

Isabelle/STARK: A Formalization of zk-STARK in Isabelle/HOL

Diego Marmosler

August 3, 2026

Department of Computer Science, University of Exeter, Exeter, UK
`d.marmosler@exeter.ac.uk`

Contents

0.0.1	Proof structure for soundness	5
0.0.2	Algebraic FRI Soundness Interface	372
0.0.3	Global FRI table bindings	424
0.1	Partial Merkle Openings	1216

```
theory Soundness-Core-Base
imports Stark-Core.Stark
begin
```

Core definitions and deterministic relations used by the STARK soundness development. Later layers separate Merkle binding, verifier execution, bad events, FRI, random-oracle accounting, and the public reduction theorems.

```
locale soundness =
  verifier concat omega shift scale clength powers to-nat of-nat size spec rounds spec2
for concat :: 'f::proth-field  $\Rightarrow$  'f  $\Rightarrow$  'f
  and omega :: 'f
  and shift :: 'f
  and scale :: nat
  and clength :: nat
  and powers :: nat
  and to-nat :: 'f  $\Rightarrow$  nat
  and of-nat :: nat  $\Rightarrow$  'f
  and size :: nat
  and spec :: (('f poly list  $\Rightarrow$  'f poly)  $\times$  nat list  $\times$  nat) list
  and rounds :: nat
  and spec2 :: (('f list  $\Rightarrow$  'f)  $\times$  nat list) list
+
fixes trace-fri-error :: prob
  and composition-fri-error :: prob
  and merkle-binding-error :: prob
assumes rounds-positive: 0 < rounds
  and spec-query-margin:
    Max (degrees clength spec) +
    sum-list (map ( $\lambda(-, roots, -).$  length roots) spec)
    < clength * scale - Max (set [0.. $\leq$  powers]) * scale
begin

definition trace-satisfies-constraint
  :: 'f poly  $\Rightarrow$  ('f poly list  $\Rightarrow$  'f poly)  $\Rightarrow$  nat list  $\Rightarrow$  bool
  where
```

trace-satisfies-constraint $f\ c\ roots \longleftrightarrow$
 $(\forall r \in \text{set } roots.$
 $\text{poly } (c\ (\text{trace-powers-of } f))\ (g \wedge r) = 0)$

definition *exists-valid-trace* :: *bool*

where

exists-valid-trace $\longleftrightarrow$
 $(\exists f.$
 $\text{degree } f < \text{clength} \wedge$
 $(\forall (c, roots, d) \in \text{set spec.}$
 $\text{trace-satisfies-constraint } f\ c\ roots))$

definition *composition-error-bound* :: *prob*

where *composition-error-bound* = $1 / \text{nnreal size}$

definition *query-sample-space-size* :: *nat*

where

query-sample-space-size =
 $\text{clength} * \text{scale} - \text{Max } (\text{set } [0..\text{powers}]) * \text{scale}$

definition *query-sample-space* :: *nat set*

where *query-sample-space* = $\{idx. idx < \text{query-sample-space-size}\}$

definition *all-constraint-roots* :: '*f list*

where

all-constraint-roots =
 $\text{remdups } (\text{List.concat } (\text{map } (\lambda(c, roots, d). \text{g-map } roots)\ \text{spec}))$

definition *query-agreement-bound* :: *nat*

where *query-agreement-bound* = $\text{maxDegree} + \text{length all-constraint-roots}$

definition *query-error-bound* :: *prob*

where

query-error-bound =
 $\text{nnreal query-agreement-bound} / \text{nnreal query-sample-space-size}$

definition *soundness-bound* :: *prob*

where

soundness-bound =
 $\text{composition-error-bound} +$
 $\text{trace-fri-error} +$
 $\text{composition-fri-error} +$
 $\text{nnreal rounds} * \text{query-error-bound}$

definition *soundness-bound-with-merkle* :: *prob*

where

soundness-bound-with-merkle =
 $\text{soundness-bound} + \text{merkle-binding-error}$

Soundness should expose a transcript-only verifier interface: the adversary chooses the

proof transcript, while the verifier starts from an empty random-oracle/hash map and the initial Fiat-Shamir state.

definition *verifier-initial-state* :: 'f list $\Rightarrow$ 'f protocol-channel

where

verifier-initial-state tr =
 (λ HashMap = fmempty, PState = 0, PTranscript = tr,
 PTraceFriCounter = 0, PCompositionFriCounter = 0,
 PAlphaCounter = 0, PQueryCounter = 0)

definition *verify-transcript*

where

verify-transcript tr =
 execute verify-monad (*verifier-initial-state* tr)

lemma *verifier-initial-state-simps*[simp]:

HashMap (*verifier-initial-state* tr) = fmempty
 PState (*verifier-initial-state* tr) = 0
 PTranscript (*verifier-initial-state* tr) = tr
 PTraceFriCounter (*verifier-initial-state* tr) = 0
 PCompositionFriCounter (*verifier-initial-state* tr) = 0
 PAlphaCounter (*verifier-initial-state* tr) = 0
 PQueryCounter (*verifier-initial-state* tr) = 0
unfolding *verifier-initial-state-def* **by** simp-all

definition *alpha-future-fresh*

:: ('f, 'a) protocol-channel-scheme $\Rightarrow$ bool

where

alpha-future-fresh s $\longleftrightarrow$
 ($\forall$ i x.
 PAlphaCounter s $\leq$ i $\longrightarrow$
 fmllookup (HashMap s) (AlphaChallenge i x) = None)

definition *trace-fri-future-fresh*

:: ('f, 'a) protocol-channel-scheme $\Rightarrow$ bool

where

trace-fri-future-fresh s $\longleftrightarrow$
 ($\forall$ i x.
 PTraceFriCounter s $\leq$ i $\longrightarrow$
 fmllookup (HashMap s) (TraceFriChallenge i x) = None)

definition *composition-fri-future-fresh*

:: ('f, 'a) protocol-channel-scheme $\Rightarrow$ bool

where

composition-fri-future-fresh s $\longleftrightarrow$
 ($\forall$ i x.
 PCompositionFriCounter s $\leq$ i $\longrightarrow$
 fmllookup (HashMap s) (CompositionFriChallenge i x) = None)

definition *query-future-fresh*


```

:: ('f, 'a) protocol-channel-scheme  $\Rightarrow$  bool
where
  query-future-fresh s  $\longleftrightarrow$ 
    ( $\forall$  i x.
      PQueryCounter s  $\leq$  i  $\longrightarrow$ 
        fmlookup (HashMap s) (QueryIndexChallenge i x) = None)

```

```

lemma alpha-future-fresh-current:
assumes alpha-future-fresh s
shows
  fmlookup (HashMap s)
    (AlphaChallenge (PAlphaCounter s) (PState s)) = None
using assms unfolding alpha-future-fresh-def by simp

```

```

lemma query-future-fresh-current:
assumes query-future-fresh s
shows
  fmlookup (HashMap s)
    (QueryIndexChallenge (PQueryCounter s) (PState s)) = None
using assms unfolding query-future-fresh-def by simp

```

```

lemma verifier-initial-alpha-future-fresh[simp]:
  alpha-future-fresh (verifier-initial-state tr)
unfolding alpha-future-fresh-def by simp

```

```

lemma verifier-initial-trace-fri-future-fresh[simp]:
  trace-fri-future-fresh (verifier-initial-state tr)
unfolding trace-fri-future-fresh-def by simp

```

```

lemma verifier-initial-composition-fri-future-fresh[simp]:
  composition-fri-future-fresh (verifier-initial-state tr)
unfolding composition-fri-future-fresh-def by simp

```

```

lemma verifier-initial-query-future-fresh[simp]:
  query-future-fresh (verifier-initial-state tr)
unfolding query-future-fresh-def by simp

```

The current bound has no separate Merkle-binding error term. Merkle binding is represented by the conditional premise *initial-merkle-binding-no-bad* below, because the present hash-map model does not assign a collision-resistance probability to Merkle roots.

0.0.1 Proof structure for soundness

The proof should decompose an accepting verifier execution for a false statement into four bad events:

- the trace FRI accepts a table that is not the evaluation of a trace polynomial satisfying $\text{degree } f < \text{clength}$;

- the composition FRI accepts a table that is not low-degree;
- the random linear combination hides a violated constraint;
- the verifier's sampled query indices miss all disagreement points.

The following predicates and lemmas are the intended interface between the probabilistic proof over *verify-monad* and the deterministic algebraic arguments.

definition *accepted* :: ('x × 's) option ⇒ bool
where *accepted out* ⇔ ¬ Option.is-none out

definition *trace-table-low-degree* :: 'f list ⇒ bool
where
trace-table-low-degree ys ⇔
 (∃ f. degree f < clength ∧ ys = map (poly f) eval-domain)

definition *composition-table-low-degree* :: nat ⇒ 'f list ⇒ bool
where
composition-table-low-degree d ys ⇔
 (∃ q. degree q ≤ d ∧ ys = map (poly q) eval-domain)

definition *trace-table-valid* :: 'f list ⇒ bool
where
trace-table-valid ys ⇔
 (∃ f.
 degree f < clength ∧
 ys = map (poly f) eval-domain ∧
 (∀ (c, roots, d) ∈ set spec.
 trace-satisfies-constraint f c roots))

definition *query-consistent-at* :: 'f list ⇒ 'f list ⇒ 'f list ⇒ nat ⇒ bool
where
query-consistent-at trace-table composition-table as idx ⇔
 idx < clength * scale ∧
 length trace-table = clength * scale ∧
 length composition-table = clength * scale ∧
 (∀ i ∈ set (powers-scaled idx). i < length trace-table) ∧
 composition-table ! idx =
 cp-eval as (map (!) trace-table) (powers-scaled idx) (h ^ idx * shift)

lemma *query-consistent-at-cong-opened-values*:
assumes *trace-len*:
 length trace-table = length trace-table'
and *composition-len*:
 length composition-table = length composition-table'
and *trace-values*:
 map (!) trace-table (powers-scaled idx) =
 map (!) trace-table' (powers-scaled idx)
and *composition-value*:
 composition-table ! idx = composition-table' ! idx
shows

$query-consistent-at\ trace-table\ composition-table\ as\ idx \longleftrightarrow$
 $query-consistent-at\ trace-table'\ composition-table'\ as\ idx$
proof –
have *cp-eq*:
 $cp-eval\ as\ (map\ (!)\ trace-table)\ (powers-scaled\ idx)$
 $(h \wedge idx * shift) =$
 $cp-eval\ as\ (map\ (!)\ trace-table')\ (powers-scaled\ idx)$
 $(h \wedge idx * shift)$
by (*rule arg-cong*[**where**
 $f = \lambda xs. cp-eval\ as\ xs\ (h \wedge idx * shift),\ OF\ trace-values$])
show *?thesis*
unfolding *query-consistent-at-def*
using *trace-len composition-len composition-value cp-eq* **by** *simp*
qed

definition *all-queries-consistent* :: '*f list* $\Rightarrow$ '*f list* $\Rightarrow$ '*f list* $\Rightarrow$ *bool*
where
 $all-queries-consistent\ trace-table\ composition-table\ as \longleftrightarrow$
 $(\forall\ idx \in query-sample-space.$
 $query-consistent-at\ trace-table\ composition-table\ as\ idx)$

definition *bad-query-indices* :: '*f list* $\Rightarrow$ '*f list* $\Rightarrow$ '*f list* $\Rightarrow$ *nat set*
where
 $bad-query-indices\ trace-table\ composition-table\ as =$
 $\{idx \in query-sample-space.$
 $\neg\ query-consistent-at\ trace-table\ composition-table\ as\ idx\}$

definition *query-agreement-indices* :: '*f list* $\Rightarrow$ '*f list* $\Rightarrow$ '*f list* $\Rightarrow$ *nat set*
where
 $query-agreement-indices\ trace-table\ composition-table\ as =$
 $\{idx \in query-sample-space.$
 $query-consistent-at\ trace-table\ composition-table\ as\ idx\}$

definition *query-disagreement-indices* :: '*f list* $\Rightarrow$ '*f list* $\Rightarrow$ '*f list* $\Rightarrow$ *nat set*
where
 $query-disagreement-indices\ trace-table\ composition-table\ as =$
 $bad-query-indices\ trace-table\ composition-table\ as$

definition *query-samples-miss-disagreements*
:: '*f list* $\Rightarrow$ '*f list* $\Rightarrow$ '*f list* $\Rightarrow$ *nat list* $\Rightarrow$ *bool*
where
 $query-samples-miss-disagreements\ trace-table\ composition-table\ as\ query-idxs \longleftrightarrow$
 $query-disagreement-indices\ trace-table\ composition-table\ as \neq \{\}$ $\wedge$
 $set\ query-idxs \subseteq query-agreement-indices\ trace-table\ composition-table\ as$

definition *violated-constraints*
:: '*f poly* $\Rightarrow$ ((*f poly list* $\Rightarrow$ '*f poly*) $\times$ *nat list* $\times$ *nat*) *set*
where
 $violated-constraints\ f =$

$\{ \text{entry} \in \text{set spec.} \\ \text{case entry of } (c, \text{roots}, d) \Rightarrow \\ \neg \text{trace-satisfies-constraint } f \ c \ \text{roots} \}$

definition *constraint-remainder-bad*
 $:: 'f \text{ poly} \Rightarrow (('f \text{ poly list} \Rightarrow 'f \text{ poly}) \times \text{nat list} \times \text{nat}) \Rightarrow \text{bool}$
where
constraint-remainder-bad $f \text{ entry} \longleftrightarrow$
 $(\text{case entry of } (c, \text{roots}, d) \Rightarrow$
 $(\exists r \in \text{set roots.}$
 $\text{poly } (c \ (\text{trace-powers-of } f)) \ (g \wedge r) \neq 0))$

definition *composition-vanishes-on-constraint-domain* $:: 'f \text{ poly} \Rightarrow 'f \text{ list} \Rightarrow \text{bool}$
where
composition-vanishes-on-constraint-domain $f \text{ as} \longleftrightarrow$
 $(\forall x \in \text{set } G. \text{poly } (cp \text{ as } (\text{trace-powers-of } f)) \ x = 0)$

definition *constraint-root-residual*
 $:: 'f \text{ poly} \Rightarrow \text{nat} \Rightarrow$
 $('f \times (('f \text{ poly list} \Rightarrow 'f \text{ poly}) \times \text{nat list} \times \text{nat})) \Rightarrow 'f$
where
constraint-root-residual $f \ r \ \text{weighted-entry} =$
 $(\text{case weighted-entry of } (a, (c, \text{roots}, d)) \Rightarrow$
 $\text{if } r \in \text{set roots}$
 $\text{then } a * \text{poly } (c \ (\text{trace-powers-of } f)) \ (g \wedge r)$
 $\text{else } 0)$

definition *composition-root-residual* $:: 'f \text{ poly} \Rightarrow 'f \text{ list} \Rightarrow \text{nat} \Rightarrow 'f$
where
composition-root-residual $f \text{ as } r =$
 $\text{sum-list } (\text{map } (\text{constraint-root-residual } f \ r) \ (\text{zip as spec}))$

definition *random-combination-hides-violations* $:: 'f \text{ poly} \Rightarrow 'f \text{ list} \Rightarrow \text{bool}$
where
random-combination-hides-violations $f \text{ as} \longleftrightarrow$
 $\text{violated-constraints } f \neq \{\} \wedge$
 $(\forall \text{entry} \in \text{violated-constraints } f.$
 $\text{case entry of } (c, \text{roots}, d) \Rightarrow$
 $(\forall r \in \text{set roots. } \text{composition-root-residual } f \text{ as } r = 0))$

definition *constraint-root-cofactor* $:: \text{nat list} \Rightarrow 'f \text{ poly}$
where
constraint-root-cofactor $\text{roots} =$
 $\text{prod } (\text{filter } (\lambda x. x \notin \text{set } (g\text{-map roots})) \ \text{all-constraint-roots})$

definition *common-denominator-poly* $:: 'f \text{ poly}$
where *common-denominator-poly* $= \text{prod all-constraint-roots}$

definition *common-denominator-numerator-term*

$:: 'f \text{ poly} \Rightarrow$
 $(('f \times ((('f \text{ poly list} \Rightarrow 'f \text{ poly}) \times \text{nat list} \times \text{nat})) \Rightarrow$
 $'f \text{ poly}$

where

$\text{common-denominator-numerator-term } f \text{ weighted-entry} =$
 $(\text{case weighted-entry of } (a, (c, \text{roots}, d)) \Rightarrow$
 $\text{CP } a * c (\text{trace-powers-of } f) * \text{constraint-root-cofactor roots})$

definition $\text{common-denominator-numerator} :: 'f \text{ poly} \Rightarrow 'f \text{ list} \Rightarrow 'f \text{ poly}$

where

$\text{common-denominator-numerator } f \text{ as} =$
 sum-list
 $(\text{map } (\text{common-denominator-numerator-term } f) (\text{zip as spec}))$

definition $\text{composition-spec-eval-term}$

$:: 'f \text{ poly} \Rightarrow 'f \Rightarrow$
 $(('f \times ((('f \text{ poly list} \Rightarrow 'f \text{ poly}) \times \text{nat list} \times \text{nat})) \Rightarrow$
 $'f$

where

$\text{composition-spec-eval-term } f \text{ x weighted-entry} =$
 $(\text{case weighted-entry of } (a, (c, \text{roots}, d)) \Rightarrow$
 $a * (\text{poly } (c (\text{trace-powers-of } f)) \text{ x} /$
 $\text{poly } (\text{prod } (g\text{-map roots}) \text{ x}))$

definition $\text{composition-spec-eval} :: 'f \text{ poly} \Rightarrow 'f \text{ list} \Rightarrow 'f \Rightarrow 'f$

where

$\text{composition-spec-eval } f \text{ as } x =$
 $\text{sum-list } (\text{map } (\text{composition-spec-eval-term } f \text{ x}) (\text{zip as spec}))$

definition $\text{common-denominator-constraint-root-residual}$

$:: 'f \text{ poly} \Rightarrow \text{nat} \Rightarrow$
 $(('f \times ((('f \text{ poly list} \Rightarrow 'f \text{ poly}) \times \text{nat list} \times \text{nat})) \Rightarrow 'f$

where

$\text{common-denominator-constraint-root-residual } f \text{ r weighted-entry} =$
 $(\text{case weighted-entry of } (a, (c, \text{roots}, d)) \Rightarrow$
 $\text{if } r \in \text{set roots}$
 then
 $a * \text{poly } (c (\text{trace-powers-of } f)) (g \wedge r) *$
 $\text{poly } (\text{constraint-root-cofactor roots}) (g \wedge r)$
 $\text{else } 0)$

definition $\text{common-denominator-composition-root-residual}$

$:: 'f \text{ poly} \Rightarrow 'f \text{ list} \Rightarrow \text{nat} \Rightarrow 'f$

where

$\text{common-denominator-composition-root-residual } f \text{ as } r =$
 sum-list
 $(\text{map } (\text{common-denominator-constraint-root-residual } f \text{ r}) (\text{zip as spec}))$

definition $\text{random-combination-common-denominator-hides-violations}$

$:: 'f \text{ poly} \Rightarrow 'f \text{ list} \Rightarrow \text{bool}$

where

$\text{random-combination-common-denominator-hides-violations } f \text{ as} \longleftrightarrow$
 $\text{violated-constraints } f \neq \{\}$ $\wedge$
 $(\forall \text{entry} \in \text{violated-constraints } f.$
 $\text{case entry of } (c, \text{roots}, d) \Rightarrow$
 $(\forall r \in \text{set roots.}$
 $\text{common-denominator-composition-root-residual } f \text{ as } r = 0))$

definition $\text{common-denominator-degree-bounds} :: 'f \text{ poly} \Rightarrow 'f \text{ list} \Rightarrow \text{bool}$

where

$\text{common-denominator-degree-bounds } f \text{ as} \longleftrightarrow$
 $\text{degree } (\text{common-denominator-numerator } f \text{ as}) < \text{query-sample-space-size} \wedge$
 $(\forall q. \text{degree } q \leq \text{maxDegree} \longrightarrow$
 $\text{degree } (\text{common-denominator-poly} * q) < \text{query-sample-space-size})$

definition $\text{query-agreement-margin} :: \text{bool}$

where

$\text{query-agreement-margin} \longleftrightarrow$
 $\text{query-agreement-bound} < \text{query-sample-space-size}$

definition $\text{spec-degree-wellformed} :: \text{bool}$

where

$\text{spec-degree-wellformed} \longleftrightarrow \text{query-agreement-margin}$

lemma $\text{spec-degree-wellformedI}:$

assumes $\text{query-agreement-margin}$

shows $\text{spec-degree-wellformed}$

using assms **unfolding** $\text{spec-degree-wellformed-def}$ **by** simp

lemma $\text{spec-degree-wellformed-query-agreement-margin}:$

assumes $\text{spec-degree-wellformed}$

shows $\text{query-agreement-margin}$

using assms **unfolding** $\text{spec-degree-wellformed-def}$ **by** simp

lemma $\text{spec-degree-wellformed-constraint-degree}:$

assumes $\text{wf: spec-degree-wellformed}$

and $\text{deg-f: degree } f < \text{clength}$

and $\text{spec-entry: } (c, \text{roots}, d) \in \text{set spec}$

shows $\text{degree } (c (\text{trace-powers-of } f)) \leq d * (\text{clength} - 1)$

by $(\text{rule constraint-degree-wellformed[OF deg-f spec-entry]})$

lemma $\text{query-agreement-marginD}:$

assumes $\text{query-agreement-margin}$

shows $\text{query-agreement-bound} < \text{query-sample-space-size}$

using assms **unfolding** $\text{query-agreement-margin-def}$ **by** simp

lemma $\text{spec-degree-wellformed-query-margin}:$

assumes $\text{spec-degree-wellformed}$

shows *query-agreement-bound* < *query-sample-space-size*
by (rule *query-agreement-marginD*
[*OF spec-degree-wellformed-query-agreement-margin*[*OF assms*]])

lemma *all-constraint-roots-length-le-sum-roots*:
length all-constraint-roots ≤
sum-list (map (λ(-, roots, -). *length roots*) *spec*)
proof –
have *length all-constraint-roots* ≤
length (*List.concat* (map (λ(*c*, roots, *d*). *g-map roots*) *spec*))
unfolding *all-constraint-roots-def*
by (rule *length-remdups-leq*)
also have ... =
sum-list (map (λ(-, roots, -). *length roots*) *spec*)
unfolding *g-map-def*
by (*induction spec*) *auto*
finally show ?thesis .
qed

lemma *spec-degree-wellformed-from-spec-query-margin*:
spec-degree-wellformed
proof –
have *margin: query-agreement-bound* < *query-sample-space-size*
using *spec-query-margin all-constraint-roots-length-le-sum-roots*
unfolding *query-agreement-bound-def query-sample-space-size-def*
maxDegree-def
by *linarith*
then show ?thesis
unfolding *spec-degree-wellformed-def query-agreement-margin-def* **by** *simp*
qed

lemma *exists-valid-traceI*:
assumes *degree f* < *clength*
and $\bigwedge c \text{ roots } d. (c, \text{roots}, d) \in \text{set spec} \implies$
trace-satisfies-constraint f c roots
shows *exists-valid-trace*
using *assms* **unfolding** *exists-valid-trace-def* **by** *auto*

lemma *trace-table-valid-imp-exists-valid-trace*:
assumes *trace-table-valid ys*
shows *exists-valid-trace*
using *assms* **unfolding** *trace-table-valid-def*
by (*auto intro!: exists-valid-traceI*)

lemma *false-statement-violated-constraints*:
assumes *false-statement*: $\neg \text{exists-valid-trace}$
and *low-degree*: *degree f* < *clength*
shows *violated-constraints f* ≠ {}
proof

```

assume empty: violated-constraints  $f = \{\}$ 
have  $\bigwedge c \text{ roots } d. (c, \text{roots}, d) \in \text{set spec} \implies$ 
   trace-satisfies-constraint  $f \ c \ \text{roots}$ 
using empty unfolding violated-constraints-def by force
then have exists-valid-trace
   using low-degree unfolding exists-valid-trace-def by auto
then show False
   using false-statement by contradiction
qed

```

```

lemma constraint-remainder-bad-iff-not-trace-satisfies:
  constraint-remainder-bad  $f \ (c, \text{roots}, d) \longleftrightarrow$ 
     $\neg \text{trace-satisfies-constraint } f \ c \ \text{roots}$ 
unfolding constraint-remainder-bad-def trace-satisfies-constraint-def
by auto

```

```

lemma violated-constraints-has-remainder-bad:
  assumes violated-constraints  $f \neq \{\}$ 
shows  $\exists \text{entry} \in \text{violated-constraints } f. \text{constraint-remainder-bad } f \ \text{entry}$ 
using assms
unfolding violated-constraints-def
by (auto simp: constraint-remainder-bad-iff-not-trace-satisfies)

```

```

lemma random-combination-hides-violations-has-remainder-bad:
  assumes random-combination-hides-violations  $f \text{ as}$ 
shows  $\exists \text{entry} \in \text{violated-constraints } f. \text{constraint-remainder-bad } f \ \text{entry}$ 
using assms violated-constraints-has-remainder-bad
unfolding random-combination-hides-violations-def
by blast

```

```

lemma linear-equation-solution-unique:

```

```

  fixes  $c \ b \ x \ y :: 'f$ 
  assumes c-nonzero:  $c \neq 0$ 
    and x-sol:  $c * x + b = 0$ 
    and y-sol:  $c * y + b = 0$ 
shows  $x = y$ 

```

```

proof -

```

```

  have x-eq:  $c * x = - b$ 

```

```

proof -

```

```

  have  $c * x + b + (- b) = 0 + (- b)$ 

```

```

    using x-sol by simp

```

```

  then show ?thesis

```

```

    by simp

```

```

qed

```

```

have y-eq:  $c * y = - b$ 

```

```

proof -

```

```

  have  $c * y + b + (- b) = 0 + (- b)$ 

```

```

    using y-sol by simp

```

```

  then show ?thesis

```



```

    by simp
qed
have  $c * (x - y) = 0$ 
  using  $x\text{-eq } y\text{-eq}$  by (simp add: algebra-simps)
then have  $x - y = 0$ 
  using  $c\text{-nonzero}$  by simp
then show ?thesis
  by simp
qed

lemma composition-root-residual-sum-nth:
  assumes  $\text{len: length } as = \text{length } spec$ 
  shows
    composition-root-residual  $f$   $as$   $r =$ 
      ( $\sum i \in \{0..<\text{length } spec\}$ .
        constraint-root-residual  $f$   $r$  ( $as ! i, spec ! i$ ))
proof -
  have composition-root-residual  $f$   $as$   $r =$ 
    sum-list (map (constraint-root-residual  $f$   $r$ ) (zip  $as$   $spec$ ))
    by (simp add: composition-root-residual-def)
  also have ... =
    ( $\sum i \in \{0..<\text{length } (zip\ as\ spec)\}$ .
      map (constraint-root-residual  $f$   $r$ ) (zip  $as$   $spec$ ) !  $i$ )
    by (simp add: sum-list-sum-nth)
  also have ... =
    ( $\sum i \in \{0..<\text{length } spec\}$ .
      constraint-root-residual  $f$   $r$  ( $as ! i, spec ! i$ ))
    using  $\text{len}$  by (intro sum.cong) auto
  finally show ?thesis .
qed

lemma composition-root-residual-split-index:
  assumes  $\text{len: length } as = \text{length } spec$ 
  and  $i\text{-bound: } i < \text{length } spec$ 
  shows
    composition-root-residual  $f$   $as$   $r =$ 
      constraint-root-residual  $f$   $r$  ( $as ! i, spec ! i$ ) +
      ( $\sum j \in \{0..<\text{length } spec\} - \{i\}$ .
        constraint-root-residual  $f$   $r$  ( $as ! j, spec ! j$ ))
proof -
  let ?F =  $\lambda j. \text{constraint-root-residual } f \ r \ (as ! j, spec ! j)$ 
  have composition-root-residual  $f$   $as$   $r = (\sum j \in \{0..<\text{length } spec\}. ?F\ j)$ 
    by (rule composition-root-residual-sum-nth[OF len])
  also have ... = ?F  $i$  + ( $\sum j \in \{0..<\text{length } spec\} - \{i\}. ?F\ j$ )
    using  $i\text{-bound}$  by (simp add: sum.remove)
  finally show ?thesis .
qed

lemma composition-root-residual-zero-unique-nth:

```

```

fixes as bs :: 'f list
assumes len-as: length as = length spec
  and len-bs: length bs = length spec
  and i-bound: i < length spec
  and spec-i: spec ! i = (c, roots, d)
  and r-in: r ∈ set roots
  and nonzero:
    poly (c (trace-powers-of f)) (g ^ r) ≠ 0
  and outside:
     $\bigwedge j. j < \text{length spec} \implies j \neq i \implies \text{as} ! j = \text{bs} ! j$ 
  and zero-as: composition-root-residual f as r = 0
  and zero-bs: composition-root-residual f bs r = 0
shows as ! i = bs ! i
proof -
  let ?C = poly (c (trace-powers-of f)) (g ^ r)
  let ?B =
    ( $\sum j \in \{0..<\text{length spec}\} - \{i\}.$ 
      constraint-root-residual f r (as ! j, spec ! j))
  have B-eq:
    ( $\sum j \in \{0..<\text{length spec}\} - \{i\}.$ 
      constraint-root-residual f r (bs ! j, spec ! j)) = ?B
  by (intro sum.cong) (use outside in auto)
  have as-eq:
    composition-root-residual f as r = ?C * (as ! i) + ?B
  using composition-root-residual-split-index[OF len-as i-bound]
    spec-i r-in
  unfolding constraint-root-residual-def
  by (simp add: algebra-simps)
  have bs-eq:
    composition-root-residual f bs r = ?C * (bs ! i) + ?B
  using composition-root-residual-split-index[OF len-bs i-bound]
    spec-i r-in B-eq
  unfolding constraint-root-residual-def
  by (simp add: algebra-simps)
  show ?thesis
proof (rule linear-equation-solution-unique[OF nonzero])
  show ?C * (as ! i) + ?B = 0
    using as-eq zero-as by simp
  show ?C * (bs ! i) + ?B = 0
    using bs-eq zero-bs by simp
qed
qed

lemma random-combination-hides-violations-unique-nth:
  fixes as bs :: 'f list
  assumes len-as: length as = length spec
    and len-bs: length bs = length spec
    and i-bound: i < length spec
    and spec-i: spec ! i = (c, roots, d)

```

```

and violated:  $(c, \text{roots}, d) \in \text{violated-constraints } f$ 
and r-in:  $r \in \text{set roots}$ 
and nonzero:
   $\text{poly } (c \text{ (trace-powers-of } f)) (g \wedge r) \neq 0$ 
and outside:
   $\bigwedge j. j < \text{length spec} \implies j \neq i \implies \text{as } ! j = \text{bs } ! j$ 
and hides-as: random-combination-hides-violations  $f \text{ as}$ 
and hides-bs: random-combination-hides-violations  $f \text{ bs}$ 
shows  $\text{as } ! i = \text{bs } ! i$ 
proof –
  have zero-as: composition-root-residual  $f \text{ as } r = 0$ 
    using hides-as violated r-in
    unfolding random-combination-hides-violations-def by auto
  have zero-bs: composition-root-residual  $f \text{ bs } r = 0$ 
    using hides-bs violated r-in
    unfolding random-combination-hides-violations-def by auto
  show ?thesis
    by (rule composition-root-residual-zero-unique-nth
      [OF len-as len-bs i-bound spec-i r-in nonzero outside zero-as zero-bs])
qed

lemma bad-query-indices-subset:
  bad-query-indices trace-table composition-table as  $\subseteq \{0..<\text{clength} * \text{scale}\}$ 
  unfolding bad-query-indices-def query-sample-space-def query-sample-space-size-def
  by auto

lemma index-less-domain:
  index x  $< \text{clength} * \text{scale}$ 
proof –
  have max-lt: Max (set  $[0..<\text{powers}]$ )  $< \text{clength}$ 
  proof –
    have Max (set  $[0..<\text{powers}]$ )  $= \text{powers} - 1$ 
      using powers-pos by (intro Max-eqI) auto
    also have  $\dots < \text{clength}$ 
      using powers-pos powers-le-clength by simp
    finally show ?thesis .
  qed
  have mod-pos:  $0 < \text{clength} * \text{scale} - \text{Max} (\text{set } [0..<\text{powers}]) * \text{scale}$ 
    using max-lt scale-pos
    by (simp add: diff-mult-distrib2)
  have index x  $< \text{clength} * \text{scale} - \text{Max} (\text{set } [0..<\text{powers}]) * \text{scale}$ 
    unfolding index-def using mod-pos by simp
  moreover have  $\text{clength} * \text{scale} - \text{Max} (\text{set } [0..<\text{powers}]) * \text{scale} \leq \text{clength} * \text{scale}$ 
    by simp
  ultimately show ?thesis
    by linarith
qed

lemma query-sample-space-size-pos:

```

```

0 < query-sample-space-size
proof -
  have max-lt: Max (set [0..

```

```

assumes idx-in:  $idx \in \text{query-sample-space}$ 
and i-in:  $i \in \text{set } (\text{powers-scaled } idx)$ 
shows  $i < \text{clength} * \text{scale}$ 
proof –
  obtain k where k-bound:  $k < \text{powers}$  and i-eq:  $i = idx + k * \text{scale}$ 
    using i-in unfolding powers-scaled-def by auto
  have max-eq:  $\text{Max } (\text{set } [0..<\text{powers}]) = \text{powers} - 1$ 
    using powers-pos by (intro Max-eqI) auto
  have k-le-max:  $k \leq \text{Max } (\text{set } [0..<\text{powers}])$ 
    using k-bound unfolding max-eq by simp
  have idx-bound:
     $idx < \text{clength} * \text{scale} - \text{Max } (\text{set } [0..<\text{powers}]) * \text{scale}$ 
    using query-sample-spaceD[OF idx-in]
    unfolding query-sample-space-size-def .
  have  $idx + k * \text{scale} \leq idx + \text{Max } (\text{set } [0..<\text{powers}]) * \text{scale}$ 
    using k-le-max by simp
  also have  $\dots < \text{clength} * \text{scale}$ 
    using idx-bound by simp
  finally show ?thesis
    unfolding i-eq .
qed

definition query-sample-domain :: 'f list
where query-sample-domain =  $\text{map } (\lambda idx. h \wedge idx * \text{shift}) [0..<\text{query-sample-space-size}]$ 

lemma eval-domain-length:
   $\text{length } \text{eval-domain} = \text{clength} * \text{scale}$ 
  unfolding eval-domain-def H-def by simp

lemma eval-domain-nth:
  assumes  $i < \text{clength} * \text{scale}$ 
  shows  $\text{eval-domain} ! i = h \wedge i * \text{shift}$ 
  using assms
  unfolding eval-domain-def H-def
  by (simp add: mult.commute)

lemma h-nonzero:
   $h \neq 0$ 
  unfolding h-def using omega-nonzero-derived by simp

lemma h-power-inj-on-eval-domain:
  assumes i-bound:  $i < \text{clength} * \text{scale}$ 
    and j-bound:  $j < \text{clength} * \text{scale}$ 
    and eq:  $h \wedge i = h \wedge j$ 
  shows  $i = j$ 
proof (rule ccontr)
  let ?N =  $\text{clength} * \text{scale}$ 
  assume neg:  $i \neq j$ 
  consider  $i < j \mid j < i$ 

```

```

    using neq by linarith
  then show False
proof cases
  case 1
  have diff-pos:  $0 < j - i$ 
    using 1 by simp
  have diff-lt:  $j - i < ?N$ 
    using j-bound by simp
  have hi-nonzero:  $h^i \neq 0$ 
    using h-nonzero by simp
  have j-decomp:  $j = i + (j - i)$ 
    using 1 by simp
  have mult-eq:  $h^j = h^i * h^{(j - i)}$ 
    by (subst j-decomp) (simp add: power-add)
  have cancel-eq:  $h^i * 1 = h^i * h^{(j - i)}$ 
    using eq mult-eq by simp
  have  $h^i = 0 \vee 1 = h^{(j - i)}$ 
    using cancel-eq by (simp only: mult-cancel-left)
  then have  $h^{(j - i)} = 1$ 
    using hi-nonzero by auto
  then show False
    using h-exact-order diff-pos diff-lt unfolding exact-order-def by blast
next
  case 2
  have diff-pos:  $0 < i - j$ 
    using 2 by simp
  have diff-lt:  $i - j < ?N$ 
    using i-bound by simp
  have hj-nonzero:  $h^j \neq 0$ 
    using h-nonzero by simp
  have i-decomp:  $i = j + (i - j)$ 
    using 2 by simp
  have mult-eq:  $h^i = h^j * h^{(i - j)}$ 
    by (subst i-decomp) (simp add: power-add)
  have cancel-eq:  $h^j * 1 = h^j * h^{(i - j)}$ 
    using eq mult-eq by simp
  have  $h^j = 0 \vee 1 = h^{(i - j)}$ 
    using cancel-eq by (simp only: mult-cancel-left)
  then have  $h^{(i - j)} = 1$ 
    using hj-nonzero by auto
  then show False
    using h-exact-order diff-pos diff-lt unfolding exact-order-def by blast
qed
qed

lemma H-distinct:
  distinct H
proof -
  have inj: inj-on (( $\cap$ ) h) (set [0.. $\text{scale} * \text{clength}$ ])

```

```

proof (intro inj-onI)
  fix x y
  assume x-in:  $x \in \text{set } [0..<\text{scale} * \text{clength}]$ 
  and y-in:  $y \in \text{set } [0..<\text{scale} * \text{clength}]$ 
  and eq:  $h \wedge x = h \wedge y$ 
  have x-bound:  $x < \text{clength} * \text{scale}$ 
  using x-in by (simp add: mult.commute)
  have y-bound:  $y < \text{clength} * \text{scale}$ 
  using y-in by (simp add: mult.commute)
  show  $x = y$ 
  by (rule h-power-inj-on-eval-domain[OF x-bound y-bound eq])
qed
show ?thesis
  unfolding H-def using inj by (simp add: distinct-map)
qed

```

```

lemma eval-domain-distinct:
  distinct eval-domain
proof -
  have inj: inj-on ((*) shift) (set H)
  using shift-nonzero by (auto intro!: inj-onI)
  show ?thesis
  unfolding eval-domain-def
  using H-distinct inj by (simp add: distinct-map)
qed

```

```

lemma card-set-eval-domain:
  card (set eval-domain) = clength * scale
using eval-domain-distinct eval-domain-length
by (simp add: distinct-card)

```

```

lemma poly-eq-on-eval-domainI:
  assumes deg-p: degree p < clength * scale
  and deg-q: degree q < clength * scale
  and agree:
     $\bigwedge \text{idx}. \text{idx} < \text{clength} * \text{scale} \implies$ 
     $\text{poly } p (h \wedge \text{idx} * \text{shift}) = \text{poly } q (h \wedge \text{idx} * \text{shift})$ 
  shows  $p = q$ 
proof (rule poly-eqI-degree[where A = set eval-domain])
  fix x
  assume x-in:  $x \in \text{set eval-domain}$ 
  then obtain idx where idx-bound:  $\text{idx} < \text{clength} * \text{scale}$ 
  and x-eq:  $x = h \wedge \text{idx} * \text{shift}$ 
  unfolding eval-domain-def H-def
  by (auto simp: mult.commute)
  show  $\text{poly } p x = \text{poly } q x$ 
  using agree[OF idx-bound] x-eq by simp
next
  show degree p < card (set eval-domain)

```

```

    using deg-p card-set-eval-domain by simp
next
  show degree q < card (set eval-domain)
    using deg-q card-set-eval-domain by simp
qed

lemma query-sample-domain-distinct:
  distinct query-sample-domain
proof -
  have inj: inj-on ( $\lambda idx. h \wedge idx * shift$ ) (set [0.. $query\_sample\_space\_size$ ])
  proof (intro inj-onI)
    fix i j
    assume i-in:  $i \in set [0.. $query\_sample\_space\_size$ ]$ 
      and j-in:  $j \in set [0.. $query\_sample\_space\_size$ ]$ 
      and eq:  $h \wedge i * shift = h \wedge j * shift$ 
    have i-bound:  $i < clength * scale$ 
      using i-in query-sample-space-size-le-domain by simp
    have j-bound:  $j < clength * scale$ 
      using j-in query-sample-space-size-le-domain by simp
    have  $h \wedge i = h \wedge j$ 
      using eq shift-nonzero by simp
    then show  $i = j$ 
      by (rule h-power-inj-on-eval-domain[OF i-bound j-bound])
  qed
  show ?thesis
    unfolding query-sample-domain-def using inj by (simp add: distinct-map)
qed

lemma card-set-query-sample-domain:
  card (set query-sample-domain) = query-sample-space-size
proof -
  have card (set query-sample-domain) = length query-sample-domain
    using query-sample-domain-distinct by (simp add: distinct-card)
  also have ... = query-sample-space-size
    unfolding query-sample-domain-def by simp
  finally show ?thesis .
qed

lemma poly-eq-on-query-sample-domainI:
  assumes deg-p: degree p < query-sample-space-size
    and deg-q: degree q < query-sample-space-size
    and agree:
       $\bigwedge idx. idx \in query\_sample\_space \implies$ 
         $poly\ p\ (h \wedge idx * shift) = poly\ q\ (h \wedge idx * shift)$ 
  shows  $p = q$ 
proof (rule poly-eqI-degree[where A = set query-sample-domain])
  fix x
  assume x-in:  $x \in set\ query\_sample\_domain$ 
  then obtain idx where idx-bound:  $idx < query\_sample\_space\_size$ 

```



```

    and  $x\text{-eq}$ :  $x = h \wedge \text{idx} * \text{shift}$ 
    unfolding query-sample-domain-def by auto
  have idx-in:  $\text{idx} \in \text{query-sample-space}$ 
    using idx-bound unfolding query-sample-space-def by simp
  show poly  $p\ x = \text{poly}\ q\ x$ 
    using agree[OF idx-in] x-eq by simp
next
  show  $\text{degree}\ p < \text{card}\ (\text{set}\ \text{query-sample-domain})$ 
    using deg-p card-set-query-sample-domain by simp
next
  show  $\text{degree}\ q < \text{card}\ (\text{set}\ \text{query-sample-domain})$ 
    using deg-q card-set-query-sample-domain by simp
qed

```

```

lemma prod-fold-acc:
  fixes roots :: 'f list
  shows  $\text{fold}\ (\lambda s\ \text{acc}.\ (\text{XP}\ 1 - \text{CP}\ s) * \text{acc})\ \text{roots}\ \text{acc} =$ 
     $\text{fold}\ (\lambda s\ \text{acc}.\ (\text{XP}\ 1 - \text{CP}\ s) * \text{acc})\ \text{roots}\ (1 :: 'f\ \text{poly}) * \text{acc}$ 
proof (induction roots arbitrary: acc)
  case Nil
  then show ?case by simp
next
  case (Cons r roots)
  let ?F =  $\lambda s\ \text{acc}.\ (\text{XP}\ 1 - \text{CP}\ s) * \text{acc}$ 
  let ?fac =  $\text{XP}\ 1 - \text{CP}\ r$ 
  have lhs:
     $\text{fold}\ ?F\ (r \# \text{roots})\ \text{acc} =$ 
       $\text{fold}\ ?F\ \text{roots}\ 1 * (?fac * \text{acc})$ 
    using Cons.IH[of ?fac * acc] by simp
  have  $\text{fold}\ ?F\ (r \# \text{roots})\ \text{acc} =$ 
     $(\text{fold}\ ?F\ \text{roots}\ 1 * ?fac) * \text{acc}$ 
    using lhs by (simp only: mult.assoc)
  also have ... =  $\text{fold}\ ?F\ \text{roots}\ ?fac * \text{acc}$ 
    using Cons.IH[of ?fac, symmetric] by simp
  also have ... =  $\text{fold}\ ?F\ (r \# \text{roots})\ 1 * \text{acc}$ 
    by simp
  finally show ?case .
qed

```

```

lemma prod-Cons:
  fixes r :: 'f and roots :: 'f list
  shows  $\text{prod}\ (r \# \text{roots}) = (\text{XP}\ 1 - \text{CP}\ r) * (\text{prod}\ \text{roots} :: 'f\ \text{poly})$ 
proof -
  let ?F =  $\lambda s\ \text{acc}.\ (\text{XP}\ 1 - \text{CP}\ s) * \text{acc}$ 
  have  $\text{prod}\ (r \# \text{roots}) = \text{fold}\ ?F\ \text{roots}\ ((\text{XP}\ 1 - \text{CP}\ r) * 1)$ 
    unfolding prod-def by simp
  also have ... =  $\text{fold}\ ?F\ \text{roots}\ 1 * ((\text{XP}\ 1 - \text{CP}\ r) * 1)$ 
    by (rule prod-fold-acc)
  also have ... =  $(\text{prod}\ \text{roots} :: 'f\ \text{poly}) * (\text{XP}\ 1 - \text{CP}\ r)$ 

```

```

    unfolding prod-def by simp
    also have ... = (XP 1 - CP r) * (prod roots :: 'f poly)
    by (rule mult.commute)
    finally show ?thesis .
qed

```

```

lemma prod-append:
  fixes xs ys :: 'f list
  shows prod (xs @ ys) = (prod xs :: 'f poly) * prod ys
proof (induction xs)
  case Nil
  then show ?case
    unfolding prod-def by simp
next
  case (Cons x xs)
  then show ?case
    by (simp add: prod-Cons algebra-simps)
qed

```

```

lemma prod-alt:
  fixes roots :: 'f list
  shows prod roots =
    prod-list (map (λr. XP 1 - CP r) roots :: 'f poly list)
proof (induction roots)
  case Nil
  then show ?case
    unfolding prod-def by simp
next
  case (Cons r roots)
  then show ?case
    by (simp add: prod-Cons)
qed

```

```

lemma degree-prod-le-length:
  fixes roots :: 'f list
  shows degree (prod roots :: 'f poly) ≤ length roots
proof (induction roots)
  case Nil
  then show ?case
    unfolding prod-def by simp
next
  case (Cons r roots)
  have factor: degree (XP 1 - CP r :: 'f poly) ≤ 1
  proof -
    have deg-x: degree (Polynomial.monom (1::'f) 1) ≤ 1
    by (rule degree-monom-le)
    have deg-c: degree (Polynomial.monom r 0) ≤ 1
    proof -
      have degree (Polynomial.monom r 0) ≤ 0

```

```

      by (rule degree-monom-le)
    then show ?thesis by simp
  qed
  show ?thesis
    unfolding XP-def CP-def
    by (rule degree-diff-le[OF deg-x deg-c])
  qed
  have degree (prod (r # roots) :: 'f poly) =
    degree ((XP 1 - CP r) * (prod roots :: 'f poly))
    by (simp add: prod-Cons)
  also have ... ≤
    degree (XP 1 - CP r :: 'f poly) + degree (prod roots :: 'f poly)
    by (rule degree-mult-le)
  also have ... ≤ Suc (length roots)
    using factor Cons.IH by simp
  finally show ?case by simp
  qed

```

```

lemma degree-sum-list-less:
  fixes ps :: 'f poly list
  assumes  $\bigwedge p. p \in \text{set } ps \implies \text{degree } p < n$ 
  and  $0 < n$ 
  shows  $\text{degree } (\text{sum-list } ps) < n$ 
  using assms
proof (induction ps)
  case Nil
  then show ?case by simp
next
  case (Cons p ps)
  have p-lt:  $\text{degree } p < n$ 
  using Cons.prem by simp
  have ps-lt:  $\text{degree } (\text{sum-list } ps) < n$ 
  by (rule Cons.IH) (use Cons.prem in auto)
  show ?case
  using degree-add-less[OF p-lt ps-lt] by simp
  qed

```

```

lemma degree-sum-list-le:
  fixes ps :: 'f poly list
  assumes  $\bigwedge p. p \in \text{set } ps \implies \text{degree } p \leq n$ 
  shows  $\text{degree } (\text{sum-list } ps) \leq n$ 
  using assms
proof (induction ps)
  case Nil
  then show ?case by simp
next
  case (Cons p ps)
  have p-le:  $\text{degree } p \leq n$ 
  using Cons.prem by simp

```

```

have ps-le: degree (sum-list ps) ≤ n
  by (rule Cons.IH) (use Cons.prem in auto)
show ?case
proof -
  have degree (p + sum-list ps) ≤ max (degree p) (degree (sum-list ps))
    by (rule degree-add-le-max)
  also have ... ≤ n
    using p-le ps-le by simp
  finally show ?thesis by simp
qed
qed

```

```

lemma prod-mset-eq:
  fixes xs ys :: 'f list
  assumes mset xs = mset ys
  shows prod xs = (prod ys :: 'f poly)
  using assms
  by (simp add: prod-alt flip: prod-mset-prod-list)

```

```

lemma prod-eval-nonzero:
  fixes roots :: 'f list
  assumes x ∉ set roots
  shows poly (prod roots :: 'f poly) x ≠ 0
  using assms
proof (induction roots)
  case Nil
  then show ?case
    unfolding prod-def by simp
next
  case (Cons r roots)
  have x-ne: x ≠ r
    using Cons.prem by simp
  have x-notin: x ∉ set roots
    using Cons.prem by simp
  have poly (prod (r # roots) :: 'f poly) x =
    (x - r) * poly (prod roots :: 'f poly) x
    unfolding prod-Cons XP-def CP-def
    by (simp add: poly-monom)
  then show ?case
    using x-ne Cons.IH[OF x-notin] by simp
qed

```

```

lemma prod-eval-zero:
  fixes roots :: 'f list
  assumes x ∈ set roots
  shows poly (prod roots :: 'f poly) x = 0
  using assms
proof (induction roots)
  case Nil

```

```

    then show ?case by simp
next
case (Cons r roots)
show ?case
proof (cases x = r)
  case True
  then show ?thesis
    unfolding prod-Cons XP-def CP-def
    by (simp add: poly-monom)
next
case False
then have x ∈ set roots
  using Cons.prem by simp
then show ?thesis
  using Cons.IH
  unfolding prod-Cons XP-def CP-def
  by (simp add: poly-monom)
qed
qed

lemma query-denominator-nonzero:
  assumes spec-entry: (c, roots, d) ∈ set spec
  and idx-bound: idx < clength * scale
  shows poly (prod (g-map roots)) (h ^ idx * shift) ≠ 0
proof -
  have h ^ idx * shift ∉ set (g-map roots)
    by (rule query-domain-disjoint[OF spec-entry idx-bound])
  then show ?thesis
    by (rule prod-eval-nonzero)
qed

lemma common-denominator-nonzero-on-eval-domain:
  assumes idx-bound: idx < clength * scale
  shows poly common-denominator-poly (h ^ idx * shift) ≠ 0
proof -
  have notin: h ^ idx * shift ∉ set all-constraint-roots
  proof
    assume in-roots: h ^ idx * shift ∈ set all-constraint-roots
    then obtain c roots d where spec-entry: (c, roots, d) ∈ set spec
      and in-g-map: h ^ idx * shift ∈ set (g-map roots)
    unfolding all-constraint-roots-def by (auto split: prod.splits)
    have h ^ idx * shift ∉ set (g-map roots)
      by (rule query-domain-disjoint[OF spec-entry idx-bound])
    then show False
      using in-g-map by simp
  qed
then show ?thesis
  unfolding common-denominator-poly-def
  by (rule prod-eval-nonzero)

```

qed

lemma *g-power-in-g-map*:

assumes $r \in \text{set roots}$
shows $g \wedge r \in \text{set } (g\text{-map roots})$
using *assms* **unfolding** *g-map-def* **by** *simp*

lemma *all-constraint-roots-distinct*[*simp*]:

distinct all-constraint-roots
unfolding *all-constraint-roots-def* **by** *simp*

lemma *g-map-subset-all-constraint-roots*:

assumes *spec-entry*: $(c, \text{roots}, d) \in \text{set spec}$
shows $\text{set } (g\text{-map roots}) \subseteq \text{set all-constraint-roots}$
using *spec-entry*
unfolding *all-constraint-roots-def*
by (*auto split: prod.splits*)

lemma *common-denominator-factor*:

assumes *spec-entry*: $(c, \text{roots}, d) \in \text{set spec}$
shows
 $\text{common-denominator-poly} =$
 $\text{prod } (g\text{-map roots}) * \text{constraint-root-cofactor roots}$

proof –

let *?rest* = $\text{filter } (\lambda x. x \notin \text{set } (g\text{-map roots})) \text{ all-constraint-roots}$
have *subset*: $\text{set } (g\text{-map roots}) \subseteq \text{set all-constraint-roots}$
by (*rule g-map-subset-all-constraint-roots[OF spec-entry]*)
have *distinct-roots*: *distinct* $(g\text{-map roots})$
by (*rule g-map-distinct[OF spec-entry]*)
have *distinct-all*: *distinct all-constraint-roots*
by *simp*
have *distinct-split*: *distinct* $(g\text{-map roots} @ \text{?rest})$
using *distinct-roots distinct-all subset* **by** *auto*
have *set-split*: $\text{set } (g\text{-map roots} @ \text{?rest}) = \text{set all-constraint-roots}$
using *subset* **by** *auto*
have *mset-eq*:
 $\text{mset all-constraint-roots} = \text{mset } (g\text{-map roots} @ \text{?rest})$
using *distinct-all distinct-split set-split*
by (*metis set-eq-iff-mset-eq-distinct*)
have *common-denominator-poly* = $\text{prod all-constraint-roots}$
unfolding *common-denominator-poly-def* **by** *simp*
also have $\dots = \text{prod } (g\text{-map roots} @ \text{?rest})$
by (*rule prod-mset-eq[OF mset-eq]*)
also have $\dots = \text{prod } (g\text{-map roots}) * \text{prod ?rest}$
by (*rule prod-append*)
finally show *?thesis*
unfolding *constraint-root-cofactor-def* .

qed

lemma *declared-quotient-degree-le-maxDegree*:
assumes *spec-entry*: $(c, \text{roots}, d) \in \text{set spec}$
shows $d * (\text{clength} - 1) - \text{length roots} \leq \text{maxDegree}$
proof –
have *deg-set*:
 $\{d * (\text{clength} - 1) - \text{length rs} \mid a \text{ rs } d. (a, \text{rs}, d) \in \text{set spec}\} =$
 $(\lambda(a, \text{rs}, d). d * (\text{clength} - 1) - \text{length rs}) \text{ ` set spec}$
by *force*
have *finite-degrees*: *finite* (*degrees clength spec*)
unfolding *degrees-def deg-set* **by** *simp*
have *entry-degree*:
 $d * (\text{clength} - 1) - \text{length roots} \in \text{degrees clength spec}$
using *spec-entry* **unfolding** *degrees-def* **by** *blast*
show *?thesis*
unfolding *maxDegree-def*
by (*rule Max-ge*[*OF finite-degrees entry-degree*])
qed

lemma *constraint-roots-length-le-all-constraint-roots*:
assumes *spec-entry*: $(c, \text{roots}, d) \in \text{set spec}$
shows $\text{length roots} \leq \text{length all-constraint-roots}$
proof –
have *subset*: $\text{set } (g\text{-map roots}) \subseteq \text{set all-constraint-roots}$
by (*rule g-map-subset-all-constraint-roots*[*OF spec-entry*])
have *distinct-roots*: *distinct* (*g-map roots*)
by (*rule g-map-distinct*[*OF spec-entry*])
have *card-le*:
 $\text{card } (\text{set } (g\text{-map roots})) \leq \text{card } (\text{set all-constraint-roots})$
by (*rule card-mono*) (*use subset in auto*)
have $\text{length roots} = \text{length } (g\text{-map roots})$
unfolding *g-map-def* **by** *simp*
also have $\dots = \text{card } (\text{set } (g\text{-map roots}))$
using *distinct-roots* **by** (*simp add: distinct-card*)
also have $\dots \leq \text{card } (\text{set all-constraint-roots})$
by (*rule card-le*)
also have $\dots = \text{length all-constraint-roots}$
by (*simp add: distinct-card*)
finally show *?thesis* .
qed

lemma *constraint-root-cofactor-degree-le*:
assumes *spec-entry*: $(c, \text{roots}, d) \in \text{set spec}$
shows
 $\text{degree } (\text{constraint-root-cofactor roots}) \leq$
 $\text{length all-constraint-roots} - \text{length roots}$
proof –
let *?rest* = *filter* ($\lambda x. x \notin \text{set } (g\text{-map roots})$) *all-constraint-roots*
have *subset*: $\text{set } (g\text{-map roots}) \subseteq \text{set all-constraint-roots}$
by (*rule g-map-subset-all-constraint-roots*[*OF spec-entry*])

```

have distinct-roots: distinct (g-map roots)
  by (rule g-map-distinct[OF spec-entry])
have distinct-rest: distinct ?rest
  by simp
have rest-set: set ?rest = set all-constraint-roots - set (g-map roots)
  by auto
have rest-len:
  length ?rest = length all-constraint-roots - length roots
proof -
  have length ?rest = card (set ?rest)
    using distinct-card[OF distinct-rest] by simp
  also have ... = card (set all-constraint-roots - set (g-map roots))
    using rest-set by simp
  also have ... =
    card (set all-constraint-roots) - card (set (g-map roots))
    by (rule card-Diff-subset) (use subset in auto)
  also have ... = length all-constraint-roots - length roots
proof -
  have card (set (g-map roots)) = length (g-map roots)
    using distinct-roots by (simp add: distinct-card)
  also have ... = length roots
    unfolding g-map-def by simp
  finally show ?thesis
    by (simp add: distinct-card)
qed
finally show ?thesis .
qed
have degree (constraint-root-cofactor roots) ≤ length ?rest
  unfolding constraint-root-cofactor-def by (rule degree-prod-le-length)
then show ?thesis
  unfolding rest-len .
qed

lemma common-denominator-poly-degree-le:
  degree common-denominator-poly ≤ length all-constraint-roots
  unfolding common-denominator-poly-def
  by (rule degree-prod-le-length)

lemma common-denominator-numerator-term-degree-lt:
  assumes wf: spec-degree-wellformed
  and deg-f: degree f < clength
  and spec-entry: (c, roots, d) ∈ set spec
  shows
    degree (common-denominator-numerator-term f (a, (c, roots, d))) <
      query-sample-space-size
proof -
  let ?dc = d * (clength - 1)
  let ?all = length all-constraint-roots
  have deg-c:

```



```

    degree (c (trace-powers-of f)) ≤ ?dc
  by (rule spec-degree-wellformed-constraint-degree
      [OF wf deg-f spec-entry])
have deg-cofactor:
  degree (constraint-root-cofactor roots) ≤ ?all - length roots
  by (rule constraint-root-cofactor-degree-le[OF spec-entry])
have deg-cp: degree (CP a :: 'f poly) ≤ 0
  unfolding CP-def by (rule degree-monom-le)
have term-le:
  degree (CP a * c (trace-powers-of f) *
    constraint-root-cofactor roots) ≤
    ?dc + (?all - length roots)
proof -
  have degree (CP a * c (trace-powers-of f) *
    constraint-root-cofactor roots) ≤
    degree (CP a * c (trace-powers-of f)) +
    degree (constraint-root-cofactor roots)
  by (rule degree-mult-le)
  also have ... ≤
    degree (CP a :: 'f poly) + degree (c (trace-powers-of f)) +
    degree (constraint-root-cofactor roots)
  using degree-mult-le[of CP a :: 'f poly c (trace-powers-of f)]
  by linarith
  also have ... ≤ ?dc + (?all - length roots)
  using deg-cp deg-c deg-cofactor by linarith
  finally show ?thesis .
qed
have roots-le-all: length roots ≤ ?all
  by (rule constraint-roots-length-le-all-constraint-roots[OF spec-entry])
have quotient-le:
  ?dc - length roots ≤ maxDegree
  by (rule declared-quotient-degree-le-maxDegree[OF spec-entry])
have arith:
  ?dc + (?all - length roots) ≤ maxDegree + ?all
  using roots-le-all quotient-le by linarith
have margin: maxDegree + ?all < query-sample-space-size
  using spec-degree-wellformed-query-margin[OF wf]
  unfolding query-agreement-bound-def by simp
show ?thesis
  using term-le arith margin
  unfolding common-denominator-numerator-term-def
  by simp
qed

lemma common-denominator-numerator-degree-lt:
  assumes wf: spec-degree-wellformed
  and deg-f: degree f < clength
  shows degree (common-denominator-numerator f as) < query-sample-space-size
proof -

```

```

have term-lt:
   $\bigwedge p. p \in$ 
    set (map (common-denominator-numerator-term f) (zip as spec))  $\implies$ 
      degree p < query-sample-space-size
proof –
  fix p
  assume p-in:
    p  $\in$  set (map (common-denominator-numerator-term f) (zip as spec))
  then obtain z where
    z-in: z  $\in$  set (zip as spec)
    and p-eq: p = common-denominator-numerator-term f z
    by auto
  obtain a entry where z-eq: z = (a, entry)
    by (cases z) auto
  obtain c roots d where entry-eq: entry = (c, roots, d)
    by (cases entry) auto
  have spec-entry: (c, roots, d)  $\in$  set spec
    using z-in unfolding z-eq entry-eq by (auto simp: set-zip)
  show degree p < query-sample-space-size
    unfolding p-eq z-eq entry-eq
    by (rule common-denominator-numerator-term-degree-lt
      [OF wf deg-f spec-entry])
  qed
show ?thesis
  unfolding common-denominator-numerator-def
  by (rule degree-sum-list-less[OF term-lt query-sample-space-size-pos])
qed

lemma common-denominator-product-degree-lt:
  assumes wf: spec-degree-wellformed
  and deg-q: degree q  $\leq$  maxDegree
  shows degree (common-denominator-poly * q) < query-sample-space-size
proof –
  have degree (common-denominator-poly * q)  $\leq$ 
    degree common-denominator-poly + degree q
    by (rule degree-mult-le)
  also have ...  $\leq$  length all-constraint-roots + maxDegree
    using common-denominator-poly-degree-le deg-q by linarith
  also have ... = maxDegree + length all-constraint-roots
    by simp
  also have ... < query-sample-space-size
    using spec-degree-wellformed-query-margin[OF wf]
    unfolding query-agreement-bound-def by simp
  finally show ?thesis .
qed

lemma common-denominator-numerator-term-degree-le-bound:
  assumes deg-f: degree f < clength
  and spec-entry: (c, roots, d)  $\in$  set spec

```

```

shows
  degree (common-denominator-numerator-term f (a, (c, roots, d))) ≤
    query-agreement-bound
proof -
  let ?dc = d * (clength - 1)
  let ?all = length all-constraint-roots
  have deg-c:
    degree (c (trace-powers-of f)) ≤ ?dc
  by (rule constraint-degree-wellformed[OF deg-f spec-entry])
  have deg-cofactor:
    degree (constraint-root-cofactor roots) ≤ ?all - length roots
  by (rule constraint-root-cofactor-degree-le[OF spec-entry])
  have deg-cp: degree (CP a :: 'f poly) ≤ 0
  unfolding CP-def by (rule degree-monom-le)
  have term-le:
    degree (CP a * c (trace-powers-of f) *
      constraint-root-cofactor roots) ≤
      ?dc + (?all - length roots)
  proof -
    have degree (CP a * c (trace-powers-of f) *
      constraint-root-cofactor roots) ≤
      degree (CP a * c (trace-powers-of f)) +
      degree (constraint-root-cofactor roots)
    by (rule degree-mult-le)
    also have ... ≤
      degree (CP a :: 'f poly) + degree (c (trace-powers-of f)) +
      degree (constraint-root-cofactor roots)
    using degree-mult-le[of CP a :: 'f poly c (trace-powers-of f)]
    by linarith
    also have ... ≤ ?dc + (?all - length roots)
    using deg-cp deg-c deg-cofactor by linarith
    finally show ?thesis .
  qed
  have roots-le-all: length roots ≤ ?all
  by (rule constraint-roots-length-le-all-constraint-roots[OF spec-entry])
  have quotient-le:
    ?dc - length roots ≤ maxDegree
  by (rule declared-quotient-degree-le-maxDegree[OF spec-entry])
  have ?dc + (?all - length roots) ≤ maxDegree + ?all
  using roots-le-all quotient-le by linarith
  then show ?thesis
  using term-le unfolding common-denominator-numerator-term-def
    query-agreement-bound-def
  by simp
qed

lemma common-denominator-numerator-degree-le-bound:
  assumes deg-f: degree f < clength
  shows degree (common-denominator-numerator f as) ≤ query-agreement-bound

```

```

proof –
  have term-le:
     $\bigwedge p. p \in$ 
       $\text{set } (\text{map } (\text{common-denominator-numerator-term } f) (\text{zip } \text{as } \text{spec})) \implies$ 
       $\text{degree } p \leq \text{query-agreement-bound}$ 
proof –
  fix p
  assume p-in:
     $p \in \text{set } (\text{map } (\text{common-denominator-numerator-term } f) (\text{zip } \text{as } \text{spec}))$ 
  then obtain z where
    z-in:  $z \in \text{set } (\text{zip } \text{as } \text{spec})$ 
    and p-eq:  $p = \text{common-denominator-numerator-term } f \ z$ 
    by auto
  obtain a entry where z-eq:  $z = (a, \text{entry})$ 
    by (cases z) auto
  obtain c roots d where entry-eq:  $\text{entry} = (c, \text{roots}, d)$ 
    by (cases entry) auto
  have spec-entry:  $(c, \text{roots}, d) \in \text{set } \text{spec}$ 
    using z-in unfolding z-eq entry-eq by (auto simp: set-zip)
  show  $\text{degree } p \leq \text{query-agreement-bound}$ 
    unfolding p-eq z-eq entry-eq
    by (rule common-denominator-numerator-term-degree-le-bound
      [OF deg-f spec-entry])
qed
show ?thesis
  unfolding common-denominator-numerator-def
  by (rule degree-sum-list-le [OF term-le])
qed

lemma common-denominator-product-degree-le-bound:
  assumes deg-q:  $\text{degree } q \leq \text{maxDegree}$ 
  shows  $\text{degree } (\text{common-denominator-poly} * q) \leq \text{query-agreement-bound}$ 
proof –
  have  $\text{degree } (\text{common-denominator-poly} * q) \leq$ 
     $\text{degree } \text{common-denominator-poly} + \text{degree } q$ 
    by (rule degree-mult-le)
  also have  $\dots \leq \text{length } \text{all-constraint-roots} + \text{maxDegree}$ 
    using common-denominator-poly-degree-le deg-q by linarith
  also have  $\dots = \text{query-agreement-bound}$ 
    unfolding query-agreement-bound-def by simp
  finally show ?thesis .
qed

lemma common-denominator-residual-degree-le-bound:
  assumes deg-f:  $\text{degree } f < \text{clength}$ 
  and deg-q:  $\text{degree } q \leq \text{maxDegree}$ 
  shows
     $\text{degree } (\text{common-denominator-numerator } f \text{ as } - \text{common-denominator-poly} * q)$ 
     $\leq \text{query-agreement-bound}$ 

```

proof –
have *num-le*:
 $\text{degree } (\text{common-denominator-numerator } f \text{ as}) \leq \text{query-agreement-bound}$
by (rule *common-denominator-numerator-degree-le-bound*[*OF deg-f*])
have *prod-le*:
 $\text{degree } (\text{common-denominator-poly} * q) \leq \text{query-agreement-bound}$
by (rule *common-denominator-product-degree-le-bound*[*OF deg-q*])
show ?thesis
using *degree-diff-le*[*OF num-le prod-le*] .
qed

lemma *common-denominator-degree-bounds-from-spec-degree-wellformed*:
assumes *wf*: *spec-degree-wellformed*
and *deg-f*: $\text{degree } f < \text{clength}$
shows *common-denominator-degree-bounds f as*
unfolding *common-denominator-degree-bounds-def*
using *common-denominator-numerator-degree-lt*[*OF wf deg-f*]
common-denominator-product-degree-lt[*OF wf*]
by *simp*

lemma *common-denominator-vanishes-on-constraint-root*:
assumes *spec-entry*: $(c, \text{roots}, d) \in \text{set spec}$
and *r-in*: $r \in \text{set roots}$
shows $\text{poly } \text{common-denominator-poly } (g \wedge r) = 0$
proof –
have *factor*:
 $\text{common-denominator-poly} =$
 $\text{prod } (g\text{-map roots}) * \text{constraint-root-cofactor roots}$
by (rule *common-denominator-factor*[*OF spec-entry*])
have *root-in*: $g \wedge r \in \text{set } (g\text{-map roots})$
by (rule *g-power-in-g-map*[*OF r-in*])
have $\text{poly } (\text{prod } (g\text{-map roots})) (g \wedge r) = 0$
by (rule *prod-eval-zero*[*OF root-in*])
then show ?thesis
using *factor* **by** *simp*
qed

lemma *g-power-notin-g-map-if-root-notin*:
assumes *spec-entry*: $(c, \text{roots}, d) \in \text{set spec}$
and *r-bound*: $r < \text{clength}$
and *r-notin*: $r \notin \text{set roots}$
shows $g \wedge r \notin \text{set } (g\text{-map roots})$
proof
assume $g \wedge r \in \text{set } (g\text{-map roots})$
then obtain *r'* **where** *r'-in*: $r' \in \text{set roots}$
and *eq*: $g \wedge r = g \wedge r'$
unfolding *g-map-def* **by** *auto*
have *r'-bound*: $r' < \text{clength}$
by (rule *spec-roots-in-range*[*OF spec-entry r'-in*])

```

have r = r'
  by (rule g-power-inj-on-range[OF r-bound r'-bound eq])
then show False
  using r-notin r'-in by simp
qed

lemma constraint-root-cofactor-eval-nonzero:
  assumes spec-entry: (c, roots, d) ∈ set spec
  and r-in: r ∈ set roots
  shows poly (constraint-root-cofactor roots) (g ^ r) ≠ 0
proof -
  have root-in: g ^ r ∈ set (g-map roots)
  by (rule g-power-in-g-map[OF r-in])
  have g ^ r ∉
    set (filter (λx. x ∉ set (g-map roots)) all-constraint-roots)
  using root-in by auto
  then show ?thesis
  unfolding constraint-root-cofactor-def
  by (rule prod-eval-nonzero)
qed

lemma constraint-root-cofactor-kills-other-root:
  assumes root-spec: (c0, roots0, d0) ∈ set spec
  and r-in0: r ∈ set roots0
  and spec-entry: (c, roots, d) ∈ set spec
  and r-notin: r ∉ set roots
  shows poly (constraint-root-cofactor roots) (g ^ r) = 0
proof -
  have r-bound: r < clength
  by (rule spec-roots-in-range[OF root-spec r-in0])
  have root-in-all: g ^ r ∈ set all-constraint-roots
  using g-map-subset-all-constraint-roots[OF root-spec]
  g-power-in-g-map[OF r-in0]
  by auto
  have root-notin-entry: g ^ r ∉ set (g-map roots)
  by (rule g-power-notin-g-map-if-root-notin
    [OF spec-entry r-bound r-notin])
  have g ^ r ∈
    set (filter (λx. x ∉ set (g-map roots)) all-constraint-roots)
  using root-in-all root-notin-entry by simp
  then show ?thesis
  unfolding constraint-root-cofactor-def
  by (rule prod-eval-zero)
qed

lemma common-denominator-residual-coefficient-nonzero:
  assumes spec-entry: (c, roots, d) ∈ set spec
  and r-in: r ∈ set roots
  and nonzero:

```

```

    poly (c (trace-powers-of f)) (g ^ r) ≠ 0
shows
    poly (c (trace-powers-of f)) (g ^ r) *
      poly (constraint-root-cofactor roots) (g ^ r) ≠ 0
using nonzero constraint-root-cofactor-eval-nonzero[OF spec-entry r-in]
by simp

lemma common-denominator-numerator-root-eval:
assumes len: length as = length spec
and root-spec: (c0, roots0, d0) ∈ set spec
and r-in0: r ∈ set roots0
shows
    poly (common-denominator-numerator f as) (g ^ r) =
      common-denominator-composition-root-residual f as r
proof -
have lhs:
    poly (common-denominator-numerator f as) (g ^ r) =
      (∑ i ∈ {0..using len
unfolding common-denominator-numerator-def
by (simp add: sum-list-sum-nth poly-sum
      common-denominator-numerator-term-def split-beta' case-prod-unfold
      CP-def poly-monom algebra-simps)
have rhs:
    common-denominator-composition-root-residual f as r =
      (∑ i ∈ {0..using len
unfolding common-denominator-composition-root-residual-def
by (simp add: sum-list-sum-nth split-beta')
have pointwise:
    ∧i. i ∈ {0..proof -
fix i
assume i-in: i ∈ {0..then have i-bound: i < length spec

```

```

  by simp
obtain c roots d where spec-i: spec ! i = (c, roots, d)
  by (cases spec ! i) auto
have spec-entry: (c, roots, d) ∈ set spec
  using nth-mem[OF i-bound] spec-i by simp
show
  poly
    (case spec ! i of (c, roots, d) ⇒
      CP (as ! i) * c (trace-powers-of f) *
        constraint-root-cofactor roots)
    (g ^ r) =
  common-denominator-constraint-root-residual f r
  (as ! i, spec ! i)
proof (cases r ∈ set roots)
case True
  then show ?thesis
    using spec-i
    unfolding common-denominator-constraint-root-residual-def
    by (simp add: CP-def poly-monom)
next
case False
  have kill:
    poly (constraint-root-cofactor roots) (g ^ r) = 0
  by (rule constraint-root-cofactor-kills-other-root
    [OF root-spec r-in0 spec-entry False])
  show ?thesis
    using spec-i False kill
    unfolding common-denominator-constraint-root-residual-def
    by (simp add: CP-def poly-monom)
qed
qed
show ?thesis
  unfolding lhs rhs
  by (intro sum.cong refl pointwise) simp
qed

lemma common-denominator-mult-composition-spec-eval:
  assumes denom-nonzero:
     $\bigwedge c \text{ roots } d. (c, \text{roots}, d) \in \text{set spec} \implies$ 
    poly (prod (g-map roots)) x  $\neq 0$ 
  shows
    poly common-denominator-poly x * composition-spec-eval f as x =
    poly (common-denominator-numerator f as) x
proof -
  let ?D = poly common-denominator-poly x
  have term-eval:
     $\bigwedge z. z \in \text{set } (\text{zip as spec}) \implies$ 
    ?D * composition-spec-eval-term f x z =
    poly (common-denominator-numerator-term f z) x

```



```

proof –
  fix  $z$ 
  assume  $z\text{-in}: z \in \text{set } (\text{zip as spec})$ 
  obtain  $a \text{ entry}$  where  $z\text{-eq}: z = (a, \text{entry})$ 
    by  $(\text{cases } z) \text{ auto}$ 
  obtain  $c \text{ roots } d$  where  $\text{entry}\text{-eq}: \text{entry} = (c, \text{roots}, d)$ 
    by  $(\text{cases entry}) \text{ auto}$ 
  have  $\text{entry}\text{-in}: \text{entry} \in \text{set spec}$ 
  proof –
    obtain  $i$  where  $i\text{-bound}: i < \text{length as } i < \text{length spec}$ 
      and  $z\text{-i}: z = (\text{as } ! i, \text{spec } ! i)$ 
      using  $z\text{-in}$  by  $(\text{auto simp: set-zip})$ 
    have  $\text{entry} = \text{spec } ! i$ 
      using  $z\text{-eq } z\text{-i}$  by  $\text{simp}$ 
    then show  $?thesis$ 
      using  $\text{nth}\text{-mem}[\text{OF } i\text{-bound}(2)]$  by  $\text{simp}$ 
  qed
  have  $\text{spec}\text{-entry}: (c, \text{roots}, d) \in \text{set spec}$ 
    using  $\text{entry}\text{-in } \text{entry}\text{-eq}$  by  $\text{simp}$ 
  have  $\text{factor}:$ 
     $?D =$ 
     $\text{poly } (\text{prod } (g\text{-map roots})) x *$ 
     $\text{poly } (\text{constraint}\text{-root}\text{-cofactor roots}) x$ 
    using  $\text{common}\text{-denominator}\text{-factor}[\text{OF spec}\text{-entry}]$ 
    by  $\text{simp}$ 
  have  $\text{denom}: \text{poly } (\text{prod } (g\text{-map roots})) x \neq 0$ 
    by  $(\text{rule denom}\text{-nonzero}[\text{OF spec}\text{-entry}])$ 
  show
     $?D * \text{composition}\text{-spec}\text{-eval}\text{-term } f x z =$ 
     $\text{poly } (\text{common}\text{-denominator}\text{-numerator}\text{-term } f z) x$ 
    using  $z\text{-eq } \text{entry}\text{-eq } \text{factor } \text{denom}$ 
    unfolding  $\text{composition}\text{-spec}\text{-eval}\text{-term}\text{-def}$ 
     $\text{common}\text{-denominator}\text{-numerator}\text{-term}\text{-def}$ 
    by  $(\text{simp add: field}\text{-simps CP}\text{-def poly}\text{-monom})$ 
  qed
  have  $\text{mult}\text{-sum}:$ 
     $?D * \text{sum}\text{-list } (\text{map } (\text{composition}\text{-spec}\text{-eval}\text{-term } f x) (\text{zip as spec})) =$ 
     $\text{sum}\text{-list}$ 
     $(\text{map } (\lambda z. \text{poly } (\text{common}\text{-denominator}\text{-numerator}\text{-term } f z) x)$ 
     $(\text{zip as spec}))$ 
  proof –
    have  $?D * \text{sum}\text{-list } (\text{map } (\text{composition}\text{-spec}\text{-eval}\text{-term } f x) (\text{zip as spec})) =$ 
     $\text{sum}\text{-list}$ 
     $(\text{map } (\lambda z. ?D * \text{composition}\text{-spec}\text{-eval}\text{-term } f x z)$ 
     $(\text{zip as spec}))$ 
    by  $(\text{simp add: sum}\text{-list}\text{-const}\text{-mult algebra}\text{-simps})$ 
  also have  $\dots =$ 
     $\text{sum}\text{-list}$ 
     $(\text{map } (\lambda z. \text{poly } (\text{common}\text{-denominator}\text{-numerator}\text{-term } f z) x)$ 

```

```

      (zip as spec))
    using term-eval by (intro arg-cong[where f=sum-list] map-cong) auto
  finally show ?thesis .
qed
have rhs:
  poly (common-denominator-numerator f as) x =
    sum-list
      (map (λz. poly (common-denominator-numerator-term f z) x)
        (zip as spec))
  unfolding common-denominator-numerator-def
  apply (simp add: poly-sum-list)
  apply (intro arg-cong[where f=sum-list] map-cong refl)
  apply (auto simp: common-denominator-numerator-term-def
    split-beta' case-prod-unfold algebra-simps CP-def poly-monom
    split: prod.splits)
  done
show ?thesis
  unfolding composition-spec-eval-def
  using mult-sum rhs by simp
qed

lemma common-denominator-composition-root-residual-sum-nth:
  assumes len: length as = length spec
  shows
    common-denominator-composition-root-residual f as r =
      (∑ i ∈ {0..

```

$(\sum j \in \{0..<\text{length spec}\} - \{i\}.$
 $\text{common-denominator-constraint-root-residual } f \ r \ (as ! j, \text{spec} ! j))$
proof –
let $?F =$
 $\lambda j. \text{common-denominator-constraint-root-residual } f \ r \ (as ! j, \text{spec} ! j)$
have $\text{common-denominator-composition-root-residual } f \ as \ r =$
 $(\sum j \in \{0..<\text{length spec}\}. ?F \ j)$
by $(\text{rule common-denominator-composition-root-residual-sum-nth}[OF \ \text{len}])$
also have $\dots = ?F \ i + (\sum j \in \{0..<\text{length spec}\} - \{i\}. ?F \ j)$
using $i\text{-bound}$ **by** $(\text{simp add: sum.remove})$
finally show $?thesis$.
qed

lemma $\text{common-denominator-composition-root-residual-zero-unique-nth}$:
fixes $as \ bs :: 'f \ \text{list}$
assumes $\text{len-as: length } as = \text{length spec}$
and $\text{len-bs: length } bs = \text{length spec}$
and $i\text{-bound: } i < \text{length spec}$
and $\text{spec-i: spec} ! i = (c, \text{roots}, d)$
and $r\text{-in: } r \in \text{set roots}$
and nonzero:
 $\text{poly } (c \ (\text{trace-powers-of } f)) \ (g \wedge r) \neq 0$
and outside:
 $\bigwedge j. j < \text{length spec} \implies j \neq i \implies as ! j = bs ! j$
and $\text{zero-as: common-denominator-composition-root-residual } f \ as \ r = 0$
and $\text{zero-bs: common-denominator-composition-root-residual } f \ bs \ r = 0$
shows $as ! i = bs ! i$

proof –
let $?C =$
 $\text{poly } (c \ (\text{trace-powers-of } f)) \ (g \wedge r) *$
 $\text{poly } (\text{constraint-root-cofactor roots}) \ (g \wedge r)$
let $?B =$
 $(\sum j \in \{0..<\text{length spec}\} - \{i\}.$
 $\text{common-denominator-constraint-root-residual } f \ r \ (as ! j, \text{spec} ! j))$
have $\text{spec-entry: } (c, \text{roots}, d) \in \text{set spec}$
using $\text{nth-mem}[OF \ i\text{-bound}] \ \text{spec-i}$ **by** simp
have $C\text{-nonzero: } ?C \neq 0$
by $(\text{rule common-denominator-residual-coefficient-nonzero}$
 $[OF \ \text{spec-entry } r\text{-in nonzero}])$
have $B\text{-eq:}$
 $(\sum j \in \{0..<\text{length spec}\} - \{i\}.$
 $\text{common-denominator-constraint-root-residual } f \ r \ (bs ! j, \text{spec} ! j)) = ?B$
by $(\text{intro sum.cong}) \ (\text{use outside in auto})$
have $as\text{-eq:}$
 $\text{common-denominator-composition-root-residual } f \ as \ r =$
 $?C * (as ! i) + ?B$
using $\text{common-denominator-composition-root-residual-split-index}$
 $[OF \ \text{len-as } i\text{-bound}]$
 $\text{spec-i } r\text{-in}$

```

unfolding common-denominator-constraint-root-residual-def
by (simp add: algebra-simps)
have bs-eq:
  common-denominator-composition-root-residual f bs r =
    ?C * (bs ! i) + ?B
using common-denominator-composition-root-residual-split-index
  [OF len-bs i-bound]
  spec-i r-in B-eq
unfolding common-denominator-constraint-root-residual-def
by (simp add: algebra-simps)
show ?thesis
proof (rule linear-equation-solution-unique[OF C-nonzero])
  show ?C * (as ! i) + ?B = 0
    using as-eq zero-as by simp
  show ?C * (bs ! i) + ?B = 0
    using bs-eq zero-bs by simp
qed
qed

lemma random-combination-common-denominator-hides-violations-unique-nth:
  fixes as bs :: 'f list
  assumes len-as: length as = length spec
    and len-bs: length bs = length spec
    and i-bound: i < length spec
    and spec-i: spec ! i = (c, roots, d)
    and violated: (c, roots, d) ∈ violated-constraints f
    and r-in: r ∈ set roots
    and nonzero:
      poly (c (trace-powers-of f)) (g ^ r) ≠ 0
    and outside:
       $\bigwedge j. j < \text{length spec} \implies j \neq i \implies \text{as ! } j = \text{bs ! } j$ 
    and hides-as: random-combination-common-denominator-hides-violations f as
    and hides-bs: random-combination-common-denominator-hides-violations f bs
  shows as ! i = bs ! i
proof –
  have zero-as:
    common-denominator-composition-root-residual f as r = 0
    using hides-as violated r-in
    unfolding random-combination-common-denominator-hides-violations-def
    by auto
  have zero-bs:
    common-denominator-composition-root-residual f bs r = 0
    using hides-bs violated r-in
    unfolding random-combination-common-denominator-hides-violations-def
    by auto
  show ?thesis
    by (rule common-denominator-composition-root-residual-zero-unique-nth
      [OF len-as len-bs i-bound spec-i r-in nonzero outside zero-as zero-bs])
qed

```

lemma *violated-constraints-has-common-denominator-coefficient:*

assumes *violated:* *violated-constraints* $f \neq \{\}$

obtains i c *roots* d r

where $i < \text{length } \text{spec}$

and $\text{spec } ! i = (c, \text{roots}, d)$

and $(c, \text{roots}, d) \in \text{violated-constraints } f$

and $r \in \text{set roots}$

and

$\text{poly } (c (\text{trace-powers-of } f)) (g \wedge r) *$

$\text{poly } (\text{constraint-root-cofactor roots}) (g \wedge r) \neq 0$

proof –

from *violated-constraints-has-remainder-bad* [*OF* *violated*]

obtain *entry* **where** *entry0*:

$\text{entry} \in \text{violated-constraints } f$

$\text{constraint-remainder-bad } f \text{ entry}$

by *blast*

obtain c *roots* d **where** *entry-eq*: $\text{entry} = (c, \text{roots}, d)$

by (*cases entry*) *auto*

have *entry*:

$(c, \text{roots}, d) \in \text{violated-constraints } f$

$\text{constraint-remainder-bad } f (c, \text{roots}, d)$

using *entry0* **unfolding** *entry-eq* **by** *simp-all*

then obtain r **where** r :

$r \in \text{set roots}$

$\text{poly } (c (\text{trace-powers-of } f)) (g \wedge r) \neq 0$

unfolding *constraint-remainder-bad-def* **by** *auto*

have *spec-entry*: $(c, \text{roots}, d) \in \text{set spec}$

using *entry(1)* **unfolding** *violated-constraints-def* **by** *auto*

then obtain i **where** i :

$i < \text{length spec}$

$\text{spec } ! i = (c, \text{roots}, d)$

by (*auto simp: in-set-conv-nth*)

have *coeff*:

$\text{poly } (c (\text{trace-powers-of } f)) (g \wedge r) *$

$\text{poly } (\text{constraint-root-cofactor roots}) (g \wedge r) \neq 0$

by (*rule common-denominator-residual-coefficient-nonzero*
[*OF spec-entry r(1) r(2)*])

show *?thesis*

by (*rule that* [*OF i entry(1) r(1) coeff*])

qed

lemma *random-combination-common-denominator-hides-has-coefficient:*

assumes *hides:* *random-combination-common-denominator-hides-violations* f *as*

obtains i c *roots* d r

where $i < \text{length spec}$

and $\text{spec } ! i = (c, \text{roots}, d)$

and $(c, \text{roots}, d) \in \text{violated-constraints } f$

and $r \in \text{set roots}$

```

and
  poly (c (trace-powers-of f)) (g ^ r) *
  poly (constraint-root-cofactor roots) (g ^ r) ≠ 0
proof -
  have violated: violated-constraints f ≠ {}
  using hides
  unfolding random-combination-common-denominator-hides-violations-def
  by simp
  show ?thesis
  by (rule violated-constraints-has-common-denominator-coefficient[OF violated])
    (rule that)
qed

```

```

definition alpha-space :: 'f list set
where alpha-space = {as. length as = length spec}

```

```

definition common-denominator-hiding-alpha-space :: 'f poly ⇒ 'f list set
where
  common-denominator-hiding-alpha-space f =
    {as ∈ alpha-space.
      random-combination-common-denominator-hides-violations f as}

```

```

lemma finite-length-lists-UNIV:
  finite {xs :: 'f list. length xs = n}
proof -
  have {xs :: 'f list. length xs = n} =
    {xs. set xs ⊆ (UNIV :: 'f set) ∧ length xs = n}
  by auto
  then show ?thesis
  using finite-lists-length-eq[of UNIV :: 'f set n] by simp
qed

```

```

lemma card-length-lists-UNIV:
  card {xs :: 'f list. length xs = n} = CARD('f) ^ n
proof -
  have {xs :: 'f list. length xs = n} =
    {xs. set xs ⊆ (UNIV :: 'f set) ∧ length xs = n}
  by auto
  then show ?thesis
  using card-lists-length-eq[of UNIV :: 'f set n] by simp
qed

```

```

definition fri-challenge-space :: nat ⇒ 'f list set
where fri-challenge-space n = {bs. length bs = n}

```

```

lemma finite-fri-challenge-space[simp]:
  finite (fri-challenge-space n)
  unfolding fri-challenge-space-def
  by (rule finite-length-lists-UNIV)

```

```

lemma card-fri-challenge-space:
  card (fri-challenge-space n) = CARD('f) ^ n
  unfolding fri-challenge-space-def
  by (rule card-length-lists-UNIV)

lemma card-alpha-space[simp]:
  card alpha-space = CARD('f) ^ length spec
  unfolding alpha-space-def by (rule card-length-lists-UNIV)

lemma common-denominator-hiding-alpha-space-card-bound:
  card (common-denominator-hiding-alpha-space f) ≤
    (if length spec = 0 then 0 else CARD('f) ^ (length spec - 1))
proof (cases common-denominator-hiding-alpha-space f = {})
  case True
  then show ?thesis by simp
next
  case False
  let ?S = common-denominator-hiding-alpha-space f
  from False obtain as0 where as0-in: as0 ∈ ?S
  by auto
  have hides0: random-combination-common-denominator-hides-violations f as0
  using as0-in unfolding common-denominator-hiding-alpha-space-def by simp
  from random-combination-common-denominator-hides-has-coefficient[OF hides0]
  obtain i c roots d r where
    i-bound: i < length spec
    and spec-i: spec ! i = (c, roots, d)
    and violated: (c, roots, d) ∈ violated-constraints f
    and r-in: r ∈ set roots
    and coeff-nonzero:
      poly (c (trace-powers-of f)) (g ^ r) *
      poly (constraint-root-cofactor roots) (g ^ r) ≠ 0
  by blast
  have nonzero: poly (c (trace-powers-of f)) (g ^ r) ≠ 0
  using coeff-nonzero by auto
  let ?key = λxs :: 'f list. (take i xs, drop (Suc i) xs)
  have inj: inj-on ?key ?S
  proof (rule inj-onI)
    fix xs ys
    assume xs-in: xs ∈ ?S
    and ys-in: ys ∈ ?S
    and key-eq: ?key xs = ?key ys
    have len-xs: length xs = length spec
    using xs-in
    unfolding common-denominator-hiding-alpha-space-def alpha-space-def
    by simp
    have len-ys: length ys = length spec
    using ys-in
    unfolding common-denominator-hiding-alpha-space-def alpha-space-def

```

```

  by simp
have hides-xs:
  random-combination-common-denominator-hides-violations f xs
  using xs-in unfolding common-denominator-hiding-alpha-space-def by simp
have hides-ys:
  random-combination-common-denominator-hides-violations f ys
  using ys-in unfolding common-denominator-hiding-alpha-space-def by simp
have take-eq: take i xs = take i ys
  using key-eq by simp
have drop-eq: drop (Suc i) xs = drop (Suc i) ys
  using key-eq by simp
have outside:
   $\bigwedge j. j < \text{length spec} \implies j \neq i \implies xs ! j = ys ! j$ 
proof -
  fix j
  assume j-bound:  $j < \text{length spec}$ 
  and j-ne:  $j \neq i$ 
  show  $xs ! j = ys ! j$ 
  proof (cases  $j < i$ )
    case True
    have  $xs ! j = \text{take } i \text{ } xs ! j$ 
    using True by simp
    also have  $\dots = \text{take } i \text{ } ys ! j$ 
    using take-eq by simp
    also have  $\dots = ys ! j$ 
    using True by simp
    finally show ?thesis .
  next
    case False
    then have i-lt-j:  $i < j$ 
    using j-ne by linarith
    have j-decomp:  $j = \text{Suc } i + (j - \text{Suc } i)$ 
    using i-lt-j by simp
    have drop-idx-bound-xs:  $j - \text{Suc } i < \text{length } (\text{drop } (\text{Suc } i) \text{ } xs)$ 
    using j-bound i-lt-j len-xs by simp
    have drop-idx-bound-ys:  $j - \text{Suc } i < \text{length } (\text{drop } (\text{Suc } i) \text{ } ys)$ 
    using j-bound i-lt-j len-ys by simp
    have  $xs ! j = \text{drop } (\text{Suc } i) \text{ } xs ! (j - \text{Suc } i)$ 
    using drop-idx-bound-xs j-decomp by simp
    also have  $\dots = \text{drop } (\text{Suc } i) \text{ } ys ! (j - \text{Suc } i)$ 
    using drop-eq by simp
    also have  $\dots = ys ! j$ 
    using drop-idx-bound-ys j-decomp by simp
    finally show ?thesis .
  qed
qed
have nth-i:  $xs ! i = ys ! i$ 
  by (rule random-combination-common-denominator-hides-violations-unique-nth
    [OF len-xs len-ys i-bound spec-i violated r-in nonzero outside

```



```

      hides-xs hides-ys])
show xs = ys
proof (rule nth-equalityI)
  show length xs = length ys
    using len-xs len-ys by simp
next
  fix j
  assume j-bound: j < length xs
  then have j-spec: j < length spec
    using len-xs by simp
  show xs ! j = ys ! j
  proof (cases j = i)
    case True
    then show ?thesis
      using nth-i by simp
  next
    case False
    then show ?thesis
      by (rule outside[OF j-spec])
  qed
qed
qed
let ?A = {xs :: 'f list. length xs = i}
let ?B = {xs :: 'f list. length xs = length spec - Suc i}
have image-subset: ?key ' ?S  $\subseteq$  ?A  $\times$  ?B
proof
  fix x
  assume x  $\in$  ?key ' ?S
  then obtain xs where xs-in: xs  $\in$  ?S and x-eq: x = ?key xs
    by auto
  have len-xs: length xs = length spec
    using xs-in
    unfolding common-denominator-hiding-alpha-space-def alpha-space-def
    by simp
  show x  $\in$  ?A  $\times$  ?B
    using len-xs i-bound unfolding x-eq by simp
qed
have finite-A: finite ?A
  by (rule finite-length-lists-UNIV)
have finite-B: finite ?B
  by (rule finite-length-lists-UNIV)
have finite-prod: finite (?A  $\times$  ?B)
  using finite-A finite-B by simp
have finite-alpha: finite alpha-space
  unfolding alpha-space-def by (rule finite-length-lists-UNIV)
have finite-S: finite ?S
  unfolding common-denominator-hiding-alpha-space-def
  using finite-alpha by simp
have card-image-le: card (?key ' ?S)  $\leq$  card (?A  $\times$  ?B)

```

```

    by (rule card-mono[OF finite-prod image-subset])
  have card-S: card ?S = card (?key ‘ ?S)
    using card-image[OF inj] by simp
  have card-prod:
    card (?A × ?B) =
      CARD('f) ^ i * CARD('f) ^ (length spec - Suc i)
    using finite-A finite-B by (simp add: card-length-lists-UNIV)
  have exp-eq:
    i + (length spec - Suc i) = length spec - 1
    using i-bound by simp
  have card ?S ≤ CARD('f) ^ i * CARD('f) ^ (length spec - Suc i)
    using card-image-le card-S card-prod by simp
  also have ... = CARD('f) ^ (length spec - 1)
    using exp-eq by (simp add: power-add[symmetric])
  finally have bound:
    card ?S ≤ CARD('f) ^ (length spec - 1) .
  have length spec ≠ 0
    using i-bound by auto
  then show ?thesis
    using bound by simp
qed

lemma nnreal-nat-divide-right-mono:
  assumes a ≤ b
  shows nnreal a / nnreal c ≤ nnreal b / nnreal c
  using assms
  by transfer (simp add: divide-right-mono)

lemma nnreal-nat-divide-self:
  assumes k ≠ 0
  shows nnreal k / nnreal k = 1
  using assms by transfer simp

lemma nnreal-power-fraction-cancel:
  assumes k ≠ 0
  shows nnreal (k ^ n) / nnreal (k ^ Suc n) = 1 / nnreal k
  using assms
  by transfer (simp add: power-Suc field-simps)

lemma common-denominator-hiding-alpha-space-fraction-bound:
  nnreal (card (common-denominator-hiding-alpha-space f)) /
  nnreal (CARD('f) ^ length spec) ≤ composition-error-bound
proof (cases length spec)
  case 0
  then have card (common-denominator-hiding-alpha-space f) = 0
    using common-denominator-hiding-alpha-space-card-bound[of f] by simp
  then show ?thesis
    unfolding composition-error-bound-def by simp
next

```

```

case (Suc n)
have card-le:
  card (common-denominator-hiding-alpha-space f) ≤ CARD('f) ^ n
  using common-denominator-hiding-alpha-space-card-bound[of f]
  unfolding Suc by simp
have field-nonzero: CARD('f) ≠ 0
  by simp
have nnreal (card (common-denominator-hiding-alpha-space f)) /
  nnreal (CARD('f) ^ length spec) ≤
  nnreal (CARD('f) ^ n) / nnreal (CARD('f) ^ Suc n)
  unfolding Suc
  using card-le
  by (rule nnreal-nat-divide-right-mono)
also have ... = 1 / nnreal (CARD('f))
  by (rule nnreal-power-fraction-cancel[OF field-nonzero])
also have ... = composition-error-bound
  using size-card unfolding composition-error-bound-def by simp
finally show ?thesis .
qed

lemma low-degree-trace-table-unique:
  assumes deg-f: degree f < clength
  and deg-q: degree q < clength
  and table-eq: map (poly f) eval-domain = map (poly q) eval-domain
  shows f = q
proof -
  have clength-le-domain: clength ≤ clength * scale
  using scale-pos by simp
  have deg-f-domain: degree f < clength * scale
  using deg-f clength-le-domain by linarith
  have deg-q-domain: degree q < clength * scale
  using deg-q clength-le-domain by linarith
  show ?thesis
proof (rule poly-eq-on-eval-domainI[OF deg-f-domain deg-q-domain])
  fix idx
  assume idx-bound: idx < clength * scale
  have idx-len: idx < length eval-domain
  using idx-bound eval-domain-length by simp
  have map (poly f) eval-domain ! idx = map (poly q) eval-domain ! idx
  using table-eq idx-len by simp
  then show poly f (h ^ idx * shift) = poly q (h ^ idx * shift)
  using idx-len idx-bound eval-domain-nth by simp
qed
qed

definition low-degree-trace-witness :: 'f list ⇒ 'f poly
where
  low-degree-trace-witness trace-table =
    (SOME f. degree f < clength ∧

```

$trace-table = map (poly f) eval-domain)$

lemma *low-degree-trace-witness-eq:*

assumes *deg-f: degree f < clength*

and *trace-table: trace-table = map (poly f) eval-domain*

shows *low-degree-trace-witness trace-table = f*

unfolding *low-degree-trace-witness-def*

proof (*rule some-equality*)

show *degree f < clength $\wedge$ trace-table = map (poly f) eval-domain*

using *deg-f trace-table* **by** *simp*

next

fix *q*

assume *q:*

degree q < clength $\wedge$ trace-table = map (poly q) eval-domain

then have *table-eq: map (poly f) eval-domain = map (poly q) eval-domain*

using *trace-table* **by** *simp*

show *q = f*

using *low-degree-trace-table-unique[OF deg-f conjunct1[OF q] table-eq]*

by *simp*

qed

definition *composition-trace-bad-alpha-space :: 'f list $\Rightarrow$ 'f list set*

where

composition-trace-bad-alpha-space trace-table =

(let f = low-degree-trace-witness trace-table in

if degree f < clength $\wedge$

trace-table = map (poly f) eval-domain $\wedge$

violated-constraints f $\neq$ {}

then

{as $\in$ common-denominator-hiding-alpha-space f.

common-denominator-degree-bounds f as}

else {})

lemma *finite-alpha-space:*

finite alpha-space

unfolding *alpha-space-def* **by** (*rule finite-length-lists-UNIV*)

lemma *common-denominator-hiding-alpha-space-subset-alpha-space:*

common-denominator-hiding-alpha-space f $\subseteq$ alpha-space

unfolding *common-denominator-hiding-alpha-space-def* **by** *auto*

lemma *finite-common-denominator-hiding-alpha-space:*

finite (common-denominator-hiding-alpha-space f)

using *finite-alpha-space*

unfolding *common-denominator-hiding-alpha-space-def* **by** *simp*

lemma *composition-trace-bad-alpha-space-subset-alpha-space:*

composition-trace-bad-alpha-space trace-table $\subseteq$ alpha-space

proof —

```

let ?f = low-degree-trace-witness trace-table
have composition-trace-bad-alpha-space trace-table  $\subseteq$ 
  common-denominator-hiding-alpha-space ?f
  unfolding composition-trace-bad-alpha-space-def Let-def by auto
also have ...  $\subseteq$  alpha-space
  by (rule common-denominator-hiding-alpha-space-subset-alpha-space)
finally show ?thesis .
qed

```

```

lemma composition-trace-bad-alpha-space-fraction-bound:
  nnreal (card (composition-trace-bad-alpha-space trace-table)) /
  nnreal (CARD('f) ^ length spec)  $\leq$  composition-error-bound
proof -
  let ?f = low-degree-trace-witness trace-table
  have subset:
    composition-trace-bad-alpha-space trace-table  $\subseteq$ 
      common-denominator-hiding-alpha-space ?f
    unfolding composition-trace-bad-alpha-space-def Let-def by auto
  have card-le:
    card (composition-trace-bad-alpha-space trace-table)  $\leq$ 
      card (common-denominator-hiding-alpha-space ?f)
    by (rule card-mono[OF finite-common-denominator-hiding-alpha-space subset])
  have nnreal (card (composition-trace-bad-alpha-space trace-table)) /
    nnreal (CARD('f) ^ length spec)  $\leq$ 
      nnreal (card (common-denominator-hiding-alpha-space ?f)) /
      nnreal (CARD('f) ^ length spec)
    by (rule nnreal-nat-divide-right-mono[OF card-le])
  also have ...  $\leq$  composition-error-bound
    by (rule common-denominator-hiding-alpha-space-fraction-bound)
  finally show ?thesis .
qed

```

```

lemma composition-trace-bad-alpha-space-fraction-bound-alpha-space:
  nnreal (card (composition-trace-bad-alpha-space trace-table)) /
  nnreal (card alpha-space)  $\leq$  composition-error-bound
  using composition-trace-bad-alpha-space-fraction-bound[of trace-table]
  by simp

```

```

lemma composition-trace-bad-alpha-space-card-bound:
  card (composition-trace-bad-alpha-space trace-table)  $\leq$ 
    (if length spec = 0 then 0 else CARD('f) ^ (length spec - 1))
proof -
  let ?f = low-degree-trace-witness trace-table
  have subset:
    composition-trace-bad-alpha-space trace-table  $\subseteq$ 
      common-denominator-hiding-alpha-space ?f
    unfolding composition-trace-bad-alpha-space-def Let-def by auto
  have card-le:
    card (composition-trace-bad-alpha-space trace-table)  $\leq$ 

```

card (common-denominator-hiding-alpha-space ?f)
 by (rule card-mono[OF finite-common-denominator-hiding-alpha-space subset])
 also have ... $\leq$
 (if length spec = 0 then 0 else CARD('f) $\wedge$ (length spec - 1))
 by (rule common-denominator-hiding-alpha-space-card-bound)
 finally show ?thesis .
 qed

lemma trace-powers-of-eval-nth:
 assumes $k < \text{powers}$
 shows poly (trace-powers-of f ! k) x = poly f (g $\wedge$ k * x)
 using assms
 unfolding trace-powers-of-def XP-def
 by (simp add: poly-pcompose poly-monom)

lemma query-trace-power-eval-nth:
 assumes k-bound: $k < \text{powers}$
 and idx-bound: $\text{idx} + k * \text{scale} < \text{clength} * \text{scale}$
 shows
 map (poly f) eval-domain ! (idx + k * scale) =
 poly (trace-powers-of f ! k) (h $\wedge$ idx * shift)
proof –
 have eval:
 map (poly f) eval-domain ! (idx + k * scale) =
 poly f (h $\wedge$ (idx + k * scale) * shift)
 using eval-domain-nth[OF idx-bound] idx-bound eval-domain-length by simp
 have arg:
 g $\wedge$ k * (h $\wedge$ idx * shift) = h $\wedge$ (idx + k * scale) * shift
proof –
 have gpow: g $\wedge$ k = h $\wedge$ (scale * k)
 using domain-alignment by (simp add: power-mult)
 have g $\wedge$ k * (h $\wedge$ idx * shift) =
 h $\wedge$ (scale * k) * h $\wedge$ idx * shift
 using gpow by (simp add: algebra-simps)
 also have ... = h $\wedge$ (idx + k * scale) * shift
 by (simp add: power-add algebra-simps mult.commute)
 finally show ?thesis .
 qed
 show ?thesis
 using eval trace-powers-of-eval-nth[OF k-bound, of f h $\wedge$ idx * shift] arg
 by simp
 qed

lemma query-values-trace-powers:
 assumes trace-table: trace-table = map (poly f) eval-domain
 and scaled-bound:
 $\bigwedge i. i \in \text{set (powers-scaled idx)} \implies i < \text{clength} * \text{scale}$
 shows
 map (!) trace-table (powers-scaled idx) =

```

      map ( $\lambda q. \text{poly } q (h \wedge \text{idx} * \text{shift})$ ) ( $\text{trace-powers-of } f$ )
proof ( $\text{rule nth-equalityI}$ )
  show  $\text{length} (\text{map} (! \text{trace-table}) (\text{powers-scaled idx})) =$ 
     $\text{length} (\text{map} (\lambda q. \text{poly } q (h \wedge \text{idx} * \text{shift})) (\text{trace-powers-of } f))$ 
    unfolding  $\text{powers-scaled-def}$  by  $\text{simp}$ 
next
  fix  $k$ 
  assume  $k\text{-bound}$ :
     $k < \text{length} (\text{map} (! \text{trace-table}) (\text{powers-scaled idx}))$ 
  then have  $k\text{-lt}$ :  $k < \text{powers}$ 
    unfolding  $\text{powers-scaled-def}$  by  $\text{simp}$ 
  have  $\text{scaled-idx-in}$ :
     $\text{idx} + k * \text{scale} \in \text{set} (\text{powers-scaled idx})$ 
    using  $k\text{-lt}$  unfolding  $\text{powers-scaled-def}$  by  $\text{auto}$ 
  have  $\text{idx-bound}$ :  $\text{idx} + k * \text{scale} < \text{clength} * \text{scale}$ 
    by ( $\text{rule scaled-bound}[OF \text{scaled-idx-in}]$ )
  have  $\text{left}$ :
     $\text{map} (! \text{trace-table}) (\text{powers-scaled idx}) ! k =$ 
       $\text{trace-table} ! (\text{idx} + k * \text{scale})$ 
    using  $k\text{-lt}$  unfolding  $\text{powers-scaled-def}$  by  $\text{simp}$ 
  have  $\text{right}$ :
     $\text{map} (\lambda q. \text{poly } q (h \wedge \text{idx} * \text{shift})) (\text{trace-powers-of } f) ! k =$ 
       $\text{poly} (\text{trace-powers-of } f ! k) (h \wedge \text{idx} * \text{shift})$ 
    using  $k\text{-lt}$  by  $\text{simp}$ 
  show
     $\text{map} (! \text{trace-table}) (\text{powers-scaled idx}) ! k =$ 
       $\text{map} (\lambda q. \text{poly } q (h \wedge \text{idx} * \text{shift})) (\text{trace-powers-of } f) ! k$ 
    using  $\text{query-trace-power-eval-nth}[OF k\text{-lt idx-bound}]$   $\text{left right trace-table}$ 
    by  $\text{simp}$ 
qed

lemma  $\text{query-consistent-at-trace-powers}$ :
  assumes  $qc$ :  $\text{query-consistent-at trace-table composition-table as idx}$ 
  and  $\text{trace-table}$ :  $\text{trace-table} = \text{map} (\text{poly } f) \text{eval-domain}$ 
  shows
     $\text{composition-table} ! \text{idx} =$ 
       $\text{cp-eval as} (\text{map} (\lambda q. \text{poly } q (h \wedge \text{idx} * \text{shift})) (\text{trace-powers-of } f))$ 
       $(h \wedge \text{idx} * \text{shift})$ 
proof –
  have  $qc\text{-unfolded}$ :
     $\text{idx} < \text{clength} * \text{scale} \wedge$ 
     $\text{length trace-table} = \text{clength} * \text{scale} \wedge$ 
     $\text{length composition-table} = \text{clength} * \text{scale} \wedge$ 
     $(\forall i \in \text{set} (\text{powers-scaled idx}). i < \text{length trace-table}) \wedge$ 
     $\text{composition-table} ! \text{idx} =$ 
       $\text{cp-eval as} (\text{map} (! \text{trace-table}) (\text{powers-scaled idx})) (h \wedge \text{idx} * \text{shift})$ 
    using  $qc$  unfolding  $\text{query-consistent-at-def}$  .
  have  $\text{trace-len}$ :  $\text{length trace-table} = \text{clength} * \text{scale}$ 
    using  $qc\text{-unfolded}$  by  $\text{blast}$ 

```

```

have scaled-bound:
   $\bigwedge i. i \in \text{set } (\text{powers-scaled } \text{idx}) \implies i < \text{clength} * \text{scale}$ 
proof -
  fix i
  assume i-in:  $i \in \text{set } (\text{powers-scaled } \text{idx})$ 
  have i < length trace-table
    using qc-unfolded i-in by blast
  then show i < clength * scale
    using trace-len by simp
qed
have vals-eq:
  map (! trace-table) (powers-scaled idx) =
    map ( $\lambda q. \text{poly } q (h \wedge \text{idx} * \text{shift})$ ) (trace-powers-of f)
  by (rule query-values-trace-powers[OF trace-table scaled-bound])
have comp:
  composition-table ! idx =
    cp-eval as (map (! trace-table) (powers-scaled idx)) ( $h \wedge \text{idx} * \text{shift}$ )
  using qc-unfolded by blast
have cp-eq:
  cp-eval as (map (! trace-table) (powers-scaled idx)) ( $h \wedge \text{idx} * \text{shift}$ ) =
    cp-eval as (map ( $\lambda q. \text{poly } q (h \wedge \text{idx} * \text{shift})$ ) (trace-powers-of f))
      ( $h \wedge \text{idx} * \text{shift}$ )
  using vals-eq by (rule arg-cong)
show ?thesis
  using comp cp-eq by simp
qed

lemma fold-add-sum-list:
  fixes f :: 'x  $\Rightarrow$  'b::comm-monoid-add
  shows fold ( $\lambda x \text{ acc}. f x + \text{acc}$ ) xs acc = sum-list (map f xs) + acc
proof (induction xs arbitrary: acc)
  case Nil
  then show ?case by simp
next
  case (Cons x xs)
  then show ?case
    by (simp add: ac-simps)
qed

lemma cp-eval-agrees-with-spec-lists:
  fixes specs :: ( $(f \text{ poly list} \Rightarrow f \text{ poly}) \times \text{nat list} \times \text{nat}$ ) list
  and spec2s :: ( $(f \text{ list} \Rightarrow f) \times \text{nat list}$ ) list
  and qs :: 'f poly list
  assumes len: length spec2s = length specs
  and zip-subset:  $\text{set } (\text{zip specs spec2s}) \subseteq \text{set } (\text{zip spec spec2})$ 
  and qs-len: powers  $\leq$  length qs
  shows
    fold
      ( $\lambda(a, (c2, \text{roots2})).$ 

```



```

      (+) (a * (c2 (map (λq. poly q x) qs) /
        poly (prod (g-map roots2)) x)))
    (zip as spec2s) vacc =
sum-list
  (map
    (λ(a, (c, roots, d)).
      a * (poly (c qs) x / poly (prod (g-map roots)) x))
    (zip as specs)) + vacc
proof -
  let ?vterm =
    λz. case z of (a, entry) ⇒
      case entry of (c2, roots2) ⇒
        a * (c2 (map (λq. poly q x) qs) /
          poly (prod (g-map roots2)) x)
  let ?sterm =
    λz. case z of (a, entry) ⇒
      case entry of (c, roots, d) ⇒
        a * (poly (c qs) x / poly (prod (g-map roots)) x)
  have terms-eq:
    map ?vterm (zip as spec2s) = map ?sterm (zip as specs)
proof (rule nth-equalityI)
  show length (map ?vterm (zip as spec2s)) =
    length (map ?sterm (zip as specs))
  using len by simp
next
  fix i
  assume i-bound: i < length (map ?vterm (zip as spec2s))
  then have i-as: i < length as
    by simp
  have i-spec2: i < length spec2s
    using i-bound by simp
  have i-specs: i < length specs
    using i-spec2 len by simp
  obtain c roots d where specs-i: specs ! i = (c, roots, d)
    by (cases specs ! i) auto
  obtain c2 roots2 where spec2s-i: spec2s ! i = (c2, roots2)
    by (cases spec2s ! i) auto
  have pair-in:
    ((c, roots, d), (c2, roots2)) ∈ set (zip spec spec2)
proof -
  have (specs ! i, spec2s ! i) ∈ set (zip specs spec2s)
  proof -
    have i < length (zip specs spec2s)
      using i-specs i-spec2 by simp
    then have zip specs spec2s ! i ∈ set (zip specs spec2s)
      by (rule nth-mem)
    then show ?thesis
      using i-specs i-spec2 by simp
qed

```

```

then show ?thesis
  using zip-subset specs-i spec2s-i by auto
qed
have roots-eq: roots2 = roots
  and raw-eq:
    c2 (map (λq. poly q x) qs) = poly (c qs) x
  using specs-agree[OF pair-in qs-len, of x] by simp-all
show map ?vterm (zip as spec2s) ! i =
  map ?sterm (zip as specs) ! i
  using i-as i-spec2 i-specs specs-i spec2s-i roots-eq raw-eq
  by simp
qed
have fold-eq:
  fold
    (λ(a, (c2, roots2)).
      (+) (a * (c2 (map (λq. poly q x) qs) /
        poly (prod (g-map roots2)) x)))
    (zip as spec2s) vacc =
  fold (λz acc. ?vterm z + acc) (zip as spec2s) vacc
  by (rule fold-cong) (auto simp: fun-eq-iff split: prod.splits)
have
  fold
    (λ(a, (c2, roots2)).
      (+) (a * (c2 (map (λq. poly q x) qs) /
        poly (prod (g-map roots2)) x)))
    (zip as spec2s) vacc =
  sum-list (map ?vterm (zip as spec2s)) + vacc
  unfolding fold-eq
  by (rule fold-add-sum-list)
also have ... =
  sum-list
    (map
      (λ(a, (c, roots, d)).
        a * (poly (c qs) x / poly (prod (g-map roots)) x))
      (zip as specs)) + vacc
  using terms-eq by simp
finally show ?thesis .
qed

```

lemma cp-eval-trace-powers-agrees-with-spec:
 cp-eval as (map (λq. poly q x) (trace-powers-of f)) x =
 composition-spec-eval f as x

proof –

have aligned:

```

  fold
    (λ(a, (c2, roots2)).
      (+) (a * (c2
        (map (λq. poly q x) (trace-powers-of f)) /
        poly (prod (g-map roots2)) x)))

```

```

  (zip as spec2) 0 =
sum-list
  (map
    (λ(a, (c, roots, d)).
      a * (poly (c (trace-powers-of f)) x /
        poly (prod (g-map roots)) x))
    (zip as spec)) + 0
by (rule cp-eval-agrees-with-spec-lists[OF spec2-length subset-refl])
simp
have spec-sum:
sum-list
  (map
    (λ(a, (c, roots, d)).
      a * (poly (c (trace-powers-of f)) x /
        poly (prod (g-map roots)) x))
    (zip as spec)) =
sum-list (map (composition-spec-eval-term f x) (zip as spec))
apply (intro arg-cong[where f=sum-list] map-cong refl)
apply (auto simp: composition-spec-eval-term-def split-beta'
  case-prod-unfold split: prod.splits)
done
show ?thesis
unfolding cp-eval-def composition-spec-eval-def
using aligned spec-sum by simp
qed

lemma common-denominator-mult-cp-eval-trace-powers:
assumes idx-bound: idx < clength * scale
shows
poly common-denominator-poly (h ^ idx * shift) *
cp-eval as
  (map (λq. poly q (h ^ idx * shift)) (trace-powers-of f))
  (h ^ idx * shift) =
poly (common-denominator-numerator f as) (h ^ idx * shift)
proof -
have denom:
  ∧ c roots d. (c, roots, d) ∈ set spec ⇒
  poly (prod (g-map roots)) (h ^ idx * shift) ≠ 0
proof -
fix c roots d
assume spec-entry: (c, roots, d) ∈ set spec
show poly (prod (g-map roots)) (h ^ idx * shift) ≠ 0
  by (rule query-denominator-nonzero[OF spec-entry idx-bound])
qed
have mult:
poly common-denominator-poly (h ^ idx * shift) *
  composition-spec-eval f as (h ^ idx * shift) =
poly (common-denominator-numerator f as) (h ^ idx * shift)
by (rule common-denominator-mult-composition-spec-eval[OF denom])

```

```

show ?thesis
  using mult cp-eval-trace-powers-agrees-with-spec[of as  $h \wedge \text{idx} * \text{shift } f$ ]
  by simp
qed

lemma common-denominator-numerator-identity-residual-zero:
  assumes len: length as = length spec
  and identity:
    common-denominator-numerator f as = common-denominator-poly * q
  and spec-entry: (c, roots, d) ∈ set spec
  and r-in: r ∈ set roots
  shows common-denominator-composition-root-residual f as r = 0
proof –
  have numerator-eval:
    poly (common-denominator-numerator f as) (g  $\wedge$  r) =
      common-denominator-composition-root-residual f as r
  by (rule common-denominator-numerator-root-eval[OF len spec-entry r-in])
  have root-zero: poly common-denominator-poly (g  $\wedge$  r) = 0
  by (rule common-denominator-vanishes-on-constraint-root
    [OF spec-entry r-in])
  have poly (common-denominator-numerator f as) (g  $\wedge$  r) = 0
  using identity root-zero by simp
  then show ?thesis
  using numerator-eval by simp
qed

```

```

lemma common-denominator-numerator-identity-hides-violations:
  assumes len: length as = length spec
  and violated: violated-constraints f ≠ {}
  and identity:
    common-denominator-numerator f as = common-denominator-poly * q
  shows random-combination-common-denominator-hides-violations f as
proof –
  have residual-zero:
     $\bigwedge c \text{ roots } d \text{ r.}$ 
    (c, roots, d) ∈ violated-constraints f  $\implies$ 
    r ∈ set roots  $\implies$ 
    common-denominator-composition-root-residual f as r = 0
  proof –
    fix c roots d r
    assume entry: (c, roots, d) ∈ violated-constraints f
    and r-in: r ∈ set roots
    have spec-entry: (c, roots, d) ∈ set spec
    using entry unfolding violated-constraints-def by simp
    show common-denominator-composition-root-residual f as r = 0
    by (rule common-denominator-numerator-identity-residual-zero
      [OF len identity spec-entry r-in])
  qed
show ?thesis

```

```

using violated residual-zero
unfolding random-combination-common-denominator-hides-violations-def
by auto
qed

lemma composition-table-agrees-with-rational-expression:
  assumes composition-low:
    composition-table-low-degree maxDegree composition-table
  and trace-table: trace-table = map (poly f) eval-domain
  and all-queries: all-queries-consistent trace-table composition-table as
obtains q where
  degree q ≤ maxDegree
and  $\bigwedge idx. idx \in \text{query-sample-space} \implies$ 
  poly q (h ^ idx * shift) =
    cp-eval as (map (λp. poly p (h ^ idx * shift)) (trace-powers-of f))
    (h ^ idx * shift)
proof –
  from composition-low obtain q where deg-q: degree q ≤ maxDegree
  and table-q: composition-table = map (poly q) eval-domain
  unfolding composition-table-low-degree-def by blast
  have agree:
     $\bigwedge idx. idx \in \text{query-sample-space} \implies$ 
    poly q (h ^ idx * shift) =
      cp-eval as (map (λp. poly p (h ^ idx * shift)) (trace-powers-of f))
      (h ^ idx * shift)
  proof –
    fix idx
    assume idx-in: idx ∈ query-sample-space
    have idx-bound: idx < clength * scale
      by (rule query-sample-space-less-domain[OF idx-in])
    have qc: query-consistent-at trace-table composition-table as idx
      using all-queries idx-in unfolding all-queries-consistent-def by simp
    have comp-value:
      composition-table ! idx = poly q (h ^ idx * shift)
      using table-q eval-domain-nth[OF idx-bound] idx-bound eval-domain-length
      by simp
    show poly q (h ^ idx * shift) =
      cp-eval as (map (λp. poly p (h ^ idx * shift)) (trace-powers-of f))
      (h ^ idx * shift)
      using query-consistent-at-trace-powers[OF qc trace-table] comp-value
      by simp
  qed
  show ?thesis
    by (rule that[OF deg-q agree])
qed

lemma common-denominator-numerator-identity-from-queries:
  assumes composition-low:
    composition-table-low-degree maxDegree composition-table

```

and *trace-table*: $\text{trace-table} = \text{map } (\text{poly } f) \text{ eval-domain}$
and *all-queries*: $\text{all-queries-consistent trace-table composition-table as}$
and *num-deg*:
 $\text{degree } (\text{common-denominator-numerator } f \text{ as}) < \text{query-sample-space-size}$
and *prod-deg*:
 $\bigwedge q. \text{degree } q \leq \text{maxDegree} \implies$
 $\text{degree } (\text{common-denominator-poly} * q) < \text{query-sample-space-size}$
obtains *q* **where**
 $\text{degree } q \leq \text{maxDegree}$
and $\text{common-denominator-numerator } f \text{ as} = \text{common-denominator-poly} * q$
proof –
from *composition-table-agrees-with-rational-expression*
 $[OF \text{ composition-low trace-table all-queries}]$
obtain *q* **where** *deg-q*: $\text{degree } q \leq \text{maxDegree}$
and *agree*:
 $\bigwedge \text{idx}. \text{idx} \in \text{query-sample-space} \implies$
 $\text{poly } q \text{ (} h \wedge \text{idx} * \text{shift} \text{)} =$
 cp-eval as
 $(\text{map } (\lambda p. \text{poly } p \text{ (} h \wedge \text{idx} * \text{shift} \text{)}) (\text{trace-powers-of } f))$
 $(h \wedge \text{idx} * \text{shift})$
by *blast*
have *eval-agree*:
 $\bigwedge \text{idx}. \text{idx} \in \text{query-sample-space} \implies$
 $\text{poly } (\text{common-denominator-numerator } f \text{ as}) \text{ (} h \wedge \text{idx} * \text{shift} \text{)} =$
 $\text{poly } (\text{common-denominator-poly} * q) \text{ (} h \wedge \text{idx} * \text{shift} \text{)}$
proof –
fix *idx*
assume *idx-in*: $\text{idx} \in \text{query-sample-space}$
have *idx-bound*: $\text{idx} < \text{clength} * \text{scale}$
by $(\text{rule query-sample-space-less-domain}[OF \text{ idx-in}])$
have *mult*:
 $\text{poly common-denominator-poly (} h \wedge \text{idx} * \text{shift} \text{)} *$
 cp-eval as
 $(\text{map } (\lambda p. \text{poly } p \text{ (} h \wedge \text{idx} * \text{shift} \text{)}) (\text{trace-powers-of } f))$
 $(h \wedge \text{idx} * \text{shift}) =$
 $\text{poly } (\text{common-denominator-numerator } f \text{ as}) \text{ (} h \wedge \text{idx} * \text{shift} \text{)}$
by $(\text{rule common-denominator-mult-cp-eval-trace-powers}[OF \text{ idx-bound}])$
have $\text{poly } (\text{common-denominator-poly} * q) \text{ (} h \wedge \text{idx} * \text{shift} \text{)} =$
 $\text{poly common-denominator-poly (} h \wedge \text{idx} * \text{shift} \text{)} *$
 $\text{poly } q \text{ (} h \wedge \text{idx} * \text{shift} \text{)}$
by *simp*
also have ... =
 $\text{poly common-denominator-poly (} h \wedge \text{idx} * \text{shift} \text{)} *$
 cp-eval as
 $(\text{map } (\lambda p. \text{poly } p \text{ (} h \wedge \text{idx} * \text{shift} \text{)}) (\text{trace-powers-of } f))$
 $(h \wedge \text{idx} * \text{shift})$
using *agree*[*OF idx-in*] **by** *simp*
also have ... = $\text{poly } (\text{common-denominator-numerator } f \text{ as}) \text{ (} h \wedge \text{idx} * \text{shift} \text{)}$
using *mult* **by** *simp*

finally show
 $\text{poly } (\text{common-denominator-numerator } f \text{ as}) (h \hat{=} \text{idx} * \text{shift}) =$
 $\text{poly } (\text{common-denominator-poly} * q) (h \hat{=} \text{idx} * \text{shift})$
by *simp*
qed
have *identity*:
 $\text{common-denominator-numerator } f \text{ as} = \text{common-denominator-poly} * q$
by (rule *poly-eq-on-query-sample-domainI*[*OF num-deg prod-deg*[*OF deg-q*]])
(rule *eval-agree*)
show *?thesis*
by (rule *that*[*OF deg-q identity*])
qed

lemma *common-denominator-numerator-identity-from-queries-bounds*:
assumes *composition-low*:
 $\text{composition-table-low-degree } \text{maxDegree } \text{composition-table}$
and *trace-table*: $\text{trace-table} = \text{map } (\text{poly } f) \text{ eval-domain}$
and *all-queries*: $\text{all-queries-consistent } \text{trace-table } \text{composition-table } \text{as}$
and *bounds*: $\text{common-denominator-degree-bounds } f \text{ as}$
obtains *q* **where**
 $\text{degree } q \leq \text{maxDegree}$
and $\text{common-denominator-numerator } f \text{ as} = \text{common-denominator-poly} * q$
proof –
have *num-deg*:
 $\text{degree } (\text{common-denominator-numerator } f \text{ as}) < \text{query-sample-space-size}$
using *bounds unfolding common-denominator-degree-bounds-def* **by** *simp*
have *prod-deg*:
 $\bigwedge q. \text{degree } q \leq \text{maxDegree} \implies$
 $\text{degree } (\text{common-denominator-poly} * q) < \text{query-sample-space-size}$
using *bounds unfolding common-denominator-degree-bounds-def* **by** *simp*
from *common-denominator-numerator-identity-from-queries*
[*OF composition-low trace-table all-queries num-deg prod-deg*]
obtain *q* **where** *deg-q*: $\text{degree } q \leq \text{maxDegree}$
and *identity*:
 $\text{common-denominator-numerator } f \text{ as} = \text{common-denominator-poly} * q$
by *blast*
show *?thesis*
by (rule *that*[*OF deg-q identity*])
qed

lemma *random-combination-common-denominator-hides-from-queries*:
assumes *len-as*: $\text{length } \text{as} = \text{length } \text{spec}$
and *false-statement*: $\neg \text{exists-valid-trace}$
and *deg-f*: $\text{degree } f < \text{clength}$
and *trace-table*: $\text{trace-table} = \text{map } (\text{poly } f) \text{ eval-domain}$
and *composition-low*:
 $\text{composition-table-low-degree } \text{maxDegree } \text{composition-table}$
and *all-queries*: $\text{all-queries-consistent } \text{trace-table } \text{composition-table } \text{as}$
and *bounds*: $\text{common-denominator-degree-bounds } f \text{ as}$

shows *random-combination-common-denominator-hides-violations* f as
proof –
have *violated*: *violated-constraints* $f \neq \{\}$
by (*rule false-statement-violated-constraints* [*OF false-statement deg-f*])
from *common-denominator-numerator-identity-from-queries-bounds*
[*OF composition-low trace-table all-queries bounds*]
obtain q **where**
degree $q \leq \text{maxDegree}$
and *identity*:
common-denominator-numerator f as = *common-denominator-poly* * q
by *blast*
show *?thesis*
by (*rule common-denominator-numerator-identity-hides-violations*
[*OF len-as violated identity*])
qed

lemma *all-queries-consistent-iff-no-bad-queries*:
all-queries-consistent trace-table composition-table as $\longleftrightarrow$
bad-query-indices trace-table composition-table as = $\{\}$
unfolding *all-queries-consistent-def bad-query-indices-def* **by** *auto*

lemma *query-disagreement-indices-nonempty-iff*:
query-disagreement-indices trace-table composition-table as $\neq \{\}$ $\longleftrightarrow$
 $\neg$ *all-queries-consistent trace-table composition-table* as
unfolding *query-disagreement-indices-def*
using *all-queries-consistent-iff-no-bad-queries* **by** *simp*

lemma *query-consistent-at-from-common-denominator-residual-zero*:
assumes *trace-table*: *trace-table* = *map* (*poly f*) *eval-domain*
and *composition-table*: *composition-table* = *map* (*poly q*) *eval-domain*
and *residual-zero*:
common-denominator-numerator f as – *common-denominator-poly* * q = 0
and *idx-in*: $\text{idx} \in \text{query-sample-space}$
shows *query-consistent-at trace-table composition-table* as idx
proof –
have *idx-bound*: $\text{idx} < \text{clength} * \text{scale}$
by (*rule query-sample-space-less-domain* [*OF idx-in*])
have *trace-len*: $\text{length trace-table} = \text{clength} * \text{scale}$
using *trace-table eval-domain-length* **by** *simp*
have *comp-len*: $\text{length composition-table} = \text{clength} * \text{scale}$
using *composition-table eval-domain-length* **by** *simp*
have *powers-bound*:
 $\forall i \in \text{set (powers-scaled idx)}. i < \text{length trace-table}$
proof
fix i
assume $i\text{-in}$: $i \in \text{set (powers-scaled idx)}$
show $i < \text{length trace-table}$
using *query-sample-space-powers-scaled-bound* [*OF idx-in i-in*] *trace-len*
by *simp*


```

qed
have comp-value:
  composition-table ! idx = poly q (h ^ idx * shift)
  using composition-table eval-domain-nth[OF idx-bound] idx-bound eval-domain-length
  by simp
have trace-powers:
  map (! trace-table) (powers-scaled idx) =
    map ( $\lambda p$ . poly p (h ^ idx * shift)) (trace-powers-of f)
  by (rule query-values-trace-powers[OF trace-table])
    (rule query-sample-space-powers-scaled-bound[OF idx-in])
have mult:
  poly common-denominator-poly (h ^ idx * shift) *
    cp-eval as
      (map ( $\lambda p$ . poly p (h ^ idx * shift)) (trace-powers-of f))
      (h ^ idx * shift) =
    poly (common-denominator-numerator f as) (h ^ idx * shift)
  by (rule common-denominator-mult-cp-eval-trace-powers[OF idx-bound])
have denom-nonzero:
  poly common-denominator-poly (h ^ idx * shift)  $\neq$  0
  by (rule common-denominator-nonzero-on-eval-domain[OF idx-bound])
have residual-eval:
  poly (common-denominator-numerator f as) (h ^ idx * shift) =
    poly common-denominator-poly (h ^ idx * shift) *
      poly q (h ^ idx * shift)
  using residual-zero by simp
have cp-eq:
  cp-eval as
    (map ( $\lambda p$ . poly p (h ^ idx * shift)) (trace-powers-of f))
    (h ^ idx * shift) =
    poly q (h ^ idx * shift)
  using mult residual-eval denom-nonzero by simp
show ?thesis
  unfolding query-consistent-at-def
  using idx-bound trace-len comp-len powers-bound comp-value trace-powers cp-eq
  by simp
qed

lemma all-queries-consistent-from-common-denominator-residual-zero:
  assumes trace-table: trace-table = map (poly f) eval-domain
    and composition-table: composition-table = map (poly q) eval-domain
    and residual-zero:
      common-denominator-numerator f as - common-denominator-poly * q = 0
  shows all-queries-consistent trace-table composition-table as
  unfolding all-queries-consistent-def
  using query-consistent-at-from-common-denominator-residual-zero
    [OF trace-table composition-table residual-zero]
  by blast

lemma common-denominator-residual-nonzero-from-query-disagreement:

```

assumes *trace-table*: $\text{trace-table} = \text{map } (\text{poly } f) \text{ eval-domain}$
and *composition-table*: $\text{composition-table} = \text{map } (\text{poly } q) \text{ eval-domain}$
and *not-all*: $\neg \text{all-queries-consistent trace-table composition-table as}$
shows *common-denominator-numerator f as* $-\text{common-denominator-poly} * q \neq 0$
proof
assume *zero*:
 $\text{common-denominator-numerator } f \text{ as } -\text{common-denominator-poly} * q = 0$
have *all-queries-consistent trace-table composition-table as*
by (*rule all-queries-consistent-from-common-denominator-residual-zero*
 $[\text{OF trace-table composition-table zero}]$)
then show *False*
using *not-all* **by** *simp*
qed

lemma *query-agreement-indices-card-bound*:
assumes *trace-low*: $\text{trace-table-low-degree trace-table}$
and *composition-low*:
 $\text{composition-table-low-degree maxDegree composition-table}$
and *not-all*:
 $\neg \text{all-queries-consistent trace-table composition-table as}$
shows *card (query-agreement-indices trace-table composition-table as)* $\leq$
 $\text{query-agreement-bound}$
proof $-$
from *trace-low* **obtain** *f* **where** *deg-f*: $\text{degree } f < \text{clength}$
and *trace-table*: $\text{trace-table} = \text{map } (\text{poly } f) \text{ eval-domain}$
unfolding *trace-table-low-degree-def* **by** *blast*
from *composition-low* **obtain** *q* **where** *deg-q*: $\text{degree } q \leq \text{maxDegree}$
and *composition-table*: $\text{composition-table} = \text{map } (\text{poly } q) \text{ eval-domain}$
unfolding *composition-table-low-degree-def* **by** *blast*
let *?A* $= \text{query-agreement-indices trace-table composition-table as}$
let *?R* $= \text{common-denominator-numerator } f \text{ as } -\text{common-denominator-poly} * q$
have *A-subset*: $?A \subseteq \text{query-sample-space}$
unfolding *query-agreement-indices-def* **by** *auto*
have *finite-A*: *finite* *?A*
unfolding *query-agreement-indices-def query-sample-space-def* **by** *simp*
have *inj*: *inj-on* $(\lambda \text{idx. } h \wedge \text{idx} * \text{shift}) ?A$
proof (*intro inj-onI*)
fix *i j*
assume *i-in*: $i \in ?A$
and *j-in*: $j \in ?A$
and *eq*: $h \wedge i * \text{shift} = h \wedge j * \text{shift}$
have *i-bound*: $i < \text{clength} * \text{scale}$
by (*rule query-sample-space-less-domain* $[\text{OF set-mp} [\text{OF } A\text{-subset } i\text{-in}]]$)
have *j-bound*: $j < \text{clength} * \text{scale}$
by (*rule query-sample-space-less-domain* $[\text{OF set-mp} [\text{OF } A\text{-subset } j\text{-in}]]$)
have $h \wedge i = h \wedge j$
using *eq shift-nonzero* **by** *simp*
then show $i = j$
by (*rule h-power-inj-on-eval-domain* $[\text{OF } i\text{-bound } j\text{-bound}]$)

```

qed
have residual-nonzero:
  ?R ≠ 0
  by (rule common-denominator-residual-nonzero-from-query-disagreement
    [OF trace-table composition-table not-all])
have image-subset:
  (λidx. h ^ idx * shift) ‘ ?A ⊆ {x. poly ?R x = 0}
proof
  fix x
  assume x-in: x ∈ (λidx. h ^ idx * shift) ‘ ?A
  then obtain idx where idx-in: idx ∈ ?A
    and x-eq: x = h ^ idx * shift
    by blast
  have idx-sample: idx ∈ query-sample-space
    by (rule set-mp[OF A-subset idx-in])
  have idx-bound: idx < clength * scale
    by (rule query-sample-space-less-domain[OF idx-sample])
  have qc: query-consistent-at trace-table composition-table as idx
    using idx-in unfolding query-agreement-indices-def by simp
  have comp-value:
    composition-table ! idx = poly q (h ^ idx * shift)
    using composition-table eval-domain-nth[OF idx-bound] idx-bound
      eval-domain-length by simp
  have trace-powers:
    map (! trace-table) (powers-scaled idx) =
    map (λp. poly p (h ^ idx * shift)) (trace-powers-of f)
    by (rule query-values-trace-powers[OF trace-table])
      (rule query-sample-space-powers-scaled-bound[OF idx-sample])
  have mult:
    poly common-denominator-poly (h ^ idx * shift) *
    cp-eval as
    (map (λp. poly p (h ^ idx * shift)) (trace-powers-of f))
    (h ^ idx * shift) =
    poly (common-denominator-numerator f as) (h ^ idx * shift)
    by (rule common-denominator-mult-cp-eval-trace-powers[OF idx-bound])
  have cp-eq:
    cp-eval as
    (map (λp. poly p (h ^ idx * shift)) (trace-powers-of f))
    (h ^ idx * shift) =
    poly q (h ^ idx * shift)
    using qc comp-value trace-powers
    unfolding query-consistent-at-def by simp
  have poly (common-denominator-numerator f as) (h ^ idx * shift) =
    poly common-denominator-poly (h ^ idx * shift) *
    poly q (h ^ idx * shift)
    using mult cp-eq by simp
  then have poly ?R (h ^ idx * shift) = 0
    by simp
  then show x ∈ {x. poly ?R x = 0}

```

```

    unfolding x-eq by simp
  qed
  have card ?A = card ((λidx. h ^ idx * shift) ‘ ?A)
    using card-image[OF inj] by simp
  also have  $\dots \leq \text{card } \{x. \text{poly } ?R \ x = 0\}$ 
    by (rule card-mono[OF poly-roots-finite[OF residual-nonzero] image-subset])
  also have  $\dots \leq \text{degree } ?R$ 
    by (rule card-poly-roots-bound[OF residual-nonzero])
  also have  $\dots \leq \text{query-agreement-bound}$ 
    by (rule common-denominator-residual-degree-le-bound[OF deg-f deg-q])
  finally show ?thesis .
qed

end

end

```

```

theory Soundness-Core-WP
  imports Soundness-Core-Base
begin

```

Generic probability and WP combinators used by soundness reductions.

```

context soundness
begin

```

```

lemma wp-event-union-bound:
  fixes P Q :: ('x × 's) option ⇒ bool
  shows
     $\text{wp-event } m \ (\lambda \text{out. } P \ \text{out} \vee Q \ \text{out}) \ s \leq$ 
     $\text{wp-event } m \ P \ s + \text{wp-event } m \ Q \ s$ 
proof –
  have pointwise:
     $\bigwedge \text{out } p. p * (\text{if } P \ \text{out} \vee Q \ \text{out} \text{ then } 1 \text{ else } 0) \leq$ 
     $p * ((\text{if } P \ \text{out} \text{ then } 1 \text{ else } 0) + (\text{if } Q \ \text{out} \text{ then } 1 \text{ else } 0)) :: \text{prob}$ 
    by (simp add: mult-left-mono)
  have  $\text{wp-event } m \ (\lambda \text{out. } P \ \text{out} \vee Q \ \text{out}) \ s \leq$ 
     $\text{wp } m \ (\lambda \text{out. } (\text{if } P \ \text{out} \text{ then } 1 \text{ else } 0) + (\text{if } Q \ \text{out} \text{ then } 1 \text{ else } 0)) \ s$ 
    unfolding wp-event-def wp-def dist-expect-def
    by (intro sum-mono pointwise)
  also have  $\dots = \text{wp-event } m \ P \ s + \text{wp-event } m \ Q \ s$ 
    unfolding wp-event-def wp-def dist-expect-def
    by (simp add: algebra-simps sum.distrib)
  finally show ?thesis .
qed

```

```

lemma wp-event-union-bound4:
  fixes A B C D :: ('x × 's) option ⇒ bool
  shows

```

$wp\text{-event } m (\lambda out. A \text{ out} \vee B \text{ out} \vee C \text{ out} \vee D \text{ out}) s \leq$
 $wp\text{-event } m A s + wp\text{-event } m B s + wp\text{-event } m C s + wp\text{-event } m D s$
proof –
have *pointwise*:
 $\bigwedge out p. p * (if A \text{ out} \vee B \text{ out} \vee C \text{ out} \vee D \text{ out} \text{ then } 1 \text{ else } 0) \leq$
 $p * ((if A \text{ out} \text{ then } 1 \text{ else } 0) + (if B \text{ out} \text{ then } 1 \text{ else } 0) +$
 $(if C \text{ out} \text{ then } 1 \text{ else } 0) + (if D \text{ out} \text{ then } 1 \text{ else } 0) :: prob)$
by (*simp add: mult-left-mono*)
have $wp\text{-event } m (\lambda out. A \text{ out} \vee B \text{ out} \vee C \text{ out} \vee D \text{ out}) s \leq$
 $wp m (\lambda out.$
 $(if A \text{ out} \text{ then } 1 \text{ else } 0) + (if B \text{ out} \text{ then } 1 \text{ else } 0) +$
 $(if C \text{ out} \text{ then } 1 \text{ else } 0) + (if D \text{ out} \text{ then } 1 \text{ else } 0)) s$
unfolding *wp-event-def wp-def dist-expect-def*
by (*intro sum-mono pointwise*)
also have ... =
 $wp\text{-event } m A s + wp\text{-event } m B s + wp\text{-event } m C s + wp\text{-event } m D s$
unfolding *wp-event-def wp-def dist-expect-def*
by (*simp add: algebra-simps sum.distrib*)
finally show *?thesis*
qed

lemma *wp-event-finite-UN-bound*:
fixes $P :: 'a \Rightarrow ('x \times 's) \text{ option} \Rightarrow bool$
and $C :: 'a \Rightarrow prob$
assumes *finite-A: finite A*
and *bounds: $\bigwedge a. a \in A \implies wp\text{-event } m (P a) s \leq C a$*
shows
 $wp\text{-event } m (\lambda out. \exists a \in A. P a \text{ out}) s \leq (\sum a \in A. C a)$
proof –
have *pointwise*:
 $\bigwedge out p. p * (if \exists a \in A. P a \text{ out} \text{ then } 1 \text{ else } 0) \leq$
 $p * (\sum a \in A. if P a \text{ out} \text{ then } 1 \text{ else } 0 :: prob)$
proof –
fix *out p*
have *indicator-le*:
 $(if \exists a \in A. P a \text{ out} \text{ then } 1 \text{ else } 0 :: prob) \leq$
 $(\sum a \in A. if P a \text{ out} \text{ then } 1 \text{ else } 0)$
proof (*cases $\exists a \in A. P a \text{ out}$*)
case *True*
then obtain *a where* $a: a \in A \ P a \text{ out}$
by *blast*
have $1 = (if P a \text{ out} \text{ then } 1 \text{ else } 0 :: prob)$
using *a* **by** *simp*
also have ... $\leq (\sum a \in A. if P a \text{ out} \text{ then } 1 \text{ else } 0)$
by (*rule member-le-sum*) (*use a finite-A in simp-all*)
finally show *?thesis*
using *True* **by** *simp*
next

```

    case False
    then show ?thesis by simp
  qed
  show  $p * (\text{if } \exists a \in A. P \ a \ \text{out then } 1 \ \text{else } 0) \leq$ 
     $p * (\sum_{a \in A}. \text{if } P \ a \ \text{out then } 1 \ \text{else } 0 :: \text{prob})$ 
    by (simp add: mult-left-mono[OF indicator-le])
  qed
  have wp-event m ( $\lambda \text{out}. \exists a \in A. P \ a \ \text{out}$ ) s  $\leq$ 
    wp m ( $\lambda \text{out}. (\sum_{a \in A}. \text{if } P \ a \ \text{out then } 1 \ \text{else } 0)$ ) s
    unfolding wp-event-def wp-def dist-expect-def
    by (intro sum-mono pointwise)
  also have ... =  $(\sum_{a \in A}. \text{wp-event } m \ (P \ a) \ s)$ 
    using finite-A
  proof (induction A)
    case empty
    then show ?case
      unfolding wp-event-def wp-def dist-expect-def by simp
  next
    case (insert a A)
    have wp m ( $\lambda \text{out}. \sum_{x \in \text{insert } a \ A}. \text{if } P \ x \ \text{out then } 1 \ \text{else } 0$ ) s =
      wp m ( $\lambda \text{out}. (\text{if } P \ a \ \text{out then } 1 \ \text{else } 0) + (\sum_{x \in A}. \text{if } P \ x \ \text{out then } 1 \ \text{else } 0)$ ) s
      using insert.hyps by simp
    also have ... =
      wp-event m (P a) s +
      wp m ( $\lambda \text{out}. \sum_{x \in A}. \text{if } P \ x \ \text{out then } 1 \ \text{else } 0$ ) s
      unfolding wp-event-def wp-def dist-expect-def
      by (simp add: sum.distrib algebra-simps)
    also have ... = wp-event m (P a) s +  $(\sum_{x \in A}. \text{wp-event } m \ (P \ x) \ s)$ 
      using insert.IH by simp
    also have ... =  $(\sum_{x \in \text{insert } a \ A}. \text{wp-event } m \ (P \ x) \ s)$ 
      using insert.hyps by simp
    finally show ?case .
  qed
  also have ...  $\leq (\sum_{a \in A}. C \ a)$ 
    by (rule sum-mono) (rule bounds, assumption)
  finally show ?thesis .
qed

lemma wp-event-mono:
  fixes P Q :: ('x  $\times$  's) option  $\Rightarrow$  bool
  assumes  $\bigwedge \text{out}. P \ \text{out} \Longrightarrow Q \ \text{out}$ 
  shows wp-event m P s  $\leq$  wp-event m Q s
proof -
  have pointwise:
     $\bigwedge \text{out } p. p * (\text{if } P \ \text{out then } 1 \ \text{else } 0) \leq$ 
       $p * (\text{if } Q \ \text{out then } 1 \ \text{else } 0 :: \text{prob})$ 
    using assms by (simp add: mult-left-mono)

```

```

show ?thesis
  unfolding wp-event-def wp-def dist-expect-def
  by (intro sum-mono pointwise)
qed

lemma wp-event-le-1:
  fixes P :: ('x × 's) option ⇒ bool
  shows wp-event m P s ≤ 1
proof -
  have pointwise:
     $\bigwedge out p. p * (if P out then 1 else 0 :: prob) \leq p$ 
  by (simp add: mult-left-le)
  have wp-event m P s ≤
     $(\sum out \in dom (dist (execute m s)).$ 
     $the (dist (execute m s) out))$ 
  unfolding wp-event-def wp-def dist-expect-def
  by (intro sum-mono pointwise)
  also have ... = 1
  unfolding sum-map-def[symmetric] by simp
  finally show ?thesis .
qed

lemma wp-le-const-on-support:
  assumes  $\bigwedge out. out \in set-dist (execute m s) \implies Q out \leq C$ 
  shows wp m Q s ≤ C
proof -
  have pointwise:
     $\bigwedge out p. out \in dom (dist (execute m s)) \implies$ 
     $p * Q out \leq p * C$ 
  using assms unfolding set-dist-def by (simp add: mult-left-mono)
  have wp m Q s ≤
     $(\sum out \in dom (dist (execute m s)).$ 
     $the (dist (execute m s) out) * C)$ 
  unfolding wp-def dist-expect-def
  by (intro sum-mono pointwise)
  also have ... =
     $(\sum out \in dom (dist (execute m s)).$ 
     $the (dist (execute m s) out)) * C$ 
  by (simp add: sum-distrib-right)
  also have ... = C
  unfolding sum-map-def[symmetric] by simp
  finally show ?thesis .
qed

lemma wp-mono-on-support:
  assumes  $\bigwedge out. out \in set-dist (execute m s) \implies Q out \leq R out$ 
  shows wp m Q s ≤ wp m R s
proof -
  have pointwise:

```

```

 $\bigwedge out\ p. out \in dom\ (dist\ (execute\ m\ s)) \implies$ 
 $p * Q\ out \leq p * R\ out$ 
using assms unfolding set-dist-def by (simp add: mult-left-mono)
show ?thesis
unfolding wp-def dist-expect-def
by (intro sum-mono pointwise) simp
qed

```

```

lemma wp-event-mono-on-support:
  fixes  $P\ Q :: ('x \times 's)\ option \Rightarrow bool$ 
  assumes  $\bigwedge out. out \in set-dist\ (execute\ m\ s) \implies P\ out \implies Q\ out$ 
  shows  $wp-event\ m\ P\ s \leq wp-event\ m\ Q\ s$ 

```

```

proof -
  have pointwise:
     $\bigwedge out\ p. out \in dom\ (dist\ (execute\ m\ s)) \implies$ 
     $p * (if\ P\ out\ then\ 1\ else\ 0) \leq$ 
     $p * (if\ Q\ out\ then\ 1\ else\ 0 :: prob)$ 
    using assms unfolding set-dist-def by (simp add: mult-left-mono)
  show ?thesis
  unfolding wp-event-def wp-def dist-expect-def
  by (intro sum-mono pointwise) simp
qed

```

```

lemma wp-event-pos-of-support:
  assumes  $out: out \in set-dist\ (execute\ m\ s)$ 
  and event:  $P\ out$ 
  shows  $0 < wp-event\ m\ P\ s$ 
proof -
  let  $?D = dom\ (dist\ (execute\ m\ s))$ 
  have out-dom:  $out \in ?D$ 
  using out unfolding set-dist-def .
  have out-weight-pos:  $0 < the\ (dist\ (execute\ m\ s)\ out)$ 
  using dist-nonzero-on-dom[OF out-dom] by simp
  have  $0 < (\sum_{x \in ?D} the\ (dist\ (execute\ m\ s)\ x) * (if\ P\ x\ then\ 1\ else\ 0 :: prob))$ 
proof (rule sum-pos2)
  show finite ?D
  by simp
  show  $out \in ?D$ 
  by (rule out-dom)
  show  $0 < the\ (dist\ (execute\ m\ s)\ out) * (if\ P\ out\ then\ 1\ else\ 0 :: prob)$ 
  using event out-weight-pos by simp
qed simp
then show ?thesis
unfolding wp-event-def wp-def dist-expect-def .
qed

```

```

lemma wp-event-pos-imp-exists-support:

```



```

assumes  $0 < \text{wp-event } m \ P \ s$ 
obtains  $\text{out}$  where  $\text{out} \in \text{set-dist } (\text{execute } m \ s)$  and  $P \ \text{out}$ 
proof –
  have  $\exists \text{out} \in \text{set-dist } (\text{execute } m \ s). \ P \ \text{out}$ 
  proof (rule ccontr)
    assume  $\neg (\exists \text{out} \in \text{set-dist } (\text{execute } m \ s). \ P \ \text{out})$ 
    then have no-event:
       $\bigwedge \text{out}. \text{out} \in \text{dom } (\text{dist } (\text{execute } m \ s)) \implies \neg P \ \text{out}$ 
    unfolding set-dist-def by blast
    have  $\text{wp-event } m \ P \ s = 0$ 
    unfolding wp-event-def wp-def dist-expect-def
    by (intro sum.neutral) (simp add: no-event)
    then show False
    using assms by simp
  qed
  then show ?thesis
  using that by blast
qed

```

```

lemma wp-event-bind-bound-by-head-event:
  fixes  $m :: ('x, 's) \text{state-monad}$ 
  and  $k :: 'x \Rightarrow ('y, 's) \text{state-monad}$ 
  assumes  $m\text{-bound}: \text{wp-event } m \ P \ s \leq C$ 
  and  $\text{none-imp}: Q \ \text{None} \implies P \ \text{None}$ 
  and  $\text{cont-imp}$ :
     $\bigwedge x \ t \ \text{out}. \text{Some } (x, t) \in \text{set-dist } (\text{execute } m \ s) \implies$ 
     $\text{out} \in \text{set-dist } (\text{execute } (k \ x) \ t) \implies$ 
     $Q \ \text{out} \implies P \ (\text{Some } (x, t))$ 
  shows  $\text{wp-event } (m \gg k) \ Q \ s \leq C$ 
proof –
  have bind-eq:
     $\text{wp-event } (m \gg k) \ Q \ s =$ 
     $\text{wp } m \ (\lambda \text{out}. \text{case out of}$ 
     $\text{None} \Rightarrow \text{if } Q \ \text{None} \text{ then } 1 \text{ else } 0$ 
     $\mid \text{Some } (x, t) \Rightarrow \text{wp-event } (k \ x) \ Q \ t) \ s$ 
  unfolding wp-event-def by (simp add: wpsimps)
  have bind-head:
     $\text{wp-event } (m \gg k) \ Q \ s \leq \text{wp-event } m \ P \ s$ 
proof –
  have  $\text{wp-event } (m \gg k) \ Q \ s =$ 
     $\text{wp } m \ (\lambda \text{out}. \text{case out of}$ 
     $\text{None} \Rightarrow \text{if } Q \ \text{None} \text{ then } 1 \text{ else } 0$ 
     $\mid \text{Some } (x, t) \Rightarrow \text{wp-event } (k \ x) \ Q \ t) \ s$ 
  by (rule bind-eq)
  also have  $\dots \leq \text{wp-event } m \ P \ s$ 
  unfolding wp-event-def
proof (rule wp-mono-on-support)
  fix  $\text{out}$ 
  assume  $\text{out}: \text{out} \in \text{set-dist } (\text{execute } m \ s)$ 

```

```

show
  (case out of None  $\Rightarrow$  if  $Q$  None then 1 else 0
   | Some  $(x, t) \Rightarrow$ 
     wp  $(k\ x)$  ( $\lambda out.$  if  $Q$  out then 1 else 0)  $t$ )
   $\leq$  (if  $P$  out then 1 else 0)
proof (cases out)
  case None
  then show ?thesis
    using none-imp by (cases  $Q$  None) simp-all
next
  case (Some  $xt$ )
  then obtain  $x\ t$  where  $xt: xt = (x, t)$ 
    by (cases  $xt$ ) auto
  have out-eq: out = Some  $(x, t)$ 
    using Some  $xt$  by simp
  have head-out: Some  $(x, t) \in \text{set-dist (execute } m\ s)$ 
    using out out-eq by simp
  have wp-bound:
    wp  $(k\ x)$  ( $\lambda out.$  if  $Q$  out then 1 else 0)  $t \leq$ 
      (if  $P$  (Some  $(x, t)$ ) then 1 else 0 :: prob)
  proof (rule wp-le-const-on-support)
    fix out'
    assume out': out'  $\in \text{set-dist (execute (} k\ x)\ t)$ 
    show (if  $Q$  out' then 1 else 0)  $\leq$ 
      (if  $P$  (Some  $(x, t)$ ) then 1 else 0 :: prob)
    proof (cases  $Q$  out')
      case True
      then have  $P$  (Some  $(x, t)$ )
        by (rule cont-imp[OF head-out out'])
      then show ?thesis
        using True by simp
    next
      case False
      then show ?thesis by simp
    qed
  qed
  show ?thesis
    using wp-bound out-eq by simp
  qed
qed

```

```

lemma wp-event-bind-cong-cont:
  fixes  $m :: ('x, 's)$  state-monad
  and  $k :: 'x \Rightarrow ('y, 's)$  state-monad

```

and $l :: 'x \Rightarrow ('z, 's) \text{ state-monad}$
assumes $\text{none}: Q \text{ None} = R \text{ None}$
and cont :
 $\bigwedge x t. \text{wp-event } (k x) Q t = \text{wp-event } (l x) R t$
shows $\text{wp-event } (m \gg= k) Q s = \text{wp-event } (m \gg= l) R s$
proof –
have left :
 $\text{wp-event } (m \gg= k) Q s =$
 $\text{wp } m (\lambda \text{out}. \text{case out of}$
 $\quad \text{None} \Rightarrow \text{if } Q \text{ None then } 1 \text{ else } 0$
 $\quad | \text{Some } (x, t) \Rightarrow \text{wp-event } (k x) Q t) s$
unfolding wp-event-def **by** $(\text{simp add: wpsimps})$
have right :
 $\text{wp-event } (m \gg= l) R s =$
 $\text{wp } m (\lambda \text{out}. \text{case out of}$
 $\quad \text{None} \Rightarrow \text{if } R \text{ None then } 1 \text{ else } 0$
 $\quad | \text{Some } (x, t) \Rightarrow \text{wp-event } (l x) R t) s$
unfolding wp-event-def **by** $(\text{simp add: wpsimps})$
have fun-eq :
 $(\lambda \text{out}. \text{case out of}$
 $\quad \text{None} \Rightarrow \text{if } Q \text{ None then } 1 \text{ else } 0$
 $\quad | \text{Some } (x, t) \Rightarrow \text{wp-event } (k x) Q t) =$
 $(\lambda \text{out}. \text{case out of}$
 $\quad \text{None} \Rightarrow \text{if } R \text{ None then } 1 \text{ else } 0$
 $\quad | \text{Some } (x, t) \Rightarrow \text{wp-event } (l x) R t)$
by $(\text{rule ext}) (\text{simp add: none cont split: option.splits prod.splits})$
show $?thesis$
unfolding left right
by $(\text{simp add: fun-eq})$
qed

lemma $\text{wp-event-bind-bound-by-head-and-cont}$:
fixes $m :: ('x, 's) \text{ state-monad}$
and $k :: 'x \Rightarrow ('y, 's) \text{ state-monad}$
assumes $\text{none}: \neg Q \text{ None}$
and cont-zero :
 $\bigwedge x t. \text{Some } (x, t) \in \text{set-dist } (\text{execute } m s) \Longrightarrow$
 $\neg P (\text{Some } (x, t)) \Longrightarrow \text{wp-event } (k x) Q t = 0$
and cont-bound :
 $\bigwedge x t. \text{Some } (x, t) \in \text{set-dist } (\text{execute } m s) \Longrightarrow$
 $P (\text{Some } (x, t)) \Longrightarrow \text{wp-event } (k x) Q t \leq D$
shows $\text{wp-event } (m \gg= k) Q s \leq \text{wp-event } m P s * D$
proof –
have bind-eq :
 $\text{wp-event } (m \gg= k) Q s =$
 $\text{wp } m (\lambda \text{out}. \text{case out of}$
 $\quad \text{None} \Rightarrow \text{if } Q \text{ None then } 1 \text{ else } 0$
 $\quad | \text{Some } (x, t) \Rightarrow \text{wp-event } (k x) Q t) s$
unfolding wp-event-def **by** $(\text{simp add: wpsimps})$

```

have wp-event (m  $\gg$  k) Q s ≤
  wp m (λout. (if P out then 1 else 0) * D) s
unfolding bind-eq
proof (rule wp-mono-on-support)
  fix out
  assume out: out ∈ set-dist (execute m s)
  show (case out of None ⇒ if Q None then 1 else 0
    | Some (x, t) ⇒ wp-event (k x) Q t)
    ≤ (if P out then 1 else 0) * D
  proof (cases out)
    case None
    then show ?thesis
    using none by simp
  next
    case (Some xt)
    then obtain x t where xt: xt = (x, t)
    by (cases xt) auto
    have head: Some (x, t) ∈ set-dist (execute m s)
    using out Some xt by simp
    show ?thesis
    proof (cases P (Some (x, t)))
      case True
      then show ?thesis
      using cont-bound[OF head True] Some xt by simp
    next
      case False
      then show ?thesis
      using cont-zero[OF head False] Some xt by simp
    qed
  qed
qed
also have ... = D * wp-event m P s
  unfolding wp-event-def wp-def dist-expect-def
  by (simp add: sum-distrib-left algebra-simps)
also have ... = wp-event m P s * D
  by (simp add: mult.commute)
finally show ?thesis .
qed

```

```

lemma wp-event-bind-bound-by-cont:
  fixes m :: ('x, 's) state-monad
  and k :: 'x ⇒ ('y, 's) state-monad
  assumes none: ¬ Q None
  and cont-bound:
    ∧ x t. Some (x, t) ∈ set-dist (execute m s) ⇒
      wp-event (k x) Q t ≤ C
  shows wp-event (m  $\gg$  k) Q s ≤ C
proof -
  have wp-event (m  $\gg$  k) Q s =

```

```

    wp m (λout. case out of
      None ⇒ if Q None then 1 else 0
      | Some (x, t) ⇒ wp-event (k x) Q t) s
  unfolding wp-event-def by (simp add: wpsimps)
also have ... ≤ C
proof (rule wp-le-const-on-support)
  fix out
  assume out: out ∈ set-dist (execute m s)
  show (case out of None ⇒ if Q None then 1 else 0
    | Some (x, t) ⇒ wp-event (k x) Q t) ≤ C
  proof (cases out)
    case None
    then show ?thesis
    using none by simp
  next
    case (Some xt)
    then obtain x t where xt: xt = (x, t)
    by (cases xt) auto
    show ?thesis
    using cont-bound[of x t] out Some xt by simp
  qed
qed
finally show ?thesis .
qed

lemma sm-bind-return-map:
  ((m ≫ (λx. return (f x))) ≫ k) = (m ≫ (λx. k (f x)))
  by (simp add: sm-bind-assoc)

lemma wp-bind-return-map:
  fixes m :: ('x, 's) state-monad
  shows
    wp (m ≫ (λx. return (f x))) Q s =
    wp m (λout. case out of None ⇒ Q None
      | Some (x, t) ⇒ Q (Some (f x, t))) s
proof -
  have cont:
    (λr. case r of None ⇒ Q None
      | Some (x, t) ⇒ wp (return (f x)) Q t) =
    (λout. case out of None ⇒ Q None
      | Some (x, t) ⇒ Q (Some (f x, t)))
  by (rule ext) (simp add: wp-return split: option.splits prod.splits)
  show ?thesis
  by (subst wp-bind) (simp add: cont)
qed

lemma wp-event-bind-return-map:
  fixes m :: ('x, 's) state-monad
  shows

```

```

wp-event (m ≫ (λx. return (f x))) P s =
  wp-event m (λout. case out of None ⇒ P None
    | Some (x, t) ⇒ P (Some (f x, t))) s
proof -
  have indicator-eq:
    (λout. case out of None ⇒ if P None then 1 else 0
      | Some (x, t) ⇒ if P (Some (f x, t)) then 1 else 0) =
    (λout. if (case out of None ⇒ P None
      | Some (x, t) ⇒ P (Some (f x, t))) then 1 else 0)
  by (rule ext) (simp split: option.splits prod.splits)
  show ?thesis
    unfolding wp-event-def
    by (simp add: wp-bind-return-map indicator-eq)
qed

lemma set-dist-bindI:
  fixes m :: ('x, 's) state-monad
  and k :: 'x ⇒ ('y, 's) state-monad
  assumes head: Some (x, t) ∈ set-dist (execute m s)
  and tail: out ∈ set-dist (execute (k x) t)
  shows out ∈ set-dist (execute (m ≫ k) s)
proof -
  let ?M = dist (execute m s)
  let ?K = λr. bind-cont-map (λx t. dist (execute (k x) t)) r
  let ?h = Some (x, t)
  have head-dom: ?h ∈ dom ?M
    using head unfolding set-dist-def .
  have tail-dom: out ∈ dom (?K ?h)
    using tail unfolding set-dist-def bind-cont-map-def by simp
  have tail-dist-dom: out ∈ dom (dist (execute (k x) t))
    using tail unfolding set-dist-def .
  have head-pos: 0 < the (?M ?h)
    using dist-nonzero-on-dom[OF head-dom] by simp
  have tail-dist-pos: 0 < the (dist (execute (k x) t) out)
    using dist-nonzero-on-dom[OF tail-dist-dom] by simp
  obtain p where p-eq: dist (execute (k x) t) out = Some p
    using tail-dist-dom by auto
  have p-pos: 0 < p
    using tail-dist-pos p-eq by simp
  have tail-pos: 0 < option-default (?K ?h out)
    using p-eq p-pos by (simp add: bind-cont-map-def)
  have term-pos:
    0 < the (?M ?h) * option-default (?K ?h out)
    using head-pos tail-pos by simp
  have sum-pos:
    0 < (∑ r ∈ dom ?M. the (?M r) * option-default (?K r out))
  proof (rule sum-pos2)
    show finite (dom ?M)
      by simp

```

```

show ?h ∈ dom ?M
  by (rule head-dom)
show 0 < the (?M ?h) * option-default (?K ?h out)
  by (rule term-pos)
qed simp
let ?p = (∑ r ∈ dom ?M. the (?M r) * option-default (?K r out))
have p-nonzero: ?p ≠ 0
  using sum-pos by simp
have dist-eq: dist (execute (m ≫ k) s) out = Some ?p
  unfolding sm-bind.rep-eq dist-bind.rep-eq map-bind-def
  by (simp add: Let-def o-def map-fun-def p-nonzero)
then show ?thesis
  unfolding set-dist-def by auto
qed

end

end

```

```

theory Soundness-Core-Events
imports Soundness-Core-WP
begin

```

Transcript-shape, Merkle binding, and alpha/header event definitions.

```

context soundness
begin

```

definition verifier-header-messages

```

:: 'f ⇒ 'f list ⇒ 'f ⇒ 'f list ⇒ 'f ⇒
  'f list ⇒ 'f ⇒ 'f list

```

where

```

verifier-header-messages fr f-fri-roots f-final as dg composition-fri-roots final =
  [fr] @ f-fri-roots @ [f-final] @ as @ [dg] @
  composition-fri-roots @ [final]

```

definition verifier-header-state

```

:: ('f, 'a) protocol-channel-scheme ⇒ 'f ⇒ 'f list ⇒ 'f ⇒
  'f list ⇒ 'f ⇒ 'f list ⇒ 'f ⇒ 'f

```

where

```

verifier-header-state s fr f-fri-roots f-final as dg composition-fri-roots final =
  foldl concat (PState s)
  (verifier-header-messages fr f-fri-roots f-final as dg
   composition-fri-roots final)

```

definition verifier-header-transcript

```

:: ('f, 'a) protocol-channel-scheme ⇒ 'f ⇒ 'f list ⇒ 'f ⇒
  'f list ⇒ 'f ⇒ 'f list ⇒ 'f ⇒ 'f list ⇒ bool

```

where

```

verifier-header-transcript s fr f-fri-roots f-final as dg
composition-fri-roots final rest  $\longleftrightarrow$ 
PTranscript s =
  verifier-header-messages fr f-fri-roots f-final as dg
  composition-fri-roots final @ rest  $\wedge$ 
length f-fri-roots = ceil-log clength  $\wedge$ 
length as = length spec  $\wedge$ 
length composition-fri-roots = ceil-log (to-nat dg + 1)

```

definition *merkle-root-binds-table*
 $:: 'f \Rightarrow 'f \text{ list} \Rightarrow ('f, 'a) \text{ protocol-channel-scheme} \Rightarrow \text{bool}$
where
 $\text{merkle-root-binds-table } rt \text{ table } s \longleftrightarrow$
 $(\exists \text{ tree} :: 'f \text{ tree. created-tree table tree } s \wedge rt = \text{value tree})$

definition *merkle-root-binding-collision*
 $:: 'f \Rightarrow ('f, 'a) \text{ protocol-channel-scheme} \Rightarrow \text{bool}$
where
 $\text{merkle-root-binding-collision } rt \text{ s} \longleftrightarrow$
 $(\exists \text{ table table'.$
 $\text{merkle-root-binds-table } rt \text{ table } s \wedge$
 $\text{merkle-root-binds-table } rt \text{ table' } s \wedge$
 $\text{table} \neq \text{table'})$

definition *hash-map-output-collision*
 $:: ('f, 'a) \text{ protocol-channel-scheme} \Rightarrow \text{bool}$
where
 $\text{hash-map-output-collision } s \longleftrightarrow$
 $(\exists (x :: 'f \text{ protocol-hash-input}) (y :: 'f \text{ protocol-hash-input}) z.$
 $x \neq y \wedge$
 $\text{fmlookup (HashMap } s) x = \text{Some } z \wedge$
 $\text{fmlookup (HashMap } s) y = \text{Some } z)$

definition *hash-map-output-values*
 $:: ('f, 'a) \text{ protocol-channel-scheme} \Rightarrow 'f \text{ set}$
where
 $\text{hash-map-output-values } s =$
 $\{z. \exists x :: 'f \text{ protocol-hash-input. fmlookup (HashMap } s) x = \text{Some } z\}$

lemma *hash-map-output-values-verifier-initial-state[simp]:*
 $\text{hash-map-output-values (verifier-initial-state tr)} = \{\}$
unfolding *hash-map-output-values-def* **by** *simp*

lemma *merkle-root-binding-collisionI:*
assumes *merkle-root-binds-table* $rt \text{ table } s$
and *merkle-root-binds-table* $rt \text{ table' } s$
and $\text{table} \neq \text{table'}$
shows *merkle-root-binding-collision* $rt \text{ s}$
using *assms* **unfolding** *merkle-root-binding-collision-def* **by** *blast*

lemma *merkle-root-binds-table-unique-if-no-collision*:
assumes $\neg$ *merkle-root-binding-collision* *rt s*
and *merkle-root-binds-table* *rt table s*
and *merkle-root-binds-table* *rt table' s*
shows *table = table'*
using *assms* **unfolding** *merkle-root-binding-collision-def* **by** *blast*

lemma *merkle-root-binds-table-mono*:
assumes *bind*: *merkle-root-binds-table* *rt table s*
and *ext*: $s \leq t$
shows *merkle-root-binds-table* *rt table t*

proof –
from *bind* **obtain** *tree* **where**
created: *created-tree* *table tree s*
and *root*: *rt = value tree*
unfolding *merkle-root-binds-table-def* **by** *blast*
have *created-tree* *table tree t*
by (*rule created-tree-mono*[*OF created ext*])
then show *?thesis*
unfolding *merkle-root-binds-table-def*
using *root* **by** *blast*
qed

definition *hash-maps-compatible*
 $:: (f, 'a) \text{ protocol-channel-scheme} \Rightarrow$
 $(f, 'a) \text{ protocol-channel-scheme} \Rightarrow \text{bool}$
where
hash-maps-compatible $s \ t \longleftrightarrow$
 $(\forall x \ y \ z.$
 $\text{fmlookup } (\text{HashMap } s) \ x = \text{Some } y \longrightarrow$
 $\text{fmlookup } (\text{HashMap } t) \ x = \text{Some } z \longrightarrow$
 $y = z)$

definition *hash-map-value-conflict*
 $:: (f, 'a) \text{ protocol-channel-scheme} \Rightarrow$
 $(f, 'a) \text{ protocol-channel-scheme} \Rightarrow \text{bool}$
where
hash-map-value-conflict $s \ t \longleftrightarrow$
 $(\exists x \ y \ z.$
 $\text{fmlookup } (\text{HashMap } s) \ x = \text{Some } y \wedge$
 $\text{fmlookup } (\text{HashMap } t) \ x = \text{Some } z \wedge$
 $y \neq z)$

lemma *hash-maps-compatible-iff-no-value-conflict*:
hash-maps-compatible $s \ t \longleftrightarrow \neg \text{hash-map-value-conflict } s \ t$
unfolding *hash-maps-compatible-def* *hash-map-value-conflict-def* **by** *blast*

lemma *hash-map-value-conflict-sym*:

```

assumes hash-map-value-conflict s t
shows hash-map-value-conflict t s
using assms unfolding hash-map-value-conflict-def by blast

lemma hash-maps-compatible-sym:
assumes hash-maps-compatible s t
shows hash-maps-compatible t s
using assms
unfolding hash-maps-compatible-iff-no-value-conflict
using hash-map-value-conflict-sym by blast

lemma hash-map-value-conflict-no-common-extension:
assumes conflict: hash-map-value-conflict s t
and s-ext:  $s \leq u$ 
and t-ext:  $t \leq u$ 
shows False
proof -
from conflict obtain x y z where
  s-lookup: fmlookup (HashMap s) x = Some y
  and t-lookup: fmlookup (HashMap t) x = Some z
  and neq:  $y \neq z$ 
  unfolding hash-map-value-conflict-def by blast
have u-y: fmlookup (HashMap u) x = Some y
  by (rule hash-extension-lookup[OF s-lookup s-ext])
have u-z: fmlookup (HashMap u) x = Some z
  by (rule hash-extension-lookup[OF t-lookup t-ext])
show False
  using u-y u-z neq by simp
qed

lemma common-extension-imp-no-hash-map-value-conflict:
assumes s-ext:  $s \leq u$ 
and t-ext:  $t \leq u$ 
shows  $\neg$  hash-map-value-conflict s t
using hash-map-value-conflict-no-common-extension[OF s-ext t-ext] by blast

lemma hash-map-value-conflict-blocks-clean-merge-obligation:
assumes hash-map-value-conflict s t
shows  $\neg$  (hash-maps-compatible s t  $\wedge$ 
   $\neg$  hash-map-output-collision (hash-state-merge s t))
using assms unfolding hash-maps-compatible-iff-no-value-conflict by simp

fun merkle-hash-key :: 'f protocol-hash-input  $\Rightarrow$  bool
where
  merkle-hash-key (MerkleLeaf -)  $\longleftrightarrow$  True
| merkle-hash-key (MerkleNode - -)  $\longleftrightarrow$  True
| merkle-hash-key (FiatShamirChallenge -)  $\longleftrightarrow$  False
| merkle-hash-key (TraceFriChallenge - -)  $\longleftrightarrow$  False
| merkle-hash-key (CompositionFriChallenge - -)  $\longleftrightarrow$  False

```

| merkle-hash-key (AlphaChallenge - -) $\longleftrightarrow$ False
 | merkle-hash-key (QueryIndexChallenge - -) $\longleftrightarrow$ False

definition merkle-hash-maps-compatible

:: ('f, 'a) protocol-channel-scheme $\Rightarrow$
 ('f, 'a) protocol-channel-scheme $\Rightarrow$ bool

where

merkle-hash-maps-compatible s t $\longleftrightarrow$
 ($\forall$ x y z.
 merkle-hash-key x $\longrightarrow$
 fmllookup (HashMap s) x = Some y $\longrightarrow$
 fmllookup (HashMap t) x = Some z $\longrightarrow$
 y = z)

definition merkle-hash-value-conflict

:: ('f, 'a) protocol-channel-scheme $\Rightarrow$
 ('f, 'a) protocol-channel-scheme $\Rightarrow$ bool

where

merkle-hash-value-conflict s t $\longleftrightarrow$
 ($\exists$ x y z.
 merkle-hash-key x $\wedge$
 fmllookup (HashMap s) x = Some y $\wedge$
 fmllookup (HashMap t) x = Some z $\wedge$
 y $\neq$ z)

lemma merkle-hash-maps-compatible-iff-no-value-conflict:

merkle-hash-maps-compatible s t $\longleftrightarrow$
 $\neg$ merkle-hash-value-conflict s t

unfolding merkle-hash-maps-compatible-def merkle-hash-value-conflict-def
by blast

lemma merkle-hash-value-conflict-sym:

assumes merkle-hash-value-conflict s t
shows merkle-hash-value-conflict t s
using assms **unfolding** merkle-hash-value-conflict-def **by** blast

lemma merkle-hash-maps-compatible-sym:

assumes merkle-hash-maps-compatible s t
shows merkle-hash-maps-compatible t s
using assms **unfolding** merkle-hash-maps-compatible-def **by** blast

definition merkle-hash-extends

:: ('f, 'a) protocol-channel-scheme $\Rightarrow$
 ('f, 'a) protocol-channel-scheme $\Rightarrow$ bool

where

merkle-hash-extends s t $\longleftrightarrow$
 ($\forall$ x y.
 merkle-hash-key x $\longrightarrow$
 fmllookup (HashMap s) x = Some y $\longrightarrow$

$fmlookup (HashMap\ t)\ x = Some\ y)$

lemma *merkle-hash-value-conflict-no-common-extension:*

assumes *conflict: merkle-hash-value-conflict s t*

and *ext-s: merkle-hash-extends s u*

and *ext-t: merkle-hash-extends t u*

shows *False*

proof –

from *conflict* **obtain** $x\ y\ z$ **where**

merkle: merkle-hash-key x

and *s-lookup: fmlookup (HashMap s) x = Some y*

and *t-lookup: fmlookup (HashMap t) x = Some z*

and *neg: y ≠ z*

unfolding *merkle-hash-value-conflict-def* **by** *blast*

have *u-y: fmlookup (HashMap u) x = Some y*

using *ext-s merkle s-lookup* **unfolding** *merkle-hash-extends-def* **by** *blast*

have *u-z: fmlookup (HashMap u) x = Some z*

using *ext-t merkle t-lookup* **unfolding** *merkle-hash-extends-def* **by** *blast*

show *False*

using *u-y u-z neg* **by** *simp*

qed

lemma *common-merkle-extension-imp-no-merkle-hash-value-conflict:*

assumes *ext-s: merkle-hash-extends s u*

and *ext-t: merkle-hash-extends t u*

shows $\neg \text{merkle-hash-value-conflict } s\ t$

using *merkle-hash-value-conflict-no-common-extension* [*OF ext-s ext-t*]

by *blast*

lemma *merkle-hash-maps-compatible-if-common-merkle-extension:*

assumes *ext-s: merkle-hash-extends s u*

and *ext-t: merkle-hash-extends t u*

shows *merkle-hash-maps-compatible s t*

using *common-merkle-extension-imp-no-merkle-hash-value-conflict*

[*OF ext-s ext-t*]

unfolding *merkle-hash-maps-compatible-iff-no-value-conflict* .

definition *merkle-hash-state-merge*

$:: (f, 'a)\ \text{protocol-channel-scheme} \Rightarrow$

$(f, 'a)\ \text{protocol-channel-scheme} \Rightarrow$

$(f, 'a)\ \text{protocol-channel-scheme}$

where

merkle-hash-state-merge s t =

$s \langle HashMap :=$

$fmfilter\ merkle\text{-}hash\text{-}key\ (HashMap\ s) ++_f$

$fmfilter\ merkle\text{-}hash\text{-}key\ (HashMap\ t) \rangle$

lemma *merkle-hash-state-merge-extends-right:*

merkle-hash-extends t (merkle-hash-state-merge s t)

```

unfolding merkle-hash-extends-def merkle-hash-state-merge-def
by (simp add: fmlookup-dom-iff)

lemma merkle-hash-state-merge-extends-left-if-compatible:
  assumes compatible: merkle-hash-maps-compatible s t
  shows merkle-hash-extends s (merkle-hash-state-merge s t)
  unfolding merkle-hash-extends-def
proof (intro allI impI)
  fix x y
  assume merkle: merkle-hash-key x
  and s-lookup: fmlookup (HashMap s) x = Some y
  show fmlookup (HashMap (merkle-hash-state-merge s t)) x = Some y
proof (cases fmlookup (HashMap t) x)
  case None
  then show ?thesis
    using merkle s-lookup
    unfolding merkle-hash-state-merge-def
    by (simp add: fmlookup-dom-iff)
  next
  case (Some z)
  have z = y
    using compatible merkle s-lookup Some
    unfolding merkle-hash-maps-compatible-def by blast
  then show ?thesis
    using merkle Some
    unfolding merkle-hash-state-merge-def
    by (simp add: fmlookup-dom-iff)
qed
qed

lemma merkle-hash-state-merge-lookupD:
  assumes lookup:
    fmlookup (HashMap (merkle-hash-state-merge s t)) x = Some z
  obtains
    merkle-hash-key x fmlookup (HashMap s) x = Some z
    | merkle-hash-key x fmlookup (HashMap t) x = Some z
proof -
  have merkle: merkle-hash-key x
  using lookup
  unfolding merkle-hash-state-merge-def
  by (cases merkle-hash-key x) auto
show ?thesis
proof (cases fmlookup (HashMap t) x)
  case (Some z')
  then have z-eq: z' = z
    using lookup merkle
    unfolding merkle-hash-state-merge-def
    by (simp add: fmlookup-dom-iff)
  show ?thesis

```

```

    by (rule that(2)[OF merkle Some[unfolded z-eq]])
next
case None
have s-lookup: fmllookup (HashMap s) x = Some z
  using lookup merkle None
  unfolding merkle-hash-state-merge-def
  by (simp add: fmllookup-dom-iff)
show ?thesis
  by (rule that(1)[OF merkle s-lookup])
qed
qed

lemma merkle-hash-state-merge-clean-if-common-clean-merkle-extension:
  assumes ext-s: merkle-hash-extends s u
  and ext-t: merkle-hash-extends t u
  and clean:  $\neg$  hash-map-output-collision u
  shows  $\neg$  hash-map-output-collision (merkle-hash-state-merge s t)
proof
  assume collision: hash-map-output-collision (merkle-hash-state-merge s t)
  then obtain x y z where neq:  $x \neq y$ 
  and x-lookup:
    fmllookup (HashMap (merkle-hash-state-merge s t)) x = Some z
  and y-lookup:
    fmllookup (HashMap (merkle-hash-state-merge s t)) y = Some z
  unfolding hash-map-output-collision-def by blast
  have x-lookup-u: fmllookup (HashMap u) x = Some z
  proof -
    from x-lookup show ?thesis
  proof (rule merkle-hash-state-merge-lookupD)
    assume merkle: merkle-hash-key x
    and lookup-s: fmllookup (HashMap s) x = Some z
    show ?thesis
      using ext-s merkle lookup-s
      unfolding merkle-hash-extends-def by blast
  next
    assume merkle: merkle-hash-key x
    and lookup-t: fmllookup (HashMap t) x = Some z
    show ?thesis
      using ext-t merkle lookup-t
      unfolding merkle-hash-extends-def by blast
  qed
qed
have y-lookup-u: fmllookup (HashMap u) y = Some z
proof -
  from y-lookup show ?thesis
proof (rule merkle-hash-state-merge-lookupD)
  assume merkle: merkle-hash-key y
  and lookup-s: fmllookup (HashMap s) y = Some z
  show ?thesis

```

```

    using ext-s merkle lookup-s
    unfolding merkle-hash-extends-def by blast
next
  assume merkle: merkle-hash-key y
  and lookup-t: fmlookup (HashMap t) y = Some z
  show ?thesis
    using ext-t merkle lookup-t
    unfolding merkle-hash-extends-def by blast
qed
qed
have hash-map-output-collision u
  unfolding hash-map-output-collision-def
  using neq x-lookup-u y-lookup-u by blast
then show False
  using clean by contradiction
qed

lemma common-clean-merkle-extension-imp-pairwise-merkle-clean:
  assumes ext-s: merkle-hash-extends s u
  and ext-t: merkle-hash-extends t u
  and clean:  $\neg$  hash-map-output-collision u
  shows
     $\neg$  merkle-hash-value-conflict s t  $\wedge$ 
     $\neg$  hash-map-output-collision (merkle-hash-state-merge s t)
  using common-merkle-extension-imp-no-merkle-hash-value-conflict
    [OF ext-s ext-t]
    merkle-hash-state-merge-clean-if-common-clean-merkle-extension
    [OF ext-s ext-t clean]
  by simp

definition merkle-hash-cross-output-collision
  :: ('f, 'a) protocol-channel-scheme  $\Rightarrow$ 
    ('f, 'a) protocol-channel-scheme  $\Rightarrow$  bool
  where
    merkle-hash-cross-output-collision s t  $\longleftrightarrow$ 
      ( $\exists x y z.$ 
        merkle-hash-key x  $\wedge$  merkle-hash-key y  $\wedge$  x  $\neq$  y  $\wedge$ 
        ((fmlookup (HashMap s) x = Some z  $\wedge$ 
          fmlookup (HashMap t) y = Some z)  $\vee$ 
          (fmlookup (HashMap t) x = Some z  $\wedge$ 
            fmlookup (HashMap s) y = Some z)))

definition merkle-hash-pairwise-coupling-bad
  :: ('f, 'a) protocol-channel-scheme  $\Rightarrow$ 
    ('f, 'a) protocol-channel-scheme  $\Rightarrow$  bool
  where
    merkle-hash-pairwise-coupling-bad s t  $\longleftrightarrow$ 
      merkle-hash-value-conflict s t  $\vee$ 
      merkle-hash-cross-output-collision s t

```

lemma *merkle-hash-cross-output-collision-sym*:
assumes *merkle-hash-cross-output-collision s t*
shows *merkle-hash-cross-output-collision t s*
using *assms* **unfolding** *merkle-hash-cross-output-collision-def* **by** *blast*

lemma *merkle-hash-pairwise-coupling-bad-sym*:
assumes *merkle-hash-pairwise-coupling-bad s t*
shows *merkle-hash-pairwise-coupling-bad t s*
using *assms*
unfolding *merkle-hash-pairwise-coupling-bad-def*
using *merkle-hash-value-conflict-sym*
merkle-hash-cross-output-collision-sym **by** *blast*

lemma *merkle-hash-state-merge-collision-imp-local-or-cross*:
assumes *collision*:
hash-map-output-collision (merkle-hash-state-merge s t)
shows
hash-map-output-collision s $\vee$
hash-map-output-collision t $\vee$
merkle-hash-cross-output-collision s t

proof –
from *collision* **obtain** *x y z* **where** *neq: x $\neq$ y*
and *x-lookup*:
fmlookup (HashMap (merkle-hash-state-merge s t)) x = Some z
and *y-lookup*:
fmlookup (HashMap (merkle-hash-state-merge s t)) y = Some z
unfolding *hash-map-output-collision-def* **by** *blast*
from *x-lookup* **consider**
(xs) merkle-hash-key x fmlookup (HashMap s) x = Some z
 $|$ *(xt) merkle-hash-key x fmlookup (HashMap t) x = Some z*
by *(rule merkle-hash-state-merge-lookupD)*
then show *?thesis*
proof *cases*
case *xs*
from *y-lookup* **consider**
(ys) merkle-hash-key y fmlookup (HashMap s) y = Some z
 $|$ *(yt) merkle-hash-key y fmlookup (HashMap t) y = Some z*
by *(rule merkle-hash-state-merge-lookupD)*
then show *?thesis*
proof *cases*
case *ys*
have *hash-map-output-collision s*
unfolding *hash-map-output-collision-def*
using *neq xs ys* **by** *blast*
then show *?thesis* **by** *blast*
next
case *yt*
have *merkle-hash-cross-output-collision s t*


```

    unfolding merkle-hash-cross-output-collision-def
    using neq xs yt by blast
    then show ?thesis by blast
qed
next
case xt
from y-lookup consider
  (ys) merkle-hash-key y fmlookup (HashMap s) y = Some z
  | (yt) merkle-hash-key y fmlookup (HashMap t) y = Some z
  by (rule merkle-hash-state-merge-lookupD)
then show ?thesis
proof cases
case ys
have merkle-hash-cross-output-collision s t
  unfolding merkle-hash-cross-output-collision-def
  using neq xt ys by blast
then show ?thesis by blast
next
case yt
have hash-map-output-collision t
  unfolding hash-map-output-collision-def
  using neq xt yt by blast
then show ?thesis by blast
qed
qed
qed

lemma merkle-hash-pairwise-bad-imp-local-collision-or-coupling-bad:
  assumes pair-bad:
    merkle-hash-value-conflict s t  $\vee$ 
    hash-map-output-collision (merkle-hash-state-merge s t)
  shows
    hash-map-output-collision s  $\vee$ 
    hash-map-output-collision t  $\vee$ 
    merkle-hash-pairwise-coupling-bad s t
proof (cases merkle-hash-value-conflict s t)
case True
then show ?thesis
  unfolding merkle-hash-pairwise-coupling-bad-def by blast
next
case False
then have merge-collision:
  hash-map-output-collision (merkle-hash-state-merge s t)
  using pair-bad by blast
from merkle-hash-state-merge-collision-imp-local-or-cross
  [OF merge-collision]
show ?thesis
  unfolding merkle-hash-pairwise-coupling-bad-def by blast
qed

```

definition *hash-state-merge*

```

:: ('f, 'a) protocol-channel-scheme =>
   ('f, 'a) protocol-channel-scheme =>
   ('f, 'a) protocol-channel-scheme
where
  hash-state-merge s t =
    s(HashMap := HashMap s ++f HashMap t)

```

lemma *hash-state-merge-extends-right:*

```

t ≤ hash-state-merge s t
unfolding hash-state-merge-def less-eq-hash-ext-def less-eq-fmap-def
by auto

```

lemma *hash-state-merge-extends-left-if-compatible:*

```

assumes compatible: hash-maps-compatible s t
shows s ≤ hash-state-merge s t
proof -
  have lookup:
    ∧x y. fmlookup (HashMap s) x = Some y =>
      fmlookup (HashMap (hash-state-merge s t)) x = Some y
  proof -
    fix x y
    assume s-lookup: fmlookup (HashMap s) x = Some y
    show fmlookup (HashMap (hash-state-merge s t)) x = Some y
    proof (cases x |∈| fmdom (HashMap t))
      case True
        then obtain z where t-lookup: fmlookup (HashMap t) x = Some z
        unfolding fmlookup-dom-iff by blast
        have z = y
          using compatible s-lookup t-lookup
          unfolding hash-maps-compatible-def by blast
        then show ?thesis
          using True t-lookup
          unfolding hash-state-merge-def by simp
      next
        case False
        then show ?thesis
          using s-lookup unfolding hash-state-merge-def by simp
    qed
  qed
  show ?thesis
    unfolding less-eq-hash-ext-def less-eq-fmap-def
  proof
    fix x
    show fmlookup (HashMap s) x = None ∨
      fmlookup (HashMap s) x =
        fmlookup (HashMap (hash-state-merge s t)) x
    proof (cases fmlookup (HashMap s) x)

```

```

    case None
    then show ?thesis by simp
next
case (Some y)
then have fmlookup (HashMap (hash-state-merge s t)) x = Some y
  by (rule lookup)
then show ?thesis
  using Some by simp
qed
qed
qed

lemma hash-maps-compatible-if-common-extension:
  assumes s-ext:  $s \leq u$ 
  and t-ext:  $t \leq u$ 
  shows hash-maps-compatible s t
  unfolding hash-maps-compatible-def
proof (intro allI impI)
  fix x y z
  assume s-lookup: fmlookup (HashMap s) x = Some y
  and t-lookup: fmlookup (HashMap t) x = Some z
  have u-y: fmlookup (HashMap u) x = Some y
    by (rule hash-extension-lookup[OF s-lookup s-ext])
  have u-z: fmlookup (HashMap u) x = Some z
    by (rule hash-extension-lookup[OF t-lookup t-ext])
  show y = z
    using u-y u-z by simp
qed

lemma hash-state-merge-le-common-extension:
  assumes s-ext:  $s \leq u$ 
  and t-ext:  $t \leq u$ 
  shows hash-state-merge s t  $\leq u$ 
  unfolding less-eq-hash-ext-def less-eq-fmap-def
proof
  fix x
  show fmlookup (HashMap (hash-state-merge s t)) x = None  $\vee$ 
    fmlookup (HashMap (hash-state-merge s t)) x = fmlookup (HashMap u) x
proof (cases x  $\in$  fmdom (HashMap t))
case True
then obtain z where t-lookup: fmlookup (HashMap t) x = Some z
  unfolding fmlookup-dom-iff by blast
have merge-lookup:
  fmlookup (HashMap (hash-state-merge s t)) x = Some z
  using True t-lookup unfolding hash-state-merge-def by simp
have u-lookup: fmlookup (HashMap u) x = Some z
  by (rule hash-extension-lookup[OF t-lookup t-ext])
show ?thesis
  using merge-lookup u-lookup by simp

```

```

next
  case False
  show ?thesis
  proof (cases fmlookup (HashMap s) x)
    case None
    then have fmlookup (HashMap (hash-state-merge s t)) x = None
      using False unfolding hash-state-merge-def by simp
    then show ?thesis
      by simp
  next
  case (Some y)
  then have merge-lookup:
    fmlookup (HashMap (hash-state-merge s t)) x = Some y
    using False unfolding hash-state-merge-def by simp
  have u-lookup: fmlookup (HashMap u) x = Some y
    by (rule hash-extension-lookup[OF Some s-ext])
  show ?thesis
    using merge-lookup u-lookup by simp
qed
qed
qed

```

```

lemma hash-state-merge-clean-if-common-clean-extension:
  assumes s-ext:  $s \leq u$ 
  and t-ext:  $t \leq u$ 
  and clean:  $\neg \text{hash-map-output-collision } u$ 
  shows  $\neg \text{hash-map-output-collision (hash-state-merge s t)}$ 
proof
  assume collision: hash-map-output-collision (hash-state-merge s t)
  have merge-ext: hash-state-merge s t  $\leq u$ 
    by (rule hash-state-merge-le-common-extension[OF s-ext t-ext])
  from collision obtain x y z where neg:  $x \neq y$ 
  and x-lookup:
    fmlookup (HashMap (hash-state-merge s t)) x = Some z
  and y-lookup:
    fmlookup (HashMap (hash-state-merge s t)) y = Some z
  unfolding hash-map-output-collision-def by blast
  have x-lookup-u: fmlookup (HashMap u) x = Some z
    by (rule hash-extension-lookup[OF x-lookup merge-ext])
  have y-lookup-u: fmlookup (HashMap u) y = Some z
    by (rule hash-extension-lookup[OF y-lookup merge-ext])
  have hash-map-output-collision u
    unfolding hash-map-output-collision-def
    using neg x-lookup-u y-lookup-u by blast
  then show False
    using clean by contradiction
qed

```

```

lemma hash-map-output-collisionI:

```

```

fixes  $x\ y :: 'f\ \text{protocol-hash-input}$ 
assumes  $x \neq y$ 
  and  $\text{fmlookup}\ (\text{HashMap}\ s)\ x = \text{Some}\ z$ 
  and  $\text{fmlookup}\ (\text{HashMap}\ s)\ y = \text{Some}\ z$ 
shows  $\text{hash-map-output-collision}\ s$ 
using  $\text{assms}\ \text{unfolding}\ \text{hash-map-output-collision-def}\ \text{by}\ \text{blast}$ 

lemma  $\text{hash-map-output-collision-merkle-leafI}$ :
assumes  $x \neq y$ 
  and  $\text{fmlookup}\ (\text{HashMap}\ s)\ (\text{MerkleLeaf}\ x) = \text{Some}\ z$ 
  and  $\text{fmlookup}\ (\text{HashMap}\ s)\ (\text{MerkleLeaf}\ y) = \text{Some}\ z$ 
shows  $\text{hash-map-output-collision}\ s$ 
by ( $\text{rule}\ \text{hash-map-output-collisionI}[\text{OF} - \text{assms}(2,3)]$ )
  ( $\text{use}\ \text{assms}(1)\ \text{in}\ \text{simp}$ )

lemma  $\text{hash-map-output-collision-merkle-nodeI}$ :
assumes  $(x1, x2) \neq (y1, y2)$ 
  and  $\text{fmlookup}\ (\text{HashMap}\ s)\ (\text{MerkleNode}\ x1\ x2) = \text{Some}\ z$ 
  and  $\text{fmlookup}\ (\text{HashMap}\ s)\ (\text{MerkleNode}\ y1\ y2) = \text{Some}\ z$ 
shows  $\text{hash-map-output-collision}\ s$ 
by ( $\text{rule}\ \text{hash-map-output-collisionI}[\text{OF} - \text{assms}(2,3)]$ )
  ( $\text{use}\ \text{assms}(1)\ \text{in}\ \text{auto}$ )

lemma  $\text{hash-map-output-collision-merkle-leaf-nodeI}$ :
assumes  $\text{fmlookup}\ (\text{HashMap}\ s)\ (\text{MerkleLeaf}\ x) = \text{Some}\ z$ 
  and  $\text{fmlookup}\ (\text{HashMap}\ s)\ (\text{MerkleNode}\ y1\ y2) = \text{Some}\ z$ 
shows  $\text{hash-map-output-collision}\ s$ 
by ( $\text{rule}\ \text{hash-map-output-collisionI}[\text{OF} - \text{assms}]$ )
   $\text{simp}$ 

lemma  $\text{hash-map-output-valuesI}$ :
fixes  $x :: 'f\ \text{protocol-hash-input}$ 
assumes  $\text{fmlookup}\ (\text{HashMap}\ s)\ x = \text{Some}\ z$ 
shows  $z \in \text{hash-map-output-values}\ s$ 
using  $\text{assms}\ \text{unfolding}\ \text{hash-map-output-values-def}\ \text{by}\ \text{blast}$ 

lemma  $\text{hash-map-output-values-alt-def}$ :
 $\text{hash-map-output-values}\ s =$ 
  ( $\lambda x. \text{the}\ (\text{fmlookup}\ (\text{HashMap}\ s)\ x))\ 'f\text{mdom}'\ (\text{HashMap}\ s)$ 
unfolding  $\text{hash-map-output-values-def}$ 
proof
show  $\{z. \exists x. \text{fmlookup}\ (\text{HashMap}\ s)\ x = \text{Some}\ z\}$ 
   $\subseteq (\lambda x. \text{the}\ (\text{fmlookup}\ (\text{HashMap}\ s)\ x))\ 'f\text{mdom}'\ (\text{HashMap}\ s)$ 
proof
fix  $z$ 
assume  $z \in \{z. \exists x. \text{fmlookup}\ (\text{HashMap}\ s)\ x = \text{Some}\ z\}$ 
then obtain  $x$  where  $\text{lookup:}\ \text{fmlookup}\ (\text{HashMap}\ s)\ x = \text{Some}\ z$ 
by  $\text{blast}$ 
have  $x \in \text{fmdom}'\ (\text{HashMap}\ s)$ 

```

```

    using lookup by (simp add: fmlookup-dom'-iff)
  moreover have the (fmlookup (HashMap s) x) = z
    using lookup by simp
  ultimately show z ∈
    (λx. the (fmlookup (HashMap s) x)) 'fmdom' (HashMap s)
    by blast
qed
show (λx. the (fmlookup (HashMap s) x)) 'fmdom' (HashMap s)
  ⊆ {z. ∃ x. fmlookup (HashMap s) x = Some z}
  by (auto simp: fmlookup-dom'-iff)
qed

```

```

lemma finite-hash-map-output-values[simp]:
  finite (hash-map-output-values s)
  unfolding hash-map-output-values-alt-def by simp

```

```

lemma card-hash-map-output-values-le-fmdom:
  card (hash-map-output-values s) ≤ card (fmdom' (HashMap s))
  unfolding hash-map-output-values-alt-def
  by (rule card-image-le) simp

```

```

lemma hash-map-output-values-after-update-subset:
  fixes x :: 'f protocol-hash-input
  shows
    hash-map-output-values
      (s(HashMap := fmupd x y (HashMap s))) ⊆
    insert y (hash-map-output-values s)
  unfolding hash-map-output-values-def
  by auto

```

```

lemma card-hash-map-output-values-after-update-le:
  fixes x :: 'f protocol-hash-input
  shows
    card (hash-map-output-values
      (s(HashMap := fmupd x y (HashMap s)))) ≤
    Suc (card (hash-map-output-values s))

```

```

proof -
  have card (hash-map-output-values
    (s(HashMap := fmupd x y (HashMap s)))) ≤
    card (insert y (hash-map-output-values s))
    by (rule card-mono)
    (simp-all add: hash-map-output-values-after-update-subset)
  also have ... ≤ Suc (card (hash-map-output-values s))
    by (simp add: card-insert-if)
  finally show ?thesis .
qed

```

```

lemma hash-map-output-collision-after-fresh-updateD:
  fixes x :: 'f protocol-hash-input

```

```

assumes clean:  $\neg$  hash-map-output-collision s
and fresh: fmlookup (HashMap s) x = None
and collision:
  hash-map-output-collision
  (s(HashMap := fmupd x y (HashMap s)))
shows y  $\in$  hash-map-output-values s
proof –
obtain a b z where neq: a  $\neq$  b
and a-lookup:
  fmlookup (fmupd x y (HashMap s)) a = Some z
and b-lookup:
  fmlookup (fmupd x y (HashMap s)) b = Some z
using collision unfolding hash-map-output-collision-def by auto
show ?thesis
proof (cases a = x)
  case True
    then have z-y: z = y
      using a-lookup by simp
    have b-ne-x: b  $\neq$  x
      using neq True by simp
    have fmlookup (HashMap s) b = Some y
      using b-lookup b-ne-x z-y by simp
    then show ?thesis
      by (rule hash-map-output-valuesI)
  next
    case a-ne-x: False
    show ?thesis
    proof (cases b = x)
      case True
        then have z-y: z = y
          using b-lookup by simp
        have fmlookup (HashMap s) a = Some y
          using a-lookup a-ne-x z-y by simp
        then show ?thesis
          by (rule hash-map-output-valuesI)
      next
        case b-ne-x: False
        have a-old: fmlookup (HashMap s) a = Some z
          using a-lookup a-ne-x by simp
        have b-old: fmlookup (HashMap s) b = Some z
          using b-lookup b-ne-x by simp
        have hash-map-output-collision s
          by (rule hash-map-output-collisionI[OF neq a-old b-old])
        then show ?thesis
          using clean by contradiction
    qed
  qed
qed

```

lemma *no-hash-map-output-collision-after-known-update*:
fixes $x :: 'f$ *protocol-hash-input*
assumes *clean*: $\neg$ *hash-map-output-collision* s
and *known*: *fmlookup* (*HashMap* s) $x = \text{Some } y$
shows $\neg$ *hash-map-output-collision*
 $(s(\text{HashMap} := \text{fmupd } x \ y \ (\text{HashMap } s)))$
proof
assume *collision*:
hash-map-output-collision
 $(s(\text{HashMap} := \text{fmupd } x \ y \ (\text{HashMap } s)))$
obtain $a \ b \ z$ **where** *neq*: $a \neq b$
and *a-lookup*:
fmlookup (*fmupd* $x \ y \ (\text{HashMap } s)$) $a = \text{Some } z$
and *b-lookup*:
fmlookup (*fmupd* $x \ y \ (\text{HashMap } s)$) $b = \text{Some } z$
using *collision* **unfolding** *hash-map-output-collision-def* **by** *auto*
have *a-old*: *fmlookup* (*HashMap* s) $a = \text{Some } z$
using *a-lookup* *known* **by** (*cases* $a = x$) *simp-all*
have *b-old*: *fmlookup* (*HashMap* s) $b = \text{Some } z$
using *b-lookup* *known* **by** (*cases* $b = x$) *simp-all*
have *hash-map-output-collision* s
by (*rule* *hash-map-output-collisionI*[*OF* *neq* *a-old* *b-old*])
then show *False*
using *clean* **by** *contradiction*
qed

lemma *dist-expect-uniform-fUNIV-indicator*:
fixes $S :: 'f$ *set*
shows
dist-expect (*dist-uniform* (*fUNIV* $:: 'f$ *fset*))
 $(\lambda x. \text{if } x \in S \text{ then } 1 \text{ else } 0) =$
 $\text{nnreal } (\text{card } S) / \text{nnreal size}$
proof –
let $?U = (\text{fUNIV} :: 'f \text{ fset})$
let $?w = 1 / \text{nnreal } (\text{CARD}('f))$
let $?B = ((\lambda x :: 'f. (x, ?w)) \mid ?U)$

have *U-nonempty*: $?U \neq \{\mid\}$
proof –
have (*undefined* $:: 'f$) $\mid \in ?U$
by *simp*
then show *?thesis*
by *auto*
qed
have *fcard-U*: *fcard* $?U = \text{CARD}('f)$
by (*simp add*: *fcard.rep-eq*)
have *total-nonzero*:
 $\text{fsum snd } ((\lambda x :: 'f. (x, 1::\text{prob})) \mid ?U) \neq 0$
using *fsum-snd-uniform-ones*[*of* $?U$] *fcard-U* **by** *simp*

have *dist-eq*:
 $\text{dist } (\text{dist-uniform } ?U) = \text{map-of } ?B$
unfolding *dist-uniform-def dist-of-fset.rep-eq map-of-fset-def*
using *norm-fset-uniform-ones*[*OF U-nonempty*] *total-nonzero fcard-U*
by (*simp add: Let-def*)

have *lookup*:
 $\text{dist } (\text{dist-uniform } ?U) \ x = \text{Some } ?w \text{ for } x :: 'f$

proof –
have *mem*: $(x, ?w) \in | ?B$
by *simp*
have *uniq*:
 $\bigwedge y \ z. (x, y) \in | ?B \implies (x, z) \in | ?B \implies y = z$
by *auto*
have *val*: $\text{map-of-val } ?B \ x = ?w$
by (*rule map-of-val-unique-key*[*OF uniq mem*])
have $x \in | \text{fimage fst } ?B$
using *mem* **by** *force*
then have $\text{map-of } ?B \ x = \text{Some } (\text{map-of-val } ?B \ x)$
unfolding *map-of-def* **by** *simp*
also have $\dots = \text{Some } ?w$
using *val* **by** *simp*
finally show *?thesis*
using *dist-eq* **by** *simp*
qed

have *dom-eq*: $\text{dom } (\text{dist } (\text{dist-uniform } ?U)) = (\text{UNIV} :: 'f \text{ set})$
using *lookup* **by** *auto*

have *dist-expect* $(\text{dist-uniform } ?U) \ (\lambda x. \text{if } x \in S \text{ then } 1 \text{ else } 0) =$
 $(\sum_{x \in \text{UNIV}} ?w * (\text{if } x \in S \text{ then } 1 \text{ else } 0))$
unfolding *dist-expect-def dom-eq*
by (*intro sum.cong refl*) (*simp add: lookup*)
also have $\dots = (\sum_{x \in \text{UNIV}} \text{if } x \in S \text{ then } ?w \text{ else } 0)$
by (*intro sum.cong refl*) (*simp split: if-splits*)
also have $\dots = (\sum_{x \in S} ?w)$
proof –
have $(\sum_{x \in \text{UNIV}} \text{if } x \in S \text{ then } ?w \text{ else } 0) =$
 $(\sum_{x \in \text{UNIV} \cap S} ?w)$
by (*subst sum.inter-restrict*[*symmetric*]) *simp-all*
also have $\dots = (\sum_{x \in S} ?w)$
by *simp*
finally show *?thesis* .

qed
also have $\dots = (\sum_{x \in S} (1 :: \text{prob})) / \text{nnreal } (\text{CARD } ('f))$
by (*rule sum-divide-nnreal*) *simp*
also have $\dots = \text{nnreal } (\text{card } S) / \text{nnreal } (\text{CARD } ('f))$
by (*simp add: sum-1-nnreal*)
also have $\dots = \text{nnreal } (\text{card } S) / \text{nnreal size}$

```

    using size-card by simp
    finally show ?thesis .
qed

lemma dist-expect-hash-dist-fresh-output-values:
  fixes x :: 'f protocol-hash-input
  assumes fresh: fmllookup (HashMap s) x = None
  shows
    dist-expect (hash-dist x s)
      (λy. if y ∈ hash-map-output-values s then 1 else 0) =
      nnreal (card (hash-map-output-values s)) / nnreal size
  using fresh
  unfolding hash-dist-def option-default-dist-def hash-default-def
  by (simp add: dist-expect-uniform-fUNIV-indicator)

lemma dist-expect-hash-dist-fresh-indicator:
  fixes x :: 'f protocol-hash-input
  and s :: ('f, 'a) protocol-channel-scheme
  assumes fresh: fmllookup (HashMap s) x = None
  shows
    dist-expect (hash-dist x s) (λy. if y ∈ B then 1 else 0) =
      nnreal (card B) / nnreal size
  using fresh
  unfolding hash-dist-def option-default-dist-def hash-default-def
  by (simp add: dist-expect-uniform-fUNIV-indicator)

lemma wp-hash-fresh-set:
  fixes x :: 'f protocol-hash-input
  and s :: ('f, 'a) protocol-channel-scheme
  assumes fresh: fmllookup (HashMap s) x = None
  shows
    wp-event (hash x) (λout. case out of
      None ⇒ False
      | Some (y, -) ⇒ y ∈ B) s =
      nnreal (card B) / nnreal size
  unfolding wp-event-def
  by (simp add: wp-hash dist-expect-hash-dist-fresh-indicator[OF fresh])

lemma wp-hash-fresh-singleton-outcome:
  fixes x :: 'f protocol-hash-input
  and s :: ('f, 'a) protocol-channel-scheme
  assumes fresh: fmllookup (HashMap s) x = None
  shows
    wp-event (hash x)
      (λout. out =
        Some (y, s(HashMap := fmapd x y (HashMap s)))) s =
      1 / nnreal size
  proof -
    have indicator:

```

```

(λxa. if xa = y ∧
  s(HashMap := fmupd x xa (HashMap s)) =
  s(HashMap := fmupd x y (HashMap s))
  then 1 else 0 :: prob) =
(λxa. if xa ∈ {y} then 1 else 0)
by auto
have wp-event (hash x)
  (λout. out =
    Some (y, s(HashMap := fmupd x y (HashMap s)))) s =
  dist-expect (hash-dist x s) (λxa. if xa ∈ {y} then 1 else 0)
  unfolding wp-event-def by (simp add: wp-hash indicator)
also have ... = nnreal (card ({y} :: 'f set)) / nnreal size
  by (rule dist-expect-hash-dist-fresh-indicator[OF fresh])
also have ... = 1 / nnreal size
  by simp
finally show ?thesis .
qed

```

lemma *hash-fresh-outcome*:

```

fixes x :: 'f protocol-hash-input
  and s :: ('f, 'a) protocol-channel-scheme
assumes fresh: fmlookup (HashMap s) x = None
shows
  Some (y, s(HashMap := fmupd x y (HashMap s))) ∈
    set-dist (execute (hash x) s)
proof -
  have positive:
    0 < wp-event (hash x)
    (λout. out =
      Some (y, s(HashMap := fmupd x y (HashMap s)))) s
  using wp-hash-fresh-singleton-outcome[OF fresh, of y]
    size-card by simp
  then obtain out where support:
    out ∈ set-dist (execute (hash x) s)
  and out-eq:
    out = Some (y, s(HashMap := fmupd x y (HashMap s)))
  by (rule wp-event-pos-imp-exists-support)
  show ?thesis
    using support out-eq by simp
qed

```

lemma *fresh-merkle-hash-has-pairwise-value-conflict*:

```

fixes x :: 'f protocol-hash-input
  and s :: ('f, 'a) protocol-channel-scheme
assumes merkle: merkle-hash-key x
  and fresh: fmlookup (HashMap s) x = None
  and neq: y ≠ z
shows
  ∃ t u.

```

$\text{Some } (y, t) \in \text{set-dist } (\text{execute } (\text{hash } x) s) \wedge$
 $\text{Some } (z, u) \in \text{set-dist } (\text{execute } (\text{hash } x) s) \wedge$
 $\text{merkle-hash-value-conflict } t u \wedge$
 $\text{merkle-hash-pairwise-coupling-bad } t u$
proof –
let $?t = s(\text{HashMap} := \text{fmupd } x y (\text{HashMap } s))$
let $?u = s(\text{HashMap} := \text{fmupd } x z (\text{HashMap } s))$
have $\text{out-y}: \text{Some } (y, ?t) \in \text{set-dist } (\text{execute } (\text{hash } x) s)$
by $(\text{rule hash-fresh-outcome}[\text{OF fresh}])$
have $\text{out-z}: \text{Some } (z, ?u) \in \text{set-dist } (\text{execute } (\text{hash } x) s)$
by $(\text{rule hash-fresh-outcome}[\text{OF fresh}])$
have $\text{conflict}: \text{merkle-hash-value-conflict } ?t ?u$
unfolding $\text{merkle-hash-value-conflict-def}$
using $\text{merkle neq by auto}$
then have $\text{coupling}: \text{merkle-hash-pairwise-coupling-bad } ?t ?u$
unfolding $\text{merkle-hash-pairwise-coupling-bad-def by simp}$
show $?thesis$
using $\text{out-y out-z conflict coupling by blast}$
qed

lemma $\text{wp-receive-tagged-random-field-element-fresh-set}:$
fixes $s :: ('f, 'a) \text{ protocol-channel-scheme}$
assumes $\text{fresh}: \text{fmlookup } (\text{HashMap } s) (\text{tag } (PState s)) = \text{None}$
shows
 $\text{wp-event } (\text{receive-tagged-random-field-element tag})$
 $(\lambda \text{out. case out of None} \Rightarrow \text{False} \mid \text{Some } (y, -) \Rightarrow y \in B) s =$
 $\text{nnreal } (\text{card } B) / \text{nnreal size}$
unfolding $\text{wp-event-def receive-tagged-random-field-element-def}$
 $\text{protocol-receive-tagged-random-field-element-def}$
by $(\text{simp add: wpsimps dist-expect-hash-dist-fresh-indicator}[\text{OF fresh}])$

lemma $\text{wp-receive-counted-tagged-random-field-element-fresh-set}:$
fixes $s :: ('f, 'a) \text{ protocol-channel-scheme}$
assumes $\text{fresh}:$
 $\text{fmlookup } (\text{HashMap } s) (\text{tag } (\text{counter } s) (PState s)) = \text{None}$
shows
 wp-event
 $(\text{protocol-receive-counted-tagged-random-field-element counter bump tag})$
 $(\lambda \text{out. case out of None} \Rightarrow \text{False} \mid \text{Some } (y, -) \Rightarrow y \in B) s =$
 $\text{nnreal } (\text{card } B) / \text{nnreal size}$
unfolding $\text{wp-event-def protocol-receive-counted-tagged-random-field-element-def}$
by $(\text{simp add: wpsimps dist-expect-hash-dist-fresh-indicator}[\text{OF fresh}])$

lemma $\text{wp-receive-alpha-challenge-fresh-set}:$
fixes $s :: ('f, 'a) \text{ protocol-channel-scheme}$
assumes $\text{fresh}: \text{fmlookup } (\text{HashMap } s) (\text{AlphaChallenge } (PAlphaCounter s) (PState s)) = \text{None}$
shows
 $\text{wp-event receive-alpha-challenge}$
 $(\lambda \text{out. case out of None} \Rightarrow \text{False} \mid \text{Some } (y, -) \Rightarrow y \in B) s =$

$nnreal (card B) / nnreal size$
unfolding *receive-alpha-challenge-def*
by (*rule wp-receive-counted-tagged-random-field-element-fresh-set*
[**where** *counter=PAlphaCounter*
and *bump=λs. s(PAlphaCounter := Suc (PAlphaCounter s))*
and *tag=AlphaChallenge and B=B and s=s, OF fresh*])

lemma *wp-receive-query-index-challenge-fresh-set*:
fixes *s :: ('f, 'a) protocol-channel-scheme*
assumes *fresh*:
fmlookup (HashMap s) (QueryIndexChallenge (PQueryCounter s) (PState s)) = None
shows
wp-event receive-query-index-challenge
($\lambda out. case out of None \Rightarrow False \mid Some (y, -) \Rightarrow y \in B$) *s =*
 $nnreal (card B) / nnreal size$
unfolding *receive-query-index-challenge-def*
by (*rule wp-receive-counted-tagged-random-field-element-fresh-set*
[**where** *counter=PQueryCounter*
and *bump=λs. s(PQueryCounter := Suc (PQueryCounter s))*
and *tag=QueryIndexChallenge and B=B and s=s, OF fresh*])

definition *query-index-raw-preimage :: nat set $\Rightarrow$ 'f set*
where
query-index-raw-preimage B =
 $\{raw. index (to-nat raw) \in B\}$

definition *query-index-nat-preimage :: nat set $\Rightarrow$ nat set*
where
query-index-nat-preimage B =
 $\{n \in range\ to-nat. n \bmod query-sample-space-size \in B\}$

lemma *finite-query-index-raw-preimage[simp]*:
finite (query-index-raw-preimage B)
unfolding *query-index-raw-preimage-def* **by** *simp*

lemma *finite-query-index-nat-preimage[simp]*:
finite (query-index-nat-preimage B)
unfolding *query-index-nat-preimage-def* **by** *simp*

lemma *query-index-raw-preimage-image-to-nat*:
 $to-nat \text{ ` } query-index-raw-preimage B = query-index-nat-preimage B$
unfolding *query-index-raw-preimage-def query-index-nat-preimage-def*
index-def query-sample-space-size-def
by *auto*

lemma *card-query-index-raw-preimage-eq-nat-preimage*:
 $card (query-index-raw-preimage B) = card (query-index-nat-preimage B)$
proof –
have *inj: inj-on to-nat (query-index-raw-preimage B)*

```

    using to-nat-inj by (rule inj-on-subset) simp
  have card (query-index-raw-preimage B) =
    card (to-nat ' query-index-raw-preimage B)
    by (rule card-image[symmetric, OF inj])
  also have ... = card (query-index-nat-preimage B)
    unfolding query-index-raw-preimage-image-to-nat ..
  finally show ?thesis .
qed

lemma wp-receive-query-index-challenge-fresh-index-set:
  fixes s :: ('f, 'a) protocol-channel-scheme
  assumes fresh:
    fmllookup (HashMap s) (QueryIndexChallenge (PQueryCounter s) (PState s)) = None
  shows
    wp-event receive-query-index-challenge
      (λout. case out of
        None ⇒ False
        | Some (raw, -) ⇒ index (to-nat raw) ∈ B) s =
      nnreal (card (query-index-raw-preimage B)) / nnreal size
proof -
  have event-eq:
    (λout. case out of
      None ⇒ False
      | Some (raw, -) ⇒ index (to-nat raw) ∈ B) =
    (λout. case out of
      None ⇒ False
      | Some (raw, -) ⇒ raw ∈ query-index-raw-preimage B)
  unfolding query-index-raw-preimage-def
  by (rule ext) (auto split: option.splits prod.splits)
show ?thesis
  unfolding event-eq
  by (rule wp-receive-query-index-challenge-fresh-set[OF fresh])
qed

```

```

lemma card-mod-lessThan-dvd:
  assumes q-pos: 0 < q
  and dvd: q dvd N
  and b-lt: b < q
  shows card {n. n < N ∧ n mod q = b} = N div q
proof -
  let ?A = {0..<N div q}
  let ?B = {n. n < N ∧ n mod q = b}
  have N-eq: N = (N div q) * q
    using dvd q-pos by (simp add: dvd-eq-mod-eq-0)
  have image-eq: (λk. k * q + b) ' ?A = ?B
  proof
    show (λk. k * q + b) ' ?A ⊆ ?B
  proof
    fix n

```

```

assume  $n \in (\lambda k. k * q + b) \text{ ' } ?A$ 
then obtain  $k$  where  $k\text{-lt}: k < N \text{ div } q$  and  $n\text{-eq}: n = k * q + b$ 
  by auto
have  $k * q + b < (N \text{ div } q) * q$ 
proof –
  have  $sk\text{-le}: \text{Suc } k \leq N \text{ div } q$ 
    using  $k\text{-lt}$  by simp
  have  $skq\text{-le}: \text{Suc } k * q \leq (N \text{ div } q) * q$ 
    by (rule mult-le-mono1 [OF  $sk\text{-le}$ ])
  have  $k * q + b < k * q + q$ 
    using  $b\text{-lt}$  by simp
  also have  $\dots = \text{Suc } k * q$ 
    by simp
  also have  $\dots \leq (N \text{ div } q) * q$ 
    by (rule skq-le)
  finally show  $?thesis$  .
qed
then have  $n < N$ 
  using  $N\text{-eq}$   $n\text{-eq}$  by simp
moreover have  $n \bmod q = b$ 
  using  $n\text{-eq}$   $b\text{-lt}$  by simp
ultimately show  $n \in ?B$ 
  by simp
qed
show  $?B \subseteq (\lambda k. k * q + b) \text{ ' } ?A$ 
proof
  fix  $n$ 
  assume  $n\text{-in}: n \in ?B$ 
  then have  $n\text{-lt}: n < N$  and  $n\text{-mod}: n \bmod q = b$ 
    by simp-all
  have  $n\text{-eq}: n = (n \text{ div } q) * q + b$ 
    using div-mult-mod-eq[of  $n$   $q$ ]  $n\text{-mod}$  by simp
  have  $div\text{-lt}: n \text{ div } q < N \text{ div } q$ 
  proof (rule ccontr)
    assume  $\neg n \text{ div } q < N \text{ div } q$ 
    then have  $ge: N \text{ div } q \leq n \text{ div } q$ 
      by simp
    have  $N = (N \text{ div } q) * q$ 
      by (rule N-eq)
    also have  $\dots \leq (n \text{ div } q) * q$ 
      using  $ge$  by simp
    also have  $\dots \leq (n \text{ div } q) * q + b$ 
      by simp
    also have  $\dots = n$ 
      using  $n\text{-eq}$  by simp
    finally show False
      using  $n\text{-lt}$  by simp
  qed
then show  $n \in (\lambda k. k * q + b) \text{ ' } ?A$ 

```

```

    using n-eq by auto
  qed
qed
have inj: inj-on ( $\lambda k. k * q + b$ ) ?A
  using q-pos unfolding inj-on-def by auto
have card ?B = card (( $\lambda k. k * q + b$ ) ‘ ?A)
  using image-eq by simp
also have ... = card ?A
  by (rule card-image[OF inj])
also have ... = N div q
  by simp
finally show ?thesis .
qed

lemma card-query-index-nat-preimage-uniform-range:
  assumes range-eq: range to-nat = {0..size}
    and dvd: query-sample-space-size dvd size
    and subset:  $B \subseteq \text{query-sample-space}$ 
  shows
    card (query-index-nat-preimage B) =
      (size div query-sample-space-size) * card B
proof -
  let ?q = query-sample-space-size
  let ?A =  $\lambda b. \{n. n < \text{size} \wedge n \bmod ?q = b\}$ 
  have finite-space: finite query-sample-space
    unfolding query-sample-space-def by simp
  have finite-B: finite B
    by (rule finite-subset[OF subset finite-space])
  have B-lt:  $\bigwedge b. b \in B \implies b < ?q$ 
    using subset unfolding query-sample-space-def by auto
  have preimage-eq: query-index-nat-preimage B =  $\bigcup (?A \text{ ‘ } B)$ 
    unfolding query-index-nat-preimage-def range-eq by auto
  have finite-A:  $\forall b \in B. \text{finite } (?A \text{ } b)$ 
    by simp
  have disjoint:
     $\forall b \in B. \forall c \in B. b \neq c \implies ?A \text{ } b \cap ?A \text{ } c = \{\}$ 
    by auto
  have card (query-index-nat-preimage B) = card ( $\bigcup (?A \text{ ‘ } B)$ )
    unfolding preimage-eq ..
  also have ... =  $(\sum_{b \in B. \text{card } (?A \text{ } b)})$ 
    by (rule card-UN-disjoint[OF finite-B finite-A disjoint])
  also have ... =  $(\sum_{b \in B. \text{size div } ?q})$ 
    by (rule sum.cong)
    (use B-lt in
       $\langle \text{simp-all add: card-mod-lessThan-dvd} [\text{OF query-sample-space-size-pos dvd}] \rangle$ )
  also have ... = (size div ?q) * card B
    by simp
  finally show ?thesis .
qed

```


lemma *wp-receive-trace-fri-challenge-fresh-set:*
fixes $s :: ('f, 'a) \text{ protocol-channel-scheme}$
assumes *fresh:*
 $\text{fmlookup } (\text{HashMap } s) (\text{TraceFriChallenge } (P\text{TraceFriCounter } s) (P\text{State } s)) = \text{None}$
shows
 $\text{wp-event receive-trace-fri-challenge}$
 $(\lambda \text{out. case out of None} \Rightarrow \text{False} \mid \text{Some } (y, -) \Rightarrow y \in B) s =$
 $\text{nnreal } (\text{card } B) / \text{nnreal size}$
unfolding *receive-trace-fri-challenge-def*
by (*rule wp-receive-counted-tagged-random-field-element-fresh-set*
[**where** $\text{counter} = P\text{TraceFriCounter}$
and $\text{bump} = \lambda s. s(P\text{TraceFriCounter} := \text{Suc } (P\text{TraceFriCounter } s))$
and $\text{tag} = \text{TraceFriChallenge}$ **and** $B = B$ **and** $s = s$, *OF fresh*])

lemma *wp-receive-composition-fri-challenge-fresh-set:*
fixes $s :: ('f, 'a) \text{ protocol-channel-scheme}$
assumes *fresh:*
 $\text{fmlookup } (\text{HashMap } s) (\text{CompositionFriChallenge } (P\text{CompositionFriCounter } s) (P\text{State } s))$
 $= \text{None}$
shows
 $\text{wp-event receive-composition-fri-challenge}$
 $(\lambda \text{out. case out of None} \Rightarrow \text{False} \mid \text{Some } (y, -) \Rightarrow y \in B) s =$
 $\text{nnreal } (\text{card } B) / \text{nnreal size}$
unfolding *receive-composition-fri-challenge-def*
by (*rule wp-receive-counted-tagged-random-field-element-fresh-set*
[**where** $\text{counter} = P\text{CompositionFriCounter}$
and $\text{bump} = \lambda s. s(P\text{CompositionFriCounter} := \text{Suc } (P\text{CompositionFriCounter } s))$
and $\text{tag} = \text{CompositionFriChallenge}$ **and** $B = B$ **and** $s = s$, *OF fresh*])

fun *merkle-path-root-in-map*
 $:: ('f, 'a) \text{ protocol-channel-scheme} \Rightarrow$
 $\text{nat} \Rightarrow \text{nat} \Rightarrow 'f \Rightarrow 'f \text{ list} \Rightarrow 'f \text{ option}$
where
 $\text{merkle-path-root-in-map } s \text{ len idx leaf } [] =$
 $\text{fmlookup } (\text{HashMap } s) (\text{MerkleLeaf leaf})$
 $\mid \text{merkle-path-root-in-map } s \text{ len idx leaf } (a \# \text{path}) =$
 $(\text{if } \text{idx} < \text{len div } 2$
 $\text{then case merkle-path-root-in-map } s (\text{len div } 2) \text{ idx leaf path of}$
 $\text{None} \Rightarrow \text{None}$
 $\mid \text{Some } x \Rightarrow \text{fmlookup } (\text{HashMap } s) (\text{MerkleNode } x a)$
 $\text{else case merkle-path-root-in-map } s (\text{len div } 2) (\text{idx} - \text{len div } 2)$
 leaf path of
 $\text{None} \Rightarrow \text{None}$
 $\mid \text{Some } x \Rightarrow \text{fmlookup } (\text{HashMap } s) (\text{MerkleNode } a x))$

lemma *check-authentication-path-outcome-merkle-path-root:*
fixes $\text{root leaf} :: 'f$
and $s \text{ t u} :: ('f, 'a) \text{ protocol-channel-scheme}$

```

assumes outcome:
  Some (root, t) ∈
    set-dist (execute (check-authentication-path len idx leaf path) s)
  and ext: t ≤ u
shows merkle-path-root-in-map u len idx leaf path = Some root
using outcome ext
unfolding check-authentication-path-def protocol-check-authentication-path-def
proof (induction path arbitrary: len idx leaf s root t)
case Nil
then have hash-step:
  Some (root, t) ∈ set-dist (execute (hash (MerkleLeaf leaf)) s)
  by simp
have lookup-t:
  fmlookup (HashMap t) (MerkleLeaf leaf) = Some root
  by (rule hash-outcome(2)[OF hash-step])
show ?case
  using hash-extension-lookup[OF lookup-t Nil.prems(2)] by simp
next
case (Cons a path)
show ?case
proof (cases idx < len div 2)
case True
from Cons.prems(1) obtain child s1 where
  child-check:
    Some (child, s1) ∈
      set-dist (execute
        (protocol-merkle.check-authentication-path
          (len div 2) idx (MerkleLeaf leaf) path) s)
  and root-hash:
    Some (root, t) ∈
      set-dist (execute (hash (MerkleNode child a)) s1)
  using True by (auto elim!: set-dist-bindE)
have s1-t: s1 ≤ t
  by (rule hash-outcome(1)[OF root-hash])
have s1-u: s1 ≤ u
  using s1-t Cons.prems(2) by (meson hash-ext-trans)
have child-root:
  merkle-path-root-in-map u (len div 2) idx leaf path = Some child
  by (rule Cons.IH[OF child-check s1-u])
have lookup-t:
  fmlookup (HashMap t) (MerkleNode child a) = Some root
  by (rule hash-outcome(2)[OF root-hash])
have lookup-u:
  fmlookup (HashMap u) (MerkleNode child a) = Some root
  by (rule hash-extension-lookup[OF lookup-t Cons.prems(2)])
show ?thesis
  using True child-root lookup-u by simp
next
case False

```

```

from Cons.prems(1) obtain child s1 where
  child-check:
    Some (child, s1) ∈
      set-dist (execute
        (protocol-merkle.check-authentication-path
          (len div 2) (idx - len div 2) (MerkleLeaf leaf) path) s)
  and root-hash:
    Some (root, t) ∈
      set-dist (execute (hash (MerkleNode a child)) s1)
  using False by (auto elim!; set-dist-bindE)
have s1-t: s1 ≤ t
  by (rule hash-outcome(1)[OF root-hash])
have s1-u: s1 ≤ u
  using s1-t Cons.prems(2) by (meson hash-ext-trans)
have child-root:
  merkle-path-root-in-map u (len div 2) (idx - len div 2) leaf path =
    Some child
  by (rule Cons.IH[OF child-check s1-u])
have lookup-t:
  fmlookup (HashMap t) (MerkleNode a child) = Some root
  by (rule hash-outcome(2)[OF root-hash])
have lookup-u:
  fmlookup (HashMap u) (MerkleNode a child) = Some root
  by (rule hash-extension-lookup[OF lookup-t Cons.prems(2)])
show ?thesis
  using False child-root lookup-u by simp
qed
qed

lemma merkle-path-root-same-index-eq-or-hash-collision:
  fixes root leaf leaf' :: 'f
  and s :: ('f, 'a) protocol-channel-scheme
  assumes root1:
    merkle-path-root-in-map s len idx leaf path = Some root
  and root2:
    merkle-path-root-in-map s len idx leaf' path' = Some root
  shows (leaf = leaf' ∧ path = path')  $\vee$  hash-map-output-collision s
  using root1 root2
proof (induction path arbitrary: len idx leaf path' leaf' root)
case Nil
show ?case
proof (cases path')
case Nil
show ?thesis
proof (cases leaf = leaf')
case True
then show ?thesis
  using Nil by simp
next

```

```

    case False
    have lookup-leaf: fmlookup (HashMap s) (MerkleLeaf leaf) = Some root
      using Nil.prem1 by simp
    have lookup-leaf':
      fmlookup (HashMap s) (MerkleLeaf leaf') = Some root
      using Nil.prem2 unfolding Nil by simp
    have hash-map-output-collision s
      by (rule hash-map-output-collision-merkle-leafI
        [OF False lookup-leaf lookup-leaf'])
    then show ?thesis by simp
  qed
next
case (Cons a rest)
have lookup-leaf: fmlookup (HashMap s) (MerkleLeaf leaf) = Some root
  using Nil.prem1 by simp
have hash-map-output-collision s
proof (cases idx < len div 2)
  case True
  from Nil.prem2[unfolded Cons] True obtain child where
    lookup-node: fmlookup (HashMap s) (MerkleNode child a) = Some root
    by (auto split: option.splits)
  show ?thesis
    by (rule hash-map-output-collision-merkle-leaf-nodeI
      [OF lookup-leaf lookup-node])
  next
  case False
  from Nil.prem2[unfolded Cons] False obtain child where
    lookup-node: fmlookup (HashMap s) (MerkleNode a child) = Some root
    by (auto split: option.splits)
  show ?thesis
    by (rule hash-map-output-collision-merkle-leaf-nodeI
      [OF lookup-leaf lookup-node])
  qed
  then show ?thesis by simp
qed
next
case (Cons a path)
show ?case
proof (cases path')
  case Nil
  have lookup-leaf': fmlookup (HashMap s) (MerkleLeaf leaf') = Some root
    using Cons.prem2 unfolding Nil by simp
  have hash-map-output-collision s
  proof (cases idx < len div 2)
    case True
    from Cons.prem1 True obtain child where
      lookup-node: fmlookup (HashMap s) (MerkleNode child a) = Some root
      by (auto split: option.splits)
    show ?thesis

```

```

    by (rule hash-map-output-collisionI
      [of MerkleNode child a MerkleLeaf leaf' s root])
      (use lookup-node lookup-leaf' in simp-all)
  next
  case False
  from Cons.premis(1) False obtain child where
    lookup-node: fmlookup (HashMap s) (MerkleNode a child) = Some root
    by (auto split: option.splits)
  show ?thesis
    by (rule hash-map-output-collisionI
      [of MerkleNode a child MerkleLeaf leaf' s root])
      (use lookup-node lookup-leaf' in simp-all)
  qed
  then show ?thesis by simp
next
case (Cons b path'')
show ?thesis
proof (cases idx < len div 2)
  case True
  from Cons.premis(1) obtain child where
    child1:
      merkle-path-root-in-map s (len div 2) idx leaf path =
        Some child
    and lookup1:
      fmlookup (HashMap s) (MerkleNode child a) = Some root
    using True by (auto split: option.splits)
  from Cons.premis(2)[unfolded Cons] obtain child' where
    child2:
      merkle-path-root-in-map s (len div 2) idx leaf' path'' =
        Some child'
    and lookup2:
      fmlookup (HashMap s) (MerkleNode child' b) = Some root
    using True by (auto split: option.splits)
  show ?thesis
  proof (cases (child, a) = (child', b))
    case False
    have hash-map-output-collision s
      by (rule hash-map-output-collision-merkle-nodeI
        [OF False lookup1 lookup2])
    then show ?thesis by simp
  next
  case pair-eq: True
  have child-eq: child = child' and a-eq: a = b
    using pair-eq by simp-all
  have rec:
    (leaf = leaf' ∧ path = path'') ∨ hash-map-output-collision s
    by (rule Cons.IH[OF child1 child2[unfolded child-eq[symmetric]]])
  then show ?thesis
    using a-eq Cons by auto

```

```

qed
next
case False
from Cons.premis(1) obtain child where
  child1:
    merkle-path-root-in-map s (len div 2) (idx - len div 2)
    leaf path = Some child
  and lookup1:
    fmlookup (HashMap s) (MerkleNode a child) = Some root
  using False by (auto split: option.splits)
from Cons.premis(2)[unfolded Cons] obtain child' where
  child2:
    merkle-path-root-in-map s (len div 2) (idx - len div 2)
    leaf' path'' = Some child'
  and lookup2:
    fmlookup (HashMap s) (MerkleNode b child') = Some root
  using False by (auto split: option.splits)
show ?thesis
proof (cases (a, child) = (b, child'))
  case False
  have hash-map-output-collision s
  by (rule hash-map-output-collision-merkle-nodeI
    [OF False lookup1 lookup2])
  then show ?thesis by simp
next
case pair-eq: True
have child-eq: child = child' and a-eq: a = b
  using pair-eq by simp-all
have rec:
  (leaf = leaf'  $\wedge$  path = path'')  $\vee$  hash-map-output-collision s
  by (rule Cons.IH[OF child1 child2[unfolded child-eq[symmetric]]])
  then show ?thesis
using a-eq Cons by auto
qed
qed
qed
qed

```

```

lemma protocol-created-tree-singletonD:
  assumes protocol-created-tree [x] t s
  obtains h where
    t =  $\langle \langle \rangle, h, \langle \rangle \rangle$ 
    fmlookup (HashMap s) (MerkleLeaf x) = Some h
  using assms
  unfolding protocol-created-tree-def
  by (auto simp: protocol-merkle.created-tree.simps)

```

```

lemma protocol-created-tree-internalD:
  assumes created: protocol-created-tree xs t s

```

```

    and len:  $2 \leq \text{length } xs$ 
  obtains l h r i where
    i = length xs div 2
    t = ⟨l, h, r⟩
    protocol-created-tree (take i xs) l s
    protocol-created-tree (drop i xs) r s
    fmlookup (HashMap s) (MerkleNode (value l) (value r)) = Some h
proof -
  obtain a b cs where xs-eq: xs = a # b # cs
  using len by (cases xs; cases tl xs) auto
  obtain l h r where
    t: t = ⟨l, h, r⟩
  and left:
    protocol-merkle.created-tree
      (take (length xs div 2) (map MerkleLeaf xs)) l s
  and right:
    protocol-merkle.created-tree
      (drop (length xs div 2) (map MerkleLeaf xs)) r s
  and root:
    fmlookup (HashMap s) (MerkleNode (value l) (value r)) = Some h
  using created len
  unfolding protocol-created-tree-def
  by (auto simp: xs-eq protocol-merkle.created-tree.simps Let-def)
  have left':
    protocol-created-tree (take (length xs div 2) xs) l s
  using left
  unfolding protocol-created-tree-def by (simp add: take-map)
  have right':
    protocol-created-tree (drop (length xs div 2) xs) r s
  using right
  unfolding protocol-created-tree-def by (simp add: drop-map)
  show ?thesis
  by (rule that[OF refl t left' right' root])
qed

```

```

lemma protocol-created-tree-path-root-in-map:
  assumes created: protocol-created-tree xs tree s
  and len-pow: length xs =  $2^n$ 
  and i-bound: i < length xs
  shows
    merkle-path-root-in-map s (length xs) i (xs ! i)
    (get-authentication-path (length xs) i tree) = Some (value tree)
  using created len-pow i-bound
proof (induction n arbitrary: xs tree i)
  case 0
  then have len-xs: length xs = 1
  by simp
  then obtain x where xs-eq: xs = [x]
  by (cases xs) auto

```

```

from protocol-created-tree-singletonD[OF 0.premis(1)[unfolded xs-eq]]
obtain h where tree-eq:
  tree = ⟨⟨⟩, h, ⟨⟩⟩
  and lookup: fmlookup (HashMap s) (MerkleLeaf x) = Some h
  by blast
have i-eq: i = 0
  using 0.premis(3) unfolding xs-eq by simp
show ?case
  using lookup unfolding xs-eq tree-eq i-eq by simp
next
case (Suc n)
let ?mid = length xs div 2
have len-xs: length xs = 2 ^ Suc n
  by (rule Suc.premis(2))
have len-ge: 2 ≤ length xs
  using len-xs by simp
have len-not-one: length xs ≠ 1
  using len-ge by simp
obtain l h r mid where mid-def: mid = ?mid
  and tree-eq: tree = ⟨l, h, r⟩
  and left: protocol-created-tree (take mid xs) l s
  and right: protocol-created-tree (drop mid xs) r s
  and root-lookup:
    fmlookup (HashMap s) (MerkleNode (value l) (value r)) = Some h
  by (rule protocol-created-tree-internalD[OF Suc.premis(1) len-ge])
have mid-pow: mid = 2 ^ n
  using len-xs unfolding mid-def by simp
have mid-pos: 0 < mid
  using mid-pow by simp
have mid-lt: mid < length xs
  using len-xs mid-pow by simp
have take-len: length (take mid xs) = 2 ^ n
  using mid-lt mid-pow by simp
have drop-len: length (drop mid xs) = 2 ^ n
  using len-xs mid-pow by simp
show ?case
proof (cases i < mid)
case True
have idx-take: i < length (take mid xs)
  using True take-len mid-pow by simp
have child:
  merkle-path-root-in-map s (length (take mid xs)) i
    (take mid xs ! i)
    (get-authentication-path (length (take mid xs)) i l) =
    Some (value l)
  by (rule Suc.IH[OF left take-len idx-take])
have child-mid:
  merkle-path-root-in-map s mid i (xs ! i)
    (get-authentication-path mid i l) = Some (value l)

```



```

    using child True take-len mid-pow by simp
show ?thesis
    using True child-mid root-lookup len-not-one
    unfolding tree-eq mid-def by simp
next
case False
have idx-drop:  $i - \text{mid} < \text{length } (\text{drop mid } xs)$ 
    using False Suc.premis(3) drop-len mid-pow mid-def by simp
have child:
    merkle-path-root-in-map s (length (drop mid xs)) (i - mid)
    (drop mid xs ! (i - mid))
    (get-authentication-path (length (drop mid xs)) (i - mid) r) =
    Some (value r)
    by (rule Suc.IH[OF right drop-len idx-drop])
have child-mid:
    merkle-path-root-in-map s mid (i - mid) (xs ! i)
    (get-authentication-path mid (i - mid) r) = Some (value r)
    using child False Suc.premis(3) drop-len mid-pow by simp
show ?thesis
    using False child-mid root-lookup len-not-one
    unfolding tree-eq mid-def by simp
qed
qed

lemma protocol-created-tree-merkle-mono:
  assumes created: protocol-created-tree xs t s
  and ext: merkle-hash-extends s u
  shows protocol-created-tree xs t u
  using created
proof (induction xs arbitrary: t rule: length-induct)
  case (1 xs)
  show ?case
  proof (cases xs = [])
    case True
    then show ?thesis
      using 1.premis
      unfolding protocol-created-tree-def
      by (simp add: protocol-merkle.created-tree.simps)
  next
  case xs-nonempty: False
  show ?thesis
  proof (cases length xs = 1)
    case True
    then obtain x where xs-eq:  $xs = [x]$ 
      using xs-nonempty by (cases xs) auto
    from protocol-created-tree-singletonD[OF 1.premis[unfolded xs-eq]]
    obtain h where
      t-eq:  $t = \langle \rangle, h, \langle \rangle$ 
      and lookup-s:  $\text{fmlookup } (\text{HashMap } s) (\text{MerkleLeaf } x) = \text{Some } h$ 

```

```

    by blast
  have lookup-u: fmlookup (HashMap u) (MerkleLeaf x) = Some h
    using ext lookup-s unfolding merkle-hash-extends-def by simp
  show ?thesis
    unfolding protocol-created-tree-def xs-eq t-eq
    using lookup-u by (simp add: protocol-merkle.created-tree.simps)
next
case not-singleton: False
have len-ge:  $2 \leq \text{length } xs$ 
  using xs-nonempty not-singleton by (cases xs; cases tl xs) auto
obtain l h r i where
  i-def:  $i = \text{length } xs \text{ div } 2$ 
  and t-eq:  $t = \langle l, h, r \rangle$ 
  and left-created: protocol-created-tree (take i xs) l s
  and right-created: protocol-created-tree (drop i xs) r s
  and root-lookup:
    fmlookup (HashMap s) (MerkleNode (value l) (value r)) = Some h
  by (rule protocol-created-tree-internalD[OF 1.prem len-ge])
have i-pos:  $0 < i$ 
  using i-def len-ge by simp
have i-lt:  $i < \text{length } xs$ 
  using i-def len-ge by simp
have take-less:  $\text{length } (\text{take } i \text{ xs}) < \text{length } xs$ 
  using i-lt by simp
have drop-less:  $\text{length } (\text{drop } i \text{ xs}) < \text{length } xs$ 
  using i-pos i-lt by simp
have left-u: protocol-created-tree (take i xs) l u
  by (rule 1.IH[rule-format, OF take-less left-created])
have right-u: protocol-created-tree (drop i xs) r u
  by (rule 1.IH[rule-format, OF drop-less right-created])
have root-lookup-u:
  fmlookup (HashMap u) (MerkleNode (value l) (value r)) = Some h
  using ext root-lookup unfolding merkle-hash-extends-def by simp
obtain a b cs where xs-eq:  $xs = a \# b \# cs$ 
  using len-ge by (cases xs; cases tl xs) auto
have take-leaf-eq:
  MerkleLeaf a #
    take (length cs div 2) (MerkleLeaf b # map MerkleLeaf cs) =
  MerkleLeaf a #
    map MerkleLeaf (take (length cs div 2) (b # cs))
  by (metis list.map(2) take-map)
have drop-leaf-eq:
  drop (length cs div 2) (MerkleLeaf b # map MerkleLeaf cs) =
  map MerkleLeaf (drop (length cs div 2) (b # cs))
  by (metis list.map(2) drop-map)
have left-u-raw:
  protocol-merkle.created-tree
    (MerkleLeaf a #
      take (length cs div 2) (MerkleLeaf b # map MerkleLeaf cs))

```

```

      l u
    using left-u
    unfolding protocol-created-tree-def i-def xs-eq
    by (simp add: take-leaf-eq)
  have right-u-raw:
    protocol-merkle.created-tree
      (drop (length cs div 2) (MerkleLeaf b # map MerkleLeaf cs))
      r u
    using right-u
    unfolding protocol-created-tree-def i-def xs-eq
    by (simp add: drop-leaf-eq)
  show ?thesis
    using left-u-raw right-u-raw root-lookup-u
    unfolding protocol-created-tree-def t-eq xs-eq
    by (auto simp: protocol-merkle.created-tree.simps Let-def)
qed
qed
qed

```

```

lemma created-tree-merkle-mono:
  assumes created: created-tree xs t s
  and ext: merkle-hash-extends s u
  shows created-tree xs t u
  using protocol-created-tree-merkle-mono
    [OF created[unfolded created-tree-def] ext]
  unfolding created-tree-def .

```

```

lemma merkle-root-binds-table-merkle-mono:
  assumes bind: merkle-root-binds-table rt table s
  and ext: merkle-hash-extends s u
  shows merkle-root-binds-table rt table u
proof -
  from bind obtain tree where
    created: created-tree table tree s
  and root: rt = value tree
  unfolding merkle-root-binds-table-def by blast
  have created-tree table tree u
  by (rule created-tree-merkle-mono[OF created ext])
  then show ?thesis
    unfolding merkle-root-binds-table-def
    using root by blast
qed

```

```

lemma protocol-created-tree-same-length-root-eq-or-hash-collision:
  assumes left-created: protocol-created-tree xs tx s
  and right-created: protocol-created-tree ys ty s
  and same-len: length xs = length ys
  and nonempty-xs: xs ≠ []
  and nonempty-ys: ys ≠ []

```

```

    and same-root: value tx = value ty
  shows  $xs = ys \vee \text{hash-map-output-collision } s$ 
  using assms
  proof (induction xs arbitrary: ys tx ty rule: length-induct)
  case (1 xs)
  show ?case
  proof (cases length xs = 1)
  case True
  then obtain x where xs-eq:  $xs = [x]$ 
    using 1.premis(4) by (cases xs) auto
  from True 1.premis(3,5) obtain y where ys-eq:  $ys = [y]$ 
    by (cases ys) auto
  from protocol-created-tree-singletonD[OF 1.premis(1)[unfolded xs-eq]]
  obtain hx where tx:  $tx = \langle \langle \rangle, hx, \langle \rangle \rangle$ 
    and hx-lookup:  $\text{fmlookup } (\text{HashMap } s) (\text{MerkleLeaf } x) = \text{Some } hx$ 
    by blast
  from protocol-created-tree-singletonD[OF 1.premis(2)[unfolded ys-eq]]
  obtain hy where ty:  $ty = \langle \langle \rangle, hy, \langle \rangle \rangle$ 
    and hy-lookup:  $\text{fmlookup } (\text{HashMap } s) (\text{MerkleLeaf } y) = \text{Some } hy$ 
    by blast
  have h-eq:  $hx = hy$ 
    using 1.premis(6) unfolding tx ty by simp
  show ?thesis
  proof (cases x = y)
  case True
  then show ?thesis
    using xs-eq ys-eq by simp
  next
  case False
  have hy-lookup':  $\text{fmlookup } (\text{HashMap } s) (\text{MerkleLeaf } y) = \text{Some } hx$ 
    using hy-lookup h-eq by simp
  then have hash-map-output-collision s
    by (rule hash-map-output-collision-merkle-leafI[OF False hx-lookup])
  then show ?thesis by simp
  qed
  next
  case False
  have 0 < length xs
    using 1.premis(4) by simp
  have len-ge:  $2 \leq \text{length } xs$ 
    using False <0 < length xs> by linarith
  have len-ge-y:  $2 \leq \text{length } ys$ 
    using len-ge 1.premis(3) by simp
  obtain lx hx rx i where i-def:  $i = \text{length } xs \text{ div } 2$ 
    and tx:  $tx = \langle lx, hx, rx \rangle$ 
    and lx-created:  $\text{protocol-created-tree } (\text{take } i \text{ } xs) \text{ } lx \text{ } s$ 
    and rx-created:  $\text{protocol-created-tree } (\text{drop } i \text{ } xs) \text{ } rx \text{ } s$ 
    and hx-lookup:
       $\text{fmlookup } (\text{HashMap } s) (\text{MerkleNode } (\text{value } lx) (\text{value } rx)) = \text{Some } hx$ 

```

```

  by (rule protocol-created-tree-internalD[OF 1.premis(1) len-ge])
obtain ly hy ry j where j-def: j = length ys div 2
and ty: ty = ⟨ly, hy, ry⟩
and ly-created: protocol-created-tree (take j ys) ly s
and ry-created: protocol-created-tree (drop j ys) ry s
and hy-lookup:
  fmllookup (HashMap s) (MerkleNode (value ly) (value ry)) = Some hy
by (rule protocol-created-tree-internalD[OF 1.premis(2) len-ge-y])
have j-i: j = i
  using i-def j-def 1.premis(3) by simp
have ly-created-i: protocol-created-tree (take i ys) ly s
  using ly-created j-i by simp
have ry-created-i: protocol-created-tree (drop i ys) ry s
  using ry-created j-i by simp
have h-eq: hx = hy
  using 1.premis(6) unfolding tx ty by simp
show ?thesis
proof (cases (value lx, value rx) = (value ly, value ry))
  case False
  have hy-lookup':
    fmllookup (HashMap s) (MerkleNode (value ly) (value ry)) = Some hx
  using hy-lookup h-eq by simp
  then have hash-map-output-collision s
    by (rule hash-map-output-collision-merkle-nodeI[OF False hx-lookup])
  then show ?thesis by simp
next
  case True
  then have left-root: value lx = value ly
    and right-root: value rx = value ry
    by simp-all
  have i-pos: 0 < i
    using i-def len-ge by simp
  have i-lt: i < length xs
    using i-def len-ge by simp
  have i-lt-y: i < length ys
    using i-lt 1.premis(3) by simp
  have length-take-xs: length (take i xs) = i
    using i-lt by simp
  have length-take-ys: length (take i ys) = i
    using i-lt-y by simp
  have left-len: length (take i xs) = length (take i ys)
    using i-def j-i 1.premis(3) by simp
  have right-len: length (drop i xs) = length (drop i ys)
    using i-def j-i 1.premis(3) by simp
  have left-nonempty-xs: take i xs ≠ []
    using i-pos length-take-xs by auto
  have left-nonempty-ys: take i ys ≠ []
    using i-pos length-take-ys by auto
  have right-nonempty-xs: drop i xs ≠ []

```

```

    using i-lt by simp
  have right-nonempty-ys: drop i ys ≠ []
    using i-lt-y by simp
  have take-less: length (take i xs) < length xs
    using i-lt by simp
  have drop-less: length (drop i xs) < length xs
    using i-pos i-lt by simp
  show ?thesis
  proof (cases take i xs = take i ys)
    case False
    have take i xs = take i ys ∨ hash-map-output-collision s
      by (rule 1.IH[rule-format, OF take-less lx-created ly-created-i
        left-len left-nonempty-xs left-nonempty-ys left-root])
    then show ?thesis
      using False by simp
  next
    case left-eq: True
    show ?thesis
    proof (cases drop i xs = drop i ys)
      case True
      then have xs = ys
        using left-eq i-def j-i 1.premis(3)
        by (metis append-take-drop-id)
      then show ?thesis by simp
    next
      case False
      have drop i xs = drop i ys ∨ hash-map-output-collision s
        by (rule 1.IH[rule-format, OF drop-less rx-created ry-created-i
          right-len right-nonempty-xs right-nonempty-ys right-root])
      then show ?thesis
        using False by simp
    qed
  qed
qed
qed
qed
qed

```

lemma merkle-root-binds-same-length-tables-collision-imp-hash-collision:

```

  assumes bind1: merkle-root-binds-table rt table s
    and bind2: merkle-root-binds-table rt table' s
    and same-len: length table = length table'
    and nonempty: table ≠ []
    and neg: table ≠ table'
  shows hash-map-output-collision s
  proof -
    from bind1 obtain tree where tree:
      created-tree table tree s rt = value tree
    unfolding merkle-root-binds-table-def by blast
    from bind2 obtain tree' where tree':

```

```

    created-tree table' tree' s rt = value tree'
  unfolding merkle-root-binds-table-def by blast
  have nonempty': table' ≠ []
  using same-len nonempty by auto
  have same-root: value tree = value tree'
  using tree tree' by simp
  have table = table' ∨ hash-map-output-collision s
  by (rule protocol-created-tree-same-length-root-eq-or-hash-collision
      [OF tree(1)[unfolded created-tree-def]
        tree'(1)[unfolded created-tree-def]
        same-len nonempty nonempty' same-root])
  then show ?thesis
  using neq by simp
qed

lemma merkle-root-binds-same-length-tables-unique-if-no-hash-collision:
  assumes bind1: merkle-root-binds-table rt table s
    and bind2: merkle-root-binds-table rt table' s
    and same-len: length table = length table'
    and clean: ¬ hash-map-output-collision s
  shows table = table'
proof (cases table = [])
  case True
  then show ?thesis
  using same-len by simp
next
  case False
  show ?thesis
  proof (rule ccontr)
    assume neq: table ≠ table'
    have hash-map-output-collision s
    by (rule merkle-root-binds-same-length-tables-collision-imp-hash-collision
        [OF bind1 bind2 same-len False neq])
    then show False
    using clean by contradiction
  qed
qed

lemma merkle-root-binds-same-length-tables-common-extension-collision:
  assumes bind1: merkle-root-binds-table rt table s1
    and bind2: merkle-root-binds-table rt table' s2
    and ext1: s1 ≤ u
    and ext2: s2 ≤ u
    and same-len: length table = length table'
    and nonempty: table ≠ []
    and neq: table ≠ table'
  shows hash-map-output-collision u
proof -
  have bind1-u: merkle-root-binds-table rt table u

```

```

    by (rule merkle-root-binds-table-mono[OF bind1 ext1])
  have bind2-u: merkle-root-binds-table rt table' u
    by (rule merkle-root-binds-table-mono[OF bind2 ext2])
  show ?thesis
    by (rule merkle-root-binds-same-length-tables-collision-imp-hash-collision
        [OF bind1-u bind2-u same-len nonempty neq])
qed

lemma merkle-root-binds-same-length-tables-common-extension-unique-if-clean:
  assumes clean:  $\neg$  hash-map-output-collision u
    and bind1: merkle-root-binds-table rt table s1
    and bind2: merkle-root-binds-table rt table' s2
    and ext1:  $s1 \leq u$ 
    and ext2:  $s2 \leq u$ 
    and same-len: length table = length table'
    and nonempty: table  $\neq$  []
  shows table = table'
proof (rule ccontr)
  assume table  $\neq$  table'
  then have hash-map-output-collision u
    by (rule merkle-root-binds-same-length-tables-common-extension-collision
        [OF bind1 bind2 ext1 ext2 same-len nonempty])
  then show False
    using clean by contradiction
qed

lemma merkle-root-binds-same-length-tables-compatible-states-unique-if-clean-merge:
  assumes compatible: hash-maps-compatible s1 s2
    and clean:  $\neg$  hash-map-output-collision (hash-state-merge s1 s2)
    and bind1: merkle-root-binds-table rt table s1
    and bind2: merkle-root-binds-table rt table' s2
    and same-len: length table = length table'
    and nonempty: table  $\neq$  []
  shows table = table'
proof -
  have ext1:  $s1 \leq$  hash-state-merge s1 s2
    by (rule hash-state-merge-extends-left-if-compatible[OF compatible])
  have ext2:  $s2 \leq$  hash-state-merge s1 s2
    by (rule hash-state-merge-extends-right)
  show ?thesis
    by (rule merkle-root-binds-same-length-tables-common-extension-unique-if-clean
        [OF clean bind1 bind2 ext1 ext2 same-len nonempty])
qed

lemma merkle-root-binds-same-length-tables-merkle-compatible-unique-if-clean-merge:
  assumes compatible: merkle-hash-maps-compatible s1 s2
    and clean:
       $\neg$  hash-map-output-collision (merkle-hash-state-merge s1 s2)
    and bind1: merkle-root-binds-table rt table s1

```



```

    and bind2: merkle-root-binds-table rt table' s2
    and same-len: length table = length table'
    and nonempty: table ≠ []
  shows table = table'
proof (rule ccontr)
  let ?u = merkle-hash-state-merge s1 s2
  assume neg: table ≠ table'
  have ext1: merkle-hash-extends s1 ?u
    by (rule merkle-hash-state-merge-extends-left-if-compatible[OF compatible])
  have ext2: merkle-hash-extends s2 ?u
    by (rule merkle-hash-state-merge-extends-right)
  have bind1-u: merkle-root-binds-table rt table ?u
    by (rule merkle-root-binds-table-merkle-mono[OF bind1 ext1])
  have bind2-u: merkle-root-binds-table rt table' ?u
    by (rule merkle-root-binds-table-merkle-mono[OF bind2 ext2])
  have hash-map-output-collision ?u
    by (rule merkle-root-binds-same-length-tables-collision-imp-hash-collision
        [OF bind1-u bind2-u same-len nonempty neg])
  then show False
    using clean by contradiction
qed

```

definition *state-after-query-chunks* :: '*f* ⇒ '*f* list list ⇒ nat ⇒ '*f*'
 where
state-after-query-chunks st chunks *i* =
 foldl (λacc chunk. foldl concat acc chunk) st (take *i* chunks)

lemma *concat-append-eq-concat-same-chunk-lengths*:
 assumes *len*: length *xs* = length *ys*
 and *lens*:
 ∧ *i*. *i* < length *xs* ⇒ length (*xs* ! *i*) = length (*ys* ! *i*)
 and *concat-eq*: List.concat *xs* @ trailing = List.concat *ys*
 shows *xs* = *ys* ∧ trailing = []
 using *len lens concat-eq*
proof (induction *xs* arbitrary: *ys* trailing)
 case Nil
 then show ?case
 by (cases *ys*) simp-all
next
 case (Cons *x xs*)
 then obtain *y ys'* where *ys-eq*: *ys* = *y* # *ys'*
proof (cases *ys*)
 case Nil
 then show ?thesis
 using Cons.premis(1) by simp
next
 case (Cons *y ys'*)
 then show ?thesis
 by (rule that)

```

qed
have head-len: length x = length y
  using Cons.premis(2)[of 0] unfolding ys-eq by simp
have append-eq:
  x @ (List.concat xs @ trailing) = y @ List.concat ys'
  using Cons.premis(3) unfolding ys-eq by simp
have head-eq: x = y
  using append-eq head-len
  by (metis append-eq-conv-conj)
have tail-concat-eq:
  List.concat xs @ trailing = List.concat ys'
  using append-eq head-eq by simp
have tail-len: length xs = length ys'
  using Cons.premis(1) unfolding ys-eq by simp
have tail-lens:
   $\bigwedge i. i < \text{length } xs \implies \text{length } (xs ! i) = \text{length } (ys' ! i)$ 
proof -
  fix i
  assume i-bound:  $i < \text{length } xs$ 
  then have Suc i < length (x # xs)
    by simp
  from Cons.premis(2)[OF this] show
    length (xs ! i) = length (ys' ! i)
    unfolding ys-eq by simp
qed
have xs = ys'  $\wedge$  trailing = []
  by (rule Cons.IH[OF tail-len tail-lens tail-concat-eq])
then show ?case
  using head-eq unfolding ys-eq by simp
qed

fun fri-query-transcript-length :: nat  $\Rightarrow$  nat  $\Rightarrow$  nat
  where
    fri-query-transcript-length 0 len = 0
  | fri-query-transcript-length (Suc n) len =
    2 + 2 * floor-log len + fri-query-transcript-length n (len div 2)

definition query-decommitment-transcript-length :: nat  $\Rightarrow$  nat
  where
    query-decommitment-transcript-length idx =
      length (powers-scaled idx) * (1 + floor-log (clength * scale))

definition query-leaf-path-chunk :: nat  $\Rightarrow$  'f  $\Rightarrow$  'f list  $\Rightarrow$  'f list  $\Rightarrow$  bool
  where
    query-leaf-path-chunk len leaf path chunk  $\longleftrightarrow$ 
      length path = floor-log len  $\wedge$ 
      chunk = [leaf] @ path

definition query-decommitment-transcript

```

```

:: nat ⇒ 'f list ⇒ 'f list list ⇒ 'f list ⇒ bool
where
  query-decommitment-transcript idx leaves paths chunk ⇔
    length leaves = length (powers-scaled idx) ∧
    length paths = length (powers-scaled idx) ∧
    chunk = List.concat
      (map (λ(leaf, path).
        [leaf] @ path) (zip leaves paths)) ∧
    (∀ path ∈ set paths. length path = floor-log (clength * scale))

lemma query-decommitment-transcript-length:
  assumes query-decommitment-transcript idx leaves paths chunk
  shows length chunk = query-decommitment-transcript-length idx
proof -
  have leaves-len: length leaves = length (powers-scaled idx)
  using assms unfolding query-decommitment-transcript-def by simp
  have paths-len: length paths = length (powers-scaled idx)
  using assms unfolding query-decommitment-transcript-def by simp
  have leaves-paths-len: length leaves = length paths
  using leaves-len paths-len by simp
  have map2-eq:
    map2 (λleaf path. [leaf] @ path) leaves paths =
    map2 (λleaf path. leaf # path) leaves paths
  proof (induction leaves arbitrary: paths)
    case Nil
    then show ?case
    by (cases paths) simp-all
  next
    case (Cons leaf leaves)
    then show ?case
    by (cases paths) simp-all
  qed
  have chunk-eq:
    chunk =
    List.concat (map2 (λleaf path. leaf # path) leaves paths)
  using assms map2-eq unfolding query-decommitment-transcript-def
  by simp
  have path-lens:
    ∧path. path ∈ set paths ⇒
    length path = floor-log (clength * scale)
  using assms unfolding query-decommitment-transcript-def by blast
  have concat-len:
    length (List.concat (map2 (λleaf path. leaf # path)
      leaves paths)) =
    length leaves * (1 + floor-log (clength * scale))
  using leaves-paths-len path-lens
  proof (induction leaves arbitrary: paths)
    case Nil
    then show ?case

```

```

    by (cases paths) simp-all
next
case (Cons leaf leaves)
then obtain path paths' where paths-eq: paths = path # paths'
  by (cases paths) auto
have len-rest: length leaves = length paths'
  using Cons.prem1(1) unfolding paths-eq by simp
have path-len: length path = floor-log (clength * scale)
  using Cons.prem1(2) unfolding paths-eq by simp
have paths'-lens:
   $\bigwedge p. p \in \text{set } \text{paths}' \implies$ 
   $\text{length } p = \text{floor-log } (\text{clength} * \text{scale})$ 
  using Cons.prem1(2) unfolding paths-eq by simp
have rest:
   $\text{length } (\text{List.concat } (\text{map2 } (\lambda \text{leaf path. leaf} \# \text{path})$ 
     $\text{leaves paths}')) =$ 
   $\text{length leaves} * (1 + \text{floor-log } (\text{clength} * \text{scale}))$ 
  by (rule Cons.IH[OF len-rest paths'-lens])
show ?case
  unfolding paths-eq using path-len rest by simp
qed
have length chunk =
   $\text{length leaves} * (1 + \text{floor-log } (\text{clength} * \text{scale}))$ 
  unfolding chunk-eq by (rule concat-len)
also have ... =
   $\text{length } (\text{powers-scaled idx}) * (1 + \text{floor-log } (\text{clength} * \text{scale}))$ 
  using leaves-len by simp
finally show ?thesis
  unfolding query-decommitment-transcript-length-def .
qed

```

definition *fri-layer-opening-chunk*

```

:: nat  $\Rightarrow$  'f  $\Rightarrow$  'f list  $\Rightarrow$  'f  $\Rightarrow$  'f list  $\Rightarrow$  'f list  $\Rightarrow$  bool
where
  fri-layer-opening-chunk len xp xp-path xn xn-path chunk  $\longleftrightarrow$ 
    length xp-path = floor-log len  $\wedge$ 
    length xn-path = floor-log len  $\wedge$ 
    chunk = [xp] @ xp-path @ [xn] @ xn-path

```

lemma *fri-layer-opening-chunk-length*:

```

assumes fri-layer-opening-chunk len xp xp-path xn xn-path chunk
shows length chunk = 2 + 2 * floor-log len
using assms unfolding fri-layer-opening-chunk-def by simp

```

fun *fri-layers-transcript-length* :: nat $\Rightarrow$ nat $\Rightarrow$ nat

```

where
  fri-layers-transcript-length 0 len = 0
| fri-layers-transcript-length (Suc n) len =
  2 + 2 * floor-log len + fri-layers-transcript-length n (len div 2)

```

```

fun fri-layer-lengths :: nat ⇒ nat ⇒ nat list
  where
    fri-layer-lengths 0 len = []
    | fri-layer-lengths (Suc n) len = len # fri-layer-lengths n (len div 2)

fun fri-layer-indices :: nat ⇒ nat ⇒ nat ⇒ nat list
  where
    fri-layer-indices 0 idx len = []
    | fri-layer-indices (Suc n) idx len =
      idx # fri-layer-indices n (idx mod (len div 2)) (len div 2)

definition fri-sibling-index :: nat ⇒ nat ⇒ nat
  where fri-sibling-index len idx = (idx + len div 2) mod len

definition fri-fold-denominator :: 'f ⇒ nat ⇒ 'f
  where fri-fold-denominator x pw = 2 * x ^ pw

definition fri-fold-value :: 'f ⇒ 'f ⇒ 'f ⇒ 'f ⇒ 'f
  where
    fri-fold-value b xp xn denom =
      (xp + xn) div 2 + b * ((xp - xn) div denom)

lemma length-fri-layer-lengths[simp]:
  length (fri-layer-lengths n len) = n
  by (induction n arbitrary: len) simp-all

lemma length-fri-layer-indices[simp]:
  length (fri-layer-indices n idx len) = n
  by (induction n arbitrary: idx len) simp-all

definition fri-layers-transcript
  :: nat ⇒ nat ⇒ 'f list list ⇒ 'f list ⇒ bool
  where
    fri-layers-transcript n len layer-chunks chunk ⟷
      length layer-chunks = n ∧
      chunk = List.concat layer-chunks ∧
      (∀ (layer-len, layer-chunk) ∈ set (zip (fri-layer-lengths n len) layer-chunks).
        ∃ xp xp-path xn xn-path.
          fri-layer-opening-chunk layer-len xp xp-path xn xn-path layer-chunk)

lemma fri-layers-transcript-length:
  assumes fri-layers-transcript n len layer-chunks chunk
  shows length chunk = fri-layers-transcript-length n len
  using assms
proof (induction n arbitrary: len layer-chunks chunk)
  case 0
  then show ?case
  unfolding fri-layers-transcript-def by simp

```

```

next
  case (Suc n)
  from Suc.premis have len-chunks:
    length layer-chunks = Suc n
  and chunk-eq: chunk = List.concat layer-chunks
  and openings:
     $\forall (layer-len, layer-chunk) \in$ 
      set (zip (fri-layer-lengths (Suc n) len) layer-chunks).
     $\exists xp\ xp-path\ xn\ xn-path.$ 
      fri-layer-opening-chunk layer-len xp xp-path xn xn-path
      layer-chunk
    unfolding fri-layers-transcript-def by simp-all
  obtain first rest where chunks-eq: layer-chunks = first # rest
  using len-chunks by (cases layer-chunks) auto
  from openings obtain xp xp-path xn xn-path where first-open:
    fri-layer-opening-chunk len xp xp-path xn xn-path first
  unfolding chunks-eq by auto
  have rest-layers:
    fri-layers-transcript n (len div 2) rest (List.concat rest)
  unfolding fri-layers-transcript-def
  proof (intro conjI)
    show length rest = n
      using len-chunks unfolding chunks-eq by simp
  next
    show List.concat rest = List.concat rest
      by simp
  next
    show  $\forall (layer-len, layer-chunk) \in$ 
      set (zip (fri-layer-lengths n (len div 2)) rest).
       $\exists xp\ xp-path\ xn\ xn-path.$ 
        fri-layer-opening-chunk layer-len xp xp-path xn xn-path
        layer-chunk
      using openings unfolding chunks-eq by auto
  qed
  have length chunk = length first + length (List.concat rest)
  unfolding chunk-eq chunks-eq by simp
  also have ... =
    (2 + 2 * floor-log len) +
    fri-layers-transcript-length n (len div 2)
  using fri-layer-opening-chunk-length[OF first-open]
    Suc.IH[OF rest-layers] by simp
  finally show ?case
    by simp
qed

lemma fri-layers-transcript-nth-opening:
  assumes layers: fri-layers-transcript n len layer-chunks chunk
  and j-bound: j < n
  obtains xp xp-path xn xn-path

```

```

where fri-layer-opening-chunk (fri-layer-lengths n len ! j)
  xp xp-path xn xn-path (layer-chunks ! j)
proof –
  have len-chunks: length layer-chunks = n
    using layers unfolding fri-layers-transcript-def by simp
  have pair-in:
    (fri-layer-lengths n len ! j, layer-chunks ! j) ∈
      set (zip (fri-layer-lengths n len) layer-chunks)
  proof –
    have j-zip:
      j < length (zip (fri-layer-lengths n len) layer-chunks)
      using j-bound len-chunks by simp
    have zip (fri-layer-lengths n len) layer-chunks ! j =
      (fri-layer-lengths n len ! j, layer-chunks ! j)
      using j-bound len-chunks by simp
    then show ?thesis
      using nth-mem[OF j-zip] by metis
  qed
then obtain xp xp-path xn xn-path where
  fri-layer-opening-chunk (fri-layer-lengths n len ! j)
  xp xp-path xn xn-path (layer-chunks ! j)
  using layers unfolding fri-layers-transcript-def by blast
then show ?thesis
  by (rule that)
qed

definition verifier-query-round-transcript-length
  :: nat ⇒ 'f list ⇒ 'f list ⇒ nat
where
  verifier-query-round-transcript-length idx f-fri-roots composition-fri-roots =
    query-decommitment-transcript-length idx +
    fri-layers-transcript-length (length f-fri-roots) (clength * scale) +
    fri-layers-transcript-length (length composition-fri-roots) (clength * scale)

definition verifier-query-round-chunk
  :: nat ⇒ 'f list ⇒ 'f list ⇒ 'f list ⇒ bool
where
  verifier-query-round-chunk idx f-fri-roots composition-fri-roots chunk ⟷
    (∃ query-chunk trace-fri-chunk composition-fri-chunk
      leaves paths trace-layer-chunks composition-layer-chunks.
      chunk = query-chunk @ trace-fri-chunk @ composition-fri-chunk ∧
      query-decommitment-transcript idx leaves paths query-chunk ∧
      fri-layers-transcript (length f-fri-roots) (clength * scale)
      trace-layer-chunks trace-fri-chunk ∧
      fri-layers-transcript (length composition-fri-roots) (clength * scale)
      composition-layer-chunks composition-fri-chunk)

lemma verifier-query-round-chunk-length:
  assumes verifier-query-round-chunk idx f-fri-roots composition-fri-roots chunk

```

shows
length chunk =
verifier-query-round-transcript-length idx f-fri-roots
composition-fri-roots

proof –
from *assms* **obtain** *query-chunk trace-fri-chunk composition-fri-chunk*
leaves paths trace-layer-chunks composition-layer-chunks **where**
chunk-eq: chunk = query-chunk @ trace-fri-chunk @ composition-fri-chunk
and *query:*
query-decommitment-transcript idx leaves paths query-chunk
and *trace:*
*fri-layers-transcript (length f-fri-roots) (clength * scale)*
trace-layer-chunks trace-fri-chunk
and *composition:*
*fri-layers-transcript (length composition-fri-roots) (clength * scale)*
composition-layer-chunks composition-fri-chunk
unfolding *verifier-query-round-chunk-def* **by** *blast*
show *?thesis*
unfolding *chunk-eq verifier-query-round-transcript-length-def*
using *query-decommitment-transcript-length[OF query]*
fri-layers-transcript-length[OF trace]
fri-layers-transcript-length[OF composition]
by *simp*
qed

definition *query-round-fri-layer-transcripts*
 $:: \text{nat} \Rightarrow 'f \text{ list} \Rightarrow 'f \text{ list} \Rightarrow 'f \text{ list} \Rightarrow$
 $'f \text{ list list} \Rightarrow 'f \text{ list list} \Rightarrow \text{bool}$

where
query-round-fri-layer-transcripts idx trace-roots composition-roots
chunk trace-layer-chunks composition-layer-chunks $\longleftrightarrow$
 $(\exists \text{ query-chunk trace-fri-chunk composition-fri-chunk leaves paths.}$
 $\text{ chunk = query-chunk @ trace-fri-chunk @ composition-fri-chunk } \wedge$
 $\text{ query-decommitment-transcript idx leaves paths query-chunk } \wedge$
 $\text{ fri-layers-transcript (length trace-roots) (clength * scale)}$
 $\text{ trace-layer-chunks trace-fri-chunk } \wedge$
 $\text{ fri-layers-transcript (length composition-roots) (clength * scale)}$
 $\text{ composition-layer-chunks composition-fri-chunk})$

lemma *verifier-query-round-chunk-fri-layer-transcriptsE:*
assumes *verifier-query-round-chunk idx trace-roots composition-roots chunk*
obtains *trace-layer-chunks composition-layer-chunks*
where *query-round-fri-layer-transcripts idx trace-roots composition-roots*
chunk trace-layer-chunks composition-layer-chunks
using *assms*
unfolding *verifier-query-round-chunk-def query-round-fri-layer-transcripts-def*
by *blast*

definition *verifier-query-indices-derived*

$:: ('f, 'a) \text{ protocol-channel-scheme} \Rightarrow$
 $(\text{unit list} \times ('f, 'a) \text{ protocol-channel-scheme}) \text{ option} \Rightarrow$
 $'f \Rightarrow 'f \text{ list} \Rightarrow 'f \text{ list} \Rightarrow 'f \text{ list} \Rightarrow \text{nat list} \Rightarrow \text{bool}$
where
 $\text{verifier-query-indices-derived } s \text{ out query-start-state rest}$
 $f\text{-fri-roots composition-fri-roots query-idxs} \longleftrightarrow$
 $(\exists \text{result final-state raw-idxs query-chunks trailing.}$
 $\text{out} = \text{Some (result, final-state)} \wedge$
 $\text{length raw-idxs} = \text{rounds} \wedge$
 $\text{query-idxs} = \text{map } (\lambda \text{raw. index (to-nat raw)}) \text{ raw-idxs} \wedge$
 $\text{length query-chunks} = \text{rounds} \wedge$
 $\text{List.concat query-chunks @ trailing} = \text{rest} \wedge$
 $(\forall i < \text{rounds.}$
 $\text{verifier-query-round-chunk (query-idxs ! i)}$
 $f\text{-fri-roots composition-fri-roots (query-chunks ! i)}) \wedge$
 $(\forall i < \text{rounds.}$
 $\text{fmlookup (HashMap final-state)}$
 $(\text{QueryIndexChallenge (PQueryCounter s + i)}$
 $(\text{state-after-query-chunks query-start-state query-chunks i})) =$
 $\text{Some (raw-idxs ! i)}) \wedge$
 $(\forall \text{idx} \in \text{set query-idxs. idx} < \text{clength} * \text{scale}))$

definition *accepted-transcript-shape*

$:: ('f, 'a) \text{ protocol-channel-scheme} \Rightarrow$
 $(\text{unit list} \times ('f, 'a) \text{ protocol-channel-scheme}) \text{ option} \Rightarrow$
 $'f \text{ list} \Rightarrow \text{nat list} \Rightarrow \text{bool}$

where

$\text{accepted-transcript-shape } s \text{ out alphas query-idxs} \longleftrightarrow$
 $(\exists \text{result final-state fr f-fri-roots f-final dg composition-fri-roots final rest.}$
 $\text{out} = \text{Some (result, final-state)} \wedge$
 $\text{verifier-header-transcript } s \text{ fr f-fri-roots f-final alphas dg}$
 $\text{composition-fri-roots final rest} \wedge$
 $\text{verifier-query-indices-derived } s \text{ out}$
 $(\text{verifier-header-state } s \text{ fr f-fri-roots f-final alphas dg}$
 $\text{composition-fri-roots final})$
 $\text{rest f-fri-roots composition-fri-roots query-idxs})$

lemma *accepted-transcript-shape-query-chunksE*:

assumes *shape*: $\text{accepted-transcript-shape } s \text{ out alphas query-idxs}$

obtains $\text{result final-state fr f-fri-roots f-final dg}$

$\text{composition-fri-roots final rest raw-idxs query-chunks trailing}$

where

$\text{out} = \text{Some (result, final-state)}$
 $\text{verifier-header-transcript } s \text{ fr f-fri-roots f-final alphas dg}$
 $\text{composition-fri-roots final rest}$
 $\text{length raw-idxs} = \text{rounds}$
 $\text{query-idxs} = \text{map } (\lambda \text{raw. index (to-nat raw)}) \text{ raw-idxs}$
 $\text{length query-chunks} = \text{rounds}$
 $\text{List.concat query-chunks @ trailing} = \text{rest}$

```

 $\bigwedge i. i < \text{rounds} \implies$ 
  verifier-query-round-chunk (query-idxs ! i)
  f-fri-roots composition-fri-roots (query-chunks ! i)
 $\bigwedge i. i < \text{rounds} \implies$ 
  fmlookup (HashMap final-state)
    (QueryIndexChallenge (PQueryCounter s + i)
      (state-after-query-chunks
        (verifier-header-state s fr f-fri-roots f-final alphas dg
          composition-fri-roots final)
        query-chunks i)) =
    Some (raw-idxs ! i)
 $\forall idx \in \text{set query-idxs}. idx < \text{clength} * \text{scale}$ 
proof -
from shape obtain result final-state fr f-fri-roots f-final dg
  composition-fri-roots final rest where
  out-eq: out = Some (result, final-state)
and header:
  verifier-header-transcript s fr f-fri-roots f-final alphas dg
  composition-fri-roots final rest
and query:
  verifier-query-indices-derived s out
  (verifier-header-state s fr f-fri-roots f-final alphas dg
    composition-fri-roots final)
  rest f-fri-roots composition-fri-roots query-idxs
unfolding accepted-transcript-shape-def by blast
from query obtain result' final-state' raw-idxs query-chunks trailing where
  out-eq': out = Some (result', final-state')
and len-raw: length raw-idxs = rounds
and query-idxs-eq:
  query-idxs = map ( $\lambda \text{raw}. \text{index } (\text{to-nat } \text{raw})$ ) raw-idxs
and len-chunks: length query-chunks = rounds
and concat-chunks: List.concat query-chunks @ trailing = rest
and chunk-shape:
 $\bigwedge i. i < \text{rounds} \implies$ 
  verifier-query-round-chunk (query-idxs ! i)
  f-fri-roots composition-fri-roots (query-chunks ! i)
and lookup:
 $\bigwedge i. i < \text{rounds} \implies$ 
  fmlookup (HashMap final-state')
    (QueryIndexChallenge (PQueryCounter s + i)
      (state-after-query-chunks
        (verifier-header-state s fr f-fri-roots f-final alphas dg
          composition-fri-roots final)
        query-chunks i)) =
    Some (raw-idxs ! i)
and idx-bound:
 $\forall idx \in \text{set query-idxs}. idx < \text{clength} * \text{scale}$ 
unfolding verifier-query-indices-derived-def by blast
have final-state-eq: final-state' = final-state

```

```

    using out-eq out-eq' by simp
  show ?thesis
  by (rule that[OF out-eq header len-raw query-idxs-eq len-chunks
    concat-chunks chunk-shape])
    (use lookup idx-bound final-state-eq in simp-all)
qed

lemma accepted-transcript-shape-query-chunk-lengthsE:
  assumes shape: accepted-transcript-shape s out alphas query-idxs
  obtains result final-state fr f-fri-roots f-final dg
    composition-fri-roots final rest raw-idxs query-chunks trailing
  where
    out = Some (result, final-state)
    verifier-header-transcript s fr f-fri-roots f-final alphas dg
      composition-fri-roots final rest
    length raw-idxs = rounds
    query-idxs = map (λraw. index (to-nat raw)) raw-idxs
    length query-chunks = rounds
    List.concat query-chunks @ trailing = rest
    ∧ i. i < rounds ⇒
      length (query-chunks ! i) =
        verifier-query-round-transcript-length (query-idxs ! i)
        f-fri-roots composition-fri-roots
    ∧ i. i < rounds ⇒
      fmlookup (HashMap final-state)
        (QueryIndexChallenge (PQueryCounter s + i)
          (state-after-query-chunks
            (verifier-header-state s fr f-fri-roots f-final alphas dg
              composition-fri-roots final)
            query-chunks i)) =
        Some (raw-idxs ! i)
    ∀ idx ∈ set query-idxs. idx < clength * scale
proof -
  from accepted-transcript-shape-query-chunksE[OF shape]
  obtain result final-state fr f-fri-roots f-final dg
    composition-fri-roots final rest raw-idxs query-chunks trailing where
    out-eq: out = Some (result, final-state)
  and header:
    verifier-header-transcript s fr f-fri-roots f-final alphas dg
      composition-fri-roots final rest
  and len-raw: length raw-idxs = rounds
  and query-idxs-eq:
    query-idxs = map (λraw. index (to-nat raw)) raw-idxs
  and len-chunks: length query-chunks = rounds
  and concat-chunks: List.concat query-chunks @ trailing = rest
  and chunk-shape:
    ∧ i. i < rounds ⇒
      verifier-query-round-chunk (query-idxs ! i)
        f-fri-roots composition-fri-roots (query-chunks ! i)

```

and *lookup*:
 $\bigwedge i. i < \text{rounds} \implies$
 $\text{fmlookup } (\text{HashMap } \text{final-state})$
 $(\text{QueryIndexChallenge } (P\text{QueryCounter } s + i)$
 $(\text{state-after-query-chunks}$
 $(\text{verifier-header-state } s \text{ fr } f\text{-fri-roots } f\text{-final } \text{alphas } dg$
 $\text{composition-fri-roots } \text{final})$
 $\text{query-chunks } i)) =$
 $\text{Some } (\text{raw-idxs } ! i)$
and *idx-bound*:
 $\forall \text{idx} \in \text{set query-idxs}. \text{idx} < \text{clength} * \text{scale}$
by *blast*
have *chunk-len*:
 $\bigwedge i. i < \text{rounds} \implies$
 $\text{length } (\text{query-chunks } ! i) =$
 $\text{verifier-query-round-transcript-length } (\text{query-idxs } ! i)$
 $f\text{-fri-roots } \text{composition-fri-roots}$
using *verifier-query-round-chunk-length*[*OF chunk-shape*] **by** *blast*
show *?thesis*
by (*rule that*[*OF out-eq header len-raw query-idxs-eq len-chunks*
concat-chunks chunk-len lookup idx-bound])
qed

definition *accepted-with-tables*

$:: (f, 'a) \text{protocol-channel-scheme} \implies$
 $(\text{unit list} \times (f, 'a) \text{protocol-channel-scheme}) \text{option} \implies$
 $'f \text{list} \implies 'f \text{list} \implies 'f \text{list} \implies \text{nat list} \implies \text{bool}$

where

$\text{accepted-with-tables } s \text{ out trace-table composition-table alphas query-idxs} \longleftrightarrow$
 $\text{accepted-transcript-shape } s \text{ out alphas query-idxs} \wedge$
 $(\forall \text{idx} \in \text{set query-idxs}. \text{query-consistent-at trace-table composition-table alphas idx}) \wedge$
 $\text{length trace-table} = \text{clength} * \text{scale} \wedge$
 $\text{length composition-table} = \text{clength} * \text{scale} \wedge$
 $\text{length alphas} = \text{length spec}$

definition *accepted-with-bound-tables*

$:: (f, 'a) \text{protocol-channel-scheme} \implies$
 $(\text{unit list} \times (f, 'a) \text{protocol-channel-scheme}) \text{option} \implies$
 $'f \text{list} \implies 'f \text{list} \implies 'f \text{list} \implies \text{nat list} \implies \text{bool}$

where

$\text{accepted-with-bound-tables } s \text{ out trace-table composition-table alphas query-idxs} \longleftrightarrow$
 $\text{accepted-with-tables } s \text{ out trace-table composition-table alphas query-idxs} \wedge$
 $(\exists \text{result final-state fr } f\text{-fri-roots } f\text{-final } dg \text{ composition-fri-roots final rest.}$
 $\text{out} = \text{Some } (\text{result}, \text{final-state}) \wedge$
 $\text{verifier-header-transcript } s \text{ fr } f\text{-fri-roots } f\text{-final } \text{alphas } dg$
 $\text{composition-fri-roots final rest} \wedge$
 $\text{merkle-root-binds-table fr trace-table final-state} \wedge$
 $\text{composition-fri-roots} \neq [] \wedge$

merkle-root-binds-table (hd composition-fri-roots) composition-table final-state)

definition *alpha-list-set-hit*

$:: ('f, 'a) \text{ protocol-channel-scheme} \Rightarrow 'f \text{ list set} \Rightarrow$
 $(\text{unit list} \times ('f, 'a) \text{ protocol-channel-scheme}) \text{ option} \Rightarrow \text{bool}$

where

alpha-list-set-hit s B out $\longleftrightarrow$
 $(\exists \text{ as query-idxs.}$
 $\text{accepted-transcript-shape s out as query-idxs} \wedge \text{as} \in B)$

definition *alpha-header-list-set-hit*

$:: ('f, 'a) \text{ protocol-channel-scheme} \Rightarrow$
 $('f \Rightarrow 'f \text{ list} \Rightarrow 'f \Rightarrow 'f \text{ list set}) \Rightarrow$
 $(\text{unit list} \times ('f, 'a) \text{ protocol-channel-scheme}) \text{ option} \Rightarrow \text{bool}$

where

alpha-header-list-set-hit s B out $\longleftrightarrow$
 $(\exists \text{ as query-idxs result final-state fr f-fri-roots f-final dg}$
 $\text{composition-fri-roots final rest.}$
 $\text{accepted-transcript-shape s out as query-idxs} \wedge$
 $\text{out} = \text{Some (result, final-state)} \wedge$
 $\text{verifier-header-transcript s fr f-fri-roots f-final as dg}$
 $\text{composition-fri-roots final rest} \wedge$
 $\text{as} \in B \text{ fr f-fri-roots f-final})$

end

end

theory *Soundness-Core*

imports *Soundness-Core-Events*

begin

Compatibility umbrella for core soundness definitions and lemmas.

end

theory *Soundness-Merkle*

imports

Soundness-Core

begin

context *soundness*

begin

A successful verifier execution only checks the sampled authentication paths. Turning those local checks into full trace and composition tables bound to the initial committed roots is a Merkle binding/collision-resistance property. The current random-oracle hash-map model records enough information to replay checked paths, but it does not

by itself imply existence or uniqueness of a complete maliciously committed table. We therefore expose the missing part as a bad event over accepted verifier executions. Binding for intermediate FRI layers is not hidden here; it is part of the FRI bad-challenge-set reductions introduced below.

definition *initial-merkle-binding-bad*

$:: ('f, 'a) \text{ protocol-channel-scheme} \Rightarrow$
 $(\text{unit list} \times ('f, 'a) \text{ protocol-channel-scheme}) \text{ option} \Rightarrow \text{bool}$

where

$\text{initial-merkle-binding-bad } s \text{ out} \longleftrightarrow$
 $(\exists \text{ alphas query-idxs.}$
 $\text{accepted-transcript-shape } s \text{ out alphas query-idxs} \wedge$
 $\neg (\exists \text{ trace-table composition-table.}$
 $\text{accepted-with-bound-tables } s \text{ out trace-table composition-table}$
 $\text{alphas query-idxs}))$

definition *initial-merkle-binding-no-bad* $:: ('f, 'a) \text{ protocol-channel-scheme} \Rightarrow \text{bool}$

where

$\text{initial-merkle-binding-no-bad } s \longleftrightarrow$
 $(\forall \text{ out} \in \text{set-dist } (\text{execute verify-monad } s).$
 $\neg \text{initial-merkle-binding-bad } s \text{ out})$

definition *initial-merkle-binding-collision-bad*

$:: ('f, 'a) \text{ protocol-channel-scheme} \Rightarrow$
 $(\text{unit list} \times ('f, 'a) \text{ protocol-channel-scheme}) \text{ option} \Rightarrow \text{bool}$

where

$\text{initial-merkle-binding-collision-bad } s \text{ out} \longleftrightarrow$
 $\text{accepted out} \wedge$
 $(\exists \text{ result final-state rt.}$
 $\text{out} = \text{Some (result, final-state)} \wedge$
 $\text{merkle-root-binding-collision rt final-state})$

definition *merkle-root-binding-collision-bad*

$:: ('f, 'a) \text{ protocol-channel-scheme} \Rightarrow$
 $(\text{unit list} \times ('f, 'a) \text{ protocol-channel-scheme}) \text{ option} \Rightarrow \text{bool}$

where

$\text{merkle-root-binding-collision-bad } s \text{ out} \longleftrightarrow$
 $\text{accepted out} \wedge$
 $(\exists \text{ result final-state rt.}$
 $\text{out} = \text{Some (result, final-state)} \wedge$
 $\text{merkle-root-binding-collision rt final-state})$

definition *hash-map-output-collision-bad*

$:: ('f, 'a) \text{ protocol-channel-scheme} \Rightarrow$
 $(\text{unit list} \times ('f, 'a) \text{ protocol-channel-scheme}) \text{ option} \Rightarrow \text{bool}$

where

$\text{hash-map-output-collision-bad } s \text{ out} \longleftrightarrow$
 $\text{accepted out} \wedge$
 $(\exists \text{ result final-state.}$
 $\text{out} = \text{Some (result, final-state)} \wedge$

hash-map-output-collision final-state)

lemma *initial-merkle-binding-collision-bad-eq-root-collision-bad:*

initial-merkle-binding-collision-bad s out =
merkle-root-binding-collision-bad s out

unfolding *initial-merkle-binding-collision-bad-def*

merkle-root-binding-collision-bad-def

by *simp*

lemma *merkle-root-binding-collision-bad-imp-accepted:*

assumes *merkle-root-binding-collision-bad s out*

shows *accepted out*

using *assms unfolding merkle-root-binding-collision-bad-def by simp*

lemma *hash-map-output-collision-bad-imp-accepted:*

assumes *hash-map-output-collision-bad s out*

shows *accepted out*

using *assms unfolding hash-map-output-collision-bad-def by simp*

lemma *merkle-root-binding-collision-bad-mono-accepted:*

wp-event verify-monad (merkle-root-binding-collision-bad s) s ≤

wp-event verify-monad accepted s

by *(rule wp-event-mono)*

(simp add: merkle-root-binding-collision-bad-imp-accepted)

lemma *hash-map-output-collision-bad-mono-accepted:*

wp-event verify-monad (hash-map-output-collision-bad s) s ≤

wp-event verify-monad accepted s

by *(rule wp-event-mono)*

(simp add: hash-map-output-collision-bad-imp-accepted)

lemma *accepted-inconsistent-merkle-bindings-imp-collision-bad:*

assumes *out: out = Some (result, final-state)*

and *bind1: merkle-root-binds-table rt table final-state*

and *bind2: merkle-root-binds-table rt table' final-state*

and *neq: table ≠ table'*

shows *initial-merkle-binding-collision-bad s out*

proof *–*

have *collision: merkle-root-binding-collision rt final-state*

by *(rule merkle-root-binding-collisionI[OF bind1 bind2 neq])*

show *?thesis*

proof *(unfold initial-merkle-binding-collision-bad-def accepted-def, intro conjI)*

show *¬ Option.is-none out*

using *out by simp*

show *∃ result final-state rt.*

out = Some (result, final-state) ∧

merkle-root-binding-collision rt final-state

using *out collision by blast*

qed

qed

lemma *accepted-inconsistent-merkle-bindings-imp-root-collision-bad*:
 assumes *out*: *out* = *Some* (*result*, *final-state*)
 and *bind1*: *merkle-root-binds-table* *rt* *table* *final-state*
 and *bind2*: *merkle-root-binds-table* *rt* *table'* *final-state*
 and *neg*: *table* $\neq$ *table'*
 shows *merkle-root-binding-collision-bad* *s* *out*
 using *accepted-inconsistent-merkle-bindings-imp-collision-bad*
 [*OF out bind1 bind2 neg*]
 unfolding *initial-merkle-binding-collision-bad-eq-root-collision-bad* .

lemma *hash-map-output-collision-badI*:
 assumes *out*: *out* = *Some* (*result*, *final-state*)
 and *neg*: *x* $\neq$ *y*
 and *x-lookup*: *fmlookup* (*HashMap* *final-state*) *x* = *Some* *z*
 and *y-lookup*: *fmlookup* (*HashMap* *final-state*) *y* = *Some* *z*
 shows *hash-map-output-collision-bad* *s* *out*
proof –
 have *collision*: *hash-map-output-collision* *final-state*
 by (*rule* *hash-map-output-collisionI* [*OF neg x-lookup y-lookup*])
 show ?*thesis*
 unfolding *hash-map-output-collision-bad-def* *accepted-def*
 using *out collision* **by** *auto*

qed

lemma *accepted-inconsistent-same-length-merkle-bindings-imp-hash-collision-bad*:
 assumes *out*: *out* = *Some* (*result*, *final-state*)
 and *bind1*: *merkle-root-binds-table* *rt* *table* *final-state*
 and *bind2*: *merkle-root-binds-table* *rt* *table'* *final-state*
 and *same-len*: *length* *table* = *length* *table'*
 and *nonempty*: *table* $\neq$ []
 and *neg*: *table* $\neq$ *table'*
 shows *hash-map-output-collision-bad* *s* *out*
proof –
 have *collision*: *hash-map-output-collision* *final-state*
 by (*rule* *merkle-root-binds-same-length-tables-collision-imp-hash-collision*
 [*OF bind1 bind2 same-len nonempty neg*])
 show ?*thesis*
 unfolding *hash-map-output-collision-bad-def* *accepted-def*
 using *out collision* **by** *auto*

qed

lemma *initial-merkle-binding-collision-bad-no-collisionD*:
 assumes $\neg$ *initial-merkle-binding-collision-bad* *s* (*Some* (*result*, *final-state*))
 shows $\neg$ *merkle-root-binding-collision* *rt* *final-state*
 using *assms* **unfolding** *initial-merkle-binding-collision-bad-def* *accepted-def*
 by *auto*

lemma *accepted-with-tables-imp-accepted:*
assumes *accepted-with-tables s out trace-table composition-table alphas query-idxs*
shows *accepted out*
using *assms unfolding accepted-with-tables-def accepted-transcript-shape-def*
verifier-query-indices-derived-def
accepted-def
by *auto*

lemma *accepted-transcript-shape-imp-accepted:*
assumes *accepted-transcript-shape s out alphas query-idxs*
shows *accepted out*
using *assms unfolding accepted-transcript-shape-def verifier-query-indices-derived-def*
accepted-def
by *auto*

lemma *accepted-with-tables-imp-accepted-transcript-shape:*
assumes *accepted-with-tables s out trace-table composition-table alphas query-idxs*
shows *accepted-transcript-shape s out alphas query-idxs*
using *assms unfolding accepted-with-tables-def* **by** *simp*

lemma *accepted-with-tables-shapes:*
assumes *accepted-with-tables s out trace-table composition-table alphas query-idxs*
shows *length trace-table = clength * scale*
and *length composition-table = clength * scale*
and *length alphas = length spec*
and *length query-idxs = rounds*
and $\bigwedge idx. idx \in \text{set query-idxs} \implies idx < \text{clength} * \text{scale}$
using *assms*
unfolding *accepted-with-tables-def accepted-transcript-shape-def*
verifier-query-indices-derived-def
by *auto*

lemma *accepted-transcript-shape-query-shapes:*
assumes *accepted-transcript-shape s out alphas query-idxs*
shows *length query-idxs = rounds*
and $\bigwedge idx. idx \in \text{set query-idxs} \implies idx < \text{clength} * \text{scale}$
using *assms*
unfolding *accepted-transcript-shape-def verifier-query-indices-derived-def*
by *auto*

lemma *verifier-query-indices-derived-query-sample-space:*
assumes *verifier-query-indices-derived s out query-start-state rest*
f-fri-roots composition-fri-roots query-idxs
shows $\bigwedge idx. idx \in \text{set query-idxs} \implies idx \in \text{query-sample-space}$
using *assms index-less-query-sample-space*
unfolding *verifier-query-indices-derived-def query-sample-space-def*
by *auto*

lemma *accepted-transcript-shape-query-sample-space:*

assumes *accepted-transcript-shape s out alphas query-idxs*
shows $\bigwedge idx. idx \in \text{set query-idxs} \implies idx \in \text{query-sample-space}$
proof –
fix *idx*
assume *idx-in: idx ∈ set query-idxs*
obtain *result final-state fr f-fri-roots f-final dg composition-fri-roots final rest*
where *derived:*
verifier-query-indices-derived s out
(verifier-header-state s fr f-fri-roots f-final alphas dg
composition-fri-roots final)
rest f-fri-roots composition-fri-roots query-idxs
using *assms unfolding accepted-transcript-shape-def by blast*
show *idx ∈ query-sample-space*
by *(rule verifier-query-indices-derived-query-sample-space[OF derived idx-in])*
qed

lemma *accepted-with-tables-query-sample-space:*
assumes *accepted-with-tables s out trace-table composition-table alphas query-idxs*
shows $\bigwedge idx. idx \in \text{set query-idxs} \implies idx \in \text{query-sample-space}$
using *accepted-transcript-shape-query-sample-space*
[OF accepted-with-tables-imp-accepted-transcript-shape[OF assms]]
by *blast*

lemma *accepted-with-bound-tables-query-sample-space:*
assumes *accepted-with-bound-tables s out trace-table composition-table alphas query-idxs*
shows $\bigwedge idx. idx \in \text{set query-idxs} \implies idx \in \text{query-sample-space}$
proof –
have *tables: accepted-with-tables s out trace-table composition-table alphas query-idxs*
using *assms unfolding accepted-with-bound-tables-def by simp*
show $\bigwedge idx. idx \in \text{set query-idxs} \implies idx \in \text{query-sample-space}$
by *(rule accepted-with-tables-query-sample-space[OF tables])*
qed

lemma *accepted-with-tables-authenticated-roots:*
assumes *accepted-with-bound-tables s out trace-table composition-table alphas query-idxs*
obtains *result final-state fr composition-root*
where *out = Some (result, final-state)*
and *merkle-root-binds-table fr trace-table final-state*
and *merkle-root-binds-table composition-root composition-table final-state*
using *assms unfolding accepted-with-bound-tables-def by auto*

lemma *accepted-with-bound-tables-imp-accepted-with-tables:*
assumes *accepted-with-bound-tables s out trace-table composition-table alphas query-idxs*
shows *accepted-with-tables s out trace-table composition-table alphas query-idxs*
using *assms unfolding accepted-with-bound-tables-def by simp*

lemma *accepted-with-bound-tables-shapes:*
assumes *accepted-with-bound-tables s out trace-table composition-table alphas query-idxs*
shows *length trace-table = clength * scale*

and *length composition-table* = *clength* * *scale*
and *length alphas* = *length spec*
and *length query-idxs* = *rounds*
and $\bigwedge \text{idx. idx} \in \text{set query-idxs} \implies \text{idx} < \text{clength} * \text{scale}$
using *accepted-with-tables-shapes*
 $[OF \text{ accepted-with-bound-tables-imp-accepted-with-tables}[OF \text{ assms}]]$
by *blast*

lemma *accepted-with-bound-tables-imp-accepted*:
assumes *accepted-with-bound-tables s out trace-table composition-table alphas query-idxs*
shows *accepted out*
using *accepted-with-bound-tables-imp-accepted-with-tables*[*OF assms*]
by (*rule accepted-with-tables-imp-accepted*)

definition *supported-hash-output-collision-possible*
 $:: (f, 'a) \text{ protocol-channel-scheme} \Rightarrow \text{bool}$
where
 $\text{supported-hash-output-collision-possible } s \longleftrightarrow$
 $(\exists \text{ out} \in \text{set-dist } (\text{execute verify-monad } s).$
 $\text{hash-map-output-collision-bad } s \text{ out})$

lemma *accepted-with-bound-tables-hash-collision-imp-supported-hash-output-collision-possible*:
assumes *outcome*:
 $\text{Some } (\text{result}, \text{final-state}) \in \text{set-dist } (\text{execute verify-monad } s)$
and *bound*:
 $\text{accepted-with-bound-tables } s (\text{Some } (\text{result}, \text{final-state}))$
 $\text{trace-table composition-table as query-idxs}$
and *collision*: *hash-map-output-collision final-state*
shows *supported-hash-output-collision-possible s*
proof –
have *accepted*: *accepted* (*Some* (*result*, *final-state*))
by (*rule accepted-with-bound-tables-imp-accepted*[*OF bound*])
have *bad*:
 $\text{hash-map-output-collision-bad } s (\text{Some } (\text{result}, \text{final-state}))$
unfolding *hash-map-output-collision-bad-def*
using *accepted collision* **by** *blast*
show *?thesis*
unfolding *supported-hash-output-collision-possible-def*
using *outcome bad* **by** *blast*
qed

definition *alpha-bad-set-hit*
 $:: (f, 'a) \text{ protocol-channel-scheme} \Rightarrow$
 $(f \text{ list} \Rightarrow f \text{ list set}) \Rightarrow$
 $(\text{unit list} \times (f, 'a) \text{ protocol-channel-scheme}) \text{ option} \Rightarrow \text{bool}$
where
 $\text{alpha-bad-set-hit } s \text{ bad-sets out} \longleftrightarrow$
 $(\exists \text{ trace-table composition-table as query-idxs.}$
 $\text{accepted-with-bound-tables } s \text{ out trace-table composition-table as query-idxs} \wedge$

$as \in \text{bad-sets trace-table})$

lemma *alpha-bad-set-hit-imp-alpha-list-set-hit:*

assumes *hit: alpha-bad-set-hit s bad-sets out*
and *envelope: $\bigwedge \text{trace-table. bad-sets trace-table} \subseteq B$*
shows *alpha-list-set-hit s B out*

proof –

from *hit* **obtain** *trace-table composition-table as query-idxs* **where**
bound:

accepted-with-bound-tables s out trace-table composition-table as query-idxs

and *as-bad: $as \in \text{bad-sets trace-table}$*

unfolding *alpha-bad-set-hit-def* **by** *blast*

have *shape:*

accepted-transcript-shape s out as query-idxs

using *bound* **unfolding** *accepted-with-bound-tables-def*

accepted-with-tables-def **by** *simp*

have *as $\in B$*

using *as-bad envelope[of trace-table]* **by** *auto*

then show *?thesis*

unfolding *alpha-list-set-hit-def* **using** *shape* **by** *blast*

qed

definition *alpha-header-trace-table-candidates*

$:: (f, 'a) \text{ protocol-channel-scheme} \Rightarrow$

$f \Rightarrow f \text{ list} \Rightarrow f \Rightarrow f \text{ list set}$

where

alpha-header-trace-table-candidates s fr f-fri-roots f-final =
 $\{\text{trace-table.}$

$\exists \text{ out composition-table as query-idxs dg composition-fri-roots final rest.}$

$\text{accepted-with-bound-tables s out trace-table composition-table}$
 $\text{as query-idxs} \wedge$

$\text{verifier-header-transcript s fr f-fri-roots f-final as dg}$
 $\text{composition-fri-roots final rest}\}$

definition *alpha-header-union-bad-sets*

$:: (f, 'a) \text{ protocol-channel-scheme} \Rightarrow$

$(f \text{ list} \Rightarrow f \text{ list set}) \Rightarrow$

$f \Rightarrow f \text{ list} \Rightarrow f \Rightarrow f \text{ list set}$

where

alpha-header-union-bad-sets s bad-sets fr f-fri-roots f-final =
 $\{as \in \text{alpha-space.}$

$\exists \text{ trace-table} \in$

$\text{alpha-header-trace-table-candidates s fr f-fri-roots f-final.}$

$as \in \text{bad-sets trace-table}\}$

definition *alpha-header-supported-trace-table-candidates*

$:: (f, 'a) \text{ protocol-channel-scheme} \Rightarrow$

$f \Rightarrow f \text{ list} \Rightarrow f \Rightarrow f \text{ list set}$

where

$\text{alpha-header-supported-trace-table-candidates } s \text{ fr } f\text{-fri-roots } f\text{-final} =$
 $\{ \text{trace-table.}$
 $\quad \exists \text{ out composition-table as query-idxs dg composition-fri-roots final rest.}$
 $\quad \text{out} \in \text{set-dist (execute verify-monad } s) \wedge$
 $\quad \text{accepted-with-bound-tables } s \text{ out trace-table composition-table}$
 $\quad \text{as query-idxs} \wedge$
 $\quad \text{verifier-header-transcript } s \text{ fr } f\text{-fri-roots } f\text{-final as dg}$
 $\quad \text{composition-fri-roots final rest} \}$

definition $\text{alpha-header-supported-union-bad-sets}$

$:: ('f, 'a) \text{ protocol-channel-scheme} \Rightarrow$
 $('f \text{ list} \Rightarrow 'f \text{ list set}) \Rightarrow$
 $'f \Rightarrow 'f \text{ list} \Rightarrow 'f \Rightarrow 'f \text{ list set}$

where

$\text{alpha-header-supported-union-bad-sets } s \text{ bad-sets fr } f\text{-fri-roots } f\text{-final} =$
 $\{ \text{as} \in \text{alpha-space.}$
 $\quad \exists \text{ trace-table} \in$
 $\quad \text{alpha-header-supported-trace-table-candidates } s \text{ fr } f\text{-fri-roots}$
 $\quad f\text{-final.}$
 $\quad \text{as} \in \text{bad-sets trace-table} \}$

lemma $\text{alpha-header-union-bad-sets-subset-alpha-space:}$

$\text{alpha-header-union-bad-sets } s \text{ bad-sets fr } f\text{-fri-roots } f\text{-final} \subseteq$
 alpha-space

unfolding $\text{alpha-header-union-bad-sets-def}$ **by** auto

lemma $\text{alpha-header-supported-union-bad-sets-subset-alpha-space:}$

$\text{alpha-header-supported-union-bad-sets } s \text{ bad-sets fr } f\text{-fri-roots } f\text{-final}$
 $\subseteq \text{alpha-space}$

unfolding $\text{alpha-header-supported-union-bad-sets-def}$ **by** auto

lemma $\text{alpha-header-union-bad-sets-fraction-bound-if-unique-candidate:}$

fixes $C :: \text{prob}$

assumes unique:

$\exists \text{ trace-table.}$
 $\text{alpha-header-trace-table-candidates } s \text{ fr } f\text{-fri-roots } f\text{-final} \subseteq$
 $\{ \text{trace-table} \}$

and $\text{subset: } \bigwedge \text{trace-table. bad-sets trace-table} \subseteq \text{alpha-space}$

and bound:

$\bigwedge \text{trace-table.}$
 $\text{nnreal (card (bad-sets trace-table)) / nnreal (card alpha-space)} \leq C$

shows

nnreal
 $(\text{card (alpha-header-union-bad-sets } s \text{ bad-sets fr } f\text{-fri-roots } f\text{-final})) /$
 $\text{nnreal (card alpha-space)} \leq C$

proof –

from unique **obtain** trace-table **where** candidates:

$\text{alpha-header-trace-table-candidates } s \text{ fr } f\text{-fri-roots } f\text{-final} \subseteq$
 $\{ \text{trace-table} \}$

```

    by blast
  have union-subset:
    alpha-header-union-bad-sets s bad-sets fr f-fri-roots f-final  $\subseteq$ 
      bad-sets trace-table
  unfolding alpha-header-union-bad-sets-def
  using candidates by auto
  have finite-bad: finite (bad-sets trace-table)
    by (rule finite-subset[OF subset finite-alpha-space])
  have card-le:
    card (alpha-header-union-bad-sets s bad-sets fr f-fri-roots f-final)  $\leq$ 
      card (bad-sets trace-table)
    by (rule card-mono[OF finite-bad union-subset])
  have nnreal
    (card (alpha-header-union-bad-sets s bad-sets fr f-fri-roots f-final)) /
    nnreal (card alpha-space)  $\leq$ 
    nnreal (card (bad-sets trace-table)) / nnreal (card alpha-space)
    by (rule nnreal-nat-divide-right-mono[OF card-le])
  also have ...  $\leq C$ 
    by (rule bound)
  finally show ?thesis .
qed

lemma alpha-header-supported-union-bad-sets-fraction-bound-if-unique-candidate:
  fixes C :: prob
  assumes unique:
     $\exists$  trace-table.
      alpha-header-supported-trace-table-candidates s fr f-fri-roots
        f-final  $\subseteq \{ \text{trace-table} \}$ 
  and subset:  $\bigwedge \text{trace-table. bad-sets trace-table} \subseteq \text{alpha-space}$ 
  and bound:
     $\bigwedge \text{trace-table. nnreal (card (bad-sets trace-table)) / nnreal (card alpha-space)} \leq C$ 
  shows
    nnreal
      (card
        (alpha-header-supported-union-bad-sets s bad-sets fr f-fri-roots
          f-final)) /
      nnreal (card alpha-space)  $\leq C$ 
proof -
  from unique obtain trace-table where candidates:
    alpha-header-supported-trace-table-candidates s fr f-fri-roots f-final
       $\subseteq \{ \text{trace-table} \}$ 
  by blast
  have union-subset:
    alpha-header-supported-union-bad-sets s bad-sets fr f-fri-roots f-final
       $\subseteq$  bad-sets trace-table
  unfolding alpha-header-supported-union-bad-sets-def
  using candidates by auto
  have finite-bad: finite (bad-sets trace-table)

```

```

    by (rule finite-subset[OF subset finite-alpha-space])
  have card-le:
    card
      (alpha-header-supported-union-bad-sets s bad-sets fr f-fri-roots
        f-final) ≤ card (bad-sets trace-table)
    by (rule card-mono[OF finite-bad union-subset])
  have nnreal
    (card
      (alpha-header-supported-union-bad-sets s bad-sets fr f-fri-roots
        f-final)) /
    nnreal (card alpha-space) ≤
    nnreal (card (bad-sets trace-table)) / nnreal (card alpha-space)
  by (rule nnreal-nat-divide-right-mono[OF card-le])
  also have ... ≤ C
  by (rule bound)
  finally show ?thesis .
qed

lemma composition-trace-alpha-header-union-fraction-bound-if-unique-candidate:
  assumes unique:
    ∃ trace-table.
      alpha-header-trace-table-candidates s fr f-fri-roots f-final ⊆
        {trace-table}
  shows
    nnreal
      (card
        (alpha-header-union-bad-sets s composition-trace-bad-alpha-space
          fr f-fri-roots f-final)) /
      nnreal (card alpha-space) ≤ composition-error-bound
  by (rule alpha-header-union-bad-sets-fraction-bound-if-unique-candidate
    [OF unique composition-trace-bad-alpha-space-subset-alpha-space
      composition-trace-bad-alpha-space-fraction-bound-alpha-space])

lemma composition-trace-alpha-header-supported-union-fraction-bound-if-unique-candidate:
  assumes unique:
    ∃ trace-table.
      alpha-header-supported-trace-table-candidates s fr f-fri-roots
        f-final ⊆ {trace-table}
  shows
    nnreal
      (card
        (alpha-header-supported-union-bad-sets s
          composition-trace-bad-alpha-space fr f-fri-roots f-final)) /
      nnreal (card alpha-space) ≤ composition-error-bound
  by (rule alpha-header-supported-union-bad-sets-fraction-bound-if-unique-candidate
    [OF unique composition-trace-bad-alpha-space-subset-alpha-space
      composition-trace-bad-alpha-space-fraction-bound-alpha-space])

lemma alpha-bad-set-hit-imp-alpha-header-union-bad-set-hit:

```

assumes *hit*: $\alpha\text{-bad-set-hit } s \text{ bad-sets out}$
and *subset*: $\bigwedge \text{trace-table. bad-sets trace-table} \subseteq \alpha\text{-space}$
shows
 $\alpha\text{-header-list-set-hit } s$
 $(\alpha\text{-header-union-bad-sets } s \text{ bad-sets}) \text{ out}$
proof –
from *hit* **obtain** *trace-table composition-table as query-idxs* **where**
bound:
 $\text{accepted-with-bound-tables } s \text{ out trace-table composition-table}$
 as query-idxs
and *as-bad*: $as \in \text{bad-sets trace-table}$
unfolding $\alpha\text{-bad-set-hit-def}$ **by** *blast*
from *bound* **obtain** *result final-state fr f-fri-roots f-final dg*
 $\text{composition-fri-roots final rest}$ **where**
out-eq: $\text{out} = \text{Some } (\text{result}, \text{final-state})$
and *header*:
 $\text{verifier-header-transcript } s \text{ fr f-fri-roots f-final as dg}$
 $\text{composition-fri-roots final rest}$
unfolding $\text{accepted-with-bound-tables-def}$ **by** *blast*
have *shape*:
 $\text{accepted-transcript-shape } s \text{ out as query-idxs}$
using $\text{accepted-with-bound-tables-imp-accepted-with-tables}[OF \text{ bound}]$
 $\text{accepted-with-tables-imp-accepted-transcript-shape}$
by *blast*
have *candidate*:
 $\text{trace-table} \in$
 $\alpha\text{-header-trace-table-candidates } s \text{ fr f-fri-roots f-final}$
unfolding $\alpha\text{-header-trace-table-candidates-def}$
using *bound header* **by** *blast*
have *as-union*:
 $as \in \alpha\text{-header-union-bad-sets } s \text{ bad-sets fr f-fri-roots f-final}$
unfolding $\alpha\text{-header-union-bad-sets-def}$
using *as-bad candidate subset[of trace-table]* **by** *auto*
show *?thesis*
unfolding $\alpha\text{-header-list-set-hit-def}$
using *shape out-eq header as-union* **by** *blast*
qed

lemma $\alpha\text{-bad-set-hit-imp-}\alpha\text{-header-supported-union-bad-set-hit}$:

assumes *outcome*: $\text{out} \in \text{set-dist } (\text{execute verify-monad } s)$
and *hit*: $\alpha\text{-bad-set-hit } s \text{ bad-sets out}$
and *subset*: $\bigwedge \text{trace-table. bad-sets trace-table} \subseteq \alpha\text{-space}$
shows

$\alpha\text{-header-list-set-hit } s$
 $(\alpha\text{-header-supported-union-bad-sets } s \text{ bad-sets}) \text{ out}$

proof –

from *hit* **obtain** *trace-table composition-table as query-idxs* **where**
bound:
 $\text{accepted-with-bound-tables } s \text{ out trace-table composition-table}$


```

    as query-idxs
  and as-bad: as ∈ bad-sets trace-table
  unfolding alpha-bad-set-hit-def by blast
from bound obtain result final-state fr f-fri-roots f-final dg
  composition-fri-roots final rest where
  out-eq: out = Some (result, final-state)
  and header:
    verifier-header-transcript s fr f-fri-roots f-final as dg
    composition-fri-roots final rest
  unfolding accepted-with-bound-tables-def by blast
have shape:
  accepted-transcript-shape s out as query-idxs
  using accepted-with-bound-tables-imp-accepted-with-tables[OF bound]
    accepted-with-tables-imp-accepted-transcript-shape
  by blast
have candidate:
  trace-table ∈
    alpha-header-supported-trace-table-candidates s fr f-fri-roots f-final
  unfolding alpha-header-supported-trace-table-candidates-def
  using outcome bound header by blast
have as-union:
  as ∈
    alpha-header-supported-union-bad-sets s bad-sets fr f-fri-roots f-final
  unfolding alpha-header-supported-union-bad-sets-def
  using as-bad candidate subset[of trace-table] by auto
show ?thesis
  unfolding alpha-header-list-set-hit-def
  using shape out-eq header as-union by blast
qed

```

definition *alpha-challenge-freshness-assumption*
 $:: (f, 'a) \text{ protocol-channel-scheme} \Rightarrow \text{bool}$
where
 $\text{alpha-challenge-freshness-assumption } s \longleftrightarrow$
 $(\forall \text{ bad-sets.}$
 $(\forall \text{ trace-table. bad-sets trace-table} \subseteq \text{alpha-space}) \longrightarrow$
 $(\forall \text{ trace-table.}$
 $\text{nnreal (card (bad-sets trace-table)) /}$
 $\text{nnreal (card alpha-space)} \leq \text{composition-error-bound}) \longrightarrow$
 $\text{wp-event verify-monad (alpha-bad-set-hit s bad-sets) } s \leq$
 $\text{composition-error-bound})$

The protocol currently represents all Fiat-Shamir challenges through the same random-oracle hash map, without protocol-level domain-separated labels for alpha, query-index, and FRI challenges. The following predicates below state the precise freshness obligations needed for each challenge family. Phase 4E bundles them into one random-oracle model interface rather than adding labels to the protocol in this refactor. Merkle hash uses are not represented as freshness obligations here; their binding role remains the explicit *initial-merkle-binding-no-bad* premise.

lemma *initial-merkle-binding-no-badE*:
assumes *bind: initial-merkle-binding-no-bad s*
and *outcome:*
Some (result, final-state) ∈ set-dist (execute verify-monad s)
and *shape: accepted-transcript-shape s (Some (result, final-state)) alphas query-idxs*
obtains *trace-table composition-table*
where *accepted-with-bound-tables s (Some (result, final-state))*
trace-table composition-table alphas query-idxs
using *assms*
unfolding *initial-merkle-binding-no-bad-def initial-merkle-binding-bad-def*
by *blast*

lemma *initial-merkle-binding-bad-bound*:
assumes *bind: initial-merkle-binding-no-bad s*
shows *wp-event verify-monad (initial-merkle-binding-bad s) s ≤ 0*
proof –
have *wp-event verify-monad (initial-merkle-binding-bad s) s ≤*
wp-event verify-monad
(λ- :: (unit list × ('f, 'a) protocol-channel-scheme) option. False) s
by *(rule wp-event-mono-on-support)*
(use bind in ⟨auto simp: initial-merkle-binding-no-bad-def⟩)
also have *... = 0*
unfolding *wp-event-def wp-def dist-expect-def* **by** *simp*
finally show *?thesis* .
qed

lemma *accepted-with-bound-tables-query-consistency*:
assumes *bound:*
accepted-with-bound-tables s out trace-table composition-table alphas query-idxs
shows $\bigwedge idx. idx \in \text{set query-idxs} \implies$
query-consistent-at trace-table composition-table alphas idx
using *bound* **unfolding** *accepted-with-bound-tables-def accepted-with-tables-def* **by** *simp*

lemma *accepted-with-bound-tables-bound-roots*:
assumes *bound:*
accepted-with-bound-tables s out trace-table composition-table alphas query-idxs
obtains *result final-state fr composition-root*
where *out = Some (result, final-state)*
and *merkle-root-binds-table fr trace-table final-state*
and *merkle-root-binds-table composition-root composition-table final-state*
using *accepted-with-tables-authenticated-roots[OF bound]* **by** *blast*

lemma *verifier-header-transcript-shapes*:
assumes *verifier-header-transcript s fr f-fri-roots f-final as dg*
composition-fri-roots final rest
shows *PTranscript s =*
verifier-header-messages fr f-fri-roots f-final as dg
composition-fri-roots final @ rest
and *length f-fri-roots = ceil-log clength*

and $\text{length } as = \text{length } spec$
and $\text{length } \text{composition-fri-roots} = \text{ceil-log } (\text{to-nat } dg + 1)$
using *assms* **unfolding** *verifier-header-transcript-def* **by** *simp-all*

lemma *verifier-header-transcript-unique*:

assumes *h1*:
 $\text{verifier-header-transcript } s \text{ fr } f\text{-fri-roots } f\text{-final } as \text{ dg}$
 $\text{composition-fri-roots } final \text{ rest}$
and *h2*:
 $\text{verifier-header-transcript } s \text{ fr}' f'\text{-fri-roots}' f'\text{-final}' as' \text{ dg}'$
 $\text{composition-fri-roots}' final' \text{ rest}'$

shows

$fr' = fr \wedge$
 $f\text{-fri-roots}' = f\text{-fri-roots} \wedge$
 $f\text{-final}' = f\text{-final} \wedge$
 $as' = as \wedge$
 $dg' = dg \wedge$
 $\text{composition-fri-roots}' = \text{composition-fri-roots} \wedge$
 $final' = final \wedge$
 $rest' = rest$

proof –

have *lengths*:
 $\text{length } f\text{-fri-roots} = \text{ceil-log } clength$
 $\text{length } as = \text{length } spec$
 $\text{length } \text{composition-fri-roots} = \text{ceil-log } (\text{to-nat } dg + 1)$
 $\text{length } f\text{-fri-roots}' = \text{ceil-log } clength$
 $\text{length } as' = \text{length } spec$
 $\text{length } \text{composition-fri-roots}' = \text{ceil-log } (\text{to-nat } dg' + 1)$
using *h1 h2* **unfolding** *verifier-header-transcript-def* **by** *simp-all*
have *msg-eq*:
 $\text{verifier-header-messages } fr \text{ f-fri-roots } f\text{-final } as \text{ dg}$
 $\text{composition-fri-roots } final @ \text{ rest} =$
 $\text{verifier-header-messages } fr' \text{ f-fri-roots}' f'\text{-final}' as' \text{ dg}'$
 $\text{composition-fri-roots}' final' @ \text{ rest}'$
using *h1 h2* **unfolding** *verifier-header-transcript-def* **by** *simp*
then show *?thesis*
using *lengths*
unfolding *verifier-header-messages-def*
by (*auto simp: append-eq-append-conv-if split: if-splits*)

qed

definition *alpha-header-trace-table-candidate-state*

$:: (f, 'a) \text{ protocol-channel-scheme} \Rightarrow$
 $'f \Rightarrow 'f \text{ list} \Rightarrow 'f \Rightarrow 'f \text{ list} \Rightarrow$
 $(f, 'a) \text{ protocol-channel-scheme} \Rightarrow \text{bool}$

where

$\text{alpha-header-trace-table-candidate-state } s \text{ fr } f\text{-fri-roots } f\text{-final}$
 $\text{trace-table } final\text{-state} \longleftrightarrow$
 $(\exists \text{ result } \text{composition-table } as \text{ query-idxs } dg \text{ composition-fri-roots } final$

rest.
accepted-with-bound-tables s (Some (result, final-state))
trace-table composition-table as query-idxs $\wedge$
verifier-header-transcript s fr f-fri-roots f-final as dg
composition-fri-roots final rest $\wedge$
merkle-root-binds-table fr trace-table final-state)

definition *alpha-header-supported-trace-table-candidate-state*

$:: (f, 'a) \text{ protocol-channel-scheme} \Rightarrow$
 $'f \Rightarrow 'f \text{ list} \Rightarrow 'f \Rightarrow 'f \text{ list} \Rightarrow$
 $(f, 'a) \text{ protocol-channel-scheme} \Rightarrow \text{bool}$

where

alpha-header-supported-trace-table-candidate-state s fr f-fri-roots
f-final trace-table final-state $\longleftrightarrow$
 $(\exists \text{result composition-table as query-idxs dg composition-fri-roots final}$
rest.
 $\text{Some (result, final-state)} \in \text{set-dist (execute verify-monad s)} \wedge$
accepted-with-bound-tables s (Some (result, final-state))
trace-table composition-table as query-idxs $\wedge$
verifier-header-transcript s fr f-fri-roots f-final as dg
composition-fri-roots final rest $\wedge$
merkle-root-binds-table fr trace-table final-state)

lemma *alpha-header-trace-table-candidates-iff-state:*

trace-table $\in$
alpha-header-trace-table-candidates s fr f-fri-roots f-final $\longleftrightarrow$
 $(\exists \text{final-state.}$
alpha-header-trace-table-candidate-state s fr f-fri-roots f-final
trace-table final-state)

proof

assume *candidate:*

trace-table $\in$
alpha-header-trace-table-candidates s fr f-fri-roots f-final

then obtain *out composition-table as query-idxs dg composition-fri-roots*
final rest **where**

bound:

accepted-with-bound-tables s out trace-table composition-table as
query-idxs

and *header:*

verifier-header-transcript s fr f-fri-roots f-final as dg
composition-fri-roots final rest

unfolding *alpha-header-trace-table-candidates-def* **by** *blast*

from *bound* **obtain** *result final-state fr' f-fri-roots' f-final' dg'*
composition-fri-roots' final' rest' where

out-eq: out = Some (result, final-state)

and *header':*

verifier-header-transcript s fr' f-fri-roots' f-final' as dg'
composition-fri-roots' final' rest'

and *bind:*

$\text{merkle-root-binds-table } fr' \text{ trace-table final-state}$
unfolding $\text{accepted-with-bound-tables-def}$ **by** blast
have $fr\text{-eq}: fr' = fr$
using $\text{verifier-header-transcript-unique}[OF \text{ header' header}]$ **by** simp
have bound-some :
 $\text{accepted-with-bound-tables } s \text{ (Some (result, final-state))}$
 $\text{trace-table composition-table as query-idxs}$
using bound out-eq **by** simp
have state :
 $\text{alpha-header-trace-table-candidate-state } s \text{ fr f-fri-roots f-final}$
 $\text{trace-table final-state}$
unfolding $\text{alpha-header-trace-table-candidate-state-def}$
by $(\text{intro } exI[of - result] \text{ } exI[of - composition-table] \text{ } exI[of - as]$
 $\text{ } exI[of - query-idxs] \text{ } exI[of - dg] \text{ } exI[of - composition-fri-roots]$
 $\text{ } exI[of - final] \text{ } exI[of - rest])$
 $(\text{use bound-some header bind fr-eq in simp})$
then show
 $\exists \text{ final-state.}$
 $\text{alpha-header-trace-table-candidate-state } s \text{ fr f-fri-roots f-final}$
 $\text{trace-table final-state}$
by blast
next
assume
 $\exists \text{ final-state.}$
 $\text{alpha-header-trace-table-candidate-state } s \text{ fr f-fri-roots f-final}$
 $\text{trace-table final-state}$
then obtain $\text{final-state result composition-table as query-idxs dg}$
 $\text{composition-fri-roots final rest}$ **where**
 bound :
 $\text{accepted-with-bound-tables } s \text{ (Some (result, final-state))}$
 $\text{trace-table composition-table as query-idxs}$
and header :
 $\text{verifier-header-transcript } s \text{ fr f-fri-roots f-final as dg}$
 $\text{composition-fri-roots final rest}$
unfolding $\text{alpha-header-trace-table-candidate-state-def}$ **by** blast
show
 $\text{trace-table} \in$
 $\text{alpha-header-trace-table-candidates } s \text{ fr f-fri-roots f-final}$
unfolding $\text{alpha-header-trace-table-candidates-def}$
using bound header **by** blast
qed

lemma $\text{alpha-header-supported-trace-table-candidates-iff-state}$:
 $\text{trace-table} \in$
 $\text{alpha-header-supported-trace-table-candidates } s \text{ fr f-fri-roots}$
 $\text{f-final} \longleftrightarrow$
 $(\exists \text{ final-state.}$
 $\text{alpha-header-supported-trace-table-candidate-state } s \text{ fr f-fri-roots}$
 $\text{f-final trace-table final-state})$

```

proof
  assume candidate:
    trace-table ∈
      alpha-header-supported-trace-table-candidates s fr f-fri-roots f-final
  then obtain out composition-table as query-idxs dg composition-fri-roots
    final rest where
    outcome: out ∈ set-dist (execute verify-monad s)
  and bound:
    accepted-with-bound-tables s out trace-table composition-table as
      query-idxs
  and header:
    verifier-header-transcript s fr f-fri-roots f-final as dg
      composition-fri-roots final rest
  unfolding alpha-header-supported-trace-table-candidates-def by blast
from bound obtain result final-state fr' f-fri-roots' f-final' dg'
  composition-fri-roots' final' rest' where
  out-eq: out = Some (result, final-state)
  and header':
    verifier-header-transcript s fr' f-fri-roots' f-final' as dg'
      composition-fri-roots' final' rest'
  and bind:
    merkle-root-binds-table fr' trace-table final-state
  unfolding accepted-with-bound-tables-def by blast
have fr-eq: fr' = fr
  using verifier-header-transcript-unique[OF header' header] by simp
have state:
  alpha-header-supported-trace-table-candidate-state s fr f-fri-roots
    f-final trace-table final-state
  unfolding alpha-header-supported-trace-table-candidate-state-def
by (intro exI[of - result] exI[of - composition-table] exI[of - as]
    exI[of - query-idxs] exI[of - dg] exI[of - composition-fri-roots]
    exI[of - final] exI[of - rest])
    (use outcome bound header bind fr-eq out-eq in simp)
then show
  ∃ final-state.
    alpha-header-supported-trace-table-candidate-state s fr f-fri-roots
      f-final trace-table final-state
  by blast
next
  assume
    ∃ final-state.
      alpha-header-supported-trace-table-candidate-state s fr f-fri-roots
        f-final trace-table final-state
  then obtain final-state result composition-table as query-idxs dg
    composition-fri-roots final rest where
  outcome:
    Some (result, final-state) ∈ set-dist (execute verify-monad s)
  and bound:
    accepted-with-bound-tables s (Some (result, final-state))

```

```

    trace-table composition-table as query-idxs
and header:
    verifier-header-transcript s fr f-fri-roots f-final as dg
    composition-fri-roots final rest
unfolding alpha-header-supported-trace-table-candidate-state-def by blast
show
    trace-table ∈
    alpha-header-supported-trace-table-candidates s fr f-fri-roots f-final
unfolding alpha-header-supported-trace-table-candidates-def
using outcome bound header by blast
qed

lemma alpha-header-supported-trace-table-candidate-state-imp-candidate-state:
assumes
    alpha-header-supported-trace-table-candidate-state s fr f-fri-roots
    f-final trace-table final-state
shows
    alpha-header-trace-table-candidate-state s fr f-fri-roots f-final
    trace-table final-state
using assms
unfolding alpha-header-supported-trace-table-candidate-state-def
    alpha-header-trace-table-candidate-state-def
by blast

lemma alpha-header-trace-table-candidate-states-common-extension-unique-if-clean:
assumes clean:  $\neg$  hash-map-output-collision u
and candidate1:
    alpha-header-trace-table-candidate-state s fr f-fri-roots f-final
    trace-table final-state
and candidate2:
    alpha-header-trace-table-candidate-state s fr f-fri-roots f-final
    trace-table' final-state'
and ext1: final-state  $\leq$  u
and ext2: final-state'  $\leq$  u
shows trace-table = trace-table'
proof –
from candidate1 obtain result composition-table as query-idxs dg
    composition-fri-roots final rest where
    bound1:
        accepted-with-bound-tables s (Some (result, final-state))
        trace-table composition-table as query-idxs
and bind1: merkle-root-binds-table fr trace-table final-state
unfolding alpha-header-trace-table-candidate-state-def by blast
from candidate2 obtain result' composition-table' as' query-idxs' dg'
    composition-fri-roots' final' rest' where
    bound2:
        accepted-with-bound-tables s (Some (result', final-state'))
        trace-table' composition-table' as' query-idxs'
and bind2: merkle-root-binds-table fr trace-table' final-state'

```

```

unfolding alpha-header-trace-table-candidate-state-def by blast
have same-len: length trace-table = length trace-table'
using accepted-with-bound-tables-shapes(1)[OF bound1]
      accepted-with-bound-tables-shapes(1)[OF bound2]
by simp
have nonempty: trace-table ≠ []
using accepted-with-bound-tables-shapes(1)[OF bound1]
      eval-domain-nontrivial by auto
show ?thesis
by (rule merkle-root-binds-same-length-tables-common-extension-unique-if-clean
      [OF clean bind1 bind2 ext1 ext2 same-len nonempty])
qed

```

lemma *alpha-header-trace-table-candidate-states-compatible-unique-if-clean-merge*:

assumes *compatible*: *hash-maps-compatible final-state final-state'*

and *clean*:

$\neg$ *hash-map-output-collision*
 (*hash-state-merge final-state final-state'*)

and *candidate1*:

alpha-header-trace-table-candidate-state s fr f-fri-roots f-final
 trace-table final-state

and *candidate2*:

alpha-header-trace-table-candidate-state s fr f-fri-roots f-final
 trace-table' final-state'

shows *trace-table = trace-table'*

proof —

from *candidate1* **obtain** *result composition-table as query-idxs dg*

composition-fri-roots final rest **where**

bound1:

accepted-with-bound-tables s (Some (result, final-state))
 trace-table composition-table as query-idxs

and *bind1*: *merkle-root-binds-table fr trace-table final-state*

unfolding *alpha-header-trace-table-candidate-state-def* **by** *blast*

from *candidate2* **obtain** *result' composition-table' as' query-idxs' dg'*

composition-fri-roots' final' rest' **where**

bound2:

accepted-with-bound-tables s (Some (result', final-state'))
 trace-table' composition-table' as' query-idxs'

and *bind2*: *merkle-root-binds-table fr trace-table' final-state'*

unfolding *alpha-header-trace-table-candidate-state-def* **by** *blast*

have *same-len*: *length trace-table = length trace-table'*

using *accepted-with-bound-tables-shapes(1)[OF bound1]*

accepted-with-bound-tables-shapes(1)[OF bound2]

by *simp*

have *nonempty*: *trace-table ≠ []*

using *accepted-with-bound-tables-shapes(1)[OF bound1]*

eval-domain-nontrivial **by** *auto*

show *?thesis*

by (*rule*

merkle-root-binds-same-length-tables-compatible-states-unique-if-clean-merge
 $[OF\ compatible\ clean\ bind1\ bind2\ same-len\ nonempty]$

qed

lemma *alpha-header-trace-table-candidate-states-merkle-compatible-unique-if-clean-merge:*
assumes *compatible:*
merkle-hash-maps-compatible final-state final-state'
and *clean:*
 $\neg hash-map-output-collision$
(merkle-hash-state-merge final-state final-state')
and *candidate1:*
alpha-header-trace-table-candidate-state s fr f-fri-roots f-final
trace-table final-state
and *candidate2:*
alpha-header-trace-table-candidate-state s fr f-fri-roots f-final
trace-table' final-state'
shows *trace-table = trace-table'*

proof –
from *candidate1* **obtain** *result composition-table as query-idxs dg*
composition-fri-roots final rest **where**
bound1:
accepted-with-bound-tables s (Some (result, final-state))
trace-table composition-table as query-idxs
and *bind1:* *merkle-root-binds-table fr trace-table final-state*
unfolding *alpha-header-trace-table-candidate-state-def* **by** *blast*
from *candidate2* **obtain** *result' composition-table' as' query-idxs' dg'*
composition-fri-roots' final' rest' **where**
bound2:
accepted-with-bound-tables s (Some (result', final-state'))
trace-table' composition-table' as' query-idxs'
and *bind2:* *merkle-root-binds-table fr trace-table' final-state'*
unfolding *alpha-header-trace-table-candidate-state-def* **by** *blast*
have *same-len:* *length trace-table = length trace-table'*
using *accepted-with-bound-tables-shapes(1)[OF bound1]*
accepted-with-bound-tables-shapes(1)[OF bound2]
by *simp*
have *nonempty:* *trace-table $\neq$ []*
using *accepted-with-bound-tables-shapes(1)[OF bound1]*
eval-domain-nontrivial **by** *auto*
show *?thesis*
by *(rule*
merkle-root-binds-same-length-tables-merkle-compatible-unique-if-clean-merge
 $[OF\ compatible\ clean\ bind1\ bind2\ same-len\ nonempty])$

qed

lemma *alpha-header-trace-table-candidates-subset-singleton-if-common-clean-extension:*
assumes *clean:* $\neg hash-map-output-collision\ u$
and *common-ext:*
 $\bigwedge trace-table\ final-state.$

```

    alpha-header-trace-table-candidate-state s fr f-fri-roots f-final
    trace-table final-state  $\implies$ 
    final-state  $\leq u$ 
  shows
     $\exists$  trace-table.
    alpha-header-trace-table-candidates s fr f-fri-roots f-final  $\subseteq$ 
    {trace-table}
  proof (cases
    alpha-header-trace-table-candidates s fr f-fri-roots f-final = {})
  case True
  then show ?thesis
    by (intro exI[of - []]) simp
next
  case False
  then obtain trace-table0 where table0:
    trace-table0  $\in$ 
    alpha-header-trace-table-candidates s fr f-fri-roots f-final
  by blast
  then obtain final-state0 where state0:
    alpha-header-trace-table-candidate-state s fr f-fri-roots f-final
    trace-table0 final-state0
  using alpha-header-trace-table-candidates-iff-state by blast
  have ext0: final-state0  $\leq u$ 
  by (rule common-ext[OF state0])
  have subset:
    alpha-header-trace-table-candidates s fr f-fri-roots f-final  $\subseteq$ 
    {trace-table0}
  proof
    fix trace-table
    assume table:
      trace-table  $\in$ 
      alpha-header-trace-table-candidates s fr f-fri-roots f-final
    then obtain final-state where state:
      alpha-header-trace-table-candidate-state s fr f-fri-roots f-final
      trace-table final-state
    using alpha-header-trace-table-candidates-iff-state by blast
    have ext: final-state  $\leq u$ 
    by (rule common-ext[OF state])
    have trace-table = trace-table0
    by (rule
      alpha-header-trace-table-candidate-states-common-extension-unique-if-clean
      [OF clean state state0 ext ext0])
    then show trace-table  $\in$  {trace-table0}
    by simp
  qed
  then show ?thesis
  by blast
qed

```

lemma *alpha-header-trace-table-candidates-subset-singleton-if-pairwise-clean-merge:*

assumes *pairwise:*

$\bigwedge \text{trace-table final-state trace-table' final-state'.$
 $\text{alpha-header-trace-table-candidate-state } s \text{ fr } f\text{-fri-roots } f\text{-final}$
 $\text{trace-table final-state} \implies$
 $\text{alpha-header-trace-table-candidate-state } s \text{ fr } f\text{-fri-roots } f\text{-final}$
 $\text{trace-table' final-state'} \implies$
 $\text{hash-maps-compatible final-state final-state'} \wedge$
 $\neg \text{hash-map-output-collision}$
 $(\text{hash-state-merge final-state final-state'})$

shows

$\exists \text{trace-table.}$
 $\text{alpha-header-trace-table-candidates } s \text{ fr } f\text{-fri-roots } f\text{-final} \subseteq$
 $\{\text{trace-table}\}$

proof (*cases*

$\text{alpha-header-trace-table-candidates } s \text{ fr } f\text{-fri-roots } f\text{-final} = \{\}$)

case *True*

then show *?thesis*

by (*intro exI[of - []] simp*)

next

case *False*

then obtain *trace-table0* **where** *table0:*

$\text{trace-table0} \in$
 $\text{alpha-header-trace-table-candidates } s \text{ fr } f\text{-fri-roots } f\text{-final}$
by *blast*

then obtain *final-state0* **where** *state0:*

$\text{alpha-header-trace-table-candidate-state } s \text{ fr } f\text{-fri-roots } f\text{-final}$
 $\text{trace-table0 final-state0}$

using *alpha-header-trace-table-candidates-iff-state* **by** *blast*

have *subset:*

$\text{alpha-header-trace-table-candidates } s \text{ fr } f\text{-fri-roots } f\text{-final} \subseteq$
 $\{\text{trace-table0}\}$

proof

fix *trace-table*

assume *table:*

$\text{trace-table} \in$
 $\text{alpha-header-trace-table-candidates } s \text{ fr } f\text{-fri-roots } f\text{-final}$

then obtain *final-state* **where** *state:*

$\text{alpha-header-trace-table-candidate-state } s \text{ fr } f\text{-fri-roots } f\text{-final}$
 $\text{trace-table final-state}$

using *alpha-header-trace-table-candidates-iff-state* **by** *blast*

have *compatible:*

$\text{hash-maps-compatible final-state final-state0}$

using *pairwise[OF state state0]* **by** *simp*

have *clean:*

$\neg \text{hash-map-output-collision}$
 $(\text{hash-state-merge final-state final-state0})$

using *pairwise[OF state state0]* **by** *simp*

have $\text{trace-table} = \text{trace-table0}$

```

    by (rule
        alpha-header-trace-table-candidate-states-compatible-unique-if-clean-merge
        [OF compatible clean state state0])
    then show trace-table ∈ {trace-table0}
    by simp
qed
then show ?thesis
    by blast
qed

lemma alpha-header-supported-trace-table-candidates-subset-singleton-if-pairwise-clean-merge:
  assumes pairwise:
    ∧ trace-table final-state trace-table' final-state'.
    alpha-header-supported-trace-table-candidate-state s fr f-fri-roots
    f-final trace-table final-state ⇒
    alpha-header-supported-trace-table-candidate-state s fr f-fri-roots
    f-final trace-table' final-state' ⇒
    hash-maps-compatible final-state final-state' ∧
    ¬ hash-map-output-collision
    (hash-state-merge final-state final-state')
  shows
    ∃ trace-table.
    alpha-header-supported-trace-table-candidates s fr f-fri-roots f-final
    ⊆ {trace-table}
proof (cases
  alpha-header-supported-trace-table-candidates s fr f-fri-roots f-final =
    {})
case True
  then show ?thesis
    by (intro exI[of - []]) simp
next
case False
  then obtain trace-table0 where table0:
    trace-table0 ∈
    alpha-header-supported-trace-table-candidates s fr f-fri-roots f-final
  by blast
  then obtain final-state0 where state0:
    alpha-header-supported-trace-table-candidate-state s fr f-fri-roots
    f-final trace-table0 final-state0
  using alpha-header-supported-trace-table-candidates-iff-state by blast
  have state0-plain:
    alpha-header-trace-table-candidate-state s fr f-fri-roots f-final
    trace-table0 final-state0
  by (rule alpha-header-supported-trace-table-candidate-state-imp-candidate-state
    [OF state0])
  have subset:
    alpha-header-supported-trace-table-candidates s fr f-fri-roots f-final
    ⊆ {trace-table0}
proof

```

```

fix trace-table
assume table:
  trace-table ∈
    alpha-header-supported-trace-table-candidates s fr f-fri-roots f-final
then obtain final-state where state:
  alpha-header-supported-trace-table-candidate-state s fr f-fri-roots
    f-final trace-table final-state
using alpha-header-supported-trace-table-candidates-iff-state by blast
have state-plain:
  alpha-header-trace-table-candidate-state s fr f-fri-roots f-final
    trace-table final-state
by (rule
  alpha-header-supported-trace-table-candidate-state-imp-candidate-state
    [OF state])
have compatible:
  hash-maps-compatible final-state final-state0
using pairwise[OF state state0] by simp
have clean:
  ¬ hash-map-output-collision
    (hash-state-merge final-state final-state0)
using pairwise[OF state state0] by simp
have trace-table = trace-table0
by (rule
  alpha-header-trace-table-candidate-states-compatible-unique-if-clean-merge
    [OF compatible clean state-plain state0-plain])
then show trace-table ∈ {trace-table0}
by simp
qed
then show ?thesis
by blast
qed

lemma alpha-header-supported-trace-table-candidates-subset-singleton-if-pairwise-no-value-conflict-clean-merge:
assumes pairwise:
  ∧ trace-table final-state trace-table' final-state'.
  alpha-header-supported-trace-table-candidate-state s fr f-fri-roots
    f-final trace-table final-state ⇒
  alpha-header-supported-trace-table-candidate-state s fr f-fri-roots
    f-final trace-table' final-state' ⇒
  ¬ hash-map-value-conflict final-state final-state' ∧
  ¬ hash-map-output-collision
    (hash-state-merge final-state final-state')
shows
  ∃ trace-table.
  alpha-header-supported-trace-table-candidates s fr f-fri-roots f-final
    ⊆ {trace-table}
proof (rule
  alpha-header-supported-trace-table-candidates-subset-singleton-if-pairwise-clean-merge)
fix trace-table final-state trace-table' final-state'

```

```

assume candidate:
  alpha-header-supported-trace-table-candidate-state s fr f-fri-roots
  f-final trace-table final-state
and candidate':
  alpha-header-supported-trace-table-candidate-state s fr f-fri-roots
  f-final trace-table' final-state'
have no-conflict:
   $\neg$  hash-map-value-conflict final-state final-state'
  using pairwise[OF candidate candidate'] by simp
have compatible:
  hash-maps-compatible final-state final-state'
  using no-conflict
  unfolding hash-maps-compatible-iff-no-value-conflict .
have clean:
   $\neg$  hash-map-output-collision
  (hash-state-merge final-state final-state')
  using pairwise[OF candidate candidate'] by simp
show hash-maps-compatible final-state final-state'  $\wedge$ 
   $\neg$  hash-map-output-collision
  (hash-state-merge final-state final-state')
  using compatible clean by simp
qed

lemma alpha-header-supported-trace-table-candidates-subset-singleton-if-pairwise-merkle-clean-merge:
assumes pairwise:
   $\bigwedge$  trace-table final-state trace-table' final-state'.
  alpha-header-supported-trace-table-candidate-state s fr f-fri-roots
  f-final trace-table final-state  $\implies$ 
  alpha-header-supported-trace-table-candidate-state s fr f-fri-roots
  f-final trace-table' final-state'  $\implies$ 
  merkle-hash-maps-compatible final-state final-state'  $\wedge$ 
   $\neg$  hash-map-output-collision
  (merkle-hash-state-merge final-state final-state')
shows
   $\exists$  trace-table.
  alpha-header-supported-trace-table-candidates s fr f-fri-roots f-final
   $\subseteq \{trace-table\}$ 
proof (cases
  alpha-header-supported-trace-table-candidates s fr f-fri-roots f-final =
   $\{\}$ )
case True
then show ?thesis
  by (intro exI[of - []]) simp
next
case False
then obtain trace-table0 where table0:
  trace-table0  $\in$ 
  alpha-header-supported-trace-table-candidates s fr f-fri-roots f-final
  by blast

```

then obtain *final-state0* **where** *state0*:
 alpha-header-supported-trace-table-candidate-state s fr f-fri-roots
 f-final trace-table0 final-state0
 using *alpha-header-supported-trace-table-candidates-iff-state* **by** *blast*
have *state0-plain*:
 alpha-header-trace-table-candidate-state s fr f-fri-roots f-final
 trace-table0 final-state0
 by (rule *alpha-header-supported-trace-table-candidate-state-imp-candidate-state*
 [*OF state0*])
have *subset*:
 alpha-header-supported-trace-table-candidates s fr f-fri-roots f-final
 $\subseteq \{ \text{trace-table0} \}$
proof
 fix *trace-table*
 assume *table*:
 trace-table $\in$
 alpha-header-supported-trace-table-candidates s fr f-fri-roots f-final
 then obtain *final-state* **where** *state*:
 alpha-header-supported-trace-table-candidate-state s fr f-fri-roots
 f-final trace-table final-state
 using *alpha-header-supported-trace-table-candidates-iff-state* **by** *blast*
 have *state-plain*:
 alpha-header-trace-table-candidate-state s fr f-fri-roots f-final
 trace-table final-state
 by (rule
 alpha-header-supported-trace-table-candidate-state-imp-candidate-state
 [*OF state*])
 have *compatible*:
 merkle-hash-maps-compatible final-state final-state0
 using *pairwise*[*OF state state0*] **by** *simp*
 have *clean*:
 $\neg$ *hash-map-output-collision*
 (*merkle-hash-state-merge final-state final-state0*)
 using *pairwise*[*OF state state0*] **by** *simp*
 have *trace-table = trace-table0*
 by (rule
 alpha-header-trace-table-candidate-states-merkle-compatible-unique-if-clean-merge
 [*OF compatible clean state-plain state0-plain*])
 then show *trace-table* $\in \{ \text{trace-table0} \}$
 by *simp*
qed
then show *?thesis*
 by *blast*
qed

lemma *alpha-header-supported-trace-table-candidates-subset-singleton-if-pairwise-no-merkle-value-conflict-clean-merge*
assumes *pairwise*:
 $\bigwedge \text{trace-table final-state trace-table' final-state'.$
 alpha-header-supported-trace-table-candidate-state s fr f-fri-roots

$f\text{-final trace-table final-state} \implies$
 $\alpha\text{-header-supported-trace-table-candidate-state } s \text{ fr } f\text{-fri-roots}$
 $f\text{-final trace-table' final-state'} \implies$
 $\neg \text{merkle-hash-value-conflict final-state final-state'} \wedge$
 $\neg \text{hash-map-output-collision}$
 $(\text{merkle-hash-state-merge final-state final-state'})$
shows
 $\exists \text{trace-table.}$
 $\alpha\text{-header-supported-trace-table-candidates } s \text{ fr } f\text{-fri-roots } f\text{-final}$
 $\subseteq \{\text{trace-table}\}$
proof (rule
 $\alpha\text{-header-supported-trace-table-candidates-subset-singleton-if-pairwise-merkle-clean-merge})$
fix $\text{trace-table final-state trace-table' final-state'}$
assume candidate:
 $\alpha\text{-header-supported-trace-table-candidate-state } s \text{ fr } f\text{-fri-roots}$
 $f\text{-final trace-table final-state}$
and candidate':
 $\alpha\text{-header-supported-trace-table-candidate-state } s \text{ fr } f\text{-fri-roots}$
 $f\text{-final trace-table' final-state'}$
have no-conflict:
 $\neg \text{merkle-hash-value-conflict final-state final-state'}$
using $\text{pairwise}[OF \text{ candidate candidate'}] \text{ by simp}$
have compatible:
 $\text{merkle-hash-maps-compatible final-state final-state'}$
using no-conflict
unfolding $\text{merkle-hash-maps-compatible-iff-no-value-conflict .}$
have clean:
 $\neg \text{hash-map-output-collision}$
 $(\text{merkle-hash-state-merge final-state final-state'})$
using $\text{pairwise}[OF \text{ candidate candidate'}] \text{ by simp}$
show $\text{merkle-hash-maps-compatible final-state final-state'} \wedge$
 $\neg \text{hash-map-output-collision}$
 $(\text{merkle-hash-state-merge final-state final-state'})$
using $\text{compatible clean by simp}$
qed

lemma $\alpha\text{-header-supported-trace-table-candidates-subset-singleton-if-common-clean-merkle-extension:}$
assumes $\text{clean: } \neg \text{hash-map-output-collision } u$
and common-ext:
 $\bigwedge \text{trace-table final-state.}$
 $\alpha\text{-header-supported-trace-table-candidate-state } s \text{ fr } f\text{-fri-roots}$
 $f\text{-final trace-table final-state} \implies$
 $\text{merkle-hash-extends final-state } u$
shows
 $\exists \text{trace-table.}$
 $\alpha\text{-header-supported-trace-table-candidates } s \text{ fr } f\text{-fri-roots } f\text{-final}$
 $\subseteq \{\text{trace-table}\}$
proof (rule
 $\alpha\text{-header-supported-trace-table-candidates-subset-singleton-if-pairwise-no-merkle-value-conflict-clean-merge})$


```

fix trace-table final-state trace-table' final-state'
assume candidate:
  alpha-header-supported-trace-table-candidate-state s fr f-fri-roots
  f-final trace-table final-state
and candidate':
  alpha-header-supported-trace-table-candidate-state s fr f-fri-roots
  f-final trace-table' final-state'
have ext: merkle-hash-extends final-state u
  by (rule common-ext[OF candidate])
have ext': merkle-hash-extends final-state' u
  by (rule common-ext[OF candidate'])
show  $\neg$  merkle-hash-value-conflict final-state final-state'  $\wedge$ 
   $\neg$  hash-map-output-collision
  (merkle-hash-state-merge final-state final-state')
by (rule common-clean-merkle-extension-imp-pairwise-merkle-clean
  [OF ext ext' clean])
qed

lemma alpha-header-supported-trace-table-candidate-states-distinct-imp-merkle-conflict-or-merge-collision:
assumes candidate:
  alpha-header-supported-trace-table-candidate-state s fr f-fri-roots
  f-final trace-table final-state
and candidate':
  alpha-header-supported-trace-table-candidate-state s fr f-fri-roots
  f-final trace-table' final-state'
and distinct-tables: trace-table  $\neq$  trace-table'
shows
  merkle-hash-value-conflict final-state final-state'  $\vee$ 
  hash-map-output-collision
  (merkle-hash-state-merge final-state final-state')
proof (rule ccontr)
assume no-bad:
   $\neg$  (merkle-hash-value-conflict final-state final-state'  $\vee$ 
  hash-map-output-collision
  (merkle-hash-state-merge final-state final-state'))
have compatible:
  merkle-hash-maps-compatible final-state final-state'
using no-bad unfolding merkle-hash-maps-compatible-iff-no-value-conflict
by simp
have clean:
   $\neg$  hash-map-output-collision
  (merkle-hash-state-merge final-state final-state')
using no-bad by simp
have candidate-plain:
  alpha-header-trace-table-candidate-state s fr f-fri-roots f-final
  trace-table final-state
by (rule alpha-header-supported-trace-table-candidate-state-imp-candidate-state
  [OF candidate])
have candidate-plain':

```

```

    alpha-header-trace-table-candidate-state s fr f-fri-roots f-final
    trace-table' final-state'
  by (rule alpha-header-supported-trace-table-candidate-state-imp-candidate-state
      [OF candidate])
  have trace-table = trace-table'
  by (rule alpha-header-trace-table-candidate-states-merkle-compatible-unique-if-clean-merge
      [OF compatible clean candidate-plain candidate-plain])
  then show False
  using distinct-tables by contradiction
qed

```

lemma *alpha-header-supported-trace-table-candidates-not-singleton-imp-pairwise-merkle-bad:*

assumes *not-unique:*

```

  ¬ (∃ trace-table.
    alpha-header-supported-trace-table-candidates s fr f-fri-roots
    f-final ⊆ {trace-table})

```

shows

```

  ∃ trace-table final-state trace-table' final-state'.
    alpha-header-supported-trace-table-candidate-state s fr f-fri-roots
    f-final trace-table final-state ∧
    alpha-header-supported-trace-table-candidate-state s fr f-fri-roots
    f-final trace-table' final-state' ∧
    trace-table ≠ trace-table' ∧
    (merkle-hash-value-conflict final-state final-state' ∨
     hash-map-output-collision
     (merkle-hash-state-merge final-state final-state'))

```

proof –

let $?C =$

```

  alpha-header-supported-trace-table-candidates s fr f-fri-roots f-final

```

have *nonempty:* $?C \neq \{\}$

using *not-unique* **by** *auto*

then obtain *trace-table* **where** *table:* $trace-table \in ?C$

by *blast*

have $\neg ?C \subseteq \{trace-table\}$

using *not-unique* **by** *blast*

then obtain *trace-table'* **where** *table':*

```

  trace-table' ∈ ?C

```

and *distinct-tables:* $trace-table \neq trace-table'$

by *auto*

from *table* **obtain** *final-state* **where** *candidate:*

```

  alpha-header-supported-trace-table-candidate-state s fr f-fri-roots
  f-final trace-table final-state

```

```

  f-final trace-table final-state

```

using *alpha-header-supported-trace-table-candidates-iff-state* **by** *blast*

from *table'* **obtain** *final-state'* **where** *candidate':*

```

  alpha-header-supported-trace-table-candidate-state s fr f-fri-roots
  f-final trace-table' final-state'

```

```

  f-final trace-table' final-state'

```

using *alpha-header-supported-trace-table-candidates-iff-state* **by** *blast*

have *bad:*

```

  merkle-hash-value-conflict final-state final-state' ∨

```

hash-map-output-collision
 (merkle-hash-state-merge final-state final-state')
by (rule
 alpha-header-supported-trace-table-candidate-states-distinct-imp-merkle-conflict-or-merge-collision
 [OF candidate candidate' distinct-tables])
show ?thesis
 using candidate candidate' distinct-tables bad **by** blast
qed

definition *alpha-header-supported-pairwise-merkle-bad*
 :: ('f, 'a) protocol-channel-scheme $\Rightarrow$
 'f $\Rightarrow$ 'f list $\Rightarrow$ 'f $\Rightarrow$ bool
where
 alpha-header-supported-pairwise-merkle-bad s fr f-fri-roots f-final
 $\longleftrightarrow$
 ($\exists$ trace-table final-state trace-table' final-state'.
 alpha-header-supported-trace-table-candidate-state s fr f-fri-roots
 f-final trace-table final-state $\wedge$
 alpha-header-supported-trace-table-candidate-state s fr f-fri-roots
 f-final trace-table' final-state' $\wedge$
 trace-table $\neq$ trace-table' $\wedge$
 (merkle-hash-value-conflict final-state final-state' $\vee$
 hash-map-output-collision
 (merkle-hash-state-merge final-state final-state'))))

definition *alpha-supported-pairwise-merkle-bad*
 :: ('f, 'a) protocol-channel-scheme $\Rightarrow$ bool
where
 alpha-supported-pairwise-merkle-bad s $\longleftrightarrow$
 ($\exists$ fr f-fri-roots f-final.
 alpha-header-supported-pairwise-merkle-bad s fr f-fri-roots f-final)

lemma *alpha-header-supported-candidate-unique-if-no-pairwise-merkle-bad*:

assumes no-bad:

$\neg$ alpha-header-supported-pairwise-merkle-bad s fr f-fri-roots f-final

shows

$\exists$ trace-table.

alpha-header-supported-trace-table-candidates s fr f-fri-roots
 f-final $\subseteq$ {trace-table}

proof (rule ccontr)

assume not-unique:

$\neg$ ($\exists$ trace-table.

alpha-header-supported-trace-table-candidates s fr f-fri-roots
 f-final $\subseteq$ {trace-table})

have alpha-header-supported-pairwise-merkle-bad s fr f-fri-roots f-final

using

alpha-header-supported-trace-table-candidates-not-singleton-imp-pairwise-merkle-bad
 [OF not-unique]

unfolding alpha-header-supported-pairwise-merkle-bad-def

```

    by blast
  then show False
    using no-bad by contradiction
qed

lemma alpha-header-supported-candidate-unique-if-no-global-pairwise-merkle-bad:
  assumes no-bad:  $\neg$  alpha-supported-pairwise-merkle-bad s
  shows
     $\exists$  trace-table.
      alpha-header-supported-trace-table-candidates s fr f-fri-roots
      f-final  $\subseteq$  {trace-table}
proof (rule alpha-header-supported-candidate-unique-if-no-pairwise-merkle-bad)
  show  $\neg$  alpha-header-supported-pairwise-merkle-bad s fr f-fri-roots f-final
    using no-bad unfolding alpha-supported-pairwise-merkle-bad-def by blast
qed

lemma accepted-with-bound-tables-common-roots:
  assumes bound1:
    accepted-with-bound-tables s out trace-table composition-table
    alphas query-idxs
  and bound2:
    accepted-with-bound-tables s out trace-table' composition-table'
    alphas' query-idxs'
  obtains result final-state fr composition-root
  where out = Some (result, final-state)
    and merkle-root-binds-table fr trace-table final-state
    and merkle-root-binds-table fr trace-table' final-state
    and merkle-root-binds-table composition-root composition-table final-state
    and merkle-root-binds-table composition-root composition-table' final-state
proof -
  from bound1 obtain result1 final-state1 fr f-fri-roots f-final dg
    composition-fri-roots final rest where
    out1: out = Some (result1, final-state1)
  and header1:
    verifier-header-transcript s fr f-fri-roots f-final alphas dg
    composition-fri-roots final rest
  and trace-bind1:
    merkle-root-binds-table fr trace-table final-state1
  and comp-bind1:
    merkle-root-binds-table (hd composition-fri-roots)
    composition-table final-state1
  unfolding accepted-with-bound-tables-def by blast
  from bound2 obtain result2 final-state2 fr' f-fri-roots' f-final' dg'
    composition-fri-roots' final' rest' where
    out2: out = Some (result2, final-state2)
  and header2:
    verifier-header-transcript s fr' f-fri-roots' f-final' alphas' dg'
    composition-fri-roots' final' rest'
  and trace-bind2:

```

```

    merkle-root-binds-table fr' trace-table' final-state2
and comp-bind2:
    merkle-root-binds-table (hd composition-fri-roots')
    composition-table' final-state2
unfolding accepted-with-bound-tables-def by blast
have final-state-eq: final-state2 = final-state1
using out1 out2 by simp
have roots-eq:
    fr' = fr  $\wedge$  composition-fri-roots' = composition-fri-roots
using verifier-header-transcript-unique[OF header1 header2] by simp
have trace-bind2':
    merkle-root-binds-table fr trace-table' final-state1
using trace-bind2 roots-eq final-state-eq by simp
have comp-bind2':
    merkle-root-binds-table (hd composition-fri-roots)
    composition-table' final-state1
using comp-bind2 roots-eq final-state-eq by simp
show ?thesis
by (rule that[OF out1 trace-bind1 trace-bind2' comp-bind1 comp-bind2'])
qed

```

```

lemma accepted-with-bound-tables-trace-table-unique-or-hash-collision-bad:
assumes bound1:
    accepted-with-bound-tables s out trace-table composition-table
    alphas query-idxs
and bound2:
    accepted-with-bound-tables s out trace-table' composition-table'
    alphas' query-idxs'
shows trace-table = trace-table'  $\vee$  hash-map-output-collision-bad s out
proof (cases trace-table = trace-table')
case True
then show ?thesis by simp
next
case neg: False
from accepted-with-bound-tables-common-roots[OF bound1 bound2]
obtain result final-state fr composition-root where
    out: out = Some (result, final-state)
and bind1: merkle-root-binds-table fr trace-table final-state
and bind2: merkle-root-binds-table fr trace-table' final-state
by blast
have same-len: length trace-table = length trace-table'
using accepted-with-bound-tables-shapes(1)[OF bound1]
    accepted-with-bound-tables-shapes(1)[OF bound2]
by simp
have nonempty: trace-table  $\neq$  []
using accepted-with-bound-tables-shapes(1)[OF bound1]
    eval-domain-nontrivial
by auto
have hash-map-output-collision-bad s out

```

```

    by (rule accepted-inconsistent-same-length-merkle-bindings-imp-hash-collision-bad
        [OF out bind1 bind2 same-len nonempty neq])
  then show ?thesis by simp
qed

```

lemma *accepted-with-bound-tables-composition-table-unique-or-hash-collision-bad:*

```

  assumes bound1:
    accepted-with-bound-tables s out trace-table composition-table
    alphas query-idxs
  and bound2:
    accepted-with-bound-tables s out trace-table' composition-table'
    alphas' query-idxs'
  shows composition-table = composition-table' ∨
    hash-map-output-collision-bad s out
proof (cases composition-table = composition-table')
  case True
  then show ?thesis by simp
next
  case neg: False
  from accepted-with-bound-tables-common-roots[OF bound1 bound2]
  obtain result final-state fr composition-root where
    out: out = Some (result, final-state)
  and bind1:
    merkle-root-binds-table composition-root composition-table final-state
  and bind2:
    merkle-root-binds-table composition-root composition-table' final-state
  by blast
  have same-len: length composition-table = length composition-table'
  using accepted-with-bound-tables-shapes(2)[OF bound1]
    accepted-with-bound-tables-shapes(2)[OF bound2]
  by simp
  have nonempty: composition-table ≠ []
  using accepted-with-bound-tables-shapes(2)[OF bound1]
    eval-domain-nontrivial
  by auto
  have hash-map-output-collision-bad s out
  by (rule accepted-inconsistent-same-length-merkle-bindings-imp-hash-collision-bad
      [OF out bind1 bind2 same-len nonempty neq])
  then show ?thesis by simp
qed

```

lemma *accepted-with-bound-tables-tables-unique-or-hash-collision-bad:*

```

  assumes bound1:
    accepted-with-bound-tables s out trace-table composition-table
    alphas query-idxs
  and bound2:
    accepted-with-bound-tables s out trace-table' composition-table'
    alphas' query-idxs'
  shows

```

$(\text{trace-table} = \text{trace-table}' \wedge \text{composition-table} = \text{composition-table}') \vee$
 $\text{hash-map-output-collision-bad } s \text{ out}$
proof –
have *trace-unique*:
 $\text{trace-table} = \text{trace-table}' \vee \text{hash-map-output-collision-bad } s \text{ out}$
by (rule *accepted-with-bound-tables-trace-table-unique-or-hash-collision-bad*
 $[\text{OF bound1 bound2}]$)
have *comp-unique*:
 $\text{composition-table} = \text{composition-table}' \vee$
 $\text{hash-map-output-collision-bad } s \text{ out}$
by (rule *accepted-with-bound-tables-composition-table-unique-or-hash-collision-bad*
 $[\text{OF bound1 bound2}]$)
show *?thesis*
using *trace-unique comp-unique* **by** *blast*
qed

lemma *accepted-with-bound-tables-trace-table-unique-if-no-hash-collision-bad*:
assumes *no-collision*: $\neg \text{hash-map-output-collision-bad } s \text{ out}$
and *bound1*:
 $\text{accepted-with-bound-tables } s \text{ out trace-table composition-table}$
 alphas query-idxs
and *bound2*:
 $\text{accepted-with-bound-tables } s \text{ out trace-table}' \text{ composition-table}'$
 $\text{alphas}' \text{ query-idxs}'$
shows $\text{trace-table} = \text{trace-table}'$
using *accepted-with-bound-tables-trace-table-unique-or-hash-collision-bad*
 $[\text{OF bound1 bound2}]$ *no-collision*
by *blast*

lemma *accepted-with-bound-tables-composition-table-unique-if-no-hash-collision-bad*:
assumes *no-collision*: $\neg \text{hash-map-output-collision-bad } s \text{ out}$
and *bound1*:
 $\text{accepted-with-bound-tables } s \text{ out trace-table composition-table}$
 alphas query-idxs
and *bound2*:
 $\text{accepted-with-bound-tables } s \text{ out trace-table}' \text{ composition-table}'$
 $\text{alphas}' \text{ query-idxs}'$
shows $\text{composition-table} = \text{composition-table}'$
using *accepted-with-bound-tables-composition-table-unique-or-hash-collision-bad*
 $[\text{OF bound1 bound2}]$ *no-collision*
by *blast*

definition *accepted-bound-table-ambiguity-bad*
 $:: (\text{'f}, \text{'a}) \text{ protocol-channel-scheme} \Rightarrow$
 $(\text{unit list} \times (\text{'f}, \text{'a}) \text{ protocol-channel-scheme}) \text{ option} \Rightarrow \text{bool}$
where
 $\text{accepted-bound-table-ambiguity-bad } s \text{ out} \longleftrightarrow$
 $(\exists \text{ trace-table composition-table alphas query-idxs}$
 $\text{trace-table}' \text{ composition-table}' \text{ alphas}' \text{ query-idxs}').$

$\text{accepted-with-bound-tables } s \text{ out trace-table composition-table}$
 $\text{alphas query-idxs} \wedge$
 $\text{accepted-with-bound-tables } s \text{ out trace-table' composition-table'}$
 $\text{alphas' query-idxs'} \wedge$
 $(\text{trace-table} \neq \text{trace-table'} \vee$
 $\text{composition-table} \neq \text{composition-table'})$

lemma *accepted-bound-table-ambiguity-bad-imp-hash-map-output-collision-bad:*

assumes *ambiguity: accepted-bound-table-ambiguity-bad s out*

shows *hash-map-output-collision-bad s out*

proof –

from *ambiguity* **obtain** *trace-table composition-table alphas query-idxs*

trace-table' composition-table' alphas' query-idxs' **where**

bound:

accepted-with-bound-tables s out trace-table composition-table

alphas query-idxs

and *bound':*

accepted-with-bound-tables s out trace-table' composition-table'

alphas' query-idxs'

and *distinct:*

trace-table $\neq$ *trace-table'* $\vee$

composition-table $\neq$ *composition-table'*

unfolding *accepted-bound-table-ambiguity-bad-def* **by** *blast*

have

$(\text{trace-table} = \text{trace-table'} \wedge$

$\text{composition-table} = \text{composition-table'}) \vee$

hash-map-output-collision-bad s out

by (rule *accepted-with-bound-tables-tables-unique-or-hash-collision-bad*

[OF *bound bound'*])

then show *?thesis*

using *distinct* **by** *blast*

qed

lemma *accepted-bound-table-ambiguity-bad-mono-hash-map-output-collision-bad:*

wp-event verify-monad (accepted-bound-table-ambiguity-bad s) s $\leq$

wp-event verify-monad (hash-map-output-collision-bad s) s

by (rule *wp-event-mono*)

(rule

accepted-bound-table-ambiguity-bad-imp-hash-map-output-collision-bad)

end

end

theory *Soundness-Execution-Query*

imports *Soundness-Merkle*

begin

Verifier query-round and authenticated-opening execution lemmas.

context *soundness*
begin

lemma *receive-random-field-element-outcome:*

assumes *outcome: Some (a, t) ∈ set-dist (execute receive-random-field-element s)*

shows

$s \leq t \wedge$

$PState\ t = PState\ s \wedge$

$PTranscript\ t = PTranscript\ s \wedge$

$fmlookup\ (HashMap\ t)\ (FiatShamirChallenge\ (PState\ s)) = Some\ a$

using *protocol-receive-random-field-element-outcome*

$[OF\ outcome[unfolded\ receive-random-field-element-def]]$

by *simp*

lemma *throw-no-outcome:*

Some (x, t) ∉ set-dist (execute throw s)

unfolding *set-dist-def throw.rep-eq dist-throw-def dist-delta-dist delta-map-def*

by *simp*

lemma *read-outcome:*

assumes *outcome: Some (x, t) ∈ set-dist (execute read s)*

shows

$\exists\ rest.$

$PTranscript\ s = x \# rest \wedge$

$PState\ t = concat\ (PState\ s)\ x \wedge$

$PTranscript\ t = rest \wedge$

$s \leq t \wedge$

$PTraceFriCounter\ t = PTraceFriCounter\ s \wedge$

$PCompositionFriCounter\ t = PCompositionFriCounter\ s \wedge$

$PAlphaCounter\ t = PAlphaCounter\ s \wedge$

$PQueryCounter\ t = PQueryCounter\ s$

proof $-$

obtain *rest* **where** *tr-s: PTranscript s = x # rest*

proof $-$

have $PTranscript\ s \neq []$

using *outcome unfolding read-def protocol-read-def assert-def*

by $(cases\ PTranscript\ s)\ (auto\ simp:\ throw-no-outcome\ elim!:\ set-dist-bindE)$

then obtain *y ys* **where** *ys: PTranscript s = y # ys*

by $(cases\ PTranscript\ s)\ auto$

have *x-y: x = y*

using *read-nonempty-outcome[OF outcome ys]* **by** *simp*

show *?thesis*

using *that ys x-y* **by** *auto*

qed

have *t-res:*

$t = s(PState := concat\ (PState\ s)\ x,\ PTranscript := rest)$

using *read-nonempty-outcome[OF outcome tr-s]* **by** *simp*

have $s \leq t$

unfolding *t-res less-eq-hash-ext-def less-eq-fmap-def* **by** *simp*

```

    then show ?thesis
      using tr-s t-res by auto
qed

lemma read-preserves-hash-map:
  assumes outcome: Some (x, t) ∈ set-dist (execute read s)
  shows HashMap t = HashMap s
proof -
  obtain rest where tr-s: PTranscript s = x # rest
  proof -
    have PTranscript s ≠ []
      using outcome unfolding read-def protocol-read-def assert-def
      by (cases PTranscript s) (auto simp: throw-no-outcome elim!: set-dist-bindE)
    then obtain y ys where ys: PTranscript s = y # ys
      by (cases PTranscript s) auto
    have x-y: x = y
      using read-nonempty-outcome[OF outcome ys] by simp
    show ?thesis
      using that ys x-y by auto
  qed
  show ?thesis
    using read-nonempty-outcome[OF outcome tr-s] by simp
qed

lemma read-preserves-alpha-future-fresh:
  assumes future: alpha-future-fresh s
  and outcome: Some (x, t) ∈ set-dist (execute read s)
  shows alpha-future-fresh t
proof -
  have hash-eq: HashMap t = HashMap s
    by (rule read-preserves-hash-map[OF outcome])
  from read-outcome[OF outcome] have counter-eq:
    PAlphaCounter t = PAlphaCounter s
    by blast
  show ?thesis
    using future hash-eq counter-eq
    unfolding alpha-future-fresh-def by simp
qed

lemma read-preserves-trace-fri-future-fresh:
  assumes future: trace-fri-future-fresh s
  and outcome: Some (x, t) ∈ set-dist (execute read s)
  shows trace-fri-future-fresh t
proof -
  have hash-eq: HashMap t = HashMap s
    by (rule read-preserves-hash-map[OF outcome])
  from read-outcome[OF outcome] have counter-eq:
    PTraceFriCounter t = PTraceFriCounter s
    by blast

```

```

show ?thesis
  using future hash-eq counter-eq
  unfolding trace-fri-future-fresh-def by simp
qed

lemma read-preserves-composition-fri-future-fresh:
  assumes future: composition-fri-future-fresh s
  and outcome: Some (x, t) ∈ set-dist (execute read s)
  shows composition-fri-future-fresh t
proof –
  have hash-eq: HashMap t = HashMap s
  by (rule read-preserves-hash-map[OF outcome])
  from read-outcome[OF outcome] have counter-eq:
    PCompositionFriCounter t = PCompositionFriCounter s
  by blast
  show ?thesis
  using future hash-eq counter-eq
  unfolding composition-fri-future-fresh-def by simp
qed

lemma read-preserves-query-future-fresh:
  assumes future: query-future-fresh s
  and outcome: Some (x, t) ∈ set-dist (execute read s)
  shows query-future-fresh t
proof –
  have hash-eq: HashMap t = HashMap s
  by (rule read-preserves-hash-map[OF outcome])
  from read-outcome[OF outcome] have counter-eq:
    PQueryCounter t = PQueryCounter s
  by blast
  show ?thesis
  using future hash-eq counter-eq
  unfolding query-future-fresh-def by simp
qed

lemma ntimes-read-any-outcome:
  assumes outcome: Some (xs, t) ∈ set-dist (execute (ntimes read n) s)
  shows
    length xs = n ∧
    PTranscript s = xs @ PTranscript t ∧
    PState t = foldl concat (PState s) xs ∧
    s ≤ t ∧
    PTraceFriCounter t = PTraceFriCounter s ∧
    PCompositionFriCounter t = PCompositionFriCounter s ∧
    PAlphaCounter t = PAlphaCounter s ∧
    PQueryCounter t = PQueryCounter s
  using outcome
proof (induction n arbitrary: s xs t)
  case 0

```

```

then show ?case
  by (simp add: hash-ext-refl)
next
case (Suc n)
from Suc.premis obtain x xs' s1 where
  read-x: Some (x, s1) ∈ set-dist (execute read s)
  and reads:
    Some (xs', t) ∈ set-dist (execute (ntimes read n) s1)
  and xs-eq: xs = x # xs'
  by (auto elim!: set-dist-bindE)
from read-outcome[OF read-x] obtain rest1 where
  tr-s: PTranscript s = x # rest1
  and st-s1: PState s1 = concat (PState s) x
  and tr-s1: PTranscript s1 = rest1
  and ext-s-s1: s ≤ s1
  and trace-count-s1: PTraceFriCounter s1 = PTraceFriCounter s
  and comp-count-s1: PCompositionFriCounter s1 = PCompositionFriCounter s
  and alpha-count-s1: PAlphaCounter s1 = PAlphaCounter s
  and query-count-s1: PQueryCounter s1 = PQueryCounter s
  by blast
have tail:
  length xs' = n ∧
  PTranscript s1 = xs' @ PTranscript t ∧
  PState t = foldl concat (PState s1) xs' ∧
  s1 ≤ t ∧
  PTraceFriCounter t = PTraceFriCounter s1 ∧
  PCompositionFriCounter t = PCompositionFriCounter s1 ∧
  PAlphaCounter t = PAlphaCounter s1 ∧
  PQueryCounter t = PQueryCounter s1
  using Suc.IH[OF reads] .
have s ≤ t
  using ext-s-s1 tail by (meson hash-ext-trans)
then show ?case
  using tr-s st-s1 tr-s1 tail xs-eq trace-count-s1 comp-count-s1
    alpha-count-s1 query-count-s1
  by simp
qed

```

```

lemma ntimes-read-preserves-lookup:
  assumes outcome: Some (xs, t) ∈ set-dist (execute (ntimes read n) s)
  shows fmlookup (HashMap t) key = fmlookup (HashMap s) key
proof (rule ntimes-preserves-lookup[OF - outcome])
  fix x s t
  assume read-x: Some (x, t) ∈ set-dist (execute read s)
  show fmlookup (HashMap t) key = fmlookup (HashMap s) key
  using read-preserves-hash-map[OF read-x] by simp
qed

```

```

lemma check-authentication-path-hash-extends:

```

```

assumes outcome:
  Some (hh, t) ∈
    set-dist (execute (check-authentication-path len i v path) s)
shows s ≤ t
using outcome
unfolding check-authentication-path-def protocol-check-authentication-path-def
proof (induction path arbitrary: len i v s hh t)
  case Nil
  then have hash-step:
    Some (hh, t) ∈ set-dist (execute (hash (MerkleLeaf v)) s)
    by simp
  show ?case
    by (rule hash-outcome(1)[OF hash-step])
next
  case (Cons a path)
  show ?case
  proof (cases i < len div 2)
    case True
    from Cons.prems obtain x u where
      rec:
        Some (x, u) ∈
          set-dist (execute
            (protocol-merkle.check-authentication-path
              (len div 2) i (MerkleLeaf v) path) s)
        and hash:
          Some (hh, t) ∈ set-dist (execute (hash (MerkleNode x a)) u)
        using True by (auto elim!: set-dist-bindE)
      have s ≤ u
      by (rule Cons.IH[OF rec])
      moreover have u ≤ t
      using hash-outcome(1)[OF hash] .
      ultimately show ?thesis
      by (rule hash-ext-trans)
    next
    case False
    from Cons.prems obtain x u where
      rec:
        Some (x, u) ∈
          set-dist (execute
            (protocol-merkle.check-authentication-path
              (len div 2) (i - len div 2) (MerkleLeaf v) path) s)
        and hash:
          Some (hh, t) ∈ set-dist (execute (hash (MerkleNode a x)) u)
        using False by (auto elim!: set-dist-bindE)
      have s ≤ u
      by (rule Cons.IH[OF rec])
      moreover have u ≤ t
      using hash-outcome(1)[OF hash] .
      ultimately show ?thesis

```

by (rule hash-ext-trans)
qed
qed

lemma *check-decommit-on-query-step-outcome:*

assumes *outcome:*

Some (qh, t) ∈ set-dist (execute
(do {
qh ← read;
*let len = scale * clength;*
qh-path ← ntimes read (floor-log len);
ap ← check-authentication-path len i qh qh-path;
assert (ap = fr);
return qh
}) s)

shows

∃ path.
*length path = floor-log (scale * clength) ∧*
PTranscript s = ([qh] @ path) @ PTranscript t ∧
PState t = foldl concat (PState s) ([qh] @ path) ∧
s ≤ t ∧
PQueryCounter t = PQueryCounter s

proof –

let *?len = scale * clength*

from *outcome* **obtain** *s1 path s2 ap s3 s4* **where**

read-qh: Some (qh, s1) ∈ set-dist (execute read s)

and *read-path:*

Some (path, s2) ∈
set-dist (execute (ntimes read (floor-log ?len)) s1)

and *check:*

Some (ap, s3) ∈
set-dist (execute (check-authentication-path ?len i qh path) s2)

and *assert-ap: Some ((), s4) ∈ set-dist (execute (assert (ap = fr)) s3)*

and *ret-qh: Some (qh, t) ∈ set-dist (execute (return qh) s4)*

by (*auto simp: Let-def elim!: set-dist-bindE*)

from *read-outcome[OF read-qh]* **obtain** *rest1* **where**

tr-s: PTranscript s = qh # rest1

and *st-s1: PState s1 = concat (PState s) qh*

and *tr-s1: PTranscript s1 = rest1*

and *ext-s-s1: s ≤ s1*

and *query-count-s1: PQueryCounter s1 = PQueryCounter s*

by *blast*

have *path-res:*

length path = floor-log ?len ∧
PTranscript s1 = path @ PTranscript s2 ∧
PState s2 = foldl concat (PState s1) path ∧
s1 ≤ s2 ∧

PQueryCounter s2 = PQueryCounter s1

using *ntimes-read-any-outcome[OF read-path]* **by** *simp*

```

have s2-s3:  $s2 \leq s3$ 
  by (rule check-authentication-path-hash-extends[OF check])
have st-s3: PState s3 = PState s2
  using check-authentication-path-preserves-channel(1)[OF check] .
have tr-s3: PTranscript s3 = PTranscript s2
  using check-authentication-path-preserves-channel(2)[OF check] .
have query-count-s3: PQueryCounter s3 = PQueryCounter s2
  using check-authentication-path-preserves-channel(6)[OF check] .
have ap-eq: ap = fr
  using assert-ap unfolding assert-def
  by (cases ap = fr) (auto simp: throw-no-outcome)
have s4-eq: s4 = s3
  using assert-ap ap-eq unfolding assert-def by simp
have t-eq: t = s4
  using ret-gh by simp
have s ≤ t
  using ext-s-s1 path-res s2-s3 unfolding t-eq s4-eq
  by (meson hash-ext-trans)
then show ?thesis
  using tr-s st-s1 tr-s1 path-res st-s3 tr-s3 query-count-s1
    query-count-s3 s4-eq t-eq
  by auto
qed

```

definition query-decommitment-step

where

```

query-decommitment-step fr i =
  (do {
    gh ← read;
    let len = scale * clength;
    gh-path ← ntimes read (floor-log len);
    ap ← check-authentication-path len i gh gh-path;
    assert (ap = fr);
    return gh
  })

```

lemma query-decommitment-step-outcome:

assumes outcome:

Some (gh, t) ∈ set-dist (execute (query-decommitment-step fr i) s)

shows

```

∃ path.
  length path = floor-log (scale * clength) ∧
  PTranscript s = ([gh] @ path) @ PTranscript t ∧
  PState t = foldl concat (PState s) ([gh] @ path) ∧
  s ≤ t ∧
  PQueryCounter t = PQueryCounter s

```

using check-decommit-on-query-step-outcome[OF outcome[unfolded query-decommitment-step-def]]

.

lemma *mmap-query-decommitment-steps-outcome*:

assumes *outcome*:

Some (*leaves*, *t*) ∈
set-dist (*execute* (*mmap* (*map* (*query-decommitment-step* *fr*) *idxs*)) *s*)

shows

∃ *paths* *chunk*.
length *leaves* = *length* *idxs* ∧
length *paths* = *length* *idxs* ∧
chunk = *List.concat* (*map* (λ(*leaf*, *path*). [*leaf*] @ *path*) (*zip* *leaves* *paths*)) ∧
(∀ *path* ∈ *set* *paths*. *length* *path* = *floor-log* (*scale* * *clength*)) ∧
PTranscript *s* = *chunk* @ *PTranscript* *t* ∧
PState *t* = *foldl concat* (*PState* *s*) *chunk* ∧
s ≤ *t* ∧
PQueryCounter *t* = *PQueryCounter* *s*

using *outcome*

proof (*induction* *idxs* *arbitrary*: *s* *leaves* *t*)

case *Nil*

then show ?*case*

by (*intro* *exI*[*of* - []]) (*simp* *add*: *hash-ext-refl*)

next

case (*Cons* *i* *idxs*)

from *Cons.prem*s **obtain** *leaf* *leaves'* *s1* **where**

head:
Some (*leaf*, *s1*) ∈
set-dist (*execute* (*query-decommitment-step* *fr* *i*) *s*)

and *tail*:
Some (*leaves'*, *t*) ∈
set-dist (*execute* (*mmap* (*map* (*query-decommitment-step* *fr*) *idxs*)) *s1*)

and *leaves-eq*: *leaves* = *leaf* # *leaves'*

by (*auto* *elim*!: *set-dist-bindE*)

from *query-decommitment-step-outcome*[*OF* *head*] **obtain** *path* **where**

path-len: *length* *path* = *floor-log* (*scale* * *clength*)
and *tr-s*: *PTranscript* *s* = ([*leaf*] @ *path*) @ *PTranscript* *s1*
and *st-s1*: *PState* *s1* = *foldl concat* (*PState* *s*) ([*leaf*] @ *path*)
and *ext-s-s1*: *s* ≤ *s1*
and *query-count-s1*: *PQueryCounter* *s1* = *PQueryCounter* *s*
by *blast*

from *Cons.IH*[*OF* *tail*] **obtain** *paths* *chunk-tail* **where**

len-leaves': *length* *leaves'* = *length* *idxs*
and *len-paths*: *length* *paths* = *length* *idxs*
and *chunk-tail*:
chunk-tail =
List.concat (*map* (λ(*leaf*, *path*). [*leaf*] @ *path*) (*zip* *leaves'* *paths*))

and *paths-len*:
∀ *path* ∈ *set* *paths*. *length* *path* = *floor-log* (*scale* * *clength*)
and *tr-s1*: *PTranscript* *s1* = *chunk-tail* @ *PTranscript* *t*
and *st-t*: *PState* *t* = *foldl concat* (*PState* *s1*) *chunk-tail*
and *ext-s1-t*: *s1* ≤ *t*
and *query-count-t*: *PQueryCounter* *t* = *PQueryCounter* *s1*


```

    by blast
  let ?paths = path # paths
  let ?chunk = ([leaf] @ path) @ chunk-tail
  have  $s \leq t$ 
    using ext-s-s1 ext-s1-t by (rule hash-ext-trans)
  then show ?case
    apply (intro exI[of - ?paths] exI[of - ?chunk])
    using leaves-eq path-len tr-s tr-s1 st-s1 st-t len-leaves' len-paths
      chunk-tail paths-len query-count-s1 query-count-t
    by simp
qed

```

lemma *check-decommit-on-query-outcome:*

assumes *outcome:*

Some (leaves, t) ∈

set-dist (execute (mmap (check-decommit-on-query fr idx)) s)

shows

$\exists$ *paths chunk.*

query-decommitment-transcript idx leaves paths chunk ∧

PTranscript s = chunk @ PTranscript t ∧

PState t = foldl concat (PState s) chunk ∧

s ≤ t ∧

PQueryCounter t = PQueryCounter s

proof –

have *map-eq:*

check-decommit-on-query fr idx =

map (query-decommitment-step fr) (powers-scaled idx)

unfolding *check-decommit-on-query-def query-decommitment-step-def* **by** *simp*

from *mmap-query-decommitment-steps-outcome[OF outcome[unfolded map-eq]]*

obtain *paths chunk where*

len-leaves: length leaves = length (powers-scaled idx)

and *len-paths: length paths = length (powers-scaled idx)*

and *chunk:*

chunk = List.concat

(map (λ(leaf, path). [leaf] @ path) (zip leaves paths))

and *paths-len:*

$\forall$ *path ∈ set paths. length path = floor-log (scale * clength)*

and *tr-s: PTranscript s = chunk @ PTranscript t*

and *st-t: PState t = foldl concat (PState s) chunk*

and *ext: s ≤ t*

and *query-count-t: PQueryCounter t = PQueryCounter s*

by *blast*

have *qtr: query-decommitment-transcript idx leaves paths chunk*

using *len-leaves len-paths chunk paths-len*

unfolding *query-decommitment-transcript-def*

by *(simp add: mult.commute case-prod-unfold)*

show *?thesis*

using *qtr tr-s st-t ext query-count-t* **by** *blast*

qed

lemma *query-decommitment-step-preserves-query-lookup*:
assumes *outcome*:
Some (qh, t) ∈ set-dist (execute (query-decommitment-step fr i) s)
shows
fmlookup (HashMap t) (QueryIndexChallenge j x) =
fmlookup (HashMap s) (QueryIndexChallenge j x)
proof –
let *?len = scale * clength*
from *outcome* **obtain** *s1 path s2 ap s3 s4* **where**
read-gh: Some (qh, s1) ∈ set-dist (execute read s)
and *read-path*:
Some (path, s2) ∈
set-dist (execute (ntimes read (floor-log ?len)) s1)
and *check*:
Some (ap, s3) ∈
set-dist (execute (check-authentication-path ?len i qh path) s2)
and *assert-ap: Some ((), s4) ∈ set-dist (execute (assert (ap = fr)) s3)*
and *ret-gh: Some (qh, t) ∈ set-dist (execute (return qh) s4)*
unfolding *query-decommitment-step-def*
by (*auto simp: Let-def elim!: set-dist-bindE*)
have *lookup-s1*:
fmlookup (HashMap s1) (QueryIndexChallenge j x) =
fmlookup (HashMap s) (QueryIndexChallenge j x)
using *read-preserves-hash-map[OF read-gh]* **by** *simp*
have *lookup-s2*:
fmlookup (HashMap s2) (QueryIndexChallenge j x) =
fmlookup (HashMap s1) (QueryIndexChallenge j x)
by (*rule ntimes-read-preserves-lookup[OF read-path]*)
have *lookup-s3*:
fmlookup (HashMap s3) (QueryIndexChallenge j x) =
fmlookup (HashMap s2) (QueryIndexChallenge j x)
by (*rule check-authentication-path-preserves-challenge-lookups(2)*
[OF check])
have *s4-eq: s4 = s3*
using *assert-ap* **unfolding** *assert-def*
by (*cases ap = fr*) (*auto simp: throw-no-outcome*)
have *t-eq: t = s4*
using *ret-gh* **by** *simp*
show *?thesis*
using *lookup-s1 lookup-s2 lookup-s3 s4-eq t-eq* **by** *simp*
qed

lemma *mmap-query-decommitment-steps-preserves-query-lookup*:
assumes *outcome*:
Some (leaves, t) ∈
set-dist (execute (mmap (map (query-decommitment-step fr) idxs)) s)
shows
fmlookup (HashMap t) (QueryIndexChallenge j x) =

```

      fmlookup (HashMap s) (QueryIndexChallenge j x)
    using outcome
  proof (induction idxs arbitrary: s leaves t)
    case Nil
    then show ?case
      by simp
  next
    case (Cons i idxs)
    from Cons.prem obtain leaf leaves' s1 where
      head:
        Some (leaf, s1) ∈
          set-dist (execute (query-decommitment-step fr i) s)
      and tail:
        Some (leaves', t) ∈
          set-dist (execute (mmap (map (query-decommitment-step fr) idxs)) s1)
      by (auto elim!: set-dist-bindE)
    have head-pres:
      fmlookup (HashMap s1) (QueryIndexChallenge j x) =
        fmlookup (HashMap s) (QueryIndexChallenge j x)
      by (rule query-decommitment-step-preserves-query-lookup[OF head])
    have tail-pres:
      fmlookup (HashMap t) (QueryIndexChallenge j x) =
        fmlookup (HashMap s1) (QueryIndexChallenge j x)
      by (rule Cons.IH[OF tail])
    show ?case
      using head-pres tail-pres by simp
  qed

lemma check-decommit-on-query-preserves-query-lookup:
  assumes outcome:
    Some (leaves, t) ∈
      set-dist (execute (mmap (check-decommit-on-query fr idx)) s)
  shows
    fmlookup (HashMap t) (QueryIndexChallenge j x) =
      fmlookup (HashMap s) (QueryIndexChallenge j x)
  proof -
    have map-eq:
      check-decommit-on-query fr idx =
        map (query-decommitment-step fr) (powers-scaled idx)
    unfolding check-decommit-on-query-def query-decommitment-step-def by simp
    show ?thesis
      by (rule mmap-query-decommitment-steps-preserves-query-lookup
        [OF outcome[unfolded map-eq]])
  qed

definition fri-layer-opening-step
  where
    fri-layer-opening-step bf st =
      (case bf of (b, f) ⇒

```

```

case st of (i, x, len, pw) ⇒
do {
  xp ← read;
  xp' ← ntimes read (floor-log len);
  xn ← read;
  xn' ← ntimes read (floor-log len);
  assert (xp = x);
  ap ← check-authentication-path len i xp xp';
  assert (ap = f);
  let idx = (i + len div 2) mod len;
  ap ← check-authentication-path len idx xn xn';
  assert (ap = f);
  let gp = (xp + xn) div 2;
  let hp = (xp - xn) div (2*((h^i) * shift)^pw);
  let x = gp + b * hp;
  return (i mod (len div 2), x, len div 2, pw+pw)
})

```

definition *fri-layer-opening-finish*

where

```

fri-layer-opening-finish b f i x len pw xp xp-path xn xn-path =
do {
  assert (xp = x);
  ap ← check-authentication-path len i xp xp-path;
  assert (ap = f);
  let idx = (i + len div 2) mod len;
  ap ← check-authentication-path len idx xn xn-path;
  assert (ap = f);
  let gp = (xp + xn) div 2;
  let hp = (xp - xn) div (2*((h^i) * shift)^pw);
  let x = gp + b * hp;
  return (i mod (len div 2), x, len div 2, pw+pw)
}

```

lemma *fri-layer-opening-step-unfold-finish*:

fri-layer-opening-step (b, f) (i, x, len, pw) =

```

do {
  xp ← read;
  xp-path ← ntimes read (floor-log len);
  xn ← read;
  xn-path ← ntimes read (floor-log len);
  fri-layer-opening-finish b f i x len pw xp xp-path xn xn-path
}

```

unfolding *fri-layer-opening-step-def* *fri-layer-opening-finish-def*
by *simp*

lemma *fri-layer-opening-step-outcome*:

assumes *outcome*:

Some (out, t) ∈

$set-dist \ (execute \ (fri-layer-opening-step \ (b, f) \ (i, x, len, pw)) \ s)$
shows
 $\exists xp \ xp-path \ xn \ xn-path \ x'.$
 $out = (i \bmod (len \div 2), x', len \div 2, pw + pw) \wedge$
 $xp = x \wedge$
 $x' = fri-fold-value \ b \ xp \ xn$
 $(fri-fold-denominator \ ((h \wedge i) * shift) \ pw) \wedge$
 $fri-layer-opening-chunk \ len \ xp \ xp-path \ xn \ xn-path$
 $([xp] @ xp-path @ [xn] @ xn-path) \wedge$
 $PTranscript \ s = ([xp] @ xp-path @ [xn] @ xn-path) @ PTranscript \ t \wedge$
 $PState \ t = foldl \ concat \ (PState \ s) \ ([xp] @ xp-path @ [xn] @ xn-path) \wedge$
 $s \leq t \wedge$
 $PQueryCounter \ t = PQueryCounter \ s$
proof –
from *outcome* **obtain** $xp \ s1 \ xp-path \ s2 \ xn \ s3 \ xn-path \ s4 \ s5 \ ap1 \ s6 \ s7$
 $ap2 \ s8 \ s9$ **where**
 $read-xp: Some \ (xp, s1) \in set-dist \ (execute \ read \ s)$
and $read-xp-path:$
 $Some \ (xp-path, s2) \in$
 $set-dist \ (execute \ (ntimes \ read \ (floor-log \ len)) \ s1)$
and $read-xn: Some \ (xn, s3) \in set-dist \ (execute \ read \ s2)$
and $read-xn-path:$
 $Some \ (xn-path, s4) \in$
 $set-dist \ (execute \ (ntimes \ read \ (floor-log \ len)) \ s3)$
and $assert-xp: Some \ ((), s5) \in set-dist \ (execute \ (assert \ (xp = x)) \ s4)$
and $check1:$
 $Some \ (ap1, s6) \in$
 $set-dist \ (execute \ (check-authentication-path \ len \ i \ xp \ xp-path) \ s5)$
and $assert-ap1: Some \ ((), s7) \in set-dist \ (execute \ (assert \ (ap1 = f)) \ s6)$
and $check2:$
 $Some \ (ap2, s8) \in$
 $set-dist \ (execute$
 $(check-authentication-path \ len \ ((i + len \div 2) \bmod len) \ xn \ xn-path) \ s7)$
and $assert-ap2: Some \ ((), s9) \in set-dist \ (execute \ (assert \ (ap2 = f)) \ s8)$
and $ret:$
 $Some \ (out, t) \in$
 $set-dist \ (execute$
 $(return$
 $(i \bmod (len \div 2),$
 $(xp + xn) \div 2 +$
 $b * ((xp - xn) \div (2 * ((h \wedge i) * shift) \wedge pw)),$
 $len \div 2, pw + pw)) \ s9)$
unfolding $fri-layer-opening-step-def$
by $(auto \ simp: Let-def \ elim!: set-dist-bindE)$
from $read-outcome[OF \ read-xp]$ **obtain** $rest1$ **where**
 $tr-s: PTranscript \ s = xp \# \ rest1$
and $st-s1: PState \ s1 = concat \ (PState \ s) \ xp$
and $tr-s1: PTranscript \ s1 = rest1$
and $ext-s-s1: s \leq s1$

```

and query-count-s1: PQueryCounter s1 = PQueryCounter s
by blast
have xp-path-res:
  length xp-path = floor-log len  $\wedge$ 
  PTranscript s1 = xp-path @ PTranscript s2  $\wedge$ 
  PState s2 = foldl concat (PState s1) xp-path  $\wedge$ 
  s1  $\leq$  s2  $\wedge$ 
  PQueryCounter s2 = PQueryCounter s1
using ntimes-read-any-outcome[OF read-xp-path] by simp
from read-outcome[OF read-xn] obtain rest3 where
  tr-s2: PTranscript s2 = xn # rest3
and st-s3: PState s3 = concat (PState s2) xn
and tr-s3: PTranscript s3 = rest3
and ext-s2-s3: s2  $\leq$  s3
and query-count-s3: PQueryCounter s3 = PQueryCounter s2
by blast
have xn-path-res:
  length xn-path = floor-log len  $\wedge$ 
  PTranscript s3 = xn-path @ PTranscript s4  $\wedge$ 
  PState s4 = foldl concat (PState s3) xn-path  $\wedge$ 
  s3  $\leq$  s4  $\wedge$ 
  PQueryCounter s4 = PQueryCounter s3
using ntimes-read-any-outcome[OF read-xn-path] by simp
have xp-eq: xp = x
using assert-xp unfolding assert-def
by (cases xp = x) (auto simp: throw-no-outcome)
have s5-eq: s5 = s4
using assert-xp xp-eq unfolding assert-def by simp
have s5-s6: s5  $\leq$  s6
by (rule check-authentication-path-hash-extends[OF check1])
have st-s6: PState s6 = PState s5
using check-authentication-path-preserves-channel(1)[OF check1] .
have tr-s6: PTranscript s6 = PTranscript s5
using check-authentication-path-preserves-channel(2)[OF check1] .
have query-count-s6: PQueryCounter s6 = PQueryCounter s5
using check-authentication-path-preserves-channel(6)[OF check1] .
have ap1-eq: ap1 = f
using assert-ap1 unfolding assert-def
by (cases ap1 = f) (auto simp: throw-no-outcome)
have s7-eq: s7 = s6
using assert-ap1 ap1-eq unfolding assert-def by simp
have s7-s8: s7  $\leq$  s8
by (rule check-authentication-path-hash-extends[OF check2])
have st-s8: PState s8 = PState s7
using check-authentication-path-preserves-channel(1)[OF check2] .
have tr-s8: PTranscript s8 = PTranscript s7
using check-authentication-path-preserves-channel(2)[OF check2] .
have query-count-s8: PQueryCounter s8 = PQueryCounter s7
using check-authentication-path-preserves-channel(6)[OF check2] .

```

```

have ap2-eq: ap2 = f
  using assert-ap2 unfolding assert-def
  by (cases ap2 = f) (auto simp: throw-no-outcome)
have s9-eq: s9 = s8
  using assert-ap2 ap2-eq unfolding assert-def by simp
let ?x' =
  (xp + xn) div 2 +
  b * ((xp - xn) div (2 * ((h ^ i) * shift) ^ pw))
have out-t: out = (i mod (len div 2), ?x', len div 2, pw + pw) ∧ t = s9
  using ret by simp
have x'-fold:
  ?x' = fri-fold-value b xp xn
  (fri-fold-denominator ((h ^ i) * shift) pw)
  unfolding fri-fold-value-def fri-fold-denominator-def by simp
have chunk-shape:
  fri-layer-opening-chunk len xp xp-path xn xn-path
  ([xp] @ xp-path @ [xn] @ xn-path)
  using xp-path-res xn-path-res unfolding fri-layer-opening-chunk-def by simp
have tr-t:
  PTranscript s = ([xp] @ xp-path @ [xn] @ xn-path) @ PTranscript t
  using tr-s tr-s1 xp-path-res tr-s2 tr-s3 xn-path-res s5-eq tr-s6
  s7-eq tr-s8 s9-eq out-t
  by simp
have st-t:
  PState t = foldl concat (PState s) ([xp] @ xp-path @ [xn] @ xn-path)
  using st-s1 xp-path-res st-s3 xn-path-res s5-eq st-s6 s7-eq st-s8
  s9-eq out-t
  by simp
have s-t: s ≤ t
  using ext-s-s1 xp-path-res ext-s2-s3 xn-path-res s5-s6 s7-s8 out-t
  unfolding s5-eq s7-eq s9-eq
  by (meson hash-ext-trans)
have query-count-t: PQueryCounter t = PQueryCounter s
  using out-t query-count-s1
  xp-path-res query-count-s3 xn-path-res query-count-s6 query-count-s8
  s5-eq s7-eq s9-eq
  by simp
show ?thesis
  apply (rule exI[where x=xp])
  apply (rule exI[where x=xp-path])
  apply (rule exI[where x=xn])
  apply (rule exI[where x=xn-path])
  apply (rule exI[where x=?x'])
  using out-t xp-eq x'-fold chunk-shape tr-t st-t s-t query-count-t
  by simp
qed

```

lemma *mfold-fri-layer-openings-outcome:*
assumes *outcome:*

```

Some (out, t) ∈
  set-dist (execute (mfold (i, x, len, pw) (map fri-layer-opening-step bfs)) s)
shows
  ∃ layer-chunks chunk.
    fri-layers-transcript (length bfs) len layer-chunks chunk ∧
    PTranscript s = chunk @ PTranscript t ∧
    PState t = foldl concat (PState s) chunk ∧
    s ≤ t ∧
    PQueryCounter t = PQueryCounter s
using outcome
proof (induction bfs arbitrary: i x len pw s out t)
case Nil
then show ?case
  by (intro exI[of - []]) (simp add: fri-layers-transcript-def hash-ext-refl)
next
case (Cons bf bfs)
obtain b f where bf-eq: bf = (b, f)
  by (cases bf)
from Cons.premis obtain out1 s1 where
  head:
    Some (out1, s1) ∈
      set-dist (execute (fri-layer-opening-step bf (i, x, len, pw)) s)
  and tail:
    Some (out, t) ∈
      set-dist (execute (mfold out1 (map fri-layer-opening-step bfs)) s1)
  by (auto elim!: set-dist-bindE)
from fri-layer-opening-step-outcome[OF head[unfolded bf-eq]]
obtain xp xp-path xn xn-path x' where
  out1-eq: out1 = (i mod (len div 2), x', len div 2, pw + pw)
  and head-chunk:
    fri-layer-opening-chunk len xp xp-path xn xn-path
      ([xp] @ xp-path @ [xn] @ xn-path)
  and tr-s: PTranscript s = ([xp] @ xp-path @ [xn] @ xn-path) @ PTranscript s1
  and st-s1:
    PState s1 = foldl concat (PState s) ([xp] @ xp-path @ [xn] @ xn-path)
  and ext-s-s1: s ≤ s1
  and query-count-s1: PQueryCounter s1 = PQueryCounter s
  by blast
from Cons.IH[OF tail[unfolded out1-eq]]
obtain layer-chunks chunk-tail where
  tail-layers:
    fri-layers-transcript (length bfs) (len div 2) layer-chunks chunk-tail
  and tr-s1: PTranscript s1 = chunk-tail @ PTranscript t
  and st-t: PState t = foldl concat (PState s1) chunk-tail
  and ext-s1-t: s1 ≤ t
  and query-count-t: PQueryCounter t = PQueryCounter s1
  by blast
let ?head-chunk = [xp] @ xp-path @ [xn] @ xn-path
let ?chunks = ?head-chunk # layer-chunks

```



```

let ?chunk = ?head-chunk @ chunk-tail
have layers:
  fri-layers-transcript (length (bf # bfs)) len ?chunks ?chunk
  using head-chunk tail-layers
  unfolding fri-layers-transcript-def
  by auto
have s ≤ t
  using ext-s-s1 ext-s1-t by (rule hash-ext-trans)
then show ?case
  apply (intro exI[of - ?chunks] exI[of - ?chunk])
  using layers tr-s tr-s1 st-s1 st-t query-count-s1 query-count-t
  by simp
qed

lemma receive-query-commits-layers-outcome:
  assumes outcome:
    Some (out, t) ∈
      set-dist (execute (mfold (i, x, len, pw) (receive-query-commits bfs)) s)
  shows
    ∃ layer-chunks chunk.
      fri-layers-transcript (length bfs) len layer-chunks chunk ∧
      PTranscript s = chunk @ PTranscript t ∧
      PState t = foldl concat (PState s) chunk ∧
      s ≤ t ∧
      PQueryCounter t = PQueryCounter s
  proof -
    have map-eq: receive-query-commits bfs = map fri-layer-opening-step bfs
      unfolding receive-query-commits-def fri-layer-opening-step-def
      by (simp add: fun-eq-iff split: prod.splits)
    show ?thesis
      using mfold-fri-layer-openings-outcome[OF outcome[unfolded map-eq]] .
  qed

lemma fri-layer-opening-step-preserves-query-lookup:
  assumes outcome:
    Some (out, t) ∈ set-dist (execute (fri-layer-opening-step bf st) s)
  shows
    fmlookup (HashMap t) (QueryIndexChallenge j x) =
      fmlookup (HashMap s) (QueryIndexChallenge j x)
  proof -
    obtain b f where bf-eq: bf = (b, f)
      by (cases bf)
    obtain i0 x0 len pw where st-eq: st = (i0, x0, len, pw)
      by (cases st)
    from outcome[unfolded bf-eq st-eq fri-layer-opening-step-def]
    obtain xp s1 xp-path s2 xn s3 xn-path s4 s5 ap1 s6 s7 ap2 s8 s9 where
      read-xp: Some (xp, s1) ∈ set-dist (execute read s)
    and read-xp-path:
      Some (xp-path, s2) ∈

```

```

    set-dist (execute (ntimes read (floor-log len)) s1)
and read-xn: Some (xn, s3) ∈ set-dist (execute read s2)
and read-xn-path:
    Some (xn-path, s4) ∈
    set-dist (execute (ntimes read (floor-log len)) s3)
and assert-xp:
    Some ((), s5) ∈ set-dist (execute (assert (xp = x0)) s4)
and check1:
    Some (ap1, s6) ∈
    set-dist (execute (check-authentication-path len i0 xp xp-path) s5)
and assert-ap1:
    Some ((), s7) ∈ set-dist (execute (assert (ap1 = f)) s6)
and check2:
    Some (ap2, s8) ∈
    set-dist (execute
      (check-authentication-path len ((i0 + len div 2) mod len) xn
        xn-path) s7)
and assert-ap2:
    Some ((), s9) ∈ set-dist (execute (assert (ap2 = f)) s8)
and ret:
    Some (out, t) ∈
    set-dist
      (execute
        (return (i0 mod (len div 2),
          (xp + xn) div 2 +
            b * ((xp - xn) div (2 * ((h ^ i0) * shift) ^ pw)),
          len div 2, pw + pw)) s9)
by (auto elim!: set-dist-bindE)
have lookup-s1:
  fmlookup (HashMap s1) (QueryIndexChallenge j x) =
  fmlookup (HashMap s) (QueryIndexChallenge j x)
using read-preserves-hash-map[OF read-xp] by simp
have lookup-s2:
  fmlookup (HashMap s2) (QueryIndexChallenge j x) =
  fmlookup (HashMap s1) (QueryIndexChallenge j x)
by (rule ntimes-read-preserves-lookup[OF read-xp-path])
have lookup-s3:
  fmlookup (HashMap s3) (QueryIndexChallenge j x) =
  fmlookup (HashMap s2) (QueryIndexChallenge j x)
using read-preserves-hash-map[OF read-xn] by simp
have lookup-s4:
  fmlookup (HashMap s4) (QueryIndexChallenge j x) =
  fmlookup (HashMap s3) (QueryIndexChallenge j x)
by (rule ntimes-read-preserves-lookup[OF read-xn-path])
have s5-eq: s5 = s4
using assert-xp unfolding assert-def
by (cases xp = x0) (auto simp: throw-no-outcome)
have lookup-s6:
  fmlookup (HashMap s6) (QueryIndexChallenge j x) =

```

```

    fmlookup (HashMap s5) (QueryIndexChallenge j x)
  by (rule check-authentication-path-preserves-challenge-lookups(2)
    [OF check1])
  have s7-eq: s7 = s6
    using assert-ap1 unfolding assert-def
    by (cases ap1 = f) (auto simp: throw-no-outcome)
  have lookup-s8:
    fmlookup (HashMap s8) (QueryIndexChallenge j x) =
      fmlookup (HashMap s7) (QueryIndexChallenge j x)
    by (rule check-authentication-path-preserves-challenge-lookups(2)
      [OF check2])
  have s9-eq: s9 = s8
    using assert-ap2 unfolding assert-def
    by (cases ap2 = f) (auto simp: throw-no-outcome)
  have t-eq: t = s9
    using ret by simp
  show ?thesis
    using lookup-s1 lookup-s2 lookup-s3 lookup-s4 lookup-s6 lookup-s8
      s5-eq s7-eq s9-eq t-eq
    by simp
qed

lemma mfold-fri-layer-openings-preserves-query-lookup:
  assumes outcome:
    Some (out, t) ∈
      set-dist (execute (mfold st (map fri-layer-opening-step bfs)) s)
  shows
    fmlookup (HashMap t) (QueryIndexChallenge j x) =
      fmlookup (HashMap s) (QueryIndexChallenge j x)
  using outcome
proof (induction bfs arbitrary: st s out t)
  case Nil
  then show ?case
    by simp
next
  case (Cons bf bfs)
  from Cons.prem obtain out1 s1 where
    head:
      Some (out1, s1) ∈
        set-dist (execute (fri-layer-opening-step bf st) s)
  and tail:
    Some (out, t) ∈
      set-dist (execute (mfold out1 (map fri-layer-opening-step bfs)) s1)
  by (auto elim!: set-dist-bindE)
  have head-pres:
    fmlookup (HashMap s1) (QueryIndexChallenge j x) =
      fmlookup (HashMap s) (QueryIndexChallenge j x)
    by (rule fri-layer-opening-step-preserves-query-lookup[OF head])
  have tail-pres:

```

```

    fmlookup (HashMap t) (QueryIndexChallenge j x) =
      fmlookup (HashMap s1) (QueryIndexChallenge j x)
  by (rule Cons.IH[OF tail])
show ?case
  using head-pres tail-pres by simp
qed

```

lemma *receive-query-commits-layers-preserves-query-lookup:*
assumes *outcome:*
Some (out, t) ∈
set-dist (execute (mfold st (receive-query-commits bfs)) s)
shows
fmlookup (HashMap t) (QueryIndexChallenge j x) =
fmlookup (HashMap s) (QueryIndexChallenge j x)
proof –
have *map-eq: receive-query-commits bfs = map fri-layer-opening-step bfs*
unfolding *receive-query-commits-def fri-layer-opening-step-def*
by (*simp add: fun-eq-iff split: prod.splits*)
show *?thesis*
by (*rule mfold-fri-layer-openings-preserves-query-lookup*
[OF outcome[unfolded map-eq]])
qed

definition *verifier-query-round-after-index-program*
where
verifier-query-round-after-index-program fr f-fl f-final as fl final raw =
(let idx' = index (to-nat raw) in
do {
fv ← mmap (check-decommit-on-query fr idx');

(f-i, f-x, f-len, f-pow) ←
*mfold (idx', hd fv, clength * scale, 1)*
(receive-query-commits f-fl);
assert (f-x = f-final);

(i, x, len, pow) ←
*mfold (idx', cp-eval as fv (h^{idx'} * shift), clength * scale, 1)*
(receive-query-commits fl);

assert (x = final)
})

definition *verifier-query-round-program*
where
verifier-query-round-program fr f-fl f-final as fl final =
(do {
idx ← receive-query-index-challenge;
let idx' = index (to-nat idx);
fv ← mmap (check-decommit-on-query fr idx');

```

(f-i, f-x, f-len, f-pow) ←
  mfold (idx', hd fv, clength * scale, 1) (receive-query-commits f-fl);
assert (f-x = f-final);

(i, x, len, pow) ←
  mfold (idx', cp-eval as fv (h ^ idx' * shift), clength * scale, 1)
    (receive-query-commits fl);

assert (x = final)
})

```

lemma *verifier-query-round-program-alt-def*:
verifier-query-round-program fr f-fl f-final as fl final =
 (receive-query-index-challenge >>=
 verifier-query-round-after-index-program fr f-fl f-final as fl final)
unfolding *verifier-query-round-program-def*
verifier-query-round-after-index-program-def
by (simp add: Let-def)

lemma *verifier-query-round-after-index-program-outcome*:
assumes *outcome*:
 Some ((), t) ∈
 set-dist (execute
 (verifier-query-round-after-index-program fr f-fl f-final as fl final raw) s)
shows s ≤ t ∧ PQueryCounter t = PQueryCounter s
proof –
let ?idx = index (to-nat raw)
from *outcome* **obtain** fv s1 f-out s2 s3 c-out s4 **where**
query-decommit:
 Some (fv, s1) ∈
 set-dist (execute (mmap (check-decommit-on-query fr ?idx)) s)
and *trace-fri*:
 Some (f-out, s2) ∈
 set-dist (execute
 (mfold (?idx, hd fv, clength * scale, 1)
 (receive-query-commits f-fl)) s1)
and *assert-trace*:
 Some ((), s3) ∈
 set-dist
 (execute
 (assert (case f-out of (_, f-x, -, -) ⇒ f-x = f-final)) s2)
and *comp-fri*:
 Some (c-out, s4) ∈
 set-dist (execute
 (mfold (?idx, cp-eval as fv (h ^ ?idx * shift),
 clength * scale, 1)
 (receive-query-commits fl)) s3)
and *assert-comp*:

```

Some (((), t) ∈
  set-dist
    (execute
      (assert (case c-out of (-, x, -, -) ⇒ x = final)) s4)
    unfolding verifier-query-round-after-index-program-def
    by (auto simp: Let-def split: prod.splits elim!: set-dist-bindE)
  from check-decommit-on-query-outcome[OF query-decommit]
  obtain query-paths query-chunk where
    ext-s-s1: s ≤ s1
    and query-count-s1: PQueryCounter s1 = PQueryCounter s
    by blast
  from receive-query-commits-layers-outcome[OF trace-fri]
  obtain trace-layer-chunks trace-fri-chunk where
    ext-s1-s2: s1 ≤ s2
    and query-count-s2: PQueryCounter s2 = PQueryCounter s1
    by blast
  have s3-eq: s3 = s2
    using assert-trace unfolding assert-def
    by (cases case f-out of (-, f-x, -, -) ⇒ f-x = f-final)
      (auto simp: throw-no-outcome)
  from receive-query-commits-layers-outcome[OF comp-fri]
  obtain comp-layer-chunks comp-fri-chunk where
    ext-s3-s4: s3 ≤ s4
    and query-count-s4: PQueryCounter s4 = PQueryCounter s3
    by blast
  have t-eq: t = s4
    using assert-comp unfolding assert-def
    by (cases case c-out of (-, x, -, -) ⇒ x = final)
      (auto simp: throw-no-outcome)
  have s ≤ t
    using ext-s-s1 ext-s1-s2 ext-s3-s4 t-eq unfolding s3-eq
    by (meson hash-ext-trans)
  moreover have PQueryCounter t = PQueryCounter s
    using query-count-s1 query-count-s2 query-count-s4 s3-eq t-eq by simp
  ultimately show ?thesis
    by simp
qed

```

lemma *verifier-query-round-after-index-program-preserves-query-lookup:*
assumes *outcome:*
Some (((), t) ∈
set-dist (execute
(verifier-query-round-after-index-program fr f-fl f-final as fl final raw) s)
shows
fmlookup (HashMap t) (QueryIndexChallenge j x) =
fmlookup (HashMap s) (QueryIndexChallenge j x)
proof –
let *?idx = index (to-nat raw)*
from *outcome* **obtain** *fv s1 f-out s2 s3 c-out s4* **where**

```

query-decommit:
  Some (fv, s1) ∈
    set-dist (execute (mmap (check-decommit-on-query fr ?idx)) s)
and trace-fri:
  Some (f-out, s2) ∈
    set-dist (execute
      (mfold (?idx, hd fv, clength * scale, 1)
        (receive-query-commits f-fl)) s1)
and assert-trace:
  Some ((), s3) ∈
    set-dist
      (execute
        (assert (case f-out of (-, f-x, -, -) ⇒ f-x = f-final)) s2)
and comp-fri:
  Some (c-out, s4) ∈
    set-dist (execute
      (mfold (?idx, cp-eval as fv (h ^ ?idx * shift),
        clength * scale, 1)
        (receive-query-commits fl)) s3)
and assert-comp:
  Some ((), t) ∈
    set-dist
      (execute
        (assert (case c-out of (-, x, -, -) ⇒ x = final)) s4)
unfolding verifier-query-round-after-index-program-def
by (auto simp: Let-def split: prod.splits elim!: set-dist-bindE)
have lookup-s1:
  fmlookup (HashMap s1) (QueryIndexChallenge j x) =
  fmlookup (HashMap s) (QueryIndexChallenge j x)
by (rule check-decommit-on-query-preserves-query-lookup
  [OF query-decommit])
have lookup-s2:
  fmlookup (HashMap s2) (QueryIndexChallenge j x) =
  fmlookup (HashMap s1) (QueryIndexChallenge j x)
by (rule receive-query-commits-layers-preserves-query-lookup
  [OF trace-fri])
have s3-eq: s3 = s2
using assert-trace unfolding assert-def
by (cases case f-out of (-, f-x, -, -) ⇒ f-x = f-final)
  (auto simp: throw-no-outcome)
have lookup-s4:
  fmlookup (HashMap s4) (QueryIndexChallenge j x) =
  fmlookup (HashMap s3) (QueryIndexChallenge j x)
by (rule receive-query-commits-layers-preserves-query-lookup
  [OF comp-fri])
have t-eq: t = s4
using assert-comp unfolding assert-def
by (cases case c-out of (-, x, -, -) ⇒ x = final)
  (auto simp: throw-no-outcome)

```

```

show ?thesis
  using lookup-s1 lookup-s2 lookup-s4 s3-eq t-eq by simp
qed

lemma receive-query-index-challenge-preserves-query-future-fresh:
  assumes future: query-future-fresh s
  and outcome:
    Some (raw, t) ∈ set-dist (execute receive-query-index-challenge s)
  shows query-future-fresh t
proof –
  have counter-t: PQueryCounter t = Suc (PQueryCounter s)
  using receive-query-index-challenge-counter-outcome[OF outcome] by simp
  show ?thesis
  unfolding query-future-fresh-def
  proof (intro allI impI)
  fix i x
  assume i-ge-t: PQueryCounter t ≤ i
  have i-ge-s: PQueryCounter s ≤ i
  using i-ge-t counter-t by simp
  have neg:
    QueryIndexChallenge i x ≠
      QueryIndexChallenge (PQueryCounter s) (PState s)
  using i-ge-t counter-t by auto
  have fmlookup (HashMap t) (QueryIndexChallenge i x) =
    fmlookup (HashMap s) (QueryIndexChallenge i x)
  by (rule receive-query-index-challenge-preserves-other-lookup
    [OF outcome neg])
  also have ... = None
  using future i-ge-s unfolding query-future-fresh-def by blast
  finally show fmlookup (HashMap t) (QueryIndexChallenge i x) = None .
qed
qed

lemma verifier-query-round-program-preserves-query-future-fresh:
  assumes future: query-future-fresh s
  and outcome:
    Some (( ), t) ∈
      set-dist
        (execute
          (verifier-query-round-program fr f-fl f-final as fl final) s)
  shows query-future-fresh t
proof –
  from outcome[unfolded verifier-query-round-program-alt-def]
  obtain raw s0 where
    rand:
      Some (raw, s0) ∈
        set-dist (execute receive-query-index-challenge s)
  and tail:
    Some (( ), t) ∈

```



```

    set-dist
    (execute
      (verifier-query-round-after-index-program fr f-fl f-final as fl
        final raw) s0)
  by (auto elim!: set-dist-bindE)
have future-s0: query-future-fresh s0
  by (rule receive-query-index-challenge-preserves-query-future-fresh
    [OF future rand])
have counter-t: PQueryCounter t = PQueryCounter s0
  using verifier-query-round-after-index-program-outcome[OF tail] by simp
show ?thesis
  unfolding query-future-fresh-def
proof (intro allI impI)
  fix i x
  assume i-ge-t: PQueryCounter t ≤ i
  have i-ge-s0: PQueryCounter s0 ≤ i
    using i-ge-t counter-t by simp
  have fmlookup (HashMap t) (QueryIndexChallenge i x) =
    fmlookup (HashMap s0) (QueryIndexChallenge i x)
    by (rule verifier-query-round-after-index-program-preserves-query-lookup
      [OF tail])
  also have ... = None
    using future-s0 i-ge-s0 unfolding query-future-fresh-def by blast
  finally show fmlookup (HashMap t) (QueryIndexChallenge i x) = None .
qed
qed

```

lemma *ntimes-verifier-query-round-program-preserves-query-future-fresh:*

```

assumes outcome:
  Some (results, t) ∈
    set-dist
    (execute
      (ntimes (verifier-query-round-program fr f-fl f-final as fl final)
        n) s)
and future: query-future-fresh s
shows query-future-fresh t
using outcome future
proof (induction n arbitrary: s results t)
case 0
then show ?case
  by simp
next
case (Suc n)
from Suc.premis obtain u results' s1 where
  head:
    Some (u, s1) ∈
      set-dist
      (execute
        (verifier-query-round-program fr f-fl f-final as fl final) s)

```

```

and tail:
  Some (results', t) ∈
    set-dist
      (execute
        (ntimes (verifier-query-round-program fr f-fl f-final as fl final)
          n) s1)
  by (auto elim!: set-dist-bindE)
have u-eq: u = ()
  by (cases u) simp
have future-s1: query-future-fresh s1
  by (rule verifier-query-round-program-preserves-query-future-fresh
    [OF Suc.prems(2) head[unfolded u-eq]])
show ?case
  by (rule Suc.IH[OF tail future-s1])
qed

lemma verifier-query-round-program-outcome:
assumes outcome:
  Some ((), t) ∈
    set-dist (execute (verifier-query-round-program fr f-fl f-final as fl final) s)
obtains raw-idx idx chunk
where
  idx = index (to-nat raw-idx)
  and verifier-query-round-chunk idx (map snd f-fl) (map snd fl) chunk
  and PTranscript s = chunk @ PTranscript t
  and PState t = foldl concat (PState s) chunk
  and s ≤ t
  and PQueryCounter t = Suc (PQueryCounter s)
  and fmlookup (HashMap t) (QueryIndexChallenge (PQueryCounter s) (PState s)) = Some
raw-idx
proof –
from outcome obtain raw-idx s0 idx fv s1 f-out s2 s3 c-out s4 where
  rand:
    Some (raw-idx, s0) ∈
      set-dist (execute receive-query-index-challenge s)
  and idx-def: idx = index (to-nat raw-idx)
  and query-decommit:
    Some (fv, s1) ∈
      set-dist (execute (mmap (check-decommit-on-query fr idx)) s0)
  and trace-fri:
    Some (f-out, s2) ∈
      set-dist (execute
        (mfold (idx, hd fv, clength * scale, 1) (receive-query-commits f-fl)) s1)
  and assert-trace:
    Some ((), s3) ∈ set-dist (execute (assert (case f-out of (-, f-x, -, -) ⇒ f-x = f-final)) s2)
  and comp-fri:
    Some (c-out, s4) ∈
      set-dist (execute
        (mfold

```

```

      (idx, cp-eval as fv (h ^ idx * shift), clength * scale, 1)
      (receive-query-commits fl)) s3)
and assert-comp:
  Some (((), t) ∈ set-dist (execute (assert (case c-out of (-, x, -, -) ⇒ x = final)) s4))
unfolding verifier-query-round-program-def
by (auto simp: Let-def split: prod.splits elim!: set-dist-bindE)
have rand-res:
  s ≤ s0 ∧
  PState s0 = PState s ∧
  PTranscript s0 = PTranscript s ∧
  fmllookup (HashMap s0) (QueryIndexChallenge (PQueryCounter s) (PState s)) = Some
raw-idx
  using receive-query-index-challenge-outcome[OF rand] .
have rand-counter:
  PQueryCounter s0 = Suc (PQueryCounter s)
  using receive-query-index-challenge-counter-outcome[OF rand] by simp
from check-decommit-on-query-outcome[OF query-decommit]
obtain query-paths query-chunk where
  query-chunk-shape:
    query-decommitment-transcript idx fv query-paths query-chunk
  and tr-s0: PTranscript s0 = query-chunk @ PTranscript s1
  and st-s1: PState s1 = foldl concat (PState s0) query-chunk
  and ext-s0-s1: s0 ≤ s1
  and query-count-s1: PQueryCounter s1 = PQueryCounter s0
  by blast
from receive-query-commits-layers-outcome[OF trace-fri]
obtain trace-layer-chunks trace-fri-chunk where
  trace-layers:
    fri-layers-transcript (length f-fl) (clength * scale)
    trace-layer-chunks trace-fri-chunk
  and tr-s1: PTranscript s1 = trace-fri-chunk @ PTranscript s2
  and st-s2: PState s2 = foldl concat (PState s1) trace-fri-chunk
  and ext-s1-s2: s1 ≤ s2
  and query-count-s2: PQueryCounter s2 = PQueryCounter s1
  by blast
have s3-eq: s3 = s2
  using assert-trace unfolding assert-def
  by (cases case f-out of (-, f-x, -, -) ⇒ f-x = f-final)
    (auto simp: throw-no-outcome)
from receive-query-commits-layers-outcome[OF comp-fri]
obtain comp-layer-chunks comp-fri-chunk where
  comp-layers:
    fri-layers-transcript (length fl) (clength * scale)
    comp-layer-chunks comp-fri-chunk
  and tr-s3: PTranscript s3 = comp-fri-chunk @ PTranscript s4
  and st-s4: PState s4 = foldl concat (PState s3) comp-fri-chunk
  and ext-s3-s4: s3 ≤ s4
  and query-count-s4: PQueryCounter s4 = PQueryCounter s3
  by blast

```

```

have t-eq: t = s4
  using assert-comp unfolding assert-def
  by (cases case c-out of (-, x, -, -) ⇒ x = final)
    (auto simp: throw-no-outcome)
let ?chunk = query-chunk @ trace-fri-chunk @ comp-fri-chunk
have round-chunk:
  verifier-query-round-chunk idx (map snd f-fl) (map snd fl) ?chunk
  unfolding verifier-query-round-chunk-def
  apply (rule exI[where x=query-chunk])
  apply (rule exI[where x=trace-fri-chunk])
  apply (rule exI[where x=comp-fri-chunk])
  apply (rule exI[where x=fv])
  apply (rule exI[where x=query-paths])
  apply (rule exI[where x=trace-layer-chunks])
  apply (rule exI[where x=comp-layer-chunks])
  using query-chunk-shape trace-layers comp-layers
  by simp
have tr-t: PTranscript s = ?chunk @ PTranscript t
  using rand-res tr-s0 tr-s1 tr-s3 s3-eq t-eq by simp
have st-t: PState t = foldl concat (PState s) ?chunk
  using rand-res st-s1 st-s2 st-s4 s3-eq t-eq by simp
have ext-s0-t: s0 ≤ t
  using ext-s0-s1 ext-s1-s2 ext-s3-s4 t-eq unfolding s3-eq
  by (meson hash-ext-trans)
have ext-s-t: s ≤ t
  using rand-res ext-s0-t by (meson hash-ext-trans)
have lookup-t: fmllookup (HashMap t) (QueryIndexChallenge (PQueryCounter s) (PState s))
= Some raw-idx
  using rand-res hash-extension-lookup[OF conjunct2[OF conjunct2[OF conjunct2[OF rand-res]]]
ext-s0-t]
  by simp
have query-count-t: PQueryCounter t = Suc (PQueryCounter s)
  using rand-counter query-count-s1 query-count-s2 query-count-s4 s3-eq t-eq
  by simp
show ?thesis
  by (rule that[OF idx-def round-chunk tr-t st-t ext-s-t query-count-t lookup-t])
qed

```

```

lemma ntimes-verifier-query-rounds-outcome:
  assumes outcome:
    Some (results, t) ∈
      set-dist (execute
        (ntimes (verifier-query-round-program fr f-fl f-final as fl final) n) s)
  shows
    ∃ raw-idxs query-idxs query-chunks.
      length raw-idxs = n ∧
      query-idxs = map (λraw. index (to-nat raw)) raw-idxs ∧
      length query-chunks = n ∧
      PTranscript s = List.concat query-chunks @ PTranscript t ∧

```

```

PState t = state-after-query-chunks (PState s) query-chunks n ∧
s ≤ t ∧
PQueryCounter t = PQueryCounter s + n ∧
(∀ i < n.
  verifier-query-round-chunk (query-idxs ! i)
    (map snd f-fl) (map snd fl) (query-chunks ! i)) ∧
(∀ i < n.
  fmllookup (HashMap t)
    (QueryIndexChallenge (PQueryCounter s + i)
      (state-after-query-chunks (PState s) query-chunks i)) =
    Some (raw-idxs ! i)) ∧
(∀ idx ∈ set query-idxs. idx < clength * scale)
using outcome
proof (induction n arbitrary: s results t)
case 0
then show ?case
  by (intro exI[of - []]) (simp add: state-after-query-chunks-def hash-ext-refl)
next
case (Suc n)
from Suc.premis obtain u results' s1 where
  head:
    Some (u, s1) ∈
      set-dist (execute (verifier-query-round-program fr f-fl f-final as fl final) s)
  and tail:
    Some (results', t) ∈
      set-dist (execute
        (ntimes (verifier-query-round-program fr f-fl f-final as fl final) n) s1)
  and results-eq: results = u # results'
  by (auto elim!: set-dist-bindE)
have u-unit: u = ()
  by (cases u) simp
from verifier-query-round-program-outcome[OF head[unfolded u-unit]]
obtain raw idx chunk where
  idx-def: idx = index (to-nat raw)
  and round-chunk:
    verifier-query-round-chunk idx (map snd f-fl) (map snd fl) chunk
  and tr-s: PTranscript s = chunk @ PTranscript s1
  and st-s1: PState s1 = foldl concat (PState s) chunk
  and ext-s-s1: s ≤ s1
  and query-count-s1: PQueryCounter s1 = Suc (PQueryCounter s)
  and lookup-s1: fmllookup (HashMap s1) (QueryIndexChallenge (PQueryCounter s) (PState
s)) = Some raw
  by this
from Suc.IH[OF tail] obtain raw-tail idx-tail chunk-tail where
  len-raw-tail: length raw-tail = n
  and idx-tail-def: idx-tail = map (λraw. index (to-nat raw)) raw-tail
  and len-chunk-tail: length chunk-tail = n
  and tr-s1: PTranscript s1 = List.concat chunk-tail @ PTranscript t
  and st-t-tail:

```

```

  PState t = state-after-query-chunks (PState s1) chunk-tail n
and ext-s1-t: s1 ≤ t
and query-count-t: PQueryCounter t = PQueryCounter s1 + n
and round-tail:
  ∧ i. i < n ⇒
    verifier-query-round-chunk (idx-tail ! i)
    (map snd f-fl) (map snd fl) (chunk-tail ! i)
and lookup-tail:
  ∧ i. i < n ⇒
    fmllookup (HashMap t)
    (QueryIndexChallenge (PQueryCounter s1 + i)
      (state-after-query-chunks (PState s1) chunk-tail i)) =
    Some (raw-tail ! i)
and idx-tail-bound:
  ∀ idx ∈ set idx-tail. idx < clength * scale
by blast
let ?raws = raw # raw-tail
let ?idxs = idx # idx-tail
let ?chunks = chunk # chunk-tail
have idxs-def: ?idxs = map (λ raw. index (to-nat raw)) ?raws
  using idx-def idx-tail-def by simp
have tr-all: PTranscript s = List.concat ?chunks @ PTranscript t
  using tr-s tr-s1 by simp
have state-shift:
  ∧ i. state-after-query-chunks (PState s) ?chunks (Suc i) =
    state-after-query-chunks (PState s1) chunk-tail i
  using st-s1 unfolding state-after-query-chunks-def by simp
have st-all:
  PState t = state-after-query-chunks (PState s) ?chunks (Suc n)
  using st-t-tail state-shift[of n] by simp
have ext-s-t: s ≤ t
  using ext-s-s1 ext-s1-t by (rule hash-ext-trans)
have query-count-all: PQueryCounter t = PQueryCounter s + Suc n
  using query-count-s1 query-count-t by simp
have round-all:
  ∧ i. i < Suc n ⇒
    verifier-query-round-chunk (?idxs ! i)
    (map snd f-fl) (map snd fl) (?chunks ! i)
proof -
  fix i
  assume i-bound: i < Suc n
  show verifier-query-round-chunk (?idxs ! i)
    (map snd f-fl) (map snd fl) (?chunks ! i)
  proof (cases i)
    case 0
    then show ?thesis
      using round-chunk by simp
  next
    case (Suc j)

```

```

    then show ?thesis
      using round-tail[of j] i-bound by simp
  qed
qed
have lookup-head-t:
  fmlookup (HashMap t) (QueryIndexChallenge (PQueryCounter s) (PState s)) = Some raw
  using hash-extension-lookup[OF lookup-s1 ext-s1-t] .
have lookup-all:
   $\wedge i. i < \text{Suc } n \implies$ 
  fmlookup (HashMap t)
    (QueryIndexChallenge (PQueryCounter s + i)
      (state-after-query-chunks (PState s) ?chunks i)) =
    Some (?raws ! i)
proof -
  fix i
  assume i-bound: i < Suc n
  show fmlookup (HashMap t)
    (QueryIndexChallenge (PQueryCounter s + i)
      (state-after-query-chunks (PState s) ?chunks i)) =
    Some (?raws ! i)
  proof (cases i)
    case 0
    then show ?thesis
      using lookup-head-t unfolding state-after-query-chunks-def by simp
  next
    case (Suc j)
    then show ?thesis
      using lookup-tail[of j] i-bound state-shift[of j] query-count-s1 by simp
  qed
qed
have idx-bound-all:
   $\forall \text{idx} \in \text{set } ?\text{idxs}. \text{idx} < \text{clength} * \text{scale}$ 
  using idx-tail-bound idx-def index-less-domain by auto
show ?case
  apply (intro exI[of - ?raws] exI[of - ?idxs] exI[of - ?chunks] conjI)
  apply (simp add: len-raw-tail)
  apply (rule idxs-def)
  apply (simp add: len-chunk-tail)
  apply (rule tr-all)
  apply (rule st-all)
  apply (rule ext-s-t)
  apply (rule query-count-all)
  apply (intro allI impI, rule round-all, assumption)
  apply (intro allI impI, rule lookup-all, assumption)
  using idx-bound-all by simp
qed

lemma verifier-query-round-program-preserves-past-query-lookup:
  assumes past: j < PQueryCounter s

```

```

and outcome:
  Some ((), t) ∈
    set-dist
      (execute
        (verifier-query-round-program fr f-fl f-final as fl final) s)
shows
  fmlookup (HashMap t) (QueryIndexChallenge j x) =
    fmlookup (HashMap s) (QueryIndexChallenge j x)
proof –
from outcome[unfolded verifier-query-round-program-alt-def]
obtain raw s0 where
  rand:
    Some (raw, s0) ∈
      set-dist (execute receive-query-index-challenge s)
and tail:
  Some ((), t) ∈
    set-dist
      (execute
        (verifier-query-round-after-index-program fr f-fl f-final as fl
          final raw) s0)
by (auto elim!: set-dist-bindE)
have neg:
  QueryIndexChallenge j x ≠
    QueryIndexChallenge (PQueryCounter s) (PState s)
using past by auto
have lookup-s0:
  fmlookup (HashMap s0) (QueryIndexChallenge j x) =
    fmlookup (HashMap s) (QueryIndexChallenge j x)
by (rule receive-query-index-challenge-preserves-other-lookup
  [OF rand neg])
have lookup-t:
  fmlookup (HashMap t) (QueryIndexChallenge j x) =
    fmlookup (HashMap s0) (QueryIndexChallenge j x)
by (rule verifier-query-round-after-index-program-preserves-query-lookup
  [OF tail])
show ?thesis
  using lookup-s0 lookup-t by simp
qed

```

```

lemma ntimes-verifier-query-round-program-preserves-past-query-lookup:
assumes past: j < PQueryCounter s
and outcome:
  Some (results, t) ∈
    set-dist
      (execute
        (ntimes (verifier-query-round-program fr f-fl f-final as fl final)
          n) s)
shows
  fmlookup (HashMap t) (QueryIndexChallenge j x) =

```



```

      fmlookup (HashMap s) (QueryIndexChallenge j x)
    using outcome past
  proof (induction n arbitrary: s results t)
    case 0
    then show ?case
      by simp
  next
    case (Suc n)
    from Suc.prem1 obtain u results' s1 where
      head:
        Some (u, s1) ∈
          set-dist
            (execute
              (verifier-query-round-program fr f-fl f-final as fl final) s)
      and tail:
        Some (results', t) ∈
          set-dist
            (execute
              (ntimes (verifier-query-round-program fr f-fl f-final as fl final)
                n) s1)
      by (auto elim!: set-dist-bindE)
    have u-eq: u = ()
      by (cases u) simp
    have head-pres:
      fmlookup (HashMap s1) (QueryIndexChallenge j x) =
        fmlookup (HashMap s) (QueryIndexChallenge j x)
      by (rule verifier-query-round-program-preserves-past-query-lookup
        [OF Suc.prem2(2) head[unfolded u-eq]])
    have counter-s1:
      PQueryCounter s1 = Suc (PQueryCounter s)
      using verifier-query-round-program-outcome[OF head[unfolded u-eq]]
      by blast
    have past-s1: j < PQueryCounter s1
      using Suc.prem1(2) counter-s1 by simp
    have tail-pres:
      fmlookup (HashMap t) (QueryIndexChallenge j x) =
        fmlookup (HashMap s1) (QueryIndexChallenge j x)
      by (rule Suc.IH[OF tail past-s1])
    show ?case
      using head-pres tail-pres by simp
  qed

lemma receive-fri-commits-outcome:
  assumes outcome: Some ((b, r), t) ∈ set-dist (execute receive-fri-commits s)
  shows
    ∃ rest.
      PTranscript s = r # rest ∧
      PState t = concat (PState s) r ∧
      PTranscript t = rest ∧

```

```

     $s \leq t \wedge$ 
     $\text{fmlookup } (\text{HashMap } t) (\text{FiatShamirChallenge } (\text{concat } (\text{PState } s) r)) = \text{Some } b$ 
proof –
from outcome obtain s-read where
  read-r:  $\text{Some } (r, s\text{-read}) \in \text{set-dist } (\text{execute read } s)$ 
and rand-b:
   $\text{Some } (b, t) \in \text{set-dist } (\text{execute receive-random-field-element } s\text{-read})$ 
unfolding receive-fri-commits-def
by (auto elim!: set-dist-bindE)
obtain rest where tr-s:  $\text{PTranscript } s = r \# \text{rest}$ 
proof –
  have  $\text{PTranscript } s \neq []$ 
  using read-r unfolding read-def protocol-read-def assert-def
  by (cases PTranscript s) (auto simp: throw-no-outcome elim!: set-dist-bindE)
then obtain x xs where xs:  $\text{PTranscript } s = x \# xs$ 
  by (cases PTranscript s) auto
have r-x:  $r = x$ 
  using read-nonempty-outcome[OF read-r xs] by simp
show ?thesis
  using that xs r-x by auto
qed
have read-res:
   $r = r \wedge$ 
   $s\text{-read} = s(\text{PState} := \text{concat } (\text{PState } s) r, \text{PTranscript} := \text{rest})$ 
  using read-nonempty-outcome[OF read-r tr-s] by simp
have rand-res:
   $s\text{-read} \leq t \wedge$ 
   $\text{PState } t = \text{PState } s\text{-read} \wedge$ 
   $\text{PTranscript } t = \text{PTranscript } s\text{-read} \wedge$ 
   $\text{fmlookup } (\text{HashMap } t) (\text{FiatShamirChallenge } (\text{PState } s\text{-read})) = \text{Some } b$ 
  using receive-random-field-element-outcome[OF rand-b] .
have s-read-ext:  $s \leq s\text{-read}$ 
  using read-res unfolding less-eq-hash-ext-def less-eq-fmap-def by simp
have s-t:  $s \leq t$ 
  using s-read-ext rand-res by (meson hash-ext-trans)
show ?thesis
  using tr-s read-res rand-res s-t by auto
qed

```

lemma *ntimes-receive-fri-commits-outcome*:

assumes *outcome*:

$\text{Some } (brs, t) \in$
 $\text{set-dist } (\text{execute } (\text{ntimes receive-fri-commits } n) s)$

shows

$\text{length } brs = n \wedge$
 $\text{PTranscript } s = \text{map snd } brs @ \text{PTranscript } t \wedge$
 $\text{PState } t = \text{foldl concat } (\text{PState } s) (\text{map snd } brs) \wedge$
 $s \leq t \wedge$
 $(\forall i < n.$

```

    fmlookup (HashMap t)
      (FiatShamirChallenge (foldl concat (PState s) (take (Suc i) (map snd brs)))) =
      Some (fst (brs ! i)))
  using outcome
proof (induction n arbitrary: s brs t)
  case 0
  then show ?case
    by (simp add: hash-ext-refl)
next
  case (Suc n)
  from Suc.premis obtain br brs' s1 where
    head: Some (br, s1) ∈ set-dist (execute receive-fri-commits s)
    and tail:
      Some (brs', t) ∈
        set-dist (execute (ntimes receive-fri-commits n) s1)
    and brs-eq: brs = br # brs'
    by (auto elim!: set-dist-bindE)
  obtain b r where br-eq: br = (b, r)
    by (cases br)
  from receive-fri-commits-outcome[OF head[unfolded br-eq]]
  obtain rest1 where
    tr-s: PTranscript s = r # rest1
    and st-s1: PState s1 = concat (PState s) r
    and tr-s1: PTranscript s1 = rest1
    and ext-head: s ≤ s1
    and lookup-head:
      fmlookup (HashMap s1) (FiatShamirChallenge (concat (PState s) r)) = Some b
    by blast
  have tail-res:
    length brs' = n ∧
    PTranscript s1 = map snd brs' @ PTranscript t ∧
    PState t = foldl concat (PState s1) (map snd brs') ∧
    s1 ≤ t ∧
    (∀ i < n.
      fmlookup (HashMap t)
        (FiatShamirChallenge (foldl concat (PState s1) (take (Suc i) (map snd brs')))) =
        Some (fst (brs' ! i)))
    using Suc.IH[OF tail] .
  have ext-s1-t: s1 ≤ t
    using tail-res by simp
  have lookup-head-t:
    fmlookup (HashMap t) (FiatShamirChallenge (concat (PState s) r)) = Some b
    using hash-extension-lookup[OF lookup-head ext-s1-t] .
  have ext-combined: s ≤ t
    using ext-head ext-s1-t by (meson hash-ext-trans)
  have lookups:
    ∀ i < Suc n.
      fmlookup (HashMap t)
        (FiatShamirChallenge (foldl concat (PState s) (take (Suc i) (map snd brs)))) =

```

```

      Some (fst (brs ! i))
proof (intro allI impI)
  fix i
  assume i-bound: i < Suc n
  show fmlookup (HashMap t)
    (FiatShamirChallenge (foldl concat (PState s) (take (Suc i) (map snd brs)))) =
    Some (fst (brs ! i))
proof (cases i)
  case 0
  then show ?thesis
    using lookup-head-t brs-eq br-eq by simp
next
  case (Suc j)
  then have j-bound: j < n
    using i-bound by simp
  have key-eq:
    foldl concat (PState s) (take (Suc i) (map snd brs)) =
    foldl concat (PState s1) (take (Suc j) (map snd brs'))
  using Suc brs-eq br-eq st-s1 by simp
  show ?thesis
    using tail-res j-bound Suc brs-eq key-eq by simp
qed
qed
show ?case
  using tail-res tr-s tr-s1 st-s1 ext-combined lookups brs-eq br-eq
  by simp
qed

end

end

```

```

theory Soundness-Execution-Prefix
  imports Soundness-Execution-Query
begin

```

Verifier alpha/FRI prefix execution and challenge-space bounds.

```

context soundness
begin

```

```

definition alpha-round
  where
    alpha-round =
      (do {
        a0 ← receive-alpha-challenge;
        let a0' = a0;
        a1 ← read;
        let a1' = a1;

```

```

    assert (a0' = a1');
    return a1'
  })

```

definition *verifier-alpha-prefix*

```

:: ('f × ('f × 'f) list × 'f × 'f list,
   ('f, 'a) protocol-channel-scheme) state-monad

```

where

```

verifier-alpha-prefix =
  (do {
    fr ← read;
    f-fl ← ntimes receive-trace-fri-commits (ceil-log clength);
    f-final ← read;
    as ← mmap (replicate (length spec) alpha-round);
    return (fr, f-fl, f-final, as)
  })

```

definition *verifier-trace-fri-prefix*

```

:: ('f × ('f × 'f) list,
   ('f, 'a) protocol-channel-scheme) state-monad

```

where

```

verifier-trace-fri-prefix =
  (do {
    fr ← read;
    f-fl ← ntimes receive-trace-fri-commits (ceil-log clength);
    return (fr, f-fl)
  })

```

definition *verifier-after-trace-fri*

```

:: 'f × ('f × 'f) list ⇒
   (unit list, ('f, 'a) protocol-channel-scheme) state-monad

```

where

```

verifier-after-trace-fri header =
  (case header of (fr, f-fl) ⇒
    do {
      f-final ← read;
      as ← mmap (replicate (length spec) alpha-round);
      dg ← read;
      assert (to-nat dg ≤ maxDegree);
      fl ← ntimes receive-composition-fri-commits
        (ceil-log (to-nat dg + 1));
      final ← read;
      ntimes (verifier-query-round-program fr f-fl f-final as fl final)
        rounds
    })

```

definition *verifier-composition-fri-prefix*

```

:: ('f × ('f × 'f) list × 'f × 'f list ×
   'f × ('f × 'f) list,

```

('f, 'a) protocol-channel-scheme) state-monad
where
 verifier-composition-fri-prefix =
 (do {
 fr ← read;
 f-fl ← ntimes receive-trace-fri-commits (ceil-log clength);
 f-final ← read;
 as ← mmap (replicate (length spec) alpha-round);
 dg ← read;
 assert (to-nat dg ≤ maxDegree);
 fl ← ntimes receive-composition-fri-commits
 (ceil-log (to-nat dg + 1));
 return (fr, f-fl, f-final, as, dg, fl)
 })

definition verifier-after-composition-fri
 :: 'f × ('f × 'f) list × 'f × 'f list ×
 'f × ('f × 'f) list ⇒
 (unit list, ('f, 'a) protocol-channel-scheme) state-monad
where
 verifier-after-composition-fri header =
 (case header of (fr, f-fl, f-final, as, dg, fl) ⇒
 do {
 final ← read;
 ntimes (verifier-query-round-program fr f-fl f-final as fl final)
 rounds
 })

definition verifier-after-alpha
 :: 'f × ('f × 'f) list × 'f × 'f list ⇒
 (unit list, ('f, 'a) protocol-channel-scheme) state-monad
where
 verifier-after-alpha header =
 (case header of (fr, f-fl, f-final, as) ⇒
 do {
 dg ← read;
 assert (to-nat dg ≤ maxDegree);
 fl ← ntimes receive-composition-fri-commits
 (ceil-log (to-nat dg + 1));
 final ← read;
 ntimes (verifier-query-round-program fr f-fl f-final as fl final)
 rounds
 })

lemma verify-monad-alpha-decomposition:
 verify-monad = (verifier-alpha-prefix ≫= verifier-after-alpha)
unfolding verify-monad-def verifier-alpha-prefix-def verifier-after-alpha-def
 alpha-round-def verifier-query-round-program-def
by (simp add: Let-def sm-bind-assoc sm-bind-return-map split: prod.splits)

lemma *verify-monad-trace-fri-decomposition:*

verify-monad = (*verifier-trace-fri-prefix* $\gg$ *verifier-after-trace-fri*)

unfolding *verify-monad-def* *verifier-trace-fri-prefix-def*

verifier-after-trace-fri-def *alpha-round-def* *verifier-query-round-program-def*

by (*simp add: Let-def sm-bind-assoc sm-bind-return-map split: prod.splits*)

lemma *verify-monad-composition-fri-decomposition:*

verify-monad =

(*verifier-composition-fri-prefix* $\gg$ *verifier-after-composition-fri*)

unfolding *verify-monad-def* *verifier-composition-fri-prefix-def*

verifier-after-composition-fri-def *alpha-round-def*

verifier-query-round-program-def

by (*simp add: Let-def sm-bind-assoc sm-bind-return-map split: prod.splits*)

lemma *alpha-round-outcome:*

assumes *outcome: Some (a, t) $\in$ set-dist (execute alpha-round s)*

shows

$\exists$ *rest.*

PTranscript s = *a* # *rest* $\wedge$

PState t = *concat (PState s) a* $\wedge$

PTranscript t = *rest* $\wedge$

s $\leq$ *t* $\wedge$

PTraceFriCounter t = *PTraceFriCounter s* $\wedge$

PCompositionFriCounter t = *PCompositionFriCounter s* $\wedge$

PAlphaCounter t = *Suc (PAlphaCounter s)* $\wedge$

PQueryCounter t = *PQueryCounter s* $\wedge$

fmlookup (HashMap t) (AlphaChallenge (PAlphaCounter s) (PState s)) = *Some a*

proof –

from *outcome* **obtain** *a0 s0 a1 s1 s2* **where**

rand: Some (a0, s0) $\in$ set-dist (execute receive-alpha-challenge s)

and *read-a: Some (a1, s1) $\in$ set-dist (execute read s0)*

and *assert-a: Some ((), s2) $\in$ set-dist (execute (assert (a0 = a1)) s1)*

and *ret-a: Some (a, t) $\in$ set-dist (execute (return a1) s2)*

unfolding *alpha-round-def*

by (*auto simp: Let-def elim!: set-dist-bindE*)

have *rand-res:*

s $\leq$ *s0* $\wedge$

PState s0 = *PState s* $\wedge$

PTranscript s0 = *PTranscript s* $\wedge$

fmlookup (HashMap s0) (AlphaChallenge (PAlphaCounter s) (PState s)) = *Some a0*

using *receive-alpha-challenge-outcome[OF rand]* .

have *rand-counter:*

PTraceFriCounter s0 = *PTraceFriCounter s* $\wedge$

PCompositionFriCounter s0 = *PCompositionFriCounter s* $\wedge$

PAlphaCounter s0 = *Suc (PAlphaCounter s)* $\wedge$

PQueryCounter s0 = *PQueryCounter s*

using *receive-alpha-challenge-counter-outcome[OF rand]* **by** *simp*

obtain *rest* **where** *tr-s0: PTranscript s0* = *a1* # *rest*

```

proof –
  have PTranscript s0 ≠ []
    using read-a unfolding read-def protocol-read-def assert-def
    by (cases PTranscript s0) (auto simp: throw-no-outcome elim!: set-dist-bindE)
  then obtain x xs where xs: PTranscript s0 = x # xs
    by (cases PTranscript s0) auto
  have a1-x: a1 = x
    using read-nonempty-outcome[OF read-a xs] by simp
  show ?thesis
    using that xs a1-x by auto
qed
have read-res:
  s1 = s0(PState := concat (PState s0) a1, PTranscript := rest)
  using read-nonempty-outcome[OF read-a tr-s0] by simp
have a-eq: a0 = a1
  using assert-a unfolding assert-def
  by (cases a0 = a1) (auto simp: throw-no-outcome)
have s2-eq: s2 = s1
  using assert-a a-eq unfolding assert-def by simp
have t-eq: t = s1 and a-ret: a = a1
  using ret-a s2-eq by simp-all
have lookup-t: fmllookup (HashMap t) (AlphaChallenge (PAlphaCounter s) (PState s)) = Some
a
  using rand-res read-res t-eq a-ret a-eq
  unfolding less-eq-hash-ext-def less-eq-fmap-def by simp
have ext: s ≤ t
  using rand-res read-res t-eq a-ret a-eq
  unfolding less-eq-hash-ext-def less-eq-fmap-def by auto
show ?thesis
  using rand-res rand-counter tr-s0 read-res t-eq a-ret a-eq lookup-t ext
  by auto
qed

lemma alpha-round-preserves-alpha-future-fresh:
  assumes future: alpha-future-fresh s
    and outcome: Some (a, t) ∈ set-dist (execute alpha-round s)
  shows alpha-future-fresh t
proof –
  from outcome obtain a0 s0 a1 s1 s2 where
    rand: Some (a0, s0) ∈ set-dist (execute receive-alpha-challenge s)
    and read-a: Some (a1, s1) ∈ set-dist (execute read s0)
    and assert-a: Some ((), s2) ∈ set-dist (execute (assert (a0 = a1)) s1)
    and ret-a: Some (a, t) ∈ set-dist (execute (return a1) s2)
    unfolding alpha-round-def
    by (auto simp: Let-def elim!: set-dist-bindE)
  have counter-t: PAlphaCounter t = Suc (PAlphaCounter s)
    using alpha-round-outcome[OF outcome] by blast
  have future-s0:
     $\bigwedge i x. \text{Suc } (PAlphaCounter\ s) \leq i \implies$ 

```



```

    fmlookup (HashMap s0) (AlphaChallenge i x) = None
  proof -
    fix i x
    assume i-ge: Suc (PAlphaCounter s) ≤ i
    have neg:
      AlphaChallenge i x ≠
        AlphaChallenge (PAlphaCounter s) (PState s)
    using i-ge by auto
    have fmlookup (HashMap s0) (AlphaChallenge i x) =
      fmlookup (HashMap s) (AlphaChallenge i x)
    by (rule receive-alpha-challenge-preserves-other-lookup[OF rand neg])
    also have ... = None
    using future i-ge unfolding alpha-future-fresh-def by auto
    finally show
      fmlookup (HashMap s0) (AlphaChallenge i x) = None .
  qed
  have hash-s1: HashMap s1 = HashMap s0
  by (rule read-preserves-hash-map[OF read-a])
  have s2-eq: s2 = s1
  using assert-a unfolding assert-def
  by (cases a0 = a1) (auto simp: throw-no-outcome)
  have t-eq: t = s2
  using ret-a by simp
  show ?thesis
  unfolding alpha-future-fresh-def
  proof (intro allI impI)
    fix i x
    assume PAlphaCounter t ≤ i
    then have Suc (PAlphaCounter s) ≤ i
    using counter-t by simp
    then show fmlookup (HashMap t) (AlphaChallenge i x) = None
    using future-s0[of i x] hash-s1 unfolding t-eq s2-eq by simp
  qed
qed

lemma alpha-round-preserves-composition-fri-future-fresh:
  assumes future: composition-fri-future-fresh s
  and outcome: Some (a, t) ∈ set-dist (execute alpha-round s)
  shows composition-fri-future-fresh t
  proof -
    from outcome obtain a0 s0 a1 s1 s2 where
      rand: Some (a0, s0) ∈ set-dist (execute receive-alpha-challenge s)
      and read-a: Some (a1, s1) ∈ set-dist (execute read s0)
      and assert-a: Some ((), s2) ∈ set-dist (execute (assert (a0 = a1)) s1)
      and ret-a: Some (a, t) ∈ set-dist (execute (return a1) s2)
    unfolding alpha-round-def
    by (auto simp: Let-def elim!: set-dist-bindE)
  have counter-s0:
    PCompositionFriCounter s0 = PCompositionFriCounter s

```

```

using receive-alpha-challenge-counter-outcome[OF rand] by simp
have future-s0:
  composition-fri-future-fresh s0
unfolding composition-fri-future-fresh-def
proof (intro allI impI)
  fix i x
  assume i-ge-s0: PCompositionFriCounter s0 ≤ i
  have i-ge-s: PCompositionFriCounter s ≤ i
    using i-ge-s0 counter-s0 by simp
  have neg:
    CompositionFriChallenge i x ≠
      AlphaChallenge (PAlphaCounter s) (PState s)
    by simp
  have fmlookup (HashMap s0) (CompositionFriChallenge i x) =
    fmlookup (HashMap s) (CompositionFriChallenge i x)
    by (rule receive-alpha-challenge-preserves-other-lookup[OF rand neg])
  also have ... = None
    using future i-ge-s
    unfolding composition-fri-future-fresh-def by blast
  finally show
    fmlookup (HashMap s0) (CompositionFriChallenge i x) = None .
qed
have future-s1:
  composition-fri-future-fresh s1
by (rule read-preserves-composition-fri-future-fresh
  [OF future-s0 read-a])
have s2-eq: s2 = s1
  using assert-a unfolding assert-def
  by (cases a0 = a1) (auto simp: throw-no-outcome)
have t-eq: t = s2
  using ret-a by simp
show ?thesis
  using future-s1 unfolding t-eq s2-eq .
qed

lemma alpha-round-preserves-query-future-fresh:
  assumes future: query-future-fresh s
  and outcome: Some (a, t) ∈ set-dist (execute alpha-round s)
  shows query-future-fresh t
proof –
  from outcome obtain a0 s0 a1 s1 s2 where
    rand: Some (a0, s0) ∈ set-dist (execute receive-alpha-challenge s)
    and read-a: Some (a1, s1) ∈ set-dist (execute read s0)
    and assert-a: Some ((), s2) ∈ set-dist (execute (assert (a0 = a1)) s1)
    and ret-a: Some (a, t) ∈ set-dist (execute (return a1) s2)
    unfolding alpha-round-def
    by (auto simp: Let-def elim!: set-dist-bindE)
  have counter-s0:
    PQueryCounter s0 = PQueryCounter s

```

```

    using receive-alpha-challenge-counter-outcome[OF rand] by simp
  have future-s0:
    query-future-fresh s0
  unfolding query-future-fresh-def
proof (intro allI impI)
  fix i x
  assume i-ge-s0: PQueryCounter s0 ≤ i
  have i-ge-s: PQueryCounter s ≤ i
    using i-ge-s0 counter-s0 by simp
  have neg:
    QueryIndexChallenge i x ≠
      AlphaChallenge (PAlphaCounter s) (PState s)
    by simp
  have fmllookup (HashMap s0) (QueryIndexChallenge i x) =
    fmllookup (HashMap s) (QueryIndexChallenge i x)
    by (rule receive-alpha-challenge-preserves-other-lookup[OF rand neg])
  also have ... = None
    using future i-ge-s
  unfolding query-future-fresh-def by blast
  finally show
    fmllookup (HashMap s0) (QueryIndexChallenge i x) = None .
qed
have future-s1:
  query-future-fresh s1
  by (rule read-preserves-query-future-fresh[OF future-s0 read-a])
have s2-eq: s2 = s1
  using assert-a unfolding assert-def
  by (cases a0 = a1) (auto simp: throw-no-outcome)
have t-eq: t = s2
  using ret-a by simp
show ?thesis
  using future-s1 unfolding t-eq s2-eq .
qed

lemma wp-alpha-round-fresh-set:
  assumes future: alpha-future-fresh s
  shows
    wp-event alpha-round
      (λout. case out of None ⇒ False | Some (a, -) ⇒ a ∈ B) s
      ≤ nnreal (card B) / nnreal size
proof -
  let ?P = λout. case out of None ⇒ False | Some (a, -) ⇒ a ∈ B
  let ?k =
    λa0. read ≧ (λa1.
      assert (a0 = a1) ≧ (λ-. return a1))
  have head-bound:
    wp-event receive-alpha-challenge ?P s ≤
      nnreal (card B) / nnreal size
  using wp-receive-alpha-challenge-fresh-set

```

```

    [OF alpha-future-fresh-current[OF future], of B]
  by simp
have round-eq: alpha-round = (receive-alpha-challenge  $\gg$  ?k)
  unfolding alpha-round-def by (simp add: Let-def)
show ?thesis
  unfolding round-eq
proof (rule wp-event-bind-bound-by-head-event[OF head-bound])
  show ?P None  $\implies$  ?P None
    by simp
next
fix a0 s0 out
assume cont: out  $\in$  set-dist (execute (?k a0) s0)
  and hit: ?P out
show ?P (Some (a0, s0))
proof (cases out)
  case None
  then show ?thesis
    using hit by simp
next
case (Some pair)
obtain a t where pair-eq: pair = (a, t)
  by (cases pair)
from cont[unfolded Some pair-eq]
obtain a1 s1 s2 where
  read-a: Some (a1, s1)  $\in$  set-dist (execute read s0)
  and assert-a:
    Some ((), s2)  $\in$  set-dist (execute (assert (a0 = a1)) s1)
  and ret-a: Some (a, t)  $\in$  set-dist (execute (return a1) s2)
  by (auto elim!: set-dist-bindE)
have a0-eq: a0 = a1
  using assert-a unfolding assert-def
  by (cases a0 = a1) (auto simp: throw-no-outcome)
have a-eq: a = a1
  using ret-a by simp
have a  $\in$  B
  using hit Some pair-eq by simp
then show ?thesis
  using a0-eq a-eq by simp
qed
qed
qed

lemma wp-alpha-round-success-bound:
  assumes future: alpha-future-fresh s
  shows
    wp-event alpha-round
    ( $\lambda$ out. case out of None  $\implies$  False | Some -  $\implies$  True) s
     $\leq$  1 / nnreal size
proof (cases PTranscript s)

```

```

case Nil
have no-success:
   $\bigwedge out. out \in \text{set-dist } (\text{execute alpha-round } s) \implies$ 
     $(\text{case out of None} \Rightarrow \text{False} \mid \text{Some } - \Rightarrow \text{True}) \implies \text{False}$ 
proof –
  fix out
  assume out:  $out \in \text{set-dist } (\text{execute alpha-round } s)$ 
    and success:  $(\text{case out of None} \Rightarrow \text{False} \mid \text{Some } - \Rightarrow \text{True})$ 
  from success obtain a t where out-eq:  $out = \text{Some } (a, t)$ 
    by  $(\text{cases out}) \text{ auto}$ 
  from alpha-round-outcome[OF out[unfolded out-eq]]
  obtain rest where  $P\text{Transcript } s = a \# \text{rest}$ 
    by blast
  then show False
    using Nil by simp
qed
have wp-event alpha-round
   $(\lambda out. \text{case out of None} \Rightarrow \text{False} \mid \text{Some } - \Rightarrow \text{True}) \ s$ 
   $\leq \text{wp-event alpha-round}$ 
   $(\lambda - :: ('f \times ('f, 'a) \text{ protocol-channel-scheme}) \text{ option. False}) \ s$ 
  by  $(\text{rule wp-event-mono-on-support}) (\text{rule no-success})$ 
also have  $\dots = 0$ 
  unfolding wp-event-def wp-def dist-expect-def by simp
also have  $\dots \leq 1 / \text{nnreal size}$ 
  by simp
finally show ?thesis .
next
case  $(\text{Cons } x \text{ rest})$ 
let ?Q =  $\lambda out. \text{case out of None} \Rightarrow \text{False} \mid \text{Some } - \Rightarrow \text{True}$ 
let ?P =  $\lambda out. \text{case out of None} \Rightarrow \text{False} \mid \text{Some } (a, -) \Rightarrow a \in \{x\}$ 
let ?k =
   $\lambda a0. \text{read} \gg (\lambda a1.$ 
     $\text{assert } (a0 = a1) \gg (\lambda -. \text{return } a1))$ 
have head-bound:
   $\text{wp-event receive-alpha-challenge } ?P \ s \leq 1 / \text{nnreal size}$ 
proof –
  have wp-event receive-alpha-challenge ?P s =
     $\text{nnreal } (\text{card } (\{x\} :: 'f \text{ set})) / \text{nnreal size}$ 
  using wp-receive-alpha-challenge-fresh-set
     $[\text{OF alpha-future-fresh-current}[\text{OF future}], \text{ of } \{x\}]$ 
  by simp
  then show ?thesis
    by simp
qed
have round-eq:  $\text{alpha-round} = (\text{receive-alpha-challenge} \gg ?k)$ 
  unfolding alpha-round-def by  $(\text{simp add: Let-def})$ 
show ?thesis
  unfolding round-eq
proof  $(\text{rule wp-event-bind-bound-by-head-event}[\text{OF head-bound}])$ 

```

```

show ?Q None  $\implies$  ?P None
  by simp
next
fix a0 s0 out
assume rand:
  Some (a0, s0)  $\in$  set-dist (execute receive-alpha-challenge s)
  and cont: out  $\in$  set-dist (execute (?k a0) s0)
  and success: ?Q out
from success obtain a t where out-eq: out = Some (a, t)
  by (cases out) auto
from cont[unfolded out-eq]
obtain a1 s1 s2 where
  read-a: Some (a1, s1)  $\in$  set-dist (execute read s0)
  and assert-a:
    Some ((), s2)  $\in$  set-dist (execute (assert (a0 = a1)) s1)
  and ret-a: Some (a, t)  $\in$  set-dist (execute (return a1) s2)
  by (auto elim!: set-dist-bindE)
have tr-s0: PTranscript s0 = x # rest
  using receive-alpha-challenge-outcome[OF rand] Cons by simp
have a1-eq: a1 = x
  using read-nonempty-outcome[OF read-a tr-s0] by simp
have a0-eq: a0 = a1
  using assert-a unfolding assert-def
  by (cases a0 = a1) (auto simp: throw-no-outcome)
show ?P (Some (a0, s0))
  using a0-eq a1-eq by simp
qed
qed

lemma mmap-alpha-round-outcome:
assumes outcome:
  Some (as, t)  $\in$  set-dist (execute (mmap (replicate n alpha-round)) s)
shows
  length as = n  $\wedge$ 
  PTranscript s = as @ PTranscript t  $\wedge$ 
  PState t = foldl concat (PState s) as  $\wedge$ 
  s  $\leq$  t  $\wedge$ 
  PTraceFriCounter t = PTraceFriCounter s  $\wedge$ 
  PCompositionFriCounter t = PCompositionFriCounter s  $\wedge$ 
  PAlphaCounter t = PAlphaCounter s + n  $\wedge$ 
  PQueryCounter t = PQueryCounter s  $\wedge$ 
  ( $\forall i < n.$ 
    fmllookup (HashMap t)
      (AlphaChallenge (PAlphaCounter s + i)
        (foldl concat (PState s) (take i as))) =
      Some (as ! i))
using outcome
proof (induction n arbitrary: s as t)
case 0

```

```

then show ?case
  by (simp add: hash-ext-refl)
next
case (Suc n)
from Suc.premis obtain a as' s1 where
  head: Some (a, s1) ∈ set-dist (execute alpha-round s)
  and tail:
    Some (as', t) ∈ set-dist (execute (mmap (replicate n alpha-round)) s1)
  and as-eq: as = a # as'
  by (auto elim!: set-dist-bindE)
from alpha-round-outcome[OF head] obtain rest1 where
  tr-s: PTranscript s = a # rest1
  and st-s1: PState s1 = concat (PState s) a
  and tr-s1: PTranscript s1 = rest1
  and ext-head: s ≤ s1
  and trace-count-s1: PTraceFriCounter s1 = PTraceFriCounter s
  and comp-count-s1: PCompositionFriCounter s1 = PCompositionFriCounter s
  and alpha-count-s1: PAlphaCounter s1 = Suc (PAlphaCounter s)
  and query-count-s1: PQueryCounter s1 = PQueryCounter s
  and lookup-head: fmllookup (HashMap s1) (AlphaChallenge (PAlphaCounter s) (PState s))
= Some a
  by blast
have tail-res:
  length as' = n ∧
  PTranscript s1 = as' @ PTranscript t ∧
  PState t = foldl concat (PState s1) as' ∧
  s1 ≤ t ∧
  PTraceFriCounter t = PTraceFriCounter s1 ∧
  PCompositionFriCounter t = PCompositionFriCounter s1 ∧
  PAlphaCounter t = PAlphaCounter s1 + n ∧
  PQueryCounter t = PQueryCounter s1 ∧
  (∀ i < n.
    fmllookup (HashMap t)
      (AlphaChallenge (PAlphaCounter s1 + i)
        (foldl concat (PState s1) (take i as')))) =
    Some (as' ! i))
  using Suc.IH[OF tail] .
have ext-s1-t: s1 ≤ t
  using tail-res by simp
have lookup-head-t:
  fmllookup (HashMap t) (AlphaChallenge (PAlphaCounter s) (PState s)) = Some a
  using hash-extension-lookup[OF lookup-head ext-s1-t] .
have ext-combined: s ≤ t
  using ext-head ext-s1-t by (meson hash-ext-trans)
have counters-t:
  PTraceFriCounter t = PTraceFriCounter s ∧
  PCompositionFriCounter t = PCompositionFriCounter s ∧
  PAlphaCounter t = PAlphaCounter s + Suc n ∧
  PQueryCounter t = PQueryCounter s

```

```

using tail-res trace-count-s1 comp-count-s1 alpha-count-s1 query-count-s1
by simp
have lookups:
   $\forall i < \text{Suc } n.$ 
    fmlookup (HashMap t)
      (AlphaChallenge (PAlphaCounter s + i)
        (foldl concat (PState s) (take i as))) =
      Some (as ! i)
proof (intro allI impI)
  fix i
  assume i-bound:  $i < \text{Suc } n$ 
  show fmlookup (HashMap t)
    (AlphaChallenge (PAlphaCounter s + i)
      (foldl concat (PState s) (take i as))) =
    Some (as ! i)
proof (cases i)
  case 0
  then show ?thesis
    using lookup-head-t as-eq by simp
next
  case (Suc j)
  then have j-bound:  $j < n$ 
    using i-bound by simp
  have key-eq:
    foldl concat (PState s) (take i as) =
    foldl concat (PState s1) (take j as')
    using Suc as-eq st-s1 by simp
  show ?thesis
    using tail-res j-bound Suc as-eq key-eq alpha-count-s1 by simp
qed
qed
show ?case
  using tail-res tr-s tr-s1 st-s1 ext-combined lookups as-eq
    counters-t
  by simp
qed

lemma mmap-alpha-round-preserves-alpha-future-fresh:
  assumes future: alpha-future-fresh s
  and outcome:
    Some (as, t)  $\in$ 
      set-dist (execute (mmap (replicate n alpha-round)) s)
  shows alpha-future-fresh t
  using outcome future
proof (induction n arbitrary: s as t)
  case 0
  then show ?case
    by simp
next

```


case (*Suc n*)
from *Suc.prem*s **obtain** *a as' s1* **where**
head: *Some (a, s1) ∈ set-dist (execute alpha-round s)*
and *tail*:
Some (as', t) ∈
set-dist (execute (mmap (replicate n alpha-round)) s1)
by (*auto elim!:* *set-dist-bindE*)
have *future-s1*: *alpha-future-fresh s1*
by (*rule alpha-round-preserves-alpha-future-fresh*[*OF Suc.prem*s(2) *head*])
show ?*case*
by (*rule Suc.IH*[*OF tail future-s1*])
qed

lemma *mmap-alpha-round-preserves-composition-fri-future-fresh*:
assumes *future*: *composition-fri-future-fresh s*
and *outcome*:
Some (as, t) ∈
set-dist (execute (mmap (replicate n alpha-round)) s)
shows *composition-fri-future-fresh t*
using *outcome future*
proof (*induction n arbitrary: s as t*)
case 0
then show ?*case*
by *simp*
next
case (*Suc n*)
from *Suc.prem*s **obtain** *a as' s1* **where**
head: *Some (a, s1) ∈ set-dist (execute alpha-round s)*
and *tail*:
Some (as', t) ∈
set-dist (execute (mmap (replicate n alpha-round)) s1)
by (*auto elim!:* *set-dist-bindE*)
have *future-s1*: *composition-fri-future-fresh s1*
by (*rule alpha-round-preserves-composition-fri-future-fresh*
[*OF Suc.prem*s(2) *head*])
show ?*case*
by (*rule Suc.IH*[*OF tail future-s1*])
qed

lemma *mmap-alpha-round-preserves-query-future-fresh*:
assumes *future*: *query-future-fresh s*
and *outcome*:
Some (as, t) ∈
set-dist (execute (mmap (replicate n alpha-round)) s)
shows *query-future-fresh t*
using *outcome future*
proof (*induction n arbitrary: s as t*)
case 0
then show ?*case*

```

    by simp
next
case (Suc n)
from Suc.prem1 obtain a as' s1 where
  head: Some (a, s1) ∈ set-dist (execute alpha-round s)
  and tail:
    Some (as', t) ∈
      set-dist (execute (mmap (replicate n alpha-round)) s1)
  by (auto elim!: set-dist-bindE)
have future-s1: query-future-fresh s1
  by (rule alpha-round-preserves-query-future-fresh
      [OF Suc.prem2(2) head])
show ?case
  by (rule Suc.IH[OF tail future-s1])
qed

```

```

lemma wp-mmap-alpha-round-success-bound:
  assumes future: alpha-future-fresh s
  shows
    wp-event (mmap (replicate n alpha-round))
      (λout. case out of None ⇒ False | Some - ⇒ True) s
      ≤ (1 / nnreal size) ^ n
  using future
proof (induction n arbitrary: s)
  case 0
  then show ?case
    by (simp add: wp-event-def wpsimps)
next
case (Suc n)
let ?ListSuccess =
  λout :: ('f list × ('f, 'a) protocol-channel-scheme) option.
    case out of None ⇒ False | Some - ⇒ True
let ?HeadSuccess =
  λout :: ('f × ('f, 'a) protocol-channel-scheme) option.
    case out of None ⇒ False | Some - ⇒ True
let ?tail =
  (mmap (replicate n alpha-round) ::
    ('f list, ('f, 'a) protocol-channel-scheme) state-monad)
let ?D = (1 / nnreal size) ^ n
have bind-bound:
  wp-event
    (alpha-round >>=
      (λa :: 'f. ?tail >>= (λas :: 'f list. return (a # as))))
    ?ListSuccess s
    ≤ wp-event alpha-round ?HeadSuccess s * ?D
proof (rule wp-event-bind-bound-by-head-and-cont
  [where P = ?HeadSuccess])
  show ¬ ?ListSuccess None
    by simp

```

```

next
  fix a t
  assume  $\neg ?HeadSuccess (Some (a, t))$ 
  then show
    wp-event ( $?tail \gg (\lambda as :: 'f list. return (a \# as))$ )
      ?ListSuccess t = 0
    by simp
next
  fix a t
  assume head:  $Some (a, t) \in set-dist (execute\ alpha-round\ s)$ 
  have future-t:  $\alpha\text{-future-fresh}\ t$ 
    by (rule  $\alpha\text{-round-preserves-}\alpha\text{-future-fresh}$  [OF Suc.prem head])
  have tail-bound: wp-event  $?tail$  ?ListSuccess t  $\leq ?D$ 
    by (rule Suc.IH [OF future-t])
  have cont-eq:
    ( $\lambda out. case\ out\ of$ 
      None  $\Rightarrow (0 :: prob)$ 
      | Some (as, u)  $\Rightarrow$ 
        wp (return (a # as))
        ( $\lambda out. if\ ?ListSuccess\ out\ then\ 1\ else\ 0$ ) u) =
    ( $\lambda out. if\ ?ListSuccess\ out\ then\ 1\ else\ 0$ )
    by (rule ext) (auto simp: wpsimps split: option.splits prod.splits)
  have wp-event ( $?tail \gg (\lambda as :: 'f list. return (a \# as))$ )
    ?ListSuccess t =
    wp-event  $?tail$  ?ListSuccess t
  unfolding wp-event-def
  by (simp add: wpsimps cont-eq)
  also have ...  $\leq ?D$ 
    by (rule tail-bound)
  finally show
    wp-event ( $?tail \gg (\lambda as :: 'f list. return (a \# as))$ )
      ?ListSuccess t  $\leq ?D$  .
qed
have wp-event (mmap (replicate (Suc n)  $\alpha\text{-round}$ )) ?ListSuccess s
   $\leq$  wp-event  $\alpha\text{-round}$  ?HeadSuccess s * ?D
  using bind-bound by simp
also have ...  $\leq (1 / nnreal\ size) * ?D$ 
  by (rule mult-right-mono [OF wp- $\alpha\text{-round-success-bound}$  [OF Suc.prem]])
  simp
also have ...  $= (1 / nnreal\ size) \wedge Suc\ n$ 
  by simp
finally show ?case .
qed

lemma wp-mmap-alpha-round-finite-set-bound:
  assumes future:  $\alpha\text{-future-fresh}\ s$ 
  and finite-B: finite B
  shows
    wp-event (mmap (replicate n  $\alpha\text{-round}$ ))

```

```

      (λout. case out of None ⇒ False | Some (as, -) ⇒ as ∈ B) s
      ≤ nnreal (card B) / (nnreal size) ^ n
proof (cases B = {})
  case True
  have wp-event (mmap (replicate n alpha-round))
    (λout. case out of None ⇒ False | Some (as, -) ⇒ as ∈ B) s
    ≤ wp-event (mmap (replicate n alpha-round))
      (λ- :: ('f list × ('f, 'a) protocol-channel-scheme) option. False) s
  unfolding True
  by (rule wp-event-mono) (auto split: option.splits prod.splits)
  also have ... = 0
  unfolding wp-event-def wp-def dist-expect-def by simp
  finally show ?thesis
  unfolding True by simp
next
  case False
  have card-pos: 0 < card B
  using finite-B False by (simp add: card-gt-0-iff)
  have event-success:
    wp-event (mmap (replicate n alpha-round))
      (λout. case out of None ⇒ False | Some (as, -) ⇒ as ∈ B) s
      ≤ wp-event (mmap (replicate n alpha-round))
        (λout. case out of None ⇒ False | Some - ⇒ True) s
  by (rule wp-event-mono) (auto split: option.splits prod.splits)
  also have ... ≤ (1 / nnreal size) ^ n
  by (rule wp-mmap-alpha-round-success-bound[OF future])
  also have ... = 1 / (nnreal size) ^ n
  using size-card by transfer (simp add: power-divide)
  also have ... = nnreal (1::nat) / nnreal (size ^ n)
  by simp
  also have ... ≤ nnreal (card B) / nnreal (size ^ n)
  by (rule nnreal-nat-divide-right-mono) (use card-pos in simp)
  also have ... ≤ nnreal (card B) / (nnreal size) ^ n
  by simp
  finally show ?thesis .
qed

lemma wp-mmap-alpha-round-alpha-space-set-bound:
  assumes future: alpha-future-fresh s
  and subset: B ⊆ alpha-space
  shows
    wp-event (mmap (replicate (length spec) alpha-round))
      (λout. case out of None ⇒ False | Some (as, -) ⇒ as ∈ B) s
      ≤ nnreal (card B) / nnreal (card alpha-space)
proof -
  have finite-B: finite B
  by (rule finite-subset[OF subset finite-alpha-space])
  have wp-event (mmap (replicate (length spec) alpha-round))
    (λout. case out of None ⇒ False | Some (as, -) ⇒ as ∈ B) s

```

$\leq \text{nnreal } (\text{card } B) / (\text{nnreal size}) \wedge \text{length spec}$
by (rule *wp-mmap-alpha-round-finite-set-bound*[*OF future finite-B*])
also have ... = $\text{nnreal } (\text{card } B) / \text{nnreal } (\text{card alpha-space})$
using *size-card* **by** *simp*
finally show ?thesis .
qed

lemma *receive-fri-commits-with-outcome*:

assumes *receiver-outcome*:

$\bigwedge b \text{ s-read } t.$
 $\text{Some } (b, t) \in \text{set-dist } (\text{execute receive-challenge s-read}) \implies$
 $s\text{-read} \leq t \wedge$
 $P\text{State } t = P\text{State } s\text{-read} \wedge$
 $P\text{Transcript } t = P\text{Transcript } s\text{-read} \wedge$
 $\text{fmlookup } (\text{HashMap } t) (\text{tag } (P\text{State } s\text{-read})) = \text{Some } b$
and *outcome*:
 $\text{Some } ((b, r), t) \in$
 $\text{set-dist } (\text{execute } (\text{receive-fri-commits-with receive-challenge}) s)$

shows

$\exists \text{rest}.$
 $P\text{Transcript } s = r \# \text{rest} \wedge$
 $P\text{State } t = \text{concat } (P\text{State } s) r \wedge$
 $P\text{Transcript } t = \text{rest} \wedge$
 $s \leq t \wedge$
 $\text{fmlookup } (\text{HashMap } t) (\text{tag } (\text{concat } (P\text{State } s) r)) = \text{Some } b$

proof –

from *outcome* **obtain** *s-read* **where**

read-r: $\text{Some } (r, s\text{-read}) \in \text{set-dist } (\text{execute read } s)$

and *rand-b*:

$\text{Some } (b, t) \in \text{set-dist } (\text{execute receive-challenge s-read})$

unfolding *receive-fri-commits-with-def*

by (*auto elim!*: *set-dist-bindE*)

obtain *rest* **where** *tr-s*: $P\text{Transcript } s = r \# \text{rest}$

proof –

have $P\text{Transcript } s \neq []$

using *read-r* **unfolding** *read-def protocol-read-def assert-def*

by (*cases* $P\text{Transcript } s$) (*auto simp*: *throw-no-outcome elim!*: *set-dist-bindE*)

then obtain *x xs* **where** *xs*: $P\text{Transcript } s = x \# xs$

by (*cases* $P\text{Transcript } s$) *auto*

have *r-x*: $r = x$

using *read-nonempty-outcome*[*OF read-r xs*] **by** *simp*

show ?thesis

using *that xs r-x* **by** *auto*

qed

have *read-res*:

$r = r \wedge$

$s\text{-read} = s[P\text{State} := \text{concat } (P\text{State } s) r, P\text{Transcript} := \text{rest}]$

using *read-nonempty-outcome*[*OF read-r tr-s*] **by** *simp*

have *rand-res*:

```

  s-read ≤ t ∧
  PState t = PState s-read ∧
  PTranscript t = PTranscript s-read ∧
  fmlookup (HashMap t) (tag (PState s-read)) = Some b
  using receiver-outcome[OF rand-b] .
have s-read-ext: s ≤ s-read
  using read-res unfolding less-eq-hash-ext-def less-eq-fmap-def by simp
have s-t: s ≤ t
  using s-read-ext rand-res by (meson hash-ext-trans)
show ?thesis
  using tr-s read-res rand-res s-t by auto
qed

```

lemma *receive-trace-fri-commits-outcome:*

assumes *outcome:*

Some ((b, r), t) ∈
set-dist (execute receive-trace-fri-commits s)

shows

$\exists$ *rest.*

PTranscript s = r # rest ∧
PState t = concat (PState s) r ∧
PTranscript t = rest ∧
s ≤ t ∧
PTraceFriCounter t = Suc (PTraceFriCounter s) ∧
PCompositionFriCounter t = PCompositionFriCounter s ∧
PAlphaCounter t = PAlphaCounter s ∧
PQueryCounter t = PQueryCounter s ∧
fmlookup (HashMap t)
(TraceFriChallenge (PTraceFriCounter s) (concat (PState s) r)) =
Some b

proof –

from *outcome* **obtain** *s-read* **where**

read-r: Some (r, s-read) ∈ set-dist (execute read s)

and *rand-b:*

Some (b, t) ∈ set-dist (execute receive-trace-fri-challenge s-read)

unfolding *receive-trace-fri-commits-def receive-fri-commits-with-def*

by (*auto elim!: set-dist-bindE*)

obtain *rest* **where** *tr-s: PTranscript s = r # rest*

proof –

have *PTranscript s ≠ []*

using *read-r* **unfolding** *read-def protocol-read-def assert-def*

by (*cases PTranscript s*) (*auto simp: throw-no-outcome elim!: set-dist-bindE*)

then obtain *x xs* **where** *xs: PTranscript s = x # xs*

by (*cases PTranscript s*) *auto*

have *r-x: r = x*

using *read-nonempty-outcome[OF read-r xs]* **by** *simp*

show *?thesis*

using *that xs r-x* **by** *auto*

qed

```

have read-res:
  s-read = s(PState := concat (PState s) r, PTranscript := rest)
  using read-nonempty-outcome[OF read-r tr-s] by simp
have rand-res:
  s-read ≤ t ∧
  PState t = PState s-read ∧
  PTranscript t = PTranscript s-read ∧
  fmlookup (HashMap t)
    (TraceFriChallenge (PTraceFriCounter s-read) (PState s-read)) =
    Some b
  using receive-trace-fri-challenge-outcome[OF rand-b] .
have rand-counter:
  PTraceFriCounter t = Suc (PTraceFriCounter s-read) ∧
  PCompositionFriCounter t = PCompositionFriCounter s-read ∧
  PAlphaCounter t = PAlphaCounter s-read ∧
  PQueryCounter t = PQueryCounter s-read
  using receive-trace-fri-challenge-counter-outcome[OF rand-b] by simp
have s-read-ext: s ≤ s-read
  using read-res unfolding less-eq-hash-ext-def less-eq-fmap-def by simp
have s-t: s ≤ t
  using s-read-ext rand-res by (meson hash-ext-trans)
show ?thesis
  using tr-s read-res rand-res rand-counter s-t by auto
qed

```

```

lemma receive-trace-fri-commits-preserves-trace-fri-future-fresh:
  assumes future: trace-fri-future-fresh s
  and outcome:
    Some ((b, r), t) ∈
      set-dist (execute receive-trace-fri-commits s)
  shows trace-fri-future-fresh t
proof –
  from outcome obtain s-read where
    read-r: Some (r, s-read) ∈ set-dist (execute read s)
  and rand-b:
    Some (b, t) ∈ set-dist (execute receive-trace-fri-challenge s-read)
  unfolding receive-trace-fri-commits-def receive-fri-commits-with-def
  by (auto elim!: set-dist-bindE)
  from read-outcome[OF read-r] have read-counts:
    PTraceFriCounter s-read = PTraceFriCounter s
  by blast
  have read-hash: HashMap s-read = HashMap s
  by (rule read-preserves-hash-map[OF read-r])
  have counter-t:
    PTraceFriCounter t = Suc (PTraceFriCounter s)
  using receive-trace-fri-challenge-counter-outcome[OF rand-b] read-counts
  by simp
  show ?thesis
  unfolding trace-fri-future-fresh-def

```

```

proof (intro allI impI)
  fix i x
  assume i-ge-t: PTraceFriCounter t ≤ i
  have i-ge-s: PTraceFriCounter s ≤ i
    using i-ge-t counter-t by simp
  have neg:
    TraceFriChallenge i x ≠
      TraceFriChallenge (PTraceFriCounter s-read) (PState s-read)
    using i-ge-t counter-t read-counts by auto
  have fmlookup (HashMap t) (TraceFriChallenge i x) =
    fmlookup (HashMap s-read) (TraceFriChallenge i x)
    by (rule receive-trace-fri-challenge-preserves-other-lookup
      [OF rand-b neg])
  also have ... = fmlookup (HashMap s) (TraceFriChallenge i x)
    using read-hash by simp
  also have ... = None
    using future i-ge-s unfolding trace-fri-future-fresh-def by blast
  finally show fmlookup (HashMap t) (TraceFriChallenge i x) = None .
qed
qed

lemma receive-trace-fri-commits-preserves-alpha-future-fresh:
  assumes future: alpha-future-fresh s
  and outcome:
    Some ((b, r), t) ∈
      set-dist (execute receive-trace-fri-commits s)
  shows alpha-future-fresh t
proof –
  from outcome obtain s-read where
    read-r: Some (r, s-read) ∈ set-dist (execute read s)
  and rand-b:
    Some (b, t) ∈ set-dist (execute receive-trace-fri-challenge s-read)
  unfolding receive-trace-fri-commits-def receive-fri-commits-with-def
  by (auto elim!: set-dist-bindE)
  from read-outcome[OF read-r] have read-counts:
    PAlphaCounter s-read = PAlphaCounter s
  by blast
  have read-hash: HashMap s-read = HashMap s
    by (rule read-preserves-hash-map[OF read-r])
  have counter-t:
    PAlphaCounter t = PAlphaCounter s
    using receive-trace-fri-challenge-counter-outcome[OF rand-b]
    read-counts
  by simp
  show ?thesis
    unfolding alpha-future-fresh-def
proof (intro allI impI)
  fix i x
  assume i-ge-t: PAlphaCounter t ≤ i

```



```

have i-ge-s: PAlphaCounter s ≤ i
  using i-ge-t counter-t by simp
have neg:
  AlphaChallenge i x ≠
    TraceFriChallenge (PTraceFriCounter s-read) (PState s-read)
  by simp
have fmlookup (HashMap t) (AlphaChallenge i x) =
  fmlookup (HashMap s-read) (AlphaChallenge i x)
  by (rule receive-trace-fri-challenge-preserves-other-lookup
    [OF rand-b neg])
also have ... = fmlookup (HashMap s) (AlphaChallenge i x)
  using read-hash by simp
also have ... = None
  using future i-ge-s unfolding alpha-future-fresh-def by blast
finally show fmlookup (HashMap t) (AlphaChallenge i x) = None .
qed
qed

lemma receive-trace-fri-commits-preserves-composition-fri-future-fresh:
assumes future: composition-fri-future-fresh s
and outcome:
  Some ((b, r), t) ∈
    set-dist (execute receive-trace-fri-commits s)
shows composition-fri-future-fresh t
proof –
from outcome obtain s-read where
  read-r: Some (r, s-read) ∈ set-dist (execute read s)
and rand-b:
  Some (b, t) ∈ set-dist (execute receive-trace-fri-challenge s-read)
unfolding receive-trace-fri-commits-def receive-fri-commits-with-def
by (auto elim!: set-dist-bindE)
from read-outcome[OF read-r] have read-counts:
  PCompositionFriCounter s-read = PCompositionFriCounter s
by blast
have read-hash: HashMap s-read = HashMap s
by (rule read-preserves-hash-map[OF read-r])
have counter-t:
  PCompositionFriCounter t = PCompositionFriCounter s
using receive-trace-fri-challenge-counter-outcome[OF rand-b]
  read-counts
by simp
show ?thesis
unfolding composition-fri-future-fresh-def
proof (intro allI impI)
  fix i x
  assume i-ge-t: PCompositionFriCounter t ≤ i
  have i-ge-s: PCompositionFriCounter s ≤ i
    using i-ge-t counter-t by simp
  have neg:

```

```

    CompositionFriChallenge i x ≠
      TraceFriChallenge (PTraceFriCounter s-read) (PState s-read)
  by simp
  have fmlookup (HashMap t) (CompositionFriChallenge i x) =
    fmlookup (HashMap s-read) (CompositionFriChallenge i x)
  by (rule receive-trace-fri-challenge-preserves-other-lookup
      [OF rand-b neg])
  also have ... = fmlookup (HashMap s) (CompositionFriChallenge i x)
    using read-hash by simp
  also have ... = None
    using future i-ge-s
    unfolding composition-fri-future-fresh-def by blast
  finally show
    fmlookup (HashMap t) (CompositionFriChallenge i x) = None .
qed
qed

lemma receive-trace-fri-commits-preserves-query-future-fresh:
  assumes future: query-future-fresh s
  and outcome:
    Some ((b, r), t) ∈
      set-dist (execute receive-trace-fri-commits s)
  shows query-future-fresh t
proof -
  from outcome obtain s-read where
    read-r: Some (r, s-read) ∈ set-dist (execute read s)
  and rand-b:
    Some (b, t) ∈ set-dist (execute receive-trace-fri-challenge s-read)
  unfolding receive-trace-fri-commits-def receive-fri-commits-with-def
  by (auto elim!: set-dist-bindE)
  from read-outcome[OF read-r] have read-counts:
    PQueryCounter s-read = PQueryCounter s
  by blast
  have read-hash: HashMap s-read = HashMap s
  by (rule read-preserves-hash-map[OF read-r])
  have counter-t:
    PQueryCounter t = PQueryCounter s
  using receive-trace-fri-challenge-counter-outcome[OF rand-b]
  read-counts
  by simp
  show ?thesis
  unfolding query-future-fresh-def
proof (intro allI impI)
  fix i x
  assume i-ge-t: PQueryCounter t ≤ i
  have i-ge-s: PQueryCounter s ≤ i
  using i-ge-t counter-t by simp
  have neg:
    QueryIndexChallenge i x ≠

```

```

    TraceFriChallenge (PTraceFriCounter s-read) (PState s-read)
  by simp
  have fmlookup (HashMap t) (QueryIndexChallenge i x) =
    fmlookup (HashMap s-read) (QueryIndexChallenge i x)
  by (rule receive-trace-fri-challenge-preserves-other-lookup
    [OF rand-b neq])
  also have ... = fmlookup (HashMap s) (QueryIndexChallenge i x)
    using read-hash by simp
  also have ... = None
    using future i-ge-s unfolding query-future-fresh-def by blast
  finally show fmlookup (HashMap t) (QueryIndexChallenge i x) = None .
qed
qed

```

lemma *wp-receive-trace-fri-commits-fresh-set:*

assumes *future: trace-fri-future-fresh s*

shows

wp-event receive-trace-fri-commits

($\lambda out. \text{case out of}$

$\text{None} \Rightarrow \text{False}$

$\mid \text{Some } ((b, r), -) \Rightarrow b \in B) s$

$\leq \text{nnreal } (\text{card } B) / \text{nnreal size}$

unfolding *receive-trace-fri-commits-def receive-fri-commits-with-def*

proof (rule *wp-event-bind-bound-by-cont*)

show $\neg (\text{case None of}$

$\text{None} \Rightarrow \text{False} \mid \text{Some } ((b, r), -) \Rightarrow b \in B)$

by *simp*

next

fix *r s-read*

assume *read-r: Some (r, s-read) $\in$ set-dist (execute read s)*

have *read-counts:*

PTraceFriCounter s-read = PTraceFriCounter s

using *read-outcome[OF read-r]* **by** *blast*

have *read-hash: HashMap s-read = HashMap s*

by (rule *read-preserves-hash-map[OF read-r]*)

have *fresh:*

fmlookup (HashMap s-read)

(*TraceFriChallenge (PTraceFriCounter s-read) (PState s-read)*) = *None*

using *future read-counts read-hash*

unfolding *trace-fri-future-fresh-def* **by** *simp*

let *?Q =*

$\lambda out :: ((f \times f) \times (f, 'a) \text{ protocol-channel-scheme}) \text{ option.}$

$\text{case out of } \text{None} \Rightarrow \text{False} \mid \text{Some } ((b, r), -) \Rightarrow b \in B$

have *cont-eq:*

($\lambda out. \text{case out of}$

$\text{None} \Rightarrow (0 :: \text{prob})$

$\mid \text{Some } (b, u) \Rightarrow$

$\text{wp } (\text{return } (b, r)) (\lambda out. \text{if } ?Q \text{ out then } 1 \text{ else } 0) u) =$

$(\lambda out :: (f \times (f, 'a) \text{ protocol-channel-scheme}) \text{ option.}$

```

    if (case out of None  $\Rightarrow$  False | Some (b, -)  $\Rightarrow$  b  $\in$  B)
    then 1 else 0)
  by (rule ext) (auto simp: wpsimps split: option.splits prod.splits)
have wp-event
  (receive-trace-fri-challenge  $\gg$  ( $\lambda b :: 'f$ . return (b, r)))
  ?Q s-read =
  wp-event receive-trace-fri-challenge
    ( $\lambda$ out. case out of None  $\Rightarrow$  False | Some (b, -)  $\Rightarrow$  b  $\in$  B)
    s-read
  unfolding wp-event-def by (simp add: wpsimps cont-eq)
also have ... = nnreal (card B) / nnreal size
  by (rule wp-receive-trace-fri-challenge-fresh-set[OF fresh])
finally show
  wp-event
    (receive-trace-fri-challenge  $\gg$  ( $\lambda b :: 'f$ . return (b, r)))
    ( $\lambda$ out. case out of None  $\Rightarrow$  False | Some ((b, r), -)  $\Rightarrow$  b  $\in$  B)
    s-read
     $\leq$  nnreal (card B) / nnreal size
  by simp
qed

lemma ntimes-receive-trace-fri-commits-preserves-trace-fri-future-fresh:
  assumes future: trace-fri-future-fresh s
  and outcome:
    Some (brs, t)  $\in$ 
      set-dist (execute (ntimes receive-trace-fri-commits n) s)
  shows trace-fri-future-fresh t
  using outcome future
proof (induction n arbitrary: s brs t)
  case 0
  then show ?case
    by simp
next
  case (Suc n)
  from Suc.prem1 obtain br brs' s1 where
    head:
      Some (br, s1)  $\in$  set-dist (execute receive-trace-fri-commits s)
  and tail:
    Some (brs', t)  $\in$ 
      set-dist (execute (ntimes receive-trace-fri-commits n) s1)
  by (auto elim!: set-dist-bindE)
  obtain b r where br-eq: br = (b, r)
  by (cases br)
  have future-s1:
    trace-fri-future-fresh s1
  by (rule receive-trace-fri-commits-preserves-trace-fri-future-fresh
    [OF Suc.prem2(2) head[unfolded br-eq]])
  show ?case
  by (rule Suc.IH[OF tail future-s1])

```

qed

lemma *ntimes-receive-trace-fri-commits-preserves-alpha-future-fresh:*

assumes *outcome:*

Some (brs, t) ∈

set-dist (execute (ntimes receive-trace-fri-commits n) s)

and *future: alpha-future-fresh s*

shows *alpha-future-fresh t*

using *outcome future*

proof (*induction n arbitrary: s brs t*)

case 0

then show ?*case*

by *simp*

next

case (*Suc n*)

from *Suc.prem1* **obtain** *br brs' s1* **where**

head:

Some (br, s1) ∈ set-dist (execute receive-trace-fri-commits s)

and *tail:*

Some (brs', t) ∈

set-dist (execute (ntimes receive-trace-fri-commits n) s1)

by (*auto elim!: set-dist-bindE*)

obtain *b r* **where** *br-eq: br = (b, r)*

by (*cases br*)

have *future-s1:*

alpha-future-fresh s1

by (*rule receive-trace-fri-commits-preserves-alpha-future-fresh*

[OF Suc.prem2 head[unfolded br-eq]])

show ?*case*

by (*rule Suc.IH[OF tail future-s1]*)

qed

lemma *ntimes-receive-trace-fri-commits-preserves-composition-fri-future-fresh:*

assumes *outcome:*

Some (brs, t) ∈

set-dist (execute (ntimes receive-trace-fri-commits n) s)

and *future: composition-fri-future-fresh s*

shows *composition-fri-future-fresh t*

using *outcome future*

proof (*induction n arbitrary: s brs t*)

case 0

then show ?*case*

by *simp*

next

case (*Suc n*)

from *Suc.prem1* **obtain** *br brs' s1* **where**

head:

Some (br, s1) ∈ set-dist (execute receive-trace-fri-commits s)

and *tail:*

Some $(brs', t) \in$
 $set-dist \ (execute \ (ntimes \ receive-trace-fri-commits \ n) \ s1)$
 by $(auto \ elim!: \ set-dist-bindE)$
obtain $b \ r$ **where** $br-eq: \ br = (b, r)$
 by $(cases \ br)$
have $future-s1:$
 $composition-fri-future-fresh \ s1$
 by $(rule \ receive-trace-fri-commits-preserves-composition-fri-future-fresh$
 $[OF \ Suc.prem(2) \ head[unfolded \ br-eq]])$
show $?case$
 by $(rule \ Suc.IH[OF \ tail \ future-s1])$
qed

lemma $ntimes-receive-trace-fri-commits-preserves-query-future-fresh:$

assumes $outcome:$
 Some $(brs, t) \in$
 $set-dist \ (execute \ (ntimes \ receive-trace-fri-commits \ n) \ s)$
 and $future: \ query-future-fresh \ s$
shows $query-future-fresh \ t$
using $outcome \ future$
proof $(induction \ n \ arbitrary: \ s \ brs \ t)$
case 0
then show $?case$
 by $simp$
next
case $(Suc \ n)$
from $Suc.prem(1)$ **obtain** $br \ brs' \ s1$ **where**
 $head:$
 Some $(br, s1) \in set-dist \ (execute \ receive-trace-fri-commits \ s)$
 and $tail:$
 Some $(brs', t) \in$
 $set-dist \ (execute \ (ntimes \ receive-trace-fri-commits \ n) \ s1)$
 by $(auto \ elim!: \ set-dist-bindE)$
obtain $b \ r$ **where** $br-eq: \ br = (b, r)$
 by $(cases \ br)$
have $future-s1:$
 $query-future-fresh \ s1$
 by $(rule \ receive-trace-fri-commits-preserves-query-future-fresh$
 $[OF \ Suc.prem(2) \ head[unfolded \ br-eq]])$
show $?case$
 by $(rule \ Suc.IH[OF \ tail \ future-s1])$
qed

lemma $verifier-alpha-prefix-preserves-query-future-fresh:$

assumes $future: \ query-future-fresh \ s$
and $outcome:$
 Some $(header, t) \in set-dist \ (execute \ verifier-alpha-prefix \ s)$
shows $query-future-fresh \ t$
proof —

```

from outcome obtain fr s1 f-fl s2 f-final s3 as where
  read-fr: Some (fr, s1) ∈ set-dist (execute read s)
and trace-fri:
  Some (f-fl, s2) ∈
    set-dist (execute (ntimes receive-trace-fri-commits (ceil-log clength)) s1)
and read-final: Some (f-final, s3) ∈ set-dist (execute read s2)
and alpha:
  Some (as, t) ∈
    set-dist (execute (mmap (replicate (length spec) alpha-round)) s3)
unfolding verifier-alpha-prefix-def
by (auto elim!: set-dist-bindE)
have future-s1: query-future-fresh s1
  by (rule read-preserves-query-future-fresh[OF future read-fr])
have future-s2: query-future-fresh s2
  by (rule ntimes-receive-trace-fri-commits-preserves-query-future-fresh
    [OF trace-fri future-s1])
have future-s3: query-future-fresh s3
  by (rule read-preserves-query-future-fresh[OF future-s2 read-final])
show ?thesis
  by (rule mmap-alpha-round-preserves-query-future-fresh
    [OF future-s3 alpha])
qed

```

lemma wp-verifier-alpha-prefix-alpha-space-set-bound:

assumes future: alpha-future-fresh s
and subset: $B \subseteq \text{alpha-space}$

shows

wp-event verifier-alpha-prefix
 ($\lambda \text{out. case out of}$
 None $\Rightarrow$ False
 | Some ((-, -, -, as'), -) $\Rightarrow$ $as' \in B$) s
 $\leq \text{nnreal (card B) / nnreal (card alpha-space)}$

proof –

let ?Q =

$\lambda \text{out. case out of}$
 None $\Rightarrow$ False
 | Some ((-, -, -, as'), -) $\Rightarrow as' \in B$

let ?C = $\text{nnreal (card B) / nnreal (card alpha-space)}$

show ?thesis

unfolding verifier-alpha-prefix-def

proof (rule wp-event-bind-bound-by-cont)

show $\neg ?Q$ None

by simp

next

fix fr s1

assume read-fr: Some (fr, s1) ∈ set-dist (execute read s)

have future-s1: alpha-future-fresh s1

by (rule read-preserves-alpha-future-fresh[OF future read-fr])

show wp-event

```

  (ntimes receive-trace-fri-commits (ceil-log clength)  $\gg$ 
    ( $\lambda$ f-fl. read  $\gg$ 
      ( $\lambda$ f-final.
        mmap (replicate (length spec) alpha-round)  $\gg$ 
          ( $\lambda$ as. return (fr, f-fl, f-final, as))))))
  ?Q s1  $\leq$  ?C
proof (rule wp-event-bind-bound-by-cont)
  show  $\neg$  ?Q None
  by simp
next
  fix f-fl s2
  assume trace-fri:
    Some (f-fl, s2)  $\in$ 
      set-dist
      (execute (ntimes receive-trace-fri-commits (ceil-log clength)) s1)
  have future-s2: alpha-future-fresh s2
  by (rule ntimes-receive-trace-fri-commits-preserves-alpha-future-fresh
    [OF trace-fri future-s1])
  show wp-event
  (read  $\gg$ 
    ( $\lambda$ f-final.
      mmap (replicate (length spec) alpha-round)  $\gg$ 
        ( $\lambda$ as. return (fr, f-fl, f-final, as))))
  ?Q s2  $\leq$  ?C
proof (rule wp-event-bind-bound-by-cont)
  show  $\neg$  ?Q None
  by simp
next
  fix f-final s3
  assume read-final:
    Some (f-final, s3)  $\in$  set-dist (execute read s2)
  have future-s3: alpha-future-fresh s3
  by (rule read-preserves-alpha-future-fresh[OF future-s2 read-final])
  let ?A =
     $\lambda$ out. case out of
      None  $\Rightarrow$  False
      | Some (as, -)  $\Rightarrow$  as  $\in$  B
  have alpha-bound:
    wp-event (mmap (replicate (length spec) alpha-round)) ?A s3  $\leq$  ?C
  by (rule wp-mmap-alpha-round-alpha-space-set-bound
    [OF future-s3 subset])
  show wp-event
    (mmap (replicate (length spec) alpha-round)  $\gg$ 
      ( $\lambda$ as. return (fr, f-fl, f-final, as)))
  ?Q s3  $\leq$  ?C
proof (rule wp-event-bind-bound-by-head-event[OF alpha-bound])
  show ?Q None  $\Rightarrow$  ?A None
  by simp
next

```



```

fix as t out
assume cont:
  out ∈
    set-dist
      (execute (return (fr, f-fl, f-final, as)) t)
  and hit: ?Q out
have out-eq: out = Some ((fr, f-fl, f-final, as), t)
  using cont
  unfolding set-dist-def return.rep-eq dist-return-def
    dist-delta-dist delta-map-def
  by simp
show ?A (Some (as, t))
  using hit unfolding out-eq by simp
qed
qed
qed
qed
qed

lemma wp-verifier-alpha-prefix-dependent-alpha-space-set-bound:
  fixes C :: prob
  assumes future: alpha-future-fresh s
  and subset:
     $\bigwedge fr\ f-fl\ f-final. B\ fr\ f-fl\ f-final \subseteq \alpha\text{-space}$ 
  and bound:
     $\bigwedge fr\ f-fl\ f-final.$ 
       $nnreal\ (card\ (B\ fr\ f-fl\ f-final)) / nnreal\ (card\ \alpha\text{-space}) \leq C$ 
  shows
    wp-event verifier-alpha-prefix
      ( $\lambda out. case\ out\ of$ 
        None  $\Rightarrow False$ 
        | Some ((fr, f-fl, f-final, as'), -)  $\Rightarrow$ 
           $as' \in B\ fr\ f-fl\ f-final$ ) s
       $\leq C$ 
  proof –
  let ?Q =
     $\lambda out. case\ out\ of$ 
      None  $\Rightarrow False$ 
      | Some ((fr, f-fl, f-final, as'), -)  $\Rightarrow$ 
         $as' \in B\ fr\ f-fl\ f-final$ 
  show ?thesis
  unfolding verifier-alpha-prefix-def
  proof (rule wp-event-bind-bound-by-cont)
  show  $\neg ?Q\ None$ 
  by simp
next
  fix fr s1
  assume read-fr: Some (fr, s1) ∈ set-dist (execute read s)
  have future-s1: alpha-future-fresh s1

```

```

    by (rule read-preserves-alpha-future-fresh[OF future read-fr])
  show wp-event
    (ntimes receive-trace-fri-commits (ceil-log clength)  $\gg$ 
      ( $\lambda f\text{-fl}.$  read  $\gg$ 
        ( $\lambda f\text{-final}.$ 
          mmap (replicate (length spec) alpha-round)  $\gg$ 
            ( $\lambda as.$  return (fr, f-fl, f-final, as))))))
    ?Q s1  $\leq$  C
  proof (rule wp-event-bind-bound-by-cont)
    show  $\neg$  ?Q None
      by simp
  next
    fix f-fl s2
    assume trace-fri:
      Some (f-fl, s2)  $\in$ 
        set-dist
          (execute (ntimes receive-trace-fri-commits (ceil-log clength)) s1)
    have future-s2: alpha-future-fresh s2
      by (rule ntimes-receive-trace-fri-commits-preserves-alpha-future-fresh
        [OF trace-fri future-s1])
    show wp-event
      (read  $\gg$ 
        ( $\lambda f\text{-final}.$ 
          mmap (replicate (length spec) alpha-round)  $\gg$ 
            ( $\lambda as.$  return (fr, f-fl, f-final, as))))
      ?Q s2  $\leq$  C
    proof (rule wp-event-bind-bound-by-cont)
      show  $\neg$  ?Q None
        by simp
    next
      fix f-final s3
      assume read-final:
        Some (f-final, s3)  $\in$  set-dist (execute read s2)
      have future-s3: alpha-future-fresh s3
        by (rule read-preserves-alpha-future-fresh[OF future-s2 read-final])
      let ?B = B fr f-fl f-final
      let ?A =
         $\lambda out.$  case out of
          None  $\Rightarrow$  False
          | Some (as, -)  $\Rightarrow$  as  $\in$  ?B
      have alpha-bound:
        wp-event (mmap (replicate (length spec) alpha-round)) ?A s3  $\leq$ 
          nnreal (card ?B) / nnreal (card alpha-space)
        by (rule wp-mmap-alpha-round-alpha-space-set-bound
          [OF future-s3 subset[of fr f-fl f-final]])
      have alpha-bound-C:
        wp-event (mmap (replicate (length spec) alpha-round)) ?A s3  $\leq$  C
        by (rule order-trans[OF alpha-bound bound[of fr f-fl f-final]])
      show wp-event

```

```

      (mmap (replicate (length spec) alpha-round) >>=
        (λas. return (fr, f-fl, f-final, as)))
    ?Q s3 ≤ C
  proof (rule wp-event-bind-bound-by-head-event[OF alpha-bound-C])
    show ?Q None ⇒ ?A None
      by simp
  next
    fix as t out
    assume cont:
      out ∈
        set-dist
          (execute (return (fr, f-fl, f-final, as)) t)
    and hit: ?Q out
    have out-eq: out = Some ((fr, f-fl, f-final, as), t)
      using cont
    unfolding set-dist-def return.rep-eq dist-return-def
      dist-delta-dist delta-map-def
    by simp
    show ?A (Some (as, t))
      using hit unfolding out-eq by simp
  qed
qed
qed
qed
qed

```

lemma *wp-ntimes-receive-trace-fri-commits-exact-challenges-bound:*

```

  assumes future: trace-fri-future-fresh s
  and len: length bs = n
  shows
    wp-event (ntimes receive-trace-fri-commits n)
      (λout. case out of
        None ⇒ False
        | Some (brs, -) ⇒ map fst brs = bs) s
      ≤ (1 / nnreal size) ^ n
  using len future
proof (induction n arbitrary: s bs)
  case 0
  then have bs-empty: bs = []
    by simp
  show ?case
    unfolding bs-empty by (simp add: wp-event-def wpsimps)
next
  case (Suc n)
  obtain b bs' where bs-eq: bs = b # bs'
    using Suc.prem1 by (cases bs) auto
  have len-bs': length bs' = n
    using Suc.prem1 bs-eq by simp
  let ?tail =

```

```

(ntimes receive-trace-fri-commits n ::
  (('f × 'f) list, ('f, 'a) protocol-channel-scheme) state-monad)
let ?Q =
  λout :: (((('f × 'f) list) ×
    ('f, 'a) protocol-channel-scheme) option.
    case out of None ⇒ False | Some (brs, -) ⇒ map fst brs = bs
let ?Head =
  λout :: (((('f × 'f) × ('f, 'a) protocol-channel-scheme) option.
    case out of None ⇒ False | Some ((b0, r0), -) ⇒ b0 ∈ {b}
have head-bound:
  wp-event receive-trace-fri-commits ?Head s ≤ 1 / nnreal size
proof -
  have wp-event receive-trace-fri-commits ?Head s ≤
    nnreal (card ({b} :: 'f set)) / nnreal size
    by (rule wp-receive-trace-fri-commits-fresh-set[OF Suc.prem2])
  then show ?thesis by simp
qed
have bind-bound:
  wp-event
    (receive-trace-fri-commits ≫
      (λbr :: 'f × 'f.
        ?tail ≫ (λbrs :: ('f × 'f) list. return (br # brs))))
  ?Q s
  ≤ wp-event receive-trace-fri-commits ?Head s *
    (1 / nnreal size) ^ n
proof (rule wp-event-bind-bound-by-head-and-cont[where P = ?Head])
  show ¬ ?Q None
    by simp
next
  fix br t
  assume not-head: ¬ ?Head (Some (br, t))
  obtain b0 r0 where br-eq: br = (b0, r0)
    by (cases br)
  have b0-ne: b0 ≠ b
    using not-head unfolding br-eq by simp
  have cont-false:
    wp-event (?tail ≫ (λbrs :: ('f × 'f) list. return (br # brs)))
      ?Q t
    ≤ wp-event (?tail ≫ (λbrs :: ('f × 'f) list. return (br # brs)))
      (λ- :: (((('f × 'f) list) ×
        ('f, 'a) protocol-channel-scheme) option. False) t
    by (rule wp-event-mono-on-support)
      (use b0-ne bs-eq br-eq in
        ⟨auto simp: wpsimps elim!: set-dist-bindE split: option.splits prod.splits⟩)
  also have ... = 0
    unfolding wp-event-def wp-def dist-expect-def by simp
  finally have le-zero:
    wp-event (?tail ≫ (λbrs :: ('f × 'f) list. return (br # brs)))
      ?Q t ≤ 0 .

```

```

show
  wp-event (?tail  $\gg$  ( $\lambda$ brs :: ('f  $\times$  'f) list. return (br # brs)))
    ?Q t = 0
  by (rule antisym[OF le-zero]) simp
next
fix br t
assume head:
  Some (br, t)  $\in$  set-dist (execute receive-trace-fri-commits s)
  and head-hit: ?Head (Some (br, t))
obtain b0 r0 where br-eq: br = (b0, r0)
  by (cases br)
have b0-eq: b0 = b
  using head-hit unfolding br-eq by simp
have future-t: trace-fri-future-fresh t
  by (rule receive-trace-fri-commits-preserves-trace-fri-future-fresh
    [OF Suc.premis(2) head[unfolded br-eq]])
have tail-bound:
  wp-event ?tail
    ( $\lambda$ out. case out of None  $\Rightarrow$  False
      | Some (brs, -)  $\Rightarrow$  map fst brs = bs') t
     $\leq$  (1 / nnreal size)  $^n$ 
  by (rule Suc.IH[OF len-bs' future-t])
have cont-eq:
  ( $\lambda$ out. case out of
    None  $\Rightarrow$  (0 :: prob)
    | Some (brs, u)  $\Rightarrow$ 
      wp (return (br # brs)) ( $\lambda$ out. if ?Q out then 1 else 0) u) =
  ( $\lambda$ out :: (('f  $\times$  'f) list)  $\times$ 
    ('f, 'a) protocol-channel-scheme) option.
    if (case out of None  $\Rightarrow$  False
      | Some (brs, -)  $\Rightarrow$  map fst brs = bs')
      then 1 else 0)
  by (rule ext)
  (auto simp: wpsimps b0-eq bs-eq br-eq
    split: option.splits prod.splits)
have wp-event (?tail  $\gg$  ( $\lambda$ brs :: ('f  $\times$  'f) list. return (br # brs)))
  ?Q t =
  wp-event ?tail
    ( $\lambda$ out. case out of None  $\Rightarrow$  False
      | Some (brs, -)  $\Rightarrow$  map fst brs = bs') t
  unfolding wp-event-def by (simp add: wpsimps cont-eq)
also have ...  $\leq$  (1 / nnreal size)  $^n$ 
  by (rule tail-bound)
finally show
  wp-event (?tail  $\gg$  ( $\lambda$ brs :: ('f  $\times$  'f) list. return (br # brs)))
  ?Q t  $\leq$  (1 / nnreal size)  $^n$  .
qed
have wp-event (ntimes receive-trace-fri-commits (Suc n)) ?Q s
   $\leq$  wp-event receive-trace-fri-commits ?Head s *

```

$(1 / \text{nnreal size}) \wedge n$
using *bind-bound* **by** *simp*
also have $\dots \leq (1 / \text{nnreal size}) * (1 / \text{nnreal size}) \wedge n$
by (*rule mult-right-mono*[*OF head-bound*]) *simp*
also have $\dots = (1 / \text{nnreal size}) \wedge \text{Suc } n$
by *simp*
finally show *?case* .
qed

lemma *wp-ntimes-receive-trace-fri-commits-finite-set-bound:*

assumes *future: trace-fri-future-fresh s*
and *finite-B: finite B*
and *lengths: $\bigwedge bs. bs \in B \implies \text{length } bs = n$*
shows

wp-event (ntimes receive-trace-fri-commits n)
 $(\lambda \text{out. case out of}$
 $\quad \text{None} \Rightarrow \text{False}$
 $\quad | \text{Some (brs, -)} \Rightarrow \text{map fst brs} \in B) \text{ s}$
 $\leq \text{nnreal (card B) / (nnreal size)} \wedge n$

proof –

let *?P* =
 $\lambda bs \text{ out. case out of}$
 $\quad \text{None} \Rightarrow \text{False}$
 $\quad | \text{Some (brs, -)} \Rightarrow \text{map fst brs} = bs$

have *event-mono:*

wp-event (ntimes receive-trace-fri-commits n)
 $(\lambda \text{out. case out of}$
 $\quad \text{None} \Rightarrow \text{False}$
 $\quad | \text{Some (brs, -)} \Rightarrow \text{map fst brs} \in B) \text{ s}$
 $\leq \text{wp-event (ntimes receive-trace-fri-commits n)}$
 $(\lambda \text{out. } \exists bs \in B. ?P \text{ bs out}) \text{ s}$

by (*rule wp-event-mono*) (*auto split: option.splits prod.splits*)

also have $\dots \leq (\sum bs \in B. (1 / \text{nnreal size}) \wedge n)$

by (*rule wp-event-finite-UN-bound*[*OF finite-B*])

$(\text{rule wp-ntimes-receive-trace-fri-commits-exact-challenges-bound}$
 $\quad [\text{OF future lengths}])$

also have $\dots = \text{nnreal (card B) } * ((1 / \text{nnreal size}) \wedge n)$

using *finite-B* **by** *simp*

also have $\dots = \text{nnreal (card B) / (nnreal size)} \wedge n$

using *size-card* **by** *transfer (simp add: power-divide)*

finally show *?thesis* .

qed

lemma *wp-ntimes-receive-trace-fri-commits-challenge-space-set-bound:*

assumes *future: trace-fri-future-fresh s*

and *subset: $B \subseteq \text{fri-challenge-space } n$*

shows

wp-event (ntimes receive-trace-fri-commits n)
 $(\lambda \text{out. case out of}$

```

      None  $\Rightarrow$  False
    | Some (brs, -)  $\Rightarrow$  map fst brs  $\in B$  s
     $\leq$  nnreal (card B) / nnreal (CARD('f)  $\wedge$  n)
  proof -
    have finite-B: finite B
    by (rule finite-subset[OF subset finite-fri-challenge-space])
    have lengths:  $\bigwedge$  bs. bs  $\in B \Rightarrow$  length bs = n
    using subset unfolding fri-challenge-space-def by auto
    have wp-event (ntimes receive-trace-fri-commits n)
      ( $\lambda$ out. case out of
        None  $\Rightarrow$  False
      | Some (brs, -)  $\Rightarrow$  map fst brs  $\in B$  s
       $\leq$  nnreal (card B) / (nnreal size)  $\wedge$  n
      by (rule wp-ntimes-receive-trace-fri-commits-finite-set-bound
        [OF future finite-B lengths])
    also have ... = nnreal (card B) / nnreal (CARD('f)  $\wedge$  n)
    using size-card by simp
    finally show ?thesis .
  qed

lemma wp-verifier-trace-fri-prefix-challenge-space-set-bound:
  assumes future: trace-fri-future-fresh s
  and subset: B  $\subseteq$  fri-challenge-space (ceil-log clength)
  shows
    wp-event verifier-trace-fri-prefix
      ( $\lambda$ out. case out of
        None  $\Rightarrow$  False
      | Some ((-, f-fl), -)  $\Rightarrow$  map fst f-fl  $\in B$  s
       $\leq$  nnreal (card B) / nnreal (CARD('f)  $\wedge$  ceil-log clength)
  proof -
    let ?Q =
       $\lambda$ out. case out of
        None  $\Rightarrow$  False
      | Some ((-, f-fl), -)  $\Rightarrow$  map fst f-fl  $\in B$ 
    let ?C = nnreal (card B) / nnreal (CARD('f)  $\wedge$  ceil-log clength)
    show ?thesis
      unfolding verifier-trace-fri-prefix-def
    proof (rule wp-event-bind-bound-by-cont)
      show  $\neg$  ?Q None
      by simp
    next
      fix fr s1
      assume read-fr: Some (fr, s1)  $\in$  set-dist (execute read s)
      have future-s1: trace-fri-future-fresh s1
      by (rule read-preserves-trace-fri-future-fresh[OF future read-fr])
      let ?Head =
         $\lambda$ out. case out of
          None  $\Rightarrow$  False
        | Some (f-fl, -)  $\Rightarrow$  map fst f-fl  $\in B$ 

```

```

have trace-bound:
  wp-event (ntimes receive-trace-fri-commits (ceil-log clength))
    ?Head s1 ≤ ?C
  by (rule wp-ntimes-receive-trace-fri-commits-challenge-space-set-bound
    [OF future-s1 subset])
show wp-event
  (ntimes receive-trace-fri-commits (ceil-log clength) ≫=
    (λf-fl. return (fr, f-fl)))
  ?Q s1 ≤ ?C
proof (rule wp-event-bind-bound-by-head-event[OF trace-bound])
  show ?Q None ⇒ ?Head None
  by simp
next
  fix f-fl t out
  assume cont:
    out ∈ set-dist (execute (return (fr, f-fl)) t)
  and hit: ?Q out
  have out-eq: out = Some ((fr, f-fl), t)
  using cont
  unfolding set-dist-def return.rep-eq dist-return-def
    dist-delta-dist delta-map-def
  by simp
  show ?Head (Some (f-fl, t))
  using hit unfolding out-eq by simp
qed
qed
qed

lemma wp-verifier-trace-fri-prefix-dependent-challenge-space-set-bound:
  fixes C :: prob
  assumes future: trace-fri-future-fresh s
  and subset:  $\bigwedge fr. B\ fr \subseteq \text{fri-challenge-space } (\text{ceil-log } \text{clength})$ 
  and bound:
     $\bigwedge fr. \text{nnreal } (\text{card } (B\ fr)) /$ 
     $\text{nnreal } (\text{CARD}(f) \wedge \text{ceil-log } \text{clength}) \leq C$ 
  shows
    wp-event verifier-trace-fri-prefix
      (λout. case out of
        None ⇒ False
        | Some ((fr, f-fl), -) ⇒ map fst f-fl ∈ B fr) s
      ≤ C
proof –
  let ?Q =
    λout. case out of
      None ⇒ False
      | Some ((fr, f-fl), -) ⇒ map fst f-fl ∈ B fr
  show ?thesis
  unfolding verifier-trace-fri-prefix-def
proof (rule wp-event-bind-bound-by-cont)

```



```

show  $\neg ?Q \text{ None}$ 
  by simp
next
fix fr s1
assume read-fr:  $\text{Some } (fr, s1) \in \text{set-dist } (\text{execute read } s)$ 
have future-s1:  $\text{trace-fri-future-fresh } s1$ 
  by (rule read-preserves-trace-fri-future-fresh[OF future read-fr])
let ?Head =
   $\lambda out. \text{case out of}$ 
     $\text{None} \Rightarrow \text{False}$ 
     $|\text{Some } (f\text{-fl}, -) \Rightarrow \text{map fst } f\text{-fl} \in B \text{ fr}$ 
have trace-bound:
   $\text{wp-event } (ntimes \text{ receive-trace-fri-commits } (\text{ceil-log clength}))$ 
   $\text{?Head } s1 \leq$ 
   $\text{nnreal } (\text{card } (B \text{ fr})) / \text{nnreal } (\text{CARD('f)} \wedge \text{ceil-log clength})$ 
  by (rule wp-ntimes-receive-trace-fri-commits-challenge-space-set-bound
    [OF future-s1 subset[of fr]])
have trace-bound-C:
   $\text{wp-event } (ntimes \text{ receive-trace-fri-commits } (\text{ceil-log clength}))$ 
   $\text{?Head } s1 \leq C$ 
  by (rule order-trans[OF trace-bound bound[of fr]])
show wp-event
   $(ntimes \text{ receive-trace-fri-commits } (\text{ceil-log clength})) \gg=$ 
   $(\lambda f\text{-fl}. \text{return } (fr, f\text{-fl}))$ 
   $\text{?Q } s1 \leq C$ 
proof (rule wp-event-bind-bound-by-head-event[OF trace-bound-C])
  show  $\text{?Q None} \implies \text{?Head None}$ 
  by simp
next
fix f-fl t out
assume cont:
   $out \in \text{set-dist } (\text{execute } (\text{return } (fr, f\text{-fl})) \text{ } t)$ 
  and hit:  $\text{?Q out}$ 
have out-eq:  $out = \text{Some } ((fr, f\text{-fl}), t)$ 
  using cont
  unfolding set-dist-def return.rep-eq dist-return-def
  dist-delta-dist delta-map-def
  by simp
show  $\text{?Head } (\text{Some } (f\text{-fl}, t))$ 
  using hit unfolding out-eq by simp
qed
qed
qed

lemma receive-composition-fri-commits-outcome:
assumes outcome:
   $\text{Some } ((b, r), t) \in$ 
   $\text{set-dist } (\text{execute receive-composition-fri-commits } s)$ 
shows

```

```

 $\exists$  rest.
  PTranscript s = r # rest  $\wedge$ 
  PState t = concat (PState s) r  $\wedge$ 
  PTranscript t = rest  $\wedge$ 
  s  $\leq$  t  $\wedge$ 
  PTraceFriCounter t = PTraceFriCounter s  $\wedge$ 
  PCompositionFriCounter t = Suc (PCompositionFriCounter s)  $\wedge$ 
  PAlphaCounter t = PAlphaCounter s  $\wedge$ 
  PQueryCounter t = PQueryCounter s  $\wedge$ 
  fmlookup (HashMap t)
    (CompositionFriChallenge (PCompositionFriCounter s)
      (concat (PState s) r)) =
    Some b
proof -
  from outcome obtain s-read where
    read-r: Some (r, s-read)  $\in$  set-dist (execute read s)
  and rand-b:
    Some (b, t)  $\in$  set-dist (execute receive-composition-fri-challenge s-read)
  unfolding receive-composition-fri-commits-def receive-fri-commits-with-def
  by (auto elim!: set-dist-bindE)
  obtain rest where tr-s: PTranscript s = r # rest
  proof -
    have PTranscript s  $\neq$  []
      using read-r unfolding read-def protocol-read-def assert-def
      by (cases PTranscript s) (auto simp: throw-no-outcome elim!: set-dist-bindE)
    then obtain x xs where xs: PTranscript s = x # xs
      by (cases PTranscript s) auto
    have r-x: r = x
      using read-nonempty-outcome[OF read-r xs] by simp
    show ?thesis
      using that xs r-x by auto
  qed
  have read-res:
    s-read = s(PState := concat (PState s) r, PTranscript := rest)
    using read-nonempty-outcome[OF read-r tr-s] by simp
  have rand-res:
    s-read  $\leq$  t  $\wedge$ 
    PState t = PState s-read  $\wedge$ 
    PTranscript t = PTranscript s-read  $\wedge$ 
    fmlookup (HashMap t)
      (CompositionFriChallenge (PCompositionFriCounter s-read)
        (PState s-read)) =
    Some b
    using receive-composition-fri-challenge-outcome[OF rand-b] .
  have rand-counter:
    PTraceFriCounter t = PTraceFriCounter s-read  $\wedge$ 
    PCompositionFriCounter t = Suc (PCompositionFriCounter s-read)  $\wedge$ 
    PAlphaCounter t = PAlphaCounter s-read  $\wedge$ 
    PQueryCounter t = PQueryCounter s-read

```

```

    using receive-composition-fri-challenge-counter-outcome[OF rand-b] by simp
  have s-read-ext:  $s \leq s\text{-read}$ 
    using read-res unfolding less-eq-hash-ext-def less-eq-fmap-def by simp
  have s-t:  $s \leq t$ 
    using s-read-ext rand-res by (meson hash-ext-trans)
  show ?thesis
    using tr-s read-res rand-res rand-counter s-t by auto
qed

```

lemma *receive-composition-fri-commits-preserves-composition-fri-future-fresh:*

assumes *future: composition-fri-future-fresh s*

and *outcome:*

Some ((b, r), t) ∈ set-dist (execute receive-composition-fri-commits s)

shows *composition-fri-future-fresh t*

proof –

from *outcome* **obtain** *s-read* **where**

read-r: Some (r, s-read) ∈ set-dist (execute read s)

and *rand-b:*

Some (b, t) ∈ set-dist (execute receive-composition-fri-challenge s-read)

unfolding *receive-composition-fri-commits-def receive-fri-commits-with-def*

by (*auto elim!: set-dist-bindE*)

from *read-outcome[OF read-r]* **have** *read-counts:*

PCompositionFriCounter s-read = PCompositionFriCounter s

by *blast*

have *read-hash: HashMap s-read = HashMap s*

by (*rule read-preserves-hash-map[OF read-r]*)

have *counter-t:*

PCompositionFriCounter t = Suc (PCompositionFriCounter s)

using *receive-composition-fri-challenge-counter-outcome[OF rand-b]*

read-counts

by *simp*

show ?thesis

unfolding *composition-fri-future-fresh-def*

proof (*intro allI impI*)

fix *i x*

assume *i-ge-t: PCompositionFriCounter t ≤ i*

have *i-ge-s: PCompositionFriCounter s ≤ i*

using *i-ge-t counter-t* **by** *simp*

have *neq:*

CompositionFriChallenge i x ≠

CompositionFriChallenge

(PCompositionFriCounter s-read) (PState s-read)

using *i-ge-t counter-t read-counts* **by** *auto*

have *fmlookup (HashMap t) (CompositionFriChallenge i x) =*

fmlookup (HashMap s-read) (CompositionFriChallenge i x)

by (*rule receive-composition-fri-challenge-preserves-other-lookup [OF rand-b neq]*)

```

    also have ... = fmlookup (HashMap s) (CompositionFriChallenge i x)
      using read-hash by simp
    also have ... = None
      using future i-ge-s unfolding composition-fri-future-fresh-def by blast
    finally show fmlookup (HashMap t) (CompositionFriChallenge i x) = None .
  qed
qed

lemma receive-composition-fri-commits-preserves-query-future-fresh:
  assumes future: query-future-fresh s
  and outcome:
    Some ((b, r), t) ∈
      set-dist (execute receive-composition-fri-commits s)
  shows query-future-fresh t
proof -
  from outcome obtain s-read where
    read-r: Some (r, s-read) ∈ set-dist (execute read s)
  and rand-b:
    Some (b, t) ∈
      set-dist (execute receive-composition-fri-challenge s-read)
  unfolding receive-composition-fri-commits-def receive-fri-commits-with-def
  by (auto elim!: set-dist-bindE)
  from read-outcome[OF read-r] have read-counts:
    PQueryCounter s-read = PQueryCounter s
  by blast
  have read-hash: HashMap s-read = HashMap s
  by (rule read-preserves-hash-map[OF read-r])
  have counter-t:
    PQueryCounter t = PQueryCounter s
  using receive-composition-fri-challenge-counter-outcome[OF rand-b]
  read-counts
  by simp
  show ?thesis
  unfolding query-future-fresh-def
proof (intro allI impI)
  fix i x
  assume i-ge-t: PQueryCounter t ≤ i
  have i-ge-s: PQueryCounter s ≤ i
    using i-ge-t counter-t by simp
  have neg:
    QueryIndexChallenge i x ≠
      CompositionFriChallenge
        (PCompositionFriCounter s-read) (PState s-read)
  by simp
  have fmlookup (HashMap t) (QueryIndexChallenge i x) =
    fmlookup (HashMap s-read) (QueryIndexChallenge i x)
  by (rule receive-composition-fri-challenge-preserves-other-lookup
    [OF rand-b neg])
  also have ... = fmlookup (HashMap s) (QueryIndexChallenge i x)

```

```

    using read-hash by simp
  also have ... = None
    using future i-ge-s unfolding query-future-fresh-def by blast
  finally show fmlookup (HashMap t) (QueryIndexChallenge i x) = None .
qed
qed

lemma wp-receive-composition-fri-commits-fresh-set:
  assumes future: composition-fri-future-fresh s
  shows
    wp-event receive-composition-fri-commits
      (λout. case out of
        None ⇒ False
        | Some ((b, r), -) ⇒ b ∈ B) s
      ≤ nnreal (card B) / nnreal size
    unfolding receive-composition-fri-commits-def receive-fri-commits-with-def
  proof (rule wp-event-bind-bound-by-cont)
    show ¬ (case None of
      None ⇒ False | Some ((b, r), -) ⇒ b ∈ B)
      by simp
  next
    fix r s-read
    assume read-r: Some (r, s-read) ∈ set-dist (execute read s)
    have read-counts:
      PCompositionFriCounter s-read = PCompositionFriCounter s
      using read-outcome[OF read-r] by blast
    have read-hash: HashMap s-read = HashMap s
      by (rule read-preserves-hash-map[OF read-r])
    have fresh:
      fmlookup (HashMap s-read)
        (CompositionFriChallenge
          (PCompositionFriCounter s-read) (PState s-read)) = None
      using future read-counts read-hash
      unfolding composition-fri-future-fresh-def by simp
    let ?Q =
      λout :: (('f × 'f) × ('f, 'a) protocol-channel-scheme) option.
        case out of None ⇒ False | Some ((b, r), -) ⇒ b ∈ B
    have cont-eq:
      (λout. case out of
        None ⇒ (0 :: prob)
        | Some (b, u) ⇒
          wp (return (b, r)) (λout. if ?Q out then 1 else 0) u) =
      (λout :: ('f × ('f, 'a) protocol-channel-scheme) option.
        if (case out of None ⇒ False | Some (b, -) ⇒ b ∈ B)
        then 1 else 0)
      by (rule ext) (auto simp: wpsimps split: option.splits prod.splits)
    have wp-event
      (receive-composition-fri-challenge ≫ (λb :: 'f. return (b, r)))
      ?Q s-read =

```

$wp\text{-event } receive\text{-composition-fri-challenge}$
 $(\lambda out. \text{case out of None} \Rightarrow \text{False} \mid \text{Some } (b, -) \Rightarrow b \in B)$
 $s\text{-read}$
unfolding $wp\text{-event-def}$ **by** ($simp \text{ add: } wpsimps \text{ cont-eq}$)
also have $\dots = nnreal \text{ (card } B) / nnreal \text{ size}$
by ($rule \text{ wp-receive-composition-fri-challenge-fresh-set[OF fresh]}$)
finally show
 $wp\text{-event}$
 $(receive\text{-composition-fri-challenge} \gg= (\lambda b :: 'f. \text{return } (b, r)))$
 $(\lambda out. \text{case out of None} \Rightarrow \text{False} \mid \text{Some } ((b, r), -) \Rightarrow b \in B)$
 $s\text{-read}$
 $\leq nnreal \text{ (card } B) / nnreal \text{ size}$
by $simp$
qed

lemma $ntimes\text{-receive-composition-fri-commits-preserves-composition-fri-future-fresh}$:
assumes $outcome$:
 $\text{Some } (brs, t) \in$
 $set\text{-dist } (execute \text{ (ntimes receive-composition-fri-commits } n) \text{ } s)$
and $future$: $composition\text{-fri-future-fresh } s$
shows $composition\text{-fri-future-fresh } t$
using $outcome \text{ future}$
proof ($induction \text{ } n \text{ arbitrary: } s \text{ } brs \text{ } t$)
case 0
then show $?case$ **by** $simp$
next
case ($Suc \text{ } n$)
from $Suc.prem(1)$ **obtain** $br \text{ } brs' \text{ } s1$ **where**
 $head$:
 $\text{Some } (br, s1) \in$
 $set\text{-dist } (execute \text{ receive-composition-fri-commits } s)$
and $tail$:
 $\text{Some } (brs', t) \in$
 $set\text{-dist } (execute \text{ (ntimes receive-composition-fri-commits } n) \text{ } s1)$
and $brs\text{-eq}$: $brs = br \# brs'$
by ($auto \text{ elim!: } set\text{-dist-bindE}$)
obtain $b \text{ } r$ **where** $br\text{-eq}$: $br = (b, r)$
by ($cases \text{ } br$)
have $future\text{-}s1$:
 $composition\text{-fri-future-fresh } s1$
by ($rule \text{ receive-composition-fri-commits-preserves-composition-fri-future-fresh}$
 $[OF \text{ } Suc.prem(2) \text{ } head[unfolding \text{ } br\text{-eq}]]$)
show $?case$
by ($rule \text{ } Suc.IH[OF \text{ } tail \text{ } future\text{-}s1]$)
qed

lemma $ntimes\text{-receive-composition-fri-commits-preserves-query-future-fresh}$:
assumes $outcome$:
 $\text{Some } (brs, t) \in$

```

      set-dist (execute (ntimes receive-composition-fri-commits n) s)
    and future: query-future-fresh s
  shows query-future-fresh t
  using outcome future
proof (induction n arbitrary: s brs t)
  case 0
  then show ?case by simp
next
  case (Suc n)
  from Suc.prem1(1) obtain br brs' s1 where
    head:
      Some (br, s1) ∈
        set-dist (execute receive-composition-fri-commits s)
    and tail:
      Some (brs', t) ∈
        set-dist (execute (ntimes receive-composition-fri-commits n) s1)
  by (auto elim!: set-dist-bindE)
  obtain b r where br-eq: br = (b, r)
  by (cases br)
  have future-s1:
    query-future-fresh s1
  by (rule receive-composition-fri-commits-preserves-query-future-fresh
      [OF Suc.prem1(2) head[unfolded br-eq]])
  show ?case
  by (rule Suc.IH[OF tail future-s1])
qed

lemma wp-ntimes-receive-composition-fri-commits-exact-challenges-bound:
  assumes future: composition-fri-future-fresh s
  and len: length bs = n
  shows
    wp-event (ntimes receive-composition-fri-commits n)
      (λout. case out of
        None ⇒ False
        | Some (brs, -) ⇒ map fst brs = bs) s
      ≤ (1 / nnreal size) ^ n
  using len future
proof (induction n arbitrary: s bs)
  case 0
  then have bs-empty: bs = []
  by simp
  show ?case
  unfolding bs-empty by (simp add: wp-event-def wpsimps)
next
  case (Suc n)
  obtain b bs' where bs-eq: bs = b # bs'
  using Suc.prem1(1) by (cases bs) auto
  have len-bs': length bs' = n
  using Suc.prem1(1) bs-eq by simp

```

```

let ?tail =
  (ntimes receive-composition-fri-commits n ::
    (('f × 'f) list, ('f, 'a) protocol-channel-scheme) state-monad)
let ?Q =
  λout :: (((('f × 'f) list) ×
    ('f, 'a) protocol-channel-scheme) option.
    case out of None ⇒ False | Some (brs, -) ⇒ map fst brs = bs
let ?Head =
  λout :: (((('f × 'f) × ('f, 'a) protocol-channel-scheme) option.
    case out of None ⇒ False | Some ((b0, r0), -) ⇒ b0 ∈ {b}
have head-bound:
  wp-event receive-composition-fri-commits ?Head s ≤ 1 / nnreal size
proof -
  have wp-event receive-composition-fri-commits ?Head s ≤
    nnreal (card ({b} :: 'f set)) / nnreal size
  by (rule wp-receive-composition-fri-commits-fresh-set[OF Suc.premis(2)])
  then show ?thesis by simp
qed
have bind-bound:
  wp-event
    (receive-composition-fri-commits ≫
      (λbr :: 'f × 'f.
        ?tail ≫ (λbrs :: ('f × 'f) list. return (br # brs))))
  ?Q s
  ≤ wp-event receive-composition-fri-commits ?Head s *
    (1 / nnreal size) ^ n
proof (rule wp-event-bind-bound-by-head-and-cont[where P = ?Head])
  show ¬ ?Q None
  by simp
next
  fix br t
  assume not-head: ¬ ?Head (Some (br, t))
  obtain b0 r0 where br-eq: br = (b0, r0)
  by (cases br)
  have b0-ne: b0 ≠ b
  using not-head unfolding br-eq by simp
  have cont-false:
    wp-event (?tail ≫ (λbrs :: ('f × 'f) list. return (br # brs)))
    ?Q t
    ≤ wp-event (?tail ≫ (λbrs :: ('f × 'f) list. return (br # brs)))
      (λ- :: (((('f × 'f) list) ×
        ('f, 'a) protocol-channel-scheme) option. False) t
  by (rule wp-event-mono-on-support)
  (use b0-ne bs-eq br-eq in
    ⟨auto simp: wpsimps elim!: set-dist-bindE split: option.splits prod.splits⟩)
  also have ... = 0
  unfolding wp-event-def wp-def dist-expect-def by simp
  finally have le-zero:
    wp-event (?tail ≫ (λbrs :: ('f × 'f) list. return (br # brs)))

```



```

    ?Q t ≤ 0 .
show
  wp-event (?tail ≫ (λbrs :: ('f × 'f) list. return (br # brs)))
    ?Q t = 0
  by (rule antisym[OF le-zero]) simp
next
fix br t
assume head:
  Some (br, t) ∈
    set-dist (execute receive-composition-fri-commits s)
  and head-hit: ?Head (Some (br, t))
obtain b0 r0 where br-eq: br = (b0, r0)
  by (cases br)
have b0-eq: b0 = b
  using head-hit unfolding br-eq by simp
have future-t: composition-fri-future-fresh t
  by (rule receive-composition-fri-commits-preserves-composition-fri-future-fresh
    [OF Suc.premis(2) head[unfolded br-eq]])
have tail-bound:
  wp-event ?tail
    (λout. case out of None ⇒ False
      | Some (brs, -) ⇒ map fst brs = bs') t
    ≤ (1 / nnreal size) ^ n
  by (rule Suc.IH[OF len-bs' future-t])
have cont-eq:
  (λout. case out of
    None ⇒ (0 :: prob)
    | Some (brs, u) ⇒
      wp (return (br # brs)) (λout. if ?Q out then 1 else 0) u) =
  (λout :: (('f × 'f) list) ×
    ('f, 'a) protocol-channel-scheme) option.
    if (case out of None ⇒ False
      | Some (brs, -) ⇒ map fst brs = bs')
      then 1 else 0)
  by (rule ext)
  (auto simp: wpsimps b0-eq bs-eq br-eq
    split: option.splits prod.splits)
have wp-event (?tail ≫ (λbrs :: ('f × 'f) list. return (br # brs)))
  ?Q t =
  wp-event ?tail
    (λout. case out of None ⇒ False
      | Some (brs, -) ⇒ map fst brs = bs') t
  unfolding wp-event-def by (simp add: wpsimps cont-eq)
also have ... ≤ (1 / nnreal size) ^ n
  by (rule tail-bound)
finally show
  wp-event (?tail ≫ (λbrs :: ('f × 'f) list. return (br # brs)))
    ?Q t ≤ (1 / nnreal size) ^ n .
qed

```

have *wp-event* (*ntimes receive-composition-fri-commits* (*Suc n*)) *?Q s*
 $\leq$ *wp-event receive-composition-fri-commits ?Head s* *
 $(1 / \text{nnreal size})^{\wedge} n$
using *bind-bound* **by** *simp*
also have ... $\leq (1 / \text{nnreal size}) * (1 / \text{nnreal size})^{\wedge} n$
by (*rule mult-right-mono[OF head-bound]*) *simp*
also have ... $= (1 / \text{nnreal size})^{\wedge} \text{Suc } n$
by *simp*
finally show *?case* .
qed

lemma *wp-ntimes-receive-composition-fri-commits-finite-set-bound*:
assumes *future: composition-fri-future-fresh s*
and *finite-B: finite B*
and *lengths: $\bigwedge bs. bs \in B \implies \text{length } bs = n$*
shows
wp-event (ntimes receive-composition-fri-commits n)
 $(\lambda \text{out. case out of}$
 $\quad \text{None} \Rightarrow \text{False}$
 $\quad | \text{Some (brs, -)} \Rightarrow \text{map fst brs} \in B) \text{ s}$
 $\leq \text{nnreal (card B) / (nnreal size)}^{\wedge} n$

proof –
let *?P* =
 $\lambda bs \text{ out. case out of}$
 $\quad \text{None} \Rightarrow \text{False}$
 $\quad | \text{Some (brs, -)} \Rightarrow \text{map fst brs} = bs$
have *event-mono*:
wp-event (ntimes receive-composition-fri-commits n)
 $(\lambda \text{out. case out of}$
 $\quad \text{None} \Rightarrow \text{False}$
 $\quad | \text{Some (brs, -)} \Rightarrow \text{map fst brs} \in B) \text{ s}$
 $\leq \text{wp-event (ntimes receive-composition-fri-commits n)}$
 $(\lambda \text{out. } \exists bs \in B. ?P \text{ bs out}) \text{ s}$
by (*rule wp-event-mono*) (*auto split: option.splits prod.splits*)
also have ... $\leq (\sum bs \in B. (1 / \text{nnreal size})^{\wedge} n)$
by (*rule wp-event-finite-UN-bound[OF finite-B]*)
 $(\text{rule wp-ntimes-receive-composition-fri-commits-exact-challenges-bound$
 $\quad \text{[OF future lengths]})$
also have ... $= \text{nnreal (card B)} * ((1 / \text{nnreal size})^{\wedge} n)$
using *finite-B* **by** *simp*
also have ... $= \text{nnreal (card B) / (nnreal size)}^{\wedge} n$
using *size-card* **by** *transfer (simp add: power-divide)*
finally show *?thesis* .
qed

lemma *wp-ntimes-receive-composition-fri-commits-challenge-space-set-bound*:
assumes *future: composition-fri-future-fresh s*
and *subset: $B \subseteq \text{fri-challenge-space } n$*
shows

```

wp-event (ntimes receive-composition-fri-commits n)
  (λout. case out of
    None ⇒ False
    | Some (brs, -) ⇒ map fst brs ∈ B) s
  ≤ nnreal (card B) / nnreal (CARD('f) ^ n)
proof -
  have finite-B: finite B
  by (rule finite-subset[OF subset finite-fri-challenge-space])
  have lengths: ∧bs. bs ∈ B ⇒ length bs = n
  using subset unfolding fri-challenge-space-def by auto
  have wp-event (ntimes receive-composition-fri-commits n)
    (λout. case out of
      None ⇒ False
      | Some (brs, -) ⇒ map fst brs ∈ B) s
    ≤ nnreal (card B) / (nnreal size) ^ n
  by (rule wp-ntimes-receive-composition-fri-commits-finite-set-bound
    [OF future finite-B lengths])
  also have ... = nnreal (card B) / nnreal (CARD('f) ^ n)
  using size-card by simp
  finally show ?thesis .
qed

lemma wp-verifier-composition-fri-prefix-challenge-space-set-bound:
  assumes future: composition-fri-future-fresh s
  and subset:
    ∧dg. B dg ⊆ fri-challenge-space (ceil-log (to-nat dg + 1))
  and bound:
    ∧dg. nnreal (card (B dg)) /
      nnreal (CARD('f) ^ ceil-log (to-nat dg + 1)) ≤ C
  shows
    wp-event verifier-composition-fri-prefix
      (λout. case out of
        None ⇒ False
        | Some ((-, -, -, dg, fl), -) ⇒ map fst fl ∈ B dg) s
      ≤ C
proof -
  let ?Q =
    λout. case out of
      None ⇒ False
      | Some ((-, -, -, dg, fl), -) ⇒ map fst fl ∈ B dg
  show ?thesis
  unfolding verifier-composition-fri-prefix-def
proof (rule wp-event-bind-bound-by-cont)
  show ¬ ?Q None
  by simp
next
fix fr s1
assume read-fr: Some (fr, s1) ∈ set-dist (execute read s)
have future-s1: composition-fri-future-fresh s1

```

```

    by (rule read-preserves-composition-fri-future-fresh
        [OF future read-fr])
  show wp-event
    (ntimes receive-trace-fri-commits (ceil-log clength)  $\gg$ 
      ( $\lambda f.fl.$  read  $\gg$ 
        ( $\lambda f.final.$ 
          mmap (replicate (length spec) alpha-round)  $\gg$ 
            ( $\lambda as.$  read  $\gg$ 
              ( $\lambda dg.$ 
                assert (to-nat dg  $\leq$  maxDegree)  $\gg$ 
                  ( $\lambda -.$  ntimes receive-composition-fri-commits
                    (ceil-log (to-nat dg + 1))  $\gg$ 
                      ( $\lambda fl.$ 
                        return (fr, f-fl, f-final, as, dg, fl)))))))
    ?Q s1  $\leq$  C
  proof (rule wp-event-bind-bound-by-cont)
    show  $\neg$  ?Q None
      by simp
  next
    fix f-fl s2
    assume trace-fri:
      Some (f-fl, s2)  $\in$ 
        set-dist
          (execute (ntimes receive-trace-fri-commits (ceil-log clength)) s1)
    have future-s2: composition-fri-future-fresh s2
      by (rule ntimes-receive-trace-fri-commits-preserves-composition-fri-future-fresh
          [OF trace-fri future-s1])
    show wp-event
      (read  $\gg$ 
        ( $\lambda f.final.$ 
          mmap (replicate (length spec) alpha-round)  $\gg$ 
            ( $\lambda as.$  read  $\gg$ 
              ( $\lambda dg.$ 
                assert (to-nat dg  $\leq$  maxDegree)  $\gg$ 
                  ( $\lambda -.$  ntimes receive-composition-fri-commits
                    (ceil-log (to-nat dg + 1))  $\gg$ 
                      ( $\lambda fl.$ 
                        return (fr, f-fl, f-final, as, dg, fl)))))))
      ?Q s2  $\leq$  C
  proof (rule wp-event-bind-bound-by-cont)
    show  $\neg$  ?Q None
      by simp
  next
    fix f-final s3
    assume read-final: Some (f-final, s3)  $\in$  set-dist (execute read s2)
    have future-s3: composition-fri-future-fresh s3
      by (rule read-preserves-composition-fri-future-fresh
          [OF future-s2 read-final])
    show wp-event

```

```

    (mmap (replicate (length spec) alpha-round) >>=
      (λas. read >>=
        (λdg.
          assert (to-nat dg ≤ maxDegree) >>=
            (λ-. ntimes receive-composition-fri-commits
              (ceil-log (to-nat dg + 1)) >>=
                (λfl.
                  return (fr, f-fl, f-final, as, dg, fl))))))
  ?Q s3 ≤ C
proof (rule wp-event-bind-bound-by-cont)
  show ¬ ?Q None
  by simp
next
  fix as s4
  assume alpha:
    Some (as, s4) ∈
      set-dist
        (execute (mmap (replicate (length spec) alpha-round)) s3)
  have future-s4: composition-fri-future-fresh s4
  by (rule mmap-alpha-round-preserves-composition-fri-future-fresh
    [OF future-s3 alpha])
  show wp-event
    (read >>=
      (λdg.
        assert (to-nat dg ≤ maxDegree) >>=
          (λ-. ntimes receive-composition-fri-commits
            (ceil-log (to-nat dg + 1)) >>=
              (λfl.
                return (fr, f-fl, f-final, as, dg, fl))))))
  ?Q s4 ≤ C
proof (rule wp-event-bind-bound-by-cont)
  show ¬ ?Q None
  by simp
next
  fix dg s5
  assume read-dg: Some (dg, s5) ∈ set-dist (execute read s4)
  have future-s5: composition-fri-future-fresh s5
  by (rule read-preserves-composition-fri-future-fresh
    [OF future-s4 read-dg])
  show wp-event
    (assert (to-nat dg ≤ maxDegree) >>=
      (λ-. ntimes receive-composition-fri-commits
        (ceil-log (to-nat dg + 1)) >>=
          (λfl.
            return (fr, f-fl, f-final, as, dg, fl))))
  ?Q s5 ≤ C
proof (rule wp-event-bind-bound-by-cont)
  show ¬ ?Q None
  by simp

```

```

next
  fix u s6
  assume assert-out:
    Some (u, s6) ∈
      set-dist (execute (assert (to-nat dg ≤ maxDegree)) s5)
  have s6-eq: s6 = s5
    using assert-out unfolding assert-def
    by (cases to-nat dg ≤ maxDegree)
      (auto simp: throw-no-outcome)
  have future-s6: composition-fri-future-fresh s6
    using future-s5 unfolding s6-eq .
  let ?Head =
    λout. case out of
      None ⇒ False
    | Some (fl, -) ⇒ map fst fl ∈ B dg
  have comp-bound:
    wp-event
      (ntimes receive-composition-fri-commits
        (ceil-log (to-nat dg + 1)))
    ?Head s6 ≤
      nnreal (card (B dg)) /
      nnreal (CARD('f) ^ ceil-log (to-nat dg + 1))
    by (rule
      wp-ntimes-receive-composition-fri-commits-challenge-space-set-bound
      [OF future-s6 subset])
  have comp-bound-C:
    wp-event
      (ntimes receive-composition-fri-commits
        (ceil-log (to-nat dg + 1)))
    ?Head s6 ≤ C
    by (rule order-trans[OF comp-bound bound])
  show wp-event
    (ntimes receive-composition-fri-commits
      (ceil-log (to-nat dg + 1)) ≫=
      (λfl.
        return (fr, f-fl, f-final, as, dg, fl)))
    ?Q s6 ≤ C
  proof (rule wp-event-bind-bound-by-head-event[OF comp-bound-C])
    show ?Q None ⇒ ?Head None
      by simp
  next
    fix fl t out
    assume cont:
      out ∈
        set-dist
          (execute
            (return (fr, f-fl, f-final, as, dg, fl)) t)
    and hit: ?Q out
    have out-eq:

```

```

    out = Some ((fr, f-fl, f-final, as, dg, fl), t)
  using cont
  unfolding set-dist-def return.rep-eq dist-return-def
    dist-delta-dist delta-map-def
  by simp
  show ?Head (Some (fl, t))
    using hit unfolding out-eq by simp
qed
qed
qed
qed
qed
qed
qed
qed

```

lemma *ntimes-receive-fri-commits-with-outcome:*

```

fixes receive-commits :: ('f, 'f × 'f, 'a) protocol-c-monad
and counter :: ('f, 'a) protocol-channel-scheme ⇒ nat
and preserve1 :: ('f, 'a) protocol-channel-scheme ⇒ nat
and preserve2 :: ('f, 'a) protocol-channel-scheme ⇒ nat
and preserve3 :: ('f, 'a) protocol-channel-scheme ⇒ nat
and tag :: nat ⇒ 'f ⇒ 'f protocol-hash-input

```

assumes *step-outcome:*

```

  ∧ s b r t. Some ((b, r), t) ∈ set-dist (execute receive-commits s) ⇒
    ∃ rest.

```

```

  PTranscript s = r # rest ∧
  PState t = concat (PState s) r ∧
  PTranscript t = rest ∧
  s ≤ t ∧

```

```

  counter t = Suc (counter s) ∧

```

```

  preserve1 t = preserve1 s ∧

```

```

  preserve2 t = preserve2 s ∧

```

```

  preserve3 t = preserve3 s ∧

```

```

  fmlookup (HashMap t) (tag (counter s) (concat (PState s) r)) = Some b

```

and *outcome:*

```

  Some (brs, t) ∈ set-dist (execute (ntimes receive-commits n) s)

```

shows

```

  length brs = n ∧

```

```

  PTranscript s = map snd brs @ PTranscript t ∧

```

```

  PState t = foldl concat (PState s) (map snd brs) ∧

```

```

  s ≤ t ∧

```

```

  counter t = counter s + n ∧

```

```

  preserve1 t = preserve1 s ∧

```

```

  preserve2 t = preserve2 s ∧

```

```

  preserve3 t = preserve3 s ∧

```

```

  (∀ i < n.

```

```

    fmlookup (HashMap t)

```

```

      (tag (counter s + i)

```

```

      (foldl concat (PState s) (take (Suc i) (map snd brs)))) =
      Some (fst (brs ! i)))
  using outcome
proof (induction n arbitrary: s brs t)
  case 0
  then show ?case
    by (simp add: hash-ext-refl)
next
  case (Suc n)
  from Suc.premis obtain br brs' s1 where
    head: Some (br, s1) ∈ set-dist (execute receive-commits s)
    and tail:
      Some (brs', t) ∈
        set-dist (execute (ntimes receive-commits n) s1)
    and brs-eq: brs = br # brs'
    by (auto elim!: set-dist-bindE)
  obtain b r where br-eq: br = (b, r)
    by (cases br)
  from step-outcome[OF head[unfolded br-eq]]
  obtain rest1 where
    tr-s: PTranscript s = r # rest1
    and st-s1: PState s1 = concat (PState s) r
    and tr-s1: PTranscript s1 = rest1
    and ext-head: s ≤ s1
    and counter-s1: counter s1 = Suc (counter s)
    and preserve1-s1: preserve1 s1 = preserve1 s
    and preserve2-s1: preserve2 s1 = preserve2 s
    and preserve3-s1: preserve3 s1 = preserve3 s
    and lookup-head:
      fmlookup (HashMap s1) (tag (counter s) (concat (PState s) r)) = Some b
    by blast
  have tail-res:
    length brs' = n ∧
    PTranscript s1 = map snd brs' @ PTranscript t ∧
    PState t = foldl concat (PState s1) (map snd brs') ∧
    s1 ≤ t ∧
    counter t = counter s1 + n ∧
    preserve1 t = preserve1 s1 ∧
    preserve2 t = preserve2 s1 ∧
    preserve3 t = preserve3 s1 ∧
    (∀ i < n.
      fmlookup (HashMap t)
        (tag (counter s1 + i)
          (foldl concat (PState s1) (take (Suc i) (map snd brs')))) =
          Some (fst (brs' ! i)))
    using Suc.IH[OF tail] .
  have ext-s1-t: s1 ≤ t
    using tail-res by simp
  have lookup-head-t:

```



```

  fmlookup (HashMap t) (tag (counter s) (concat (PState s) r)) = Some b
  using hash-extension-lookup[OF lookup-head ext-s1-t] .
have ext-combined:  $s \leq t$ 
  using ext-head ext-s1-t by (meson hash-ext-trans)
have counter-t: counter t = counter s + Suc n
  using tail-res counter-s1 by simp
have preserves-t:
  preserve1 t = preserve1 s  $\wedge$ 
  preserve2 t = preserve2 s  $\wedge$ 
  preserve3 t = preserve3 s
  using tail-res preserve1-s1 preserve2-s1 preserve3-s1 by simp
have lookups:
   $\forall i < \text{Suc } n.$ 
  fmlookup (HashMap t)
    (tag (counter s + i)
      (foldl concat (PState s) (take (Suc i) (map snd brs)))) =
    Some (fst (brs ! i))
proof (intro allI impI)
  fix i
  assume i-bound:  $i < \text{Suc } n$ 
  show fmlookup (HashMap t)
    (tag (counter s + i)
      (foldl concat (PState s) (take (Suc i) (map snd brs)))) =
    Some (fst (brs ! i))
  proof (cases i)
    case 0
    then show ?thesis
      using brs-eq br-eq lookup-head-t by simp
  next
    case (Suc j)
    have j-bound:  $j < n$ 
    using i-bound Suc by simp
    have state-shift:
      foldl concat (PState s) (take (Suc (Suc j)) (map snd brs)) =
      foldl concat (PState s1) (take (Suc j) (map snd brs'))
    using brs-eq br-eq st-s1 by simp
    have brs-nth:  $\text{brs} ! \text{Suc } j = \text{brs}' ! j$ 
    using brs-eq j-bound by simp
    show ?thesis
      using tail-res j-bound Suc state-shift brs-nth counter-s1 by simp
  qed
qed
have as-eq:
  PTranscript s = map snd brs @ PTranscript t  $\wedge$ 
  PState t = foldl concat (PState s) (map snd brs)
  using tail-res tr-s tr-s1 st-s1 brs-eq br-eq by simp
show ?case
  using tail-res tr-s tr-s1 st-s1 ext-combined lookups as-eq brs-eq br-eq
  counter-t preserves-t

```

by simp
qed

lemma *ntimes-receive-trace-fri-commits-outcome:*

assumes *outcome:*

Some (brs, t) ∈
set-dist (execute (ntimes receive-trace-fri-commits n) s)

shows

length brs = n ∧
PTranscript s = map snd brs @ PTranscript t ∧
PState t = foldl concat (PState s) (map snd brs) ∧
s ≤ t ∧
PTraceFriCounter t = PTraceFriCounter s + n ∧
PCompositionFriCounter t = PCompositionFriCounter s ∧
PAlphaCounter t = PAlphaCounter s ∧
PQueryCounter t = PQueryCounter s ∧
(∀ i < n.
fmlookup (HashMap t)
(TraceFriChallenge (PTraceFriCounter s + i)
(foldl concat (PState s) (take (Suc i) (map snd brs)))) =
Some (fst (brs ! i)))

proof –

have *res:*

length brs = n ∧
PTranscript s = map snd brs @ PTranscript t ∧
PState t = foldl concat (PState s) (map snd brs) ∧
s ≤ t ∧
PTraceFriCounter t = PTraceFriCounter s + n ∧
PCompositionFriCounter t = PCompositionFriCounter s ∧
PAlphaCounter t = PAlphaCounter s ∧
PQueryCounter t = PQueryCounter s ∧
(∀ i < n.
fmlookup (HashMap t)
(TraceFriChallenge (PTraceFriCounter s + i)
(foldl concat (PState s) (take (Suc i) (map snd brs)))) =
Some (fst (brs ! i)))

by (rule *ntimes-receive-fri-commits-with-outcome*
[of *receive-trace-fri-commits PTraceFriCounter PCompositionFriCounter*
PAlphaCounter PQueryCounter TraceFriChallenge,
OF receive-trace-fri-commits-outcome outcome])

then show *?thesis* .

qed

lemma *ntimes-receive-composition-fri-commits-outcome:*

assumes *outcome:*

Some (brs, t) ∈
set-dist (execute (ntimes receive-composition-fri-commits n) s)

shows

length brs = n ∧

$PTranscript\ s = \text{map}\ \text{snd}\ \text{brs} \ @\ PTranscript\ t \wedge$
 $PState\ t = \text{foldl}\ \text{concat}\ (PState\ s)\ (\text{map}\ \text{snd}\ \text{brs}) \wedge$
 $s \leq t \wedge$
 $PCompositionFriCounter\ t = PCompositionFriCounter\ s + n \wedge$
 $PTraceFriCounter\ t = PTraceFriCounter\ s \wedge$
 $PAlphaCounter\ t = PAlphaCounter\ s \wedge$
 $PQueryCounter\ t = PQueryCounter\ s \wedge$
 $(\forall i < n.$
 $\quad \text{fmlookup}\ (\text{HashMap}\ t)$
 $\quad (\text{CompositionFriChallenge}\ (PCompositionFriCounter\ s + i)$
 $\quad (\text{foldl}\ \text{concat}\ (PState\ s)\ (\text{take}\ (\text{Suc}\ i)\ (\text{map}\ \text{snd}\ \text{brs})))) =$
 $\quad \text{Some}\ (\text{fst}\ (\text{brs}\ !\ i)))$

proof –

have *composition-step-outcome*:

$\bigwedge s\ b\ r\ t. \text{Some}\ ((b, r), t) \in$
 $\quad \text{set-dist}\ (\text{execute}\ \text{receive-composition-fri-commits}\ s) \implies$
 $\exists \text{rest}.$
 $\quad PTranscript\ s = r \ \# \ \text{rest} \wedge$
 $\quad PState\ t = \text{concat}\ (PState\ s)\ r \wedge$
 $\quad PTranscript\ t = \text{rest} \wedge$
 $\quad s \leq t \wedge$
 $\quad PCompositionFriCounter\ t = \text{Suc}\ (PCompositionFriCounter\ s) \wedge$
 $\quad PTraceFriCounter\ t = PTraceFriCounter\ s \wedge$
 $\quad PAlphaCounter\ t = PAlphaCounter\ s \wedge$
 $\quad PQueryCounter\ t = PQueryCounter\ s \wedge$
 $\quad \text{fmlookup}\ (\text{HashMap}\ t)$
 $\quad (\text{CompositionFriChallenge}\ (PCompositionFriCounter\ s)$
 $\quad (\text{concat}\ (PState\ s)\ r)) =$
 $\quad \text{Some}\ b$

using *receive-composition-fri-commits-outcome* **by** *blast*

have *res*:

$\text{length}\ \text{brs} = n \wedge$
 $PTranscript\ s = \text{map}\ \text{snd}\ \text{brs} \ @\ PTranscript\ t \wedge$
 $PState\ t = \text{foldl}\ \text{concat}\ (PState\ s)\ (\text{map}\ \text{snd}\ \text{brs}) \wedge$
 $s \leq t \wedge$
 $PCompositionFriCounter\ t = PCompositionFriCounter\ s + n \wedge$
 $PTraceFriCounter\ t = PTraceFriCounter\ s \wedge$
 $PAlphaCounter\ t = PAlphaCounter\ s \wedge$
 $PQueryCounter\ t = PQueryCounter\ s \wedge$
 $(\forall i < n.$
 $\quad \text{fmlookup}\ (\text{HashMap}\ t)$
 $\quad (\text{CompositionFriChallenge}\ (PCompositionFriCounter\ s + i)$
 $\quad (\text{foldl}\ \text{concat}\ (PState\ s)\ (\text{take}\ (\text{Suc}\ i)\ (\text{map}\ \text{snd}\ \text{brs})))) =$
 $\quad \text{Some}\ (\text{fst}\ (\text{brs}\ !\ i)))$

using *composition-step-outcome outcome*

by (*rule ntimes-receive-fri-commits-with-outcome*
 $[\text{of}\ \text{receive-composition-fri-commits}\ PCompositionFriCounter$
 $PTraceFriCounter\ PAlphaCounter\ PQueryCounter\ \text{CompositionFriChallenge}]$)

then show *?thesis* .

qed

lemma *ntimes-receive-composition-fri-commits-roots-outcome:*

assumes *outcome:*

Some (brs, t) ∈

set-dist (execute (ntimes receive-composition-fri-commits n) s)

shows *map snd brs = take n (PTranscript s)*

proof –

have *res:*

length brs = n ∧ PTranscript s = map snd brs @ PTranscript t

using *ntimes-receive-composition-fri-commits-outcome[OF outcome]* **by** *simp*

have *len: length (map snd brs) = n*

using *res* **by** *simp*

have *take n (PTranscript s) = take n (map snd brs @ PTranscript t)*

using *res* **by** *simp*

also have *... = map snd brs*

using *len* **by** *simp*

finally show *?thesis*

by *simp*

qed

lemma *wp-ntimes-receive-composition-fri-commits-root-dependent-challenge-space-set-bound:*

fixes *B :: 'f list ⇒ 'f list set*

assumes *future: composition-fri-future-fresh s*

and *subset: ⋀ roots. B roots ⊆ fri-challenge-space n*

shows

wp-event (ntimes receive-composition-fri-commits n)

(λ out. case out of

None ⇒ False

| Some (brs, -) ⇒ map fst brs ∈ B (map snd brs)) s

≤ nnreal (card (B (take n (PTranscript s)))) /

nnreal (CARD('f) ^ n)

proof –

let *?roots = take n (PTranscript s)*

let *?Q =*

λ out. case out of

None ⇒ False

| Some (brs, -) ⇒ map fst brs ∈ B (map snd brs)

let *?Q' =*

λ out. case out of

None ⇒ False

| Some (brs, -) ⇒ map fst brs ∈ B ?roots

have *fixed-bound:*

wp-event (ntimes receive-composition-fri-commits n) ?Q' s

≤ nnreal (card (B ?roots)) / nnreal (CARD('f) ^ n)

by (*rule wp-ntimes-receive-composition-fri-commits-challenge-space-set-bound*
[OF future subset])

have *event-le:*

wp-event (ntimes receive-composition-fri-commits n) ?Q s

```

    ≤ wp-event (ntimes receive-composition-fri-commits n) ?Q' s
proof (rule wp-event-mono-on-support)
  fix out
  assume out-supp:
    out ∈ set-dist (execute (ntimes receive-composition-fri-commits n) s)
    and hit: ?Q out
  show ?Q' out
proof (cases out)
  case None
  then show ?thesis
    using hit by simp
  next
  case (Some pair)
  then obtain brs t where out-eq: out = Some (brs, t)
    by (cases pair) auto
  have roots-eq: map snd brs = ?roots
    by (rule ntimes-receive-composition-fri-commits-roots-outcome)
    (use out-supp out-eq in simp)
  show ?thesis
    using hit roots-eq out-eq by simp
qed
qed
show ?thesis
  by (rule order-trans[OF event-le fixed-bound])
qed

lemma wp-ntimes-receive-composition-fri-commits-root-dependent-bound:
  fixes C :: prob
  assumes future: composition-fri-future-fresh s
  and subset:  $\bigwedge \text{roots}. B \text{ roots} \subseteq \text{fri-challenge-space } n$ 
  and bound:
     $\bigwedge \text{roots}. \text{nnreal } (\text{card } (B \text{ roots})) / \text{nnreal } (\text{CARD}('f) ^ n) \leq C$ 
  shows
    wp-event (ntimes receive-composition-fri-commits n)
      (λout. case out of
        None ⇒ False
        | Some (brs, -) ⇒ map fst brs ∈ B (map snd brs)) s
    ≤ C
proof –
  have root-bound:
    wp-event (ntimes receive-composition-fri-commits n)
      (λout. case out of
        None ⇒ False
        | Some (brs, -) ⇒ map fst brs ∈ B (map snd brs)) s
    ≤ nnreal (card (B (take n (PTranscript s)))) /
      nnreal (CARD('f) ^ n)
  by (rule
    wp-ntimes-receive-composition-fri-commits-root-dependent-challenge-space-set-bound

```

```

    [OF future subset])
  also have ... ≤ C
  by (rule bound)
  finally show ?thesis .
qed

lemma wp-verifier-composition-fri-prefix-root-dependent-challenge-space-set-bound:
  fixes C :: prob
  assumes future: composition-fri-future-fresh s
  and subset:
    ∧fr f-fl f-final as dg roots.
    B fr f-fl f-final as dg roots ⊆
    fri-challenge-space (ceil-log (to-nat dg + 1))
  and bound:
    ∧fr f-fl f-final as dg roots.
    nnreal (card (B fr f-fl f-final as dg roots)) /
    nnreal (CARD('f) ^ ceil-log (to-nat dg + 1)) ≤ C
  shows
    wp-event verifier-composition-fri-prefix
    (λout. case out of
      None ⇒ False
      | Some ((fr, f-fl, f-final, as, dg, fl), -) ⇒
        map fst fl ∈ B fr f-fl f-final as dg (map snd fl)) s
    ≤ C
  proof -
    let ?Q =
      λout. case out of
        None ⇒ False
        | Some ((fr, f-fl, f-final, as, dg, fl), -) ⇒
          map fst fl ∈ B fr f-fl f-final as dg (map snd fl)
    show ?thesis
      unfolding verifier-composition-fri-prefix-def
    proof (rule wp-event-bind-bound-by-cont)
      show ¬ ?Q None
        by simp
    next
      fix fr s1
      assume read-fr: Some (fr, s1) ∈ set-dist (execute read s)
      have future-s1: composition-fri-future-fresh s1
        by (rule read-preserves-composition-fri-future-fresh
          [OF future read-fr])
      show wp-event
        (ntimes receive-trace-fri-commits (ceil-log clength) ≫=
          (λf-fl. read ≫=
            (λf-final.
              mmap (replicate (length spec) alpha-round) ≫=
                (λas. read ≫=
                  (λdg.
                    assert (to-nat dg ≤ maxDegree) ≫=

```

```

      (λ-. ntimes receive-composition-fri-commits
        (ceil-log (to-nat dg + 1)) >=>=
        (λfl.
          return (fr, f-fl, f-final, as, dg, fl))))))
    ?Q s1 ≤ C
  proof (rule wp-event-bind-bound-by-cont)
    show ¬ ?Q None
      by simp
  next
    fix f-fl s2
    assume trace-fri:
      Some (f-fl, s2) ∈
        set-dist
          (execute (ntimes receive-trace-fri-commits (ceil-log clength)) s1)
    have future-s2: composition-fri-future-fresh s2
      by (rule ntimes-receive-trace-fri-commits-preserves-composition-fri-future-fresh
        [OF trace-fri future-s1])
    show wp-event
      (read >=>=
        (λf-final.
          mmap (replicate (length spec) alpha-round) >=>=
            (λas. read >=>=
              (λdg.
                assert (to-nat dg ≤ maxDegree) >=>=
                  (λ-. ntimes receive-composition-fri-commits
                    (ceil-log (to-nat dg + 1)) >=>=
                    (λfl.
                      return (fr, f-fl, f-final, as, dg, fl)))))))
        ?Q s2 ≤ C
    proof (rule wp-event-bind-bound-by-cont)
      show ¬ ?Q None
        by simp
    next
      fix f-final s3
      assume read-final: Some (f-final, s3) ∈ set-dist (execute read s2)
      have future-s3: composition-fri-future-fresh s3
        by (rule read-preserves-composition-fri-future-fresh
          [OF future-s2 read-final])
      show wp-event
        (mmap (replicate (length spec) alpha-round) >=>=
          (λas. read >=>=
            (λdg.
              assert (to-nat dg ≤ maxDegree) >=>=
                (λ-. ntimes receive-composition-fri-commits
                  (ceil-log (to-nat dg + 1)) >=>=
                  (λfl.
                    return (fr, f-fl, f-final, as, dg, fl)))))))
        ?Q s3 ≤ C
    proof (rule wp-event-bind-bound-by-cont)

```

```

show  $\neg ?Q \text{ None}$ 
  by simp
next
  fix as s4
  assume alpha:
    Some (as, s4) ∈
      set-dist
      (execute (mmap (replicate (length spec) alpha-round)) s3)
  have future-s4: composition-fri-future-fresh s4
    by (rule mmap-alpha-round-preserves-composition-fri-future-fresh
      [OF future-s3 alpha])
  show wp-event
    (read  $\gg$ 
      ( $\lambda dg.$ 
        assert (to-nat dg ≤ maxDegree)  $\gg$ 
        ( $\lambda.$  ntimes receive-composition-fri-commits
          (ceil-log (to-nat dg + 1))  $\gg$ 
          ( $\lambda fl.$ 
            return (fr, f-fl, f-final, as, dg, fl)))))
     $?Q s4 \leq C$ 
  proof (rule wp-event-bind-bound-by-cont)
    show  $\neg ?Q \text{ None}$ 
      by simp
  next
  fix dg s5
  assume read-dg: Some (dg, s5) ∈ set-dist (execute read s4)
  have future-s5: composition-fri-future-fresh s5
    by (rule read-preserves-composition-fri-future-fresh
      [OF future-s4 read-dg])
  show wp-event
    (assert (to-nat dg ≤ maxDegree)  $\gg$ 
      ( $\lambda.$  ntimes receive-composition-fri-commits
        (ceil-log (to-nat dg + 1))  $\gg$ 
        ( $\lambda fl.$ 
          return (fr, f-fl, f-final, as, dg, fl)))))
     $?Q s5 \leq C$ 
  proof (rule wp-event-bind-bound-by-cont)
    show  $\neg ?Q \text{ None}$ 
      by simp
  next
  fix u s6
  assume assert-out:
    Some (u, s6) ∈
      set-dist (execute (assert (to-nat dg ≤ maxDegree)) s5)
  have s6-eq: s6 = s5
    using assert-out unfolding assert-def
    by (cases to-nat dg ≤ maxDegree
      (auto simp: throw-no-outcome))
  have future-s6: composition-fri-future-fresh s6

```



```

    using future-s5 unfolding s6-eq .
  let ?Head =
    λout. case out of
      None ⇒ False
    | Some (fl, -) ⇒
      map fst fl ∈ B fr f-fl f-final as dg (map snd fl)
  have comp-bound-C:
    wp-event
      (ntimes receive-composition-fri-commits
        (ceil-log (to-nat dg + 1)))
    ?Head s6 ≤ C
  by (rule
      wp-ntimes-receive-composition-fri-commits-root-dependent-bound
      [where B = B fr f-fl f-final as dg,
        OF future-s6])
    (use subset bound in simp-all)
  show wp-event
    (ntimes receive-composition-fri-commits
      (ceil-log (to-nat dg + 1)) >>=
      (λfl.
        return (fr, f-fl, f-final, as, dg, fl)))
    ?Q s6 ≤ C
  proof (rule wp-event-bind-bound-by-head-event[OF comp-bound-C])
    show ?Q None ⇒ ?Head None
    by simp
  next
    fix fl t out
    assume cont:
      out ∈
        set-dist
          (execute
            (return (fr, f-fl, f-final, as, dg, fl)) t)
    and hit: ?Q out
    have out-eq:
      out = Some ((fr, f-fl, f-final, as, dg, fl), t)
    using cont
    unfolding set-dist-def return.rep-eq dist-return-def
      dist-delta-dist delta-map-def
    by simp
    show ?Head (Some (fl, t))
    using hit unfolding out-eq by simp
  qed
qed
qed
qed
qed
qed
qed
qed

```

lemma *verifier-alpha-prefix-outcome*:

assumes *outcome*:

Some ((fr, f-fl, f-final, as), t) ∈
set-dist (execute verifier-alpha-prefix s)

shows

length f-fl = ceil-log clength ∧
length as = length spec ∧
PTranscript s =
[fr] @ map snd f-fl @ [f-final] @ as @ PTranscript t ∧
PState t =
foldl concat
(concat (foldl concat (concat (PState s) fr) (map snd f-fl))
f-final)
as ∧
PQueryCounter t = PQueryCounter s

proof –

from *outcome* **obtain** *s1 s2 s3 s4* **where**

read-fr: Some (fr, s1) ∈ set-dist (execute read s)

and *trace-fri*:

Some (f-fl, s2) ∈
set-dist (execute
(ntimes receive-trace-fri-commits (ceil-log clength)) s1)

and *read-final: Some (f-final, s3) ∈ set-dist (execute read s2)*

and *alpha-out*:

Some (as, s4) ∈
set-dist (execute (mmap (replicate (length spec) alpha-round)) s3)

and *ret*:

Some ((fr, f-fl, f-final, as), t) ∈
set-dist (execute (return (fr, f-fl, f-final, as)) s4)

unfolding *verifier-alpha-prefix-def*

by (*auto elim!: set-dist-bindE*)

from *read-outcome[OF read-fr]* **obtain** *rest1* **where**

tr-s: PTranscript s = fr # rest1

and *st-s1: PState s1 = concat (PState s) fr*

and *tr-s1: PTranscript s1 = rest1*

and *query-count-s1: PQueryCounter s1 = PQueryCounter s*

by *blast*

have *trace-res*:

length f-fl = ceil-log clength ∧
PTranscript s1 = map snd f-fl @ PTranscript s2 ∧
PState s2 = foldl concat (PState s1) (map snd f-fl) ∧
PQueryCounter s2 = PQueryCounter s1

using *ntimes-receive-trace-fri-commits-outcome[OF trace-fri]* **by** *simp*

from *read-outcome[OF read-final]* **obtain** *rest3* **where**

tr-s2: PTranscript s2 = f-final # rest3

and *st-s3: PState s3 = concat (PState s2) f-final*

and *tr-s3: PTranscript s3 = rest3*

and *query-count-s3: PQueryCounter s3 = PQueryCounter s2*

```

  by blast
have alpha-res:
  length as = length spec ∧
  PTranscript s3 = as @ PTranscript s4 ∧
  PState s4 = foldl concat (PState s3) as ∧
  PQueryCounter s4 = PQueryCounter s3
  using mmap-alpha-round-outcome[OF alpha-out] by simp
have t-eq: t = s4
  using ret by simp
show ?thesis
  using tr-s tr-s1 trace-res tr-s2 tr-s3 alpha-res st-s1 st-s3
    query-count-s1 query-count-s3 t-eq
  by simp
qed

lemma verifier-trace-fri-prefix-outcome:
  assumes outcome:
    Some ((fr, f-fl), t) ∈
      set-dist (execute verifier-trace-fri-prefix s)
  shows
    length f-fl = ceil-log clength ∧
    PTranscript s = [fr] @ map snd f-fl @ PTranscript t ∧
    PState t = foldl concat (concat (PState s) fr) (map snd f-fl) ∧
    s ≤ t ∧
    PTraceFriCounter t = PTraceFriCounter s + ceil-log clength ∧
    PCompositionFriCounter t = PCompositionFriCounter s ∧
    PAlphaCounter t = PAlphaCounter s ∧
    PQueryCounter t = PQueryCounter s ∧
    (∀ i < length f-fl.
      fmllookup (HashMap t)
        (TraceFriChallenge (PTraceFriCounter s + i)
          (foldl concat (concat (PState s) fr)
            (take (Suc i) (map snd f-fl)))) =
        Some (fst (f-fl ! i)))
  proof -
    from outcome obtain s1 s2 where
      read-fr: Some (fr, s1) ∈ set-dist (execute read s)
    and trace-fri:
      Some (f-fl, s2) ∈
        set-dist (execute
          (ntimes receive-trace-fri-commits (ceil-log clength)) s1)
    and ret:
      Some ((fr, f-fl), t) ∈ set-dist (execute (return (fr, f-fl)) s2)
    unfolding verifier-trace-fri-prefix-def
    by (auto elim!: set-dist-bindE)
  from read-outcome[OF read-fr] obtain rest1 where
    tr-s: PTranscript s = fr # rest1
  and st-s1: PState s1 = concat (PState s) fr
  and tr-s1: PTranscript s1 = rest1

```

```

and ext-s-s1:  $s \leq s1$ 
and trace-count-s1:  $PTraceFriCounter\ s1 = PTraceFriCounter\ s$ 
and comp-count-s1:  $PCompositionFriCounter\ s1 = PCompositionFriCounter\ s$ 
and alpha-count-s1:  $PAlphaCounter\ s1 = PAlphaCounter\ s$ 
and query-count-s1:  $PQueryCounter\ s1 = PQueryCounter\ s$ 
by blast
have trace-res:
  length f-fl = ceil-log clength  $\wedge$ 
  PTranscript s1 = map snd f-fl @ PTranscript s2  $\wedge$ 
  PState s2 = foldl concat (PState s1) (map snd f-fl)  $\wedge$ 
  s1  $\leq$  s2  $\wedge$ 
  PTraceFriCounter s2 = PTraceFriCounter s1 + ceil-log clength  $\wedge$ 
  PCompositionFriCounter s2 = PCompositionFriCounter s1  $\wedge$ 
  PAlphaCounter s2 = PAlphaCounter s1  $\wedge$ 
  PQueryCounter s2 = PQueryCounter s1  $\wedge$ 
  ( $\forall i < \text{ceil-log clength}.$ 
    fmlookup (HashMap s2)
      (TraceFriChallenge (PTraceFriCounter s1 + i)
        (foldl concat (PState s1) (take (Suc i) (map snd f-fl)))) =
      Some (fst (f-fl ! i)))
  using ntimes-receive-trace-fri-commits-outcome[OF trace-fri] by simp
have t-eq:  $t = s2$ 
  using ret by simp
have ext-s-t:  $s \leq t$ 
  using ext-s-s1 trace-res t-eq by (meson hash-ext-trans)
have lookups:
   $\forall i < \text{length f-fl}.$ 
    fmlookup (HashMap t)
      (TraceFriChallenge (PTraceFriCounter s + i)
        (foldl concat (concat (PState s) fr)
          (take (Suc i) (map snd f-fl)))) =
      Some (fst (f-fl ! i))
  using trace-res st-s1 trace-count-s1 t-eq by simp
show ?thesis
  using tr-s tr-s1 st-s1 trace-res t-eq ext-s-t trace-count-s1
    comp-count-s1 alpha-count-s1 query-count-s1 lookups
  by simp
qed

```

```

lemma verifier-composition-fri-prefix-outcome:
assumes outcome:
  Some ((fr, f-fl, f-final, as, dg, fl), t)  $\in$ 
    set-dist (execute verifier-composition-fri-prefix s)
shows
  length f-fl = ceil-log clength  $\wedge$ 
  length as = length spec  $\wedge$ 
  length fl = ceil-log (to-nat dg + 1)  $\wedge$ 
  PTranscript s =
    [fr] @ map snd f-fl @ [f-final] @ as @ [dg] @

```

```

    map snd fl @ PTranscript t ∧
PState t =
  foldl concat
    (concat
      (foldl concat
        (concat
          (foldl concat (concat (PState s) fr) (map snd f-fl))
            f-final)
        as)
      dg)
    (map snd fl) ∧
s ≤ t ∧
PQueryCounter t = PQueryCounter s ∧
(∀ i < length f-fl.
  fmllookup (HashMap t)
    (TraceFriChallenge (PTraceFriCounter s + i)
      (foldl concat (concat (PState s) fr)
        (take (Suc i) (map snd f-fl)))) =
    Some (fst (f-fl ! i))) ∧
(∀ i < length fl.
  fmllookup (HashMap t)
    (CompositionFriChallenge (PCompositionFriCounter s + i)
      (foldl concat
        (concat
          (foldl concat
            (concat
              (foldl concat (concat (PState s) fr) (map snd f-fl))
                f-final)
            as)
          dg)
        (take (Suc i) (map snd fl)))) =
    Some (fst (fl ! i)))
proof –
from outcome obtain s1 s2 s3 s4 s5 s6 s7 where
  read-fr: Some (fr, s1) ∈ set-dist (execute read s)
and trace-fri:
  Some (f-fl, s2) ∈
    set-dist (execute
      (ntimes receive-trace-fri-commits (ceil-log clength)) s1)
and read-f-final: Some (f-final, s3) ∈ set-dist (execute read s2)
and alpha-out:
  Some (as, s4) ∈
    set-dist (execute (mmap (replicate (length spec) alpha-round)) s3)
and read-dg: Some (dg, s5) ∈ set-dist (execute read s4)
and degree-assert:
  Some ((), s6) ∈
    set-dist (execute (assert (to-nat dg ≤ maxDegree)) s5)
and comp-fri:
  Some (fl, s7) ∈

```

```

    set-dist (execute
      (ntimes receive-composition-fri-commits
        (ceil-log (to-nat dg + 1))) s6)
  and ret:
    Some ((fr, f-fl, f-final, as, dg, fl), t) ∈
      set-dist (execute (return (fr, f-fl, f-final, as, dg, fl)) s7)
  unfolding verifier-composition-fri-prefix-def
  by (auto elim!: set-dist-bindE)
from read-outcome[OF read-fr] obtain rest1 where
  tr-s: PTranscript s = fr # rest1
  and st-s1: PState s1 = concat (PState s) fr
  and tr-s1: PTranscript s1 = rest1
  and ext-s-s1: s ≤ s1
  and trace-count-s1: PTraceFriCounter s1 = PTraceFriCounter s
  and comp-count-s1: PCompositionFriCounter s1 = PCompositionFriCounter s
  and alpha-count-s1: PAlphaCounter s1 = PAlphaCounter s
  and query-count-s1: PQueryCounter s1 = PQueryCounter s
  by blast
have trace-res:
  length f-fl = ceil-log clength ∧
  PTranscript s1 = map snd f-fl @ PTranscript s2 ∧
  PState s2 = foldl concat (PState s1) (map snd f-fl) ∧
  s1 ≤ s2 ∧
  PTraceFriCounter s2 = PTraceFriCounter s1 + ceil-log clength ∧
  PCompositionFriCounter s2 = PCompositionFriCounter s1 ∧
  PAlphaCounter s2 = PAlphaCounter s1 ∧
  PQueryCounter s2 = PQueryCounter s1 ∧
  (∀ i < ceil-log clength.
    fmllookup (HashMap s2)
      (TraceFriChallenge (PTraceFriCounter s1 + i)
        (foldl concat (PState s1) (take (Suc i) (map snd f-fl)))) =
      Some (fst (f-fl ! i)))
  using ntimes-receive-trace-fri-commits-outcome[OF trace-fr] by simp
from read-outcome[OF read-f-final] obtain rest3 where
  tr-s2: PTranscript s2 = f-final # rest3
  and st-s3: PState s3 = concat (PState s2) f-final
  and tr-s3: PTranscript s3 = rest3
  and ext-s2-s3: s2 ≤ s3
  and comp-count-s3: PCompositionFriCounter s3 = PCompositionFriCounter s2
  and query-count-s3: PQueryCounter s3 = PQueryCounter s2
  by blast
have alpha-res:
  length as = length spec ∧
  PTranscript s3 = as @ PTranscript s4 ∧
  PState s4 = foldl concat (PState s3) as ∧
  s3 ≤ s4 ∧
  PCompositionFriCounter s4 = PCompositionFriCounter s3 ∧
  PQueryCounter s4 = PQueryCounter s3
  using mmap-alpha-round-outcome[OF alpha-out] by simp

```

```

from read-outcome[OF read-dg] obtain rest5 where
  tr-s4: PTranscript s4 = dg # rest5
  and st-s5: PState s5 = concat (PState s4) dg
  and tr-s5: PTranscript s5 = rest5
  and ext-s4-s5: s4 ≤ s5
  and comp-count-s5: PCompositionFriCounter s5 = PCompositionFriCounter s4
  and query-count-s5: PQueryCounter s5 = PQueryCounter s4
  by blast
have s6-eq: s6 = s5
  using degree-assert unfolding assert-def
  by (cases to-nat dg ≤ maxDegree) (auto simp: throw-no-outcome)
have ext-s5-s6: s5 ≤ s6
  unfolding s6-eq by (rule hash-ext-refl)
have comp-res:
  length fl = ceil-log (to-nat dg + 1) ∧
  PTranscript s6 = map snd fl @ PTranscript s7 ∧
  PState s7 = foldl concat (PState s6) (map snd fl) ∧
  s6 ≤ s7 ∧
  PCompositionFriCounter s7 =
    PCompositionFriCounter s6 + ceil-log (to-nat dg + 1) ∧
  PQueryCounter s7 = PQueryCounter s6 ∧
  (∀ i < ceil-log (to-nat dg + 1).
    fmllookup (HashMap s7)
      (CompositionFriChallenge (PCompositionFriCounter s6 + i)
        (foldl concat (PState s6) (take (Suc i) (map snd fl)))) =
      Some (fst (fl ! i)))
  using ntimes-receive-composition-fri-commits-outcome[OF comp-fri]
  by simp
have t-eq: t = s7
  using ret by simp
have ext-s-t: s ≤ t
  using ext-s-s1 trace-res ext-s2-s3 alpha-res ext-s4-s5 ext-s5-s6
    comp-res t-eq
  by (meson hash-ext-trans)
have ext-s2-t: s2 ≤ t
  using ext-s2-s3 alpha-res ext-s4-s5 ext-s5-s6 comp-res t-eq
  by (meson hash-ext-trans)
have trace-lookup:
  ∀ i < length f-fl.
    fmllookup (HashMap t)
      (TraceFriChallenge (PTraceFriCounter s + i)
        (foldl concat (concat (PState s) fr)
          (take (Suc i) (map snd f-fl)))) =
      Some (fst (f-fl ! i))
proof (intro allI impI)
  fix i
  assume i-bound: i < length f-fl
  have lookup-s2:
    fmllookup (HashMap s2)

```

```

    (TraceFriChallenge (PTraceFriCounter s1 + i)
      (foldl concat (PState s1) (take (Suc i) (map snd f-fl)))) =
    Some (fst (f-fl ! i))
  using trace-res i-bound by simp
show fmlookup (HashMap t)
  (TraceFriChallenge (PTraceFriCounter s + i)
    (foldl concat (concat (PState s) fr)
      (take (Suc i) (map snd f-fl)))) =
  Some (fst (f-fl ! i))
  using hash-extension-lookup[OF lookup-s2 ext-s2-t] st-s1
    trace-count-s1
  by simp
qed
have comp-lookup:
  ∀ i < length fl.
  fmlookup (HashMap t)
    (CompositionFriChallenge (PCompositionFriCounter s + i)
      (foldl concat
        (concat
          (foldl concat
            (concat
              (foldl concat (concat (PState s) fr) (map snd f-fl))
                f-final)
            as)
          dg)
        (take (Suc i) (map snd fl)))) =
    Some (fst (fl ! i))
proof (intro allI impI)
  fix i
  assume i-bound: i < length fl
  have lookup-s7:
    fmlookup (HashMap s7)
      (CompositionFriChallenge (PCompositionFriCounter s6 + i)
        (foldl concat (PState s6) (take (Suc i) (map snd fl)))) =
    Some (fst (fl ! i))
  using comp-res i-bound by simp
show fmlookup (HashMap t)
  (CompositionFriChallenge (PCompositionFriCounter s + i)
    (foldl concat
      (concat
        (foldl concat
          (concat
            (foldl concat (concat (PState s) fr) (map snd f-fl))
              f-final)
            as)
          dg)
        (take (Suc i) (map snd fl)))) =
  Some (fst (fl ! i))
  using lookup-s7 t-eq st-s1 trace-res st-s3 alpha-res st-s5 s6-eq

```



```

      comp-count-s1 comp-count-s3 comp-count-s5
    by simp
  qed
  have query-count-t: PQueryCounter t = PQueryCounter s
    using query-count-s1 trace-res query-count-s3 alpha-res query-count-s5
      s6-eq comp-res t-eq
    by simp
  show ?thesis
    using tr-s tr-s1 trace-res tr-s2 tr-s3 alpha-res tr-s4 tr-s5 s6-eq
      comp-res st-s1 st-s3 st-s5 t-eq ext-s-t query-count-t trace-lookup
      comp-lookup
    by simp
  qed
end
end
end

```

```

theory Soundness-Execution-Extraction
imports Soundness-Execution-Prefix
begin

```

Accepted verifier execution transcript extraction and witness lemmas.

```

context soundness
begin

```

```

lemma verify-monad-hash-extends:
  assumes outcome:
    Some (result, final-state) ∈ set-dist (execute verify-monad s)
  shows s ≤ final-state
proof —
  from outcome[unfolded verify-monad-composition-fri-decomposition]
  obtain header prefix-state where
    prefix:
      Some (header, prefix-state) ∈
        set-dist (execute verifier-composition-fri-prefix s)
  and suffix:
    Some (result, final-state) ∈
      set-dist
        (execute (verifier-after-composition-fri header) prefix-state)
  by (auto elim!: set-dist-bindE)
  obtain fr f-fl f-final as dg fl where header-eq:
    header = (fr, f-fl, f-final, as, dg, fl)
  by (cases header) auto
  have ext-prefix: s ≤ prefix-state
  using verifier-composition-fri-prefix-outcome
    [OF prefix[unfolded header-eq]]
  by simp

```

from *suffix*[*unfolded header-eq verifier-after-composition-fri-def*]
obtain *final query-state* **where**
read-final:
Some (final, query-state) ∈ set-dist (execute read prefix-state)
and *query-out*:
Some (result, final-state) ∈
set-dist
(execute
(ntimes
(verifier-query-round-program fr f-fl f-final as fl final)
rounds)
query-state)
by (*auto elim!:* *set-dist-bindE*)
have *ext-query: prefix-state ≤ query-state*
using *read-outcome[OF read-final]* **by** *blast*
have *ext-final: query-state ≤ final-state*
using *ntimes-verifier-query-rounds-outcome[OF query-out]* **by** *blast*
show *?thesis*
using *ext-prefix ext-query ext-final* **by** (*meson hash-ext-trans*)
qed

lemma *alpha-header-supported-trace-table-candidate-state-hash-extends*:
assumes
alpha-header-supported-trace-table-candidate-state s fr f-fri-roots
f-final trace-table final-state
shows *s ≤ final-state*
proof –
from *assms* **obtain** *result composition-table as query-idxs dg*
composition-fri-roots final rest **where**
outcome:
Some (result, final-state) ∈ set-dist (execute verify-monad s)
unfolding *alpha-header-supported-trace-table-candidate-state-def*
by *blast*
show *?thesis*
by (*rule verify-monad-hash-extends[OF outcome]*)
qed

lemma *alpha-header-supported-pairwise-merkle-bad-extends-initial*:
assumes *bad*:
alpha-header-supported-pairwise-merkle-bad s fr f-fri-roots f-final
obtains *trace-table final-state trace-table' final-state'*
where
alpha-header-supported-trace-table-candidate-state s fr f-fri-roots
f-final trace-table final-state
alpha-header-supported-trace-table-candidate-state s fr f-fri-roots
f-final trace-table' final-state'
trace-table ≠ trace-table'
merkle-hash-value-conflict final-state final-state' ∨
hash-map-output-collision

$(\text{merkle-hash-state-merge } \text{final-state } \text{final-state}')$
 $s \leq \text{final-state}$
 $s \leq \text{final-state}'$
proof –
from *bad* **obtain** *trace-table final-state trace-table' final-state'* **where**
candidate:
 $\text{alpha-header-supported-trace-table-candidate-state } s \text{ fr } f\text{-fri-roots}$
 $f\text{-final trace-table final-state}$
and *candidate':*
 $\text{alpha-header-supported-trace-table-candidate-state } s \text{ fr } f\text{-fri-roots}$
 $f\text{-final trace-table' final-state'}$
and *distinct:* $\text{trace-table} \neq \text{trace-table}'$
and *pair-bad:*
 $\text{merkle-hash-value-conflict final-state final-state}' \vee$
 $\text{hash-map-output-collision}$
 $(\text{merkle-hash-state-merge final-state final-state}')$
unfolding *alpha-header-supported-pairwise-merkle-bad-def* **by** *blast*
have *ext:* $s \leq \text{final-state}$
by (*rule alpha-header-supported-trace-table-candidate-state-hash-extends*
 $[\text{OF candidate}]$)
have *ext':* $s \leq \text{final-state}'$
by (*rule alpha-header-supported-trace-table-candidate-state-hash-extends*
 $[\text{OF candidate}']$)
show *?thesis*
by (*rule that* $[\text{OF candidate candidate' distinct pair-bad ext ext'}]$)
qed

definition *alpha-header-supported-pairwise-coupling-bad*
 $:: (f, 'a) \text{ protocol-channel-scheme} \Rightarrow$
 $f \Rightarrow f \text{ list} \Rightarrow f \Rightarrow \text{bool}$
where
 $\text{alpha-header-supported-pairwise-coupling-bad } s \text{ fr } f\text{-fri-roots } f\text{-final}$
 $\longleftrightarrow$
 $(\exists \text{ trace-table final-state trace-table' final-state'.$
 $\text{alpha-header-supported-trace-table-candidate-state } s \text{ fr } f\text{-fri-roots}$
 $f\text{-final trace-table final-state} \wedge$
 $\text{alpha-header-supported-trace-table-candidate-state } s \text{ fr } f\text{-fri-roots}$
 $f\text{-final trace-table' final-state'} \wedge$
 $\text{trace-table} \neq \text{trace-table}' \wedge$
 $\text{merkle-hash-pairwise-coupling-bad final-state final-state'})$

definition *alpha-supported-pairwise-coupling-bad*
 $:: (f, 'a) \text{ protocol-channel-scheme} \Rightarrow \text{bool}$
where
 $\text{alpha-supported-pairwise-coupling-bad } s \longleftrightarrow$
 $(\exists \text{ fr } f\text{-fri-roots } f\text{-final.}$
 $\text{alpha-header-supported-pairwise-coupling-bad } s \text{ fr } f\text{-fri-roots}$
 $f\text{-final})$

lemma *alpha-header-supported-pairwise-merkle-bad-imp-local-collision-or-coupling-bad:*

assumes *bad:*

alpha-header-supported-pairwise-merkle-bad s fr f-fri-roots f-final

shows

$(\exists \text{ trace-table final-state trace-table' final-state'.$
alpha-header-supported-trace-table-candidate-state s fr f-fri-roots
f-final trace-table final-state $\wedge$
alpha-header-supported-trace-table-candidate-state s fr f-fri-roots
f-final trace-table' final-state' $\wedge$
 $\text{trace-table} \neq \text{trace-table}' \wedge$
 $(\text{hash-map-output-collision final-state} \vee$
 $\text{hash-map-output-collision final-state'})) \vee$
alpha-header-supported-pairwise-coupling-bad s fr f-fri-roots f-final

proof —

from *bad* **obtain** *trace-table final-state trace-table' final-state'*

where *candidate:*

alpha-header-supported-trace-table-candidate-state s fr f-fri-roots
f-final trace-table final-state

and *candidate':*

alpha-header-supported-trace-table-candidate-state s fr f-fri-roots
f-final trace-table' final-state'

and *distinct:* $\text{trace-table} \neq \text{trace-table}'$

and *pair-bad:*

merkle-hash-value-conflict final-state final-state' $\vee$
hash-map-output-collision
 $(\text{merkle-hash-state-merge final-state final-state'})$

unfolding *alpha-header-supported-pairwise-merkle-bad-def* **by** *blast*

have *decomp:*

hash-map-output-collision final-state $\vee$
hash-map-output-collision final-state' $\vee$
merkle-hash-pairwise-coupling-bad final-state final-state'

by $(\text{rule merkle-hash-pairwise-bad-imp-local-collision-or-coupling-bad}$
 $[\text{OF pair-bad}])$

show *?thesis*

using *candidate candidate' distinct decomp*

unfolding *alpha-header-supported-pairwise-coupling-bad-def* **by** *blast*

qed

lemma *alpha-supported-pairwise-merkle-bad-imp-local-collision-or-coupling-bad:*

assumes *bad:* *alpha-supported-pairwise-merkle-bad s*

shows

$(\exists \text{ fr f-fri-roots f-final trace-table final-state trace-table'}$
 $\text{final-state'.$
alpha-header-supported-trace-table-candidate-state s fr f-fri-roots
f-final trace-table final-state $\wedge$
alpha-header-supported-trace-table-candidate-state s fr f-fri-roots
f-final trace-table' final-state' $\wedge$
 $\text{trace-table} \neq \text{trace-table}' \wedge$
 $(\text{hash-map-output-collision final-state} \vee$

$\text{hash-map-output-collision final-state'}) \vee$
 $\text{alpha-supported-pairwise-coupling-bad } s$
proof –
from bad **obtain** $\text{fr f-fri-roots f-final}$ **where** header-bad :
 $\text{alpha-header-supported-pairwise-merkle-bad } s \text{ fr f-fri-roots f-final}$
unfolding $\text{alpha-supported-pairwise-merkle-bad-def}$ **by** blast
from
 $\text{alpha-header-supported-pairwise-merkle-bad-imp-local-collision-or-coupling-bad}$
 $[\text{OF header-bad}]$
show $?thesis$
unfolding $\text{alpha-supported-pairwise-coupling-bad-def}$ **by** blast
qed

lemma $\text{alpha-supported-pairwise-merkle-bad-imp-supported-hash-collision-possible-or-coupling-bad}$:
assumes $\text{bad: alpha-supported-pairwise-merkle-bad } s$
shows
 $\text{supported-hash-output-collision-possible } s \vee$
 $\text{alpha-supported-pairwise-coupling-bad } s$
proof –
have decomp :
 $(\exists \text{fr f-fri-roots f-final trace-table final-state trace-table'}$
 final-state'
 $\text{alpha-header-supported-trace-table-candidate-state } s \text{ fr f-fri-roots}$
 $\text{f-final trace-table final-state} \wedge$
 $\text{alpha-header-supported-trace-table-candidate-state } s \text{ fr f-fri-roots}$
 $\text{f-final trace-table' final-state'} \wedge$
 $\text{trace-table} \neq \text{trace-table'} \wedge$
 $(\text{hash-map-output-collision final-state} \vee$
 $\text{hash-map-output-collision final-state'}) \vee$
 $\text{alpha-supported-pairwise-coupling-bad } s$
by $(\text{rule}$
 $\text{alpha-supported-pairwise-merkle-bad-imp-local-collision-or-coupling-bad}$
 $[\text{OF bad}])$
then show $?thesis$
proof
assume local :
 $\exists \text{fr f-fri-roots f-final trace-table final-state trace-table'}$
 final-state'
 $\text{alpha-header-supported-trace-table-candidate-state } s \text{ fr f-fri-roots}$
 $\text{f-final trace-table final-state} \wedge$
 $\text{alpha-header-supported-trace-table-candidate-state } s \text{ fr f-fri-roots}$
 $\text{f-final trace-table' final-state'} \wedge$
 $\text{trace-table} \neq \text{trace-table'} \wedge$
 $(\text{hash-map-output-collision final-state} \vee$
 $\text{hash-map-output-collision final-state'})$
then obtain $\text{fr f-fri-roots f-final trace-table final-state trace-table'}$
 final-state' **where**
 candidate :
 $\text{alpha-header-supported-trace-table-candidate-state } s \text{ fr f-fri-roots}$

```

    f-final trace-table final-state
  and candidate':
    alpha-header-supported-trace-table-candidate-state s fr f-fri-roots
    f-final trace-table' final-state'
  and collision:
    hash-map-output-collision final-state  $\vee$ 
    hash-map-output-collision final-state'
  by blast
from collision show ?thesis
proof
  assume collision: hash-map-output-collision final-state
  from candidate obtain result composition-table as query-idxs dg
    composition-fri-roots final rest where
  outcome:
    Some (result, final-state)  $\in$  set-dist (execute verify-monad s)
  and bound:
    accepted-with-bound-tables s (Some (result, final-state))
    trace-table composition-table as query-idxs
  unfolding alpha-header-supported-trace-table-candidate-state-def
  by blast
have supported-hash-output-collision-possible s
  by (rule
    accepted-with-bound-tables-hash-collision-imp-supported-hash-output-collision-possible
    [OF outcome bound collision])
then show ?thesis by simp
next
  assume collision': hash-map-output-collision final-state'
  from candidate' obtain result' composition-table' as' query-idxs' dg'
    composition-fri-roots' final' rest' where
  outcome':
    Some (result', final-state')  $\in$  set-dist (execute verify-monad s)
  and bound':
    accepted-with-bound-tables s (Some (result', final-state'))
    trace-table' composition-table' as' query-idxs'
  unfolding alpha-header-supported-trace-table-candidate-state-def
  by blast
have supported-hash-output-collision-possible s
  by (rule
    accepted-with-bound-tables-hash-collision-imp-supported-hash-output-collision-possible
    [OF outcome' bound' collision'])
then show ?thesis by simp
qed
next
  assume alpha-supported-pairwise-coupling-bad s
  then show ?thesis by simp
qed
qed

```

lemma *verifier-composition-fri-prefix-preserves-query-future-fresh:*

```

assumes future: query-future-fresh s
and outcome:
  Some (header, t) ∈
    set-dist (execute verifier-composition-fri-prefix s)
shows query-future-fresh t
proof –
obtain fr f-fl f-final as dg fl where header-eq:
  header = (fr, f-fl, f-final, as, dg, fl)
by (cases header) auto
from outcome[unfolded header-eq] obtain s1 s2 s3 s4 s5 s6 where
  read-fr: Some (fr, s1) ∈ set-dist (execute read s)
and trace-fri:
  Some (f-fl, s2) ∈
    set-dist (execute
      (ntimes receive-trace-fri-commits (ceil-log clength) s1)
    )
and read-f-final:
  Some (f-final, s3) ∈ set-dist (execute read s2)
and alpha-out:
  Some (as, s4) ∈
    set-dist (execute (mmap (replicate (length spec) alpha-round)) s3)
and read-dg: Some (dg, s5) ∈ set-dist (execute read s4)
and degree-assert:
  Some ((), s6) ∈
    set-dist (execute (assert (to-nat dg ≤ maxDegree)) s5)
and comp-fri:
  Some (fl, t) ∈
    set-dist (execute
      (ntimes receive-composition-fri-commits
        (ceil-log (to-nat dg + 1))) s6)
    )
unfolding verifier-composition-fri-prefix-def
by (auto elim! set-dist-bindE)
have future-s1: query-future-fresh s1
by (rule read-preserves-query-future-fresh[OF future read-fr])
have future-s2: query-future-fresh s2
by (rule ntimes-receive-trace-fri-commits-preserves-query-future-fresh
  [OF trace-fri future-s1])
have future-s3: query-future-fresh s3
by (rule read-preserves-query-future-fresh[OF future-s2 read-f-final])
have future-s4: query-future-fresh s4
by (rule mmap-alpha-round-preserves-query-future-fresh
  [OF future-s3 alpha-out])
have future-s5: query-future-fresh s5
by (rule read-preserves-query-future-fresh[OF future-s4 read-dg])
have s6-eq: s6 = s5
using degree-assert unfolding assert-def
by (cases to-nat dg ≤ maxDegree)
  (auto simp: throw-no-outcome)
have future-s6: query-future-fresh s6
using future-s5 unfolding s6-eq .

```

```

show ?thesis
  by (rule ntimes-receive-composition-fri-commits-preserves-query-future-fresh
    [OF comp-fri future-s6])
qed

lemma verify-monad-header-extraction:
  fixes s :: ('f, 'a) protocol-channel-scheme
  assumes outcome:
    Some (result, final-state) ∈ set-dist (execute verify-monad s)
  obtains fr f-fl f-final as dg fl final query-state
  where
    verifier-header-transcript s fr (map snd f-fl) f-final as dg
      (map snd fl) final (PTranscript query-state)
  and PState query-state =
    verifier-header-state s fr (map snd f-fl) f-final as dg (map snd fl) final
  and Some (result, final-state) ∈
    set-dist (execute
      (ntimes (verifier-query-round-program fr f-fl f-final as fl final) rounds)
      query-state)
  and PQueryCounter query-state = PQueryCounter s
  and length f-fl = ceil-log clength
  and length as = length spec
  and length fl = ceil-log (to-nat dg + 1)
proof –
  let ?alpha =
    do {
      a0 ← receive-alpha-challenge;
      let a0' = a0;
      a1 ← read;
      let a1' = a1;
      assert (a0' = a1');
      return a1'
    }
  from outcome obtain fr s1 f-fl s2 f-final s3 as s4 dg s5 s6 fl s7 final query-state
  where read-fr: Some (fr, s1) ∈ set-dist (execute read s)
  and trace-fri:
    Some (f-fl, s2) ∈
    set-dist (execute
      (ntimes receive-trace-fri-commits (ceil-log clength) s1)
    and read-f-final: Some (f-final, s3) ∈ set-dist (execute read s2)
  and alpha-out:
    Some (as, s4) ∈
    set-dist (execute (mmap (replicate (length spec) ?alpha)) s3)
  and read-dg: Some (dg, s5) ∈ set-dist (execute read s4)
  and degree-assert:
    Some ((), s6) ∈ set-dist (execute (assert (to-nat dg ≤ maxDegree)) s5)
  and comp-fri:
    Some (fl, s7) ∈
    set-dist (execute

```



```

      (ntimes receive-composition-fri-commits
        (ceil-log (to-nat dg + 1))) s6)
    and read-final: Some (final, query-state) ∈ set-dist (execute read s7)
    and query-out:
      Some (result, final-state) ∈
        set-dist (execute
          (ntimes (verifier-query-round-program fr f-fl f-final as fl final) rounds)
            query-state)
    unfolding verify-monad-def verifier-query-round-program-def
    by (auto simp: Let-def split: prod.splits elim!: set-dist-bindE)
  from read-outcome[OF read-fr] obtain rest1 where
    tr-s: PTranscript s = fr # rest1
    and st-s1: PState s1 = concat (PState s) fr
    and tr-s1: PTranscript s1 = rest1
    and query-count-s1: PQueryCounter s1 = PQueryCounter s
    by blast
  have trace-fri-res:
    length f-fl = ceil-log clength ∧
    PTranscript s1 = map snd f-fl @ PTranscript s2 ∧
    PState s2 = foldl concat (PState s1) (map snd f-fl)
    using ntimes-receive-trace-fri-commits-outcome[OF trace-fri] by simp
  have query-count-s2: PQueryCounter s2 = PQueryCounter s1
    using ntimes-receive-trace-fri-commits-outcome[OF trace-fri] by simp
  from read-outcome[OF read-f-final] obtain rest3 where
    tr-s2: PTranscript s2 = f-final # rest3
    and st-s3: PState s3 = concat (PState s2) f-final
    and tr-s3: PTranscript s3 = rest3
    and query-count-s3: PQueryCounter s3 = PQueryCounter s2
    by blast
  have alpha-out':
    Some (as, s4) ∈
      set-dist (execute (mmap (replicate (length spec) alpha-round)) s3)
    using alpha-out by (simp add: alpha-round-def)
  have alpha-res:
    length as = length spec ∧
    PTranscript s3 = as @ PTranscript s4 ∧
    PState s4 = foldl concat (PState s3) as
    using mmap-alpha-round-outcome[OF alpha-out'] by simp
  have query-count-s4: PQueryCounter s4 = PQueryCounter s3
    using mmap-alpha-round-outcome[OF alpha-out'] by simp
  from read-outcome[OF read-dg] obtain rest5 where
    tr-s4: PTranscript s4 = dg # rest5
    and st-s5: PState s5 = concat (PState s4) dg
    and tr-s5: PTranscript s5 = rest5
    and query-count-s5: PQueryCounter s5 = PQueryCounter s4
    by blast
  have s6-eq: s6 = s5
    using degree-assert unfolding assert-def
    by (cases to-nat dg ≤ maxDegree) (auto simp: throw-no-outcome)

```

```

have comp-fri-res:
  length fl = ceil-log (to-nat dg + 1) ∧
    PTranscript s6 = map snd fl @ PTranscript s7 ∧
    PState s7 = foldl concat (PState s6) (map snd fl)
  using ntimes-receive-composition-fri-commits-outcome[OF comp-fri]
  by simp
have query-count-s7: PQueryCounter s7 = PQueryCounter s6
  using ntimes-receive-composition-fri-commits-outcome[OF comp-fri]
  by simp
from read-outcome[OF read-final] obtain rest-query where
  tr-s7: PTranscript s7 = final # rest-query
  and st-query: PState query-state = concat (PState s7) final
  and tr-query: PTranscript query-state = rest-query
  and query-count-query: PQueryCounter query-state = PQueryCounter s7
  by blast
have header-tr:
  verifier-header-transcript s fr (map snd f-fl) f-final as dg
    (map snd fl) final (PTranscript query-state)
  using tr-s tr-s1 trace-fri-res tr-s2 tr-s3 alpha-res tr-s4 tr-s5
    s6-eq comp-fri-res tr-s7 tr-query
  unfolding verifier-header-transcript-def verifier-header-messages-def
  by simp
have header-state:
  PState query-state =
    verifier-header-state s fr (map snd f-fl) f-final as dg (map snd fl) final
  using st-s1 trace-fri-res st-s3 alpha-res st-s5 s6-eq comp-fri-res st-query
  unfolding verifier-header-state-def verifier-header-messages-def
  by simp
have query-count-header:
  PQueryCounter query-state = PQueryCounter s
  using query-count-s1 query-count-s2 query-count-s3 query-count-s4
    query-count-s5 s6-eq query-count-s7 query-count-query
  by simp
show ?thesis
  by (rule that[OF header-tr header-state query-out query-count-header])
    (use trace-fri-res alpha-res comp-fri-res in simp-all)
qed

```

```

lemma verify-monad-header-query-extraction:
  fixes s :: ('f, 'a) protocol-channel-scheme
  assumes outcome:
    Some (result, final-state) ∈ set-dist (execute verify-monad s)
  obtains fr f-fl f-final as dg fl final query-state
  where
    verifier-header-transcript s fr (map snd f-fl) f-final as dg
      (map snd fl) final (PTranscript query-state)
  and Some (result, final-state) ∈
    set-dist (execute
      (ntimes

```

```

      (verifier-query-round-program fr f-fl f-final as fl final) rounds)
    query-state)
proof (rule verify-monad-header-extraction[OF outcome])
fix fr f-fl f-final as dg fl final query-state
assume header:
  verifier-header-transcript s fr (map snd f-fl) f-final as dg
    (map snd fl) final (PTranscript query-state)
and query-out:
  Some (result, final-state) ∈
    set-dist
      (execute
        (ntimes
          (verifier-query-round-program fr f-fl f-final as fl final)
            rounds)
          query-state)
show ?thesis
by (rule that[OF header query-out])
qed

lemma verify-monad-header-random-oracle-replay:
fixes s :: ('f, 'a) protocol-channel-scheme
assumes outcome:
  Some (result, final-state) ∈ set-dist (execute verify-monad s)
obtains fr f-fl f-final as dg fl final query-state
where
  verifier-header-transcript s fr (map snd f-fl) f-final as dg
    (map snd fl) final (PTranscript query-state)
and PState query-state =
  verifier-header-state s fr (map snd f-fl) f-final as dg (map snd fl) final
and Some (result, final-state) ∈
  set-dist (execute
    (ntimes (verifier-query-round-program fr f-fl f-final as fl final) rounds)
      query-state)
and PQueryCounter query-state = PQueryCounter s
and s ≤ query-state
and  $\bigwedge i. i < \text{length } f\text{-fl} \implies$ 
  fmlookup (HashMap query-state)
    (TraceFriChallenge (PTraceFriCounter s + i)
      (foldl concat (concat (PState s) fr) (take (Suc i) (map snd f-fl)))) =
    Some (fst (f-fl ! i))
and  $\bigwedge i. i < \text{length } as \implies$ 
  fmlookup (HashMap query-state)
    (AlphaChallenge (PAlphaCounter s + i) (foldl concat
      (concat (foldl concat (concat (PState s) fr) (map snd f-fl)) f-final)
        (take i as))) =
    Some (as ! i)
and  $\bigwedge i. i < \text{length } fl \implies$ 
  fmlookup (HashMap query-state)
    (CompositionFriChallenge (PCompositionFriCounter s + i) (foldl concat

```

```

    (concat
      (foldl concat
        (concat (foldl concat (concat (PState s) fr) (map snd f-fl)) f-final)
        as)
      dg)
    (take (Suc i) (map snd fl)))) =
  Some (fst (fl ! i))
and to-nat dg ≤ maxDegree
proof –
let ?alpha =
  do {
    a0 ← receive-alpha-challenge;
    let a0' = a0;
    a1 ← read;
    let a1' = a1;
    assert (a0' = a1');
    return a1'
  }
from outcome obtain fr s1 f-fl s2 f-final s3 as s4 dg s5 s6 fl s7 final query-state
where read-fr: Some (fr, s1) ∈ set-dist (execute read s)
and trace-fri:
  Some (f-fl, s2) ∈
    set-dist (execute
      (ntimes receive-trace-fri-commits (ceil-log clength)) s1)
and read-f-final: Some (f-final, s3) ∈ set-dist (execute read s2)
and alpha-out:
  Some (as, s4) ∈
    set-dist (execute (mmap (replicate (length spec) ?alpha)) s3)
and read-dg: Some (dg, s5) ∈ set-dist (execute read s4)
and degree-assert:
  Some ((), s6) ∈ set-dist (execute (assert (to-nat dg ≤ maxDegree)) s5)
and comp-fri:
  Some (fl, s7) ∈
    set-dist (execute
      (ntimes receive-composition-fri-commits
        (ceil-log (to-nat dg + 1))) s6)
and read-final: Some (final, query-state) ∈ set-dist (execute read s7)
and query-out:
  Some (result, final-state) ∈
    set-dist (execute
      (ntimes (verifier-query-round-program fr f-fl f-final as fl final) rounds)
      query-state)
unfolding verify-monad-def verifier-query-round-program-def
by (auto simp: Let-def split: prod.splits elim!: set-dist-bindE)
from read-outcome[OF read-fr] obtain rest1 where
  tr-s: PTranscript s = fr # rest1
and st-s1: PState s1 = concat (PState s) fr
and tr-s1: PTranscript s1 = rest1
and ext-s-s1: s ≤ s1

```

```

and trace-count-s1: PTraceFriCounter s1 = PTraceFriCounter s
and comp-count-s1: PCompositionFriCounter s1 = PCompositionFriCounter s
and alpha-count-s1: PAlphaCounter s1 = PAlphaCounter s
and query-count-s1: PQueryCounter s1 = PQueryCounter s
by blast
have trace-fri-res:
  length f-fl = ceil-log clength  $\wedge$ 
  PTranscript s1 = map snd f-fl @ PTranscript s2  $\wedge$ 
  PState s2 = foldl concat (PState s1) (map snd f-fl)  $\wedge$ 
  s1  $\leq$  s2  $\wedge$ 
  PTraceFriCounter s2 = PTraceFriCounter s1 + ceil-log clength  $\wedge$ 
  PCompositionFriCounter s2 = PCompositionFriCounter s1  $\wedge$ 
  PAlphaCounter s2 = PAlphaCounter s1  $\wedge$ 
  PQueryCounter s2 = PQueryCounter s1  $\wedge$ 
  ( $\forall i < \text{ceil-log clength}.$ 
    fmllookup (HashMap s2)
      (TraceFriChallenge (PTraceFriCounter s1 + i)
        (foldl concat (PState s1) (take (Suc i) (map snd f-fl)))) =
      Some (fst (f-fl ! i)))
  using ntimes-receive-trace-fri-commits-outcome[OF trace-fri] by simp
from read-outcome[OF read-f-final] obtain rest3 where
  tr-s2: PTranscript s2 = f-final # rest3
and st-s3: PState s3 = concat (PState s2) f-final
and tr-s3: PTranscript s3 = rest3
and ext-s2-s3: s2  $\leq$  s3
and trace-count-s3: PTraceFriCounter s3 = PTraceFriCounter s2
and comp-count-s3: PCompositionFriCounter s3 = PCompositionFriCounter s2
and alpha-count-s3: PAlphaCounter s3 = PAlphaCounter s2
and query-count-s3: PQueryCounter s3 = PQueryCounter s2
by blast
have alpha-out':
  Some (as, s4)  $\in$ 
    set-dist (execute (mmap (replicate (length spec) alpha-round)) s3)
  using alpha-out by (simp add: alpha-round-def)
have alpha-res:
  length as = length spec  $\wedge$ 
  PTranscript s3 = as @ PTranscript s4  $\wedge$ 
  PState s4 = foldl concat (PState s3) as  $\wedge$ 
  s3  $\leq$  s4  $\wedge$ 
  PTraceFriCounter s4 = PTraceFriCounter s3  $\wedge$ 
  PCompositionFriCounter s4 = PCompositionFriCounter s3  $\wedge$ 
  PAlphaCounter s4 = PAlphaCounter s3 + length spec  $\wedge$ 
  PQueryCounter s4 = PQueryCounter s3  $\wedge$ 
  ( $\forall i < \text{length spec}.$ 
    fmllookup (HashMap s4)
      (AlphaChallenge (PAlphaCounter s3 + i)
        (foldl concat (PState s3) (take i as))) =
      Some (as ! i))
  using mmap-alpha-round-outcome[OF alpha-out'] by simp

```

```

from read-outcome[OF read-dg] obtain rest5 where
  tr-s4: PTranscript s4 = dg # rest5
  and st-s5: PState s5 = concat (PState s4) dg
  and tr-s5: PTranscript s5 = rest5
  and ext-s4-s5: s4 ≤ s5
  and trace-count-s5: PTraceFriCounter s5 = PTraceFriCounter s4
  and comp-count-s5: PCompositionFriCounter s5 = PCompositionFriCounter s4
  and alpha-count-s5: PAlphaCounter s5 = PAlphaCounter s4
  and query-count-s5: PQueryCounter s5 = PQueryCounter s4
  by blast
have s6-eq: s6 = s5
  using degree-assert unfolding assert-def
  by (cases to-nat dg ≤ maxDegree) (auto simp: throw-no-outcome)
have degree-bound: to-nat dg ≤ maxDegree
  using degree-assert unfolding assert-def
  by (cases to-nat dg ≤ maxDegree) (auto simp: throw-no-outcome)
have ext-s5-s6: s5 ≤ s6
  unfolding s6-eq by (rule hash-ext-refl)
have comp-fri-res:
  length fl = ceil-log (to-nat dg + 1) ∧
  PTranscript s6 = map snd fl @ PTranscript s7 ∧
  PState s7 = foldl concat (PState s6) (map snd fl) ∧
  s6 ≤ s7 ∧
  PTraceFriCounter s7 = PTraceFriCounter s6 ∧
  PCompositionFriCounter s7 = PCompositionFriCounter s6 + ceil-log (to-nat dg + 1) ∧
  PAlphaCounter s7 = PAlphaCounter s6 ∧
  PQueryCounter s7 = PQueryCounter s6 ∧
  (∀ i < ceil-log (to-nat dg + 1).
    fmllookup (HashMap s7)
      (CompositionFriChallenge (PCompositionFriCounter s6 + i)
        (foldl concat (PState s6) (take (Suc i) (map snd fl)))) =
        Some (fst (fl ! i)))
  using ntimes-receive-composition-fri-commits-outcome[OF comp-fri]
  by simp
from read-outcome[OF read-final] obtain rest-query where
  tr-s7: PTranscript s7 = final # rest-query
  and st-query: PState query-state = concat (PState s7) final
  and tr-query: PTranscript query-state = rest-query
  and ext-s7-query: s7 ≤ query-state
  and query-count-query: PQueryCounter query-state = PQueryCounter s7
  by blast
have ext-s2-query: s2 ≤ query-state
  using ext-s2-s3 alpha-res ext-s4-s5 ext-s5-s6 comp-fri-res ext-s7-query
  by (meson hash-ext-trans)
have ext-s4-query: s4 ≤ query-state
  using ext-s4-s5 ext-s5-s6 comp-fri-res ext-s7-query
  by (meson hash-ext-trans)
have ext-s7-query': s7 ≤ query-state
  using ext-s7-query .

```

```

have ext-s1-s2:  $s1 \leq s2$ 
  using trace-fri-res by simp
have ext-s3-s4:  $s3 \leq s4$ 
  using alpha-res by simp
have ext-s6-s7:  $s6 \leq s7$ 
  using comp-fri-res by simp
have ext-s-s2:  $s \leq s2$ 
  by (rule hash-ext-trans[OF ext-s-s1 ext-s1-s2])
have ext-s-s3:  $s \leq s3$ 
  by (rule hash-ext-trans[OF ext-s-s2 ext-s2-s3])
have ext-s-s4:  $s \leq s4$ 
  by (rule hash-ext-trans[OF ext-s-s3 ext-s3-s4])
have ext-s-s5:  $s \leq s5$ 
  by (rule hash-ext-trans[OF ext-s-s4 ext-s4-s5])
have ext-s-s6:  $s \leq s6$ 
  by (rule hash-ext-trans[OF ext-s-s5 ext-s5-s6])
have ext-s-s7:  $s \leq s7$ 
  by (rule hash-ext-trans[OF ext-s-s6 ext-s6-s7])
have ext-s-query:  $s \leq \text{query-state}$ 
  by (rule hash-ext-trans[OF ext-s-s7 ext-s7-query])
have header-tr:
  verifier-header-transcript s fr (map snd f-fl) f-final as dg
    (map snd fl) final (PTranscript query-state)
  using tr-s tr-s1 trace-fri-res tr-s2 tr-s3 alpha-res tr-s4 tr-s5
    s6-eq comp-fri-res tr-s7 tr-query
  unfolding verifier-header-transcript-def verifier-header-messages-def
  by simp
have header-state:
  PState query-state =
    verifier-header-state s fr (map snd f-fl) f-final as dg (map snd fl) final
  using st-s1 trace-fri-res st-s3 alpha-res st-s5 s6-eq comp-fri-res st-query
  unfolding verifier-header-state-def verifier-header-messages-def
  by simp
have trace-lookup:
   $\bigwedge i. i < \text{length } f\text{-fl} \implies$ 
    fmlookup (HashMap query-state)
      (TraceFriChallenge (PTraceFriCounter s + i)
        (foldl concat (concat (PState s) fr) (take (Suc i) (map snd f-fl)))) =
      Some (fst (f-fl ! i))
proof -
  fix i
  assume i-bound:  $i < \text{length } f\text{-fl}$ 
  have lookup-s2:
    fmlookup (HashMap s2)
      (TraceFriChallenge (PTraceFriCounter s1 + i)
        (foldl concat (PState s1) (take (Suc i) (map snd f-fl)))) =
      Some (fst (f-fl ! i))
  using trace-fri-res i-bound by simp
  show fmlookup (HashMap query-state)

```

```

    (TraceFriChallenge (PTraceFriCounter s + i)
      (foldl concat (concat (PState s) fr) (take (Suc i) (map snd f-fl)))) =
    Some (fst (f-fl ! i))
using hash-extension-lookup[OF lookup-s2 ext-s2-query] st-s1
    trace-count-s1
by simp
qed
have alpha-lookup:
   $\bigwedge i. i < \text{length } as \implies$ 
    fmllookup (HashMap query-state)
      (AlphaChallenge (PAlphaCounter s + i) (foldl concat
        (concat (foldl concat (concat (PState s) fr) (map snd f-fl)) f-final)
        (take i as))) =
    Some (as ! i)
proof -
  fix i
  assume i-bound:  $i < \text{length } as$ 
  have lookup-s4:
    fmllookup (HashMap s4)
      (AlphaChallenge (PAlphaCounter s3 + i)
        (foldl concat (PState s3) (take i as))) =
    Some (as ! i)
  using alpha-res i-bound by simp
  show fmllookup (HashMap query-state)
    (AlphaChallenge (PAlphaCounter s + i) (foldl concat
      (concat (foldl concat (concat (PState s) fr) (map snd f-fl)) f-final)
      (take i as))) =
    Some (as ! i)
  using hash-extension-lookup[OF lookup-s4 ext-s4-query]
    st-s1 trace-fri-res st-s3 alpha-count-s1 alpha-count-s3
  by simp
qed
have comp-lookup:
   $\bigwedge i. i < \text{length } fl \implies$ 
    fmllookup (HashMap query-state)
      (CompositionFriChallenge (PCompositionFriCounter s + i) (foldl concat
        (concat
          (foldl concat
            (concat (foldl concat (concat (PState s) fr) (map snd f-fl)) f-final)
            as)
          dg)
        (take (Suc i) (map snd fl)))) =
    Some (fst (fl ! i))
proof -
  fix i
  assume i-bound:  $i < \text{length } fl$ 
  have lookup-s7:
    fmllookup (HashMap s7)
      (CompositionFriChallenge (PCompositionFriCounter s6 + i)

```



```

    (foldl concat (PState s6) (take (Suc i) (map snd fl)))) =
    Some (fst (fl ! i))
  using comp-fri-res i-bound by simp
show fmlookup (HashMap query-state)
  (CompositionFriChallenge (PCompositionFriCounter s + i) (foldl concat
    (concat
      (foldl concat
        (concat (foldl concat (concat (PState s) fr) (map snd f-fl)) f-final)
        as)
      dg)
    (take (Suc i) (map snd fl)))) =
    Some (fst (fl ! i))
  using hash-extension-lookup[OF lookup-s7 ext-s7-query]
    st-s1 trace-fri-res st-s3 alpha-res st-s5 s6-eq
    comp-count-s1 comp-count-s3 comp-count-s5
  by simp
qed
have query-count-header:
  PQueryCounter query-state = PQueryCounter s
  using query-count-s1 trace-fri-res query-count-s3 alpha-res query-count-s5
    s6-eq comp-fri-res query-count-query
  by simp
show ?thesis
  by (rule that[OF header-tr header-state query-out query-count-header
    ext-s-query trace-lookup alpha-lookup comp-lookup degree-bound])
qed

```

lemma *verify-monad-random-oracle-key-extraction:*

fixes $s :: ('f, 'a) \text{protocol-channel-scheme}$

assumes *outcome:*

$\text{Some}(\text{result}, \text{final-state}) \in \text{set-dist}(\text{execute verify-monad } s)$

obtains $\text{fr} :: 'f$

and $\text{f-fl} :: ('f \times 'f) \text{list}$

and $\text{f-final} :: 'f$

and $\text{as} :: 'f \text{list}$

and $\text{dg} :: 'f$

and $\text{fl} :: ('f \times 'f) \text{list}$

and $\text{final} :: 'f$

and $\text{query-state} :: ('f, 'a) \text{protocol-channel-scheme}$

and $\text{raw-idxs} :: 'f \text{list}$

and $\text{query-idxs} :: \text{nat list}$

and $\text{query-chunks} :: 'f \text{list list}$

where

$\text{verifier-header-transcript } s \text{ fr } (\text{map snd f-fl}) \text{ f-final as dg}$

$(\text{map snd fl}) \text{ final } (P\text{Transcript query-state})$

and $P\text{State query-state} =$

$\text{verifier-header-state } s \text{ fr } (\text{map snd f-fl}) \text{ f-final as dg } (\text{map snd fl}) \text{ final}$

and $\text{length raw-idxs} = \text{rounds}$

and $\text{query-idxs} = \text{map } (\lambda \text{raw. index (to-nat raw)}) \text{ raw-idxs}$

```

and length query-chunks = rounds
and PTranscript query-state =
  List.concat query-chunks @ PTranscript final-state
and  $\bigwedge i. i < \text{length } f\text{-fl} \implies$ 
  fmlookup (HashMap query-state)
    (TraceFriChallenge
      (PTraceFriCounter s + i)
      (foldl concat (concat (PState s) fr)
        (take (Suc i) (map snd f-fl)))) =
    Some (fst (f-fl ! i))
and  $\bigwedge i. i < \text{length } as \implies$ 
  fmlookup (HashMap query-state)
    (AlphaChallenge (PAlphaCounter s + i) (foldl concat
      (concat (foldl concat (concat (PState s) fr) (map snd f-fl)) f-final)
      (take i as))) =
    Some (as ! i)
and  $\bigwedge i. i < \text{length } fl \implies$ 
  fmlookup (HashMap query-state)
    (CompositionFriChallenge (PCompositionFriCounter s + i) (foldl concat
      (concat
        (foldl concat
          (concat (foldl concat (concat (PState s) fr) (map snd f-fl)) f-final)
          as)
        dg)
      (take (Suc i) (map snd fl)))) =
    Some (fst (fl ! i))
and  $\bigwedge i. i < \text{rounds} \implies$ 
  verifier-query-round-chunk (query-idxs ! i)
    (map snd f-fl) (map snd fl) (query-chunks ! i)
and  $\bigwedge i. i < \text{rounds} \implies$ 
  fmlookup (HashMap final-state)
    (QueryIndexChallenge
      (PQueryCounter query-state + i) (state-after-query-chunks (PState query-state) query-chunks
i)) =
    Some (raw-idxs ! i)
and  $\bigwedge idx. idx \in \text{set query-idxs} \implies idx < \text{clength} * \text{scale}$ 
proof –
from verify-monad-header-random-oracle-replay[OF outcome]
obtain fr f-fl f-final as dg fl final query-state where
  header:
    verifier-header-transcript s fr (map snd f-fl) f-final as dg
    (map snd fl) final (PTranscript query-state)
and header-state:
  PState query-state =
    verifier-header-state s fr (map snd f-fl) f-final as dg
    (map snd fl) final
and query-out:
  Some (result, final-state) ∈
    set-dist (execute

```

```

    (ntimes (verifier-query-round-program fr f-fl f-final as fl final) rounds)
    query-state)
and query-count-header:
    PQueryCounter query-state = PQueryCounter s
and ext-s-query-header:  $s \leq$  query-state
and trace-lookup:
     $\bigwedge i. i < \text{length } f\text{-fl} \implies$ 
    fmllookup (HashMap query-state)
      (TraceFriChallenge
        (PTraceFriCounter s + i) (foldl concat (concat (PState s) fr)
          (take (Suc i) (map snd f-fl)))) =
    Some (fst (f-fl ! i))
and alpha-lookup:
     $\bigwedge i. i < \text{length } as \implies$ 
    fmllookup (HashMap query-state)
      (AlphaChallenge (PAlphaCounter s + i) (foldl concat
        (concat (foldl concat (concat (PState s) fr) (map snd f-fl)) f-final)
          (take i as))) =
    Some (as ! i)
and comp-lookup:
     $\bigwedge i. i < \text{length } fl \implies$ 
    fmllookup (HashMap query-state)
      (CompositionFriChallenge (PCompositionFriCounter s + i) (foldl concat
        (concat
          (foldl concat
            (concat
              (foldl concat (concat (PState s) fr) (map snd f-fl))
                f-final)
            as)
          dg)
          (take (Suc i) (map snd fl)))) =
    Some (fst (fl ! i))
and degree-bound:  $\text{to-nat } dg \leq \text{maxDegree}$ 
by blast
from ntimes-verifier-query-rounds-outcome[OF query-out]
obtain raw-idxs query-idxs query-chunks where
  len-raw:  $\text{length } \text{raw-idxs} = \text{rounds}$ 
and query-idxs-def:  $\text{query-idxs} = \text{map } (\lambda \text{raw}. \text{index } (\text{to-nat } \text{raw})) \text{ raw-idxs}$ 
and len-chunks:  $\text{length } \text{query-chunks} = \text{rounds}$ 
and tr-query:
    PTranscript query-state =
    List.concat query-chunks @ PTranscript final-state
and round-chunks:
     $\forall i < \text{rounds}.$ 
    verifier-query-round-chunk (query-idxs ! i)
      (map snd f-fl) (map snd fl) (query-chunks ! i)
and query-lookup:
     $\forall i < \text{rounds}.$ 
    fmllookup (HashMap final-state)

```

```

      (QueryIndexChallenge
        (PQueryCounter query-state + i) (state-after-query-chunks (PState query-state)
query-chunks i)) =
      Some (raw-idxs ! i)
    and idx-bounds:  $\forall idx \in \text{set query-idxs}. idx < \text{clength} * \text{scale}$ 
    by blast
  show ?thesis
proof (rule that[of fr f-fl f-final as dg fl final query-state
  raw-idxs query-idxs query-chunks])
  show verifier-header-transcript s fr (map snd f-fl) f-final as dg
    (map snd fl) final (PTranscript query-state)
  by (rule header)
next
  show PState query-state =
    verifier-header-state s fr (map snd f-fl) f-final as dg
    (map snd fl) final
  by (rule header-state)
next
  show length raw-idxs = rounds
  by (rule len-raw)
next
  show query-idxs = map ( $\lambda \text{raw}. \text{index} (\text{to-nat raw})$ ) raw-idxs
  by (rule query-idxs-def)
next
  show length query-chunks = rounds
  by (rule len-chunks)
next
  show PTranscript query-state =
    List.concat query-chunks @ PTranscript final-state
  by (rule tr-query)
next
  fix i
  assume i < length f-fl
  then show fmlookup (HashMap query-state)
    (TraceFriChallenge
      (PTraceFriCounter s + i)
      (foldl concat (concat (PState s) fr)
        (take (Suc i) (map snd f-fl)))) =
    Some (fst (f-fl ! i))
  by (rule trace-lookup)
next
  fix i
  assume i < length as
  then show fmlookup (HashMap query-state)
    (AlphaChallenge (PAlphaCounter s + i) (foldl concat
      (concat (foldl concat (concat (PState s) fr) (map snd f-fl)) f-final)
      (take i as))) =
    Some (as ! i)
  by (rule alpha-lookup)

```

```

next
  fix i
  assume i < length fl
  then show fmlookup (HashMap query-state)
    (CompositionFriChallenge (PCompositionFriCounter s + i) (foldl concat
      (concat
        (foldl concat
          (concat (foldl concat (concat (PState s) fr) (map snd f-fl)) f-final)
          as)
        dg)
        (take (Suc i) (map snd fl)))) =
      Some (fst (fl ! i))
    by (rule comp-lookup)
next
  fix i
  assume i < rounds
  then show verifier-query-round-chunk (query-idxs ! i)
    (map snd f-fl) (map snd fl) (query-chunks ! i)
    using round-chunks by blast
next
  fix i
  assume i < rounds
  then show fmlookup (HashMap final-state)
    (QueryIndexChallenge
      (PQueryCounter query-state + i) (state-after-query-chunks (PState query-state) query-chunks
i)) =
      Some (raw-idxs ! i)
    using query-lookup by blast
next
  fix idx
  assume idx ∈ set query-idxs
  then show idx < clength * scale
    using idx-bounds by blast
qed
qed

lemma verify-monad-accepted-transcript-shape:
  fixes s :: ('f, 'a) protocol-channel-scheme
  assumes outcome:
    Some (result, final-state) ∈ set-dist (execute verify-monad s)
  shows ∃ alphas query-idxs.
    accepted-transcript-shape s (Some (result, final-state)) alphas query-idxs
proof -
  from verify-monad-header-extraction[OF outcome]
  obtain fr f-fl f-final as dg fl final query-state where
    header:
      verifier-header-transcript s fr (map snd f-fl) f-final as dg
      (map snd fl) final (PTranscript query-state)
  and header-state:

```

```

PState query-state =
  verifier-header-state s fr (map snd f-fl) f-final as dg (map snd fl) final
and query-out:
  Some (result, final-state) ∈
    set-dist (execute
      (ntimes (verifier-query-round-program fr f-fl f-final as fl final) rounds)
      query-state)
and query-count-header:
  PQueryCounter query-state = PQueryCounter s
and len-f-fl: length f-fl = ceil-log clength
and len-as: length as = length spec
and len-fl: length fl = ceil-log (to-nat dg + 1)
by blast
from ntimes-verifier-query-rounds-outcome[OF query-out]
obtain raw-idxs query-idxs query-chunks where
  len-raw: length raw-idxs = rounds
and query-idxs-def: query-idxs = map (λraw. index (to-nat raw)) raw-idxs
and len-chunks: length query-chunks = rounds
and tr-query:
  PTranscript query-state = List.concat query-chunks @ PTranscript final-state
and round-chunks:
  ∀ i < rounds.
    verifier-query-round-chunk (query-idxs ! i)
      (map snd f-fl) (map snd fl) (query-chunks ! i)
and lookups:
  ∀ i < rounds.
    fmlookup (HashMap final-state)
      (QueryIndexChallenge (PQueryCounter query-state + i) (state-after-query-chunks (PState
query-state) query-chunks i)) =
      Some (raw-idxs ! i)
and idx-bounds: ∀ idx ∈ set query-idxs. idx < clength * scale
by auto
have derived:
  verifier-query-indices-derived s (Some (result, final-state))
    (verifier-header-state s fr (map snd f-fl) f-final as dg (map snd fl) final)
    (PTranscript query-state) (map snd f-fl) (map snd fl) query-idxs
proof –
have tr-query':
  List.concat query-chunks @ PTranscript final-state = PTranscript query-state
using tr-query by simp
have lookups':
  ∀ i < rounds.
    fmlookup (HashMap final-state)
      (QueryIndexChallenge (PQueryCounter s + i) (state-after-query-chunks
      (verifier-header-state s fr (map snd f-fl) f-final as dg (map snd fl) final)
      query-chunks i)) =
      Some (raw-idxs ! i)
using lookups header-state query-count-header by simp
show ?thesis

```

```

unfolding verifier-query-indices-derived-def
apply (rule exI[where  $x=result$ ])
apply (rule exI[where  $x=final-state$ ])
apply (rule exI[where  $x=raw-idxs$ ])
apply (rule exI[where  $x=query-chunks$ ])
apply (rule exI[where  $x=PTranscript\ final-state$ ])
apply (intro conjI)
  apply simp
  apply (rule len-raw)
  apply (rule query-idxs-def)
  apply (rule len-chunks)
  apply (rule tr-query')
  apply (rule round-chunks)
  apply (rule lookups')
apply (rule idx-bounds)
done
qed
have shape:
  accepted-transcript-shape s (Some (result, final-state)) as query-idxs
unfolding accepted-transcript-shape-def
apply (rule exI[where  $x=result$ ])
apply (rule exI[where  $x=final-state$ ])
apply (rule exI[where  $x=fr$ ])
apply (rule exI[where  $x=map\ snd\ f.fl$ ])
apply (rule exI[where  $x=f.final$ ])
apply (rule exI[where  $x=dg$ ])
apply (rule exI[where  $x=map\ snd\ fl$ ])
apply (rule exI[where  $x=final$ ])
apply (rule exI[where  $x=PTranscript\ query-state$ ])
apply (intro conjI)
  apply simp
  apply (rule header)
apply (rule derived)
done
show ?thesis
  using shape by blast
qed

lemma verify-monad-supplied-header-transcript-shape:
fixes  $s :: ('f, 'a)\ protocol-channel-scheme$ 
assumes outcome:
   $Some\ (result, final-state) \in set-dist\ (execute\ verify-monad\ s)$ 
and header0:
  verifier-header-transcript s fr f-fri-roots f-final as dg
  composition-fri-roots final rest
obtains query-idxs raw-idxs query-chunks where
  accepted-transcript-shape s (Some (result, final-state)) as query-idxs
   $length\ raw-idxs = rounds$ 
   $query-idxs = map\ (\lambda raw.\ index\ (to-nat\ raw))\ raw-idxs$ 

```

```

length query-chunks = rounds
List.concat query-chunks @ PTranscript final-state = rest
 $\wedge i. i < \text{rounds} \implies$ 
  verifier-query-round-chunk (query-idxs ! i)
  f-fri-roots composition-fri-roots (query-chunks ! i)
 $\wedge i. i < \text{rounds} \implies$ 
  fmlookup (HashMap final-state)
  (QueryIndexChallenge (PQueryCounter s + i)
   (state-after-query-chunks
    (verifier-header-state s fr f-fri-roots f-final as dg
     composition-fri-roots final)
    query-chunks i)) =
  Some (raw-idxs ! i)
proof –
from verify-monad-header-extraction[OF outcome]
obtain fr' f-fl f-final' as' dg' fl final' query-state where
  header:
    verifier-header-transcript s fr' (map snd f-fl) f-final' as' dg'
    (map snd fl) final' (PTranscript query-state)
and header-state:
  PState query-state =
    verifier-header-state s fr' (map snd f-fl) f-final' as' dg'
    (map snd fl) final'
and query-out:
  Some (result, final-state)  $\in$ 
    set-dist
    (execute
     (ntimes
      (verifier-query-round-program fr' f-fl f-final' as' fl
       final') rounds)
     query-state)
and query-count-header:
  PQueryCounter query-state = PQueryCounter s
by metis
have eqs:
  fr' = fr  $\wedge$ 
  map snd f-fl = f-fri-roots  $\wedge$ 
  f-final' = f-final  $\wedge$ 
  as' = as  $\wedge$ 
  dg' = dg  $\wedge$ 
  map snd fl = composition-fri-roots  $\wedge$ 
  final' = final  $\wedge$ 
  PTranscript query-state = rest
using verifier-header-transcript-unique[OF header header0] by simp
from ntimes-verifier-query-rounds-outcome[OF query-out]
obtain raw-idxs query-idxs query-chunks where
  len-raw: length raw-idxs = rounds
and query-idxs-eq:
  query-idxs = map ( $\lambda$ raw. index (to-nat raw)) raw-idxs

```



```

and len-chunks: length query-chunks = rounds
and tr-query:
  PTranscript query-state =
    List.concat query-chunks @ PTranscript final-state
and chunks:
   $\forall i < \text{rounds}.$ 
    verifier-query-round-chunk (query-idxs ! i)
      (map snd f-fl) (map snd fl) (query-chunks ! i)
and lookups:
   $\forall i < \text{rounds}.$ 
    fmlookup (HashMap final-state)
      (QueryIndexChallenge (PQueryCounter query-state + i)
        (state-after-query-chunks (PState query-state) query-chunks i)) =
      Some (raw-idxs ! i)
and idx-bounds:  $\forall \text{idx} \in \text{set query-idxs}. \text{idx} < \text{clength} * \text{scale}$ 
by blast
have tr-query':
  List.concat query-chunks @ PTranscript final-state = rest
using tr-query eqs by simp
have chunks':
   $\bigwedge i. i < \text{rounds} \implies$ 
    verifier-query-round-chunk (query-idxs ! i)
      f-fri-roots composition-fri-roots (query-chunks ! i)
using chunks eqs by simp
have lookups':
   $\bigwedge i. i < \text{rounds} \implies$ 
    fmlookup (HashMap final-state)
      (QueryIndexChallenge (PQueryCounter s + i)
        (state-after-query-chunks
          (verifier-header-state s fr f-fri-roots f-final as dg
            composition-fri-roots final)
          query-chunks i)) =
      Some (raw-idxs ! i)
using lookups header-state query-count-header eqs by simp
have derived:
  verifier-query-indices-derived s (Some (result, final-state))
    (verifier-header-state s fr f-fri-roots f-final as dg
      composition-fri-roots final)
    rest f-fri-roots composition-fri-roots query-idxs
unfolding verifier-query-indices-derived-def
by (intro exI[of - result] exI[of - final-state]
  exI[of - raw-idxs] exI[of - query-chunks]
  exI[of - PTranscript final-state] conjI)
  (use len-raw query-idxs-eq len-chunks tr-query' chunks' lookups'
    idx-bounds in auto)
have shape:
  accepted-transcript-shape s (Some (result, final-state)) as query-idxs
unfolding accepted-transcript-shape-def
by (intro exI[of - result] exI[of - final-state] exI[of - fr]

```

```

    exI[of - f-fri-roots] exI[of - f-final] exI[of - dg]
    exI[of - composition-fri-roots] exI[of - final]
    exI[of - rest] conjI)
  (use header0 derived in simp-all)
show ?thesis
  by (rule that[OF shape len-raw query-idxs-eq len-chunks tr-query'
    chunks' lookups'])
qed

```

```

lemma verify-monad-supplied-header-query-rounds:
fixes s :: ('f, 'a) protocol-channel-scheme
assumes outcome:
  Some (result, final-state) ∈ set-dist (execute verify-monad s)
and header0:
  verifier-header-transcript s fr f-fri-roots f-final as dg
  composition-fri-roots final rest
obtains f-fl fl query-state where
  map snd f-fl = f-fri-roots
  map snd fl = composition-fri-roots
  PTranscript query-state = rest
  PState query-state =
    verifier-header-state s fr f-fri-roots f-final as dg
    composition-fri-roots final
  PQueryCounter query-state = PQueryCounter s
  Some (result, final-state) ∈
    set-dist
    (execute
      (ntimes
        (verifier-query-round-program fr f-fl f-final as fl final)
        rounds)
      query-state)

```

```

proof –
from verify-monad-header-extraction[OF outcome]
obtain fr' f-fl f-final' as' dg' fl final' query-state where
  header:
    verifier-header-transcript s fr' (map snd f-fl) f-final' as' dg'
    (map snd fl) final' (PTranscript query-state)
and header-state:
  PState query-state =
    verifier-header-state s fr' (map snd f-fl) f-final' as' dg'
    (map snd fl) final'
and query-out:
  Some (result, final-state) ∈
    set-dist
    (execute
      (ntimes
        (verifier-query-round-program fr' f-fl f-final' as' fl
          final') rounds)
      query-state)

```

and *query-count-header*:
PQueryCounter query-state = PQueryCounter s
by *metis*
have *eqs*:
fr' = fr $\wedge$
map snd f-fl = f-fri-roots $\wedge$
f-final' = f-final $\wedge$
as' = as $\wedge$
dg' = dg $\wedge$
map snd fl = composition-fri-roots $\wedge$
final' = final $\wedge$
PTranscript query-state = rest
using *verifier-header-transcript-unique*[*OF header header0*] **by** *simp*
show *?thesis*
by (*rule that*[*of f-fl fl query-state*])
(use eqs header-state query-count-header query-out in simp-all)
qed

lemma *verify-monad-supplied-empty-header-query-rounds*:
fixes *s :: ('f, 'a) protocol-channel-scheme*
assumes *outcome*:
Some (result, final-state) ∈ set-dist (execute verify-monad s)
and *header0*:
verifier-header-transcript s fr f-fri-roots f-final as dg []
final rest
obtains *f-fl query-state* **where**
map snd f-fl = f-fri-roots
PTranscript query-state = rest
PState query-state =
verifier-header-state s fr f-fri-roots f-final as dg [] final
PQueryCounter query-state = PQueryCounter s
Some (result, final-state) ∈
set-dist
(execute
(ntimes
(verifier-query-round-program fr f-fl f-final as [] final)
rounds)
query-state)

proof –
from *verify-monad-supplied-header-query-rounds*[*OF outcome header0*]
obtain *f-fl fl query-state* **where** *f-roots*:
map snd f-fl = f-fri-roots
and *fl-roots*: *map snd fl = []*
and *tr*: *PTranscript query-state = rest*
and *st*:
PState query-state =
verifier-header-state s fr f-fri-roots f-final as dg [] final
and *qc*: *PQueryCounter query-state = PQueryCounter s*
and *out*:

```

    Some (result, final-state) ∈
      set-dist
        (execute
          (ntimes
            (verifier-query-round-program fr f-fl f-final as fl final)
            rounds)
          query-state)
  by blast
have fl-empty: fl = []
  using fl-roots by (cases fl) simp-all
show ?thesis
  by (rule that[of f-fl query-state])
    (use f-roots tr st qc out fl-empty in simp-all)
qed

lemma verifier-after-alpha-accepted-transcript-shape:
  fixes s :: ('f, 'a) protocol-channel-scheme
  assumes prefix:
    Some ((fr, f-fl, f-final, as), prefix-state) ∈
      set-dist (execute verifier-alpha-prefix s)
  and suffix:
    Some (result, final-state) ∈
      set-dist
        (execute (verifier-after-alpha (fr, f-fl, f-final, as))
          prefix-state)
  shows ∃ query-idxs.
    accepted-transcript-shape s (Some (result, final-state)) as query-idxs
proof -
  have prefix-res:
    length f-fl = ceil-log clength ∧
    length as = length spec ∧
    PTranscript s =
      [fr] @ map snd f-fl @ [f-final] @ as @ PTranscript prefix-state ∧
    PState prefix-state =
      foldl concat
        (concat (foldl concat (concat (PState s) fr) (map snd f-fl))
          f-final)
      as ∧
    PQueryCounter prefix-state = PQueryCounter s
  by (rule verifier-alpha-prefix-outcome[OF prefix])
from suffix obtain dg s5 s6 fl s7 final query-state where
  read-dg: Some (dg, s5) ∈ set-dist (execute read prefix-state)
  and degree-assert:
    Some ((), s6) ∈
      set-dist (execute (assert (to-nat dg ≤ maxDegree)) s5)
  and comp-fri:
    Some (fl, s7) ∈
      set-dist (execute
        (ntimes receive-composition-fri-commits

```

```

      (ceil-log (to-nat dg + 1))) s6)
and read-final: Some (final, query-state) ∈ set-dist (execute read s7)
and query-out:
  Some (result, final-state) ∈
    set-dist (execute
      (ntimes (verifier-query-round-program fr f-fl f-final as fl final)
        rounds)
      query-state)
unfolding verifier-after-alpha-def
by (auto elim!: set-dist-bindE)
from read-outcome[OF read-dg] obtain rest5 where
  tr-prefix: PTranscript prefix-state = dg # rest5
and st-s5: PState s5 = concat (PState prefix-state) dg
and tr-s5: PTranscript s5 = rest5
and query-count-s5: PQueryCounter s5 = PQueryCounter prefix-state
by blast
have s6-eq: s6 = s5
  using degree-assert unfolding assert-def
  by (cases to-nat dg ≤ maxDegree) (auto simp: throw-no-outcome)
have comp-res:
  length fl = ceil-log (to-nat dg + 1) ∧
    PTranscript s6 = map snd fl @ PTranscript s7 ∧
    PState s7 = foldl concat (PState s6) (map snd fl) ∧
    PQueryCounter s7 = PQueryCounter s6
  using ntimes-receive-composition-fri-commits-outcome[OF comp-fri]
  by simp
from read-outcome[OF read-final] obtain rest-query where
  tr-s7: PTranscript s7 = final # rest-query
and st-query: PState query-state = concat (PState s7) final
and tr-query: PTranscript query-state = rest-query
and query-count-query: PQueryCounter query-state = PQueryCounter s7
by blast
have header:
  verifier-header-transcript s fr (map snd f-fl) f-final as dg
    (map snd fl) final (PTranscript query-state)
  using prefix-res tr-prefix tr-s5 s6-eq comp-res tr-s7 tr-query
  unfolding verifier-header-transcript-def verifier-header-messages-def
  by simp
have header-state:
  PState query-state =
    verifier-header-state s fr (map snd f-fl) f-final as dg
      (map snd fl) final
  using prefix-res st-s5 s6-eq comp-res st-query
  unfolding verifier-header-state-def verifier-header-messages-def
  by simp
have query-count-header:
  PQueryCounter query-state = PQueryCounter s
  using prefix-res query-count-s5 s6-eq comp-res query-count-query
  by simp

```

```

from ntimes-verifier-query-rounds-outcome[OF query-out]
obtain raw-idxs query-idxs query-chunks where
  len-raw: length raw-idxs = rounds
  and query-idxs-def: query-idxs = map (λraw. index (to-nat raw)) raw-idxs
  and len-chunks: length query-chunks = rounds
  and tr-query-rounds:
    PTranscript query-state =
      List.concat query-chunks @ PTranscript final-state
  and round-chunks:
     $\forall i < \text{rounds}.$ 
      verifier-query-round-chunk (query-idxs ! i)
        (map snd f-fl) (map snd fl) (query-chunks ! i)
  and lookups:
     $\forall i < \text{rounds}.$ 
      fmlookup (HashMap final-state)
        (QueryIndexChallenge (PQueryCounter query-state + i)
          (state-after-query-chunks (PState query-state) query-chunks i)) =
          Some (raw-idxs ! i)
  and idx-bounds:  $\forall \text{idx} \in \text{set query-idxs}. \text{idx} < \text{clength} * \text{scale}$ 
by blast
have derived:
  verifier-query-indices-derived s (Some (result, final-state))
    (verifier-header-state s fr (map snd f-fl) f-final as dg
      (map snd fl) final)
    (PTranscript query-state) (map snd f-fl) (map snd fl) query-idxs
proof –
  have tr-query-rounds':
    List.concat query-chunks @ PTranscript final-state =
      PTranscript query-state
  using tr-query-rounds by simp
have lookups':
   $\forall i < \text{rounds}.$ 
    fmlookup (HashMap final-state)
      (QueryIndexChallenge (PQueryCounter s + i)
        (state-after-query-chunks
          (verifier-header-state s fr (map snd f-fl) f-final as dg
            (map snd fl) final)
          query-chunks i)) =
        Some (raw-idxs ! i)
  using lookups header-state query-count-header by simp
show ?thesis
  unfolding verifier-query-indices-derived-def
  apply (rule exI[where x=result])
  apply (rule exI[where x=final-state])
  apply (rule exI[where x=raw-idxs])
  apply (rule exI[where x=query-chunks])
  apply (rule exI[where x=PTranscript final-state])
  apply (intro conjI)
  apply simp

```

```

    apply (rule len-raw)
    apply (rule query-idxs-def)
    apply (rule len-chunks)
    apply (rule tr-query-rounds')
    apply (rule round-chunks)
    apply (rule lookups')
    apply (rule idx-bounds)
  done
qed
have shape:
  accepted-transcript-shape s (Some (result, final-state)) as query-idxs
  unfolding accepted-transcript-shape-def
  apply (rule exI[where x=result])
  apply (rule exI[where x=final-state])
  apply (rule exI[where x=fr])
  apply (rule exI[where x=map snd f-fl])
  apply (rule exI[where x=f-final])
  apply (rule exI[where x=dg])
  apply (rule exI[where x=map snd fl])
  apply (rule exI[where x=final])
  apply (rule exI[where x=PTranscript query-state])
  apply (intro conjI)
    apply simp
    apply (rule header)
    apply (rule derived)
  done
show ?thesis
  using shape by blast
qed

lemma verifier-after-alpha-accepted-transcript-header:
  fixes s :: ('f, 'a) protocol-channel-scheme
  assumes prefix:
    Some ((fr, f-fl, f-final, as), prefix-state) ∈
      set-dist (execute verifier-alpha-prefix s)
  and suffix:
    Some (result, final-state) ∈
      set-dist
        (execute (verifier-after-alpha (fr, f-fl, f-final, as))
          prefix-state)
  obtains dg composition-fri-roots final rest query-idxs
  where
    verifier-header-transcript s fr (map snd f-fl) f-final as dg
      composition-fri-roots final rest
    and accepted-transcript-shape s (Some (result, final-state))
      as query-idxs
proof -
  have prefix-res:
    length f-fl = ceil-log clength ∧

```

```

length as = length spec ∧
PTranscript s =
  [fr] @ map snd f-fl @ [f-final] @ as @ PTranscript prefix-state
using verifier-alpha-prefix-outcome[OF prefix] by simp
from suffix obtain dg s5 s6 fl s7 final query-state where
  read-dg: Some (dg, s5) ∈ set-dist (execute read prefix-state)
and degree-assert:
  Some ((), s6) ∈
    set-dist (execute (assert (to-nat dg ≤ maxDegree)) s5)
and comp-fri:
  Some (fl, s7) ∈
    set-dist (execute
      (ntimes receive-composition-fri-commits
        (ceil-log (to-nat dg + 1))) s6)
and read-final: Some (final, query-state) ∈ set-dist (execute read s7)
unfolding verifier-after-alpha-def
by (auto elim!: set-dist-bindE)
from read-outcome[OF read-dg] obtain rest5 where
  tr-prefix: PTranscript prefix-state = dg # rest5
and tr-s5: PTranscript s5 = rest5
by blast
have s6-eq: s6 = s5
using degree-assert unfolding assert-def
by (cases to-nat dg ≤ maxDegree) (auto simp: throw-no-outcome)
have comp-res:
  length fl = ceil-log (to-nat dg + 1) ∧
  PTranscript s6 = map snd fl @ PTranscript s7
using ntimes-receive-composition-fri-commits-outcome[OF comp-fri]
by simp
from read-outcome[OF read-final] obtain rest-query where
  tr-s7: PTranscript s7 = final # rest-query
and tr-query: PTranscript query-state = rest-query
by blast
have header:
  verifier-header-transcript s fr (map snd f-fl) f-final as dg
    (map snd fl) final (PTranscript query-state)
using prefix-res tr-prefix tr-s5 s6-eq comp-res tr-s7 tr-query
unfolding verifier-header-transcript-def verifier-header-messages-def
by simp
from verifier-after-alpha-accepted-transcript-shape[OF prefix suffix]
obtain query-idxs where shape:
  accepted-transcript-shape s (Some (result, final-state)) as query-idxs
by blast
show ?thesis
by (rule that[of dg map snd fl final PTranscript query-state query-idxs,
  OF header shape])
qed

```

lemma verify-monad-merkle-bound-witnesses:

fixes $s :: ('f, 'a) \text{ protocol-channel-scheme}$
assumes $\text{bind: initial-merkle-binding-no-bad } s$
and outcome:
 $\text{Some (result, final-state)} \in \text{set-dist (execute verify-monad } s)$
shows $\exists \text{ trace-table composition-table alphas query-idxs.}$
 $\text{accepted-with-bound-tables } s \text{ (Some (result, final-state))}$
 $\text{trace-table composition-table alphas query-idxs}$
proof –
from $\text{verify-monad-accepted-transcript-shape}[OF \text{ outcome}]$
obtain $\text{alphas query-idxs where}$
 shape:
 $\text{accepted-transcript-shape } s \text{ (Some (result, final-state)) alphas query-idxs}$
by blast
from $\text{initial-merkle-binding-no-badE}[OF \text{ bind outcome shape}]$
obtain $\text{trace-table composition-table where}$
 bound:
 $\text{accepted-with-bound-tables } s \text{ (Some (result, final-state))}$
 $\text{trace-table composition-table alphas query-idxs}$
by blast
show $?thesis$
using bound by blast
qed

lemma $\text{verify-monad-merkle-bound-tables:}$
fixes $s :: ('f, 'a) \text{ protocol-channel-scheme}$
assumes $\text{bind: initial-merkle-binding-no-bad } s$
and outcome:
 $\text{Some (result, final-state)} \in \text{set-dist (execute verify-monad } s)$
obtains $\text{trace-table composition-table alphas query-idxs}$
where
 $\text{accepted-with-bound-tables } s \text{ (Some (result, final-state))}$
 $\text{trace-table composition-table alphas query-idxs}$
and $\text{accepted-with-tables } s \text{ (Some (result, final-state))}$
 $\text{trace-table composition-table alphas query-idxs}$
and $\bigwedge \text{idx. idx} \in \text{set query-idxs} \implies$
 $\text{query-consistent-at trace-table composition-table alphas idx}$
and $\exists \text{fr composition-root.}$
 $\text{merkle-root-binds-table fr trace-table final-state} \wedge$
 $\text{merkle-root-binds-table composition-root composition-table final-state}$
proof –
from $\text{verify-monad-merkle-bound-witnesses}[OF \text{ bind outcome}]$
obtain $\text{trace-table composition-table alphas query-idxs where}$
 bound:
 $\text{accepted-with-bound-tables } s \text{ (Some (result, final-state))}$
 $\text{trace-table composition-table alphas query-idxs}$
by blast
have tables:
 $\text{accepted-with-tables } s \text{ (Some (result, final-state))}$
 $\text{trace-table composition-table alphas query-idxs}$

```

  by (rule accepted-with-bound-tables-imp-accepted-with-tables[OF bound])
have query-cons:
   $\bigwedge idx. idx \in \text{set query-idxs} \implies$ 
    query-consistent-at trace-table composition-table alphas idx
  by (rule accepted-with-bound-tables-query-consistency[OF bound])
have roots:
   $\exists fr$  composition-root.
    merkle-root-binds-table fr trace-table final-state  $\wedge$ 
    merkle-root-binds-table composition-root composition-table final-state
  using accepted-with-bound-tables-bound-roots[OF bound] by blast
show ?thesis
  by (rule that[OF bound tables query-cons roots])
qed

```

lemma *verifier-query-indices-derivedE*:

assumes *verifier-query-indices-derived s out query-start-state rest*
f-fri-roots composition-fri-roots query-idxs

obtains *result final-state raw-idxs query-chunks trailing*

where *out = Some (result, final-state)*
and length raw-idxs = rounds
and query-idxs = map ($\lambda raw. \text{index (to-nat raw)}$) raw-idxs
and length query-chunks = rounds
and List.concat query-chunks @ trailing = rest
and $\bigwedge i. i < \text{rounds} \implies$
verifier-query-round-chunk (query-idxs ! i)
f-fri-roots composition-fri-roots (query-chunks ! i)
and $\bigwedge i. i < \text{rounds} \implies$
fmlookup (HashMap final-state)
(QueryIndexChallenge (PQueryCounter s + i)
(state-after-query-chunks query-start-state query-chunks i)) =
Some (raw-idxs ! i)
*and $\bigwedge idx. idx \in \text{set query-idxs} \implies idx < \text{clength} * \text{scale}$*
using *assms* **unfolding** *verifier-query-indices-derived-def* **by** *blast*

lemma *accepted-transcript-shape-header-query-extraction*:

assumes *accepted-transcript-shape s out alphas query-idxs*

obtains *result final-state fr f-fri-roots f-final dg composition-fri-roots final rest*

where *out = Some (result, final-state)*
and verifier-header-transcript s fr f-fri-roots f-final alphas dg
composition-fri-roots final rest
and verifier-query-indices-derived s out
(verifier-header-state s fr f-fri-roots f-final alphas dg composition-fri-roots final)
rest f-fri-roots composition-fri-roots query-idxs
using *assms* **unfolding** *accepted-transcript-shape-def* **by** *blast*

lemma *accepted-transcript-shape-alphas-unique*:

assumes *shape1: accepted-transcript-shape s out alphas query-idxs*
and shape2: accepted-transcript-shape s out alphas' query-idxs'

shows *alphas' = alphas*

proof –
from *shape1* **obtain** *result final-state fr f-fri-roots f-final dg*
composition-fri-roots final rest **where**
out1: out = Some (result, final-state)
and *header1:*
verifier-header-transcript s fr f-fri-roots f-final alphas dg
composition-fri-roots final rest
by (*elim accepted-transcript-shape-header-query-extraction*)
from *shape2* **obtain** *result' final-state' fr' f-fri-roots' f-final' dg'*
composition-fri-roots' final' rest' **where**
out2: out = Some (result', final-state')
and *header2:*
verifier-header-transcript s fr' f-fri-roots' f-final' alphas' dg'
composition-fri-roots' final' rest'
by (*elim accepted-transcript-shape-header-query-extraction*)
show *?thesis*
using *verifier-header-transcript-unique[OF header1 header2]* **by** *simp*
qed

lemma *accepted-with-bound-tables-alphas-unique:*
assumes *bound1:*
accepted-with-bound-tables s out trace-table composition-table
alphas query-idxs
and *bound2:*
accepted-with-bound-tables s out trace-table' composition-table'
alphas' query-idxs'
shows *alphas' = alphas*

proof –
have *shape1: accepted-transcript-shape s out alphas query-idxs*
using *bound1 unfolding accepted-with-bound-tables-def*
accepted-with-tables-def **by** *simp*
have *shape2: accepted-transcript-shape s out alphas' query-idxs'*
using *bound2 unfolding accepted-with-bound-tables-def*
accepted-with-tables-def **by** *simp*
show *?thesis*
by (*rule accepted-transcript-shape-alphas-unique[OF shape1 shape2]*)
qed

lemma *wp-verify-monad-alpha-list-set-bound:*
assumes *future: alpha-future-fresh s*
and *subset: B ⊆ alpha-space*
shows
wp-event verify-monad (alpha-list-set-hit s B) s ≤
nnreal (card B) / nnreal (card alpha-space)

proof –
let *?Head =*
λout. case out of
None ⇒ False
| Some ((-, -, -, as), -) ⇒ as ∈ B

```

have prefix-bound:
  wp-event verifier-alpha-prefix ?Head s ≤
    nnreal (card B) / nnreal (card alpha-space)
by (rule wp-verifier-alpha-prefix-alpha-space-set-bound
    [OF future subset])
show ?thesis
  unfolding verify-monad-alpha-decomposition
proof (rule wp-event-bind-bound-by-head-event[OF prefix-bound])
  show alpha-list-set-hit s B None ⇒ ?Head None
    unfolding alpha-list-set-hit-def accepted-transcript-shape-def by blast
next
fix header prefix-state out
assume prefix:
  Some (header, prefix-state) ∈
    set-dist (execute verifier-alpha-prefix s)
and suffix:
  out ∈
    set-dist (execute (verifier-after-alpha header) prefix-state)
and hit: alpha-list-set-hit s B out
obtain fr f-fl f-final as where header-eq:
  header = (fr, f-fl, f-final, as)
by (cases header) auto
from hit obtain as' query-idxs where
  shape-hit: accepted-transcript-shape s out as' query-idxs
and as'-B: as' ∈ B
unfolding alpha-list-set-hit-def by blast
from shape-hit obtain result final-state where out-eq:
  out = Some (result, final-state)
unfolding accepted-transcript-shape-def by blast
have prefix':
  Some ((fr, f-fl, f-final, as), prefix-state) ∈
    set-dist (execute verifier-alpha-prefix s)
using prefix header-eq by simp
have suffix':
  Some (result, final-state) ∈
    set-dist
      (execute (verifier-after-alpha (fr, f-fl, f-final, as))
        prefix-state)
using suffix header-eq out-eq by simp
from verifier-after-alpha-accepted-transcript-shape[OF prefix' suffix']
obtain query-idxs' where
  shape-prefix:
    accepted-transcript-shape s (Some (result, final-state))
    as query-idxs'
by blast
have as' = as
by (rule accepted-transcript-shape-alphas-unique
  [OF shape-prefix shape-hit[unfolded out-eq]])
then show ?Head (Some (header, prefix-state))

```

```

    using as'-B header-eq by simp
  qed
qed

lemma wp-verify-monad-alpha-header-list-set-bound:
  fixes C :: prob
  assumes future: alpha-future-fresh s
  and subset:
     $\bigwedge fr\ f\text{-fri-roots}\ f\text{-final}.$ 
     $B\ fr\ f\text{-fri-roots}\ f\text{-final} \subseteq \text{alpha-space}$ 
  and bound:
     $\bigwedge fr\ f\text{-fri-roots}\ f\text{-final}.$ 
     $\text{nnreal}(\text{card}(B\ fr\ f\text{-fri-roots}\ f\text{-final})) /$ 
     $\text{nnreal}(\text{card}\ \text{alpha-space}) \leq C$ 
  shows
    wp-event verify-monad (alpha-header-list-set-hit s B) s  $\leq C$ 
proof -
  let ?Head =
     $\lambda out.$  case out of
      None  $\Rightarrow$  False
    | Some ((fr, f-fl, f-final, as), -)  $\Rightarrow$ 
      as  $\in B\ fr\ (\text{map}\ \text{snd}\ f\text{-fl})\ f\text{-final}$ 
  have prefix-bound:
    wp-event verifier-alpha-prefix ?Head s  $\leq C$ 
  by (rule wp-verifier-alpha-prefix-dependent-alpha-space-set-bound
    [OF future])
    (use subset bound in simp-all)
  show ?thesis
    unfolding verify-monad-alpha-decomposition
  proof (rule wp-event-bind-bound-by-head-event[OF prefix-bound])
    show alpha-header-list-set-hit s B None  $\implies$  ?Head None
      unfolding alpha-header-list-set-hit-def
      accepted-transcript-shape-def by blast
  next
    fix header prefix-state out
    assume prefix:
      Some (header, prefix-state)  $\in$ 
      set-dist (execute verifier-alpha-prefix s)
    and suffix:
      out  $\in$ 
      set-dist (execute (verifier-after-alpha header) prefix-state)
    and hit: alpha-header-list-set-hit s B out
    obtain fr f-fl f-final as where header-eq:
      header = (fr, f-fl, f-final, as)
    by (cases header) auto
    from hit obtain as' query-idxs result final-state fr' f-fri-roots'
      f-final' dg' composition-fri-roots' final' rest' where
      shape-hit:
        accepted-transcript-shape s out as' query-idxs

```

```

and out-eq: out = Some (result, final-state)
and header-hit:
  verifier-header-transcript s fr' f-fri-roots' f-final' as' dg'
  composition-fri-roots' final' rest'
and as'-B: as' ∈ B fr' f-fri-roots' f-final'
unfolding alpha-header-list-set-hit-def by blast
have prefix':
  Some ((fr, f-fl, f-final, as), prefix-state) ∈
    set-dist (execute verifier-alpha-prefix s)
using prefix header-eq by simp
have suffix':
  Some (result, final-state) ∈
    set-dist
      (execute (verifier-after-alpha (fr, f-fl, f-final, as))
        prefix-state)
using suffix header-eq out-eq by simp
from verifier-after-alpha-accepted-transcript-header[OF prefix' suffix']
obtain dg composition-fri-roots final rest query-idxs' where
  header-prefix:
    verifier-header-transcript s fr (map snd f-fl) f-final as dg
    composition-fri-roots final rest
and shape-prefix:
  accepted-transcript-shape s (Some (result, final-state))
  as query-idxs'
by metis
have unique-header:
  fr' = fr ∧
  f-fri-roots' = map snd f-fl ∧
  f-final' = f-final ∧
  as' = as
using verifier-header-transcript-unique[OF header-prefix header-hit]
by simp
then show ?Head (Some (header, prefix-state))
using as'-B header-eq by simp
qed
qed

lemma wp-alpha-bad-set-hit-bound-via-header-union:
fixes C :: prob
assumes future: alpha-future-fresh s
and subset:  $\bigwedge \text{trace-table}. \text{bad-sets trace-table} \subseteq \text{alpha-space}$ 
and union-bound:
   $\bigwedge \text{fr f-fri-roots f-final.}$ 
  nnreal
  (card
    (alpha-header-union-bad-sets s bad-sets fr f-fri-roots f-final)) /
  nnreal (card alpha-space) ≤ C
shows wp-event verify-monad (alpha-bad-set-hit s bad-sets) s ≤ C
proof –

```

have *header-bound*:
 wp-event verify-monad
 (*alpha-header-list-set-hit s*
 (*alpha-header-union-bad-sets s bad-sets*)) $s \leq C$
by (*rule wp-verify-monad-alpha-header-list-set-bound*[*OF future*])
 (*use alpha-header-union-bad-sets-subset-alpha-space union-bound in simp-all*)
have *wp-event verify-monad (alpha-bad-set-hit s bad-sets) s ≤*
 wp-event verify-monad
 (*alpha-header-list-set-hit s*
 (*alpha-header-union-bad-sets s bad-sets*)) s
by (*rule wp-event-mono*)
 (*rule alpha-bad-set-hit-imp-alpha-header-union-bad-set-hit*[*OF - subset*])
also have $\dots \leq C$
 by (rule header-bound)
finally show *?thesis* .
qed

lemma *wp-alpha-bad-set-hit-bound-via-supported-header-union*:
fixes $C :: \text{prob}$
assumes *future: alpha-future-fresh s*
and *subset: $\bigwedge \text{trace-table. bad-sets trace-table} \subseteq \text{alpha-space}$*
and *union-bound*:
 $\bigwedge \text{fr f-fri-roots f-final.}$
 nnreal
 (*card*
 (*alpha-header-supported-union-bad-sets s bad-sets fr f-fri-roots*
 f-final)) /
 nnreal (card alpha-space) $\leq C$
shows *wp-event verify-monad (alpha-bad-set-hit s bad-sets) s ≤ C*
proof –
have *header-bound*:
 wp-event verify-monad
 (*alpha-header-list-set-hit s*
 (*alpha-header-supported-union-bad-sets s bad-sets*)) $s \leq C$
by (*rule wp-verify-monad-alpha-header-list-set-bound*[*OF future*])
 (*use alpha-header-supported-union-bad-sets-subset-alpha-space*
 union-bound in simp-all)
have *wp-event verify-monad (alpha-bad-set-hit s bad-sets) s ≤*
 wp-event verify-monad
 (*alpha-header-list-set-hit s*
 (*alpha-header-supported-union-bad-sets s bad-sets*)) s
by (*rule wp-event-mono-on-support*)
 (*rule alpha-bad-set-hit-imp-alpha-header-supported-union-bad-set-hit*
 [*OF - - subset*])
also have $\dots \leq C$
 by (rule header-bound)
finally show *?thesis* .
qed

lemma *accepted-with-tables-header-query-extraction:*

assumes *accepted-with-tables s out trace-table composition-table alphas query-idxs*

obtains *result final-state fr f-fri-roots f-final dg composition-fri-roots final rest*

where *out = Some (result, final-state)*

and *verifier-header-transcript s fr f-fri-roots f-final alphas dg
composition-fri-roots final rest*

and *verifier-query-indices-derived s out
(verifier-header-state s fr f-fri-roots f-final alphas dg composition-fri-roots final)
rest f-fri-roots composition-fri-roots query-idxs*

and $\bigwedge idx. idx \in \text{set query-idxs} \implies$
query-consistent-at trace-table composition-table alphas idx

and *length trace-table = clength * scale*

and *length composition-table = clength * scale*

and *length alphas = length spec*

using *assms*

unfolding *accepted-with-tables-def accepted-transcript-shape-def*

by *blast*

lemma *accepted-with-tables-existential-witnesses:*

assumes *accepted-with-tables s out trace-table composition-table alphas query-idxs*

shows $\exists \text{trace-table composition-table as query-idxs.}$

accepted-with-tables s out trace-table composition-table as query-idxs

using *assms by blast*

lemma *accepted-transcript-shape-header-extraction:*

assumes *shape: accepted-transcript-shape s (Some (result, final-state)) alphas query-idxs*

shows $\exists \text{fr f-fri-roots f-final as dg composition-fri-roots final rest.}$

*verifier-header-transcript s fr f-fri-roots f-final as dg
composition-fri-roots final rest*

using *shape unfolding accepted-transcript-shape-def by blast*

lemma *accepted-transcript-shape-query-indices-extraction:*

assumes *shape: accepted-transcript-shape s (Some (result, final-state)) alphas query-idxs*

shows $\exists \text{fr f-fri-roots f-final dg composition-fri-roots final rest.}$

*verifier-query-indices-derived s (Some (result, final-state))
(verifier-header-state s fr f-fri-roots f-final alphas dg
composition-fri-roots final)*

rest f-fri-roots composition-fri-roots query-idxs

using *shape unfolding accepted-transcript-shape-def by blast*

lemma *accepted-with-tables-has-witnesses:*

assumes *tables: accepted-with-tables s (Some (result, final-state))*

trace-table composition-table alphas query-idxs

shows $\exists \text{trace-table composition-table as query-idxs.}$

accepted-with-tables s (Some (result, final-state))

trace-table composition-table as query-idxs

using *tables by blast*

end

end

theory *Soundness-Execution*
 imports *Soundness-Execution-Extraction*
begin

Compatibility umbrella for verifier execution soundness lemmas.

end

theory *Soundness-Bad-Events*
 imports
 Soundness-Execution
begin

context *soundness*
begin

At this stage *accepted-with-tables* is the boundary between the verifier execution and the algebraic proof. It records the trace table, composition table, and verifier challenges associated with an accepted execution. It now also records that the initial verifier transcript has the expected header shape and that the sampled query positions are locally consistent with the associated tables. Merkle binding is separated into *accepted-with-bound-tables*, because malicious binding requires additional Merkle/collision-resistance reasoning.

definition *composition-bad-context*

$:: ('f, 'a) \text{ protocol-channel-scheme} \Rightarrow$
 $(\text{unit list} \times ('f, 'a) \text{ protocol-channel-scheme}) \text{ option} \Rightarrow$
 $'f \text{ list} \Rightarrow 'f \text{ list} \Rightarrow 'f \text{ list} \Rightarrow \text{nat list} \Rightarrow$
 $'f \text{ poly} \Rightarrow \text{bool}$

where

composition-bad-context *s* out *trace-table* *composition-table* as *query-idxs* *f* $\longleftrightarrow$
 accepted-with-bound-tables *s* out *trace-table* *composition-table* as *query-idxs* $\wedge$
 $\text{degree } f < \text{clength} \wedge$
 $\text{trace-table} = \text{map } (\text{poly } f) \text{ eval-domain} \wedge$
 $\text{violated-constraints } f \neq \{\}$ $\wedge$
 $\text{composition-table-low-degree } \text{maxDegree } \text{composition-table} \wedge$
 all-queries-consistent *trace-table* *composition-table* as

definition *composition-degree-bad*

$:: ('f, 'a) \text{ protocol-channel-scheme} \Rightarrow$
 $(\text{unit list} \times ('f, 'a) \text{ protocol-channel-scheme}) \text{ option} \Rightarrow \text{bool}$

where

composition-degree-bad *s* out $\longleftrightarrow$
 $(\exists \text{ trace-table } \text{composition-table as query-idxs } f.$
 composition-bad-context *s* out *trace-table* *composition-table* as *query-idxs* *f* $\wedge$

$\neg$ *common-denominator-degree-bounds* *f as*)

definition *composition-randomization-bad*

$:: (f, 'a) \text{ protocol-channel-scheme} \Rightarrow$
 $(\text{unit list} \times (f, 'a) \text{ protocol-channel-scheme}) \text{ option} \Rightarrow \text{bool}$

where

composition-randomization-bad *s out* $\longleftrightarrow$
 $(\exists \text{ trace-table } \text{composition-table as query-idxs } f.$
 $\text{composition-bad-context } s \text{ out trace-table } \text{composition-table as query-idxs } f \wedge$
 $\text{common-denominator-degree-bounds } f \text{ as} \wedge$
 $\text{random-combination-common-denominator-hides-violations } f \text{ as})$

definition *composition-bad*

$:: (f, 'a) \text{ protocol-channel-scheme} \Rightarrow$
 $(\text{unit list} \times (f, 'a) \text{ protocol-channel-scheme}) \text{ option} \Rightarrow \text{bool}$

where

composition-bad *s out* $\longleftrightarrow$
 $\text{composition-degree-bad } s \text{ out} \vee \text{composition-randomization-bad } s \text{ out}$

lemma *composition-bad-has-common-denominator-witness:*

assumes *composition-bad* *s out*

shows $\exists \text{ trace-table } \text{composition-table as query-idxs } f.$

$\text{accepted-with-bound-tables } s \text{ out trace-table } \text{composition-table as query-idxs} \wedge$
 $\text{degree } f < \text{clength} \wedge$

$\text{trace-table} = \text{map } (\text{poly } f) \text{ eval-domain} \wedge$

$\text{violated-constraints } f \neq \{\}$ $\wedge$

$\text{composition-table-low-degree } \text{maxDegree } \text{composition-table} \wedge$

$\text{all-queries-consistent } \text{trace-table } \text{composition-table as} \wedge$

$(\neg \text{common-denominator-degree-bounds } f \text{ as} \vee$

$\text{random-combination-common-denominator-hides-violations } f \text{ as})$

using *assms*

unfolding *composition-bad-def* *composition-degree-bad-def*

composition-randomization-bad-def *composition-bad-context-def*

by *blast*

lemma *composition-bad-imp-spec-nonempty:*

assumes *composition-bad* *s out*

shows $\text{spec} \neq []$

proof –

obtain *trace-table* *composition-table as query-idxs* *f*

where $\text{violated-constraints } f \neq \{\}$

using *composition-bad-has-common-denominator-witness*[*OF assms*] **by** *blast*

then obtain *entry* **where** $\text{entry} \in \text{violated-constraints } f$

by *auto*

then have $\text{entry} \in \text{set spec}$

unfolding *violated-constraints-def* **by** *simp*

then show *?thesis*

by *auto*

qed

definition *trace-fri-bad*

$:: ('f, 'a) \text{ protocol-channel-scheme} \Rightarrow$
 $(\text{unit list} \times ('f, 'a) \text{ protocol-channel-scheme}) \text{ option} \Rightarrow \text{bool}$

where

$\text{trace-fri-bad } s \text{ out} \longleftrightarrow$
 $(\exists \text{ trace-table composition-table as query-idxs.}$
 $\text{accepted-with-bound-tables } s \text{ out trace-table composition-table as query-idxs} \wedge$
 $\neg \text{ trace-table-low-degree trace-table})$

definition *composition-fri-bad*

$:: ('f, 'a) \text{ protocol-channel-scheme} \Rightarrow$
 $(\text{unit list} \times ('f, 'a) \text{ protocol-channel-scheme}) \text{ option} \Rightarrow \text{bool}$

where

$\text{composition-fri-bad } s \text{ out} \longleftrightarrow$
 $(\exists \text{ trace-table composition-table as query-idxs.}$
 $\text{accepted-with-bound-tables } s \text{ out trace-table composition-table as query-idxs} \wedge$
 $\text{trace-table-low-degree trace-table} \wedge$
 $\neg \text{ composition-table-low-degree maxDegree composition-table})$

definition *query-bad*

$:: ('f, 'a) \text{ protocol-channel-scheme} \Rightarrow$
 $(\text{unit list} \times ('f, 'a) \text{ protocol-channel-scheme}) \text{ option} \Rightarrow \text{bool}$

where

$\text{query-bad } s \text{ out} \longleftrightarrow$
 $(\exists \text{ trace-table composition-table as query-idxs.}$
 $\text{accepted-with-bound-tables } s \text{ out trace-table composition-table as query-idxs} \wedge$
 $\text{trace-table-low-degree trace-table} \wedge$
 $\text{composition-table-low-degree maxDegree composition-table} \wedge$
 $\neg \text{ all-queries-consistent trace-table composition-table as})$

definition *soundness-bad-event*

$:: ('f, 'a) \text{ protocol-channel-scheme} \Rightarrow$
 $(\text{unit list} \times ('f, 'a) \text{ protocol-channel-scheme}) \text{ option} \Rightarrow \text{bool}$

where

$\text{soundness-bad-event } s \text{ out} \longleftrightarrow$
 $\text{composition-bad } s \text{ out} \vee$
 $\text{trace-fri-bad } s \text{ out} \vee$
 $\text{composition-fri-bad } s \text{ out} \vee$
 $\text{query-bad } s \text{ out}$

definition *query-sampling-success-space*

$:: 'f \text{ list} \Rightarrow 'f \text{ list} \Rightarrow 'f \text{ list} \Rightarrow \text{nat set}$

where

$\text{query-sampling-success-space trace-table composition-table as} =$
 $(\text{if trace-table-low-degree trace-table} \wedge$
 $\text{composition-table-low-degree maxDegree composition-table} \wedge$
 $\neg \text{ all-queries-consistent trace-table composition-table as}$
 $\text{then query-agreement-indices trace-table composition-table as})$

else {})

definition *query-index-set-hit*

$:: ('f, 'a) \text{ protocol-channel-scheme} \Rightarrow$
 $('f \text{ list} \Rightarrow 'f \text{ list} \Rightarrow 'f \text{ list} \Rightarrow \text{nat set}) \Rightarrow$
 $(\text{unit list} \times ('f, 'a) \text{ protocol-channel-scheme}) \text{ option} \Rightarrow \text{bool}$

where

$\text{query-index-set-hit } s \text{ good-sets out} \longleftrightarrow$
 $(\exists \text{ trace-table composition-table as query-idxs.}$
 $\text{accepted-with-bound-tables } s \text{ out trace-table composition-table as query-idxs} \wedge$
 $\text{set query-idxs} \subseteq \text{good-sets trace-table composition-table as})$

definition *query-index-round-set-hit-at*

$:: ('f, 'a) \text{ protocol-channel-scheme} \Rightarrow$
 $('f \text{ list} \Rightarrow 'f \text{ list} \Rightarrow 'f \text{ list} \Rightarrow \text{nat set}) \Rightarrow$
 $\text{nat} \Rightarrow (\text{unit list} \times ('f, 'a) \text{ protocol-channel-scheme}) \text{ option} \Rightarrow \text{bool}$

where

$\text{query-index-round-set-hit-at } s \text{ good-sets } i \text{ out} \longleftrightarrow$
 $(\exists \text{ trace-table composition-table as query-idxs.}$
 $\text{accepted-with-bound-tables } s \text{ out trace-table composition-table as}$
 $\text{query-idxs} \wedge$
 $i < \text{rounds} \wedge$
 $\text{query-idxs} ! i \in \text{good-sets trace-table composition-table as})$

definition *query-index-round-set-hit*

$:: ('f, 'a) \text{ protocol-channel-scheme} \Rightarrow$
 $('f \text{ list} \Rightarrow 'f \text{ list} \Rightarrow 'f \text{ list} \Rightarrow \text{nat set}) \Rightarrow$
 $(\text{unit list} \times ('f, 'a) \text{ protocol-channel-scheme}) \text{ option} \Rightarrow \text{bool}$

where

$\text{query-index-round-set-hit } s \text{ good-sets out} \longleftrightarrow$
 $(\exists i < \text{rounds. query-index-round-set-hit-at } s \text{ good-sets } i \text{ out})$

definition *query-round-index-set-hit*

$:: ('f, 'a) \text{ protocol-channel-scheme} \Rightarrow \text{nat set} \Rightarrow$
 $(\text{unit} \times ('f, 'a) \text{ protocol-channel-scheme}) \text{ option} \Rightarrow \text{bool}$

where

$\text{query-round-index-set-hit } s \text{ B out} \longleftrightarrow$
 $(\exists \text{ raw } t.$
 $\text{out} = \text{Some } ((), t) \wedge$
 $\text{fmlookup } (\text{HashMap } t)$
 $(\text{QueryIndexChallenge } (P\text{QueryCounter } s) (P\text{State } s)) = \text{Some raw} \wedge$
 $\text{index } (\text{to-nat raw}) \in B)$

definition *query-round-any-index-set-hit*

$:: ('f, 'a) \text{ protocol-channel-scheme} \Rightarrow \text{nat set} \Rightarrow$
 $(\text{unit} \times ('f, 'a) \text{ protocol-channel-scheme}) \text{ option} \Rightarrow \text{bool}$

where

$\text{query-round-any-index-set-hit } s \text{ B out} \longleftrightarrow$
 $(\exists \text{ raw } t x.$

$out = \text{Some } ((), t) \wedge$
 $fmlookup \ (HashMap \ t)$
 $(\text{QueryIndexChallenge } (PQueryCounter \ s) \ x) = \text{Some } raw \wedge$
 $index \ (to\text{-}nat \ raw) \in B$

definition *query-rounds-any-index-set-hit*

$:: (f, 'a) \text{ protocol-channel-scheme} \Rightarrow nat \text{ set} \Rightarrow nat \Rightarrow$
 $(unit \ list \times (f, 'a) \text{ protocol-channel-scheme}) \text{ option} \Rightarrow bool$

where

$query\text{-}rounds\text{-}any\text{-}index\text{-}set\text{-}hit \ s \ B \ n \ out \longleftrightarrow$
 $(\exists \text{ results } t \ i \ x \ raw.$
 $out = \text{Some } (results, t) \wedge$
 $i < n \wedge$
 $fmlookup \ (HashMap \ t)$
 $(\text{QueryIndexChallenge } (PQueryCounter \ s + i) \ x) = \text{Some } raw \wedge$
 $index \ (to\text{-}nat \ raw) \in B)$

lemma *accepted-with-bound-tables-query-rounds-any-index-set-hit:*

assumes *bound:*

$accepted\text{-}with\text{-}bound\text{-}tables \ s \ out \ \text{trace-table} \ \text{composition-table} \ as$
 $query\text{-}idxs$

and *i-bound:* $i < rounds$

and *hit:* $query\text{-}idxs \ ! \ i \in B$

shows $query\text{-}rounds\text{-}any\text{-}index\text{-}set\text{-}hit \ s \ B \ rounds \ out$

proof –

have *shape:* $accepted\text{-}transcript\text{-}shape \ s \ out \ as \ query\text{-}idxs$

using $accepted\text{-}with\text{-}bound\text{-}tables\text{-}imp\text{-}accepted\text{-}with\text{-}tables[OF \ bound]$

$accepted\text{-}with\text{-}tables\text{-}imp\text{-}accepted\text{-}transcript\text{-}shape$ **by** *blast*

then obtain *result final-state fr f-fri-roots f-final dg*

$composition\text{-}fri\text{-}roots \ final \ rest \ raw\text{-}idxs \ query\text{-}chunks \ trailing$ **where**

out-eq: $out = \text{Some } (result, final\text{-}state)$

and *len-raw:* $length \ raw\text{-}idxs = rounds$

and *query-idxs-eq:* $query\text{-}idxs = map \ (\lambda raw. index \ (to\text{-}nat \ raw)) \ raw\text{-}idxs$

and *lookup:*

$\forall i < rounds.$

$fmlookup \ (HashMap \ final\text{-}state)$
 $(\text{QueryIndexChallenge } (PQueryCounter \ s + i)$
 $(state\text{-}after\text{-}query\text{-}chunks$
 $(verifier\text{-}header\text{-}state \ s \ fr \ f\text{-}fri\text{-}roots \ f\text{-}final \ as \ dg$
 $composition\text{-}fri\text{-}roots \ final)$
 $query\text{-}chunks \ i)) =$
 $\text{Some } (raw\text{-}idxs \ ! \ i)$

unfolding $accepted\text{-}transcript\text{-}shape\text{-}def \ verifier\text{-}query\text{-}indices\text{-}derived\text{-}def$

by *blast*

have *raw-hit:* $index \ (to\text{-}nat \ (raw\text{-}idxs \ ! \ i)) \in B$

using *hit query-idxs-eq len-raw i-bound* **by** *simp*

have *lookup-i:*

$fmlookup \ (HashMap \ final\text{-}state)$
 $(\text{QueryIndexChallenge } (PQueryCounter \ s + i)$

```

    (state-after-query-chunks
      (verifier-header-state s fr f-fri-roots f-final as dg
        composition-fri-roots final)
      query-chunks i)) =
    Some (raw-idxs ! i)
  using lookup i-bound by simp
show ?thesis
  unfolding query-rounds-any-index-set-hit-def
  apply (rule exI[where x=result])
  apply (rule exI[where x=final-state])
  apply (rule exI[where x=i])
  apply (rule exI[
    where x=state-after-query-chunks
      (verifier-header-state s fr f-fri-roots f-final as dg
        composition-fri-roots final)
      query-chunks i])
  apply (rule exI[where x=raw-idxs ! i])
  using out-eq i-bound lookup-i raw-hit by simp
qed

lemma query-index-round-set-hit-at-imp-query-rounds-any-index-set-hit:
  assumes hit: query-index-round-set-hit-at s good-sets i out
  shows
     $\exists$  trace-table composition-table as.
      query-rounds-any-index-set-hit s
        (good-sets trace-table composition-table as) rounds out
  proof -
    from hit obtain trace-table composition-table as query-idxs where
      bound:
        accepted-with-bound-tables s out trace-table composition-table as
          query-idxs
    and i-bound:  $i < \text{rounds}$ 
    and query-hit:
      query-idxs !  $i \in \text{good-sets trace-table composition-table as}$ 
    unfolding query-index-round-set-hit-at-def by blast
    have round-hit:
      query-rounds-any-index-set-hit s
        (good-sets trace-table composition-table as) rounds out
    by (rule accepted-with-bound-tables-query-rounds-any-index-set-hit
      [OF bound i-bound query-hit])
    show ?thesis
      using round-hit by blast
  qed

definition query-header-supported-table-candidates
  :: ('f, 'a) protocol-channel-scheme  $\Rightarrow$ 
    'f  $\Rightarrow$  'f list  $\Rightarrow$  'f  $\Rightarrow$  'f list  $\Rightarrow$  'f  $\Rightarrow$ 
    'f list  $\Rightarrow$  'f  $\Rightarrow$  ('f list  $\times$  'f list) set
  where

```

query-header-supported-table-candidates *s* *fr* *f-fri-roots* *f-final* *as* *dg*
composition-fri-roots *final* =
 {(*trace-table*, *composition-table*).
 ∃ *out query-idxs rest*.
out ∈ *set-dist* (*execute verify-monad s*) ∧
accepted-with-bound-tables s out trace-table composition-table as
query-idxs ∧
verifier-header-transcript s fr f-fri-roots f-final as dg
composition-fri-roots final rest}

definition *query-header-supported-table-candidate-state*

:: (*f*, *'a*) *protocol-channel-scheme* ⇒
'f ⇒ *'f list* ⇒ *'f* ⇒ *'f list* ⇒ *'f* ⇒
'f list ⇒ *'f* ⇒ *'f list* ⇒ *'f list* ⇒
 (*f*, *'a*) *protocol-channel-scheme* ⇒ *bool*

where

query-header-supported-table-candidate-state s fr f-fri-roots f-final
as dg composition-fri-roots final trace-table composition-table
final-state ⇔
 (∃ *result query-idxs rest*.
Some (result, final-state) ∈ *set-dist* (*execute verify-monad s*) ∧
accepted-with-bound-tables s (Some (result, final-state))
trace-table composition-table as query-idxs ∧
verifier-header-transcript s fr f-fri-roots f-final as dg
composition-fri-roots final rest ∧
merkle-root-binds-table fr trace-table final-state ∧
composition-fri-roots ≠ [] ∧
merkle-root-binds-table (hd composition-fri-roots) composition-table
final-state)

lemma *query-header-supported-table-candidates-iff-state:*

(*trace-table*, *composition-table*) ∈
query-header-supported-table-candidates s fr f-fri-roots f-final as dg
composition-fri-roots final ⇔
 (∃ *final-state*.
query-header-supported-table-candidate-state s fr f-fri-roots f-final
as dg composition-fri-roots final trace-table composition-table
final-state)

proof

assume *candidate:*

(*trace-table*, *composition-table*) ∈
query-header-supported-table-candidates s fr f-fri-roots f-final as dg
composition-fri-roots final

then obtain *out query-idxs rest* **where**

outcome: *out* ∈ *set-dist* (*execute verify-monad s*)

and *bound:*

accepted-with-bound-tables s out trace-table composition-table as
query-idxs

and *header:*

```

    verifier-header-transcript s fr f-fri-roots f-final as dg
    composition-fri-roots final rest
unfolding query-header-supported-table-candidates-def by blast
from bound obtain result final-state fr' f-fri-roots' f-final' dg'
    composition-fri-roots' final' rest' where
    out-eq: out = Some (result, final-state)
and header':
    verifier-header-transcript s fr' f-fri-roots' f-final' as dg'
    composition-fri-roots' final' rest'
and trace-bind:
    merkle-root-binds-table fr' trace-table final-state
and comp-nonempty: composition-fri-roots' ≠ []
and comp-bind:
    merkle-root-binds-table (hd composition-fri-roots')
    composition-table final-state
unfolding accepted-with-bound-tables-def by blast
have eqs:
    fr' = fr ∧
    f-fri-roots' = f-fri-roots ∧
    f-final' = f-final ∧
    dg' = dg ∧
    composition-fri-roots' = composition-fri-roots ∧
    final' = final ∧
    rest' = rest
using verifier-header-transcript-unique[OF header' header] by simp
have state:
    query-header-supported-table-candidate-state s fr f-fri-roots f-final
    as dg composition-fri-roots final trace-table composition-table
    final-state
unfolding query-header-supported-table-candidate-state-def
by (intro exI[of - result] exI[of - query-idxs] exI[of - rest])
    (use outcome out-eq bound header trace-bind comp-nonempty comp-bind eqs
    in simp)
then show
    ∃ final-state.
    query-header-supported-table-candidate-state s fr f-fri-roots f-final
    as dg composition-fri-roots final trace-table composition-table
    final-state
by blast
next
assume
    ∃ final-state.
    query-header-supported-table-candidate-state s fr f-fri-roots f-final
    as dg composition-fri-roots final trace-table composition-table
    final-state
then obtain final-state result query-idxs rest where
    outcome:
    Some (result, final-state) ∈ set-dist (execute verify-monad s)
and bound:

```


accepted-with-bound-tables s (Some (result, final-state))
 trace-table composition-table as query-idxs
and header:
 verifier-header-transcript s fr f-fri-roots f-final as dg
 composition-fri-roots final rest
unfolding query-header-supported-table-candidate-state-def **by** blast
show (trace-table, composition-table) $\in$
 query-header-supported-table-candidates s fr f-fri-roots f-final as dg
 composition-fri-roots final
unfolding query-header-supported-table-candidates-def
using outcome bound header **by** blast
qed

lemma query-header-supported-table-candidate-state-hash-extends:
assumes
 query-header-supported-table-candidate-state s fr f-fri-roots f-final
 as dg composition-fri-roots final trace-table composition-table
 final-state
shows $s \leq$ final-state
proof –
from assms **obtain** result query-idxs rest **where**
 outcome:
 Some (result, final-state) $\in$ set-dist (execute verify-monad s)
unfolding query-header-supported-table-candidate-state-def **by** blast
show ?thesis
by (rule verify-monad-hash-extends[OF outcome])
qed

lemma query-header-supported-table-candidate-states-merkle-compatible-unique-if-clean-merge:
assumes compatible:
 merkle-hash-maps-compatible final-state final-state'
and clean:
 $\neg$ hash-map-output-collision
 (merkle-hash-state-merge final-state final-state')
and candidate:
 query-header-supported-table-candidate-state s fr f-fri-roots f-final
 as dg composition-fri-roots final trace-table composition-table
 final-state
and candidate':
 query-header-supported-table-candidate-state s fr f-fri-roots f-final
 as dg composition-fri-roots final trace-table' composition-table'
 final-state'
shows
 trace-table = trace-table' $\wedge$
 composition-table = composition-table'
proof –
from candidate **obtain** result query-idxs rest **where**
 bound:
 accepted-with-bound-tables s (Some (result, final-state))

```

    trace-table composition-table as query-idxs
and trace-bind:
    merkle-root-binds-table fr trace-table final-state
and comp-nonempty: composition-fri-roots ≠ []
and comp-bind:
    merkle-root-binds-table (hd composition-fri-roots) composition-table
    final-state
unfolding query-header-supported-table-candidate-state-def by blast
from candidate' obtain result' query-idxs' rest' where
bound':
    accepted-with-bound-tables s (Some (result', final-state'))
    trace-table' composition-table' as query-idxs'
and trace-bind':
    merkle-root-binds-table fr trace-table' final-state'
and comp-bind':
    merkle-root-binds-table (hd composition-fri-roots) composition-table'
    final-state'
unfolding query-header-supported-table-candidate-state-def by blast
have same-trace-len: length trace-table = length trace-table'
using accepted-with-bound-tables-shapes(1)[OF bound]
    accepted-with-bound-tables-shapes(1)[OF bound']
by simp
have trace-nonempty: trace-table ≠ []
using accepted-with-bound-tables-shapes(1)[OF bound]
    eval-domain-nontrivial by auto
have trace-eq: trace-table = trace-table'
by (rule
    merkle-root-binds-same-length-tables-merkle-compatible-unique-if-clean-merge
    [OF compatible clean trace-bind trace-bind' same-trace-len
    trace-nonempty])
have same-comp-len: length composition-table = length composition-table'
using accepted-with-bound-tables-shapes(2)[OF bound]
    accepted-with-bound-tables-shapes(2)[OF bound']
by simp
have comp-nonempty': composition-table ≠ []
using accepted-with-bound-tables-shapes(2)[OF bound]
    eval-domain-nontrivial by auto
have comp-eq: composition-table = composition-table'
by (rule
    merkle-root-binds-same-length-tables-merkle-compatible-unique-if-clean-merge
    [OF compatible clean comp-bind comp-bind' same-comp-len
    comp-nonempty'])
show ?thesis
using trace-eq comp-eq by simp
qed

lemma query-header-supported-table-candidates-subset-singleton-if-pairwise-merkle-clean-merge:
assumes pairwise:
     $\bigwedge$  trace-table composition-table final-state trace-table'

```

```

    composition-table' final-state'.
  query-header-supported-table-candidate-state s fr f-fri-roots f-final
    as dg composition-fri-roots final trace-table composition-table
    final-state  $\implies$ 
  query-header-supported-table-candidate-state s fr f-fri-roots f-final
    as dg composition-fri-roots final trace-table' composition-table'
    final-state'  $\implies$ 
  merkle-hash-maps-compatible final-state final-state'  $\wedge$ 
   $\neg$  hash-map-output-collision
    (merkle-hash-state-merge final-state final-state')
shows
   $\exists$  trace-table composition-table.
  query-header-supported-table-candidates s fr f-fri-roots f-final as dg
    composition-fri-roots final  $\subseteq$ 
    {(trace-table, composition-table)}
proof (cases
  query-header-supported-table-candidates s fr f-fri-roots f-final as dg
    composition-fri-roots final = {})
case True
then show ?thesis
  by (intro exI[of -] exI[of -]) simp
next
case False
then obtain table-pair0 where table-pair0:
  table-pair0  $\in$ 
  query-header-supported-table-candidates s fr f-fri-roots f-final as dg
    composition-fri-roots final
  by blast
obtain trace-table0 composition-table0 where table-pair0-eq:
  table-pair0 = (trace-table0, composition-table0)
  by (cases table-pair0)
have table0:
  (trace-table0, composition-table0)  $\in$ 
  query-header-supported-table-candidates s fr f-fri-roots f-final as dg
    composition-fri-roots final
  using table-pair0 unfolding table-pair0-eq .
then obtain final-state0 where state0:
  query-header-supported-table-candidate-state s fr f-fri-roots f-final
    as dg composition-fri-roots final trace-table0 composition-table0
    final-state0
  using query-header-supported-table-candidates-iff-state by blast
have subset:
  query-header-supported-table-candidates s fr f-fri-roots f-final as dg
    composition-fri-roots final  $\subseteq$ 
    {(trace-table0, composition-table0)}
proof
  fix table-pair
  assume pair-in:
    table-pair  $\in$ 

```

```

    query-header-supported-table-candidates s fr f-fri-roots f-final as
    dg composition-fri-roots final
obtain trace-table composition-table where pair-eq:
    table-pair = (trace-table, composition-table)
by (cases table-pair)
from pair-in obtain final-state where state:
    query-header-supported-table-candidate-state s fr f-fri-roots f-final
    as dg composition-fri-roots final trace-table composition-table
    final-state
unfolding pair-eq
using query-header-supported-table-candidates-iff-state by blast
have compatible:
    merkle-hash-maps-compatible final-state final-state0
using pairwise[OF state state0] by simp
have clean:
    ¬ hash-map-output-collision
    (merkle-hash-state-merge final-state final-state0)
using pairwise[OF state state0] by simp
have eqs:
    trace-table = trace-table0 ∧
    composition-table = composition-table0
by (rule
    query-header-supported-table-candidate-states-merkle-compatible-unique-if-clean-merge
    [OF compatible clean state state0])
show table-pair ∈ {(trace-table0, composition-table0)}
using pair-eq eqs by simp
qed
then show ?thesis
by blast
qed

lemma query-header-supported-table-candidates-subset-singleton-if-pairwise-no-merkle-value-conflict-clean-merge:
assumes pairwise:
    ∧ trace-table composition-table final-state trace-table'
    composition-table' final-state'.
    query-header-supported-table-candidate-state s fr f-fri-roots f-final
    as dg composition-fri-roots final trace-table composition-table
    final-state ⇒
    query-header-supported-table-candidate-state s fr f-fri-roots f-final
    as dg composition-fri-roots final trace-table' composition-table'
    final-state' ⇒
    ¬ merkle-hash-value-conflict final-state final-state' ∧
    ¬ hash-map-output-collision
    (merkle-hash-state-merge final-state final-state')
shows
    ∃ trace-table composition-table.
    query-header-supported-table-candidates s fr f-fri-roots f-final as dg
    composition-fri-roots final ⊆
    {(trace-table, composition-table)}

```

```

proof (rule
  query-header-supported-table-candidates-subset-singleton-if-pairwise-merkle-clean-merge)
fix trace-table composition-table final-state trace-table'
  composition-table' final-state'
assume candidate:
  query-header-supported-table-candidate-state s fr f-fri-roots f-final
  as dg composition-fri-roots final trace-table composition-table
  final-state
and candidate':
  query-header-supported-table-candidate-state s fr f-fri-roots f-final
  as dg composition-fri-roots final trace-table' composition-table'
  final-state'
have no-conflict:
  ¬ merkle-hash-value-conflict final-state final-state'
  using pairwise[OF candidate candidate'] by simp
have compatible:
  merkle-hash-maps-compatible final-state final-state'
  using no-conflict
  unfolding merkle-hash-maps-compatible-iff-no-value-conflict .
have clean:
  ¬ hash-map-output-collision
  (merkle-hash-state-merge final-state final-state')
  using pairwise[OF candidate candidate'] by simp
show merkle-hash-maps-compatible final-state final-state' ∧
  ¬ hash-map-output-collision
  (merkle-hash-state-merge final-state final-state')
  using compatible clean by simp
qed

lemma query-header-supported-table-candidates-subset-singleton-if-common-clean-merkle-extension:
assumes clean: ¬ hash-map-output-collision u
and common-ext:
  ∧ trace-table composition-table final-state.
  query-header-supported-table-candidate-state s fr f-fri-roots f-final
  as dg composition-fri-roots final trace-table composition-table
  final-state ⇒
  merkle-hash-extends final-state u
shows
  ∃ trace-table composition-table.
  query-header-supported-table-candidates s fr f-fri-roots f-final as dg
  composition-fri-roots final ⊆
  {(trace-table, composition-table)}
proof (rule
  query-header-supported-table-candidates-subset-singleton-if-pairwise-no-merkle-value-conflict-clean-merge)
fix trace-table composition-table final-state trace-table'
  composition-table' final-state'
assume candidate:
  query-header-supported-table-candidate-state s fr f-fri-roots f-final
  as dg composition-fri-roots final trace-table composition-table

```

```

    final-state
  and candidate':
    query-header-supported-table-candidate-state s fr f-fri-roots f-final
    as dg composition-fri-roots final trace-table' composition-table'
    final-state'
  have ext: merkle-hash-extends final-state u
  by (rule common-ext[OF candidate])
  have ext': merkle-hash-extends final-state' u
  by (rule common-ext[OF candidate'])
  show  $\neg$  merkle-hash-value-conflict final-state final-state'  $\wedge$ 
     $\neg$  hash-map-output-collision
    (merkle-hash-state-merge final-state final-state')
  by (rule common-clean-merkle-extension-imp-pairwise-merkle-clean
    [OF ext ext' clean])
qed

lemma query-header-supported-table-candidate-states-distinct-imp-merkle-conflict-or-merge-collision:
  assumes candidate:
    query-header-supported-table-candidate-state s fr f-fri-roots f-final
    as dg composition-fri-roots final trace-table composition-table
    final-state
  and candidate':
    query-header-supported-table-candidate-state s fr f-fri-roots f-final
    as dg composition-fri-roots final trace-table' composition-table'
    final-state'
  and distinct-tables:
    trace-table  $\neq$  trace-table'  $\vee$ 
    composition-table  $\neq$  composition-table'
  shows
    merkle-hash-value-conflict final-state final-state'  $\vee$ 
    hash-map-output-collision
    (merkle-hash-state-merge final-state final-state')
proof (rule ccontr)
  assume no-bad:
     $\neg$  (merkle-hash-value-conflict final-state final-state'  $\vee$ 
      hash-map-output-collision
      (merkle-hash-state-merge final-state final-state'))
  have compatible:
    merkle-hash-maps-compatible final-state final-state'
  using no-bad unfolding merkle-hash-maps-compatible-iff-no-value-conflict
  by simp
  have clean:
     $\neg$  hash-map-output-collision
    (merkle-hash-state-merge final-state final-state')
  using no-bad by simp
  have eqs:
    trace-table = trace-table'  $\wedge$ 
    composition-table = composition-table'
  by (rule

```

$query_header_supported_table_candidate_states_merkle_compatible_unique_if_clean_merge$
 $[OF\ compatible\ clean\ candidate\ candidate']$
then show *False*
using *distinct-tables* **by** *simp*
qed

definition *query-header-supported-pairwise-merkle-bad*

$:: (f, 'a)\ protocol_channel_scheme \Rightarrow$
 $'f \Rightarrow 'f\ list \Rightarrow 'f \Rightarrow 'f\ list \Rightarrow 'f \Rightarrow$
 $'f\ list \Rightarrow 'f \Rightarrow bool$

where

$query_header_supported_pairwise_merkle_bad\ s\ fr\ f_fri_roots\ f_final\ as$
 $dg\ composition_fri_roots\ final \longleftrightarrow$
 $(\exists\ trace_table\ composition_table\ final_state\ trace_table'$
 $\quad composition_table'\ final_state'.$
 $query_header_supported_table_candidate_state\ s\ fr\ f_fri_roots\ f_final$
 $\quad as\ dg\ composition_fri_roots\ final\ trace_table\ composition_table$
 $\quad final_state \wedge$
 $query_header_supported_table_candidate_state\ s\ fr\ f_fri_roots\ f_final$
 $\quad as\ dg\ composition_fri_roots\ final\ trace_table'\ composition_table'$
 $\quad final_state' \wedge$
 $(trace_table \neq trace_table' \vee$
 $\quad composition_table \neq composition_table') \wedge$
 $(merkle_hash_value_conflict\ final_state\ final_state' \vee$
 $\quad hash_map_output_collision$
 $\quad (merkle_hash_state_merge\ final_state\ final_state'))))$

definition *query-supported-pairwise-merkle-bad*

$:: (f, 'a)\ protocol_channel_scheme \Rightarrow bool$

where

$query_supported_pairwise_merkle_bad\ s \longleftrightarrow$
 $(\exists\ fr\ f_fri_roots\ f_final\ as\ dg\ composition_fri_roots\ final.$
 $\quad query_header_supported_pairwise_merkle_bad\ s\ fr\ f_fri_roots\ f_final$
 $\quad as\ dg\ composition_fri_roots\ final)$

lemma *query-header-supported-table-candidates-not-singleton-imp-pairwise-merkle-bad:*

assumes *not-unique:*

$\neg (\exists\ trace_table\ composition_table.$
 $\quad query_header_supported_table_candidates\ s\ fr\ f_fri_roots\ f_final\ as\ dg$
 $\quad composition_fri_roots\ final \subseteq$
 $\quad \{(trace_table, composition_table)\})$

shows

$\exists\ trace_table\ composition_table\ final_state\ trace_table'$
 $\quad composition_table'\ final_state'.$
 $query_header_supported_table_candidate_state\ s\ fr\ f_fri_roots\ f_final$
 $\quad as\ dg\ composition_fri_roots\ final\ trace_table\ composition_table$
 $\quad final_state \wedge$
 $query_header_supported_table_candidate_state\ s\ fr\ f_fri_roots\ f_final$
 $\quad as\ dg\ composition_fri_roots\ final\ trace_table'\ composition_table'$

```

    final-state'  $\wedge$ 
    (trace-table  $\neq$  trace-table'  $\vee$ 
     composition-table  $\neq$  composition-table')  $\wedge$ 
    (merkle-hash-value-conflict final-state final-state'  $\vee$ 
     hash-map-output-collision
     (merkle-hash-state-merge final-state final-state'))
proof -
let ?C =
  query-header-supported-table-candidates s fr f-fri-roots f-final as dg
  composition-fri-roots final
have nonempty: ?C  $\neq$  {}
  using not-unique by auto
then obtain table-pair where table-pair:
  table-pair  $\in$  ?C
  by blast
obtain trace-table composition-table where table-pair-eq:
  table-pair = (trace-table, composition-table)
  by (cases table-pair)
have table:
  (trace-table, composition-table)  $\in$  ?C
  using table-pair unfolding table-pair-eq .
have  $\neg$  ?C  $\subseteq$  {(trace-table, composition-table)}
  using not-unique by blast
then obtain table-pair' where table-pair':
  table-pair'  $\in$  ?C
  and table-pair'-not:
    table-pair'  $\notin$  {(trace-table, composition-table)}
  by auto
obtain trace-table' composition-table' where table-pair'-eq:
  table-pair' = (trace-table', composition-table')
  by (cases table-pair')
have table':
  (trace-table', composition-table')  $\in$  ?C
  using table-pair' unfolding table-pair'-eq .
have distinct-tables:
  trace-table  $\neq$  trace-table'  $\vee$ 
  composition-table  $\neq$  composition-table'
  using table-pair'-not unfolding table-pair'-eq by auto
from table obtain final-state where candidate:
  query-header-supported-table-candidate-state s fr f-fri-roots f-final
  as dg composition-fri-roots final trace-table composition-table
  final-state
  using query-header-supported-table-candidates-iff-state by blast
from table' obtain final-state' where candidate':
  query-header-supported-table-candidate-state s fr f-fri-roots f-final
  as dg composition-fri-roots final trace-table' composition-table'
  final-state'
  using query-header-supported-table-candidates-iff-state by blast
have bad:

```


$\text{merkle-hash-value-conflict } \text{final-state } \text{final-state}' \vee$
 $\text{hash-map-output-collision}$
 $(\text{merkle-hash-state-merge } \text{final-state } \text{final-state}')$
by (rule
 $\text{query-header-supported-table-candidate-states-distinct-imp-merkle-conflict-or-merge-collision}$
 $[\text{OF candidate candidate}' \text{ distinct-tables}])$
show ?thesis
using candidate candidate' distinct-tables bad **by** blast
qed

lemma query-header-supported-pairwise-merkle-bad-extends-initial:
assumes bad:
 $\text{query-header-supported-pairwise-merkle-bad } s \text{ fr } f\text{-fri-roots } f\text{-final as}$
 $\text{dg composition-fri-roots final}$
obtains trace-table composition-table final-state trace-table'
 $\text{composition-table}' \text{ final-state}'$
where
 $\text{query-header-supported-table-candidate-state } s \text{ fr } f\text{-fri-roots } f\text{-final}$
 $\text{as dg composition-fri-roots final trace-table composition-table}$
 final-state
 $\text{query-header-supported-table-candidate-state } s \text{ fr } f\text{-fri-roots } f\text{-final}$
 $\text{as dg composition-fri-roots final trace-table}' \text{ composition-table}'$
 $\text{final-state}'$
 $\text{trace-table} \neq \text{trace-table}' \vee$
 $\text{composition-table} \neq \text{composition-table}'$
 $\text{merkle-hash-value-conflict } \text{final-state } \text{final-state}' \vee$
 $\text{hash-map-output-collision}$
 $(\text{merkle-hash-state-merge } \text{final-state } \text{final-state}')$
 $s \leq \text{final-state}$
 $s \leq \text{final-state}'$

proof –
from bad **obtain** trace-table composition-table final-state trace-table'
 $\text{composition-table}' \text{ final-state}'$ **where**
candidate:
 $\text{query-header-supported-table-candidate-state } s \text{ fr } f\text{-fri-roots } f\text{-final}$
 $\text{as dg composition-fri-roots final trace-table composition-table}$
 final-state
and candidate':
 $\text{query-header-supported-table-candidate-state } s \text{ fr } f\text{-fri-roots } f\text{-final}$
 $\text{as dg composition-fri-roots final trace-table}' \text{ composition-table}'$
 $\text{final-state}'$
and distinct:
 $\text{trace-table} \neq \text{trace-table}' \vee$
 $\text{composition-table} \neq \text{composition-table}'$
and pair-bad:
 $\text{merkle-hash-value-conflict } \text{final-state } \text{final-state}' \vee$
 $\text{hash-map-output-collision}$
 $(\text{merkle-hash-state-merge } \text{final-state } \text{final-state}')$
unfolding query-header-supported-pairwise-merkle-bad-def **by** blast

have $ext: s \leq final-state$
by (rule *query-header-supported-table-candidate-state-hash-extends*
 $[OF\ candidate]$)
have $ext': s \leq final-state'$
by (rule *query-header-supported-table-candidate-state-hash-extends*
 $[OF\ candidate']$)
show *?thesis*
by (rule *that* $[OF\ candidate\ candidate'\ distinct\ pair-bad\ ext\ ext']$)
qed

definition *query-header-supported-pairwise-coupling-bad*

$:: (f, 'a)\ protocol-channel-scheme \Rightarrow$
 $'f \Rightarrow 'f\ list \Rightarrow 'f \Rightarrow 'f\ list \Rightarrow 'f \Rightarrow$
 $'f\ list \Rightarrow 'f \Rightarrow bool$

where

query-header-supported-pairwise-coupling-bad $s\ fr\ f-fri-roots\ f-final\ as$
 $dg\ composition-fri-roots\ final \longleftrightarrow$
 $(\exists\ trace-table\ composition-table\ final-state\ trace-table'$
 $composition-table'\ final-state'.$
 $query-header-supported-table-candidate-state\ s\ fr\ f-fri-roots\ f-final$
 $as\ dg\ composition-fri-roots\ final\ trace-table\ composition-table$
 $final-state \wedge$
 $query-header-supported-table-candidate-state\ s\ fr\ f-fri-roots\ f-final$
 $as\ dg\ composition-fri-roots\ final\ trace-table'\ composition-table'$
 $final-state' \wedge$
 $(trace-table \neq trace-table' \vee$
 $composition-table \neq composition-table') \wedge$
 $merkle-hash-pairwise-coupling-bad\ final-state\ final-state')$

definition *query-supported-pairwise-coupling-bad*

$:: (f, 'a)\ protocol-channel-scheme \Rightarrow bool$

where

query-supported-pairwise-coupling-bad $s \longleftrightarrow$
 $(\exists\ fr\ f-fri-roots\ f-final\ as\ dg\ composition-fri-roots\ final.$
 $query-header-supported-pairwise-coupling-bad\ s\ fr\ f-fri-roots\ f-final$
 $as\ dg\ composition-fri-roots\ final)$

lemma *query-header-supported-pairwise-merkle-bad-imp-local-collision-or-coupling-bad:*

assumes *bad:*

$query-header-supported-pairwise-merkle-bad\ s\ fr\ f-fri-roots\ f-final\ as$
 $dg\ composition-fri-roots\ final$

shows

$(\exists\ trace-table\ composition-table\ final-state\ trace-table'$
 $composition-table'\ final-state'.$
 $query-header-supported-table-candidate-state\ s\ fr\ f-fri-roots\ f-final$
 $as\ dg\ composition-fri-roots\ final\ trace-table\ composition-table$
 $final-state \wedge$
 $query-header-supported-table-candidate-state\ s\ fr\ f-fri-roots\ f-final$
 $as\ dg\ composition-fri-roots\ final\ trace-table'\ composition-table')$

$final-state' \wedge$
 $(trace-table \neq trace-table' \vee$
 $composition-table \neq composition-table') \wedge$
 $(hash-map-output-collision\ final-state \vee$
 $hash-map-output-collision\ final-state')) \vee$
 $query-header-supported-pairwise-coupling-bad\ s\ fr\ f-fri-roots\ f-final\ as$
 $dg\ composition-fri-roots\ final$
proof –
from *bad* **obtain** $trace-table\ composition-table\ final-state\ trace-table'$
 $composition-table'\ final-state'$
where *candidate*:
 $query-header-supported-table-candidate-state\ s\ fr\ f-fri-roots\ f-final$
 $as\ dg\ composition-fri-roots\ final\ trace-table\ composition-table$
 $final-state$
and *candidate'*:
 $query-header-supported-table-candidate-state\ s\ fr\ f-fri-roots\ f-final$
 $as\ dg\ composition-fri-roots\ final\ trace-table'\ composition-table'$
 $final-state'$
and *distinct*:
 $trace-table \neq trace-table' \vee$
 $composition-table \neq composition-table'$
and *pair-bad*:
 $merkle-hash-value-conflict\ final-state\ final-state' \vee$
 $hash-map-output-collision$
 $(merkle-hash-state-merge\ final-state\ final-state')$
unfolding *query-header-supported-pairwise-merkle-bad-def* **by** *blast*
have *decomp*:
 $hash-map-output-collision\ final-state \vee$
 $hash-map-output-collision\ final-state' \vee$
 $merkle-hash-pairwise-coupling-bad\ final-state\ final-state'$
by (*rule merkle-hash-pairwise-bad-imp-local-collision-or-coupling-bad*
 $[OF\ pair-bad]$)
show *?thesis*
using *candidate candidate' distinct decomp*
unfolding *query-header-supported-pairwise-coupling-bad-def* **by** *blast*
qed

lemma *query-supported-pairwise-merkle-bad-imp-local-collision-or-coupling-bad*:
assumes *bad*: *query-supported-pairwise-merkle-bad s*
shows
 $(\exists\ fr\ f-fri-roots\ f-final\ as\ dg\ composition-fri-roots\ final$
 $trace-table\ composition-table\ final-state\ trace-table'$
 $composition-table'\ final-state'.$
 $query-header-supported-table-candidate-state\ s\ fr\ f-fri-roots\ f-final$
 $as\ dg\ composition-fri-roots\ final\ trace-table\ composition-table$
 $final-state \wedge$
 $query-header-supported-table-candidate-state\ s\ fr\ f-fri-roots\ f-final$
 $as\ dg\ composition-fri-roots\ final\ trace-table'\ composition-table'$
 $final-state' \wedge$

$(\text{trace-table} \neq \text{trace-table}' \vee$
 $\text{composition-table} \neq \text{composition-table}') \wedge$
 $(\text{hash-map-output-collision final-state} \vee$
 $\text{hash-map-output-collision final-state}')) \vee$
 $\text{query-supported-pairwise-coupling-bad } s$

proof –

from *bad* **obtain** *fr f-fri-roots f-final as dg composition-fri-roots final*

where *header-bad*:

query-header-supported-pairwise-merkle-bad s fr f-fri-roots f-final
as dg composition-fri-roots final

unfolding *query-supported-pairwise-merkle-bad-def* **by** *blast*

from

query-header-supported-pairwise-merkle-bad-imp-local-collision-or-coupling-bad
 $[OF \text{ header-bad}]$

show *?thesis*

unfolding *query-supported-pairwise-coupling-bad-def* **by** *blast*

qed

lemma *query-supported-pairwise-merkle-bad-imp-supported-hash-collision-possible-or-coupling-bad*:

assumes *bad*: *query-supported-pairwise-merkle-bad s*

shows

supported-hash-output-collision-possible s $\vee$
query-supported-pairwise-coupling-bad s

proof –

have *decomp*:

$(\exists \text{ fr f-fri-roots f-final as dg composition-fri-roots final}$
 $\text{trace-table composition-table final-state trace-table'}$
 $\text{composition-table' final-state'}.)$

query-header-supported-table-candidate-state s fr f-fri-roots f-final
as dg composition-fri-roots final trace-table composition-table
final-state $\wedge$

query-header-supported-table-candidate-state s fr f-fri-roots f-final
as dg composition-fri-roots final trace-table' composition-table'
final-state' $\wedge$

$(\text{trace-table} \neq \text{trace-table}' \vee$
 $\text{composition-table} \neq \text{composition-table}') \wedge$
 $(\text{hash-map-output-collision final-state} \vee$
 $\text{hash-map-output-collision final-state}')) \vee$
 $\text{query-supported-pairwise-coupling-bad } s$

by $(\text{rule } \text{query-supported-pairwise-merkle-bad-imp-local-collision-or-coupling-bad}$
 $[OF \text{ bad}])$

then show *?thesis*

proof

assume *local*:

$\exists \text{ fr f-fri-roots f-final as dg composition-fri-roots final}$
 $\text{trace-table composition-table final-state trace-table'}$
 $\text{composition-table' final-state'}.)$

query-header-supported-table-candidate-state s fr f-fri-roots f-final
as dg composition-fri-roots final trace-table composition-table

```

    final-state  $\wedge$ 
    query-header-supported-table-candidate-state  $s$  fr  $f$ -fri-roots  $f$ -final
      as dg composition-fri-roots final trace-table' composition-table'
      final-state'  $\wedge$ 
      (trace-table  $\neq$  trace-table'  $\vee$ 
       composition-table  $\neq$  composition-table')  $\wedge$ 
      (hash-map-output-collision final-state  $\vee$ 
       hash-map-output-collision final-state')
  then obtain fr  $f$ -fri-roots  $f$ -final as dg composition-fri-roots final
    trace-table composition-table final-state trace-table'
    composition-table' final-state' where
  candidate:
    query-header-supported-table-candidate-state  $s$  fr  $f$ -fri-roots  $f$ -final
      as dg composition-fri-roots final trace-table composition-table
      final-state
  and candidate':
    query-header-supported-table-candidate-state  $s$  fr  $f$ -fri-roots  $f$ -final
      as dg composition-fri-roots final trace-table' composition-table'
      final-state'
  and collision:
    hash-map-output-collision final-state  $\vee$ 
    hash-map-output-collision final-state'
  by blast
from collision show ?thesis
proof
  assume collision-final: hash-map-output-collision final-state
  from candidate obtain result query-idxs rest where
    outcome:
      Some (result, final-state)  $\in$  set-dist (execute verify-monad  $s$ )
  and bound:
    accepted-with-bound-tables  $s$  (Some (result, final-state))
    trace-table composition-table as query-idxs
  unfolding query-header-supported-table-candidate-state-def by blast
have supported-hash-output-collision-possible  $s$ 
  by (rule
      accepted-with-bound-tables-hash-collision-imp-supported-hash-output-collision-possible
      [OF outcome bound collision-final])
then show ?thesis by simp
next
  assume collision-final': hash-map-output-collision final-state'
  from candidate' obtain result' query-idxs' rest' where
    outcome':
      Some (result', final-state')  $\in$  set-dist (execute verify-monad  $s$ )
  and bound':
    accepted-with-bound-tables  $s$  (Some (result', final-state'))
    trace-table' composition-table' as query-idxs'
  unfolding query-header-supported-table-candidate-state-def by blast
have supported-hash-output-collision-possible  $s$ 
  by (rule

```

$\text{accepted-with-bound-tables-hash-collision-imp-supported-hash-output-collision-possible}$
 $[OF \text{ outcome}' \text{ bound}' \text{ collision-final}]$)
then show $?thesis$ **by** $simp$
qed
next
assume $query\text{-supported-pairwise-coupling-bad } s$
then show $?thesis$ **by** $simp$
qed
qed

lemma $query\text{-header-supported-candidate-unique-if-no-pairwise-merkle-bad}$:
assumes $no\text{-bad}$:
 $\neg query\text{-header-supported-pairwise-merkle-bad } s \text{ fr } f\text{-fri-roots } f\text{-final}$
 $as dg \text{ composition-fri-roots final}$
shows
 $\exists trace\text{-table } composition\text{-table.}$
 $query\text{-header-supported-table-candidates } s \text{ fr } f\text{-fri-roots } f\text{-final } as dg$
 $composition\text{-fri-roots final} \subseteq$
 $\{(trace\text{-table}, composition\text{-table})\}$
proof (rule $ccontr$)
assume $not\text{-unique}$:
 $\neg (\exists trace\text{-table } composition\text{-table.}$
 $query\text{-header-supported-table-candidates } s \text{ fr } f\text{-fri-roots } f\text{-final } as dg$
 $composition\text{-fri-roots final} \subseteq$
 $\{(trace\text{-table}, composition\text{-table})\})$
have $query\text{-header-supported-pairwise-merkle-bad } s \text{ fr } f\text{-fri-roots } f\text{-final}$
 $as dg \text{ composition-fri-roots final}$
using
 $query\text{-header-supported-table-candidates-not-singleton-imp-pairwise-merkle-bad}$
 $[OF not\text{-unique}]$
unfolding $query\text{-header-supported-pairwise-merkle-bad-def}$
by $blast$
then show $False$
using $no\text{-bad}$ **by** $contradiction$
qed

lemma $query\text{-header-supported-candidate-unique-if-no-global-pairwise-merkle-bad}$:
assumes $no\text{-bad}$: $\neg query\text{-supported-pairwise-merkle-bad } s$
shows
 $\exists trace\text{-table } composition\text{-table.}$
 $query\text{-header-supported-table-candidates } s \text{ fr } f\text{-fri-roots } f\text{-final } as dg$
 $composition\text{-fri-roots final} \subseteq$
 $\{(trace\text{-table}, composition\text{-table})\}$
proof (rule $query\text{-header-supported-candidate-unique-if-no-pairwise-merkle-bad}$)
show $\neg query\text{-header-supported-pairwise-merkle-bad } s \text{ fr } f\text{-fri-roots}$
 $f\text{-final } as dg \text{ composition-fri-roots final}$
using $no\text{-bad}$ **unfolding** $query\text{-supported-pairwise-merkle-bad-def}$ **by** $blast$
qed

definition *query-header-supported-union-good-sets*

$:: (f, 'a) \text{ protocol-channel-scheme} \Rightarrow$
 $(f \text{ list} \Rightarrow f \text{ list} \Rightarrow f \text{ list} \Rightarrow \text{nat set}) \Rightarrow$
 $f \Rightarrow f \text{ list} \Rightarrow f \Rightarrow f \text{ list} \Rightarrow f \Rightarrow$
 $f \text{ list} \Rightarrow f \Rightarrow \text{nat set}$

where

query-header-supported-union-good-sets s good-sets fr f-fri-roots
f-final as dg composition-fri-roots final =
 $\{idx \in \text{query-sample-space}.$
 $\exists \text{ trace-table composition-table}.$
 $(\text{trace-table}, \text{composition-table}) \in$
 $\text{query-header-supported-table-candidates } s \text{ fr } f\text{-fri-roots } f\text{-final}$
 $\text{ as } dg \text{ composition-fri-roots final} \wedge$
 $idx \in \text{good-sets trace-table composition-table as}\}$

lemma *query-header-supported-union-good-sets-subset:*

query-header-supported-union-good-sets s good-sets fr f-fri-roots
f-final as dg composition-fri-roots final $\subseteq$ query-sample-space

unfolding *query-header-supported-union-good-sets-def* **by** *auto*

definition *query-header-rounds-any-index-set-hit*

$:: (f, 'a) \text{ protocol-channel-scheme} \Rightarrow$
 $(f \Rightarrow f \text{ list} \Rightarrow f \Rightarrow f \text{ list} \Rightarrow f \Rightarrow$
 $f \text{ list} \Rightarrow f \Rightarrow \text{nat set}) \Rightarrow$
 $(\text{unit list} \times (f, 'a) \text{ protocol-channel-scheme}) \text{ option} \Rightarrow \text{bool}$

where

query-header-rounds-any-index-set-hit s B out $\longleftrightarrow$
 $(\exists \text{ fr } f\text{-fri-roots } f\text{-final as } dg \text{ composition-fri-roots final rest}.$
 $\text{verifier-header-transcript } s \text{ fr } f\text{-fri-roots } f\text{-final as } dg$
 $\text{composition-fri-roots final rest} \wedge$
 $\text{query-rounds-any-index-set-hit } s$
 $(B \text{ fr } f\text{-fri-roots } f\text{-final as } dg \text{ composition-fri-roots final})$
 $\text{rounds out})$

lemma *query-header-supported-union-good-sets-fraction-bound-if-unique-candidate:*

assumes *unique:*

$\exists \text{ trace-table composition-table}.$
 $\text{query-header-supported-table-candidates } s \text{ fr } f\text{-fri-roots } f\text{-final as}$
 $dg \text{ composition-fri-roots final} \subseteq$
 $\{(\text{trace-table}, \text{composition-table})\}$

and *subset:*

$\bigwedge \text{ trace-table composition-table as}.$
 $\text{good-sets trace-table composition-table as} \subseteq \text{query-sample-space}$

and *bound:*

$\bigwedge \text{ trace-table composition-table as}.$
 $\text{nnreal } (\text{card } (\text{good-sets trace-table composition-table as})) /$
 $\text{nnreal } (\text{card } \text{query-sample-space}) \leq \text{query-error-bound}$

shows

nnreal

```

    (card
      (query-header-supported-union-good-sets s good-sets fr f-fri-roots
        f-final as dg composition-fri-roots final)) /
    nnreal (card query-sample-space) ≤ query-error-bound
  proof -
    from unique obtain trace-table composition-table where candidates:
      query-header-supported-table-candidates s fr f-fri-roots f-final as dg
      composition-fri-roots final ⊆
      {(trace-table, composition-table)}
    by blast
  have union-subset:
    query-header-supported-union-good-sets s good-sets fr f-fri-roots
      f-final as dg composition-fri-roots final ⊆
      good-sets trace-table composition-table as
  unfolding query-header-supported-union-good-sets-def
  using candidates by auto
  have finite-good: finite (good-sets trace-table composition-table as)
  by (rule finite-subset[OF subset finite-query-sample-space])
  have card-le:
    card
      (query-header-supported-union-good-sets s good-sets fr f-fri-roots
        f-final as dg composition-fri-roots final) ≤
    card (good-sets trace-table composition-table as)
  by (rule card-mono[OF finite-good union-subset])
  have nnreal
    (card
      (query-header-supported-union-good-sets s good-sets fr f-fri-roots
        f-final as dg composition-fri-roots final)) /
    nnreal (card query-sample-space) ≤
    nnreal (card (good-sets trace-table composition-table as)) /
    nnreal (card query-sample-space)
  by (rule nnreal-nat-divide-right-mono[OF card-le])
  also have ... ≤ query-error-bound
  by (rule bound)
  finally show ?thesis .
qed

lemma query-header-supported-union-good-sets-fraction-bound-if-no-pairwise-merkle-bad:
  assumes no-bad:
    ¬ query-header-supported-pairwise-merkle-bad s fr f-fri-roots
      f-final as dg composition-fri-roots final
  and subset:
    ∧ trace-table composition-table as.
      good-sets trace-table composition-table as ⊆ query-sample-space
  and bound:
    ∧ trace-table composition-table as.
      nnreal (card (good-sets trace-table composition-table as)) /
      nnreal (card query-sample-space) ≤ query-error-bound
  shows

```


$nnreal$
 $(card$
 $(query\text{-}header\text{-}supported\text{-}union\text{-}good\text{-}sets\ s\ good\text{-}sets\ fr\ f\text{-}fri\text{-}roots$
 $f\text{-}final\ as\ dg\ composition\text{-}fri\text{-}roots\ final))\ /\$
 $nnreal\ (card\ query\text{-}sample\text{-}space) \leq query\text{-}error\text{-}bound$

proof –

have *unique*:

$\exists\ trace\text{-}table\ composition\text{-}table.$
 $query\text{-}header\text{-}supported\text{-}table\text{-}candidates\ s\ fr\ f\text{-}fri\text{-}roots\ f\text{-}final\ as\ dg$
 $composition\text{-}fri\text{-}roots\ final \subseteq$
 $\{(trace\text{-}table, composition\text{-}table)\}$

by (rule *query-header-supported-candidate-unique-if-no-pairwise-merkle-bad*
 $[OF\ no\text{-}bad]$)

show *?thesis*

by (rule *query-header-supported-union-good-sets-fraction-bound-if-unique-candidate*
 $[OF\ unique\ subset\ bound]$)

qed

lemma *query-header-supported-union-good-sets-fraction-bound-if-no-global-pairwise-merkle-bad*:

assumes *no-bad*: $\neg query\text{-}supported\text{-}pairwise\text{-}merkle\text{-}bad\ s$

and *subset*:

$\bigwedge trace\text{-}table\ composition\text{-}table\ as.$
 $good\text{-}sets\ trace\text{-}table\ composition\text{-}table\ as \subseteq query\text{-}sample\text{-}space$

and *bound*:

$\bigwedge trace\text{-}table\ composition\text{-}table\ as.$
 $nnreal\ (card\ (good\text{-}sets\ trace\text{-}table\ composition\text{-}table\ as))\ /\$
 $nnreal\ (card\ query\text{-}sample\text{-}space) \leq query\text{-}error\text{-}bound$

shows

$nnreal$
 $(card$
 $(query\text{-}header\text{-}supported\text{-}union\text{-}good\text{-}sets\ s\ good\text{-}sets\ fr\ f\text{-}fri\text{-}roots$
 $f\text{-}final\ as\ dg\ composition\text{-}fri\text{-}roots\ final))\ /\$
 $nnreal\ (card\ query\text{-}sample\text{-}space) \leq query\text{-}error\text{-}bound$

proof –

have *no-header-bad*:

$\neg query\text{-}header\text{-}supported\text{-}pairwise\text{-}merkle\text{-}bad\ s\ fr\ f\text{-}fri\text{-}roots$
 $f\text{-}final\ as\ dg\ composition\text{-}fri\text{-}roots\ final$

using *no-bad* **unfolding** *query-supported-pairwise-merkle-bad-def* **by** *blast*

show *?thesis*

by (rule
 $query\text{-}header\text{-}supported\text{-}union\text{-}good\text{-}sets\text{-}fraction\text{-}bound\text{-}if\text{-}no\text{-}pairwise\text{-}merkle\text{-}bad$
 $[OF\ no\text{-}header\text{-}bad\ subset\ bound]$)

qed

definition *query-header-committed-table-candidates*

$:: (f, 'a)\ protocol\text{-}channel\text{-}scheme \Rightarrow$
 $f \Rightarrow f\ list \Rightarrow (f\ list \times f\ list)\ set$

where

$query\text{-}header\text{-}committed\text{-}table\text{-}candidates\ s\ fr\ composition\text{-}fri\text{-}roots =$

$\{(trace-table, composition-table).$
 $merkle-root-binds-table \text{ fr } trace-table \ s \wedge$
 $composition-fri-roots \neq [] \wedge$
 $merkle-root-binds-table (hd \ composition-fri-roots) \ composition-table$
 $s\}$

definition *query-header-committed-union-good-sets*

$:: (f, 'a) \text{ protocol-channel-scheme} \Rightarrow$
 $(f \text{ list} \Rightarrow f \text{ list} \Rightarrow f \text{ list} \Rightarrow \text{nat set}) \Rightarrow$
 $f \Rightarrow f \text{ list} \Rightarrow f \text{ list} \Rightarrow \text{nat set}$

where

query-header-committed-union-good-sets s *good-sets* *fr* *as*
 $composition-fri-roots =$
 $\{idx \in query-sample-space.$
 $\exists trace-table \ composition-table.$
 $(trace-table, composition-table) \in$
 $query-header-committed-table-candidates \ s \text{ fr}$
 $composition-fri-roots \wedge$
 $idx \in good-sets \ trace-table \ composition-table \ as\}$

lemma *query-header-committed-union-good-sets-subset:*

query-header-committed-union-good-sets s *good-sets* *fr* *as*
 $composition-fri-roots \subseteq query-sample-space$

unfolding *query-header-committed-union-good-sets-def* **by** *auto*

lemma *query-header-committed-union-good-sets-fraction-bound-if-unique-candidate:*

assumes *unique:*

$\exists trace-table \ composition-table.$
 $query-header-committed-table-candidates \ s \text{ fr } composition-fri-roots$
 $\subseteq \{(trace-table, composition-table)\}$

and *subset:*

$\bigwedge trace-table \ composition-table \ as.$
 $good-sets \ trace-table \ composition-table \ as \subseteq query-sample-space$

and *bound:*

$\bigwedge trace-table \ composition-table \ as.$
 $nnreal (card (good-sets \ trace-table \ composition-table \ as)) /$
 $nnreal (card \ query-sample-space) \leq query-error-bound$

shows

$nnreal$
 $(card$
 $(query-header-committed-union-good-sets \ s \text{ good-sets } fr \ as$
 $composition-fri-roots)) /$
 $nnreal (card \ query-sample-space) \leq query-error-bound$

proof –

from *unique* **obtain** *trace-table* *composition-table* **where** *candidates:*

$query-header-committed-table-candidates \ s \text{ fr } composition-fri-roots$
 $\subseteq \{(trace-table, composition-table)\}$

by *blast*

have *union-subset:*

```

    query-header-committed-union-good-sets s good-sets fr as
      composition-fri-roots  $\subseteq$  good-sets trace-table composition-table as
unfolding query-header-committed-union-good-sets-def
using candidates by auto
have finite-good: finite (good-sets trace-table composition-table as)
  by (rule finite-subset[OF subset finite-query-sample-space])
have card-le:
  card
    (query-header-committed-union-good-sets s good-sets fr as
      composition-fri-roots)  $\leq$ 
    card (good-sets trace-table composition-table as)
  by (rule card-mono[OF finite-good union-subset])
have nnreal
  (card
    (query-header-committed-union-good-sets s good-sets fr as
      composition-fri-roots)) /
  nnreal (card query-sample-space)  $\leq$ 
  nnreal (card (good-sets trace-table composition-table as)) /
  nnreal (card query-sample-space)
  by (rule nnreal-nat-divide-right-mono[OF card-le])
also have ...  $\leq$  query-error-bound
  by (rule bound)
finally show ?thesis .
qed

lemma query-header-committed-table-candidates-subset-singleton-if-no-root-collisions:
assumes trace-clean:  $\neg$  merkle-root-binding-collision fr s
and comp-clean:
   $\bigwedge rt. \text{composition-fri-roots} \neq [] \implies$ 
   $rt = \text{hd composition-fri-roots} \implies$ 
   $\neg \text{merkle-root-binding-collision } rt \ s$ 
shows
   $\exists \text{trace-table composition-table.}$ 
   $\text{query-header-committed-table-candidates } s \text{ fr composition-fri-roots}$ 
   $\subseteq \{(\text{trace-table}, \text{composition-table})\}$ 
proof (cases
  query-header-committed-table-candidates s fr composition-fri-roots = {})
case True
then show ?thesis
  by (intro exI[of -] exI[of -]) simp
next
case False
then obtain pair where pair-in:
  pair  $\in$ 
  query-header-committed-table-candidates s fr composition-fri-roots
by blast
then obtain trace-table composition-table where pair-eq:
  pair = (trace-table, composition-table)
by (cases pair)

```

```

have candidate:
  (trace-table, composition-table) ∈
    query-header-committed-table-candidates s fr composition-fri-roots
  using pair-in unfolding pair-eq .
have trace-bind:
  merkle-root-binds-table fr trace-table s
  using candidate unfolding query-header-committed-table-candidates-def
  by simp
have comp-nonempty: composition-fri-roots ≠ []
  using candidate unfolding query-header-committed-table-candidates-def
  by simp
have comp-bind:
  merkle-root-binds-table (hd composition-fri-roots) composition-table s
  using candidate unfolding query-header-committed-table-candidates-def
  by simp
have candidates:
  query-header-committed-table-candidates s fr composition-fri-roots
    ⊆ {(trace-table, composition-table)}
proof
  fix pair
  assume pair-in:
    pair ∈
      query-header-committed-table-candidates s fr composition-fri-roots
  obtain trace-table' composition-table' where pair-eq:
    pair = (trace-table', composition-table')
  by (cases pair)
  have trace-bind':
    merkle-root-binds-table fr trace-table' s
    using pair-in unfolding pair-eq
    query-header-committed-table-candidates-def
    by simp
  have comp-bind':
    merkle-root-binds-table (hd composition-fri-roots) composition-table'
      s
    using pair-in unfolding pair-eq
    query-header-committed-table-candidates-def
    by simp
  have trace-eq: trace-table' = trace-table
    using merkle-root-binds-table-unique-if-no-collision
    [OF trace-clean trace-bind' trace-bind] by simp
  have comp-eq: composition-table' = composition-table
    using merkle-root-binds-table-unique-if-no-collision
    [OF comp-clean [OF comp-nonempty refl] comp-bind' comp-bind]
    by simp
  show pair ∈ {(trace-table, composition-table)}
    unfolding pair-eq trace-eq comp-eq by simp
qed
show ?thesis
  using candidates by blast

```

qed

lemma *query-index-round-set-hit-at-imp-query-header-supported-union-hit:*

assumes *outcome: out* $\in$ *set-dist* (*execute verify-monad s*)

and hit: *query-index-round-set-hit-at s good-sets i out*

shows

$\exists$ *fr f-fri-roots f-final as dg composition-fri-roots final.*

query-rounds-any-index-set-hit s

(*query-header-supported-union-good-sets s good-sets fr f-fri-roots*
f-final as dg composition-fri-roots final)

rounds out

proof –

from *hit* **obtain** *trace-table composition-table as query-idxs* **where**
bound:

accepted-with-bound-tables s out trace-table composition-table as
query-idxs

and *i-bound: i < rounds*

and *query-hit:*

query-idxs ! i $\in$ *good-sets trace-table composition-table as*

unfolding *query-index-round-set-hit-at-def* **by** *blast*

from *bound* **obtain** *result final-state fr f-fri-roots f-final dg*

composition-fri-roots final rest **where**

out-eq: out = Some (result, final-state)

and *header:*

verifier-header-transcript s fr f-fri-roots f-final as dg
composition-fri-roots final rest

unfolding *accepted-with-bound-tables-def* **by** *blast*

have *candidate:*

(*trace-table, composition-table*) $\in$

query-header-supported-table-candidates s fr f-fri-roots f-final as dg
composition-fri-roots final

unfolding *query-header-supported-table-candidates-def*

using *outcome bound header* **by** *blast*

have *len: length query-idxs = rounds*

by (*rule accepted-with-bound-tables-shapes(4)[OF bound]*)

have *idx-mem: query-idxs ! i* $\in$ *set query-idxs*

by (*rule nth-mem*) (*use i-bound len in simp*)

have *idx-sample: query-idxs ! i* $\in$ *query-sample-space*

by (*rule accepted-with-bound-tables-query-sample-space*
[*OF bound idx-mem*])

have *union-hit:*

query-idxs ! i $\in$

query-header-supported-union-good-sets s good-sets fr f-fri-roots
f-final as dg composition-fri-roots final

unfolding *query-header-supported-union-good-sets-def*

using *idx-sample candidate query-hit* **by** *blast*

have *round-hit:*

query-rounds-any-index-set-hit s

(*query-header-supported-union-good-sets s good-sets fr f-fri-roots*

f-final as dg composition-fri-roots final)
rounds out
by (rule *accepted-with-bound-tables-query-rounds-any-index-set-hit*
[OF bound i-bound union-hit])
show *?thesis*
using *round-hit* **by** *blast*
qed

lemma *query-index-round-set-hit-at-imp-query-header-supported-rounds-hit:*
assumes *outcome: out ∈ set-dist (execute verify-monad s)*
and *hit: query-index-round-set-hit-at s good-sets i out*
shows
query-header-rounds-any-index-set-hit s
(query-header-supported-union-good-sets s good-sets) out
proof –
from *hit* **obtain** *trace-table composition-table as query-idxs* **where**
bound:
accepted-with-bound-tables s out trace-table composition-table as
query-idxs
and *i-bound: i < rounds*
and *query-hit:*
query-idxs ! i ∈ good-sets trace-table composition-table as
unfolding *query-index-round-set-hit-at-def* **by** *blast*
from *bound* **obtain** *result final-state fr f-fri-roots f-final dg*
composition-fri-roots final rest **where**
header:
verifier-header-transcript s fr f-fri-roots f-final as dg
composition-fri-roots final rest
unfolding *accepted-with-bound-tables-def* **by** *blast*
from *query-index-round-set-hit-at-imp-query-header-supported-union-hit*
[OF outcome hit]
obtain *fr' f-fri-roots' f-final' as' dg' composition-fri-roots' final'*
where *round-hit':*
query-rounds-any-index-set-hit s
(query-header-supported-union-good-sets s good-sets fr' f-fri-roots'
f-final' as' dg' composition-fri-roots' final')
rounds out
by *blast*
have *candidate:*
(trace-table, composition-table) ∈
query-header-supported-table-candidates s fr f-fri-roots f-final as dg
composition-fri-roots final
unfolding *query-header-supported-table-candidates-def*
using *outcome bound header* **by** *blast*
have *len: length query-idxs = rounds*
by (rule *accepted-with-bound-tables-shapes(4)* *[OF bound]*)
have *idx-mem: query-idxs ! i ∈ set query-idxs*
by (rule *nth-mem*) (use *i-bound len* **in** *simp*)
have *idx-sample: query-idxs ! i ∈ query-sample-space*

```

    by (rule accepted-with-bound-tables-query-sample-space
        [OF bound idx-mem])
  have union-hit:
    query-idxs ! i ∈
      query-header-supported-union-good-sets s good-sets fr f-fri-roots
      f-final as dg composition-fri-roots final
    unfolding query-header-supported-union-good-sets-def
    using idx-sample candidate query-hit by blast
  have round-hit:
    query-rounds-any-index-set-hit s
      (query-header-supported-union-good-sets s good-sets fr f-fri-roots
        f-final as dg composition-fri-roots final)
    rounds out
    by (rule accepted-with-bound-tables-query-rounds-any-index-set-hit
        [OF bound i-bound union-hit])
  show ?thesis
    unfolding query-header-rounds-any-index-set-hit-def
    using header round-hit by blast
qed

lemma query-index-round-set-hit-imp-query-header-supported-rounds-hit:
  assumes outcome: out ∈ set-dist (execute verify-monad s)
  and hit: query-index-round-set-hit s good-sets out
  shows
    query-header-rounds-any-index-set-hit s
      (query-header-supported-union-good-sets s good-sets) out
  proof -
    from hit obtain i where
      hit-at: query-index-round-set-hit-at s good-sets i out
    unfolding query-index-round-set-hit-def by blast
  show ?thesis
    by (rule query-index-round-set-hit-at-imp-query-header-supported-rounds-hit
        [OF outcome hit-at])
qed

lemma query-index-set-hit-empty-queries:
  assumes bound:
    accepted-with-bound-tables s out trace-table composition-table as []
  shows query-index-set-hit s good-sets out
  unfolding query-index-set-hit-def
  apply (rule exI[where x=trace-table])
  apply (rule exI[where x=composition-table])
  apply (rule exI[where x=as])
  apply (rule exI[where x=[]])
  using bound by simp

lemma query-index-set-hit-vacuous-if-rounds-zero:
  assumes rounds-zero: rounds = 0
  and bound:

```

accepted-with-bound-tables s out trace-table composition-table as query-idxs
shows *query-index-set-hit s good-sets out*
proof –
 have *length query-idxs = 0*
 using *accepted-with-bound-tables-shapes(4)[OF bound] rounds-zero* **by** *simp*
 then have *query-idxs-empty: query-idxs = []*
 by *simp*
 show *?thesis*
 using *bound unfolding query-idxs-empty*
 by (rule *query-index-set-hit-empty-queries*)
qed

lemma *query-index-set-hit-imp-round-set-hit:*
 assumes *hit: query-index-set-hit s good-sets out*
 shows *query-index-round-set-hit s good-sets out*
proof –
 from *hit* **obtain** *trace-table composition-table as query-idxs* **where**
bound:
accepted-with-bound-tables s out trace-table composition-table as
query-idxs
and *subset:*
set query-idxs $\subseteq$ good-sets trace-table composition-table as
unfolding *query-index-set-hit-def* **by** *blast*
 have *len: length query-idxs = rounds*
 by (rule *accepted-with-bound-tables-shapes(4)[OF bound]*)
 have *zero-bound: 0 < length query-idxs*
 using *len rounds-positive* **by** *simp*
 have *nth-in: query-idxs ! 0 $\in$ set query-idxs*
 by (rule *nth-mem[OF zero-bound]*)
 have *round-hit:*
query-index-round-set-hit-at s good-sets 0 out
unfolding *query-index-round-set-hit-at-def*
apply (rule *exI[where x=trace-table]*)
apply (rule *exI[where x=composition-table]*)
apply (rule *exI[where x=as]*)
apply (rule *exI[where x=query-idxs]*)
 using *bound subset nth-in rounds-positive* **by** *auto*
 show *?thesis*
unfolding *query-index-round-set-hit-def*
 using *round-hit rounds-positive* **by** *blast*
qed

lemma *accepted-with-bound-tables-query-samples-miss-if-not-all-consistent:*
 assumes *bound:*
accepted-with-bound-tables s out trace-table composition-table as query-idxs
and *not-all:*
 $\neg$ *all-queries-consistent trace-table composition-table as*
 shows *query-samples-miss-disagreements trace-table composition-table as query-idxs*
proof –


```

have disagreements:
  query-disagreement-indices trace-table composition-table as  $\neq \{\}$ 
  using not-all query-disagreement-indices-nonempty-iff by blast
have samples-agree:
  set query-idxs  $\subseteq$  query-agreement-indices trace-table composition-table as
proof
  fix idx
  assume idx-in: idx  $\in$  set query-idxs
  have sample: idx  $\in$  query-sample-space
    by (rule accepted-with-bound-tables-query-sample-space[OF bound idx-in])
  have consistent:
    query-consistent-at trace-table composition-table as idx
    by (rule accepted-with-bound-tables-query-consistency[OF bound idx-in])
  show idx  $\in$  query-agreement-indices trace-table composition-table as
    using sample consistent unfolding query-agreement-indices-def by simp
qed
show ?thesis
  using disagreements samples-agree
  unfolding query-samples-miss-disagreements-def by simp
qed

lemma query-sampling-success-space-subset:
  query-sampling-success-space trace-table composition-table as  $\subseteq$  query-sample-space
  unfolding query-sampling-success-space-def query-agreement-indices-def by auto

lemma query-sampling-success-space-fraction-bound:
  shows
    nnreal (card (query-sampling-success-space trace-table composition-table as)) /
      nnreal query-sample-space-size  $\leq$  query-error-bound
proof (cases
  trace-table-low-degree trace-table  $\wedge$ 
  composition-table-low-degree maxDegree composition-table  $\wedge$ 
   $\neg$  all-queries-consistent trace-table composition-table as)
case True
then have space-eq:
  query-sampling-success-space trace-table composition-table as =
  query-agreement-indices trace-table composition-table as
  unfolding query-sampling-success-space-def by simp
have card-le:
  card (query-sampling-success-space trace-table composition-table as)  $\leq$ 
  query-agreement-bound
  unfolding space-eq
  by (rule query-agreement-indices-card-bound)
  (use True in auto)
have nnreal (card (query-sampling-success-space trace-table composition-table as)) /
  nnreal query-sample-space-size  $\leq$ 
  nnreal query-agreement-bound / nnreal query-sample-space-size
  by (rule nnreal-nat-divide-right-mono[OF card-le])
then show ?thesis

```

```

    unfolding query-error-bound-def .
next
case False
then have space-eq:
  query-sampling-success-space trace-table composition-table as = {}
  unfolding query-sampling-success-space-def by auto
show ?thesis
  unfolding space-eq by simp
qed

lemma query-sampling-success-space-fraction-bound-query-sample-space:
shows
  nnreal (card (query-sampling-success-space trace-table composition-table as)) /
  nnreal (card query-sample-space) ≤ query-error-bound
using query-sampling-success-space-fraction-bound[of trace-table composition-table as]
by simp

lemma query-sampling-success-header-supported-union-fraction-bound-if-no-global-pairwise-merkle-bad:
assumes no-bad: ¬ query-supported-pairwise-merkle-bad s
shows
  nnreal
    (card
      (query-header-supported-union-good-sets s query-sampling-success-space
        fr f-fri-roots f-final as dg composition-fri-roots final)) /
  nnreal (card query-sample-space) ≤ query-error-bound
by (rule
  query-header-supported-union-good-sets-fraction-bound-if-no-global-pairwise-merkle-bad
  [OF no-bad query-sampling-success-space-subset
    query-sampling-success-space-fraction-bound-query-sample-space])

definition query-index-raw-preimage-bound :: bool
where
  query-index-raw-preimage-bound  $\longleftrightarrow$ 
  (∀ B.
    B ⊆ query-sample-space  $\longrightarrow$ 
    nnreal (card (query-index-raw-preimage B)) / nnreal size ≤
    nnreal (card B) / nnreal (card query-sample-space))

lemma query-index-raw-preimage-bound-if-uniform-range:
assumes range-eq: range to-nat = {0..

```

```

    (size div ?q) * card B
  unfolding card-query-index-raw-preimage-eq-nat-preimage
  by (rule card-query-index-nat-preimage-uniform-range
    [OF range-eq dvd subset])
  have size-eq: size = (size div ?q) * ?q
    using dvd query-sample-space-size-pos by (simp add: dvd-eq-mod-eq-0)
  have size-pos: 0 < size
    using size-card by simp
  have div-pos: 0 < size div ?q
  proof (cases size div ?q)
    case 0
    then show ?thesis
      using size-eq size-pos by simp
  next
    case (Suc n)
    then show ?thesis
      by simp
  qed
  have frac-eq:
    nnreal ((size div ?q) * card B) / nnreal ((size div ?q) * ?q) =
      nnreal (card B) / nnreal ?q
    using div-pos query-sample-space-size-pos
    by transfer (simp add: field-simps)
  have lhs-eq:
    nnreal (card (query-index-raw-preimage B)) / nnreal size =
      nnreal ((size div ?q) * card B) / nnreal ((size div ?q) * ?q)
    using card-raw size-eq by simp
  show
    nnreal (card (query-index-raw-preimage B)) / nnreal size ≤
      nnreal (card B) / nnreal (card query-sample-space)
    using lhs-eq frac-eq by simp
  qed

lemma query-index-raw-preimage-bound-from-sampler-wellformed:
  query-index-raw-preimage-bound
proof (rule query-index-raw-preimage-bound-if-uniform-range)
  show range to-nat = {0..

```

```

proof –
  have subset: {b} ⊆ query-sample-space
    using b-in by simp
  have nnreal (card (query-index-row-preimage {b})) / nnreal size ≤
    nnreal (card {b}) / nnreal (card query-sample-space)
    using raw-bound subset unfolding query-index-row-preimage-bound-def
    by blast
  then show ?thesis
    by simp
qed

lemma query-index-row-preimage-fraction-bound:
  assumes raw-bound: query-index-row-preimage-bound
  and subset: B ⊆ query-sample-space
  and frac:
    nnreal (card B) / nnreal (card query-sample-space) ≤
      query-error-bound
  shows
    nnreal (card (query-index-row-preimage B)) / nnreal size ≤
      query-error-bound
proof –
  have nnreal (card (query-index-row-preimage B)) / nnreal size ≤
    nnreal (card B) / nnreal (card query-sample-space)
    using raw-bound subset unfolding query-index-row-preimage-bound-def by blast
  also have ... ≤ query-error-bound
    by (rule frac)
  finally show ?thesis .
qed

lemma wp-receive-query-index-challenge-fresh-index-set-raw-bound:
  assumes future: query-future-fresh s
  and raw-bound: query-index-row-preimage-bound
  and subset: B ⊆ query-sample-space
  shows
    wp-event receive-query-index-challenge
      (λout. case out of None ⇒ False
        | Some (raw, -) ⇒ index (to-nat raw) ∈ B) s ≤
      nnreal (card B) / nnreal (card query-sample-space)
proof –
  have fresh:
    fmlookup (HashMap s)
      (QueryIndexChallenge (PQueryCounter s) (PState s)) = None
    by (rule query-future-fresh-current[OF future])
  have exact:
    wp-event receive-query-index-challenge
      (λout. case out of None ⇒ False
        | Some (raw, -) ⇒ index (to-nat raw) ∈ B) s =
      nnreal (card (query-index-row-preimage B)) / nnreal size
    by (rule wp-receive-query-index-challenge-fresh-index-set[OF fresh])

```

```

have raw:
  nnreal (card (query-index-raw-preimage B)) / nnreal size ≤
    nnreal (card B) / nnreal (card query-sample-space)
using raw-bound subset
unfolding query-index-raw-preimage-bound-def by blast
show ?thesis
  unfolding exact by (rule raw)
qed

lemma wp-receive-query-index-challenge-fresh-index-set-query-error-bound:
assumes future: query-future-fresh s
and raw-bound: query-index-raw-preimage-bound
and subset:  $B \subseteq \text{query-sample-space}$ 
and frac:
  nnreal (card B) / nnreal (card query-sample-space) ≤
    query-error-bound
shows
  wp-event receive-query-index-challenge
    ( $\lambda \text{out. case out of None} \Rightarrow \text{False}$ 
      |  $\text{Some (raw, -)} \Rightarrow \text{index (to-nat raw)} \in B$ ) s ≤
    query-error-bound
by (rule order-trans
  [OF wp-receive-query-index-challenge-fresh-index-set-raw-bound
  [OF future raw-bound subset] frac])

lemma wp-verifier-query-round-program-index-set-bound:
assumes future: query-future-fresh s
and raw-bound: query-index-raw-preimage-bound
and subset:  $B \subseteq \text{query-sample-space}$ 
shows
  wp-event
    (verifier-query-round-program fr f-fl f-final as fl final)
    (query-round-index-set-hit s B) s ≤
    nnreal (card B) / nnreal (card query-sample-space)
proof –
let ?P =
   $\lambda \text{out. case out of None} \Rightarrow \text{False}$ 
    |  $\text{Some (raw, -)} \Rightarrow \text{index (to-nat raw)} \in B$ 
have head-bound:
  wp-event receive-query-index-challenge ?P s ≤
    nnreal (card B) / nnreal (card query-sample-space)
by (rule wp-receive-query-index-challenge-fresh-index-set-raw-bound
  [OF future raw-bound subset])
show ?thesis
  unfolding verifier-query-round-program-alt-def
proof (rule wp-event-bind-bound-by-head-event[OF head-bound])
  show query-round-index-set-hit s B None  $\implies$  ?P None
    unfolding query-round-index-set-hit-def by simp
next

```

```

fix raw s0 out
assume head:
  Some (raw, s0) ∈
    set-dist (execute receive-query-index-challenge s)
and tail:
  out ∈
    set-dist
      (execute
        (verifier-query-round-after-index-program fr f-fl f-final as fl
          final raw) s0)
and hit: query-round-index-set-hit s B out
from hit obtain raw' t where out-eq: out = Some ((), t)
and lookup-t:
  fmlookup (HashMap t)
    (QueryIndexChallenge (PQueryCounter s) (PState s)) = Some raw'
and raw'-in: index (to-nat raw') ∈ B
unfolding query-round-index-set-hit-def by blast
have tail-out:
  Some ((), t) ∈
    set-dist
      (execute
        (verifier-query-round-after-index-program fr f-fl f-final as fl
          final raw) s0)
using tail out-eq by simp
have ext-s0-t: s0 ≤ t
using verifier-query-round-after-index-program-outcome[OF tail-out]
by simp
have lookup-s0:
  fmlookup (HashMap s0)
    (QueryIndexChallenge (PQueryCounter s) (PState s)) = Some raw
using receive-query-index-challenge-outcome[OF head] by blast
have lookup-t-raw:
  fmlookup (HashMap t)
    (QueryIndexChallenge (PQueryCounter s) (PState s)) = Some raw
by (rule hash-extension-lookup[OF lookup-s0 ext-s0-t])
have raw = raw'
using lookup-t-raw lookup-t by simp
then show ?P (Some (raw, s0))
using raw'-in by simp
qed
qed

lemma wp-verifier-query-round-program-index-set-query-error-bound:
assumes future: query-future-fresh s
and raw-bound: query-index-raw-preimage-bound
and subset: B ⊆ query-sample-space
and frac:
  nnreal (card B) / nnreal (card query-sample-space) ≤
    query-error-bound

```

```

shows
  wp-event
    (verifier-query-round-program fr f-fl f-final as fl final)
    (query-round-index-set-hit s B) s ≤ query-error-bound
by (rule order-trans
  [OF wp-verifier-query-round-program-index-set-bound
  [OF future raw-bound subset] frac])

lemma wp-verifier-query-round-program-any-index-set-bound:
assumes future: query-future-fresh s
and raw-bound: query-index-raw-preimage-bound
and subset: B ⊆ query-sample-space
shows
  wp-event
    (verifier-query-round-program fr f-fl f-final as fl final)
    (query-round-any-index-set-hit s B) s ≤
    nnreal (card B) / nnreal (card query-sample-space)
proof -
let ?P =
  λout. case out of None ⇒ False
    | Some (raw, -) ⇒ index (to-nat raw) ∈ B
have head-bound:
  wp-event receive-query-index-challenge ?P s ≤
    nnreal (card B) / nnreal (card query-sample-space)
by (rule wp-receive-query-index-challenge-fresh-index-set-raw-bound
  [OF future raw-bound subset])
show ?thesis
unfolding verifier-query-round-program-alt-def
proof (rule wp-event-bind-bound-by-head-event[OF head-bound])
show query-round-any-index-set-hit s B None ⇒ ?P None
unfolding query-round-any-index-set-hit-def by simp
next
fix raw s0 out
assume head:
  Some (raw, s0) ∈
    set-dist (execute receive-query-index-challenge s)
and tail:
  out ∈
    set-dist
      (execute
        (verifier-query-round-after-index-program fr f-fl f-final as fl
          final raw) s0)
and hit: query-round-any-index-set-hit s B out
from hit obtain raw' t x where out-eq: out = Some ((), t)
and lookup-t:
  fmlookup (HashMap t)
    (QueryIndexChallenge (PQueryCounter s) x) = Some raw'
and raw'-in: index (to-nat raw') ∈ B
unfolding query-round-any-index-set-hit-def by blast

```

```

have tail-out:
  Some ((), t) ∈
    set-dist
      (execute
        (verifier-query-round-after-index-program fr f-fl f-final as fl
          final raw) s0)
using tail out-eq by simp
have lookup-s0-x:
  fmlookup (HashMap s0)
    (QueryIndexChallenge (PQueryCounter s) x) = Some raw'
using verifier-query-round-after-index-program-preserves-query-lookup
  [OF tail-out, of PQueryCounter s x]
  lookup-t
by simp
have lookup-current:
  fmlookup (HashMap s0)
    (QueryIndexChallenge (PQueryCounter s) (PState s)) = Some raw
using receive-query-index-challenge-outcome[OF head] by blast
have key-eq:
  QueryIndexChallenge (PQueryCounter s) x =
    QueryIndexChallenge (PQueryCounter s) (PState s)
proof (rule ccontr)
  assume neg:
    QueryIndexChallenge (PQueryCounter s) x ≠
      QueryIndexChallenge (PQueryCounter s) (PState s)
  have lookup-s0-x':
    fmlookup (HashMap s0)
      (QueryIndexChallenge (PQueryCounter s) x) =
        fmlookup (HashMap s0)
          (QueryIndexChallenge (PQueryCounter s) x)
    by (rule receive-query-index-challenge-preserves-other-lookup
      [OF head neg])
  have fmlookup (HashMap s)
    (QueryIndexChallenge (PQueryCounter s) x) = None
    using future unfolding query-future-fresh-def by simp
  then show False
    using lookup-s0-x lookup-s0-x' by simp
qed
have raw' = raw
  using lookup-s0-x lookup-current key-eq by simp
then show ?P (Some (raw, s0))
  using raw'-in by simp
qed
qed

lemma wp-ntimes-verifier-query-round-program-any-index-set-bound:
  assumes future: query-future-fresh s
  and raw-bound: query-index-raw-preimage-bound
  and subset: B ⊆ query-sample-space

```



```

shows
  wp-event
    (ntimes (verifier-query-round-program fr f-fl f-final as fl final) n)
    (query-rounds-any-index-set-hit s B n) s ≤
    nnreal n *
    (nnreal (card B) / nnreal (card query-sample-space))
using future
proof (induction n arbitrary: s)
  case 0
  show ?case
    unfolding query-rounds-any-index-set-hit-def
    by (simp add: wp-event-def upsimps)
next
  case (Suc n)
  let ?prog = verifier-query-round-program fr f-fl f-final as fl final
  let ?C = nnreal (card B) / nnreal (card query-sample-space)
  let ?Q = query-rounds-any-index-set-hit s B (Suc n)
  let ?Head =
    λout :: (unit list × ('f, 'a) protocol-channel-scheme) option.
      case out of None ⇒ False
      | Some (-, t) ⇒
        (∃ raw x.
          fmlookup (HashMap t)
            (QueryIndexChallenge (PQueryCounter s) x) = Some raw ∧
            index (to-nat raw) ∈ B)
  let ?Tail =
    λout :: (unit list × ('f, 'a) protocol-channel-scheme) option.
      case out of None ⇒ False
      | Some (-, t) ⇒
        (∃ i < n. ∃ x raw.
          fmlookup (HashMap t)
            (QueryIndexChallenge (PQueryCounter s + Suc i) x) =
              Some raw ∧
              index (to-nat raw) ∈ B)
  have split-hit:
    ∧out. ?Q out ⇒ ?Head out ∨ ?Tail out
proof -
  fix out
  assume ?Q out
  then obtain results t i x raw where out-eq: out = Some (results, t)
  and i-bound: i < Suc n
  and lookup:
    fmlookup (HashMap t)
      (QueryIndexChallenge (PQueryCounter s + i) x) = Some raw
  and raw-in: index (to-nat raw) ∈ B
  unfolding query-rounds-any-index-set-hit-def by blast
  show ?Head out ∨ ?Tail out
proof (cases i)
  case 0

```

```

then have ?Head out
  using out-eq lookup raw-in by simp blast
then show ?thesis by simp
next
case (Suc j)
then have j-bound:  $j < n$ 
  using i-bound by simp
have ?Tail out
  using out-eq lookup raw-in j-bound Suc by simp blast
then show ?thesis by simp
qed
qed
have head-bound:
  wp-event (ntimes ?prog (Suc n)) ?Head  $s \leq ?C$ 
proof -
  have one-bound:
    wp-event ?prog (query-round-any-index-set-hit s B)  $s \leq ?C$ 
    by (rule wp-verifier-query-round-program-any-index-set-bound
      [OF Suc.prem1 raw-bound subset])
  show ?thesis
    unfolding ntimes.simps
  proof (rule wp-event-bind-bound-by-head-event[OF one-bound])
    show ?Head None  $\implies$  query-round-any-index-set-hit s B None
      by simp
  next
  fix u s1 out
  assume head:
    Some (u, s1)  $\in$  set-dist (execute ?prog s)
  and tail:
    out  $\in$ 
      set-dist
        (execute
          (ntimes ?prog n  $\gg$  ( $\lambda xs.$  return (u # xs))) s1)
  and hit: ?Head out
  have u-eq:  $u = ()$ 
    by (cases u) simp
  from hit obtain ys0 t0 where out-some: out = Some (ys0, t0)
    by (cases out) auto
  from tail[unfolded out-some] obtain results t where
    tail-rounds:
      Some (results, t)  $\in$  set-dist (execute (ntimes ?prog n) s1)
  and out-ret:
    Some (ys0, t0)  $\in$  set-dist (execute (return (u # results)) t)
    by (rule set-dist-bindE)
  have out-eq: out = Some (u # results, t)
    using out-ret out-some
    by simp
  from hit obtain raw x where lookup-t:
    fmllookup (HashMap t)

```

```

    (QueryIndexChallenge (PQueryCounter s) x) = Some raw
  and raw-in: index (to-nat raw) ∈ B
  using out-eq by auto
have counter-s1: PQueryCounter s1 = Suc (PQueryCounter s)
  using verifier-query-round-program-outcome[OF head[unfolded u-eq]]
  by blast
have past: PQueryCounter s < PQueryCounter s1
  using counter-s1 by simp
have lookup-s1:
  fmllookup (HashMap s1)
    (QueryIndexChallenge (PQueryCounter s) x) = Some raw
  using ntimes-verifier-query-round-program-preserves-past-query-lookup
    [OF past tail-rounds, of x]
    lookup-t
  by simp
show query-round-any-index-set-hit s B (Some (u, s1))
  unfolding query-round-any-index-set-hit-def
  using lookup-s1 raw-in u-eq by blast
qed
qed
have tail-bound:
  wp-event (ntimes ?prog (Suc n)) ?Tail s ≤ nnreal n * ?C
proof -
  show ?thesis
    unfolding ntimes.simps
  proof (rule wp-event-bind-bound-by-cont)
    show ¬ ?Tail None
      by simp
  next
    fix u s1
    assume head:
      Some (u, s1) ∈ set-dist (execute ?prog s)
    have u-eq: u = ()
      by (cases u) simp
    have future-s1: query-future-fresh s1
      by (rule verifier-query-round-program-preserves-query-future-fresh
        [OF Suc.prem head[unfolded u-eq]])
    have counter-s1: PQueryCounter s1 = Suc (PQueryCounter s)
      using verifier-query-round-program-outcome[OF head[unfolded u-eq]]
      by blast
    have tail-induct:
      wp-event (ntimes ?prog n)
        (query-rounds-any-index-set-hit s1 B n) s1 ≤
        nnreal n * ?C
      by (rule Suc.IH[OF future-s1])
    show wp-event
      (ntimes ?prog n ≫ (λxs. return (u # xs))) ?Tail s1
      ≤ nnreal n * ?C
  proof (rule wp-event-bind-bound-by-head-event[OF tail-induct])

```

```

show ?Tail None  $\implies$  query-rounds-any-index-set-hit s1 B n None
  by simp
next
  fix results t out
  assume tail-rounds:
    Some (results, t)  $\in$ 
      set-dist (execute (ntimes ?prog n) s1)
  and ret:
    out  $\in$  set-dist (execute (return (u # results)) t)
  and hit: ?Tail out
  have out-eq: out = Some (u # results, t)
  using ret
  unfolding set-dist-def return.rep-eq dist-return-def
    dist-delta-dist delta-map-def
  by simp
  from hit obtain i x raw where i-bound: i < n
  and lookup:
    fmlookup (HashMap t)
      (QueryIndexChallenge (PQueryCounter s + Suc i) x) =
        Some raw
  and raw-in: index (to-nat raw)  $\in$  B
  using out-eq by auto
  have key-eq:
    Suc (PQueryCounter s + i) = PQueryCounter s1 + i
  using counter-s1 by presburger
  have lookup':
    fmlookup (HashMap t)
      (QueryIndexChallenge (PQueryCounter s1 + i) x) = Some raw
  using lookup key-eq by simp
  show query-rounds-any-index-set-hit s1 B n (Some (results, t))
  unfolding query-rounds-any-index-set-hit-def
  apply (rule exI[where x=results])
  apply (rule exI[where x=t])
  apply (rule exI[where x=i])
  apply (rule exI[where x=x])
  apply (rule exI[where x=raw])
  using i-bound lookup' raw-in by simp
  qed
qed
qed
have wp-event (ntimes ?prog (Suc n)) ?Q s  $\leq$ 
  wp-event (ntimes ?prog (Suc n))
    ( $\lambda$ out. ?Head out  $\vee$  ?Tail out) s
  by (rule wp-event-mono) (use split-hit in blast)
also have ...  $\leq$  wp-event (ntimes ?prog (Suc n)) ?Head s +
  wp-event (ntimes ?prog (Suc n)) ?Tail s
  by (rule wp-event-union-bound)
also have ...  $\leq$  ?C + nnreal n * ?C
  by (intro add-mono head-bound tail-bound)

```

```

also have ... = nnreal (Suc n) * ?C
  by (simp add: algebra-simps)
finally show ?case .
qed

lemma wp-ntimes-verifier-query-round-program-any-index-set-query-error-bound:
  assumes future: query-future-fresh s
  and raw-bound: query-index-raw-preimage-bound
  and subset:  $B \subseteq \text{query-sample-space}$ 
  and frac:
    nnreal (card B) / nnreal (card query-sample-space)  $\leq$ 
      query-error-bound
  shows
    wp-event
      (ntimes (verifier-query-round-program fr f-fl f-final as fl final) n)
      (query-rounds-any-index-set-hit s B n) s  $\leq$ 
        nnreal n * query-error-bound
proof -
  have base:
    wp-event
      (ntimes (verifier-query-round-program fr f-fl f-final as fl final) n)
      (query-rounds-any-index-set-hit s B n) s  $\leq$ 
        nnreal n *
          (nnreal (card B) / nnreal (card query-sample-space))
  by (rule wp-ntimes-verifier-query-round-program-any-index-set-bound
      [OF future raw-bound subset])
  have mono:
    nnreal n *
      (nnreal (card B) / nnreal (card query-sample-space))  $\leq$ 
        nnreal n * query-error-bound
  by (rule mult-left-mono) (use frac in simp-all)
  show ?thesis
    by (rule order-trans[OF base mono])
qed

lemma wp-verifier-after-composition-fri-query-rounds-any-index-set-bound:
  assumes future: query-future-fresh t
  and counter: PQueryCounter t = PQueryCounter s
  and raw-bound: query-index-raw-preimage-bound
  and subset:  $B \subseteq \text{query-sample-space}$ 
  shows
    wp-event (verifier-after-composition-fri header)
      (query-rounds-any-index-set-hit s B rounds) t  $\leq$ 
        nnreal rounds *
          (nnreal (card B) / nnreal (card query-sample-space))
proof -
  obtain fr f-fl f-final as dg fl where header-eq:
    header = (fr, f-fl, f-final, as, dg, fl)
  by (cases header) auto

```

```

have after-eq:
  verifier-after-composition-fri header =
    (read  $\gg$ 
      ( $\lambda$ final.
        ntimes
          (verifier-query-round-program fr f-fl f-final as fl final)
          rounds))
  unfolding header-eq verifier-after-composition-fri-def by simp
show ?thesis
  unfolding after-eq
proof (rule wp-event-bind-bound-by-cont)
  show  $\neg$  query-rounds-any-index-set-hit s B rounds None
    unfolding query-rounds-any-index-set-hit-def by simp
next
  fix final query-state
  assume read-final:
    Some (final, query-state)  $\in$  set-dist (execute read t)
  have future-query: query-future-fresh query-state
    by (rule read-preserves-query-future-fresh[OF future read-final])
  from read-outcome[OF read-final] obtain rest where
    counter-query-t: PQueryCounter query-state = PQueryCounter t
    by blast
  have counter-query: PQueryCounter query-state = PQueryCounter s
    using counter-query-t counter by simp
  have event-imp:
     $\bigwedge$ out. query-rounds-any-index-set-hit s B rounds out  $\implies$ 
      query-rounds-any-index-set-hit query-state B rounds out
    using counter-query
    unfolding query-rounds-any-index-set-hit-def by auto
  have wp-event
    (ntimes
      (verifier-query-round-program fr f-fl f-final as fl final)
      rounds)
    (query-rounds-any-index-set-hit s B rounds) query-state  $\leq$ 
  wp-event
    (ntimes
      (verifier-query-round-program fr f-fl f-final as fl final)
      rounds)
    (query-rounds-any-index-set-hit query-state B rounds) query-state
    by (rule wp-event-mono) (rule event-imp)
  also have ...  $\leq$ 
    nnreal rounds *
      (nnreal (card B) / nnreal (card query-sample-space))
    by (rule wp-ntimes-verifier-query-round-program-any-index-set-bound
      [OF future-query raw-bound subset])
  finally show
    wp-event
      (ntimes
        (verifier-query-round-program fr f-fl f-final as fl final)

```

```

      rounds)
      (query-rounds-any-index-set-hit s B rounds) query-state ≤
nnreal rounds *
      (nnreal (card B) / nnreal (card query-sample-space)) .
qed
qed

lemma wp-verify-monad-query-rounds-any-index-set-bound:
  assumes future: query-future-fresh s
    and raw-bound: query-index-raw-preimage-bound
    and subset: B ⊆ query-sample-space
  shows
    wp-event verify-monad (query-rounds-any-index-set-hit s B rounds) s ≤
      nnreal rounds *
        (nnreal (card B) / nnreal (card query-sample-space))
  unfolding verify-monad-composition-fri-decomposition
proof (rule wp-event-bind-bound-by-cont)
  show ¬ query-rounds-any-index-set-hit s B rounds None
    unfolding query-rounds-any-index-set-hit-def by simp
next
  fix header t
  assume prefix:
    Some (header, t) ∈
      set-dist (execute verifier-composition-fri-prefix s)
  obtain fr f-fl f-final as dg fl where header-eq:
    header = (fr, f-fl, f-final, as, dg, fl)
  by (cases header) auto
  have future-t: query-future-fresh t
  by (rule verifier-composition-fri-prefix-preserves-query-future-fresh
    [OF future prefix])
  have counter-t: PQueryCounter t = PQueryCounter s
  using verifier-composition-fri-prefix-outcome[OF prefix[unfolded header-eq]]
  by simp
  show
    wp-event (verifier-after-composition-fri header)
      (query-rounds-any-index-set-hit s B rounds) t ≤
      nnreal rounds *
        (nnreal (card B) / nnreal (card query-sample-space))
  by (rule
    wp-verifier-after-composition-fri-query-rounds-any-index-set-bound
    [OF future-t counter-t raw-bound subset])
qed

lemma wp-verify-monad-query-rounds-any-index-set-query-error-bound:
  assumes future: query-future-fresh s
    and raw-bound: query-index-raw-preimage-bound
    and subset: B ⊆ query-sample-space
  and frac:
    nnreal (card B) / nnreal (card query-sample-space) ≤

```

query-error-bound

shows

$$wp\text{-event } verify\text{-monad } (query\text{-rounds-any-index-set-hit } s \ B \ rounds) \ s \leq nnreal \ rounds * query\text{-error-bound}$$

proof –
have *base*:

$$wp\text{-event } verify\text{-monad } (query\text{-rounds-any-index-set-hit } s \ B \ rounds) \ s \leq nnreal \ rounds * (nnreal \ (card \ B) / nnreal \ (card \ query\text{-sample-space}))$$
by (rule *wp-verify-monad-query-rounds-any-index-set-bound* [OF *future raw-bound subset*])
have *mono*:

$$nnreal \ rounds * (nnreal \ (card \ B) / nnreal \ (card \ query\text{-sample-space})) \leq nnreal \ rounds * query\text{-error-bound}$$
by (rule *mult-left-mono*) (use *frac* **in** *simp-all*)
show *?thesis*
by (rule *order-trans*[OF *base mono*])
qed

lemma *wp-verify-monad-query-header-rounds-any-index-set-bound*:
fixes *C* :: *prob*
assumes *future*: *query-future-fresh s*
and *raw-bound*: *query-index-raw-preimage-bound*
and *subset*:

$$\bigwedge fr \ f\text{-fri-roots } f\text{-final as dg composition-fri-roots final.} \\
B \ fr \ f\text{-fri-roots } f\text{-final as dg composition-fri-roots final} \\
\subseteq query\text{-sample-space}$$
and *bound*:

$$\bigwedge fr \ f\text{-fri-roots } f\text{-final as dg composition-fri-roots final.} \\
nnreal \\
(card \\
(B \ fr \ f\text{-fri-roots } f\text{-final as dg composition-fri-roots final})) / \\
nnreal \ (card \ query\text{-sample-space}) \leq C$$
shows

$$wp\text{-event } verify\text{-monad } (query\text{-header-rounds-any-index-set-hit } s \ B) \ s \leq nnreal \ rounds * C$$
unfolding *verify-monad-composition-fri-decomposition*
proof (rule *wp-event-bind-bound-by-cont*)
show $\neg query\text{-header-rounds-any-index-set-hit } s \ B \ None$
unfolding *query-header-rounds-any-index-set-hit-def*
query-rounds-any-index-set-hit-def
by *auto*
next
fix *header prefix-state*
assume *prefix*:

$$Some \ (header, \ prefix\text{-state}) \in set\text{-dist } (execute \ verifier\text{-composition-fri-prefix } s)$$
obtain *fr f-fl f-final as dg fl* **where** *header-eq*:


```

header = (fr, f-fl, f-final, as, dg, fl)
by (cases header) auto
have prefix-res:
length f-fl = ceil-log clength ∧
length as = length spec ∧
length fl = ceil-log (to-nat dg + 1) ∧
PTranscript s =
  [fr] @ map snd f-fl @ [f-final] @ as @ [dg] @
  map snd fl @ PTranscript prefix-state ∧
  PQueryCounter prefix-state = PQueryCounter s
using verifier-composition-fri-prefix-outcome
[OF prefix[unfolded header-eq]]
by simp
have future-prefix: query-future-fresh prefix-state
by (rule verifier-composition-fri-prefix-preserves-query-future-fresh
[OF future prefix])
have after-eq:
verifier-after-composition-fri header =
  (read >>=
    (λfinal.
      ntimes (verifier-query-round-program fr f-fl f-final as fl final)
        rounds))
unfolding header-eq verifier-after-composition-fri-def by simp
show
wp-event (verifier-after-composition-fri header)
  (query-header-rounds-any-index-set-hit s B) prefix-state ≤
  nnreal rounds * C
unfolding after-eq
proof (rule wp-event-bind-bound-by-cont)
show ¬ query-header-rounds-any-index-set-hit s B None
  unfolding query-header-rounds-any-index-set-hit-def
    query-rounds-any-index-set-hit-def
  by auto
next
fix final query-state
assume read-final:
  Some (final, query-state) ∈ set-dist (execute read prefix-state)
from read-outcome[OF read-final] obtain rest where
  tr-prefix: PTranscript prefix-state = final # rest
and tr-query: PTranscript query-state = rest
and query-counter:
  PQueryCounter query-state = PQueryCounter prefix-state
by blast
have future-query: query-future-fresh query-state
by (rule read-preserves-query-future-fresh[OF future-prefix read-final])
have counter-query: PQueryCounter query-state = PQueryCounter s
using prefix-res query-counter by simp
have header-prefix:
  verifier-header-transcript s fr (map snd f-fl) f-final as dg

```

```

    (map snd fl) final (PTranscript query-state)
using prefix-res tr-prefix tr-query
unfolding verifier-header-transcript-def verifier-header-messages-def
by simp
let ?B = B fr (map snd f-fl) f-final as dg (map snd fl) final
let ?prog = verifier-query-round-program fr f-fl f-final as fl final
have base:
  wp-event (ntimes ?prog rounds)
    (query-rounds-any-index-set-hit query-state ?B rounds)
    query-state ≤
    nnreal rounds *
    (nnreal (card ?B) / nnreal (card query-sample-space))
by (rule wp-ntimes-verifier-query-round-program-any-index-set-bound
    [OF future-query raw-bound subset])
have base-C:
  wp-event (ntimes ?prog rounds)
    (query-rounds-any-index-set-hit query-state ?B rounds)
    query-state ≤ nnreal rounds * C
proof —
  have nnreal rounds *
    (nnreal (card ?B) / nnreal (card query-sample-space)) ≤
    nnreal rounds * C
  by (rule mult-left-mono) (use bound in simp-all)
  then show ?thesis
    by (rule order-trans[OF base])
qed
show
  wp-event
    (ntimes ?prog rounds)
    (query-header-rounds-any-index-set-hit s B) query-state ≤
    nnreal rounds * C
proof (rule order-trans[OF - base-C])
show
  wp-event
    (ntimes ?prog rounds)
    (query-header-rounds-any-index-set-hit s B) query-state ≤
  wp-event
    (ntimes ?prog rounds)
    (query-rounds-any-index-set-hit query-state ?B rounds) query-state
proof (rule wp-event-mono-on-support)
fix out
assume out-support:
  out ∈ set-dist (execute (ntimes ?prog rounds) query-state)
  and hit: query-header-rounds-any-index-set-hit s B out
from hit obtain fr' f-fri-roots' f-final' as' dg'
  composition-fri-roots' final' rest' where
  header-hit:
  verifier-header-transcript s fr' f-fri-roots' f-final' as' dg'
  composition-fri-roots' final' rest'

```

```

and round-hit:
  query-rounds-any-index-set-hit s
    (B fr' f-fri-roots' f-final' as' dg'
      composition-fri-roots' final')
  rounds out
unfolding query-header-rounds-any-index-set-hit-def by blast
have eqs:
  fr' = fr  $\wedge$ 
  f-fri-roots' = map snd f-fl  $\wedge$ 
  f-final' = f-final  $\wedge$ 
  as' = as  $\wedge$ 
  dg' = dg  $\wedge$ 
  composition-fri-roots' = map snd fl  $\wedge$ 
  final' = final
using verifier-header-transcript-unique[OF header-prefix header-hit]
by simp
have round-hit-s:
  query-rounds-any-index-set-hit s ?B rounds out
using round-hit eqs by simp
show
  query-rounds-any-index-set-hit query-state ?B rounds out
using round-hit-s counter-query
unfolding query-rounds-any-index-set-hit-def by auto
qed
qed
qed
qed

```

lemma wp-query-index-round-set-hit-bound-via-supported-header-union:

```

assumes future: query-future-fresh s
and raw-bound: query-index-raw-preimage-bound
and union-bound:
   $\bigwedge_{\text{fr f-fri-roots f-final as dg composition-fri-roots final.}}$ 
  nnreal
  (card
    (query-header-supported-union-good-sets s good-sets fr
      f-fri-roots f-final as dg composition-fri-roots final)) /
  nnreal (card query-sample-space)  $\leq$  query-error-bound

```

shows

```

wp-event verify-monad (query-index-round-set-hit s good-sets) s  $\leq$ 
  nnreal rounds * query-error-bound

```

proof –

```

have header-bound:
  wp-event verify-monad
    (query-header-rounds-any-index-set-hit s
      (query-header-supported-union-good-sets s good-sets)) s  $\leq$ 
  nnreal rounds * query-error-bound
by (rule wp-verify-monad-query-header-rounds-any-index-set-bound
  [OF future raw-bound])

```

(use query-header-supported-union-good-sets-subset union-bound in
 simp-all)
have wp-event verify-monad (query-index-round-set-hit s good-sets) s ≤
 wp-event verify-monad
 (query-header-rounds-any-index-set-hit s
 (query-header-supported-union-good-sets s good-sets)) s
by (rule wp-event-mono-on-support)
 (rule query-index-round-set-hit-imp-query-header-supported-rounds-hit)
also have ... ≤ nnreal rounds * query-error-bound
by (rule header-bound)
finally show ?thesis .
qed

lemma verifier-initial-query-rounds-any-index-set-error-bound:

assumes raw-bound: query-index-raw-preimage-bound
and subset: $B \subseteq \text{query-sample-space}$
and frac:
 nnreal (card B) / nnreal (card query-sample-space) ≤
 query-error-bound

shows

wp-event verify-monad
 (query-rounds-any-index-set-hit (verifier-initial-state tr) B rounds)
 (verifier-initial-state tr) ≤
 nnreal rounds * query-error-bound
by (rule wp-verify-monad-query-rounds-any-index-set-query-error-bound
 [OF verifier-initial-query-future-fresh raw-bound subset frac])

definition query-index-freshness-assumption

:: (f, 'a) protocol-channel-scheme ⇒ bool

where

query-index-freshness-assumption s ⇔
 (∀ good-sets.
 (∀ trace-table composition-table as.
 good-sets trace-table composition-table as ⊆ query-sample-space) →
 (∀ trace-table composition-table as.
 nnreal (card (good-sets trace-table composition-table as)) /
 nnreal (card query-sample-space) ≤ query-error-bound) →
 wp-event verify-monad (query-index-set-hit s good-sets) s ≤
 nnreal rounds * query-error-bound)

definition query-index-round-freshness-assumption

:: (f, 'a) protocol-channel-scheme ⇒ bool

where

query-index-round-freshness-assumption s ⇔
 (∀ good-sets.
 (∀ trace-table composition-table as.
 good-sets trace-table composition-table as ⊆ query-sample-space) →
 (∀ trace-table composition-table as.
 nnreal (card (good-sets trace-table composition-table as)) /

$$\text{nnreal } (\text{card } \text{query-sample-space}) \leq \text{query-error-bound}) \longrightarrow \\ \text{wp-event verify-monad } (\text{query-index-round-set-hit } s \text{ good-sets}) s \leq \\ \text{nnreal rounds} * \text{query-error-bound})$$

lemma *query-index-freshness-from-round-freshness:*

assumes *fresh-round: query-index-round-freshness-assumption s*

shows *query-index-freshness-assumption s*

unfolding *query-index-freshness-assumption-def*

proof (*intro allI impI*)

fix *good-sets :: 'f list $\Rightarrow$ 'f list $\Rightarrow$ 'f list $\Rightarrow$ nat set*

assume *subset:*

$\forall$ *trace-table composition-table as.*

good-sets trace-table composition-table as $\subseteq$ query-sample-space

and *frac:*

$\forall$ *trace-table composition-table as.*

nnreal (card (good-sets trace-table composition-table as)) /

nnreal (card query-sample-space) $\leq$ query-error-bound

have *wp-event verify-monad (query-index-set-hit s good-sets) s $\leq$*

wp-event verify-monad (query-index-round-set-hit s good-sets) s

by (*rule wp-event-mono*) (*rule query-index-set-hit-imp-round-set-hit*)

also have *... $\leq$ nnreal rounds * query-error-bound*

using *fresh-round subset frac*

unfolding *query-index-round-freshness-assumption-def* **by** *blast*

finally show *wp-event verify-monad (query-index-set-hit s good-sets) s $\leq$*

*nnreal rounds * query-error-bound .*

qed

lemma *query-index-round-freshness-if-no-query-supported-pairwise-merkle-bad:*

assumes *future: query-future-fresh s*

and *raw-bound: query-index-raw-preimage-bound*

and *no-bad: $\neg$ query-supported-pairwise-merkle-bad s*

shows *query-index-round-freshness-assumption s*

unfolding *query-index-round-freshness-assumption-def*

proof (*intro allI impI*)

fix *good-sets :: 'f list $\Rightarrow$ 'f list $\Rightarrow$ 'f list $\Rightarrow$ nat set*

assume *subset:*

$\forall$ *trace-table composition-table as.*

good-sets trace-table composition-table as $\subseteq$ query-sample-space

and *bounded:*

$\forall$ *trace-table composition-table as.*

nnreal (card (good-sets trace-table composition-table as)) /

nnreal (card query-sample-space) $\leq$ query-error-bound

have *union-bound:*

$\bigwedge$ *fr f-fri-roots f-final as dg composition-fri-roots final.*

nnreal

(card

(query-header-supported-union-good-sets s good-sets fr f-fri-roots

f-final as dg composition-fri-roots final)) /

nnreal (card query-sample-space) $\leq$ query-error-bound

by (rule
 query-header-supported-union-good-sets-fraction-bound-if-no-global-pairwise-merkle-bad
 [*OF no-bad*])
 (use subset bounded in blast)+
 show wp-event verify-monad (*query-index-round-set-hit s good-sets*) $s \leq$
 nnreal rounds * *query-error-bound*
 by (rule wp-query-index-round-set-hit-bound-via-supported-header-union
 [*OF future raw-bound union-bound*])
 qed

lemma *query-index-freshness-if-no-query-supported-pairwise-merkle-bad*:
 assumes future: *query-future-fresh s*
 and raw-bound: *query-index-raw-preimage-bound*
 and no-bad: $\neg$ *query-supported-pairwise-merkle-bad s*
 shows *query-index-freshness-assumption s*
 by (rule *query-index-freshness-from-round-freshness*)
 (rule *query-index-round-freshness-if-no-query-supported-pairwise-merkle-bad*
 [*OF future raw-bound no-bad*])

lemma *verifier-initial-query-index-freshness-if-no-query-supported-pairwise-merkle-bad*:
 assumes raw-bound: *query-index-raw-preimage-bound*
 and no-bad:
 $\neg$ *query-supported-pairwise-merkle-bad (verifier-initial-state tr)*
 shows *query-index-freshness-assumption (verifier-initial-state tr)*
 by (rule *query-index-freshness-if-no-query-supported-pairwise-merkle-bad*)
 (use raw-bound no-bad in simp-all)

lemma *verifier-initial-query-index-freshness-if-uniform-sampler-and-no-query-supported-pairwise-merkle-bad*:
 assumes range-eq: *range to-nat = {0..*size*}*
 and dvd: *query-sample-space-size dvd size*
 and no-bad:
 $\neg$ *query-supported-pairwise-merkle-bad (verifier-initial-state tr)*
 shows *query-index-freshness-assumption (verifier-initial-state tr)*
proof –
 have raw-bound: *query-index-raw-preimage-bound*
 by (rule *query-index-raw-preimage-bound-if-uniform-range*
 [*OF range-eq dvd*])
 show ?thesis
 by (rule
 verifier-initial-query-index-freshness-if-no-query-supported-pairwise-merkle-bad
 [*OF raw-bound no-bad*])
 qed

lemma *query-index-freshness-rounds-zero-obstruction*:
 assumes rounds-zero: *rounds = 0*
 and outcome: *out $\in$ set-dist (execute verify-monad s)*
 and bound:
 accepted-with-bound-tables s out trace-table composition-table as query-idxs
 shows $\neg$ *query-index-freshness-assumption s*

```

proof
  assume fresh: query-index-freshness-assumption s
  let ?good-sets =  $\lambda - -. \{ \}$ 
  have hit: query-index-set-hit s ?good-sets out
    by (rule query-index-set-hit-vacuous-if-rounds-zero
      [OF rounds-zero bound])
  have positive:
     $0 < \text{wp-event verify-monad } (\text{query-index-set-hit } s \text{ ?good-sets}) \ s$ 
    by (rule wp-event-pos-of-support
      [where out=out and m=verify-monad and s=s
        and P=query-index-set-hit s ?good-sets, OF outcome hit])
  have subset:
     $\wedge \text{trace-table composition-table as.}$ 
     $\text{?good-sets trace-table composition-table as} \subseteq \text{query-sample-space}$ 
    by simp
  have frac:
     $\wedge \text{trace-table composition-table as.}$ 
     $\text{nnreal } (\text{card } (\text{?good-sets trace-table composition-table as})) /$ 
     $\text{nnreal } (\text{card query-sample-space}) \leq \text{query-error-bound}$ 
    by simp
  have zero-bound:
     $\text{wp-event verify-monad } (\text{query-index-set-hit } s \text{ ?good-sets}) \ s \leq 0$ 
    using fresh subset frac rounds-zero
    unfolding query-index-freshness-assumption-def by simp
  have zero:
     $\text{wp-event verify-monad } (\text{query-index-set-hit } s \text{ ?good-sets}) \ s = 0$ 
    by (rule antisym[OF zero-bound]) simp
  show False
    using positive zero by simp
qed

lemma query-bad-imp-query-index-set-hit:
  assumes query-bad s out
  shows query-index-set-hit s query-sampling-success-space out
proof –
  from assms obtain trace-table composition-table as query-idxs
  where bound:
    accepted-with-bound-tables s out trace-table composition-table as query-idxs
  and trace-low: trace-table-low-degree trace-table
  and composition-low:
    composition-table-low-degree maxDegree composition-table
  and not-all:
     $\neg \text{all-queries-consistent trace-table composition-table as}$ 
  unfolding query-bad-def by blast
  have miss:
    query-samples-miss-disagreements trace-table composition-table as query-idxs
    by (rule accepted-with-bound-tables-query-samples-miss-if-not-all-consistent
      [OF bound not-all])
  have success-space:

```

```

    query-sampling-success-space trace-table composition-table as =
      query-agreement-indices trace-table composition-table as
using trace-low composition-low not-all
unfolding query-sampling-success-space-def by simp
have samples:
  set query-idxs  $\subseteq$ 
    query-sampling-success-space trace-table composition-table as
using miss unfolding query-samples-miss-disagreements-def success-space by simp
show ?thesis
  unfolding query-index-set-hit-def
  using bound samples by blast
qed

```

```

lemma query-bad-imp-query-index-round-set-hit:
  assumes query-bad s out
  shows query-index-round-set-hit s query-sampling-success-space out
by (rule query-index-set-hit-imp-round-set-hit)
  (rule query-bad-imp-query-index-set-hit[OF assms])

```

```

lemma query-bad-bound-from-phase4B:
  assumes fresh: query-index-freshness-assumption s
  shows wp-event verify-monad (query-bad s) s  $\leq$ 
    nnreal rounds * query-error-bound
proof –
  have hit-bound:
    wp-event verify-monad
      (query-index-set-hit s query-sampling-success-space) s  $\leq$ 
        nnreal rounds * query-error-bound
  using fresh query-sampling-success-space-subset
    query-sampling-success-space-fraction-bound-query-sample-space
  unfolding query-index-freshness-assumption-def
  by blast
  have wp-event verify-monad (query-bad s) s  $\leq$ 
    wp-event verify-monad
      (query-index-set-hit s query-sampling-success-space) s
  by (rule wp-event-mono) (rule query-bad-imp-query-index-set-hit)
  also have ...  $\leq$  nnreal rounds * query-error-bound
  by (rule hit-bound)
  finally show ?thesis .
qed

```

```

lemma accepted-bound-trace-low-degree-or-trace-fri-bad:
  assumes bound:
    accepted-with-bound-tables s out trace-table composition-table as query-idxs
  shows trace-table-low-degree trace-table  $\vee$  trace-fri-bad s out
proof (cases trace-table-low-degree trace-table)
  case True
  then show ?thesis by simp
next

```



```

case False
then have trace-fri-bad s out
  unfolding trace-fri-bad-def using bound by blast
then show ?thesis by simp
qed

lemma accepted-bound-composition-low-degree-or-composition-fri-bad:
  assumes bound:
    accepted-with-bound-tables s out trace-table composition-table as query-idxs
    and trace-low: trace-table-low-degree trace-table
  shows composition-table-low-degree maxDegree composition-table  $\vee$ 
    composition-fri-bad s out
proof (cases composition-table-low-degree maxDegree composition-table)
  case True
    then show ?thesis by simp
  next
    case False
    then have composition-fri-bad s out
      unfolding composition-fri-bad-def using bound trace-low by blast
    then show ?thesis by simp
  qed

lemma accepted-bound-all-queries-consistent-or-query-bad:
  assumes bound:
    accepted-with-bound-tables s out trace-table composition-table as query-idxs
    and trace-low: trace-table-low-degree trace-table
    and composition-low: composition-table-low-degree maxDegree composition-table
  shows all-queries-consistent trace-table composition-table as  $\vee$ 
    query-bad s out
proof (cases all-queries-consistent trace-table composition-table as)
  case True
    then show ?thesis by simp
  next
    case False
    then have query-bad s out
      unfolding query-bad-def using bound trace-low composition-low by blast
    then show ?thesis by simp
  qed

lemma accepted-bound-composition-bad-if-globally-consistent:
  assumes false-statement:  $\neg$  exists-valid-trace
  and bound:
    accepted-with-bound-tables s out trace-table composition-table as query-idxs
    and trace-low: trace-table-low-degree trace-table
    and composition-low: composition-table-low-degree maxDegree composition-table
    and all-queries: all-queries-consistent trace-table composition-table as
  shows composition-bad s out
proof –
  from trace-low obtain f where

```

```

deg-f: degree f < clength
and trace-table: trace-table = map (poly f) eval-domain
unfolding trace-table-low-degree-def by blast
have violated: violated-constraints f ≠ {}
  by (rule false-statement-violated-constraints[OF false-statement deg-f])
have len-as: length as = length spec
  using accepted-with-tables-shapes(3)
  [OF accepted-with-bound-tables-imp-accepted-with-tables[OF bound]]
  by simp
have hide-if-bounds:
  common-denominator-degree-bounds f as ⇒
    random-combination-common-denominator-hides-violations f as
  by (rule random-combination-common-denominator-hides-from-queries
    [OF len-as false-statement deg-f trace-table composition-low all-queries])
show ?thesis
  unfolding composition-bad-def composition-degree-bad-def
    composition-randomization-bad-def composition-bad-context-def
  using bound deg-f trace-table violated composition-low all-queries hide-if-bounds
  by blast
qed

lemma accepted-bound-table-partition:
  assumes false-statement: ¬ exists-valid-trace
  and bound:
    accepted-with-bound-tables s out trace-table composition-table as query-idxs
  shows
    composition-bad s out ∨ trace-fri-bad s out ∨
    composition-fri-bad s out ∨ query-bad s out
proof -
  from accepted-bound-trace-low-degree-or-trace-fri-bad[OF bound]
  consider
    (trace-low) trace-table-low-degree trace-table
  | (trace-bad) trace-fri-bad s out
  by blast
  then show ?thesis
proof cases
  case trace-bad
  then show ?thesis by blast
next
  case trace-low
  from accepted-bound-composition-low-degree-or-composition-fri-bad
  [OF bound trace-low]
  consider
    (composition-low) composition-table-low-degree maxDegree composition-table
  | (composition-bad) composition-fri-bad s out
  by blast
  then show ?thesis
proof cases
  case composition-bad

```

```

    then show ?thesis by blast
next
case composition-low
from accepted-bound-all-queries-consistent-or-query-bad
  [OF bound trace-low composition-low]
consider
  (all-queries) all-queries-consistent trace-table composition-table as
  | (query-bad) query-bad s out
  by blast
then show ?thesis
proof cases
case query-bad
  then show ?thesis by blast
next
case all-queries
  have composition-bad s out
    by (rule accepted-bound-composition-bad-if-globally-consistent
      [OF false-statement bound trace-low composition-low all-queries])
  then show ?thesis by blast
qed
qed
qed
qed

lemma composition-randomization-bad-imp-alpha-bad-set-hit:
  assumes bad: composition-randomization-bad s out
  shows alpha-bad-set-hit s composition-trace-bad-alpha-space out
proof -
  from bad obtain trace-table composition-table as query-idxs f where
    ctx:
      composition-bad-context s out trace-table composition-table as query-idxs f
  and bounds: common-denominator-degree-bounds f as
  and hides: random-combination-common-denominator-hides-violations f as
  unfolding composition-randomization-bad-def by blast
  have bound:
    accepted-with-bound-tables s out trace-table composition-table as query-idxs
  using ctx unfolding composition-bad-context-def by blast
  have deg-f: degree f < clength
  using ctx unfolding composition-bad-context-def by blast
  have trace-table: trace-table = map (poly f) eval-domain
  using ctx unfolding composition-bad-context-def by blast
  have violated: violated-constraints f ≠ {}
  using ctx unfolding composition-bad-context-def by blast
  have len-as: length as = length spec
  using accepted-with-tables-shapes(3)
  [OF accepted-with-bound-tables-imp-accepted-with-tables[OF bound]] .
  have as-in:
    as ∈ common-denominator-hiding-alpha-space f
  using len-as hides

```

```

unfolding common-denominator-hiding-alpha-space-def alpha-space-def
by simp
have witness-eq: low-degree-trace-witness trace-table = f
  by (rule low-degree-trace-witness-eq[OF deg-f trace-table])
have as ∈ composition-trace-bad-alpha-space trace-table
  using deg-f trace-table violated bounds as-in
  unfolding composition-trace-bad-alpha-space-def witness-eq
  by simp
then show ?thesis
  using bound unfolding alpha-bad-set-hit-def by blast
qed

```

```

lemma composition-randomization-bad-bound-from-alpha-freshness:
  assumes fresh: alpha-challenge-freshness-assumption s
  shows wp-event verify-monad (composition-randomization-bad s) s ≤
    composition-error-bound
proof —
  have hit-bound:
    wp-event verify-monad
      (alpha-bad-set-hit s composition-trace-bad-alpha-space) s ≤
        composition-error-bound
  using fresh composition-trace-bad-alpha-space-subset-alpha-space
    composition-trace-bad-alpha-space-fraction-bound-alpha-space
  unfolding alpha-challenge-freshness-assumption-def
  by blast
  have wp-event verify-monad (composition-randomization-bad s) s ≤
    wp-event verify-monad
      (alpha-bad-set-hit s composition-trace-bad-alpha-space) s
  by (rule wp-event-mono)
    (rule composition-randomization-bad-imp-alpha-bad-set-hit)
  also have ... ≤ composition-error-bound
  by (rule hit-bound)
  finally show ?thesis .
qed

```

```

lemma composition-randomization-bad-bound-via-header-union:
  assumes future: alpha-future-fresh s
  and union-bound:
     $\bigwedge_{fr} f \text{ fri-roots } f \text{ final.}$ 
    nnreal
      (card
        (alpha-header-union-bad-sets s
          (composition-trace-bad-alpha-space fr f-fri-roots f-final))) /
      nnreal (card alpha-space) ≤ composition-error-bound
  shows wp-event verify-monad (composition-randomization-bad s) s ≤
    composition-error-bound
proof —
  have hit-bound:
    wp-event verify-monad

```

$(\text{alpha-bad-set-hit } s \text{ composition-trace-bad-alpha-space}) s \leq$
 $\text{composition-error-bound}$
by (rule *wp-alpha-bad-set-hit-bound-via-header-union*[*OF future*])
 $(\text{use composition-trace-bad-alpha-space-subset-alpha-space union-bound in}$
 $\text{simp-all})$
have *wp-event verify-monad* (*composition-randomization-bad s*) $s \leq$
 $\text{wp-event verify-monad}$
 $(\text{alpha-bad-set-hit } s \text{ composition-trace-bad-alpha-space}) s$
by (rule *wp-event-mono*)
 $(\text{rule composition-randomization-bad-imp-alpha-bad-set-hit})$
also have $\dots \leq \text{composition-error-bound}$
by (rule *hit-bound*)
finally show *?thesis* .
qed

lemma *composition-randomization-bad-bound-via-supported-header-union:*

assumes *future: alpha-future-fresh s*

and *union-bound:*

$\bigwedge \text{fr } f\text{-fri-roots } f\text{-final.}$

nnreal

$(\text{card}$

$(\text{alpha-header-supported-union-bad-sets } s$

$\text{composition-trace-bad-alpha-space fr } f\text{-fri-roots } f\text{-final})) /$

$\text{nnreal } (\text{card } \text{alpha-space}) \leq \text{composition-error-bound}$

shows *wp-event verify-monad* (*composition-randomization-bad s*) $s \leq$
 $\text{composition-error-bound}$

proof –

have *hit-bound:*

wp-event verify-monad

$(\text{alpha-bad-set-hit } s \text{ composition-trace-bad-alpha-space}) s \leq$
 $\text{composition-error-bound}$

by (rule *wp-alpha-bad-set-hit-bound-via-supported-header-union*[*OF future*])
 $(\text{use composition-trace-bad-alpha-space-subset-alpha-space union-bound in}$
 $\text{simp-all})$

have *wp-event verify-monad* (*composition-randomization-bad s*) $s \leq$
 $\text{wp-event verify-monad}$

$(\text{alpha-bad-set-hit } s \text{ composition-trace-bad-alpha-space}) s$

by (rule *wp-event-mono*)

$(\text{rule composition-randomization-bad-imp-alpha-bad-set-hit})$

also have $\dots \leq \text{composition-error-bound}$

by (rule *hit-bound*)

finally show *?thesis* .

qed

lemma *composition-randomization-bad-bound-if-alpha-header-candidate-unique:*

assumes *future: alpha-future-fresh s*

and *unique:*

$\bigwedge \text{fr } f\text{-fri-roots } f\text{-final.}$

$\exists \text{ trace-table.}$

$\text{alpha-header-trace-table-candidates } s \text{ fr } f\text{-fri-roots } f\text{-final} \subseteq \{\text{trace-table}\}$
shows $\text{wp-event verify-monad } (\text{composition-randomization-bad } s) \text{ } s \leq \text{composition-error-bound}$
by (rule $\text{composition-randomization-bad-bound-via-header-union}[OF \text{ future}]$)
(rule $\text{composition-trace-alpha-header-union-fraction-bound-if-unique-candidate}[OF \text{ unique}]$)

lemma $\text{composition-randomization-bad-bound-if-alpha-header-supported-candidate-unique}$:
assumes $\text{future: alpha-future-fresh } s$
and unique :
 $\bigwedge \text{fr } f\text{-fri-roots } f\text{-final.}$
 $\exists \text{ trace-table.}$
 $\text{alpha-header-supported-trace-table-candidates } s \text{ fr } f\text{-fri-roots } f\text{-final} \subseteq \{\text{trace-table}\}$
shows $\text{wp-event verify-monad } (\text{composition-randomization-bad } s) \text{ } s \leq \text{composition-error-bound}$
by (rule $\text{composition-randomization-bad-bound-via-supported-header-union}[OF \text{ future}]$)
(rule $\text{composition-trace-alpha-header-supported-union-fraction-bound-if-unique-candidate}[OF \text{ unique}]$)

lemma $\text{composition-randomization-bad-bound-if-no-alpha-supported-pairwise-merkle-bad}$:
assumes $\text{future: alpha-future-fresh } s$
and $\text{no-bad: } \neg \text{alpha-supported-pairwise-merkle-bad } s$
shows $\text{wp-event verify-monad } (\text{composition-randomization-bad } s) \text{ } s \leq \text{composition-error-bound}$
proof (rule $\text{composition-randomization-bad-bound-if-alpha-header-supported-candidate-unique}[OF \text{ future}]$)
fix $\text{fr } f\text{-fri-roots } f\text{-final}$
show
 $\exists \text{ trace-table.}$
 $\text{alpha-header-supported-trace-table-candidates } s \text{ fr } f\text{-fri-roots } f\text{-final} \subseteq \{\text{trace-table}\}$
by (rule $\text{alpha-header-supported-candidate-unique-if-no-global-pairwise-merkle-bad}[OF \text{ no-bad}]$)

qed

lemma $\text{composition-degree-bad-false}$:
assumes $\text{wf: spec-degree-wellformed}$
shows $\neg \text{composition-degree-bad } s \text{ out}$
using $\text{common-denominator-degree-bounds-from-spec-degree-wellformed}[OF \text{ wf}]$
unfolding $\text{composition-degree-bad-def } \text{composition-bad-context-def}$
by blast

lemma $\text{composition-degree-bad-bound}$:
assumes $\text{wf: spec-degree-wellformed}$
shows $\text{wp-event verify-monad } (\text{composition-degree-bad } s) \text{ } s \leq 0$

```

proof –
  have composition-degree-bad s =
    ( $\lambda out :: (unit\ list \times ('f, 'a)\ protocol-channel-scheme)\ option.\ False$ )
  by (rule ext) (simp add: composition-degree-bad-false[OF wf])
  then show ?thesis
    unfolding wp-event-def wp-def dist-expect-def by simp
qed

lemma composition-bad-bound-from-phase4A:
  assumes wf: spec-degree-wellformed
  and alpha-fresh: alpha-challenge-freshness-assumption s
  shows wp-event verify-monad (composition-bad s) s ≤ composition-error-bound
proof –
  have degree-bound:
    wp-event verify-monad (composition-degree-bad s) s ≤ 0
  by (rule composition-degree-bad-bound[OF wf])
  have randomization-bound:
    wp-event verify-monad (composition-randomization-bad s) s ≤
      composition-error-bound
  by (rule composition-randomization-bad-bound-from-alpha-freshness[OF alpha-fresh])
  have wp-event verify-monad (composition-bad s) s =
    wp-event verify-monad
      ( $\lambda out.\ composition-degree-bad\ s\ out \vee$ 
       composition-randomization-bad s out) s
  unfolding composition-bad-def by simp
  also have  $\dots \leq$ 
    wp-event verify-monad (composition-degree-bad s) s +
    wp-event verify-monad (composition-randomization-bad s) s
  by (rule wp-event-union-bound)
  also have  $\dots \leq 0 + composition-error-bound$ 
  by (intro add-mono degree-bound randomization-bound)
  finally show ?thesis by simp
qed

end

end

theory Soundness-FRI
  imports
    Soundness-Bad-Events
begin

context soundness
begin

```

0.0.2 Algebraic FRI Soundness Interface

The next definitions isolate the algebraic content of the verifier's FRI fold equation from transcript extraction and Merkle authentication. They are intentionally protocol-independent except for using the same affine fold as *receive-query-commits*.

definition *fri-table-fold-value*

$:: 'f \Rightarrow 'f \text{ list} \Rightarrow 'f \text{ list} \Rightarrow \text{nat} \Rightarrow \text{nat} \Rightarrow \text{nat} \Rightarrow 'f$

where

$\text{fri-table-fold-value } b \text{ layer fri-dom len pw } i =$
 $\text{fri-fold-value } b \text{ (layer ! } i \text{) (layer ! fri-sibling-index len } i \text{)}$
 $(\text{fri-fold-denominator (fri-dom ! } i \text{) } 1)$

definition *fri-algebraic-layer-relation*

$:: \text{nat} \Rightarrow \text{nat} \Rightarrow 'f \Rightarrow 'f \text{ list} \Rightarrow 'f \text{ list} \Rightarrow$

$'f \text{ list} \Rightarrow 'f \text{ list} \Rightarrow \text{bool}$

where

$\text{fri-algebraic-layer-relation len pw } b \text{ fri-dom layer next-dom}$
 $\text{next-layer} \longleftrightarrow$
 $0 < \text{len} \wedge$
 $\text{len} \leq \text{length fri-dom} \wedge$
 $\text{len} \leq \text{length layer} \wedge$
 $\text{len div } 2 \leq \text{length next-dom} \wedge$
 $\text{len div } 2 \leq \text{length next-layer} \wedge$
 $(\forall i < \text{len div } 2.$
 $\text{next-dom ! } i = (\text{fri-dom ! } i)^2 \wedge$
 $\text{next-layer ! } i = \text{fri-table-fold-value } b \text{ layer fri-dom len pw } i)$

lemma *fri-algebraic-layer-relationD:*

assumes $\text{fri-algebraic-layer-relation len pw } b \text{ fri-dom layer next-dom}$

next-layer

and $i < \text{len div } 2$

shows $\text{next-dom ! } i = (\text{fri-dom ! } i)^2$

and $\text{next-layer ! } i = \text{fri-table-fold-value } b \text{ layer fri-dom len pw } i$

using *assms unfolding fri-algebraic-layer-relation-def by auto*

definition *fri-opening-matches-table*

$:: \text{nat} \Rightarrow \text{nat} \Rightarrow 'f \text{ list} \Rightarrow 'f \Rightarrow 'f \Rightarrow \text{bool}$

where

$\text{fri-opening-matches-table len idx layer xp xn} \longleftrightarrow$
 $\text{idx} < \text{len} \wedge$
 $\text{len} \leq \text{length layer} \wedge$
 $\text{layer ! idx} = \text{xp} \wedge$
 $\text{layer ! fri-sibling-index len idx} = \text{xn}$

lemma *fri-opening-matches-tableD:*

assumes $\text{fri-opening-matches-table len idx layer xp xn}$

shows $\text{idx} < \text{len}$

and $\text{len} \leq \text{length layer}$

and $\text{layer ! idx} = \text{xp}$

and $\text{layer} \mid \text{fri-sibling-index len idx} = \text{xn}$
using *assms* **unfolding** *fri-opening-matches-table-def* **by** *auto*

lemma *fri-fold-value-eq-table-fold-value*:
assumes *dom-idx*: $\text{fri-dom} \mid \text{idx} = (h \hat{\ } \text{idx} * \text{shift}) \hat{\ } \text{pw}$
and *match*: $\text{fri-opening-matches-table len idx layer xp xn}$
shows
 $\text{fri-fold-value } b \text{ xp xn } (\text{fri-fold-denominator } (h \hat{\ } \text{idx} * \text{shift}) \text{ pw}) =$
 $\text{fri-table-fold-value } b \text{ layer fri-dom len pw idx}$
using *dom-idx* *fri-opening-matches-tableD(3,4)[OF match]*
unfolding *fri-table-fold-value-def* *fri-fold-denominator-def* **by** *simp*

lemma *fri-layer-opening-step-outcome-fold-table-if-opening-matches*:
assumes *outcome*:
 $\text{Some } (\text{out}, t) \in$
 $\text{set-dist } (\text{execute } (\text{fri-layer-opening-step } (b, f) (i, x, \text{len}, \text{pw})) s)$
and *dom-i*: $\text{fri-dom} \mid i = (h \hat{\ } i * \text{shift}) \hat{\ } \text{pw}$
obtains *xp xp-path xn xn-path x'*
where
 $\text{out} = (i \bmod (\text{len div } 2), x', \text{len div } 2, \text{pw} + \text{pw})$
and $\text{xp} = x$
and *fri-layer-opening-chunk len xp xp-path xn xn-path*
 $([xp] @ \text{xp-path} @ [xn] @ \text{xn-path})$
and *PTranscript s* = $([xp] @ \text{xp-path} @ [xn] @ \text{xn-path}) @ \text{PTranscript } t$
and *PState t* = $\text{foldl concat } (\text{PState } s) ([xp] @ \text{xp-path} @ [xn] @ \text{xn-path})$
and $s \leq t$
and *fri-opening-matches-table len i layer xp xn* $\implies$
 $x' = \text{fri-table-fold-value } b \text{ layer fri-dom len pw } i$

proof –
from *fri-layer-opening-step-outcome[OF outcome]*
obtain *xp xp-path xn xn-path x'* **where**
 $\text{out-eq: out} = (i \bmod (\text{len div } 2), x', \text{len div } 2, \text{pw} + \text{pw})$
and *xp-eq*: $\text{xp} = x$
and *x'-eq*:
 $x' = \text{fri-fold-value } b \text{ xp xn}$
 $(\text{fri-fold-denominator } ((h \hat{\ } i) * \text{shift}) \text{ pw})$
and *chunk*:
 $\text{fri-layer-opening-chunk len xp xp-path xn xn-path}$
 $([xp] @ \text{xp-path} @ [xn] @ \text{xn-path})$
and *tr*:
 $\text{PTranscript } s = ([xp] @ \text{xp-path} @ [xn] @ \text{xn-path}) @ \text{PTranscript } t$
and *st*:
 $\text{PState } t = \text{foldl concat } (\text{PState } s) ([xp] @ \text{xp-path} @ [xn] @ \text{xn-path})$
and *ext*: $s \leq t$
by *blast*
have *table-fold*:
 $\text{fri-opening-matches-table len i layer xp xn} \implies$
 $x' = \text{fri-table-fold-value } b \text{ layer fri-dom len pw } i$
proof –

```

assume match: fri-opening-matches-table len i layer xp xn
have fri-fold-value b xp xn
  (fri-fold-denominator ((h  $\wedge$  i) * shift) pw) =
  fri-table-fold-value b layer fri-dom len pw i
  by (rule fri-fold-value-eq-table-fold-value[OF dom-i match])
then show x' = fri-table-fold-value b layer fri-dom len pw i
  using x'-eq by simp
qed
show ?thesis
  by (rule that[OF out-eq xp-eq chunk tr st ext table-fold])
qed

definition fri-fold-coeff-pair :: 'f  $\Rightarrow$  'f  $\Rightarrow$  'f  $\Rightarrow$  'f  $\times$  'f
where
  fri-fold-coeff-pair xp xn denom =
    ((xp + xn) div 2, (xp - xn) div denom)

definition fri-local-hiding-challenges
  :: 'f  $\Rightarrow$  'f  $\Rightarrow$  'f  $\Rightarrow$  'f  $\Rightarrow$  'f  $\Rightarrow$  'f set
where
  fri-local-hiding-challenges xp xn yp yn denom =
    {b. fri-fold-value b xp xn denom = fri-fold-value b yp yn denom}

lemma fri-affine-collision-challenges-card-le-one:
  fixes a c a' c' :: 'f
  assumes pair-diff: (a, c)  $\neq$  (a', c')
  shows card {b. a + b * c = a' + b * c'}  $\leq$  (1::nat)
proof -
  show ?thesis
proof (cases c = c')
  case True
  then have a-diff: a  $\neq$  a'
    using pair-diff by auto
  have {b. a + b * c = a' + b * c'} = {}
  proof (rule ccontr)
  assume {b. a + b * c = a' + b * c'}  $\neq$  {}
  then obtain b where
    b-in: b  $\in$  {b. a + b * c = a' + b * c'}
    by blast
  then have eq: a + b * c = a' + b * c'
    by simp
  then have a + b * c' = a' + b * c'
    using True by simp
  then have (a + b * c') - b * c' = (a' + b * c') - b * c'
    by simp
  then have a = a'
    by simp
  then show False
    using a-diff by contradiction

```

```

qed
then show ?thesis by simp
next
case False
have subset:
  {b. a + b * c = a' + b * c'} ⊆
  {(a' - a) div (c - c')}
proof
  fix b
  assume b-in: b ∈ {b. a + b * c = a' + b * c'}
  then have eq: a + b * c = a' + b * c'
    by simp
  have b * (c - c') = a' - a
    using eq by (simp add: algebra-simps)
  then have b = (a' - a) div (c - c')
    using False by (simp add: field-simps)
  then show b ∈ {(a' - a) div (c - c')}
    by simp
qed
have card {b. a + b * c = a' + b * c'} ≤
  card {(a' - a) div (c - c')}
  by (rule card-mono) (use subset in auto)
then show ?thesis by simp
qed
qed

lemma fri-local-hiding-challenges-card-le-one:
  assumes pair-diff:
    fri-fold-coeff-pair xp xn denom ≠ fri-fold-coeff-pair yp yn denom
  shows card (fri-local-hiding-challenges xp xn yp yn denom) ≤ 1
proof -
  let ?a = (xp + xn) div 2
  let ?c = (xp - xn) div denom
  let ?a' = (yp + yn) div 2
  let ?c' = (yp - yn) div denom
  have pair-diff': (?a, ?c) ≠ (?a', ?c')
    using pair-diff unfolding fri-fold-coeff-pair-def by simp
  have fri-local-hiding-challenges xp xn yp yn denom =
    {b. ?a + b * ?c = ?a' + b * ?c'}
  unfolding fri-local-hiding-challenges-def fri-fold-value-def by simp
  then show ?thesis
    using fri-affine-collision-challenges-card-le-one[OF pair-diff'] by simp
qed

```

```

lemma fri-one-step-disagreement-hiding-bound:
  assumes
    fri-fold-coeff-pair xp xn denom ≠ fri-fold-coeff-pair yp yn denom
  shows card (fri-local-hiding-challenges xp xn yp yn denom) ≤ 1
  by (rule fri-local-hiding-challenges-card-le-one[OF assms])

```

lemma *fri-one-step-disagreement-outside-hiding*:
assumes $b \notin \text{fri-local-hiding-challenges } xp \ xn \ yp \ yn \ \text{denom}$
shows $\text{fri-fold-value } b \ xp \ xn \ \text{denom} \neq \text{fri-fold-value } b \ yp \ yn \ \text{denom}$
using *assms* **unfolding** *fri-local-hiding-challenges-def* **by** *simp*

definition *fri-table-local-hiding-challenges*
 $:: 'f \text{ list} \Rightarrow 'f \text{ list} \Rightarrow 'f \text{ list} \Rightarrow \text{nat} \Rightarrow \text{nat} \Rightarrow \text{nat} \Rightarrow$
 $'f \text{ set}$
where
 $\text{fri-table-local-hiding-challenges } \text{layer } \text{claimed } \text{fri-dom } \text{len } \text{pw } i =$
 $\text{fri-local-hiding-challenges}$
 $(\text{layer } ! i) (\text{layer } ! \text{fri-sibling-index } \text{len } i)$
 $(\text{claimed } ! i) (\text{claimed } ! \text{fri-sibling-index } \text{len } i)$
 $(\text{fri-fold-denominator } (\text{fri-dom } ! i) \text{pw})$

lemma *fri-table-local-hiding-challenges-card-le-one*:
assumes
 $\text{fri-fold-coeff-pair}$
 $(\text{layer } ! i) (\text{layer } ! \text{fri-sibling-index } \text{len } i)$
 $(\text{fri-fold-denominator } (\text{fri-dom } ! i) \text{pw}) \neq$
 $\text{fri-fold-coeff-pair}$
 $(\text{claimed } ! i) (\text{claimed } ! \text{fri-sibling-index } \text{len } i)$
 $(\text{fri-fold-denominator } (\text{fri-dom } ! i) \text{pw})$
shows $\text{card } (\text{fri-table-local-hiding-challenges } \text{layer } \text{claimed } \text{fri-dom } \text{len } \text{pw } i)$
 ≤ 1
unfolding *fri-table-local-hiding-challenges-def*
by (*rule* *fri-local-hiding-challenges-card-le-one*[*OF* *assms*])

definition *fri-one-step-disagreement-challenges*
 $:: \text{nat} \Rightarrow \text{nat} \Rightarrow 'f \text{ list} \Rightarrow 'f \text{ list} \Rightarrow 'f \text{ list} \Rightarrow 'f \text{ set}$
where
 $\text{fri-one-step-disagreement-challenges } \text{len } \text{pw } \text{layer } \text{claimed } \text{fri-dom} =$
 $\{b. \exists i < \text{len div } 2.$
 $\text{fri-fold-coeff-pair}$
 $(\text{layer } ! i) (\text{layer } ! \text{fri-sibling-index } \text{len } i)$
 $(\text{fri-fold-denominator } (\text{fri-dom } ! i) \text{pw}) \neq$
 $\text{fri-fold-coeff-pair}$
 $(\text{claimed } ! i) (\text{claimed } ! \text{fri-sibling-index } \text{len } i)$
 $(\text{fri-fold-denominator } (\text{fri-dom } ! i) \text{pw}) \wedge$
 $b \in \text{fri-table-local-hiding-challenges } \text{layer } \text{claimed } \text{fri-dom } \text{len } \text{pw } i\}$

lemma *fri-one-step-disagreement-challenges-card-bound*:
 $\text{card } (\text{fri-one-step-disagreement-challenges } \text{len } \text{pw } \text{layer } \text{claimed } \text{fri-dom})$
 $\leq \text{CARD}('f)$
proof –
have *finite-univ*: *finite* (*UNIV* :: *'f* *set*)
by *simp*
have $\text{card } (\text{fri-one-step-disagreement-challenges } \text{len } \text{pw } \text{layer } \text{claimed } \text{fri-dom})$

```

    ≤ card (UNIV :: 'f set)
  by (rule card-mono[OF finite-univ]) auto
then show ?thesis by simp
qed

lemma fri-one-step-disagreement-challenges-card-bound-local:
  card (fri-one-step-disagreement-challenges len pw layer claimed fri-dom)
    ≤ len div 2
proof -
  let ?S =
    λi. if
      fri-fold-coeff-pair
        (layer ! i) (layer ! fri-sibling-index len i)
        (fri-fold-denominator (fri-dom ! i) pw) ≠
      fri-fold-coeff-pair
        (claimed ! i) (claimed ! fri-sibling-index len i)
        (fri-fold-denominator (fri-dom ! i) pw)
      then fri-table-local-hiding-challenges layer claimed fri-dom len pw i
      else {}
  have set-eq:
    fri-one-step-disagreement-challenges len pw layer claimed fri-dom =
      (⋃ i ∈ {..

```

```

      card ( $\bigcup i \in \{..<len \text{ div } 2\}. ?S i$ )
    using set-eq by simp
  also have ...  $\leq (\sum i \in \{..<len \text{ div } 2\}. card (?S i))$ 
    by (rule card-UN-le) simp
  also have ...  $\leq (\sum i \in \{..<len \text{ div } 2\}. 1)$ 
    by (rule sum-mono) (rule local-bound, simp)
  also have ... = len div 2
    by simp
  finally show ?thesis .
qed

```

definition *fri-multiround-bad-challenge-lists*
 $:: nat \Rightarrow (nat \Rightarrow 'f \text{ list} \Rightarrow 'f \text{ set}) \Rightarrow 'f \text{ list set}$
where
fri-multiround-bad-challenge-lists $n \text{ bad} =$
 $\{bs \in \text{fri-challenge-space } n.$
 $\exists i < n. bs ! i \in \text{bad } i \text{ (take } i \text{ bs)}\}$

definition *fri-bad-challenge-lists-bounded-by*
 $:: nat \Rightarrow 'f \text{ list set} \Rightarrow nat \Rightarrow bool$
where
fri-bad-challenge-lists-bounded-by $n \text{ bad } bd \longleftrightarrow$
 $\text{bad} \subseteq \text{fri-challenge-space } n \wedge \text{card bad} \leq bd$

lemma *fri-multiround-bad-challenge-lists-subset*:
fri-multiround-bad-challenge-lists $n \text{ bad} \subseteq \text{fri-challenge-space } n$
unfolding *fri-multiround-bad-challenge-lists-def* **by** *auto*

lemma *fri-multiround-bad-challenge-lists-card-bound*:
 $\text{card} (\text{fri-multiround-bad-challenge-lists } n \text{ bad}) \leq \text{CARD}('f) \wedge n$
proof –
have $\text{card} (\text{fri-multiround-bad-challenge-lists } n \text{ bad}) \leq$
 $\text{card} (\text{fri-challenge-space } n)$
by (rule card-mono[OF *finite-fri-challenge-space*])
(auto simp: *fri-multiround-bad-challenge-lists-subset*)
also have ... = $\text{CARD}('f) \wedge n$
by (rule card-fri-challenge-space)
finally show ?thesis .
qed

lemma *fri-multiround-bad-challenge-lists-bounded-by*:
fri-bad-challenge-lists-bounded-by n
 $(\text{fri-multiround-bad-challenge-lists } n \text{ bad}) (\text{CARD}('f) \wedge n)$
unfolding *fri-bad-challenge-lists-bounded-by-def*
using *fri-multiround-bad-challenge-lists-subset*
fri-multiround-bad-challenge-lists-card-bound
by *simp*

definition *fri-final-constant-consistent* $:: 'f \text{ list} \Rightarrow 'f \Rightarrow bool$

where

$fri_final_constant_consistent\ layer\ final \longleftrightarrow$
 $(\forall x \in set\ layer. x = final)$

lemma *fri-final-constant-consistent-nth*:

assumes *fri-final-constant-consistent layer final*
and $i < length\ layer$
shows $layer\ !\ i = final$
using *assms unfolding fri-final-constant-consistent-def*
by (*simp add: nth-mem*)

lemma *fri-final-constant-consistent-hd*:

assumes *fri-final-constant-consistent layer final*
and $layer \neq []$
shows $hd\ layer = final$
using *assms fri-final-constant-consistent-nth[of layer final 0]*
by (*cases layer*) *auto*

definition *fri-table-low-degree-on* :: $nat \Rightarrow 'f\ list \Rightarrow 'f\ list \Rightarrow bool$

where

$fri_table_low_degree_on\ d\ fri_dom\ table \longleftrightarrow$
 $(\exists p. degree\ p \leq d \wedge table = map\ (poly\ p)\ fri_dom)$

fun *fri-degree-after* :: $nat \Rightarrow nat \Rightarrow nat$

where

$fri_degree_after\ 0\ d = d$
 $| fri_degree_after\ (Suc\ n)\ d = fri_degree_after\ n\ d\ div\ 2$

lemma *fri-degree-after-Suc[simp]*:

$fri_degree_after\ (Suc\ n)\ d = fri_degree_after\ n\ d\ div\ 2$
by *simp*

definition *fri-one-round-proximity-step*

:: $nat \Rightarrow nat \Rightarrow nat \Rightarrow 'f\ list \Rightarrow 'f\ list \Rightarrow$
 $'f\ list \Rightarrow 'f\ list \Rightarrow 'f\ set \Rightarrow bool$

where

$fri_one_round_proximity_step\ d\ len\ pw\ fri_dom\ layer\ next_dom\ next_layer\ bad \longleftrightarrow$
 $(\forall b. b \notin bad \longrightarrow$
 $fri_algebraic_layer_relation\ len\ pw\ b\ fri_dom\ layer\ next_dom\ next_layer \longrightarrow$
 $fri_table_low_degree_on\ (d\ div\ 2)\ next_dom\ next_layer \longrightarrow$
 $fri_table_low_degree_on\ d\ fri_dom\ layer)$

lemma *fri-one-round-proximity-step-bad-or-next-not-low*:

assumes *step*:

$fri_one_round_proximity_step\ d\ len\ pw\ fri_dom\ layer\ next_dom\ next_layer\ bad$

and *relation*:

$fri_algebraic_layer_relation\ len\ pw\ b\ fri_dom\ layer\ next_dom\ next_layer$

and *current-not-low*: $\neg fri_table_low_degree_on\ d\ fri_dom\ layer$

and *challenge-not-bad*: $b \notin bad$

shows $\neg \text{fri-table-low-degree-on } (d \text{ div } 2) \text{ next-dom next-layer}$
using *assms* **unfolding** *fri-one-round-proximity-step-def* **by** *blast*

definition *fri-multiround-proximity-steps*

$:: \text{nat} \Rightarrow \text{nat} \Rightarrow \text{nat list} \Rightarrow \text{nat list} \Rightarrow$
 $\quad 'f \text{ list list} \Rightarrow 'f \text{ list list} \Rightarrow$
 $\quad (\text{nat} \Rightarrow 'f \text{ list} \Rightarrow 'f \text{ set}) \Rightarrow \text{bool}$

where

fri-multiround-proximity-steps *n d lens pws doms layers bad* $\longleftrightarrow$
 $\text{length lens} = n \wedge$
 $\text{length pws} = n \wedge$
 $\text{length doms} = \text{Suc } n \wedge$
 $\text{length layers} = \text{Suc } n \wedge$
 $(\forall bs \in \text{fri-challenge-space } n.$
 $\quad \forall i < n.$
 $\quad \text{fri-one-round-proximity-step } (\text{fri-degree-after } i \text{ } d)$
 $\quad (\text{lens } ! \text{ } i) (\text{pws } ! \text{ } i) (\text{doms } ! \text{ } i) (\text{layers } ! \text{ } i)$
 $\quad (\text{doms } ! \text{ } \text{Suc } i) (\text{layers } ! \text{ } \text{Suc } i) (\text{bad } i (\text{take } i \text{ } bs)))$

lemma *fri-multiround-bad-challenge-lists-notinD*:

assumes *challenges*: $bs \in \text{fri-challenge-space } n$
and *not-bad*: $bs \notin \text{fri-multiround-bad-challenge-lists } n \text{ bad}$
and *i-bound*: $i < n$
shows $bs ! i \notin \text{bad } i (\text{take } i \text{ } bs)$
using *assms* **unfolding** *fri-multiround-bad-challenge-lists-def* **by** *blast*

lemma *fri-multiround-proximity-skeleton*:

assumes *steps*: *fri-multiround-proximity-steps* *n d lens pws doms layers bad*
and *challenges*: $bs \in \text{fri-challenge-space } n$
and *relations*:
 $\bigwedge i. i < n \implies$
 $\quad \text{fri-algebraic-layer-relation } (\text{lens } ! \text{ } i) (\text{pws } ! \text{ } i) (bs ! i)$
 $\quad (\text{doms } ! \text{ } i) (\text{layers } ! \text{ } i) (\text{doms } ! \text{ } \text{Suc } i) (\text{layers } ! \text{ } \text{Suc } i)$
and *start-not-low*:
 $\neg \text{fri-table-low-degree-on } d (\text{doms } ! 0) (\text{layers } ! 0)$
and *final-low*:
 $\text{fri-table-low-degree-on } (\text{fri-degree-after } n \text{ } d) (\text{doms } ! n) (\text{layers } ! n)$
shows $bs \in \text{fri-multiround-bad-challenge-lists } n \text{ bad}$

proof (rule *ccontr*)

assume *not-bad*: $bs \notin \text{fri-multiround-bad-challenge-lists } n \text{ bad}$

have *step-at*:

$\bigwedge i. i < n \implies$
 $\quad \text{fri-one-round-proximity-step } (\text{fri-degree-after } i \text{ } d)$
 $\quad (\text{lens } ! \text{ } i) (\text{pws } ! \text{ } i) (\text{doms } ! \text{ } i) (\text{layers } ! \text{ } i)$
 $\quad (\text{doms } ! \text{ } \text{Suc } i) (\text{layers } ! \text{ } \text{Suc } i) (\text{bad } i (\text{take } i \text{ } bs))$

using *steps challenges* **unfolding** *fri-multiround-proximity-steps-def* **by** *blast*

have *not-low*:

$\bigwedge i. i \leq n \implies$
 $\quad \neg \text{fri-table-low-degree-on } (\text{fri-degree-after } i \text{ } d) (\text{doms } ! i) (\text{layers } ! i)$


```

proof –
  fix  $i$ 
  assume  $i \leq n$ 
  then show
     $\neg \text{fri-table-low-degree-on } (\text{fri-degree-after } i \ d) \ (doms \ ! \ i) \ (layers \ ! \ i)$ 
  proof (induction  $i$ )
    case 0
    then show ?case
    using start-not-low by simp
  next
    case (Suc  $i$ )
    have i-bound:  $i < n$ 
    using Suc.prems by simp
    have prev-not-low:
       $\neg \text{fri-table-low-degree-on } (\text{fri-degree-after } i \ d) \ (doms \ ! \ i) \ (layers \ ! \ i)$ 
    using Suc.IH Suc.prems by simp
    have challenge-not-bad:  $bs \ ! \ i \notin \text{bad } i \ (take \ i \ bs)$ 
    by (rule fri-multiround-bad-challenge-lists-notinD
      [OF challenges not-bad i-bound])
    have next-not-low:
       $\neg \text{fri-table-low-degree-on } (\text{fri-degree-after } i \ d \ \text{div } 2) \ (doms \ ! \ Suc \ i) \ (layers \ ! \ Suc \ i)$ 
    by (rule fri-one-round-proximity-step-bad-or-next-not-low
      [OF step-at[OF i-bound] relations[OF i-bound] prev-not-low
        challenge-not-bad])
    then show ?case
    by simp
  qed
qed
have  $\neg \text{fri-table-low-degree-on } (\text{fri-degree-after } n \ d) \ (doms \ ! \ n) \ (layers \ ! \ n)$ 
by (rule not-low) simp
then show False
using final-low by contradiction
qed

definition fri-round-count-for-degree-bound ::  $nat \Rightarrow nat$ 
  where fri-round-count-for-degree-bound  $d = \text{ceil-log } (d + 1)$ 

definition trace-fri-algebraic-round-count ::  $nat$ 
  where
    trace-fri-algebraic-round-count =
      fri-round-count-for-degree-bound (clength – 1)

definition composition-fri-algebraic-round-count ::  $'f \Rightarrow nat$ 
  where
    composition-fri-algebraic-round-count  $dg =$ 
      fri-round-count-for-degree-bound (to-nat  $dg$ )

```

lemma *trace-fri-algebraic-round-count-eq*:
 $\text{trace-fri-algebraic-round-count} = \text{ceil-log clength}$
unfolding *trace-fri-algebraic-round-count-def*
fri-round-count-for-degree-bound-def
using *clength-pos* **by** *simp*

lemma *composition-fri-algebraic-round-count-eq*:
 $\text{composition-fri-algebraic-round-count } dg =$
 $\text{ceil-log } (\text{to-nat } dg + 1)$
unfolding *composition-fri-algebraic-round-count-def*
fri-round-count-for-degree-bound-def
by *simp*

lemma *trace-fri-algebraic-challenge-space*:
 $\text{fri-challenge-space } \text{trace-fri-algebraic-round-count} =$
 $\text{fri-challenge-space } (\text{ceil-log clength})$
by (*simp add: trace-fri-algebraic-round-count-eq*)

lemma *composition-fri-algebraic-challenge-space*:
 $\text{fri-challenge-space } (\text{composition-fri-algebraic-round-count } dg) =$
 $\text{fri-challenge-space } (\text{ceil-log } (\text{to-nat } dg + 1))$
by (*simp add: composition-fri-algebraic-round-count-eq*)

definition *trace-fri-multiround-bad-sets*
 $:: ('f \text{ list} \Rightarrow \text{nat} \Rightarrow 'f \text{ list} \Rightarrow 'f \text{ set}) \Rightarrow$
 $'f \text{ list} \Rightarrow 'f \text{ list set}$
where
 $\text{trace-fri-multiround-bad-sets } \text{bad } \text{trace-table} =$
 $\text{fri-multiround-bad-challenge-lists } \text{trace-fri-algebraic-round-count}$
 $(\text{bad } \text{trace-table})$

definition *composition-fri-multiround-bad-sets*
 $:: ('f \Rightarrow 'f \text{ list} \Rightarrow \text{nat} \Rightarrow 'f \text{ list} \Rightarrow 'f \text{ set}) \Rightarrow$
 $'f \Rightarrow 'f \text{ list} \Rightarrow 'f \text{ list set}$
where
 $\text{composition-fri-multiround-bad-sets } \text{bad } dg \text{ composition-table} =$
 $\text{fri-multiround-bad-challenge-lists}$
 $(\text{composition-fri-algebraic-round-count } dg)$
 $(\text{bad } dg \text{ composition-table})$

lemma *trace-fri-multiround-bad-sets-subset*:
 $\text{trace-fri-multiround-bad-sets } \text{bad } \text{trace-table} \subseteq$
 $\text{fri-challenge-space } (\text{ceil-log clength})$
proof –
have $\text{trace-fri-multiround-bad-sets } \text{bad } \text{trace-table} \subseteq$
 $\text{fri-challenge-space } \text{trace-fri-algebraic-round-count}$
unfolding *trace-fri-multiround-bad-sets-def*
by (*rule fri-multiround-bad-challenge-lists-subset*)
also have $\dots = \text{fri-challenge-space } (\text{ceil-log clength})$

by (rule trace-fri-algebraic-challenge-space)
 finally show ?thesis .
 qed

lemma composition-fri-multiround-bad-sets-subset:
 composition-fri-multiround-bad-sets bad dg composition-table $\subseteq$
 fri-challenge-space (ceil-log (to-nat dg + 1))
proof –
 have composition-fri-multiround-bad-sets bad dg composition-table $\subseteq$
 fri-challenge-space (composition-fri-algebraic-round-count dg)
 unfolding composition-fri-multiround-bad-sets-def
 by (rule fri-multiround-bad-challenge-lists-subset)
 also have ... = fri-challenge-space (ceil-log (to-nat dg + 1))
 by (rule composition-fri-algebraic-challenge-space)
 finally show ?thesis .
 qed

lemma trace-fri-multiround-bad-sets-card-bound:
 card (trace-fri-multiround-bad-sets bad trace-table) $\leq$
 CARD('f) ^ ceil-log clength
 unfolding trace-fri-multiround-bad-sets-def
 using fri-multiround-bad-challenge-lists-card-bound
 by (simp add: trace-fri-algebraic-round-count-eq)

lemma composition-fri-multiround-bad-sets-card-bound:
 card (composition-fri-multiround-bad-sets bad dg composition-table) $\leq$
 CARD('f) ^ ceil-log (to-nat dg + 1)
 unfolding composition-fri-multiround-bad-sets-def
 using fri-multiround-bad-challenge-lists-card-bound
 by (simp add: composition-fri-algebraic-round-count-eq)

definition accepted-fri-challenges
 :: ('f, 'a) protocol-channel-scheme $\Rightarrow$
 (unit list $\times$ ('f, 'a) protocol-channel-scheme) option $\Rightarrow$
 'f list $\Rightarrow$ 'f $\Rightarrow$ 'f list $\Rightarrow$ bool
where
 accepted-fri-challenges s out trace-bs dg comp-bs $\longleftrightarrow$
 ($\exists$ result final-state fr f-fl f-final as fl final query-state.
 out = Some (result, final-state) $\wedge$
 verifier-header-transcript s fr (map snd f-fl) f-final as dg
 (map snd fl) final (PTranscript query-state) $\wedge$
 PState query-state =
 verifier-header-state s fr (map snd f-fl) f-final as dg
 (map snd fl) final $\wedge$
 Some (result, final-state) $\in$
 set-dist (execute
 (ntimes
 (verifier-query-round-program fr f-fl f-final as fl final)
 rounds)

```

    query-state) ∧
  PQueryCounter query-state = PQueryCounter s ∧
  trace-bs = map fst f-fl ∧
  comp-bs = map fst fl ∧
  (∀ i < length f-fl.
    fmlookup (HashMap query-state)
      (TraceFriChallenge (PTraceFriCounter s + i) (foldl concat (concat (PState s) fr)
        (take (Suc i) (map snd f-fl)))) =
      Some (fst (f-fl ! i))) ∧
  (∀ i < length fl.
    fmlookup (HashMap query-state)
      (CompositionFriChallenge (PCompositionFriCounter s + i) (foldl concat
        (concat
          (foldl concat
            (concat
              (foldl concat (concat (PState s) fr) (map snd f-fl))
              f-final)
            as)
          dg)
        (take (Suc i) (map snd fl)))) =
      Some (fst (fl ! i)))

```

lemma *verify-monad-accepted-fri-challenges*:

fixes $s :: ('f, 'a) \text{ protocol-channel-scheme}$

assumes *outcome*:

$\text{Some (result, final-state)} \in \text{set-dist (execute verify-monad s)}$

obtains *trace-bs dg comp-bs*

where *accepted-fri-challenges s (Some (result, final-state))*

trace-bs dg comp-bs

proof –

from *verify-monad-header-random-oracle-replay[OF outcome]*

obtain *fr f-fl f-final as dg fl final query-state* **where**

header:

verifier-header-transcript s fr (map snd f-fl) f-final as dg

(map snd fl) final (PTranscript query-state)

and *query-state-eq*:

PState query-state =

verifier-header-state s fr (map snd f-fl) f-final as dg

(map snd fl) final

and *query-out*:

Some (result, final-state) ∈

set-dist (execute

(ntimes

(verifier-query-round-program fr f-fl f-final as fl final)

rounds)

query-state)

and *query-count-header*:

PQueryCounter query-state = PQueryCounter s

and *ext-s-query-header: s ≤ query-state*

```

and trace-lookup:
   $\bigwedge i. i < \text{length } f\text{-}fl \implies$ 
    fmllookup (HashMap query-state)
      (TraceFriChallenge (PTraceFriCounter s + i) (foldl concat (concat (PState s) fr)
        (take (Suc i) (map snd f-fl)))) =
      Some (fst (f-fl ! i))
and alpha-lookup:
   $\bigwedge i. i < \text{length } as \implies$ 
    fmllookup (HashMap query-state)
      (AlphaChallenge (PAlphaCounter s + i) (foldl concat
        (concat (foldl concat (concat (PState s) fr) (map snd f-fl))
          f-final)
        (take i as))) =
      Some (as ! i)
and comp-lookup:
   $\bigwedge i. i < \text{length } fl \implies$ 
    fmllookup (HashMap query-state)
      (CompositionFriChallenge (PCompositionFriCounter s + i) (foldl concat
        (concat
          (foldl concat
            (concat
              (foldl concat (concat (PState s) fr) (map snd f-fl))
              f-final)
            as)
          dg)
        (take (Suc i) (map snd fl)))) =
      Some (fst (fl ! i))
and degree-bound: to-nat dg  $\leq$  maxDegree
by blast
have accepted-fri-challenges s (Some (result, final-state))
  (map fst f-fl) dg (map fst fl)
proof -
have trace-lookup-all:
   $\forall i < \text{length } f\text{-}fl.$ 
    fmllookup (HashMap query-state)
      (TraceFriChallenge (PTraceFriCounter s + i) (foldl concat (concat (PState s) fr)
        (take (Suc i) (map snd f-fl)))) =
      Some (fst (f-fl ! i))
using trace-lookup by blast
have comp-lookup-all:
   $\forall i < \text{length } fl.$ 
    fmllookup (HashMap query-state)
      (CompositionFriChallenge (PCompositionFriCounter s + i) (foldl concat
        (concat
          (foldl concat
            (concat
              (foldl concat (concat (PState s) fr) (map snd f-fl))
              f-final)
            as)
          dg)
        (take (Suc i) (map snd fl)))) =
      Some (fst (fl ! i))

```

```

      dg)
      (take (Suc i) (map snd fl)))) =
    Some (fst (fl ! i))
  using comp-lookup by blast
show ?thesis
  unfolding accepted-fri-challenges-def
  by (intro exI[of - result] exI[of - final-state] exI[of - fr]
      exI[of - f-fl] exI[of - f-final] exI[of - as] exI[of - fl]
      exI[of - final] exI[of - query-state])
      (use header query-state-eq query-out query-count-header trace-lookup-all
        comp-lookup-all in simp)
qed
then show ?thesis
  by (rule that)
qed

```

lemma *accepted-fri-challenges-trace-space*:

assumes *accepted-fri-challenges s out trace-bs dg comp-bs*

shows $\text{trace-bs} \in \text{fri-challenge-space } (\text{ceil-log } \text{clength})$

using *assms*

unfolding *accepted-fri-challenges-def verifier-header-transcript-def*
fri-challenge-space-def

by *auto*

lemma *accepted-fri-challenges-composition-space*:

assumes *accepted-fri-challenges s out trace-bs dg comp-bs*

shows $\text{comp-bs} \in \text{fri-challenge-space } (\text{ceil-log } (\text{to-nat } dg + 1))$

using *assms*

unfolding *accepted-fri-challenges-def verifier-header-transcript-def*
fri-challenge-space-def

by *auto*

lemma *accepted-fri-challenges-unique*:

assumes *c1*:

accepted-fri-challenges s (Some (result, final-state))
trace-bs dg comp-bs

and *c2*:

accepted-fri-challenges s (Some (result, final-state))
trace-bs' dg' comp-bs'

shows $\text{trace-bs}' = \text{trace-bs} \wedge dg' = dg \wedge \text{comp-bs}' = \text{comp-bs}$

proof –

from *c1* **obtain** *fr f-fl f-final as fl final query-state* **where**
header1:

verifier-header-transcript s fr (map snd f-fl) f-final as dg
(map snd fl) final (PTranscript query-state)

and *query-state1*:

PState query-state =
verifier-header-state s fr (map snd f-fl) f-final as dg
(map snd fl) final

```

and query-out1:
  Some (result, final-state) ∈
    set-dist (execute
      (ntimes
        (verifier-query-round-program fr f-fl f-final as fl final)
        rounds)
      query-state)
and trace-bs-eq: trace-bs = map fst f-fl
and comp-bs-eq: comp-bs = map fst fl
and trace-lookup1:
  ∧ i. i < length f-fl ⇒
    fmlookup (HashMap query-state)
      (TraceFriChallenge (PTraceFriCounter s + i)
        (foldl concat (concat (PState s) fr)
          (take (Suc i) (map snd f-fl)))) =
      Some (fst (f-fl ! i))
and comp-lookup1:
  ∧ i. i < length fl ⇒
    fmlookup (HashMap query-state)
      (CompositionFriChallenge (PCompositionFriCounter s + i)
        (foldl concat
          (concat
            (foldl concat
              (concat
                (foldl concat (concat (PState s) fr) (map snd f-fl))
                f-final)
              as)
            dg)
          (take (Suc i) (map snd fl)))) =
      Some (fst (fl ! i))
unfolding accepted-fri-challenges-def by blast
from c2 obtain fr' f-fl' f-final' as' fl' final' query-state' where
  header2:
    verifier-header-transcript s fr' (map snd f-fl') f-final' as' dg'
    (map snd fl') final' (PTranscript query-state')
and query-state2:
  PState query-state' =
    verifier-header-state s fr' (map snd f-fl') f-final' as' dg'
    (map snd fl') final'
and query-out2:
  Some (result, final-state) ∈
    set-dist (execute
      (ntimes
        (verifier-query-round-program fr' f-fl' f-final' as' fl' final')
        rounds)
      query-state')
and trace-bs'-eq: trace-bs' = map fst f-fl'
and comp-bs'-eq: comp-bs' = map fst fl'
and trace-lookup2:

```

```

 $\wedge i. i < \text{length } f\text{-}fl' \implies$ 
   $\text{fmlookup } (\text{HashMap } \text{query-state}')$ 
     $(\text{TraceFriChallenge } (P\text{TraceFriCounter } s + i)$ 
       $(\text{foldl concat } (\text{concat } (P\text{State } s) \text{ fr}')$ 
         $(\text{take } (\text{Suc } i) (\text{map snd } f\text{-}fl')))) =$ 
       $\text{Some } (\text{fst } (f\text{-}fl' ! i))$ 
and comp-lookup2:
 $\wedge i. i < \text{length } fl' \implies$ 
   $\text{fmlookup } (\text{HashMap } \text{query-state}')$ 
     $(\text{CompositionFriChallenge } (P\text{CompositionFriCounter } s + i)$ 
       $(\text{foldl concat}$ 
         $(\text{concat}$ 
           $(\text{foldl concat}$ 
             $(\text{concat}$ 
               $(\text{foldl concat } (\text{concat } (P\text{State } s) \text{ fr}') (\text{map snd } f\text{-}fl'))$ 
               $f\text{-}final')$ 
             $as')$ 
           $dg')$ 
         $(\text{take } (\text{Suc } i) (\text{map snd } fl')))) =$ 
       $\text{Some } (\text{fst } (fl' ! i))$ 
unfolding accepted-fri-challenges-def by blast
have header-unique:
   $fr' = fr \wedge$ 
   $\text{map snd } f\text{-}fl' = \text{map snd } f\text{-}fl \wedge$ 
   $f\text{-}final' = f\text{-}final \wedge$ 
   $as' = as \wedge$ 
   $dg' = dg \wedge$ 
   $\text{map snd } fl' = \text{map snd } fl \wedge$ 
   $final' = final \wedge$ 
   $P\text{Transcript } \text{query-state}' = P\text{Transcript } \text{query-state}$ 
by (rule verifier-header-transcript-unique[OF header1 header2])
have ext1:  $\text{query-state} \leq \text{final-state}$ 
using ntimes-verifier-query-rounds-outcome[OF query-out1] by blast
have ext2:  $\text{query-state}' \leq \text{final-state}$ 
using ntimes-verifier-query-rounds-outcome[OF query-out2] by blast
have len-f-fl':  $\text{length } f\text{-}fl' = \text{length } f\text{-}fl$ 
using header-unique by (metis length-map)
have f-fl-fst-eq:  $\text{map fst } f\text{-}fl' = \text{map fst } f\text{-}fl$ 
proof (rule nth-equalityI)
  show  $\text{length } (\text{map fst } f\text{-}fl') = \text{length } (\text{map fst } f\text{-}fl)$ 
  using len-f-fl' by simp
next
fix i
assume i-bound:  $i < \text{length } (\text{map fst } f\text{-}fl')$ 
then have i-bound':  $i < \text{length } f\text{-}fl'$ 
by simp
have i-bound-old:  $i < \text{length } f\text{-}fl$ 
using i-bound' len-f-fl' by simp
let ?key =

```



```

    TraceFriChallenge (PTraceFriCounter s + i)
      (foldl concat (concat (PState s) fr)
        (take (Suc i) (map snd f-fl)))
  have lookup1-final:
    fmllookup (HashMap final-state) ?key =
      Some (fst (f-fl ! i))
  by (rule hash-extension-lookup[OF trace-lookup1[OF i-bound-old] ext1])
  have key-eq:
    TraceFriChallenge (PTraceFriCounter s + i)
      (foldl concat (concat (PState s) fr')
        (take (Suc i) (map snd f-fl'))) = ?key
  using header-unique by simp
  have lookup2-final:
    fmllookup (HashMap final-state) ?key =
      Some (fst (f-fl' ! i))
  using hash-extension-lookup[OF trace-lookup2[OF i-bound'] ext2]
  unfolding key-eq .
  show map fst f-fl' ! i = map fst f-fl ! i
  using lookup1-final lookup2-final i-bound' i-bound-old by simp
qed
have len-fl': length fl' = length fl
  using header-unique by (metis length-map)
have fl-fst-eq: map fst fl' = map fst fl
proof (rule nth-equalityI)
  show length (map fst fl') = length (map fst fl)
  using len-fl' by simp
next
fix i
assume i-bound: i < length (map fst fl')
then have i-bound': i < length fl'
  by simp
have i-bound-old: i < length fl
  using i-bound' len-fl' by simp
let ?key =
  CompositionFriChallenge (PCompositionFriCounter s + i)
    (foldl concat
      (concat
        (foldl concat
          (concat
            (foldl concat (concat (PState s) fr) (map snd f-fl))
            f-final)
          as)
        dg)
      (take (Suc i) (map snd fl)))
  have lookup1-final:
    fmllookup (HashMap final-state) ?key =
      Some (fst (fl ! i))
  by (rule hash-extension-lookup[OF comp-lookup1[OF i-bound-old] ext1])
  have key-eq:

```

```

CompositionFriChallenge (PCompositionFriCounter s + i)
  (foldl concat
    (concat
      (foldl concat
        (concat
          (foldl concat (concat (PState s) fr') (map snd f-fl'))
            f-final')
          as')
        dg')
      (take (Suc i) (map snd fl')))) = ?key
using header-unique by simp
have lookup2-final:
  fmlookup (HashMap final-state) ?key =
    Some (fst (fl' ! i))
using hash-extension-lookup[OF comp-lookup2[OF i-bound'] ext2]
unfolding key-eq .
show map fst fl' ! i = map fst fl ! i
using lookup1-final lookup2-final i-bound' i-bound-old by simp
qed
show ?thesis
using header-unique trace-bs-eq trace-bs'-eq comp-bs-eq comp-bs'-eq
  f-fl-fst-eq fl-fst-eq
by simp
qed

lemma verify-monad-trace-fri-prequery-lookup:
fixes s :: ('f, 'a) protocol-channel-scheme
assumes outcome:
  Some (result, final-state) ∈ set-dist (execute verify-monad s)
and challenges:
  accepted-fri-challenges s (Some (result, final-state))
  trace-bs dg comp-bs
and header:
  verifier-header-transcript s fr roots trace-final as dg
  composition-roots final rest
and i-len: i < length roots
and lookup-s:
  fmlookup (HashMap s)
    (TraceFriChallenge (PTraceFriCounter s + i)
      (foldl concat (concat (PState s) fr) (take (Suc i) roots))) =
    Some y
shows i < length trace-bs ∧ y = trace-bs ! i
proof (rule verify-monad-header-random-oracle-replay[OF outcome])
fix fr' f-fl trace-final' as' dg' fl' final' query-state
assume header-replay:
  verifier-header-transcript s fr' (map snd f-fl) trace-final' as' dg'
  (map snd fl') final' (PTranscript query-state)
and query-state-eq:
  PState query-state =

```

```

    verifier-header-state s fr' (map snd f-fl) trace-final' as' dg'
    (map snd fl') final'
and query-out:
    Some (result, final-state) ∈
    set-dist
    (execute
    (ntimes
    (verifier-query-round-program fr' f-fl trace-final' as' fl'
    final')
    rounds)
    query-state)
and query-count-header: PQueryCounter query-state = PQueryCounter s
and ext-s-query: s ≤ query-state
and trace-lookup:
    ∧j. j < length f-fl ⇒
    fmlookup (HashMap query-state)
    (TraceFriChallenge (PTraceFriCounter s + j)
    (foldl concat (concat (PState s) fr')
    (take (Suc j) (map snd f-fl)))) =
    Some (fst (f-fl ! j))
and comp-lookup:
    ∧j. j < length fl' ⇒
    fmlookup (HashMap query-state)
    (CompositionFriChallenge (PCompositionFriCounter s + j)
    (foldl concat
    (concat
    (foldl concat
    (concat
    (foldl concat (concat (PState s) fr') (map snd f-fl))
    trace-final')
    as')
    dg')
    (take (Suc j) (map snd fl')))) =
    Some (fst (fl' ! j))
and degree-bound: to-nat dg' ≤ maxDegree
have replay-challenges:
    accepted-fri-challenges s (Some (result, final-state))
    (map fst f-fl) dg' (map fst fl')
unfolding accepted-fri-challenges-def
by (intro exI[of - result] exI[of - final-state] exI[of - fr']
    exI[of - f-fl] exI[of - trace-final'] exI[of - as']
    exI[of - fl'] exI[of - final'] exI[of - query-state])
    (use header-replay query-state-eq query-out query-count-header
    trace-lookup comp-lookup in auto)
have trace-bs-eq: trace-bs = map fst f-fl
using accepted-fri-challenges-unique[OF replay-challenges challenges]
by simp
have header-unique:
    fr = fr' ∧

```

```

    roots = map snd f-fl ∧
    trace-final = trace-final' ∧
    as = as' ∧
    dg = dg' ∧
    composition-roots = map snd fl' ∧
    final = final' ∧
    rest = PTranscript query-state
using verifier-header-transcript-unique[OF header header-replay]
by simp
have i-f-fl:  $i < \text{length } f\text{-fl}$ 
  using i-len header-unique by simp
have lookup-query-from-s:
  fmlookup (HashMap query-state)
    (TraceFriChallenge (PTraceFriCounter s + i)
      (foldl concat (concat (PState s) fr) (take (Suc i) roots))) =
    Some y
  by (rule hash-extension-lookup[OF lookup-s ext-s-query])
have lookup-query-from-trace:
  fmlookup (HashMap query-state)
    (TraceFriChallenge (PTraceFriCounter s + i)
      (foldl concat (concat (PState s) fr) (take (Suc i) roots))) =
    Some (trace-bs ! i)
  using trace-lookup[OF i-f-fl] header-unique trace-bs-eq i-f-fl
  by simp
then have y-eq:  $y = \text{trace-bs } ! i$ 
  using lookup-query-from-s by simp
have  $i < \text{length } \text{trace-bs}$ 
  using i-f-fl trace-bs-eq by simp
then show ?thesis
  using y-eq by simp
qed

```

```

lemma verify-monad-composition-fri-prequery-lookup:
fixes s :: ('f, 'a) protocol-channel-scheme
assumes outcome:
  Some (result, final-state) ∈ set-dist (execute verify-monad s)
and challenges:
  accepted-fri-challenges s (Some (result, final-state))
  trace-bs dg comp-bs
and header:
  verifier-header-transcript s fr trace-roots trace-final as dg
  composition-roots final rest
and i-len:  $i < \text{length } \text{composition-roots}$ 
and lookup-s:
  fmlookup (HashMap s)
    (CompositionFriChallenge (PCompositionFriCounter s + i)
      (foldl concat
        (concat
          (foldl concat

```

```

      (concat
        (foldl concat (concat (PState s) fr) trace-roots)
        trace-final)
      as)
    dg)
  (take (Suc i) composition-roots))) =
  Some y
shows  $i < \text{length comp-bs} \wedge y = \text{comp-bs} ! i$ 
proof (rule verify-monad-header-random-oracle-replay[OF outcome])
fix fr' f-fl trace-final' as' dg' fl' final' query-state
assume header-replay:
  verifier-header-transcript s fr' (map snd f-fl) trace-final' as' dg'
  (map snd fl') final' (PTranscript query-state)
and query-state-eq:
  PState query-state =
    verifier-header-state s fr' (map snd f-fl) trace-final' as' dg'
    (map snd fl') final'
and query-out:
  Some (result, final-state)  $\in$ 
    set-dist
    (execute
      (ntimes
        (verifier-query-round-program fr' f-fl trace-final' as' fl'
          final')
        rounds)
      query-state)
and query-count-header: PQueryCounter query-state = PQueryCounter s
and ext-s-query:  $s \leq \text{query-state}$ 
and trace-lookup:
   $\bigwedge j. j < \text{length f-fl} \implies$ 
    fmlookup (HashMap query-state)
      (TraceFriChallenge (PTraceFriCounter s + j)
        (foldl concat (concat (PState s) fr')
          (take (Suc j) (map snd f-fl)))) =
      Some (fst (f-fl ! j))
and comp-lookup:
   $\bigwedge j. j < \text{length fl'} \implies$ 
    fmlookup (HashMap query-state)
      (CompositionFriChallenge (PCompositionFriCounter s + j)
        (foldl concat
          (concat
            (foldl concat
              (concat
                (foldl concat (concat (PState s) fr') (map snd f-fl))
                trace-final')
              as')
            dg')
          (take (Suc j) (map snd fl')))) =
      Some (fst (fl' ! j))

```

```

and degree-bound: to-nat dg' ≤ maxDegree
have replay-challenges:
  accepted-fri-challenges s (Some (result, final-state))
    (map fst f-fl) dg' (map fst fl')
unfolding accepted-fri-challenges-def
by (intro exI[of - result] exI[of - final-state] exI[of - fr']
    exI[of - f-fl] exI[of - trace-final'] exI[of - as']
    exI[of - fl'] exI[of - final'] exI[of - query-state])
    (use header-replay query-state-eq query-out query-count-header
      trace-lookup comp-lookup in auto)
have comp-bs-eq: comp-bs = map fst fl'
using accepted-fri-challenges-unique[OF replay-challenges challenges]
by simp
have header-unique:
  fr = fr' ∧
  trace-roots = map snd f-fl ∧
  trace-final = trace-final' ∧
  as = as' ∧
  dg = dg' ∧
  composition-roots = map snd fl' ∧
  final = final' ∧
  rest = PTranscript query-state
using verifier-header-transcript-unique[OF header header-replay]
by simp
have i-fl: i < length fl'
using i-len header-unique by simp
have lookup-query-from-s:
  fmlookup (HashMap query-state)
    (CompositionFriChallenge (PCompositionFriCounter s + i)
      (foldl concat
        (concat
          (foldl concat
            (concat
              (foldl concat (concat (PState s) fr) trace-roots)
              trace-final)
            as)
          dg)
        (take (Suc i) composition-roots)))) =
    Some y
by (rule hash-extension-lookup[OF lookup-s ext-s-query])
have lookup-query-from-comp:
  fmlookup (HashMap query-state)
    (CompositionFriChallenge (PCompositionFriCounter s + i)
      (foldl concat
        (concat
          (foldl concat
            (concat
              (foldl concat (concat (PState s) fr) trace-roots)
              trace-final)
            as)
          dg)
        (take (Suc i) composition-roots)))) =
    Some y

```

```

      as)
      dg)
      (take (Suc i) composition-roots))) =
    Some (comp-bs ! i)
  using comp-lookup[OF i-fl] header-unique comp-bs-eq i-fl
  by simp
then have y-eq: y = comp-bs ! i
  using lookup-query-from-s by simp
have i < length comp-bs
  using i-fl comp-bs-eq by simp
then show ?thesis
  using y-eq by simp
qed

lemma verify-monad-accepted-fri-challenges-degree-bound:
  fixes s :: ('f, 'a) protocol-channel-scheme
  assumes outcome:
    Some (result, final-state) ∈ set-dist (execute verify-monad s)
  and challenges:
    accepted-fri-challenges s (Some (result, final-state))
    trace-bs dg comp-bs
  shows to-nat dg ≤ maxDegree
proof -
  from verify-monad-header-random-oracle-replay[OF outcome]
  obtain fr f-fl f-final as dg' fl final query-state where
    header:
      verifier-header-transcript s fr (map snd f-fl) f-final as dg'
      (map snd fl) final (PTranscript query-state)
  and query-state-eq:
    PState query-state =
      verifier-header-state s fr (map snd f-fl) f-final as dg'
      (map snd fl) final
  and query-out:
    Some (result, final-state) ∈
      set-dist (execute
        (ntimes
          (verifier-query-round-program fr f-fl f-final as fl final)
          rounds)
        query-state)
  and query-count-header:
    PQueryCounter query-state = PQueryCounter s
  and trace-lookup:
    ∧ i. i < length f-fl ⇒
      fmlookup (HashMap query-state)
        (TraceFriChallenge (PTraceFriCounter s + i) (foldl concat
          (concat (PState s) fr) (take (Suc i) (map snd f-fl)))) =
        Some (fst (f-fl ! i))
  and comp-lookup:
    ∧ i. i < length fl ⇒

```

```

    fmllookup (HashMap query-state)
      (CompositionFriChallenge (PCompositionFriCounter s + i) (foldl concat
        (concat
          (foldl concat
            (concat
              (foldl concat (concat (PState s) fr) (map snd f-fl))
              f-final)
            as)
          dg')
        (take (Suc i) (map snd fl)))))) =
    Some (fst (fl ! i))
  and degree-bound: to-nat dg' ≤ maxDegree
  by metis
have trace-lookup-all:
  ∀ i < length f-fl.
    fmllookup (HashMap query-state)
      (TraceFriChallenge (PTraceFriCounter s + i) (foldl concat
        (concat (PState s) fr) (take (Suc i) (map snd f-fl)))) =
    Some (fst (f-fl ! i))
  using trace-lookup by blast
have comp-lookup-all:
  ∀ i < length fl.
    fmllookup (HashMap query-state)
      (CompositionFriChallenge (PCompositionFriCounter s + i) (foldl concat
        (concat
          (foldl concat
            (concat
              (foldl concat (concat (PState s) fr) (map snd f-fl))
              f-final)
            as)
          dg')
        (take (Suc i) (map snd fl)))))) =
    Some (fst (fl ! i))
  using comp-lookup by blast
have challenges':
  accepted-fri-challenges s (Some (result, final-state))
    (map fst f-fl) dg' (map fst fl)
  unfolding accepted-fri-challenges-def
  by (intro exI[of - result] exI[of - final-state] exI[of - fr]
    exI[of - f-fl] exI[of - f-final] exI[of - as] exI[of - fl]
    exI[of - final] exI[of - query-state])
    (use header query-state-eq query-out query-count-header
      trace-lookup-all comp-lookup-all in simp)
have dg' = dg
  using accepted-fri-challenges-unique[OF challenges challenges'] by simp
then show ?thesis
  using degree-bound by simp
qed

```


lemma *verifier-after-trace-fri-accepted-fri-challenges*:

fixes $s :: ('f, 'a) \text{ protocol-channel-scheme}$

assumes *prefix*:

$\text{Some } ((fr, f-fl), \text{prefix-state}) \in$
 $\text{set-dist } (\text{execute } \text{verifier-trace-fri-prefix } s)$

and *suffix*:

$\text{Some } (\text{result}, \text{final-state}) \in$
 set-dist
 $(\text{execute } (\text{verifier-after-trace-fri } (fr, f-fl)) \text{ prefix-state})$

shows $\exists dg \text{ comp-bs.}$
 $\text{accepted-fri-challenges } s (\text{Some } (\text{result}, \text{final-state}))$
 $(\text{map fst } f-fl) \text{ dg comp-bs}$

proof –

have *prefix-res*:

$\text{length } f-fl = \text{ceil-log } \text{clength} \wedge$
 $P\text{Transcript } s = [fr] @ \text{map snd } f-fl @ P\text{Transcript } \text{prefix-state} \wedge$
 $P\text{State } \text{prefix-state} =$
 $\text{foldl concat } (\text{concat } (P\text{State } s) \text{ fr}) (\text{map snd } f-fl) \wedge$
 $s \leq \text{prefix-state} \wedge$
 $P\text{TraceFriCounter } \text{prefix-state} = P\text{TraceFriCounter } s + \text{ceil-log } \text{clength} \wedge$
 $P\text{CompositionFriCounter } \text{prefix-state} = P\text{CompositionFriCounter } s \wedge$
 $P\text{AlphaCounter } \text{prefix-state} = P\text{AlphaCounter } s \wedge$
 $P\text{QueryCounter } \text{prefix-state} = P\text{QueryCounter } s \wedge$
 $(\forall i < \text{length } f-fl.$
 $\text{fmlookup } (\text{HashMap } \text{prefix-state})$
 $(\text{TraceFriChallenge } (P\text{TraceFriCounter } s + i)$
 $(\text{foldl concat } (\text{concat } (P\text{State } s) \text{ fr})$
 $(\text{take } (\text{Suc } i) (\text{map snd } f-fl)))) =$
 $\text{Some } (\text{fst } (f-fl ! i)))$

by (rule *verifier-trace-fri-prefix-outcome*[*OF prefix*])

from *suffix* **obtain** $f\text{-final } s3 \text{ as } s4 \text{ dg } s5 \text{ s6 } fl \text{ s7 final query-state}$ **where**

read-trace-final:

$\text{Some } (f\text{-final}, s3) \in \text{set-dist } (\text{execute } \text{read } \text{prefix-state})$

and *alpha-out*:

$\text{Some } (as, s4) \in$
 $\text{set-dist } (\text{execute } (\text{mmap } (\text{replicate } (\text{length } \text{spec}) \text{ alpha-round})) s3)$

and *read-dg*: $\text{Some } (dg, s5) \in \text{set-dist } (\text{execute } \text{read } s4)$

and *degree-assert*:

$\text{Some } ((), s6) \in$
 $\text{set-dist } (\text{execute } (\text{assert } (\text{to-nat } dg \leq \text{maxDegree})) s5)$

and *comp-fri*:

$\text{Some } (fl, s7) \in$
 $\text{set-dist } (\text{execute}$
 $(\text{ntimes } \text{receive-composition-fri-commits}$
 $(\text{ceil-log } (\text{to-nat } dg + 1))) s6)$

and *read-final*: $\text{Some } (\text{final}, \text{query-state}) \in \text{set-dist } (\text{execute } \text{read } s7)$

and *query-out*:

$\text{Some } (\text{result}, \text{final-state}) \in$
 $\text{set-dist } (\text{execute}$

```

    (ntimes (verifier-query-round-program fr f-fl f-final as fl final)
      rounds)
    query-state)
  unfolding verifier-after-trace-fri-def
  by (auto elim!: set-dist-bindE)
from read-outcome[OF read-trace-final] obtain rest3 where
  tr-prefix: PTranscript prefix-state = f-final # rest3
  and st-s3: PState s3 = concat (PState prefix-state) f-final
  and tr-s3: PTranscript s3 = rest3
  and ext-prefix-s3: prefix-state ≤ s3
  and trace-count-s3: PTraceFriCounter s3 = PTraceFriCounter prefix-state
  and comp-count-s3: PCompositionFriCounter s3 = PCompositionFriCounter prefix-state
  and alpha-count-s3: PAlphaCounter s3 = PAlphaCounter prefix-state
  and query-count-s3: PQueryCounter s3 = PQueryCounter prefix-state
  by blast
have alpha-res:
  length as = length spec ∧
  PTranscript s3 = as @ PTranscript s4 ∧
  PState s4 = foldl concat (PState s3) as ∧
  s3 ≤ s4 ∧
  PTraceFriCounter s4 = PTraceFriCounter s3 ∧
  PCompositionFriCounter s4 = PCompositionFriCounter s3 ∧
  PAlphaCounter s4 = PAlphaCounter s3 + length spec ∧
  PQueryCounter s4 = PQueryCounter s3
  using mmap-alpha-round-outcome[OF alpha-out] by simp
from read-outcome[OF read-dg] obtain rest5 where
  tr-s4: PTranscript s4 = dg # rest5
  and st-s5: PState s5 = concat (PState s4) dg
  and tr-s5: PTranscript s5 = rest5
  and ext-s4-s5: s4 ≤ s5
  and trace-count-s5: PTraceFriCounter s5 = PTraceFriCounter s4
  and comp-count-s5: PCompositionFriCounter s5 = PCompositionFriCounter s4
  and alpha-count-s5: PAlphaCounter s5 = PAlphaCounter s4
  and query-count-s5: PQueryCounter s5 = PQueryCounter s4
  by blast
have s6-eq: s6 = s5
  using degree-assert unfolding assert-def
  by (cases to-nat dg ≤ maxDegree) (auto simp: throw-no-outcome)
have ext-s5-s6: s5 ≤ s6
  unfolding s6-eq by (rule hash-ext-refl)
have comp-res:
  length fl = ceil-log (to-nat dg + 1) ∧
  PTranscript s6 = map snd fl @ PTranscript s7 ∧
  PState s7 = foldl concat (PState s6) (map snd fl) ∧
  s6 ≤ s7 ∧
  PTraceFriCounter s7 = PTraceFriCounter s6 ∧
  PCompositionFriCounter s7 =
    PCompositionFriCounter s6 + ceil-log (to-nat dg + 1) ∧
  PAlphaCounter s7 = PAlphaCounter s6 ∧

```

```

PQueryCounter s7 = PQueryCounter s6 ∧
(∀ i < ceil-log (to-nat dg + 1).
  fmllookup (HashMap s7)
    (CompositionFriChallenge (PCompositionFriCounter s6 + i)
      (foldl concat (PState s6) (take (Suc i) (map snd fl)))) =
    Some (fst (fl ! i)))
using ntimes-receive-composition-fri-commits-outcome[OF comp-fri]
by simp
from read-outcome[OF read-final] obtain rest-query where
  tr-s7: PTranscript s7 = final # rest-query
and st-query: PState query-state = concat (PState s7) final
and tr-query: PTranscript query-state = rest-query
and ext-s7-query: s7 ≤ query-state
and query-count-query: PQueryCounter query-state = PQueryCounter s7
by blast
have ext-prefix-query: prefix-state ≤ query-state
using ext-prefix-s3 alpha-res ext-s4-s5 ext-s5-s6 comp-res ext-s7-query
by (meson hash-ext-trans)
have header:
  verifier-header-transcript s fr (map snd f-fl) f-final as dg
    (map snd fl) final (PTranscript query-state)
using prefix-res tr-prefix tr-s3 alpha-res tr-s4 tr-s5 s6-eq comp-res
  tr-s7 tr-query
unfolding verifier-header-transcript-def verifier-header-messages-def
by simp
have header-state:
  PState query-state =
    verifier-header-state s fr (map snd f-fl) f-final as dg
      (map snd fl) final
using prefix-res st-s3 alpha-res st-s5 s6-eq comp-res st-query
unfolding verifier-header-state-def verifier-header-messages-def
by simp
have query-count-header:
  PQueryCounter query-state = PQueryCounter s
using prefix-res query-count-s3 alpha-res query-count-s5 s6-eq comp-res
  query-count-query
by simp
have trace-lookup:
  ∧ i. i < length f-fl ⇒
    fmllookup (HashMap query-state)
      (TraceFriChallenge (PTraceFriCounter s + i)
        (foldl concat (concat (PState s) fr)
          (take (Suc i) (map snd f-fl)))) =
      Some (fst (f-fl ! i))
proof –
  fix i
  assume i-bound: i < length f-fl
  have lookup-prefix:
    fmllookup (HashMap prefix-state)

```

```

    (TraceFriChallenge (PTraceFriCounter s + i)
      (foldl concat (concat (PState s) fr)
        (take (Suc i) (map snd f-fl)))) =
    Some (fst (f-fl ! i))
  using prefix-res i-bound by simp
show fmlookup (HashMap query-state)
  (TraceFriChallenge (PTraceFriCounter s + i)
    (foldl concat (concat (PState s) fr)
      (take (Suc i) (map snd f-fl)))) =
  Some (fst (f-fl ! i))
  by (rule hash-extension-lookup[OF lookup-prefix ext-prefix-query])
qed
have comp-lookup:
   $\bigwedge i. i < \text{length } fl \implies$ 
  fmlookup (HashMap query-state)
    (CompositionFriChallenge (PCompositionFriCounter s + i) (foldl concat
      (concat
        (foldl concat
          (concat (foldl concat (concat (PState s) fr) (map snd f-fl)) f-final)
            as)
          dg)
        (take (Suc i) (map snd fl)))))) =
  Some (fst (fl ! i))
proof -
  fix i
  assume i-bound:  $i < \text{length } fl$ 
  have lookup-s7:
    fmlookup (HashMap s7)
      (CompositionFriChallenge (PCompositionFriCounter s6 + i)
        (foldl concat (PState s6) (take (Suc i) (map snd fl)))) =
    Some (fst (fl ! i))
  using comp-res i-bound by simp
show fmlookup (HashMap query-state)
  (CompositionFriChallenge (PCompositionFriCounter s + i) (foldl concat
    (concat
      (foldl concat
        (concat (foldl concat (concat (PState s) fr) (map snd f-fl)) f-final)
          as)
        dg)
        (take (Suc i) (map snd fl)))))) =
  Some (fst (fl ! i))
  using hash-extension-lookup[OF lookup-s7 ext-s7-query]
    prefix-res st-s3 alpha-res st-s5 s6-eq comp-count-s3 comp-count-s5
  by simp
qed
have challenges:
  accepted-fri-challenges s (Some (result, final-state))
  (map fst f-fl) dg (map fst fl)
unfolding accepted-fri-challenges-def

```

```

apply (rule exI[where x=result])
apply (rule exI[where x=final-state])
apply (rule exI[where x=fr])
apply (rule exI[where x=f-fl])
apply (rule exI[where x=f-final])
apply (rule exI[where x=as])
apply (rule exI[where x=fl])
apply (rule exI[where x=final])
apply (rule exI[where x=query-state])
apply (intro conjI)
  apply simp
  apply (rule header)
  apply (rule header-state)
  apply (rule query-out)
  apply (rule query-count-header)
  apply simp
  apply simp
  apply (intro allI impI, rule trace-lookup, assumption)
  apply (intro allI impI, rule comp-lookup, assumption)
done
show ?thesis
  using challenges by blast
qed

```

```

lemma verifier-after-composition-fri-accepted-fri-challenges:
  fixes s :: ('f, 'a) protocol-channel-scheme
  assumes prefix:
    Some ((fr, f-fl, f-final, as, dg, fl), prefix-state) ∈
      set-dist (execute verifier-composition-fri-prefix s)
  and suffix:
    Some (result, final-state) ∈
      set-dist
        (execute
          (verifier-after-composition-fri
            (fr, f-fl, f-final, as, dg, fl))
          prefix-state)
  shows accepted-fri-challenges s (Some (result, final-state))
    (map fst f-fl) dg (map fst fl)
proof –

```

```

  have prefix-res:
    length f-fl = ceil-log clength ∧
    length as = length spec ∧
    length fl = ceil-log (to-nat dg + 1) ∧
    PTranscript s =
      [fr] @ map snd f-fl @ [f-final] @ as @ [dg] @
      map snd fl @ PTranscript prefix-state ∧
    PState prefix-state =
      foldl concat
        (concat

```

```

    (foldl concat
      (concat
        (foldl concat (concat (PState s) fr) (map snd f-fl))
        f-final)
      as)
    dg)
  (map snd fl) ∧
  s ≤ prefix-state ∧
  PQueryCounter prefix-state = PQueryCounter s ∧
  (∀ i < length f-fl.
    fmllookup (HashMap prefix-state)
      (TraceFriChallenge (PTraceFriCounter s + i)
        (foldl concat (concat (PState s) fr)
          (take (Suc i) (map snd f-fl)))) =
    Some (fst (f-fl ! i))) ∧
  (∀ i < length fl.
    fmllookup (HashMap prefix-state)
      (CompositionFriChallenge (PCompositionFriCounter s + i)
        (foldl concat
          (concat
            (foldl concat
              (concat
                (foldl concat (concat (PState s) fr) (map snd f-fl))
                f-final)
              as)
            dg)
          (take (Suc i) (map snd fl)))) =
    Some (fst (fl ! i)))
  by (rule verifier-composition-fri-prefix-outcome[OF prefix])
from suffix obtain final query-state where
  read-final:
    Some (final, query-state) ∈
    set-dist (execute read prefix-state)
  and query-out:
    Some (result, final-state) ∈
    set-dist (execute
      (ntimes (verifier-query-round-program fr f-fl f-final as fl final)
        rounds)
      query-state)
  unfolding verifier-after-composition-fri-def
  by (auto elim!: set-dist-bindE)
from read-outcome[OF read-final] obtain rest-query where
  tr-prefix: PTranscript prefix-state = final # rest-query
  and st-query: PState query-state = concat (PState prefix-state) final
  and tr-query: PTranscript query-state = rest-query
  and ext-prefix-query: prefix-state ≤ query-state
  and query-count-query:
    PQueryCounter query-state = PQueryCounter prefix-state
by blast

```

```

have header:
  verifier-header-transcript s fr (map snd f-fl) f-final as dg
    (map snd fl) final (PTranscript query-state)
using prefix-res tr-prefix tr-query
unfolding verifier-header-transcript-def verifier-header-messages-def
by simp
have header-state:
  PState query-state =
    verifier-header-state s fr (map snd f-fl) f-final as dg
      (map snd fl) final
using prefix-res st-query
unfolding verifier-header-state-def verifier-header-messages-def
by simp
have query-count-header:
  PQueryCounter query-state = PQueryCounter s
using prefix-res query-count-query by simp
have trace-lookup:
   $\wedge i. i < \text{length } f\text{-fl} \implies$ 
    fmlookup (HashMap query-state)
      (TraceFriChallenge (PTraceFriCounter s + i)
        (foldl concat (concat (PState s) fr)
          (take (Suc i) (map snd f-fl)))) =
      Some (fst (f-fl ! i))
proof -
  fix i
  assume i-bound:  $i < \text{length } f\text{-fl}$ 
  have lookup-prefix:
    fmlookup (HashMap prefix-state)
      (TraceFriChallenge (PTraceFriCounter s + i)
        (foldl concat (concat (PState s) fr)
          (take (Suc i) (map snd f-fl)))) =
    Some (fst (f-fl ! i))
  using prefix-res i-bound by simp
  show fmlookup (HashMap query-state)
    (TraceFriChallenge (PTraceFriCounter s + i)
      (foldl concat (concat (PState s) fr)
        (take (Suc i) (map snd f-fl)))) =
    Some (fst (f-fl ! i))
  by (rule hash-extension-lookup[OF lookup-prefix ext-prefix-query])
qed
have comp-lookup:
   $\wedge i. i < \text{length } fl \implies$ 
    fmlookup (HashMap query-state)
      (CompositionFriChallenge (PCompositionFriCounter s + i)
        (foldl concat
          (concat
            (foldl concat
              (concat
                (foldl concat (concat (PState s) fr) (map snd f-fl))

```

```

      f-final)
    as)
  dg)
  (take (Suc i) (map snd fl)))) =
  Some (fst (fl ! i))
proof -
  fix i
  assume i-bound: i < length fl
  have lookup-prefix:
    fmlookup (HashMap prefix-state)
      (CompositionFriChallenge (PCompositionFriCounter s + i)
        (foldl concat
          (concat
            (foldl concat
              (concat
                (foldl concat (concat (PState s) fr) (map snd f-fl))
                f-final)
              as)
            dg)
          (take (Suc i) (map snd fl)))))) =
    Some (fst (fl ! i))
  using prefix-res i-bound by simp
  show fmlookup (HashMap query-state)
    (CompositionFriChallenge (PCompositionFriCounter s + i)
      (foldl concat
        (concat
          (foldl concat
            (concat
              (foldl concat (concat (PState s) fr) (map snd f-fl))
              f-final)
            as)
          dg)
        (take (Suc i) (map snd fl)))))) =
    Some (fst (fl ! i))
  by (rule hash-extension-lookup[OF lookup-prefix ext-prefix-query])
qed
show ?thesis
  unfolding accepted-fri-challenges-def
  apply (rule exI[where x=result])
  apply (rule exI[where x=final-state])
  apply (rule exI[where x=fr])
  apply (rule exI[where x=f-fl])
  apply (rule exI[where x=f-final])
  apply (rule exI[where x=as])
  apply (rule exI[where x=fl])
  apply (rule exI[where x=final])
  apply (rule exI[where x=query-state])
  apply (intro conjI)
  apply simp

```



```

    apply (rule header)
    apply (rule header-state)
    apply (rule query-out)
    apply (rule query-count-header)
    apply simp
    apply simp
    apply (intro allI impI, rule trace-lookup, assumption)
    apply (intro allI impI, rule comp-lookup, assumption)
  done
qed

```

definition *trace-fri-challenge-list-set-hit*
 $:: (f, 'a) \text{ protocol-channel-scheme} \Rightarrow 'f \text{ list set} \Rightarrow$
 $(\text{unit list} \times (f, 'a) \text{ protocol-channel-scheme}) \text{ option} \Rightarrow \text{bool}$
where
 $\text{trace-fri-challenge-list-set-hit } s \ B \ \text{out} \longleftrightarrow$
 $(\exists \text{ trace-bs } dg \ \text{comp-bs.}$
 $\text{accepted-fri-challenges } s \ \text{out trace-bs } dg \ \text{comp-bs} \wedge$
 $\text{trace-bs} \in B)$

definition *trace-fri-root-list-set-hit*
 $:: (f, 'a) \text{ protocol-channel-scheme} \Rightarrow$
 $(f \Rightarrow 'f \text{ list set}) \Rightarrow$
 $(\text{unit list} \times (f, 'a) \text{ protocol-channel-scheme}) \text{ option} \Rightarrow \text{bool}$
where
 $\text{trace-fri-root-list-set-hit } s \ B \ \text{out} \longleftrightarrow$
 $(\exists \text{ trace-bs } dg \ \text{comp-bs } fr \ f\text{-fri-roots } f\text{-final } as \ \text{composition-fri-roots}$
 final rest.
 $\text{accepted-fri-challenges } s \ \text{out trace-bs } dg \ \text{comp-bs} \wedge$
 $\text{verifier-header-transcript } s \ fr \ f\text{-fri-roots } f\text{-final } as \ dg$
 $\text{composition-fri-roots final rest} \wedge$
 $\text{trace-bs} \in B \ fr)$

definition *composition-fri-challenge-list-set-hit*
 $:: (f, 'a) \text{ protocol-channel-scheme} \Rightarrow (f \Rightarrow 'f \text{ list set}) \Rightarrow$
 $(\text{unit list} \times (f, 'a) \text{ protocol-channel-scheme}) \text{ option} \Rightarrow \text{bool}$
where
 $\text{composition-fri-challenge-list-set-hit } s \ B \ \text{out} \longleftrightarrow$
 $(\exists \text{ trace-bs } dg \ \text{comp-bs.}$
 $\text{accepted-fri-challenges } s \ \text{out trace-bs } dg \ \text{comp-bs} \wedge$
 $\text{comp-bs} \in B \ dg)$

definition *composition-fri-root-list-set-hit*
 $:: (f, 'a) \text{ protocol-channel-scheme} \Rightarrow$
 $(f \Rightarrow 'f \text{ list} \Rightarrow f \Rightarrow 'f \text{ list} \Rightarrow f \Rightarrow$
 $'f \text{ list} \Rightarrow 'f \text{ list set}) \Rightarrow$
 $(\text{unit list} \times (f, 'a) \text{ protocol-channel-scheme}) \text{ option} \Rightarrow \text{bool}$
where
 $\text{composition-fri-root-list-set-hit } s \ B \ \text{out} \longleftrightarrow$

$(\exists \text{ trace-bs } dg \text{ comp-bs } fr \text{ f-fri-roots } f\text{-final as}$
 $\quad \text{composition-fri-roots final rest.}$
 $\text{accepted-fri-challenges } s \text{ out trace-bs } dg \text{ comp-bs } \wedge$
 $\text{verifier-header-transcript } s \text{ fr f-fri-roots } f\text{-final as } dg$
 $\quad \text{composition-fri-roots final rest } \wedge$
 $\text{comp-bs} \in B \text{ fr f-fri-roots } f\text{-final as } dg \text{ composition-fri-roots})$

lemma *wp-verify-monad-trace-fri-challenge-list-set-bound:*

assumes *future: trace-fri-future-fresh s*

and *subset: $B \subseteq \text{fri-challenge-space } (\text{ceil-log clength})$*

shows

wp-event verify-monad (trace-fri-challenge-list-set-hit s B) s ≤
nnreal (card B) / nnreal (CARD('f) ^ ceil-log clength)

proof –

let *?Head =*

λout. case out of

None ⇒ False

| Some ((-, f-fl), -) ⇒ map fst f-fl ∈ B

have *prefix-bound:*

wp-event verifier-trace-fri-prefix ?Head s ≤
nnreal (card B) / nnreal (CARD('f) ^ ceil-log clength)

by (*rule wp-verifier-trace-fri-prefix-challenge-space-set-bound*
[OF future subset])

show *?thesis*

unfolding *verify-monad-trace-fri-decomposition*

proof (*rule wp-event-bind-bound-by-head-event[OF prefix-bound]*)

show *trace-fri-challenge-list-set-hit s B None ⇒ ?Head None*

unfolding *trace-fri-challenge-list-set-hit-def*

accepted-fri-challenges-def **by** *blast*

next

fix *header prefix-state out*

assume *prefix:*

Some (header, prefix-state) ∈

set-dist (execute verifier-trace-fri-prefix s)

and *suffix:*

out ∈

set-dist (execute (verifier-after-trace-fri header) prefix-state)

and *hit: trace-fri-challenge-list-set-hit s B out*

obtain *fr f-fl* **where** *header-eq: header = (fr, f-fl)*

by (*cases header*) *auto*

from *hit* **obtain** *trace-bs dg comp-bs* **where**

challenges-hit: accepted-fri-challenges s out trace-bs dg comp-bs

and *trace-bs-B: trace-bs ∈ B*

unfolding *trace-fri-challenge-list-set-hit-def* **by** *blast*

from *challenges-hit* **obtain** *result final-state* **where** *out-eq:*

out = Some (result, final-state)

unfolding *accepted-fri-challenges-def* **by** *blast*

have *prefix':*

Some ((fr, f-fl), prefix-state) ∈

```

    set-dist (execute verifier-trace-fri-prefix s)
  using prefix header-eq by simp
have suffix':
  Some (result, final-state) ∈
    set-dist
      (execute (verifier-after-trace-fri (fr, f-fl)) prefix-state)
  using suffix header-eq out-eq by simp
from verifier-after-trace-fri-accepted-fri-challenges[OF prefix' suffix']
obtain dg' comp-bs' where
  challenges-prefix:
    accepted-fri-challenges s (Some (result, final-state))
    (map fst f-fl) dg' comp-bs'
  by blast
have trace-bs = map fst f-fl
  using accepted-fri-challenges-unique
  [OF challenges-prefix challenges-hit[unfolded out-eq]]
  by simp
then show ?Head (Some (header, prefix-state))
  using trace-bs-B header-eq by simp
qed
qed

lemma wp-verify-monad-trace-fri-root-list-set-bound:
  fixes C :: prob
  assumes future: trace-fri-future-fresh s
  and subset:  $\bigwedge fr. B \text{ fr} \subseteq \text{fri-challenge-space } (\text{ceil-log } \text{clength})$ 
  and bound:
     $\bigwedge fr. \text{nnreal } (\text{card } (B \text{ fr})) /$ 
     $\text{nnreal } (\text{CARD}('f) \wedge \text{ceil-log } \text{clength}) \leq C$ 
  shows
    wp-event verify-monad (trace-fri-root-list-set-hit s B) s ≤ C
proof -
  let ?Head =
    λout. case out of
      None ⇒ False
    | Some ((fr, f-fl), -) ⇒ map fst f-fl ∈ B fr
  have prefix-bound:
    wp-event verifier-trace-fri-prefix ?Head s ≤ C
  by (rule wp-verifier-trace-fri-prefix-dependent-challenge-space-set-bound
    [OF future subset bound])
  show ?thesis
    unfolding verify-monad-trace-fri-decomposition
  proof (rule wp-event-bind-bound-by-head-event[OF prefix-bound])
    show trace-fri-root-list-set-hit s B None ⇒ ?Head None
      unfolding trace-fri-root-list-set-hit-def
      accepted-fri-challenges-def by blast
  next
    fix header prefix-state out
    assume prefix:

```

```

Some (header, prefix-state) ∈
  set-dist (execute verifier-trace-fri-prefix s)
and suffix:
  out ∈
    set-dist (execute (verifier-after-trace-fri header) prefix-state)
and hit: trace-fri-root-list-set-hit s B out
obtain fr f-fl where header-eq: header = (fr, f-fl)
by (cases header) auto
from hit obtain trace-bs dg comp-bs fr' f-fri-roots' f-final' as'
  composition-fri-roots' final' rest' where
    challenges-hit: accepted-fri-challenges s out trace-bs dg comp-bs
and header-hit:
  verifier-header-transcript s fr' f-fri-roots' f-final' as' dg
  composition-fri-roots' final' rest'
and trace-bs-B: trace-bs ∈ B fr'
unfolding trace-fri-root-list-set-hit-def by blast
from challenges-hit obtain result final-state where out-eq:
  out = Some (result, final-state)
unfolding accepted-fri-challenges-def by blast
have prefix':
  Some ((fr, f-fl), prefix-state) ∈
    set-dist (execute verifier-trace-fri-prefix s)
using prefix header-eq by simp
have suffix':
  Some (result, final-state) ∈
    set-dist
      (execute (verifier-after-trace-fri (fr, f-fl)) prefix-state)
using suffix header-eq out-eq by simp
have prefix-res:
  PTranscript s = [fr] @ map snd f-fl @ PTranscript prefix-state
using verifier-trace-fri-prefix-outcome[OF prefix'] by simp
have fr-eq: fr' = fr
using prefix-res header-hit
unfolding verifier-header-transcript-def verifier-header-messages-def
by simp
from verifier-after-trace-fri-accepted-fri-challenges[OF prefix' suffix']
obtain dg' comp-bs' where
  challenges-prefix:
    accepted-fri-challenges s (Some (result, final-state))
    (map fst f-fl) dg' comp-bs'
by blast
have trace-bs-eq: trace-bs = map fst f-fl
using accepted-fri-challenges-unique
  [OF challenges-prefix challenges-hit[unfolded out-eq]]
by simp
then show ?Head (Some (header, prefix-state))
using trace-bs-B fr-eq header-eq by simp
qed
qed

```

```

lemma wp-verify-monad-composition-fri-root-list-set-bound:
  fixes  $C :: \text{prob}$ 
  assumes future: composition-fri-future-fresh s
  and subset:
     $\bigwedge \text{fr } f\text{-fri-roots } f\text{-final as dg composition-fri-roots.}$ 
     $B \text{ fr } f\text{-fri-roots } f\text{-final as dg composition-fri-roots} \subseteq$ 
     $\text{fri-challenge-space } (\text{ceil-log } (\text{to-nat dg} + 1))$ 
  and bound:
     $\bigwedge \text{fr } f\text{-fri-roots } f\text{-final as dg composition-fri-roots.}$ 
     $\text{nnreal}$ 
     $(\text{card}$ 
       $(B \text{ fr } f\text{-fri-roots } f\text{-final as dg composition-fri-roots})) /$ 
     $\text{nnreal } (\text{CARD}('f) \wedge \text{ceil-log } (\text{to-nat dg} + 1)) \leq C$ 
  shows
     $\text{wp-event verify-monad } (\text{composition-fri-root-list-set-hit } s \ B) \ s \leq C$ 
proof –
  let ?Head =
     $\lambda \text{out. case out of}$ 
     $\text{None} \Rightarrow \text{False}$ 
     $| \text{Some } ((\text{fr}, f\text{-fl}, f\text{-final}, \text{as}, \text{dg}, \text{fl}), -) \Rightarrow$ 
     $\text{map fst fl} \in B \text{ fr } (\text{map snd } f\text{-fl}) \ f\text{-final as dg } (\text{map snd fl})$ 
  have prefix-bound:
     $\text{wp-event verifier-composition-fri-prefix } ?\text{Head } s \leq C$ 
  by (rule wp-verifier-composition-fri-prefix-root-dependent-challenge-space-set-bound
    [OF future subset bound])
  show ?thesis
  unfolding verify-monad-composition-fri-decomposition
proof (rule wp-event-bind-bound-by-head-event[OF prefix-bound])
  show composition-fri-root-list-set-hit s B None  $\implies$  ?Head None
  unfolding composition-fri-root-list-set-hit-def
    accepted-fri-challenges-def by blast
next
  fix header prefix-state out
  assume prefix:
     $\text{Some } (\text{header}, \text{prefix-state}) \in$ 
     $\text{set-dist } (\text{execute verifier-composition-fri-prefix } s)$ 
  and suffix:
     $\text{out} \in$ 
     $\text{set-dist } (\text{execute}$ 
       $(\text{verifier-after-composition-fri header}) \ \text{prefix-state})$ 
  and hit: composition-fri-root-list-set-hit s B out
obtain fr f-fl f-final as dg fl where header-eq:
     $\text{header} = (\text{fr}, f\text{-fl}, f\text{-final}, \text{as}, \text{dg}, \text{fl})$ 
  by (cases header) auto
from hit obtain trace-bs dg' comp-bs fr' f-fri-roots' f-final' as'
    composition-fri-roots' final' rest' where
    challenges-hit:
       $\text{accepted-fri-challenges } s \ \text{out } \text{trace-bs } dg' \ \text{comp-bs}$ 

```

```

and header-hit:
  verifier-header-transcript s fr' f-fri-roots' f-final' as' dg'
  composition-fri-roots' final' rest'
and comp-bs-B:
  comp-bs  $\in$ 
    B fr' f-fri-roots' f-final' as' dg' composition-fri-roots'
  unfolding composition-fri-root-list-set-hit-def by blast
from challenges-hit obtain result final-state where out-eq:
  out = Some (result, final-state)
  unfolding accepted-fri-challenges-def by blast
have prefix':
  Some ((fr, f-fl, f-final, as, dg, fl), prefix-state)  $\in$ 
    set-dist (execute verifier-composition-fri-prefix s)
  using prefix header-eq by simp
have suffix':
  Some (result, final-state)  $\in$ 
    set-dist
    (execute
      (verifier-after-composition-fri
        (fr, f-fl, f-final, as, dg, fl))
      prefix-state)
  using suffix header-eq out-eq by simp
from suffix' [unfolded verifier-after-composition-fri-def]
obtain final query-state where
  read-final:
    Some (final, query-state)  $\in$  set-dist (execute read prefix-state)
  and query-out:
    Some (result, final-state)  $\in$ 
      set-dist
      (execute
        (ntimes
          (verifier-query-round-program fr f-fl f-final as fl final)
          rounds)
        query-state)
    by (auto elim!: set-dist-bindE)
have prefix-res:
  length f-fl = ceil-log clength  $\wedge$ 
  length as = length spec  $\wedge$ 
  length fl = ceil-log (to-nat dg + 1)  $\wedge$ 
  PTranscript s =
    [fr] @ map snd f-fl @ [f-final] @ as @ [dg] @
    map snd fl @ PTranscript prefix-state
  using verifier-composition-fri-prefix-outcome[OF prefix'] by simp
from read-outcome[OF read-final] obtain rest-query where
  tr-prefix: PTranscript prefix-state = final # rest-query
  and tr-query: PTranscript query-state = rest-query
  by blast
have header-prefix:
  verifier-header-transcript s fr (map snd f-fl) f-final as dg

```

```

    (map snd fl) final (PTranscript query-state)
using prefix-res tr-prefix tr-query
unfolding verifier-header-transcript-def verifier-header-messages-def
by simp
have header-eqs:
  fr' = fr  $\wedge$ 
  f-fri-roots' = map snd f-fl  $\wedge$ 
  f-final' = f-final  $\wedge$ 
  as' = as  $\wedge$ 
  dg' = dg  $\wedge$ 
  composition-fri-roots' = map snd fl
using verifier-header-transcript-unique[OF header-prefix header-hit]
by simp
have challenges-prefix:
  accepted-fri-challenges s (Some (result, final-state))
    (map fst f-fl) dg (map fst fl)
by (rule verifier-after-composition-fri-accepted-fri-challenges
    [OF prefix' suffix'])
have comp-bs-eq: comp-bs = map fst fl
using accepted-fri-challenges-unique
  [OF challenges-prefix challenges-hit[unfolded out-eq]]
by simp
show ?Head (Some (header, prefix-state))
using comp-bs-B comp-bs-eq header-eqs header-eq by simp
qed
qed

lemma wp-verify-monad-composition-fri-challenge-list-set-bound:
  fixes C :: prob
  assumes future: composition-fri-future-fresh s
  and subset:
     $\bigwedge dg. B \ dg \subseteq \text{fri-challenge-space } (\text{ceil-log } (\text{to-nat } dg + 1))$ 
  and bound:
     $\bigwedge dg. \text{nnreal } (\text{card } (B \ dg)) /$ 
     $\text{nnreal } (\text{CARD}(f) \wedge \text{ceil-log } (\text{to-nat } dg + 1)) \leq C$ 
  shows
    wp-event verify-monad (composition-fri-challenge-list-set-hit s B) s  $\leq C$ 
proof -
  let ?Head =
     $\lambda out. \text{case out of}$ 
     $\text{None} \Rightarrow \text{False}$ 
     $| \text{Some } ((-, -, -, dg, fl), -) \Rightarrow \text{map fst fl} \in B \ dg$ 
  have prefix-bound:
    wp-event verifier-composition-fri-prefix ?Head s  $\leq C$ 
  by (rule wp-verifier-composition-fri-prefix-challenge-space-set-bound
    [OF future subset bound])
  show ?thesis
    unfolding verify-monad-composition-fri-decomposition
  proof (rule wp-event-bind-bound-by-head-event[OF prefix-bound])

```

```

show composition-fri-challenge-list-set-hit s B None  $\impl$  ?Head None
  unfolding composition-fri-challenge-list-set-hit-def
    accepted-fri-challenges-def by blast
next
fix header prefix-state out
assume prefix:
  Some (header, prefix-state) ∈
    set-dist (execute verifier-composition-fri-prefix s)
and suffix:
  out ∈
    set-dist (execute
      (verifier-after-composition-fri header) prefix-state)
and hit: composition-fri-challenge-list-set-hit s B out
obtain fr f-fl f-final as dg fl where header-eq:
  header = (fr, f-fl, f-final, as, dg, fl)
by (cases header) auto
from hit obtain trace-bs dg' comp-bs where
  challenges-hit: accepted-fri-challenges s out trace-bs dg' comp-bs
and comp-bs-B: comp-bs ∈ B dg'
unfolding composition-fri-challenge-list-set-hit-def by blast
from challenges-hit obtain result final-state where out-eq:
  out = Some (result, final-state)
unfolding accepted-fri-challenges-def by blast
have prefix':
  Some ((fr, f-fl, f-final, as, dg, fl), prefix-state) ∈
    set-dist (execute verifier-composition-fri-prefix s)
using prefix header-eq by simp
have suffix':
  Some (result, final-state) ∈
    set-dist
      (execute
        (verifier-after-composition-fri
          (fr, f-fl, f-final, as, dg, fl))
        prefix-state)
using suffix header-eq out-eq by simp
have challenges-prefix:
  accepted-fri-challenges s (Some (result, final-state))
    (map fst f-fl) dg (map fst fl)
by (rule verifier-after-composition-fri-accepted-fri-challenges
    [OF prefix' suffix'])
have eqs:
  trace-bs = map fst f-fl ∧ dg' = dg ∧ comp-bs = map fst fl
using accepted-fri-challenges-unique
    [OF challenges-prefix challenges-hit[unfolded out-eq]]
by simp
then show ?Head (Some (header, prefix-state))
using comp-bs-B header-eq by simp
qed
qed

```


lemma *verifier-initial-alpha-list-set-error-bound*:
assumes *subset*: $B \subseteq \text{alpha-space}$
and *bounded*:
 $\text{nnreal } (\text{card } B) / \text{nnreal } (\text{card } \text{alpha-space}) \leq$
 $\text{composition-error-bound}$
shows
 $\text{wp-event verify-monad}$
 $(\text{alpha-list-set-hit } (\text{verifier-initial-state } tr) B)$
 $(\text{verifier-initial-state } tr) \leq \text{composition-error-bound}$
proof –
have $\text{wp-event verify-monad}$
 $(\text{alpha-list-set-hit } (\text{verifier-initial-state } tr) B)$
 $(\text{verifier-initial-state } tr) \leq$
 $\text{nnreal } (\text{card } B) / \text{nnreal } (\text{card } \text{alpha-space})$
by (rule *wp-verify-monad-alpha-list-set-bound*) (use *subset* **in** *simp-all*)
also have $\dots \leq \text{composition-error-bound}$
by (rule *bounded*)
finally show *?thesis* .
qed

lemma *verifier-initial-trace-fri-challenge-list-set-error-bound*:
assumes *subset*: $B \subseteq \text{fri-challenge-space } (\text{ceil-log } \text{clength})$
and *bounded*:
 $\text{nnreal } (\text{card } B) / \text{nnreal } (\text{CARD}'f) \wedge \text{ceil-log } \text{clength}) \leq$
 trace-fri-error
shows
 $\text{wp-event verify-monad}$
 $(\text{trace-fri-challenge-list-set-hit } (\text{verifier-initial-state } tr) B)$
 $(\text{verifier-initial-state } tr) \leq \text{trace-fri-error}$
proof –
have $\text{wp-event verify-monad}$
 $(\text{trace-fri-challenge-list-set-hit } (\text{verifier-initial-state } tr) B)$
 $(\text{verifier-initial-state } tr) \leq$
 $\text{nnreal } (\text{card } B) / \text{nnreal } (\text{CARD}'f) \wedge \text{ceil-log } \text{clength})$
by (rule *wp-verify-monad-trace-fri-challenge-list-set-bound*)
 (use *subset* **in** *simp-all*)
also have $\dots \leq \text{trace-fri-error}$
by (rule *bounded*)
finally show *?thesis* .
qed

lemma *verifier-initial-composition-fri-challenge-list-set-error-bound*:
assumes *subset*:
 $\bigwedge dg. B \ dg \subseteq \text{fri-challenge-space } (\text{ceil-log } (\text{to-nat } dg + 1))$
and *bounded*:
 $\bigwedge dg. \text{nnreal } (\text{card } (B \ dg)) /$
 $\text{nnreal } (\text{CARD}'f) \wedge \text{ceil-log } (\text{to-nat } dg + 1)) \leq$
 $\text{composition-fri-error}$

shows

wp-event verify-monad
(composition-fri-challenge-list-set-hit (verifier-initial-state tr) B)
(verifier-initial-state tr) ≤ composition-fri-error
by (*rule wp-verify-monad-composition-fri-challenge-list-set-bound*)
(use subset bounded in simp-all)

definition *accepted-fri-opening-transcript*

:: ('f, 'a) protocol-channel-scheme ⇒
(unit list × ('f, 'a) protocol-channel-scheme) option ⇒
'f list ⇒ 'f list ⇒ 'f ⇒
'f ⇒ 'f list ⇒ 'f list ⇒ 'f ⇒
nat list ⇒ 'f list list list ⇒ 'f list list list ⇒ bool

where

accepted-fri-opening-transcript s out trace-roots trace-bs trace-final
dg composition-roots composition-bs composition-final query-idxs
trace-round-layers composition-round-layers ↔
(∃ result final-state fr f-fl as fl query-state raw-idxs query-chunks.
out = Some (result, final-state) ∧
trace-roots = map snd f-fl ∧
trace-bs = map fst f-fl ∧
composition-roots = map snd fl ∧
composition-bs = map fst fl ∧
accepted-fri-challenges s out trace-bs dg composition-bs ∧
verifier-header-transcript s fr trace-roots trace-final as dg
composition-roots composition-final (PTranscript query-state) ∧
PState query-state =
verifier-header-state s fr trace-roots trace-final as dg
composition-roots composition-final ∧
PQueryCounter query-state = PQueryCounter s ∧
Some (result, final-state) ∈
set-dist (execute
(ntimes
(verifier-query-round-program fr f-fl trace-final as fl
composition-final)
rounds)
query-state) ∧
verifier-query-indices-derived s out
(verifier-header-state s fr trace-roots trace-final as dg
composition-roots composition-final)
(PTranscript query-state) trace-roots composition-roots query-idxs ∧
length trace-round-layers = rounds ∧
length composition-round-layers = rounds ∧
length raw-idxs = rounds ∧
query-idxs = map (λraw. index (to-nat raw)) raw-idxs ∧
length query-chunks = rounds ∧
PTranscript query-state =
List.concat query-chunks @ PTranscript final-state ∧
(∀ i < rounds.

```

    verifier-query-round-chunk (query-idxs ! i)
      trace-roots composition-roots (query-chunks ! i)) ∧
  (∀ i < rounds.
    query-round-fri-layer-transcripts (query-idxs ! i)
      trace-roots composition-roots (query-chunks ! i)
        (trace-round-layers ! i) (composition-round-layers ! i)) ∧
  (∀ i < rounds.
    fmlookup (HashMap final-state)
      (QueryIndexChallenge (PQueryCounter query-state + i) (state-after-query-chunks
(PState query-state) query-chunks i)) =
      Some (raw-idxs ! i)) ∧
  (∀ idx ∈ set query-idxs. idx < clength * scale))

```

lemma *verify-monad-accepted-fri-opening-transcript:*

fixes *s* :: ('f, 'a) protocol-channel-scheme

assumes *outcome*:

Some (result, final-state) ∈ set-dist (execute verify-monad s)

obtains *trace-roots trace-bs trace-final dg composition-roots composition-bs*
composition-final query-idxs trace-round-layers composition-round-layers

where *accepted-fri-opening-transcript s (Some (result, final-state))*

trace-roots trace-bs trace-final dg composition-roots composition-bs

composition-final query-idxs trace-round-layers composition-round-layers

proof —

from *verify-monad-accepted-fri-challenges[OF outcome]*

obtain *trace-bs dg composition-bs* **where** *challenges:*

accepted-fri-challenges s (Some (result, final-state))

trace-bs dg composition-bs

by *blast*

then obtain *result' final-state' fr f-fl trace-final as fl composition-final*

query-state **where**

out-eq: Some (result, final-state) = Some (result', final-state')

and *trace-bs-eq: trace-bs = map fst f-fl*

and *composition-bs-eq: composition-bs = map fst fl*

and *header:*

verifier-header-transcript s fr (map snd f-fl) trace-final as dg

(map snd fl) composition-final (PTranscript query-state)

and *query-state-eq:*

PState query-state =

verifier-header-state s fr (map snd f-fl) trace-final as dg

(map snd fl) composition-final

and *query-out:*

Some (result', final-state') ∈

set-dist (execute

(ntimes

(verifier-query-round-program fr f-fl trace-final as fl

composition-final)

rounds)

query-state)

and *query-count-header:*

```

    PQueryCounter query-state = PQueryCounter s
  unfolding accepted-fri-challenges-def by blast
have result'-eq: result' = result
  using out-eq by simp
have final-state'-eq: final-state' = final-state
  using out-eq by simp
have query-out':
  Some (result, final-state) ∈
    set-dist (execute
      (ntimes
        (verifier-query-round-program fr f-fl trace-final as fl
          composition-final)
        rounds)
      query-state)
  using query-out result'-eq final-state'-eq by simp
from ntimes-verifier-query-rounds-outcome[OF query-out]
obtain raw-idxs query-idxs query-chunks where
  len-raw: length raw-idxs = rounds
  and query-idxs-def: query-idxs = map (λraw. index (to-nat raw)) raw-idxs
  and len-chunks: length query-chunks = rounds
  and tr-query:
    PTranscript query-state = List.concat query-chunks @ PTranscript final-state
  and round-chunks:
    ∧ i. i < rounds ⇒
      verifier-query-round-chunk (query-idxs ! i)
      (map snd f-fl) (map snd fl) (query-chunks ! i)
  and lookups:
    ∧ i. i < rounds ⇒
      fmllookup (HashMap final-state)
      (QueryIndexChallenge (PQueryCounter query-state + i) (state-after-query-chunks (PState
        query-state) query-chunks i)) =
        Some (raw-idxs ! i)
  and idx-bounds:
    ∀ idx ∈ set query-idxs. idx < clength * scale
  by auto
have layer-exists:
  ∀ i < rounds.
    ∃ trace-layers composition-layers.
      query-round-fri-layer-transcripts (query-idxs ! i)
      (map snd f-fl) (map snd fl) (query-chunks ! i)
      trace-layers composition-layers
proof (intro allI impI)
  fix i
  assume i-bound: i < rounds
  from verifier-query-round-chunk-fri-layer-transcriptsE
  [OF round-chunks[OF i-bound]]
  show ∃ trace-layers composition-layers.
    query-round-fri-layer-transcripts (query-idxs ! i)
    (map snd f-fl) (map snd fl) (query-chunks ! i)

```

```

      trace-layers composition-layers
    by blast
  qed
  then obtain trace-layers-of composition-layers-of where layer-of:
     $\wedge i. i < \text{rounds} \implies$ 
      query-round-fri-layer-transcripts (query-idxs ! i)
      (map snd f-fl) (map snd fl) (query-chunks ! i)
      (trace-layers-of i) (composition-layers-of i)
    by metis
  let ?trace-round-layers = map trace-layers-of [0.. $\text{rounds}$ ]
  let ?composition-round-layers = map composition-layers-of [0.. $\text{rounds}$ ]
  have layers:
     $\forall i < \text{rounds}.$ 
      query-round-fri-layer-transcripts (query-idxs ! i)
      (map snd f-fl) (map snd fl) (query-chunks ! i)
      (?trace-round-layers ! i) (?composition-round-layers ! i)
  proof (intro allI impI)
    fix i
    assume i-bound:  $i < \text{rounds}$ 
    then show query-round-fri-layer-transcripts (query-idxs ! i)
      (map snd f-fl) (map snd fl) (query-chunks ! i)
      (?trace-round-layers ! i) (?composition-round-layers ! i)
      using layer-of[OF i-bound] by simp
  qed
  have derived:
    verifier-query-indices-derived s (Some (result, final-state))
    (verifier-header-state s fr (map snd f-fl) trace-final as dg
     (map snd fl) composition-final)
    (PTranscript query-state) (map snd f-fl) (map snd fl) query-idxs
  proof -
    have tr-query':
      List.concat query-chunks @ PTranscript final-state = PTranscript query-state
    using tr-query by simp
    have lookups':
       $\forall i < \text{rounds}.$ 
        fmlookup (HashMap final-state)
        (QueryIndexChallenge (PQueryCounter s + i) (state-after-query-chunks
          (verifier-header-state s fr (map snd f-fl) trace-final as dg
            (map snd fl) composition-final)
          query-chunks i)) =
        Some (raw-idxs ! i)
    using lookups query-state-eq query-count-header by simp
  show ?thesis
    unfolding verifier-query-indices-derived-def
    apply (rule exI[where x=result])
    apply (rule exI[where x=final-state])
    apply (rule exI[where x=raw-idxs])
    apply (rule exI[where x=query-chunks])
    apply (rule exI[where x=PTranscript final-state])

```

```

apply (intro conjI)
  apply simp
  apply (rule len-raw)
  apply (rule query-idxs-def)
  apply (rule len-chunks)
  apply (rule tr-query')
apply (intro allI impI, rule round-chunks, assumption)
apply (rule lookups')
using idx-bounds by simp
qed
have tr-query-tail:
  List.concat query-chunks @ PTranscript final-state = PTranscript query-state
  using tr-query by simp
have lookups-all:
   $\forall i < \text{rounds}.$ 
  fmlookup (HashMap final-state)
    (QueryIndexChallenge (PQueryCounter query-state + i) (state-after-query-chunks (PState
query-state) query-chunks i)) =
    Some (raw-idxs ! i)
  using lookups by blast
have lookups-all-header:
   $\forall i < \text{rounds}.$ 
  fmlookup (HashMap final-state)
    (QueryIndexChallenge (PQueryCounter query-state + i)
      (state-after-query-chunks
        (verifier-header-state s fr (map snd f-fl) trace-final as dg
          (map snd fl) composition-final)
        query-chunks i)) =
    Some (raw-idxs ! i)
  using lookups-all query-state-eq by simp
have challenges-maps:
  accepted-fri-challenges s (Some (result, final-state))
    (map fst f-fl) dg (map fst fl)
  using challenges trace-bs-eq composition-bs-eq by simp
have derived-maps:
  verifier-query-indices-derived s (Some (result, final-state))
    (verifier-header-state s fr (map snd f-fl) trace-final as dg
      (map snd fl) composition-final)
    (PTranscript query-state) (map snd f-fl) (map snd fl)
    (map ( $\lambda \text{raw}.$  index (to-nat raw)) raw-idxs)
  using derived query-idxs-def by simp
have layers-maps:
   $\forall i < \text{rounds}.$ 
  query-round-fri-layer-transcripts (index (to-nat (raw-idxs ! i)))
    (map snd f-fl) (map snd fl) (query-chunks ! i)
    (trace-layers-of i) (composition-layers-of i)
  using layers query-idxs-def len-raw by auto
have round-chunks-raw:
   $\forall i < \text{rounds}.$ 

```

```

    verifier-query-round-chunk (index (to-nat (raw-idxs ! i)))
      (map snd f-fl) (map snd fl) (query-chunks ! i)
  using round-chunks query-idxs-def len-raw by auto
have lookups-header:
  ∀ i < rounds.
    fmlookup (HashMap final-state)
      (QueryIndexChallenge (PQueryCounter s + i) (state-after-query-chunks
        (verifier-header-state s fr (map snd f-fl) trace-final as dg
          (map snd fl) composition-final)
        query-chunks i)) =
      Some (raw-idxs ! i)
  using lookups-all query-state-eq query-count-header by simp
have accepted:
  accepted-fri-opening-transcript s (Some (result, final-state))
    (map snd f-fl) trace-bs trace-final dg (map snd fl) composition-bs
    composition-final query-idxs ?trace-round-layers ?composition-round-layers
  unfolding accepted-fri-opening-transcript-def
  apply (rule exI[where x=result])
  apply (rule exI[where x=final-state])
  apply (rule exI[where x=fr])
  apply (rule exI[where x=f-fl])
  apply (rule exI[where x=as])
  apply (rule exI[where x=fl])
  apply (rule exI[where x=query-state])
  apply (rule exI[where x=raw-idxs])
  apply (rule exI[where x=query-chunks])
  apply (simp add: challenges header query-state-eq query-out' derived len-raw
    query-count-header query-idxs-def len-chunks tr-query-tail layers
    lookups-all idx-bounds trace-bs-eq composition-bs-eq challenges-maps
    derived-maps layers-maps lookups-header lookups-all-header round-chunks)
  apply (rule round-chunks-raw)
done
then show ?thesis
  by (rule that)
qed

```

```

lemma accepted-fri-opening-transcript-challenges:
  assumes fri-openings:
    accepted-fri-opening-transcript s out trace-roots trace-bs trace-final
    dg composition-roots composition-bs composition-final query-idxs
    trace-round-layers composition-round-layers
  shows accepted-fri-challenges s out trace-bs dg composition-bs
  using fri-openings
  unfolding accepted-fri-opening-transcript-def
  apply (elim exE conjE)
  apply assumption
done

```

```

lemma accepted-fri-opening-transcript-trace-challenge-list-set-hit:

```

assumes *fri-openings*:
accepted-fri-opening-transcript s out trace-roots trace-bs trace-final
dg composition-roots composition-bs composition-final query-idxs
trace-round-layers composition-round-layers
and *member*: *trace-bs* $\in B$
shows *trace-fri-challenge-list-set-hit s B out*
using *fri-openings*
unfolding *accepted-fri-opening-transcript-def trace-fri-challenge-list-set-hit-def*
apply (*elim exE conjE*)
apply (*intro exI conjI*)
apply *assumption*
apply (*rule member*)
done

lemma *accepted-fri-opening-transcript-composition-challenge-list-set-hit*:

assumes *fri-openings*:
accepted-fri-opening-transcript s out trace-roots trace-bs trace-final
dg composition-roots composition-bs composition-final query-idxs
trace-round-layers composition-round-layers
and *member*: *composition-bs* $\in B$ *dg*
shows *composition-fri-challenge-list-set-hit s B out*
using *fri-openings*
unfolding *accepted-fri-opening-transcript-def*
composition-fri-challenge-list-set-hit-def
apply (*elim exE conjE*)
apply (*intro exI conjI*)
apply *assumption*
apply (*rule member*)
done

lemma *accepted-fri-opening-transcript-shapes*:

assumes *fri-openings*:
accepted-fri-opening-transcript s out trace-roots trace-bs trace-final
dg composition-roots composition-bs composition-final query-idxs
trace-round-layers composition-round-layers
shows *length trace-roots = ceil-log clength*
and *length trace-bs = length trace-roots*
and *length composition-roots = ceil-log (to-nat dg + 1)*
and *length composition-bs = length composition-roots*
and *length trace-round-layers = rounds*
and *length composition-round-layers = rounds*
and *length query-idxs = rounds*
using *fri-openings*
unfolding *accepted-fri-opening-transcript-def verifier-header-transcript-def*
by *auto*

lemma *accepted-fri-opening-transcript-algebraic-round-counts*:

assumes *fri-openings*:
accepted-fri-opening-transcript s out trace-roots trace-bs trace-final


```

    dg composition-roots composition-bs composition-final query-idxs
    trace-round-layers composition-round-layers
shows length trace-roots = trace-fri-algebraic-round-count
and length trace-bs = trace-fri-algebraic-round-count
and length composition-roots = composition-fri-algebraic-round-count dg
and length composition-bs = composition-fri-algebraic-round-count dg
and length trace-round-layers = rounds
and length composition-round-layers = rounds
and length query-idxs = rounds
proof –
  have shapes:
    length trace-roots = ceil-log clength
    length trace-bs = length trace-roots
    length composition-roots = ceil-log (to-nat dg + 1)
    length composition-bs = length composition-roots
    length trace-round-layers = rounds
    length composition-round-layers = rounds
    length query-idxs = rounds
    using accepted-fri-opening-transcript-shapes[OF fri-openings] by blast+
  show length trace-roots = trace-fri-algebraic-round-count
    using shapes(1) by (simp add: trace-fri-algebraic-round-count-eq)
  show length trace-bs = trace-fri-algebraic-round-count
    using shapes(1,2) by (simp add: trace-fri-algebraic-round-count-eq)
  show length composition-roots = composition-fri-algebraic-round-count dg
    using shapes(3) by (simp add: composition-fri-algebraic-round-count-eq)
  show length composition-bs = composition-fri-algebraic-round-count dg
    using shapes(3,4) by (simp add: composition-fri-algebraic-round-count-eq)
  show length trace-round-layers = rounds
    using shapes(5) .
  show length composition-round-layers = rounds
    using shapes(6) .
  show length query-idxs = rounds
    using shapes(7) .
qed

```

```

lemma accepted-fri-opening-transcript-trace-round-layer-length:
  assumes fri-openings:
    accepted-fri-opening-transcript s out trace-roots trace-bs trace-final
    dg composition-roots composition-bs composition-final query-idxs
    trace-round-layers composition-round-layers
    and round-bound: round-idx < rounds
  shows length (trace-round-layers ! round-idx) = length trace-roots
proof –
  obtain result final-state fr f-fl as fl query-state raw-idxs query-chunks
  where layers:
     $\bigwedge i. i < rounds \implies$ 
    query-round-fri-layer-transcripts (query-idxs ! i)
    trace-roots composition-roots (query-chunks ! i)
    (trace-round-layers ! i) (composition-round-layers ! i)

```

using *fri-openings* **unfolding** *accepted-fri-opening-transcript-def* **by** *blast*
from *layers*[*OF round-bound*]
obtain *query-chunk trace-fri-chunk composition-fri-chunk leaves paths* **where**
trace-layers:
fri-layers-transcript (*length trace-roots*) (*clength * scale*)
(*trace-round-layers ! round-idx*) *trace-fri-chunk*
unfolding *query-round-fri-layer-transcripts-def* **by** *blast*
then show *?thesis*
unfolding *fri-layers-transcript-def* **by** *simp*
qed

lemma *accepted-fri-opening-transcript-composition-round-layer-length:*
assumes *fri-openings:*
accepted-fri-opening-transcript s out trace-roots trace-bs trace-final
dg composition-roots composition-bs composition-final query-idxs
trace-round-layers composition-round-layers
and *round-bound: round-idx < rounds*
shows *length (composition-round-layers ! round-idx) = length composition-roots*
proof –
obtain *result final-state fr f-fl as fl query-state raw-idxs query-chunks*
where *layers:*
 $\bigwedge i. i < rounds \implies$
query-round-fri-layer-transcripts (*query-idxs ! i*)
trace-roots composition-roots (*query-chunks ! i*)
(*trace-round-layers ! i*) (*composition-round-layers ! i*)
using *fri-openings* **unfolding** *accepted-fri-opening-transcript-def* **by** *blast*
from *layers*[*OF round-bound*]
obtain *query-chunk trace-fri-chunk composition-fri-chunk leaves paths* **where**
composition-layers:
fri-layers-transcript (*length composition-roots*) (*clength * scale*)
(*composition-round-layers ! round-idx*) *composition-fri-chunk*
unfolding *query-round-fri-layer-transcripts-def* **by** *blast*
then show *?thesis*
unfolding *fri-layers-transcript-def* **by** *simp*
qed

lemma *accepted-fri-opening-transcript-trace-layer-opening:*
assumes *fri-openings:*
accepted-fri-opening-transcript s out trace-roots trace-bs trace-final
dg composition-roots composition-bs composition-final query-idxs
trace-round-layers composition-round-layers
and *round-bound: round-idx < rounds*
and *layer-bound: layer-idx < length trace-roots*
obtains *root challenge len idx sibling-idx xp xp-path xn xn-path*
where *root = trace-roots ! layer-idx*
and *challenge = trace-bs ! layer-idx*
and *len = fri-layer-lengths (length trace-roots) (clength * scale) ! layer-idx*
and *idx =*
fri-layer-indices (length trace-roots) (query-idxs ! round-idx)

```

    (clength * scale) ! layer-idx
  and sibling-idx = fri-sibling-index len idx
  and fri-layer-opening-chunk len xp xp-path xn xn-path
    (trace-round-layers ! round-idx ! layer-idx)
proof -
  obtain result final-state fr f-fl as fl query-state raw-idxs query-chunks
  where layers:
     $\bigwedge i. i < rounds \implies$ 
      query-round-fri-layer-transcripts (query-idxs ! i)
      trace-roots composition-roots (query-chunks ! i)
      (trace-round-layers ! i) (composition-round-layers ! i)
  using fri-openings unfolding accepted-fri-opening-transcript-def by blast
  from layers[OF round-bound]
  obtain query-chunk trace-fri-chunk composition-fri-chunk leaves paths where
    trace-layers:
      fri-layers-transcript (length trace-roots) (clength * scale)
      (trace-round-layers ! round-idx) trace-fri-chunk
  unfolding query-round-fri-layer-transcripts-def by blast
  from fri-layers-transcript-nth-opening[OF trace-layers layer-bound]
  obtain xp xp-path xn xn-path where layer-opening:
    fri-layer-opening-chunk
    (fri-layer-lengths (length trace-roots) (clength * scale) ! layer-idx)
    xp xp-path xn xn-path (trace-round-layers ! round-idx ! layer-idx)
  by blast
  let ?len = fri-layer-lengths (length trace-roots) (clength * scale) ! layer-idx
  let ?idx =
    fri-layer-indices (length trace-roots) (query-idxs ! round-idx)
    (clength * scale) ! layer-idx
  let ?sidx = fri-sibling-index ?len ?idx
  show ?thesis
  by (rule that[of trace-roots ! layer-idx trace-bs ! layer-idx
    ?len ?idx ?sidx xp xp-path xn xn-path])
    (use layer-opening in simp-all)
qed

```

```

lemma accepted-fri-opening-transcript-composition-layer-opening:
  assumes fri-openings:
    accepted-fri-opening-transcript s out trace-roots trace-bs trace-final
    dg composition-roots composition-bs composition-final query-idxs
    trace-round-layers composition-round-layers
  and round-bound: round-idx < rounds
  and layer-bound: layer-idx < length composition-roots
  obtains root challenge len idx sibling-idx xp xp-path xn xn-path
  where root = composition-roots ! layer-idx
  and challenge = composition-bs ! layer-idx
  and len =
    fri-layer-lengths (length composition-roots) (clength * scale) ! layer-idx
  and idx =
    fri-layer-indices (length composition-roots) (query-idxs ! round-idx)

```

```

    (clength * scale) ! layer-idx
  and sibling-idx = fri-sibling-index len idx
  and fri-layer-opening-chunk len xp xp-path xn xn-path
    (composition-round-layers ! round-idx ! layer-idx)
proof -
  obtain result final-state fr f-fl as fl query-state raw-idxs query-chunks
  where layers:
     $\bigwedge i. i < rounds \implies$ 
      query-round-fri-layer-transcripts (query-idxs ! i)
      trace-roots composition-roots (query-chunks ! i)
      (trace-round-layers ! i) (composition-round-layers ! i)
  using fri-openings unfolding accepted-fri-opening-transcript-def by blast
  from layers[OF round-bound]
  obtain query-chunk trace-fri-chunk composition-fri-chunk leaves paths where
    composition-layers:
      fri-layers-transcript (length composition-roots) (clength * scale)
      (composition-round-layers ! round-idx) composition-fri-chunk
  unfolding query-round-fri-layer-transcripts-def by blast
  from fri-layers-transcript-nth-opening[OF composition-layers layer-bound]
  obtain xp xp-path xn xn-path where layer-opening:
    fri-layer-opening-chunk
    (fri-layer-lengths (length composition-roots) (clength * scale) ! layer-idx)
    xp xp-path xn xn-path
    (composition-round-layers ! round-idx ! layer-idx)
  by blast
  let ?len =
    fri-layer-lengths (length composition-roots) (clength * scale) ! layer-idx
  let ?idx =
    fri-layer-indices (length composition-roots) (query-idxs ! round-idx)
    (clength * scale) ! layer-idx
  let ?sidx = fri-sibling-index ?len ?idx
  show ?thesis
  by (rule that[of composition-roots ! layer-idx composition-bs ! layer-idx
    ?len ?idx ?sidx xp xp-path xn xn-path])
    (use layer-opening in simp-all)
qed

```

0.0.3 Global FRI table bindings

The verifier sees authenticated local openings. The following predicates describe the stronger deterministic object needed by the algebraic FRI argument: complete layer tables bound to the Merkle roots, with all local openings matching the corresponding table entries.

definition *fri-layer-tables-shape*
 $:: nat \Rightarrow 'f\ list\ list \Rightarrow bool$
where
fri-layer-tables-shape $n\ tables \longleftrightarrow$
 $length\ tables = n \wedge$

$(\forall j < n.$
 $\text{length } (tables ! j) =$
 $\text{fri-layer-lengths } n \text{ (clength * scale) ! } j)$

definition *fri-roots-bind-tables*

$:: 'f \text{ list} \Rightarrow 'f \text{ list list} \Rightarrow$
 $('f, 'a) \text{ protocol-channel-scheme} \Rightarrow \text{bool}$

where

$\text{fri-roots-bind-tables roots tables final-state} \longleftrightarrow$
 $\text{length tables} = \text{length roots} \wedge$
 $(\forall j < \text{length roots}.$
 $\text{merkle-root-binds-table (roots ! } j) \text{ (tables ! } j) \text{ final-state})$

definition *fri-bound-layer-tables*

$:: 'f \text{ list} \Rightarrow 'f \text{ list list} \Rightarrow$
 $('f, 'a) \text{ protocol-channel-scheme} \Rightarrow \text{bool}$

where

$\text{fri-bound-layer-tables roots tables final-state} \longleftrightarrow$
 $\text{fri-layer-tables-shape (length roots) tables} \wedge$
 $\text{fri-roots-bind-tables roots tables final-state}$

definition *fri-layer-opening-matches-bound-table*

$:: \text{nat} \Rightarrow \text{nat} \Rightarrow 'f \text{ list list} \Rightarrow 'f \text{ list list} \Rightarrow \text{bool}$

where

$\text{fri-layer-opening-matches-bound-table query-idx layer-idx tables}$
 $\text{layer-chunks} \longleftrightarrow$
 $\text{layer-idx} < \text{length tables} \wedge$
 $\text{length layer-chunks} = \text{length tables} \wedge$
 $(\text{let len} =$
 $\text{fri-layer-lengths (length tables) (clength * scale) ! layer-idx};$
 $\text{idx} =$
 $\text{fri-layer-indices (length tables) query-idx (clength * scale) !}$
 layer-idx
 $\text{in } \exists xp \text{ xp-path } xn \text{ xn-path}.$
 $\text{fri-layer-opening-chunk len xp xp-path xn xn-path}$
 $(\text{layer-chunks ! layer-idx}) \wedge$
 $\text{fri-opening-matches-table len idx (tables ! layer-idx) xp xn})$

definition *fri-round-openings-match-tables*

$:: 'f \text{ list list} \Rightarrow \text{nat} \Rightarrow 'f \text{ list list} \Rightarrow \text{bool}$

where

$\text{fri-round-openings-match-tables tables query-idx layer-chunks} \longleftrightarrow$
 $\text{length layer-chunks} = \text{length tables} \wedge$
 $(\forall j < \text{length tables}.$
 $\text{fri-layer-opening-matches-bound-table query-idx } j \text{ tables}$
 $\text{layer-chunks})$

definition *fri-all-round-openings-match-tables*

$:: 'f \text{ list list} \Rightarrow \text{nat list} \Rightarrow 'f \text{ list list list} \Rightarrow \text{bool}$

where

fri-all-round-openings-match-tables tables query-idxs round-layers $\longleftrightarrow$
length round-layers = length query-idxs $\wedge$
 $(\forall i < \text{length query-idxs}.$
fri-round-openings-match-tables tables (query-idxs ! i)
(round-layers ! i))

definition *fri-final-table-matches*

$:: 'f \text{ list} \Rightarrow 'f \Rightarrow \text{bool}$

where

fri-final-table-matches final-table final $\longleftrightarrow$
final-table $\neq []$ $\wedge$
fri-final-constant-consistent final-table final

definition *trace-fri-tables-bound*

$:: ('f, 'a) \text{ protocol-channel-scheme} \Rightarrow$
 $(\text{unit list} \times ('f, 'a) \text{ protocol-channel-scheme}) \text{ option} \Rightarrow$
 $'f \text{ list} \Rightarrow 'f \text{ list} \Rightarrow 'f \Rightarrow \text{nat list} \Rightarrow$
 $'f \text{ list list list} \Rightarrow 'f \text{ list list} \Rightarrow 'f \text{ list} \Rightarrow \text{bool}$

where

trace-fri-tables-bound s out trace-roots trace-bs trace-final
query-idxs trace-round-layers trace-tables trace-final-table $\longleftrightarrow$
 $(\exists \text{result final-state dg composition-roots composition-bs}$
composition-final composition-round-layers.
out = Some (result, final-state) $\wedge$
accepted-fri-opening-transcript s out trace-roots trace-bs
trace-final dg composition-roots composition-bs composition-final
query-idxs trace-round-layers composition-round-layers $\wedge$
fri-bound-layer-tables trace-roots trace-tables final-state $\wedge$
fri-all-round-openings-match-tables trace-tables query-idxs
trace-round-layers $\wedge$
fri-final-table-matches trace-final-table trace-final)

definition *composition-fri-tables-bound*

$:: ('f, 'a) \text{ protocol-channel-scheme} \Rightarrow$
 $(\text{unit list} \times ('f, 'a) \text{ protocol-channel-scheme}) \text{ option} \Rightarrow$
 $'f \Rightarrow 'f \text{ list} \Rightarrow 'f \text{ list} \Rightarrow 'f \Rightarrow \text{nat list} \Rightarrow$
 $'f \text{ list list list} \Rightarrow 'f \text{ list list} \Rightarrow 'f \text{ list} \Rightarrow \text{bool}$

where

composition-fri-tables-bound s out dg composition-roots
composition-bs composition-final query-idxs composition-round-layers
composition-tables composition-final-table $\longleftrightarrow$
 $(\exists \text{result final-state trace-roots trace-bs trace-final}$
trace-round-layers.
out = Some (result, final-state) $\wedge$
accepted-fri-opening-transcript s out trace-roots trace-bs
trace-final dg composition-roots composition-bs composition-final
query-idxs trace-round-layers composition-round-layers $\wedge$
fri-bound-layer-tables composition-roots composition-tables

$final-state \wedge$
 $fri-all-round-openings-match-tables \text{ composition-tables query-idxs}$
 $composition-round-layers \wedge$
 $fri-final-table-matches \text{ composition-final-table composition-final})$

definition *fri-initial-table-agrees*

$:: 'f \text{ list} \Rightarrow 'f \text{ list list} \Rightarrow \text{bool}$

where

$fri-initial-table-agrees \text{ initial-table tables} \longleftrightarrow$
 $tables = [] \vee hd \text{ tables} = \text{initial-table}$

definition *fri-merkle-binding-good*

$:: ('f, 'a) \text{ protocol-channel-scheme} \Rightarrow$
 $(\text{unit list} \times ('f, 'a) \text{ protocol-channel-scheme}) \text{ option} \Rightarrow \text{bool}$

where

$fri-merkle-binding-good \text{ s out} \longleftrightarrow$
 $\text{accepted out} \longrightarrow$
 $(\exists \text{ trace-table composition-table as query-idxs}$
 $\text{trace-roots trace-bs trace-final dg composition-roots}$
 $\text{composition-bs composition-final trace-round-layers}$
 $\text{composition-round-layers trace-tables composition-tables}$
 $\text{trace-final-table composition-final-table.}$
 $\text{accepted-with-bound-tables s out trace-table composition-table as}$
 $\text{query-idxs} \wedge$
 $\text{accepted-fri-opening-transcript s out trace-roots trace-bs}$
 $\text{trace-final dg composition-roots composition-bs}$
 $\text{composition-final query-idxs trace-round-layers}$
 $\text{composition-round-layers} \wedge$
 $\text{trace-fri-tables-bound s out trace-roots trace-bs trace-final}$
 $\text{query-idxs trace-round-layers trace-tables trace-final-table} \wedge$
 $\text{composition-fri-tables-bound s out dg composition-roots}$
 $\text{composition-bs composition-final query-idxs}$
 $\text{composition-round-layers composition-tables}$
 $\text{composition-final-table} \wedge$
 $\text{fri-initial-table-agrees trace-table trace-tables} \wedge$
 $\text{fri-initial-table-agrees composition-table composition-tables})$

definition *fri-merkle-binding-bad*

$:: ('f, 'a) \text{ protocol-channel-scheme} \Rightarrow$
 $(\text{unit list} \times ('f, 'a) \text{ protocol-channel-scheme}) \text{ option} \Rightarrow \text{bool}$

where

$fri-merkle-binding-bad \text{ s out} \longleftrightarrow$
 $\text{accepted out} \wedge \neg \text{fri-merkle-binding-good s out}$

definition *fri-merkle-binding-bound*

$:: ('f, 'a) \text{ protocol-channel-scheme} \Rightarrow \text{bool}$

where

$fri-merkle-binding-bound \text{ s} \longleftrightarrow$
 $\text{wp-event verify-monad (fri-merkle-binding-bad s)} \text{ s} \leq$

merkle-binding-error

end

end

theory *Soundness-Oracle-Target*
 imports *Soundness-FRI*
begin

Random-oracle target-hit events and target-budget calculus.

context *soundness*
begin

definition *hash-map-output-collision-bound*
 $:: (f, 'a) \text{ protocol-channel-scheme} \Rightarrow \text{bool}$
 where
 $\text{hash-map-output-collision-bound } s \longleftrightarrow$
 $\text{wp-event verify-monad } (\text{hash-map-output-collision-bad } s) \ s \leq$
 $\text{merkle-binding-error}$

definition *hash-map-new-output-collision*
 $:: (f, 'a) \text{ protocol-channel-scheme} \Rightarrow$
 $(f, 'a) \text{ protocol-channel-scheme} \Rightarrow \text{bool}$
 where
 $\text{hash-map-new-output-collision initial final} \longleftrightarrow$
 $\neg \text{hash-map-output-collision initial} \wedge$
 $\text{hash-map-output-collision final}$

definition *hash-new-collision-event*
 $:: (f, 'a) \text{ protocol-channel-scheme} \Rightarrow$
 $(r \times (f, 'a) \text{ protocol-channel-scheme}) \text{ option} \Rightarrow \text{bool}$
 where
 $\text{hash-new-collision-event } s \text{ out} \longleftrightarrow$
 $(\text{case out of}$
 $\text{None} \Rightarrow \text{False}$
 $| \text{Some } (-, t) \Rightarrow \text{hash-map-new-output-collision } s \ t)$

definition *hash-map-new-output-hit*
 $:: f \text{ set} \Rightarrow$
 $(f, 'a) \text{ protocol-channel-scheme} \Rightarrow$
 $(f, 'a) \text{ protocol-channel-scheme} \Rightarrow \text{bool}$
 where
 $\text{hash-map-new-output-hit } B \text{ initial final} \longleftrightarrow$
 $(\exists x \ y.$
 $\text{fmlookup } (\text{HashMap initial}) \ x = \text{None} \wedge$
 $\text{fmlookup } (\text{HashMap final}) \ x = \text{Some } y \wedge$
 $y \in B)$

definition *hash-new-output-hit-event*
 $:: 'f \text{ set} \Rightarrow$
 $(f, 'a) \text{ protocol-channel-scheme} \Rightarrow$
 $('r \times (f, 'a) \text{ protocol-channel-scheme}) \text{ option} \Rightarrow \text{bool}$
where
 $\text{hash-new-output-hit-event } B \text{ s out} \longleftrightarrow$
 $(\text{case out of}$
 $\text{None} \Rightarrow \text{False}$
 $| \text{Some } (-, t) \Rightarrow \text{hash-map-new-output-hit } B \text{ s } t)$

lemma *hash-map-new-output-hit-subset*:
assumes *subset*: $A \subseteq B$
and *hit*: $\text{hash-map-new-output-hit } A \text{ s } t$
shows $\text{hash-map-new-output-hit } B \text{ s } t$
using *subset hit unfolding hash-map-new-output-hit-def* **by** *blast*

lemma *protocol-merkle-created-tree-pullback-or-new-output-hit*:
assumes *created*: $\text{protocol-merkle.created-tree } xs \text{ tree } t$
and *ext*: $s \leq t$
shows
 $\text{protocol-merkle.created-tree } xs \text{ tree } s \vee$
 $\text{hash-map-new-output-hit } (\text{set-tree tree}) \text{ s } t$
using *created ext*

proof (*induction length xs arbitrary: xs tree s t rule: less-induct*)
case *less*
show *?case*
proof (*cases xs*)
case *Nil*
then show *?thesis*
using *less.prem*
by (*simp add: protocol-merkle.created-tree.simps*)
next
case (*Cons x rest*)
have *xs-cons*: $xs = x \# rest$
using *Cons* **by** *simp*
show *?thesis*
proof (*cases rest*)
case *Nil*
then obtain *h* **where**
 $\text{tree-eq}: \text{tree} = \langle \rangle, h, \langle \rangle$
and *lookup-t*: $\text{fmlookup } (\text{HashMap } t) \text{ x} = \text{Some } h$
using *less.prem Cons*
by (*auto simp: protocol-merkle.created-tree.simps*)
show *?thesis*
proof (*cases fmlookup (HashMap s) x*)
case *None*
then have $\text{hash-map-new-output-hit } (\text{set-tree tree}) \text{ s } t$
unfolding *hash-map-new-output-hit-def tree-eq*

```

    using lookup-t by auto
  then show ?thesis by simp
next
  case (Some h')
  have lookup-t': fmllookup (HashMap t) x = Some h'
    by (rule hash-extension-lookup[OF Some less.prem(2)])
  then have h'-eq: h' = h
    using lookup-t by simp
  have protocol-merkle.created-tree [x] tree s
    unfolding tree-eq
    using Some h'-eq
    by (simp add: protocol-merkle.created-tree.simps)
  then show ?thesis
    using Cons Nil by simp
qed
next
  case (Cons y ys)
  let ?xs = x # y # ys
  let ?i = length ?xs div 2
  have xs-eq: xs = ?xs
    using xs-cons Cons by simp
  from less.prem(1) obtain l h r where
    tree-eq: tree = ⟨l, h, r⟩
    and left-t:
      protocol-merkle.created-tree (take ?i ?xs) l t
    and right-t:
      protocol-merkle.created-tree (drop ?i ?xs) r t
    and lookup-t:
      fmllookup (HashMap t) (MerkleNode (value l) (value r)) =
        Some h
    unfolding xs-eq
    by (auto simp: protocol-merkle.created-tree.simps Let-def)
  have i-pos: 0 < ?i
    by simp
  have i-lt: ?i < length ?xs
    by simp
  have len-left: length (take ?i ?xs) < length xs
    using i-lt unfolding xs-eq by simp
  have len-right: length (drop ?i ?xs) < length xs
    using i-pos unfolding xs-eq by simp
  have left-split:
    protocol-merkle.created-tree (take ?i ?xs) l s ∨
    hash-map-new-output-hit (set-tree l) s t
    by (rule less.hyps[OF len-left])
    (use left-t less.prem(2) in simp-all)
  from left-split show ?thesis
proof
  assume left-s:
    protocol-merkle.created-tree (take ?i ?xs) l s

```

```

have right-split:
  protocol-merkle.created-tree (drop ?i ?xs) r s ∨
    hash-map-new-output-hit (set-tree r) s t
  by (rule less.hyps[OF len-right])
    (use right-t less.prems(2) in simp-all)
from right-split show ?thesis
proof
  assume right-s:
    protocol-merkle.created-tree (drop ?i ?xs) r s
  show ?thesis
  proof (cases
    fmlookup (HashMap s)
      (MerkleNode (value l) (value r)))
    case None
      then have hash-map-new-output-hit (set-tree tree) s t
        unfolding hash-map-new-output-hit-def tree-eq
        using lookup-t by auto
      then show ?thesis by simp
    next
      case (Some h')
        have lookup-t':
          fmlookup (HashMap t) (MerkleNode (value l) (value r)) =
            Some h'
          by (rule hash-extension-lookup[OF Some less.prems(2)])
        then have h'-eq: h' = h
          using lookup-t by simp
        have protocol-merkle.created-tree ?xs tree s
          unfolding tree-eq
          using left-s right-s Some h'-eq
          by (auto simp add: protocol-merkle.created-tree.simps Let-def)
        then have protocol-merkle.created-tree xs tree s
          unfolding xs-eq .
        then show ?thesis by blast
      qed
    next
      assume right-hit: hash-map-new-output-hit (set-tree r) s t
      have hash-map-new-output-hit (set-tree tree) s t
        by (rule hash-map-new-output-hit-subset[OF - right-hit])
          (auto simp add: tree-eq)
      then show ?thesis by simp
      qed
    next
      assume left-hit: hash-map-new-output-hit (set-tree l) s t
      have hash-map-new-output-hit (set-tree tree) s t
        by (rule hash-map-new-output-hit-subset[OF - left-hit])
          (auto simp add: tree-eq)
      then show ?thesis by simp
      qed
    qed

```

qed
qed

lemma *protocol-created-tree-pullback-or-new-output-hit*:
assumes *created*: *protocol-created-tree xs tree t*
and *ext*: $s \leq t$
shows
protocol-created-tree xs tree s $\vee$
hash-map-new-output-hit (set-tree tree) s t
using *protocol-merkle-created-tree-pullback-or-new-output-hit*
 $[OF\ created[unfolded\ protocol-created-tree-def]\ ext]$
unfolding *protocol-created-tree-def* .

lemma *protocol-created-tree-nonempty-root-lookup*:
assumes *created*: *protocol-created-tree xs tree s*
and *nonempty*: $xs \neq []$
obtains *input* **where**
fmlookup (HashMap s) input = Some (value tree)
proof (*cases length xs = 1*)
case *True*
then obtain *x* **where** *xs-eq*: $xs = [x]$
using *nonempty* **by** (*cases xs*) *auto*
from *protocol-created-tree-singletonD[OF created[unfolded xs-eq]]*
obtain *h* **where**
tree-eq: $tree = \langle \rangle, h, \langle \rangle$
and *lookup*: *fmlookup (HashMap s) (MerkleLeaf x) = Some h*
by *blast*
show *?thesis*
by (*rule that[of MerkleLeaf x] (use lookup in <simp add: tree-eq>)*)
next
case *False*
have *len*: $2 \leq \text{length } xs$
using *nonempty False* **by** (*cases xs; cases tl xs*) *auto*
from *protocol-created-tree-internalD[OF created len]*
obtain *l h r i* **where**
tree-eq: $tree = \langle l, h, r \rangle$
and *lookup*:
fmlookup (HashMap s) (MerkleNode (value l) (value r)) = Some h
by *blast*
show *?thesis*
by (*rule that[of MerkleNode (value l) (value r)]*)
 $(\text{use } lookup \text{ in } \langle simp \text{ add: tree-eq } \rangle)$
qed

lemma *merkle-root-binds-nonempty-table-output-lookup*:
fixes *rt* :: 'f
assumes *bind*: *merkle-root-binds-table rt table s*
and *nonempty*: $table \neq []$
obtains *input* **where**

```

    fmlookup (HashMap s) input = Some rt
proof -
  from bind obtain tree where
    created: created-tree table tree s
    and root-eq: rt = value tree
    unfolding merkle-root-binds-table-def by blast
  from protocol-created-tree-nonempty-root-lookup
    [OF created[unfolded created-tree-def] nonempty]
  obtain input where
    fmlookup (HashMap s) input = Some (value tree)
    by blast
  then have lookup: fmlookup (HashMap s) input = Some rt
    using root-eq by simp
  show ?thesis
    by (rule that[OF lookup])
qed

lemma merkle-root-binds-table-pullback-or-new-tree-output-hit:
  assumes ext: prefix-state  $\leq$  final-state
    and bind: merkle-root-binds-table rt table final-state
  obtains tree where
    created-tree table tree final-state
    rt = value tree
    merkle-root-binds-table rt table prefix-state  $\vee$ 
    hash-map-new-output-hit (set-tree tree) prefix-state final-state
proof -
  from bind obtain tree where
    created: created-tree table tree final-state
    and rt-eq: rt = value tree
    unfolding merkle-root-binds-table-def by blast
  have pullback:
    created-tree table tree prefix-state  $\vee$ 
    hash-map-new-output-hit (set-tree tree) prefix-state final-state
  using protocol-created-tree-pullback-or-new-output-hit
    [OF created[unfolded created-tree-def] ext]
  unfolding created-tree-def .
  have
    merkle-root-binds-table rt table prefix-state  $\vee$ 
    hash-map-new-output-hit (set-tree tree) prefix-state final-state
proof -
  from pullback show ?thesis
proof
  assume created-tree table tree prefix-state
  then have merkle-root-binds-table rt table prefix-state
  unfolding merkle-root-binds-table-def
  using rt-eq by blast
  then show ?thesis by simp
next
  assume hash-map-new-output-hit (set-tree tree) prefix-state final-state

```

```

    then show ?thesis by simp
  qed
qed
then show ?thesis
  by (rule that[OF created rt-eq])
qed

lemma merkle-root-binds-nonempty-table-late-root-output-preexisting-or-new-hit:
  fixes rt :: 'f
  assumes ext:  $s \leq t$ 
  and bind: merkle-root-binds-table rt table t
  and nonempty:  $\text{table} \neq []$ 
  shows
     $(\exists \text{input}. \text{fmlookup} (\text{HashMap } s) \text{ input} = \text{Some } rt) \vee$ 
     $\text{hash-map-new-output-hit } \{rt\} s t$ 
proof -
  from merkle-root-binds-nonempty-table-output-lookup[OF bind nonempty]
  obtain input where lookup-t:
     $\text{fmlookup} (\text{HashMap } t) \text{ input} = \text{Some } rt$ 
  by blast
  show ?thesis
proof (cases  $\text{fmlookup} (\text{HashMap } s) \text{ input}$ )
  case None
  then have hash-map-new-output-hit  $\{rt\} s t$ 
    unfolding hash-map-new-output-hit-def
    using lookup-t by blast
  then show ?thesis by simp
next
  case (Some old)
  have lookup-t-old:  $\text{fmlookup} (\text{HashMap } t) \text{ input} = \text{Some } old$ 
    by (rule hash-extension-lookup[OF Some ext])
  then have old-eq:  $old = rt$ 
    using lookup-t by simp
  then show ?thesis
    using Some by blast
qed
qed

lemma accepted-with-bound-tables-trace-root-late-output-preexisting-or-new-hit:
  assumes ext:  $\text{prefix-state} \leq \text{final-state}$ 
  and bound:
    accepted-with-bound-tables s (Some (result, final-state))
    trace-table composition-table as query-idxs
  obtains fr f-fri-roots f-final dg composition-fri-roots final rest
  where
    verifier-header-transcript s fr f-fri-roots f-final as dg
    composition-fri-roots final rest
    merkle-root-binds-table fr trace-table final-state
     $(\exists \text{input}. \text{fmlookup} (\text{HashMap } \text{prefix-state}) \text{ input} = \text{Some } fr) \vee$ 

```

$\text{hash-map-new-output-hit } \{fr\} \text{ prefix-state final-state}$
proof –
from bound **obtain** $fr \text{ f-fri-roots f-final dg composition-fri-roots final}$
 rest **where**
 header:
 $\text{verifier-header-transcript } s \text{ fr f-fri-roots f-final as dg}$
 $\text{composition-fri-roots final rest}$
and $\text{bind: merkle-root-binds-table fr trace-table final-state}$
unfolding $\text{accepted-with-bound-tables-def}$ **by** blast
have $\text{nonempty: trace-table} \neq []$
using $\text{accepted-with-bound-tables-shapes}(1)[OF \text{ bound}]$
 $\text{eval-domain-nontrivial}$ **by** auto
have late:
 $(\exists \text{ input. fmlookup (HashMap prefix-state) input = Some fr}) \vee$
 $\text{hash-map-new-output-hit } \{fr\} \text{ prefix-state final-state}$
by $(\text{rule}$
 $\text{merkle-root-binds-nonempty-table-late-root-output-preexisting-or-new-hit}$
 $[OF \text{ ext bind nonempty}])$
show $?thesis$
by $(\text{rule that}[OF \text{ header bind late}])$
qed

lemma $\text{accepted-with-bound-tables-composition-root-late-output-preexisting-or-new-hit:}$
assumes $\text{ext: prefix-state} \leq \text{final-state}$
and bound:
 $\text{accepted-with-bound-tables } s \text{ (Some (result, final-state))}$
 $\text{trace-table composition-table as query-idxs}$
obtains $fr \text{ f-fri-roots f-final dg composition-fri-roots final rest}$
where
 $\text{verifier-header-transcript } s \text{ fr f-fri-roots f-final as dg}$
 $\text{composition-fri-roots final rest}$
 $\text{composition-fri-roots} \neq []$
 $\text{merkle-root-binds-table (hd composition-fri-roots) composition-table}$
 final-state
 $(\exists \text{ input.}$
 $\text{fmlookup (HashMap prefix-state) input =}$
 $\text{Some (hd composition-fri-roots)}) \vee$
 $\text{hash-map-new-output-hit } \{\text{hd composition-fri-roots}\} \text{ prefix-state}$
 final-state
proof –
from bound **obtain** $fr \text{ f-fri-roots f-final dg composition-fri-roots final}$
 rest **where**
 header:
 $\text{verifier-header-transcript } s \text{ fr f-fri-roots f-final as dg}$
 $\text{composition-fri-roots final rest}$
and $\text{comp-nonempty: composition-fri-roots} \neq []$
and bind:
 $\text{merkle-root-binds-table (hd composition-fri-roots)}$
 $\text{composition-table final-state}$

unfolding *accepted-with-bound-tables-def* **by** *blast*
have *nonempty: composition-table* $\neq []$
using *accepted-with-bound-tables-shapes*(2)[*OF bound*]
eval-domain-nontrivial **by** *auto*
have *late:*
 $(\exists \text{ input.}$
 $\quad \text{fmlookup (HashMap prefix-state) input =}$
 $\quad \text{Some (hd composition-fri-roots))} \vee$
 $\quad \text{hash-map-new-output-hit \{hd composition-fri-roots\} prefix-state}$
 $\quad \text{final-state}$
by (rule
 $\quad \text{merkle-root-binds-nonempty-table-late-root-output-preexisting-or-new-hit}$
 $\quad \text{[OF ext bind nonempty]})$
show *?thesis*
by (rule that[*OF header comp-nonempty bind late*])
qed

lemma *accepted-with-bound-tables-initial-roots-late-output-preexisting-or-new-hit:*
assumes *ext: prefix-state* $\leq$ *final-state*
and *bound:*
 $\quad \text{accepted-with-bound-tables s (Some (result, final-state))}$
 $\quad \text{trace-table composition-table as query-idxs}$
obtains *fr f-fri-roots f-final dg composition-fri-roots final rest*
where
 $\quad \text{verifier-header-transcript s fr f-fri-roots f-final as dg}$
 $\quad \text{composition-fri-roots final rest}$
 $\quad \text{composition-fri-roots} \neq []$
 $\quad \text{merkle-root-binds-table fr trace-table final-state}$
 $\quad \text{merkle-root-binds-table (hd composition-fri-roots) composition-table}$
 $\quad \text{final-state}$
 $\quad (\exists \text{ input. fmlookup (HashMap prefix-state) input = Some fr}) \vee$
 $\quad \text{hash-map-new-output-hit \{fr\} prefix-state final-state}$
 $\quad (\exists \text{ input.}$
 $\quad \quad \text{fmlookup (HashMap prefix-state) input =}$
 $\quad \quad \text{Some (hd composition-fri-roots))} \vee$
 $\quad \quad \text{hash-map-new-output-hit \{hd composition-fri-roots\} prefix-state}$
 $\quad \quad \text{final-state}$
proof –
from *accepted-with-bound-tables-trace-root-late-output-preexisting-or-new-hit*
 $\quad \text{[OF ext bound]}$
obtain *fr f-fri-roots f-final dg composition-fri-roots final rest* **where**
header:
 $\quad \text{verifier-header-transcript s fr f-fri-roots f-final as dg}$
 $\quad \text{composition-fri-roots final rest}$
and *trace-bind:*
 $\quad \text{merkle-root-binds-table fr trace-table final-state}$
and *trace-late:*
 $\quad (\exists \text{ input. fmlookup (HashMap prefix-state) input = Some fr}) \vee$
 $\quad \text{hash-map-new-output-hit \{fr\} prefix-state final-state}$

by *blast*
from *accepted-with-bound-tables-composition-root-late-output-preexisting-or-new-hit*
[OF ext bound]
obtain *fr' f-fri-roots' f-final' dg' composition-fri-roots' final' rest'*
where *header'*:
verifier-header-transcript s fr' f-fri-roots' f-final' as dg'
composition-fri-roots' final' rest'
and *comp-nonempty'*: *composition-fri-roots' ≠ []*
and *comp-bind'*:
merkle-root-binds-table (hd composition-fri-roots')
composition-table final-state
and *comp-late'*:
(∃ input.
fmlookup (HashMap prefix-state) input =
Some (hd composition-fri-roots')) ∨
hash-map-new-output-hit {hd composition-fri-roots'} prefix-state
final-state
by *blast*
have *eqs*:
fr' = fr ∧
f-fri-roots' = f-fri-roots ∧
f-final' = f-final ∧
dg' = dg ∧
composition-fri-roots' = composition-fri-roots ∧
final' = final ∧
rest' = rest
using *verifier-header-transcript-unique[OF header header'] by simp*
show *?thesis*
by *(rule that[OF header])*
(use comp-nonempty' comp-bind' comp-late' trace-bind trace-late eqs
in simp-all)
qed

lemma *accepted-with-bound-tables-initial-roots-late-output-preexisting-or-union-hit*:
assumes *ext: prefix-state ≤ final-state*
and *bound*:
accepted-with-bound-tables s (Some (result, final-state))
trace-table composition-table as query-idxs
obtains *fr f-fri-roots f-final dg composition-fri-roots final rest*
where
verifier-header-transcript s fr f-fri-roots f-final as dg
composition-fri-roots final rest
composition-fri-roots ≠ []
merkle-root-binds-table fr trace-table final-state
merkle-root-binds-table (hd composition-fri-roots) composition-table
final-state
((∃ input. fmlookup (HashMap prefix-state) input = Some fr) ∧
(∃ input.
fmlookup (HashMap prefix-state) input =

```

    Some (hd composition-fri-roots))) ∨
    hash-map-new-output-hit {fr, hd composition-fri-roots} prefix-state
    final-state
proof –
from accepted-with-bound-tables-initial-roots-late-output-preexisting-or-new-hit
    [OF ext bound]
obtain fr f-fri-roots f-final dg composition-fri-roots final rest where
    header:
    verifier-header-transcript s fr f-fri-roots f-final as dg
    composition-fri-roots final rest
and comp-nonempty: composition-fri-roots ≠ []
and trace-bind:
    merkle-root-binds-table fr trace-table final-state
and comp-bind:
    merkle-root-binds-table (hd composition-fri-roots)
    composition-table final-state
and trace-late:
    (∃ input. fmlookup (HashMap prefix-state) input = Some fr) ∨
    hash-map-new-output-hit {fr} prefix-state final-state
and comp-late:
    (∃ input.
    fmlookup (HashMap prefix-state) input =
    Some (hd composition-fri-roots)) ∨
    hash-map-new-output-hit {hd composition-fri-roots} prefix-state
    final-state
by blast
have late-union:
    ((∃ input. fmlookup (HashMap prefix-state) input = Some fr) ∧
    (∃ input.
    fmlookup (HashMap prefix-state) input =
    Some (hd composition-fri-roots))) ∨
    hash-map-new-output-hit {fr, hd composition-fri-roots} prefix-state
    final-state
proof (cases
    (∃ input. fmlookup (HashMap prefix-state) input = Some fr) ∧
    (∃ input.
    fmlookup (HashMap prefix-state) input =
    Some (hd composition-fri-roots)))
case True
then show ?thesis by simp
next
case False
then have
    hash-map-new-output-hit {fr} prefix-state final-state ∨
    hash-map-new-output-hit {hd composition-fri-roots} prefix-state
    final-state
using trace-late comp-late by blast
then show ?thesis
proof

```

```

assume hash-map-new-output-hit {fr} prefix-state final-state
then have
  hash-map-new-output-hit {fr, hd composition-fri-roots}
    prefix-state final-state
using hash-map-new-output-hit-subset[of {fr}
  {fr, hd composition-fri-roots} prefix-state final-state]
by auto
then show ?thesis by simp
next
assume
  hash-map-new-output-hit {hd composition-fri-roots} prefix-state
    final-state
then have
  hash-map-new-output-hit {fr, hd composition-fri-roots}
    prefix-state final-state
using hash-map-new-output-hit-subset
  [of {hd composition-fri-roots}
    {fr, hd composition-fri-roots} prefix-state final-state]
by auto
then show ?thesis by simp
qed
qed
show ?thesis
by (rule that[OF header comp-nonempty trace-bind comp-bind late-union])
qed

lemma accepted-with-bound-tables-initial-roots-pullback-or-new-tree-output-hit:
assumes ext: prefix-state  $\leq$  final-state
and bound:
  accepted-with-bound-tables s (Some (result, final-state))
  trace-table composition-table as query-idxs
obtains fr f-fri-roots f-final dg composition-fri-roots final rest
  trace-tree composition-tree
where
  verifier-header-transcript s fr f-fri-roots f-final as dg
    composition-fri-roots final rest
  composition-fri-roots  $\neq \square$ 
  created-tree trace-table trace-tree final-state
  fr = value trace-tree
  created-tree composition-table composition-tree final-state
  hd composition-fri-roots = value composition-tree
  (merkle-root-binds-table fr trace-table prefix-state  $\wedge$ 
    merkle-root-binds-table (hd composition-fri-roots) composition-table
    prefix-state)  $\vee$ 
  hash-map-new-output-hit
    (set-tree trace-tree  $\cup$  set-tree composition-tree)
    prefix-state final-state
proof —
from bound obtain fr f-fri-roots f-final dg composition-fri-roots final

```

```

    rest where
header:
    verifier-header-transcript s fr f-fri-roots f-final as dg
    composition-fri-roots final rest
and comp-nonempty: composition-fri-roots  $\neq$  []
and trace-bind:
    merkle-root-binds-table fr trace-table final-state
and comp-bind:
    merkle-root-binds-table (hd composition-fri-roots)
    composition-table final-state
unfolding accepted-with-bound-tables-def by blast
from merkle-root-binds-table-pullback-or-new-tree-output-hit
    [OF ext trace-bind]
obtain trace-tree where
    trace-created: created-tree trace-table trace-tree final-state
and trace-root: fr = value trace-tree
and trace-split:
    merkle-root-binds-table fr trace-table prefix-state  $\vee$ 
    hash-map-new-output-hit (set-tree trace-tree) prefix-state
    final-state
by blast
from merkle-root-binds-table-pullback-or-new-tree-output-hit
    [OF ext comp-bind]
obtain composition-tree where
    comp-created:
        created-tree composition-table composition-tree final-state
and comp-root:
    hd composition-fri-roots = value composition-tree
and comp-split:
    merkle-root-binds-table (hd composition-fri-roots)
    composition-table prefix-state  $\vee$ 
    hash-map-new-output-hit (set-tree composition-tree) prefix-state
    final-state
by blast
have split:
    (merkle-root-binds-table fr trace-table prefix-state  $\wedge$ 
    merkle-root-binds-table (hd composition-fri-roots) composition-table
    prefix-state)  $\vee$ 
    hash-map-new-output-hit
    (set-tree trace-tree  $\cup$  set-tree composition-tree)
    prefix-state final-state
proof -
from trace-split show ?thesis
proof
    assume trace-prefix:
        merkle-root-binds-table fr trace-table prefix-state
from comp-split show ?thesis
proof
    assume comp-prefix:

```

```

    merkle-root-binds-table (hd composition-fri-roots)
    composition-table prefix-state
then show ?thesis
    using trace-prefix by simp
next
    assume comp-hit:
        hash-map-new-output-hit (set-tree composition-tree)
        prefix-state final-state
    have hash-map-new-output-hit
        (set-tree trace-tree  $\cup$  set-tree composition-tree)
        prefix-state final-state
    by (rule hash-map-new-output-hit-subset[OF - comp-hit])
        simp
    then show ?thesis by simp
qed
next
    assume trace-hit:
        hash-map-new-output-hit (set-tree trace-tree) prefix-state
        final-state
    have hash-map-new-output-hit
        (set-tree trace-tree  $\cup$  set-tree composition-tree)
        prefix-state final-state
    by (rule hash-map-new-output-hit-subset[OF - trace-hit])
        simp
    then show ?thesis by simp
qed
qed
show ?thesis
    by (rule that[OF header comp-nonempty trace-created trace-root
        comp-created comp-root split])
qed

lemma hash-map-new-output-hit-after-update:
    hash-map-new-output-hit B s
    (s( $\downarrow$ HashMap := fmupd x y (HashMap s)))  $\longleftrightarrow$ 
    fmlookup (HashMap s) x = None  $\wedge$  y  $\in$  B
proof
    assume hit:
        hash-map-new-output-hit B s
        (s( $\downarrow$ HashMap := fmupd x y (HashMap s)))
    then obtain k z where fresh:
        fmlookup (HashMap s) k = None
    and lookup:
        fmlookup (fmupd x y (HashMap s)) k = Some z
    and z-in: z  $\in$  B
    unfolding hash-map-new-output-hit-def by auto
    have k = x
    proof (rule ccontr)
        assume k  $\neq$  x

```

```

    then have fmlookup (HashMap s) k = Some z
      using lookup by simp
    then show False
      using fresh by simp
  qed
  then show fmlookup (HashMap s) x = None  $\wedge$   $y \in B$ 
    using fresh lookup z-in by simp
next
  assume fmlookup (HashMap s) x = None  $\wedge$   $y \in B$ 
  then show
    hash-map-new-output-hit B s
    (s(HashMap := fmupd x y (HashMap s)))
    unfolding hash-map-new-output-hit-def by auto
  qed

```

lemma *hash-extension-none*:

```

  assumes ext:  $s \leq t$ 
    and none: fmlookup (HashMap t) x = None
  shows fmlookup (HashMap s) x = None
proof (cases fmlookup (HashMap s) x)
  case None
    then show ?thesis .
next
  case (Some y)
  have fmlookup (HashMap t) x = Some y
    by (rule hash-extension-lookup[OF Some ext])
  then show ?thesis
    using none by simp
qed

```

lemma *hash-map-new-output-hit-extend-initial*:

```

  assumes ext:  $s \leq t$ 
    and hit: hash-map-new-output-hit B t u
  shows hash-map-new-output-hit B s u
proof –
  from hit obtain x y where fresh-t:
    fmlookup (HashMap t) x = None
    and lookup-u: fmlookup (HashMap u) x = Some y
    and y-in:  $y \in B$ 
    unfolding hash-map-new-output-hit-def by blast
  have fresh-s: fmlookup (HashMap s) x = None
    by (rule hash-extension-none[OF ext fresh-t])
  show ?thesis
    unfolding hash-map-new-output-hit-def
    using fresh-s lookup-u y-in by blast
qed

```

lemma *hash-map-new-output-hit-trans-decomp*:

```

  assumes ext-st:  $s \leq t$ 

```

```

    and ext-tu:  $t \leq u$ 
    and hit: hash-map-new-output-hit  $B\ s\ u$ 
  shows
    hash-map-new-output-hit  $B\ s\ t \vee$ 
    hash-map-new-output-hit  $B\ t\ u$ 
  proof -
    from hit obtain  $x\ y$  where fresh-s:
      fmlookup (HashMap  $s$ )  $x = \text{None}$ 
    and lookup-u: fmlookup (HashMap  $u$ )  $x = \text{Some } y$ 
    and y-in:  $y \in B$ 
    unfolding hash-map-new-output-hit-def by blast
  show ?thesis
  proof (cases fmlookup (HashMap  $t$ )  $x$ )
    case None
    then have hash-map-new-output-hit  $B\ t\ u$ 
      unfolding hash-map-new-output-hit-def
      using lookup-u y-in by blast
    then show ?thesis by simp
  next
    case (Some  $z$ )
    have lookup-u-z: fmlookup (HashMap  $u$ )  $x = \text{Some } z$ 
      by (rule hash-extension-lookup[OF Some ext-tu])
    have  $z = y$ 
      using lookup-u lookup-u-z by simp
    then have hash-map-new-output-hit  $B\ s\ t$ 
      unfolding hash-map-new-output-hit-def
      using fresh-s Some y-in by blast
    then show ?thesis by simp
  qed
qed

definition hash-extension-preserving
  :: ('r, ('f, 'a) protocol-channel-scheme) state-monad  $\Rightarrow$  bool
  where
    hash-extension-preserving  $m \longleftrightarrow$ 
      ( $\forall s\ x\ t.$ 
        Some  $(x, t) \in \text{set-dist } (\text{execute } m\ s) \longrightarrow s \leq t$ )

definition hash-map-preserving
  :: ('r, ('f, 'a) protocol-channel-scheme) state-monad  $\Rightarrow$  bool
  where
    hash-map-preserving  $m \longleftrightarrow$ 
      ( $\forall s\ x\ t.$ 
        Some  $(x, t) \in \text{set-dist } (\text{execute } m\ s) \longrightarrow$ 
        HashMap  $t = \text{HashMap } s$ )

lemma hash-map-preserving-imp-hash-extension-preserving:
  assumes hash-map-preserving  $m$ 
  shows hash-extension-preserving  $m$ 

```

```

unfolding hash-extension-preserving-def
proof (intro allI impI)
  fix s x t
  assume outcome: Some (x, t) ∈ set-dist (execute m s)
  have HashMap t = HashMap s
    using assms outcome unfolding hash-map-preserving-def by blast
  then show s ≤ t
    unfolding less-eq-hash-ext-def less-eq-fmap-def by simp
qed

lemma hash-extension-preserving-return:
  hash-extension-preserving (return x)
  unfolding hash-extension-preserving-def
  by (intro allI impI) (simp add: hash-ext-refl)

lemma hash-extension-preserving-hash:
  fixes x :: 'f protocol-hash-input
  shows
    hash-extension-preserving
      (hash x :: ('f, ('f, 'a) protocol-channel-scheme) state-monad)
  unfolding hash-extension-preserving-def
  by (intro allI impI) (rule hash-outcome(1))

lemma hash-extension-preserving-bind:
  fixes m :: ('x, ('f, 'a) protocol-channel-scheme) state-monad
  and k :: 'x ⇒ ('y, ('f, 'a) protocol-channel-scheme) state-monad
  assumes m-ext: hash-extension-preserving m
  and k-ext:  $\bigwedge x. \text{hash-extension-preserving } (k\ x)$ 
  shows hash-extension-preserving (m  $\gg$  k)
  unfolding hash-extension-preserving-def
proof (intro allI impI)
  fix s y u
  assume outcome: Some (y, u) ∈ set-dist (execute (m  $\gg$  k) s)
  from outcome obtain x t where m-out:
    Some (x, t) ∈ set-dist (execute m s)
  and k-out: Some (y, u) ∈ set-dist (execute (k x) t)
  by (auto elim!: set-dist-bindE)
  have ext-st: s ≤ t
    using m-ext m-out unfolding hash-extension-preserving-def by blast
  have ext-tu: t ≤ u
    using k-ext[of x] k-out unfolding hash-extension-preserving-def by blast
  show s ≤ u
    by (rule hash-ext-trans[OF ext-st ext-tu])
qed

lemma wp-hash-new-output-hit:
  fixes x :: 'f protocol-hash-input
  and s :: ('f, 'a) protocol-channel-scheme
  shows

```



```

wp-event (hash x) (hash-new-output-hit-event B s) s =
  (if fmlookup (HashMap s) x = None
    then nnreal (card B) / nnreal size
    else 0)
proof (cases fmlookup (HashMap s) x)
case None
have indicator:
  (λy. if hash-map-new-output-hit B s
    (s(HashMap := fmupd x y (HashMap s)))
    then 1 else 0 :: prob) =
  (λy. if y ∈ B then 1 else 0)
by (rule ext)
  (simp add: hash-map-new-output-hit-after-update None)
have wp-event (hash x) (hash-new-output-hit-event B s) s =
  dist-expect (hash-dist x s) (λy. if y ∈ B then 1 else 0)
unfolding wp-event-def hash-new-output-hit-event-def
by (simp add: wp-hash indicator)
also have ... = nnreal (card B) / nnreal size
by (rule dist-expect-hash-dist-fresh-indicator[OF None])
finally show ?thesis
  using None by simp
next
case (Some y)
have no-hit:
  ¬ hash-map-new-output-hit B s
  (s(HashMap := fmupd x y (HashMap s)))
by (simp add: hash-map-new-output-hit-after-update Some)
show ?thesis
unfolding wp-event-def hash-new-output-hit-event-def
by (simp add: wp-hash Some hash-dist-def option-default-dist-def no-hit)
qed

lemma wp-hash-new-output-hit-bound:
fixes x :: 'f protocol-hash-input
and s :: ('f, 'a) protocol-channel-scheme
shows
  wp-event (hash x) (hash-new-output-hit-event B s) s ≤
  nnreal (card B) / nnreal size
unfolding wp-hash-new-output-hit
by simp

definition hash-target-budget-value
  :: 'f set ⇒ nat ⇒ prob
where
  hash-target-budget-value B n =
  nnreal (n * card B) / nnreal size

definition hash-target-budget
  :: 'f set ⇒ nat ⇒

```

```

    ('r, ('f, 'a) protocol-channel-scheme) state-monad  $\Rightarrow$  bool
where
  hash-target-budget  $B\ n\ m \longleftrightarrow$ 
    ( $\forall\ s.$ 
      wp-event  $m\ (\text{hash-new-output-hit-event } B\ s)\ s \leq$ 
        hash-target-budget-value  $B\ n$ )

lemma hash-target-budget-value-add:
  hash-target-budget-value  $B\ n + \text{hash-target-budget-value } B\ m =$ 
    hash-target-budget-value  $B\ (n + m)$ 
unfolding hash-target-budget-value-def
by (simp add: add-divide-nnreal algebra-simps)

lemma hash-new-output-hit-event-step-bound:
  fixes  $m :: ('x, ('f, 'a) \text{protocol-channel-scheme}) \text{state-monad}$ 
  assumes ext-st:  $s \leq t$ 
  and ext-m: hash-extension-preserving  $m$ 
  shows
    wp-event  $m\ (\text{hash-new-output-hit-event } B\ s)\ t \leq$ 
      (if hash-map-new-output-hit  $B\ s\ t$  then 1
        else wp-event  $m\ (\text{hash-new-output-hit-event } B\ t)\ t)$ 
proof (cases hash-map-new-output-hit  $B\ s\ t$ )
  case True
    then show ?thesis
      using wp-event-le-1 [of  $m\ \text{hash-new-output-hit-event } B\ s\ t$ ]
      by simp
  next
    case False
    have mono:
      wp-event  $m\ (\text{hash-new-output-hit-event } B\ s)\ t \leq$ 
        wp-event  $m\ (\text{hash-new-output-hit-event } B\ t)\ t$ 
    proof (rule wp-event-mono-on-support)
      fix out
      assume support:  $out \in \text{set-dist } (\text{execute } m\ t)$ 
      and hit-s: hash-new-output-hit-event  $B\ s\ out$ 
      show hash-new-output-hit-event  $B\ t\ out$ 
      proof (cases out)
        case None
          then show ?thesis
            using hit-s unfolding hash-new-output-hit-event-def by simp
        case Some  $xu$ 
          then obtain  $x\ u$  where  $xu = (x, u)$ 
          by (cases  $xu$ ) simp
          have out:  $\text{Some } (x, u) \in \text{set-dist } (\text{execute } m\ t)$ 
          using support  $\text{Some } xu$  by simp
          have ext-tu:  $t \leq u$ 
          using ext-m out unfolding hash-extension-preserving-def by blast
          have hit-su: hash-map-new-output-hit  $B\ s\ u$ 

```

```

    using hit-s Some xu unfolding hash-new-output-hit-event-def by simp
  have hash-map-new-output-hit B s t  $\vee$ 
    hash-map-new-output-hit B t u
  by (rule hash-map-new-output-hit-trans-decomp
      [OF ext-st ext-tu hit-su])
  then have hash-map-new-output-hit B t u
    using False by blast
  then show ?thesis
    using Some xu unfolding hash-new-output-hit-event-def by simp
qed
qed
show ?thesis
  using False mono by simp
qed

```

```

lemma hash-map-new-output-hit-refl[simp]:
   $\neg$  hash-map-new-output-hit B s s
  unfolding hash-map-new-output-hit-def by auto

```

```

lemma hash-target-budget-return:
  hash-target-budget B 0 (return x)
  unfolding hash-target-budget-def hash-target-budget-value-def
    hash-new-output-hit-event-def
  by (simp add: wp-event-def wpsimps)

```

```

lemma hash-map-preserving-no-new-output-hit-on-support:
  assumes preserving: hash-map-preserving m
    and support: out  $\in$  set-dist (execute m s)
  shows  $\neg$  hash-new-output-hit-event B s out
proof (cases out)
  case None
  then show ?thesis
    unfolding hash-new-output-hit-event-def by simp
next
  case (Some xt)
  then obtain x t where xt: xt = (x, t)
    by (cases xt) simp
  have out: Some (x, t)  $\in$  set-dist (execute m s)
    using support Some xt by simp
  have map-eq: HashMap t = HashMap s
    using preserving out unfolding hash-map-preserving-def by blast
  have  $\neg$  hash-map-new-output-hit B s t
    using map-eq unfolding hash-map-new-output-hit-def by auto
  then show ?thesis
    using Some xt unfolding hash-new-output-hit-event-def by simp
qed

```

```

lemma hash-map-preserving-imp-hash-target-budget-zero:
  assumes preserving: hash-map-preserving m

```

shows *hash-target-budget* $B\ 0\ m$
unfolding *hash-target-budget-def* *hash-target-budget-value-def*
apply (*intro allI*)
apply (*rule order-trans* [
 where $y = wp_event\ m\ (\lambda\cdot. False)\ -]$)
 apply (*rule wp-event-mono-on-support*)
 apply (*meson hash-map-preserving-no-new-output-hit-on-support preserving*)
unfolding *wp-event-def* *wp-def* *dist-expect-def*
by *simp*

lemma *hash-target-budget-hash*:
fixes $x :: 'f\ protocol_hash_input$
shows
 hash-target-budget $B\ 1$
 $(hash\ x :: ('f, ('f, 'a)\ protocol_channel_scheme)\ state_monad)$
unfolding *hash-target-budget-def*
proof (*intro allI*)
fix $s :: ('f, 'a)\ protocol_channel_scheme$
show
 $wp_event\ (hash\ x)\ (hash_new_output_hit_event\ B\ s)\ s \leq$
 hash-target-budget-value $B\ 1$
 using *wp-hash-new-output-hit-bound* [**where** $x=x$ **and** $s=s$ **and** $B=B$]
 unfolding *hash-target-budget-value-def* **by** *simp*
qed

lemma *hash-target-budget-bind*:
fixes $m :: ('x, ('f, 'a)\ protocol_channel_scheme)\ state_monad$
 and $k :: 'x \Rightarrow ('y, ('f, 'a)\ protocol_channel_scheme)\ state_monad$
assumes *m-ext*: *hash-extension-preserving* m
 and *m-budget*: *hash-target-budget* $B\ n\ m$
 and *k-ext*: $\bigwedge x. hash_extension_preserving\ (k\ x)$
 and *k-budget*: $\bigwedge x. hash_target_budget\ B\ n'\ (k\ x)$
shows *hash-target-budget* $B\ (n + n')\ (m \gg k)$
unfolding *hash-target-budget-def*
proof (*intro allI*)
fix $s :: ('f, 'a)\ protocol_channel_scheme$
let $?E = hash_new_output_hit_event\ B\ s$
let $?Tail =$
 $\lambda out. case\ out\ of$
 $None \Rightarrow 0$
 | $Some\ (x, t) \Rightarrow$
 $if\ hash_map_new_output_hit\ B\ s\ t\ then\ 0$
 $else\ wp_event\ (k\ x)\ (hash_new_output_hit_event\ B\ t)\ t$
let $?R = \lambda out. (if\ ?E\ out\ then\ 1\ else\ 0) + ?Tail\ out$
have *split*:
 $wp_event\ (m \gg k)\ ?E\ s \leq wp\ m\ ?R\ s$
proof –
 have *exact*:
 $wp_event\ (m \gg k)\ ?E\ s =$

```

wp m
  (λout. case out of
    None ⇒ 0
    | Some (x, t) ⇒ wp-event (k x) ?E t) s
unfolding wp-event-def hash-new-output-hit-event-def
by (simp add: wpsimps)
have wp m
  (λout. case out of
    None ⇒ 0
    | Some (x, t) ⇒ wp-event (k x) ?E t) s
  ≤ wp m ?R s
proof (rule wp-mono-on-support)
fix out
assume support: out ∈ set-dist (execute m s)
show (case out of
  None ⇒ 0
  | Some (x, t) ⇒ wp-event (k x) ?E t) ≤ ?R out
proof (cases out)
case None
then show ?thesis
  unfolding hash-new-output-hit-event-def by simp
next
case (Some xt)
then obtain x t where xt: xt = (x, t)
  by (cases xt) simp
have m-out: Some (x, t) ∈ set-dist (execute m s)
  using support Some xt by simp
have ext-st: s ≤ t
  using m-ext m-out unfolding hash-extension-preserving-def by blast
have step:
  wp-event (k x) ?E t ≤
    (if hash-map-new-output-hit B s t then 1
     else wp-event (k x) (hash-new-output-hit-event B t) t)
  by (rule hash-new-output-hit-event-step-bound
    [OF ext-st k-ext[of x]])
show ?thesis
proof (cases hash-map-new-output-hit B s t)
case True
then show ?thesis
  using Some xt step
  unfolding hash-new-output-hit-event-def by simp
next
case False
then show ?thesis
  using Some xt step
  unfolding hash-new-output-hit-event-def by simp
qed
qed
qed

```

```

    then show ?thesis
      unfolding exact .
qed
have head-bound:
  wp-event m ?E s ≤ hash-target-budget-value B n
  using m-budget unfolding hash-target-budget-def by blast
have tail-bound:
  wp m ?Tail s ≤ hash-target-budget-value B n'
proof (rule wp-le-const-on-support)
  fix out
  assume support: out ∈ set-dist (execute m s)
  show ?Tail out ≤ hash-target-budget-value B n'
  proof (cases out)
    case None
    then show ?thesis
      unfolding hash-target-budget-value-def by simp
  next
    case (Some xt)
    then obtain x t where xt: xt = (x, t)
      by (cases xt) simp
    have tail:
      wp-event (k x) (hash-new-output-hit-event B t) t ≤
        hash-target-budget-value B n'
      using k-budget[of x] unfolding hash-target-budget-def by blast
    show ?thesis
      using Some xt tail by simp
  qed
qed
have wp-event (m ≫ k) ?E s ≤ wp m ?R s
  by (rule split)
also have ... = wp-event m ?E s + wp m ?Tail s
  unfolding wp-event-def wp-def dist-expect-def
  by (simp add: sum.distrib algebra-simps
      hash-new-output-hit-event-def)
also have ... ≤
  hash-target-budget-value B n + hash-target-budget-value B n'
  by (intro add-mono head-bound tail-bound)
also have ... = hash-target-budget-value B (n + n')
  by (rule hash-target-budget-value-add)
finally show
  wp-event (m ≫ k) (hash-new-output-hit-event B s) s ≤
    hash-target-budget-value B (n + n') .
qed

```

```

lemma hash-target-budget-value-mono:
  assumes m ≤ n
  shows hash-target-budget-value B m ≤ hash-target-budget-value B n
  unfolding hash-target-budget-value-def
  apply (subst nn2real-le-iff[symmetric])

```

```

using assms by (simp add: divide-right-mono mult-right-mono)

lemma hash-target-budget-mono:
  assumes  $m \leq n$ 
  and budget: hash-target-budget  $B$   $m$   $p$ 
  shows hash-target-budget  $B$   $n$   $p$ 
  unfolding hash-target-budget-def
proof (intro allI)
  fix  $s$ 
  have wp-event  $p$  (hash-new-output-hit-event  $B$   $s$ )  $s \leq$ 
    hash-target-budget-value  $B$   $m$ 
  using budget unfolding hash-target-budget-def by blast
  also have  $\dots \leq$  hash-target-budget-value  $B$   $n$ 
  by (rule hash-target-budget-value-mono[OF assms(1)])
  finally show
    wp-event  $p$  (hash-new-output-hit-event  $B$   $s$ )  $s \leq$ 
      hash-target-budget-value  $B$   $n$  .
qed

definition hash-target-program
  :: 'f set  $\Rightarrow$  nat  $\Rightarrow$ 
    ('r, ('f, 'a) protocol-channel-scheme) state-monad  $\Rightarrow$  bool
where
  hash-target-program  $B$   $n$   $m \longleftrightarrow$ 
    hash-extension-preserving  $m \wedge$  hash-target-budget  $B$   $n$   $m$ 

lemma hash-target-program-extension:
  assumes hash-target-program  $B$   $n$   $m$ 
  shows hash-extension-preserving  $m$ 
  using assms unfolding hash-target-program-def by simp

lemma hash-target-program-budget:
  assumes hash-target-program  $B$   $n$   $m$ 
  shows hash-target-budget  $B$   $n$   $m$ 
  using assms unfolding hash-target-program-def by simp

lemma hash-target-program-mono:
  assumes  $m \leq n$ 
  and program: hash-target-program  $B$   $m$   $p$ 
  shows hash-target-program  $B$   $n$   $p$ 
  unfolding hash-target-program-def
proof
  show hash-extension-preserving  $p$ 
  using program unfolding hash-target-program-def by simp
  have hash-target-budget  $B$   $m$   $p$ 
  using program unfolding hash-target-program-def by simp
  then show hash-target-budget  $B$   $n$   $p$ 
  by (rule hash-target-budget-mono[OF assms(1)])
qed

```

lemma *hash-target-program-return*:
hash-target-program B 0 (return x)
unfolding *hash-target-program-def*
by (intro conjI *hash-extension-preserving-return* *hash-target-budget-return*)

lemma *hash-target-program-hash*:
fixes $x :: 'f$ *protocol-hash-input*
shows
hash-target-program B 1
 $(\text{hash } x :: ('f, ('f, 'a) \text{ protocol-channel-scheme}) \text{ state-monad})$
unfolding *hash-target-program-def*
by (intro conjI *hash-extension-preserving-hash* *hash-target-budget-hash*)

lemma *hash-map-preserving-imp-hash-target-program-zero*:
assumes *hash-map-preserving* m
shows *hash-target-program* B 0 m
unfolding *hash-target-program-def*
using *assms* *hash-map-preserving-imp-hash-extension-preserving*
hash-map-preserving-imp-hash-target-budget-zero
by *blast*

lemma *hash-target-program-bind*:
fixes $m :: ('x, ('f, 'a) \text{ protocol-channel-scheme}) \text{ state-monad}$
and $k :: 'x \Rightarrow ('y, ('f, 'a) \text{ protocol-channel-scheme}) \text{ state-monad}$
assumes m : *hash-target-program* B n m
and k : $\bigwedge x. \text{hash-target-program } B \ n' \ (k \ x)$
shows *hash-target-program* B $(n + n')$ $(m \ggg k)$
unfolding *hash-target-program-def*
proof (intro conjI)
show *hash-extension-preserving* $(m \ggg k)$
by (rule *hash-extension-preserving-bind*)
 $(\text{use } m \ k \text{ in } \langle \text{simp-all add: hash-target-program-def} \rangle)$
show *hash-target-budget* B $(n + n')$ $(m \ggg k)$
by (rule *hash-target-budget-bind*)
 $(\text{use } m \ k \text{ in } \langle \text{simp-all add: hash-target-program-def} \rangle)$
qed

lemma *hash-extension-preserving-bind-on-outcomes*:
fixes $m :: ('x, ('f, 'a) \text{ protocol-channel-scheme}) \text{ state-monad}$
and $k :: 'x \Rightarrow ('y, ('f, 'a) \text{ protocol-channel-scheme}) \text{ state-monad}$
assumes $m\text{-ext}$: *hash-extension-preserving* m
and $k\text{-ext}$:
 $\bigwedge s \ x \ t. \text{Some } (x, t) \in \text{set-dist } (\text{execute } m \ s) \implies$
hash-extension-preserving $(k \ x)$
shows *hash-extension-preserving* $(m \ggg k)$
unfolding *hash-extension-preserving-def*
proof (intro allI impI)
fix $s \ y \ u$

assume *outcome*: $\text{Some } (y, u) \in \text{set-dist } (\text{execute } (m \ggg k) s)$
from *outcome* **obtain** $x\ t$ **where** *m-out*:
 $\text{Some } (x, t) \in \text{set-dist } (\text{execute } m\ s)$
 and *k-out*: $\text{Some } (y, u) \in \text{set-dist } (\text{execute } (k\ x)\ t)$
 by (*auto elim!*: *set-dist-bindE*)
have *ext-st*: $s \leq t$
 using *m-ext m-out* **unfolding** *hash-extension-preserving-def* **by** *blast*
have *ext-tu*: $t \leq u$
 using *k-ext*[*OF m-out*] *k-out*
 unfolding *hash-extension-preserving-def* **by** *blast*
show $s \leq u$
 by (*rule hash-ext-trans*[*OF ext-st ext-tu*])
qed

lemma *hash-target-budget-bind-on-outcomes*:
fixes $m :: ('x, ('f, 'a) \text{ protocol-channel-scheme}) \text{ state-monad}$
and $k :: 'x \Rightarrow ('y, ('f, 'a) \text{ protocol-channel-scheme}) \text{ state-monad}$
assumes *m-ext*: *hash-extension-preserving m*
and *m-budget*: *hash-target-budget B n m*
and *k-ext*:
 $\bigwedge s\ x\ t. \text{Some } (x, t) \in \text{set-dist } (\text{execute } m\ s) \implies$
 hash-extension-preserving (k x)
and *k-budget*:
 $\bigwedge s\ x\ t. \text{Some } (x, t) \in \text{set-dist } (\text{execute } m\ s) \implies$
 hash-target-budget B n' (k x)
shows *hash-target-budget B (n + n') (m \ggg k)*
unfolding *hash-target-budget-def*
proof (*intro allI*)
fix $s :: ('f, 'a) \text{ protocol-channel-scheme}$
let $?E = \text{hash-new-output-hit-event } B\ s$
let $?Tail =$
 $\lambda out. \text{case out of}$
 $\text{None} \Rightarrow 0$
 $| \text{Some } (x, t) \Rightarrow$
 $\text{if hash-map-new-output-hit } B\ s\ t \text{ then } 0$
 $\text{else wp-event } (k\ x) (\text{hash-new-output-hit-event } B\ t)\ t$
let $?R = \lambda out. (\text{if } ?E\ out \text{ then } 1 \text{ else } 0) + ?Tail\ out$
have *split*:
 $\text{wp-event } (m \ggg k) ?E\ s \leq \text{wp } m\ ?R\ s$
proof –
have *exact*:
 $\text{wp-event } (m \ggg k) ?E\ s =$
 $\text{wp } m$
 $(\lambda out. \text{case out of}$
 $\text{None} \Rightarrow 0$
 $| \text{Some } (x, t) \Rightarrow \text{wp-event } (k\ x) ?E\ t)\ s$
unfolding *wp-event-def hash-new-output-hit-event-def*
by (*simp add: wpsimps*)
have *wp m*

```

    ( $\lambda out.$  case out of
      None  $\Rightarrow 0$ 
      | Some (x, t)  $\Rightarrow wp\text{-event } (k\ x) \ ?E\ t) \ s$ 
     $\leq wp\ m \ ?R\ s$ 
  proof (rule wp-mono-on-support)
    fix out
    assume support: out  $\in set\text{-dist } (execute\ m\ s)$ 
    show (case out of
      None  $\Rightarrow 0$ 
      | Some (x, t)  $\Rightarrow wp\text{-event } (k\ x) \ ?E\ t) \leq ?R\ out$ 
    proof (cases out)
      case None
      then show ?thesis
        unfolding hash-new-output-hit-event-def by simp
    next
      case (Some xt)
      then obtain x t where xt: xt = (x, t)
        by (cases xt) simp
      have m-out: Some (x, t)  $\in set\text{-dist } (execute\ m\ s)$ 
        using support Some xt by simp
      have ext-st: s  $\leq t$ 
        using m-ext m-out unfolding hash-extension-preserving-def by blast
      have step:
        wp-event (k x) ?E t  $\leq$ 
        (if hash-map-new-output-hit B s t then 1
         else wp-event (k x) (hash-new-output-hit-event B t) t)
        by (rule hash-new-output-hit-event-step-bound
          [OF ext-st k-ext[OF m-out]])
      show ?thesis
        using Some xt step
        unfolding hash-new-output-hit-event-def
        by (cases hash-map-new-output-hit B s t) simp-all
    qed
  qed
  then show ?thesis
    unfolding exact .
qed
have head-bound:
  wp-event m ?E s  $\leq hash\text{-target-budget-value } B\ n$ 
  using m-budget unfolding hash-target-budget-def by blast
have tail-bound:
  wp m ?Tail s  $\leq hash\text{-target-budget-value } B\ n'$ 
proof (rule wp-le-const-on-support)
  fix out
  assume support: out  $\in set\text{-dist } (execute\ m\ s)$ 
  show ?Tail out  $\leq hash\text{-target-budget-value } B\ n'$ 
  proof (cases out)
    case None
    then show ?thesis

```

```

    unfolding hash-target-budget-value-def by simp
next
  case (Some xt)
  then obtain x t where xt: xt = (x, t)
    by (cases xt) simp
  have m-out: Some (x, t) ∈ set-dist (execute m s)
    using support Some xt by simp
  have tail:
    wp-event (k x) (hash-new-output-hit-event B t) t ≤
      hash-target-budget-value B n'
    using k-budget[OF m-out] unfolding hash-target-budget-def by blast
  show ?thesis
    using Some xt tail by simp
qed
qed
have wp-event (m ≫ k) ?E s ≤ wp m ?R s
  by (rule split)
also have ... = wp-event m ?E s + wp m ?Tail s
  unfolding wp-event-def wp-def dist-expect-def
  by (simp add: sum.distrib algebra-simps
    hash-new-output-hit-event-def)
also have ... ≤
  hash-target-budget-value B n + hash-target-budget-value B n'
  by (intro add-mono head-bound tail-bound)
also have ... = hash-target-budget-value B (n + n')
  by (rule hash-target-budget-value-add)
finally show
  wp-event (m ≫ k) (hash-new-output-hit-event B s) s ≤
    hash-target-budget-value B (n + n') .
qed

lemma hash-target-program-bind-on-outcomes:
  fixes m :: ('x, ('f, 'a) protocol-channel-scheme) state-monad
  and k :: 'x ⇒ ('y, ('f, 'a) protocol-channel-scheme) state-monad
  assumes m: hash-target-program B n m
  and k:
     $\bigwedge s x t. \text{Some } (x, t) \in \text{set-dist } (\text{execute } m s) \implies$ 
    hash-target-program B n' (k x)
  shows hash-target-program B (n + n') (m ≫ k)
  unfolding hash-target-program-def
proof (intro conjI)
  show hash-extension-preserving (m ≫ k)
    by (rule hash-extension-preserving-bind-on-outcomes)
    (use m k in ⟨simp-all add: hash-target-program-def⟩)
  show hash-target-budget B (n + n') (m ≫ k)
    by (rule hash-target-budget-bind-on-outcomes)
    (use m k in ⟨simp-all add: hash-target-program-def⟩)
qed

```

```

lemma hash-target-program-ntimes:
  fixes  $m :: ('x, ('f, 'a) \text{protocol-channel-scheme}) \text{state-monad}$ 
  assumes  $m: \text{hash-target-program } B \ n \ m$ 
  shows  $\text{hash-target-program } B \ (k * n) \ (\text{ntimes } m \ k)$ 
  using  $m$ 
proof (induction k)
  case 0
  then show ?case
    by (simp add: hash-target-program-return)
next
  case (Suc k)
  have tail:  $\text{hash-target-program } B \ (k * n) \ (\text{ntimes } m \ k)$ 
    using Suc.IH Suc.prems by simp
  have cont:
     $\text{hash-target-program } B \ (k * n + 0)$ 
     $(\text{ntimes } m \ k \gg (\lambda xs. \text{return } (x \# xs)))$  for  $x$ 
    by (rule hash-target-program-bind[OF tail hash-target-program-return])
  have  $\text{hash-target-program } B \ (n + (k * n + 0))$ 
     $(m \gg (\lambda x. \text{ntimes } m \ k \gg (\lambda xs. \text{return } (x \# xs))))$ 
    by (rule hash-target-program-bind[OF Suc.prems cont])
  then show ?case by simp
qed

```

```

lemma hash-target-program-mmap:
  fixes  $ms :: ('x, ('f, 'a) \text{protocol-channel-scheme}) \text{state-monad list}$ 
  assumes  $ms: \bigwedge m. m \in \text{set } ms \implies \text{hash-target-program } B \ n \ m$ 
  shows  $\text{hash-target-program } B \ (\text{length } ms * n) \ (\text{mmap } ms)$ 
  using  $ms$ 
proof (induction ms)
  case Nil
  then show ?case
    by (simp add: hash-target-program-return)
next
  case (Cons m ms)
  have head:  $\text{hash-target-program } B \ n \ m$ 
    using Cons.prems by simp
  have tail:  $\text{hash-target-program } B \ (\text{length } ms * n) \ (\text{mmap } ms)$ 
    using Cons.IH Cons.prems by simp
  have cont:
     $\text{hash-target-program } B \ (\text{length } ms * n + 0)$ 
     $(\text{mmap } ms \gg (\lambda xs. \text{return } (x \# xs)))$  for  $x$ 
    by (rule hash-target-program-bind[OF tail hash-target-program-return])
  have  $\text{hash-target-program } B \ (n + (\text{length } ms * n + 0))$ 
     $(m \gg (\lambda x. \text{mmap } ms \gg (\lambda xs. \text{return } (x \# xs))))$ 
    by (rule hash-target-program-bind[OF head cont])
  then show ?case by simp
qed

```

```

lemma hash-target-program-mfold:

```

```

fixes steps ::
  ('x  $\Rightarrow$  ('x, ('f, 'a) protocol-channel-scheme) state-monad) list
assumes steps:
   $\bigwedge \text{step } x. \text{step} \in \text{set steps} \implies$ 
    hash-target-program B n (step x)
shows hash-target-program B (length steps * n) (mfold x steps)
using steps
proof (induction steps arbitrary: x)
  case Nil
  then show ?case
    by (simp add: hash-target-program-return)
next
  case (Cons step steps)
  have head: hash-target-program B n (step x)
    using Cons.prem by simp
  have tail:
     $\bigwedge y. \text{hash-target-program B (length steps * n) (mfold y steps)}$ 
    using Cons.IH Cons.prem by simp
  have hash-target-program B (n + length steps * n)
    (step x  $\gg$  ( $\lambda y. \text{mfold y steps}$ ))
    by (rule hash-target-program-bind[OF head tail])
  then show ?case by simp
qed

end

end

```

```

theory Soundness-Oracle-Budgets
imports Soundness-Oracle-Target
begin

```

Random-oracle range and collision budget calculus for protocol programs.

```

context soundness
begin

```

```

definition hash-collision-budget-value :: nat  $\Rightarrow$  nat  $\Rightarrow$  prob
where
  hash-collision-budget-value c n =
    nnreal (n * (c + n)) / nnreal size

```

```

definition hash-range-budget
:: nat  $\Rightarrow$  ('r, ('f, 'a) protocol-channel-scheme) state-monad  $\Rightarrow$  bool
where
  hash-range-budget n m  $\longleftrightarrow$ 
    ( $\forall s \ x \ t. \text{Some } (x, t) \in \text{set-dist (execute m s)} \longrightarrow$ 
      card (hash-map-output-values t)  $\leq$ 

```

$\text{card } (\text{hash-map-output-values } s) + n)$

definition *hash-collision-budget*

$:: \text{nat} \Rightarrow ('r, ('f, 'a) \text{ protocol-channel-scheme}) \text{ state-monad} \Rightarrow \text{bool}$

where

$\text{hash-collision-budget } n \ m \longleftrightarrow$
 $(\forall s. \neg \text{hash-map-output-collision } s \longrightarrow$
 $\text{wp-event } m \ (\text{hash-new-collision-event } s) \ s \leq$
 $\text{hash-collision-budget-value } (\text{card } (\text{hash-map-output-values } s)) \ n)$

lemma *hash-collision-budget-value-mono-left:*

assumes $c \leq d$

shows $\text{hash-collision-budget-value } c \ n \leq$

$\text{hash-collision-budget-value } d \ n$

unfolding *hash-collision-budget-value-def*

apply (*subst nn2real-le-iff[symmetric]*)

using *assms* **by** (*simp add: divide-right-mono add-mono mult-left-mono*)

lemma *hash-collision-budget-value-mono-right:*

assumes $m \leq n$

shows $\text{hash-collision-budget-value } c \ m \leq$

$\text{hash-collision-budget-value } c \ n$

unfolding *hash-collision-budget-value-def*

apply (*subst nn2real-le-iff[symmetric]*)

using *assms* **by** (*simp add: divide-right-mono add-mono mult-mono*)

lemma *hash-range-budget-mono:*

fixes $p :: ('r, ('f, 'a) \text{ protocol-channel-scheme}) \text{ state-monad}$

assumes $m \leq n$

and *budget: hash-range-budget m p*

shows *hash-range-budget n p*

using *assms* **unfolding** *hash-range-budget-def* **by** *force*

lemma *hash-collision-budget-mono:*

fixes $p :: ('r, ('f, 'a) \text{ protocol-channel-scheme}) \text{ state-monad}$

assumes $m \leq n$

and *budget: hash-collision-budget m p*

shows *hash-collision-budget n p*

unfolding *hash-collision-budget-def*

proof (*intro allI impI*)

fix $s :: ('f, 'a) \text{ protocol-channel-scheme}$

assume *clean: $\neg \text{hash-map-output-collision } s$*

have *wp-event p (hash-new-collision-event s) s $\leq$*

hash-collision-budget-value (card (hash-map-output-values s)) m

using *budget clean* **unfolding** *hash-collision-budget-def* **by** *blast*

also have $\dots \leq$

hash-collision-budget-value (card (hash-map-output-values s)) n

by (*rule hash-collision-budget-value-mono-right[OF assms(1)]*)

finally show *wp-event p (hash-new-collision-event s) s $\leq$*

hash-collision-budget-value (*card* (*hash-map-output-values* *s*)) *n* .
qed

lemma *hash-collision-budget-value-bind-le*:
hash-collision-budget-value *c* *n* +
hash-collision-budget-value (*c* + *n*) *n'* ≤
hash-collision-budget-value *c* (*n* + *n'*)
unfolding *hash-collision-budget-value-def*
by (*simp add: add-divide-nnreal algebra-simps*)

lemma *hash-range-budget-bind*:
fixes *m* :: ('*x*, ('*f*, '*a*) *protocol-channel-scheme*) *state-monad*
and *k* :: '*x* ⇒ ('*y*, ('*f*, '*a*) *protocol-channel-scheme*) *state-monad*
assumes *m*: *hash-range-budget* *n* *m*
and *k*: $\bigwedge x. \text{hash-range-budget } n' (k\ x)$
shows *hash-range-budget* (*n* + *n'*) (*m* $\ggg$ *k*)
unfolding *hash-range-budget-def*
proof (*intro allI impI*)
fix *s y t*
assume *outcome*: *Some* (*y*, *t*) ∈ *set-dist* (*execute* (*m* $\ggg$ *k*) *s*)
from *outcome* **obtain** *x s1* **where**
m-out: *Some* (*x*, *s1*) ∈ *set-dist* (*execute* *m* *s*)
and *k-out*: *Some* (*y*, *t*) ∈ *set-dist* (*execute* (*k* *x*) *s1*)
by (*auto elim!: set-dist-bindE*)
have *m-range*:
card (*hash-map-output-values* *s1*) ≤
card (*hash-map-output-values* *s*) + *n*
using *m m-out* **unfolding** *hash-range-budget-def* **by** *blast*
have *k-range*:
card (*hash-map-output-values* *t*) ≤
card (*hash-map-output-values* *s1*) + *n'*
using *k k-out* **unfolding** *hash-range-budget-def* **by** *blast*
show *card* (*hash-map-output-values* *t*)
≤ *card* (*hash-map-output-values* *s*) + (*n* + *n'*)
using *m-range k-range* **by** *linarith*
qed

lemma *hash-range-budget-bind-on-outcomes*:
fixes *m* :: ('*x*, ('*f*, '*a*) *protocol-channel-scheme*) *state-monad*
and *k* :: '*x* ⇒ ('*y*, ('*f*, '*a*) *protocol-channel-scheme*) *state-monad*
assumes *m*: *hash-range-budget* *n* *m*
and *k*:
 $\bigwedge s\ x\ t. \text{Some } (x, t) \in \text{set-dist } (\text{execute } m\ s) \implies$
hash-range-budget *n'* (*k* *x*)
shows *hash-range-budget* (*n* + *n'*) (*m* $\ggg$ *k*)
unfolding *hash-range-budget-def*
proof (*intro allI impI*)
fix *s y t*
assume *outcome*: *Some* (*y*, *t*) ∈ *set-dist* (*execute* (*m* $\ggg$ *k*) *s*)

```

from outcome obtain  $x\ s1$  where
   $m\text{-out}$ : Some  $(x, s1) \in \text{set-dist (execute } m\ s)$ 
  and  $k\text{-out}$ : Some  $(y, t) \in \text{set-dist (execute (} k\ x) s1)$ 
  by (auto elim!: set-dist-bindE)
have  $m\text{-range}$ :
   $\text{card (hash-map-output-values } s1) \leq$ 
     $\text{card (hash-map-output-values } s) + n$ 
  using  $m\ m\text{-out}$  unfolding hash-range-budget-def by blast
have  $k\text{-range}$ :
   $\text{card (hash-map-output-values } t) \leq$ 
     $\text{card (hash-map-output-values } s1) + n'$ 
  using  $k[OF\ m\text{-out}]\ k\text{-out}$  unfolding hash-range-budget-def by blast
show  $\text{card (hash-map-output-values } t)$ 
   $\leq \text{card (hash-map-output-values } s) + (n + n')$ 
  using  $m\text{-range}\ k\text{-range}$  by linarith
qed

lemma hash-new-collision-event-step-bound:
  fixes  $m :: ('x, ('f, 'a)\ \text{protocol-channel-scheme})\ \text{state-monad}$ 
  and  $s\ t :: ('f, 'a)\ \text{protocol-channel-scheme}$ 
  assumes clean:  $\neg \text{hash-map-output-collision } s$ 
  shows
     $\text{wp-event } m\ (\text{hash-new-collision-event } s)\ t \leq$ 
      (if  $\text{hash-map-new-output-collision } s\ t$  then 1
       else  $\text{wp-event } m\ (\text{hash-new-collision-event } t)\ t)$ 
proof (cases hash-map-new-output-collision s t)
  case True
  then show ?thesis
    using  $\text{wp-event-le-1}[of\ m\ \text{hash-new-collision-event } s\ t]$  by simp
next
  case False
  then have  $\text{clean-t}$ :  $\neg \text{hash-map-output-collision } t$ 
  using clean unfolding hash-map-new-output-collision-def by simp
  have same:
     $\text{hash-new-collision-event } s\ \text{out} = \text{hash-new-collision-event } t\ \text{out}$ 
  for  $\text{out} :: ('x \times ('f, 'a)\ \text{protocol-channel-scheme})\ \text{option}$ 
  using clean clean-t
  unfolding hash-new-collision-event-def hash-map-new-output-collision-def
  by (cases out) auto
  have  $\text{wp-event } m\ (\text{hash-new-collision-event } s)\ t =$ 
     $\text{wp-event } m\ (\text{hash-new-collision-event } t)\ t$ 
  unfolding wp-event-def by (simp add: same)
  show ?thesis
    using False  $\langle \text{wp-event } m\ (\text{hash-new-collision-event } s)\ t =$ 
       $\text{wp-event } m\ (\text{hash-new-collision-event } t)\ t \rangle$  by simp
qed

```

```

lemma hash-collision-budget-bind:
  fixes  $m :: ('x, ('f, 'a)\ \text{protocol-channel-scheme})\ \text{state-monad}$ 

```



```

    and k :: 'x ⇒ ('y, ('f, 'a) protocol-channel-scheme) state-monad
  assumes range-m: hash-range-budget n m
    and coll-m: hash-collision-budget n m
    and range-k:  $\bigwedge x. \text{hash-range-budget } n' (k \ x)$ 
    and coll-k:  $\bigwedge x. \text{hash-collision-budget } n' (k \ x)$ 
  shows hash-collision-budget (n + n') (m  $\ggg$  k)
  unfolding hash-collision-budget-def
proof (intro allI impI)
  fix s :: ('f, 'a) protocol-channel-scheme
  assume clean:  $\neg \text{hash-map-output-collision } s$ 
  let ?c = card (hash-map-output-values s)
  let ?B1 = hash-collision-budget-value ?c n
  let ?B2 = hash-collision-budget-value (?c + n) n'
  let ?Em = hash-new-collision-event s
  let ?S =
     $\lambda \text{out. case out of}$ 
     $\text{None} \Rightarrow 0$ 
    |  $\text{Some } (x, s1) \Rightarrow$ 
       $\text{if hash-map-new-output-collision } s \ s1 \text{ then } 0$ 
       $\text{else wp-event } (k \ x) (\text{hash-new-collision-event } s1) \ s1$ 
  let ?R =
     $\lambda \text{out. (if ?Em out then } 1 \text{ else } 0) + ?S \text{ out}$ 
  have split:
     $\text{wp-event } (m \ggg k) (\text{hash-new-collision-event } s) \ s \leq \text{wp } m \ ?R \ s$ 
  proof -
    have exact:
       $\text{wp-event } (m \ggg k) (\text{hash-new-collision-event } s) \ s =$ 
       $\text{wp } m$ 
       $(\lambda \text{out. case out of}$ 
       $\text{None} \Rightarrow 0$ 
      |  $\text{Some } (x, s1) \Rightarrow$ 
         $\text{wp-event } (k \ x) (\text{hash-new-collision-event } s) \ s1) \ s$ 
      unfolding wp-event-def hash-new-collision-event-def
      by (simp add: wpsimps)
    have wp m
       $(\lambda \text{out. case out of}$ 
       $\text{None} \Rightarrow 0$ 
      |  $\text{Some } (x, s1) \Rightarrow$ 
         $\text{wp-event } (k \ x) (\text{hash-new-collision-event } s) \ s1) \ s$ 
       $\leq \text{wp } m \ ?R \ s$ 
    proof (rule wp-mono-on-support)
      fix out
      assume out  $\in \text{set-dist } (\text{execute } m \ s)$ 
      show (case out of
         $\text{None} \Rightarrow 0$ 
        |  $\text{Some } (x, s1) \Rightarrow$ 
           $\text{wp-event } (k \ x) (\text{hash-new-collision-event } s) \ s1) \leq ?R \text{ out}$ 
      proof (cases out)
        case None

```

```

then show ?thesis
  unfolding hash-new-collision-event-def by simp
next
case (Some xs)
then obtain x s1 where xs: xs = (x, s1)
  by (cases xs) simp
have step:
  wp-event (k x) (hash-new-collision-event s) s1 ≤
    (if hash-map-new-output-collision s s1 then 1
     else wp-event (k x) (hash-new-collision-event s1) s1)
  by (rule hash-new-collision-event-step-bound[OF clean])
have R-eq:
  ?R out =
    (if hash-map-new-output-collision s s1 then 1
     else wp-event (k x) (hash-new-collision-event s1) s1)
  using Some xs unfolding hash-new-collision-event-def by simp
have L-eq:
  (case out of
   None ⇒ 0
  | Some (x, s1) ⇒
    wp-event (k x) (hash-new-collision-event s) s1) =
    wp-event (k x) (hash-new-collision-event s) s1
  using Some xs by simp
show ?thesis
  using step unfolding L-eq R-eq .
qed
qed
then show ?thesis
  unfolding exact .
qed
have m-bound:
  wp-event m ?Em s ≤ ?B1
  using coll-m clean unfolding hash-collision-budget-def by blast
have S-bound: wp m ?S s ≤ ?B2
proof (rule wp-le-const-on-support)
  fix out
  assume out-support: out ∈ set-dist (execute m s)
  show ?S out ≤ ?B2
  proof (cases out)
  case None
  then show ?thesis by simp
next
case (Some xs)
then obtain x s1 where xs: xs = (x, s1)
  by (cases xs) simp
show ?thesis
proof (cases hash-map-new-output-collision s s1)
  case True
  then show ?thesis

```

```

    using Some xs by simp
next
case False
then have clean-s1:  $\neg$  hash-map-output-collision s1
    using clean unfolding hash-map-new-output-collision-def by simp
have k-bound:
    wp-event (k x) (hash-new-collision-event s1) s1  $\leq$ 
        hash-collision-budget-value
        (card (hash-map-output-values s1)) n'
    using coll-k[of x] clean-s1 unfolding hash-collision-budget-def
    by blast
have m-out:
    Some (x, s1)  $\in$  set-dist (execute m s)
    using out-support Some xs by simp
have range:
    card (hash-map-output-values s1)  $\leq$  ?c + n
    using range-m m-out unfolding hash-range-budget-def by blast
have hash-collision-budget-value
    (card (hash-map-output-values s1)) n'  $\leq$  ?B2
    by (rule hash-collision-budget-value-mono-left[OF range])
then show ?thesis
    using Some xs False k-bound by simp
qed
qed
qed
have wp-event (m  $\gg$  k) (hash-new-collision-event s) s
     $\leq$  wp m ?R s
    by (rule split)
also have ... =
    wp-event m ?Em s + wp m ?S s
    unfolding wp-event-def wp-def dist-expect-def
    by (simp add: sum.distrib algebra-simps hash-new-collision-event-def)
also have ...  $\leq$  ?B1 + ?B2
    using m-bound S-bound by (intro add-mono)
also have ...  $\leq$  hash-collision-budget-value ?c (n + n')
    by (rule hash-collision-budget-value-bind-le)
finally show
    wp-event (m  $\gg$  k) (hash-new-collision-event s) s
     $\leq$  hash-collision-budget-value ?c (n + n') .
qed

lemma hash-collision-budget-bind-on-outcomes:
  fixes m :: ('x, ('f, 'a) protocol-channel-scheme) state-monad
  and k :: 'x  $\Rightarrow$  ('y, ('f, 'a) protocol-channel-scheme) state-monad
  assumes range-m: hash-range-budget n m
  and coll-m: hash-collision-budget n m
  and range-k:
     $\bigwedge s x t. \text{Some } (x, t) \in \text{set-dist } (\text{execute } m \ s) \implies$ 
      hash-range-budget n' (k x)

```

and *coll-k*:
 $\bigwedge s \ x \ t. \text{Some } (x, t) \in \text{set-dist } (\text{execute } m \ s) \implies$
 $\text{hash-collision-budget } n' \ (k \ x)$
shows *hash-collision-budget* $(n + n') \ (m \ggg k)$
unfolding *hash-collision-budget-def*
proof (*intro allI impI*)
fix *s* :: (*f*, *a*) *protocol-channel-scheme*
assume *clean*: $\neg \text{hash-map-output-collision } s$
let *?c* = *card* (*hash-map-output-values* *s*)
let *?B1* = *hash-collision-budget-value* *?c* *n*
let *?B2* = *hash-collision-budget-value* (*?c* + *n*) *n'*
let *?Em* = *hash-new-collision-event* *s*
let *?S* =
 $\lambda \text{out. case out of}$
 $\text{None} \Rightarrow 0$
 $| \text{Some } (x, s1) \Rightarrow$
 $\text{if hash-map-new-output-collision } s \ s1 \text{ then } 0$
 $\text{else wp-event } (k \ x) \ (\text{hash-new-collision-event } s1) \ s1$
let *?R* =
 $\lambda \text{out. (if ?Em out then } 1 \text{ else } 0) + ?S \text{ out}$
have *split*:
 $\text{wp-event } (m \ggg k) \ (\text{hash-new-collision-event } s) \ s \leq \text{wp } m \ ?R \ s$
proof –
have *exact*:
 $\text{wp-event } (m \ggg k) \ (\text{hash-new-collision-event } s) \ s =$
 $\text{wp } m$
 $(\lambda \text{out. case out of}$
 $\text{None} \Rightarrow 0$
 $| \text{Some } (x, s1) \Rightarrow$
 $\text{wp-event } (k \ x) \ (\text{hash-new-collision-event } s) \ s1) \ s$
unfolding *wp-event-def* *hash-new-collision-event-def*
by (*simp add: wpsimps*)
have *wp m*
 $(\lambda \text{out. case out of}$
 $\text{None} \Rightarrow 0$
 $| \text{Some } (x, s1) \Rightarrow$
 $\text{wp-event } (k \ x) \ (\text{hash-new-collision-event } s) \ s1) \ s$
 $\leq \text{wp } m \ ?R \ s$
proof (*rule wp-mono-on-support*)
fix *out*
assume *out* $\in \text{set-dist } (\text{execute } m \ s)$
show (*case out of*
 $\text{None} \Rightarrow 0$
 $| \text{Some } (x, s1) \Rightarrow$
 $\text{wp-event } (k \ x) \ (\text{hash-new-collision-event } s) \ s1) \leq ?R \ \text{out}$
proof (*cases out*)
case *None*
then show *?thesis*
unfolding *hash-new-collision-event-def* **by** *simp*

```

next
  case (Some xs)
  then obtain  $x$   $s1$  where  $xs$ :  $xs = (x, s1)$ 
    by (cases xs) simp
  have step:
    wp-event ( $k$   $x$ ) (hash-new-collision-event  $s$ )  $s1 \leq$ 
      (if hash-map-new-output-collision  $s$   $s1$  then 1
       else wp-event ( $k$   $x$ ) (hash-new-collision-event  $s1$ )  $s1$ )
    by (rule hash-new-collision-event-step-bound[OF clean])
  have R-eq:
    ?R out =
      (if hash-map-new-output-collision  $s$   $s1$  then 1
       else wp-event ( $k$   $x$ ) (hash-new-collision-event  $s1$ )  $s1$ )
    using Some xs unfolding hash-new-collision-event-def by simp
  have L-eq:
    (case out of
     None  $\Rightarrow$  0
     | Some ( $x, s1$ )  $\Rightarrow$ 
       wp-event ( $k$   $x$ ) (hash-new-collision-event  $s$ )  $s1$ ) =
       wp-event ( $k$   $x$ ) (hash-new-collision-event  $s$ )  $s1$ 
    using Some xs by simp
  show ?thesis
    using step unfolding L-eq R-eq .
qed
qed
then show ?thesis
  unfolding exact .
qed
have m-bound:
  wp-event  $m$  ?Em  $s \leq$  ?B1
  using coll-m clean unfolding hash-collision-budget-def by blast
have S-bound: wp  $m$  ?S  $s \leq$  ?B2
proof (rule wp-le-const-on-support)
  fix out
  assume out-support: out  $\in$  set-dist (execute  $m$   $s$ )
  show ?S out  $\leq$  ?B2
  proof (cases out)
    case None
    then show ?thesis by simp
  next
    case (Some xs)
    then obtain  $x$   $s1$  where  $xs$ :  $xs = (x, s1)$ 
      by (cases xs) simp
    have m-out:
      Some ( $x, s1$ )  $\in$  set-dist (execute  $m$   $s$ )
      using out-support Some xs by simp
    show ?thesis
    proof (cases hash-map-new-output-collision  $s$   $s1$ )
      case True

```

```

then show ?thesis
  using Some xs by simp
next
case False
then have clean-s1:  $\neg$  hash-map-output-collision s1
  using clean unfolding hash-map-new-output-collision-def by simp
have k-bound:
  wp-event (k x) (hash-new-collision-event s1) s1  $\leq$ 
    hash-collision-budget-value
    (card (hash-map-output-values s1)) n'
  using coll-k[OF m-out] clean-s1
  unfolding hash-collision-budget-def by blast
have range:
  card (hash-map-output-values s1)  $\leq$  ?c + n
  using range-m m-out unfolding hash-range-budget-def by blast
have hash-collision-budget-value
  (card (hash-map-output-values s1)) n'  $\leq$  ?B2
  by (rule hash-collision-budget-value-mono-left[OF range])
then show ?thesis
  using Some xs False k-bound by simp
qed
qed
qed
have wp-event (m  $\gg$  k) (hash-new-collision-event s) s
   $\leq$  wp m ?R s
  by (rule split)
also have ... =
  wp-event m ?Em s + wp m ?S s
  unfolding wp-event-def wp-def dist-expect-def
  by (simp add: sum.distrib algebra-simps hash-new-collision-event-def)
also have ...  $\leq$  ?B1 + ?B2
  using m-bound S-bound by (intro add-mono)
also have ...  $\leq$  hash-collision-budget-value ?c (n + n')
  by (rule hash-collision-budget-value-bind-le)
finally show
  wp-event (m  $\gg$  k) (hash-new-collision-event s) s
   $\leq$  hash-collision-budget-value ?c (n + n') .
qed

lemma wp-hash-new-output-collision-bound:
fixes x :: 'f protocol-hash-input
and s :: ('f, 'a) protocol-channel-scheme
assumes clean:  $\neg$  hash-map-output-collision s
shows
  wp-event (hash x)
  (hash-new-collision-event s) s
   $\leq$  nnreal (card (hash-map-output-values s)) / nnreal size
proof (cases fmlookup (HashMap s) x)
case None

```

```

let ?P =
  λy. hash-map-new-output-collision s
    (s(HashMap := fmupd x y (HashMap s)))
let ?Q = λy. y ∈ hash-map-output-values s
have mono:
  dist-expect (hash-dist x s) (λy. if ?P y then 1 else 0) ≤
    dist-expect (hash-dist x s) (λy. if ?Q y then 1 else 0)
  unfolding dist-expect-def
proof (intro sum-mono)
  fix y
  assume y ∈ dom (dist (hash-dist x s))
  show the (dist (hash-dist x s) y) * (if ?P y then 1 else 0)
    ≤ the (dist (hash-dist x s) y) * (if ?Q y then 1 else 0)
  proof (cases ?P y)
    case True
    then have ?Q y
      using hash-map-output-collision-after-fresh-updateD
        [OF clean None]
      unfolding hash-map-new-output-collision-def by simp
    then show ?thesis
      using True by simp
  next
    case False
    then show ?thesis by simp
  qed
qed
show ?thesis
proof -
  have wp-event (hash x)
    (hash-new-collision-event s) s =
    dist-expect (hash-dist x s) (λy. if ?P y then 1 else 0)
  unfolding wp-event-def hash-new-collision-event-def
  by (simp add: wp-hash)
  also have ... ≤
    dist-expect (hash-dist x s) (λy. if ?Q y then 1 else 0)
  by (rule mono)
  also have ... =
    nnreal (card (hash-map-output-values s)) / nnreal size
  by (rule dist-expect-hash-dist-fresh-output-values[OF None])
  finally show ?thesis .
qed
next
case (Some y)
have no-new:
  ¬ hash-map-new-output-collision s
    (s(HashMap := fmupd x y (HashMap s)))
  using no-hash-map-output-collision-after-known-update[OF clean Some]
  unfolding hash-map-new-output-collision-def by simp
show ?thesis

```

unfolding *wp-event-def hash-new-collision-event-def*
by (*simp add: wp-hash Some hash-dist-def option-default-dist-def no-new*)
qed

lemma *hash-range-budget-return:*
hash-range-budget 0 (return x)
unfolding *hash-range-budget-def* **by** *simp*

lemma *hash-collision-budget-return:*
hash-collision-budget 0 (return x)
unfolding *hash-collision-budget-def hash-new-collision-event-def*
hash-collision-budget-value-def hash-map-new-output-collision-def
by (*simp add: wp-event-def wpsimps*)

lemma *hash-range-budget-hash:*
fixes *x :: 'f protocol-hash-input*
shows
hash-range-budget 1
(hash x :: ('f, ('f, 'a) protocol-channel-scheme) state-monad)
unfolding *hash-range-budget-def*
proof (*intro allI impI*)
fix *s y t*
assume *outcome:*
Some (y, t) ∈
set-dist (execute
(hash x :: ('f, ('f, 'a) protocol-channel-scheme) state-monad) s)
have *t-eq: t = s(HashMap := fmupd x y (HashMap s))*
proof –
from *outcome* **obtain** *a s1* **where**
apply-step:
Some (a, s1) ∈ set-dist (execute (apply-hash x) s)
and *rest:*
Some (y, t) ∈
set-dist (execute
(modify-HashMap x a >>= (λ-. return a)) s1)
proof –
have *Some (y, t) ∈*
set-dist (execute
(apply-hash x >>=
(λa. modify-HashMap x a >>= (λ-. return a))) s)
using *outcome* **unfolding** *hash-def* .
then show *?thesis*
by (*rule set-dist-bindE*) (*rule that*)
qed
have *apply-img:*
Some (a, s1) ∈ (λa. Some (a, s)) ‘ set-dist (hash-dist x s)
using *apply-step*
unfolding *apply-hash-def lift.rep-eq lift-dist-def*
by (*simp add: set-dist-dist-map*)


```

then have s1-eq:  $s1 = s$ 
  by auto
from rest obtain  $u\ s2$  where
  mod:
     $\text{Some } (u, s2) \in \text{set-dist } (\text{execute } (\text{modify-HashMap } x\ a)\ s1)$ 
  and ret:
     $\text{Some } (y, t) \in \text{set-dist } (\text{execute } (\text{return } a)\ s2)$ 
  by (elim set-dist-bindE)
have s2-eq:  $s2 = s1 \langle \text{HashMap} := \text{fmupd } x\ a\ (\text{HashMap } s1) \rangle$ 
  using mod unfolding modify-HashMap-def modify-def
  by (auto elim!: set-dist-bindE)
have y-eq:  $y = a$  and t-eq-s2:  $t = s2$ 
  using ret by auto
show ?thesis
  using s1-eq s2-eq y-eq t-eq-s2 by simp
qed
show card (hash-map-output-values t)
   $\leq \text{card } (\text{hash-map-output-values } s) + 1$ 
unfolding t-eq
using card-hash-map-output-values-after-update-le[of x y s]
by simp
qed

lemma hash-collision-budget-hash:
  fixes  $x :: 'f \text{ protocol-hash-input}$ 
  shows
    hash-collision-budget 1
     $(\text{hash } x :: ('f, ('f, 'a) \text{ protocol-channel-scheme}) \text{ state-monad})$ 
  unfolding hash-collision-budget-def
proof (intro allI impI)
  fix  $s :: ('f, 'a) \text{ protocol-channel-scheme}$ 
  assume clean:  $\neg \text{hash-map-output-collision } s$ 
  have one-step:
     $\text{wp-event } (\text{hash } x) (\text{hash-new-collision-event } s) s \leq$ 
     $\text{nnreal } (\text{card } (\text{hash-map-output-values } s)) / \text{nnreal size}$ 
    by (rule wp-hash-new-output-collision-bound[OF clean])
  also have  $\dots \leq$ 
     $\text{hash-collision-budget-value } (\text{card } (\text{hash-map-output-values } s))\ 1$ 
  unfolding hash-collision-budget-value-def
  apply (subst nn2real-le-iff[symmetric])
  by (simp add: divide-right-mono)
  finally show
     $\text{wp-event } (\text{hash } x) (\text{hash-new-collision-event } s) s \leq$ 
     $\text{hash-collision-budget-value } (\text{card } (\text{hash-map-output-values } s))\ 1 .$ 
qed

lemma hash-range-budget-get:
  hash-range-budget 0
  (get ::  $((f, 'a) \text{ protocol-channel-scheme},$ 

```

(*'f, 'a*) *protocol-channel-scheme*) *state-monad*)
unfolding *hash-range-budget-def* **by** *simp*

lemma *hash-collision-budget-get*:
hash-collision-budget 0
(*get* :: ((*'f, 'a*) *protocol-channel-scheme*,
(*'f, 'a*) *protocol-channel-scheme*) *state-monad*)
unfolding *hash-collision-budget-def* *hash-new-collision-event-def*
hash-map-new-output-collision-def *hash-collision-budget-value-def*
by (*simp add: wp-event-def wpsimps*)

lemma *hash-range-budget-assert*:
hash-range-budget 0
(*assert* *b* :: (*unit*, (*'f, 'a*) *protocol-channel-scheme*) *state-monad*)
unfolding *hash-range-budget-def* *assert-def*
by (*auto simp: throw-no-outcome*)

lemma *hash-collision-budget-assert*:
hash-collision-budget 0
(*assert* *b* :: (*unit*, (*'f, 'a*) *protocol-channel-scheme*) *state-monad*)
unfolding *hash-collision-budget-def* *hash-new-collision-event-def*
hash-map-new-output-collision-def *hash-collision-budget-value-def*
assert-def
by (*simp add: wp-event-def wpsimps*)

lemma *hash-range-budget-send*:
hash-range-budget 0 (*send* *x*)
unfolding *hash-range-budget-def* *send-def* *protocol-send-def* *modify-def*
by (*auto simp: hash-map-output-values-def elim!: set-dist-bindE*)

lemma *hash-collision-budget-send*:
hash-collision-budget 0 (*send* *x*)
unfolding *hash-collision-budget-def* *hash-new-collision-event-def*
hash-map-new-output-collision-def *hash-collision-budget-value-def*
hash-map-output-collision-def
send-def *protocol-send-def*
by (*simp add: wp-event-def wpsimps*)

lemma *hash-range-budget-read*:
hash-range-budget 0 (*read* :: (*'f, 'f, 'a*) *protocol-c-monad*)
unfolding *hash-range-budget-def*
proof (*intro allI impI*)
fix *s x t*
assume *out*: *Some* (*x, t*) ∈
set-dist (*execute* (*read* :: (*'f, 'f, 'a*) *protocol-c-monad*) *s*)
obtain *y ys* **where** *tr*: *PTranscript* *s* = *y* # *ys*
proof –
have *PTranscript* *s* ≠ []
using *out* **unfolding** *read-def* *protocol-read-def* *assert-def*

```

    by (cases PTranscript s)
      (auto simp: throw-no-outcome elim!: set-dist-bindE)
  then show ?thesis
    by (cases PTranscript s) (auto intro: that)
qed
have t-eq:
  t = s(PState := concat (PState s) y, PTranscript := ys)
  using read-nonempty-outcome[OF out tr] by simp
show card (hash-map-output-values t)
  ≤ card (hash-map-output-values s) + 0
  unfolding t-eq hash-map-output-values-def by simp
qed

lemma hash-collision-budget-read:
  hash-collision-budget 0
  (read :: ('f, 'a) protocol-c-monad)
  unfolding hash-collision-budget-def hash-new-collision-event-def
    hash-map-new-output-collision-def hash-collision-budget-value-def
    hash-map-output-collision-def
    read-def protocol-read-def assert-def
  by (simp add: wp-event-def wpsimps)

lemma hash-range-budget-modify-preserves-hash-map:
  assumes  $\bigwedge s. \text{HashMap } (f\ s) = \text{HashMap } s$ 
  shows hash-range-budget 0 (modify f)
  using assms
  unfolding hash-range-budget-def modify-def hash-map-output-values-def
  by (auto elim!: set-dist-bindE)

lemma hash-collision-budget-modify-preserves-hash-map:
  assumes  $\bigwedge s. \text{HashMap } (f\ s) = \text{HashMap } s$ 
  shows hash-collision-budget 0 (modify f)
  using assms
  unfolding hash-collision-budget-def hash-new-collision-event-def
    hash-map-new-output-collision-def hash-collision-budget-value-def
    hash-map-output-collision-def modify-def
  by (simp add: wp-event-def wpsimps)

lemma hash-map-preserving-get:
  hash-map-preserving
  (get :: (('f, 'a) protocol-channel-scheme,
    ('f, 'a) protocol-channel-scheme) state-monad)
  unfolding hash-map-preserving-def by simp

lemma hash-map-preserving-assert:
  hash-map-preserving
  (assert b :: (unit, ('f, 'a) protocol-channel-scheme) state-monad)
  unfolding hash-map-preserving-def assert-def
  by (auto simp: throw-no-outcome elim!: set-dist-bindE)

```

lemma *hash-map-preserving-send*:
hash-map-preserving (*send* *x*)
unfolding *hash-map-preserving-def* *send-def* *protocol-send-def* *modify-def*
by (*auto elim!*: *set-dist-bindE*)

lemma *hash-map-preserving-read*:
hash-map-preserving (*read* :: ('f, 'f, 'a) *protocol-c-monad*)
unfolding *hash-map-preserving-def*
proof (*intro allI impI*)
fix *s x t*
assume *out*:
Some (*x*, *t*) ∈
set-dist (*execute* (*read* :: ('f, 'f, 'a) *protocol-c-monad*) *s*)
obtain *y ys* **where** *tr*: *PTranscript* *s* = *y* # *ys*
proof –
have *PTranscript* *s* ≠ []
using *out* **unfolding** *read-def* *protocol-read-def* *assert-def*
by (*cases* *PTranscript* *s*)
(*auto simp*: *throw-no-outcome elim!*: *set-dist-bindE*)
then show ?thesis
by (*cases* *PTranscript* *s*) (*auto intro*: *that*)
qed
have *t-eq*:
t = *s*(*PState* := *concat* (*PState* *s*) *y*, *PTranscript* := *ys*)
using *read-nonempty-outcome*[*OF out tr*] **by** *simp*
show *HashMap* *t* = *HashMap* *s*
unfolding *t-eq* **by** *simp*
qed

lemma *hash-map-preserving-modify*:
assumes $\bigwedge s. \text{HashMap } (f\ s) = \text{HashMap } s$
shows *hash-map-preserving* (*modify* *f*)
using *assms* **unfolding** *hash-map-preserving-def* *modify-def*
by (*auto elim!*: *set-dist-bindE*)

lemma *hash-target-program-get*:
hash-target-program *B* 0
(*get* :: (('f, 'a) *protocol-channel-scheme*,
('f, 'a) *protocol-channel-scheme*) *state-monad*)
by (*rule* *hash-map-preserving-imp-hash-target-program-zero*)
(*rule* *hash-map-preserving-get*)

lemma *hash-target-program-assert*:
hash-target-program *B* 0
(*assert* *b* :: (*unit*, ('f, 'a) *protocol-channel-scheme*) *state-monad*)
by (*rule* *hash-map-preserving-imp-hash-target-program-zero*)
(*rule* *hash-map-preserving-assert*)

lemma *hash-target-program-send*:
hash-target-program B 0 (send x)
by (rule *hash-map-preserving-imp-hash-target-program-zero*)
(rule *hash-map-preserving-send*)

lemma *hash-target-program-read*:
hash-target-program B 0 (read :: ('f, 'f, 'a) protocol-c-monad)
by (rule *hash-map-preserving-imp-hash-target-program-zero*)
(rule *hash-map-preserving-read*)

lemma *hash-target-program-ntimes-read-bind*:
fixes *k :: 'f list* $\Rightarrow$
('y, ('f, 'a) protocol-channel-scheme) state-monad
assumes *k*:
 $\bigwedge xs. \text{length } xs = n \implies \text{hash-target-program } B \ b \ (k \ xs)$
shows
hash-target-program B b
((*ntimes read n* ::
('f list, ('f, 'a) protocol-channel-scheme) state-monad) $\ggg$ *k*)

proof –
have *reads*:
*hash-target-program B (n * 0)*
(*ntimes read n* ::
('f list, ('f, 'a) protocol-channel-scheme) state-monad)
by (rule *hash-target-program-ntimes*[OF *hash-target-program-read*])
have *hash-target-program B (n * 0 + b)*
((*ntimes read n* ::
('f list, ('f, 'a) protocol-channel-scheme) state-monad) $\ggg$ *k*)
proof (rule *hash-target-program-bind-on-outcomes*[OF *reads*])
fix *s xs t*
assume *out*:
Some (*xs*, *t*) $\in$
set-dist (execute
(*ntimes read n* ::
('f list, ('f, 'a) protocol-channel-scheme) state-monad) *s*)
show *hash-target-program B b (k xs)*
by (rule *k*) (use *ntimes-read-any-outcome*[OF *out*] **in** *simp*)
qed
then show ?thesis **by** *simp*
qed

lemma *hash-target-program-modify*:
assumes $\bigwedge s. \text{HashMap } (f \ s) = \text{HashMap } s$
shows *hash-target-program B 0 (modify f)*
by (rule *hash-map-preserving-imp-hash-target-program-zero*)
(rule *hash-map-preserving-modify*[OF *assms*])

lemma *hash-target-program-receive-random-field-element*:
hash-target-program B 1

```

    (receive-random-field-element :: ('f, 'f, 'a) protocol-c-monad)
proof –
  have tail:
    hash-target-program B (1 + 0)
    (hash (FiatShamirChallenge (PState s))  $\gg=$  return ::
      ('f, ('f, 'a) protocol-channel-scheme) state-monad)
    for s :: ('f, 'a) protocol-channel-scheme
    by (rule hash-target-program-bind)
    (rule hash-target-program-hash, rule hash-target-program-return)
  have hash-target-program B (0 + (1 + 0))
    (get  $\gg=$ 
      ( $\lambda$ s :: ('f, 'a) protocol-channel-scheme.
        hash (FiatShamirChallenge (PState s))  $\gg=$  return))
    by (rule hash-target-program-bind[OF hash-target-program-get tail])
  then show ?thesis
    unfolding receive-random-field-element-def
      protocol-receive-random-field-element-def
      protocol-receive-tagged-random-field-element-def
    by simp
qed

```

lemma hash-target-program-receive-tagged-random-field-element:

```

  hash-target-program B 1
  (receive-tagged-random-field-element tag ::
    ('f, 'f, 'a) protocol-c-monad)

```

```

proof –
  have tail:
    hash-target-program B (1 + 0)
    (hash (tag (PState s))  $\gg=$  return ::
      ('f, ('f, 'a) protocol-channel-scheme) state-monad)
    for s :: ('f, 'a) protocol-channel-scheme
    by (rule hash-target-program-bind)
    (rule hash-target-program-hash, rule hash-target-program-return)
  have hash-target-program B (0 + (1 + 0))
    (get  $\gg=$ 
      ( $\lambda$ s :: ('f, 'a) protocol-channel-scheme.
        hash (tag (PState s))  $\gg=$  return))
    by (rule hash-target-program-bind[OF hash-target-program-get tail])
  then show ?thesis
    unfolding receive-tagged-random-field-element-def
      protocol-receive-tagged-random-field-element-def
    by simp
qed

```

lemma hash-target-program-receive-counted-tagged-random-field-element:

```

assumes bump:  $\bigwedge s. \text{HashMap } (\text{bump } s) = \text{HashMap } s$ 
shows hash-target-program B 1
  (protocol-receive-counted-tagged-random-field-element counter bump tag ::
    ('f, 'f, 'a) protocol-c-monad)

```

proof –

have *modify-return*:

hash-target-program $B (0 + 0)$

$(\text{modify bump} \gg (\lambda-. \text{return } r) ::$

$('f, ('f, 'a) \text{protocol-channel-scheme}) \text{state-monad})$

for $r :: 'f$

by (*rule* *hash-target-program-bind*)

(*rule* *hash-target-program-modify*[*OF* *bump*],

rule *hash-target-program-return*)

have *hash-tail*:

hash-target-program $B (1 + (0 + 0))$

$(\text{hash } (\text{tag } (\text{counter } s) (PState\ s)) \gg$

$(\lambda r. \text{modify bump} \gg (\lambda-. \text{return } r)) ::$

$('f, ('f, 'a) \text{protocol-channel-scheme}) \text{state-monad})$

for $s :: ('f, 'a) \text{protocol-channel-scheme}$

by (*rule* *hash-target-program-bind*)

(*rule* *hash-target-program-hash*, *rule* *modify-return*)

have *hash-target-program* $B (0 + (1 + (0 + 0)))$

(*get* $\gg$

$(\lambda s :: ('f, 'a) \text{protocol-channel-scheme}.$

$\text{hash } (\text{tag } (\text{counter } s) (PState\ s)) \gg$

$(\lambda r. \text{modify bump} \gg (\lambda-. \text{return } r))))$

by (*rule* *hash-target-program-bind*[*OF* *hash-target-program-get* *hash-tail*])

then show *?thesis*

unfolding *protocol-receive-counted-tagged-random-field-element-def*

by *simp*

qed

lemma *hash-target-program-receive-alpha-challenge*:

hash-target-program $B\ 1$

$(\text{receive-alpha-challenge} :: ('f, 'f, 'a) \text{protocol-c-monad})$

unfolding *receive-alpha-challenge-def*

by (*rule* *hash-target-program-receive-counted-tagged-random-field-element*)

simp

lemma *hash-target-program-receive-query-index-challenge*:

hash-target-program $B\ 1$

$(\text{receive-query-index-challenge} :: ('f, 'f, 'a) \text{protocol-c-monad})$

unfolding *receive-query-index-challenge-def*

by (*rule* *hash-target-program-receive-counted-tagged-random-field-element*)

simp

lemma *hash-target-program-receive-trace-fri-challenge*:

hash-target-program $B\ 1$

$(\text{receive-trace-fri-challenge} :: ('f, 'f, 'a) \text{protocol-c-monad})$

unfolding *receive-trace-fri-challenge-def*

by (*rule* *hash-target-program-receive-counted-tagged-random-field-element*)

simp

lemma *hash-target-program-receive-composition-fri-challenge*:
hash-target-program B 1
(receive-composition-fri-challenge :: ('f, 'f, 'a) protocol-c-monad)
unfolding *receive-composition-fri-challenge-def*
by (*rule hash-target-program-receive-counted-tagged-random-field-element*)
simp

lemma *hash-range-budget-receive-random-field-element*:
hash-range-budget 1
(receive-random-field-element :: ('f, 'f, 'a) protocol-c-monad)
proof –
have *get-step*:
hash-range-budget 0
(get :: (('f, 'a) protocol-channel-scheme,
('f, 'a) protocol-channel-scheme) state-monad)
by (*rule hash-range-budget-get*)
have *hash-then-return*:
hash-range-budget (1 + 0)
(hash (FiatShamirChallenge (PState s)) $\gg$ return ::
('f, ('f, 'a) protocol-channel-scheme) state-monad)
for *s :: ('f, 'a) protocol-channel-scheme*
by (*rule hash-range-budget-bind*)
(rule hash-range-budget-hash, rule hash-range-budget-return)
have *hash-range-budget (0 + (1 + 0))*
(get $\gg$
($\lambda s :: ('f, 'a) protocol-channel-scheme.$
hash (FiatShamirChallenge (PState s)) $\gg$ return))
by (*rule hash-range-budget-bind[OF get-step hash-then-return]*)
then show *?thesis*
unfolding *receive-random-field-element-def*
protocol-receive-random-field-element-def
protocol-receive-tagged-random-field-element-def
by *simp*
qed

lemma *hash-range-budget-receive-tagged-random-field-element*:
hash-range-budget 1
(receive-tagged-random-field-element tag :: ('f, 'f, 'a) protocol-c-monad)
proof –
have *get-step*:
hash-range-budget 0
(get :: (('f, 'a) protocol-channel-scheme,
('f, 'a) protocol-channel-scheme) state-monad)
by (*rule hash-range-budget-get*)
have *hash-then-return*:
hash-range-budget (1 + 0)
(hash (tag (PState s)) $\gg$ return ::
('f, ('f, 'a) protocol-channel-scheme) state-monad)
for *s :: ('f, 'a) protocol-channel-scheme*


```

    by (rule hash-range-budget-bind)
      (rule hash-range-budget-hash, rule hash-range-budget-return)
  have hash-range-budget (0 + (1 + 0))
    (get  $\gg$ 
      ( $\lambda s :: ('f, 'a)$  protocol-channel-scheme.
        hash (tag (PState s))  $\gg$  return))
  by (rule hash-range-budget-bind[OF get-step hash-then-return])
then show ?thesis
  unfolding receive-tagged-random-field-element-def
    protocol-receive-tagged-random-field-element-def
  by simp
qed

lemma hash-range-budget-receive-counted-tagged-random-field-element:
  assumes bump:  $\bigwedge s. \text{HashMap } (\text{bump } s) = \text{HashMap } s$ 
  shows hash-range-budget 1
    (protocol-receive-counted-tagged-random-field-element counter bump tag ::
      ('f, 'f, 'a) protocol-c-monad)
proof -
  have get-step:
    hash-range-budget 0
    (get :: (('f, 'a) protocol-channel-scheme,
      ('f, 'a) protocol-channel-scheme) state-monad)
  by (rule hash-range-budget-get)
  have modify-return:
    hash-range-budget (0 + 0)
    (modify bump  $\gg$  ( $\lambda -. \text{return } r$ ) ::
      ('f, ('f, 'a) protocol-channel-scheme) state-monad)
  for r :: 'f
  by (rule hash-range-budget-bind)
    (rule hash-range-budget-modify-preserves-hash-map[OF bump],
      rule hash-range-budget-return)
  have hash-then-tail:
    hash-range-budget (1 + (0 + 0))
    (hash (tag (counter s) (PState s))  $\gg$ 
      ( $\lambda r. \text{modify bump } \gg (\lambda -. \text{return } r)$ ) ::
      ('f, ('f, 'a) protocol-channel-scheme) state-monad)
  for s :: ('f, 'a) protocol-channel-scheme
  by (rule hash-range-budget-bind)
    (rule hash-range-budget-hash, rule modify-return)
  have hash-range-budget (0 + (1 + (0 + 0)))
    (get  $\gg$ 
      ( $\lambda s :: ('f, 'a)$  protocol-channel-scheme.
        hash (tag (counter s) (PState s))  $\gg$ 
          ( $\lambda r. \text{modify bump } \gg (\lambda -. \text{return } r)$ )))
  by (rule hash-range-budget-bind[OF get-step hash-then-tail])
then show ?thesis
  unfolding protocol-receive-counted-tagged-random-field-element-def
  by simp

```

qed

lemma *hash-range-budget-receive-alpha-challenge:*

hash-range-budget 1

(receive-alpha-challenge :: ('f, 'f, 'a) protocol-c-monad)

unfolding *receive-alpha-challenge-def*

by *(rule hash-range-budget-receive-counted-tagged-random-field-element) simp*

lemma *hash-range-budget-receive-query-index-challenge:*

hash-range-budget 1

(receive-query-index-challenge :: ('f, 'f, 'a) protocol-c-monad)

unfolding *receive-query-index-challenge-def*

by *(rule hash-range-budget-receive-counted-tagged-random-field-element) simp*

lemma *hash-range-budget-receive-trace-fri-challenge:*

hash-range-budget 1

(receive-trace-fri-challenge :: ('f, 'f, 'a) protocol-c-monad)

unfolding *receive-trace-fri-challenge-def*

by *(rule hash-range-budget-receive-counted-tagged-random-field-element) simp*

lemma *hash-range-budget-receive-composition-fri-challenge:*

hash-range-budget 1

(receive-composition-fri-challenge :: ('f, 'f, 'a) protocol-c-monad)

unfolding *receive-composition-fri-challenge-def*

by *(rule hash-range-budget-receive-counted-tagged-random-field-element) simp*

lemma *hash-collision-budget-receive-random-field-element:*

hash-collision-budget 1

(receive-random-field-element :: ('f, 'f, 'a) protocol-c-monad)

proof –

have *get-range:*

hash-range-budget 0

*(get :: (('f, 'a) protocol-channel-scheme,
(f, 'a) protocol-channel-scheme) state-monad)*

by *(rule hash-range-budget-get)*

have *get-coll:*

hash-collision-budget 0

*(get :: (('f, 'a) protocol-channel-scheme,
(f, 'a) protocol-channel-scheme) state-monad)*

by *(rule hash-collision-budget-get)*

have *hash-then-return-range:*

hash-range-budget (1 + 0)

*(hash (FiatShamirChallenge (PState s)) $\gg$ return ::
(f, (f, 'a) protocol-channel-scheme) state-monad)*

for *s :: ('f, 'a) protocol-channel-scheme*

by *(rule hash-range-budget-bind)*

(rule hash-range-budget-hash, rule hash-range-budget-return)

have *hash-then-return-coll:*

hash-collision-budget (1 + 0)

```

    (hash (FiatShamirChallenge (PState s))  $\gg$  return ::
      ('f, ('f, 'a) protocol-channel-scheme) state-monad)
  for s :: ('f, 'a) protocol-channel-scheme
  by (rule hash-collision-budget-bind)
    (rule hash-range-budget-hash, rule hash-collision-budget-hash,
      rule hash-range-budget-return, rule hash-collision-budget-return)
  have hash-collision-budget (0 + (1 + 0))
    (get  $\gg$ 
      ( $\lambda$ s :: ('f, 'a) protocol-channel-scheme.
        hash (FiatShamirChallenge (PState s))  $\gg$  return))
  by (rule hash-collision-budget-bind
    [OF get-range get-coll hash-then-return-range
      hash-then-return-coll])
  then show ?thesis
  unfolding receive-random-field-element-def
    protocol-receive-random-field-element-def
    protocol-receive-tagged-random-field-element-def
  by simp
qed

lemma hash-collision-budget-receive-tagged-random-field-element:
  hash-collision-budget 1
  (receive-tagged-random-field-element tag :: ('f, 'f, 'a) protocol-c-monad)
proof -
  have get-range:
    hash-range-budget 0
    (get :: (('f, 'a) protocol-channel-scheme,
      ('f, 'a) protocol-channel-scheme) state-monad)
  by (rule hash-range-budget-get)
  have get-coll:
    hash-collision-budget 0
    (get :: (('f, 'a) protocol-channel-scheme,
      ('f, 'a) protocol-channel-scheme) state-monad)
  by (rule hash-collision-budget-get)
  have hash-then-return-range:
    hash-range-budget (1 + 0)
    (hash (tag (PState s))  $\gg$  return ::
      ('f, ('f, 'a) protocol-channel-scheme) state-monad)
  for s :: ('f, 'a) protocol-channel-scheme
  by (rule hash-range-budget-bind)
    (rule hash-range-budget-hash, rule hash-range-budget-return)
  have hash-then-return-coll:
    hash-collision-budget (1 + 0)
    (hash (tag (PState s))  $\gg$  return ::
      ('f, ('f, 'a) protocol-channel-scheme) state-monad)
  for s :: ('f, 'a) protocol-channel-scheme
  by (rule hash-collision-budget-bind)
    (rule hash-range-budget-hash, rule hash-collision-budget-hash,
      rule hash-range-budget-return, rule hash-collision-budget-return)

```

```

have hash-collision-budget ( $0 + (1 + 0)$ )
  (get  $\gg$ 
    ( $\lambda s :: ('f, 'a)$  protocol-channel-scheme.
      hash (tag (PState s))  $\gg$  return))
by (rule hash-collision-budget-bind
  [OF get-range get-coll hash-then-return-range
    hash-then-return-coll])
then show ?thesis
  unfolding receive-tagged-random-field-element-def
    protocol-receive-tagged-random-field-element-def
by simp
qed

lemma hash-collision-budget-receive-counted-tagged-random-field-element:
  assumes bump:  $\bigwedge s.$  HashMap (bump s) = HashMap s
  shows hash-collision-budget 1
    (protocol-receive-counted-tagged-random-field-element counter bump tag ::
      ('f, 'f, 'a) protocol-c-monad)
proof –
  have get-range:
    hash-range-budget 0
    (get :: (('f, 'a) protocol-channel-scheme,
      ('f, 'a) protocol-channel-scheme) state-monad)
by (rule hash-range-budget-get)
  have get-coll:
    hash-collision-budget 0
    (get :: (('f, 'a) protocol-channel-scheme,
      ('f, 'a) protocol-channel-scheme) state-monad)
by (rule hash-collision-budget-get)
  have modify-return-range:
    hash-range-budget ( $0 + 0$ )
    (modify bump  $\gg$  ( $\lambda -.$  return r) ::
      ('f, ('f, 'a) protocol-channel-scheme) state-monad)
for r :: 'f
by (rule hash-range-budget-bind
    (rule hash-range-budget-modify-preserves-hash-map[OF bump],
      rule hash-range-budget-return))
  have modify-return-coll:
    hash-collision-budget ( $0 + 0$ )
    (modify bump  $\gg$  ( $\lambda -.$  return r) ::
      ('f, ('f, 'a) protocol-channel-scheme) state-monad)
for r :: 'f
by (rule hash-collision-budget-bind
    (rule hash-range-budget-modify-preserves-hash-map[OF bump],
      rule hash-collision-budget-modify-preserves-hash-map[OF bump],
      rule hash-range-budget-return, rule hash-collision-budget-return))
  have hash-then-tail-range:
    hash-range-budget ( $1 + (0 + 0)$ )
    (hash (tag (counter s) (PState s))  $\gg$ 

```

```

    (λr. modify bump ≫ (λ-. return r)) ::
    ('f, ('f, 'a) protocol-channel-scheme) state-monad)
  for s :: ('f, 'a) protocol-channel-scheme
  by (rule hash-range-budget-bind)
    (rule hash-range-budget-hash, rule modify-return-range)
  have hash-then-tail-coll:
    hash-collision-budget (1 + (0 + 0))
    (hash (tag (counter s) (PState s)) ≫
    (λr. modify bump ≫ (λ-. return r)) ::
    ('f, ('f, 'a) protocol-channel-scheme) state-monad)
  for s :: ('f, 'a) protocol-channel-scheme
  by (rule hash-collision-budget-bind)
    (rule hash-range-budget-hash, rule hash-collision-budget-hash,
    rule modify-return-range, rule modify-return-coll)
  have hash-collision-budget (0 + (1 + (0 + 0)))
    (get ≫
    (λs :: ('f, 'a) protocol-channel-scheme.
    hash (tag (counter s) (PState s)) ≫
    (λr. modify bump ≫ (λ-. return r))))
  by (rule hash-collision-budget-bind
    [OF get-range get-coll hash-then-tail-range hash-then-tail-coll])
  then show ?thesis
    unfolding protocol-receive-counted-tagged-random-field-element-def
    by simp
qed

```

lemma hash-collision-budget-receive-alpha-challenge:
 hash-collision-budget 1
 (receive-alpha-challenge :: ('f, 'f, 'a) protocol-c-monad)
 unfolding receive-alpha-challenge-def
 by (rule hash-collision-budget-receive-counted-tagged-random-field-element) simp

lemma hash-collision-budget-receive-query-index-challenge:
 hash-collision-budget 1
 (receive-query-index-challenge :: ('f, 'f, 'a) protocol-c-monad)
 unfolding receive-query-index-challenge-def
 by (rule hash-collision-budget-receive-counted-tagged-random-field-element) simp

lemma hash-collision-budget-receive-trace-fri-challenge:
 hash-collision-budget 1
 (receive-trace-fri-challenge :: ('f, 'f, 'a) protocol-c-monad)
 unfolding receive-trace-fri-challenge-def
 by (rule hash-collision-budget-receive-counted-tagged-random-field-element) simp

lemma hash-collision-budget-receive-composition-fri-challenge:
 hash-collision-budget 1
 (receive-composition-fri-challenge :: ('f, 'f, 'a) protocol-c-monad)
 unfolding receive-composition-fri-challenge-def
 by (rule hash-collision-budget-receive-counted-tagged-random-field-element) simp

```

lemma hash-range-budget-ntimes:
  fixes  $m :: ('x, ('f, 'a) \text{protocol-channel-scheme}) \text{state-monad}$ 
  assumes  $m$ : hash-range-budget  $n$   $m$ 
  shows hash-range-budget  $(k * n)$   $(\text{ntimes } m \ k)$ 
  using  $m$ 
proof (induction  $k$ )
  case  $0$ 
  then show ?case
    by (simp add: hash-range-budget-return)
next
  case (Suc  $k$ )
  have tail: hash-range-budget  $(k * n)$   $(\text{ntimes } m \ k)$ 
    using Suc.IH Suc.prems by simp
  have cont:
    hash-range-budget  $(k * n + 0)$ 
       $(\text{ntimes } m \ k \gg (\lambda xs. \text{return } (x \# xs)))$  for  $x$ 
    by (rule hash-range-budget-bind[OF tail hash-range-budget-return])
  have hash-range-budget  $(n + (k * n + 0))$ 
     $(m \gg (\lambda x. \text{ntimes } m \ k \gg (\lambda xs. \text{return } (x \# xs))))$ 
    by (rule hash-range-budget-bind[OF Suc.prems cont])
  then show ?case
    by simp
qed

```

```

lemma hash-collision-budget-ntimes:
  fixes  $m :: ('x, ('f, 'a) \text{protocol-channel-scheme}) \text{state-monad}$ 
  assumes range-m: hash-range-budget  $n$   $m$ 
  and coll-m: hash-collision-budget  $n$   $m$ 
  shows hash-collision-budget  $(k * n)$   $(\text{ntimes } m \ k)$ 
  using range-m coll-m
proof (induction  $k$ )
  case  $0$ 
  then show ?case
    by (simp add: hash-collision-budget-return)
next
  case (Suc  $k$ )
  have tail-range: hash-range-budget  $(k * n)$   $(\text{ntimes } m \ k)$ 
    by (rule hash-range-budget-ntimes[OF Suc.prems(1)])
  have tail-coll: hash-collision-budget  $(k * n)$   $(\text{ntimes } m \ k)$ 
    using Suc.IH Suc.prems by simp
  have cont-range:
    hash-range-budget  $(k * n + 0)$ 
       $(\text{ntimes } m \ k \gg (\lambda xs. \text{return } (x \# xs)))$  for  $x$ 
    by (rule hash-range-budget-bind[OF tail-range hash-range-budget-return])
  have cont-coll:
    hash-collision-budget  $(k * n + 0)$ 
       $(\text{ntimes } m \ k \gg (\lambda xs. \text{return } (x \# xs)))$  for  $x$ 
    by (rule hash-collision-budget-bind

```

```

      [OF tail-range tail-coll hash-range-budget-return
        hash-collision-budget-return])
have hash-collision-budget (n + (k * n + 0))
  (m  $\gg$  ( $\lambda x. ntimes\ m\ k\ \gg$  ( $\lambda xs. return\ (x\ \# \ xs)$ )))
by (rule hash-collision-budget-bind
  [OF Suc.premis(1) Suc.premis(2) cont-range cont-coll])
then show ?case
  by simp
qed

lemma hash-range-budget-ntimes-read-bind:
fixes k :: 'f list  $\Rightarrow$ 
  ('y, ('f, 'a) protocol-channel-scheme) state-monad
assumes k:  $\bigwedge xs. length\ xs = n \implies hash-range-budget\ b\ (k\ xs)$ 
shows
  hash-range-budget b
  ((ntimes read n :: ('f list, ('f, 'a) protocol-channel-scheme) state-monad)
     $\gg$  k)
proof -
have read-range:
  hash-range-budget (n * 0)
  (ntimes read n :: ('f list, ('f, 'a) protocol-channel-scheme) state-monad)
by (rule hash-range-budget-ntimes[OF hash-range-budget-read])
have hash-range-budget (n * 0 + b)
  ((ntimes read n :: ('f list, ('f, 'a) protocol-channel-scheme) state-monad)
     $\gg$  k)
proof (rule hash-range-budget-bind-on-outcomes[OF read-range])
fix s xs t
assume out:
  Some (xs, t)  $\in$ 
  set-dist (execute
    (ntimes read n ::
      ('f list, ('f, 'a) protocol-channel-scheme) state-monad) s)
show hash-range-budget b (k xs)
  by (rule k) (use ntimes-read-any-outcome[OF out] in simp)
qed
then show ?thesis by simp
qed

lemma hash-collision-budget-ntimes-read-bind:
fixes k :: 'f list  $\Rightarrow$ 
  ('y, ('f, 'a) protocol-channel-scheme) state-monad
assumes range-k:  $\bigwedge xs. length\ xs = n \implies hash-range-budget\ b\ (k\ xs)$ 
  and coll-k:  $\bigwedge xs. length\ xs = n \implies hash-collision-budget\ b\ (k\ xs)$ 
shows
  hash-collision-budget b
  ((ntimes read n :: ('f list, ('f, 'a) protocol-channel-scheme) state-monad)
     $\gg$  k)
proof -

```

```

have read-range:
  hash-range-budget (n * 0)
    (ntimes read n :: ('f list, ('f, 'a) protocol-channel-scheme) state-monad)
  by (rule hash-range-budget-ntimes[OF hash-range-budget-read])
have read-coll:
  hash-collision-budget (n * 0)
    (ntimes read n :: ('f list, ('f, 'a) protocol-channel-scheme) state-monad)
  by (rule hash-collision-budget-ntimes
    [OF hash-range-budget-read hash-collision-budget-read])
have hash-collision-budget (n * 0 + b)
  ((ntimes read n :: ('f list, ('f, 'a) protocol-channel-scheme) state-monad)
     $\ggg$  k)
proof (rule hash-collision-budget-bind-on-outcomes
  [OF read-range read-coll])
  fix s xs t
  assume out:
    Some (xs, t)  $\in$ 
      set-dist (execute
        (ntimes read n ::
          ('f list, ('f, 'a) protocol-channel-scheme) state-monad) s)
  have len: length xs = n
    using ntimes-read-any-outcome[OF out] by simp
  show hash-range-budget b (k xs)
    by (rule range-k[OF len])
  show hash-collision-budget b (k xs)
    by (rule coll-k[OF len])
  qed
  then show ?thesis by simp
qed

lemma hash-range-budget-mmap:
  fixes ms :: ('x, ('f, 'a) protocol-channel-scheme) state-monad list
  assumes ms:  $\bigwedge m. m \in \text{set } ms \implies \text{hash-range-budget } n \ m$ 
  shows hash-range-budget (length ms * n) (mmap ms)
  using ms
proof (induction ms)
  case Nil
  then show ?case
    by (simp add: hash-range-budget-return)
next
  case (Cons m ms)
  have head: hash-range-budget n m
    using Cons.prem1 by simp
  have tail: hash-range-budget (length ms * n) (mmap ms)
    using Cons.IH Cons.prem1 by simp
  have cont:
    hash-range-budget (length ms * n + 0)
      (mmap ms  $\ggg$  ( $\lambda xs. \text{return } (x \# xs)$ )) for x
    by (rule hash-range-budget-bind[OF tail hash-range-budget-return])

```



```

have hash-range-budget (n + (length ms * n + 0))
  (m ≫ (λx. mmap ms ≫ (λxs. return (x # xs))))
  by (rule hash-range-budget-bind[OF head cont])
then show ?case
  by simp
qed

lemma hash-collision-budget-mmap:
  fixes ms :: ('x, ('f, 'a) protocol-channel-scheme) state-monad list
  assumes range-ms:  $\bigwedge m. m \in \text{set } ms \implies \text{hash-range-budget } n \ m$ 
  and coll-ms:  $\bigwedge m. m \in \text{set } ms \implies \text{hash-collision-budget } n \ m$ 
  shows hash-collision-budget (length ms * n) (mmap ms)
  using range-ms coll-ms
proof (induction ms)
  case Nil
  then show ?case
    by (simp add: hash-collision-budget-return)
next
  case (Cons m ms)
  have head-range: hash-range-budget n m
    using Cons.prem1 by simp
  have head-coll: hash-collision-budget n m
    using Cons.prem1 by simp
  have tail-range: hash-range-budget (length ms * n) (mmap ms)
    by (rule hash-range-budget-mmap) (use Cons.prem2 in auto)
  have tail-coll: hash-collision-budget (length ms * n) (mmap ms)
    using Cons.IH Cons.prem2 by simp
  have cont-range:
    hash-range-budget (length ms * n + 0)
    (mmap ms ≫ (λxs. return (x # xs))) for x
    by (rule hash-range-budget-bind[OF tail-range hash-range-budget-return])
  have cont-coll:
    hash-collision-budget (length ms * n + 0)
    (mmap ms ≫ (λxs. return (x # xs))) for x
    by (rule hash-collision-budget-bind
      [OF tail-range tail-coll hash-range-budget-return
        hash-collision-budget-return])
  have hash-collision-budget (n + (length ms * n + 0))
    (m ≫ (λx. mmap ms ≫ (λxs. return (x # xs))))
    by (rule hash-collision-budget-bind
      [OF head-range head-coll cont-range cont-coll])
  then show ?case
    by simp
qed

lemma hash-range-budget-mfold:
  fixes steps :: ('x  $\Rightarrow$  ('x, ('f, 'a) protocol-channel-scheme) state-monad) list
  assumes steps:  $\bigwedge \text{step } x. \text{step} \in \text{set steps} \implies$ 
    hash-range-budget n (step x)

```

```

shows hash-range-budget (length steps * n) (mfold x steps)
using steps
proof (induction steps arbitrary: x)
  case Nil
  then show ?case
    by (simp add: hash-range-budget-return)
next
case (Cons step steps)
have head: hash-range-budget n (step x)
  using Cons.prem by simp
have tail:  $\bigwedge y. \text{hash-range-budget } (length\ steps * n) (mfold\ y\ steps)$ 
  using Cons.IH Cons.prem by simp
have hash-range-budget (n + length steps * n)
  (step x  $\gg$  ( $\lambda y. mfold\ y\ steps$ ))
  by (rule hash-range-budget-bind[OF head tail])
then show ?case
  by simp
qed

lemma hash-collision-budget-mfold:
  fixes steps :: ('x  $\Rightarrow$  ('f, 'a) protocol-channel-scheme) state-monad) list
  assumes range-steps:  $\bigwedge step\ x. step \in set\ steps \Rightarrow$ 
    hash-range-budget n (step x)
  and coll-steps:  $\bigwedge step\ x. step \in set\ steps \Rightarrow$ 
    hash-collision-budget n (step x)
  shows hash-collision-budget (length steps * n) (mfold x steps)
  using range-steps coll-steps
proof (induction steps arbitrary: x)
  case Nil
  then show ?case
    by (simp add: hash-collision-budget-return)
next
case (Cons step steps)
have head-range: hash-range-budget n (step x)
  using Cons.prem by simp
have head-coll: hash-collision-budget n (step x)
  using Cons.prem by simp
have tail-range:
   $\bigwedge y. \text{hash-range-budget } (length\ steps * n) (mfold\ y\ steps)$ 
  by (rule hash-range-budget-mfold) (use Cons.prem in auto)
have tail-coll:
   $\bigwedge y. \text{hash-collision-budget } (length\ steps * n) (mfold\ y\ steps)$ 
  using Cons.IH Cons.prem by simp
have hash-collision-budget (n + length steps * n)
  (step x  $\gg$  ( $\lambda y. mfold\ y\ steps$ ))
  by (rule hash-collision-budget-bind
    [OF head-range head-coll tail-range tail-coll])
then show ?case
  by simp

```

qed

lemma *hash-range-budget-mfold-invariant:*

fixes *steps* :: ('x $\Rightarrow$ ('x, ('f, 'a) protocol-channel-scheme) state-monad) list

assumes *init*: $I\ x$

and *step-budget*:

$\bigwedge \text{step } y. \text{ step } \in \text{set steps} \implies I\ y \implies$
 $\text{hash-range-budget } n\ (\text{step } y)$

and *step-inv*:

$\bigwedge \text{step } y\ z\ s\ t. \text{ step } \in \text{set steps} \implies I\ y \implies$
 $\text{Some } (z, t) \in \text{set-dist } (\text{execute } (\text{step } y)\ s) \implies I\ z$

shows $\text{hash-range-budget } (\text{length steps} * n)\ (\text{mfold } x\ \text{steps})$

using *init step-budget step-inv*

proof (*induction steps arbitrary: x*)

case *Nil*

then show *?case*

by (*simp add: hash-range-budget-return*)

next

case (*Cons step steps*)

have *Ix*: $I\ x$

using *Cons.prem*s **by** *simp*

have *head*: $\text{hash-range-budget } n\ (\text{step } x)$

by (*rule Cons.prem*s(2)) (*use Ix in auto*)

have *tail*:

$\text{hash-range-budget } (\text{length steps} * n)\ (\text{mfold } y\ \text{steps})$

if *out*: $\text{Some } (y, t) \in \text{set-dist } (\text{execute } (\text{step } x)\ s)$

for $s\ y\ t$

proof (*rule Cons.IH*)

show $I\ y$

proof –

have *mem*: $\text{step} \in \text{set } (\text{step} \# \text{steps})$

by *simp*

have *inv*: $\bigwedge z\ s\ t.$

$\text{Some } (z, t) \in \text{set-dist } (\text{execute } (\text{step } x)\ s) \implies I\ z$

by (*rule Cons.prem*s(3)[*OF mem Ix*])

show *?thesis*

by (*rule inv*[*OF out*])

qed

show $\bigwedge \text{step } y. \text{ step } \in \text{set steps} \implies I\ y \implies$

$\text{hash-range-budget } n\ (\text{step } y)$

by (*rule Cons.prem*s(2)) *simp-all*

show $\bigwedge \text{step } y\ z\ s\ t.$

$\text{step} \in \text{set steps} \implies$

$I\ y \implies \text{Some } (z, t) \in \text{set-dist } (\text{execute } (\text{step } y)\ s) \implies I\ z$

by (*rule Cons.prem*s(3)) *simp-all*

qed

have $\text{hash-range-budget } (n + \text{length steps} * n)$

$(\text{step } x \gg (\lambda y. \text{mfold } y\ \text{steps}))$

by (*rule hash-range-budget-bind-on-outcomes*[*OF head*]) (*rule tail*)

```

then show ?case
  by simp
qed

lemma hash-collision-budget-mfold-invariant:
  fixes steps :: ('x  $\Rightarrow$  ('x, ('f, 'a) protocol-channel-scheme) state-monad) list
  assumes init:  $I\ x$ 
  and range-step:
     $\bigwedge \text{step } y. \text{step} \in \text{set steps} \Rightarrow I\ y \Rightarrow$ 
    hash-range-budget  $n\ (\text{step } y)$ 
  and coll-step:
     $\bigwedge \text{step } y. \text{step} \in \text{set steps} \Rightarrow I\ y \Rightarrow$ 
    hash-collision-budget  $n\ (\text{step } y)$ 
  and step-inv:
     $\bigwedge \text{step } y\ z\ s\ t. \text{step} \in \text{set steps} \Rightarrow I\ y \Rightarrow$ 
    Some  $(z, t) \in \text{set-dist } (\text{execute } (\text{step } y)\ s) \Rightarrow I\ z$ 
  shows hash-collision-budget  $(\text{length steps} * n)\ (\text{mfold } x\ \text{steps})$ 
  using init range-step coll-step step-inv
proof (induction steps arbitrary:  $x$ )
  case Nil
  then show ?case
    by (simp add: hash-collision-budget-return)
next
  case (Cons step steps)
  have  $Ix: I\ x$ 
  using Cons.prem1 by simp
  have head-range: hash-range-budget  $n\ (\text{step } x)$ 
  by (rule Cons.prem2) (use  $Ix$  in auto)
  have head-coll: hash-collision-budget  $n\ (\text{step } x)$ 
  by (rule Cons.prem3) (use  $Ix$  in auto)
  have tail-range:
    hash-range-budget  $(\text{length steps} * n)\ (\text{mfold } y\ \text{steps})$ 
  if out: Some  $(y, t) \in \text{set-dist } (\text{execute } (\text{step } x)\ s)$ 
  for  $s\ y\ t$ 
proof (rule hash-range-budget-mfold-invariant)
  show  $I\ y$ 
proof -
  have mem:  $\text{step} \in \text{set } (\text{step} \# \text{steps})$ 
  by simp
  have inv:  $\bigwedge z\ s\ t. \text{Some } (z, t) \in \text{set-dist } (\text{execute } (\text{step } x)\ s) \Rightarrow I\ z$ 
  by (rule Cons.prem4)[OF mem  $Ix$ ]
  show ?thesis
  by (rule inv[OF out])
qed
show  $\bigwedge \text{step } y. \text{step} \in \text{set steps} \Rightarrow I\ y \Rightarrow$ 
  hash-range-budget  $n\ (\text{step } y)$ 
  by (rule Cons.prem2) simp-all
show  $\bigwedge \text{step } y\ z\ s\ t. \text{Some } (z, t) \in \text{set-dist } (\text{execute } (\text{step } y)\ s) \Rightarrow I\ z$ 

```

```

    step ∈ set steps ⇒
    I y ⇒ Some (z, t) ∈ set-dist (execute (step y) s) ⇒ I z
    by (rule Cons.premis(4)) simp-all
qed
have tail-coll:
  hash-collision-budget (length steps * n) (mfold y steps)
  if out: Some (y, t) ∈ set-dist (execute (step x) s)
  for s y t
proof (rule Cons.IH)
  show I y
  proof -
    have mem: step ∈ set (step # steps)
    by simp
    have inv:  $\bigwedge z s t.$ 
      Some (z, t) ∈ set-dist (execute (step x) s) ⇒ I z
    by (rule Cons.premis(4)[OF mem It])
    show ?thesis
    by (rule inv[OF out])
  qed
  show  $\bigwedge \text{step } y. \text{step} \in \text{set steps} \Rightarrow I y \Rightarrow$ 
    hash-range-budget n (step y)
  by (rule Cons.premis(2)) simp-all
  show  $\bigwedge \text{step } y. \text{step} \in \text{set steps} \Rightarrow I y \Rightarrow$ 
    hash-collision-budget n (step y)
  by (rule Cons.premis(3)) simp-all
  show  $\bigwedge \text{step } y z s t.$ 
    step ∈ set steps ⇒
    I y ⇒ Some (z, t) ∈ set-dist (execute (step y) s) ⇒ I z
  by (rule Cons.premis(4)) simp-all
qed
have hash-collision-budget (n + length steps * n)
  (step x  $\gg$  ( $\lambda y. \text{mfold } y \text{ steps}$ ))
  by (rule hash-collision-budget-bind-on-outcomes
    [OF head-range head-coll tail-range tail-coll])
  then show ?case
  by simp
qed

lemma hash-range-budget-fri-state-mfold-invariant:
  fixes st :: nat × 'f × nat × nat
  and steps ::
    (nat × 'f × nat × nat ⇒
     (nat × 'f × nat × nat,
      ('f, 'a) protocol-channel-scheme) state-monad) list
  and I :: nat × 'f × nat × nat ⇒ bool
  assumes init: I st
  and step-budget:
     $\bigwedge \text{step } y. \text{step} \in \text{set steps} \Rightarrow I y \Rightarrow$ 
    hash-range-budget n (step y)

```

and *step-inv*:
 $\bigwedge \text{step } y \ z \ s \ t. \text{ step} \in \text{set steps} \implies I \ y \implies$
 $\text{Some } (z, t) \in \text{set-dist } (\text{execute } (\text{step } y) \ s) \implies I \ z$
shows *hash-range-budget* (length steps * n) (mfold st steps)
using *assms* **by** (rule hash-range-budget-mfold-invariant)

lemma *hash-collision-budget-fri-state-mfold-invariant*:
fixes *st* :: nat × 'f × nat × nat
and *steps* ::
 $(\text{nat} \times 'f \times \text{nat} \times \text{nat} \Rightarrow$
 $(\text{nat} \times 'f \times \text{nat} \times \text{nat},$
 $(f, a) \text{ protocol-channel-scheme}) \text{ state-monad}) \text{ list}$
and *I* :: nat × 'f × nat × nat ⇒ bool
assumes *init*: *I st*
and *range-step*:
 $\bigwedge \text{step } y. \text{ step} \in \text{set steps} \implies I \ y \implies$
 $\text{hash-range-budget } n \ (\text{step } y)$
and *coll-step*:
 $\bigwedge \text{step } y. \text{ step} \in \text{set steps} \implies I \ y \implies$
 $\text{hash-collision-budget } n \ (\text{step } y)$
and *step-inv*:
 $\bigwedge \text{step } y \ z \ s \ t. \text{ step} \in \text{set steps} \implies I \ y \implies$
 $\text{Some } (z, t) \in \text{set-dist } (\text{execute } (\text{step } y) \ s) \implies I \ z$
shows *hash-collision-budget* (length steps * n) (mfold st steps)
using *assms* **by** (rule hash-collision-budget-mfold-invariant)

lemma *hash-target-program-receive-fri-commits*:
 $\text{hash-target-program } B \ 1 \ \text{receive-fri-commits}$
proof –
have *tail*:
 $\text{hash-target-program } B \ (1 + 0)$
 $(\text{receive-random-field-element} \gg (\lambda b. \text{return } (b, r)))$
for *r*
by (rule hash-target-program-bind)
 $(\text{rule hash-target-program-receive-random-field-element},$
 $\text{rule hash-target-program-return})$
have $\text{hash-target-program } B \ (0 + (1 + 0))$
 $(\text{read} \gg$
 $(\lambda r. \text{receive-random-field-element} \gg$
 $(\lambda b. \text{return } (b, r))))$
by (rule hash-target-program-bind[OF hash-target-program-read tail])
then show *?thesis*
unfolding *receive-fri-commits-def* **by** *simp*
qed

lemma *hash-target-program-receive-fri-commits-with*:
assumes *challenge*: $\text{hash-target-program } B \ 1 \ \text{receive-challenge}$

shows *hash-target-program B 1*
(receive-fri-commits-with receive-challenge)
proof –
have *tail*:
hash-target-program B (1 + 0)
(receive-challenge $\gg$ (λb . return (b, r)))
for *r*
by (*rule hash-target-program-bind[OF challenge hash-target-program-return]*)
have *hash-target-program B (0 + (1 + 0))*
(read $\gg$
(λr . receive-challenge $\gg$
(λb . return (b, r))))
by (*rule hash-target-program-bind[OF hash-target-program-read tail]*)
then show *?thesis*
unfolding *receive-fri-commits-with-def* **by** *simp*
qed

lemma *hash-target-program-receive-trace-fri-commits*:
hash-target-program B 1 receive-trace-fri-commits
unfolding *receive-trace-fri-commits-def*
by (*rule hash-target-program-receive-fri-commits-with*)
(rule hash-target-program-receive-trace-fri-challenge)

lemma *hash-target-program-receive-composition-fri-commits*:
hash-target-program B 1 receive-composition-fri-commits
unfolding *receive-composition-fri-commits-def*
by (*rule hash-target-program-receive-fri-commits-with*)
(rule hash-target-program-receive-composition-fri-challenge)

lemma *hash-range-budget-receive-fri-commits*:
hash-range-budget 1 receive-fri-commits
proof –
have *read-then-random*:
hash-range-budget (0 + (1 + 0))
(read $\gg$
(λr . receive-random-field-element $\gg$
(λb . return (b, r))))
by (*rule hash-range-budget-bind*)
(rule hash-range-budget-read,
rule hash-range-budget-bind
[OF hash-range-budget-receive-random-field-element
hash-range-budget-return])
then show *?thesis*
unfolding *receive-fri-commits-def* **by** *simp*
qed

lemma *hash-collision-budget-receive-fri-commits*:
hash-collision-budget 1 receive-fri-commits
proof –

have *random-then-return-range*:
hash-range-budget $(1 + 0)$
 $(\text{receive-random-field-element} \gg (\lambda b. \text{return } (b, r)))$ **for** r
by (*rule hash-range-budget-bind*
 $[OF \text{ hash-range-budget-receive-random-field-element}$
 $\text{ hash-range-budget-return}]$)
have *random-then-return-coll*:
hash-collision-budget $(1 + 0)$
 $(\text{receive-random-field-element} \gg (\lambda b. \text{return } (b, r)))$ **for** r
by (*rule hash-collision-budget-bind*
 $[OF \text{ hash-range-budget-receive-random-field-element}$
 $\text{ hash-collision-budget-receive-random-field-element}$
 $\text{ hash-range-budget-return hash-collision-budget-return}]$)
have *hash-collision-budget* $(0 + (1 + 0))$
 $(\text{read} \gg$
 $(\lambda r. \text{receive-random-field-element} \gg$
 $(\lambda b. \text{return } (b, r))))$
by (*rule hash-collision-budget-bind*
 $[OF \text{ hash-range-budget-read hash-collision-budget-read}$
 $\text{ random-then-return-range random-then-return-coll}]$)
then show *?thesis*
unfolding *receive-fri-commits-def* **by** *simp*
qed

lemma *hash-range-budget-receive-fri-commits-with*:
assumes *challenge*: *hash-range-budget 1 receive-challenge*
shows *hash-range-budget 1 (receive-fri-commits-with receive-challenge)*
proof –
have *read-then-challenge*:
hash-range-budget $(0 + (1 + 0))$
 $(\text{read} \gg$
 $(\lambda r. \text{receive-challenge} \gg$
 $(\lambda b. \text{return } (b, r))))$
by (*rule hash-range-budget-bind*)
 $(\text{rule hash-range-budget-read},$
 $\text{rule hash-range-budget-bind } [OF \text{ challenge hash-range-budget-return}])$
then show *?thesis*
unfolding *receive-fri-commits-with-def* **by** *simp*
qed

lemma *hash-collision-budget-receive-fri-commits-with*:
assumes *challenge-range*: *hash-range-budget 1 receive-challenge*
and *challenge-collision*: *hash-collision-budget 1 receive-challenge*
shows *hash-collision-budget 1 (receive-fri-commits-with receive-challenge)*
proof –
have *challenge-then-return-range*:
hash-range-budget $(1 + 0)$
 $(\text{receive-challenge} \gg (\lambda b. \text{return } (b, r)))$ **for** r
by (*rule hash-range-budget-bind*


```

    [OF challenge-range hash-range-budget-return])
have challenge-then-return-coll:
  hash-collision-budget (1 + 0)
  (receive-challenge  $\gg$  ( $\lambda b.$  return (b, r))) for r
by (rule hash-collision-budget-bind
    [OF challenge-range challenge-collision
      hash-range-budget-return hash-collision-budget-return])
have hash-collision-budget (0 + (1 + 0))
  (read  $\gg$ 
    ( $\lambda r.$  receive-challenge  $\gg$ 
      ( $\lambda b.$  return (b, r))))
by (rule hash-collision-budget-bind
    [OF hash-range-budget-read hash-collision-budget-read
      challenge-then-return-range challenge-then-return-coll])
then show ?thesis
  unfolding receive-fri-commits-with-def by simp
qed

```

```

lemma hash-range-budget-receive-trace-fri-commits:
  hash-range-budget 1 receive-trace-fri-commits
unfolding receive-trace-fri-commits-def
by (rule hash-range-budget-receive-fri-commits-with)
  (rule hash-range-budget-receive-trace-fri-challenge)

```

```

lemma hash-collision-budget-receive-trace-fri-commits:
  hash-collision-budget 1 receive-trace-fri-commits
unfolding receive-trace-fri-commits-def
by (rule hash-collision-budget-receive-fri-commits-with)
  (rule hash-range-budget-receive-trace-fri-challenge,
    rule hash-collision-budget-receive-trace-fri-challenge)

```

```

lemma hash-range-budget-receive-composition-fri-commits:
  hash-range-budget 1 receive-composition-fri-commits
unfolding receive-composition-fri-commits-def
by (rule hash-range-budget-receive-fri-commits-with)
  (rule hash-range-budget-receive-composition-fri-challenge)

```

```

lemma hash-collision-budget-receive-composition-fri-commits:
  hash-collision-budget 1 receive-composition-fri-commits
unfolding receive-composition-fri-commits-def
by (rule hash-collision-budget-receive-fri-commits-with)
  (rule hash-range-budget-receive-composition-fri-challenge,
    rule hash-collision-budget-receive-composition-fri-challenge)

```

```

lemma hash-target-program-protocol-check-authentication-path-raw:
fixes leaf :: 'f protocol-hash-input
shows hash-target-program B (Suc (length path))
  (protocol-merkle.check-authentication-path len i leaf path ::
    ('f, ('f, 'a) protocol-channel-scheme) state-monad)

```

```

proof (induction path arbitrary: len i)
  case Nil
  have hash-target-program B 1
    (hash leaf :: ('f, ('f, 'a) protocol-channel-scheme) state-monad)
  by (rule hash-target-program-hash)
  then show ?case
    by (simp add: protocol-merkle.check-authentication-path.simps)
next
  case (Cons a path)
  show ?case
  proof (cases i < len div 2)
    case True
    have rec:
      hash-target-program B (Suc (length path))
      (protocol-merkle.check-authentication-path (len div 2) i leaf path ::
        ('f, ('f, 'a) protocol-channel-scheme) state-monad)
    using Cons.IH[of len div 2 i] by simp
    have bound:
      hash-target-program B (Suc (length path) + 1)
      ((protocol-merkle.check-authentication-path (len div 2) i leaf path ::
        ('f, ('f, 'a) protocol-channel-scheme) state-monad)  $\gg$ 
        ( $\lambda x$ . hash (MerkleNode x a)))
    by (rule hash-target-program-bind[OF rec hash-target-program-hash])
    show ?thesis
    using True bound
    by (simp add: protocol-merkle.check-authentication-path.simps)
  next
  case False
  have rec:
    hash-target-program B (Suc (length path))
    (protocol-merkle.check-authentication-path
      (len div 2) (i - len div 2) leaf path ::
      ('f, ('f, 'a) protocol-channel-scheme) state-monad)
  using Cons.IH[of len div 2 i - len div 2] by simp
  have bound:
    hash-target-program B (Suc (length path) + 1)
    ((protocol-merkle.check-authentication-path
      (len div 2) (i - len div 2) leaf path ::
      ('f, ('f, 'a) protocol-channel-scheme) state-monad)  $\gg$ 
      ( $\lambda x$ . hash (MerkleNode a x)))
  by (rule hash-target-program-bind[OF rec hash-target-program-hash])
  show ?thesis
  using False bound
  by (simp add: protocol-merkle.check-authentication-path.simps)
qed
qed

```

lemma hash-target-program-check-authentication-path:
 hash-target-program B (Suc (length path))

```

    (check-authentication-path len i v path)
unfolding check-authentication-path-def protocol-check-authentication-path-def
by (rule hash-target-program-protocol-check-authentication-path-raw)

lemma hash-target-program-check-authentication-path-assert-return:
  fixes path :: 'f list
    and v root :: 'f
    and r :: 'r
  shows
    hash-target-program B (Suc (length path))
      ((check-authentication-path len i v path ::
        ('f, ('f, 'a) protocol-channel-scheme) state-monad)  $\gg=$ 
        ( $\lambda$ ap. assert (ap = root)  $\gg=$  ( $\lambda$ -. return r)))
proof -
  have tail:
    hash-target-program B (0 + 0)
      (assert (ap = root)  $\gg=$  ( $\lambda$ -. return r) ::
        ('r, ('f, 'a) protocol-channel-scheme) state-monad)
    for ap
  by (rule hash-target-program-bind)
    (rule hash-target-program-assert, rule hash-target-program-return)
have hash-target-program B (Suc (length path) + (0 + 0))
      ((check-authentication-path len i v path ::
        ('f, ('f, 'a) protocol-channel-scheme) state-monad)  $\gg=$ 
        ( $\lambda$ ap. assert (ap = root)  $\gg=$  ( $\lambda$ -. return r)))
    by (rule hash-target-program-bind)
      (rule hash-target-program-check-authentication-path, rule tail)
  then show ?thesis by simp
qed

lemma hash-target-program-check-authentication-path-assert-bind:
  fixes path :: 'f list
    and v root :: 'f
    and k :: unit  $\Rightarrow$ 
      ('r, ('f, 'a) protocol-channel-scheme) state-monad
  assumes k:  $\bigwedge u$ . hash-target-program B n (k u)
  shows
    hash-target-program B (Suc (length path) + n)
      ((check-authentication-path len i v path ::
        ('f, ('f, 'a) protocol-channel-scheme) state-monad)  $\gg=$ 
        ( $\lambda$ ap. assert (ap = root)  $\gg=$  k))
proof -
  have tail:
    hash-target-program B (0 + n)
      (assert (ap = root)  $\gg=$  k ::
        ('r, ('f, 'a) protocol-channel-scheme) state-monad)
    for ap
  by (rule hash-target-program-bind[OF hash-target-program-assert k])
have hash-target-program B (Suc (length path) + (0 + n))

```

```

    ((check-authentication-path len i v path ::
      ('f, ('f, 'a) protocol-channel-scheme) state-monad) >>=
      (λap. assert (ap = root) >>= k))
  by (rule hash-target-program-bind)
    (rule hash-target-program-check-authentication-path, rule tail)
  then show ?thesis by simp
qed

lemma hash-target-program-fri-layer-opening-finish:
  assumes xp-path-len: length xp-path = floor-log len
  and xn-path-len: length xn-path = floor-log len
  shows
    hash-target-program B
      (Suc (floor-log len) + Suc (floor-log len))
      (fri-layer-opening-finish b f i x len pw xp xp-path xn xn-path)
proof -
  let ?H = Suc (floor-log len)
  let ?sidx = (i + len div 2) mod len
  let ?out =
    (i mod (len div 2),
     (xp + xn) div 2 +
     b * ((xp - xn) div (2 * ((h ^ i) * shift) ^ pw)),
     len div 2, pw + pw)
  have second:
    hash-target-program B ?H
      ((check-authentication-path len ?sidx xn xn-path ::
        ('f, ('f, 'a) protocol-channel-scheme) state-monad) >>=
        (λap. assert (ap = f) >>= (λ-. return ?out)))
  by (subst xn-path-len[symmetric])
    (rule hash-target-program-check-authentication-path-assert-return)
  have first0:
    hash-target-program B (Suc (length xp-path) + ?H)
      ((check-authentication-path len i xp xp-path ::
        ('f, ('f, 'a) protocol-channel-scheme) state-monad) >>=
        (λap. assert (ap = f) >>=
          (λ-.
            (check-authentication-path len ?sidx xn xn-path ::
              ('f, ('f, 'a) protocol-channel-scheme) state-monad) >>=
              (λap. assert (ap = f) >>= (λ-. return ?out))))))
  by (rule hash-target-program-check-authentication-path-assert-bind)
    (use second in simp)
  have first:
    hash-target-program B (?H + ?H)
      ((check-authentication-path len i xp xp-path ::
        ('f, ('f, 'a) protocol-channel-scheme) state-monad) >>=
        (λap. assert (ap = f) >>=
          (λ-.
            (check-authentication-path len ?sidx xn xn-path ::
              ('f, ('f, 'a) protocol-channel-scheme) state-monad) >>=
              (λap. assert (ap = f) >>= (λ-. return ?out))))))

```

```

      (λap. assert (ap = f) ≫= (λ-. return ?out))))))
using first0 xp-path-len by simp
have raw:
  hash-target-program B (0 + (?H + ?H))
  ((assert (xp = x) ::
    (unit, ('f, 'a) protocol-channel-scheme) state-monad) ≫=
    (λ-.
      (check-authentication-path len i xp xp-path ::
        ('f, ('f, 'a) protocol-channel-scheme) state-monad) ≫=
        (λap. assert (ap = f) ≫=
          (λ-.
            (check-authentication-path len ?sidx xn xn-path ::
              ('f, ('f, 'a) protocol-channel-scheme) state-monad) ≫=
              (λap. assert (ap = f) ≫= (λ-. return ?out)))))))
    by (rule hash-target-program-bind[OF hash-target-program-assert first])
show ?thesis
  using raw unfolding fri-layer-opening-finish-def
  by (simp add: Let-def)
qed

```

```

lemma hash-target-program-fri-layer-opening-step:
  hash-target-program B
  (Suc (floor-log len) + Suc (floor-log len))
  (fri-layer-opening-step (b, f) (i, x, len, pw))
proof -
  let ?N = Suc (floor-log len) + Suc (floor-log len)
  have after-xn-path:
    hash-target-program B ?N
    ((ntimes read (floor-log len) ::
      ('f list, ('f, 'a) protocol-channel-scheme) state-monad) ≫=
      (λxn-path.
        fri-layer-opening-finish b f i x len pw xp xp-path xn xn-path))
  if xp-path-len: length xp-path = floor-log len
  for xp xp-path xn
proof (rule hash-target-program-ntimes-read-bind)
  fix xn-path :: 'f list
  assume xn-path-len: length xn-path = floor-log len
  show hash-target-program B ?N
    (fri-layer-opening-finish b f i x len pw xp xp-path xn xn-path)
  by (rule hash-target-program-fri-layer-opening-finish
    [OF xp-path-len xn-path-len])
qed

```

```

have after-xn:
  hash-target-program B ?N
  (read ≫=
    (λxn.
      (ntimes read (floor-log len) ::
        ('f list, ('f, 'a) protocol-channel-scheme) state-monad) ≫=
        (λxn-path.

```

```

      fri-layer-opening-finish b f i x len pw xp xp-path xn xn-path)))
  if xp-path-len: length xp-path = floor-log len
  for xp xp-path
  proof -
    have hash-target-program B (0 + ?N)
      (read  $\gg$ 
        ( $\lambda$ xn.
          (ntimes read (floor-log len) ::
            ('f list, ('f, 'a) protocol-channel-scheme) state-monad)  $\gg$ 
            ( $\lambda$ xn-path.
              fri-layer-opening-finish b f i x len pw xp xp-path xn xn-path))))
    by (rule hash-target-program-bind
      [OF hash-target-program-read after-xn-path[OF xp-path-len]])
  then show ?thesis by simp
qed
have after-xp-path:
  hash-target-program B ?N
  ((ntimes read (floor-log len) ::
    ('f list, ('f, 'a) protocol-channel-scheme) state-monad)  $\gg$ 
    ( $\lambda$ xp-path.
      read  $\gg$ 
        ( $\lambda$ xn.
          (ntimes read (floor-log len) ::
            ('f list, ('f, 'a) protocol-channel-scheme) state-monad)  $\gg$ 
            ( $\lambda$ xn-path.
              fri-layer-opening-finish b f i x len pw xp xp-path xn xn-path))))))
  for xp
proof (rule hash-target-program-ntimes-read-bind)
  fix xp-path :: 'f list
  assume xp-path-len: length xp-path = floor-log len
  show hash-target-program B ?N
    (read  $\gg$ 
      ( $\lambda$ xn.
        (ntimes read (floor-log len) ::
          ('f list, ('f, 'a) protocol-channel-scheme) state-monad)  $\gg$ 
          ( $\lambda$ xn-path.
            fri-layer-opening-finish b f i x len pw xp xp-path xn xn-path))))
  by (rule after-xn[OF xp-path-len])
qed
have hash-target-program B (0 + ?N)
  (read  $\gg$ 
    ( $\lambda$ xp.
      (ntimes read (floor-log len) ::
        ('f list, ('f, 'a) protocol-channel-scheme) state-monad)  $\gg$ 
        ( $\lambda$ xp-path.
          read  $\gg$ 
            ( $\lambda$ xn.
              (ntimes read (floor-log len) ::
                ('f list, ('f, 'a) protocol-channel-scheme) state-monad)  $\gg$ 
                fri-layer-opening-finish b f i x len pw xp xp-path xn xn-path))))))

```

```

      (λxn-path.
        fri-layer-opening-finish b f i x len pw xp xp-path xn xn-path))))))
  by (rule hash-target-program-bind
      [OF hash-target-program-read after-xp-path])
  then show ?thesis
    unfolding fri-layer-opening-step-unfold-finish by simp
qed

lemma hash-range-budget-protocol-check-authentication-path-raw:
  fixes leaf :: 'f protocol-hash-input
  shows hash-range-budget (Suc (length path))
    (protocol-merkle.check-authentication-path len i leaf path ::
      ('f, ('f, 'a) protocol-channel-scheme) state-monad)
  proof (induction path arbitrary: len i)
    case Nil
    have hash-budget:
      hash-range-budget 1
      (hash leaf :: ('f, ('f, 'a) protocol-channel-scheme) state-monad)
    by (rule hash-range-budget-hash)
    then show ?case
      by (simp add: protocol-merkle.check-authentication-path.simps)
  next
    case (Cons a path)
    show ?case
    proof (cases i < len div 2)
      case True
      have rec:
        hash-range-budget (Suc (length path))
        (protocol-merkle.check-authentication-path (len div 2) i leaf path ::
          ('f, ('f, 'a) protocol-channel-scheme) state-monad)
      using Cons.IH[of len div 2 i] by simp
      have bound:
        hash-range-budget (Suc (length path) + 1)
        ((protocol-merkle.check-authentication-path (len div 2) i leaf path ::
          ('f, ('f, 'a) protocol-channel-scheme) state-monad) ≫=
          (λx. hash (MerkleNode x a)))
      by (rule hash-range-budget-bind[OF rec hash-range-budget-hash])
      show ?thesis
        using True bound
        by (simp add: protocol-merkle.check-authentication-path.simps)
    next
      case False
      have rec:
        hash-range-budget (Suc (length path))
        (protocol-merkle.check-authentication-path
          (len div 2) (i - len div 2) leaf path ::
          ('f, ('f, 'a) protocol-channel-scheme) state-monad)
      using Cons.IH[of len div 2 i - len div 2] by simp
      have bound:

```

```

    hash-range-budget (Suc (length path) + 1)
      ((protocol-merkle.check-authentication-path
        (len div 2) (i - len div 2) leaf path ::
        ('f, ('f, 'a) protocol-channel-scheme) state-monad) >>=
        (λx. hash (MerkleNode a x)))
    by (rule hash-range-budget-bind[OF rec hash-range-budget-hash])
  show ?thesis
    using False bound
    by (simp add: protocol-merkle.check-authentication-path.simps)
qed
qed

lemma hash-range-budget-check-authentication-path:
  hash-range-budget (Suc (length path))
    (check-authentication-path len i v path)
  unfolding check-authentication-path-def protocol-check-authentication-path-def
  by (rule hash-range-budget-protocol-check-authentication-path-raw)

lemma hash-collision-budget-protocol-check-authentication-path-raw:
  fixes leaf :: 'f protocol-hash-input
  shows hash-collision-budget (Suc (length path))
    (protocol-merkle.check-authentication-path len i leaf path ::
    ('f, ('f, 'a) protocol-channel-scheme) state-monad)
  proof (induction path arbitrary: len i)
    case Nil
    have hash-budget:
      hash-collision-budget 1
        (hash leaf :: ('f, ('f, 'a) protocol-channel-scheme) state-monad)
    by (rule hash-collision-budget-hash)
    then show ?case
      by (simp add: protocol-merkle.check-authentication-path.simps)
  next
    case (Cons a path)
    show ?case
    proof (cases i < len div 2)
      case True
      have rec-range:
        hash-range-budget (Suc (length path))
          (protocol-merkle.check-authentication-path (len div 2) i leaf path ::
          ('f, ('f, 'a) protocol-channel-scheme) state-monad)
      by (rule hash-range-budget-protocol-check-authentication-path-raw)
      have rec-coll:
        hash-collision-budget (Suc (length path))
          (protocol-merkle.check-authentication-path (len div 2) i leaf path ::
          ('f, ('f, 'a) protocol-channel-scheme) state-monad)
      using Cons.IH[of len div 2 i] by simp
      have bound:
        hash-collision-budget (Suc (length path) + 1)
          ((protocol-merkle.check-authentication-path (len div 2) i leaf path ::

```



```

      ('f, ('f, 'a) protocol-channel-scheme) state-monad) >>=
      (λx. hash (MerkleNode x a)))
  by (rule hash-collision-budget-bind
      [OF rec-range rec-coll hash-range-budget-hash
        hash-collision-budget-hash])
  show ?thesis
  using True bound
  by (simp add: protocol-merkle.check-authentication-path.simps)
next
case False
have rec-range:
  hash-range-budget (Suc (length path))
  (protocol-merkle.check-authentication-path
   (len div 2) (i - len div 2) leaf path ::
    ('f, ('f, 'a) protocol-channel-scheme) state-monad)
  by (rule hash-range-budget-protocol-check-authentication-path-raw)
have rec-coll:
  hash-collision-budget (Suc (length path))
  (protocol-merkle.check-authentication-path
   (len div 2) (i - len div 2) leaf path ::
    ('f, ('f, 'a) protocol-channel-scheme) state-monad)
  using Cons.IH[of len div 2 i - len div 2] by simp
have bound:
  hash-collision-budget (Suc (length path) + 1)
  ((protocol-merkle.check-authentication-path
   (len div 2) (i - len div 2) leaf path ::
    ('f, ('f, 'a) protocol-channel-scheme) state-monad) >>=
   (λx. hash (MerkleNode a x)))
  by (rule hash-collision-budget-bind
      [OF rec-range rec-coll hash-range-budget-hash
        hash-collision-budget-hash])
  show ?thesis
  using False bound
  by (simp add: protocol-merkle.check-authentication-path.simps)
qed
qed

lemma hash-collision-budget-check-authentication-path:
  hash-collision-budget (Suc (length path))
  (check-authentication-path len i v path)
  unfolding check-authentication-path-def protocol-check-authentication-path-def
  by (rule hash-collision-budget-protocol-check-authentication-path-raw)

lemma hash-range-budget-check-authentication-path-assert-return:
  fixes path :: 'f list
  and v root :: 'f
  and r :: 'r
  shows
  hash-range-budget (Suc (length path))

```

```

(((check-authentication-path len i v path ::
  ('f, ('f, 'a) protocol-channel-scheme) state-monad) >>=
  (λap. assert (ap = root) >>= (λ-. return r))))
proof -
  have check-budget:
    hash-range-budget (Suc (length path))
    (check-authentication-path len i v path ::
      ('f, ('f, 'a) protocol-channel-scheme) state-monad)
  by (rule hash-range-budget-check-authentication-path)
  have assert-return:
    hash-range-budget (0 + 0)
    (assert (ap = root) >>= (λ-. return r) ::
      ('r, ('f, 'a) protocol-channel-scheme) state-monad) for ap
  by (rule hash-range-budget-bind
    [OF hash-range-budget-assert hash-range-budget-return])
  have hash-range-budget (Suc (length path) + (0 + 0))
    (((check-authentication-path len i v path ::
      ('f, ('f, 'a) protocol-channel-scheme) state-monad) >>=
      (λap. assert (ap = root) >>= (λ-. return r))))
    by (rule hash-range-budget-bind[OF check-budget assert-return])
  then show ?thesis by simp
qed

```

lemma hash-collision-budget-check-authentication-path-assert-return:

```

fixes path :: 'f list
and v root :: 'f
and r :: 'r

```

shows

```

hash-collision-budget (Suc (length path))
  (((check-authentication-path len i v path ::
    ('f, ('f, 'a) protocol-channel-scheme) state-monad) >>=
    (λap. assert (ap = root) >>= (λ-. return r))))

```

proof -

```

have check-range:
  hash-range-budget (Suc (length path))
  (check-authentication-path len i v path ::
    ('f, ('f, 'a) protocol-channel-scheme) state-monad)
by (rule hash-range-budget-check-authentication-path)
have check-coll:
  hash-collision-budget (Suc (length path))
  (check-authentication-path len i v path ::
    ('f, ('f, 'a) protocol-channel-scheme) state-monad)
by (rule hash-collision-budget-check-authentication-path)
have assert-return-range:
  hash-range-budget (0 + 0)
  (assert (ap = root) >>= (λ-. return r) ::
    ('r, ('f, 'a) protocol-channel-scheme) state-monad) for ap
by (rule hash-range-budget-bind
  [OF hash-range-budget-assert hash-range-budget-return])

```

```

have assert-return-coll:
  hash-collision-budget (0 + 0)
    (assert (ap = root)  $\gg$  ( $\lambda$ -. return r) ::
      ('r, ('f, 'a) protocol-channel-scheme) state-monad) for ap
by (rule hash-collision-budget-bind
  [OF hash-range-budget-assert hash-collision-budget-assert
   hash-range-budget-return hash-collision-budget-return])
have hash-collision-budget (Suc (length path) + (0 + 0))
  (((check-authentication-path len i v path ::
    ('f, ('f, 'a) protocol-channel-scheme) state-monad)  $\gg$ 
    ( $\lambda$ ap. assert (ap = root)  $\gg$  ( $\lambda$ -. return r))))
by (rule hash-collision-budget-bind
  [OF check-range check-coll assert-return-range assert-return-coll])
then show ?thesis by simp
qed

```

```

lemma hash-range-budget-check-authentication-path-assert-bind:
  fixes path :: 'f list
  and v root :: 'f
  and k :: unit  $\Rightarrow$  ('r, ('f, 'a) protocol-channel-scheme) state-monad
assumes k:  $\bigwedge u$ . hash-range-budget n (k u)
shows
  hash-range-budget (Suc (length path) + n)
  (((check-authentication-path len i v path ::
    ('f, ('f, 'a) protocol-channel-scheme) state-monad)  $\gg$ 
    ( $\lambda$ ap. assert (ap = root)  $\gg$  k)))

```

```

proof –
  have check-budget:
    hash-range-budget (Suc (length path))
      (check-authentication-path len i v path ::
        ('f, ('f, 'a) protocol-channel-scheme) state-monad)
    by (rule hash-range-budget-check-authentication-path)
  have assert-cont:
    hash-range-budget (0 + n)
      (assert (ap = root)  $\gg$  k ::
        ('r, ('f, 'a) protocol-channel-scheme) state-monad) for ap
    by (rule hash-range-budget-bind[OF hash-range-budget-assert k])
  have hash-range-budget (Suc (length path) + (0 + n))
    (((check-authentication-path len i v path ::
      ('f, ('f, 'a) protocol-channel-scheme) state-monad)  $\gg$ 
      ( $\lambda$ ap. assert (ap = root)  $\gg$  k)))
    by (rule hash-range-budget-bind[OF check-budget assert-cont])
  then show ?thesis by simp
qed

```

```

lemma hash-collision-budget-check-authentication-path-assert-bind:
  fixes path :: 'f list
  and v root :: 'f
  and k :: unit  $\Rightarrow$  ('r, ('f, 'a) protocol-channel-scheme) state-monad

```

```

assumes range-k:  $\bigwedge u. \text{hash-range-budget } n \ (k \ u)$ 
and coll-k:  $\bigwedge u. \text{hash-collision-budget } n \ (k \ u)$ 
shows
  hash-collision-budget (Suc (length path) + n)
    (((check-authentication-path len i v path ::
      ('f, ('f, 'a) protocol-channel-scheme) state-monad)  $\ggg$ 
      ( $\lambda ap. \text{assert } (ap = \text{root}) \ggg k$ )))
proof -
  have check-range:
    hash-range-budget (Suc (length path))
      (check-authentication-path len i v path ::
        ('f, ('f, 'a) protocol-channel-scheme) state-monad)
  by (rule hash-range-budget-check-authentication-path)
  have check-coll:
    hash-collision-budget (Suc (length path))
      (check-authentication-path len i v path ::
        ('f, ('f, 'a) protocol-channel-scheme) state-monad)
  by (rule hash-collision-budget-check-authentication-path)
  have assert-range:
    hash-range-budget (0 + n)
      (assert (ap = root)  $\ggg k$  ::
        ('r, ('f, 'a) protocol-channel-scheme) state-monad) for ap
  by (rule hash-range-budget-bind[OF hash-range-budget-assert range-k])
  have assert-coll:
    hash-collision-budget (0 + n)
      (assert (ap = root)  $\ggg k$  ::
        ('r, ('f, 'a) protocol-channel-scheme) state-monad) for ap
  by (rule hash-collision-budget-bind
    [OF hash-range-budget-assert hash-collision-budget-assert
      range-k coll-k])
  have hash-collision-budget (Suc (length path) + (0 + n))
    (((check-authentication-path len i v path ::
      ('f, ('f, 'a) protocol-channel-scheme) state-monad)  $\ggg$ 
      ( $\lambda ap. \text{assert } (ap = \text{root}) \ggg k$ )))
  by (rule hash-collision-budget-bind
    [OF check-range check-coll assert-range assert-coll])
  then show ?thesis by simp
qed

```

```

lemma hash-range-budget-fri-layer-opening-finish:
assumes xp-path-len: length xp-path = floor-log len
and xn-path-len: length xn-path = floor-log len
shows
  hash-range-budget (Suc (floor-log len) + Suc (floor-log len))
    (fri-layer-opening-finish b f i x len pw xp xp-path xn xn-path)
proof -
  let ?b = Suc (floor-log len)
  let ?sidx = (i + len div 2) mod len
  let ?out =

```

```

    (i mod (len div 2),
      (xp + xn) div 2 +
        b * ((xp - xn) div (2 * ((h ^ i) * shift) ^ pw)),
      len div 2, pw + pw)
have second:
  hash-range-budget ?b
  (((check-authentication-path len ?sidx xn xn-path ::
    ('f, ('f, 'a) protocol-channel-scheme) state-monad) >>=
    (λap. assert (ap = f) >>= (λ-. return ?out))))
  by (subst xn-path-len[symmetric])
    (rule hash-range-budget-check-authentication-path-assert-return)
have second-cont:
  ∧u. hash-range-budget ?b
  (((check-authentication-path len ?sidx xn xn-path ::
    ('f, ('f, 'a) protocol-channel-scheme) state-monad) >>=
    (λap. assert (ap = f) >>= (λ-. return ?out))))
  using second by simp
have first0:
  hash-range-budget (Suc (length xp-path) + ?b)
  (((check-authentication-path len i xp xp-path ::
    ('f, ('f, 'a) protocol-channel-scheme) state-monad) >>=
    (λap. assert (ap = f) >>=
      (λ-.
        (check-authentication-path len ?sidx xn xn-path ::
          ('f, ('f, 'a) protocol-channel-scheme) state-monad) >>=
          (λap. assert (ap = f) >>= (λ-. return ?out)))))))
  by (rule hash-range-budget-check-authentication-path-assert-bind
    [OF second-cont])
have first:
  hash-range-budget (?b + ?b)
  (((check-authentication-path len i xp xp-path ::
    ('f, ('f, 'a) protocol-channel-scheme) state-monad) >>=
    (λap. assert (ap = f) >>=
      (λ-.
        (check-authentication-path len ?sidx xn xn-path ::
          ('f, ('f, 'a) protocol-channel-scheme) state-monad) >>=
          (λap. assert (ap = f) >>= (λ-. return ?out)))))))
  using first0 xp-path-len by simp
have raw:
  hash-range-budget (0 + (?b + ?b))
  ((assert (xp = x) ::
    (unit, ('f, 'a) protocol-channel-scheme) state-monad) >>=
    (λ-.
      (check-authentication-path len i xp xp-path ::
        ('f, ('f, 'a) protocol-channel-scheme) state-monad) >>=
        (λap. assert (ap = f) >>=
          (λ-.
            (check-authentication-path len ?sidx xn xn-path ::
              ('f, ('f, 'a) protocol-channel-scheme) state-monad) >>=
              (λap. assert (ap = f) >>= (λ-. return ?out))))))))

```

```

      (λap. assert (ap = f) ≫ (λ-. return ?out))))))
  by (rule hash-range-budget-bind[OF hash-range-budget-assert first])
have all:
  hash-range-budget (?b + ?b)
  ((assert (xp = x) ::
    (unit, ('f, 'a) protocol-channel-scheme) state-monad) ≫=
    (λ-.
      (check-authentication-path len i xp xp-path ::
        ('f, ('f, 'a) protocol-channel-scheme) state-monad) ≫=
        (λap. assert (ap = f) ≫=
          (λ-.
            (check-authentication-path len ?sidx xn xn-path ::
              ('f, ('f, 'a) protocol-channel-scheme) state-monad) ≫=
              (λap. assert (ap = f) ≫ (λ-. return ?out)))))))
  using raw by simp
from all show ?thesis
  unfolding fri-layer-opening-finish-def
  by (simp add: Let-def)
qed

```

lemma *hash-collision-budget-fri-layer-opening-finish:*
assumes *xp-path-len:* $\text{length } xp\text{-path} = \text{floor-log } len$
and *xn-path-len:* $\text{length } xn\text{-path} = \text{floor-log } len$
shows
 $\text{hash-collision-budget } (\text{Suc } (\text{floor-log } len) + \text{Suc } (\text{floor-log } len))$
 $(\text{fri-layer-opening-finish } b \ f \ i \ x \ len \ pw \ xp \ xp\text{-path } xn \ xn\text{-path})$
proof –
let $?b = \text{Suc } (\text{floor-log } len)$
let $?sidx = (i + len \text{ div } 2) \text{ mod } len$
let $?out =$
 $(i \text{ mod } (len \text{ div } 2),$
 $(xp + xn) \text{ div } 2 +$
 $b * ((xp - xn) \text{ div } (2 * ((h \wedge i) * shift) \wedge pw)),$
 $len \text{ div } 2, pw + pw)$
have *second-range:*
 $\text{hash-range-budget } ?b$
 $((\text{check-authentication-path } len \ ?sidx \ xn \ xn\text{-path} ::$
 $(\text{'f}, (\text{'f}, \text{'a}) \text{ protocol-channel-scheme}) \text{ state-monad}) \gg=$
 $(\lambda ap. \text{assert } (ap = f) \gg (\lambda -. \text{return } ?out))))$
by (*subst xn-path-len[symmetric]*)
 $(\text{rule hash-range-budget-check-authentication-path-assert-return})$
have *second-coll:*
 $\text{hash-collision-budget } ?b$
 $((\text{check-authentication-path } len \ ?sidx \ xn \ xn\text{-path} ::$
 $(\text{'f}, (\text{'f}, \text{'a}) \text{ protocol-channel-scheme}) \text{ state-monad}) \gg=$
 $(\lambda ap. \text{assert } (ap = f) \gg (\lambda -. \text{return } ?out))))$
by (*subst xn-path-len[symmetric]*)
 $(\text{rule hash-collision-budget-check-authentication-path-assert-return})$
have *second-cont-range:*

```

 $\wedge$ u. hash-range-budget ?b
  (((check-authentication-path len ?sidx xn xn-path ::
    ('f, ('f, 'a) protocol-channel-scheme) state-monad)  $\gg$ 
    ( $\lambda$ ap. assert (ap = f)  $\gg$  ( $\lambda$ -. return ?out))))
  using second-range by simp
have second-cont-coll:
 $\wedge$ u. hash-collision-budget ?b
  (((check-authentication-path len ?sidx xn xn-path ::
    ('f, ('f, 'a) protocol-channel-scheme) state-monad)  $\gg$ 
    ( $\lambda$ ap. assert (ap = f)  $\gg$  ( $\lambda$ -. return ?out))))
  using second-coll by simp
have first-range0:
  hash-range-budget (Suc (length xp-path) + ?b)
  (((check-authentication-path len i xp xp-path ::
    ('f, ('f, 'a) protocol-channel-scheme) state-monad)  $\gg$ 
    ( $\lambda$ ap. assert (ap = f)  $\gg$ 
      ( $\lambda$ -.
        (check-authentication-path len ?sidx xn xn-path ::
          ('f, ('f, 'a) protocol-channel-scheme) state-monad)  $\gg$ 
          ( $\lambda$ ap. assert (ap = f)  $\gg$  ( $\lambda$ -. return ?out)))))))
  by (rule hash-range-budget-check-authentication-path-assert-bind
    [OF second-cont-range])
have first-coll0:
  hash-collision-budget (Suc (length xp-path) + ?b)
  (((check-authentication-path len i xp xp-path ::
    ('f, ('f, 'a) protocol-channel-scheme) state-monad)  $\gg$ 
    ( $\lambda$ ap. assert (ap = f)  $\gg$ 
      ( $\lambda$ -.
        (check-authentication-path len ?sidx xn xn-path ::
          ('f, ('f, 'a) protocol-channel-scheme) state-monad)  $\gg$ 
          ( $\lambda$ ap. assert (ap = f)  $\gg$  ( $\lambda$ -. return ?out)))))))
  by (rule hash-collision-budget-check-authentication-path-assert-bind
    [OF second-cont-range second-cont-coll])
have first-range:
  hash-range-budget (?b + ?b)
  (((check-authentication-path len i xp xp-path ::
    ('f, ('f, 'a) protocol-channel-scheme) state-monad)  $\gg$ 
    ( $\lambda$ ap. assert (ap = f)  $\gg$ 
      ( $\lambda$ -.
        (check-authentication-path len ?sidx xn xn-path ::
          ('f, ('f, 'a) protocol-channel-scheme) state-monad)  $\gg$ 
          ( $\lambda$ ap. assert (ap = f)  $\gg$  ( $\lambda$ -. return ?out)))))))
  using first-range0 xp-path-len by simp
have first-coll:
  hash-collision-budget (?b + ?b)
  (((check-authentication-path len i xp xp-path ::
    ('f, ('f, 'a) protocol-channel-scheme) state-monad)  $\gg$ 
    ( $\lambda$ ap. assert (ap = f)  $\gg$ 
      ( $\lambda$ -.

```

```

      (check-authentication-path len ?sidx xn xn-path ::
        ('f, ('f, 'a) protocol-channel-scheme) state-monad) >>=
      (λap. assert (ap = f) >>= (λ-. return ?out))))))
  using first-coll0 xp-path-len by simp
have raw:
  hash-collision-budget (0 + (?b + ?b))
  ((assert (xp = x) ::
    (unit, ('f, 'a) protocol-channel-scheme) state-monad) >>=
    (λ-.
      (check-authentication-path len i xp xp-path ::
        ('f, ('f, 'a) protocol-channel-scheme) state-monad) >>=
      (λap. assert (ap = f) >>=
        (λ-.
          (check-authentication-path len ?sidx xn xn-path ::
            ('f, ('f, 'a) protocol-channel-scheme) state-monad) >>=
            (λap. assert (ap = f) >>= (λ-. return ?out)))))))
  by (rule hash-collision-budget-bind
    [OF hash-range-budget-assert hash-collision-budget-assert
      first-range first-coll])
then show ?thesis
  unfolding fri-layer-opening-finish-def
  by (simp add: Let-def)
qed

```

lemma hash-range-budget-fri-layer-opening-step:

```

  hash-range-budget (Suc (floor-log len) + Suc (floor-log len))
  (fri-layer-opening-step (b, f) (i, x, len, pw))

```

proof –

```

  let ?B = Suc (floor-log len) + Suc (floor-log len)
  have after-xn-path:
    hash-range-budget ?B
    ((ntimes read (floor-log len) ::
      ('f list, ('f, 'a) protocol-channel-scheme) state-monad) >>=
      (λxn-path.
        fri-layer-opening-finish b f i x len pw xp xp-path xn xn-path))
  if xp-path-len: length xp-path = floor-log len
  for xp xp-path xn

```

proof (rule hash-range-budget-ntimes-read-bind)

```

  fix xn-path :: 'f list
  assume xn-path-len: length xn-path = floor-log len
  show hash-range-budget ?B
    (fri-layer-opening-finish b f i x len pw xp xp-path xn xn-path)
  by (rule hash-range-budget-fri-layer-opening-finish
    [OF xp-path-len xn-path-len])

```

qed

have after-xn:

```

  hash-range-budget ?B
  (read >>=
    (λxn.

```



```

    (ntimes read (floor-log len) ::
      ('f list, ('f, 'a) protocol-channel-scheme) state-monad) >>=
    (λxn-path.
      fri-layer-opening-finish b f i x len pw xp xp-path xn xn-path)))
  if xp-path-len: length xp-path = floor-log len
  for xp xp-path
proof -
  have hash-range-budget (0 + ?B)
    (read >>=
      (λxn.
        (ntimes read (floor-log len) ::
          ('f list, ('f, 'a) protocol-channel-scheme) state-monad) >>=
          (λxn-path.
            fri-layer-opening-finish b f i x len pw xp xp-path xn xn-path))))
    by (rule hash-range-budget-bind
      [OF hash-range-budget-read after-xn-path[OF xp-path-len]])
  then show ?thesis by simp
qed
have after-xp-path:
  hash-range-budget ?B
  ((ntimes read (floor-log len) ::
    ('f list, ('f, 'a) protocol-channel-scheme) state-monad) >>=
    (λxp-path.
      read >>=
      (λxn.
        (ntimes read (floor-log len) ::
          ('f list, ('f, 'a) protocol-channel-scheme) state-monad) >>=
          (λxn-path.
            fri-layer-opening-finish b f i x len pw xp xp-path xn xn-path))))))
  for xp
proof (rule hash-range-budget-ntimes-read-bind)
  fix xp-path :: 'f list
  assume xp-path-len: length xp-path = floor-log len
  show hash-range-budget ?B
    (read >>=
      (λxn.
        (ntimes read (floor-log len) ::
          ('f list, ('f, 'a) protocol-channel-scheme) state-monad) >>=
          (λxn-path.
            fri-layer-opening-finish b f i x len pw xp xp-path xn xn-path))))
    by (rule after-xn[OF xp-path-len])
qed
have hash-range-budget (0 + ?B)
  (read >>=
    (λxp.
      (ntimes read (floor-log len) ::
        ('f list, ('f, 'a) protocol-channel-scheme) state-monad) >>=
        (λxp-path.
          read >>=
            (λxn.
              (ntimes read (floor-log len) ::
                ('f list, ('f, 'a) protocol-channel-scheme) state-monad) >>=
                (λxn-path.
                  fri-layer-opening-finish b f i x len pw xp xp-path xn xn-path))))))))

```

```

      (λxn.
        (ntimes read (floor-log len) ::
          ('f list, ('f, 'a) protocol-channel-scheme) state-monad) >>=
          (λxn-path.
            fri-layer-opening-finish b f i x len pw xp xp-path xn xn-path))))
    by (rule hash-range-budget-bind
      [OF hash-range-budget-read after-xp-path])
  then show ?thesis
    unfolding fri-layer-opening-step-unfold-finish by simp
qed

```

lemma hash-collision-budget-fri-layer-opening-step:
 hash-collision-budget (Suc (floor-log len) + Suc (floor-log len))
 (fri-layer-opening-step (b, f) (i, x, len, pw))

proof –
 let ?B = Suc (floor-log len) + Suc (floor-log len)
 have after-xn-path-range:
 hash-range-budget ?B
 ((ntimes read (floor-log len) ::
 ('f list, ('f, 'a) protocol-channel-scheme) state-monad) >>=
 (λxn-path.
 fri-layer-opening-finish b f i x len pw xp xp-path xn xn-path))
 if xp-path-len: length xp-path = floor-log len
 for xp xp-path xn
proof (rule hash-range-budget-ntimes-read-bind)
 fix xn-path :: 'f list
 assume xn-path-len: length xn-path = floor-log len
 show hash-range-budget ?B
 (fri-layer-opening-finish b f i x len pw xp xp-path xn xn-path)
 by (rule hash-range-budget-fri-layer-opening-finish
 [OF xp-path-len xn-path-len])

qed

have after-xn-path-coll:
 hash-collision-budget ?B
 ((ntimes read (floor-log len) ::
 ('f list, ('f, 'a) protocol-channel-scheme) state-monad) >>=
 (λxn-path.
 fri-layer-opening-finish b f i x len pw xp xp-path xn xn-path))
 if xp-path-len: length xp-path = floor-log len
 for xp xp-path xn
proof (rule hash-collision-budget-ntimes-read-bind)
 fix xn-path :: 'f list
 assume xn-path-len: length xn-path = floor-log len
 show hash-range-budget ?B
 (fri-layer-opening-finish b f i x len pw xp xp-path xn xn-path)
 by (rule hash-range-budget-fri-layer-opening-finish
 [OF xp-path-len xn-path-len])
 show hash-collision-budget ?B
 (fri-layer-opening-finish b f i x len pw xp xp-path xn xn-path)

```

    by (rule hash-collision-budget-fri-layer-opening-finish
        [OF xp-path-len xn-path-len])
qed
have after-xn-range:
  hash-range-budget ?B
  (read  $\gg$ 
    ( $\lambda xn.$ 
      (ntimes read (floor-log len) ::
        ('f list, ('f, 'a) protocol-channel-scheme) state-monad)  $\gg$ 
      ( $\lambda xn$ -path.
        fri-layer-opening-finish b f i x len pw xp xp-path xn xn-path)))
  if xp-path-len: length xp-path = floor-log len
  for xp xp-path
proof -
  have hash-range-budget (0 + ?B)
    (read  $\gg$ 
      ( $\lambda xn.$ 
        (ntimes read (floor-log len) ::
          ('f list, ('f, 'a) protocol-channel-scheme) state-monad)  $\gg$ 
          ( $\lambda xn$ -path.
            fri-layer-opening-finish b f i x len pw xp xp-path xn xn-path)))
    by (rule hash-range-budget-bind
        [OF hash-range-budget-read after-xn-path-range[OF xp-path-len]])
  then show ?thesis by simp
qed
have after-xn-coll:
  hash-collision-budget ?B
  (read  $\gg$ 
    ( $\lambda xn.$ 
      (ntimes read (floor-log len) ::
        ('f list, ('f, 'a) protocol-channel-scheme) state-monad)  $\gg$ 
      ( $\lambda xn$ -path.
        fri-layer-opening-finish b f i x len pw xp xp-path xn xn-path)))
  if xp-path-len: length xp-path = floor-log len
  for xp xp-path
proof -
  have hash-collision-budget (0 + ?B)
    (read  $\gg$ 
      ( $\lambda xn.$ 
        (ntimes read (floor-log len) ::
          ('f list, ('f, 'a) protocol-channel-scheme) state-monad)  $\gg$ 
          ( $\lambda xn$ -path.
            fri-layer-opening-finish b f i x len pw xp xp-path xn xn-path)))
    by (rule hash-collision-budget-bind
        [OF hash-range-budget-read hash-collision-budget-read
          after-xn-path-range[OF xp-path-len]
          after-xn-path-coll[OF xp-path-len]])
  then show ?thesis by simp
qed

```

```

have after-xp-path-range:
  hash-range-budget ?B
  ((ntimes read (floor-log len) ::
    ('f list, ('f, 'a) protocol-channel-scheme) state-monad)  $\gg$ 
    ( $\lambda$ xp-path.
      read  $\gg$ 
      ( $\lambda$ xn.
        (ntimes read (floor-log len) ::
          ('f list, ('f, 'a) protocol-channel-scheme) state-monad)  $\gg$ 
          ( $\lambda$ xn-path.
            fri-layer-opening-finish b f i x len pw xp xp-path xn xn-path))))))
for xp
proof (rule hash-range-budget-ntimes-read-bind)
fix xp-path :: 'f list
assume xp-path-len: length xp-path = floor-log len
show hash-range-budget ?B
  (read  $\gg$ 
    ( $\lambda$ xn.
      (ntimes read (floor-log len) ::
        ('f list, ('f, 'a) protocol-channel-scheme) state-monad)  $\gg$ 
        ( $\lambda$ xn-path.
          fri-layer-opening-finish b f i x len pw xp xp-path xn xn-path))))
by (rule after-xn-range[OF xp-path-len])
qed
have after-xp-path-coll:
  hash-collision-budget ?B
  ((ntimes read (floor-log len) ::
    ('f list, ('f, 'a) protocol-channel-scheme) state-monad)  $\gg$ 
    ( $\lambda$ xp-path.
      read  $\gg$ 
      ( $\lambda$ xn.
        (ntimes read (floor-log len) ::
          ('f list, ('f, 'a) protocol-channel-scheme) state-monad)  $\gg$ 
          ( $\lambda$ xn-path.
            fri-layer-opening-finish b f i x len pw xp xp-path xn xn-path))))))
for xp
proof (rule hash-collision-budget-ntimes-read-bind)
fix xp-path :: 'f list
assume xp-path-len: length xp-path = floor-log len
show hash-range-budget ?B
  (read  $\gg$ 
    ( $\lambda$ xn.
      (ntimes read (floor-log len) ::
        ('f list, ('f, 'a) protocol-channel-scheme) state-monad)  $\gg$ 
        ( $\lambda$ xn-path.
          fri-layer-opening-finish b f i x len pw xp xp-path xn xn-path))))
by (rule after-xn-range[OF xp-path-len])
show hash-collision-budget ?B
  (read  $\gg$ 

```

```

    (λxn.
      (ntimes read (floor-log len) ::
        ('f list, ('f, 'a) protocol-channel-scheme) state-monad) >>=
      (λxp-path.
        fri-layer-opening-finish b f i x len pw xp xp-path xn xn-path)))
  by (rule after-xn-coll[OF xp-path-len])
qed
have hash-collision-budget (0 + ?B)
  (read >>=
    (λxp.
      (ntimes read (floor-log len) ::
        ('f list, ('f, 'a) protocol-channel-scheme) state-monad) >>=
      (λxp-path.
        read >>=
        (λxn.
          (ntimes read (floor-log len) ::
            ('f list, ('f, 'a) protocol-channel-scheme) state-monad) >>=
            (λxn-path.
              fri-layer-opening-finish b f i x len pw xp xp-path xn xn-path))))))
  by (rule hash-collision-budget-bind
    [OF hash-range-budget-read hash-collision-budget-read
      after-xp-path-range after-xp-path-coll])
then show ?thesis
  unfolding fri-layer-opening-step-unfold-finish by simp
qed

```

lemma *floor-log-div2-le-self*:
 $\text{floor-log } (n \text{ div } 2) \leq \text{floor-log } n$
 by (rule floor-log-le-iff) simp

lemma *receive-query-commits-eq-fri-layer-opening-steps*:
 $\text{receive-query-commits fl} = \text{map fri-layer-opening-step fl}$
unfolding *receive-query-commits-def fri-layer-opening-step-def*
 by (simp add: fun-eq-iff split-def)

lemma *hash-target-program-fri-layer-opening-steps-mfold-state*:
fixes $st :: \text{nat} \times 'f \times \text{nat} \times \text{nat}$
assumes *init*:
 $\text{case } st \text{ of } (i, x, \text{len}, pw) \Rightarrow \text{floor-log len} \leq L$
shows
 $\text{hash-target-program } B$
 $(\text{length fl} * (\text{Suc } L + \text{Suc } L))$
 $(\text{mfold } st (\text{map fri-layer-opening-step fl}) ::$
 $(\text{nat} \times 'f \times \text{nat} \times \text{nat},$
 $(\text{'f}, \text{'a}) \text{ protocol-channel-scheme}) \text{ state-monad})$
using *init*
proof (induction fl arbitrary: st)
case *Nil*
then show ?case

```

    by (simp add: hash-target-program-return)
next
case (Cons bf fl)
obtain i x len pw where st-eq: st = (i, x, len, pw)
  by (cases st)
obtain b f where bf-eq: bf = (b, f)
  by (cases bf)
let ?N = Suc L + Suc L
have head-exact:
  hash-target-program B
    (Suc (floor-log len) + Suc (floor-log len))
    (fri-layer-opening-step (b, f) (i, x, len, pw))
  by (rule hash-target-program-fri-layer-opening-step)
have head-le:
  Suc (floor-log len) + Suc (floor-log len) ≤ ?N
  using Cons.prem1 unfolding st-eq by simp
have head:
  hash-target-program B ?N
    (fri-layer-opening-step bf st ::
      (nat × 'f × nat × nat,
        ('f, 'a) protocol-channel-scheme) state-monad)
  unfolding bf-eq st-eq
  by (rule hash-target-program-mono[OF head-le head-exact])
have tail:
  hash-target-program B (length fl * ?N)
    (mfold z (map fri-layer-opening-step fl) ::
      (nat × 'f × nat × nat,
        ('f, 'a) protocol-channel-scheme) state-monad)
  if out:
    Some (z, t) ∈
      set-dist (execute (fri-layer-opening-step bf st) s)
  for z and s :: ('f, 'a) protocol-channel-scheme
  and t :: ('f, 'a) protocol-channel-scheme
proof -
  from fri-layer-opening-step-outcome[OF out[unfolded bf-eq st-eq]]
  obtain xp xp-path xn xn-path x' where
    z-eq: z = (i mod (len div 2), x', len div 2, pw + pw)
  by blast
have len-le: floor-log len ≤ L
  using Cons.prem1 unfolding st-eq by simp
have next-len: floor-log (len div 2) ≤ L
  using floor-log-div2-le-self[of len] len-le by linarith
have z-inv:
  case z of (i, x, len, pw) ⇒ floor-log len ≤ L
  unfolding z-eq using next-len by simp
show ?thesis
  by (rule Cons.IH[OF z-inv])
qed
have hash-target-program B (?N + length fl * ?N)

```

```

    ((fri-layer-opening-step bf st ::
      (nat × 'f × nat × nat,
        ('f, 'a) protocol-channel-scheme) state-monad) >>=
      (λz. mfold z (map fri-layer-opening-step fl)))
  by (rule hash-target-program-bind-on-outcomes[OF head]) (rule tail)
then show ?case
  by (simp add: add.assoc add.commute add.left-commute)
qed

```

lemma *hash-target-program-receive-query-commits-mfold:*

assumes *init:* $\text{floor-log } \text{len} \leq L$

shows

hash-target-program B
 $(\text{length } fl * (\text{Suc } L + \text{Suc } L))$
 $(\text{mfold } (i, x, \text{len}, pw) (\text{receive-query-commits } fl))$

unfolding *receive-query-commits-eq-fri-layer-opening-steps*

proof –

have *st-inv:*

case (i, x, len, pw) *of*
 $(i, x, \text{len}, pw) \Rightarrow \text{floor-log } \text{len} \leq L$

using *init* **by** *simp*

show *hash-target-program B* $(\text{length } fl * (\text{Suc } L + \text{Suc } L))$

$(\text{mfold } (i, x, \text{len}, pw) (\text{map fri-layer-opening-step } fl) ::$
 $(\text{nat} \times 'f \times \text{nat} \times \text{nat},$
 $(f, 'a) \text{ protocol-channel-scheme}) \text{ state-monad})$

by (rule *hash-target-program-fri-layer-opening-steps-mfold-state*
 $[OF \text{ st-inv}]$)

qed

lemma *hash-range-budget-fri-layer-opening-steps-mfold-state:*

fixes *st* :: $\text{nat} \times 'f \times \text{nat} \times \text{nat}$

assumes *init:*

case *st* *of* $(i, x, \text{len}, pw) \Rightarrow \text{floor-log } \text{len} \leq L$

shows

hash-range-budget
 $(\text{length } fl * (\text{Suc } L + \text{Suc } L))$
 $(\text{mfold } st (\text{map fri-layer-opening-step } fl) ::$
 $(\text{nat} \times 'f \times \text{nat} \times \text{nat},$
 $(f, 'a) \text{ protocol-channel-scheme}) \text{ state-monad})$

using *init*

proof (*induction fl arbitrary: st*)

case *Nil*

then show ?*case*

by (*simp add: hash-range-budget-return*)

next

case (*Cons bf fl*)

obtain *i x len pw* **where** *st-eq:* $st = (i, x, \text{len}, pw)$

by (*cases st*)

obtain *b f* **where** *bf-eq:* $bf = (b, f)$

```

  by (cases bf)
let ?B = Suc L + Suc L
have head-exact:
  hash-range-budget (Suc (floor-log len) + Suc (floor-log len))
    (fri-layer-opening-step (b, f) (i, x, len, pw))
  by (rule hash-range-budget-fri-layer-opening-step)
have head-le:
  Suc (floor-log len) + Suc (floor-log len) ≤ ?B
  using Cons.premis unfolding st-eq by simp
have head:
  hash-range-budget ?B
    (fri-layer-opening-step bf st ::
      (nat × 'f × nat × nat,
        ('f, 'a) protocol-channel-scheme) state-monad)
  unfolding bf-eq st-eq
  by (rule hash-range-budget-mono[OF head-le head-exact])
have tail:
  hash-range-budget (length fl * ?B)
    (mfold z (map fri-layer-opening-step fl) ::
      (nat × 'f × nat × nat,
        ('f, 'a) protocol-channel-scheme) state-monad)
  if out:
    Some (z, t) ∈
      set-dist (execute (fri-layer-opening-step bf st) s)
  for z and s :: ('f, 'a) protocol-channel-scheme
  and t :: ('f, 'a) protocol-channel-scheme
proof -
  from fri-layer-opening-step-outcome[OF out[unfolded bf-eq st-eq]]
  obtain xp xp-path xn xn-path x' where
    z-eq: z = (i mod (len div 2), x', len div 2, pw + pw)
  by blast
have len-le: floor-log len ≤ L
  using Cons.premis unfolding st-eq by simp
have next-len: floor-log (len div 2) ≤ L
  using floor-log-div2-le-self[of len] len-le by linarith
have z-inv:
  case z of (i, x, len, pw) ⇒ floor-log len ≤ L
  unfolding z-eq using next-len by simp
have ih-z:
  hash-range-budget (length fl * ?B)
    (mfold z (map fri-layer-opening-step fl) ::
      (nat × 'f × nat × nat,
        ('f, 'a) protocol-channel-scheme) state-monad)
  using Cons.IH[OF z-inv] by assumption
show ?thesis
  by (rule ih-z)
qed
have hash-range-budget (?B + length fl * ?B)
  ((fri-layer-opening-step bf st ::

```



```

    (nat × 'f × nat × nat,
     ('f, 'a) protocol-channel-scheme) state-monad) >>=
    (λz. mfold z (map fri-layer-opening-step fl)))
  by (rule hash-range-budget-bind-on-outcomes[OF head]) (rule tail)
then show ?case
  by (simp add: add.assoc add.commute add.left-commute)
qed

```

lemma *hash-range-budget-receive-query-commits-mfold:*
assumes *init:* $\text{floor-log len} \leq L$
shows
hash-range-budget
 $(\text{length fl} * (\text{Suc } L + \text{Suc } L))$
 $(\text{mfold } (i, x, \text{len}, \text{pw}) (\text{receive-query-commits fl}))$
unfolding *receive-query-commits-eq-fri-layer-opening-steps*

proof –
have *st-inv:*
case $(i, x, \text{len}, \text{pw})$ *of*
 $(i, x, \text{len}, \text{pw}) \Rightarrow \text{floor-log len} \leq L$
using *init* **by** *simp*
show *hash-range-budget* $(\text{length fl} * (\text{Suc } L + \text{Suc } L))$
 $(\text{mfold } (i, x, \text{len}, \text{pw}) (\text{map fri-layer-opening-step fl})) ::$
 $(\text{nat} \times 'f \times \text{nat} \times \text{nat},$
 $(\text{'f}, \text{'a}) \text{ protocol-channel-scheme}) \text{ state-monad})$
by (rule *hash-range-budget-fri-layer-opening-steps-mfold-state*
 $[\text{OF } \text{st-inv}]$)

qed

lemma *hash-collision-budget-fri-layer-opening-steps-mfold-state:*

fixes *st* :: $\text{nat} \times 'f \times \text{nat} \times \text{nat}$
assumes *init:*
case *st* *of* $(i, x, \text{len}, \text{pw}) \Rightarrow \text{floor-log len} \leq L$

shows
hash-collision-budget
 $(\text{length fl} * (\text{Suc } L + \text{Suc } L))$
 $(\text{mfold } \text{st} (\text{map fri-layer-opening-step fl})) ::$
 $(\text{nat} \times 'f \times \text{nat} \times \text{nat},$
 $(\text{'f}, \text{'a}) \text{ protocol-channel-scheme}) \text{ state-monad})$

using *init*

proof (*induction fl arbitrary: st*)

case *Nil*

then show ?case

by (*simp add: hash-collision-budget-return*)

next

case (*Cons bf fl*)

obtain *i x len pw* **where** *st-eq:* $\text{st} = (i, x, \text{len}, \text{pw})$

by (*cases st*)

obtain *b f* **where** *bf-eq:* $\text{bf} = (b, f)$

by (*cases bf*)

```

let ?B = Suc L + Suc L
have head-range-exact:
  hash-range-budget (Suc (floor-log len) + Suc (floor-log len))
    (fri-layer-opening-step (b, f) (i, x, len, pw))
  by (rule hash-range-budget-fri-layer-opening-step)
have head-coll-exact:
  hash-collision-budget (Suc (floor-log len) + Suc (floor-log len))
    (fri-layer-opening-step (b, f) (i, x, len, pw))
  by (rule hash-collision-budget-fri-layer-opening-step)
have head-le:
  Suc (floor-log len) + Suc (floor-log len) ≤ ?B
  using Cons.premis unfolding st-eq by simp
have head-range:
  hash-range-budget ?B
    (fri-layer-opening-step bf st ::
      (nat × 'f × nat × nat,
        ('f, 'a) protocol-channel-scheme) state-monad)
  unfolding bf-eq st-eq
  by (rule hash-range-budget-mono[OF head-le head-range-exact])
have head-coll:
  hash-collision-budget ?B
    (fri-layer-opening-step bf st ::
      (nat × 'f × nat × nat,
        ('f, 'a) protocol-channel-scheme) state-monad)
  unfolding bf-eq st-eq
  by (rule hash-collision-budget-mono[OF head-le head-coll-exact])
have tail-range:
  hash-range-budget (length fl * ?B)
    (mfold z (map fri-layer-opening-step fl) ::
      (nat × 'f × nat × nat,
        ('f, 'a) protocol-channel-scheme) state-monad)
  if out:
    Some (z, t) ∈
      set-dist (execute (fri-layer-opening-step bf st) s)
  for z and s :: ('f, 'a) protocol-channel-scheme
  and t :: ('f, 'a) protocol-channel-scheme
proof –
  from fri-layer-opening-step-outcome[OF out[unfolded bf-eq st-eq]]
  obtain xp xp-path xn xn-path x' where
    z-eq: z = (i mod (len div 2), x', len div 2, pw + pw)
  by blast
  have len-le: floor-log len ≤ L
  using Cons.premis unfolding st-eq by simp
  have next-len: floor-log (len div 2) ≤ L
  using floor-log-div2-le-self[of len] len-le by linarith
  have z-inv:
    case z of (i, x, len, pw) ⇒ floor-log len ≤ L
  unfolding z-eq using next-len by simp
  show ?thesis

```

```

    by (rule hash-range-budget-fri-layer-opening-steps-mfold-state
        [OF z-inv])
qed
have tail-coll:
  hash-collision-budget (length fl * ?B)
    (mfold z (map fri-layer-opening-step fl) ::
      (nat × 'f × nat × nat,
        ('f, 'a) protocol-channel-scheme) state-monad)
  if out:
    Some (z, t) ∈
      set-dist (execute (fri-layer-opening-step bf st) s)
  for z and s :: ('f, 'a) protocol-channel-scheme
    and t :: ('f, 'a) protocol-channel-scheme
proof -
  from fri-layer-opening-step-outcome[OF out[unfolded bf-eq st-eq]]
  obtain xp xp-path xn xn-path x' where
    z-eq: z = (i mod (len div 2), x', len div 2, pw + pw)
    by blast
  have len-le: floor-log len ≤ L
    using Cons.premis unfolding st-eq by simp
  have next-len: floor-log (len div 2) ≤ L
    using floor-log-div2-le-self[of len] len-le by linarith
  have z-inv:
    case z of (i, x, len, pw) ⇒ floor-log len ≤ L
    unfolding z-eq using next-len by simp
  have ih-z:
    hash-collision-budget (length fl * ?B)
      (mfold z (map fri-layer-opening-step fl) ::
        (nat × 'f × nat × nat,
          ('f, 'a) protocol-channel-scheme) state-monad)
    using Cons.IH[OF z-inv] by assumption
  show ?thesis
    by (rule ih-z)
qed
have hash-collision-budget (?B + length fl * ?B)
  ((fri-layer-opening-step bf st ::
    (nat × 'f × nat × nat,
      ('f, 'a) protocol-channel-scheme) state-monad) ≫
    (λz. mfold z (map fri-layer-opening-step fl)))
  by (rule hash-collision-budget-bind-on-outcomes
      [OF head-range head-coll tail-range tail-coll])
then show ?case
  by (simp add: add.assoc add.commute add.left-commute)
qed

lemma hash-collision-budget-receive-query-commits-mfold:
  assumes init: floor-log len ≤ L
  shows
    hash-collision-budget

```

```

    (length fl * (Suc L + Suc L))
    (mfold (i, x, len, pw) (receive-query-commits fl))
  unfolding receive-query-commits-eq-fri-layer-opening-steps
proof -
  have st-inv:
    case (i, x, len, pw) of
      (i, x, len, pw)  $\Rightarrow$  floor-log len  $\leq$  L
    using init by simp
  show hash-collision-budget (length fl * (Suc L + Suc L))
    (mfold (i, x, len, pw) (map fri-layer-opening-step fl)) ::
      (nat  $\times$  'f  $\times$  nat  $\times$  nat,
       ('f, 'a) protocol-channel-scheme) state-monad
  by (rule hash-collision-budget-fri-layer-opening-steps-mfold-state
      [OF st-inv])
qed

lemma hash-target-program-query-decommitment-step:
  hash-target-program B (Suc (floor-log (scale * clength)))
  (query-decommitment-step fr i)
proof -
  let ?len = scale * clength
  have after-path:
    hash-target-program B (Suc (floor-log ?len))
    ((ntimes read (floor-log ?len)) ::
      ('f list, ('f, 'a) protocol-channel-scheme) state-monad)  $\gg=$ 
    ( $\lambda$ path.
      (check-authentication-path ?len i qh path ::
        ('f, ('f, 'a) protocol-channel-scheme) state-monad)  $\gg=$ 
        ( $\lambda$ ap. assert (ap = fr)  $\gg=$  ( $\lambda$ -. return qh))))
  for qh
proof (rule hash-target-program-ntimes-read-bind)
  fix path :: 'f list
  assume path-len: length path = floor-log ?len
  have hash-target-program B (Suc (length path))
    ((check-authentication-path ?len i qh path ::
      ('f, ('f, 'a) protocol-channel-scheme) state-monad)  $\gg=$ 
      ( $\lambda$ ap. assert (ap = fr)  $\gg=$  ( $\lambda$ -. return qh)))
  by (rule hash-target-program-check-authentication-path-assert-return)
  then show hash-target-program B (Suc (floor-log ?len))
    ((check-authentication-path ?len i qh path ::
      ('f, ('f, 'a) protocol-channel-scheme) state-monad)  $\gg=$ 
      ( $\lambda$ ap. assert (ap = fr)  $\gg=$  ( $\lambda$ -. return qh)))
  by (subst path-len[symmetric])
qed
have hash-target-program B (0 + Suc (floor-log ?len))
  (read  $\gg=$ 
    ( $\lambda$ qh. (ntimes read (floor-log ?len)) ::
      ('f list, ('f, 'a) protocol-channel-scheme) state-monad)  $\gg=$ 
    ( $\lambda$ path.

```

```

      (check-authentication-path ?len i qh path ::
        ('f, ('f, 'a) protocol-channel-scheme) state-monad) >>=
        (λap. assert (ap = fr) >>= (λ-. return qh))))
  by (rule hash-target-program-bind
      [OF hash-target-program-read after-path])
  then show ?thesis
    unfolding query-decommitment-step-def by (simp add: Let-def)
qed

lemma hash-target-program-check-decommit-on-query:
  hash-target-program B
    (powers * Suc (floor-log (scale * clength)))
    (mmap (check-decommit-on-query fr idx))
proof -
  let ?b = Suc (floor-log (scale * clength))
  let ?steps =
    (map (query-decommitment-step fr) (powers-scaled idx) ::
      ('f, ('f, 'a) protocol-channel-scheme) state-monad list)
  have step-budget:
    ∧(m :: ('f, ('f, 'a) protocol-channel-scheme) state-monad).
      m ∈ set ?steps ⇒
        hash-target-program B ?b m
  proof -
    fix m :: ('f, ('f, 'a) protocol-channel-scheme) state-monad
    assume m ∈ set ?steps
    then obtain i where i ∈ set (powers-scaled idx)
      and m-eq: m = query-decommitment-step fr i
      by auto
    show hash-target-program B ?b m
      unfolding m-eq by (rule hash-target-program-query-decommitment-step)
  qed
  have map-budget:
    hash-target-program B (length ?steps * ?b) (mmap ?steps)
    by (rule hash-target-program-mmap) (rule step-budget)
  then have hash-target-program B
    (length (powers-scaled idx) * ?b) (mmap ?steps)
    by simp
  then show ?thesis
    unfolding check-decommit-on-query-def query-decommitment-step-def
      powers-scaled-def
    by simp
qed

lemma hash-range-budget-query-decommitment-step:
  hash-range-budget (Suc (floor-log (scale * clength)))
    (query-decommitment-step fr i)
proof -
  let ?len = scale * clength
  have after-path:

```

```

hash-range-budget (Suc (floor-log ?len))
  ((ntimes read (floor-log ?len) ::
    ('f list, ('f, 'a) protocol-channel-scheme) state-monad) >>=
    (λpath.
      (check-authentication-path ?len i qh path ::
        ('f, ('f, 'a) protocol-channel-scheme) state-monad) >>=
        (λap. assert (ap = fr) >>= (λ-. return qh)))) for qh
proof (rule hash-range-budget-ntimes-read-bind)
  fix path :: 'f list
  assume path-len: length path = floor-log ?len
  have hash-range-budget (Suc (length path))
    (((check-authentication-path ?len i qh path ::
      ('f, ('f, 'a) protocol-channel-scheme) state-monad) >>=
      (λap. assert (ap = fr) >>= (λ-. return qh))))
    by (rule hash-range-budget-check-authentication-path-assert-return)
  note path-budget = this
  show hash-range-budget (Suc (floor-log ?len))
    ((check-authentication-path ?len i qh path ::
      ('f, ('f, 'a) protocol-channel-scheme) state-monad) >>=
      (λap. assert (ap = fr) >>= (λ-. return qh)))
    by (subst path-len[symmetric]) (rule path-budget)
qed
have hash-range-budget (0 + Suc (floor-log ?len))
  (read >>=
    (λqh. (ntimes read (floor-log ?len) ::
      ('f list, ('f, 'a) protocol-channel-scheme) state-monad) >>=
      (λpath.
        (check-authentication-path ?len i qh path ::
          ('f, ('f, 'a) protocol-channel-scheme) state-monad) >>=
          (λap. assert (ap = fr) >>= (λ-. return qh))))))
  by (rule hash-range-budget-bind[OF hash-range-budget-read after-path])
then show ?thesis
  unfolding query-decommitment-step-def by (simp add: Let-def)
qed

```

lemma hash-collision-budget-query-decommitment-step:
 hash-collision-budget (Suc (floor-log (scale * clength)))
 (query-decommitment-step fr i)

```

proof -
  let ?len = scale * clength
  have after-path-range:
    hash-range-budget (Suc (floor-log ?len))
      ((ntimes read (floor-log ?len) ::
        ('f list, ('f, 'a) protocol-channel-scheme) state-monad) >>=
        (λpath.
          (check-authentication-path ?len i qh path ::
            ('f, ('f, 'a) protocol-channel-scheme) state-monad) >>=
            (λap. assert (ap = fr) >>= (λ-. return qh)))) for qh
  proof (rule hash-range-budget-ntimes-read-bind)

```

```

fix path :: 'f list
assume path-len: length path = floor-log ?len
have hash-range-budget (Suc (length path))
  (((check-authentication-path ?len i qh path ::
    ('f, ('f, 'a) protocol-channel-scheme) state-monad) >>=
    (λap. assert (ap = fr) >>= (λ-. return qh))))
  by (rule hash-range-budget-check-authentication-path-assert-return)
note path-budget = this
show hash-range-budget (Suc (floor-log ?len))
  ((check-authentication-path ?len i qh path ::
    ('f, ('f, 'a) protocol-channel-scheme) state-monad) >>=
    (λap. assert (ap = fr) >>= (λ-. return qh)))
  by (subst path-len[symmetric]) (rule path-budget)
qed
have after-path-coll:
  hash-collision-budget (Suc (floor-log ?len))
  ((ntimes read (floor-log ?len) ::
    ('f list, ('f, 'a) protocol-channel-scheme) state-monad) >>=
    (λpath.
      (check-authentication-path ?len i qh path ::
        ('f, ('f, 'a) protocol-channel-scheme) state-monad) >>=
        (λap. assert (ap = fr) >>= (λ-. return qh)))) for qh
proof (rule hash-collision-budget-ntimes-read-bind)
  fix path :: 'f list
  assume path-len: length path = floor-log ?len
  have range-bind:
    hash-range-budget (Suc (length path))
      (((check-authentication-path ?len i qh path ::
        ('f, ('f, 'a) protocol-channel-scheme) state-monad) >>=
        (λap. assert (ap = fr) >>= (λ-. return qh))))
      by (rule hash-range-budget-check-authentication-path-assert-return)
  show hash-range-budget (Suc (floor-log ?len))
    ((check-authentication-path ?len i qh path ::
      ('f, ('f, 'a) protocol-channel-scheme) state-monad) >>=
      (λap. assert (ap = fr) >>= (λ-. return qh)))
    by (subst path-len[symmetric]) (rule range-bind)
  have hash-collision-budget (Suc (length path))
    (((check-authentication-path ?len i qh path ::
      ('f, ('f, 'a) protocol-channel-scheme) state-monad) >>=
      (λap. assert (ap = fr) >>= (λ-. return qh))))
    by (rule hash-collision-budget-check-authentication-path-assert-return)
  note path-budget = this
  show hash-collision-budget (Suc (floor-log ?len))
    ((check-authentication-path ?len i qh path ::
      ('f, ('f, 'a) protocol-channel-scheme) state-monad) >>=
      (λap. assert (ap = fr) >>= (λ-. return qh)))
    by (subst path-len[symmetric]) (rule path-budget)
qed
have hash-collision-budget (0 + Suc (floor-log ?len))

```

```

(read >>=
  (λqh. (ntimes read (floor-log ?len) ::
    ('f list, ('f, 'a) protocol-channel-scheme) state-monad) >>=
    (λpath.
      (check-authentication-path ?len i qh path ::
        ('f, ('f, 'a) protocol-channel-scheme) state-monad) >>=
        (λap. assert (ap = fr) >>= (λ-. return qh))))))
by (rule hash-collision-budget-bind
  [OF hash-range-budget-read hash-collision-budget-read
    after-path-range after-path-coll])
then show ?thesis
  unfolding query-decommitment-step-def by (simp add: Let-def)
qed

lemma hash-range-budget-check-decommit-on-query:
  hash-range-budget
    (powers * Suc (floor-log (scale * clength)))
    (mmap (check-decommit-on-query fr idx))
proof -
  let ?b = Suc (floor-log (scale * clength))
  let ?steps =
    (map (query-decommitment-step fr) (powers-scaled idx) ::
      ('f, ('f, 'a) protocol-channel-scheme) state-monad list)
  have step-budget:
    ∧(m :: ('f, ('f, 'a) protocol-channel-scheme) state-monad).
      m ∈ set ?steps ⇒
        hash-range-budget ?b m
  proof -
    fix m :: ('f, ('f, 'a) protocol-channel-scheme) state-monad
    assume m ∈ set ?steps
    then obtain i where i ∈ set (powers-scaled idx)
      and m-eq: m = query-decommitment-step fr i
      by auto
    show hash-range-budget ?b m
      unfolding m-eq by (rule hash-range-budget-query-decommitment-step)
  qed
  have map-budget:
    hash-range-budget
      (length ?steps * ?b) (mmap ?steps)
    by (rule hash-range-budget-mmap) (rule step-budget)
  then have hash-range-budget (length (powers-scaled idx) * ?b)
    (mmap ?steps)
    by simp
  then show ?thesis
    unfolding check-decommit-on-query-def query-decommitment-step-def
      powers-scaled-def
    by simp
qed

```



```

lemma hash-collision-budget-check-decommit-on-query:
  hash-collision-budget
    (powers * Suc (floor-log (scale * clength)))
    (mmap (check-decommit-on-query fr idx))
proof -
  let ?b = Suc (floor-log (scale * clength))
  let ?steps =
    (map (query-decommitment-step fr) (powers-scaled idx) ::
      ('f, ('f, 'a) protocol-channel-scheme) state-monad list)
  have step-range:
     $\bigwedge (m :: ('f, ('f, 'a) protocol-channel-scheme) state-monad).$ 
     $m \in \text{set } ?steps \implies$ 
    hash-range-budget ?b m
proof -
  fix m :: ('f, ('f, 'a) protocol-channel-scheme) state-monad
  assume m  $\in$  set ?steps
  then obtain i where i  $\in$  set (powers-scaled idx)
    and m-eq: m = query-decommitment-step fr i
    by auto
  show hash-range-budget ?b m
    unfolding m-eq by (rule hash-range-budget-query-decommitment-step)
qed
  have step-coll:
     $\bigwedge (m :: ('f, ('f, 'a) protocol-channel-scheme) state-monad).$ 
     $m \in \text{set } ?steps \implies$ 
    hash-collision-budget ?b m
proof -
  fix m :: ('f, ('f, 'a) protocol-channel-scheme) state-monad
  assume m  $\in$  set ?steps
  then obtain i where i  $\in$  set (powers-scaled idx)
    and m-eq: m = query-decommitment-step fr i
    by auto
  show hash-collision-budget ?b m
    unfolding m-eq by (rule hash-collision-budget-query-decommitment-step)
qed
  have map-budget:
    hash-collision-budget
      (length ?steps * ?b) (mmap ?steps)
    apply (rule hash-collision-budget-mmap)
    apply (rule step-range, assumption)
    apply (rule step-coll, assumption)
    done
  then have hash-collision-budget (length (powers-scaled idx) * ?b)
    (mmap ?steps)
    by simp
  then show ?thesis
    unfolding check-decommit-on-query-def query-decommitment-step-def
      powers-scaled-def
    by simp

```

qed

end

end

theory *Soundness-Oracle-Verifier*
 imports *Soundness-Oracle-Budgets*
begin

Concrete verifier hash-query budget bounds.

context *soundness*
begin

lemma *ceil-log-mono*:

assumes $m \leq n$

shows $\text{ceil-log } m \leq \text{ceil-log } n$

proof (*cases* $m \leq 1$)

case *True*

then show *?thesis*

unfolding *ceil-log-def* **by** *simp*

next

case *False*

then have *m-pos*: $0 < m$ **and** *m-minus*: $m - 1 \leq n - 1$

using *assms* **by** *simp-all*

have *n-gt*: $1 < n$

using *False assms* **by** *linarith*

have $\text{floor-log } (m - 1) \leq \text{floor-log } (n - 1)$

using *floor-log-le-iff m-minus* **by** *blast*

then show *?thesis*

using *False n-gt* **unfolding** *ceil-log-def* **by** *simp*

qed

definition *verifier-fri-layer-hash-budget* :: *nat*

where *verifier-fri-layer-hash-budget* =

Suc (*floor-log* (*clength* * *scale*)) + *Suc* (*floor-log* (*clength* * *scale*))

definition *verifier-query-decommit-hash-budget* :: *nat*

where *verifier-query-decommit-hash-budget* =

powers * *Suc* (*floor-log* (*clength* * *scale*))

definition *verifier-query-round-hash-budget* :: *nat*

where *verifier-query-round-hash-budget* =

 1 + *verifier-query-decommit-hash-budget* +
 (*ceil-log* *clength* + *ceil-log* (*maxDegree* + 1)) *
 verifier-fri-layer-hash-budget

definition *verifier-header-hash-budget* :: *nat*

where *verifier-header-hash-budget* =
 $\text{ceil-log } \text{clength} + \text{length spec} + \text{ceil-log } (\text{maxDegree} + 1)$

definition *verifier-hash-query-budget* :: *nat*

where *verifier-hash-query-budget* =
 $\text{verifier-header-hash-budget} + \text{rounds} * \text{verifier-query-round-hash-budget}$

definition *concrete-merkle-binding-error* :: *prob*

where
 $\text{concrete-merkle-binding-error} =$
 $\text{hash-collision-budget-value } 0 \text{ verifier-hash-query-budget}$

definition *concrete-transcript-target-error* :: '*f list* $\Rightarrow$ *prob*

where
 $\text{concrete-transcript-target-error } tr =$
 $\text{nnreal } (\text{card } (\text{set } tr) * \text{verifier-hash-query-budget}) / \text{nnreal size}$

lemma *hash-target-program-alpha-challenge-step*:

hash-target-program B 1
 (do {
 a0 $\leftarrow$ *receive-alpha-challenge*;
 let *a0'* = *a0*;
 a1 $\leftarrow$ *read*;
 let *a1'* = *a1*;
 assert (*a0'* = *a1'*);
 return *a1'*
 })

proof –

have *assert-return*:
 $\text{hash-target-program } B \ (0 + 0)$
 $(\text{assert } (a0 = a1) \gg (\lambda-. \text{return } a1)) ::$
 $(f, (f, 'a) \text{ protocol-channel-scheme}) \text{ state-monad}$
for *a0 a1*
by (rule *hash-target-program-bind*
 [OF *hash-target-program-assert hash-target-program-return*])

have *after-read*:
 $\text{hash-target-program } B \ (0 + 0 + 0)$
 $(\text{read} \gg (\lambda a1. \text{assert } (a0 = a1) \gg (\lambda-. \text{return } a1))) ::$
 $(f, (f, 'a) \text{ protocol-channel-scheme}) \text{ state-monad}$
for *a0*

proof –

have *hash-target-program B (0 + (0 + 0))*
 $(\text{read} \gg (\lambda a1. \text{assert } (a0 = a1) \gg (\lambda-. \text{return } a1))) ::$
 $(f, (f, 'a) \text{ protocol-channel-scheme}) \text{ state-monad}$
by (rule *hash-target-program-bind*
 [OF *hash-target-program-read assert-return*])

then show *?thesis* **by** *simp*

qed

have *hash-target-program B (1 + (0 + 0 + 0))*

```

    (receive-alpha-challenge  $\gg$ 
      ( $\lambda a0$ . read  $\gg$  ( $\lambda a1$ .
        assert ( $a0 = a1$ )  $\gg$  ( $\lambda$ -. return  $a1$ ))) ::
      ('f, ('f, 'a) protocol-channel-scheme) state-monad)
  by (rule hash-target-program-bind
    [OF hash-target-program-receive-alpha-challenge after-read])
  then show ?thesis
    by simp
qed

```

lemma hash-range-budget-alpha-challenge-step:

```

hash-range-budget 1
  (do {
    a0  $\leftarrow$  receive-alpha-challenge;
    let a0' = a0;
    a1  $\leftarrow$  read;
    let a1' = a1;
    assert ( $a0' = a1'$ );
    return a1'
  })
proof -
  have assert-return:
    hash-range-budget (0 + 0)
    (assert ( $a0 = a1$ )  $\gg$  ( $\lambda$ -. return  $a1$ )) ::
    ('f, ('f, 'a) protocol-channel-scheme) state-monad for a0 a1
  by (rule hash-range-budget-bind
    [OF hash-range-budget-assert hash-range-budget-return])
  have after-read:
    hash-range-budget (0 + 0 + 0)
    (read  $\gg$  ( $\lambda a1$ . assert ( $a0 = a1$ )  $\gg$  ( $\lambda$ -. return  $a1$ ))) ::
    ('f, ('f, 'a) protocol-channel-scheme) state-monad for a0
  proof -
    have hash-range-budget (0 + (0 + 0))
      (read  $\gg$  ( $\lambda a1$ . assert ( $a0 = a1$ )  $\gg$  ( $\lambda$ -. return  $a1$ ))) ::
      ('f, ('f, 'a) protocol-channel-scheme) state-monad
    by (rule hash-range-budget-bind
      [OF hash-range-budget-read assert-return])
    then show ?thesis by simp
  qed
  have hash-range-budget (1 + (0 + 0 + 0))
    (receive-alpha-challenge  $\gg$ 
      ( $\lambda a0$ . read  $\gg$  ( $\lambda a1$ .
        assert ( $a0 = a1$ )  $\gg$  ( $\lambda$ -. return  $a1$ ))) ::
      ('f, ('f, 'a) protocol-channel-scheme) state-monad)
  by (rule hash-range-budget-bind
    [OF hash-range-budget-receive-alpha-challenge after-read])
  then show ?thesis
    by simp
qed

```

lemma *hash-collision-budget-alpha-challenge-step*:
hash-collision-budget 1
 (do {
a0 ← *receive-alpha-challenge*;
 let *a0'* = *a0*;
a1 ← *read*;
 let *a1'* = *a1*;
 assert (*a0'* = *a1'*);
 return *a1'*
 })
proof –
have *assert-return-range*:
hash-range-budget (0 + 0)
 (assert (*a0* = *a1*) $\gg$ (λ -. return *a1*)) ::
 (*f*, (*f*, 'a) *protocol-channel-scheme*) *state-monad*) **for** *a0 a1*
by (rule *hash-range-budget-bind*
 [OF *hash-range-budget-assert hash-range-budget-return*])
have *assert-return-coll*:
hash-collision-budget (0 + 0)
 (assert (*a0* = *a1*) $\gg$ (λ -. return *a1*)) ::
 (*f*, (*f*, 'a) *protocol-channel-scheme*) *state-monad*) **for** *a0 a1*
by (rule *hash-collision-budget-bind*
 [OF *hash-range-budget-assert hash-collision-budget-assert*
hash-range-budget-return hash-collision-budget-return])
have *after-read-range*:
hash-range-budget (0 + 0 + 0)
 (read $\gg$ (λ a1. assert (*a0* = *a1*) $\gg$ (λ -. return *a1*))) ::
 (*f*, (*f*, 'a) *protocol-channel-scheme*) *state-monad*) **for** *a0*
proof –
have *hash-range-budget (0 + (0 + 0))*
 (read $\gg$ (λ a1. assert (*a0* = *a1*) $\gg$ (λ -. return *a1*))) ::
 (*f*, (*f*, 'a) *protocol-channel-scheme*) *state-monad*)
by (rule *hash-range-budget-bind*
 [OF *hash-range-budget-read assert-return-range*])
then show ?thesis **by** *simp*
qed
have *after-read-coll*:
hash-collision-budget (0 + 0 + 0)
 (read $\gg$ (λ a1. assert (*a0* = *a1*) $\gg$ (λ -. return *a1*))) ::
 (*f*, (*f*, 'a) *protocol-channel-scheme*) *state-monad*) **for** *a0*
proof –
have *hash-collision-budget (0 + (0 + 0))*
 (read $\gg$ (λ a1. assert (*a0* = *a1*) $\gg$ (λ -. return *a1*))) ::
 (*f*, (*f*, 'a) *protocol-channel-scheme*) *state-monad*)
by (rule *hash-collision-budget-bind*
 [OF *hash-range-budget-read hash-collision-budget-read*
assert-return-range assert-return-coll])
then show ?thesis **by** *simp*

```

qed
have hash-collision-budget (1 + (0 + 0 + 0))
  (receive-alpha-challenge  $\gg$ 
    ( $\lambda a0$ . read  $\gg$  ( $\lambda a1$ .
      assert ( $a0 = a1$ )  $\gg$  ( $\lambda$ -. return  $a1$ ))) ::
    ('f, ('f, 'a) protocol-channel-scheme) state-monad)
by (rule hash-collision-budget-bind
  [OF hash-range-budget-receive-alpha-challenge
    hash-collision-budget-receive-alpha-challenge
    after-read-range after-read-coll])
then show ?thesis
by simp
qed

lemma hash-target-program-verifier-query-round-program-exact:
  hash-target-program B
  (1 + verifier-query-decommit-hash-budget +
    (length f-fl + length fl) * verifier-fri-layer-hash-budget)
  (verifier-query-round-program fr f-fl f-final as fl final)
proof -
let ?L = floor-log (clength * scale)
let ?B = verifier-fri-layer-hash-budget
have B-eq: ?B = Suc ?L + Suc ?L
unfolding verifier-fri-layer-hash-budget-def by simp
have after-comp:
  hash-target-program B (length fl * ?B + 0)
  ((mfold
    (idx, cp-eval as fv ( $h \hat{\ } idx * shift$ ), clength * scale, 1)
    (receive-query-commits fl) ::
    (nat  $\times$  'f  $\times$  nat  $\times$  nat,
      ('f, 'a) protocol-channel-scheme) state-monad)  $\gg$ 
    ( $\lambda(i, x, len, pw)$ . assert ( $x = final$ ))))
for idx fv
proof (rule hash-target-program-bind)
show hash-target-program B (length fl * ?B)
  (mfold
    (idx, cp-eval as fv ( $h \hat{\ } idx * shift$ ), clength * scale, 1)
    (receive-query-commits fl) ::
    (nat  $\times$  'f  $\times$  nat  $\times$  nat,
      ('f, 'a) protocol-channel-scheme) state-monad)
unfolding B-eq
by (rule hash-target-program-receive-query-commits-mfold) simp
show  $\bigwedge x$ . hash-target-program B 0
  ((case x of (i, x, len, pw)  $\Rightarrow$  assert ( $x = final$ )) ::
    (unit, ('f, 'a) protocol-channel-scheme) state-monad)
by (simp add: hash-target-program-assert split: prod.splits)
qed
have after-trace-assert:
  hash-target-program B (0 + (length fl * ?B + 0))

```

```

((assert (f-x = f-final) ::
  (unit, ('f, 'a) protocol-channel-scheme) state-monad) >>=
(λ-. (mfold
  (idx, cp-eval as fv (h ^ idx * shift), clength * scale, 1)
  (receive-query-commits fl) ::
  (nat × 'f × nat × nat,
    ('f, 'a) protocol-channel-scheme) state-monad) >>=
  (λ(i, x, len, pw). assert (x = final))))
for idx fv f-i f-x f-len f-pow
by (rule hash-target-program-bind
  [OF hash-target-program-assert after-comp])
have after-trace:
  hash-target-program B
  (length f-fl * ?B + (0 + (length fl * ?B + 0)))
  ((mfold (idx, hd fv, clength * scale, 1)
    (receive-query-commits f-fl) ::
    (nat × 'f × nat × nat,
      ('f, 'a) protocol-channel-scheme) state-monad) >>=
    (λ(f-i, f-x, f-len, f-pow).
      assert (f-x = f-final) >>=
      (λ-. (mfold
        (idx, cp-eval as fv (h ^ idx * shift), clength * scale, 1)
        (receive-query-commits fl) ::
        (nat × 'f × nat × nat,
          ('f, 'a) protocol-channel-scheme) state-monad) >>=
          (λ(i, x, len, pw). assert (x = final))))))
for idx fv
proof (rule hash-target-program-bind)
show hash-target-program B (length f-fl * ?B)
  (mfold (idx, hd fv, clength * scale, 1)
    (receive-query-commits f-fl) ::
    (nat × 'f × nat × nat,
      ('f, 'a) protocol-channel-scheme) state-monad)
unfolding B-eq
by (rule hash-target-program-receive-query-commits-mfold) simp
fix x :: nat × 'f × nat × nat
obtain f-i f-x f-len f-pow where x-eq:
  x = (f-i, f-x, f-len, f-pow)
by (cases x) auto
have case-budget:
  hash-target-program B (0 + (length fl * ?B + 0))
  ((assert (f-x = f-final) ::
    (unit, ('f, 'a) protocol-channel-scheme) state-monad) >>=
    (λ-. (mfold
      (idx, cp-eval as fv (h ^ idx * shift), clength * scale, 1)
      (receive-query-commits fl) ::
      (nat × 'f × nat × nat,
        ('f, 'a) protocol-channel-scheme) state-monad) >>=
        (λ(i, x, len, pw). assert (x = final))))))

```

```

    by (rule hash-target-program-bind
        [OF hash-target-program-assert after-comp])
  show hash-target-program B (0 + (length fl * ?B + 0))
    ((case x of (f-i, f-x, f-len, f-pow) =>
      assert (f-x = f-final) >>=
      (λ-. (mfold
        (idx, cp-eval as fv (h ^ idx * shift), clength * scale, 1)
        (receive-query-commits fl) ::
        (nat × 'f × nat × nat,
          ('f, 'a) protocol-channel-scheme) state-monad) >>=
        (λ(i, x, len, pw). assert (x = final)))) ::
        (unit, ('f, 'a) protocol-channel-scheme) state-monad)
  using case-budget
  unfolding x-eq
  by (simp split: prod.splits)
qed
have after-query:
  hash-target-program B
    (verifier-query-decommit-hash-budget +
      (length f-fl * ?B + (0 + (length fl * ?B + 0))))
    (mmap (check-decommit-on-query fr idx) >>=
      (λfv. (mfold (idx, hd fv, clength * scale, 1)
        (receive-query-commits f-fl) ::
        (nat × 'f × nat × nat,
          ('f, 'a) protocol-channel-scheme) state-monad) >>=
        (λ(f-i, f-x, f-len, f-pow).
          assert (f-x = f-final) >>=
          (λ-. (mfold
            (idx, cp-eval as fv (h ^ idx * shift), clength * scale, 1)
            (receive-query-commits fl) ::
            (nat × 'f × nat × nat,
              ('f, 'a) protocol-channel-scheme) state-monad) >>=
            (λ(i, x, len, pw). assert (x = final)))))))
  for idx
proof -
  have decommit:
    hash-target-program B verifier-query-decommit-hash-budget
      (mmap (check-decommit-on-query fr idx))
  proof -
    have exact:
      hash-target-program B
        (powers * Suc (floor-log (scale * clength)))
        (mmap (check-decommit-on-query fr idx))
    by (rule hash-target-program-check-decommit-on-query)
  show ?thesis
    using exact
    unfolding verifier-query-decommit-hash-budget-def
    by (simp add: mult commute)
qed

```



```

show ?thesis
  by (rule hash-target-program-bind[OF decommit after-trace])
qed
have after-query-simple:
  hash-target-program B
    (verifier-query-decommit-hash-budget +
     (length f-fl * ?B + length fl * ?B))
    (mmap (check-decommit-on-query fr idx) >>=
     (λfv. (mfold (idx, hd fv, clength * scale, 1)
      (receive-query-commits f-fl) ::
      (nat × 'f × nat × nat,
       ('f, 'a) protocol-channel-scheme) state-monad) >>=
      (λ(f-i, f-x, f-len, f-pow).
       assert (f-x = f-final) >>=
       (λ-. (mfold
        (idx, cp-eval as fv (h ^ idx * shift), clength * scale, 1)
        (receive-query-commits fl) ::
        (nat × 'f × nat × nat,
         ('f, 'a) protocol-channel-scheme) state-monad) >>=
        (λ(i, x, len, pw). assert (x = final)))))))
  for idx
  using after-query[of idx] by simp
have after-random:
  hash-target-program B
    (verifier-query-decommit-hash-budget +
     (length f-fl * ?B + length fl * ?B))
    ((let idx' = index (to-nat idx) in
     mmap (check-decommit-on-query fr idx') >>=
     (λfv. (mfold (idx', hd fv, clength * scale, 1)
      (receive-query-commits f-fl) ::
      (nat × 'f × nat × nat,
       ('f, 'a) protocol-channel-scheme) state-monad) >>=
      (λ(f-i, f-x, f-len, f-pow).
       assert (f-x = f-final) >>=
       (λ-. (mfold
        (idx', cp-eval as fv (h ^ idx' * shift), clength * scale, 1)
        (receive-query-commits fl) ::
        (nat × 'f × nat × nat,
         ('f, 'a) protocol-channel-scheme) state-monad) >>=
        (λ(i, x, len, pw). assert (x = final)))))))
  for idx
  using after-query-simple[of index (to-nat idx)]
  by (simp add: Let-def algebra-simps mult.commute split: prod.splits)
have hash-target-program B
  (1 + (verifier-query-decommit-hash-budget +
   (length f-fl * ?B + length fl * ?B)))
  (receive-query-index-challenge >>=
   (λidx.
    let idx' = index (to-nat idx) in

```

```

mmap (check-decommit-on-query fr idx') >>=
(λfv. (mfold (idx', hd fv, clength * scale, 1)
  (receive-query-commits f-fl) ::
  (nat × 'f × nat × nat,
    ('f, 'a) protocol-channel-scheme) state-monad) >>=
  (λ(f-i, f-x, f-len, f-pw).
    assert (f-x = f-final) >>=
    (λ-. (mfold
      (idx', cp-eval as fv (h ^ idx' * shift), clength * scale, 1)
      (receive-query-commits fl) ::
      (nat × 'f × nat × nat,
        ('f, 'a) protocol-channel-scheme) state-monad) >>=
      (λ(i, x, len, pw). assert (x = final))))))
by (rule hash-target-program-bind
  [OF hash-target-program-receive-query-index-challenge])
(rule after-random)
then show ?thesis
unfolding verifier-query-round-program-def
by (simp add: Let-def algebra-simps mult.commute split: prod.splits)
qed

```

lemma hash-range-budget-verifier-query-round-program-exact:
hash-range-budget

```

(1 + verifier-query-decommit-hash-budget +
  (length f-fl + length fl) * verifier-fri-layer-hash-budget)
(verifier-query-round-program fr f-fl f-final as fl final)

```

proof –

```

let ?L = floor-log (clength * scale)
let ?B = verifier-fri-layer-hash-budget
have B-eq: ?B = Suc ?L + Suc ?L
unfolding verifier-fri-layer-hash-budget-def by simp
have after-comp:
hash-range-budget (length fl * ?B + 0)
  ((mfold
    (idx, cp-eval as fv (h ^ idx * shift), clength * scale, 1)
    (receive-query-commits fl) ::
    (nat × 'f × nat × nat,
      ('f, 'a) protocol-channel-scheme) state-monad) >>=
    (λ(i, x, len, pw). assert (x = final))))

```

for idx fv

proof (rule hash-range-budget-bind)

show hash-range-budget (length fl * ?B)

```

(mfold
  (idx, cp-eval as fv (h ^ idx * shift), clength * scale, 1)
  (receive-query-commits fl) ::
  (nat × 'f × nat × nat,
    ('f, 'a) protocol-channel-scheme) state-monad)

```

unfolding B-eq

```

  by (rule hash-range-budget-receive-query-commits-mfold) simp
show  $\bigwedge x. \text{hash-range-budget } 0$ 
  ((case x of (i, x, len, pw)  $\Rightarrow$  assert (x = final)) ::
    (unit, ('f, 'a) protocol-channel-scheme) state-monad)
  by (simp add: hash-range-budget-assert split: prod.splits)
qed
have after-trace-assert:
  hash-range-budget (0 + (length fl * ?B + 0))
  ((assert (f-x = f-final) ::
    (unit, ('f, 'a) protocol-channel-scheme) state-monad)  $\gg=$ 
    ( $\lambda$ -. (mfold
      (idx, cp-eval as fv (h ^ idx * shift), clength * scale, 1)
      (receive-query-commits fl) ::
      (nat  $\times$  'f  $\times$  nat  $\times$  nat,
        ('f, 'a) protocol-channel-scheme) state-monad)  $\gg=$ 
      ( $\lambda$ (i, x, len, pw). assert (x = final))))))
  for idx fv f-i f-x f-len f-pow
  by (rule hash-range-budget-bind
    [OF hash-range-budget-assert after-comp])
have after-trace:
  hash-range-budget
  (length f-fl * ?B + (0 + (length fl * ?B + 0)))
  ((mfold (idx, hd fv, clength * scale, 1)
    (receive-query-commits f-fl) ::
    (nat  $\times$  'f  $\times$  nat  $\times$  nat,
      ('f, 'a) protocol-channel-scheme) state-monad)  $\gg=$ 
    ( $\lambda$ (f-i, f-x, f-len, f-pow).
      assert (f-x = f-final)  $\gg=$ 
      ( $\lambda$ -. (mfold
        (idx, cp-eval as fv (h ^ idx * shift), clength * scale, 1)
        (receive-query-commits fl) ::
        (nat  $\times$  'f  $\times$  nat  $\times$  nat,
          ('f, 'a) protocol-channel-scheme) state-monad)  $\gg=$ 
        ( $\lambda$ (i, x, len, pw). assert (x = final))))))
  for idx fv
proof (rule hash-range-budget-bind)
show hash-range-budget (length f-fl * ?B)
  (mfold (idx, hd fv, clength * scale, 1)
    (receive-query-commits f-fl) ::
    (nat  $\times$  'f  $\times$  nat  $\times$  nat,
      ('f, 'a) protocol-channel-scheme) state-monad)
  unfolding B-eq
  by (rule hash-range-budget-receive-query-commits-mfold) simp
fix x :: nat  $\times$  'f  $\times$  nat  $\times$  nat
obtain f-i f-x f-len f-pow where x-eq:
  x = (f-i, f-x, f-len, f-pow)
  by (cases x) auto
have case-budget:
  hash-range-budget (0 + (length fl * ?B + 0))

```

```

((assert (f-x = f-final) ::
  (unit, ('f, 'a) protocol-channel-scheme) state-monad) >>=
  (λ-. (mfold
    (idx, cp-eval as fv (h ^ idx * shift), clength * scale, 1)
    (receive-query-commits fl) ::
    (nat × 'f × nat × nat,
      ('f, 'a) protocol-channel-scheme) state-monad) >>=
    (λ(i, x, len, pw). assert (x = final))))
by (rule hash-range-budget-bind
  [OF hash-range-budget-assert after-comp])
show hash-range-budget (0 + (length fl * ?B + 0))
((case x of (f-i, f-x, f-len, f-pow) ⇒
  assert (f-x = f-final) >>=
  (λ-. (mfold
    (idx, cp-eval as fv (h ^ idx * shift), clength * scale, 1)
    (receive-query-commits fl) ::
    (nat × 'f × nat × nat,
      ('f, 'a) protocol-channel-scheme) state-monad) >>=
    (λ(i, x, len, pw). assert (x = final)))) ::
  (unit, ('f, 'a) protocol-channel-scheme) state-monad)
using case-budget
unfolding x-eq
by (simp split: prod.splits)
qed
have after-query:
  hash-range-budget
    (verifier-query-decommit-hash-budget +
      (length f-fl * ?B + (0 + (length fl * ?B + 0))))
  (mmap (check-decommit-on-query fr idx) >>=
    (λfv. (mfold (idx, hd fv, clength * scale, 1)
      (receive-query-commits f-fl) ::
      (nat × 'f × nat × nat,
        ('f, 'a) protocol-channel-scheme) state-monad) >>=
      (λ(f-i, f-x, f-len, f-pow).
        assert (f-x = f-final) >>=
        (λ-. (mfold
          (idx, cp-eval as fv (h ^ idx * shift), clength * scale, 1)
          (receive-query-commits fl) ::
          (nat × 'f × nat × nat,
            ('f, 'a) protocol-channel-scheme) state-monad) >>=
          (λ(i, x, len, pw). assert (x = final)))))))
for idx
proof —
have decommit:
  hash-range-budget verifier-query-decommit-hash-budget
    (mmap (check-decommit-on-query fr idx))
proof —
have exact:
  hash-range-budget

```

```

    (powers * Suc (floor-log (scale * clength)))
    (mmap (check-decommit-on-query fr idx))
  by (rule hash-range-budget-check-decommit-on-query)
show ?thesis
  using exact
  unfolding verifier-query-decommit-hash-budget-def
  by (simp add: mult.commute)
qed
show ?thesis
  by (rule hash-range-budget-bind[OF decommit after-trace])
qed
have after-query-simple:
  hash-range-budget
    (verifier-query-decommit-hash-budget +
      (length f-fl * ?B + length fl * ?B))
    (mmap (check-decommit-on-query fr idx) >>=
      (λfv. (mfold (idx, hd fv, clength * scale, 1)
        (receive-query-commits f-fl) ::
          (nat × 'f × nat × nat,
            ('f, 'a) protocol-channel-scheme) state-monad) >>=
        (λ(f-i, f-x, f-len, f-pow).
          assert (f-x = f-final) >>=
            (λ-. (mfold
              (idx, cp-eval as fv (h ^ idx * shift), clength * scale, 1)
              (receive-query-commits fl) ::
                (nat × 'f × nat × nat,
                  ('f, 'a) protocol-channel-scheme) state-monad) >>=
                (λ(i, x, len, pw). assert (x = final)))))))
  for idx
  using after-query[of idx] by simp
have after-random:
  hash-range-budget
    (verifier-query-decommit-hash-budget +
      (length f-fl * ?B + length fl * ?B))
    ((let idx' = index (to-nat idx) in
      mmap (check-decommit-on-query fr idx') >>=
      (λfv. (mfold (idx', hd fv, clength * scale, 1)
        (receive-query-commits f-fl) ::
          (nat × 'f × nat × nat,
            ('f, 'a) protocol-channel-scheme) state-monad) >>=
        (λ(f-i, f-x, f-len, f-pow).
          assert (f-x = f-final) >>=
            (λ-. (mfold
              (idx', cp-eval as fv (h ^ idx' * shift), clength * scale, 1)
              (receive-query-commits fl) ::
                (nat × 'f × nat × nat,
                  ('f, 'a) protocol-channel-scheme) state-monad) >>=
                (λ(i, x, len, pw). assert (x = final)))))))
  for idx

```

```

using after-query-simple[of index (to-nat idx)]
by (simp add: Let-def algebra-simps mult.commute split: prod.splits)
have hash-range-budget
  (1 + (verifier-query-decommit-hash-budget +
    (length f-fl * ?B + length fl * ?B)))
  (receive-query-index-challenge  $\gg$ 
    ( $\lambda$ idx.
      let idx' = index (to-nat idx) in
      mmap (check-decommit-on-query fr idx')  $\gg$ 
      ( $\lambda$ fv. (mfold (idx', hd fv, clength * scale, 1)
        (receive-query-commits f-fl) ::
        (nat  $\times$  'f  $\times$  nat  $\times$  nat,
          ('f, 'a) protocol-channel-scheme) state-monad)  $\gg$ 
        ( $\lambda$ (f-i, f-x, f-len, f-pow).
          assert (f-x = f-final)  $\gg$ 
          ( $\lambda$ -. (mfold
            (idx', cp-eval as fv (h ^ idx' * shift), clength * scale, 1)
            (receive-query-commits fl) ::
            (nat  $\times$  'f  $\times$  nat  $\times$  nat,
              ('f, 'a) protocol-channel-scheme) state-monad)  $\gg$ 
            ( $\lambda$ (i, x, len, pw). assert (x = final)))))))
by (rule hash-range-budget-bind
  [OF hash-range-budget-receive-query-index-challenge]
  (rule after-random))
then show ?thesis
unfolding verifier-query-round-program-def
by (simp add: Let-def algebra-simps mult.commute split: prod.splits)
qed

```

lemma hash-target-program-verifier-query-round-program:

assumes len-f: length f-fl $\leq$ ceil-log clength
and len-c: length fl $\leq$ ceil-log (maxDegree + 1)

shows

hash-target-program B verifier-query-round-hash-budget
 (verifier-query-round-program fr f-fl f-final as fl final)

proof –

let ?N = verifier-fri-layer-hash-budget

have exact:

hash-target-program B
 (1 + verifier-query-decommit-hash-budget +
 (length f-fl + length fl) * ?N)
 (verifier-query-round-program fr f-fl f-final as fl final)
by (rule hash-target-program-verifier-query-round-program-exact)

have le:

1 + verifier-query-decommit-hash-budget +
 (length f-fl + length fl) * ?N
 $\leq$ verifier-query-round-hash-budget

using len-f len-c

unfolding verifier-query-round-hash-budget-def

```

    by simp
  show ?thesis
    by (rule hash-target-program-mono[OF le exact])
qed

lemma hash-range-budget-verifier-query-round-program:
  assumes len-f: length f-fl ≤ ceil-log clength
    and len-c: length fl ≤ ceil-log (maxDegree + 1)
  shows
    hash-range-budget verifier-query-round-hash-budget
      (verifier-query-round-program fr f-fl f-final as fl final)
proof -
  let ?B = verifier-fri-layer-hash-budget
  have exact:
    hash-range-budget
      (1 + verifier-query-decommit-hash-budget +
        (length f-fl + length fl) * ?B)
      (verifier-query-round-program fr f-fl f-final as fl final)
    by (rule hash-range-budget-verifier-query-round-program-exact)
  have le:
    1 + verifier-query-decommit-hash-budget +
      (length f-fl + length fl) * ?B
    ≤ verifier-query-round-hash-budget
  using len-f len-c
  unfolding verifier-query-round-hash-budget-def
  by simp
  show ?thesis
    by (rule hash-range-budget-mono[OF le exact])
qed

```

```

lemma hash-collision-budget-verifier-query-round-program-exact:
  hash-collision-budget
    (1 + verifier-query-decommit-hash-budget +
      (length f-fl + length fl) * verifier-fri-layer-hash-budget)
    (verifier-query-round-program fr f-fl f-final as fl final)
proof -
  let ?L = floor-log (clength * scale)
  let ?B = verifier-fri-layer-hash-budget
  have B-eq: ?B = Suc ?L + Suc ?L
    unfolding verifier-fri-layer-hash-budget-def by simp
  have after-comp-range:
    hash-range-budget (length fl * ?B + 0)
      ((mfold
        (idx, cp-eval as fv (h ^ idx * shift), clength * scale, 1)
        (receive-query-commits fl) ::
        (nat × 'f × nat × nat,
          ('f, 'a) protocol-channel-scheme) state-monad) >>=
        (λ(i, x, len, pw). assert (x = final))))
  for idx fv

```

```

proof (rule hash-range-budget-bind)
  show hash-range-budget (length fl * ?B)
    (mfold
      (idx, cp-eval as fv (h ^ idx * shift), clength * scale, 1)
      (receive-query-commits fl) ::
      (nat × 'f × nat × nat,
        ('f, 'a) protocol-channel-scheme) state-monad)
    unfolding B-eq
    by (rule hash-range-budget-receive-query-commits-mfold) simp
show  $\bigwedge x. \text{hash-range-budget } 0$ 
  ((case x of (i, x, len, pw)  $\Rightarrow$  assert (x = final)) ::
    (unit, ('f, 'a) protocol-channel-scheme) state-monad)
  by (simp add: hash-range-budget-assert split: prod.splits)
qed
have after-comp-coll:
  hash-collision-budget (length fl * ?B + 0)
    ((mfold
      (idx, cp-eval as fv (h ^ idx * shift), clength * scale, 1)
      (receive-query-commits fl) ::
      (nat × 'f × nat × nat,
        ('f, 'a) protocol-channel-scheme) state-monad)  $\gg$ 
      ( $\lambda(i, x, len, pw). \text{assert } (x = \text{final})$ )))
  for idx fv
proof (rule hash-collision-budget-bind)
  show hash-range-budget (length fl * ?B)
    (mfold
      (idx, cp-eval as fv (h ^ idx * shift), clength * scale, 1)
      (receive-query-commits fl) ::
      (nat × 'f × nat × nat,
        ('f, 'a) protocol-channel-scheme) state-monad)
    unfolding B-eq
    by (rule hash-range-budget-receive-query-commits-mfold) simp
show hash-collision-budget (length fl * ?B)
  (mfold
    (idx, cp-eval as fv (h ^ idx * shift), clength * scale, 1)
    (receive-query-commits fl) ::
    (nat × 'f × nat × nat,
      ('f, 'a) protocol-channel-scheme) state-monad)
    unfolding B-eq
    by (rule hash-collision-budget-receive-query-commits-mfold) simp
show  $\bigwedge x. \text{hash-range-budget } 0$ 
  ((case x of (i, x, len, pw)  $\Rightarrow$  assert (x = final)) ::
    (unit, ('f, 'a) protocol-channel-scheme) state-monad)
  by (simp add: hash-range-budget-assert split: prod.splits)
show  $\bigwedge x. \text{hash-collision-budget } 0$ 
  ((case x of (i, x, len, pw)  $\Rightarrow$  assert (x = final)) ::
    (unit, ('f, 'a) protocol-channel-scheme) state-monad)
  by (simp add: hash-collision-budget-assert split: prod.splits)
qed

```



```

have trace-cont-range:
  hash-range-budget (0 + (length fl * ?B + 0))
  ((assert (f-x = f-final) ::
    (unit, ('f, 'a) protocol-channel-scheme) state-monad) >>=
    (λ-. (mfold
      (idx, cp-eval as fv (h ^ idx * shift), clength * scale, 1)
      (receive-query-commits fl) ::
      (nat × 'f × nat × nat,
        ('f, 'a) protocol-channel-scheme) state-monad) >>=
      (λ(i, x, len, pw). assert (x = final))))))
for idx fv f-x
by (rule hash-range-budget-bind
  [OF hash-range-budget-assert after-comp-range])
have trace-cont-coll:
  hash-collision-budget (0 + (length fl * ?B + 0))
  ((assert (f-x = f-final) ::
    (unit, ('f, 'a) protocol-channel-scheme) state-monad) >>=
    (λ-. (mfold
      (idx, cp-eval as fv (h ^ idx * shift), clength * scale, 1)
      (receive-query-commits fl) ::
      (nat × 'f × nat × nat,
        ('f, 'a) protocol-channel-scheme) state-monad) >>=
      (λ(i, x, len, pw). assert (x = final))))))
for idx fv f-x
by (rule hash-collision-budget-bind
  [OF hash-range-budget-assert hash-collision-budget-assert
    after-comp-range after-comp-coll])
have after-trace-range:
  hash-range-budget
  (length f-fl * ?B + (0 + (length fl * ?B + 0)))
  ((mfold (idx, hd fv, clength * scale, 1)
    (receive-query-commits f-fl) ::
    (nat × 'f × nat × nat,
      ('f, 'a) protocol-channel-scheme) state-monad) >>=
    (λ(f-i, f-x, f-len, fpw).
      assert (f-x = f-final) >>=
      (λ-. (mfold
        (idx, cp-eval as fv (h ^ idx * shift), clength * scale, 1)
        (receive-query-commits fl) ::
        (nat × 'f × nat × nat,
          ('f, 'a) protocol-channel-scheme) state-monad) >>=
        (λ(i, x, len, pw). assert (x = final)))))))
for idx fv
proof (rule hash-range-budget-bind)
show hash-range-budget (length f-fl * ?B)
  (mfold (idx, hd fv, clength * scale, 1)
    (receive-query-commits f-fl) ::
    (nat × 'f × nat × nat,
      ('f, 'a) protocol-channel-scheme) state-monad)

```

```

unfolding B-eq
  by (rule hash-range-budget-receive-query-commits-mfold) simp
fix x :: nat × 'f × nat × nat
obtain f-i f-x f-len fpw where x-eq: x = (f-i, f-x, f-len, fpw)
  by (cases x) auto
have case-budget:
  hash-range-budget (0 + (length fl * ?B + 0))
    ((assert (f-x = f-final) ::
      (unit, ('f, 'a) protocol-channel-scheme) state-monad) ≫=
      (λ-. (mfold
        (idx, cp-eval as fv (h ^ idx * shift), clength * scale, 1)
        (receive-query-commits fl) ::
        (nat × 'f × nat × nat,
          ('f, 'a) protocol-channel-scheme) state-monad) ≫=
          (λ(i, x, len, pw). assert (x = final))))))
  by (rule trace-cont-range)
show hash-range-budget (0 + (length fl * ?B + 0))
  ((case x of (f-i, f-x, f-len, fpw) ⇒
    assert (f-x = f-final) ≫=
    (λ-. (mfold
      (idx, cp-eval as fv (h ^ idx * shift), clength * scale, 1)
      (receive-query-commits fl) ::
      (nat × 'f × nat × nat,
        ('f, 'a) protocol-channel-scheme) state-monad) ≫=
        (λ(i, x, len, pw). assert (x = final)))) ::
    (unit, ('f, 'a) protocol-channel-scheme) state-monad)
  using case-budget unfolding x-eq by (simp split: prod.splits)
qed
have after-trace-coll:
  hash-collision-budget
    (length f-fl * ?B + (0 + (length fl * ?B + 0)))
    ((mfold (idx, hd fv, clength * scale, 1)
      (receive-query-commits f-fl) ::
      (nat × 'f × nat × nat,
        ('f, 'a) protocol-channel-scheme) state-monad) ≫=
      (λ(f-i, f-x, f-len, fpw).
        assert (f-x = f-final) ≫=
        (λ-. (mfold
          (idx, cp-eval as fv (h ^ idx * shift), clength * scale, 1)
          (receive-query-commits fl) ::
          (nat × 'f × nat × nat,
            ('f, 'a) protocol-channel-scheme) state-monad) ≫=
            (λ(i, x, len, pw). assert (x = final))))))
  for idx fv
proof (rule hash-collision-budget-bind)
show hash-range-budget (length f-fl * ?B)
  (mfold (idx, hd fv, clength * scale, 1)
    (receive-query-commits f-fl) ::
    (nat × 'f × nat × nat,
```

```

    ('f, 'a) protocol-channel-scheme) state-monad)
  unfolding B-eq
  by (rule hash-range-budget-receive-query-commits-mfold) simp
show hash-collision-budget (length fl * ?B)
  (mfold (idx, hd fv, clength * scale, 1)
    (receive-query-commits fl) ::
    (nat × 'f × nat × nat,
      ('f, 'a) protocol-channel-scheme) state-monad)
  unfolding B-eq
  by (rule hash-collision-budget-receive-query-commits-mfold) simp
fix x :: nat × 'f × nat × nat
obtain f-i f-x f-len fpw where x-eq: x = (f-i, f-x, f-len, fpw)
  by (cases x) auto
have range-case:
  hash-range-budget (0 + (length fl * ?B + 0))
    ((assert (f-x = f-final) ::
      (unit, ('f, 'a) protocol-channel-scheme) state-monad) >>=
      (λ-. (mfold
        (idx, cp-eval as fv (h ^ idx * shift), clength * scale, 1)
        (receive-query-commits fl) ::
        (nat × 'f × nat × nat,
          ('f, 'a) protocol-channel-scheme) state-monad) >>=
        (λ(i, x, len, pw). assert (x = final))))))
  by (rule trace-cont-range)
show hash-range-budget (0 + (length fl * ?B + 0))
  ((case x of (f-i, f-x, f-len, fpw) ⇒
    assert (f-x = f-final) >>=
    (λ-. (mfold
      (idx, cp-eval as fv (h ^ idx * shift), clength * scale, 1)
      (receive-query-commits fl) ::
      (nat × 'f × nat × nat,
        ('f, 'a) protocol-channel-scheme) state-monad) >>=
      (λ(i, x, len, pw). assert (x = final)))))) ::
    (unit, ('f, 'a) protocol-channel-scheme) state-monad)
  using range-case unfolding x-eq by (simp split: prod.splits)
have coll-case:
  hash-collision-budget (0 + (length fl * ?B + 0))
    ((assert (f-x = f-final) ::
      (unit, ('f, 'a) protocol-channel-scheme) state-monad) >>=
      (λ-. (mfold
        (idx, cp-eval as fv (h ^ idx * shift), clength * scale, 1)
        (receive-query-commits fl) ::
        (nat × 'f × nat × nat,
          ('f, 'a) protocol-channel-scheme) state-monad) >>=
        (λ(i, x, len, pw). assert (x = final))))))
  by (rule trace-cont-coll)
show hash-collision-budget (0 + (length fl * ?B + 0))
  ((case x of (f-i, f-x, f-len, fpw) ⇒
    assert (f-x = f-final) >>=

```

```

(λ-. (mfold
  (idx, cp-eval as fv (h ^ idx * shift), clength * scale, 1)
  (receive-query-commits fl) ::
  (nat × 'f × nat × nat,
    ('f, 'a) protocol-channel-scheme) state-monad) >>=
  (λ(i, x, len, pw). assert (x = final)))) ::
  (unit, ('f, 'a) protocol-channel-scheme) state-monad)
using coll-case unfolding x-eq by (simp split: prod.splits)
qed
have after-query-range:
  hash-range-budget
  (verifier-query-decommit-hash-budget +
    (length f-fl * ?B + (0 + (length fl * ?B + 0))))
  (mmap (check-decommit-on-query fr idx) >>=
    (λfv. (mfold (idx, hd fv, clength * scale, 1)
      (receive-query-commits f-fl) ::
      (nat × 'f × nat × nat,
        ('f, 'a) protocol-channel-scheme) state-monad) >>=
      (λ(f-i, f-x, f-len, fpw).
        assert (f-x = f-final) >>=
        (λ-. (mfold
          (idx, cp-eval as fv (h ^ idx * shift), clength * scale, 1)
          (receive-query-commits fl) ::
          (nat × 'f × nat × nat,
            ('f, 'a) protocol-channel-scheme) state-monad) >>=
          (λ(i, x, len, pw). assert (x = final)))))))
for idx
proof -
have decommit:
  hash-range-budget verifier-query-decommit-hash-budget
  (mmap (check-decommit-on-query fr idx))
proof -
have exact:
  hash-range-budget
  (powers * Suc (floor-log (scale * clength)))
  (mmap (check-decommit-on-query fr idx))
by (rule hash-range-budget-check-decommit-on-query)
show ?thesis
using exact
unfolding verifier-query-decommit-hash-budget-def
by (simp add: mult.commute)
qed
show ?thesis
by (rule hash-range-budget-bind[OF decommit after-trace-range])
qed
have after-query-coll:
  hash-collision-budget
  (verifier-query-decommit-hash-budget +
    (length f-fl * ?B + (0 + (length fl * ?B + 0))))

```

```

(mmap (check-decommit-on-query fr idx) >>=
  (λfv. (mfold (idx, hd fv, clength * scale, 1)
    (receive-query-commits f-fl) ::
    (nat × 'f × nat × nat,
      ('f, 'a) protocol-channel-scheme) state-monad) >>=
    (λ(f-i, f-x, f-len, fpw).
      assert (f-x = f-final) >>=
      (λ-. (mfold
        (idx, cp-eval as fv (h ^ idx * shift), clength * scale, 1)
        (receive-query-commits fl) ::
        (nat × 'f × nat × nat,
          ('f, 'a) protocol-channel-scheme) state-monad) >>=
        (λ(i, x, len, pw). assert (x = final)))))))
for idx
proof -
have decommit-range:
  hash-range-budget verifier-query-decommit-hash-budget
  (mmap (check-decommit-on-query fr idx))
proof -
  have exact:
    hash-range-budget
    (powers * Suc (floor-log (scale * clength)))
    (mmap (check-decommit-on-query fr idx))
  by (rule hash-range-budget-check-decommit-on-query)
show ?thesis
  using exact
  unfolding verifier-query-decommit-hash-budget-def
  by (simp add: mult.commute)
qed
have decommit-coll:
  hash-collision-budget verifier-query-decommit-hash-budget
  (mmap (check-decommit-on-query fr idx))
proof -
  have exact:
    hash-collision-budget
    (powers * Suc (floor-log (scale * clength)))
    (mmap (check-decommit-on-query fr idx))
  by (rule hash-collision-budget-check-decommit-on-query)
show ?thesis
  using exact
  unfolding verifier-query-decommit-hash-budget-def
  by (simp add: mult.commute)
qed
show ?thesis
  by (rule hash-collision-budget-bind
    [OF decommit-range decommit-coll after-trace-range after-trace-coll])
qed
have after-query-range-simple:
  hash-range-budget

```

```

(verifier-query-decommit-hash-budget +
 (length f-fl * ?B + length fl * ?B))
(mmap (check-decommit-on-query fr idx) >>=
 (λfv. (mfold (idx, hd fv, clength * scale, 1)
 (receive-query-commits f-fl) ::
 (nat × 'f × nat × nat,
 ('f, 'a) protocol-channel-scheme) state-monad) >>=
 (λ(f-i, f-x, f-len, fpw).
 assert (f-x = f-final) >>=
 (λ-. (mfold
 (idx, cp-eval as fv (h ^ idx * shift), clength * scale, 1)
 (receive-query-commits fl) ::
 (nat × 'f × nat × nat,
 ('f, 'a) protocol-channel-scheme) state-monad) >>=
 (λ(i, x, len, pw). assert (x = final)))))))
for idx
using after-query-range[of idx] by simp
have after-query-coll-simple:
hash-collision-budget
(verifier-query-decommit-hash-budget +
 (length f-fl * ?B + length fl * ?B))
(mmap (check-decommit-on-query fr idx) >>=
 (λfv. (mfold (idx, hd fv, clength * scale, 1)
 (receive-query-commits f-fl) ::
 (nat × 'f × nat × nat,
 ('f, 'a) protocol-channel-scheme) state-monad) >>=
 (λ(f-i, f-x, f-len, fpw).
 assert (f-x = f-final) >>=
 (λ-. (mfold
 (idx, cp-eval as fv (h ^ idx * shift), clength * scale, 1)
 (receive-query-commits fl) ::
 (nat × 'f × nat × nat,
 ('f, 'a) protocol-channel-scheme) state-monad) >>=
 (λ(i, x, len, pw). assert (x = final)))))))
for idx
using after-query-coll[of idx] by simp
have after-random-range:
hash-range-budget
(verifier-query-decommit-hash-budget +
 (length f-fl * ?B + length fl * ?B))
((let idx' = index (to-nat idx) in
 mmap (check-decommit-on-query fr idx') >>=
 (λfv. (mfold (idx', hd fv, clength * scale, 1)
 (receive-query-commits f-fl) ::
 (nat × 'f × nat × nat,
 ('f, 'a) protocol-channel-scheme) state-monad) >>=
 (λ(f-i, f-x, f-len, fpw).
 assert (f-x = f-final) >>=
 (λ-. (mfold

```

```

      (idx', cp-eval as fv (h ^ idx' * shift), clength * scale, 1)
      (receive-query-commits fl) ::
      (nat × 'f × nat × nat,
        ('f, 'a) protocol-channel-scheme) state-monad) >>=
      (λ(i, x, len, pw). assert (x = final))))))
  for idx
  using after-query-range-simple[of index (to-nat idx)]
  by (simp add: Let-def algebra-simps mult.commute split: prod.splits)
have after-random-coll:
  hash-collision-budget
  (verifier-query-decommit-hash-budget +
    (length f-fl * ?B + length fl * ?B))
  ((let idx' = index (to-nat idx) in
    mmap (check-decommit-on-query fr idx') >>=
    (λfv. (mfold (idx', hd fv, clength * scale, 1)
      (receive-query-commits f-fl) ::
      (nat × 'f × nat × nat,
        ('f, 'a) protocol-channel-scheme) state-monad) >>=
      (λ(f-i, f-x, f-len, fpw).
        assert (f-x = f-final) >>=
        (λ-. (mfold
          (idx', cp-eval as fv (h ^ idx' * shift), clength * scale, 1)
          (receive-query-commits fl) ::
          (nat × 'f × nat × nat,
            ('f, 'a) protocol-channel-scheme) state-monad) >>=
          (λ(i, x, len, pw). assert (x = final))))))))
  for idx
  using after-query-coll-simple[of index (to-nat idx)]
  by (simp add: Let-def algebra-simps mult.commute split: prod.splits)
have hash-collision-budget
  (1 + (verifier-query-decommit-hash-budget +
    (length f-fl * ?B + length fl * ?B)))
  (receive-query-index-challenge >>=
    (λidx.
      let idx' = index (to-nat idx) in
      mmap (check-decommit-on-query fr idx') >>=
      (λfv. (mfold (idx', hd fv, clength * scale, 1)
        (receive-query-commits f-fl) ::
        (nat × 'f × nat × nat,
          ('f, 'a) protocol-channel-scheme) state-monad) >>=
        (λ(f-i, f-x, f-len, fpw).
          assert (f-x = f-final) >>=
          (λ-. (mfold
            (idx', cp-eval as fv (h ^ idx' * shift), clength * scale, 1)
            (receive-query-commits fl) ::
            (nat × 'f × nat × nat,
              ('f, 'a) protocol-channel-scheme) state-monad) >>=
            (λ(i, x, len, pw). assert (x = final))))))))
  by (rule hash-collision-budget-bind

```

```

    [OF hash-range-budget-receive-query-index-challenge
     hash-collision-budget-receive-query-index-challenge])
  (rule after-random-range, rule after-random-coll)
then show ?thesis
  unfolding verifier-query-round-program-def
  by (simp add: Let-def algebra-simps mult.commute split: prod.splits)
qed

lemma hash-collision-budget-verifier-query-round-program:
  assumes len-f: length f-fl ≤ ceil-log clength
    and len-c: length fl ≤ ceil-log (maxDegree + 1)
  shows
    hash-collision-budget verifier-query-round-hash-budget
      (verifier-query-round-program fr f-fl f-final as fl final)
proof -
  let ?B = verifier-fri-layer-hash-budget
  have exact:
    hash-collision-budget
      (1 + verifier-query-decommit-hash-budget +
       (length f-fl + length fl) * ?B)
      (verifier-query-round-program fr f-fl f-final as fl final)
  by (rule hash-collision-budget-verifier-query-round-program-exact)
  have le:
    1 + verifier-query-decommit-hash-budget +
      (length f-fl + length fl) * ?B
    ≤ verifier-query-round-hash-budget
  using len-f len-c
  unfolding verifier-query-round-hash-budget-def
  by simp
  show ?thesis
  by (rule hash-collision-budget-mono[OF le exact])
qed

lemma ceil-log-to-nat-degree-bound:
  assumes to-nat dg ≤ maxDegree
  shows ceil-log (to-nat dg + 1) ≤ ceil-log (maxDegree + 1)
  by (rule ceil-log-mono) (use assms in simp)

lemma hash-range-budget-ntimes-receive-fri-commits-bound:
  assumes n ≤ N
  shows hash-range-budget N (ntimes receive-fri-commits n)
proof -
  have exact: hash-range-budget (n * 1) (ntimes receive-fri-commits n)
  by (rule hash-range-budget-ntimes[OF hash-range-budget-receive-fri-commits])
  show ?thesis
  by (rule hash-range-budget-mono[OF - exact]) (use assms in simp)
qed

lemma hash-target-program-ntimes-receive-trace-fri-commits-bound:

```



```

assumes  $n \leq N$ 
shows hash-target-program B N (ntimes receive-trace-fri-commits n)
proof –
  have exact:
    hash-target-program B (n * 1) (ntimes receive-trace-fri-commits n)
  by (rule hash-target-program-ntimes
    [OF hash-target-program-receive-trace-fri-commits])
  show ?thesis
  by (rule hash-target-program-mono[OF - exact]) (use assms in simp)
qed

lemma hash-target-program-ntimes-receive-composition-fri-commits-bound:
  assumes  $n \leq N$ 
  shows hash-target-program B N (ntimes receive-composition-fri-commits n)
proof –
  have exact:
    hash-target-program B (n * 1) (ntimes receive-composition-fri-commits n)
  by (rule hash-target-program-ntimes
    [OF hash-target-program-receive-composition-fri-commits])
  show ?thesis
  by (rule hash-target-program-mono[OF - exact]) (use assms in simp)
qed

lemma hash-target-program-alpha-round:
  hash-target-program B 1 alpha-round
unfolding alpha-round-def
by (rule hash-target-program-alpha-challenge-step)

lemma hash-target-program-verify-monad:
  hash-target-program B verifier-hash-query-budget verify-monad
proof –
  let ?Q = rounds * verifier-query-round-hash-budget
  have alpha-map-range:
    hash-target-program B (length spec)
    (mmap (replicate (length spec)
      (alpha-round ::
        (f, (f, 'a) protocol-channel-scheme) state-monad)))
  proof –
    let ?steps =
      replicate (length spec)
      (alpha-round ::
        (f, (f, 'a) protocol-channel-scheme) state-monad)
    have step-range:
       $\bigwedge m. m \in \text{set } ?steps \implies \text{hash-target-program } B \ 1 \ m$ 
  proof –
    fix m
    assume  $m \in \text{set } ?steps$ 
    then have m-eq:
       $m =$ 

```

```

    (alpha-round ::
      ('f, ('f, 'a) protocol-channel-scheme) state-monad)
  by simp
show hash-target-program B 1 m
  unfolding m-eq
  by (rule hash-target-program-alpha-round)
qed
have hash-target-program B (length ?steps * 1) (mmap ?steps)
  by (rule hash-target-program-mmap)
  (rule step-range)
then show ?thesis by simp
qed
have after-final:
  hash-target-program B (0 + ?Q)
  (read  $\gg$ 
    ( $\lambda$ final. ntimes
      (verifier-query-round-program fr f-fl f-final as fl final)
      rounds))
  if len-f: length f-fl  $\leq$  ceil-log clength
  and len-fl: length fl  $\leq$  ceil-log (maxDegree + 1)
  for fr f-fl f-final as fl
  by (rule hash-target-program-bind
    [OF hash-target-program-read
      hash-target-program-ntimes
      [OF hash-target-program-verifier-query-round-program
        [OF len-f len-fl]]])
have after-comp-fri:
  hash-target-program B (ceil-log (maxDegree + 1) + (0 + ?Q))
  ((ntimes receive-composition-fri-commits (ceil-log (to-nat dg + 1)) ::
    (((f  $\times$  'f) list),
      ('f, 'a) protocol-channel-scheme) state-monad)  $\gg$ 
    ( $\lambda$ fl. read  $\gg$ 
      ( $\lambda$ final. ntimes
        (verifier-query-round-program fr f-fl f-final as fl final)
        rounds))))
  if len-f: length f-fl  $\leq$  ceil-log clength
  and dg-bound: to-nat dg  $\leq$  maxDegree
  for fr f-fl f-final as dg
proof (rule hash-target-program-bind-on-outcomes)
show hash-target-program B (ceil-log (maxDegree + 1))
  (ntimes receive-composition-fri-commits (ceil-log (to-nat dg + 1)) ::
    (((f  $\times$  'f) list),
      ('f, 'a) protocol-channel-scheme) state-monad)
  by (rule hash-target-program-ntimes-receive-composition-fri-commits-bound)
  (rule ceil-log-to-nat-degree-bound[OF dg-bound])
fix s fl t
assume out:
  Some (fl, t)  $\in$ 
  set-dist (execute

```

```

      (ntimes receive-composition-fri-commits (ceil-log (to-nat dg + 1)) ::
        (((f × 'f) list),
         ('f, 'a) protocol-channel-scheme) state-monad) s)
have len-fl: length fl ≤ ceil-log (maxDegree + 1)
  using ntimes-receive-composition-fri-commits-outcome[OF out]
    ceil-log-to-nat-degree-bound[OF dg-bound]
  by simp
show hash-target-program B (0 + ?Q)
  (read ≥=
    (λfinal. ntimes
      (verifier-query-round-program fr f-fl f-final as fl final)
      rounds))
  by (rule after-final[OF len-f len-fl])
qed
have after-degree-assert:
  hash-target-program B (0 + (ceil-log (maxDegree + 1) + (0 + ?Q)))
  (assert (to-nat dg ≤ maxDegree) ≥=
    (λ-. (ntimes receive-composition-fri-commits (ceil-log (to-nat dg + 1)) ::
      (((f × 'f) list),
       ('f, 'a) protocol-channel-scheme) state-monad) ≥=
      (λfl. read ≥=
        (λfinal. ntimes
          (verifier-query-round-program fr f-fl f-final as fl final)
          rounds))))))
  if len-f: length f-fl ≤ ceil-log clength
  for fr f-fl f-final as dg
proof (rule hash-target-program-bind-on-outcomes)
  show hash-target-program B 0
    (assert (to-nat dg ≤ maxDegree) ::
      (unit, ('f, 'a) protocol-channel-scheme) state-monad)
    by (rule hash-target-program-assert)
  fix s u t
  assume out:
    Some (u, t) ∈
      set-dist (execute
        (assert (to-nat dg ≤ maxDegree) ::
          (unit, ('f, 'a) protocol-channel-scheme) state-monad) s)
  have dg-bound: to-nat dg ≤ maxDegree
    using out unfolding assert-def
    by (cases to-nat dg ≤ maxDegree) (auto simp: throw-no-outcome)
  show hash-target-program B (ceil-log (maxDegree + 1) + (0 + ?Q))
    ((ntimes receive-composition-fri-commits (ceil-log (to-nat dg + 1)) ::
      (((f × 'f) list),
       ('f, 'a) protocol-channel-scheme) state-monad) ≥=
      (λfl. read ≥=
        (λfinal. ntimes
          (verifier-query-round-program fr f-fl f-final as fl final)
          rounds))))
    by (rule after-comp-fri[OF len-f dg-bound])

```

```

qed
have after-read-dg:
  hash-target-program B
  (0 + (0 + (ceil-log (maxDegree + 1) + (0 + ?Q))))
  (read  $\gg$ 
    ( $\lambda dg$ . assert (to-nat dg  $\leq$  maxDegree)  $\gg$ 
      ( $\lambda$ -. (ntimes receive-composition-fri-commits (ceil-log (to-nat dg + 1)) ::
        (((f  $\times$  'f) list),
          ('f, 'a) protocol-channel-scheme) state-monad)  $\gg$ 
        ( $\lambda fl$ . read  $\gg$ 
          ( $\lambda final$ . ntimes
            (verifier-query-round-program fr f-fl f-final as fl final)
            rounds))))))
  if len-f: length f-fl  $\leq$  ceil-log clength
  for fr f-fl f-final as
  by (rule hash-target-program-bind
    [OF hash-target-program-read after-degree-assert[OF len-f]])
have after-alpha:
  hash-target-program B
  (length spec +
    (0 + (0 + (ceil-log (maxDegree + 1) + (0 + ?Q))))))
  (mmap (replicate (length spec) alpha-round)  $\gg$ 
    ( $\lambda as$ . read  $\gg$ 
      ( $\lambda dg$ . assert (to-nat dg  $\leq$  maxDegree)  $\gg$ 
        ( $\lambda$ -. (ntimes receive-composition-fri-commits (ceil-log (to-nat dg + 1)) ::
          (((f  $\times$  'f) list),
            ('f, 'a) protocol-channel-scheme) state-monad)  $\gg$ 
            ( $\lambda fl$ . read  $\gg$ 
              ( $\lambda final$ . ntimes
                (verifier-query-round-program fr f-fl f-final as fl final)
                rounds))))))
      if len-f: length f-fl  $\leq$  ceil-log clength
      for fr f-fl f-final
      by (rule hash-target-program-bind
        [OF alpha-map-range after-read-dg[OF len-f]])
have after-read-f-final:
  hash-target-program B
  (0 + (length spec +
    (0 + (0 + (ceil-log (maxDegree + 1) + (0 + ?Q))))))
  (read  $\gg$ 
    ( $\lambda f$ -final. mmap (replicate (length spec) alpha-round)  $\gg$ 
      ( $\lambda as$ . read  $\gg$ 
        ( $\lambda dg$ . assert (to-nat dg  $\leq$  maxDegree)  $\gg$ 
          ( $\lambda$ -. (ntimes receive-composition-fri-commits (ceil-log (to-nat dg + 1)) ::
            (((f  $\times$  'f) list),
              ('f, 'a) protocol-channel-scheme) state-monad)  $\gg$ 
              ( $\lambda fl$ . read  $\gg$ 
                ( $\lambda final$ . ntimes
                  (verifier-query-round-program fr f-fl f-final as fl final)
                  rounds))))))
          
```

```

rounds))))))
if len-f: length f-fl ≤ ceil-log clength
for fr f-fl
by (rule hash-target-program-bind
  [OF hash-target-program-read after-alpha[OF len-f]])
have after-trace-fri:
  hash-target-program B
  (ceil-log clength +
    (0 + (length spec +
      (0 + (0 + (ceil-log (maxDegree + 1) + (0 + ?Q)))))))
  ((ntimes receive-trace-fri-commits (ceil-log clength) ::
    (((f × 'f) list),
      (f, 'a) protocol-channel-scheme) state-monad) >>=
    (λf-fl. read >>=
      (λf-final. mmap (replicate (length spec) alpha-round) >>=
        (λas. read >>=
          (λdg. assert (to-nat dg ≤ maxDegree) >>=
            (λ-. (ntimes receive-composition-fri-commits (ceil-log (to-nat dg + 1)) ::
              (((f × 'f) list),
                (f, 'a) protocol-channel-scheme) state-monad) >>=
              (λfl. read >>=
                (λfinal. ntimes
                  (verifier-query-round-program fr f-fl f-final as fl final)
                  rounds))))))))))
for fr
proof (rule hash-target-program-bind-on-outcomes)
show hash-target-program B (ceil-log clength)
  (ntimes receive-trace-fri-commits (ceil-log clength) ::
    (((f × 'f) list),
      (f, 'a) protocol-channel-scheme) state-monad)
by (rule hash-target-program-ntimes-receive-trace-fri-commits-bound) simp
fix s f-fl t
assume out:
  Some (f-fl, t) ∈
    set-dist (execute
      (ntimes receive-trace-fri-commits (ceil-log clength) ::
        (((f × 'f) list),
          (f, 'a) protocol-channel-scheme) state-monad) s)
have len-f: length f-fl ≤ ceil-log clength
using ntimes-receive-trace-fri-commits-outcome[OF out] by simp
show hash-target-program B
  (0 + (length spec +
    (0 + (0 + (ceil-log (maxDegree + 1) + (0 + ?Q))))))
  (read >>=
    (λf-final. mmap (replicate (length spec) alpha-round) >>=
      (λas. read >>=
        (λdg. assert (to-nat dg ≤ maxDegree) >>=
          (λ-. (ntimes receive-composition-fri-commits (ceil-log (to-nat dg + 1)) ::
            (((f × 'f) list),
              (f, 'a) protocol-channel-scheme) state-monad) >>=
            (λfl. read >>=
              (λfinal. ntimes
                (verifier-query-round-program fr f-fl f-final as fl final)
                rounds))))))))))

```

```

      ('f, 'a) protocol-channel-scheme) state-monad) >>=
    (λfl. read >>=
      (λfinal. ntimes
        (verifier-query-round-program fr f-fl f-final as fl final)
        rounds))))))
  by (rule after-read-f-final[OF len-f])
qed
have top:
  hash-target-program B
  (0 + (ceil-log clength +
    (0 + (length spec +
      (0 + (0 + (ceil-log (maxDegree + 1) + (0 + ?Q))))))))
  (read >>=
    (λfr. (ntimes receive-trace-fri-commits (ceil-log clength) ::
      (((f × 'f) list),
        ('f, 'a) protocol-channel-scheme) state-monad) >>=
      (λf-fl. read >>=
        (λf-final. mmap (replicate (length spec) alpha-round) >>=
          (λas. read >>=
            (λdg. assert (to-nat dg ≤ maxDegree) >>=
              (λ-. (ntimes receive-composition-fri-commits (ceil-log (to-nat dg + 1)) ::
                (((f × 'f) list),
                  ('f, 'a) protocol-channel-scheme) state-monad) >>=
                  (λfl. read >>=
                    (λfinal. ntimes
                      (verifier-query-round-program fr f-fl f-final as fl final)
                      rounds))))))))))
    by (rule hash-target-program-bind
      [OF hash-target-program-read after-trace-fri])
  show ?thesis
  using top
  unfolding verify-monad-def verifier-query-round-program-def alpha-round-def
    verifier-hash-query-budget-def verifier-header-hash-budget-def
  by (simp add: algebra-simps)
qed

```

lemma *hash-target-budget-verify-monad:*
hash-target-budget B verifier-hash-query-budget verify-monad
by (rule *hash-target-program-budget*[OF *hash-target-program-verify-monad*])

lemma *wp-verify-monad-hash-new-output-hit-bound:*
wp-event verify-monad (hash-new-output-hit-event B s) s ≤
hash-target-budget-value B verifier-hash-query-budget
using *hash-target-budget-verify-monad*
unfolding *hash-target-budget-def* **by** *blast*

lemma *wp-verify-monad-transcript-hash-target-hit-bound:*
wp-event verify-monad
(hash-new-output-hit-event (set tr) (verifier-initial-state tr))

```

      (verifier-initial-state tr)
    ≤ nnreal (card (set tr) * verifier-hash-query-budget) / nnreal size
using wp-verify-monad-hash-new-output-hit-bound
    [of set tr verifier-initial-state tr]
unfolding hash-target-budget-value-def
by (simp add: mult.commute)

```

lemma *hash-range-budget-ntimes-receive-trace-fri-commits-bound*:

```

assumes  $n \leq N$ 
shows hash-range-budget  $N$  (ntimes receive-trace-fri-commits  $n$ )
proof –
  have exact:
    hash-range-budget ( $n * 1$ ) (ntimes receive-trace-fri-commits  $n$ )
  by (rule hash-range-budget-ntimes
    [OF hash-range-budget-receive-trace-fri-commits])
  show ?thesis
    by (rule hash-range-budget-mono[OF - exact]) (use assms in simp)
qed

```

lemma *hash-range-budget-ntimes-receive-composition-fri-commits-bound*:

```

assumes  $n \leq N$ 
shows hash-range-budget  $N$  (ntimes receive-composition-fri-commits  $n$ )
proof –
  have exact:
    hash-range-budget ( $n * 1$ ) (ntimes receive-composition-fri-commits  $n$ )
  by (rule hash-range-budget-ntimes
    [OF hash-range-budget-receive-composition-fri-commits])
  show ?thesis
    by (rule hash-range-budget-mono[OF - exact]) (use assms in simp)
qed

```

lemma *hash-range-budget-alpha-round*:

```

hash-range-budget 1 alpha-round
unfolding alpha-round-def
by (rule hash-range-budget-alpha-challenge-step)

```

lemma *hash-range-budget-verify-monad*:

```

hash-range-budget verifier-hash-query-budget verify-monad

```

proof –

```

let ?Q = rounds * verifier-query-round-hash-budget
have alpha-map-range:
  hash-range-budget (length spec)
    (mmap (replicate (length spec)
      (alpha-round ::
        ('f, ('f, 'a) protocol-channel-scheme) state-monad)))

```

proof –

```

let ?steps =
  replicate (length spec)

```

```

(alpha-round ::
  ('f, ('f, 'a) protocol-channel-scheme) state-monad)
have step-range:
   $\bigwedge m. m \in \text{set } ?\text{steps} \implies \text{hash-range-budget } 1 \ m$ 
proof –
  fix m
  assume m  $\in \text{set } ?\text{steps}$ 
  then have m-eq:
    m =
      (alpha-round ::
        ('f, ('f, 'a) protocol-channel-scheme) state-monad)
  by simp
  show hash-range-budget 1 m
  unfolding m-eq
  by (rule hash-range-budget-alpha-round)
qed
have hash-range-budget (length ?steps * 1) (mmap ?steps)
  by (rule hash-range-budget-mmap)
  (rule step-range)
then show ?thesis by simp
qed
have after-final:
  hash-range-budget (0 + ?Q)
  (read  $\gg=$ 
    ( $\lambda \text{final}. \text{ntimes}$ 
      (verifier-query-round-program fr f-fl f-final as fl final)
      rounds))
  if len-f: length f-fl  $\leq \text{ceil-log } \text{clength}$ 
  and len-fl: length fl  $\leq \text{ceil-log } (\text{maxDegree} + 1)$ 
  for fr f-fl f-final as fl
  by (rule hash-range-budget-bind
    [OF hash-range-budget-read
      hash-range-budget-ntimes
      [OF hash-range-budget-verifier-query-round-program
        [OF len-f len-fl]]])
have after-comp-fri:
  hash-range-budget (ceil-log (maxDegree + 1) + (0 + ?Q))
  ((ntimes receive-composition-fri-commits (ceil-log (to-nat dg + 1)) ::
    (((f  $\times$  'f) list),
      ('f, 'a) protocol-channel-scheme) state-monad)  $\gg=$ 
    ( $\lambda \text{fl}. \text{read } \gg=$ 
      ( $\lambda \text{final}. \text{ntimes}$ 
        (verifier-query-round-program fr f-fl f-final as fl final)
        rounds)))
  if len-f: length f-fl  $\leq \text{ceil-log } \text{clength}$ 
  and dg-bound: to-nat dg  $\leq \text{maxDegree}$ 
  for fr f-fl f-final as dg
proof (rule hash-range-budget-bind-on-outcomes)
show hash-range-budget (ceil-log (maxDegree + 1))

```



```

  (ntimes receive-composition-fri-commits (ceil-log (to-nat dg + 1)) ::
    (((f × 'f) list),
     (f, 'a) protocol-channel-scheme) state-monad)
  by (rule hash-range-budget-ntimes-receive-composition-fri-commits-bound)
    (rule ceil-log-to-nat-degree-bound[OF dg-bound])
fix s fl t
assume out:
  Some (fl, t) ∈
    set-dist (execute
      (ntimes receive-composition-fri-commits (ceil-log (to-nat dg + 1)) ::
        (((f × 'f) list),
         (f, 'a) protocol-channel-scheme) state-monad) s)
have len-fl: length fl ≤ ceil-log (maxDegree + 1)
  using ntimes-receive-composition-fri-commits-outcome[OF out]
    ceil-log-to-nat-degree-bound[OF dg-bound]
  by simp
show hash-range-budget (0 + ?Q)
  (read >>=
    (λfinal. ntimes
      (verifier-query-round-program fr f-fl f-final as fl final)
      rounds))
  by (rule after-final[OF len-f len-fl])
qed
have after-degree-assert:
  hash-range-budget (0 + (ceil-log (maxDegree + 1) + (0 + ?Q)))
  (assert (to-nat dg ≤ maxDegree) >>=
    (λ-. (ntimes receive-composition-fri-commits (ceil-log (to-nat dg + 1)) ::
      (((f × 'f) list),
       (f, 'a) protocol-channel-scheme) state-monad) >>=
      (λfl. read >>=
        (λfinal. ntimes
          (verifier-query-round-program fr f-fl f-final as fl final)
          rounds))))))
if len-f: length f-fl ≤ ceil-log clength
for fr f-fl f-final as dg
proof (rule hash-range-budget-bind-on-outcomes)
show hash-range-budget 0
  (assert (to-nat dg ≤ maxDegree) ::
    (unit, (f, 'a) protocol-channel-scheme) state-monad)
  by (rule hash-range-budget-assert)
fix s u t
assume out:
  Some (u, t) ∈
    set-dist (execute
      (assert (to-nat dg ≤ maxDegree) ::
        (unit, (f, 'a) protocol-channel-scheme) state-monad) s)
have dg-bound: to-nat dg ≤ maxDegree
  using out unfolding assert-def
  by (cases to-nat dg ≤ maxDegree) (auto simp: throw-no-outcome)

```

```

show hash-range-budget (ceil-log (maxDegree + 1) + (0 + ?Q))
  ((ntimes receive-composition-fri-commits (ceil-log (to-nat dg + 1)) ::
    (((f × 'f) list),
     (f, 'a) protocol-channel-scheme) state-monad) ≫=
    (λfl. read ≫=
      (λfinal. ntimes
        (verifier-query-round-program fr f-fl f-final as fl final)
        rounds))))
  by (rule after-comp-fri[OF len-f dg-bound])
qed
have after-read-dg:
  hash-range-budget
    (0 + (0 + (ceil-log (maxDegree + 1) + (0 + ?Q))))
    (read ≫=
      (λdg. assert (to-nat dg ≤ maxDegree) ≫=
        (λ-. (ntimes receive-composition-fri-commits (ceil-log (to-nat dg + 1)) ::
          (((f × 'f) list),
           (f, 'a) protocol-channel-scheme) state-monad) ≫=
            (λfl. read ≫=
              (λfinal. ntimes
                (verifier-query-round-program fr f-fl f-final as fl final)
                rounds))))))
  if len-f: length f-fl ≤ ceil-log clength
  for fr f-fl f-final as
  by (rule hash-range-budget-bind
    [OF hash-range-budget-read after-degree-assert[OF len-f]])
have after-alpha:
  hash-range-budget
    (length spec +
     (0 + (0 + (ceil-log (maxDegree + 1) + (0 + ?Q)))))
    (mmap (replicate (length spec) alpha-round) ≫=
      (λas. read ≫=
        (λdg. assert (to-nat dg ≤ maxDegree) ≫=
          (λ-. (ntimes receive-composition-fri-commits (ceil-log (to-nat dg + 1)) ::
            (((f × 'f) list),
             (f, 'a) protocol-channel-scheme) state-monad) ≫=
              (λfl. read ≫=
                (λfinal. ntimes
                  (verifier-query-round-program fr f-fl f-final as fl final)
                  rounds))))))
  if len-f: length f-fl ≤ ceil-log clength
  for fr f-fl f-final
  by (rule hash-range-budget-bind
    [OF alpha-map-range after-read-dg[OF len-f]])
have after-read-f-final:
  hash-range-budget
    (0 + (length spec +
     (0 + (0 + (ceil-log (maxDegree + 1) + (0 + ?Q)))))
    (read ≫=

```

```

(λf-final. mmap (replicate (length spec) alpha-round) >>=
  (λas. read >>=
    (λdg. assert (to-nat dg ≤ maxDegree) >>=
      (λ-. (ntimes receive-composition-fri-commits (ceil-log (to-nat dg + 1)) ::
        (((f × 'f) list),
          ('f, 'a) protocol-channel-scheme) state-monad) >>=
        (λfl. read >>=
          (λfinal. ntimes
            (verifier-query-round-program fr f-fl f-final as fl final)
            rounds)))))))
if len-f: length f-fl ≤ ceil-log clength
for fr f-fl
by (rule hash-range-budget-bind
  [OF hash-range-budget-read after-alpha[OF len-f]])
have after-trace-fri:
  hash-range-budget
    (ceil-log clength +
      (0 + (length spec +
        (0 + (0 + (ceil-log (maxDegree + 1) + (0 + ?Q)))))))
    ((ntimes receive-trace-fri-commits (ceil-log clength) ::
      (((f × 'f) list),
        ('f, 'a) protocol-channel-scheme) state-monad) >>=
      (λf-fl. read >>=
        (λf-final. mmap (replicate (length spec) alpha-round) >>=
          (λas. read >>=
            (λdg. assert (to-nat dg ≤ maxDegree) >>=
              (λ-. (ntimes receive-composition-fri-commits (ceil-log (to-nat dg + 1)) ::
                (((f × 'f) list),
                  ('f, 'a) protocol-channel-scheme) state-monad) >>=
                (λfl. read >>=
                  (λfinal. ntimes
                    (verifier-query-round-program fr f-fl f-final as fl final)
                    rounds))))))))))
for fr
proof (rule hash-range-budget-bind-on-outcomes)
show hash-range-budget (ceil-log clength)
  (ntimes receive-trace-fri-commits (ceil-log clength) ::
    (((f × 'f) list),
      ('f, 'a) protocol-channel-scheme) state-monad)
by (rule hash-range-budget-ntimes-receive-trace-fri-commits-bound) simp
fix s f-fl t
assume out:
  Some (f-fl, t) ∈
    set-dist (execute
      (ntimes receive-trace-fri-commits (ceil-log clength) ::
        (((f × 'f) list),
          ('f, 'a) protocol-channel-scheme) state-monad) s)
have len-f: length f-fl ≤ ceil-log clength
using ntimes-receive-trace-fri-commits-outcome[OF out] by simp

```

```

show hash-range-budget
  (0 + (length spec +
    (0 + (0 + (ceil-log (maxDegree + 1) + (0 + ?Q))))))
  (read >=
    (λf-final. mmap (replicate (length spec) alpha-round) >=
      (λas. read >=
        (λdg. assert (to-nat dg ≤ maxDegree) >=
          (λ-. (ntimes receive-composition-fri-commits (ceil-log (to-nat dg + 1)) ::
            (((f × 'f) list),
              (f, 'a) protocol-channel-scheme) state-monad) >=
            (λfl. read >=
              (λfinal. ntimes
                (verifier-query-round-program fr f-fl f-final as fl final)
                rounds))))))))
    by (rule after-read-f-final[OF len-f])
qed
have top:
  hash-range-budget
    (0 + (ceil-log clength +
      (0 + (length spec +
        (0 + (0 + (ceil-log (maxDegree + 1) + (0 + ?Q)))))))
    (read >=
      (λfr. (ntimes receive-trace-fri-commits (ceil-log clength) ::
        (((f × 'f) list),
          (f, 'a) protocol-channel-scheme) state-monad) >=
      (λf-fl. read >=
        (λf-final. mmap (replicate (length spec) alpha-round) >=
          (λas. read >=
            (λdg. assert (to-nat dg ≤ maxDegree) >=
              (λ-. (ntimes receive-composition-fri-commits (ceil-log (to-nat dg + 1)) ::
                (((f × 'f) list),
                  (f, 'a) protocol-channel-scheme) state-monad) >=
                (λfl. read >=
                  (λfinal. ntimes
                    (verifier-query-round-program fr f-fl f-final as fl final)
                    rounds))))))))
            by (rule hash-range-budget-bind
              [OF hash-range-budget-read after-trace-fri])
show ?thesis
  using top
  unfolding verify-monad-def verifier-query-round-program-def alpha-round-def
    verifier-hash-query-budget-def verifier-header-hash-budget-def
  by (simp add: algebra-simps)
qed

lemma hash-collision-budget-ntimes-receive-fri-commits-bound:
  assumes  $n \leq N$ 
  shows hash-collision-budget  $N$  (ntimes receive-fri-commits  $n$ )
proof –

```

```

have exact: hash-collision-budget ( $n * 1$ ) (ntimes receive-fri-commits  $n$ )
  by (rule hash-collision-budget-ntimes
    [OF hash-range-budget-receive-fri-commits
      hash-collision-budget-receive-fri-commits])
show ?thesis
  by (rule hash-collision-budget-mono[OF - exact]) (use assms in simp)
qed

lemma hash-collision-budget-ntimes-receive-trace-fri-commits-bound:
  assumes  $n \leq N$ 
  shows hash-collision-budget  $N$  (ntimes receive-trace-fri-commits  $n$ )
proof –
  have exact:
    hash-collision-budget ( $n * 1$ ) (ntimes receive-trace-fri-commits  $n$ )
  by (rule hash-collision-budget-ntimes
    [OF hash-range-budget-receive-trace-fri-commits
      hash-collision-budget-receive-trace-fri-commits])
show ?thesis
  by (rule hash-collision-budget-mono[OF - exact]) (use assms in simp)
qed

lemma hash-collision-budget-ntimes-receive-composition-fri-commits-bound:
  assumes  $n \leq N$ 
  shows hash-collision-budget  $N$  (ntimes receive-composition-fri-commits  $n$ )
proof –
  have exact:
    hash-collision-budget ( $n * 1$ )
      (ntimes receive-composition-fri-commits  $n$ )
  by (rule hash-collision-budget-ntimes
    [OF hash-range-budget-receive-composition-fri-commits
      hash-collision-budget-receive-composition-fri-commits])
show ?thesis
  by (rule hash-collision-budget-mono[OF - exact]) (use assms in simp)
qed

lemma hash-collision-budget-alpha-round:
  hash-collision-budget 1 alpha-round
  unfolding alpha-round-def
  by (rule hash-collision-budget-alpha-challenge-step)

lemma hash-collision-budget-verify-monad:
  hash-collision-budget verifier-hash-query-budget verify-monad
proof –
  let  $?Q = \text{rounds} * \text{verifier-query-round-hash-budget}$ 
  let  $?alpha\text{-steps} =$ 
    replicate (length spec)
      (alpha-round ::
        ( $'f, ('f, 'a)$  protocol-channel-scheme) state-monad)
  have alpha-step-range:

```

```

 $\wedge m. m \in \text{set } ?\text{alpha-steps} \implies \text{hash-range-budget } 1 \ m$ 
proof –
  fix  $m$ 
  assume  $m \in \text{set } ?\text{alpha-steps}$ 
  then have  $m\text{-eq}$ :
     $m =$ 
      ( $\text{alpha-round} ::$ 
        ( $f, (f, 'a) \text{ protocol-channel-scheme} \rangle \text{ state-monad}$ )
      )
    by simp
  show  $\text{hash-range-budget } 1 \ m$ 
    unfolding  $m\text{-eq}$  by (rule hash-range-budget-alpha-round)
qed
have  $\text{alpha-step-coll}$ :
   $\wedge m. m \in \text{set } ?\text{alpha-steps} \implies \text{hash-collision-budget } 1 \ m$ 
proof –
  fix  $m$ 
  assume  $m \in \text{set } ?\text{alpha-steps}$ 
  then have  $m\text{-eq}$ :
     $m =$ 
      ( $\text{alpha-round} ::$ 
        ( $f, (f, 'a) \text{ protocol-channel-scheme} \rangle \text{ state-monad}$ )
      )
    by simp
  show  $\text{hash-collision-budget } 1 \ m$ 
    unfolding  $m\text{-eq}$  by (rule hash-collision-budget-alpha-round)
qed
have  $\text{alpha-map-range}$ :
   $\text{hash-range-budget } (\text{length spec}) \ (\text{mmap } ?\text{alpha-steps})$ 
proof –
  have  $\text{hash-range-budget } (\text{length } ?\text{alpha-steps} * 1) \ (\text{mmap } ?\text{alpha-steps})$ 
    by (rule hash-range-budget-mmap) (rule alpha-step-range)
  then show  $?thesis$  by simp
qed
have  $\text{alpha-map-coll}$ :
   $\text{hash-collision-budget } (\text{length spec}) \ (\text{mmap } ?\text{alpha-steps})$ 
proof –
  have  $\text{hash-collision-budget } (\text{length } ?\text{alpha-steps} * 1) \ (\text{mmap } ?\text{alpha-steps})$ 
proof (rule hash-collision-budget-mmap)
    fix  $m$ 
    assume  $m\text{-in}: m \in \text{set } ?\text{alpha-steps}$ 
    show  $\text{hash-range-budget } 1 \ m$ 
      by (rule alpha-step-range[OF m-in])
    show  $\text{hash-collision-budget } 1 \ m$ 
      by (rule alpha-step-coll[OF m-in])
    qed
  then show  $?thesis$  by simp
qed
have  $\text{after-final-range}$ :
   $\text{hash-range-budget } (0 + ?Q)$ 
  (read  $\gg=$ 

```

```

    ( $\lambda$ final. ntimes
      (verifier-query-round-program fr f-fl f-final as fl final)
      rounds))
  if len-f: length f-fl  $\leq$  ceil-log clength
    and len-fl: length fl  $\leq$  ceil-log (maxDegree + 1)
  for fr f-fl f-final as fl
  by (rule hash-range-budget-bind
      [OF hash-range-budget-read
        hash-range-budget-ntimes
        [OF hash-range-budget-verifier-query-round-program
          [OF len-f len-fl]]])
  have after-final-coll:
    hash-collision-budget (0 + ?Q)
    (read  $\gg$ 
      ( $\lambda$ final. ntimes
        (verifier-query-round-program fr f-fl f-final as fl final)
        rounds))
  if len-f: length f-fl  $\leq$  ceil-log clength
    and len-fl: length fl  $\leq$  ceil-log (maxDegree + 1)
  for fr f-fl f-final as fl
  proof -
    have query-range:
       $\bigwedge$ final. hash-range-budget verifier-query-round-hash-budget
        (verifier-query-round-program fr f-fl f-final as fl final)
      by (rule hash-range-budget-verifier-query-round-program
          [OF len-f len-fl])
    have query-coll:
       $\bigwedge$ final. hash-collision-budget verifier-query-round-hash-budget
        (verifier-query-round-program fr f-fl f-final as fl final)
      by (rule hash-collision-budget-verifier-query-round-program
          [OF len-f len-fl])
    have rounds-range:
       $\bigwedge$ final. hash-range-budget ?Q
        (ntimes (verifier-query-round-program fr f-fl f-final as fl final)
          rounds)
      by (rule hash-range-budget-ntimes) (rule query-range)
    have rounds-coll:
       $\bigwedge$ final. hash-collision-budget ?Q
        (ntimes (verifier-query-round-program fr f-fl f-final as fl final)
          rounds)
      by (rule hash-collision-budget-ntimes)
        (rule query-range, rule query-coll)
    show ?thesis
      by (rule hash-collision-budget-bind
          [OF hash-range-budget-read hash-collision-budget-read
            rounds-range rounds-coll])
  qed
  have after-comp-fri-range:
    hash-range-budget (ceil-log (maxDegree + 1) + (0 + ?Q))

```

```

((ntimes receive-composition-fri-commits (ceil-log (to-nat dg + 1)) ::
  (((f × 'f) list),
   (f, 'a) protocol-channel-scheme) state-monad) >>=
  (λfl. read >>=
   (λfinal. ntimes
    (verifier-query-round-program fr f-fl f-final as fl final)
    rounds))))
if len-f: length f-fl ≤ ceil-log clength
  and dg-bound: to-nat dg ≤ maxDegree
for fr f-fl f-final as dg
proof (rule hash-range-budget-bind-on-outcomes)
show hash-range-budget (ceil-log (maxDegree + 1))
  (ntimes receive-composition-fri-commits (ceil-log (to-nat dg + 1)) ::
   (((f × 'f) list),
    (f, 'a) protocol-channel-scheme) state-monad)
  by (rule hash-range-budget-ntimes-receive-composition-fri-commits-bound)
      (rule ceil-log-to-nat-degree-bound[OF dg-bound])
fix s fl t
assume out:
  Some (fl, t) ∈
    set-dist (execute
      (ntimes receive-composition-fri-commits (ceil-log (to-nat dg + 1)) ::
        (((f × 'f) list),
         (f, 'a) protocol-channel-scheme) state-monad) s)
have len-fl: length fl ≤ ceil-log (maxDegree + 1)
  using ntimes-receive-composition-fri-commits-outcome[OF out]
      ceil-log-to-nat-degree-bound[OF dg-bound]
  by simp
show hash-range-budget (0 + ?Q)
  (read >>=
   (λfinal. ntimes
    (verifier-query-round-program fr f-fl f-final as fl final)
    rounds))
  by (rule after-final-range[OF len-f len-fl])
qed
have after-comp-fri-coll:
  hash-collision-budget (ceil-log (maxDegree + 1) + (0 + ?Q))
  ((ntimes receive-composition-fri-commits (ceil-log (to-nat dg + 1)) ::
   (((f × 'f) list),
    (f, 'a) protocol-channel-scheme) state-monad) >>=
   (λfl. read >>=
    (λfinal. ntimes
     (verifier-query-round-program fr f-fl f-final as fl final)
     rounds))))
if len-f: length f-fl ≤ ceil-log clength
  and dg-bound: to-nat dg ≤ maxDegree
for fr f-fl f-final as dg
proof (rule hash-collision-budget-bind-on-outcomes)
show hash-range-budget (ceil-log (maxDegree + 1))

```



```

  (ntimes receive-composition-fri-commits (ceil-log (to-nat dg + 1)) ::
    (((f × 'f) list),
     (f, 'a) protocol-channel-scheme) state-monad)
  by (rule hash-range-budget-ntimes-receive-composition-fri-commits-bound)
    (rule ceil-log-to-nat-degree-bound[OF dg-bound])
show hash-collision-budget (ceil-log (maxDegree + 1))
  (ntimes receive-composition-fri-commits (ceil-log (to-nat dg + 1)) ::
    (((f × 'f) list),
     (f, 'a) protocol-channel-scheme) state-monad)
  by (rule hash-collision-budget-ntimes-receive-composition-fri-commits-bound)
    (rule ceil-log-to-nat-degree-bound[OF dg-bound])
fix s fl t
assume out:
  Some (fl, t) ∈
    set-dist (execute
      (ntimes receive-composition-fri-commits (ceil-log (to-nat dg + 1)) ::
        (((f × 'f) list),
         (f, 'a) protocol-channel-scheme) state-monad) s)
have len-fl: length fl ≤ ceil-log (maxDegree + 1)
  using ntimes-receive-composition-fri-commits-outcome[OF out]
    ceil-log-to-nat-degree-bound[OF dg-bound]
  by simp
show hash-range-budget (0 + ?Q)
  (read >=
    (λfinal. ntimes
      (verifier-query-round-program fr f-fl f-final as fl final)
      rounds))
  by (rule after-final-range[OF len-f len-fl])
show hash-collision-budget (0 + ?Q)
  (read >=
    (λfinal. ntimes
      (verifier-query-round-program fr f-fl f-final as fl final)
      rounds))
  by (rule after-final-coll[OF len-f len-fl])
qed
have after-degree-assert-range:
  hash-range-budget (0 + (ceil-log (maxDegree + 1) + (0 + ?Q)))
  (assert (to-nat dg ≤ maxDegree) >=
    (λ-. (ntimes receive-composition-fri-commits (ceil-log (to-nat dg + 1)) ::
      (((f × 'f) list),
       (f, 'a) protocol-channel-scheme) state-monad) >=
      (λfl. read >=
        (λfinal. ntimes
          (verifier-query-round-program fr f-fl f-final as fl final)
          rounds))))))
  if len-f: length f-fl ≤ ceil-log clength
  for fr f-fl f-final as dg
proof (rule hash-range-budget-bind-on-outcomes)
  show hash-range-budget 0

```

```

    (assert (to-nat dg ≤ maxDegree) ::
      (unit, ('f, 'a) protocol-channel-scheme) state-monad)
  by (rule hash-range-budget-assert)
fix s u t
assume out:
  Some (u, t) ∈
    set-dist (execute
      (assert (to-nat dg ≤ maxDegree) ::
        (unit, ('f, 'a) protocol-channel-scheme) state-monad) s)
have dg-bound: to-nat dg ≤ maxDegree
  using out unfolding assert-def
  by (cases to-nat dg ≤ maxDegree) (auto simp: throw-no-outcome)
show hash-range-budget (ceil-log (maxDegree + 1) + (0 + ?Q))
  ((ntimes receive-composition-fri-commits (ceil-log (to-nat dg + 1)) ::
    (((('f × 'f) list),
      ('f, 'a) protocol-channel-scheme) state-monad) ≫=
    (λfl. read ≫=
      (λfinal. ntimes
        (verifier-query-round-program fr f-fl f-final as fl final)
        rounds))))))
  by (rule after-comp-fri-range[OF len-f dg-bound])
qed
have after-degree-assert-coll:
  hash-collision-budget (0 + (ceil-log (maxDegree + 1) + (0 + ?Q)))
  (assert (to-nat dg ≤ maxDegree) ≫=
    (λ-. (ntimes receive-composition-fri-commits (ceil-log (to-nat dg + 1)) ::
      (((('f × 'f) list),
        ('f, 'a) protocol-channel-scheme) state-monad) ≫=
        (λfl. read ≫=
          (λfinal. ntimes
            (verifier-query-round-program fr f-fl f-final as fl final)
            rounds))))))
  if len-f: length f-fl ≤ ceil-log clength
  for fr f-fl f-final as dg
proof (rule hash-collision-budget-bind-on-outcomes)
show hash-range-budget 0
  (assert (to-nat dg ≤ maxDegree) ::
    (unit, ('f, 'a) protocol-channel-scheme) state-monad)
  by (rule hash-range-budget-assert)
show hash-collision-budget 0
  (assert (to-nat dg ≤ maxDegree) ::
    (unit, ('f, 'a) protocol-channel-scheme) state-monad)
  by (rule hash-collision-budget-assert)
fix s u t
assume out:
  Some (u, t) ∈
    set-dist (execute
      (assert (to-nat dg ≤ maxDegree) ::
        (unit, ('f, 'a) protocol-channel-scheme) state-monad) s)

```

```

have dg-bound: to-nat dg ≤ maxDegree
  using out unfolding assert-def
  by (cases to-nat dg ≤ maxDegree) (auto simp: throw-no-outcome)
show hash-range-budget (ceil-log (maxDegree + 1) + (0 + ?Q))
  ((ntimes receive-composition-fri-commits (ceil-log (to-nat dg + 1)) ::
    (((f × 'f) list),
     (f, 'a) protocol-channel-scheme) state-monad) ≫=
    (λfl. read ≫=
      (λfinal. ntimes
        (verifier-query-round-program fr f-fl f-final as fl final)
        rounds))))
  by (rule after-comp-fri-range[OF len-f dg-bound])
show hash-collision-budget (ceil-log (maxDegree + 1) + (0 + ?Q))
  ((ntimes receive-composition-fri-commits (ceil-log (to-nat dg + 1)) ::
    (((f × 'f) list),
     (f, 'a) protocol-channel-scheme) state-monad) ≫=
    (λfl. read ≫=
      (λfinal. ntimes
        (verifier-query-round-program fr f-fl f-final as fl final)
        rounds))))
  by (rule after-comp-fri-coll[OF len-f dg-bound])
qed
have after-read-dg-range:
  hash-range-budget
  (0 + (0 + (ceil-log (maxDegree + 1) + (0 + ?Q))))
  (read ≫=
    (λdg. assert (to-nat dg ≤ maxDegree) ≫=
      (λ-. (ntimes receive-composition-fri-commits (ceil-log (to-nat dg + 1)) ::
        (((f × 'f) list),
         (f, 'a) protocol-channel-scheme) state-monad) ≫=
        (λfl. read ≫=
          (λfinal. ntimes
            (verifier-query-round-program fr f-fl f-final as fl final)
            rounds))))))
  if len-f: length f-fl ≤ ceil-log clength
  for fr f-fl f-final as
  by (rule hash-range-budget-bind
    [OF hash-range-budget-read after-degree-assert-range[OF len-f]])
have after-read-dg-coll:
  hash-collision-budget
  (0 + (0 + (ceil-log (maxDegree + 1) + (0 + ?Q))))
  (read ≫=
    (λdg. assert (to-nat dg ≤ maxDegree) ≫=
      (λ-. (ntimes receive-composition-fri-commits (ceil-log (to-nat dg + 1)) ::
        (((f × 'f) list),
         (f, 'a) protocol-channel-scheme) state-monad) ≫=
        (λfl. read ≫=
          (λfinal. ntimes
            (verifier-query-round-program fr f-fl f-final as fl final)
            rounds))))))

```

```

        rounds))))))
if len-f: length f-fl ≤ ceil-log clength
for fr f-fl f-final as
by (rule hash-collision-budget-bind
    [OF hash-range-budget-read hash-collision-budget-read
     after-degree-assert-range[OF len-f]
     after-degree-assert-coll[OF len-f]])
have after-alpha-range:
    hash-range-budget
    (length spec +
     (0 + (0 + (ceil-log (maxDegree + 1) + (0 + ?Q))))))
    (mmap ?alpha-steps >>=
     (λas. read >>=
      (λdg. assert (to-nat dg ≤ maxDegree) >>=
       (λ-. (ntimes receive-composition-fri-commits (ceil-log (to-nat dg + 1)) ::
        (((f × f) list),
         (f, a) protocol-channel-scheme) state-monad) >>=
        (λfl. read >>=
         (λfinal. ntimes
          (verifier-query-round-program fr f-fl f-final as fl final)
          rounds)))))))
if len-f: length f-fl ≤ ceil-log clength
for fr f-fl f-final
by (rule hash-range-budget-bind
    [OF alpha-map-range after-read-dg-range[OF len-f]])
have after-alpha-coll:
    hash-collision-budget
    (length spec +
     (0 + (0 + (ceil-log (maxDegree + 1) + (0 + ?Q))))))
    (mmap ?alpha-steps >>=
     (λas. read >>=
      (λdg. assert (to-nat dg ≤ maxDegree) >>=
       (λ-. (ntimes receive-composition-fri-commits (ceil-log (to-nat dg + 1)) ::
        (((f × f) list),
         (f, a) protocol-channel-scheme) state-monad) >>=
        (λfl. read >>=
         (λfinal. ntimes
          (verifier-query-round-program fr f-fl f-final as fl final)
          rounds)))))))
if len-f: length f-fl ≤ ceil-log clength
for fr f-fl f-final
by (rule hash-collision-budget-bind
    [OF alpha-map-range alpha-map-coll
     after-read-dg-range[OF len-f] after-read-dg-coll[OF len-f]])
have after-read-f-final-range:
    hash-range-budget
    (0 + (length spec +
     (0 + (0 + (ceil-log (maxDegree + 1) + (0 + ?Q))))))
    (read >>=

```



```

      (λfl. read >>=
        (λfinal. ntimes
          (verifier-query-round-program fr f-fl f-final as fl final)
          rounds)))))))))
for fr
proof (rule hash-range-budget-bind-on-outcomes)
show hash-range-budget (ceil-log clength)
  (ntimes receive-trace-fri-commits (ceil-log clength) ::
    (((f × 'f) list),
      (f, 'a) protocol-channel-scheme) state-monad)
  by (rule hash-range-budget-ntimes-receive-trace-fri-commits-bound) simp
fix s f-fl t
assume out:
  Some (f-fl, t) ∈
    set-dist (execute
      (ntimes receive-trace-fri-commits (ceil-log clength) ::
        (((f × 'f) list),
          (f, 'a) protocol-channel-scheme) state-monad) s)
have len-f: length f-fl ≤ ceil-log clength
  using ntimes-receive-trace-fri-commits-outcome[OF out] by simp
show hash-range-budget
  (0 + (length spec +
    (0 + (0 + (ceil-log (maxDegree + 1) + (0 + ?Q))))))
  (read >>=
    (λf-final. mmap ?alpha-steps >>=
      (λas. read >>=
        (λdg. assert (to-nat dg ≤ maxDegree) >>=
          (λ-. (ntimes receive-composition-fri-commits (ceil-log (to-nat dg + 1)) ::
            (((f × 'f) list),
              (f, 'a) protocol-channel-scheme) state-monad) >>=
            (λfl. read >>=
              (λfinal. ntimes
                (verifier-query-round-program fr f-fl f-final as fl final)
                rounds))))))))))
  by (rule after-read-f-final-range[OF len-f])
qed
have after-trace-fri-coll:
  hash-collision-budget
  (ceil-log clength +
    (0 + (length spec +
      (0 + (0 + (ceil-log (maxDegree + 1) + (0 + ?Q))))))
  ((ntimes receive-trace-fri-commits (ceil-log clength) ::
    (((f × 'f) list),
      (f, 'a) protocol-channel-scheme) state-monad) >>=
    (λf-fl. read >>=
      (λf-final. mmap ?alpha-steps >>=
        (λas. read >>=
          (λdg. assert (to-nat dg ≤ maxDegree) >>=
            (λ-. (ntimes receive-composition-fri-commits (ceil-log (to-nat dg + 1)) ::

```

```

      (((('f × 'f) list),
        ('f, 'a) protocol-channel-scheme) state-monad) >>=
      (λfl. read >>=
        (λfinal. ntimes
          (verifier-query-round-program fr f-fl f-final as fl final)
          rounds)))))))))
for fr
proof (rule hash-collision-budget-bind-on-outcomes)
show hash-range-budget (ceil-log clength)
  (ntimes receive-trace-fri-commits (ceil-log clength) ::
    (((('f × 'f) list),
      ('f, 'a) protocol-channel-scheme) state-monad)
    by (rule hash-range-budget-ntimes-receive-trace-fri-commits-bound) simp
show hash-collision-budget (ceil-log clength)
  (ntimes receive-trace-fri-commits (ceil-log clength) ::
    (((('f × 'f) list),
      ('f, 'a) protocol-channel-scheme) state-monad)
    by (rule hash-collision-budget-ntimes-receive-trace-fri-commits-bound) simp
fix s f-fl t
assume out:
  Some (f-fl, t) ∈
    set-dist (execute
      (ntimes receive-trace-fri-commits (ceil-log clength) ::
        (((('f × 'f) list),
          ('f, 'a) protocol-channel-scheme) state-monad) s)
      len-f: length f-fl ≤ ceil-log clength
      using ntimes-receive-trace-fri-commits-outcome[OF out] by simp
show hash-range-budget
    (0 + (length spec +
      (0 + (0 + (ceil-log (maxDegree + 1) + (0 + ?Q))))))
    (read >>=
      (λf-final. mmap ?alpha-steps >>=
        (λas. read >>=
          (λdg. assert (to-nat dg ≤ maxDegree) >>=
            (λ-. (ntimes receive-composition-fri-commits (ceil-log (to-nat dg + 1)) ::
              (((('f × 'f) list),
                ('f, 'a) protocol-channel-scheme) state-monad) >>=
              (λfl. read >>=
                (λfinal. ntimes
                  (verifier-query-round-program fr f-fl f-final as fl final)
                  rounds))))))))))
        by (rule after-read-f-final-range[OF len-f])
show hash-collision-budget
    (0 + (length spec +
      (0 + (0 + (ceil-log (maxDegree + 1) + (0 + ?Q))))))
    (read >>=
      (λf-final. mmap ?alpha-steps >>=
        (λas. read >>=
          (λdg. assert (to-nat dg ≤ maxDegree) >>=

```

```

    (λ-. (ntimes receive-composition-fri-commits (ceil-log (to-nat dg + 1)) ::
      (((('f × 'f) list),
        ('f, 'a) protocol-channel-scheme) state-monad) >=>=
      (λfl. read >=>=
        (λfinal. ntimes
          (verifier-query-round-program fr f-fl f-final as fl final)
          rounds))))))
  by (rule after-read-f-final-coll[OF len-f])
qed
have top-range:
  hash-range-budget
    (0 + (ceil-log clength +
      (0 + (length spec +
        (0 + (0 + (ceil-log (maxDegree + 1) + (0 + ?Q))))))))
  (read >=>=
    (λfr. (ntimes receive-trace-fri-commits (ceil-log clength) ::
      (((('f × 'f) list),
        ('f, 'a) protocol-channel-scheme) state-monad) >=>=
      (λf-fl. read >=>=
        (λf-final. mmap ?alpha-steps >=>=
          (λas. read >=>=
            (λdg. assert (to-nat dg ≤ maxDegree) >=>=
              (λ-. (ntimes receive-composition-fri-commits (ceil-log (to-nat dg + 1)) ::
                (((('f × 'f) list),
                  ('f, 'a) protocol-channel-scheme) state-monad) >=>=
                  (λfl. read >=>=
                    (λfinal. ntimes
                      (verifier-query-round-program fr f-fl f-final as fl final)
                      rounds))))))))))
            by (rule hash-range-budget-bind
              [OF hash-range-budget-read after-trace-fri-range])
  have top-coll:
    hash-collision-budget
      (0 + (ceil-log clength +
        (0 + (length spec +
          (0 + (0 + (ceil-log (maxDegree + 1) + (0 + ?Q))))))))
      (read >=>=
        (λfr. (ntimes receive-trace-fri-commits (ceil-log clength) ::
          (((('f × 'f) list),
            ('f, 'a) protocol-channel-scheme) state-monad) >=>=
          (λf-fl. read >=>=
            (λf-final. mmap ?alpha-steps >=>=
              (λas. read >=>=
                (λdg. assert (to-nat dg ≤ maxDegree) >=>=
                  (λ-. (ntimes receive-composition-fri-commits (ceil-log (to-nat dg + 1)) ::
                    (((('f × 'f) list),
                      ('f, 'a) protocol-channel-scheme) state-monad) >=>=
                      (λfl. read >=>=
                        (λfinal. ntimes
                          (verifier-query-round-program fr f-fl f-final as fl final)
                          rounds))))))))))
                  by (rule hash-range-budget-bind
                    [OF hash-range-budget-read after-trace-fri-range])

```



```

      (verifier-query-round-program fr f-fl f-final as fl final)
      rounds)))))))))
  by (rule hash-collision-budget-bind
      [OF hash-range-budget-read hash-collision-budget-read
         after-trace-fri-range after-trace-fri-coll])
  show ?thesis
  using top-coll
  unfolding verify-monad-def verifier-query-round-program-def alpha-round-def
    verifier-hash-query-budget-def verifier-header-hash-budget-def
  by (simp add: algebra-simps)
qed

end

end

```

```

theory Soundness-Oracle-Merkle
imports Soundness-Oracle-Verifier
begin

```

Merkle-binding collision covers and concrete binding interfaces.

```

context soundness
begin

```

```

definition hash-map-new-output-collision-bad
:: ('f, 'a) protocol-channel-scheme  $\Rightarrow$ 
  (unit list  $\times$  ('f, 'a) protocol-channel-scheme) option  $\Rightarrow$  bool
where
  hash-map-new-output-collision-bad s out  $\longleftrightarrow$ 
    accepted out  $\wedge$ 
    ( $\exists$  result final-state.
      out = Some (result, final-state)  $\wedge$ 
      hash-map-new-output-collision s final-state)

```

```

definition hash-map-new-output-collision-bound
:: ('f, 'a) protocol-channel-scheme  $\Rightarrow$  bool
where
  hash-map-new-output-collision-bound s  $\longleftrightarrow$ 
    wp-event verify-monad (hash-map-new-output-collision-bad s) s  $\leq$ 
    merkle-binding-error

```

```

lemma hash-map-new-output-collision-bound-if-error-ge-one:
assumes 1  $\leq$  merkle-binding-error
shows hash-map-new-output-collision-bound s
unfolding hash-map-new-output-collision-bound-def
using order-trans[OF wp-event-le-1 assms] .

```

```

lemma hash-map-new-output-collision-bad-imp-hash-new-collision-event:

```

```

assumes hash-map-new-output-collision-bad s out
shows hash-new-collision-event s out
using asms
unfolding hash-map-new-output-collision-bad-def hash-new-collision-event-def
by auto

lemma hash-map-new-output-collision-event-bound-from-verify-hash-budget:
  assumes clean:  $\neg$  hash-map-output-collision s
  shows
    wp-event verify-monad (hash-map-new-output-collision-bad s) s  $\leq$ 
      hash-collision-budget-value (card (hash-map-output-values s))
      verifier-hash-query-budget
proof –
  have wp-event verify-monad (hash-map-new-output-collision-bad s) s  $\leq$ 
    wp-event verify-monad (hash-new-collision-event s) s
  by (rule wp-event-mono)
    (rule hash-map-new-output-collision-bad-imp-hash-new-collision-event)
  also have ...  $\leq$ 
    hash-collision-budget-value (card (hash-map-output-values s))
    verifier-hash-query-budget
  using hash-collision-budget-verify-monad clean
  unfolding hash-collision-budget-def by blast
  finally show ?thesis .
qed

lemma hash-map-new-output-collision-bound-from-verify-hash-budget:
  assumes clean:  $\neg$  hash-map-output-collision s
  and error:
    hash-collision-budget-value (card (hash-map-output-values s))
    verifier-hash-query-budget  $\leq$  merkle-binding-error
  shows hash-map-new-output-collision-bound s
proof –
  have wp-event verify-monad (hash-map-new-output-collision-bad s) s  $\leq$ 
    hash-collision-budget-value (card (hash-map-output-values s))
    verifier-hash-query-budget
  by (rule hash-map-new-output-collision-event-bound-from-verify-hash-budget
    [OF clean])
  also have ...  $\leq$  merkle-binding-error
  by (rule error)
  finally show ?thesis
    unfolding hash-map-new-output-collision-bound-def .
qed

lemma hash-map-new-output-collision-bound-from-verify-hash-budget-empty:
  assumes empty-hashmap: HashMap s = fmempty
  and error:
    hash-collision-budget-value (card (hash-map-output-values s))
    verifier-hash-query-budget  $\leq$  merkle-binding-error
  shows hash-map-new-output-collision-bound s

```

```

proof –
  have clean:  $\neg$  hash-map-output-collision s
    using empty-hashmap unfolding hash-map-output-collision-def by simp
  show ?thesis
    by (rule hash-map-new-output-collision-bound-from-verify-hash-budget
      [OF clean error])
qed

lemma verifier-initial-hash-map-new-output-collision-bound:
  assumes error:
    hash-collision-budget-value 0 verifier-hash-query-budget  $\leq$ 
      merkle-binding-error
  shows
    hash-map-new-output-collision-bound (verifier-initial-state tr)
  by (rule hash-map-new-output-collision-bound-from-verify-hash-budget-empty)
    (use error in simp-all)

lemma verifier-initial-hash-map-new-output-collision-bad-bound-concrete:
  wp-event verify-monad
    (hash-map-new-output-collision-bad (verifier-initial-state tr))
    (verifier-initial-state tr)  $\leq$  concrete-merkle-binding-error
proof –
  have clean:
     $\neg$  hash-map-output-collision (verifier-initial-state tr)
    unfolding verifier-initial-state-def hash-map-output-collision-def
    by simp
  have bound:
    wp-event verify-monad
      (hash-map-new-output-collision-bad (verifier-initial-state tr))
      (verifier-initial-state tr)  $\leq$ 
        hash-collision-budget-value
          (card (hash-map-output-values (verifier-initial-state tr)))
          verifier-hash-query-budget
    by (rule hash-map-new-output-collision-event-bound-from-verify-hash-budget
      [OF clean])
  show ?thesis
    using bound unfolding concrete-merkle-binding-error-def by simp
qed

lemma hash-map-output-collision-bad-imp-new-collision-bad-if-initial-clean:
  assumes clean:  $\neg$  hash-map-output-collision s
    and bad: hash-map-output-collision-bad s out
  shows hash-map-new-output-collision-bad s out
  using assms
  unfolding hash-map-output-collision-bad-def
    hash-map-new-output-collision-bad-def
    hash-map-new-output-collision-def
  by auto

```

lemma *hash-map-output-collision-bound-from-new-collision-bound*:
assumes *clean*: $\neg$ *hash-map-output-collision s*
and *new-bound*: *hash-map-new-output-collision-bound s*
shows *hash-map-output-collision-bound s*
proof –
have *wp-event verify-monad (hash-map-output-collision-bad s) s* $\leq$
wp-event verify-monad (hash-map-new-output-collision-bad s) s
by (*rule wp-event-mono*)
(*rule hash-map-output-collision-bad-imp-new-collision-bad-if-initial-clean*
[*OF clean*])
also have ... $\leq$ *merkle-binding-error*
using *new-bound* **unfolding** *hash-map-new-output-collision-bound-def* .
finally show ?thesis
unfolding *hash-map-output-collision-bound-def* .
qed

definition *fri-merkle-binding-bad-covered-by-hash-collision*
 $:: (f, 'a)$ *protocol-channel-scheme* $\Rightarrow$ *bool*
where
fri-merkle-binding-bad-covered-by-hash-collision s $\longleftrightarrow$
 $(\forall out \in \text{set-dist } (\text{execute verify-monad } s).$
fri-merkle-binding-bad s out $\longrightarrow$
hash-map-output-collision-bad s out)

lemma *fri-merkle-binding-bound-from-hash-collision-cover*:
assumes *cover*: *fri-merkle-binding-bad-covered-by-hash-collision s*
and *collision-bound*: *hash-map-output-collision-bound s*
shows *fri-merkle-binding-bound s*
proof –
have *wp-event verify-monad (fri-merkle-binding-bad s) s* $\leq$
wp-event verify-monad (hash-map-output-collision-bad s) s
by (*rule wp-event-mono-on-support*)
(*use cover in*
 $\langle \text{auto simp: fri-merkle-binding-bad-covered-by-hash-collision-def} \rangle$)
also have ... $\leq$ *merkle-binding-error*
using *collision-bound* **unfolding** *hash-map-output-collision-bound-def* .
finally show ?thesis
unfolding *fri-merkle-binding-bound-def* .
qed

lemma *fri-merkle-binding-bound-from-new-hash-collision-cover*:
assumes *clean*: $\neg$ *hash-map-output-collision s*
and *cover*: *fri-merkle-binding-bad-covered-by-hash-collision s*
and *new-bound*: *hash-map-new-output-collision-bound s*
shows *fri-merkle-binding-bound s*
proof –
have *collision-bound*: *hash-map-output-collision-bound s*
by (*rule hash-map-output-collision-bound-from-new-collision-bound*
[*OF clean new-bound*])

```

show ?thesis
  by (rule fri-merkle-binding-bound-from-hash-collision-cover
    [OF cover collision-bound])
qed

lemma hash-map-output-collision-empty-hashmap[simp]:
  assumes HashMap s = fmempty
  shows  $\neg$  hash-map-output-collision s
  using assms unfolding hash-map-output-collision-def by simp

lemma verifier-initial-no-hash-map-output-collision[simp]:
   $\neg$  hash-map-output-collision (verifier-initial-state tr)
  by simp

lemma verifier-initial-fri-merkle-binding-bound-concrete:
  assumes cover:
    fri-merkle-binding-bad-covered-by-hash-collision
      (verifier-initial-state tr)
  shows
    wp-event verify-monad
      (fri-merkle-binding-bad (verifier-initial-state tr))
      (verifier-initial-state tr)  $\leq$  concrete-merkle-binding-error
proof -
  let ?s = verifier-initial-state tr
  have wp-event verify-monad (fri-merkle-binding-bad ?s) ?s  $\leq$ 
    wp-event verify-monad (hash-map-output-collision-bad ?s) ?s
  by (rule wp-event-mono-on-support)
    (use cover in
       $\langle$ auto simp: fri-merkle-binding-bad-covered-by-hash-collision-def $\rangle$ )
  also have ...  $\leq$ 
    wp-event verify-monad (hash-map-new-output-collision-bad ?s) ?s
  by (rule wp-event-mono)
    (rule hash-map-output-collision-bad-imp-new-collision-bad-if-initial-clean,
      simp, assumption)
  also have ...  $\leq$  concrete-merkle-binding-error
  by (rule verifier-initial-hash-map-new-output-collision-bad-bound-concrete)
  finally show ?thesis .
qed

lemma fri-merkle-binding-bound-from-new-hash-collision-cover-empty:
  assumes HashMap s = fmempty
  and cover: fri-merkle-binding-bad-covered-by-hash-collision s
  and new-bound: hash-map-new-output-collision-bound s
  shows fri-merkle-binding-bound s
  by (rule fri-merkle-binding-bound-from-new-hash-collision-cover
    [OF - cover new-bound])
    (use assms(1) in simp)

lemma fri-bound-layer-tables-length:

```

assumes *fri-bound-layer-tables roots tables final-state*
shows *length tables = length roots*
using *assms*
unfolding *fri-bound-layer-tables-def fri-layer-tables-shape-def* **by** *simp*

lemma *fri-bound-layer-tables-root*:
assumes *fri-bound-layer-tables roots tables final-state*
and *j < length roots*
shows *merkle-root-binds-table (roots ! j) (tables ! j) final-state*
using *assms*
unfolding *fri-bound-layer-tables-def fri-roots-bind-tables-def* **by** *simp*

lemma *fri-bound-layer-tables-table-length*:
assumes *fri-bound-layer-tables roots tables final-state*
and *j < length roots*
shows
length (tables ! j) =
*fri-layer-lengths (length roots) (clength * scale) ! j*
using *assms*
unfolding *fri-bound-layer-tables-def fri-layer-tables-shape-def*
by *simp*

lemma *fri-bound-layer-tables-same-root-collision-imp-hash-collision-bad*:
assumes *out: out = Some (result, final-state)*
and *bound1: fri-bound-layer-tables roots tables final-state*
and *bound2: fri-bound-layer-tables roots tables' final-state*
and *j-bound: j < length roots*
and *nonempty: tables ! j ≠ []*
and *neq: tables ! j ≠ tables' ! j*
shows *hash-map-output-collision-bad s out*
proof –
have *bind1*:
merkle-root-binds-table (roots ! j) (tables ! j) final-state
by (rule *fri-bound-layer-tables-root*[OF *bound1 j-bound*])
have *bind2*:
merkle-root-binds-table (roots ! j) (tables' ! j) final-state
by (rule *fri-bound-layer-tables-root*[OF *bound2 j-bound*])
have *same-len*: *length (tables ! j) = length (tables' ! j)*
using *fri-bound-layer-tables-table-length*[OF *bound1 j-bound*]
fri-bound-layer-tables-table-length[OF *bound2 j-bound*]
by *simp*
show *?thesis*
by (rule *accepted-inconsistent-same-length-merkle-bindings-imp-hash-collision-bad*
[OF *out bind1 bind2 same-len nonempty neq*])
qed

lemma *fri-layer-opening-matches-bound-tableD*:
assumes *match*:
fri-layer-opening-matches-bound-table query-idx layer-idx tables

```

    layer-chunks
obtains len idx xp xp-path xn xn-path
where layer-idx < length tables
    and length layer-chunks = length tables
    and len =
        fri-layer-lengths (length tables) (clength * scale) ! layer-idx
    and idx =
        fri-layer-indices (length tables) query-idx (clength * scale) !
        layer-idx
    and fri-layer-opening-chunk len xp xp-path xn xn-path
        (layer-chunks ! layer-idx)
    and fri-opening-matches-table len idx (tables ! layer-idx) xp xn
using match
unfolding fri-layer-opening-matches-bound-table-def Let-def
by blast

```

lemma *fri-round-openings-match-tablesD*:

```

assumes fri-round-openings-match-tables tables query-idx layer-chunks
    and j < length tables
shows
    fri-layer-opening-matches-bound-table query-idx j tables layer-chunks
using assms unfolding fri-round-openings-match-tables-def by simp

```

lemma *fri-all-round-openings-match-tablesD*:

```

assumes
    fri-all-round-openings-match-tables tables query-idxs round-layers
    and i < length query-idxs
shows
    fri-round-openings-match-tables tables (query-idxs ! i)
        (round-layers ! i)
using assms unfolding fri-all-round-openings-match-tables-def by simp

```

lemma *trace-fri-tables-bound-opening-matches*:

```

assumes bound:
    trace-fri-tables-bound s out trace-roots trace-bs trace-final
    query-idxs trace-round-layers trace-tables trace-final-table
    and i-bound: i < length query-idxs
    and j-bound: j < length trace-tables
shows
    fri-layer-opening-matches-bound-table (query-idxs ! i) j
        trace-tables (trace-round-layers ! i)

```

proof –

```

have rounds:
    fri-all-round-openings-match-tables trace-tables query-idxs
        trace-round-layers
    using bound unfolding trace-fri-tables-bound-def by blast
from fri-all-round-openings-match-tablesD[OF rounds i-bound]
have fri-round-openings-match-tables trace-tables (query-idxs ! i)
    (trace-round-layers ! i) .

```

then show *?thesis*
using *j-bound* **by** (rule *fri-round-openings-match-tablesD*)
qed

lemma *composition-fri-tables-bound-opening-matches*:
assumes *bound*:
composition-fri-tables-bound s out dg composition-roots composition-bs
composition-final query-idxs composition-round-layers
composition-tables composition-final-table
and *i-bound*: $i < \text{length } \text{query-idxs}$
and *j-bound*: $j < \text{length } \text{composition-tables}$
shows
fri-layer-opening-matches-bound-table (query-idxs ! i) j
composition-tables (composition-round-layers ! i)
proof –
have *rounds*:
fri-all-round-openings-match-tables composition-tables query-idxs
composition-round-layers
using *bound* **unfolding** *composition-fri-tables-bound-def* **by** *blast*
from *fri-all-round-openings-match-tablesD*[*OF rounds i-bound*]
have *fri-round-openings-match-tables composition-tables (query-idxs ! i)*
(composition-round-layers ! i) .
then show *?thesis*
using *j-bound* **by** (rule *fri-round-openings-match-tablesD*)
qed

lemma *fri-merkle-binding-bad-imp-accepted*:
assumes *fri-merkle-binding-bad s out*
shows *accepted out*
using *assms* **unfolding** *fri-merkle-binding-bad-def* **by** *simp*

lemma *fri-merkle-binding-bad-mono-accepted*:
wp-event verify-monad (fri-merkle-binding-bad s) s ≤
wp-event verify-monad accepted s
by (rule *wp-event-mono*)
(simp add: fri-merkle-binding-bad-imp-accepted)

lemma *accepted-good-or-fri-merkle-binding-bad*:
assumes *accepted out*
shows *fri-merkle-binding-good s out ∨ fri-merkle-binding-bad s out*
using *assms* **unfolding** *fri-merkle-binding-bad-def* **by** *blast*

lemma *fri-merkle-binding-goodE*:
assumes *good: fri-merkle-binding-good s out*
and *acc: accepted out*
obtains *trace-table composition-table as query-idxs*
trace-roots trace-bs trace-final dg composition-roots composition-bs
composition-final trace-round-layers composition-round-layers
trace-tables composition-tables trace-final-table composition-final-table

where
accepted-with-bound-tables s out trace-table composition-table as
query-idxs
and *accepted-fri-opening-transcript s out trace-roots trace-bs*
trace-final dg composition-roots composition-bs composition-final
query-idxs trace-round-layers composition-round-layers
and *trace-fri-tables-bound s out trace-roots trace-bs trace-final*
query-idxs trace-round-layers trace-tables trace-final-table
and *composition-fri-tables-bound s out dg composition-roots*
composition-bs composition-final query-idxs composition-round-layers
composition-tables composition-final-table
and *fri-initial-table-agrees trace-table trace-tables*
and *fri-initial-table-agrees composition-table composition-tables*
using *good acc* **unfolding** *fri-merkle-binding-good-def* **by** *blast*

end

end

theory *Soundness-Oracle*
imports *Soundness-Oracle-Merkle*
begin

Compatibility umbrella for random-oracle soundness infrastructure.

end

theory *Controlled-RO*
imports
Soundness-Oracle
begin

context *soundness*
begin

Controlled random-oracle programs provide a capability-safe interface for attacker stages. The constructors expose ordinary probabilistic computation and oracle queries, but no operation that can directly modify the oracle map or protocol-local channel fields.

inductive *controlled-ro-program*
 $:: \text{nat} \Rightarrow$
 $(\text{'r}, \text{'f protocol-channel}) \text{state-monad} \Rightarrow \text{bool}$
where
Return[intro]:
controlled-ro-program 0 (return x)
| *Fail[intro]:*
controlled-ro-program 0 throw
| *Sample[intro]:*
controlled-ro-program 0 (lift ($\lambda\cdot$. d))

| *Query*[intro]:
 $(\forall y. \text{controlled-ro-program } q \ (k \ y)) \implies$
 $\text{controlled-ro-program } (\text{Suc } q)$
 $((\text{hash } x :: ('f, 'f \text{ protocol-channel}) \text{ state-monad})$
 $\ggg k)$

| *Bind*[intro]:
 $\text{controlled-ro-program } q \ m \implies$
 $(\forall x. \text{controlled-ro-program } r \ (k \ x)) \implies$
 $\text{controlled-ro-program } (q + r) \ (m \ggg k)$

| *Weaken*:
 $\text{controlled-ro-program } q \ m \implies q \leq r \implies$
 $\text{controlled-ro-program } r \ m$

lemma *hash-map-preserving-throw*:
hash-map-preserving
 $(\text{throw} :: ('r, 'f \text{ protocol-channel}) \text{ state-monad})$
unfolding *hash-map-preserving-def*
by (*simp add: throw-no-outcome*)

lemma *hash-map-preserving-return*:
hash-map-preserving
 $(\text{return } x :: ('r, 'f \text{ protocol-channel}) \text{ state-monad})$
unfolding *hash-map-preserving-def* **by** *simp*

lemma *hash-map-preserving-state-independent-sample*:
hash-map-preserving
 $(\text{lift } (\lambda-. d) ::$
 $('r, 'f \text{ protocol-channel}) \text{ state-monad})$
unfolding *hash-map-preserving-def lift.rep-eq lift-dist-def*
by (*auto simp: set-dist-dist-map*)

definition *protocol-fields-preserving*
 $:: ('r, 'f \text{ protocol-channel}) \text{ state-monad} \Rightarrow \text{bool}$
where
 $\text{protocol-fields-preserving } m \longleftrightarrow$
 $(\forall s \ x \ t.$
 $\text{Some } (x, t) \in \text{set-dist } (\text{execute } m \ s) \longrightarrow$
 $P\text{State } t = P\text{State } s \wedge$
 $P\text{Transcript } t = P\text{Transcript } s \wedge$
 $P\text{TraceFriCounter } t = P\text{TraceFriCounter } s \wedge$
 $P\text{CompositionFriCounter } t = P\text{CompositionFriCounter } s \wedge$
 $P\text{AlphaCounter } t = P\text{AlphaCounter } s \wedge$
 $P\text{QueryCounter } t = P\text{QueryCounter } s)$

lemma *controlled-ro-program-preserves-protocol-fields*:
assumes *controlled-ro-program* $q \ m$
shows *protocol-fields-preserving* m
using *assms*
proof (*induction rule: controlled-ro-program.induct*)

```

case (Return  $x$ )
then show ?case
  unfolding protocol-fields-preserving-def by simp
next
case Fail
then show ?case
  unfolding protocol-fields-preserving-def
  by (simp add: throw-no-outcome)
next
case (Sample  $d$ )
then show ?case
  unfolding protocol-fields-preserving-def lift.rep-eq lift-dist-def
  by (auto simp: set-dist-dist-map)
next
case (Query  $q\ k\ x$ )
show ?case
  unfolding protocol-fields-preserving-def
proof (intro allI impI)
  fix  $s\ result\ t$ 
  assume outcome:
     $Some\ (result,\ t) \in$ 
      set-dist
      (execute
        ( $hash\ x :: ('f,\ 'f\ protocol-channel)\ state-monad) \gg k) s$ )
  then obtain  $h\ u$  where
    hash-out:
       $Some\ (h,\ u) \in$ 
        set-dist
        (execute ( $hash\ x :: ('f,\ 'f\ protocol-channel)\ state-monad$ )  $s$ )
  and rest:  $Some\ (result,\ t) \in set-dist\ (execute\ (k\ h)\ u)$ 
  by (auto elim!: set-dist-bindE)
have fields-u:
   $PState\ u = PState\ s \wedge$ 
   $PTranscript\ u = PTranscript\ s \wedge$ 
   $PTraceFriCounter\ u = PTraceFriCounter\ s \wedge$ 
   $PCompositionFriCounter\ u = PCompositionFriCounter\ s \wedge$ 
   $PAlphaCounter\ u = PAlphaCounter\ s \wedge$ 
   $PQueryCounter\ u = PQueryCounter\ s$ 
  using hash-channel-preserves[OF hash-out] by simp
have preserving-k: protocol-fields-preserving ( $k\ h$ )
  using Query.IH by blast
have fields-t:
   $PState\ t = PState\ u \wedge$ 
   $PTranscript\ t = PTranscript\ u \wedge$ 
   $PTraceFriCounter\ t = PTraceFriCounter\ u \wedge$ 
   $PCompositionFriCounter\ t = PCompositionFriCounter\ u \wedge$ 
   $PAlphaCounter\ t = PAlphaCounter\ u \wedge$ 
   $PQueryCounter\ t = PQueryCounter\ u$ 
  using preserving-k rest

```

```

  unfolding protocol-fields-preserving-def by blast
show
  PState t = PState s ∧
  PTranscript t = PTranscript s ∧
  PTraceFriCounter t = PTraceFriCounter s ∧
  PCompositionFriCounter t = PCompositionFriCounter s ∧
  PAlphaCounter t = PAlphaCounter s ∧
  PQueryCounter t = PQueryCounter s
  using fields-u fields-t by simp
qed
next
case (Bind q m r k)
show ?case
  unfolding protocol-fields-preserving-def
proof (intro allI impI)
  fix s result t
  assume outcome:
    Some (result, t) ∈ set-dist (execute (m ≫ k) s)
  then obtain x u where
    first: Some (x, u) ∈ set-dist (execute m s)
    and rest: Some (result, t) ∈ set-dist (execute (k x) u)
    by (auto elim!: set-dist-bindE)
  have fields-u:
    PState u = PState s ∧
    PTranscript u = PTranscript s ∧
    PTraceFriCounter u = PTraceFriCounter s ∧
    PCompositionFriCounter u = PCompositionFriCounter s ∧
    PAlphaCounter u = PAlphaCounter s ∧
    PQueryCounter u = PQueryCounter s
    using Bind.IH(1) first
  unfolding protocol-fields-preserving-def by blast
  have preserving-k: protocol-fields-preserving (k x)
    using Bind.IH(2) by blast
  have fields-t:
    PState t = PState u ∧
    PTranscript t = PTranscript u ∧
    PTraceFriCounter t = PTraceFriCounter u ∧
    PCompositionFriCounter t = PCompositionFriCounter u ∧
    PAlphaCounter t = PAlphaCounter u ∧
    PQueryCounter t = PQueryCounter u
    using preserving-k rest
  unfolding protocol-fields-preserving-def by blast
show
  PState t = PState s ∧
  PTranscript t = PTranscript s ∧
  PTraceFriCounter t = PTraceFriCounter s ∧
  PCompositionFriCounter t = PCompositionFriCounter s ∧
  PAlphaCounter t = PAlphaCounter s ∧
  PQueryCounter t = PQueryCounter s

```

```

    using fields-u fields-t by simp
  qed
next
  case (Weaken q m r)
  then show ?case by simp
qed

lemma hash-range-budget-throw:
  hash-range-budget 0
  (throw :: ('r, 'f protocol-channel) state-monad)
  unfolding hash-range-budget-def
  by (simp add: throw-no-outcome)

lemma hash-collision-budget-throw:
  hash-collision-budget 0
  (throw :: ('r, 'f protocol-channel) state-monad)
  unfolding hash-collision-budget-def wp-event-def wp-def
  throw.rep-eq dist-throw-def dist-delta-dist delta-map-def
  hash-collision-budget-value-def hash-new-collision-event-def
  by simp

lemma hash-target-program-throw:
  hash-target-program B 0
  (throw :: ('r, 'f protocol-channel) state-monad)
  by (rule hash-map-preserving-imp-hash-target-program-zero)
  (rule hash-map-preserving-throw)

lemma hash-range-budget-state-independent-sample:
  hash-range-budget 0
  (lift (λ-. d) ::
    ('r, 'f protocol-channel) state-monad)
  unfolding hash-range-budget-def lift.rep-eq lift-dist-def
  by (auto simp: set-dist-dist-map hash-map-output-values-def)

lemma hash-map-preserving-imp-hash-collision-budget-zero:
  fixes m :: ('r, 'f protocol-channel) state-monad
  assumes preserving: hash-map-preserving m
  shows hash-collision-budget 0 m
  unfolding hash-collision-budget-def
proof (intro allI impI)
  fix s :: 'f protocol-channel
  assume clean: ¬ hash-map-output-collision s
  have no-event:
     $\bigwedge out. out \in \text{set-dist } (\text{execute } m \ s) \implies$ 
     $\neg \text{hash-new-collision-event } s \ out$ 
  proof -
    fix out
    assume support: out  $\in$  set-dist (execute m s)
    show ¬ hash-new-collision-event s out
  end
end

```

```

proof (cases out)
  case None
  then show ?thesis
    unfolding hash-new-collision-event-def by simp
next
  case (Some xt)
  then obtain x t where xt: xt = (x, t)
    by (cases xt) simp
  have same: HashMap t = HashMap s
    using preserving support Some xt
    unfolding hash-map-preserving-def by blast
  show ?thesis
    using clean same Some xt
    unfolding hash-new-collision-event-def
      hash-map-new-output-collision-def hash-map-output-collision-def
    by simp
qed
qed
have no-event-dom:
   $\bigwedge out. out \in \text{dom} (\text{dist} (\text{execute } m \ s)) \implies$ 
   $\neg \text{hash-new-collision-event } s \ out$ 
  using no-event unfolding set-dist-def by blast
have wp-event m (hash-new-collision-event s) s = 0
  unfolding wp-event-def wp-def dist-expect-def
  by (intro sum.neutral) (simp add: no-event-dom)
then show
  wp-event m (hash-new-collision-event s) s  $\leq$ 
  hash-collision-budget-value
  (card (hash-map-output-values s)) 0
  unfolding hash-collision-budget-value-def by simp
qed

lemma hash-collision-budget-state-independent-sample:
  hash-collision-budget 0
  (lift ( $\lambda \cdot. d$ ) ::
    ('r, 'f protocol-channel) state-monad)
by (rule hash-map-preserving-imp-hash-collision-budget-zero)
  (rule hash-map-preserving-state-independent-sample)

lemma hash-target-program-state-independent-sample:
  hash-target-program B 0
  (lift ( $\lambda \cdot. d$ ) ::
    ('r, 'f protocol-channel) state-monad)
by (rule hash-map-preserving-imp-hash-target-program-zero)
  (rule hash-map-preserving-state-independent-sample)

lemma controlled-ro-program-extension:
  assumes controlled-ro-program q m
  shows hash-extension-preserving m

```

```

    using assms
  proof (induction rule: controlled-ro-program.induct)
    case (Return x)
    then show ?case by (rule hash-extension-preserving-return)
  next
    case Fail
    then show ?case
      by (rule hash-map-preserving-imp-hash-extension-preserving)
        (rule hash-map-preserving-throw)
  next
    case (Sample d)
    then show ?case
      by (rule hash-map-preserving-imp-hash-extension-preserving)
        (rule hash-map-preserving-state-independent-sample)
  next
    case (Query q k x)
    have ext-k:  $\bigwedge y. \text{hash-extension-preserving } (k \ y)$ 
      using Query.IH by blast
    show ?case
      by (rule hash-extension-preserving-bind)
        (rule hash-extension-preserving-hash, rule ext-k)
  next
    case (Bind q m r k)
    have ext-k:  $\bigwedge x. \text{hash-extension-preserving } (k \ x)$ 
      using Bind.IH(2) by blast
    show ?case
      by (rule hash-extension-preserving-bind[OF Bind.IH(1) ext-k])
  next
    case (Weaken q m r)
    then show ?case by simp
qed

lemma controlled-ro-program-range:
  assumes controlled-ro-program q m
  shows hash-range-budget q m
  using assms
proof (induction rule: controlled-ro-program.induct)
  case (Return x)
  then show ?case by (rule hash-range-budget-return)
next
  case Fail
  then show ?case by (rule hash-range-budget-throw)
next
  case (Sample d)
  then show ?case by (rule hash-range-budget-state-independent-sample)
next
  case (Query q k x)
  have range-k:  $\bigwedge y. \text{hash-range-budget } q \ (k \ y)$ 
    using Query.IH by blast

```

```

have hash-range-budget (1 + q)
  ((hash x :: ('f, 'f protocol-channel) state-monad)
     $\gg$  k)
  by (rule hash-range-budget-bind)
    (rule hash-range-budget-hash, rule range-k)
then show ?case by simp
next
  case (Bind q m r k)
  have range-k:  $\bigwedge x. \text{hash-range-budget } r \ (k \ x)$ 
    using Bind.IH(2) by blast
  show ?case
    by (rule hash-range-budget-bind[OF Bind.IH(1) range-k])
next
  case (Weaken q m r)
  show ?case
    by (rule hash-range-budget-mono[OF Weaken.hyps(2) Weaken.IH])
qed

lemma controlled-ro-program-collision:
  assumes controlled-ro-program q m
  shows hash-collision-budget q m
  using assms
proof (induction rule: controlled-ro-program.induct)
  case (Return x)
  then show ?case by (rule hash-collision-budget-return)
next
  case Fail
  then show ?case by (rule hash-collision-budget-throw)
next
  case (Sample d)
  then show ?case by (rule hash-collision-budget-state-independent-sample)
next
  case (Query q k x)
  have range-k:  $\bigwedge y. \text{hash-range-budget } q \ (k \ y)$ 
    using Query(1) by (blast intro: controlled-ro-program-range)
  have collision-k:  $\bigwedge y. \text{hash-collision-budget } q \ (k \ y)$ 
    using Query.IH by blast
  have hash-collision-budget (1 + q)
    ((hash x :: ('f, 'f protocol-channel) state-monad)
       $\gg$  k)
    by (rule hash-collision-budget-bind)
      (rule hash-range-budget-hash, rule hash-collision-budget-hash,
        rule range-k, rule collision-k)
  then show ?case by simp
next
  case (Bind q m r k)
  have range-m: hash-range-budget q m
    by (rule controlled-ro-program-range[OF Bind(1)])
  have range-k:  $\bigwedge x. \text{hash-range-budget } r \ (k \ x)$ 

```



```

    using Bind(3) by (blast intro: controlled-ro-program-range)
    have collision-k:  $\bigwedge x. \text{hash-collision-budget } r \ (k \ x)$ 
    using Bind.IH(2) by blast
    show ?case
    by (rule hash-collision-budget-bind)
      (rule range-m, rule Bind.IH(1), rule range-k, rule collision-k)
next
  case (Weaken q m r)
  show ?case
  by (rule hash-collision-budget-mono[OF Weaken.hyps(2) Weaken.IH])
qed

lemma controlled-ro-program-target:
  assumes controlled-ro-program q m
  shows hash-target-program B q m
  using assms
proof (induction rule: controlled-ro-program.induct)
  case (Return x)
  then show ?case by (rule hash-target-program-return)
next
  case Fail
  then show ?case by (rule hash-target-program-throw)
next
  case (Sample d)
  then show ?case by (rule hash-target-program-state-independent-sample)
next
  case (Query q k x)
  have target-k:  $\bigwedge y. \text{hash-target-program } B \ q \ (k \ y)$ 
  using Query.IH by blast
  have hash-target-program B (1 + q)
    ((hash x :: ('f, 'f protocol-channel) state-monad)
      $\gg= k$ )
  by (rule hash-target-program-bind)
    (rule hash-target-program-hash, rule target-k)
  then show ?case by simp
next
  case (Bind q m r k)
  have target-k:  $\bigwedge x. \text{hash-target-program } B \ r \ (k \ x)$ 
  using Bind.IH(2) by blast
  show ?case
  by (rule hash-target-program-bind[OF Bind.IH(1) target-k])
next
  case (Weaken q m r)
  show ?case
  by (rule hash-target-program-mono[OF Weaken.hyps(2) Weaken.IH])
qed

definition controlled-ro-admissible
  :: nat  $\Rightarrow$ 

```

$(\text{'r}, \text{'f protocol-channel}) \text{ state-monad} \Rightarrow \text{bool}$
where
 $\text{controlled-ro-admissible } q \ m \longleftrightarrow$
 $\text{hash-extension-preserving } m \wedge$
 $\text{hash-range-budget } q \ m \wedge$
 $\text{hash-collision-budget } q \ m \wedge$
 $(\forall B. \text{hash-target-program } B \ q \ m)$

lemma *controlled-ro-program-admissible:*
assumes *controlled-ro-program* $q \ m$
shows *controlled-ro-admissible* $q \ m$
unfolding *controlled-ro-admissible-def*
using *controlled-ro-program-extension* $[OF \ \text{assms}]$
controlled-ro-program-range $[OF \ \text{assms}]$
controlled-ro-program-collision $[OF \ \text{assms}]$
controlled-ro-program-target $[OF \ \text{assms}]$
by *blast*

definition *hash-relation-hit*
 $:: (\text{'f protocol-hash-input} \Rightarrow \text{'f} \Rightarrow \text{bool}) \Rightarrow$
 $\text{'f protocol-channel} \Rightarrow \text{'f protocol-channel} \Rightarrow \text{bool}$
where
 $\text{hash-relation-hit } R \ \text{initial} \ \text{final} \longleftrightarrow$
 $(\exists x \ y.$
 $\text{fmlookup } (\text{HashMap } \text{initial}) \ x = \text{None} \wedge$
 $\text{fmlookup } (\text{HashMap } \text{final}) \ x = \text{Some } y \wedge$
 $R \ x \ y)$

definition *hash-relation-hit-event*
 $:: (\text{'f protocol-hash-input} \Rightarrow \text{'f} \Rightarrow \text{bool}) \Rightarrow$
 $\text{'f protocol-channel} \Rightarrow$
 $(\text{'r} \times \text{'f protocol-channel}) \ \text{option} \Rightarrow \text{bool}$
where
 $\text{hash-relation-hit-event } R \ \text{initial} \ \text{out} \longleftrightarrow$
 $(\text{case out of}$
 $\text{None} \Rightarrow \text{False}$
 $| \text{Some } (-, \text{final}) \Rightarrow \text{hash-relation-hit } R \ \text{initial} \ \text{final})$

lemma *hash-relation-hit-after-update:*
 $\text{hash-relation-hit } R \ s$
 $(s \downarrow \text{HashMap} := \text{fmupd } x \ y \ (\text{HashMap } s)) \longleftrightarrow$
 $\text{fmlookup } (\text{HashMap } s) \ x = \text{None} \wedge R \ x \ y$
proof
assume *hit:*
 $\text{hash-relation-hit } R \ s$
 $(s \downarrow \text{HashMap} := \text{fmupd } x \ y \ (\text{HashMap } s))$
then obtain $k \ z$ **where** *fresh:*
 $\text{fmlookup } (\text{HashMap } s) \ k = \text{None}$
and *lookup:*

```

    fmlookup (fmupd x y (HashMap s)) k = Some z
  and rel: R k z
  by (auto simp: hash-relation-hit-def)
  have k = x
  proof (rule ccontr)
    assume k ≠ x
    then have fmlookup (HashMap s) k = Some z
      using lookup by simp
    then show False using fresh by simp
  qed
  then show fmlookup (HashMap s) x = None ∧ R x y
    using fresh lookup rel by simp
next
  assume fmlookup (HashMap s) x = None ∧ R x y
  then show
    hash-relation-hit R s
    (s(HashMap := fmupd x y (HashMap s)))
  unfolding hash-relation-hit-def by auto
qed

lemma wp-hash-relation-hit:
  fixes x :: 'f protocol-hash-input
  and s :: 'f protocol-channel
  shows
    wp-event (hash x) (hash-relation-hit-event R s) s =
      (if fmlookup (HashMap s) x = None
       then nnreal (card {y. R x y}) / nnreal size
       else 0)
  proof (cases fmlookup (HashMap s) x)
  case None
  have indicator:
    (λy. if hash-relation-hit R s
      (s(HashMap := fmupd x y (HashMap s)))
      then 1 else 0 :: prob) =
    (λy. if y ∈ {y. R x y} then 1 else 0)
  by (rule ext)
    (simp add: hash-relation-hit-after-update None)
  have wp-event (hash x) (hash-relation-hit-event R s) s =
    dist-expect (hash-dist x s)
    (λy. if y ∈ {y. R x y} then 1 else 0)
  unfolding wp-event-def hash-relation-hit-event-def
  by (simp add: wp-hash indicator)
  also have ... = nnreal (card {y. R x y}) / nnreal size
    by (rule dist-expect-hash-dist-fresh-indicator[OF None])
  finally show ?thesis using None by simp
next
  case (Some y)
  have no-hit:
    ¬ hash-relation-hit R s

```

```

    (s(|HashMap := fmupd x y (HashMap s)|))
  by (simp add: hash-relation-hit-after-update Some)
show ?thesis
  unfolding wp-event-def hash-relation-hit-event-def
  by (simp add: wp-hash Some hash-dist-def option-default-dist-def no-hit)
qed

```

definition *hash-relation-budget-value* :: $\text{nat} \Rightarrow \text{nat} \Rightarrow \text{prob}$
where
hash-relation-budget-value $b\ q =$
 $\text{nnreal } (q * b) / \text{nnreal size}$

definition *hash-relation-budget*
:: $(f \text{ protocol-hash-input} \Rightarrow f \Rightarrow \text{bool}) \Rightarrow$
 $\text{nat} \Rightarrow \text{nat} \Rightarrow$
 $(r, f \text{ protocol-channel}) \text{ state-monad} \Rightarrow \text{bool}$
where
hash-relation-budget $R\ b\ q\ m \longleftrightarrow$
 $(\forall s.$
 $\text{wp-event } m (\text{hash-relation-hit-event } R\ s) \ s \leq$
 $\text{hash-relation-budget-value } b\ q)$

definition *hash-relation-program*
:: $(f \text{ protocol-hash-input} \Rightarrow f \Rightarrow \text{bool}) \Rightarrow$
 $\text{nat} \Rightarrow \text{nat} \Rightarrow$
 $(r, f \text{ protocol-channel}) \text{ state-monad} \Rightarrow \text{bool}$
where
hash-relation-program $R\ b\ q\ m \longleftrightarrow$
 $\text{hash-extension-preserving } m \wedge \text{hash-relation-budget } R\ b\ q\ m$

lemma *hash-relation-budget-value-add*:
 $\text{hash-relation-budget-value } b\ q + \text{hash-relation-budget-value } b\ r =$
 $\text{hash-relation-budget-value } b\ (q + r)$
unfolding *hash-relation-budget-value-def*
by (simp add: add-divide-nnreal algebra-simps)

lemma *hash-relation-budget-value-mono*:
assumes $q \leq r$
shows $\text{hash-relation-budget-value } b\ q \leq \text{hash-relation-budget-value } b\ r$
unfolding *hash-relation-budget-value-def*
apply (subst nn2real-le-iff[symmetric])
using *assms* **by** (simp add: divide-right-mono mult-right-mono)

lemma *hash-relation-budget-value-mono-left*:
assumes $b \leq c$
shows $\text{hash-relation-budget-value } b\ q \leq \text{hash-relation-budget-value } c\ q$
unfolding *hash-relation-budget-value-def*
apply (subst nn2real-le-iff[symmetric])
using *assms* **by** (simp add: divide-right-mono mult-left-mono)

```

lemma hash-relation-hit-event-step-bound:
  fixes  $m :: ('r, 'f \text{ protocol-channel}) \text{ state-monad}$ 
  assumes  $\text{ext-st}: s \leq t$ 
  and  $\text{ext-m}: \text{hash-extension-preserving } m$ 
  shows
     $\text{wp-event } m (\text{hash-relation-hit-event } R \ s) \ t \leq$ 
       $(\text{if } \text{hash-relation-hit } R \ s \ t \text{ then } 1$ 
         $\text{else } \text{wp-event } m (\text{hash-relation-hit-event } R \ t) \ t)$ 
  proof  $(\text{cases } \text{hash-relation-hit } R \ s \ t)$ 
  case True
  then show ?thesis
    using  $\text{wp-event-le-1} [\text{of } m \ \text{hash-relation-hit-event } R \ s \ t]$  by simp
  next
  case False
  have mono:
     $\text{wp-event } m (\text{hash-relation-hit-event } R \ s) \ t \leq$ 
       $\text{wp-event } m (\text{hash-relation-hit-event } R \ t) \ t$ 
  proof  $(\text{rule } \text{wp-event-mono-on-support})$ 
  fix out
  assume support:  $\text{out} \in \text{set-dist } (\text{execute } m \ t)$ 
  and hit-s:  $\text{hash-relation-hit-event } R \ s \ \text{out}$ 
  show  $\text{hash-relation-hit-event } R \ t \ \text{out}$ 
  proof  $(\text{cases } \text{out})$ 
  case None
  then show ?thesis
    using hit-s unfolding  $\text{hash-relation-hit-event-def}$  by simp
  next
  case  $(\text{Some } xu)$ 
  then obtain  $x \ u$  where  $xu: xu = (x, u)$ 
  by  $(\text{cases } xu) \ \text{simp}$ 
  have out:  $\text{Some } (x, u) \in \text{set-dist } (\text{execute } m \ t)$ 
  using support  $\text{Some } xu$  by simp
  have ext-tu:  $t \leq u$ 
  using ext-m out unfolding  $\text{hash-extension-preserving-def}$  by blast
  from hit-s obtain  $k \ y$  where fresh-s:
     $\text{fmlookup } (\text{HashMap } s) \ k = \text{None}$ 
    and  $\text{lookup-u}: \text{fmlookup } (\text{HashMap } u) \ k = \text{Some } y$ 
    and  $\text{rel}: R \ k \ y$ 
  unfolding  $\text{hash-relation-hit-event-def}$   $\text{hash-relation-hit-def}$ 
  using  $\text{Some } xu$  by auto
  show ?thesis
  proof  $(\text{cases } \text{fmlookup } (\text{HashMap } t) \ k)$ 
  case None
  then show ?thesis
    using  $\text{Some } xu \ \text{lookup-u} \ \text{rel}$ 
    unfolding  $\text{hash-relation-hit-event-def}$   $\text{hash-relation-hit-def}$ 
    by auto
  next

```

```

    case (Some z)
    have lookup-u-z: fmllookup (HashMap u) k = Some z
      by (rule hash-extension-lookup[OF Some ext-tu])
    have z = y using lookup-u lookup-u-z by simp
    then have hash-relation-hit R s t
      unfolding hash-relation-hit-def
      using fresh-s Some rel by blast
    then show ?thesis using False by contradiction
  qed
qed
qed
then show ?thesis using False by simp
qed

```

```

lemma hash-relation-budget-bind:
  fixes m :: ('x, 'f protocol-channel) state-monad
  and k :: 'x  $\Rightarrow$  ('y, 'f protocol-channel) state-monad
  assumes m-ext: hash-extension-preserving m
  and m-budget: hash-relation-budget R b q m
  and k-ext:  $\bigwedge x. \text{hash-extension-preserving } (k\ x)$ 
  and k-budget:  $\bigwedge x. \text{hash-relation-budget } R\ b\ r\ (k\ x)$ 
  shows hash-relation-budget R b (q + r) (m  $\ggg$  k)
  unfolding hash-relation-budget-def
proof (intro allI)
  fix s :: 'f protocol-channel
  let ?E = hash-relation-hit-event R s
  let ?Tail =
     $\lambda out. \text{case out of}$ 
     $\text{None} \Rightarrow 0$ 
     $| \text{Some } (x, t) \Rightarrow$ 
     $\text{if hash-relation-hit } R\ s\ t \text{ then } 0$ 
     $\text{else wp-event } (k\ x) (\text{hash-relation-hit-event } R\ t)\ t$ 
  let ?R =  $\lambda out. (\text{if } ?E\ out \text{ then } 1 \text{ else } 0) + ?Tail\ out$ 
  have split:
     $\text{wp-event } (m \ggg k) ?E\ s \leq \text{wp } m\ ?R\ s$ 
proof -
  have exact:
     $\text{wp-event } (m \ggg k) ?E\ s =$ 
     $\text{wp } m$ 
     $(\lambda out. \text{case out of}$ 
     $\text{None} \Rightarrow 0$ 
     $| \text{Some } (x, t) \Rightarrow \text{wp-event } (k\ x) ?E\ t)\ s$ 
  unfolding wp-event-def hash-relation-hit-event-def
  by (simp add: wpsimps)
  have wp m
     $(\lambda out. \text{case out of}$ 
     $\text{None} \Rightarrow 0$ 
     $| \text{Some } (x, t) \Rightarrow \text{wp-event } (k\ x) ?E\ t)\ s$ 
     $\leq \text{wp } m\ ?R\ s$ 

```

```

proof (rule wp-mono-on-support)
  fix out
  assume support: out  $\in$  set-dist (execute m s)
  show (case out of
    None  $\Rightarrow$  0
    | Some (x, t)  $\Rightarrow$  wp-event (k x) ?E t  $\leq$  ?R out
  )
  proof (cases out)
    case None
    then show ?thesis
      unfolding hash-relation-hit-event-def by simp
  next
    case (Some xt)
    then obtain x t where xt: xt = (x, t)
      by (cases xt) simp
    have m-out: Some (x, t)  $\in$  set-dist (execute m s)
      using support Some xt by simp
    have ext-st: s  $\leq$  t
      using m-ext m-out unfolding hash-extension-preserving-def by blast
    have step:
      wp-event (k x) ?E t  $\leq$ 
        (if hash-relation-hit R s t then 1
         else wp-event (k x) (hash-relation-hit-event R t) t)
      by (rule hash-relation-hit-event-step-bound
        [OF ext-st k-ext[of x]])
    show ?thesis
      using Some xt step
      unfolding hash-relation-hit-event-def
      by (cases hash-relation-hit R s t) simp-all
  qed
qed
then show ?thesis unfolding exact .
qed
have head-bound:
  wp-event m ?E s  $\leq$  hash-relation-budget-value b q
  using m-budget unfolding hash-relation-budget-def by blast
have tail-bound:
  wp m ?Tail s  $\leq$  hash-relation-budget-value b r
proof (rule wp-le-const-on-support)
  fix out
  assume support: out  $\in$  set-dist (execute m s)
  show ?Tail out  $\leq$  hash-relation-budget-value b r
  proof (cases out)
    case None
    then show ?thesis
      unfolding hash-relation-budget-value-def by simp
  next
    case (Some xt)
    then obtain x t where xt: xt = (x, t)
      by (cases xt) simp

```

```

have tail:
  wp-event (k x) (hash-relation-hit-event R t) t ≤
    hash-relation-budget-value b r
  using k-budget[of x] unfolding hash-relation-budget-def by blast
show ?thesis using Some xt tail by simp
qed
qed
have wp-event (m ≫ k) ?E s ≤ wp m ?R s
  by (rule split)
also have ... = wp-event m ?E s + wp m ?Tail s
  unfolding wp-event-def wp-def dist-expect-def
  by (simp add: sum.distrib algebra-simps
    hash-relation-hit-event-def)
also have ... ≤
  hash-relation-budget-value b q + hash-relation-budget-value b r
  by (intro add-mono head-bound tail-bound)
also have ... = hash-relation-budget-value b (q + r)
  by (rule hash-relation-budget-value-add)
finally show
  wp-event (m ≫ k) (hash-relation-hit-event R s) s ≤
    hash-relation-budget-value b (q + r) .
qed

lemma hash-relation-budget-bind-on-outcomes:
  fixes m :: ('x, 'f protocol-channel) state-monad
  and k :: 'x ⇒ ('y, 'f protocol-channel) state-monad
  assumes m-ext: hash-extension-preserving m
  and m-budget: hash-relation-budget R b q m
  and k-ext:
    ∧ s x t. Some (x, t) ∈ set-dist (execute m s) ⇒
      hash-extension-preserving (k x)
  and k-budget:
    ∧ s x t. Some (x, t) ∈ set-dist (execute m s) ⇒
      hash-relation-budget R b r (k x)
  shows hash-relation-budget R b (q + r) (m ≫ k)
  unfolding hash-relation-budget-def
proof (intro allI)
  fix s :: 'f protocol-channel
  let ?E = hash-relation-hit-event R s
  let ?Tail =
    λ out. case out of
      None ⇒ 0
    | Some (x, t) ⇒
      if hash-relation-hit R s t then 0
      else wp-event (k x) (hash-relation-hit-event R t) t
  let ?R = λ out. (if ?E out then 1 else 0) + ?Tail out
  have split:
    wp-event (m ≫ k) ?E s ≤ wp m ?R s
  proof –

```



```

have exact:
  wp-event (m  $\gg$  k) ?E s =
    wp m
    ( $\lambda out.$  case out of
      None  $\Rightarrow 0$ 
      | Some (x, t)  $\Rightarrow$  wp-event (k x) ?E t) s
  unfolding wp-event-def hash-relation-hit-event-def
  by (simp add: wpsimps)
have wp m
  ( $\lambda out.$  case out of
    None  $\Rightarrow 0$ 
    | Some (x, t)  $\Rightarrow$  wp-event (k x) ?E t) s
   $\leq$  wp m ?R s
proof (rule wp-mono-on-support)
  fix out
  assume support: out  $\in$  set-dist (execute m s)
  show (case out of
    None  $\Rightarrow 0$ 
    | Some (x, t)  $\Rightarrow$  wp-event (k x) ?E t)  $\leq$  ?R out
  proof (cases out)
    case None
    then show ?thesis
    unfolding hash-relation-hit-event-def by simp
  next
    case (Some xt)
    then obtain x t where xt: xt = (x, t)
    by (cases xt) simp
    have m-out: Some (x, t)  $\in$  set-dist (execute m s)
    using support Some xt by simp
    have ext-st: s  $\leq$  t
    using m-ext m-out unfolding hash-extension-preserving-def by blast
    have step:
      wp-event (k x) ?E t  $\leq$ 
        (if hash-relation-hit R s t then 1
         else wp-event (k x) (hash-relation-hit-event R t) t)
    by (rule hash-relation-hit-event-step-bound
      [OF ext-st k-ext[OF m-out]])
    show ?thesis
    using Some xt step
    unfolding hash-relation-hit-event-def
    by (cases hash-relation-hit R s t) simp-all
  qed
qed
then show ?thesis unfolding exact .
qed
have head-bound:
  wp-event m ?E s  $\leq$  hash-relation-budget-value b q
  using m-budget unfolding hash-relation-budget-def by blast
have tail-bound:

```

```

  wp m ?Tail s ≤ hash-relation-budget-value b r
proof (rule wp-le-const-on-support)
  fix out
  assume support: out ∈ set-dist (execute m s)
  show ?Tail out ≤ hash-relation-budget-value b r
  proof (cases out)
    case None
    then show ?thesis
      unfolding hash-relation-budget-value-def by simp
  next
  case (Some xt)
  then obtain x t where xt: xt = (x, t)
    by (cases xt) simp
  have m-out: Some (x, t) ∈ set-dist (execute m s)
    using support Some xt by simp
  have tail:
    wp-event (k x) (hash-relation-hit-event R t) t ≤
      hash-relation-budget-value b r
    using k-budget[OF m-out] unfolding hash-relation-budget-def
    by blast
  show ?thesis using Some xt tail by simp
qed
qed
have wp-event (m ≫ k) ?E s ≤ wp m ?R s
  by (rule split)
also have ... = wp-event m ?E s + wp m ?Tail s
  unfolding wp-event-def wp-def dist-expect-def
  by (simp add: sum.distrib algebra-simps
    hash-relation-hit-event-def)
also have ... ≤
  hash-relation-budget-value b q + hash-relation-budget-value b r
  by (intro add-mono head-bound tail-bound)
also have ... = hash-relation-budget-value b (q + r)
  by (rule hash-relation-budget-value-add)
finally show
  wp-event (m ≫ k) (hash-relation-hit-event R s) s ≤
    hash-relation-budget-value b (q + r) .
qed

lemma hash-relation-program-bind:
  assumes m: hash-relation-program R b q m
    and k:  $\bigwedge x. \text{hash-relation-program } R \text{ b r } (k \ x)$ 
  shows hash-relation-program R b (q + r) (m ≫ k)
  unfolding hash-relation-program-def
proof
  show hash-extension-preserving (m ≫ k)
    by (rule hash-extension-preserving-bind)
    (use m k in ⟨simp-all add: hash-relation-program-def⟩)
  show hash-relation-budget R b (q + r) (m ≫ k)

```

```

    by (rule hash-relation-budget-bind)
      (use m k in ⟨simp-all add: hash-relation-program-def⟩)
qed

lemma hash-relation-program-bind-on-outcomes:
  fixes m :: ('x, 'f protocol-channel) state-monad
    and k :: 'x ⇒ ('y, 'f protocol-channel) state-monad
  assumes m: hash-relation-program R b q m
    and k:
       $\bigwedge s\ x\ t. \text{Some } (x, t) \in \text{set-dist } (\text{execute } m\ s) \implies$ 
      hash-relation-program R b r (k x)
  shows hash-relation-program R b (q + r) (m  $\gg$  k)
  unfolding hash-relation-program-def
proof
  show hash-extension-preserving (m  $\gg$  k)
    by (rule hash-extension-preserving-bind-on-outcomes)
      (use m k in ⟨simp-all add: hash-relation-program-def⟩)
  show hash-relation-budget R b (q + r) (m  $\gg$  k)
    by (rule hash-relation-budget-bind-on-outcomes)
      (use m k in ⟨simp-all add: hash-relation-program-def⟩)
qed

lemma hash-relation-program-zero:
  assumes preserving: hash-map-preserving m
  shows hash-relation-program R b 0 m
  unfolding hash-relation-program-def hash-relation-budget-def
proof
  show hash-extension-preserving m
    by (rule hash-map-preserving-imp-hash-extension-preserving[OF preserving])
  show  $\forall s.$ 
    wp-event m (hash-relation-hit-event R s) s  $\leq$ 
    hash-relation-budget-value b 0
proof
  fix s
  have no-event-support:
     $\bigwedge \text{out}. \text{out} \in \text{set-dist } (\text{execute } m\ s) \implies$ 
     $\neg \text{hash-relation-hit-event } R\ s\ \text{out}$ 
proof -
  fix out
  assume support: out  $\in \text{set-dist } (\text{execute } m\ s)$ 
  show  $\neg \text{hash-relation-hit-event } R\ s\ \text{out}$ 
  proof (cases out)
    case None
    then show ?thesis
      unfolding hash-relation-hit-event-def by simp
  next
    case (Some xt)
    then obtain x t where xt: xt = (x, t)
      by (cases xt) simp

```

```

    have same: HashMap t = HashMap s
      using preserving support Some xt
      unfolding hash-map-preserving-def by blast
    show ?thesis
      using same Some xt
      unfolding hash-relation-hit-event-def hash-relation-hit-def
      by auto
  qed
qed
have no-event:
   $\bigwedge out. out \in \text{dom } (\text{dist } (\text{execute } m \ s)) \implies$ 
   $\neg \text{hash-relation-hit-event } R \ s \ out$ 
  using no-event-support unfolding set-dist-def by blast
have wp-event m (hash-relation-hit-event R s) s = 0
  unfolding wp-event-def wp-def dist-expect-def
  by (intro sum.neutral) (simp add: no-event)
then show
  wp-event m (hash-relation-hit-event R s) s  $\leq$ 
  hash-relation-budget-value b 0
  unfolding hash-relation-budget-value-def by simp
qed
qed

lemma hash-relation-program-hash:
  assumes fibers:  $\bigwedge x. \text{card } \{y. R \ x \ y\} \leq b$ 
  shows
    hash-relation-program R b 1
    (hash x :: ('f, 'f protocol-channel) state-monad)
  unfolding hash-relation-program-def hash-relation-budget-def
proof
  show hash-extension-preserving
    (hash x :: ('f, 'f protocol-channel) state-monad)
    by (rule hash-extension-preserving-hash)
  show  $\forall s.$ 
    wp-event (hash x) (hash-relation-hit-event R s) s  $\leq$ 
    hash-relation-budget-value b 1
proof
  fix s :: 'f protocol-channel
  have
    wp-event (hash x) (hash-relation-hit-event R s) s  $\leq$ 
    nnreal (card {y. R x y}) / nnreal size
    unfolding wp-hash-relation-hit by simp
  also have ...  $\leq$  nnreal b / nnreal size
    by (rule nnreal-nat-divide-right-mono)
    (rule fibers)
  also have ... = hash-relation-budget-value b 1
    unfolding hash-relation-budget-value-def by simp
  finally show
    wp-event (hash x) (hash-relation-hit-event R s) s  $\leq$ 

```

```

      hash-relation-budget-value b 1 .
qed
qed

lemma hash-relation-program-mono:
  assumes qr:  $q \leq r$ 
  and program: hash-relation-program R b q m
  shows hash-relation-program R b r m
  unfolding hash-relation-program-def hash-relation-budget-def
proof
  show hash-extension-preserving m
  using program unfolding hash-relation-program-def by simp
  show  $\forall s.$ 
    wp-event m (hash-relation-hit-event R s)  $s \leq$ 
      hash-relation-budget-value b r
proof
  fix s
  have
    wp-event m (hash-relation-hit-event R s)  $s \leq$ 
      hash-relation-budget-value b q
  using program
  unfolding hash-relation-program-def hash-relation-budget-def by blast
  also have  $\dots \leq$  hash-relation-budget-value b r
  by (rule hash-relation-budget-value-mono[OF qr])
  finally show
    wp-event m (hash-relation-hit-event R s)  $s \leq$ 
      hash-relation-budget-value b r .
qed
qed

lemma controlled-ro-program-relation:
  assumes controlled: controlled-ro-program q m
  and fibers:  $\bigwedge x. \text{card } \{y. R \ x \ y\} \leq b$ 
  shows hash-relation-program R b q m
  using controlled
proof (induction rule: controlled-ro-program.induct)
  case (Return x)
  show ?case
  by (rule hash-relation-program-zero)
    (rule hash-map-preserving-return)
next
  case Fail
  show ?case
  by (rule hash-relation-program-zero)
    (rule hash-map-preserving-throw)
next
  case (Sample d)
  show ?case
  by (rule hash-relation-program-zero)

```

```

      (rule hash-map-preserving-state-independent-sample)
next
  case (Query q k x)
  have tail:  $\bigwedge y. \text{hash-relation-program } R \ b \ q \ (k \ y)$ 
    using Query.IH fibers by blast
  have hash-relation-program  $R \ b \ (1 + q)$ 
    ((hash x :: ('f, 'f protocol-channel) state-monad)  $\ggg k$ )
    by (rule hash-relation-program-bind)
    (rule hash-relation-program-hash[OF fibers], rule tail)
  then show ?case by simp
next
  case (Bind q m r k)
  have tail:  $\bigwedge x. \text{hash-relation-program } R \ b \ r \ (k \ x)$ 
    using Bind.IH(2) by blast
  show ?case
    by (rule hash-relation-program-bind[OF Bind.IH(1) tail])
next
  case (Weaken q m r)
  show ?case
    by (rule hash-relation-program-mono[OF Weaken.hyps(2)
      Weaken.IH])
qed

theorem controlled-ro-program-relation-bound:
  assumes controlled: controlled-ro-program q m
  and fibers:  $\bigwedge x. \text{card } \{y. R \ x \ y\} \leq b$ 
  shows
    wp-event m (hash-relation-hit-event R s)  $s \leq$ 
      nnreal (q * b) / nnreal size
proof –
  have program: hash-relation-program R b q m
    by (rule controlled-ro-program-relation[OF controlled fibers])
  have budget: hash-relation-budget R b q m
    using program unfolding hash-relation-program-def by simp
  show ?thesis
    using budget unfolding hash-relation-budget-def
      hash-relation-budget-value-def by blast
qed

end

end

theory Security-Experiment
imports
  Soundness-Oracle
begin

```

type-synonym 'f semantic-adversary =
 ('f list, 'f protocol-channel) state-monad

context soundness
begin

A semantic adversary is an arbitrary probabilistic state-monad program that returns a complete proof transcript. Admissibility states the random-oracle laws needed by the security proof: oracle consistency, bounded range growth, bounded collisions, and bounded hits on every fixed target set.

This interface deliberately does not prescribe an attacker programming language. A future capability-safe command language may expose only probabilistic computation and random-oracle queries, with a refinement theorem showing that every query-bounded command program satisfies *admissible-adversary*.

definition *adversary-initial-state* :: 'f protocol-channel
where
adversary-initial-state =
 (λ HashMap = fmempty, PState = 0, PTranscript = [],
 PTraceFriCounter = 0, PCompositionFriCounter = 0,
 PAlphaCounter = 0, PQueryCounter = 0)

definition *admissible-adversary*
 :: nat $\Rightarrow$ 'f semantic-adversary $\Rightarrow$ bool
where
admissible-adversary q A $\longleftrightarrow$
 hash-extension-preserving A $\wedge$
 hash-range-budget q A $\wedge$
 hash-collision-budget q A $\wedge$
 ($\forall$ B. hash-target-program B q A)

lemma *adversary-initial-state-simps*[simp]:
 HashMap *adversary-initial-state* = fmempty
 PState *adversary-initial-state* = 0
 PTranscript *adversary-initial-state* = []
 PTraceFriCounter *adversary-initial-state* = 0
 PCompositionFriCounter *adversary-initial-state* = 0
 PAlphaCounter *adversary-initial-state* = 0
 PQueryCounter *adversary-initial-state* = 0
unfolding *adversary-initial-state-def* **by** *simp-all*

lemma *adversary-initial-state-eq-verifier-initial-state*:
adversary-initial-state = *verifier-initial-state* []
unfolding *adversary-initial-state-def* *verifier-initial-state-def* **by** *simp*

lemma *admissible-adversary-extension*:
assumes *admissible-adversary* q A
shows hash-extension-preserving A
using *assms* **unfolding** *admissible-adversary-def* **by** *simp*

lemma *admissible-adversary-range*:
assumes *admissible-adversary* q A
shows *hash-range-budget* q A
using *assms* **unfolding** *admissible-adversary-def* **by** *simp*

lemma *admissible-adversary-collision*:
assumes *admissible-adversary* q A
shows *hash-collision-budget* q A
using *assms* **unfolding** *admissible-adversary-def* **by** *simp*

lemma *admissible-adversary-target*:
assumes *admissible-adversary* q A
shows *hash-target-program* B q A
using *assms* **unfolding** *admissible-adversary-def* **by** *simp*

definition *verifier-state-from-adversary*
 $:: 'f \text{ protocol-channel} \Rightarrow 'f \text{ list} \Rightarrow 'f \text{ protocol-channel}$
where
verifier-state-from-adversary s $tr =$
 $(\text{HashMap} = \text{HashMap } s, \text{PState} = 0, \text{PTranscript} = tr,$
 $\text{PTraceFriCounter} = 0, \text{PCompositionFriCounter} = 0,$
 $\text{PAlphaCounter} = 0, \text{PQueryCounter} = 0)$

lemma *verifier-state-from-adversary-simps*[*simp*]:
 $\text{HashMap } (\text{verifier-state-from-adversary } s \text{ } tr) = \text{HashMap } s$
 $\text{PState } (\text{verifier-state-from-adversary } s \text{ } tr) = 0$
 $\text{PTranscript } (\text{verifier-state-from-adversary } s \text{ } tr) = tr$
 $\text{PTraceFriCounter } (\text{verifier-state-from-adversary } s \text{ } tr) = 0$
 $\text{PCompositionFriCounter } (\text{verifier-state-from-adversary } s \text{ } tr) = 0$
 $\text{PAlphaCounter } (\text{verifier-state-from-adversary } s \text{ } tr) = 0$
 $\text{PQueryCounter } (\text{verifier-state-from-adversary } s \text{ } tr) = 0$
unfolding *verifier-state-from-adversary-def* **by** *simp-all*

lemma *verifier-state-from-adversary-empty*:
 $\text{verifier-state-from-adversary adversary-initial-state } tr =$
 $\text{verifier-initial-state } tr$
unfolding *verifier-state-from-adversary-def* *adversary-initial-state-def*
 $\text{verifier-initial-state-def}$
by *simp*

lemma *verifier-state-from-adversary-hash-extends*:
assumes $s \leq t$
shows $\text{verifier-state-from-adversary } s \text{ } tr \leq$
 $\text{verifier-state-from-adversary } t \text{ } tr$
using *assms*
unfolding *verifier-state-from-adversary-def* *less-eq-hash-ext-def*
 less-eq-fmap-def
by *simp*

definition *verifier-state-transfer*
 $:: 'f \text{ list} \Rightarrow (\text{unit}, 'f \text{ protocol-channel}) \text{ state-monad}$
where
verifier-state-transfer *tr* =
 $\text{get} \gg (\lambda s. \text{put} (\text{verifier-state-from-adversary } s \text{ tr}))$

lemma *verifier-state-transfer-outcomeE*:
assumes
 $\text{Some } ((), t) \in \text{set-dist } (\text{execute } (\text{verifier-state-transfer } tr) \ s)$
shows $t = \text{verifier-state-from-adversary } s \text{ tr}$
using *assms* **unfolding** *verifier-state-transfer-def*
by (*auto elim!*: *set-dist-bindE*)

lemma *hash-map-preserving-verifier-state-transfer*:
hash-map-preserving (*verifier-state-transfer* *tr*)
unfolding *hash-map-preserving-def*
using *verifier-state-transfer-outcomeE* **by** *fastforce*

lemma *hash-map-preserving-imp-hash-range-budget-zero-semantic*:
fixes $m :: ('r, 'f \text{ protocol-channel}) \text{ state-monad}$
assumes *preserving*: *hash-map-preserving* *m*
shows *hash-range-budget* 0 *m*
using *preserving*
unfolding *hash-range-budget-def* *hash-map-preserving-def*
hash-map-output-values-def
by *auto*

lemma *hash-map-preserving-imp-hash-collision-budget-zero-semantic*:
fixes $m :: ('r, 'f \text{ protocol-channel}) \text{ state-monad}$
assumes *preserving*: *hash-map-preserving* *m*
shows *hash-collision-budget* 0 *m*
unfolding *hash-collision-budget-def*

proof (*intro allI impI*)
fix $s :: 'f \text{ protocol-channel}$
assume *clean*: $\neg \text{hash-map-output-collision } s$
have *no-event*:
 $\bigwedge \text{out}. \text{out} \in \text{set-dist } (\text{execute } m \ s) \implies$
 $\neg \text{hash-new-collision-event } s \text{ out}$

proof –
fix *out*
assume *support*: $\text{out} \in \text{set-dist } (\text{execute } m \ s)$
show $\neg \text{hash-new-collision-event } s \text{ out}$
proof (*cases out*)
case *None*
then show *?thesis*
unfolding *hash-new-collision-event-def* **by** *simp*

next
case (*Some xt*)

```

then obtain  $x\ t$  where  $xt: xt = (x, t)$ 
  by (cases  $xt$ ) simp
have same:  $\text{HashMap } t = \text{HashMap } s$ 
  using preserving support Some xt
  unfolding hash-map-preserving-def by blast
show ?thesis
  using clean same Some xt
  unfolding hash-new-collision-event-def
    hash-map-new-output-collision-def hash-map-output-collision-def
  by simp
qed
qed
have no-event-dom:
   $\bigwedge \text{out. out} \in \text{dom } (\text{dist } (\text{execute } m\ s)) \implies$ 
     $\neg \text{hash-new-collision-event } s\ \text{out}$ 
  using no-event unfolding set-dist-def by blast
have wp-event m (hash-new-collision-event s) s = 0
  unfolding wp-event-def wp-def dist-expect-def
  by (intro sum.neutral) (simp add: no-event-dom)
then show
   $\text{wp-event } m\ (\text{hash-new-collision-event } s)\ s \leq$ 
     $\text{hash-collision-budget-value}$ 
     $(\text{card } (\text{hash-map-output-values } s))\ 0$ 
  unfolding hash-collision-budget-value-def by simp
qed

lemma hash-range-budget-verifier-state-transfer:
   $\text{hash-range-budget } 0\ (\text{verifier-state-transfer } tr)$ 
  by (rule hash-map-preserving-imp-hash-range-budget-zero-semantic)
    (rule hash-map-preserving-verifier-state-transfer)

lemma hash-collision-budget-verifier-state-transfer:
   $\text{hash-collision-budget } 0\ (\text{verifier-state-transfer } tr)$ 
  by (rule hash-map-preserving-imp-hash-collision-budget-zero-semantic)
    (rule hash-map-preserving-verifier-state-transfer)

lemma hash-range-budget-verifier-after-adversary:
   $\text{hash-range-budget } \text{verifier-hash-query-budget}$ 
   $(\text{verifier-state-transfer } tr \gg (\lambda\cdot. \text{verify-monad}))$ 
proof –
  have range:
     $\text{hash-range-budget } (0 + \text{verifier-hash-query-budget})$ 
     $(\text{verifier-state-transfer } tr \gg (\lambda\cdot. \text{verify-monad}))$ 
  by (rule hash-range-budget-bind)
    (rule hash-range-budget-verifier-state-transfer,
      rule hash-range-budget-verify-monad)
  then show ?thesis by simp
qed

```

lemma *hash-collision-budget-verifier-after-adversary*:
hash-collision-budget verifier-hash-query-budget
(verifier-state-transfer tr $\gg$ (λ -. verify-monad))

proof –
have *collision*:
hash-collision-budget (0 + verifier-hash-query-budget)
(verifier-state-transfer tr $\gg$ (λ -. verify-monad))
by (*rule hash-collision-budget-bind*)
(rule hash-range-budget-verifier-state-transfer,
rule hash-collision-budget-verifier-state-transfer,
rule hash-range-budget-verify-monad,
rule hash-collision-budget-verify-monad)
then show *?thesis* **by** *simp*

qed

definition *security-experiment*
 $:: 'f \text{ semantic-adversary} \Rightarrow$
 $(\text{unit list}, 'f \text{ protocol-channel}) \text{ state-monad}$
where
security-experiment A =
 $A \gg (\lambda tr.$
 $\text{verifier-state-transfer tr } \gg (\lambda \text{-. verify-monad}))$

definition *adversary-acceptance-probability*
 $:: 'f \text{ semantic-adversary} \Rightarrow \text{prob}$
where
adversary-acceptance-probability A =
 $\text{wp-event } (\text{security-experiment } A)$
 $(\lambda \text{out. } \neg \text{Option.is-none out}) \text{ adversary-initial-state}$

lemma *security-experiment-outcomeE*:
assumes *outcome*:
 $\text{Some } (\text{result}, \text{final-state}) \in$
 $\text{set-dist } (\text{execute } (\text{security-experiment } A) \text{ initial-state})$
obtains *tr attacker-state* **where**
 $\text{Some } (tr, \text{attacker-state}) \in \text{set-dist } (\text{execute } A \text{ initial-state})$
 $\text{Some } (\text{result}, \text{final-state}) \in$
 set-dist
 $(\text{execute verify-monad}$
 $(\text{verifier-state-from-adversary attacker-state tr}))$
using *outcome*
unfolding *security-experiment-def verifier-state-transfer-def*
by (*auto elim!:* *set-dist-bindE* *intro:* *that*)

lemma *accepted-security-experimentE*:
assumes *outcome*:
 $\text{out} \in \text{set-dist}$
 $(\text{execute } (\text{security-experiment } A) \text{ adversary-initial-state})$
and *accepted*: $\neg \text{Option.is-none out}$

```

obtains result final-state tr attacker-state where
  out = Some (result, final-state)
  Some (tr, attacker-state) ∈
    set-dist (execute A adversary-initial-state)
  Some (result, final-state) ∈
    set-dist
      (execute verify-monad
        (verifier-state-from-adversary attacker-state tr))
proof –
  from accepted obtain result final-state where
    out: out = Some (result, final-state)
    by (cases out) auto
  from security-experiment-outcomeE[OF outcome[unfolded out]]
  obtain tr attacker-state where
    attacker:
      Some (tr, attacker-state) ∈
        set-dist (execute A adversary-initial-state)
    and verifier:
      Some (result, final-state) ∈
        set-dist
          (execute verify-monad
            (verifier-state-from-adversary attacker-state tr))
    by blast
  show ?thesis
    by (rule that[OF out attacker verifier])
qed

lemma attacker-failure-not-accepted[simp]:
  ¬ (¬ Option.is-none
    (None :: (unit list × 'f protocol-channel) option))
  by simp

lemma security-experiment-success-requires-attacker-success:
  assumes no-success:
    ∧ tr attacker-state.
      Some (tr, attacker-state) ∉ set-dist (execute A initial-state)
  shows
    Some (result, final-state) ∉
      set-dist (execute (security-experiment A) initial-state)
proof
  assume outcome:
    Some (result, final-state) ∈
      set-dist (execute (security-experiment A) initial-state)
  from security-experiment-outcomeE[OF outcome]
  obtain tr attacker-state where
    Some (tr, attacker-state) ∈ set-dist (execute A initial-state)
    by blast
  then show False
    using no-success[of tr attacker-state] by blast

```

qed

lemma *admissible-adversary-outcome-extends*:
 assumes *admissible*: *admissible-adversary* q A
 and *outcome*:
 Some (tr , *attacker-state*) $\in$
 set-dist (*execute* A *adversary-initial-state*)
 shows *adversary-initial-state* $\leq$ *attacker-state*
 using *admissible-adversary-extension*[*OF* *admissible*] *outcome*
 unfolding *hash-extension-preserving-def* **by** *blast*

lemma *hash-map-output-values-adversary-initial-state*[*simp*]:
 hash-map-output-values *adversary-initial-state* = $\{\}$
 unfolding *adversary-initial-state-eq-verifier-initial-state* **by** *simp*

lemma *adversary-initial-state-no-output-collision*[*simp*]:
 $\neg$ *hash-map-output-collision* *adversary-initial-state*
 unfolding *hash-map-output-collision-def* **by** *simp*

lemma *admissible-adversary-output-range-bound*:
 assumes *admissible*: *admissible-adversary* q A
 and *outcome*:
 Some (tr , *attacker-state*) $\in$
 set-dist (*execute* A *adversary-initial-state*)
 shows *card* (*hash-map-output-values* *attacker-state*) $\leq q$
proof –
 have
 card (*hash-map-output-values* *attacker-state*) $\leq$
 card (*hash-map-output-values* *adversary-initial-state*) + q
 using *admissible-adversary-range*[*OF* *admissible*] *outcome*
 unfolding *hash-range-budget-def* **by** *blast*
 then show ?thesis **by** *simp*
qed

lemma *admissible-adversary-collision-bound*:
 assumes *admissible*: *admissible-adversary* q A
 shows
 wp-event A (*hash-new-collision-event* *adversary-initial-state*)
 adversary-initial-state $\leq$ *hash-collision-budget-value* 0 q
proof –
 have *collision-budget*: *hash-collision-budget* q A
 by (*rule* *admissible-adversary-collision*[*OF* *admissible*])
 have
 wp-event A (*hash-new-collision-event* *adversary-initial-state*)
 adversary-initial-state $\leq$
 hash-collision-budget-value
 (*card* (*hash-map-output-values* *adversary-initial-state*)) q
 using *collision-budget*
 adversary-initial-state-no-output-collision

```

    unfolding hash-collision-budget-def by blast
  then show ?thesis by simp
qed

lemma admissible-adversary-target-bound:
  assumes admissible: admissible-adversary q A
  shows
    wp-event A
    (hash-new-output-hit-event B adversary-initial-state)
    adversary-initial-state  $\leq$  hash-target-budget-value B q
  proof -
    have hash-target-budget B q A
    by (rule hash-target-program-budget)
      (rule admissible-adversary-target[OF admissible])
    then show ?thesis
      unfolding hash-target-budget-def by blast
  qed

lemma hash-range-budget-security-experiment:
  assumes admissible: admissible-adversary q A
  shows
    hash-range-budget (q + verifier-hash-query-budget)
    (security-experiment A)
  unfolding security-experiment-def
  by (rule hash-range-budget-bind)
    (rule admissible-adversary-range[OF admissible],
     rule hash-range-budget-verifier-after-adversary)

lemma hash-collision-budget-security-experiment:
  assumes admissible: admissible-adversary q A
  shows
    hash-collision-budget (q + verifier-hash-query-budget)
    (security-experiment A)
  unfolding security-experiment-def
  by (rule hash-collision-budget-bind)
    (rule admissible-adversary-range[OF admissible],
     rule admissible-adversary-collision[OF admissible],
     rule hash-range-budget-verifier-after-adversary,
     rule hash-collision-budget-verifier-after-adversary)

lemma security-experiment-hash-new-collision-bound:
  assumes admissible: admissible-adversary q A
  shows
    wp-event (security-experiment A)
    (hash-new-collision-event adversary-initial-state)
    adversary-initial-state  $\leq$ 
    hash-collision-budget-value 0 (q + verifier-hash-query-budget)
  proof -
    have collision-budget:

```

```

    hash-collision-budget (q + verifier-hash-query-budget)
    (security-experiment A)
  by (rule hash-collision-budget-security-experiment[OF admissible])
have
  wp-event (security-experiment A)
    (hash-new-collision-event adversary-initial-state)
    adversary-initial-state ≤
    hash-collision-budget-value
      (card (hash-map-output-values adversary-initial-state))
      (q + verifier-hash-query-budget)
  using collision-budget adversary-initial-state-no-output-collision
  unfolding hash-collision-budget-def by blast
then show ?thesis by simp
qed

lemma hash-map-output-collision-bad-imp-security-new-collision:
  assumes bad: hash-map-output-collision-bad adversary-initial-state out
  shows hash-new-collision-event adversary-initial-state out
proof -
  have new-bad: hash-map-new-output-collision-bad adversary-initial-state out
  by (rule hash-map-output-collision-bad-imp-new-collision-bad-if-initial-clean)
    (use bad in simp-all)
  show ?thesis
  by (rule hash-map-new-output-collision-bad-imp-hash-new-collision-event
    [OF new-bad])
qed

lemma security-experiment-hash-map-output-collision-bad-bound:
  assumes admissible: admissible-adversary q A
  shows
    wp-event (security-experiment A)
      (hash-map-output-collision-bad adversary-initial-state)
      adversary-initial-state ≤
      hash-collision-budget-value 0 (q + verifier-hash-query-budget)
proof -
  have wp-event (security-experiment A)
    (hash-map-output-collision-bad adversary-initial-state)
    adversary-initial-state ≤
    wp-event (security-experiment A)
      (hash-new-collision-event adversary-initial-state)
      adversary-initial-state
  by (rule wp-event-mono)
    (rule hash-map-output-collision-bad-imp-security-new-collision)
  also have ... ≤
    hash-collision-budget-value 0 (q + verifier-hash-query-budget)
  by (rule security-experiment-hash-new-collision-bound[OF admissible])
  finally show ?thesis .
qed

```

end

end

theory *Staged-Security-Experiment-Core*
 imports *Controlled-RO Security-Experiment*
begin

Core staged-adversary records, transcript builders, and experiment projections.

record *staged-budgets* =
 trace-root-budget :: *nat*
 trace-fri-budgets :: *nat list*
 trace-final-budget :: *nat*
 degree-budget :: *nat*
 composition-fri-budgets :: *nat list*
 composition-final-budget :: *nat*
 query-opening-budgets :: *nat list*

record *'f staged-adversary* =
 trace-root-stage :: (*'f*, *'f protocol-channel*) *state-monad*
 trace-fri-root-stage ::
 nat $\Rightarrow$ *'f list* $\Rightarrow$ (*'f*, *'f protocol-channel*) *state-monad*
 trace-final-stage ::
 'f list $\Rightarrow$ (*'f*, *'f protocol-channel*) *state-monad*
 degree-stage ::
 'f list $\Rightarrow$ (*'f*, *'f protocol-channel*) *state-monad*
 composition-fri-root-stage ::
 'f $\Rightarrow$ *nat* $\Rightarrow$ *'f list* $\Rightarrow$
 (*'f*, *'f protocol-channel*) *state-monad*
 composition-final-stage ::
 'f $\Rightarrow$ *'f list* $\Rightarrow$ (*'f*, *'f protocol-channel*) *state-monad*
 query-opening-stage ::
 nat $\Rightarrow$ *'f* $\Rightarrow$ (*'f list*, *'f protocol-channel*) *state-monad*

record *'f staged-proof-data* =
 staged-trace-root :: *'f*
 staged-trace-fri-roots :: *'f list*
 staged-trace-fri-challenges :: *'f list*
 staged-trace-final :: *'f*
 staged-alphas :: *'f list*
 staged-degree :: *'f*
 staged-composition-fri-roots :: *'f list*
 staged-composition-fri-challenges :: *'f list*
 staged-composition-final :: *'f*
 staged-query-chunks :: *'f list list*

record *'f staged-composition-prefix-data* =
 scp-trace-root :: *'f*


```

    scp-trace-fri-roots :: 'f list
    scp-trace-fri-challenges :: 'f list
    scp-trace-final :: 'f
    scp-alphas :: 'f list
    scp-degree :: 'f
    scp-composition-fri-roots :: 'f list
    scp-composition-fri-challenges :: 'f list

record 'f staged-query-prefix-data =
  sqp-trace-root :: 'f
  sqp-trace-fri-roots :: 'f list
  sqp-trace-fri-challenges :: 'f list
  sqp-trace-final :: 'f
  sqp-alphas :: 'f list
  sqp-degree :: 'f
  sqp-composition-fri-roots :: 'f list
  sqp-composition-fri-challenges :: 'f list
  sqp-composition-final :: 'f
  sqp-query-chunks :: 'f list list

context soundness
begin

lemma wp-event-bind-return-map:
  fixes m :: ('x, 'f protocol-channel) state-monad
  shows
    wp-event (m  $\gg$  (λx. return (f x))) P s =
    wp-event m (λout. case out of None  $\Rightarrow$  P None
      | Some (x, t)  $\Rightarrow$  P (Some (f x, t))) s
proof -
  have indicator-eq:
    (λout. case out of None  $\Rightarrow$  if P None then 1 else 0
      | Some (x, t)  $\Rightarrow$  if P (Some (f x, t)) then 1 else 0) =
    (λout. if (case out of None  $\Rightarrow$  P None
      | Some (x, t)  $\Rightarrow$  P (Some (f x, t))) then 1 else 0)
  by (rule ext) (simp split: option.splits prod.splits)
  show ?thesis
  unfolding wp-event-def
  apply (subst wp-bind)
  apply (rule arg-cong[where f=λQ. wp m Q s])
  apply (rule ext)
  apply (simp add: wp-return indicator-eq
    split: option.splits prod.splits)
  done
qed

lemma wp-event-bind-mono-cont:
  fixes m :: ('x, 'f protocol-channel) state-monad
  and k :: 'x  $\Rightarrow$  ('y, 'f protocol-channel) state-monad

```

```

and  $l :: 'x \Rightarrow ('z, 'f \text{ protocol-channel}) \text{ state-monad}$ 
assumes  $\text{none}: P \text{ None} \Longrightarrow Q \text{ None}$ 
and  $\text{cont}$ :
   $\bigwedge x t. \text{Some } (x, t) \in \text{set-dist } (\text{execute } m \ s) \Longrightarrow$ 
     $\text{wp-event } (k \ x) \ P \ t \leq \text{wp-event } (l \ x) \ Q \ t$ 
shows  $\text{wp-event } (m \ggg k) \ P \ s \leq \text{wp-event } (m \ggg l) \ Q \ s$ 
proof –
  have  $\text{left}$ :
     $\text{wp-event } (m \ggg k) \ P \ s =$ 
     $\text{wp } m \ (\lambda \text{out}. \text{case out of}$ 
       $\text{None} \Rightarrow \text{if } P \text{ None then } 1 \text{ else } 0$ 
       $| \text{Some } (x, t) \Rightarrow \text{wp-event } (k \ x) \ P \ t) \ s$ 
    unfolding  $\text{wp-event-def}$  by  $(\text{simp add: wpsimps})$ 
  have  $\text{right}$ :
     $\text{wp-event } (m \ggg l) \ Q \ s =$ 
     $\text{wp } m \ (\lambda \text{out}. \text{case out of}$ 
       $\text{None} \Rightarrow \text{if } Q \text{ None then } 1 \text{ else } 0$ 
       $| \text{Some } (x, t) \Rightarrow \text{wp-event } (l \ x) \ Q \ t) \ s$ 
    unfolding  $\text{wp-event-def}$  by  $(\text{simp add: wpsimps})$ 
  show  $?thesis$ 
    unfolding  $\text{left right}$ 
  proof  $(\text{rule wp-mono-on-support})$ 
    fix  $\text{out}$ 
    assume  $\text{out}: \text{out} \in \text{set-dist } (\text{execute } m \ s)$ 
    show  $(\text{case out of } \text{None} \Rightarrow \text{if } P \text{ None then } 1 \text{ else } 0$ 
       $| \text{Some } (x, t) \Rightarrow \text{wp-event } (k \ x) \ P \ t)$ 
       $\leq (\text{case out of } \text{None} \Rightarrow \text{if } Q \text{ None then } 1 \text{ else } 0$ 
       $| \text{Some } (x, t) \Rightarrow \text{wp-event } (l \ x) \ Q \ t)$ 
    proof  $(\text{cases out})$ 
      case  $\text{None}$ 
      then show  $?thesis$ 
        using  $\text{none}$  by  $(\text{cases } P \text{ None}) \text{ simp-all}$ 
      next
      case  $(\text{Some } xt)$ 
      then obtain  $x \ t$  where  $xt: xt = (x, t)$ 
      by  $(\text{cases } xt) \text{ auto}$ 
      show  $?thesis$ 
        using  $\text{cont}[\text{of } x \ t] \ \text{out } \text{Some } xt$  by  $\text{simp}$ 
    qed
  qed
qed

lemma  $\text{wp-event-assert-bind-le}$ :
  fixes  $k :: \text{unit} \Rightarrow ('x, 'f \text{ protocol-channel}) \text{ state-monad}$ 
  assumes  $\text{none}: \neg P \text{ None}$ 
  shows  $\text{wp-event } (\text{assert } b \ggg k) \ P \ s \leq \text{wp-event } (k \ ()) \ P \ s$ 
proof  $(\text{cases } b)$ 
  case  $\text{True}$ 
  then show  $?thesis$ 

```

```

    unfolding assert-def by simp
next
  case False
  then show ?thesis
    using none unfolding assert-def
    by (simp add: wp-event-def wpsimps throw-no-outcome)
qed

```

definition *staged-budget-wellformed* :: *staged-budgets* $\Rightarrow$ *bool*
where
staged-budget-wellformed budgets $\longleftrightarrow$
length (*trace-fri-budgets budgets*) = *ceil-log clength* $\wedge$
length (*composition-fri-budgets budgets*) =
ceil-log (*maxDegree* + 1) $\wedge$
length (*query-opening-budgets budgets*) = *rounds*

definition *staged-adversary-controlled*
:: *staged-budgets* $\Rightarrow$ 'f *staged-adversary* $\Rightarrow$ *bool*
where
staged-adversary-controlled budgets A $\longleftrightarrow$
controlled-ro-program (*trace-root-budget budgets*)
(*trace-root-stage A*) $\wedge$
($\forall$ *i bs.*
i < *length* (*trace-fri-budgets budgets*) $\longrightarrow$
controlled-ro-program (*trace-fri-budgets budgets ! i*)
(*trace-fri-root-stage A i bs*)) $\wedge$
($\forall$ *bs.*
controlled-ro-program (*trace-final-budget budgets*)
(*trace-final-stage A bs*)) $\wedge$
($\forall$ *as.*
controlled-ro-program (*degree-budget budgets*)
(*degree-stage A as*)) $\wedge$
($\forall$ *dg i bs.*
i < *length* (*composition-fri-budgets budgets*) $\longrightarrow$
controlled-ro-program (*composition-fri-budgets budgets ! i*)
(*composition-fri-root-stage A dg i bs*)) $\wedge$
($\forall$ *dg bs.*
controlled-ro-program (*composition-final-budget budgets*)
(*composition-final-stage A dg bs*)) $\wedge$
($\forall$ *i raw.*
i < *length* (*query-opening-budgets budgets*) $\longrightarrow$
controlled-ro-program (*query-opening-budgets budgets ! i*)
(*query-opening-stage A i raw*))

definition *staged-attacker-query-budget* :: *staged-budgets* $\Rightarrow$ *nat*
where
staged-attacker-query-budget budgets =
trace-root-budget budgets +
sum-list (*trace-fri-budgets budgets*) +

trace-final-budget budgets +
degree-budget budgets +
sum-list (composition-fri-budgets budgets) +
composition-final-budget budgets +
sum-list (query-opening-budgets budgets)

definition *staged-challenge-query-budget* :: nat
where
staged-challenge-query-budget =
ceil-log clength + length spec +
ceil-log (maxDegree + 1) + rounds

definition *staged-trace-fri-search-queries*
 :: staged-budgets $\Rightarrow$ nat $\Rightarrow$ nat
where
staged-trace-fri-search-queries budgets i =
trace-root-budget budgets +
sum-list (take (Suc i) (trace-fri-budgets budgets)) + i

definition *staged-alpha-search-queries*
 :: staged-budgets $\Rightarrow$ nat $\Rightarrow$ nat
where
staged-alpha-search-queries budgets i =
trace-root-budget budgets +
sum-list (trace-fri-budgets budgets) +
trace-final-budget budgets +
ceil-log clength + i

definition *staged-composition-fri-search-queries*
 :: staged-budgets $\Rightarrow$ nat $\Rightarrow$ nat
where
staged-composition-fri-search-queries budgets i =
trace-root-budget budgets +
sum-list (trace-fri-budgets budgets) +
trace-final-budget budgets +
degree-budget budgets +
sum-list (take (Suc i) (composition-fri-budgets budgets)) +
ceil-log clength + length spec + i

definition *staged-composition-fri-vector-search-queries*
 :: staged-budgets $\Rightarrow$ nat
where
staged-composition-fri-vector-search-queries budgets =
staged-alpha-search-queries budgets 0 +
length spec +
degree-budget budgets +
sum-list (composition-fri-budgets budgets) +
ceil-log (maxDegree + 1)

definition *staged-query-search-queries*

$:: \text{staged-budgets} \Rightarrow \text{nat} \Rightarrow \text{nat}$

where

staged-query-search-queries budgets $i =$
 $\text{trace-root-budget budgets} +$
 $\text{sum-list (trace-fri-budgets budgets)} +$
 $\text{trace-final-budget budgets} +$
 $\text{degree-budget budgets} +$
 $\text{sum-list (composition-fri-budgets budgets)} +$
 $\text{composition-final-budget budgets} +$
 $\text{sum-list (take i (query-opening-budgets budgets))} +$
 $\text{ceil-log clength} + \text{length spec} +$
 $\text{ceil-log (maxDegree} + 1) + i$

definition *staged-phase-relation-error*

$:: \text{nat} \Rightarrow \text{nat} \Rightarrow \text{prob}$

where

staged-phase-relation-error fiber-bound query-bound =
 $\text{hash-relation-budget-value fiber-bound query-bound}$

definition *staged-phase-target-error*

$:: 'f \text{ set} \Rightarrow \text{nat} \Rightarrow \text{prob}$

where

staged-phase-target-error B query-bound =
 $\text{hash-target-budget-value B query-bound}$

lemma *staged-phase-query-index-target-error-fraction-bound:*

assumes raw-bound: *query-index-raw-preimage-bound*

and subset: $B \subseteq \text{query-sample-space}$

shows

staged-phase-target-error (query-index-raw-preimage B) $q \leq$
 $\text{nnreal } q * (\text{nnreal (card B)} / \text{nnreal (card query-sample-space)})$

proof –

have raw:

$\text{nnreal (card (query-index-raw-preimage B))} / \text{nnreal size} \leq$
 $\text{nnreal (card B)} / \text{nnreal (card query-sample-space)}$

using raw-bound subset **unfolding** *query-index-raw-preimage-bound-def*

by blast

have eq:

staged-phase-target-error (query-index-raw-preimage B) $q =$
 $\text{nnreal } q *$

$(\text{nnreal (card (query-index-raw-preimage B))} / \text{nnreal size})$

unfolding *staged-phase-target-error-def* *hash-target-budget-value-def*

by transfer (simp add: field-simps)

show ?thesis

unfolding eq **by** (rule mult-left-mono) (use raw in simp-all)

qed

lemma *staged-phase-query-index-target-error-query-bound:*

```

assumes raw-bound: query-index-raw-preimage-bound
and subset:  $B \subseteq \text{query-sample-space}$ 
and frac:
   $\text{nnreal } (\text{card } B) / \text{nnreal } (\text{card } \text{query-sample-space}) \leq C$ 
shows
   $\text{staged-phase-target-error } (\text{query-index-raw-preimage } B) \ q \leq$ 
     $\text{nnreal } q * C$ 
proof –
have
   $\text{staged-phase-target-error } (\text{query-index-raw-preimage } B) \ q \leq$ 
     $\text{nnreal } q * (\text{nnreal } (\text{card } B) / \text{nnreal } (\text{card } \text{query-sample-space}))$ 
by (rule staged-phase-query-index-target-error-fraction-bound
    [OF raw-bound subset])
also have  $\dots \leq \text{nnreal } q * C$ 
by (rule mult-left-mono) (use frac in simp-all)
finally show ?thesis .
qed

```

```

definition record-staged-message
  :: 'f  $\Rightarrow$  (unit, 'f protocol-channel) state-monad
where
  record-staged-message  $x =$ 
    modify
      ( $\lambda s. s \langle$ 
         $PState := \text{concat } (PState \ s) \ x,$ 
         $PTranscript := PTranscript \ s \ @ \ [x] \rangle$ )

```

```

definition record-staged-messages
  :: 'f list  $\Rightarrow$  (unit, 'f protocol-channel) state-monad
where
  record-staged-messages  $xs = \text{mfold2 } \text{record-staged-message } xs$ 

```

```

primrec staged-trace-fri-program
  :: 'f staged-adversary  $\Rightarrow$  nat  $\Rightarrow$  nat  $\Rightarrow$  'f list  $\Rightarrow$ 
    ('f list  $\times$  'f list, 'f protocol-channel) state-monad
where
  staged-trace-fri-program  $A \ i \ 0 \ bs = \text{return } ([], bs)$ 
| staged-trace-fri-program  $A \ i \ (\text{Suc } n) \ bs =$ 
  do {
     $root \leftarrow \text{trace-fri-root-stage } A \ i \ bs;$ 
     $\text{record-staged-message } root;$ 
     $b \leftarrow \text{receive-trace-fri-challenge};$ 
     $(\text{roots}, bs') \leftarrow \text{staged-trace-fri-program } A \ (\text{Suc } i) \ n \ (bs \ @ \ [b]);$ 
     $\text{return } (root \ \# \ \text{roots}, bs')$ 
  }

```

```

primrec staged-alpha-program
  :: nat  $\Rightarrow$  ('f list, 'f protocol-channel) state-monad
where

```

```

    staged-alpha-program 0 = return []
| staged-alpha-program (Suc n) =
    do {
        a ← receive-alpha-challenge;
        record-staged-message a;
        as ← staged-alpha-program n;
        return (a # as)
    }

primrec staged-composition-fri-program
:: 'f staged-adversary ⇒ 'f ⇒ nat ⇒ nat ⇒
   'f list ⇒
   ('f list × 'f list, 'f protocol-channel) state-monad
where
    staged-composition-fri-program A dg i 0 bs = return ([], bs)
| staged-composition-fri-program A dg i (Suc n) bs =
    do {
        root ← composition-fri-root-stage A dg i bs;
        record-staged-message root;
        b ← receive-composition-fri-challenge;
        (roots, bs') ←
            staged-composition-fri-program A dg (Suc i) n (bs @ [b]);
        return (root # roots, bs')
    }

primrec staged-query-program
:: 'f staged-adversary ⇒ nat ⇒ nat ⇒
   ('f list list, 'f protocol-channel) state-monad
where
    staged-query-program A i 0 = return []
| staged-query-program A i (Suc n) =
    do {
        raw ← receive-query-index-challenge;
        chunk ← query-opening-stage A i raw;
        record-staged-messages chunk;
        chunks ← staged-query-program A (Suc i) n;
        return (chunk # chunks)
    }

primrec checked-staged-query-program
:: 'f staged-adversary ⇒ 'f list ⇒ 'f list ⇒
   nat ⇒ nat ⇒ ('f list list, 'f protocol-channel) state-monad
where
    checked-staged-query-program A trace-roots composition-roots i 0 =
        return []
| checked-staged-query-program A trace-roots composition-roots i (Suc n) =
    do {
        raw ← receive-query-index-challenge;
        let idx = index (to-nat raw);

```

```

    chunk ← query-opening-stage A i raw;
    assert
      (verifier-query-round-chunk idx trace-roots composition-roots chunk);
    record-staged-messages chunk;
    chunks ←
      checked-staged-query-program A trace-roots composition-roots
        (Suc i) n;
    return (chunk # chunks)
  }

```

definition *staged-transcript-program*

```

:: 'f staged-adversary ⇒
  ('f staged-proof-data, 'f protocol-channel) state-monad
where
  staged-transcript-program A =
    do {
      fr ← trace-root-stage A;
      record-staged-message fr;
      (trace-roots, trace-bs) ←
        staged-trace-fri-program A 0 (ceil-log clength) [];
      trace-final ← trace-final-stage A trace-bs;
      record-staged-message trace-final;
      as ← staged-alpha-program (length spec);
      dg ← degree-stage A as;
      record-staged-message dg;
      let composition-rounds = ceil-log (to-nat dg + 1);
      assert (composition-rounds ≤ ceil-log (maxDegree + 1));
      (composition-roots, composition-bs) ←
        staged-composition-fri-program A dg 0 composition-rounds [];
      composition-final ←
        composition-final-stage A dg composition-bs;
      record-staged-message composition-final;
      query-chunks ← staged-query-program A 0 rounds;
      return
        (|staged-trace-root = fr,
          staged-trace-fri-roots = trace-roots,
          staged-trace-fri-challenges = trace-bs,
          staged-trace-final = trace-final,
          staged-alphas = as,
          staged-degree = dg,
          staged-composition-fri-roots = composition-roots,
          staged-composition-fri-challenges = composition-bs,
          staged-composition-final = composition-final,
          staged-query-chunks = query-chunks|)
    }

```

definition *checked-staged-transcript-program*

```

:: 'f staged-adversary ⇒
  ('f staged-proof-data, 'f protocol-channel) state-monad

```


where

```

checked-staged-transcript-program A =
do {
  fr ← trace-root-stage A;
  record-staged-message fr;
  (trace-roots, trace-bs) ←
    staged-trace-fri-program A 0 (ceil-log clength) [];
  trace-final ← trace-final-stage A trace-bs;
  record-staged-message trace-final;
  as ← staged-alpha-program (length spec);
  dg ← degree-stage A as;
  record-staged-message dg;
  let composition-rounds = ceil-log (to-nat dg + 1);
  assert (composition-rounds ≤ ceil-log (maxDegree + 1));
  (composition-roots, composition-bs) ←
    staged-composition-fri-program A dg 0 composition-rounds [];
  composition-final ←
    composition-final-stage A dg composition-bs;
  record-staged-message composition-final;
  query-chunks ←
    checked-staged-query-program A trace-roots composition-roots
      0 rounds;
  return
    (|staged-trace-root = fr,
     staged-trace-fri-roots = trace-roots,
     staged-trace-fri-challenges = trace-bs,
     staged-trace-final = trace-final,
     staged-alphas = as,
     staged-degree = dg,
     staged-composition-fri-roots = composition-roots,
     staged-composition-fri-challenges = composition-bs,
     staged-composition-final = composition-final,
     staged-query-chunks = query-chunks|)
}

```

definition *staged-alpha-prefix-program*

```

:: 'f staged-adversary ⇒
  ('f × 'f list × 'f list × 'f, 'f protocol-channel)
  state-monad

```

where

```

staged-alpha-prefix-program A =
do {
  fr ← trace-root-stage A;
  record-staged-message fr;
  (trace-roots, trace-bs) ←
    staged-trace-fri-program A 0 (ceil-log clength) [];
  trace-final ← trace-final-stage A trace-bs;
  record-staged-message trace-final;
  return (fr, trace-roots, trace-bs, trace-final)
}

```

}

definition *staged-trace-fri-challenge-prefix-program*

$:: 'f \text{ staged-adversary} \Rightarrow \text{nat} \Rightarrow$
 $('f \times 'f \text{ list} \times 'f, 'f \text{ protocol-channel}) \text{ state-monad}$

where

staged-trace-fri-challenge-prefix-program $A \ i =$
 $\text{do } \{$
 $\text{fr} \leftarrow \text{trace-root-stage } A;$
 $\text{record-staged-message fr};$
 $(-, \text{trace-bs}) \leftarrow \text{staged-trace-fri-program } A \ 0 \ i \ [];$
 $\text{root} \leftarrow \text{trace-fri-root-stage } A \ i \ \text{trace-bs};$
 $\text{record-staged-message root};$
 $\text{return } (\text{fr}, \text{trace-bs}, \text{root})$
 $\}$

definition *staged-alpha-challenge-prefix-program*

$:: 'f \text{ staged-adversary} \Rightarrow \text{nat} \Rightarrow$
 $(('f \times 'f \text{ list} \times 'f \text{ list} \times 'f) \times 'f \text{ list},$
 $'f \text{ protocol-channel}) \text{ state-monad}$

where

staged-alpha-challenge-prefix-program $A \ i =$
 $\text{do } \{$
 $\text{prefix} \leftarrow \text{staged-alpha-prefix-program } A;$
 $\text{as-prefix} \leftarrow \text{staged-alpha-program } i;$
 $\text{return } (\text{prefix}, \text{as-prefix})$
 $\}$

definition *staged-composition-fri-challenge-prefix-program*

$:: 'f \text{ staged-adversary} \Rightarrow \text{nat} \Rightarrow$
 $('f \times 'f \text{ list} \times 'f, 'f \text{ protocol-channel}) \text{ state-monad}$

where

staged-composition-fri-challenge-prefix-program $A \ i =$
 $\text{do } \{$
 $(-, -, \text{trace-bs}, -) \leftarrow \text{staged-alpha-prefix-program } A;$
 $\text{as} \leftarrow \text{staged-alpha-program } (\text{length spec});$
 $\text{dg} \leftarrow \text{degree-stage } A \ \text{as};$
 $\text{record-staged-message dg};$
 $\text{let composition-rounds} = \text{ceil-log } (\text{to-nat dg} + 1);$
 $\text{assert } (\text{Suc } i \leq \text{composition-rounds});$
 $\text{assert } (\text{composition-rounds} \leq \text{ceil-log } (\text{maxDegree} + 1));$
 $(-, \text{composition-bs}) \leftarrow \text{staged-composition-fri-program } A \ \text{dg} \ 0 \ i \ [];$
 $\text{root} \leftarrow \text{composition-fri-root-stage } A \ \text{dg} \ i \ \text{composition-bs};$
 $\text{record-staged-message root};$
 $\text{return } (\text{dg}, \text{composition-bs}, \text{root})$
 $\}$

definition *staged-composition-fri-vector-program*

$:: 'f \text{ staged-adversary} \Rightarrow$

$(\text{'f} \times \text{'f list}, \text{'f protocol-channel}) \text{ state-monad}$
where
 $\text{staged-composition-fri-vector-program } A =$
 $\text{do } \{$
 $\quad (-, -, \text{trace-bs}, -) \leftarrow \text{staged-alpha-prefix-program } A;$
 $\quad as \leftarrow \text{staged-alpha-program } (\text{length spec});$
 $\quad dg \leftarrow \text{degree-stage } A \text{ as};$
 $\quad \text{record-staged-message } dg;$
 $\quad \text{let composition-rounds} = \text{ceil-log } (\text{to-nat } dg + 1);$
 $\quad \text{assert } (\text{composition-rounds} \leq \text{ceil-log } (\text{maxDegree} + 1));$
 $\quad (-, \text{composition-bs}) \leftarrow$
 $\quad \quad \text{staged-composition-fri-program } A \text{ dg } 0 \text{ composition-rounds } [];$
 $\quad \text{return } (dg, \text{composition-bs})$
 $\}$

definition $\text{staged-composition-fri-full-vector-program}$
 $:: \text{'f staged-adversary} \Rightarrow$
 $(\text{'f staged-composition-prefix-data}, \text{'f protocol-channel}) \text{ state-monad}$
where
 $\text{staged-composition-fri-full-vector-program } A =$
 $\text{do } \{$
 $\quad (\text{fr}, \text{trace-roots}, \text{trace-bs}, \text{trace-final}) \leftarrow$
 $\quad \quad \text{staged-alpha-prefix-program } A;$
 $\quad as \leftarrow \text{staged-alpha-program } (\text{length spec});$
 $\quad dg \leftarrow \text{degree-stage } A \text{ as};$
 $\quad \text{record-staged-message } dg;$
 $\quad \text{let composition-rounds} = \text{ceil-log } (\text{to-nat } dg + 1);$
 $\quad \text{assert } (\text{composition-rounds} \leq \text{ceil-log } (\text{maxDegree} + 1));$
 $\quad (\text{composition-roots}, \text{composition-bs}) \leftarrow$
 $\quad \quad \text{staged-composition-fri-program } A \text{ dg } 0 \text{ composition-rounds } [];$
 $\quad \text{return}$
 $\quad \quad (\text{scp-trace-root} = \text{fr},$
 $\quad \quad \text{scp-trace-fri-roots} = \text{trace-roots},$
 $\quad \quad \text{scp-trace-fri-challenges} = \text{trace-bs},$
 $\quad \quad \text{scp-trace-final} = \text{trace-final},$
 $\quad \quad \text{scp-alphas} = as,$
 $\quad \quad \text{scp-degree} = dg,$
 $\quad \quad \text{scp-composition-fri-roots} = \text{composition-roots},$
 $\quad \quad \text{scp-composition-fri-challenges} = \text{composition-bs})$
 $\}$

definition $\text{staged-query-challenge-prefix-program}$
 $:: \text{'f staged-adversary} \Rightarrow \text{nat} \Rightarrow$
 $(\text{'f} \times \text{'f} \times \text{'f list list}, \text{'f protocol-channel}) \text{ state-monad}$
where
 $\text{staged-query-challenge-prefix-program } A \text{ } i =$
 $\text{do } \{$
 $\quad (-, -, \text{trace-bs}, -) \leftarrow \text{staged-alpha-prefix-program } A;$
 $\quad as \leftarrow \text{staged-alpha-program } (\text{length spec});$

```

    dg ← degree-stage A as;
    record-staged-message dg;
    let composition-rounds = ceil-log (to-nat dg + 1);
    assert (composition-rounds ≤ ceil-log (maxDegree + 1));
    (-, composition-bs) ←
      staged-composition-fri-program A dg 0 composition-rounds [];
    composition-final ←
      composition-final-stage A dg composition-bs;
    record-staged-message composition-final;
    query-chunks ← staged-query-program A 0 i;
    return (dg, composition-final, query-chunks)
  }

```

definition *checked-staged-query-challenge-prefix-program*

:: 'f staged-adversary ⇒ nat ⇒
('f staged-query-prefix-data, 'f protocol-channel) state-monad

where

```

checked-staged-query-challenge-prefix-program A i =
do {
  (fr, trace-roots, trace-bs, trace-final) ←
    staged-alpha-prefix-program A;
  as ← staged-alpha-program (length spec);
  dg ← degree-stage A as;
  record-staged-message dg;
  let composition-rounds = ceil-log (to-nat dg + 1);
  assert (composition-rounds ≤ ceil-log (maxDegree + 1));
  (composition-roots, composition-bs) ←
    staged-composition-fri-program A dg 0 composition-rounds [];
  composition-final ←
    composition-final-stage A dg composition-bs;
  record-staged-message composition-final;
  query-chunks ←
    checked-staged-query-program A trace-roots composition-roots 0 i;
  return
    (|sqp-trace-root = fr,
     sqp-trace-fri-roots = trace-roots,
     sqp-trace-fri-challenges = trace-bs,
     sqp-trace-final = trace-final,
     sqp-alphas = as,
     sqp-degree = dg,
     sqp-composition-fri-roots = composition-roots,
     sqp-composition-fri-challenges = composition-bs,
     sqp-composition-final = composition-final,
     sqp-query-chunks = query-chunks|)
}

```

definition *staged-proof-transcript* *:: 'f staged-proof-data ⇒ 'f list*

where

staged-proof-transcript data =

```

verifier-header-messages
  (staged-trace-root data)
  (staged-trace-fri-roots data)
  (staged-trace-final data)
  (staged-alphas data)
  (staged-degree data)
  (staged-composition-fri-roots data)
  (staged-composition-final data) @
List.concat (staged-query-chunks data)

```

definition *staged-trace-fri-start-hash* :: 'f staged-proof-data $\Rightarrow$ 'f
where
staged-trace-fri-start-hash data =
 concat 0 (staged-trace-root data)

definition *staged-composition-fri-start-hash*
 :: 'f staged-proof-data $\Rightarrow$ 'f
where
staged-composition-fri-start-hash data =
 concat
 (foldl concat
 (concat
 (foldl concat
 (staged-trace-fri-start-hash data)
 (staged-trace-fri-roots data))
 (staged-trace-final data))
 (staged-alphas data))
 (staged-degree data)

definition *staged-query-start-hash* :: 'f staged-proof-data $\Rightarrow$ 'f
where
staged-query-start-hash data =
 concat
 (foldl concat
 (staged-composition-fri-start-hash data)
 (staged-composition-fri-roots data))
 (staged-composition-final data)

definition *staged-query-chunks-match-verifier-lengths*
 :: 'f staged-proof-data $\Rightarrow$ nat list $\Rightarrow$ bool
where
staged-query-chunks-match-verifier-lengths data query-idxs $\longleftrightarrow$
 length (staged-query-chunks data) = rounds $\wedge$
 ($\forall i < \text{rounds}.$
 length (staged-query-chunks data ! i) =
 verifier-query-round-transcript-length (query-idxs ! i)
 (staged-trace-fri-roots data)
 (staged-composition-fri-roots data))

lemma *query-chunks-eq-staged-if-matching-lengths*:
assumes *parser-len*: $\text{length query-chunks} = \text{rounds}$
and *concat-eq*:
 $\text{List.concat query-chunks @ trailing} =$
 $\text{List.concat (staged-query-chunks data)}$
and *parser-chunk-len*:
 $\bigwedge i. i < \text{rounds} \implies$
 $\text{length (query-chunks ! } i) =$
 $\text{verifier-query-round-transcript-length (query-idxs ! } i)$
 $(\text{staged-trace-fri-roots data})$
 $(\text{staged-composition-fri-roots data})$
and *staged-len*:
 $\text{staged-query-chunks-match-verifier-lengths data query-idxs}$
shows $\text{query-chunks} = \text{staged-query-chunks data} \wedge \text{trailing} = []$
proof –
have *staged-chunks-len*:
 $\text{length (staged-query-chunks data)} = \text{rounds}$
using *staged-len*
unfolding *staged-query-chunks-match-verifier-lengths-def* **by** *simp*
have *same-len*:
 $\text{length query-chunks} = \text{length (staged-query-chunks data)}$
using *parser-len staged-chunks-len* **by** *simp*
have *same-chunk-lens*:
 $\bigwedge i. i < \text{length query-chunks} \implies$
 $\text{length (query-chunks ! } i) =$
 $\text{length (staged-query-chunks data ! } i)$
proof –
fix *i*
assume *i-bound*: $i < \text{length query-chunks}$
then have *i-round*: $i < \text{rounds}$
using *parser-len* **by** *simp*
show
 $\text{length (query-chunks ! } i) =$
 $\text{length (staged-query-chunks data ! } i)$
using *parser-chunk-len[OF i-round] staged-len i-round*
unfolding *staged-query-chunks-match-verifier-lengths-def* **by** *simp*
qed
show *?thesis*
by (*rule concat-append-eq-concat-same-chunk-lengths*
 $[\text{OF same-len same-chunk-lens concat-eq}])$
qed

lemma *verifier-query-round-transcript-length-index-irrelevant*:
 $\text{verifier-query-round-transcript-length idx trace-roots composition-roots} =$
 $\text{verifier-query-round-transcript-length idx' trace-roots composition-roots}$
unfolding *verifier-query-round-transcript-length-def*
 $\text{query-decommitment-transcript-length-def powers-scaled-def}$
by *simp*

```

lemma staged-query-chunks-match-verifier-lengths-transfer:
  assumes match: staged-query-chunks-match-verifier-lengths data query-idxs
    and len: length query-idxs' = rounds
  shows staged-query-chunks-match-verifier-lengths data query-idxs'
proof -
  have chunks-len: length (staged-query-chunks data) = rounds
    using match unfolding staged-query-chunks-match-verifier-lengths-def
    by simp
  have chunk-lens:
     $\forall i < \text{rounds}.$ 
      length (staged-query-chunks data ! i) =
        verifier-query-round-transcript-length (query-idxs' ! i)
          (staged-trace-fri-roots data)
          (staged-composition-fri-roots data)
  proof (intro allI impI)
    fix i
    assume i-bound: i < rounds
    have old:
      length (staged-query-chunks data ! i) =
        verifier-query-round-transcript-length (query-idxs ! i)
          (staged-trace-fri-roots data)
          (staged-composition-fri-roots data)
    using match i-bound
    unfolding staged-query-chunks-match-verifier-lengths-def by simp
  show
    length (staged-query-chunks data ! i) =
      verifier-query-round-transcript-length (query-idxs' ! i)
        (staged-trace-fri-roots data)
        (staged-composition-fri-roots data)
    using old
      verifier-query-round-transcript-length-index-irrelevant
      [of query-idxs ! i
        staged-trace-fri-roots data
        staged-composition-fri-roots data query-idxs' ! i]
    by simp
  qed
show ?thesis
  unfolding staged-query-chunks-match-verifier-lengths-def
  using chunks-len chunk-lens by simp
qed

```

```

definition staged-transcript-trace-fri-vector-hit
  :: 'f list set  $\Rightarrow$ 
    ('f staged-proof-data  $\times$  'f protocol-channel) option  $\Rightarrow$  bool
where
  staged-transcript-trace-fri-vector-hit B out  $\longleftrightarrow$ 
    (case out of
      None  $\Rightarrow$  False
      | Some (data, -)  $\Rightarrow$  staged-trace-fri-challenges data  $\in$  B)

```

definition *staged-transcript-alpha-list-hit*
 $:: 'f \text{ list set} \Rightarrow$
 $('f \text{ staged-proof-data} \times 'f \text{ protocol-channel}) \text{ option} \Rightarrow \text{bool}$
where
 $\text{staged-transcript-alpha-list-hit } B \text{ out} \longleftrightarrow$
 $(\text{case out of}$
 $\quad \text{None} \Rightarrow \text{False}$
 $\quad | \text{Some } (data, -) \Rightarrow \text{staged-alphas } data \in B)$

definition *staged-transcript-composition-fri-vector-hit*
 $:: ('f \Rightarrow 'f \text{ list set}) \Rightarrow$
 $('f \text{ staged-proof-data} \times 'f \text{ protocol-channel}) \text{ option} \Rightarrow \text{bool}$
where
 $\text{staged-transcript-composition-fri-vector-hit } B \text{ out} \longleftrightarrow$
 $(\text{case out of}$
 $\quad \text{None} \Rightarrow \text{False}$
 $\quad | \text{Some } (data, -) \Rightarrow$
 $\quad \quad \text{staged-composition-fri-challenges } data \in$
 $\quad \quad B (\text{staged-degree } data))$

definition *staged-transcript-composition-fri-vector-guarded-hit*
 $:: ('f \Rightarrow 'f \text{ list set}) \Rightarrow$
 $('f \text{ staged-proof-data} \times 'f \text{ protocol-channel}) \text{ option} \Rightarrow \text{bool}$
where
 $\text{staged-transcript-composition-fri-vector-guarded-hit } B \text{ out} \longleftrightarrow$
 $(\text{case out of}$
 $\quad \text{None} \Rightarrow \text{False}$
 $\quad | \text{Some } (data, -) \Rightarrow$
 $\quad \quad \text{staged-composition-fri-challenges } data \in$
 $\quad \quad B (\text{staged-degree } data) \wedge$
 $\quad \quad 0 < \text{ceil-log } (to\text{-nat } (\text{staged-degree } data) + 1) \wedge$
 $\quad \quad \text{ceil-log } (to\text{-nat } (\text{staged-degree } data) + 1) \leq$
 $\quad \quad \text{ceil-log } (\text{maxDegree} + 1))$

definition *staged-transcript-query-index-set-hit*
 $:: \text{nat set} \Rightarrow$
 $('f \text{ staged-proof-data} \times 'f \text{ protocol-channel}) \text{ option} \Rightarrow \text{bool}$
where
 $\text{staged-transcript-query-index-set-hit } B \text{ out} \longleftrightarrow$
 $(\text{case out of}$
 $\quad \text{None} \Rightarrow \text{False}$
 $\quad | \text{Some } (data, t) \Rightarrow$
 $\quad \quad (\exists i \text{ raw. } i < \text{rounds} \wedge$
 $\quad \quad \text{fmlookup } (\text{HashMap } t)$
 $\quad \quad (\text{QueryIndexChallenge } i$
 $\quad \quad (\text{state-after-query-chunks}$
 $\quad \quad (\text{staged-query-start-hash } data)$
 $\quad \quad (\text{staged-query-chunks } data) i)) =$

$\text{Some raw} \wedge$
 $\text{index } (\text{to-nat raw}) \in B)$

definition *staged-security-with-data-trace-fri-vector-hit*

$:: 'f \text{ list set} \Rightarrow$
 $(('f \text{ staged-proof-data} \times \text{unit list}) \times 'f \text{ protocol-channel})$
 $\text{option} \Rightarrow \text{bool}$

where

$\text{staged-security-with-data-trace-fri-vector-hit } B \text{ out} \longleftrightarrow$
 $(\text{case out of}$
 $\text{None} \Rightarrow \text{False}$
 $| \text{Some } ((\text{data}, -), -) \Rightarrow \text{staged-trace-fri-challenges data} \in B)$

definition *staged-security-with-data-alpha-list-hit*

$:: 'f \text{ list set} \Rightarrow$
 $(('f \text{ staged-proof-data} \times \text{unit list}) \times 'f \text{ protocol-channel})$
 $\text{option} \Rightarrow \text{bool}$

where

$\text{staged-security-with-data-alpha-list-hit } B \text{ out} \longleftrightarrow$
 $(\text{case out of}$
 $\text{None} \Rightarrow \text{False}$
 $| \text{Some } ((\text{data}, -), -) \Rightarrow \text{staged-alphas data} \in B)$

definition *staged-security-with-data-alpha-bad-set-hit*

$:: ('f \text{ list} \Rightarrow 'f \text{ list set}) \Rightarrow$
 $(('f \text{ staged-proof-data} \times \text{unit list}) \times 'f \text{ protocol-channel})$
 $\text{option} \Rightarrow \text{bool}$

where

$\text{staged-security-with-data-alpha-bad-set-hit bad-sets out} \longleftrightarrow$
 $(\text{case out of}$
 $\text{None} \Rightarrow \text{False}$
 $| \text{Some } ((\text{data}, -), -) \Rightarrow$
 $(\exists \text{ trace-table. staged-alphas data} \in \text{bad-sets trace-table}))$

definition *staged-security-with-data-composition-fri-vector-guarded-hit*

$:: ('f \Rightarrow 'f \text{ list set}) \Rightarrow$
 $(('f \text{ staged-proof-data} \times \text{unit list}) \times 'f \text{ protocol-channel})$
 $\text{option} \Rightarrow \text{bool}$

where

$\text{staged-security-with-data-composition-fri-vector-guarded-hit } B \text{ out} \longleftrightarrow$
 $(\text{case out of}$
 $\text{None} \Rightarrow \text{False}$
 $| \text{Some } ((\text{data}, -), -) \Rightarrow$
 $\text{staged-composition-fri-challenges data} \in$
 $B (\text{staged-degree data}) \wedge$
 $0 < \text{ceil-log } (\text{to-nat } (\text{staged-degree data}) + 1) \wedge$
 $\text{ceil-log } (\text{to-nat } (\text{staged-degree data}) + 1) \leq$
 $\text{ceil-log } (\text{maxDegree} + 1))$

definition *staged-security-with-data-query-index-set-hit*

```
:: nat set =>
  (('f staged-proof-data × unit list) × 'f protocol-channel)
  option => bool
```

where

```
staged-security-with-data-query-index-set-hit B out <=>
  (case out of
    None => False
  | Some ((data, -), t) =>
    (∃ i raw. i < rounds ∧
      fmllookup (HashMap t)
        (QueryIndexChallenge i
          (state-after-query-chunks
            (staged-query-start-hash data)
            (staged-query-chunks data) i)) =
        Some raw ∧
      index (to-nat raw) ∈ B))
```

definition *staged-security-with-data-query-bad-hit*

```
:: (('f staged-proof-data × unit list) × 'f protocol-channel)
  option => bool
```

where

```
staged-security-with-data-query-bad-hit out <=>
  (case out of
    None => False
  | Some ((-, result), final-state) =>
    (∃ s.
      Some (result, final-state) ∈ set-dist (execute verify-monad s) ∧
      query-bad s (Some (result, final-state))))
```

definition *staged-security-with-data-query-sampling-hit*

```
:: (('f staged-proof-data × unit list) × 'f protocol-channel)
  option => bool
```

where

```
staged-security-with-data-query-sampling-hit out <=>
  (case out of
    None => False
  | Some ((-, result), final-state) =>
    (∃ s trace-table composition-table as.
      query-rounds-any-index-set-hit s
        (query-sampling-success-space trace-table composition-table as)
        rounds
      (Some (result, final-state))))
```

definition *staged-security-with-data-query-header-sampling-hit*

```
:: (('f staged-proof-data × unit list) × 'f protocol-channel)
  option => bool
```

where

```
staged-security-with-data-query-header-sampling-hit out <=>
```

(case out of
 None $\Rightarrow$ False
 | Some ((-, result), final-state) $\Rightarrow$
 ($\exists$ s.
 query-header-rounds-any-index-set-hit s
 (query-header-supported-union-good-sets s
 query-sampling-success-space)
 (Some (result, final-state))))

definition staged-security-with-data-state-query-bad-hit
 :: (((('f staged-proof-data $\times$ 'f protocol-channel) $\times$ unit list) $\times$
 'f protocol-channel) option $\Rightarrow$ bool

where

staged-security-with-data-state-query-bad-hit out $\longleftrightarrow$
 (case out of
 None $\Rightarrow$ False
 | Some (((data, attacker-state), result), final-state) $\Rightarrow$
 (let s =
 verifier-state-from-adversary attacker-state
 (staged-proof-transcript data)
 in
 Some (result, final-state) $\in$ set-dist (execute verify-monad s) $\wedge$
 query-bad s (Some (result, final-state))))

definition staged-security-with-data-state-query-header-sampling-hit
 :: (((('f staged-proof-data $\times$ 'f protocol-channel) $\times$ unit list) $\times$
 'f protocol-channel) option $\Rightarrow$ bool

where

staged-security-with-data-state-query-header-sampling-hit out $\longleftrightarrow$
 (case out of
 None $\Rightarrow$ False
 | Some (((data, attacker-state), result), final-state) $\Rightarrow$
 (let s =
 verifier-state-from-adversary attacker-state
 (staged-proof-transcript data)
 in
 query-header-rounds-any-index-set-hit s
 (query-header-supported-union-good-sets s
 query-sampling-success-space)
 (Some (result, final-state))))

definition staged-security-with-data-state-query-header-key-hit
 :: (((('f staged-proof-data $\times$ 'f protocol-channel) $\times$ unit list) $\times$
 'f protocol-channel) option $\Rightarrow$ bool

where

staged-security-with-data-state-query-header-key-hit out $\longleftrightarrow$
 (case out of
 None $\Rightarrow$ False
 | Some (((data, attacker-state), -), final-state) $\Rightarrow$

```

(let s =
  verifier-state-from-adversary attacker-state
  (staged-proof-transcript data);
  B =
  query-header-supported-union-good-sets s
  query-sampling-success-space
  (staged-trace-root data)
  (staged-trace-fri-roots data)
  (staged-trace-final data)
  (staged-alphas data)
  (staged-degree data)
  (staged-composition-fri-roots data)
  (staged-composition-final data)
  in
  (∃ i raw. i < rounds ∧
    fmlookup (HashMap final-state)
      (QueryIndexChallenge i
        (state-after-query-chunks
          (staged-query-start-hash data)
          (staged-query-chunks data) i)) =
        Some raw ∧
        index (to-nat raw) ∈ B)))

```

definition *staged-security-with-data-state-verifier-event*

```

:: ('f protocol-channel ⇒
  (unit list × 'f protocol-channel) option ⇒ bool) ⇒
  (((('f staged-proof-data × 'f protocol-channel) × unit list) ×
    'f protocol-channel) option ⇒ bool

```

where

```

staged-security-with-data-state-verifier-event P out ⟷
  (case out of
    None ⇒ False
  | Some (((data, attacker-state), result), final-state) ⇒
    P
    (verifier-state-from-adversary attacker-state
      (staged-proof-transcript data))
    (Some (result, final-state)))

```

definition *staged-security-with-data-state-transcript-pre-hit*

```

:: (((('f staged-proof-data × 'f protocol-channel) × unit list) ×
  'f protocol-channel) option ⇒ bool

```

where

```

staged-security-with-data-state-transcript-pre-hit out ⟷
  (case out of
    None ⇒ False
  | Some (((data, attacker-state), -), -) ⇒
    hash-map-output-values
      (verifier-state-from-adversary attacker-state
        (staged-proof-transcript data)) ∩

```

$\text{set } (\text{staged-proof-transcript data}) \neq \{\}$

definition *staged-security-with-data-state-transcript-new-hit*
 $:: ((('f \text{ staged-proof-data} \times 'f \text{ protocol-channel}) \times \text{unit list}) \times 'f \text{ protocol-channel}) \text{ option} \Rightarrow \text{bool}$

where

$\text{staged-security-with-data-state-transcript-new-hit out} \longleftrightarrow$
 (case out of
 None $\Rightarrow$ False
 | Some (((data, attacker-state), -), final-state) $\Rightarrow$
 hash-map-new-output-hit (set (staged-proof-transcript data))
 (verifier-state-from-adversary attacker-state
 (staged-proof-transcript data))
 final-state)

definition *staged-composition-prefix-fri-vector-hit*

$:: ('f \Rightarrow 'f \text{ list set}) \Rightarrow$
 $('f \text{ staged-composition-prefix-data} \times 'f \text{ protocol-channel}) \text{ option} \Rightarrow \text{bool}$

where

$\text{staged-composition-prefix-fri-vector-hit B out} \longleftrightarrow$
 (case out of
 None $\Rightarrow$ False
 | Some (prefix, -) $\Rightarrow$
 scp-composition-fri-challenges prefix $\in$
 B (scp-degree prefix) $\wedge$
 $0 < \text{ceil-log } (\text{to-nat } (\text{scp-degree prefix}) + 1) \wedge$
 $\text{ceil-log } (\text{to-nat } (\text{scp-degree prefix}) + 1) \leq$
 $\text{ceil-log } (\text{maxDegree} + 1)$)

definition *staged-after-alpha-prefix-program*

$:: 'f \text{ staged-adversary} \Rightarrow$
 $'f \times 'f \text{ list} \times 'f \text{ list} \times 'f \Rightarrow$
 $('f \text{ staged-proof-data}, 'f \text{ protocol-channel}) \text{ state-monad}$

where

$\text{staged-after-alpha-prefix-program A prefix} =$
 (case prefix of (fr, trace-roots, trace-bs, trace-final) $\Rightarrow$
 do {
 as $\leftarrow$ staged-alpha-program (length spec);
 dg $\leftarrow$ degree-stage A as;
 record-staged-message dg;
 let composition-rounds = $\text{ceil-log } (\text{to-nat dg} + 1)$;
 assert (composition-rounds $\leq$ $\text{ceil-log } (\text{maxDegree} + 1)$);
 (composition-roots, composition-bs) $\leftarrow$
 staged-composition-fri-program A dg 0 composition-rounds [];
 composition-final $\leftarrow$
 composition-final-stage A dg composition-bs;
 record-staged-message composition-final;
 query-chunks $\leftarrow$ staged-query-program A 0 rounds;

```

return
  (|staged-trace-root = fr,
   staged-trace-fri-roots = trace-roots,
   staged-trace-fri-challenges = trace-bs,
   staged-trace-final = trace-final,
   staged-alphas = as,
   staged-degree = dg,
   staged-composition-fri-roots = composition-roots,
   staged-composition-fri-challenges = composition-bs,
   staged-composition-final = composition-final,
   staged-query-chunks = query-chunks|)
})

```

definition *staged-after-composition-fri-full-vector-program*
 $:: 'f \text{ staged-adversary} \Rightarrow 'f \text{ staged-composition-prefix-data} \Rightarrow$
 $('f \text{ staged-proof-data}, 'f \text{ protocol-channel}) \text{ state-monad}$
where

```

staged-after-composition-fri-full-vector-program A prefix =
do {
  composition-final ←
    composition-final-stage A
    (scp-degree prefix)
    (scp-composition-fri-challenges prefix);
  record-staged-message composition-final;
  query-chunks ← staged-query-program A 0 rounds;
  return
    (|staged-trace-root = scp-trace-root prefix,
     staged-trace-fri-roots = scp-trace-fri-roots prefix,
     staged-trace-fri-challenges = scp-trace-fri-challenges prefix,
     staged-trace-final = scp-trace-final prefix,
     staged-alphas = scp-alphas prefix,
     staged-degree = scp-degree prefix,
     staged-composition-fri-roots =
       scp-composition-fri-roots prefix,
     staged-composition-fri-challenges =
       scp-composition-fri-challenges prefix,
     staged-composition-final = composition-final,
     staged-query-chunks = query-chunks|)
}

```

lemma *staged-transcript-program-alpha-prefix-decomp:*

```

staged-transcript-program A =
  staged-alpha-prefix-program A  $\gg$ 
  staged-after-alpha-prefix-program A

```

unfolding *staged-transcript-program-def staged-alpha-prefix-program-def*
staged-after-alpha-prefix-program-def *Let-def*

by (*simp add: sm-bind-assoc split-def split: prod.splits*)

lemma *staged-transcript-program-composition-fri-full-vector-decomp:*

```

staged-transcript-program A =
  staged-composition-fri-full-vector-program A  $\gg$ 
  staged-after-composition-fri-full-vector-program A
unfolding staged-transcript-program-def
  staged-composition-fri-full-vector-program-def
  staged-after-composition-fri-full-vector-program-def
  staged-alpha-prefix-program-def Let-def
by (simp add: sm-bind-assoc split-def split: prod.splits)

lemma staged-composition-fri-vector-program-projection:
  staged-composition-fri-vector-program A =
  staged-composition-fri-full-vector-program A  $\gg$ 
  ( $\lambda$ prefix. return
    (scp-degree prefix, scp-composition-fri-challenges prefix))
unfolding staged-composition-fri-vector-program-def
  staged-composition-fri-full-vector-program-def
  staged-alpha-prefix-program-def Let-def
by (simp add: sm-bind-assoc split-def split: prod.splits)

lemma staged-after-alpha-prefix-program-trace-fri-vector-hitD:
assumes outcome:
  out  $\in$ 
    set-dist
    (execute
      (staged-after-alpha-prefix-program A
        (fr, trace-roots, trace-bs, trace-final)) s)
  and hit: staged-transcript-trace-fri-vector-hit B out
shows trace-bs  $\in$  B
proof (cases out)
case None
then show ?thesis
  using hit unfolding staged-transcript-trace-fri-vector-hit-def by simp
next
case (Some result)
then obtain data u where out-eq: out = Some (data, u)
  by (cases result) simp
from outcome[unfolded out-eq]
obtain as s1 dg s2 s3 composition-roots composition-bs s4
  composition-final s5 s6 query-chunks where
  ret:
    Some (data, u)  $\in$ 
      set-dist
      (execute
        (return
          ( $\lambda$ staged-trace-root = fr,
            staged-trace-fri-roots = trace-roots,
            staged-trace-fri-challenges = trace-bs,
            staged-trace-final = trace-final,
            staged-alphas = as,

```

```

    staged-degree = dg,
    staged-composition-fri-roots = composition-roots,
    staged-composition-fri-challenges = composition-bs,
    staged-composition-final = composition-final,
    staged-query-chunks = query-chunks))
s6)
unfolding staged-after-alpha-prefix-program-def Let-def
by (auto elim!: set-dist-bindE split: prod.splits)
have data-eq:
  data =
    (|staged-trace-root = fr,
     staged-trace-fri-roots = trace-roots,
     staged-trace-fri-challenges = trace-bs,
     staged-trace-final = trace-final,
     staged-alphas = as,
     staged-degree = dg,
     staged-composition-fri-roots = composition-roots,
     staged-composition-fri-challenges = composition-bs,
     staged-composition-final = composition-final,
     staged-query-chunks = query-chunks|)
  using ret by simp
show ?thesis
  using hit unfolding out-eq data-eq
    staged-transcript-trace-fri-vector-hit-def
  by simp
qed

lemma staged-after-alpha-prefix-program-alpha-list-hitD:
assumes outcome:
  out ∈
    set-dist
      (execute
        (staged-after-alpha-prefix-program A
          (fr, trace-roots, trace-bs, trace-final)) s)
and hit: staged-transcript-alpha-list-hit B out
shows
  ∃ as s1.
    Some (as, s1) ∈
      set-dist (execute (staged-alpha-program (length spec)) s) ∧
    as ∈ B
proof (cases out)
case None
then show ?thesis
  using hit unfolding staged-transcript-alpha-list-hit-def by simp
next
case (Some result)
then obtain data u where out-eq: out = Some (data, u)
  by (cases result) simp
from outcome[unfolded out-eq]

```



```

obtain as s1 dg s2 s3 composition-roots composition-bs s4
  composition-final s5 s6 query-chunks where
  alpha-out:
    Some (as, s1) ∈
      set-dist (execute (staged-alpha-program (length spec)) s)
and ret:
    Some (data, u) ∈
      set-dist
        (execute
          (return
            (staged-trace-root = fr,
             staged-trace-fri-roots = trace-roots,
             staged-trace-fri-challenges = trace-bs,
             staged-trace-final = trace-final,
             staged-alphas = as,
             staged-degree = dg,
             staged-composition-fri-roots = composition-roots,
             staged-composition-fri-challenges = composition-bs,
             staged-composition-final = composition-final,
             staged-query-chunks = query-chunks))
          s6)
    unfolding staged-after-alpha-prefix-program-def Let-def
    by (auto elim!: set-dist-bindE split: prod.splits)
have data-eq:
  data =
    (staged-trace-root = fr,
     staged-trace-fri-roots = trace-roots,
     staged-trace-fri-challenges = trace-bs,
     staged-trace-final = trace-final,
     staged-alphas = as,
     staged-degree = dg,
     staged-composition-fri-roots = composition-roots,
     staged-composition-fri-challenges = composition-bs,
     staged-composition-final = composition-final,
     staged-query-chunks = query-chunks)
    using ret by simp
have as-in: as ∈ B
    using hit unfolding out-eq data-eq
      staged-transcript-alpha-list-hit-def
    by simp
show ?thesis
    using alpha-out as-in by blast
qed

lemma staged-after-alpha-prefix-program-alpha-list-bound-from-alpha-program:
assumes alpha-bound:
  wp-event (staged-alpha-program (length spec))
  (λout. case out of None ⇒ False
    | Some (as, -) ⇒ as ∈ B) s ≤ C

```

shows

wp-event

(*staged-after-alpha-prefix-program* *A*
 (*fr*, *trace-roots*, *trace-bs*, *trace-final*))
 (*staged-transcript-alpha-list-hit* *B*) $s \leq C$

proof –

have *decomp*:

staged-after-alpha-prefix-program *A*
 (*fr*, *trace-roots*, *trace-bs*, *trace-final*) =
 (*staged-alpha-program* (*length spec*) $\gg$
 (λ *as*. *degree-stage* *A as* $\gg$
 (λ *dg*. *record-staged-message* *dg* $\gg$
 (λ -. *let composition-rounds* =
 ceil-log (*to-nat dg* + 1)
 in assert
 (*composition-rounds* $\leq$ *ceil-log* (*maxDegree* + 1)) $\gg$
 (λ -. *staged-composition-fri-program* *A dg 0*
 composition-rounds [] $\gg$
 (λ (*composition-roots*, *composition-bs*).
 composition-final-stage *A dg composition-bs* $\gg$
 (λ *composition-final*.
 record-staged-message composition-final $\gg$
 (λ -. *staged-query-program* *A 0 rounds* $\gg$
 (λ *query-chunks*.
 return
 (λ (*staged-trace-root* = *fr*,
 staged-trace-fri-roots = *trace-roots*,
 staged-trace-fri-challenges = *trace-bs*,
 staged-trace-final = *trace-final*,
 staged-alphas = *as*,
 staged-degree = *dg*,
 staged-composition-fri-roots =
 composition-roots,
 staged-composition-fri-challenges =
 composition-bs,
 staged-composition-final =
 composition-final,
 staged-query-chunks =
 query-chunks))))))))))))))))

unfolding *staged-after-alpha-prefix-program-def* *Let-def*

by *simp*

show *?thesis*

unfolding *decomp*

proof (*rule wp-event-bind-bound-by-head-event*[*OF alpha-bound*])

show *staged-transcript-alpha-list-hit* *B* *None* $\implies$

(*case* *None* of *None* $\implies$ *False* | *Some* (*as*, -) $\implies$ *as* $\in$ *B*)

unfolding *staged-transcript-alpha-list-hit-def* **by** *simp*

next

fix *as s1 out*

```

assume alpha-out:
  Some (as, s1) ∈
    set-dist (execute (staged-alpha-program (length spec)) s)
and cont:
  out ∈
    set-dist
      (execute
        (degree-stage A as >>=
          (λdg. record-staged-message dg >>=
            (λ-. let composition-rounds =
              ceil-log (to-nat dg + 1)
            in assert
              (composition-rounds ≤ ceil-log (maxDegree + 1)) >>=
                (λ-. staged-composition-fri-program A dg 0
                  composition-rounds [] >>=
                    (λ(composition-roots, composition-bs).
                      composition-final-stage A dg composition-bs >>=
                        (λcomposition-final.
                          record-staged-message composition-final >>=
                            (λ-. staged-query-program A 0 rounds >>=
                              (λquery-chunks.
                                return
                                  (λ(staged-trace-root = fr,
                                    staged-trace-fri-roots = trace-roots,
                                    staged-trace-fri-challenges = trace-bs,
                                    staged-trace-final = trace-final,
                                    staged-alphas = as,
                                    staged-degree = dg,
                                    staged-composition-fri-roots =
                                      composition-roots,
                                    staged-composition-fri-challenges =
                                      composition-bs,
                                    staged-composition-final =
                                      composition-final,
                                    staged-query-chunks =
                                      query-chunks))))))))))
                                  s1)
                                )
                              )
                            )
                          )
                        )
                      )
                    )
                  )
                )
              )
            )
          )
        )
      )
    and hit: staged-transcript-alpha-list-hit B out
show (case Some (as, s1) of None ⇒ False
  | Some (as, -) ⇒ as ∈ B)
proof (cases out)
  case None
  then show ?thesis
    using hit unfolding staged-transcript-alpha-list-hit-def by simp
next
  case (Some result)
  then obtain data u where out-eq: out = Some (data, u)
    by (cases result) simp
  from cont[unfolded out-eq]

```

```

obtain dg s2 s3 composition-roots composition-bs s4
  composition-final s5 s6 query-chunks where
  ret:
    Some (data, u) ∈
      set-dist
      (execute
      (return
        (|staged-trace-root = fr,
          staged-trace-fri-roots = trace-roots,
          staged-trace-fri-challenges = trace-bs,
          staged-trace-final = trace-final,
          staged-alphas = as,
          staged-degree = dg,
          staged-composition-fri-roots = composition-roots,
          staged-composition-fri-challenges = composition-bs,
          staged-composition-final = composition-final,
          staged-query-chunks = query-chunks|))
      s6)
  by (auto simp: Let-def assert-def
    elim!: set-dist-bindE split: prod.splits)
have data-eq:
  data =
    (|staged-trace-root = fr,
      staged-trace-fri-roots = trace-roots,
      staged-trace-fri-challenges = trace-bs,
      staged-trace-final = trace-final,
      staged-alphas = as,
      staged-degree = dg,
      staged-composition-fri-roots = composition-roots,
      staged-composition-fri-challenges = composition-bs,
      staged-composition-final = composition-final,
      staged-query-chunks = query-chunks|)
  using ret by simp
show ?thesis
  using hit unfolding out-eq data-eq
    staged-transcript-alpha-list-hit-def by simp
qed
qed
qed

lemma staged-after-composition-fri-full-vector-program-hitD:
assumes outcome:
  out ∈
    set-dist
    (execute
      (staged-after-composition-fri-full-vector-program A prefix) s)
and hit:
  staged-transcript-composition-fri-vector-guarded-hit B out
shows staged-composition-prefix-fri-vector-hit B (Some (prefix, s))

```

```

proof (cases out)
  case None
  then show ?thesis
    using hit
    unfolding staged-transcript-composition-fri-vector-guarded-hit-def
    by simp
next
  case (Some result)
  then obtain data u where out-eq: out = Some (data, u)
    by (cases result) simp
  from outcome[unfolded out-eq]
  obtain composition-final s1 s2 query-chunks where
    ret:
      Some (data, u) ∈
        set-dist
          (execute
            (return
              (staged-trace-root = scp-trace-root prefix,
               staged-trace-fri-roots = scp-trace-fri-roots prefix,
               staged-trace-fri-challenges =
                 scp-trace-fri-challenges prefix,
               staged-trace-final = scp-trace-final prefix,
               staged-alphas = scp-alphas prefix,
               staged-degree = scp-degree prefix,
               staged-composition-fri-roots =
                 scp-composition-fri-roots prefix,
               staged-composition-fri-challenges =
                 scp-composition-fri-challenges prefix,
               staged-composition-final = composition-final,
               staged-query-chunks = query-chunks))
            s2)
          unfolding staged-after-composition-fri-full-vector-program-def
          by (auto elim!: set-dist-bindE split: prod.splits)
  have data-eq:
    data =
      (staged-trace-root = scp-trace-root prefix,
       staged-trace-fri-roots = scp-trace-fri-roots prefix,
       staged-trace-fri-challenges = scp-trace-fri-challenges prefix,
       staged-trace-final = scp-trace-final prefix,
       staged-alphas = scp-alphas prefix,
       staged-degree = scp-degree prefix,
       staged-composition-fri-roots =
         scp-composition-fri-roots prefix,
       staged-composition-fri-challenges =
         scp-composition-fri-challenges prefix,
       staged-composition-final = composition-final,
       staged-query-chunks = query-chunks)
    using ret by simp
  show ?thesis

```

```

using hit
unfolding out-eq data-eq
  staged-transcript-composition-fri-vector-guarded-hit-def
  staged-composition-prefix-fri-vector-hit-def
by simp
qed

lemma staged-proof-transcript-verifier-header-transcript:
  assumes trace-len:
    length (staged-trace-fri-roots data) = ceil-log clength
  and alpha-len:
    length (staged-alphas data) = length spec
  and composition-len:
    length (staged-composition-fri-roots data) =
      ceil-log (to-nat (staged-degree data) + 1)
  shows
    verifier-header-transcript
      (verifier-state-from-adversary s (staged-proof-transcript data))
      (staged-trace-root data)
      (staged-trace-fri-roots data)
      (staged-trace-final data)
      (staged-alphas data)
      (staged-degree data)
      (staged-composition-fri-roots data)
      (staged-composition-final data)
      (List.concat (staged-query-chunks data))
  using trace-len alpha-len composition-len
  unfolding verifier-header-transcript-def staged-proof-transcript-def
  by simp

```

```

definition staged-security-experiment
  :: 'f staged-adversary  $\Rightarrow$ 
    (unit list, 'f protocol-channel) state-monad
  where
    staged-security-experiment A =
      staged-transcript-program A  $\gg=$  ( $\lambda$ data.
        get  $\gg=$  ( $\lambda$ s.
          put
            (verifier-state-from-adversary s
              (staged-proof-transcript data))  $\gg=$ 
            ( $\lambda$ -. verify-monad)))

definition staged-security-experiment-with-data
  :: 'f staged-adversary  $\Rightarrow$ 
    ('f staged-proof-data  $\times$  unit list, 'f protocol-channel) state-monad
  where
    staged-security-experiment-with-data A =
      staged-transcript-program A  $\gg=$  ( $\lambda$ data.
        get  $\gg=$  ( $\lambda$ s.

```

$$\begin{aligned} & \text{put} \\ & \quad (\text{verifier-state-from-adversary } s \\ & \quad \quad (\text{staged-proof-transcript data})) \gg= \\ & \quad (\lambda-. \text{verify-monad} \gg= (\lambda \text{result}. \text{return } (\text{data}, \text{result})))) \end{aligned}$$

definition *staged-security-experiment-with-data-state*

$$\begin{aligned} & :: 'f \text{ staged-adversary} \Rightarrow \\ & \quad (('f \text{ staged-proof-data} \times 'f \text{ protocol-channel}) \times \text{unit list}, \\ & \quad 'f \text{ protocol-channel}) \text{ state-monad} \end{aligned}$$

where

$$\begin{aligned} & \text{staged-security-experiment-with-data-state } A = \\ & \quad \text{staged-transcript-program } A \gg= (\lambda \text{data}. \\ & \quad \text{get} \gg= (\lambda s. \\ & \quad \quad \text{put} \\ & \quad \quad \quad (\text{verifier-state-from-adversary } s \\ & \quad \quad \quad \quad (\text{staged-proof-transcript data})) \gg= \\ & \quad \quad (\lambda-. \\ & \quad \quad \quad \text{verify-monad} \gg= \\ & \quad \quad \quad \quad (\lambda \text{result}. \text{return } ((\text{data}, s), \text{result})))) \end{aligned}$$

definition *checked-staged-security-experiment*

$$\begin{aligned} & :: 'f \text{ staged-adversary} \Rightarrow \\ & \quad (\text{unit list}, 'f \text{ protocol-channel}) \text{ state-monad} \end{aligned}$$

where

$$\begin{aligned} & \text{checked-staged-security-experiment } A = \\ & \quad \text{checked-staged-transcript-program } A \gg= (\lambda \text{data}. \\ & \quad \text{get} \gg= (\lambda s. \\ & \quad \quad \text{put} \\ & \quad \quad \quad (\text{verifier-state-from-adversary } s \\ & \quad \quad \quad \quad (\text{staged-proof-transcript data})) \gg= \\ & \quad \quad (\lambda-. \text{verify-monad}))) \end{aligned}$$

definition *checked-staged-security-experiment-with-data*

$$\begin{aligned} & :: 'f \text{ staged-adversary} \Rightarrow \\ & \quad (('f \text{ staged-proof-data} \times \text{unit list}, 'f \text{ protocol-channel}) \text{ state-monad} \end{aligned}$$

where

$$\begin{aligned} & \text{checked-staged-security-experiment-with-data } A = \\ & \quad \text{checked-staged-transcript-program } A \gg= (\lambda \text{data}. \\ & \quad \text{get} \gg= (\lambda s. \\ & \quad \quad \text{put} \\ & \quad \quad \quad (\text{verifier-state-from-adversary } s \\ & \quad \quad \quad \quad (\text{staged-proof-transcript data})) \gg= \\ & \quad \quad (\lambda-. \text{verify-monad} \gg= (\lambda \text{result}. \text{return } (\text{data}, \text{result})))) \end{aligned}$$

definition *checked-staged-security-experiment-with-data-state*

$$\begin{aligned} & :: 'f \text{ staged-adversary} \Rightarrow \\ & \quad (('f \text{ staged-proof-data} \times 'f \text{ protocol-channel}) \times \text{unit list}, \\ & \quad 'f \text{ protocol-channel}) \text{ state-monad} \end{aligned}$$

where

```

checked-staged-security-experiment-with-data-state A =
  checked-staged-transcript-program A >>= (λdata.
    get >>= (λs.
      put
        (verifier-state-from-adversary s
          (staged-proof-transcript data)) >>=
        (λ-.
          verify-monad >>=
            (λresult. return ((data, s), result))))))

```

lemma *staged-security-experiment-projection:*
staged-security-experiment-with-data A >>=
(λx. return (snd x)) =
staged-security-experiment A
unfolding *staged-security-experiment-with-data-def*
staged-security-experiment-def
by (*simp add: sm-bind-assoc split: prod.splits*)

lemma *staged-security-experiment-with-data-state-projection:*
staged-security-experiment-with-data-state A >>=
(λx. return (snd x)) =
staged-security-experiment A
unfolding *staged-security-experiment-with-data-state-def*
staged-security-experiment-def
by (*simp add: sm-bind-assoc split: prod.splits*)

lemma *staged-security-experiment-with-data-state-projection-to-data:*
staged-security-experiment-with-data-state A >>=
(λx. case x of ((data, -), result) =>
 return (data, result)) =
staged-security-experiment-with-data A
unfolding *staged-security-experiment-with-data-state-def*
staged-security-experiment-with-data-def
by (*simp add: sm-bind-assoc split: prod.splits*)

lemma *checked-staged-security-experiment-projection:*
checked-staged-security-experiment-with-data A >>=
(λx. return (snd x)) =
checked-staged-security-experiment A
unfolding *checked-staged-security-experiment-with-data-def*
checked-staged-security-experiment-def
by (*simp add: sm-bind-assoc split: prod.splits*)

lemma *checked-staged-security-experiment-with-data-state-projection:*
checked-staged-security-experiment-with-data-state A >>=
(λx. return (snd x)) =
checked-staged-security-experiment A
unfolding *checked-staged-security-experiment-with-data-state-def*
checked-staged-security-experiment-def

by (*simp add: sm-bind-assoc split: prod.splits*)

lemma *checked-staged-security-experiment-with-data-state-projection-to-data:*

checked-staged-security-experiment-with-data-state A $\gg=$

($\lambda x.$ *case x of ((data, -), result)* $\Rightarrow$

return (data, result)) =

checked-staged-security-experiment-with-data A

unfolding *checked-staged-security-experiment-with-data-state-def*

checked-staged-security-experiment-with-data-def

by (*simp add: sm-bind-assoc split: prod.splits*)

lemma *staged-security-experiment-acceptance-with-data:*

wp-event (staged-security-experiment A) accepted adversary-initial-state =

wp-event (staged-security-experiment-with-data A) accepted

adversary-initial-state

proof –

have *proj:*

staged-security-experiment A =

staged-security-experiment-with-data A $\gg=$

($\lambda x.$ *return (snd x)*)

using *staged-security-experiment-projection* **by** *simp*

show *?thesis*

unfolding *proj*

by (*subst wp-event-bind-return-map[where f=snd]*)

(*unfold wp-event-def accepted-def,*

rule arg-cong[where

f= $\lambda Q.$ wp (staged-security-experiment-with-data A) Q

adversary-initial-state],

rule ext, simp split: option.splits prod.splits)

qed

lemma *staged-security-experiment-acceptance-with-data-state:*

wp-event (staged-security-experiment A) accepted adversary-initial-state =

wp-event (staged-security-experiment-with-data-state A) accepted

adversary-initial-state

proof –

have *proj:*

staged-security-experiment A =

staged-security-experiment-with-data-state A $\gg=$

($\lambda x.$ *return (snd x)*)

using *staged-security-experiment-with-data-state-projection* **by** *simp*

show *?thesis*

unfolding *proj*

by (*subst wp-event-bind-return-map[where f=snd]*)

(*unfold wp-event-def accepted-def,*

rule arg-cong[where

f= $\lambda Q.$ wp (staged-security-experiment-with-data-state A) Q

adversary-initial-state],

rule ext, simp split: option.splits prod.splits)

qed

lemma *checked-staged-security-experiment-acceptance-with-data:*

wp-event (checked-staged-security-experiment A) accepted
adversary-initial-state =
wp-event (checked-staged-security-experiment-with-data A) accepted
adversary-initial-state

proof –

have *proj*:

checked-staged-security-experiment A =
checked-staged-security-experiment-with-data A $\gg$
(λx . return (snd x))

using *checked-staged-security-experiment-projection* **by** *simp*

show *?thesis*

unfolding *proj*

by (*subst wp-event-bind-return-map[where f=snd]*)
(*unfold wp-event-def accepted-def*,
rule arg-cong[where
f= λQ . wp (checked-staged-security-experiment-with-data A) Q
adversary-initial-state],
rule ext, simp split: option.splits prod.splits)

qed

lemma *checked-staged-security-experiment-acceptance-with-data-state:*

wp-event (checked-staged-security-experiment A) accepted
adversary-initial-state =
wp-event (checked-staged-security-experiment-with-data-state A) accepted
adversary-initial-state

proof –

have *proj*:

checked-staged-security-experiment A =
checked-staged-security-experiment-with-data-state A $\gg$
(λx . return (snd x))

using *checked-staged-security-experiment-with-data-state-projection*

by *simp*

show *?thesis*

unfolding *proj*

by (*subst wp-event-bind-return-map[where f=snd]*)
(*unfold wp-event-def accepted-def*,
rule arg-cong[where
f= λQ . wp (checked-staged-security-experiment-with-data-state A) Q
adversary-initial-state],
rule ext, simp split: option.splits prod.splits)

qed

lemma *staged-security-experiment-with-data-event-from-data-state:*

wp-event (staged-security-experiment-with-data A) P adversary-initial-state =
wp-event (staged-security-experiment-with-data-state A)
(λout . case out of None $\Rightarrow$ P None

```

      | Some (((data, -), result), t)  $\Rightarrow$  P (Some ((data, result), t)))
    adversary-initial-state
  proof -
    have proj:
      staged-security-experiment-with-data A =
        staged-security-experiment-with-data-state A  $\ggg$ 
        ( $\lambda x$ . case x of ((data, -), result)  $\Rightarrow$ 
          return (data, result))
    using staged-security-experiment-with-data-state-projection-to-data
    by simp
  show ?thesis
    unfolding proj wp-event-def
    apply (subst wp-bind)
    apply (rule arg-cong[where
      f= $\lambda Q$ . wp (staged-security-experiment-with-data-state A) Q
      adversary-initial-state])
    apply (rule ext)
    apply (simp add: wp-return split: option.splits prod.splits)
    done
qed

lemma checked-staged-security-experiment-with-data-event-from-data-state:
  wp-event (checked-staged-security-experiment-with-data A) P
  adversary-initial-state =
  wp-event (checked-staged-security-experiment-with-data-state A)
  ( $\lambda out$ . case out of None  $\Rightarrow$  P None
    | Some (((data, -), result), t)  $\Rightarrow$  P (Some ((data, result), t)))
  adversary-initial-state
  proof -
    have proj:
      checked-staged-security-experiment-with-data A =
        checked-staged-security-experiment-with-data-state A  $\ggg$ 
        ( $\lambda x$ . case x of ((data, -), result)  $\Rightarrow$ 
          return (data, result))
    using checked-staged-security-experiment-with-data-state-projection-to-data
    by simp
  show ?thesis
    unfolding proj wp-event-def
    apply (subst wp-bind)
    apply (rule arg-cong[where
      f= $\lambda Q$ . wp (checked-staged-security-experiment-with-data-state A) Q
      adversary-initial-state])
    apply (rule ext)
    apply (simp add: wp-return split: option.splits prod.splits)
    done
qed

lemma staged-security-experiment-with-data-state-outcomeE:
  assumes

```

$\text{Some } (((\text{data}, \text{attacker-state}), \text{result}), \text{final-state}) \in$
 set-dist
 $(\text{execute } (\text{staged-security-experiment-with-data-state } A) \text{ initial-state})$
obtains
 $\text{Some } (\text{data}, \text{attacker-state}) \in$
 $\text{set-dist } (\text{execute } (\text{staged-transcript-program } A) \text{ initial-state})$
 $\text{Some } (\text{result}, \text{final-state}) \in$
 set-dist
 $(\text{execute } \text{verify-monad}$
 $(\text{verifier-state-from-adversary } \text{attacker-state}$
 $(\text{staged-proof-transcript } \text{data})))$
using *assms* **unfolding** *staged-security-experiment-with-data-state-def*
by (*auto elim!*: *set-dist-bindE* *intro*: *that*)

lemma *checked-staged-security-experiment-with-data-state-outcomeE*:

assumes
 $\text{Some } (((\text{data}, \text{attacker-state}), \text{result}), \text{final-state}) \in$
 set-dist
 $(\text{execute}$
 $(\text{checked-staged-security-experiment-with-data-state } A)$
 $\text{initial-state})$
obtains
 $\text{Some } (\text{data}, \text{attacker-state}) \in$
 set-dist
 $(\text{execute } (\text{checked-staged-transcript-program } A) \text{ initial-state})$
 $\text{Some } (\text{result}, \text{final-state}) \in$
 set-dist
 $(\text{execute } \text{verify-monad}$
 $(\text{verifier-state-from-adversary } \text{attacker-state}$
 $(\text{staged-proof-transcript } \text{data})))$
using *assms*
unfolding *checked-staged-security-experiment-with-data-state-def*
by (*auto elim!*: *set-dist-bindE* *intro*: *that*)

lemma *checked-staged-security-with-data-state-verifier-event-bound-from-cont*:

assumes *no-none*: $\bigwedge s. \neg P \ s \ \text{None}$
and *cont-bound*:
 $\bigwedge \text{data } \text{attacker-state}.$
 $\text{Some } (\text{data}, \text{attacker-state}) \in$
 set-dist
 $(\text{execute } (\text{checked-staged-transcript-program } A)$
 $\text{adversary-initial-state}) \implies$
 $\text{wp-event } \text{verify-monad}$
 $(P$
 $(\text{verifier-state-from-adversary } \text{attacker-state}$
 $(\text{staged-proof-transcript } \text{data})))$
 $(\text{verifier-state-from-adversary } \text{attacker-state}$
 $(\text{staged-proof-transcript } \text{data})) \leq C$
shows

```

    wp-event (checked-staged-security-experiment-with-data-state A)
      (staged-security-with-data-state-verifier-event P)
      adversary-initial-state ≤ C
  unfolding checked-staged-security-experiment-with-data-state-def
proof (rule wp-event-bind-bound-by-cont)
  show ¬ staged-security-with-data-state-verifier-event P None
    unfolding staged-security-with-data-state-verifier-event-def by simp
next
fix data attacker-state
assume builder:
  Some (data, attacker-state) ∈
    set-dist
      (execute (checked-staged-transcript-program A)
        adversary-initial-state)
let ?s =
  verifier-state-from-adversary attacker-state
  (staged-proof-transcript data)
have cont-eq:
  wp-event
    (get >=>
      (λs. put
        (verifier-state-from-adversary s
          (staged-proof-transcript data)) >=>
        (λ-. verify-monad >=>
          (λresult. return ((data, s), result))))))
    (staged-security-with-data-state-verifier-event P) attacker-state =
  wp-event verify-monad (P ?s) ?s
unfolding wp-event-def staged-security-with-data-state-verifier-event-def
apply (simp add: wpsimps)
apply (rule arg-cong[where f=λQ. wp verify-monad Q ?s])
apply (rule ext)
apply (simp add: no-none wp-return split: option.splits prod.splits)
done
show
  wp-event
    (get >=>
      (λs. put
        (verifier-state-from-adversary s
          (staged-proof-transcript data)) >=>
        (λ-. verify-monad >=>
          (λresult. return ((data, s), result))))))
    (staged-security-with-data-state-verifier-event P) attacker-state ≤ C
  unfolding cont-eq by (rule cont-bound[OF builder])
qed

lemma checked-staged-security-with-data-state-bound-from-data-cont:
  assumes none: ¬ P None
  and cont-bound:
    ∧ data attacker-state.

```

```

Some (data, attacker-state) ∈
  set-dist
  (execute (checked-staged-transcript-program A)
    adversary-initial-state) ⇒
wp-event verify-monad
  (λout.
    P (case out of
      None ⇒ None
      | Some (result, final-state) ⇒
        Some (((data, attacker-state), result), final-state)))
    (verifier-state-from-adversary attacker-state
      (staged-proof-transcript data)) ≤ C
shows
  wp-event (checked-staged-security-experiment-with-data-state A) P
  adversary-initial-state ≤ C
unfolding checked-staged-security-experiment-with-data-state-def
proof (rule wp-event-bind-bound-by-cont)
show ¬ P None
by (rule none)
next
fix data attacker-state
assume builder:
  Some (data, attacker-state) ∈
    set-dist
    (execute (checked-staged-transcript-program A)
      adversary-initial-state)
let ?s =
  verifier-state-from-adversary attacker-state
  (staged-proof-transcript data)
have cont-eq:
  wp-event
  (get ≫=
    (λs. put
      (verifier-state-from-adversary s
        (staged-proof-transcript data)) ≫=
      (λ-. verify-monad ≫=
        (λresult. return ((data, s), result)))))
  P attacker-state =
wp-event verify-monad
  (λout.
    P (case out of
      None ⇒ None
      | Some (result, final-state) ⇒
        Some (((data, attacker-state), result), final-state)))
  ?s
unfolding wp-event-def
apply (simp add: none wpsimps)
apply (rule arg-cong[where f=λQ. wp verify-monad Q ?s])
apply (rule ext)

```

```

apply (rename-tac out)
apply (case-tac out)
  apply (simp add: none)
apply (rename-tac result-state)
apply (case-tac result-state)
by (simp add: wp-return)
show
  wp-event
  (get  $\gg$ 
    ( $\lambda s$ . put
      (verifier-state-from-adversary s
        (staged-proof-transcript data))  $\gg$ 
      ( $\lambda$ -. verify-monad  $\gg$ 
        ( $\lambda$ result. return ((data, s), result))))))
  P attacker-state  $\leq$  C
unfolding cont-eq by (rule cont-bound[OF builder])
qed

lemma checked-staged-security-with-data-state-query-bad-verifier-event-bound:
assumes query-bound:
  wp-event (checked-staged-security-experiment-with-data-state A)
  staged-security-with-data-state-query-bad-hit
  adversary-initial-state  $\leq$  C
shows
  wp-event (checked-staged-security-experiment-with-data-state A)
  (staged-security-with-data-state-verifier-event query-bad)
  adversary-initial-state  $\leq$  C
proof –
have wp-event (checked-staged-security-experiment-with-data-state A)
  (staged-security-with-data-state-verifier-event query-bad)
  adversary-initial-state  $\leq$ 
  wp-event (checked-staged-security-experiment-with-data-state A)
  staged-security-with-data-state-query-bad-hit
  adversary-initial-state
proof (rule wp-event-mono-on-support)
fix out
assume support:
  out  $\in$ 
  set-dist
  (execute
    (checked-staged-security-experiment-with-data-state A)
    adversary-initial-state)
and event:
  staged-security-with-data-state-verifier-event query-bad out
show staged-security-with-data-state-query-bad-hit out
proof (cases out)
case None
then show ?thesis
  using event unfolding staged-security-with-data-state-verifier-event-def

```

```

    by simp
next
case (Some packed)
then obtain data attacker-state result final-state where out-eq:
  out = Some (((data, attacker-state), result), final-state)
  by (cases packed, auto split: prod.splits)
let ?s =
  verifier-state-from-adversary attacker-state
  (staged-proof-transcript data)
have verifier:
  Some (result, final-state) ∈ set-dist (execute verify-monad ?s)
  using checked-staged-security-experiment-with-data-state-outcomeE
  [OF support[unfolded out-eq]]
  by blast
have bad: query-bad ?s (Some (result, final-state))
  using event
  unfolding out-eq staged-security-with-data-state-verifier-event-def
  by simp
show ?thesis
  unfolding out-eq staged-security-with-data-state-query-bad-hit-def
  Let-def
  using verifier bad by simp
qed
qed
also have ... ≤ C
  by (rule query-bound)
finally show ?thesis .
qed

```

definition *staged-semantic-adversary*
 $:: 'f \text{ staged-adversary} \Rightarrow 'f \text{ semantic-adversary}$
where
staged-semantic-adversary $A =$
staged-transcript-program $A \gg=$
 $(\lambda \text{data. return (staged-proof-transcript data)})$

lemma *staged-security-experiment-eq-security-experiment:*
staged-security-experiment $A =$
security-experiment (*staged-semantic-adversary* A)
unfolding *staged-security-experiment-def* *security-experiment-def*
staged-semantic-adversary-def *verifier-state-transfer-def*
by (*simp add: sm-bind-assoc*)

definition *staged-adversary-acceptance-probability*
 $:: 'f \text{ staged-adversary} \Rightarrow \text{prob}$
where
staged-adversary-acceptance-probability $A =$
wp-event (*staged-security-experiment* A)
 $(\lambda \text{out. } \neg \text{Option.is-none out}) \text{ adversary-initial-state}$

definition *checked-staged-adversary-acceptance-probability*
 $:: 'f \text{ staged-adversary} \Rightarrow \text{prob}$
where
checked-staged-adversary-acceptance-probability $A =$
 $\text{wp-event } (\text{checked-staged-security-experiment } A)$
 $(\lambda \text{out. } \neg \text{Option.is-none out}) \text{ adversary-initial-state}$

end

end

theory *Staged-Security-Experiment-Budgets*
imports *Staged-Security-Experiment-Core*
begin

Hash range, collision, relation, and target-budget calculus for staged programs.

context *soundness*
begin

lemma *record-staged-message-outcome*:
assumes
 $\text{Some } ((\text{ }, t) \in \text{set-dist } (\text{execute } (\text{record-staged-message } x) s))$
shows
 $t = s \langle \langle$
 $\quad P\text{State} := \text{concat } (P\text{State } s) x,$
 $\quad P\text{Transcript} := P\text{Transcript } s @ [x] \rangle \rangle$
using *assms unfolding record-staged-message-def modify-def*
by *(auto elim!: set-dist-bindE)*

lemma *record-staged-message-simps[simp]*:
 HashMap
 $(s \langle \langle P\text{State} := \text{concat } (P\text{State } s) x,$
 $\quad P\text{Transcript} := P\text{Transcript } s @ [x] \rangle \rangle) = \text{HashMap } s$
 $P\text{State}$
 $(s \langle \langle P\text{State} := \text{concat } (P\text{State } s) x,$
 $\quad P\text{Transcript} := P\text{Transcript } s @ [x] \rangle \rangle) =$
 $\text{concat } (P\text{State } s) x$
 $P\text{Transcript}$
 $(s \langle \langle P\text{State} := \text{concat } (P\text{State } s) x,$
 $\quad P\text{Transcript} := P\text{Transcript } s @ [x] \rangle \rangle) =$
 $P\text{Transcript } s @ [x]$
 $P\text{TraceFriCounter}$
 $(s \langle \langle P\text{State} := \text{concat } (P\text{State } s) x,$
 $\quad P\text{Transcript} := P\text{Transcript } s @ [x] \rangle \rangle) =$
 $P\text{TraceFriCounter } s$
 $P\text{CompositionFriCounter}$
 $(s \langle \langle P\text{State} := \text{concat } (P\text{State } s) x,$

```

      PTranscript := PTranscript s @ [x])) =
    PCompositionFriCounter s
  PAlphaCounter
    (s(PState := concat (PState s) x,
      PTranscript := PTranscript s @ [x])) =
    PAlphaCounter s
  PQueryCounter
    (s(PState := concat (PState s) x,
      PTranscript := PTranscript s @ [x])) =
    PQueryCounter s
  by simp-all

```

lemma *record-staged-message-preserves-hash-map:*

hash-map-preserving (record-staged-message x)

unfolding *hash-map-preserving-def record-staged-message-def modify-def*

by (*auto elim!: set-dist-bindE*)

lemma *hash-range-budget-record-staged-message:*

hash-range-budget 0 (record-staged-message x)

unfolding *record-staged-message-def*

by (*rule hash-range-budget-modify-preserves-hash-map*) *simp*

lemma *hash-collision-budget-record-staged-message:*

hash-collision-budget 0 (record-staged-message x)

unfolding *record-staged-message-def*

by (*rule hash-collision-budget-modify-preserves-hash-map*) *simp*

lemma *hash-target-program-record-staged-message:*

hash-target-program B 0 (record-staged-message x)

by (*rule hash-map-preserving-imp-hash-target-program-zero*)

(*rule record-staged-message-preserves-hash-map*)

lemma *hash-relation-program-get:*

hash-relation-program R b 0

(*get :: ('f protocol-channel, 'f protocol-channel) state-monad*)

by (*rule hash-relation-program-zero*)

(*rule hash-map-preserving-get*)

lemma *hash-relation-program-assert:*

hash-relation-program R b 0

(*assert x :: (unit, 'f protocol-channel) state-monad*)

by (*rule hash-relation-program-zero*)

(*rule hash-map-preserving-assert*)

lemma *hash-relation-program-modify:*

assumes $\bigwedge s. \text{HashMap } (f\ s) = \text{HashMap } s$

shows *hash-relation-program R b 0 (modify f)*

by (*rule hash-relation-program-zero*)

(*rule hash-map-preserving-modify[OF assms]*)

lemma *hash-relation-program-record-staged-message:*
hash-relation-program R b 0 (*record-staged-message* x)
by (*rule hash-relation-program-zero*)
(*rule record-staged-message-preserves-hash-map*)

lemma *record-staged-messages-outcome:*
assumes
Some $((\), t) \in$
set-dist (*execute* (*record-staged-messages* xs) s)
shows
 $t = s\langle$
PState $:= \text{foldl concat } (PState\ s)\ xs,$
PTranscript $:= PTranscript\ s\ @\ xs\)$
using *assms*
proof (*induction xs arbitrary: s*)
case *Nil*
then show *?case*
unfolding *record-staged-messages-def* **by** *simp*
next
case (*Cons x xs*)
from *Cons.prem*s **obtain** u **where**
head:
Some $((\), u) \in$
set-dist (*execute* (*record-staged-message* x) s)
and *tail:*
Some $((\), t) \in$
set-dist (*execute* (*record-staged-messages* xs) u)
unfolding *record-staged-messages-def*
by (*auto elim!: set-dist-bindE*)
have $u:$
 $u = s\langle$
PState $:= \text{concat } (PState\ s)\ x,$
PTranscript $:= PTranscript\ s\ @\ [x]\)$
by (*rule record-staged-message-outcome[OF head]*)
have $t:$
 $t = u\langle$
PState $:= \text{foldl concat } (PState\ u)\ xs,$
PTranscript $:= PTranscript\ u\ @\ xs\)$
by (*rule Cons.IH[OF tail]*)
show *?case*
unfolding $t\ u$ **by** *simp*
qed

lemma *record-staged-messages-preserves-hash-map:*
hash-map-preserving (*record-staged-messages* xs)
unfolding *hash-map-preserving-def*
using *record-staged-messages-outcome* **by** *fastforce*

lemma *hash-range-budget-record-staged-messages:*
hash-range-budget 0 (record-staged-messages xs)
using *record-staged-messages-preserves-hash-map*
unfolding *hash-map-preserving-def hash-range-budget-def*
hash-map-output-values-def
by *auto*

lemma *hash-collision-budget-record-staged-messages:*
hash-collision-budget 0 (record-staged-messages xs)
by (*rule hash-map-preserving-imp-hash-collision-budget-zero-semantic*)
(*rule record-staged-messages-preserves-hash-map*)

lemma *hash-target-program-record-staged-messages:*
hash-target-program B 0 (record-staged-messages xs)
by (*rule hash-map-preserving-imp-hash-target-program-zero*)
(*rule record-staged-messages-preserves-hash-map*)

lemma *hash-relation-program-record-staged-messages:*
hash-relation-program R b 0 (record-staged-messages xs)
by (*rule hash-relation-program-zero*)
(*rule record-staged-messages-preserves-hash-map*)

lemma *hash-relation-program-receive-counted-tagged-random-field-element:*
assumes *bump: $\bigwedge s. \text{HashMap } (\text{bump } s) = \text{HashMap } s$*
and *fibers: $\bigwedge x. \text{card } \{y. R \ x \ y\} \leq b$*
shows *hash-relation-program R b 1*
(*protocol-receive-counted-tagged-random-field-element counter bump tag ::*
(*'f, 'f, unit*) *protocol-c-monad*)

proof –
have *modify-return:*
hash-relation-program R b (0 + 0)
(*modify bump $\gg=$ ($\lambda\cdot$. return r) ::*
(*'f, 'f protocol-channel*) *state-monad*)
for *r :: 'f*
by (*rule hash-relation-program-bind*)
(*rule hash-relation-program-modify[OF bump],*
rule hash-relation-program-zero[OF hash-map-preserving-return])

have *hash-tail:*
hash-relation-program R b (1 + (0 + 0))
(*hash (tag (counter s) (PState s)) $\gg=$*
(*λr . modify bump $\gg=$ ($\lambda\cdot$. return r) ::*
(*'f, 'f protocol-channel*) *state-monad*)
for *s :: 'f protocol-channel*
by (*rule hash-relation-program-bind*)
(*rule hash-relation-program-hash[OF fibers], rule modify-return*)

have *hash-relation-program R b (0 + (1 + (0 + 0)))*
(*get $\gg=$*
(*$\lambda s :: 'f$ protocol-channel.*
hash (tag (counter s) (PState s)) $\gg=$

```

      (λr. modify bump ≫ (λ-. return r))))
  by (rule hash-relation-program-bind
      [OF hash-relation-program-get hash-tail])
  then show ?thesis
    unfolding protocol-receive-counted-tagged-random-field-element-def
    by simp
qed

lemma hash-relation-program-receive-query-index-challenge:
  assumes fibers:  $\bigwedge x. \text{card } \{y. R \ x \ y\} \leq b$ 
  shows hash-relation-program  $R \ b \ 1$ 
    (receive-query-index-challenge :: ('f, 'f, unit) protocol-c-monad)
  unfolding receive-query-index-challenge-def
  by (rule hash-relation-program-receive-counted-tagged-random-field-element)
    (simp-all add: fibers)

lemma hash-relation-program-receive-trace-fri-challenge:
  assumes fibers:  $\bigwedge x. \text{card } \{y. R \ x \ y\} \leq b$ 
  shows hash-relation-program  $R \ b \ 1$ 
    (receive-trace-fri-challenge :: ('f, 'f, unit) protocol-c-monad)
  unfolding receive-trace-fri-challenge-def
  by (rule hash-relation-program-receive-counted-tagged-random-field-element)
    (simp-all add: fibers)

lemma hash-relation-program-receive-composition-fri-challenge:
  assumes fibers:  $\bigwedge x. \text{card } \{y. R \ x \ y\} \leq b$ 
  shows hash-relation-program  $R \ b \ 1$ 
    (receive-composition-fri-challenge :: ('f, 'f, unit) protocol-c-monad)
  unfolding receive-composition-fri-challenge-def
  by (rule hash-relation-program-receive-counted-tagged-random-field-element)
    (simp-all add: fibers)

lemma hash-relation-program-receive-alpha-challenge:
  assumes fibers:  $\bigwedge x. \text{card } \{y. R \ x \ y\} \leq b$ 
  shows hash-relation-program  $R \ b \ 1$ 
    (receive-alpha-challenge :: ('f, 'f, unit) protocol-c-monad)
  unfolding receive-alpha-challenge-def
  by (rule hash-relation-program-receive-counted-tagged-random-field-element)
    (simp-all add: fibers)

lemma controlled-stage-outcome-fields:
  assumes controlled: controlled-ro-program  $q \ m$ 
    and outcome:  $\text{Some } (x, t) \in \text{set-dist } (\text{execute } m \ s)$ 
  shows
    PState  $t = PState \ s$ 
    PTranscript  $t = PTranscript \ s$ 
    PTraceFriCounter  $t = PTraceFriCounter \ s$ 
    PCompositionFriCounter  $t = PCompositionFriCounter \ s$ 
    PAlphaCounter  $t = PAlphaCounter \ s$ 

```

```

    PQueryCounter t = PQueryCounter s
using controlled-ro-program-preserves-protocol-fields[OF controlled]
    outcome
unfolding protocol-fields-preserving-def
by blast+

```

```

lemma sum-list-take-Suc-drop:
  assumes i < length xs
  shows sum-list (take (Suc n) (drop i xs)) =
    xs ! i + sum-list (take n (drop (Suc i) xs))
proof -
  have drop-i:
    drop i xs = xs ! i # drop (Suc i) xs
    using Cons-nth-drop-Suc[OF assms] by simp
  show ?thesis
    unfolding drop-i by simp
qed

```

```

lemma sum-list-take-le:
  fixes xs :: nat list
  shows sum-list (take n xs) ≤ sum-list xs
proof (induction n arbitrary: xs)
  case 0
  then show ?case by simp
next
  case (Suc n)
  then show ?case
    by (cases xs) simp-all
qed

```

```

lemma hash-range-budget-staged-trace-fri-program:
  assumes controlled: staged-adversary-controlled budgets A
    and bound: i + n ≤ length (trace-fri-budgets budgets)
  shows
    hash-range-budget
      (sum-list (take n (drop i (trace-fri-budgets budgets))) + n)
      (staged-trace-fri-program A i n bs)
  using bound
proof (induction n arbitrary: i bs)
  case 0
  then show ?case by (simp add: hash-range-budget-return)
next
  case (Suc n)
  have i-bound: i < length (trace-fri-budgets budgets)
    using Suc.premis by simp
  have stage:
    hash-range-budget (trace-fri-budgets budgets ! i)
      (trace-fri-root-stage A i bs)
    using controlled i-bound

```

```

unfolding staged-adversary-controlled-def
by (blast intro: controlled-ro-program-range)
have record-budget:
   $\wedge$  root. hash-range-budget 0 (record-staged-message root)
  by (rule hash-range-budget-record-staged-message)
have challenge: hash-range-budget 1 receive-trace-fri-challenge
  by (rule hash-range-budget-receive-trace-fri-challenge)
have tail:
   $\wedge$  b. hash-range-budget
    (sum-list (take n (drop (Suc i) (trace-fri-budgets budgets))) + n)
    (staged-trace-fri-program A (Suc i) n (bs @ [b]))
  by (rule Suc.IH) (use Suc.premis in simp)
have tail-return:
   $\wedge$  b root. hash-range-budget
    (sum-list (take n (drop (Suc i) (trace-fri-budgets budgets))) + n + 0)
    (staged-trace-fri-program A (Suc i) n (bs @ [b])  $\gg$ 
      ( $\lambda$ (roots, bs'). return (root # roots, bs')))
  by (rule hash-range-budget-bind)
    (rule tail, simp add: hash-range-budget-return split: prod.splits)
have after-challenge:
   $\wedge$  root. hash-range-budget
    (1 +
      (sum-list (take n (drop (Suc i) (trace-fri-budgets budgets))) +
        n + 0))
    (receive-trace-fri-challenge  $\gg$ 
      ( $\lambda$ b. staged-trace-fri-program A (Suc i) n (bs @ [b])  $\gg$ 
        ( $\lambda$ (roots, bs'). return (root # roots, bs'))))
  by (rule hash-range-budget-bind)
    (rule challenge, rule tail-return)
have after-record:
   $\wedge$  root. hash-range-budget
    (0 +
      (1 +
        (sum-list (take n (drop (Suc i) (trace-fri-budgets budgets))) +
          n + 0)))
    (record-staged-message root  $\gg$ 
      ( $\lambda$ -. receive-trace-fri-challenge  $\gg$ 
        ( $\lambda$ b. staged-trace-fri-program A (Suc i) n (bs @ [b])  $\gg$ 
          ( $\lambda$ (roots, bs'). return (root # roots, bs')))))
  by (rule hash-range-budget-bind)
    (rule record-budget, rule after-challenge)
have whole:
  hash-range-budget
    (trace-fri-budgets budgets ! i +
      (0 +
        (1 +
          (sum-list (take n (drop (Suc i) (trace-fri-budgets budgets))) +
            n + 0))))
    (staged-trace-fri-program A i (Suc n) bs)

```

```

unfolding staged-trace-fri-program.simps
by (rule hash-range-budget-bind)
    (rule stage, rule after-record)
have budget-eq:
  trace-fri-budgets budgets ! i +
    (0 +
      (1 +
        (sum-list (take n (drop (Suc i) (trace-fri-budgets budgets))) +
          n + 0))) =
    sum-list (take (Suc n) (drop i (trace-fri-budgets budgets))) + Suc n
using sum-list-take-Suc-drop[OF i-bound, of n] by simp
show ?case
using whole unfolding budget-eq .
qed

lemma hash-collision-budget-staged-trace-fri-program:
assumes controlled: staged-adversary-controlled budgets A
and bound:  $i + n \leq \text{length } (\text{trace-fri-budgets budgets})$ 
shows
  hash-collision-budget
    (sum-list (take n (drop i (trace-fri-budgets budgets))) + n)
    (staged-trace-fri-program A i n bs)
using bound
proof (induction n arbitrary: i bs)
case 0
then show ?case by (simp add: hash-collision-budget-return)
next
case (Suc n)
have i-bound:  $i < \text{length } (\text{trace-fri-budgets budgets})$ 
using Suc.prem1 by simp
have stage-range:
  hash-range-budget (trace-fri-budgets budgets ! i)
    (trace-fri-root-stage A i bs)
using controlled i-bound
unfolding staged-adversary-controlled-def
by (blast intro: controlled-ro-program-range)
have stage-coll:
  hash-collision-budget (trace-fri-budgets budgets ! i)
    (trace-fri-root-stage A i bs)
using controlled i-bound
unfolding staged-adversary-controlled-def
by (blast intro: controlled-ro-program-collision)
have record-range:
   $\wedge \text{root. hash-range-budget } 0 \text{ (record-staged-message root)}$ 
by (rule hash-range-budget-record-staged-message)
have record-coll:
   $\wedge \text{root. hash-collision-budget } 0 \text{ (record-staged-message root)}$ 
by (rule hash-collision-budget-record-staged-message)
have challenge-range: hash-range-budget 1 receive-trace-fri-challenge

```



```

  by (rule hash-range-budget-receive-trace-fri-challenge)
have challenge-coll: hash-collision-budget 1 receive-trace-fri-challenge
  by (rule hash-collision-budget-receive-trace-fri-challenge)
have tail-range:
   $\wedge b.$  hash-range-budget
    (sum-list (take n (drop (Suc i) (trace-fri-budgets budgets))) + n)
    (staged-trace-fri-program A (Suc i) n (bs @ [b])))
  by (rule hash-range-budget-staged-trace-fri-program[OF controlled])
    (use Suc.premis in simp)
have tail-coll:
   $\wedge b.$  hash-collision-budget
    (sum-list (take n (drop (Suc i) (trace-fri-budgets budgets))) + n)
    (staged-trace-fri-program A (Suc i) n (bs @ [b])))
  by (rule Suc.IH) (use Suc.premis in simp)
have tail-return-range:
   $\wedge b$  root. hash-range-budget
    (sum-list (take n (drop (Suc i) (trace-fri-budgets budgets))) + n + 0)
    (staged-trace-fri-program A (Suc i) n (bs @ [b])  $\gg$ 
      ( $\lambda$ (roots, bs'). return (root # roots, bs'))))
  by (rule hash-range-budget-bind)
    (rule tail-range, simp add: hash-range-budget-return split: prod.splits)
have tail-return-coll:
   $\wedge b$  root. hash-collision-budget
    (sum-list (take n (drop (Suc i) (trace-fri-budgets budgets))) + n + 0)
    (staged-trace-fri-program A (Suc i) n (bs @ [b])  $\gg$ 
      ( $\lambda$ (roots, bs'). return (root # roots, bs'))))
  by (rule hash-collision-budget-bind)
    (rule tail-range, rule tail-coll,
      simp add: hash-range-budget-return split: prod.splits,
      simp add: hash-collision-budget-return split: prod.splits)
have after-challenge-range:
   $\wedge$ root. hash-range-budget
    (1 +
      (sum-list (take n (drop (Suc i) (trace-fri-budgets budgets))) +
        n + 0))
    (receive-trace-fri-challenge  $\gg$ 
      ( $\lambda b.$  staged-trace-fri-program A (Suc i) n (bs @ [b])  $\gg$ 
        ( $\lambda$ (roots, bs'). return (root # roots, bs')))))
  by (rule hash-range-budget-bind)
    (rule challenge-range, rule tail-return-range)
have after-challenge-coll:
   $\wedge$ root. hash-collision-budget
    (1 +
      (sum-list (take n (drop (Suc i) (trace-fri-budgets budgets))) +
        n + 0))
    (receive-trace-fri-challenge  $\gg$ 
      ( $\lambda b.$  staged-trace-fri-program A (Suc i) n (bs @ [b])  $\gg$ 
        ( $\lambda$ (roots, bs'). return (root # roots, bs')))))
  by (rule hash-collision-budget-bind)

```

```

(rule challenge-range, rule challenge-coll, rule tail-return-range,
 rule tail-return-coll)
have after-record-range:
 $\wedge$ root. hash-range-budget
  (0 +
    (1 +
      (sum-list (take n (drop (Suc i) (trace-fri-budgets budgets))) +
        n + 0)))
    (record-staged-message root  $\gg$ 
      ( $\lambda$ -. receive-trace-fri-challenge  $\gg$ 
        ( $\lambda$ b. staged-trace-fri-program A (Suc i) n (bs @ [b])  $\gg$ 
          ( $\lambda$ (roots, bs'). return (root # roots, bs')))))
  by (rule hash-range-budget-bind)
    (rule record-range, rule after-challenge-range)
have after-record-coll:
 $\wedge$ root. hash-collision-budget
  (0 +
    (1 +
      (sum-list (take n (drop (Suc i) (trace-fri-budgets budgets))) +
        n + 0)))
    (record-staged-message root  $\gg$ 
      ( $\lambda$ -. receive-trace-fri-challenge  $\gg$ 
        ( $\lambda$ b. staged-trace-fri-program A (Suc i) n (bs @ [b])  $\gg$ 
          ( $\lambda$ (roots, bs'). return (root # roots, bs')))))
  by (rule hash-collision-budget-bind)
    (rule record-range, rule record-coll, rule after-challenge-range,
      rule after-challenge-coll)
have whole:
  hash-collision-budget
  (trace-fri-budgets budgets ! i +
    (0 +
      (1 +
        (sum-list (take n (drop (Suc i) (trace-fri-budgets budgets))) +
          n + 0))))
    (staged-trace-fri-program A i (Suc n) bs)
unfolding staged-trace-fri-program.simps
by (rule hash-collision-budget-bind)
  (rule stage-range, rule stage-coll, rule after-record-range,
    rule after-record-coll)
have budget-eq:
  trace-fri-budgets budgets ! i +
    (0 +
      (1 +
        (sum-list (take n (drop (Suc i) (trace-fri-budgets budgets))) +
          n + 0))) =
    sum-list (take (Suc n) (drop i (trace-fri-budgets budgets))) + Suc n
using sum-list-take-Suc-drop[OF i-bound, of n] by simp
show ?case
using whole unfolding budget-eq .

```

qed

lemma *hash-range-budget-staged-alpha-program:*
 hash-range-budget *n* (*staged-alpha-program* *n*)
proof (*induction* *n*)
 case 0
 then show ?case by (simp add: *hash-range-budget-return*)
next
 case (Suc *n*)
 have challenge: *hash-range-budget* 1 *receive-alpha-challenge*
 by (rule *hash-range-budget-receive-alpha-challenge*)
 have record-budget:
 $\wedge a.$ *hash-range-budget* 0 (*record-staged-message* *a*)
 by (rule *hash-range-budget-record-staged-message*)
 have tail-return:
 $\wedge a.$ *hash-range-budget* (*n* + 0)
 (*staged-alpha-program* *n* $\ggg$ ($\lambda as.$ *return* (*a* # *as*)))
 by (rule *hash-range-budget-bind*)
 (rule *Suc.IH*, rule *hash-range-budget-return*)
 have after-record:
 $\wedge a.$ *hash-range-budget* (0 + (*n* + 0))
 (*record-staged-message* *a* $\ggg$
 ($\lambda.$ *staged-alpha-program* *n* $\ggg$ ($\lambda as.$ *return* (*a* # *as*))))
 by (rule *hash-range-budget-bind*)
 (rule *record-budget*, rule *tail-return*)
 have *hash-range-budget* (1 + (0 + (*n* + 0)))
 (*staged-alpha-program* (*Suc* *n*))
 unfolding *staged-alpha-program.simps*
 by (rule *hash-range-budget-bind*)
 (rule *challenge*, rule *after-record*)
 then show ?case by simp
qed

lemma *hash-collision-budget-staged-alpha-program:*
 hash-collision-budget *n* (*staged-alpha-program* *n*)
proof (*induction* *n*)
 case 0
 then show ?case by (simp add: *hash-collision-budget-return*)
next
 case (Suc *n*)
 have challenge-range: *hash-range-budget* 1 *receive-alpha-challenge*
 by (rule *hash-range-budget-receive-alpha-challenge*)
 have challenge-coll: *hash-collision-budget* 1 *receive-alpha-challenge*
 by (rule *hash-collision-budget-receive-alpha-challenge*)
 have record-range:
 $\wedge a.$ *hash-range-budget* 0 (*record-staged-message* *a*)
 by (rule *hash-range-budget-record-staged-message*)
 have record-coll:
 $\wedge a.$ *hash-collision-budget* 0 (*record-staged-message* *a*)

```

    by (rule hash-collision-budget-record-staged-message)
  have tail-return-range:
     $\bigwedge a. \text{hash-range-budget } (n + 0)$ 
    (staged-alpha-program  $n \gg= (\lambda as. \text{return } (a \# as))$ )
  by (rule hash-range-budget-bind)
    (rule hash-range-budget-staged-alpha-program,
      rule hash-range-budget-return)
  have tail-return-coll:
     $\bigwedge a. \text{hash-collision-budget } (n + 0)$ 
    (staged-alpha-program  $n \gg= (\lambda as. \text{return } (a \# as))$ )
  by (rule hash-collision-budget-bind)
    (rule hash-range-budget-staged-alpha-program, rule Suc.IH,
      rule hash-range-budget-return, rule hash-collision-budget-return)
  have after-record-range:
     $\bigwedge a. \text{hash-range-budget } (0 + (n + 0))$ 
    (record-staged-message  $a \gg=$ 
      ( $\lambda -. \text{staged-alpha-program } n \gg= (\lambda as. \text{return } (a \# as))$ )))
  by (rule hash-range-budget-bind)
    (rule record-range, rule tail-return-range)
  have after-record-coll:
     $\bigwedge a. \text{hash-collision-budget } (0 + (n + 0))$ 
    (record-staged-message  $a \gg=$ 
      ( $\lambda -. \text{staged-alpha-program } n \gg= (\lambda as. \text{return } (a \# as))$ )))
  proof (rule hash-collision-budget-bind)
    fix a
    show hash-range-budget 0 (record-staged-message a)
      by (rule record-range)
    show hash-collision-budget 0 (record-staged-message a)
      by (rule record-coll)
    show  $\bigwedge x. \text{hash-range-budget } (n + 0)$ 
      (staged-alpha-program  $n \gg= (\lambda as. \text{return } (a \# as))$ )
      by (rule tail-return-range)
    show  $\bigwedge x. \text{hash-collision-budget } (n + 0)$ 
      (staged-alpha-program  $n \gg= (\lambda as. \text{return } (a \# as))$ )
      by (rule tail-return-coll)
  qed
  have hash-collision-budget (1 + (0 + (n + 0)))
    (staged-alpha-program (Suc n))
  unfolding staged-alpha-program.simps
  by (rule hash-collision-budget-bind)
    (rule challenge-range, rule challenge-coll, rule after-record-range,
      rule after-record-coll)
  then show ?case by simp
qed

lemma hash-range-budget-staged-composition-fri-program:
  assumes controlled: staged-adversary-controlled budgets A
  and bound:  $i + n \leq \text{length } (\text{composition-fri-budgets budgets})$ 
  shows

```

```

    hash-range-budget
      (sum-list (take n (drop i (composition-fri-budgets budgets))) + n)
      (staged-composition-fri-program A dg i n bs)
  using bound
proof (induction n arbitrary: i bs)
  case 0
  then show ?case by (simp add: hash-range-budget-return)
next
  case (Suc n)
  have i-bound: i < length (composition-fri-budgets budgets)
    using Suc.premis by simp
  have stage:
    hash-range-budget (composition-fri-budgets budgets ! i)
      (composition-fri-root-stage A dg i bs)
    using controlled i-bound
  unfolding staged-adversary-controlled-def
  by (blast intro: controlled-ro-program-range)
  have record-budget:
     $\wedge$  root. hash-range-budget 0 (record-staged-message root)
    by (rule hash-range-budget-record-staged-message)
  have challenge:
    hash-range-budget 1 receive-composition-fri-challenge
    by (rule hash-range-budget-receive-composition-fri-challenge)
  have tail:
     $\wedge$  b. hash-range-budget
      (sum-list
        (take n (drop (Suc i) (composition-fri-budgets budgets))) + n)
        (staged-composition-fri-program A dg (Suc i) n (bs @ [b])))
    by (rule Suc.IH) (use Suc.premis in simp)
  have tail-return:
     $\wedge$  b root. hash-range-budget
      (sum-list
        (take n (drop (Suc i) (composition-fri-budgets budgets))) + n + 0)
        (staged-composition-fri-program A dg (Suc i) n (bs @ [b])  $\gg$ 
          ( $\lambda$ (roots, bs'). return (root # roots, bs'))))
    by (rule hash-range-budget-bind)
      (rule tail, simp add: hash-range-budget-return split: prod.splits)
  have after-challenge:
     $\wedge$  root. hash-range-budget
      (1 +
        (sum-list
          (take n (drop (Suc i) (composition-fri-budgets budgets))) +
          n + 0))
        (receive-composition-fri-challenge  $\gg$ 
          ( $\lambda$ b. staged-composition-fri-program A dg (Suc i) n
            (bs @ [b])  $\gg$ 
              ( $\lambda$ (roots, bs'). return (root # roots, bs')))))
    by (rule hash-range-budget-bind)
      (rule challenge, rule tail-return)

```

```

have after-record:
   $\wedge$  root. hash-range-budget
    (0 +
      (1 +
        (sum-list
          (take n (drop (Suc i) (composition-fri-budgets budgets))) +
            n + 0)))
      (record-staged-message root  $\gg$ 
        ( $\lambda$ -. receive-composition-fri-challenge  $\gg$ 
          ( $\lambda$ b. staged-composition-fri-program A dg (Suc i) n
            (bs @ [b])  $\gg$ 
              ( $\lambda$ (roots, bs'). return (root # roots, bs')))))
    by (rule hash-range-budget-bind)
      (rule record-budget, rule after-challenge)
have whole:
  hash-range-budget
    (composition-fri-budgets budgets ! i +
      (0 +
        (1 +
          (sum-list
            (take n (drop (Suc i) (composition-fri-budgets budgets))) +
              n + 0))))
      (staged-composition-fri-program A dg i (Suc n) bs)
  unfolding staged-composition-fri-program.simps
  by (rule hash-range-budget-bind)
    (rule stage, rule after-record)
have budget-eq:
  composition-fri-budgets budgets ! i +
    (0 +
      (1 +
        (sum-list
          (take n (drop (Suc i) (composition-fri-budgets budgets))) +
            n + 0))) =
    sum-list (take (Suc n) (drop i (composition-fri-budgets budgets))) +
      Suc n
  using sum-list-take-Suc-drop[OF i-bound, of n] by simp
show ?case
  using whole unfolding budget-eq .
qed

lemma hash-collision-budget-staged-composition-fri-program:
  assumes controlled: staged-adversary-controlled budgets A
  and bound:  $i + n \leq \text{length } (\text{composition-fri-budgets budgets})$ 
shows
  hash-collision-budget
    (sum-list (take n (drop i (composition-fri-budgets budgets))) + n)
    (staged-composition-fri-program A dg i n bs)
  using bound
proof (induction n arbitrary: i bs)

```

```

case 0
then show ?case by (simp add: hash-collision-budget-return)
next
case (Suc n)
have i-bound:  $i < \text{length } (\text{composition-fri-budgets } \text{budgets})$ 
  using Suc.prem by simp
have stage-range:
  hash-range-budget (composition-fri-budgets budgets ! i)
    (composition-fri-root-stage A dg i bs)
  using controlled i-bound
  unfolding staged-adversary-controlled-def
  by (blast intro: controlled-ro-program-range)
have stage-coll:
  hash-collision-budget (composition-fri-budgets budgets ! i)
    (composition-fri-root-stage A dg i bs)
  using controlled i-bound
  unfolding staged-adversary-controlled-def
  by (blast intro: controlled-ro-program-collision)
have record-range:
   $\wedge \text{root. hash-range-budget } 0 \text{ (record-staged-message root)}$ 
  by (rule hash-range-budget-record-staged-message)
have record-coll:
   $\wedge \text{root. hash-collision-budget } 0 \text{ (record-staged-message root)}$ 
  by (rule hash-collision-budget-record-staged-message)
have challenge-range:
  hash-range-budget 1 receive-composition-fri-challenge
  by (rule hash-range-budget-receive-composition-fri-challenge)
have challenge-coll:
  hash-collision-budget 1 receive-composition-fri-challenge
  by (rule hash-collision-budget-receive-composition-fri-challenge)
have tail-range:
   $\wedge b. \text{hash-range-budget}$ 
    (sum-list
      (take n (drop (Suc i) (composition-fri-budgets budgets))) + n)
      (staged-composition-fri-program A dg (Suc i) n (bs @ [b])))
  by (rule hash-range-budget-staged-composition-fri-program
    [OF controlled])
    (use Suc.prem in simp)
have tail-coll:
   $\wedge b. \text{hash-collision-budget}$ 
    (sum-list
      (take n (drop (Suc i) (composition-fri-budgets budgets))) + n)
      (staged-composition-fri-program A dg (Suc i) n (bs @ [b])))
  by (rule Suc.IH) (use Suc.prem in simp)
have tail-return-range:
   $\wedge b \text{ root. hash-range-budget}$ 
    (sum-list
      (take n (drop (Suc i) (composition-fri-budgets budgets))) + n + 0)
      (staged-composition-fri-program A dg (Suc i) n (bs @ [b])))  $\gg$ 

```

```

    (λ(roots, bs'). return (root # roots, bs'))))
  by (rule hash-range-budget-bind)
    (rule tail-range, simp add: hash-range-budget-return split: prod.splits)
have tail-return-coll:
  ∧b root. hash-collision-budget
    (sum-list
      (take n (drop (Suc i) (composition-fri-budgets budgets))) + n + 0)
    (staged-composition-fri-program A dg (Suc i) n (bs @ [b]) >>=
      (λ(roots, bs'). return (root # roots, bs'))))
  by (rule hash-collision-budget-bind)
    (rule tail-range, rule tail-coll,
      simp add: hash-range-budget-return split: prod.splits,
      simp add: hash-collision-budget-return split: prod.splits)
have after-challenge-range:
  ∧root. hash-range-budget
    (1 +
      (sum-list
        (take n (drop (Suc i) (composition-fri-budgets budgets))) +
          n + 0))
    (receive-composition-fri-challenge >>=
      (λb. staged-composition-fri-program A dg (Suc i) n
        (bs @ [b]) >>=
          (λ(roots, bs'). return (root # roots, bs')))))
  by (rule hash-range-budget-bind)
    (rule challenge-range, rule tail-return-range)
have after-challenge-coll:
  ∧root. hash-collision-budget
    (1 +
      (sum-list
        (take n (drop (Suc i) (composition-fri-budgets budgets))) +
          n + 0))
    (receive-composition-fri-challenge >>=
      (λb. staged-composition-fri-program A dg (Suc i) n
        (bs @ [b]) >>=
          (λ(roots, bs'). return (root # roots, bs')))))
  by (rule hash-collision-budget-bind)
    (rule challenge-range, rule challenge-coll, rule tail-return-range,
      rule tail-return-coll)
have after-record-range:
  ∧root. hash-range-budget
    (0 +
      (1 +
        (sum-list
          (take n (drop (Suc i) (composition-fri-budgets budgets))) +
            n + 0)))
    (record-staged-message root >>=
      (λ-. receive-composition-fri-challenge >>=
        (λb. staged-composition-fri-program A dg (Suc i) n
          (bs @ [b]) >>=
            (λ(roots, bs'). return (root # roots, bs')))))

```



```

    (λ(roots, bs'). return (root # roots, bs')))))
  by (rule hash-range-budget-bind)
    (rule record-range, rule after-challenge-range)
have after-record-coll:
  ∧ root. hash-collision-budget
    (0 +
      (1 +
        (sum-list
          (take n (drop (Suc i) (composition-fri-budgets budgets))) +
            n + 0)))
    (record-staged-message root >=>
      (λ-. receive-composition-fri-challenge >=>
        (λb. staged-composition-fri-program A dg (Suc i) n
          (bs @ [b]) >=>
            (λ(roots, bs'). return (root # roots, bs')))))
  by (rule hash-collision-budget-bind)
    (rule record-range, rule record-coll, rule after-challenge-range,
      rule after-challenge-coll)
have whole:
  hash-collision-budget
    (composition-fri-budgets budgets ! i +
      (0 +
        (1 +
          (sum-list
            (take n (drop (Suc i) (composition-fri-budgets budgets))) +
              n + 0))))
    (staged-composition-fri-program A dg i (Suc n) bs)
unfolding staged-composition-fri-program.simps
by (rule hash-collision-budget-bind)
  (rule stage-range, rule stage-coll, rule after-record-range,
    rule after-record-coll)
have budget-eq:
  composition-fri-budgets budgets ! i +
    (0 +
      (1 +
        (sum-list
          (take n (drop (Suc i) (composition-fri-budgets budgets))) +
            n + 0))) =
    sum-list (take (Suc n) (drop i (composition-fri-budgets budgets))) +
      Suc n
  using sum-list-take-Suc-drop[OF i-bound, of n] by simp
show ?case
  using whole unfolding budget-eq .
qed

lemma hash-range-budget-staged-query-program:
  assumes controlled: staged-adversary-controlled budgets A
    and bound: i + n ≤ length (query-opening-budgets budgets)
  shows

```

```

    hash-range-budget
      (sum-list (take n (drop i (query-opening-budgets budgets))) + n)
      (staged-query-program A i n)
  using bound
proof (induction n arbitrary: i)
  case 0
  then show ?case by (simp add: hash-range-budget-return)
next
  case (Suc n)
  have i-bound: i < length (query-opening-budgets budgets)
    using Suc.prem by simp
  have challenge: hash-range-budget 1 receive-query-index-challenge
    by (rule hash-range-budget-receive-query-index-challenge)
  have stage:
     $\bigwedge$ raw. hash-range-budget (query-opening-budgets budgets ! i)
      (query-opening-stage A i raw)
    using controlled i-bound
    unfolding staged-adversary-controlled-def
    by (blast intro: controlled-ro-program-range)
  have record-budget:
     $\bigwedge$ chunk. hash-range-budget 0 (record-staged-messages chunk)
    by (rule hash-range-budget-record-staged-messages)
  have tail:
    hash-range-budget
      (sum-list (take n (drop (Suc i) (query-opening-budgets budgets))) +
        n)
      (staged-query-program A (Suc i) n)
    by (rule Suc.IH) (use Suc.prem in simp)
  have tail-return:
     $\bigwedge$ chunk. hash-range-budget
      (sum-list (take n (drop (Suc i) (query-opening-budgets budgets))) +
        n + 0)
      (staged-query-program A (Suc i) n  $\gg$ 
        ( $\lambda$ chunks. return (chunk # chunks)))
    by (rule hash-range-budget-bind)
      (rule tail, rule hash-range-budget-return)
  have after-record:
     $\bigwedge$ chunk. hash-range-budget
      (0 +
        (sum-list
          (take n (drop (Suc i) (query-opening-budgets budgets))) +
          n + 0))
      (record-staged-messages chunk  $\gg$ 
        ( $\lambda$ -. staged-query-program A (Suc i) n  $\gg$ 
          ( $\lambda$ chunks. return (chunk # chunks)))))
    by (rule hash-range-budget-bind)
      (rule record-budget, rule tail-return)
  have after-stage:
     $\bigwedge$ raw. hash-range-budget

```

```

    (query-opening-budgets budgets ! i +
      (0 +
        (sum-list
          (take n (drop (Suc i) (query-opening-budgets budgets))) +
            n + 0)))
    (query-opening-stage A i raw >>=
      (λchunk. record-staged-messages chunk >>=
        (λ-. staged-query-program A (Suc i) n >>=
          (λchunks. return (chunk # chunks)))))
  by (rule hash-range-budget-bind)
    (rule stage, rule after-record)
have whole:
  hash-range-budget
    (1 +
      (query-opening-budgets budgets ! i +
        (0 +
          (sum-list
            (take n (drop (Suc i) (query-opening-budgets budgets))) +
              n + 0))))
    (staged-query-program A i (Suc n))
  unfolding staged-query-program.simps
  by (rule hash-range-budget-bind)
    (rule challenge, rule after-stage)
have budget-eq:
  1 +
    (query-opening-budgets budgets ! i +
      (0 +
        (sum-list
          (take n (drop (Suc i) (query-opening-budgets budgets))) +
            n + 0))) =
    sum-list (take (Suc n) (drop i (query-opening-budgets budgets))) +
      Suc n
  using sum-list-take-Suc-drop[OF i-bound, of n] by simp
show ?case
  using whole unfolding budget-eq .
qed

lemma hash-collision-budget-staged-query-program:
  assumes controlled: staged-adversary-controlled budgets A
    and bound: i + n ≤ length (query-opening-budgets budgets)
  shows
    hash-collision-budget
      (sum-list (take n (drop i (query-opening-budgets budgets))) + n)
      (staged-query-program A i n)
  using bound
proof (induction n arbitrary: i)
  case 0
  then show ?case by (simp add: hash-collision-budget-return)
next

```

```

case (Suc n)
have i-bound:  $i < \text{length } (\text{query-opening-budgets } \text{budgets})$ 
  using Suc.premis by simp
have challenge-range: hash-range-budget 1 receive-query-index-challenge
  by (rule hash-range-budget-receive-query-index-challenge)
have challenge-coll: hash-collision-budget 1 receive-query-index-challenge
  by (rule hash-collision-budget-receive-query-index-challenge)
have stage-range:
   $\bigwedge \text{raw. hash-range-budget } (\text{query-opening-budgets } \text{budgets } ! i)$ 
     $(\text{query-opening-stage } A \ i \ \text{raw})$ 
  using controlled i-bound
  unfolding staged-adversary-controlled-def
  by (blast intro: controlled-ro-program-range)
have stage-coll:
   $\bigwedge \text{raw. hash-collision-budget } (\text{query-opening-budgets } \text{budgets } ! i)$ 
     $(\text{query-opening-stage } A \ i \ \text{raw})$ 
  using controlled i-bound
  unfolding staged-adversary-controlled-def
  by (blast intro: controlled-ro-program-collision)
have record-range:
   $\bigwedge \text{chunk. hash-range-budget } 0 \ (\text{record-staged-messages } \text{chunk})$ 
  by (rule hash-range-budget-record-staged-messages)
have record-coll:
   $\bigwedge \text{chunk. hash-collision-budget } 0 \ (\text{record-staged-messages } \text{chunk})$ 
  by (rule hash-collision-budget-record-staged-messages)
have tail-range:
  hash-range-budget
     $(\text{sum-list } (\text{take } n \ (\text{drop } (\text{Suc } i) \ (\text{query-opening-budgets } \text{budgets})))) +$ 
     $n)$ 
     $(\text{staged-query-program } A \ (\text{Suc } i) \ n)$ 
  by (rule hash-range-budget-staged-query-program[OF controlled])
    (use Suc.premis in simp)
have tail-coll:
  hash-collision-budget
     $(\text{sum-list } (\text{take } n \ (\text{drop } (\text{Suc } i) \ (\text{query-opening-budgets } \text{budgets})))) +$ 
     $n)$ 
     $(\text{staged-query-program } A \ (\text{Suc } i) \ n)$ 
  by (rule Suc.IH) (use Suc.premis in simp)
have tail-return-range:
   $\bigwedge \text{chunk. hash-range-budget}$ 
     $(\text{sum-list } (\text{take } n \ (\text{drop } (\text{Suc } i) \ (\text{query-opening-budgets } \text{budgets})))) +$ 
     $n + 0)$ 
     $(\text{staged-query-program } A \ (\text{Suc } i) \ n \gg=$ 
     $(\lambda \text{chunks. return } (\text{chunk } \# \ \text{chunks})))$ 
  by (rule hash-range-budget-bind)
    (rule tail-range, rule hash-range-budget-return)
have tail-return-coll:
   $\bigwedge \text{chunk. hash-collision-budget}$ 
     $(\text{sum-list } (\text{take } n \ (\text{drop } (\text{Suc } i) \ (\text{query-opening-budgets } \text{budgets})))) +$ 

```

```

    n + 0)
  (staged-query-program A (Suc i) n >=>
    (λchunks. return (chunk # chunks)))
by (rule hash-collision-budget-bind)
    (rule tail-range, rule tail-coll, rule hash-range-budget-return,
    rule hash-collision-budget-return)
have after-record-range:
  ∧chunk. hash-range-budget
    (0 +
      (sum-list
        (take n (drop (Suc i) (query-opening-budgets budgets))) +
        n + 0))
    (record-staged-messages chunk >=>
      (λ-. staged-query-program A (Suc i) n >=>
        (λchunks. return (chunk # chunks)))))
by (rule hash-range-budget-bind)
    (rule record-range, rule tail-return-range)
have after-record-coll:
  ∧chunk. hash-collision-budget
    (0 +
      (sum-list
        (take n (drop (Suc i) (query-opening-budgets budgets))) +
        n + 0))
    (record-staged-messages chunk >=>
      (λ-. staged-query-program A (Suc i) n >=>
        (λchunks. return (chunk # chunks)))))
by (rule hash-collision-budget-bind)
    (rule record-range, rule record-coll, rule tail-return-range,
    rule tail-return-coll)
have after-stage-range:
  ∧raw. hash-range-budget
    (query-opening-budgets budgets ! i +
      (0 +
        (sum-list
          (take n (drop (Suc i) (query-opening-budgets budgets))) +
          n + 0)))
    (query-opening-stage A i raw >=>
      (λchunk. record-staged-messages chunk >=>
        (λ-. staged-query-program A (Suc i) n >=>
          (λchunks. return (chunk # chunks)))))
by (rule hash-range-budget-bind)
    (rule stage-range, rule after-record-range)
have after-stage-coll:
  ∧raw. hash-collision-budget
    (query-opening-budgets budgets ! i +
      (0 +
        (sum-list
          (take n (drop (Suc i) (query-opening-budgets budgets))) +
          n + 0)))

```

```

    (query-opening-stage A i raw >>=
      (λchunk. record-staged-messages chunk >>=
        (λ-. staged-query-program A (Suc i) n >>=
          (λchunks. return (chunk # chunks))))))
  by (rule hash-collision-budget-bind)
    (rule stage-range, rule stage-coll, rule after-record-range,
      rule after-record-coll)
  have whole:
    hash-collision-budget
      (1 +
        (query-opening-budgets budgets ! i +
          (0 +
            (sum-list
              (take n (drop (Suc i) (query-opening-budgets budgets))) +
              n + 0))))
        (staged-query-program A i (Suc n))
      )
  unfolding staged-query-program.simps
  by (rule hash-collision-budget-bind)
    (rule challenge-range, rule challenge-coll, rule after-stage-range,
      rule after-stage-coll)
  have budget-eq:
    1 +
      (query-opening-budgets budgets ! i +
        (0 +
          (sum-list
            (take n (drop (Suc i) (query-opening-budgets budgets))) +
            n + 0))) =
      sum-list (take (Suc n) (drop i (query-opening-budgets budgets))) +
      Suc n
  using sum-list-take-Suc-drop[OF i-bound, of n] by simp
  show ?case
    using whole unfolding budget-eq .
qed

lemma hash-target-program-staged-trace-fri-program:
  assumes controlled: staged-adversary-controlled budgets A
    and bound: i + n ≤ length (trace-fri-budgets budgets)
  shows
    hash-target-program B
      (sum-list (take n (drop i (trace-fri-budgets budgets))) + n)
      (staged-trace-fri-program A i n bs)
  using bound
proof (induction n arbitrary: i bs)
  case 0
  then show ?case by (simp add: hash-target-program-return)
next
  case (Suc n)
  have i-bound: i < length (trace-fri-budgets budgets)
  using Suc.premis by simp

```

```

have stage:
  hash-target-program B (trace-fri-budgets budgets ! i)
    (trace-fri-root-stage A i bs)
  using controlled i-bound
  unfolding staged-adversary-controlled-def
  by (blast intro: controlled-ro-program-target)
have record-budget:
   $\wedge$  root. hash-target-program B 0 (record-staged-message root)
  by (rule hash-target-program-record-staged-message)
have challenge: hash-target-program B 1 receive-trace-fri-challenge
  by (rule hash-target-program-receive-trace-fri-challenge)
have tail:
   $\wedge$  b. hash-target-program B
    (sum-list (take n (drop (Suc i) (trace-fri-budgets budgets))) + n)
    (staged-trace-fri-program A (Suc i) n (bs @ [b]))
  by (rule Suc.IH) (use Suc.premis in simp)
have tail-return:
   $\wedge$  b root. hash-target-program B
    (sum-list (take n (drop (Suc i) (trace-fri-budgets budgets))) + n + 0)
    (staged-trace-fri-program A (Suc i) n (bs @ [b])  $\gg$ 
      ( $\lambda$ (roots, bs'). return (root # roots, bs')))
  by (rule hash-target-program-bind)
    (rule tail, simp add: hash-target-program-return split: prod.splits)
have after-challenge:
   $\wedge$  root. hash-target-program B
    (1 +
      (sum-list (take n (drop (Suc i) (trace-fri-budgets budgets))) +
        n + 0))
    (receive-trace-fri-challenge  $\gg$ 
      ( $\lambda$ b. staged-trace-fri-program A (Suc i) n (bs @ [b])  $\gg$ 
        ( $\lambda$ (roots, bs'). return (root # roots, bs')))))
  by (rule hash-target-program-bind)
    (rule challenge, rule tail-return)
have after-record:
   $\wedge$  root. hash-target-program B
    (0 +
      (1 +
        (sum-list (take n (drop (Suc i) (trace-fri-budgets budgets))) +
          n + 0)))
    (record-staged-message root  $\gg$ 
      ( $\lambda$ -. receive-trace-fri-challenge  $\gg$ 
        ( $\lambda$ b. staged-trace-fri-program A (Suc i) n (bs @ [b])  $\gg$ 
          ( $\lambda$ (roots, bs'). return (root # roots, bs')))))
  by (rule hash-target-program-bind)
    (rule record-budget, rule after-challenge)
have whole:
  hash-target-program B
    (trace-fri-budgets budgets ! i +
      (0 +

```

```

      (1 +
        (sum-list (take n (drop (Suc i) (trace-fri-budgets budgets))) +
          n + 0))))
    (staged-trace-fri-program A i (Suc n) bs)
  unfolding staged-trace-fri-program.simps
  by (rule hash-target-program-bind)
    (rule stage, rule after-record)
have budget-eq:
  trace-fri-budgets budgets ! i +
    (0 +
      (1 +
        (sum-list (take n (drop (Suc i) (trace-fri-budgets budgets))) +
          n + 0))) =
    sum-list (take (Suc n) (drop i (trace-fri-budgets budgets))) + Suc n
  using sum-list-take-Suc-drop[OF i-bound, of n] by simp
show ?case
  using whole unfolding budget-eq .
qed

```

```

lemma hash-target-program-staged-alpha-program:
  hash-target-program B n (staged-alpha-program n)
proof (induction n)
  case 0
  then show ?case by (simp add: hash-target-program-return)
next
  case (Suc n)
  have challenge: hash-target-program B 1 receive-alpha-challenge
  by (rule hash-target-program-receive-alpha-challenge)
  have record-budget:
     $\bigwedge a.$  hash-target-program B 0 (record-staged-message a)
  by (rule hash-target-program-record-staged-message)
  have tail-return:
     $\bigwedge a.$  hash-target-program B (n + 0)
      (staged-alpha-program n  $\gg=$  ( $\lambda as.$  return (a # as)))
  by (rule hash-target-program-bind)
    (rule Suc.IH, rule hash-target-program-return)
  have after-record:
     $\bigwedge a.$  hash-target-program B (0 + (n + 0))
      (record-staged-message a  $\gg=$ 
        ( $\lambda -.$  staged-alpha-program n  $\gg=$  ( $\lambda as.$  return (a # as))))
  by (rule hash-target-program-bind)
    (rule record-budget, rule tail-return)
  have hash-target-program B (1 + (0 + (n + 0)))
    (staged-alpha-program (Suc n))
  unfolding staged-alpha-program.simps
  by (rule hash-target-program-bind)
    (rule challenge, rule after-record)
  then show ?case by simp
qed

```


lemma *hash-target-program-staged-composition-fri-program:*
assumes *controlled: staged-adversary-controlled budgets A*
and *bound: $i + n \leq \text{length } (\text{composition-fri-budgets budgets})$*
shows
hash-target-program B
(sum-list (take n (drop i (composition-fri-budgets budgets))) + n)
(staged-composition-fri-program A dg i n bs)
using *bound*
proof (*induction n arbitrary: i bs*)
case 0
then show ?*case* **by** (*simp add: hash-target-program-return*)
next
case (*Suc n*)
have *i-bound: $i < \text{length } (\text{composition-fri-budgets budgets})$*
using *Suc.prem*s **by** *simp*
have *stage:*
hash-target-program B (composition-fri-budgets budgets ! i)
(composition-fri-root-stage A dg i bs)
using *controlled i-bound*
unfolding *staged-adversary-controlled-def*
by (*blast intro: controlled-ro-program-target*)
have *record-budget:*
 $\wedge \text{root. hash-target-program B 0 (record-staged-message root)}$
by (*rule hash-target-program-record-staged-message*)
have *challenge:*
hash-target-program B 1 receive-composition-fri-challenge
by (*rule hash-target-program-receive-composition-fri-challenge*)
have *tail:*
 $\wedge b. \text{hash-target-program B}$
(sum-list
(take n (drop (Suc i) (composition-fri-budgets budgets))) + n)
(staged-composition-fri-program A dg (Suc i) n (bs @ [b]))
by (*rule Suc.IH*) (*use Suc.prem*s **in** *simp*)
have *tail-return:*
 $\wedge b \text{ root. hash-target-program B}$
(sum-list
(take n (drop (Suc i) (composition-fri-budgets budgets))) + n + 0)
(staged-composition-fri-program A dg (Suc i) n (bs @ [b])) $\gg$
($\lambda(\text{roots}, \text{bs}'). \text{return } (\text{root} \# \text{roots}, \text{bs}')$)
by (*rule hash-target-program-bind*)
(rule tail, simp add: hash-target-program-return split: prod.splits)
have *after-challenge:*
 $\wedge \text{root. hash-target-program B}$
(1 +
(sum-list
(take n (drop (Suc i) (composition-fri-budgets budgets))) +
n + 0))
(receive-composition-fri-challenge $\gg$

```

    (λb. staged-composition-fri-program A dg (Suc i) n
      (bs @ [b]) >>=
      (λ(roots, bs'). return (root # roots, bs'))))
  by (rule hash-target-program-bind)
    (rule challenge, rule tail-return)
have after-record:
  ∧root. hash-target-program B
    (0 +
      (1 +
        (sum-list
          (take n (drop (Suc i) (composition-fri-budgets budgets))) +
          n + 0)))
    (record-staged-message root >>=
      (λ-. receive-composition-fri-challenge >>=
        (λb. staged-composition-fri-program A dg (Suc i) n
          (bs @ [b]) >>=
          (λ(roots, bs'). return (root # roots, bs')))))
  by (rule hash-target-program-bind)
    (rule record-budget, rule after-challenge)
have whole:
  hash-target-program B
    (composition-fri-budgets budgets ! i +
      (0 +
        (1 +
          (sum-list
            (take n (drop (Suc i) (composition-fri-budgets budgets))) +
            n + 0))))
    (staged-composition-fri-program A dg i (Suc n) bs)
unfolding staged-composition-fri-program.simps
by (rule hash-target-program-bind)
    (rule stage, rule after-record)
have budget-eq:
  composition-fri-budgets budgets ! i +
    (0 +
      (1 +
        (sum-list
          (take n (drop (Suc i) (composition-fri-budgets budgets))) +
          n + 0))) =
    sum-list (take (Suc n) (drop i (composition-fri-budgets budgets))) +
    Suc n
using sum-list-take-Suc-drop[OF i-bound, of n] by simp
show ?case
using whole unfolding budget-eq .
qed

```

lemma hash-target-program-staged-query-program:
assumes controlled: staged-adversary-controlled budgets A
and bound: $i + n \leq \text{length } (\text{query-opening-budgets } \text{budgets})$
shows

```

    hash-target-program B
      (sum-list (take n (drop i (query-opening-budgets budgets))) + n)
      (staged-query-program A i n)
  using bound
proof (induction n arbitrary: i)
  case 0
  then show ?case by (simp add: hash-target-program-return)
next
  case (Suc n)
  have i-bound: i < length (query-opening-budgets budgets)
    using Suc.prem by simp
  have challenge: hash-target-program B 1 receive-query-index-challenge
    by (rule hash-target-program-receive-query-index-challenge)
  have stage:
     $\bigwedge$ raw. hash-target-program B (query-opening-budgets budgets ! i)
      (query-opening-stage A i raw)
    using controlled i-bound
    unfolding staged-adversary-controlled-def
    by (blast intro: controlled-ro-program-target)
  have record-budget:
     $\bigwedge$ chunk. hash-target-program B 0 (record-staged-messages chunk)
    by (rule hash-target-program-record-staged-messages)
  have tail:
    hash-target-program B
      (sum-list (take n (drop (Suc i) (query-opening-budgets budgets))) +
        n)
      (staged-query-program A (Suc i) n)
    by (rule Suc.IH) (use Suc.prem in simp)
  have tail-return:
     $\bigwedge$ chunk. hash-target-program B
      (sum-list (take n (drop (Suc i) (query-opening-budgets budgets))) +
        n + 0)
      (staged-query-program A (Suc i) n  $\gg$ 
        ( $\lambda$ chunks. return (chunk # chunks)))
    by (rule hash-target-program-bind)
      (rule tail, rule hash-target-program-return)
  have after-record:
     $\bigwedge$ chunk. hash-target-program B
      (0 +
        (sum-list
          (take n (drop (Suc i) (query-opening-budgets budgets))) +
          n + 0))
      (record-staged-messages chunk  $\gg$ 
        ( $\lambda$ -. staged-query-program A (Suc i) n  $\gg$ 
          ( $\lambda$ chunks. return (chunk # chunks)))))
    by (rule hash-target-program-bind)
      (rule record-budget, rule tail-return)
  have after-stage:
     $\bigwedge$ raw. hash-target-program B

```

```

    (query-opening-budgets budgets ! i +
      (0 +
        (sum-list
          (take n (drop (Suc i) (query-opening-budgets budgets))) +
            n + 0)))
    (query-opening-stage A i raw >=>
      (λchunk. record-staged-messages chunk >=>
        (λ-. staged-query-program A (Suc i) n >=>
          (λchunks. return (chunk # chunks)))))
  by (rule hash-target-program-bind)
    (rule stage, rule after-record)
have whole:
  hash-target-program B
    (1 +
      (query-opening-budgets budgets ! i +
        (0 +
          (sum-list
            (take n (drop (Suc i) (query-opening-budgets budgets))) +
              n + 0))))
    (staged-query-program A i (Suc n))
  unfolding staged-query-program.simps
  by (rule hash-target-program-bind)
    (rule challenge, rule after-stage)
have budget-eq:
  1 +
    (query-opening-budgets budgets ! i +
      (0 +
        (sum-list
          (take n (drop (Suc i) (query-opening-budgets budgets))) +
            n + 0))) =
    sum-list (take (Suc n) (drop i (query-opening-budgets budgets))) +
      Suc n
  using sum-list-take-Suc-drop[OF i-bound, of n] by simp
show ?case
  using whole unfolding budget-eq .
qed

lemma hash-target-program-checked-staged-query-program:
  assumes controlled: staged-adversary-controlled budgets A
    and bound: i + n ≤ length (query-opening-budgets budgets)
  shows
    hash-target-program B
      (sum-list (take n (drop i (query-opening-budgets budgets))) + n)
      (checked-staged-query-program A trace-roots composition-roots i n)
  using bound
proof (induction n arbitrary: i)
  case 0
  then show ?case by (simp add: hash-target-program-return)
next

```

```

case (Suc n)
have i-bound:  $i < \text{length } (\text{query-opening-budgets } \text{budgets})$ 
  using Suc.prem by simp
have challenge: hash-target-program B 1 receive-query-index-challenge
  by (rule hash-target-program-receive-query-index-challenge)
have stage:
   $\wedge \text{raw. hash-target-program } B \text{ (query-opening-budgets } \text{budgets } ! i)$ 
    (query-opening-stage A i raw)
  using controlled i-bound
  unfolding staged-adversary-controlled-def
  by (blast intro: controlled-ro-program-target)
have check:
   $\wedge \text{raw chunk. hash-target-program } B \text{ } 0$ 
    (assert
      (verifier-query-round-chunk (index (to-nat raw))
        trace-roots composition-roots chunk))
    by (rule hash-target-program-assert)
have record-budget:
   $\wedge \text{chunk. hash-target-program } B \text{ } 0 \text{ (record-staged-messages chunk)}$ 
  by (rule hash-target-program-record-staged-messages)
have tail:
  hash-target-program B
    (sum-list (take n (drop (Suc i) (query-opening-budgets budgets))) +
      n)
    (checked-staged-query-program A trace-roots composition-roots
      (Suc i) n)
  by (rule Suc.IH) (use Suc.prem in simp)
have tail-return:
   $\wedge \text{chunk. hash-target-program } B$ 
    (sum-list (take n (drop (Suc i) (query-opening-budgets budgets))) +
      n + 0)
    (checked-staged-query-program A trace-roots composition-roots
      (Suc i) n  $\gg=$ 
      ( $\lambda \text{chunks. return (chunk \# chunks)}$ )))
  by (rule hash-target-program-bind)
    (rule tail, rule hash-target-program-return)
have after-record:
   $\wedge \text{chunk. hash-target-program } B$ 
    (0 +
      (sum-list
        (take n (drop (Suc i) (query-opening-budgets budgets))) +
        n + 0))
    (record-staged-messages chunk  $\gg=$ 
      ( $\lambda \cdot \text{checked-staged-query-program } A \text{ trace-roots composition-roots}$ 
        (Suc i) n  $\gg=$ 
        ( $\lambda \text{chunks. return (chunk \# chunks)}$ ))))
  by (rule hash-target-program-bind)
    (rule record-budget, rule tail-return)
have after-check:

```

```

 $\wedge$ raw chunk. hash-target-program B
  (0 +
    (0 +
      (sum-list
        (take n (drop (Suc i) (query-opening-budgets budgets))) +
        n + 0))))
  (assert
    (verifier-query-round-chunk (index (to-nat raw))
      trace-roots composition-roots chunk)  $\gg$ 
    ( $\lambda$ -. record-staged-messages chunk  $\gg$ 
      ( $\lambda$ -. checked-staged-query-program A trace-roots
        composition-roots (Suc i) n  $\gg$ 
        ( $\lambda$ chunks. return (chunk # chunks)))))
  by (rule hash-target-program-bind)
    (rule check, rule after-record)
have after-stage:
 $\wedge$ raw. hash-target-program B
  (query-opening-budgets budgets ! i +
    (0 +
      (0 +
        (sum-list
          (take n (drop (Suc i) (query-opening-budgets budgets))) +
          n + 0))))
  (query-opening-stage A i raw  $\gg$ 
    ( $\lambda$ chunk. assert
      (verifier-query-round-chunk (index (to-nat raw))
        trace-roots composition-roots chunk)  $\gg$ 
      ( $\lambda$ -. record-staged-messages chunk  $\gg$ 
        ( $\lambda$ -. checked-staged-query-program A trace-roots
          composition-roots (Suc i) n  $\gg$ 
          ( $\lambda$ chunks. return (chunk # chunks)))))
  by (rule hash-target-program-bind)
    (rule stage, rule after-check)
have whole:
  hash-target-program B
  (1 +
    (query-opening-budgets budgets ! i +
      (0 +
        (0 +
          (sum-list
            (take n (drop (Suc i) (query-opening-budgets budgets))) +
            n + 0))))
    (checked-staged-query-program A trace-roots composition-roots
      i (Suc n))
  unfolding checked-staged-query-program.simps Let-def
  by (rule hash-target-program-bind)
    (rule challenge, rule after-stage)
have budget-eq:
  1 +

```

```

    (query-opening-budgets budgets ! i +
      (0 +
        (0 +
          (sum-list
            (take n (drop (Suc i) (query-opening-budgets budgets))) +
            n + 0)))) =
    sum-list (take (Suc n) (drop i (query-opening-budgets budgets))) +
    Suc n
  using sum-list-take-Suc-drop[OF i-bound, of n] by simp
show ?case
  using whole unfolding budget-eq .
qed

lemma hash-relation-program-checked-staged-query-program:
  assumes controlled: staged-adversary-controlled budgets A
    and bound:  $i + n \leq \text{length } (\text{query-opening-budgets budgets})$ 
    and fibers:  $\bigwedge x. \text{card } \{y. R \ x \ y\} \leq b$ 
  shows
    hash-relation-program R b
      (sum-list (take n (drop i (query-opening-budgets budgets))) + n)
      (checked-staged-query-program A trace-roots composition-roots i n)
  using bound
proof (induction n arbitrary: i)
  case 0
  then show ?case
    by (simp add:
      hash-relation-program-zero[OF hash-map-preserving-return])
next
  case (Suc n)
  have i-bound:  $i < \text{length } (\text{query-opening-budgets budgets})$ 
    using Suc.prem1 by simp
  have challenge:
    hash-relation-program R b 1 receive-query-index-challenge
  by (rule hash-relation-program-receive-query-index-challenge[OF fibers])
  have stage:
     $\bigwedge \text{raw}. \text{hash-relation-program } R \ b$ 
      (query-opening-budgets budgets ! i)
      (query-opening-stage A i raw)
  proof -
    fix raw
    have stage-controlled:
      controlled-ro-program (query-opening-budgets budgets ! i)
      (query-opening-stage A i raw)
    using controlled i-bound
    unfolding staged-adversary-controlled-def by blast
    show hash-relation-program R b
      (query-opening-budgets budgets ! i)
      (query-opening-stage A i raw)
    by (rule controlled-ro-program-relation

```

```

      [OF stage-controlled fibers])
qed
have check:
   $\bigwedge$ raw chunk. hash-relation-program R b 0
    (assert
      (verifier-query-round-chunk (index (to-nat raw))
        trace-roots composition-roots chunk))
    by (rule hash-relation-program-assert)
have record-budget:
   $\bigwedge$ chunk. hash-relation-program R b 0
    (record-staged-messages chunk)
    by (rule hash-relation-program-record-staged-messages)
have tail:
  hash-relation-program R b
    (sum-list (take n (drop (Suc i) (query-opening-budgets budgets))) +
      n)
    (checked-staged-query-program A trace-roots composition-roots
      (Suc i) n)
    by (rule Suc.IH) (use Suc.premis in simp)
have tail-return:
   $\bigwedge$ chunk. hash-relation-program R b
    (sum-list (take n (drop (Suc i) (query-opening-budgets budgets))) +
      n + 0)
    (checked-staged-query-program A trace-roots composition-roots
      (Suc i) n  $\gg=$ 
      ( $\lambda$ chunks. return (chunk # chunks)))
    by (rule hash-relation-program-bind)
    (rule tail,
      rule hash-relation-program-zero[OF hash-map-preserving-return])
have after-record:
   $\bigwedge$ chunk. hash-relation-program R b
    (0 +
      (sum-list
        (take n (drop (Suc i) (query-opening-budgets budgets))) +
        n + 0))
    (record-staged-messages chunk  $\gg=$ 
      ( $\lambda$ -. checked-staged-query-program A trace-roots composition-roots
        (Suc i) n  $\gg=$ 
        ( $\lambda$ chunks. return (chunk # chunks)))))
    by (rule hash-relation-program-bind)
    (rule record-budget, rule tail-return)
have after-check:
   $\bigwedge$ raw chunk. hash-relation-program R b
    (0 +
      (0 +
        (sum-list
          (take n (drop (Suc i) (query-opening-budgets budgets))) +
          n + 0))))
    (assert

```



```

    (verifier-query-round-chunk (index (to-nat raw))
      trace-roots composition-roots chunk) >>=
    (λ-. record-staged-messages chunk >>=
      (λ-. checked-staged-query-program A trace-roots
        composition-roots (Suc i) n >>=
          (λchunks. return (chunk # chunks))))))
  by (rule hash-relation-program-bind)
    (rule check, rule after-record)
have after-stage:
  ∧raw. hash-relation-program R b
    (query-opening-budgets budgets ! i +
      (0 +
        (0 +
          (sum-list
            (take n (drop (Suc i) (query-opening-budgets budgets))) +
              n + 0))))))
    (query-opening-stage A i raw >>=
      (λchunk. assert
        (verifier-query-round-chunk (index (to-nat raw))
          trace-roots composition-roots chunk) >>=
        (λ-. record-staged-messages chunk >>=
          (λ-. checked-staged-query-program A trace-roots
            composition-roots (Suc i) n >>=
              (λchunks. return (chunk # chunks)))))))
  by (rule hash-relation-program-bind)
    (rule stage, rule after-check)
have whole:
  hash-relation-program R b
    (1 +
      (query-opening-budgets budgets ! i +
        (0 +
          (0 +
            (sum-list
              (take n (drop (Suc i) (query-opening-budgets budgets))) +
                n + 0))))))
    (checked-staged-query-program A trace-roots composition-roots
      i (Suc n))
unfolding checked-staged-query-program.simps Let-def
by (rule hash-relation-program-bind)
    (rule challenge, rule after-stage)
have budget-eq:
  1 +
    (query-opening-budgets budgets ! i +
      (0 +
        (0 +
          (sum-list
            (take n (drop (Suc i) (query-opening-budgets budgets))) +
              n + 0)))))) =
    sum-list (take (Suc n) (drop i (query-opening-budgets budgets))) +

```

```

      Suc n
    using sum-list-take-Suc-drop[OF i-bound, of n] by simp
  show ?case
    using whole unfolding budget-eq .
qed

lemma hash-relation-program-staged-trace-fri-program:
  assumes controlled: staged-adversary-controlled budgets A
    and bound:  $i + n \leq \text{length } (\text{trace-fri-budgets budgets})$ 
    and fibers:  $\bigwedge x. \text{card } \{y. R \ x \ y\} \leq b$ 
  shows
    hash-relation-program R b
      (sum-list (take n (drop i (trace-fri-budgets budgets))) + n)
      (staged-trace-fri-program A i n bs)
  using bound
proof (induction n arbitrary: i bs)
  case 0
  then show ?case
    by (simp add:
        hash-relation-program-zero[OF hash-map-preserving-return])
next
  case (Suc n)
  have i-bound:  $i < \text{length } (\text{trace-fri-budgets budgets})$ 
    using Suc.prem by simp
  have stage:
    hash-relation-program R b (trace-fri-budgets budgets ! i)
    (trace-fri-root-stage A i bs)
  proof -
    have stage-controlled:
      controlled-ro-program (trace-fri-budgets budgets ! i)
      (trace-fri-root-stage A i bs)
    using controlled i-bound
    unfolding staged-adversary-controlled-def by blast
  show ?thesis
    by (rule controlled-ro-program-relation
        [OF stage-controlled fibers])
qed
have record-budget:
   $\bigwedge \text{root}. \text{hash-relation-program } R \ b \ 0 \ (\text{record-staged-message root})$ 
  by (rule hash-relation-program-record-staged-message)
have challenge:
  hash-relation-program R b 1 receive-trace-fri-challenge
  by (rule hash-relation-program-receive-trace-fri-challenge[OF fibers])
have tail:
   $\bigwedge \text{root } b'. \text{hash-relation-program } R \ b$ 
  (sum-list (take n (drop (Suc i) (trace-fri-budgets budgets))) + n)
  (staged-trace-fri-program A (Suc i) n (bs @ [b']))
  by (rule Suc.IH) (use Suc.prem in simp)
have tail-return:

```

```

 $\wedge$ root b'. hash-relation-program R b
  (sum-list (take n (drop (Suc i) (trace-fri-budgets budgets))) +
    n + 0)
  (staged-trace-fri-program A (Suc i) n (bs @ [b'])  $\gg$ 
    ( $\lambda$ (roots, bs'). return (root # roots, bs')))
by (rule hash-relation-program-bind)
  (rule tail,
    simp add: hash-relation-program-zero[OF hash-map-preserving-return]
      split: prod.splits)
have after-challenge:
 $\wedge$ root. hash-relation-program R b
  (1 +
    (sum-list (take n (drop (Suc i) (trace-fri-budgets budgets))) +
      n + 0))
  (receive-trace-fri-challenge  $\gg$ 
    ( $\lambda$ b'. staged-trace-fri-program A (Suc i) n (bs @ [b'])  $\gg$ 
      ( $\lambda$ (roots, bs'). return (root # roots, bs'))))
by (rule hash-relation-program-bind)
  (rule challenge, rule tail-return)
have after-record:
 $\wedge$ root. hash-relation-program R b
  (0 +
    (1 +
      (sum-list (take n (drop (Suc i) (trace-fri-budgets budgets))) +
        n + 0))))
  (record-staged-message root  $\gg$ 
    ( $\lambda$ -. receive-trace-fri-challenge  $\gg$ 
      ( $\lambda$ b'. staged-trace-fri-program A (Suc i) n (bs @ [b'])  $\gg$ 
        ( $\lambda$ (roots, bs'). return (root # roots, bs')))))
by (rule hash-relation-program-bind)
  (rule record-budget, rule after-challenge)
have whole:
  hash-relation-program R b
  (trace-fri-budgets budgets ! i +
    (0 +
      (1 +
        (sum-list (take n (drop (Suc i) (trace-fri-budgets budgets))) +
          n + 0))))
    (staged-trace-fri-program A i (Suc n) bs)
unfolding staged-trace-fri-program.simps
by (rule hash-relation-program-bind)
  (rule stage, rule after-record)
have budget-eq:
  trace-fri-budgets budgets ! i +
    (0 +
      (1 +
        (sum-list (take n (drop (Suc i) (trace-fri-budgets budgets))) +
          n + 0))) =
    sum-list (take (Suc n) (drop i (trace-fri-budgets budgets))) +

```

```

      Suc n
    using sum-list-take-Suc-drop[OF i-bound, of n] by simp
  show ?case
    using whole unfolding budget-eq .
qed

lemma hash-relation-program-staged-alpha-program:
  assumes fibers:  $\bigwedge x. \text{card } \{y. R \ x \ y\} \leq b$ 
  shows hash-relation-program R b n (staged-alpha-program n)
proof (induction n)
  case 0
  then show ?case
    by (simp add:
        hash-relation-program-zero[OF hash-map-preserving-return])
next
  case (Suc n)
  have challenge: hash-relation-program R b 1 receive-alpha-challenge
    by (rule hash-relation-program-receive-alpha-challenge[OF fibers])
  have record-budget:
     $\bigwedge a. \text{hash-relation-program } R \ b \ 0 \ (\text{record-staged-message } a)$ 
    by (rule hash-relation-program-record-staged-message)
  have tail-return:
     $\bigwedge a. \text{hash-relation-program } R \ b \ (n + 0)$ 
     $(\text{staged-alpha-program } n \gg (\lambda as. \text{return } (a \# as)))$ 
    by (rule hash-relation-program-bind)
    (rule Suc.IH,
        rule hash-relation-program-zero[OF hash-map-preserving-return])
  have after-record:
     $\bigwedge a. \text{hash-relation-program } R \ b \ (0 + (n + 0))$ 
     $(\text{record-staged-message } a \gg$ 
     $(\lambda -. \text{staged-alpha-program } n \gg (\lambda as. \text{return } (a \# as))))$ 
    by (rule hash-relation-program-bind)
    (rule record-budget, rule tail-return)
  have hash-relation-program R b (1 + (0 + (n + 0)))
    (staged-alpha-program (Suc n))
    unfolding staged-alpha-program.simps
    by (rule hash-relation-program-bind)
    (rule challenge, rule after-record)
  then show ?case by simp
qed

```

```

lemma hash-relation-program-staged-composition-fri-program:
  assumes controlled: staged-adversary-controlled budgets A
  and bound:  $i + n \leq \text{length } (\text{composition-fri-budgets } \text{budgets})$ 
  and fibers:  $\bigwedge x. \text{card } \{y. R \ x \ y\} \leq b$ 
  shows
    hash-relation-program R b
    (sum-list (take n (drop i (composition-fri-budgets budgets))) + n)
    (staged-composition-fri-program A dg i n bs)

```

```

using bound
proof (induction n arbitrary: i bs)
  case 0
  then show ?case
    by (simp add:
      hash-relation-program-zero[OF hash-map-preserving-return])
next
  case (Suc n)
  have i-bound: i < length (composition-fri-budgets budgets)
    using Suc.premis by simp
  have stage:
    hash-relation-program R b (composition-fri-budgets budgets ! i)
    (composition-fri-root-stage A dg i bs)
  proof –
    have stage-controlled:
      controlled-ro-program (composition-fri-budgets budgets ! i)
      (composition-fri-root-stage A dg i bs)
    using controlled i-bound
    unfolding staged-adversary-controlled-def by blast
  show ?thesis
    by (rule controlled-ro-program-relation
      [OF stage-controlled fibers])
  qed
  have record-budget:
     $\bigwedge \text{root}. \text{hash-relation-program } R \text{ } b \text{ } 0 \text{ (record-staged-message root)}$ 
    by (rule hash-relation-program-record-staged-message)
  have challenge:
    hash-relation-program R b 1 receive-composition-fri-challenge
    by (rule
      hash-relation-program-receive-composition-fri-challenge[OF fibers])
  have tail:
     $\bigwedge \text{root } b'. \text{hash-relation-program } R \text{ } b$ 
    (sum-list
      (take n (drop (Suc i) (composition-fri-budgets budgets))) + n)
    (staged-composition-fri-program A dg (Suc i) n (bs @ [b']))
    by (rule Suc.IH) (use Suc.premis in simp)
  have tail-return:
     $\bigwedge \text{root } b'. \text{hash-relation-program } R \text{ } b$ 
    (sum-list
      (take n (drop (Suc i) (composition-fri-budgets budgets))) + n + 0)
    (staged-composition-fri-program A dg (Suc i) n (bs @ [b']))  $\ggg$ 
    ( $\lambda(\text{roots}, \text{bs}'). \text{return (root \# roots, bs')}$ ))
    by (rule hash-relation-program-bind)
    (rule tail,
      simp add: hash-relation-program-zero[OF hash-map-preserving-return]
      split: prod.splits)
  have after-challenge:
     $\bigwedge \text{root}. \text{hash-relation-program } R \text{ } b$ 
    (1 +

```

```

    (sum-list
      (take n (drop (Suc i) (composition-fri-budgets budgets))) +
      n + 0))
  (receive-composition-fri-challenge >>=
    (λb'. staged-composition-fri-program A dg (Suc i) n
      (bs @ [b']) >>=
        (λ(roots, bs'). return (root # roots, bs')))))
by (rule hash-relation-program-bind)
    (rule challenge, rule tail-return)
have after-record:
  ∧root. hash-relation-program R b
    (0 +
      (1 +
        (sum-list
          (take n (drop (Suc i) (composition-fri-budgets budgets))) +
          n + 0))))
    (record-staged-message root >>=
      (λ-. receive-composition-fri-challenge >>=
        (λb'. staged-composition-fri-program A dg (Suc i) n
          (bs @ [b']) >>=
            (λ(roots, bs'). return (root # roots, bs'))))))
by (rule hash-relation-program-bind)
    (rule record-budget, rule after-challenge)
have whole:
  hash-relation-program R b
    (composition-fri-budgets budgets ! i +
      (0 +
        (1 +
          (sum-list
            (take n (drop (Suc i) (composition-fri-budgets budgets))) +
            n + 0))))
      (staged-composition-fri-program A dg i (Suc n) bs)
unfolding staged-composition-fri-program.simps
by (rule hash-relation-program-bind)
    (rule stage, rule after-record)
have budget-eq:
  composition-fri-budgets budgets ! i +
    (0 +
      (1 +
        (sum-list
          (take n (drop (Suc i) (composition-fri-budgets budgets))) +
          n + 0))) =
    sum-list (take (Suc n) (drop i (composition-fri-budgets budgets))) +
    Suc n
using sum-list-take-Suc-drop[OF i-bound, of n] by simp
show ?case
using whole unfolding budget-eq .
qed

```

lemma *hash-relation-program-guarded-staged-composition-fri-program:*
assumes *wf: staged-budget-wellformed budgets*
and *controlled: staged-adversary-controlled budgets A*
and *fibers: $\bigwedge x. \text{card } \{y. R \ x \ y\} \leq b$*
shows
hash-relation-program R b
(sum-list (composition-fri-budgets budgets) +
ceil-log (maxDegree + 1))
(assert (ceil-log (to-nat dg + 1) ≤ ceil-log (maxDegree + 1))) $\gg$
(λ -. staged-composition-fri-program A dg 0
(ceil-log (to-nat dg + 1)) []))
proof –
let *?B =*
sum-list (composition-fri-budgets budgets) +
ceil-log (maxDegree + 1)
have *hash-relation-program R b (0 + ?B)*
(assert (ceil-log (to-nat dg + 1) ≤ ceil-log (maxDegree + 1))) $\gg$
(λ -. staged-composition-fri-program A dg 0
(ceil-log (to-nat dg + 1)) []))
proof (rule *hash-relation-program-bind-on-outcomes*
[**where** *q=0 and r=?B*])
show *hash-relation-program R b 0*
(assert (ceil-log (to-nat dg + 1) ≤ ceil-log (maxDegree + 1)))
by (rule *hash-relation-program-assert*)
next
fix *s x t*
assume *out:*
Some (x, t) ∈
set-dist
(execute
(assert
(ceil-log (to-nat dg + 1) ≤
ceil-log (maxDegree + 1))) s)
have *round-bound:*
ceil-log (to-nat dg + 1) ≤ ceil-log (maxDegree + 1)
using *out unfolding assert-def*
by (cases *ceil-log (to-nat dg + 1) ≤ ceil-log (maxDegree + 1)*)
(auto simp: throw-no-outcome)
have *len:*
0 + ceil-log (to-nat dg + 1) ≤
length (composition-fri-budgets budgets)
using *round-bound wf unfolding staged-budget-wellformed-def* **by** *simp*
have *exact:*
hash-relation-program R b
(sum-list
(take (ceil-log (to-nat dg + 1))
(drop 0 (composition-fri-budgets budgets))) +
ceil-log (to-nat dg + 1))
(staged-composition-fri-program A dg 0

```

    (ceil-log (to-nat dg + 1)) [])
  by (rule hash-relation-program-staged-composition-fri-program
    [OF controlled len fibers])
have le:
  sum-list
    (take (ceil-log (to-nat dg + 1))
      (drop 0 (composition-fri-budgets budgets))) +
    ceil-log (to-nat dg + 1) ≤
  sum-list (composition-fri-budgets budgets) +
    ceil-log (maxDegree + 1)
using round-bound sum-list-take-le[of ceil-log (to-nat dg + 1)
  composition-fri-budgets budgets]
by simp
show hash-relation-program R b ?B
  (staged-composition-fri-program A dg 0
    (ceil-log (to-nat dg + 1)) [])
  by (rule hash-relation-program-mono[OF le exact])
qed
then show ?thesis by simp
qed

lemma hash-relation-program-staged-alpha-prefix-program:
  assumes wf: staged-budget-wellformed budgets
  and controlled: staged-adversary-controlled budgets A
  and fibers:  $\bigwedge x. \text{card } \{y. R \ x \ y\} \leq b$ 
  shows
    hash-relation-program R b
      (staged-alpha-search-queries budgets 0)
      (staged-alpha-prefix-program A)
proof -
  let ?trace-root = trace-root-budget budgets
  let ?trace-fri =
    sum-list (trace-fri-budgets budgets) + ceil-log clength
  let ?trace-final = trace-final-budget budgets
  have trace-root:
    hash-relation-program R b ?trace-root (trace-root-stage A)
proof -
  have controlled-root:
    controlled-ro-program ?trace-root (trace-root-stage A)
  using controlled unfolding staged-adversary-controlled-def by blast
  show ?thesis
  by (rule controlled-ro-program-relation
    [OF controlled-root fibers])
qed
have record-root:
   $\bigwedge fr. \text{hash-relation-program } R \ b \ 0 \ (\text{record-staged-message } fr)$ 
  by (rule hash-relation-program-record-staged-message)
have trace-fri:
  hash-relation-program R b ?trace-fri

```



```

    (staged-trace-fri-program A 0 (ceil-log clength) [])
  proof -
    have len:
      0 + ceil-log clength ≤ length (trace-fri-budgets budgets)
      using wf unfolding staged-budget-wellformed-def by simp
    have exact:
      hash-relation-program R b
      (sum-list
        (take (ceil-log clength) (drop 0 (trace-fri-budgets budgets))) +
        ceil-log clength)
      (staged-trace-fri-program A 0 (ceil-log clength) [])
    by (rule hash-relation-program-staged-trace-fri-program
      [OF controlled len fibers])
    have le:
      sum-list
        (take (ceil-log clength) (drop 0 (trace-fri-budgets budgets))) +
        ceil-log clength ≤
        sum-list (trace-fri-budgets budgets) + ceil-log clength
      using sum-list-take-le[of ceil-log clength
        trace-fri-budgets budgets]
      by simp
    show ?thesis
      by (rule hash-relation-program-mono[OF le exact])
  qed
  have trace-final:
    ∧ trace-roots trace-bs. hash-relation-program R b ?trace-final
    (trace-final-stage A trace-bs)
  proof -
    fix trace-roots trace-bs
    have controlled-final:
      controlled-ro-program ?trace-final
      (trace-final-stage A trace-bs)
      using controlled unfolding staged-adversary-controlled-def by blast
    show hash-relation-program R b ?trace-final
      (trace-final-stage A trace-bs)
      by (rule controlled-ro-program-relation
        [OF controlled-final fibers])
  qed
  have record-final:
    ∧ trace-final. hash-relation-program R b 0
    (record-staged-message trace-final)
    by (rule hash-relation-program-record-staged-message)
  have after-final-record:
    ∧ fr trace-roots trace-bs trace-final.
    hash-relation-program R b 0
    (record-staged-message trace-final ≫=
      (λ-. return (fr, trace-roots, trace-bs, trace-final)))
  proof -
    fix fr trace-roots trace-bs trace-final

```

```

have hash-relation-program  $R\ b\ (0 + 0)$ 
  (record-staged-message trace-final  $\gg=$ 
    ( $\lambda$ -. return (fr, trace-roots, trace-bs, trace-final)))
by (rule hash-relation-program-bind)
  (rule record-final,
    rule hash-relation-program-zero[OF hash-map-preserving-return])
then show hash-relation-program  $R\ b\ 0$ 
  (record-staged-message trace-final  $\gg=$ 
    ( $\lambda$ -. return (fr, trace-roots, trace-bs, trace-final)))
by simp
qed
have after-final:
 $\bigwedge$ fr trace-roots trace-bs. hash-relation-program  $R\ b$ 
  (?trace-final + 0)
  (trace-final-stage  $A\ trace-bs\ \gg=$ 
    ( $\lambda$ trace-final. record-staged-message trace-final  $\gg=$ 
      ( $\lambda$ -. return (fr, trace-roots, trace-bs, trace-final))))
by (rule hash-relation-program-bind)
  (rule trace-final, rule after-final-record)
have after-trace-fri:
 $\bigwedge$ fr. hash-relation-program  $R\ b$ 
  (?trace-fri + (?trace-final + 0))
  (staged-trace-fri-program  $A\ 0\ (\text{ceil-log}\ clength)\ \square\ \gg=$ 
    ( $\lambda$ (trace-roots, trace-bs).
      trace-final-stage  $A\ trace-bs\ \gg=$ 
        ( $\lambda$ trace-final. record-staged-message trace-final  $\gg=$ 
          ( $\lambda$ -. return (fr, trace-roots, trace-bs, trace-final))))))
proof -
fix fr
have cont:
 $\bigwedge$ x. hash-relation-program  $R\ b\ (?trace-final + 0)$ 
  (case x of (trace-roots, trace-bs)  $\Rightarrow$ 
    trace-final-stage  $A\ trace-bs\ \gg=$ 
      ( $\lambda$ trace-final. record-staged-message trace-final  $\gg=$ 
        ( $\lambda$ -. return (fr, trace-roots, trace-bs, trace-final))))
proof -
fix x
show hash-relation-program  $R\ b\ (?trace-final + 0)$ 
  (case x of (trace-roots, trace-bs)  $\Rightarrow$ 
    trace-final-stage  $A\ trace-bs\ \gg=$ 
      ( $\lambda$ trace-final. record-staged-message trace-final  $\gg=$ 
        ( $\lambda$ -. return (fr, trace-roots, trace-bs, trace-final))))
proof (cases x)
case (Pair trace-roots trace-bs)
have step:
  hash-relation-program  $R\ b\ (?trace-final + 0)$ 
  (trace-final-stage  $A\ trace-bs\ \gg=$ 
    ( $\lambda$ trace-final. record-staged-message trace-final  $\gg=$ 
      ( $\lambda$ -. return (fr, trace-roots, trace-bs, trace-final))))

```

```

      by (rule after-final)
    show ?thesis
      unfolding Pair using step by simp
    qed
  qed
  have hash-relation-program R b (?trace-fri + (?trace-final + 0))
    (staged-trace-fri-program A 0 (ceil-log clength) [] >=>
      (λ(trace-roots, trace-bs).
        trace-final-stage A trace-bs >=>
          (λtrace-final. record-staged-message trace-final >=>
            (λ-. return (fr, trace-roots, trace-bs, trace-final))))))
    by (rule hash-relation-program-bind)
      (rule trace-fri, rule cont)
  then show hash-relation-program R b
    (?trace-fri + (?trace-final + 0))
    (staged-trace-fri-program A 0 (ceil-log clength) [] >=>
      (λ(trace-roots, trace-bs).
        trace-final-stage A trace-bs >=>
          (λtrace-final. record-staged-message trace-final >=>
            (λ-. return (fr, trace-roots, trace-bs, trace-final)))))) .
  qed
  have after-root-record:
    ∧fr. hash-relation-program R b
      (0 + (?trace-fri + (?trace-final + 0)))
      (record-staged-message fr >=>
        (λ-. staged-trace-fri-program A 0 (ceil-log clength) [] >=>
          (λ(trace-roots, trace-bs).
            trace-final-stage A trace-bs >=>
              (λtrace-final. record-staged-message trace-final >=>
                (λ-. return (fr, trace-roots, trace-bs, trace-final))))))
      by (rule hash-relation-program-bind)
        (rule record-root, rule after-trace-fri)
  have whole:
    hash-relation-program R b
      (?trace-root + (0 + (?trace-fri + (?trace-final + 0))))
      (staged-alpha-prefix-program A)
    unfolding staged-alpha-prefix-program-def
    by (rule hash-relation-program-bind)
      (rule trace-root, rule after-root-record)
  show ?thesis
    using whole
    unfolding staged-alpha-search-queries-def
    by (simp add: add.assoc add.commute add.left-commute)
  qed

lemma hash-range-budget-staged-alpha-prefix-program:
  assumes wf: staged-budget-wellformed budgets
    and controlled: staged-adversary-controlled budgets A
  shows

```

```

    hash-range-budget
      (staged-alpha-search-queries budgets 0)
      (staged-alpha-prefix-program A)
  proof -
    let ?trace-root = trace-root-budget budgets
    let ?trace-fri =
      sum-list (trace-fri-budgets budgets) + ceil-log clength
    let ?trace-final = trace-final-budget budgets
    have trace-root:
      hash-range-budget ?trace-root (trace-root-stage A)
      using controlled unfolding staged-adversary-controlled-def
      by (blast intro: controlled-ro-program-range)
    have trace-fri:
      hash-range-budget ?trace-fri
        (staged-trace-fri-program A 0 (ceil-log clength) [])
    proof -
      have len:
        0 + ceil-log clength ≤ length (trace-fri-budgets budgets)
        using wf unfolding staged-budget-wellformed-def by simp
      have exact:
        hash-range-budget
          (sum-list
            (take (ceil-log clength)
              (drop 0 (trace-fri-budgets budgets))) +
            ceil-log clength)
          (staged-trace-fri-program A 0 (ceil-log clength) [])
        by (rule hash-range-budget-staged-trace-fri-program
          [OF controlled len])
      then show ?thesis
        using wf unfolding staged-budget-wellformed-def by simp
    qed
    have trace-final:
      ∧ bs. hash-range-budget ?trace-final (trace-final-stage A bs)
      using controlled unfolding staged-adversary-controlled-def
      by (blast intro: controlled-ro-program-range)
    have after-trace-final-record:
      ∧ fr trace-roots trace-bs trace-final.
        hash-range-budget 0
          (record-staged-message trace-final ≫=
            (λ-. return (fr, trace-roots, trace-bs, trace-final)))
    proof -
      fix fr trace-roots trace-bs trace-final
      have hash-range-budget (0 + 0)
        (record-staged-message trace-final ≫=
          (λ-. return (fr, trace-roots, trace-bs, trace-final)))
      by (rule hash-range-budget-bind)
        (rule hash-range-budget-record-staged-message,
          rule hash-range-budget-return)
    then show

```

```

    hash-range-budget 0
    (record-staged-message trace-final  $\gg$ 
      ( $\lambda$ -. return (fr, trace-roots, trace-bs, trace-final)))
  by simp
qed
have after-trace-final:
   $\wedge$ fr trace-roots trace-bs.
  hash-range-budget (?trace-final + 0)
  (trace-final-stage A trace-bs  $\gg$ 
    ( $\lambda$ trace-final.
      record-staged-message trace-final  $\gg$ 
        ( $\lambda$ -. return (fr, trace-roots, trace-bs, trace-final))))
  by (rule hash-range-budget-bind)
    (rule trace-final, rule after-trace-final-record)
have after-trace-fri:
   $\wedge$ fr. hash-range-budget (?trace-fri + (?trace-final + 0))
  (staged-trace-fri-program A 0 (ceil-log clength) [])  $\gg$ 
  ( $\lambda$ (trace-roots, trace-bs).
    trace-final-stage A trace-bs  $\gg$ 
      ( $\lambda$ trace-final.
        record-staged-message trace-final  $\gg$ 
          ( $\lambda$ -. return (fr, trace-roots, trace-bs, trace-final))))
proof (rule hash-range-budget-bind)
  fix fr
  show hash-range-budget ?trace-fri
    (staged-trace-fri-program A 0 (ceil-log clength) [])
  by (rule trace-fri)
next
  fix fr x
  show hash-range-budget (?trace-final + 0)
    (case x of (trace-roots, trace-bs)  $\Rightarrow$ 
      trace-final-stage A trace-bs  $\gg$ 
        ( $\lambda$ trace-final.
          record-staged-message trace-final  $\gg$ 
            ( $\lambda$ -. return (fr, trace-roots, trace-bs, trace-final))))
proof (cases x)
  case (Pair trace-roots trace-bs)
  have step:
    hash-range-budget (?trace-final + 0)
    (trace-final-stage A trace-bs  $\gg$ 
      ( $\lambda$ trace-final.
        record-staged-message trace-final  $\gg$ 
          ( $\lambda$ -. return (fr, trace-roots, trace-bs, trace-final))))
  by (rule after-trace-final)
  show ?thesis
  unfolding Pair using step by simp
qed
qed
have after-trace-root-record:

```

```

 $\wedge$ fr. hash-range-budget (0 + (?trace-fri + (?trace-final + 0)))
  (record-staged-message fr  $\gg$ 
    ( $\lambda$ -. staged-trace-fri-program A 0 (ceil-log clength) []  $\gg$ 
      ( $\lambda$ (trace-roots, trace-bs).
        trace-final-stage A trace-bs  $\gg$ 
          ( $\lambda$ trace-final.
            record-staged-message trace-final  $\gg$ 
              ( $\lambda$ -. return
                (fr, trace-roots, trace-bs, trace-final))))))
  by (rule hash-range-budget-bind)
    (rule hash-range-budget-record-staged-message, rule after-trace-fri)
have whole:
  hash-range-budget
    (?trace-root + (0 + (?trace-fri + (?trace-final + 0))))
    (staged-alpha-prefix-program A)
  unfolding staged-alpha-prefix-program-def
  by (rule hash-range-budget-bind)
    (rule trace-root, rule after-trace-root-record)
show ?thesis
  using whole
  unfolding staged-alpha-search-queries-def
  by (simp add: add.assoc add.commute add.left-commute)
qed

```

```

lemma hash-collision-budget-staged-alpha-prefix-program:
  assumes wf: staged-budget-wellformed budgets
    and controlled: staged-adversary-controlled budgets A
  shows
    hash-collision-budget
      (staged-alpha-search-queries budgets 0)
      (staged-alpha-prefix-program A)
proof -
  let ?trace-root = trace-root-budget budgets
  let ?trace-fri =
    sum-list (trace-fri-budgets budgets) + ceil-log clength
  let ?trace-final = trace-final-budget budgets
  have trace-root-range:
    hash-range-budget ?trace-root (trace-root-stage A)
  using controlled unfolding staged-adversary-controlled-def
  by (blast intro: controlled-ro-program-range)
  have trace-root-coll:
    hash-collision-budget ?trace-root (trace-root-stage A)
  using controlled unfolding staged-adversary-controlled-def
  by (blast intro: controlled-ro-program-collision)
  have trace-fri-range':
    hash-range-budget ?trace-fri
      (staged-trace-fri-program A 0 (ceil-log clength) [])
proof -
  have len:

```

```

0 + ceil-log clength ≤ length (trace-fri-budgets budgets)
using wf unfolding staged-budget-wellformed-def by simp
have exact:
  hash-range-budget
    (sum-list
      (take (ceil-log clength)
        (drop 0 (trace-fri-budgets budgets))) +
      ceil-log clength)
    (staged-trace-fri-program A 0 (ceil-log clength) [])
  by (rule hash-range-budget-staged-trace-fri-program
    [OF controlled len])
then show ?thesis
  using wf unfolding staged-budget-wellformed-def by simp
qed
have trace-fri-coll:
  hash-collision-budget ?trace-fri
    (staged-trace-fri-program A 0 (ceil-log clength) [])
proof –
  have len:
    0 + ceil-log clength ≤ length (trace-fri-budgets budgets)
    using wf unfolding staged-budget-wellformed-def by simp
  have exact:
    hash-collision-budget
      (sum-list
        (take (ceil-log clength)
          (drop 0 (trace-fri-budgets budgets))) +
        ceil-log clength)
      (staged-trace-fri-program A 0 (ceil-log clength) [])
    by (rule hash-collision-budget-staged-trace-fri-program
      [OF controlled len])
  then show ?thesis
    using wf unfolding staged-budget-wellformed-def by simp
qed
have trace-final-range:
  ∧bs. hash-range-budget ?trace-final (trace-final-stage A bs)
  using controlled unfolding staged-adversary-controlled-def
  by (blast intro: controlled-ro-program-range)
have trace-final-coll:
  ∧bs. hash-collision-budget ?trace-final (trace-final-stage A bs)
  using controlled unfolding staged-adversary-controlled-def
  by (blast intro: controlled-ro-program-collision)
have after-trace-final-record-range:
  ∧fr trace-roots trace-bs trace-final.
  hash-range-budget 0
    (record-staged-message trace-final ≫=
      (λ-. return (fr, trace-roots, trace-bs, trace-final)))
proof –
  fix fr trace-roots trace-bs trace-final
  have hash-range-budget (0 + 0)

```

```

      (record-staged-message trace-final  $\gg$ 
        ( $\lambda$ -. return (fr, trace-roots, trace-bs, trace-final)))
    by (rule hash-range-budget-bind)
      (rule hash-range-budget-record-staged-message,
        rule hash-range-budget-return)
  then show hash-range-budget 0
    (record-staged-message trace-final  $\gg$ 
      ( $\lambda$ -. return (fr, trace-roots, trace-bs, trace-final)))
  by simp
qed
have after-trace-final-record-coll:
   $\wedge$ fr trace-roots trace-bs trace-final.
  hash-collision-budget 0
  (record-staged-message trace-final  $\gg$ 
    ( $\lambda$ -. return (fr, trace-roots, trace-bs, trace-final)))
proof -
  fix fr trace-roots trace-bs trace-final
  have hash-collision-budget (0 + 0)
    (record-staged-message trace-final  $\gg$ 
      ( $\lambda$ -. return (fr, trace-roots, trace-bs, trace-final)))
  by (rule hash-collision-budget-bind)
    (rule hash-range-budget-record-staged-message,
      rule hash-collision-budget-record-staged-message,
      rule hash-range-budget-return, rule hash-collision-budget-return)
  then show hash-collision-budget 0
    (record-staged-message trace-final  $\gg$ 
      ( $\lambda$ -. return (fr, trace-roots, trace-bs, trace-final)))
  by simp
qed
have after-trace-final-range:
   $\wedge$ fr trace-roots trace-bs.
  hash-range-budget (?trace-final + 0)
  (trace-final-stage A trace-bs  $\gg$ 
    ( $\lambda$ trace-final.
      record-staged-message trace-final  $\gg$ 
        ( $\lambda$ -. return (fr, trace-roots, trace-bs, trace-final))))
  by (rule hash-range-budget-bind)
    (rule trace-final-range, rule after-trace-final-record-range)
have after-trace-final-coll:
   $\wedge$ fr trace-roots trace-bs.
  hash-collision-budget (?trace-final + 0)
  (trace-final-stage A trace-bs  $\gg$ 
    ( $\lambda$ trace-final.
      record-staged-message trace-final  $\gg$ 
        ( $\lambda$ -. return (fr, trace-roots, trace-bs, trace-final))))
  by (rule hash-collision-budget-bind)
    (rule trace-final-range, rule trace-final-coll,
      rule after-trace-final-record-range,
      rule after-trace-final-record-coll)

```



```

have after-trace-fri-range:
   $\wedge$ fr. hash-range-budget (?trace-fri + (?trace-final + 0))
    (staged-trace-fri-program A 0 (ceil-log clength) []  $\gg$ 
      ( $\lambda$ (trace-roots, trace-bs).
        trace-final-stage A trace-bs  $\gg$ 
          ( $\lambda$ trace-final.
            record-staged-message trace-final  $\gg$ 
              ( $\lambda$ -. return (fr, trace-roots, trace-bs, trace-final))))))
proof (rule hash-range-budget-bind)
  fix fr
  show hash-range-budget ?trace-fri
    (staged-trace-fri-program A 0 (ceil-log clength) [])
    by (rule trace-fri-range')
next
  fix fr x
  show hash-range-budget (?trace-final + 0)
    (case x of (trace-roots, trace-bs)  $\Rightarrow$ 
      trace-final-stage A trace-bs  $\gg$ 
        ( $\lambda$ trace-final.
          record-staged-message trace-final  $\gg$ 
            ( $\lambda$ -. return (fr, trace-roots, trace-bs, trace-final))))
  proof (cases x)
    case (Pair trace-roots trace-bs)
    have step:
      hash-range-budget (?trace-final + 0)
        (trace-final-stage A trace-bs  $\gg$ 
          ( $\lambda$ trace-final.
            record-staged-message trace-final  $\gg$ 
              ( $\lambda$ -. return (fr, trace-roots, trace-bs, trace-final))))
    by (rule after-trace-final-range)
    show ?thesis
      unfolding Pair using step by simp
  qed
qed
have after-trace-fri-coll:
   $\wedge$ fr. hash-collision-budget (?trace-fri + (?trace-final + 0))
    (staged-trace-fri-program A 0 (ceil-log clength) []  $\gg$ 
      ( $\lambda$ (trace-roots, trace-bs).
        trace-final-stage A trace-bs  $\gg$ 
          ( $\lambda$ trace-final.
            record-staged-message trace-final  $\gg$ 
              ( $\lambda$ -. return (fr, trace-roots, trace-bs, trace-final))))))
proof (rule hash-collision-budget-bind)
  fix fr
  show hash-range-budget ?trace-fri
    (staged-trace-fri-program A 0 (ceil-log clength) [])
    by (rule trace-fri-range')
  show hash-collision-budget ?trace-fri
    (staged-trace-fri-program A 0 (ceil-log clength) [])

```

```

    by (rule trace-fri-coll)
next
fix fr x
show hash-range-budget (?trace-final + 0)
  (case x of (trace-roots, trace-bs)  $\Rightarrow$ 
    trace-final-stage A trace-bs  $\gg=$ 
    ( $\lambda$ trace-final.
      record-staged-message trace-final  $\gg=$ 
      ( $\lambda$ -. return (fr, trace-roots, trace-bs, trace-final))))
proof (cases x)
case (Pair trace-roots trace-bs)
have step:
  hash-range-budget (?trace-final + 0)
  (trace-final-stage A trace-bs  $\gg=$ 
  ( $\lambda$ trace-final.
    record-staged-message trace-final  $\gg=$ 
    ( $\lambda$ -. return (fr, trace-roots, trace-bs, trace-final))))
  by (rule after-trace-final-range)
show ?thesis
  unfolding Pair using step by simp
qed
show hash-collision-budget (?trace-final + 0)
  (case x of (trace-roots, trace-bs)  $\Rightarrow$ 
    trace-final-stage A trace-bs  $\gg=$ 
    ( $\lambda$ trace-final.
      record-staged-message trace-final  $\gg=$ 
      ( $\lambda$ -. return (fr, trace-roots, trace-bs, trace-final))))
proof (cases x)
case (Pair trace-roots trace-bs)
have step:
  hash-collision-budget (?trace-final + 0)
  (trace-final-stage A trace-bs  $\gg=$ 
  ( $\lambda$ trace-final.
    record-staged-message trace-final  $\gg=$ 
    ( $\lambda$ -. return (fr, trace-roots, trace-bs, trace-final))))
  by (rule after-trace-final-coll)
show ?thesis
  unfolding Pair using step by simp
qed
qed
have after-trace-root-record-range:
   $\wedge$ fr. hash-range-budget (0 + (?trace-fri + (?trace-final + 0)))
  (record-staged-message fr  $\gg=$ 
  ( $\lambda$ -. staged-trace-fri-program A 0 (ceil-log clength)  $\square$   $\gg=$ 
  ( $\lambda$ (trace-roots, trace-bs).
    trace-final-stage A trace-bs  $\gg=$ 
    ( $\lambda$ trace-final.
      record-staged-message trace-final  $\gg=$ 
      ( $\lambda$ -. return

```

```

      (fr, trace-roots, trace-bs, trace-final))))))
  by (rule hash-range-budget-bind)
    (rule hash-range-budget-record-staged-message,
      rule after-trace-fri-range)
  have after-trace-root-record-coll:
     $\wedge$ fr. hash-collision-budget
      (0 + (?trace-fri + (?trace-final + 0)))
      (record-staged-message fr  $\gg$ 
        ( $\lambda$ -. staged-trace-fri-program A 0 (ceil-log clength) []  $\gg$ 
          ( $\lambda$ (trace-roots, trace-bs).
            trace-final-stage A trace-bs  $\gg$ 
              ( $\lambda$ trace-final.
                record-staged-message trace-final  $\gg$ 
                  ( $\lambda$ -. return
                    (fr, trace-roots, trace-bs, trace-final))))))
  by (rule hash-collision-budget-bind)
    (rule hash-range-budget-record-staged-message,
      rule hash-collision-budget-record-staged-message,
      rule after-trace-fri-range, rule after-trace-fri-coll)
  have whole:
    hash-collision-budget
      (?trace-root + (0 + (?trace-fri + (?trace-final + 0))))
      (staged-alpha-prefix-program A)
  unfolding staged-alpha-prefix-program-def
  by (rule hash-collision-budget-bind)
    (rule trace-root-range, rule trace-root-coll,
      rule after-trace-root-record-range,
      rule after-trace-root-record-coll)
  show ?thesis
  using whole
  unfolding staged-alpha-search-queries-def
  by (simp add: add.assoc add.commute add.left-commute)
qed

lemma hash-range-budget-checked-staged-query-program:
  assumes controlled: staged-adversary-controlled budgets A
    and bound:  $i + n \leq \text{length } (\text{query-opening-budgets budgets})$ 
  shows
    hash-range-budget
      (sum-list (take n (drop i (query-opening-budgets budgets))) + n)
      (checked-staged-query-program A trace-roots composition-roots i n)
  using bound
  proof (induction n arbitrary: i)
    case 0
    then show ?case by (simp add: hash-range-budget-return)
  next
    case (Suc n)
    have i-bound:  $i < \text{length } (\text{query-opening-budgets budgets})$ 
    using Suc.premis by simp

```

```

have challenge: hash-range-budget 1 receive-query-index-challenge
  by (rule hash-range-budget-receive-query-index-challenge)
have stage:
   $\wedge$ raw. hash-range-budget (query-opening-budgets budgets ! i)
    (query-opening-stage A i raw)
  using controlled i-bound
  unfolding staged-adversary-controlled-def
  by (blast intro: controlled-ro-program-range)
have check:
   $\wedge$ raw chunk. hash-range-budget 0
    (assert
      (verifier-query-round-chunk (index (to-nat raw))
        trace-roots composition-roots chunk))
    by (rule hash-range-budget-assert)
have record-budget:
   $\wedge$ chunk. hash-range-budget 0 (record-staged-messages chunk)
  by (rule hash-range-budget-record-staged-messages)
have tail:
  hash-range-budget
    (sum-list (take n (drop (Suc i) (query-opening-budgets budgets))) +
      n)
    (checked-staged-query-program A trace-roots composition-roots
      (Suc i) n)
  by (rule Suc.IH) (use Suc.prem in simp)
have tail-return:
   $\wedge$ chunk. hash-range-budget
    (sum-list (take n (drop (Suc i) (query-opening-budgets budgets))) +
      n + 0)
    (checked-staged-query-program A trace-roots composition-roots
      (Suc i) n  $\gg$ 
      ( $\lambda$ chunks. return (chunk # chunks)))
  by (rule hash-range-budget-bind)
    (rule tail, rule hash-range-budget-return)
have after-record:
   $\wedge$ chunk. hash-range-budget
    (0 +
      (sum-list
        (take n (drop (Suc i) (query-opening-budgets budgets))) +
        n + 0))
    (record-staged-messages chunk  $\gg$ 
      ( $\lambda$ -. checked-staged-query-program A trace-roots composition-roots
        (Suc i) n  $\gg$ 
        ( $\lambda$ chunks. return (chunk # chunks)))))
  by (rule hash-range-budget-bind)
    (rule record-budget, rule tail-return)
have after-check:
   $\wedge$ raw chunk. hash-range-budget
    (0 +
      (0 +

```

```

      (sum-list
        (take n (drop (Suc i) (query-opening-budgets budgets))) +
        n + 0)))
    (assert
      (verifier-query-round-chunk (index (to-nat raw))
        trace-roots composition-roots chunk) >=>
      (λ-. record-staged-messages chunk >=>
        (λ-. checked-staged-query-program A trace-roots
          composition-roots (Suc i) n >=>
            (λchunks. return (chunk # chunks))))))
  by (rule hash-range-budget-bind)
    (rule check, rule after-record)
have after-stage:
  ∧ raw. hash-range-budget
    (query-opening-budgets budgets ! i +
      (0 +
        (0 +
          (sum-list
            (take n (drop (Suc i) (query-opening-budgets budgets))) +
            n + 0))))))
    (query-opening-stage A i raw >=>
      (λchunk. assert
        (verifier-query-round-chunk (index (to-nat raw))
          trace-roots composition-roots chunk) >=>
        (λ-. record-staged-messages chunk >=>
          (λ-. checked-staged-query-program A trace-roots
            composition-roots (Suc i) n >=>
              (λchunks. return (chunk # chunks))))))
    by (rule hash-range-budget-bind)
      (rule stage, rule after-check)
have whole:
  hash-range-budget
    (1 +
      (query-opening-budgets budgets ! i +
        (0 +
          (0 +
            (sum-list
              (take n (drop (Suc i) (query-opening-budgets budgets))) +
              n + 0))))))
    (checked-staged-query-program A trace-roots composition-roots
      i (Suc n))
unfolding checked-staged-query-program.simps Let-def
by (rule hash-range-budget-bind)
    (rule challenge, rule after-stage)
have budget-eq:
  1 +
    (query-opening-budgets budgets ! i +
      (0 +
        (0 +

```

```

      (sum-list
        (take n (drop (Suc i) (query-opening-budgets budgets))) +
        n + 0))) =
    sum-list (take (Suc n) (drop i (query-opening-budgets budgets))) +
    Suc n
  using sum-list-take-Suc-drop[OF i-bound, of n] by simp
show ?case
  using whole unfolding budget-eq .
qed

lemma hash-collision-budget-checked-staged-query-program:
  assumes controlled: staged-adversary-controlled budgets A
    and bound:  $i + n \leq \text{length } (\text{query-opening-budgets budgets})$ 
  shows
    hash-collision-budget
      (sum-list (take n (drop i (query-opening-budgets budgets))) + n)
      (checked-staged-query-program A trace-roots composition-roots i n)
  using bound
proof (induction n arbitrary: i)
  case 0
  then show ?case by (simp add: hash-collision-budget-return)
next
  case (Suc n)
  have i-bound:  $i < \text{length } (\text{query-opening-budgets budgets})$ 
  using Suc.prem1 by simp
  have challenge-range: hash-range-budget 1 receive-query-index-challenge
  by (rule hash-range-budget-receive-query-index-challenge)
  have challenge-coll: hash-collision-budget 1 receive-query-index-challenge
  by (rule hash-collision-budget-receive-query-index-challenge)
  have stage-range:
     $\bigwedge \text{raw. hash-range-budget } (\text{query-opening-budgets budgets } ! i)$ 
     $(\text{query-opening-stage } A \ i \ \text{raw})$ 
  using controlled i-bound
  unfolding staged-adversary-controlled-def
  by (blast intro: controlled-ro-program-range)
  have stage-coll:
     $\bigwedge \text{raw. hash-collision-budget } (\text{query-opening-budgets budgets } ! i)$ 
     $(\text{query-opening-stage } A \ i \ \text{raw})$ 
  using controlled i-bound
  unfolding staged-adversary-controlled-def
  by (blast intro: controlled-ro-program-collision)
  have check-range:
     $\bigwedge \text{raw chunk. hash-range-budget } 0$ 
    (assert
      (verifier-query-round-chunk (index (to-nat raw))
        trace-roots composition-roots chunk))
  by (rule hash-range-budget-assert)
  have check-coll:
     $\bigwedge \text{raw chunk. hash-collision-budget } 0$ 

```

```

    (assert
      (verifier-query-round-chunk (index (to-nat raw))
        trace-roots composition-roots chunk))
  by (rule hash-collision-budget-assert)
have record-range:
   $\wedge$ chunk. hash-range-budget 0 (record-staged-messages chunk)
  by (rule hash-range-budget-record-staged-messages)
have record-coll:
   $\wedge$ chunk. hash-collision-budget 0 (record-staged-messages chunk)
  by (rule hash-collision-budget-record-staged-messages)
have tail-range:
  hash-range-budget
    (sum-list (take n (drop (Suc i) (query-opening-budgets budgets))) +
      n)
    (checked-staged-query-program A trace-roots composition-roots
      (Suc i) n)
  by (rule hash-range-budget-checked-staged-query-program[OF controlled])
    (use Suc.prem in simp)
have tail-coll:
  hash-collision-budget
    (sum-list (take n (drop (Suc i) (query-opening-budgets budgets))) +
      n)
    (checked-staged-query-program A trace-roots composition-roots
      (Suc i) n)
  by (rule Suc.IH) (use Suc.prem in simp)
have tail-return-range:
   $\wedge$ chunk. hash-range-budget
    (sum-list (take n (drop (Suc i) (query-opening-budgets budgets))) +
      n + 0)
    (checked-staged-query-program A trace-roots composition-roots
      (Suc i) n  $\gg$ 
      ( $\lambda$ chunks. return (chunk # chunks)))
  by (rule hash-range-budget-bind)
    (rule tail-range, rule hash-range-budget-return)
have tail-return-coll:
   $\wedge$ chunk. hash-collision-budget
    (sum-list (take n (drop (Suc i) (query-opening-budgets budgets))) +
      n + 0)
    (checked-staged-query-program A trace-roots composition-roots
      (Suc i) n  $\gg$ 
      ( $\lambda$ chunks. return (chunk # chunks)))
  by (rule hash-collision-budget-bind)
    (rule tail-range, rule tail-coll, rule hash-range-budget-return,
      rule hash-collision-budget-return)
have after-record-range:
   $\wedge$ chunk. hash-range-budget
    (0 +
      (sum-list
        (take n (drop (Suc i) (query-opening-budgets budgets))) +

```

```

    n + 0))
  (record-staged-messages chunk  $\gg$ 
    ( $\lambda$ -. checked-staged-query-program A trace-roots composition-roots
      (Suc i) n  $\gg$ 
      ( $\lambda$ chunks. return (chunk # chunks))))
  by (rule hash-range-budget-bind)
    (rule record-range, rule tail-return-range)
have after-record-coll:
 $\wedge$ chunk. hash-collision-budget
  (0 +
    (sum-list
      (take n (drop (Suc i) (query-opening-budgets budgets))) +
      n + 0))
  (record-staged-messages chunk  $\gg$ 
    ( $\lambda$ -. checked-staged-query-program A trace-roots composition-roots
      (Suc i) n  $\gg$ 
      ( $\lambda$ chunks. return (chunk # chunks))))
  by (rule hash-collision-budget-bind)
    (rule record-range, rule record-coll, rule tail-return-range,
      rule tail-return-coll)
have after-check-range:
 $\wedge$ raw chunk. hash-range-budget
  (0 +
    (0 +
      (sum-list
        (take n (drop (Suc i) (query-opening-budgets budgets))) +
        n + 0))))
  (assert
    (verifier-query-round-chunk (index (to-nat raw))
      trace-roots composition-roots chunk)  $\gg$ 
    ( $\lambda$ -. record-staged-messages chunk  $\gg$ 
      ( $\lambda$ -. checked-staged-query-program A trace-roots
        composition-roots (Suc i) n  $\gg$ 
        ( $\lambda$ chunks. return (chunk # chunks))))))
  by (rule hash-range-budget-bind)
    (rule check-range, rule after-record-range)
have after-check-coll:
 $\wedge$ raw chunk. hash-collision-budget
  (0 +
    (0 +
      (sum-list
        (take n (drop (Suc i) (query-opening-budgets budgets))) +
        n + 0))))
  (assert
    (verifier-query-round-chunk (index (to-nat raw))
      trace-roots composition-roots chunk)  $\gg$ 
    ( $\lambda$ -. record-staged-messages chunk  $\gg$ 
      ( $\lambda$ -. checked-staged-query-program A trace-roots
        composition-roots (Suc i) n  $\gg$ 
        ( $\lambda$ chunks. return (chunk # chunks))))))

```



```

      (λchunks. return (chunk # chunks))))))
  by (rule hash-collision-budget-bind)
    (rule check-range, rule check-coll, rule after-record-range,
     rule after-record-coll)
  have after-stage-range:
    ∧raw. hash-range-budget
      (query-opening-budgets budgets ! i +
       (0 +
        (0 +
         (sum-list
          (take n (drop (Suc i) (query-opening-budgets budgets))) +
           n + 0))))))
      (query-opening-stage A i raw >=>
       (λchunk. assert
        (verifier-query-round-chunk (index (to-nat raw))
         trace-roots composition-roots chunk) >=>
        (λ-. record-staged-messages chunk >=>
         (λ-. checked-staged-query-program A trace-roots
          composition-roots (Suc i) n >=>
           (λchunks. return (chunk # chunks)))))))
  by (rule hash-range-budget-bind)
    (rule stage-range, rule after-check-range)
  have after-stage-coll:
    ∧raw. hash-collision-budget
      (query-opening-budgets budgets ! i +
       (0 +
        (0 +
         (sum-list
          (take n (drop (Suc i) (query-opening-budgets budgets))) +
           n + 0))))))
      (query-opening-stage A i raw >=>
       (λchunk. assert
        (verifier-query-round-chunk (index (to-nat raw))
         trace-roots composition-roots chunk) >=>
        (λ-. record-staged-messages chunk >=>
         (λ-. checked-staged-query-program A trace-roots
          composition-roots (Suc i) n >=>
           (λchunks. return (chunk # chunks)))))))
  by (rule hash-collision-budget-bind)
    (rule stage-range, rule stage-coll, rule after-check-range,
     rule after-check-coll)
  have whole:
    hash-collision-budget
      (1 +
       (query-opening-budgets budgets ! i +
        (0 +
         (0 +
          (sum-list
           (take n (drop (Suc i) (query-opening-budgets budgets))) +
            n + 0))))))

```

```

      n + 0))))))
    (checked-staged-query-program A trace-roots composition-roots
      i (Suc n))
  unfolding checked-staged-query-program.simps Let-def
  by (rule hash-collision-budget-bind)
    (rule challenge-range, rule challenge-coll, rule after-stage-range,
      rule after-stage-coll)
  have budget-eq:
    1 +
      (query-opening-budgets budgets ! i +
        (0 +
          (0 +
            (sum-list
              (take n (drop (Suc i) (query-opening-budgets budgets))) +
              n + 0)))) =
      sum-list (take (Suc n) (drop i (query-opening-budgets budgets))) +
        Suc n
  using sum-list-take-Suc-drop[OF i-bound, of n] by simp
  show ?case
    using whole unfolding budget-eq .
qed

lemma hash-range-budget-guarded-staged-composition-fri-program:
  assumes wf: staged-budget-wellformed budgets
  and controlled: staged-adversary-controlled budgets A
  shows
    hash-range-budget
      (sum-list (composition-fri-budgets budgets) +
        ceil-log (maxDegree + 1))
      (assert (ceil-log (to-nat dg + 1) ≤ ceil-log (maxDegree + 1)) ≫=
        (λ-. staged-composition-fri-program A dg 0
          (ceil-log (to-nat dg + 1)) []))
  proof -
    let ?B =
      sum-list (composition-fri-budgets budgets) +
        ceil-log (maxDegree + 1)
    have hash-range-budget (0 + ?B)
      (assert (ceil-log (to-nat dg + 1) ≤ ceil-log (maxDegree + 1)) ≫=
        (λ-. staged-composition-fri-program A dg 0
          (ceil-log (to-nat dg + 1)) []))
    proof (rule hash-range-budget-bind-on-outcomes
      [where n=0 and n'=?B])
      show hash-range-budget 0
        (assert (ceil-log (to-nat dg + 1) ≤ ceil-log (maxDegree + 1)))
      unfolding hash-range-budget-def assert-def
      by (auto simp: throw-no-outcome)
    next
      fix s x t
      assume out:

```

```

Some (x, t) ∈
  set-dist
  (execute
    (assert
      (ceil-log (to-nat dg + 1) ≤
        ceil-log (maxDegree + 1))) s)
have round-bound:
  ceil-log (to-nat dg + 1) ≤ ceil-log (maxDegree + 1)
using out unfolding assert-def
by (cases ceil-log (to-nat dg + 1) ≤ ceil-log (maxDegree + 1))
  (auto simp: throw-no-outcome)
have len:
  0 + ceil-log (to-nat dg + 1) ≤
    length (composition-fri-budgets budgets)
using round-bound wf unfolding staged-budget-wellformed-def by simp
have exact:
  hash-range-budget
  (sum-list
    (take (ceil-log (to-nat dg + 1))
      (drop 0 (composition-fri-budgets budgets))) +
    ceil-log (to-nat dg + 1))
  (staged-composition-fri-program A dg 0
    (ceil-log (to-nat dg + 1)) [])
by (rule hash-range-budget-staged-composition-fri-program
  [OF controlled len])
have le:
  sum-list
    (take (ceil-log (to-nat dg + 1))
      (drop 0 (composition-fri-budgets budgets))) +
    ceil-log (to-nat dg + 1) ≤
  sum-list (composition-fri-budgets budgets) +
    ceil-log (maxDegree + 1)
using round-bound sum-list-take-le[of ceil-log (to-nat dg + 1)
  composition-fri-budgets budgets]
by simp
show hash-range-budget
  (sum-list (composition-fri-budgets budgets) +
    ceil-log (maxDegree + 1))
  (staged-composition-fri-program A dg 0
    (ceil-log (to-nat dg + 1)) [])
by (rule hash-range-budget-mono[OF le exact])
qed
then show ?thesis by simp
qed

lemma hash-collision-budget-guarded-staged-composition-fri-program:
assumes wf: staged-budget-wellformed budgets
and controlled: staged-adversary-controlled budgets A
shows

```

```

hash-collision-budget
  (sum-list (composition-fri-budgets budgets) +
    ceil-log (maxDegree + 1))
  (assert (ceil-log (to-nat dg + 1) ≤ ceil-log (maxDegree + 1)) >>=
    (λ-. staged-composition-fri-program A dg 0
      (ceil-log (to-nat dg + 1)) []))
proof -
  let ?B =
    sum-list (composition-fri-budgets budgets) +
    ceil-log (maxDegree + 1)
  have hash-collision-budget (0 + ?B)
    (assert (ceil-log (to-nat dg + 1) ≤ ceil-log (maxDegree + 1)) >>=
      (λ-. staged-composition-fri-program A dg 0
        (ceil-log (to-nat dg + 1)) []))
  proof (rule hash-collision-budget-bind-on-outcomes
    [where n=0 and n'=?B])
  show hash-range-budget 0
    (assert (ceil-log (to-nat dg + 1) ≤ ceil-log (maxDegree + 1)))
  unfolding hash-range-budget-def assert-def
  by (auto simp: throw-no-outcome)
  show hash-collision-budget 0
    (assert (ceil-log (to-nat dg + 1) ≤ ceil-log (maxDegree + 1)))
  unfolding hash-collision-budget-def hash-new-collision-event-def
    hash-map-new-output-collision-def hash-collision-budget-value-def
    assert-def
  by (simp add: wp-event-def wpsimps)
next
fix s x t
assume out:
  Some (x, t) ∈
    set-dist
      (execute
        (assert
          (ceil-log (to-nat dg + 1) ≤
            ceil-log (maxDegree + 1)))) s)
have round-bound:
  ceil-log (to-nat dg + 1) ≤ ceil-log (maxDegree + 1)
using out unfolding assert-def
by (cases ceil-log (to-nat dg + 1) ≤ ceil-log (maxDegree + 1))
  (auto simp: throw-no-outcome)
have len:
  0 + ceil-log (to-nat dg + 1) ≤
    length (composition-fri-budgets budgets)
using round-bound wf unfolding staged-budget-wellformed-def by simp
have exact:
  hash-range-budget
    (sum-list
      (take (ceil-log (to-nat dg + 1))
        (drop 0 (composition-fri-budgets budgets)))) +

```

```

      ceil-log (to-nat dg + 1))
    (staged-composition-fri-program A dg 0
      (ceil-log (to-nat dg + 1)) [])
  by (rule hash-range-budget-staged-composition-fri-program
    [OF controlled len])
have le:
  sum-list
    (take (ceil-log (to-nat dg + 1))
      (drop 0 (composition-fri-budgets budgets))) +
    ceil-log (to-nat dg + 1) ≤
  sum-list (composition-fri-budgets budgets) +
    ceil-log (maxDegree + 1)
  using round-bound sum-list-take-le[of ceil-log (to-nat dg + 1)
    composition-fri-budgets budgets]
  by simp
show hash-range-budget
  (sum-list (composition-fri-budgets budgets) +
    ceil-log (maxDegree + 1))
  (staged-composition-fri-program A dg 0
    (ceil-log (to-nat dg + 1)) [])
  by (rule hash-range-budget-mono[OF le exact])
next
fix s x t
assume out:
  Some (x, t) ∈
    set-dist
      (execute
        (assert
          (ceil-log (to-nat dg + 1) ≤
            ceil-log (maxDegree + 1)))) s)
have round-bound:
  ceil-log (to-nat dg + 1) ≤ ceil-log (maxDegree + 1)
  using out unfolding assert-def
  by (cases ceil-log (to-nat dg + 1) ≤ ceil-log (maxDegree + 1))
    (auto simp: throw-no-outcome)
have len:
  0 + ceil-log (to-nat dg + 1) ≤
    length (composition-fri-budgets budgets)
  using round-bound wf unfolding staged-budget-wellformed-def by simp
have exact:
  hash-collision-budget
    (sum-list
      (take (ceil-log (to-nat dg + 1))
        (drop 0 (composition-fri-budgets budgets))) +
      ceil-log (to-nat dg + 1))
    (staged-composition-fri-program A dg 0
      (ceil-log (to-nat dg + 1)) [])
  by (rule hash-collision-budget-staged-composition-fri-program
    [OF controlled len])

```

```

have le:
  sum-list
    (take (ceil-log (to-nat dg + 1))
      (drop 0 (composition-fri-budgets budgets))) +
    ceil-log (to-nat dg + 1) ≤
  sum-list (composition-fri-budgets budgets) +
    ceil-log (maxDegree + 1)
using round-bound sum-list-take-le[of ceil-log (to-nat dg + 1)
  composition-fri-budgets budgets]
by simp
show hash-collision-budget
  (sum-list (composition-fri-budgets budgets) +
    ceil-log (maxDegree + 1))
  (staged-composition-fri-program A dg 0
    (ceil-log (to-nat dg + 1)) [])
by (rule hash-collision-budget-mono[OF le exact])
qed
then show ?thesis by simp
qed

lemma hash-target-program-guarded-staged-composition-fri-program:
assumes wf: staged-budget-wellformed budgets
and controlled: staged-adversary-controlled budgets A
shows
  hash-target-program B
    (sum-list (composition-fri-budgets budgets) +
      ceil-log (maxDegree + 1))
    (assert (ceil-log (to-nat dg + 1) ≤ ceil-log (maxDegree + 1)) ≫=
      (λ-. staged-composition-fri-program A dg 0
        (ceil-log (to-nat dg + 1)) []))
proof -
let ?B =
  sum-list (composition-fri-budgets budgets) +
    ceil-log (maxDegree + 1)
have hash-target-program B (0 + ?B)
  (assert (ceil-log (to-nat dg + 1) ≤ ceil-log (maxDegree + 1)) ≫=
    (λ-. staged-composition-fri-program A dg 0
      (ceil-log (to-nat dg + 1)) []))
proof (rule hash-target-program-bind-on-outcomes
  [where n=0 and n'=?B])
show hash-target-program B 0
  (assert (ceil-log (to-nat dg + 1) ≤ ceil-log (maxDegree + 1)))
by (rule hash-target-program-assert)
next
fix s x t
assume out:
  Some (x, t) ∈
    set-dist
      (execute

```

```

      (assert
        (ceil-log (to-nat dg + 1) ≤
          ceil-log (maxDegree + 1))) s)
  have round-bound:
    ceil-log (to-nat dg + 1) ≤ ceil-log (maxDegree + 1)
  using out unfolding assert-def
  by (cases ceil-log (to-nat dg + 1) ≤ ceil-log (maxDegree + 1))
    (auto simp: throw-no-outcome)
  have len:
    0 + ceil-log (to-nat dg + 1) ≤
      length (composition-fri-budgets budgets)
  using round-bound wf unfolding staged-budget-wellformed-def by simp
  have exact:
    hash-target-program B
      (sum-list
        (take (ceil-log (to-nat dg + 1))
          (drop 0 (composition-fri-budgets budgets)))) +
        ceil-log (to-nat dg + 1))
      (staged-composition-fri-program A dg 0
        (ceil-log (to-nat dg + 1)) [])
  by (rule hash-target-program-staged-composition-fri-program
    [OF controlled len])
  have le:
    sum-list
      (take (ceil-log (to-nat dg + 1))
        (drop 0 (composition-fri-budgets budgets))) +
        ceil-log (to-nat dg + 1) ≤
      sum-list (composition-fri-budgets budgets) +
        ceil-log (maxDegree + 1)
  using round-bound sum-list-take-le[of ceil-log (to-nat dg + 1)
    composition-fri-budgets budgets]
  by simp
  show hash-target-program B
    (sum-list (composition-fri-budgets budgets) +
      ceil-log (maxDegree + 1))
    (staged-composition-fri-program A dg 0
      (ceil-log (to-nat dg + 1)) [])
  by (rule hash-target-program-mono[OF le exact])
qed
then show ?thesis by simp
qed

lemma hash-target-program-staged-alpha-prefix-program:
  assumes wf: staged-budget-wellformed budgets
  and controlled: staged-adversary-controlled budgets A
  shows
    hash-target-program B
      (staged-alpha-search-queries budgets 0)
      (staged-alpha-prefix-program A)

```

```

proof –
  let ?trace-root = trace-root-budget budgets
  let ?trace-fri =
    sum-list (trace-fri-budgets budgets) + ceil-log length
  let ?trace-final = trace-final-budget budgets
  have trace-root-target:
    hash-target-program B ?trace-root (trace-root-stage A)
    using controlled unfolding staged-adversary-controlled-def
    by (blast intro: controlled-ro-program-target)
  have trace-fri-target:
    hash-target-program B ?trace-fri
    (staged-trace-fri-program A 0 (ceil-log length) [])
  proof –
    have len:
      0 + ceil-log length ≤ length (trace-fri-budgets budgets)
      using wf unfolding staged-budget-wellformed-def by simp
    have exact:
      hash-target-program B
      (sum-list
        (take (ceil-log length)
          (drop 0 (trace-fri-budgets budgets))) +
        ceil-log length)
      (staged-trace-fri-program A 0 (ceil-log length) [])
      by (rule hash-target-program-staged-trace-fri-program
        [OF controlled len])
    then show ?thesis
      using wf unfolding staged-budget-wellformed-def by simp
  qed
  have trace-final-target:
    ∧bs. hash-target-program B ?trace-final (trace-final-stage A bs)
    using controlled unfolding staged-adversary-controlled-def
    by (blast intro: controlled-ro-program-target)
  have after-trace-final-record:
    ∧fr trace-roots trace-bs trace-final.
    hash-target-program B 0
    (record-staged-message trace-final ≫=
      (λ-. return (fr, trace-roots, trace-bs, trace-final)))
  proof –
    fix fr trace-roots trace-bs trace-final
    have hash-target-program B (0 + 0)
      (record-staged-message trace-final ≫=
        (λ-. return (fr, trace-roots, trace-bs, trace-final)))
    by (rule hash-target-program-bind)
      (rule hash-target-program-record-staged-message,
        rule hash-target-program-return)
    then show
      hash-target-program B 0
      (record-staged-message trace-final ≫=
        (λ-. return (fr, trace-roots, trace-bs, trace-final)))

```



```

    by simp
qed
have after-trace-final:
   $\bigwedge fr$  trace-roots trace-bs.
    hash-target-program B (?trace-final + 0)
      (trace-final-stage A trace-bs  $\gg=$ 
        ( $\lambda$ trace-final.
          record-staged-message trace-final  $\gg=$ 
            ( $\lambda$ -. return (fr, trace-roots, trace-bs, trace-final))))
  by (rule hash-target-program-bind)
    (rule trace-final-target, rule after-trace-final-record)
have after-trace-fri:
   $\bigwedge fr$ . hash-target-program B (?trace-fri + (?trace-final + 0))
    (staged-trace-fri-program A 0 (ceil-log clength) [])  $\gg=$ 
      ( $\lambda$ (trace-roots, trace-bs).
        trace-final-stage A trace-bs  $\gg=$ 
          ( $\lambda$ trace-final.
            record-staged-message trace-final  $\gg=$ 
              ( $\lambda$ -. return (fr, trace-roots, trace-bs, trace-final))))))
proof (rule hash-target-program-bind)
  fix fr
  show hash-target-program B ?trace-fri
    (staged-trace-fri-program A 0 (ceil-log clength) [])
  by (rule trace-fri-target)
next
  fix fr x
  show hash-target-program B (?trace-final + 0)
    (case x of (trace-roots, trace-bs)  $\Rightarrow$ 
      trace-final-stage A trace-bs  $\gg=$ 
        ( $\lambda$ trace-final.
          record-staged-message trace-final  $\gg=$ 
            ( $\lambda$ -. return (fr, trace-roots, trace-bs, trace-final))))
proof (cases x)
  case (Pair trace-roots trace-bs)
  have target:
    hash-target-program B (?trace-final + 0)
      (trace-final-stage A trace-bs  $\gg=$ 
        ( $\lambda$ trace-final.
          record-staged-message trace-final  $\gg=$ 
            ( $\lambda$ -. return
              (fr, trace-roots, trace-bs, trace-final))))
  by (rule hash-target-program-bind)
    (rule trace-final-target, rule after-trace-final-record)
  show ?thesis
  unfolding Pair using target by simp
qed
qed
have after-trace-root-record:
   $\bigwedge fr$ . hash-target-program B (0 + (?trace-fri + (?trace-final + 0)))

```

```

    (record-staged-message fr  $\gg$ 
      ( $\lambda$ -. staged-trace-fri-program A 0 (ceil-log clength) []  $\gg$ 
        ( $\lambda$ (trace-roots, trace-bs).
          trace-final-stage A trace-bs  $\gg$ 
            ( $\lambda$ trace-final.
              record-staged-message trace-final  $\gg$ 
                ( $\lambda$ -. return
                  (fr, trace-roots, trace-bs, trace-final))))))
  by (rule hash-target-program-bind)
  (rule hash-target-program-record-staged-message, rule after-trace-fri)
have whole:
  hash-target-program B
    (?trace-root + (0 + (?trace-fri + (?trace-final + 0))))
    (staged-alpha-prefix-program A)
  unfolding staged-alpha-prefix-program-def
  by (rule hash-target-program-bind)
  (rule trace-root-target, rule after-trace-root-record)
show ?thesis
  using whole
  unfolding staged-alpha-search-queries-def
  by (simp add: add.assoc add.commute add.left-commute)
qed

lemma staged-alpha-prefix-target-hit-bound:
  assumes wf: staged-budget-wellformed budgets
  and controlled: staged-adversary-controlled budgets A
  shows
    wp-event (staged-alpha-prefix-program A)
      (hash-new-output-hit-event B s) s  $\leq$ 
      staged-phase-target-error B
      (staged-alpha-search-queries budgets 0)
proof -
  have budget:
    hash-target-budget B (staged-alpha-search-queries budgets 0)
    (staged-alpha-prefix-program A)
  using hash-target-program-staged-alpha-prefix-program[OF wf controlled]
  unfolding hash-target-program-def by simp
  show ?thesis
    using budget unfolding hash-target-budget-def
      staged-phase-target-error-def by blast
qed

lemma hash-target-program-staged-alpha-challenge-prefix-program:
  assumes wf: staged-budget-wellformed budgets
  and controlled: staged-adversary-controlled budgets A
  and i-bound: i  $\leq$  length spec
  shows
    hash-target-program B (staged-alpha-search-queries budgets i)
    (staged-alpha-challenge-prefix-program A i)

```

```

proof –
  let ?alpha-prefix = staged-alpha-search-queries budgets 0
  have prefix-target:
    hash-target-program B ?alpha-prefix (staged-alpha-prefix-program A)
    by (rule hash-target-program-staged-alpha-prefix-program[OF wf controlled])
  have alpha-target:
     $\bigwedge$ prefix. hash-target-program B i
      (staged-alpha-program i  $\gg$ 
        ( $\lambda$ as-prefix. return (prefix, as-prefix)))
  proof –
    fix prefix
    have hash-target-program B (i + 0)
      (staged-alpha-program i  $\gg$ 
        ( $\lambda$ as-prefix. return (prefix, as-prefix)))
    by (rule hash-target-program-bind)
      (rule hash-target-program-staged-alpha-program,
        rule hash-target-program-return)
    then show hash-target-program B i
      (staged-alpha-program i  $\gg$ 
        ( $\lambda$ as-prefix. return (prefix, as-prefix)))
    by simp
  qed
  have whole:
    hash-target-program B (?alpha-prefix + i)
    (staged-alpha-challenge-prefix-program A i)
    unfolding staged-alpha-challenge-prefix-program-def
    by (rule hash-target-program-bind[OF prefix-target alpha-target])
  show ?thesis
    using whole i-bound
    unfolding staged-alpha-search-queries-def
    by simp
qed

lemma staged-alpha-prefix-target-hit-bound-i:
  assumes wf: staged-budget-wellformed budgets
  and controlled: staged-adversary-controlled budgets A
  and i-bound:  $i \leq \text{length spec}$ 
  shows
    wp-event (staged-alpha-challenge-prefix-program A i)
      (hash-new-output-hit-event B s) s  $\leq$ 
      staged-phase-target-error B
      (staged-alpha-search-queries budgets i)
proof –
  have budget:
    hash-target-budget B (staged-alpha-search-queries budgets i)
    (staged-alpha-challenge-prefix-program A i)
  using hash-target-program-staged-alpha-challenge-prefix-program
    [OF wf controlled i-bound]
  unfolding hash-target-program-def by simp

```

```

show ?thesis
  using budget unfolding hash-target-budget-def
    staged-phase-target-error-def by blast
qed

lemma hash-target-program-staged-composition-fri-vector-program:
  assumes wf: staged-budget-wellformed budgets
  and controlled: staged-adversary-controlled budgets A
  shows
    hash-target-program B
      (staged-composition-fri-vector-search-queries budgets)
      (staged-composition-fri-vector-program A)
proof –
  let ?alpha-prefix = staged-alpha-search-queries budgets 0
  let ?alpha = length spec
  let ?degree = degree-budget budgets
  let ?composition =
    sum-list (composition-fri-budgets budgets) +
    ceil-log (maxDegree + 1)
  have alpha-prefix-target:
    hash-target-program B ?alpha-prefix (staged-alpha-prefix-program A)
  by (rule hash-target-program-staged-alpha-prefix-program[OF wf controlled])
  have alpha-target:
    hash-target-program B ?alpha (staged-alpha-program (length spec))
  by (rule hash-target-program-staged-alpha-program)
  have degree-target:
     $\wedge$ as. hash-target-program B ?degree (degree-stage A as)
  using controlled unfolding staged-adversary-controlled-def
  by (blast intro: controlled-ro-program-target)
  have guarded-return:
     $\wedge$ dg. hash-target-program B (?composition + 0)
    ((assert
      (ceil-log (to-nat dg + 1)  $\leq$  ceil-log (maxDegree + 1))  $\gg=$ 
      ( $\lambda$ -. staged-composition-fri-program A dg 0
        (ceil-log (to-nat dg + 1)) []))  $\gg=$ 
      ( $\lambda$ x. case x of (-, composition-bs)  $\Rightarrow$ 
        return (dg, composition-bs)))
  by (rule hash-target-program-bind)
    (rule hash-target-program-guarded-staged-composition-fri-program
      [OF wf controlled], simp add: hash-target-program-return
      split: prod.splits)
  have after-record:
     $\wedge$ dg. hash-target-program B (0 + (?composition + 0))
    (record-staged-message dg  $\gg=$ 
      ( $\lambda$ -. (assert
        (ceil-log (to-nat dg + 1)  $\leq$  ceil-log (maxDegree + 1))  $\gg=$ 
        ( $\lambda$ -. staged-composition-fri-program A dg 0
          (ceil-log (to-nat dg + 1)) []))  $\gg=$ 
        ( $\lambda$ x. case x of (-, composition-bs)  $\Rightarrow$ 

```

```

      return (dg, composition-bs))))
  by (rule hash-target-program-bind)
    (rule hash-target-program-record-staged-message, rule guarded-return)
have after-degree:
 $\wedge$ as. hash-target-program B
  (?degree + (0 + (?composition + 0)))
  (degree-stage A as  $\gg$ 
    ( $\lambda$ dg. record-staged-message dg  $\gg$ 
      ( $\lambda$ -. let composition-rounds = ceil-log (to-nat dg + 1) in
        assert (composition-rounds  $\leq$  ceil-log (maxDegree + 1))  $\gg$ 
          ( $\lambda$ -. staged-composition-fri-program A dg 0
            composition-rounds  $\square$   $\gg$ 
              ( $\lambda$ (-, composition-bs). return (dg, composition-bs)))))))
  unfolding Let-def
proof (rule hash-target-program-bind)
  fix as
  show hash-target-program B ?degree (degree-stage A as)
    by (rule degree-target)
next
  fix as dg
  show hash-target-program B (0 + (?composition + 0))
    (record-staged-message dg  $\gg$ 
      ( $\lambda$ -. assert
        (ceil-log (to-nat dg + 1)  $\leq$  ceil-log (maxDegree + 1))  $\gg$ 
          ( $\lambda$ -. staged-composition-fri-program A dg 0
            (ceil-log (to-nat dg + 1))  $\square$   $\gg$ 
              ( $\lambda$ (-, composition-bs). return (dg, composition-bs))))))
    using after-record[of dg] by (simp add: sm-bind-assoc)
qed
have after-alpha:
 $\wedge$ x. hash-target-program B
  (?alpha + (?degree + (0 + (?composition + 0))))
  (staged-alpha-program (length spec)  $\gg$ 
    ( $\lambda$ as. degree-stage A as  $\gg$ 
      ( $\lambda$ dg. record-staged-message dg  $\gg$ 
        ( $\lambda$ -. let composition-rounds = ceil-log (to-nat dg + 1) in
          assert (composition-rounds  $\leq$  ceil-log (maxDegree + 1))  $\gg$ 
            ( $\lambda$ -. staged-composition-fri-program A dg 0
              composition-rounds  $\square$   $\gg$ 
                ( $\lambda$ (-, composition-bs). return (dg, composition-bs)))))))
proof (rule hash-target-program-bind)
  fix x
  show hash-target-program B ?alpha
    (staged-alpha-program (length spec))
    by (rule alpha-target)
next
  fix x as
  show hash-target-program B (?degree + (0 + (?composition + 0)))
    (degree-stage A as  $\gg$ 

```

```

    (λdg. record-staged-message dg >>=
      (λ-. let composition-rounds = ceil-log (to-nat dg + 1)
        in assert
          (composition-rounds ≤ ceil-log (maxDegree + 1)) >>=
            (λ-. staged-composition-fri-program A dg 0
              composition-rounds [] >>=
                (λ(-, composition-bs). return (dg, composition-bs))))))
  by (rule after-degree)
qed
have whole:
  hash-target-program B
    (?alpha-prefix +
      (?alpha + (?degree + (0 + (?composition + 0)))))
    (staged-composition-fri-vector-program A)
  unfolding staged-composition-fri-vector-program-def
proof (rule hash-target-program-bind)
  show hash-target-program B ?alpha-prefix
    (staged-alpha-prefix-program A)
    by (rule alpha-prefix-target)
next
fix x
show hash-target-program B
  (?alpha + (?degree + (0 + (?composition + 0)))))
  (case x of
    (a, b, trace-bs, c) =>
      staged-alpha-program (length spec) >>=
        (λas. degree-stage A as >>=
          (λdg. record-staged-message dg >>=
            (λ-. let composition-rounds =
              ceil-log (to-nat dg + 1)
            in assert
              (composition-rounds ≤ ceil-log (maxDegree + 1)) >>=
                (λ-. staged-composition-fri-program A dg 0
                  composition-rounds [] >>=
                    (λ(r, composition-bs).
                      return (dg, composition-bs)))))))
    using after-alpha by (cases x) simp
qed
show ?thesis
  using whole
  unfolding staged-composition-fri-vector-search-queries-def
  by (simp add: add.assoc add.commute add.left-commute)
qed

lemma staged-composition-fri-vector-target-hit-bound:
  assumes wf: staged-budget-wellformed budgets
    and controlled: staged-adversary-controlled budgets A
  shows
    wp-event (staged-composition-fri-vector-program A)

```

```

    (hash-new-output-hit-event B s) s ≤
    staged-phase-target-error B
    (staged-composition-fri-vector-search-queries budgets)
proof –
  have budget:
    hash-target-budget B
    (staged-composition-fri-vector-search-queries budgets)
    (staged-composition-fri-vector-program A)
  using hash-target-program-staged-composition-fri-vector-program
    [OF wf controlled]
  unfolding hash-target-program-def by simp
  show ?thesis
  using budget unfolding hash-target-budget-def
    staged-phase-target-error-def by blast
qed

lemma hash-target-program-staged-trace-fri-challenge-prefix-program:
assumes controlled: staged-adversary-controlled budgets A
and i-bound: i < length (trace-fri-budgets budgets)
shows
  hash-target-program B
  (staged-trace-fri-search-queries budgets i)
  (staged-trace-fri-challenge-prefix-program A i)
proof –
  let ?trace-root = trace-root-budget budgets
  let ?completed = sum-list (take i (trace-fri-budgets budgets)) + i
  let ?current = trace-fri-budgets budgets ! i
  have trace-root-target:
    hash-target-program B ?trace-root (trace-root-stage A)
    using controlled unfolding staged-adversary-controlled-def
    by (blast intro: controlled-ro-program-target)
  have completed-target:
    hash-target-program B ?completed
    (staged-trace-fri-program A 0 i [])
  proof –
    have len: 0 + i ≤ length (trace-fri-budgets budgets)
    using i-bound by simp
    have exact:
      hash-target-program B
      (sum-list (take i (drop 0 (trace-fri-budgets budgets))) + i)
      (staged-trace-fri-program A 0 i [])
    by (rule hash-target-program-staged-trace-fri-program
      [OF controlled len])
    then show ?thesis by simp
  qed
  have current-target:
    ∧ trace-bs. hash-target-program B ?current
    (trace-fri-root-stage A i trace-bs)
  using controlled i-bound unfolding staged-adversary-controlled-def

```

```

    by (blast intro: controlled-ro-program-target)
  have after-current-record:
     $\bigwedge$ fr trace-bs root. hash-target-program B 0
      (record-staged-message root  $\gg$ 
        ( $\lambda$ -. return (fr, trace-bs, root)))
  proof -
    fix fr trace-bs root
    have hash-target-program B (0 + 0)
      (record-staged-message root  $\gg$ 
        ( $\lambda$ -. return (fr, trace-bs, root)))
    by (rule hash-target-program-bind)
      (rule hash-target-program-record-staged-message,
        rule hash-target-program-return)
    then show hash-target-program B 0
      (record-staged-message root  $\gg$ 
        ( $\lambda$ -. return (fr, trace-bs, root)))
    by simp
  qed
  have after-current:
     $\bigwedge$ fr trace-bs. hash-target-program B (?current + 0)
      (trace-fri-root-stage A i trace-bs  $\gg$ 
        ( $\lambda$ root. record-staged-message root  $\gg$ 
          ( $\lambda$ -. return (fr, trace-bs, root))))
    by (rule hash-target-program-bind)
      (rule current-target, rule after-current-record)
  have after-completed:
     $\bigwedge$ fr. hash-target-program B (?completed + (?current + 0))
      (staged-trace-fri-program A 0 i []  $\gg$ 
        ( $\lambda$ (-, trace-bs).
          trace-fri-root-stage A i trace-bs  $\gg$ 
            ( $\lambda$ root. record-staged-message root  $\gg$ 
              ( $\lambda$ -. return (fr, trace-bs, root))))))
  proof (rule hash-target-program-bind)
    fix fr
    show hash-target-program B ?completed
      (staged-trace-fri-program A 0 i [])
    by (rule completed-target)
  next
    fix fr x
    show hash-target-program B (?current + 0)
      (case x of (-, trace-bs)  $\Rightarrow$ 
        trace-fri-root-stage A i trace-bs  $\gg$ 
          ( $\lambda$ root. record-staged-message root  $\gg$ 
            ( $\lambda$ -. return (fr, trace-bs, root))))
  proof (cases x)
    case (Pair roots trace-bs)
    have target:
      hash-target-program B (?current + 0)
        (trace-fri-root-stage A i trace-bs  $\gg$ 

```



```

      (λroot. record-staged-message root >>=
        (λ-. return (fr, trace-bs, root))))
    by (rule after-current)
  show ?thesis
    unfolding Pair using target by simp
qed
qed
have after-trace-root-record:
  ∧fr. hash-target-program B (0 + (?completed + (?current + 0)))
    (record-staged-message fr >>=
      (λ-. staged-trace-fri-program A 0 i [] >>=
        (λ(-, trace-bs).
          trace-fri-root-stage A i trace-bs >>=
            (λroot. record-staged-message root >>=
              (λ-. return (fr, trace-bs, root)))))))
  by (rule hash-target-program-bind)
    (rule hash-target-program-record-staged-message, rule after-completed)
have whole:
  hash-target-program B
    (?trace-root + (0 + (?completed + (?current + 0))))
    (staged-trace-fri-challenge-prefix-program A i)
  unfolding staged-trace-fri-challenge-prefix-program-def
  by (rule hash-target-program-bind)
    (rule trace-root-target, rule after-trace-root-record)
have sum-eq:
  ?current + sum-list (take i (trace-fri-budgets budgets)) =
    sum-list (take (Suc i) (trace-fri-budgets budgets))
proof -
  have take (Suc i) (trace-fri-budgets budgets) =
    take i (trace-fri-budgets budgets) @
    [trace-fri-budgets budgets ! i]
  by (rule take-Suc-conv-app-nth[OF i-bound])
  then show ?thesis by simp
qed
show ?thesis
  using whole
  unfolding staged-trace-fri-search-queries-def
  by (simp add: sum-eq add.assoc add.commute add.left-commute)
qed

lemma staged-trace-fri-prefix-target-hit-bound:
  assumes controlled: staged-adversary-controlled budgets A
    and i-bound: i < length (trace-fri-budgets budgets)
  shows
    wp-event (staged-trace-fri-challenge-prefix-program A i)
      (hash-new-output-hit-event B s) s ≤
      staged-phase-target-error B
      (staged-trace-fri-search-queries budgets i)
proof -

```

```

have budget:
  hash-target-budget  $B$  (staged-trace-fri-search-queries budgets  $i$ )
    (staged-trace-fri-challenge-prefix-program  $A$   $i$ )
  using hash-target-program-staged-trace-fri-challenge-prefix-program
    [OF controlled  $i$ -bound]
  unfolding hash-target-program-def by simp
show ?thesis
  using budget unfolding hash-target-budget-def
    staged-phase-target-error-def by blast
qed

lemma hash-target-program-staged-composition-fri-challenge-prefix-program:
  assumes wf: staged-budget-wellformed budgets
  and controlled: staged-adversary-controlled budgets  $A$ 
  and  $i$ -bound:  $i < \text{ceil-log } (\text{maxDegree} + 1)$ 
  shows
    hash-target-program  $B$ 
      (staged-composition-fri-search-queries budgets  $i$ )
      (staged-composition-fri-challenge-prefix-program  $A$   $i$ )
proof –
  let ?alpha-prefix = staged-alpha-search-queries budgets 0
  let ?alpha = length spec
  let ?degree = degree-budget budgets
  let ?completed =
    sum-list (take  $i$  (composition-fri-budgets budgets)) +  $i$ 
  let ?current = composition-fri-budgets budgets !  $i$ 
  have comp- $i$ -bound:  $i < \text{length } (\text{composition-fri-budgets budgets})$ 
    using wf  $i$ -bound unfolding staged-budget-wellformed-def by simp
  have alpha-prefix-target:
    hash-target-program  $B$  ?alpha-prefix (staged-alpha-prefix-program  $A$ )
    by (rule hash-target-program-staged-alpha-prefix-program[OF wf controlled])
  have alpha-target:
    hash-target-program  $B$  ?alpha (staged-alpha-program (length spec))
    by (rule hash-target-program-staged-alpha-program)
  have degree-target:
     $\wedge$  as. hash-target-program  $B$  ?degree (degree-stage  $A$  as)
    using controlled unfolding staged-adversary-controlled-def
    by (blast intro: controlled-ro-program-target)
  have completed-target:
     $\wedge$  dg. hash-target-program  $B$  ?completed
      (staged-composition-fri-program  $A$  dg 0  $i$  [])
proof –
  fix dg
  have len:  $0 + i \leq \text{length } (\text{composition-fri-budgets budgets})$ 
    using comp- $i$ -bound by simp
  have exact:
    hash-target-program  $B$ 
      (sum-list (take  $i$  (drop 0 (composition-fri-budgets budgets)))) +  $i$ 
      (staged-composition-fri-program  $A$  dg 0  $i$  [])

```

```

    by (rule hash-target-program-staged-composition-fri-program
        [OF controlled len])
  then show
    hash-target-program B ?completed
      (staged-composition-fri-program A dg 0 i [])
    by simp
qed
have current-target:
   $\bigwedge dg$  composition-bs. hash-target-program B ?current
    (composition-fri-root-stage A dg i composition-bs)
  using controlled comp-i-bound
  unfolding staged-adversary-controlled-def
  by (blast intro: controlled-ro-program-target)
have after-current-record:
   $\bigwedge dg$  composition-bs root. hash-target-program B 0
    (record-staged-message root  $\gg=$ 
      ( $\lambda$ -. return (dg, composition-bs, root)))
proof -
  fix dg composition-bs root
  have hash-target-program B (0 + 0)
    (record-staged-message root  $\gg=$ 
      ( $\lambda$ -. return (dg, composition-bs, root)))
  by (rule hash-target-program-bind)
    (rule hash-target-program-record-staged-message,
      rule hash-target-program-return)
  then show
    hash-target-program B 0
      (record-staged-message root  $\gg=$ 
        ( $\lambda$ -. return (dg, composition-bs, root)))
    by simp
qed
have after-current:
   $\bigwedge dg$  composition-bs. hash-target-program B (?current + 0)
    (composition-fri-root-stage A dg i composition-bs  $\gg=$ 
      ( $\lambda$ root. record-staged-message root  $\gg=$ 
        ( $\lambda$ -. return (dg, composition-bs, root)))))
  by (rule hash-target-program-bind)
    (rule current-target, rule after-current-record)
have after-completed:
   $\bigwedge dg$ . hash-target-program B (?completed + (?current + 0))
    (staged-composition-fri-program A dg 0 i []  $\gg=$ 
      ( $\lambda$ (-, composition-bs).
        composition-fri-root-stage A dg i composition-bs  $\gg=$ 
          ( $\lambda$ root. record-staged-message root  $\gg=$ 
            ( $\lambda$ -. return (dg, composition-bs, root))))))
proof (rule hash-target-program-bind)
  fix dg
  show hash-target-program B ?completed
    (staged-composition-fri-program A dg 0 i [])

```

```

    by (rule completed-target)
next
fix dg x
show hash-target-program B (?current + 0)
  (case x of (_, composition-bs) ⇒
    composition-fri-root-stage A dg i composition-bs ≫=
      (λroot. record-staged-message root ≫=
        (λ-. return (dg, composition-bs, root))))
proof (cases x)
  case (Pair roots composition-bs)
  have target:
    hash-target-program B (?current + 0)
      (composition-fri-root-stage A dg i composition-bs ≫=
        (λroot. record-staged-message root ≫=
          (λ-. return (dg, composition-bs, root))))
  by (rule after-current)
show ?thesis
  unfolding Pair using target by simp
qed
qed
have after-second-assert:
  ∧dg. hash-target-program B (0 + (?completed + (?current + 0)))
    (assert (ceil-log (to-nat dg + 1) ≤ ceil-log (maxDegree + 1)) ≫=
      (λ-. staged-composition-fri-program A dg 0 i [] ≫=
        (λ(-, composition-bs).
          composition-fri-root-stage A dg i composition-bs ≫=
            (λroot. record-staged-message root ≫=
              (λ-. return (dg, composition-bs, root)))))))
  by (rule hash-target-program-bind)
    (rule hash-target-program-assert, rule after-completed)
have after-first-assert:
  ∧dg. hash-target-program B
    (0 + (0 + (?completed + (?current + 0))))
    (assert (Suc i ≤ ceil-log (to-nat dg + 1)) ≫=
      (λ-. assert
        (ceil-log (to-nat dg + 1) ≤ ceil-log (maxDegree + 1)) ≫=
          (λ-. staged-composition-fri-program A dg 0 i [] ≫=
            (λ(-, composition-bs).
              composition-fri-root-stage A dg i composition-bs ≫=
                (λroot. record-staged-message root ≫=
                  (λ-. return (dg, composition-bs, root))))))))
  by (rule hash-target-program-bind)
    (rule hash-target-program-assert, rule after-second-assert)
have after-degree-record:
  ∧dg. hash-target-program B
    (0 + (0 + (0 + (?completed + (?current + 0)))))
    (record-staged-message dg ≫=
      (λ-. assert (Suc i ≤ ceil-log (to-nat dg + 1)) ≫=
        (λ-. assert

```

```

    (ceil-log (to-nat dg + 1) ≤ ceil-log (maxDegree + 1)) >>=
    (λ-. staged-composition-fri-program A dg 0 i [] >>=
    (λ(-, composition-bs).
    composition-fri-root-stage A dg i composition-bs >>=
    (λroot. record-staged-message root >>=
    (λ-. return (dg, composition-bs, root))))))
  by (rule hash-target-program-bind)
  (rule hash-target-program-record-staged-message, rule after-first-assert)
have after-degree:
  ∧ as. hash-target-program B
  (?degree + (0 + (0 + (0 + (?completed + (?current + 0))))))
  (degree-stage A as >>=
  (λdg. record-staged-message dg >>=
  (λ-. let composition-rounds = ceil-log (to-nat dg + 1) in
  assert (Suc i ≤ composition-rounds) >>=
  (λ-. assert (composition-rounds ≤ ceil-log (maxDegree + 1)) >>=
  (λ-. staged-composition-fri-program A dg 0 i [] >>=
  (λ(-, composition-bs).
  composition-fri-root-stage A dg i composition-bs >>=
  (λroot. record-staged-message root >>=
  (λ-. return (dg, composition-bs, root)))))))))
  unfolding Let-def
  by (rule hash-target-program-bind)
  (rule degree-target, rule after-degree-record)
have after-alpha:
  ∧ x. hash-target-program B
  (?alpha + (?degree + (0 + (0 + (0 + (?completed + (?current + 0))))))
  (staged-alpha-program (length spec) >>=
  (λ as. degree-stage A as >>=
  (λ dg. record-staged-message dg >>=
  (λ -. let composition-rounds = ceil-log (to-nat dg + 1) in
  assert (Suc i ≤ composition-rounds) >>=
  (λ -. assert
  (composition-rounds ≤ ceil-log (maxDegree + 1)) >>=
  (λ -. staged-composition-fri-program A dg 0 i [] >>=
  (λ (-, composition-bs).
  composition-fri-root-stage A dg i composition-bs >>=
  (λ root. record-staged-message root >>=
  (λ -. return
  (dg, composition-bs, root)))))))))
  by (rule hash-target-program-bind)
  (rule alpha-target, rule after-degree)
have after-alpha-prefix:
  hash-target-program B
  (?alpha-prefix +
  (?alpha + (?degree +
  (0 + (0 + (0 + (?completed + (?current + 0))))))
  (staged-composition-fri-challenge-prefix-program A i)
  unfolding staged-composition-fri-challenge-prefix-program-def

```

```

proof (rule hash-target-program-bind)
  show hash-target-program B ?alpha-prefix
    (staged-alpha-prefix-program A)
    by (rule alpha-prefix-target)
next
  fix x
  show hash-target-program B
    (?alpha + (?degree +
      (0 + (0 + (0 + (?completed + (?current + 0)))))))
    (case x of (_, -, trace-bs, -) =>
      staged-alpha-program (length spec) >=>
        (λas. degree-stage A as >=>
          (λdg. record-staged-message dg >=>
            (λ-. let composition-rounds = ceil-log (to-nat dg + 1) in
              assert (Suc i ≤ composition-rounds) >=>
                (λ-. assert
                  (composition-rounds ≤ ceil-log (maxDegree + 1)) >=>
                    (λ-. staged-composition-fri-program A dg 0 i [] >=>
                      (λ(-, composition-bs).
                        composition-fri-root-stage A dg i composition-bs >=>
                          (λroot. record-staged-message root >=>
                            (λ-. return
                              (dg, composition-bs, root))))))))))
proof (cases x)
  case (fields a b c d)
  have target:
    hash-target-program B
      (?alpha + (?degree +
        (0 + (0 + (0 + (?completed + (?current + 0)))))))
      (staged-alpha-program (length spec) >=>
        (λas. degree-stage A as >=>
          (λdg. record-staged-message dg >=>
            (λ-. let composition-rounds = ceil-log (to-nat dg + 1) in
              assert (Suc i ≤ composition-rounds) >=>
                (λ-. assert
                  (composition-rounds ≤ ceil-log (maxDegree + 1)) >=>
                    (λ-. staged-composition-fri-program A dg 0 i [] >=>
                      (λ(-, composition-bs).
                        composition-fri-root-stage A dg i composition-bs >=>
                          (λroot. record-staged-message root >=>
                            (λ-. return
                              (dg, composition-bs, root))))))))))
      by (rule after-alpha)
  show ?thesis
    unfolding fields
    using target
    by (simp add: add.assoc add.commute add.left-commute)
qed
qed

```

```

have sum-eq:
  ?current + sum-list (take i (composition-fri-budgets budgets)) =
    sum-list (take (Suc i) (composition-fri-budgets budgets))
proof –
  have take (Suc i) (composition-fri-budgets budgets) =
    take i (composition-fri-budgets budgets) @
    [composition-fri-budgets budgets ! i]
  by (rule take-Suc-conv-app-nth[OF comp-i-bound])
  then show ?thesis by simp
qed
show ?thesis
  using after-alpha-prefix
  unfolding staged-composition-fri-search-queries-def
    staged-alpha-search-queries-def
  by (simp add: sum-eq add.assoc add.commute add.left-commute)
qed

lemma staged-composition-fri-prefix-target-hit-bound:
  assumes wf: staged-budget-wellformed budgets
  and controlled: staged-adversary-controlled budgets A
  and i-bound: i < ceil-log (maxDegree + 1)
  shows
    wp-event (staged-composition-fri-challenge-prefix-program A i)
      (hash-new-output-hit-event B s) s ≤
      staged-phase-target-error B
      (staged-composition-fri-search-queries budgets i)
proof –
  have budget:
    hash-target-budget B (staged-composition-fri-search-queries budgets i)
      (staged-composition-fri-challenge-prefix-program A i)
  using
    hash-target-program-staged-composition-fri-challenge-prefix-program
      [OF wf controlled i-bound]
  unfolding hash-target-program-def by simp
  show ?thesis
    using budget unfolding hash-target-budget-def
      staged-phase-target-error-def by blast
qed

lemma hash-target-program-staged-query-challenge-prefix-program:
  assumes wf: staged-budget-wellformed budgets
  and controlled: staged-adversary-controlled budgets A
  and i-bound: i ≤ rounds
  shows
    hash-target-program B
      (staged-query-search-queries budgets i)
      (staged-query-challenge-prefix-program A i)
proof –
  let ?alpha-prefix = staged-alpha-search-queries budgets 0

```

```

let ?alpha = length spec
let ?degree = degree-budget budgets
let ?composition-fri =
  sum-list (composition-fri-budgets budgets) +
  ceil-log (maxDegree + 1)
let ?composition-final = composition-final-budget budgets
let ?query = sum-list (take i (query-opening-budgets budgets)) + i
have alpha-prefix-target:
  hash-target-program B ?alpha-prefix (staged-alpha-prefix-program A)
  by (rule hash-target-program-staged-alpha-prefix-program[OF wf controlled])
have alpha-target:
  hash-target-program B ?alpha (staged-alpha-program (length spec))
  by (rule hash-target-program-staged-alpha-program)
have degree-target:
   $\wedge$  as. hash-target-program B ?degree (degree-stage A as)
  using controlled unfolding staged-adversary-controlled-def
  by (blast intro: controlled-ro-program-target)
have composition-fri-target:
   $\wedge$  dg. hash-target-program B ?composition-fri
  (assert (ceil-log (to-nat dg + 1)  $\leq$  ceil-log (maxDegree + 1))  $\gg$ 
    ( $\lambda$ -. staged-composition-fri-program A dg 0
      (ceil-log (to-nat dg + 1)) []))
  by (rule hash-target-program-guarded-staged-composition-fri-program
    [OF wf controlled])
have composition-final-target:
   $\wedge$  dg bs. hash-target-program B ?composition-final
  (composition-final-stage A dg bs)
  using controlled unfolding staged-adversary-controlled-def
  by (blast intro: controlled-ro-program-target)
have query-target:
  hash-target-program B ?query (staged-query-program A 0 i)
proof -
  have len: 0 + i  $\leq$  length (query-opening-budgets budgets)
  using wf i-bound unfolding staged-budget-wellformed-def by simp
  have exact:
    hash-target-program B
      (sum-list (take i (drop 0 (query-opening-budgets budgets))) + i)
      (staged-query-program A 0 i)
  by (rule hash-target-program-staged-query-program
    [OF controlled len])
  then show ?thesis by simp
qed
have after-query:
   $\wedge$  dg composition-final. hash-target-program B (?query + 0)
  (staged-query-program A 0 i  $\gg$ 
    ( $\lambda$ query-chunks.
      return (dg, composition-final, query-chunks)))
  by (rule hash-target-program-bind)
  (rule query-target, rule hash-target-program-return)

```



```

have after-composition-final-record:
   $\wedge dg$  composition-final. hash-target-program  $B$  ( $0 + (?query + 0)$ )
    (record-staged-message composition-final  $\gg$ 
      ( $\lambda$ -. staged-query-program  $A$   $0$   $i$   $\gg$ 
        ( $\lambda$ query-chunks.
          return ( $dg$ , composition-final, query-chunks))))
  by (rule hash-target-program-bind)
    (rule hash-target-program-record-staged-message, rule after-query)
have after-composition-final:
   $\wedge dg$  composition-bs. hash-target-program  $B$ 
    (?composition-final + ( $0 + (?query + 0)$ ))
    (composition-final-stage  $A$   $dg$  composition-bs  $\gg$ 
      ( $\lambda$ composition-final.
        record-staged-message composition-final  $\gg$ 
          ( $\lambda$ -. staged-query-program  $A$   $0$   $i$   $\gg$ 
            ( $\lambda$ query-chunks.
              return ( $dg$ , composition-final, query-chunks))))))
  by (rule hash-target-program-bind)
    (rule composition-final-target, rule after-composition-final-record)
have after-composition-fri:
   $\wedge dg$ . hash-target-program  $B$ 
    (?composition-fri + (?composition-final + ( $0 + (?query + 0)$ )))
    ((assert (ceil-log (to-nat  $dg + 1$ )  $\leq$  ceil-log (maxDegree + 1))  $\gg$ 
      ( $\lambda$ -. staged-composition-fri-program  $A$   $dg$   $0$ 
        (ceil-log (to-nat  $dg + 1$ )) []))  $\gg$ 
      ( $\lambda$ (-, composition-bs).
        composition-final-stage  $A$   $dg$  composition-bs  $\gg$ 
          ( $\lambda$ composition-final.
            record-staged-message composition-final  $\gg$ 
              ( $\lambda$ -. staged-query-program  $A$   $0$   $i$   $\gg$ 
                ( $\lambda$ query-chunks.
                  return ( $dg$ , composition-final, query-chunks)))))))
  proof (rule hash-target-program-bind)
    fix  $dg$ 
    show hash-target-program  $B$  ?composition-fri
      (assert (ceil-log (to-nat  $dg + 1$ )  $\leq$  ceil-log (maxDegree + 1))  $\gg$ 
        ( $\lambda$ -. staged-composition-fri-program  $A$   $dg$   $0$ 
          (ceil-log (to-nat  $dg + 1$ )) []))
    by (rule composition-fri-target)
  next
    fix  $dg$   $x$ 
    show hash-target-program  $B$ 
      (?composition-final + ( $0 + (?query + 0)$ ))
      (case  $x$  of (-, composition-bs)  $\Rightarrow$ 
        composition-final-stage  $A$   $dg$  composition-bs  $\gg$ 
          ( $\lambda$ composition-final.
            record-staged-message composition-final  $\gg$ 
              ( $\lambda$ -. staged-query-program  $A$   $0$   $i$   $\gg$ 
                ( $\lambda$ query-chunks.

```

```

      return (dg, composition-final, query-chunks))))))
proof (cases x)
  case (Pair composition-roots composition-bs)
  have target:
    hash-target-program B
    (?composition-final + (0 + (?query + 0)))
    (composition-final-stage A dg composition-bs  $\gg$ 
      ( $\lambda$ composition-final.
        record-staged-message composition-final  $\gg$ 
          ( $\lambda$ -. staged-query-program A 0 i  $\gg$ 
            ( $\lambda$ query-chunks.
              return (dg, composition-final, query-chunks))))))
  by (rule after-composition-final)
show ?thesis
  unfolding Pair using target by simp
qed
qed
have after-degree-record:
   $\wedge$  dg. hash-target-program B
  (0 + (?composition-fri + (?composition-final + (0 + (?query + 0)))))
  (record-staged-message dg  $\gg$ 
    ( $\lambda$ -. (assert
      (ceil-log (to-nat dg + 1)  $\leq$  ceil-log (maxDegree + 1))  $\gg$ 
      ( $\lambda$ -. staged-composition-fri-program A dg 0
        (ceil-log (to-nat dg + 1))  $\square$ ))  $\gg$ 
      ( $\lambda$ (-, composition-bs).
        composition-final-stage A dg composition-bs  $\gg$ 
          ( $\lambda$ composition-final.
            record-staged-message composition-final  $\gg$ 
              ( $\lambda$ -. staged-query-program A 0 i  $\gg$ 
                ( $\lambda$ query-chunks.
                  return (dg, composition-final, query-chunks)))))))
  by (rule hash-target-program-bind)
  (rule hash-target-program-record-staged-message,
    rule after-composition-fri)
have after-degree-record-expanded:
   $\wedge$  dg. hash-target-program B
  (0 + (?composition-fri + (?composition-final + (0 + (?query + 0)))))
  (record-staged-message dg  $\gg$ 
    ( $\lambda$ -. assert
      (ceil-log (to-nat dg + 1)  $\leq$  ceil-log (maxDegree + 1))  $\gg$ 
      ( $\lambda$ -. staged-composition-fri-program A dg 0
        (ceil-log (to-nat dg + 1))  $\square$   $\gg$ 
      ( $\lambda$ (-, composition-bs).
        composition-final-stage A dg composition-bs  $\gg$ 
          ( $\lambda$ composition-final.
            record-staged-message composition-final  $\gg$ 
              ( $\lambda$ -. staged-query-program A 0 i  $\gg$ 
                ( $\lambda$ query-chunks.

```

```

    return (dg, composition-final, query-chunks)))))))))
using after-degree-record
by (simp add: sm-bind-assoc split: prod.splits)
have after-degree:
 $\wedge$ as. hash-target-program B
  (?degree +
    (0 + (?composition-fri + (?composition-final + (0 + (?query + 0))))))
  (degree-stage A as  $\gg$ 
    ( $\lambda$ dg. record-staged-message dg  $\gg$ 
      ( $\lambda$ -. let composition-rounds = ceil-log (to-nat dg + 1) in
        assert (composition-rounds  $\leq$  ceil-log (maxDegree + 1))  $\gg$ 
        ( $\lambda$ -. staged-composition-fri-program A dg 0
          composition-rounds []  $\gg$ 
            ( $\lambda$ (-, composition-bs).
              composition-final-stage A dg composition-bs  $\gg$ 
                ( $\lambda$ composition-final.
                  record-staged-message composition-final  $\gg$ 
                    ( $\lambda$ -. staged-query-program A 0 i  $\gg$ 
                      ( $\lambda$ query-chunks.
                        return
                          (dg, composition-final, query-chunks)))))))))
unfolding Let-def
by (rule hash-target-program-bind)
  (rule degree-target, rule after-degree-record-expanded)
have after-alpha:
 $\wedge$ x. hash-target-program B
  (?alpha +
    (?degree +
      (0 + (?composition-fri +
        (?composition-final + (0 + (?query + 0))))))
    (staged-alpha-program (length spec)  $\gg$ 
      ( $\lambda$ as. degree-stage A as  $\gg$ 
        ( $\lambda$ dg. record-staged-message dg  $\gg$ 
          ( $\lambda$ -. let composition-rounds = ceil-log (to-nat dg + 1) in
            assert (composition-rounds  $\leq$  ceil-log (maxDegree + 1))  $\gg$ 
            ( $\lambda$ -. staged-composition-fri-program A dg 0
              composition-rounds []  $\gg$ 
                ( $\lambda$ (-, composition-bs).
                  composition-final-stage A dg composition-bs  $\gg$ 
                    ( $\lambda$ composition-final.
                      record-staged-message composition-final  $\gg$ 
                        ( $\lambda$ -. staged-query-program A 0 i  $\gg$ 
                          ( $\lambda$ query-chunks.
                            return
                              (dg, composition-final, query-chunks)))))))))
by (rule hash-target-program-bind)
  (rule alpha-target, rule after-degree)
have after-alpha-prefix:
  hash-target-program B

```

```

(?alpha-prefix +
  (?alpha +
    (?degree +
      (0 + (?composition-fri +
        (?composition-final + (0 + (?query + 0)))))))
    (staged-query-challenge-prefix-program A i)
  unfolding staged-query-challenge-prefix-program-def
proof (rule hash-target-program-bind)
show hash-target-program B ?alpha-prefix
  (staged-alpha-prefix-program A)
  by (rule alpha-prefix-target)
next
fix x
show hash-target-program B
  (?alpha +
    (?degree +
      (0 + (?composition-fri +
        (?composition-final + (0 + (?query + 0)))))))
  (case x of (-, -, trace-bs, -) =>
    staged-alpha-program (length spec) >=>
    (λas. degree-stage A as >=>
      (λdg. record-staged-message dg >=>
        (λ-. let composition-rounds = ceil-log (to-nat dg + 1) in
          assert (composition-rounds ≤ ceil-log (maxDegree + 1)) >=>
          (λ-. staged-composition-fri-program A dg 0
            composition-rounds [] >=>
            (λ(-, composition-bs).
              composition-final-stage A dg composition-bs >=>
              (λcomposition-final.
                record-staged-message composition-final >=>
                (λ-. staged-query-program A 0 i >=>
                  (λquery-chunks.
                    return
                      (dg, composition-final,
                        query-chunks))))))))))
proof (cases x)
case (fields a b c d)
have target:
  hash-target-program B
  (?alpha +
    (?degree +
      (0 + (?composition-fri +
        (?composition-final + (0 + (?query + 0)))))))
  (staged-alpha-program (length spec) >=>
    (λas. degree-stage A as >=>
      (λdg. record-staged-message dg >=>
        (λ-. let composition-rounds = ceil-log (to-nat dg + 1) in
          assert
            (composition-rounds ≤ ceil-log (maxDegree + 1)) >=>

```

```

      (λ-. staged-composition-fri-program A dg 0
        composition-rounds [] ≫=
        (λ(-, composition-bs).
          composition-final-stage A dg composition-bs ≫=
          (λcomposition-final.
            record-staged-message composition-final ≫=
            (λ-. staged-query-program A 0 i ≫=
              (λquery-chunks.
                return
                (dg, composition-final,
                  query-chunks))))))))))
    by (rule after-alpha)
  show ?thesis
    unfolding fields
    using target
    by (simp add: add.assoc add.commute add.left-commute)
qed
qed
show ?thesis
  using after-alpha-prefix
  unfolding staged-query-search-queries-def staged-alpha-search-queries-def
  by (simp add: add.assoc add.commute add.left-commute)
qed

```

lemma *hash-range-budget-staged-query-challenge-prefix-program:*

assumes *wf: staged-budget-wellformed budgets*
and *controlled: staged-adversary-controlled budgets A*
and *i-bound: i ≤ rounds*

shows

hash-range-budget
(staged-query-search-queries budgets i)
(staged-query-challenge-prefix-program A i)

proof –

let *?alpha-prefix = staged-alpha-search-queries budgets 0*
let *?alpha = length spec*
let *?degree = degree-budget budgets*
let *?composition-fri =*
sum-list (composition-fri-budgets budgets) +
ceil-log (maxDegree + 1)
let *?composition-final = composition-final-budget budgets*
let *?query = sum-list (take i (query-opening-budgets budgets)) + i*
have *alpha-prefix:*
hash-range-budget ?alpha-prefix (staged-alpha-prefix-program A)
by (rule *hash-range-budget-staged-alpha-prefix-program*
[OF wf controlled])
have *alpha: hash-range-budget ?alpha (staged-alpha-program (length spec))*
by (rule *hash-range-budget-staged-alpha-program*)
have *degree:*
 $\wedge$ *as. hash-range-budget ?degree (degree-stage A as)*

```

using controlled unfolding staged-adversary-controlled-def
by (blast intro: controlled-ro-program-range)
have composition-fri:
 $\wedge dg.$  hash-range-budget ?composition-fri
  (assert (ceil-log (to-nat dg + 1)  $\leq$  ceil-log (maxDegree + 1)))  $\gg$ 
  ( $\lambda$ -. staged-composition-fri-program A dg 0
    (ceil-log (to-nat dg + 1)) []))
by (rule hash-range-budget-guarded-staged-composition-fri-program
  [OF wf controlled])
have composition-final:
 $\wedge dg$  bs. hash-range-budget ?composition-final
  (composition-final-stage A dg bs)
using controlled unfolding staged-adversary-controlled-def
by (blast intro: controlled-ro-program-range)
have query: hash-range-budget ?query (staged-query-program A 0 i)
proof –
  have len: 0 + i  $\leq$  length (query-opening-budgets budgets)
    using wf i-bound unfolding staged-budget-wellformed-def by simp
  have exact:
    hash-range-budget
      (sum-list (take i (drop 0 (query-opening-budgets budgets))))  $+$  i)
      (staged-query-program A 0 i)
    by (rule hash-range-budget-staged-query-program[OF controlled len])
  then show ?thesis by simp
qed
have after-query:
 $\wedge dg$  composition-final. hash-range-budget (?query + 0)
  (staged-query-program A 0 i  $\gg$ 
    ( $\lambda$ query-chunks. return (dg, composition-final, query-chunks)))
by (rule hash-range-budget-bind)
  (rule query, rule hash-range-budget-return)
have after-composition-final-record:
 $\wedge dg$  composition-final. hash-range-budget (0 + (?query + 0))
  (record-staged-message composition-final  $\gg$ 
    ( $\lambda$ -. staged-query-program A 0 i  $\gg$ 
      ( $\lambda$ query-chunks. return
        (dg, composition-final, query-chunks))))
by (rule hash-range-budget-bind)
  (rule hash-range-budget-record-staged-message, rule after-query)
have after-composition-final:
 $\wedge dg$  bs. hash-range-budget
  (?composition-final + (0 + (?query + 0)))
  (composition-final-stage A dg bs  $\gg$ 
    ( $\lambda$ composition-final.
      record-staged-message composition-final  $\gg$ 
        ( $\lambda$ -. staged-query-program A 0 i  $\gg$ 
          ( $\lambda$ query-chunks. return
            (dg, composition-final, query-chunks))))))
by (rule hash-range-budget-bind)

```

```

(rule composition-final, rule after-composition-final-record)
have after-composition-fri:
   $\wedge$  dg. hash-range-budget
    (?composition-fri + (?composition-final + (0 + (?query + 0))))
    ((assert (ceil-log (to-nat dg + 1)  $\leq$  ceil-log (maxDegree + 1))  $\gg$ 
      ( $\lambda$ -. staged-composition-fri-program A dg 0
        (ceil-log (to-nat dg + 1)) []))  $\gg$ 
      ( $\lambda$ (-, composition-bs).
        composition-final-stage A dg composition-bs  $\gg$ 
        ( $\lambda$ composition-final.
          record-staged-message composition-final  $\gg$ 
          ( $\lambda$ -. staged-query-program A 0 i  $\gg$ 
            ( $\lambda$ query-chunks. return
              (dg, composition-final, query-chunks))))))
proof (rule hash-range-budget-bind)
  fix dg
  show hash-range-budget ?composition-fri
    (assert (ceil-log (to-nat dg + 1)  $\leq$  ceil-log (maxDegree + 1))  $\gg$ 
      ( $\lambda$ -. staged-composition-fri-program A dg 0
        (ceil-log (to-nat dg + 1)) []))
    by (rule composition-fri)
next
  fix dg x
  show hash-range-budget
    (?composition-final + (0 + (?query + 0)))
    (case x of (-, composition-bs)  $\Rightarrow$ 
      composition-final-stage A dg composition-bs  $\gg$ 
      ( $\lambda$ composition-final.
        record-staged-message composition-final  $\gg$ 
        ( $\lambda$ -. staged-query-program A 0 i  $\gg$ 
          ( $\lambda$ query-chunks. return
            (dg, composition-final, query-chunks))))))
proof (cases x)
  case (Pair composition-roots composition-bs)
  have step:
    hash-range-budget
      (?composition-final + (0 + (?query + 0)))
      (composition-final-stage A dg composition-bs  $\gg$ 
        ( $\lambda$ composition-final.
          record-staged-message composition-final  $\gg$ 
          ( $\lambda$ -. staged-query-program A 0 i  $\gg$ 
            ( $\lambda$ query-chunks. return
              (dg, composition-final, query-chunks))))))
    by (rule after-composition-final)
  show ?thesis
    unfolding Pair using step by simp
qed
qed
have after-degree-record:

```

```

 $\wedge dg.$  hash-range-budget
  (0 + (?composition-fri + (?composition-final + (0 + (?query + 0))))))
  (record-staged-message dg  $\gg$ 
    ( $\lambda$ -. (assert
      (ceil-log (to-nat dg + 1)  $\leq$  ceil-log (maxDegree + 1))  $\gg$ 
      ( $\lambda$ -. staged-composition-fri-program A dg 0
        (ceil-log (to-nat dg + 1))  $\square$ ))  $\gg$ 
      ( $\lambda$ (-, composition-bs).
        composition-final-stage A dg composition-bs  $\gg$ 
        ( $\lambda$ composition-final.
          record-staged-message composition-final  $\gg$ 
          ( $\lambda$ -. staged-query-program A 0 i  $\gg$ 
            ( $\lambda$ query-chunks. return
              (dg, composition-final, query-chunks)))))))
  by (rule hash-range-budget-bind)
  (rule hash-range-budget-record-staged-message,
    rule after-composition-fri)
have after-degree-record-expanded:
 $\wedge dg.$  hash-range-budget
  (0 + (?composition-fri + (?composition-final + (0 + (?query + 0))))))
  (record-staged-message dg  $\gg$ 
    ( $\lambda$ -. assert
      (ceil-log (to-nat dg + 1)  $\leq$  ceil-log (maxDegree + 1))  $\gg$ 
      ( $\lambda$ -. staged-composition-fri-program A dg 0
        (ceil-log (to-nat dg + 1))  $\square$   $\gg$ 
        ( $\lambda$ (-, composition-bs).
          composition-final-stage A dg composition-bs  $\gg$ 
          ( $\lambda$ composition-final.
            record-staged-message composition-final  $\gg$ 
            ( $\lambda$ -. staged-query-program A 0 i  $\gg$ 
              ( $\lambda$ query-chunks. return
                (dg, composition-final, query-chunks)))))))
    using after-degree-record
  by (simp add: sm-bind-assoc split: prod.splits)
have after-degree:
 $\wedge as.$  hash-range-budget
  (?degree +
    (0 + (?composition-fri +
      (?composition-final + (0 + (?query + 0))))))
  (degree-stage A as  $\gg$ 
    ( $\lambda dg.$  record-staged-message dg  $\gg$ 
      ( $\lambda$ -. let composition-rounds = ceil-log (to-nat dg + 1) in
        assert (composition-rounds  $\leq$  ceil-log (maxDegree + 1))  $\gg$ 
        ( $\lambda$ -. staged-composition-fri-program A dg 0
          composition-rounds  $\square$   $\gg$ 
          ( $\lambda$ (-, composition-bs).
            composition-final-stage A dg composition-bs  $\gg$ 
            ( $\lambda$ composition-final.
              record-staged-message composition-final  $\gg$ 

```



```

      (λ-. staged-query-program A 0 i >>=
        (λquery-chunks. return
          (dg, composition-final, query-chunks)))))))))
unfolding Let-def
by (rule hash-range-budget-bind)
    (rule degree, rule after-degree-record-expanded)
have after-alpha:
  ∧x. hash-range-budget
    (?alpha +
      (?degree +
        (0 + (?composition-fri +
          (?composition-final + (0 + (?query + 0)))))))
    (staged-alpha-program (length spec) >>=
      (λas. degree-stage A as >>=
        (λdg. record-staged-message dg >>=
          (λ-. let composition-rounds = ceil-log (to-nat dg + 1) in
            assert (composition-rounds ≤ ceil-log (maxDegree + 1)) >>=
              (λ-. staged-composition-fri-program A dg 0
                composition-rounds [] >>=
                  (λ(-, composition-bs).
                    composition-final-stage A dg composition-bs >>=
                      (λcomposition-final.
                        record-staged-message composition-final >>=
                          (λ-. staged-query-program A 0 i >>=
                            (λquery-chunks. return
                              (dg, composition-final, query-chunks))))))))))
by (rule hash-range-budget-bind)
    (rule alpha, rule after-degree)
have after-alpha-prefix:
  hash-range-budget
    (?alpha-prefix +
      (?alpha +
        (?degree +
          (0 + (?composition-fri +
            (?composition-final + (0 + (?query + 0)))))))
    (staged-query-challenge-prefix-program A i)
unfolding staged-query-challenge-prefix-program-def
proof (rule hash-range-budget-bind)
show hash-range-budget ?alpha-prefix
  (staged-alpha-prefix-program A)
by (rule alpha-prefix)
next
fix x
show hash-range-budget
  (?alpha +
    (?degree +
      (0 + (?composition-fri +
        (?composition-final + (0 + (?query + 0)))))))
  (case x of (-, -, trace-bs, -) ⇒

```

```

    staged-alpha-program (length spec) >>=
      (λas. degree-stage A as >>=
        (λdg. record-staged-message dg >>=
          (λ-. let composition-rounds = ceil-log (to-nat dg + 1) in
            assert (composition-rounds ≤ ceil-log (maxDegree + 1)) >>=
              (λ-. staged-composition-fri-program A dg 0
                composition-rounds [] >>=
                  (λ(-, composition-bs).
                    composition-final-stage A dg composition-bs >>=
                      (λcomposition-final.
                        record-staged-message composition-final >>=
                          (λ-. staged-query-program A 0 i >>=
                            (λquery-chunks. return
                              (dg, composition-final,
                                query-chunks))))))))))
  proof (cases x)
  case (fields a b c d)
  have step:
    hash-range-budget
      (?alpha +
        (?degree +
          (0 + (?composition-fri +
            (?composition-final + (0 + (?query + 0)))))))
    (staged-alpha-program (length spec) >>=
      (λas. degree-stage A as >>=
        (λdg. record-staged-message dg >>=
          (λ-. let composition-rounds = ceil-log (to-nat dg + 1) in
            assert (composition-rounds ≤ ceil-log (maxDegree + 1)) >>=
              (λ-. staged-composition-fri-program A dg 0
                composition-rounds [] >>=
                  (λ(-, composition-bs).
                    composition-final-stage A dg composition-bs >>=
                      (λcomposition-final.
                        record-staged-message composition-final >>=
                          (λ-. staged-query-program A 0 i >>=
                            (λquery-chunks. return
                              (dg, composition-final,
                                query-chunks))))))))))
      by (rule after-alpha)
    show ?thesis
      unfolding fields using step
      by (simp add: add.assoc add.commute add.left-commute)
  qed
qed
show ?thesis
  using after-alpha-prefix
  unfolding staged-query-search-queries-def staged-alpha-search-queries-def
  by (simp add: add.assoc add.commute add.left-commute)
qed

```

lemma *hash-collision-budget-staged-query-challenge-prefix-program:*
assumes *wf: staged-budget-wellformed budgets*
and *controlled: staged-adversary-controlled budgets A*
and *i-bound: i ≤ rounds*
shows
hash-collision-budget
(staged-query-search-queries budgets i)
(staged-query-challenge-prefix-program A i)
proof –
let *?alpha-prefix = staged-alpha-search-queries budgets 0*
let *?alpha = length spec*
let *?degree = degree-budget budgets*
let *?composition-fri =*
sum-list (composition-fri-budgets budgets) +
ceil-log (maxDegree + 1)
let *?composition-final = composition-final-budget budgets*
let *?query = sum-list (take i (query-opening-budgets budgets)) + i*
have *alpha-prefix-range:*
hash-range-budget ?alpha-prefix (staged-alpha-prefix-program A)
by (rule *hash-range-budget-staged-alpha-prefix-program*
[OF wf controlled])
have *alpha-prefix-coll:*
hash-collision-budget ?alpha-prefix (staged-alpha-prefix-program A)
by (rule *hash-collision-budget-staged-alpha-prefix-program*
[OF wf controlled])
have *alpha-range:*
hash-range-budget ?alpha (staged-alpha-program (length spec))
by (rule *hash-range-budget-staged-alpha-program*)
have *alpha-coll:*
hash-collision-budget ?alpha (staged-alpha-program (length spec))
by (rule *hash-collision-budget-staged-alpha-program*)
have *degree-range:*
∧ as. hash-range-budget ?degree (degree-stage A as)
using *controlled unfolding staged-adversary-controlled-def*
by (blast *intro: controlled-ro-program-range*)
have *degree-coll:*
∧ as. hash-collision-budget ?degree (degree-stage A as)
using *controlled unfolding staged-adversary-controlled-def*
by (blast *intro: controlled-ro-program-collision*)
have *composition-fri-range:*
∧ dg. hash-range-budget ?composition-fri
(assert (ceil-log (to-nat dg + 1) ≤ ceil-log (maxDegree + 1)) ≫=
(λ-. staged-composition-fri-program A dg 0
(ceil-log (to-nat dg + 1)) []))
by (rule *hash-range-budget-guarded-staged-composition-fri-program*
[OF wf controlled])
have *composition-fri-coll:*
∧ dg. hash-collision-budget ?composition-fri

```

    (assert (ceil-log (to-nat dg + 1) ≤ ceil-log (maxDegree + 1)) >>=
      (λ-. staged-composition-fri-program A dg 0
        (ceil-log (to-nat dg + 1)) []))
  by (rule hash-collision-budget-guarded-staged-composition-fri-program
    [OF wf controlled])
have composition-final-range:
  ∧ dg bs. hash-range-budget ?composition-final
    (composition-final-stage A dg bs)
  using controlled unfolding staged-adversary-controlled-def
  by (blast intro: controlled-ro-program-range)
have composition-final-coll:
  ∧ dg bs. hash-collision-budget ?composition-final
    (composition-final-stage A dg bs)
  using controlled unfolding staged-adversary-controlled-def
  by (blast intro: controlled-ro-program-collision)
have query-range: hash-range-budget ?query (staged-query-program A 0 i)
proof -
  have len: 0 + i ≤ length (query-opening-budgets budgets)
    using wf i-bound unfolding staged-budget-wellformed-def by simp
  have exact:
    hash-range-budget
      (sum-list (take i (drop 0 (query-opening-budgets budgets))) + i)
      (staged-query-program A 0 i)
    by (rule hash-range-budget-staged-query-program[OF controlled len])
  then show ?thesis by simp
qed
have query-coll: hash-collision-budget ?query (staged-query-program A 0 i)
proof -
  have len: 0 + i ≤ length (query-opening-budgets budgets)
    using wf i-bound unfolding staged-budget-wellformed-def by simp
  have exact:
    hash-collision-budget
      (sum-list (take i (drop 0 (query-opening-budgets budgets))) + i)
      (staged-query-program A 0 i)
    by (rule hash-collision-budget-staged-query-program[OF controlled len])
  then show ?thesis by simp
qed
have after-query-range:
  ∧ dg composition-final. hash-range-budget (?query + 0)
    (staged-query-program A 0 i >>=
      (λquery-chunks. return (dg, composition-final, query-chunks)))
  by (rule hash-range-budget-bind)
    (rule query-range, rule hash-range-budget-return)
have after-query-coll:
  ∧ dg composition-final. hash-collision-budget (?query + 0)
    (staged-query-program A 0 i >>=
      (λquery-chunks. return (dg, composition-final, query-chunks)))
  by (rule hash-collision-budget-bind)
    (rule query-range, rule query-coll,

```

```

    rule hash-range-budget-return, rule hash-collision-budget-return)
have after-composition-final-record-range:
   $\wedge dg$  composition-final. hash-range-budget  $(0 + (?query + 0))$ 
  (record-staged-message composition-final  $\gg$ 
    ( $\lambda$ -. staged-query-program A 0 i  $\gg$ 
      ( $\lambda$ query-chunks. return
        (dg, composition-final, query-chunks))))
by (rule hash-range-budget-bind)
  (rule hash-range-budget-record-staged-message, rule after-query-range)
have after-composition-final-record-coll:
   $\wedge dg$  composition-final. hash-collision-budget  $(0 + (?query + 0))$ 
  (record-staged-message composition-final  $\gg$ 
    ( $\lambda$ -. staged-query-program A 0 i  $\gg$ 
      ( $\lambda$ query-chunks. return
        (dg, composition-final, query-chunks))))
by (rule hash-collision-budget-bind)
  (rule hash-range-budget-record-staged-message,
    rule hash-collision-budget-record-staged-message,
    rule after-query-range, rule after-query-coll)
have after-composition-final-range:
   $\wedge dg$  bs. hash-range-budget
  (?composition-final +  $(0 + (?query + 0))$ )
  (composition-final-stage A dg bs  $\gg$ 
    ( $\lambda$ composition-final.
      record-staged-message composition-final  $\gg$ 
        ( $\lambda$ -. staged-query-program A 0 i  $\gg$ 
          ( $\lambda$ query-chunks. return
            (dg, composition-final, query-chunks))))))
by (rule hash-range-budget-bind)
  (rule composition-final-range,
    rule after-composition-final-record-range)
have after-composition-final-coll:
   $\wedge dg$  bs. hash-collision-budget
  (?composition-final +  $(0 + (?query + 0))$ )
  (composition-final-stage A dg bs  $\gg$ 
    ( $\lambda$ composition-final.
      record-staged-message composition-final  $\gg$ 
        ( $\lambda$ -. staged-query-program A 0 i  $\gg$ 
          ( $\lambda$ query-chunks. return
            (dg, composition-final, query-chunks))))))
by (rule hash-collision-budget-bind)
  (rule composition-final-range, rule composition-final-coll,
    rule after-composition-final-record-range,
    rule after-composition-final-record-coll)
have after-composition-fri-range:
   $\wedge dg$ . hash-range-budget
  (?composition-fri + (?composition-final +  $(0 + (?query + 0))$ ))
  ((assert (ceil-log (to-nat dg + 1)  $\leq$  ceil-log (maxDegree + 1)))  $\gg$ 
    ( $\lambda$ -. staged-composition-fri-program A dg 0

```

```

    (ceil-log (to-nat dg + 1)) []) >=>
  (λ(-, composition-bs).
    composition-final-stage A dg composition-bs >=>
    (λcomposition-final.
      record-staged-message composition-final >=>
      (λ-. staged-query-program A 0 i >=>
        (λquery-chunks. return
          (dg, composition-final, query-chunks))))))
proof (rule hash-range-budget-bind)
  fix dg
  show hash-range-budget ?composition-fri
    (assert (ceil-log (to-nat dg + 1) ≤ ceil-log (maxDegree + 1)) >=>
      (λ-. staged-composition-fri-program A dg 0
        (ceil-log (to-nat dg + 1)) []))
    by (rule composition-fri-range)
next
  fix dg x
  show hash-range-budget
    (?composition-final + (0 + (?query + 0)))
    (case x of (-, composition-bs) =>
      composition-final-stage A dg composition-bs >=>
      (λcomposition-final.
        record-staged-message composition-final >=>
        (λ-. staged-query-program A 0 i >=>
          (λquery-chunks. return
            (dg, composition-final, query-chunks))))))
  proof (cases x)
  case (Pair composition-roots composition-bs)
  have step:
    hash-range-budget
      (?composition-final + (0 + (?query + 0)))
      (composition-final-stage A dg composition-bs >=>
        (λcomposition-final.
          record-staged-message composition-final >=>
          (λ-. staged-query-program A 0 i >=>
            (λquery-chunks. return
              (dg, composition-final, query-chunks))))))
    by (rule after-composition-final-range)
  show ?thesis
    unfolding Pair using step by simp
  qed
qed
have after-composition-fri-coll:
  ∧ dg. hash-collision-budget
    (?composition-fri + (?composition-final + (0 + (?query + 0))))
    ((assert (ceil-log (to-nat dg + 1) ≤ ceil-log (maxDegree + 1)) >=>
      (λ-. staged-composition-fri-program A dg 0
        (ceil-log (to-nat dg + 1)) [])) >=>
      (λ(-, composition-bs).

```

```

    composition-final-stage A dg composition-bs >>=
      (λcomposition-final.
        record-staged-message composition-final >>=
          (λ-. staged-query-program A 0 i >>=
            (λquery-chunks. return
              (dg, composition-final, query-chunks))))))
  proof (rule hash-collision-budget-bind)
    fix dg
    show hash-range-budget ?composition-fri
      (assert (ceil-log (to-nat dg + 1) ≤ ceil-log (maxDegree + 1)) >>=
        (λ-. staged-composition-fri-program A dg 0
          (ceil-log (to-nat dg + 1)) []))
      by (rule composition-fri-range)
    show hash-collision-budget ?composition-fri
      (assert (ceil-log (to-nat dg + 1) ≤ ceil-log (maxDegree + 1)) >>=
        (λ-. staged-composition-fri-program A dg 0
          (ceil-log (to-nat dg + 1)) []))
      by (rule composition-fri-coll)
  next
    fix dg x
    show hash-range-budget
      (?composition-final + (0 + (?query + 0)))
      (case x of (-, composition-bs) ⇒
        composition-final-stage A dg composition-bs >>=
          (λcomposition-final.
            record-staged-message composition-final >>=
              (λ-. staged-query-program A 0 i >>=
                (λquery-chunks. return
                  (dg, composition-final, query-chunks))))))
    proof (cases x)
      case (Pair composition-roots composition-bs)
      have step:
        hash-range-budget
          (?composition-final + (0 + (?query + 0)))
          (composition-final-stage A dg composition-bs >>=
            (λcomposition-final.
              record-staged-message composition-final >>=
                (λ-. staged-query-program A 0 i >>=
                  (λquery-chunks. return
                    (dg, composition-final, query-chunks))))))
        by (rule after-composition-final-range)
      show ?thesis
        unfolding Pair using step by simp
    qed
    show hash-collision-budget
      (?composition-final + (0 + (?query + 0)))
      (case x of (-, composition-bs) ⇒
        composition-final-stage A dg composition-bs >>=
          (λcomposition-final.

```

```

    record-staged-message composition-final >>=
      (λ-. staged-query-program A 0 i >>=
        (λquery-chunks. return
          (dg, composition-final, query-chunks))))))
  proof (cases x)
  case (Pair composition-roots composition-bs)
  have step:
    hash-collision-budget
      (?composition-final + (0 + (?query + 0)))
      (composition-final-stage A dg composition-bs >>=
        (λcomposition-final.
          record-staged-message composition-final >>=
            (λ-. staged-query-program A 0 i >>=
              (λquery-chunks. return
                (dg, composition-final, query-chunks))))))
      by (rule after-composition-final-coll)
  show ?thesis
    unfolding Pair using step by simp
  qed
qed
have after-composition-fri-range-expanded:
  ∧ dg. hash-range-budget
    (?composition-fri + (?composition-final + (0 + (?query + 0))))
    (assert (ceil-log (to-nat dg + 1) ≤ ceil-log (maxDegree + 1)) >>=
      (λ-. staged-composition-fri-program A dg 0
        (ceil-log (to-nat dg + 1)) [] >>=
          (λ(-, composition-bs).
            composition-final-stage A dg composition-bs >>=
              (λcomposition-final.
                record-staged-message composition-final >>=
                  (λ-. staged-query-program A 0 i >>=
                    (λquery-chunks. return
                      (dg, composition-final, query-chunks))))))))
  using after-composition-fri-range
  by (simp add: sm-bind-assoc split: prod.splits)
have after-composition-fri-coll-expanded:
  ∧ dg. hash-collision-budget
    (?composition-fri + (?composition-final + (0 + (?query + 0))))
    (assert (ceil-log (to-nat dg + 1) ≤ ceil-log (maxDegree + 1)) >>=
      (λ-. staged-composition-fri-program A dg 0
        (ceil-log (to-nat dg + 1)) [] >>=
          (λ(-, composition-bs).
            composition-final-stage A dg composition-bs >>=
              (λcomposition-final.
                record-staged-message composition-final >>=
                  (λ-. staged-query-program A 0 i >>=
                    (λquery-chunks. return
                      (dg, composition-final, query-chunks))))))))
  using after-composition-fri-coll

```



```

by (simp add: sm-bind-assoc split: prod.splits)
have after-degree-record-range:
   $\wedge dg.$  hash-range-budget
   $(0 + (?composition-fri + (?composition-final + (0 + (?query + 0))))))$ 
  (record-staged-message dg  $\gg$ 
    ( $\lambda -.$  assert
      ( $ceil-log (to-nat dg + 1) \leq ceil-log (maxDegree + 1)$ )  $\gg$ 
      ( $\lambda -.$  staged-composition-fri-program A dg 0
        ( $ceil-log (to-nat dg + 1)$ )  $\square$   $\gg$ 
        ( $\lambda(-, composition-bs).$ 
          composition-final-stage A dg composition-bs  $\gg$ 
          ( $\lambda composition-final.$ 
            record-staged-message composition-final  $\gg$ 
            ( $\lambda -.$  staged-query-program A 0 i  $\gg$ 
              ( $\lambda query-chunks.$  return
                (dg, composition-final, query-chunks))))))))))
by (rule hash-range-budget-bind)
  (rule hash-range-budget-record-staged-message,
   rule after-composition-fri-range-expanded)
have after-degree-record-coll:
   $\wedge dg.$  hash-collision-budget
   $(0 + (?composition-fri + (?composition-final + (0 + (?query + 0))))))$ 
  (record-staged-message dg  $\gg$ 
    ( $\lambda -.$  assert
      ( $ceil-log (to-nat dg + 1) \leq ceil-log (maxDegree + 1)$ )  $\gg$ 
      ( $\lambda -.$  staged-composition-fri-program A dg 0
        ( $ceil-log (to-nat dg + 1)$ )  $\square$   $\gg$ 
        ( $\lambda(-, composition-bs).$ 
          composition-final-stage A dg composition-bs  $\gg$ 
          ( $\lambda composition-final.$ 
            record-staged-message composition-final  $\gg$ 
            ( $\lambda -.$  staged-query-program A 0 i  $\gg$ 
              ( $\lambda query-chunks.$  return
                (dg, composition-final, query-chunks))))))))))
by (rule hash-collision-budget-bind)
  (rule hash-range-budget-record-staged-message,
   rule hash-collision-budget-record-staged-message,
   rule after-composition-fri-range-expanded,
   rule after-composition-fri-coll-expanded)
have after-degree-range:
   $\wedge as.$  hash-range-budget
  (?degree +
     $(0 + (?composition-fri +$ 
       $(?composition-final + (0 + (?query + 0))))))$ 
  (degree-stage A as  $\gg$ 
    ( $\lambda dg.$  record-staged-message dg  $\gg$ 
      ( $\lambda -.$  let composition-rounds =  $ceil-log (to-nat dg + 1)$  in
        assert ( $composition-rounds \leq ceil-log (maxDegree + 1)$ )  $\gg$ 
        ( $\lambda -.$  staged-composition-fri-program A dg 0

```

```

composition-rounds [] >=
(λ(-, composition-bs).
  composition-final-stage A dg composition-bs >=
  (λcomposition-final.
    record-staged-message composition-final >=
    (λ-. staged-query-program A 0 i >=
      (λquery-chunks. return
        (dg, composition-final, query-chunks)))))))))
unfolding Let-def
by (rule hash-range-budget-bind)
  (rule degree-range, rule after-degree-record-range)
have after-degree-coll:
  ∧as. hash-collision-budget
  (?degree +
    (0 + (?composition-fri +
      (?composition-final + (0 + (?query + 0))))))
  (degree-stage A as >=
    (λdg. record-staged-message dg >=
      (λ-. let composition-rounds = ceil-log (to-nat dg + 1) in
        assert (composition-rounds ≤ ceil-log (maxDegree + 1)) >=
        (λ-. staged-composition-fri-program A dg 0
          composition-rounds [] >=
          (λ(-, composition-bs).
            composition-final-stage A dg composition-bs >=
            (λcomposition-final.
              record-staged-message composition-final >=
              (λ-. staged-query-program A 0 i >=
                (λquery-chunks. return
                  (dg, composition-final, query-chunks)))))))))
unfolding Let-def
by (rule hash-collision-budget-bind)
  (rule degree-range, rule degree-coll,
    rule after-degree-record-range, rule after-degree-record-coll)
have after-alpha-range:
  ∧x. hash-range-budget
  (?alpha +
    (?degree +
      (0 + (?composition-fri +
        (?composition-final + (0 + (?query + 0))))))
  (staged-alpha-program (length spec) >=
    (λas. degree-stage A as >=
      (λdg. record-staged-message dg >=
        (λ-. let composition-rounds = ceil-log (to-nat dg + 1) in
          assert (composition-rounds ≤ ceil-log (maxDegree + 1)) >=
          (λ-. staged-composition-fri-program A dg 0
            composition-rounds [] >=
            (λ(-, composition-bs).
              composition-final-stage A dg composition-bs >=
              (λcomposition-final.

```

```

      record-staged-message composition-final >>=
      (λ-. staged-query-program A 0 i >>=
        (λquery-chunks. return
          (dg, composition-final, query-chunks)))))))))
  by (rule hash-range-budget-bind)
    (rule alpha-range, rule after-degree-range)
have after-alpha-coll:
  ∧x. hash-collision-budget
    (?alpha +
      (?degree +
        (0 + (?composition-fri +
          (?composition-final + (0 + (?query + 0)))))))
    (staged-alpha-program (length spec) >>=
      (λas. degree-stage A as >>=
        (λdg. record-staged-message dg >>=
          (λ-. let composition-rounds = ceil-log (to-nat dg + 1) in
            assert (composition-rounds ≤ ceil-log (maxDegree + 1)) >>=
              (λ-. staged-composition-fri-program A dg 0
                composition-rounds [] >>=
                  (λ(-, composition-bs).
                    composition-final-stage A dg composition-bs >>=
                      (λcomposition-final.
                        record-staged-message composition-final >>=
                          (λ-. staged-query-program A 0 i >>=
                            (λquery-chunks. return
                              (dg, composition-final, query-chunks))))))))))
    by (rule hash-collision-budget-bind)
      (rule alpha-range, rule alpha-coll,
        rule after-degree-range, rule after-degree-coll)
have after-alpha-prefix:
  hash-collision-budget
    (?alpha-prefix +
      (?alpha +
        (?degree +
          (0 + (?composition-fri +
            (?composition-final + (0 + (?query + 0)))))))
    (staged-query-challenge-prefix-program A i)
  unfolding staged-query-challenge-prefix-program-def
proof (rule hash-collision-budget-bind)
  show hash-range-budget ?alpha-prefix
    (staged-alpha-prefix-program A)
  by (rule alpha-prefix-range)
  show hash-collision-budget ?alpha-prefix
    (staged-alpha-prefix-program A)
  by (rule alpha-prefix-coll)
next
fix x
show hash-range-budget
  (?alpha +

```

```

    (?degree +
      (0 + (?composition-fri +
        (?composition-final + (0 + (?query + 0)))))))
  (case x of (-, -, trace-bs, -) =>
    staged-alpha-program (length spec) >=>
    (λas. degree-stage A as >=>
      (λdg. record-staged-message dg >=>
        (λ-. let composition-rounds = ceil-log (to-nat dg + 1) in
          assert (composition-rounds ≤ ceil-log (maxDegree + 1)) >=>
            (λ-. staged-composition-fri-program A dg 0
              composition-rounds [] >=>
                (λ(-, composition-bs).
                  composition-final-stage A dg composition-bs >=>
                    (λcomposition-final.
                      record-staged-message composition-final >=>
                        (λ-. staged-query-program A 0 i >=>
                          (λquery-chunks. return
                            (dg, composition-final,
                              query-chunks))))))))))
  proof (cases x)
  case (fields a b c d)
  show ?thesis
    unfolding fields using after-alpha-range
    by (simp add: add.assoc add.commute add.left-commute)
qed
show hash-collision-budget
  (?alpha +
    (?degree +
      (0 + (?composition-fri +
        (?composition-final + (0 + (?query + 0)))))))
  (case x of (-, -, trace-bs, -) =>
    staged-alpha-program (length spec) >=>
    (λas. degree-stage A as >=>
      (λdg. record-staged-message dg >=>
        (λ-. let composition-rounds = ceil-log (to-nat dg + 1) in
          assert (composition-rounds ≤ ceil-log (maxDegree + 1)) >=>
            (λ-. staged-composition-fri-program A dg 0
              composition-rounds [] >=>
                (λ(-, composition-bs).
                  composition-final-stage A dg composition-bs >=>
                    (λcomposition-final.
                      record-staged-message composition-final >=>
                        (λ-. staged-query-program A 0 i >=>
                          (λquery-chunks. return
                            (dg, composition-final,
                              query-chunks))))))))))
  proof (cases x)
  case (fields a b c d)
  show ?thesis

```

```

    unfolding fields using after-alpha-coll
    by (simp add: add.assoc add.commute add.left-commute)
  qed
qed
show ?thesis
  using after-alpha-prefix
  unfolding staged-query-search-queries-def staged-alpha-search-queries-def
  by (simp add: add.assoc add.commute add.left-commute)
qed

```

```

lemma staged-query-prefix-target-hit-bound:
  assumes wf: staged-budget-wellformed budgets
    and controlled: staged-adversary-controlled budgets A
    and i-bound:  $i \leq \text{rounds}$ 
  shows
    wp-event (staged-query-challenge-prefix-program A i)
      (hash-new-output-hit-event B s)  $s \leq$ 
      staged-phase-target-error B
      (staged-query-search-queries budgets i)
  proof -
    have budget:
      hash-target-budget B (staged-query-search-queries budgets i)
      (staged-query-challenge-prefix-program A i)
    using hash-target-program-staged-query-challenge-prefix-program
      [OF wf controlled i-bound]
    unfolding hash-target-program-def by simp
    show ?thesis
      using budget unfolding hash-target-budget-def
      staged-phase-target-error-def by blast
  qed
end
end
end

```

```

theory Staged-Security-Experiment-Targets
  imports Staged-Security-Experiment-Budgets
begin

```

Bad-value target events and checked-prefix probability bounds.

```

context soundness
begin

```

```

definition staged-query-prefix-bad-index-error
  :: staged-budgets  $\Rightarrow$  nat set  $\Rightarrow$  nat  $\Rightarrow$  prob
where
  staged-query-prefix-bad-index-error budgets B i =
    staged-phase-target-error (query-index-raw-preimage B)

```

(staged-query-search-queries budgets i)

definition *fri-vector-position-values* :: 'f list set $\Rightarrow$ nat $\Rightarrow$ 'f set
where

fri-vector-position-values $B\ i =$
 $\{b. \exists bs \in B. i < \text{length } bs \wedge bs ! i = b\}$

lemma *fri-vector-position-valueI*:

assumes $bs \in B$

and $i < \text{length } bs$

shows $bs ! i \in \text{fri-vector-position-values } B\ i$

using *assms* **unfolding** *fri-vector-position-values-def* **by** *blast*

lemma *fri-vector-member-imp-position-value*:

assumes *subset*: $B \subseteq \text{fri-challenge-space } n$

and *member*: $bs \in B$

and *i-bound*: $i < n$

shows $bs ! i \in \text{fri-vector-position-values } B\ i$

proof –

have $\text{length } bs = n$

using *subset member* **unfolding** *fri-challenge-space-def* **by** *auto*

then have $i < \text{length } bs$

using *i-bound* **by** *simp*

then show *?thesis*

by (rule *fri-vector-position-valueI*[*OF member*])

qed

lemma *fri-vector-member-imp-first-position-value*:

assumes *subset*: $B \subseteq \text{fri-challenge-space } n$

and *member*: $bs \in B$

and *nonempty*: $0 < n$

shows $bs ! 0 \in \text{fri-vector-position-values } B\ 0$

by (rule *fri-vector-member-imp-position-value*[*OF subset member*])

(use *nonempty* **in** *simp*)

lemma *fri-vector-extend-member-imp-position-value*:

assumes *member*: $bs @ [b] \in B$

and *length*: $\text{length } bs = i$

shows $b \in \text{fri-vector-position-values } B\ i$

using *assms* **unfolding** *fri-vector-position-values-def* **by** *force*

lemma *fri-vector-position-values-card-le*:

assumes *subset*: $B \subseteq \text{fri-challenge-space } n$

and *i-bound*: $i < n$

shows $\text{card } (\text{fri-vector-position-values } B\ i) \leq \text{card } B$

proof –

have *finite-B*: *finite* B

by (rule *finite-subset*[*OF subset finite-fri-challenge-space*])

have *values-eq*:

```

fri-vector-position-values  $B\ i = (\lambda bs. bs\ !\ i)\ \text{' } B$ 
using subset i-bound
unfolding fri-vector-position-values-def fri-challenge-space-def
by auto
show ?thesis
unfolding values-eq by (rule card-image-le[OF finite-B])
qed

```

```

lemma fri-vector-member-coordinate-cover:
  assumes subset:  $B \subseteq \text{fri-challenge-space } n$ 
  and member:  $bs \in B$ 
  and nonempty:  $0 < n$ 
  shows  $\exists i < n. bs\ !\ i \in \text{fri-vector-position-values } B\ i$ 
proof -
  have  $bs\ !\ 0 \in \text{fri-vector-position-values } B\ 0$ 
  by (rule fri-vector-member-imp-first-position-value
    [OF subset member nonempty])
  then show ?thesis
  using nonempty by blast
qed

```

```

definition trace-fri-challenge-values-absent
  :: 'f set  $\Rightarrow$  ('f, 'a) protocol-channel-scheme  $\Rightarrow$  bool
where
  trace-fri-challenge-values-absent  $B\ s \longleftrightarrow$ 
    ( $\forall i\ x\ y.$ 
      fmlookup (HashMap  $s$ ) (TraceFriChallenge  $i\ x$ ) = Some  $y \longrightarrow$ 
         $y \notin B$ )

```

```

definition composition-fri-challenge-values-absent
  :: 'f set  $\Rightarrow$  ('f, 'a) protocol-channel-scheme  $\Rightarrow$  bool
where
  composition-fri-challenge-values-absent  $B\ s \longleftrightarrow$ 
    ( $\forall i\ x\ y.$ 
      fmlookup (HashMap  $s$ ) (CompositionFriChallenge  $i\ x$ ) = Some  $y \longrightarrow$ 
         $y \notin B$ )

```

```

definition alpha-challenge-values-absent
  :: 'f set  $\Rightarrow$  ('f, 'a) protocol-channel-scheme  $\Rightarrow$  bool
where
  alpha-challenge-values-absent  $B\ s \longleftrightarrow$ 
    ( $\forall i\ x\ y.$ 
      fmlookup (HashMap  $s$ ) (AlphaChallenge  $i\ x$ ) = Some  $y \longrightarrow$ 
         $y \notin B$ )

```

```

definition query-index-raw-preimage-absent
  :: nat set  $\Rightarrow$  ('f, 'a) protocol-channel-scheme  $\Rightarrow$  bool
where
  query-index-raw-preimage-absent  $B\ s \longleftrightarrow$ 

```

$(\forall i \ x \ \text{raw}.$
 $\text{fmlookup} (\text{HashMap } s) (\text{QueryIndexChallenge } i \ x) = \text{Some } \text{raw} \longrightarrow$
 $\text{raw} \notin \text{query-index-raw-preimage } B)$

lemma *staged-query-prefix-bad-index-error-eq*:
staged-query-prefix-bad-index-error budgets B i =
hash-target-budget-value (query-index-raw-preimage B)
(staged-query-search-queries budgets i)
unfolding *staged-query-prefix-bad-index-error-def*
staged-phase-target-error-def **by** *simp*

lemma *trace-fri-challenge-values-absent-empty[simp]*:
trace-fri-challenge-values-absent B
(s(|HashMap := fmempty|))
unfolding *trace-fri-challenge-values-absent-def* **by** *simp*

lemma *composition-fri-challenge-values-absent-empty[simp]*:
composition-fri-challenge-values-absent B
(s(|HashMap := fmempty|))
unfolding *composition-fri-challenge-values-absent-def* **by** *simp*

lemma *alpha-challenge-values-absent-empty[simp]*:
alpha-challenge-values-absent B
(s(|HashMap := fmempty|))
unfolding *alpha-challenge-values-absent-def* **by** *simp*

lemma *staged-query-prefix-bad-index-target-bound*:
assumes *wf: staged-budget-wellformed budgets*
and *controlled: staged-adversary-controlled budgets A*
and *i-bound: i ≤ rounds*
shows
wp-event (staged-query-challenge-prefix-program A i)
(hash-new-output-hit-event (query-index-raw-preimage B) s) s ≤
staged-query-prefix-bad-index-error budgets B i
unfolding *staged-query-prefix-bad-index-error-def*
by *(rule staged-query-prefix-target-hit-bound[OF wf controlled i-bound])*

lemma *query-index-raw-preimage-absent-empty[simp]*:
query-index-raw-preimage-absent B
(s(|HashMap := fmempty|))
unfolding *query-index-raw-preimage-absent-def* **by** *simp*

lemma *staged-trace-fri-prefix-receive-bad-value-imp-new-output-hit*:
assumes *controlled: staged-adversary-controlled budgets A*
and *i-bound: i < length (trace-fri-budgets budgets)*
and *absent: trace-fri-challenge-values-absent B s*
and *out:*
Some (b, t) ∈
set-dist


```

      (execute
        (staged-trace-fri-challenge-prefix-program A i  $\gg$ 
          ( $\lambda$ -. receive-trace-fri-challenge)) s)
    and bad:  $b \in B$ 
  shows hash-new-output-hit-event B s (Some (b, t))
proof -
  obtain prefix-out u where prefix:
    Some (prefix-out, u)  $\in$ 
      set-dist
        (execute (staged-trace-fri-challenge-prefix-program A i) s)
  and recv:
    Some (b, t)  $\in$  set-dist (execute receive-trace-fri-challenge u)
  using out by (auto elim!: set-dist-bindE)
  have prefix-ext:  $s \leq u$ 
  using hash-target-program-staged-trace-fri-challenge-prefix-program
    [OF controlled i-bound, of B] prefix
  unfolding hash-target-program-def hash-extension-preserving-def
  by blast
  have recv-out:
     $u \leq t \wedge$ 
    fmlookup (HashMap t)
      (TraceFriChallenge (PTraceFriCounter u) (PState u)) = Some b
  using receive-trace-fri-challenge-outcome[OF recv] by blast
  show ?thesis
proof (cases
  fmlookup (HashMap s)
  (TraceFriChallenge (PTraceFriCounter u) (PState u)))
  case None
  then have hash-map-new-output-hit B s t
    unfolding hash-map-new-output-hit-def
    using recv-out bad by blast
  then show ?thesis
    unfolding hash-new-output-hit-event-def by simp
next
  case (Some old-b)
  have old-lookup-t:
    fmlookup (HashMap t)
      (TraceFriChallenge (PTraceFriCounter u) (PState u)) = Some old-b
  by (rule hash-extension-lookup[OF Some])
  (rule hash-ext-trans[OF prefix-ext conjunct1[OF recv-out]])
  then have old-eq:  $old-b = b$ 
    using recv-out by simp
  have old-b  $\notin B$ 
    using absent Some
    unfolding trace-fri-challenge-values-absent-def by blast
  then show ?thesis
    using old-eq bad by simp
qed
qed

```

lemma *staged-trace-fri-prefix-receive-bad-value-bound*:

assumes *controlled*: *staged-adversary-controlled budgets A*
and *i-bound*: $i < \text{length } (\text{trace-fri-budgets budgets})$
and *absent*: *trace-fri-challenge-values-absent B s*
shows

wp-event
 $(\text{staged-trace-fri-challenge-prefix-program } A \ i \gg=$
 $(\lambda-. \text{receive-trace-fri-challenge}))$
 $(\lambda \text{out. case out of None} \Rightarrow \text{False}$
 $| \text{Some } (b, -) \Rightarrow b \in B) \ s \leq$
 $\text{staged-phase-target-error } B$
 $(\text{Suc } (\text{staged-trace-fri-search-queries budgets } i)))$

proof –

let *?m* =
 $\text{staged-trace-fri-challenge-prefix-program } A \ i \gg=$
 $(\lambda-. \text{receive-trace-fri-challenge})$

let *?bad* =
 $\lambda \text{out. case out of None} \Rightarrow \text{False}$
 $| \text{Some } (b, -) \Rightarrow b \in B$

let *?hit* = *hash-new-output-hit-event B s*

have *bad-imp-hit*:
 $\text{wp-event } ?m \ ?bad \ s \leq \text{wp-event } ?m \ ?hit \ s$

proof (rule *wp-event-mono-on-support*)

fix *out*

assume *support*: $\text{out} \in \text{set-dist } (\text{execute } ?m \ s)$
and *bad*: *?bad out*

show *?hit out*

proof (*cases out*)

case *None*
then show *?thesis* **using** *bad by simp*

next

case (*Some bt*)

then obtain *b t* **where** *out-eq*: $\text{out} = \text{Some } (b, t)$
by (*cases bt*) *simp*

have *out'*:
 $\text{Some } (b, t) \in \text{set-dist } (\text{execute } ?m \ s)$
using *support out-eq* **by** *simp*

have *b-in*: $b \in B$
using *bad out-eq* **by** *simp*

show *?thesis*
unfolding *out-eq*
by (rule *staged-trace-fri-prefix-receive-bad-value-imp-new-output-hit*
 $[\text{OF controlled } i\text{-bound absent out' } b\text{-in}]$)

qed

qed

have *prefix-target*:
 $\text{hash-target-program } B$
 $(\text{staged-trace-fri-search-queries budgets } i)$

```

    (staged-trace-fri-challenge-prefix-program A i)
  by (rule hash-target-program-staged-trace-fri-challenge-prefix-program
      [OF controlled i-bound])
  have target:
    hash-target-program B
    (staged-trace-fri-search-queries budgets i + 1) ?m
  by (rule hash-target-program-bind[OF prefix-target])
    (rule hash-target-program-receive-trace-fri-challenge)
  have hit-bound:
    wp-event ?m ?hit s ≤
    staged-phase-target-error B
    (staged-trace-fri-search-queries budgets i + 1)
  using target unfolding hash-target-program-def hash-target-budget-def
    staged-phase-target-error-def by blast
  show ?thesis
    using order-trans[OF bad-imp-hit hit-bound] by simp
qed

lemma staged-composition-fri-prefix-receive-bad-value-imp-new-output-hit:
  assumes wf: staged-budget-wellformed budgets
    and controlled: staged-adversary-controlled budgets A
    and i-bound: i < ceil-log (maxDegree + 1)
    and absent: composition-fri-challenge-values-absent B s
    and out:
      Some (b, t) ∈
        set-dist
          (execute
            (staged-composition-fri-challenge-prefix-program A i >>=
              (λ-. receive-composition-fri-challenge)) s)
    and bad: b ∈ B
  shows hash-new-output-hit-event B s (Some (b, t))
proof -
  obtain prefix-out u where prefix:
    Some (prefix-out, u) ∈
      set-dist
        (execute (staged-composition-fri-challenge-prefix-program A i) s)
    and recv:
      Some (b, t) ∈
        set-dist (execute receive-composition-fri-challenge u)
  using out by (auto elim!: set-dist-bindE)
  have prefix-ext: s ≤ u
  using hash-target-program-staged-composition-fri-challenge-prefix-program
    [OF wf controlled i-bound, of B] prefix
  unfolding hash-target-program-def hash-extension-preserving-def
  by blast
  have recv-out:
    u ≤ t ∧
    fmlookup (HashMap t)
      (CompositionFriChallenge (PCompositionFriCounter u) (PState u)) =

```

```

    Some b
  using receive-composition-fri-challenge-outcome[OF recv] by blast
show ?thesis
proof (cases
  fmlookup (HashMap s)
    (CompositionFriChallenge (PCompositionFriCounter u) (PState u)))
case None
then have hash-map-new-output-hit B s t
  unfolding hash-map-new-output-hit-def
  using recv-out bad by blast
then show ?thesis
  unfolding hash-new-output-hit-event-def by simp
next
case (Some old-b)
have old-lookup-t:
  fmlookup (HashMap t)
    (CompositionFriChallenge (PCompositionFriCounter u) (PState u)) =
  Some old-b
  by (rule hash-extension-lookup[OF Some])
  (rule hash-ext-trans[OF prefix-ext conjunct1[OF recv-out]])
then have old-eq: old-b = b
  using recv-out by simp
have old-b ∉ B
  using absent Some
  unfolding composition-fri-challenge-values-absent-def by blast
then show ?thesis
  using old-eq bad by simp
qed
qed

lemma staged-composition-fri-prefix-recv-bad-value-bound:
  assumes wf: staged-budget-wellformed budgets
  and controlled: staged-adversary-controlled budgets A
  and i-bound: i < ceil-log (maxDegree + 1)
  and absent: composition-fri-challenge-values-absent B s
  shows
    wp-event
      (staged-composition-fri-challenge-prefix-program A i ≫=
        (λ-. receive-composition-fri-challenge))
      (λout. case out of None ⇒ False
        | Some (b, -) ⇒ b ∈ B) s ≤
      staged-phase-target-error B
      (Suc (staged-composition-fri-search-queries budgets i))
proof -
  let ?m =
    staged-composition-fri-challenge-prefix-program A i ≫=
      (λ-. receive-composition-fri-challenge)
  let ?bad =
    λout. case out of None ⇒ False

```

```

    | Some (b, -)  $\Rightarrow$   $b \in B$ 
  let ?hit = hash-new-output-hit-event B s
  have bad-imp-hit:
    wp-event ?m ?bad s  $\leq$  wp-event ?m ?hit s
  proof (rule wp-event-mono-on-support)
    fix out
    assume support: out  $\in$  set-dist (execute ?m s)
    and bad: ?bad out
    show ?hit out
  proof (cases out)
    case None
    then show ?thesis using bad by simp
  next
    case (Some bt)
    then obtain b t where out-eq: out = Some (b, t)
    by (cases bt) simp
    have out':
      Some (b, t)  $\in$  set-dist (execute ?m s)
    using support out-eq by simp
    have b-in:  $b \in B$ 
    using bad out-eq by simp
    show ?thesis
    unfolding out-eq
    by (rule
      staged-composition-fri-prefix-receive-bad-value-imp-new-output-hit
      [OF wf controlled i-bound absent out' b-in])
  qed
qed
have prefix-target:
  hash-target-program B
  (staged-composition-fri-search-queries budgets i)
  (staged-composition-fri-challenge-prefix-program A i)
  by (rule
    hash-target-program-staged-composition-fri-challenge-prefix-program
    [OF wf controlled i-bound])
have target:
  hash-target-program B
  (staged-composition-fri-search-queries budgets  $i + 1$ ) ?m
  by (rule hash-target-program-bind [OF prefix-target])
  (rule hash-target-program-receive-composition-fri-challenge)
have hit-bound:
  wp-event ?m ?hit s  $\leq$ 
  staged-phase-target-error B
  (staged-composition-fri-search-queries budgets  $i + 1$ )
  using target unfolding hash-target-program-def hash-target-budget-def
  staged-phase-target-error-def by blast
show ?thesis
  using order-trans [OF bad-imp-hit hit-bound] by simp
qed

```

lemma *staged-query-prefix-receive-bad-index-imp-new-output-hit*:
assumes *wf*: *staged-budget-wellformed budgets*
and *controlled*: *staged-adversary-controlled budgets A*
and *i-bound*: $i < \text{rounds}$
and *absent*: *query-index-raw-preimage-absent B s*
and *out*:
 Some (*raw*, *t*) $\in$
 set-dist
 (*execute*
 (*staged-query-challenge-prefix-program A i* $\gg$
 ($\lambda\cdot$. *receive-query-index-challenge*)) *s*)
and *bad*: *index (to-nat raw) $\in$ B*
shows
 hash-new-output-hit-event (query-index-raw-preimage B) s
 (*Some* (*raw*, *t*))
proof –
obtain *prefix-out u* **where** *prefix*:
 Some (*prefix-out*, *u*) $\in$
 set-dist (*execute* (*staged-query-challenge-prefix-program A i*) *s*)
and *recv*:
 Some (*raw*, *t*) $\in$
 set-dist (*execute receive-query-index-challenge u*)
using *out* **by** (*auto elim!*: *set-dist-bindE*)
have *prefix-ext*:
 $s \leq u$
using *hash-target-program-staged-query-challenge-prefix-program*
 [*OF wf controlled i-bound, of query-index-raw-preimage B*]
 prefix
unfolding *hash-target-program-def hash-extension-preserving-def*
by *blast*
have *recv-out*:
 $u \leq t \wedge$
 fmlookup (*HashMap t*)
 (*QueryIndexChallenge* (*PQueryCounter u*) (*PState u*)) = *Some raw*
using *receive-query-index-challenge-outcome*[*OF recv*] **by** *blast*
have *raw-in*: *raw $\in$ query-index-raw-preimage B*
using *bad* **unfolding** *query-index-raw-preimage-def* **by** *simp*
show *?thesis*
proof (*cases*
 fmlookup (*HashMap s*)
 (*QueryIndexChallenge* (*PQueryCounter u*) (*PState u*)))
case *None*
then have *hash-map-new-output-hit (query-index-raw-preimage B) s t*
 unfolding *hash-map-new-output-hit-def*
 using *recv-out raw-in* **by** *blast*
then show *?thesis*
 unfolding *hash-new-output-hit-event-def* **by** *simp*
next

```

case (Some old-row)
have old-lookup-t:
  fmlookup (HashMap t)
    (QueryIndexChallenge (PQueryCounter u) (PState u)) =
    Some old-row
by (rule hash-extension-lookup[OF Some])
    (rule hash-ext-trans[OF prefix-ext conjunct1[OF recv-out]])
then have old-eq: old-row = raw
  using recv-out by simp
have old-row  $\notin$  query-index-row-preimage B
  using absent Some
  unfolding query-index-row-preimage-absent-def by blast
then show ?thesis
  using old-eq raw-in by simp
qed
qed

lemma staged-query-prefix-receive-bad-index-bound:
assumes wf: staged-budget-wellformed budgets
and controlled: staged-adversary-controlled budgets A
and i-bound:  $i \leq \text{rounds}$ 
and absent: query-index-row-preimage-absent B s
shows
  wp-event
    (staged-query-challenge-prefix-program A i  $\gg$ 
      ( $\lambda$ -. receive-query-index-challenge))
    ( $\lambda$ out. case out of None  $\Rightarrow$  False
      | Some (raw, -)  $\Rightarrow$  index (to-nat raw)  $\in$  B) s  $\leq$ 
    staged-phase-target-error (query-index-row-preimage B)
      (Suc (staged-query-search-queries budgets i))

proof -
let ?m =
  staged-query-challenge-prefix-program A i  $\gg$ 
    ( $\lambda$ -. receive-query-index-challenge)
let ?B = query-index-row-preimage B
let ?bad =
   $\lambda$ out. case out of None  $\Rightarrow$  False
    | Some (raw, -)  $\Rightarrow$  index (to-nat raw)  $\in$  B
let ?hit = hash-new-output-hit-event ?B s
have bad-imp-hit:
  wp-event ?m ?bad s  $\leq$  wp-event ?m ?hit s
proof (rule wp-event-mono-on-support)
fix out
assume support: out  $\in$  set-dist (execute ?m s)
and bad: ?bad out
show ?hit out
proof (cases out)
case None
then show ?thesis using bad by simp

```

```

next
  case (Some rt)
  then obtain raw t where out-eq: out = Some (raw, t)
    by (cases rt) simp
  have out':
    Some (raw, t) ∈ set-dist (execute ?m s)
    using support out-eq by simp
  have bad-raw: index (to-nat raw) ∈ B
    using bad out-eq by simp
  show ?thesis
    unfolding out-eq
    by (rule staged-query-prefix-receive-bad-index-imp-new-output-hit
      [OF wf controlled i-bound absent out' bad-raw])
  qed
qed
have prefix-target:
  hash-target-program ?B
    (staged-query-search-queries budgets i)
    (staged-query-challenge-prefix-program A i)
  by (rule hash-target-program-staged-query-challenge-prefix-program
    [OF wf controlled i-bound])
have target:
  hash-target-program ?B
    (staged-query-search-queries budgets i + 1) ?m
  by (rule hash-target-program-bind [OF prefix-target])
    (rule hash-target-program-receive-query-index-challenge)
have hit-bound:
  wp-event ?m ?hit s ≤
    staged-phase-target-error ?B
    (staged-query-search-queries budgets i + 1)
  using target unfolding hash-target-program-def hash-target-budget-def
    staged-phase-target-error-def by blast
show ?thesis
  using order-trans [OF bad-imp-hit hit-bound] by simp
qed

lemma receive-alpha-challenge-bad-value-imp-new-output-hit:
assumes absent: alpha-challenge-values-absent B s
and out:
  Some (a, t) ∈ set-dist (execute receive-alpha-challenge s)
and bad: a ∈ B
shows hash-new-output-hit-event B s (Some (a, t))
proof –
have recv-out:
  s ≤ t ∧
    fmlookup (HashMap t)
      (AlphaChallenge (PAlphaCounter s) (PState s)) = Some a
  using receive-alpha-challenge-outcome [OF out] by blast
show ?thesis

```



```

proof (cases
  fmlookup (HashMap s)
    (AlphaChallenge (PAlphaCounter s) (PState s)))
case None
then have hash-map-new-output-hit B s t
  unfolding hash-map-new-output-hit-def
  using recv-out bad by blast
then show ?thesis
  unfolding hash-new-output-hit-event-def by simp
next
case (Some old-a)
have old-lookup-t:
  fmlookup (HashMap t)
    (AlphaChallenge (PAlphaCounter s) (PState s)) = Some old-a
  by (rule hash-extension-lookup[OF Some conjunct1[OF recv-out]])
then have old-eq: old-a = a
  using recv-out by simp
have old-a  $\notin$  B
  using absent Some
  unfolding alpha-challenge-values-absent-def by blast
then show ?thesis
  using old-eq bad by simp
qed
qed

lemma receive-alpha-challenge-bad-value-bound:
assumes absent: alpha-challenge-values-absent B s
shows
  wp-event receive-alpha-challenge
    ( $\lambda out. \text{case } out \text{ of } None \Rightarrow False$ 
      |  $\text{Some } (a, -) \Rightarrow a \in B$ ) s  $\leq$ 
    staged-phase-target-error B 1
proof -
let ?bad =
   $\lambda out. \text{case } out \text{ of } None \Rightarrow False$ 
    |  $\text{Some } (a, -) \Rightarrow a \in B$ 
let ?hit = hash-new-output-hit-event B s
have bad-imp-hit:
  wp-event receive-alpha-challenge ?bad s  $\leq$ 
    wp-event receive-alpha-challenge ?hit s
proof (rule wp-event-mono-on-support)
fix out
assume support: out  $\in$  set-dist (execute receive-alpha-challenge s)
  and bad: ?bad out
show ?hit out
proof (cases out)
  case None
  then show ?thesis using bad by simp
next

```

```

case (Some pair)
obtain a t where pair-eq: pair = (a, t)
  by (cases pair)
have out-eq: out = Some (a, t)
  using Some pair-eq by simp
have out':
  Some (a, t) ∈ set-dist (execute receive-alpha-challenge s)
  using support out-eq by simp
have a-in: a ∈ B
  using bad out-eq by simp
show ?thesis
  unfolding out-eq
  by (rule receive-alpha-challenge-bad-value-imp-new-output-hit
    [OF absent out' a-in])
qed
qed
have target:
  hash-target-program B 1 receive-alpha-challenge
  by (rule hash-target-program-receive-alpha-challenge)
have hit-bound:
  wp-event receive-alpha-challenge ?hit s ≤
  staged-phase-target-error B 1
  using target unfolding hash-target-program-def hash-target-budget-def
  staged-phase-target-error-def by blast
show ?thesis
  using order-trans[OF bad-imp-hit hit-bound] .
qed

lemma staged-alpha-prefix-receive-bad-value-imp-new-output-hit:
assumes wf: staged-budget-wellformed budgets
and controlled: staged-adversary-controlled budgets A
and i-bound: i ≤ length spec
and absent: alpha-challenge-values-absent B s
and out:
  Some (a, t) ∈
  set-dist
  (execute
    (staged-alpha-challenge-prefix-program A i  $\gg$ 
      ( $\lambda\cdot$ . receive-alpha-challenge)) s)
and bad: a ∈ B
shows hash-new-output-hit-event B s (Some (a, t))
proof –
obtain prefix-out u where prefix:
  Some (prefix-out, u) ∈
  set-dist (execute (staged-alpha-challenge-prefix-program A i) s)
and recv:
  Some (a, t) ∈ set-dist (execute receive-alpha-challenge u)
  using out by (auto elim!: set-dist-bindE)
have prefix-ext:

```

```

 $s \leq u$ 
using hash-target-program-staged-alpha-challenge-prefix-program
  [OF wf controlled i-bound, of B] prefix
unfolding hash-target-program-def hash-extension-preserving-def
by blast
have recv-out:
 $u \leq t \wedge$ 
  fmlookup (HashMap t)
    (AlphaChallenge (PAlphaCounter u) (PState u)) = Some a
using receive-alpha-challenge-outcome[OF recv] by blast
show ?thesis
proof (cases
  fmlookup (HashMap s)
    (AlphaChallenge (PAlphaCounter u) (PState u)))
case None
then have hash-map-new-output-hit B s t
  unfolding hash-map-new-output-hit-def
  using recv-out bad by blast
then show ?thesis
  unfolding hash-new-output-hit-event-def by simp
next
case (Some old-a)
have old-lookup-t:
  fmlookup (HashMap t)
    (AlphaChallenge (PAlphaCounter u) (PState u)) = Some old-a
by (rule hash-extension-lookup[OF Some])
  (rule hash-ext-trans[OF prefix-ext conjunct1[OF recv-out]])
then have old-eq: old-a = a
  using recv-out by simp
have old-a  $\notin$  B
  using absent Some
  unfolding alpha-challenge-values-absent-def by blast
then show ?thesis
  using old-eq bad by simp
qed
qed

lemma staged-alpha-prefix-receive-bad-value-bound:
assumes wf: staged-budget-wellformed budgets
and controlled: staged-adversary-controlled budgets A
and i-bound:  $i \leq \text{length spec}$ 
and absent: alpha-challenge-values-absent B s
shows
  wp-event
    (staged-alpha-challenge-prefix-program A i  $\gg=$ 
      ( $\lambda$ -. receive-alpha-challenge))
    ( $\lambda$ out. case out of None  $\Rightarrow$  False
      | Some (a, -)  $\Rightarrow$   $a \in B$ )  $s \leq$ 
    staged-phase-target-error B

```

```

      (Suc (staged-alpha-search-queries budgets i))
proof -
  let ?m =
    staged-alpha-challenge-prefix-program A i  $\gg$ 
    ( $\lambda$ -. receive-alpha-challenge)
  let ?bad =
     $\lambda$ out. case out of None  $\Rightarrow$  False
      | Some (a, -)  $\Rightarrow$  a  $\in$  B
  let ?hit = hash-new-output-hit-event B s
  have bad-imp-hit:
    wp-event ?m ?bad s  $\leq$  wp-event ?m ?hit s
  proof (rule wp-event-mono-on-support)
    fix out
    assume support: out  $\in$  set-dist (execute ?m s)
    and bad: ?bad out
    show ?hit out
    proof (cases out)
      case None
      then show ?thesis using bad by simp
    next
      case (Some pair)
      obtain a t where pair-eq: pair = (a, t)
      by (cases pair)
      have out-eq: out = Some (a, t)
      using Some pair-eq by simp
      have out':
        Some (a, t)  $\in$  set-dist (execute ?m s)
      using support out-eq by simp
      have a-in: a  $\in$  B
      using bad out-eq by simp
      show ?thesis
      unfolding out-eq
      by (rule staged-alpha-prefix-receive-bad-value-imp-new-output-hit
        [OF wf controlled i-bound absent out' a-in])
    qed
  qed
  have prefix-target:
    hash-target-program B (staged-alpha-search-queries budgets i)
    (staged-alpha-challenge-prefix-program A i)
    by (rule hash-target-program-staged-alpha-challenge-prefix-program
      [OF wf controlled i-bound])
  have target:
    hash-target-program B
    (staged-alpha-search-queries budgets i + 1) ?m
    by (rule hash-target-program-bind[OF prefix-target])
    (rule hash-target-program-receive-alpha-challenge)
  have hit-bound:
    wp-event ?m ?hit s  $\leq$ 
    staged-phase-target-error B

```

```

    (staged-alpha-search-queries budgets i + 1)
  using target unfolding hash-target-program-def hash-target-budget-def
    staged-phase-target-error-def by blast
show ?thesis
  using order-trans[OF bad-imp-hit hit-bound] by simp
qed

lemma hash-range-budget-staged-transcript-program:
  assumes wf: staged-budget-wellformed budgets
    and controlled: staged-adversary-controlled budgets A
  shows
    hash-range-budget
      (staged-attacker-query-budget budgets + staged-challenge-query-budget)
      (staged-transcript-program A)
proof -
  let ?trace-root = trace-root-budget budgets
  let ?trace-fri =
    sum-list (trace-fri-budgets budgets) + ceil-log clength
  let ?trace-final = trace-final-budget budgets
  let ?alpha = length spec
  let ?degree = degree-budget budgets
  let ?composition-fri =
    sum-list (composition-fri-budgets budgets) +
    ceil-log (maxDegree + 1)
  let ?composition-final = composition-final-budget budgets
  let ?query =
    sum-list (query-opening-budgets budgets) + rounds
  have trace-root-range:
    hash-range-budget ?trace-root (trace-root-stage A)
    using controlled unfolding staged-adversary-controlled-def
    by (blast intro: controlled-ro-program-range)
  have trace-fri-range:
    hash-range-budget ?trace-fri
      (staged-trace-fri-program A 0 (ceil-log clength) [])
proof -
  have len:
    0 + ceil-log clength ≤ length (trace-fri-budgets budgets)
    using wf unfolding staged-budget-wellformed-def by simp
  have exact:
    hash-range-budget
      (sum-list
        (take (ceil-log clength)
          (drop 0 (trace-fri-budgets budgets)))) +
      ceil-log clength
      (staged-trace-fri-program A 0 (ceil-log clength) [])
    by (rule hash-range-budget-staged-trace-fri-program
      [OF controlled len])
  then show ?thesis
    using wf unfolding staged-budget-wellformed-def by simp

```

```

qed
have trace-final-range:
   $\wedge$  bs. hash-range-budget ?trace-final (trace-final-stage A bs)
  using controlled unfolding staged-adversary-controlled-def
  by (blast intro: controlled-ro-program-range)
have alpha-range:
  hash-range-budget ?alpha (staged-alpha-program (length spec))
  by (rule hash-range-budget-staged-alpha-program)
have degree-range:
   $\wedge$  as. hash-range-budget ?degree (degree-stage A as)
  using controlled unfolding staged-adversary-controlled-def
  by (blast intro: controlled-ro-program-range)
have composition-fri-range:
   $\wedge$  dg. hash-range-budget ?composition-fri
    (assert (ceil-log (to-nat dg + 1)  $\leq$  ceil-log (maxDegree + 1))  $\gg$ 
      ( $\lambda$ -. staged-composition-fri-program A dg 0
        (ceil-log (to-nat dg + 1)) []))
  by (rule hash-range-budget-guarded-staged-composition-fri-program
    [OF wf controlled])
have composition-final-range:
   $\wedge$  dg bs. hash-range-budget ?composition-final
    (composition-final-stage A dg bs)
  using controlled unfolding staged-adversary-controlled-def
  by (blast intro: controlled-ro-program-range)
have query-range:
  hash-range-budget ?query (staged-query-program A 0 rounds)
proof -
  have len:
    0 + rounds  $\leq$  length (query-opening-budgets budgets)
    using wf unfolding staged-budget-wellformed-def by simp
  have exact:
    hash-range-budget
      (sum-list
        (take rounds (drop 0 (query-opening-budgets budgets))) +
        rounds)
      (staged-query-program A 0 rounds)
    by (rule hash-range-budget-staged-query-program[OF controlled len])
  then show ?thesis
    using wf unfolding staged-budget-wellformed-def by simp
qed
have query-tail:
   $\wedge$  F. hash-range-budget (?query + 0)
    (staged-query-program A 0 rounds  $\gg$ 
      ( $\lambda$ query-chunks. return (F query-chunks)))
  by (rule hash-range-budget-bind)
    (rule query-range, rule hash-range-budget-return)
have after-composition-final-record:
   $\wedge$  composition-final F. hash-range-budget (0 + (?query + 0))
    (record-staged-message composition-final  $\gg$ 

```

```

    (λ-. staged-query-program A 0 rounds >>=
      (λquery-chunks. return (F query-chunks))))
  by (rule hash-range-budget-bind)
    (rule hash-range-budget-record-staged-message, rule query-tail)
have after-composition-final:
  ∧dg bs F. hash-range-budget
    (?composition-final + (0 + (?query + 0)))
    (composition-final-stage A dg bs >>=
      (λcomposition-final.
        record-staged-message composition-final >>=
          (λ-. staged-query-program A 0 rounds >>=
            (λquery-chunks.
              return (F composition-final query-chunks))))))
  by (rule hash-range-budget-bind)
    (rule composition-final-range, rule after-composition-final-record)
have after-composition-fri:
  ∧dg F. hash-range-budget
    (?composition-fri +
      (?composition-final + (0 + (?query + 0))))
    ((assert (ceil-log (to-nat dg + 1) ≤
      ceil-log (maxDegree + 1)) >>=
      (λ-. staged-composition-fri-program A dg 0
        (ceil-log (to-nat dg + 1)) [])) >>=
      (λ(composition-roots, composition-bs).
        composition-final-stage A dg composition-bs >>=
          (λcomposition-final.
            record-staged-message composition-final >>=
              (λ-. staged-query-program A 0 rounds >>=
                (λquery-chunks.
                  return
                    (F composition-roots composition-bs composition-final
                      query-chunks)))))))
proof (rule hash-range-budget-bind)
  fix dg F
  show hash-range-budget ?composition-fri
    (assert (ceil-log (to-nat dg + 1) ≤
      ceil-log (maxDegree + 1)) >>=
      (λ-. staged-composition-fri-program A dg 0
        (ceil-log (to-nat dg + 1)) []))
    by (rule composition-fri-range)
  fix dg F x
  show hash-range-budget
    (?composition-final + (0 + (?query + 0)))
    (case x of (composition-roots, composition-bs) ⇒
      composition-final-stage A dg composition-bs >>=
        (λcomposition-final.
          record-staged-message composition-final >>=
            (λ-. staged-query-program A 0 rounds >>=
              (λquery-chunks.

```

```

      return
      (F composition-roots composition-bs composition-final
       query-chunks))))))
proof (cases x)
  case (Pair composition-roots composition-bs)
  have hash-range-budget
    (?composition-final + (0 + (?query + 0)))
    (composition-final-stage A dg composition-bs  $\gg$ 
     (λcomposition-final.
      record-staged-message composition-final  $\gg$ 
      (λ-. staged-query-program A 0 rounds  $\gg$ 
       (λquery-chunks.
        return
        (F composition-roots composition-bs composition-final
         query-chunks))))))
  by (rule after-composition-final)
  then show ?thesis
  unfolding Pair by simp
qed
qed
have after-degree-record:
  ∧dg F. hash-range-budget
    (0 +
     (?composition-fri +
      (?composition-final + (0 + (?query + 0)))))
  (record-staged-message dg  $\gg$ 
   (λ-. (assert (ceil-log (to-nat dg + 1) ≤
    ceil-log (maxDegree + 1))  $\gg$ 
    (λ-. staged-composition-fri-program A dg 0
     (ceil-log (to-nat dg + 1)) []))  $\gg$ 
    (λ(composition-roots, composition-bs).
     composition-final-stage A dg composition-bs  $\gg$ 
     (λcomposition-final.
      record-staged-message composition-final  $\gg$ 
      (λ-. staged-query-program A 0 rounds  $\gg$ 
       (λquery-chunks.
        return
        (F composition-roots composition-bs composition-final
         query-chunks)))))))))
  by (rule hash-range-budget-bind)
  (rule hash-range-budget-record-staged-message, rule after-composition-fri)
have after-degree:
  ∧as F. hash-range-budget
    (?degree +
     (0 +
      (?composition-fri +
       (?composition-final + (0 + (?query + 0)))))
    (degree-stage A as  $\gg$ 
     (λdg. record-staged-message dg  $\gg$ 

```



```

(λ-. (assert (ceil-log (to-nat dg + 1) ≤
  ceil-log (maxDegree + 1)) >>=
  (λ-. staged-composition-fri-program A dg 0
    (ceil-log (to-nat dg + 1)) [])) >>=
  (λ(composition-roots, composition-bs).
    composition-final-stage A dg composition-bs >>=
    (λcomposition-final.
      record-staged-message composition-final >>=
      (λ-. staged-query-program A 0 rounds >>=
        (λquery-chunks.
          return
            (F dg composition-roots composition-bs
              composition-final query-chunks)))))))))
by (rule hash-range-budget-bind)
  (rule degree-range, rule after-degree-record)
have after-alpha:
  ∧F. hash-range-budget
  (?alpha +
    (?degree +
      (0 +
        (?composition-fri +
          (?composition-final + (0 + (?query + 0)))))))
  (staged-alpha-program (length spec) >>=
    (λas. degree-stage A as >>=
      (λdg. record-staged-message dg >>=
        (λ-. (assert (ceil-log (to-nat dg + 1) ≤
          ceil-log (maxDegree + 1)) >>=
            (λ-. staged-composition-fri-program A dg 0
              (ceil-log (to-nat dg + 1)) [])) >>=
              (λ(composition-roots, composition-bs).
                composition-final-stage A dg composition-bs >>=
                (λcomposition-final.
                  record-staged-message composition-final >>=
                  (λ-. staged-query-program A 0 rounds >>=
                    (λquery-chunks.
                      return
                        (F as dg composition-roots composition-bs
                          composition-final query-chunks)))))))))
        (rule hash-range-budget-bind)
        (rule alpha-range, rule after-degree)
have after-trace-final-record:
  ∧trace-final F. hash-range-budget
  (0 +
    (?alpha +
      (?degree +
        (0 +
          (?composition-fri +
            (?composition-final + (0 + (?query + 0)))))))
  (record-staged-message trace-final >>=

```

```

(λ-. staged-alpha-program (length spec) >>=
  (λas. degree-stage A as >>=
    (λdg. record-staged-message dg >>=
      (λ-. (assert (ceil-log (to-nat dg + 1) ≤
        ceil-log (maxDegree + 1)) >>=
        (λ-. staged-composition-fri-program A dg 0
          (ceil-log (to-nat dg + 1)) [])) >>=
        (λ(composition-roots, composition-bs).
          composition-final-stage A dg composition-bs >>=
            (λcomposition-final.
              record-staged-message composition-final >>=
                (λ-. staged-query-program A 0 rounds >>=
                  (λquery-chunks.
                    return
                      (F as dg composition-roots composition-bs
                        composition-final query-chunks))))))))))
  by (rule hash-range-budget-bind)
  (rule hash-range-budget-record-staged-message, rule after-alpha)
have after-trace-final:
  ∧ trace-bs F. hash-range-budget
  (?trace-final +
    (0 +
      (?alpha +
        (?degree +
          (0 +
            (?composition-fri +
              (?composition-final + (0 + (?query + 0))))))))))
  (trace-final-stage A trace-bs >>=
    (λtrace-final. record-staged-message trace-final >>=
      (λ-. staged-alpha-program (length spec) >>=
        (λas. degree-stage A as >>=
          (λdg. record-staged-message dg >>=
            (λ-. (assert (ceil-log (to-nat dg + 1) ≤
              ceil-log (maxDegree + 1)) >>=
              (λ-. staged-composition-fri-program A dg 0
                (ceil-log (to-nat dg + 1)) [])) >>=
              (λ(composition-roots, composition-bs).
                composition-final-stage A dg composition-bs >>=
                  (λcomposition-final.
                    record-staged-message composition-final >>=
                      (λ-. staged-query-program A 0 rounds >>=
                        (λquery-chunks.
                          return
                            (F trace-final as dg composition-roots
                              composition-bs composition-final
                                query-chunks))))))))))
          by (rule hash-range-budget-bind)
          (rule trace-final-range, rule after-trace-final-record)
have after-trace-fri:

```

```

 $\wedge F.$  hash-range-budget
  (?trace-fri +
    (?trace-final +
      (0 +
        (?alpha +
          (?degree +
            (0 +
              (?composition-fri +
                (?composition-final + (0 + (?query + 0))))))))))
  (staged-trace-fri-program A 0 (ceil-log clength) [])  $\gg$ 
  ( $\lambda$ (trace-roots, trace-bs).
    trace-final-stage A trace-bs  $\gg$ 
    ( $\lambda$ trace-final. record-staged-message trace-final  $\gg$ 
      ( $\lambda$ -. staged-alpha-program (length spec)  $\gg$ 
        ( $\lambda$ as. degree-stage A as  $\gg$ 
          ( $\lambda$ dg. record-staged-message dg  $\gg$ 
            ( $\lambda$ -. (assert (ceil-log (to-nat dg + 1))  $\leq$ 
              ceil-log (maxDegree + 1))  $\gg$ 
              ( $\lambda$ -. staged-composition-fri-program A dg 0
                (ceil-log (to-nat dg + 1)) []))  $\gg$ 
              ( $\lambda$ (composition-roots, composition-bs).
                composition-final-stage A dg composition-bs  $\gg$ 
                ( $\lambda$ composition-final.
                  record-staged-message composition-final  $\gg$ 
                  ( $\lambda$ -. staged-query-program A 0 rounds  $\gg$ 
                    ( $\lambda$ query-chunks.
                      return
                        (F trace-roots trace-bs trace-final as dg
                          composition-roots composition-bs
                          composition-final query-chunks))))))))))
proof (rule hash-range-budget-bind)
  fix F
  show hash-range-budget ?trace-fri
    (staged-trace-fri-program A 0 (ceil-log clength) [])
  by (rule trace-fri-range)
  fix F x
  show hash-range-budget
    (?trace-final +
      (0 +
        (?alpha +
          (?degree +
            (0 +
              (?composition-fri +
                (?composition-final + (0 + (?query + 0))))))))))
  (case x of (trace-roots, trace-bs)  $\Rightarrow$ 
    trace-final-stage A trace-bs  $\gg$ 
    ( $\lambda$ trace-final. record-staged-message trace-final  $\gg$ 
      ( $\lambda$ -. staged-alpha-program (length spec)  $\gg$ 
        ( $\lambda$ as. degree-stage A as  $\gg$ 

```

```

      (λdg. record-staged-message dg ≫=
        (λ-. (assert (ceil-log (to-nat dg + 1) ≤
          ceil-log (maxDegree + 1)) ≫=
            (λ-. staged-composition-fri-program A dg 0
              (ceil-log (to-nat dg + 1)) [])) ≫=
              (λ(composition-roots, composition-bs).
                composition-final-stage A dg composition-bs ≫=
                  (λcomposition-final.
                    record-staged-message composition-final ≫=
                      (λ-. staged-query-program A 0 rounds ≫=
                        (λquery-chunks.
                          return
                            (F trace-roots trace-bs trace-final as dg
                              composition-roots composition-bs
                                composition-final query-chunks))))))))))
    proof (cases x)
    case (Pair trace-roots trace-bs)
    have hash-range-budget
      (?trace-final +
        (0 +
          (?alpha +
            (?degree +
              (0 +
                (?composition-fri +
                  (?composition-final + (0 + (?query + 0))))))))))
    (trace-final-stage A trace-bs ≫=
      (λtrace-final. record-staged-message trace-final ≫=
        (λ-. staged-alpha-program (length spec) ≫=
          (λas. degree-stage A as ≫=
            (λdg. record-staged-message dg ≫=
              (λ-. (assert (ceil-log (to-nat dg + 1) ≤
                ceil-log (maxDegree + 1)) ≫=
                  (λ-. staged-composition-fri-program A dg 0
                    (ceil-log (to-nat dg + 1)) [])) ≫=
                    (λ(composition-roots, composition-bs).
                      composition-final-stage A dg composition-bs ≫=
                        (λcomposition-final.
                          record-staged-message composition-final ≫=
                            (λ-. staged-query-program A 0 rounds ≫=
                              (λquery-chunks.
                                return
                                  (F trace-roots trace-bs trace-final as dg
                                    composition-roots composition-bs
                                      composition-final query-chunks))))))))))
          by (rule after-trace-final)
        then show ?thesis
          unfolding Pair by simp
      qed
    qed

```

```

have after-trace-root-record:
   $\wedge$ fr F. hash-range-budget
    (0 +
      (?trace-fri +
        (?trace-final +
          (0 +
            (?alpha +
              (?degree +
                (0 +
                  (?composition-fri +
                    (?composition-final + (0 + (?query + 0))))))))))
      (record-staged-message fr  $\gg$ 
        ( $\lambda$ -. staged-trace-fri-program A 0 (ceil-log clength) []  $\gg$ 
          ( $\lambda$ (trace-roots, trace-bs).
            trace-final-stage A trace-bs  $\gg$ 
              ( $\lambda$ trace-final. record-staged-message trace-final  $\gg$ 
                ( $\lambda$ -. staged-alpha-program (length spec)  $\gg$ 
                  ( $\lambda$ as. degree-stage A as  $\gg$ 
                    ( $\lambda$ dg. record-staged-message dg  $\gg$ 
                      ( $\lambda$ -. (assert (ceil-log (to-nat dg + 1)  $\leq$ 
                        ceil-log (maxDegree + 1))  $\gg$ 
                        ( $\lambda$ -. staged-composition-fri-program A dg 0
                          (ceil-log (to-nat dg + 1) []))  $\gg$ 
                        ( $\lambda$ (composition-roots, composition-bs).
                          composition-final-stage A dg composition-bs  $\gg$ 
                            ( $\lambda$ composition-final.
                              record-staged-message composition-final  $\gg$ 
                                ( $\lambda$ -. staged-query-program A 0 rounds  $\gg$ 
                                  ( $\lambda$ query-chunks.
                                    return
                                      (F trace-roots trace-bs trace-final as dg
                                        composition-roots composition-bs
                                        composition-final query-chunks))))))))))
          by (rule hash-range-budget-bind)
            (rule hash-range-budget-record-staged-message, rule after-trace-fri)
        have whole:
          hash-range-budget
            (?trace-root +
              (0 +
                (?trace-fri +
                  (?trace-final +
                    (0 +
                      (?alpha +
                        (?degree +
                          (0 +
                            (?composition-fri +
                              (?composition-final + (0 + (?query + 0))))))))))
              (staged-transcript-program A)
            unfolding staged-transcript-program-def Let-def

```

```

proof (rule hash-range-budget-bind)
  show hash-range-budget ?trace-root (trace-root-stage A)
    by (rule trace-root-range)
fix fr
show hash-range-budget
  (0 +
    (?trace-fri +
      (?trace-final +
        (0 +
          (?alpha +
            (?degree +
              (0 +
                (?composition-fri +
                  (?composition-final + (0 + (?query + 0))))))))))
  (record-staged-message fr >=
    (λ-. staged-trace-fri-program A 0 (ceil-log clength) [] >=
      (λ(trace-roots, trace-bs).
        trace-final-stage A trace-bs >=
          (λtrace-final.
            record-staged-message trace-final >=
              (λ-. staged-alpha-program (length spec) >=
                (λas. degree-stage A as >=
                  (λdg. record-staged-message dg >=
                    (λ-. assert (ceil-log (to-nat dg + 1) ≤
                      ceil-log (maxDegree + 1)) >=
                    (λ-. staged-composition-fri-program A dg 0
                      (ceil-log (to-nat dg + 1)) [] >=
                    (λ(composition-roots, composition-bs).
                      composition-final-stage A dg composition-bs >=
                    (λcomposition-final.
                      record-staged-message
                        composition-final >=
                    (λ-. staged-query-program A 0
                      rounds >=
                    (λquery-chunks.
                      return
                        (|staged-trace-root = fr,
                          staged-trace-fri-roots =
                            trace-roots,
                          staged-trace-fri-challenges =
                            trace-bs,
                          staged-trace-final =
                            trace-final,
                          staged-alphas = as,
                          staged-degree = dg,
                          staged-composition-fri-roots =
                            composition-roots,
                          staged-composition-fri-challenges =
                            composition-bs,

```

```

                                staged-composition-final =
                                composition-final,
                                staged-query-chunks =
                                query-chunks)))))))))))))
using after-trace-root-record
  [of fr
    λtrace-roots trace-bs trace-final as dg composition-roots
      composition-bs composition-final query-chunks.
    (|staged-trace-root = fr,
      staged-trace-fri-roots = trace-roots,
      staged-trace-fri-challenges = trace-bs,
      staged-trace-final = trace-final,
      staged-alphas = as,
      staged-degree = dg,
      staged-composition-fri-roots = composition-roots,
      staged-composition-fri-challenges = composition-bs,
      staged-composition-final = composition-final,
      staged-query-chunks = query-chunks|)]
  by (simp add: sm-bind-assoc)
qed
have budget-eq:
  ?trace-root +
  (0 +
    (?trace-fri +
      (?trace-final +
        (0 +
          (?alpha +
            (?degree +
              (0 +
                (?composition-fri +
                  (?composition-final + (0 + (?query + 0)))))))))) =
    staged-attacker-query-budget budgets + staged-challenge-query-budget
unfolding staged-attacker-query-budget-def
  staged-challenge-query-budget-def
by (simp add: add.assoc add.commute add.left-commute)
show ?thesis
  using whole unfolding budget-eq .
qed

lemma hash-range-budget-staged-semantic-adversary:
  assumes wf: staged-budget-wellformed budgets
  and controlled: staged-adversary-controlled budgets A
  shows
    hash-range-budget
    (staged-attacker-query-budget budgets + staged-challenge-query-budget)
    (staged-semantic-adversary A)
proof -
  have hash-range-budget
    ((staged-attacker-query-budget budgets + staged-challenge-query-budget) +

```

```

0)
(staged-semantic-adversary A)
unfolding staged-semantic-adversary-def
proof (rule hash-range-budget-bind
  [where  $n = \text{staged-attacker-query-budget budgets} +$ 
     $\text{staged-challenge-query-budget}$  and  $n' = 0$ ])
show hash-range-budget
  (staged-attacker-query-budget budgets + staged-challenge-query-budget)
  (staged-transcript-program A)
  by (rule hash-range-budget-staged-transcript-program[OF wf controlled])
show  $\bigwedge x. \text{hash-range-budget } 0 \text{ (return (staged-proof-transcript } x))$ 
  by (rule hash-range-budget-return)
qed
then show ?thesis by simp
qed

lemma hash-collision-budget-staged-transcript-program:
assumes wf: staged-budget-wellformed budgets
and controlled: staged-adversary-controlled budgets A
shows
  hash-collision-budget
    (staged-attacker-query-budget budgets + staged-challenge-query-budget)
    (staged-transcript-program A)
proof –
let ?trace-root = trace-root-budget budgets
let ?trace-fri =
  sum-list (trace-fri-budgets budgets) + ceil-log clength
let ?trace-final = trace-final-budget budgets
let ?alpha = length spec
let ?degree = degree-budget budgets
let ?composition-fri =
  sum-list (composition-fri-budgets budgets) +
  ceil-log (maxDegree + 1)
let ?composition-final = composition-final-budget budgets
let ?query =
  sum-list (query-opening-budgets budgets) + rounds
have trace-root-range:
  hash-range-budget ?trace-root (trace-root-stage A)
using controlled unfolding staged-adversary-controlled-def
by (blast intro: controlled-ro-program-range)
have trace-root-coll:
  hash-collision-budget ?trace-root (trace-root-stage A)
using controlled unfolding staged-adversary-controlled-def
by (blast intro: controlled-ro-program-collision)
have trace-fri-range:
  hash-range-budget ?trace-fri
    (staged-trace-fri-program A 0 (ceil-log clength) [])
proof –
have len:

```



```

    0 + ceil-log clength ≤ length (trace-fri-budgets budgets)
using wf unfolding staged-budget-wellformed-def by simp
have exact:
  hash-range-budget
    (sum-list
      (take (ceil-log clength)
        (drop 0 (trace-fri-budgets budgets))) +
      ceil-log clength)
    (staged-trace-fri-program A 0 (ceil-log clength) [])
by (rule hash-range-budget-staged-trace-fri-program
  [OF controlled len])
then show ?thesis
  using wf unfolding staged-budget-wellformed-def by simp
qed
have trace-fri-coll:
  hash-collision-budget ?trace-fri
    (staged-trace-fri-program A 0 (ceil-log clength) [])
proof –
  have len:
    0 + ceil-log clength ≤ length (trace-fri-budgets budgets)
    using wf unfolding staged-budget-wellformed-def by simp
  have exact:
    hash-collision-budget
      (sum-list
        (take (ceil-log clength)
          (drop 0 (trace-fri-budgets budgets))) +
        ceil-log clength)
      (staged-trace-fri-program A 0 (ceil-log clength) [])
    by (rule hash-collision-budget-staged-trace-fri-program
      [OF controlled len])
  then show ?thesis
    using wf unfolding staged-budget-wellformed-def by simp
qed
have trace-final-range:
  ∧ bs. hash-range-budget ?trace-final (trace-final-stage A bs)
  using controlled unfolding staged-adversary-controlled-def
  by (blast intro: controlled-ro-program-range)
have trace-final-coll:
  ∧ bs. hash-collision-budget ?trace-final (trace-final-stage A bs)
  using controlled unfolding staged-adversary-controlled-def
  by (blast intro: controlled-ro-program-collision)
have alpha-range:
  hash-range-budget ?alpha (staged-alpha-program (length spec))
  by (rule hash-range-budget-staged-alpha-program)
have alpha-coll:
  hash-collision-budget ?alpha (staged-alpha-program (length spec))
  by (rule hash-collision-budget-staged-alpha-program)
have degree-range:
  ∧ as. hash-range-budget ?degree (degree-stage A as)

```

```

using controlled unfolding staged-adversary-controlled-def
by (blast intro: controlled-ro-program-range)
have degree-coll:
   $\wedge as. \text{hash-collision-budget } ?degree \text{ (degree-stage } A \text{ as)}$ 
using controlled unfolding staged-adversary-controlled-def
by (blast intro: controlled-ro-program-collision)
have composition-fri-range:
   $\wedge dg. \text{hash-range-budget } ?composition-fri$ 
  (assert (ceil-log (to-nat dg + 1)  $\leq$  ceil-log (maxDegree + 1))  $\gg$ 
    ( $\lambda-. \text{staged-composition-fri-program } A \text{ dg } 0$ 
      (ceil-log (to-nat dg + 1)) []))
by (rule hash-range-budget-guarded-staged-composition-fri-program
  [OF wf controlled])
have composition-fri-coll:
   $\wedge dg. \text{hash-collision-budget } ?composition-fri$ 
  (assert (ceil-log (to-nat dg + 1)  $\leq$  ceil-log (maxDegree + 1))  $\gg$ 
    ( $\lambda-. \text{staged-composition-fri-program } A \text{ dg } 0$ 
      (ceil-log (to-nat dg + 1)) []))
by (rule hash-collision-budget-guarded-staged-composition-fri-program
  [OF wf controlled])
have composition-final-range:
   $\wedge dg \text{ bs. hash-range-budget } ?composition-final$ 
  (composition-final-stage A dg bs)
using controlled unfolding staged-adversary-controlled-def
by (blast intro: controlled-ro-program-range)
have composition-final-coll:
   $\wedge dg \text{ bs. hash-collision-budget } ?composition-final$ 
  (composition-final-stage A dg bs)
using controlled unfolding staged-adversary-controlled-def
by (blast intro: controlled-ro-program-collision)
have query-range:
  hash-range-budget ?query (staged-query-program A 0 rounds)
proof –
  have len:
     $0 + \text{rounds} \leq \text{length (query-opening-budgets budgets)}$ 
    using wf unfolding staged-budget-wellformed-def by simp
  have exact:
    hash-range-budget
    (sum-list
      (take rounds (drop 0 (query-opening-budgets budgets)))) +
      rounds)
    (staged-query-program A 0 rounds)
    by (rule hash-range-budget-staged-query-program[OF controlled len])
  then show ?thesis
    using wf unfolding staged-budget-wellformed-def by simp
qed
have query-coll:
  hash-collision-budget ?query (staged-query-program A 0 rounds)
proof –

```

```

have len:
   $0 + \text{rounds} \leq \text{length } (\text{query-opening-budgets budgets})$ 
  using wf unfolding staged-budget-wellformed-def by simp
have exact:
  hash-collision-budget
    (sum-list
      (take rounds (drop 0 (query-opening-budgets budgets)))) +
      rounds)
    (staged-query-program A 0 rounds)
  by (rule hash-collision-budget-staged-query-program[OF controlled len])
then show ?thesis
  using wf unfolding staged-budget-wellformed-def by simp
qed
have query-tail-range:
   $\wedge F. \text{hash-range-budget } (?query + 0)$ 
    (staged-query-program A 0 rounds  $\gg=$ 
      ( $\lambda \text{query-chunks. return } (F \text{ query-chunks})$ )))
  by (rule hash-range-budget-bind)
    (rule query-range, rule hash-range-budget-return)
have query-tail-coll:
   $\wedge F. \text{hash-collision-budget } (?query + 0)$ 
    (staged-query-program A 0 rounds  $\gg=$ 
      ( $\lambda \text{query-chunks. return } (F \text{ query-chunks})$ )))
  by (rule hash-collision-budget-bind)
    (rule query-range, rule query-coll, rule hash-range-budget-return,
      rule hash-collision-budget-return)
have after-composition-final-record-range:
   $\wedge \text{composition-final } F. \text{hash-range-budget } (0 + (?query + 0))$ 
    (record-staged-message composition-final  $\gg=$ 
      ( $\lambda -. \text{staged-query-program A 0 rounds } \gg=$ 
        ( $\lambda \text{query-chunks. return } (F \text{ query-chunks})$ ))))
  by (rule hash-range-budget-bind)
    (rule hash-range-budget-record-staged-message, rule query-tail-range)
have after-composition-final-record-coll:
   $\wedge \text{composition-final } F. \text{hash-collision-budget } (0 + (?query + 0))$ 
    (record-staged-message composition-final  $\gg=$ 
      ( $\lambda -. \text{staged-query-program A 0 rounds } \gg=$ 
        ( $\lambda \text{query-chunks. return } (F \text{ query-chunks})$ ))))
  by (rule hash-collision-budget-bind)
    (rule hash-range-budget-record-staged-message,
      rule hash-collision-budget-record-staged-message,
      rule query-tail-range, rule query-tail-coll)
have after-composition-final-range:
   $\wedge dg \text{ bs } F. \text{hash-range-budget}$ 
    ( $?composition-final + (0 + (?query + 0))$ )
    (composition-final-stage A dg bs  $\gg=$ 
      ( $\lambda \text{composition-final.}$ 
        record-staged-message composition-final  $\gg=$ 
          ( $\lambda -. \text{staged-query-program A 0 rounds } \gg=$ 
            ( $\lambda \text{query-chunks. return } (F \text{ query-chunks})$ ))))))

```

```

      (λquery-chunks.
        return (F composition-final query-chunks))))))
  by (rule hash-range-budget-bind)
    (rule composition-final-range,
     rule after-composition-final-record-range)
  have after-composition-final-coll:
    ∧dg bs F. hash-collision-budget
      (?composition-final + (0 + (?query + 0)))
      (composition-final-stage A dg bs ≍
        (λcomposition-final.
          record-staged-message composition-final ≍
            (λ-. staged-query-program A 0 rounds ≍
              (λquery-chunks.
                return (F composition-final query-chunks))))))
  by (rule hash-collision-budget-bind)
    (rule composition-final-range, rule composition-final-coll,
     rule after-composition-final-record-range,
     rule after-composition-final-record-coll)
  have after-composition-fri-range:
    ∧dg F. hash-range-budget
      (?composition-fri +
        (?composition-final + (0 + (?query + 0))))
      ((assert (ceil-log (to-nat dg + 1) ≤
        ceil-log (maxDegree + 1)) ≍
        (λ-. staged-composition-fri-program A dg 0
          (ceil-log (to-nat dg + 1)) [])) ≍
        (λ(composition-roots, composition-bs).
          composition-final-stage A dg composition-bs ≍
            (λcomposition-final.
              record-staged-message composition-final ≍
                (λ-. staged-query-program A 0 rounds ≍
                  (λquery-chunks.
                    return
                      (F composition-roots composition-bs composition-final
                        query-chunks))))))
  proof (rule hash-range-budget-bind)
    fix dg F
    show hash-range-budget ?composition-fri
      (assert (ceil-log (to-nat dg + 1) ≤
        ceil-log (maxDegree + 1)) ≍
        (λ-. staged-composition-fri-program A dg 0
          (ceil-log (to-nat dg + 1)) []))
      by (rule composition-fri-range)
    fix dg F x
    show hash-range-budget
      (?composition-final + (0 + (?query + 0)))
      (case x of (composition-roots, composition-bs) ⇒
        composition-final-stage A dg composition-bs ≍
          (λcomposition-final.

```

```

    record-staged-message composition-final >>=
    (λ-. staged-query-program A 0 rounds >>=
    (λquery-chunks.
    return
    (F composition-roots composition-bs composition-final
    query-chunks))))))
proof (cases x)
  case (Pair composition-roots composition-bs)
  have hash-range-budget
    (?composition-final + (0 + (?query + 0)))
    (composition-final-stage A dg composition-bs >>=
    (λcomposition-final.
    record-staged-message composition-final >>=
    (λ-. staged-query-program A 0 rounds >>=
    (λquery-chunks.
    return
    (F composition-roots composition-bs composition-final
    query-chunks))))))
  by (rule after-composition-final-range)
  then show ?thesis
    unfolding Pair by simp
  qed
qed
have after-composition-fri-coll:
  ∧ dg F. hash-collision-budget
    (?composition-fri +
    (?composition-final + (0 + (?query + 0))))
    ((assert (ceil-log (to-nat dg + 1) ≤
    ceil-log (maxDegree + 1)) >>=
    (λ-. staged-composition-fri-program A dg 0
    (ceil-log (to-nat dg + 1)) [])) >>=
    (λ(composition-roots, composition-bs).
    composition-final-stage A dg composition-bs >>=
    (λcomposition-final.
    record-staged-message composition-final >>=
    (λ-. staged-query-program A 0 rounds >>=
    (λquery-chunks.
    return
    (F composition-roots composition-bs composition-final
    query-chunks))))))
proof (rule hash-collision-budget-bind)
  fix dg F
  show hash-range-budget ?composition-fri
    (assert (ceil-log (to-nat dg + 1) ≤
    ceil-log (maxDegree + 1)) >>=
    (λ-. staged-composition-fri-program A dg 0
    (ceil-log (to-nat dg + 1)) []))
  by (rule composition-fri-range)
  fix dg F

```

```

show hash-collision-budget ?composition-fri
  (assert (ceil-log (to-nat dg + 1) ≤
    ceil-log (maxDegree + 1)) >>=
    (λ-. staged-composition-fri-program A dg 0
      (ceil-log (to-nat dg + 1)) []))
  by (rule composition-fri-coll)
fix dg F x
show hash-range-budget
  (?composition-final + (0 + (?query + 0)))
  (case x of (composition-roots, composition-bs) ⇒
    composition-final-stage A dg composition-bs >>=
    (λcomposition-final.
      record-staged-message composition-final >>=
      (λ-. staged-query-program A 0 rounds >>=
        (λquery-chunks.
          return
            (F composition-roots composition-bs composition-final
              query-chunks)))))
proof (cases x)
  case (Pair composition-roots composition-bs)
  have hash-range-budget
    (?composition-final + (0 + (?query + 0)))
    (composition-final-stage A dg composition-bs >>=
      (λcomposition-final.
        record-staged-message composition-final >>=
        (λ-. staged-query-program A 0 rounds >>=
          (λquery-chunks.
            return
              (F composition-roots composition-bs composition-final
                query-chunks)))))
  by (rule after-composition-final-range)
  then show ?thesis
    unfolding Pair by simp
qed
fix dg F x
show hash-collision-budget
  (?composition-final + (0 + (?query + 0)))
  (case x of (composition-roots, composition-bs) ⇒
    composition-final-stage A dg composition-bs >>=
    (λcomposition-final.
      record-staged-message composition-final >>=
      (λ-. staged-query-program A 0 rounds >>=
        (λquery-chunks.
          return
            (F composition-roots composition-bs composition-final
              query-chunks)))))
proof (cases x)
  case (Pair composition-roots composition-bs)
  have hash-collision-budget

```

```

    (?composition-final + (0 + (?query + 0)))
    (composition-final-stage A dg composition-bs >>=
      (λcomposition-final.
        record-staged-message composition-final >>=
          (λ-. staged-query-program A 0 rounds >>=
            (λquery-chunks.
              return
                (F composition-roots composition-bs composition-final
                  query-chunks))))))
  by (rule after-composition-final-coll)
then show ?thesis
  unfolding Pair by simp
qed
qed
have after-degree-record-range:
  ∧dg F. hash-range-budget
    (0 +
      (?composition-fri +
        (?composition-final + (0 + (?query + 0)))))
  (record-staged-message dg >>=
    (λ-. (assert (ceil-log (to-nat dg + 1) ≤
      ceil-log (maxDegree + 1)) >>=
        (λ-. staged-composition-fri-program A dg 0
          (ceil-log (to-nat dg + 1)) [])) >>=
      (λ(composition-roots, composition-bs).
        composition-final-stage A dg composition-bs >>=
          (λcomposition-final.
            record-staged-message composition-final >>=
              (λ-. staged-query-program A 0 rounds >>=
                (λquery-chunks.
                  return
                    (F composition-roots composition-bs composition-final
                      query-chunks)))))))
  by (rule hash-range-budget-bind)
    (rule hash-range-budget-record-staged-message,
      rule after-composition-fri-range)
have after-degree-record-coll:
  ∧dg F. hash-collision-budget
    (0 +
      (?composition-fri +
        (?composition-final + (0 + (?query + 0)))))
  (record-staged-message dg >>=
    (λ-. (assert (ceil-log (to-nat dg + 1) ≤
      ceil-log (maxDegree + 1)) >>=
        (λ-. staged-composition-fri-program A dg 0
          (ceil-log (to-nat dg + 1)) [])) >>=
      (λ(composition-roots, composition-bs).
        composition-final-stage A dg composition-bs >>=
          (λcomposition-final.
            record-staged-message composition-final >>=
              (λ-. staged-query-program A 0 rounds >>=
                (λquery-chunks.
                  return
                    (F composition-roots composition-bs composition-final
                      query-chunks)))))))

```

```

    record-staged-message composition-final >>=
      (λ-. staged-query-program A 0 rounds >>=
        (λquery-chunks.
          return
            (F composition-roots composition-bs composition-final
              query-chunks))))))
  by (rule hash-collision-budget-bind)
    (rule hash-range-budget-record-staged-message,
      rule hash-collision-budget-record-staged-message,
      rule after-composition-fri-range, rule after-composition-fri-coll)
  have after-degree-range:
    ∧as F. hash-range-budget
      (?degree +
        (0 +
          (?composition-fri +
            (?composition-final + (0 + (?query + 0))))))
      (degree-stage A as >>=
        (λdg. record-staged-message dg >>=
          (λ-. (assert (ceil-log (to-nat dg + 1) ≤
            ceil-log (maxDegree + 1)) >>=
              (λ-. staged-composition-fri-program A dg 0
                (ceil-log (to-nat dg + 1)) [])) >>=
                (λ(composition-roots, composition-bs).
                  composition-final-stage A dg composition-bs >>=
                    (λcomposition-final.
                      record-staged-message composition-final >>=
                        (λ-. staged-query-program A 0 rounds >>=
                          (λquery-chunks.
                            return
                              (F dg composition-roots composition-bs
                                composition-final query-chunks))))))))
          by (rule hash-range-budget-bind)
            (rule degree-range, rule after-degree-record-range)
        have after-degree-coll:
          ∧as F. hash-collision-budget
            (?degree +
              (0 +
                (?composition-fri +
                  (?composition-final + (0 + (?query + 0))))))
            (degree-stage A as >>=
              (λdg. record-staged-message dg >>=
                (λ-. (assert (ceil-log (to-nat dg + 1) ≤
                  ceil-log (maxDegree + 1)) >>=
                    (λ-. staged-composition-fri-program A dg 0
                      (ceil-log (to-nat dg + 1)) [])) >>=
                      (λ(composition-roots, composition-bs).
                        composition-final-stage A dg composition-bs >>=
                          (λcomposition-final.
                            record-staged-message composition-final >>=

```



```

      (λ-. staged-query-program A 0 rounds >>=
        (λquery-chunks.
          return
            (F dg composition-roots composition-bs
              composition-final query-chunks)))))))))
  by (rule hash-collision-budget-bind)
    (rule degree-range, rule degree-coll, rule after-degree-record-range,
      rule after-degree-record-coll)
  have after-alpha-range:
    ∧F. hash-range-budget
      (?alpha +
        (?degree +
          (0 +
            (?composition-fri +
              (?composition-final + (0 + (?query + 0)))))))
        (staged-alpha-program (length spec) >>=
          (λas. degree-stage A as >>=
            (λdg. record-staged-message dg >>=
              (λ-. (assert (ceil-log (to-nat dg + 1)) ≤
                ceil-log (maxDegree + 1)) >>=
                (λ-. staged-composition-fri-program A dg 0
                  (ceil-log (to-nat dg + 1)) [])) >>=
                (λ(composition-roots, composition-bs).
                  composition-final-stage A dg composition-bs >>=
                    (λcomposition-final.
                      record-staged-message composition-final >>=
                        (λ-. staged-query-program A 0 rounds >>=
                          (λquery-chunks.
                            return
                              (F as dg composition-roots composition-bs
                                composition-final query-chunks))))))))))
            by (rule hash-range-budget-bind)
              (rule alpha-range, rule after-degree-range)
  have after-alpha-coll:
    ∧F. hash-collision-budget
      (?alpha +
        (?degree +
          (0 +
            (?composition-fri +
              (?composition-final + (0 + (?query + 0)))))))
        (staged-alpha-program (length spec) >>=
          (λas. degree-stage A as >>=
            (λdg. record-staged-message dg >>=
              (λ-. (assert (ceil-log (to-nat dg + 1)) ≤
                ceil-log (maxDegree + 1)) >>=
                (λ-. staged-composition-fri-program A dg 0
                  (ceil-log (to-nat dg + 1)) [])) >>=
                (λ(composition-roots, composition-bs).
                  composition-final-stage A dg composition-bs >>=

```

```

      (λcomposition-final.
        record-staged-message composition-final ≫=
        (λ-. staged-query-program A 0 rounds ≫=
        (λquery-chunks.
          return
            (F as dg composition-roots composition-bs
              composition-final query-chunks)))))))))
by (rule hash-collision-budget-bind)
  (rule alpha-range, rule alpha-coll, rule after-degree-range,
    rule after-degree-coll)
have after-trace-final-record-range:
  ∧trace-final F. hash-range-budget
  (0 +
    (?alpha +
      (?degree +
        (0 +
          (?composition-fri +
            (?composition-final + (0 + (?query + 0))))))))))
  (record-staged-message trace-final ≫=
    (λ-. staged-alpha-program (length spec) ≫=
      (λas. degree-stage A as ≫=
        (λdg. record-staged-message dg ≫=
          (λ-. (assert (ceil-log (to-nat dg + 1) ≤
            ceil-log (maxDegree + 1)) ≫=
            (λ-. staged-composition-fri-program A dg 0
              (ceil-log (to-nat dg + 1)) [])) ≫=
            (λ(composition-roots, composition-bs).
              composition-final-stage A dg composition-bs ≫=
              (λcomposition-final.
                record-staged-message composition-final ≫=
                (λ-. staged-query-program A 0 rounds ≫=
                (λquery-chunks.
                  return
                    (F as dg composition-roots composition-bs
                      composition-final query-chunks)))))))))))))
by (rule hash-range-budget-bind)
  (rule hash-range-budget-record-staged-message,
    rule after-alpha-range)
have after-trace-final-record-coll:
  ∧trace-final F. hash-collision-budget
  (0 +
    (?alpha +
      (?degree +
        (0 +
          (?composition-fri +
            (?composition-final + (0 + (?query + 0))))))))))
  (record-staged-message trace-final ≫=
    (λ-. staged-alpha-program (length spec) ≫=
      (λas. degree-stage A as ≫=

```

```

(λdg. record-staged-message dg ≫=
  (λ-. (assert (ceil-log (to-nat dg + 1) ≤
    ceil-log (maxDegree + 1)) ≫=
    (λ-. staged-composition-fri-program A dg 0
      (ceil-log (to-nat dg + 1)) [])) ≫=
    (λ(composition-roots, composition-bs).
      composition-final-stage A dg composition-bs ≫=
      (λcomposition-final.
        record-staged-message composition-final ≫=
        (λ-. staged-query-program A 0 rounds ≫=
          (λquery-chunks.
            return
              (F as dg composition-roots composition-bs
                composition-final query-chunks))))))))))
by (rule hash-collision-budget-bind)
  (rule hash-range-budget-record-staged-message,
    rule hash-collision-budget-record-staged-message,
    rule after-alpha-range, rule after-alpha-coll)
have after-trace-final-range:
  ∧trace-bs F. hash-range-budget
  (?trace-final +
    (0 +
      (?alpha +
        (?degree +
          (0 +
            (?composition-fri +
              (?composition-final + (0 + (?query + 0))))))))))
  (trace-final-stage A trace-bs ≫=
    (λtrace-final. record-staged-message trace-final ≫=
      (λ-. staged-alpha-program (length spec) ≫=
        (λas. degree-stage A as ≫=
          (λdg. record-staged-message dg ≫=
            (λ-. (assert (ceil-log (to-nat dg + 1) ≤
              ceil-log (maxDegree + 1)) ≫=
              (λ-. staged-composition-fri-program A dg 0
                (ceil-log (to-nat dg + 1)) [])) ≫=
              (λ(composition-roots, composition-bs).
                composition-final-stage A dg composition-bs ≫=
                (λcomposition-final.
                  record-staged-message composition-final ≫=
                  (λ-. staged-query-program A 0 rounds ≫=
                    (λquery-chunks.
                      return
                        (F trace-final as dg composition-roots
                          composition-bs composition-final
                          query-chunks))))))))))))))
by (rule hash-range-budget-bind)
  (rule trace-final-range, rule after-trace-final-record-range)
have after-trace-final-coll:

```

```

 $\wedge$ trace-bs F. hash-collision-budget
  (?trace-final +
    (0 +
      (?alpha +
        (?degree +
          (0 +
            (?composition-fri +
              (?composition-final + (0 + (?query + 0))))))))))
  (trace-final-stage A trace-bs  $\gg$ 
    ( $\lambda$ trace-final. record-staged-message trace-final  $\gg$ 
      ( $\lambda$ -. staged-alpha-program (length spec)  $\gg$ 
        ( $\lambda$ as. degree-stage A as  $\gg$ 
          ( $\lambda$ dg. record-staged-message dg  $\gg$ 
            ( $\lambda$ -. (assert (ceil-log (to-nat dg + 1)  $\leq$ 
              ceil-log (maxDegree + 1))  $\gg$ 
              ( $\lambda$ -. staged-composition-fri-program A dg 0
                (ceil-log (to-nat dg + 1)) []))  $\gg$ 
              ( $\lambda$ (composition-roots, composition-bs).
                composition-final-stage A dg composition-bs  $\gg$ 
                  ( $\lambda$ composition-final.
                    record-staged-message composition-final  $\gg$ 
                      ( $\lambda$ -. staged-query-program A 0 rounds  $\gg$ 
                        ( $\lambda$ query-chunks.
                          return
                            (F trace-final as dg composition-roots
                              composition-bs composition-final
                                query-chunks))))))))))
  by (rule hash-collision-budget-bind)
    (rule trace-final-range, rule trace-final-coll,
      rule after-trace-final-record-range,
      rule after-trace-final-record-coll)
have after-trace-fri-range:
 $\wedge$ F. hash-range-budget
  (?trace-fri +
    (?trace-final +
      (0 +
        (?alpha +
          (?degree +
            (0 +
              (?composition-fri +
                (?composition-final + (0 + (?query + 0))))))))))
  (staged-trace-fri-program A 0 (ceil-log clength) []  $\gg$ 
    ( $\lambda$ (trace-roots, trace-bs).
      trace-final-stage A trace-bs  $\gg$ 
        ( $\lambda$ trace-final. record-staged-message trace-final  $\gg$ 
          ( $\lambda$ -. staged-alpha-program (length spec)  $\gg$ 
            ( $\lambda$ as. degree-stage A as  $\gg$ 
              ( $\lambda$ dg. record-staged-message dg  $\gg$ 
                ( $\lambda$ -. (assert (ceil-log (to-nat dg + 1)  $\leq$ 

```

```

      ceil-log (maxDegree + 1)) >>=
      (λ-. staged-composition-fri-program A dg 0
        (ceil-log (to-nat dg + 1)) [])) >>=
      (λ(composition-roots, composition-bs).
        composition-final-stage A dg composition-bs >>=
        (λcomposition-final.
          record-staged-message composition-final >>=
          (λ-. staged-query-program A 0 rounds >>=
            (λquery-chunks.
              return
              (F trace-roots trace-bs trace-final as dg
                composition-roots composition-bs
                composition-final query-chunks))))))))))
proof (rule hash-range-budget-bind)
fix F
show hash-range-budget ?trace-fri
  (staged-trace-fri-program A 0 (ceil-log clength) [])
  by (rule trace-fri-range)
fix F x
show hash-range-budget
  (?trace-final +
    (0 +
      (?alpha +
        (?degree +
          (0 +
            (?composition-fri +
              (?composition-final + (0 + (?query + 0))))))))))
  (case x of (trace-roots, trace-bs) =>
    trace-final-stage A trace-bs >>=
    (λtrace-final. record-staged-message trace-final >>=
      (λ-. staged-alpha-program (length spec) >>=
        (λas. degree-stage A as >>=
          (λdg. record-staged-message dg >>=
            (λ-. (assert (ceil-log (to-nat dg + 1) ≤
              ceil-log (maxDegree + 1)) >>=
              (λ-. staged-composition-fri-program A dg 0
                (ceil-log (to-nat dg + 1)) [])) >>=
              (λ(composition-roots, composition-bs).
                composition-final-stage A dg composition-bs >>=
                (λcomposition-final.
                  record-staged-message composition-final >>=
                  (λ-. staged-query-program A 0 rounds >>=
                    (λquery-chunks.
                      return
                      (F trace-roots trace-bs trace-final as dg
                        composition-roots composition-bs
                        composition-final query-chunks))))))))))
proof (cases x)
case (Pair trace-roots trace-bs)

```

```

have hash-range-budget
  (?trace-final +
    (0 +
      (?alpha +
        (?degree +
          (0 +
            (?composition-fri +
              (?composition-final + (0 + (?query + 0))))))))))
  (trace-final-stage A trace-bs >=>
    (λtrace-final. record-staged-message trace-final >=>
      (λ-. staged-alpha-program (length spec) >=>
        (λas. degree-stage A as >=>
          (λdg. record-staged-message dg >=>
            (λ-. (assert (ceil-log (to-nat dg + 1) ≤
              ceil-log (maxDegree + 1)) >=>
                (λ-. staged-composition-fri-program A dg 0
                  (ceil-log (to-nat dg + 1)) [])) >=>
                  (λ(composition-roots, composition-bs).
                    composition-final-stage A dg composition-bs >=>
                      (λcomposition-final.
                        record-staged-message composition-final >=>
                          (λ-. staged-query-program A 0 rounds >=>
                            (λquery-chunks.
                              return
                                (F trace-roots trace-bs trace-final as dg
                                  composition-roots composition-bs
                                  composition-final query-chunks))))))))))
    by (rule after-trace-final-range)
  then show ?thesis
    unfolding Pair by simp
  qed
qed
have after-trace-fri-coll:
  ∧F. hash-collision-budget
    (?trace-fri +
      (?trace-final +
        (0 +
          (?alpha +
            (?degree +
              (0 +
                (?composition-fri +
                  (?composition-final + (0 + (?query + 0))))))))))
    (staged-trace-fri-program A 0 (ceil-log clength) [] >=>
      (λ(trace-roots, trace-bs).
        trace-final-stage A trace-bs >=>
          (λtrace-final. record-staged-message trace-final >=>
            (λ-. staged-alpha-program (length spec) >=>
              (λas. degree-stage A as >=>
                (λdg. record-staged-message dg >=>

```

```

(λ-. (assert (ceil-log (to-nat dg + 1) ≤
  ceil-log (maxDegree + 1)) >>=
  (λ-. staged-composition-fri-program A dg 0
    (ceil-log (to-nat dg + 1)) [])) >>=
  (λ(composition-roots, composition-bs).
    composition-final-stage A dg composition-bs >>=
    (λcomposition-final.
      record-staged-message composition-final >>=
      (λ-. staged-query-program A 0 rounds >>=
        (λquery-chunks.
          return
            (F trace-roots trace-bs trace-final as dg
              composition-roots composition-bs
              composition-final query-chunks))))))))))
proof (rule hash-collision-budget-bind)
fix F
show hash-range-budget ?trace-fri
  (staged-trace-fri-program A 0 (ceil-log clength) [])
  by (rule trace-fri-range)
fix F
show hash-collision-budget ?trace-fri
  (staged-trace-fri-program A 0 (ceil-log clength) [])
  by (rule trace-fri-coll)
fix F x
show hash-range-budget
  (?trace-final +
    (0 +
      (?alpha +
        (?degree +
          (0 +
            (?composition-fri +
              (?composition-final + (0 + (?query + 0))))))))))
  (case x of (trace-roots, trace-bs) ⇒
    trace-final-stage A trace-bs >>=
    (λtrace-final. record-staged-message trace-final >>=
      (λ-. staged-alpha-program (length spec) >>=
        (λas. degree-stage A as >>=
          (λdg. record-staged-message dg >>=
            (λ-. (assert (ceil-log (to-nat dg + 1) ≤
              ceil-log (maxDegree + 1)) >>=
              (λ-. staged-composition-fri-program A dg 0
                (ceil-log (to-nat dg + 1)) [])) >>=
              (λ(composition-roots, composition-bs).
                composition-final-stage A dg composition-bs >>=
                (λcomposition-final.
                  record-staged-message composition-final >>=
                  (λ-. staged-query-program A 0 rounds >>=
                    (λquery-chunks.
                      return

```

```

(F trace-roots trace-bs trace-final as dg
 composition-roots composition-bs
 composition-final query-chunks))))))))))
proof (cases x)
case (Pair trace-roots trace-bs)
have hash-range-budget
  (?trace-final +
   (0 +
    (?alpha +
     (?degree +
      (0 +
       (?composition-fri +
        (?composition-final + (0 + (?query + 0))))))))))
(trace-final-stage A trace-bs >=>
 (λtrace-final. record-staged-message trace-final >=>
  (λ-. staged-alpha-program (length spec) >=>
   (λas. degree-stage A as >=>
    (λdg. record-staged-message dg >=>
     (λ-. (assert (ceil-log (to-nat dg + 1) ≤
      ceil-log (maxDegree + 1)) >=>
       (λ-. staged-composition-fri-program A dg 0
        (ceil-log (to-nat dg + 1)) [])) >=>
        (λ(composition-roots, composition-bs).
         composition-final-stage A dg composition-bs >=>
          (λcomposition-final.
           record-staged-message composition-final >=>
            (λ-. staged-query-program A 0 rounds >=>
             (λquery-chunks.
              return
               (F trace-roots trace-bs trace-final as dg
                composition-roots composition-bs
                composition-final query-chunks))))))))))
    by (rule after-trace-final-range)
  then show ?thesis
    unfolding Pair by simp
qed
fix F x
show hash-collision-budget
  (?trace-final +
   (0 +
    (?alpha +
     (?degree +
      (0 +
       (?composition-fri +
        (?composition-final + (0 + (?query + 0))))))))))
(case x of (trace-roots, trace-bs) =>
 trace-final-stage A trace-bs >=>
 (λtrace-final. record-staged-message trace-final >=>
  (λ-. staged-alpha-program (length spec) >=>

```



```

(λas. degree-stage A as >=
  (λdg. record-staged-message dg >=
    (λ-. (assert (ceil-log (to-nat dg + 1) ≤
      ceil-log (maxDegree + 1)) >=
      (λ-. staged-composition-fri-program A dg 0
        (ceil-log (to-nat dg + 1)) [])) >=
      (λ(composition-roots, composition-bs).
        composition-final-stage A dg composition-bs >=
        (λcomposition-final.
          record-staged-message composition-final >=
          (λ-. staged-query-program A 0 rounds >=
            (λquery-chunks.
              return
              (F trace-roots trace-bs trace-final as dg
                composition-roots composition-bs
                composition-final query-chunks))))))))))
proof (cases x)
case (Pair trace-roots trace-bs)
have hash-collision-budget
  (?trace-final +
    (0 +
      (?alpha +
        (?degree +
          (0 +
            (?composition-fri +
              (?composition-final + (0 + (?query + 0))))))))))
  (trace-final-stage A trace-bs >=
    (λtrace-final. record-staged-message trace-final >=
      (λ-. staged-alpha-program (length spec) >=
        (λas. degree-stage A as >=
          (λdg. record-staged-message dg >=
            (λ-. (assert (ceil-log (to-nat dg + 1) ≤
              ceil-log (maxDegree + 1)) >=
              (λ-. staged-composition-fri-program A dg 0
                (ceil-log (to-nat dg + 1)) [])) >=
              (λ(composition-roots, composition-bs).
                composition-final-stage A dg composition-bs >=
                (λcomposition-final.
                  record-staged-message composition-final >=
                  (λ-. staged-query-program A 0 rounds >=
                    (λquery-chunks.
                      return
                      (F trace-roots trace-bs trace-final as dg
                        composition-roots composition-bs
                        composition-final query-chunks))))))))))
        by (rule after-trace-final-coll)
      then show ?thesis
        unfolding Pair by simp
    qed

```

qed

have *after-trace-root-record-range:*

```

 $\wedge$ fr F. hash-range-budget
  (0 +
    (?trace-fri +
      (?trace-final +
        (0 +
          (?alpha +
            (?degree +
              (0 +
                (?composition-fri +
                  (?composition-final + (0 + (?query + 0))))))))))
    (record-staged-message fr  $\gg$ 
      ( $\lambda$ -. staged-trace-fri-program A 0 (ceil-log clength) []  $\gg$ 
        ( $\lambda$ (trace-roots, trace-bs).
          trace-final-stage A trace-bs  $\gg$ 
            ( $\lambda$ trace-final. record-staged-message trace-final  $\gg$ 
              ( $\lambda$ -. staged-alpha-program (length spec)  $\gg$ 
                ( $\lambda$ as. degree-stage A as  $\gg$ 
                  ( $\lambda$ dg. record-staged-message dg  $\gg$ 
                    ( $\lambda$ -. (assert (ceil-log (to-nat dg + 1))  $\leq$ 
                      ceil-log (maxDegree + 1))  $\gg$ 
                      ( $\lambda$ -. staged-composition-fri-program A dg 0
                        (ceil-log (to-nat dg + 1)) []))  $\gg$ 
                      ( $\lambda$ (composition-roots, composition-bs).
                        composition-final-stage A dg composition-bs  $\gg$ 
                          ( $\lambda$ composition-final.
                            record-staged-message composition-final  $\gg$ 
                              ( $\lambda$ -. staged-query-program A 0 rounds  $\gg$ 
                                ( $\lambda$ query-chunks.
                                  return
                                    (F trace-roots trace-bs trace-final as dg
                                      composition-roots composition-bs
                                      composition-final query-chunks))))))))))

```

by (rule *hash-range-budget-bind*)

(rule *hash-range-budget-record-staged-message*,
rule *after-trace-fri-range*)

have *after-trace-root-record-coll:*

```

 $\wedge$ fr F. hash-collision-budget
  (0 +
    (?trace-fri +
      (?trace-final +
        (0 +
          (?alpha +
            (?degree +
              (0 +
                (?composition-fri +
                  (?composition-final + (0 + (?query + 0))))))))))
    (record-staged-message fr  $\gg$ 

```

```

(λ-. staged-trace-fri-program A 0 (ceil-log clength) [] >>=
  (λ(trace-roots, trace-bs).
    trace-final-stage A trace-bs >>=
      (λtrace-final. record-staged-message trace-final >>=
        (λ-. staged-alpha-program (length spec) >>=
          (λas. degree-stage A as >>=
            (λdg. record-staged-message dg >>=
              (λ-. (assert (ceil-log (to-nat dg + 1) ≤
                ceil-log (maxDegree + 1)) >>=
                  (λ-. staged-composition-fri-program A dg 0
                    (ceil-log (to-nat dg + 1)) [])) >>=
                    (λ(composition-roots, composition-bs).
                      composition-final-stage A dg composition-bs >>=
                        (λcomposition-final.
                          record-staged-message composition-final >>=
                            (λ-. staged-query-program A 0 rounds >>=
                              (λquery-chunks.
                                return
                                  (F trace-roots trace-bs trace-final as dg
                                    composition-roots composition-bs
                                    composition-final query-chunks))))))))))
  by (rule hash-collision-budget-bind)
  (rule hash-range-budget-record-staged-message,
    rule hash-collision-budget-record-staged-message,
    rule after-trace-fri-range, rule after-trace-fri-coll)
have whole:
  hash-collision-budget
    (?trace-root +
      (0 +
        (?trace-fri +
          (?trace-final +
            (0 +
              (?alpha +
                (?degree +
                  (0 +
                    (?composition-fri +
                      (?composition-final + (0 + (?query + 0))))))))))
      (staged-transcript-program A)
    )
  unfolding staged-transcript-program-def Let-def
proof (rule hash-collision-budget-bind)
show hash-range-budget ?trace-root (trace-root-stage A)
  by (rule trace-root-range)
show hash-collision-budget ?trace-root (trace-root-stage A)
  by (rule trace-root-coll)
fix fr
show hash-range-budget
  (0 +
    (?trace-fri +
      (?trace-final +

```

```

(0 +
  (?alpha +
    (?degree +
      (0 +
        (?composition-fri +
          (?composition-final + (0 + (?query + 0))))))))))
(record-staged-message fr >>=
  (λ-. staged-trace-fri-program A 0 (ceil-log clength) [] >>=
    (λ(trace-roots, trace-bs).
      trace-final-stage A trace-bs >>=
        (λtrace-final.
          record-staged-message trace-final >>=
            (λ-. staged-alpha-program (length spec) >>=
              (λas. degree-stage A as >>=
                (λdg. record-staged-message dg >>=
                  (λ-. assert (ceil-log (to-nat dg + 1) ≤
                    ceil-log (maxDegree + 1)) >>=
                    (λ-. staged-composition-fri-program A dg 0
                      (ceil-log (to-nat dg + 1)) [] >>=
                        (λ(composition-roots, composition-bs).
                          composition-final-stage A dg composition-bs >>=
                            (λcomposition-final.
                              record-staged-message
                                composition-final >>=
                                  (λ-. staged-query-program A 0
                                    rounds >>=
                                      (λquery-chunks.
                                        return
                                          (|staged-trace-root = fr,
                                            staged-trace-fri-roots =
                                              trace-roots,
                                            staged-trace-fri-challenges =
                                              trace-bs,
                                            staged-trace-final =
                                              trace-final,
                                            staged-alphas = as,
                                            staged-degree = dg,
                                            staged-composition-fri-roots =
                                              composition-roots,
                                            staged-composition-fri-challenges =
                                              composition-bs,
                                            staged-composition-final =
                                              composition-final,
                                            staged-query-chunks =
                                              query-chunks|)))))))))))))
using after-trace-root-record-range
[of fr
  λtrace-roots trace-bs trace-final as dg composition-roots
  composition-bs composition-final query-chunks.

```

```

    (| staged-trace-root = fr,
      staged-trace-fri-roots = trace-roots,
      staged-trace-fri-challenges = trace-bs,
      staged-trace-final = trace-final,
      staged-alphas = as,
      staged-degree = dg,
      staged-composition-fri-roots = composition-roots,
      staged-composition-fri-challenges = composition-bs,
      staged-composition-final = composition-final,
      staged-query-chunks = query-chunks|)]
  by (simp add: sm-bind-assoc)
fix fr
show hash-collision-budget
  (0 +
    (?trace-fri +
      (?trace-final +
        (0 +
          (?alpha +
            (?degree +
              (0 +
                (?composition-fri +
                  (?composition-final + (0 + (?query + 0))))))))))
  (record-staged-message fr >=>
    (λ-. staged-trace-fri-program A 0 (ceil-log clength) [] >=>
      (λ(trace-roots, trace-bs).
        trace-final-stage A trace-bs >=>
          (λtrace-final.
            record-staged-message trace-final >=>
              (λ-. staged-alpha-program (length spec) >=>
                (λas. degree-stage A as >=>
                  (λdg. record-staged-message dg >=>
                    (λ-. assert (ceil-log (to-nat dg + 1) ≤
                      ceil-log (maxDegree + 1)) >=>
                      (λ-. staged-composition-fri-program A dg 0
                        (ceil-log (to-nat dg + 1)) [] >=>
                          (λ(composition-roots, composition-bs).
                            composition-final-stage A dg composition-bs >=>
                              (λcomposition-final.
                                record-staged-message
                                  composition-final >=>
                                    (λ-. staged-query-program A 0
                                      rounds >=>
                                        (λquery-chunks.
                                          return
                                            (| staged-trace-root = fr,
                                              staged-trace-fri-roots =
                                                trace-roots,
                                              staged-trace-fri-challenges =
                                                trace-bs,

```

```

    staged-trace-final =
      trace-final,
    staged-alphas = as,
    staged-degree = dg,
    staged-composition-fri-roots =
      composition-roots,
    staged-composition-fri-challenges =
      composition-bs,
    staged-composition-final =
      composition-final,
    staged-query-chunks =
      query-chunks)))))))))))))
using after-trace-root-record-coll
  [of fr
    λtrace-roots trace-bs trace-final as dg composition-roots
      composition-bs composition-final query-chunks.
    (|staged-trace-root = fr,
      staged-trace-fri-roots = trace-roots,
      staged-trace-fri-challenges = trace-bs,
      staged-trace-final = trace-final,
      staged-alphas = as,
      staged-degree = dg,
      staged-composition-fri-roots = composition-roots,
      staged-composition-fri-challenges = composition-bs,
      staged-composition-final = composition-final,
      staged-query-chunks = query-chunks|)]
  by (simp add: sm-bind-assoc)
qed
have budget-eq:
  ?trace-root +
  (0 +
    (?trace-fri +
      (?trace-final +
        (0 +
          (?alpha +
            (?degree +
              (0 +
                (?composition-fri +
                  (?composition-final + (0 + (?query + 0)))))))))) =
    staged-attacker-query-budget budgets + staged-challenge-query-budget
unfolding staged-attacker-query-budget-def
  staged-challenge-query-budget-def
by (simp add: add.assoc add.commute add.left-commute)
show ?thesis
using whole unfolding budget-eq .
qed

lemma hash-collision-budget-staged-semantic-adversary:
  assumes wf: staged-budget-wellformed budgets

```

```

and controlled: staged-adversary-controlled budgets A
shows
  hash-collision-budget
    (staged-attacker-query-budget budgets + staged-challenge-query-budget)
    (staged-semantic-adversary A)
proof –
  have hash-collision-budget
    ((staged-attacker-query-budget budgets + staged-challenge-query-budget) +
     0)
    (staged-semantic-adversary A)
  unfolding staged-semantic-adversary-def
proof (rule hash-collision-budget-bind
  [where n=staged-attacker-query-budget budgets +
    staged-challenge-query-budget and n'=0])
  show hash-range-budget
    (staged-attacker-query-budget budgets + staged-challenge-query-budget)
    (staged-transcript-program A)
  by (rule hash-range-budget-staged-transcript-program[OF wf controlled])
  show hash-collision-budget
    (staged-attacker-query-budget budgets + staged-challenge-query-budget)
    (staged-transcript-program A)
  by (rule hash-collision-budget-staged-transcript-program
    [OF wf controlled])
  show  $\bigwedge x.$  hash-range-budget 0 (return (staged-proof-transcript x))
  by (rule hash-range-budget-return)
  show  $\bigwedge x.$  hash-collision-budget 0
    (return (staged-proof-transcript x))
  by (rule hash-collision-budget-return)
qed
then show ?thesis by simp
qed

lemma hash-target-program-staged-transcript-program:
assumes wf: staged-budget-wellformed budgets
and controlled: staged-adversary-controlled budgets A
shows
  hash-target-program B
    (staged-attacker-query-budget budgets + staged-challenge-query-budget)
    (staged-transcript-program A)
proof –
  let ?trace-root = trace-root-budget budgets
  let ?trace-fri =
    sum-list (trace-fri-budgets budgets) + ceil-log clength
  let ?trace-final = trace-final-budget budgets
  let ?alpha = length spec
  let ?degree = degree-budget budgets
  let ?composition-fri =
    sum-list (composition-fri-budgets budgets) +
    ceil-log (maxDegree + 1)

```

```

let ?composition-final = composition-final-budget budgets
let ?query =
  sum-list (query-opening-budgets budgets) + rounds
have trace-root-target:
  hash-target-program B ?trace-root (trace-root-stage A)
  using controlled unfolding staged-adversary-controlled-def
  by (blast intro: controlled-ro-program-target)
have trace-fri-target:
  hash-target-program B ?trace-fri
  (staged-trace-fri-program A 0 (ceil-log clength) [])
proof -
  have len:
    0 + ceil-log clength ≤ length (trace-fri-budgets budgets)
    using wf unfolding staged-budget-wellformed-def by simp
  have exact:
    hash-target-program B
    (sum-list
      (take (ceil-log clength)
        (drop 0 (trace-fri-budgets budgets))) +
      ceil-log clength)
    (staged-trace-fri-program A 0 (ceil-log clength) [])
    by (rule hash-target-program-staged-trace-fri-program
      [OF controlled len])
  then show ?thesis
    using wf unfolding staged-budget-wellformed-def by simp
qed
have trace-final-target:
  ∧ bs. hash-target-program B ?trace-final (trace-final-stage A bs)
  using controlled unfolding staged-adversary-controlled-def
  by (blast intro: controlled-ro-program-target)
have alpha-target:
  hash-target-program B ?alpha (staged-alpha-program (length spec))
  by (rule hash-target-program-staged-alpha-program)
have degree-target:
  ∧ as. hash-target-program B ?degree (degree-stage A as)
  using controlled unfolding staged-adversary-controlled-def
  by (blast intro: controlled-ro-program-target)
have composition-fri-target:
  ∧ dg. hash-target-program B ?composition-fri
  (assert (ceil-log (to-nat dg + 1) ≤ ceil-log (maxDegree + 1)) >>=
    (λ-. staged-composition-fri-program A dg 0
      (ceil-log (to-nat dg + 1)) []))
  by (rule hash-target-program-guarded-staged-composition-fri-program
    [OF wf controlled])
have composition-final-target:
  ∧ dg bs. hash-target-program B ?composition-final
  (composition-final-stage A dg bs)
  using controlled unfolding staged-adversary-controlled-def
  by (blast intro: controlled-ro-program-target)

```



```

have query-target:
  hash-target-program B ?query (staged-query-program A 0 rounds)
proof –
  have len:
    0 + rounds ≤ length (query-opening-budgets budgets)
    using wf unfolding staged-budget-wellformed-def by simp
  have exact:
    hash-target-program B
      (sum-list
        (take rounds (drop 0 (query-opening-budgets budgets))) +
        rounds)
      (staged-query-program A 0 rounds)
    by (rule hash-target-program-staged-query-program[OF controlled len])
  then show ?thesis
    using wf unfolding staged-budget-wellformed-def by simp
qed
have query-tail:
  ∧F. hash-target-program B (?query + 0)
    (staged-query-program A 0 rounds >=>
      (λquery-chunks. return (F query-chunks)))
  by (rule hash-target-program-bind)
    (rule query-target, rule hash-target-program-return)
have after-composition-final-record:
  ∧composition-final F. hash-target-program B (0 + (?query + 0))
    (record-staged-message composition-final >=>
      (λ-. staged-query-program A 0 rounds >=>
        (λquery-chunks. return (F query-chunks))))
  by (rule hash-target-program-bind)
    (rule hash-target-program-record-staged-message, rule query-tail)
have after-composition-final:
  ∧dg bs F. hash-target-program B
    (?composition-final + (0 + (?query + 0)))
    (composition-final-stage A dg bs >=>
      (λcomposition-final.
        record-staged-message composition-final >=>
          (λ-. staged-query-program A 0 rounds >=>
            (λquery-chunks.
              return (F composition-final query-chunks))))))
  by (rule hash-target-program-bind)
    (rule composition-final-target, rule after-composition-final-record)
have after-composition-fri:
  ∧dg F. hash-target-program B
    (?composition-fri +
      (?composition-final + (0 + (?query + 0))))
    ((assert (ceil-log (to-nat dg + 1) ≤
      ceil-log (maxDegree + 1)) >=>
      (λ-. staged-composition-fri-program A dg 0
        (ceil-log (to-nat dg + 1)) [])) >=>
      (λ(composition-roots, composition-bs).

```

```

    composition-final-stage A dg composition-bs >>=
      (λcomposition-final.
        record-staged-message composition-final >>=
          (λ-. staged-query-program A 0 rounds >>=
            (λquery-chunks.
              return
                (F composition-roots composition-bs composition-final
                  query-chunks))))))
proof (rule hash-target-program-bind)
  fix dg F
  show hash-target-program B ?composition-fri
    (assert (ceil-log (to-nat dg + 1) ≤
      ceil-log (maxDegree + 1)) >>=
      (λ-. staged-composition-fri-program A dg 0
        (ceil-log (to-nat dg + 1)) []))
    by (rule composition-fri-target)
  fix dg F x
  show hash-target-program B
    (?composition-final + (0 + (?query + 0)))
    (case x of (composition-roots, composition-bs) =>
      composition-final-stage A dg composition-bs >>=
        (λcomposition-final.
          record-staged-message composition-final >>=
            (λ-. staged-query-program A 0 rounds >>=
              (λquery-chunks.
                return
                  (F composition-roots composition-bs composition-final
                    query-chunks))))))
  proof (cases x)
  case (Pair composition-roots composition-bs)
  have hash-target-program B
    (?composition-final + (0 + (?query + 0)))
    (composition-final-stage A dg composition-bs >>=
      (λcomposition-final.
        record-staged-message composition-final >>=
          (λ-. staged-query-program A 0 rounds >>=
            (λquery-chunks.
              return
                (F composition-roots composition-bs composition-final
                  query-chunks))))))
    by (rule after-composition-final)
  then show ?thesis
    unfolding Pair by simp
  qed
qed
have after-degree-record:
  ∧ dg F. hash-target-program B
    (0 +
      (?composition-fri +

```

```

    (?composition-final + (0 + (?query + 0))))))
  (record-staged-message dg >>=
    (λ-. (assert (ceil-log (to-nat dg + 1) ≤
      ceil-log (maxDegree + 1)) >>=
      (λ-. staged-composition-fri-program A dg 0
        (ceil-log (to-nat dg + 1)) [])) >>=
      (λ(composition-roots, composition-bs).
        composition-final-stage A dg composition-bs >>=
        (λcomposition-final.
          record-staged-message composition-final >>=
          (λ-. staged-query-program A 0 rounds >>=
            (λquery-chunks.
              return
                (F composition-roots composition-bs composition-final
                  query-chunks)))))))
  by (rule hash-target-program-bind)
  (rule hash-target-program-record-staged-message, rule after-composition-fri)
have after-degree:
  ∧as F. hash-target-program B
  (?degree +
    (0 +
      (?composition-fri +
        (?composition-final + (0 + (?query + 0))))))
  (degree-stage A as >>=
    (λdg. record-staged-message dg >>=
      (λ-. (assert (ceil-log (to-nat dg + 1) ≤
        ceil-log (maxDegree + 1)) >>=
        (λ-. staged-composition-fri-program A dg 0
          (ceil-log (to-nat dg + 1)) [])) >>=
        (λ(composition-roots, composition-bs).
          composition-final-stage A dg composition-bs >>=
          (λcomposition-final.
            record-staged-message composition-final >>=
            (λ-. staged-query-program A 0 rounds >>=
              (λquery-chunks.
                return
                  (F dg composition-roots composition-bs
                    composition-final query-chunks)))))))
    by (rule hash-target-program-bind)
    (rule degree-target, rule after-degree-record)
have after-alpha:
  ∧F. hash-target-program B
  (?alpha +
    (?degree +
      (0 +
        (?composition-fri +
          (?composition-final + (0 + (?query + 0))))))
  (staged-alpha-program (length spec) >>=
    (λas. degree-stage A as >>=

```

```

( $\lambda dg$ . record-staged-message  $dg \gg=$ 
  ( $\lambda$ -. (assert (ceil-log (to-nat  $dg + 1$ )  $\leq$ 
    ceil-log (maxDegree + 1))  $\gg=$ 
    ( $\lambda$ -. staged-composition-fri-program  $A \ dg \ 0$ 
      (ceil-log (to-nat  $dg + 1$ ) []))  $\gg=$ 
      ( $\lambda$ (composition-roots, composition-bs).
        composition-final-stage  $A \ dg \ composition-bs \gg=$ 
        ( $\lambda composition-final$ .
          record-staged-message composition-final  $\gg=$ 
          ( $\lambda$ -. staged-query-program  $A \ 0 \ rounds \gg=$ 
            ( $\lambda query-chunks$ .
              return
                ( $F \ as \ dg \ composition-roots \ composition-bs$ 
                  composition-final query-chunks))))))))))
by (rule hash-target-program-bind)
  (rule alpha-target, rule after-degree)
have after-trace-final-record:
 $\wedge trace-final \ F$ . hash-target-program  $B$ 
  ( $0 +$ 
    (?alpha +
      (?degree +
        ( $0 +$ 
          (?composition-fri +
            (?composition-final + ( $0 +$  (?query +  $0$ ))))))))))
  (record-staged-message trace-final  $\gg=$ 
    ( $\lambda$ -. staged-alpha-program (length spec)  $\gg=$ 
      ( $\lambda as$ . degree-stage  $A \ as \gg=$ 
        ( $\lambda dg$ . record-staged-message  $dg \gg=$ 
          ( $\lambda$ -. (assert (ceil-log (to-nat  $dg + 1$ )  $\leq$ 
            ceil-log (maxDegree + 1))  $\gg=$ 
            ( $\lambda$ -. staged-composition-fri-program  $A \ dg \ 0$ 
              (ceil-log (to-nat  $dg + 1$ ) []))  $\gg=$ 
              ( $\lambda$ (composition-roots, composition-bs).
                composition-final-stage  $A \ dg \ composition-bs \gg=$ 
                ( $\lambda composition-final$ .
                  record-staged-message composition-final  $\gg=$ 
                  ( $\lambda$ -. staged-query-program  $A \ 0 \ rounds \gg=$ 
                    ( $\lambda query-chunks$ .
                      return
                        ( $F \ as \ dg \ composition-roots \ composition-bs$ 
                          composition-final query-chunks))))))))))
          (rule hash-target-program-record-staged-message, rule after-alpha)
have after-trace-final:
 $\wedge trace-bs \ F$ . hash-target-program  $B$ 
  (?trace-final +
    ( $0 +$ 
      (?alpha +
        (?degree +

```

```

(0 +
  (?composition-fri +
    (?composition-final + (0 + (?query + 0))))))
(trace-final-stage A trace-bs >>=
  (λtrace-final. record-staged-message trace-final >>=
    (λ-. staged-alpha-program (length spec) >>=
      (λas. degree-stage A as >>=
        (λdg. record-staged-message dg >>=
          (λ-. (assert (ceil-log (to-nat dg + 1) ≤
            ceil-log (maxDegree + 1)) >>=
              (λ-. staged-composition-fri-program A dg 0
                (ceil-log (to-nat dg + 1)) [])) >>=
                (λ(composition-roots, composition-bs).
                  composition-final-stage A dg composition-bs >>=
                    (λcomposition-final.
                      record-staged-message composition-final >>=
                        (λ-. staged-query-program A 0 rounds >>=
                          (λquery-chunks.
                            return
                              (F trace-final as dg composition-roots
                                composition-bs composition-final
                                  query-chunks))))))))))
by (rule hash-target-program-bind)
      (rule trace-final-target, rule after-trace-final-record)
have after-trace-fri:
  ∧F. hash-target-program B
  (?trace-fri +
    (?trace-final +
      (0 +
        (?alpha +
          (?degree +
            (0 +
              (?composition-fri +
                (?composition-final + (0 + (?query + 0)))))))))
  (staged-trace-fri-program A 0 (ceil-log clength) [] >>=
    (λ(trace-roots, trace-bs).
      trace-final-stage A trace-bs >>=
        (λtrace-final. record-staged-message trace-final >>=
          (λ-. staged-alpha-program (length spec) >>=
            (λas. degree-stage A as >>=
              (λdg. record-staged-message dg >>=
                (λ-. (assert (ceil-log (to-nat dg + 1) ≤
                  ceil-log (maxDegree + 1)) >>=
                    (λ-. staged-composition-fri-program A dg 0
                      (ceil-log (to-nat dg + 1)) [])) >>=
                      (λ(composition-roots, composition-bs).
                        composition-final-stage A dg composition-bs >>=
                          (λcomposition-final.
                            record-staged-message composition-final >>=

```

```

      (λ-. staged-query-program A 0 rounds >>=
        (λquery-chunks.
          return
            (F trace-roots trace-bs trace-final as dg
              composition-roots composition-bs
              composition-final query-chunks)))))))))
proof (rule hash-target-program-bind)
  fix F
  show hash-target-program B ?trace-fri
    (staged-trace-fri-program A 0 (ceil-log clength) [])
    by (rule trace-fri-target)
  fix F x
  show hash-target-program B
    (?trace-final +
      (0 +
        (?alpha +
          (?degree +
            (0 +
              (?composition-fri +
                (?composition-final + (0 + (?query + 0))))))))))
    (case x of (trace-roots, trace-bs) =>
      trace-final-stage A trace-bs >>=
        (λtrace-final. record-staged-message trace-final >>=
          (λ-. staged-alpha-program (length spec) >>=
            (λas. degree-stage A as >>=
              (λdg. record-staged-message dg >>=
                (λ-. (assert (ceil-log (to-nat dg + 1)) ≤
                  ceil-log (maxDegree + 1)) >>=
                  (λ-. staged-composition-fri-program A dg 0
                    (ceil-log (to-nat dg + 1)) [])) >>=
                  (λ(composition-roots, composition-bs).
                    composition-final-stage A dg composition-bs >>=
                    (λcomposition-final.
                      record-staged-message composition-final >>=
                      (λ-. staged-query-program A 0 rounds >>=
                        (λquery-chunks.
                          return
                            (F trace-roots trace-bs trace-final as dg
                              composition-roots composition-bs
                              composition-final query-chunks))))))))))
proof (cases x)
  case (Pair trace-roots trace-bs)
  have hash-target-program B
    (?trace-final +
      (0 +
        (?alpha +
          (?degree +
            (0 +
              (?composition-fri +

```

```

      (?composition-final + (0 + (?query + 0)))))))))
(trace-final-stage A trace-bs >>=
  (λtrace-final. record-staged-message trace-final >>=
    (λ-. staged-alpha-program (length spec) >>=
      (λas. degree-stage A as >>=
        (λdg. record-staged-message dg >>=
          (λ-. (assert (ceil-log (to-nat dg + 1) ≤
            ceil-log (maxDegree + 1)) >>=
              (λ-. staged-composition-fri-program A dg 0
                (ceil-log (to-nat dg + 1)) [])) >>=
                (λ(composition-roots, composition-bs).
                  composition-final-stage A dg composition-bs >>=
                    (λcomposition-final.
                      record-staged-message composition-final >>=
                        (λ-. staged-query-program A 0 rounds >>=
                          (λquery-chunks.
                            return
                              (F trace-roots trace-bs trace-final as dg
                                composition-roots composition-bs
                                composition-final query-chunks))))))))))
    by (rule after-trace-final)
  then show ?thesis
    unfolding Pair by simp
qed
qed
have after-trace-root-record:
  ∧fr F. hash-target-program B
  (0 +
    (?trace-fri +
      (?trace-final +
        (0 +
          (?alpha +
            (?degree +
              (0 +
                (?composition-fri +
                  (?composition-final + (0 + (?query + 0))))))))))
  (record-staged-message fr >>=
    (λ-. staged-trace-fri-program A 0 (ceil-log clength) [] >>=
      (λ(trace-roots, trace-bs).
        trace-final-stage A trace-bs >>=
          (λtrace-final. record-staged-message trace-final >>=
            (λ-. staged-alpha-program (length spec) >>=
              (λas. degree-stage A as >>=
                (λdg. record-staged-message dg >>=
                  (λ-. (assert (ceil-log (to-nat dg + 1) ≤
                    ceil-log (maxDegree + 1)) >>=
                      (λ-. staged-composition-fri-program A dg 0
                        (ceil-log (to-nat dg + 1)) [])) >>=
                        (λ(composition-roots, composition-bs).

```

```

composition-final-stage A dg composition-bs >>=
  (λcomposition-final.
    record-staged-message composition-final >>=
      (λ-. staged-query-program A 0 rounds >>=
        (λquery-chunks.
          return
            (F trace-roots trace-bs trace-final as dg
              composition-roots composition-bs
              composition-final query-chunks)))))))))
by (rule hash-target-program-bind)
  (rule hash-target-program-record-staged-message, rule after-trace-fri)
have whole:
  hash-target-program B
  (?trace-root +
    (0 +
      (?trace-fri +
        (?trace-final +
          (0 +
            (?alpha +
              (?degree +
                (0 +
                  (?composition-fri +
                    (?composition-final + (0 + (?query + 0)))))))))))
    (staged-transcript-program A)
  unfolding staged-transcript-program-def Let-def
proof (rule hash-target-program-bind)
show hash-target-program B ?trace-root (trace-root-stage A)
  by (rule trace-root-target)
fix fr
show hash-target-program B
  (0 +
    (?trace-fri +
      (?trace-final +
        (0 +
          (?alpha +
            (?degree +
              (0 +
                (?composition-fri +
                  (?composition-final + (0 + (?query + 0))))))))))
  (record-staged-message fr >>=
    (λ-. staged-trace-fri-program A 0 (ceil-log clength) [] >>=
      (λ(trace-roots, trace-bs).
        trace-final-stage A trace-bs >>=
          (λtrace-final.
            record-staged-message trace-final >>=
              (λ-. staged-alpha-program (length spec) >>=
                (λas. degree-stage A as >>=
                  (λdg. record-staged-message dg >>=
                    (λ-. assert (ceil-log (to-nat dg + 1) ≤

```



```

    ceil-log (maxDegree + 1)) >>=
  (λ-. staged-composition-fri-program A dg 0
    (ceil-log (to-nat dg + 1)) [] >>=
    (λ(composition-roots, composition-bs).
      composition-final-stage A dg composition-bs >>=
      (λcomposition-final.
        record-staged-message
          composition-final >>=
          (λ-. staged-query-program A 0
            rounds >>=
            (λquery-chunks.
              return
                (λ(staged-trace-root = fr,
                  staged-trace-fri-roots =
                    trace-roots,
                  staged-trace-fri-challenges =
                    trace-bs,
                  staged-trace-final =
                    trace-final,
                  staged-alphas = as,
                  staged-degree = dg,
                  staged-composition-fri-roots =
                    composition-roots,
                  staged-composition-fri-challenges =
                    composition-bs,
                  staged-composition-final =
                    composition-final,
                  staged-query-chunks =
                    query-chunks)))))))))))))
using after-trace-root-record
  [of fr
    λtrace-roots trace-bs trace-final as dg composition-roots
      composition-bs composition-final query-chunks.
      (λ(staged-trace-root = fr,
        staged-trace-fri-roots = trace-roots,
        staged-trace-fri-challenges = trace-bs,
        staged-trace-final = trace-final,
        staged-alphas = as,
        staged-degree = dg,
        staged-composition-fri-roots = composition-roots,
        staged-composition-fri-challenges = composition-bs,
        staged-composition-final = composition-final,
        staged-query-chunks = query-chunks))]
by (simp add: sm-bind-assoc)
qed
have budget-eq:
  ?trace-root +
    (0 +
      (?trace-fri +

```

```

      (?trace-final +
      (0 +
      (?alpha +
      (?degree +
      (0 +
      (?composition-fri +
      (?composition-final + (0 + (?query + 0)))))))))) =
    staged-attacker-query-budget budgets + staged-challenge-query-budget
  unfolding staged-attacker-query-budget-def
    staged-challenge-query-budget-def
  by (simp add: add.assoc add.commute add.left-commute)
show ?thesis
  using whole unfolding budget-eq .
qed

```

```

lemma hash-target-program-checked-staged-transcript-program:
  assumes wf: staged-budget-wellformed budgets
    and controlled: staged-adversary-controlled budgets A
  shows
    hash-target-program B
      (staged-attacker-query-budget budgets + staged-challenge-query-budget)
      (checked-staged-transcript-program A)
proof -
  let ?trace-root = trace-root-budget budgets
  let ?trace-fri =
    sum-list (trace-fri-budgets budgets) + ceil-log clength
  let ?trace-final = trace-final-budget budgets
  let ?alpha = length spec
  let ?degree = degree-budget budgets
  let ?composition-fri =
    sum-list (composition-fri-budgets budgets) +
    ceil-log (maxDegree + 1)
  let ?composition-final = composition-final-budget budgets
  let ?query =
    sum-list (query-opening-budgets budgets) + rounds
  have trace-root-target:
    hash-target-program B ?trace-root (trace-root-stage A)
  using controlled unfolding staged-adversary-controlled-def
  by (blast intro: controlled-ro-program-target)
  have trace-fri-target:
    hash-target-program B ?trace-fri
      (staged-trace-fri-program A 0 (ceil-log clength) [])
proof -
  have len:
    0 + ceil-log clength ≤ length (trace-fri-budgets budgets)
  using wf unfolding staged-budget-wellformed-def by simp
  have exact:
    hash-target-program B
      (sum-list

```

```

    (take (ceil-log clength)
      (drop 0 (trace-fri-budgets budgets))) +
    ceil-log clength)
  (staged-trace-fri-program A 0 (ceil-log clength) [])
  by (rule hash-target-program-staged-trace-fri-program
    [OF controlled len])
  then show ?thesis
    using wf unfolding staged-budget-wellformed-def by simp
qed
have trace-final-target:
   $\wedge bs.$  hash-target-program B ?trace-final (trace-final-stage A bs)
  using controlled unfolding staged-adversary-controlled-def
  by (blast intro: controlled-ro-program-target)
have alpha-target:
  hash-target-program B ?alpha (staged-alpha-program (length spec))
  by (rule hash-target-program-staged-alpha-program)
have degree-target:
   $\wedge as.$  hash-target-program B ?degree (degree-stage A as)
  using controlled unfolding staged-adversary-controlled-def
  by (blast intro: controlled-ro-program-target)
have composition-fri-target:
   $\wedge dg.$  hash-target-program B ?composition-fri
    (assert (ceil-log (to-nat dg + 1)  $\leq$  ceil-log (maxDegree + 1))  $\gg$ 
      ( $\lambda.$  staged-composition-fri-program A dg 0
        (ceil-log (to-nat dg + 1)) []))
  by (rule hash-target-program-guarded-staged-composition-fri-program
    [OF wf controlled])
have composition-final-target:
   $\wedge dg$  bs. hash-target-program B ?composition-final
    (composition-final-stage A dg bs)
  using controlled unfolding staged-adversary-controlled-def
  by (blast intro: controlled-ro-program-target)
have query-target:
   $\wedge$  trace-roots composition-roots.
    hash-target-program B ?query
      (checked-staged-query-program A trace-roots composition-roots
        0 rounds)
proof -
  fix trace-roots composition-roots
  have len:
    0 + rounds  $\leq$  length (query-opening-budgets budgets)
    using wf unfolding staged-budget-wellformed-def by simp
  have exact:
    hash-target-program B
      (sum-list
        (take rounds (drop 0 (query-opening-budgets budgets))) +
        rounds)
      (checked-staged-query-program A trace-roots composition-roots
        0 rounds)

```

```

    by (rule hash-target-program-checked-staged-query-program
        [OF controlled len])
  then show
    hash-target-program B ?query
      (checked-staged-query-program A trace-roots composition-roots
        0 rounds)
    using wf unfolding staged-budget-wellformed-def by simp
qed
have query-tail:
   $\wedge$  trace-roots composition-roots F. hash-target-program B (?query + 0)
    (checked-staged-query-program A trace-roots composition-roots
      0 rounds  $\gg$ 
      ( $\lambda$  query-chunks. return (F query-chunks)))
  by (rule hash-target-program-bind)
    (rule query-target, rule hash-target-program-return)
have after-composition-final-record:
   $\wedge$  trace-roots composition-roots composition-final F.
    hash-target-program B (0 + (?query + 0))
    (record-staged-message composition-final  $\gg$ 
      ( $\lambda$ -. checked-staged-query-program A trace-roots
        composition-roots 0 rounds  $\gg$ 
        ( $\lambda$  query-chunks. return (F query-chunks))))
  by (rule hash-target-program-bind)
    (rule hash-target-program-record-staged-message, rule query-tail)
have after-composition-final:
   $\wedge$  trace-roots dg composition-roots bs F. hash-target-program B
    (?composition-final + (0 + (?query + 0)))
    (composition-final-stage A dg bs  $\gg$ 
      ( $\lambda$  composition-final.
        record-staged-message composition-final  $\gg$ 
        ( $\lambda$ -. checked-staged-query-program A trace-roots
          composition-roots 0 rounds  $\gg$ 
          ( $\lambda$  query-chunks.
            return (F composition-final query-chunks))))))
  by (rule hash-target-program-bind)
    (rule composition-final-target, rule after-composition-final-record)
have after-composition-fri:
   $\wedge$  trace-roots dg F. hash-target-program B
    (?composition-fri +
      (?composition-final + (0 + (?query + 0))))
    ((assert (ceil-log (to-nat dg + 1)  $\leq$ 
      ceil-log (maxDegree + 1))  $\gg$ 
      ( $\lambda$ -. staged-composition-fri-program A dg 0
        (ceil-log (to-nat dg + 1)) []))  $\gg$ 
      ( $\lambda$ (composition-roots, composition-bs).
        composition-final-stage A dg composition-bs  $\gg$ 
        ( $\lambda$  composition-final.
          record-staged-message composition-final  $\gg$ 
          ( $\lambda$ -. checked-staged-query-program A trace-roots
            composition-roots 0 rounds  $\gg$ 
            ( $\lambda$  query-chunks. return (F composition-final query-chunks))))))

```

```

      composition-roots 0 rounds  $\gg$ 
      ( $\lambda$ query-chunks.
        return
          (F composition-roots composition-bs composition-final
            query-chunks))))))
proof (rule hash-target-program-bind)
  fix trace-roots dg F
  show hash-target-program B ?composition-fri
    (assert (ceil-log (to-nat dg + 1)  $\leq$ 
      ceil-log (maxDegree + 1))  $\gg$ 
      ( $\lambda$ -. staged-composition-fri-program A dg 0
        (ceil-log (to-nat dg + 1)) []))
    by (rule composition-fri-target)
next
  fix trace-roots dg F x
  show hash-target-program B
    (?composition-final + (0 + (?query + 0)))
    (case x of (composition-roots, composition-bs)  $\Rightarrow$ 
      composition-final-stage A dg composition-bs  $\gg$ 
      ( $\lambda$ composition-final.
        record-staged-message composition-final  $\gg$ 
        ( $\lambda$ -. checked-staged-query-program A trace-roots
          composition-roots 0 rounds  $\gg$ 
          ( $\lambda$ query-chunks.
            return
              (F composition-roots composition-bs composition-final
                query-chunks))))))
proof (cases x)
  case (Pair composition-roots composition-bs)
  have hash-target-program B
    (?composition-final + (0 + (?query + 0)))
    (composition-final-stage A dg composition-bs  $\gg$ 
    ( $\lambda$ composition-final.
      record-staged-message composition-final  $\gg$ 
      ( $\lambda$ -. checked-staged-query-program A trace-roots
        composition-roots 0 rounds  $\gg$ 
        ( $\lambda$ query-chunks.
          return
            (F composition-roots composition-bs composition-final
              query-chunks))))))
    by (rule after-composition-final)
  then show ?thesis
    unfolding Pair by simp
qed
qed
have after-degree-record:
   $\wedge$ trace-roots dg F. hash-target-program B
    (0 +
      (?composition-fri +

```

```

    (?composition-final + (0 + (?query + 0))))))
  (record-staged-message dg >>=
    (λ-. (assert (ceil-log (to-nat dg + 1) ≤
      ceil-log (maxDegree + 1)) >>=
    (λ-. staged-composition-fri-program A dg 0
      (ceil-log (to-nat dg + 1)) [])) >>=
    (λ(composition-roots, composition-bs).
      composition-final-stage A dg composition-bs >>=
      (λcomposition-final.
        record-staged-message composition-final >>=
        (λ-. checked-staged-query-program A trace-roots
          composition-roots 0 rounds >>=
          (λquery-chunks.
            return
              (F composition-roots composition-bs composition-final
                query-chunks)))))))
  by (rule hash-target-program-bind)
  (rule hash-target-program-record-staged-message,
    rule after-composition-fri)
have after-degree:
  ∧trace-roots as F. hash-target-program B
  (?degree +
    (0 +
      (?composition-fri +
        (?composition-final + (0 + (?query + 0))))))
  (degree-stage A as >>=
    (λdg. record-staged-message dg >>=
      (λ-. (assert (ceil-log (to-nat dg + 1) ≤
        ceil-log (maxDegree + 1)) >>=
      (λ-. staged-composition-fri-program A dg 0
        (ceil-log (to-nat dg + 1)) [])) >>=
      (λ(composition-roots, composition-bs).
        composition-final-stage A dg composition-bs >>=
        (λcomposition-final.
          record-staged-message composition-final >>=
          (λ-. checked-staged-query-program A trace-roots
            composition-roots 0 rounds >>=
            (λquery-chunks.
              return
                (F dg composition-roots composition-bs
                  composition-final query-chunks)))))))
  by (rule hash-target-program-bind)
  (rule degree-target, rule after-degree-record)
have after-alpha:
  ∧trace-roots F. hash-target-program B
  (?alpha +
    (?degree +
      (0 +
        (?composition-fri +

```

```

    (?composition-final + (0 + (?query + 0))))))
  (staged-alpha-program (length spec) >>=
    (λas. degree-stage A as >>=
      (λdg. record-staged-message dg >>=
        (λ-. (assert (ceil-log (to-nat dg + 1) ≤
          ceil-log (maxDegree + 1)) >>=
            (λ-. staged-composition-fri-program A dg 0
              (ceil-log (to-nat dg + 1)) [])) >>=
              (λ(composition-roots, composition-bs).
                composition-final-stage A dg composition-bs >>=
                  (λcomposition-final.
                    record-staged-message composition-final >>=
                      (λ-. checked-staged-query-program A trace-roots
                        composition-roots 0 rounds >>=
                          (λquery-chunks.
                            return
                              (F as dg composition-roots composition-bs
                                composition-final query-chunks))))))))))
  by (rule hash-target-program-bind)
  (rule alpha-target, rule after-degree)
have after-trace-final-record:
  ∧trace-roots trace-final F. hash-target-program B
  (0 +
    (?alpha +
      (?degree +
        (0 +
          (?composition-fri +
            (?composition-final + (0 + (?query + 0))))))))
  (record-staged-message trace-final >>=
    (λ-. staged-alpha-program (length spec) >>=
      (λas. degree-stage A as >>=
        (λdg. record-staged-message dg >>=
          (λ-. (assert (ceil-log (to-nat dg + 1) ≤
            ceil-log (maxDegree + 1)) >>=
              (λ-. staged-composition-fri-program A dg 0
                (ceil-log (to-nat dg + 1)) [])) >>=
                (λ(composition-roots, composition-bs).
                  composition-final-stage A dg composition-bs >>=
                    (λcomposition-final.
                      record-staged-message composition-final >>=
                        (λ-. checked-staged-query-program A trace-roots
                          composition-roots 0 rounds >>=
                            (λquery-chunks.
                              return
                                (F as dg composition-roots composition-bs
                                  composition-final query-chunks))))))))))
  by (rule hash-target-program-bind)
  (rule hash-target-program-record-staged-message, rule after-alpha)
have after-trace-final:

```

```

 $\wedge$ trace-roots trace-bs  $F$ . hash-target-program  $B$ 
  (?trace-final +
    (0 +
      (?alpha +
        (?degree +
          (0 +
            (?composition-fri +
              (?composition-final + (0 + (?query + 0))))))))))
  (trace-final-stage  $A$  trace-bs  $\gg$ 
    ( $\lambda$ trace-final. record-staged-message trace-final  $\gg$ 
      ( $\lambda$ -. staged-alpha-program (length spec)  $\gg$ 
        ( $\lambda$ as. degree-stage  $A$  as  $\gg$ 
          ( $\lambda$ dg. record-staged-message dg  $\gg$ 
            ( $\lambda$ -. (assert (ceil-log (to-nat dg + 1)  $\leq$ 
              ceil-log (maxDegree + 1))  $\gg$ 
              ( $\lambda$ -. staged-composition-fri-program  $A$  dg 0
                (ceil-log (to-nat dg + 1)) []))  $\gg$ 
              ( $\lambda$ (composition-roots, composition-bs).
                composition-final-stage  $A$  dg composition-bs  $\gg$ 
                ( $\lambda$ composition-final.
                  record-staged-message composition-final  $\gg$ 
                  ( $\lambda$ -. checked-staged-query-program  $A$  trace-roots
                    composition-roots 0 rounds  $\gg$ 
                    ( $\lambda$ query-chunks.
                      return
                        ( $F$  trace-final as dg composition-roots
                          composition-bs composition-final
                          query-chunks))))))))))
  by (rule hash-target-program-bind)
  (rule trace-final-target, rule after-trace-final-record)
have after-trace-fri:
 $\wedge F$ . hash-target-program  $B$ 
  (?trace-fri +
    (?trace-final +
      (0 +
        (?alpha +
          (?degree +
            (0 +
              (?composition-fri +
                (?composition-final + (0 + (?query + 0))))))))))
  (staged-trace-fri-program  $A$  0 (ceil-log clength) []  $\gg$ 
    ( $\lambda$ (trace-roots, trace-bs).
      trace-final-stage  $A$  trace-bs  $\gg$ 
      ( $\lambda$ trace-final. record-staged-message trace-final  $\gg$ 
        ( $\lambda$ -. staged-alpha-program (length spec)  $\gg$ 
          ( $\lambda$ as. degree-stage  $A$  as  $\gg$ 
            ( $\lambda$ dg. record-staged-message dg  $\gg$ 
              ( $\lambda$ -. (assert (ceil-log (to-nat dg + 1)  $\leq$ 
                ceil-log (maxDegree + 1))  $\gg$ 

```



```

      (λ-. staged-composition-fri-program A dg 0
        (ceil-log (to-nat dg + 1)) [])) >>=
      (λ(composition-roots, composition-bs).
        composition-final-stage A dg composition-bs >>=
        (λcomposition-final.
          record-staged-message composition-final >>=
          (λ-. checked-staged-query-program A
            trace-roots composition-roots 0 rounds >>=
            (λquery-chunks.
              return
              (F trace-roots trace-bs trace-final as dg
                composition-roots composition-bs
                composition-final query-chunks)))))))))
proof (rule hash-target-program-bind)
fix F
show hash-target-program B ?trace-fri
  (staged-trace-fri-program A 0 (ceil-log clength) [])
  by (rule trace-fri-target)
next
fix F x
show hash-target-program B
  (?trace-final +
    (0 +
      (?alpha +
        (?degree +
          (0 +
            (?composition-fri +
              (?composition-final + (0 + (?query + 0))))))))))
  (case x of (trace-roots, trace-bs) =>
    trace-final-stage A trace-bs >>=
    (λtrace-final. record-staged-message trace-final >>=
      (λ-. staged-alpha-program (length spec) >>=
        (λas. degree-stage A as >>=
          (λdg. record-staged-message dg >>=
            (λ-. (assert (ceil-log (to-nat dg + 1)) ≤
              ceil-log (maxDegree + 1)) >>=
              (λ-. staged-composition-fri-program A dg 0
                (ceil-log (to-nat dg + 1)) [])) >>=
              (λ(composition-roots, composition-bs).
                composition-final-stage A dg composition-bs >>=
                (λcomposition-final.
                  record-staged-message composition-final >>=
                  (λ-. checked-staged-query-program A
                    trace-roots composition-roots 0 rounds >>=
                    (λquery-chunks.
                      return
                      (F trace-roots trace-bs trace-final as dg
                        composition-roots composition-bs
                        composition-final query-chunks)))))))))

```

```

proof (cases x)
  case (Pair trace-roots trace-bs)
  have hash-target-program B
    (?trace-final +
      (0 +
        (?alpha +
          (?degree +
            (0 +
              (?composition-fri +
                (?composition-final + (0 + (?query + 0))))))))))
    (trace-final-stage A trace-bs  $\gg$ 
      ( $\lambda$ trace-final. record-staged-message trace-final  $\gg$ 
        ( $\lambda$ -. staged-alpha-program (length spec)  $\gg$ 
          ( $\lambda$ as. degree-stage A as  $\gg$ 
            ( $\lambda$ dg. record-staged-message dg  $\gg$ 
              ( $\lambda$ -. (assert (ceil-log (to-nat dg + 1)  $\leq$ 
                ceil-log (maxDegree + 1))  $\gg$ 
                ( $\lambda$ -. staged-composition-fri-program A dg 0
                  (ceil-log (to-nat dg + 1)) [])  $\gg$ 
                  ( $\lambda$ (composition-roots, composition-bs).
                    composition-final-stage A dg composition-bs  $\gg$ 
                    ( $\lambda$ composition-final.
                      record-staged-message composition-final  $\gg$ 
                      ( $\lambda$ -. checked-staged-query-program A
                        trace-roots composition-roots 0 rounds  $\gg$ 
                        ( $\lambda$ query-chunks.
                          return
                            (F trace-roots trace-bs trace-final as dg
                              composition-roots composition-bs
                              composition-final query-chunks))))))))))
      by (rule after-trace-final)
      then show ?thesis
      unfolding Pair by simp
    qed
  qed
have after-trace-root-record:
   $\wedge$ fr F. hash-target-program B
    (0 +
      (?trace-fri +
        (?trace-final +
          (0 +
            (?alpha +
              (?degree +
                (0 +
                  (?composition-fri +
                    (?composition-final + (0 + (?query + 0))))))))))
    (record-staged-message fr  $\gg$ 
      ( $\lambda$ -. staged-trace-fri-program A 0 (ceil-log clength) [])  $\gg$ 
        ( $\lambda$ (trace-roots, trace-bs).

```

```

    trace-final-stage A trace-bs >>=
    (λtrace-final. record-staged-message trace-final >>=
      (λ-. staged-alpha-program (length spec) >>=
        (λas. degree-stage A as >>=
          (λdg. record-staged-message dg >>=
            (λ-. (assert (ceil-log (to-nat dg + 1) ≤
              ceil-log (maxDegree + 1))) >>=
              (λ-. staged-composition-fri-program A dg 0
                (ceil-log (to-nat dg + 1)) [])) >>=
              (λ(composition-roots, composition-bs).
                composition-final-stage A dg composition-bs >>=
                (λcomposition-final.
                  record-staged-message composition-final >>=
                  (λ-. checked-staged-query-program A
                    trace-roots composition-roots 0 rounds >>=
                    (λquery-chunks.
                      return
                      (F trace-roots trace-bs trace-final as dg
                        composition-roots composition-bs
                        composition-final query-chunks))))))))))
  by (rule hash-target-program-bind)
  (rule hash-target-program-record-staged-message, rule after-trace-fri)
have whole:
  hash-target-program B
  (?trace-root +
    (0 +
      (?trace-fri +
        (?trace-final +
          (0 +
            (?alpha +
              (?degree +
                (0 +
                  (?composition-fri +
                    (?composition-final + (0 + (?query + 0))))))))))
    (checked-staged-transcript-program A)
  unfolding checked-staged-transcript-program-def Let-def
proof (rule hash-target-program-bind)
  show hash-target-program B ?trace-root (trace-root-stage A)
  by (rule trace-root-target)
next
fix fr
show hash-target-program B
  (0 +
    (?trace-fri +
      (?trace-final +
        (0 +
          (?alpha +
            (?degree +
              (0 +

```

```

      (?composition-fri +
        (?composition-final + (0 + (?query + 0)))))))))
(record-staged-message fr >>=
  (λ-. staged-trace-fri-program A 0 (ceil-log clength) [] >>=
    (λ(trace-roots, trace-bs).
      trace-final-stage A trace-bs >>=
        (λtrace-final.
          record-staged-message trace-final >>=
            (λ-. staged-alpha-program (length spec) >>=
              (λas. degree-stage A as >>=
                (λdg. record-staged-message dg >>=
                  (λ-. assert (ceil-log (to-nat dg + 1) ≤
                    ceil-log (maxDegree + 1)) >>=
                    (λ-. staged-composition-fri-program A dg 0
                      (ceil-log (to-nat dg + 1)) [] >>=
                        (λ(composition-roots, composition-bs).
                          composition-final-stage A dg composition-bs >>=
                            (λcomposition-final.
                              record-staged-message
                                composition-final >>=
                                  (λ-. checked-staged-query-program A
                                    trace-roots composition-roots 0
                                      rounds >>=
                                        (λquery-chunks.
                                          return
                                            (λ(staged-trace-root = fr,
                                              staged-trace-fri-roots =
                                                trace-roots,
                                              staged-trace-fri-challenges =
                                                trace-bs,
                                              staged-trace-final =
                                                trace-final,
                                              staged-alphas = as,
                                              staged-degree = dg,
                                              staged-composition-fri-roots =
                                                composition-roots,
                                              staged-composition-fri-challenges =
                                                composition-bs,
                                              staged-composition-final =
                                                composition-final,
                                              staged-query-chunks =
                                                query-chunks)))))))))))))
using after-trace-root-record
[of fr
  λtrace-roots trace-bs trace-final as dg composition-roots
    composition-bs composition-final query-chunks.
    (λ(staged-trace-root = fr,
      staged-trace-fri-roots = trace-roots,
      staged-trace-fri-challenges = trace-bs,

```

```

    staged-trace-final = trace-final,
    staged-alphas = as,
    staged-degree = dg,
    staged-composition-fri-roots = composition-roots,
    staged-composition-fri-challenges = composition-bs,
    staged-composition-final = composition-final,
    staged-query-chunks = query-chunks)]
  by (simp add: sm-bind-assoc)
qed
have budget-eq:
  ?trace-root +
  (0 +
    (?trace-fri +
      (?trace-final +
        (0 +
          (?alpha +
            (?degree +
              (0 +
                (?composition-fri +
                  (?composition-final + (0 + (?query + 0)))))))))) =
    staged-attacker-query-budget budgets + staged-challenge-query-budget
  unfolding staged-attacker-query-budget-def
    staged-challenge-query-budget-def
  by (simp add: add.assoc add.commute add.left-commute)
show ?thesis
  using whole unfolding budget-eq .
qed

lemma hash-target-program-staged-semantic-adversary:
  assumes wf: staged-budget-wellformed budgets
  and controlled: staged-adversary-controlled budgets A
  shows
    hash-target-program B
      (staged-attacker-query-budget budgets + staged-challenge-query-budget)
      (staged-semantic-adversary A)
proof -
  have hash-target-program B
    ((staged-attacker-query-budget budgets + staged-challenge-query-budget) +
     0)
    (staged-semantic-adversary A)
  unfolding staged-semantic-adversary-def
proof (rule hash-target-program-bind
  [where n=staged-attacker-query-budget budgets +
    staged-challenge-query-budget and n'=0])
  show hash-target-program B
    (staged-attacker-query-budget budgets + staged-challenge-query-budget)
    (staged-transcript-program A)
  by (rule hash-target-program-staged-transcript-program[OF wf controlled])
  show  $\bigwedge x.$  hash-target-program B 0

```

```

    (return (staged-proof-transcript x))
  by (rule hash-target-program-return)
qed
then show ?thesis by simp
qed

lemma hash-extension-preserving-staged-semantic-adversary:
  assumes wf: staged-budget-wellformed budgets
    and controlled: staged-adversary-controlled budgets A
  shows hash-extension-preserving (staged-semantic-adversary A)
  by (rule hash-target-program-extension
    [OF hash-target-program-staged-semantic-adversary
      [OF wf controlled, of {}]])

lemma admissible-staged-semantic-adversary:
  assumes wf: staged-budget-wellformed budgets
    and controlled: staged-adversary-controlled budgets A
  shows
    admissible-adversary
      (staged-attacker-query-budget budgets + staged-challenge-query-budget)
      (staged-semantic-adversary A)
  unfolding admissible-adversary-def
proof (intro conjI allI)
  show hash-extension-preserving (staged-semantic-adversary A)
    by (rule hash-extension-preserving-staged-semantic-adversary
      [OF wf controlled])
  show hash-range-budget
    (staged-attacker-query-budget budgets + staged-challenge-query-budget)
    (staged-semantic-adversary A)
    by (rule hash-range-budget-staged-semantic-adversary[OF wf controlled])
  show hash-collision-budget
    (staged-attacker-query-budget budgets + staged-challenge-query-budget)
    (staged-semantic-adversary A)
    by (rule hash-collision-budget-staged-semantic-adversary
      [OF wf controlled])
fix B
  show hash-target-program B
    (staged-attacker-query-budget budgets + staged-challenge-query-budget)
    (staged-semantic-adversary A)
    by (rule hash-target-program-staged-semantic-adversary[OF wf controlled])
qed

lemma staged-trace-fri-program-Suc-outcomeE:
  assumes outcome:
    Some ((roots, bs'), t) ∈
      set-dist (execute (staged-trace-fri-program A i (Suc n) bs) s)
  obtains root b roots' s1 s2 s3 where
    Some (root, s1) ∈
      set-dist (execute (trace-fri-root-stage A i bs) s)

```

```

Some ((), s2) ∈
  set-dist (execute (record-staged-message root) s1)
Some (b, s3) ∈
  set-dist (execute receive-trace-fri-challenge s2)
Some ((roots', bs'), t) ∈
  set-dist
    (execute
      (staged-trace-fri-program A (Suc i) n (bs @ [b])) s3)
roots = root # roots'
using outcome
by (auto elim!: set-dist-bindE intro: that)

```

lemma staged-composition-fri-program-Suc-outcomeE:

```

assumes outcome:
  Some ((roots, bs'), t) ∈
    set-dist
      (execute
        (staged-composition-fri-program A dg i (Suc n) bs) s)
obtains root b roots' s1 s2 s3 where
  Some (root, s1) ∈
    set-dist (execute (composition-fri-root-stage A dg i bs) s)
  Some ((), s2) ∈
    set-dist (execute (record-staged-message root) s1)
  Some (b, s3) ∈
    set-dist (execute receive-composition-fri-challenge s2)
  Some ((roots', bs'), t) ∈
    set-dist
      (execute
        (staged-composition-fri-program
          A dg (Suc i) n (bs @ [b])) s3)
  roots = root # roots'
using outcome
by (auto elim!: set-dist-bindE intro: that)

```

lemma staged-alpha-program-Suc-outcomeE:

```

assumes outcome:
  Some (as, t) ∈
    set-dist (execute (staged-alpha-program (Suc n)) s)
obtains a as' s1 s2 where
  Some (a, s1) ∈
    set-dist (execute receive-alpha-challenge s)
  Some ((), s2) ∈
    set-dist (execute (record-staged-message a) s1)
  Some (as', t) ∈
    set-dist (execute (staged-alpha-program n) s2)
  as = a # as'
using outcome
by (auto elim!: set-dist-bindE intro: that)

```

lemma *staged-query-program-Suc-outcomeE*:

assumes *outcome*:

Some (chunks, t) ∈
set-dist (execute (staged-query-program A i (Suc n)) s)

obtains *raw chunk chunks' s1 s2 s3 where*

Some (raw, s1) ∈
set-dist (execute receive-query-index-challenge s)
Some (chunk, s2) ∈
set-dist (execute (query-opening-stage A i raw) s1)
Some ((), s3) ∈
set-dist (execute (record-staged-messages chunk) s2)
Some (chunks', t) ∈
set-dist (execute (staged-query-program A (Suc i) n) s3)
chunks = chunk # chunks'

using *outcome*

by (*auto elim! set-dist-bindE intro that*)

lemma *checked-staged-query-program-Suc-outcomeE*:

assumes *outcome*:

Some (chunks, t) ∈
set-dist
(execute
(checked-staged-query-program A trace-roots composition-roots
i (Suc n)) s)

obtains *raw chunk chunks' s1 s2 s3 where*

Some (raw, s1) ∈
set-dist (execute receive-query-index-challenge s)
Some (chunk, s2) ∈
set-dist (execute (query-opening-stage A i raw) s1)
verifier-query-round-chunk (index (to-nat raw))
trace-roots composition-roots chunk
Some ((), s3) ∈
set-dist (execute (record-staged-messages chunk) s2)
Some (chunks', t) ∈
set-dist
(execute
(checked-staged-query-program A trace-roots composition-roots
(Suc i) n) s3)
chunks = chunk # chunks'

using *outcome*

unfolding *checked-staged-query-program.simps assert-def Let-def*

by (*auto simp: throw-no-outcome elim! set-dist-bindE intro that*
split: if-splits)

lemma *checked-staged-query-program-outcome-imp-staged-query-program*:

assumes *outcome*:

Some (chunks, t) ∈
set-dist
(execute


```

      (checked-staged-query-program A trace-roots composition-roots
        i n) s)
shows
  Some (chunks, t) ∈
    set-dist (execute (staged-query-program A i n) s)
using outcome
proof (induction n arbitrary: i chunks s t)
  case 0
  then show ?case
  by simp
next
  case (Suc n)
  obtain raw chunk chunks' s1 s2 s3 where
    challenge-out:
      Some (raw, s1) ∈
        set-dist (execute receive-query-index-challenge s)
  and stage-out:
      Some (chunk, s2) ∈
        set-dist (execute (query-opening-stage A i raw) s1)
  and record-out:
      Some ((), s3) ∈
        set-dist (execute (record-staged-messages chunk) s2)
  and rest-out:
      Some (chunks', t) ∈
        set-dist
          (execute
            (checked-staged-query-program A trace-roots composition-roots
              (Suc i) n) s3)
  and chunks-eq: chunks = chunk # chunks'
proof (rule checked-staged-query-program-Suc-outcomeE[OF Suc.prem])
  fix raw chunk chunks' s1 s2 s3
  assume challenge:
    Some (raw, s1) ∈
      set-dist (execute receive-query-index-challenge s)
  and stage:
    Some (chunk, s2) ∈
      set-dist (execute (query-opening-stage A i raw) s1)
  and recorded:
    Some ((), s3) ∈
      set-dist (execute (record-staged-messages chunk) s2)
  and rest:
    Some (chunks', t) ∈
      set-dist
        (execute
          (checked-staged-query-program A trace-roots composition-roots
            (Suc i) n) s3)
  and chunks: chunks = chunk # chunks'
show ?thesis
  by (rule that[OF challenge stage recorded rest chunks])

```

```

qed
have rest-unchecked:
  Some (chunks', t) ∈
    set-dist (execute (staged-query-program A (Suc i) n) s3)
  by (rule Suc.IH[OF rest-out])
show ?case
  unfolding chunks-eq staged-query-program.simps
  by (intro set-dist-bindI[OF challenge-out]
    set-dist-bindI[OF stage-out]
    set-dist-bindI[OF record-out]
    set-dist-bindI[OF rest-unchecked])
  simp
qed

lemma checked-staged-transcript-program-outcome-imp-staged-transcript-program:
assumes outcome:
  Some (data, t) ∈
    set-dist
      (execute (checked-staged-transcript-program A) s)
shows
  Some (data, t) ∈
    set-dist (execute (staged-transcript-program A) s)
using outcome
unfolding checked-staged-transcript-program-def staged-transcript-program-def
  Let-def
by (auto elim!: set-dist-bindE
  intro!: set-dist-bindI
  intro: checked-staged-query-program-outcome-imp-staged-query-program
  split: prod.splits)

lemma
checked-staged-query-challenge-prefix-outcome-imp-staged-query-prefix:
assumes outcome:
  Some (prefix, t) ∈
    set-dist
      (execute (checked-staged-query-challenge-prefix-program A i) s)
shows
  Some ((sqp-degree prefix, sqp-composition-final prefix,
    sqp-query-chunks prefix), t) ∈
    set-dist (execute (staged-query-challenge-prefix-program A i) s)
using outcome
unfolding checked-staged-query-challenge-prefix-program-def
  staged-query-challenge-prefix-program-def Let-def
by (auto elim!: set-dist-bindE
  intro!: set-dist-bindI
  intro: checked-staged-query-program-outcome-imp-staged-query-program
  split: prod.splits)

lemma checked-staged-query-program-wp-event-le-staged:

```

```

assumes none:  $\neg P \text{ None}$ 
shows
  wp-event
    (checked-staged-query-program A trace-roots composition-roots i n)
    P s  $\leq$ 
    wp-event (staged-query-program A i n) P s
using none
proof (induction n arbitrary: i s P)
  case 0
  then show ?case by simp
next
  case (Suc n)
  let ?checked =
    checked-staged-query-program A trace-roots composition-roots i (Suc n)
  let ?plain = staged-query-program A i (Suc n)
  show ?case
    unfolding checked-staged-query-program.simps
      staged-query-program.simps Let-def
  proof (rule wp-event-bind-mono-cont)
    show P None  $\implies$  P None .
next
  fix raw s1
  assume raw-out:
    Some (raw, s1)  $\in$ 
      set-dist (execute receive-query-index-challenge s)
  show
    wp-event
      (query-opening-stage A i raw  $\gg=$ 
        ( $\lambda$ chunk.
          assert
            (verifier-query-round-chunk (index (to-nat raw))
              trace-roots composition-roots chunk)  $\gg=$ 
            ( $\lambda$ -.
              record-staged-messages chunk  $\gg=$ 
              ( $\lambda$ -.
                checked-staged-query-program A trace-roots
                  composition-roots (Suc i) n  $\gg=$ 
                  ( $\lambda$ chunks. return (chunk # chunks)))))))
      P s1
     $\leq$ 
    wp-event
      (query-opening-stage A i raw  $\gg=$ 
        ( $\lambda$ chunk.
          record-staged-messages chunk  $\gg=$ 
          ( $\lambda$ -.
            staged-query-program A (Suc i) n  $\gg=$ 
            ( $\lambda$ chunks. return (chunk # chunks))))))
      P s1
  proof (rule wp-event-bind-mono-cont)

```

```

show  $P \text{ None} \implies P \text{ None} .$ 
next
fix  $chunk\ s2$ 
assume  $chunk\text{-out}:$ 
   $Some\ (chunk, s2) \in$ 
     $set\text{-}dist\ (execute\ (query\text{-}opening\text{-}stage\ A\ i\ raw)\ s1)$ 
let  $?tailP =$ 
   $\lambda out. case\ out\ of\ None \Rightarrow P\ None$ 
     $| Some\ (chunks, t) \Rightarrow P\ (Some\ (chunk\ \# \ chunks, t))$ 
have  $tail\text{-}none: \neg ?tailP\ None$ 
  using  $Suc.premis\ by\ simp$ 
have  $checked\text{-}tail\text{-}le:$ 
   $\bigwedge s'.$ 
   $wp\text{-}event$ 
     $(checked\text{-}staged\text{-}query\text{-}program\ A\ trace\text{-}roots\ composition\text{-}roots$ 
       $(Suc\ i)\ n)$ 
     $?tailP\ s'$ 
   $\leq$ 
   $wp\text{-}event\ (staged\text{-}query\text{-}program\ A\ (Suc\ i)\ n)\ ?tailP\ s'$ 
by  $(rule\ Suc.IH)\ (rule\ tail\text{-}none)$ 
have  $after\text{-}record:$ 
   $wp\text{-}event$ 
     $(record\text{-}staged\text{-}messages\ chunk \gg=$ 
       $(\lambda\text{-}.$ 
         $checked\text{-}staged\text{-}query\text{-}program\ A\ trace\text{-}roots$ 
         $composition\text{-}roots\ (Suc\ i)\ n \gg=$ 
         $(\lambda chunks. return\ (chunk\ \# \ chunks))))$ 
     $P\ s2$ 
   $\leq$ 
   $wp\text{-}event$ 
     $(record\text{-}staged\text{-}messages\ chunk \gg=$ 
       $(\lambda\text{-}.$ 
         $staged\text{-}query\text{-}program\ A\ (Suc\ i)\ n \gg=$ 
         $(\lambda chunks. return\ (chunk\ \# \ chunks))))$ 
     $P\ s2$ 
proof  $(rule\ wp\text{-}event\text{-}bind\text{-}mono\text{-}cont)$ 
show  $P \text{ None} \implies P \text{ None} .$ 
next
fix  $u\ s3$ 
assume  $Some\ (u, s3) \in$ 
   $set\text{-}dist\ (execute\ (record\text{-}staged\text{-}messages\ chunk)\ s2)$ 
show
   $wp\text{-}event$ 
     $(checked\text{-}staged\text{-}query\text{-}program\ A\ trace\text{-}roots\ composition\text{-}roots$ 
       $(Suc\ i)\ n \gg=$ 
       $(\lambda chunks. return\ (chunk\ \# \ chunks)))$ 
     $P\ s3$ 
   $\leq$ 
   $wp\text{-}event$ 

```

```

      (staged-query-program A (Suc i) n  $\gg$ 
        ( $\lambda$ chunks. return (chunk # chunks)))
    P s3
  unfolding wp-event-bind-return-map
  using checked-tail-le[of s3]
  by simp
qed
have checked-assert:
  wp-event
  (assert
    (verifier-query-round-chunk (index (to-nat raw))
      trace-roots composition-roots chunk)  $\gg$ 
    ( $\lambda$ -.
      record-staged-messages chunk  $\gg$ 
      ( $\lambda$ -.
        checked-staged-query-program A trace-roots
          composition-roots (Suc i) n  $\gg$ 
          ( $\lambda$ chunks. return (chunk # chunks))))))
  P s2
 $\leq$ 
  wp-event
  (record-staged-messages chunk  $\gg$ 
    ( $\lambda$ -.
      checked-staged-query-program A trace-roots
        composition-roots (Suc i) n  $\gg$ 
        ( $\lambda$ chunks. return (chunk # chunks))))
  P s2
by (rule wp-event-assert-bind-le) (rule Suc.prem)
show
  wp-event
  (assert
    (verifier-query-round-chunk (index (to-nat raw))
      trace-roots composition-roots chunk)  $\gg$ 
    ( $\lambda$ -.
      record-staged-messages chunk  $\gg$ 
      ( $\lambda$ -.
        checked-staged-query-program A trace-roots
          composition-roots (Suc i) n  $\gg$ 
          ( $\lambda$ chunks. return (chunk # chunks))))))
  P s2
 $\leq$ 
  wp-event
  (record-staged-messages chunk  $\gg$ 
    ( $\lambda$ -.
      staged-query-program A (Suc i) n  $\gg$ 
      ( $\lambda$ chunks. return (chunk # chunks))))
  P s2
by (rule order-trans[OF checked-assert after-record])
qed

```

qed
qed

lemma *checked-staged-transcript-program-wp-event-le-staged:*
 assumes *none:* $\neg P \text{ None}$
 shows
 wp-event (checked-staged-transcript-program A) P s ≤
 wp-event (staged-transcript-program A) P s
 unfolding *checked-staged-transcript-program-def staged-transcript-program-def*
 Let-def
 apply (rule *wp-event-bind-mono-cont*)
 apply *simp*
 apply (rename-tac fr s1)
 apply (rule *wp-event-bind-mono-cont*)
 apply *simp*
 apply (rename-tac u1 s2)
 apply (rule *wp-event-bind-mono-cont*)
 apply *simp*
 apply (rename-tac trace-pair s3)
 apply (case-tac trace-pair)
 apply (rename-tac trace-roots trace-bs)
 apply *simp*
 apply (rule *wp-event-bind-mono-cont*)
 apply *simp*
 apply (rename-tac trace-final s4)
 apply (rule *wp-event-bind-mono-cont*)
 apply *simp*
 apply (rename-tac u2 s5)
 apply (rule *wp-event-bind-mono-cont*)
 apply *simp*
 apply (rename-tac as s6)
 apply (rule *wp-event-bind-mono-cont*)
 apply *simp*
 apply (rename-tac dg s7)
 apply (rule *wp-event-bind-mono-cont*)
 apply *simp*
 apply (rename-tac u3 s8)
 apply (rule *wp-event-bind-mono-cont*)
 apply *simp*
 apply (rename-tac u4 s9)
 apply (rule *wp-event-bind-mono-cont*)
 apply *simp*
 apply (rename-tac composition-pair s10)
 apply (case-tac composition-pair)
 apply (rename-tac composition-roots composition-bs)
 apply *simp*
 apply (rule *wp-event-bind-mono-cont*)
 apply *simp*
 apply (rename-tac composition-final s11)

```

apply (rule wp-event-bind-mono-cont)
apply simp
apply (rename-tac u5 s12)
apply (subst wp-event-bind-return-map)
apply (subst wp-event-bind-return-map)
apply (rule checked-staged-query-program-wp-event-le-staged)
apply (simp add: none)
done

```

```

lemma hash-range-budget-checked-staged-query-challenge-prefix-program:
  assumes wf: staged-budget-wellformed budgets
    and controlled: staged-adversary-controlled budgets A
    and i-bound:  $i \leq \text{rounds}$ 
  shows
    hash-range-budget
      (staged-query-search-queries budgets i)
      (checked-staged-query-challenge-prefix-program A i)
  unfolding hash-range-budget-def
proof (intro allI impI)
  fix s prefix t
  assume outcome:
    Some (prefix, t)  $\in$ 
      set-dist
        (execute (checked-staged-query-challenge-prefix-program A i) s)
  have unchecked-out:
    Some ((sqp-degree prefix, sqp-composition-final prefix,
      sqp-query-chunks prefix), t)  $\in$ 
      set-dist (execute (staged-query-challenge-prefix-program A i) s)
  by (rule checked-staged-query-challenge-prefix-outcome-imp-staged-query-prefix
    [OF outcome])
  have budget:
    hash-range-budget
      (staged-query-search-queries budgets i)
      (staged-query-challenge-prefix-program A i)
  by (rule hash-range-budget-staged-query-challenge-prefix-program
    [OF wf controlled i-bound])
  show card (hash-map-output-values t)
     $\leq$  card (hash-map-output-values s) +
      staged-query-search-queries budgets i
  using budget unchecked-out unfolding hash-range-budget-def by blast
qed

```

```

lemma hash-collision-budget-checked-staged-query-challenge-prefix-program:
  assumes wf: staged-budget-wellformed budgets
    and controlled: staged-adversary-controlled budgets A
    and i-bound:  $i \leq \text{rounds}$ 
  shows
    hash-collision-budget
      (staged-query-search-queries budgets i)

```

```

    (checked-staged-query-challenge-prefix-program A i)
  unfolding hash-collision-budget-def
proof (intro allI impI)
  fix s :: 'f protocol-channel
  assume clean:  $\neg$  hash-map-output-collision s
  have checked-le:
    wp-event (checked-staged-query-challenge-prefix-program A i)
      (hash-new-collision-event s) s  $\leq$ 
    wp-event (staged-query-challenge-prefix-program A i)
      (hash-new-collision-event s) s
  unfolding checked-staged-query-challenge-prefix-program-def
    staged-query-challenge-prefix-program-def Let-def
  apply (rule wp-event-bind-mono-cont)
  apply (simp add: hash-new-collision-event-def)
  apply (rename-tac alpha-prefix s1)
  apply (case-tac alpha-prefix)
  apply (rename-tac fr trace-roots trace-bs trace-final)
  apply simp
  apply (rule wp-event-bind-mono-cont)
  apply (simp add: hash-new-collision-event-def)
  apply (rename-tac as s2)
  apply (rule wp-event-bind-mono-cont)
  apply (simp add: hash-new-collision-event-def)
  apply (rename-tac dg s3)
  apply (rule wp-event-bind-mono-cont)
  apply (simp add: hash-new-collision-event-def)
  apply (rename-tac u s4)
  apply (rule wp-event-bind-mono-cont)
  apply (simp add: hash-new-collision-event-def)
  apply (rename-tac u2 s5)
  apply (rule wp-event-bind-mono-cont)
  apply (simp add: hash-new-collision-event-def)
  apply (rename-tac composition-pair s6)
  apply (case-tac composition-pair)
  apply (rename-tac composition-roots composition-bs)
  apply simp
  apply (rule wp-event-bind-mono-cont)
  apply (simp add: hash-new-collision-event-def)
  apply (rename-tac composition-final s7)
  apply (rule wp-event-bind-mono-cont)
  apply (simp add: hash-new-collision-event-def)
  apply (rename-tac u3 s8)
  apply (subst wp-event-bind-return-map)
  apply (subst wp-event-bind-return-map)
  apply (simp add: hash-new-collision-event-def)
  apply (rule checked-staged-query-program-wp-event-le-staged)
  apply (simp add: hash-new-collision-event-def)
  done
  have unchecked-bound:

```



```

wp-event (staged-query-challenge-prefix-program A i)
  (hash-new-collision-event s) s ≤
  hash-collision-budget-value (card (hash-map-output-values s))
  (staged-query-search-queries budgets i)
using hash-collision-budget-staged-query-challenge-prefix-program
  [OF wf controlled i-bound] clean
unfolding hash-collision-budget-def by blast
show wp-event (checked-staged-query-challenge-prefix-program A i)
  (hash-new-collision-event s) s
  ≤ hash-collision-budget-value (card (hash-map-output-values s))
  (staged-query-search-queries budgets i)
by (rule order-trans[OF checked-le unchecked-bound])
qed

lemma hash-range-budget-checked-staged-transcript-program:
assumes wf: staged-budget-wellformed budgets
and controlled: staged-adversary-controlled budgets A
shows
  hash-range-budget
    (staged-attacker-query-budget budgets + staged-challenge-query-budget)
    (checked-staged-transcript-program A)
unfolding hash-range-budget-def
proof (intro allI impI)
fix s data t
assume outcome:
  Some (data, t) ∈
    set-dist (execute (checked-staged-transcript-program A) s)
have unchecked-out:
  Some (data, t) ∈ set-dist (execute (staged-transcript-program A) s)
by (rule checked-staged-transcript-program-outcome-imp-staged-transcript-program
    [OF outcome])
have budget:
  hash-range-budget
    (staged-attacker-query-budget budgets + staged-challenge-query-budget)
    (staged-transcript-program A)
by (rule hash-range-budget-staged-transcript-program[OF wf controlled])
show card (hash-map-output-values t)
  ≤ card (hash-map-output-values s) +
    (staged-attacker-query-budget budgets + staged-challenge-query-budget)
using budget unchecked-out unfolding hash-range-budget-def by blast
qed

lemma hash-collision-budget-checked-staged-transcript-program:
assumes wf: staged-budget-wellformed budgets
and controlled: staged-adversary-controlled budgets A
shows
  hash-collision-budget
    (staged-attacker-query-budget budgets + staged-challenge-query-budget)
    (checked-staged-transcript-program A)

```

```

unfolding hash-collision-budget-def
proof (intro allI impI)
  fix s :: 'f protocol-channel
  assume clean:  $\neg$  hash-map-output-collision s
  let ?budget =
    staged-attacker-query-budget budgets + staged-challenge-query-budget
  have checked-le:
    wp-event (checked-staged-transcript-program A)
      (hash-new-collision-event s) s  $\leq$ 
    wp-event (staged-transcript-program A)
      (hash-new-collision-event s) s
  by (rule checked-staged-transcript-program-wp-event-le-staged)
    (simp add: hash-new-collision-event-def)
  have unchecked-bound:
    wp-event (staged-transcript-program A)
      (hash-new-collision-event s) s  $\leq$ 
    hash-collision-budget-value (card (hash-map-output-values s)) ?budget
  using hash-collision-budget-staged-transcript-program[OF wf controlled]
    clean
  unfolding hash-collision-budget-def by blast
  show wp-event (checked-staged-transcript-program A)
    (hash-new-collision-event s) s
     $\leq$  hash-collision-budget-value (card (hash-map-output-values s)) ?budget
  by (rule order-trans[OF checked-le unchecked-bound])
qed

```

lemma checked-staged-query-prefix-target-hit-bound:

assumes wf: staged-budget-wellformed budgets
and controlled: staged-adversary-controlled budgets A
and i-bound: $i \leq$ rounds

shows

wp-event (checked-staged-query-challenge-prefix-program A i)
 (hash-new-output-hit-event B s) s $\leq$
 staged-phase-target-error B
 (staged-query-search-queries budgets i)

proof –

have checked-le-unchecked:

wp-event (checked-staged-query-challenge-prefix-program A i)
 (hash-new-output-hit-event B s) s $\leq$
 wp-event (staged-query-challenge-prefix-program A i)
 (hash-new-output-hit-event B s) s

unfolding checked-staged-query-challenge-prefix-program-def
 staged-query-challenge-prefix-program-def Let-def

apply (rule wp-event-bind-mono-cont)

apply (simp add: hash-new-output-hit-event-def)

apply (rename-tac alpha-prefix s1)

apply (case-tac alpha-prefix)

apply (rename-tac fr trace-roots trace-bs trace-final)

apply simp

```

apply (rule wp-event-bind-mono-cont)
  apply (simp add: hash-new-output-hit-event-def)
apply (rename-tac as s2)
apply (rule wp-event-bind-mono-cont)
  apply (simp add: hash-new-output-hit-event-def)
apply (rename-tac dg s3)
apply (rule wp-event-bind-mono-cont)
  apply (simp add: hash-new-output-hit-event-def)
apply (rename-tac u s4)
apply (rule wp-event-bind-mono-cont)
  apply (simp add: hash-new-output-hit-event-def)
apply (rename-tac u2 s5)
apply (rule wp-event-bind-mono-cont)
  apply (simp add: hash-new-output-hit-event-def)
apply (rename-tac composition-pair s6)
apply (case-tac composition-pair)
apply (rename-tac composition-roots composition-bs)
apply simp
apply (rule wp-event-bind-mono-cont)
  apply (simp add: hash-new-output-hit-event-def)
apply (rename-tac composition-final s7)
apply (rule wp-event-bind-mono-cont)
  apply (simp add: hash-new-output-hit-event-def)
apply (rename-tac u3 s8)
apply (subst wp-event-bind-return-map)
apply (subst wp-event-bind-return-map)
apply (simp add: hash-new-output-hit-event-def)
apply (rule checked-staged-query-program-wp-event-le-staged)
apply simp
done
also have ...  $\leq$ 
  staged-phase-target-error B
  (staged-query-search-queries budgets i)
by (rule staged-query-prefix-target-hit-bound[OF wf controlled i-bound])
finally show ?thesis .
qed

lemma checked-staged-query-prefix-bad-index-target-bound:
assumes wf: staged-budget-wellformed budgets
and controlled: staged-adversary-controlled budgets A
and i-bound:  $i \leq \text{rounds}$ 
shows
  wp-event (checked-staged-query-challenge-prefix-program A i)
    (hash-new-output-hit-event (query-index-raw-preimage B) s) s  $\leq$ 
    staged-query-prefix-bad-index-error budgets B i
unfolding staged-query-prefix-bad-index-error-def
by (rule checked-staged-query-prefix-target-hit-bound
  [OF wf controlled i-bound])

```

end

end

theory *Staged-Security-Experiment-Checked-Prefix*
 imports *Staged-Security-Experiment-Targets*
begin

Checked staged-prefix replay, alignment, and transcript extraction lemmas.

context *soundness*
begin

lemma *hash-relation-program-checked-staged-query-challenge-prefix-program:*

assumes *wf: staged-budget-wellformed budgets*
 and *controlled: staged-adversary-controlled budgets A*
 and *i-bound: $i \leq \text{rounds}$*
 and *fibers: $\bigwedge x. \text{card } \{y. R \ x \ y\} \leq b$*

shows

hash-relation-program R b
 (staged-query-search-queries budgets i)
 (checked-staged-query-challenge-prefix-program A i)

proof —

let *?alpha-prefix = staged-alpha-search-queries budgets 0*

let *?alpha = length spec*

let *?degree = degree-budget budgets*

let *?composition-fri =*
 sum-list (composition-fri-budgets budgets) +
 ceil-log (maxDegree + 1)

let *?composition-final = composition-final-budget budgets*

let *?query = sum-list (take i (query-opening-budgets budgets)) + i*

have *alpha-prefix:*

hash-relation-program R b ?alpha-prefix
 (staged-alpha-prefix-program A)

by (rule *hash-relation-program-staged-alpha-prefix-program*
 [*OF wf controlled fibers*])

have *alpha:*

hash-relation-program R b ?alpha
 (staged-alpha-program (length spec))

by (rule *hash-relation-program-staged-alpha-program*[*OF fibers*])

have *degree:*

$\bigwedge as. \text{hash-relation-program } R \ b \ ?degree \ (\text{degree-stage } A \ as)$

proof —

fix *as*

have *controlled-degree:*

controlled-ro-program ?degree (degree-stage A as)

using *controlled unfolding staged-adversary-controlled-def* **by** *blast*

show *hash-relation-program R b ?degree (degree-stage A as)*

by (rule *controlled-ro-program-relation*

```

    [OF controlled-degree fibers])
qed
have composition-fri:
   $\bigwedge dg.$  hash-relation-program  $R$   $b$  ?composition-fri
    (assert (ceil-log (to-nat dg + 1)  $\leq$  ceil-log (maxDegree + 1))  $\gg$ 
      ( $\lambda$ -. staged-composition-fri-program  $A$  dg 0
        (ceil-log (to-nat dg + 1)) []))
  by (rule hash-relation-program-guarded-staged-composition-fri-program
    [OF wf controlled fibers])
have composition-final:
   $\bigwedge dg$  composition-bs. hash-relation-program  $R$   $b$  ?composition-final
    (composition-final-stage  $A$  dg composition-bs)
proof -
  fix dg composition-bs
  have controlled-final:
    controlled-ro-program ?composition-final
      (composition-final-stage  $A$  dg composition-bs)
    using controlled unfolding staged-adversary-controlled-def by blast
  show hash-relation-program  $R$   $b$  ?composition-final
    (composition-final-stage  $A$  dg composition-bs)
    by (rule controlled-ro-program-relation
      [OF controlled-final fibers])
qed
have query:
   $\bigwedge$  trace-roots composition-roots.
    hash-relation-program  $R$   $b$  ?query
      (checked-staged-query-program  $A$  trace-roots composition-roots 0  $i$ )
proof -
  fix trace-roots composition-roots
  have len:  $0 + i \leq \text{length}(\text{query-opening-budgets budgets})$ 
    using wf i-bound unfolding staged-budget-wellformed-def by simp
  have exact:
    hash-relation-program  $R$   $b$ 
      (sum-list (take  $i$  (drop 0 (query-opening-budgets budgets))) +  $i$ )
      (checked-staged-query-program  $A$  trace-roots composition-roots 0  $i$ )
    by (rule hash-relation-program-checked-staged-query-program
      [OF controlled len fibers])
  then show hash-relation-program  $R$   $b$  ?query
    (checked-staged-query-program  $A$  trace-roots composition-roots 0  $i$ )
    by simp
qed
have after-query:
   $\bigwedge$  fr trace-roots trace-bs trace-final as dg composition-roots
    composition-bs composition-final.
    hash-relation-program  $R$   $b$  (?query + 0)
      (checked-staged-query-program  $A$  trace-roots composition-roots 0  $i$   $\gg$ 
        ( $\lambda$ query-chunks. return
          ( $\sqcup$ sqp-trace-root = fr,
            sqp-trace-fri-roots = trace-roots,

```

```

    sqp-trace-fri-challenges = trace-bs,
    sqp-trace-final = trace-final,
    sqp-alphas = as,
    sqp-degree = dg,
    sqp-composition-fri-roots = composition-roots,
    sqp-composition-fri-challenges = composition-bs,
    sqp-composition-final = composition-final,
    sqp-query-chunks = query-chunks)))
  by (rule hash-relation-program-bind)
    (rule query,
      rule hash-relation-program-zero[OF hash-map-preserving-return])
  have after-composition-final-record:
     $\bigwedge$ fr trace-roots trace-bs trace-final as dg composition-roots
      composition-bs composition-final.
    hash-relation-program R b (0 + (?query + 0))
    (record-staged-message composition-final  $\gg$ 
      ( $\lambda$ -. checked-staged-query-program A trace-roots
        composition-roots 0 i  $\gg$ 
          ( $\lambda$ query-chunks. return
            ( $\lambda$ sqp-trace-root = fr,
              sqp-trace-fri-roots = trace-roots,
              sqp-trace-fri-challenges = trace-bs,
              sqp-trace-final = trace-final,
              sqp-alphas = as,
              sqp-degree = dg,
              sqp-composition-fri-roots = composition-roots,
              sqp-composition-fri-challenges = composition-bs,
              sqp-composition-final = composition-final,
              sqp-query-chunks = query-chunks))))))
  by (rule hash-relation-program-bind)
    (rule hash-relation-program-record-staged-message, rule after-query)
  have after-composition-final:
     $\bigwedge$ fr trace-roots trace-bs trace-final as dg composition-roots
      composition-bs.
    hash-relation-program R b
    (?composition-final + (0 + (?query + 0)))
    (composition-final-stage A dg composition-bs  $\gg$ 
      ( $\lambda$ composition-final.
        record-staged-message composition-final  $\gg$ 
          ( $\lambda$ -. checked-staged-query-program A trace-roots
            composition-roots 0 i  $\gg$ 
              ( $\lambda$ query-chunks. return
                ( $\lambda$ sqp-trace-root = fr,
                  sqp-trace-fri-roots = trace-roots,
                  sqp-trace-fri-challenges = trace-bs,
                  sqp-trace-final = trace-final,
                  sqp-alphas = as,
                  sqp-degree = dg,
                  sqp-composition-fri-roots = composition-roots,

```

```

      sqp-composition-fri-challenges = composition-bs,
      sqp-composition-final = composition-final,
      sqp-query-chunks = query-chunks))))))
  by (rule hash-relation-program-bind)
    (rule composition-final, rule after-composition-final-record)
  have after-composition-fri:
     $\bigwedge$  fr trace-roots trace-bs trace-final as dg.
    hash-relation-program R b
      (?composition-fri +
        (?composition-final + (0 + (?query + 0))))
      ((assert (ceil-log (to-nat dg + 1)  $\leq$  ceil-log (maxDegree + 1))  $\gg$ 
        ( $\lambda$ -. staged-composition-fri-program A dg 0
          (ceil-log (to-nat dg + 1)) []))  $\gg$ 
        ( $\lambda$ (composition-roots, composition-bs).
          composition-final-stage A dg composition-bs  $\gg$ 
            ( $\lambda$ composition-final.
              record-staged-message composition-final  $\gg$ 
                ( $\lambda$ -. checked-staged-query-program A trace-roots
                  composition-roots 0 i  $\gg$ 
                    ( $\lambda$ query-chunks. return
                      ( $\downarrow$ sqp-trace-root = fr,
                        sqp-trace-fri-roots = trace-roots,
                        sqp-trace-fri-challenges = trace-bs,
                        sqp-trace-final = trace-final,
                        sqp-alphas = as,
                        sqp-degree = dg,
                        sqp-composition-fri-roots = composition-roots,
                        sqp-composition-fri-challenges = composition-bs,
                        sqp-composition-final = composition-final,
                        sqp-query-chunks = query-chunks)))))))
  proof -
  fix fr trace-roots trace-bs trace-final as dg
  have cont:
     $\bigwedge$  x. hash-relation-program R b
      (?composition-final + (0 + (?query + 0)))
      (case x of (composition-roots, composition-bs)  $\Rightarrow$ 
        composition-final-stage A dg composition-bs  $\gg$ 
          ( $\lambda$ composition-final.
            record-staged-message composition-final  $\gg$ 
              ( $\lambda$ -. checked-staged-query-program A trace-roots
                composition-roots 0 i  $\gg$ 
                  ( $\lambda$ query-chunks. return
                    ( $\downarrow$ sqp-trace-root = fr,
                      sqp-trace-fri-roots = trace-roots,
                      sqp-trace-fri-challenges = trace-bs,
                      sqp-trace-final = trace-final,
                      sqp-alphas = as,
                      sqp-degree = dg,
                      sqp-composition-fri-roots = composition-roots,

```

```

      sqp-composition-fri-challenges = composition-bs,
      sqp-composition-final = composition-final,
      sqp-query-chunks = query-chunks))))))
proof –
  fix x
  show hash-relation-program R b
    (?composition-final + (0 + (?query + 0)))
    (case x of (composition-roots, composition-bs) ⇒
      composition-final-stage A dg composition-bs ≫=
      (λcomposition-final.
        record-staged-message composition-final ≫=
        (λ-. checked-staged-query-program A trace-roots
          composition-roots 0 i ≫=
          (λquery-chunks. return
            (|sqp-trace-root = fr,
              sqp-trace-fri-roots = trace-roots,
              sqp-trace-fri-challenges = trace-bs,
              sqp-trace-final = trace-final,
              sqp-alphas = as,
              sqp-degree = dg,
              sqp-composition-fri-roots = composition-roots,
              sqp-composition-fri-challenges = composition-bs,
              sqp-composition-final = composition-final,
              sqp-query-chunks = query-chunks))))))
proof (cases x)
  case (Pair composition-roots composition-bs)
  have step:
    hash-relation-program R b
      (?composition-final + (0 + (?query + 0)))
      (composition-final-stage A dg composition-bs ≫=
      (λcomposition-final.
        record-staged-message composition-final ≫=
        (λ-. checked-staged-query-program A trace-roots
          composition-roots 0 i ≫=
          (λquery-chunks. return
            (|sqp-trace-root = fr,
              sqp-trace-fri-roots = trace-roots,
              sqp-trace-fri-challenges = trace-bs,
              sqp-trace-final = trace-final,
              sqp-alphas = as,
              sqp-degree = dg,
              sqp-composition-fri-roots = composition-roots,
              sqp-composition-fri-challenges = composition-bs,
              sqp-composition-final = composition-final,
              sqp-query-chunks = query-chunks))))))
    by (rule after-composition-final)
  show ?thesis
    unfolding Pair using step by simp
qed

```



```

qed
show hash-relation-program R b
  (?composition-fri +
    (?composition-final + (0 + (?query + 0))))
  ((assert (ceil-log (to-nat dg + 1) ≤ ceil-log (maxDegree + 1)) >>=
    (λ-. staged-composition-fri-program A dg 0
      (ceil-log (to-nat dg + 1)) [])) >>=
    (λ(composition-roots, composition-bs).
      composition-final-stage A dg composition-bs >>=
        (λcomposition-final.
          record-staged-message composition-final >>=
            (λ-. checked-staged-query-program A trace-roots
              composition-roots 0 i >>=
                (λquery-chunks. return
                  (|spp-trace-root = fr,
                    spp-trace-fri-roots = trace-roots,
                    spp-trace-fri-challenges = trace-bs,
                    spp-trace-final = trace-final,
                    spp-alphas = as,
                    spp-degree = dg,
                    spp-composition-fri-roots = composition-roots,
                    spp-composition-fri-challenges = composition-bs,
                    spp-composition-final = composition-final,
                    spp-query-chunks = query-chunks|))))))
  by (rule hash-relation-program-bind)
      (rule composition-fri, rule cont)

```

qed

have *after-degree-record*:

```

  ∧ fr trace-roots trace-bs trace-final as dg.
  hash-relation-program R b
    (0 + (?composition-fri +
      (?composition-final + (0 + (?query + 0)))))
  (record-staged-message dg >>=
    (λ-. (assert
      (ceil-log (to-nat dg + 1) ≤ ceil-log (maxDegree + 1)) >>=
        (λ-. staged-composition-fri-program A dg 0
          (ceil-log (to-nat dg + 1)) [])) >>=
          (λ(composition-roots, composition-bs).
            composition-final-stage A dg composition-bs >>=
              (λcomposition-final.
                record-staged-message composition-final >>=
                  (λ-. checked-staged-query-program A trace-roots
                    composition-roots 0 i >>=
                      (λquery-chunks. return
                        (|spp-trace-root = fr,
                          spp-trace-fri-roots = trace-roots,
                          spp-trace-fri-challenges = trace-bs,
                          spp-trace-final = trace-final,
                          spp-alphas = as,

```

```

      sqp-degree = dg,
      sqp-composition-fri-roots = composition-roots,
      sqp-composition-fri-challenges = composition-bs,
      sqp-composition-final = composition-final,
      sqp-query-chunks = query-chunks))))))
by (rule hash-relation-program-bind)
  (rule hash-relation-program-record-staged-message,
   rule after-composition-fri)
have after-degree:
   $\bigwedge$  fr trace-roots trace-bs trace-final as.
    hash-relation-program R b
      (?degree + (0 + (?composition-fri +
        (?composition-final + (0 + (?query + 0))))))
      (degree-stage A as  $\gg$ 
        ( $\lambda$  dg. record-staged-message dg  $\gg$ 
          ( $\lambda$ -. let composition-rounds = ceil-log (to-nat dg + 1) in
            assert (composition-rounds  $\leq$  ceil-log (maxDegree + 1))  $\gg$ 
              ( $\lambda$ -. staged-composition-fri-program A dg 0
                composition-rounds  $\square$   $\gg$ 
                  ( $\lambda$ (composition-roots, composition-bs).
                    composition-final-stage A dg composition-bs  $\gg$ 
                      ( $\lambda$ composition-final.
                        record-staged-message composition-final  $\gg$ 
                          ( $\lambda$ -. checked-staged-query-program A trace-roots
                            composition-roots 0 i  $\gg$ 
                              ( $\lambda$ query-chunks. return
                                ( $\sqcup$ sqp-trace-root = fr,
                                  sqp-trace-fri-roots = trace-roots,
                                  sqp-trace-fri-challenges = trace-bs,
                                  sqp-trace-final = trace-final,
                                  sqp-alphas = as,
                                  sqp-degree = dg,
                                  sqp-composition-fri-roots = composition-roots,
                                  sqp-composition-fri-challenges = composition-bs,
                                  sqp-composition-final = composition-final,
                                  sqp-query-chunks = query-chunks)))))))))

```

proof –

fix fr trace-roots trace-bs trace-final as

have cont:

```

 $\bigwedge$  dg. hash-relation-program R b
  (0 + (?composition-fri +
    (?composition-final + (0 + (?query + 0))))))
(record-staged-message dg  $\gg$ 
  ( $\lambda$ -. assert (ceil-log (to-nat dg + 1)  $\leq$  ceil-log (maxDegree + 1))  $\gg$ 
    ( $\lambda$ -. staged-composition-fri-program A dg 0
      (ceil-log (to-nat dg + 1))  $\square$   $\gg$ 
        ( $\lambda$ (composition-roots, composition-bs).
          composition-final-stage A dg composition-bs  $\gg$ 
            ( $\lambda$ composition-final.

```

```

record-staged-message composition-final >>=
(λ-. checked-staged-query-program A trace-roots
composition-roots 0 i >>=
(λquery-chunks. return
(⌊spp-trace-root = fr,
spp-trace-fri-roots = trace-roots,
spp-trace-fri-challenges = trace-bs,
spp-trace-final = trace-final,
spp-alphas = as,
spp-degree = dg,
spp-composition-fri-roots = composition-roots,
spp-composition-fri-challenges = composition-bs,
spp-composition-final = composition-final,
spp-query-chunks = query-chunks⌋))))))

proof –
fix dg
have step:
hash-relation-program R b
(0 + (?composition-fri +
(?composition-final + (0 + (?query + 0)))))
(record-staged-message dg >>=
(λ-. (assert
(ceil-log (to-nat dg + 1) ≤ ceil-log (maxDegree + 1)) >>=
(λ-. staged-composition-fri-program A dg 0
(ceil-log (to-nat dg + 1)) [])) >>=
(λ(composition-roots, composition-bs).
composition-final-stage A dg composition-bs >>=
(λcomposition-final.
record-staged-message composition-final >>=
(λ-. checked-staged-query-program A trace-roots
composition-roots 0 i >>=
(λquery-chunks. return
(⌊spp-trace-root = fr,
spp-trace-fri-roots = trace-roots,
spp-trace-fri-challenges = trace-bs,
spp-trace-final = trace-final,
spp-alphas = as,
spp-degree = dg,
spp-composition-fri-roots = composition-roots,
spp-composition-fri-challenges = composition-bs,
spp-composition-final = composition-final,
spp-query-chunks = query-chunks⌋))))))

by (rule after-degree-record)
show hash-relation-program R b
(0 + (?composition-fri +
(?composition-final + (0 + (?query + 0)))))
(record-staged-message dg >>=
(λ-. assert (ceil-log (to-nat dg + 1) ≤ ceil-log (maxDegree + 1)) >>=
(λ-. staged-composition-fri-program A dg 0

```

```

    (ceil-log (to-nat dg + 1)) [] >>=
    (λ(composition-roots, composition-bs).
      composition-final-stage A dg composition-bs >>=
      (λcomposition-final.
        record-staged-message composition-final >>=
        (λ-. checked-staged-query-program A trace-roots
          composition-roots 0 i >>=
          (λquery-chunks. return
            (⊔sqp-trace-root = fr,
              sqp-trace-fri-roots = trace-roots,
              sqp-trace-fri-challenges = trace-bs,
              sqp-trace-final = trace-final,
              sqp-alphas = as,
              sqp-degree = dg,
              sqp-composition-fri-roots = composition-roots,
              sqp-composition-fri-challenges = composition-bs,
              sqp-composition-final = composition-final,
              sqp-query-chunks = query-chunks)))))))))
  using step by (simp add: sm-bind-assoc)
qed
show hash-relation-program R b
  (?degree + (0 + (?composition-fri +
    (?composition-final + (0 + (?query + 0))))))
  (degree-stage A as >>=
    (λdg. record-staged-message dg >>=
      (λ-. let composition-rounds = ceil-log (to-nat dg + 1) in
        assert (composition-rounds ≤ ceil-log (maxDegree + 1)) >>=
        (λ-. staged-composition-fri-program A dg 0
          composition-rounds [] >>=
          (λ(composition-roots, composition-bs).
            composition-final-stage A dg composition-bs >>=
            (λcomposition-final.
              record-staged-message composition-final >>=
              (λ-. checked-staged-query-program A trace-roots
                composition-roots 0 i >>=
                (λquery-chunks. return
                  (⊔sqp-trace-root = fr,
                    sqp-trace-fri-roots = trace-roots,
                    sqp-trace-fri-challenges = trace-bs,
                    sqp-trace-final = trace-final,
                    sqp-alphas = as,
                    sqp-degree = dg,
                    sqp-composition-fri-roots = composition-roots,
                    sqp-composition-fri-challenges = composition-bs,
                    sqp-composition-final = composition-final,
                    sqp-query-chunks = query-chunks)))))))))
  unfolding Let-def
  by (rule hash-relation-program-bind)
    (rule degree, rule cont)

```

```

qed
have after-alpha:
   $\bigwedge$ fr trace-roots trace-bs trace-final.
  hash-relation-program R b
    (?alpha + (?degree + (0 + (?composition-fri +
      (?composition-final + (0 + (?query + 0)))))))
    (staged-alpha-program (length spec)  $\gg$ 
      ( $\lambda$ as. degree-stage A as  $\gg$ 
        ( $\lambda$ dg. record-staged-message dg  $\gg$ 
          ( $\lambda$ -. let composition-rounds = ceil-log (to-nat dg + 1) in
            assert (composition-rounds  $\leq$  ceil-log (maxDegree + 1))  $\gg$ 
              ( $\lambda$ -. staged-composition-fri-program A dg 0
                composition-rounds []  $\gg$ 
                  ( $\lambda$ (composition-roots, composition-bs).
                    composition-final-stage A dg composition-bs  $\gg$ 
                      ( $\lambda$ composition-final.
                        record-staged-message composition-final  $\gg$ 
                          ( $\lambda$ -. checked-staged-query-program A trace-roots
                            composition-roots 0 i  $\gg$ 
                              ( $\lambda$ query-chunks. return
                                ( $\lambda$ sqp-trace-root = fr,
                                  sqp-trace-fri-roots = trace-roots,
                                  sqp-trace-fri-challenges = trace-bs,
                                  sqp-trace-final = trace-final,
                                  sqp-alphas = as,
                                  sqp-degree = dg,
                                  sqp-composition-fri-roots = composition-roots,
                                  sqp-composition-fri-challenges = composition-bs,
                                  sqp-composition-final = composition-final,
                                  sqp-query-chunks = query-chunks))))))))))
      by (rule hash-relation-program-bind)
      (rule alpha, rule after-degree)
have after-alpha-prefix:
  hash-relation-program R b
    (?alpha-prefix +
      (?alpha + (?degree + (0 + (?composition-fri +
        (?composition-final + (0 + (?query + 0)))))))
      (checked-staged-query-challenge-prefix-program A i)
  unfolding checked-staged-query-challenge-prefix-program-def
proof (rule hash-relation-program-bind)
  show hash-relation-program R b ?alpha-prefix
    (staged-alpha-prefix-program A)
    by (rule alpha-prefix)
next
fix x
show hash-relation-program R b
  (?alpha + (?degree + (0 + (?composition-fri +
    (?composition-final + (0 + (?query + 0)))))))
  (case x of (fr, trace-roots, trace-bs, trace-final)  $\Rightarrow$ 

```

```

staged-alpha-program (length spec) >>=
  (λas. degree-stage A as >>=
    (λdg. record-staged-message dg >>=
      (λ-. let composition-rounds = ceil-log (to-nat dg + 1) in
        assert (composition-rounds ≤ ceil-log (maxDegree + 1)) >>=
          (λ-. staged-composition-fri-program A dg 0
            composition-rounds [] >>=
              (λ(composition-roots, composition-bs).
                composition-final-stage A dg composition-bs >>=
                  (λcomposition-final.
                    record-staged-message composition-final >>=
                      (λ-. checked-staged-query-program A trace-roots
                        composition-roots 0 i >>=
                          (λquery-chunks. return
                            (λsqp-trace-root = fr,
                              sqp-trace-fri-roots = trace-roots,
                              sqp-trace-fri-challenges = trace-bs,
                              sqp-trace-final = trace-final,
                              sqp-alphas = as,
                              sqp-degree = dg,
                              sqp-composition-fri-roots = composition-roots,
                              sqp-composition-fri-challenges = composition-bs,
                              sqp-composition-final = composition-final,
                              sqp-query-chunks = query-chunks))))))))))

```

proof (cases x)

case (fields fr trace-roots trace-bs trace-final)

have step:

```

hash-relation-program R b
  (?alpha + (?degree + (0 + (?composition-fri +
    (?composition-final + (0 + (?query + 0)))))))
  (staged-alpha-program (length spec) >>=
    (λas. degree-stage A as >>=
      (λdg. record-staged-message dg >>=
        (λ-. let composition-rounds = ceil-log (to-nat dg + 1) in
          assert (composition-rounds ≤ ceil-log (maxDegree + 1)) >>=
            (λ-. staged-composition-fri-program A dg 0
              composition-rounds [] >>=
                (λ(composition-roots, composition-bs).
                  composition-final-stage A dg composition-bs >>=
                    (λcomposition-final.
                      record-staged-message composition-final >>=
                        (λ-. checked-staged-query-program A trace-roots
                          composition-roots 0 i >>=
                            (λquery-chunks. return
                              (λsqp-trace-root = fr,
                                sqp-trace-fri-roots = trace-roots,
                                sqp-trace-fri-challenges = trace-bs,
                                sqp-trace-final = trace-final,
                                sqp-alphas = as,

```

```

      sqp-degree = dg,
      sqp-composition-fri-roots = composition-roots,
      sqp-composition-fri-challenges = composition-bs,
      sqp-composition-final = composition-final,
      sqp-query-chunks = query-chunks))))))))))
    by (rule after-alpha)
  show ?thesis
    unfolding fields using step by simp
qed
qed
show ?thesis
  using after-alpha-prefix
  unfolding staged-query-search-queries-def
    staged-alpha-search-queries-def
  by (simp add: add.assoc add.commute add.left-commute)
qed

```

lemma *staged-trace-fri-program-challenge-prefix-pre-alignment:*

```

  assumes outcome:
    Some ((roots, bs'), t) ∈
      set-dist (execute (staged-trace-fri-program A i n bs) s)
  shows take (length bs) bs' = bs
  using outcome
proof (induction n arbitrary: i bs roots bs' s t)
  case 0
  then show ?case by simp
next
  case (Suc n)
  from Suc.premis obtain root b roots' s1 s2 s3 where
    rest-out:
      Some ((roots', bs'), t) ∈
        set-dist
          (execute
            (staged-trace-fri-program A (Suc i) n (bs @ [b])) s3)
  unfolding staged-trace-fri-program.simps
  by (auto elim!: set-dist-bindE)
have prefix-suc:
  take (length (bs @ [b])) bs' = bs @ [b]
  by (rule Suc.IH[OF rest-out])
have take (length bs) bs' =
  take (length bs) (take (length (bs @ [b])) bs')
  by simp
also have ... = bs
  using prefix-suc by simp
finally show ?case .
qed

```

lemma *staged-trace-fri-program-alignment:*

assumes *controlled: staged-adversary-controlled budgets A*

```

and bound:  $i + n \leq \text{length } (\text{trace-fri-budgets } \text{budgets})$ 
and outcome:
  Some  $((\text{roots}, \text{bs}'), t) \in$ 
    set-dist (execute (staged-trace-fri-program  $A$   $i$   $n$   $\text{bs}$ )  $s$ )
shows
  length roots =  $n \wedge$ 
  length bs' = length bs +  $n \wedge$ 
  PTranscript  $t = P\text{Transcript } s @ \text{roots} \wedge$ 
  PState  $t = \text{foldl concat } (P\text{State } s) \text{ roots} \wedge$ 
   $s \leq t \wedge$ 
  PTraceFriCounter  $t = P\text{TraceFriCounter } s + n \wedge$ 
  PCompositionFriCounter  $t = P\text{CompositionFriCounter } s \wedge$ 
  PAlphaCounter  $t = P\text{AlphaCounter } s \wedge$ 
  PQueryCounter  $t = P\text{QueryCounter } s \wedge$ 
   $(\forall j < n.$ 
    fmlookup (HashMap  $t$ )
      (TraceFriChallenge (PTraceFriCounter  $s + j$ )
        (foldl concat (PState  $s$ ) (take (Suc  $j$ ) roots))) =
    Some  $(\text{bs}' ! (\text{length } \text{bs} + j)))$ 
  using bound outcome
proof (induction  $n$  arbitrary:  $i$   $\text{bs}$  roots  $\text{bs}'$   $s$   $t$ )
  case 0
  then show ?case by (simp add: hash-ext-refl)
next
  case (Suc  $n$ )
  from staged-trace-fri-program-Suc-outcomeE[OF Suc.prem(2)]
  obtain root  $b$  roots'  $s1$   $s2$   $s3$  where
    stage-out:
      Some (root,  $s1$ )  $\in$ 
        set-dist (execute (trace-fri-root-stage  $A$   $i$   $\text{bs}$ )  $s$ )
    and record-out:
      Some  $((), s2) \in$ 
        set-dist (execute (record-staged-message root)  $s1$ )
    and challenge-out:
      Some  $(b, s3) \in$ 
        set-dist (execute receive-trace-fri-challenge  $s2$ )
    and rest-out:
      Some  $((\text{roots}', \text{bs}'), t) \in$ 
        set-dist
          (execute
            (staged-trace-fri-program  $A$  (Suc  $i$ )  $n$  ( $\text{bs} @ [b]$ )  $s3$ )
          )
    and roots-eq: roots = root # roots'
  .
have i-bound:  $i < \text{length } (\text{trace-fri-budgets } \text{budgets})$ 
using Suc.prem(1) by simp
have stage-controlled:
  controlled-ro-program (trace-fri-budgets budgets !  $i$ )
    (trace-fri-root-stage  $A$   $i$   $\text{bs}$ )
using controlled i-bound

```


unfolding *staged-adversary-controlled-def* **by** *blast*
have *stage-fields*:
 $PState\ s1 = PState\ s \wedge$
 $PTranscript\ s1 = PTranscript\ s \wedge$
 $PTraceFriCounter\ s1 = PTraceFriCounter\ s \wedge$
 $PCompositionFriCounter\ s1 = PCompositionFriCounter\ s \wedge$
 $PAlphaCounter\ s1 = PAlphaCounter\ s \wedge$
 $PQueryCounter\ s1 = PQueryCounter\ s$
using *controlled-stage-outcome-fields*[*OF stage-controlled stage-out*]
by *simp*
have *ext-s-s1*: $s \leq s1$
using *controlled-ro-program-extension*[*OF stage-controlled*] *stage-out*
unfolding *hash-extension-preserving-def* **by** *blast*
have *s2*:
 $s2 = s1 \mid$
 $PState := concat\ (PState\ s1)\ root,$
 $PTranscript := PTranscript\ s1\ @\ [root]]$
by (*rule record-staged-message-outcome*[*OF record-out*])
have *ext-s1-s2*: $s1 \leq s2$
unfolding *s2 less-eq-hash-ext-def less-eq-fmap-def* **by** *simp*
have *s2-fields*:
 $PState\ s2 = concat\ (PState\ s)\ root \wedge$
 $PTranscript\ s2 = PTranscript\ s\ @\ [root] \wedge$
 $PTraceFriCounter\ s2 = PTraceFriCounter\ s \wedge$
 $PCompositionFriCounter\ s2 = PCompositionFriCounter\ s \wedge$
 $PAlphaCounter\ s2 = PAlphaCounter\ s \wedge$
 $PQueryCounter\ s2 = PQueryCounter\ s$
unfolding *s2 using stage-fields* **by** *simp*
have *challenge-fields*:
 $PState\ s3 = PState\ s2 \wedge$
 $PTranscript\ s3 = PTranscript\ s2$
using *receive-trace-fri-challenge-outcome*[*OF challenge-out*] **by** *simp*
have *ext-s2-s3*: $s2 \leq s3$
using *receive-trace-fri-challenge-outcome*[*OF challenge-out*] **by** *simp*
have *challenge-counters*:
 $PTraceFriCounter\ s3 = Suc\ (PTraceFriCounter\ s2) \wedge$
 $PCompositionFriCounter\ s3 = PCompositionFriCounter\ s2 \wedge$
 $PAlphaCounter\ s3 = PAlphaCounter\ s2 \wedge$
 $PQueryCounter\ s3 = PQueryCounter\ s2$
using *receive-trace-fri-challenge-counter-outcome*[*OF challenge-out*] **by** *simp*
have *rest-bound*:
 $Suc\ i + n \leq length\ (trace-fri-budgets\ budgets)$
using *Suc.premis(1)* **by** *simp*
have *rest-alignment*:
 $length\ roots' = n \wedge$
 $length\ bs' = length\ (bs\ @\ [b]) + n \wedge$
 $PTranscript\ t = PTranscript\ s3\ @\ roots' \wedge$
 $PState\ t = foldl\ concat\ (PState\ s3)\ roots' \wedge$
 $s3 \leq t \wedge$

```

PTraceFriCounter t = PTraceFriCounter s3 + n ∧
PCompositionFriCounter t = PCompositionFriCounter s3 ∧
PAlphaCounter t = PAlphaCounter s3 ∧
PQueryCounter t = PQueryCounter s3 ∧
(∀ j < n.
  fmlookup (HashMap t)
    (TraceFriChallenge (PTraceFriCounter s3 + j)
      (foldl concat (PState s3) (take (Suc j) roots'))) =
      Some (bs' ! (length (bs @ [b] + j)))
  by (rule Suc.IH[OF rest-bound rest-out])
have ext-s3-t: s3 ≤ t
  using rest-alignment by simp
have ext-s-t: s ≤ t
  by (rule hash-ext-trans[OF ext-s-s1])
    (rule hash-ext-trans[OF ext-s1-s2],
      rule hash-ext-trans[OF ext-s2-s3 ext-s3-t])
have challenge-lookup-s3:
  fmlookup (HashMap s3)
    (TraceFriChallenge (PTraceFriCounter s)
      (concat (PState s) root)) = Some b
  using receive-trace-fri-challenge-outcome[OF challenge-out]
    s2-fields by simp
have challenge-lookup-t:
  fmlookup (HashMap t)
    (TraceFriChallenge (PTraceFriCounter s)
      (concat (PState s) root)) = Some b
  by (rule hash-extension-lookup[OF challenge-lookup-s3 ext-s3-t])
have prefix-suc:
  take (length (bs @ [b])) bs' = bs @ [b]
  by (rule staged-trace-fri-program-challenge-prefix-pre-alignment
    [OF rest-out])
have lookup-all:
  ∀ j < Suc n.
    fmlookup (HashMap t)
      (TraceFriChallenge (PTraceFriCounter s + j)
        (foldl concat (PState s) (take (Suc j) (root # roots')))) =
        Some (bs' ! (length bs + j))
proof (intro allI impI)
  fix j
  assume j-bound: j < Suc n
  show
    fmlookup (HashMap t)
      (TraceFriChallenge (PTraceFriCounter s + j)
        (foldl concat (PState s) (take (Suc j) (root # roots')))) =
        Some (bs' ! (length bs + j))
proof (cases j)
  case 0
  have bs' ! length bs = b
  proof -

```

```

    have idx-lt: length bs < length (bs @ [b])
      by simp
    have bs' ! length bs =
      take (length (bs @ [b])) bs' ! length bs
      using idx-lt by simp
    also have ... = (bs @ [b]) ! length bs
      using prefix-suc by simp
    also have ... = b
      by simp
    finally show ?thesis .
  qed
  then show ?thesis
    using challenge-lookup-t unfolding 0 by simp
next
  case (Suc k)
  then have k-bound: k < n
    using j-bound by simp
  show ?thesis
    using rest-alignment k-bound s2-fields challenge-fields
      challenge-counters
    unfolding Suc
    by simp
  qed
qed
show ?case
  unfolding roots-eq
  using s2-fields challenge-fields challenge-counters rest-alignment
    ext-s-t lookup-all
  by simp
qed

lemma staged-trace-fri-program-challenge-prefix:
  assumes outcome:
    Some ((roots, bs'), t) ∈
      set-dist (execute (staged-trace-fri-program A i n bs) s)
  shows take (length bs) bs' = bs
  using outcome
proof (induction n arbitrary: i bs roots bs' s t)
  case 0
  then show ?case by (simp add: hash-ext-refl)
next
  case (Suc n)
  from Suc.prems obtain root b roots' s1 s2 s3 where
    rest-out:
      Some ((roots', bs'), t) ∈
        set-dist
          (execute
            (staged-trace-fri-program A (Suc i) n (bs @ [b]) s3))
  unfolding staged-trace-fri-program.simps

```

```

    by (auto elim!: set-dist-bindE)
  have prefix-suc:
    take (length (bs @ [b])) bs' = bs @ [b]
    by (rule Suc.IH[OF rest-out])
  have take (length bs) bs' =
    take (length bs) (take (length (bs @ [b])) bs')
    by simp
  also have ... = bs
    using prefix-suc by simp
  finally show ?case .
qed

lemma staged-trace-fri-program-challenge-length:
  assumes outcome:
    Some ((roots, bs'), t) ∈
      set-dist (execute (staged-trace-fri-program A i n bs) s)
  shows length bs' = length bs + n
  using outcome
proof (induction n arbitrary: i bs roots bs' s t)
  case 0
  then show ?case by (simp add: hash-ext-refl)
next
  case (Suc n)
  from Suc.prem1 obtain root b roots' s1 s2 s3 where
    rest-out:
      Some ((roots', bs'), t) ∈
        set-dist
          (execute
            (staged-trace-fri-program A (Suc i) n (bs @ [b])) s3)
    unfolding staged-trace-fri-program.simps
    by (auto elim!: set-dist-bindE)
  have length bs' = length (bs @ [b]) + n
    by (rule Suc.IH[OF rest-out])
  then show ?case by simp
qed

lemma staged-alpha-prefix-program-trace-challenge-length:
  assumes outcome:
    Some ((fr, trace-roots, trace-bs, trace-final), t) ∈
      set-dist (execute (staged-alpha-prefix-program A) s)
  shows length trace-bs = ceil-log clength
proof -
  from outcome obtain s1 s2 s3 where
    fri-out:
      Some ((trace-roots, trace-bs), s2) ∈
        set-dist
          (execute
            (staged-trace-fri-program A 0 (ceil-log clength) []) s1)
    unfolding staged-alpha-prefix-program-def

```

```

    by (auto elim!: set-dist-bindE)
  have length trace-bs = length ([] :: 'f list) + ceil-log clength
    by (rule staged-trace-fri-program-challenge-length[OF fri-out])
  then show ?thesis by simp
qed

lemma staged-alpha-prefix-trace-vector-member-imp-position-value:
  assumes outcome:
    Some ((fr, trace-roots, trace-bs, trace-final), t) ∈
      set-dist (execute (staged-alpha-prefix-program A) s)
    and subset:  $B \subseteq \text{fri-challenge-space } (\text{ceil-log clength})$ 
    and member:  $\text{trace-bs} \in B$ 
    and i-bound:  $i < \text{ceil-log clength}$ 
  shows  $\text{trace-bs} ! i \in \text{fri-vector-position-values } B i$ 
  by (rule fri-vector-member-imp-position-value
      [OF subset member i-bound])

lemma staged-alpha-prefix-bad-trace-vector-coordinate-cover-bound:
  assumes subset:  $B \subseteq \text{fri-challenge-space } (\text{ceil-log clength})$ 
    and nonempty:  $0 < \text{ceil-log clength}$ 
  shows
    wp-event (staged-alpha-prefix-program A)
      ( $\lambda \text{out. case out of None} \Rightarrow \text{False}$ 
        |  $\text{Some } ((-, -, \text{trace-bs}, -), -) \Rightarrow \text{trace-bs} \in B$ )  $s \leq$ 
      ( $\sum i < \text{ceil-log clength.}$ 
        wp-event (staged-alpha-prefix-program A)
          ( $\lambda \text{out. case out of None} \Rightarrow \text{False}$ 
            |  $\text{Some } ((-, -, \text{trace-bs}, -), -) \Rightarrow$ 
               $\text{trace-bs} ! i \in \text{fri-vector-position-values } B i$ )  $s$ )

proof -
  let ?bad =
     $\lambda \text{out. case out of None} \Rightarrow \text{False}$ 
    |  $\text{Some } ((-, -, \text{trace-bs}, -), -) \Rightarrow \text{trace-bs} \in B$ 
  let ?coord =
     $\lambda i \text{ out. case out of None} \Rightarrow \text{False}$ 
    |  $\text{Some } ((-, -, \text{trace-bs}, -), -) \Rightarrow$ 
       $\text{trace-bs} ! i \in \text{fri-vector-position-values } B i$ 
  have bad-to-coord:
    wp-event (staged-alpha-prefix-program A) ?bad  $s \leq$ 
    wp-event (staged-alpha-prefix-program A)
      ( $\lambda \text{out.} \exists i \in \{.. < \text{ceil-log clength}\}. ?coord i \text{ out}$ )  $s$ 
  proof (rule wp-event-mono)
    fix out
    assume bad: ?bad out
    show  $\exists i \in \{.. < \text{ceil-log clength}\}. ?coord i \text{ out}$ 
  proof (cases out)
    case None
    then show ?thesis using bad by simp
  next

```

```

case (Some result)
then obtain fr trace-roots trace-bs trace-final t where
  out-eq:
    out = Some ((fr, trace-roots, trace-bs, trace-final), t)
  by (cases result, auto split: prod.splits)
have member: trace-bs ∈ B
  using bad unfolding out-eq by simp
from fri-vector-member-coordinate-cover[OF subset member nonempty]
obtain i where i-bound: i < ceil-log clength
  and coord: trace-bs ! i ∈ fri-vector-position-values B i
  by blast
show ?thesis
  by (rule bexI[of - i]) (use i-bound coord out-eq in simp-all)
qed
qed
also have ... ≤
  ( $\sum i < \text{ceil-log clength}$ .
    wp-event (staged-alpha-prefix-program A) (?coord i) s)
  by (rule wp-event-finite-UN-bound) simp-all
finally show ?thesis .
qed

lemma staged-alpha-prefix-trace-coordinate-hit-imp-new-output-hit:
assumes wf: staged-budget-wellformed budgets
and controlled: staged-adversary-controlled budgets A
and i-bound: i < ceil-log clength
and absent:
  trace-fri-challenge-values-absent
  (fri-vector-position-values B i) s
and outcome:
  Some ((fr, trace-roots, trace-bs, trace-final), t) ∈
  set-dist (execute (staged-alpha-prefix-program A) s)
and hit: trace-bs ! i ∈ fri-vector-position-values B i
shows
  hash-new-output-hit-event (fri-vector-position-values B i) s
  (Some ((fr, trace-roots, trace-bs, trace-final), t))
proof –
from outcome obtain s1 s2 s3 s4 where
  root-out:
    Some (fr, s1) ∈ set-dist (execute (trace-root-stage A) s)
and record-root:
    Some ((), s2) ∈ set-dist (execute (record-staged-message fr) s1)
and fri-out:
    Some ((trace-roots, trace-bs), s3) ∈
    set-dist
    (execute
      (staged-trace-fri-program A 0 (ceil-log clength) []) s2)
and final-out:
    Some (trace-final, s4) ∈

```

```

    set-dist (execute (trace-final-stage A trace-bs) s3)
and record-final:
    Some ((), t) ∈
    set-dist (execute (record-staged-message trace-final) s4)
unfolding staged-alpha-prefix-program-def
by (auto elim!: set-dist-bindE)
have root-controlled:
    controlled-ro-program (trace-root-budget budgets)
    (trace-root-stage A)
using controlled unfolding staged-adversary-controlled-def by simp
have final-controlled:
    controlled-ro-program (trace-final-budget budgets)
    (trace-final-stage A trace-bs)
using controlled unfolding staged-adversary-controlled-def by blast
have ext-s-s1:  $s \leq s1$ 
using controlled-ro-program-extension[OF root-controlled] root-out
unfolding hash-extension-preserving-def by blast
have s2-eq:
    s2 = s1 (
      PState := concat (PState s1) fr,
      PTranscript := PTranscript s1 @ [fr])
by (rule record-staged-message-outcome[OF record-root])
have ext-s1-s2:  $s1 \leq s2$ 
unfolding s2-eq less-eq-hash-ext-def less-eq-fmap-def by simp
have trace-bound:
     $0 + \text{ceil-log } \text{clength} \leq \text{length } (\text{trace-fri-budgets } \text{budgets})$ 
using wf unfolding staged-budget-wellformed-def by simp
have trace-alignment:
    length trace-roots = ceil-log clength ∧
    length trace-bs = length ([] :: 'f list) + ceil-log clength ∧
    PTranscript s3 = PTranscript s2 @ trace-roots ∧
    PState s3 = foldl concat (PState s2) trace-roots ∧
    s2 ≤ s3 ∧
    PTraceFriCounter s3 = PTraceFriCounter s2 + ceil-log clength ∧
    PCompositionFriCounter s3 = PCompositionFriCounter s2 ∧
    PAlphaCounter s3 = PAlphaCounter s2 ∧
    PQueryCounter s3 = PQueryCounter s2 ∧
    (∀ j < ceil-log clength.
      fmllookup (HashMap s3)
        (TraceFriChallenge (PTraceFriCounter s2 + j)
          (foldl concat (PState s2) (take (Suc j) trace-roots))) =
        Some (trace-bs ! (length ([] :: 'f list) + j)))
by (rule staged-trace-fri-program-alignment
    [OF controlled trace-bound fri-out])
let ?key =
    TraceFriChallenge (PTraceFriCounter s2 + i)
    (foldl concat (PState s2) (take (Suc i) trace-roots))
have lookup-s3:
    fmllookup (HashMap s3) ?key = Some (trace-bs ! i)

```

```

    using trace-alignment i-bound by simp
have ext-s-s3:  $s \leq s3$ 
  by (rule hash-ext-trans[OF ext-s-s1])
    (rule hash-ext-trans[OF ext-s1-s2], use trace-alignment in simp)
have ext-s3-s4:  $s3 \leq s4$ 
  using controlled-ro-program-extension[OF final-controlled] final-out
  unfolding hash-extension-preserving-def by blast
have t-eq:
   $t = s4 \langle$ 
    PState := concat (PState  $s4$ ) trace-final,
    PTranscript := PTranscript  $s4$  @ [trace-final] $\rangle$ 
  by (rule record-staged-message-outcome[OF record-final])
have ext-s4-t:  $s4 \leq t$ 
  unfolding t-eq less-eq-hash-ext-def less-eq-fmap-def by simp
have ext-s3-t:  $s3 \leq t$ 
  by (rule hash-ext-trans[OF ext-s3-s4 ext-s4-t])
have lookup-t:
  fmlookup (HashMap  $t$ ) ?key = Some (trace-bs !  $i$ )
  by (rule hash-extension-lookup[OF lookup-s3 ext-s3-t])
have none-s: fmlookup (HashMap  $s$ ) ?key = None
proof (cases fmlookup (HashMap  $s$ ) ?key)
  case None
  then show ?thesis .
next
  case (Some old)
  have lookup-s3-old: fmlookup (HashMap  $s3$ ) ?key = Some old
    by (rule hash-extension-lookup[OF Some ext-s-s3])
  then have old-eq:  $old = trace-bs ! i$ 
    using lookup-s3 by simp
  have old  $\notin$  fri-vector-position-values  $B$   $i$ 
    using absent Some
    unfolding trace-fri-challenge-values-absent-def by blast
  then show ?thesis
    using old-eq hit by simp
qed
have hash-map-new-output-hit (fri-vector-position-values  $B$   $i$ )  $s$   $t$ 
  unfolding hash-map-new-output-hit-def
  using none-s lookup-t hit by blast
then show ?thesis
  unfolding hash-new-output-hit-event-def by simp
qed

lemma staged-alpha-prefix-trace-coordinate-bound:
  assumes wf: staged-budget-wellformed budgets
  and controlled: staged-adversary-controlled budgets  $A$ 
  and i-bound:  $i < \text{ceil-log } \text{clength}$ 
  and absent:
    trace-fri-challenge-values-absent
    (fri-vector-position-values  $B$   $i$ )  $s$ 

```


shows

wp-event (staged-alpha-prefix-program A)
(λout. case out of None ⇒ False
| Some ((-, -, trace-bs, -), -) ⇒
trace-bs ! i ∈ fri-vector-position-values B i) s ≤
staged-phase-target-error (fri-vector-position-values B i)
(staged-alpha-search-queries budgets 0)

proof –

let ?P =

λout. case out of None ⇒ False
| Some ((-, -, trace-bs, -), -) ⇒
trace-bs ! i ∈ fri-vector-position-values B i

let ?H =

hash-new-output-hit-event (fri-vector-position-values B i) s

have *event-imp-hit*:

wp-event (staged-alpha-prefix-program A) ?P s ≤
wp-event (staged-alpha-prefix-program A) ?H s

proof (rule *wp-event-mono-on-support*)

fix *out*

assume *support*:

out ∈ set-dist (execute (staged-alpha-prefix-program A) s)
and *event*: ?P *out*

show ?H *out*

proof (*cases out*)

case *None*

then show ?thesis **using** *event by simp*

next

case (*Some result*)

then obtain *fr trace-roots trace-bs trace-final t* **where**

out-eq:

out = Some ((fr, trace-roots, trace-bs, trace-final), t)

by (*cases result, auto split: prod.splits*)

have *outcome*:

Some ((fr, trace-roots, trace-bs, trace-final), t) ∈
set-dist (execute (staged-alpha-prefix-program A) s)

using *support unfolding out-eq* .

have *hit*: *trace-bs ! i ∈ fri-vector-position-values B i*

using *event unfolding out-eq by simp*

show ?thesis

unfolding *out-eq*

by (rule

staged-alpha-prefix-trace-coordinate-hit-imp-new-output-hit
[OF wf controlled i-bound absent outcome hit])

qed

qed

also have

wp-event (staged-alpha-prefix-program A) ?H s ≤
staged-phase-target-error (fri-vector-position-values B i)
(staged-alpha-search-queries budgets 0)

by (rule staged-alpha-prefix-target-hit-bound[OF wf controlled])
 finally show ?thesis .
 qed

lemma staged-trace-fri-challenge-prefix-program-length:

assumes outcome:

Some ((fr, trace-bs, rt), t) ∈

set-dist

(execute (staged-trace-fri-challenge-prefix-program A i) s)

shows length trace-bs = i

proof –

from outcome **obtain** roots pre-state post-state **where**

fri-out:

Some ((roots, trace-bs), post-state) ∈

set-dist

(execute (staged-trace-fri-program A 0 i []) pre-state)

unfolding staged-trace-fri-challenge-prefix-program-def

by (auto elim!: set-dist-bindE)

have length trace-bs = length ([] :: 'f list) + i

by (rule staged-trace-fri-program-challenge-length[OF fri-out])

then show ?thesis **by** simp

qed

lemma staged-trace-fri-prefix-vector-member-imp-position-value:

assumes outcome:

Some ((fr, trace-bs, rt), t) ∈

set-dist

(execute (staged-trace-fri-challenge-prefix-program A i) s)

and subset: $B \subseteq \text{fri-challenge-space } i$

and member: trace-bs ∈ B

and j-bound: $j < i$

shows trace-bs ! j ∈ fri-vector-position-values B j

by (rule fri-vector-member-imp-position-value

[OF subset member j-bound])

lemma staged-trace-fri-prefix-head-challenge-imp-position-value:

assumes outcome:

Some (((fr, trace-bs, rt), b), t) ∈

set-dist

(execute

(staged-trace-fri-challenge-prefix-program A i $\gg$

(λ prefix. receive-trace-fri-challenge $\gg$

(λb . return (prefix, b)))) s)

and member: trace-bs @ [b] ∈ B

shows b ∈ fri-vector-position-values B i

proof –

from outcome **obtain** s1 **where**

prefix-out:

Some ((fr, trace-bs, rt), s1) ∈

```

      set-dist
      (execute (staged-trace-fri-challenge-prefix-program A i) s)
    by (auto elim!: set-dist-bindE)
  have len: length trace-bs = i
  by (rule staged-trace-fri-challenge-prefix-program-length
      [OF prefix-out])
  show ?thesis
  by (rule fri-vector-extend-member-imp-position-value
      [OF member len])
qed

```

lemma *staged-alpha-program-alignment*:

assumes *outcome*:

Some (as, t) ∈ set-dist (execute (staged-alpha-program n) s)

shows

length as = n ∧

PTranscript t = PTranscript s @ as ∧

PState t = foldl concat (PState s) as ∧

PTraceFriCounter t = PTraceFriCounter s ∧

PCompositionFriCounter t = PCompositionFriCounter s ∧

PAlphaCounter t = PAlphaCounter s + n ∧

PQueryCounter t = PQueryCounter s

using *outcome*

proof (*induction n arbitrary: as s t*)

case 0

then show ?*case* **by** *simp*

next

case (*Suc n*)

from *staged-alpha-program-Suc-outcomeE*[*OF Suc.premis*]

obtain *a as' s1 s2* **where**

challenge-out:

Some (a, s1) ∈

set-dist (execute receive-alpha-challenge s)

and *record-out*:

Some ((), s2) ∈

set-dist (execute (record-staged-message a) s1)

and *rest-out*:

Some (as', t) ∈

set-dist (execute (staged-alpha-program n) s2)

and *as-eq*: *as = a # as'*

.

have *challenge-fields*:

PState s1 = PState s ∧

PTranscript s1 = PTranscript s

using *receive-alpha-challenge-outcome*[*OF challenge-out*] **by** *simp*

have *challenge-counters*:

PTraceFriCounter s1 = PTraceFriCounter s ∧

PCompositionFriCounter s1 = PCompositionFriCounter s ∧

PAlphaCounter s1 = Suc (PAlphaCounter s) ∧

```

    PQueryCounter s1 = PQueryCounter s
using receive-alpha-challenge-counter-outcome[OF challenge-out] by simp
have s2:
  s2 = s1(|
    PState := concat (PState s1) a,
    PTranscript := PTranscript s1 @ [a])
  by (rule record-staged-message-outcome[OF record-out])
have s2-fields:
  PState s2 = concat (PState s) a ∧
  PTranscript s2 = PTranscript s @ [a] ∧
  PTraceFriCounter s2 = PTraceFriCounter s ∧
  PCompositionFriCounter s2 = PCompositionFriCounter s ∧
  PAlphaCounter s2 = Suc (PAlphaCounter s) ∧
  PQueryCounter s2 = PQueryCounter s
  unfolding s2 using challenge-fields challenge-counters by simp
have rest-alignment:
  length as' = n ∧
  PTranscript t = PTranscript s2 @ as' ∧
  PState t = foldl concat (PState s2) as' ∧
  PTraceFriCounter t = PTraceFriCounter s2 ∧
  PCompositionFriCounter t = PCompositionFriCounter s2 ∧
  PAlphaCounter t = PAlphaCounter s2 + n ∧
  PQueryCounter t = PQueryCounter s2
  by (rule Suc.IH[OF rest-out])
show ?case
  unfolding as-eq
  using s2-fields rest-alignment
  by simp
qed

```

lemma staged-receive-alpha-challenge-preserves-alpha-future-fresh:

assumes future: alpha-future-fresh s

and outcome: Some (a, t) ∈ set-dist (execute receive-alpha-challenge s)

shows alpha-future-fresh t

proof –

have counter-t: PAlphaCounter t = Suc (PAlphaCounter s)

using receive-alpha-challenge-counter-outcome[OF outcome] **by** simp

show ?thesis

unfolding alpha-future-fresh-def

proof (intro allI impI)

fix i x

assume i-future: PAlphaCounter t ≤ i

have i-ge-s: PAlphaCounter s ≤ i

using counter-t i-future **by** simp

have neg:

AlphaChallenge i x ≠

AlphaChallenge (PAlphaCounter s) (PState s)

using counter-t i-future **by** auto

have fmllookup (HashMap t) (AlphaChallenge i x) =

```

      fmlookup (HashMap s) (AlphaChallenge i x)
    by (rule receive-alpha-challenge-preserves-other-lookup
        [OF outcome neg])
    also have ... = None
      using future i-ge-s unfolding alpha-future-fresh-def by blast
    finally show fmlookup (HashMap t) (AlphaChallenge i x) = None .
  qed
qed

```

```

lemma record-staged-message-preserves-alpha-future-fresh:
  assumes future: alpha-future-fresh s
  and outcome:
    Some ((), t) ∈ set-dist (execute (record-staged-message x) s)
  shows alpha-future-fresh t
proof -
  have t-eq:
    t = s⟦
      PState := concat (PState s) x,
      PTranscript := PTranscript s @ [x]⟧
  by (rule record-staged-message-outcome[OF outcome])
  show ?thesis
    using future unfolding t-eq alpha-future-fresh-def by simp
qed

```

```

lemma staged-alpha-program-preserves-alpha-future-fresh:
  assumes future: alpha-future-fresh s
  and outcome:
    Some (as, t) ∈ set-dist (execute (staged-alpha-program n) s)
  shows alpha-future-fresh t
  using future outcome
proof (induction n arbitrary: as s t)
  case 0
  then show ?case by simp
next
  case (Suc n)
  show ?case
  proof (cases rule:
    staged-alpha-program-Suc-outcomeE[OF Suc.prem(2)])
  case (1 a as' s1 s2)
  have future-s1:
    alpha-future-fresh s1
  by (rule staged-receive-alpha-challenge-preserves-alpha-future-fresh
      [OF Suc.prem(1) 1(1)])
  have future-s2:
    alpha-future-fresh s2
  by (rule record-staged-message-preserves-alpha-future-fresh
      [OF future-s1 1(2)])
  show ?thesis
    by (rule Suc.IH[OF future-s2 1(3)])

```

```

qed
qed

lemma wp-staged-alpha-program-exact-challenges-bound:
  assumes len: length as = n
    and future: alpha-future-fresh s
  shows
    wp-event (staged-alpha-program n)
      (λout. case out of None ⇒ False
        | Some (as', -) ⇒ as' = as) s ≤
      (1 / nnreal size) ^ n
  using len future
proof (induction n arbitrary: as s)
  case 0
  then have as-empty: as = []
    by simp
  show ?case
    unfolding as-empty by (simp add: wp-event-def wpsimps)
next
  case (Suc n)
  obtain a as-tail where as-eq: as = a # as-tail
    using Suc.prem1 by (cases as) auto
  have len-tail: length as-tail = n
    using Suc.prem1 as-eq by simp
  let ?tail =
    staged-alpha-program n ::
      ('f list, 'f protocol-channel) state-monad
  let ?Q =
    λout :: ('f list ×
      'f protocol-channel) option.
      case out of None ⇒ False | Some (as', -) ⇒ as' = as
  let ?Head =
    λout :: ('f ×
      'f protocol-channel) option.
      case out of None ⇒ False | Some (a0, -) ⇒ a0 ∈ {a}
  have head-bound:
    wp-event receive-alpha-challenge ?Head s ≤ 1 / nnreal size
  proof -
    have current-fresh:
      fmlookup (HashMap s)
        (AlphaChallenge (PAlphaCounter s) (PState s)) = None
      by (rule alpha-future-fresh-current[OF Suc.prem2])
    have wp-event receive-alpha-challenge ?Head s =
      nnreal (card ({a} :: 'f set)) / nnreal size
      by (rule wp-receive-alpha-challenge-fresh-set[OF current-fresh])
    then show ?thesis by simp
  qed
  have bind-bound:
    wp-event

```

```

    (receive-alpha-challenge  $\gg$ 
      ( $\lambda a0$ . record-staged-message  $a0$   $\gg$ 
        ( $\lambda$ -. ?tail  $\gg$ 
          ( $\lambda as'$ . return ( $a0$  #  $as'$ ))))))
    ?Q s  $\leq$ 
    wp-event receive-alpha-challenge ?Head s *
    (1 / nreal size)  $\wedge$  n
proof (rule wp-event-bind-bound-by-head-and-cont[where P = ?Head])
show  $\neg$  ?Q None
by simp
next
fix a0 t
assume recv:
  Some (a0, t)  $\in$  set-dist (execute receive-alpha-challenge s)
and not-head:  $\neg$  ?Head (Some (a0, t))
have a0-ne: a0  $\neq$  a
using not-head by simp
have cont-false:
  wp-event
    (record-staged-message a0  $\gg$ 
      ( $\lambda$ -. ?tail  $\gg$  ( $\lambda as'$ . return ( $a0$  #  $as'$ ))))
  ?Q t  $\leq$ 
  wp-event
    (record-staged-message a0  $\gg$ 
      ( $\lambda$ -. ?tail  $\gg$  ( $\lambda as'$ . return ( $a0$  #  $as'$ ))))
    ( $\lambda$ -. ('f list  $\times$ 
      'f protocol-channel) option. False) t
by (rule wp-event-mono-on-support)
    (use a0-ne as-eq in
       $\langle$ auto simp: wpsimps elim!: set-dist-bindE
        split: option.splits prod.splits $\rangle$ )
also have ... = 0
unfolding wp-event-def wp-def dist-expect-def by simp
finally have le-zero:
  wp-event
    (record-staged-message a0  $\gg$ 
      ( $\lambda$ -. ?tail  $\gg$  ( $\lambda as'$ . return ( $a0$  #  $as'$ ))))
  ?Q t  $\leq$  0 .
show
  wp-event
    (record-staged-message a0  $\gg$ 
      ( $\lambda$ -. ?tail  $\gg$  ( $\lambda as'$ . return ( $a0$  #  $as'$ ))))
  ?Q t = 0
by (rule antisym[OF le-zero]) simp
next
fix a0 t
assume recv:
  Some (a0, t)  $\in$  set-dist (execute receive-alpha-challenge s)
and head: ?Head (Some (a0, t))

```

```

have a0-eq: a0 = a
  using head by simp
have future-t: alpha-future-fresh t
  by (rule staged-receive-alpha-challenge-preserves-alpha-future-fresh
    [OF Suc.premis(2) recv])
have record-cont-bound:
  wp-event
    (record-staged-message a0  $\gg$ 
      ( $\lambda$ -. ?tail  $\gg$  ( $\lambda$ as'. return (a0 # as'))))
    ?Q t  $\leq$  (1 / nnreal size) ^ n
proof (rule wp-event-bind-bound-by-cont)
  show  $\neg$  ?Q None
    by simp
next
  fix uu u
  assume record-out:
    Some (uu, u)  $\in$ 
      set-dist (execute (record-staged-message a0) t)
  have record-unit:
    Some ((), u)  $\in$ 
      set-dist (execute (record-staged-message a0) t)
    using record-out by (cases uu) simp
  have future-u: alpha-future-fresh u
    by (rule record-staged-message-preserves-alpha-future-fresh
      [OF future-t record-unit])
  have tail-bound:
    wp-event ?tail
      ( $\lambda$ out. case out of None  $\Rightarrow$  False
        | Some (as', -)  $\Rightarrow$  as' = as-tail) u
       $\leq$  (1 / nnreal size) ^ n
    by (rule Suc.IH[OF len-tail future-u])
  have cont-eq:
    wp-event
      (?tail  $\gg$  ( $\lambda$ as'. return (a0 # as'))) ?Q u =
    wp-event ?tail
      ( $\lambda$ out. case out of None  $\Rightarrow$  False
        | Some (as', -)  $\Rightarrow$  as' = as-tail) u
    by (simp add: wp-event-bind-return-map a0-eq as-eq
      split: option.splits prod.splits)
  show
    wp-event (?tail  $\gg$  ( $\lambda$ as'. return (a0 # as'))) ?Q u
       $\leq$  (1 / nnreal size) ^ n
    unfolding cont-eq by (rule tail-bound)
qed
show
  wp-event
    (record-staged-message a0  $\gg$ 
      ( $\lambda$ -. ?tail  $\gg$  ( $\lambda$ as'. return (a0 # as'))))
    ?Q t  $\leq$  (1 / nnreal size) ^ n

```



```

    by (rule record-cont-bound)
  qed
  have wp-event (staged-alpha-program (Suc n)) ?Q s
    ≤ wp-event receive-alpha-challenge ?Head s *
      (1 / nnreal size) ^ n
    unfolding staged-alpha-program.simps by (rule bind-bound)
  also have ... ≤ (1 / nnreal size) * (1 / nnreal size) ^ n
    by (rule mult-right-mono[OF head-bound]) simp
  also have ... = (1 / nnreal size) ^ Suc n
    by simp
  finally show ?case .
  qed

```

lemma *wp-staged-alpha-program-finite-set-bound:*

assumes *future:* alpha-future-fresh *s*
and *finite-B:* finite *B*
and *lengths:* $\bigwedge as. as \in B \implies \text{length } as = n$
shows

wp-event (staged-alpha-program n)
 $(\lambda out. \text{case out of None} \Rightarrow \text{False}$
 $\quad | \text{Some } (as, -) \Rightarrow as \in B) s \leq$
 $nnreal (\text{card } B) / (nnreal \text{ size}) ^ n$

proof –

let ?P =

$\lambda as \text{ out. case out of None} \Rightarrow \text{False}$
 $\quad | \text{Some } (as', -) \Rightarrow as' = as$

have *event-mono:*

wp-event (staged-alpha-program n)
 $(\lambda out. \text{case out of None} \Rightarrow \text{False}$
 $\quad | \text{Some } (as, -) \Rightarrow as \in B) s \leq$
wp-event (staged-alpha-program n)
 $(\lambda out. \exists as \in B. ?P as \text{ out}) s$
by (rule wp-event-mono) (auto split: option.splits prod.splits)

also have ... ≤ $(\sum as \in B. (1 / nnreal \text{ size}) ^ n)$

by (rule wp-event-finite-UN-bound[OF finite-B])
 (rule wp-staged-alpha-program-exact-challenges-bound
 [OF lengths future])

also have ... = $nnreal (\text{card } B) * ((1 / nnreal \text{ size}) ^ n)$

using *finite-B* **by** *simp*

also have ... = $nnreal (\text{card } B) / (nnreal \text{ size}) ^ n$

using *size-card* **by** *transfer (simp add: power-divide)*

finally show ?thesis .

qed

lemma *wp-staged-alpha-program-alpha-space-set-bound:*

assumes *future:* alpha-future-fresh *s*

and *subset:* $B \subseteq \text{alpha-space}$

shows

wp-event (staged-alpha-program (length spec))

```

    (λout. case out of None ⇒ False
      | Some (as, -) ⇒ as ∈ B) s ≤
    nnreal (card B) / nnreal (CARD('f) ^ length spec)
proof -
  have finite-B: finite B
    by (rule finite-subset[OF subset])
      (simp add: alpha-space-def finite-length-lists-UNIV)
  have lengths:  $\bigwedge as. as \in B \implies \text{length } as = \text{length spec}$ 
    using subset unfolding alpha-space-def by auto
  have wp-event (staged-alpha-program (length spec))
    (λout. case out of None ⇒ False
      | Some (as, -) ⇒ as ∈ B) s ≤
    nnreal (card B) / (nnreal size) ^ length spec
    by (rule wp-staged-alpha-program-finite-set-bound
      [OF future finite-B lengths])
  also have ... =
    nnreal (card B) / nnreal (CARD('f) ^ length spec)
    using size-card by simp
  finally show ?thesis .
qed

lemma staged-after-alpha-prefix-program-alpha-space-list-bound-from-fresh:
  assumes future: alpha-future-fresh s
  and subset:  $B \subseteq \text{alpha-space}$ 
  shows
    wp-event
      (staged-after-alpha-prefix-program A
        (fr, trace-roots, trace-bs, trace-final))
      (staged-transcript-alpha-list-hit B) s ≤
      nnreal (card B) / nnreal (CARD('f) ^ length spec)
proof -
  have alpha-bound:
    wp-event (staged-alpha-program (length spec))
      (λout. case out of None ⇒ False
        | Some (as, -) ⇒ as ∈ B) s ≤
      nnreal (card B) / nnreal (CARD('f) ^ length spec)
    by (rule wp-staged-alpha-program-alpha-space-set-bound
      [OF future subset])
  show ?thesis
    by (rule staged-after-alpha-prefix-program-alpha-list-bound-from-alpha-program
      [OF alpha-bound])
qed

lemma staged-transcript-program-alpha-space-list-bound-from-fresh-prefix:
  assumes prefix-future:
     $\bigwedge \text{prefix } t.$ 
    Some (prefix, t) ∈
      set-dist (execute (staged-alpha-prefix-program A)
        adversary-initial-state)  $\implies$ 

```

```

    alpha-future-fresh t
  and subset:  $B \subseteq \text{alpha-space}$ 
shows
  wp-event (staged-transcript-program A)
    (staged-transcript-alpha-list-hit B) adversary-initial-state  $\leq$ 
    nnreal (card B) / nnreal ( $\text{CARD}(f) \wedge \text{length spec}$ )
  unfolding staged-transcript-program-alpha-prefix-decomp
proof (rule wp-event-bind-bound-by-cont)
  show  $\neg \text{staged-transcript-alpha-list-hit B None}$ 
    unfolding staged-transcript-alpha-list-hit-def by simp
next
  fix prefix t
  assume prefix-out:
    Some (prefix, t)  $\in$ 
    set-dist (execute (staged-alpha-prefix-program A)
      adversary-initial-state)
  obtain fr trace-roots trace-bs trace-final
    where prefix-eq: prefix = (fr, trace-roots, trace-bs, trace-final)
    by (cases prefix) auto
  have future-t: alpha-future-fresh t
    by (rule prefix-future[OF prefix-out])
  show
    wp-event (staged-after-alpha-prefix-program A prefix)
      (staged-transcript-alpha-list-hit B) t  $\leq$ 
      nnreal (card B) / nnreal ( $\text{CARD}(f) \wedge \text{length spec}$ )
    unfolding prefix-eq
    by (rule staged-after-alpha-prefix-program-alpha-space-list-bound-from-fresh
      [OF future-t subset])
qed

```

lemma staged-composition-fri-program-challenge-prefix-pre-alignment:

```

  assumes outcome:
    Some ((roots, bs'), t)  $\in$ 
    set-dist
      (execute (staged-composition-fri-program A dg i n bs) s)
  shows take (length bs) bs' = bs
  using outcome
proof (induction n arbitrary: i bs roots bs' s t)
  case 0
  then show ?case by simp
next
  case (Suc n)
  from Suc.premis obtain root b roots' s1 s2 s3 where
    rest-out:
      Some ((roots', bs'), t)  $\in$ 
      set-dist
        (execute
          (staged-composition-fri-program
            A dg (Suc i) n (bs @ [b])) s3)

```

```

unfolding staged-composition-fri-program.simps
by (auto elim!: set-dist-bindE)
have prefix-suc:
  take (length (bs @ [b])) bs' = bs @ [b]
  by (rule Suc.IH[OF rest-out])
have take (length bs) bs' =
  take (length bs) (take (length (bs @ [b])) bs')
  by simp
also have ... = bs
  using prefix-suc by simp
finally show ?case .
qed

lemma staged-composition-fri-program-alignment:
assumes controlled: staged-adversary-controlled budgets A
and bound:
   $i + n \leq \text{length } (\text{composition-fri-budgets } \text{budgets})$ 
and outcome:
  Some ((roots, bs'), t) ∈
    set-dist
    (execute
      (staged-composition-fri-program A dg i n bs) s)
shows
  length roots = n ∧
  length bs' = length bs + n ∧
  PTranscript t = PTranscript s @ roots ∧
  PState t = foldl concat (PState s) roots ∧
  s ≤ t ∧
  PTraceFriCounter t = PTraceFriCounter s ∧
  PCompositionFriCounter t = PCompositionFriCounter s + n ∧
  PAlphaCounter t = PAlphaCounter s ∧
  PQueryCounter t = PQueryCounter s ∧
  (∀ j < n.
    fmllookup (HashMap t)
      (CompositionFriChallenge (PCompositionFriCounter s + j)
        (foldl concat (PState s) (take (Suc j) roots))) =
      Some (bs' ! (length bs + j)))
  using bound outcome
proof (induction n arbitrary: i bs roots bs' s t)
case 0
  then show ?case by (simp add: hash-ext-refl)
next
case (Suc n)
from staged-composition-fri-program-Suc-outcomeE[OF Suc.prem1(2)]
obtain root b roots' s1 s2 s3 where
  stage-out:
  Some (root, s1) ∈
    set-dist
    (execute (composition-fri-root-stage A dg i bs) s)

```

and *record-out*:
 Some $((\text{ }, s2) \in$
 set-dist (*execute* (*record-staged-message* *root*) *s1*)
and *challenge-out*:
 Some $(b, s3) \in$
 set-dist (*execute* *receive-composition-fri-challenge* *s2*)
and *rest-out*:
 Some $((\text{roots}', bs'), t) \in$
 set-dist
 (*execute*
 (*staged-composition-fri-program*
 A dg (Suc i) n (bs @ [b]) s3)
and *roots-eq*: *roots* = *root* # *roots'*
 .
have *i-bound*: $i < \text{length } (\text{composition-fri-budgets } \text{budgets})$
 using *Suc.prem*(1) **by** *simp*
have *stage-controlled*:
 controlled-ro-program (*composition-fri-budgets* *budgets* ! *i*)
 (*composition-fri-root-stage* *A dg i bs*)
 using *controlled i-bound*
 unfolding *staged-adversary-controlled-def* **by** *blast*
have *stage-fields*:
 PState *s1* = *PState* *s* $\wedge$
 PTranscript *s1* = *PTranscript* *s* $\wedge$
 PTraceFriCounter *s1* = *PTraceFriCounter* *s* $\wedge$
 PCompositionFriCounter *s1* = *PCompositionFriCounter* *s* $\wedge$
 PAlphaCounter *s1* = *PAlphaCounter* *s* $\wedge$
 PQueryCounter *s1* = *PQueryCounter* *s*
 using *controlled-stage-outcome-fields*[*OF stage-controlled stage-out*]
 by *simp*
have *ext-s-s1*: $s \leq s1$
 using *controlled-ro-program-extension*[*OF stage-controlled*] *stage-out*
 unfolding *hash-extension-preserving-def* **by** *blast*
have *s2*:
 s2 = *s1* ($\parallel$
 PState := *concat* (*PState* *s1*) *root*,
 PTranscript := *PTranscript* *s1* @ [*root*])
 by (*rule record-staged-message-outcome*[*OF record-out*])
have *ext-s1-s2*: $s1 \leq s2$
 unfolding *s2 less-eq-hash-ext-def less-eq-fmap-def* **by** *simp*
have *s2-fields*:
 PState *s2* = *concat* (*PState* *s*) *root* $\wedge$
 PTranscript *s2* = *PTranscript* *s* @ [*root*] $\wedge$
 PTraceFriCounter *s2* = *PTraceFriCounter* *s* $\wedge$
 PCompositionFriCounter *s2* = *PCompositionFriCounter* *s* $\wedge$
 PAlphaCounter *s2* = *PAlphaCounter* *s* $\wedge$
 PQueryCounter *s2* = *PQueryCounter* *s*
 unfolding *s2* **using** *stage-fields* **by** *simp*
have *challenge-fields*:

```

PState s3 = PState s2 ∧
PTranscript s3 = PTranscript s2
using receive-composition-fri-challenge-outcome[OF challenge-out]
by simp
have ext-s2-s3: s2 ≤ s3
using receive-composition-fri-challenge-outcome[OF challenge-out]
by simp
have challenge-counters:
PTraceFriCounter s3 = PTraceFriCounter s2 ∧
PCompositionFriCounter s3 =
  Suc (PCompositionFriCounter s2) ∧
PAlphaCounter s3 = PAlphaCounter s2 ∧
PQueryCounter s3 = PQueryCounter s2
using
  receive-composition-fri-challenge-counter-outcome[OF challenge-out]
by simp
have rest-bound:
  Suc i + n ≤ length (composition-fri-budgets budgets)
using Suc.premis(1) by simp
have rest-alignment:
length roots' = n ∧
length bs' = length (bs @ [b]) + n ∧
PTranscript t = PTranscript s3 @ roots' ∧
PState t = foldl concat (PState s3) roots' ∧
s3 ≤ t ∧
PTraceFriCounter t = PTraceFriCounter s3 ∧
PCompositionFriCounter t = PCompositionFriCounter s3 + n ∧
PAlphaCounter t = PAlphaCounter s3 ∧
PQueryCounter t = PQueryCounter s3 ∧
(∀ j < n.
  fmllookup (HashMap t)
    (CompositionFriChallenge (PCompositionFriCounter s3 + j)
      (foldl concat (PState s3) (take (Suc j) roots')))) =
    Some (bs' ! (length (bs @ [b]) + j)))
by (rule Suc.IH[OF rest-bound rest-out])
have ext-s3-t: s3 ≤ t
using rest-alignment by simp
have ext-s-t: s ≤ t
by (rule hash-ext-trans[OF ext-s-s1])
  (rule hash-ext-trans[OF ext-s1-s2],
   rule hash-ext-trans[OF ext-s2-s3 ext-s3-t])
have challenge-lookup-s3:
fmllookup (HashMap s3)
  (CompositionFriChallenge (PCompositionFriCounter s)
    (concat (PState s) root)) = Some b
using receive-composition-fri-challenge-outcome[OF challenge-out]
  s2-fields by simp
have challenge-lookup-t:
fmllookup (HashMap t)

```

```

    (CompositionFriChallenge (PCompositionFriCounter s)
      (concat (PState s) root)) = Some b
  by (rule hash-extension-lookup[OF challenge-lookup-s3 ext-s3-t])
have prefix-suc:
  take (length (bs @ [b])) bs' = bs @ [b]
  by (rule staged-composition-fri-program-challenge-prefix-pre-alignment
    [OF rest-out])
have lookup-all:
  ∀ j < Suc n.
    fmlookup (HashMap t)
      (CompositionFriChallenge (PCompositionFriCounter s + j)
        (foldl concat (PState s) (take (Suc j) (root # roots')))) =
      Some (bs' ! (length bs + j))
proof (intro allI impI)
  fix j
  assume j-bound: j < Suc n
  show
    fmlookup (HashMap t)
      (CompositionFriChallenge (PCompositionFriCounter s + j)
        (foldl concat (PState s) (take (Suc j) (root # roots')))) =
      Some (bs' ! (length bs + j))
proof (cases j)
  case 0
  have bs' ! length bs = b
  proof -
    have idx-lt: length bs < length (bs @ [b])
    by simp
    have bs' ! length bs =
      take (length (bs @ [b])) bs' ! length bs
    using idx-lt by simp
    also have ... = (bs @ [b]) ! length bs
    using prefix-suc by simp
    also have ... = b
    by simp
    finally show ?thesis .
  qed
  then show ?thesis
    using challenge-lookup-t unfolding 0 by simp
next
  case (Suc k)
  then have k-bound: k < n
    using j-bound by simp
  show ?thesis
    using rest-alignment k-bound s2-fields challenge-fields
      challenge-counters
    unfolding Suc
    by simp
qed
qed

```

```

show ?case
  unfolding roots-eq
  using s2-fields challenge-fields challenge-counters rest-alignment
    ext-s-t lookup-all
  by simp
qed

```

lemma *staged-composition-fri-program-challenge-prefix:*

```

assumes outcome:
  Some ((roots, bs'), t) ∈
    set-dist
      (execute (staged-composition-fri-program A dg i n bs) s)
shows take (length bs) bs' = bs
using outcome
proof (induction n arbitrary: i bs roots bs' s t)
  case 0
  then show ?case by simp
next
  case (Suc n)
  from Suc.premis obtain root b roots' s1 s2 s3 where
    rest-out:
      Some ((roots', bs'), t) ∈
        set-dist
          (execute
            (staged-composition-fri-program
              A dg (Suc i) n (bs @ [b])) s3)
  unfolding staged-composition-fri-program.simps
  by (auto elim!: set-dist-bindE)
  have prefix-suc:
    take (length (bs @ [b])) bs' = bs @ [b]
  by (rule Suc.IH[OF rest-out])
  have take (length bs) bs' =
    take (length bs) (take (length (bs @ [b])) bs')
  by simp
  also have ... = bs
  using prefix-suc by simp
  finally show ?case .
qed

```

lemma *staged-composition-fri-program-challenge-length:*

```

assumes outcome:
  Some ((roots, bs'), t) ∈
    set-dist
      (execute (staged-composition-fri-program A dg i n bs) s)
shows length bs' = length bs + n
using outcome
proof (induction n arbitrary: i bs roots bs' s t)
  case 0
  then show ?case by simp

```



```

next
case (Suc n)
from Suc.premis obtain root b roots' s1 s2 s3 where
  rest-out:
    Some ((roots', bs'), t) ∈
      set-dist
        (execute
          (staged-composition-fri-program
            A dg (Suc i) n (bs @ [b])) s3)
  unfolding staged-composition-fri-program.simps
  by (auto elim!: set-dist-bindE)
have length bs' = length (bs @ [b]) + n
  by (rule Suc.IH[OF rest-out])
then show ?case by simp
qed

lemma staged-composition-fri-challenge-prefix-program-length:
  assumes outcome:
    Some ((dg, composition-bs, rt), t) ∈
      set-dist
        (execute (staged-composition-fri-challenge-prefix-program A i) s)
  shows length composition-bs = i
proof -
  from outcome obtain as s1 dg0 s2 s3 where
    inner:
      Some ((dg, composition-bs, rt), t) ∈
        set-dist
          (execute
            (let composition-rounds = ceil-log (to-nat dg0 + 1)
              in assert (Suc i ≤ composition-rounds) >>=
                (λ-. assert
                  (composition-rounds ≤ ceil-log (maxDegree + 1)) >>=
                    (λ-.
                      staged-composition-fri-program A dg0 0 i [] >>=
                        (λ(composition-roots, composition-bs).
                          composition-fri-root-stage A dg0 i
                            composition-bs >>=
                              (λrt.
                                record-staged-message rt >>=
                                  (λ-. return
                                    (dg0, composition-bs, rt)))))))
              s3)
  unfolding staged-composition-fri-challenge-prefix-program-def
  by (auto elim!: set-dist-bindE)
let ?rounds = ceil-log (to-nat dg0 + 1)
have fri-out:
  ∃ composition-roots s4.
    Some ((composition-roots, composition-bs), s4) ∈
      set-dist

```

```

      (execute (staged-composition-fri-program A dg0 0 i []) s3)
proof (cases Suc i ≤ ?rounds)
  case False
  then show ?thesis
    using inner
    by (auto simp: assert-def throw-no-outcome elim!: set-dist-bindE)
next
  case first-guard: True
  show ?thesis
proof (cases ?rounds ≤ ceil-log (maxDegree + 1))
  case False
  then show ?thesis
    using inner first-guard
    by (auto simp: assert-def throw-no-outcome elim!: set-dist-bindE)
next
  case second-guard: True
  from inner obtain composition-roots s4 where
    Some ((composition-roots, composition-bs), s4) ∈
      set-dist
      (execute (staged-composition-fri-program A dg0 0 i []) s3)
    using first-guard second-guard
    by (auto simp: assert-def elim!: set-dist-bindE split: prod.splits)
  then show ?thesis by blast
qed
qed
then obtain composition-roots s4 where fri-out:
  Some ((composition-roots, composition-bs), s4) ∈
    set-dist
    (execute (staged-composition-fri-program A dg0 0 i []) s3)
  by blast
have length composition-bs = length ([] :: 'f list) + i
  by (rule staged-composition-fri-program-challenge-length[OF fri-out])
then show ?thesis by simp
qed

```

lemma staged-composition-fri-prefix-vector-member-imp-position-value:

assumes outcome:

Some ((dg, composition-bs, rt), t) ∈
 set-dist
 (execute (staged-composition-fri-challenge-prefix-program A i) s)

and subset: $B \subseteq \text{fri-challenge-space } i$
and member: $\text{composition-bs} \in B$
and j-bound: $j < i$

shows $\text{composition-bs} ! j \in \text{fri-vector-position-values } B j$
by (rule fri-vector-member-imp-position-value
 [OF subset member j-bound])

lemma staged-composition-fri-prefix-head-challenge-imp-position-value:

assumes outcome:

$\text{Some } (((dg, \text{composition-bs}, rt), b), t) \in$
 set-dist
 $(\text{execute}$
 $(\text{staged-composition-fri-challenge-prefix-program } A \ i \gg=$
 $(\lambda \text{prefix}. \text{receive-composition-fri-challenge } \gg=$
 $(\lambda b. \text{return } (\text{prefix}, b)))) \ s)$
and $\text{member: composition-bs @ [b] } \in B$
shows $b \in \text{fri-vector-position-values } B \ i$
proof –
from *outcome* **obtain** *s1* **where**
 prefix-out:
 $\text{Some } ((dg, \text{composition-bs}, rt), s1) \in$
 set-dist
 $(\text{execute } (\text{staged-composition-fri-challenge-prefix-program } A \ i) \ s)$
by $(\text{auto elim!}: \text{set-dist-bindE})$
have $\text{len: length composition-bs} = i$
by $(\text{rule staged-composition-fri-challenge-prefix-program-length}$
 $[\text{OF prefix-out}])$
show *?thesis*
by $(\text{rule fri-vector-extend-member-imp-position-value}$
 $[\text{OF member len}])$
qed

lemma *staged-composition-fri-vector-bad-coordinate-cover-bound:*
assumes *subset:*
 $\bigwedge dg. B \ dg \subseteq \text{fri-challenge-space } (\text{ceil-log } (\text{to-nat } dg + 1))$
shows
 $\text{wp-event } (\text{staged-composition-fri-vector-program } A)$
 $(\lambda \text{out}. \text{case out of None} \Rightarrow \text{False}$
 $\mid \text{Some } ((dg, \text{composition-bs}), -) \Rightarrow$
 $\text{composition-bs} \in B \ dg \wedge$
 $0 < \text{ceil-log } (\text{to-nat } dg + 1) \wedge$
 $\text{ceil-log } (\text{to-nat } dg + 1) \leq \text{ceil-log } (\text{maxDegree} + 1)) \ s \leq$
 $(\sum i < \text{ceil-log } (\text{maxDegree} + 1).$
 $\text{wp-event } (\text{staged-composition-fri-vector-program } A)$
 $(\lambda \text{out}. \text{case out of None} \Rightarrow \text{False}$
 $\mid \text{Some } ((dg, \text{composition-bs}), -) \Rightarrow$
 $i < \text{ceil-log } (\text{to-nat } dg + 1) \wedge$
 $\text{composition-bs ! } i \in$
 $\text{fri-vector-position-values } (B \ dg \ i) \ s)$

proof –
let *?bad* =
 $\lambda \text{out}. \text{case out of None} \Rightarrow \text{False}$
 $\mid \text{Some } ((dg, \text{composition-bs}), -) \Rightarrow$
 $\text{composition-bs} \in B \ dg \wedge$
 $0 < \text{ceil-log } (\text{to-nat } dg + 1) \wedge$
 $\text{ceil-log } (\text{to-nat } dg + 1) \leq \text{ceil-log } (\text{maxDegree} + 1)$
let *?coord* =
 $\lambda i \text{ out}. \text{case out of None} \Rightarrow \text{False}$

```

| Some ((dg, composition-bs), -) ⇒
  i < ceil-log (to-nat dg + 1) ∧
  composition-bs ! i ∈ fri-vector-position-values (B dg) i
have bad-to-coord:
  wp-event (staged-composition-fri-vector-program A) ?bad s ≤
    wp-event (staged-composition-fri-vector-program A)
    (λout. ∃ i ∈ {.. $\text{ceil-log (maxDegree + 1)}$ }. ?coord i out) s
proof (rule wp-event-mono)
  fix out
  assume bad: ?bad out
  show ∃ i ∈ {.. $\text{ceil-log (maxDegree + 1)}$ }. ?coord i out
  proof (cases out)
    case None
    then show ?thesis using bad by simp
  next
    case (Some result)
    then obtain dg composition-bs t where
      out-eq: out = Some ((dg, composition-bs), t)
      by (cases result, auto split: prod.splits)
    let ?n = ceil-log (to-nat dg + 1)
    have member: composition-bs ∈ B dg
      and nonempty: 0 < ?n
      and rounds-bound: ?n ≤ ceil-log (maxDegree + 1)
      using bad unfolding out-eq by simp-all
    from fri-vector-member-coordinate-cover
      [OF subset[of dg] member nonempty]
    obtain i where i-bound: i < ?n
      and coord:
        composition-bs ! i ∈ fri-vector-position-values (B dg) i
      by blast
    have i-max: i < ceil-log (maxDegree + 1)
      using i-bound rounds-bound by simp
    show ?thesis
      by (rule bexI[of - i])
      (use i-bound i-max coord out-eq in simp-all)
  qed
qed
also have ... ≤
  (∑ i < ceil-log (maxDegree + 1).
    wp-event (staged-composition-fri-vector-program A) (?coord i) s)
  by (rule wp-event-finite-UN-bound) simp-all
finally show ?thesis .
qed

```

lemma
staged-composition-fri-vector-coordinate-hit-imp-new-output-hit:
assumes wf: staged-budget-wellformed budgets
and controlled: staged-adversary-controlled budgets A
and i-bound: i < ceil-log (to-nat dg + 1)

and absent:
composition-fri-challenge-values-absent
(fri-vector-position-values (B dg) i) s
and outcome:
Some ((dg, composition-bs), t) ∈
set-dist (execute (staged-composition-fri-vector-program A) s)
and hit:
composition-bs ! i ∈ fri-vector-position-values (B dg) i
shows
hash-new-output-hit-event (fri-vector-position-values (B dg) i) s
(Some ((dg, composition-bs), t))
proof –
from outcome obtain *alpha-prefix s1 as s2 s3 s4 s5 roots* **where**
alpha-prefix-out:
Some (alpha-prefix, s1) ∈
set-dist (execute (staged-alpha-prefix-program A) s)
and alpha-out:
Some (as, s2) ∈
set-dist (execute (staged-alpha-program (length spec)) s1)
and degree-out:
Some (dg, s3) ∈ set-dist (execute (degree-stage A as) s2)
and record-degree:
Some ((), s4) ∈
set-dist (execute (record-staged-message dg) s3)
and assert-out:
Some ((), s5) ∈
set-dist
(execute
(assert
(ceil-log (to-nat dg + 1) ≤
ceil-log (maxDegree + 1))) s4)
and fri-out:
Some ((roots, composition-bs), t) ∈
set-dist
(execute
(staged-composition-fri-program A dg 0
(ceil-log (to-nat dg + 1)) []) s5)
unfolding *staged-composition-fri-vector-program-def Let-def*
by *(auto elim!: set-dist-bindE split: prod.splits)*
have *round-bound:*
ceil-log (to-nat dg + 1) ≤ ceil-log (maxDegree + 1)
using *assert-out* **unfolding** *assert-def*
by *(cases*
ceil-log (to-nat dg + 1) ≤ ceil-log (maxDegree + 1))
(auto simp: throw-no-outcome)
have *len:*
0 + ceil-log (to-nat dg + 1) ≤
length (composition-fri-budgets budgets)
using *round-bound wf* **unfolding** *staged-budget-wellformed-def* **by** *simp*

```

have alignment:
  length roots = ceil-log (to-nat dg + 1) ∧
  length composition-bs =
    length ([] :: 'f list) + ceil-log (to-nat dg + 1) ∧
  PTranscript t = PTranscript s5 @ roots ∧
  PState t = foldl concat (PState s5) roots ∧
  s5 ≤ t ∧
  PTraceFriCounter t = PTraceFriCounter s5 ∧
  PCompositionFriCounter t =
    PCompositionFriCounter s5 + ceil-log (to-nat dg + 1) ∧
  PAlphaCounter t = PAlphaCounter s5 ∧
  PQueryCounter t = PQueryCounter s5 ∧
  (∀ j < ceil-log (to-nat dg + 1).
    fmlookup (HashMap t)
      (CompositionFriChallenge (PCompositionFriCounter s5 + j)
        (foldl concat (PState s5) (take (Suc j) roots))) =
      Some (composition-bs ! (length ([] :: 'f list) + j)))
  by (rule staged-composition-fri-program-alignment
    [OF controlled len fri-out])
let ?key =
  CompositionFriChallenge (PCompositionFriCounter s5 + i)
    (foldl concat (PState s5) (take (Suc i) roots))
have lookup-t:
  fmlookup (HashMap t) ?key = Some (composition-bs ! i)
  using alignment i-bound by simp
have ext-s-t: s ≤ t
  using hash-target-program-staged-composition-fri-vector-program
    [OF wf controlled,
      of fri-vector-position-values (B dg) i] outcome
  unfolding hash-target-program-def hash-extension-preserving-def
  by blast
have none-s: fmlookup (HashMap s) ?key = None
proof (cases fmlookup (HashMap s) ?key)
  case None
  then show ?thesis .
next
  case (Some old)
  have old-lookup-t:
    fmlookup (HashMap t) ?key = Some old
    by (rule hash-extension-lookup[OF Some ext-s-t])
  then have old-eq: old = composition-bs ! i
  using lookup-t by simp
  have old ∉ fri-vector-position-values (B dg) i
  using absent Some
  unfolding composition-fri-challenge-values-absent-def by blast
  then show ?thesis
  using old-eq hit by simp
qed
have hash-map-new-output-hit

```

```

    (fri-vector-position-values (B dg) i) s t
  unfolding hash-map-new-output-hit-def
  using none-s lookup-t hit by blast
then show ?thesis
  unfolding hash-new-output-hit-event-def by simp
qed

lemma staged-composition-fri-vector-coordinate-bound-fixed-degree:
  assumes wf: staged-budget-wellformed budgets
  and controlled: staged-adversary-controlled budgets A
  and i-bound: i < ceil-log (to-nat dg + 1)
  and absent:
    composition-fri-challenge-values-absent
    (fri-vector-position-values (B dg) i) s
  shows
    wp-event (staged-composition-fri-vector-program A)
      (λout. case out of None ⇒ False
        | Some ((dg', composition-bs), -) ⇒
          dg' = dg ∧
            composition-bs ! i ∈ fri-vector-position-values (B dg) i) s ≤
      staged-phase-target-error (fri-vector-position-values (B dg) i)
        (staged-composition-fri-vector-search-queries budgets)
proof -
  let ?P =
    λout. case out of None ⇒ False
      | Some ((dg', composition-bs), -) ⇒
        dg' = dg ∧
          composition-bs ! i ∈ fri-vector-position-values (B dg) i
  let ?H =
    hash-new-output-hit-event (fri-vector-position-values (B dg) i) s
  have event-imp-hit:
    wp-event (staged-composition-fri-vector-program A) ?P s ≤
    wp-event (staged-composition-fri-vector-program A) ?H s
proof (rule wp-event-mono-on-support)
  fix out
  assume support:
    out ∈ set-dist
    (execute (staged-composition-fri-vector-program A) s)
  and event: ?P out
  show ?H out
proof (cases out)
  case None
  then show ?thesis using event by simp
next
  case (Some result)
  then obtain dg' composition-bs t where
    out-eq: out = Some ((dg', composition-bs), t)
    by (cases result, auto split: prod.splits)
  have dg-eq: dg' = dg

```

```

    using event unfolding out-eq by simp
  have outcome:
    Some ((dg, composition-bs), t) ∈
      set-dist
        (execute (staged-composition-fri-vector-program A) s)
    using support unfolding out-eq dg-eq .
  have hit:
    composition-bs ! i ∈ fri-vector-position-values (B dg) i
    using event unfolding out-eq by simp
  show ?thesis
    unfolding out-eq dg-eq
    by (rule
      staged-composition-fri-vector-coordinate-hit-imp-new-output-hit
      [where B=B, OF wf controlled i-bound absent outcome hit])
qed
qed
also have
  wp-event (staged-composition-fri-vector-program A) ?H s ≤
    staged-phase-target-error (fri-vector-position-values (B dg) i)
    (staged-composition-fri-vector-search-queries budgets)
  by (rule staged-composition-fri-vector-target-hit-bound
    [OF wf controlled])
  finally show ?thesis .
qed

lemma composition-fri-challenge-values-absent-subset:
  assumes subset:  $A \subseteq B$ 
  and absent: composition-fri-challenge-values-absent B s
  shows composition-fri-challenge-values-absent A s
  using subset absent
  unfolding composition-fri-challenge-values-absent-def by blast

lemma hash-map-new-output-hit-mono-set:
  assumes subset:  $A \subseteq B$ 
  and hit: hash-map-new-output-hit A s t
  shows hash-map-new-output-hit B s t
  using subset hit unfolding hash-map-new-output-hit-def by blast

lemma hash-new-output-hit-event-mono-set:
  assumes subset:  $A \subseteq B$ 
  and hit: hash-new-output-hit-event A s out
  shows hash-new-output-hit-event B s out
proof (cases out)
  case None
  then show ?thesis
    using hit unfolding hash-new-output-hit-event-def by simp
next
  case (Some result)
  then obtain x t where out-eq: out = Some (x, t)

```



```

  by (cases result) simp
have map-hit: hash-map-new-output-hit A s t
  using hit unfolding out-eq hash-new-output-hit-event-def by simp
have hash-map-new-output-hit B s t
  by (rule hash-map-new-output-hit-mono-set[OF subset map-hit])
show ?thesis
  using ⟨hash-map-new-output-hit B s t⟩
  unfolding out-eq hash-new-output-hit-event-def by simp
qed

```

lemma staged-composition-fri-vector-coordinate-bound-union:

assumes wf: staged-budget-wellformed budgets
and controlled: staged-adversary-controlled budgets A
and absent:

composition-fri-challenge-values-absent
 $(\bigcup dg. \text{fri-vector-position-values } (B \ dg) \ i) \ s$

shows

wp-event (staged-composition-fri-vector-program A)
 $(\lambda out. \text{case out of None} \Rightarrow \text{False}$
 $\mid \text{Some } ((dg, \text{composition-bs}), -) \Rightarrow$
 $i < \text{ceil-log } (\text{to-nat } dg + 1) \wedge$
 $\text{composition-bs} ! i \in$
 $\text{fri-vector-position-values } (B \ dg) \ i) \ s \leq$
staged-phase-target-error
 $(\bigcup dg. \text{fri-vector-position-values } (B \ dg) \ i)$
(staged-composition-fri-vector-search-queries budgets)

proof –

let ?U = $(\bigcup dg. \text{fri-vector-position-values } (B \ dg) \ i)$

let ?P =

$\lambda out. \text{case out of None} \Rightarrow \text{False}$
 $\mid \text{Some } ((dg, \text{composition-bs}), -) \Rightarrow$
 $i < \text{ceil-log } (\text{to-nat } dg + 1) \wedge$
 $\text{composition-bs} ! i \in \text{fri-vector-position-values } (B \ dg) \ i$

let ?H = hash-new-output-hit-event ?U s

have event-imp-hit:

wp-event (staged-composition-fri-vector-program A) ?P s ≤
wp-event (staged-composition-fri-vector-program A) ?H s

proof (rule wp-event-mono-on-support)

fix out

assume support:

out ∈ set-dist
(execute (staged-composition-fri-vector-program A) s)

and event: ?P out

show ?H out

proof (cases out)

case None

then show ?thesis **using** event **by** simp

next

case (Some result)

then obtain dg *composition-bs* t **where**
out-eq: $out = Some ((dg, composition-bs), t)$
by (*cases result*, *auto split*: *prod.splits*)
have *outcome*:
 $Some ((dg, composition-bs), t) \in$
set-dist
 $(execute (staged-composition-fri-vector-program A) s)$
using support unfolding *out-eq* .
have *i-bound*: $i < ceil-log (to-nat dg + 1)$
using event unfolding *out-eq* **by** *simp*
have *hit*:
 $composition-bs ! i \in fri-vector-position-values (B dg) i$
using event unfolding *out-eq* **by** *simp*
have *small-absent*:
 $composition-fri-challenge-values-absent$
 $(fri-vector-position-values (B dg) i) s$
by (*rule composition-fri-challenge-values-absent-subset*
 $[OF - absent]$) *blast*
have *small-hit*:
 $hash-new-output-hit-event$
 $(fri-vector-position-values (B dg) i) s$
 $(Some ((dg, composition-bs), t))$
by (*rule*
 $staged-composition-fri-vector-coordinate-hit-imp-new-output-hit$
 $[where B=B,$
 $OF wf controlled i-bound small-absent outcome hit]$)
have *subset*:
 $fri-vector-position-values (B dg) i \subseteq ?U$
by *blast*
show *?thesis*
unfolding *out-eq*
by (*rule hash-new-output-hit-event-mono-set*
 $[OF subset small-hit]$)
qed
qed
also have
 $wp-event (staged-composition-fri-vector-program A) ?H s \leq$
 $staged-phase-target-error ?U$
 $(staged-composition-fri-vector-search-queries budgets)$
by (*rule staged-composition-fri-vector-target-hit-bound*
 $[OF wf controlled]$)
finally show *?thesis* .
qed

lemma *staged-composition-fri-vector-bad-bound-union*:
assumes *wf*: *staged-budget-wellformed budgets*
and *controlled*: *staged-adversary-controlled budgets A*
and *subset*:
 $\bigwedge dg. B dg \subseteq fri-challenge-space (ceil-log (to-nat dg + 1))$

and *absent*:
 $\bigwedge i. i < \text{ceil-log } (\text{maxDegree} + 1) \Rightarrow$
 $\text{composition-fri-challenge-values-absent}$
 $(\bigcup dg. \text{fri-vector-position-values } (B \text{ dg}) \ i) \ s$
shows
 $\text{wp-event } (\text{staged-composition-fri-vector-program } A)$
 $(\lambda \text{out. case out of None} \Rightarrow \text{False}$
 $\mid \text{Some } ((dg, \text{composition-bs}), -) \Rightarrow$
 $\text{composition-bs} \in B \text{ dg} \wedge$
 $0 < \text{ceil-log } (\text{to-nat } dg + 1) \wedge$
 $\text{ceil-log } (\text{to-nat } dg + 1) \leq \text{ceil-log } (\text{maxDegree} + 1)) \ s \leq$
 $(\sum i < \text{ceil-log } (\text{maxDegree} + 1).$
 $\text{staged-phase-target-error}$
 $(\bigcup dg. \text{fri-vector-position-values } (B \text{ dg}) \ i)$
 $(\text{staged-composition-fri-vector-search-queries budgets}))$
proof –
let *?coord* =
 $\lambda i \text{ out. case out of None} \Rightarrow \text{False}$
 $\mid \text{Some } ((dg, \text{composition-bs}), -) \Rightarrow$
 $i < \text{ceil-log } (\text{to-nat } dg + 1) \wedge$
 $\text{composition-bs} ! i \in \text{fri-vector-position-values } (B \text{ dg}) \ i$
have *cover*:
 $\text{wp-event } (\text{staged-composition-fri-vector-program } A)$
 $(\lambda \text{out. case out of None} \Rightarrow \text{False}$
 $\mid \text{Some } ((dg, \text{composition-bs}), -) \Rightarrow$
 $\text{composition-bs} \in B \text{ dg} \wedge$
 $0 < \text{ceil-log } (\text{to-nat } dg + 1) \wedge$
 $\text{ceil-log } (\text{to-nat } dg + 1) \leq$
 $\text{ceil-log } (\text{maxDegree} + 1)) \ s \leq$
 $(\sum i < \text{ceil-log } (\text{maxDegree} + 1).$
 $\text{wp-event } (\text{staged-composition-fri-vector-program } A)$
 $(\text{?coord } i) \ s)$
by (*rule staged-composition-fri-vector-bad-coordinate-cover-bound*
 $[\text{OF subset}]$)
also have $\dots \leq$
 $(\sum i < \text{ceil-log } (\text{maxDegree} + 1).$
 $\text{staged-phase-target-error}$
 $(\bigcup dg. \text{fri-vector-position-values } (B \text{ dg}) \ i)$
 $(\text{staged-composition-fri-vector-search-queries budgets}))$
proof (*rule sum-mono*)
fix *i*
assume *i-in*: $i \in \{.. < \text{ceil-log } (\text{maxDegree} + 1)\}$
then have *i-bound*: $i < \text{ceil-log } (\text{maxDegree} + 1)$
by *simp*
show
 $\text{wp-event } (\text{staged-composition-fri-vector-program } A)$
 $(\text{?coord } i) \ s \leq$
 $\text{staged-phase-target-error}$
 $(\bigcup dg. \text{fri-vector-position-values } (B \text{ dg}) \ i)$

```

      (staged-composition-fri-vector-search-queries budgets)
    by (rule staged-composition-fri-vector-coordinate-bound-union
        [OF wf controlled absent[OF i-bound]])
  qed
  finally show ?thesis .
qed

```

lemma *staged-composition-fri-full-vector-bad-bound-union:*
assumes *wf: staged-budget-wellformed budgets*
and *controlled: staged-adversary-controlled budgets A*
and *subset:*
 $\bigwedge dg. B \ dg \subseteq \text{fri-challenge-space } (\text{ceil-log } (to\text{-nat } dg + 1))$
and *absent:*
 $\bigwedge i. i < \text{ceil-log } (\text{maxDegree} + 1) \implies$
composition-fri-challenge-values-absent
 $(\bigcup dg. \text{fri-vector-position-values } (B \ dg) \ i)$
adversary-initial-state

shows

```

wp-event (staged-composition-fri-full-vector-program A)
(staged-composition-prefix-fri-vector-hit B)
adversary-initial-state ≤
(∑ i < ceil-log (maxDegree + 1).
  staged-phase-target-error
    (⋃ dg. fri-vector-position-values (B dg) i)
    (staged-composition-fri-vector-search-queries budgets))

```

proof –

```

let ?Q =
  λout. case out of None ⇒ False
    | Some ((dg, composition-bs), -) ⇒
      composition-bs ∈ B dg ∧
      0 < ceil-log (to-nat dg + 1) ∧
      ceil-log (to-nat dg + 1) ≤ ceil-log (maxDegree + 1)

```

have *projected:*

```

wp-event (staged-composition-fri-vector-program A) ?Q
adversary-initial-state ≤
(∑ i < ceil-log (maxDegree + 1).
  staged-phase-target-error
    (⋃ dg. fri-vector-position-values (B dg) i)
    (staged-composition-fri-vector-search-queries budgets))

```

```

by (rule staged-composition-fri-vector-bad-bound-union
    [OF wf controlled subset absent])

```

have *eq:*

```

wp-event (staged-composition-fri-vector-program A) ?Q
adversary-initial-state =
wp-event (staged-composition-fri-full-vector-program A)
(staged-composition-prefix-fri-vector-hit B)
adversary-initial-state

```

```

unfolding staged-composition-fri-vector-program-projection
wp-event-bind-return-map

```

```

    staged-composition-prefix-fri-vector-hit-def
  by (simp split: option.splits prod.splits)
show ?thesis
  using projected unfolding eq .
qed

lemma staged-transcript-program-composition-fri-vector-bound:
  assumes wf: staged-budget-wellformed budgets
  and controlled: staged-adversary-controlled budgets A
  and subset:
     $\bigwedge dg. B \ dg \subseteq \text{fri-challenge-space } (\text{ceil-log } (\text{to-nat } dg + 1))$ 
  and absent:
     $\bigwedge i. i < \text{ceil-log } (\text{maxDegree} + 1) \implies$ 
      composition-fri-challenge-values-absent
       $(\bigcup dg. \text{fri-vector-position-values } (B \ dg) \ i)$ 
      adversary-initial-state
  shows
    wp-event (staged-transcript-program A)
      (staged-transcript-composition-fri-vector-guarded-hit B)
      adversary-initial-state  $\leq$ 
       $(\sum i < \text{ceil-log } (\text{maxDegree} + 1).$ 
        staged-phase-target-error
         $(\bigcup dg. \text{fri-vector-position-values } (B \ dg) \ i)$ 
        (staged-composition-fri-vector-search-queries budgets))
  proof -
    have prefix-bound:
      wp-event (staged-composition-fri-full-vector-program A)
        (staged-composition-prefix-fri-vector-hit B)
        adversary-initial-state  $\leq$ 
         $(\sum i < \text{ceil-log } (\text{maxDegree} + 1).$ 
          staged-phase-target-error
           $(\bigcup dg. \text{fri-vector-position-values } (B \ dg) \ i)$ 
          (staged-composition-fri-vector-search-queries budgets))
      by (rule staged-composition-fri-full-vector-bad-bound-union
        [OF wf controlled subset absent])
    show ?thesis
      unfolding staged-transcript-program-composition-fri-full-vector-decomp
    proof (rule wp-event-bind-bound-by-head-event[OF prefix-bound])
      show
        staged-transcript-composition-fri-vector-guarded-hit B None  $\implies$ 
        staged-composition-prefix-fri-vector-hit B None
      unfolding staged-transcript-composition-fri-vector-guarded-hit-def
        staged-composition-prefix-fri-vector-hit-def
      by simp
    next
      fix prefix t out
      assume cont:
        out  $\in$ 
        set-dist

```

```

      (execute
        (staged-after-composition-fri-full-vector-program A prefix) t)
    and hit:
      staged-transcript-composition-fri-vector-guarded-hit B out
  show staged-composition-prefix-fri-vector-hit B (Some (prefix, t))
  by (rule staged-after-composition-fri-full-vector-program-hitD
    [OF cont hit])
qed
qed

```

lemma *staged-query-program-alignment*:

```

assumes controlled: staged-adversary-controlled budgets A
and bound:  $i + n \leq \text{length } (\text{query-opening-budgets budgets})$ 
and outcome:
  Some (chunks, t)  $\in$ 
    set-dist (execute (staged-query-program A i n) s)

```

shows

```

length chunks = n  $\wedge$ 
PTranscript t = PTranscript s @ List.concat chunks  $\wedge$ 
PState t = foldl concat (PState s) (List.concat chunks)  $\wedge$ 
PTraceFriCounter t = PTraceFriCounter s  $\wedge$ 
PCompositionFriCounter t = PCompositionFriCounter s  $\wedge$ 
PAlphaCounter t = PAlphaCounter s  $\wedge$ 
PQueryCounter t = PQueryCounter s + n

```

using bound outcome

proof (induction n arbitrary: i chunks s t)

case 0

then show ?case **by** simp

next

case (Suc n)

from staged-query-program-Suc-outcomeE[OF Suc.premis(2)]

obtain raw chunk chunks' s1 s2 s3 **where**

challenge-out:

```

Some (raw, s1)  $\in$ 
  set-dist (execute receive-query-index-challenge s)

```

and stage-out:

```

Some (chunk, s2)  $\in$ 
  set-dist (execute (query-opening-stage A i raw) s1)

```

and record-out:

```

Some ((), s3)  $\in$ 
  set-dist (execute (record-staged-messages chunk) s2)

```

and rest-out:

```

Some (chunks', t)  $\in$ 
  set-dist (execute (staged-query-program A (Suc i) n) s3)

```

and chunks-eq: chunks = chunk # chunks'

.

have challenge-fields:

PState s1 = PState s $\wedge$

PTranscript s1 = PTranscript s

```

using receive-query-index-challenge-outcome[OF challenge-out] by simp
have challenge-counters:
  PTraceFriCounter s1 = PTraceFriCounter s ∧
  PCompositionFriCounter s1 = PCompositionFriCounter s ∧
  PAlphaCounter s1 = PAlphaCounter s ∧
  PQueryCounter s1 = Suc (PQueryCounter s)
using receive-query-index-challenge-counter-outcome[OF challenge-out]
by simp
have i-bound:  $i < \text{length } (\text{query-opening-budgets } \text{budgets})$ 
using Suc.premis(1) by simp
have stage-controlled:
  controlled-ro-program (query-opening-budgets budgets ! i)
    (query-opening-stage A i raw)
using controlled i-bound
unfolding staged-adversary-controlled-def by blast
have stage-fields:
  PState s2 = PState s1 ∧
  PTranscript s2 = PTranscript s1 ∧
  PTraceFriCounter s2 = PTraceFriCounter s1 ∧
  PCompositionFriCounter s2 = PCompositionFriCounter s1 ∧
  PAlphaCounter s2 = PAlphaCounter s1 ∧
  PQueryCounter s2 = PQueryCounter s1
using controlled-stage-outcome-fields[OF stage-controlled stage-out]
by simp
have s3:
  s3 = s2(|
    PState := foldl concat (PState s2) chunk,
    PTranscript := PTranscript s2 @ chunk|)
by (rule record-staged-messages-outcome[OF record-out])
have s3-fields:
  PState s3 = foldl concat (PState s) chunk ∧
  PTranscript s3 = PTranscript s @ chunk ∧
  PTraceFriCounter s3 = PTraceFriCounter s ∧
  PCompositionFriCounter s3 = PCompositionFriCounter s ∧
  PAlphaCounter s3 = PAlphaCounter s ∧
  PQueryCounter s3 = Suc (PQueryCounter s)
unfolding s3
using challenge-fields challenge-counters stage-fields by simp
have rest-bound:
  Suc i + n ≤ length (query-opening-budgets budgets)
using Suc.premis(1) by simp
have rest-alignment:
  length chunks' = n ∧
  PTranscript t = PTranscript s3 @ List.concat chunks' ∧
  PState t =
    foldl concat (PState s3) (List.concat chunks') ∧
  PTraceFriCounter t = PTraceFriCounter s3 ∧
  PCompositionFriCounter t = PCompositionFriCounter s3 ∧
  PAlphaCounter t = PAlphaCounter s3 ∧

```

```

    PQueryCounter t = PQueryCounter s3 + n
  by (rule Suc.IH[OF rest-bound rest-out])
show ?case
  unfolding chunks-eq
  using s3-fields rest-alignment
  by (simp add: foldl-append)
qed

lemma staged-query-program-outcome-with-rows:
  assumes controlled: staged-adversary-controlled budgets A
    and bound: i + n ≤ length (query-opening-budgets budgets)
    and outcome:
      Some (chunks, t) ∈
        set-dist (execute (staged-query-program A i n) s)
  shows
    ∃ raw-idxs query-idxs.
      length raw-idxs = n ∧
      query-idxs = map (λraw. index (to-nat raw)) raw-idxs ∧
      length chunks = n ∧
      PTranscript t = PTranscript s @ List.concat chunks ∧
      PState t = state-after-query-chunks (PState s) chunks n ∧
      s ≤ t ∧
      PQueryCounter t = PQueryCounter s + n ∧
      (∀ j < n.
        fmllookup (HashMap t)
          (QueryIndexChallenge (PQueryCounter s + j)
            (state-after-query-chunks (PState s) chunks j)) =
          Some (raw-idxs ! j)) ∧
      (∀ idx ∈ set query-idxs. idx < clength * scale)
    using bound outcome
  proof (induction n arbitrary: i chunks s t)
    case 0
    then show ?case
      by (intro exI[of - []])
        (simp add: state-after-query-chunks-def hash-ext-refl)
  next
    case (Suc n)
    from staged-query-program-Suc-outcomeE[OF Suc.premis(2)]
    obtain raw chunk chunks' s1 s2 s3 where
      challenge-out:
        Some (raw, s1) ∈
          set-dist (execute receive-query-index-challenge s)
      and stage-out:
        Some (chunk, s2) ∈
          set-dist (execute (query-opening-stage A i raw) s1)
      and record-out:
        Some (( ), s3) ∈
          set-dist (execute (record-staged-messages chunk) s2)
      and rest-out:

```



```

    Some (chunks', t) ∈
      set-dist (execute (staged-query-program A (Suc i) n) s3)
  and chunks-eq: chunks = chunk # chunks'
  .
have i-bound: i < length (query-opening-budgets budgets)
  using Suc.premis(1) by simp
have stage-controlled:
  controlled-ro-program (query-opening-budgets budgets ! i)
    (query-opening-stage A i raw)
  using controlled i-bound
  unfolding staged-adversary-controlled-def by blast
have challenge:
  s ≤ s1 ∧
  PState s1 = PState s ∧
  PTranscript s1 = PTranscript s ∧
  fmlookup (HashMap s1)
    (QueryIndexChallenge (PQueryCounter s) (PState s)) = Some raw
  by (rule receive-query-index-challenge-outcome[OF challenge-out])
have challenge-counters:
  PQueryCounter s1 = Suc (PQueryCounter s)
  using receive-query-index-challenge-counter-outcome[OF challenge-out]
  by simp
have ext-s1-s2: s1 ≤ s2
  using controlled-ro-program-extension[OF stage-controlled] stage-out
  unfolding hash-extension-preserving-def by blast
have stage-fields:
  PState s2 = PState s1 ∧
  PTranscript s2 = PTranscript s1 ∧
  PQueryCounter s2 = PQueryCounter s1
  using controlled-stage-outcome-fields[OF stage-controlled stage-out]
  by simp
have s3-eq:
  s3 = s2(
    PState := foldl concat (PState s2) chunk,
    PTranscript := PTranscript s2 @ chunk)
  by (rule record-staged-messages-outcome[OF record-out])
have ext-s2-s3: s2 ≤ s3
  unfolding s3-eq less-eq-hash-ext-def less-eq-fmap-def by simp
have s3-fields:
  PState s3 = foldl concat (PState s) chunk ∧
  PTranscript s3 = PTranscript s @ chunk ∧
  PQueryCounter s3 = Suc (PQueryCounter s)
  unfolding s3-eq using challenge stage-fields challenge-counters by simp
have rest-bound:
  Suc i + n ≤ length (query-opening-budgets budgets)
  using Suc.premis(1) by simp
from Suc.IH[OF rest-bound rest-out]
obtain raw-tail idx-tail where
  len-raw-tail: length raw-tail = n

```

```

and idx-tail-def:
  idx-tail = map ( $\lambda$ raw. index (to-nat raw)) raw-tail
and len-chunks-tail: length chunks' = n
and tr-tail:
  PTranscript t = PTranscript s3 @ List.concat chunks'
and st-tail:
  PState t = state-after-query-chunks (PState s3) chunks' n
and ext-s3-t: s3 ≤ t
and query-count-tail:
  PQueryCounter t = PQueryCounter s3 + n
and lookup-tail:
   $\bigwedge j. j < n \implies$ 
    fmlookup (HashMap t)
      (QueryIndexChallenge (PQueryCounter s3 + j)
        (state-after-query-chunks (PState s3) chunks' j)) =
      Some (raw-tail ! j)
and idx-tail-bound:
   $\forall idx \in \text{set } idx\text{-tail}. idx < \text{clength} * \text{scale}$ 
by blast
let ?raws = raw # raw-tail
let ?idxs = index (to-nat raw) # idx-tail
let ?chunks = chunk # chunks'
have idxs-def: ?idxs = map ( $\lambda$ raw. index (to-nat raw)) ?raws
  using idx-tail-def by simp
have tr-all:
  PTranscript t = PTranscript s @ List.concat ?chunks
  using tr-tail s3-fields by simp
have state-shift:
   $\bigwedge j. \text{state-after-query-chunks } (PState\ s) \ ?chunks\ (Suc\ j) =$ 
    state-after-query-chunks (PState s3) chunks' j
  using s3-fields unfolding state-after-query-chunks-def by simp
have st-all:
  PState t = state-after-query-chunks (PState s) ?chunks (Suc n)
  using st-tail state-shift[of n] by simp
have ext-s-s2: s ≤ s2
  by (rule hash-ext-trans[OF conjunct1[OF challenge] ext-s1-s2])
have ext-s-s3: s ≤ s3
  by (rule hash-ext-trans[OF ext-s-s2 ext-s2-s3])
have ext-s-t: s ≤ t
  by (rule hash-ext-trans[OF ext-s-s3 ext-s3-t])
have query-count-all: PQueryCounter t = PQueryCounter s + Suc n
  using query-count-tail s3-fields by simp
have ext-s1-t: s1 ≤ t
  by (rule hash-ext-trans[OF ext-s1-s2])
  (rule hash-ext-trans[OF ext-s2-s3 ext-s3-t])
have lookup-head-t:
  fmlookup (HashMap t)
    (QueryIndexChallenge (PQueryCounter s) (PState s)) =
    Some raw

```

```

    by (rule hash-extension-lookup[OF conjunct2[OF conjunct2[OF conjunct2[OF challenge]]]
ext-s1-t])
  have lookup-all:
     $\bigwedge j. j < \text{Suc } n \implies$ 
    fmlookup (HashMap t)
      (QueryIndexChallenge (PQueryCounter s + j)
        (state-after-query-chunks (PState s) ?chunks j)) =
      Some (?rows ! j)
  proof -
    fix j
    assume j-bound:  $j < \text{Suc } n$ 
    show
      fmlookup (HashMap t)
        (QueryIndexChallenge (PQueryCounter s + j)
          (state-after-query-chunks (PState s) ?chunks j)) =
        Some (?rows ! j)
    proof (cases j)
      case 0
      then show ?thesis
        using lookup-head-t unfolding state-after-query-chunks-def by simp
    next
      case (Suc k)
      then show ?thesis
        using lookup-tail[of k] j-bound state-shift[of k] s3-fields by simp
    qed
  qed
  have idx-bound-all:
     $\forall \text{idx} \in \text{set } ?\text{idxs}. \text{idx} < \text{clength} * \text{scale}$ 
    using idx-tail-bound index-less-domain by auto
  show ?case
    unfolding chunks-eq
    apply (intro exI[of - ?rows] exI[of - ?idxs] conjI)
      apply (simp add: len-raw-tail)
      apply (rule idxs-def)
      apply (simp add: len-chunks-tail)
      apply (rule tr-all)
      apply (rule st-all)
      apply (rule ext-s-t)
      apply (rule query-count-all)
    apply (intro allI impI, rule lookup-all, assumption)
    using idx-bound-all by simp
  qed

lemma checked-staged-query-program-chunks-match-verifier:
  assumes outcome:
    Some (chunks, t)  $\in$ 
    set-dist
    (execute
      (checked-staged-query-program A trace-roots composition-roots

```

```

      i n) s)
shows
  ∃ raw-idxs query-idxs.
    length raw-idxs = n ∧
    query-idxs = map (λ raw. index (to-nat raw)) raw-idxs ∧
    length chunks = n ∧
    (∀ j < n.
      verifier-query-round-chunk (query-idxs ! j)
      trace-roots composition-roots (chunks ! j)) ∧
    (∀ idx ∈ set query-idxs. idx < clength * scale)
using outcome
proof (induction n arbitrary: i chunks s t)
case 0
then show ?case
  by (intro exI[of -] exI[of -]) simp
next
case (Suc n)
obtain raw chunk chunks' s1 s2 s3 where
  chunk-shape:
    verifier-query-round-chunk (index (to-nat raw))
    trace-roots composition-roots chunk
and rest-out:
  Some (chunks', t) ∈
  set-dist
  (execute
    (checked-staged-query-program A trace-roots composition-roots
      (Suc i) n) s3)
and chunks-eq: chunks = chunk # chunks'
proof (rule checked-staged-query-program-Suc-outcomeE[OF Suc.prem])
fix raw chunk chunks' s1 s2 s3
assume shape:
  verifier-query-round-chunk (index (to-nat raw))
  trace-roots composition-roots chunk
and rest:
  Some (chunks', t) ∈
  set-dist
  (execute
    (checked-staged-query-program A trace-roots composition-roots
      (Suc i) n) s3)
and chunks: chunks = chunk # chunks'
show ?thesis
  by (rule that[OF shape rest chunks])
qed
from Suc.IH[OF rest-out]
obtain raw-tail idx-tail where
  len-raw-tail: length raw-tail = n
and idx-tail-def:
  idx-tail = map (λ raw. index (to-nat raw)) raw-tail
and len-chunks-tail: length chunks' = n

```

```

and tail-shape:
   $\forall j < n.$ 
    verifier-query-round-chunk (idx-tail ! j)
    trace-roots composition-roots (chunks' ! j)
and idx-tail-bound:
   $\forall idx \in \text{set } idx\text{-tail}. idx < \text{clength} * \text{scale}$ 
by blast
let ?raws = raw # raw-tail
let ?idxs = index (to-nat raw) # idx-tail
have idxs-def: ?idxs = map ( $\lambda raw. \text{index } (to\text{-nat } raw)$ ) ?raws
  using idx-tail-def by simp
have shape-all:
   $\forall j < \text{Suc } n.$ 
    verifier-query-round-chunk (?idxs ! j)
    trace-roots composition-roots ((chunk # chunks') ! j)
proof (intro allI impI)
  fix j
  assume j-bound: j < Suc n
  show verifier-query-round-chunk (?idxs ! j)
    trace-roots composition-roots ((chunk # chunks') ! j)
  proof (cases j)
    case 0
    then show ?thesis
      using chunk-shape by simp
  next
    case (Suc k)
    then have k-bound: k < n
      using j-bound by simp
    then show ?thesis
      using tail-shape Suc by simp
  qed
qed
have idx-bound-all:
   $\forall idx \in \text{set } ?idxs. idx < \text{clength} * \text{scale}$ 
  using idx-tail-bound index-less-domain by auto
show ?case
  unfolding chunks-eq
  by (intro exI[of - ?raws] exI[of - ?idxs] conjI)
    (use len-raw-tail idxs-def len-chunks-tail shape-all idx-bound-all
      in simp-all)
qed

lemma checked-staged-query-program-outcome-with-raws:
assumes controlled: staged-adversary-controlled budgets A
and bound:  $i + n \leq \text{length } (\text{query-opening-budgets } \text{budgets})$ 
and outcome:
  Some (chunks, t)  $\in$ 
    set-dist
    (execute

```

(*checked-staged-query-program A trace-roots composition-roots*
i n) s)

shows

$\exists \text{raw-idxs query-idxs.}$
 $\text{length raw-idxs} = n \wedge$
 $\text{query-idxs} = \text{map } (\lambda \text{raw. index (to-nat raw)}) \text{ raw-idxs} \wedge$
 $\text{length chunks} = n \wedge$
 $P\text{Transcript } t = P\text{Transcript } s @ \text{List.concat chunks} \wedge$
 $P\text{State } t = \text{state-after-query-chunks } (P\text{State } s) \text{ chunks } n \wedge$
 $s \leq t \wedge$
 $P\text{QueryCounter } t = P\text{QueryCounter } s + n \wedge$
 $(\forall j < n.$
 $\quad \text{verifier-query-round-chunk (query-idxs ! } j)$
 $\quad \text{trace-roots composition-roots (chunks ! } j)) \wedge$
 $(\forall j < n.$
 $\quad \text{fmlookup (HashMap } t)$
 $\quad (\text{QueryIndexChallenge } (P\text{QueryCounter } s + j)$
 $\quad (\text{state-after-query-chunks } (P\text{State } s) \text{ chunks } j)) =$
 $\quad \text{Some (raw-idxs ! } j)) \wedge$
 $(\forall \text{idx} \in \text{set query-idxs. idx} < \text{clength} * \text{scale})$
using *bound outcome*

proof (*induction n arbitrary: i chunks s t*)

case 0

then show ?*case*

by (*intro exI[of - []] exI[of - []]*)
(simp add: state-after-query-chunks-def hash-ext-refl)

next

case (*Suc n*)

from *checked-staged-query-program-Suc-outcomeE[OF Suc.prem(2)]*

obtain *raw chunk chunks' s1 s2 s3* **where**

challenge-out:
 $\text{Some (raw, s1)} \in$
 $\text{set-dist (execute receive-query-index-challenge s)}$

and *stage-out:*
 $\text{Some (chunk, s2)} \in$
 $\text{set-dist (execute (query-opening-stage A i raw) s1)}$

and *chunk-shape:*
 $\text{verifier-query-round-chunk (index (to-nat raw))}$
 $\text{trace-roots composition-roots chunk}$

and *record-out:*
 $\text{Some } ((), s3) \in$
 $\text{set-dist (execute (record-staged-messages chunk) s2)}$

and *rest-out:*
 $\text{Some (chunks', t)} \in$
 set-dist
 $(\text{execute}$
 $\quad (\text{checked-staged-query-program A trace-roots composition-roots}$
 $\quad (\text{Suc } i) \text{ n) } s3)$

and *chunks-eq: chunks = chunk # chunks'*

```

.
have i-bound:  $i < \text{length } (\text{query-opening-budgets } \text{budgets})$ 
  using  $\text{Suc.prem}(1)$  by simp
have stage-controlled:
  controlled-ro-program  $(\text{query-opening-budgets } \text{budgets } ! i)$ 
     $(\text{query-opening-stage } A \ i \ \text{raw})$ 
  using controlled i-bound
  unfolding staged-adversary-controlled-def by blast
have challenge:
   $s \leq s1 \wedge$ 
   $P\text{State } s1 = P\text{State } s \wedge$ 
   $P\text{Transcript } s1 = P\text{Transcript } s \wedge$ 
   $\text{fmlookup } (\text{HashMap } s1)$ 
   $(\text{QueryIndexChallenge } (P\text{QueryCounter } s) (P\text{State } s)) = \text{Some raw}$ 
  by (rule receive-query-index-challenge-outcome[OF challenge-out])
have challenge-counters:
   $P\text{QueryCounter } s1 = \text{Suc } (P\text{QueryCounter } s)$ 
  using receive-query-index-challenge-counter-outcome[OF challenge-out]
  by simp
have ext-s1-s2:  $s1 \leq s2$ 
  using controlled-ro-program-extension[OF stage-controlled] stage-out
  unfolding hash-extension-preserving-def by blast
have stage-fields:
   $P\text{State } s2 = P\text{State } s1 \wedge$ 
   $P\text{Transcript } s2 = P\text{Transcript } s1 \wedge$ 
   $P\text{QueryCounter } s2 = P\text{QueryCounter } s1$ 
  using controlled-stage-outcome-fields[OF stage-controlled stage-out]
  by simp
have s3-eq:
   $s3 = s2 \langle$ 
     $P\text{State} := \text{foldl concat } (P\text{State } s2) \ \text{chunk},$ 
     $P\text{Transcript} := P\text{Transcript } s2 \ @ \ \text{chunk} \rangle$ 
  by (rule record-staged-messages-outcome[OF record-out])
have ext-s2-s3:  $s2 \leq s3$ 
  unfolding s3-eq less-eq-hash-ext-def less-eq-fmap-def by simp
have s3-fields:
   $P\text{State } s3 = \text{foldl concat } (P\text{State } s) \ \text{chunk} \wedge$ 
   $P\text{Transcript } s3 = P\text{Transcript } s \ @ \ \text{chunk} \wedge$ 
   $P\text{QueryCounter } s3 = \text{Suc } (P\text{QueryCounter } s)$ 
  unfolding s3-eq using challenge stage-fields challenge-counters by simp
have rest-bound:
   $\text{Suc } i + n \leq \text{length } (\text{query-opening-budgets } \text{budgets})$ 
  using  $\text{Suc.prem}(1)$  by simp
from  $\text{Suc.IH}$ [OF rest-bound rest-out]
obtain raw-tail idx-tail where
  len-raw-tail:  $\text{length raw-tail} = n$ 
  and idx-tail-def:
     $\text{idx-tail} = \text{map } (\lambda \text{raw}. \text{index } (\text{to-nat raw})) \ \text{raw-tail}$ 
  and len-chunks-tail:  $\text{length chunks}' = n$ 

```

```

and tr-tail:
  PTranscript t = PTranscript s3 @ List.concat chunks'
and st-tail:
  PState t = state-after-query-chunks (PState s3) chunks' n
and ext-s3-t: s3 ≤ t
and query-count-tail:
  PQueryCounter t = PQueryCounter s3 + n
and chunk-shape-tail:
   $\bigwedge j. j < n \implies$ 
    verifier-query-round-chunk (idx-tail ! j)
    trace-roots composition-roots (chunks' ! j)
and lookup-tail:
   $\bigwedge j. j < n \implies$ 
    fmlookup (HashMap t)
    (QueryIndexChallenge (PQueryCounter s3 + j)
     (state-after-query-chunks (PState s3) chunks' j)) =
    Some (raw-tail ! j)
and idx-tail-bound:
   $\forall \text{idx} \in \text{set } \text{idx-tail}. \text{idx} < \text{clength} * \text{scale}$ 
by blast
let ?raws = raw # raw-tail
let ?idxs = index (to-nat raw) # idx-tail
let ?chunks = chunk # chunks'
have idxs-def: ?idxs = map ( $\lambda \text{raw}. \text{index } (\text{to-nat } \text{raw})$ ) ?raws
  using idx-tail-def by simp
have tr-all:
  PTranscript t = PTranscript s @ List.concat ?chunks
  using tr-tail s3-fields by simp
have state-shift:
   $\bigwedge j. \text{state-after-query-chunks } (\text{PState } s) \text{ ?chunks } (\text{Suc } j) =$ 
    state-after-query-chunks (PState s3) chunks' j
  using s3-fields unfolding state-after-query-chunks-def by simp
have st-all:
  PState t = state-after-query-chunks (PState s) ?chunks (Suc n)
  using st-tail state-shift[of n] by simp
have ext-s-s2: s ≤ s2
  by (rule hash-ext-trans[OF conjunct1[OF challenge] ext-s1-s2])
have ext-s-s3: s ≤ s3
  by (rule hash-ext-trans[OF ext-s-s2 ext-s2-s3])
have ext-s-t: s ≤ t
  by (rule hash-ext-trans[OF ext-s-s3 ext-s3-t])
have query-count-all: PQueryCounter t = PQueryCounter s + Suc n
  using query-count-tail s3-fields by simp
have ext-s1-t: s1 ≤ t
  by (rule hash-ext-trans[OF ext-s1-s2])
  (rule hash-ext-trans[OF ext-s2-s3 ext-s3-t])
have lookup-head-t:
  fmlookup (HashMap t)
  (QueryIndexChallenge (PQueryCounter s) (PState s)) =

```



```

    Some raw
  by (rule hash-extension-lookup
    [OF conjunct2[OF conjunct2[OF conjunct2[OF challenge]]] ext-s1-t])
have chunk-shape-all:
   $\bigwedge j. j < \text{Suc } n \implies$ 
    verifier-query-round-chunk (?idxs ! j)
    trace-roots composition-roots (?chunks ! j)
proof -
  fix j
  assume j-bound:  $j < \text{Suc } n$ 
  show
    verifier-query-round-chunk (?idxs ! j)
    trace-roots composition-roots (?chunks ! j)
proof (cases j)
  case 0
  then show ?thesis
    using chunk-shape by simp
next
  case (Suc k)
  then show ?thesis
    using chunk-shape-tail[of k] j-bound by simp
qed
qed
have lookup-all:
   $\bigwedge j. j < \text{Suc } n \implies$ 
    fmlookup (HashMap t)
    (QueryIndexChallenge (PQueryCounter s + j)
      (state-after-query-chunks (PState s) ?chunks j)) =
    Some (?raws ! j)
proof -
  fix j
  assume j-bound:  $j < \text{Suc } n$ 
  show
    fmlookup (HashMap t)
    (QueryIndexChallenge (PQueryCounter s + j)
      (state-after-query-chunks (PState s) ?chunks j)) =
    Some (?raws ! j)
proof (cases j)
  case 0
  then show ?thesis
    using lookup-head-t unfolding state-after-query-chunks-def by simp
next
  case (Suc k)
  then show ?thesis
    using lookup-tail[of k] j-bound state-shift[of k] s3-fields by simp
qed
qed
have idx-bound-all:
   $\forall \text{idx} \in \text{set } ?\text{idxs}. \text{idx} < \text{clength} * \text{scale}$ 

```

```

    using idx-tail-bound index-less-domain by auto
  show ?case
    unfolding chunks-eq
    by (intro exI[of - ?raws] exI[of - ?idxs] conjI)
      (use len-raw-tail idxs-def len-chunks-tail tr-all st-all ext-s-t
        query-count-all chunk-shape-all lookup-all idx-bound-all in simp-all)
qed

lemma checked-staged-query-program-chunks-match-verifier-lengths:
  assumes outcome:
    Some (chunks, t) ∈
      set-dist
        (execute
          (checked-staged-query-program A trace-roots composition-roots
            i rounds) s)
  shows
    ∃ query-idxs.
      staged-query-chunks-match-verifier-lengths
        (|staged-trace-root = fr,
          staged-trace-fri-roots = trace-roots,
          staged-trace-fri-challenges = trace-bs,
          staged-trace-final = trace-final,
          staged-alphas = as,
          staged-degree = dg,
          staged-composition-fri-roots = composition-roots,
          staged-composition-fri-challenges = composition-bs,
          staged-composition-final = composition-final,
          staged-query-chunks = chunks|)
        query-idxs
  proof -
    from checked-staged-query-program-chunks-match-verifier[OF outcome]
    obtain raw-idxs query-idxs where
      len-chunks: length chunks = rounds
    and chunk-shape:
      ∀ j < rounds.
        verifier-query-round-chunk (query-idxs ! j)
          trace-roots composition-roots (chunks ! j)
    by blast
    have chunk-lens:
      ∀ j < rounds.
        length (chunks ! j) =
          verifier-query-round-transcript-length (query-idxs ! j)
            trace-roots composition-roots
    using chunk-shape verifier-query-round-chunk-length by blast
    show ?thesis
      by (intro exI[of - query-idxs])
        (use len-chunks chunk-lens in
          ⟨simp add: staged-query-chunks-match-verifier-lengths-def⟩)
  qed

```

lemma *checked-staged-transcript-program-query-chunks-match-verifier-lengths:*

assumes *outcome:*

Some (data, t) ∈
set-dist
(execute (checked-staged-transcript-program A) s)

shows $\exists$ *query-idxs.*

staged-query-chunks-match-verifier-lengths data query-idxs

proof –

from *outcome* **obtain** *fr s1 s2 trace-roots trace-bs s3 trace-final s4 s5*
as s6 dg s7 s8 s9 composition-roots composition-bs s10
composition-final s11 s12 query-chunks **where**

query-out:

Some (query-chunks, s12) ∈
set-dist
(execute
(checked-staged-query-program A trace-roots composition-roots
0 rounds)
s11)

and *data-eq:*

data =
(|staged-trace-root = fr,
staged-trace-fri-roots = trace-roots,
staged-trace-fri-challenges = trace-bs,
staged-trace-final = trace-final,
staged-alphas = as,
staged-degree = dg,
staged-composition-fri-roots = composition-roots,
staged-composition-fri-challenges = composition-bs,
staged-composition-final = composition-final,
staged-query-chunks = query-chunks|)

unfolding *checked-staged-transcript-program-def Let-def*

by *(auto elim!: set-dist-bindE split: prod.splits)*

show *?thesis*

unfolding *data-eq*

by *(rule checked-staged-query-program-chunks-match-verifier-lengths*
[OF query-out])

qed

lemma *checked-staged-security-experiment-with-data-state-query-lengthsE:*

assumes *outcome:*

Some (((data, attacker-state), result), final-state) ∈
set-dist
(execute
(checked-staged-security-experiment-with-data-state A)
initial-state)

obtains *query-idxs* **where**

staged-query-chunks-match-verifier-lengths data query-idxs

proof –

```

from checked-staged-security-experiment-with-data-state-outcomeE
  [OF outcome]
have builder:
  Some (data, attacker-state) ∈
    set-dist
    (execute (checked-staged-transcript-program A) initial-state)
  by blast
from checked-staged-transcript-program-query-chunks-match-verifier-lengths
  [OF builder]
obtain query-idxs where
  staged-query-chunks-match-verifier-lengths data query-idxs
  by blast
then show ?thesis
  by (rule that)
qed

```

```

lemma verifier-query-round-program-replays-shaped-chunk:
assumes outcome:
  Some ((), t) ∈
    set-dist
    (execute
      (verifier-query-round-program fr f-fl f-final as fl final) s)
and lookup:
  fmlookup (HashMap s)
    (QueryIndexChallenge (PQueryCounter s) (PState s)) = Some raw
and chunk-shape:
  verifier-query-round-chunk (index (to-nat raw))
    (map snd f-fl) (map snd fl) chunk
and transcript: PTranscript s = chunk @ rest
shows
  PTranscript t = rest ∧
  PState t = foldl concat (PState s) chunk ∧
  PQueryCounter t = Suc (PQueryCounter s) ∧
  s ≤ t
proof –
from verifier-query-round-program-outcome[OF outcome]
obtain raw' idx' chunk' where
  idx'-def: idx' = index (to-nat raw')
and chunk'-shape:
  verifier-query-round-chunk idx' (map snd f-fl) (map snd fl)
    chunk'
and transcript': PTranscript s = chunk' @ PTranscript t
and state-t: PState t = foldl concat (PState s) chunk'
and ext: s ≤ t
and counter-t: PQueryCounter t = Suc (PQueryCounter s)
and lookup-t:
  fmlookup (HashMap t)
    (QueryIndexChallenge (PQueryCounter s) (PState s)) = Some raw'
by blast

```

```

have lookup-t-from-s:
  fmlookup (HashMap t)
    (QueryIndexChallenge (PQueryCounter s) (PState s)) = Some raw
  by (rule hash-extension-lookup[OF lookup ext])
have raw'-eq: raw' = raw
  using lookup-t lookup-t-from-s by simp
have idx'-eq: idx' = index (to-nat raw)
  unfolding idx'-def raw'-eq by simp
have len-eq: length chunk' = length chunk
  using verifier-query-round-chunk-length[OF chunk'-shape]
    verifier-query-round-chunk-length[OF chunk-shape]
  unfolding idx'-eq by simp
have chunk'-eq: chunk' = chunk
proof -
  have chunk' = take (length chunk') (PTranscript s)
    using transcript' by simp
  also have ... = chunk
    using transcript len-eq by simp
  finally show ?thesis .
qed
have transcript-t: PTranscript t = rest
  using transcript' transcript unfolding chunk'-eq by simp
show ?thesis
  using transcript-t state-t counter-t ext unfolding chunk'-eq by simp
qed

```

lemma *ntimes-verifier-query-round-program-replays-shaped-chunks:*

```

assumes outcome:
  Some (results, t) ∈
    set-dist
      (execute
        (ntimes
          (verifier-query-round-program fr f-fl f-final as fl final)
        n) s)
and len-raw: length raw-idxs = n
and len-chunks: length chunks = n
and chunk-shape:
   $\bigwedge i. i < n \implies$ 
    verifier-query-round-chunk (index (to-nat (raw-idxs ! i)))
      (map snd f-fl) (map snd fl) (chunks ! i)
and lookup:
   $\bigwedge i. i < n \implies$ 
    fmlookup (HashMap s)
      (QueryIndexChallenge (PQueryCounter s + i)
        (state-after-query-chunks (PState s) chunks i)) =
      Some (raw-idxs ! i)
and transcript: PTranscript s = List.concat chunks @ rest
shows
  PTranscript t = rest  $\wedge$ 

```

```

    PState t = state-after-query-chunks (PState s) chunks n ∧
    PQueryCounter t = PQueryCounter s + n ∧
    s ≤ t
  using outcome len-row len-chunks chunk-shape lookup transcript
proof (induction n arbitrary: s results t raw-idxs chunks rest)
case 0
then show ?case
  by (simp add: state-after-query-chunks-def hash-ext-refl)
next
case (Suc n)
from Suc.premis(2) obtain raw raw-tail where raw-eq:
  raw-idxs = raw # raw-tail
  by (cases raw-idxs) simp-all
from Suc.premis(3) obtain chunk chunk-tail where chunks-eq:
  chunks = chunk # chunk-tail
  by (cases chunks) simp-all
from Suc.premis(1) obtain u s1 results' where
  head:
    Some (u, s1) ∈
      set-dist
        (execute
          (verifier-query-round-program fr f-fl f-final as fl final) s)
  and tail:
    Some (results', t) ∈
      set-dist
        (execute
          (ntimes
            (verifier-query-round-program fr f-fl f-final as fl final)
            n) s1)
  and results-eq: results = u # results'
  by (auto elim!: set-dist-bindE)
have u-eq: u = ()
  by (cases u) simp
have len-row-tail: length raw-tail = n
  using Suc.premis(2) unfolding raw-eq by simp
have len-chunk-tail: length chunk-tail = n
  using Suc.premis(3) unfolding chunks-eq by simp
have head-shape:
  verifier-query-round-chunk (index (to-nat raw))
    (map snd f-fl) (map snd fl) chunk
  using Suc.premis(4)[of 0] unfolding raw-eq chunks-eq by simp
have head-lookup:
  fmlookup (HashMap s)
    (QueryIndexChallenge (PQueryCounter s) (PState s)) =
    Some raw
  using Suc.premis(5)[of 0]
  unfolding raw-eq chunks-eq state-after-query-chunks-def by simp
have head-transcript:
  PTranscript s = chunk @ (List.concat chunk-tail @ rest)

```

```

using Suc.prems(6) unfolding chunks-eq by simp
from verifier-query-round-program-replays-shaped-chunk
  [OF head[unfolded u-eq] head-lookup head-shape head-transcript]
have replay-head:
  PTranscript s1 = List.concat chunk-tail @ rest ∧
  PState s1 = foldl concat (PState s) chunk ∧
  PQueryCounter s1 = Suc (PQueryCounter s) ∧
  s ≤ s1 .
have tail-shape:
  ∧i. i < n ⇒
    verifier-query-round-chunk (index (to-nat (raw-tail ! i)))
      (map snd f-fl) (map snd fl) (chunk-tail ! i)
proof –
  fix i
  assume i-bound: i < n
  then show
    verifier-query-round-chunk (index (to-nat (raw-tail ! i)))
      (map snd f-fl) (map snd fl) (chunk-tail ! i)
    using Suc.prems(4)[of Suc i]
    unfolding raw-eq chunks-eq by simp
qed
have state-shift:
  ∧i. state-after-query-chunks (PState s) (chunk # chunk-tail)
    (Suc i) =
    state-after-query-chunks (PState s1) chunk-tail i
  using replay-head unfolding state-after-query-chunks-def by simp
have tail-lookup:
  ∧i. i < n ⇒
    fmlookup (HashMap s1)
      (QueryIndexChallenge (PQueryCounter s1 + i)
        (state-after-query-chunks (PState s1) chunk-tail i)) =
    Some (raw-tail ! i)
proof –
  fix i
  assume i-bound: i < n
  have lookup-s:
    fmlookup (HashMap s)
      (QueryIndexChallenge (PQueryCounter s + Suc i)
        (state-after-query-chunks (PState s) (chunk # chunk-tail)
          (Suc i))) =
    Some (raw-tail ! i)
  using Suc.prems(5)[of Suc i] i-bound
  unfolding raw-eq chunks-eq by simp
have key-eq:
    QueryIndexChallenge (PQueryCounter s + Suc i)
      (state-after-query-chunks (PState s) (chunk # chunk-tail)
        (Suc i)) =
    QueryIndexChallenge (PQueryCounter s1 + i)
      (state-after-query-chunks (PState s1) chunk-tail i)

```

```

    using replay-head state-shift[of i] by simp
  show
    fmlookup (HashMap s1)
      (QueryIndexChallenge (PQueryCounter s1 + i)
        (state-after-query-chunks (PState s1) chunk-tail i)) =
      Some (raw-tail ! i)
    using hash-extension-lookup[OF lookup-s conjunct2[OF conjunct2[OF conjunct2[OF re-
play-head]]]]
    unfolding key-eq .
  qed
  from Suc.IH[OF tail len-raw-tail len-chunk-tail tail-shape tail-lookup
    conjunct1[OF replay-head]]
  have replay-tail:
    PTranscript t = rest  $\wedge$ 
    PState t = state-after-query-chunks (PState s1) chunk-tail n  $\wedge$ 
    PQueryCounter t = PQueryCounter s1 + n  $\wedge$ 
    s1  $\leq$  t .
  have state-all:
    PState t =
      state-after-query-chunks (PState s) (chunk # chunk-tail) (Suc n)
    using replay-tail state-shift[of n] by simp
  have counter-all: PQueryCounter t = PQueryCounter s + Suc n
    using replay-head replay-tail by simp
  have ext-all: s  $\leq$  t
    using replay-head replay-tail by (blast intro: hash-ext-trans)
  show ?case
    unfolding raw-eq chunks-eq
    using replay-tail state-all counter-all ext-all by simp
  qed

```

```

lemma checked-staged-query-program-ntimes-replay:
  assumes controlled: staged-adversary-controlled budgets A
  and bound: i + n  $\leq$  length (query-opening-budgets budgets)
  and checked:
    Some (chunks, attacker-t)  $\in$ 
      set-dist
        (execute
          (checked-staged-query-program A (map snd f-fl) (map snd fl)
            i n) attacker-s)
  and verifier:
    Some (results, verifier-t)  $\in$ 
      set-dist
        (execute
          (ntimes
            (verifier-query-round-program fr f-fl f-final as fl final)
            n) verifier-s)
  and hash-ext: attacker-t  $\leq$  verifier-s
  and state-eq: PState verifier-s = PState attacker-s
  and counter-eq: PQueryCounter verifier-s = PQueryCounter attacker-s

```



```

and transcript-eq:
  PTranscript verifier-s = List.concat chunks @ rest
shows
  PTranscript verifier-t = rest  $\wedge$ 
  PState verifier-t =
    state-after-query-chunks (PState verifier-s) chunks n  $\wedge$ 
    PQueryCounter verifier-t = PQueryCounter verifier-s + n  $\wedge$ 
    verifier-s  $\leq$  verifier-t
proof -
from checked-staged-query-program-outcome-with-raws
  [OF controlled bound checked]
obtain raw-idxs query-idxs where
  len-raw: length raw-idxs = n
and query-idxs-def:
  query-idxs = map ( $\lambda$ raw. index (to-nat raw)) raw-idxs
and len-chunks: length chunks = n
and chunk-shape:
   $\bigwedge j. j < n \implies$ 
    verifier-query-round-chunk (query-idxs ! j)
      (map snd f-fl) (map snd fl) (chunks ! j)
and lookup:
   $\bigwedge j. j < n \implies$ 
    fmllookup (HashMap attacker-t)
      (QueryIndexChallenge (PQueryCounter attacker-s + j)
        (state-after-query-chunks (PState attacker-s) chunks j)) =
      Some (raw-idxs ! j)
by blast
have chunk-shape-raw:
   $\bigwedge j. j < n \implies$ 
    verifier-query-round-chunk (index (to-nat (raw-idxs ! j)))
      (map snd f-fl) (map snd fl) (chunks ! j)
proof -
  fix j
  assume j-bound: j < n
  have query-idxs ! j = index (to-nat (raw-idxs ! j))
    using query-idxs-def len-raw j-bound by simp
  then show
    verifier-query-round-chunk (index (to-nat (raw-idxs ! j)))
      (map snd f-fl) (map snd fl) (chunks ! j)
    using chunk-shape[OF j-bound] by simp
qed
have lookup-verifier:
   $\bigwedge j. j < n \implies$ 
    fmllookup (HashMap verifier-s)
      (QueryIndexChallenge (PQueryCounter verifier-s + j)
        (state-after-query-chunks (PState verifier-s) chunks j)) =
      Some (raw-idxs ! j)
proof -
  fix j

```

```

assume j-bound:  $j < n$ 
have lookup-attacker:
  fmlookup (HashMap attacker-t)
    (QueryIndexChallenge (PQueryCounter attacker-s + j)
      (state-after-query-chunks (PState attacker-s) chunks j)) =
    Some (raw-idxs ! j)
  by (rule lookup[OF j-bound])
have lookup-verifier-s:
  fmlookup (HashMap verifier-s)
    (QueryIndexChallenge (PQueryCounter attacker-s + j)
      (state-after-query-chunks (PState attacker-s) chunks j)) =
    Some (raw-idxs ! j)
  by (rule hash-extension-lookup[OF lookup-attacker hash-ext])
show
  fmlookup (HashMap verifier-s)
    (QueryIndexChallenge (PQueryCounter verifier-s + j)
      (state-after-query-chunks (PState verifier-s) chunks j)) =
    Some (raw-idxs ! j)
  using lookup-verifier-s state-eq counter-eq by simp
qed
show ?thesis
  by (rule ntimes-verifier-query-round-program-replays-shaped-chunks
    [OF verifier len-row len-chunks chunk-shape-row lookup-verifier
      transcript-eq])
qed

lemma checked-staged-transcript-program-outcome-shape:
  assumes wf: staged-budget-wellformed budgets
  and controlled: staged-adversary-controlled budgets A
  and outcome:
    Some (data, t) ∈
      set-dist
        (execute (checked-staged-transcript-program A)
          adversary-initial-state)
  shows
    length (staged-trace-fri-roots data) = ceil-log clength ∧
    length (staged-trace-fri-challenges data) = ceil-log clength ∧
    length (staged-alphas data) = length spec ∧
    length (staged-composition-fri-roots data) =
      ceil-log (to-nat (staged-degree data) + 1) ∧
    length (staged-composition-fri-challenges data) =
      ceil-log (to-nat (staged-degree data) + 1) ∧
    ceil-log (to-nat (staged-degree data) + 1) ≤
      ceil-log (maxDegree + 1) ∧
    length (staged-query-chunks data) = rounds
proof –
  from outcome obtain fr s1 s2 trace-roots trace-bs s3 trace-final s4 s5
  as s6 dg s7 s8 s9 composition-roots composition-bs s10
  composition-final s11 s12 query-chunks where

```

```

trace-out:
  Some ((trace-roots, trace-bs), s3) ∈
    set-dist
      (execute
        (staged-trace-fri-program A 0 (ceil-log clength) [] s2)
and alpha-out:
  Some (as, s6) ∈
    set-dist (execute (staged-alpha-program (length spec)) s5)
and degree-eq: staged-degree data = dg
and assert-out:
  Some ((), s9) ∈
    set-dist
      (execute
        (assert
          (ceil-log (to-nat dg + 1) ≤
            ceil-log (maxDegree + 1))) s8)
and composition-out:
  Some ((composition-roots, composition-bs), s10) ∈
    set-dist
      (execute
        (staged-composition-fri-program A dg 0
          (ceil-log (to-nat dg + 1) [] s9))
and query-out:
  Some (query-chunks, s12) ∈
    set-dist
      (execute
        (checked-staged-query-program A trace-roots composition-roots
          0 rounds)
          s11)
and data-eq:
  data =
    (staged-trace-root = fr,
     staged-trace-fri-roots = trace-roots,
     staged-trace-fri-challenges = trace-bs,
     staged-trace-final = trace-final,
     staged-alphas = as,
     staged-degree = dg,
     staged-composition-fri-roots = composition-roots,
     staged-composition-fri-challenges = composition-bs,
     staged-composition-final = composition-final,
     staged-query-chunks = query-chunks)
unfolding checked-staged-transcript-program-def Let-def
by (auto elim!: set-dist-bindE split: prod.splits)
have trace-len:
  length trace-roots = ceil-log clength ∧
  length trace-bs = ceil-log clength
proof –
  have len:
    0 + ceil-log clength ≤ length (trace-fri-budgets budgets)

```

```

using wf unfolding staged-budget-wellformed-def by simp
have alignment:
  length trace-roots = ceil-log clength  $\wedge$ 
  length trace-bs = length ([] :: 'f list) + ceil-log clength  $\wedge$ 
  PTranscript s3 = PTranscript s2 @ trace-roots  $\wedge$ 
  PState s3 = foldl concat (PState s2) trace-roots  $\wedge$ 
  s2  $\leq$  s3  $\wedge$ 
  PTraceFriCounter s3 = PTraceFriCounter s2 + ceil-log clength  $\wedge$ 
  PCompositionFriCounter s3 = PCompositionFriCounter s2  $\wedge$ 
  PAlphaCounter s3 = PAlphaCounter s2  $\wedge$ 
  PQueryCounter s3 = PQueryCounter s2  $\wedge$ 
  ( $\forall j < \text{ceil-log clength}.$ 
    fmlookup (HashMap s3)
      (TraceFriChallenge (PTraceFriCounter s2 + j)
        (foldl concat (PState s2) (take (Suc j) trace-roots))) =
        Some (trace-bs ! (length ([] :: 'f list) + j)))
  by (rule staged-trace-fri-program-alignment
    [OF controlled len trace-out])
then show ?thesis by simp
qed
have alpha-len: length as = length spec
  using staged-alpha-program-alignment[OF alpha-out] by simp
have round-bound:
  ceil-log (to-nat dg + 1)  $\leq$  ceil-log (maxDegree + 1)
  using assert-out unfolding assert-def
  by (cases
    ceil-log (to-nat dg + 1)  $\leq$  ceil-log (maxDegree + 1))
    (auto simp: throw-no-outcome)
have composition-len:
  length composition-roots = ceil-log (to-nat dg + 1)  $\wedge$ 
  length composition-bs = ceil-log (to-nat dg + 1)
proof –
  have len:
    0 + ceil-log (to-nat dg + 1)  $\leq$ 
    length (composition-fri-budgets budgets)
    using wf round-bound unfolding staged-budget-wellformed-def by simp
  have alignment:
    length composition-roots = ceil-log (to-nat dg + 1)  $\wedge$ 
    length composition-bs =
      length ([] :: 'f list) + ceil-log (to-nat dg + 1)  $\wedge$ 
    PTranscript s10 = PTranscript s9 @ composition-roots  $\wedge$ 
    PState s10 = foldl concat (PState s9) composition-roots  $\wedge$ 
    s9  $\leq$  s10  $\wedge$ 
    PTraceFriCounter s10 = PTraceFriCounter s9  $\wedge$ 
    PCompositionFriCounter s10 =
      PCompositionFriCounter s9 + ceil-log (to-nat dg + 1)  $\wedge$ 
    PAlphaCounter s10 = PAlphaCounter s9  $\wedge$ 
    PQueryCounter s10 = PQueryCounter s9  $\wedge$ 
    ( $\forall j < \text{ceil-log (to-nat dg + 1)}.$ 

```

```

      fmlookup (HashMap s10)
      (CompositionFriChallenge (PCompositionFriCounter s9 + j)
      (foldl concat (PState s9) (take (Suc j) composition-roots))) =
      Some (composition-bs ! (length ([] :: 'f list) + j)))
    by (rule staged-composition-fri-program-alignment
      [OF controlled len composition-out])
  then show ?thesis by simp
qed
have query-len: length query-chunks = rounds
  using checked-staged-query-program-chunks-match-verifier[OF query-out]
  by blast
show ?thesis
  unfolding data-eq degree-eq
  using trace-len alpha-len composition-len round-bound query-len by simp
qed

```

lemma *checked-staged-transcript-program-outcome-header-transcript:*

```

assumes wf: staged-budget-wellformed budgets
and controlled: staged-adversary-controlled budgets A
and outcome:
  Some (data, attacker-state) ∈
    set-dist
      (execute (checked-staged-transcript-program A)
        adversary-initial-state)

```

shows

```

  verifier-header-transcript
    (verifier-state-from-adversary attacker-state
      (staged-proof-transcript data))
    (staged-trace-root data)
    (staged-trace-fri-roots data)
    (staged-trace-final data)
    (staged-alphas data)
    (staged-degree data)
    (staged-composition-fri-roots data)
    (staged-composition-final data)
    (List.concat (staged-query-chunks data))

```

proof –

have *shape:*

```

  length (staged-trace-fri-roots data) = ceil-log clength ∧
  length (staged-trace-fri-challenges data) = ceil-log clength ∧
  length (staged-alphas data) = length spec ∧
  length (staged-composition-fri-roots data) =
    ceil-log (to-nat (staged-degree data) + 1) ∧
  length (staged-composition-fri-challenges data) =
    ceil-log (to-nat (staged-degree data) + 1) ∧
  ceil-log (to-nat (staged-degree data) + 1) ≤
    ceil-log (maxDegree + 1) ∧
  length (staged-query-chunks data) = rounds
by (rule checked-staged-transcript-program-outcome-shape

```

```

    [OF wf controlled outcome])
  show ?thesis
  by (rule staged-proof-transcript-verifier-header-transcript)
    (use shape in simp-all)
qed

lemma staged-transcript-program-outcome-shape:
  assumes wf: staged-budget-wellformed budgets
  and controlled: staged-adversary-controlled budgets A
  and outcome:
    Some (data, t) ∈
      set-dist
        (execute (staged-transcript-program A)
          adversary-initial-state)
  shows
    length (staged-trace-fri-roots data) = ceil-log clength ∧
    length (staged-trace-fri-challenges data) = ceil-log clength ∧
    length (staged-alphas data) = length spec ∧
    length (staged-composition-fri-roots data) =
      ceil-log (to-nat (staged-degree data) + 1) ∧
    length (staged-composition-fri-challenges data) =
      ceil-log (to-nat (staged-degree data) + 1) ∧
    ceil-log (to-nat (staged-degree data) + 1) ≤
      ceil-log (maxDegree + 1) ∧
    length (staged-query-chunks data) = rounds
proof -
  from outcome obtain fr s1 s2 trace-roots trace-bs s3 trace-final s4 s5
    as s6 dg s7 s8 s9 composition-roots composition-bs s10
    composition-final s11 s12 query-chunks where
  trace-out:
    Some ((trace-roots, trace-bs), s3) ∈
      set-dist
        (execute
          (staged-trace-fri-program A 0 (ceil-log clength) []) s2)
  and alpha-out:
    Some (as, s6) ∈
      set-dist (execute (staged-alpha-program (length spec)) s5)
  and degree-eq: staged-degree data = dg
  and assert-out:
    Some ((), s9) ∈
      set-dist
        (execute
          (assert
            (ceil-log (to-nat dg + 1) ≤
              ceil-log (maxDegree + 1)))) s8)
  and composition-out:
    Some ((composition-roots, composition-bs), s10) ∈
      set-dist
        (execute

```

```

      (staged-composition-fri-program A dg 0
       (ceil-log (to-nat dg + 1)) []) s9)
and query-out:
  Some (query-chunks, s12) ∈
    set-dist (execute (staged-query-program A 0 rounds) s11)
and data-eq:
  data =
    (|staged-trace-root = fr,
     staged-trace-fri-roots = trace-roots,
     staged-trace-fri-challenges = trace-bs,
     staged-trace-final = trace-final,
     staged-alphas = as,
     staged-degree = dg,
     staged-composition-fri-roots = composition-roots,
     staged-composition-fri-challenges = composition-bs,
     staged-composition-final = composition-final,
     staged-query-chunks = query-chunks|)
  unfolding staged-transcript-program-def Let-def
  by (auto elim!: set-dist-bindE split: prod.splits)
have trace-len:
  length trace-roots = ceil-log clength ∧
  length trace-bs = ceil-log clength
proof -
  have len:
    0 + ceil-log clength ≤ length (trace-fri-budgets budgets)
  using wf unfolding staged-budget-wellformed-def by simp
have alignment:
  length trace-roots = ceil-log clength ∧
  length trace-bs = length ([] :: 'f list) + ceil-log clength ∧
  PTranscript s3 = PTranscript s2 @ trace-roots ∧
  PState s3 = foldl concat (PState s2) trace-roots ∧
  s2 ≤ s3 ∧
  PTraceFriCounter s3 = PTraceFriCounter s2 + ceil-log clength ∧
  PCompositionFriCounter s3 = PCompositionFriCounter s2 ∧
  PAlphaCounter s3 = PAlphaCounter s2 ∧
  PQueryCounter s3 = PQueryCounter s2 ∧
  (∀ j < ceil-log clength.
   fmlookup (HashMap s3)
     (TraceFriChallenge (PTraceFriCounter s2 + j)
      (foldl concat (PState s2) (take (Suc j) trace-roots)))) =
   Some (trace-bs ! (length ([] :: 'f list) + j)))
  by (rule staged-trace-fri-program-alignment
      [OF controlled len trace-out])
  then show ?thesis by simp
qed
have alpha-len: length as = length spec
  using staged-alpha-program-alignment[OF alpha-out] by simp
have round-bound:
  ceil-log (to-nat dg + 1) ≤ ceil-log (maxDegree + 1)

```

```

using assert-out unfolding assert-def
by (cases
   $\text{ceil-log } (\text{to-nat } dg + 1) \leq \text{ceil-log } (\text{maxDegree} + 1)$ )
  (auto simp: throw-no-outcome)
have composition-len:
   $\text{length } \text{composition-roots} = \text{ceil-log } (\text{to-nat } dg + 1) \wedge$ 
   $\text{length } \text{composition-bs} = \text{ceil-log } (\text{to-nat } dg + 1)$ 
proof –
  have len:
     $0 + \text{ceil-log } (\text{to-nat } dg + 1) \leq$ 
     $\text{length } (\text{composition-fri-budgets } \text{budgets})$ 
    using wf round-bound unfolding staged-budget-wellformed-def by simp
  have alignment:
     $\text{length } \text{composition-roots} = \text{ceil-log } (\text{to-nat } dg + 1) \wedge$ 
     $\text{length } \text{composition-bs} =$ 
     $\text{length } ([ ] :: 'f \text{ list}) + \text{ceil-log } (\text{to-nat } dg + 1) \wedge$ 
     $P\text{Transcript } s10 = P\text{Transcript } s9 @ \text{composition-roots} \wedge$ 
     $P\text{State } s10 = \text{foldl } \text{concat } (P\text{State } s9) \text{composition-roots} \wedge$ 
     $s9 \leq s10 \wedge$ 
     $P\text{TraceFriCounter } s10 = P\text{TraceFriCounter } s9 \wedge$ 
     $P\text{CompositionFriCounter } s10 =$ 
     $P\text{CompositionFriCounter } s9 + \text{ceil-log } (\text{to-nat } dg + 1) \wedge$ 
     $P\text{AlphaCounter } s10 = P\text{AlphaCounter } s9 \wedge$ 
     $P\text{QueryCounter } s10 = P\text{QueryCounter } s9 \wedge$ 
     $(\forall j < \text{ceil-log } (\text{to-nat } dg + 1).$ 
     $\text{fmlookup } (\text{HashMap } s10)$ 
     $(\text{CompositionFriChallenge } (P\text{CompositionFriCounter } s9 + j)$ 
     $(\text{foldl } \text{concat } (P\text{State } s9) (\text{take } (\text{Suc } j) \text{composition-roots}))) =$ 
     $\text{Some } (\text{composition-bs } ! (\text{length } ([ ] :: 'f \text{ list}) + j)))$ 
    by (rule staged-composition-fri-program-alignment
    [OF controlled len composition-out])
  then show ?thesis by simp
qed
have query-len:
   $\text{length } \text{query-chunks} = \text{rounds}$ 
proof –
  have len:  $0 + \text{rounds} \leq \text{length } (\text{query-opening-budgets } \text{budgets})$ 
    using wf unfolding staged-budget-wellformed-def by simp
  have alignment:
     $\text{length } \text{query-chunks} = \text{rounds} \wedge$ 
     $P\text{Transcript } s12 = P\text{Transcript } s11 @ \text{List.concat } \text{query-chunks} \wedge$ 
     $P\text{State } s12 =$ 
     $\text{foldl } \text{concat } (P\text{State } s11) (\text{List.concat } \text{query-chunks}) \wedge$ 
     $P\text{TraceFriCounter } s12 = P\text{TraceFriCounter } s11 \wedge$ 
     $P\text{CompositionFriCounter } s12 = P\text{CompositionFriCounter } s11 \wedge$ 
     $P\text{AlphaCounter } s12 = P\text{AlphaCounter } s11 \wedge$ 
     $P\text{QueryCounter } s12 = P\text{QueryCounter } s11 + \text{rounds}$ 
    by (rule staged-query-program-alignment[OF controlled len query-out])
  then show ?thesis by simp

```


qed
 show ?thesis
 unfolding data-eq degree-eq
 using trace-len alpha-len composition-len round-bound query-len by simp
 qed

lemma hash-target-program-outcome-extension:
 assumes target: hash-target-program B q m
 and outcome: $\text{Some } (x, t) \in \text{set-dist } (\text{execute } m \ s)$
 shows $s \leq t$
 using hash-target-program-extension[OF target] outcome
 unfolding hash-extension-preserving-def by blast

lemma staged-transcript-program-outcome-query-lookups:
 assumes wf: staged-budget-wellformed budgets
 and controlled: staged-adversary-controlled budgets A
 and outcome:
 $\text{Some } (data, \text{attacker-state}) \in$
 set-dist
 $(\text{execute } (\text{staged-transcript-program } A)$
 $\text{adversary-initial-state})$
 shows
 $\exists \text{raw-idxs query-idxs.}$
 $\text{length raw-idxs} = \text{rounds} \wedge$
 $\text{query-idxs} = \text{map } (\lambda \text{raw. index } (\text{to-nat raw})) \text{ raw-idxs} \wedge$
 $\text{length } (\text{staged-query-chunks } data) = \text{rounds} \wedge$
 $P\text{State attacker-state} =$
 $\text{state-after-query-chunks}$
 $(\text{staged-query-start-hash } data)$
 $(\text{staged-query-chunks } data) \text{ rounds} \wedge$
 $P\text{QueryCounter attacker-state} = \text{rounds} \wedge$
 $(\forall j < \text{rounds.}$
 $\text{fmlookup } (\text{HashMap attacker-state})$
 $(\text{QueryIndexChallenge } j$
 $(\text{state-after-query-chunks}$
 $(\text{staged-query-start-hash } data)$
 $(\text{staged-query-chunks } data) \ j)) =$
 $\text{Some } (\text{raw-idxs } ! \ j)) \wedge$
 $(\forall \text{idx} \in \text{set query-idxs. idx} < \text{clength} * \text{scale})$

proof –
 from outcome **obtain** $fr \ s1 \ s2 \ \text{trace-roots} \ \text{trace-bs} \ s3 \ \text{trace-final} \ s4 \ s5$
 $as \ s6 \ dg \ s7 \ s8 \ s9 \ \text{composition-roots} \ \text{composition-bs} \ s10$
 $\text{composition-final} \ s11 \ s12 \ \text{query-chunks}$ **where**
 root-out:
 $\text{Some } (fr, s1) \in$
 $\text{set-dist } (\text{execute } (\text{trace-root-stage } A) \ \text{adversary-initial-state})$
and record-root:
 $\text{Some } ((), s2) \in \text{set-dist } (\text{execute } (\text{record-staged-message } fr) \ s1)$
and trace-out:

```

    Some ((trace-roots, trace-bs), s3) ∈
      set-dist
        (execute
          (staged-trace-fri-program A 0 (ceil-log clength) []) s2)
  and final-out:
    Some (trace-final, s4) ∈
      set-dist (execute (trace-final-stage A trace-bs) s3)
  and record-final:
    Some ((), s5) ∈
      set-dist (execute (record-staged-message trace-final) s4)
  and alpha-out:
    Some (as, s6) ∈
      set-dist (execute (staged-alpha-program (length spec)) s5)
  and degree-out:
    Some (dg, s7) ∈ set-dist (execute (degree-stage A as) s6)
  and record-degree:
    Some ((), s8) ∈ set-dist (execute (record-staged-message dg) s7)
  and assert-out:
    Some ((), s9) ∈
      set-dist
        (execute
          (assert
            (ceil-log (to-nat dg + 1) ≤
              ceil-log (maxDegree + 1))) s8)
  and composition-out:
    Some ((composition-roots, composition-bs), s10) ∈
      set-dist
        (execute
          (staged-composition-fri-program A dg 0
            (ceil-log (to-nat dg + 1)) []) s9)
  and composition-final-out:
    Some (composition-final, s11) ∈
      set-dist
        (execute (composition-final-stage A dg composition-bs) s10)
  and record-composition-final:
    Some ((), s12) ∈
      set-dist (execute (record-staged-message composition-final) s11)
  and query-out:
    Some (query-chunks, attacker-state) ∈
      set-dist (execute (staged-query-program A 0 rounds) s12)
  and data-eq:
    data =
      (|staged-trace-root = fr,
        staged-trace-fri-roots = trace-roots,
        staged-trace-fri-challenges = trace-bs,
        staged-trace-final = trace-final,
        staged-alphas = as,
        staged-degree = dg,
        staged-composition-fri-roots = composition-roots,

```

```

    staged-composition-fri-challenges = composition-bs,
    staged-composition-final = composition-final,
    staged-query-chunks = query-chunks)
unfolding staged-transcript-program-def Let-def
by (auto elim!: set-dist-bindE split: prod.splits)
have root-controlled:
  controlled-ro-program (trace-root-budget budgets)
    (trace-root-stage A)
  using controlled unfolding staged-adversary-controlled-def by simp
have trace-final-controlled:
  controlled-ro-program (trace-final-budget budgets)
    (trace-final-stage A trace-bs)
  using controlled unfolding staged-adversary-controlled-def by blast
have degree-controlled:
  controlled-ro-program (degree-budget budgets) (degree-stage A as)
  using controlled unfolding staged-adversary-controlled-def by blast
have composition-final-controlled:
  controlled-ro-program (composition-final-budget budgets)
    (composition-final-stage A dg composition-bs)
  using controlled unfolding staged-adversary-controlled-def by blast
have root-state: PState s1 = 0
  using controlled-stage-outcome-fields[OF root-controlled root-out]
  by simp
have s2-eq:
  s2 = s1 (
    PState := concat (PState s1) fr,
    PTranscript := PTranscript s1 @ [fr])
  by (rule record-staged-message-outcome[OF record-root])
have trace-bound:
  0 + ceil-log clength ≤ length (trace-fri-budgets budgets)
  using wf unfolding staged-budget-wellformed-def by simp
have trace-alignment:
  PState s3 = foldl concat (PState s2) trace-roots
  using staged-trace-fri-program-alignment
    [OF controlled trace-bound trace-out]
  by simp
have state-s3:
  PState s3 = foldl concat (concat 0 fr) trace-roots
  using root-state trace-alignment unfolding s2-eq by simp
have state-s4: PState s4 = PState s3
  using controlled-stage-outcome-fields
    [OF trace-final-controlled final-out]
  by simp
have s5-eq:
  s5 = s4 (
    PState := concat (PState s4) trace-final,
    PTranscript := PTranscript s4 @ [trace-final])
  by (rule record-staged-message-outcome[OF record-final])
have alpha-alignment:

```

```

PState s6 = foldl concat (PState s5) as
using staged-alpha-program-alignment[OF alpha-out] by simp
have state-s6:
PState s6 =
  foldl concat
    (concat (foldl concat (concat 0 fr) trace-roots) trace-final)
  as
  using state-s3 state-s4 alpha-alignment unfolding s5-eq by simp
have state-s7: PState s7 = PState s6
  using controlled-stage-outcome-fields[OF degree-controlled degree-out]
  by simp
have s8-eq:
s8 = s7(
  PState := concat (PState s7) dg,
  PTranscript := PTranscript s7 @ [dg])
by (rule record-staged-message-outcome[OF record-degree])
have s9-eq: s9 = s8
  using assert-out unfolding assert-def
  by (cases
    ceil-log (to-nat dg + 1) ≤ ceil-log (maxDegree + 1))
    (auto simp: throw-no-outcome)
have round-bound:
  ceil-log (to-nat dg + 1) ≤ ceil-log (maxDegree + 1)
  using assert-out unfolding assert-def
  by (cases
    ceil-log (to-nat dg + 1) ≤ ceil-log (maxDegree + 1))
    (auto simp: throw-no-outcome)
have composition-bound:
  0 + ceil-log (to-nat dg + 1) ≤
    length (composition-fri-budgets budgets)
  using wf round-bound unfolding staged-budget-wellformed-def by simp
have composition-alignment:
  PState s10 = foldl concat (PState s9) composition-roots
  using staged-composition-fri-program-alignment
    [OF controlled composition-bound composition-out]
  by simp
have state-s10:
PState s10 =
  foldl concat
    (concat
      (foldl concat
        (concat (foldl concat (concat 0 fr) trace-roots) trace-final)
        as)
      dg)
    composition-roots
  using state-s6 state-s7 composition-alignment
  unfolding s8-eq s9-eq by simp
have state-s11: PState s11 = PState s10
  using controlled-stage-outcome-fields

```

```

    [OF composition-final-controlled composition-final-out]
  by simp
have s12-eq:
  s12 = s11 (
    PState := concat (PState s11) composition-final,
    PTranscript := PTranscript s11 @ [composition-final])
  by (rule record-staged-message-outcome[OF record-composition-final])
have state-s12:
  PState s12 =
    concat
      (foldl concat
        (concat
          (foldl concat
            (concat (foldl concat (concat 0 fr) trace-roots) trace-final)
            as)
          dg)
        composition-roots)
      composition-final
  using state-s10 state-s11 unfolding s12-eq by simp
have query-counter-s12: PQueryCounter s12 = 0
proof -
  have root-fields:
    PQueryCounter s1 = PQueryCounter adversary-initial-state
    using controlled-stage-outcome-fields[OF root-controlled root-out]
    by simp
  have trace-counters:
    PQueryCounter s3 = PQueryCounter s2
    using staged-trace-fri-program-alignment
      [OF controlled trace-bound trace-out]
    by simp
  have final-counter: PQueryCounter s4 = PQueryCounter s3
    using controlled-stage-outcome-fields
      [OF trace-final-controlled final-out]
    by simp
  have alpha-counter: PQueryCounter s6 = PQueryCounter s5
    using staged-alpha-program-alignment[OF alpha-out] by simp
  have degree-counter: PQueryCounter s7 = PQueryCounter s6
    using controlled-stage-outcome-fields[OF degree-controlled degree-out]
    by simp
  have comp-counter: PQueryCounter s10 = PQueryCounter s9
    using staged-composition-fri-program-alignment
      [OF controlled composition-bound composition-out]
    by simp
  have comp-final-counter: PQueryCounter s11 = PQueryCounter s10
    using controlled-stage-outcome-fields
      [OF composition-final-controlled composition-final-out]
    by simp
show ?thesis
  using root-fields trace-counters final-counter alpha-counter

```

```

    degree-counter comp-counter comp-final-counter
  unfolding s2-eq s5-eq s8-eq s9-eq s12-eq by simp
qed
have query-bound:  $0 + \text{rounds} \leq \text{length } (\text{query-opening-budgets budgets})$ 
  using wf unfolding staged-budget-wellformed-def by simp
from staged-query-program-outcome-with-raws
  [OF controlled query-bound query-out]
obtain raw-idxs query-idxs where
  len-raw:  $\text{length raw-idxs} = \text{rounds}$ 
  and query-idxs-def:
    query-idxs =  $\text{map } (\lambda \text{raw}. \text{index } (\text{to-nat raw})) \text{ raw-idxs}$ 
  and len-chunks:  $\text{length query-chunks} = \text{rounds}$ 
  and state-attacker:
    PState attacker-state =
      state-after-query-chunks (PState s12) query-chunks rounds
  and query-counter-attacker:
    PQueryCounter attacker-state = PQueryCounter s12 + rounds
  and lookup:
     $\bigwedge j. j < \text{rounds} \implies$ 
      fmlookup (HashMap attacker-state)
        (QueryIndexChallenge (PQueryCounter s12 + j)
          (state-after-query-chunks (PState s12) query-chunks j)) =
        Some (raw-idxs ! j)
  and idx-bound:
     $\forall \text{idx} \in \text{set query-idxs}. \text{idx} < \text{clength} * \text{scale}$ 
  by blast
have start-eq:
  PState s12 = staged-query-start-hash data
  unfolding data-eq staged-query-start-hash-def
    staged-composition-fri-start-hash-def staged-trace-fri-start-hash-def
  using state-s12 by simp
have chunks-eq: staged-query-chunks data = query-chunks
  unfolding data-eq by simp
have state-attacker':
  PState attacker-state =
    state-after-query-chunks
      (staged-query-start-hash data)
      (staged-query-chunks data) rounds
  using state-attacker start-eq chunks-eq by simp
have query-counter-attacker':
  PQueryCounter attacker-state = rounds
  using query-counter-attacker query-counter-s12 by simp
have lookup':
   $\bigwedge j. j < \text{rounds} \implies$ 
    fmlookup (HashMap attacker-state)
      (QueryIndexChallenge j
        (state-after-query-chunks
          (staged-query-start-hash data)
          (staged-query-chunks data) j)) =

```

```

    Some (raw-idxs ! j)
  using lookup start-eq query-counter-s12 chunks-eq by simp
show ?thesis
by (intro exI[of - raw-idxs] exI[of - query-idxs] conjI)
  (use len-raw query-idxs-def chunks-eq len-chunks state-attacker'
    query-counter-attacker' lookup' idx-bound
    in simp-all)
qed

lemma checked-staged-transcript-program-outcome-query-lookups:
  assumes wf: staged-budget-wellformed budgets
  and controlled: staged-adversary-controlled budgets A
  and outcome:
    Some (data, attacker-state) ∈
      set-dist
        (execute (checked-staged-transcript-program A)
          adversary-initial-state)
  shows
    ∃ raw-idxs query-idxs.
      length raw-idxs = rounds ∧
      query-idxs = map (λraw. index (to-nat raw)) raw-idxs ∧
      length (staged-query-chunks data) = rounds ∧
      PState attacker-state =
        state-after-query-chunks
          (staged-query-start-hash data)
          (staged-query-chunks data) rounds ∧
      PQueryCounter attacker-state = rounds ∧
      (∀ j < rounds.
        fmlookup (HashMap attacker-state)
          (QueryIndexChallenge j
            (state-after-query-chunks
              (staged-query-start-hash data)
              (staged-query-chunks data) j)) =
          Some (raw-idxs ! j)) ∧
      (∀ idx ∈ set query-idxs. idx < clength * scale)
proof -
  have unchecked-out:
    Some (data, attacker-state) ∈
      set-dist
        (execute (staged-transcript-program A) adversary-initial-state)
  by (rule checked-staged-transcript-program-outcome-imp-staged-transcript-program
    [OF outcome])
show ?thesis
by (rule staged-transcript-program-outcome-query-lookups
  [OF wf controlled unchecked-out])
qed

lemma checked-staged-transcript-program-query-phaseE:
  assumes wf: staged-budget-wellformed budgets

```

and *controlled*: staged-adversary-controlled budgets A
and *outcome*:
Some ($data, attacker-state$) $\in$
set-dist
(execute (checked-staged-transcript-program A)
adversary-initial-state)
obtains fr *trace-roots* *trace-bs* *trace-final* *as* dg *composition-roots*
composition-bs *composition-final* *query-start* *query-chunks*
where
Some ($query-chunks, attacker-state$) $\in$
set-dist
(execute
*(checked-staged-query-program A *trace-roots* *composition-roots**
0 rounds)
query-start)
 $data =$
(*staged-trace-root* $= fr$,
staged-trace-fri-roots $= trace-roots$,
staged-trace-fri-challenges $= trace-bs$,
staged-trace-final $= trace-final$,
staged-alphas $= as$,
staged-degree $= dg$,
staged-composition-fri-roots $= composition-roots$,
staged-composition-fri-challenges $= composition-bs$,
staged-composition-final $= composition-final$,
staged-query-chunks $= query-chunks$)
PState *query-start* $= staged-query-start-hash$ $data$
PQueryCounter *query-start* $= 0$
proof –
from *outcome* **obtain** fr $s1$ $s2$ *trace-roots* *trace-bs* $s3$ *trace-final* $s4$ $s5$
as $s6$ dg $s7$ $s8$ $s9$ *composition-roots* *composition-bs* $s10$
composition-final $s11$ *query-start* *query-chunks* **where**
root-out:
Some ($fr, s1$) $\in$
set-dist (execute (trace-root-stage A) adversary-initial-state)
and *record-root*:
Some ($()$, $s2$) $\in$ *set-dist (execute (record-staged-message fr) $s1$)*
and *trace-out*:
Some ($(trace-roots, trace-bs), s3$) $\in$
set-dist
(execute
(staged-trace-fri-program A 0 (ceil-log clength) []) $s2$)
and *final-out*:
Some (*trace-final*, $s4$) $\in$
*set-dist (execute (trace-final-stage A *trace-bs*) $s3$)*
and *record-final*:
Some ($()$, $s5$) $\in$
*set-dist (execute (record-staged-message *trace-final*) $s4$)*
and *alpha-out*:


```

    Some (as, s6) ∈
      set-dist (execute (staged-alpha-program (length spec)) s5)
and degree-out:
    Some (dg, s7) ∈ set-dist (execute (degree-stage A as) s6)
and record-degree:
    Some ((), s8) ∈ set-dist (execute (record-staged-message dg) s7)
and assert-out:
    Some ((), s9) ∈
      set-dist
        (execute
          (assert
            (ceil-log (to-nat dg + 1) ≤
              ceil-log (maxDegree + 1))) s8)
and composition-out:
    Some ((composition-roots, composition-bs), s10) ∈
      set-dist
        (execute
          (staged-composition-fri-program A dg 0
            (ceil-log (to-nat dg + 1)) []) s9)
and composition-final-out:
    Some (composition-final, s11) ∈
      set-dist
        (execute (composition-final-stage A dg composition-bs) s10)
and record-composition-final:
    Some ((), query-start) ∈
      set-dist (execute (record-staged-message composition-final) s11)
and query-out:
    Some (query-chunks, attacker-state) ∈
      set-dist
        (execute
          (checked-staged-query-program A trace-roots composition-roots
            0 rounds)
          query-start)
and data-eq:
    data =
      (|staged-trace-root = fr,
        staged-trace-fri-roots = trace-roots,
        staged-trace-fri-challenges = trace-bs,
        staged-trace-final = trace-final,
        staged-alphas = as,
        staged-degree = dg,
        staged-composition-fri-roots = composition-roots,
        staged-composition-fri-challenges = composition-bs,
        staged-composition-final = composition-final,
        staged-query-chunks = query-chunks)
unfolding checked-staged-transcript-program-def Let-def
by (auto elim!: set-dist-bindE split: prod.splits)
have root-controlled:
    controlled-ro-program (trace-root-budget budgets)

```

```

    (trace-root-stage A)
  using controlled unfolding staged-adversary-controlled-def by simp
have trace-final-controlled:
  controlled-ro-program (trace-final-budget budgets)
    (trace-final-stage A trace-bs)
  using controlled unfolding staged-adversary-controlled-def by blast
have degree-controlled:
  controlled-ro-program (degree-budget budgets) (degree-stage A as)
  using controlled unfolding staged-adversary-controlled-def by blast
have composition-final-controlled:
  controlled-ro-program (composition-final-budget budgets)
    (composition-final-stage A dg composition-bs)
  using controlled unfolding staged-adversary-controlled-def by blast
have root-state: PState s1 = 0
  using controlled-stage-outcome-fields[OF root-controlled root-out]
  by simp
have s2-eq:
  s2 = s1(
    PState := concat (PState s1) fr,
    PTranscript := PTranscript s1 @ [fr])
  by (rule record-staged-message-outcome[OF record-root])
have trace-bound:
  0 + ceil-log clength ≤ length (trace-fri-budgets budgets)
  using wf unfolding staged-budget-wellformed-def by simp
have trace-alignment:
  PState s3 = foldl concat (PState s2) trace-roots
  using staged-trace-fri-program-alignment
    [OF controlled trace-bound trace-out]
  by simp
have state-s3:
  PState s3 = foldl concat (concat 0 fr) trace-roots
  using root-state trace-alignment unfolding s2-eq by simp
have state-s4: PState s4 = PState s3
  using controlled-stage-outcome-fields
    [OF trace-final-controlled final-out]
  by simp
have s5-eq:
  s5 = s4(
    PState := concat (PState s4) trace-final,
    PTranscript := PTranscript s4 @ [trace-final])
  by (rule record-staged-message-outcome[OF record-final])
have alpha-alignment:
  PState s6 = foldl concat (PState s5) as
  using staged-alpha-program-alignment[OF alpha-out] by simp
have state-s6:
  PState s6 =
    foldl concat
      (concat (foldl concat (concat 0 fr) trace-roots) trace-final)
    as

```

```

using state-s3 state-s4 alpha-alignment unfolding s5-eq by simp
have state-s7: PState s7 = PState s6
using controlled-stage-outcome-fields[OF degree-controlled degree-out]
by simp
have s8-eq:
  s8 = s7(
    PState := concat (PState s7) dg,
    PTranscript := PTranscript s7 @ [dg])
by (rule record-staged-message-outcome[OF record-degree])
have s9-eq: s9 = s8
using assert-out unfolding assert-def
by (cases
  ceil-log (to-nat dg + 1) ≤ ceil-log (maxDegree + 1))
  (auto simp: throw-no-outcome)
have round-bound:
  ceil-log (to-nat dg + 1) ≤ ceil-log (maxDegree + 1)
using assert-out unfolding assert-def
by (cases
  ceil-log (to-nat dg + 1) ≤ ceil-log (maxDegree + 1))
  (auto simp: throw-no-outcome)
have composition-bound:
  0 + ceil-log (to-nat dg + 1) ≤
  length (composition-fri-budgets budgets)
using wf round-bound unfolding staged-budget-wellformed-def by simp
have composition-alignment:
  PState s10 = foldl concat (PState s9) composition-roots
using staged-composition-fri-program-alignment
  [OF controlled composition-bound composition-out]
by simp
have state-s10:
  PState s10 =
    foldl concat
      (concat
        (foldl concat
          (concat (foldl concat (concat 0 fr) trace-roots) trace-final)
          as)
        dg)
      composition-roots
using state-s6 state-s7 composition-alignment
unfolding s8-eq s9-eq by simp
have state-s11: PState s11 = PState s10
using controlled-stage-outcome-fields
  [OF composition-final-controlled composition-final-out]
by simp
have query-start-eq:
  query-start = s11(
    PState := concat (PState s11) composition-final,
    PTranscript := PTranscript s11 @ [composition-final])
by (rule record-staged-message-outcome[OF record-composition-final])

```

```

have state-query-start:
  PState query-start =
    concat
      (foldl concat
        (concat
          (foldl concat
            (concat (foldl concat (concat 0 fr) trace-roots) trace-final)
            as)
          dg)
        composition-roots)
      composition-final
    using state-s10 state-s11 unfolding query-start-eq by simp
have query-counter-start: PQueryCounter query-start = 0
proof –
  have root-fields:
    PQueryCounter s1 = PQueryCounter adversary-initial-state
    using controlled-stage-outcome-fields[OF root-controlled root-out]
    by simp
  have trace-counters:
    PQueryCounter s3 = PQueryCounter s2
    using staged-trace-fri-program-alignment
      [OF controlled trace-bound trace-out]
    by simp
  have final-counter: PQueryCounter s4 = PQueryCounter s3
    using controlled-stage-outcome-fields
      [OF trace-final-controlled final-out]
    by simp
  have alpha-counter: PQueryCounter s6 = PQueryCounter s5
    using staged-alpha-program-alignment[OF alpha-out] by simp
  have degree-counter: PQueryCounter s7 = PQueryCounter s6
    using controlled-stage-outcome-fields[OF degree-controlled degree-out]
    by simp
  have comp-counter: PQueryCounter s10 = PQueryCounter s9
    using staged-composition-fri-program-alignment
      [OF controlled composition-bound composition-out]
    by simp
  have comp-final-counter: PQueryCounter s11 = PQueryCounter s10
    using controlled-stage-outcome-fields
      [OF composition-final-controlled composition-final-out]
    by simp
  show ?thesis
    using root-fields trace-counters final-counter alpha-counter
      degree-counter comp-counter comp-final-counter
    unfolding s2-eq s5-eq s8-eq s9-eq query-start-eq by simp
qed
have start-eq:
  PState query-start = staged-query-start-hash data
  unfolding data-eq staged-query-start-hash-def
    staged-composition-fri-start-hash-def staged-trace-fri-start-hash-def

```

```

    using state-query-start by simp
  show ?thesis
  by (rule that[OF query-out data-eq start-eq query-counter-start])
qed

lemma checked-staged-security-with-data-state-accepted-shape-query-keys:
  assumes wf: staged-budget-wellformed budgets
  and controlled: staged-adversary-controlled budgets A
  and outcome:
    Some (((data, attacker-state), result), final-state) ∈
      set-dist (execute
        (checked-staged-security-experiment-with-data-state A)
        adversary-initial-state)
  and shape:
    accepted-transcript-shape
      (verifier-state-from-adversary attacker-state
        (staged-proof-transcript data))
    (Some (result, final-state)) as query-idxs
  shows
    as = staged-alphas data ∧
    (∃ raw-idxs.
      length raw-idxs = rounds ∧
      query-idxs = map (λraw. index (to-nat raw)) raw-idxs ∧
      (∀ i < rounds.
        fmllookup (HashMap final-state)
          (QueryIndexChallenge i
            (state-after-query-chunks
              (staged-query-start-hash data)
              (staged-query-chunks data) i)) =
          Some (raw-idxs ! i)) ∧
      (∀ idx ∈ set query-idxs. idx < clength * scale))
  proof -
    let ?s = verifier-state-from-adversary attacker-state
      (staged-proof-transcript data)
    from checked-staged-security-experiment-with-data-state-outcomeE
      [OF outcome]
    have builder:
      Some (data, attacker-state) ∈
        set-dist (execute (checked-staged-transcript-program A)
          adversary-initial-state)
    and verifier:
      Some (result, final-state) ∈
        set-dist (execute verify-monad ?s)
    by blast+
    have staged-header:
      verifier-header-transcript ?s
      (staged-trace-root data)
      (staged-trace-fri-roots data)
      (staged-trace-final data)

```

```

    (staged-alphas data)
    (staged-degree data)
    (staged-composition-fri-roots data)
    (staged-composition-final data)
    (List.concat (staged-query-chunks data))
  by (rule checked-staged-transcript-program-outcome-header-transcript
      [OF wf controlled builder])
from accepted-transcript-shape-query-chunksE[OF shape]
obtain result' final-state' fr f-fri-roots f-final dg
  composition-fri-roots final rest raw-idxs query-chunks trailing
where out-eq:
  Some (result, final-state) = Some (result', final-state')
and header:
  verifier-header-transcript ?s fr f-fri-roots f-final as dg
  composition-fri-roots final rest
and len-raw: length raw-idxs = rounds
and query-idxs-eq:
  query-idxs = map (λraw. index (to-nat raw)) raw-idxs
and len-chunks: length query-chunks = rounds
and concat-chunks: List.concat query-chunks @ trailing = rest
and chunk-shape:
  ∧ i. i < rounds ⇒
    verifier-query-round-chunk (query-idxs ! i)
    f-fri-roots composition-fri-roots (query-chunks ! i)
and lookup:
  ∧ i. i < rounds ⇒
    fmlookup (HashMap final-state')
    (QueryIndexChallenge (PQueryCounter ?s + i)
      (state-after-query-chunks
        (verifier-header-state ?s fr f-fri-roots f-final as dg
          composition-fri-roots final)
        query-chunks i)) =
    Some (raw-idxs ! i)
and idx-bound:
  ∀ idx ∈ set query-idxs. idx < clength * scale
by blast
have header-eq:
  fr = staged-trace-root data ∧
  f-fri-roots = staged-trace-fri-roots data ∧
  f-final = staged-trace-final data ∧
  as = staged-alphas data ∧
  dg = staged-degree data ∧
  composition-fri-roots = staged-composition-fri-roots data ∧
  final = staged-composition-final data ∧
  rest = List.concat (staged-query-chunks data)
using verifier-header-transcript-unique[OF staged-header header]
by simp
have final-state-eq: final-state' = final-state
using out-eq by simp

```

```

have header-state-eq:
  verifier-header-state ?s fr f-fri-roots f-final as dg
    composition-fri-roots final =
    staged-query-start-hash data
using header-eq
unfolding verifier-header-state-def verifier-header-messages-def
  staged-query-start-hash-def staged-composition-fri-start-hash-def
  staged-trace-fri-start-hash-def
by simp
from checked-staged-transcript-program-query-chunks-match-verifier-lengths
  [OF builder]
obtain staged-query-idxs where
  staged-match:
    staged-query-chunks-match-verifier-lengths data staged-query-idxs
by blast
have query-idxs-len: length query-idxs = rounds
  using query-idxs-eq len-raw by simp
have staged-match-query-idxs:
  staged-query-chunks-match-verifier-lengths data query-idxs
by (rule staged-query-chunks-match-verifier-lengths-transfer
  [OF staged-match query-idxs-len])
have parser-chunk-len:
 $\bigwedge i. i < \text{rounds} \implies$ 
  length (query-chunks ! i) =
    verifier-query-round-transcript-length (query-idxs ! i)
    (staged-trace-fri-roots data)
    (staged-composition-fri-roots data)
proof -
  fix i
  assume i-bound: i < rounds
  show
    length (query-chunks ! i) =
      verifier-query-round-transcript-length (query-idxs ! i)
      (staged-trace-fri-roots data)
      (staged-composition-fri-roots data)
    using verifier-query-round-chunk-length[OF chunk-shape[OF i-bound]]
    header-eq by simp
qed
have concat-staged:
  List.concat query-chunks @ trailing =
    List.concat (staged-query-chunks data)
using concat-chunks header-eq by simp
have query-chunks-eq:
  query-chunks = staged-query-chunks data  $\wedge$  trailing = []
by (rule query-chunks-eq-staged-if-matching-lengths
  [OF len-chunks concat-staged parser-chunk-len
  staged-match-query-idxs])
have lookup-staged:
 $\bigwedge i. i < \text{rounds} \implies$ 

```

```

    fmlookup (HashMap final-state)
      (QueryIndexChallenge i
        (state-after-query-chunks
          (staged-query-start-hash data)
          (staged-query-chunks data) i)) =
        Some (raw-idxs ! i)
  proof -
    fix i
    assume i-bound: i < rounds
    have lookup-i:
      fmlookup (HashMap final-state')
        (QueryIndexChallenge (PQueryCounter ?s + i)
          (state-after-query-chunks
            (verifier-header-state ?s fr f-fri-roots f-final as dg
              composition-fri-roots final)
            query-chunks i)) =
          Some (raw-idxs ! i)
    by (rule lookup[OF i-bound])
  show
    fmlookup (HashMap final-state)
      (QueryIndexChallenge i
        (state-after-query-chunks
          (staged-query-start-hash data)
          (staged-query-chunks data) i)) =
        Some (raw-idxs ! i)
    using lookup-i final-state-eq header-state-eq query-chunks-eq by simp
  qed
  show ?thesis
    using header-eq len-raw query-idxs-eq lookup-staged idx-bound by blast
  qed

lemma checked-staged-security-with-data-state-accepted-shape-query-set-hit:
  assumes wf: staged-budget-wellformed budgets
  and controlled: staged-adversary-controlled budgets A
  and outcome:
    Some (((data, attacker-state), result), final-state) ∈
      set-dist (execute
        (checked-staged-security-experiment-with-data-state A)
        adversary-initial-state)
  and shape:
    accepted-transcript-shape
      (verifier-state-from-adversary attacker-state
        (staged-proof-transcript data))
      (Some (result, final-state)) as query-idxs
  and hit: ∃ i < rounds. query-idxs ! i ∈ B
  shows staged-transcript-query-index-set-hit B (Some (data, attacker-state))
  proof -
    let ?s = verifier-state-from-adversary attacker-state
      (staged-proof-transcript data)

```



```

from checked-staged-security-experiment-with-data-state-outcomeE
  [OF outcome]
have builder:
  Some (data, attacker-state) ∈
    set-dist (execute (checked-staged-transcript-program A)
      adversary-initial-state)
and verifier:
  Some (result, final-state) ∈
    set-dist (execute verify-monad ?s)
by blast+
from checked-staged-transcript-program-outcome-query-lookups
  [OF wf controlled builder]
obtain attacker-raw-idxs where
  lookup-attacker:
     $\forall j < \text{rounds}.$ 
      fmlookup (HashMap attacker-state)
        (QueryIndexChallenge j
          (state-after-query-chunks (staged-query-start-hash data)
            (staged-query-chunks data j))) =
        Some (attacker-raw-idxs ! j)
by blast
from checked-staged-security-with-data-state-accepted-shape-query-keys
  [OF wf controlled outcome shape]
obtain raw-idxs where
  len-raw: length raw-idxs = rounds
and query-idxs-eq:
  query-idxs = map ( $\lambda \text{raw}. \text{index (to-nat raw)}$ ) raw-idxs
and lookup:
   $\bigwedge i. i < \text{rounds} \implies$ 
    fmlookup (HashMap final-state)
      (QueryIndexChallenge i
        (state-after-query-chunks (staged-query-start-hash data)
          (staged-query-chunks data i))) =
      Some (raw-idxs ! i)
by blast
from hit obtain i where i-bound: i < rounds
and query-hit: query-idxs ! i ∈ B
by blast
have raw-i: i < length raw-idxs
using len-raw i-bound by simp
have raw-hit: index (to-nat (raw-idxs ! i)) ∈ B
using query-hit query-idxs-eq raw-i by (simp add: nth-map)
have ext: ?s ≤ final-state
by (rule verify-monad-hash-extends[OF verifier])
have lookup-initial:
  fmlookup (HashMap ?s)
    (QueryIndexChallenge i
      (state-after-query-chunks (staged-query-start-hash data)
        (staged-query-chunks data i))) =

```

```

    Some (attacker-raw-idxs ! i)
  using lookup-attacker i-bound by simp
have lookup-final-from-initial:
  fmlookup (HashMap final-state)
    (QueryIndexChallenge i
      (state-after-query-chunks (staged-query-start-hash data)
        (staged-query-chunks data) i)) =
    Some (attacker-raw-idxs ! i)
  by (rule hash-extension-lookup[OF lookup-initial ext])
have raw-eq: attacker-raw-idxs ! i = raw-idxs ! i
  using lookup-final-from-initial lookup[OF i-bound] by simp
have attacker-raw-hit: index (to-nat (attacker-raw-idxs ! i)) ∈ B
  using raw-hit raw-eq by simp
have lookup-attacker-i:
  fmlookup (HashMap attacker-state)
    (QueryIndexChallenge i
      (state-after-query-chunks (staged-query-start-hash data)
        (staged-query-chunks data) i)) =
    Some (attacker-raw-idxs ! i)
  using lookup-attacker i-bound by simp
show ?thesis
  unfolding staged-transcript-query-index-set-hit-def
  using i-bound lookup-attacker-i attacker-raw-hit by auto
qed

lemma staged-security-with-data-query-index-set-bound-from-transcript:
  assumes wf: staged-budget-wellformed budgets
  and controlled: staged-adversary-controlled budgets A
  and transcript-bound:
    wp-event (staged-transcript-program A)
      (staged-transcript-query-index-set-hit B)
    adversary-initial-state ≤ C
  shows
    wp-event (staged-security-experiment-with-data A)
      (staged-security-with-data-query-index-set-hit B)
    adversary-initial-state ≤ C
  unfolding staged-security-experiment-with-data-def
proof (rule wp-event-bind-bound-by-head-event[OF transcript-bound])
  show staged-security-with-data-query-index-set-hit B None ⇒
    staged-transcript-query-index-set-hit B None
  unfolding staged-security-with-data-query-index-set-hit-def
    staged-transcript-query-index-set-hit-def
  by simp
next
fix data attacker-state out
assume builder:
  Some (data, attacker-state) ∈
    set-dist
      (execute (staged-transcript-program A)

```

```

      adversary-initial-state)
and cont:
  out ∈
    set-dist
      (execute
        (get  $\gg$ 
          ( $\lambda s$ . put
            (verifier-state-from-adversary s
              (staged-proof-transcript data))  $\gg$ 
            ( $\lambda$ -. verify-monad  $\gg$ 
              ( $\lambda$ result. return (data, result))))))
        attacker-state)
    and hit: staged-security-with-data-query-index-set-hit B out
from staged-transcript-program-outcome-query-lookups
  [OF wf controlled builder]
obtain raw-idxs query-idxs where
  len-raw: length raw-idxs = rounds
and lookup-attacker:
   $\forall j < \text{rounds}$ .
    fmlookup (HashMap attacker-state)
      (QueryIndexChallenge j
        (state-after-query-chunks
          (staged-query-start-hash data)
          (staged-query-chunks data) j)) =
        Some (raw-idxs ! j)
by blast
from hit obtain data' result final-state i raw where
  out-eq: out = Some ((data', result), final-state)
and i-bound: i < rounds
and lookup-final:
  fmlookup (HashMap final-state)
    (QueryIndexChallenge i
      (state-after-query-chunks
        (staged-query-start-hash data')
        (staged-query-chunks data') i)) =
        Some raw
and raw-hit: index (to-nat raw) ∈ B
unfolding staged-security-with-data-query-index-set-hit-def
by (auto split: option.splits prod.splits)
from cont[unfolded out-eq] obtain verifier-state verifier-result where
  verifier:
    Some (verifier-result, final-state) ∈
      set-dist
        (execute verify-monad
          (verifier-state-from-adversary attacker-state
            (staged-proof-transcript data)))
    and data'-eq: data' = data
by (auto elim!: set-dist-bindE)
have ext:

```

```

    verifier-state-from-adversary attacker-state
      (staged-proof-transcript data) ≤ final-state
  by (rule verify-monad-hash-extends[OF verifier])
have lookup-initial:
  fmlookup
    (HashMap
      (verifier-state-from-adversary attacker-state
        (staged-proof-transcript data)))
    (QueryIndexChallenge i
      (state-after-query-chunks
        (staged-query-start-hash data)
        (staged-query-chunks data) i)) =
      Some (raw-idxs ! i)
  using lookup-attacker i-bound by simp
have lookup-final-from-initial:
  fmlookup (HashMap final-state)
    (QueryIndexChallenge i
      (state-after-query-chunks
        (staged-query-start-hash data)
        (staged-query-chunks data) i)) =
      Some (raw-idxs ! i)
  by (rule hash-extension-lookup[OF lookup-initial ext])
have raw-eq: raw = raw-idxs ! i
  using lookup-final lookup-final-from-initial data'-eq by simp
have lookup-attacker-i:
  fmlookup (HashMap attacker-state)
    (QueryIndexChallenge i
      (state-after-query-chunks
        (staged-query-start-hash data)
        (staged-query-chunks data) i)) =
      Some (raw-idxs ! i)
  using lookup-attacker i-bound by simp
have raw-idxs-hit: index (to-nat (raw-idxs ! i)) ∈ B
  using raw-hit raw-eq by simp
show staged-transcript-query-index-set-hit B
  (Some (data, attacker-state))
  unfolding staged-transcript-query-index-set-hit-def
  using i-bound lookup-attacker-i raw-idxs-hit by auto
qed

```

lemma *checked-staged-security-with-data-query-index-set-bound-from-transcript:*
assumes *wf: staged-budget-wellformed budgets*
and *controlled: staged-adversary-controlled budgets A*
and *transcript-bound:*
 wp-event (checked-staged-transcript-program A)
 (staged-transcript-query-index-set-hit B)
 adversary-initial-state ≤ C
shows
 wp-event (checked-staged-security-experiment-with-data A)

```

    (staged-security-with-data-query-index-set-hit B)
    adversary-initial-state ≤ C
unfolding checked-staged-security-experiment-with-data-def
proof (rule wp-event-bind-bound-by-head-event[OF transcript-bound])
show staged-security-with-data-query-index-set-hit B None ⇒
    staged-transcript-query-index-set-hit B None
unfolding staged-security-with-data-query-index-set-hit-def
    staged-transcript-query-index-set-hit-def
by simp
next
fix data attacker-state out
assume builder:
    Some (data, attacker-state) ∈
        set-dist
            (execute (checked-staged-transcript-program A)
                adversary-initial-state)
and cont:
    out ∈
        set-dist
            (execute
                (get >>=
                    (λs. put
                        (verifier-state-from-adversary s
                            (staged-proof-transcript data)) >>=
                            (λ-. verify-monad >>=
                                (λresult. return (data, result))))))
                    attacker-state)
and hit: staged-security-with-data-query-index-set-hit B out
from checked-staged-transcript-program-outcome-query-lookups
    [OF wf controlled builder]
obtain raw-idxs query-idxs where
    len-raw: length raw-idxs = rounds
and lookup-attacker:
    ∀ j < rounds.
        fmllookup (HashMap attacker-state)
            (QueryIndexChallenge j
                (state-after-query-chunks
                    (staged-query-start-hash data)
                    (staged-query-chunks data) j)) =
            Some (raw-idxs ! j)
by blast
from hit obtain data' result final-state i raw where
    out-eq: out = Some ((data', result), final-state)
and i-bound: i < rounds
and lookup-final:
    fmllookup (HashMap final-state)
        (QueryIndexChallenge i
            (state-after-query-chunks
                (staged-query-start-hash data')

```

```

      (staged-query-chunks data' i)) =
    Some raw
  and raw-hit: index (to-nat raw) ∈ B
  unfolding staged-security-with-data-query-index-set-hit-def
  by (auto split: option.splits prod.splits)
  from cont[unfolded out-eq] obtain verifier-state verifier-result where
  verifier:
    Some (verifier-result, final-state) ∈
      set-dist
        (execute verify-monad
          (verifier-state-from-adversary attacker-state
            (staged-proof-transcript data)))
  and data'-eq: data' = data
  by (auto elim!: set-dist-bindE)
  have ext:
    verifier-state-from-adversary attacker-state
      (staged-proof-transcript data) ≤ final-state
  by (rule verify-monad-hash-extends[OF verifier])
  have lookup-initial:
    fmlookup
      (HashMap
        (verifier-state-from-adversary attacker-state
          (staged-proof-transcript data)))
      (QueryIndexChallenge i
        (state-after-query-chunks
          (staged-query-start-hash data)
          (staged-query-chunks data) i)) =
      Some (raw-idxs ! i)
  using lookup-attacker i-bound by simp
  have lookup-final-from-initial:
    fmlookup (HashMap final-state)
      (QueryIndexChallenge i
        (state-after-query-chunks
          (staged-query-start-hash data)
          (staged-query-chunks data) i)) =
      Some (raw-idxs ! i)
  by (rule hash-extension-lookup[OF lookup-initial ext])
  have raw-eq: raw = raw-idxs ! i
  using lookup-final lookup-final-from-initial data'-eq by simp
  have lookup-attacker-i:
    fmlookup (HashMap attacker-state)
      (QueryIndexChallenge i
        (state-after-query-chunks
          (staged-query-start-hash data)
          (staged-query-chunks data) i)) =
      Some (raw-idxs ! i)
  using lookup-attacker i-bound by simp
  have raw-idxs-hit: index (to-nat (raw-idxs ! i)) ∈ B
  using raw-hit raw-eq by simp

```

```

show staged-transcript-query-index-set-hit B
  (Some (data, attacker-state))
unfolding staged-transcript-query-index-set-hit-def
using i-bound lookup-attacker-i raw-idxs-hit by auto
qed

lemma staged-transcript-query-index-set-hit-imp-target-hit:
assumes hit: staged-transcript-query-index-set-hit B out
shows
  hash-new-output-hit-event (query-index-raw-preimage B)
  adversary-initial-state out
proof (cases out)
case None
then show ?thesis
  using hit unfolding staged-transcript-query-index-set-hit-def by simp
next
case (Some result)
then obtain data t where out-eq: out = Some (data, t)
  by (cases result) simp
from hit[unfolded out-eq staged-transcript-query-index-set-hit-def]
obtain i raw where lookup:
  fmlookup (HashMap t)
  (QueryIndexChallenge i
   (state-after-query-chunks
    (staged-query-start-hash data)
    (staged-query-chunks data) i)) =
    Some raw
  and raw-in: raw  $\in$  query-index-raw-preimage B
  unfolding query-index-raw-preimage-def by auto
have new-hit:
  hash-map-new-output-hit (query-index-raw-preimage B)
  adversary-initial-state t
  unfolding hash-map-new-output-hit-def
  using lookup raw-in by auto
show ?thesis
  unfolding out-eq hash-new-output-hit-event-def
  using new-hit by simp
qed

lemma staged-transcript-query-index-set-hit-bound-from-target-program:
assumes target:
  hash-target-program (query-index-raw-preimage B) q m
shows
  wp-event m (staged-transcript-query-index-set-hit B)
  adversary-initial-state  $\leq$ 
  staged-phase-target-error (query-index-raw-preimage B) q
proof –
have event-mono:
  wp-event m (staged-transcript-query-index-set-hit B)

```

$adversary-initial-state \leq$
 $wp-event\ m$
 $(hash-new-output-hit-event\ (query-index-raw-preimage\ B)$
 $\quad adversary-initial-state)$
 $adversary-initial-state$
by (rule $wp-event-mono$)
 $(rule\ staged-transcript-query-index-set-hit-imp-target-hit)$
have $target-bound$:
 $wp-event\ m$
 $(hash-new-output-hit-event\ (query-index-raw-preimage\ B)$
 $\quad adversary-initial-state)$
 $adversary-initial-state \leq$
 $staged-phase-target-error\ (query-index-raw-preimage\ B)\ q$
using $target$ **unfolding** $hash-target-program-def\ hash-target-budget-def$
 $staged-phase-target-error-def$ **by** $blast$
show $?thesis$
by (rule $order-trans[OF\ event-mono\ target-bound]$)
qed

lemma $staged-transcript-query-index-set-hit-bound$:
assumes wf : $staged-budget-wellformed\ budgets$
and $controlled$: $staged-adversary-controlled\ budgets\ A$
shows
 $wp-event\ (staged-transcript-program\ A)$
 $(staged-transcript-query-index-set-hit\ B)$
 $adversary-initial-state \leq$
 $staged-phase-target-error\ (query-index-raw-preimage\ B)$
 $(staged-attacker-query-budget\ budgets +$
 $\quad staged-challenge-query-budget)$
by (rule $staged-transcript-query-index-set-hit-bound-from-target-program$)
 $(rule\ hash-target-program-staged-transcript-program[OF\ wf\ controlled])$

lemma $checked-staged-transcript-query-index-set-hit-bound-from-target-program$:
assumes $target$:
 $hash-target-program\ (query-index-raw-preimage\ B)\ q$
 $(checked-staged-transcript-program\ A)$
shows
 $wp-event\ (checked-staged-transcript-program\ A)$
 $(staged-transcript-query-index-set-hit\ B)$
 $adversary-initial-state \leq$
 $staged-phase-target-error\ (query-index-raw-preimage\ B)\ q$
by (rule $staged-transcript-query-index-set-hit-bound-from-target-program$
 $[OF\ target]$)

lemma $checked-staged-transcript-query-index-set-hit-bound$:
assumes wf : $staged-budget-wellformed\ budgets$
and $controlled$: $staged-adversary-controlled\ budgets\ A$
shows
 $wp-event\ (checked-staged-transcript-program\ A)$

(staged-transcript-query-index-set-hit B)
 adversary-initial-state $\leq$
 staged-phase-target-error (query-index-raw-preimage B)
 (staged-attacker-query-budget budgets +
 staged-challenge-query-budget)
by (rule checked-staged-transcript-query-index-set-hit-bound-from-target-program)
 (rule hash-target-program-checked-staged-transcript-program
 [OF wf controlled])

lemma staged-security-with-data-query-index-set-hit-bound:
assumes wf: staged-budget-wellformed budgets
and controlled: staged-adversary-controlled budgets A
shows
 wp-event (staged-security-experiment-with-data A)
 (staged-security-with-data-query-index-set-hit B)
 adversary-initial-state $\leq$
 staged-phase-target-error (query-index-raw-preimage B)
 (staged-attacker-query-budget budgets +
 staged-challenge-query-budget)
by (rule staged-security-with-data-query-index-set-bound-from-transcript
 [OF wf controlled
 staged-transcript-query-index-set-hit-bound[OF wf controlled]])

lemma checked-staged-security-with-data-query-index-set-hit-bound-from-target-program:
assumes wf: staged-budget-wellformed budgets
and controlled: staged-adversary-controlled budgets A
and target:
 hash-target-program (query-index-raw-preimage B) q
 (checked-staged-transcript-program A)
shows
 wp-event (checked-staged-security-experiment-with-data A)
 (staged-security-with-data-query-index-set-hit B)
 adversary-initial-state $\leq$
 staged-phase-target-error (query-index-raw-preimage B) q
by (rule checked-staged-security-with-data-query-index-set-bound-from-transcript
 [OF wf controlled
 checked-staged-transcript-query-index-set-hit-bound-from-target-program
 [OF target]])

lemma checked-staged-security-with-data-query-index-set-hit-bound:
assumes wf: staged-budget-wellformed budgets
and controlled: staged-adversary-controlled budgets A
shows
 wp-event (checked-staged-security-experiment-with-data A)
 (staged-security-with-data-query-index-set-hit B)
 adversary-initial-state $\leq$
 staged-phase-target-error (query-index-raw-preimage B)
 (staged-attacker-query-budget budgets +
 staged-challenge-query-budget)

```

    by (rule checked-staged-security-with-data-query-index-set-hit-bound-from-target-program
        [OF wf controlled
         hash-target-program-checked-staged-transcript-program
         [OF wf controlled]])

end

end

theory Staged-Security-Experiment-Checked
  imports Staged-Security-Experiment-Checked-Prefix
begin

Final checked staged query-bad and collision bounds.

context soundness
begin

lemma query-bad-imp-query-rounds-sampling-success-hit:
  assumes bad: query-bad s out
  shows
     $\exists$  trace-table composition-table as.
      query-rounds-any-index-set-hit s
        (query-sampling-success-space trace-table composition-table as)
        rounds out
  proof —
    have round-hit:
      query-index-round-set-hit s query-sampling-success-space out
    by (rule query-bad-imp-query-index-round-set-hit[OF bad])
    then obtain i where hit-at:
      query-index-round-set-hit-at s query-sampling-success-space i out
    unfolding query-index-round-set-hit-def by blast
    show ?thesis
      by (rule query-index-round-set-hit-at-imp-query-rounds-any-index-set-hit
          [OF hit-at])
  qed

lemma query-bad-imp-query-header-sampling-success-hit:
  assumes outcome: out  $\in$  set-dist (execute verify-monad s)
  and bad: query-bad s out
  shows
    query-header-rounds-any-index-set-hit s
      (query-header-supported-union-good-sets s query-sampling-success-space)
      out
  proof —
    have round-hit:
      query-index-round-set-hit s query-sampling-success-space out
    by (rule query-bad-imp-query-index-round-set-hit[OF bad])
    show ?thesis

```

```

    by (rule query-index-round-set-hit-imp-query-header-supported-rounds-hit
        [OF outcome round-hit])
qed

lemma staged-security-with-data-query-bad-imp-sampling-hit:
  assumes bad: staged-security-with-data-query-bad-hit out
  shows staged-security-with-data-query-sampling-hit out
proof (cases out)
  case None
  then show ?thesis
    using bad unfolding staged-security-with-data-query-bad-hit-def by simp
next
  case (Some result)
  then obtain data verifier-result final-state where out-eq:
    out = Some ((data, verifier-result), final-state)
    by (cases result, auto split: prod.splits)
  from bad[unfolded out-eq staged-security-with-data-query-bad-hit-def]
  obtain s where query-bad:
    query-bad s (Some (verifier-result, final-state))
    by auto
  from query-bad-imp-query-rounds-sampling-success-hit[OF query-bad]
  obtain trace-table composition-table as where hit:
    query-rounds-any-index-set-hit s
    (query-sampling-success-space trace-table composition-table as)
    rounds (Some (verifier-result, final-state))
    by blast
  have witness:
     $\exists s$  trace-table composition-table as.
    query-rounds-any-index-set-hit s
    (query-sampling-success-space trace-table composition-table as)
    rounds (Some (verifier-result, final-state))
    by (intro exI[of - s] exI[of - trace-table]
        exI[of - composition-table] exI[of - as])
        (rule hit)
  show ?thesis
    unfolding out-eq staged-security-with-data-query-sampling-hit-def
    using witness by simp
qed

```

```

lemma staged-security-with-data-query-bad-imp-header-sampling-hit:
  assumes bad: staged-security-with-data-query-bad-hit out
  shows staged-security-with-data-query-header-sampling-hit out
proof (cases out)
  case None
  then show ?thesis
    using bad unfolding staged-security-with-data-query-bad-hit-def by simp
next
  case (Some result)
  then obtain data verifier-result final-state where out-eq:

```

```

    out = Some ((data, verifier-result), final-state)
  by (cases result, auto split: prod.splits)
from bad[unfolded out-eq staged-security-with-data-query-bad-hit-def]
obtain s where outcome:
  Some (verifier-result, final-state) ∈
    set-dist (execute verify-monad s)
and query-bad:
  query-bad s (Some (verifier-result, final-state))
by auto
have hit:
  query-header-rounds-any-index-set-hit s
    (query-header-supported-union-good-sets s query-sampling-success-space)
    (Some (verifier-result, final-state))
  by (rule query-bad-imp-query-header-sampling-success-hit
    [OF outcome query-bad])
then show ?thesis
  unfolding out-eq staged-security-with-data-query-header-sampling-hit-def
  by auto
qed

```

```

lemma staged-security-with-data-state-query-bad-imp-header-sampling-hit:
  assumes bad: staged-security-with-data-state-query-bad-hit out
  shows staged-security-with-data-state-query-header-sampling-hit out
proof (cases out)
  case None
  then show ?thesis
    using bad
    unfolding staged-security-with-data-state-query-bad-hit-def by simp
next
  case (Some result)
  then obtain data attacker-state verifier-result final-state where out-eq:
    out = Some (((data, attacker-state), verifier-result), final-state)
  by (cases result, auto split: prod.splits)
  let ?s =
    verifier-state-from-adversary attacker-state
      (staged-proof-transcript data)
  from bad[unfolded out-eq staged-security-with-data-state-query-bad-hit-def]
  have outcome:
    Some (verifier-result, final-state) ∈
      set-dist (execute verify-monad ?s)
  and query-bad:
    query-bad ?s (Some (verifier-result, final-state))
  by (simp-all add: Let-def)
  have hit:
    query-header-rounds-any-index-set-hit ?s
      (query-header-supported-union-good-sets ?s query-sampling-success-space)
      (Some (verifier-result, final-state))
  by (rule query-bad-imp-query-header-sampling-success-hit
    [OF outcome query-bad])

```

```

then show ?thesis
  unfolding out-eq
    staged-security-with-data-state-query-header-sampling-hit-def
  by simp
qed

lemma checked-staged-security-with-data-state-query-bad-imp-header-key-hit-on-support:
  assumes wf: staged-budget-wellformed budgets
  and controlled: staged-adversary-controlled budgets A
  and support:
    out ∈
      set-dist
        (execute
          (checked-staged-security-experiment-with-data-state A)
          adversary-initial-state)
  and bad: staged-security-with-data-state-query-bad-hit out
  shows staged-security-with-data-state-query-header-key-hit out
proof (cases out)
  case None
  then show ?thesis
    using bad unfolding staged-security-with-data-state-query-bad-hit-def
    by simp
next
  case (Some result-pack)
  then obtain data attacker-state result final-state where out-eq:
    out = Some (((data, attacker-state), result), final-state)
  by (cases result-pack, auto split: prod.splits)
  let ?s =
    verifier-state-from-adversary attacker-state
      (staged-proof-transcript data)
  have support-some:
    Some (((data, attacker-state), result), final-state) ∈
      set-dist
        (execute
          (checked-staged-security-experiment-with-data-state A)
          adversary-initial-state)
  using support unfolding out-eq .
from checked-staged-security-experiment-with-data-state-outcomeE
  [OF support-some]
have builder:
  Some (data, attacker-state) ∈
    set-dist
      (execute (checked-staged-transcript-program A)
        adversary-initial-state)
  and verifier:
    Some (result, final-state) ∈ set-dist (execute verify-monad ?s)
  by blast+
have staged-header:
  verifier-header-transcript ?s

```

```

    (staged-trace-root data)
    (staged-trace-fri-roots data)
    (staged-trace-final data)
    (staged-alphas data)
    (staged-degree data)
    (staged-composition-fri-roots data)
    (staged-composition-final data)
    (List.concat (staged-query-chunks data))
  by (rule checked-staged-transcript-program-outcome-header-transcript
      [OF wf controlled builder])
from bad[unfolded out-eq
    staged-security-with-data-state-query-bad-hit-def Let-def]
have query-bad: query-bad ?s (Some (result, final-state))
  by simp
from query-bad-imp-query-index-round-set-hit[OF query-bad]
obtain i trace-table composition-table as query-idxs where
  bound:
    accepted-with-bound-tables ?s (Some (result, final-state))
    trace-table composition-table as query-idxs
and i-bound: i < rounds
and query-hit:
  query-idxs ! i ∈
    query-sampling-success-space trace-table composition-table as
unfolding query-index-round-set-hit-def
  query-index-round-set-hit-at-def
by blast
have shape:
  accepted-transcript-shape ?s (Some (result, final-state))
  as query-idxs
using bound
unfolding accepted-with-bound-tables-def accepted-with-tables-def
by simp
from checked-staged-security-with-data-state-accepted-shape-query-keys
  [OF wf controlled support-some shape]
obtain raw-idxs where
  as-eq: as = staged-alphas data
and len-raw: length raw-idxs = rounds
and query-idxs-eq:
  query-idxs = map (λraw. index (to-nat raw)) raw-idxs
and lookup:
   $\bigwedge j. j < rounds \implies$ 
    fmlookup (HashMap final-state)
      (QueryIndexChallenge j
        (state-after-query-chunks
          (staged-query-start-hash data)
          (staged-query-chunks data) j)) =
      Some (raw-idxs ! j)
and idx-bound:
   $\forall idx \in \text{set query-idxs}. idx < \text{clength} * \text{scale}$ 

```

```

    by blast
  have idx-sample:
    query-idxs ! i ∈ query-sample-space
  proof -
    have len-query: length query-idxs = rounds
      using query-idxs-eq len-raw by simp
    have query-idxs ! i ∈ set query-idxs
      by (rule nth-mem) (use i-bound len-query in simp)
    then show ?thesis
      by (rule accepted-with-bound-tables-query-sample-space[OF bound])
  qed
  have candidate:
    (trace-table, composition-table) ∈
    query-header-supported-table-candidates ?s
    (staged-trace-root data)
    (staged-trace-fri-roots data)
    (staged-trace-final data)
    (staged-alphas data)
    (staged-degree data)
    (staged-composition-fri-roots data)
    (staged-composition-final data)
  proof -
    have bound-staged:
      accepted-with-bound-tables ?s (Some (result, final-state))
      trace-table composition-table (staged-alphas data) query-idxs
    using bound as-eq by simp
    have witness:
      ∃ out query-idxs rest.
      out ∈ set-dist (execute verify-monad ?s) ∧
      accepted-with-bound-tables ?s out trace-table composition-table
      (staged-alphas data) query-idxs ∧
      verifier-header-transcript ?s
      (staged-trace-root data)
      (staged-trace-fri-roots data)
      (staged-trace-final data)
      (staged-alphas data)
      (staged-degree data)
      (staged-composition-fri-roots data)
      (staged-composition-final data) rest
    by (intro exI[where x=Some (result, final-state)]
      exI[where x=query-idxs]
      exI[where x=List.concat (staged-query-chunks data)] conjI)
      (use verifier bound-staged staged-header in simp-all)
    show ?thesis
      using witness
      unfolding query-header-supported-table-candidates-def by simp
  qed
  have raw-hit:
    index (to-nat (raw-idxs ! i)) ∈

```

```

    query-header-supported-union-good-sets ?s
      query-sampling-success-space
      (staged-trace-root data)
      (staged-trace-fri-roots data)
      (staged-trace-final data)
      (staged-alphas data)
      (staged-degree data)
      (staged-composition-fri-roots data)
      (staged-composition-final data)
  proof -
    have raw-idx-eq: index (to-nat (raw-idxs ! i)) = query-idxs ! i
      using query-idxs-eq len-raw i-bound by simp
    show ?thesis
      unfolding query-header-supported-union-good-sets-def raw-idx-eq
      using idx-sample candidate query-hit as-eq by blast
  qed
  have lookup-i:
    fmlookup (HashMap final-state)
      (QueryIndexChallenge i
        (state-after-query-chunks
          (staged-query-start-hash data)
          (staged-query-chunks data) i)) =
      Some (raw-idxs ! i)
    by (rule lookup[OF i-bound])
  show ?thesis
    unfolding out-eq staged-security-with-data-state-query-header-key-hit-def
      Let-def
    using i-bound lookup-i raw-hit by auto
  qed

lemma staged-security-with-data-state-query-bad-bound-from-header-sampling-hit:
  assumes header-bound:
    wp-event (staged-security-experiment-with-data-state A)
    staged-security-with-data-state-query-header-sampling-hit
    adversary-initial-state  $\leq C$ 
  shows
    wp-event (staged-security-experiment-with-data-state A)
    staged-security-with-data-state-query-bad-hit
    adversary-initial-state  $\leq C$ 
  proof -
    have wp-event (staged-security-experiment-with-data-state A)
      staged-security-with-data-state-query-bad-hit
      adversary-initial-state  $\leq$ 
    wp-event (staged-security-experiment-with-data-state A)
      staged-security-with-data-state-query-header-sampling-hit
      adversary-initial-state
    by (rule wp-event-mono)
      (rule staged-security-with-data-state-query-bad-imp-header-sampling-hit)
  also have ...  $\leq C$ 

```


by (rule header-bound)
 finally show ?thesis .
 qed

lemma *checked-staged-security-with-data-state-query-bad-bound-from-header-key-hit*:
 assumes *wf*: staged-budget-wellformed budgets
 and *controlled*: staged-adversary-controlled budgets *A*
 and *header-key-bound*:
 wp-event (checked-staged-security-experiment-with-data-state *A*)
 staged-security-with-data-state-query-header-key-hit
 adversary-initial-state $\leq C$
 shows
 wp-event (checked-staged-security-experiment-with-data-state *A*)
 staged-security-with-data-state-query-bad-hit
 adversary-initial-state $\leq C$
proof –
 have *wp-event* (checked-staged-security-experiment-with-data-state *A*)
 staged-security-with-data-state-query-bad-hit
 adversary-initial-state $\leq$
 wp-event (checked-staged-security-experiment-with-data-state *A*)
 staged-security-with-data-state-query-header-key-hit
 adversary-initial-state
proof (rule *wp-event-mono-on-support*)
 fix *out*
 assume *support*:
 out $\in$
 set-dist
 (*execute*
 (checked-staged-security-experiment-with-data-state *A*)
 adversary-initial-state)
 and *bad*: *staged-security-with-data-state-query-bad-hit out*
 show *staged-security-with-data-state-query-header-key-hit out*
 by (rule
 checked-staged-security-with-data-state-query-bad-imp-header-key-hit-on-support
 [*OF wf controlled support bad*])
 qed
 also have ... $\leq C$
 by (rule *header-key-bound*)
 finally show ?thesis .
 qed

lemma
checked-staged-security-with-data-state-header-key-hit-bound-from-index-set:
 assumes *cover*:
 $\bigwedge$ *data* *attacker-state*.
 query-header-supported-union-good-sets
 (*verifier-state-from-adversary attacker-state*
 (*staged-proof-transcript data*))
 query-sampling-success-space

```

    (staged-trace-root data)
    (staged-trace-fri-roots data)
    (staged-trace-final data)
    (staged-alphas data)
    (staged-degree data)
    (staged-composition-fri-roots data)
    (staged-composition-final data)  $\subseteq B$ 
  and index-bound:
    wp-event (checked-staged-security-experiment-with-data A)
      (staged-security-with-data-query-index-set-hit B)
      adversary-initial-state  $\leq C$ 
  shows
    wp-event (checked-staged-security-experiment-with-data-state A)
      staged-security-with-data-state-query-header-key-hit
      adversary-initial-state  $\leq C$ 
  proof -
    let ?projected =
       $\lambda out. case\ out\ of\ None \Rightarrow$ 
        staged-security-with-data-query-index-set-hit B None
      | Some (((data, -), result), t)  $\Rightarrow$ 
        staged-security-with-data-query-index-set-hit B
        (Some ((data, result), t))
    have event-le:
      wp-event (checked-staged-security-experiment-with-data-state A)
        staged-security-with-data-state-query-header-key-hit
        adversary-initial-state  $\leq$ 
      wp-event (checked-staged-security-experiment-with-data-state A)
        ?projected adversary-initial-state
    proof (rule wp-event-mono)
      fix out
      assume hit:
        staged-security-with-data-state-query-header-key-hit out
      show ?projected out
      proof (cases out)
        case None
        then show ?thesis
          using hit
          unfolding staged-security-with-data-state-query-header-key-hit-def
          by simp
      next
        case (Some packed)
        then obtain data attacker-state result final-state where out-eq:
          out = Some (((data, attacker-state), result), final-state)
          by (cases packed, auto split: prod.splits)
        let ?s =
          verifier-state-from-adversary attacker-state
          (staged-proof-transcript data)
        let ?B =
          query-header-supported-union-good-sets ?s

```

```

    query-sampling-success-space
    (staged-trace-root data)
    (staged-trace-fri-roots data)
    (staged-trace-final data)
    (staged-alphas data)
    (staged-degree data)
    (staged-composition-fri-roots data)
    (staged-composition-final data)
  from hit[unfolded out-eq
    staged-security-with-data-state-query-header-key-hit-def Let-def]
  obtain i raw where i-bound: i < rounds
  and lookup:
    fmlookup (HashMap final-state)
    (QueryIndexChallenge i
      (state-after-query-chunks
        (staged-query-start-hash data)
        (staged-query-chunks data) i)) =
      Some raw
  and raw-hit: index (to-nat raw) ∈ ?B
  by auto
  have raw-in-B: index (to-nat raw) ∈ B
  proof -
    have ?B ⊆ B
    by (rule cover)
    then show ?thesis
    using raw-hit by blast
  qed
  show ?thesis
    unfolding out-eq staged-security-with-data-query-index-set-hit-def
    using i-bound lookup raw-in-B by auto
  qed
  qed
  also have
    wp-event (checked-staged-security-experiment-with-data-state A)
      ?projected adversary-initial-state =
    wp-event (checked-staged-security-experiment-with-data A)
      (staged-security-with-data-query-index-set-hit B)
      adversary-initial-state
  using checked-staged-security-experiment-with-data-event-from-data-state
    [of A staged-security-with-data-query-index-set-hit B]
  by simp
  also have ... ≤ C
  by (rule index-bound)
  finally show ?thesis .
  qed

lemma
  checked-staged-security-with-data-state-header-key-hit-bound-from-index-set-controlled:
  assumes wf: staged-budget-wellformed budgets

```

and controlled: staged-adversary-controlled budgets A

and cover:

$\bigwedge$ data attacker-state.
query-header-supported-union-good-sets
(verifier-state-from-adversary attacker-state
(staged-proof-transcript data))
query-sampling-success-space
(staged-trace-root data)
(staged-trace-fri-roots data)
(staged-trace-final data)
(staged-alphas data)
(staged-degree data)
(staged-composition-fri-roots data)
(staged-composition-final data) $\subseteq B$

shows

wp-event (checked-staged-security-experiment-with-data-state A)
staged-security-with-data-state-query-header-key-hit
adversary-initial-state $\leq$
staged-phase-target-error (query-index-raw-preimage B)
(staged-attacker-query-budget budgets +
staged-challenge-query-budget)

by (rule

checked-staged-security-with-data-state-header-key-hit-bound-from-index-set
[OF cover])

(rule checked-staged-security-with-data-query-index-set-hit-bound
[OF wf controlled])

lemma

checked-staged-security-with-data-state-query-bad-bound-from-index-set-controlled:

assumes wf: staged-budget-wellformed budgets

and controlled: staged-adversary-controlled budgets A

and cover:

$\bigwedge$ data attacker-state.
query-header-supported-union-good-sets
(verifier-state-from-adversary attacker-state
(staged-proof-transcript data))
query-sampling-success-space
(staged-trace-root data)
(staged-trace-fri-roots data)
(staged-trace-final data)
(staged-alphas data)
(staged-degree data)
(staged-composition-fri-roots data)
(staged-composition-final data) $\subseteq B$

shows

wp-event (checked-staged-security-experiment-with-data-state A)
staged-security-with-data-state-query-bad-hit
adversary-initial-state $\leq$
staged-phase-target-error (query-index-raw-preimage B)

(staged-attacker-query-budget budgets +
 staged-challenge-query-budget)
by (rule checked-staged-security-with-data-state-query-bad-bound-from-header-key-hit
 [OF wf controlled])
 (rule
 checked-staged-security-with-data-state-header-key-hit-bound-from-index-set-controlled
 [OF wf controlled cover])

lemma

checked-staged-security-with-data-state-query-bad-bound-from-fixed-query-error-cover:

assumes wf: staged-budget-wellformed budgets

and controlled: staged-adversary-controlled budgets A

and cover:

$\bigwedge$ data attacker-state.
 query-header-supported-union-good-sets
 (verifier-state-from-adversary attacker-state
 (staged-proof-transcript data))
 query-sampling-success-space
 (staged-trace-root data)
 (staged-trace-fri-roots data)
 (staged-trace-final data)
 (staged-alphas data)
 (staged-degree data)
 (staged-composition-fri-roots data)
 (staged-composition-final data) $\subseteq B$

and raw-bound: query-index-raw-preimage-bound

and subset: $B \subseteq \text{query-sample-space}$

and frac:

$\text{nnreal}(\text{card } B) / \text{nnreal}(\text{card query-sample-space}) \leq$
 query-error-bound

shows

wp-event (checked-staged-security-experiment-with-data-state A)
 staged-security-with-data-state-query-bad-hit
 adversary-initial-state $\leq$
 nnreal
 (staged-attacker-query-budget budgets +
 staged-challenge-query-budget) * query-error-bound

proof –

have target:

wp-event (checked-staged-security-experiment-with-data-state A)
 staged-security-with-data-state-query-bad-hit
 adversary-initial-state $\leq$
 staged-phase-target-error (query-index-raw-preimage B)
 (staged-attacker-query-budget budgets +
 staged-challenge-query-budget)

by (rule

checked-staged-security-with-data-state-query-bad-bound-from-index-set-controlled
 [OF wf controlled cover])

also have ... $\leq$

$nnreal$
 $(staged-attacker-query-budget\ budgets +$
 $staged-challenge-query-budget) * query-error-bound$
by (rule *staged-phase-query-index-target-error-query-bound*
 $[OF\ raw-bound\ subset\ frac]$)
finally show *?thesis* .
qed

lemma

checked-staged-security-with-data-state-query-bad-verifier-event-bound-from-index-set-controlled:

assumes *wf: staged-budget-wellformed budgets*

and *controlled: staged-adversary-controlled budgets A*

and *cover:*

$\bigwedge data\ attacker-state.$
 $query-header-supported-union-good-sets$
 $(verifier-state-from-adversary\ attacker-state$
 $(staged-proof-transcript\ data))$
 $query-sampling-success-space$
 $(staged-trace-root\ data)$
 $(staged-trace-fri-roots\ data)$
 $(staged-trace-final\ data)$
 $(staged-alphas\ data)$
 $(staged-degree\ data)$
 $(staged-composition-fri-roots\ data)$
 $(staged-composition-final\ data) \subseteq B$

shows

$wp-event\ (checked-staged-security-experiment-with-data-state\ A)$
 $(staged-security-with-data-state-verifier-event\ query-bad)$
 $adversary-initial-state \leq$
 $staged-phase-target-error\ (query-index-raw-preimage\ B)$
 $(staged-attacker-query-budget\ budgets +$
 $staged-challenge-query-budget)$

by (rule

checked-staged-security-with-data-state-query-bad-verifier-event-bound)

(rule

checked-staged-security-with-data-state-query-bad-bound-from-index-set-controlled
 $[OF\ wf\ controlled\ cover]$)

lemma

checked-staged-security-with-data-state-query-bad-verifier-event-bound-from-fixed-query-error-cover:

assumes *wf: staged-budget-wellformed budgets*

and *controlled: staged-adversary-controlled budgets A*

and *cover:*

$\bigwedge data\ attacker-state.$
 $query-header-supported-union-good-sets$
 $(verifier-state-from-adversary\ attacker-state$
 $(staged-proof-transcript\ data))$
 $query-sampling-success-space$
 $(staged-trace-root\ data)$

(staged-trace-fri-roots data)
 (staged-trace-final data)
 (staged-alphas data)
 (staged-degree data)
 (staged-composition-fri-roots data)
 (staged-composition-final data) $\subseteq B$
and raw-bound: query-index-raw-preimage-bound
and subset: $B \subseteq \text{query-sample-space}$
and frac:
 $\text{nnreal } (\text{card } B) / \text{nnreal } (\text{card } \text{query-sample-space}) \leq$
 query-error-bound
shows
 $\text{wp-event } (\text{checked-staged-security-experiment-with-data-state } A)$
 $(\text{staged-security-with-data-state-verifier-event } \text{query-bad})$
 $\text{adversary-initial-state} \leq$
 nnreal
 $(\text{staged-attacker-query-budget budgets} +$
 $\text{staged-challenge-query-budget}) * \text{query-error-bound}$
by (rule
 $\text{checked-staged-security-with-data-state-query-bad-verifier-event-bound})$
(rule
 $\text{checked-staged-security-with-data-state-query-bad-bound-from-fixed-query-error-cover}$
 $[\text{OF wf controlled cover raw-bound subset frac}]$)

lemma staged-security-with-data-query-bad-bound-from-sampling-hit:
assumes sampling-bound:
 $\text{wp-event } (\text{staged-security-experiment-with-data } A)$
 $\text{staged-security-with-data-query-sampling-hit}$
 $\text{adversary-initial-state} \leq C$
shows
 $\text{wp-event } (\text{staged-security-experiment-with-data } A)$
 $\text{staged-security-with-data-query-bad-hit}$
 $\text{adversary-initial-state} \leq C$

proof –
have $\text{wp-event } (\text{staged-security-experiment-with-data } A)$
 $\text{staged-security-with-data-query-bad-hit}$
 $\text{adversary-initial-state} \leq$
 $\text{wp-event } (\text{staged-security-experiment-with-data } A)$
 $\text{staged-security-with-data-query-sampling-hit}$
 $\text{adversary-initial-state}$
by (rule wp-event-mono)
(rule staged-security-with-data-query-bad-imp-sampling-hit)
also have $\dots \leq C$
by (rule sampling-bound)
finally show ?thesis .
qed

lemma staged-security-with-data-query-bad-bound-from-header-sampling-hit:
assumes header-bound:

$wp\text{-}event \text{ (staged-security-experiment-with-data } A)$
 $staged\text{-}security\text{-}with\text{-}data\text{-}query\text{-}header\text{-}sampling\text{-}hit$
 $adversary\text{-}initial\text{-}state \leq C$
shows
 $wp\text{-}event \text{ (staged-security-experiment-with-data } A)$
 $staged\text{-}security\text{-}with\text{-}data\text{-}query\text{-}bad\text{-}hit$
 $adversary\text{-}initial\text{-}state \leq C$
proof –
have $wp\text{-}event \text{ (staged-security-experiment-with-data } A)$
 $staged\text{-}security\text{-}with\text{-}data\text{-}query\text{-}bad\text{-}hit$
 $adversary\text{-}initial\text{-}state \leq$
 $wp\text{-}event \text{ (staged-security-experiment-with-data } A)$
 $staged\text{-}security\text{-}with\text{-}data\text{-}query\text{-}header\text{-}sampling\text{-}hit$
 $adversary\text{-}initial\text{-}state$
by (rule $wp\text{-}event\text{-}mono$)
(rule $staged\text{-}security\text{-}with\text{-}data\text{-}query\text{-}bad\text{-}imp\text{-}header\text{-}sampling\text{-}hit$)
also have $\dots \leq C$
by (rule $header\text{-}bound$)
finally show $?thesis$.
qed

lemma *staged-transcript-program-outcome-trace-challenge-lookup*:
assumes wf : *staged-budget-wellformed budgets*
and *controlled*: *staged-adversary-controlled budgets A*
and *outcome*:
 $Some \text{ (data, attacker-state)} \in$
 $set\text{-}dist$
 $(execute \text{ (staged-transcript-program } A)$
 $adversary\text{-}initial\text{-}state)$
and *i-bound*: $i < \text{ceil-log } clength$
shows
 $fmllookup \text{ (HashMap attacker-state)}$
 $(TraceFriChallenge \text{ } i$
 $(foldl \text{ concat (staged-trace-fri-start-hash data)}$
 $(take \text{ (Suc } i) \text{ (staged-trace-fri-roots data)}))) =$
 $Some \text{ (staged-trace-fri-challenges data ! } i)$
proof –
from *outcome* **obtain** $fr \text{ } s1 \text{ } s2 \text{ trace-roots trace-bs } s3 \text{ trace-final } s4 \text{ } s5$
 $as \text{ } s6 \text{ } dg \text{ } s7 \text{ } s8 \text{ } s9 \text{ composition-roots composition-bs } s10$
 $composition\text{-}final \text{ } s11 \text{ } s12 \text{ query-chunks}$ **where**
root-out:
 $Some \text{ (fr, } s1) \in$
 $set\text{-}dist \text{ (execute (trace-root-stage } A) adversary\text{-}initial\text{-}state)$
and *record-root*:
 $Some \text{ ((), } s2) \in set\text{-}dist \text{ (execute (record-staged-message fr) } s1)$
and *trace-out*:
 $Some \text{ ((trace-roots, trace-bs), } s3) \in$
 $set\text{-}dist$
 $(execute$


```

    (staged-trace-fri-program A 0 (ceil-log clength) []) s2)
and final-out:
    Some (trace-final, s4) ∈
    set-dist (execute (trace-final-stage A trace-bs) s3)
and record-final:
    Some ((), s5) ∈
    set-dist (execute (record-staged-message trace-final) s4)
and alpha-out:
    Some (as, s6) ∈
    set-dist (execute (staged-alpha-program (length spec)) s5)
and degree-out:
    Some (dg, s7) ∈ set-dist (execute (degree-stage A as) s6)
and record-degree:
    Some ((), s8) ∈ set-dist (execute (record-staged-message dg) s7)
and assert-out:
    Some ((), s9) ∈
    set-dist
    (execute
    (assert
    (ceil-log (to-nat dg + 1) ≤
    ceil-log (maxDegree + 1))) s8)
and composition-out:
    Some ((composition-roots, composition-bs), s10) ∈
    set-dist
    (execute
    (staged-composition-fri-program A dg 0
    (ceil-log (to-nat dg + 1)) []) s9)
and composition-final-out:
    Some (composition-final, s11) ∈
    set-dist
    (execute (composition-final-stage A dg composition-bs) s10)
and record-composition-final:
    Some ((), s12) ∈
    set-dist (execute (record-staged-message composition-final) s11)
and query-out:
    Some (query-chunks, attacker-state) ∈
    set-dist (execute (staged-query-program A 0 rounds) s12)
and data-eq:
    data =
    (|staged-trace-root = fr,
    staged-trace-fri-roots = trace-roots,
    staged-trace-fri-challenges = trace-bs,
    staged-trace-final = trace-final,
    staged-alphas = as,
    staged-degree = dg,
    staged-composition-fri-roots = composition-roots,
    staged-composition-fri-challenges = composition-bs,
    staged-composition-final = composition-final,
    staged-query-chunks = query-chunks|)

```

```

unfolding staged-transcript-program-def Let-def
by (auto elim!: set-dist-bindE split: prod.splits)
have root-controlled:
  controlled-ro-program (trace-root-budget budgets)
    (trace-root-stage A)
using controlled unfolding staged-adversary-controlled-def by simp
have trace-final-controlled:
  controlled-ro-program (trace-final-budget budgets)
    (trace-final-stage A trace-bs)
using controlled unfolding staged-adversary-controlled-def by blast
have degree-controlled:
  controlled-ro-program (degree-budget budgets) (degree-stage A as)
using controlled unfolding staged-adversary-controlled-def by blast
have composition-final-controlled:
  controlled-ro-program (composition-final-budget budgets)
    (composition-final-stage A dg composition-bs)
using controlled unfolding staged-adversary-controlled-def by blast
have root-fields:
  PState s1 = 0 ∧
  PTraceFriCounter s1 = 0
using controlled-stage-outcome-fields[OF root-controlled root-out]
by simp
have s2-eq:
  s2 = s1 (
    PState := concat (PState s1) fr,
    PTranscript := PTranscript s1 @ [fr])
by (rule record-staged-message-outcome[OF record-root])
have trace-bound:
  0 + ceil-log clength ≤ length (trace-fri-budgets budgets)
using wf unfolding staged-budget-wellformed-def by simp
have trace-alignment:
  length trace-roots = ceil-log clength ∧
  length trace-bs = length ([] :: 'f list) + ceil-log clength ∧
  PTranscript s3 = PTranscript s2 @ trace-roots ∧
  PState s3 = foldl concat (PState s2) trace-roots ∧
  s2 ≤ s3 ∧
  PTraceFriCounter s3 = PTraceFriCounter s2 + ceil-log clength ∧
  PCompositionFriCounter s3 = PCompositionFriCounter s2 ∧
  PAlphaCounter s3 = PAlphaCounter s2 ∧
  PQueryCounter s3 = PQueryCounter s2 ∧
  (∀ j < ceil-log clength.
    fmlookup (HashMap s3)
      (TraceFriChallenge (PTraceFriCounter s2 + j)
        (foldl concat (PState s2) (take (Suc j) trace-roots))) =
        Some (trace-bs ! (length ([] :: 'f list) + j)))
by (rule staged-trace-fri-program-alignment
  [OF controlled trace-bound trace-out])
let ?key =
  TraceFriChallenge i

```

```

    (foldl concat (concat 0 fr) (take (Suc i) trace-roots))
have lookup-s3: fmlookup (HashMap s3) ?key = Some (trace-bs ! i)
  using trace-alignment i-bound root-fields
  unfolding s2-eq by simp
have ext-s3-s4: s3 ≤ s4
  using controlled-ro-program-extension[OF trace-final-controlled]
    final-out
  unfolding hash-extension-preserving-def by blast
have s5-eq:
  s5 = s4 (
    PState := concat (PState s4) trace-final,
    PTranscript := PTranscript s4 @ [trace-final])
  by (rule record-staged-message-outcome[OF record-final])
have ext-s4-s5: s4 ≤ s5
  unfolding s5-eq less-eq-hash-ext-def less-eq-fmap-def by simp
have ext-s5-s6: s5 ≤ s6
  by (rule hash-target-program-outcome-extension
    [OF hash-target-program-staged-alpha-program alpha-out])
have ext-s6-s7: s6 ≤ s7
  using controlled-ro-program-extension[OF degree-controlled] degree-out
  unfolding hash-extension-preserving-def by blast
have s8-eq:
  s8 = s7 (
    PState := concat (PState s7) dg,
    PTranscript := PTranscript s7 @ [dg])
  by (rule record-staged-message-outcome[OF record-degree])
have ext-s7-s8: s7 ≤ s8
  unfolding s8-eq less-eq-hash-ext-def less-eq-fmap-def by simp
have s9-eq: s9 = s8
  using assert-out unfolding assert-def
  by (cases
    ceil-log (to-nat dg + 1) ≤ ceil-log (maxDegree + 1))
    (auto simp: throw-no-outcome)
have ext-s8-s9: s8 ≤ s9
  unfolding s9-eq by (simp add: hash-ext-refl)
have round-bound:
  ceil-log (to-nat dg + 1) ≤ ceil-log (maxDegree + 1)
  using assert-out unfolding assert-def
  by (cases
    ceil-log (to-nat dg + 1) ≤ ceil-log (maxDegree + 1))
    (auto simp: throw-no-outcome)
have composition-bound:
  0 + ceil-log (to-nat dg + 1) ≤
    length (composition-fri-budgets budgets)
  using wf round-bound unfolding staged-budget-wellformed-def by simp
have ext-s9-s10: s9 ≤ s10
  using staged-composition-fri-program-alignment
    [OF controlled composition-bound composition-out]
  by simp

```

```

have ext-s10-s11:  $s10 \leq s11$ 
  using controlled-ro-program-extension[OF composition-final-controlled]
    composition-final-out
  unfolding hash-extension-preserving-def by blast
have s12-eq:
  s12 = s11()
    PState := concat (PState s11) composition-final,
    PTranscript := PTranscript s11 @ [composition-final])
  by (rule record-staged-message-outcome[OF record-composition-final])
have ext-s11-s12:  $s11 \leq s12$ 
  unfolding s12-eq less-eq-hash-ext-def less-eq-fmap-def by simp
have query-bound:  $0 + \text{rounds} \leq \text{length} (\text{query-opening-budgets budgets})$ 
  using wf unfolding staged-budget-wellformed-def by simp
have ext-s12-attacker:  $s12 \leq \text{attacker-state}$ 
  by (rule hash-target-program-outcome-extension
    [OF hash-target-program-staged-query-program[OF controlled query-bound]
      query-out])
have ext-s3-s5:  $s3 \leq s5$ 
  by (rule hash-ext-trans[OF ext-s3-s4 ext-s4-s5])
have ext-s3-s6:  $s3 \leq s6$ 
  by (rule hash-ext-trans[OF ext-s3-s5 ext-s5-s6])
have ext-s3-s7:  $s3 \leq s7$ 
  by (rule hash-ext-trans[OF ext-s3-s6 ext-s6-s7])
have ext-s3-s8:  $s3 \leq s8$ 
  by (rule hash-ext-trans[OF ext-s3-s7 ext-s7-s8])
have ext-s3-s9:  $s3 \leq s9$ 
  by (rule hash-ext-trans[OF ext-s3-s8 ext-s8-s9])
have ext-s3-s10:  $s3 \leq s10$ 
  by (rule hash-ext-trans[OF ext-s3-s9 ext-s9-s10])
have ext-s3-s11:  $s3 \leq s11$ 
  by (rule hash-ext-trans[OF ext-s3-s10 ext-s10-s11])
have ext-s3-s12:  $s3 \leq s12$ 
  by (rule hash-ext-trans[OF ext-s3-s11 ext-s11-s12])
have ext-s3-attacker:  $s3 \leq \text{attacker-state}$ 
  by (rule hash-ext-trans[OF ext-s3-s12 ext-s12-attacker])
have lookup-attacker:
  fmlookup (HashMap attacker-state) ?key = Some (trace-bs ! i)
  by (rule hash-extension-lookup[OF lookup-s3 ext-s3-attacker])
show ?thesis
  unfolding data-eq staged-trace-fri-start-hash-def
  using lookup-attacker by simp
qed

```

```

lemma staged-transcript-program-outcome-composition-challenge-lookup:
  assumes wf: staged-budget-wellformed budgets
  and controlled: staged-adversary-controlled budgets A
  and outcome:
    Some (data, attacker-state) ∈
      set-dist

```

```

      (execute (staged-transcript-program A)
        adversary-initial-state)
    and i-bound:
      i < ceil-log (to-nat (staged-degree data) + 1)
  shows
    fmlookup (HashMap attacker-state)
      (CompositionFriChallenge i
        (foldl concat (staged-composition-fri-start-hash data)
          (take (Suc i) (staged-composition-fri-roots data)))) =
      Some (staged-composition-fri-challenges data ! i)
  proof -
  from outcome obtain fr s1 s2 trace-roots trace-bs s3 trace-final s4 s5
    as s6 dg s7 s8 s9 composition-roots composition-bs s10
    composition-final s11 s12 query-chunks where
  root-out:
    Some (fr, s1) ∈
      set-dist (execute (trace-root-stage A) adversary-initial-state)
  and record-root:
    Some ((), s2) ∈ set-dist (execute (record-staged-message fr) s1)
  and trace-out:
    Some ((trace-roots, trace-bs), s3) ∈
      set-dist
        (execute
          (staged-trace-fri-program A 0 (ceil-log clength) []) s2)
  and final-out:
    Some (trace-final, s4) ∈
      set-dist (execute (trace-final-stage A trace-bs) s3)
  and record-final:
    Some ((), s5) ∈
      set-dist (execute (record-staged-message trace-final) s4)
  and alpha-out:
    Some (as, s6) ∈
      set-dist (execute (staged-alpha-program (length spec)) s5)
  and degree-out:
    Some (dg, s7) ∈ set-dist (execute (degree-stage A as) s6)
  and record-degree:
    Some ((), s8) ∈ set-dist (execute (record-staged-message dg) s7)
  and assert-out:
    Some ((), s9) ∈
      set-dist
        (execute
          (assert
            (ceil-log (to-nat dg + 1) ≤
              ceil-log (maxDegree + 1))) s8)
  and composition-out:
    Some ((composition-roots, composition-bs), s10) ∈
      set-dist
        (execute
          (staged-composition-fri-program A dg 0

```

```

      (ceil-log (to-nat dg + 1)) []) s9)
and composition-final-out:
  Some (composition-final, s11) ∈
    set-dist
      (execute (composition-final-stage A dg composition-bs) s10)
and record-composition-final:
  Some ((), s12) ∈
    set-dist (execute (record-staged-message composition-final) s11)
and query-out:
  Some (query-chunks, attacker-state) ∈
    set-dist (execute (staged-query-program A 0 rounds) s12)
and data-eq:
  data =
    (|staged-trace-root = fr,
     staged-trace-fri-roots = trace-roots,
     staged-trace-fri-challenges = trace-bs,
     staged-trace-final = trace-final,
     staged-alphas = as,
     staged-degree = dg,
     staged-composition-fri-roots = composition-roots,
     staged-composition-fri-challenges = composition-bs,
     staged-composition-final = composition-final,
     staged-query-chunks = query-chunks|)
unfolding staged-transcript-program-def Let-def
by (auto elim!: set-dist-bindE split: prod.splits)
have root-controlled:
  controlled-ro-program (trace-root-budget budgets)
    (trace-root-stage A)
using controlled unfolding staged-adversary-controlled-def by simp
have trace-final-controlled:
  controlled-ro-program (trace-final-budget budgets)
    (trace-final-stage A trace-bs)
using controlled unfolding staged-adversary-controlled-def by blast
have degree-controlled:
  controlled-ro-program (degree-budget budgets) (degree-stage A as)
using controlled unfolding staged-adversary-controlled-def by blast
have composition-final-controlled:
  controlled-ro-program (composition-final-budget budgets)
    (composition-final-stage A dg composition-bs)
using controlled unfolding staged-adversary-controlled-def by blast
have root-fields:
  PState s1 = 0 ∧
  PCompositionFriCounter s1 = 0
using controlled-stage-outcome-fields[OF root-controlled root-out]
by simp
have s2-eq:
  s2 = s1 (|
    PState := concat (PState s1) fr,
    PTranscript := PTranscript s1 @ [fr]|)

```

```

  by (rule record-staged-message-outcome[OF record-root])
have s2-fields:
  PState s2 = concat 0 fr  $\wedge$ 
  PCompositionFriCounter s2 = 0
  unfolding s2-eq using root-fields by simp
have trace-bound:
  0 + ceil-log clength  $\leq$  length (trace-fri-budgets budgets)
  using wf unfolding staged-budget-wellformed-def by simp
have trace-alignment:
  length trace-roots = ceil-log clength  $\wedge$ 
  length trace-bs = length ([] :: 'f list) + ceil-log clength  $\wedge$ 
  PTranscript s3 = PTranscript s2 @ trace-roots  $\wedge$ 
  PState s3 = foldl concat (PState s2) trace-roots  $\wedge$ 
  s2  $\leq$  s3  $\wedge$ 
  PTraceFriCounter s3 = PTraceFriCounter s2 + ceil-log clength  $\wedge$ 
  PCompositionFriCounter s3 = PCompositionFriCounter s2  $\wedge$ 
  PAlphaCounter s3 = PAlphaCounter s2  $\wedge$ 
  PQueryCounter s3 = PQueryCounter s2  $\wedge$ 
  ( $\forall j < \text{ceil-log clength}.$ 
    fmllookup (HashMap s3)
      (TraceFriChallenge (PTraceFriCounter s2 + j)
        (foldl concat (PState s2) (take (Suc j) trace-roots))) =
        Some (trace-bs ! (length ([] :: 'f list) + j)))
  by (rule staged-trace-fri-program-alignment
    [OF controlled trace-bound trace-out])
have final-fields:
  PState s4 = PState s3  $\wedge$ 
  PCompositionFriCounter s4 = PCompositionFriCounter s3
  using controlled-stage-outcome-fields
    [OF trace-final-controlled final-out]
  by simp
have s5-eq:
  s5 = s4 (
    PState := concat (PState s4) trace-final,
    PTranscript := PTranscript s4 @ [trace-final])
  by (rule record-staged-message-outcome[OF record-final])
have s5-fields:
  PState s5 =
    concat (foldl concat (concat 0 fr) trace-roots) trace-final  $\wedge$ 
  PCompositionFriCounter s5 = 0
  unfolding s5-eq
  using s2-fields trace-alignment final-fields by simp
have alpha-alignment:
  length as = length spec  $\wedge$ 
  PTranscript s6 = PTranscript s5 @ as  $\wedge$ 
  PState s6 = foldl concat (PState s5) as  $\wedge$ 
  PTraceFriCounter s6 = PTraceFriCounter s5  $\wedge$ 
  PCompositionFriCounter s6 = PCompositionFriCounter s5  $\wedge$ 
  PAlphaCounter s6 = PAlphaCounter s5 + length spec  $\wedge$ 

```

```

    PQueryCounter s6 = PQueryCounter s5
  by (rule staged-alpha-program-alignment[OF alpha-out])
have degree-fields:
  PState s7 = PState s6 ∧
  PCompositionFriCounter s7 = PCompositionFriCounter s6
  using controlled-stage-outcome-fields[OF degree-controlled degree-out]
  by simp
have s8-eq:
  s8 = s7(
    PState := concat (PState s7) dg,
    PTranscript := PTranscript s7 @ [dg])
  by (rule record-staged-message-outcome[OF record-degree])
have s9-eq: s9 = s8
  using assert-out unfolding assert-def
  by (cases
    ceil-log (to-nat dg + 1) ≤ ceil-log (maxDegree + 1))
    (auto simp: throw-no-outcome)
have start-fields:
  PState s9 =
    concat
      (foldl concat
        (concat
          (foldl concat (concat 0 fr) trace-roots)
          trace-final)
        as)
      dg ∧
  PCompositionFriCounter s9 = 0
unfolding s9-eq s8-eq
using s5-fields alpha-alignment degree-fields
  by simp
have round-bound:
  ceil-log (to-nat dg + 1) ≤ ceil-log (maxDegree + 1)
  using assert-out unfolding assert-def
  by (cases
    ceil-log (to-nat dg + 1) ≤ ceil-log (maxDegree + 1))
    (auto simp: throw-no-outcome)
have composition-bound:
  0 + ceil-log (to-nat dg + 1) ≤
    length (composition-fri-budgets budgets)
  using wf round-bound unfolding staged-budget-wellformed-def by simp
have composition-alignment:
  length composition-roots = ceil-log (to-nat dg + 1) ∧
  length composition-bs =
    length ([] :: 'f list) + ceil-log (to-nat dg + 1) ∧
  PTranscript s10 = PTranscript s9 @ composition-roots ∧
  PState s10 = foldl concat (PState s9) composition-roots ∧
  s9 ≤ s10 ∧
  PTraceFriCounter s10 = PTraceFriCounter s9 ∧
  PCompositionFriCounter s10 =

```



```

    PCompositionFriCounter s9 + ceil-log (to-nat dg + 1) ∧
    PAlphaCounter s10 = PAlphaCounter s9 ∧
    PQueryCounter s10 = PQueryCounter s9 ∧
    (∀ j < ceil-log (to-nat dg + 1).
      fmlookup (HashMap s10)
        (CompositionFriChallenge (PCompositionFriCounter s9 + j)
          (foldl concat (PState s9) (take (Suc j) composition-roots))) =
          Some (composition-bs ! (length [] :: 'f list) + j)))
  by (rule staged-composition-fri-program-alignment
    [OF controlled composition-bound composition-out])
let ?key =
  CompositionFriChallenge i
  (foldl concat
    (concat
      (foldl concat
        (concat
          (foldl concat (concat 0 fr) trace-roots)
          trace-final)
        as)
      dg)
    (take (Suc i) composition-roots))
have i-bound': i < ceil-log (to-nat dg + 1)
  using i-bound unfolding data-eq by simp
have lookup-s10:
  fmlookup (HashMap s10) ?key = Some (composition-bs ! i)
  using composition-alignment i-bound' start-fields by simp
have ext-s10-s11: s10 ≤ s11
  using controlled-ro-program-extension[OF composition-final-controlled]
  composition-final-out
  unfolding hash-extension-preserving-def by blast
have s12-eq:
  s12 = s11 (
    PState := concat (PState s11) composition-final,
    PTranscript := PTranscript s11 @ [composition-final])
  by (rule record-staged-message-outcome[OF record-composition-final])
have ext-s11-s12: s11 ≤ s12
  unfolding s12-eq less-eq-hash-ext-def less-eq-fmap-def by simp
have query-bound: 0 + rounds ≤ length (query-opening-budgets budgets)
  using wf unfolding staged-budget-wellformed-def by simp
have ext-s12-attacker: s12 ≤ attacker-state
  by (rule hash-target-program-outcome-extension
    [OF hash-target-program-staged-query-program[OF controlled query-bound]
    query-out])
have ext-s10-s12: s10 ≤ s12
  by (rule hash-ext-trans[OF ext-s10-s11 ext-s11-s12])
have ext-s10-attacker: s10 ≤ attacker-state
  by (rule hash-ext-trans[OF ext-s10-s12 ext-s12-attacker])
have lookup-attacker:
  fmlookup (HashMap attacker-state) ?key = Some (composition-bs ! i)

```

```

    by (rule hash-extension-lookup[OF lookup-s10 ext-s10-attacker])
  show ?thesis
    unfolding data-eq staged-composition-fri-start-hash-def
      staged-trace-fri-start-hash-def
    using lookup-attacker by simp
qed

```

lemma *staged-transcript-program-outcome-header-transcript:*

```

assumes wf: staged-budget-wellformed budgets
and controlled: staged-adversary-controlled budgets A
and outcome:
  Some (data, attacker-state) ∈
    set-dist
      (execute (staged-transcript-program A)
        adversary-initial-state)

```

shows

```

  verifier-header-transcript
    (verifier-state-from-adversary attacker-state
      (staged-proof-transcript data))
    (staged-trace-root data)
    (staged-trace-fri-roots data)
    (staged-trace-final data)
    (staged-alphas data)
    (staged-degree data)
    (staged-composition-fri-roots data)
    (staged-composition-final data)
    (List.concat (staged-query-chunks data))

```

proof –

have *shape:*

```

  length (staged-trace-fri-roots data) = ceil-log clength ∧
  length (staged-trace-fri-challenges data) = ceil-log clength ∧
  length (staged-alphas data) = length spec ∧
  length (staged-composition-fri-roots data) =
    ceil-log (to-nat (staged-degree data) + 1) ∧
  length (staged-composition-fri-challenges data) =
    ceil-log (to-nat (staged-degree data) + 1) ∧
  ceil-log (to-nat (staged-degree data) + 1) ≤
    ceil-log (maxDegree + 1) ∧
  length (staged-query-chunks data) = rounds

```

```

by (rule staged-transcript-program-outcome-shape
  [OF wf controlled outcome])

```

show ?thesis

```

by (rule staged-proof-transcript-verifier-header-transcript)
  (use shape in simp-all)

```

qed

lemma *staged-security-experiment-outcomeE:*

assumes

```

  Some (result, final-state) ∈

```

```

    set-dist (execute (staged-security-experiment A) initial-state)
obtains data attacker-state where
  Some (data, attacker-state) ∈
    set-dist (execute (staged-transcript-program A) initial-state)
  Some (result, final-state) ∈
    set-dist
      (execute verify-monad
        (verifier-state-from-adversary attacker-state
          (staged-proof-transcript data)))
using assms unfolding staged-security-experiment-def
by (auto elim!: set-dist-bindE intro: that)

lemma accepted-staged-security-experimentE:
  assumes outcome:
    out ∈ set-dist
      (execute (staged-security-experiment A) adversary-initial-state)
  and accepted: ¬ Option.is-none out
obtains result final-state data attacker-state where
  out = Some (result, final-state)
  Some (data, attacker-state) ∈
    set-dist (execute (staged-transcript-program A) adversary-initial-state)
  Some (result, final-state) ∈
    set-dist
      (execute verify-monad
        (verifier-state-from-adversary attacker-state
          (staged-proof-transcript data)))
proof –
  from accepted obtain result final-state where
    out: out = Some (result, final-state)
  by (cases out) auto
  from staged-security-experiment-outcomeE[OF outcome[unfolded out]]
obtain data attacker-state where
  builder:
    Some (data, attacker-state) ∈
      set-dist (execute (staged-transcript-program A)
        adversary-initial-state)
  and verifier:
    Some (result, final-state) ∈
      set-dist
        (execute verify-monad
          (verifier-state-from-adversary attacker-state
            (staged-proof-transcript data)))
  by blast
  show ?thesis by (rule that[OF out builder verifier])
qed

```

```

lemma accepted-staged-security-experiment-headerE:
  assumes wf: staged-budget-wellformed budgets
  and controlled: staged-adversary-controlled budgets A

```

and *outcome*:
 out $\in$ *set-dist*
 (*execute* (*staged-security-experiment* *A*) *adversary-initial-state*)
and *accepted*: $\neg$ *Option.is-none out*
obtains *result final-state data attacker-state* **where**
 out = *Some (result, final-state)*
 Some (data, attacker-state) $\in$
 set-dist (*execute* (*staged-transcript-program* *A*)
 adversary-initial-state)
 Some (result, final-state) $\in$
 set-dist
 (*execute verify-monad*
 (*verifier-state-from-adversary attacker-state*
 (*staged-proof-transcript data*)))
verifier-header-transcript
 (*verifier-state-from-adversary attacker-state*
 (*staged-proof-transcript data*))
 (*staged-trace-root data*)
 (*staged-trace-fri-roots data*)
 (*staged-trace-final data*)
 (*staged-alphas data*)
 (*staged-degree data*)
 (*staged-composition-fri-roots data*)
 (*staged-composition-final data*)
 (*List.concat (staged-query-chunks data)*)
proof –
from *accepted-staged-security-experimentE*[*OF outcome accepted*]
obtain *result final-state data attacker-state* **where**
 out-eq: *out* = *Some (result, final-state)*
and *builder*:
 Some (data, attacker-state) $\in$
 set-dist (*execute* (*staged-transcript-program* *A*)
 adversary-initial-state)
and *verifier*:
 Some (result, final-state) $\in$
 set-dist
 (*execute verify-monad*
 (*verifier-state-from-adversary attacker-state*
 (*staged-proof-transcript data*)))
by *blast*
have *header*:
 verifier-header-transcript
 (*verifier-state-from-adversary attacker-state*
 (*staged-proof-transcript data*))
 (*staged-trace-root data*)
 (*staged-trace-fri-roots data*)
 (*staged-trace-final data*)
 (*staged-alphas data*)
 (*staged-degree data*)

```

    (staged-composition-fri-roots data)
    (staged-composition-final data)
    (List.concat (staged-query-chunks data))
  by (rule staged-transcript-program-outcome-header-transcript
      [OF wf controlled builder])
  show ?thesis
  by (rule that[OF out-eq builder verifier header])
qed

```

```

lemma accepted-staged-security-experiment-transcript-shapeE:
assumes wf: staged-budget-wellformed budgets
and controlled: staged-adversary-controlled budgets A
and outcome:
  out ∈ set-dist
  (execute (staged-security-experiment A) adversary-initial-state)
and accepted: ¬ Option.is-none out
obtains result final-state data attacker-state query-idxs where
  out = Some (result, final-state)
  Some (data, attacker-state) ∈
    set-dist (execute (staged-transcript-program A)
      adversary-initial-state)
  Some (result, final-state) ∈
    set-dist
    (execute verify-monad
      (verifier-state-from-adversary attacker-state
        (staged-proof-transcript data)))
  accepted-transcript-shape
    (verifier-state-from-adversary attacker-state
      (staged-proof-transcript data))
  out
  (staged-alphas data)
  query-idxs
proof –
from accepted-staged-security-experiment-headerE
  [OF wf controlled outcome accepted]
obtain result final-state data attacker-state where
  out-eq: out = Some (result, final-state)
and builder:
  Some (data, attacker-state) ∈
    set-dist (execute (staged-transcript-program A)
      adversary-initial-state)
and verifier:
  Some (result, final-state) ∈
    set-dist
    (execute verify-monad
      (verifier-state-from-adversary attacker-state
        (staged-proof-transcript data)))
and staged-header:
  verifier-header-transcript

```

```

    (verifier-state-from-adversary attacker-state
      (staged-proof-transcript data))
    (staged-trace-root data)
    (staged-trace-fri-roots data)
    (staged-trace-final data)
    (staged-alphas data)
    (staged-degree data)
    (staged-composition-fri-roots data)
    (staged-composition-final data)
    (List.concat (staged-query-chunks data))
  by blast
let ?s =
  verifier-state-from-adversary attacker-state
    (staged-proof-transcript data)
from verify-monad-accepted-transcript-shape[OF verifier]
obtain as query-idxs where shape:
  accepted-transcript-shape ?s (Some (result, final-state))
  as query-idxs
by blast
from shape obtain result' final-state' fr f-fri-roots f-final dg
  composition-fri-roots final rest where
  shape-out: Some (result, final-state) = Some (result', final-state')
and shape-header:
  verifier-header-transcript ?s fr f-fri-roots f-final as dg
  composition-fri-roots final rest
by (elim accepted-transcript-shape-header-query-extraction)
have as-eq: as = staged-alphas data
  using verifier-header-transcript-unique[OF staged-header shape-header]
  by simp
have shape-staged:
  accepted-transcript-shape ?s out (staged-alphas data) query-idxs
  using shape unfolding out-eq as-eq .
show ?thesis
  by (rule that[OF out-eq builder verifier shape-staged])
qed

lemma accepted-staged-security-experiment-query-chunksE:
  assumes wf: staged-budget-wellformed budgets
  and controlled: staged-adversary-controlled budgets A
  and outcome:
    out ∈ set-dist
    (execute (staged-security-experiment A) adversary-initial-state)
  and accepted: ¬ Option.is-none out
  obtains result final-state data attacker-state raw-idxs query-idxs
    query-chunks trailing
  where
    out = Some (result, final-state)
    Some (data, attacker-state) ∈
      set-dist (execute (staged-transcript-program A)

```

```

    adversary-initial-state)
Some (result, final-state) ∈
  set-dist
    (execute verify-monad
      (verifier-state-from-adversary attacker-state
        (staged-proof-transcript data)))
length raw-idxs = rounds
query-idxs = map (λraw. index (to-nat raw)) raw-idxs
length query-chunks = rounds
List.concat query-chunks @ trailing =
  List.concat (staged-query-chunks data)
Λi. i < rounds ⇒
  verifier-query-round-chunk (query-idxs ! i)
    (staged-trace-fri-roots data)
    (staged-composition-fri-roots data)
    (query-chunks ! i)
Λi. i < rounds ⇒
  length (query-chunks ! i) =
    verifier-query-round-transcript-length (query-idxs ! i)
      (staged-trace-fri-roots data)
      (staged-composition-fri-roots data)
Λi. i < rounds ⇒
  fmlookup (HashMap final-state)
    (QueryIndexChallenge i
      (state-after-query-chunks
        (staged-query-start-hash data) query-chunks i)) =
    Some (raw-idxs ! i)
∀ idx ∈ set query-idxs. idx < clength * scale
proof –
from accepted-staged-security-experiment-headerE
  [OF wf controlled outcome accepted]
obtain result final-state data attacker-state where
  out-eq: out = Some (result, final-state)
and builder:
  Some (data, attacker-state) ∈
    set-dist (execute (staged-transcript-program A)
      adversary-initial-state)
and verifier:
  Some (result, final-state) ∈
    set-dist
      (execute verify-monad
        (verifier-state-from-adversary attacker-state
          (staged-proof-transcript data)))
and staged-header:
  verifier-header-transcript
    (verifier-state-from-adversary attacker-state
      (staged-proof-transcript data))
    (staged-trace-root data)
    (staged-trace-fri-roots data)

```

```

    (staged-trace-final data)
    (staged-alphas data)
    (staged-degree data)
    (staged-composition-fri-roots data)
    (staged-composition-final data)
    (List.concat (staged-query-chunks data))
  by blast
let ?s =
  verifier-state-from-adversary attacker-state
  (staged-proof-transcript data)
from verify-monad-accepted-transcript-shape[OF verifier]
obtain as accepted-query-idxs where shape:
  accepted-transcript-shape ?s (Some (result, final-state))
  as accepted-query-idxs
  by blast
from accepted-transcript-shape-query-chunksE[OF shape]
obtain result' final-state' fr f-fri-roots f-final dg
  composition-fri-roots final rest raw-idxs query-chunks trailing
where shape-out:
  Some (result, final-state) = Some (result', final-state')
and header:
  verifier-header-transcript ?s fr f-fri-roots f-final as dg
  composition-fri-roots final rest
and len-raw: length raw-idxs = rounds
and query-idxs-eq:
  accepted-query-idxs = map (λraw. index (to-nat raw)) raw-idxs
and len-chunks: length query-chunks = rounds
and concat-chunks: List.concat query-chunks @ trailing = rest
and chunk-shape:
  ∧ i. i < rounds ⇒
    verifier-query-round-chunk (accepted-query-idxs ! i)
    f-fri-roots composition-fri-roots (query-chunks ! i)
and lookup:
  ∧ i. i < rounds ⇒
    fmlookup (HashMap final-state')
    (QueryIndexChallenge (PQueryCounter ?s + i)
    (state-after-query-chunks
    (verifier-header-state ?s fr f-fri-roots f-final as dg
    composition-fri-roots final)
    query-chunks i)) =
    Some (raw-idxs ! i)
and idx-bound:
  ∀ idx ∈ set accepted-query-idxs. idx < clength * scale
  by blast
have header-eq:
  fr = staged-trace-root data ∧
  f-fri-roots = staged-trace-fri-roots data ∧
  f-final = staged-trace-final data ∧
  as = staged-alphas data ∧

```



```

    dg = staged-degree data ∧
    composition-fri-roots = staged-composition-fri-roots data ∧
    final = staged-composition-final data ∧
    rest = List.concat (staged-query-chunks data)
  using verifier-header-transcript-unique[OF staged-header header]
  by simp
  have final-state-eq: final-state' = final-state
    using shape-out by simp
  have header-state-eq:
    verifier-header-state ?s fr f-fri-roots f-final as dg
    composition-fri-roots final =
    staged-query-start-hash data
  using header-eq
  unfolding verifier-header-state-def verifier-header-messages-def
    staged-query-start-hash-def staged-composition-fri-start-hash-def
    staged-trace-fri-start-hash-def
  by simp
  have lookup':
    ∧i. i < rounds ⇒
    fmlookup (HashMap final-state)
      (QueryIndexChallenge i
        (state-after-query-chunks
          (staged-query-start-hash data) query-chunks i)) =
    Some (raw-idxs ! i)
  using lookup final-state-eq header-state-eq by simp
  have chunk-shape':
    ∧i. i < rounds ⇒
    verifier-query-round-chunk (accepted-query-idxs ! i)
      (staged-trace-fri-roots data)
      (staged-composition-fri-roots data)
      (query-chunks ! i)
  using chunk-shape header-eq by simp
  have chunk-len:
    ∧i. i < rounds ⇒
    length (query-chunks ! i) =
    verifier-query-round-transcript-length (accepted-query-idxs ! i)
      (staged-trace-fri-roots data)
      (staged-composition-fri-roots data)
  by (rule verifier-query-round-chunk-length[OF chunk-shape'])
  show ?thesis
  by (rule that[OF out-eq builder verifier len-raw query-idxs-eq
    len-chunks])
    (use concat-chunks header-eq chunk-shape' chunk-len lookup' idx-bound
      in simp-all)
qed

```

```

lemma staged-transcript-shape-alpha-badD:
  assumes shape:
    accepted-transcript-shape s out staged-as staged-query-idxs

```

and *bad*: *alpha-bad-set-hit* *s* *B* out
 shows $\exists$ *trace-table*. *staged-as* $\in$ *B* *trace-table*
proof –
 from *bad* **obtain** *trace-table* *composition-table* *as* *query-idxs* **where**
 bound:
 accepted-with-bound-tables *s* out *trace-table* *composition-table*
 as *query-idxs*
 and *as-bad*: *as* $\in$ *B* *trace-table*
 unfolding *alpha-bad-set-hit-def* **by** *blast*
 have *shape-bad*: *accepted-transcript-shape* *s* out *as* *query-idxs*
 using *bound* unfolding *accepted-with-bound-tables-def*
 accepted-with-tables-def **by** *simp*
 have *as* = *staged-as*
 by (rule *accepted-transcript-shape-alphas-unique*[*OF* *shape* *shape-bad*])
 then show ?*thesis*
 using *as-bad* **by** *blast*
qed

lemma *staged-security-with-data-alpha-bad-set-hit-imp-alpha-list-hit*:
 assumes *envelope*: $\bigwedge$ *trace-table*. *bad-sets* *trace-table* $\subseteq$ *B*
 and *hit*: *staged-security-with-data-alpha-bad-set-hit* *bad-sets* out
 shows *staged-security-with-data-alpha-list-hit* *B* out
 using *assms*
 unfolding *staged-security-with-data-alpha-bad-set-hit-def*
 staged-security-with-data-alpha-list-hit-def
by (*auto* *split*: *option.splits* *prod.splits*)

lemma *staged-security-with-data-alpha-list-bound-from-transcript*:
 assumes *transcript-bound*:
 wp-event (*staged-transcript-program* *A*)
 (*staged-transcript-alpha-list-hit* *B*) *adversary-initial-state* $\leq$ *C*
 shows
 wp-event (*staged-security-experiment-with-data* *A*)
 (*staged-security-with-data-alpha-list-hit* *B*)
 adversary-initial-state $\leq$ *C*
 unfolding *staged-security-experiment-with-data-def*
proof (rule *wp-event-bind-bound-by-head-event*[*OF* *transcript-bound*])
 show *staged-security-with-data-alpha-list-hit* *B* *None* $\implies$
 staged-transcript-alpha-list-hit *B* *None*
 unfolding *staged-security-with-data-alpha-list-hit-def*
 staged-transcript-alpha-list-hit-def
by *simp*
next
 fix *data* *attacker-state* out
 assume *cont*:
 out $\in$
 set-dist
 (*execute*
 (*get* $\gg$

```

      (λs. put
        (verifier-state-from-adversary s
          (staged-proof-transcript data)) >>=
        (λ-. verify-monad >>=
          (λresult. return (data, result))))
      attacker-state)
  and hit: staged-security-with-data-alpha-list-hit B out
have data-bad: staged-alphas data ∈ B
proof (cases out)
  case None
  then show ?thesis
    using hit unfolding staged-security-with-data-alpha-list-hit-def
    by simp
  next
  case (Some result)
  then obtain data' result' final-state where out-eq:
    out = Some ((data', result'), final-state)
    by (cases result, auto split: prod.splits)
  from cont[unfolded out-eq]
  obtain s3 where
    ret:
      Some ((data', result'), final-state) ∈
        set-dist (execute (return (data, result')) s3)
    by (auto elim!: set-dist-bindE)
  have data'-eq: data' = data
    using ret by simp
  show ?thesis
    using hit unfolding out-eq data'-eq
      staged-security-with-data-alpha-list-hit-def
    by simp
qed
show staged-transcript-alpha-list-hit B
  (Some (data, attacker-state))
  using data-bad unfolding staged-transcript-alpha-list-hit-def by simp
qed

lemma staged-security-with-data-alpha-bad-set-bound-from-alpha-list:
  assumes envelope:  $\bigwedge \text{trace-table. bad-sets trace-table} \subseteq B$ 
  and list-bound:
    wp-event (staged-security-experiment-with-data A)
      (staged-security-with-data-alpha-list-hit B)
      adversary-initial-state  $\leq C$ 
  shows
    wp-event (staged-security-experiment-with-data A)
      (staged-security-with-data-alpha-bad-set-hit bad-sets)
      adversary-initial-state  $\leq C$ 
proof -
  have wp-event (staged-security-experiment-with-data A)
    (staged-security-with-data-alpha-bad-set-hit bad-sets)

```

```

    adversary-initial-state ≤
wp-event (staged-security-experiment-with-data A)
  (staged-security-with-data-alpha-list-hit B)
  adversary-initial-state
by (rule wp-event-mono)
  (rule staged-security-with-data-alpha-bad-set-hit-imp-alpha-list-hit
    [OF envelope])
also have ... ≤ C
  by (rule list-bound)
finally show ?thesis .
qed

```

definition *checked-staged-security-with-data-state-alpha-list-hit*

```

:: 'f list set ⇒
  (((('f staged-proof-data × 'f protocol-channel) × unit list) ×
    'f protocol-channel) option ⇒ bool

```

where

```

checked-staged-security-with-data-state-alpha-list-hit B out ⟷
  (case out of
    None ⇒ False
  | Some (((data, -), -), -) ⇒ staged-alphas data ∈ B)

```

definition *checked-staged-security-with-data-state-alpha-bad-set-hit*

```

:: ('f list ⇒ 'f list set) ⇒
  (((('f staged-proof-data × 'f protocol-channel) × unit list) ×
    'f protocol-channel) option ⇒ bool

```

where

```

checked-staged-security-with-data-state-alpha-bad-set-hit bad-sets out
  ⟷
  (case out of
    None ⇒ False
  | Some (((data, -), -), -) ⇒
    (∃ trace-table. staged-alphas data ∈ bad-sets trace-table))

```

definition *checked-staged-security-with-data-state-alpha-header-supported-bad-set-hit*

```

:: ('f list ⇒ 'f list set) ⇒
  (((('f staged-proof-data × 'f protocol-channel) × unit list) ×
    'f protocol-channel) option ⇒ bool

```

where

```

checked-staged-security-with-data-state-alpha-header-supported-bad-set-hit
  bad-sets out ⟷
  (case out of
    None ⇒ False
  | Some (((data, attacker-state), -), -) ⇒
    (let s =
      verifier-state-from-adversary attacker-state
      (staged-proof-transcript data)
    in staged-alphas data ∈
      alpha-header-supported-union-bad-sets s bad-sets)

```

(staged-trace-root data)
(staged-trace-fri-roots data)
(staged-trace-final data)))

lemma *checked-staged-security-with-data-state-alpha-bad-set-hit-imp-alpha-list-hit:*

assumes *envelope:* $\bigwedge \text{trace-table. bad-sets trace-table} \subseteq B$

and *hit:*

checked-staged-security-with-data-state-alpha-bad-set-hit bad-sets out

shows *checked-staged-security-with-data-state-alpha-list-hit B out*

using *assms*

unfolding *checked-staged-security-with-data-state-alpha-bad-set-hit-def*

checked-staged-security-with-data-state-alpha-list-hit-def

by (*auto split: option.splits prod.splits*)

lemma *checked-staged-security-with-data-state-alpha-list-bound-from-transcript:*

assumes *transcript-bound:*

wp-event (checked-staged-transcript-program A)

(staged-transcript-alpha-list-hit B) adversary-initial-state $\leq C$

shows

wp-event (checked-staged-security-experiment-with-data-state A)

(checked-staged-security-with-data-state-alpha-list-hit B)

adversary-initial-state $\leq C$

unfolding *checked-staged-security-experiment-with-data-state-def*

proof (*rule wp-event-bind-bound-by-head-event[OF transcript-bound]*)

show

checked-staged-security-with-data-state-alpha-list-hit B None $\implies$

staged-transcript-alpha-list-hit B None

unfolding *checked-staged-security-with-data-state-alpha-list-hit-def*

staged-transcript-alpha-list-hit-def

by *simp*

next

fix *data attacker-state out*

assume *cont:*

out $\in$

set-dist

(execute

(get $\gg=$

($\lambda s.$ put

(verifier-state-from-adversary s

(staged-proof-transcript data)) $\gg=$

($\lambda -. \text{verify-monad } \gg=$

($\lambda \text{result. return } ((\text{data}, s), \text{result}))))$

attacker-state)

and *hit:* *checked-staged-security-with-data-state-alpha-list-hit B out*

have *data-bad:* *staged-alphas data $\in B$*

proof (*cases out*)

case *None*

then show *?thesis*

using *hit*

```

    unfolding checked-staged-security-with-data-state-alpha-list-hit-def
    by simp
next
  case (Some result-pack)
  then obtain data' attacker-state' result final-state where out-eq:
    out = Some (((data', attacker-state'), result), final-state)
    by (cases result-pack, auto split: prod.splits)
  from cont[unfolded out-eq]
  obtain saved-state state-after-get where
    ret:
      Some (((data', attacker-state'), result), final-state) ∈
        set-dist (execute (return ((data, saved-state), result))
          final-state)
    by (auto elim!: set-dist-bindE)
  have data'-eq: data' = data
    using ret by simp
  show ?thesis
    using hit
    unfolding out-eq data'-eq
      checked-staged-security-with-data-state-alpha-list-hit-def
    by simp
qed
show staged-transcript-alpha-list-hit B
  (Some (data, attacker-state))
  using data-bad unfolding staged-transcript-alpha-list-hit-def by simp
qed

lemma checked-staged-security-with-data-state-alpha-bad-set-bound-from-alpha-list:
  assumes envelope:  $\bigwedge \text{trace-table. bad-sets trace-table} \subseteq B$ 
  and list-bound:
    wp-event (checked-staged-security-experiment-with-data-state A)
      (checked-staged-security-with-data-state-alpha-list-hit B)
      adversary-initial-state  $\leq C$ 
  shows
    wp-event (checked-staged-security-experiment-with-data-state A)
      (checked-staged-security-with-data-state-alpha-bad-set-hit bad-sets)
      adversary-initial-state  $\leq C$ 
proof –
  have wp-event (checked-staged-security-experiment-with-data-state A)
    (checked-staged-security-with-data-state-alpha-bad-set-hit bad-sets)
    adversary-initial-state  $\leq$ 
  wp-event (checked-staged-security-experiment-with-data-state A)
    (checked-staged-security-with-data-state-alpha-list-hit B)
    adversary-initial-state
  by (rule wp-event-mono)
  (rule
    checked-staged-security-with-data-state-alpha-bad-set-hit-imp-alpha-list-hit
    [OF envelope])
also have ...  $\leq C$ 

```

by (rule list-bound)
 finally show ?thesis .
 qed

lemma *checked-staged-security-with-data-state-composition-randomization-bad-imp-alpha-header-supported-bad-set-h*

assumes *wf: staged-budget-wellformed budgets*

and *controlled: staged-adversary-controlled budgets A*

and *support:*

out $\in$

set-dist

(*execute (checked-staged-security-experiment-with-data-state A)*
adversary-initial-state)

and *bad:*

staged-security-with-data-state-verifier-event
composition-randomization-bad out

shows

checked-staged-security-with-data-state-alpha-header-supported-bad-set-hit
composition-trace-bad-alpha-space out

proof (cases *out*)

case *None*

then show ?thesis

using *bad*

unfolding *staged-security-with-data-state-verifier-event-def*

by *simp*

next

case (*Some result-pack*)

then obtain *data attacker-state result final-state* **where** *out-eq:*

out = Some (((data, attacker-state), result), final-state)

by (cases *result-pack*, auto *split: prod.splits*)

let ?s =

verifier-state-from-adversary attacker-state
(staged-proof-transcript data)

have *support-some:*

Some (((data, attacker-state), result), final-state) $\in$
set-dist

(*execute (checked-staged-security-experiment-with-data-state A)*
adversary-initial-state)

using *support* **unfolding** *out-eq* .

from *checked-staged-security-experiment-with-data-state-outcomeE*

[*OF support-some*]

have *builder:*

Some (data, attacker-state) $\in$
set-dist

(*execute (checked-staged-transcript-program A)*
adversary-initial-state)

and *verifier:*

Some (result, final-state) $\in$ set-dist (execute verify-monad ?s)

by *blast+*

have *staged-header:*

```

verifier-header-transcript ?s
  (staged-trace-root data)
  (staged-trace-fri-roots data)
  (staged-trace-final data)
  (staged-alphas data)
  (staged-degree data)
  (staged-composition-fri-roots data)
  (staged-composition-final data)
  (List.concat (staged-query-chunks data))
by (rule checked-staged-transcript-program-outcome-header-transcript
  [OF wf controlled builder])
from bad[unfolded out-eq
  staged-security-with-data-state-verifier-event-def]
have random-bad:
  composition-randomization-bad ?s (Some (result, final-state))
by simp
have alpha-hit:
  alpha-bad-set-hit ?s composition-trace-bad-alpha-space
  (Some (result, final-state))
by (rule composition-randomization-bad-imp-alpha-bad-set-hit
  [OF random-bad])
from alpha-hit obtain trace-table composition-table as query-idxs where
  bound:
    accepted-with-bound-tables ?s (Some (result, final-state))
    trace-table composition-table as query-idxs
and as-bad: as ∈ composition-trace-bad-alpha-space trace-table
unfolding alpha-bad-set-hit-def by blast
have shape:
  accepted-transcript-shape ?s (Some (result, final-state))
  as query-idxs
using bound
unfolding accepted-with-bound-tables-def accepted-with-tables-def
by simp
from shape obtain result' final-state' fr f-fri-roots f-final dg
  composition-fri-roots final rest where
  header:
    verifier-header-transcript ?s fr f-fri-roots f-final as dg
    composition-fri-roots final rest
by (elim accepted-transcript-shape-header-query-extraction)
have as-eq: as = staged-alphas data
using verifier-header-transcript-unique[OF staged-header header]
by simp
have bound-staged:
  accepted-with-bound-tables ?s (Some (result, final-state))
  trace-table composition-table (staged-alphas data) query-idxs
using bound as-eq by simp
have staged-bad:
  staged-alphas data ∈ composition-trace-bad-alpha-space trace-table
using as-bad as-eq by simp

```



```

have candidate:
  trace-table  $\in$ 
    alpha-header-supported-trace-table-candidates ?s
    (staged-trace-root data)
    (staged-trace-fri-roots data)
    (staged-trace-final data)
  unfolding alpha-header-supported-trace-table-candidates-def
  using verifier bound-staged staged-header by blast
have union-bad:
  staged-alphas data  $\in$ 
    alpha-header-supported-union-bad-sets ?s
    composition-trace-bad-alpha-space
    (staged-trace-root data)
    (staged-trace-fri-roots data)
    (staged-trace-final data)
  unfolding alpha-header-supported-union-bad-sets-def
  using candidate staged-bad
    composition-trace-bad-alpha-space-subset-alpha-space[of trace-table]
  by auto
show ?thesis
  using union-bad
  unfolding out-eq
    checked-staged-security-with-data-state-alpha-header-supported-bad-set-hit-def
  by simp
qed

```

lemma checked-staged-security-with-data-state-composition-randomization-bad-bound-from-alpha-header-supported-b

```

assumes wf: staged-budget-wellformed budgets
and controlled: staged-adversary-controlled budgets A
and alpha-bound:
  wp-event (checked-staged-security-experiment-with-data-state A)
  (checked-staged-security-with-data-state-alpha-header-supported-bad-set-hit
    composition-trace-bad-alpha-space)
  adversary-initial-state  $\leq C$ 

```

shows

```

  wp-event (checked-staged-security-experiment-with-data-state A)
  (staged-security-with-data-state-verifier-event
    composition-randomization-bad)
  adversary-initial-state  $\leq C$ 

```

proof –

```

have wp-event (checked-staged-security-experiment-with-data-state A)
  (staged-security-with-data-state-verifier-event
    composition-randomization-bad)
  adversary-initial-state  $\leq$ 
  wp-event (checked-staged-security-experiment-with-data-state A)
  (checked-staged-security-with-data-state-alpha-header-supported-bad-set-hit
    composition-trace-bad-alpha-space)
  adversary-initial-state
by (rule wp-event-mono-on-support)

```

```

    (rule
      checked-staged-security-with-data-state-composition-randomization-bad-imp-alpha-header-supported-bad-set-hit-on-support
      [OF wf controlled])
  also have ... ≤ C
    by (rule alpha-bound)
  finally show ?thesis .
qed

lemma checked-staged-security-with-data-state-composition-randomization-bad-imp-alpha-bad-set-hit-on-support:
  assumes wf: staged-budget-wellformed budgets
  and controlled: staged-adversary-controlled budgets A
  and support:
    out ∈
      set-dist
        (execute (checked-staged-security-experiment-with-data-state A)
          adversary-initial-state)
  and bad:
    staged-security-with-data-state-verifier-event
    composition-randomization-bad out
  shows
    checked-staged-security-with-data-state-alpha-bad-set-hit
    composition-trace-bad-alpha-space out
proof (cases out)
case None
  then show ?thesis
    using bad
    unfolding staged-security-with-data-state-verifier-event-def
    by simp
next
case (Some result-pack)
  then obtain data attacker-state result final-state where out-eq:
    out = Some (((data, attacker-state), result), final-state)
    by (cases result-pack, auto split: prod.splits)
  let ?s =
    verifier-state-from-adversary attacker-state
    (staged-proof-transcript data)
  have support-some:
    Some (((data, attacker-state), result), final-state) ∈
      set-dist
        (execute (checked-staged-security-experiment-with-data-state A)
          adversary-initial-state)
    using support unfolding out-eq .
  from checked-staged-security-experiment-with-data-state-outcomeE
  [OF support-some]
  have builder:
    Some (data, attacker-state) ∈
      set-dist
        (execute (checked-staged-transcript-program A)
          adversary-initial-state)

```

```

by blast
have staged-header:
  verifier-header-transcript ?s
    (staged-trace-root data)
    (staged-trace-fri-roots data)
    (staged-trace-final data)
    (staged-alphas data)
    (staged-degree data)
    (staged-composition-fri-roots data)
    (staged-composition-final data)
    (List.concat (staged-query-chunks data))
by (rule checked-staged-transcript-program-outcome-header-transcript
      [OF wf controlled builder])
from bad[unfolded out-eq
  staged-security-with-data-state-verifier-event-def]
have random-bad:
  composition-randomization-bad ?s (Some (result, final-state))
by simp
have alpha-hit:
  alpha-bad-set-hit ?s composition-trace-bad-alpha-space
    (Some (result, final-state))
by (rule composition-randomization-bad-imp-alpha-bad-set-hit
      [OF random-bad])
from alpha-hit obtain trace-table composition-table as query-idxs where
  bound:
    accepted-with-bound-tables ?s (Some (result, final-state))
    trace-table composition-table as query-idxs
and as-bad: as ∈ composition-trace-bad-alpha-space trace-table
unfolding alpha-bad-set-hit-def by blast
have shape:
  accepted-transcript-shape ?s (Some (result, final-state))
    as query-idxs
using bound
unfolding accepted-with-bound-tables-def accepted-with-tables-def
by simp
from shape obtain result' final-state' fr f-fri-roots f-final dg
  composition-fri-roots final rest where
  header:
    verifier-header-transcript ?s fr f-fri-roots f-final as dg
    composition-fri-roots final rest
by (elim accepted-transcript-shape-header-query-extraction)
have as-eq: as = staged-alphas data
using verifier-header-transcript-unique[OF staged-header header]
by simp
have staged-bad:
  staged-alphas data ∈ composition-trace-bad-alpha-space trace-table
using as-bad as-eq by simp
show ?thesis
using staged-bad

```

```

unfolding out-eq
  checked-staged-security-with-data-state-alpha-bad-set-hit-def
by auto
qed

lemma checked-staged-security-with-data-state-composition-randomization-bad-bound-from-alpha-bad-set:
  assumes wf: staged-budget-wellformed budgets
  and controlled: staged-adversary-controlled budgets A
  and alpha-bound:
    wp-event (checked-staged-security-experiment-with-data-state A)
    (checked-staged-security-with-data-state-alpha-bad-set-hit
      composition-trace-bad-alpha-space)
    adversary-initial-state  $\leq C$ 
  shows
    wp-event (checked-staged-security-experiment-with-data-state A)
    (staged-security-with-data-state-verifier-event
      composition-randomization-bad)
    adversary-initial-state  $\leq C$ 
  proof -
    have wp-event (checked-staged-security-experiment-with-data-state A)
      (staged-security-with-data-state-verifier-event
        composition-randomization-bad)
      adversary-initial-state  $\leq$ 
    wp-event (checked-staged-security-experiment-with-data-state A)
      (checked-staged-security-with-data-state-alpha-bad-set-hit
        composition-trace-bad-alpha-space)
      adversary-initial-state
    by (rule wp-event-mono-on-support)
      (rule
        checked-staged-security-with-data-state-composition-randomization-bad-imp-alpha-bad-set-hit-on-support
        [OF wf controlled])
    also have ...  $\leq C$ 
    by (rule alpha-bound)
    finally show ?thesis .
  qed

lemma checked-staged-security-with-data-state-composition-degree-bad-bound:
  shows
    wp-event (checked-staged-security-experiment-with-data-state A)
      (staged-security-with-data-state-verifier-event composition-degree-bad)
      adversary-initial-state  $\leq 0$ 
  proof -
    have event-eq:
      staged-security-with-data-state-verifier-event composition-degree-bad =
        ( $\lambda$ out :: ((( $\lambda$ f staged-proof-data  $\times$   $\lambda$ f protocol-channel)  $\times$ 
          unit list)  $\times$   $\lambda$ f protocol-channel) option. False)
    by (rule ext)
      (auto simp: staged-security-with-data-state-verifier-event-def
        composition-degree-bad-false[OF spec-degree-wellformed-from-spec-query-margin])

```

```

    split: option.splits prod.splits)
show ?thesis
  unfolding event-eq wp-event-def wp-def dist-expect-def by simp
qed

lemma checked-staged-security-with-data-state-composition-bad-bound-from-alpha-bad-set:
  assumes wf: staged-budget-wellformed budgets
  and controlled: staged-adversary-controlled budgets A
  and alpha-bound:
    wp-event (checked-staged-security-experiment-with-data-state A)
      (checked-staged-security-with-data-state-alpha-bad-set-hit
        composition-trace-bad-alpha-space)
    adversary-initial-state ≤ composition-error-bound
  shows
    wp-event (checked-staged-security-experiment-with-data-state A)
      (staged-security-with-data-state-verifier-event composition-bad)
    adversary-initial-state ≤ composition-error-bound
proof -
  let ?M = checked-staged-security-experiment-with-data-state A
  let ?degree =
    staged-security-with-data-state-verifier-event composition-degree-bad
  let ?random =
    staged-security-with-data-state-verifier-event
    composition-randomization-bad
  have event-eq:
    staged-security-with-data-state-verifier-event composition-bad =
      (λout. ?degree out ∨ ?random out)
  by (rule ext)
    (auto simp: staged-security-with-data-state-verifier-event-def
      composition-bad-def split: option.splits prod.splits)
  have degree-bound:
    wp-event ?M ?degree adversary-initial-state ≤ 0
  by (rule
    checked-staged-security-with-data-state-composition-degree-bad-bound)
  have random-bound:
    wp-event ?M ?random adversary-initial-state ≤
      composition-error-bound
  by (rule
    checked-staged-security-with-data-state-composition-randomization-bad-bound-from-alpha-bad-set
    [OF wf controlled alpha-bound])
  have wp-event ?M
    (staged-security-with-data-state-verifier-event composition-bad)
    adversary-initial-state ≤
    wp-event ?M ?degree adversary-initial-state +
    wp-event ?M ?random adversary-initial-state
  unfolding event-eq by (rule wp-event-union-bound)
  also have ... ≤ 0 + composition-error-bound
  by (intro add-mono degree-bound random-bound)
  finally show ?thesis by simp

```

qed

lemma *checked-staged-security-with-data-state-composition-bad-bound-from-alpha-header-supported-bad-set:*

assumes *wf: staged-budget-wellformed budgets*
and *controlled: staged-adversary-controlled budgets A*
and *alpha-bound:*
 wp-event (checked-staged-security-experiment-with-data-state A)
 (checked-staged-security-with-data-state-alpha-header-supported-bad-set-hit
 composition-trace-bad-alpha-space)
 adversary-initial-state ≤ composition-error-bound

shows

wp-event (checked-staged-security-experiment-with-data-state A)
 (staged-security-with-data-state-verifier-event composition-bad)
 adversary-initial-state ≤ composition-error-bound

proof –

let *?M = checked-staged-security-experiment-with-data-state A*
let *?degree =*
 staged-security-with-data-state-verifier-event composition-degree-bad
let *?random =*
 staged-security-with-data-state-verifier-event
 composition-randomization-bad

have *event-eq:*

staged-security-with-data-state-verifier-event composition-bad =
 (λout. ?degree out ∨ ?random out)

by *(rule ext)*

(auto simp: staged-security-with-data-state-verifier-event-def
 composition-bad-def split: option.splits prod.splits)

have *degree-bound:*

wp-event ?M ?degree adversary-initial-state ≤ 0

by *(rule*

checked-staged-security-with-data-state-composition-degree-bad-bound)

have *random-bound:*

wp-event ?M ?random adversary-initial-state ≤
 composition-error-bound

by *(rule*

checked-staged-security-with-data-state-composition-randomization-bad-bound-from-alpha-header-supported-bad-s
 [OF wf controlled alpha-bound])

have *wp-event ?M*

(staged-security-with-data-state-verifier-event composition-bad)
 adversary-initial-state ≤

wp-event ?M ?degree adversary-initial-state +

wp-event ?M ?random adversary-initial-state

unfolding *event-eq* **by** *(rule wp-event-union-bound)*

also have *... ≤ 0 + composition-error-bound*

by *(intro add-mono degree-bound random-bound)*

finally show *?thesis* **by** *simp*

qed

lemma *checked-staged-security-with-data-state-composition-bad-bound-from-alpha-header-supported-bad-set-general:*

```

assumes wf: staged-budget-wellformed budgets
and controlled: staged-adversary-controlled budgets A
and alpha-bound:
  wp-event (checked-staged-security-experiment-with-data-state A)
    (checked-staged-security-with-data-state-alpha-header-supported-bad-set-hit
      composition-trace-bad-alpha-space)
  adversary-initial-state  $\leq C$ 
shows
  wp-event (checked-staged-security-experiment-with-data-state A)
    (staged-security-with-data-state-verifier-event composition-bad)
  adversary-initial-state  $\leq C$ 
proof –
  let ?M = checked-staged-security-experiment-with-data-state A
  let ?degree =
    staged-security-with-data-state-verifier-event composition-degree-bad
  let ?random =
    staged-security-with-data-state-verifier-event
    composition-randomization-bad
  have event-eq:
    staged-security-with-data-state-verifier-event composition-bad =
      ( $\lambda$ out. ?degree out  $\vee$  ?random out)
  by (rule ext)
    (auto simp: staged-security-with-data-state-verifier-event-def
      composition-bad-def split: option.splits prod.splits)
  have degree-bound:
    wp-event ?M ?degree adversary-initial-state  $\leq 0$ 
  by (rule
    checked-staged-security-with-data-state-composition-degree-bad-bound)
  have random-bound:
    wp-event ?M ?random adversary-initial-state  $\leq C$ 
  by (rule
    checked-staged-security-with-data-state-composition-randomization-bad-bound-from-alpha-header-supported-bad-s
    [OF wf controlled alpha-bound])
  have wp-event ?M
    (staged-security-with-data-state-verifier-event composition-bad)
    adversary-initial-state  $\leq$ 
    wp-event ?M ?degree adversary-initial-state +
    wp-event ?M ?random adversary-initial-state
  unfolding event-eq by (rule wp-event-union-bound)
  also have ...  $\leq 0 + C$ 
  by (intro add-mono degree-bound random-bound)
  finally show ?thesis by simp
qed

```

lemma *staged-composition-alpha-header-supported-union-fraction-bound-if-no-pairwise-merkle-bad*:

```

assumes no-bad:
   $\neg$  alpha-supported-pairwise-merkle-bad
  (verifier-state-from-adversary attacker-state
    (staged-proof-transcript data))

```

shows

nnreal
 (*card*
 (*alpha-header-supported-union-bad-sets*
 (*verifier-state-from-adversary attacker-state*
 (*staged-proof-transcript data*)))
 composition-trace-bad-alpha-space
 (*staged-trace-root data*)
 (*staged-trace-fri-roots data*)
 (*staged-trace-final data*))) /
 nnreal (card alpha-space) ≤ composition-error-bound
by (*rule*
 composition-trace-alpha-header-supported-union-fraction-bound-if-unique-candidate)
 (*rule*
 alpha-header-supported-candidate-unique-if-no-global-pairwise-merkle-bad
 [*OF no-bad*])

lemma *accepted-staged-security-experiment-fri-challengesE*:

assumes *wf*: *staged-budget-wellformed budgets*
and *controlled*: *staged-adversary-controlled budgets A*
and *outcome*:
 out ∈ set-dist
 (*execute (staged-security-experiment A) adversary-initial-state*)
and *accepted*: $\neg \text{Option.is-none } out$
obtains *result final-state data attacker-state* **where**
 out = Some (result, final-state)
 Some (data, attacker-state) ∈
 set-dist (execute (staged-transcript-program A)
 adversary-initial-state)
 Some (result, final-state) ∈
 set-dist
 (*execute verify-monad*
 (*verifier-state-from-adversary attacker-state*
 (*staged-proof-transcript data*))))
 accepted-fri-challenges
 (*verifier-state-from-adversary attacker-state*
 (*staged-proof-transcript data*))
 out
 (*staged-trace-fri-challenges data*)
 (*staged-degree data*)
 (*staged-composition-fri-challenges data*)

proof –

from *accepted-staged-security-experiment-headerE*
 [*OF wf controlled outcome accepted*]
obtain *result final-state data attacker-state* **where**
 out-eq: *out = Some (result, final-state)*
and *builder*:
 Some (data, attacker-state) ∈
 set-dist (execute (staged-transcript-program A)


```

    adversary-initial-state)
and verifier:
    Some (result, final-state) ∈
    set-dist
    (execute verify-monad
    (verifier-state-from-adversary attacker-state
    (staged-proof-transcript data)))
and staged-header:
    verifier-header-transcript
    (verifier-state-from-adversary attacker-state
    (staged-proof-transcript data))
    (staged-trace-root data)
    (staged-trace-fri-roots data)
    (staged-trace-final data)
    (staged-alphas data)
    (staged-degree data)
    (staged-composition-fri-roots data)
    (staged-composition-final data)
    (List.concat (staged-query-chunks data))
by blast
let ?s =
    verifier-state-from-adversary attacker-state
    (staged-proof-transcript data)
from verify-monad-header-random-oracle-replay[OF verifier]
obtain fr f-fl f-final as dg fl final query-state where
    header:
    verifier-header-transcript ?s fr (map snd f-fl) f-final as dg
    (map snd fl) final (PTranscript query-state)
and query-state-eq:
    PState query-state =
    verifier-header-state ?s fr (map snd f-fl) f-final as dg
    (map snd fl) final
and query-out:
    Some (result, final-state) ∈
    set-dist (execute
    (ntimes
    (verifier-query-round-program fr f-fl f-final as fl final)
    rounds)
    query-state)
and query-count-header: PQueryCounter query-state = PQueryCounter ?s
and ext-s-query: ?s ≤ query-state
and trace-lookup:
     $\bigwedge i. i < \text{length } f\text{-fl} \implies$ 
    fmlookup (HashMap query-state)
    (TraceFriChallenge (PTraceFriCounter ?s + i)
    (foldl concat (concat (PState ?s) fr)
    (take (Suc i) (map snd f-fl)))) =
    Some (fst (f-fl ! i))
and alpha-lookup:

```

```

 $\bigwedge i. i < \text{length } as \implies$ 
  fmllookup (HashMap query-state)
    (AlphaChallenge (PAlphaCounter ?s + i) (foldl concat
      (concat (foldl concat (concat (PState ?s) fr) (map snd f-fl))
        f-final)
      (take i as))) =
    Some (as ! i)
and comp-lookup:
 $\bigwedge i. i < \text{length } fl \implies$ 
  fmllookup (HashMap query-state)
    (CompositionFriChallenge (PCompositionFriCounter ?s + i)
      (foldl concat
        (concat
          (foldl concat
            (concat
              (foldl concat (concat (PState ?s) fr) (map snd f-fl))
                f-final)
            as)
          dg)
        (take (Suc i) (map snd fl)))) =
    Some (fst (fl ! i))
and degree-bound: to-nat dg  $\leq$  maxDegree
by blast
have header-unique:
  fr = staged-trace-root data  $\wedge$ 
  map snd f-fl = staged-trace-fri-roots data  $\wedge$ 
  f-final = staged-trace-final data  $\wedge$ 
  as = staged-alphas data  $\wedge$ 
  dg = staged-degree data  $\wedge$ 
  map snd fl = staged-composition-fri-roots data  $\wedge$ 
  final = staged-composition-final data  $\wedge$ 
  PTranscript query-state = List.concat (staged-query-chunks data)
using verifier-header-transcript-unique[OF staged-header header] by simp
have shape:
  length (staged-trace-fri-roots data) = ceil-log clength  $\wedge$ 
  length (staged-trace-fri-challenges data) = ceil-log clength  $\wedge$ 
  length (staged-alphas data) = length spec  $\wedge$ 
  length (staged-composition-fri-roots data) =
    ceil-log (to-nat (staged-degree data) + 1)  $\wedge$ 
  length (staged-composition-fri-challenges data) =
    ceil-log (to-nat (staged-degree data) + 1)  $\wedge$ 
  ceil-log (to-nat (staged-degree data) + 1)  $\leq$ 
    ceil-log (maxDegree + 1)  $\wedge$ 
  length (staged-query-chunks data) = rounds
by (rule staged-transcript-program-outcome-shape
  [OF wf controlled builder])
have trace-bs-eq:
  map fst f-fl = staged-trace-fri-challenges data
proof (rule nth-equalityI)

```

```

show length (map fst f-fl) =
  length (staged-trace-fri-challenges data)
  using header-unique shape by (metis length-map)
next
fix i
assume i-bound:  $i < \text{length (map fst f-fl)}$ 
then have i-f-fl:  $i < \text{length f-fl}$ 
  by simp
have i-round:  $i < \text{ceil-log clength}$ 
  using i-f-fl header-unique shape by (metis length-map)
have staged-lookup-attacker:
  fmlookup (HashMap attacker-state)
    (TraceFriChallenge i
      (foldl concat (staged-trace-fri-start-hash data)
        (take (Suc i) (staged-trace-fri-roots data)))) =
    Some (staged-trace-fri-challenges data ! i)
  by (rule staged-transcript-program-outcome-trace-challenge-lookup
    [OF wf controlled builder i-round])
have staged-lookup-s:
  fmlookup (HashMap ?s)
    (TraceFriChallenge i
      (foldl concat (staged-trace-fri-start-hash data)
        (take (Suc i) (staged-trace-fri-roots data)))) =
    Some (staged-trace-fri-challenges data ! i)
  using staged-lookup-attacker by simp
have staged-lookup-query:
  fmlookup (HashMap query-state)
    (TraceFriChallenge i
      (foldl concat (staged-trace-fri-start-hash data)
        (take (Suc i) (staged-trace-fri-roots data)))) =
    Some (staged-trace-fri-challenges data ! i)
  by (rule hash-extension-lookup[OF staged-lookup-s ext-s-query])
have verifier-lookup-query:
  fmlookup (HashMap query-state)
    (TraceFriChallenge i
      (foldl concat (staged-trace-fri-start-hash data)
        (take (Suc i) (staged-trace-fri-roots data)))) =
    Some (fst (f-fl ! i))
  using trace-lookup[OF i-f-fl] header-unique
  unfolding staged-trace-fri-start-hash-def
  by simp
show map fst f-fl ! i = staged-trace-fri-challenges data ! i
  using verifier-lookup-query staged-lookup-query i-f-fl
  by simp
qed
have comp-bs-eq:
  map fst fl = staged-composition-fri-challenges data
proof (rule nth-equalityI)
  show length (map fst fl) =

```

```

    length (staged-composition-fri-challenges data)
  using header-unique shape by (metis length-map)
next
fix i
assume i-bound: i < length (map fst fl)
then have i-fl: i < length fl
  by simp
have i-round:
  i < ceil-log (to-nat (staged-degree data) + 1)
  using i-fl header-unique shape by (metis length-map)
have staged-lookup-attacker:
  fmlookup (HashMap attacker-state)
    (CompositionFriChallenge i
      (foldl concat (staged-composition-fri-start-hash data)
        (take (Suc i) (staged-composition-fri-roots data)))) =
    Some (staged-composition-fri-challenges data ! i)
  by (rule staged-transcript-program-outcome-composition-challenge-lookup
    [OF wf controlled builder i-round])
have staged-lookup-s:
  fmlookup (HashMap ?s)
    (CompositionFriChallenge i
      (foldl concat (staged-composition-fri-start-hash data)
        (take (Suc i) (staged-composition-fri-roots data)))) =
    Some (staged-composition-fri-challenges data ! i)
  using staged-lookup-attacker by simp
have staged-lookup-query:
  fmlookup (HashMap query-state)
    (CompositionFriChallenge i
      (foldl concat (staged-composition-fri-start-hash data)
        (take (Suc i) (staged-composition-fri-roots data)))) =
    Some (staged-composition-fri-challenges data ! i)
  by (rule hash-extension-lookup[OF staged-lookup-s ext-s-query])
have verifier-lookup-query:
  fmlookup (HashMap query-state)
    (CompositionFriChallenge i
      (foldl concat (staged-composition-fri-start-hash data)
        (take (Suc i) (staged-composition-fri-roots data)))) =
    Some (fst (fl ! i))
  using comp-lookup[OF i-fl] header-unique
  unfolding staged-composition-fri-start-hash-def
    staged-trace-fri-start-hash-def
  by simp
show map fst fl ! i =
  staged-composition-fri-challenges data ! i
  using verifier-lookup-query staged-lookup-query i-fl
  by simp
qed
have trace-lookup-all:
  ∀ i < length f-fl.

```

```

    fmlookup (HashMap query-state)
      (TraceFriChallenge (PTraceFriCounter ?s + i)
        (foldl concat (concat (PState ?s) fr)
          (take (Suc i) (map snd f-fl)))) =
    Some (fst (f-fl ! i))
  using trace-lookup by blast
have comp-lookup-all:
  ∀ i < length fl.
    fmlookup (HashMap query-state)
      (CompositionFriChallenge (PCompositionFriCounter ?s + i)
        (foldl concat
          (concat
            (foldl concat
              (concat
                (foldl concat (concat (PState ?s) fr) (map snd f-fl))
                f-final)
              as)
            dg)
          (take (Suc i) (map snd fl)))) =
    Some (fst (fl ! i))
  using comp-lookup by blast
have challenges:
  accepted-fri-challenges ?s (Some (result, final-state))
    (staged-trace-fri-challenges data)
    (staged-degree data)
    (staged-composition-fri-challenges data)
  unfolding accepted-fri-challenges-def
  by (intro exI[of - result] exI[of - final-state] exI[of - fr]
    exI[of - f-fl] exI[of - f-final] exI[of - as] exI[of - fl]
    exI[of - final] exI[of - query-state])
    (use header query-state-eq query-out query-count-header
      header-unique trace-bs-eq comp-bs-eq trace-lookup-all comp-lookup-all
      in simp)
have challenges-out:
  accepted-fri-challenges ?s out
    (staged-trace-fri-challenges data)
    (staged-degree data)
    (staged-composition-fri-challenges data)
  using challenges out-eq by simp
show ?thesis
  by (rule that[OF out-eq builder verifier challenges-out])
qed

lemma staged-budget-trace-length:
  assumes staged-budget-wellformed budgets
  shows length (trace-fri-budgets budgets) = ceil-log clength
  using assms unfolding staged-budget-wellformed-def by simp

lemma staged-budget-composition-length:

```

assumes *staged-budget-wellformed budgets*
shows
 $\text{length } (\text{composition-fri-budgets budgets}) =$
 $\text{ceil-log } (\text{maxDegree} + 1)$
using *assms unfolding staged-budget-wellformed-def by simp*

lemma *staged-budget-query-length:*
assumes *staged-budget-wellformed budgets*
shows $\text{length } (\text{query-opening-budgets budgets}) = \text{rounds}$
using *assms unfolding staged-budget-wellformed-def by simp*

lemma *staged-trace-fri-search-queries-le-total:*
assumes *wf: staged-budget-wellformed budgets*
and *i-bound: $i < \text{ceil-log clength}$*
shows
 $\text{staged-trace-fri-search-queries budgets } i \leq$
 $\text{staged-attacker-query-budget budgets} + \text{staged-challenge-query-budget}$
proof –
have *take-le:*
 $\text{sum-list } (\text{take } (\text{Suc } i) (\text{trace-fri-budgets budgets})) \leq$
 $\text{sum-list } (\text{trace-fri-budgets budgets})$
by *(rule sum-list-take-le)*
have *i-le: $i \leq \text{ceil-log clength}$*
using *i-bound by simp*
show *?thesis*
unfolding *staged-trace-fri-search-queries-def*
staged-attacker-query-budget-def staged-challenge-query-budget-def
using *take-le i-le by simp*
qed

lemma *staged-alpha-search-queries-le-total:*
assumes *$i \leq \text{length spec}$*
shows
 $\text{staged-alpha-search-queries budgets } i \leq$
 $\text{staged-attacker-query-budget budgets} + \text{staged-challenge-query-budget}$
unfolding *staged-alpha-search-queries-def*
staged-attacker-query-budget-def staged-challenge-query-budget-def
using *assms by simp*

lemma *staged-composition-fri-search-queries-le-total:*
assumes *wf: staged-budget-wellformed budgets*
and *i-bound: $i < \text{ceil-log } (\text{maxDegree} + 1)$*
shows
 $\text{staged-composition-fri-search-queries budgets } i \leq$
 $\text{staged-attacker-query-budget budgets} + \text{staged-challenge-query-budget}$
proof –
have *take-le:*
 $\text{sum-list } (\text{take } (\text{Suc } i) (\text{composition-fri-budgets budgets})) \leq$
 $\text{sum-list } (\text{composition-fri-budgets budgets})$

```

    by (rule sum-list-take-le)
  have i-le:  $i \leq \text{ceil-log } (\text{maxDegree} + 1)$ 
    using i-bound by simp
  show ?thesis
    unfolding staged-composition-fri-search-queries-def
      staged-attacker-query-budget-def staged-challenge-query-budget-def
    using take-le i-le by simp
qed

lemma staged-query-search-queries-le-total:
  assumes wf: staged-budget-wellformed budgets
    and i-bound:  $i < \text{rounds}$ 
  shows
    staged-query-search-queries budgets  $i \leq$ 
      staged-attacker-query-budget budgets + staged-challenge-query-budget
proof -
  have take-le:
    sum-list (take i (query-opening-budgets budgets))  $\leq$ 
      sum-list (query-opening-budgets budgets)
    by (rule sum-list-take-le)
  have i-le:  $i \leq \text{rounds}$ 
    using i-bound by simp
  show ?thesis
    unfolding staged-query-search-queries-def
      staged-attacker-query-budget-def staged-challenge-query-budget-def
    using take-le i-le by simp
qed

lemma staged-phase-relation-error-mono:
  assumes  $q \leq r$ 
  shows
    staged-phase-relation-error fiber-bound  $q \leq$ 
      staged-phase-relation-error fiber-bound  $r$ 
  unfolding staged-phase-relation-error-def
  by (rule hash-relation-budget-value-mono[OF assms])

lemma staged-phase-target-error-mono:
  assumes  $q \leq r$ 
  shows staged-phase-target-error  $B$   $q \leq$  staged-phase-target-error  $B$   $r$ 
  unfolding staged-phase-target-error-def
  by (rule hash-target-budget-value-mono[OF assms])

lemma staged-phase-target-error-card-mono:
  assumes card  $B \leq b$ 
  shows staged-phase-target-error  $B$   $q \leq$ 
    staged-phase-relation-error  $b$   $q$ 
  unfolding staged-phase-target-error-def staged-phase-relation-error-def
    hash-target-budget-value-def hash-relation-budget-value-def
  apply (subst nn2real-le-iff[symmetric])

```

using *assms* **by** (*simp add: divide-right-mono mult-left-mono*)

lemma *staged-dynamic-query-set-subset:*

query-header-supported-union-good-sets
 (*verifier-state-from-adversary attacker-state*
 (*staged-proof-transcript data*))
query-sampling-success-space
 (*staged-trace-root data*)
 (*staged-trace-fri-roots data*)
 (*staged-trace-final data*)
 (*staged-alphas data*)
 (*staged-degree data*)
 (*staged-composition-fri-roots data*)
 (*staged-composition-final data*) $\subseteq$ *query-sample-space*
by (*rule query-header-supported-union-good-sets-subset*)

lemma *staged-dynamic-query-set-fraction-bound-if-no-pairwise-merkle-bad:*

assumes *no-bad:*

$\neg$ *query-supported-pairwise-merkle-bad*
 (*verifier-state-from-adversary attacker-state*
 (*staged-proof-transcript data*))

shows

nnreal
 (*card*
 (*query-header-supported-union-good-sets*
 (*verifier-state-from-adversary attacker-state*
 (*staged-proof-transcript data*))
query-sampling-success-space
 (*staged-trace-root data*)
 (*staged-trace-fri-roots data*)
 (*staged-trace-final data*)
 (*staged-alphas data*)
 (*staged-degree data*)
 (*staged-composition-fri-roots data*)
 (*staged-composition-final data*))) /
nnreal (*card query-sample-space*) $\leq$ *query-error-bound*

by (*rule*

query-header-supported-union-good-sets-fraction-bound-if-no-global-pairwise-merkle-bad
 [*OF no-bad query-sampling-success-space-subset*
query-sampling-success-space-fraction-bound-query-sample-space])

lemma *staged-dynamic-query-set-target-error-bound-if-no-pairwise-merkle-bad:*

assumes *raw-bound: query-index-raw-preimage-bound*

and *no-bad:*

$\neg$ *query-supported-pairwise-merkle-bad*
 (*verifier-state-from-adversary attacker-state*
 (*staged-proof-transcript data*))

shows

staged-phase-target-error

(query-index-raw-preimage
 (query-header-supported-union-good-sets
 (verifier-state-from-adversary attacker-state
 (staged-proof-transcript data)))
 query-sampling-success-space
 (staged-trace-root data)
 (staged-trace-fri-roots data)
 (staged-trace-final data)
 (staged-alphas data)
 (staged-degree data)
 (staged-composition-fri-roots data)
 (staged-composition-final data))) $q \leq$
 $nnreal\ q * query-error-bound$
by (rule staged-phase-query-index-target-error-query-bound
 [OF raw-bound staged-dynamic-query-set-subset
 staged-dynamic-query-set-fraction-bound-if-no-pairwise-merkle-bad
 [OF no-bad]])

lemma staged-alpha-prefix-recv-bad-vector-position-bound:

assumes wf: staged-budget-wellformed budgets
and controlled: staged-adversary-controlled budgets A
and i-bound: $i < length\ spec$
and absent:
 alpha-challenge-values-absent
 (fri-vector-position-values B i) s

shows

wp-event
 (staged-alpha-challenge-prefix-program A i $\gg$
 (λ -. receive-alpha-challenge))
 (λ out. case out of None $\Rightarrow$ False
 | Some (a, -) $\Rightarrow$ $a \in fri-vector-position-values\ B\ i$) s $\leq$
 staged-phase-target-error (fri-vector-position-values B i)
 (Suc (staged-alpha-search-queries budgets i)))
by (rule staged-alpha-prefix-recv-bad-value-bound
 [OF wf controlled - absent])
 (use i-bound **in** simp)

lemma staged-alpha-prefix-recv-bad-vector-position-relation-bound:

assumes wf: staged-budget-wellformed budgets
and controlled: staged-adversary-controlled budgets A
and subset: $B \subseteq fri-challenge-space\ (length\ spec)$
and i-bound: $i < length\ spec$
and absent:
 alpha-challenge-values-absent
 (fri-vector-position-values B i) s

shows

wp-event
 (staged-alpha-challenge-prefix-program A i $\gg$
 (λ -. receive-alpha-challenge))

```

    (λout. case out of None ⇒ False
      | Some (a, -) ⇒ a ∈ fri-vector-position-values B i) s ≤
    staged-phase-relation-error (card B)
    (Suc (staged-alpha-search-queries budgets i))
  proof -
    have target:
      wp-event
      (staged-alpha-challenge-prefix-program A i ≫=
        (λ-. receive-alpha-challenge))
      (λout. case out of None ⇒ False
        | Some (a, -) ⇒ a ∈ fri-vector-position-values B i) s ≤
      staged-phase-target-error (fri-vector-position-values B i)
      (Suc (staged-alpha-search-queries budgets i))
    by (rule staged-alpha-prefix-receive-bad-vector-position-bound
      [OF wf controlled i-bound absent])
    also have ... ≤
      staged-phase-relation-error (card B)
      (Suc (staged-alpha-search-queries budgets i))
    by (rule staged-phase-target-error-card-mono)
      (rule fri-vector-position-values-card-le[OF subset i-bound])
    finally show ?thesis .
  qed

  lemma staged-alpha-prefix-receive-bad-vector-position-relation-bound-total:
    assumes wf: staged-budget-wellformed budgets
    and controlled: staged-adversary-controlled budgets A
    and subset: B ⊆ fri-challenge-space (length spec)
    and i-bound: i < length spec
    and absent:
      alpha-challenge-values-absent
      (fri-vector-position-values B i) s
    shows
      wp-event
      (staged-alpha-challenge-prefix-program A i ≫=
        (λ-. receive-alpha-challenge))
      (λout. case out of None ⇒ False
        | Some (a, -) ⇒ a ∈ fri-vector-position-values B i) s ≤
      staged-phase-relation-error (card B)
      (Suc
        (staged-attacker-query-budget budgets +
          staged-challenge-query-budget))
  proof -
    have local:
      wp-event
      (staged-alpha-challenge-prefix-program A i ≫=
        (λ-. receive-alpha-challenge))
      (λout. case out of None ⇒ False
        | Some (a, -) ⇒ a ∈ fri-vector-position-values B i) s ≤
      staged-phase-relation-error (card B)

```

```

    (Suc (staged-alpha-search-queries budgets i))
  by (rule staged-alpha-prefix-receive-bad-vector-position-relation-bound
      [OF wf controlled subset i-bound absent])
also have
  staged-phase-relation-error (card B)
    (Suc (staged-alpha-search-queries budgets i)) ≤
  staged-phase-relation-error (card B)
    (Suc
      (staged-attacker-query-budget budgets +
       staged-challenge-query-budget))
  by (rule staged-phase-relation-error-mono)
    (simp add: staged-alpha-search-queries-le-total[OF less-imp-le[OF i-bound]])
finally show ?thesis .
qed

lemma staged-alpha-prefix-receive-keep-prefix-bad-vector-position-relation-bound-total:
  assumes wf: staged-budget-wellformed budgets
    and controlled: staged-adversary-controlled budgets A
    and subset: B ⊆ fri-challenge-space (length spec)
    and i-bound: i < length spec
    and absent:
      alpha-challenge-values-absent
      (fri-vector-position-values B i) s
  shows
    wp-event
      (staged-alpha-challenge-prefix-program A i ≫≡
        (λprefix. receive-alpha-challenge ≫≡
          (λa. return (prefix, a))))
      (λout. case out of None ⇒ False
        | Some ((-, a), -) ⇒ a ∈ fri-vector-position-values B i) s ≤
      staged-phase-relation-error (card B)
        (Suc
          (staged-attacker-query-budget budgets +
           staged-challenge-query-budget))

proof -
  let ?P =
    λout. case out of None ⇒ False
    | Some (a, -) ⇒ a ∈ fri-vector-position-values B i
  let ?Q =
    λout. case out of None ⇒ False
    | Some ((-, a), -) ⇒ a ∈ fri-vector-position-values B i
  have cont-eq:
    ∧prefix t. wp-event
      (receive-alpha-challenge ≫≡ (λa. return (prefix, a))) ?Q t =
      wp-event receive-alpha-challenge ?P t
  proof -
    fix prefix t
    show wp-event
      (receive-alpha-challenge ≫≡ (λa. return (prefix, a))) ?Q t =

```

```

    wp-event receive-alpha-challenge ?P t
  unfolding wp-event-def
  apply (subst wp-bind)
  apply (rule arg-cong[where
    f= $\lambda Q. wp\ receive-alpha-challenge\ Q\ t$ ])
  apply (rule ext)
  apply (simp add: wp-return split: option.splits prod.splits)
  done
qed
have eq:
  wp-event
    (staged-alpha-challenge-prefix-program A i  $\gg$ 
      ( $\lambda prefix. receive-alpha-challenge$   $\gg$ 
        ( $\lambda a. return\ (prefix, a)$ ))) ?Q s =
  wp-event
    (staged-alpha-challenge-prefix-program A i  $\gg$ 
      ( $\lambda \cdot. receive-alpha-challenge$ )) ?P s
  by (rule wp-event-bind-cong-cont) (simp-all add: cont-eq)
show ?thesis
  unfolding eq
  by (rule staged-alpha-prefix-receive-bad-vector-position-relation-bound-total
    [OF wf controlled subset i-bound absent])
qed

lemma staged-alpha-prefix-bad-trace-vector-bound:
  assumes wf: staged-budget-wellformed budgets
  and controlled: staged-adversary-controlled budgets A
  and subset:  $B \subseteq fri-challenge-space\ (ceil-log\ clength)$ 
  and nonempty:  $0 < ceil-log\ clength$ 
  and absent:
     $\bigwedge i. i < ceil-log\ clength \implies$ 
      trace-fri-challenge-values-absent
        (fri-vector-position-values B i) s
  shows
    wp-event (staged-alpha-prefix-program A)
      ( $\lambda out. case\ out\ of\ None \Rightarrow False$ 
        | Some ((-, -, trace-bs, -), -)  $\Rightarrow trace-bs \in B$ ) s  $\leq$ 
      nnreal (ceil-log clength) *
        staged-phase-relation-error (card B)
        (staged-alpha-search-queries budgets 0)
proof -
  let ?coord =
     $\lambda i out. case\ out\ of\ None \Rightarrow False$ 
    | Some ((-, -, trace-bs, -), -)  $\Rightarrow$ 
      trace-bs ! i  $\in fri-vector-position-values\ B\ i$ 
  have cover:
    wp-event (staged-alpha-prefix-program A)
      ( $\lambda out. case\ out\ of\ None \Rightarrow False$ 
        | Some ((-, -, trace-bs, -), -)  $\Rightarrow trace-bs \in B$ ) s  $\leq$ 

```

$(\sum i < \text{ceil-log } \text{clength}.$
 $\text{wp-event } (\text{staged-alpha-prefix-program } A) \text{ } (?coord \ i) \ s$
by (rule staged-alpha-prefix-bad-trace-vector-coordinate-cover-bound
 $[OF \ \text{subset } \text{nonempty}]$)
also have ... $\leq$
 $(\sum i < \text{ceil-log } \text{clength}.$
 $\text{staged-phase-relation-error } (\text{card } B)$
 $(\text{staged-alpha-search-queries } \text{budgets } 0))$
proof (rule sum-mono)
fix i
assume $i\text{-in}: i \in \{..<\text{ceil-log } \text{clength}\}$
then have $i\text{-bound}: i < \text{ceil-log } \text{clength}$
by simp
have target:
 $\text{wp-event } (\text{staged-alpha-prefix-program } A) \text{ } (?coord \ i) \ s \leq$
 $\text{staged-phase-target-error } (\text{fri-vector-position-values } B \ i)$
 $(\text{staged-alpha-search-queries } \text{budgets } 0)$
by (rule staged-alpha-prefix-trace-coordinate-bound
 $[OF \ \text{wf controlled } i\text{-bound absent}[OF \ i\text{-bound}]]$)
also have ... $\leq$
 $\text{staged-phase-relation-error } (\text{card } B)$
 $(\text{staged-alpha-search-queries } \text{budgets } 0)$
by (rule staged-phase-target-error-card-mono)
 $(\text{rule } \text{fri-vector-position-values-card-le}[OF \ \text{subset } i\text{-bound}])$
finally show
 $\text{wp-event } (\text{staged-alpha-prefix-program } A) \text{ } (?coord \ i) \ s \leq$
 $\text{staged-phase-relation-error } (\text{card } B)$
 $(\text{staged-alpha-search-queries } \text{budgets } 0) .$
qed
also have ... $=$
 $\text{nnreal } (\text{ceil-log } \text{clength}) *$
 $\text{staged-phase-relation-error } (\text{card } B)$
 $(\text{staged-alpha-search-queries } \text{budgets } 0)$
by simp
finally show ?thesis .
qed

lemma staged-transcript-program-trace-fri-vector-bound:
assumes wf: staged-budget-wellformed budgets
and controlled: staged-adversary-controlled budgets A
and subset: $B \subseteq \text{fri-challenge-space } (\text{ceil-log } \text{clength})$
and nonempty: $0 < \text{ceil-log } \text{clength}$
and absent:
 $\bigwedge i. i < \text{ceil-log } \text{clength} \implies$
 $\text{trace-fri-challenge-values-absent}$
 $(\text{fri-vector-position-values } B \ i) \ \text{adversary-initial-state}$
shows
 $\text{wp-event } (\text{staged-transcript-program } A)$
 $(\text{staged-transcript-trace-fri-vector-hit } B)$

```

    adversary-initial-state ≤
    nnreal (ceil-log clength) *
    staged-phase-relation-error (card B)
    (staged-alpha-search-queries budgets 0)
proof –
  let ?Head =
    λout. case out of
      None ⇒ False
    | Some ((-, -, trace-bs, -), -) ⇒ trace-bs ∈ B
  have prefix-bound:
    wp-event (staged-alpha-prefix-program A) ?Head
    adversary-initial-state ≤
    nnreal (ceil-log clength) *
    staged-phase-relation-error (card B)
    (staged-alpha-search-queries budgets 0)
  by (rule staged-alpha-prefix-bad-trace-vector-bound
    [OF wf controlled subset nonempty absent])
  show ?thesis
    unfolding staged-transcript-program-alpha-prefix-decomp
  proof (rule wp-event-bind-bound-by-head-event[OF prefix-bound])
  show staged-transcript-trace-fri-vector-hit B None ⇒
    ?Head None
    unfolding staged-transcript-trace-fri-vector-hit-def by simp
  next
    fix x t out
    assume cont:
      out ∈
      set-dist
      (execute (staged-after-alpha-prefix-program A x) t)
    and hit: staged-transcript-trace-fri-vector-hit B out
  obtain fr trace-roots trace-bs trace-final where x-eq:
    x = (fr, trace-roots, trace-bs, trace-final)
    by (cases x) auto
  have trace-bs-in: trace-bs ∈ B
    by (rule staged-after-alpha-prefix-program-trace-fri-vector-hitD
      [OF cont[unfolded x-eq] hit])
  show ?Head (Some (x, t))
    unfolding x-eq
    using trace-bs-in by simp
  qed
qed

```

```

lemma staged-security-with-data-trace-fri-vector-bound:
  assumes wf: staged-budget-wellformed budgets
  and controlled: staged-adversary-controlled budgets A
  and subset: B ⊆ fri-challenge-space (ceil-log clength)
  and nonempty: 0 < ceil-log clength
  and absent:
    ∧ i. i < ceil-log clength ⇒

```

```

    trace-fri-challenge-values-absent
    (fri-vector-position-values B i) adversary-initial-state
shows
  wp-event (staged-security-experiment-with-data A)
    (staged-security-with-data-trace-fri-vector-hit B)
  adversary-initial-state ≤
  nnreal (ceil-log clength) *
    staged-phase-relation-error (card B)
    (staged-alpha-search-queries budgets 0)
proof -
  have transcript-bound:
    wp-event (staged-transcript-program A)
      (staged-transcript-trace-fri-vector-hit B)
    adversary-initial-state ≤
    nnreal (ceil-log clength) *
      staged-phase-relation-error (card B)
      (staged-alpha-search-queries budgets 0)
  by (rule staged-transcript-program-trace-fri-vector-bound
      [OF wf controlled subset nonempty absent])
show ?thesis
  unfolding staged-security-experiment-with-data-def
proof (rule wp-event-bind-bound-by-head-event[OF transcript-bound])
  show staged-security-with-data-trace-fri-vector-hit B None ⇒
    staged-transcript-trace-fri-vector-hit B None
  unfolding staged-security-with-data-trace-fri-vector-hit-def
    staged-transcript-trace-fri-vector-hit-def
  by simp
next
  fix data attacker-state out
  assume cont:
    out ∈
      set-dist
      (execute
        (get >=>
          (λs. put
            (verifier-state-from-adversary s
              (staged-proof-transcript data)) >=>
            (λ-. verify-monad >=>
              (λresult. return (data, result))))))
        attacker-state)
  and hit: staged-security-with-data-trace-fri-vector-hit B out
  have data-bad: staged-trace-fri-challenges data ∈ B
  proof (cases out)
    case None
    then show ?thesis
      using hit unfolding staged-security-with-data-trace-fri-vector-hit-def
      by simp
  next
    case (Some result)

```

```

then obtain data' result' final-state where out-eq:
  out = Some ((data', result'), final-state)
  by (cases result, auto split: prod.splits)
from cont[unfolded out-eq]
obtain s1 s2 s3 where
  ret:
    Some ((data', result'), final-state) ∈
      set-dist (execute (return (data, result')) s3)
  by (auto elim!: set-dist-bindE)
have data'-eq: data' = data
  using ret by simp
show ?thesis
  using hit unfolding out-eq data'-eq
    staged-security-with-data-trace-fri-vector-hit-def
  by simp
qed
show staged-transcript-trace-fri-vector-hit B
  (Some (data, attacker-state))
  using data-bad unfolding staged-transcript-trace-fri-vector-hit-def
  by simp
qed
qed

lemma staged-security-with-data-composition-fri-vector-bound:
  assumes wf: staged-budget-wellformed budgets
  and controlled: staged-adversary-controlled budgets A
  and subset:
     $\bigwedge dg. B \ dg \subseteq \text{fri-challenge-space } (\text{ceil-log } (to\text{-nat } dg + 1))$ 
  and absent:
     $\bigwedge i. i < \text{ceil-log } (\text{maxDegree} + 1) \implies$ 
      composition-fri-challenge-values-absent
      ( $\bigcup dg. \text{fri-vector-position-values } (B \ dg) \ i$ )
      adversary-initial-state
  shows
    wp-event (staged-security-experiment-with-data A)
    (staged-security-with-data-composition-fri-vector-guarded-hit B)
    adversary-initial-state ≤
    ( $\sum i < \text{ceil-log } (\text{maxDegree} + 1).$ 
      staged-phase-target-error
      ( $\bigcup dg. \text{fri-vector-position-values } (B \ dg) \ i$ )
      (staged-composition-fri-vector-search-queries budgets))

proof –
  have transcript-bound:
    wp-event (staged-transcript-program A)
    (staged-transcript-composition-fri-vector-guarded-hit B)
    adversary-initial-state ≤
    ( $\sum i < \text{ceil-log } (\text{maxDegree} + 1).$ 
      staged-phase-target-error
      ( $\bigcup dg. \text{fri-vector-position-values } (B \ dg) \ i$ )

```



```

    (staged-composition-fri-vector-search-queries budgets))
  by (rule staged-transcript-program-composition-fri-vector-bound
    [OF wf controlled subset absent])
show ?thesis
  unfolding staged-security-experiment-with-data-def
proof (rule wp-event-bind-bound-by-head-event[OF transcript-bound])
  show
    staged-security-with-data-composition-fri-vector-guarded-hit B None
     $\implies$  staged-transcript-composition-fri-vector-guarded-hit B None
  unfolding
    staged-security-with-data-composition-fri-vector-guarded-hit-def
    staged-transcript-composition-fri-vector-guarded-hit-def
  by simp
next
fix data attacker-state out
assume cont:
  out  $\in$ 
  set-dist
  (execute
    (get  $\gg$ 
      ( $\lambda s$ . put
        (verifier-state-from-adversary s
          (staged-proof-transcript data))  $\gg$ 
        ( $\lambda \cdot$ . verify-monad  $\gg$ 
          ( $\lambda result$ . return (data, result))))))
    attacker-state)
and hit:
  staged-security-with-data-composition-fri-vector-guarded-hit B out
have data-bad:
  staged-composition-fri-challenges data  $\in$ 
  B (staged-degree data)  $\wedge$ 
  0 < ceil-log (to-nat (staged-degree data) + 1)  $\wedge$ 
  ceil-log (to-nat (staged-degree data) + 1)  $\leq$ 
  ceil-log (maxDegree + 1)
proof (cases out)
case None
then show ?thesis
  using hit
  unfolding
    staged-security-with-data-composition-fri-vector-guarded-hit-def
  by simp
next
case (Some result)
then obtain data' result' final-state where out-eq:
  out = Some ((data', result'), final-state)
  by (cases result, auto split: prod.splits)
from cont[unfolded out-eq]
obtain s1 s2 s3 where
  ret:

```

```

    Some ((data', result'), final-state) ∈
      set-dist (execute (return (data, result')) s3)
  by (auto elim!: set-dist-bindE)
have data'-eq: data' = data
  using ret by simp
show ?thesis
  using hit unfolding out-eq data'-eq
    staged-security-with-data-composition-fri-vector-guarded-hit-def
  by simp
qed
show staged-transcript-composition-fri-vector-guarded-hit B
  (Some (data, attacker-state))
  using data-bad
  unfolding staged-transcript-composition-fri-vector-guarded-hit-def
  by simp
qed
qed

lemma staged-trace-fri-prefix-receive-bad-value-bound-total:
  assumes wf: staged-budget-wellformed budgets
  and controlled: staged-adversary-controlled budgets A
  and i-bound: i < ceil-log clength
  and absent: trace-fri-challenge-values-absent B s
  shows
    wp-event
      (staged-trace-fri-challenge-prefix-program A i ≫=
        (λ-. receive-trace-fri-challenge))
      (λout. case out of None ⇒ False
        | Some (b, -) ⇒ b ∈ B) s ≤
      staged-phase-target-error B
      (Suc
        (staged-attacker-query-budget budgets +
          staged-challenge-query-budget))

proof -
  have i-len: i < length (trace-fri-budgets budgets)
  using wf i-bound unfolding staged-budget-wellformed-def by simp
  have local:
    wp-event
      (staged-trace-fri-challenge-prefix-program A i ≫=
        (λ-. receive-trace-fri-challenge))
      (λout. case out of None ⇒ False
        | Some (b, -) ⇒ b ∈ B) s ≤
      staged-phase-target-error B
      (Suc (staged-trace-fri-search-queries budgets i))
  by (rule staged-trace-fri-prefix-receive-bad-value-bound
    [OF controlled i-len absent])
  also have
    staged-phase-target-error B
      (Suc (staged-trace-fri-search-queries budgets i)) ≤

```

staged-phase-target-error B
 (Suc
 (*staged-attacker-query-budget budgets +*
 staged-challenge-query-budget))
 by (rule *staged-phase-target-error-mono*)
 (*simp add: staged-trace-fri-search-queries-le-total[OF wf i-bound]*)
 finally show ?thesis .
 qed

lemma *staged-trace-fri-prefix-receive-bad-vector-position-bound-total*:
assumes *wf: staged-budget-wellformed budgets*
and *controlled: staged-adversary-controlled budgets A*
and *i-bound: i < ceil-log clength*
and *absent:*
 trace-fri-challenge-values-absent
 (*fri-vector-position-values B i*) *s*
shows
 wp-event
 (*staged-trace-fri-challenge-prefix-program A i* $\gg$
 ($\lambda\cdot$. *receive-trace-fri-challenge*))
 (λ out. case out of None $\Rightarrow$ False
 | Some (b, -) $\Rightarrow$ b $\in$ *fri-vector-position-values B i*) *s* $\leq$
 staged-phase-target-error (fri-vector-position-values B i)
 (Suc
 (*staged-attacker-query-budget budgets +*
 staged-challenge-query-budget))
by (rule *staged-trace-fri-prefix-receive-bad-value-bound-total*
 [*OF wf controlled i-bound absent*])

lemma *staged-trace-fri-prefix-receive-bad-vector-position-relation-bound*:
assumes *wf: staged-budget-wellformed budgets*
and *controlled: staged-adversary-controlled budgets A*
and *subset: B $\subseteq$ fri-challenge-space (ceil-log clength)*
and *i-bound: i < ceil-log clength*
and *absent:*
 trace-fri-challenge-values-absent
 (*fri-vector-position-values B i*) *s*
shows
 wp-event
 (*staged-trace-fri-challenge-prefix-program A i* $\gg$
 ($\lambda\cdot$. *receive-trace-fri-challenge*))
 (λ out. case out of None $\Rightarrow$ False
 | Some (b, -) $\Rightarrow$ b $\in$ *fri-vector-position-values B i*) *s* $\leq$
 staged-phase-relation-error (card B)
 (Suc
 (*staged-attacker-query-budget budgets +*
 staged-challenge-query-budget))

proof –
have *target*:

```

wp-event
  (staged-trace-fri-challenge-prefix-program A i  $\gg$ 
    ( $\lambda$ -. receive-trace-fri-challenge))
  ( $\lambda$ out. case out of None  $\Rightarrow$  False
    | Some (b, -)  $\Rightarrow$  b  $\in$  fri-vector-position-values B i) s  $\leq$ 
    staged-phase-target-error (fri-vector-position-values B i)
    (Suc
      (staged-attacker-query-budget budgets +
        staged-challenge-query-budget))
  by (rule staged-trace-fri-prefix-receive-bad-vector-position-bound-total
    [OF wf controlled i-bound absent])
also have ...  $\leq$ 
  staged-phase-relation-error (card B)
  (Suc
    (staged-attacker-query-budget budgets +
      staged-challenge-query-budget))
  by (rule staged-phase-target-error-card-mono)
    (rule fri-vector-position-values-card-le[OF subset i-bound])
finally show ?thesis .
qed

lemma staged-trace-fri-prefix-receive-keep-prefix-bad-vector-position-relation-bound:
  assumes wf: staged-budget-wellformed budgets
  and controlled: staged-adversary-controlled budgets A
  and subset: B  $\subseteq$  fri-challenge-space (ceil-log clength)
  and i-bound: i < ceil-log clength
  and absent:
    trace-fri-challenge-values-absent
    (fri-vector-position-values B i) s
  shows
    wp-event
      (staged-trace-fri-challenge-prefix-program A i  $\gg$ 
        ( $\lambda$ prefix. receive-trace-fri-challenge  $\gg$ 
          ( $\lambda$ b. return (prefix, b))))
      ( $\lambda$ out. case out of None  $\Rightarrow$  False
        | Some ((-, b), -)  $\Rightarrow$  b  $\in$  fri-vector-position-values B i) s  $\leq$ 
        staged-phase-relation-error (card B)
        (Suc
          (staged-attacker-query-budget budgets +
            staged-challenge-query-budget))

proof -
  let ?P =
     $\lambda$ out. case out of None  $\Rightarrow$  False
    | Some (b, -)  $\Rightarrow$  b  $\in$  fri-vector-position-values B i
  let ?Q =
     $\lambda$ out. case out of None  $\Rightarrow$  False
    | Some ((-, b), -)  $\Rightarrow$  b  $\in$  fri-vector-position-values B i
  have cont-eq:
     $\bigwedge$ prefix t. wp-event

```

```

    (receive-trace-fri-challenge  $\gg$  ( $\lambda b. \text{return } (\text{prefix}, b)$ )) ?Q t =
    wp-event receive-trace-fri-challenge ?P t
proof –
  fix prefix t
  show wp-event
    (receive-trace-fri-challenge  $\gg$  ( $\lambda b. \text{return } (\text{prefix}, b)$ )) ?Q t =
    wp-event receive-trace-fri-challenge ?P t
  unfolding wp-event-def
  apply (subst wp-bind)
  apply (rule arg-cong[where
    f= $\lambda Q. \text{wp receive-trace-fri-challenge } Q \ t$ ])
  apply (rule ext)
  apply (simp add: wp-return split: option.splits prod.splits)
  done
qed
have eq:
  wp-event
    (staged-trace-fri-challenge-prefix-program A i  $\gg$ 
      ( $\lambda \text{prefix}. \text{receive-trace-fri-challenge } \gg$ 
        ( $\lambda b. \text{return } (\text{prefix}, b)$ ))) ?Q s =
  wp-event
    (staged-trace-fri-challenge-prefix-program A i  $\gg$ 
      ( $\lambda -. \text{receive-trace-fri-challenge}$ )) ?P s
  by (rule wp-event-bind-cong-cont) (simp-all add: cont-eq)
show ?thesis
  unfolding eq
  by (rule staged-trace-fri-prefix-receive-bad-vector-position-relation-bound
    [OF wf controlled subset i-bound absent])
qed

lemma staged-trace-fri-bad-vector-coordinate-sum-bound:
assumes wf: staged-budget-wellformed budgets
and controlled: staged-adversary-controlled budgets A
and subset:  $B \subseteq \text{fri-challenge-space } (\text{ceil-log } \text{clength})$ 
and absent:
   $\bigwedge i. i < \text{ceil-log } \text{clength} \implies$ 
    trace-fri-challenge-values-absent
    (fri-vector-position-values B i) s
shows
  ( $\sum i < \text{ceil-log } \text{clength}.$ 
    wp-event
      (staged-trace-fri-challenge-prefix-program A i  $\gg$ 
        ( $\lambda -. \text{receive-trace-fri-challenge}$ ))
      ( $\lambda \text{out}. \text{case out of None} \Rightarrow \text{False}$ 
        |  $\text{Some } (b, -) \Rightarrow b \in \text{fri-vector-position-values } B \ i$ ) s)  $\leq$ 
    nnreal (ceil-log clength) *
    staged-phase-relation-error (card B)
    (Suc
      (staged-attacker-query-budget budgets +

```

staged-challenge-query-budget))

proof –

have

($\sum i < \text{ceil-log clength}$.
wp-event
 (*staged-trace-fri-challenge-prefix-program* $A\ i \gg=$
 ($\lambda\cdot$. *receive-trace-fri-challenge*))
 (λout . *case out of None* $\Rightarrow$ *False*
 | *Some* ($b, - \Rightarrow b \in \text{fri-vector-position-values } B\ i$) s) $\leq$
 ($\sum i < \text{ceil-log clength}$.
staged-phase-relation-error (*card* B)
 (*Suc*
 (*staged-attacker-query-budget budgets* +
staged-challenge-query-budget)))

by (*rule sum-mono*)
 (*rule staged-trace-fri-prefix-receive-bad-vector-position-relation-bound*
 [*OF wf controlled subset*], *simp-all add: absent*)

also have ... =

nnreal (*ceil-log clength*) *
staged-phase-relation-error (*card* B)
 (*Suc*
 (*staged-attacker-query-budget budgets* +
staged-challenge-query-budget))

by *simp*

finally show *?thesis* .

qed

lemma *staged-trace-fri-bad-vector-coordinate-sum-keep-prefix-bound*:

assumes *wf*: *staged-budget-wellformed budgets*
and *controlled*: *staged-adversary-controlled budgets A*
and *subset*: $B \subseteq \text{fri-challenge-space } (\text{ceil-log clength})$
and *absent*:

$\bigwedge i. i < \text{ceil-log clength} \Rightarrow$
trace-fri-challenge-values-absent
 (*fri-vector-position-values* $B\ i$) s

shows

($\sum i < \text{ceil-log clength}$.
wp-event
 (*staged-trace-fri-challenge-prefix-program* $A\ i \gg=$
 (λprefix . *receive-trace-fri-challenge* $\gg=$
 (λb . *return* (*prefix*, b))))
 (λout . *case out of None* $\Rightarrow$ *False*
 | *Some* ($(-, b), - \Rightarrow$
 $b \in \text{fri-vector-position-values } B\ i$) s) $\leq$
nnreal (*ceil-log clength*) *
staged-phase-relation-error (*card* B)
 (*Suc*
 (*staged-attacker-query-budget budgets* +
staged-challenge-query-budget)))

proof –
have
 $(\sum i < \text{ceil-log } \text{clength}.$
 wp-event
 $(\text{staged-trace-fri-challenge-prefix-program } A \ i \gg=$
 $(\lambda \text{prefix}. \text{receive-trace-fri-challenge } \gg=$
 $(\lambda b. \text{return } (\text{prefix}, b))))$
 $(\lambda \text{out}. \text{case out of None} \Rightarrow \text{False}$
 $| \text{Some } ((-, b), -) \Rightarrow$
 $b \in \text{fri-vector-position-values } B \ i) \ s) \leq$
 $(\sum i < \text{ceil-log } \text{clength}.$
 $\text{staged-phase-relation-error } (\text{card } B)$
 $(\text{Suc}$
 $(\text{staged-attacker-query-budget } \text{budgets} +$
 $\text{staged-challenge-query-budget})))$
by (rule sum-mono)
 $(\text{rule}$
 $\text{staged-trace-fri-prefix-keep-prefix-bad-vector-position-relation-bound}$
 $[\text{OF wf controlled subset}], \text{simp-all add: absent})$
also have ... =
 $\text{nnreal } (\text{ceil-log } \text{clength}) *$
 $\text{staged-phase-relation-error } (\text{card } B)$
 $(\text{Suc}$
 $(\text{staged-attacker-query-budget } \text{budgets} +$
 $\text{staged-challenge-query-budget})))$
by simp
finally show ?thesis .
qed

lemma staged-query-prefix-bad-index-error-mono:
assumes $i\text{-bound}: i < \text{rounds}$
and $\text{wf}: \text{staged-budget-wellformed } \text{budgets}$
shows
 $\text{staged-query-prefix-bad-index-error } \text{budgets } B \ i \leq$
 $\text{staged-phase-target-error } (\text{query-index-raw-preimage } B)$
 $(\text{staged-attacker-query-budget } \text{budgets} +$
 $\text{staged-challenge-query-budget})$
unfolding staged-query-prefix-bad-index-error-def
by (rule staged-phase-target-error-mono)
 $(\text{rule staged-query-search-queries-le-total}[\text{OF wf } i\text{-bound}])$

lemma staged-query-prefix-bad-index-target-bound-total:
assumes $\text{wf}: \text{staged-budget-wellformed } \text{budgets}$
and $\text{controlled}: \text{staged-adversary-controlled } \text{budgets } A$
and $i\text{-bound}: i < \text{rounds}$
shows
 $\text{wp-event } (\text{staged-query-challenge-prefix-program } A \ i)$
 $(\text{hash-new-output-hit-event } (\text{query-index-raw-preimage } B) \ s) \ s \leq$
 $\text{staged-phase-target-error } (\text{query-index-raw-preimage } B)$

(staged-attacker-query-budget budgets +
 staged-challenge-query-budget)
proof –
 have local:
 wp-event (staged-query-challenge-prefix-program A i)
 (hash-new-output-hit-event (query-index-raw-preimage B) s) s ≤
 staged-query-prefix-bad-index-error budgets B i
 by (rule staged-query-prefix-bad-index-target-bound)
 (use wf controlled i-bound in simp-all)
 also have
 staged-query-prefix-bad-index-error budgets B i ≤
 staged-phase-target-error (query-index-raw-preimage B)
 (staged-attacker-query-budget budgets +
 staged-challenge-query-budget)
 by (rule staged-query-prefix-bad-index-error-mono[OF i-bound wf])
 finally show ?thesis .
qed

lemma checked-staged-query-prefix-bad-index-target-bound-total:
 assumes wf: staged-budget-wellformed budgets
 and controlled: staged-adversary-controlled budgets A
 and i-bound: i < rounds
 shows
 wp-event (checked-staged-query-challenge-prefix-program A i)
 (hash-new-output-hit-event (query-index-raw-preimage B) s) s ≤
 staged-phase-target-error (query-index-raw-preimage B)
 (staged-attacker-query-budget budgets +
 staged-challenge-query-budget)
proof –
 have local:
 wp-event (checked-staged-query-challenge-prefix-program A i)
 (hash-new-output-hit-event (query-index-raw-preimage B) s) s ≤
 staged-query-prefix-bad-index-error budgets B i
 by (rule checked-staged-query-prefix-bad-index-target-bound)
 (use wf controlled i-bound in simp-all)
 also have
 staged-query-prefix-bad-index-error budgets B i ≤
 staged-phase-target-error (query-index-raw-preimage B)
 (staged-attacker-query-budget budgets +
 staged-challenge-query-budget)
 by (rule staged-query-prefix-bad-index-error-mono[OF i-bound wf])
 finally show ?thesis .
qed

lemma staged-query-prefix-receive-bad-index-bound-total:
 assumes wf: staged-budget-wellformed budgets
 and controlled: staged-adversary-controlled budgets A
 and i-bound: i < rounds
 and absent: query-index-raw-preimage-absent B s

shows

wp-event
 (staged-query-challenge-prefix-program A i $\gg=$
 (λ -. receive-query-index-challenge))
 (λ out. case out of None $\Rightarrow$ False
 | Some (raw, -) $\Rightarrow$ index (to-nat raw) $\in B$) $s \leq$
 staged-phase-target-error (query-index-raw-preimage B)
 (Suc
 (staged-attacker-query-budget budgets +
 staged-challenge-query-budget))

proof –

have *local*:

wp-event
 (staged-query-challenge-prefix-program A i $\gg=$
 (λ -. receive-query-index-challenge))
 (λ out. case out of None $\Rightarrow$ False
 | Some (raw, -) $\Rightarrow$ index (to-nat raw) $\in B$) $s \leq$
 staged-phase-target-error (query-index-raw-preimage B)
 (Suc (staged-query-search-queries budgets i))
by (rule staged-query-prefix-receive-bad-index-bound)
 (use wf controlled i -bound absent **in** simp-all)

also have

staged-phase-target-error (query-index-raw-preimage B)
 (Suc (staged-query-search-queries budgets i)) $\leq$
 staged-phase-target-error (query-index-raw-preimage B)
 (Suc
 (staged-attacker-query-budget budgets +
 staged-challenge-query-budget))
by (rule staged-phase-target-error-mono)
 (simp add: staged-query-search-queries-le-total[OF wf i -bound])

finally show ?thesis .

qed

lemma staged-query-prefix-receive-bad-index-query-bound-total:

assumes wf: staged-budget-wellformed budgets

and controlled: staged-adversary-controlled budgets A

and i -bound: $i < \text{rounds}$

and absent: query-index-raw-preimage-absent B s

and raw-bound: query-index-raw-preimage-bound

and subset: $B \subseteq \text{query-sample-space}$

and frac:

$\text{nnreal} (\text{card } B) / \text{nnreal} (\text{card } \text{query-sample-space}) \leq C$

shows

wp-event
 (staged-query-challenge-prefix-program A i $\gg=$
 (λ -. receive-query-index-challenge))
 (λ out. case out of None $\Rightarrow$ False
 | Some (raw, -) $\Rightarrow$ index (to-nat raw) $\in B$) $s \leq$
 nnreal

```

      (Suc
        (staged-attacker-query-budget budgets +
         staged-challenge-query-budget)) * C
proof –
  have local:
    wp-event
      (staged-query-challenge-prefix-program A i  $\gg$ 
        ( $\lambda$ -. receive-query-index-challenge))
      ( $\lambda$ out. case out of None  $\Rightarrow$  False
        | Some (raw, -)  $\Rightarrow$  index (to-nat raw)  $\in$  B) s  $\leq$ 
        staged-phase-target-error (query-index-raw-preimage B)
      (Suc
        (staged-attacker-query-budget budgets +
         staged-challenge-query-budget))
    by (rule staged-query-prefix-receive-bad-index-bound-total
      [OF wf controlled i-bound absent])
  also have ...  $\leq$ 
    nnreal
      (Suc
        (staged-attacker-query-budget budgets +
         staged-challenge-query-budget)) * C
    by (rule staged-phase-query-index-target-error-query-bound
      [OF raw-bound subset frac])
  finally show ?thesis .
qed

```

lemma staged-query-prefix-receive-header-supported-sampling-bound-total:

```

assumes wf: staged-budget-wellformed budgets
and controlled: staged-adversary-controlled budgets A
and i-bound: i < rounds
and absent:
  query-index-raw-preimage-absent
  (query-header-supported-union-good-sets verifier-state
    query-sampling-success-space fr f-fri-roots f-final as dg
    composition-fri-roots final)
  prefix-state
and raw-bound: query-index-raw-preimage-bound
and no-bad:  $\neg$  query-supported-pairwise-merkle-bad verifier-state
shows
  wp-event
    (staged-query-challenge-prefix-program A i  $\gg$ 
      ( $\lambda$ -. receive-query-index-challenge))
    ( $\lambda$ out. case out of None  $\Rightarrow$  False
      | Some (raw, -)  $\Rightarrow$ 
        index (to-nat raw)  $\in$ 
        query-header-supported-union-good-sets verifier-state
        query-sampling-success-space fr f-fri-roots f-final as dg
        composition-fri-roots final)
    prefix-state  $\leq$ 

```

```

    nnreal
    (Suc
      (staged-attacker-query-budget budgets +
       staged-challenge-query-budget)) * query-error-bound
  proof -
    let ?B =
      query-header-supported-union-good-sets verifier-state
      query-sampling-success-space fr f-fri-roots f-final as dg
      composition-fri-roots final
    have subset: ?B ⊆ query-sample-space
    by (rule query-header-supported-union-good-sets-subset)
    have frac:
      nnreal (card ?B) / nnreal (card query-sample-space) ≤
      query-error-bound
    by (rule
        query-header-supported-union-good-sets-fraction-bound-if-no-global-pairwise-merkle-bad
        [OF no-bad query-sampling-success-space-subset
         query-sampling-success-space-fraction-bound-query-sample-space])
    show ?thesis
    by (rule staged-query-prefix-receive-bad-index-query-bound-total
        [OF wf controlled i-bound absent raw-bound subset frac])
  qed

```

lemma *staged-composition-fri-prefix-receive-bad-value-bound-total:*

assumes *wf: staged-budget-wellformed budgets*
and *controlled: staged-adversary-controlled budgets A*
and *i-bound: i < ceil-log (maxDegree + 1)*
and *absent: composition-fri-challenge-values-absent B s*
shows

```

  wp-event
  (staged-composition-fri-challenge-prefix-program A i ≫=
   (λ-. receive-composition-fri-challenge))
  (λout. case out of None ⇒ False
   | Some (b, -) ⇒ b ∈ B) s ≤
  staged-phase-target-error B
  (Suc
    (staged-attacker-query-budget budgets +
     staged-challenge-query-budget))

```

proof -

have *local:*

```

  wp-event
  (staged-composition-fri-challenge-prefix-program A i ≫=
   (λ-. receive-composition-fri-challenge))
  (λout. case out of None ⇒ False
   | Some (b, -) ⇒ b ∈ B) s ≤
  staged-phase-target-error B
  (Suc (staged-composition-fri-search-queries budgets i))
  by (rule staged-composition-fri-prefix-receive-bad-value-bound
      [OF wf controlled i-bound absent])

```

also have

staged-phase-target-error B
 (Suc (staged-composition-fri-search-queries budgets i)) ≤
staged-phase-target-error B
 (Suc
 (staged-attacker-query-budget budgets +
 staged-challenge-query-budget))
 by (rule staged-phase-target-error-mono)
 (simp add: staged-composition-fri-search-queries-le-total
 [OF wf i-bound])

finally show ?thesis .

qed

lemma *staged-composition-fri-prefix-recv-bad-vector-position-bound-total:*

assumes wf: staged-budget-wellformed budgets

and controlled: staged-adversary-controlled budgets A

and i-bound: $i < \text{ceil-log } (\text{maxDegree} + 1)$

and absent:

composition-fri-challenge-values-absent
 (fri-vector-position-values B i) s

shows

wp-event
 (staged-composition-fri-challenge-prefix-program A i $\gg$
 (λ-. receive-composition-fri-challenge))
 (λout. case out of None $\Rightarrow$ False
 | Some (b, -) $\Rightarrow$ $b \in \text{fri-vector-position-values B i}$ s ≤
 staged-phase-target-error (fri-vector-position-values B i)
 (Suc
 (staged-attacker-query-budget budgets +
 staged-challenge-query-budget))

by (rule staged-composition-fri-prefix-recv-bad-value-bound-total
 [OF wf controlled i-bound absent])

lemma

staged-composition-fri-prefix-recv-bad-vector-position-relation-bound:

assumes wf: staged-budget-wellformed budgets

and controlled: staged-adversary-controlled budgets A

and subset:

$B \subseteq \text{fri-challenge-space } (\text{ceil-log } (\text{to-nat dg} + 1))$

and i-bound: $i < \text{ceil-log } (\text{to-nat dg} + 1)$

and dg-bound: $\text{ceil-log } (\text{to-nat dg} + 1) \leq \text{ceil-log } (\text{maxDegree} + 1)$

and absent:

composition-fri-challenge-values-absent
 (fri-vector-position-values B i) s

shows

wp-event
 (staged-composition-fri-challenge-prefix-program A i $\gg$
 (λ-. receive-composition-fri-challenge))
 (λout. case out of None $\Rightarrow$ False

$| \text{Some } (b, -) \Rightarrow b \in \text{fri-vector-position-values } B \ i) \ s \leq$
 $\text{staged-phase-relation-error } (\text{card } B)$
 $(\text{Suc}$
 $\quad (\text{staged-attacker-query-budget budgets} +$
 $\quad \text{staged-challenge-query-budget}))$
proof –
have $i\text{-max}$: $i < \text{ceil-log } (\text{maxDegree} + 1)$
using $i\text{-bound}$ $dg\text{-bound}$ **by** simp
have target :
 wp-event
 $(\text{staged-composition-fri-challenge-prefix-program } A \ i \gg=$
 $\quad (\lambda -. \text{receive-composition-fri-challenge}))$
 $(\lambda \text{out. case out of None} \Rightarrow \text{False}$
 $\quad | \text{Some } (b, -) \Rightarrow b \in \text{fri-vector-position-values } B \ i) \ s \leq$
 $\text{staged-phase-target-error } (\text{fri-vector-position-values } B \ i)$
 $(\text{Suc}$
 $\quad (\text{staged-attacker-query-budget budgets} +$
 $\quad \text{staged-challenge-query-budget}))$
by $(\text{rule}$
 $\quad \text{staged-composition-fri-prefix-receive-bad-vector-position-bound-total}$
 $\quad [\text{OF wf controlled } i\text{-max absent}])$
also have $\dots \leq$
 $\text{staged-phase-relation-error } (\text{card } B)$
 $(\text{Suc}$
 $\quad (\text{staged-attacker-query-budget budgets} +$
 $\quad \text{staged-challenge-query-budget}))$
by $(\text{rule staged-phase-target-error-card-mono})$
 $(\text{rule fri-vector-position-values-card-le}[\text{OF subset } i\text{-bound}])$
finally show $?thesis$.
qed

lemma

$\text{staged-composition-fri-prefix-receive-keep-prefix-bad-vector-position-relation-bound}$:

assumes wf : $\text{staged-budget-wellformed budgets}$

and controlled : $\text{staged-adversary-controlled budgets } A$

and subset :

$B \subseteq \text{fri-challenge-space } (\text{ceil-log } (\text{to-nat } dg + 1))$

and $i\text{-bound}$: $i < \text{ceil-log } (\text{to-nat } dg + 1)$

and $dg\text{-bound}$: $\text{ceil-log } (\text{to-nat } dg + 1) \leq \text{ceil-log } (\text{maxDegree} + 1)$

and absent :

$\text{composition-fri-challenge-values-absent}$

$(\text{fri-vector-position-values } B \ i) \ s$

shows

wp-event

$(\text{staged-composition-fri-challenge-prefix-program } A \ i \gg=$

$\quad (\lambda \text{prefix. receive-composition-fri-challenge} \gg=$

$\quad \quad (\lambda b. \text{return } (\text{prefix}, b))))$

$(\lambda \text{out. case out of None} \Rightarrow \text{False}$

$\quad | \text{Some } ((-, b), -) \Rightarrow b \in \text{fri-vector-position-values } B \ i) \ s \leq$

```

    staged-phase-relation-error (card B)
    (Suc
      (staged-attacker-query-budget budgets +
        staged-challenge-query-budget))
  proof -
    let ?P =
      λout. case out of None ⇒ False
      | Some (b, -) ⇒ b ∈ fri-vector-position-values B i
    let ?Q =
      λout. case out of None ⇒ False
      | Some ((-, b), -) ⇒ b ∈ fri-vector-position-values B i
    have cont-eq:
      ∧prefix t. wp-event
        (receive-composition-fri-challenge ≫=
          (λb. return (prefix, b))) ?Q t =
        wp-event receive-composition-fri-challenge ?P t
    proof -
      fix prefix t
      show wp-event
        (receive-composition-fri-challenge ≫=
          (λb. return (prefix, b))) ?Q t =
        wp-event receive-composition-fri-challenge ?P t
      unfolding wp-event-def
      apply (subst wp-bind)
      apply (rule arg-cong[where
        f=λQ. wp receive-composition-fri-challenge Q t])
      apply (rule ext)
      apply (simp add: wp-return split: option.splits prod.splits)
      done
    qed
  have eq:
    wp-event
      (staged-composition-fri-challenge-prefix-program A i ≫=
        (λprefix. receive-composition-fri-challenge ≫=
          (λb. return (prefix, b)))) ?Q s =
    wp-event
      (staged-composition-fri-challenge-prefix-program A i ≫=
        (λ-. receive-composition-fri-challenge)) ?P s
  by (rule wp-event-bind-cong-cont) (simp-all add: cont-eq)
  show ?thesis
  unfolding eq
  by (rule
    staged-composition-fri-prefix-receive-bad-vector-position-relation-bound
    [OF wf controlled subset i-bound dg-bound absent])
  qed

```

lemma staged-composition-fri-bad-vector-coordinate-sum-bound:
assumes wf: staged-budget-wellformed budgets
and controlled: staged-adversary-controlled budgets A

and *subset*:
 $B \subseteq \text{fri-challenge-space } (\text{ceil-log } (\text{to-nat } dg + 1))$
and *dg-bound*:
 $\text{ceil-log } (\text{to-nat } dg + 1) \leq \text{ceil-log } (\text{maxDegree} + 1)$
and *absent*:
 $\bigwedge i. i < \text{ceil-log } (\text{to-nat } dg + 1) \implies$
 $\text{composition-fri-challenge-values-absent}$
 $(\text{fri-vector-position-values } B \ i) \ s$
shows
 $(\sum i < \text{ceil-log } (\text{to-nat } dg + 1)).$
 wp-event
 $(\text{staged-composition-fri-challenge-prefix-program } A \ i \gg=$
 $(\lambda-. \text{receive-composition-fri-challenge}))$
 $(\lambda \text{out}. \text{case out of None} \Rightarrow \text{False}$
 $| \text{Some } (b, -) \Rightarrow b \in \text{fri-vector-position-values } B \ i) \ s) \leq$
 $\text{nnreal } (\text{ceil-log } (\text{to-nat } dg + 1)) *$
 $\text{staged-phase-relation-error } (\text{card } B)$
 $(\text{Suc}$
 $(\text{staged-attacker-query-budget budgets} +$
 $\text{staged-challenge-query-budget}))$
proof –
have
 $(\sum i < \text{ceil-log } (\text{to-nat } dg + 1)).$
 wp-event
 $(\text{staged-composition-fri-challenge-prefix-program } A \ i \gg=$
 $(\lambda-. \text{receive-composition-fri-challenge}))$
 $(\lambda \text{out}. \text{case out of None} \Rightarrow \text{False}$
 $| \text{Some } (b, -) \Rightarrow b \in \text{fri-vector-position-values } B \ i) \ s) \leq$
 $(\sum i < \text{ceil-log } (\text{to-nat } dg + 1)).$
 $\text{staged-phase-relation-error } (\text{card } B)$
 $(\text{Suc}$
 $(\text{staged-attacker-query-budget budgets} +$
 $\text{staged-challenge-query-budget}))$
proof (rule *sum-mono*)
fix i
assume $i\text{-bound}: i \in \{..<\text{ceil-log } (\text{to-nat } dg + 1)\}$
then have $i\text{-lt}: i < \text{ceil-log } (\text{to-nat } dg + 1)$
by *simp*
show
 wp-event
 $(\text{staged-composition-fri-challenge-prefix-program } A \ i \gg=$
 $(\lambda-. \text{receive-composition-fri-challenge}))$
 $(\lambda \text{out}. \text{case out of None} \Rightarrow \text{False}$
 $| \text{Some } (b, -) \Rightarrow b \in \text{fri-vector-position-values } B \ i) \ s) \leq$
 $\text{staged-phase-relation-error } (\text{card } B)$
 $(\text{Suc}$
 $(\text{staged-attacker-query-budget budgets} +$
 $\text{staged-challenge-query-budget}))$
by (rule

$\text{staged-composition-fri-prefix-receive-bad-vector-position-relation-bound}$
 $[OF \text{ wf controlled subset } i\text{-lt } dg\text{-bound absent}[OF \text{ } i\text{-lt}]]$
qed
also have ... =
 $nnreal (\text{ceil-log } (to\text{-nat } dg + 1)) * \text{staged-phase-relation-error } (card \ B)$
 $(Suc$
 $(\text{staged-attacker-query-budget } budgets +$
 $\text{staged-challenge-query-budget}))$
by simp
finally show ?thesis .
qed

lemma *staged-composition-fri-bad-vector-coordinate-sum-keep-prefix-bound:*

assumes *wf: staged-budget-wellformed budgets*
and *controlled: staged-adversary-controlled budgets A*
and *subset:*
 $B \subseteq \text{fri-challenge-space } (\text{ceil-log } (to\text{-nat } dg + 1))$
and *dg-bound:*
 $\text{ceil-log } (to\text{-nat } dg + 1) \leq \text{ceil-log } (maxDegree + 1)$
and *absent:*
 $\bigwedge i. i < \text{ceil-log } (to\text{-nat } dg + 1) \implies$
 $\text{composition-fri-challenge-values-absent}$
 $(\text{fri-vector-position-values } B \ i) \ s$

shows

$(\sum i < \text{ceil-log } (to\text{-nat } dg + 1)).$
wp-event
 $(\text{staged-composition-fri-challenge-prefix-program } A \ i \gg=$
 $(\lambda prefix. \text{receive-composition-fri-challenge } \gg=$
 $(\lambda b. \text{return } (prefix, b))))$
 $(\lambda out. \text{case out of None} \Rightarrow \text{False}$
 $| \text{Some } ((-, b), -) \Rightarrow$
 $b \in \text{fri-vector-position-values } B \ i) \ s) \leq$
 $nnreal (\text{ceil-log } (to\text{-nat } dg + 1)) * \text{staged-phase-relation-error } (card \ B)$
 $(Suc$
 $(\text{staged-attacker-query-budget } budgets +$
 $\text{staged-challenge-query-budget}))$

proof –

have
 $(\sum i < \text{ceil-log } (to\text{-nat } dg + 1)).$
wp-event
 $(\text{staged-composition-fri-challenge-prefix-program } A \ i \gg=$
 $(\lambda prefix. \text{receive-composition-fri-challenge } \gg=$
 $(\lambda b. \text{return } (prefix, b))))$
 $(\lambda out. \text{case out of None} \Rightarrow \text{False}$
 $| \text{Some } ((-, b), -) \Rightarrow$
 $b \in \text{fri-vector-position-values } B \ i) \ s) \leq$
 $(\sum i < \text{ceil-log } (to\text{-nat } dg + 1)).$


```

    staged-phase-relation-error (card B)
    (Suc
      (staged-attacker-query-budget budgets +
        staged-challenge-query-budget)))
proof (rule sum-mono)
  fix i
  assume i-bound:  $i \in \{..<\text{ceil-log } (\text{to-nat } dg + 1)\}$ 
  then have i-lt:  $i < \text{ceil-log } (\text{to-nat } dg + 1)$ 
  by simp
  show
    wp-event
    (staged-composition-fri-challenge-prefix-program A i  $\gg$ 
      ( $\lambda$ prefix. receive-composition-fri-challenge  $\gg$ 
        ( $\lambda$ b. return (prefix, b))))
    ( $\lambda$ out. case out of None  $\Rightarrow$  False
      | Some ((-, b), -)  $\Rightarrow$ 
         $b \in \text{fri-vector-position-values } B \ i \ s \leq$ 
        staged-phase-relation-error (card B)
        (Suc
          (staged-attacker-query-budget budgets +
            staged-challenge-query-budget)))
  by (rule
    staged-composition-fri-prefix-keep-prefix-bad-vector-position-relation-bound
    [OF wf controlled subset i-lt dg-bound absent[OF i-lt]])
qed
also have ... =
  nnreal (ceil-log (to-nat dg + 1)) *
  staged-phase-relation-error (card B)
  (Suc
    (staged-attacker-query-budget budgets +
      staged-challenge-query-budget))
  by simp
finally show ?thesis .
qed

```

```

lemma staged-controlled-trace-root:
assumes staged-adversary-controlled budgets A
shows
  controlled-ro-program (trace-root-budget budgets)
  (trace-root-stage A)
using assms unfolding staged-adversary-controlled-def by simp

```

```

lemma staged-controlled-trace-fri:
assumes controlled: staged-adversary-controlled budgets A
and  $i < \text{length } (\text{trace-fri-budgets budgets})$ 
shows
  controlled-ro-program (trace-fri-budgets budgets ! i)
  (trace-fri-root-stage A i bs)
using assms unfolding staged-adversary-controlled-def by blast

```

lemma *staged-controlled-trace-final*:
assumes *staged-adversary-controlled budgets A*
shows
controlled-ro-program (trace-final-budget budgets)
(trace-final-stage A bs)
using *assms unfolding staged-adversary-controlled-def* **by** *blast*

lemma *staged-controlled-degree*:
assumes *staged-adversary-controlled budgets A*
shows
controlled-ro-program (degree-budget budgets)
(degree-stage A as)
using *assms unfolding staged-adversary-controlled-def* **by** *blast*

lemma *staged-controlled-composition-fri*:
assumes *controlled: staged-adversary-controlled budgets A*
and *i < length (composition-fri-budgets budgets)*
shows
controlled-ro-program (composition-fri-budgets budgets ! i)
(composition-fri-root-stage A dg i bs)
using *assms unfolding staged-adversary-controlled-def* **by** *blast*

lemma *staged-controlled-composition-final*:
assumes *staged-adversary-controlled budgets A*
shows
controlled-ro-program (composition-final-budget budgets)
(composition-final-stage A dg bs)
using *assms unfolding staged-adversary-controlled-def* **by** *blast*

lemma *staged-controlled-query*:
assumes *controlled: staged-adversary-controlled budgets A*
and *i < length (query-opening-budgets budgets)*
shows
controlled-ro-program (query-opening-budgets budgets ! i)
(query-opening-stage A i raw)
using *assms unfolding staged-adversary-controlled-def* **by** *blast*

lemma *staged-security-experiment-hash-map-output-collision-bad-bound*:
assumes *admissible*:
admissible-adversary q (staged-semantic-adversary A)
shows
wp-event (staged-security-experiment A)
(hash-map-output-collision-bad adversary-initial-state)
adversary-initial-state ≤
hash-collision-budget-value 0 (q + verifier-hash-query-budget)
unfolding *staged-security-experiment-eq-security-experiment*
by *(rule security-experiment-hash-map-output-collision-bad-bound*
[OF admissible])

lemma *staged-security-experiment-hash-map-output-collision-bad-bound-controlled:*

assumes *wf: staged-budget-wellformed budgets*

and *controlled: staged-adversary-controlled budgets A*

shows

wp-event (staged-security-experiment A)
(hash-map-output-collision-bad adversary-initial-state)
adversary-initial-state ≤
hash-collision-budget-value 0
(staged-attacker-query-budget budgets +
staged-challenge-query-budget + verifier-hash-query-budget)

by (*rule staged-security-experiment-hash-map-output-collision-bad-bound*
[OF admissible-staged-semantic-adversary[OF wf controlled]])

lemma *hash-range-budget-checked-staged-security-experiment:*

assumes *wf: staged-budget-wellformed budgets*

and *controlled: staged-adversary-controlled budgets A*

shows

hash-range-budget
(staged-attacker-query-budget budgets +
staged-challenge-query-budget + verifier-hash-query-budget)
(checked-staged-security-experiment A)

proof –

have *cont:*

∧ data. hash-range-budget verifier-hash-query-budget
(get >>=
(λ s. put
(verifier-state-from-adversary s
(staged-proof-transcript data)) >>=
(λ -. verify-monad)))

using *hash-range-budget-verifier-after-adversary*

[of staged-proof-transcript data for data]

unfolding *verifier-state-transfer-def sm-bind-assoc .*

have *budget:*

hash-range-budget
((staged-attacker-query-budget budgets +
staged-challenge-query-budget) +
verifier-hash-query-budget)
(checked-staged-security-experiment A)
unfolding *checked-staged-security-experiment-def*
by (*rule hash-range-budget-bind*)
(rule hash-range-budget-checked-staged-transcript-program
[OF wf controlled], rule cont)

then show *?thesis* **by** (*simp add: add.assoc*)

qed

lemma *hash-collision-budget-checked-staged-security-experiment:*

assumes *wf: staged-budget-wellformed budgets*

and *controlled: staged-adversary-controlled budgets A*

shows
hash-collision-budget
 (*staged-attacker-query-budget budgets* +
staged-challenge-query-budget + *verifier-hash-query-budget*)
 (*checked-staged-security-experiment A*)

proof –
have *cont-range*:
 $\wedge$ *data. hash-range-budget verifier-hash-query-budget*
 (*get* $\gg$
 ($\lambda s. \text{put}$
 (*verifier-state-from-adversary s*
 (*staged-proof-transcript data*)) $\gg$
 ($\lambda-. \text{verify-monad}$)))
using *hash-range-budget-verifier-after-adversary*
 [*of staged-proof-transcript data for data*]
unfolding *verifier-state-transfer-def sm-bind-assoc* .
have *cont-collision*:
 $\wedge$ *data. hash-collision-budget verifier-hash-query-budget*
 (*get* $\gg$
 ($\lambda s. \text{put}$
 (*verifier-state-from-adversary s*
 (*staged-proof-transcript data*)) $\gg$
 ($\lambda-. \text{verify-monad}$)))
using *hash-collision-budget-verifier-after-adversary*
 [*of staged-proof-transcript data for data*]
unfolding *verifier-state-transfer-def sm-bind-assoc* .
have *budget*:
hash-collision-budget
 ((*staged-attacker-query-budget budgets* +
staged-challenge-query-budget) +
verifier-hash-query-budget)
 (*checked-staged-security-experiment A*)
unfolding *checked-staged-security-experiment-def*
by (*rule hash-collision-budget-bind*)
 (*rule hash-range-budget-checked-staged-transcript-program*
 [*OF wf controlled*],
rule hash-collision-budget-checked-staged-transcript-program
 [*OF wf controlled*],
rule cont-range,
rule cont-collision)
then show *?thesis* **by** (*simp add: add.assoc*)
qed

lemma *checked-staged-security-experiment-hash-new-collision-bound-controlled*:
assumes *wf: staged-budget-wellformed budgets*
and *controlled: staged-adversary-controlled budgets A*
shows
wp-event (checked-staged-security-experiment A)
(hash-new-collision-event adversary-initial-state)

$adversary-initial-state \leq$
 $hash-collision-budget-value\ 0$
 $(staged-attacker-query-budget\ budgets +$
 $staged-challenge-query-budget + verifier-hash-query-budget)$
proof –
let $?q =$
 $staged-attacker-query-budget\ budgets +$
 $staged-challenge-query-budget + verifier-hash-query-budget$
have $collision-budget:$
 $hash-collision-budget\ ?q\ (checked-staged-security-experiment\ A)$
by $(rule\ hash-collision-budget-checked-staged-security-experiment$
 $[OF\ wf\ controlled])$
have $wp-event\ (checked-staged-security-experiment\ A)$
 $(hash-new-collision-event\ adversary-initial-state)$
 $adversary-initial-state \leq$
 $hash-collision-budget-value$
 $(card\ (hash-map-output-values\ adversary-initial-state))\ ?q$
using $collision-budget\ adversary-initial-state-no-output-collision$
unfolding $hash-collision-budget-def$ **by** $blast$
then show $?thesis$ **by** $simp$
qed

lemma $checked-staged-security-experiment-hash-map-output-collision-bad-bound-controlled:$
assumes $wf: staged-budget-wellformed\ budgets$
and $controlled: staged-adversary-controlled\ budgets\ A$
shows
 $wp-event\ (checked-staged-security-experiment\ A)$
 $(hash-map-output-collision-bad\ adversary-initial-state)$
 $adversary-initial-state \leq$
 $hash-collision-budget-value\ 0$
 $(staged-attacker-query-budget\ budgets +$
 $staged-challenge-query-budget + verifier-hash-query-budget)$
proof –
have $wp-event\ (checked-staged-security-experiment\ A)$
 $(hash-map-output-collision-bad\ adversary-initial-state)$
 $adversary-initial-state \leq$
 $wp-event\ (checked-staged-security-experiment\ A)$
 $(hash-new-collision-event\ adversary-initial-state)$
 $adversary-initial-state$
by $(rule\ wp-event-mono)$
 $(rule\ hash-map-output-collision-bad-imp-security-new-collision)$
also have $\dots \leq$
 $hash-collision-budget-value\ 0$
 $(staged-attacker-query-budget\ budgets +$
 $staged-challenge-query-budget + verifier-hash-query-budget)$
by $(rule\ checked-staged-security-experiment-hash-new-collision-bound-controlled$
 $[OF\ wf\ controlled])$
finally show $?thesis$.
qed

lemma *checked-staged-security-with-data-state-hash-map-output-collision-bad-bound-controlled:*
assumes *wf: staged-budget-wellformed budgets*
and *controlled: staged-adversary-controlled budgets A*
shows
wp-event (checked-staged-security-experiment-with-data-state A)
(staged-security-with-data-state-verifier-event
hash-map-output-collision-bad)
adversary-initial-state ≤
hash-collision-budget-value 0
(staged-attacker-query-budget budgets +
staged-challenge-query-budget + verifier-hash-query-budget)

proof –
let *?M = checked-staged-security-experiment-with-data-state A*
let *?P = hash-map-output-collision-bad adversary-initial-state*
let *?projected =*
λout. case out of None ⇒ ?P None
| Some (x, t) ⇒ ?P (Some (snd x, t))
have *event-le-projected:*
wp-event ?M
(staged-security-with-data-state-verifier-event
hash-map-output-collision-bad)
adversary-initial-state ≤
wp-event ?M ?projected adversary-initial-state

proof (rule *wp-event-mono*)
fix *out*
assume *bad:*
staged-security-with-data-state-verifier-event
hash-map-output-collision-bad out
show *?projected out*
proof (cases *out*)
case *None*
then show *?thesis*
using *bad unfolding staged-security-with-data-state-verifier-event-def*
by *simp*

next
case (Some *packed*)
then obtain *data attacker-state result final-state* **where** *out-eq:*
out = Some (((data, attacker-state), result), final-state)
by (cases *packed*, auto split: *prod.splits*)
have *local-bad:*
hash-map-output-collision-bad
(verifier-state-from-adversary attacker-state
(staged-proof-transcript data))
(Some (result, final-state))
using *bad unfolding out-eq*
staged-security-with-data-state-verifier-event-def **by** *simp*
then have *global-bad:*
hash-map-output-collision-bad adversary-initial-state

```

    (Some (result, final-state))
  unfolding hash-map-output-collision-bad-def by simp
show ?thesis
  unfolding out-eq using global-bad by simp
qed
qed
have projected-eq:
  wp-event ?M ?projected adversary-initial-state =
    wp-event (checked-staged-security-experiment A) ?P
    adversary-initial-state
proof -
  have ?M  $\gg$  ( $\lambda x$ . return (snd x)) =
    checked-staged-security-experiment A
  by (rule checked-staged-security-experiment-with-data-state-projection)
  then have wp-event (checked-staged-security-experiment A) ?P
    adversary-initial-state =
    wp-event (?M  $\gg$  ( $\lambda x$ . return (snd x))) ?P
    adversary-initial-state
  by simp
  also have ... = wp-event ?M ?projected adversary-initial-state
  by (subst wp-event-bind-return-map[where f=snd]) simp
  finally show ?thesis by simp
qed
have checked-bound:
  wp-event (checked-staged-security-experiment A)
    (hash-map-output-collision-bad adversary-initial-state)
    adversary-initial-state  $\leq$ 
    hash-collision-budget-value 0
    (staged-attacker-query-budget budgets +
      staged-challenge-query-budget + verifier-hash-query-budget)
  by (rule
    checked-staged-security-experiment-hash-map-output-collision-bad-bound-controlled
    [OF wf controlled])
have wp-event (checked-staged-security-experiment-with-data-state A)
  (staged-security-with-data-state-verifier-event
    hash-map-output-collision-bad)
  adversary-initial-state  $\leq$ 
  wp-event ?M ?projected adversary-initial-state
  by (rule event-le-projected)
also have ... =
  wp-event (checked-staged-security-experiment A) ?P
  adversary-initial-state
  by (rule projected-eq)
also have ...  $\leq$ 
  hash-collision-budget-value 0
  (staged-attacker-query-budget budgets +
    staged-challenge-query-budget + verifier-hash-query-budget)
  by (rule checked-bound)
finally show ?thesis .

```

qed

end

end

theory *Soundness-Oracle-Dynamic-Target*
 imports *Soundness-Oracle-Budgets*
begin

State-dependent target-hit combinators.

These lemmas are deliberately generic: the selected target set may depend on an intermediate oracle state, but all probability bounds are still discharged through the existing random-oracle target-budget interface.

context *soundness*
begin

definition *state-dependent-output-hit*
:: ((*f*, *a*) *protocol-channel-scheme* $\Rightarrow$ *f set*) $\Rightarrow$
 (*f*, *a*) *protocol-channel-scheme* $\Rightarrow$
 (*f* $\times$ (*f*, *a*) *protocol-channel-scheme*) *option* $\Rightarrow$ *bool*
where
 state-dependent-output-hit *B s out* $\longleftrightarrow$
 (case out of
 None $\Rightarrow$ *False*
 | *Some* (*y*, -) $\Rightarrow$ *y* $\in$ *B s*)

definition *state-dependent-new-output-hit*
:: ((*f*, *a*) *protocol-channel-scheme* $\Rightarrow$ *f set*) $\Rightarrow$
 (*f*, *a*) *protocol-channel-scheme* $\Rightarrow$
 (*r* $\times$ (*f*, *a*) *protocol-channel-scheme*) *option* $\Rightarrow$ *bool*
where
 state-dependent-new-output-hit *B s out* $\longleftrightarrow$
 hash-new-output-hit-event (*B s*) *s out*

definition *state-dependent-prequeried-hash-hit*
:: ((*f*, *a*) *protocol-channel-scheme* $\Rightarrow$ *f set*) $\Rightarrow$
 f protocol-hash-input $\Rightarrow$
 (*f*, *a*) *protocol-channel-scheme* $\Rightarrow$ *bool*
where
 state-dependent-prequeried-hash-hit *B x s* $\longleftrightarrow$
 ($\exists y. \text{fmlookup } (\text{HashMap } s) \ x = \text{Some } y \wedge y \in B \ s$)

lemma *state-dependent-output-hit-None[simp]*:
 $\neg$ *state-dependent-output-hit* *B s None*
 unfolding *state-dependent-output-hit-def* **by** *simp*

lemma *state-dependent-new-output-hit-None[simp]*:

$\neg$ *state-dependent-new-output-hit* $B\ s\ None$
unfolding *state-dependent-new-output-hit-def* *hash-new-output-hit-event-def*
by *simp*

lemma *state-dependent-prequeried-hash-hitI*:
assumes *fmlookup* (*HashMap* s) $x = Some\ y$
and $y \in B\ s$
shows *state-dependent-prequeried-hash-hit* $B\ x\ s$
using *assms* **unfolding** *state-dependent-prequeried-hash-hit-def* **by** *blast*

lemma *state-dependent-output-hit-mono*:
assumes *subset*: $\bigwedge s. B\ s \subseteq C\ s$
and *hit*: *state-dependent-output-hit* $B\ s\ out$
shows *state-dependent-output-hit* $C\ s\ out$
using *subset hit*
unfolding *state-dependent-output-hit-def*
by (*cases out*) *auto*

lemma *state-dependent-new-output-hit-mono*:
assumes *subset*: $\bigwedge s. B\ s \subseteq C\ s$
and *hit*: *state-dependent-new-output-hit* $B\ s\ out$
shows *state-dependent-new-output-hit* $C\ s\ out$
using *subset hit*
unfolding *state-dependent-new-output-hit-def* *hash-new-output-hit-event-def*
by (*cases out*) (*auto intro: hash-map-new-output-hit-subset*)

lemma *state-dependent-output-hit-union*:
state-dependent-output-hit $(\lambda s. B\ s \cup C\ s)\ s\ out \longleftrightarrow$
state-dependent-output-hit $B\ s\ out \vee$
state-dependent-output-hit $C\ s\ out$
unfolding *state-dependent-output-hit-def*
by (*cases out*) *auto*

lemma *state-dependent-new-output-hit-union-imp*:
assumes *hit*: *state-dependent-new-output-hit* $(\lambda s. B\ s \cup C\ s)\ s\ out$
shows
state-dependent-new-output-hit $B\ s\ out \vee$
state-dependent-new-output-hit $C\ s\ out$
proof (*cases out*)
case *None*
then show *?thesis*
using *hit* **by** *simp*
next
case (*Some* rt)
then obtain $r\ t$ **where** $rt = (r, t)$
by (*cases rt*) *simp*
obtain $x\ y$ **where**
none: *fmlookup* (*HashMap* s) $x = None$
and *some*: *fmlookup* (*HashMap* t) $x = Some\ y$

```

and  $y: y \in B \ s \cup C \ s$ 
using hit Some rt
unfolding state-dependent-new-output-hit-def hash-new-output-hit-event-def
  hash-map-new-output-hit-def
by auto
show ?thesis
proof (cases y ∈ B s)
  case True
  then show ?thesis
    using Some rt none some
    unfolding state-dependent-new-output-hit-def hash-new-output-hit-event-def
      hash-map-new-output-hit-def
    by auto
  next
  case False
  then have  $y \in C \ s$ 
    using  $y$  by simp
  then show ?thesis
    using Some rt none some
    unfolding state-dependent-new-output-hit-def hash-new-output-hit-event-def
      hash-map-new-output-hit-def
    by auto
qed
qed

```

lemma *wp-event-state-dependent-output-union-bound:*

```

wp-event  $m$ 
  (state-dependent-output-hit ( $\lambda s. B \ s \cup C \ s$ )  $s$ )  $s \leq$ 
  wp-event  $m$  (state-dependent-output-hit  $B \ s$ )  $s$  +
  wp-event  $m$  (state-dependent-output-hit  $C \ s$ )  $s$ 
unfolding state-dependent-output-hit-union
by (rule wp-event-union-bound)

```

lemma *wp-event-state-dependent-new-output-union-bound:*

```

wp-event  $m$ 
  (state-dependent-new-output-hit ( $\lambda s. B \ s \cup C \ s$ )  $s$ )  $s \leq$ 
  wp-event  $m$  (state-dependent-new-output-hit  $B \ s$ )  $s$  +
  wp-event  $m$  (state-dependent-new-output-hit  $C \ s$ )  $s$ 
proof –
  have wp-event  $m$ 
    (state-dependent-new-output-hit ( $\lambda s. B \ s \cup C \ s$ )  $s$ )  $s \leq$ 
    wp-event  $m$ 
      ( $\lambda out. \text{state-dependent-new-output-hit } B \ s \ out \vee$ 
        state-dependent-new-output-hit  $C \ s \ out$ )  $s$ 
    by (rule wp-event-mono) (rule state-dependent-new-output-hit-union-imp)
  also have ...  $\leq$ 
    wp-event  $m$  (state-dependent-new-output-hit  $B \ s$ )  $s$  +
    wp-event  $m$  (state-dependent-new-output-hit  $C \ s$ )  $s$ 
    by (rule wp-event-union-bound)

```

```

finally show ?thesis .
qed

lemma wp-hash-state-dependent-new-output-hit:
  fixes x :: 'f protocol-hash-input
  shows
    wp-event (hash x) (state-dependent-new-output-hit B s) s =
      (if fmllookup (HashMap s) x = None
        then nnreal (card (B s)) / nnreal size
        else 0)
  unfolding state-dependent-new-output-hit-def
  by (rule wp-hash-new-output-hit)

lemma wp-hash-state-dependent-new-output-hit-bound:
  fixes x :: 'f protocol-hash-input
  shows
    wp-event (hash x) (state-dependent-new-output-hit B s) s ≤
      nnreal (card (B s)) / nnreal size
  unfolding state-dependent-new-output-hit-def
  by (rule wp-hash-new-output-hit-bound)

lemma wp-hash-state-dependent-new-output-hit-bound-card:
  fixes x :: 'f protocol-hash-input
  assumes card-bound: card (B s) ≤ N
  shows
    wp-event (hash x) (state-dependent-new-output-hit B s) s ≤
      nnreal N / nnreal size
proof –
  have wp-event (hash x) (state-dependent-new-output-hit B s) s ≤
    nnreal (card (B s)) / nnreal size
  by (rule wp-hash-state-dependent-new-output-hit-bound)
  also have ... ≤ nnreal N / nnreal size
  by (rule nnreal-nat-divide-right-mono[OF card-bound])
  finally show ?thesis .
qed

lemma hash-state-dependent-output-hit-fresh-or-prequeried:
  fixes x :: 'f protocol-hash-input
  assumes out: out ∈ set-dist (execute (hash x) s)
  and hit: state-dependent-output-hit B s out
  shows
    state-dependent-new-output-hit B s out ∨
    state-dependent-prequeried-hash-hit B x s
proof (cases out)
  case None
  then show ?thesis
  using hit by simp
next
  case (Some ht)

```

```

then obtain  $h\ t$  where  $ht$ :  $ht = (h, t)$ 
  by  $(cases\ ht)\ simp$ 
have  $hash-out$ :  $Some\ (h, t) \in set-dist\ (execute\ (hash\ x)\ s)$ 
  using  $out\ Some\ ht$  by  $simp$ 
have  $h-in$ :  $h \in B\ s$ 
  using  $hit\ Some\ ht$  unfolding  $state-dependent-output-hit-def$  by  $simp$ 
have  $lookup-t$ :  $fmlookup\ (HashMap\ t)\ x = Some\ h$ 
  using  $protocol-merkle.hash-outcome(2)[OF\ hash-out]$  .
show  $?thesis$ 
proof  $(cases\ fmlookup\ (HashMap\ s)\ x)$ 
  case  $None$ 
    then have  $hash-map-new-output-hit\ (B\ s)\ s\ t$ 
      unfolding  $hash-map-new-output-hit-def$ 
      using  $lookup-t\ h-in$  by  $blast$ 
    then show  $?thesis$ 
      using  $Some\ ht$ 
      unfolding  $state-dependent-new-output-hit-def\ hash-new-output-hit-event-def$ 
      by  $simp$ 
  next
    case  $(Some\ y)$ 
    then have  $y = h$ 
    using  $lookup-t\ hash-extension-lookup[OF\ Some\ protocol-merkle.hash-outcome(1)[OF\ hash-out]]$ 
      by  $simp$ 
    then show  $?thesis$ 
      using  $Some\ h-in$ 
      unfolding  $state-dependent-prequeried-hash-hit-def$ 
      by  $blast$ 
  qed
qed

lemma  $receive-alpha-challenge-state-dependent-hit-fresh-or-prequeried$ :
  assumes  $out$ :  $Some\ (a, t) \in set-dist\ (execute\ receive-alpha-challenge\ s)$ 
  and  $hit$ :  $a \in B\ s$ 
  shows
     $state-dependent-new-output-hit\ B\ s\ (Some\ (a, t)) \vee$ 
     $state-dependent-prequeried-hash-hit\ B$ 
     $(AlphaChallenge\ (PAlphaCounter\ s)\ (PState\ s))\ s$ 
proof –
  have  $recv$ :
     $s \leq t \wedge$ 
     $fmlookup\ (HashMap\ t)$ 
     $(AlphaChallenge\ (PAlphaCounter\ s)\ (PState\ s)) = Some\ a$ 
  using  $receive-alpha-challenge-outcome[OF\ out]$  by  $blast$ 
show  $?thesis$ 
proof  $(cases$ 
   $fmlookup\ (HashMap\ s)$ 
   $(AlphaChallenge\ (PAlphaCounter\ s)\ (PState\ s)))$ 
  case  $None$ 
    then have  $hash-map-new-output-hit\ (B\ s)\ s\ t$ 

```

```

    unfolding hash-map-new-output-hit-def
    using recv hit by blast
  then show ?thesis
    unfolding state-dependent-new-output-hit-def
      hash-new-output-hit-event-def
    by simp
next
case (Some old-a)
have old-lookup-t:
  fmlookup (HashMap t)
  (AlphaChallenge (PAlphaCounter s) (PState s)) = Some old-a
  by (rule hash-extension-lookup[OF Some conjunct1[OF recv]])
then have old-a = a
  using recv by simp
then show ?thesis
  using Some hit
  unfolding state-dependent-prequeried-hash-hit-def
  by blast
qed
qed

lemma wp-receive-alpha-challenge-state-dependent-hit-split:
  wp-event receive-alpha-challenge
    ( $\lambda out. \text{case out of None} \Rightarrow \text{False}$ 
     |  $\text{Some } (a, -) \Rightarrow a \in B \ s$ )  $s \leq$ 
  wp-event receive-alpha-challenge
    (state-dependent-new-output-hit B s)  $s +$ 
  (if state-dependent-prequeried-hash-hit B
    (AlphaChallenge (PAlphaCounter s) (PState s)) s
    then 1 else 0)
proof -
  let ?bad =
     $\lambda out. \text{case out of None} \Rightarrow \text{False}$ 
    |  $\text{Some } (a, -) \Rightarrow a \in B \ s$ 
  let ?fresh = state-dependent-new-output-hit B s
  let ?pre =
    state-dependent-prequeried-hash-hit B
    (AlphaChallenge (PAlphaCounter s) (PState s)) s
  have wp-event receive-alpha-challenge ?bad  $s \leq$ 
    wp-event receive-alpha-challenge
      ( $\lambda out. ?fresh \text{ out} \vee ?pre$ ) s
  proof (rule wp-event-mono-on-support)
    fix out
    assume support:  $out \in \text{set-dist } (\text{execute receive-alpha-challenge } s)$ 
    and bad: ?bad out
    show ?fresh out  $\vee ?pre$ 
    proof (cases out)
      case None
      then show ?thesis

```

```

    using bad by simp
next
case (Some pair)
then obtain a t where out-eq: out = Some (a, t)
  by (cases pair) auto
have a-in: a ∈ B s
  using bad unfolding out-eq by simp
show ?thesis
  unfolding out-eq
  by (rule
    receive-alpha-challenge-state-dependent-hit-fresh-or-prequeried
    [where B=B and s=s and a=a and t=t, OF - a-in])
    (use support out-eq in simp)
qed
qed
also have ... ≤
  wp-event receive-alpha-challenge ?fresh s +
  wp-event receive-alpha-challenge (λ-. ?pre) s
  by (rule wp-event-union-bound)
also have ... ≤
  wp-event receive-alpha-challenge ?fresh s +
  (if ?pre then 1 else 0)
proof (cases ?pre)
case True
have wp-event receive-alpha-challenge (λ-. ?pre) s ≤ 1
  by (rule wp-event-le-1)
then show ?thesis
  using True by simp
next
case False
then show ?thesis
  by (simp add: wp-event-def wp-def)
qed
finally show ?thesis .
qed

lemma wp-event-bind-state-dependent-output-bound-by-cont:
  fixes m :: ('x, ('f, 'a) protocol-channel-scheme) state-monad
  and k :: 'x ⇒ ('f, ('f, 'a) protocol-channel-scheme) state-monad
  assumes none: ¬ E None
  and event-imp:
    ∧ x t out. Some (x, t) ∈ set-dist (execute m s) ⇒
      out ∈ set-dist (execute (k x) t) ⇒
      E out ⇒ state-dependent-output-hit B t out
  and cont-bound:
    ∧ x t. Some (x, t) ∈ set-dist (execute m s) ⇒
      wp-event (k x) (state-dependent-output-hit B t) t ≤ C
  shows wp-event (m ≫ k) E s ≤ C
proof (rule wp-event-bind-bound-by-cont

```

```

[where  $Q=E$  and  $m=m$  and  $k=k$  and  $s=s$  and  $C=C$ ]
show  $\neg E \text{ None}$ 
  by (rule none)
next
fix  $x \ t$ 
assume head:  $\text{Some } (x, t) \in \text{set-dist } (\text{execute } m \ s)$ 
have wp-event  $(k \ x) \ E \ t \leq$ 
  wp-event  $(k \ x) \ (\text{state-dependent-output-hit } B \ t) \ t$ 
  by (rule wp-event-mono-on-support)
  (use head event-imp in blast)
also have  $\dots \leq C$ 
  by (rule cont-bound[OF head])
finally show wp-event  $(k \ x) \ E \ t \leq C$  .
qed

```

lemma *wp-event-bind-state-dependent-new-output-bound-by-cont:*
fixes $m :: ('x, ('f, 'a) \text{protocol-channel-scheme}) \text{state-monad}$
and $k :: 'x \Rightarrow ('y, ('f, 'a) \text{protocol-channel-scheme}) \text{state-monad}$
assumes none: $\neg E \text{ None}$
and event-imp:
 $\bigwedge x \ t \text{ out. } \text{Some } (x, t) \in \text{set-dist } (\text{execute } m \ s) \implies$
 $\text{out} \in \text{set-dist } (\text{execute } (k \ x) \ t) \implies$
 $E \text{ out} \implies \text{state-dependent-new-output-hit } B \ t \text{ out}$
and cont-bound:
 $\bigwedge x \ t. \text{Some } (x, t) \in \text{set-dist } (\text{execute } m \ s) \implies$
 $\text{wp-event } (k \ x) \ (\text{state-dependent-new-output-hit } B \ t) \ t \leq C$
shows wp-event $(m \gg k) \ E \ s \leq C$
proof (rule wp-event-bind-bound-by-cont
[where $Q=E$ and $m=m$ and $k=k$ and $s=s$ and $C=C$])
show $\neg E \text{ None}$
 by (rule none)
next
fix $x \ t$
assume head: $\text{Some } (x, t) \in \text{set-dist } (\text{execute } m \ s)$
have wp-event $(k \ x) \ E \ t \leq$
 wp-event $(k \ x) \ (\text{state-dependent-new-output-hit } B \ t) \ t$
 by (rule wp-event-mono-on-support)
 (use head event-imp in blast)
also have $\dots \leq C$
 by (rule cont-bound[OF head])
finally show wp-event $(k \ x) \ E \ t \leq C$.
qed

lemma *wp-event-bind-state-dependent-new-output-bound-by-target-budget:*
fixes $m :: ('x, ('f, 'a) \text{protocol-channel-scheme}) \text{state-monad}$
and $k :: 'x \Rightarrow ('y, ('f, 'a) \text{protocol-channel-scheme}) \text{state-monad}$
assumes none: $\neg E \text{ None}$
and event-imp:
 $\bigwedge x \ t \text{ out. } \text{Some } (x, t) \in \text{set-dist } (\text{execute } m \ s) \implies$

```

    out ∈ set-dist (execute (k x) t) ⇒
    E out ⇒ state-dependent-new-output-hit B t out
  and budget:
    ∧x t. Some (x, t) ∈ set-dist (execute m s) ⇒
    hash-target-budget (B t) n (k x)
  and value-bound:
    ∧x t. Some (x, t) ∈ set-dist (execute m s) ⇒
    hash-target-budget-value (B t) n ≤ C
  shows
    wp-event (m ≫ k) E s ≤ C
proof (rule wp-event-bind-state-dependent-new-output-bound-by-cont
  [where E=E and m=m and k=k and s=s and B=B and C=C])
  show ¬ E None
  by (rule none)
next
  show ∧x t out.
    Some (x, t) ∈ set-dist (execute m s) ⇒
    out ∈ set-dist (execute (k x) t) ⇒
    E out ⇒ state-dependent-new-output-hit B t out
  by (rule event-imp)
next
  fix x t
  assume head: Some (x, t) ∈ set-dist (execute m s)
  have wp-event (k x) (state-dependent-new-output-hit B t) t ≤
    hash-target-budget-value (B t) n
  using budget[OF head]
  unfolding hash-target-budget-def state-dependent-new-output-hit-def
  by blast
  also have ... ≤ C
  by (rule value-bound[OF head])
  finally show
    wp-event (k x) (state-dependent-new-output-hit B t) t ≤ C .
qed

end

end

```

```

theory Staged-Security-Experiment-Alpha-Prequery
imports
  Staged-Security-Experiment-Checked-Prefix
  Soundness-Oracle-Dynamic-Target
begin

```

Alpha-key prequery support for staged soundness.

This layer records deterministic facts about the alpha challenge key path. It is proof infrastructure for accounting adversarial prequeries to future alpha keys; it does not add assumptions or change the oracle model.

context *soundness*
begin

definition *alpha-challenge-key-at*
 $:: 'f \text{ protocol-channel} \Rightarrow \text{nat} \Rightarrow 'f \text{ list} \Rightarrow$
 $\quad 'f \text{ protocol-hash-input}$
where
 $\text{alpha-challenge-key-at } s \ i \ as =$
 $\quad \text{AlphaChallenge } (P\text{AlphaCounter } s + i)$
 $\quad (\text{foldl concat } (P\text{State } s) (\text{take } i \ as))$

definition *alpha-vector-key-relation*
 $:: 'f \text{ protocol-channel} \Rightarrow 'f \text{ list set} \Rightarrow \text{nat} \Rightarrow$
 $\quad 'f \text{ protocol-hash-input} \Rightarrow 'f \Rightarrow \text{bool}$
where
 $\text{alpha-vector-key-relation } s \ B \ n \ x \ y \longleftrightarrow$
 $\quad (\exists as \in B. \exists i < n.$
 $\quad \quad i < \text{length } as \wedge$
 $\quad \quad x = \text{alpha-challenge-key-at } s \ i \ as \wedge$
 $\quad \quad y = as \ ! \ i)$

definition *alpha-vector-prequery-hit-event*
 $:: 'f \text{ protocol-channel} \Rightarrow 'f \text{ list set} \Rightarrow \text{nat} \Rightarrow$
 $\quad ('r \times 'f \text{ protocol-channel}) \text{ option} \Rightarrow \text{bool}$
where
 $\text{alpha-vector-prequery-hit-event } s \ B \ n =$
 $\quad \text{hash-relation-hit-event } (\text{alpha-vector-key-relation } s \ B \ n) \ s$

definition *alpha-vector-prequeried-in-state*
 $:: 'f \text{ protocol-channel} \Rightarrow 'f \text{ list} \Rightarrow \text{nat} \Rightarrow \text{bool}$
where
 $\text{alpha-vector-prequeried-in-state } s \ as \ n \longleftrightarrow$
 $\quad (\exists i < n.$
 $\quad \quad i < \text{length } as \wedge$
 $\quad \quad \text{fmlookup } (\text{HashMap } s) (\text{alpha-challenge-key-at } s \ i \ as) =$
 $\quad \quad \text{Some } (as \ ! \ i))$

definition *alpha-vector-path-fresh*
 $:: 'f \text{ protocol-channel} \Rightarrow 'f \text{ list} \Rightarrow \text{nat} \Rightarrow \text{bool}$
where
 $\text{alpha-vector-path-fresh } s \ as \ n \longleftrightarrow$
 $\quad (\forall i < n.$
 $\quad \quad i < \text{length } as \longrightarrow$
 $\quad \quad \text{fmlookup } (\text{HashMap } s) (\text{alpha-challenge-key-at } s \ i \ as) = \text{None})$

lemma *alpha-vector-key-relation-fiber-card-bound:*
assumes *finite-B: finite B*
shows $\text{card } \{y. \text{alpha-vector-key-relation } s \ B \ n \ x \ y\} \leq \text{card } B * n$
proof —

```

let ?pairs =  $B \times \{..<n\}$ 
let ?values =  $(\lambda(as, i). as ! i) \text{ ` } ?pairs$ 
have subset:
   $\{y. \text{alpha-vector-key-relation } s \ B \ n \ x \ y\} \subseteq ?values$ 
  unfolding alpha-vector-key-relation-def
  by force
have finite-values: finite ?values
  using finite-B by simp
have card  $\{y. \text{alpha-vector-key-relation } s \ B \ n \ x \ y\} \leq \text{card } ?values$ 
  by (rule card-mono[OF finite-values subset])
also have  $\dots \leq \text{card } ?pairs$ 
  by (rule card-image-le) (use finite-B in simp)
also have  $\dots = \text{card } B * n$ 
  using finite-B by simp
finally show ?thesis .
qed

lemma hash-relation-program-staged-alpha-prefix-alpha-vector-prequery:
  assumes wf: staged-budget-wellformed budgets
    and controlled: staged-adversary-controlled budgets A
    and finite-B: finite B
  shows
    hash-relation-program (alpha-vector-key-relation s B n) (card B * n)
      (staged-alpha-search-queries budgets 0)
      (staged-alpha-prefix-program A)
  by (rule hash-relation-program-staged-alpha-prefix-program
      [OF wf controlled])
    (rule alpha-vector-key-relation-fiber-card-bound[OF finite-B])

lemma staged-alpha-prefix-alpha-vector-prequery-hit-bound:
  assumes wf: staged-budget-wellformed budgets
    and controlled: staged-adversary-controlled budgets A
    and finite-B: finite B
  shows
    wp-event (staged-alpha-prefix-program A)
      (alpha-vector-prequery-hit-event s B n) s  $\leq$ 
      hash-relation-budget-value (card B * n)
      (staged-alpha-search-queries budgets 0)
proof -
  have program:
    hash-relation-program (alpha-vector-key-relation s B n) (card B * n)
      (staged-alpha-search-queries budgets 0)
      (staged-alpha-prefix-program A)
  by (rule hash-relation-program-staged-alpha-prefix-alpha-vector-prequery
      [OF wf controlled finite-B])
  then have budget:
    hash-relation-budget (alpha-vector-key-relation s B n) (card B * n)
      (staged-alpha-search-queries budgets 0)
      (staged-alpha-prefix-program A)

```

```

    unfolding hash-relation-program-def by simp
show ?thesis
    using budget
    unfolding alpha-vector-prequery-hit-event-def
    hash-relation-budget-def by blast
qed

lemma staged-alpha-prefix-event-bound-from-hash-relation:
  assumes wf: staged-budget-wellformed budgets
    and controlled: staged-adversary-controlled budgets A
    and fibers:  $\bigwedge x. \text{card } \{y. R \ x \ y\} \leq b$ 
    and event-imp:
       $\bigwedge x \ t. \text{Some } (x, t) \in$ 
         $\text{set-dist } (\text{execute } (\text{staged-alpha-prefix-program } A) \ s) \implies$ 
         $E \ (\text{Some } (x, t)) \implies \text{hash-relation-hit } R \ s \ t$ 
    and none:  $\neg E \ \text{None}$ 
  shows
     $\text{wp-event } (\text{staged-alpha-prefix-program } A) \ E \ s \leq$ 
       $\text{hash-relation-budget-value } b \ (\text{staged-alpha-search-queries } \text{budgets } 0)$ 
proof -
  have program:
     $\text{hash-relation-program } R \ b \ (\text{staged-alpha-search-queries } \text{budgets } 0)$ 
     $(\text{staged-alpha-prefix-program } A)$ 
  by (rule hash-relation-program-staged-alpha-prefix-program
    [OF wf controlled fibers])
  have
     $\text{wp-event } (\text{staged-alpha-prefix-program } A) \ E \ s \leq$ 
     $\text{wp-event } (\text{staged-alpha-prefix-program } A)$ 
     $(\text{hash-relation-hit-event } R \ s) \ s$ 
proof (rule wp-event-mono-on-support)
  fix out
  assume support:
     $\text{out} \in \text{set-dist } (\text{execute } (\text{staged-alpha-prefix-program } A) \ s)$ 
    and event:  $E \ \text{out}$ 
  show hash-relation-hit-event  $R \ s \ \text{out}$ 
proof (cases out)
  case None
  then show ?thesis using none event by simp
next
  case (Some xt)
  then obtain  $x \ t$  where  $xt: xt = (x, t)$ 
  by (cases xt) simp
  have hash-relation-hit  $R \ s \ t$ 
  by (rule event-imp[of x t]) (use support event Some xt in simp-all)
  then show ?thesis
  unfolding Some xt hash-relation-hit-event-def by simp
qed
qed
also have ...  $\leq$ 

```

```

    hash-relation-budget-value b (staged-alpha-search-queries budgets 0)
  using program unfolding hash-relation-program-def hash-relation-budget-def
  by blast
  finally show ?thesis .
qed

```

```

lemma alpha-vector-path-fresh-Cons-tail:
  assumes fresh: alpha-vector-path-fresh s (a # xs) (Suc n)
  and recv:
    Some (a, t) ∈ set-dist (execute receive-alpha-challenge s)
  and record-out:
    Some ((), u) ∈ set-dist (execute (record-staged-message a) t)
  shows alpha-vector-path-fresh u xs n

```

```

proof -
  have recv-fields:
    PAlphaCounter t = Suc (PAlphaCounter s)
    PState t = PState s
    fmlookup (HashMap t)
      (AlphaChallenge (PAlphaCounter s) (PState s)) = Some a
  using receive-alpha-challenge-outcome[OF recv]
    receive-alpha-challenge-counter-outcome[OF recv]
  by simp-all
  have u-eq:
    u = t |
      PState := concat (PState t) a,
      PTranscript := PTranscript t @ [a]]
  by (rule record-staged-message-outcome[OF record-out])
  show ?thesis
    unfolding alpha-vector-path-fresh-def
  proof (intro allI impI)
    fix i
    assume i-bound: i < n
    and i-len: i < length xs
    let ?tail-key = alpha-challenge-key-at u i xs
    let ?full-key = alpha-challenge-key-at s (Suc i) (a # xs)
    have key-eq: ?tail-key = ?full-key
      unfolding alpha-challenge-key-at-def u-eq
      using recv-fields
    by simp
    have full-none:
      fmlookup (HashMap s) ?full-key = None
    using fresh i-bound i-len
    unfolding alpha-vector-path-fresh-def
    by simp
    have key-ne:
      ?full-key ≠ AlphaChallenge (PAlphaCounter s) (PState s)
    unfolding alpha-challenge-key-at-def by simp
    have lookup-t:
      fmlookup (HashMap t) ?full-key = None

```

```

    using receive-alpha-challenge-preserves-other-lookup[OF recv key-ne]
      full-none
    by simp
  show fmlookup (HashMap u) ?tail-key = None
    using lookup-t key-eq unfolding u-eq by simp
qed
qed

lemma wp-staged-alpha-program-exact-challenges-bound-from-path-fresh:
  assumes len: length as = n
    and fresh: alpha-vector-path-fresh s as n
  shows
    wp-event (staged-alpha-program n)
      (λout. case out of None ⇒ False
        | Some (as', -) ⇒ as' = as) s ≤
      (1 / nnreal size) ^ n
    using len fresh
  proof (induction n arbitrary: as s)
    case 0
    then have as-empty: as = []
      by simp
    show ?case
      unfolding as-empty by (simp add: wp-event-def wpsimps)
  next
    case (Suc n)
    obtain a as-tail where as-eq: as = a # as-tail
      using Suc.prem1 by (cases as) auto
    have len-tail: length as-tail = n
      using Suc.prem1 as-eq by simp
    let ?tail =
      staged-alpha-program n ::
        ('f list, 'f protocol-channel) state-monad
    let ?Q =
      λout :: ('f list ×
        'f protocol-channel) option.
        case out of None ⇒ False | Some (as', -) ⇒ as' = as
    let ?Head =
      λout :: ('f ×
        'f protocol-channel) option.
        case out of None ⇒ False | Some (a0, -) ⇒ a0 ∈ {a}
    have head-fresh:
      fmlookup (HashMap s)
        (AlphaChallenge (PAlphaCounter s) (PState s)) = None
  proof -
    have fmlookup (HashMap s)
      (alpha-challenge-key-at s 0 (a # as-tail)) = None
      using Suc.prem2
      unfolding as-eq alpha-vector-path-fresh-def by simp
    then show ?thesis

```

```

    unfolding alpha-challenge-key-at-def by simp
qed
have head-bound:
  wp-event receive-alpha-challenge ?Head s ≤ 1 / nnreal size
proof -
  have wp-event receive-alpha-challenge ?Head s =
    nnreal (card ({a} :: 'f set)) / nnreal size
  by (rule wp-receive-alpha-challenge-fresh-set[OF head-fresh])
  then show ?thesis by simp
qed
have bind-bound:
  wp-event
    (receive-alpha-challenge >>=
      (λa0. record-staged-message a0 >>=
        (λ-. ?tail >>=
          (λas'. return (a0 # as')))))
    ?Q s ≤
  wp-event receive-alpha-challenge ?Head s *
    (1 / nnreal size) ^ n
proof (rule wp-event-bind-bound-by-head-and-cont[where P = ?Head])
  show ¬ ?Q None
  by simp
next
  fix a0 t
  assume recv:
    Some (a0, t) ∈ set-dist (execute receive-alpha-challenge s)
  and not-head: ¬ ?Head (Some (a0, t))
  have a0-ne: a0 ≠ a
  using not-head by simp
  have cont-false:
    wp-event
      (record-staged-message a0 >>=
        (λ-. ?tail >>= (λas'. return (a0 # as'))))
    ?Q t ≤
  wp-event
    (record-staged-message a0 >>=
      (λ-. ?tail >>= (λas'. return (a0 # as'))))
    (λ- :: ('f list × 'f protocol-channel) option. False) t
  by (rule wp-event-mono-on-support)
  (use a0-ne as-eq in
    ⟨auto simp: wpsimps elim!: set-dist-bindE
      split: option.splits prod.splits⟩)
  also have ... = 0
  unfolding wp-event-def wp-def dist-expect-def by simp
  finally have le-zero:
    wp-event
      (record-staged-message a0 >>=
        (λ-. ?tail >>= (λas'. return (a0 # as'))))
    ?Q t ≤ 0 .

```

```

show
  wp-event
    (record-staged-message a0  $\gg$ 
      ( $\lambda$ -. ?tail  $\gg$  ( $\lambda$ as'. return (a0 # as'))))
    ?Q t = 0
  by (rule antisym[OF le-zero]) simp
next
fix a0 t
assume recv:
  Some (a0, t)  $\in$  set-dist (execute receive-alpha-challenge s)
  and head: ?Head (Some (a0, t))
have a0-eq: a0 = a
  using head by simp
have record-cont-bound:
  wp-event
    (record-staged-message a0  $\gg$ 
      ( $\lambda$ -. ?tail  $\gg$  ( $\lambda$ as'. return (a0 # as'))))
    ?Q t  $\leq$  (1 / nnreal size) ^ n
proof (rule wp-event-bind-bound-by-cont)
  show  $\neg$  ?Q None
  by simp
next
fix uu u
assume record-out:
  Some (uu, u)  $\in$ 
    set-dist (execute (record-staged-message a0) t)
have record-unit:
  Some ((), u)  $\in$ 
    set-dist (execute (record-staged-message a0) t)
  using record-out by (cases uu) simp
have tail-fresh: alpha-vector-path-fresh u as-tail n
proof -
  have fresh-cons:
    alpha-vector-path-fresh s (a # as-tail) (Suc n)
    using Suc.premis(2) unfolding as-eq .
  have recv-a:
    Some (a, t)  $\in$  set-dist (execute receive-alpha-challenge s)
    using recv a0-eq by simp
  have record-a:
    Some ((), u)  $\in$  set-dist (execute (record-staged-message a) t)
    using record-unit a0-eq by simp
  show ?thesis
  by (rule alpha-vector-path-fresh-Cons-tail
    [OF fresh-cons recv-a record-a])
qed
have tail-bound:
  wp-event ?tail
    ( $\lambda$ out. case out of None  $\Rightarrow$  False
      | Some (as', -)  $\Rightarrow$  as' = as-tail) u

```

```

    ≤ (1 / nnreal size) ^ n
  by (rule Suc.IH[OF len-tail tail-fresh])
have cont-eq:
  wp-event
    (?tail >>= (λas'. return (a0 # as'))) ?Q u =
  wp-event ?tail
    (λout. case out of None ⇒ False
      | Some (as', -) ⇒ as' = as-tail) u
  by (simp add: wp-event-bind-return-map a0-eq as-eq
    split: option.splits prod.splits)
show
  wp-event (?tail >>= (λas'. return (a0 # as'))) ?Q u
    ≤ (1 / nnreal size) ^ n
  unfolding cont-eq by (rule tail-bound)
qed
show
  wp-event
    (record-staged-message a0 >>=
      (λ-. ?tail >>= (λas'. return (a0 # as'))))
    ?Q t ≤ (1 / nnreal size) ^ n
  by (rule record-cont-bound)
qed
have wp-event (staged-alpha-program (Suc n)) ?Q s
  ≤ wp-event receive-alpha-challenge ?Head s *
    (1 / nnreal size) ^ n
  unfolding staged-alpha-program.simps by (rule bind-bound)
also have ... ≤ (1 / nnreal size) * (1 / nnreal size) ^ n
  by (rule mult-right-mono[OF head-bound]) simp
also have ... = (1 / nnreal size) ^ Suc n
  by simp
finally show ?case .
qed

lemma wp-staged-alpha-program-finite-set-bound-from-path-fresh:
  assumes finite-B: finite B
  and lengths: ⋀as. as ∈ B ⇒ length as = n
  and fresh: ⋀as. as ∈ B ⇒ alpha-vector-path-fresh s as n
  shows
    wp-event (staged-alpha-program n)
      (λout. case out of None ⇒ False
        | Some (as, -) ⇒ as ∈ B) s ≤
      nnreal (card B) / (nnreal size) ^ n
proof -
  let ?P =
    λa out. case out of None ⇒ False
      | Some (as', -) ⇒ as' = a
  have event-mono:
    wp-event (staged-alpha-program n)
      (λout. case out of None ⇒ False

```



```

    | Some (as, -)  $\Rightarrow$  as  $\in$  B) s  $\leq$ 
    wp-event (staged-alpha-program n)
    ( $\lambda$ out.  $\exists$  as $\in$ B. ?P as out) s
  by (rule wp-event-mono) (auto split: option.splits prod.splits)
also have ...  $\leq$ 
  ( $\sum$  a $\in$ B. wp-event (staged-alpha-program n) (?P a) s)
  by (rule wp-event-finite-UN-bound[where A = B and P = ?P])
    (use finite-B in simp-all)
also have ...  $\leq$  ( $\sum$  as $\in$ B. (1 / nnreal size) ^ n)
proof (rule sum-mono)
  fix as
  assume as-in: as  $\in$  B
  show
    wp-event (staged-alpha-program n) (?P as) s  $\leq$ 
    (1 / nnreal size) ^ n
  by (rule wp-staged-alpha-program-exact-challenges-bound-from-path-fresh
    [OF lengths[OF as-in] fresh[OF as-in]])
qed
also have ... = nnreal (card B) * ((1 / nnreal size) ^ n)
  using finite-B by simp
also have ... = nnreal (card B) / (nnreal size) ^ n
  using size-card by transfer (simp add: power-divide)
finally show ?thesis .
qed

lemma alpha-vector-prequeried-in-state-imp-hash-relation-hit:
  assumes prequeried: alpha-vector-prequeried-in-state s as n
  and member: as  $\in$  B
  and initial-absent:
     $\bigwedge i. i < n \Rightarrow i < \text{length } as \Rightarrow$ 
      fmllookup (HashMap initial) (alpha-challenge-key-at s i as) = None
  shows hash-relation-hit (alpha-vector-key-relation s B n) initial s
proof -
  from prequeried obtain i where
    i-n: i < n
  and i-len: i < length as
  and lookup:
    fmllookup (HashMap s) (alpha-challenge-key-at s i as) =
      Some (as ! i)
  unfolding alpha-vector-prequeried-in-state-def by blast
  have relation:
    alpha-vector-key-relation s B n (alpha-challenge-key-at s i as)
    (as ! i)
  unfolding alpha-vector-key-relation-def
  using member i-n i-len by blast
  show ?thesis
  unfolding hash-relation-hit-def
  using initial-absent[OF i-n i-len] lookup relation by blast
qed

```

lemma *alpha-vector-prequeried-in-state-imp-hash-relation-hit-empty-initial:*
assumes *prequeried:* *alpha-vector-prequeried-in-state s as n*
and *member:* *as ∈ B*
and *empty:* *HashMap initial = fmempty*
shows *hash-relation-hit (alpha-vector-key-relation s B n) initial s*
by (*rule alpha-vector-prequeried-in-state-imp-hash-relation-hit*
[OF prequeried member])
(simp add: empty)

lemma *staged-alpha-program-outcome-lookup:*
assumes *outcome:*
Some (as, t) ∈ set-dist (execute (staged-alpha-program n) s)
and *i-bound:* *i < n*
shows
fmlookup (HashMap t)
(AlphaChallenge (PAlphaCounter s + i)
(foldl concat (PState s) (take i as))) = Some (as ! i)
using *outcome i-bound*
proof (*induction n arbitrary: as s t i*)
case 0
then show ?*case* **by** *simp*
next
case (*Suc n*)
from *staged-alpha-program-Suc-outcomeE[OF Suc.premis(1)]*
obtain *a as' s1 s2* **where**
challenge-out:
Some (a, s1) ∈ set-dist (execute receive-alpha-challenge s)
and *record-out:*
Some ((), s2) ∈ set-dist (execute (record-staged-message a) s1)
and *rest-out:*
Some (as', t) ∈
set-dist (execute (staged-alpha-program n) s2)
and *as-eq:* *as = a # as'*
.
have *challenge-lookup:*
fmlookup (HashMap s1)
(AlphaChallenge (PAlphaCounter s) (PState s)) = Some a
using *receive-alpha-challenge-outcome[OF challenge-out]* **by** *blast*
have *challenge-counter:*
PAlphaCounter s1 = Suc (PAlphaCounter s)
using *receive-alpha-challenge-counter-outcome[OF challenge-out]* **by** *simp*
have *record-eq:*
s2 = s1 ∪
PState := concat (PState s1) a,
PTranscript := PTranscript s1 @ [a])
by (*rule record-staged-message-outcome[OF record-out]*)
have *s2-fields:*
HashMap s2 = HashMap s1

```

PState s2 = concat (PState s) a
PAlphaCounter s2 = Suc (PAlphaCounter s)
unfolding record-eq
using receive-alpha-challenge-outcome[OF challenge-out]
      challenge-counter
by simp-all
have rest-ext:  $s2 \leq t$ 
proof -
  have hash-target-program {} n (staged-alpha-program n)
    by (rule hash-target-program-staged-alpha-program)
  then show ?thesis
    using rest-out
    unfolding hash-target-program-def hash-extension-preserving-def
    by blast
qed
show ?case
proof (cases i)
  case 0
  have lookup-s2:
    fmlookup (HashMap s2)
      (AlphaChallenge (PAlphaCounter s) (PState s)) = Some a
    using challenge-lookup unfolding s2-fields by simp
  have lookup-t:
    fmlookup (HashMap t)
      (AlphaChallenge (PAlphaCounter s) (PState s)) = Some a
    by (rule hash-extension-lookup[OF lookup-s2 rest-ext])
  then show ?thesis
    using 0 as-eq by simp
next
  case (Suc j)
  then have j-bound:  $j < n$ 
    using Suc.premis(2) by simp
  have rest-lookup:
    fmlookup (HashMap t)
      (AlphaChallenge (PAlphaCounter s2 + j)
        (foldl concat (PState s2) (take j as')))) = Some (as' ! j)
    by (rule Suc.IH[OF rest-out j-bound])
  show ?thesis
    using rest-lookup
    unfolding Suc as-eq s2-fields
    by simp
qed
qed

lemma staged-alpha-program-outcome-not-path-fresh-imp-prequeried:
assumes outcome:
  Some (as, t)  $\in$  set-dist (execute (staged-alpha-program n) s)
and not-fresh:  $\neg$  alpha-vector-path-fresh s as n
shows alpha-vector-prequeried-in-state s as n

```

```

proof –
  from not-fresh obtain i old where
    i-bound:  $i < n$ 
    and i-len:  $i < \text{length } as$ 
    and lookup-s:
       $\text{fmlookup } (\text{HashMap } s) (\text{alpha-challenge-key-at } s \ i \ as) = \text{Some } old$ 
    unfolding alpha-vector-path-fresh-def
    by (metis option.exhaust)
  have lookup-t:
     $\text{fmlookup } (\text{HashMap } t) (\text{alpha-challenge-key-at } s \ i \ as) =$ 
       $\text{Some } (as \ ! \ i)$ 
    using staged-alpha-program-outcome-lookup[OF outcome i-bound]
    unfolding alpha-challenge-key-at-def .
  have ext:  $s \leq t$ 
  proof –
    have hash-target-program  $\{ \} \ n$  (staged-alpha-program n)
      by (rule hash-target-program-staged-alpha-program)
    then show ?thesis
      using outcome
      unfolding hash-target-program-def hash-extension-preserving-def
      by blast
  qed
  have lookup-t-old:
     $\text{fmlookup } (\text{HashMap } t) (\text{alpha-challenge-key-at } s \ i \ as) = \text{Some } old$ 
    by (rule hash-extension-lookup[OF lookup-s ext])
  have old-eq:  $old = as \ ! \ i$ 
    using lookup-t lookup-t-old by simp
  show ?thesis
    unfolding alpha-vector-prequeried-in-state-def
    using i-bound i-len lookup-s old-eq by blast
  qed

lemma wp-staged-alpha-program-set-bound-from-path-fresh-or-prequery:
  assumes subset:  $B \subseteq \text{alpha-space}$ 
  shows
     $\text{wp-event } (\text{staged-alpha-program } (\text{length spec}))$ 
     $(\lambda out. \text{case out of None} \Rightarrow \text{False}$ 
       $\mid \text{Some } (as, -) \Rightarrow as \in B) \ s \leq$ 
     $\text{nnreal } (\text{card } B) / \text{nnreal } (\text{CARD('f)} \wedge \text{length spec}) +$ 
     $(\text{if } \exists as \in B.$ 
       $\text{alpha-vector-prequeried-in-state } s \ as \ (\text{length spec})$ 
       $\text{then } 1 \text{ else } 0)$ 
  proof (cases  $\exists as \in B.$ 
     $\text{alpha-vector-prequeried-in-state } s \ as \ (\text{length spec}))$ 
  case True
  have wp-event (staged-alpha-program (length spec))
     $(\lambda out. \text{case out of None} \Rightarrow \text{False}$ 
       $\mid \text{Some } (as, -) \Rightarrow as \in B) \ s \leq 1$ 
    by (rule wp-event-le-1)

```

```

also have ...  $\leq$ 
  nnreal (card B) / nnreal (CARD('f)  $\wedge$  length spec) + 1
by simp
finally show ?thesis
  using True by simp
next
case False
let ?BF =
  {as  $\in$  B. alpha-vector-path-fresh s as (length spec)}
have finite-B: finite B
  by (rule finite-subset[OF subset])
  (simp add: alpha-space-def finite-length-lists-UNIV)
have finite-BF: finite ?BF
  using finite-B by simp
have lengths:
   $\bigwedge as. as \in ?BF \implies \text{length } as = \text{length spec}$ 
  using subset unfolding alpha-space-def by auto
have fresh:
   $\bigwedge as. as \in ?BF \implies$ 
    alpha-vector-path-fresh s as (length spec)
  by simp
have event-mono:
  wp-event (staged-alpha-program (length spec))
    ( $\lambda out. \text{case out of None} \Rightarrow \text{False}$ 
      |  $\text{Some } (as, -) \Rightarrow as \in B$ ) s  $\leq$ 
  wp-event (staged-alpha-program (length spec))
    ( $\lambda out. \text{case out of None} \Rightarrow \text{False}$ 
      |  $\text{Some } (as, -) \Rightarrow as \in ?BF$ ) s
proof (rule wp-event-mono-on-support)
fix out
assume support:
  out  $\in$  set-dist
  (execute (staged-alpha-program (length spec)) s)
and hit:
  ( $\text{case out of None} \Rightarrow \text{False}$ 
    |  $\text{Some } (as, -) \Rightarrow as \in B$ )
show
  ( $\text{case out of None} \Rightarrow \text{False}$ 
    |  $\text{Some } (as, -) \Rightarrow as \in ?BF$ )
proof (cases out)
case None
  then show ?thesis using hit by simp
next
case (Some result)
  then obtain as t where out-eq: out = Some (as, t)
  by (cases result) simp
have as-in: as  $\in$  B
  using hit unfolding out-eq by simp
have outcome:

```

```

    Some (as, t) ∈
      set-dist (execute (staged-alpha-program (length spec)) s)
    using support unfolding out-eq .
  have path-fresh:
    alpha-vector-path-fresh s as (length spec)
  proof (rule ccontr)
    assume ¬ alpha-vector-path-fresh s as (length spec)
    then have alpha-vector-prequeried-in-state s as (length spec)
      by (rule
        staged-alpha-program-outcome-not-path-fresh-imp-prequeried
        [OF outcome])
    then show False
      using False as-in by blast
  qed
  show ?thesis
    unfolding out-eq using as-in path-fresh by simp
  qed
  also have ... ≤
    nnreal (card ?BF) / (nnreal size) ^ length spec
  by (rule wp-staged-alpha-program-finite-set-bound-from-path-fresh
    [OF finite-BF lengths fresh])
  also have ... ≤ nnreal (card B) / (nnreal size) ^ length spec
  proof -
    have card ?BF ≤ card B
      by (rule card-mono[OF finite-B]) blast
    then show ?thesis
      apply (subst nn2real-le-iff[symmetric])
      by (simp add: divide-right-mono)
  qed
  also have ... =
    nnreal (card B) / nnreal (CARD('f) ^ length spec)
  using size-card by simp
  finally show ?thesis
    using False by simp
  qed

lemma wp-staged-alpha-program-set-bound-excluding-prequeried:
  assumes subset: B ⊆ alpha-space
  shows
    wp-event (staged-alpha-program (length spec))
      (λout. case out of None ⇒ False
        | Some (as, -) ⇒
          as ∈ B ∧
            ¬ alpha-vector-prequeried-in-state s as (length spec))
      s ≤ nnreal (card B) / nnreal (CARD('f) ^ length spec)
  proof -
    let ?BF =
      {as ∈ B. alpha-vector-path-fresh s as (length spec)}

```

```

have finite-B: finite B
  by (rule finite-subset[OF subset])
    (simp add: alpha-space-def finite-length-lists-UNIV)
have finite-BF: finite ?BF
  using finite-B by simp
have lengths:
   $\bigwedge as. as \in ?BF \implies length\ as = length\ spec$ 
  using subset unfolding alpha-space-def by auto
have fresh:
   $\bigwedge as. as \in ?BF \implies$ 
    alpha-vector-path-fresh s as (length spec)
  by simp
have event-mono:
  wp-event (staged-alpha-program (length spec))
    ( $\lambda out. case\ out\ of\ None \Rightarrow False$ 
      | Some (as, -)  $\Rightarrow$ 
         $as \in B \wedge$ 
         $\neg\ alpha\text{-vector}\text{-prequeried-in-state}\ s\ as\ (length\ spec)$ )
     $s \leq$ 
    wp-event (staged-alpha-program (length spec))
      ( $\lambda out. case\ out\ of\ None \Rightarrow False$ 
        | Some (as, -)  $\Rightarrow as \in ?BF$ ) s
proof (rule wp-event-mono-on-support)
fix out
assume support:
  out  $\in set\text{-}dist$ 
  (execute (staged-alpha-program (length spec)) s)
and hit:
  (case out of None  $\Rightarrow False$ 
    | Some (as, -)  $\Rightarrow$ 
       $as \in B \wedge$ 
       $\neg\ alpha\text{-vector}\text{-prequeried-in-state}\ s\ as\ (length\ spec)$ )
show
  (case out of None  $\Rightarrow False$ 
    | Some (as, -)  $\Rightarrow as \in ?BF$ )
proof (cases out)
  case None
    then show ?thesis using hit by simp
next
  case (Some result)
  then obtain as t where out-eq: out = Some (as, t)
  by (cases result) simp
  have as-in:  $as \in B$ 
  and not-prequeried:
     $\neg\ alpha\text{-vector}\text{-prequeried-in-state}\ s\ as\ (length\ spec)$ 
  using hit unfolding out-eq by simp-all
  have outcome:
    Some (as, t)  $\in$ 
    set-dist (execute (staged-alpha-program (length spec)) s)

```

```

    using support unfolding out-eq .
  have path-fresh:
    alpha-vector-path-fresh s as (length spec)
  proof (rule ccontr)
    assume  $\neg$  alpha-vector-path-fresh s as (length spec)
    then have alpha-vector-prequeried-in-state s as (length spec)
      by (rule
        staged-alpha-program-outcome-not-path-fresh-imp-prequeried
        [OF outcome])
    then show False using not-prequeried by contradiction
  qed
  show ?thesis
    unfolding out-eq using as-in path-fresh by simp
  qed
  qed
  also have ...  $\leq$ 
    nnreal (card ?BF) / (nnreal size)  $\wedge$  length spec
  by (rule wp-staged-alpha-program-finite-set-bound-from-path-fresh
    [OF finite-BF lengths fresh])
  also have ...  $\leq$  nnreal (card B) / (nnreal size)  $\wedge$  length spec
  proof -
    have card ?BF  $\leq$  card B
      by (rule card-mono[OF finite-B]) blast
    then show ?thesis
      apply (subst nn2real-le-iff[symmetric])
      by (simp add: divide-right-mono)
  qed
  also have ... =
    nnreal (card B) / nnreal (CARD('f)  $\wedge$  length spec)
    using size-card by simp
  finally show ?thesis .
  qed

lemma staged-after-alpha-prefix-program-alpha-list-bound-from-path-fresh-or-prequery:
  assumes subset:  $B \subseteq \text{alpha-space}$ 
  shows
    wp-event
      (staged-after-alpha-prefix-program A
        (fr, trace-roots, trace-bs, trace-final))
      (staged-transcript-alpha-list-hit B) s  $\leq$ 
      nnreal (card B) / nnreal (CARD('f)  $\wedge$  length spec) +
      (if  $\exists as \in B$ .
        alpha-vector-prequeried-in-state s as (length spec)
        then 1 else 0)
  proof -
    have alpha-bound:
      wp-event (staged-alpha-program (length spec))
      ( $\lambda out. \text{case out of None} \Rightarrow \text{False}$ 
        |  $\text{Some } (as, -) \Rightarrow as \in B$ ) s  $\leq$ 

```



```

      nnreal (card B) / nnreal (CARD('f) ^ length spec) +
      (if  $\exists as \in B$ .
        alpha-vector-prequeried-in-state s as (length spec)
        then 1 else 0)
    by (rule wp-staged-alpha-program-set-bound-from-path-fresh-or-prequery
        [OF subset])
  show ?thesis
  by (rule staged-after-alpha-prefix-program-alpha-list-bound-from-alpha-program
      [OF alpha-bound])
qed

end

end

```

```

theory Staged-Security-Experiment-Composition
imports
  Staged-Security-Experiment-Checked
  Staged-Security-Experiment-Alpha-Prequery
begin

```

Prefix-fixed composition alpha targets for staged soundness.

```

context soundness
begin

```

```

definition checked-staged-after-alpha-prefix-program
  :: 'f staged-adversary  $\Rightarrow$ 
    ('f  $\times$  'f list  $\times$  'f list  $\times$  'f)  $\Rightarrow$ 
    ('f staged-proof-data, 'f protocol-channel) state-monad
where
  checked-staged-after-alpha-prefix-program A prefix =
    (case prefix of (fr, trace-roots, trace-bs, trace-final)  $\Rightarrow$ 
      do {
        as  $\leftarrow$  staged-alpha-program (length spec);
        dg  $\leftarrow$  degree-stage A as;
        record-staged-message dg;
        let composition-rounds = ceil-log (to-nat dg + 1);
        assert (composition-rounds  $\leq$  ceil-log (maxDegree + 1));
        (composition-roots, composition-bs)  $\leftarrow$ 
          staged-composition-fri-program A dg 0 composition-rounds [];
        composition-final  $\leftarrow$ 
          composition-final-stage A dg composition-bs;
        record-staged-message composition-final;
        query-chunks  $\leftarrow$ 
          checked-staged-query-program A trace-roots composition-roots
            0 rounds;
        return
          ( $\mid$ staged-trace-root = fr,

```

```

    staged-trace-fri-roots = trace-roots,
    staged-trace-fri-challenges = trace-bs,
    staged-trace-final = trace-final,
    staged-alphas = as,
    staged-degree = dg,
    staged-composition-fri-roots = composition-roots,
    staged-composition-fri-challenges = composition-bs,
    staged-composition-final = composition-final,
    staged-query-chunks = query-chunks)
  })

```

lemma *checked-staged-transcript-program-alpha-prefix-decomp:*

```

checked-staged-transcript-program A =
  staged-alpha-prefix-program A  $\gg$ 
  checked-staged-after-alpha-prefix-program A
unfolding checked-staged-transcript-program-def
  staged-alpha-prefix-program-def
  checked-staged-after-alpha-prefix-program-def Let-def
by (simp add: sm-bind-assoc split-def split: prod.splits)

```

lemma *sm-bind-get-ignore:*

```

(get  $\gg$  ( $\lambda\cdot. m$ )) = m
apply transfer
apply (rule ext)
apply (rule dist-inject[THEN iffD1])
unfolding dist-bind.rep-eq dist-get-def
apply (simp add: o-def dist-delta-dist)
apply (subst map-bind-delta-left-state[where  $f=\lambda s. \text{Some } (s, s)$ ])
apply (auto simp: bind-cont-map-def split: option.splits prod.splits)
done

```

lemma *checked-staged-after-alpha-prefix-program-alpha-list-bound-from-alpha-program:*

```

assumes alpha-bound:
  wp-event (staged-alpha-program (length spec))
  ( $\lambda out. \text{case out of None} \Rightarrow \text{False}$ 
    |  $\text{Some } (as, -) \Rightarrow as \in B$ )  $s \leq C$ 

```

shows

```

  wp-event (checked-staged-after-alpha-prefix-program A prefix)
  (staged-transcript-alpha-list-hit B)  $s \leq C$ 

```

proof –

obtain *fr trace-roots trace-bs trace-final where prefix-eq:*

```

  prefix = (fr, trace-roots, trace-bs, trace-final)
by (cases prefix) auto

```

let $?P =$

```

   $\lambda out. \text{case out of None} \Rightarrow \text{False}$ 
  |  $\text{Some } (as, -) \Rightarrow as \in B$ 

```

have *after-eq:*

```

checked-staged-after-alpha-prefix-program A prefix =
  staged-alpha-program (length spec)  $\gg$ 

```

```

( $\lambda$ as. degree-stage A as  $\gg$ 
( $\lambda$ dg. record-staged-message dg  $\gg$ 
( $\lambda$ -. let composition-rounds = ceil-log (to-nat dg + 1)
in assert
  (composition-rounds  $\leq$  ceil-log (maxDegree + 1))  $\gg$ 
( $\lambda$ -. staged-composition-fri-program A dg 0
  composition-rounds []  $\gg$ 
( $\lambda$ (composition-roots, composition-bs).
  composition-final-stage A dg composition-bs  $\gg$ 
( $\lambda$ composition-final.
  record-staged-message composition-final  $\gg$ 
( $\lambda$ -. checked-staged-query-program A trace-roots
  composition-roots 0 rounds  $\gg$ 
( $\lambda$ query-chunks.
  return
    ( $\lambda$ staged-trace-root = fr,
    staged-trace-fri-roots = trace-roots,
    staged-trace-fri-challenges = trace-bs,
    staged-trace-final = trace-final,
    staged-alphas = as,
    staged-degree = dg,
    staged-composition-fri-roots =
      composition-roots,
    staged-composition-fri-challenges =
      composition-bs,
    staged-composition-final = composition-final,
    staged-query-chunks =
      query-chunks)))))))))
unfolding checked-staged-after-alpha-prefix-program-def prefix-eq Let-def
by simp
show ?thesis
unfolding after-eq staged-transcript-alpha-list-hit-def
proof (rule wp-event-bind-bound-by-head-event[OF alpha-bound])
show (case None of None  $\Rightarrow$  False
  | Some (data, -)  $\Rightarrow$  staged-alphas data  $\in$  B)  $\Rightarrow$ 
  ?P None
by simp
next
fix as t out
assume cont:
  out  $\in$ 
  set-dist
  (execute
    (degree-stage A as  $\gg$ 
    ( $\lambda$ dg. record-staged-message dg  $\gg$ 
    ( $\lambda$ -. let composition-rounds =
      ceil-log (to-nat dg + 1)
    in assert
      (composition-rounds  $\leq$  ceil-log (maxDegree + 1))  $\gg$ 

```

```

(λ-. staged-composition-fri-program A dg 0
  composition-rounds [] >>=
  (λ(composition-roots, composition-bs).
    composition-final-stage A dg composition-bs >>=
    (λcomposition-final.
      record-staged-message composition-final >>=
      (λ-. checked-staged-query-program A trace-roots
        composition-roots 0 rounds >>=
        (λquery-chunks.
          return
            (|staged-trace-root = fr,
              staged-trace-fri-roots = trace-roots,
              staged-trace-fri-challenges = trace-bs,
              staged-trace-final = trace-final,
              staged-alphas = as,
              staged-degree = dg,
              staged-composition-fri-roots =
                composition-roots,
              staged-composition-fri-challenges =
                composition-bs,
              staged-composition-final = composition-final,
              staged-query-chunks =
                query-chunks|)))))))))
  t)
and hit:
  (case out of None ⇒ False
    | Some (data, -) ⇒ staged-alphas data ∈ B)
show ?P (Some (as, t))
proof (cases out)
  case None
  then show ?thesis using hit by simp
next
  case (Some result)
  then obtain data u where out-eq: out = Some (data, u)
  by (cases result) simp
have data-as: staged-alphas data = as
  using cont unfolding out-eq Let-def
  by (auto elim!: set-dist-bindE split: prod.splits)
have staged-alphas data ∈ B
  using hit unfolding out-eq by simp
then show ?thesis
  using data-as by simp
qed
qed
qed

lemma checked-staged-after-alpha-prefix-program-alpha-list-bound-from-path-fresh-or-prequery:
  assumes subset: B ⊆ alpha-space
  shows

```

```

wp-event (checked-staged-after-alpha-prefix-program A prefix)
  (staged-transcript-alpha-list-hit B) s ≤
  nnreal (card B) / nnreal (CARD('f) ^ length spec) +
  (if ∃ as ∈ B.
    alpha-vector-prequeried-in-state s as (length spec)
  then 1 else 0)
proof –
  have alpha-bound:
    wp-event (staged-alpha-program (length spec))
      (λout. case out of None ⇒ False
        | Some (as, -) ⇒ as ∈ B) s ≤
      nnreal (card B) / nnreal (CARD('f) ^ length spec) +
      (if ∃ as ∈ B.
        alpha-vector-prequeried-in-state s as (length spec)
      then 1 else 0)
  by (rule wp-staged-alpha-program-set-bound-from-path-fresh-or-prequery
    [OF subset])
  show ?thesis
  by (rule
    checked-staged-after-alpha-prefix-program-alpha-list-bound-from-alpha-program
    [OF alpha-bound])
qed

lemma checked-staged-after-alpha-prefix-program-alpha-list-bound-excluding-prequeried:
  assumes subset: B ⊆ alpha-space
  shows
    wp-event (checked-staged-after-alpha-prefix-program A prefix)
      (λout. case out of None ⇒ False
        | Some (data, -) ⇒
          staged-alphas data ∈ B ∧
          ¬ alpha-vector-prequeried-in-state s
            (staged-alphas data) (length spec))
      s ≤ nnreal (card B) / nnreal (CARD('f) ^ length spec)
proof –
  obtain fr trace-roots trace-bs trace-final where prefix-eq:
    prefix = (fr, trace-roots, trace-bs, trace-final)
  by (cases prefix) auto
  let ?P =
    λout. case out of None ⇒ False
      | Some (as, -) ⇒
        as ∈ B ∧
        ¬ alpha-vector-prequeried-in-state s as (length spec)
  have alpha-bound:
    wp-event (staged-alpha-program (length spec))
      ?P s ≤ nnreal (card B) / nnreal (CARD('f) ^ length spec)
  by (rule wp-staged-alpha-program-set-bound-excluding-prequeried
    [OF subset])
  have after-eq:
    checked-staged-after-alpha-prefix-program A prefix =

```

```

staged-alpha-program (length spec)  $\gg=$ 
  ( $\lambda$ as. degree-stage A as  $\gg=$ 
    ( $\lambda$ dg. record-staged-message dg  $\gg=$ 
      ( $\lambda$ -. let composition-rounds = ceil-log (to-nat dg + 1)
        in assert
          (composition-rounds  $\leq$  ceil-log (maxDegree + 1))  $\gg=$ 
            ( $\lambda$ -. staged-composition-fri-program A dg 0
              composition-rounds []  $\gg=$ 
                ( $\lambda$ (composition-roots, composition-bs).
                  composition-final-stage A dg composition-bs  $\gg=$ 
                    ( $\lambda$ composition-final.
                      record-staged-message composition-final  $\gg=$ 
                        ( $\lambda$ -. checked-staged-query-program A trace-roots
                          composition-roots 0 rounds  $\gg=$ 
                            ( $\lambda$ query-chunks.
                              return
                                ( $\lambda$ (staged-trace-root = fr,
                                  staged-trace-fri-roots = trace-roots,
                                  staged-trace-fri-challenges = trace-bs,
                                  staged-trace-final = trace-final,
                                  staged-alphas = as,
                                  staged-degree = dg,
                                  staged-composition-fri-roots =
                                    composition-roots,
                                  staged-composition-fri-challenges =
                                    composition-bs,
                                  staged-composition-final = composition-final,
                                  staged-query-chunks =
                                    query-chunks))))))))))
          unfolding checked-staged-after-alpha-prefix-program-def prefix-eq Let-def
          by simp
        show ?thesis
          unfolding after-eq
        proof (rule wp-event-bind-bound-by-head-event[OF alpha-bound])
          show (case None of None  $\Rightarrow$  False
            | Some (data, -)  $\Rightarrow$ 
              staged-alphas data  $\in B \wedge$ 
               $\neg$  alpha-vector-prequeried-in-state s
              (staged-alphas data) (length spec)  $\Rightarrow$ 
                ?P None
              by simp
            next
            fix as t out
            assume cont:
              out  $\in$ 
                set-dist
                (execute
                  (degree-stage A as  $\gg=$ 
                    ( $\lambda$ dg. record-staged-message dg  $\gg=$ 

```

```

(λ-. let composition-rounds =
      ceil-log (to-nat dg + 1)
    in assert
      (composition-rounds ≤ ceil-log (maxDegree + 1)) >>=
      (λ-. staged-composition-fri-program A dg 0
        composition-rounds [] >>=
        (λ(composition-roots, composition-bs).
          composition-final-stage A dg composition-bs >>=
          (λcomposition-final.
            record-staged-message composition-final >>=
            (λ-. checked-staged-query-program A trace-roots
              composition-roots 0 rounds >>=
              (λquery-chunks.
                return
                  (|staged-trace-root = fr,
                    staged-trace-fri-roots = trace-roots,
                    staged-trace-fri-challenges = trace-bs,
                    staged-trace-final = trace-final,
                    staged-alphas = as,
                    staged-degree = dg,
                    staged-composition-fri-roots =
                      composition-roots,
                    staged-composition-fri-challenges =
                      composition-bs,
                    staged-composition-final = composition-final,
                    staged-query-chunks =
                      query-chunks|)))))))))
t)
and hit:
  (case out of None ⇒ False
   | Some (data, -) ⇒
     staged-alphas data ∈ B ∧
     ¬ alpha-vector-prequeried-in-state s
     (staged-alphas data) (length spec))
show ?P (Some (as, t))
proof (cases out)
  case None
  then show ?thesis using hit by simp
next
  case (Some result)
  then obtain data u where out-eq: out = Some (data, u)
  by (cases result) simp
have data-as: staged-alphas data = as
  using cont unfolding out-eq Let-def
  by (auto elim!: set-dist-bindE split: prod.splits)
have staged-alphas data ∈ B
  ¬ alpha-vector-prequeried-in-state s
  (staged-alphas data) (length spec)
  using hit unfolding out-eq by simp-all

```

```

    then show ?thesis
    using data-as by simp
qed
qed
qed

```

definition *checked-staged-transcript-with-alpha-prefix-program*

```

:: 'f staged-adversary  $\Rightarrow$ 
  (((('f  $\times$  'f list  $\times$  'f list  $\times$  'f)  $\times$ 
    'f protocol-channel)  $\times$  'f staged-proof-data,
    'f protocol-channel) state-monad

```

where

```

checked-staged-transcript-with-alpha-prefix-program A =
  staged-alpha-prefix-program A  $\gg=$ 
    ( $\lambda$ prefix. get  $\gg=$ 
      ( $\lambda$ prefix-state.
        checked-staged-after-alpha-prefix-program A prefix  $\gg=$ 
          ( $\lambda$ data. return ((prefix, prefix-state), data))))

```

definition *checked-staged-security-experiment-with-actual-alpha-prefix-data-state*

```

:: 'f staged-adversary  $\Rightarrow$ 
  ((((((('f  $\times$  'f list  $\times$  'f list  $\times$  'f)  $\times$ 
    'f protocol-channel)  $\times$  'f staged-proof-data)  $\times$ 
    'f protocol-channel)  $\times$  unit list,
    'f protocol-channel) state-monad

```

where

```

checked-staged-security-experiment-with-actual-alpha-prefix-data-state A =
  checked-staged-transcript-with-alpha-prefix-program A  $\gg=$ 
    ( $\lambda$ packed. get  $\gg=$ 
      ( $\lambda$ s.
        put
          (verifier-state-from-adversary s
            (staged-proof-transcript (snd packed)))  $\gg=$ 
            ( $\lambda$ -.
              verify-monad  $\gg=$ 
                ( $\lambda$ result. return ((packed, s), result))))

```

lemma *checked-staged-transcript-with-alpha-prefix-program-projection:*

```

checked-staged-transcript-with-alpha-prefix-program A  $\gg=$ 
  ( $\lambda$ packed. return (snd packed)) =
  checked-staged-transcript-program A

```

unfolding *checked-staged-transcript-with-alpha-prefix-program-def*

```

checked-staged-transcript-program-alpha-prefix-decomp
by (simp add: sm-bind-assoc sm-bind-get-ignore)

```

lemma *checked-staged-security-experiment-with-actual-alpha-prefix-data-state-projection:*

```

checked-staged-security-experiment-with-actual-alpha-prefix-data-state A  $\gg=$ 
  ( $\lambda$ x. return ((snd (fst (fst x)), snd (fst x)), snd x)) =
  checked-staged-security-experiment-with-data-state A

```



```

proof –
  let ?K =
    λdata. get >>=
      (λs. put
        (verifier-state-from-adversary s
          (staged-proof-transcript data)) >>=
        (λ-. verify-monad >>=
          (λresult. return ((data, s), result))))
  have
    checked-staged-security-experiment-with-actual-alpha-prefix-data-state A >>=
      (λx. return ((snd (fst (fst x)), snd (fst x)), snd x)) =
    checked-staged-transcript-with-alpha-prefix-program A >>=
      (λpacked. ?K (snd packed))
  unfolding
    checked-staged-security-experiment-with-actual-alpha-prefix-data-state-def
  by (simp add: sm-bind-assoc split-def)
  also have ... =
    (checked-staged-transcript-with-alpha-prefix-program A >>=
      (λpacked. return (snd packed))) >>= ?K
  by (simp add: sm-bind-assoc)
  also have ... =
    checked-staged-transcript-program A >>= ?K
  unfolding checked-staged-transcript-with-alpha-prefix-program-projection
  by simp
  also have ... =
    checked-staged-security-experiment-with-data-state A
  unfolding checked-staged-security-experiment-with-data-state-def by simp
  finally show ?thesis .
qed

```

```

lemma alpha-prefix-hash-extends-verifier-state-from-adversary-right:
  assumes ext:  $s \leq t$ 
  shows  $s \leq \text{verifier-state-from-adversary } t \text{ tr}$ 
  using ext
  unfolding verifier-state-from-adversary-def less-eq-hash-ext-def
    less-eq-fmap-def
  by simp

```

```

definition alpha-prefix-trace-table-candidates
  :: 'f protocol-channel  $\Rightarrow$  'f  $\Rightarrow$  'f list set
  where
    alpha-prefix-trace-table-candidates prefix-state fr =
      {trace-table.
        length trace-table = clength * scale  $\wedge$ 
        merkle-root-binds-table fr trace-table prefix-state}

```

```

lemma alpha-prefix-trace-table-candidateI:
  assumes len: length trace-table = clength * scale
  and bind: merkle-root-binds-table fr trace-table prefix-state

```

shows $\text{trace-table} \in \text{alpha-prefix-trace-table-candidates prefix-state fr}$
using *assms* **unfolding** *alpha-prefix-trace-table-candidates-def* **by** *simp*

definition *alpha-prefix-union-bad-sets*

$:: 'f \text{ protocol-channel} \Rightarrow ('f \text{ list} \Rightarrow 'f \text{ list set}) \Rightarrow$
 $'f \Rightarrow 'f \text{ list set}$

where

$\text{alpha-prefix-union-bad-sets prefix-state bad-sets fr} =$
 $\{as \in \text{alpha-space}.$
 $\exists \text{ trace-table} \in$
 $\text{alpha-prefix-trace-table-candidates prefix-state fr}.$
 $as \in \text{bad-sets trace-table}\}$

definition *checked-staged-transcript-actual-alpha-prefix-bad-set-hit*

$:: ('f \text{ list} \Rightarrow 'f \text{ list set}) \Rightarrow$
 $((('f \times 'f \text{ list} \times 'f \text{ list} \times 'f) \times$
 $'f \text{ protocol-channel}) \times 'f \text{ staged-proof-data}) \times$
 $'f \text{ protocol-channel}) \text{ option} \Rightarrow \text{bool}$

where

$\text{checked-staged-transcript-actual-alpha-prefix-bad-set-hit bad-sets out}$
 $\longleftrightarrow$
 $(\text{case out of}$
 $\text{None} \Rightarrow \text{False}$
 $| \text{Some } (((\text{fr}, -, -, -), \text{prefix-state}), \text{data}), -) \Rightarrow$
 $\text{staged-alphas data} \in$
 $\text{alpha-prefix-union-bad-sets prefix-state bad-sets fr})$

definition *checked-staged-transcript-actual-alpha-prefix-prequery-hit*

$:: ('f \text{ list} \Rightarrow 'f \text{ list set}) \Rightarrow$
 $((('f \times 'f \text{ list} \times 'f \text{ list} \times 'f) \times$
 $'f \text{ protocol-channel}) \times 'f \text{ staged-proof-data}) \times$
 $'f \text{ protocol-channel}) \text{ option} \Rightarrow \text{bool}$

where

$\text{checked-staged-transcript-actual-alpha-prefix-prequery-hit bad-sets out}$
 $\longleftrightarrow$
 $(\text{case out of}$
 $\text{None} \Rightarrow \text{False}$
 $| \text{Some } (((\text{fr}, -, -, -), \text{prefix-state}), \text{data}), -) \Rightarrow$
 $\text{staged-alphas data} \in$
 $\text{alpha-prefix-union-bad-sets prefix-state bad-sets fr} \wedge$
 $\text{alpha-vector-prequeried-in-state prefix-state}$
 $(\text{staged-alphas data}) (\text{length spec}))$

lemma *checked-staged-transcript-actual-alpha-prefix-prequery-hit-imp-bad-set-hit:*

assumes

$\text{checked-staged-transcript-actual-alpha-prefix-prequery-hit bad-sets out}$

shows $\text{checked-staged-transcript-actual-alpha-prefix-bad-set-hit bad-sets out}$

using *assms*

unfolding *checked-staged-transcript-actual-alpha-prefix-prequery-hit-def*

checked-staged-transcript-actual-alpha-prefix-bad-set-hit-def
by (*auto split: option.splits prod.splits*)

definition *checked-staged-transcript-actual-alpha-prefix-clean-fresh-bad-set-hit*
 $:: ('f \text{ list} \Rightarrow 'f \text{ list set}) \Rightarrow$
 $((('f \times 'f \text{ list} \times 'f \text{ list} \times 'f) \times$
 $'f \text{ protocol-channel}) \times 'f \text{ staged-proof-data}) \times$
 $'f \text{ protocol-channel}) \text{ option} \Rightarrow \text{bool}$

where

checked-staged-transcript-actual-alpha-prefix-clean-fresh-bad-set-hit
bad-sets out $\longleftrightarrow$
(case out of
 $\text{None} \Rightarrow \text{False}$
 $| \text{Some } (((\text{prefix}, \text{prefix-state}), \text{data}), -) \Rightarrow$
 $\neg \text{hash-map-output-collision } \text{prefix-state} \wedge$
 $\text{staged-alphas } \text{data} \in$
 $\text{alpha-prefix-union-bad-sets } \text{prefix-state } \text{bad-sets } (\text{fst } \text{prefix}) \wedge$
 $\neg \text{alpha-vector-prequeried-in-state } \text{prefix-state}$
 $(\text{staged-alphas } \text{data}) (\text{length spec}))$

lemma *checked-staged-transcript-actual-alpha-prefix-bad-set-hit-split:*

assumes

checked-staged-transcript-actual-alpha-prefix-bad-set-hit bad-sets out

shows

checked-staged-transcript-actual-alpha-prefix-clean-fresh-bad-set-hit
bad-sets out $\vee$
checked-staged-transcript-actual-alpha-prefix-prequery-hit bad-sets out $\vee$
(case out of
 $\text{None} \Rightarrow \text{False}$
 $| \text{Some } (((-, \text{prefix-state}), -), -) \Rightarrow$
 $\text{hash-map-output-collision } \text{prefix-state}))$

using *assms*

unfolding *checked-staged-transcript-actual-alpha-prefix-bad-set-hit-def*
checked-staged-transcript-actual-alpha-prefix-clean-fresh-bad-set-hit-def
checked-staged-transcript-actual-alpha-prefix-prequery-hit-def
by (*auto split: option.splits prod.splits*)

lemma *alpha-prefix-union-bad-sets-subset-alpha-space:*

alpha-prefix-union-bad-sets prefix-state bad-sets fr $\subseteq$ *alpha-space*

unfolding *alpha-prefix-union-bad-sets-def* **by** *auto*

lemma *alpha-prefix-union-bad-sets-finite:*

finite (alpha-prefix-union-bad-sets prefix-state bad-sets fr)

by (*rule finite-subset[OF alpha-prefix-union-bad-sets-subset-alpha-space*
finite-alpha-space])

lemma *alpha-prefix-union-bad-sets-fraction-bound-if-unique-candidate:*

fixes *C* $::$ *prob*

assumes *unique:*

```

     $\exists$  trace-table.
      alpha-prefix-trace-table-candidates prefix-state fr  $\subseteq$ 
        {trace-table}
    and subset:  $\bigwedge$  trace-table. bad-sets trace-table  $\subseteq$  alpha-space
    and bound:
       $\bigwedge$  trace-table.
        nnreal (card (bad-sets trace-table)) / nnreal (card alpha-space)  $\leq C$ 
  shows
    nnreal (card (alpha-prefix-union-bad-sets prefix-state bad-sets fr)) /
      nnreal (card alpha-space)  $\leq C$ 
  proof -
    from unique obtain trace-table where candidates:
      alpha-prefix-trace-table-candidates prefix-state fr  $\subseteq$ 
        {trace-table}
    by blast
    have union-subset:
      alpha-prefix-union-bad-sets prefix-state bad-sets fr  $\subseteq$ 
        bad-sets trace-table
    unfolding alpha-prefix-union-bad-sets-def
    using candidates by auto
    have finite-bad: finite (bad-sets trace-table)
    by (rule finite-subset[OF subset finite-alpha-space])
    have card-le:
      card (alpha-prefix-union-bad-sets prefix-state bad-sets fr)  $\leq$ 
        card (bad-sets trace-table)
    by (rule card-mono[OF finite-bad union-subset])
    have nnreal (card (alpha-prefix-union-bad-sets prefix-state bad-sets fr)) /
      nnreal (card alpha-space)  $\leq$ 
      nnreal (card (bad-sets trace-table)) / nnreal (card alpha-space)
    by (rule nnreal-nat-divide-right-mono[OF card-le])
    also have ...  $\leq C$ 
    by (rule bound)
    finally show ?thesis .
  qed

```

```

lemma alpha-prefix-trace-table-candidates-unique-if-no-hash-collision:
  assumes clean:  $\neg$  hash-map-output-collision prefix-state
  shows
     $\exists$  trace-table.
      alpha-prefix-trace-table-candidates prefix-state fr  $\subseteq$ 
        {trace-table}
  proof (cases alpha-prefix-trace-table-candidates prefix-state fr = {})
    case True
    then show ?thesis
    by blast
  next
    case False
    then obtain trace-table where table-in:
      trace-table  $\in$  alpha-prefix-trace-table-candidates prefix-state fr

```

```

    by blast
  have table-len: length trace-table = clength * scale
    using table-in unfolding alpha-prefix-trace-table-candidates-def by simp
  have table-bind: merkle-root-binds-table fr trace-table prefix-state
    using table-in unfolding alpha-prefix-trace-table-candidates-def by simp
  have subset-single:
    alpha-prefix-trace-table-candidates prefix-state fr  $\subseteq$ 
      {trace-table}
proof
  fix trace-table'
  assume table'-in:
    trace-table'  $\in$  alpha-prefix-trace-table-candidates prefix-state fr
  have table'-len: length trace-table' = clength * scale
    using table'-in unfolding alpha-prefix-trace-table-candidates-def
    by simp
  have table'-bind:
    merkle-root-binds-table fr trace-table' prefix-state
    using table'-in unfolding alpha-prefix-trace-table-candidates-def
    by simp
  have trace-table' = trace-table
    by (rule merkle-root-binds-same-length-tables-unique-if-no-hash-collision
      [OF table'-bind table-bind])
      (use table'-len table-len clean in simp-all)
  then show trace-table'  $\in$  {trace-table}
    by simp
qed
then show ?thesis by blast
qed

```

lemma *composition-alpha-prefix-union-fraction-bound-if-no-hash-collision:*
assumes clean: $\neg$ hash-map-output-collision prefix-state
shows

```

  nnreal
    (card
      (alpha-prefix-union-bad-sets prefix-state
        composition-trace-bad-alpha-space fr)) /
  nnreal (card alpha-space)  $\leq$  composition-error-bound
by (rule alpha-prefix-union-bad-sets-fraction-bound-if-unique-candidate)
(rule alpha-prefix-trace-table-candidates-unique-if-no-hash-collision
  [OF clean],
  rule composition-trace-bad-alpha-space-subset-alpha-space,
  rule composition-trace-bad-alpha-space-fraction-bound-alpha-space)

```

lemma *alpha-prefix-union-bad-sets-card-bound-if-unique-candidate:*
assumes unique:
 $\exists$ trace-table.
 alpha-prefix-trace-table-candidates prefix-state fr $\subseteq$
 {trace-table}
and subset: $\bigwedge$ trace-table. bad-sets trace-table $\subseteq$ alpha-space

and *bound*:
 $\bigwedge \text{trace-table}. \text{card } (\text{bad-sets trace-table}) \leq C$
shows
 $\text{card } (\text{alpha-prefix-union-bad-sets prefix-state bad-sets fr}) \leq C$
proof –
from *unique* **obtain** *trace-table* **where** *candidates*:
 $\text{alpha-prefix-trace-table-candidates prefix-state fr} \subseteq \{\text{trace-table}\}$
by *blast*
have *union-subset*:
 $\text{alpha-prefix-union-bad-sets prefix-state bad-sets fr} \subseteq \text{bad-sets trace-table}$
unfolding *alpha-prefix-union-bad-sets-def*
using *candidates* **by** *auto*
have *finite-bad*: $\text{finite } (\text{bad-sets trace-table})$
by (rule *finite-subset*[*OF subset finite-alpha-space*])
have $\text{card } (\text{alpha-prefix-union-bad-sets prefix-state bad-sets fr}) \leq \text{card } (\text{bad-sets trace-table})$
by (rule *card-mono*[*OF finite-bad union-subset*])
also have $\dots \leq C$
by (rule *bound*)
finally show *?thesis* .
qed

lemma *composition-alpha-prefix-union-card-bound-if-no-hash-collision*:
assumes *clean*: $\neg \text{hash-map-output-collision prefix-state}$
shows
 $\text{card } (\text{alpha-prefix-union-bad-sets prefix-state composition-trace-bad-alpha-space fr}) \leq (\text{if length spec} = 0 \text{ then } 0 \text{ else } \text{CARD}('f) ^ (\text{length spec} - 1))$
by (rule *alpha-prefix-union-bad-sets-card-bound-if-unique-candidate*)
(rule *alpha-prefix-trace-table-candidates-unique-if-no-hash-collision* [*OF clean*],
rule *composition-trace-bad-alpha-space-subset-alpha-space*,
rule *composition-trace-bad-alpha-space-card-bound*)

lemma *checked-staged-transcript-with-alpha-prefix-clean-fresh-composition-bound*:
shows
 $\text{wp-event } (\text{checked-staged-transcript-with-alpha-prefix-program } A)$
 $(\text{checked-staged-transcript-actual-alpha-prefix-clean-fresh-bad-set-hit composition-trace-bad-alpha-space})$
 $\text{adversary-initial-state} \leq \text{composition-error-bound}$
unfolding *checked-staged-transcript-with-alpha-prefix-program-def*
proof (rule *wp-event-bind-bound-by-cont*)
show
 $\neg \text{checked-staged-transcript-actual-alpha-prefix-clean-fresh-bad-set-hit composition-trace-bad-alpha-space None}$
unfolding

```

    checked-staged-transcript-actual-alpha-prefix-clean-fresh-bad-set-hit-def
  by simp
next
  fix prefix prefix-state
  assume prefix-out:
    Some (prefix, prefix-state) ∈
      set-dist
        (execute (staged-alpha-prefix-program A) adversary-initial-state)
  obtain fr trace-roots trace-bs trace-final where prefix-eq:
    prefix = (fr, trace-roots, trace-bs, trace-final)
  by (cases prefix) auto
  let ?B =
    alpha-prefix-union-bad-sets prefix-state
    composition-trace-bad-alpha-space fr
  let ?Q =
    checked-staged-transcript-actual-alpha-prefix-clean-fresh-bad-set-hit
    composition-trace-bad-alpha-space
  let ?E =
    λout. case out of
      None ⇒ False
    | Some (data, -) ⇒
      ¬ hash-map-output-collision prefix-state ∧
      staged-alphas data ∈ ?B ∧
      ¬ alpha-vector-prequeried-in-state prefix-state
      (staged-alphas data) (length spec)
  have get-eq:
    wp-event
      (get >>=
        (λprefix-state'.
          checked-staged-after-alpha-prefix-program A prefix >>=
            (λdata. return ((prefix, prefix-state'), data))))
    ?Q prefix-state =
    wp-event
      (checked-staged-after-alpha-prefix-program A prefix >>=
        (λdata. return ((prefix, prefix-state), data)))
    ?Q
    prefix-state
  unfolding wp-event-def by (simp add: wpsimps)
  have return-eq:
    wp-event
      (checked-staged-after-alpha-prefix-program A prefix >>=
        (λdata. return ((prefix, prefix-state), data)))
    ?Q prefix-state =
    wp-event (checked-staged-after-alpha-prefix-program A prefix) ?E
    prefix-state
  apply (subst wp-event-bind-return-map)
  unfolding
    checked-staged-transcript-actual-alpha-prefix-clean-fresh-bad-set-hit-def
    prefix-eq

```

```

  by (simp split: option.splits prod.splits)
have cont-eq:
  wp-event
  (get  $\gg$ 
    ( $\lambda$ prefix-state'.
      checked-staged-after-alpha-prefix-program A prefix  $\gg$ 
      ( $\lambda$ data. return ((prefix, prefix-state'), data))))
  ?Q prefix-state =
  wp-event (checked-staged-after-alpha-prefix-program A prefix) ?E
  prefix-state
using get-eq return-eq by simp
show
  wp-event
  (get  $\gg$ 
    ( $\lambda$ prefix-state.
      checked-staged-after-alpha-prefix-program A prefix  $\gg$ 
      ( $\lambda$ data. return ((prefix, prefix-state), data))))
  ?Q prefix-state  $\leq$  composition-error-bound
proof (cases hash-map-output-collision prefix-state)
case True
have zero:
  wp-event (checked-staged-after-alpha-prefix-program A prefix) ?E
  prefix-state = 0
proof -
  have indicator-zero:
    ( $\lambda x$ . if ?E x then 1 else 0 :: prob) = ( $\lambda$ -. 0)
  using True
  by (auto simp: fun-eq-iff split: option.splits prod.splits)
show ?thesis
  unfolding wp-event-def indicator-zero wp-def by simp
qed
show ?thesis
  unfolding cont-eq zero by simp
next
case False
have subset: ?B  $\subseteq$  alpha-space
  by (rule alpha-prefix-union-bad-sets-subset-alpha-space)
have suffix-bound:
  wp-event (checked-staged-after-alpha-prefix-program A prefix)
  ( $\lambda$ out. case out of None  $\Rightarrow$  False
    | Some (data, -)  $\Rightarrow$ 
      staged-alphas data  $\in$  ?B  $\wedge$ 
       $\neg$  alpha-vector-prequeried-in-state prefix-state
      (staged-alphas data) (length spec))
  prefix-state  $\leq$ 
  nnreal (card ?B) / nnreal (CARD('f)  $\wedge$  length spec)
by (rule
  checked-staged-after-alpha-prefix-program-alpha-list-bound-excluding-prequeried
  [OF subset])

```



```

have clean-bound:
  nnreal (card ?B) / nnreal (CARD('f) ^ length spec) ≤
    composition-error-bound
using composition-alpha-prefix-union-fraction-bound-if-no-hash-collision
  [OF False, of fr]
by (simp add: card-alpha-space)
have event-eq:
  ?E =
    (λout. case out of None ⇒ False
      | Some (data, -) ⇒
        staged-alphas data ∈ ?B ∧
        ¬ alpha-vector-prequeried-in-state prefix-state
          (staged-alphas data) (length spec))
using False
by (auto simp: fun-eq-iff split: option.splits prod.splits)
show ?thesis
unfolding cont-eq event-eq
by (rule order-trans[OF suffix-bound clean-bound])
qed
qed

lemma alpha-prefix-composition-vector-key-relation-fiber-bound-if-no-hash-collision:
assumes clean: ¬ hash-map-output-collision prefix-state
shows
  card
    {y. alpha-vector-key-relation prefix-state
      (alpha-prefix-union-bad-sets prefix-state
        composition-trace-bad-alpha-space fr)
      (length spec) x y}
    ≤ (if length spec = 0 then 0 else CARD('f) ^ (length spec - 1)) *
      length spec
proof -
have finite-bad:
  finite
    (alpha-prefix-union-bad-sets prefix-state
      composition-trace-bad-alpha-space fr)
by (rule alpha-prefix-union-bad-sets-finite)
have relation-bound:
  card
    {y. alpha-vector-key-relation prefix-state
      (alpha-prefix-union-bad-sets prefix-state
        composition-trace-bad-alpha-space fr)
      (length spec) x y}
    ≤
  card
    (alpha-prefix-union-bad-sets prefix-state
      composition-trace-bad-alpha-space fr) *
    length spec
by (rule alpha-vector-key-relation-fiber-card-bound[OF finite-bad])

```

```

also have ... ≤
  (if length spec = 0 then 0 else CARD('f) ^ (length spec - 1)) *
    length spec
proof (cases length spec = 0)
  case True
  then show ?thesis by simp
next
  case False
  have card
    (alpha-prefix-union-bad-sets prefix-state
     composition-trace-bad-alpha-space fr)
    ≤ CARD('f) ^ (length spec - 1)
  using composition-alpha-prefix-union-card-bound-if-no-hash-collision
    [OF clean] False
  by simp
  then show ?thesis
  using False
  by (intro mult-right-mono) simp-all
qed
finally show ?thesis .
qed

```

definition *checked-staged-security-with-data-state-alpha-prefix-bad-set-hit*
 $:: 'f \text{ staged-adversary} \Rightarrow ('f \text{ list} \Rightarrow 'f \text{ list set}) \Rightarrow$
 $((('f \text{ staged-proof-data} \times 'f \text{ protocol-channel}) \times \text{unit list}) \times$
 $'f \text{ protocol-channel}) \text{ option} \Rightarrow \text{bool}$

where

```

checked-staged-security-with-data-state-alpha-prefix-bad-set-hit A
  bad-sets out  $\longleftrightarrow$ 
  (case out of
    None  $\Rightarrow$  False
  | Some (((data, -), -), -)  $\Rightarrow$ 
    ( $\exists$  prefix prefix-state.
      Some (prefix, prefix-state)  $\in$ 
        set-dist
          (execute (staged-alpha-prefix-program A)
            adversary-initial-state)  $\wedge$ 
            staged-trace-root data = fst prefix  $\wedge$ 
            staged-trace-fri-roots data = fst (snd prefix)  $\wedge$ 
            staged-trace-fri-challenges data = fst (snd (snd prefix))  $\wedge$ 
            staged-trace-final data = snd (snd (snd prefix))  $\wedge$ 
            staged-alphas data  $\in$ 
              alpha-prefix-union-bad-sets prefix-state bad-sets
                (staged-trace-root data))))

```

definition *checked-staged-transcript-alpha-prefix-prequery-hit*

```

 $:: 'f \text{ staged-adversary} \Rightarrow ('f \text{ list} \Rightarrow 'f \text{ list set}) \Rightarrow$   

 $('f \text{ staged-proof-data} \times 'f \text{ protocol-channel}) \text{ option} \Rightarrow \text{bool}$ 

```

where

checked-staged-transcript-alpha-prefix-prequery-hit A bad-sets out
 $\longleftrightarrow$
 (case out of
 None $\Rightarrow$ False
 | Some (data, -) $\Rightarrow$
 ($\exists$ prefix prefix-state.
 Some (prefix, prefix-state) $\in$
 set-dist
 (execute (staged-alpha-prefix-program A)
 adversary-initial-state) $\wedge$
 staged-trace-root data = fst prefix $\wedge$
 staged-trace-fri-roots data = fst (snd prefix) $\wedge$
 staged-trace-fri-challenges data = fst (snd (snd prefix)) $\wedge$
 staged-trace-final data = snd (snd (snd prefix)) $\wedge$
 staged-alphas data $\in$
 alpha-prefix-union-bad-sets prefix-state bad-sets
 (staged-trace-root data) $\wedge$
 alpha-vector-prequeried-in-state prefix-state
 (staged-alphas data) (length spec)))

definition *checked-staged-transcript-alpha-prefix-bad-set-hit*
 $:: 'f \text{ staged-adversary} \Rightarrow ('f \text{ list} \Rightarrow 'f \text{ list set}) \Rightarrow$
 $('f \text{ staged-proof-data} \times 'f \text{ protocol-channel}) \text{ option} \Rightarrow \text{bool}$
where

checked-staged-transcript-alpha-prefix-bad-set-hit A bad-sets out
 $\longleftrightarrow$
 (case out of
 None $\Rightarrow$ False
 | Some (data, -) $\Rightarrow$
 ($\exists$ prefix prefix-state.
 Some (prefix, prefix-state) $\in$
 set-dist
 (execute (staged-alpha-prefix-program A)
 adversary-initial-state) $\wedge$
 staged-trace-root data = fst prefix $\wedge$
 staged-trace-fri-roots data = fst (snd prefix) $\wedge$
 staged-trace-fri-challenges data = fst (snd (snd prefix)) $\wedge$
 staged-trace-final data = snd (snd (snd prefix)) $\wedge$
 staged-alphas data $\in$
 alpha-prefix-union-bad-sets prefix-state bad-sets
 (staged-trace-root data)))

lemma *checked-staged-transcript-alpha-prefix-prequery-hit-imp-bad-set-hit:*
assumes *hit:*
 checked-staged-transcript-alpha-prefix-prequery-hit A bad-sets out
shows *checked-staged-transcript-alpha-prefix-bad-set-hit A bad-sets out*
using *hit*
unfolding *checked-staged-transcript-alpha-prefix-prequery-hit-def*
 checked-staged-transcript-alpha-prefix-bad-set-hit-def

by (*auto split: option.splits prod.splits*)

lemma *checked-staged-transcript-alpha-prefix-prequery-hitE*:

assumes *hit*:

checked-staged-transcript-alpha-prefix-prequery-hit A bad-sets
(Some (data, attacker-state))

obtains *prefix prefix-state* **where**

Some (prefix, prefix-state) ∈

set-dist

(execute (staged-alpha-prefix-program A) adversary-initial-state)

staged-trace-root data = fst prefix

staged-trace-fri-roots data = fst (snd prefix)

staged-trace-fri-challenges data = fst (snd (snd prefix))

staged-trace-final data = snd (snd (snd prefix))

staged-alphas data ∈

alpha-prefix-union-bad-sets prefix-state bad-sets

(staged-trace-root data)

alpha-vector-prequeried-in-state prefix-state

(staged-alphas data) (length spec)

hash-relation-hit

(alpha-vector-key-relation prefix-state

(alpha-prefix-union-bad-sets prefix-state bad-sets

(staged-trace-root data))

(length spec))

adversary-initial-state prefix-state

proof —

from *hit* **obtain** *prefix prefix-state* **where**

prefix-out:

Some (prefix, prefix-state) ∈

set-dist

(execute (staged-alpha-prefix-program A)

adversary-initial-state)

and *root-eq*: *staged-trace-root data = fst prefix*

and *trace-roots-eq*:

staged-trace-fri-roots data = fst (snd prefix)

and *trace-bs-eq*:

staged-trace-fri-challenges data = fst (snd (snd prefix))

and *trace-final-eq*:

staged-trace-final data = snd (snd (snd prefix))

and *member*:

staged-alphas data ∈

alpha-prefix-union-bad-sets prefix-state bad-sets

(staged-trace-root data)

and *prequeried*:

alpha-vector-prequeried-in-state prefix-state

(staged-alphas data) (length spec)

unfolding *checked-staged-transcript-alpha-prefix-prequery-hit-def*

by *auto*

have *relation-hit*:

```

hash-relation-hit
  (alpha-vector-key-relation prefix-state
    (alpha-prefix-union-bad-sets prefix-state bad-sets
      (staged-trace-root data))
    (length spec))
  adversary-initial-state prefix-state
by (rule alpha-vector-prequeried-in-state-imp-hash-relation-hit-empty-initial
  [OF prequeried member])
simp
show ?thesis
by (rule that[OF prefix-out root-eq trace-roots-eq trace-bs-eq
  trace-final-eq member prequeried relation-hit])
qed

```

definition *staged-alpha-prefix-dynamic-prequery-hit*

```

:: ('f list  $\Rightarrow$  'f list set)  $\Rightarrow$ 
  (('f  $\times$  'f list  $\times$  'f list  $\times$  'f)  $\times$ 
    'f protocol-channel) option  $\Rightarrow$  bool
where
  staged-alpha-prefix-dynamic-prequery-hit bad-sets out  $\longleftrightarrow$ 
    (case out of
      None  $\Rightarrow$  False
    | Some ((fr, -), prefix-state)  $\Rightarrow$ 
      ( $\exists$  as  $\in$  alpha-prefix-union-bad-sets prefix-state bad-sets fr.
        alpha-vector-prequeried-in-state prefix-state as
          (length spec)))

```

definition *staged-alpha-prefix-dynamic-prequery-relation*

```

:: 'f staged-adversary  $\Rightarrow$  'f protocol-channel  $\Rightarrow$ 
  ('f list  $\Rightarrow$  'f list set)  $\Rightarrow$ 
  'f protocol-hash-input  $\Rightarrow$  'f  $\Rightarrow$  bool
where
  staged-alpha-prefix-dynamic-prequery-relation A s bad-sets x y
     $\longleftrightarrow$ 
    ( $\exists$  fr trace-roots trace-bs trace-final prefix-state as i.
      Some ((fr, trace-roots, trace-bs, trace-final), prefix-state)  $\in$ 
        set-dist (execute (staged-alpha-prefix-program A) s)  $\wedge$ 
      as  $\in$  alpha-prefix-union-bad-sets prefix-state bad-sets fr  $\wedge$ 
      i < length spec  $\wedge$ 
      i < length as  $\wedge$ 
      x = alpha-challenge-key-at prefix-state i as  $\wedge$ 
      y = as ! i)

```

lemma *staged-alpha-prefix-dynamic-prequery-hit-imp-relation-hit:*

```

assumes empty: HashMap s = fmempty
and support:
  Some ((fr, trace-roots, trace-bs, trace-final), prefix-state)  $\in$ 
    set-dist (execute (staged-alpha-prefix-program A) s)
and hit:

```

```

    staged-alpha-prefix-dynamic-prequery-hit bad-sets
    (Some ((fr, trace-roots, trace-bs, trace-final), prefix-state))
  shows
    hash-relation-hit-event
    (staged-alpha-prefix-dynamic-prequery-relation A s bad-sets)
    s (Some ((fr, trace-roots, trace-bs, trace-final), prefix-state))
proof -
  from hit obtain as i where member:
    as ∈ alpha-prefix-union-bad-sets prefix-state bad-sets fr
  and i-bound: i < length spec
  and i-len: i < length as
  and lookup:
    fmllookup (HashMap prefix-state)
    (alpha-challenge-key-at prefix-state i as) = Some (as ! i)
  unfolding staged-alpha-prefix-dynamic-prequery-hit-def
    alpha-vector-prequeried-in-state-def
  by auto
  have initial-none:
    fmllookup (HashMap s) (alpha-challenge-key-at prefix-state i as) =
      None
  using empty by simp
  have relation:
    staged-alpha-prefix-dynamic-prequery-relation A s bad-sets
    (alpha-challenge-key-at prefix-state i as) (as ! i)
  unfolding staged-alpha-prefix-dynamic-prequery-relation-def
  using support member i-bound i-len by blast
  have hit-relation:
    hash-relation-hit
    (staged-alpha-prefix-dynamic-prequery-relation A s bad-sets)
    s prefix-state
  unfolding hash-relation-hit-def
  by (intro exI[of - alpha-challenge-key-at prefix-state i as]
    exI[of - as ! i] conjI)
    (use initial-none lookup relation in simp-all)
  show ?thesis
    unfolding hash-relation-hit-event-def using hit-relation by simp
qed

lemma staged-alpha-prefix-dynamic-prequery-hit-bound-by-relation-hit:
  assumes empty: HashMap s = fmempty
  shows
    wp-event (staged-alpha-prefix-program A)
    (staged-alpha-prefix-dynamic-prequery-hit bad-sets) s ≤
    wp-event (staged-alpha-prefix-program A)
    (hash-relation-hit-event
    (staged-alpha-prefix-dynamic-prequery-relation A s bad-sets) s) s
proof (rule wp-event-mono-on-support)
  fix out
  assume support:

```

```

    out ∈ set-dist (execute (staged-alpha-prefix-program A) s)
  and prehit: staged-alpha-prefix-dynamic-prequery-hit bad-sets out
show
  hash-relation-hit-event
    (staged-alpha-prefix-dynamic-prequery-relation A s bad-sets)
    s out
proof (cases out)
case None
then show ?thesis
  using prehit unfolding staged-alpha-prefix-dynamic-prequery-hit-def
  by simp
next
case (Some packed)
then obtain fr trace-roots trace-bs trace-final prefix-state
  where out-eq:
    out =
      Some ((fr, trace-roots, trace-bs, trace-final), prefix-state)
  by (cases packed, auto split: prod.splits)
show ?thesis
  unfolding out-eq
  by (rule staged-alpha-prefix-dynamic-prequery-hit-imp-relation-hit
      [OF empty])
    (use support prehit out-eq in simp-all)
qed
qed

lemma staged-alpha-prefix-dynamic-prequery-relation-fiber-bound-size:
  card
    {y. staged-alpha-prefix-dynamic-prequery-relation A s bad-sets x y}
  ≤ size
proof -
  have subset:
    {y. staged-alpha-prefix-dynamic-prequery-relation A s bad-sets x y}
    ⊆ (UNIV :: 'f set)
  by simp
  have finite-field: finite (UNIV :: 'f set)
  by simp
  have card
    {y. staged-alpha-prefix-dynamic-prequery-relation A s bad-sets x y}
    ≤ card (UNIV :: 'f set)
  by (rule card-mono[OF finite-field subset])
  also have ... = size
  using size-card by simp
  finally show ?thesis .
qed

lemma staged-alpha-prefix-dynamic-prequery-hit-bound:
  assumes wf: staged-budget-wellformed budgets
  and controlled: staged-adversary-controlled budgets A

```

shows

wp-event (*staged-alpha-prefix-program A*)
 (*staged-alpha-prefix-dynamic-prequery-hit bad-sets*)
adversary-initial-state $\leq$
staged-phase-relation-error size
 (*staged-alpha-search-queries budgets 0*)

proof –

have *prehit-le-relation*:

wp-event (*staged-alpha-prefix-program A*)
 (*staged-alpha-prefix-dynamic-prequery-hit bad-sets*)
adversary-initial-state $\leq$
wp-event (*staged-alpha-prefix-program A*)
 (*hash-relation-hit-event*
 (*staged-alpha-prefix-dynamic-prequery-relation A*
adversary-initial-state bad-sets)
adversary-initial-state)
adversary-initial-state

by (*rule staged-alpha-prefix-dynamic-prequery-hit-bound-by-relation-hit*)
 (*simp add: adversary-initial-state-def*)

also have ... $\leq$

staged-phase-relation-error size
 (*staged-alpha-search-queries budgets 0*)

proof –

have *program*:

hash-relation-program
 (*staged-alpha-prefix-dynamic-prequery-relation A*
adversary-initial-state bad-sets)
size (*staged-alpha-search-queries budgets 0*)
 (*staged-alpha-prefix-program A*)

by (*rule hash-relation-program-staged-alpha-prefix-program*
 [*OF wf controlled*])
 (*rule staged-alpha-prefix-dynamic-prequery-relation-fiber-bound-size*)

show *?thesis*

using *program*

unfolding *hash-relation-program-def hash-relation-budget-def*
staged-phase-relation-error-def

by *blast*

qed

finally show *?thesis* .

qed

definition *checked-staged-transcript-actual-alpha-prefix-collision-hit*

$:: (((('f \times 'f \text{ list} \times 'f \text{ list} \times 'f) \times$
 $'f \text{ protocol-channel}) \times 'f \text{ staged-proof-data}) \times$
 $'f \text{ protocol-channel}) \text{ option} \Rightarrow \text{bool}$

where

checked-staged-transcript-actual-alpha-prefix-collision-hit out
 $\longleftrightarrow$
 (*case out of*

$None \Rightarrow False$
 $| Some ((-, prefix-state), -, -) \Rightarrow$
 $hash-map-output-collision\ prefix-state)$

lemma *checked-staged-transcript-actual-alpha-prefix-bad-set-hit-split-events:*

assumes

checked-staged-transcript-actual-alpha-prefix-bad-set-hit bad-sets out

shows

checked-staged-transcript-actual-alpha-prefix-clean-fresh-bad-set-hit

bad-sets out $\vee$

checked-staged-transcript-actual-alpha-prefix-prequery-hit bad-sets out $\vee$

checked-staged-transcript-actual-alpha-prefix-collision-hit out

using *checked-staged-transcript-actual-alpha-prefix-bad-set-hit-split*

[OF assms]

unfolding *checked-staged-transcript-actual-alpha-prefix-collision-hit-def*

by (*auto split: option.splits prod.splits*)

lemma *checked-staged-transcript-with-alpha-prefix-prequery-bound:*

shows

wp-event (checked-staged-transcript-with-alpha-prefix-program A)

(checked-staged-transcript-actual-alpha-prefix-prequery-hit bad-sets)

adversary-initial-state $\leq$

wp-event (staged-alpha-prefix-program A)

(staged-alpha-prefix-dynamic-prequery-hit bad-sets)

adversary-initial-state

unfolding *checked-staged-transcript-with-alpha-prefix-program-def*

proof (*rule wp-event-bind-bound-by-head-event*)

show

wp-event (staged-alpha-prefix-program A)

(staged-alpha-prefix-dynamic-prequery-hit bad-sets)

adversary-initial-state $\leq$

wp-event (staged-alpha-prefix-program A)

(staged-alpha-prefix-dynamic-prequery-hit bad-sets)

adversary-initial-state

by *simp*

next

show

checked-staged-transcript-actual-alpha-prefix-prequery-hit

bad-sets None $\implies$

staged-alpha-prefix-dynamic-prequery-hit bad-sets None

unfolding *checked-staged-transcript-actual-alpha-prefix-prequery-hit-def*

staged-alpha-prefix-dynamic-prequery-hit-def

by *simp*

next

fix *prefix prefix-state out*

assume *prefix-out:*

Some (prefix, prefix-state) $\in$

set-dist

(execute (staged-alpha-prefix-program A) adversary-initial-state)

```

and cont:
  out ∈
    set-dist
      (execute
        (get  $\gg$ 
          ( $\lambda$ prefix-state.
            checked-staged-after-alpha-prefix-program A prefix  $\gg$ 
              ( $\lambda$ data. return ((prefix, prefix-state), data))))
            prefix-state)
and hit:
  checked-staged-transcript-actual-alpha-prefix-prequery-hit
    bad-sets out
obtain fr trace-roots trace-bs trace-final where prefix-eq:
  prefix = (fr, trace-roots, trace-bs, trace-final)
  by (cases prefix) auto
from hit cont show
  staged-alpha-prefix-dynamic-prequery-hit bad-sets
    (Some (prefix, prefix-state))
unfolding checked-staged-transcript-actual-alpha-prefix-prequery-hit-def
  staged-alpha-prefix-dynamic-prequery-hit-def prefix-eq
  by (auto elim!: set-dist-bindE split: option.splits prod.splits)
qed

lemma checked-staged-transcript-with-alpha-prefix-collision-bound:
shows
  wp-event (checked-staged-transcript-with-alpha-prefix-program A)
    checked-staged-transcript-actual-alpha-prefix-collision-hit
      adversary-initial-state  $\leq$ 
  wp-event (staged-alpha-prefix-program A)
    (hash-new-collision-event adversary-initial-state)
    adversary-initial-state
unfolding checked-staged-transcript-with-alpha-prefix-program-def
proof (rule wp-event-bind-bound-by-head-event)
show
  wp-event (staged-alpha-prefix-program A)
    (hash-new-collision-event adversary-initial-state)
    adversary-initial-state  $\leq$ 
  wp-event (staged-alpha-prefix-program A)
    (hash-new-collision-event adversary-initial-state)
    adversary-initial-state
  by simp
next
show
  checked-staged-transcript-actual-alpha-prefix-collision-hit None  $\implies$ 
    hash-new-collision-event adversary-initial-state None
unfolding checked-staged-transcript-actual-alpha-prefix-collision-hit-def
  hash-new-collision-event-def
  by simp
next

```

```

fix prefix prefix-state out
assume prefix-out:
  Some (prefix, prefix-state) ∈
    set-dist
      (execute (staged-alpha-prefix-program A) adversary-initial-state)
and cont:
  out ∈
    set-dist
      (execute
        (get >>=
          (λprefix-state.
            checked-staged-after-alpha-prefix-program A prefix >>=
              (λdata. return ((prefix, prefix-state), data))))
        prefix-state)
and hit:
  checked-staged-transcript-actual-alpha-prefix-collision-hit out
have collision: hash-map-output-collision prefix-state
using hit cont
unfolding checked-staged-transcript-actual-alpha-prefix-collision-hit-def
by (auto elim!: set-dist-bindE split: option.splits prod.splits)
show
  hash-new-collision-event adversary-initial-state
    (Some (prefix, prefix-state))
using collision adversary-initial-state-no-output-collision
unfolding hash-new-collision-event-def hash-map-new-output-collision-def
by simp
qed

lemma checked-staged-transcript-with-alpha-prefix-actual-bad-set-hit-bound:
shows
  wp-event (checked-staged-transcript-with-alpha-prefix-program A)
    (checked-staged-transcript-actual-alpha-prefix-bad-set-hit
      composition-trace-bad-alpha-space)
  adversary-initial-state ≤
    composition-error-bound +
  wp-event (staged-alpha-prefix-program A)
    (staged-alpha-prefix-dynamic-prequery-hit
      composition-trace-bad-alpha-space)
  adversary-initial-state +
  wp-event (staged-alpha-prefix-program A)
    (hash-new-collision-event adversary-initial-state)
  adversary-initial-state
proof –
let ?m = checked-staged-transcript-with-alpha-prefix-program A
let ?bad =
  checked-staged-transcript-actual-alpha-prefix-bad-set-hit
    composition-trace-bad-alpha-space
let ?clean =
  checked-staged-transcript-actual-alpha-prefix-clean-fresh-bad-set-hit

```

```

    composition-trace-bad-alpha-space
  let ?pre =
    checked-staged-transcript-actual-alpha-prefix-prequery-hit
    composition-trace-bad-alpha-space
  let ?coll = checked-staged-transcript-actual-alpha-prefix-collision-hit
  have bad-le-split:
    wp-event ?m ?bad adversary-initial-state ≤
      wp-event ?m (λout. ?clean out ∨ ?pre out ∨ ?coll out)
      adversary-initial-state
  by (rule wp-event-mono)
    (rule
      checked-staged-transcript-actual-alpha-prefix-bad-set-hit-split-events)
  also have ... ≤
    wp-event ?m ?clean adversary-initial-state +
    wp-event ?m ?pre adversary-initial-state +
    wp-event ?m ?coll adversary-initial-state
  proof -
    have ... ≤
      wp-event ?m ?clean adversary-initial-state +
      wp-event ?m (λout. ?pre out ∨ ?coll out)
      adversary-initial-state
    by (rule wp-event-union-bound)
  also have ... ≤
    wp-event ?m ?clean adversary-initial-state +
    (wp-event ?m ?pre adversary-initial-state +
     wp-event ?m ?coll adversary-initial-state)
  by (intro add-mono order.refl wp-event-union-bound)
  finally show ?thesis
    by (simp add: add.assoc)
qed
also have ... ≤
  composition-error-bound +
  wp-event (staged-alpha-prefix-program A)
    (staged-alpha-prefix-dynamic-prequery-hit
     composition-trace-bad-alpha-space)
  adversary-initial-state +
  wp-event (staged-alpha-prefix-program A)
    (hash-new-collision-event adversary-initial-state)
  adversary-initial-state
  by (intro add-mono
    checked-staged-transcript-with-alpha-prefix-clean-fresh-composition-bound
    checked-staged-transcript-with-alpha-prefix-prequery-bound
    checked-staged-transcript-with-alpha-prefix-collision-bound)
  finally show ?thesis .
qed

lemma checked-staged-transcript-with-alpha-prefix-actual-bad-set-hit-bound-from-budgets:
  assumes wf: staged-budget-wellformed budgets
  and controlled: staged-adversary-controlled budgets A

```

shows

wp-event (*checked-staged-transcript-with-alpha-prefix-program A*)
 (*checked-staged-transcript-actual-alpha-prefix-bad-set-hit*
composition-trace-bad-alpha-space)
adversary-initial-state $\leq$
composition-error-bound +
staged-phase-relation-error size
 (*staged-alpha-search-queries budgets 0*) +
hash-collision-budget-value 0
 (*staged-alpha-search-queries budgets 0*)

proof –

have *prequery-bound*:

wp-event (*staged-alpha-prefix-program A*)
 (*staged-alpha-prefix-dynamic-prequery-hit*
composition-trace-bad-alpha-space)
adversary-initial-state $\leq$
staged-phase-relation-error size
 (*staged-alpha-search-queries budgets 0*)
by (*rule staged-alpha-prefix-dynamic-prequery-hit-bound*
 [*OF wf controlled*])

have *collision-budget*:

hash-collision-budget
 (*staged-alpha-search-queries budgets 0*)
 (*staged-alpha-prefix-program A*)
by (*rule hash-collision-budget-staged-alpha-prefix-program*
 [*OF wf controlled*])

have *collision-bound*:

wp-event (*staged-alpha-prefix-program A*)
 (*hash-new-collision-event adversary-initial-state*)
adversary-initial-state $\leq$
hash-collision-budget-value 0
 (*staged-alpha-search-queries budgets 0*)

proof –

have

wp-event (*staged-alpha-prefix-program A*)
 (*hash-new-collision-event adversary-initial-state*)
adversary-initial-state $\leq$
hash-collision-budget-value
 (*card (hash-map-output-values adversary-initial-state)*)
 (*staged-alpha-search-queries budgets 0*)
using *collision-budget adversary-initial-state-no-output-collision*
unfolding *hash-collision-budget-def* **by** *blast*

also have ... =

hash-collision-budget-value 0
 (*staged-alpha-search-queries budgets 0*)

by *simp*

finally show *?thesis* .

qed

have

```

wp-event (checked-staged-transcript-with-alpha-prefix-program A)
  (checked-staged-transcript-actual-alpha-prefix-bad-set-hit
    composition-trace-bad-alpha-space)
adversary-initial-state ≤
composition-error-bound +
wp-event (staged-alpha-prefix-program A)
  (staged-alpha-prefix-dynamic-prequery-hit
    composition-trace-bad-alpha-space)
adversary-initial-state +
wp-event (staged-alpha-prefix-program A)
  (hash-new-collision-event adversary-initial-state)
adversary-initial-state
by (rule checked-staged-transcript-with-alpha-prefix-actual-bad-set-hit-bound)
also have ... ≤
  composition-error-bound +
  staged-phase-relation-error size
  (staged-alpha-search-queries budgets 0) +
  hash-collision-budget-value 0
  (staged-alpha-search-queries budgets 0)
by (intro add-mono order.refl prequery-bound collision-bound)
finally show ?thesis .
qed

```

definition *checked-staged-security-with-actual-alpha-prefix-bad-set-hit*

```

:: ('f list ⇒ 'f list set) ⇒
  ((((((('f × 'f list × 'f list × 'f) ×
    'f protocol-channel) × 'f staged-proof-data) ×
    'f protocol-channel) × unit list) ×
    'f protocol-channel) option ⇒ bool

```

where

```

checked-staged-security-with-actual-alpha-prefix-bad-set-hit
bad-sets out ⇔
(case out of
  None ⇒ False
| Some (x, -) ⇒
  (let packed = fst (fst x);
    prefix = fst (fst packed);
    prefix-state = snd (fst packed);
    data = snd packed
  in staged-alphas data ∈
    alpha-prefix-union-bad-sets prefix-state bad-sets (fst prefix)))

```

definition *checked-staged-security-with-actual-alpha-prefix-header-supported-bad-set-hit*

```

:: ('f list ⇒ 'f list set) ⇒
  ((((((('f × 'f list × 'f list × 'f) ×
    'f protocol-channel) × 'f staged-proof-data) ×
    'f protocol-channel) × unit list) ×
    'f protocol-channel) option ⇒ bool

```

where

checked-staged-security-with-actual-alpha-prefix-header-supported-bad-set-hit
bad-sets out $\longleftrightarrow$
 (case out of
 None $\Rightarrow$ False
 | Some (x, -) $\Rightarrow$
 (let packed = fst (fst x);
 data = snd packed;
 attacker-state = snd (fst x)
 in staged-alphas data $\in$
 alpha-header-supported-union-bad-sets
 (verifier-state-from-adversary attacker-state
 (staged-proof-transcript data))
 bad-sets
 (staged-trace-root data)
 (staged-trace-fri-roots data)
 (staged-trace-final data)))

definition *checked-staged-security-with-actual-alpha-prefix-tree-output-hit*

:: ((((((f $\times$ 'f list $\times$ 'f list $\times$ 'f) $\times$
 'f protocol-channel) $\times$ 'f staged-proof-data) $\times$
 'f protocol-channel) $\times$ unit list) $\times$
 'f protocol-channel) option $\Rightarrow$ bool

where

checked-staged-security-with-actual-alpha-prefix-tree-output-hit out
 $\longleftrightarrow$
 (case out of
 None $\Rightarrow$ False
 | Some (x, -) $\Rightarrow$
 (let packed = fst (fst x);
 prefix-state = snd (fst packed);
 data = snd packed;
 attacker-state = snd (fst x);
 s =
 verifier-state-from-adversary attacker-state
 (staged-proof-transcript data)
 in $\exists$ result final-state trace-table composition-table as
 query-idxs dg composition-fri-roots final rest trace-tree
 composition-tree.
 Some (result, final-state) $\in$
 set-dist (execute verify-monad s) $\wedge$
 accepted-with-bound-tables s (Some (result, final-state))
 trace-table composition-table as query-idxs $\wedge$
 verifier-header-transcript s
 (staged-trace-root data)
 (staged-trace-fri-roots data)
 (staged-trace-final data)
 as dg composition-fri-roots final rest $\wedge$
 created-tree trace-table trace-tree final-state $\wedge$
 created-tree composition-table composition-tree final-state $\wedge$

hash-map-new-output-hit
 (set-tree trace-tree $\cup$ set-tree composition-tree)
 prefix-state final-state))

lemma *checked-staged-security-with-actual-alpha-prefix-bad-set-hit-bound-from-transcript:*

assumes *transcript-bound:*

wp-event (*checked-staged-transcript-with-alpha-prefix-program* *A*)
 (*checked-staged-transcript-actual-alpha-prefix-bad-set-hit* *bad-sets*)
adversary-initial-state $\leq C$

shows

wp-event
 (*checked-staged-security-experiment-with-actual-alpha-prefix-data-state* *A*)
 (*checked-staged-security-with-actual-alpha-prefix-bad-set-hit* *bad-sets*)
adversary-initial-state $\leq C$

unfolding

checked-staged-security-experiment-with-actual-alpha-prefix-data-state-def

proof (rule *wp-event-bind-bound-by-head-event*[*OF transcript-bound*])

show

checked-staged-security-with-actual-alpha-prefix-bad-set-hit
bad-sets *None* $\implies$

checked-staged-transcript-actual-alpha-prefix-bad-set-hit *bad-sets* *None*

unfolding *checked-staged-security-with-actual-alpha-prefix-bad-set-hit-def*

checked-staged-transcript-actual-alpha-prefix-bad-set-hit-def

by *simp*

next

fix *packed attacker-state out*

assume *cont:*

out $\in$
set-dist
 (*execute*
 (*get* $\gg$
 ($\lambda s.$ *put*
 (*verifier-state-from-adversary* *s*
 (*staged-proof-transcript* (*snd* *packed*)))) $\gg$
 ($\lambda.$ *verify-monad* $\gg$
 (λ result. *return* ((*packed*, *s*), *result*))))
attacker-state)

and *hit:*

checked-staged-security-with-actual-alpha-prefix-bad-set-hit
bad-sets *out*

show

checked-staged-transcript-actual-alpha-prefix-bad-set-hit *bad-sets*
 (*Some* (*packed*, *attacker-state*))

using *cont hit*

unfolding *checked-staged-security-with-actual-alpha-prefix-bad-set-hit-def*

checked-staged-transcript-actual-alpha-prefix-bad-set-hit-def

by (*auto elim!*: *set-dist-bindE* *split*: *option.splits* *prod.splits*)

qed

lemma *checked-staged-security-with-data-state-alpha-header-supported-bad-set-hit-actual-projection:*

```

wp-event (checked-staged-security-experiment-with-data-state A)
  (checked-staged-security-with-data-state-alpha-header-supported-bad-set-hit
    bad-sets)
adversary-initial-state =
wp-event
  (checked-staged-security-experiment-with-actual-alpha-prefix-data-state A)
  (checked-staged-security-with-actual-alpha-prefix-header-supported-bad-set-hit
    bad-sets)
adversary-initial-state

```

proof –

```

let ?project =
  λx. ((snd (fst (fst x)), snd (fst x)), snd x)

```

have

```

wp-event (checked-staged-security-experiment-with-data-state A)
  (checked-staged-security-with-data-state-alpha-header-supported-bad-set-hit
    bad-sets)
adversary-initial-state =
wp-event
  (checked-staged-security-experiment-with-actual-alpha-prefix-data-state A
    ≫ (λx. return (?project x)))
  (checked-staged-security-with-data-state-alpha-header-supported-bad-set-hit
    bad-sets)
adversary-initial-state

```

using

```

checked-staged-security-experiment-with-actual-alpha-prefix-data-state-projection
[of A]

```

by *simp*

also have ... =

```

wp-event
  (checked-staged-security-experiment-with-actual-alpha-prefix-data-state A)
  (λout. case out of
    None ⇒
      checked-staged-security-with-data-state-alpha-header-supported-bad-set-hit
        bad-sets None
    | Some (x, t) ⇒
      checked-staged-security-with-data-state-alpha-header-supported-bad-set-hit
        bad-sets (Some (?project x, t)))
  adversary-initial-state

```

by (rule *wp-event-bind-return-map*)

also have ... =

```

wp-event
  (checked-staged-security-experiment-with-actual-alpha-prefix-data-state A)
  (checked-staged-security-with-actual-alpha-prefix-header-supported-bad-set-hit
    bad-sets)
adversary-initial-state

```

unfolding

```

checked-staged-security-with-data-state-alpha-header-supported-bad-set-hit-def
checked-staged-security-with-actual-alpha-prefix-header-supported-bad-set-hit-def

```

```

apply (rule arg-cong[where
   $f = \lambda P. \text{wp-event}$ 
  (checked-staged-security-experiment-with-actual-alpha-prefix-data-state A)
   $P \text{ adversary-initial-state}$ ])
apply (rule ext)
apply (simp add: Let-def split: option.splits prod.splits)
done
finally show ?thesis .
qed

```

lemma *checked-staged-security-with-actual-alpha-prefix-tree-output-hitI:*

```

assumes verify-out:
   $\text{Some } (result, final\text{-}state) \in$ 
    set-dist
    (execute verify-monad
      (verifier-state-from-adversary attacker-state
        (staged-proof-transcript data)))
and bound:
  accepted-with-bound-tables
    (verifier-state-from-adversary attacker-state
      (staged-proof-transcript data))
    ( $\text{Some } (result, final\text{-}state)$ ) trace-table composition-table as
      query-idxs
and header:
  verifier-header-transcript
    (verifier-state-from-adversary attacker-state
      (staged-proof-transcript data))
    (staged-trace-root data)
    (staged-trace-fri-roots data)
    (staged-trace-final data)
    as dg composition-fri-roots final rest
and trace-created: created-tree trace-table trace-tree final-state
and composition-created:
  created-tree composition-table composition-tree final-state
and tree-hit:
  hash-map-new-output-hit
    (set-tree trace-tree  $\cup$  set-tree composition-tree)
    prefix-state final-state
shows
  checked-staged-security-with-actual-alpha-prefix-tree-output-hit
    ( $\text{Some } (((((prefix, prefix\text{-}state), data), attacker\text{-}state),$ 
      checked-result), checked-final-state))

```

proof –

```

have witness:
   $\exists result\ final\text{-}state\ trace\text{-}table\ composition\text{-}table\ as\ query\text{-}idxs\ dg$ 
    composition-fri-roots final rest trace-tree composition-tree.
   $\text{Some } (result, final\text{-}state) \in$ 
    set-dist
    (execute verify-monad

```

```

      (verifier-state-from-adversary attacker-state
        (staged-proof-transcript data))) ∧
    accepted-with-bound-tables
      (verifier-state-from-adversary attacker-state
        (staged-proof-transcript data))
      (Some (result, final-state)) trace-table composition-table as
        query-idxs ∧
    verifier-header-transcript
      (verifier-state-from-adversary attacker-state
        (staged-proof-transcript data))
      (staged-trace-root data)
      (staged-trace-fri-roots data)
      (staged-trace-final data)
      as dg composition-fri-roots final rest ∧
    created-tree trace-table trace-tree final-state ∧
    created-tree composition-table composition-tree final-state ∧
    hash-map-new-output-hit
      (set-tree trace-tree ∪ set-tree composition-tree)
      prefix-state final-state
  using verify-out bound header trace-created composition-created tree-hit
  by blast
show ?thesis
unfolding
  checked-staged-security-with-actual-alpha-prefix-tree-output-hit-def
  Let-def
using witness by simp
qed

```

definition *checked-staged-security-with-data-state-alpha-prefix-tree-output-hit*

```

:: 'f staged-adversary ⇒
  (((('f staged-proof-data × 'f protocol-channel) × unit list) ×
    'f protocol-channel) option ⇒ bool

```

where

```

checked-staged-security-with-data-state-alpha-prefix-tree-output-hit A
  out ⟷
  (case out of
    None ⇒ False
  | Some (((data, attacker-state), -), -) ⇒
    (let s =
      verifier-state-from-adversary attacker-state
        (staged-proof-transcript data)
    in ∃ prefix prefix-state result final-state trace-table
      composition-table as query-idxs dg composition-fri-roots final
      rest trace-tree composition-tree.
      Some (prefix, prefix-state) ∈
        set-dist
          (execute (staged-alpha-prefix-program A)
            adversary-initial-state) ∧
          staged-trace-root data = fst prefix ∧

```

staged-trace-fri-roots data = fst (snd prefix) ∧
staged-trace-fri-challenges data = fst (snd (snd prefix)) ∧
staged-trace-final data = snd (snd (snd prefix)) ∧
Some (result, final-state) ∈
set-dist (execute verify-monad s) ∧
accepted-with-bound-tables s (Some (result, final-state))
trace-table composition-table as query-idxs ∧
verifier-header-transcript s
(staged-trace-root data)
(staged-trace-fri-roots data)
(staged-trace-final data)
as dg composition-fri-roots final rest ∧
created-tree trace-table trace-tree final-state ∧
created-tree composition-table composition-tree final-state ∧
hash-map-new-output-hit
(set-tree trace-tree ∪ set-tree composition-tree)
prefix-state final-state))

lemma *checked-staged-security-with-data-state-alpha-prefix-bad-set-hit-alphas-in-space:*

assumes *hit:*

checked-staged-security-with-data-state-alpha-prefix-bad-set-hit A
bad-sets out

shows

case out of

None ⇒ False

| Some (((data, -), -), -) ⇒ staged-alphas data ∈ alpha-space

using *hit*

unfolding *checked-staged-security-with-data-state-alpha-prefix-bad-set-hit-def*

alpha-prefix-union-bad-sets-def

by (*auto split: option.splits prod.splits*)

lemma *checked-staged-security-with-data-state-alpha-prefix-bad-set-hit-bound-from-transcript:*

assumes *transcript-bound:*

wp-event (checked-staged-transcript-program A)
(checked-staged-transcript-alpha-prefix-bad-set-hit A bad-sets)
adversary-initial-state ≤ C

shows

wp-event (checked-staged-security-experiment-with-data-state A)
(checked-staged-security-with-data-state-alpha-prefix-bad-set-hit A
bad-sets)
adversary-initial-state ≤ C

unfolding *checked-staged-security-experiment-with-data-state-def*

proof (*rule wp-event-bind-bound-by-head-event[OF transcript-bound]*)

show

checked-staged-security-with-data-state-alpha-prefix-bad-set-hit A
bad-sets None ⇒

checked-staged-transcript-alpha-prefix-bad-set-hit A bad-sets None

unfolding

checked-staged-security-with-data-state-alpha-prefix-bad-set-hit-def

```

      checked-staged-transcript-alpha-prefix-bad-set-hit-def
    by simp
next
fix data attacker-state out
assume cont:
  out ∈
    set-dist
      (execute
        (get >>=
          (λs. put
            (verifier-state-from-adversary s
              (staged-proof-transcript data)) >>=
            (λ-. verify-monad >>=
              (λresult. return ((data, s), result))))))
        attacker-state)
and hit:
  checked-staged-security-with-data-state-alpha-prefix-bad-set-hit A
  bad-sets out
show
  checked-staged-transcript-alpha-prefix-bad-set-hit A bad-sets
  (Some (data, attacker-state))
proof (cases out)
case None
then show ?thesis
  using hit
  unfolding
    checked-staged-security-with-data-state-alpha-prefix-bad-set-hit-def
  by simp
next
case (Some result-pack)
then obtain data' attacker-state' result final-state where out-eq:
  out = Some (((data', attacker-state'), result), final-state)
  by (cases result-pack, auto split: prod.splits)
have data-eq: data' = data
  and attacker-eq: attacker-state' = attacker-state
  using cont unfolding out-eq
  by (auto elim!: set-dist-bindE)
show ?thesis
  using hit unfolding out-eq data-eq attacker-eq
    checked-staged-security-with-data-state-alpha-prefix-bad-set-hit-def
    checked-staged-transcript-alpha-prefix-bad-set-hit-def
  by simp
qed
qed

lemma staged-after-alpha-prefix-program-outcome-extension:
  assumes wf: staged-budget-wellformed budgets
  and controlled: staged-adversary-controlled budgets A
  and outcome:

```

```

    Some (data, t) ∈
      set-dist
        (execute (staged-after-alpha-prefix-program A prefix) s)
  shows  $s \leq t$ 
proof -
  obtain fr trace-roots trace-bs trace-final where prefix-eq:
    prefix = (fr, trace-roots, trace-bs, trace-final)
  by (cases prefix) simp
  from outcome[unfolded prefix-eq]
  obtain as s1 dg s2 s3 s4 composition-roots composition-bs s5
    composition-final s6 s7 query-chunks where
    alpha-out:
      Some (as, s1) ∈
        set-dist (execute (staged-alpha-program (length spec)) s)
  and degree-out:
      Some (dg, s2) ∈ set-dist (execute (degree-stage A as) s1)
  and record-degree-out:
      Some ((), s3) ∈ set-dist (execute (record-staged-message dg) s2)
  and assert-out:
      Some ((), s4) ∈
        set-dist
          (execute
            (assert
              (ceil-log (to-nat dg + 1) ≤ ceil-log (maxDegree + 1)))
            s3)
  and composition-out:
      Some ((composition-roots, composition-bs), s5) ∈
        set-dist
          (execute
            (staged-composition-fri-program A dg 0
              (ceil-log (to-nat dg + 1)) []))
            s4)
  and composition-final-out:
      Some (composition-final, s6) ∈
        set-dist (execute (composition-final-stage A dg composition-bs) s5)
  and record-composition-final-out:
      Some ((), s7) ∈
        set-dist (execute (record-staged-message composition-final) s6)
  and query-out:
      Some (query-chunks, t) ∈
        set-dist (execute (staged-query-program A 0 rounds) s7)
  unfolding staged-after-alpha-prefix-program-def Let-def
  by (auto elim!: set-dist-bindE split: prod.splits)
  have degree-controlled:
    controlled-ro-program (degree-budget budgets) (degree-stage A as)
  using controlled unfolding staged-adversary-controlled-def by blast
  have composition-final-controlled:
    controlled-ro-program (composition-final-budget budgets)
      (composition-final-stage A dg composition-bs)

```

```

using controlled unfolding staged-adversary-controlled-def by blast
have ext-s-s1:  $s \leq s1$ 
  by (rule hash-target-program-outcome-extension
    [OF hash-target-program-staged-alpha-program alpha-out])
have ext-s1-s2:  $s1 \leq s2$ 
  using controlled-ro-program-extension[OF degree-controlled] degree-out
  unfolding hash-extension-preserving-def by blast
have s3-eq:
   $s3 = s2 \langle$ 
     $PState := \text{concat } (PState\ s2)\ dg,$ 
     $PTranscript := PTranscript\ s2\ @\ [dg]\rangle$ 
  by (rule record-staged-message-outcome[OF record-degree-out])
have ext-s2-s3:  $s2 \leq s3$ 
  unfolding s3-eq less-eq-hash-ext-def less-eq-fmap-def by simp
have s4-eq:  $s4 = s3$ 
  using assert-out unfolding assert-def
  by (cases
     $\text{ceil-log } (to\text{-nat } dg + 1) \leq \text{ceil-log } (maxDegree + 1))$ 
    (auto simp: throw-no-outcome)
  )
have ext-s3-s4:  $s3 \leq s4$ 
  unfolding s4-eq by (simp add: hash-ext-refl)
have round-bound:
   $\text{ceil-log } (to\text{-nat } dg + 1) \leq \text{ceil-log } (maxDegree + 1)$ 
  using assert-out unfolding assert-def
  by (cases
     $\text{ceil-log } (to\text{-nat } dg + 1) \leq \text{ceil-log } (maxDegree + 1))$ 
    (auto simp: throw-no-outcome)
  )
have composition-bound:
   $0 + \text{ceil-log } (to\text{-nat } dg + 1) \leq$ 
   $\text{length } (composition\text{-fri}\text{-budgets } budgets)$ 
  using wf round-bound unfolding staged-budget-wellformed-def by simp
have ext-s4-s5:  $s4 \leq s5$ 
  using staged-composition-fri-program-alignment
  [OF controlled composition-bound composition-out]
  by simp
have ext-s5-s6:  $s5 \leq s6$ 
  using controlled-ro-program-extension[OF composition-final-controlled]
  composition-final-out
  unfolding hash-extension-preserving-def by blast
have s7-eq:
   $s7 = s6 \langle$ 
     $PState := \text{concat } (PState\ s6)\ composition\text{-final},$ 
     $PTranscript := PTranscript\ s6\ @\ [composition\text{-final}]\rangle$ 
  by (rule record-staged-message-outcome[OF record-composition-final-out])
have ext-s6-s7:  $s6 \leq s7$ 
  unfolding s7-eq less-eq-hash-ext-def less-eq-fmap-def by simp
have query-bound:  $0 + \text{rounds} \leq \text{length } (query\text{-opening}\text{-budgets } budgets)$ 
  using wf unfolding staged-budget-wellformed-def by simp
have ext-s7-t:  $s7 \leq t$ 

```

by (rule hash-target-program-outcome-extension
 [OF hash-target-program-staged-query-program[OF controlled query-bound]
 query-out])
 show ?thesis
 by (rule hash-ext-trans[OF ext-s-s1])
 (rule hash-ext-trans[OF ext-s1-s2],
 rule hash-ext-trans[OF ext-s2-s3],
 rule hash-ext-trans[OF ext-s3-s4],
 rule hash-ext-trans[OF ext-s4-s5],
 rule hash-ext-trans[OF ext-s5-s6],
 rule hash-ext-trans[OF ext-s6-s7 ext-s7-t])
 qed

lemma checked-staged-after-alpha-prefix-program-outcome-extension:
assumes wf: staged-budget-wellformed budgets
and controlled: staged-adversary-controlled budgets A
and outcome:
 Some (data, t) ∈
 set-dist
 (execute (checked-staged-after-alpha-prefix-program A prefix) s)
shows s ≤ t
proof –
obtain fr trace-roots trace-bs trace-final **where** prefix-eq:
 prefix = (fr, trace-roots, trace-bs, trace-final)
by (cases prefix) simp
from outcome[unfolded prefix-eq]
obtain as s1 dg s2 s3 s4 composition-roots composition-bs s5
 composition-final s6 s7 query-chunks **where**
 alpha-out:
 Some (as, s1) ∈
 set-dist (execute (staged-alpha-program (length spec)) s)
and degree-out:
 Some (dg, s2) ∈ set-dist (execute (degree-stage A as) s1)
and record-degree-out:
 Some ((), s3) ∈ set-dist (execute (record-staged-message dg) s2)
and assert-out:
 Some ((), s4) ∈
 set-dist
 (execute
 (assert
 (ceil-log (to-nat dg + 1) ≤ ceil-log (maxDegree + 1)))
 s3)
and composition-out:
 Some ((composition-roots, composition-bs), s5) ∈
 set-dist
 (execute
 (staged-composition-fri-program A dg 0
 (ceil-log (to-nat dg + 1)) [])
 s4)


```

and composition-final-out:
  Some (composition-final, s6) ∈
    set-dist (execute (composition-final-stage A dg composition-bs) s5)
and record-composition-final-out:
  Some ((), s7) ∈
    set-dist (execute (record-staged-message composition-final) s6)
and query-out:
  Some (query-chunks, t) ∈
    set-dist
      (execute
        (checked-staged-query-program A trace-roots composition-roots
          0 rounds)
        s7)
  unfolding checked-staged-after-alpha-prefix-program-def Let-def
  by (auto elim!: set-dist-bindE split: prod.splits)
have degree-controlled:
  controlled-ro-program (degree-budget budgets) (degree-stage A as)
  using controlled unfolding staged-adversary-controlled-def by blast
have composition-final-controlled:
  controlled-ro-program (composition-final-budget budgets)
    (composition-final-stage A dg composition-bs)
  using controlled unfolding staged-adversary-controlled-def by blast
have ext-s-s1: s ≤ s1
  by (rule hash-target-program-outcome-extension
    [OF hash-target-program-staged-alpha-program alpha-out])
have ext-s1-s2: s1 ≤ s2
  using controlled-ro-program-extension[OF degree-controlled] degree-out
  unfolding hash-extension-preserving-def by blast
have s3-eq:
  s3 = s2(|
    PState := concat (PState s2) dg,
    PTranscript := PTranscript s2 @ [dg])
  by (rule record-staged-message-outcome[OF record-degree-out])
have ext-s2-s3: s2 ≤ s3
  unfolding s3-eq less-eq-hash-ext-def less-eq-fmap-def by simp
have s4-eq: s4 = s3
  using assert-out unfolding assert-def
  by (cases
    ceil-log (to-nat dg + 1) ≤ ceil-log (maxDegree + 1))
    (auto simp: throw-no-outcome)
have ext-s3-s4: s3 ≤ s4
  unfolding s4-eq by (simp add: hash-ext-refl)
have round-bound:
  ceil-log (to-nat dg + 1) ≤ ceil-log (maxDegree + 1)
  using assert-out unfolding assert-def
  by (cases
    ceil-log (to-nat dg + 1) ≤ ceil-log (maxDegree + 1))
    (auto simp: throw-no-outcome)
have composition-bound:

```

```

0 + ceil-log (to-nat dg + 1) ≤
  length (composition-fri-budgets budgets)
using wf round-bound unfolding staged-budget-wellformed-def by simp
have ext-s4-s5: s4 ≤ s5
using staged-composition-fri-program-alignment
  [OF controlled composition-bound composition-out]
by simp
have ext-s5-s6: s5 ≤ s6
using controlled-ro-program-extension[OF composition-final-controlled]
  composition-final-out
unfolding hash-extension-preserving-def by blast
have s7-eq:
s7 = s6(
  PState := concat (PState s6) composition-final,
  PTranscript := PTranscript s6 @ [composition-final])
by (rule record-staged-message-outcome[OF record-composition-final-out])
have ext-s6-s7: s6 ≤ s7
unfolding s7-eq less-eq-hash-ext-def less-eq-fmap-def by simp
have query-bound: 0 + rounds ≤ length (query-opening-budgets budgets)
using wf unfolding staged-budget-wellformed-def by simp
have query-program:
hash-relation-program (λ- -. False) 0
  (sum-list (take rounds (drop 0 (query-opening-budgets budgets))) +
    rounds)
  (checked-staged-query-program A trace-roots composition-roots 0
    rounds)
by (rule hash-relation-program-checked-staged-query-program
  [OF controlled query-bound])
  simp
have ext-s7-t: s7 ≤ t
using query-program query-out
unfolding hash-relation-program-def hash-extension-preserving-def
by blast
show ?thesis
by (rule hash-ext-trans[OF ext-s-s1])
  (rule hash-ext-trans[OF ext-s1-s2],
  rule hash-ext-trans[OF ext-s2-s3],
  rule hash-ext-trans[OF ext-s3-s4],
  rule hash-ext-trans[OF ext-s4-s5],
  rule hash-ext-trans[OF ext-s5-s6],
  rule hash-ext-trans[OF ext-s6-s7 ext-s7-t])
qed

lemma checked-staged-transcript-with-alpha-prefix-program-support:
assumes wf: staged-budget-wellformed budgets
and controlled: staged-adversary-controlled budgets A
and outcome:
Some (((prefix, prefix-state), data), attacker-state) ∈
  set-dist

```

```

      (execute (checked-staged-transcript-with-alpha-prefix-program A)
        adversary-initial-state)
obtains
  Some (prefix, prefix-state) ∈
    set-dist
      (execute (staged-alpha-prefix-program A) adversary-initial-state)
  staged-trace-root data = fst prefix
  staged-trace-fri-roots data = fst (snd prefix)
  staged-trace-fri-challenges data = fst (snd (snd prefix))
  staged-trace-final data = snd (snd (snd prefix))
  prefix-state ≤ attacker-state
proof –
from outcome obtain after-out where prefix-out:
  Some (prefix, prefix-state) ∈
    set-dist
      (execute (staged-alpha-prefix-program A) adversary-initial-state)
and after-out:
  Some (data, attacker-state) ∈
    set-dist
      (execute (checked-staged-after-alpha-prefix-program A prefix)
        prefix-state)
unfolding checked-staged-transcript-with-alpha-prefix-program-def
by (auto elim!: set-dist-bindE)
have prefix-ext: prefix-state ≤ attacker-state
by (rule checked-staged-after-alpha-prefix-program-outcome-extension
  [OF - - after-out])
  (fact wf, fact controlled)
have root-eq:
  staged-trace-root data = fst prefix
and trace-roots-eq:
  staged-trace-fri-roots data = fst (snd prefix)
and trace-bs-eq:
  staged-trace-fri-challenges data = fst (snd (snd prefix))
and trace-final-eq:
  staged-trace-final data = snd (snd (snd prefix))
proof –
obtain fr trace-roots trace-bs trace-final where prefix-eq:
  prefix = (fr, trace-roots, trace-bs, trace-final)
by (cases prefix) simp
from after-out[unfolded prefix-eq]
obtain as s1 dg s2 s3 composition-roots composition-bs s4
  composition-final s5 s6 query-chunks where ret:
  Some (data, attacker-state) ∈
    set-dist
      (execute
        (return
          (|staged-trace-root = fr,
            staged-trace-fri-roots = trace-roots,
            staged-trace-fri-challenges = trace-bs,

```

```

    staged-trace-final = trace-final,
    staged-alphas = as,
    staged-degree = dg,
    staged-composition-fri-roots = composition-roots,
    staged-composition-fri-challenges = composition-bs,
    staged-composition-final = composition-final,
    staged-query-chunks = query-chunks))
s6)
unfolding checked-staged-after-alpha-prefix-program-def Let-def
by (auto elim!: set-dist-bindE split: prod.splits)
have data-eq:
  data =
    (|staged-trace-root = fr,
     staged-trace-fri-roots = trace-roots,
     staged-trace-fri-challenges = trace-bs,
     staged-trace-final = trace-final,
     staged-alphas = as,
     staged-degree = dg,
     staged-composition-fri-roots = composition-roots,
     staged-composition-fri-challenges = composition-bs,
     staged-composition-final = composition-final,
     staged-query-chunks = query-chunks)
  using ret by simp
show staged-trace-root data = fst prefix
  unfolding prefix-eq data-eq by simp
show staged-trace-fri-roots data = fst (snd prefix)
  unfolding prefix-eq data-eq by simp
show staged-trace-fri-challenges data = fst (snd (snd prefix))
  unfolding prefix-eq data-eq by simp
show staged-trace-final data = snd (snd (snd prefix))
  unfolding prefix-eq data-eq by simp
qed
show ?thesis
  by (rule that[OF prefix-out root-eq trace-roots-eq trace-bs-eq
    trace-final-eq prefix-ext])
qed

lemma checked-staged-transcript-program-alpha-prefix-support:
assumes outcome:
  Some (data, attacker-state) ∈
    set-dist
      (execute (checked-staged-transcript-program A)
        adversary-initial-state)
obtains prefix prefix-state
where
  Some (prefix, prefix-state) ∈
    set-dist
      (execute (staged-alpha-prefix-program A)
        adversary-initial-state)

```

```

    staged-trace-root data = fst prefix
    staged-trace-fri-roots data = fst (snd prefix)
    staged-trace-fri-challenges data = fst (snd (snd prefix))
    staged-trace-final data = snd (snd (snd prefix))
proof –
  have staged-out:
    Some (data, attacker-state) ∈
      set-dist
        (execute (staged-transcript-program A) adversary-initial-state)
  by (rule checked-staged-transcript-program-outcome-imp-staged-transcript-program
    [OF outcome])
  then obtain prefix prefix-state where prefix-out:
    Some (prefix, prefix-state) ∈
      set-dist
        (execute (staged-alpha-prefix-program A)
          adversary-initial-state)
  and after-out:
    Some (data, attacker-state) ∈
      set-dist
        (execute (staged-after-alpha-prefix-program A prefix)
          prefix-state)
  unfolding staged-transcript-program-alpha-prefix-decomp
  by (auto elim!: set-dist-bindE)
  have root-eq:
    staged-trace-root data = fst prefix
  and trace-roots-eq:
    staged-trace-fri-roots data = fst (snd prefix)
  and trace-bs-eq:
    staged-trace-fri-challenges data = fst (snd (snd prefix))
  and trace-final-eq:
    staged-trace-final data = snd (snd (snd prefix))
  proof –
  obtain fr trace-roots trace-bs trace-final where prefix-eq:
    prefix = (fr, trace-roots, trace-bs, trace-final)
  by (cases prefix) simp
  from after-out[unfolded prefix-eq]
  obtain as s1 dg s2 s3 composition-roots composition-bs s4
    composition-final s5 s6 query-chunks where ret:
    Some (data, attacker-state) ∈
      set-dist
        (execute
          (return
            (|staged-trace-root = fr,
              staged-trace-fri-roots = trace-roots,
              staged-trace-fri-challenges = trace-bs,
              staged-trace-final = trace-final,
              staged-alphas = as,
              staged-degree = dg,
              staged-composition-fri-roots = composition-roots,

```

```

    staged-composition-fri-challenges = composition-bs,
    staged-composition-final = composition-final,
    staged-query-chunks = query-chunks))
s6)
unfolding staged-after-alpha-prefix-program-def Let-def
by (auto elim!: set-dist-bindE split: prod.splits)
have data-eq:
  data =
    (|staged-trace-root = fr,
     staged-trace-fri-roots = trace-roots,
     staged-trace-fri-challenges = trace-bs,
     staged-trace-final = trace-final,
     staged-alphas = as,
     staged-degree = dg,
     staged-composition-fri-roots = composition-roots,
     staged-composition-fri-challenges = composition-bs,
     staged-composition-final = composition-final,
     staged-query-chunks = query-chunks)
  using ret by simp
show staged-trace-root data = fst prefix
  unfolding prefix-eq data-eq by simp
show staged-trace-fri-roots data = fst (snd prefix)
  unfolding prefix-eq data-eq by simp
show staged-trace-fri-challenges data = fst (snd (snd prefix))
  unfolding prefix-eq data-eq by simp-all
show staged-trace-final data = snd (snd (snd prefix))
  unfolding prefix-eq data-eq by simp
qed
show ?thesis
  by (rule that[OF prefix-out root-eq trace-roots-eq trace-bs-eq
    trace-final-eq])
qed

lemma checked-staged-transcript-program-alpha-prefix-match-support:
assumes wf: staged-budget-wellformed budgets
and controlled: staged-adversary-controlled budgets A
and outcome:
  Some (data, attacker-state) ∈
    set-dist
      (execute (checked-staged-transcript-program A)
        adversary-initial-state)
obtains prefix prefix-state
where
  Some (prefix, prefix-state) ∈
    set-dist
      (execute (staged-alpha-prefix-program A)
        adversary-initial-state)
  staged-trace-root data = fst prefix
  staged-trace-fri-roots data = fst (snd prefix)

```

staged-trace-fri-challenges data = fst (snd (snd prefix))
staged-trace-final data = snd (snd (snd prefix))
prefix-state ≤ attacker-state
proof –
have *staged-out*:
Some (data, attacker-state) ∈
set-dist
(execute (staged-transcript-program A) adversary-initial-state)
by (*rule checked-staged-transcript-program-outcome-imp-staged-transcript-program*
[OF outcome])
then obtain *prefix prefix-state* **where** *prefix-out*:
Some (prefix, prefix-state) ∈
set-dist
(execute (staged-alpha-prefix-program A)
adversary-initial-state)
and *after-out*:
Some (data, attacker-state) ∈
set-dist
(execute (staged-after-alpha-prefix-program A prefix)
prefix-state)
unfolding *staged-transcript-program-alpha-prefix-decomp*
by (*auto elim! set-dist-bindE*)
have *prefix-ext*: *prefix-state ≤ attacker-state*
by (*rule staged-after-alpha-prefix-program-outcome-extension*
[OF wf controlled after-out])
have *root-eq*:
staged-trace-root data = fst prefix
and *trace-roots-eq*:
staged-trace-fri-roots data = fst (snd prefix)
and *trace-bs-eq*:
staged-trace-fri-challenges data = fst (snd (snd prefix))
and *trace-final-eq*:
staged-trace-final data = snd (snd (snd prefix))
proof –
obtain *fr trace-roots trace-bs trace-final* **where** *prefix-eq*:
prefix = (fr, trace-roots, trace-bs, trace-final)
by (*cases prefix simp*)
from *after-out[unfolded prefix-eq]*
obtain *as s1 dg s2 s3 composition-roots composition-bs s4*
composition-final s5 s6 query-chunks **where** *ret*:
Some (data, attacker-state) ∈
set-dist
(execute
(return
(|staged-trace-root = fr,
staged-trace-fri-roots = trace-roots,
staged-trace-fri-challenges = trace-bs,
staged-trace-final = trace-final,
staged-alphas = as,

```

    staged-degree = dg,
    staged-composition-fri-roots = composition-roots,
    staged-composition-fri-challenges = composition-bs,
    staged-composition-final = composition-final,
    staged-query-chunks = query-chunks))
s6)
unfolding staged-after-alpha-prefix-program-def Let-def
by (auto elim!: set-dist-bindE split: prod.splits)
have data-eq:
  data =
    (|staged-trace-root = fr,
     staged-trace-fri-roots = trace-roots,
     staged-trace-fri-challenges = trace-bs,
     staged-trace-final = trace-final,
     staged-alphas = as,
     staged-degree = dg,
     staged-composition-fri-roots = composition-roots,
     staged-composition-fri-challenges = composition-bs,
     staged-composition-final = composition-final,
     staged-query-chunks = query-chunks)
  using ret by simp
show staged-trace-root data = fst prefix
  unfolding prefix-eq data-eq by simp
show staged-trace-fri-roots data = fst (snd prefix)
  unfolding prefix-eq data-eq by simp
show staged-trace-fri-challenges data = fst (snd (snd prefix))
  unfolding prefix-eq data-eq by simp
show staged-trace-final data = snd (snd (snd prefix))
  unfolding prefix-eq data-eq by simp
qed
show ?thesis
  by (rule that[OF prefix-out root-eq trace-roots-eq trace-bs-eq
    trace-final-eq prefix-ext])
qed

lemma checked-staged-security-with-data-state-alpha-prefix-tree-output-hitI:
assumes prefix-out:
  Some (prefix, prefix-state) ∈
    set-dist
      (execute (staged-alpha-prefix-program A)
        adversary-initial-state)
and root-eq: staged-trace-root data = fst prefix
and trace-roots-eq:
  staged-trace-fri-roots data = fst (snd prefix)
and trace-bs-eq:
  staged-trace-fri-challenges data = fst (snd (snd prefix))
and trace-final-eq:
  staged-trace-final data = snd (snd (snd prefix))
and verify-out:

```



```

Some (result, final-state) ∈
  set-dist
    (execute verify-monad
      (verifier-state-from-adversary attacker-state
        (staged-proof-transcript data)))
and bound:
  accepted-with-bound-tables
    (verifier-state-from-adversary attacker-state
      (staged-proof-transcript data))
    (Some (result, final-state)) trace-table composition-table as
      query-idxs
and header:
  verifier-header-transcript
    (verifier-state-from-adversary attacker-state
      (staged-proof-transcript data))
    (staged-trace-root data)
    (staged-trace-fri-roots data)
    (staged-trace-final data)
    as dg composition-fri-roots final rest
and trace-created: created-tree trace-table trace-tree final-state
and composition-created:
  created-tree composition-table composition-tree final-state
and tree-hit:
  hash-map-new-output-hit
    (set-tree trace-tree ∪ set-tree composition-tree)
    prefix-state final-state
shows
  checked-staged-security-with-data-state-alpha-prefix-tree-output-hit A
    (Some (((data, attacker-state), checked-result), checked-final-state))
proof –
have prefix-tuple:
  prefix =
    (staged-trace-root data, staged-trace-fri-roots data,
      staged-trace-fri-challenges data, staged-trace-final data)
  by (cases prefix; cases snd prefix; cases snd (snd prefix))
    (use root-eq trace-roots-eq trace-bs-eq trace-final-eq in auto)
show ?thesis
  unfolding checked-staged-security-with-data-state-alpha-prefix-tree-output-hit-def
    Let-def
  apply (simp add: prefix-tuple)
  apply (rule exI[where x=prefix-state])
  apply (rule conjI)
  using prefix-out prefix-tuple
  apply simp
  apply (rule exI[where x=result])
  apply (rule exI[where x=final-state])
  apply (rule conjI)
  using verify-out
  apply simp

```

```

apply (rule exI[where  $x=trace-table$ ])
apply (rule exI[where  $x=composition-table$ ])
apply (rule exI[where  $x=as$ ])
apply (rule conjI)
  apply (rule exI[where  $x=query-idxs$ ])
  using bound
  apply simp
apply (rule conjI)
apply (rule exI[where  $x=dg$ ])
apply (rule exI[where  $x=composition-fri-roots$ ])
apply (rule exI[where  $x=final$ ])
apply (rule exI[where  $x=rest$ ])
  using header
  apply simp
apply (rule exI[where  $x=trace-tree$ ])
apply (rule conjI)
  using trace-created
  apply simp
apply (rule exI[where  $x=composition-tree$ ])
using composition-created tree-hit
apply simp
done
qed

lemma checked-staged-security-with-data-state-alpha-header-supported-bad-set-hit-imp-prefix-or-tree-output-hit-on-su
  assumes wf: staged-budget-wellformed budgets
  and controlled: staged-adversary-controlled budgets A
  and support:
     $out \in$ 
    set-dist
    (execute (checked-staged-security-experiment-with-data-state A)
      adversary-initial-state)
  and hit:
    checked-staged-security-with-data-state-alpha-header-supported-bad-set-hit
    bad-sets out
  shows
    checked-staged-security-with-data-state-alpha-prefix-bad-set-hit A
    bad-sets out  $\vee$ 
    checked-staged-security-with-data-state-alpha-prefix-tree-output-hit A
    out
proof (cases out)
  case None
  then show ?thesis
    using hit
    unfolding
      checked-staged-security-with-data-state-alpha-header-supported-bad-set-hit-def
    by simp
next
  case (Some result-pack)

```

```

then obtain data attacker-state checked-result checked-final-state where
  out-eq:
    out =
      Some (((data, attacker-state), checked-result),
            checked-final-state)
  by (cases result-pack, auto split: prod.splits)
let ?s =
  verifier-state-from-adversary attacker-state
  (staged-proof-transcript data)
have support-some:
  Some (((data, attacker-state), checked-result),
        checked-final-state)  $\in$ 
  set-dist
    (execute (checked-staged-security-experiment-with-data-state A)
             adversary-initial-state)
  using support unfolding out-eq .
from checked-staged-security-experiment-with-data-state-outcomeE
  [OF support-some]
have builder:
  Some (data, attacker-state)  $\in$ 
  set-dist
    (execute (checked-staged-transcript-program A)
             adversary-initial-state)
  and checked-verify:
  Some (checked-result, checked-final-state)  $\in$ 
  set-dist (execute verify-monad ?s)
  by blast+
from checked-staged-transcript-program-alpha-prefix-match-support
  [OF wf controlled builder]
obtain prefix prefix-state where prefix-out:
  Some (prefix, prefix-state)  $\in$ 
  set-dist
    (execute (staged-alpha-prefix-program A)
             adversary-initial-state)
  and root-eq: staged-trace-root data = fst prefix
  and trace-roots-eq:
    staged-trace-fri-roots data = fst (snd prefix)
  and trace-bs-eq:
    staged-trace-fri-challenges data = fst (snd (snd prefix))
  and trace-final-eq:
    staged-trace-final data = snd (snd (snd prefix))
  and prefix-ext: prefix-state  $\leq$  attacker-state
  by blast
have prefix-tuple:
  prefix =
    (staged-trace-root data, staged-trace-fri-roots data,
     staged-trace-fri-challenges data, staged-trace-final data)
  by (cases prefix; cases snd prefix; cases snd (snd prefix))
    (use root-eq trace-roots-eq trace-bs-eq trace-final-eq in auto)

```

```

have union-bad:
  staged-alphas data ∈
    alpha-header-supported-union-bad-sets ?s bad-sets
    (staged-trace-root data)
    (staged-trace-fri-roots data)
    (staged-trace-final data)
  using hit unfolding out-eq
    checked-staged-security-with-data-state-alpha-header-supported-bad-set-hit-def
  by simp
then have alphas-space: staged-alphas data ∈ alpha-space
  unfolding alpha-header-supported-union-bad-sets-def by auto
from union-bad obtain trace-table where header-candidate:
  trace-table ∈
    alpha-header-supported-trace-table-candidates ?s
    (staged-trace-root data)
    (staged-trace-fri-roots data)
    (staged-trace-final data)
and bad:
  staged-alphas data ∈ bad-sets trace-table
  unfolding alpha-header-supported-union-bad-sets-def by blast
from header-candidate obtain candidate-out composition-table as query-idxs
  dg composition-fri-roots final rest where candidate-out:
  candidate-out ∈ set-dist (execute verify-monad ?s)
and bound:
  accepted-with-bound-tables ?s candidate-out trace-table
  composition-table as query-idxs
and candidate-header:
  verifier-header-transcript ?s
  (staged-trace-root data)
  (staged-trace-fri-roots data)
  (staged-trace-final data)
  as dg composition-fri-roots final rest
  unfolding alpha-header-supported-trace-table-candidates-def by blast
from bound obtain result final-state where candidate-out-eq:
  candidate-out = Some (result, final-state)
  unfolding accepted-with-bound-tables-def by blast
have verifier-out:
  Some (result, final-state) ∈ set-dist (execute verify-monad ?s)
  using candidate-out candidate-out-eq by simp
have bound-some:
  accepted-with-bound-tables ?s (Some (result, final-state))
  trace-table composition-table as query-idxs
  using bound candidate-out-eq by simp
have prefix-s: prefix-state ≤ ?s
  by (rule
    alpha-prefix-hash-extends-verifier-state-from-adversary-right
    [OF prefix-ext])
have s-final: ?s ≤ final-state
  by (rule verify-monad-hash-extends[OF verifier-out])

```

```

have prefix-final: prefix-state ≤ final-state
  by (rule hash-ext-trans[OF prefix-s s-final])
show ?thesis
proof (rule
  accepted-with-bound-tables-initial-roots-pullback-or-new-tree-output-hit
  [OF prefix-final bound-some])
fix fr f-fri-roots f-final dg' composition-fri-roots' final'
  rest' trace-tree composition-tree
assume pull-header:
  verifier-header-transcript ?s fr f-fri-roots f-final as dg'
  composition-fri-roots' final' rest'
and trace-created:
  created-tree trace-table trace-tree final-state
and composition-created:
  created-tree composition-table composition-tree final-state
and split:
  (merkle-root-binds-table fr trace-table prefix-state ∧
   merkle-root-binds-table (hd composition-fri-roots')
   composition-table prefix-state) ∨
  hash-map-new-output-hit
  (set-tree trace-tree ∪ set-tree composition-tree)
  prefix-state final-state
have pull-header-eq:
  fr = staged-trace-root data ∧
  f-fri-roots = staged-trace-fri-roots data ∧
  f-final = staged-trace-final data ∧
  dg' = dg ∧
  composition-fri-roots' = composition-fri-roots ∧
  final' = final ∧
  rest' = rest
using verifier-header-transcript-unique[OF candidate-header pull-header]
by simp
then have fr-eq: fr = staged-trace-root data
by simp
from split show ?thesis
proof
assume binds:
  merkle-root-binds-table fr trace-table prefix-state ∧
  merkle-root-binds-table (hd composition-fri-roots')
  composition-table prefix-state
have trace-len: length trace-table = clength * scale
  using bound-some unfolding accepted-with-bound-tables-def
  accepted-with-tables-def by simp
have prefix-candidate:
  trace-table ∈
  alpha-prefix-trace-table-candidates prefix-state
  (staged-trace-root data)
by (rule alpha-prefix-trace-table-candidateI[OF trace-len])
  (use binds fr-eq in simp)

```

```

have prefix-bad:
  staged-alphas data  $\in$ 
    alpha-prefix-union-bad-sets prefix-state bad-sets
      (staged-trace-root data)
  unfolding alpha-prefix-union-bad-sets-def
  using alphas-space prefix-candidate bad by auto
have
  checked-staged-security-with-data-state-alpha-prefix-bad-set-hit A
    bad-sets out
  unfolding out-eq
    checked-staged-security-with-data-state-alpha-prefix-bad-set-hit-def
  apply simp
  apply (rule exI[where  $x = \text{prefix-state}$ ])
  using prefix-out prefix-tuple prefix-bad
  apply simp
  done
then show ?thesis by simp
next
assume tree-hit:
  hash-map-new-output-hit
    (set-tree trace-tree  $\cup$  set-tree composition-tree)
  prefix-state final-state
have
  checked-staged-security-with-data-state-alpha-prefix-tree-output-hit A
    out
  unfolding out-eq
  by (rule
    checked-staged-security-with-data-state-alpha-prefix-tree-output-hitI
    [OF prefix-out root-eq trace-roots-eq trace-bs-eq trace-final-eq
      verifier-out bound-some candidate-header trace-created
      composition-created tree-hit])
  then show ?thesis by simp
qed
qed
qed

lemma checked-staged-security-with-actual-alpha-prefix-header-supported-bad-set-hit-imp-prefix-or-tree-output-hit-on-
assumes wf: staged-budget-wellformed budgets
and controlled: staged-adversary-controlled budgets A
and support:
  out  $\in$ 
    set-dist
      (execute
        (checked-staged-security-experiment-with-actual-alpha-prefix-data-state
          A)
        adversary-initial-state)
and hit:
  checked-staged-security-with-actual-alpha-prefix-header-supported-bad-set-hit
    bad-sets out

```

```

shows
  checked-staged-security-with-actual-alpha-prefix-bad-set-hit
    bad-sets out  $\vee$ 
    checked-staged-security-with-actual-alpha-prefix-tree-output-hit out
proof (cases out)
case None
then show ?thesis
  using hit
  unfolding
    checked-staged-security-with-actual-alpha-prefix-header-supported-bad-set-hit-def
  by simp
next
case (Some result-pack)
then obtain prefix prefix-state data attacker-state checked-result
  checked-final-state where out-eq:
    out =
      Some (((((prefix, prefix-state), data), attacker-state),
        checked-result), checked-final-state)
  by (cases result-pack, auto split: prod.splits)
let ?s =
  verifier-state-from-adversary attacker-state
  (staged-proof-transcript data)
have support-some:
  Some (((((prefix, prefix-state), data), attacker-state),
    checked-result), checked-final-state)  $\in$ 
  set-dist
    (execute
      (checked-staged-security-experiment-with-actual-alpha-prefix-data-state
        A)
      adversary-initial-state)
  using support unfolding out-eq .
from support-some obtain trans-out where trans-out:
  Some (((prefix, prefix-state), data), attacker-state)  $\in$ 
  set-dist
    (execute (checked-staged-transcript-with-alpha-prefix-program A)
      adversary-initial-state)
and checked-verify:
  Some (checked-result, checked-final-state)  $\in$ 
  set-dist (execute verify-monad ?s)
unfolding
  checked-staged-security-experiment-with-actual-alpha-prefix-data-state-def
by (auto elim!: set-dist-bindE split: prod.splits)
from checked-staged-transcript-with-alpha-prefix-program-support
  [OF wf controlled trans-out]
have prefix-out:
  Some (prefix, prefix-state)  $\in$ 
  set-dist
    (execute (staged-alpha-prefix-program A)
      adversary-initial-state)

```

```

and root-eq: staged-trace-root data = fst prefix
and trace-roots-eq:
  staged-trace-fri-roots data = fst (snd prefix)
and trace-bs-eq:
  staged-trace-fri-challenges data = fst (snd (snd prefix))
and trace-final-eq:
  staged-trace-final data = snd (snd (snd prefix))
and prefix-ext: prefix-state  $\leq$  attacker-state
by blast+
have union-bad:
  staged-alphas data  $\in$ 
    alpha-header-supported-union-bad-sets ?s bad-sets
    (staged-trace-root data)
    (staged-trace-fri-roots data)
    (staged-trace-final data)
using hit unfolding out-eq
  checked-staged-security-with-actual-alpha-prefix-header-supported-bad-set-hit-def
  Let-def
by simp
then have alphas-space: staged-alphas data  $\in$  alpha-space
  unfolding alpha-header-supported-union-bad-sets-def by auto
from union-bad obtain trace-table where header-candidate:
  trace-table  $\in$ 
    alpha-header-supported-trace-table-candidates ?s
    (staged-trace-root data)
    (staged-trace-fri-roots data)
    (staged-trace-final data)
and bad:
  staged-alphas data  $\in$  bad-sets trace-table
  unfolding alpha-header-supported-union-bad-sets-def by blast
from header-candidate obtain candidate-out composition-table as query-idxs
  dg composition-fri-roots final rest where candidate-out:
  candidate-out  $\in$  set-dist (execute verify-monad ?s)
and bound:
  accepted-with-bound-tables ?s candidate-out trace-table
  composition-table as query-idxs
and candidate-header:
  verifier-header-transcript ?s
  (staged-trace-root data)
  (staged-trace-fri-roots data)
  (staged-trace-final data)
  as dg composition-fri-roots final rest
  unfolding alpha-header-supported-trace-table-candidates-def by blast
from bound obtain result final-state where candidate-out-eq:
  candidate-out = Some (result, final-state)
  unfolding accepted-with-bound-tables-def by blast
have verifier-out:
  Some (result, final-state)  $\in$  set-dist (execute verify-monad ?s)
  using candidate-out candidate-out-eq by simp

```



```

have bound-some:
  accepted-with-bound-tables ?s (Some (result, final-state))
  trace-table composition-table as query-idxs
  using bound candidate-out-eq by simp
have prefix-s: prefix-state  $\leq$  ?s
  by (rule
    alpha-prefix-hash-extends-verifier-state-from-adversary-right
    [OF prefix-ext])
have s-final: ?s  $\leq$  final-state
  by (rule verify-monad-hash-extends[OF verifier-out])
have prefix-final: prefix-state  $\leq$  final-state
  by (rule hash-ext-trans[OF prefix-s s-final])
show ?thesis
proof (rule
  accepted-with-bound-tables-initial-roots-pullback-or-new-tree-output-hit
  [OF prefix-final bound-some])
  fix fr f-fri-roots f-final dg' composition-fri-roots' final'
  rest' trace-tree composition-tree
  assume pull-header:
    verifier-header-transcript ?s fr f-fri-roots f-final as dg'
    composition-fri-roots' final' rest'
  and trace-created:
    created-tree trace-table trace-tree final-state
  and composition-created:
    created-tree composition-table composition-tree final-state
  and split:
    (merkle-root-binds-table fr trace-table prefix-state  $\wedge$ 
      merkle-root-binds-table (hd composition-fri-roots')
        composition-table prefix-state)  $\vee$ 
    hash-map-new-output-hit
      (set-tree trace-tree  $\cup$  set-tree composition-tree)
      prefix-state final-state
  have pull-header-eq:
    fr = staged-trace-root data  $\wedge$ 
    f-fri-roots = staged-trace-fri-roots data  $\wedge$ 
    f-final = staged-trace-final data  $\wedge$ 
    dg' = dg  $\wedge$ 
    composition-fri-roots' = composition-fri-roots  $\wedge$ 
    final' = final  $\wedge$ 
    rest' = rest
  using verifier-header-transcript-unique[OF candidate-header pull-header]
  by simp
  then have fr-eq: fr = staged-trace-root data
  by simp
  from split show ?thesis
proof
  assume binds:
    merkle-root-binds-table fr trace-table prefix-state  $\wedge$ 
    merkle-root-binds-table (hd composition-fri-roots')

```

```

    composition-table prefix-state
  have trace-len: length trace-table = clength * scale
    using bound-some unfolding accepted-with-bound-tables-def
    accepted-with-tables-def by simp
  have prefix-candidate:
    trace-table ∈
      alpha-prefix-trace-table-candidates prefix-state (fst prefix)
    by (rule alpha-prefix-trace-table-candidateI[OF trace-len])
      (use binds fr-eq root-eq in simp)
  have prefix-bad:
    staged-alphas data ∈
      alpha-prefix-union-bad-sets prefix-state bad-sets (fst prefix)
    unfolding alpha-prefix-union-bad-sets-def
    using alphas-space prefix-candidate bad by auto
  have
    checked-staged-security-with-actual-alpha-prefix-bad-set-hit
      bad-sets out
    unfolding out-eq
      checked-staged-security-with-actual-alpha-prefix-bad-set-hit-def
      Let-def
    using prefix-bad
    by simp
  then show ?thesis by simp
next
  assume tree-hit:
    hash-map-new-output-hit
      (set-tree trace-tree ∪ set-tree composition-tree)
    prefix-state final-state
  have
    checked-staged-security-with-actual-alpha-prefix-tree-output-hit
      out
    unfolding out-eq
    by (rule
      checked-staged-security-with-actual-alpha-prefix-tree-output-hitI
      [OF verifier-out bound-some candidate-header trace-created
        composition-created tree-hit])
  then show ?thesis by simp
qed
qed
qed

lemma checked-staged-security-with-data-state-alpha-header-supported-bad-set-hit-bound-from-actual-prefix-and-tree:
  assumes wf: staged-budget-wellformed budgets
  and controlled: staged-adversary-controlled budgets A
  and prefix-bound:
    wp-event
      (checked-staged-security-experiment-with-actual-alpha-prefix-data-state
        A)
    (checked-staged-security-with-actual-alpha-prefix-bad-set-hit

```

```

    bad-sets)
    adversary-initial-state  $\leq P$ 
and tree-bound:
    wp-event
    (checked-staged-security-experiment-with-actual-alpha-prefix-data-state
     A)
    checked-staged-security-with-actual-alpha-prefix-tree-output-hit
    adversary-initial-state  $\leq T$ 
shows
    wp-event (checked-staged-security-experiment-with-data-state A)
    (checked-staged-security-with-data-state-alpha-header-supported-bad-set-hit
     bad-sets)
    adversary-initial-state  $\leq P + T$ 
proof –
    let ?M =
    checked-staged-security-experiment-with-actual-alpha-prefix-data-state A
    let ?Prefix =
    checked-staged-security-with-actual-alpha-prefix-bad-set-hit bad-sets
    let ?Tree = checked-staged-security-with-actual-alpha-prefix-tree-output-hit
    let ?Header =
    checked-staged-security-with-actual-alpha-prefix-header-supported-bad-set-hit
    bad-sets
    have wp-event (checked-staged-security-experiment-with-data-state A)
    (checked-staged-security-with-data-state-alpha-header-supported-bad-set-hit
     bad-sets)
    adversary-initial-state =
    wp-event ?M ?Header adversary-initial-state
    by (rule
    checked-staged-security-with-data-state-alpha-header-supported-bad-set-hit-actual-projection)
    also have ...  $\leq$  wp-event ?M ( $\lambda$ out. ?Prefix out  $\vee$  ?Tree out)
    adversary-initial-state
    proof (rule wp-event-mono-on-support)
    fix out
    assume support: out  $\in$  set-dist (execute ?M adversary-initial-state)
    and hit: ?Header out
    show ?Prefix out  $\vee$  ?Tree out
    by (rule
    checked-staged-security-with-actual-alpha-prefix-header-supported-bad-set-hit-imp-prefix-or-tree-output-hit-on-su
    [OF wf controlled support hit])
    qed
    also have ...  $\leq$ 
    wp-event ?M ?Prefix adversary-initial-state +
    wp-event ?M ?Tree adversary-initial-state
    by (rule wp-event-union-bound)
    also have ...  $\leq P + T$ 
    by (rule add-mono[OF prefix-bound tree-bound])
    finally show ?thesis .
qed

```

lemma *checked-staged-security-with-data-state-alpha-header-supported-bad-set-hit-bound-from-prefix-and-tree:*

assumes *wf: staged-budget-wellformed budgets*

and *controlled: staged-adversary-controlled budgets A*

and *prefix-bound:*

wp-event (checked-staged-security-experiment-with-data-state A)
(checked-staged-security-with-data-state-alpha-prefix-bad-set-hit A
bad-sets)
adversary-initial-state ≤ P

and *tree-bound:*

wp-event (checked-staged-security-experiment-with-data-state A)
(checked-staged-security-with-data-state-alpha-prefix-tree-output-hit A)
adversary-initial-state ≤ T

shows

wp-event (checked-staged-security-experiment-with-data-state A)
(checked-staged-security-with-data-state-alpha-header-supported-bad-set-hit
bad-sets)
adversary-initial-state ≤ P + T

proof –

let *?M = checked-staged-security-experiment-with-data-state A*

let *?Prefix =*
checked-staged-security-with-data-state-alpha-prefix-bad-set-hit A
bad-sets

let *?Tree =*
checked-staged-security-with-data-state-alpha-prefix-tree-output-hit A

let *?Header =*
checked-staged-security-with-data-state-alpha-header-supported-bad-set-hit
bad-sets

have *wp-event ?M ?Header adversary-initial-state ≤*
wp-event ?M (λout. ?Prefix out ∨ ?Tree out)
adversary-initial-state

proof (rule *wp-event-mono-on-support*)

fix *out*

assume *support:*

out ∈ set-dist (execute ?M adversary-initial-state)

and *hit: ?Header out*

show *?Prefix out ∨ ?Tree out*

by (rule
checked-staged-security-with-data-state-alpha-header-supported-bad-set-hit-imp-prefix-or-tree-output-hit-on-supp
[OF wf controlled support hit])

qed

also have *... ≤*
wp-event ?M ?Prefix adversary-initial-state +
wp-event ?M ?Tree adversary-initial-state

by (rule *wp-event-union-bound*)

also have *... ≤ P + T*

by (rule *add-mono[OF prefix-bound tree-bound]*)

finally show *?thesis .*

qed

lemma *checked-staged-security-with-data-state-composition-bad-bound-from-alpha-prefix-and-tree:*

assumes *wf: staged-budget-wellformed budgets*

and *controlled: staged-adversary-controlled budgets A*

and *prefix-bound:*

wp-event (checked-staged-security-experiment-with-data-state A)
(checked-staged-security-with-data-state-alpha-prefix-bad-set-hit A
composition-trace-bad-alpha-space)
adversary-initial-state ≤ P

and *tree-bound:*

wp-event (checked-staged-security-experiment-with-data-state A)
(checked-staged-security-with-data-state-alpha-prefix-tree-output-hit A)
adversary-initial-state ≤ T

shows

wp-event (checked-staged-security-experiment-with-data-state A)
(staged-security-with-data-state-verifier-event composition-bad)
adversary-initial-state ≤ P + T

proof –

have *alpha-bound:*

wp-event (checked-staged-security-experiment-with-data-state A)
(checked-staged-security-with-data-state-alpha-header-supported-bad-set-hit
composition-trace-bad-alpha-space)
adversary-initial-state ≤ P + T

by (rule

checked-staged-security-with-data-state-alpha-header-supported-bad-set-hit-bound-from-prefix-and-tree
[OF wf controlled prefix-bound tree-bound])

show *?thesis*

by (rule

checked-staged-security-with-data-state-composition-bad-bound-from-alpha-header-supported-bad-set-general
[OF wf controlled alpha-bound])

qed

lemma *checked-staged-security-with-data-state-composition-bad-bound-from-actual-alpha-prefix-and-tree:*

assumes *wf: staged-budget-wellformed budgets*

and *controlled: staged-adversary-controlled budgets A*

and *prefix-bound:*

wp-event
(checked-staged-security-experiment-with-actual-alpha-prefix-data-state
A)
(checked-staged-security-with-actual-alpha-prefix-bad-set-hit
composition-trace-bad-alpha-space)
adversary-initial-state ≤ P

and *tree-bound:*

wp-event
(checked-staged-security-experiment-with-actual-alpha-prefix-data-state
A)
checked-staged-security-with-actual-alpha-prefix-tree-output-hit
adversary-initial-state ≤ T

shows

wp-event (checked-staged-security-experiment-with-data-state A)

```

    (staged-security-with-data-state-verifier-event composition-bad)
    adversary-initial-state  $\leq P + T$ 
proof –
  have alpha-bound:
    wp-event (checked-staged-security-experiment-with-data-state A)
      (checked-staged-security-with-data-state-alpha-header-supported-bad-set-hit
        composition-trace-bad-alpha-space)
    adversary-initial-state  $\leq P + T$ 
  by (rule
    checked-staged-security-with-data-state-alpha-header-supported-bad-set-hit-bound-from-actual-prefix-and-tree
    [OF wf controlled prefix-bound tree-bound])
  show ?thesis
  by (rule
    checked-staged-security-with-data-state-composition-bad-bound-from-alpha-header-supported-bad-set-general
    [OF wf controlled alpha-bound])
qed

end

end

```

theory *Staged-Security-Experiment-Composition-Tree-Output*
imports *Staged-Security-Experiment-Composition*
begin

Bridges for the composition tree-output side event.

The actual-alpha-prefix experiment carries the concrete alpha-prefix state through the verifier run. This layer relates that path-specific event back to the ordinary checked data-state experiment, so later bounds can be proved once for the data-state event.

context *soundness*
begin

lemma *hash-map-new-output-hit-Un-iff*:
 hash-map-new-output-hit $(A \cup B) s t \longleftrightarrow$
 hash-map-new-output-hit $A s t \vee$ hash-map-new-output-hit $B s t$
unfolding hash-map-new-output-hit-def **by** blast

lemma *hash-new-output-hit-event-Un-iff*:
 hash-new-output-hit-event $(A \cup B) s out \longleftrightarrow$
 hash-new-output-hit-event $A s out \vee$
 hash-new-output-hit-event $B s out$
unfolding hash-new-output-hit-event-def
by (cases out) (auto simp: hash-map-new-output-hit-Un-iff)

lemma *hash-map-new-output-hit-tree-root-or-subtree*:
assumes hit: hash-map-new-output-hit (set-tree tree) $s t$
shows
 hash-map-new-output-hit {value tree} $s t \vee$

```

    (∃ l v r. tree = ⟨l, v, r⟩ ∧
      hash-map-new-output-hit (set-tree l ∪ set-tree r) s t)
using hit
unfolding hash-map-new-output-hit-def
by (cases tree) auto

lemma hash-map-new-output-hit-two-trees-root-or-subtree:
assumes hit:
  hash-map-new-output-hit (set-tree trace-tree ∪ set-tree composition-tree)
    s t
shows
  hash-map-new-output-hit {value trace-tree, value composition-tree} s t ∨
    (∃ l v r. trace-tree = ⟨l, v, r⟩ ∧
      hash-map-new-output-hit (set-tree l ∪ set-tree r) s t) ∨
    (∃ l v r. composition-tree = ⟨l, v, r⟩ ∧
      hash-map-new-output-hit (set-tree l ∪ set-tree r) s t)
proof -
from hit have split:
  hash-map-new-output-hit (set-tree trace-tree) s t ∨
  hash-map-new-output-hit (set-tree composition-tree) s t
unfolding hash-map-new-output-hit-Un-iff .
from split show ?thesis
proof
assume trace-hit: hash-map-new-output-hit (set-tree trace-tree) s t
from hash-map-new-output-hit-tree-root-or-subtree[OF trace-hit]
show ?thesis
proof
assume root-hit: hash-map-new-output-hit {value trace-tree} s t
have hash-map-new-output-hit
  {value trace-tree, value composition-tree} s t
by (rule hash-map-new-output-hit-subset[OF - root-hit]) auto
then show ?thesis by simp
next
assume ∃ l v r. trace-tree = ⟨l, v, r⟩ ∧
  hash-map-new-output-hit (set-tree l ∪ set-tree r) s t
then show ?thesis by blast
qed
next
assume composition-hit:
  hash-map-new-output-hit (set-tree composition-tree) s t
from hash-map-new-output-hit-tree-root-or-subtree[OF composition-hit]
show ?thesis
proof
assume root-hit:
  hash-map-new-output-hit {value composition-tree} s t
have hash-map-new-output-hit
  {value trace-tree, value composition-tree} s t
by (rule hash-map-new-output-hit-subset[OF - root-hit]) auto
then show ?thesis by simp

```

```

next
  assume  $\exists l\ v\ r. \text{composition-tree} = \langle l, v, r \rangle \wedge$ 
    hash-map-new-output-hit (set-tree  $l \cup \text{set-tree } r$ )  $s\ t$ 
  then show ?thesis by blast
qed
qed
qed

lemma card-set-tree-le-inorder-length:
  card (set-tree  $t$ )  $\leq$  length (inorder  $t$ )
  using card-length[of inorder  $t$ ]
  by simp

lemma protocol-created-tree-inorder-length-bound:
  assumes created: protocol-created-tree  $xs\ t\ s$ 
  shows length (inorder  $t$ )  $\leq 2 * \text{length } xs - 1$ 
  using created
proof (induction  $xs$  arbitrary:  $t$  rule: length-induct)
  case (1  $xs$ )
  show ?case
  proof (cases length  $xs = 0$ )
    case True
    then have  $xs\text{-eq}: xs = []$ 
      by simp
    then show ?thesis
      using 1.prem1
      unfolding protocol-created-tree-def
      by (simp add: protocol-merkle.created-tree.simps)
  next
  case False
  show ?thesis
  proof (cases length  $xs = 1$ )
    case True
    then obtain  $x$  where  $xs\text{-eq}: xs = [x]$ 
      by (cases  $xs$ ) auto
    from protocol-created-tree-singletonD[OF 1.prem1[unfolded  $xs\text{-eq}$ ]]
    obtain  $h$  where  $t\text{-eq}: t = \langle \langle \rangle, h, \langle \rangle \rangle$ 
      by blast
    then show ?thesis
      using  $xs\text{-eq}$  by simp
  next
  case False
  then have  $len\text{-ge}: 2 \leq \text{length } xs$ 
    using  $\langle \text{length } xs \neq 0 \rangle$  by linarith
  let ? $i$  = length  $xs \text{ div } 2$ 
  have  $i\text{-pos}: 0 < ?i$ 
    using  $len\text{-ge}$  by simp
  have  $i\text{-lt}: ?i < \text{length } xs$ 
    using  $len\text{-ge}$  by simp

```



```

from 1.prems obtain l h r where
  i-def: ?i = length xs div 2
  and
    t-eq: t = ⟨l, h, r⟩
  and left: protocol-created-tree (take ?i xs) l s
  and right: protocol-created-tree (drop ?i xs) r s
  using protocol-created-tree-internalD[OF 1.prems len-ge]
  by blast
have drop-lt: length (drop ?i xs) < length xs
  using i-pos by simp
have right-bound: length (inorder r) ≤
  2 * length (drop ?i xs) - 1
  using 1.IH[rule-format, OF drop-lt, of r] right by simp
have take-lt: length (take ?i xs) < length xs
  using i-lt by simp
have left-bound: length (inorder l) ≤
  2 * length (take ?i xs) - 1
  using 1.IH[rule-format, OF take-lt, of l] left by simp
have take-pos: 0 < length (take ?i xs)
  using i-pos i-lt by simp
have drop-pos: 0 < length (drop ?i xs)
  using i-lt by simp
have length (inorder t) =
  Suc (length (inorder l) + length (inorder r))
  using t-eq by simp
also have ... ≤
  Suc ((2 * length (take ?i xs) - 1) +
    (2 * length (drop ?i xs) - 1))
  using left-bound right-bound by simp
also have ... = 2 * length xs - 1
  using take-pos drop-pos i-lt by simp
finally show ?thesis .
qed
qed
qed

```

```

lemma created-tree-inorder-length-bound:
  assumes created: created-tree xs t s
  shows length (inorder t) ≤ 2 * length xs - 1
  using protocol-created-tree-inorder-length-bound[
    OF created[unfolded created-tree-def]]
  .

```

```

lemma created-tree-card-set-tree-bound:
  assumes created: created-tree xs t s
  shows card (set-tree t) ≤ 2 * length xs - 1
  by (rule order.trans[OF card-set-tree-le-inorder-length
    created-tree-inorder-length-bound[OF created]])

```

```

lemma protocol-merkle-created-tree-same-inputs-value-eq:
  assumes left: protocol-merkle.created-tree xs left-tree s
    and right: protocol-merkle.created-tree xs right-tree s
  shows value left-tree = value right-tree
  using assms
proof (induction xs arbitrary: left-tree right-tree rule: length-induct)
  case (1 xs)
  show ?case
  proof (cases xs)
    case Nil
    then show ?thesis
      using 1.premss
      by (simp add: protocol-merkle.created-tree.simps)
  next
  case (Cons x rest)
  have xs-cons: xs = x # rest
    using Cons by simp
  show ?thesis
  proof (cases rest)
    case Nil
    then obtain left-hash right-hash where
      left-tree:
        left-tree = ⟨⟨⟩, left-hash, ⟨⟩⟩
        fmlookup (HashMap s) x = Some left-hash
    and right-tree:
        right-tree = ⟨⟨⟩, right-hash, ⟨⟩⟩
        fmlookup (HashMap s) x = Some right-hash
    using 1.premss Cons Nil
    by (auto simp: protocol-merkle.created-tree.simps)
    then show ?thesis by simp
  next
  case (Cons y ys)
  have xs-eq: xs = x # y # ys
    using xs-cons Cons by simp
  let ?i = length xs div 2
  have len: 2 ≤ length xs
    using xs-eq by simp
  obtain ll lh lr i where i-def: i = ?i
    and left-tree-eq: left-tree = ⟨ll, lh, lr⟩
    and ll-created: protocol-merkle.created-tree (take i xs) ll s
    and lr-created: protocol-merkle.created-tree (drop i xs) lr s
    and lh-lookup:
      fmlookup (HashMap s) (MerkleNode (value ll) (value lr)) =
        Some lh
  using 1.premss(1) xs-eq
  by (auto simp: protocol-merkle.created-tree.simps Let-def)
  obtain rl rh rr j where j-def: j = ?i
    and right-tree-eq: right-tree = ⟨rl, rh, rr⟩
    and rl-created: protocol-merkle.created-tree (take j xs) rl s

```

```

and rr-created: protocol-merkle.created-tree (drop j xs) rr s
and rh-lookup:
  fmlookup (HashMap s) (MerkleNode (value rl) (value rr)) =
    Some rh
using 1.premis(2) xs-eq
by (auto simp: protocol-merkle.created-tree.simps Let-def)
have j-i: j = i
  using i-def j-def by simp
have i-pos: 0 < i
  using i-def len by simp
have i-lt: i < length xs
  using i-def len by simp
have take-less: length (take i xs) < length xs
  using i-lt by simp
have drop-less: length (drop i xs) < length xs
  using i-pos i-lt by simp
have left-value: value ll = value rl
  by (rule 1.IH[rule-format, OF take-less ll-created])
    (use rl-created j-i in simp)
have right-value: value lr = value rr
  by (rule 1.IH[rule-format, OF drop-less lr-created])
    (use rr-created j-i in simp)
have lh = rh
  using lh-lookup rh-lookup left-value right-value by simp
then show ?thesis
  using left-tree-eq right-tree-eq by simp
qed
qed
qed

```

```

lemma protocol-created-tree-same-table-value-eq:
  assumes left: protocol-created-tree xs left-tree s
  and right: protocol-created-tree xs right-tree s
  shows value left-tree = value right-tree
  by (rule protocol-merkle-created-tree-same-inputs-value-eq
    [OF left[unfolded protocol-created-tree-def]
      right[unfolded protocol-created-tree-def]])

```

```

lemma created-tree-same-table-value-eq:
  assumes left: created-tree xs left-tree s
  and right: created-tree xs right-tree s
  shows value left-tree = value right-tree
  by (rule protocol-created-tree-same-table-value-eq
    [OF left[unfolded created-tree-def] right[unfolded created-tree-def]])

```

```

definition merkle-node-child-output-relation
  :: 'f protocol-hash-input  $\Rightarrow$  'f  $\Rightarrow$  bool
  where
    merkle-node-child-output-relation x y  $\longleftrightarrow$ 

```

$$(\exists z. x = \text{MerkleNode } y \ z \vee x = \text{MerkleNode } z \ y)$$

lemma *merkle-node-child-output-relation-values-subset:*

$\{y. \text{merkle-node-child-output-relation } x \ y\} \subseteq$
(case x of
MerkleNode l r $\Rightarrow \{l, r\}$
| - $\Rightarrow \{\}$)

unfolding *merkle-node-child-output-relation-def*
by *(cases x) auto*

lemma *merkle-node-child-output-relation-fiber-card-bound:*

card $\{y. \text{merkle-node-child-output-relation } x \ y\} \leq 2$

proof *–*

have *finite-values:*

finite
(case x of
MerkleNode l r $\Rightarrow \{l, r\}$
| - $\Rightarrow \{\}$)

by *(cases x) simp-all*

have *card* $\{y. \text{merkle-node-child-output-relation } x \ y\} \leq$
card
(case x of
MerkleNode l r $\Rightarrow \{l, r\}$
| - $\Rightarrow \{\}$)

by *(rule card-mono[OF finite-values])*
(rule merkle-node-child-output-relation-values-subset)

also have $\dots \leq 2$

proof *(cases x)*

case *(MerkleNode l r)*

then show *?thesis*

by *(simp add: card-insert-if)*

qed *simp-all*

finally show *?thesis .*

qed

definition *merkle-root-or-child-output-relation*

$:: 'f \text{ set} \Rightarrow 'f \text{ protocol-hash-input} \Rightarrow 'f \Rightarrow \text{bool}$

where

merkle-root-or-child-output-relation roots x y $\longleftrightarrow$
y $\in$ roots $\vee$ merkle-node-child-output-relation x y

lemma *merkle-root-or-child-output-relation-values-subset:*

$\{y. \text{merkle-root-or-child-output-relation roots } x \ y\} \subseteq$
roots $\cup$
(case x of
MerkleNode l r $\Rightarrow \{l, r\}$
| - $\Rightarrow \{\}$)

unfolding *merkle-root-or-child-output-relation-def*

using *merkle-node-child-output-relation-values-subset*

by blast

lemma merkle-root-or-child-output-relation-fiber-card-bound:
assumes finite-roots: finite roots
shows card {y. merkle-root-or-child-output-relation roots x y} ≤
card roots + 2
proof –
have finite-values:
finite
(roots ∪
(case x of
MerkleNode l r ⇒ {l, r}
| - ⇒ {}))
using finite-roots **by** (cases x) simp-all
have card {y. merkle-root-or-child-output-relation roots x y} ≤
card
(roots ∪
(case x of
MerkleNode l r ⇒ {l, r}
| - ⇒ {}))
by (rule card-mono[OF finite-values])
(rule merkle-root-or-child-output-relation-values-subset)
also have ... ≤
card roots +
card
(case x of
MerkleNode l r ⇒ {l, r}
| - ⇒ {})
proof –
show ?thesis
by (rule card-Un-le)
qed
also have ... ≤ card roots + 2
proof (cases x)
case (MerkleNode l r)
then show ?thesis
by (simp add: card-insert-if)
qed simp-all
finally show ?thesis .
qed

lemma hash-relation-program-protocol-check-authentication-path-raw:
fixes leaf :: 'f protocol-hash-input
assumes fibers: $\bigwedge x. \text{card } \{y. R \ x \ y\} \leq b$
shows hash-relation-program R b (Suc (length path))
(protocol-merkle.check-authentication-path len i leaf path ::
('f, 'f protocol-channel) state-monad)
proof (induction path arbitrary: len i)
case Nil

```

have hash-relation-program R b 1
  (hash leaf :: ('f, 'f protocol-channel) state-monad)
  by (rule hash-relation-program-hash[OF fibers])
then show ?case
  by (simp add: protocol-merkle.check-authentication-path.simps)
next
case (Cons a path)
show ?case
proof (cases i < len div 2)
  case True
  have rec:
    hash-relation-program R b (Suc (length path))
    (protocol-merkle.check-authentication-path (len div 2) i leaf path ::
      ('f, 'f protocol-channel) state-monad)
  using Cons.IH[of len div 2 i] by simp
  have bound:
    hash-relation-program R b (Suc (length path) + 1)
    ((protocol-merkle.check-authentication-path (len div 2) i leaf path ::
      ('f, 'f protocol-channel) state-monad)  $\gg$ 
      ( $\lambda x$ . hash (MerkleNode x a)))
  by (rule hash-relation-program-bind)
  (rule rec, rule hash-relation-program-hash[OF fibers])
show ?thesis
  using True bound
  by (simp add: protocol-merkle.check-authentication-path.simps)
next
case False
  have rec:
    hash-relation-program R b (Suc (length path))
    (protocol-merkle.check-authentication-path
      (len div 2) (i - len div 2) leaf path ::
      ('f, 'f protocol-channel) state-monad)
  using Cons.IH[of len div 2 i - len div 2] by simp
  have bound:
    hash-relation-program R b (Suc (length path) + 1)
    ((protocol-merkle.check-authentication-path
      (len div 2) (i - len div 2) leaf path ::
      ('f, 'f protocol-channel) state-monad)  $\gg$ 
      ( $\lambda x$ . hash (MerkleNode a x)))
  by (rule hash-relation-program-bind)
  (rule rec, rule hash-relation-program-hash[OF fibers])
show ?thesis
  using False bound
  by (simp add: protocol-merkle.check-authentication-path.simps)
qed
qed

```

lemma hash-relation-program-check-authentication-path:
 assumes fibers: $\bigwedge x. \text{card } \{y. R \ x \ y\} \leq b$

shows *hash-relation-program* R b (*Suc* (*length path*))
 (*check-authentication-path len i v path*)
unfolding *check-authentication-path-def protocol-check-authentication-path-def*
by (*rule hash-relation-program-protocol-check-authentication-path-raw*
 [*OF fibers*])

lemma *hash-relation-program-check-authentication-path-assert-return*:
fixes *path* :: 'f list
and *v root* :: 'f
and *r* :: 'r
assumes *fibers*: $\bigwedge x. \text{card } \{y. R \ x \ y\} \leq b$
shows
hash-relation-program R b (*Suc* (*length path*))
 ((*check-authentication-path len i v path* ::
 ('f, 'f protocol-channel) state-monad) $\gg=$
 ($\lambda ap. \text{assert } (ap = \text{root}) \gg= (\lambda -. \text{return } r)$)))
proof –
have *tail*:
hash-relation-program R b ($0 + 0$)
 ($\text{assert } (ap = \text{root}) \gg= (\lambda -. \text{return } r)$::
 ('r, 'f protocol-channel) state-monad)
for *ap*
by (*rule hash-relation-program-bind*)
 (*rule hash-relation-program-assert, rule hash-relation-program-zero*
 [*OF hash-map-preserving-return*])
have *hash-relation-program* R b (*Suc* (*length path*) + ($0 + 0$))
 ((*check-authentication-path len i v path* ::
 ('f, 'f protocol-channel) state-monad) $\gg=$
 ($\lambda ap. \text{assert } (ap = \text{root}) \gg= (\lambda -. \text{return } r)$)))
by (*rule hash-relation-program-bind*)
 (*rule hash-relation-program-check-authentication-path*[*OF fibers*],
rule tail)
then show ?thesis **by** *simp*
qed

lemma *hash-relation-program-check-authentication-path-assert-bind*:
fixes *path* :: 'f list
and *v root* :: 'f
and *k* :: unit $\Rightarrow$ ('r, 'f protocol-channel) state-monad
assumes *fibers*: $\bigwedge x. \text{card } \{y. R \ x \ y\} \leq b$
and *k*: $\bigwedge u. \text{hash-relation-program } R \ b \ n \ (k \ u)$
shows
hash-relation-program R b (*Suc* (*length path*) + *n*)
 ((*check-authentication-path len i v path* ::
 ('f, 'f protocol-channel) state-monad) $\gg=$
 ($\lambda ap. \text{assert } (ap = \text{root}) \gg= k$)))
proof –
have *tail*:
hash-relation-program R b ($0 + n$)

```

    (assert (ap = root)  $\gg$  k ::
      ('r, 'f protocol-channel) state-monad)
  for ap
  by (rule hash-relation-program-bind)
    (rule hash-relation-program-assert, rule k)
  have hash-relation-program R b (Suc (length path) + (0 + n))
    ((check-authentication-path len i v path ::
      ('f, 'f protocol-channel) state-monad)  $\gg$ 
      ( $\lambda$ ap. assert (ap = root)  $\gg$  k))
  by (rule hash-relation-program-bind)
    (rule hash-relation-program-check-authentication-path[OF fibers],
      rule tail)
  then show ?thesis by simp
qed

lemma hash-relation-program-read:
  hash-relation-program R b 0
  (read :: ('f, 'f, unit) protocol-c-monad)
  by (rule hash-relation-program-zero)
    (rule hash-map-preserving-read)

lemma hash-relation-program-ntimes:
  fixes m :: ('x, 'f protocol-channel) state-monad
  assumes m: hash-relation-program R b n m
  shows hash-relation-program R b (k * n) (ntimes m k)
  using m
proof (induction k)
  case 0
  then show ?case
    by (simp add: hash-relation-program-zero[OF hash-map-preserving-return])
next
  case (Suc k)
  have tail: hash-relation-program R b (k * n) (ntimes m k)
    using Suc.IH Suc.premis by simp
  have cont:
    hash-relation-program R b (k * n + 0)
    (ntimes m k  $\gg$  ( $\lambda$ xs. return (x # xs))) for x
  by (rule hash-relation-program-bind)
    (rule tail, rule hash-relation-program-zero[OF hash-map-preserving-return])
  have hash-relation-program R b (n + (k * n + 0))
    (m  $\gg$  ( $\lambda$ x. ntimes m k  $\gg$  ( $\lambda$ xs. return (x # xs))))
  by (rule hash-relation-program-bind[OF Suc.premis cont])
  then show ?case by simp
qed

lemma hash-relation-program-mmap:
  fixes ms :: ('x, 'f protocol-channel) state-monad list
  assumes ms:  $\bigwedge m. m \in \text{set } ms \implies \text{hash-relation-program } R \text{ b n } m$ 
  shows hash-relation-program R b (length ms * n) (mmap ms)

```



```

using ms
proof (induction ms)
  case Nil
  then show ?case
    by (simp add: hash-relation-program-zero[OF hash-map-preserving-return])
next
  case (Cons m ms)
  have head: hash-relation-program R b n m
    using Cons.prems by simp
  have tail: hash-relation-program R b (length ms * n) (mmap ms)
    using Cons.IH Cons.prems by simp
  have cont:
    hash-relation-program R b (length ms * n + 0)
    (mmap ms  $\gg$  ( $\lambda x$ . return ( $x \# xs$ ))) for x
  by (rule hash-relation-program-bind)
    (rule tail, rule hash-relation-program-zero[OF hash-map-preserving-return])
  have hash-relation-program R b (n + (length ms * n + 0))
    (m  $\gg$  ( $\lambda x$ . mmap ms  $\gg$  ( $\lambda x$ . return ( $x \# xs$ ))))
  by (rule hash-relation-program-bind[OF head cont])
  then show ?case by simp
qed

```

```

lemma hash-relation-program-mfold:
  fixes steps :: ('x  $\Rightarrow$  ('x, 'f protocol-channel) state-monad) list
  assumes steps:
     $\bigwedge \text{step } x. \text{step} \in \text{set steps} \implies$ 
    hash-relation-program R b n (step x)
  shows hash-relation-program R b (length steps * n) (mfold x steps)
  using steps
proof (induction steps arbitrary: x)
  case Nil
  then show ?case
    by (simp add: hash-relation-program-zero[OF hash-map-preserving-return])
next
  case (Cons step steps)
  have head: hash-relation-program R b n (step x)
    using Cons.prems by simp
  have tail:
     $\bigwedge y. \text{hash-relation-program } R \text{ b } (\text{length steps} * n) (\text{mfold } y \text{ steps})$ 
    using Cons.IH Cons.prems by simp
  have hash-relation-program R b (n + length steps * n)
    (step x  $\gg$  ( $\lambda y. \text{mfold } y \text{ steps}$ ))
  by (rule hash-relation-program-bind[OF head tail])
  then show ?case by simp
qed

```

```

lemma hash-relation-program-ntimes-read-bind:
  fixes k :: ('f list  $\Rightarrow$  ('y, 'f protocol-channel) state-monad)
  assumes k:

```

```

 $\bigwedge xs. \text{length } xs = n \implies \text{hash-relation-program } R \ b \ q \ (k \ xs)$ 
shows
   $\text{hash-relation-program } R \ b \ q$ 
   $((\text{ntimes read } n :: ('f \text{ list}, 'f \text{ protocol-channel}) \text{ state-monad}) \ggg k)$ 
proof –
  have reads:
     $\text{hash-relation-program } R \ b \ (n * 0)$ 
     $(\text{ntimes read } n :: ('f \text{ list}, 'f \text{ protocol-channel}) \text{ state-monad})$ 
  by (rule hash-relation-program-ntimes[OF hash-relation-program-read])
  have  $\text{hash-relation-program } R \ b \ (n * 0 + q)$ 
     $((\text{ntimes read } n :: ('f \text{ list}, 'f \text{ protocol-channel}) \text{ state-monad}) \ggg k)$ 
  proof (rule hash-relation-program-bind-on-outcomes[OF reads])
    fix  $s \ xs \ t$ 
    assume out:
       $\text{Some } (xs, t) \in$ 
        set-dist
         $(\text{execute}$ 
           $(\text{ntimes read } n :: ('f \text{ list}, 'f \text{ protocol-channel}) \text{ state-monad})$ 
           $s)$ 
        then have  $\text{length } xs = n$ 
          using ntimes-read-any-outcome by blast
        then show  $\text{hash-relation-program } R \ b \ q \ (k \ xs)$ 
          by (rule k)
    qed
  then show ?thesis by simp
qed

```

```

lemma hash-relation-program-check-decommit-on-query-step:
  assumes fibers:  $\bigwedge x. \text{card } \{y. R \ x \ y\} \leq b$ 
  shows
     $\text{hash-relation-program } R \ b \ (\text{Suc } (\text{floor-log } (\text{scale} * \text{clength})))$ 
     $(\text{do } \{$ 
       $qh \leftarrow \text{read};$ 
       $\text{let } \text{len} = \text{scale} * \text{clength};$ 
       $qh\text{-path} \leftarrow \text{ntimes read } (\text{floor-log } \text{len});$ 
       $ap \leftarrow \text{check-authentication-path } \text{len } i \ qh \ qh\text{-path};$ 
       $\text{assert } (ap = \text{fr});$ 
       $\text{return } qh$ 
     $\})$ 
  proof –
    let  $?len = \text{scale} * \text{clength}$ 
    have after-path:
       $\text{hash-relation-program } R \ b \ (\text{Suc } (\text{floor-log } ?len))$ 
       $((\text{check-authentication-path } ?len \ i \ qh \ qh\text{-path} ::$ 
         $('f, 'f \text{ protocol-channel}) \text{ state-monad}) \ggg$ 
         $(\lambda ap. \text{assert } (ap = \text{fr}) \ggg (\lambda -. \text{return } qh)))$ 
    if path-len:  $\text{length } qh\text{-path} = \text{floor-log } ?len$ 
    for  $qh \ qh\text{-path}$ 
    using path-len

```

```

by (subst path-len[symmetric])
  (rule hash-relation-program-check-authentication-path-assert-return
    [OF fibers])
have after-reads:
  hash-relation-program R b (Suc (floor-log ?len))
    ((ntimes read (floor-log ?len) ::
      ('f list, 'f protocol-channel) state-monad)  $\gg$ 
      ( $\lambda$ qh-path.
        (check-authentication-path ?len i qh qh-path ::
          ('f, 'f protocol-channel) state-monad)  $\gg$ 
          ( $\lambda$ ap. assert (ap = fr)  $\gg$  ( $\lambda$ -. return qh))))
    for qh
    by (rule hash-relation-program-ntimes-read-bind)
      (rule after-path)
have hash-relation-program R b (0 + Suc (floor-log ?len))
  (read  $\gg$ 
    ( $\lambda$ qh.
      (ntimes read (floor-log ?len) ::
        ('f list, 'f protocol-channel) state-monad)  $\gg$ 
        ( $\lambda$ qh-path.
          (check-authentication-path ?len i qh qh-path ::
            ('f, 'f protocol-channel) state-monad)  $\gg$ 
            ( $\lambda$ ap. assert (ap = fr)  $\gg$  ( $\lambda$ -. return qh))))))
    by (rule hash-relation-program-bind)
      (rule hash-relation-program-read, rule after-reads)
then show ?thesis
  by (simp add: Let-def)
qed

```

```

lemma hash-relation-program-check-decommit-on-query:
  assumes fibers:  $\bigwedge x. \text{card } \{y. R \ x \ y\} \leq b$ 
  shows
    hash-relation-program R b
      (length (powers-scaled idx) * Suc (floor-log (scale * clength)))
      (mmap (check-decommit-on-query fr idx))
proof -
  let ?step =
     $\lambda i. \text{do } \{$ 
      qh  $\leftarrow$  read;
      let len = scale * clength;
      qh-path  $\leftarrow$  ntimes read (floor-log len);
      ap  $\leftarrow$  check-authentication-path len i qh qh-path;
      assert (ap = fr);
      return qh
     $\}$ 
  have mapped-general:
     $\bigwedge xs. \text{hash-relation-program } R \ b$ 
      (length xs + length xs * floor-log (scale * clength))
      (mmap (map ?step xs))

```

```

proof –
  fix xs
  show hash-relation-program R b
    (length xs + length xs * floor-log (scale * clength))
    (mmap (map ?step xs))
  proof (induction xs)
  case Nil
  then show ?case
    by (simp add: hash-relation-program-zero[OF hash-map-preserving-return])
  next
  case (Cons i rest)
  have head:
    hash-relation-program R b (Suc (floor-log (scale * clength)))
    (?step i)
  by (rule hash-relation-program-check-decommit-on-query-step[OF fibers])
  have tail:
    hash-relation-program R b
      (length rest + length rest * floor-log (scale * clength))
      (mmap (map ?step rest))
  by (rule Cons.IH)
  have cont:
    hash-relation-program R b
      ((length rest + length rest * floor-log (scale * clength)) + 0)
      (mmap (map ?step rest)  $\gg$  ( $\lambda xs.$  return (x # xs)))
  for x
  by (rule hash-relation-program-bind)
    (rule tail, rule hash-relation-program-zero[OF hash-map-preserving-return])
  have hash-relation-program R b
    (Suc (floor-log (scale * clength)) +
      ((length rest + length rest * floor-log (scale * clength)) + 0))
    (?step i  $\gg$  ( $\lambda x.$  mmap (map ?step rest)  $\gg$ 
      ( $\lambda xs.$  return (x # xs))))
  by (rule hash-relation-program-bind[OF head cont])
  then show ?case
    by (simp add: algebra-simps)
  qed
qed
have mapped:
  hash-relation-program R b
    (length (powers-scaled idx) +
      length (powers-scaled idx) * floor-log (scale * clength))
    (mmap (map ?step (powers-scaled idx)))
  by (rule mapped-general)
show ?thesis
  using mapped
  unfolding check-decommit-on-query-def by simp
qed

```

lemma *hash-relation-program-fri-layer-opening-finish*:

```

assumes fibers:  $\bigwedge x. \text{card } \{y. R \ x \ y\} \leq b$ 
and xp-path-len:  $\text{length } xp\text{-path} = \text{floor-log } len$ 
and xn-path-len:  $\text{length } xn\text{-path} = \text{floor-log } len$ 
shows
  hash-relation-program R b
    (Suc (floor-log len) + Suc (floor-log len))
    (fri-layer-opening-finish b' f i x len pw xp xp-path xn xn-path)
proof –
  let ?H = Suc (floor-log len)
  let ?sidx = (i + len div 2) mod len
  let ?out =
    (i mod (len div 2),
     (xp + xn) div 2 +
      b' * ((xp - xn) div (2 * ((h ^ i) * shift) ^ pw)),
     len div 2, pw + pw)
  have second:
    hash-relation-program R b ?H
      ((check-authentication-path len ?sidx xn xn-path ::
        ('f, 'f protocol-channel) state-monad)  $\gg=$ 
        ( $\lambda ap. \text{assert } (ap = f) \gg= (\lambda -. \text{return } ?out)$ )))
    by (subst xn-path-len[symmetric])
      (rule hash-relation-program-check-authentication-path-assert-return
        [OF fibers])
  have first0:
    hash-relation-program R b (Suc (length xp-path) + ?H)
      ((check-authentication-path len i xp xp-path ::
        ('f, 'f protocol-channel) state-monad)  $\gg=$ 
        ( $\lambda ap. \text{assert } (ap = f) \gg=$ 
          ( $\lambda -. \text{check-authentication-path } len \ ?sidx \ xn \ xn\text{-path} ::$ 
            ('f, 'f protocol-channel) state-monad)  $\gg=$ 
            ( $\lambda ap. \text{assert } (ap = f) \gg= (\lambda -. \text{return } ?out)$ ))))))
    by (rule hash-relation-program-check-authentication-path-assert-bind
      [OF fibers])
      (use second in simp)
  have first:
    hash-relation-program R b (?H + ?H)
      ((check-authentication-path len i xp xp-path ::
        ('f, 'f protocol-channel) state-monad)  $\gg=$ 
        ( $\lambda ap. \text{assert } (ap = f) \gg=$ 
          ( $\lambda -. \text{check-authentication-path } len \ ?sidx \ xn \ xn\text{-path} ::$ 
            ('f, 'f protocol-channel) state-monad)  $\gg=$ 
            ( $\lambda ap. \text{assert } (ap = f) \gg= (\lambda -. \text{return } ?out)$ ))))))
    using first0 xp-path-len by simp
  have raw:
    hash-relation-program R b (0 + (?H + ?H))
      ((assert (xp = x) ::
        (unit, 'f protocol-channel) state-monad)  $\gg=$ 

```

```

(λ-.
  (check-authentication-path len i xp xp-path ::
    ('f, 'f protocol-channel) state-monad) >>=
  (λap. assert (ap = f) >>=
    (λ-.
      (check-authentication-path len ?sidx xn xn-path ::
        ('f, 'f protocol-channel) state-monad) >>=
        (λap. assert (ap = f) >>= (λ-. return ?out))))))
by (rule hash-relation-program-bind)
  (rule hash-relation-program-assert, rule first)
show ?thesis
  using raw unfolding fri-layer-opening-finish-def
  by (simp add: Let-def)
qed

lemma hash-relation-program-fri-layer-opening-step:
  assumes fibers:  $\bigwedge x. \text{card } \{y. R \ x \ y\} \leq b$ 
  shows
    hash-relation-program R b
      (Suc (floor-log len) + Suc (floor-log len))
      (fri-layer-opening-step (b', f) (i, x, len, pw))
proof -
  let ?N = Suc (floor-log len) + Suc (floor-log len)
  have after-xn-path:
    hash-relation-program R b ?N
      ((ntimes read (floor-log len) ::
        ('f list, 'f protocol-channel) state-monad) >>=
        (λxn-path.
          fri-layer-opening-finish b' f i x len pw xp xp-path xn xn-path)))
  if xp-path-len: length xp-path = floor-log len
  for xp xp-path xn
proof (rule hash-relation-program-ntimes-read-bind)
  fix xn-path :: 'f list
  assume xn-path-len: length xn-path = floor-log len
  show hash-relation-program R b ?N
    (fri-layer-opening-finish b' f i x len pw xp xp-path xn xn-path)
  by (rule hash-relation-program-fri-layer-opening-finish
    [OF fibers xp-path-len xn-path-len])
qed
have after-xn:
  hash-relation-program R b ?N
    (read >>=
      (λxn.
        (ntimes read (floor-log len) ::
          ('f list, 'f protocol-channel) state-monad) >>=
          (λxn-path.
            fri-layer-opening-finish b' f i x len pw xp xp-path xn xn-path))))
  if xp-path-len: length xp-path = floor-log len
  for xp xp-path

```

```

proof –
  have hash-relation-program  $R\ b\ (0 + ?N)$ 
    (read  $\gg$ 
      ( $\lambda xn.$ 
        (ntimes read (floor-log len) ::
          ('f list, 'f protocol-channel) state-monad)  $\gg$ 
          ( $\lambda xn$ -path.
            fri-layer-opening-finish  $b'\ f\ i\ x\ len\ pw\ xp\ xp$ -path  $xn\ xn$ -path))))
    by (rule hash-relation-program-bind)
      (rule hash-relation-program-read, rule after-xn-path[OF  $xp$ -path-len])
  then show ?thesis by simp
qed
have after-xp-path:
  hash-relation-program  $R\ b\ ?N$ 
    ((ntimes read (floor-log len) ::
      ('f list, 'f protocol-channel) state-monad)  $\gg$ 
      ( $\lambda xp$ -path.
        read  $\gg$ 
          ( $\lambda xn.$ 
            (ntimes read (floor-log len) ::
              ('f list, 'f protocol-channel) state-monad)  $\gg$ 
              ( $\lambda xn$ -path.
                fri-layer-opening-finish  $b'\ f\ i\ x\ len\ pw\ xp\ xp$ -path  $xn\ xn$ -path))))))
  for  $xp$ 
proof (rule hash-relation-program-ntimes-read-bind)
  fix  $xp$ -path :: 'f list
  assume  $xp$ -path-len: length  $xp$ -path = floor-log len
  show hash-relation-program  $R\ b\ ?N$ 
    (read  $\gg$ 
      ( $\lambda xn.$ 
        (ntimes read (floor-log len) ::
          ('f list, 'f protocol-channel) state-monad)  $\gg$ 
          ( $\lambda xn$ -path.
            fri-layer-opening-finish  $b'\ f\ i\ x\ len\ pw\ xp\ xp$ -path  $xn\ xn$ -path))))
    by (rule after-xn[OF  $xp$ -path-len])
qed
have hash-relation-program  $R\ b\ (0 + ?N)$ 
  (read  $\gg$ 
    ( $\lambda xp.$ 
      (ntimes read (floor-log len) ::
        ('f list, 'f protocol-channel) state-monad)  $\gg$ 
        ( $\lambda xp$ -path.
          read  $\gg$ 
            ( $\lambda xn.$ 
              (ntimes read (floor-log len) ::
                ('f list, 'f protocol-channel) state-monad)  $\gg$ 
                ( $\lambda xn$ -path.
                  fri-layer-opening-finish  $b'\ f\ i\ x\ len\ pw\ xp\ xp$ -path  $xn\ xn$ -path))))))
    by (rule hash-relation-program-bind)

```

```

    (rule hash-relation-program-read, rule after-xp-path)
  then show ?thesis
    unfolding fri-layer-opening-step-unfold-finish by simp
qed

lemma hash-relation-program-fri-layer-opening-steps-mfold-state:
  fixes st :: nat × 'f × nat × nat
  assumes fibers:  $\bigwedge x. \text{card } \{y. R \ x \ y\} \leq b$ 
  and init:
    case st of (i, x, len, pw)  $\Rightarrow \text{floor-log } \text{len} \leq L$ 
  shows
    hash-relation-program R b
      (length fl * (Suc L + Suc L))
      (mfold st (map fri-layer-opening-step fl) ::
        (nat × 'f × nat × nat, 'f protocol-channel) state-monad)
  using init
proof (induction fl arbitrary: st)
  case Nil
  then show ?case
    by (simp add: hash-relation-program-zero[OF hash-map-preserving-return])
next
  case (Cons bf fl)
  obtain i x len pw where st-eq: st = (i, x, len, pw)
  by (cases st)
  obtain b' f where bf-eq: bf = (b', f)
  by (cases bf)
  let ?N = Suc L + Suc L
  have head-exact:
    hash-relation-program R b
      (Suc (floor-log len) + Suc (floor-log len))
      (fri-layer-opening-step (b', f) (i, x, len, pw))
  by (rule hash-relation-program-fri-layer-opening-step[OF fibers])
  have head-le:
    Suc (floor-log len) + Suc (floor-log len)  $\leq$  ?N
  using Cons.premis unfolding st-eq by simp
  have head:
    hash-relation-program R b ?N
      (fri-layer-opening-step bf st ::
        (nat × 'f × nat × nat, 'f protocol-channel) state-monad)
  unfolding bf-eq st-eq
  by (rule hash-relation-program-mono[OF head-le head-exact])
  have tail:
    hash-relation-program R b (length fl * ?N)
      (mfold z (map fri-layer-opening-step fl) ::
        (nat × 'f × nat × nat, 'f protocol-channel) state-monad)
  if out:
    Some (z, t)  $\in$ 
      set-dist (execute (fri-layer-opening-step bf st) s)
  for z and s :: 'f protocol-channel and t :: 'f protocol-channel

```



```

proof –
  from fri-layer-opening-step-outcome[OF out[unfolded bf-eq st-eq]]
  obtain xp xp-path xn xn-path x' where
    z-eq:  $z = (i \bmod (\text{len} \div 2), x', \text{len} \div 2, pw + pw)$ 
    by blast
  have len-le:  $\text{floor-log len} \leq L$ 
    using Cons.premis unfolding st-eq by simp
  have next-len:  $\text{floor-log} (\text{len} \div 2) \leq L$ 
    using floor-log-div2-le-self[of len] len-le by linarith
  have z-inv:
    case z of (i, x, len, pw)  $\Rightarrow$  floor-log len  $\leq$  L
    unfolding z-eq using next-len by simp
  show ?thesis
    by (rule Cons.IH[OF z-inv])
qed
have hash-relation-program R b ( $?N + \text{length fl} * ?N$ )
  ((fri-layer-opening-step bf st ::
    ( $\text{nat} \times 'f \times \text{nat} \times \text{nat}, 'f \text{ protocol-channel}$ ) state-monad)  $\gg$ 
    ( $\lambda z. \text{mfold } z (\text{map } \text{fri-layer-opening-step fl})$ ))
  by (rule hash-relation-program-bind-on-outcomes[OF head]) (rule tail)
then show ?case
  by (simp add: add.assoc add.commute add.left-commute)
qed

lemma hash-relation-program-receive-query-commits-mfold:
  assumes fibers:  $\bigwedge x. \text{card } \{y. R \ x \ y\} \leq b$ 
  and init:  $\text{floor-log len} \leq L$ 
  shows
    hash-relation-program R b
      ( $\text{length fl} * (\text{Suc } L + \text{Suc } L)$ )
      ( $\text{mfold } (i, x, \text{len}, pw) (\text{receive-query-commits fl})$ )
  unfolding receive-query-commits-eq-fri-layer-opening-steps
proof –
  have st-inv:
    case (i, x, len, pw) of
      ( $i, x, \text{len}, pw$ )  $\Rightarrow$   $\text{floor-log len} \leq L$ 
    using init by simp
  show hash-relation-program R b ( $\text{length fl} * (\text{Suc } L + \text{Suc } L)$ )
    ( $\text{mfold } (i, x, \text{len}, pw) (\text{map } \text{fri-layer-opening-step fl})$  ::
      ( $\text{nat} \times 'f \times \text{nat} \times \text{nat}, 'f \text{ protocol-channel}$ ) state-monad)
    by (rule hash-relation-program-fri-layer-opening-steps-mfold-state
      [OF fibers st-inv])
qed

lemma hash-relation-program-check-decommit-on-query-exact:
  assumes fibers:  $\bigwedge x. \text{card } \{y. R \ x \ y\} \leq b$ 
  shows
    hash-relation-program R b verifier-query-decommit-hash-budget
      ( $\text{mmap } (\text{check-decommit-on-query fr idx})$ )

```

```

proof –
  have exact:
    hash-relation-program R b
      (length (powers-scaled idx) * Suc (floor-log (scale * clength)))
      (mmap (check-decommit-on-query fr idx))
    by (rule hash-relation-program-check-decommit-on-query[OF fibers])
  show ?thesis
    using exact
    unfolding verifier-query-decommit-hash-budget-def powers-scaled-def
    by (simp add: mult.commute)
qed

```

```

lemma hash-relation-program-verifier-query-round-program-exact:
  assumes fibers:  $\bigwedge x. \text{card } \{y. R \ x \ y\} \leq b$ 
  shows

```

```

  hash-relation-program R b
    (1 + verifier-query-decommit-hash-budget +
      (length f-fl + length fl) * verifier-fri-layer-hash-budget)
    (verifier-query-round-program fr f-fl f-final as fl final)

```

```

proof –
  let ?L = floor-log (clength * scale)
  let ?B = verifier-fri-layer-hash-budget
  have B-eq: ?B = Suc ?L + Suc ?L
    unfolding verifier-fri-layer-hash-budget-def by simp
  have after-comp:
    hash-relation-program R b (length fl * ?B + 0)
      ((mfold
        (idx, cp-eval as fv (h ^ idx * shift), clength * scale, 1)
        (receive-query-commits fl) ::
        (nat × 'f × nat × nat, 'f protocol-channel) state-monad) >>=
        (λ(i, x, len, pw). assert (x = final)))
    for idx fv
  proof (rule hash-relation-program-bind)
  show hash-relation-program R b (length fl * ?B)
    (mfold
      (idx, cp-eval as fv (h ^ idx * shift), clength * scale, 1)
      (receive-query-commits fl) ::
      (nat × 'f × nat × nat, 'f protocol-channel) state-monad)
    unfolding B-eq
    by (rule hash-relation-program-receive-query-commits-mfold)
      (rule fibers, simp)
  show  $\bigwedge x. \text{hash-relation-program } R \ b \ 0$ 
    ((case x of (i, x, len, pw) ⇒ assert (x = final)) ::
      (unit, 'f protocol-channel) state-monad)
    by (simp add: hash-relation-program-assert split: prod.splits)
qed

```

```

have after-trace:
  hash-relation-program R b
    (length f-fl * ?B + (0 + (length fl * ?B + 0)))

```

```

((mfold (idx, hd fv, clength * scale, 1)
  (receive-query-commits f-fl) ::
  (nat × 'f × nat × nat, 'f protocol-channel) state-monad) ≫=
(λ(f-i, f-x, f-len, f-pow).
  assert (f-x = f-final) ≫=
  (λ-. (mfold
    (idx, cp-eval as fv (h ^ idx * shift), clength * scale, 1)
    (receive-query-commits fl) ::
    (nat × 'f × nat × nat, 'f protocol-channel) state-monad) ≫=
    (λ(i, x, len, pw). assert (x = final))))))
for idx fv
proof (rule hash-relation-program-bind)
show hash-relation-program R b (length f-fl * ?B)
  (mfold (idx, hd fv, clength * scale, 1)
    (receive-query-commits f-fl) ::
    (nat × 'f × nat × nat, 'f protocol-channel) state-monad)
  unfolding B-eq
  by (rule hash-relation-program-receive-query-commits-mfold)
    (rule fibers, simp)
fix x :: nat × 'f × nat × nat
obtain f-i f-x f-len f-pow where x-eq:
  x = (f-i, f-x, f-len, f-pow)
  by (cases x) auto
have case-budget:
  hash-relation-program R b (0 + (length fl * ?B + 0))
  ((assert (f-x = f-final) ::
    (unit, 'f protocol-channel) state-monad) ≫=
  (λ-. (mfold
    (idx, cp-eval as fv (h ^ idx * shift), clength * scale, 1)
    (receive-query-commits fl) ::
    (nat × 'f × nat × nat, 'f protocol-channel) state-monad) ≫=
    (λ(i, x, len, pw). assert (x = final))))))
  by (rule hash-relation-program-bind)
    (rule hash-relation-program-assert, rule after-comp)
show hash-relation-program R b (0 + (length fl * ?B + 0))
  ((case x of (f-i, f-x, f-len, f-pow) ⇒
    assert (f-x = f-final) ≫=
  (λ-. (mfold
    (idx, cp-eval as fv (h ^ idx * shift), clength * scale, 1)
    (receive-query-commits fl) ::
    (nat × 'f × nat × nat, 'f protocol-channel) state-monad) ≫=
    (λ(i, x, len, pw). assert (x = final)))))) ::
  (unit, 'f protocol-channel) state-monad)
  using case-budget
  unfolding x-eq
  by (simp split: prod.splits)
qed
have after-query:
  hash-relation-program R b

```

```

(verifier-query-decommit-hash-budget +
 (length f-fl * ?B + (0 + (length fl * ?B + 0))))
(mmap (check-decommit-on-query fr idx) >>=
 (λfv. (mfold (idx, hd fv, clength * scale, 1)
 (receive-query-commits f-fl) ::
 (nat × 'f × nat × nat, 'f protocol-channel) state-monad) >>=
 (λ(f-i, f-x, f-len, f-pow).
 assert (f-x = f-final) >>=
 (λ-. (mfold
 (idx, cp-eval as fv (h ^ idx * shift), clength * scale, 1)
 (receive-query-commits fl) ::
 (nat × 'f × nat × nat, 'f protocol-channel) state-monad) >>=
 (λ(i, x, len, pw). assert (x = final)))))))
for idx
by (rule hash-relation-program-bind)
(rule hash-relation-program-check-decommit-on-query-exact[OF fibers],
 rule after-trace)
have after-query-simple:
hash-relation-program R b
(verifier-query-decommit-hash-budget +
 (length f-fl * ?B + length fl * ?B))
(mmap (check-decommit-on-query fr idx) >>=
 (λfv. (mfold (idx, hd fv, clength * scale, 1)
 (receive-query-commits f-fl) ::
 (nat × 'f × nat × nat, 'f protocol-channel) state-monad) >>=
 (λ(f-i, f-x, f-len, f-pow).
 assert (f-x = f-final) >>=
 (λ-. (mfold
 (idx, cp-eval as fv (h ^ idx * shift), clength * scale, 1)
 (receive-query-commits fl) ::
 (nat × 'f × nat × nat, 'f protocol-channel) state-monad) >>=
 (λ(i, x, len, pw). assert (x = final)))))))
for idx
using after-query[of idx] by simp
have after-random:
hash-relation-program R b
(verifier-query-decommit-hash-budget +
 (length f-fl * ?B + length fl * ?B))
((let idx' = index (to-nat idx) in
 mmap (check-decommit-on-query fr idx') >>=
 (λfv. (mfold (idx', hd fv, clength * scale, 1)
 (receive-query-commits f-fl) ::
 (nat × 'f × nat × nat, 'f protocol-channel) state-monad) >>=
 (λ(f-i, f-x, f-len, f-pow).
 assert (f-x = f-final) >>=
 (λ-. (mfold
 (idx', cp-eval as fv (h ^ idx' * shift), clength * scale, 1)
 (receive-query-commits fl) ::
 (nat × 'f × nat × nat, 'f protocol-channel) state-monad) >>=

```

```

      (λ(i, x, len, pw). assert (x = final))))))
for idx
using after-query-simple[of index (to-nat idx)]
by (simp add: Let-def algebra-simps mult.commute split: prod.splits)
have hash-relation-program R b
  (1 + (verifier-query-decommit-hash-budget +
    (length f-fl * ?B + length fl * ?B)))
  (receive-query-index-challenge >>=
    (λidx.
      let idx' = index (to-nat idx) in
      mmap (check-decommit-on-query fr idx') >>=
      (λfv. (mfold (idx', hd fv, clength * scale, 1)
        (receive-query-commits f-fl) ::
        (nat × 'f × nat × nat, 'f protocol-channel) state-monad) >>=
        (λ(f-i, f-x, f-len, f-pow).
          assert (f-x = f-final) >>=
          (λ-. (mfold
            (idx', cp-eval as fv (h ^ idx' * shift), clength * scale, 1)
            (receive-query-commits fl) ::
            (nat × 'f × nat × nat, 'f protocol-channel) state-monad) >>=
            (λ(i, x, len, pw). assert (x = final)))))))
by (rule hash-relation-program-bind)
  (rule hash-relation-program-receive-query-index-challenge[OF fibers],
    rule after-random)
then show ?thesis
unfolding verifier-query-round-program-def
by (simp add: Let-def algebra-simps mult.commute split: prod.splits)
qed

```

lemma hash-relation-program-verifier-query-round-program:

```

assumes fibers:  $\bigwedge x. \text{card } \{y. R \ x \ y\} \leq b$ 
and len-f:  $\text{length } f\text{-fl} \leq \text{ceil-log } \text{clength}$ 
and len-c:  $\text{length } fl \leq \text{ceil-log } (\text{maxDegree} + 1)$ 

```

shows

```

  hash-relation-program R b verifier-query-round-hash-budget
  (verifier-query-round-program fr f-fl f-final as fl final)

```

proof –

```

let ?N = verifier-fri-layer-hash-budget

```

have exact:

```

  hash-relation-program R b
  (1 + verifier-query-decommit-hash-budget +
    (length f-fl + length fl) * ?N)
  (verifier-query-round-program fr f-fl f-final as fl final)
by (rule hash-relation-program-verifier-query-round-program-exact
  [OF fibers])

```

have le:

```

  1 + verifier-query-decommit-hash-budget +
  (length f-fl + length fl) * ?N
  ≤ verifier-query-round-hash-budget

```

```

    using len-f len-c
    unfolding verifier-query-round-hash-budget-def
    by simp
  show ?thesis
    by (rule hash-relation-program-mono[OF le exact])
qed

lemma wp-verifier-query-round-program-hash-relation-hit-bound:
  assumes fibers:  $\bigwedge x. \text{card } \{y. R \ x \ y\} \leq b$ 
    and len-f:  $\text{length } f\text{-fl} \leq \text{ceil-log } \text{clength}$ 
    and len-c:  $\text{length } fl \leq \text{ceil-log } (\text{maxDegree} + 1)$ 
  shows
    wp-event
      (verifier-query-round-program fr f-fl f-final as fl final)
      (hash-relation-hit-event R s)  $s \leq$ 
      hash-relation-budget-value b verifier-query-round-hash-budget
  proof -
    have program:
      hash-relation-program R b verifier-query-round-hash-budget
      (verifier-query-round-program fr f-fl f-final as fl final)
    by (rule hash-relation-program-verifier-query-round-program
        [OF fibers len-f len-c])
    then show ?thesis
      unfolding hash-relation-program-def hash-relation-budget-def by blast
  qed

```

```

lemma wp-verifier-query-round-program-merkle-root-or-child-relation-bound:
  assumes finite-roots: finite roots
    and len-f:  $\text{length } f\text{-fl} \leq \text{ceil-log } \text{clength}$ 
    and len-c:  $\text{length } fl \leq \text{ceil-log } (\text{maxDegree} + 1)$ 
  shows
    wp-event
      (verifier-query-round-program fr f-fl f-final as fl final)
      (hash-relation-hit-event
        (merkle-root-or-child-output-relation roots) s)  $s \leq$ 
      hash-relation-budget-value (card roots + 2)
      verifier-query-round-hash-budget
  by (rule wp-verifier-query-round-program-hash-relation-hit-bound)
  (rule merkle-root-or-child-output-relation-fiber-card-bound
    [OF finite-roots], rule len-f, rule len-c)

```

```

lemma hash-relation-program-ntimes-verifier-query-round-program:
  assumes fibers:  $\bigwedge x. \text{card } \{y. R \ x \ y\} \leq b$ 
    and len-f:  $\text{length } f\text{-fl} \leq \text{ceil-log } \text{clength}$ 
    and len-c:  $\text{length } fl \leq \text{ceil-log } (\text{maxDegree} + 1)$ 
  shows
    hash-relation-program R b (n * verifier-query-round-hash-budget)
      (ntimes
        (verifier-query-round-program fr f-fl f-final as fl final) n)

```

proof (rule hash-relation-program-ntimes)
show $\bigwedge x.$
 hash-relation-program *R* *b* *verifier-query-round-hash-budget*
 (*verifier-query-round-program* *fr* *f-fl* *f-final* as *fl* *final*)
by (rule hash-relation-program-verifier-query-round-program
 [*OF* *fibers* *len-f* *len-c*])
qed

lemma *wp-ntimes-verifier-query-round-program-hash-relation-hit-bound*:
assumes *fibers*: $\bigwedge x. \text{card } \{y. R \ x \ y\} \leq b$
and *len-f*: $\text{length } f\text{-fl} \leq \text{ceil-log } \text{clength}$
and *len-c*: $\text{length } fl \leq \text{ceil-log } (\text{maxDegree} + 1)$
shows
 wp-event
 (*ntimes*
 (*verifier-query-round-program* *fr* *f-fl* *f-final* as *fl* *final*) *n*)
 (*hash-relation-hit-event* *R* *s*) *s* $\leq$
 hash-relation-budget-value *b*
 (*n* * *verifier-query-round-hash-budget*)
proof –
have *program*:
 hash-relation-program *R* *b* (*n* * *verifier-query-round-hash-budget*)
 (*ntimes*
 (*verifier-query-round-program* *fr* *f-fl* *f-final* as *fl* *final*) *n*)
by (rule hash-relation-program-ntimes-verifier-query-round-program
 [*OF* *fibers* *len-f* *len-c*])
then show ?thesis
 unfolding *hash-relation-program-def* *hash-relation-budget-def* **by** *blast*
qed

lemma *wp-ntimes-verifier-query-round-program-merkle-root-or-child-relation-bound*:
assumes *finite-roots*: *finite roots*
and *len-f*: $\text{length } f\text{-fl} \leq \text{ceil-log } \text{clength}$
and *len-c*: $\text{length } fl \leq \text{ceil-log } (\text{maxDegree} + 1)$
shows
 wp-event
 (*ntimes*
 (*verifier-query-round-program* *fr* *f-fl* *f-final* as *fl* *final*) *n*)
 (*hash-relation-hit-event*
 (*merkle-root-or-child-output-relation* *roots*) *s*) *s* $\leq$
 hash-relation-budget-value (*card roots* + 2)
 (*n* * *verifier-query-round-hash-budget*)
by (rule *wp-ntimes-verifier-query-round-program-hash-relation-hit-bound*)
 (rule *merkle-root-or-child-output-relation-fiber-card-bound*
 [*OF* *finite-roots*], rule *len-f*, rule *len-c*)

definition *checked-staged-security-with-actual-alpha-prefix-tree-output-case*
where
 checked-staged-security-with-actual-alpha-prefix-tree-output-case *P* *out*

$\longleftrightarrow$
 (case out of
 None $\Rightarrow$ False
 | Some (x, -) $\Rightarrow$
 (let packed = fst (fst x);
 prefix-state = snd (fst packed);
 data = snd packed;
 attacker-state = snd (fst x);
 s =
 verifier-state-from-adversary attacker-state
 (staged-proof-transcript data)
 in $\exists$ result final-state trace-table composition-table as
 query-idxs dg composition-fri-roots final rest trace-tree
 composition-tree.
 Some (result, final-state) $\in$
 set-dist (execute verify-monad s) $\wedge$
 accepted-with-bound-tables s (Some (result, final-state))
 trace-table composition-table as query-idxs $\wedge$
 verifier-header-transcript s
 (staged-trace-root data)
 (staged-trace-fri-roots data)
 (staged-trace-final data)
 as dg composition-fri-roots final rest $\wedge$
 created-tree trace-table trace-tree final-state $\wedge$
 created-tree composition-table composition-tree final-state $\wedge$
 P trace-tree composition-tree prefix-state final-state))

definition *checked-staged-security-with-actual-alpha-prefix-tree-root-output-hit*

where

checked-staged-security-with-actual-alpha-prefix-tree-root-output-hit =
checked-staged-security-with-actual-alpha-prefix-tree-output-case
 (λ trace-tree composition-tree prefix-state final-state.
 hash-map-new-output-hit
 {value trace-tree, value composition-tree}
 prefix-state final-state)

definition

checked-staged-security-with-actual-alpha-prefix-tree-subtree-output-hit

where

checked-staged-security-with-actual-alpha-prefix-tree-subtree-output-hit =
checked-staged-security-with-actual-alpha-prefix-tree-output-case
 (λ trace-tree composition-tree prefix-state final-state.
 ($\exists$ l v r. trace-tree = $\langle l, v, r \rangle \wedge$
 hash-map-new-output-hit (set-tree l $\cup$ set-tree r)
 prefix-state final-state) $\vee$
 ($\exists$ l v r. composition-tree = $\langle l, v, r \rangle \wedge$
 hash-map-new-output-hit (set-tree l $\cup$ set-tree r)
 prefix-state final-state))


```

lemma checked-staged-security-with-actual-alpha-prefix-tree-output-hit-imp-root-or-subtree:
  assumes hit:
    checked-staged-security-with-actual-alpha-prefix-tree-output-hit out
  shows
    checked-staged-security-with-actual-alpha-prefix-tree-root-output-hit out  $\vee$ 
    checked-staged-security-with-actual-alpha-prefix-tree-subtree-output-hit
      out
  proof (cases out)
    case None
    then show ?thesis
      using hit
      unfolding
        checked-staged-security-with-actual-alpha-prefix-tree-output-hit-def
      by simp
  next
    case (Some result-pack)
    then obtain x checked-final-state where out-eq:
      out = Some (x, checked-final-state)
    by (cases result-pack) simp
    let ?packed = fst (fst x)
    let ?prefix-state = snd (fst ?packed)
    let ?data = snd ?packed
    let ?attacker-state = snd (fst x)
    let ?s =
      verifier-state-from-adversary ?attacker-state
      (staged-proof-transcript ?data)
    from hit obtain result final-state trace-table composition-table as
      query-idxs dg composition-fri-roots final rest trace-tree
      composition-tree where
      verify-out:
        Some (result, final-state)  $\in$  set-dist (execute verify-monad ?s)
      and bound:
        accepted-with-bound-tables ?s (Some (result, final-state))
        trace-table composition-table as query-idxs
      and header:
        verifier-header-transcript ?s
        (staged-trace-root ?data)
        (staged-trace-fri-roots ?data)
        (staged-trace-final ?data)
        as dg composition-fri-roots final rest
      and trace-created:
        created-tree trace-table trace-tree final-state
      and composition-created:
        created-tree composition-table composition-tree final-state
      and tree-hit:
        hash-map-new-output-hit
        (set-tree trace-tree  $\cup$  set-tree composition-tree)
        ?prefix-state final-state
    unfolding out-eq

```

```

    checked-staged-security-with-actual-alpha-prefix-tree-output-hit-def
  Let-def
apply simp
by blast
have split:
  hash-map-new-output-hit
    {value trace-tree, value composition-tree}
    ?prefix-state final-state  $\vee$ 
    ( $\exists l\ v\ r.$  trace-tree =  $\langle l, v, r \rangle \wedge$ 
      hash-map-new-output-hit (set-tree l  $\cup$  set-tree r)
      ?prefix-state final-state)  $\vee$ 
    ( $\exists l\ v\ r.$  composition-tree =  $\langle l, v, r \rangle \wedge$ 
      hash-map-new-output-hit (set-tree l  $\cup$  set-tree r)
      ?prefix-state final-state)
  by (rule hash-map-new-output-hit-two-trees-root-or-subtree
    [OF tree-hit])
from split show ?thesis
proof
  assume root-hit:
    hash-map-new-output-hit
      {value trace-tree, value composition-tree}
      ?prefix-state final-state
  have checked-staged-security-with-actual-alpha-prefix-tree-root-output-hit
    out
  unfolding out-eq
    checked-staged-security-with-actual-alpha-prefix-tree-root-output-hit-def
    checked-staged-security-with-actual-alpha-prefix-tree-output-case-def
    Let-def
  using verify-out bound header trace-created composition-created root-hit
  apply simp
  by blast
  then show ?thesis by simp
next
  assume subtree-hit:
    ( $\exists l\ v\ r.$  trace-tree =  $\langle l, v, r \rangle \wedge$ 
      hash-map-new-output-hit (set-tree l  $\cup$  set-tree r)
      ?prefix-state final-state)  $\vee$ 
    ( $\exists l\ v\ r.$  composition-tree =  $\langle l, v, r \rangle \wedge$ 
      hash-map-new-output-hit (set-tree l  $\cup$  set-tree r)
      ?prefix-state final-state)
  have
    checked-staged-security-with-actual-alpha-prefix-tree-subtree-output-hit
    out
  unfolding out-eq
    checked-staged-security-with-actual-alpha-prefix-tree-subtree-output-hit-def
    checked-staged-security-with-actual-alpha-prefix-tree-output-case-def
    Let-def
  using verify-out bound header trace-created composition-created
    subtree-hit

```

```

    apply simp
    by blast
  then show ?thesis by simp
qed
qed

lemma checked-staged-security-with-actual-alpha-prefix-tree-output-hit-bound-from-root-and-subtree:
  assumes root-bound:
    wp-event m
    checked-staged-security-with-actual-alpha-prefix-tree-root-output-hit s
    ≤ R
  and subtree-bound:
    wp-event m
    checked-staged-security-with-actual-alpha-prefix-tree-subtree-output-hit s
    ≤ S
  shows
    wp-event m checked-staged-security-with-actual-alpha-prefix-tree-output-hit
    s ≤ R + S
proof −
  have
    wp-event m checked-staged-security-with-actual-alpha-prefix-tree-output-hit
    s ≤
    wp-event m
    (λout.
      checked-staged-security-with-actual-alpha-prefix-tree-root-output-hit
        out ∨
        checked-staged-security-with-actual-alpha-prefix-tree-subtree-output-hit
        out)
    s
  by (rule wp-event-mono)
    (rule
      checked-staged-security-with-actual-alpha-prefix-tree-output-hit-imp-root-or-subtree)
  also have ... ≤
    wp-event m
    checked-staged-security-with-actual-alpha-prefix-tree-root-output-hit s +
    wp-event m
    checked-staged-security-with-actual-alpha-prefix-tree-subtree-output-hit
    s
  by (rule wp-event-union-bound)
  also have ... ≤ R + S
  by (rule add-mono[OF root-bound subtree-bound])
  finally show ?thesis .
qed

definition checked-staged-security-with-actual-alpha-prefix-transcript-root-output-hit
  where
    checked-staged-security-with-actual-alpha-prefix-transcript-root-output-hit
    out ↔
    (case out of

```

```

None  $\Rightarrow$  False
| Some (x, -)  $\Rightarrow$ 
  (let packed = fst (fst x);
   prefix-state = snd (fst packed);
   data = snd packed;
   attacker-state = snd (fst x);
   s =
     verifier-state-from-adversary attacker-state
     (staged-proof-transcript data)
  in  $\exists$  result final-state trace-table composition-table as
    query-idxs dg composition-fri-roots final rest.
  Some (result, final-state)  $\in$ 
    set-dist (execute verify-monad s)  $\wedge$ 
    accepted-with-bound-tables s (Some (result, final-state))
    trace-table composition-table as query-idxs  $\wedge$ 
    verifier-header-transcript s
    (staged-trace-root data)
    (staged-trace-fri-roots data)
    (staged-trace-final data)
    as dg composition-fri-roots final rest  $\wedge$ 
    hash-map-new-output-hit
    {staged-trace-root data, hd composition-fri-roots}
    prefix-state final-state))

```

lemma *checked-staged-security-with-actual-alpha-prefix-tree-root-output-hit-imp-transcript-root-output-hit:*

assumes *root-hit:*

checked-staged-security-with-actual-alpha-prefix-tree-root-output-hit out

shows

checked-staged-security-with-actual-alpha-prefix-transcript-root-output-hit out

proof (*cases out*)

case *None*

then show *?thesis*

using *root-hit*

unfolding

checked-staged-security-with-actual-alpha-prefix-tree-root-output-hit-def

checked-staged-security-with-actual-alpha-prefix-tree-output-case-def

by *simp*

next

case (*Some result-pack*)

then obtain *x checked-final-state* **where** *out-eq:*

out = Some (x, checked-final-state)

by (*cases result-pack*) *simp*

let *?packed = fst (fst x)*

let *?prefix-state = snd (fst ?packed)*

let *?data = snd ?packed*

let *?attacker-state = snd (fst x)*

let *?s =*

verifier-state-from-adversary ?attacker-state

```

    (staged-proof-transcript ?data)
from root-hit obtain result final-state trace-table composition-table as
    query-idxs dg composition-fri-roots final rest trace-tree
    composition-tree where
verify-out:
    Some (result, final-state) ∈ set-dist (execute verify-monad ?s)
and bound:
    accepted-with-bound-tables ?s (Some (result, final-state))
    trace-table composition-table as query-idxs
and header:
    verifier-header-transcript ?s
    (staged-trace-root ?data)
    (staged-trace-fri-roots ?data)
    (staged-trace-final ?data)
    as dg composition-fri-roots final rest
and trace-created:
    created-tree trace-table trace-tree final-state
and composition-created:
    created-tree composition-table composition-tree final-state
and tree-root-hit:
    hash-map-new-output-hit
    {value trace-tree, value composition-tree}
    ?prefix-state final-state
unfolding out-eq
    checked-staged-security-with-actual-alpha-prefix-tree-root-output-hit-def
    checked-staged-security-with-actual-alpha-prefix-tree-output-case-def
    Let-def
apply simp
by blast
from bound obtain fr' f-fri-roots' f-final' dg'
    composition-fri-roots' final' rest' where
header':
    verifier-header-transcript ?s fr' f-fri-roots' f-final' as dg'
    composition-fri-roots' final' rest'
and trace-bind:
    merkle-root-binds-table fr' trace-table final-state
and composition-bind:
    merkle-root-binds-table (hd composition-fri-roots')
    composition-table final-state
unfolding accepted-with-bound-tables-def by auto
have header-eq:
    fr' = staged-trace-root ?data ∧
    f-fri-roots' = staged-trace-fri-roots ?data ∧
    f-final' = staged-trace-final ?data ∧
    dg' = dg ∧
    composition-fri-roots' = composition-fri-roots ∧
    final' = final ∧
    rest' = rest
using verifier-header-transcript-unique[OF header' header]

```

```

    by simp
  then have fr-eq:  $fr' = staged\_trace\_root \ ?data$ 
    and comp-roots-eq:
       $composition\_fri\_roots' = composition\_fri\_roots$ 
    by simp-all
  from trace-bind obtain trace-root-tree where
    trace-root-created:
       $created\_tree \ trace\_table \ trace\_root\_tree \ final\_state$ 
    and trace-root-eq:  $fr' = value \ trace\_root\_tree$ 
    unfolding merkle-root-binds-table-def by blast
  have trace-value-eq:
     $value \ trace\_tree = staged\_trace\_root \ ?data$ 
    using created-tree-same-table-value-eq
      [OF trace-created trace-root-created]
    trace-root-eq fr-eq
    by simp
  from composition-bind obtain composition-root-tree where
    composition-root-created:
       $created\_tree \ composition\_table \ composition\_root\_tree \ final\_state$ 
    and composition-root-eq:
       $hd \ composition\_fri\_roots' = value \ composition\_root\_tree$ 
    unfolding merkle-root-binds-table-def by blast
  have composition-value-eq:
     $value \ composition\_tree = hd \ composition\_fri\_roots$ 
    using created-tree-same-table-value-eq
      [OF composition-created composition-root-created]
    composition-root-eq comp-roots-eq
    by simp
  have transcript-root-hit:
    hash-map-new-output-hit
      { $staged\_trace\_root \ ?data, hd \ composition\_fri\_roots$ }
      ?prefix-state final-state
    using tree-root-hit trace-value-eq composition-value-eq by simp
  show ?thesis
    unfolding out-eq
      checked-staged-security-with-actual-alpha-prefix-transcript-root-output-hit-def
      Let-def
    using verify-out bound header transcript-root-hit
    apply simp
    by blast
qed

lemma checked-staged-security-with-actual-alpha-prefix-tree-root-output-hit-bound-from-transcript-roots:
  assumes root-bound:
    wp-event m
      checked-staged-security-with-actual-alpha-prefix-transcript-root-output-hit
       $s \leq R$ 
  shows
    wp-event m

```

checked-staged-security-with-actual-alpha-prefix-tree-root-output-hit s
 $\leq R$
by (*rule order.trans*[*OF* - *root-bound*])
 (*rule wp-event-mono*,
rule
checked-staged-security-with-actual-alpha-prefix-tree-root-output-hit-imp-transcript-root-output-hit)

definition

checked-staged-security-with-actual-alpha-prefix-transcript-root-preverifier-output-hit

where

checked-staged-security-with-actual-alpha-prefix-transcript-root-preverifier-output-hit
out $\longleftrightarrow$
 (case *out* of
 None $\Rightarrow$ *False*
 | *Some* (*x*, -) $\Rightarrow$
 (let *packed* = *fst* (*fst* *x*);
 prefix-state = *snd* (*fst* *packed*);
 data = *snd* *packed*;
 attacker-state = *snd* (*fst* *x*);
 s =
 verifier-state-from-adversary attacker-state
 (*staged-proof-transcript data*)
 in $\exists$ *result final-state trace-table composition-table* as
 query-idxs dg composition-fri-roots final rest.
 Some (*result*, *final-state*) $\in$
 set-dist (*execute verify-monad s*) $\wedge$
 accepted-with-bound-tables s (*Some* (*result*, *final-state*))
 trace-table composition-table as *query-idxs* $\wedge$
 verifier-header-transcript s
 (*staged-trace-root data*)
 (*staged-trace-fri-roots data*)
 (*staged-trace-final data*)
 as *dg composition-fri-roots final rest* $\wedge$
 hash-map-new-output-hit
 {*staged-trace-root data*, *hd composition-fri-roots*}
 prefix-state s))

definition

checked-staged-security-with-actual-alpha-prefix-transcript-root-verifier-output-hit

where

checked-staged-security-with-actual-alpha-prefix-transcript-root-verifier-output-hit
out $\longleftrightarrow$
 (case *out* of
 None $\Rightarrow$ *False*
 | *Some* (*x*, -) $\Rightarrow$
 (let *packed* = *fst* (*fst* *x*);
 data = *snd* *packed*;
 attacker-state = *snd* (*fst* *x*);
 s =

verifier-state-from-adversary attacker-state
(staged-proof-transcript data)
in $\exists$ *result final-state trace-table composition-table as*
query-idxs dg composition-fri-roots final rest.
Some (result, final-state) ∈
set-dist (execute verify-monad s) ∧
accepted-with-bound-tables s (Some (result, final-state))
trace-table composition-table as query-idxs ∧
verifier-header-transcript s
(staged-trace-root data)
(staged-trace-fri-roots data)
(staged-trace-final data)
as dg composition-fri-roots final rest ∧
hash-map-new-output-hit
{staged-trace-root data, hd composition-fri-roots}
s final-state))

lemma *checked-staged-security-with-actual-alpha-prefix-transcript-root-output-hit-imp-preverifier-or-verifier-on-suppo*

assumes *wf: staged-budget-wellformed budgets*

and *controlled: staged-adversary-controlled budgets A*

and *support:*

out ∈
set-dist
(execute
(checked-staged-security-experiment-with-actual-alpha-prefix-data-state
A)
adversary-initial-state)

and *hit:*

checked-staged-security-with-actual-alpha-prefix-transcript-root-output-hit
out

shows

checked-staged-security-with-actual-alpha-prefix-transcript-root-preverifier-output-hit
out ∨
checked-staged-security-with-actual-alpha-prefix-transcript-root-verifier-output-hit
out

proof *(cases out)*

case *None*

then show *?thesis*

using *hit*

unfolding

checked-staged-security-with-actual-alpha-prefix-transcript-root-output-hit-def

by *simp*

next

case *(Some result-pack)*

then obtain *prefix prefix-state data attacker-state checked-result*

checked-final-state **where** *out-eq:*

out =
Some (((((prefix, prefix-state), data), attacker-state),
checked-result), checked-final-state)


```

    by (cases result-pack, auto split: prod.splits)
  let ?s =
    verifier-state-from-adversary attacker-state
      (staged-proof-transcript data)
  have support-some:
    Some (((((prefix, prefix-state), data), attacker-state),
      checked-result), checked-final-state) ∈
    set-dist
      (execute
        (checked-staged-security-experiment-with-actual-alpha-prefix-data-state
          A)
        adversary-initial-state)
  using support unfolding out-eq .
  from support-some obtain trans-out where trans-out:
    Some (((((prefix, prefix-state), data), attacker-state) ∈
    set-dist
      (execute (checked-staged-transcript-with-alpha-prefix-program A)
        adversary-initial-state)
  unfolding
    checked-staged-security-experiment-with-actual-alpha-prefix-data-state-def
  by (auto elim!: set-dist-bindE split: prod.splits)
  from checked-staged-transcript-with-alpha-prefix-program-support
    [OF wf controlled trans-out]
  have prefix-ext: prefix-state ≤ attacker-state
  by blast
  have prefix-s: prefix-state ≤ ?s
  by (rule
    alpha-prefix-hash-extends-verifier-state-from-adversary-right
    [OF prefix-ext])
  from hit obtain result final-state trace-table composition-table as
    query-idxs dg composition-fri-roots final rest where
  verify-out:
    Some (result, final-state) ∈ set-dist (execute verify-monad ?s)
  and bound:
    accepted-with-bound-tables ?s (Some (result, final-state))
    trace-table composition-table as query-idxs
  and header:
    verifier-header-transcript ?s
      (staged-trace-root data)
      (staged-trace-fri-roots data)
      (staged-trace-final data)
      as dg composition-fri-roots final rest
  and root-hit:
    hash-map-new-output-hit
      {staged-trace-root data, hd composition-fri-roots}
      prefix-state final-state
  unfolding out-eq
    checked-staged-security-with-actual-alpha-prefix-transcript-root-output-hit-def
  Let-def

```

```

    by simp blast
  have s-final: ?s ≤ final-state
    by (rule verify-monad-hash-extends[OF verify-out])
  from hash-map-new-output-hit-trans-decomp
    [OF prefix-s s-final root-hit]
  show ?thesis
  proof
    assume pre:
      hash-map-new-output-hit
      {staged-trace-root data, hd composition-fri-roots}
      prefix-state ?s
    have
      checked-staged-security-with-actual-alpha-prefix-transcript-root-preverifier-output-hit
        out
      unfolding out-eq
        checked-staged-security-with-actual-alpha-prefix-transcript-root-preverifier-output-hit-def
        Let-def
      using verify-out bound header pre
      by simp blast
    then show ?thesis by simp
  next
    assume verifier:
      hash-map-new-output-hit
      {staged-trace-root data, hd composition-fri-roots}
      ?s final-state
    have
      checked-staged-security-with-actual-alpha-prefix-transcript-root-verifier-output-hit
        out
      unfolding out-eq
        checked-staged-security-with-actual-alpha-prefix-transcript-root-verifier-output-hit-def
        Let-def
      using verify-out bound header verifier
      by simp blast
    then show ?thesis by simp
  qed
qed

lemma verifier-header-transcript-initial-roots-subset-transcript:
  assumes header:
    verifier-header-transcript s fr f-fri-roots f-final as dg
    composition-fri-roots final rest
  and nonempty: composition-fri-roots ≠ []
  shows {fr, hd composition-fri-roots} ⊆ set (PTranscript s)
  using header nonempty
  unfolding verifier-header-transcript-def verifier-header-messages-def
  by (cases composition-fri-roots) auto

```

```

definition
  checked-staged-security-with-actual-alpha-prefix-verifier-transcript-output-hit

```

where

checked-staged-security-with-actual-alpha-prefix-verifier-transcript-output-hit
out $\longleftrightarrow$
 (case *out* of
 None $\Rightarrow$ *False*
 | *Some* (*x*, -) $\Rightarrow$
 (let *packed* = *fst* (*fst* *x*);
 data = *snd* *packed*;
 attacker-state = *snd* (*fst* *x*);
 s =
 verifier-state-from-adversary attacker-state
 (*staged-proof-transcript data*)
 in $\exists$ *result final-state trace-table composition-table as*
 query-idxs.
 Some (*result, final-state*) $\in$
 set-dist (*execute verify-monad s*) $\wedge$
 accepted-with-bound-tables s (*Some* (*result, final-state*))
 trace-table composition-table as query-idxs $\wedge$
 hash-map-new-output-hit (*set* (*PTranscript s*)) *s final-state*))

lemma *checked-staged-security-with-actual-alpha-prefix-transcript-root-verifier-output-hit-imp-verifier-transcript-outp*

assumes *hit*:

checked-staged-security-with-actual-alpha-prefix-transcript-root-verifier-output-hit
out

shows

checked-staged-security-with-actual-alpha-prefix-verifier-transcript-output-hit
out

proof (cases *out*)

case *None*

then show *?thesis*

using *hit*

unfolding

checked-staged-security-with-actual-alpha-prefix-transcript-root-verifier-output-hit-def

by *simp*

next

case (*Some result-pack*)

then obtain *x checked-final-state* **where** *out-eq*:

out = *Some* (*x, checked-final-state*)

by (cases *result-pack*) *simp*

let *?packed* = *fst* (*fst* *x*)

let *?data* = *snd* *?packed*

let *?attacker-state* = *snd* (*fst* *x*)

let *?s* =

verifier-state-from-adversary ?attacker-state
 (*staged-proof-transcript ?data*)

from *hit* **obtain** *result final-state trace-table composition-table as*

query-idxs dg composition-fri-roots final rest **where**

verify-out:

Some (*result, final-state*) $\in$ *set-dist* (*execute verify-monad ?s*)

```

and bound:
  accepted-with-bound-tables ?s (Some (result, final-state))
  trace-table composition-table as query-idxs
and header:
  verifier-header-transcript ?s
    (staged-trace-root ?data)
    (staged-trace-fri-roots ?data)
    (staged-trace-final ?data)
  as dg composition-fri-roots final rest
and root-hit:
  hash-map-new-output-hit
    {staged-trace-root ?data, hd composition-fri-roots}
    ?s final-state
unfolding out-eq
  checked-staged-security-with-actual-alpha-prefix-transcript-root-verifier-output-hit-def
  Let-def
by simp blast
from bound obtain result' final-state' fr' f-fri-roots' f-final' dg'
  composition-fri-roots' final' rest' where
  out-eq':
    Some (result, final-state) = Some (result', final-state')
and header':
  verifier-header-transcript ?s fr' f-fri-roots' f-final' as dg'
  composition-fri-roots' final' rest'
and nonempty': composition-fri-roots' ≠ []
unfolding accepted-with-bound-tables-def by blast
have header-eq:
  fr' = staged-trace-root ?data ∧
  f-fri-roots' = staged-trace-fri-roots ?data ∧
  f-final' = staged-trace-final ?data ∧
  dg' = dg ∧
  composition-fri-roots' = composition-fri-roots ∧
  final' = final ∧
  rest' = rest
using verifier-header-transcript-unique[OF header' header]
by simp
have nonempty: composition-fri-roots ≠ []
using nonempty' header-eq by simp
have subset:
  {staged-trace-root ?data, hd composition-fri-roots}
    ⊆ set (PTranscript ?s)
by (rule verifier-header-transcript-initial-roots-subset-transcript
    [OF header nonempty])
have transcript-hit:
  hash-map-new-output-hit (set (PTranscript ?s)) ?s final-state
by (rule hash-map-new-output-hit-subset[OF subset root-hit])
show ?thesis
unfolding out-eq
  checked-staged-security-with-actual-alpha-prefix-verifier-transcript-output-hit-def

```

```

    Let-def
using verify-out bound transcript-hit
by simp blast
qed

lemma checked-staged-security-with-actual-alpha-prefix-transcript-root-verifier-output-hit-bound-from-verifier-transcr
assumes transcript-bound:
  wp-event m
  checked-staged-security-with-actual-alpha-prefix-verifier-transcript-output-hit
   $s \leq C$ 
shows
  wp-event m
  checked-staged-security-with-actual-alpha-prefix-transcript-root-verifier-output-hit
   $s \leq C$ 
by (rule order.trans[OF - transcript-bound])
  (rule wp-event-mono,
  rule
  checked-staged-security-with-actual-alpha-prefix-transcript-root-verifier-output-hit-imp-verifier-transcript-output-hit)

lemma checked-staged-security-with-actual-alpha-prefix-tree-output-hit-imp-data-state-alpha-prefix-tree-output-hit-on-
assumes wf: staged-budget-wellformed budgets
and controlled: staged-adversary-controlled budgets A
and support:
  out  $\in$ 
  set-dist
  (execute
  (checked-staged-security-experiment-with-actual-alpha-prefix-data-state
  A)
  adversary-initial-state)
and hit: checked-staged-security-with-actual-alpha-prefix-tree-output-hit
  out
shows
  case out of
  None  $\Rightarrow$  False
  | Some (((((prefix, prefix-state), data), attacker-state),
    checked-result), checked-final-state)  $\Rightarrow$ 
    checked-staged-security-with-data-state-alpha-prefix-tree-output-hit A
    (Some (((data, attacker-state), checked-result),
      checked-final-state)))
proof (cases out)
case None
then show ?thesis
  using hit
  unfolding checked-staged-security-with-actual-alpha-prefix-tree-output-hit-def
  by simp
next
case (Some result-pack)
then obtain prefix prefix-state data attacker-state checked-result
  checked-final-state where out-eq:

```

```

    out =
      Some (((((prefix, prefix-state), data), attacker-state),
        checked-result), checked-final-state)
  by (cases result-pack, auto split: prod.splits)
let ?s =
  verifier-state-from-adversary attacker-state
  (staged-proof-transcript data)
from support[unfolded out-eq] obtain trans-out where trans-out:
  Some (((prefix, prefix-state), data), attacker-state) ∈
    set-dist
    (execute (checked-staged-transcript-with-alpha-prefix-program A)
      adversary-initial-state)
  unfolding
    checked-staged-security-experiment-with-actual-alpha-prefix-data-state-def
  by (auto elim!: set-dist-bindE split: prod.splits)
from checked-staged-transcript-with-alpha-prefix-program-support
  [OF wf controlled trans-out]
have prefix-out:
  Some (prefix, prefix-state) ∈
    set-dist
    (execute (staged-alpha-prefix-program A)
      adversary-initial-state)
and root-eq: staged-trace-root data = fst prefix
and trace-roots-eq:
  staged-trace-fri-roots data = fst (snd prefix)
and trace-bs-eq:
  staged-trace-fri-challenges data = fst (snd (snd prefix))
and trace-final-eq:
  staged-trace-final data = snd (snd (snd prefix))
by blast+
have actual-body:
  ∃ result final-state trace-table composition-table as query-idxs dg
    composition-fri-roots final rest trace-tree composition-tree.
  Some (result, final-state) ∈ set-dist (execute verify-monad ?s) ∧
  accepted-with-bound-tables ?s (Some (result, final-state))
  trace-table composition-table as query-idxs ∧
  verifier-header-transcript ?s
  (staged-trace-root data)
  (staged-trace-fri-roots data)
  (staged-trace-final data)
  as dg composition-fri-roots final rest ∧
  created-tree trace-table trace-tree final-state ∧
  created-tree composition-table composition-tree final-state ∧
  hash-map-new-output-hit
  (set-tree trace-tree ∪ set-tree composition-tree)
  prefix-state final-state
using hit
unfolding out-eq
  checked-staged-security-with-actual-alpha-prefix-tree-output-hit-def

```

Let-def
by simp
from *actual-body* **obtain** *result final-state trace-table composition-table as*
query-idxs dg composition-fri-roots final rest trace-tree
composition-tree **where**
verify-out:
Some (result, final-state) ∈ set-dist (execute verify-monad ?s)
and *bound:*
accepted-with-bound-tables ?s (Some (result, final-state))
trace-table composition-table as query-idxs
and *header:*
verifier-header-transcript ?s
(staged-trace-root data)
(staged-trace-fri-roots data)
(staged-trace-final data)
as dg composition-fri-roots final rest
and *trace-created: created-tree trace-table trace-tree final-state*
and *composition-created:*
created-tree composition-table composition-tree final-state
and *tree-hit:*
hash-map-new-output-hit
(set-tree trace-tree ∪ set-tree composition-tree)
prefix-state final-state
by blast
have *data-hit:*
checked-staged-security-with-data-state-alpha-prefix-tree-output-hit A
(Some (((data, attacker-state), checked-result),
checked-final-state))
by (*rule*
checked-staged-security-with-data-state-alpha-prefix-tree-output-hitI
[OF prefix-out root-eq trace-roots-eq trace-bs-eq trace-final-eq
verify-out bound header trace-created composition-created
tree-hit]))
show *?thesis*
unfolding *out-eq*
using *data-hit* **by simp**
qed

lemma *checked-staged-security-with-actual-alpha-prefix-tree-output-hit-bound-from-data-state:*
assumes *wf: staged-budget-wellformed budgets*
and *controlled: staged-adversary-controlled budgets A*
and *data-bound:*
wp-event (checked-staged-security-experiment-with-data-state A)
(checked-staged-security-with-data-state-alpha-prefix-tree-output-hit A)
adversary-initial-state ≤ T
shows
wp-event
(checked-staged-security-experiment-with-actual-alpha-prefix-data-state A)
checked-staged-security-with-actual-alpha-prefix-tree-output-hit

```

    adversary-initial-state ≤ T
proof –
  let ?M =
    checked-staged-security-experiment-with-actual-alpha-prefix-data-state A
  let ?project =
    λx. ((snd (fst (fst x)), snd (fst x)), snd x)
  let ?DataTree =
    checked-staged-security-with-data-state-alpha-prefix-tree-output-hit A
  have wp-event ?M checked-staged-security-with-actual-alpha-prefix-tree-output-hit
    adversary-initial-state ≤
  wp-event ?M
    (λout. case out of
      None ⇒ False
    | Some (x, t) ⇒ ?DataTree (Some (?project x, t)))
    adversary-initial-state
  proof (rule wp-event-mono-on-support)
  fix out
  assume support:
    out ∈ set-dist (execute ?M adversary-initial-state)
  and hit:
    checked-staged-security-with-actual-alpha-prefix-tree-output-hit out
  show
    (case out of None ⇒ False
    | Some (x, t) ⇒ ?DataTree (Some (?project x, t)))
  using
    checked-staged-security-with-actual-alpha-prefix-tree-output-hit-imp-data-state-alpha-prefix-tree-output-hit-on-sup
    [OF wf controlled support hit]
  by (cases out) (auto split: prod.splits)
qed
also have ... =
  wp-event (?M ≫ (λx. return (?project x))) ?DataTree
    adversary-initial-state
  unfolding
    checked-staged-security-with-data-state-alpha-prefix-tree-output-hit-def
  by (subst wp-event-bind-return-map)
    (simp split: option.splits prod.splits)
also have ... =
  wp-event (checked-staged-security-experiment-with-data-state A)
    ?DataTree adversary-initial-state
  using
    checked-staged-security-experiment-with-actual-alpha-prefix-data-state-projection
    [of A]
  by simp
also have ... ≤ T
  by (rule data-bound)
  finally show ?thesis .
qed

```

Branch-specific tree-output events.

The actual-prefix tree-output events are existential over verifier witnesses for a fixed prefix. They are useful for deterministic reductions, but too coarse for direct verifier-local WP bounds. The following auxiliary events tie the witness to the current verifier branch of the checked experiment, so target-budget lemmas can be applied without changing the public theorem interface.

definition

checked-staged-security-with-actual-alpha-prefix-branch-verifier-transcript-output-hit

where

checked-staged-security-with-actual-alpha-prefix-branch-verifier-transcript-output-hit

out $\longleftrightarrow$

(case out of

None $\Rightarrow$ *False*

| *Some* (*x*, *final-state*) $\Rightarrow$

(let *packed* = *fst* (*fst* *x*);

data = *snd* *packed*;

attacker-state = *snd* (*fst* *x*);

result = *snd* *x*;

s =

verifier-state-from-adversary attacker-state

(*staged-proof-transcript data*)

in $\exists$ *trace-table composition-table as query-idxs*.

Some (*result*, *final-state*) $\in$

set-dist (*execute verify-monad s*) $\wedge$

accepted-with-bound-tables s (*Some* (*result*, *final-state*))

trace-table composition-table as query-idxs $\wedge$

hash-map-new-output-hit (*set* (*PTranscript s*)) *s final-state*)

definition

checked-staged-security-with-actual-alpha-prefix-branch-transcript-root-verifier-output-hit

where

checked-staged-security-with-actual-alpha-prefix-branch-transcript-root-verifier-output-hit

out $\longleftrightarrow$

(case out of

None $\Rightarrow$ *False*

| *Some* (*x*, *final-state*) $\Rightarrow$

(let *packed* = *fst* (*fst* *x*);

data = *snd* *packed*;

attacker-state = *snd* (*fst* *x*);

result = *snd* *x*;

s =

verifier-state-from-adversary attacker-state

(*staged-proof-transcript data*)

in $\exists$ *trace-table composition-table as query-idxs dg*

composition-fri-roots final rest.

Some (*result*, *final-state*) $\in$

set-dist (*execute verify-monad s*) $\wedge$

accepted-with-bound-tables s (*Some* (*result*, *final-state*))

trace-table composition-table as query-idxs $\wedge$

```

    verifier-header-transcript s
      (staged-trace-root data)
      (staged-trace-fri-roots data)
      (staged-trace-final data)
      as dg composition-fri-roots final rest ∧
hash-map-new-output-hit
  {staged-trace-root data, hd composition-fri-roots}
  s final-state))

```

lemma *branch-verifier-transcript-output-hit-imp-verifier-transcript-output-hit:*
assumes *hit:*
 checked-staged-security-with-actual-alpha-prefix-branch-verifier-transcript-output-hit
 out
shows
 checked-staged-security-with-actual-alpha-prefix-verifier-transcript-output-hit
 out
using *hit*
unfolding
 checked-staged-security-with-actual-alpha-prefix-branch-verifier-transcript-output-hit-def
 checked-staged-security-with-actual-alpha-prefix-verifier-transcript-output-hit-def
 Let-def
by (*cases out*) (*fastforce split: prod.splits*)+

lemma *branch-transcript-root-verifier-output-hit-imp-branch-verifier-transcript-output-hit:*
assumes *hit:*
 checked-staged-security-with-actual-alpha-prefix-branch-transcript-root-verifier-output-hit
 out
shows
 checked-staged-security-with-actual-alpha-prefix-branch-verifier-transcript-output-hit
 out
proof (*cases out*)
case *None*
then show *?thesis*
 using *hit*
 unfolding
 checked-staged-security-with-actual-alpha-prefix-branch-transcript-root-verifier-output-hit-def
 by *simp*
next
case (*Some result-pack*)
then obtain *x final-state* **where** *out-eq: out = Some (x, final-state)*
 by (*cases result-pack*) *simp*
let *?packed = fst (fst x)*
let *?data = snd ?packed*
let *?attacker-state = snd (fst x)*
let *?result = snd x*
let *?s =*
 verifier-state-from-adversary ?attacker-state
 (*staged-proof-transcript ?data*)
from *hit* **obtain** *trace-table composition-table as query-idxs dg*

```

    composition-fri-roots final rest where
  verify-out:
    Some (?result, final-state) ∈ set-dist (execute verify-monad ?s)
and bound:
    accepted-with-bound-tables ?s (Some (?result, final-state))
    trace-table composition-table as query-idxs
and header:
    verifier-header-transcript ?s
    (staged-trace-root ?data)
    (staged-trace-fri-roots ?data)
    (staged-trace-final ?data)
    as dg composition-fri-roots final rest
and root-hit:
    hash-map-new-output-hit
    {staged-trace-root ?data, hd composition-fri-roots}
    ?s final-state
unfolding out-eq
    checked-staged-security-with-actual-alpha-prefix-branch-transcript-root-verifier-output-hit-def
    Let-def
by simp blast
from bound obtain result' final-state' fr' f-fri-roots' f-final' dg'
    composition-fri-roots' final' rest' where
  out-eq':
    Some (?result, final-state) = Some (result', final-state')
and header':
    verifier-header-transcript ?s fr' f-fri-roots' f-final' as dg'
    composition-fri-roots' final' rest'
and nonempty': composition-fri-roots' ≠ []
unfolding accepted-with-bound-tables-def by blast
have header-eq:
  fr' = staged-trace-root ?data ∧
  f-fri-roots' = staged-trace-fri-roots ?data ∧
  f-final' = staged-trace-final ?data ∧
  dg' = dg ∧
  composition-fri-roots' = composition-fri-roots ∧
  final' = final ∧
  rest' = rest
using verifier-header-transcript-unique[OF header' header]
by simp
have nonempty: composition-fri-roots ≠ []
using nonempty' header-eq by simp
have subset:
  {staged-trace-root ?data, hd composition-fri-roots}
  ⊆ set (PTranscript ?s)
by (rule verifier-header-transcript-initial-roots-subset-transcript
    [OF header nonempty])
have transcript-hit:
  hash-map-new-output-hit (set (PTranscript ?s)) ?s final-state
by (rule hash-map-new-output-hit-subset[OF subset root-hit])

```

```

show ?thesis
  unfolding out-eq
    checked-staged-security-with-actual-alpha-prefix-branch-verifier-transcript-output-hit-def
    Let-def
  using verify-out bound transcript-hit
  by simp blast
qed

lemma branch-transcript-root-verifier-output-hit-bound-from-branch-verifier-transcript-output:
  assumes transcript-bound:
    wp-event m
    checked-staged-security-with-actual-alpha-prefix-branch-verifier-transcript-output-hit
     $s \leq C$ 
  shows
    wp-event m
    checked-staged-security-with-actual-alpha-prefix-branch-transcript-root-verifier-output-hit
     $s \leq C$ 
by (rule order.trans[OF - transcript-bound])
  (rule wp-event-mono,
   rule
    branch-transcript-root-verifier-output-hit-imp-branch-verifier-transcript-output-hit)

lemma actual-alpha-prefix-branch-verifier-transcript-output-cont-imp-hash-new-output-hit:
  fixes packed and attacker-state
  defines s  $\equiv$ 
    verifier-state-from-adversary attacker-state
    (staged-proof-transcript (snd packed))
  assumes hit:
    (case out of
     None  $\Rightarrow$ 
       checked-staged-security-with-actual-alpha-prefix-branch-verifier-transcript-output-hit
       None
     | Some (result, t)  $\Rightarrow$ 
       checked-staged-security-with-actual-alpha-prefix-branch-verifier-transcript-output-hit
       (Some (((packed, attacker-state), result), t)))
  shows hash-new-output-hit-event (set (PTranscript s)) s out
proof (cases out)
case None
then show ?thesis
  using hit
  unfolding
    checked-staged-security-with-actual-alpha-prefix-branch-verifier-transcript-output-hit-def
  by simp
next
case (Some result-state)
then obtain result t where result-state: result-state = (result, t)
  by (cases result-state) simp
have hash-map-new-output-hit (set (PTranscript s)) s t
  using hit Some result-state

```

```

unfolding
  checked-staged-security-with-actual-alpha-prefix-branch-verifier-transcript-output-hit-def
  s-def Let-def
by (auto split: prod.splits)
then show ?thesis
using Some result-state
unfolding hash-new-output-hit-event-def by simp
qed

lemma wp-verify-monad-bind-return-actual-alpha-prefix-branch-verifier-transcript-output-hit-bound:
fixes packed and attacker-state
defines s  $\equiv$ 
  verifier-state-from-adversary attacker-state
  (staged-proof-transcript (snd packed))
shows
  wp-event
  (verify-monad  $\gg$ 
    ( $\lambda$ result. return ((packed, attacker-state), result)))
  checked-staged-security-with-actual-alpha-prefix-branch-verifier-transcript-output-hit
  s  $\leq$ 
  hash-target-budget-value (set (staged-proof-transcript (snd packed)))
  verifier-hash-query-budget
proof –
have mono:
  wp-event verify-monad
  ( $\lambda$ out. case out of
    None  $\Rightarrow$ 
      checked-staged-security-with-actual-alpha-prefix-branch-verifier-transcript-output-hit
      None
    | Some (x, t)  $\Rightarrow$ 
      checked-staged-security-with-actual-alpha-prefix-branch-verifier-transcript-output-hit
      (Some (((packed, attacker-state), x), t))) s  $\leq$ 
  wp-event verify-monad
  (hash-new-output-hit-event (set (PTranscript s)) s) s
proof (rule wp-event-mono)
fix out
assume hit:
  (case out of
    None  $\Rightarrow$ 
      checked-staged-security-with-actual-alpha-prefix-branch-verifier-transcript-output-hit
      None
    | Some (x, t)  $\Rightarrow$ 
      checked-staged-security-with-actual-alpha-prefix-branch-verifier-transcript-output-hit
      (Some (((packed, attacker-state), x), t)))
show hash-new-output-hit-event (set (PTranscript s)) s out
using hit
unfolding
  checked-staged-security-with-actual-alpha-prefix-branch-verifier-transcript-output-hit-def
  hash-new-output-hit-event-def s-def Let-def

```

```

    by (cases out) (auto split: prod.splits)
qed
have target:
  wp-event verify-monad
    (hash-new-output-hit-event (set (PTranscript s)) s) s ≤
    hash-target-budget-value (set (PTranscript s))
    verifier-hash-query-budget
  by (rule wp-verify-monad-hash-new-output-hit-bound)
have cont-bound:
  wp-event verify-monad
    (λout. case out of
      None ⇒
        checked-staged-security-with-actual-alpha-prefix-branch-verifier-transcript-output-hit
        None
      | Some (x, t) ⇒
        checked-staged-security-with-actual-alpha-prefix-branch-verifier-transcript-output-hit
        (Some (((packed, attacker-state), x), t))) s ≤
        hash-target-budget-value (set (staged-proof-transcript (snd packed)))
        verifier-hash-query-budget
    using order.trans[OF mono target]
    unfolding s-def by simp
show ?thesis
  apply (subst wp-event-bind-return-map)
  using cont-bound
  apply (simp split: option.splits prod.splits)
done
qed

lemma wp-verify-monad-bind-return-actual-alpha-prefix-branch-transcript-root-verifier-output-hit-bound:
  fixes packed and attacker-state
  defines s ≡
    verifier-state-from-adversary attacker-state
    (staged-proof-transcript (snd packed))
  shows
    wp-event
      (verify-monad >=>
        (λresult. return ((packed, attacker-state), result)))
      checked-staged-security-with-actual-alpha-prefix-branch-transcript-root-verifier-output-hit
      s ≤
      hash-target-budget-value (set (staged-proof-transcript (snd packed)))
      verifier-hash-query-budget
proof -
  have transcript-bound:
    wp-event
      (verify-monad >=>
        (λresult. return ((packed, attacker-state), result)))
      checked-staged-security-with-actual-alpha-prefix-branch-verifier-transcript-output-hit
      s ≤
      hash-target-budget-value (set (staged-proof-transcript (snd packed)))

```

```

    verifier-hash-query-budget
  unfolding s-def
  by (rule
    wp-verify-monad-bind-return-actual-alpha-prefix-branch-verifier-transcript-output-hit-bound)
  show ?thesis
  by (rule
    branch-transcript-root-verifier-output-hit-bound-from-branch-verifier-transcript-output
    [OF transcript-bound])
qed

lemma checked-staged-security-with-actual-alpha-prefix-branch-transcript-root-verifier-output-hit-bound-from-transcr
  assumes target-bound:
     $\bigwedge \text{packed attacker-state.}$ 
    Some (packed, attacker-state)  $\in$ 
      set-dist
        (execute (checked-staged-transcript-with-alpha-prefix-program A)
          adversary-initial-state)  $\implies$ 
        hash-target-budget-value (set (staged-proof-transcript (snd packed)))
          verifier-hash-query-budget  $\leq C$ 
  shows
    wp-event
      (checked-staged-security-experiment-with-actual-alpha-prefix-data-state A)
      checked-staged-security-with-actual-alpha-prefix-branch-transcript-root-verifier-output-hit
      adversary-initial-state  $\leq C$ 
  unfolding checked-staged-security-experiment-with-actual-alpha-prefix-data-state-def
  proof (rule wp-event-bind-bound-by-cont)
  show
     $\neg$ 
      checked-staged-security-with-actual-alpha-prefix-branch-transcript-root-verifier-output-hit
      None
  unfolding
    checked-staged-security-with-actual-alpha-prefix-branch-transcript-root-verifier-output-hit-def
  by simp
next
  fix packed attacker-state
  assume support:
    Some (packed, attacker-state)  $\in$ 
      set-dist
        (execute (checked-staged-transcript-with-alpha-prefix-program A)
          adversary-initial-state)
  let ?s =
    verifier-state-from-adversary attacker-state
    (staged-proof-transcript (snd packed))
  have cont-eq:
    wp-event
      (get  $\gg=$ 
        ( $\lambda s.$  put
          (verifier-state-from-adversary s
            (staged-proof-transcript (snd packed))))  $\gg=$ 

```

```

      (λ-. verify-monad >>=
        (λresult. return ((packed, s), result))))
checked-staged-security-with-actual-alpha-prefix-branch-transcript-root-verifier-output-hit
attacker-state =
wp-event
(verify-monad >>=
  (λresult. return ((packed, attacker-state), result)))
checked-staged-security-with-actual-alpha-prefix-branch-transcript-root-verifier-output-hit
?s
unfolding wp-event-def by (simp add: wpsimps)
have verifier-bound:
wp-event
(verify-monad >>=
  (λresult. return ((packed, attacker-state), result)))
checked-staged-security-with-actual-alpha-prefix-branch-transcript-root-verifier-output-hit
?s ≤
hash-target-budget-value (set (staged-proof-transcript (snd packed)))
  verifier-hash-query-budget
by (rule
  wp-verify-monad-bind-return-actual-alpha-prefix-branch-transcript-root-verifier-output-hit-bound)
show
wp-event
(get >>=
  (λs. put
    (verifier-state-from-adversary s
      (staged-proof-transcript (snd packed))) >>=
      (λ-. verify-monad >>=
        (λresult. return ((packed, s), result))))))
checked-staged-security-with-actual-alpha-prefix-branch-transcript-root-verifier-output-hit
attacker-state ≤ C
unfolding cont-eq
by (rule order.trans[OF verifier-bound target-bound[OF support]])
qed

lemma checked-staged-security-with-actual-alpha-prefix-branch-transcript-root-verifier-output-hit-bound-from-concrete
assumes target-bound:
  ∧packed attacker-state.
  Some (packed, attacker-state) ∈
    set-dist
      (execute (checked-staged-transcript-with-alpha-prefix-program A)
        adversary-initial-state) ⇒
    concrete-transcript-target-error (staged-proof-transcript (snd packed))
      ≤ C
shows
wp-event
  (checked-staged-security-experiment-with-actual-alpha-prefix-data-state A)
checked-staged-security-with-actual-alpha-prefix-branch-transcript-root-verifier-output-hit
adversary-initial-state ≤ C
proof (rule

```



```

checked-staged-security-with-actual-alpha-prefix-branch-transcript-root-verifier-output-hit-bound-from-transcript-target
fix packed attacker-state
assume support:
  Some (packed, attacker-state) ∈
    set-dist
    (execute (checked-staged-transcript-with-alpha-prefix-program A)
      adversary-initial-state)
show
  hash-target-budget-value (set (staged-proof-transcript (snd packed)))
    verifier-hash-query-budget ≤ C
using target-bound[OF support]
unfolding concrete-transcript-target-error-def hash-target-budget-value-def
by (simp add: mult.commute)
qed

end

end

```

```

theory Staged-Security-Experiment-Composition-Bounds
imports Staged-Security-Experiment-Composition-Tree-Output
begin

```

Budget-level composition corollaries for the staged security experiment.

This layer consumes the actual-alpha-prefix prequery accounting theorem and exposes the remaining composition side condition as the tree-output event only. The tree-output event is still Merkle/candidate proof work; the alpha-key prequery accounting itself is derived from the current staged oracle-budget interface.

```

context soundness
begin

```

```

lemma checked-staged-security-with-data-state-composition-bad-bound-from-actual-alpha-prefix-tree-and-budgets:
assumes wf: staged-budget-wellformed budgets
and controlled: staged-adversary-controlled budgets A
and tree-bound:
  wp-event
  (checked-staged-security-experiment-with-actual-alpha-prefix-data-state
    A)
  checked-staged-security-with-actual-alpha-prefix-tree-output-hit
  adversary-initial-state ≤ T
shows
  wp-event (checked-staged-security-experiment-with-data-state A)
  (staged-security-with-data-state-verifier-event composition-bad)
  adversary-initial-state ≤
  composition-error-bound +
  staged-phase-relation-error size
  (staged-alpha-search-queries budgets 0) +
  hash-collision-budget-value 0

```

```

      (staged-alpha-search-queries budgets 0) +
    T
proof -
  let ?P =
    composition-error-bound +
    staged-phase-relation-error size
    (staged-alpha-search-queries budgets 0) +
    hash-collision-budget-value 0
    (staged-alpha-search-queries budgets 0)
  have transcript-bound:
    wp-event (checked-staged-transcript-with-alpha-prefix-program A)
    (checked-staged-transcript-actual-alpha-prefix-bad-set-hit
     composition-trace-bad-alpha-space)
    adversary-initial-state ≤ ?P
  by (rule
    checked-staged-transcript-with-alpha-prefix-actual-bad-set-hit-bound-from-budgets
    [OF wf controlled])
  have prefix-bound:
    wp-event
    (checked-staged-security-experiment-with-actual-alpha-prefix-data-state
     A)
    (checked-staged-security-with-actual-alpha-prefix-bad-set-hit
     composition-trace-bad-alpha-space)
    adversary-initial-state ≤ ?P
  by (rule
    checked-staged-security-with-actual-alpha-prefix-bad-set-hit-bound-from-transcript
    [OF transcript-bound])
  have
    wp-event (checked-staged-security-experiment-with-data-state A)
    (staged-security-with-data-state-verifier-event composition-bad)
    adversary-initial-state ≤ ?P + T
  by (rule
    checked-staged-security-with-data-state-composition-bad-bound-from-actual-alpha-prefix-and-tree
    [OF wf controlled prefix-bound tree-bound])
  then show ?thesis
    by (simp add: add.assoc)
qed

```

```

lemma checked-staged-security-with-data-state-composition-bad-bound-from-data-alpha-prefix-tree-and-budgets:
  assumes wf: staged-budget-wellformed budgets
  and controlled: staged-adversary-controlled budgets A
  and data-tree-bound:
    wp-event (checked-staged-security-experiment-with-data-state A)
    (checked-staged-security-with-data-state-alpha-prefix-tree-output-hit A)
    adversary-initial-state ≤ T
  shows
    wp-event (checked-staged-security-experiment-with-data-state A)
    (staged-security-with-data-state-verifier-event composition-bad)
    adversary-initial-state ≤

```

```

    composition-error-bound +
    staged-phase-relation-error size
    (staged-alpha-search-queries budgets 0) +
    hash-collision-budget-value 0
    (staged-alpha-search-queries budgets 0) +
    T
proof —
  have actual-tree-bound:
    wp-event
    (checked-staged-security-experiment-with-actual-alpha-prefix-data-state A)
    checked-staged-security-with-actual-alpha-prefix-tree-output-hit
    adversary-initial-state ≤ T
  by (rule
    checked-staged-security-with-actual-alpha-prefix-tree-output-hit-bound-from-data-state
    [OF wf controlled data-tree-bound])
  show ?thesis
  by (rule
    checked-staged-security-with-data-state-composition-bad-bound-from-actual-alpha-prefix-tree-and-budgets
    [OF wf controlled actual-tree-bound])
qed

end

end

```

theory *Staged-Security-Experiment-Composition-Branch*
imports *Staged-Security-Experiment-Composition-Bounds*
begin

Branch-specific composition reductions.

This layer introduces branch-tied events for the verifier result actually sampled by the checked experiment. These events are suitable for verifier-local WP bounds and avoid broad existential side events.

context *soundness*
begin

definition *staged-proof-data-of-query-prefix*
 :: 'f staged-query-prefix-data ⇒ 'f staged-proof-data
where
 staged-proof-data-of-query-prefix prefix =
 (|staged-trace-root = sqp-trace-root prefix,
 staged-trace-fri-roots = sqp-trace-fri-roots prefix,
 staged-trace-fri-challenges = sqp-trace-fri-challenges prefix,
 staged-trace-final = sqp-trace-final prefix,
 staged-alphas = sqp-alphas prefix,
 staged-degree = sqp-degree prefix,
 staged-composition-fri-roots = sqp-composition-fri-roots prefix,
 staged-composition-fri-challenges =

sqp-composition-fri-challenges prefix,
staged-composition-final = sqp-composition-final prefix,
staged-query-chunks = sqp-query-chunks prefix)

lemma *checked-staged-transcript-program-query-prefix-map:*

checked-staged-transcript-program A =
checked-staged-query-challenge-prefix-program A rounds $\gg$
(λ prefix. return (staged-proof-data-of-query-prefix prefix))

unfolding *checked-staged-transcript-program-def*

checked-staged-query-challenge-prefix-program-def
staged-alpha-prefix-program-def

staged-proof-data-of-query-prefix-def Let-def

by (*simp add: sm-bind-assoc split-def split: prod.splits*)

lemma *staged-query-search-queries-rounds-eq-total:*

assumes *wf: staged-budget-wellformed budgets*

shows

staged-query-search-queries budgets rounds =
staged-attacker-query-budget budgets + staged-challenge-query-budget

using *wf*

unfolding *staged-query-search-queries-def*

staged-attacker-query-budget-def staged-challenge-query-budget-def
staged-budget-wellformed-def

by *simp*

lemma *hash-relation-program-checked-staged-transcript-program:*

assumes *wf: staged-budget-wellformed budgets*

and *controlled: staged-adversary-controlled budgets A*

and *fibers: $\bigwedge x. \text{card } \{y. R \ x \ y\} \leq b$*

shows

hash-relation-program R b
(staged-attacker-query-budget budgets + staged-challenge-query-budget)
(checked-staged-transcript-program A)

proof —

have *prefix:*

hash-relation-program R b
(staged-query-search-queries budgets rounds)
(checked-staged-query-challenge-prefix-program A rounds)

by (*rule hash-relation-program-checked-staged-query-challenge-prefix-program*
 $[OF \ wf \ controlled - fibers]$)

simp

have *mapped:*

hash-relation-program R b
(staged-query-search-queries budgets rounds + 0)
(checked-staged-query-challenge-prefix-program A rounds $\gg$
(λ prefix. return (staged-proof-data-of-query-prefix prefix)))

by (*rule hash-relation-program-bind*)

(*rule prefix,*

simp add: hash-relation-program-zero[OF hash-map-preserving-return])

have *budget-eq*:
staged-query-search-queries budgets rounds =
staged-attacker-query-budget budgets + staged-challenge-query-budget
by (rule *staged-query-search-queries-rounds-eq-total*[*OF wf*])
show *?thesis*
unfolding *checked-staged-transcript-program-query-prefix-map*
using *mapped budget-eq* **by** *simp*
qed

definition *checked-staged-transcript-initial-roots-relation*
 $:: 'f \text{ staged-adversary} \Rightarrow 'f \text{ protocol-hash-input} \Rightarrow 'f \Rightarrow \text{bool}$
where
checked-staged-transcript-initial-roots-relation A x y $\longleftrightarrow$
 $(\exists \text{ data attacker-state.}$
 $\text{Some (data, attacker-state)} \in$
 set-dist
 $(\text{execute (checked-staged-transcript-program A)}$
 $\text{adversary-initial-state}) \wedge$
 $y \in \{\text{staged-trace-root data,}$
 $\text{hd (staged-composition-fri-roots data)}\})$

lemma *checked-staged-transcript-initial-roots-relation-fiber-bound-size*:
 $\text{card } \{y. \text{checked-staged-transcript-initial-roots-relation } A \ x \ y\}$
 $\leq \text{size}$

proof –
have *subset*:
 $\{y. \text{checked-staged-transcript-initial-roots-relation } A \ x \ y\}$
 $\subseteq (\text{UNIV} :: 'f \text{ set})$
by *simp*
have *finite-univ*: *finite* (*UNIV* :: *'f set*)
by *simp*
have $\text{card } \{y. \text{checked-staged-transcript-initial-roots-relation } A \ x \ y\}$
 $\leq \text{card } (\text{UNIV} :: 'f \text{ set})$
by (rule *card-mono*[*OF finite-univ subset*])
also have $\dots = \text{size}$
using *size-card* **by** *simp*
finally show *?thesis* .
qed

lemma *checked-staged-transcript-initial-roots-relation-hit-bound*:
assumes *wf*: *staged-budget-wellformed budgets*
and *controlled*: *staged-adversary-controlled budgets A*
shows
 $\text{wp-event (checked-staged-transcript-program A)}$
 $(\text{hash-relation-hit-event}$
 $(\text{checked-staged-transcript-initial-roots-relation } A)$
 $\text{adversary-initial-state})$
 $\text{adversary-initial-state} \leq$
 $\text{staged-phase-relation-error size}$

```

      (staged-attacker-query-budget budgets + staged-challenge-query-budget)
proof -
  have program:
    hash-relation-program
      (checked-staged-transcript-initial-roots-relation A)
    size
      (staged-attacker-query-budget budgets + staged-challenge-query-budget)
      (checked-staged-transcript-program A)
  by (rule hash-relation-program-checked-staged-transcript-program
      [OF wf controlled])
      (rule checked-staged-transcript-initial-roots-relation-fiber-bound-size)
  show ?thesis
  using program
  unfolding hash-relation-program-def hash-relation-budget-def
    staged-phase-relation-error-def
  by blast
qed

lemma checked-staged-transcript-with-alpha-prefix-initial-roots-relation-hit-bound:
  assumes wf: staged-budget-wellformed budgets
  and controlled: staged-adversary-controlled budgets A
  shows
    wp-event (checked-staged-transcript-with-alpha-prefix-program A)
      (λout. case out of
        None ⇒ False
        | Some (packed, attacker-state) ⇒
          hash-relation-hit
            (checked-staged-transcript-initial-roots-relation A)
            adversary-initial-state attacker-state)
      adversary-initial-state ≤
      staged-phase-relation-error size
      (staged-attacker-query-budget budgets + staged-challenge-query-budget)
proof -
  let ?R = checked-staged-transcript-initial-roots-relation A
  let ?H =
    λout. case out of
      None ⇒ False
      | Some (packed, attacker-state) ⇒
        hash-relation-hit ?R adversary-initial-state attacker-state
  have map-eq:
    wp-event (checked-staged-transcript-with-alpha-prefix-program A) ?H
      adversary-initial-state =
    wp-event
      (checked-staged-transcript-with-alpha-prefix-program A ≫=
        (λpacked. return (snd packed)))
      (hash-relation-hit-event ?R adversary-initial-state)
      adversary-initial-state
  by (subst wp-event-bind-return-map)
      (simp add: hash-relation-hit-event-def split: option.splits prod.splits)

```

also have ... =

$$\text{wp-event } (\text{checked-staged-transcript-program } A)$$

$$(\text{hash-relation-hit-event } ?R \text{ adversary-initial-state})$$

$$\text{adversary-initial-state}$$
unfolding *checked-staged-transcript-with-alpha-prefix-program-projection*
by *simp*
also have ... ≤

$$\text{staged-phase-relation-error size}$$

$$(\text{staged-attacker-query-budget budgets} + \text{staged-challenge-query-budget})$$
by (*rule checked-staged-transcript-initial-roots-relation-hit-bound*

$$[OF \text{ wf controlled}]$$
)
finally show *?thesis* .
qed

definition *staged-proof-transcript-length-bound* :: nat
where

$$\text{staged-proof-transcript-length-bound} =$$

$$4 + \text{ceil-log clength} + \text{length spec} + \text{ceil-log (maxDegree} + 1) +$$

$$\text{rounds} *$$

$$(\text{query-decommitment-transcript-length } 0 +$$

$$\text{fri-layers-transcript-length (ceil-log clength)}$$

$$(\text{clength} * \text{scale}) +$$

$$\text{fri-layers-transcript-length (ceil-log (maxDegree} + 1))$$

$$(\text{clength} * \text{scale}))$$

definition *staged-concrete-transcript-target-error-bound* :: prob
where

$$\text{staged-concrete-transcript-target-error-bound} =$$

$$\text{nnreal}$$

$$(\text{staged-proof-transcript-length-bound} * \text{verifier-hash-query-budget}) /$$

$$\text{nnreal size}$$

lemma *fri-layers-transcript-length-mono-layers:*

assumes $m \leq n$
shows $\text{fri-layers-transcript-length } m \text{ len} \leq$
 $\text{fri-layers-transcript-length } n \text{ len}$
using *assms*
proof (*induction m arbitrary: n len*)
case 0
then show *?case* **by** *simp*
next
case (*Suc m*)
then obtain n' **where** $n\text{-eq: } n = \text{Suc } n'$
by (*cases n*) *auto*
have $m\text{-le: } m \leq n'$
using *Suc.prem*s $n\text{-eq}$ **by** *simp*
have *rec:*

$$\text{fri-layers-transcript-length } m \text{ (len div 2)} \leq$$

$$\text{fri-layers-transcript-length } n' \text{ (len div 2)}$$

```

    by (rule Suc.IH[OF m-le])
  show ?case
    unfolding n-eq by simp (rule rec)
qed

lemma length-concat-eq-sum-list-map-length:
  length (List.concat xss) = sum-list (map length xss)
  by (induction xss) simp-all

lemma sum-list-map-const:
  ( $\sum x \leftarrow xs. c$ ) = length xs * c
  by (induction xs) simp-all

lemma staged-query-chunks-total-length-le:
  assumes match: staged-query-chunks-match-verifier-lengths data query-idxs
  and trace-len:
    length (staged-trace-fri-roots data) = ceil-log clength
  and composition-len:
    length (staged-composition-fri-roots data)  $\leq$ 
      ceil-log (maxDegree + 1)
  shows
    length (List.concat (staged-query-chunks data))  $\leq$ 
      rounds *
      (query-decommitment-transcript-length 0 +
       fri-layers-transcript-length (ceil-log clength)
       (clength * scale) +
       fri-layers-transcript-length (ceil-log (maxDegree + 1))
       (clength * scale))
proof -
  let ?Q =
    query-decommitment-transcript-length 0 +
    fri-layers-transcript-length (ceil-log clength) (clength * scale) +
    fri-layers-transcript-length (ceil-log (maxDegree + 1))
    (clength * scale)
  have chunks-len: length (staged-query-chunks data) = rounds
  using match unfolding staged-query-chunks-match-verifier-lengths-def
  by simp
  have chunk-bound:
     $\bigwedge chunk. chunk \in \text{set } (\text{staged-query-chunks data}) \implies$ 
      length chunk  $\leq$  ?Q
  proof -
    fix chunk
    assume chunk-in: chunk  $\in$  set (staged-query-chunks data)
    then obtain i where i-bound: i < rounds
    and chunk-eq: chunk = staged-query-chunks data ! i
    using chunks-len
    by (metis in-set-conv-nth)
  have chunk-len:
    length chunk =

```



```

    verifier-query-round-transcript-length (query-idxs ! i)
      (staged-trace-fri-roots data)
      (staged-composition-fri-roots data)
  using match i-bound chunk-eq
  unfolding staged-query-chunks-match-verifier-lengths-def by simp
have comp-mono:
  fri-layers-transcript-length
    (length (staged-composition-fri-roots data))
    (clength * scale)
  ≤
  fri-layers-transcript-length (ceil-log (maxDegree + 1))
    (clength * scale)
  by (rule fri-layers-transcript-length-mono-layers[OF composition-len])
show length chunk ≤ ?Q
  unfolding chunk-len verifier-query-round-transcript-length-def
    trace-len query-decommitment-transcript-length-def powers-scaled-def
  using comp-mono by simp
qed
have sum-bound:
  (∑ chunk←staged-query-chunks data. length chunk) ≤
  (∑ chunk←staged-query-chunks data. ?Q)
  by (rule sum-list-mono) (rule chunk-bound)
have concat-len:
  length (List.concat (staged-query-chunks data)) =
  (∑ chunk←staged-query-chunks data. length chunk)
  by (rule length-concat-eq-sum-list-map-length)
also have ... ≤ (∑ chunk←staged-query-chunks data. ?Q)
  by (rule sum-bound)
also have ... = length (staged-query-chunks data) * ?Q
  by (rule sum-list-map-const)
also have ... = rounds * ?Q
  using chunks-len by simp
finally show ?thesis .
qed

lemma checked-staged-transcript-program-staged-proof-transcript-length-bound:
  assumes wf: staged-budget-wellformed budgets
  and controlled: staged-adversary-controlled budgets A
  and outcome:
    Some (data, attacker-state) ∈
      set-dist
        (execute (checked-staged-transcript-program A)
          adversary-initial-state)
  shows length (staged-proof-transcript data) ≤
    staged-proof-transcript-length-bound
proof -
  have shape:
    length (staged-trace-fri-roots data) = ceil-log clength ∧
    length (staged-alphas data) = length spec ∧

```

```

length (staged-composition-fri-roots data) =
  ceil-log (to-nat (staged-degree data) + 1) ∧
  ceil-log (to-nat (staged-degree data) + 1) ≤
  ceil-log (maxDegree + 1)
using checked-staged-transcript-program-outcome-shape
  [OF wf controlled outcome]
by blast
from checked-staged-transcript-program-query-chunks-match-verifier-lengths
  [OF outcome]
obtain query-idxs where match:
  staged-query-chunks-match-verifier-lengths data query-idxs
by blast
have query-len:
  length (List.concat (staged-query-chunks data)) ≤
    rounds *
    (query-decommitment-transcript-length 0 +
     fri-layers-transcript-length (ceil-log clength)
     (clength * scale) +
     fri-layers-transcript-length (ceil-log (maxDegree + 1))
     (clength * scale))
by (rule staged-query-chunks-total-length-le[OF match])
  (use shape in simp-all)
have header-len:
  length
    (verifier-header-messages
     (staged-trace-root data)
     (staged-trace-fri-roots data)
     (staged-trace-final data)
     (staged-alphas data)
     (staged-degree data)
     (staged-composition-fri-roots data)
     (staged-composition-final data))
  ≤ 4 + ceil-log clength + length spec + ceil-log (maxDegree + 1)
using shape unfolding verifier-header-messages-def by simp
show ?thesis
unfolding staged-proof-transcript-def
  staged-proof-transcript-length-bound-def
using header-len query-len by simp
qed

```

```

lemma concrete-transcript-target-error-bound-from-length:
  assumes length tr ≤ n
  shows concrete-transcript-target-error tr ≤
    nnreal (n * verifier-hash-query-budget) / nnreal size
proof -
  have card (set tr) ≤ length tr
  by (rule card-length)
  then have card-le: card (set tr) ≤ n
  using assms by linarith

```

have *mult-le*:
 $\text{card } (\text{set } tr) * \text{verifier-hash-query-budget} \leq$
 $n * \text{verifier-hash-query-budget}$
by (rule *mult-right-mono*[*OF card-le*]) *simp*
show *?thesis*
unfolding *concrete-transcript-target-error-def*
by (rule *nnreal-nat-divide-right-mono*[*OF mult-le*])
qed

lemma *checked-staged-transcript-with-alpha-prefix-program-data-outcome*:
assumes *outcome*:
 $\text{Some } (\text{packed}, \text{attacker-state}) \in$
 set-dist
 $(\text{execute } (\text{checked-staged-transcript-with-alpha-prefix-program } A)$
 $\text{adversary-initial-state})$
shows
 $\text{Some } (\text{snd } \text{packed}, \text{attacker-state}) \in$
 set-dist
 $(\text{execute } (\text{checked-staged-transcript-program } A)$
 $\text{adversary-initial-state})$
using *outcome*
unfolding *checked-staged-transcript-with-alpha-prefix-program-projection*
 $[\text{symmetric}]$
by (rule-tac *x=packed and t=attacker-state in set-dist-bindI*) *simp-all*

lemma *checked-staged-transcript-with-alpha-prefix-program-concrete-transcript-target-error-bound*:
assumes *wf*: *staged-budget-wellformed budgets*
and *controlled*: *staged-adversary-controlled budgets A*
and *outcome*:
 $\text{Some } (\text{packed}, \text{attacker-state}) \in$
 set-dist
 $(\text{execute } (\text{checked-staged-transcript-with-alpha-prefix-program } A)$
 $\text{adversary-initial-state})$
shows
 $\text{concrete-transcript-target-error } (\text{staged-proof-transcript } (\text{snd } \text{packed}))$
 $\leq \text{staged-concrete-transcript-target-error-bound}$
proof –
have *data-out*:
 $\text{Some } (\text{snd } \text{packed}, \text{attacker-state}) \in$
 set-dist
 $(\text{execute } (\text{checked-staged-transcript-program } A)$
 $\text{adversary-initial-state})$
by (rule *checked-staged-transcript-with-alpha-prefix-program-data-outcome*
 $[\text{OF outcome}]$)
have *len*:
 $\text{length } (\text{staged-proof-transcript } (\text{snd } \text{packed})) \leq$
 $\text{staged-proof-transcript-length-bound}$
by (rule *checked-staged-transcript-program-staged-proof-transcript-length-bound*
 $[\text{OF wf controlled data-out}]$)

show *?thesis*
unfolding *staged-concrete-transcript-target-error-bound-def*
by (rule *concrete-transcript-target-error-bound-from-length*[*OF len*])
qed

definition *checked-staged-security-with-actual-alpha-prefix-verifier-event*

$:: ('f \text{ protocol-channel} \Rightarrow$
 $(\text{unit list} \times 'f \text{ protocol-channel}) \text{ option} \Rightarrow \text{bool})$
 $\Rightarrow$
 $(((((('f \times 'f \text{ list} \times 'f \text{ list} \times 'f) \times$
 $'f \text{ protocol-channel}) \times 'f \text{ staged-proof-data}) \times$
 $'f \text{ protocol-channel}) \times \text{unit list}) \times$
 $'f \text{ protocol-channel}) \text{ option} \Rightarrow \text{bool}$

where

checked-staged-security-with-actual-alpha-prefix-verifier-event *P out*
 $\longleftrightarrow$
 (case out of
 None $\Rightarrow$ False
 | Some (*x*, *final-state*) $\Rightarrow$
 (let packed = fst (fst *x*);
 data = snd packed;
 attacker-state = snd (fst *x*);
 result = snd *x*;
 s =
 verifier-state-from-adversary attacker-state
 (staged-proof-transcript data)
 in *P s* (Some (result, final-state))))

definition *checked-staged-security-with-actual-alpha-prefix-branch-tree-output-hit*

$:: ((((((('f \times 'f \text{ list} \times 'f \text{ list} \times 'f) \times$
 $'f \text{ protocol-channel}) \times 'f \text{ staged-proof-data}) \times$
 $'f \text{ protocol-channel}) \times \text{unit list}) \times$
 $'f \text{ protocol-channel}) \text{ option} \Rightarrow \text{bool}$

where

checked-staged-security-with-actual-alpha-prefix-branch-tree-output-hit
out $\longleftrightarrow$
 (case out of
 None $\Rightarrow$ False
 | Some (*x*, *final-state*) $\Rightarrow$
 (let packed = fst (fst *x*);
 prefix-state = snd (fst packed);
 data = snd packed;
 attacker-state = snd (fst *x*);
 result = snd *x*;
 s =
 verifier-state-from-adversary attacker-state
 (staged-proof-transcript data)
 in $\exists$ trace-table composition-table as query-idxs dg
 composition-fri-roots final rest trace-tree composition-tree.

Some (result, final-state) ∈
set-dist (execute verify-monad s) ∧
accepted-with-bound-tables s (Some (result, final-state))
trace-table composition-table as query-idxs ∧
verifier-header-transcript s
(staged-trace-root data)
(staged-trace-fri-roots data)
(staged-trace-final data)
as dg composition-fri-roots final rest ∧
created-tree trace-table trace-tree final-state ∧
created-tree composition-table composition-tree final-state ∧
hash-map-new-output-hit
(set-tree trace-tree ∪ set-tree composition-tree)
prefix-state final-state))

definition *checked-staged-security-with-actual-alpha-prefix-branch-tree-output-case*
where

checked-staged-security-with-actual-alpha-prefix-branch-tree-output-case
P out ⟷
(case out of
None ⇒ False
| Some (x, final-state) ⇒
(let packed = fst (fst x);
prefix-state = snd (fst packed);
data = snd packed;
attacker-state = snd (fst x);
result = snd x;
s =
verifier-state-from-adversary attacker-state
(staged-proof-transcript data)
in ∃ trace-table composition-table as query-idxs dg
composition-fri-roots final rest trace-tree composition-tree.
Some (result, final-state) ∈
set-dist (execute verify-monad s) ∧
accepted-with-bound-tables s (Some (result, final-state))
trace-table composition-table as query-idxs ∧
verifier-header-transcript s
(staged-trace-root data)
(staged-trace-fri-roots data)
(staged-trace-final data)
as dg composition-fri-roots final rest ∧
created-tree trace-table trace-tree final-state ∧
created-tree composition-table composition-tree final-state ∧
P trace-tree composition-tree prefix-state final-state))

definition *checked-staged-security-with-actual-alpha-prefix-branch-tree-root-output-hit*
where

checked-staged-security-with-actual-alpha-prefix-branch-tree-root-output-hit =
checked-staged-security-with-actual-alpha-prefix-branch-tree-output-case

$(\lambda \text{trace-tree composition-tree prefix-state final-state.}$
 $\text{hash-map-new-output-hit}$
 $\{\text{value trace-tree, value composition-tree}\}$
 $\text{prefix-state final-state})$

definition

checked-staged-security-with-actual-alpha-prefix-branch-tree-subtree-output-hit

where

$\text{checked-staged-security-with-actual-alpha-prefix-branch-tree-subtree-output-hit} =$
 $\text{checked-staged-security-with-actual-alpha-prefix-branch-tree-output-case}$
 $(\lambda \text{trace-tree composition-tree prefix-state final-state.}$
 $(\exists l \ v \ r. \text{trace-tree} = \langle l, v, r \rangle \wedge$
 $\text{hash-map-new-output-hit (set-tree } l \cup \text{set-tree } r)$
 $\text{prefix-state final-state}) \vee$
 $(\exists l \ v \ r. \text{composition-tree} = \langle l, v, r \rangle \wedge$
 $\text{hash-map-new-output-hit (set-tree } l \cup \text{set-tree } r)$
 $\text{prefix-state final-state}))$

definition

checked-staged-security-with-actual-alpha-prefix-branch-tree-subtree-preverifier-output-hit

where

$\text{checked-staged-security-with-actual-alpha-prefix-branch-tree-subtree-preverifier-output-hit}$
 $\text{out} \longleftrightarrow$
 $(\text{case out of}$
 $\text{None} \Rightarrow \text{False}$
 $| \text{Some } (x, \text{final-state}) \Rightarrow$
 $(\text{let packed} = \text{fst (fst } x);$
 $\text{prefix-state} = \text{snd (fst packed);}$
 $\text{data} = \text{snd packed};$
 $\text{attacker-state} = \text{snd (fst } x);$
 $\text{result} = \text{snd } x;$
 $s =$
 $\text{verifier-state-from-adversary attacker-state}$
 $(\text{staged-proof-transcript data})$
 $\text{in } \exists \text{trace-table composition-table as query-idxs dg}$
 $\text{composition-fri-roots final rest trace-tree composition-tree.}$
 $\text{Some (result, final-state)} \in$
 $\text{set-dist (execute verify-monad } s) \wedge$
 $\text{accepted-with-bound-tables } s (\text{Some (result, final-state)})$
 $\text{trace-table composition-table as query-idxs} \wedge$
 $\text{verifier-header-transcript } s$
 $(\text{staged-trace-root data})$
 $(\text{staged-trace-fri-roots data})$
 $(\text{staged-trace-final data})$
 $\text{as dg composition-fri-roots final rest} \wedge$
 $\text{created-tree trace-table trace-tree final-state} \wedge$
 $\text{created-tree composition-table composition-tree final-state} \wedge$
 $((\exists l \ v \ r. \text{trace-tree} = \langle l, v, r \rangle \wedge$
 $\text{hash-map-new-output-hit (set-tree } l \cup \text{set-tree } r)$

$\text{prefix-state } s) \vee$
 $(\exists l \ v \ r. \text{composition-tree} = \langle l, v, r \rangle \wedge$
 $\text{hash-map-new-output-hit } (\text{set-tree } l \cup \text{set-tree } r)$
 $\text{prefix-state } s)))))$

definition

checked-staged-security-with-actual-alpha-prefix-branch-tree-subtree-verifier-output-hit

where

checked-staged-security-with-actual-alpha-prefix-branch-tree-subtree-verifier-output-hit

$\text{out} \longleftrightarrow$

(case out of

None $\Rightarrow$ False

| Some (x, final-state) $\Rightarrow$

(let packed = fst (fst x);

data = snd packed;

attacker-state = snd (fst x);

result = snd x;

s =

verifier-state-from-adversary attacker-state

(staged-proof-transcript data)

in $\exists$ trace-table composition-table as query-idxs dg

composition-fri-roots final rest trace-tree composition-tree.

Some (result, final-state) $\in$

set-dist (execute verify-monad s) $\wedge$

accepted-with-bound-tables s (Some (result, final-state))

trace-table composition-table as query-idxs $\wedge$

verifier-header-transcript s

(staged-trace-root data)

(staged-trace-fri-roots data)

(staged-trace-final data)

as dg composition-fri-roots final rest $\wedge$

created-tree trace-table trace-tree final-state $\wedge$

created-tree composition-table composition-tree final-state $\wedge$

(($\exists l \ v \ r. \text{trace-tree} = \langle l, v, r \rangle \wedge$

hash-map-new-output-hit (set-tree l $\cup$ set-tree r)

s final-state) $\vee$

($\exists l \ v \ r. \text{composition-tree} = \langle l, v, r \rangle \wedge$

hash-map-new-output-hit (set-tree l $\cup$ set-tree r)

s final-state))))

definition

checked-staged-security-with-actual-alpha-prefix-branch-transcript-root-output-hit

where

checked-staged-security-with-actual-alpha-prefix-branch-transcript-root-output-hit

$\text{out} \longleftrightarrow$

(case out of

None $\Rightarrow$ False

| Some (x, final-state) $\Rightarrow$

(let packed = fst (fst x);

```

    prefix-state = snd (fst packed);
    data = snd packed;
    attacker-state = snd (fst x);
    result = snd x;
    s =
    verifier-state-from-adversary attacker-state
      (staged-proof-transcript data)
  in  $\exists$  trace-table composition-table as query-idxs dg
    composition-fri-roots final rest.
  Some (result, final-state)  $\in$ 
    set-dist (execute verify-monad s)  $\wedge$ 
  accepted-with-bound-tables s (Some (result, final-state))
    trace-table composition-table as query-idxs  $\wedge$ 
  verifier-header-transcript s
    (staged-trace-root data)
    (staged-trace-fri-roots data)
    (staged-trace-final data)
    as dg composition-fri-roots final rest  $\wedge$ 
  hash-map-new-output-hit
    {staged-trace-root data, hd composition-fri-roots}
    prefix-state final-state))

```

definition

checked-staged-security-with-actual-alpha-prefix-branch-transcript-root-preverifier-output-hit

where

checked-staged-security-with-actual-alpha-prefix-branch-transcript-root-preverifier-output-hit

out $\longleftrightarrow$

(case out of

None $\Rightarrow$ False

| Some (x, final-state) $\Rightarrow$

(let packed = fst (fst x);

prefix-state = snd (fst packed);

data = snd packed;

attacker-state = snd (fst x);

result = snd x;

s =

verifier-state-from-adversary attacker-state

(staged-proof-transcript data)

in $\exists$ trace-table composition-table as query-idxs dg

composition-fri-roots final rest.

Some (result, final-state) $\in$

set-dist (execute verify-monad s) $\wedge$

accepted-with-bound-tables s (Some (result, final-state))

trace-table composition-table as query-idxs $\wedge$

verifier-header-transcript s

(staged-trace-root data)

(staged-trace-fri-roots data)

(staged-trace-final data)

as dg composition-fri-roots final rest $\wedge$

hash-map-new-output-hit
 {*staged-trace-root data*, *hd composition-fri-roots*}
prefix-state s))

lemma *checked-staged-security-with-actual-alpha-prefix-branch-transcript-root-preverifier-output-hit-imp-initial-roots*

assumes *wf: staged-budget-wellformed budgets*

and *controlled: staged-adversary-controlled budgets A*

and *support:*

out ∈

set-dist

(*execute*

(*checked-staged-security-experiment-with-actual-alpha-prefix-data-state*

A)

adversary-initial-state)

and *hit:*

checked-staged-security-with-actual-alpha-prefix-branch-transcript-root-preverifier-output-hit

out

shows

case out of

None ⇒ *False*

| *Some* ((((*prefix*, *prefix-state*), *data*), *attacker-state*),
result), *final-state*) ⇒

hash-relation-hit

(*checked-staged-transcript-initial-roots-relation A*)

adversary-initial-state attacker-state

proof (*cases out*)

case *None*

then show *?thesis*

using *hit*

unfolding

checked-staged-security-with-actual-alpha-prefix-branch-transcript-root-preverifier-output-hit-def

by *simp*

next

case (*Some result-pack*)

then obtain *prefix prefix-state data attacker-state result final-state*

where *out-eq:*

out =

Some ((((*prefix*, *prefix-state*), *data*), *attacker-state*),

result), *final-state*)

by (*cases result-pack*, *auto split: prod.splits*)

let *?s* =

verifier-state-from-adversary attacker-state

(*staged-proof-transcript data*)

have *support-some:*

Some ((((*prefix*, *prefix-state*), *data*), *attacker-state*),

result), *final-state*) ∈

set-dist

(*execute*

(*checked-staged-security-experiment-with-actual-alpha-prefix-data-state*

```

    A)
    adversary-initial-state)
  using support unfolding out-eq .
from support-some obtain trans-out where trans-out:
  Some (((prefix, prefix-state), data), attacker-state) ∈
  set-dist
  (execute (checked-staged-transcript-with-alpha-prefix-program A)
    adversary-initial-state)
  unfolding
    checked-staged-security-experiment-with-actual-alpha-prefix-data-state-def
  by (auto elim!: set-dist-bindE split: prod.splits)
have data-out:
  Some (data, attacker-state) ∈
  set-dist
  (execute (checked-staged-transcript-program A)
    adversary-initial-state)
  using checked-staged-transcript-with-alpha-prefix-program-data-outcome
  [OF trans-out]
  by simp
from checked-staged-transcript-with-alpha-prefix-program-support
  [OF wf controlled trans-out]
have prefix-out:
  Some (prefix, prefix-state) ∈
  set-dist
  (execute (staged-alpha-prefix-program A)
    adversary-initial-state)
  by blast
have initial-prefix: adversary-initial-state ≤ prefix-state
proof -
  have program:
    hash-target-program {} (staged-alpha-search-queries budgets 0)
    (staged-alpha-prefix-program A)
  by (rule hash-target-program-staged-alpha-prefix-program
    [OF wf controlled])
  then show ?thesis
  using prefix-out
  unfolding hash-target-program-def hash-extension-preserving-def
  by blast
qed
from hit obtain trace-table composition-table as query-idxs dg
  composition-fri-roots final rest where
  verify-out:
    Some (result, final-state) ∈ set-dist (execute verify-monad ?s)
  and bound:
    accepted-with-bound-tables ?s (Some (result, final-state))
    trace-table composition-table as query-idxs
  and header:
    verifier-header-transcript ?s
    (staged-trace-root data)

```

```

    (staged-trace-fri-roots data)
    (staged-trace-final data)
    as dg composition-fri-roots final rest
and root-hit:
    hash-map-new-output-hit
    {staged-trace-root data, hd composition-fri-roots}
    prefix-state ?s
unfolding out-eq
    checked-staged-security-with-actual-alpha-prefix-branch-transcript-root-preverifier-output-hit-def
    Let-def
by simp blast
have staged-header:
    verifier-header-transcript ?s
    (staged-trace-root data)
    (staged-trace-fri-roots data)
    (staged-trace-final data)
    (staged-alphas data)
    (staged-degree data)
    (staged-composition-fri-roots data)
    (staged-composition-final data)
    (List.concat (staged-query-chunks data))
by (rule checked-staged-transcript-program-outcome-header-transcript
    [OF wf controlled data-out])
have header-eq:
    as = staged-alphas data ∧
    dg = staged-degree data ∧
    composition-fri-roots = staged-composition-fri-roots data ∧
    final = staged-composition-final data ∧
    rest = List.concat (staged-query-chunks data)
using verifier-header-transcript-unique[OF header staged-header]
by simp
from root-hit obtain x y where
    y-in: y ∈ {staged-trace-root data, hd composition-fri-roots}
    and lookup-prefix-none: fmlookup (HashMap prefix-state) x = None
    and lookup-s: fmlookup (HashMap ?s) x = Some y
    unfolding hash-map-new-output-hit-def by blast
have lookup-initial-none: fmlookup (HashMap adversary-initial-state) x = None
proof (cases fmlookup (HashMap adversary-initial-state) x)
    case None
    then show ?thesis .
next
    case (Some old)
    have fmlookup (HashMap prefix-state) x = Some old
    by (rule hash-extension-lookup[OF Some initial-prefix])
    then show ?thesis
    using lookup-prefix-none by simp
qed
have lookup-attacker: fmlookup (HashMap attacker-state) x = Some y
    using lookup-s by simp

```

```

have relation:
  checked-staged-transcript-initial-roots-relation A x y
proof —
  have  $y \in$ 
    {staged-trace-root data, hd (staged-composition-fri-roots data)}
    using y-in header-eq by auto
  then show ?thesis
    unfolding checked-staged-transcript-initial-roots-relation-def
    using data-out by blast
qed
have
  hash-relation-hit
    (checked-staged-transcript-initial-roots-relation A)
    adversary-initial-state attacker-state
  unfolding hash-relation-hit-def
  by (intro exI[of - x] exI[of - y] conjI)
    (use lookup-initial-none lookup-attacker relation in simp-all)
  then show ?thesis
    unfolding out-eq by simp
qed

lemma checked-staged-security-with-actual-alpha-prefix-branch-transcript-root-preverifier-output-hit-bound:
  assumes wf: staged-budget-wellformed budgets
  and controlled: staged-adversary-controlled budgets A
  shows
    wp-event
      (checked-staged-security-experiment-with-actual-alpha-prefix-data-state A)
      checked-staged-security-with-actual-alpha-prefix-branch-transcript-root-preverifier-output-hit
      adversary-initial-state  $\leq$ 
      staged-phase-relation-error size
      (staged-attacker-query-budget budgets + staged-challenge-query-budget)
  unfolding checked-staged-security-experiment-with-actual-alpha-prefix-data-state-def
proof (rule wp-event-bind-bound-by-head-event)
show
  wp-event (checked-staged-transcript-with-alpha-prefix-program A)
    ( $\lambda out. case out of$ 
      None  $\Rightarrow$  False
      | Some (packed, attacker-state)  $\Rightarrow$ 
        hash-relation-hit
          (checked-staged-transcript-initial-roots-relation A)
          adversary-initial-state attacker-state)
        adversary-initial-state
         $\leq$  staged-phase-relation-error size
        (staged-attacker-query-budget budgets +
          staged-challenge-query-budget)
    by (rule
      checked-staged-transcript-with-alpha-prefix-initial-roots-relation-hit-bound
      [OF wf controlled])
next

```

```

show
  checked-staged-security-with-actual-alpha-prefix-branch-transcript-root-preverifier-output-hit
    None  $\Rightarrow$ 
    (case None of None  $\Rightarrow$  False
    | Some (packed, attacker-state)  $\Rightarrow$ 
      hash-relation-hit
        (checked-staged-transcript-initial-roots-relation A)
        adversary-initial-state attacker-state)
unfolding
  checked-staged-security-with-actual-alpha-prefix-branch-transcript-root-preverifier-output-hit-def
by simp
next
fix packed attacker-state out
assume trans-out:
  Some (packed, attacker-state)  $\in$ 
    set-dist
      (execute (checked-staged-transcript-with-alpha-prefix-program A)
        adversary-initial-state)
and cont:
  out  $\in$ 
    set-dist
      (execute
        (get  $\gg$ 
          ( $\lambda$ s. put
            (verifier-state-from-adversary s
              (staged-proof-transcript (snd packed))))  $\gg$ 
            ( $\lambda$ -. verify-monad  $\gg$ 
              ( $\lambda$ result. return ((packed, s), result))))))
        attacker-state)
and hit:
  checked-staged-security-with-actual-alpha-prefix-branch-transcript-root-preverifier-output-hit
    out
have support:
  out  $\in$ 
    set-dist
      (execute
        (checked-staged-security-experiment-with-actual-alpha-prefix-data-state
          A)
        adversary-initial-state)
unfolding
  checked-staged-security-experiment-with-actual-alpha-prefix-data-state-def
by (rule-tac x=packed and t=attacker-state in set-dist-bindI)
  (use trans-out cont in simp-all)
have rel:
  case out of
    None  $\Rightarrow$  False
  | Some (((((prefix, prefix-state), data), attacker-state),
    result), final-state)  $\Rightarrow$ 
    hash-relation-hit

```

```

      (checked-staged-transcript-initial-roots-relation A)
      adversary-initial-state attacker-state
  by (rule
    checked-staged-security-with-actual-alpha-prefix-branch-transcript-root-preverifier-output-hit-imp-initial-roots-rel
    [OF wf controlled support hit])
  show
    (case Some (packed, attacker-state) of
      None  $\Rightarrow$  False
    | Some (packed, attacker-state)  $\Rightarrow$ 
      hash-relation-hit
      (checked-staged-transcript-initial-roots-relation A)
      adversary-initial-state attacker-state)
  using cont rel
  by (cases out)
    (auto elim!: set-dist-bindE split: option.splits prod.splits)
qed

lemma checked-staged-security-with-actual-alpha-prefix-branch-tree-output-hit-imp-tree-output-hit:
  assumes hit:
    checked-staged-security-with-actual-alpha-prefix-branch-tree-output-hit
    out
  shows
    checked-staged-security-with-actual-alpha-prefix-tree-output-hit out
  using hit
  unfolding
    checked-staged-security-with-actual-alpha-prefix-branch-tree-output-hit-def
    checked-staged-security-with-actual-alpha-prefix-tree-output-hit-def
    Let-def
  by (cases out) (fastforce split: prod.splits)+

lemma checked-staged-security-with-actual-alpha-prefix-branch-tree-output-hit-imp-root-or-subtree:
  assumes hit:
    checked-staged-security-with-actual-alpha-prefix-branch-tree-output-hit
    out
  shows
    checked-staged-security-with-actual-alpha-prefix-branch-tree-root-output-hit
    out  $\vee$ 
    checked-staged-security-with-actual-alpha-prefix-branch-tree-subtree-output-hit
    out
  proof (cases out)
  case None
  then show ?thesis
    using hit
    unfolding
      checked-staged-security-with-actual-alpha-prefix-branch-tree-output-hit-def
    by simp
  next
  case (Some result-pack)
  then obtain x final-state where out-eq:

```

```

    out = Some (x, final-state)
  by (cases result-pack) simp
from hit obtain trace-table composition-table as query-idxs dg
  composition-fri-roots final rest trace-tree composition-tree where
body:
  Some (snd x, final-state) ∈
    set-dist
      (execute verify-monad
        (verifier-state-from-adversary (snd (fst x))
          (staged-proof-transcript (snd (fst (fst x))))))
    accepted-with-bound-tables
      (verifier-state-from-adversary (snd (fst x))
        (staged-proof-transcript (snd (fst (fst x))))))
      (Some (snd x, final-state)) trace-table composition-table as
        query-idxs
    verifier-header-transcript
      (verifier-state-from-adversary (snd (fst x))
        (staged-proof-transcript (snd (fst (fst x))))))
      (staged-trace-root (snd (fst (fst x))))
      (staged-trace-fri-roots (snd (fst (fst x))))
      (staged-trace-final (snd (fst (fst x))))
      as dg composition-fri-roots final rest
    created-tree trace-table trace-tree final-state
    created-tree composition-table composition-tree final-state
    hash-map-new-output-hit
      (set-tree trace-tree ∪ set-tree composition-tree)
      (snd (fst (fst (fst x)))) final-state
  unfolding out-eq
    checked-staged-security-with-actual-alpha-prefix-branch-tree-output-hit-def
    Let-def
  by (auto split: prod.splits)
from hash-map-new-output-hit-two-trees-root-or-subtree[OF body(6)]
show ?thesis
proof
  assume root-hit:
    hash-map-new-output-hit {value trace-tree, value composition-tree}
      (snd (fst (fst (fst x)))) final-state
  have
    checked-staged-security-with-actual-alpha-prefix-branch-tree-root-output-hit
      out
  unfolding out-eq
    checked-staged-security-with-actual-alpha-prefix-branch-tree-root-output-hit-def
    checked-staged-security-with-actual-alpha-prefix-branch-tree-output-case-def
    Let-def
  using body(1-5) root-hit
  by (cases x) (fastforce split: prod.splits)+
then show ?thesis by simp
next
  assume subtree:

```

```

(∃ l v r. trace-tree = ⟨l, v, r⟩ ∧
  hash-map-new-output-hit (set-tree l ∪ set-tree r)
    (snd (fst (fst (fst x)))) final-state) ∨
(∃ l v r. composition-tree = ⟨l, v, r⟩ ∧
  hash-map-new-output-hit (set-tree l ∪ set-tree r)
    (snd (fst (fst (fst x)))) final-state)
have
  checked-staged-security-with-actual-alpha-prefix-branch-tree-subtree-output-hit
    out
unfolding out-eq
  checked-staged-security-with-actual-alpha-prefix-branch-tree-subtree-output-hit-def
  checked-staged-security-with-actual-alpha-prefix-branch-tree-output-case-def
  Let-def
using body(1-5) subtree
by (cases x) (fastforce split: prod.splits)+
then show ?thesis by simp
qed
qed

lemma checked-staged-security-with-actual-alpha-prefix-branch-tree-output-hit-bound-from-root-and-subtree:
assumes root-bound:
  wp-event m
  checked-staged-security-with-actual-alpha-prefix-branch-tree-root-output-hit
  s ≤ R
and subtree-bound:
  wp-event m
  checked-staged-security-with-actual-alpha-prefix-branch-tree-subtree-output-hit
  s ≤ S
shows
  wp-event m
  checked-staged-security-with-actual-alpha-prefix-branch-tree-output-hit
  s ≤ R + S
proof -
have wp-event m
  checked-staged-security-with-actual-alpha-prefix-branch-tree-output-hit
  s ≤
  wp-event m
  (λout.
    checked-staged-security-with-actual-alpha-prefix-branch-tree-root-output-hit
    out ∨
    checked-staged-security-with-actual-alpha-prefix-branch-tree-subtree-output-hit
    out) s
by (rule wp-event-mono)
  (rule
    checked-staged-security-with-actual-alpha-prefix-branch-tree-output-hit-imp-root-or-subtree)
also have ... ≤
  wp-event m
  checked-staged-security-with-actual-alpha-prefix-branch-tree-root-output-hit
  s +

```



```

    wp-event m
    checked-staged-security-with-actual-alpha-prefix-branch-tree-subtree-output-hit
    s
    by (rule wp-event-union-bound)
    also have ...  $\leq R + S$ 
    by (intro add-mono root-bound subtree-bound)
    finally show ?thesis .
qed

lemma checked-staged-security-with-actual-alpha-prefix-branch-tree-subtree-output-hit-imp-preverifier-or-verifier-on-s
  assumes wf: staged-budget-wellformed budgets
  and controlled: staged-adversary-controlled budgets A
  and support:
    out  $\in$ 
    set-dist
    (execute
      (checked-staged-security-experiment-with-actual-alpha-prefix-data-state
        A)
      adversary-initial-state)
  and hit:
    checked-staged-security-with-actual-alpha-prefix-branch-tree-subtree-output-hit
    out
  shows
    checked-staged-security-with-actual-alpha-prefix-branch-tree-subtree-preverifier-output-hit
    out  $\vee$ 
    checked-staged-security-with-actual-alpha-prefix-branch-tree-subtree-verifier-output-hit
    out
  proof (cases out)
  case None
  then show ?thesis
  using hit
  unfolding
    checked-staged-security-with-actual-alpha-prefix-branch-tree-subtree-output-hit-def
    checked-staged-security-with-actual-alpha-prefix-branch-tree-output-case-def
  by simp
next
case (Some result-pack)
then obtain prefix prefix-state data attacker-state result final-state where
  out-eq:
    out =
      Some (((((prefix, prefix-state), data), attacker-state),
        result), final-state)
  by (cases result-pack, auto split: prod.splits)
let ?s =
  verifier-state-from-adversary attacker-state
  (staged-proof-transcript data)
have support-some:
  Some (((((prefix, prefix-state), data), attacker-state),
    result), final-state)  $\in$ 

```

```

    set-dist
    (execute
      (checked-staged-security-experiment-with-actual-alpha-prefix-data-state
        A)
      adversary-initial-state)
  using support unfolding out-eq .
from support-some obtain trans-out where trans-out:
  Some (((prefix, prefix-state), data), attacker-state) ∈
    set-dist
    (execute (checked-staged-transcript-with-alpha-prefix-program A)
      adversary-initial-state)
  unfolding
    checked-staged-security-experiment-with-actual-alpha-prefix-data-state-def
  by (auto elim!: set-dist-bindE split: prod.splits)
from checked-staged-transcript-with-alpha-prefix-program-support
  [OF wf controlled trans-out]
have prefix-ext: prefix-state ≤ attacker-state
  by blast
have prefix-s: prefix-state ≤ ?s
  by (rule
    alpha-prefix-hash-extends-verifier-state-from-adversary-right
    [OF prefix-ext])
from hit obtain trace-table composition-table as query-idxs dg
  composition-fri-roots final rest trace-tree composition-tree where
  verify-out:
    Some (result, final-state) ∈ set-dist (execute verify-monad ?s)
  and bound:
    accepted-with-bound-tables ?s (Some (result, final-state))
    trace-table composition-table as query-idxs
  and header:
    verifier-header-transcript ?s
    (staged-trace-root data)
    (staged-trace-fri-roots data)
    (staged-trace-final data)
    as dg composition-fri-roots final rest
  and trace-created:
    created-tree trace-table trace-tree final-state
  and composition-created:
    created-tree composition-table composition-tree final-state
  and subtree:
    (∃ l v r. trace-tree = ⟨l, v, r⟩ ∧
      hash-map-new-output-hit (set-tree l ∪ set-tree r)
      prefix-state final-state) ∨
    (∃ l v r. composition-tree = ⟨l, v, r⟩ ∧
      hash-map-new-output-hit (set-tree l ∪ set-tree r)
      prefix-state final-state)
  unfolding out-eq
    checked-staged-security-with-actual-alpha-prefix-branch-tree-subtree-output-hit-def
    checked-staged-security-with-actual-alpha-prefix-branch-tree-output-case-def

```

```

    Let-def
  by simp blast
have s-final:  $?s \leq \text{final-state}$ 
  by (rule verify-monad-hash-extends [OF verify-out])
from subtree show ?thesis
proof
  assume trace-subtree:
     $\exists l\ v\ r. \text{trace-tree} = \langle l, v, r \rangle \wedge$ 
    hash-map-new-output-hit (set-tree l  $\cup$  set-tree r)
    prefix-state final-state
  then obtain l v r where trace-tree-eq:  $\text{trace-tree} = \langle l, v, r \rangle$ 
  and tree-hit:
    hash-map-new-output-hit (set-tree l  $\cup$  set-tree r)
    prefix-state final-state
  by blast
from hash-map-new-output-hit-trans-decomp
  [OF prefix-s s-final tree-hit]
show ?thesis
proof
  assume pre:
    hash-map-new-output-hit (set-tree l  $\cup$  set-tree r)
    prefix-state ?s
  have
    checked-staged-security-with-actual-alpha-prefix-branch-tree-subtree-preverifier-output-hit
    out
  unfolding out-eq
checked-staged-security-with-actual-alpha-prefix-branch-tree-subtree-preverifier-output-hit-def
    Let-def
  using verify-out bound header trace-created composition-created
    trace-tree-eq pre
  by simp blast
  then show ?thesis by simp
next
  assume verifier:
    hash-map-new-output-hit (set-tree l  $\cup$  set-tree r)
    ?s final-state
  have
    checked-staged-security-with-actual-alpha-prefix-branch-tree-subtree-verifier-output-hit
    out
  unfolding out-eq
checked-staged-security-with-actual-alpha-prefix-branch-tree-subtree-verifier-output-hit-def
    Let-def
  using verify-out bound header trace-created composition-created
    trace-tree-eq verifier
  by simp blast
  then show ?thesis by simp
qed
next
  assume composition-subtree:

```

```

     $\exists l \ v \ r. \text{composition-tree} = \langle l, v, r \rangle \wedge$ 
     $\text{hash-map-new-output-hit} \ (\text{set-tree } l \cup \text{set-tree } r)$ 
     $\text{prefix-state } \text{final-state}$ 
then obtain  $l \ v \ r$  where
     $\text{composition-tree-eq}: \text{composition-tree} = \langle l, v, r \rangle$ 
and  $\text{tree-hit}:$ 
     $\text{hash-map-new-output-hit} \ (\text{set-tree } l \cup \text{set-tree } r)$ 
     $\text{prefix-state } \text{final-state}$ 
by  $\text{blast}$ 
from  $\text{hash-map-new-output-hit-trans-decomp}$ 
     $[\text{OF } \text{prefix-s } s\text{-final } \text{tree-hit}]$ 
show  $?thesis$ 
proof
  assume  $\text{pre}:$ 
     $\text{hash-map-new-output-hit} \ (\text{set-tree } l \cup \text{set-tree } r)$ 
     $\text{prefix-state } ?s$ 
  have
     $\text{checked-staged-security-with-actual-alpha-prefix-branch-tree-subtree-preverifier-output-hit}$ 
     $\text{out}$ 
    unfolding  $\text{out-eq}$ 
     $\text{checked-staged-security-with-actual-alpha-prefix-branch-tree-subtree-preverifier-output-hit-def}$ 
     $\text{Let-def}$ 
    using  $\text{verify-out bound header trace-created composition-created}$ 
     $\text{composition-tree-eq } \text{pre}$ 
    by  $\text{simp blast}$ 
  then show  $?thesis$  by  $\text{simp}$ 
next
  assume  $\text{verifier}:$ 
     $\text{hash-map-new-output-hit} \ (\text{set-tree } l \cup \text{set-tree } r)$ 
     $?s \ \text{final-state}$ 
  have
     $\text{checked-staged-security-with-actual-alpha-prefix-branch-tree-subtree-verifier-output-hit}$ 
     $\text{out}$ 
    unfolding  $\text{out-eq}$ 
     $\text{checked-staged-security-with-actual-alpha-prefix-branch-tree-subtree-verifier-output-hit-def}$ 
     $\text{Let-def}$ 
    using  $\text{verify-out bound header trace-created composition-created}$ 
     $\text{composition-tree-eq } \text{verifier}$ 
    by  $\text{simp blast}$ 
  then show  $?thesis$  by  $\text{simp}$ 
qed
qed
qed

lemma  $\text{checked-staged-security-with-actual-alpha-prefix-branch-tree-subtree-output-hit-bound-from-preverifier-and-ver}$ 
assumes  $\text{wf}: \text{staged-budget-wellformed budgets}$ 
and  $\text{controlled}: \text{staged-adversary-controlled budgets } A$ 
and  $\text{pre-bound}:$ 
   $\text{wp-event}$ 

```

```

    (checked-staged-security-experiment-with-actual-alpha-prefix-data-state
      A)
    checked-staged-security-with-actual-alpha-prefix-branch-tree-subtree-preverifier-output-hit
    adversary-initial-state  $\leq P$ 
  and verifier-bound:
    wp-event
    (checked-staged-security-experiment-with-actual-alpha-prefix-data-state
      A)
    checked-staged-security-with-actual-alpha-prefix-branch-tree-subtree-verifier-output-hit
    adversary-initial-state  $\leq V$ 
  shows
    wp-event
    (checked-staged-security-experiment-with-actual-alpha-prefix-data-state A)
    checked-staged-security-with-actual-alpha-prefix-branch-tree-subtree-output-hit
    adversary-initial-state  $\leq P + V$ 
proof -
  let ?M =
    checked-staged-security-experiment-with-actual-alpha-prefix-data-state A
  let ?Sub =
    checked-staged-security-with-actual-alpha-prefix-branch-tree-subtree-output-hit
  let ?Pre =
    checked-staged-security-with-actual-alpha-prefix-branch-tree-subtree-preverifier-output-hit
  let ?Verifier =
    checked-staged-security-with-actual-alpha-prefix-branch-tree-subtree-verifier-output-hit
  have wp-event ?M ?Sub adversary-initial-state  $\leq$ 
    wp-event ?M ( $\lambda out. ?Pre out \vee ?Verifier out$ )
    adversary-initial-state
  by (rule wp-event-mono-on-support)
  (rule
    checked-staged-security-with-actual-alpha-prefix-branch-tree-subtree-output-hit-imp-preverifier-or-verifier-on-supp
    [OF wf controlled])
  also have ...  $\leq$ 
    wp-event ?M ?Pre adversary-initial-state +
    wp-event ?M ?Verifier adversary-initial-state
  by (rule wp-event-union-bound)
  also have ...  $\leq P + V$ 
  by (intro add-mono pre-bound verifier-bound)
  finally show ?thesis .
qed

lemma checked-staged-security-with-actual-alpha-prefix-branch-tree-root-output-hit-imp-transcript-root-output-hit:
  assumes root-hit:
    checked-staged-security-with-actual-alpha-prefix-branch-tree-root-output-hit
    out
  shows
    checked-staged-security-with-actual-alpha-prefix-branch-transcript-root-output-hit
    out
proof (cases out)
  case None

```

```

then show ?thesis
  using root-hit
  unfolding
    checked-staged-security-with-actual-alpha-prefix-branch-tree-root-output-hit-def
    checked-staged-security-with-actual-alpha-prefix-branch-tree-output-case-def
  by simp
next
  case (Some result-pack)
  then obtain x final-state where out-eq:
    out = Some (x, final-state)
  by (cases result-pack) simp
  let ?packed = fst (fst x)
  let ?prefix-state = snd (fst ?packed)
  let ?data = snd ?packed
  let ?attacker-state = snd (fst x)
  let ?result = snd x
  let ?s =
    verifier-state-from-adversary ?attacker-state
    (staged-proof-transcript ?data)
  from root-hit obtain trace-table composition-table as query-idxs dg
    composition-fri-roots final rest trace-tree composition-tree where
  verify-out:
    Some (?result, final-state) ∈ set-dist (execute verify-monad ?s)
  and bound:
    accepted-with-bound-tables ?s (Some (?result, final-state))
    trace-table composition-table as query-idxs
  and header:
    verifier-header-transcript ?s
    (staged-trace-root ?data)
    (staged-trace-fri-roots ?data)
    (staged-trace-final ?data)
    as dg composition-fri-roots final rest
  and trace-created:
    created-tree trace-table trace-tree final-state
  and composition-created:
    created-tree composition-table composition-tree final-state
  and tree-root-hit:
    hash-map-new-output-hit
    {value trace-tree, value composition-tree}
    ?prefix-state final-state
  unfolding out-eq
    checked-staged-security-with-actual-alpha-prefix-branch-tree-root-output-hit-def
    checked-staged-security-with-actual-alpha-prefix-branch-tree-output-case-def
    Let-def
  by simp blast
  from bound obtain fr' f-fri-roots' f-final' dg'
    composition-fri-roots' final' rest' where
  header':
    verifier-header-transcript ?s fr' f-fri-roots' f-final' as dg'

```

```

    composition-fri-roots' final' rest'
and trace-bind:
    merkle-root-binds-table fr' trace-table final-state
and composition-bind:
    merkle-root-binds-table (hd composition-fri-roots')
    composition-table final-state
unfolding accepted-with-bound-tables-def by auto
have header-eq:
    fr' = staged-trace-root ?data ∧
    f-fri-roots' = staged-trace-fri-roots ?data ∧
    f-final' = staged-trace-final ?data ∧
    dg' = dg ∧
    composition-fri-roots' = composition-fri-roots ∧
    final' = final ∧
    rest' = rest
using verifier-header-transcript-unique[OF header' header]
by simp
then have fr-eq: fr' = staged-trace-root ?data
and comp-roots-eq:
    composition-fri-roots' = composition-fri-roots
by simp-all
from trace-bind obtain trace-root-tree where
    trace-root-created:
        created-tree trace-table trace-root-tree final-state
and trace-root-eq: fr' = value trace-root-tree
unfolding merkle-root-binds-table-def by blast
have trace-value-eq:
    value trace-tree = staged-trace-root ?data
using created-tree-same-table-value-eq
    [OF trace-created trace-root-created]
    trace-root-eq fr-eq
by simp
from composition-bind obtain composition-root-tree where
    composition-root-created:
        created-tree composition-table composition-root-tree final-state
and composition-root-eq:
    hd composition-fri-roots' = value composition-root-tree
unfolding merkle-root-binds-table-def by blast
have composition-value-eq:
    value composition-tree = hd composition-fri-roots
using created-tree-same-table-value-eq
    [OF composition-created composition-root-created]
    composition-root-eq comp-roots-eq
by simp
have transcript-root-hit:
    hash-map-new-output-hit
    {staged-trace-root ?data, hd composition-fri-roots}
    ?prefix-state final-state
using tree-root-hit trace-value-eq composition-value-eq by simp

```

```

show ?thesis
  unfolding out-eq
    checked-staged-security-with-actual-alpha-prefix-branch-transcript-root-output-hit-def
    Let-def
  using verify-out bound header transcript-root-hit
  by simp blast
qed

lemma checked-staged-security-with-actual-alpha-prefix-branch-tree-root-output-hit-bound-from-transcript-roots:
  assumes root-bound:
    wp-event m
    checked-staged-security-with-actual-alpha-prefix-branch-transcript-root-output-hit
     $s \leq R$ 
  shows
    wp-event m
    checked-staged-security-with-actual-alpha-prefix-branch-tree-root-output-hit
     $s \leq R$ 
  by (rule order.trans[OF - root-bound])
    (rule wp-event-mono,
     rule
     checked-staged-security-with-actual-alpha-prefix-branch-tree-root-output-hit-imp-transcript-root-output-hit)

lemma checked-staged-security-with-actual-alpha-prefix-branch-transcript-root-output-hit-imp-preverifier-or-verifier-output-hit:
  assumes wf: staged-budget-wellformed budgets
  and controlled: staged-adversary-controlled budgets A
  and support:
    out  $\in$ 
    set-dist
    (execute
     (checked-staged-security-experiment-with-actual-alpha-prefix-data-state
      A)
     adversary-initial-state)
  and hit:
    checked-staged-security-with-actual-alpha-prefix-branch-transcript-root-output-hit
    out
  shows
    checked-staged-security-with-actual-alpha-prefix-branch-transcript-root-preverifier-output-hit
    out  $\vee$ 
    checked-staged-security-with-actual-alpha-prefix-branch-transcript-root-verifier-output-hit
    out
proof (cases out)
  case None
  then show ?thesis
    using hit
    unfolding
      checked-staged-security-with-actual-alpha-prefix-branch-transcript-root-output-hit-def
    by simp
next
  case (Some result-pack)

```



```

then obtain prefix prefix-state data attacker-state result final-state where
  out-eq:
    out =
      Some (((((prefix, prefix-state), data), attacker-state),
        result), final-state)
    by (cases result-pack, auto split: prod.splits)
let ?s =
  verifier-state-from-adversary attacker-state
  (staged-proof-transcript data)
have support-some:
  Some (((((prefix, prefix-state), data), attacker-state),
    result), final-state)  $\in$ 
  set-dist
  (execute
    (checked-staged-security-experiment-with-actual-alpha-prefix-data-state
      A)
    adversary-initial-state)
  using support unfolding out-eq .
from support-some obtain trans-out where trans-out:
  Some (((prefix, prefix-state), data), attacker-state)  $\in$ 
  set-dist
  (execute (checked-staged-transcript-with-alpha-prefix-program A)
    adversary-initial-state)
  unfolding
    checked-staged-security-experiment-with-actual-alpha-prefix-data-state-def
  by (auto elim!: set-dist-bindE split: prod.splits)
from checked-staged-transcript-with-alpha-prefix-program-support
  [OF wf controlled trans-out]
have prefix-ext: prefix-state  $\leq$  attacker-state
  by blast
have prefix-s: prefix-state  $\leq$  ?s
  by (rule
    alpha-prefix-hash-extends-verifier-state-from-adversary-right
    [OF prefix-ext])
from hit obtain trace-table composition-table as query-idxs dg
  composition-fri-roots final rest where
  verify-out:
    Some (result, final-state)  $\in$  set-dist (execute verify-monad ?s)
  and bound:
    accepted-with-bound-tables ?s (Some (result, final-state))
    trace-table composition-table as query-idxs
  and header:
    verifier-header-transcript ?s
    (staged-trace-root data)
    (staged-trace-fri-roots data)
    (staged-trace-final data)
    as dg composition-fri-roots final rest
  and root-hit:
    hash-map-new-output-hit

```

```

    {staged-trace-root data, hd composition-fri-roots}
    prefix-state final-state
unfolding out-eq
    checked-staged-security-with-actual-alpha-prefix-branch-transcript-root-output-hit-def
    Let-def
by simp blast
have s-final: ?s ≤ final-state
    by (rule verify-monad-hash-extends[OF verify-out])
from hash-map-new-output-hit-trans-decomp
    [OF prefix-s s-final root-hit]
show ?thesis
proof
    assume pre:
        hash-map-new-output-hit
        {staged-trace-root data, hd composition-fri-roots}
        prefix-state ?s
    have
        checked-staged-security-with-actual-alpha-prefix-branch-transcript-root-preverifier-output-hit
        out
    unfolding out-eq
        checked-staged-security-with-actual-alpha-prefix-branch-transcript-root-preverifier-output-hit-def
        Let-def
    using verify-out bound header pre
    by simp blast
    then show ?thesis by simp
next
    assume verifier:
        hash-map-new-output-hit
        {staged-trace-root data, hd composition-fri-roots}
        ?s final-state
    have
        checked-staged-security-with-actual-alpha-prefix-branch-transcript-root-verifier-output-hit
        out
    unfolding out-eq
        checked-staged-security-with-actual-alpha-prefix-branch-transcript-root-verifier-output-hit-def
        Let-def
    using verify-out bound header verifier
    by simp blast
    then show ?thesis by simp
qed
qed

lemma checked-staged-security-with-actual-alpha-prefix-branch-transcript-root-output-hit-bound-from-preverifier-and-
assumes wf: staged-budget-wellformed budgets
and controlled: staged-adversary-controlled budgets A
and pre-bound:
    wp-event
    (checked-staged-security-experiment-with-actual-alpha-prefix-data-state
    A)

```

checked-staged-security-with-actual-alpha-prefix-branch-transcript-root-preverifier-output-hit
adversary-initial-state $\leq P$
and *verifier-bound*:
wp-event
(checked-staged-security-experiment-with-actual-alpha-prefix-data-state
A)
checked-staged-security-with-actual-alpha-prefix-branch-transcript-root-verifier-output-hit
adversary-initial-state $\leq V$
shows
wp-event
(checked-staged-security-experiment-with-actual-alpha-prefix-data-state A)
checked-staged-security-with-actual-alpha-prefix-branch-transcript-root-output-hit
adversary-initial-state $\leq P + V$
proof –
let *?M* =
checked-staged-security-experiment-with-actual-alpha-prefix-data-state A
let *?Root* =
checked-staged-security-with-actual-alpha-prefix-branch-transcript-root-output-hit
let *?Pre* =
checked-staged-security-with-actual-alpha-prefix-branch-transcript-root-preverifier-output-hit
let *?Verifier* =
checked-staged-security-with-actual-alpha-prefix-branch-transcript-root-verifier-output-hit
have *wp-event ?M ?Root adversary-initial-state* $\leq$
wp-event ?M (λ out. ?Pre out $\vee$?Verifier out)
adversary-initial-state
by (rule *wp-event-mono-on-support*)
(rule
checked-staged-security-with-actual-alpha-prefix-branch-transcript-root-output-hit-imp-preverifier-or-verifier-on-s
[*OF wf controlled*])
also have ... $\leq$
wp-event ?M ?Pre adversary-initial-state +
wp-event ?M ?Verifier adversary-initial-state
by (rule *wp-event-union-bound*)
also have ... $\leq P + V$
by (intro *add-mono pre-bound verifier-bound*)
finally show *?thesis* .
qed

lemma *checked-staged-security-with-actual-alpha-prefix-branch-tree-root-output-hit-bound-from-preverifier-and-verifier*
assumes *wf: staged-budget-wellformed budgets*
and *controlled: staged-adversary-controlled budgets A*
and *pre-bound*:
wp-event
(checked-staged-security-experiment-with-actual-alpha-prefix-data-state
A)
checked-staged-security-with-actual-alpha-prefix-branch-transcript-root-preverifier-output-hit
adversary-initial-state $\leq P$
and *verifier-bound*:
wp-event

(*checked-staged-security-experiment-with-actual-alpha-prefix-data-state*
A)
checked-staged-security-with-actual-alpha-prefix-branch-transcript-root-verifier-output-hit
adversary-initial-state $\leq V$
shows
wp-event
 (*checked-staged-security-experiment-with-actual-alpha-prefix-data-state A*)
checked-staged-security-with-actual-alpha-prefix-branch-tree-root-output-hit
adversary-initial-state $\leq P + V$
proof –
have *root-bound*:
wp-event
 (*checked-staged-security-experiment-with-actual-alpha-prefix-data-state A*)
checked-staged-security-with-actual-alpha-prefix-branch-transcript-root-output-hit
adversary-initial-state $\leq P + V$
by (rule
checked-staged-security-with-actual-alpha-prefix-branch-transcript-root-output-hit-bound-from-preverifier-and-ver
[OF wf controlled pre-bound verifier-bound])
show *?thesis*
by (rule
checked-staged-security-with-actual-alpha-prefix-branch-tree-root-output-hit-bound-from-transcript-roots
[OF root-bound])
qed

lemma *checked-staged-security-with-actual-alpha-prefix-branch-tree-root-output-hit-bound-from-preverifier-and-concre*
assumes *wf: staged-budget-wellformed budgets*
and *controlled: staged-adversary-controlled budgets A*
and *pre-bound*:
wp-event
 (*checked-staged-security-experiment-with-actual-alpha-prefix-data-state*
A)
checked-staged-security-with-actual-alpha-prefix-branch-transcript-root-preverifier-output-hit
adversary-initial-state $\leq P$
and *target-bound*:
 $\bigwedge$ *packed attacker-state.*
Some (packed, attacker-state) $\in$
set-dist
 (*execute (checked-staged-transcript-with-alpha-prefix-program A)*
adversary-initial-state) $\implies$
concrete-transcript-target-error (staged-proof-transcript (snd packed))
 $\leq V$
shows
wp-event
 (*checked-staged-security-experiment-with-actual-alpha-prefix-data-state A*)
checked-staged-security-with-actual-alpha-prefix-branch-tree-root-output-hit
adversary-initial-state $\leq P + V$
proof –
have *verifier-bound*:
wp-event

(*checked-staged-security-experiment-with-actual-alpha-prefix-data-state* A)
checked-staged-security-with-actual-alpha-prefix-branch-transcript-root-verifier-output-hit
adversary-initial-state $\leq V$
by (rule
checked-staged-security-with-actual-alpha-prefix-branch-transcript-root-verifier-output-hit-bound-from-concrete-tr
 [*OF target-bound*])
show *?thesis*
by (rule
checked-staged-security-with-actual-alpha-prefix-branch-tree-root-output-hit-bound-from-preverifier-and-verifier
 [*OF wf controlled pre-bound verifier-bound*])
qed

lemma *checked-staged-security-with-actual-alpha-prefix-branch-tree-output-hit-bound-from-preverifier-concrete-transc*
assumes *wf: staged-budget-wellformed budgets*
and *controlled: staged-adversary-controlled budgets A*
and *pre-bound:*
wp-event
 (*checked-staged-security-experiment-with-actual-alpha-prefix-data-state*
A)
checked-staged-security-with-actual-alpha-prefix-branch-transcript-root-preverifier-output-hit
adversary-initial-state $\leq P$
and *target-bound:*
 $\bigwedge$ *packed attacker-state.*
Some (packed, attacker-state) $\in$
set-dist
 (*execute (checked-staged-transcript-with-alpha-prefix-program A)*
adversary-initial-state) $\implies$
concrete-transcript-target-error (staged-proof-transcript (snd packed))
 $\leq V$
and *subtree-bound:*
wp-event
 (*checked-staged-security-experiment-with-actual-alpha-prefix-data-state*
A)
checked-staged-security-with-actual-alpha-prefix-branch-tree-subtree-output-hit
adversary-initial-state $\leq S$
shows
wp-event
 (*checked-staged-security-experiment-with-actual-alpha-prefix-data-state A*)
checked-staged-security-with-actual-alpha-prefix-branch-tree-output-hit
adversary-initial-state $\leq P + V + S$
proof –
have *root-bound:*
wp-event
 (*checked-staged-security-experiment-with-actual-alpha-prefix-data-state A*)
checked-staged-security-with-actual-alpha-prefix-branch-tree-root-output-hit
adversary-initial-state $\leq P + V$
by (rule
checked-staged-security-with-actual-alpha-prefix-branch-tree-root-output-hit-bound-from-preverifier-and-concrete-t
 [*OF wf controlled pre-bound target-bound*])

have
wp-event
 (checked-staged-security-experiment-with-actual-alpha-prefix-data-state A)
 checked-staged-security-with-actual-alpha-prefix-branch-tree-output-hit
 adversary-initial-state $\leq (P + V) + S$
by (rule
 checked-staged-security-with-actual-alpha-prefix-branch-tree-output-hit-bound-from-root-and-subtree
 [OF root-bound subtree-bound])
then show ?thesis
 by (simp add: add.assoc)
qed

lemma checked-staged-security-with-actual-alpha-prefix-branch-tree-output-hit-bound-from-preverifier-concrete-transc

assumes wf: staged-budget-wellformed budgets

and controlled: staged-adversary-controlled budgets A

and root-pre-bound:

wp-event

(checked-staged-security-experiment-with-actual-alpha-prefix-data-state
 A)

checked-staged-security-with-actual-alpha-prefix-branch-transcript-root-preverifier-output-hit
 adversary-initial-state $\leq P$

and target-bound:

$\bigwedge$ packed attacker-state.

Some (packed, attacker-state) $\in$

set-dist

(execute (checked-staged-transcript-with-alpha-prefix-program A)
 adversary-initial-state) $\implies$

concrete-transcript-target-error (staged-proof-transcript (snd packed))
 $\leq V$

and subtree-pre-bound:

wp-event

(checked-staged-security-experiment-with-actual-alpha-prefix-data-state
 A)

checked-staged-security-with-actual-alpha-prefix-branch-tree-subtree-preverifier-output-hit
 adversary-initial-state $\leq SP$

and subtree-verifier-bound:

wp-event

(checked-staged-security-experiment-with-actual-alpha-prefix-data-state
 A)

checked-staged-security-with-actual-alpha-prefix-branch-tree-subtree-verifier-output-hit
 adversary-initial-state $\leq SV$

shows

wp-event

(checked-staged-security-experiment-with-actual-alpha-prefix-data-state A)
 checked-staged-security-with-actual-alpha-prefix-branch-tree-output-hit
 adversary-initial-state $\leq P + V + SP + SV$

proof —

have subtree-bound:

wp-event

```

    (checked-staged-security-experiment-with-actual-alpha-prefix-data-state A)
    checked-staged-security-with-actual-alpha-prefix-branch-tree-subtree-output-hit
    adversary-initial-state  $\leq SP + SV$ 
  by (rule
    checked-staged-security-with-actual-alpha-prefix-branch-tree-subtree-output-hit-bound-from-preverifier-and-verifier
    [OF wf controlled subtree-pre-bound subtree-verifier-bound])
  have
    wp-event
    (checked-staged-security-experiment-with-actual-alpha-prefix-data-state A)
    checked-staged-security-with-actual-alpha-prefix-branch-tree-output-hit
    adversary-initial-state  $\leq P + V + (SP + SV)$ 
  by (rule
    checked-staged-security-with-actual-alpha-prefix-branch-tree-output-hit-bound-from-preverifier-concrete-transcript
    [OF wf controlled root-pre-bound target-bound subtree-bound])
  then show ?thesis
  by (simp add: add.assoc)
qed

lemma checked-staged-security-with-data-state-verifier-event-actual-alpha-prefix-projection:
  wp-event (checked-staged-security-experiment-with-data-state A)
  (staged-security-with-data-state-verifier-event P)
  adversary-initial-state =
  wp-event
  (checked-staged-security-experiment-with-actual-alpha-prefix-data-state A)
  (checked-staged-security-with-actual-alpha-prefix-verifier-event P)
  adversary-initial-state
proof -
  let ?project =
     $\lambda x. ((snd (fst (fst x)), snd (fst x)), snd x)$ 
  have
    wp-event (checked-staged-security-experiment-with-data-state A)
    (staged-security-with-data-state-verifier-event P)
    adversary-initial-state =
    wp-event
    (checked-staged-security-experiment-with-actual-alpha-prefix-data-state A
       $\ggg (\lambda x. \text{return } (?project x))$ )
    (staged-security-with-data-state-verifier-event P)
    adversary-initial-state
  using
    checked-staged-security-experiment-with-actual-alpha-prefix-data-state-projection
    [of A]
  by simp
  also have ... =
    wp-event
    (checked-staged-security-experiment-with-actual-alpha-prefix-data-state A)
    ( $\lambda out. \text{case out of}$ 
      None  $\Rightarrow$  staged-security-with-data-state-verifier-event P None
      | Some (x, t)  $\Rightarrow$ 
        staged-security-with-data-state-verifier-event P

```

```

      (Some (?project x, t)))
    adversary-initial-state
  by (rule wp-event-bind-return-map)
also have ... =
  wp-event
  (checked-staged-security-experiment-with-actual-alpha-prefix-data-state A)
  (checked-staged-security-with-actual-alpha-prefix-verifier-event P)
  adversary-initial-state
unfolding staged-security-with-data-state-verifier-event-def
  checked-staged-security-with-actual-alpha-prefix-verifier-event-def
apply (rule arg-cong[where
  f= $\lambda Q.$  wp-event
  (checked-staged-security-experiment-with-actual-alpha-prefix-data-state A)
  Q adversary-initial-state])
apply (rule ext)
apply (simp add: Let-def split: option.splits prod.splits)
done
finally show ?thesis .
qed

lemma checked-staged-security-with-actual-alpha-prefix-composition-randomization-bad-imp-prefix-or-branch-tree-on-
assumes wf: staged-budget-wellformed budgets
and controlled: staged-adversary-controlled budgets A
and support:
  out  $\in$ 
  set-dist
  (execute
    (checked-staged-security-experiment-with-actual-alpha-prefix-data-state
      A)
    adversary-initial-state)
and bad:
  checked-staged-security-with-actual-alpha-prefix-verifier-event
  composition-randomization-bad out
shows
  checked-staged-security-with-actual-alpha-prefix-bad-set-hit
  composition-trace-bad-alpha-space out  $\vee$ 
  checked-staged-security-with-actual-alpha-prefix-branch-tree-output-hit
  out
proof (cases out)
case None
then show ?thesis
  using bad
  unfolding
    checked-staged-security-with-actual-alpha-prefix-verifier-event-def
  by simp
next
case (Some result-pack)
then obtain prefix prefix-state data attacker-state result final-state where
  out-eq:

```



```

    out =
      Some (((((prefix, prefix-state), data), attacker-state),
              result), final-state)
  by (cases result-pack, auto split: prod.splits)
let ?s =
  verifier-state-from-adversary attacker-state
  (staged-proof-transcript data)
have support-some:
  Some (((((prefix, prefix-state), data), attacker-state),
          result), final-state) ∈
  set-dist
  (execute
   (checked-staged-security-experiment-with-actual-alpha-prefix-data-state
    A)
   adversary-initial-state)
  using support unfolding out-eq .
from support-some obtain trans-out where trans-out:
  Some (((prefix, prefix-state), data), attacker-state) ∈
  set-dist
  (execute (checked-staged-transcript-with-alpha-prefix-program A)
   adversary-initial-state)
and verify-out:
  Some (result, final-state) ∈ set-dist (execute verify-monad ?s)
unfolding
  checked-staged-security-experiment-with-actual-alpha-prefix-data-state-def
by (auto elim!: set-dist-bindE split: prod.splits)
from checked-staged-transcript-with-alpha-prefix-program-support
  [OF wf controlled trans-out]
have prefix-out:
  Some (prefix, prefix-state) ∈
  set-dist
  (execute (staged-alpha-prefix-program A)
   adversary-initial-state)
and root-eq: staged-trace-root data = fst prefix
and trace-roots-eq:
  staged-trace-fri-roots data = fst (snd prefix)
and trace-bs-eq:
  staged-trace-fri-challenges data = fst (snd (snd prefix))
and trace-final-eq:
  staged-trace-final data = snd (snd (snd prefix))
and prefix-ext: prefix-state ≤ attacker-state
by blast+
have prefix-tuple:
  prefix =
    (staged-trace-root data, staged-trace-fri-roots data,
     staged-trace-fri-challenges data, staged-trace-final data)
  by (cases prefix; cases snd prefix; cases snd (snd prefix))
  (use root-eq trace-roots-eq trace-bs-eq trace-final-eq in auto)
have random-bad:

```

```

composition-randomization-bad ?s (Some (result, final-state))
using bad unfolding out-eq
  checked-staged-security-with-actual-alpha-prefix-verifier-event-def
  Let-def
by simp
have alpha-hit:
  alpha-bad-set-hit ?s composition-trace-bad-alpha-space
    (Some (result, final-state))
by (rule composition-randomization-bad-imp-alpha-bad-set-hit
  [OF random-bad])
from alpha-hit obtain trace-table composition-table as query-idxs where
  bound:
    accepted-with-bound-tables ?s (Some (result, final-state))
    trace-table composition-table as query-idxs
and as-bad: as  $\in$  composition-trace-bad-alpha-space trace-table
unfolding alpha-bad-set-hit-def by blast
have staged-header:
  verifier-header-transcript ?s
    (staged-trace-root data)
    (staged-trace-fri-roots data)
    (staged-trace-final data)
    (staged-alphas data)
    (staged-degree data)
    (staged-composition-fri-roots data)
    (staged-composition-final data)
    (List.concat (staged-query-chunks data))
proof –
have projected:
  Some (data, attacker-state)  $\in$ 
    set-dist
      (execute
        (checked-staged-transcript-with-alpha-prefix-program A  $\gg$ 
          ( $\lambda$ packed. return (snd packed)))
        adversary-initial-state)
by (rule set-dist-bindI[OF trans-out]) simp
have checked-builder:
  Some (data, attacker-state)  $\in$ 
    set-dist
      (execute (checked-staged-transcript-program A)
        adversary-initial-state)
using projected
unfolding checked-staged-transcript-with-alpha-prefix-program-projection
by simp
show ?thesis
by (rule checked-staged-transcript-program-outcome-header-transcript
  [OF wf controlled checked-builder])
qed
from bound obtain fr f-fri-roots f-final dg composition-fri-roots final
  rest where

```

```

header:
  verifier-header-transcript ?s fr f-fri-roots f-final as dg
  composition-fri-roots final rest
unfolding accepted-with-bound-tables-def by blast
have header-eq:
  fr = staged-trace-root data ∧
  f-fri-roots = staged-trace-fri-roots data ∧
  f-final = staged-trace-final data ∧
  as = staged-alphas data ∧
  dg = staged-degree data ∧
  composition-fri-roots = staged-composition-fri-roots data ∧
  final = staged-composition-final data ∧
  rest = List.concat (staged-query-chunks data)
using verifier-header-transcript-unique[OF header staged-header]
by simp
have staged-bad:
  staged-alphas data ∈ composition-trace-bad-alpha-space trace-table
using as-bad header-eq by simp
have alphas-space: staged-alphas data ∈ alpha-space
using staged-bad composition-trace-bad-alpha-space-subset-alpha-space
by blast
have prefix-s: prefix-state ≤ ?s
by (rule
  alpha-prefix-hash-extends-verifier-state-from-adversary-right
  [OF prefix-ext])
have s-final: ?s ≤ final-state
by (rule verify-monad-hash-extends[OF verify-out])
have prefix-final: prefix-state ≤ final-state
by (rule hash-ext-trans[OF prefix-s s-final])
show ?thesis
proof (rule
  accepted-with-bound-tables-initial-roots-pullback-or-new-tree-output-hit
  [OF prefix-final bound])
fix fr' f-fri-roots' f-final' dg' composition-fri-roots' final'
  rest' trace-tree composition-tree
assume pull-header:
  verifier-header-transcript ?s fr' f-fri-roots' f-final' as dg'
  composition-fri-roots' final' rest'
and trace-created:
  created-tree trace-table trace-tree final-state
and composition-created:
  created-tree composition-table composition-tree final-state
and split:
  (merkle-root-binds-table fr' trace-table prefix-state ∧
  merkle-root-binds-table (hd composition-fri-roots')
  composition-table prefix-state) ∨
  hash-map-new-output-hit
  (set-tree trace-tree ∪ set-tree composition-tree)
  prefix-state final-state

```

```

have pull-eq:
  fr' = staged-trace-root data ∧
  f-fri-roots' = staged-trace-fri-roots data ∧
  f-final' = staged-trace-final data ∧
  dg' = staged-degree data ∧
  composition-fri-roots' = staged-composition-fri-roots data ∧
  final' = staged-composition-final data ∧
  rest' = List.concat (staged-query-chunks data)
using verifier-header-transcript-unique[OF pull-header staged-header]
  header-eq
by simp
from split show ?thesis
proof
  assume binds:
    merkle-root-binds-table fr' trace-table prefix-state ∧
    merkle-root-binds-table (hd composition-fri-roots')
    composition-table prefix-state
  have trace-len: length trace-table = clength * scale
    using bound unfolding accepted-with-bound-tables-def
    accepted-with-tables-def by simp
  have prefix-candidate:
    trace-table ∈
      alpha-prefix-trace-table-candidates prefix-state (fst prefix)
    by (rule alpha-prefix-trace-table-candidateI[OF trace-len])
    (use binds pull-eq root-eq in simp)
  have prefix-bad:
    staged-alphas data ∈
      alpha-prefix-union-bad-sets prefix-state
      composition-trace-bad-alpha-space (fst prefix)
    unfolding alpha-prefix-union-bad-sets-def
    using alphas-space prefix-candidate staged-bad by auto
  have
    checked-staged-security-with-actual-alpha-prefix-bad-set-hit
    composition-trace-bad-alpha-space out
    unfolding out-eq
    checked-staged-security-with-actual-alpha-prefix-bad-set-hit-def
    Let-def
    using prefix-bad by simp
  then show ?thesis by simp
next
  assume tree-hit:
    hash-map-new-output-hit
    (set-tree trace-tree ∪ set-tree composition-tree)
    prefix-state final-state
  have
    checked-staged-security-with-actual-alpha-prefix-branch-tree-output-hit
    out
    unfolding out-eq
    checked-staged-security-with-actual-alpha-prefix-branch-tree-output-hit-def

```

```

    Let-def
    using verify-out bound pull-header trace-created composition-created
      tree-hit pull-eq
    by simp blast
    then show ?thesis by simp
qed
qed
qed

lemma checked-staged-security-with-actual-alpha-prefix-composition-randomization-bad-bound-from-prefix-and-branch
  assumes wf: staged-budget-wellformed budgets
  and controlled: staged-adversary-controlled budgets A
  and prefix-bound:
    wp-event
      (checked-staged-security-experiment-with-actual-alpha-prefix-data-state
        A)
      (checked-staged-security-with-actual-alpha-prefix-bad-set-hit
        composition-trace-bad-alpha-space)
      adversary-initial-state  $\leq P$ 
  and branch-tree-bound:
    wp-event
      (checked-staged-security-experiment-with-actual-alpha-prefix-data-state
        A)
      checked-staged-security-with-actual-alpha-prefix-branch-tree-output-hit
      adversary-initial-state  $\leq T$ 
  shows
    wp-event
      (checked-staged-security-experiment-with-actual-alpha-prefix-data-state A)
      (checked-staged-security-with-actual-alpha-prefix-verifier-event
        composition-randomization-bad)
      adversary-initial-state  $\leq P + T$ 
proof -
  let ?M =
    checked-staged-security-experiment-with-actual-alpha-prefix-data-state A
  let ?random =
    checked-staged-security-with-actual-alpha-prefix-verifier-event
    composition-randomization-bad
  let ?prefix =
    checked-staged-security-with-actual-alpha-prefix-bad-set-hit
    composition-trace-bad-alpha-space
  let ?tree =
    checked-staged-security-with-actual-alpha-prefix-branch-tree-output-hit
  have wp-event ?M ?random adversary-initial-state  $\leq$ 
    wp-event ?M ( $\lambda$ out. ?prefix out  $\vee$  ?tree out)
    adversary-initial-state
  by (rule wp-event-mono-on-support)
  (rule
    checked-staged-security-with-actual-alpha-prefix-composition-randomization-bad-imp-prefix-or-branch-tree-on-sup
    [OF wf controlled])

```

also have ... $\leq$
 $wp\text{-event } ?M \text{ ?prefix adversary-initial-state} +$
 $wp\text{-event } ?M \text{ ?tree adversary-initial-state}$
 by (rule *wp-event-union-bound*)
 also have ... $\leq P + T$
 by (intro *add-mono prefix-bound branch-tree-bound*)
 finally show *?thesis* .
 qed

lemma *checked-staged-security-with-data-state-composition-bad-bound-from-actual-alpha-prefix-branch-tree*:
assumes *wf*: *staged-budget-wellformed budgets*
and *controlled*: *staged-adversary-controlled budgets A*
and *prefix-bound*:
 $wp\text{-event}$
 (*checked-staged-security-experiment-with-actual-alpha-prefix-data-state*
 A)
 (*checked-staged-security-with-actual-alpha-prefix-bad-set-hit*
 composition-trace-bad-alpha-space)
 adversary-initial-state $\leq P$
and *branch-tree-bound*:
 $wp\text{-event}$
 (*checked-staged-security-experiment-with-actual-alpha-prefix-data-state*
 A)
 checked-staged-security-with-actual-alpha-prefix-branch-tree-output-hit
 adversary-initial-state $\leq T$
shows
 $wp\text{-event}$ (*checked-staged-security-experiment-with-data-state A*)
 (*staged-security-with-data-state-verifier-event composition-bad*)
 adversary-initial-state $\leq P + T$
proof –
 let *?M* =
 checked-staged-security-experiment-with-actual-alpha-prefix-data-state A
 let *?degree* =
 checked-staged-security-with-actual-alpha-prefix-verifier-event
 composition-degree-bad
 let *?random* =
 checked-staged-security-with-actual-alpha-prefix-verifier-event
 composition-randomization-bad
 let *?bad* =
 checked-staged-security-with-actual-alpha-prefix-verifier-event
 composition-bad
have *projected*:
 $wp\text{-event}$ (*checked-staged-security-experiment-with-data-state A*)
 (*staged-security-with-data-state-verifier-event composition-bad*)
 adversary-initial-state =
 $wp\text{-event } ?M \text{ ?bad adversary-initial-state}$
 by (rule
 checked-staged-security-with-data-state-verifier-event-actual-alpha-prefix-projection)
have *event-eq*: *?bad* = ($\lambda out. ?degree \text{ out} \vee ?random \text{ out}$)

```

  by (rule ext)
    (auto simp:
      checked-staged-security-with-actual-alpha-prefix-verifier-event-def
      composition-bad-def split: option.splits prod.splits)
have degree-false: ?degree = ( $\lambda$ out. False)
  by (rule ext)
    (auto simp:
      checked-staged-security-with-actual-alpha-prefix-verifier-event-def
      composition-degree-bad-false[OF spec-degree-wellformed-from-spec-query-margin]
      split: option.splits prod.splits)
have degree-bound:
  wp-event ?M ?degree adversary-initial-state  $\leq 0$ 
  unfolding degree-false wp-event-def wp-def dist-expect-def by simp
have random-bound:
  wp-event ?M ?random adversary-initial-state  $\leq P + T$ 
  by (rule
    checked-staged-security-with-actual-alpha-prefix-composition-randomization-bad-bound-from-prefix-and-branch-tree
    [OF wf controlled prefix-bound branch-tree-bound])
have wp-event ?M ?bad adversary-initial-state  $\leq$ 
  wp-event ?M ?degree adversary-initial-state +
  wp-event ?M ?random adversary-initial-state
  unfolding event-eq by (rule wp-event-union-bound)
also have ...  $\leq 0 + (P + T)$ 
  by (intro add-mono degree-bound random-bound)
finally show ?thesis
  unfolding projected by simp
qed

lemma checked-staged-security-with-data-state-composition-bad-bound-from-actual-alpha-prefix-branch-tree-and-budget
  assumes wf: staged-budget-wellformed budgets
  and controlled: staged-adversary-controlled budgets A
  and branch-tree-bound:
    wp-event
      (checked-staged-security-experiment-with-actual-alpha-prefix-data-state
        A)
      checked-staged-security-with-actual-alpha-prefix-branch-tree-output-hit
      adversary-initial-state  $\leq T$ 
  shows
    wp-event (checked-staged-security-experiment-with-data-state A)
      (staged-security-with-data-state-verifier-event composition-bad)
      adversary-initial-state  $\leq$ 
      composition-error-bound +
      staged-phase-relation-error size
      (staged-alpha-search-queries budgets 0) +
      hash-collision-budget-value 0
      (staged-alpha-search-queries budgets 0) +
      T
proof -
  let ?P =

```

$composition-error-bound +$
 $staged-phase-relation-error\ size$
 $(staged-alpha-search-queries\ budgets\ 0) +$
 $hash-collision-budget-value\ 0$
 $(staged-alpha-search-queries\ budgets\ 0)$
have *transcript-bound*:
 $wp-event\ (checked-staged-transcript-with-alpha-prefix-program\ A)$
 $(checked-staged-transcript-actual-alpha-prefix-bad-set-hit$
 $composition-trace-bad-alpha-space)$
 $adversary-initial-state \leq ?P$
by (rule
 $checked-staged-transcript-with-alpha-prefix-actual-bad-set-hit-bound-from-budgets$
 $[OF\ wf\ controlled])$
have *prefix-bound*:
 $wp-event$
 $(checked-staged-security-experiment-with-actual-alpha-prefix-data-state$
 $A)$
 $(checked-staged-security-with-actual-alpha-prefix-bad-set-hit$
 $composition-trace-bad-alpha-space)$
 $adversary-initial-state \leq ?P$
by (rule
 $checked-staged-security-with-actual-alpha-prefix-bad-set-hit-bound-from-transcript$
 $[OF\ transcript-bound])$
have
 $wp-event\ (checked-staged-security-experiment-with-data-state\ A)$
 $(staged-security-with-data-state-verifier-event\ composition-bad)$
 $adversary-initial-state \leq ?P + T$
by (rule
 $checked-staged-security-with-data-state-composition-bad-bound-from-actual-alpha-prefix-branch-tree$
 $[OF\ wf\ controlled\ prefix-bound\ branch-tree-bound])$
then show *?thesis*
by (*simp add: add.assoc*)
qed

lemma *checked-staged-security-with-data-state-composition-bad-bound-from-branch-split-side-events*:
assumes *wf: staged-budget-wellformed budgets*
and *controlled: staged-adversary-controlled budgets A*
and *root-pre-bound*:
 $wp-event$
 $(checked-staged-security-experiment-with-actual-alpha-prefix-data-state$
 $A)$
 $checked-staged-security-with-actual-alpha-prefix-branch-transcript-root-preverifier-output-hit$
 $adversary-initial-state \leq RP$
and *target-bound*:
 $\bigwedge packed\ attacker-state.$
 $Some\ (packed,\ attacker-state) \in$
 $set-dist$
 $(execute\ (checked-staged-transcript-with-alpha-prefix-program\ A)$
 $adversary-initial-state) \implies$

$concrete-transcript-target-error \ (staged-proof-transcript \ (snd \ packed))$
 $\leq RV$
and *subtree-pre-bound*:
 $wp-event$
 $(checked-staged-security-experiment-with-actual-alpha-prefix-data-state$
 $A)$
 $checked-staged-security-with-actual-alpha-prefix-branch-tree-subtree-preverifier-output-hit$
 $adversary-initial-state \leq SP$
and *subtree-verifier-bound*:
 $wp-event$
 $(checked-staged-security-experiment-with-actual-alpha-prefix-data-state$
 $A)$
 $checked-staged-security-with-actual-alpha-prefix-branch-tree-subtree-verifier-output-hit$
 $adversary-initial-state \leq SV$
shows
 $wp-event \ (checked-staged-security-experiment-with-data-state \ A)$
 $(staged-security-with-data-state-verifier-event \ composition-bad)$
 $adversary-initial-state \leq$
 $composition-error-bound +$
 $staged-phase-relation-error \ size$
 $(staged-alpha-search-queries \ budgets \ 0) +$
 $hash-collision-budget-value \ 0$
 $(staged-alpha-search-queries \ budgets \ 0) +$
 $(RP + RV + SP + SV)$
proof –
have *branch-tree-bound*:
 $wp-event$
 $(checked-staged-security-experiment-with-actual-alpha-prefix-data-state \ A)$
 $checked-staged-security-with-actual-alpha-prefix-branch-tree-output-hit$
 $adversary-initial-state \leq RP + RV + SP + SV$
by (rule
 $checked-staged-security-with-actual-alpha-prefix-branch-tree-output-hit-bound-from-preverifier-concrete-transcript-b$
 $[OF \ wf \ controlled \ root-pre-bound \ target-bound \ subtree-pre-bound$
 $subtree-verifier-bound])$
show *?thesis*
by (rule
 $checked-staged-security-with-data-state-composition-bad-bound-from-actual-alpha-prefix-branch-tree-and-budgets$
 $[OF \ wf \ controlled \ branch-tree-bound])$
qed

lemma *checked-staged-security-with-data-state-composition-bad-bound-from-branch-split-side-events-and-transcript-b*
assumes *wf*: *staged-budget-wellformed* *budgets*
and *controlled*: *staged-adversary-controlled* *budgets* *A*
and *root-pre-bound*:
 $wp-event$
 $(checked-staged-security-experiment-with-actual-alpha-prefix-data-state$
 $A)$
 $checked-staged-security-with-actual-alpha-prefix-branch-transcript-root-preverifier-output-hit$
 $adversary-initial-state \leq RP$

and *subtree-pre-bound*:
wp-event
(checked-staged-security-experiment-with-actual-alpha-prefix-data-state
A)
checked-staged-security-with-actual-alpha-prefix-branch-tree-subtree-preverifier-output-hit
adversary-initial-state $\leq$ *SP*
and *subtree-verifier-bound*:
wp-event
(checked-staged-security-experiment-with-actual-alpha-prefix-data-state
A)
checked-staged-security-with-actual-alpha-prefix-branch-tree-subtree-verifier-output-hit
adversary-initial-state $\leq$ *SV*
shows
wp-event (checked-staged-security-experiment-with-data-state A)
(staged-security-with-data-state-verifier-event composition-bad)
adversary-initial-state $\leq$
composition-error-bound +
staged-phase-relation-error size
(staged-alpha-search-queries budgets 0) +
hash-collision-budget-value 0
(staged-alpha-search-queries budgets 0) +
(RP + staged-concrete-transcript-target-error-bound + SP + SV)
proof –
have *target-bound*:
 $\bigwedge$ *packed attacker-state.*
Some (packed, attacker-state) ∈
set-dist
(execute (checked-staged-transcript-with-alpha-prefix-program A)
adversary-initial-state) ⇒
concrete-transcript-target-error (staged-proof-transcript (snd packed))
 $\leq$ *staged-concrete-transcript-target-error-bound*
by (rule
checked-staged-transcript-with-alpha-prefix-program-concrete-transcript-target-error-bound
[*OF wf controlled*])
show ?thesis
by (rule
checked-staged-security-with-data-state-composition-bad-bound-from-branch-split-side-events
[*OF wf controlled root-pre-bound target-bound subtree-pre-bound*
subtree-verifier-bound])
qed
lemma *checked-staged-security-with-data-state-composition-bad-bound-from-branch-split-side-events-and-transcript-b*
assumes *wf: staged-budget-wellformed budgets*
and *controlled: staged-adversary-controlled budgets A*
and *subtree-pre-bound*:
wp-event
(checked-staged-security-experiment-with-actual-alpha-prefix-data-state
A)
checked-staged-security-with-actual-alpha-prefix-branch-tree-subtree-preverifier-output-hit

```

    adversary-initial-state ≤ SP
and subtree-verifier-bound:
  wp-event
    (checked-staged-security-experiment-with-actual-alpha-prefix-data-state
      A)
    checked-staged-security-with-actual-alpha-prefix-branch-tree-subtree-verifier-output-hit
    adversary-initial-state ≤ SV
shows
  wp-event (checked-staged-security-experiment-with-data-state A)
    (staged-security-with-data-state-verifier-event composition-bad)
    adversary-initial-state ≤
      composition-error-bound +
      staged-phase-relation-error size
      (staged-alpha-search-queries budgets 0) +
      hash-collision-budget-value 0
      (staged-alpha-search-queries budgets 0) +
      (staged-phase-relation-error size
        (staged-attacker-query-budget budgets + staged-challenge-query-budget) +
        staged-concrete-transcript-target-error-bound + SP + SV)
proof –
  have root-pre-bound:
    wp-event
      (checked-staged-security-experiment-with-actual-alpha-prefix-data-state
        A)
      checked-staged-security-with-actual-alpha-prefix-branch-transcript-root-preverifier-output-hit
      adversary-initial-state ≤
        staged-phase-relation-error size
        (staged-attacker-query-budget budgets + staged-challenge-query-budget)
    by (rule
      checked-staged-security-with-actual-alpha-prefix-branch-transcript-root-preverifier-output-hit-bound
      [OF wf controlled])
  show ?thesis
    by (rule
      checked-staged-security-with-data-state-composition-bad-bound-from-branch-split-side-events-and-transcript-budg
      [OF wf controlled root-pre-bound subtree-pre-bound
        subtree-verifier-bound])
qed

end

end

theory Staged-Security-Experiment
  imports Staged-Security-Experiment-Composition-Branch
begin

  Compatibility umbrella for the staged-adversary security experiment.

end

```

```

theory Soundness-Partial-Merkle
  imports
    Staged-Security-Experiment
begin

```

0.1 Partial Merkle Openings

This theory records the authenticated openings that the verifier actually observes. It provides partial table candidates, consistency predicates, and reductions from inconsistent authenticated openings to hash-collision events.

```

record 'f authenticated-opening =
  opening-root :: 'f
  opening-length :: nat
  opening-index :: nat
  opening-value :: 'f
  opening-path :: 'f list

fun merkle-path-bound
  :: 'f  $\Rightarrow$  nat  $\Rightarrow$  nat  $\Rightarrow$  'f  $\Rightarrow$  'f list  $\Rightarrow$ 
    ('f, 'a) protocol-channel-scheme  $\Rightarrow$  bool
where
  merkle-path-bound rt len idx v [] s  $\longleftrightarrow$ 
    fmlookup (HashMap s) (MerkleLeaf v) = Some rt
| merkle-path-bound rt len idx v (sibling # path) s  $\longleftrightarrow$ 
    (if idx < len div 2 then
      ( $\exists$  child.
        merkle-path-bound child (len div 2) idx v path s  $\wedge$ 
        fmlookup (HashMap s) (MerkleNode child sibling) = Some rt)
    else
      ( $\exists$  child.
        merkle-path-bound child (len div 2) (idx - len div 2)
          v path s  $\wedge$ 
        fmlookup (HashMap s) (MerkleNode sibling child) = Some rt))

context soundness
begin

definition authenticated-opening-in
  :: ('f, 'a) protocol-channel-scheme  $\Rightarrow$ 
    'f authenticated-opening  $\Rightarrow$  bool
where
  authenticated-opening-in s opening  $\longleftrightarrow$ 
    opening-index opening < opening-length opening  $\wedge$ 
    length (opening-path opening) =
      floor-log (opening-length opening)  $\wedge$ 
    merkle-path-bound

```

(opening-root opening)
 (opening-length opening)
 (opening-index opening)
 (opening-value opening)
 (opening-path opening)

s

definition *partial-authenticated-table*

:: 'f $\Rightarrow$ nat $\Rightarrow$ 'f authenticated-opening list $\Rightarrow$
 ('f, 'a) protocol-channel-scheme $\Rightarrow$ bool

where

partial-authenticated-table rt len openings *s* $\longleftrightarrow$
 ($\forall$ opening $\in$ set openings.
 opening-root opening = rt $\wedge$
 opening-length opening = len $\wedge$
 authenticated-opening-in *s* opening)

lemma *merkle-path-bound-mono*:

assumes bound: merkle-path-bound rt len idx *v* path *s*

and ext: $s \leq t$

shows merkle-path-bound rt len idx *v* path *t*

using bound

proof (induction path arbitrary: rt len idx)

case Nil

then show ?case

using hash-extension-lookup[OF - ext] **by** simp

next

case (Cons sibling path)

show ?case

proof (cases idx < len div 2)

case True

then obtain child **where**

child:

merkle-path-bound child (len div 2) idx *v* path *s*

and lookup:

fmllookup (HashMap *s*) (MerkleNode child sibling) = Some rt

using Cons.premis **by** auto

have child-t:

merkle-path-bound child (len div 2) idx *v* path *t*

by (rule Cons.IH[OF child])

have lookup-t:

fmllookup (HashMap *t*) (MerkleNode child sibling) = Some rt

by (rule hash-extension-lookup[OF lookup ext])

show ?thesis

using True child-t lookup-t

by (auto intro!: exI[of - child])

next

case False

then obtain child **where**

```

  child:
    merkle-path-bound child (len div 2) (idx - len div 2)
    v path s
  and lookup:
    fmlookup (HashMap s) (MerkleNode sibling child) = Some rt
  using Cons.premis by auto
have child-t:
  merkle-path-bound child (len div 2) (idx - len div 2)
  v path t
  by (rule Cons.IH[OF child])
have lookup-t:
  fmlookup (HashMap t) (MerkleNode sibling child) = Some rt
  by (rule hash-extension-lookup[OF lookup ext])
show ?thesis
  using False child-t lookup-t
  by (auto intro!: exI[of - child])
qed
qed

```

```

lemma authenticated-opening-in-mono:
  assumes opn-auth: authenticated-opening-in s opn
  and ext:  $s \leq t$ 
  shows authenticated-opening-in t opn
  using opn-auth merkle-path-bound-mono[OF - ext]
  unfolding authenticated-opening-in-def by blast

```

```

lemma partial-authenticated-table-mono:
  assumes table: partial-authenticated-table rt len openings s
  and ext:  $s \leq t$ 
  shows partial-authenticated-table rt len openings t
  using table authenticated-opening-in-mono[OF - ext]
  unfolding partial-authenticated-table-def by blast

```

```

lemma merkle-path-bound-root-lookup:
  assumes merkle-path-bound rt len idx v path s
  shows  $\exists \text{input. fmlookup (HashMap s) input} = \text{Some rt}$ 
  using assms
proof (induction path arbitrary: rt len idx)
  case Nil
  then show ?case
    by (intro exI[of - MerkleLeaf v]) simp
next
  case (Cons sibling path)
  then show ?case
    by (cases idx < len div 2) auto
qed

```

```

lemma authenticated-opening-root-lookup:
  assumes authenticated-opening-in s opening

```

```

shows  $\exists \text{input}. \text{fmlookup } (\text{HashMap } s) \text{ input} = \text{Some } (\text{opening-root opening})$ 
using assms
unfolding authenticated-opening-in-def
by (blast intro: merkle-path-bound-root-lookup)

lemma authenticated-opening-root-late-output-preexisting-or-new-hit:
  assumes ext: s ≤ t
  and auth: authenticated-opening-in t opening
shows
  ( $\exists \text{input}. \text{fmlookup } (\text{HashMap } s) \text{ input} = \text{Some } (\text{opening-root opening})$ )  $\vee$ 
  hash-map-new-output-hit {opening-root opening} s t
proof –
  from authenticated-opening-root-lookup[OF auth]
obtain input where lookup-t:
  fmlookup (HashMap t) input = Some (opening-root opening)
  by blast
show ?thesis
proof (cases fmlookup (HashMap s) input)
  case None
  then have hash-map-new-output-hit {opening-root opening} s t
  unfolding hash-map-new-output-hit-def
  using lookup-t by blast
  then show ?thesis by simp
next
  case (Some old)
  have lookup-t-old: fmlookup (HashMap t) input = Some old
  by (rule hash-extension-lookup[OF Some ext])
  then have old = opening-root opening
  using lookup-t by simp
  then show ?thesis
  using Some by blast
qed
qed

lemma partial-authenticated-table-root-late-output-preexisting-or-new-hit:
  assumes ext: s ≤ t
  and table: partial-authenticated-table rt len openings t
  and opn-in: opening ∈ set openings
shows
  ( $\exists \text{input}. \text{fmlookup } (\text{HashMap } s) \text{ input} = \text{Some } rt$ )  $\vee$ 
  hash-map-new-output-hit {rt} s t
proof –
  have auth: authenticated-opening-in t opening
  using table opn-in unfolding partial-authenticated-table-def by blast
  have root-eq: opening-root opening = rt
  using table opn-in unfolding partial-authenticated-table-def by blast
show ?thesis
  using authenticated-opening-root-late-output-preexisting-or-new-hit
  [OF ext auth]

```

unfolding *root-eq* .
qed

lemma *merkle-path-bound-root-in-map*:
 assumes *merkle-path-bound* *rt* *len* *idx* *v* *path* *s*
 shows *merkle-path-root-in-map* *s* *len* *idx* *v* *path* = *Some* *rt*
 using *assms*
proof (*induction* *path* *arbitrary*: *rt* *len* *idx*)
 case *Nil*
 then show ?case **by** *simp*
next
 case (*Cons* *sibling* *path*)
 then show ?case
 by (*cases* *idx* < *len* *div* 2) *auto*
 qed

lemma *merkle-path-root-in-map-some-imp-bound*:
 assumes *merkle-path-root-in-map* *s* *len* *idx* *v* *path* = *Some* *rt*
 shows *merkle-path-bound* *rt* *len* *idx* *v* *path* *s*
 using *assms*
proof (*induction* *path* *arbitrary*: *rt* *len* *idx*)
 case *Nil*
 then show ?case **by** *simp*
next
 case (*Cons* *sibling* *path*)
 show ?case
proof (*cases* *idx* < *len* *div* 2)
 case *True*
 from *Cons.prem*s **obtain** *child* **where**
 child-root:
 merkle-path-root-in-map *s* (*len* *div* 2) *idx* *v* *path* =
 Some *child*
 and *lookup*:
 fmlookup (*HashMap* *s*) (*MerkleNode* *child* *sibling*) = *Some* *rt*
 using *True* **by** (*auto* *split*: *option.splits*)
have *child-bound*:
 merkle-path-bound *child* (*len* *div* 2) *idx* *v* *path* *s*
 by (*rule* *Cons.IH*[*OF* *child-root*])
show ?thesis
 using *True* *child-bound* *lookup* **by** *auto*
next
 case *False*
 from *Cons.prem*s **obtain** *child* **where**
 child-root:
 merkle-path-root-in-map *s* (*len* *div* 2) (*idx* - *len* *div* 2)
 v *path* = *Some* *child*
 and *lookup*:
 fmlookup (*HashMap* *s*) (*MerkleNode* *sibling* *child*) = *Some* *rt*
 using *False* **by** (*auto* *split*: *option.splits*)


```

have child-bound:
  merkle-path-bound child (len div 2) (idx - len div 2) v path s
  by (rule Cons.IH[OF child-root])
show ?thesis
  using False child-bound lookup by auto
qed
qed

fun merkle-path-target-roots
  :: ('f, 'a) protocol-channel-scheme  $\Rightarrow$  'f  $\Rightarrow$  nat  $\Rightarrow$  nat  $\Rightarrow$ 
    'f  $\Rightarrow$  'f list  $\Rightarrow$  'f set
where
  merkle-path-target-roots t rt len idx v [] = {rt}
  | merkle-path-target-roots t rt len idx v (sibling # path) =
    {rt}  $\cup$ 
    (if idx < len div 2 then
      (case merkle-path-root-in-map t (len div 2) idx v path of
        None  $\Rightarrow$  {}
      | Some child  $\Rightarrow$ 
        merkle-path-target-roots t child (len div 2) idx v path)
    else
      (case merkle-path-root-in-map t (len div 2) (idx - len div 2)
        v path of
        None  $\Rightarrow$  {}
      | Some child  $\Rightarrow$ 
        merkle-path-target-roots t child (len div 2)
          (idx - len div 2) v path)))

lemma finite-merkle-path-target-roots[simp]:
  finite (merkle-path-target-roots t rt len idx v path)
  by (induction path arbitrary: rt len idx)
  (auto split: option.splits if-splits)

lemma card-merkle-path-target-roots-le:
  card (merkle-path-target-roots t rt len idx v path)  $\leq$  Suc (length path)
proof (induction path arbitrary: rt len idx)
  case Nil
  then show ?case by simp
next
  case (Cons sibling path)
  show ?case
  proof (cases idx < len div 2)
  case True
  then show ?thesis
  proof (cases merkle-path-root-in-map t (len div 2) idx v path)
  case None
  then show ?thesis
  using True by simp
  next

```

```

case (Some child)
have finite-child:
  finite (merkle-path-target-roots t child (len div 2) idx v path)
  by simp
have card-le:
  card ({rt}  $\cup$ 
    merkle-path-target-roots t child (len div 2) idx v path)  $\leq$ 
    Suc (card
      (merkle-path-target-roots t child (len div 2) idx v path))
  using finite-child by (simp add: card-insert-if)
also have ...  $\leq$  Suc (Suc (length path))
  using Cons.IH[of child len div 2 idx] by simp
finally show ?thesis
  using True Some by simp
qed
next
case False
then show ?thesis
proof (cases
  merkle-path-root-in-map t (len div 2) (idx - len div 2) v path)
case None
then show ?thesis
  using False by simp
next
case (Some child)
have finite-child:
  finite (merkle-path-target-roots t child (len div 2)
    (idx - len div 2) v path)
  by simp
have card-le:
  card ({rt}  $\cup$ 
    merkle-path-target-roots t child (len div 2)
      (idx - len div 2) v path)  $\leq$ 
    Suc (card
      (merkle-path-target-roots t child (len div 2)
        (idx - len div 2) v path))
  using finite-child by (simp add: card-insert-if)
also have ...  $\leq$  Suc (Suc (length path))
  using Cons.IH[of child len div 2 idx - len div 2] by simp
finally show ?thesis
  using False Some by simp
qed
qed
qed

lemma merkle-path-bound-pullback-or-new-output-hit:
assumes ext:  $s \leq t$ 
and bound: merkle-path-bound rt len idx v path t
shows

```

```

merkle-path-bound rt len idx v path s ∨
hash-map-new-output-hit
(merkle-path-target-roots t rt len idx v path) s t
using bound
proof (induction path arbitrary: rt len idx)
case Nil
have lookup-t: fmlookup (HashMap t) (MerkleLeaf v) = Some rt
using Nil.premis by simp
show ?case
proof (cases fmlookup (HashMap s) (MerkleLeaf v))
case None
then have
hash-map-new-output-hit
(merkle-path-target-roots t rt len idx v []) s t
unfolding hash-map-new-output-hit-def
using lookup-t by simp blast
then show ?thesis by simp
next
case (Some old)
have lookup-t-old: fmlookup (HashMap t) (MerkleLeaf v) = Some old
by (rule hash-extension-lookup[OF Some ext])
then have old = rt
using lookup-t by simp
then have merkle-path-bound rt len idx v [] s
using Some by simp
then show ?thesis by simp
qed
next
case (Cons sibling path)
show ?case
proof (cases idx < len div 2)
case True
from Cons.premis obtain child where child-bound:
merkle-path-bound child (len div 2) idx v path t
and lookup-t:
fmlookup (HashMap t) (MerkleNode child sibling) = Some rt
using True by auto
have child-root:
merkle-path-root-in-map t (len div 2) idx v path = Some child
by (rule merkle-path-bound-root-in-map[OF child-bound])
show ?thesis
proof (cases fmlookup (HashMap s) (MerkleNode child sibling))
case None
then have root-hit:
hash-map-new-output-hit {rt} s t
unfolding hash-map-new-output-hit-def
using lookup-t by blast
have subset:
{rt} ⊆

```

```

    merkle-path-target-roots t rt len idx v (sibling # path)
  using True child-root by simp
have
  hash-map-new-output-hit
    (merkle-path-target-roots t rt len idx v (sibling # path)) s t
  by (rule hash-map-new-output-hit-subset[OF subset root-hit])
then show ?thesis by simp
next
case (Some old)
have lookup-t-old:
  fmllookup (HashMap t) (MerkleNode child sibling) = Some old
  by (rule hash-extension-lookup[OF Some ext])
then have old-eq: old = rt
  using lookup-t by simp
have lookup-s:
  fmllookup (HashMap s) (MerkleNode child sibling) = Some rt
  using Some old-eq by simp
from Cons.IH[OF child-bound]
have child-pullback:
  merkle-path-bound child (len div 2) idx v path s ∨
  hash-map-new-output-hit
    (merkle-path-target-roots t child (len div 2) idx v path) s t
  .
then show ?thesis
proof
  assume child-s:
    merkle-path-bound child (len div 2) idx v path s
  then have merkle-path-bound rt len idx v (sibling # path) s
    using True lookup-s by auto
  then show ?thesis by simp
next
  assume child-hit:
    hash-map-new-output-hit
      (merkle-path-target-roots t child (len div 2) idx v path) s t
  have subset:
    merkle-path-target-roots t child (len div 2) idx v path
    ⊆
    merkle-path-target-roots t rt len idx v (sibling # path)
  using True child-root by auto
  have
    hash-map-new-output-hit
      (merkle-path-target-roots t rt len idx v (sibling # path)) s t
    by (rule hash-map-new-output-hit-subset[OF subset child-hit])
  then show ?thesis by simp
qed
qed
next
case False
from Cons.premis obtain child where child-bound:

```

```

    merkle-path-bound child (len div 2) (idx - len div 2) v path t
and lookup-t:
    fmlookup (HashMap t) (MerkleNode sibling child) = Some rt
using False by auto
have child-root:
    merkle-path-root-in-map t (len div 2) (idx - len div 2) v path =
        Some child
    by (rule merkle-path-bound-root-in-map[OF child-bound])
show ?thesis
proof (cases fmlookup (HashMap s) (MerkleNode sibling child))
  case None
    then have root-hit:
      hash-map-new-output-hit {rt} s t
      unfolding hash-map-new-output-hit-def
      using lookup-t by blast
    have subset:
      {rt}  $\subseteq$ 
        merkle-path-target-roots t rt len idx v (sibling # path)
      using False child-root by simp
    have
      hash-map-new-output-hit
        (merkle-path-target-roots t rt len idx v (sibling # path)) s t
      by (rule hash-map-new-output-hit-subset[OF subset root-hit])
    then show ?thesis by simp
  next
    case (Some old)
    have lookup-t-old:
      fmlookup (HashMap t) (MerkleNode sibling child) = Some old
      by (rule hash-extension-lookup[OF Some ext])
    then have old-eq: old = rt
      using lookup-t by simp
    have lookup-s:
      fmlookup (HashMap s) (MerkleNode sibling child) = Some rt
      using Some old-eq by simp
    from Cons.IH[OF child-bound]
    have child-pullback:
      merkle-path-bound child (len div 2) (idx - len div 2) v path s  $\vee$ 
        hash-map-new-output-hit
          (merkle-path-target-roots t child (len div 2)
            (idx - len div 2) v path) s t
      .
    then show ?thesis
  proof
    assume child-s:
      merkle-path-bound child (len div 2) (idx - len div 2) v path s
    then have merkle-path-bound rt len idx v (sibling # path) s
      using False lookup-s by auto
    then show ?thesis by simp
  next

```

```

assume child-hit:
  hash-map-new-output-hit
    (merkle-path-target-roots t child (len div 2)
      (idx - len div 2) v path) s t
have subset:
  merkle-path-target-roots t child (len div 2)
    (idx - len div 2) v path
    ⊆
  merkle-path-target-roots t rt len idx v (sibling # path)
using False child-root by auto
have
  hash-map-new-output-hit
    (merkle-path-target-roots t rt len idx v (sibling # path)) s t
  by (rule hash-map-new-output-hit-subset[OF subset child-hit])
then show ?thesis by simp
qed
qed
qed
qed

```

lemma *authenticated-opening-pullback-or-new-output-hit*:

assumes *ext*: $s \leq t$

and *auth*: *authenticated-opening-in* *t* *opening*

shows

authenticated-opening-in *s* *opening* ∨
hash-map-new-output-hit
(*merkle-path-target-roots* *t*
(*opening-root* *opening*)
(*opening-length* *opening*)
(*opening-index* *opening*)
(*opening-value* *opening*)
(*opening-path* *opening*)) *s* *t*

proof –

have *idx-bound*: *opening-index* *opening* < *opening-length* *opening*

using *auth* **unfolding** *authenticated-opening-in-def* **by** *simp*

have *path-len*:

length (*opening-path* *opening*) =
floor-log (*opening-length* *opening*)

using *auth* **unfolding** *authenticated-opening-in-def* **by** *simp*

have *bound-t*:

merkle-path-bound
(*opening-root* *opening*)
(*opening-length* *opening*)
(*opening-index* *opening*)
(*opening-value* *opening*)
(*opening-path* *opening*) *t*

using *auth* **unfolding** *authenticated-opening-in-def* **by** *simp*

from *merkle-path-bound-pullback-or-new-output-hit*[*OF ext bound-t*]

show *?thesis*

```

proof
  assume bound-s:
    merkle-path-bound
      (opening-root opening)
      (opening-length opening)
      (opening-index opening)
      (opening-value opening)
      (opening-path opening) s
  have authenticated-opening-in s opening
    unfolding authenticated-opening-in-def
    using idx-bound path-len bound-s by simp
  then show ?thesis by simp
next
  assume
    hash-map-new-output-hit
      (merkle-path-target-roots t
        (opening-root opening)
        (opening-length opening)
        (opening-index opening)
        (opening-value opening)
        (opening-path opening)) s t
  then show ?thesis by simp
qed
qed

lemma partial-authenticated-table-opening-pullback-or-new-output-hit:
  assumes ext:  $s \leq t$ 
  and table: partial-authenticated-table rt len openings t
  and opn-in: opening  $\in$  set openings
  shows
    authenticated-opening-in s opening  $\vee$ 
    hash-map-new-output-hit
      (merkle-path-target-roots t rt len
        (opening-index opening)
        (opening-value opening)
        (opening-path opening)) s t
proof –
  have auth: authenticated-opening-in t opening
    using table opn-in unfolding partial-authenticated-table-def by blast
  have root-eq: opening-root opening = rt
    using table opn-in unfolding partial-authenticated-table-def by blast
  have len-eq: opening-length opening = len
    using table opn-in unfolding partial-authenticated-table-def by blast
  show ?thesis
    using authenticated-opening-pullback-or-new-output-hit[OF ext auth]
    unfolding root-eq len-eq .
qed

```

lemma *partial-authenticated-table-pullback-or-new-output-hit*:

```

assumes ext:  $s \leq t$ 
and table: partial-authenticated-table rt len openings t
shows
  partial-authenticated-table rt len openings s  $\vee$ 
  ( $\exists$  opening  $\in$  set openings.
    hash-map-new-output-hit
      (merkle-path-target-roots t rt len
        (opening-index opening)
        (opening-value opening)
        (opening-path opening))) s t)
proof (cases
   $\exists$  opening  $\in$  set openings.
    hash-map-new-output-hit
      (merkle-path-target-roots t rt len
        (opening-index opening)
        (opening-value opening)
        (opening-path opening))) s t)
  case True
  then show ?thesis by simp
next
  case False
  have table-s:
    partial-authenticated-table rt len openings s
    unfolding partial-authenticated-table-def
  proof
    fix opn
    assume opn-in: opn  $\in$  set openings
    have root-eq: opening-root opn = rt
      using table opn-in unfolding partial-authenticated-table-def by blast
    have len-eq: opening-length opn = len
      using table opn-in unfolding partial-authenticated-table-def by blast
    have auth-s: authenticated-opening-in s opn
  proof –
    have pullback:
      authenticated-opening-in s opn  $\vee$ 
      hash-map-new-output-hit
        (merkle-path-target-roots t rt len
          (opening-index opn)
          (opening-value opn)
          (opening-path opn))) s t
      by (rule partial-authenticated-table-opening-pullback-or-new-output-hit
        [OF ext table opn-in])
    then show ?thesis
      using False opn-in by blast
  qed
show
  opening-root opn = rt  $\wedge$ 
  opening-length opn = len  $\wedge$ 
  authenticated-opening-in s opn

```



```

    using root-eq len-eq auth-s by simp
  qed
  then show ?thesis by simp
  qed

lemma protocol-created-tree-get-authentication-path-len:
  assumes created: protocol-created-tree xs tree s
    and pos: 0 < lgth
    and le: lgth ≤ length xs
  shows length (get-authentication-path lgth idx tree) = floor-log lgth
  using assms
proof (induction xs arbitrary: tree lgth idx rule: measure-induct-rule[of length])
  case (less xs)
  show ?case
  proof (cases xs)
    case Nil
    then show ?thesis
      using less.prem1 by simp
  next
    case (Cons x ys)
    show ?thesis
    proof (cases ys)
      case Nil
      then have lgth-eq: lgth = 1
        using less.prem1 Cons by simp
      obtain h where tree-eq: tree = ⟨⟨⟩, h, ⟨⟩⟩
        using protocol-created-tree-singletonD[OF less.prem1][unfolded Cons Nil]
        by blast
      show ?thesis
        unfolding tree-eq lgth-eq by (simp add: floor-log-Suc-zero)
    next
      case (Cons y zs)
      let ?mid = length xs div 2
      have xs-eq: xs = x # y # zs
        using ⟨xs = x # ys⟩ ⟨ys = y # zs⟩ by simp
      have len-ge: 2 ≤ length xs
        unfolding xs-eq by simp
      obtain l h r i where i-eq: i = ?mid
        and tree-eq: tree = ⟨l, h, r⟩
        and left-i: protocol-created-tree (take i xs) l s
        and right-i: protocol-created-tree (drop i xs) r s
        and root-lookup:
          fmlookup (HashMap s) (MerkleNode (value l) (value r)) =
            Some h
        by (rule protocol-created-tree-internalD[OF less.prem1 len-ge])
      have left: protocol-created-tree (take ?mid xs) l s
        using left-i i-eq by simp
      have right: protocol-created-tree (drop ?mid xs) r s
        using right-i i-eq by simp
    end
  end
end

```

```

show ?thesis
proof (cases lgth = 1)
  case True
  then show ?thesis
    unfolding tree-eq by (simp add: floor-log-Suc-zero)
next
  case False
  note lgth-ne-one = False
  have lgth-ge:  $2 \leq \text{lgth}$ 
    using lgth-ne-one less.prem by simp
  have half-pos:  $0 < \text{lgth} \text{ div } 2$ 
    using lgth-ge by simp
  have take-less:  $\text{length} (\text{take } ?\text{mid } \text{xs}) < \text{length } \text{xs}$ 
    using len-ge by simp
  have drop-less:  $\text{length} (\text{drop } ?\text{mid } \text{xs}) < \text{length } \text{xs}$ 
    using len-ge by simp
  have half-le-take:  $\text{lgth} \text{ div } 2 \leq \text{length} (\text{take } ?\text{mid } \text{xs})$ 
    using less.prem(3) by simp
  have half-le-drop:  $\text{lgth} \text{ div } 2 \leq \text{length} (\text{drop } ?\text{mid } \text{xs})$ 
    using less.prem(3) len-ge by simp
  have left-len:
     $\text{length} (\text{get-authentication-path } (\text{lgth} \text{ div } 2) \text{ idx } l) =$ 
     $\text{floor-log } (\text{lgth} \text{ div } 2)$ 
    by (rule less.IH[OF take-less left half-pos half-le-take])
  have right-len:
     $\text{length} (\text{get-authentication-path } (\text{lgth} \text{ div } 2)$ 
     $(\text{idx} - \text{lgth} \text{ div } 2) \text{ r}) =$ 
     $\text{floor-log } (\text{lgth} \text{ div } 2)$ 
    by (rule less.IH[OF drop-less right half-pos half-le-drop])
  have floor-eq:  $\text{floor-log } \text{lgth} = \text{Suc } (\text{floor-log } (\text{lgth} \text{ div } 2))$ 
    using floor-log-rec[OF lgth-ge] .
  show ?thesis
  proof (cases idx < lgth div 2)
    case True
    then have  $\text{length} (\text{get-authentication-path } \text{lgth} \text{ idx } \text{tree}) =$ 
       $\text{Suc } (\text{floor-log } (\text{lgth} \text{ div } 2))$ 
      using lgth-ne-one left-len unfolding tree-eq by simp
    then show ?thesis
      by (simp only: floor-eq)
  next
    case False
    then have  $\text{length} (\text{get-authentication-path } \text{lgth} \text{ idx } \text{tree}) =$ 
       $\text{Suc } (\text{floor-log } (\text{lgth} \text{ div } 2))$ 
      using lgth-ne-one right-len unfolding tree-eq by simp
    then show ?thesis
      by (simp only: floor-eq)
  qed
qed
qed

```

qed
qed

lemma *created-tree-get-authentication-path-len*:
assumes *created*: *created-tree xs tree s*
and *xs-nonempty*: $xs \neq []$
shows $\text{length } (\text{get-authentication-path } (\text{length } xs) \text{ idx tree}) =$
 $\text{floor-log } (\text{length } xs)$
by (*rule protocol-created-tree-get-authentication-path-len*
 $[OF \text{ created}[\text{unfolded created-tree-def}]]$)
(use xs-nonempty in simp-all)

lemma *merkle-root-binds-table-authenticated-openings*:
assumes *bind*: *merkle-root-binds-table rt table final-state*
and *len-table*: $\text{length table} = \text{scale} * \text{clength}$
and *idxs-bound*: $\bigwedge \text{idx. idx} \in \text{set idxs} \implies \text{idx} < \text{scale} * \text{clength}$
obtains *openings* **where**
 $\text{map opening-index openings} = \text{idxs}$
 $\text{map opening-value openings} = \text{map } (! \text{ table}) \text{ idxs}$
 $\text{partial-authenticated-table rt } (\text{scale} * \text{clength}) \text{ openings final-state}$

proof –

obtain *tree* **where** *created*:
 $\text{created-tree table tree final-state}$
and *rt-eq*: $rt = \text{value tree}$
using *bind* **unfolding** *merkle-root-binds-table-def* **by** *blast*
let $?opening = \lambda \text{idx.}$
 $(\text{opening-root} = rt,$
 $\text{opening-length} = \text{scale} * \text{clength},$
 $\text{opening-index} = \text{idx},$
 $\text{opening-value} = \text{table } ! \text{ idx},$
 $\text{opening-path} = \text{get-authentication-path } (\text{length table}) \text{ idx tree})$
let $?openings = \text{map } ?opening \text{ idxs}$
have *auth-all*:
 $\forall \text{opn} \in \text{set } ?openings.$
 $\text{opening-root opn} = rt \wedge$
 $\text{opening-length opn} = \text{scale} * \text{clength} \wedge$
 $\text{authenticated-opening-in final-state opn}$

proof

fix *opn*
assume *opn-in*: $\text{opn} \in \text{set } ?openings$
then obtain *idx* **where** *idx-in*: $\text{idx} \in \text{set idxs}$
and *opn-eq*: $\text{opn} = ?opening \text{ idx}$
by *auto*
have *idx-bound-len*: $\text{idx} < \text{length table}$
using *idxs-bound* $[OF \text{ idx-in}]$ *len-table* **by** *simp*
have *idx-bound-domain*: $\text{idx} < \text{scale} * \text{clength}$
by (*rule idxs-bound* $[OF \text{ idx-in}]$)
obtain *n* **where** *len-pow*: $\text{length table} = 2^n$
using *len-table eval-domain-length-power*

```

    by (metis mult.commute)
  have root-in-map:
    merkle-path-root-in-map final-state (length table) idx
      (table ! idx) (get-authentication-path (length table) idx tree) =
      Some rt
  using protocol-created-tree-path-root-in-map
    [OF created[unfolded created-tree-def] len-pow idx-bound-len]
  unfolding rt-eq .
  have path-bound:
    merkle-path-bound rt (length table) idx (table ! idx)
      (get-authentication-path (length table) idx tree) final-state
  by (rule merkle-path-root-in-map-some-imp-bound[OF root-in-map])
  have path-len:
    length (get-authentication-path (length table) idx tree) =
      floor-log (scale * clength)
  proof -
    have table-nonempty: table ≠ []
      using idx-bound-len by auto
    show ?thesis
      using created-tree-get-authentication-path-len[OF created table-nonempty,
        of idx] len-table
      by simp
  qed
  have auth: authenticated-opening-in final-state opn
    unfolding authenticated-opening-in-def opn-eq
    using idx-bound-domain path-len path-bound len-table by simp
  show opening-root opn = rt ∧
    opening-length opn = scale * clength ∧
    authenticated-opening-in final-state opn
    using auth unfolding opn-eq by simp
  qed
  have partial-table:
    partial-authenticated-table rt (scale * clength) ?openings
      final-state
  using auth-all unfolding partial-authenticated-table-def by blast
  have indices: map opening-index ?openings = idxs
    by (induction idxs) simp-all
  have value-map: map opening-value ?openings = map (!! table) idxs
    by (induction idxs) simp-all
  show ?thesis
    by (rule that[OF indices value-map partial-table])
  qed

lemma authenticated-opening-root-in-map:
  assumes authenticated-opening-in s opening
  shows
    merkle-path-root-in-map s
      (opening-length opening)
      (opening-index opening)

```

```

      (opening-value opening)
      (opening-path opening) =
      Some (opening-root opening)
proof –
  have bound:
    merkle-path-bound
      (opening-root opening)
      (opening-length opening)
      (opening-index opening)
      (opening-value opening)
      (opening-path opening) s
    using assms unfolding authenticated-opening-in-def by simp
  show ?thesis
    by (rule merkle-path-bound-root-in-map[OF bound])
qed

```

```

lemma check-authentication-path-outcome-bound:
  fixes s t :: ('f, 'a) protocol-channel-scheme
  assumes outcome:
    Some (rt, t) ∈
      set-dist (execute (check-authentication-path len idx v path) s)
  shows merkle-path-bound rt len idx v path t
  using outcome
proof (induction path arbitrary: rt len idx s t)
  case Nil
  have hash-out:
    Some (rt, t) ∈
      set-dist
        (execute
          (hash (MerkleLeaf v) ::
            ('f, ('f, 'a) protocol-channel-scheme) state-monad) s)
  using Nil.premis
  unfolding check-authentication-path-def
    protocol-check-authentication-path-def
  by simp
  show ?case using hash-outcome(2)[OF hash-out] by simp
next
  case (Cons sibling path)
  show ?case
  proof (cases idx < len div 2)
  case True
  from Cons.premis obtain child u where
    rec-out:
      Some (child, u) ∈
        set-dist
          (execute
            (check-authentication-path (len div 2) idx v path) s)
  and hash-out:
    Some (rt, t) ∈

```

```

    set-dist
    (execute
      (hash (MerkleNode child sibling) ::
        ('f, ('f, 'a) protocol-channel-scheme) state-monad) u)
  unfolding check-authentication-path-def
    protocol-check-authentication-path-def
  using True
  by (auto simp: protocol-merkle.check-authentication-path.simps
    elim!: set-dist-bindE)
have child-u:
  merkle-path-bound child (len div 2) idx v path u
  by (rule Cons.IH[OF rec-out])
have ext:  $u \leq t$ 
  by (rule hash-outcome(1)[OF hash-out])
have child-t:
  merkle-path-bound child (len div 2) idx v path t
  by (rule merkle-path-bound-mono[OF child-u ext])
have lookup:
  fmlookup (HashMap t) (MerkleNode child sibling) = Some rt
  by (rule hash-outcome(2)[OF hash-out])
show ?thesis
  using True child-t lookup
  by (auto intro!: exI[of - child])
next
case False
from Cons.premis obtain child u where
  rec-out:
    Some (child, u)  $\in$ 
    set-dist
    (execute
      (check-authentication-path
        (len div 2) (idx - len div 2) v path) s)
  and hash-out:
    Some (rt, t)  $\in$ 
    set-dist
    (execute
      (hash (MerkleNode sibling child) ::
        ('f, ('f, 'a) protocol-channel-scheme) state-monad) u)
  unfolding check-authentication-path-def
    protocol-check-authentication-path-def
  using False
  by (auto simp: protocol-merkle.check-authentication-path.simps
    elim!: set-dist-bindE)
have child-u:
  merkle-path-bound child (len div 2) (idx - len div 2)
    v path u
  by (rule Cons.IH[OF rec-out])
have ext:  $u \leq t$ 
  by (rule hash-outcome(1)[OF hash-out])

```

```

have child-t:
  merkle-path-bound child (len div 2) (idx - len div 2)
    v path t
  by (rule merkle-path-bound-mono[OF child-u ext])
have lookup:
  fmlookup (HashMap t) (MerkleNode sibling child) = Some rt
  by (rule hash-outcome(2)[OF hash-out])
show ?thesis
  using False child-t lookup
  by (auto intro!: exI[of - child])
qed
qed

```

```

lemma query-decommitment-step-authenticated-opening:
fixes s t :: ('f, 'a) protocol-channel-scheme
assumes idx-bound: idx < scale * clength
and outcome:
  Some (qh, t) ∈
    set-dist (execute (query-decommitment-step rt idx) s)
obtains path opn where
  opn =
    (opening-root = rt,
     opening-length = scale * clength,
     opening-index = idx,
     opening-value = qh,
     opening-path = path)
  authenticated-opening-in t opn
proof -
let ?len = scale * clength
from outcome obtain s1 path s2 ap s3 s4 where
  read-gh:
    Some (qh, s1) ∈ set-dist (execute read s)
and read-path:
    Some (path, s2) ∈
      set-dist (execute (ntimes read (floor-log ?len)) s1)
and check:
    Some (ap, s3) ∈
      set-dist
        (execute (check-authentication-path ?len idx qh path) s2)
and assert-ap:
    Some ((), s4) ∈ set-dist (execute (assert (ap = rt)) s3)
and ret:
    Some (qh, t) ∈ set-dist (execute (return qh) s4)
unfolding query-decommitment-step-def
by (auto simp: Let-def elim!: set-dist-bindE)
have path-length: length path = floor-log ?len
  using ntimes-read-any-outcome[OF read-path] by simp
have ap-eq: ap = rt
  using assert-ap unfolding assert-def

```

```

    by (cases ap = rt) (auto simp: throw-no-outcome)
  have s4: s4 = s3
    using assert-ap ap-eq unfolding assert-def by simp
  have t: t = s3
    using ret unfolding s4 by simp
  have path-bound:
    merkle-path-bound rt ?len idx qh path t
  proof -
    have merkle-path-bound ap ?len idx qh path s3
      by (rule check-authentication-path-outcome-bound[OF check])
    then show ?thesis unfolding ap-eq t .
  qed
  let ?opn =
    (opening-root = rt,
     opening-length = ?len,
     opening-index = idx,
     opening-value = qh,
     opening-path = path)
  have authenticated: authenticated-opening-in t ?opn
    unfolding authenticated-opening-in-def
    using idx-bound path-length path-bound by simp
  show ?thesis
    by (rule that[of ?opn path]) (simp-all add: authenticated)
  qed

lemma mmap-query-decommitment-steps-authenticated-openings:
  fixes s t :: ('f, 'a) protocol-channel-scheme
  assumes idxs-bound:  $\bigwedge idx. idx \in \text{set } idxs \implies idx < \text{scale} * \text{clength}$ 
  and outcome:
    Some (leaves, t)  $\in$ 
      set-dist (execute (mmap (map (query-decommitment-step rt) idxs)) s)
  shows
     $\exists \text{ openings.}$ 
      length openings = length idxs  $\wedge$ 
      map opening-value openings = leaves  $\wedge$ 
      map opening-index openings = idxs  $\wedge$ 
      partial-authenticated-table rt (scale * clength) openings t
  using outcome idxs-bound
  proof (induction idxs arbitrary: s leaves t)
    case Nil
    then have leaves = [] and t = s
      by simp-all
    moreover have
      partial-authenticated-table rt (scale * clength) [] t
      unfolding partial-authenticated-table-def by simp
    ultimately show ?case
      by (intro exI[of - []]) simp
  next
    case (Cons idx idxs)

```



```

from Cons.prems obtain leaf leaves' s1 where
  head:
    Some (leaf, s1) ∈
      set-dist (execute (query-decommitment-step rt idx) s)
  and tail:
    Some (leaves', t) ∈
      set-dist (execute (mmap (map (query-decommitment-step rt) idxs)) s1)
  and leaves-eq: leaves = leaf # leaves'
  by (auto elim!: set-dist-bindE)
have idx-bound: idx < scale * clength
  by (rule Cons.prems(2)) simp
from query-decommitment-step-authenticated-opening[OF idx-bound head]
obtain path opn where
  opn-eq:
    opn =
      (opening-root = rt,
       opening-length = scale * clength,
       opening-index = idx,
       opening-value = leaf,
       opening-path = path)
  and opn-auth-s1: authenticated-opening-in s1 opn
  by blast
have tail-bound:
  ∧ idx. idx ∈ set idxs ⇒ idx < scale * clength
  by (rule Cons.prems(2)) simp
from Cons.IH[OF tail tail-bound] obtain openings where
  len-openings: length openings = length idxs
  and values-openings: map opening-value openings = leaves'
  and idx-openings: map opening-index openings = idxs
  and table-tail:
    partial-authenticated-table rt (scale * clength) openings t
  by blast
have s1-t: s1 ≤ t
  using mmap-query-decommitment-steps-outcome[OF tail] by blast
have opn-auth-t: authenticated-opening-in t opn
  by (rule authenticated-opening-in-mono[OF opn-auth-s1 s1-t])
let ?openings = opn # openings
have table-all:
  partial-authenticated-table rt (scale * clength) ?openings t
  using opn-auth-t table-tail opn-eq
  unfolding partial-authenticated-table-def by simp
show ?case
  using len-openings values-openings idx-openings leaves-eq opn-eq table-all
  by (intro exI[of - ?openings]) simp
qed

```

lemma check-decommit-on-query-authenticated-openings:

fixes s t :: ('f, 'a) protocol-channel-scheme

assumes idx-in: idx ∈ query-sample-space

and *outcome*:
 Some (*leaves*, *t*) ∈
 set-dist (execute (mmap (check-decommit-on-query *rt* *idx*)) *s*)
obtains *openings* **where**
 length *openings* = length (powers-scaled *idx*)
 map opening-value *openings* = *leaves*
 map opening-index *openings* = powers-scaled *idx*
 partial-authenticated-table *rt* (scale * clength) *openings* *t*
proof –
have map-eq:
 check-decommit-on-query *rt* *idx* =
 map (query-decommitment-step *rt*) (powers-scaled *idx*)
unfolding check-decommit-on-query-def query-decommitment-step-def **by** simp
have *idxs-bound*:
 ∧ *i*. *i* ∈ set (powers-scaled *idx*) ⇒ *i* < scale * clength
using query-sample-space-powers-scaled-bound[OF *idx-in*]
by (simp add: mult.commute)
from mmap-query-decommitment-steps-authenticated-openings
 [OF *idxs-bound* outcome[unfolded map-eq]]
obtain *openings* **where**
 length *openings* = length (powers-scaled *idx*)
 map opening-value *openings* = *leaves*
 map opening-index *openings* = powers-scaled *idx*
 partial-authenticated-table *rt* (scale * clength) *openings* *t*
by blast
then show ?thesis **by** (rule that)
qed

lemma fri-layer-opening-finish-authenticated-openings:

fixes *s t* :: ('f, 'a) protocol-channel-scheme
assumes *idx-bound*: *idx* < len
and *sibling-bound*: (*idx* + len div 2) mod len < len
and *xp-path-length*: length *xp-path* = floor-log len
and *xn-path-length*: length *xn-path* = floor-log len
and *outcome*:
 Some (*out*, *t*) ∈
 set-dist
 (execute
 (fri-layer-opening-finish *b* *rt* *idx* *x* len *pw*
xp *xp-path* *xn* *xn-path*) *s*)

obtains *xp-opening xn-opening* **where**

xp-opening =
 (|opening-root = *rt*,
 opening-length = len,
 opening-index = *idx*,
 opening-value = *xp*,
 opening-path = *xp-path*)
xn-opening =
 (|opening-root = *rt*,

```

    opening-length = len,
    opening-index = (idx + len div 2) mod len,
    opening-value = xn,
    opening-path = xn-path)
authenticated-opening-in t xp-opening
authenticated-opening-in t xn-opening
proof –
from outcome obtain s1 ap1 s2 s3 ap2 s4 s5 where
  assert-xp:
    Some ((), s1) ∈ set-dist (execute (assert (xp = x)) s)
  and check-xp:
    Some (ap1, s2) ∈
      set-dist
      (execute (check-authentication-path len idx xp xp-path) s1)
  and assert-ap1:
    Some ((), s3) ∈ set-dist (execute (assert (ap1 = rt)) s2)
  and check-xn:
    Some (ap2, s4) ∈
      set-dist
      (execute
        (check-authentication-path len
          ((idx + len div 2) mod len) xn xn-path) s3)
  and assert-ap2:
    Some ((), s5) ∈ set-dist (execute (assert (ap2 = rt)) s4)
  and ret:
    Some (out, t) ∈
      set-dist
      (execute
        (return
          (idx mod (len div 2),
            (xp + xn) div 2 +
              b * ((xp - xn) div
                (2 * ((h ^ idx) * shift) ^ pw)),
            len div 2, pw + pw)) s5)
  unfolding fri-layer-opening-finish-def
  by (auto simp: Let-def elim!: set-dist-bindE)
have ap1-eq: ap1 = rt
  using assert-ap1 unfolding assert-def
  by (cases ap1 = rt) (auto simp: throw-no-outcome)
have ap2-eq: ap2 = rt
  using assert-ap2 unfolding assert-def
  by (cases ap2 = rt) (auto simp: throw-no-outcome)
have xp-eq: xp = x
  using assert-xp unfolding assert-def
  by (cases xp = x) (auto simp: throw-no-outcome)
have s1: s1 = s
  using assert-xp xp-eq unfolding assert-def by simp
have s3: s3 = s2
  using assert-ap1 ap1-eq unfolding assert-def by simp

```

```

have s5: s5 = s4
  using assert-ap2 ap2-eq unfolding assert-def by simp
have t: t = s4
  using ret unfolding s5 by simp
have xp-bound-s2:
  merkle-path-bound rt len idx xp xp-path s2
proof -
  have merkle-path-bound ap1 len idx xp xp-path s2
    by (rule check-authentication-path-outcome-bound[OF check-xp])
  then show ?thesis unfolding ap1-eq .
qed
have s2-s4: s2 ≤ s4
  unfolding s3[symmetric]
  by (rule check-authentication-path-hash-extends[OF check-xn])
have xp-bound:
  merkle-path-bound rt len idx xp xp-path t
  unfolding t
  by (rule merkle-path-bound-mono[OF xp-bound-s2 s2-s4])
have xn-bound:
  merkle-path-bound rt len ((idx + len div 2) mod len)
  xn xn-path t
proof -
  have merkle-path-bound ap2 len ((idx + len div 2) mod len)
    xn xn-path s4
    by (rule check-authentication-path-outcome-bound[OF check-xn])
  then show ?thesis unfolding ap2-eq t .
qed
let ?xp-opening =
  (opening-root = rt,
   opening-length = len,
   opening-index = idx,
   opening-value = xp,
   opening-path = xp-path)
let ?xn-opening =
  (opening-root = rt,
   opening-length = len,
   opening-index = (idx + len div 2) mod len,
   opening-value = xn,
   opening-path = xn-path)
have xp-authenticated: authenticated-opening-in t ?xp-opening
  unfolding authenticated-opening-in-def
  using idx-bound xp-path-length xp-bound by simp
have xn-authenticated: authenticated-opening-in t ?xn-opening
  unfolding authenticated-opening-in-def
  using sibling-bound xn-path-length xn-bound by simp
show ?thesis
  by (rule that[of ?xp-opening ?xn-opening])
  (simp-all add: xp-authenticated xn-authenticated)
qed

```

lemma *fri-layer-opening-step-authenticated-openings*:

fixes $s\ t :: ('f, 'a)\ \text{protocol-channel-scheme}$

assumes $\text{len-pos}: 0 < \text{len}$

and $\text{idx-bound}: \text{idx} < \text{len}$

and outcome :

$\text{Some } (\text{out}, t) \in$

set-dist

$(\text{execute } (\text{fri-layer-opening-step } (b, rt) (\text{idx}, x, \text{len}, pw))\ s)$

obtains $\text{xp-opening } \text{xn-opening}$ **where**

$\text{opening-root } \text{xp-opening} = rt$

$\text{opening-length } \text{xp-opening} = \text{len}$

$\text{opening-index } \text{xp-opening} = \text{idx}$

$\text{opening-root } \text{xn-opening} = rt$

$\text{opening-length } \text{xn-opening} = \text{len}$

$\text{opening-index } \text{xn-opening} = (\text{idx} + \text{len} \text{ div } 2) \text{ mod } \text{len}$

$\text{authenticated-opening-in } t\ \text{xp-opening}$

$\text{authenticated-opening-in } t\ \text{xn-opening}$

proof —

from $\text{outcome}[\text{unfolded } \text{fri-layer-opening-step-unfold-finish}]$

obtain $\text{xp } s1\ \text{xp-path } s2\ \text{xn } s3\ \text{xn-path } s4$ **where**

$\text{read-xp}: \text{Some } (\text{xp}, s1) \in \text{set-dist } (\text{execute read } s)$

and read-xp-path :

$\text{Some } (\text{xp-path}, s2) \in$

$\text{set-dist } (\text{execute } (\text{ntimes read } (\text{floor-log len}))\ s1)$

and $\text{read-xn}: \text{Some } (\text{xn}, s3) \in \text{set-dist } (\text{execute read } s2)$

and read-xn-path :

$\text{Some } (\text{xn-path}, s4) \in$

$\text{set-dist } (\text{execute } (\text{ntimes read } (\text{floor-log len}))\ s3)$

and finish :

$\text{Some } (\text{out}, t) \in$

set-dist

$(\text{execute}$

$(\text{fri-layer-opening-finish } b\ rt\ \text{idx } x\ \text{len } pw$

$\text{xp } \text{xp-path } \text{xn } \text{xn-path})\ s4)$

$\text{by } (\text{auto elim!}: \text{set-dist-bindE})$

have $\text{xp-path-length}: \text{length } \text{xp-path} = \text{floor-log len}$

using $\text{ntimes-read-any-outcome}[\text{OF read-xp-path}]$ **by** simp

have $\text{xn-path-length}: \text{length } \text{xn-path} = \text{floor-log len}$

using $\text{ntimes-read-any-outcome}[\text{OF read-xn-path}]$ **by** simp

have $\text{sibling-bound}: (\text{idx} + \text{len} \text{ div } 2) \text{ mod } \text{len} < \text{len}$

using len-pos **by** simp

from $\text{fri-layer-opening-finish-authenticated-openings}$

$[\text{OF } \text{idx-bound } \text{sibling-bound } \text{xp-path-length } \text{xn-path-length } \text{finish}]$

obtain $\text{xp-opening } \text{xn-opening}$ **where**

$\text{xp-eq}:$

$\text{xp-opening} =$

$(\text{opening-root} = rt,$

$\text{opening-length} = \text{len},$

```

    opening-index = idx,
    opening-value = xp,
    opening-path = xp-path)
and xn-eq:
  xn-opening =
    (|opening-root = rt,
     opening-length = len,
     opening-index = (idx + len div 2) mod len,
     opening-value = xn,
     opening-path = xn-path)
and xp-auth: authenticated-opening-in t xp-opening
and xn-auth: authenticated-opening-in t xn-opening
by blast
show ?thesis
by (rule that[of xp-opening xn-opening])
   (use xp-eq xn-eq xp-auth xn-auth in simp-all)
qed

lemma fri-layer-opening-step-partial-authenticated-table:
  fixes s t :: ('f, 'a) protocol-channel-scheme
  assumes len-pos: 0 < len
  and idx-bound: idx < len
  and outcome:
    Some (out, t) ∈
      set-dist
        (execute (fri-layer-opening-step (b, rt) (idx, x, len, pw)) s)
  obtains openings where
    partial-authenticated-table rt len openings t
    map opening-index openings = [idx, (idx + len div 2) mod len]
    length openings = 2
proof -
  from fri-layer-opening-step-authenticated-openings
    [OF len-pos idx-bound outcome]
  obtain xp-opening xn-opening where
    xp-root: opening-root xp-opening = rt
    and xp-len: opening-length xp-opening = len
    and xp-idx: opening-index xp-opening = idx
    and xn-root: opening-root xn-opening = rt
    and xn-len: opening-length xn-opening = len
    and xn-idx:
      opening-index xn-opening = (idx + len div 2) mod len
    and xp-auth: authenticated-opening-in t xp-opening
    and xn-auth: authenticated-opening-in t xn-opening
  by blast
let ?openings = [xp-opening, xn-opening]
have table: partial-authenticated-table rt len ?openings t
  unfolding partial-authenticated-table-def
  using xp-root xp-len xp-auth xn-root xn-len xn-auth by simp
have indices:

```

```

    map opening-index ?openings = [idx, (idx + len div 2) mod len]
    using xp-idx xn-idx by simp
  show ?thesis
    by (rule that[OF table indices]) simp
qed

lemma verifier-query-round-after-index-authenticated-trace-openings:
  fixes s t :: ('f, 'a) protocol-channel-scheme
  assumes outcome:
    Some ((), t) ∈
      set-dist
        (execute
          (verifier-query-round-after-index-program
            fr f-fl f-final as fl final raw) s)
  obtains idx leaves openings where
    idx = index (to-nat raw)
    length openings = length (powers-scaled idx)
    map opening-value openings = leaves
    map opening-index openings = powers-scaled idx
    partial-authenticated-table fr (scale * clength) openings t
proof -
  let ?idx = index (to-nat raw)
  from outcome obtain fv s1 f-out s2 s3 c-out s4 where
    query-decommit:
      Some (fv, s1) ∈
        set-dist (execute (mmap (check-decommit-on-query fr ?idx)) s)
  and trace-fri:
    Some (f-out, s2) ∈
      set-dist (execute
        (mfold (?idx, hd fv, clength * scale, 1)
          (receive-query-commits f-fl)) s1)
  and assert-trace:
    Some ((), s3) ∈
      set-dist
        (execute
          (assert (case f-out of (-, f-x, -, -) ⇒ f-x = f-final)) s2)
  and comp-fri:
    Some (c-out, s4) ∈
      set-dist (execute
        (mfold (?idx, cp-eval as fv (h ^ ?idx * shift),
          clength * scale, 1)
          (receive-query-commits fl)) s3)
  and assert-comp:
    Some ((), t) ∈
      set-dist
        (execute
          (assert (case c-out of (-, x, -, -) ⇒ x = final)) s4)
  unfolding verifier-query-round-after-index-program-def
  by (auto simp: Let-def split: prod.splits elim!: set-dist-bindE)

```

```

have idx-in: ?idx ∈ query-sample-space
  using index-less-query-sample-space
  unfolding query-sample-space-def by simp
from check-decommit-on-query-authenticated-openings
  [OF idx-in query-decommit]
obtain openings where
  len-openings: length openings = length (powers-scaled ?idx)
  and values-openings: map opening-value openings = fv
  and idx-openings: map opening-index openings = powers-scaled ?idx
  and table-s1:
    partial-authenticated-table fr (scale * clength) openings s1
  by blast
from receive-query-commits-layers-outcome[OF trace-fri]
obtain trace-layer-chunks trace-fri-chunk where
  s1-s2: s1 ≤ s2
  by blast
have s3-eq: s3 = s2
  using assert-trace unfolding assert-def
  by (cases case f-out of (-, f-x, -, -) ⇒ f-x = f-final)
    (auto simp: throw-no-outcome)
from receive-query-commits-layers-outcome[OF comp-fri]
obtain comp-layer-chunks comp-fri-chunk where
  s3-s4: s3 ≤ s4
  by blast
have t-eq: t = s4
  using assert-comp unfolding assert-def
  by (cases case c-out of (-, x, -, -) ⇒ x = final)
    (auto simp: throw-no-outcome)
have s1-t: s1 ≤ t
  using s1-s2 s3-s4 unfolding s3-eq t-eq
  by (meson hash-ext-trans)
have table-t:
  partial-authenticated-table fr (scale * clength) openings t
  by (rule partial-authenticated-table-mono[OF table-s1 s1-t])
show ?thesis
  by (rule that[OF refl len-openings values-openings idx-openings table-t])
qed

```

```

lemma verifier-query-round-after-index-authenticated-composition-openings:
fixes s t :: ('f, 'a) protocol-channel-scheme
assumes fl-eq: fl = (b, composition-root) # fl-tail
and outcome:
  Some ((), t) ∈
    set-dist
      (execute
        (verifier-query-round-after-index-program
          fr f-fl f-final as fl final raw) s)
obtains idx openings where
  idx = index (to-nat raw)

```



```

    partial-authenticated-table composition-root (scale * clength)
    openings t
    map opening-index openings =
      [idx, fri-sibling-index (scale * clength) idx]
    length openings = 2
  proof -
    let ?idx = index (to-nat raw)
    from outcome obtain fv s1 f-out s2 s3 c-out s4 where
      query-decommit:
        Some (fv, s1) ∈
          set-dist (execute (mmap (check-decommit-on-query fr ?idx)) s)
      and trace-fri:
        Some (f-out, s2) ∈
          set-dist (execute
            (mfold (?idx, hd fv, clength * scale, 1)
              (receive-query-commits f-fl)) s1)
      and assert-trace:
        Some ((), s3) ∈
          set-dist
            (execute
              (assert (case f-out of (-, f-x, -, -) ⇒ f-x = f-final)) s2)
      and comp-fri:
        Some (c-out, s4) ∈
          set-dist (execute
            (mfold (?idx, cp-eval as fv (h ^ ?idx * shift),
              clength * scale, 1)
              (receive-query-commits fl)) s3)
      and assert-comp:
        Some ((), t) ∈
          set-dist
            (execute
              (assert (case c-out of (-, x, -, -) ⇒ x = final)) s4)
    unfolding verifier-query-round-after-index-program-def
    by (auto simp: Let-def split: prod.splits elim!: set-dist-bindE)
  have comp-steps:
    receive-query-commits fl =
      fri-layer-opening-step (b, composition-root) #
      receive-query-commits fl-tail
    unfolding fl-eq receive-query-commits-def fri-layer-opening-step-def
    by simp
  from comp-fri[unfolded comp-steps]
  obtain out1 s-head where
    head:
      Some (out1, s-head) ∈
        set-dist
          (execute
            (fri-layer-opening-step (b, composition-root)
              (?idx, cp-eval as fv (h ^ ?idx * shift),
                clength * scale, 1)) s3)

```

```

and tail:
  Some (c-out, s4) ∈
    set-dist (execute (mfold out1 (receive-query-commits fl-tail))
      s-head)
  by (auto elim!: set-dist-bindE)
have len-pos:  $0 < \text{clength} * \text{scale}$ 
  using eval-domain-nontrivial by linarith
have idx-bound:  $?idx < \text{clength} * \text{scale}$ 
  by (rule index-less-domain)
from fri-layer-opening-step-partial-authenticated-table
  [OF len-pos idx-bound head]
obtain openings where
  table-head:
    partial-authenticated-table composition-root ( $\text{clength} * \text{scale}$ )
    openings s-head
and indices:
  map opening-index openings =
    [ $?idx, (?idx + (\text{clength} * \text{scale}) \text{div } 2) \bmod (\text{clength} * \text{scale})$ ]
and len-openings:  $\text{length openings} = 2$ 
by (rule fri-layer-opening-step-partial-authenticated-table
  [OF len-pos idx-bound head])
obtain i1 x1 len1 pw1 where out1-eq: out1 = (i1, x1, len1, pw1)
  by (cases out1)
from receive-query-commits-layers-outcome[OF tail[unfolded out1-eq]]
obtain tail-layer-chunks tail-chunk where s-head-s4:  $s\text{-head} \leq s4$ 
  by blast
have t-eq:  $t = s4$ 
  using assert-comp unfolding assert-def
  by (cases case c-out of ( $-, x, -, -$ )  $\Rightarrow x = \text{final}$ )
    (auto simp: throw-no-outcome)
have table-t:
  partial-authenticated-table composition-root ( $\text{scale} * \text{clength}$ )
  openings t
  using partial-authenticated-table-mono[OF table-head s-head-s4]
  unfolding t-eq by (simp add: mult.commute)
have indices':
  map openings-index openings =
    [ $?idx, \text{fri-sibling-index} (\text{scale} * \text{clength}) ?idx$ ]
  using indices
  unfolding fri-sibling-index-def
  by (simp add: mult.commute)
show ?thesis
  by (rule that[OF refl table-t indices' len-openings])
qed

```

```

lemma verifier-query-round-program-authenticated-trace-openings:
  fixes  $s\ t :: ('f, 'a) \text{protocol-channel-scheme}$ 
  assumes outcome:
    Some ( $()$ , t) ∈

```

```

    set-dist
      (execute
        (verifier-query-round-program fr f-fl f-final as fl final) s)
obtains raw idx leaves openings where
  idx = index (to-nat raw)
  length openings = length (powers-scaled idx)
  map opening-value openings = leaves
  map opening-index openings = powers-scaled idx
  partial-authenticated-table fr (scale * clength) openings t
proof –
from outcome[unfolded verifier-query-round-program-alt-def]
obtain raw s0 where
  rand:
    Some (raw, s0) ∈
      set-dist (execute receive-query-index-challenge s)
and tail:
  Some ((), t) ∈
    set-dist
      (execute
        (verifier-query-round-after-index-program
          fr f-fl f-final as fl final raw) s0)
by (auto elim!: set-dist-bindE)
from verifier-query-round-after-index-authenticated-trace-openings[OF tail]
obtain idx leaves openings where
  idx-eq: idx = index (to-nat raw)
and len-openings: length openings = length (powers-scaled idx)
and values-openings: map opening-value openings = leaves
and idx-openings: map opening-index openings = powers-scaled idx
and table-t:
  partial-authenticated-table fr (scale * clength) openings t
by blast
show ?thesis
by (rule that[OF idx-eq len-openings values-openings idx-openings table-t])
qed

```

lemma verifier-query-round-program-authenticated-composition-openings:

```

fixes s t :: ('f, 'a) protocol-channel-scheme
assumes fl-eq: fl = (b, composition-root) # fl-tail
and outcome:
  Some ((), t) ∈
    set-dist
      (execute
        (verifier-query-round-program fr f-fl f-final as fl final) s)
obtains raw idx openings where
  idx = index (to-nat raw)
  partial-authenticated-table composition-root (scale * clength)
    openings t
  map opening-index openings =
    [idx, fri-sibling-index (scale * clength) idx]

```

```

    length openings = 2
proof –
from outcome[unfolded verifier-query-round-program-alt-def]
obtain raw s0 where
  rand:
    Some (raw, s0) ∈
      set-dist (execute receive-query-index-challenge s)
and tail:
    Some ((), t) ∈
      set-dist
        (execute
          (verifier-query-round-after-index-program
            fr f-fl f-final as fl final raw) s0)
by (auto elim!: set-dist-bindE)
from verifier-query-round-after-index-authenticated-composition-openings
  [OF fl-eq tail]
obtain idx openings where
  idx-eq: idx = index (to-nat raw)
and table-t:
    partial-authenticated-table composition-root (scale * clength)
      openings t
and idx-openings:
    map opening-index openings =
      [idx, fri-sibling-index (scale * clength) idx]
and len-openings: length openings = 2
by blast
show ?thesis
by (rule that[OF idx-eq table-t idx-openings len-openings])
qed

```

lemma *ntimes-verifier-query-rounds-authenticated-trace-openings:*

fixes s t :: ('f, 'a) protocol-channel-scheme

assumes outcome:

```

  Some (results, t) ∈
    set-dist
      (execute
        (ntimes
          (verifier-query-round-program fr f-fl f-final as fl final) n)
        s)

```

obtains raw-idxs query-idxs trace-openings **where**

```

  length raw-idxs = n
  query-idxs = map (λraw. index (to-nat raw)) raw-idxs
  length trace-openings = n
  ∧ i. i < n ⇒
    map opening-index (trace-openings ! i) =
      powers-scaled (query-idxs ! i)
  ∧ i. i < n ⇒
    partial-authenticated-table fr (scale * clength)
      (trace-openings ! i) t

```

```

proof –
  have
     $\exists \text{raw-idxs } \text{query-idxs } \text{trace-openings}.$ 
     $\text{length } \text{raw-idxs} = n \wedge$ 
     $\text{query-idxs} = \text{map } (\lambda \text{raw}. \text{index } (\text{to-nat } \text{raw})) \text{ raw-idxs} \wedge$ 
     $\text{length } \text{trace-openings} = n \wedge$ 
     $(\forall i < n.$ 
       $\text{map } \text{opening-index } (\text{trace-openings } ! i) =$ 
       $\text{powers-scaled } (\text{query-idxs } ! i)) \wedge$ 
     $(\forall i < n.$ 
       $\text{partial-authenticated-table } \text{fr } (\text{scale} * \text{clength})$ 
       $(\text{trace-openings } ! i) \text{ } t)$ 
    using outcome
  proof (induction n arbitrary: s results t)
    case 0
    then show ?case
      by (intro exI[of - []] simp)
    next
    case (Suc n)
    from Suc.prems obtain u results' s1 where
      head:
         $\text{Some } (u, s1) \in$ 
         $\text{set-dist}$ 
         $(\text{execute}$ 
           $(\text{verifier-query-round-program } \text{fr } f\text{-fl } f\text{-final } \text{as } \text{fl } \text{final}) \text{ } s)$ 
        and tail:
           $\text{Some } (\text{results}', t) \in$ 
           $\text{set-dist}$ 
           $(\text{execute}$ 
             $(\text{ntimes}$ 
               $(\text{verifier-query-round-program } \text{fr } f\text{-fl } f\text{-final } \text{as } \text{fl } \text{final})$ 
               $n) \text{ } s1)$ 
          and results-eq:  $\text{results} = u \# \text{results}'$ 
          by (auto elim!: set-dist-bindE)
    have u-eq:  $u = ()$ 
    by (cases u simp)
    from verifier-query-round-program-authenticated-trace-openings
      [OF head[unfolded u-eq]]
    obtain raw idx leaves openings where
      idx-eq:  $\text{idx} = \text{index } (\text{to-nat } \text{raw})$ 
      and idx-openings:  $\text{map } \text{opening-index } \text{openings} = \text{powers-scaled } \text{idx}$ 
      and table-s1:
         $\text{partial-authenticated-table } \text{fr } (\text{scale} * \text{clength}) \text{ openings } s1$ 
      by blast
    from Suc.IH[OF tail] obtain raw-tail idx-tail openings-tail where
      len-raw-tail:  $\text{length } \text{raw-tail} = n$ 
      and idx-tail-eq:
         $\text{idx-tail} = \text{map } (\lambda \text{raw}. \text{index } (\text{to-nat } \text{raw})) \text{ raw-tail}$ 
      and len-openings-tail:  $\text{length } \text{openings-tail} = n$ 

```

```

and tail-indices:
   $\forall i < n.$ 
     $\text{map } \text{opening-index } (\text{openings-tail} ! i) =$ 
       $\text{powers-scaled } (\text{idx-tail} ! i)$ 
and tail-tables:
   $\forall i < n.$ 
     $\text{partial-authenticated-table fr } (\text{scale} * \text{clength})$ 
       $(\text{openings-tail} ! i) \ t$ 
by blast
have s1-t:  $s1 \leq t$ 
  using ntimes-verifier-query-rounds-outcome[OF tail] by blast
have table-t:
   $\text{partial-authenticated-table fr } (\text{scale} * \text{clength}) \ \text{openings } t$ 
  by (rule partial-authenticated-table-mono[OF table-s1 s1-t])
let ?rows = raw # raw-tail
let ?idxs = idx # idx-tail
let ?openings = openings # openings-tail
have idxs-eq:
   $?idxs = \text{map } (\lambda \text{raw}. \text{index } (\text{to-nat } \text{raw})) \ ?rows$ 
  using idx-eq idx-tail-eq by simp
have indices-all:
   $\forall i < \text{Suc } n.$ 
     $\text{map } \text{opening-index } (?openings ! i) =$ 
       $\text{powers-scaled } (?idxs ! i)$ 
proof (intro allI impI)
  fix i
  assume i-bound:  $i < \text{Suc } n$ 
  show  $\text{map } \text{opening-index } (?openings ! i) =$ 
     $\text{powers-scaled } (?idxs ! i)$ 
  proof (cases i)
    case 0
    then show ?thesis
    using idx-openings by simp
  next
    case (Suc j)
    then show ?thesis
    using i-bound tail-indices by simp
  qed
qed
have tables-all:
   $\forall i < \text{Suc } n.$ 
     $\text{partial-authenticated-table fr } (\text{scale} * \text{clength})$ 
       $(?openings ! i) \ t$ 
proof (intro allI impI)
  fix i
  assume i-bound:  $i < \text{Suc } n$ 
  show  $\text{partial-authenticated-table fr } (\text{scale} * \text{clength})$ 
     $(?openings ! i) \ t$ 
  proof (cases i)

```

```

    case 0
    then show ?thesis
      using table-t by simp
  next
    case (Suc j)
    then show ?thesis
      using i-bound tail-tables by simp
  qed
qed
show ?case
  apply (intro exI[of - ?raws] exI[of - ?idxs] exI[of - ?openings]
    conjI)
    apply (simp add: len-row-tail)
    apply (rule idxs-eq)
    apply (simp add: len-openings-tail)
    apply (rule indices-all)
    apply (rule tables-all)
  done
qed
then obtain raw-idxs query-idxs trace-openings where
  length raw-idxs = n
  query-idxs = map ( $\lambda$ raw. index (to-nat raw)) raw-idxs
  length trace-openings = n
 $\bigwedge i. i < n \implies$ 
  map opening-index (trace-openings ! i) =
    powers-scaled (query-idxs ! i)
 $\bigwedge i. i < n \implies$ 
  partial-authenticated-table fr (scale * clength)
    (trace-openings ! i) t
  by blast
then show ?thesis by (rule that)
qed

lemma ntimes-verifier-query-rounds-authenticated-composition-openings:
  fixes s t :: ('f, 'a) protocol-channel-scheme
  assumes fl-eq: fl = (b, composition-root) # fl-tail
  and outcome:
    Some (results, t)  $\in$ 
      set-dist
        (execute
          (ntimes
            (verifier-query-round-program fr f-fl f-final as fl final) n)
          s)
  obtains raw-idxs query-idxs composition-openings where
    length raw-idxs = n
    query-idxs = map ( $\lambda$ raw. index (to-nat raw)) raw-idxs
    length composition-openings = n
 $\bigwedge i. i < n \implies$ 
  map opening-index (composition-openings ! i) =

```

```

    [query-idxs ! i,
     fri-sibling-index (scale * clength) (query-idxs ! i)]
  ∧ i. i < n ⇒
    partial-authenticated-table composition-root (scale * clength)
      (composition-openings ! i) t
proof -
have
  ∃ raw-idxs query-idxs composition-openings.
    length raw-idxs = n ∧
    query-idxs = map (λ raw. index (to-nat raw)) raw-idxs ∧
    length composition-openings = n ∧
    (∀ i < n.
      map opening-index (composition-openings ! i) =
        [query-idxs ! i,
         fri-sibling-index (scale * clength) (query-idxs ! i)]) ∧
    (∀ i < n.
      partial-authenticated-table composition-root (scale * clength)
        (composition-openings ! i) t)
using outcome
proof (induction n arbitrary: s results t)
case 0
then show ?case
  by (intro exI[of - []]) simp
next
case (Suc n)
from Suc.premis obtain u results' s1 where
  head:
    Some (u, s1) ∈
      set-dist
        (execute
          (verifier-query-round-program fr f-fl f-final as fl final) s)
  and tail:
    Some (results', t) ∈
      set-dist
        (execute
          (ntimes
            (verifier-query-round-program fr f-fl f-final as fl final)
            n) s1)
  by (auto elim!: set-dist-bindE)
have u-eq: u = ()
  by (cases u) simp
from verifier-query-round-program-authenticated-composition-openings
  [OF fl-eq head[unfolded u-eq]]
obtain raw idx openings where
  idx-eq: idx = index (to-nat raw)
  and table-s1:
    partial-authenticated-table composition-root (scale * clength)
      openings s1
  and idx-openings:

```



```

    map opening-index openings =
      [idx, fri-sibling-index (scale * clength) idx]
  and len-openings: length openings = 2
  by (rule verifier-query-round-program-authenticated-composition-openings
      [OF fl-eq head[unfolded u-eq]])
from Suc.IH[OF tail] obtain raw-tail idx-tail openings-tail where
  len-raw-tail: length raw-tail = n
  and idx-tail-eq:
    idx-tail = map (λraw. index (to-nat raw)) raw-tail
  and len-openings-tail: length openings-tail = n
  and tail-indices:
    ∀ i < n.
      map opening-index (openings-tail ! i) =
        [idx-tail ! i,
         fri-sibling-index (scale * clength) (idx-tail ! i)]
  and tail-tables:
    ∀ i < n.
      partial-authenticated-table composition-root (scale * clength)
        (openings-tail ! i) t
  by blast
have s1-t: s1 ≤ t
  using ntimes-verifier-query-rounds-outcome[OF tail] by blast
have table-t:
  partial-authenticated-table composition-root (scale * clength)
    openings t
  by (rule partial-authenticated-table-mono[OF table-s1 s1-t])
let ?rows = raw # raw-tail
let ?idxs = idx # idx-tail
let ?openings = openings # openings-tail
have idxs-eq:
  ?idxs = map (λraw. index (to-nat raw)) ?rows
  using idx-eq idx-tail-eq by simp
have indices-all:
  ∀ i < Suc n.
    map opening-index (?openings ! i) =
      [?idxs ! i, fri-sibling-index (scale * clength) (?idxs ! i)]
proof (intro allI impI)
  fix i
  assume i-bound: i < Suc n
  show map opening-index (?openings ! i) =
    [?idxs ! i, fri-sibling-index (scale * clength) (?idxs ! i)]
  proof (cases i)
    case 0
    then show ?thesis
      using idx-openings by simp
  next
    case (Suc j)
    then show ?thesis
      using i-bound tail-indices by simp
  end
end

```

```

qed
qed
have tables-all:
   $\forall i < \text{Suc } n.$ 
    partial-authenticated-table composition-root (scale * clength)
      (?openings ! i) t
proof (intro allI impI)
  fix i
  assume i-bound:  $i < \text{Suc } n$ 
  show partial-authenticated-table composition-root (scale * clength)
    (?openings ! i) t
  proof (cases i)
    case 0
    then show ?thesis
      using table-t by simp
  next
    case (Suc j)
    then show ?thesis
      using i-bound tail-tables by simp
  qed
qed
show ?case
  apply (intro exI[of - ?rows] exI[of - ?idxs]
    exI[of - ?openings] conjI)
    apply (simp add: len-row-tail)
    apply (rule idxs-eq)
    apply (simp add: len-openings-tail)
    apply (rule indices-all)
    apply (rule tables-all)
  done
qed
then obtain raw-idxs query-idxs composition-openings where
  len-row: length raw-idxs = n
  and idxs-eq: query-idxs = map ( $\lambda \text{raw}. \text{index } (\text{to-nat } \text{raw})$ ) raw-idxs
  and len-openings: length composition-openings = n
  and indices:
     $\forall i < n.$ 
      map opening-index (composition-openings ! i) =
        [query-idxs ! i,
          fri-sibling-index (scale * clength) (query-idxs ! i)]
  and tables:
     $\forall i < n.$ 
      partial-authenticated-table composition-root (scale * clength)
        (composition-openings ! i) t
  by (elim exE conjE, blast)
show ?thesis
  by (rule that[OF len-row idxs-eq len-openings])
    (use indices tables in auto)
qed

```

lemma *ntimes-verify-verify-query-rounds-authenticated-trace-openings-no-raws:*

fixes $s\ t :: ('f, 'a)\ \text{protocol-channel-scheme}$

assumes *outcome:*

$\text{Some } (\text{results}, t) \in$

set-dist

$(\text{execute}$

$(\text{ntimes}$

$(\text{verifier-query-round-program } \text{fr } f\text{-fl } f\text{-final } \text{as } \text{fl } \text{final})\ n)$

$s)$

obtains *query-idxs trace-openings* **where**

$\text{length } \text{query-idxs} = n$

$\text{length } \text{trace-openings} = n$

$\bigwedge i. i < n \implies$

$\text{map } \text{opening-index } (\text{trace-openings } !\ i) =$

$\text{powers-scaled } (\text{query-idxs } !\ i)$

$\bigwedge i. i < n \implies$

$\text{partial-authenticated-table } \text{fr } (\text{scale} * \text{clength})$

$(\text{trace-openings } !\ i)\ t$

proof (*rule ntimes-verify-verify-query-rounds-authenticated-trace-openings*
[OF *outcome*])

fix *raw-idxs query-idxs trace-openings*

assume *len-raw:* $\text{length } \text{raw-idxs} = n$

and *idxs-eq:* $\text{query-idxs} = \text{map } (\lambda \text{raw}. \text{index } (\text{to-nat } \text{raw}))\ \text{raw-idxs}$

and *len-openings:* $\text{length } \text{trace-openings} = n$

and *indices:*

$\bigwedge i. i < n \implies$

$\text{map } \text{opening-index } (\text{trace-openings } !\ i) =$

$\text{powers-scaled } (\text{query-idxs } !\ i)$

and *tables:*

$\bigwedge i. i < n \implies$

$\text{partial-authenticated-table } \text{fr } (\text{scale} * \text{clength})$

$(\text{trace-openings } !\ i)\ t$

have *len-query-idxs:* $\text{length } \text{query-idxs} = n$

using *len-raw idxs-eq* **by** *simp*

show *?thesis*

by (*rule that*[OF *len-query-idxs len-openings indices tables*])

qed

lemma *verify-monad-authenticated-trace-openings:*

fixes $s :: ('f, 'a)\ \text{protocol-channel-scheme}$

assumes *outcome:*

$\text{Some } (\text{result}, \text{final-state}) \in \text{set-dist } (\text{execute } \text{verify-monad } s)$

obtains *fr query-idxs trace-openings* **where**

$\text{length } \text{query-idxs} = \text{rounds}$

$\text{length } \text{trace-openings} = \text{rounds}$

$\bigwedge i. i < \text{rounds} \implies$

$\text{map } \text{opening-index } (\text{trace-openings } !\ i) =$

$\text{powers-scaled } (\text{query-idxs } !\ i)$

$\bigwedge i. i < \text{rounds} \implies$
 $\text{partial-authenticated-table fr (scale * clength)}$
 $(\text{trace-openings ! } i) \text{ final-state}$
proof –
from *verify-monad-header-extraction*[*OF outcome*]
obtain *fr f-fl f-final as dg fl final query-state* **where**
query-out:
 $\text{Some (result, final-state)} \in$
 set-dist (execute
 $(\text{ntimes}$
 $(\text{verifier-query-round-program fr f-fl f-final as fl final})$
 $\text{rounds})$
 $\text{query-state})$
by *metis*
from *ntimes-verifier-query-rounds-authenticated-trace-openings*
 $[\text{OF query-out}]$
obtain *raw-idxs query-idxs trace-openings* **where**
 $\text{len-raw: length raw-idxs = rounds}$
and *query-idxs-def*:
 $\text{query-idxs = map } (\lambda \text{raw. index (to-nat raw)}) \text{ raw-idxs}$
and *len-openings*: $\text{length trace-openings = rounds}$
and *indices*:
 $\bigwedge i. i < \text{rounds} \implies$
 $\text{map opening-index (trace-openings ! } i) =$
 $\text{powers-scaled (query-idxs ! } i)$
and *tables*:
 $\bigwedge i. i < \text{rounds} \implies$
 $\text{partial-authenticated-table fr (scale * clength)}$
 $(\text{trace-openings ! } i) \text{ final-state}$
by *blast*
have *len-query-idxs*: $\text{length query-idxs = rounds}$
using *len-raw query-idxs-def* **by** *simp*
show *?thesis*
by $(\text{rule that}[\text{OF len-query-idxs len-openings indices tables}])$
qed

definition *accepted-with-partial-trace-openings*

$:: (\text{'f, 'a}) \text{ protocol-channel-scheme} \Rightarrow$
 $(\text{unit list} \times (\text{'f, 'a}) \text{ protocol-channel-scheme}) \text{ option} \Rightarrow$
 $\text{'f} \Rightarrow \text{nat list} \Rightarrow \text{'f authenticated-opening list list} \Rightarrow \text{bool}$

where

$\text{accepted-with-partial-trace-openings } s \text{ out fr query-idxs trace-openings} \longleftrightarrow$
 $\text{accepted out} \wedge$
 $(\exists \text{result final-state.}$
 $\text{out = Some (result, final-state)} \wedge$
 $\text{length query-idxs = rounds} \wedge$
 $\text{length trace-openings = rounds} \wedge$
 $(\forall i < \text{rounds.}$
 $\text{map opening-index (trace-openings ! } i) =$

$\text{powers-scaled } (\text{query-idxs } ! i) \wedge$
 $(\forall i < \text{rounds}.$
 $\text{partial-authenticated-table fr } (\text{scale} * \text{clength})$
 $(\text{trace-openings } ! i) \text{ final-state}))$

lemma *accepted-with-partial-trace-openings-imp-accepted:*
assumes *accepted-with-partial-trace-openings s out fr query-idxs trace-openings*
shows *accepted out*
using *assms unfolding accepted-with-partial-trace-openings-def by simp*

lemma *accepted-with-partial-trace-openings-shapes:*
assumes *accepted-with-partial-trace-openings s out fr query-idxs trace-openings*
shows *length query-idxs = rounds*
and *length trace-openings = rounds*
and $\bigwedge i. i < \text{rounds} \implies$
 $\text{map opening-index } (\text{trace-openings } ! i) =$
 $\text{powers-scaled } (\text{query-idxs } ! i)$
using *assms unfolding accepted-with-partial-trace-openings-def by blast+*

definition *table-agrees-with-authenticated-openings*
 $:: 'f \text{ list} \Rightarrow \text{nat} \Rightarrow 'f \text{ authenticated-opening list} \Rightarrow \text{bool}$
where
 $\text{table-agrees-with-authenticated-openings table len openings} \longleftrightarrow$
 $\text{length table} = \text{len} \wedge$
 $(\forall \text{opening} \in \text{set openings}.$
 $\text{opening-index opening} < \text{len} \wedge$
 $\text{table } ! \text{opening-index opening} = \text{opening-value opening})$

lemma *table-agrees-with-authenticated-openings-subset:*
assumes *agrees:*
 $\text{table-agrees-with-authenticated-openings table len openings}$
and *subset: set openings' $\subseteq$ set openings*
shows *table-agrees-with-authenticated-openings table len openings'*
using *assms*
unfolding *table-agrees-with-authenticated-openings-def by blast*

lemma *table-agrees-with-authenticated-openings-consistent-exists:*
assumes *idx-bound:*
 $\bigwedge \text{opn}. \text{opn} \in \text{set openings} \implies \text{opening-index opn} < \text{len}$
and *consistent:*
 $\bigwedge \text{opn opn'}. \text{opn} \in \text{set openings} \implies$
 $\text{opn'} \in \text{set openings} \implies$
 $\text{opening-index opn} = \text{opening-index opn'} \implies$
 $\text{opening-value opn} = \text{opening-value opn'}$
shows $\exists \text{table}. \text{table-agrees-with-authenticated-openings table len openings}$

proof –
let *?value =*
 $\lambda j. \text{if } \exists \text{opn} \in \text{set openings}. \text{opening-index opn} = j$

```

    then opening-value
      (SOME opn. opn ∈ set openings ∧ opening-index opn = j)
    else 0
  let ?table = map ?value [0.. $\text{len}$ ]
  have agrees:
     $\bigwedge \text{opn. opn} \in \text{set openings} \implies$ 
      ?table ! opening-index opn = opening-value opn
  proof -
    fix opn
    assume opn-in: opn ∈ set openings
    let ?chosen =
      SOME opn'. opn' ∈ set openings ∧
        opening-index opn' = opening-index opn
    have ex-chosen:
       $\exists \text{opn'. opn'} \in \text{set openings} \wedge$ 
        opening-index opn' = opening-index opn
    using opn-in by blast
    have chosen:
      ?chosen ∈ set openings ∧
        opening-index ?chosen = opening-index opn
    by (rule someI-ex[OF ex-chosen])
    have value-eq:
      opening-value ?chosen = opening-value opn
    using consistent[OF conjunct1[OF chosen] opn-in conjunct2[OF chosen]]
    by simp
    have ?table ! opening-index opn = ?value (opening-index opn)
      using idx-bound[OF opn-in] by simp
    also have ... = opening-value ?chosen
  proof -
    have  $\exists \text{opn'} \in \text{set openings. opening-index opn'} =$ 
      opening-index opn
    using opn-in by blast
    then show ?thesis by simp
  qed
  also have ... = opening-value opn
    by (rule value-eq)
  finally show ?table ! opening-index opn = opening-value opn .
  qed
  have table-agrees-with-authenticated-openings ?table len openings
    unfolding table-agrees-with-authenticated-openings-def
    using idx-bound agrees by simp
  then show ?thesis by blast
qed

```

definition *partial-trace-table-candidate*
 $:: 'f \text{ list} \implies 'f \text{ authenticated-opening list list} \implies \text{bool}$
where
partial-trace-table-candidate trace-table trace-openings $\longleftrightarrow$
 length trace-table = scale * clengeth ∧

$length\ trace_openings = rounds \wedge$
 $(\forall i < rounds.$
 $\quad table_agrees-with-authenticated-openings\ trace_table$
 $\quad (scale * clength) (trace_openings ! i))$

definition *partial-trace-table-candidates*

$:: 'f\ authenticated-opening\ list\ list \Rightarrow 'f\ list\ set$

where

$partial-trace-table-candidates\ trace_openings =$
 $\{trace_table.\ partial-trace-table-candidate\ trace_table\ trace_openings\}$

definition *accepted-with-partial-composition-openings*

$:: ('f,\ 'a)\ protocol-channel-scheme \Rightarrow$
 $\quad (unit\ list \times ('f,\ 'a)\ protocol-channel-scheme)\ option \Rightarrow$
 $\quad 'f \Rightarrow nat\ list \Rightarrow 'f\ authenticated-opening\ list\ list \Rightarrow bool$

where

$accepted-with-partial-composition-openings\ s\ out\ composition-root$
 $\quad query_idxs\ composition_openings \longleftrightarrow$
 $accepted\ out \wedge$
 $(\exists\ result\ final_state.$
 $\quad out = Some\ (result,\ final_state) \wedge$
 $\quad length\ query_idxs = rounds \wedge$
 $\quad length\ composition_openings = rounds \wedge$
 $\quad (\forall i < rounds.$
 $\quad \quad map\ opening-index\ (composition_openings ! i) =$
 $\quad \quad [query_idxs ! i,$
 $\quad \quad \quad fri-sibling-index\ (scale * clength) (query_idxs ! i)]) \wedge$
 $\quad (\forall i < rounds.$
 $\quad \quad partial-authenticated-table\ composition-root\ (scale * clength)$
 $\quad \quad (composition_openings ! i)\ final_state))$

definition *partial-composition-table-candidate*

$:: 'f\ list \Rightarrow 'f\ authenticated-opening\ list\ list \Rightarrow bool$

where

$partial-composition-table-candidate\ composition-table\ composition_openings$
 $\longleftrightarrow$
 $\quad length\ composition-table = scale * clength \wedge$
 $\quad length\ composition_openings = rounds \wedge$
 $\quad (\forall i < rounds.$
 $\quad \quad table-agrees-with-authenticated-openings\ composition-table$
 $\quad \quad (scale * clength) (composition_openings ! i))$

definition *partial-composition-table-candidates*

$:: 'f\ authenticated-opening\ list\ list \Rightarrow 'f\ list\ set$

where

$partial-composition-table-candidates\ composition_openings =$
 $\{composition-table.$
 $\quad partial-composition-table-candidate\ composition-table$
 $\quad \quad composition_openings\}$

lemma *partial-trace-table-candidateD*:
assumes *cand*: *partial-trace-table-candidate trace-table trace-openings*
and *i-bound*: $i < \text{rounds}$
and *opn-in*: $\text{opn} \in \text{set } (\text{trace-openings} \ ! \ i)$
shows $\text{length } \text{trace-table} = \text{scale} * \text{clength}$
and $\text{opening-index } \text{opn} < \text{scale} * \text{clength}$
and $\text{trace-table} \ ! \ \text{opening-index } \text{opn} = \text{opening-value } \text{opn}$
using *assms*
unfolding *partial-trace-table-candidate-def*
table-agrees-with-authenticated-openings-def
by *blast+*

lemma *table-agrees-with-authenticated-openings-map-values*:
assumes *agrees*:
table-agrees-with-authenticated-openings table len openings
and *idxs*: $\text{map } \text{opening-index } \text{openings} = \text{idxs}$
shows $\text{map } (!) \ \text{table} \ \text{idxs} = \text{map } \text{opening-value } \text{openings}$
proof (*rule nth-equalityI*)
show $\text{length } (\text{map } (!) \ \text{table} \ \text{idxs}) =$
 $\text{length } (\text{map } \text{opening-value } \text{openings})$
proof –
have $\text{length } \text{idxs} = \text{length } \text{openings}$
using *arg-cong[OF idxs, of length]* **by** *simp*
then show *?thesis* **by** *simp*
qed
next
fix *k*
assume *k-bound*: $k < \text{length } (\text{map } (!) \ \text{table} \ \text{idxs})$
have *len-idxs*: $\text{length } \text{idxs} = \text{length } \text{openings}$
using *arg-cong[OF idxs, of length]* **by** *simp*
then have *k-openings*: $k < \text{length } \text{openings}$
using *k-bound len-idxs* **by** *simp*
have *idx-eq*: $\text{idxs} \ ! \ k = \text{opening-index } (\text{openings} \ ! \ k)$
using *idxs k-openings* **by** (*metis length-map nth-map*)
have *opening-in*: $\text{openings} \ ! \ k \in \text{set } \text{openings}$
by (*rule nth-mem[OF k-openings]*)
have $\text{table} \ ! \ \text{opening-index } (\text{openings} \ ! \ k) =$
 $\text{opening-value } (\text{openings} \ ! \ k)$
using *agrees opening-in*
unfolding *table-agrees-with-authenticated-openings-def* **by** *blast*
then show $\text{map } (!) \ \text{table} \ \text{idxs} \ ! \ k =$
 $\text{map } \text{opening-value } \text{openings} \ ! \ k$
using *k-bound k-openings idx-eq* **by** *simp*
qed

lemma *partial-trace-table-candidate-query-values*:
assumes *cand*: *partial-trace-table-candidate trace-table trace-openings*
and *i-bound*: $i < \text{rounds}$

and *idxs*:
 $\text{map opening-index } (\text{trace-openings } ! i) = \text{powers-scaled idx}$
shows
 $\text{map } (! \text{ trace-table}) (\text{powers-scaled idx}) =$
 $\text{map opening-value } (\text{trace-openings } ! i)$
proof –
have *agrees*:
 $\text{table-agrees-with-authenticated-openings trace-table}$
 $(\text{scale} * \text{clength}) (\text{trace-openings } ! i)$
using *cand i-bound unfolding partial-trace-table-candidate-def* **by** *blast*
show *?thesis*
by (*rule table-agrees-with-authenticated-openings-map-values*
 $[OF \text{ agrees } idxs]$)
qed

lemma *partial-trace-table-candidates-iff*:
 $\text{trace-table} \in \text{partial-trace-table-candidates trace-openings} \longleftrightarrow$
 $\text{partial-trace-table-candidate trace-table trace-openings}$
unfolding *partial-trace-table-candidates-def* **by** *simp*

lemma *accepted-with-partial-composition-openings-imp-accepted*:
assumes
 $\text{accepted-with-partial-composition-openings } s \text{ out composition-root}$
 $\text{query-idxs composition-openings}$
shows *accepted out*
using *assms*
unfolding *accepted-with-partial-composition-openings-def* **by** *simp*

lemma *accepted-with-partial-composition-openings-shapes*:
assumes
 $\text{accepted-with-partial-composition-openings } s \text{ out composition-root}$
 $\text{query-idxs composition-openings}$
shows $\text{length query-idxs} = \text{rounds}$
and $\text{length composition-openings} = \text{rounds}$
and $\bigwedge i. i < \text{rounds} \implies$
 $\text{map opening-index } (\text{composition-openings } ! i) =$
 $[\text{query-idxs } ! i,$
 $\text{fri-sibling-index } (\text{scale} * \text{clength}) (\text{query-idxs } ! i)]$
using *assms*
unfolding *accepted-with-partial-composition-openings-def* **by** *blast+*

definition *accepted-with-partial-initial-openings*
 $:: ('f, 'a) \text{ protocol-channel-scheme} \Rightarrow$
 $(\text{unit list} \times ('f, 'a) \text{ protocol-channel-scheme}) \text{ option} \Rightarrow$
 $'f \Rightarrow 'f \text{ list} \Rightarrow 'f \Rightarrow 'f \text{ list} \Rightarrow 'f \Rightarrow$
 $'f \text{ list} \Rightarrow 'f \Rightarrow \text{nat list} \Rightarrow$
 $'f \text{ authenticated-opening list list} \Rightarrow \text{nat list} \Rightarrow$
 $'f \text{ authenticated-opening list list} \Rightarrow \text{bool}$
where

accepted-with-partial-initial-openings *s* out fr *f-fri-roots* *f-final* as *dg*
composition-fri-roots *final* *trace-query-idxs* *trace-openings*
composition-query-idxs *composition-openings* $\longleftrightarrow$
accepted out $\wedge$
composition-fri-roots $\neq \square$ $\wedge$
 $(\exists$ *rest*.
verifier-header-transcript *s* fr *f-fri-roots* *f-final* as *dg*
composition-fri-roots *final* *rest*) $\wedge$
accepted-with-partial-trace-openings *s* out fr *trace-query-idxs*
trace-openings $\wedge$
accepted-with-partial-composition-openings *s* out
 $(\text{hd } \text{composition-fri-roots})$ *composition-query-idxs* *composition-openings*

lemma *accepted-with-partial-initial-openings-imp-accepted*:

assumes

accepted-with-partial-initial-openings *s* out fr *f-fri-roots* *f-final* as *dg*
composition-fri-roots *final* *trace-query-idxs* *trace-openings*
composition-query-idxs *composition-openings*

shows *accepted* out

using *assms* **unfolding** *accepted-with-partial-initial-openings-def* **by** *simp*

lemma *accepted-with-partial-initial-openings-shapes*:

assumes

accepted-with-partial-initial-openings *s* out fr *f-fri-roots* *f-final* as *dg*
composition-fri-roots *final* *trace-query-idxs* *trace-openings*
composition-query-idxs *composition-openings*

shows *composition-fri-roots* $\neq \square$

and $\exists$ *rest*. *verifier-header-transcript* *s* fr *f-fri-roots* *f-final* as *dg*
composition-fri-roots *final* *rest*

and *accepted-with-partial-trace-openings* *s* out fr *trace-query-idxs*
trace-openings

and *accepted-with-partial-composition-openings* *s* out
 $(\text{hd } \text{composition-fri-roots})$ *composition-query-idxs* *composition-openings*

using *assms* **unfolding** *accepted-with-partial-initial-openings-def* **by** *blast+*

lemma *query-rounds-accepted-with-partial-composition-openings*:

fixes *s* *query-state* *final-state* :: (*f*, *a*) *protocol-channel-scheme*

assumes *fl-eq*: *fl* = (*b*, *composition-root*) # *fl-tail*

and *outcome*:

Some (*results*, *final-state*) $\in$
set-dist
(execute
(ntimes
(verifier-query-round-program fr *f-fl* *f-final* as *fl* *final*)
rounds)
query-state)

obtains *query-idxs* *composition-openings* **where**

accepted-with-partial-composition-openings *s*
 $(\text{Some } (\text{results}, \text{final-state}))$ *composition-root* *query-idxs*

composition-openings
proof –
from *ntimes-verifier-query-rounds-authenticated-composition-openings*
[OF fl-eq outcome]
obtain *raw-idxs query-idxs composition-openings* **where**
len-raw: length raw-idxs = rounds
and *query-idxs-def:*
query-idxs = map (λraw. index (to-nat raw)) raw-idxs
and *len-openings: length composition-openings = rounds*
and *indices:*
 $\bigwedge i. i < \text{rounds} \implies$
 $\text{map opening-index } (\text{composition-openings } ! i) =$
 $[\text{query-idxs } ! i,$
 $\text{fri-sibling-index } (\text{scale} * \text{clength}) (\text{query-idxs } ! i)]$
and *tables:*
 $\bigwedge i. i < \text{rounds} \implies$
 $\text{partial-authenticated-table } \text{composition-root } (\text{scale} * \text{clength})$
 $(\text{composition-openings } ! i) \text{ final-state}$
by *blast*
have *len-query-idxs: length query-idxs = rounds*
using *len-raw query-idxs-def* **by** *simp*
have *partial:*
accepted-with-partial-composition-openings s
(Some (results, final-state)) composition-root query-idxs
composition-openings
unfolding *accepted-with-partial-composition-openings-def accepted-def*
apply (*intro conjI exI*)
apply *simp*
apply *simp*
apply (*rule len-query-idxs*)
apply (*rule len-openings*)
using *indices tables* **by** *simp-all*
show *?thesis*
by (*rule that[OF partial]*)
qed

lemma *verify-monad-accepted-with-partial-composition-openings-if-bound-tables:*
fixes *s :: ('f, 'a) protocol-channel-scheme*
assumes *outcome:*
Some (result, final-state) ∈ set-dist (execute verify-monad s)
and *bound:*
accepted-with-bound-tables s (Some (result, final-state))
trace-table composition-table as query-idxs-bound
obtains *composition-root query-idxs composition-openings* **where**
accepted-with-partial-composition-openings s
(Some (result, final-state)) composition-root query-idxs
composition-openings
merkle-root-binds-table composition-root composition-table final-state
proof –

```

from verify-monad-header-extraction[OF outcome]
obtain fr fl f-final as' dg fl final query-state where
  header:
    verifier-header-transcript s fr (map snd f-fl) f-final as' dg
    (map snd fl) final (PTranscript query-state)
and query-out:
  Some (result, final-state) ∈
    set-dist
    (execute
      (ntimes
        (verifier-query-round-program fr f-fl f-final as' fl final)
        rounds)
      query-state)
by metis
from bound obtain fr' f-fri-roots' f-final' dg'
  composition-fri-roots' final' rest' where
  header':
    verifier-header-transcript s fr' f-fri-roots' f-final' as dg'
    composition-fri-roots' final' rest'
and comp-nonempty: composition-fri-roots' ≠ []
and comp-bind:
  merkle-root-binds-table (hd composition-fri-roots')
  composition-table final-state
unfolding accepted-with-bound-tables-def by blast
have eqs:
  fr' = fr ∧
  f-fri-roots' = map snd f-fl ∧
  f-final' = f-final ∧
  as = as' ∧
  dg' = dg ∧
  composition-fri-roots' = map snd fl ∧
  final' = final ∧
  rest' = PTranscript query-state
using verifier-header-transcript-unique[OF header' header] by simp
then have fl-nonempty: fl ≠ []
using comp-nonempty by (cases fl) simp-all
then obtain b composition-root fl-tail where fl-eq:
  fl = (b, composition-root) # fl-tail
by (cases fl) auto
have comp-root-eq: hd composition-fri-roots' = composition-root
using eqs fl-eq comp-nonempty by simp
from query-rounds-accepted-with-partial-composition-openings
  [OF fl-eq query-out]
obtain query-idxs composition-openings where partial:
  accepted-with-partial-composition-openings s
  (Some (result, final-state)) composition-root query-idxs
  composition-openings
by blast
have bind:

```

```

merkle-root-binds-table composition-root composition-table final-state
using comp-bind unfolding comp-root-eq .
show ?thesis
  by (rule that[OF partial bind])
qed

lemma verify-monad-accepted-with-partial-composition-openings-if-header-nonempty:
fixes s :: ('f, 'a) protocol-channel-scheme
assumes outcome:
  Some (result, final-state) ∈ set-dist (execute verify-monad s)
and header0:
  verifier-header-transcript s fr f-fri-roots f-final as dg
  composition-fri-roots final rest
and comp-nonempty: composition-fri-roots ≠ []
obtains query-idxs composition-openings where
  accepted-with-partial-composition-openings s
  (Some (result, final-state)) (hd composition-fri-roots) query-idxs
  composition-openings
proof —
from verify-monad-header-extraction[OF outcome]
obtain fr' f-fl f-final' as' dg' fl final' query-state where
  header:
    verifier-header-transcript s fr' (map snd f-fl) f-final' as' dg'
    (map snd fl) final' (PTranscript query-state)
and query-out:
  Some (result, final-state) ∈
    set-dist
    (execute
    (ntimes
    (verifier-query-round-program fr' f-fl f-final' as' fl
    final') rounds)
    query-state)
by metis
have eqs:
  fr' = fr ∧
  map snd f-fl = f-fri-roots ∧
  f-final' = f-final ∧
  as' = as ∧
  dg' = dg ∧
  map snd fl = composition-fri-roots ∧
  final' = final ∧
  PTranscript query-state = rest
using verifier-header-transcript-unique[OF header header0] by simp
have fl-nonempty: fl ≠ []
using comp-nonempty eqs by (cases fl) simp-all
then obtain b composition-root fl-tail where fl-eq:
  fl = (b, composition-root) # fl-tail
by (cases fl) auto
have root-eq: composition-root = hd composition-fri-roots

```

```

proof –
  have map-eq: map snd fl = composition-fri-roots
    using eqs by simp
  then have composition-root # map snd fl-tail = composition-fri-roots
    unfolding fl-eq by simp
  then have hd composition-fri-roots = composition-root
    by (metis list.sel(1))
  then show ?thesis
    by simp
qed
from query-rounds-accepted-with-partial-composition-openings
  [OF fl-eq query-out]
obtain query-idxs composition-openings where partial:
  accepted-with-partial-composition-openings s
  (Some (result, final-state)) composition-root query-idxs
  composition-openings
by blast
show ?thesis
by (rule that[of query-idxs composition-openings])
  (use partial root-eq in simp)
qed

lemma partial-composition-table-candidateD:
assumes
  cand:
    partial-composition-table-candidate composition-table
    composition-openings
and i-bound: i < rounds
and opn-in: opn ∈ set (composition-openings ! i)
shows length composition-table = scale * clength
and opening-index opn < scale * clength
and composition-table ! opening-index opn = opening-value opn
using assms
unfolding partial-composition-table-candidate-def
  table-agrees-with-authenticated-openings-def
by blast+

lemma partial-composition-table-candidate-query-values:
assumes cand:
  partial-composition-table-candidate composition-table
  composition-openings
and i-bound: i < rounds
and idxs:
  map opening-index (composition-openings ! i) =
  [idx, fri-sibling-index (scale * clength) idx]
shows
  map (!) composition-table
  [idx, fri-sibling-index (scale * clength) idx] =
  map opening-value (composition-openings ! i)

```

```

proof –
  have agrees:
    table-agrees-with-authenticated-openings composition-table
      (scale * clength) (composition-openings ! i)
  using cand i-bound
  unfolding partial-composition-table-candidate-def by blast
  show ?thesis
  by (rule table-agrees-with-authenticated-openings-map-values
    [OF agrees idxs])
qed

```

```

lemma partial-composition-table-candidate-query-value:
  assumes cand:
    partial-composition-table-candidate composition-table
      composition-openings
  and i-bound: i < rounds
  and idxs:
    map opening-index (composition-openings ! i) =
      [idx, fri-sibling-index (scale * clength) idx]
  shows
    composition-table ! idx =
      opening-value ((composition-openings ! i) ! 0)

```

```

proof –
  have vals:
    map ((!) composition-table)
      [idx, fri-sibling-index (scale * clength) idx] =
    map opening-value (composition-openings ! i)
  by (rule partial-composition-table-candidate-query-values
    [OF cand i-bound idxs])
  have len-openings: length (composition-openings ! i) = 2
  proof –
    have length (map opening-index (composition-openings ! i)) =
      length [idx, fri-sibling-index (scale * clength) idx]
    using arg-cong[OF idxs, of length] by simp
    then show ?thesis by simp
  qed
  have value0:
    map ((!) composition-table)
      [idx, fri-sibling-index (scale * clength) idx] ! 0 =
    map opening-value (composition-openings ! i) ! 0
  using vals by simp
  show ?thesis
  using value0 len-openings by (cases composition-openings ! i) auto
qed

```

```

lemma partial-composition-table-candidates-iff:
  composition-table ∈
    partial-composition-table-candidates composition-openings  $\longleftrightarrow$ 
    partial-composition-table-candidate composition-table composition-openings

```

unfolding *partial-composition-table-candidates-def* **by** *simp*

lemma *verify-monad-accepted-with-partial-trace-openings*:
fixes $s :: ('f, 'a) \text{ protocol-channel-scheme}$
assumes *outcome*:
 $\text{Some } (\text{result}, \text{final-state}) \in \text{set-dist } (\text{execute verify-monad } s)$
obtains *fr query-idxs trace-openings* **where**
 $\text{accepted-with-partial-trace-openings } s$
 $(\text{Some } (\text{result}, \text{final-state})) \text{ fr query-idxs trace-openings}$
proof –
show *?thesis*
proof (*rule verify-monad-authenticated-trace-openings[OF outcome]*)
fix *fr query-idxs trace-openings*
assume *len-query-idxs*: $\text{length query-idxs} = \text{rounds}$
and *len-openings*: $\text{length trace-openings} = \text{rounds}$
and *indices*:
 $\bigwedge i. i < \text{rounds} \implies$
 $\text{map opening-index } (\text{trace-openings } ! i) =$
 $\text{powers-scaled } (\text{query-idxs } ! i)$
and *tables*:
 $\bigwedge i. i < \text{rounds} \implies$
 $\text{partial-authenticated-table fr } (\text{scale } * \text{clength})$
 $(\text{trace-openings } ! i) \text{ final-state}$
have *partial*:
 $\text{accepted-with-partial-trace-openings } s$
 $(\text{Some } (\text{result}, \text{final-state})) \text{ fr query-idxs trace-openings}$
unfolding *accepted-with-partial-trace-openings-def accepted-def*
apply (*intro conjI exI*)
apply *simp*
apply *simp*
apply (*rule len-query-idxs*)
apply (*rule len-openings*)
using *indices tables by simp-all*
show *?thesis*
by (*rule that[OF partial]*)
qed
qed

lemma *verify-monad-accepted-with-partial-trace-openings-for-header*:
fixes $s :: ('f, 'a) \text{ protocol-channel-scheme}$
assumes *outcome*:
 $\text{Some } (\text{result}, \text{final-state}) \in \text{set-dist } (\text{execute verify-monad } s)$
and *header0*:
 $\text{verifier-header-transcript } s \text{ fr f-fri-roots f-final as dg}$
 $\text{composition-fri-roots final rest}$
obtains *query-idxs trace-openings* **where**
 $\text{accepted-with-partial-trace-openings } s$
 $(\text{Some } (\text{result}, \text{final-state})) \text{ fr query-idxs trace-openings}$
proof –


```

from verify-monad-header-extraction[OF outcome]
obtain fr' f-fl f-final' as' dg' fl final' query-state where
  header:
    verifier-header-transcript s fr' (map snd f-fl) f-final' as' dg'
    (map snd fl final' (PTranscript query-state))
and query-out:
  Some (result, final-state) ∈
    set-dist
    (execute
      (ntimes
        (verifier-query-round-program fr' f-fl f-final' as' fl
          final') rounds)
      query-state)
by metis
have fr-eq: fr' = fr
  using verifier-header-transcript-unique[OF header header0] by simp
obtain query-idxs trace-openings where
  len-query-idxs: length query-idxs = rounds
and len-openings: length trace-openings = rounds
and indices:
   $\bigwedge i. i < rounds \implies$ 
    map opening-index (trace-openings ! i) =
      powers-scaled (query-idxs ! i)
and tables:
   $\bigwedge i. i < rounds \implies$ 
    partial-authenticated-table fr' (scale * clength)
    (trace-openings ! i final-state)
proof (rule ntimes-verifier-query-rounds-authenticated-trace-openings-no-raws
  [OF query-out])
fix query-idxs trace-openings
assume len-query-idxs: length query-idxs = rounds
and len-openings: length trace-openings = rounds
and indices:
   $\bigwedge i. i < rounds \implies$ 
    map opening-index (trace-openings ! i) =
      powers-scaled (query-idxs ! i)
and tables:
   $\bigwedge i. i < rounds \implies$ 
    partial-authenticated-table fr' (scale * clength)
    (trace-openings ! i final-state)
show ?thesis
  by (rule that[OF len-query-idxs len-openings indices tables])
qed
have partial:
  accepted-with-partial-trace-openings s
  (Some (result, final-state)) fr query-idxs trace-openings
proof –
  have all-indices:
     $\forall i < rounds.$ 

```

```

    map opening-index (trace-openings ! i) =
      powers-scaled (query-idxs ! i)
  using indices by blast
have all-tables:
  ∀ i < rounds.
    partial-authenticated-table fr (scale * clength)
      (trace-openings ! i) final-state
  using tables fr-eq by simp
show ?thesis
  unfolding accepted-with-partial-trace-openings-def accepted-def
  by (intro conjI exI[of - result] exI[of - final-state])
    (use len-query-idxs len-openings all-indices all-tables in simp-all)
qed
show ?thesis
  by (rule that[OF partial])
qed

lemma verify-monad-accepted-with-partial-initial-openings-if-header-nonempty:
  fixes s :: ('f, 'a) protocol-channel-scheme
  assumes outcome:
    Some (result, final-state) ∈ set-dist (execute verify-monad s)
  and header:
    verifier-header-transcript s fr f-fri-roots f-final as dg
    composition-fri-roots final rest
  and comp-nonempty: composition-fri-roots ≠ []
  obtains trace-query-idxs trace-openings composition-query-idxs
    composition-openings where
    accepted-with-partial-trace-openings s
      (Some (result, final-state)) fr trace-query-idxs trace-openings
    accepted-with-partial-composition-openings s
      (Some (result, final-state)) (hd composition-fri-roots)
    composition-query-idxs composition-openings
proof -
  from verify-monad-accepted-with-partial-trace-openings-for-header
    [OF outcome header]
  obtain trace-query-idxs trace-openings where trace-partial:
    accepted-with-partial-trace-openings s
      (Some (result, final-state)) fr trace-query-idxs trace-openings
  by blast
  from verify-monad-accepted-with-partial-composition-openings-if-header-nonempty
    [OF outcome header comp-nonempty]
  obtain composition-query-idxs composition-openings where composition-partial:
    accepted-with-partial-composition-openings s
      (Some (result, final-state)) (hd composition-fri-roots)
    composition-query-idxs composition-openings
  by blast
  show ?thesis
    by (rule that[OF trace-partial composition-partial])
qed

```

lemma *verify-monad-accepted-with-partial-initial-openings*:
fixes $s :: ('f, 'a) \text{ protocol-channel-scheme}$
assumes *outcome*:
 $\text{Some } (\text{result}, \text{final-state}) \in \text{set-dist } (\text{execute verify-monad } s)$
and *header*:
 $\text{verifier-header-transcript } s \text{ fr } f\text{-fri-roots } f\text{-final as } dg$
 $\text{composition-fri-roots final rest}$
and *comp-nonempty*: $\text{composition-fri-roots} \neq []$
obtains $\text{trace-query-idxs trace-openings composition-query-idxs}$
 $\text{composition-openings}$ **where**
 $\text{accepted-with-partial-initial-openings } s$
 $(\text{Some } (\text{result}, \text{final-state})) \text{ fr } f\text{-fri-roots } f\text{-final as } dg$
 $\text{composition-fri-roots final trace-query-idxs trace-openings}$
 $\text{composition-query-idxs composition-openings}$
proof –
from *verify-monad-accepted-with-partial-initial-openings-if-header-nonempty*
 $[OF \text{ outcome header comp-nonempty}]$
obtain $\text{trace-query-idxs trace-openings composition-query-idxs}$
 $\text{composition-openings}$ **where**
trace-partial:
 $\text{accepted-with-partial-trace-openings } s$
 $(\text{Some } (\text{result}, \text{final-state})) \text{ fr trace-query-idxs trace-openings}$
and *composition-partial*:
 $\text{accepted-with-partial-composition-openings } s$
 $(\text{Some } (\text{result}, \text{final-state})) (\text{hd } \text{composition-fri-roots})$
 $\text{composition-query-idxs composition-openings}$
by *blast*
have *accepted-out*: $\text{accepted } (\text{Some } (\text{result}, \text{final-state}))$
by (*rule* *accepted-with-partial-trace-openings-imp-accepted*
 $[OF \text{ trace-partial}]$)
have *evidence*:
 $\text{accepted-with-partial-initial-openings } s$
 $(\text{Some } (\text{result}, \text{final-state})) \text{ fr } f\text{-fri-roots } f\text{-final as } dg$
 $\text{composition-fri-roots final trace-query-idxs trace-openings}$
 $\text{composition-query-idxs composition-openings}$
unfolding *accepted-with-partial-initial-openings-def*
by (*intro conjI exI* [*of* - *rest*])
 $(\text{use } \text{accepted-out comp-nonempty header trace-partial}$
 $\text{composition-partial in simp-all})$
show *?thesis*
by (*rule* *that* [*OF evidence*])
qed

lemma *merkle-path-bound-unique-value-if-clean*:
assumes *clean*: $\neg \text{hash-map-output-collision } s$
and *first*: $\text{merkle-path-bound } rt \text{ len } idx \text{ } v \text{ path } s$
and *second*: $\text{merkle-path-bound } rt \text{ len } idx \text{ } v' \text{ path}' s$
and *same-length*: $\text{length path} = \text{length path}'$

```

shows  $v = v'$ 
using first second same-length
proof (induction path arbitrary: path' rt len idx)
case Nil
then have path': path' = [] by simp
have first-lookup:
  fmlookup (HashMap s) (MerkleLeaf v) = Some rt
  using Nil.premis(1) by simp
have second-lookup:
  fmlookup (HashMap s) (MerkleLeaf v') = Some rt
  using Nil.premis(2) path' by simp
show ?case
proof (rule ccontr)
  assume  $v \neq v'$ 
  then have keys: MerkleLeaf v  $\neq$  MerkleLeaf v' by simp
  have hash-map-output-collision s
    by (rule hash-map-output-collisionI
      [OF keys first-lookup second-lookup])
  then show False using clean by contradiction
qed
next
case (Cons sibling path)
from Cons.premis(3) obtain sibling' path'' where
  path': path' = sibling' # path''
  and lengths: length path = length path''
  by (cases path') auto
show ?case
proof (cases idx < len div 2)
case True
  obtain child where
    child:
      merkle-path-bound child (len div 2) idx v path s
  and first-lookup:
    fmlookup (HashMap s) (MerkleNode child sibling) = Some rt
  using Cons.premis(1) True by auto
  obtain child' where
    child':
      merkle-path-bound child' (len div 2) idx v' path'' s
  and second-lookup:
    fmlookup (HashMap s) (MerkleNode child' sibling') = Some rt
  using Cons.premis(2) True path' by auto
  have node-eq:
    MerkleNode child sibling = MerkleNode child' sibling'
  proof (rule ccontr)
    assume neq:
      MerkleNode child sibling  $\neq$  MerkleNode child' sibling'
    have hash-map-output-collision s
      by (rule hash-map-output-collisionI
        [OF neq first-lookup second-lookup])

```

```

    then show False using clean by contradiction
  qed
  have child-eq: child = child' using node-eq by simp
  show ?thesis
    by (rule Cons.IH[OF child - lengths])
      (use child' child-eq in simp)
next
case False
obtain child where
  child:
    merkle-path-bound child (len div 2) (idx - len div 2)
    v path s
  and first-lookup:
    fmlookup (HashMap s) (MerkleNode sibling child) = Some rt
  using Cons.prems(1) False by auto
obtain child' where
  child':
    merkle-path-bound child' (len div 2) (idx - len div 2)
    v' path'' s
  and second-lookup:
    fmlookup (HashMap s) (MerkleNode sibling' child') = Some rt
  using Cons.prems(2) False path' by auto
have node-eq:
  MerkleNode sibling child = MerkleNode sibling' child'
proof (rule ccontr)
  assume neg:
    MerkleNode sibling child ≠ MerkleNode sibling' child'
  have hash-map-output-collision s
    by (rule hash-map-output-collisionI
      [OF neg first-lookup second-lookup])
  then show False using clean by contradiction
qed
have child-eq: child = child' using node-eq by simp
show ?thesis
  by (rule Cons.IH[OF child - lengths])
    (use child' child-eq in simp)
qed
qed

lemma inconsistent-authenticated-openings-imp-hash-collision:
  assumes first: authenticated-opening-in s opening
    and second: authenticated-opening-in s opening'
    and same-root: opening-root opening = opening-root opening'
    and same-length: opening-length opening = opening-length opening'
    and same-index: opening-index opening = opening-index opening'
    and different-value: opening-value opening ≠ opening-value opening'
  shows hash-map-output-collision s
proof (rule ccontr)
  assume clean: ¬ hash-map-output-collision s

```

```

have path-lengths:
  length (opening-path opening) = length (opening-path opening')
  using first second same-length
  unfolding authenticated-opening-in-def by simp
have first-bound:
  merkle-path-bound
    (opening-root opening)
    (opening-length opening)
    (opening-index opening)
    (opening-value opening)
    (opening-path opening)
  s
  using first unfolding authenticated-opening-in-def by blast
have second-bound:
  merkle-path-bound
    (opening-root opening')
    (opening-length opening')
    (opening-index opening')
    (opening-value opening')
    (opening-path opening')
  s
  using second unfolding authenticated-opening-in-def by blast
have values-eq:
  opening-value opening = opening-value opening'
proof (rule merkle-path-bound-unique-value-if-clean[
  OF clean first-bound - path-lengths])
show
  merkle-path-bound
    (opening-root opening)
    (opening-length opening)
    (opening-index opening)
    (opening-value opening')
    (opening-path opening')
  s
  using second-bound same-root same-length same-index by simp
qed
show False using different-value values-eq by contradiction
qed

lemma partial-authenticated-table-values-unique-if-clean:
  assumes table: partial-authenticated-table rt len openings s
  and clean:  $\neg$  hash-map-output-collision s
  and first: opening  $\in$  set openings
  and second: opening'  $\in$  set openings
  and same-index: opening-index opening = opening-index opening'
  shows opening-value opening = opening-value opening'
proof (rule ccontr)
  assume different: opening-value opening  $\neq$  opening-value opening'
  have first-auth: authenticated-opening-in s opening

```

```

    using table first unfolding partial-authenticated-table-def by blast
  have second-auth: authenticated-opening-in s opening'
    using table second unfolding partial-authenticated-table-def by blast
  have roots: opening-root opening = opening-root opening'
  proof -
    have opening-root opening = rt
      using table first unfolding partial-authenticated-table-def by blast
    moreover have opening-root opening' = rt
      using table second unfolding partial-authenticated-table-def by blast
    ultimately show ?thesis by simp
  qed
  have lengths: opening-length opening = opening-length opening'
  proof -
    have opening-length opening = len
      using table first unfolding partial-authenticated-table-def by blast
    moreover have opening-length opening' = len
      using table second unfolding partial-authenticated-table-def by blast
    ultimately show ?thesis by simp
  qed
  have hash-map-output-collision s
    by (rule inconsistent-authenticated-openings-imp-hash-collision[
      OF first-auth second-auth roots lengths same-index different])
  then show False using clean by contradiction
  qed

lemma accepted-with-partial-trace-openings-values-unique-if-clean:
  assumes partial:
    accepted-with-partial-trace-openings s (Some (result, final-state))
    fr query-idxs trace-openings
  and clean:  $\neg$  hash-map-output-collision final-state
  and i-bound:  $i < \text{rounds}$ 
  and opn-in:  $\text{opn} \in \text{set } (\text{trace-openings } ! i)$ 
  and opn'-in:  $\text{opn}' \in \text{set } (\text{trace-openings } ! i)$ 
  and same-index:  $\text{opening-index } \text{opn} = \text{opening-index } \text{opn}'$ 
  shows opening-value opn = opening-value opn'
  proof -
    have table:
      partial-authenticated-table fr (scale * clength)
      (trace-openings ! i) final-state
    using partial i-bound unfolding accepted-with-partial-trace-openings-def
    by blast
  show ?thesis
    by (rule partial-authenticated-table-values-unique-if-clean
      [OF table clean opn-in opn'-in same-index])
  qed

definition partial-trace-openings-inconsistent-bad
  :: ('f, 'a) protocol-channel-scheme  $\Rightarrow$ 
    (unit list  $\times$  ('f, 'a) protocol-channel-scheme) option  $\Rightarrow$  bool

```

where

partial-trace-openings-inconsistent-bad s *out* $\longleftrightarrow$
 $(\exists \text{ fr query-idxs trace-openings result final-state } i \text{ opn opn'.$
 $\text{out} = \text{Some}(\text{result}, \text{final-state}) \wedge$
 $\text{accepted-with-partial-trace-openings } s \text{ out fr query-idxs trace-openings} \wedge$
 $i < \text{rounds} \wedge$
 $\text{opn} \in \text{set}(\text{trace-openings } ! i) \wedge$
 $\text{opn}' \in \text{set}(\text{trace-openings } ! i) \wedge$
 $\text{opening-index } \text{opn} = \text{opening-index } \text{opn}' \wedge$
 $\text{opening-value } \text{opn} \neq \text{opening-value } \text{opn}')$

lemma *accepted-with-partial-trace-openings-values-unique-if-no-bad*:

assumes *partial*:

$\text{accepted-with-partial-trace-openings } s (\text{Some}(\text{result}, \text{final-state}))$
 $\text{fr query-idxs trace-openings}$

and *no-bad*:

$\neg \text{partial-trace-openings-inconsistent-bad } s$
 $(\text{Some}(\text{result}, \text{final-state}))$

and *i-bound*: $i < \text{rounds}$

and *opn-in*: $\text{opn} \in \text{set}(\text{trace-openings } ! i)$

and *opn'-in*: $\text{opn}' \in \text{set}(\text{trace-openings } ! i)$

and *same-index*: $\text{opening-index } \text{opn} = \text{opening-index } \text{opn}'$

shows $\text{opening-value } \text{opn} = \text{opening-value } \text{opn}'$

proof (rule *ccontr*)

assume *neg*: $\text{opening-value } \text{opn} \neq \text{opening-value } \text{opn}'$

have *partial-trace-openings-inconsistent-bad* s

$(\text{Some}(\text{result}, \text{final-state}))$

unfolding *partial-trace-openings-inconsistent-bad-def*

by (intro $\text{exI}[\text{of} - \text{fr}] \text{exI}[\text{of} - \text{query-idxs}]$
 $\text{exI}[\text{of} - \text{trace-openings}] \text{exI}[\text{of} - \text{result}]$
 $\text{exI}[\text{of} - \text{final-state}] \text{exI}[\text{of} - i] \text{exI}[\text{of} - \text{opn}]$
 $\text{exI}[\text{of} - \text{opn}']$)

(use *partial i-bound opn-in opn'-in same-index neg* **in** *simp*)

then show *False* **using** *no-bad* **by** *contradiction*

qed

lemma *merkle-bound-table-agrees-with-authenticated-opening-if-clean*:

assumes *bind*: *merkle-root-binds-table* rt *table* *final-state*

and *len-pow*: $\text{length } \text{table} = 2^n$

and *auth*: *authenticated-opening-in* *final-state* *opening*

and *root*: *opening-root* *opening* = rt

and *len*: *opening-length* *opening* = $\text{length } \text{table}$

and *clean*: $\neg \text{hash-map-output-collision } \text{final-state}$

shows

$\text{opening-index } \text{opening} < \text{length } \text{table} \wedge$
 $\text{table } ! \text{opening-index } \text{opening} = \text{opening-value } \text{opening}$

proof –

from *bind* **obtain** *tree* **where** *created*:

created-tree *table* *tree* *final-state*


```

and rt-eq: rt = value tree
unfolding merkle-root-binds-table-def by blast
have idx-bound: opening-index opening < length table
using auth len unfolding authenticated-opening-in-def by simp
have table-root:
  merkle-path-root-in-map final-state (length table)
    (opening-index opening) (table ! opening-index opening)
    (get-authentication-path (length table) (opening-index opening) tree) =
    Some rt
using protocol-created-tree-path-root-in-map[
  OF created[unfolded created-tree-def] len-pow idx-bound]
unfolding rt-eq .
have opening-root-in-map:
  merkle-path-root-in-map final-state (length table)
    (opening-index opening) (opening-value opening)
    (opening-path opening) = Some rt
using authenticated-opening-root-in-map[OF auth] root len by simp
have same:
  (table ! opening-index opening = opening-value opening ∧
    get-authentication-path (length table) (opening-index opening) tree =
    opening-path opening) ∨
    hash-map-output-collision final-state
by (rule merkle-path-root-same-index-eq-or-hash-collision[
  OF table-root opening-root-in-map])
then have value-eq:
  table ! opening-index opening = opening-value opening
using clean by blast
show ?thesis
using idx-bound value-eq by simp
qed

lemma accepted-bound-trace-table-partial-candidate-if-clean:
assumes bind: merkle-root-binds-table fr trace-table final-state
and len-table: length trace-table = scale * clength
and partial:
  accepted-with-partial-trace-openings s (Some (result, final-state))
    fr query-idxs trace-openings
and clean: ¬ hash-map-output-collision final-state
shows partial-trace-table-candidate trace-table trace-openings
proof –
obtain n where len-pow: clength * scale =  $2^n$ 
using eval-domain-length-power by blast
have table-pow: length trace-table =  $2^n$ 
using len-table len-pow by (simp add: mult.commute)
have len-openings: length trace-openings = rounds
using partial unfolding accepted-with-partial-trace-openings-def by blast
have agrees:
  ∧i. i < rounds ⇒
    table-agrees-with-authenticated-openings trace-table

```

```

      (scale * clength) (trace-openings ! i)
proof –
  fix i
  assume i-bound: i < rounds
  have table-i:
    partial-authenticated-table fr (scale * clength)
      (trace-openings ! i) final-state
  using partial i-bound
  unfolding accepted-with-partial-trace-openings-def by blast
show
  table-agrees-with-authenticated-openings trace-table
    (scale * clength) (trace-openings ! i)
proof –
  have opening-agrees:
     $\forall \text{opn} \in \text{set } (\text{trace-openings} ! i).$ 
      opening-index opn < scale * clength  $\wedge$ 
      trace-table ! opening-index opn = opening-value opn
  proof
  fix opn
  assume opn-in: opn  $\in$  set (trace-openings ! i)
  have root: opening-root opn = fr
    using table-i opn-in unfolding partial-authenticated-table-def
    by blast
  have len: opening-length opn = length trace-table
    using table-i opn-in len-table
    unfolding partial-authenticated-table-def by simp
  have auth: authenticated-opening-in final-state opn
    using table-i opn-in unfolding partial-authenticated-table-def
    by blast
  have agreement:
    opening-index opn < length trace-table  $\wedge$ 
    trace-table ! opening-index opn = opening-value opn
    by (rule merkle-bound-table-agrees-with-authenticated-opening-if-clean
      [OF bind table-pow auth root len clean])
  then show
    opening-index opn < scale * clength  $\wedge$ 
    trace-table ! opening-index opn = opening-value opn
    using len-table by simp
  qed
  show ?thesis
    unfolding table-agrees-with-authenticated-openings-def
    using len-table opening-agrees by simp
  qed
qed
show ?thesis
  unfolding partial-trace-table-candidate-def
  using len-table len-openings agrees by simp
qed

```

lemma *accepted-bound-composition-table-partial-candidate-if-clean*:

assumes *bind*:

merkle-root-binds-table composition-root composition-table
final-state

and *len-table*: *length composition-table = scale * clength*

and *partial*:

accepted-with-partial-composition-openings s
(Some (result, final-state)) composition-root query-idxs
composition-openings

and *clean*: $\neg$ *hash-map-output-collision final-state*

shows

partial-composition-table-candidate composition-table
composition-openings

proof –

obtain *n* **where** *len-pow*: *clength * scale = 2 ^ n*

using *eval-domain-length-power* **by** *blast*

have *table-pow*: *length composition-table = 2 ^ n*

using *len-table len-pow* **by** (*simp add: mult.commute*)

have *len-openings*: *length composition-openings = rounds*

using *partial*

unfolding *accepted-with-partial-composition-openings-def* **by** *blast*

have *agrees*:

$\bigwedge i. i < rounds \implies$
table-agrees-with-authenticated-openings composition-table
*(scale * clength) (composition-openings ! i)*

proof –

fix *i*

assume *i-bound*: *i < rounds*

have *table-i*:

*partial-authenticated-table composition-root (scale * clength)*
(composition-openings ! i) final-state

using *partial i-bound*

unfolding *accepted-with-partial-composition-openings-def* **by** *blast*

show

table-agrees-with-authenticated-openings composition-table
*(scale * clength) (composition-openings ! i)*

proof –

have *opening-agrees*:

$\forall \text{opn} \in \text{set } (composition-openings ! i).$
*opening-index opn < scale * clength* $\wedge$
composition-table ! opening-index opn = opening-value opn

proof

fix *opn*

assume *opn-in*: *opn* $\in$ *set (composition-openings ! i)*

have *root*: *opening-root opn = composition-root*

using *table-i opn-in* **unfolding** *partial-authenticated-table-def*
by *blast*

have *len*: *opening-length opn = length composition-table*

using *table-i opn-in len-table*

```

unfolding partial-authenticated-table-def by simp
have auth: authenticated-opening-in final-state opn
using table-i opn-in unfolding partial-authenticated-table-def
by blast
have agreement:
  opening-index opn < length composition-table  $\wedge$ 
  composition-table ! opening-index opn = opening-value opn
by (rule merkle-bound-table-agrees-with-authenticated-opening-if-clean
  [OF bind table-pow auth root len clean])
then show
  opening-index opn < scale * clength  $\wedge$ 
  composition-table ! opening-index opn = opening-value opn
using len-table by simp
qed
show ?thesis
unfolding table-agrees-with-authenticated-openings-def
using len-table opening-agrees by simp
qed
qed
show ?thesis
unfolding partial-composition-table-candidate-def
using len-table len-openings agrees by simp
qed

```

definition *partial-merkle-inconsistency-bad*

$:: (f, 'a) \text{ protocol-channel-scheme} \Rightarrow$
 $(\text{unit list} \times (f, 'a) \text{ protocol-channel-scheme}) \text{ option} \Rightarrow \text{bool}$

where

partial-merkle-inconsistency-bad s out $\longleftrightarrow$
accepted out $\wedge$
 $(\exists \text{result final-state opn opn'.$
out = Some (result, final-state) $\wedge$
authenticated-opening-in final-state opn $\wedge$
authenticated-opening-in final-state opn' $\wedge$
opening-root opn = opening-root opn' $\wedge$
opening-length opn = opening-length opn' $\wedge$
opening-index opn = opening-index opn' $\wedge$
opening-value opn $\neq$ opening-value opn')

lemma *partial-trace-openings-inconsistent-bad-imp-partial-merkle-inconsistency-bad:*

assumes *bad: partial-trace-openings-inconsistent-bad s out*

shows *partial-merkle-inconsistency-bad s out*

proof –

obtain *fr query-idxs trace-openings result final-state i opn opn'* **where**

out: out = Some (result, final-state)

and *partial:*

accepted-with-partial-trace-openings s out fr query-idxs trace-openings

and *i-bound: i < rounds*

and *opn-in: opn $\in$ set (trace-openings ! i)*

```

and opn'-in: opn'  $\in$  set (trace-openings ! i)
and same-index: opening-index opn = opening-index opn'
and diff-value: opening-value opn  $\neq$  opening-value opn'
using bad unfolding partial-trace-openings-inconsistent-bad-def by blast
have table:
  partial-authenticated-table fr (scale * clength)
    (trace-openings ! i) final-state
proof –
  obtain result' final-state' where
    partial-out: out = Some (result', final-state')
  and tables:
     $\forall i < \text{rounds}.$ 
      partial-authenticated-table fr (scale * clength)
        (trace-openings ! i) final-state'
  using partial unfolding accepted-with-partial-trace-openings-def
  by auto
have final-state-eq: final-state' = final-state
  using out partial-out by simp
show ?thesis
  using tables i-bound unfolding final-state-eq by simp
qed
have opn-auth: authenticated-opening-in final-state opn
  using table opn-in unfolding partial-authenticated-table-def by blast
have opn'-auth: authenticated-opening-in final-state opn'
  using table opn'-in unfolding partial-authenticated-table-def by blast
have same-root: opening-root opn = opening-root opn'
proof –
  have root-opn: opening-root opn = fr
    using table opn-in unfolding partial-authenticated-table-def by simp
  have root-opn': opening-root opn' = fr
    using table opn'-in unfolding partial-authenticated-table-def by simp
  show ?thesis
    using root-opn root-opn' by simp
qed
have same-length: opening-length opn = opening-length opn'
proof –
  have length-opn: opening-length opn = scale * clength
    using table opn-in unfolding partial-authenticated-table-def by simp
  have length-opn': opening-length opn' = scale * clength
    using table opn'-in unfolding partial-authenticated-table-def by simp
  show ?thesis
    using length-opn length-opn' by simp
qed
show ?thesis
  unfolding partial-merkle-inconsistency-bad-def
  using partial out opn-auth opn'-auth same-root same-length same-index
    diff-value
  unfolding accepted-with-partial-trace-openings-def
  by blast

```

qed

lemma *partial-merkle-inconsistency-bad-imp-hash-map-output-collision-bad:*

assumes *partial-merkle-inconsistency-bad s out*

shows *hash-map-output-collision-bad s out*

proof –

obtain *result final-state opn opn'* **where**

out: out = Some (result, final-state)

and *first: authenticated-opening-in final-state opn*

and *second: authenticated-opening-in final-state opn'*

and *same-root: opening-root opn = opening-root opn'*

and *same-length: opening-length opn = opening-length opn'*

and *same-index: opening-index opn = opening-index opn'*

and *different-value:*

opening-value opn ≠ opening-value opn'

using *assms* **unfolding** *partial-merkle-inconsistency-bad-def* **by** *blast*

have *collision: hash-map-output-collision final-state*

by (*rule inconsistent-authenticated-openings-imp-hash-collision* [
 OF first second same-root same-length same-index different-value])

show *?thesis*

unfolding *hash-map-output-collision-bad-def accepted-def*

using *out collision* **by** *simp*

qed

lemma *partial-trace-openings-inconsistent-bad-imp-hash-map-output-collision-bad:*

assumes *partial-trace-openings-inconsistent-bad s out*

shows *hash-map-output-collision-bad s out*

by (*rule partial-merkle-inconsistency-bad-imp-hash-map-output-collision-bad* (
 rule partial-trace-openings-inconsistent-bad-imp-partial-merkle-inconsistency-bad [
 OF assms])

lemma *partial-trace-openings-inconsistent-bad-mono-hash-map-output-collision-bad:*

wp-event verify-monad (partial-trace-openings-inconsistent-bad s) s ≤

wp-event verify-monad (hash-map-output-collision-bad s) s

by (*rule wp-event-mono*)

(*rule partial-trace-openings-inconsistent-bad-imp-hash-map-output-collision-bad*)

lemma *partial-merkle-inconsistency-bad-mono-hash-map-output-collision-bad:*

wp-event verify-monad (partial-merkle-inconsistency-bad s) s ≤

wp-event verify-monad (hash-map-output-collision-bad s) s

by (*rule wp-event-mono*)

(*rule partial-merkle-inconsistency-bad-imp-hash-map-output-collision-bad*)

lemma *partial-trace-openings-inconsistent-bad-bound-from-hash-map-output-collision:*

fixes *H :: prob*

assumes *wp-event verify-monad (hash-map-output-collision-bad s) s ≤ H*

shows *wp-event verify-monad (partial-trace-openings-inconsistent-bad s) s ≤*
H

by (*rule order-trans*)

[OF *partial-trace-openings-inconsistent-bad-mono-hash-map-output-collision-bad*
assms])

lemma *partial-merkle-inconsistency-bad-bound-from-hash-map-output-collision*:

fixes $H :: \text{prob}$

assumes *wp-event verify-monad (hash-map-output-collision-bad s) s ≤ H*

shows *wp-event verify-monad (partial-merkle-inconsistency-bad s) s ≤ H*

by (*rule order-trans*

[OF *partial-merkle-inconsistency-bad-mono-hash-map-output-collision-bad*
assms])

lemma *partial-merkle-inconsistency-bad-imp-accepted*:

assumes *partial-merkle-inconsistency-bad s out*

shows *accepted out*

using *assms unfolding partial-merkle-inconsistency-bad-def* **by** *simp*

lemma *accepted-with-partial-trace-openings-candidate-if-no-partial-merkle-bad*:

assumes *partial*:

accepted-with-partial-trace-openings s (Some (result, final-state))

fr query-idxs trace-openings

and *no-bad*:

$\neg$ *partial-merkle-inconsistency-bad s*

(Some (result, final-state))

shows $\exists$ *trace-table*.

partial-trace-table-candidate trace-table trace-openings

proof –

have *len-openings: length trace-openings = rounds*

using *partial unfolding accepted-with-partial-trace-openings-def* **by** *blast*

have *table*:

$\bigwedge i. i < \text{rounds} \implies$

*partial-authenticated-table fr (scale * clength)*

(trace-openings ! i) final-state

using *partial unfolding accepted-with-partial-trace-openings-def* **by** *blast*

have *accepted-out: accepted (Some (result, final-state))*

by (*rule accepted-with-partial-trace-openings-imp-accepted*[OF *partial*])

have *idx-bound*:

$\bigwedge \text{opn}. \text{opn} \in \text{set } (\text{List.concat } \text{trace-openings}) \implies$

*opening-index opn < scale * clength*

proof –

fix *opn*

assume *opn-in: opn ∈ set (List.concat trace-openings)*

then obtain *openings where openings-in: openings ∈ set trace-openings*

and *opn-openings: opn ∈ set openings*

by *auto*

then obtain *i where i-bound: i < length trace-openings*

and *openings-eq: trace-openings ! i = openings*

by (*meson in-set-conv-nth*)

have *table-i*:

*partial-authenticated-table fr (scale * clength)*

```

    (trace-openings ! i) final-state
  using table i-bound len-openings by simp
have auth: authenticated-opening-in final-state opn
  using table-i opn-openings unfolding openings-eq
    partial-authenticated-table-def by blast
have len: opening-length opn = scale * clength
  using table-i opn-openings unfolding openings-eq
    partial-authenticated-table-def by blast
show opening-index opn < scale * clength
  using auth len unfolding authenticated-opening-in-def by simp
qed
have consistent:
   $\bigwedge \text{opn opn}'. \text{opn} \in \text{set (List.concat trace-openings)} \implies$ 
   $\text{opn}' \in \text{set (List.concat trace-openings)} \implies$ 
   $\text{opening-index opn} = \text{opening-index opn}' \implies$ 
   $\text{opening-value opn} = \text{opening-value opn}'$ 
proof (rule ccontr)
  fix opn opn'
  assume opn-in: opn  $\in$  set (List.concat trace-openings)
    and opn'-in: opn'  $\in$  set (List.concat trace-openings)
    and same-index: opening-index opn = opening-index opn'
    and neg: opening-value opn  $\neq$  opening-value opn'
  obtain openings where openings-in: openings  $\in$  set trace-openings
    and opn-openings: opn  $\in$  set openings
    using opn-in by auto
  then obtain i where i-bound: i < length trace-openings
    and openings-eq: trace-openings ! i = openings
    by (meson in-set-conv-nth)
  obtain openings' where openings'-in: openings'  $\in$  set trace-openings
    and opn'-openings: opn'  $\in$  set openings'
    using opn'-in by auto
  then obtain j where j-bound: j < length trace-openings
    and openings'-eq: trace-openings ! j = openings'
    by (meson in-set-conv-nth)
  have table-i:
    partial-authenticated-table fr (scale * clength)
    (trace-openings ! i) final-state
    using table i-bound len-openings by simp
  have table-j:
    partial-authenticated-table fr (scale * clength)
    (trace-openings ! j) final-state
    using table j-bound len-openings by simp
  have auth: authenticated-opening-in final-state opn
    using table-i opn-openings unfolding openings-eq
      partial-authenticated-table-def by blast
  have auth': authenticated-opening-in final-state opn'
    using table-j opn'-openings unfolding openings'-eq
      partial-authenticated-table-def by blast
  have root: opening-root opn = opening-root opn'

```



```

using table-i table-j opn-openings opn'-openings
unfolding openings-eq openings'-eq partial-authenticated-table-def
by auto
have len: opening-length opn = opening-length opn'
using table-i table-j opn-openings opn'-openings
unfolding openings-eq openings'-eq partial-authenticated-table-def
by auto
have bad:
  partial-merkle-inconsistency-bad s (Some (result, final-state))
unfolding partial-merkle-inconsistency-bad-def
by (intro conjI exI[of - result] exI[of - final-state]
  exI[of - opn] exI[of - opn'])
  (use accepted-out auth auth' root len same-index neq in simp-all)
then show False using no-bad by contradiction
qed
obtain trace-table where agrees-all:
  table-agrees-with-authenticated-openings trace-table (scale * clength)
  (List.concat trace-openings)
using table-agrees-with-authenticated-openings-consistent-exists
  [OF idx-bound consistent]
by blast
have agrees:
   $\bigwedge i. i < rounds \implies$ 
  table-agrees-with-authenticated-openings trace-table
  (scale * clength) (trace-openings ! i)
proof -
  fix i
  assume i-bound: i < rounds
  have set (trace-openings ! i)  $\subseteq$ 
    set (List.concat trace-openings)
  proof
    fix opn
    assume opn-in: opn  $\in$  set (trace-openings ! i)
    have trace-openings ! i  $\in$  set trace-openings
      using i-bound len-openings by (intro nth-mem) simp
    then show opn  $\in$  set (List.concat trace-openings)
      using opn-in by auto
  qed
then show
  table-agrees-with-authenticated-openings trace-table
  (scale * clength) (trace-openings ! i)
  by (rule table-agrees-with-authenticated-openings-subset
    [OF agrees-all])
qed
have len-table: length trace-table = scale * clength
  using agrees-all
  unfolding table-agrees-with-authenticated-openings-def by simp
have partial-trace-table-candidate trace-table trace-openings
  unfolding partial-trace-table-candidate-def

```

```

    using len-table len-openings agrees by simp
  then show ?thesis by blast
qed

lemma accepted-with-partial-composition-openings-candidate-if-no-partial-merkle-bad:
  assumes partial:
    accepted-with-partial-composition-openings s
    (Some (result, final-state)) composition-root query-idxs
    composition-openings
  and no-bad:
    ¬ partial-merkle-inconsistency-bad s
    (Some (result, final-state))
  shows ∃ composition-table.
    partial-composition-table-candidate composition-table composition-openings
proof -
  have len-openings: length composition-openings = rounds
  using partial
  unfolding accepted-with-partial-composition-openings-def by blast
  have table:
    ∧i. i < rounds ⇒
      partial-authenticated-table composition-root (scale * clength)
      (composition-openings ! i) final-state
  using partial
  unfolding accepted-with-partial-composition-openings-def by blast
  have accepted-out: accepted (Some (result, final-state))
  by (rule accepted-with-partial-composition-openings-imp-accepted
    [OF partial])
  have idx-bound:
    ∧opn. opn ∈ set (List.concat composition-openings) ⇒
      opening-index opn < scale * clength
  proof -
    fix opn
    assume opn-in: opn ∈ set (List.concat composition-openings)
    then obtain openings where openings-in:
      openings ∈ set composition-openings
    and opn-openings: opn ∈ set openings
    by auto
    then obtain i where i-bound: i < length composition-openings
    and openings-eq: composition-openings ! i = openings
    by (meson in-set-conv-nth)
    have table-i:
      partial-authenticated-table composition-root (scale * clength)
      (composition-openings ! i) final-state
    using table i-bound len-openings by simp
    have auth: authenticated-opening-in final-state opn
    using table-i opn-openings unfolding openings-eq
      partial-authenticated-table-def by blast
    have len: opening-length opn = scale * clength
    using table-i opn-openings unfolding openings-eq

```

```

    partial-authenticated-table-def by blast
  show opening-index opn < scale * clength
    using auth len unfolding authenticated-opening-in-def by simp
qed
have consistent:
   $\bigwedge \text{opn opn}'. \text{opn} \in \text{set (List.concat composition-openings)} \implies$ 
     $\text{opn}' \in \text{set (List.concat composition-openings)} \implies$ 
     $\text{opening-index opn} = \text{opening-index opn}' \implies$ 
     $\text{opening-value opn} = \text{opening-value opn}'$ 
proof (rule ccontr)
  fix opn opn'
  assume opn-in: opn  $\in$  set (List.concat composition-openings)
    and opn'-in: opn'  $\in$  set (List.concat composition-openings)
    and same-index: opening-index opn = opening-index opn'
    and neg: opening-value opn  $\neq$  opening-value opn'
  obtain openings where openings-in: openings  $\in$  set composition-openings
    and opn-openings: opn  $\in$  set openings
    using opn-in by auto
  then obtain i where i-bound: i < length composition-openings
    and openings-eq: composition-openings ! i = openings
    by (meson in-set-conv-nth)
  obtain openings' where openings'-in:
    openings'  $\in$  set composition-openings
    and opn'-openings: opn'  $\in$  set openings'
    using opn'-in by auto
  then obtain j where j-bound: j < length composition-openings
    and openings'-eq: composition-openings ! j = openings'
    by (meson in-set-conv-nth)
  have table-i:
    partial-authenticated-table composition-root (scale * clength)
      (composition-openings ! i) final-state
    using table i-bound len-openings by simp
  have table-j:
    partial-authenticated-table composition-root (scale * clength)
      (composition-openings ! j) final-state
    using table j-bound len-openings by simp
  have auth: authenticated-opening-in final-state opn
    using table-i opn-openings unfolding openings-eq
      partial-authenticated-table-def by blast
  have auth': authenticated-opening-in final-state opn'
    using table-j opn'-openings unfolding openings'-eq
      partial-authenticated-table-def by blast
  have root: opening-root opn = opening-root opn'
    using table-i table-j opn-openings opn'-openings
      unfolding openings-eq openings'-eq partial-authenticated-table-def
    by auto
  have len: opening-length opn = opening-length opn'
    using table-i table-j opn-openings opn'-openings
      unfolding openings-eq openings'-eq partial-authenticated-table-def

```

```

    by auto
  have bad:
    partial-merkle-inconsistency-bad s (Some (result, final-state))
    unfolding partial-merkle-inconsistency-bad-def
    by (intro conjI exI[of - result] exI[of - final-state]
        exI[of - opn] exI[of - opn'])
        (use accepted-out auth auth' root len same-index neq in simp-all)
    then show False using no-bad by contradiction
qed
obtain composition-table where agrees-all:
  table-agrees-with-authenticated-openings composition-table
  (scale * clength) (List.concat composition-openings)
using table-agrees-with-authenticated-openings-consistent-exists
  [OF idx-bound consistent]
by blast
have agrees:
   $\bigwedge i. i < \text{rounds} \implies$ 
  table-agrees-with-authenticated-openings composition-table
  (scale * clength) (composition-openings ! i)
proof -
  fix i
  assume i-bound:  $i < \text{rounds}$ 
  have set (composition-openings ! i)  $\subseteq$ 
    set (List.concat composition-openings)
  proof
    fix opn
    assume opn-in:  $\text{opn} \in \text{set } (\text{composition-openings ! } i)$ 
    have composition-openings ! i  $\in \text{set } \text{composition-openings}$ 
      using i-bound len-openings by (intro nth-mem) simp
    then show  $\text{opn} \in \text{set } (\text{List.concat } \text{composition-openings})$ 
      using opn-in by auto
  qed
  then show
    table-agrees-with-authenticated-openings composition-table
    (scale * clength) (composition-openings ! i)
    by (rule table-agrees-with-authenticated-openings-subset
        [OF agrees-all])
qed
have len-table:  $\text{length } \text{composition-table} = \text{scale} * \text{clength}$ 
  using agrees-all
  unfolding table-agrees-with-authenticated-openings-def by simp
have partial-composition-table-candidate composition-table
  composition-openings
  unfolding partial-composition-table-candidate-def
  using len-table len-openings agrees by simp
then show ?thesis by blast
qed

lemma accepted-with-partial-initial-openings-candidates-if-no-partial-merkle-bad:

```

assumes *partial*:
accepted-with-partial-initial-openings s
(Some (result, final-state)) fr f-fri-roots f-final as dg
composition-fri-roots final trace-query-idxs trace-openings
composition-query-idxs composition-openings
and *no-bad*:
 $\neg$ *partial-merkle-inconsistency-bad s*
(Some (result, final-state))
shows $\exists$ *trace-table composition-table*.
partial-trace-table-candidate trace-table trace-openings $\wedge$
partial-composition-table-candidate composition-table
composition-openings
proof –
have *trace-partial*:
accepted-with-partial-trace-openings s
(Some (result, final-state)) fr trace-query-idxs trace-openings
using *accepted-with-partial-initial-openings-shapes(3)[OF partial]* .
have *comp-partial*:
accepted-with-partial-composition-openings s
(Some (result, final-state)) (hd composition-fri-roots)
composition-query-idxs composition-openings
using *accepted-with-partial-initial-openings-shapes(4)[OF partial]* .
obtain *trace-table* **where** *trace-candidate*:
partial-trace-table-candidate trace-table trace-openings
using *accepted-with-partial-trace-openings-candidate-if-no-partial-merkle-bad*
 $[OF \text{ trace-partial no-bad}]$
by *blast*
obtain *composition-table* **where** *comp-candidate*:
partial-composition-table-candidate composition-table
composition-openings
using
accepted-with-partial-composition-openings-candidate-if-no-partial-merkle-bad
 $[OF \text{ comp-partial no-bad}]$
by *blast*
show *?thesis*
using *trace-candidate comp-candidate* **by** *blast*
qed

lemma *verify-monad-partial-initial-candidates-or-partial-merkle-bad*:
fixes *s :: ('f, 'a) protocol-channel-scheme*
assumes *outcome*:
Some (result, final-state) $\in$ set-dist (execute verify-monad s)
and *header*:
verifier-header-transcript s fr f-fri-roots f-final as dg
composition-fri-roots final rest
and *comp-nonempty*: *composition-fri-roots $\neq []$*
shows
partial-merkle-inconsistency-bad s (Some (result, final-state)) $\vee$
($\exists$ trace-query-idxs trace-openings composition-query-idxs

```

    composition-openings trace-table composition-table.
  accepted-with-partial-initial-openings s
  (Some (result, final-state)) fr f-fri-roots f-final as dg
  composition-fri-roots final trace-query-idxs trace-openings
  composition-query-idxs composition-openings ∧
  partial-trace-table-candidate trace-table trace-openings ∧
  partial-composition-table-candidate composition-table
  composition-openings)
proof (cases
  partial-merkle-inconsistency-bad s (Some (result, final-state)))
case True
then show ?thesis by simp
next
case False
from verify-monad-accepted-with-partial-initial-openings
  [OF outcome header comp-nonempty]
obtain trace-query-idxs trace-openings composition-query-idxs
  composition-openings where partial:
  accepted-with-partial-initial-openings s
  (Some (result, final-state)) fr f-fri-roots f-final as dg
  composition-fri-roots final trace-query-idxs trace-openings
  composition-query-idxs composition-openings
by blast
obtain trace-table composition-table where candidates:
  partial-trace-table-candidate trace-table trace-openings
  partial-composition-table-candidate composition-table
  composition-openings
using accepted-with-partial-initial-openings-candidates-if-no-partial-merkle-bad
  [OF partial False]
by blast
show ?thesis
using partial candidates by blast
qed

lemma partial-merkle-inconsistency-bad-mono-accepted:
  wp-event verify-monad (partial-merkle-inconsistency-bad s) s ≤
  wp-event verify-monad accepted s
by (rule wp-event-mono)
  (rule partial-merkle-inconsistency-bad-imp-accepted)

end

end

```

```

theory Soundness-Reductions-Trace
imports Soundness-Partial-Merkle
begin

```

FRI-table acceptance, transcript-target, and trace-table candidate reductions.

context *soundness*
begin

definition *accepted-with-fri-tables*
 $:: (f, 'a) \text{ protocol-channel-scheme} \Rightarrow$
 $(\text{unit list} \times (f, 'a) \text{ protocol-channel-scheme}) \text{ option} \Rightarrow \text{bool}$
where
 $\text{accepted-with-fri-tables } s \text{ out} \longleftrightarrow$
 $\text{accepted out} \wedge \text{fri-merkle-binding-good } s \text{ out}$

lemma *accepted-with-fri-tables-imp-accepted*:
assumes *accepted-with-fri-tables s out*
shows *accepted out*
using *assms unfolding accepted-with-fri-tables-def* **by** *simp*

lemma *accepted-with-fri-tablesE*:
assumes *tables: accepted-with-fri-tables s out*
obtains *trace-table composition-table as query-idxs*
 $\text{trace-roots trace-bs trace-final dg composition-roots composition-bs}$
 $\text{composition-final trace-round-layers composition-round-layers}$
 $\text{trace-tables composition-tables trace-final-table composition-final-table}$
where
 $\text{accepted-with-bound-tables } s \text{ out trace-table composition-table as}$
 query-idxs
and $\text{accepted-fri-opening-transcript } s \text{ out trace-roots trace-bs}$
 $\text{trace-final dg composition-roots composition-bs composition-final}$
 $\text{query-idxs trace-round-layers composition-round-layers}$
and $\text{trace-fri-tables-bound } s \text{ out trace-roots trace-bs trace-final}$
 $\text{query-idxs trace-round-layers trace-tables trace-final-table}$
and $\text{composition-fri-tables-bound } s \text{ out dg composition-roots}$
 $\text{composition-bs composition-final query-idxs composition-round-layers}$
 $\text{composition-tables composition-final-table}$
and $\text{fri-initial-table-agrees trace-table trace-tables}$
and $\text{fri-initial-table-agrees composition-table composition-tables}$
using *tables*
unfolding *accepted-with-fri-tables-def fri-merkle-binding-good-def*
by (*blast intro: that*)

lemma *accepted-with-fri-tables-obtain-bound-tables*:
assumes *accepted-with-fri-tables s out*
obtains *trace-table composition-table as query-idxs*
where *accepted-with-bound-tables s out trace-table composition-table as*
 query-idxs
using *assms* **by** (*elim accepted-with-fri-tablesE*) *blast*

lemma *accepted-partition-fri-merkle-binding*:
assumes *accepted out*
shows $\text{fri-merkle-binding-bad } s \text{ out} \vee \text{accepted-with-fri-tables } s \text{ out}$
using *assms*

unfolding *fri-merkle-binding-bad-def accepted-with-fri-tables-def*
by *blast*

lemma *accepted-with-bound-tables-trace-root-target-hit:*

assumes *bound:*

accepted-with-bound-tables (verifier-initial-state tr)
(Some (result, final-state)) trace-table composition-table as
query-idxs

shows

hash-map-new-output-hit (set tr) (verifier-initial-state tr) final-state

proof –

from *bound* **obtain** *fr f-fri-roots f-final dg composition-fri-roots final rest*

where *header:*

verifier-header-transcript (verifier-initial-state tr) fr f-fri-roots
f-final as dg composition-fri-roots final rest

and *bind:*

merkle-root-binds-table fr trace-table final-state

unfolding *accepted-with-bound-tables-def* **by** *blast*

have *table-nonempty: trace-table ≠ []*

using *accepted-with-bound-tables-shapes(1)[OF bound]*

eval-domain-nontrivial **by** *auto*

from *merkle-root-binds-nonempty-table-output-lookup[OF bind table-nonempty]*

obtain *input* **where** *lookup:*

fmlookup (HashMap final-state) input = Some fr

by *blast*

have *root-in: fr ∈ set tr*

using *header*

unfolding *verifier-header-transcript-def verifier-header-messages-def*

by *simp*

have *fresh:*

fmlookup (HashMap (verifier-initial-state tr)) input = None

by *simp*

show *?thesis*

unfolding *hash-map-new-output-hit-def*

using *fresh lookup root-in* **by** *blast*

qed

lemma *accepted-with-fri-tables-imp-transcript-target-hit:*

assumes *tables:*

accepted-with-fri-tables (verifier-initial-state tr) out

shows

hash-new-output-hit-event (set tr) (verifier-initial-state tr) out

proof –

from *accepted-with-fri-tables-obtain-bound-tables[OF tables]*

obtain *trace-table composition-table as query-idxs* **where** *bound:*

accepted-with-bound-tables (verifier-initial-state tr) out trace-table
composition-table as query-idxs

by *blast*

from *accepted-with-bound-tables-imp-accepted[OF bound]*


```

obtain result final-state where out-eq: out = Some (result, final-state)
  unfolding accepted-def by (cases out) auto
have hit:
  hash-map-new-output-hit (set tr) (verifier-initial-state tr) final-state
  by (rule accepted-with-bound-tables-trace-root-target-hit)
    (use bound out-eq in simp)
show ?thesis
  unfolding hash-new-output-hit-event-def out-eq using hit by simp
qed

```

```

lemma accepted-imp-fri-merkle-binding-bad-or-transcript-target-hit:
  assumes acc: accepted out
  shows
    fri-merkle-binding-bad (verifier-initial-state tr) out  $\vee$ 
    hash-new-output-hit-event (set tr) (verifier-initial-state tr) out
proof –
  from accepted-partition-fri-merkle-binding
    [of out verifier-initial-state tr, OF acc]
  show ?thesis
proof
  assume fri-merkle-binding-bad (verifier-initial-state tr) out
  then show ?thesis by simp
next
  assume tables:
    accepted-with-fri-tables (verifier-initial-state tr) out
  have hash-new-output-hit-event (set tr) (verifier-initial-state tr) out
    by (rule accepted-with-fri-tables-imp-transcript-target-hit[OF tables])
  then show ?thesis by simp
qed
qed

```

```

lemma verify-monad-verifier-initial-trace-root-target-hit:
  assumes outcome:
    Some (result, final-state)  $\in$ 
    set-dist (execute verify-monad (verifier-initial-state tr))
  shows
    hash-map-new-output-hit (set tr) (verifier-initial-state tr)
    final-state
proof –
  let ?s = verifier-initial-state tr
  from verify-monad-header-extraction[OF outcome]
  obtain fr f-fl f-final as dg fl final query-state where
    header:
      verifier-header-transcript ?s fr (map snd f-fl) f-final as dg
      (map snd fl) final (PTranscript query-state)
  and query-out:
    Some (result, final-state)  $\in$ 
    set-dist
    (execute

```

```

      (ntimes
        (verifier-query-round-program fr f-fl f-final as fl final)
        rounds)
      query-state)
  by (rule verify-monad-header-query-extraction[OF outcome])
obtain query-idxs trace-openings where
  len-query-idxs: length query-idxs = rounds
and len-openings: length trace-openings = rounds
and indices:
   $\bigwedge i. i < \text{rounds} \implies$ 
    map opening-index (trace-openings ! i) =
      powers-scaled (query-idxs ! i)
and tables:
   $\bigwedge i. i < \text{rounds} \implies$ 
    partial-authenticated-table fr (scale * clength)
      (trace-openings ! i) final-state
proof (rule ntimes-verifier-query-rounds-authenticated-trace-openings-no-rows
  [OF query-out])
  fix query-idxs trace-openings
  assume len-query-idxs: length query-idxs = rounds
  and len-openings: length trace-openings = rounds
  and indices:
     $\bigwedge i. i < \text{rounds} \implies$ 
      map opening-index (trace-openings ! i) =
        powers-scaled (query-idxs ! i)
  and tables:
     $\bigwedge i. i < \text{rounds} \implies$ 
      partial-authenticated-table fr (scale * clength)
        (trace-openings ! i) final-state
  show ?thesis
    by (rule that[OF len-query-idxs len-openings indices tables])
qed
have round0: 0 < rounds
  by (rule rounds-positive)
have openings0-nonempty:
  trace-openings ! 0  $\neq$  []
proof -
  have map opening-index (trace-openings ! 0) =
    powers-scaled (query-idxs ! 0)
  by (rule indices[OF round0])
  moreover have powers-scaled (query-idxs ! 0)  $\neq$  []
    using powers-pos unfolding powers-scaled-def by simp
  ultimately show ?thesis
    by auto
qed
then obtain opn where opn-in:
  opn  $\in$  set (trace-openings ! 0)
  by (cases trace-openings ! 0) auto
have table0:

```

```

    partial-authenticated-table fr (scale * clength)
      (trace-openings ! 0) final-state
    using tables[OF round0] by simp
  have opn-root: opening-root opn = fr
    using table0 opn-in unfolding partial-authenticated-table-def by blast
  have opn-auth: authenticated-opening-in final-state opn
    using table0 opn-in unfolding partial-authenticated-table-def by blast
  from authenticated-opening-root-lookup[OF opn-auth]
  obtain input where lookup:
    fmlookup (HashMap final-state) input = Some fr
    using opn-root by auto
  have root-in: fr ∈ set tr
    using header
    unfolding verifier-header-transcript-def verifier-header-messages-def
    by simp
  have fresh: fmlookup (HashMap ?s) input = None
    by simp
  show ?thesis
    unfolding hash-map-new-output-hit-def
    using fresh lookup root-in by blast
qed

```

```

lemma verify-monad-trace-root-target-preexisting-or-new:
  assumes outcome:
    Some (result, final-state) ∈ set-dist (execute verify-monad s)
  and tr-eq: PTranscript s = tr
  shows
    hash-map-output-values s ∩ set tr ≠ {} ∨
    hash-map-new-output-hit (set tr) s final-state
proof -
  from verify-monad-header-extraction[OF outcome]
  obtain fr f-fl f-final as dg fl final query-state where
    header:
      verifier-header-transcript s fr (map snd f-fl) f-final as dg
      (map snd fl) final (PTranscript query-state)
  and query-out:
    Some (result, final-state) ∈
      set-dist
        (execute
          (ntimes
            (verifier-query-round-program fr f-fl f-final as fl final)
            rounds)
          query-state)
  by (rule verify-monad-header-query-extraction[OF outcome])
  obtain query-idxs trace-openings where
    indices:
       $\bigwedge i. i < rounds \implies$ 
      map opening-index (trace-openings ! i) =
      powers-scaled (query-idxs ! i)

```

```

and tables:
   $\bigwedge i. i < \text{rounds} \implies$ 
    partial-authenticated-table fr (scale * clength)
      (trace-openings ! i) final-state
proof (rule ntimes-verifier-query-rounds-authenticated-trace-openings-no-raws
  [OF query-out])
fix query-idxs trace-openings
assume indices:
   $\bigwedge i. i < \text{rounds} \implies$ 
    map opening-index (trace-openings ! i) =
      powers-scaled (query-idxs ! i)
and tables:
   $\bigwedge i. i < \text{rounds} \implies$ 
    partial-authenticated-table fr (scale * clength)
      (trace-openings ! i) final-state
show ?thesis
  by (rule that[OF indices tables])
qed
have round0:  $0 < \text{rounds}$ 
  by (rule rounds-positive)
have openings0-nonempty:
  trace-openings !  $0 \neq []$ 
proof –
  have map opening-index (trace-openings !  $0$ ) =
    powers-scaled (query-idxs !  $0$ )
    by (rule indices[OF round0])
  moreover have powers-scaled (query-idxs !  $0$ )  $\neq []$ 
    using powers-pos unfolding powers-scaled-def by simp
  ultimately show ?thesis
    by auto
qed
then obtain opn where opn-in:
  opn  $\in \text{set } (\text{trace-openings} ! 0)$ 
  by (cases trace-openings !  $0$ ) auto
have table0:
  partial-authenticated-table fr (scale * clength)
    (trace-openings !  $0$ ) final-state
  using tables[OF round0] by simp
have opn-root: opening-root opn = fr
  using table0 opn-in unfolding partial-authenticated-table-def by blast
have opn-auth: authenticated-opening-in final-state opn
  using table0 opn-in unfolding partial-authenticated-table-def by blast
from authenticated-opening-root-lookup[OF opn-auth]
obtain input where lookup-final:
  fmlookup (HashMap final-state) input = Some fr
  using opn-root by auto
have root-in: fr  $\in \text{set } \text{tr}$ 
  using header tr-eq
  unfolding verifier-header-transcript-def verifier-header-messages-def

```

```

    by simp
  have ext:  $s \leq \text{final-state}$ 
    by (rule verify-monad-hash-extends[OF outcome])
  show ?thesis
  proof (cases fmlookup (HashMap s) input)
    case None
    then have hash-map-new-output-hit (set tr) s final-state
      unfolding hash-map-new-output-hit-def
      using lookup-final root-in by blast
    then show ?thesis by simp
  next
    case (Some z)
    have lookup-final-z: fmlookup (HashMap final-state) input = Some z
      by (rule hash-extension-lookup[OF Some ext])
    then have z = fr
      using lookup-final by simp
    then have fr  $\in$  hash-map-output-values s
      using Some by (auto intro: hash-map-output-valuesI)
    then show ?thesis
      using root-in by blast
  qed
qed

```

definition *soundness-bad-event-with-merkle*
 $:: (f, 'a) \text{ protocol-channel-scheme} \Rightarrow$
 $(\text{unit list} \times (f, 'a) \text{ protocol-channel-scheme}) \text{ option} \Rightarrow \text{bool}$
where
 $\text{soundness-bad-event-with-merkle } s \text{ out} \longleftrightarrow$
 $\text{fri-merkle-binding-bad } s \text{ out} \vee \text{soundness-bad-event } s \text{ out}$

lemma *soundness-bad-event-imp-soundness-bad-event-with-merkle*:
assumes *soundness-bad-event s out*
shows *soundness-bad-event-with-merkle s out*
using *assms* **unfolding** *soundness-bad-event-with-merkle-def* **by** *simp*

lemma *fri-merkle-binding-bad-imp-soundness-bad-event-with-merkle*:
assumes *fri-merkle-binding-bad s out*
shows *soundness-bad-event-with-merkle s out*
using *assms* **unfolding** *soundness-bad-event-with-merkle-def* **by** *simp*

lemma *accepted-with-fri-tables-table-partition*:
assumes *false-statement: $\neg \text{exists-valid-trace}$*
and *tables: accepted-with-fri-tables s out*
shows *soundness-bad-event s out*
proof –
from *accepted-with-fri-tables-obtain-bound-tables[OF tables]*
obtain *trace-table composition-table as query-idxs* **where** *bound:*
 $\text{accepted-with-bound-tables } s \text{ out } \text{trace-table } \text{composition-table as}$
 query-idxs

```

    by blast
  have partition:
    composition-bad s out  $\vee$  trace-fri-bad s out  $\vee$ 
    composition-fri-bad s out  $\vee$  query-bad s out
  by (rule accepted-bound-table-partition[OF false-statement bound])
  then show ?thesis
    unfolding soundness-bad-event-def by blast
qed

lemma accepted-partition-soundness-bad-event-with-merkle:
  assumes false-statement:  $\neg$  exists-valid-trace
  and acc: accepted out
  shows soundness-bad-event-with-merkle s out
proof -
  from accepted-partition-fri-merkle-binding[OF acc]
  consider
    (merkle-bad) fri-merkle-binding-bad s out
  | (with-tables) accepted-with-fri-tables s out
  by blast
  then show ?thesis
proof cases
  case merkle-bad
  then show ?thesis
    by (rule fri-merkle-binding-bad-imp-soundness-bad-event-with-merkle)
next
  case with-tables
  have soundness-bad-event s out
  by (rule accepted-with-fri-tables-table-partition
    [OF false-statement with-tables])
  then show ?thesis
    by (rule soundness-bad-event-imp-soundness-bad-event-with-merkle)
qed
qed

```

```

definition trace-fri-challenge-set-hit
:: ('f, 'a) protocol-channel-scheme  $\Rightarrow$ 
  ('f list  $\Rightarrow$  'f list set)  $\Rightarrow$ 
  (unit list  $\times$  ('f, 'a) protocol-channel-scheme) option  $\Rightarrow$  bool
where
  trace-fri-challenge-set-hit s bad-sets out  $\longleftrightarrow$ 
    ( $\exists$  trace-table composition-table as query-idxs trace-bs dg comp-bs.
      accepted-with-bound-tables s out trace-table composition-table as query-idxs  $\wedge$ 
      accepted-fri-challenges s out trace-bs dg comp-bs  $\wedge$ 
       $\neg$  trace-table-low-degree trace-table  $\wedge$ 
      trace-bs  $\in$  bad-sets trace-table)

```

```

definition composition-fri-challenge-set-hit
:: ('f, 'a) protocol-channel-scheme  $\Rightarrow$ 
  ('f  $\Rightarrow$  'f list  $\Rightarrow$  'f list set)  $\Rightarrow$ 

```

(unit list $\times$ ('f, 'a) protocol-channel-scheme) option $\Rightarrow$ bool
where
 composition-fri-challenge-set-hit s bad-sets out $\longleftrightarrow$
 ($\exists$ trace-table composition-table as query-idxs trace-bs dg comp-bs.
 accepted-with-bound-tables s out trace-table composition-table as query-idxs $\wedge$
 accepted-fri-challenges s out trace-bs dg comp-bs $\wedge$
 trace-table-low-degree trace-table $\wedge$
 $\neg$ composition-table-low-degree maxDegree composition-table $\wedge$
 comp-bs $\in$ bad-sets dg composition-table)
lemma trace-fri-challenge-set-hit-imp-list-set-hit:
assumes hit: trace-fri-challenge-set-hit s bad-sets out
and envelope: $\bigwedge$ trace-table. bad-sets trace-table $\subseteq B$
shows trace-fri-challenge-list-set-hit s B out
proof –
from hit **obtain** trace-table composition-table as query-idxs trace-bs dg comp-bs
where challenges:
 accepted-fri-challenges s out trace-bs dg comp-bs
and trace-bad: trace-bs $\in$ bad-sets trace-table
unfolding trace-fri-challenge-set-hit-def **by** blast
have trace-bs $\in B$
using trace-bad envelope[of trace-table] **by** auto
then show ?thesis
unfolding trace-fri-challenge-list-set-hit-def
using challenges **by** blast
qed

definition trace-fri-root-trace-table-candidates
 $::$ ('f, 'a) protocol-channel-scheme $\Rightarrow$ 'f $\Rightarrow$ 'f list set
where
 trace-fri-root-trace-table-candidates s fr =
 {trace-table.
 $\exists$ out composition-table as query-idxs trace-bs dg comp-bs
 f-fri-roots f-final header-as composition-fri-roots final rest.
 accepted-with-bound-tables s out trace-table composition-table
 as query-idxs $\wedge$
 accepted-fri-challenges s out trace-bs dg comp-bs $\wedge$
 verifier-header-transcript s fr f-fri-roots f-final header-as dg
 composition-fri-roots final rest}

definition trace-fri-supported-root-trace-table-candidates
 $::$ ('f, 'a) protocol-channel-scheme $\Rightarrow$ 'f $\Rightarrow$ 'f list set
where
 trace-fri-supported-root-trace-table-candidates s fr =
 {trace-table.
 $\exists$ out composition-table as query-idxs trace-bs dg comp-bs
 f-fri-roots f-final header-as composition-fri-roots final rest.
 out $\in$ set-dist (execute verify-monad s) $\wedge$
 accepted-with-bound-tables s out trace-table composition-table

$as\ query\ idxs \wedge$
 $accepted\ fri\ challenges\ s\ out\ trace\ bs\ dg\ comp\ bs \wedge$
 $verifier\ header\ transcript\ s\ fr\ f\ fri\ roots\ f\ final\ header\ as\ dg$
 $composition\ fri\ roots\ final\ rest\}$

definition *trace-fri-supported-root-partial-trace-table-candidates*

$:: (f, 'a)\ protocol\ channel\ scheme \Rightarrow f \Rightarrow f\ list\ set$

where

$trace\ fri\ supported\ root\ partial\ trace\ table\ candidates\ s\ fr =$
 $\{trace\ table.$
 $\exists out\ query\ idxs\ trace\ openings\ trace\ bs\ dg\ comp\ bs$
 $f\ fri\ roots\ f\ final\ header\ as\ composition\ fri\ roots\ final\ rest.$
 $out \in set\ dist\ (execute\ verify\ monad\ s) \wedge$
 $accepted\ with\ partial\ trace\ openings\ s\ out\ fr\ query\ idxs$
 $trace\ openings \wedge$
 $partial\ trace\ table\ candidate\ trace\ table\ trace\ openings \wedge$
 $accepted\ fri\ challenges\ s\ out\ trace\ bs\ dg\ comp\ bs \wedge$
 $verifier\ header\ transcript\ s\ fr\ f\ fri\ roots\ f\ final\ header\ as\ dg$
 $composition\ fri\ roots\ final\ rest\}$

definition *trace-fri-supported-root-partial-trace-table-candidate-state*

$:: (f, 'a)\ protocol\ channel\ scheme \Rightarrow f \Rightarrow f\ list \Rightarrow$

$(f, 'a)\ protocol\ channel\ scheme \Rightarrow bool$

where

$trace\ fri\ supported\ root\ partial\ trace\ table\ candidate\ state\ s\ fr$
 $trace\ table\ final\ state \longleftrightarrow$
 $(\exists result\ query\ idxs\ trace\ openings\ trace\ bs\ dg\ comp\ bs$
 $f\ fri\ roots\ f\ final\ header\ as\ composition\ fri\ roots\ final\ rest.$
 $Some\ (result,\ final\ state) \in set\ dist\ (execute\ verify\ monad\ s) \wedge$
 $accepted\ with\ partial\ trace\ openings\ s$
 $(Some\ (result,\ final\ state))\ fr\ query\ idxs\ trace\ openings \wedge$
 $partial\ trace\ table\ candidate\ trace\ table\ trace\ openings \wedge$
 $accepted\ fri\ challenges\ s\ (Some\ (result,\ final\ state))\ trace\ bs\ dg$
 $comp\ bs \wedge$
 $verifier\ header\ transcript\ s\ fr\ f\ fri\ roots\ f\ final\ header\ as\ dg$
 $composition\ fri\ roots\ final\ rest)$

definition *alpha-header-supported-partial-trace-table-candidates*

$:: (f, 'a)\ protocol\ channel\ scheme \Rightarrow$

$f \Rightarrow f\ list \Rightarrow f \Rightarrow f\ list\ set$

where

$alpha\ header\ supported\ partial\ trace\ table\ candidates\ s\ fr\ f\ fri\ roots$
 $f\ final =$
 $\{trace\ table.$
 $\exists out\ query\ idxs\ trace\ openings\ as\ dg\ composition\ fri\ roots\ final\ rest.$
 $out \in set\ dist\ (execute\ verify\ monad\ s) \wedge$
 $accepted\ with\ partial\ trace\ openings\ s\ out\ fr\ query\ idxs$
 $trace\ openings \wedge$
 $partial\ trace\ table\ candidate\ trace\ table\ trace\ openings \wedge$

*verifier-header-transcript s fr f-fri-roots f-final as dg
composition-fri-roots final rest}*

lemma *alpha-header-supported-partial-trace-table-candidatesE:*

assumes

trace-table $\in$

*alpha-header-supported-partial-trace-table-candidates s fr f-fri-roots
f-final*

obtains *out query-idxs trace-openings as dg composition-fri-roots final rest*

where

out $\in$ *set-dist* (*execute verify-monad s*)

accepted-with-partial-trace-openings s out fr query-idxs

trace-openings

partial-trace-table-candidate trace-table trace-openings

verifier-header-transcript s fr f-fri-roots f-final as dg

composition-fri-roots final rest

using *assms*

unfolding *alpha-header-supported-partial-trace-table-candidates-def*

by *blast*

definition *alpha-header-supported-partial-union-bad-sets*

$:: (f, 'a) \text{ protocol-channel-scheme} \Rightarrow$

$(f \text{ list} \Rightarrow 'f \text{ list set}) \Rightarrow$

$'f \Rightarrow 'f \text{ list} \Rightarrow 'f \Rightarrow 'f \text{ list set}$

where

alpha-header-supported-partial-union-bad-sets s bad-sets fr

f-fri-roots f-final =

$\{as \in \text{alpha-space}.$

$\exists \text{ trace-table} \in$

alpha-header-supported-partial-trace-table-candidates s fr

f-fri-roots f-final.

$as \in \text{bad-sets trace-table}\}$

definition *trace-fri-root-trace-table-candidate-state*

$:: (f, 'a) \text{ protocol-channel-scheme} \Rightarrow 'f \Rightarrow 'f \text{ list} \Rightarrow$

$(f, 'a) \text{ protocol-channel-scheme} \Rightarrow \text{bool}$

where

trace-fri-root-trace-table-candidate-state s fr trace-table

final-state $\longleftrightarrow$

$(\exists \text{ result composition-table as query-idxs trace-bs dg comp-bs}$

f-fri-roots f-final header-as composition-fri-roots final rest.

accepted-with-bound-tables s (Some (result, final-state))

trace-table composition-table as query-idxs $\wedge$

accepted-fri-challenges s (Some (result, final-state)) trace-bs dg

comp-bs $\wedge$

verifier-header-transcript s fr f-fri-roots f-final header-as dg

composition-fri-roots final rest $\wedge$

merkle-root-binds-table fr trace-table final-state)

definition *trace-fri-supported-root-trace-table-candidate-state*
 $:: ('f, 'a) \text{ protocol-channel-scheme} \Rightarrow 'f \Rightarrow 'f \text{ list} \Rightarrow$
 $('f, 'a) \text{ protocol-channel-scheme} \Rightarrow \text{bool}$
where
trace-fri-supported-root-trace-table-candidate-state s fr trace-table
final-state $\longleftrightarrow$
 $(\exists \text{ result composition-table as query-idxs trace-bs dg comp-bs}$
 $\text{ f-fri-roots f-final header-as composition-fri-roots final rest.}$
 $\text{Some (result, final-state)} \in \text{set-dist (execute verify-monad s)} \wedge$
 $\text{accepted-with-bound-tables s (Some (result, final-state))}$
 $\text{trace-table composition-table as query-idxs} \wedge$
 $\text{accepted-fri-challenges s (Some (result, final-state)) trace-bs dg}$
 $\text{comp-bs} \wedge$
 $\text{verifier-header-transcript s fr f-fri-roots f-final header-as dg}$
 $\text{composition-fri-roots final rest} \wedge$
 $\text{merkle-root-binds-table fr trace-table final-state})$

lemma *trace-fri-root-trace-table-candidates-iff-state*:
 $\text{trace-table} \in \text{trace-fri-root-trace-table-candidates s fr} \longleftrightarrow$
 $(\exists \text{ final-state.}$
 $\text{trace-fri-root-trace-table-candidate-state s fr trace-table}$
 $\text{final-state})$

proof

assume *candidate*:

$\text{trace-table} \in \text{trace-fri-root-trace-table-candidates s fr}$

then obtain *out composition-table as query-idxs trace-bs dg comp-bs*
f-fri-roots f-final header-as composition-fri-roots final rest **where**
bound:

$\text{accepted-with-bound-tables s out trace-table composition-table as}$
 query-idxs

and *challenges*:

$\text{accepted-fri-challenges s out trace-bs dg comp-bs}$

and *header*:

$\text{verifier-header-transcript s fr f-fri-roots f-final header-as dg}$
 $\text{composition-fri-roots final rest}$

unfolding *trace-fri-root-trace-table-candidates-def* **by** *blast*
from *bound* **obtain** *result final-state fr' f-fri-roots' f-final' dg'*
composition-fri-roots' final' rest' where
out-eq: $\text{out} = \text{Some (result, final-state)}$

and *header'*:

$\text{verifier-header-transcript s fr' f-fri-roots' f-final' as dg'}$
 $\text{composition-fri-roots' final' rest'}$

and *bind*:

$\text{merkle-root-binds-table fr' trace-table final-state}$

unfolding *accepted-with-bound-tables-def* **by** *blast*

have *fr-eq*: $\text{fr}' = \text{fr}$

using *verifier-header-transcript-unique[OF header' header]* **by** *simp*

have *bound-some*:

$\text{accepted-with-bound-tables s (Some (result, final-state))}$

```

    trace-table composition-table as query-idxs
  using bound out-eq by simp
have challenges-some:
  accepted-fri-challenges s (Some (result, final-state)) trace-bs dg
  comp-bs
  using challenges out-eq by simp
have bind-fr: merkle-root-binds-table fr trace-table final-state
  using bind fr-eq by simp
have state:
  trace-fri-root-trace-table-candidate-state s fr trace-table
  final-state
  unfolding trace-fri-root-trace-table-candidate-state-def
  apply (rule exI[where x=result])
  apply (rule exI[where x=composition-table])
  apply (rule exI[where x=as])
  apply (rule exI[where x=query-idxs])
  apply (rule exI[where x=trace-bs])
  apply (rule exI[where x=dg])
  apply (rule exI[where x=comp-bs])
  apply (rule exI[where x=f-fri-roots])
  apply (rule exI[where x=f-final])
  apply (rule exI[where x=header-as])
  apply (rule exI[where x=composition-fri-roots])
  apply (rule exI[where x=final])
  apply (rule exI[where x=rest])
  apply (intro conjI)
    apply (rule bound-some)
    apply (rule challenges-some)
    apply (rule header)
  apply (rule bind-fr)
done
then show
  ∃ final-state.
    trace-fri-root-trace-table-candidate-state s fr trace-table
    final-state
  by blast
next
assume
  ∃ final-state.
    trace-fri-root-trace-table-candidate-state s fr trace-table
    final-state
then obtain final-state result composition-table as query-idxs trace-bs dg
  comp-bs f-fri-roots f-final header-as composition-fri-roots final rest
where
  bound:
    accepted-with-bound-tables s (Some (result, final-state))
    trace-table composition-table as query-idxs
and challenges:
  accepted-fri-challenges s (Some (result, final-state)) trace-bs dg

```

```

    comp-bs
  and header:
    verifier-header-transcript s fr f-fri-roots f-final header-as dg
    composition-fri-roots final rest
  unfolding trace-fri-root-trace-table-candidate-state-def by blast
show trace-table ∈ trace-fri-root-trace-table-candidates s fr
  unfolding trace-fri-root-trace-table-candidates-def
  using bound challenges header by blast
qed

lemma trace-fri-supported-root-trace-table-candidates-iff-state:
  trace-table ∈ trace-fri-supported-root-trace-table-candidates s fr  $\longleftrightarrow$ 
  (∃ final-state.
    trace-fri-supported-root-trace-table-candidate-state s fr trace-table
    final-state)
proof
  assume candidate:
    trace-table ∈ trace-fri-supported-root-trace-table-candidates s fr
  then obtain out composition-table as query-idxs trace-bs dg comp-bs
    f-fri-roots f-final header-as composition-fri-roots final rest where
    outcome: out ∈ set-dist (execute verify-monad s)
  and bound:
    accepted-with-bound-tables s out trace-table composition-table as
    query-idxs
  and challenges:
    accepted-fri-challenges s out trace-bs dg comp-bs
  and header:
    verifier-header-transcript s fr f-fri-roots f-final header-as dg
    composition-fri-roots final rest
  unfolding trace-fri-supported-root-trace-table-candidates-def by blast
from bound obtain result final-state fr' f-fri-roots' f-final' dg'
  composition-fri-roots' final' rest' where
  out-eq: out = Some (result, final-state)
  and header':
    verifier-header-transcript s fr' f-fri-roots' f-final' as dg'
    composition-fri-roots' final' rest'
  and bind:
    merkle-root-binds-table fr' trace-table final-state
  unfolding accepted-with-bound-tables-def by blast
have fr-eq: fr' = fr
  using verifier-header-transcript-unique[OF header' header] by simp
have state:
  trace-fri-supported-root-trace-table-candidate-state s fr trace-table
  final-state
  unfolding trace-fri-supported-root-trace-table-candidate-state-def
  apply (rule exI[where x=result])
  apply (rule exI[where x=composition-table])
  apply (rule exI[where x=as])
  apply (rule exI[where x=query-idxs])

```

```

apply (rule exI[where  $x=trace-bs$ ])
apply (rule exI[where  $x=dg$ ])
apply (rule exI[where  $x=comp-bs$ ])
apply (rule exI[where  $x=f-fri-roots$ ])
apply (rule exI[where  $x=f-final$ ])
apply (rule exI[where  $x=header-as$ ])
apply (rule exI[where  $x=composition-fri-roots$ ])
apply (rule exI[where  $x=final$ ])
apply (rule exI[where  $x=rest$ ])
apply (intro conjI)
  apply (use outcome out-eq in simp)
  apply (use bound out-eq in simp)
  apply (use challenges out-eq in simp)
  apply (rule header)
apply (use bind fr-eq in simp)
done
then show
   $\exists final-state.$ 
     $trace-fri-supported-root-trace-table-candidate-state\ s\ fr\ trace-table$ 
     $final-state$ 
  by blast
next
assume
   $\exists final-state.$ 
     $trace-fri-supported-root-trace-table-candidate-state\ s\ fr\ trace-table$ 
     $final-state$ 
then obtain  $final-state\ result\ composition-table\ as\ query-idxs\ trace-bs\ dg$ 
   $comp-bs\ f-fri-roots\ f-final\ header-as\ composition-fri-roots\ final\ rest$ 
where
  outcome:
     $Some\ (result,\ final-state) \in set-dist\ (execute\ verify-monad\ s)$ 
and bound:
     $accepted-with-bound-tables\ s\ (Some\ (result,\ final-state))$ 
     $trace-table\ composition-table\ as\ query-idxs$ 
and challenges:
     $accepted-fri-challenges\ s\ (Some\ (result,\ final-state))\ trace-bs\ dg$ 
     $comp-bs$ 
and header:
     $verifier-header-transcript\ s\ fr\ f-fri-roots\ f-final\ header-as\ dg$ 
     $composition-fri-roots\ final\ rest$ 
unfolding  $trace-fri-supported-root-trace-table-candidate-state-def$  by blast
show  $trace-table \in trace-fri-supported-root-trace-table-candidates\ s\ fr$ 
unfolding  $trace-fri-supported-root-trace-table-candidates-def$ 
using outcome bound challenges header by blast
qed

```

```

lemma  $trace-fri-supported-root-partial-trace-table-candidates-iff-state:$ 
   $trace-table \in$ 
     $trace-fri-supported-root-partial-trace-table-candidates\ s\ fr \longleftrightarrow$ 

```

$(\exists \text{ final-state.}$
 $\text{trace-fri-supported-root-partial-trace-table-candidate-state } s \text{ fr}$
 $\text{trace-table final-state})$

proof

assume *candidate*:

$\text{trace-table} \in$
 $\text{trace-fri-supported-root-partial-trace-table-candidates } s \text{ fr}$

then obtain *out query-idxs trace-openings trace-bs dg comp-bs*
 $f\text{-fri-roots } f\text{-final header-as composition-fri-roots final rest$ **where**
 $\text{outcome: out} \in \text{set-dist (execute verify-monad } s)$

and *partial*:

$\text{accepted-with-partial-trace-openings } s \text{ out fr query-idxs}$
 trace-openings

and *table*:

$\text{partial-trace-table-candidate trace-table trace-openings}$

and *challenges*:

$\text{accepted-fri-challenges } s \text{ out trace-bs dg comp-bs}$

and *header*:

$\text{verifier-header-transcript } s \text{ fr } f\text{-fri-roots } f\text{-final header-as dg}$
 $\text{composition-fri-roots final rest}$

unfolding *trace-fri-supported-root-partial-trace-table-candidates-def*
by *blast*

from *partial* **obtain** *result final-state* **where** *out-eq*:

$\text{out} = \text{Some (result, final-state)}$

unfolding *accepted-with-partial-trace-openings-def* **by** *blast*

have *state*:

$\text{trace-fri-supported-root-partial-trace-table-candidate-state } s \text{ fr}$
 $\text{trace-table final-state}$

proof —

have *outcome-some*:

$\text{Some (result, final-state)} \in \text{set-dist (execute verify-monad } s)$
using *outcome out-eq* **by** *simp*

have *partial-some*:

$\text{accepted-with-partial-trace-openings } s$
 $(\text{Some (result, final-state)}) \text{ fr query-idxs trace-openings}$
using *partial out-eq* **by** *simp*

have *challenges-some*:

$\text{accepted-fri-challenges } s (\text{Some (result, final-state)}) \text{ trace-bs dg}$
 comp-bs
using *challenges out-eq* **by** *simp*

show *?thesis*

unfolding
 $\text{trace-fri-supported-root-partial-trace-table-candidate-state-def}$
using *outcome-some partial-some table challenges-some header* **by** *blast*

qed

then show

$\exists \text{ final-state.}$
 $\text{trace-fri-supported-root-partial-trace-table-candidate-state } s \text{ fr}$
 $\text{trace-table final-state}$

by blast
 next
 assume
 $\exists$ final-state.
 trace-fri-supported-root-partial-trace-table-candidate-state s fr
 trace-table final-state
 then obtain final-state result query-idxs trace-openings trace-bs dg
 comp-bs f-fri-roots f-final header-as composition-fri-roots final rest
 where
 outcome:
 Some (result, final-state) $\in$ set-dist (execute verify-monad s)
 and partial:
 accepted-with-partial-trace-openings s
 (Some (result, final-state)) fr query-idxs trace-openings
 and table:
 partial-trace-table-candidate trace-table trace-openings
 and challenges:
 accepted-fri-challenges s (Some (result, final-state)) trace-bs dg
 comp-bs
 and header:
 verifier-header-transcript s fr f-fri-roots f-final header-as dg
 composition-fri-roots final rest
 unfolding
 trace-fri-supported-root-partial-trace-table-candidate-state-def
 by blast
 show
 trace-table $\in$
 trace-fri-supported-root-partial-trace-table-candidates s fr
 unfolding trace-fri-supported-root-partial-trace-table-candidates-def
 using outcome partial table challenges header by blast
 qed

lemma trace-fri-supported-root-partial-trace-table-candidate-length:
 assumes
 trace-fri-supported-root-partial-trace-table-candidate-state s fr
 trace-table final-state
 shows length trace-table = scale * clength
 using assms
 unfolding trace-fri-supported-root-partial-trace-table-candidate-state-def
 partial-trace-table-candidate-def
 by blast

lemma alpha-header-supported-partial-union-bad-sets-subset-alpha-space:
 alpha-header-supported-partial-union-bad-sets s bad-sets fr f-fri-roots
 f-final $\subseteq$ alpha-space
 unfolding alpha-header-supported-partial-union-bad-sets-def by auto

lemma trace-fri-supported-root-trace-table-candidate-state-imp-candidate-state:
 assumes

```

    trace-fri-supported-root-trace-table-candidate-state s fr trace-table
    final-state
shows
    trace-fri-root-trace-table-candidate-state s fr trace-table final-state
using assms
unfolding trace-fri-supported-root-trace-table-candidate-state-def
    trace-fri-root-trace-table-candidate-state-def
by blast

lemma trace-fri-supported-root-trace-table-candidate-state-hash-extends:
  assumes
    trace-fri-supported-root-trace-table-candidate-state s fr trace-table
    final-state
  shows  $s \leq \text{final-state}$ 
proof –
  from assms obtain result composition-table as query-idxs trace-bs dg comp-bs
    f-fri-roots f-final header-as composition-fri-roots final rest where
    outcome:
      Some (result, final-state)  $\in$  set-dist (execute verify-monad s)
  unfolding trace-fri-supported-root-trace-table-candidate-state-def
  by blast
  show ?thesis
  by (rule verify-monad-hash-extends[OF outcome])
qed

lemma trace-fri-supported-root-trace-table-candidate-state-imp-partial-candidate:
  assumes state:
    trace-fri-supported-root-trace-table-candidate-state s fr trace-table
    final-state
  and clean:  $\neg$  hash-map-output-collision final-state
  shows
    trace-table  $\in$ 
    trace-fri-supported-root-partial-trace-table-candidates s fr
proof –
  from state obtain result composition-table as query-idxs trace-bs dg comp-bs
    f-fri-roots f-final header-as composition-fri-roots final rest where
    outcome:
      Some (result, final-state)  $\in$  set-dist (execute verify-monad s)
  and bound:
    accepted-with-bound-tables s (Some (result, final-state))
    trace-table composition-table as query-idxs
  and challenges:
    accepted-fri-challenges s (Some (result, final-state)) trace-bs dg
    comp-bs
  and header:
    verifier-header-transcript s fr f-fri-roots f-final header-as dg
    composition-fri-roots final rest
  and bind:
    merkle-root-binds-table fr trace-table final-state

```



```

unfolding trace-fri-supported-root-trace-table-candidate-state-def
by blast
from verify-monad-header-extraction[OF outcome]
obtain fr' f-fl f-final' as' dg' fl final' query-state where
  header':
    verifier-header-transcript s fr' (map snd f-fl) f-final' as' dg'
      (map snd fl) final' (PTranscript query-state)
and query-out:
  Some (result, final-state) ∈
    set-dist
      (execute
        (ntimes
          (verifier-query-round-program fr' f-fl f-final' as' fl final')
            rounds)
          query-state)
by (rule verify-monad-header-query-extraction[OF outcome])
have fr-eq: fr' = fr
  using verifier-header-transcript-unique[OF header' header] by simp
obtain query-idxs' trace-openings where
  len-query-idxs: length query-idxs' = rounds
and len-openings: length trace-openings = rounds
and indices:
   $\bigwedge i. i < \text{rounds} \implies$ 
    map opening-index (trace-openings ! i) =
      powers-scaled (query-idxs' ! i)
and tables:
   $\bigwedge i. i < \text{rounds} \implies$ 
    partial-authenticated-table fr'
      (scale * clength) (trace-openings ! i) final-state
proof (rule ntimes-verifier-query-rounds-authenticated-trace-openings-no-raws
  [OF query-out])
fix query-idxs' trace-openings
assume len-query-idxs: length query-idxs' = rounds
and len-openings: length trace-openings = rounds
and indices:
   $\bigwedge i. i < \text{rounds} \implies$ 
    map opening-index (trace-openings ! i) =
      powers-scaled (query-idxs' ! i)
and tables:
   $\bigwedge i. i < \text{rounds} \implies$ 
    partial-authenticated-table fr'
      (scale * clength) (trace-openings ! i) final-state
show ?thesis
  by (rule that[OF len-query-idxs len-openings indices tables])
qed
have partial:
  accepted-with-partial-trace-openings s
    (Some (result, final-state)) fr query-idxs' trace-openings
proof –

```

```

have all-indices:
   $\forall i < \text{rounds}.$ 
    map opening-index (trace-openings ! i) =
      powers-scaled (query-idxs' ! i)
    using indices by blast
have all-tables:
   $\forall i < \text{rounds}.$ 
    partial-authenticated-table fr (scale * clength)
      (trace-openings ! i) final-state
    using tables fr-eq by simp
show ?thesis
  unfolding accepted-with-partial-trace-openings-def accepted-def
  by (intro conjI exI[of - result] exI[of - final-state])
    (use len-query-idxs len-openings all-indices all-tables in simp-all)
qed
have len-table: length trace-table = scale * clength
  using accepted-with-bound-tables-shapes(1)[OF bound] by simp
have partial-candidate:
  partial-trace-table-candidate trace-table trace-openings
  by (rule accepted-bound-trace-table-partial-candidate-if-clean
    [OF bind len-table partial clean])
show ?thesis
proof –
  have witness:
     $\exists$  out query-idxs trace-openings trace-bs dg comp-bs
      f-fri-roots f-final header-as composition-fri-roots final rest.
      out  $\in$  set-dist (execute verify-monad s)  $\wedge$ 
      accepted-with-partial-trace-openings s out fr query-idxs
      trace-openings  $\wedge$ 
      partial-trace-table-candidate trace-table trace-openings  $\wedge$ 
      accepted-fri-challenges s out trace-bs dg comp-bs  $\wedge$ 
      verifier-header-transcript s fr f-fri-roots f-final header-as dg
      composition-fri-roots final rest
    using outcome partial partial-candidate challenges header by blast
  show ?thesis
  using witness
  unfolding trace-fri-supported-root-partial-trace-table-candidates-def
  by simp
qed
qed

lemma alpha-header-supported-trace-table-candidate-state-imp-partial-candidate:
assumes state:
  alpha-header-supported-trace-table-candidate-state s fr f-fri-roots
  f-final trace-table final-state
and clean:  $\neg$  hash-map-output-collision final-state
shows
  trace-table  $\in$ 
    alpha-header-supported-partial-trace-table-candidates s fr f-fri-roots

```

f -final
proof –
from *state* **obtain** *result composition-table as query-idxs dg*
composition-fri-roots final rest **where**
outcome:
Some (result, final-state) ∈ set-dist (execute verify-monad s)
and *bound:*
accepted-with-bound-tables s (Some (result, final-state))
trace-table composition-table as query-idxs
and *header:*
verifier-header-transcript s fr f-fri-roots f-final as dg
composition-fri-roots final rest
and *bind:*
merkle-root-binds-table fr trace-table final-state
unfolding *alpha-header-supported-trace-table-candidate-state-def*
by *blast*
from *verify-monad-header-extraction[OF outcome]*
obtain *fr' f-fl f-final' as' dg' fl final' query-state* **where**
header':
verifier-header-transcript s fr' (map snd f-fl) f-final' as' dg'
(map snd fl) final' (PTranscript query-state)
and *query-out:*
Some (result, final-state) ∈
set-dist
(execute
(ntimes
(verifier-query-round-program fr' f-fl f-final' as' fl final')
rounds)
query-state)
by *(rule verify-monad-header-query-extraction[OF outcome])*
have *eqs:*
fr' = fr ∧ map snd f-fl = f-fri-roots ∧
f-final' = f-final ∧ as' = as ∧ dg' = dg ∧
map snd fl = composition-fri-roots ∧ final' = final ∧
PTranscript query-state = rest
using *verifier-header-transcript-unique[OF header' header]* **by** *simp*
obtain *query-idxs' trace-openings* **where**
len-query-idxs: length query-idxs' = rounds
and *len-openings: length trace-openings = rounds*
and *indices:*
 $\bigwedge i. i < \text{rounds} \implies$
 $\text{map opening-index (trace-openings ! } i) =$
 $\text{powers-scaled (query-idxs' ! } i)$
and *tables:*
 $\bigwedge i. i < \text{rounds} \implies$
 $\text{partial-authenticated-table fr'}$
 $(\text{scale} * \text{clength}) (\text{trace-openings ! } i) \text{ final-state}$
proof *(rule ntimes-verifier-query-rounds-authenticated-trace-openings-no-raws*
[OF query-out])

```

fix query-idxs' trace-openings
assume len-query-idxs: length query-idxs' = rounds
and len-openings: length trace-openings = rounds
and indices:
   $\bigwedge i. i < rounds \implies$ 
    map opening-index (trace-openings ! i) =
      powers-scaled (query-idxs' ! i)
and tables:
   $\bigwedge i. i < rounds \implies$ 
    partial-authenticated-table fr'
      (scale * clength) (trace-openings ! i) final-state
show ?thesis
  by (rule that[OF len-query-idxs len-openings indices tables])
qed
have partial:
  accepted-with-partial-trace-openings s
    (Some (result, final-state)) fr query-idxs' trace-openings
proof –
  have all-indices:
     $\forall i < rounds.$ 
      map opening-index (trace-openings ! i) =
        powers-scaled (query-idxs' ! i)
      using indices by blast
  have all-tables:
     $\forall i < rounds.$ 
      partial-authenticated-table fr (scale * clength)
        (trace-openings ! i) final-state
      using tables eqs by simp
  show ?thesis
    unfolding accepted-with-partial-trace-openings-def accepted-def
    by (intro conjI exI[of - result] exI[of - final-state])
      (use len-query-idxs len-openings all-indices all-tables in simp-all)
qed
have len-table: length trace-table = scale * clength
  using accepted-with-bound-tables-shapes(1)[OF bound] by simp
have partial-candidate:
  partial-trace-table-candidate trace-table trace-openings
  by (rule accepted-bound-trace-table-partial-candidate-if-clean
    [OF bind len-table partial clean])
show ?thesis
  unfolding alpha-header-supported-partial-trace-table-candidates-def
  by (intro CollectI exI[of - Some (result, final-state)]
    exI[of - query-idxs] exI[of - trace-openings]
    exI[of - as] exI[of - dg]
    exI[of - composition-fri-roots] exI[of - final]
    exI[of - rest] conjI)
    (use outcome partial partial-candidate header in simp-all)
qed

```

lemma *trace-fri-root-trace-table-candidate-states-common-extension-unique-if-clean:*
assumes *clean:* $\neg$ *hash-map-output-collision* *u*
and *candidate1:*
trace-fri-root-trace-table-candidate-state s fr trace-table
final-state
and *candidate2:*
trace-fri-root-trace-table-candidate-state s fr trace-table'
final-state'
and *ext1:* *final-state* $\leq$ *u*
and *ext2:* *final-state'* $\leq$ *u*
shows *trace-table = trace-table'*
proof –
from *candidate1* **obtain** *result composition-table as query-idxs trace-bs dg*
comp-bs f-fri-roots f-final header-as composition-fri-roots final rest
where
bound1:
accepted-with-bound-tables s (Some (result, final-state))
trace-table composition-table as query-idxs
and *bind1:* *merkle-root-binds-table fr trace-table final-state*
unfolding *trace-fri-root-trace-table-candidate-state-def* **by** *blast*
from *candidate2* **obtain** *result' composition-table' as' query-idxs' trace-bs'*
dg' comp-bs' f-fri-roots' f-final' header-as'
composition-fri-roots' final' rest' where
bound2:
accepted-with-bound-tables s (Some (result', final-state'))
trace-table' composition-table' as' query-idxs'
and *bind2:* *merkle-root-binds-table fr trace-table' final-state'*
unfolding *trace-fri-root-trace-table-candidate-state-def* **by** *blast*
have *same-len:* *length trace-table = length trace-table'*
using *accepted-with-bound-tables-shapes(1)[OF bound1]*
accepted-with-bound-tables-shapes(1)[OF bound2]
by *simp*
have *nonempty:* *trace-table* $\neq$ []
using *accepted-with-bound-tables-shapes(1)[OF bound1]*
eval-domain-nontrivial **by** *auto*
show *?thesis*
by (*rule merkle-root-binds-same-length-tables-common-extension-unique-if-clean*
 $[OF\ clean\ bind1\ bind2\ ext1\ ext2\ same-len\ nonempty]$)
qed

lemma *trace-fri-root-trace-table-candidate-states-compatible-unique-if-clean-merge:*
assumes *compatible:* *hash-maps-compatible final-state final-state'*
and *clean:*
 $\neg$ *hash-map-output-collision*
(hash-state-merge final-state final-state')
and *candidate1:*
trace-fri-root-trace-table-candidate-state s fr trace-table
final-state
and *candidate2:*

```

    trace-fri-root-trace-table-candidate-state s fr trace-table'
    final-state'
  shows trace-table = trace-table'
proof -
  from candidate1 obtain result composition-table as query-idxs trace-bs dg
    comp-bs f-fri-roots f-final header-as composition-fri-roots final rest
  where
    bound1:
      accepted-with-bound-tables s (Some (result, final-state))
      trace-table composition-table as query-idxs
    and bind1: merkle-root-binds-table fr trace-table final-state
    unfolding trace-fri-root-trace-table-candidate-state-def by blast
  from candidate2 obtain result' composition-table' as' query-idxs' trace-bs'
    dg' comp-bs' f-fri-roots' f-final' header-as'
    composition-fri-roots' final' rest' where
    bound2:
      accepted-with-bound-tables s (Some (result', final-state'))
      trace-table' composition-table' as' query-idxs'
    and bind2: merkle-root-binds-table fr trace-table' final-state'
    unfolding trace-fri-root-trace-table-candidate-state-def by blast
  have same-len: length trace-table = length trace-table'
  using accepted-with-bound-tables-shapes(1)[OF bound1]
    accepted-with-bound-tables-shapes(1)[OF bound2]
  by simp
  have nonempty: trace-table ≠ []
  using accepted-with-bound-tables-shapes(1)[OF bound1]
    eval-domain-nontrivial by auto
  show ?thesis
  by (rule
    merkle-root-binds-same-length-tables-compatible-states-unique-if-clean-merge
    [OF compatible clean bind1 bind2 same-len nonempty])
qed

lemma trace-fri-root-trace-table-candidate-states-merkle-compatible-unique-if-clean-merge:
  assumes compatible:
    merkle-hash-maps-compatible final-state final-state'
  and clean:
    ¬ hash-map-output-collision
    (merkle-hash-state-merge final-state final-state')
  and candidate1:
    trace-fri-root-trace-table-candidate-state s fr trace-table
    final-state
  and candidate2:
    trace-fri-root-trace-table-candidate-state s fr trace-table'
    final-state'
  shows trace-table = trace-table'
proof -
  from candidate1 obtain result composition-table as query-idxs trace-bs dg
    comp-bs f-fri-roots f-final header-as composition-fri-roots final rest

```

```

where
  bound1:
    accepted-with-bound-tables  $s$  (Some (result, final-state))
    trace-table composition-table as query-idxs
  and bind1: merkle-root-binds-table fr trace-table final-state
  unfolding trace-fri-root-trace-table-candidate-state-def by blast
from candidate2 obtain result' composition-table' as' query-idxs' trace-bs'
  dg' comp-bs' f-fri-roots' f-final' header-as'
  composition-fri-roots' final' rest' where
  bound2:
    accepted-with-bound-tables  $s$  (Some (result', final-state'))
    trace-table' composition-table' as' query-idxs'
  and bind2: merkle-root-binds-table fr trace-table' final-state'
  unfolding trace-fri-root-trace-table-candidate-state-def by blast
have same-len: length trace-table = length trace-table'
using accepted-with-bound-tables-shapes(1)[OF bound1]
  accepted-with-bound-tables-shapes(1)[OF bound2]
by simp
have nonempty: trace-table  $\neq$  []
using accepted-with-bound-tables-shapes(1)[OF bound1]
  eval-domain-nontrivial by auto
show ?thesis
by (rule
  merkle-root-binds-same-length-tables-merkle-compatible-unique-if-clean-merge
  [OF compatible clean bind1 bind2 same-len nonempty])
qed

lemma trace-fri-root-trace-table-candidates-subset-singleton-if-common-clean-extension:
assumes clean:  $\neg$  hash-map-output-collision  $u$ 
and common-ext:
   $\bigwedge$  trace-table final-state.
  trace-fri-root-trace-table-candidate-state  $s$  fr trace-table
  final-state  $\implies$ 
  final-state  $\leq u$ 
shows
   $\exists$  trace-table.
  trace-fri-root-trace-table-candidates  $s$  fr  $\subseteq \{trace-table\}$ 
proof (cases trace-fri-root-trace-table-candidates  $s$  fr = {})
case True
then show ?thesis
by (intro exI[of - []]) simp
next
case False
then obtain trace-table0 where table0:
  trace-table0  $\in$  trace-fri-root-trace-table-candidates  $s$  fr
by blast
then obtain final-state0 where state0:
  trace-fri-root-trace-table-candidate-state  $s$  fr trace-table0
  final-state0

```

```

    using trace-fri-root-trace-table-candidates-iff-state by blast
  have ext0: final-state0 ≤ u
    by (rule common-ext[OF state0])
  have subset:
    trace-fri-root-trace-table-candidates s fr ⊆ {trace-table0}
  proof
    fix trace-table
    assume table:
      trace-table ∈ trace-fri-root-trace-table-candidates s fr
    then obtain final-state where state:
      trace-fri-root-trace-table-candidate-state s fr trace-table
      final-state
    using trace-fri-root-trace-table-candidates-iff-state by blast
    have ext: final-state ≤ u
      by (rule common-ext[OF state])
    have trace-table = trace-table0
      by (rule
        trace-fri-root-trace-table-candidate-states-common-extension-unique-if-clean
        [OF clean state state0 ext ext0])
    then show trace-table ∈ {trace-table0}
      by simp
  qed
  then show ?thesis
    by blast
qed

```

lemma trace-fri-root-trace-table-candidates-subset-singleton-if-pairwise-clean-merge:

```

  assumes pairwise:
    ∧ trace-table final-state trace-table' final-state'.
      trace-fri-root-trace-table-candidate-state s fr trace-table
      final-state ⇒
      trace-fri-root-trace-table-candidate-state s fr trace-table'
      final-state' ⇒
      hash-maps-compatible final-state final-state' ∧
      ¬ hash-map-output-collision
      (hash-state-merge final-state final-state')
  shows
    ∃ trace-table.
      trace-fri-root-trace-table-candidates s fr ⊆ {trace-table}
  proof (cases trace-fri-root-trace-table-candidates s fr = {})
    case True
    then show ?thesis
      by (intro exI[of - []]) simp
  next
    case False
    then obtain trace-table0 where table0:
      trace-table0 ∈ trace-fri-root-trace-table-candidates s fr
      by blast
    then obtain final-state0 where state0:

```



```

    trace-fri-root-trace-table-candidate-state s fr trace-table0
    final-state0
    using trace-fri-root-trace-table-candidates-iff-state by blast
  have subset:
    trace-fri-root-trace-table-candidates s fr  $\subseteq$  {trace-table0}
  proof
    fix trace-table
    assume table:
      trace-table  $\in$  trace-fri-root-trace-table-candidates s fr
    then obtain final-state where state:
      trace-fri-root-trace-table-candidate-state s fr trace-table
      final-state
    using trace-fri-root-trace-table-candidates-iff-state by blast
  have compatible:
    hash-maps-compatible final-state final-state0
    using pairwise[OF state state0] by simp
  have clean:
     $\neg$  hash-map-output-collision
    (hash-state-merge final-state final-state0)
    using pairwise[OF state state0] by simp
  have trace-table = trace-table0
    by (rule
      trace-fri-root-trace-table-candidate-states-compatible-unique-if-clean-merge
      [OF compatible clean state state0])
  then show trace-table  $\in$  {trace-table0}
    by simp
qed
then show ?thesis
  by blast
qed

lemma trace-fri-supported-root-trace-table-candidates-subset-singleton-if-pairwise-clean-merge:
  assumes pairwise:
     $\bigwedge$  trace-table final-state trace-table' final-state'.
      trace-fri-supported-root-trace-table-candidate-state s fr trace-table
      final-state  $\implies$ 
      trace-fri-supported-root-trace-table-candidate-state s fr trace-table'
      final-state'  $\implies$ 
      hash-maps-compatible final-state final-state'  $\wedge$ 
       $\neg$  hash-map-output-collision
      (hash-state-merge final-state final-state')
  shows
     $\exists$  trace-table.
      trace-fri-supported-root-trace-table-candidates s fr  $\subseteq$ 
      {trace-table}
  proof (cases trace-fri-supported-root-trace-table-candidates s fr = {})
    case True
    then show ?thesis
      by (intro exI[of - []]) simp
  end

```

```

next
  case False
  then obtain trace-table0 where table0:
    trace-table0 ∈ trace-fri-supported-root-trace-table-candidates s fr
    by blast
  then obtain final-state0 where state0:
    trace-fri-supported-root-trace-table-candidate-state s fr trace-table0
    final-state0
    using trace-fri-supported-root-trace-table-candidates-iff-state by blast
  have state0-plain:
    trace-fri-root-trace-table-candidate-state s fr trace-table0
    final-state0
    by (rule
      trace-fri-supported-root-trace-table-candidate-state-imp-candidate-state
      [OF state0])
  have subset:
    trace-fri-supported-root-trace-table-candidates s fr ⊆
    {trace-table0}
  proof
    fix trace-table
    assume table:
      trace-table ∈ trace-fri-supported-root-trace-table-candidates s fr
    then obtain final-state where state:
      trace-fri-supported-root-trace-table-candidate-state s fr trace-table
      final-state
      using trace-fri-supported-root-trace-table-candidates-iff-state by blast
    have state-plain:
      trace-fri-root-trace-table-candidate-state s fr trace-table
      final-state
      by (rule
        trace-fri-supported-root-trace-table-candidate-state-imp-candidate-state
        [OF state])
    have compatible:
      hash-maps-compatible final-state final-state0
      using pairwise[OF state state0] by simp
    have clean:
      ¬ hash-map-output-collision
      (hash-state-merge final-state final-state0)
      using pairwise[OF state state0] by simp
    have trace-table = trace-table0
      by (rule
        trace-fri-root-trace-table-candidate-states-compatible-unique-if-clean-merge
        [OF compatible clean state-plain state0-plain])
    then show trace-table ∈ {trace-table0}
      by simp
  qed
  then show ?thesis
    by blast
qed

```

lemma *trace-fri-supported-root-trace-table-candidates-subset-singleton-if-pairwise-no-value-conflict-clean-merge:*

assumes *pairwise:*

$\bigwedge \text{trace-table final-state trace-table' final-state'}$.
 $\text{trace-fri-supported-root-trace-table-candidate-state } s \text{ fr trace-table}$
 $\text{final-state} \implies$
 $\text{trace-fri-supported-root-trace-table-candidate-state } s \text{ fr trace-table'}$
 $\text{final-state'} \implies$
 $\neg \text{hash-map-value-conflict final-state final-state'} \wedge$
 $\neg \text{hash-map-output-collision}$
 $(\text{hash-state-merge final-state final-state'})$

shows

$\exists \text{trace-table}$.
 $\text{trace-fri-supported-root-trace-table-candidates } s \text{ fr } \subseteq$
 $\{\text{trace-table}\}$

proof (rule

trace-fri-supported-root-trace-table-candidates-subset-singleton-if-pairwise-clean-merge)

fix *trace-table final-state trace-table' final-state'*

assume *candidate:*

$\text{trace-fri-supported-root-trace-table-candidate-state } s \text{ fr trace-table}$
 final-state

and *candidate':*

$\text{trace-fri-supported-root-trace-table-candidate-state } s \text{ fr trace-table'}$
 final-state'

have *no-conflict:*

$\neg \text{hash-map-value-conflict final-state final-state'}$
using *pairwise*[*OF candidate candidate'*] **by** *simp*

have *compatible:*

$\text{hash-maps-compatible final-state final-state'}$
using *no-conflict*
unfolding *hash-maps-compatible-iff-no-value-conflict* .

have *clean:*

$\neg \text{hash-map-output-collision}$
 $(\text{hash-state-merge final-state final-state'})$
using *pairwise*[*OF candidate candidate'*] **by** *simp*
show $\text{hash-maps-compatible final-state final-state'} \wedge$
 $\neg \text{hash-map-output-collision}$
 $(\text{hash-state-merge final-state final-state'})$

using *compatible clean* **by** *simp*

qed

lemma *trace-fri-supported-root-trace-table-candidates-subset-singleton-if-pairwise-merkle-clean-merge:*

assumes *pairwise:*

$\bigwedge \text{trace-table final-state trace-table' final-state'}$.
 $\text{trace-fri-supported-root-trace-table-candidate-state } s \text{ fr trace-table}$
 $\text{final-state} \implies$
 $\text{trace-fri-supported-root-trace-table-candidate-state } s \text{ fr trace-table'}$
 $\text{final-state'} \implies$
 $\text{merkle-hash-maps-compatible final-state final-state'} \wedge$

```

    ¬ hash-map-output-collision
      (merkle-hash-state-merge final-state final-state')
  shows
    ∃ trace-table.
      trace-fri-supported-root-trace-table-candidates s fr ⊆
        {trace-table}
  proof (cases trace-fri-supported-root-trace-table-candidates s fr = {})
    case True
      then show ?thesis
        by (intro exI[of - []]) simp
  next
    case False
      then obtain trace-table0 where table0:
        trace-table0 ∈ trace-fri-supported-root-trace-table-candidates s fr
        by blast
      then obtain final-state0 where state0:
        trace-fri-supported-root-trace-table-candidate-state s fr trace-table0
        final-state0
        using trace-fri-supported-root-trace-table-candidates-iff-state by blast
      have state0-plain:
        trace-fri-root-trace-table-candidate-state s fr trace-table0
        final-state0
        by (rule
          trace-fri-supported-root-trace-table-candidate-state-imp-candidate-state
            [OF state0])
      have subset:
        trace-fri-supported-root-trace-table-candidates s fr ⊆
          {trace-table0}
      proof
        fix trace-table
        assume table:
          trace-table ∈ trace-fri-supported-root-trace-table-candidates s fr
        then obtain final-state where state:
          trace-fri-supported-root-trace-table-candidate-state s fr trace-table
          final-state
        using trace-fri-supported-root-trace-table-candidates-iff-state by blast
      have state-plain:
        trace-fri-root-trace-table-candidate-state s fr trace-table
        final-state
        by (rule
          trace-fri-supported-root-trace-table-candidate-state-imp-candidate-state
            [OF state])
      have compatible:
        merkle-hash-maps-compatible final-state final-state0
        using pairwise[OF state state0] by simp
      have clean:
        ¬ hash-map-output-collision
          (merkle-hash-state-merge final-state final-state0)
        using pairwise[OF state state0] by simp

```

```

have trace-table = trace-table0
  by (rule
    trace-fri-root-trace-table-candidate-states-merkle-compatible-unique-if-clean-merge
    [OF compatible clean state-plain state0-plain])
then show trace-table ∈ {trace-table0}
  by simp
qed
then show ?thesis
  by blast
qed

```

lemma *trace-fri-supported-root-trace-table-candidates-subset-singleton-if-pairwise-no-merkle-value-conflict-clean-merge*

assumes *pairwise*:

```

 $\bigwedge$  trace-table final-state trace-table' final-state'.
  trace-fri-supported-root-trace-table-candidate-state s fr trace-table
    final-state  $\implies$ 
    trace-fri-supported-root-trace-table-candidate-state s fr trace-table'
      final-state'  $\implies$ 
     $\neg$  merkle-hash-value-conflict final-state final-state'  $\wedge$ 
     $\neg$  hash-map-output-collision
      (merkle-hash-state-merge final-state final-state')

```

shows

```

 $\exists$  trace-table.
  trace-fri-supported-root-trace-table-candidates s fr  $\subseteq$ 
    {trace-table}

```

proof (*rule*

trace-fri-supported-root-trace-table-candidates-subset-singleton-if-pairwise-merkle-clean-merge)

fix *trace-table final-state trace-table' final-state'*

assume *candidate*:

```

trace-fri-supported-root-trace-table-candidate-state s fr trace-table
  final-state

```

and *candidate'*:

```

trace-fri-supported-root-trace-table-candidate-state s fr trace-table'
  final-state'

```

have *no-conflict*:

```

 $\neg$  merkle-hash-value-conflict final-state final-state'

```

using *pairwise*[*OF candidate candidate'*] **by** *simp*

have *compatible*:

```

merkle-hash-maps-compatible final-state final-state'

```

using *no-conflict*

unfolding *merkle-hash-maps-compatible-iff-no-value-conflict* .

have *clean*:

```

 $\neg$  hash-map-output-collision
  (merkle-hash-state-merge final-state final-state')

```

using *pairwise*[*OF candidate candidate'*] **by** *simp*

show *merkle-hash-maps-compatible final-state final-state'* $\wedge$

```

 $\neg$  hash-map-output-collision
  (merkle-hash-state-merge final-state final-state')

```

using *compatible clean* **by** *simp*

qed

lemma *trace-fri-supported-root-trace-table-candidates-subset-singleton-if-common-clean-merkle-extension:*

assumes *clean*: $\neg$ *hash-map-output-collision* *u*

and *common-ext*:

$\bigwedge$ *trace-table* *final-state*.

trace-fri-supported-root-trace-table-candidate-state *s* *fr* *trace-table*

final-state $\implies$

merkle-hash-extends *final-state* *u*

shows

$\exists$ *trace-table*.

trace-fri-supported-root-trace-table-candidates *s* *fr* $\subseteq$

$\{ \text{trace-table} \}$

proof (rule

trace-fri-supported-root-trace-table-candidates-subset-singleton-if-pairwise-no-merkle-value-conflict-clean-merge)

fix *trace-table* *final-state* *trace-table'* *final-state'*

assume *candidate*:

trace-fri-supported-root-trace-table-candidate-state *s* *fr* *trace-table*

final-state

and *candidate'*:

trace-fri-supported-root-trace-table-candidate-state *s* *fr* *trace-table'*

final-state'

have *ext*: *merkle-hash-extends* *final-state* *u*

by (rule *common-ext*[OF *candidate*])

have *ext'*: *merkle-hash-extends* *final-state'* *u*

by (rule *common-ext*[OF *candidate'*])

show $\neg$ *merkle-hash-value-conflict* *final-state* *final-state'* $\wedge$

$\neg$ *hash-map-output-collision*

(*merkle-hash-state-merge* *final-state* *final-state'*)

by (rule *common-clean-merkle-extension-imp-pairwise-merkle-clean*

[OF *ext* *ext'* *clean*])

qed

lemma *trace-fri-supported-root-trace-table-candidate-states-distinct-imp-merkle-conflict-or-merge-collision:*

assumes *candidate*:

trace-fri-supported-root-trace-table-candidate-state *s* *fr* *trace-table*

final-state

and *candidate'*:

trace-fri-supported-root-trace-table-candidate-state *s* *fr* *trace-table'*

final-state'

and *distinct-tables*: *trace-table* $\neq$ *trace-table'*

shows

merkle-hash-value-conflict *final-state* *final-state'* $\vee$

hash-map-output-collision

(*merkle-hash-state-merge* *final-state* *final-state'*)

proof (rule *ccontr*)

assume *no-bad*:

$\neg$ (*merkle-hash-value-conflict* *final-state* *final-state'* $\vee$

hash-map-output-collision

```

    (merkle-hash-state-merge final-state final-state'))
have compatible:
  merkle-hash-maps-compatible final-state final-state'
  using no-bad unfolding merkle-hash-maps-compatible-iff-no-value-conflict
  by simp
have clean:
  ¬ hash-map-output-collision
    (merkle-hash-state-merge final-state final-state')
  using no-bad by simp
have candidate-plain:
  trace-fri-root-trace-table-candidate-state s fr trace-table
    final-state
  by (rule
    trace-fri-supported-root-trace-table-candidate-state-imp-candidate-state
    [OF candidate])
have candidate-plain':
  trace-fri-root-trace-table-candidate-state s fr trace-table'
    final-state'
  by (rule
    trace-fri-supported-root-trace-table-candidate-state-imp-candidate-state
    [OF candidate'])
have trace-table = trace-table'
  by (rule
    trace-fri-root-trace-table-candidate-states-merkle-compatible-unique-if-clean-merge
    [OF compatible clean candidate-plain candidate-plain'])
then show False
  using distinct-tables by contradiction
qed

lemma trace-fri-supported-root-trace-table-candidates-not-singleton-imp-pairwise-merkle-bad:
  assumes not-unique:
    ¬ (∃ trace-table.
      trace-fri-supported-root-trace-table-candidates s fr ⊆
        {trace-table})
  shows
    ∃ trace-table final-state trace-table' final-state'.
      trace-fri-supported-root-trace-table-candidate-state s fr trace-table
        final-state ∧
      trace-fri-supported-root-trace-table-candidate-state s fr trace-table'
        final-state' ∧
      trace-table ≠ trace-table' ∧
      (merkle-hash-value-conflict final-state final-state' ∨
        hash-map-output-collision
          (merkle-hash-state-merge final-state final-state'))
proof —
  let ?C = trace-fri-supported-root-trace-table-candidates s fr
  have nonempty: ?C ≠ {}
    using not-unique by auto
  then obtain trace-table where table: trace-table ∈ ?C

```

by blast
 have $\neg ?C \subseteq \{trace-table\}$
 using not-unique by blast
 then obtain *trace-table'* where *table'*:
 trace-table' ∈ ?C
 and *distinct-tables*: *trace-table* $\neq$ *trace-table'*
 by auto
 from *table* obtain *final-state* where *candidate*:
 trace-fri-supported-root-trace-table-candidate-state s fr trace-table
 final-state
 using *trace-fri-supported-root-trace-table-candidates-iff-state* by blast
 from *table'* obtain *final-state'* where *candidate'*:
 trace-fri-supported-root-trace-table-candidate-state s fr trace-table'
 final-state'
 using *trace-fri-supported-root-trace-table-candidates-iff-state* by blast
 have *bad*:
 merkle-hash-value-conflict final-state final-state' ∨
 hash-map-output-collision
 (*merkle-hash-state-merge final-state final-state'*)
 by (rule
 trace-fri-supported-root-trace-table-candidate-states-distinct-imp-merkle-conflict-or-merge-collision
 [*OF candidate candidate' distinct-tables*])
 show *?thesis*
 using *candidate candidate' distinct-tables bad* by blast
 qed

definition *trace-fri-supported-root-pairwise-merkle-bad*
 :: (*f*, *a*) *protocol-channel-scheme* $\Rightarrow$ *bool*
 where
 trace-fri-supported-root-pairwise-merkle-bad s fr $\longleftrightarrow$
 ($\exists$ *trace-table final-state trace-table' final-state'*.
 trace-fri-supported-root-trace-table-candidate-state s fr trace-table
 final-state $\wedge$
 trace-fri-supported-root-trace-table-candidate-state s fr trace-table'
 final-state' $\wedge$
 trace-table $\neq$ *trace-table'* $\wedge$
 (*merkle-hash-value-conflict final-state final-state' ∨*
 hash-map-output-collision
 (*merkle-hash-state-merge final-state final-state'*)))

definition *trace-fri-supported-pairwise-merkle-bad*
 :: (*f*, *a*) *protocol-channel-scheme* $\Rightarrow$ *bool*
 where
 trace-fri-supported-pairwise-merkle-bad s $\longleftrightarrow$
 ($\exists$ *fr. trace-fri-supported-root-pairwise-merkle-bad s fr*)

lemma *trace-fri-supported-root-pairwise-merkle-bad-extends-initial*:
 assumes *bad*: *trace-fri-supported-root-pairwise-merkle-bad s fr*
 obtains *trace-table final-state trace-table' final-state'*

where
trace-fri-supported-root-trace-table-candidate-state s fr trace-table
final-state
trace-fri-supported-root-trace-table-candidate-state s fr trace-table'
final-state'
trace-table $\neq$ *trace-table'*
merkle-hash-value-conflict final-state final-state' $\vee$
hash-map-output-collision
(*merkle-hash-state-merge final-state final-state'*)
s $\leq$ *final-state*
s $\leq$ *final-state'*
proof –
from *bad* **obtain** *trace-table final-state trace-table' final-state'* **where**
candidate:
trace-fri-supported-root-trace-table-candidate-state s fr trace-table
final-state
and *candidate':*
trace-fri-supported-root-trace-table-candidate-state s fr trace-table'
final-state'
and *distinct:* *trace-table* $\neq$ *trace-table'*
and *pair-bad:*
merkle-hash-value-conflict final-state final-state' $\vee$
hash-map-output-collision
(*merkle-hash-state-merge final-state final-state'*)
unfolding *trace-fri-supported-root-pairwise-merkle-bad-def* **by** *blast*
have *ext:* *s* $\leq$ *final-state*
by (*rule trace-fri-supported-root-trace-table-candidate-state-hash-extends*
[*OF candidate*])
have *ext':* *s* $\leq$ *final-state'*
by (*rule trace-fri-supported-root-trace-table-candidate-state-hash-extends*
[*OF candidate'*])
show *?thesis*
by (*rule that*[*OF candidate candidate' distinct pair-bad ext ext'*])
qed

definition *trace-fri-supported-root-pairwise-coupling-bad*
 $:: (f, 'a) \text{ protocol-channel-scheme} \Rightarrow f \Rightarrow \text{bool}$
where
trace-fri-supported-root-pairwise-coupling-bad s fr $\longleftrightarrow$
($\exists$ *trace-table final-state trace-table' final-state'*.
trace-fri-supported-root-trace-table-candidate-state s fr trace-table
final-state $\wedge$
trace-fri-supported-root-trace-table-candidate-state s fr trace-table'
final-state' $\wedge$
trace-table $\neq$ *trace-table'* $\wedge$
merkle-hash-pairwise-coupling-bad final-state final-state')

definition *trace-fri-supported-pairwise-coupling-bad*
 $:: (f, 'a) \text{ protocol-channel-scheme} \Rightarrow \text{bool}$

where

$\text{trace-fri-supported-pairwise-coupling-bad } s \longleftrightarrow$
 $(\exists \text{ fr. trace-fri-supported-root-pairwise-coupling-bad } s \text{ fr})$

lemma *trace-fri-supported-root-pairwise-merkle-bad-imp-local-collision-or-coupling-bad*:

assumes *bad*: *trace-fri-supported-root-pairwise-merkle-bad* *s* *fr*

shows

$(\exists \text{ trace-table final-state trace-table' final-state'.$
 $\text{trace-fri-supported-root-trace-table-candidate-state } s \text{ fr trace-table}$
 $\text{final-state} \wedge$
 $\text{trace-fri-supported-root-trace-table-candidate-state } s \text{ fr trace-table'}$
 $\text{final-state'} \wedge$
 $\text{trace-table} \neq \text{trace-table'} \wedge$
 $(\text{hash-map-output-collision final-state} \vee$
 $\text{hash-map-output-collision final-state'}) \vee$
 $\text{trace-fri-supported-root-pairwise-coupling-bad } s \text{ fr})$

proof —

from *bad* **obtain** *trace-table final-state trace-table' final-state'*

where *candidate*:

$\text{trace-fri-supported-root-trace-table-candidate-state } s \text{ fr trace-table}$
 final-state

and *candidate'*:

$\text{trace-fri-supported-root-trace-table-candidate-state } s \text{ fr trace-table'}$
 final-state'

and *distinct*: $\text{trace-table} \neq \text{trace-table'}$

and *pair-bad*:

$\text{merkle-hash-value-conflict final-state final-state'} \vee$
 $\text{hash-map-output-collision}$
 $(\text{merkle-hash-state-merge final-state final-state'})$

unfolding *trace-fri-supported-root-pairwise-merkle-bad-def* **by** *blast*

have *decomp*:

$\text{hash-map-output-collision final-state} \vee$
 $\text{hash-map-output-collision final-state'} \vee$
 $\text{merkle-hash-pairwise-coupling-bad final-state final-state'}$

by (rule *merkle-hash-pairwise-bad-imp-local-collision-or-coupling-bad*
 $[OF \text{ pair-bad}]$)

show *?thesis*

using *candidate candidate' distinct decomp*

unfolding *trace-fri-supported-root-pairwise-coupling-bad-def* **by** *blast*

qed

lemma *trace-fri-supported-pairwise-merkle-bad-imp-local-collision-or-coupling-bad*:

assumes *bad*: *trace-fri-supported-pairwise-merkle-bad* *s*

shows

$(\exists \text{ fr trace-table final-state trace-table' final-state'.$
 $\text{trace-fri-supported-root-trace-table-candidate-state } s \text{ fr trace-table}$
 $\text{final-state} \wedge$
 $\text{trace-fri-supported-root-trace-table-candidate-state } s \text{ fr trace-table'}$
 $\text{final-state'} \wedge$

$trace-table \neq trace-table' \wedge$
 $(hash-map-output-collision\ final-state \vee$
 $hash-map-output-collision\ final-state')) \vee$
 $trace-fri-supported-pairwise-coupling-bad\ s$
proof –
from bad **obtain** fr **where** $root-bad$:
 $trace-fri-supported-root-pairwise-merkle-bad\ s\ fr$
unfolding $trace-fri-supported-pairwise-merkle-bad-def$ **by** $blast$
from
 $trace-fri-supported-root-pairwise-merkle-bad-imp-local-collision-or-coupling-bad$
 $[OF\ root-bad]$
show $?thesis$
unfolding $trace-fri-supported-pairwise-coupling-bad-def$ **by** $blast$
qed

lemma $trace-fri-supported-pairwise-merkle-bad-imp-supported-hash-collision-possible-or-coupling-bad$:
assumes bad : $trace-fri-supported-pairwise-merkle-bad\ s$
shows
 $supported-hash-output-collision-possible\ s \vee$
 $trace-fri-supported-pairwise-coupling-bad\ s$
proof –
have $decomp$:
 $(\exists\ fr\ trace-table\ final-state\ trace-table'\ final-state'.$
 $trace-fri-supported-root-trace-table-candidate-state\ s\ fr\ trace-table$
 $final-state \wedge$
 $trace-fri-supported-root-trace-table-candidate-state\ s\ fr\ trace-table'$
 $final-state' \wedge$
 $trace-table \neq trace-table' \wedge$
 $(hash-map-output-collision\ final-state \vee$
 $hash-map-output-collision\ final-state')) \vee$
 $trace-fri-supported-pairwise-coupling-bad\ s$
by ($rule$
 $trace-fri-supported-pairwise-merkle-bad-imp-local-collision-or-coupling-bad$
 $[OF\ bad])$)
then show $?thesis$
proof
assume $local$:
 $\exists\ fr\ trace-table\ final-state\ trace-table'\ final-state'.$
 $trace-fri-supported-root-trace-table-candidate-state\ s\ fr\ trace-table$
 $final-state \wedge$
 $trace-fri-supported-root-trace-table-candidate-state\ s\ fr\ trace-table'$
 $final-state' \wedge$
 $trace-table \neq trace-table' \wedge$
 $(hash-map-output-collision\ final-state \vee$
 $hash-map-output-collision\ final-state')$
then obtain $fr\ trace-table\ final-state\ trace-table'\ final-state'$ **where**
 $candidate$:
 $trace-fri-supported-root-trace-table-candidate-state\ s\ fr\ trace-table$
 $final-state$

```

and candidate':
  trace-fri-supported-root-trace-table-candidate-state s fr trace-table'
  final-state'
and collision:
  hash-map-output-collision final-state  $\vee$ 
  hash-map-output-collision final-state'
by blast
from collision show ?thesis
proof
  assume collision-final: hash-map-output-collision final-state
  from candidate obtain result composition-table as query-idxs trace-bs dg
    comp-bs f-fri-roots f-final header-as composition-fri-roots final
    rest where
  outcome:
    Some (result, final-state)  $\in$  set-dist (execute verify-monad s)
  and bound:
    accepted-with-bound-tables s (Some (result, final-state))
    trace-table composition-table as query-idxs
  unfolding trace-fri-supported-root-trace-table-candidate-state-def
  by blast
have supported-hash-output-collision-possible s
  by (rule
    accepted-with-bound-tables-hash-collision-imp-supported-hash-output-collision-possible
    [OF outcome bound collision-final])
  then show ?thesis by simp
next
  assume collision-final': hash-map-output-collision final-state'
  from candidate' obtain result' composition-table' as' query-idxs'
    trace-bs' dg' comp-bs' f-fri-roots' f-final' header-as'
    composition-fri-roots' final' rest' where
  outcome':
    Some (result', final-state')  $\in$  set-dist (execute verify-monad s)
  and bound':
    accepted-with-bound-tables s (Some (result', final-state'))
    trace-table' composition-table' as' query-idxs'
  unfolding trace-fri-supported-root-trace-table-candidate-state-def
  by blast
have supported-hash-output-collision-possible s
  by (rule
    accepted-with-bound-tables-hash-collision-imp-supported-hash-output-collision-possible
    [OF outcome' bound' collision-final'])
  then show ?thesis by simp
qed
next
  assume trace-fri-supported-pairwise-coupling-bad s
  then show ?thesis by simp
qed
qed

```

lemma *trace-fri-supported-root-candidate-unique-if-no-pairwise-merkle-bad*:
assumes *no-bad*: $\neg \text{trace-fri-supported-root-pairwise-merkle-bad } s \text{ fr}$
shows
 $\exists \text{ trace-table.}$
 $\text{trace-fri-supported-root-trace-table-candidates } s \text{ fr} \subseteq \{\text{trace-table}\}$
proof (rule *ccontr*)
assume *not-unique*:
 $\neg (\exists \text{ trace-table.}$
 $\text{trace-fri-supported-root-trace-table-candidates } s \text{ fr} \subseteq \{\text{trace-table}\})$
have *trace-fri-supported-root-pairwise-merkle-bad* *s fr*
using
 $\text{trace-fri-supported-root-trace-table-candidates-not-singleton-imp-pairwise-merkle-bad}$
 $[OF \text{ not-unique}]$
unfolding *trace-fri-supported-root-pairwise-merkle-bad-def*
by *blast*
then show *False*
using *no-bad* **by** *contradiction*
qed

lemma *trace-fri-supported-root-candidate-unique-if-no-global-pairwise-merkle-bad*:
assumes *no-bad*: $\neg \text{trace-fri-supported-pairwise-merkle-bad } s$
shows
 $\exists \text{ trace-table.}$
 $\text{trace-fri-supported-root-trace-table-candidates } s \text{ fr} \subseteq \{\text{trace-table}\}$
proof (rule *trace-fri-supported-root-candidate-unique-if-no-pairwise-merkle-bad*)
show $\neg \text{trace-fri-supported-root-pairwise-merkle-bad } s \text{ fr}$
using *no-bad* **unfolding** *trace-fri-supported-pairwise-merkle-bad-def* **by** *blast*
qed

definition *trace-fri-root-union-bad-sets*
 $:: ('f, 'a) \text{ protocol-channel-scheme} \Rightarrow$
 $('f \text{ list} \Rightarrow 'f \text{ list set}) \Rightarrow 'f \Rightarrow 'f \text{ list set}$
where
 $\text{trace-fri-root-union-bad-sets } s \text{ bad-sets fr} =$
 $\{\text{trace-bs} \in \text{fri-challenge-space } (\text{ceil-log } \text{clength}).$
 $\exists \text{ trace-table} \in \text{trace-fri-root-trace-table-candidates } s \text{ fr.}$
 $\text{trace-bs} \in \text{bad-sets trace-table}\}$

definition *trace-fri-supported-root-union-bad-sets*
 $:: ('f, 'a) \text{ protocol-channel-scheme} \Rightarrow$
 $('f \text{ list} \Rightarrow 'f \text{ list set}) \Rightarrow 'f \Rightarrow 'f \text{ list set}$
where
 $\text{trace-fri-supported-root-union-bad-sets } s \text{ bad-sets fr} =$
 $\{\text{trace-bs} \in \text{fri-challenge-space } (\text{ceil-log } \text{clength}).$
 $\exists \text{ trace-table} \in$
 $\text{trace-fri-supported-root-trace-table-candidates } s \text{ fr.}$

$trace\text{-}bs \in bad\text{-}sets\ trace\text{-}table\}$

definition *trace-fri-supported-root-partial-union-bad-sets*

$:: ('f, 'a)\ protocol\text{-}channel\text{-}scheme \Rightarrow$
 $('f\ list \Rightarrow 'f\ list\ set) \Rightarrow 'f \Rightarrow 'f\ list\ set$

where

trace-fri-supported-root-partial-union-bad-sets $s\ bad\text{-}sets\ fr =$
 $\{ trace\text{-}bs \in fri\text{-}challenge\text{-}space\ (ceil\text{-}log\ clength).\$
 $\exists\ trace\text{-}table \in$
 $trace\text{-}fri\text{-}supported\text{-}root\text{-}partial\text{-}trace\text{-}table\text{-}candidates\ s\ fr.$
 $trace\text{-}bs \in bad\text{-}sets\ trace\text{-}table\}$

lemma *trace-fri-root-union-bad-sets-subset:*

trace-fri-root-union-bad-sets $s\ bad\text{-}sets\ fr \subseteq$
 $fri\text{-}challenge\text{-}space\ (ceil\text{-}log\ clength)$

unfolding *trace-fri-root-union-bad-sets-def* **by** *auto*

lemma *trace-fri-supported-root-union-bad-sets-subset:*

trace-fri-supported-root-union-bad-sets $s\ bad\text{-}sets\ fr \subseteq$
 $fri\text{-}challenge\text{-}space\ (ceil\text{-}log\ clength)$

unfolding *trace-fri-supported-root-union-bad-sets-def* **by** *auto*

lemma *trace-fri-supported-root-partial-union-bad-sets-subset:*

trace-fri-supported-root-partial-union-bad-sets $s\ bad\text{-}sets\ fr \subseteq$
 $fri\text{-}challenge\text{-}space\ (ceil\text{-}log\ clength)$

unfolding *trace-fri-supported-root-partial-union-bad-sets-def* **by** *auto*

lemma *trace-fri-challenge-set-hit-imp-root-union-list-set-hit:*

assumes *hit:* *trace-fri-challenge-set-hit* $s\ bad\text{-}sets\ out$

shows

trace-fri-root-list-set-hit s
 $(trace\text{-}fri\text{-}root\text{-}union\text{-}bad\text{-}sets\ s\ bad\text{-}sets)\ out$

proof —

from *hit* **obtain** *trace-table* *composition-table* *as* *query-idxs* *trace-bs* *dg* *comp-bs*

where *bound:*

accepted-with-bound-tables $s\ out\ trace\text{-}table\ composition\text{-}table\ as$
 $query\text{-}idxs$

and *challenges:* *accepted-fri-challenges* $s\ out\ trace\text{-}bs\ dg\ comp\text{-}bs$

and *trace-bs-bad:* $trace\text{-}bs \in bad\text{-}sets\ trace\text{-}table$

unfolding *trace-fri-challenge-set-hit-def* **by** *blast*

have *header-ex:*

$\exists fr\ f\text{-}fri\text{-}roots\ f\text{-}final\ header\text{-}as\ composition\text{-}fri\text{-}roots\ final\ rest.$
 $verifier\text{-}header\text{-}transcript\ s\ fr\ f\text{-}fri\text{-}roots\ f\text{-}final\ header\text{-}as\ dg$
 $composition\text{-}fri\text{-}roots\ final\ rest$

proof —

from *challenges[unfolded accepted-fri-challenges-def]* **show** *?thesis*

proof (*elim exE conjE*)

fix *result0* *final-state0* *fr0* *f-fl0* *f-final0* *as0* *fl0* *final0*
 $query\text{-}state0$

```

assume out0: out = Some (result0, final-state0)
and header0:
  verifier-header-transcript s fr0 (map snd f-fl0) f-final0 as0 dg
  (map snd fl0) final0 (PTranscript query-state0)
show ?thesis
  apply (rule exI[where x=fr0])
  apply (rule exI[where x=map snd f-fl0])
  apply (rule exI[where x=f-final0])
  apply (rule exI[where x=as0])
  apply (rule exI[where x=map snd fl0])
  apply (rule exI[where x=final0])
  apply (rule exI[where x=PTranscript query-state0])
  apply (rule header0)
  done
qed
qed
then obtain fr f-fri-roots f-final header-as composition-fri-roots final
  rest where
  header:
    verifier-header-transcript s fr f-fri-roots f-final header-as dg
    composition-fri-roots final rest
  by blast
have candidate:
  trace-table ∈ trace-fri-root-trace-table-candidates s fr
  unfolding trace-fri-root-trace-table-candidates-def
  using bound challenges header by blast
have trace-bs-space:
  trace-bs ∈ fri-challenge-space (ceil-log clength)
  by (rule accepted-fri-challenges-trace-space[OF challenges])
have trace-bs-union:
  trace-bs ∈ trace-fri-root-union-bad-sets s bad-sets fr
  unfolding trace-fri-root-union-bad-sets-def
  using trace-bs-space candidate trace-bs-bad by blast
show ?thesis
  unfolding trace-fri-root-list-set-hit-def
  using challenges header trace-bs-union by blast
qed

lemma trace-fri-challenge-set-hit-imp-supported-root-union-list-set-hit:
  assumes outcome: out ∈ set-dist (execute verify-monad s)
  and hit: trace-fri-challenge-set-hit s bad-sets out
  shows
    trace-fri-root-list-set-hit s
    (trace-fri-supported-root-union-bad-sets s bad-sets) out
proof –
  from hit obtain trace-table composition-table as query-idxs trace-bs dg comp-bs
  where bound:
    accepted-with-bound-tables s out trace-table composition-table as
    query-idxs

```

```

    and challenges: accepted-fri-challenges s out trace-bs dg comp-bs
    and not-low:  $\neg$  trace-table-low-degree trace-table
    and trace-bs-bad: trace-bs  $\in$  bad-sets trace-table
    unfolding trace-fri-challenge-set-hit-def by blast
  have header-ex:
     $\exists$  fr f-fri-roots f-final header-as composition-fri-roots final rest.
    verifier-header-transcript s fr f-fri-roots f-final header-as dg
    composition-fri-roots final rest
  proof -
    from challenges[unfolded accepted-fri-challenges-def] show ?thesis
  proof (elim exE conjE)
    fix result0 final-state0 fr0 f-fl0 f-final0 as0 fl0 final0
    query-state0
  assume
    header0:
      verifier-header-transcript s fr0 (map snd f-fl0) f-final0 as0 dg
      (map snd fl0) final0 (PTranscript query-state0)
  show ?thesis
    apply (rule exI[where x=fr0])
    apply (rule exI[where x=map snd f-fl0])
    apply (rule exI[where x=f-final0])
    apply (rule exI[where x=as0])
    apply (rule exI[where x=map snd fl0])
    apply (rule exI[where x=final0])
    apply (rule exI[where x=PTranscript query-state0])
    apply (rule header0)
  done
qed
qed
then obtain fr f-fri-roots f-final header-as composition-fri-roots final
  rest where
  header:
    verifier-header-transcript s fr f-fri-roots f-final header-as dg
    composition-fri-roots final rest
  by blast
have candidate:
  trace-table  $\in$  trace-fri-supported-root-trace-table-candidates s fr
  unfolding trace-fri-supported-root-trace-table-candidates-def
  using outcome bound challenges header by blast
have trace-bs-space:
  trace-bs  $\in$  fri-challenge-space (ceil-log clength)
  by (rule accepted-fri-challenges-trace-space[OF challenges])
have trace-bs-union:
  trace-bs  $\in$  trace-fri-supported-root-union-bad-sets s bad-sets fr
  unfolding trace-fri-supported-root-union-bad-sets-def
  using trace-bs-space candidate trace-bs-bad by blast
show ?thesis
  unfolding trace-fri-root-list-set-hit-def
  using challenges header trace-bs-union by blast

```


qed

lemma *trace-fri-challenge-set-hit-imp-partial-union-or-collision:*
assumes *outcome:* $out \in \text{set-dist } (\text{execute verify-monad } s)$
and *hit:* *trace-fri-challenge-set-hit* *s* *bad-sets* *out*
shows
trace-fri-root-list-set-hit *s*
 $(\text{trace-fri-supported-root-partial-union-bad-sets } s \text{ bad-sets}) \text{ out} \vee$
hash-map-output-collision-bad *s* *out*
proof –
from *hit* **obtain** *trace-table* *composition-table* *as* *query-idxs* *trace-bs* *dg* *comp-bs*
where *bound:*
accepted-with-bound-tables *s* *out* *trace-table* *composition-table* *as*
query-idxs
and *challenges:* *accepted-fri-challenges* *s* *out* *trace-bs* *dg* *comp-bs*
and *not-low:* $\neg \text{trace-table-low-degree } \text{trace-table}$
and *trace-bs-bad:* *trace-bs* $\in \text{bad-sets } \text{trace-table}$
unfolding *trace-fri-challenge-set-hit-def* **by** *blast*
from *accepted-with-bound-tables-imp-accepted*[*OF* *bound*]
obtain *result* *final-state* **where** *out-eq:* $out = \text{Some } (result, final-state)$
unfolding *accepted-def* **by** $(\text{cases } out) \text{ auto}$
show *?thesis*
proof $(\text{cases } \text{hash-map-output-collision } final-state)$
case *True*
then **have** *hash-map-output-collision-bad* *s* *out*
unfolding *hash-map-output-collision-bad-def* *accepted-def* *out-eq* **by** *simp*
then **show** *?thesis* **by** *simp*
next
case *clean:* *False*
have *header-ex:*
 $\exists fr \text{ f-fri-roots } f\text{-final } header\text{-as } composition\text{-fri-roots } final \text{ rest.}$
 $verifier\text{-header-transcript } s \text{ fr } f\text{-fri-roots } f\text{-final } header\text{-as } dg$
 $composition\text{-fri-roots } final \text{ rest}$
proof –
from *challenges*[*unfolded* *accepted-fri-challenges-def*] **show** *?thesis*
proof $(\text{elim } exE \text{ conjE})$
fix *result0* *final-state0* *fr0* *f-fl0* *f-final0* *as0* *fl0* *final0*
query-state0
assume
header0:
 $verifier\text{-header-transcript } s \text{ fr0 } (\text{map } snd \text{ f-fl0}) \text{ f-final0 } as0 \text{ dg}$
 $(\text{map } snd \text{ fl0}) \text{ final0 } (PTranscript \text{ query-state0})$
show *?thesis*
apply $(\text{rule } exI[\text{where } x=fr0])$
apply $(\text{rule } exI[\text{where } x=\text{map } snd \text{ f-fl0}])$
apply $(\text{rule } exI[\text{where } x=f\text{-final0}])$
apply $(\text{rule } exI[\text{where } x=as0])$
apply $(\text{rule } exI[\text{where } x=\text{map } snd \text{ fl0}])$
apply $(\text{rule } exI[\text{where } x=final0])$

```

    apply (rule exI[where  $x = P\text{Transcript } \text{query-state0}$ ])
    apply (rule header0)
  done
qed
qed
then obtain  $fr$   $f\text{-fri-roots}$   $f\text{-final}$   $header\text{-as}$   $composition\text{-fri-roots}$   $final$ 
   $rest$  where
   $header$ :
     $verifier\text{-header-transcript } s$   $fr$   $f\text{-fri-roots}$   $f\text{-final}$   $header\text{-as}$   $dg$ 
     $composition\text{-fri-roots}$   $final$   $rest$ 
  by  $blast$ 
from  $bound$  obtain  $fr'$   $f'\text{-fri-roots'}$   $f'\text{-final'}$   $dg'$ 
   $composition\text{-fri-roots'}$   $final'$   $rest'$  where
   $header\text{-bound}$ :
     $verifier\text{-header-transcript } s$   $fr'$   $f'\text{-fri-roots'}$   $f'\text{-final'}$   $as$   $dg'$ 
     $composition\text{-fri-roots'}$   $final'$   $rest'$ 
  and  $bind$ :
     $merkle\text{-root-binds-table } fr'$   $trace\text{-table}$   $final\text{-state}$ 
  unfolding  $accepted\text{-with-bound-tables-def}$   $out\text{-eq}$  by  $blast$ 
have  $fr\text{-eq}$ :  $fr' = fr$ 
  using  $verifier\text{-header-transcript-unique}[OF header\text{-bound } header]$  by  $simp$ 
have  $outcome\text{-some}$ :
   $Some (result, final\text{-state}) \in set\text{-dist } (execute\ verify\text{-monad } s)$ 
  using  $outcome\ out\text{-eq}$  by  $simp$ 
have  $bound\text{-some}$ :
   $accepted\text{-with-bound-tables } s (Some (result, final\text{-state}))$ 
   $trace\text{-table}$   $composition\text{-table}$   $as$   $query\text{-idxs}$ 
  using  $bound\ out\text{-eq}$  by  $simp$ 
have  $challenges\text{-some}$ :
   $accepted\text{-fri-challenges } s (Some (result, final\text{-state}))$   $trace\text{-bs}$   $dg$ 
   $comp\text{-bs}$ 
  using  $challenges\ out\text{-eq}$  by  $simp$ 
have  $bind\text{-fr}$ :  $merkle\text{-root-binds-table } fr$   $trace\text{-table}$   $final\text{-state}$ 
  using  $bind\ fr\text{-eq}$  by  $simp$ 
have  $state$ :
   $trace\text{-fri-supported-root-trace-table-candidate-state } s$   $fr$   $trace\text{-table}$ 
   $final\text{-state}$ 
  unfolding  $trace\text{-fri-supported-root-trace-table-candidate-state-def}$ 
  using  $outcome\text{-some}$   $bound\text{-some}$   $challenges\text{-some}$   $header$   $bind\text{-fr}$  by  $blast$ 
have  $candidate$ :
   $trace\text{-table} \in$ 
   $trace\text{-fri-supported-root-partial-trace-table-candidates } s$   $fr$ 
  by (rule
     $trace\text{-fri-supported-root-trace-table-candidate-state-imp-partial-candidate}$ 
     $[OF state\ clean]$ )
have  $trace\text{-bs-space}$ :
   $trace\text{-bs} \in fri\text{-challenge-space } (ceil\text{-log } clength)$ 
  by (rule  $accepted\text{-fri-challenges-trace-space}[OF challenges]$ )
have  $trace\text{-bs-union}$ :

```

```

    trace-bs ∈
      trace-fri-supported-root-partial-union-bad-sets s bad-sets fr
    unfolding trace-fri-supported-root-partial-union-bad-sets-def
    using trace-bs-space candidate trace-bs-bad by blast
  show ?thesis
    unfolding trace-fri-root-list-set-hit-def
    using challenges header trace-bs-union by blast
qed
qed

lemma alpha-bad-set-hit-imp-partial-union-or-collision:
  assumes outcome: out ∈ set-dist (execute verify-monad s)
  and hit: alpha-bad-set-hit s bad-sets out
  and subset:  $\bigwedge \text{trace-table. bad-sets trace-table} \subseteq \text{alpha-space}$ 
  shows
    alpha-header-list-set-hit s
      (alpha-header-supported-partial-union-bad-sets s bad-sets) out ∨
    hash-map-output-collision-bad s out
proof –
  from hit obtain trace-table composition-table as query-idxs where
    bound:
      accepted-with-bound-tables s out trace-table composition-table
      as query-idxs
  and as-bad: as ∈ bad-sets trace-table
  unfolding alpha-bad-set-hit-def by blast
from accepted-with-bound-tables-imp-accepted[OF bound]
obtain result final-state where out-eq: out = Some (result, final-state)
  unfolding accepted-def by (cases out) auto
show ?thesis
proof (cases hash-map-output-collision final-state)
  case True
  then have hash-map-output-collision-bad s out
    unfolding hash-map-output-collision-bad-def accepted-def out-eq by simp
  then show ?thesis by simp
next
  case clean: False
  from bound obtain fr f-fri-roots f-final dg composition-fri-roots final
    rest where
    header:
      verifier-header-transcript s fr f-fri-roots f-final as dg
      composition-fri-roots final rest
  and bind:
    merkle-root-binds-table fr trace-table final-state
  unfolding accepted-with-bound-tables-def out-eq by blast
have shape:
  accepted-transcript-shape s out as query-idxs
  using accepted-with-bound-tables-imp-accepted-with-tables[OF bound]
    accepted-with-tables-imp-accepted-transcript-shape
  by blast

```

```

have state:
  alpha-header-supported-trace-table-candidate-state s fr f-fri-roots
  f-final trace-table final-state
unfolding alpha-header-supported-trace-table-candidate-state-def
by (intro exI[of - result] exI[of - composition-table]
      exI[of - as] exI[of - query-idxs] exI[of - dg]
      exI[of - composition-fri-roots] exI[of - final]
      exI[of - rest])
  (use outcome out-eq bound header bind in simp)
have candidate:
  trace-table ∈
  alpha-header-supported-partial-trace-table-candidates s fr
  f-fri-roots f-final
by (rule
      alpha-header-supported-trace-table-candidate-state-imp-partial-candidate
      [OF state clean])
have as-union:
  as ∈
  alpha-header-supported-partial-union-bad-sets s bad-sets fr
  f-fri-roots f-final
unfolding alpha-header-supported-partial-union-bad-sets-def
using as-bad candidate subset[of trace-table] by auto
show ?thesis
unfolding alpha-header-list-set-hit-def
using shape out-eq header as-union by blast
qed
qed

lemma wp-trace-fri-challenge-set-hit-bound-via-partial-union-or-collision:
fixes C H :: prob
assumes partial-bound:
  wp-event verify-monad
  (trace-fri-root-list-set-hit s
   (trace-fri-supported-root-partial-union-bad-sets s bad-sets)) s ≤ C
and collision-bound:
  wp-event verify-monad (hash-map-output-collision-bad s) s ≤ H
shows
  wp-event verify-monad (trace-fri-challenge-set-hit s bad-sets) s ≤
  C + H
proof –
have mono:
  wp-event verify-monad (trace-fri-challenge-set-hit s bad-sets) s ≤
  wp-event verify-monad
  (λout.
   trace-fri-root-list-set-hit s
   (trace-fri-supported-root-partial-union-bad-sets s bad-sets) out ∨
   hash-map-output-collision-bad s out) s
by (rule wp-event-mono-on-support)
  (use trace-fri-challenge-set-hit-imp-partial-union-or-collision in blast)

```

also have $\dots \leq C + H$
 proof –
 have *wp-event verify-monad*
 ($\lambda out.$
 trace-fri-root-list-set-hit s
 (*trace-fri-supported-root-partial-union-bad-sets s bad-sets*) *out* $\vee$
 hash-map-output-collision-bad s out) $s \leq$
 wp-event verify-monad
 (*trace-fri-root-list-set-hit s*
 (*trace-fri-supported-root-partial-union-bad-sets s bad-sets*)) $s +$
 wp-event verify-monad (hash-map-output-collision-bad s) s
 by (*rule wp-event-union-bound*)
 also have $\dots \leq C + H$
 by (*intro add-mono partial-bound collision-bound*)
 finally show ?thesis .
 qed
 finally show ?thesis .
 qed

lemma *accepted-verifier-initial-imp-transcript-target-hit-on-support:*
 assumes *support:*
 $out \in \text{set-dist } (\text{execute verify-monad } (\text{verifier-initial-state } tr))$
 and *acc:* *accepted out*
 shows *hash-new-output-hit-event (set tr) (verifier-initial-state tr) out*
 proof –
 from *acc* obtain *result final-state* where *out-eq:*
 $out = \text{Some } (result, final-state)$
 unfolding *accepted-def* by (*cases out*) auto
 have *outcome:*
 $\text{Some } (result, final-state) \in$
 $\text{set-dist } (\text{execute verify-monad } (\text{verifier-initial-state } tr))$
 using *support out-eq* by *simp*
 have *hit:*
 hash-map-new-output-hit (set tr) (verifier-initial-state tr)
 final-state
 by (*rule verify-monad-verifier-initial-trace-root-target-hit[OF outcome]*)
 show ?thesis
 unfolding *hash-new-output-hit-event-def out-eq*
 using *hit* by *simp*
 qed

lemma *wp-alpha-bad-set-hit-bound-via-partial-union-or-collision:*
 fixes $C H :: \text{prob}$
 assumes *subset:* $\bigwedge \text{trace-table. bad-sets trace-table} \subseteq \text{alpha-space}$
 and *partial-bound:*
 wp-event verify-monad
 (*alpha-header-list-set-hit s*
 (*alpha-header-supported-partial-union-bad-sets s bad-sets*)) $s \leq C$
 and *collision-bound:*

$wp\text{-event } verify\text{-monad } (hash\text{-map-output-collision-bad } s) \ s \leq H$
shows
 $wp\text{-event } verify\text{-monad } (alpha\text{-bad-set-hit } s \ bad\text{-sets}) \ s \leq C + H$
proof –
have *mono*:
 $wp\text{-event } verify\text{-monad } (alpha\text{-bad-set-hit } s \ bad\text{-sets}) \ s \leq$
 $wp\text{-event } verify\text{-monad}$
 $(\lambda out.$
 $alpha\text{-header-list-set-hit } s$
 $(alpha\text{-header-supported-partial-union-bad-sets } s \ bad\text{-sets}) \ out \ \vee$
 $hash\text{-map-output-collision-bad } s \ out) \ s$
proof (rule *wp-event-mono-on-support*)
fix *out*
assume *support*: $out \in set\text{-dist } (execute \ verify\text{-monad } s)$
and *hit*: $alpha\text{-bad-set-hit } s \ bad\text{-sets} \ out$
show
 $alpha\text{-header-list-set-hit } s$
 $(alpha\text{-header-supported-partial-union-bad-sets } s \ bad\text{-sets}) \ out \ \vee$
 $hash\text{-map-output-collision-bad } s \ out$
by (rule *alpha-bad-set-hit-imp-partial-union-or-collision*
 $[OF \ support \ hit \ subset]$)
qed
also have $\dots \leq C + H$
proof –
have $wp\text{-event } verify\text{-monad}$
 $(\lambda out.$
 $alpha\text{-header-list-set-hit } s$
 $(alpha\text{-header-supported-partial-union-bad-sets } s \ bad\text{-sets}) \ out \ \vee$
 $hash\text{-map-output-collision-bad } s \ out) \ s \leq$
 $wp\text{-event } verify\text{-monad}$
 $(alpha\text{-header-list-set-hit } s$
 $(alpha\text{-header-supported-partial-union-bad-sets } s \ bad\text{-sets})) \ s +$
 $wp\text{-event } verify\text{-monad } (hash\text{-map-output-collision-bad } s) \ s$
by (rule *wp-event-union-bound*)
also have $\dots \leq C + H$
by (intro *add-mono partial-bound collision-bound*)
finally show *?thesis* .
qed
finally show *?thesis* .
qed

lemma *wp-trace-fri-challenge-set-hit-bound-via-supported-partial-union-or-collision*:
fixes $C \ H :: prob$
assumes *future*: $trace\text{-fri-future-fresh } s$
and *union-bound*:
 $\bigwedge fr. \ nnreal$
 $(card \ (trace\text{-fri-supported-root-partial-union-bad-sets } s \ bad\text{-sets} \ fr)) \ /$
 $nnreal \ (CARD('f) \wedge ceil\text{-log } clength) \leq C$
and *collision-bound*:

$wp\text{-}event\ verify\text{-}monad\ (hash\text{-}map\text{-}output\text{-}collision\text{-}bad\ s)\ s \leq H$
shows
 $wp\text{-}event\ verify\text{-}monad\ (trace\text{-}fri\text{-}challenge\text{-}set\text{-}hit\ s\ bad\text{-}sets)\ s \leq C + H$
proof –
have *partial-bound*:
 $wp\text{-}event\ verify\text{-}monad$
 $(trace\text{-}fri\text{-}root\text{-}list\text{-}set\text{-}hit\ s$
 $(trace\text{-}fri\text{-}supported\text{-}root\text{-}partial\text{-}union\text{-}bad\text{-}sets\ s\ bad\text{-}sets))\ s \leq C$
by (rule *wp-verify-monad-trace-fri-root-list-set-bound*[*OF future*])
 $(use\ trace\text{-}fri\text{-}supported\text{-}root\text{-}partial\text{-}union\text{-}bad\text{-}sets\text{-}subset\ union\text{-}bound$
 $in\ simp\text{-}all)$
show *?thesis*
by (rule *wp-trace-fri-challenge-set-hit-bound-via-partial-union-or-collision*
 $[OF\ partial\text{-}bound\ collision\text{-}bound])$
qed

lemma *wp-alpha-bad-set-hit-bound-via-supported-partial-union-or-collision*:
fixes $C\ H :: prob$
assumes *future*: $\alpha\text{-future-fresh}\ s$
and *subset*: $\bigwedge trace\text{-}table.\ bad\text{-}sets\ trace\text{-}table \subseteq \alpha\text{-space}$
and *union-bound*:
 $\bigwedge fr\ f\text{-fri-roots}\ f\text{-final}.$
 $nnreal$
 $(card$
 $(\alpha\text{-header-supported-partial-union-bad-sets}\ s\ bad\text{-}sets\ fr$
 $f\text{-fri-roots}\ f\text{-final}))\ /\$
 $nnreal\ (card\ \alpha\text{-space}) \leq C$
and *collision-bound*:
 $wp\text{-}event\ verify\text{-}monad\ (hash\text{-}map\text{-}output\text{-}collision\text{-}bad\ s)\ s \leq H$
shows $wp\text{-}event\ verify\text{-}monad\ (\alpha\text{-bad-set-hit}\ s\ bad\text{-}sets)\ s \leq C + H$
proof –
have *partial-bound*:
 $wp\text{-}event\ verify\text{-}monad$
 $(\alpha\text{-header-list-set-hit}\ s$
 $(\alpha\text{-header-supported-partial-union-bad-sets}\ s\ bad\text{-}sets))\ s \leq C$
by (rule *wp-verify-monad-alpha-header-list-set-bound*[*OF future*])
 $(use\ \alpha\text{-header-supported-partial-union-bad-sets-subset-alpha-space}$
 $union\text{-}bound\ in\ simp\text{-}all)$
show *?thesis*
by (rule *wp-alpha-bad-set-hit-bound-via-partial-union-or-collision*
 $[OF\ subset\ partial\text{-}bound\ collision\text{-}bound])$
qed

lemma *trace-fri-supported-root-partial-union-bad-sets-fraction-bound-if-unique-candidate*:
fixes $C :: prob$
assumes *unique*:
 $\exists trace\text{-}table.$
 $trace\text{-}fri\text{-}supported\text{-}root\text{-}partial\text{-}trace\text{-}table\text{-}candidates\ s\ fr \subseteq$

```

      {trace-table}
    and subset:
       $\bigwedge \text{trace-table. bad-sets trace-table} \subseteq$ 
        fri-challenge-space (ceil-log clength)
    and bound:
       $\bigwedge \text{trace-table.}$ 
        nnreal (card (bad-sets trace-table)) /
        nnreal (CARD('f)  $\wedge$  ceil-log clength)  $\leq C$ 
  shows
    nnreal
      (card (trace-fri-supported-root-partial-union-bad-sets s bad-sets fr)) /
      nnreal (CARD('f)  $\wedge$  ceil-log clength)  $\leq C$ 
  proof -
  from unique obtain trace-table where candidates:
    trace-fri-supported-root-partial-trace-table-candidates s fr  $\subseteq$ 
      {trace-table}
  by blast
  have union-subset:
    trace-fri-supported-root-partial-union-bad-sets s bad-sets fr  $\subseteq$ 
      bad-sets trace-table
  unfolding trace-fri-supported-root-partial-union-bad-sets-def
  using candidates by auto
  have finite-bad: finite (bad-sets trace-table)
  by (rule finite-subset[OF subset finite-fri-challenge-space])
  have card-le:
    card (trace-fri-supported-root-partial-union-bad-sets s bad-sets fr)  $\leq$ 
      card (bad-sets trace-table)
  by (rule card-mono[OF finite-bad union-subset])
  have nnreal
    (card
      (trace-fri-supported-root-partial-union-bad-sets s bad-sets fr)) /
    nnreal (CARD('f)  $\wedge$  ceil-log clength)  $\leq$ 
    nnreal (card (bad-sets trace-table)) /
    nnreal (CARD('f)  $\wedge$  ceil-log clength)
  by (rule nnreal-nat-divide-right-mono[OF card-le])
  also have ...  $\leq C$ 
  by (rule bound)
  finally show ?thesis .
qed

end

end

```

```

theory Soundness-Reductions-Composition
  imports Soundness-Reductions-Trace
begin

```

Composition-table, query-header, and supported candidate reductions.

context *soundness*
begin

definition *trace-fri-bad-challenge-sets-bounded*
 $:: ('f \text{ list} \Rightarrow 'f \text{ list set}) \Rightarrow \text{bool}$
where
 $\text{trace-fri-bad-challenge-sets-bounded bad-sets} \longleftrightarrow$
 $(\forall \text{ trace-table.}$
 $\text{bad-sets trace-table} \subseteq \text{fri-challenge-space (ceil-log clength)} \wedge$
 $\text{nnreal (card (bad-sets trace-table)) /}$
 $\text{nnreal (CARD('f) ^ ceil-log clength)} \leq \text{trace-fri-error})$

definition *composition-fri-bad-challenge-sets-bounded*
 $:: ('f \Rightarrow 'f \text{ list} \Rightarrow 'f \text{ list set}) \Rightarrow \text{bool}$
where
 $\text{composition-fri-bad-challenge-sets-bounded bad-sets} \longleftrightarrow$
 $(\forall \text{ dg composition-table.}$
 $\text{bad-sets dg composition-table} \subseteq$
 $\text{fri-challenge-space (ceil-log (to-nat dg + 1))} \wedge$
 $\text{nnreal (card (bad-sets dg composition-table)) /}$
 $\text{nnreal (CARD('f) ^ ceil-log (to-nat dg + 1))} \leq$
 $\text{composition-fri-error})$

lemma *wp-trace-fri-challenge-set-hit-bound-if-supported-partial-root-candidate-unique-or-collision:*

fixes $H :: \text{prob}$
assumes *future: trace-fri-future-fresh s*
and *unique:*
 $\bigwedge_{fr.}$
 $\exists \text{ trace-table.}$
 $\text{trace-fri-supported-root-partial-trace-table-candidates s fr} \subseteq$
 $\{\text{trace-table}\}$
and *bounded: trace-fri-bad-challenge-sets-bounded bad-sets*
and *collision-bound:*
 $\text{wp-event verify-monad (hash-map-output-collision-bad s) s} \leq H$
shows
 $\text{wp-event verify-monad (trace-fri-challenge-set-hit s bad-sets) s} \leq$
 $\text{trace-fri-error} + H$

proof –
have *union-bound:*
 $\bigwedge_{fr.}$
 nnreal
 $(\text{card}$
 $(\text{trace-fri-supported-root-partial-union-bad-sets s bad-sets fr})) /$
 $\text{nnreal (CARD('f) ^ ceil-log clength)} \leq \text{trace-fri-error}$
proof –
fix fr
show
 nnreal
 $(\text{card}$

```

      (trace-fri-supported-root-partial-union-bad-sets s bad-sets fr)) /
nnreal (CARD('f) ^ ceil-log clength) ≤ trace-fri-error
by (rule
  trace-fri-supported-root-partial-union-bad-sets-fraction-bound-if-unique-candidate
  [OF unique])
  (use bounded in
    ‹auto simp: trace-fri-bad-challenge-sets-bounded-def›)
qed
show ?thesis
  by (rule
    wp-trace-fri-challenge-set-hit-bound-via-supported-partial-union-or-collision
    [OF future union-bound collision-bound])
qed

lemma alpha-header-supported-partial-union-bad-sets-fraction-bound-if-unique-candidate:
  fixes C :: prob
  assumes unique:
    ∃ trace-table.
      alpha-header-supported-partial-trace-table-candidates s fr
      f-fri-roots f-final ⊆ {trace-table}
  and subset:  $\bigwedge$  trace-table. bad-sets trace-table ⊆ alpha-space
  and bound:
     $\bigwedge$  trace-table.
      nnreal (card (bad-sets trace-table)) / nnreal (card alpha-space) ≤ C
  shows
    nnreal
      (card
        (alpha-header-supported-partial-union-bad-sets s bad-sets fr
          f-fri-roots f-final)) /
      nnreal (card alpha-space) ≤ C
proof -
  from unique obtain trace-table where candidates:
    alpha-header-supported-partial-trace-table-candidates s fr f-fri-roots
    f-final ⊆ {trace-table}
  by blast
  have union-subset:
    alpha-header-supported-partial-union-bad-sets s bad-sets fr f-fri-roots
    f-final ⊆ bad-sets trace-table
  unfolding alpha-header-supported-partial-union-bad-sets-def
  using candidates by auto
  have finite-bad: finite (bad-sets trace-table)
  by (rule finite-subset[OF subset finite-alpha-space])
  have card-le:
    card
      (alpha-header-supported-partial-union-bad-sets s bad-sets fr
        f-fri-roots f-final) ≤
      card (bad-sets trace-table)
  by (rule card-mono[OF finite-bad union-subset])
  have nnreal

```

```

    (card
      (alpha-header-supported-partial-union-bad-sets s bad-sets fr
        f-fri-roots f-final)) /
    nnreal (card alpha-space) ≤
    nnreal (card (bad-sets trace-table)) / nnreal (card alpha-space)
  by (rule nnreal-nat-divide-right-mono[OF card-le])
also have ... ≤ C
  by (rule bound)
finally show ?thesis .
qed

lemma wp-alpha-bad-set-hit-bound-if-supported-partial-header-candidate-unique-or-collision:
  fixes C H :: prob
  assumes future: alpha-future-fresh s
  and unique:
     $\bigwedge fr f-fri-roots f-final.$ 
     $\exists trace-table.$ 
    alpha-header-supported-partial-trace-table-candidates s fr
     $f-fri-roots f-final \subseteq \{trace-table\}$ 
  and subset:  $\bigwedge trace-table. bad-sets trace-table \subseteq alpha-space$ 
  and bounded:
     $\bigwedge trace-table.$ 
     $nnreal (card (bad-sets trace-table)) / nnreal (card alpha-space) \leq C$ 
  and collision-bound:
    wp-event verify-monad (hash-map-output-collision-bad s) s ≤ H
  shows wp-event verify-monad (alpha-bad-set-hit s bad-sets) s ≤ C + H
proof -
  have union-bound:
     $\bigwedge fr f-fri-roots f-final.$ 
    nnreal
      (card
        (alpha-header-supported-partial-union-bad-sets s bad-sets fr
          f-fri-roots f-final)) /
      nnreal (card alpha-space) ≤ C
  proof -
    fix fr f-fri-roots f-final
    show
      nnreal
        (card
          (alpha-header-supported-partial-union-bad-sets s bad-sets fr
            f-fri-roots f-final)) /
        nnreal (card alpha-space) ≤ C
    by (rule
        alpha-header-supported-partial-union-bad-sets-fraction-bound-if-unique-candidate
        [OF unique subset bounded])
  qed
show ?thesis
  by (rule wp-alpha-bad-set-hit-bound-via-supported-partial-union-or-collision)

```

[OF future subset union-bound collision-bound])

qed

lemma *composition-randomization-bad-bound-if-alpha-header-supported-partial-candidate-unique-or-collision:*
fixes $H :: \text{prob}$
assumes *future: alpha-future-fresh s*
and *unique:*
 $\bigwedge \text{fr. } f\text{-fri-roots } f\text{-final.}$
 $\exists \text{ trace-table.}$
 $\text{alpha-header-supported-partial-trace-table-candidates } s \text{ fr}$
 $f\text{-fri-roots } f\text{-final} \subseteq \{\text{trace-table}\}$
and *collision-bound:*
 $\text{wp-event verify-monad (hash-map-output-collision-bad } s) \ s \leq H$
shows
 $\text{wp-event verify-monad (composition-randomization-bad } s) \ s \leq$
 $\text{composition-error-bound} + H$

proof –
have *alpha-bound:*
 $\text{wp-event verify-monad}$
 $(\text{alpha-bad-set-hit } s \text{ composition-trace-bad-alpha-space}) \ s \leq$
 $\text{composition-error-bound} + H$
by (rule
 $\text{wp-alpha-bad-set-hit-bound-if-supported-partial-header-candidate-unique-or-collision}$
 $[OF \text{ future unique composition-trace-bad-alpha-space-subset-alpha-space}$
 $\text{composition-trace-bad-alpha-space-fraction-bound-alpha-space}$
 $\text{collision-bound}]$)
have $\text{wp-event verify-monad (composition-randomization-bad } s) \ s \leq$
 $\text{wp-event verify-monad}$
 $(\text{alpha-bad-set-hit } s \text{ composition-trace-bad-alpha-space}) \ s$
by (rule *wp-event-mono*)
 $(\text{rule composition-randomization-bad-imp-alpha-bad-set-hit})$
also have $\dots \leq \text{composition-error-bound} + H$
by (rule *alpha-bound*)
finally show *?thesis* .

qed

lemma *wp-trace-fri-challenge-set-hit-bound-via-root-union:*
fixes $C :: \text{prob}$
assumes *future: trace-fri-future-fresh s*
and *union-bound:*
 $\bigwedge \text{fr. } \text{nnreal}$
 $(\text{card (trace-fri-root-union-bad-sets } s \text{ bad-sets fr)}) /$
 $\text{nnreal (CARD('f) } ^{\wedge} \text{ceil-log clength)} \leq C$
shows $\text{wp-event verify-monad (trace-fri-challenge-set-hit } s \text{ bad-sets}) \ s \leq C$

proof –
have *root-bound:*
 $\text{wp-event verify-monad}$
 $(\text{trace-fri-root-list-set-hit } s$
 $(\text{trace-fri-root-union-bad-sets } s \text{ bad-sets})) \ s \leq C$

by (rule wp-verify-monad-trace-fri-root-list-set-bound[OF future])
 (use trace-fri-root-union-bad-sets-subset union-bound in simp-all)
 have wp-event verify-monad (trace-fri-challenge-set-hit s bad-sets) $s \leq$
 wp-event verify-monad
 (trace-fri-root-list-set-hit s
 (trace-fri-root-union-bad-sets s bad-sets)) s
 by (rule wp-event-mono)
 (rule trace-fri-challenge-set-hit-imp-root-union-list-set-hit)
 also have ... $\leq C$
 by (rule root-bound)
 finally show ?thesis .
 qed

lemma wp-trace-fri-challenge-set-hit-bound-via-supported-root-union:
 fixes $C :: \text{prob}$
 assumes future: trace-fri-future-fresh s
 and union-bound:
 $\bigwedge fr. \text{nnreal}$
 $(\text{card} (\text{trace-fri-supported-root-union-bad-sets } s \text{ bad-sets } fr)) /$
 $\text{nnreal} (\text{CARD}('f) \wedge \text{ceil-log } \text{clength}) \leq C$
 shows wp-event verify-monad (trace-fri-challenge-set-hit s bad-sets) $s \leq C$
proof –
 have root-bound:
 wp-event verify-monad
 (trace-fri-root-list-set-hit s
 (trace-fri-supported-root-union-bad-sets s bad-sets)) $s \leq C$
 by (rule wp-verify-monad-trace-fri-root-list-set-bound[OF future])
 (use trace-fri-supported-root-union-bad-sets-subset union-bound in simp-all)
 have wp-event verify-monad (trace-fri-challenge-set-hit s bad-sets) $s \leq$
 wp-event verify-monad
 (trace-fri-root-list-set-hit s
 (trace-fri-supported-root-union-bad-sets s bad-sets)) s
 by (rule wp-event-mono-on-support)
 (rule trace-fri-challenge-set-hit-imp-supported-root-union-list-set-hit)
 also have ... $\leq C$
 by (rule root-bound)
 finally show ?thesis .
 qed

lemma trace-fri-root-union-bad-sets-fraction-bound-if-unique-candidate:
 fixes $C :: \text{prob}$
 assumes unique:
 $\exists \text{trace-table.}$
 $\text{trace-fri-root-trace-table-candidates } s \text{ fr} \subseteq \{\text{trace-table}\}$
 and subset:
 $\bigwedge \text{trace-table. bad-sets trace-table} \subseteq$
 $\text{fri-challenge-space } (\text{ceil-log } \text{clength})$
 and bound:

```

 $\bigwedge \text{trace-table.}$ 
  nnreal (card (bad-sets trace-table)) /
  nnreal (CARD('f)  $\wedge$  ceil-log clength)  $\leq C$ 
shows
  nnreal (card (trace-fri-root-union-bad-sets s bad-sets fr)) /
  nnreal (CARD('f)  $\wedge$  ceil-log clength)  $\leq C$ 
proof -
  from unique obtain trace-table where candidates:
    trace-fri-root-trace-table-candidates s fr  $\subseteq \{\text{trace-table}\}$ 
  by blast
  have union-subset:
    trace-fri-root-union-bad-sets s bad-sets fr  $\subseteq$  bad-sets trace-table
  unfolding trace-fri-root-union-bad-sets-def
  using candidates by auto
  have finite-bad: finite (bad-sets trace-table)
  by (rule finite-subset[OF subset finite-fri-challenge-space])
  have card-le:
    card (trace-fri-root-union-bad-sets s bad-sets fr)  $\leq$ 
    card (bad-sets trace-table)
  by (rule card-mono[OF finite-bad union-subset])
  have nnreal (card (trace-fri-root-union-bad-sets s bad-sets fr)) /
    nnreal (CARD('f)  $\wedge$  ceil-log clength)  $\leq$ 
    nnreal (card (bad-sets trace-table)) /
    nnreal (CARD('f)  $\wedge$  ceil-log clength)
  by (rule nnreal-nat-divide-right-mono[OF card-le])
  also have ...  $\leq C$ 
  by (rule bound)
  finally show ?thesis .
qed

lemma trace-fri-supported-root-union-bad-sets-fraction-bound-if-unique-candidate:
  fixes C :: prob
  assumes unique:
     $\exists \text{trace-table.}$ 
    trace-fri-supported-root-trace-table-candidates s fr  $\subseteq$ 
     $\{\text{trace-table}\}$ 
  and subset:
     $\bigwedge \text{trace-table. bad-sets trace-table} \subseteq$ 
    fri-challenge-space (ceil-log clength)
  and bound:
     $\bigwedge \text{trace-table.}$ 
    nnreal (card (bad-sets trace-table)) /
    nnreal (CARD('f)  $\wedge$  ceil-log clength)  $\leq C$ 
  shows
    nnreal (card (trace-fri-supported-root-union-bad-sets s bad-sets fr)) /
    nnreal (CARD('f)  $\wedge$  ceil-log clength)  $\leq C$ 
proof -
  from unique obtain trace-table where candidates:
    trace-fri-supported-root-trace-table-candidates s fr  $\subseteq$ 

```

```

    {trace-table}
  by blast
have union-subset:
  trace-fri-supported-root-union-bad-sets s bad-sets fr  $\subseteq$ 
    bad-sets trace-table
  unfolding trace-fri-supported-root-union-bad-sets-def
  using candidates by auto
have finite-bad: finite (bad-sets trace-table)
  by (rule finite-subset[OF subset finite-fri-challenge-space])
have card-le:
  card (trace-fri-supported-root-union-bad-sets s bad-sets fr)  $\leq$ 
    card (bad-sets trace-table)
  by (rule card-mono[OF finite-bad union-subset])
have nnreal
  (card (trace-fri-supported-root-union-bad-sets s bad-sets fr)) /
  nnreal (CARD('f) ^ ceil-log clength)  $\leq$ 
  nnreal (card (bad-sets trace-table)) /
  nnreal (CARD('f) ^ ceil-log clength)
  by (rule nnreal-nat-divide-right-mono[OF card-le])
also have ...  $\leq C$ 
  by (rule bound)
finally show ?thesis .
qed

lemma composition-fri-challenge-set-hit-imp-list-set-hit:
  assumes hit: composition-fri-challenge-set-hit s bad-sets out
  and envelope:
     $\bigwedge dg$  composition-table. bad-sets dg composition-table  $\subseteq B$  dg
  shows composition-fri-challenge-list-set-hit s B out
proof -
  from hit obtain trace-table composition-table as query-idxs trace-bs dg comp-bs
  where challenges:
    accepted-fri-challenges s out trace-bs dg comp-bs
  and comp-bad: comp-bs  $\in$  bad-sets dg composition-table
  unfolding composition-fri-challenge-set-hit-def by blast
have comp-bs  $\in B$  dg
  using comp-bad envelope[of dg composition-table] by auto
then show ?thesis
  unfolding composition-fri-challenge-list-set-hit-def
  using challenges by blast
qed

definition composition-fri-degree-table-candidates
  :: ('f, 'a) protocol-channel-scheme  $\Rightarrow$  'f  $\Rightarrow$  'f list set
where
  composition-fri-degree-table-candidates s dg =
    { composition-table.
       $\exists$  out trace-table as query-idxs trace-bs comp-bs.
        accepted-with-bound-tables s out trace-table composition-table
    }

```

as query-idxs $\wedge$
 accepted-fri-challenges s out trace-bs dg comp-bs}

definition composition-fri-degree-union-bad-sets

:: ('f, 'a) protocol-channel-scheme $\Rightarrow$
 ('f $\Rightarrow$ 'f list $\Rightarrow$ 'f list set) $\Rightarrow$ 'f $\Rightarrow$ 'f list set

where

composition-fri-degree-union-bad-sets s bad-sets dg =
 {comp-bs $\in$ fri-challenge-space (ceil-log (to-nat dg + 1)).
 $\exists$ composition-table $\in$ composition-fri-degree-table-candidates s dg.
 comp-bs $\in$ bad-sets dg composition-table}

lemma composition-fri-degree-union-bad-sets-subset:

composition-fri-degree-union-bad-sets s bad-sets dg $\subseteq$
 fri-challenge-space (ceil-log (to-nat dg + 1))

unfolding composition-fri-degree-union-bad-sets-def **by** auto

lemma composition-fri-challenge-set-hit-imp-degree-union-list-set-hit:

assumes hit: composition-fri-challenge-set-hit s bad-sets out

shows

composition-fri-challenge-list-set-hit s
 (composition-fri-degree-union-bad-sets s bad-sets) out

proof –

from hit **obtain** trace-table composition-table as query-idxs trace-bs dg comp-bs

where bound:

accepted-with-bound-tables s out trace-table composition-table as
 query-idxs

and challenges: accepted-fri-challenges s out trace-bs dg comp-bs

and comp-bs-bad: comp-bs $\in$ bad-sets dg composition-table

unfolding composition-fri-challenge-set-hit-def **by** blast

have candidate:

composition-table $\in$ composition-fri-degree-table-candidates s dg

unfolding composition-fri-degree-table-candidates-def

using bound challenges **by** blast

have comp-bs-space:

comp-bs $\in$ fri-challenge-space (ceil-log (to-nat dg + 1))

by (rule accepted-fri-challenges-composition-space[OF challenges])

have comp-bs-union:

comp-bs $\in$ composition-fri-degree-union-bad-sets s bad-sets dg

unfolding composition-fri-degree-union-bad-sets-def

using comp-bs-space candidate comp-bs-bad **by** blast

show ?thesis

unfolding composition-fri-challenge-list-set-hit-def

using challenges comp-bs-union **by** blast

qed

lemma wp-composition-fri-challenge-set-hit-bound-via-degree-union:

fixes C :: prob

assumes future: composition-fri-future-fresh s

and *union-bound*:
 $\bigwedge dg. \text{nnreal} \quad (\text{card} (\text{composition-fri-degree-union-bad-sets } s \text{ bad-sets } dg)) / \text{nnreal} (CARD('f) \wedge \text{ceil-log} (to\text{-nat } dg + 1)) \leq C$
shows
 $\text{wp-event verify-monad} (\text{composition-fri-challenge-set-hit } s \text{ bad-sets}) s \leq C$
proof –
have *list-bound*:
 $\text{wp-event verify-monad} \quad (\text{composition-fri-challenge-list-set-hit } s \quad (\text{composition-fri-degree-union-bad-sets } s \text{ bad-sets})) s \leq C$
by (rule *wp-verify-monad-composition-fri-challenge-list-set-bound*[*OF future*])
 $(\text{use } \text{composition-fri-degree-union-bad-sets-subset } \text{union-bound} \text{ in } \text{simp-all})$
have $\text{wp-event verify-monad} (\text{composition-fri-challenge-set-hit } s \text{ bad-sets}) s \leq \text{wp-event verify-monad} \quad (\text{composition-fri-challenge-list-set-hit } s \quad (\text{composition-fri-degree-union-bad-sets } s \text{ bad-sets})) s$
by (rule *wp-event-mono*)
 $(\text{rule } \text{composition-fri-challenge-set-hit-imp-degree-union-list-set-hit})$
also have $\dots \leq C$
by (rule *list-bound*)
finally show *?thesis* .
qed

lemma *composition-fri-degree-union-bad-sets-fraction-bound-if-unique-candidate*:
fixes $C :: \text{prob}$
assumes *unique*:
 $\exists \text{composition-table.} \quad \text{composition-fri-degree-table-candidates } s \text{ dg} \subseteq \{\text{composition-table}\}$
and *subset*:
 $\bigwedge \text{composition-table.} \text{ bad-sets } dg \text{ composition-table} \subseteq \text{fri-challenge-space} (\text{ceil-log} (to\text{-nat } dg + 1))$
and *bound*:
 $\bigwedge \text{composition-table.} \quad \text{nnreal} (\text{card} (\text{bad-sets } dg \text{ composition-table})) / \text{nnreal} (CARD('f) \wedge \text{ceil-log} (to\text{-nat } dg + 1)) \leq C$
shows
 $\text{nnreal} (\text{card} (\text{composition-fri-degree-union-bad-sets } s \text{ bad-sets } dg)) / \text{nnreal} (CARD('f) \wedge \text{ceil-log} (to\text{-nat } dg + 1)) \leq C$
proof –
from *unique* **obtain** *composition-table* **where** *candidates*:
 $\text{composition-fri-degree-table-candidates } s \text{ dg} \subseteq \{\text{composition-table}\}$
by *blast*
have *union-subset*:
 $\text{composition-fri-degree-union-bad-sets } s \text{ bad-sets } dg \subseteq \text{bad-sets } dg \text{ composition-table}$
unfolding *composition-fri-degree-union-bad-sets-def*

```

using candidates by auto
have finite-bad: finite (bad-sets dg composition-table)
by (rule finite-subset[OF subset finite-fri-challenge-space])
have card-le:
  card (composition-fri-degree-union-bad-sets s bad-sets dg) ≤
    card (bad-sets dg composition-table)
by (rule card-mono[OF finite-bad union-subset])
have nnreal (card (composition-fri-degree-union-bad-sets s bad-sets dg)) /
  nnreal (CARD('f) ^ ceil-log (to-nat dg + 1)) ≤
  nnreal (card (bad-sets dg composition-table)) /
  nnreal (CARD('f) ^ ceil-log (to-nat dg + 1))
by (rule nnreal-nat-divide-right-mono[OF card-le])
also have ... ≤ C
by (rule bound)
finally show ?thesis .
qed

```

definition composition-fri-supported-root-composition-table-candidates

```

:: ('f, 'a) protocol-channel-scheme ⇒
  'f ⇒ 'f list ⇒ 'f ⇒ 'f list ⇒ 'f ⇒
  'f list ⇒ 'f list set

```

where

```

composition-fri-supported-root-composition-table-candidates s fr
f-fri-roots f-final as dg composition-fri-roots =
{ composition-table.
  ∃ out trace-table query-idxs trace-bs comp-bs final rest.
    out ∈ set-dist (execute verify-monad s) ∧
    accepted-with-bound-tables s out trace-table composition-table as
      query-idxs ∧
    accepted-fri-challenges s out trace-bs dg comp-bs ∧
    verifier-header-transcript s fr f-fri-roots f-final as dg
      composition-fri-roots final rest }

```

definition composition-fri-supported-root-composition-table-candidate-state

```

:: ('f, 'a) protocol-channel-scheme ⇒
  'f ⇒ 'f list ⇒ 'f ⇒ 'f list ⇒ 'f ⇒
  'f list ⇒ 'f list ⇒
  ('f, 'a) protocol-channel-scheme ⇒ bool

```

where

```

composition-fri-supported-root-composition-table-candidate-state s fr
f-fri-roots f-final as dg composition-fri-roots composition-table
final-state ⇔
(∃ result trace-table query-idxs trace-bs comp-bs final rest.
  Some (result, final-state) ∈ set-dist (execute verify-monad s) ∧
  accepted-with-bound-tables s (Some (result, final-state)) trace-table
    composition-table as query-idxs ∧
  accepted-fri-challenges s (Some (result, final-state)) trace-bs dg
    comp-bs ∧
  verifier-header-transcript s fr f-fri-roots f-final as dg

```

$\text{composition-fri-roots final rest} \wedge$
 $\text{composition-fri-roots} \neq [] \wedge$
 $\text{merkle-root-binds-table (hd composition-fri-roots) composition-table}$
 $\text{final-state})$

lemma *composition-fri-supported-root-composition-table-candidates-iff-state:*

$\text{composition-table} \in$
 $\text{composition-fri-supported-root-composition-table-candidates } s \text{ fr}$
 $\text{f-fri-roots f-final as dg composition-fri-roots} \longleftrightarrow$
 $(\exists \text{ final-state.}$
 $\text{composition-fri-supported-root-composition-table-candidate-state } s \text{ fr}$
 $\text{f-fri-roots f-final as dg composition-fri-roots composition-table}$
 $\text{final-state})$

proof

assume *candidate:*

$\text{composition-table} \in$
 $\text{composition-fri-supported-root-composition-table-candidates } s \text{ fr}$
 $\text{f-fri-roots f-final as dg composition-fri-roots}$

then obtain *out trace-table query-idxs trace-bs comp-bs final rest* **where**

outcome: out $\in \text{set-dist (execute verify-monad } s)$

and *bound:*

$\text{accepted-with-bound-tables } s \text{ out trace-table composition-table as}$
 query-idxs

and *challenges: accepted-fri-challenges* $s \text{ out trace-bs dg comp-bs}$

and *header:*

$\text{verifier-header-transcript } s \text{ fr f-fri-roots f-final as dg}$
 $\text{composition-fri-roots final rest}$

unfolding *composition-fri-supported-root-composition-table-candidates-def*

by *blast*

from *bound* **obtain** *result final-state fr' f-fri-roots' f-final' dg'*

$\text{composition-fri-roots' final' rest'}$ **where**

out-eq: out $= \text{Some (result, final-state)}$

and *header':*

$\text{verifier-header-transcript } s \text{ fr' f-fri-roots' f-final' as dg'}$
 $\text{composition-fri-roots' final' rest'}$

and *comp-nonempty: composition-fri-roots' $\neq []$*

and *comp-bind:*

$\text{merkle-root-binds-table (hd composition-fri-roots')}$
 $\text{composition-table final-state}$

unfolding *accepted-with-bound-tables-def* **by** *blast*

have *eqs:*

$\text{fr'} = \text{fr} \wedge$

$\text{f-fri-roots'} = \text{f-fri-roots} \wedge$

$\text{f-final'} = \text{f-final} \wedge$

$\text{dg'} = \text{dg} \wedge$

$\text{composition-fri-roots'} = \text{composition-fri-roots} \wedge$

$\text{final'} = \text{final} \wedge$

$\text{rest'} = \text{rest}$

using *verifier-header-transcript-unique[OF header' header]* **by** *simp*

have *state*:
composition-fri-supported-root-composition-table-candidate-state s fr
f-fri-roots f-final as dg composition-fri-roots composition-table
final-state
unfolding *composition-fri-supported-root-composition-table-candidate-state-def*
by (*intro exI[of - result] exI[of - trace-table]*
exI[of - query-idxs] exI[of - trace-bs] exI[of - comp-bs]
exI[of - final] exI[of - rest])
(use outcome out-eq bound challenges header comp-nonempty comp-bind eqs
in simp)
then show
 $\exists$ *final-state*.
composition-fri-supported-root-composition-table-candidate-state s fr
f-fri-roots f-final as dg composition-fri-roots composition-table
final-state
by *blast*
next
assume
 $\exists$ *final-state*.
composition-fri-supported-root-composition-table-candidate-state s fr
f-fri-roots f-final as dg composition-fri-roots composition-table
final-state
then obtain *final-state result trace-table query-idxs trace-bs comp-bs final*
rest where
outcome:
Some (result, final-state) $\in$ set-dist (execute verify-monad s)
and *bound:*
accepted-with-bound-tables s (Some (result, final-state))
trace-table composition-table as query-idxs
and *challenges:*
accepted-fri-challenges s (Some (result, final-state)) trace-bs dg
comp-bs
and *header:*
verifier-header-transcript s fr f-fri-roots f-final as dg
composition-fri-roots final rest
unfolding *composition-fri-supported-root-composition-table-candidate-state-def*
by *blast*
show
composition-table $\in$
composition-fri-supported-root-composition-table-candidates s fr
f-fri-roots f-final as dg composition-fri-roots
unfolding *composition-fri-supported-root-composition-table-candidates-def*
using *outcome bound challenges header* **by** *blast*
qed

definition *composition-fri-supported-root-partial-composition-table-candidates*
 $:: ('f, 'a) \text{ protocol-channel-scheme} \Rightarrow$
 $'f \Rightarrow 'f \text{ list} \Rightarrow 'f \Rightarrow 'f \text{ list} \Rightarrow 'f \Rightarrow$
 $'f \text{ list} \Rightarrow 'f \text{ list set}$

where

composition-fri-supported-root-partial-composition-table-candidates s fr
f-fri-roots f-final as dg composition-fri-roots =
{ composition-table.
composition-fri-roots $\neq []$ $\wedge$
($\exists$ out query-idxs composition-openings trace-bs comp-bs final rest.
out $\in$ set-dist (execute verify-monad s) $\wedge$
accepted-with-partial-composition-openings s out
(hd composition-fri-roots) query-idxs composition-openings $\wedge$
partial-composition-table-candidate composition-table
composition-openings $\wedge$
accepted-fri-challenges s out trace-bs dg comp-bs $\wedge$
verifier-header-transcript s fr f-fri-roots f-final as dg
composition-fri-roots final rest))}

definition *composition-fri-supported-root-partial-composition-table-candidate-state*

:: (f, 'a) protocol-channel-scheme $\Rightarrow$
'f $\Rightarrow$ 'f list $\Rightarrow$ 'f $\Rightarrow$ 'f list $\Rightarrow$ 'f $\Rightarrow$
'f list $\Rightarrow$ 'f list $\Rightarrow$
(f, 'a) protocol-channel-scheme $\Rightarrow$ bool

where

composition-fri-supported-root-partial-composition-table-candidate-state
s fr f-fri-roots f-final as dg composition-fri-roots composition-table
final-state $\longleftrightarrow$
composition-fri-roots $\neq []$ $\wedge$
($\exists$ result query-idxs composition-openings trace-bs comp-bs final rest.
Some (result, final-state) $\in$ set-dist (execute verify-monad s) $\wedge$
accepted-with-partial-composition-openings s
(Some (result, final-state)) (hd composition-fri-roots) query-idxs
composition-openings $\wedge$
partial-composition-table-candidate composition-table
composition-openings $\wedge$
accepted-fri-challenges s (Some (result, final-state)) trace-bs dg
comp-bs $\wedge$
verifier-header-transcript s fr f-fri-roots f-final as dg
composition-fri-roots final rest)

lemma *composition-fri-supported-root-partial-composition-table-candidates-iff-state:*

composition-table $\in$
composition-fri-supported-root-partial-composition-table-candidates s fr
f-fri-roots f-final as dg composition-fri-roots $\longleftrightarrow$
($\exists$ final-state.
composition-fri-supported-root-partial-composition-table-candidate-state
s fr f-fri-roots f-final as dg composition-fri-roots
composition-table final-state)

proof

assume *candidate:*

composition-table $\in$
composition-fri-supported-root-partial-composition-table-candidates s fr

f-fri-roots f-final as dg composition-fri-roots
then obtain *out query-idxs composition-openings trace-bs comp-bs final rest*
where *comp-nonempty: composition-fri-roots $\neq \square$*
and *outcome: out $\in$ set-dist (execute verify-monad s)*
and *partial:*
accepted-with-partial-composition-openings s out
(hd composition-fri-roots) query-idxs composition-openings
and *table:*
partial-composition-table-candidate composition-table
composition-openings
and *challenges: accepted-fri-challenges s out trace-bs dg comp-bs*
and *header:*
verifier-header-transcript s fr f-fri-roots f-final as dg
composition-fri-roots final rest
unfolding
composition-fri-supported-root-partial-composition-table-candidates-def
by *blast*
from *partial* **obtain** *result final-state* **where** *out-eq:*
out = Some (result, final-state)
unfolding *accepted-with-partial-composition-openings-def* **by** *blast*
have *outcome-some:*
Some (result, final-state) $\in$ set-dist (execute verify-monad s)
using *outcome out-eq* **by** *simp*
have *partial-some:*
accepted-with-partial-composition-openings s
(Some (result, final-state)) (hd composition-fri-roots) query-idxs
composition-openings
using *partial out-eq* **by** *simp*
have *challenges-some:*
accepted-fri-challenges s (Some (result, final-state)) trace-bs dg
comp-bs
using *challenges out-eq* **by** *simp*
have *state:*
composition-fri-supported-root-partial-composition-table-candidate-state
s fr f-fri-roots f-final as dg composition-fri-roots
composition-table final-state
unfolding
composition-fri-supported-root-partial-composition-table-candidate-state-def
using *comp-nonempty outcome-some partial-some table challenges-some header*
by *blast*
then show
 $\exists$ *final-state.*
composition-fri-supported-root-partial-composition-table-candidate-state
s fr f-fri-roots f-final as dg composition-fri-roots
composition-table final-state
by *blast*
next
assume
 $\exists$ *final-state.*

```

    composition-fri-supported-root-partial-composition-table-candidate-state
    s fr f-fri-roots f-final as dg composition-fri-roots
    composition-table final-state
then obtain final-state result query-idxs composition-openings trace-bs
    comp-bs final rest where
    comp-nonempty: composition-fri-roots  $\neq []$ 
and outcome:
    Some (result, final-state)  $\in$  set-dist (execute verify-monad s)
and partial:
    accepted-with-partial-composition-openings s
    (Some (result, final-state)) (hd composition-fri-roots) query-idxs
    composition-openings
and table:
    partial-composition-table-candidate composition-table
    composition-openings
and challenges:
    accepted-fri-challenges s (Some (result, final-state)) trace-bs dg
    comp-bs
and header:
    verifier-header-transcript s fr f-fri-roots f-final as dg
    composition-fri-roots final rest
unfolding
    composition-fri-supported-root-partial-composition-table-candidate-state-def
by blast
show
    composition-table  $\in$ 
    composition-fri-supported-root-partial-composition-table-candidates s fr
    f-fri-roots f-final as dg composition-fri-roots
unfolding
    composition-fri-supported-root-partial-composition-table-candidates-def
using comp-nonempty outcome partial table challenges header by blast
qed

lemma composition-fri-supported-root-composition-table-candidate-state-imp-partial-candidate-state:
assumes state:
    composition-fri-supported-root-composition-table-candidate-state s fr
    f-fri-roots f-final as dg composition-fri-roots composition-table
    final-state
and clean:  $\neg$  hash-map-output-collision final-state
shows
    composition-fri-supported-root-partial-composition-table-candidate-state
    s fr f-fri-roots f-final as dg composition-fri-roots composition-table
    final-state
proof –
from state obtain result trace-table query-idxs trace-bs comp-bs final rest
where outcome:
    Some (result, final-state)  $\in$  set-dist (execute verify-monad s)
and bound:
    accepted-with-bound-tables s (Some (result, final-state))

```

```

    trace-table composition-table as query-idxs
and challenges:
    accepted-fri-challenges s (Some (result, final-state)) trace-bs dg
    comp-bs
and header:
    verifier-header-transcript s fr f-fri-roots f-final as dg
    composition-fri-roots final rest
and comp-nonempty: composition-fri-roots  $\neq$  []
and bind:
    merkle-root-binds-table (hd composition-fri-roots) composition-table
    final-state
unfolding composition-fri-supported-root-composition-table-candidate-state-def
by blast
from verify-monad-header-extraction[OF outcome]
obtain fr' f-fl' f-final' as' dg' fl' final' query-state' where
    header':
    verifier-header-transcript s fr' (map snd f-fl') f-final' as' dg'
    (map snd fl') final' (PTranscript query-state')
and query-out:
    Some (result, final-state)  $\in$ 
    set-dist
    (execute
    (ntimes
    (verifier-query-round-program fr' f-fl' f-final' as' fl'
    final') rounds)
    query-state')
by metis
have eqs:
    fr' = fr  $\wedge$ 
    map snd f-fl' = f-fri-roots  $\wedge$ 
    f-final' = f-final  $\wedge$ 
    as' = as  $\wedge$ 
    dg' = dg  $\wedge$ 
    map snd fl' = composition-fri-roots  $\wedge$ 
    final' = final  $\wedge$ 
    PTranscript query-state' = rest
using verifier-header-transcript-unique[OF header' header] by simp
then have fl-nonempty: fl'  $\neq$  []
using comp-nonempty by (cases fl') simp-all
then obtain b composition-root fl-tail where fl-eq:
    fl' = (b, composition-root) # fl-tail
by (cases fl') auto
have root-eq: composition-root = hd composition-fri-roots
proof -
have map-eq: map snd fl' = composition-fri-roots
using eqs by simp
have cons-eq: composition-root # map snd fl-tail = composition-fri-roots
using map-eq fl-eq by simp
then have hd composition-fri-roots = composition-root

```



```

    by (metis list.sel(1))
  then show ?thesis by simp
qed
from query-rounds-accepted-with-partial-composition-openings
  [OF fl-eq query-out]
obtain partial-query-idxs composition-openings where partial:
  accepted-with-partial-composition-openings s
  (Some (result, final-state)) composition-root partial-query-idxs
  composition-openings
by blast
have partial-root:
  accepted-with-partial-composition-openings s
  (Some (result, final-state)) (hd composition-fri-roots)
  partial-query-idxs composition-openings
using partial root-eq by simp
have len-table: length composition-table = scale * clength
using accepted-with-bound-tables-shapes(2)[OF bound]
by (simp add: mult.commute)
have table:
  partial-composition-table-candidate composition-table
  composition-openings
by (rule accepted-bound-composition-table-partial-candidate-if-clean
  [OF bind len-table partial-root clean])
show ?thesis
unfolding
  composition-fri-supported-root-partial-composition-table-candidate-state-def
using outcome partial-root table challenges header comp-nonempty by blast
qed

```

definition *query-header-supported-partial-table-candidates*

```

:: ('f, 'a) protocol-channel-scheme  $\Rightarrow$ 
  'f  $\Rightarrow$  'f list  $\Rightarrow$  'f  $\Rightarrow$  'f list  $\Rightarrow$  'f  $\Rightarrow$ 
  'f list  $\Rightarrow$  'f  $\Rightarrow$  ('f list  $\times$  'f list) set

```

where

```

query-header-supported-partial-table-candidates s fr f-fri-roots
f-final as dg composition-fri-roots final =
  {(trace-table, composition-table).
    composition-fri-roots  $\neq$  []  $\wedge$ 
    ( $\exists$  out trace-query-idxs composition-query-idxs trace-openings
      composition-openings rest.
      out  $\in$  set-dist (execute verify-monad s)  $\wedge$ 
      accepted-with-partial-trace-openings s out fr trace-query-idxs
      trace-openings  $\wedge$ 
      partial-trace-table-candidate trace-table trace-openings  $\wedge$ 
      accepted-with-partial-composition-openings s out
      (hd composition-fri-roots) composition-query-idxs
      composition-openings  $\wedge$ 
      partial-composition-table-candidate composition-table
      composition-openings  $\wedge$ 

```

*verifier-header-transcript s fr f-fri-roots f-final as dg
composition-fri-roots final rest)*}}

definition *query-header-supported-partial-table-candidate-state*

*:: ('f, 'a) protocol-channel-scheme $\Rightarrow$
'f $\Rightarrow$ 'f list $\Rightarrow$ 'f $\Rightarrow$ 'f list $\Rightarrow$ 'f $\Rightarrow$
'f list $\Rightarrow$ 'f $\Rightarrow$ 'f list $\Rightarrow$ 'f list $\Rightarrow$
('f, 'a) protocol-channel-scheme $\Rightarrow$ bool*

where

*query-header-supported-partial-table-candidate-state s fr f-fri-roots
f-final as dg composition-fri-roots final trace-table
composition-table final-state $\longleftrightarrow$
composition-fri-roots $\neq []$ $\wedge$
($\exists$ result trace-query-idxs composition-query-idxs trace-openings
composition-openings rest.
Some (result, final-state) $\in$ set-dist (execute verify-monad s) $\wedge$
accepted-with-partial-trace-openings s
(Some (result, final-state)) fr trace-query-idxs
trace-openings $\wedge$
partial-trace-table-candidate trace-table trace-openings $\wedge$
accepted-with-partial-composition-openings s
(Some (result, final-state)) (hd composition-fri-roots)
composition-query-idxs composition-openings $\wedge$
partial-composition-table-candidate composition-table
composition-openings $\wedge$
verifier-header-transcript s fr f-fri-roots f-final as dg
composition-fri-roots final rest)*

lemma *query-header-supported-partial-table-candidates-iff-state:*

*(trace-table, composition-table) $\in$
query-header-supported-partial-table-candidates s fr f-fri-roots
f-final as dg composition-fri-roots final $\longleftrightarrow$
($\exists$ final-state.
query-header-supported-partial-table-candidate-state s fr f-fri-roots
f-final as dg composition-fri-roots final trace-table
composition-table final-state)*

proof

assume *candidate:*

*(trace-table, composition-table) $\in$
query-header-supported-partial-table-candidates s fr f-fri-roots
f-final as dg composition-fri-roots final*

then obtain *out trace-query-idxs composition-query-idxs trace-openings
composition-openings rest* **where**

comp-nonempty: composition-fri-roots $\neq []$

and *outcome: out $\in$ set-dist (execute verify-monad s)*

and *trace-partial:*

*accepted-with-partial-trace-openings s out fr trace-query-idxs
trace-openings*

and *trace-table:*

```

    partial-trace-table-candidate trace-table trace-openings
and comp-partial:
    accepted-with-partial-composition-openings s out
    (hd composition-fri-roots) composition-query-idxs
    composition-openings
and comp-table:
    partial-composition-table-candidate composition-table
    composition-openings
and header:
    verifier-header-transcript s fr f-fri-roots f-final as dg
    composition-fri-roots final rest
unfolding query-header-supported-partial-table-candidates-def by blast
from trace-partial obtain result final-state where out-eq:
    out = Some (result, final-state)
unfolding accepted-with-partial-trace-openings-def by blast
have outcome-some:
    Some (result, final-state) ∈ set-dist (execute verify-monad s)
using outcome out-eq by simp
have trace-partial-some:
    accepted-with-partial-trace-openings s (Some (result, final-state)) fr
    trace-query-idxs trace-openings
using trace-partial out-eq by simp
have comp-partial-some:
    accepted-with-partial-composition-openings s
    (Some (result, final-state)) (hd composition-fri-roots)
    composition-query-idxs composition-openings
using comp-partial out-eq by simp
have state:
    query-header-supported-partial-table-candidate-state s fr f-fri-roots
    f-final as dg composition-fri-roots final trace-table
    composition-table final-state
unfolding query-header-supported-partial-table-candidate-state-def
using comp-nonempty outcome-some trace-partial-some trace-table
    comp-partial-some comp-table header
by blast
then show
    ∃ final-state.
    query-header-supported-partial-table-candidate-state s fr f-fri-roots
    f-final as dg composition-fri-roots final trace-table
    composition-table final-state
by blast
next
assume
    ∃ final-state.
    query-header-supported-partial-table-candidate-state s fr f-fri-roots
    f-final as dg composition-fri-roots final trace-table
    composition-table final-state
then obtain final-state result trace-query-idxs composition-query-idxs
    trace-openings composition-openings rest where

```

comp-nonempty: *composition-fri-roots* $\neq \square$
and *outcome*:
Some (*result*, *final-state*) $\in$ *set-dist* (*execute verify-monad s*)
and *trace-partial*:
accepted-with-partial-trace-openings s
(Some (result, final-state)) fr trace-query-idxs trace-openings
and *trace-table*:
partial-trace-table-candidate trace-table trace-openings
and *comp-partial*:
accepted-with-partial-composition-openings s
(Some (result, final-state)) (hd composition-fri-roots)
composition-query-idxs composition-openings
and *comp-table*:
partial-composition-table-candidate composition-table
composition-openings
and *header*:
verifier-header-transcript s fr f-fri-roots f-final as dg
composition-fri-roots final rest
unfolding *query-header-supported-partial-table-candidate-state-def*
by *blast*
show
(trace-table, composition-table) $\in$
query-header-supported-partial-table-candidates s fr f-fri-roots
f-final as dg composition-fri-roots final
unfolding *query-header-supported-partial-table-candidates-def*
using *comp-nonempty outcome trace-partial trace-table comp-partial*
comp-table header
by *blast*
qed

lemma *query-header-supported-partial-table-candidate-state-hash-extends*:

assumes

query-header-supported-partial-table-candidate-state s fr f-fri-roots
f-final as dg composition-fri-roots final trace-table
composition-table final-state

shows $s \leq \text{final-state}$

proof –

from *assms* **obtain** *result trace-query-idxs composition-query-idxs*
trace-openings composition-openings rest **where**

outcome:

Some (result, final-state) $\in$ set-dist (execute verify-monad s)

unfolding *query-header-supported-partial-table-candidate-state-def*

by *blast*

show *?thesis*

by (*rule verify-monad-hash-extends[OF outcome]*)

qed

lemma *query-header-supported-table-candidate-state-imp-partial-candidate-state*:

assumes *state*:

query-header-supported-table-candidate-state s fr f-fri-roots f-final
as dg composition-fri-roots final trace-table composition-table
final-state
and *clean*: $\neg$ *hash-map-output-collision final-state*
shows
query-header-supported-partial-table-candidate-state s fr f-fri-roots
f-final as dg composition-fri-roots final trace-table
composition-table final-state
proof –
from *state* **obtain** *result query-idxs rest* **where**
outcome:
Some (result, final-state) ∈ set-dist (execute verify-monad s)
and *bound*:
accepted-with-bound-tables s (Some (result, final-state))
trace-table composition-table as query-idxs
and *header*:
verifier-header-transcript s fr f-fri-roots f-final as dg
composition-fri-roots final rest
and *trace-bind*:
merkle-root-binds-table fr trace-table final-state
and *comp-nonempty*: *composition-fri-roots* $\neq$ []
and *comp-bind*:
merkle-root-binds-table (hd composition-fri-roots) composition-table
final-state
unfolding *query-header-supported-table-candidate-state-def* **by** *blast*
from *verify-monad-header-extraction[OF outcome]*
obtain *fr' f-fl f-final' as' dg' fl final' query-state* **where**
header':
verifier-header-transcript s fr' (map snd f-fl) f-final' as' dg'
(map snd fl) final' (PTranscript query-state)
and *query-out*:
Some (result, final-state) ∈
set-dist
(execute
(ntimes
(verifier-query-round-program fr' f-fl f-final' as' fl
final') rounds)
query-state)
by *metis*
have *eqs*:
fr' = fr ∧
map snd f-fl = f-fri-roots ∧
f-final' = f-final ∧
as' = as ∧
dg' = dg ∧
map snd fl = composition-fri-roots ∧
final' = final ∧
PTranscript query-state = rest
using *verifier-header-transcript-unique[OF header' header]* **by** *simp*

```

obtain trace-query-idxs trace-openings where
  len-trace-query-idxs: length trace-query-idxs = rounds
  and len-trace-openings: length trace-openings = rounds
  and trace-indices:
     $\bigwedge i. i < \text{rounds} \implies$ 
      map opening-index (trace-openings ! i) =
        powers-scaled (trace-query-idxs ! i)
  and trace-tables:
     $\bigwedge i. i < \text{rounds} \implies$ 
      partial-authenticated-table fr'
      (scale * clength) (trace-openings ! i) final-state
proof (rule ntimes-verifier-query-rounds-authenticated-trace-openings-no-raws
  [OF query-out])
fix trace-query-idxs trace-openings
assume len-trace-query-idxs: length trace-query-idxs = rounds
  and len-trace-openings: length trace-openings = rounds
  and trace-indices:
     $\bigwedge i. i < \text{rounds} \implies$ 
      map opening-index (trace-openings ! i) =
        powers-scaled (trace-query-idxs ! i)
  and trace-tables:
     $\bigwedge i. i < \text{rounds} \implies$ 
      partial-authenticated-table fr'
      (scale * clength) (trace-openings ! i) final-state
show ?thesis
  by (rule that[OF len-trace-query-idxs len-trace-openings
    trace-indices trace-tables])
qed
have trace-partial:
  accepted-with-partial-trace-openings s
  (Some (result, final-state)) fr trace-query-idxs trace-openings
proof –
  have all-indices:
     $\forall i < \text{rounds}.$ 
      map opening-index (trace-openings ! i) =
        powers-scaled (trace-query-idxs ! i)
    using trace-indices by blast
  have all-tables:
     $\forall i < \text{rounds}.$ 
      partial-authenticated-table fr (scale * clength)
      (trace-openings ! i) final-state
    using trace-tables eqs by simp
  show ?thesis
  unfolding accepted-with-partial-trace-openings-def accepted-def
  by (intro conjI exI[of - result] exI[of - final-state])
    (use len-trace-query-idxs len-trace-openings all-indices all-tables
      in simp-all)
qed
have trace-len: length trace-table = scale * clength

```

```

    using accepted-with-bound-tables-shapes(1)[OF bound] by simp
  have trace-candidate:
    partial-trace-table-candidate trace-table trace-openings
  by (rule accepted-bound-trace-table-partial-candidate-if-clean
    [OF trace-bind trace-len trace-partial clean])
  have fl-nonempty: fl ≠ []
    using comp-nonempty eqs by (cases fl) simp-all
  then obtain b composition-root fl-tail where fl-eq:
    fl = (b, composition-root) # fl-tail
  by (cases fl) auto
  have root-eq: composition-root = hd composition-fri-roots
  proof -
    have cons-eq: composition-root # map snd fl-tail = composition-fri-roots
      using eqs fl-eq by simp
    then have hd composition-fri-roots = composition-root
      by (metis list.sel(1))
    then show ?thesis by simp
  qed
  from query-rounds-accepted-with-partial-composition-openings
    [OF fl-eq query-out]
  obtain composition-query-idxs composition-openings where comp-partial':
    accepted-with-partial-composition-openings s
    (Some (result, final-state)) composition-root composition-query-idxs
    composition-openings
  by blast
  have comp-partial:
    accepted-with-partial-composition-openings s
    (Some (result, final-state)) (hd composition-fri-roots)
    composition-query-idxs composition-openings
  using comp-partial' root-eq by simp
  have comp-len: length composition-table = scale * clength
  using accepted-with-bound-tables-shapes(2)[OF bound]
  by (simp add: mult.commute)
  have comp-candidate:
    partial-composition-table-candidate composition-table
    composition-openings
  by (rule accepted-bound-composition-table-partial-candidate-if-clean
    [OF comp-bind comp-len comp-partial clean])
  show ?thesis
  unfolding query-header-supported-partial-table-candidate-state-def
  using comp-nonempty outcome trace-partial trace-candidate comp-partial
    comp-candidate header
  by blast
qed

lemma accepted-with-bound-tables-header-composition-fri-roots-nonempty:
  assumes bound:
    accepted-with-bound-tables s out trace-table composition-table as
    query-idxs

```

and *header*:
verifier-header-transcript s fr f-fri-roots f-final as dg
composition-fri-roots final rest
shows *composition-fri-roots* $\neq \square$
proof –
from *bound* **obtain** *result final-state fr' f-fri-roots' f-final' dg'*
composition-fri-roots' final' rest' where
out-eq: out = Some (result, final-state)
and *header'*:
verifier-header-transcript s fr' f-fri-roots' f-final' as dg'
composition-fri-roots' final' rest'
and *comp-nonempty'*: *composition-fri-roots' $\neq \square$*
unfolding *accepted-with-bound-tables-def* **by** *blast*
have *composition-fri-roots' = composition-fri-roots*
using *verifier-header-transcript-unique[OF header' header]* **by** *simp*
then show *?thesis*
using *comp-nonempty'* **by** *simp*
qed

definition *query-header-supported-partial-union-good-sets*

$:: (f, 'a) \text{ protocol-channel-scheme} \Rightarrow$
 $(f \text{ list} \Rightarrow 'f \text{ list} \Rightarrow 'f \text{ list} \Rightarrow \text{nat set}) \Rightarrow$
 $'f \Rightarrow 'f \text{ list} \Rightarrow 'f \Rightarrow 'f \text{ list} \Rightarrow 'f \Rightarrow$
 $'f \text{ list} \Rightarrow 'f \Rightarrow \text{nat set}$

where

query-header-supported-partial-union-good-sets s good-sets fr
f-fri-roots f-final as dg composition-fri-roots final =
 $\{idx \in \text{query-sample-space}.$
 $\exists \text{trace-table composition-table}.$
 $(\text{trace-table}, \text{composition-table}) \in$
 $\text{query-header-supported-partial-table-candidates } s \text{ fr}$
 $f\text{-fri-roots } f\text{-final as dg composition-fri-roots final} \wedge$
 $idx \in \text{good-sets trace-table composition-table as}\}$

lemma *query-header-supported-partial-union-good-sets-subset:*

query-header-supported-partial-union-good-sets s good-sets fr f-fri-roots
f-final as dg composition-fri-roots final $\subseteq$ query-sample-space
unfolding *query-header-supported-partial-union-good-sets-def* **by** *auto*

lemma *query-header-supported-partial-union-good-sets-fraction-bound-if-unique-candidate:*

assumes *unique:*

$\exists \text{trace-table composition-table}.$
query-header-supported-partial-table-candidates s fr f-fri-roots
f-final as dg composition-fri-roots final $\subseteq$
 $\{(\text{trace-table}, \text{composition-table})\}$

and *subset:*

$\wedge \text{trace-table composition-table as}.$
good-sets trace-table composition-table as $\subseteq$ query-sample-space

and *bound:*

$\wedge$ trace-table composition-table as.
 $\text{nnreal } (\text{card } (\text{good-sets trace-table composition-table as})) /$
 $\text{nnreal } (\text{card query-sample-space}) \leq \text{query-error-bound}$
shows
 nnreal
 $(\text{card}$
 $(\text{query-header-supported-partial-union-good-sets } s \text{ good-sets fr}$
 $\text{f-fri-roots f-final as dg composition-fri-roots final})) /$
 $\text{nnreal } (\text{card query-sample-space}) \leq \text{query-error-bound}$
proof –
from unique **obtain** trace-table composition-table **where** candidates:
 $\text{query-header-supported-partial-table-candidates } s \text{ fr f-fri-roots f-final}$
 $\text{as dg composition-fri-roots final} \subseteq$
 $\{(\text{trace-table}, \text{composition-table})\}$
by blast
have union-subset:
 $\text{query-header-supported-partial-union-good-sets } s \text{ good-sets fr}$
 $\text{f-fri-roots f-final as dg composition-fri-roots final} \subseteq$
 $\text{good-sets trace-table composition-table as}$
unfolding query-header-supported-partial-union-good-sets-def
using candidates **by** auto
have finite-good: finite (good-sets trace-table composition-table as)
by (rule finite-subset[OF subset finite-query-sample-space])
have card-le:
 card
 $(\text{query-header-supported-partial-union-good-sets } s \text{ good-sets fr}$
 $\text{f-fri-roots f-final as dg composition-fri-roots final}) \leq$
 $\text{card } (\text{good-sets trace-table composition-table as})$
by (rule card-mono[OF finite-good union-subset])
have nnreal
 $(\text{card}$
 $(\text{query-header-supported-partial-union-good-sets } s \text{ good-sets fr}$
 $\text{f-fri-roots f-final as dg composition-fri-roots final})) /$
 $\text{nnreal } (\text{card query-sample-space}) \leq$
 $\text{nnreal } (\text{card } (\text{good-sets trace-table composition-table as})) /$
 $\text{nnreal } (\text{card query-sample-space})$
by (rule nnreal-nat-divide-right-mono[OF card-le])
also have ... $\leq \text{query-error-bound}$
by (rule bound)
finally show ?thesis .
qed

lemma query-index-round-set-hit-at-imp-partial-union-or-collision:
assumes outcome: $\text{out} \in \text{set-dist } (\text{execute verify-monad } s)$
and hit: query-index-round-set-hit-at $s \text{ good-sets } i \text{ out}$
shows
 $\text{query-header-rounds-any-index-set-hit } s$
 $(\text{query-header-supported-partial-union-good-sets } s \text{ good-sets}) \text{ out} \vee$
 $\text{hash-map-output-collision-bad } s \text{ out}$

```

proof –
from hit obtain trace-table composition-table as query-idxs where
  bound:
    accepted-with-bound-tables s out trace-table composition-table as
      query-idxs
  and i-bound: i < rounds
  and query-hit:
    query-idxs ! i ∈ good-sets trace-table composition-table as
  unfolding query-index-round-set-hit-at-def by blast
from bound obtain result final-state fr f-fri-roots f-final dg
  composition-fri-roots final rest where
  out-eq: out = Some (result, final-state)
  and header:
    verifier-header-transcript s fr f-fri-roots f-final as dg
    composition-fri-roots final rest
  and trace-bind:
    merkle-root-binds-table fr trace-table final-state
  and comp-nonempty: composition-fri-roots ≠ []
  and comp-bind:
    merkle-root-binds-table (hd composition-fri-roots) composition-table
    final-state
  unfolding accepted-with-bound-tables-def by blast
show ?thesis
proof (cases hash-map-output-collision final-state)
  case True
  then have hash-map-output-collision-bad s out
    unfolding hash-map-output-collision-bad-def accepted-def out-eq by simp
  then show ?thesis by simp
next
  case clean: False
  have state:
    query-header-supported-table-candidate-state s fr f-fri-roots f-final
    as dg composition-fri-roots final trace-table composition-table
    final-state
  unfolding query-header-supported-table-candidate-state-def
  by (intro exI[of - result] exI[of - query-idxs] exI[of - rest])
    (use outcome out-eq bound header trace-bind comp-nonempty comp-bind
      in simp)
  have candidate:
    (trace-table, composition-table) ∈
    query-header-supported-partial-table-candidates s fr f-fri-roots
    f-final as dg composition-fri-roots final
  using
    query-header-supported-table-candidate-state-imp-partial-candidate-state
    [OF state clean]
    query-header-supported-partial-table-candidates-iff-state
  by blast
  have len: length query-idxs = rounds
  by (rule accepted-with-bound-tables-shapes(4) [OF bound])

```

```

have idx-mem: query-idxs ! i ∈ set query-idxs
  by (rule nth-mem) (use i-bound len in simp)
have idx-sample: query-idxs ! i ∈ query-sample-space
  by (rule accepted-with-bound-tables-query-sample-space
    [OF bound idx-mem])
have union-hit:
  query-idxs ! i ∈
    query-header-supported-partial-union-good-sets s good-sets fr
      f-fri-roots f-final as dg composition-fri-roots final
  unfolding query-header-supported-partial-union-good-sets-def
  using idx-sample candidate query-hit by blast
have round-hit:
  query-rounds-any-index-set-hit s
    (query-header-supported-partial-union-good-sets s good-sets fr
      f-fri-roots f-final as dg composition-fri-roots final)
    rounds out
  by (rule accepted-with-bound-tables-query-rounds-any-index-set-hit
    [OF bound i-bound union-hit])
show ?thesis
  unfolding query-header-rounds-any-index-set-hit-def
  using header round-hit by blast
qed
qed

```

```

lemma query-index-round-set-hit-imp-partial-union-or-collision:
  assumes outcome: out ∈ set-dist (execute verify-monad s)
  and hit: query-index-round-set-hit s good-sets out
  shows
    query-header-rounds-any-index-set-hit s
      (query-header-supported-partial-union-good-sets s good-sets) out ∨
      hash-map-output-collision-bad s out
proof –
  from hit obtain i where
    query-index-round-set-hit-at s good-sets i out
  unfolding query-index-round-set-hit-def by blast
  then show ?thesis
    by (rule query-index-round-set-hit-at-imp-partial-union-or-collision
      [OF outcome])
qed

```

```

lemma wp-query-index-round-set-hit-bound-via-partial-union-or-collision:
  fixes C H :: prob
  assumes partial-bound:
    wp-event verify-monad
      (query-header-rounds-any-index-set-hit s
        (query-header-supported-partial-union-good-sets s good-sets)) s ≤
        C
  and collision-bound:
    wp-event verify-monad (hash-map-output-collision-bad s) s ≤ H

```

shows
 $wp\text{-event } verify\text{-monad } (query\text{-index-round-set-hit } s \text{ good-sets}) \ s \leq C + H$
proof –
have *mono*:
 $wp\text{-event } verify\text{-monad } (query\text{-index-round-set-hit } s \text{ good-sets}) \ s \leq wp\text{-event } verify\text{-monad } (\lambda out. query\text{-header-rounds-any-index-set-hit } s (query\text{-header-supported-partial-union-good-sets } s \text{ good-sets}) out \vee hash\text{-map-output-collision-bad } s \ out) \ s$
by (rule *wp-event-mono-on-support*)
 (use *query-index-round-set-hit-imp-partial-union-or-collision* **in** *blast*)
also have $\dots \leq C + H$
proof –
have *wp-event verify-monad*
 $(\lambda out. query\text{-header-rounds-any-index-set-hit } s (query\text{-header-supported-partial-union-good-sets } s \text{ good-sets}) out \vee hash\text{-map-output-collision-bad } s \ out) \ s \leq wp\text{-event } verify\text{-monad } (query\text{-header-rounds-any-index-set-hit } s (query\text{-header-supported-partial-union-good-sets } s \text{ good-sets})) \ s + wp\text{-event } verify\text{-monad } (hash\text{-map-output-collision-bad } s) \ s$
by (rule *wp-event-union-bound*)
also have $\dots \leq C + H$
by (intro *add-mono partial-bound collision-bound*)
finally show *?thesis* .
qed
finally show *?thesis* .
qed

lemma *wp-query-index-round-set-hit-bound-via-supported-partial-union-or-collision*:
fixes $H :: prob$
assumes *future: query-future-fresh s*
and *raw-bound: query-index-raw-preimage-bound*
and *union-bound*:
 $\bigwedge fr \ f \text{-fri-roots } f\text{-final} \text{ as } dg \text{ composition-fri-roots } final. nnreal (card (query\text{-header-supported-partial-union-good-sets } s \text{ good-sets } fr \ f\text{-fri-roots } f\text{-final} \text{ as } dg \text{ composition-fri-roots } final)) / nnreal (card \ query\text{-sample-space}) \leq query\text{-error-bound}$
and *collision-bound*:
 $wp\text{-event } verify\text{-monad } (hash\text{-map-output-collision-bad } s) \ s \leq H$
shows
 $wp\text{-event } verify\text{-monad } (query\text{-index-round-set-hit } s \text{ good-sets}) \ s \leq$

$nnreal\ rounds * query\ error\ bound + H$
proof –
have *partial-bound*:
 $wp\ event\ verify\ monad$
 $(query\ header\ rounds\ any\ index\ set\ hit\ s$
 $(query\ header\ supported\ partial\ union\ good\ sets\ s\ good\ sets))\ s \leq$
 $nnreal\ rounds * query\ error\ bound$
by (rule *wp-verify-monad-query-header-rounds-any-index-set-bound*
 $[OF\ future\ raw\ bound]$)
 $(use\ query\ header\ supported\ partial\ union\ good\ sets\ subset\ union\ bound$
 $in\ simp\ all)$
show *?thesis*
by (rule *wp-query-index-round-set-hit-bound-via-partial-union-or-collision*
 $[OF\ partial\ bound\ collision\ bound]$)
qed

lemma *wp-query-index-set-hit-bound-via-partial-union-or-collision*:
fixes $C\ H :: prob$
assumes *partial-bound*:
 $wp\ event\ verify\ monad$
 $(query\ header\ rounds\ any\ index\ set\ hit\ s$
 $(query\ header\ supported\ partial\ union\ good\ sets\ s\ good\ sets))\ s \leq$
 C
and *collision-bound*:
 $wp\ event\ verify\ monad\ (hash\ map\ output\ collision\ bad\ s)\ s \leq H$
shows
 $wp\ event\ verify\ monad\ (query\ index\ set\ hit\ s\ good\ sets)\ s \leq C + H$
proof –
have $wp\ event\ verify\ monad\ (query\ index\ set\ hit\ s\ good\ sets)\ s \leq$
 $wp\ event\ verify\ monad\ (query\ index\ round\ set\ hit\ s\ good\ sets)\ s$
by (rule *wp-event-mono*) (rule *query-index-set-hit-imp-round-set-hit*)
also have $\dots \leq C + H$
by (rule *wp-query-index-round-set-hit-bound-via-partial-union-or-collision*
 $[OF\ partial\ bound\ collision\ bound]$)
finally show *?thesis* .
qed

lemma *wp-query-index-round-set-hit-bound-if-supported-partial-header-candidate-unique-or-collision*:
fixes $H :: prob$
assumes *future*: *query-future-fresh* s
and *raw-bound*: *query-index-raw-preimage-bound*
and *unique*:
 $\bigwedge fr\ f\ fri\ roots\ f\ final\ as\ dg\ composition\ fri\ roots\ final.$
 $\exists\ trace\ table\ composition\ table.$
 $query\ header\ supported\ partial\ table\ candidates\ s\ fr\ f\ fri\ roots$
 $f\ final\ as\ dg\ composition\ fri\ roots\ final \subseteq$
 $\{(trace\ table,\ composition\ table)\}$
and *subset*:
 $\bigwedge trace\ table\ composition\ table\ as.$

$\text{good-sets trace-table composition-table as} \subseteq \text{query-sample-space}$
and *bounded*:
 $\bigwedge \text{trace-table composition-table as.}$
 $\text{nnreal (card (good-sets trace-table composition-table as)) /}$
 $\text{nnreal (card query-sample-space)} \leq \text{query-error-bound}$
and *collision-bound*:
 $\text{wp-event verify-monad (hash-map-output-collision-bad s) s} \leq H$
shows
 $\text{wp-event verify-monad (query-index-round-set-hit s good-sets) s} \leq$
 $\text{nnreal rounds * query-error-bound} + H$
proof –
have *union-bound*:
 $\bigwedge \text{fr f-fri-roots f-final as dg composition-fri-roots final.}$
 nnreal
 $(\text{card}$
 $(\text{query-header-supported-partial-union-good-sets s good-sets fr}$
 $\text{f-fri-roots f-final as dg composition-fri-roots final})) /$
 $\text{nnreal (card query-sample-space)} \leq \text{query-error-bound}$
by (rule
 $\text{query-header-supported-partial-union-good-sets-fraction-bound-if-unique-candidate}$
 $[\text{OF unique subset bounded}]$)
show *?thesis*
by (rule
 $\text{wp-query-index-round-set-hit-bound-via-supported-partial-union-or-collision}$
 $[\text{OF future raw-bound union-bound collision-bound}]$)
qed

lemma *wp-query-index-set-hit-bound-if-supported-partial-header-candidate-unique-or-collision*:
fixes $H :: \text{prob}$
assumes *future: query-future-fresh s*
and *raw-bound: query-index-raw-preimage-bound*
and *unique*:
 $\bigwedge \text{fr f-fri-roots f-final as dg composition-fri-roots final.}$
 $\exists \text{trace-table composition-table.}$
 $\text{query-header-supported-partial-table-candidates s fr f-fri-roots}$
 $\text{f-final as dg composition-fri-roots final} \subseteq$
 $\{(\text{trace-table}, \text{composition-table})\}$
and *subset*:
 $\bigwedge \text{trace-table composition-table as.}$
 $\text{good-sets trace-table composition-table as} \subseteq \text{query-sample-space}$
and *bounded*:
 $\bigwedge \text{trace-table composition-table as.}$
 $\text{nnreal (card (good-sets trace-table composition-table as)) /}$
 $\text{nnreal (card query-sample-space)} \leq \text{query-error-bound}$
and *collision-bound*:
 $\text{wp-event verify-monad (hash-map-output-collision-bad s) s} \leq H$
shows
 $\text{wp-event verify-monad (query-index-set-hit s good-sets) s} \leq$
 $\text{nnreal rounds * query-error-bound} + H$

proof –

have $wp\text{-event}\ verify\text{-monad}\ (query\text{-index}\text{-set}\text{-hit}\ s\ good\text{-sets})\ s \leq$
 $wp\text{-event}\ verify\text{-monad}\ (query\text{-index}\text{-round}\text{-set}\text{-hit}\ s\ good\text{-sets})\ s$
by (rule $wp\text{-event}\text{-mono}$) (rule $query\text{-index}\text{-set}\text{-hit}\text{-imp}\text{-round}\text{-set}\text{-hit}$)
also have $\dots \leq nnreal\ rounds * query\text{-error}\text{-bound} + H$
by (rule
 $wp\text{-query}\text{-index}\text{-round}\text{-set}\text{-hit}\text{-bound}\text{-if}\text{-supported}\text{-partial}\text{-header}\text{-candidate}\text{-unique}\text{-or}\text{-collision}$
 $[OF\ future\ raw\text{-bound}\ unique\ subset\ bounded\ collision\text{-bound}]$)
finally show $?thesis$.

qed

lemma $wp\text{-query}\text{-sampling}\text{-success}\text{-set}\text{-hit}\text{-bound}\text{-if}\text{-supported}\text{-partial}\text{-header}\text{-candidate}\text{-unique}\text{-or}\text{-collision}$:

fixes $H :: prob$

assumes $future: query\text{-future}\text{-fresh}\ s$

and $raw\text{-bound}: query\text{-index}\text{-raw}\text{-preimage}\text{-bound}$

and $unique$:

$\bigwedge fr\ f\text{-fri}\text{-roots}\ f\text{-final}\ as\ dg\ composition\text{-fri}\text{-roots}\ final.$

$\exists trace\text{-table}\ composition\text{-table}.$

$query\text{-header}\text{-supported}\text{-partial}\text{-table}\text{-candidates}\ s\ fr\ f\text{-fri}\text{-roots}$

$f\text{-final}\ as\ dg\ composition\text{-fri}\text{-roots}\ final \subseteq$

$\{(trace\text{-table},\ composition\text{-table})\}$

and $collision\text{-bound}$:

$wp\text{-event}\ verify\text{-monad}\ (hash\text{-map}\text{-output}\text{-collision}\text{-bad}\ s)\ s \leq H$

shows

$wp\text{-event}\ verify\text{-monad}$

$(query\text{-index}\text{-set}\text{-hit}\ s\ query\text{-sampling}\text{-success}\text{-space})\ s \leq$

$nnreal\ rounds * query\text{-error}\text{-bound} + H$

by (rule

$wp\text{-query}\text{-index}\text{-set}\text{-hit}\text{-bound}\text{-if}\text{-supported}\text{-partial}\text{-header}\text{-candidate}\text{-unique}\text{-or}\text{-collision}$

$[OF\ future\ raw\text{-bound}\ unique\ query\text{-sampling}\text{-success}\text{-space}\text{-subset}$

$query\text{-sampling}\text{-success}\text{-space}\text{-fraction}\text{-bound}\text{-query}\text{-sample}\text{-space}$

$collision\text{-bound}]$)

lemma $composition\text{-fri}\text{-supported}\text{-root}\text{-composition}\text{-table}\text{-candidate}\text{-state}\text{-hash}\text{-extends}$:

assumes

$composition\text{-fri}\text{-supported}\text{-root}\text{-composition}\text{-table}\text{-candidate}\text{-state}\ s\ fr$

$f\text{-fri}\text{-roots}\ f\text{-final}\ as\ dg\ composition\text{-fri}\text{-roots}\ composition\text{-table}$

$final\text{-state}$

shows $s \leq final\text{-state}$

proof –

from $assms$ **obtain** $result\ trace\text{-table}\ query\text{-idxs}\ trace\text{-bs}\ comp\text{-bs}\ final\ rest$

where $outcome$:

$Some\ (result,\ final\text{-state}) \in set\text{-dist}\ (execute\ verify\text{-monad}\ s)$

unfolding $composition\text{-fri}\text{-supported}\text{-root}\text{-composition}\text{-table}\text{-candidate}\text{-state}\text{-def}$

by $blast$

show $?thesis$

by (rule $verify\text{-monad}\text{-hash}\text{-extends}[OF\ outcome]$)

qed

lemma *composition-fri-supported-root-composition-table-candidate-states-merkle-compatible-unique-if-clean-merge:*
assumes *compatible:*
 merkle-hash-maps-compatible final-state final-state'
and *clean:*
 $\neg$ *hash-map-output-collision*
 (*merkle-hash-state-merge final-state final-state'*)
and *candidate:*
 composition-fri-supported-root-composition-table-candidate-state s fr
 f-fri-roots f-final as dg composition-fri-roots composition-table
 final-state
and *candidate':*
 composition-fri-supported-root-composition-table-candidate-state s fr
 f-fri-roots f-final as dg composition-fri-roots composition-table'
 final-state'
shows *composition-table = composition-table'*
proof –
from *candidate* **obtain** *result trace-table query-idxs trace-bs comp-bs final*
 rest **where**
 bound:
 accepted-with-bound-tables s (Some (result, final-state))
 trace-table composition-table as query-idxs
and *comp-nonempty:* *composition-fri-roots $\neq$ []*
and *bind:*
 merkle-root-binds-table (hd composition-fri-roots) composition-table
 final-state
unfolding *composition-fri-supported-root-composition-table-candidate-state-def*
by *blast*
from *candidate'* **obtain** *result' trace-table' query-idxs' trace-bs' comp-bs'*
 final' rest' **where**
 bound':
 accepted-with-bound-tables s (Some (result', final-state'))
 trace-table' composition-table' as query-idxs'
and *bind':*
 merkle-root-binds-table (hd composition-fri-roots) composition-table'
 final-state'
unfolding *composition-fri-supported-root-composition-table-candidate-state-def*
by *blast*
have *same-len:* *length composition-table = length composition-table'*
using *accepted-with-bound-tables-shapes(2)[OF bound]*
 accepted-with-bound-tables-shapes(2)[OF bound']
by *simp*
have *nonempty:* *composition-table $\neq$ []*
using *accepted-with-bound-tables-shapes(2)[OF bound]*
 eval-domain-nontrivial **by** *auto*
show *?thesis*
by (*rule*
 merkle-root-binds-same-length-tables-merkle-compatible-unique-if-clean-merge
 [*OF compatible clean bind bind' same-len nonempty*])
qed

lemma *composition-fri-supported-root-composition-table-candidates-subset-singleton-if-common-clean-merkle-extension*
assumes *clean*: $\neg$ *hash-map-output-collision* *u*
and *common-ext*:
 $\bigwedge$ *composition-table* *final-state*.
composition-fri-supported-root-composition-table-candidate-state *s* *fr*
f-fri-roots *f-final* as *dg* *composition-fri-roots* *composition-table*
final-state $\implies$
merkle-hash-extends *final-state* *u*
shows
 $\exists$ *composition-table*.
composition-fri-supported-root-composition-table-candidates *s* *fr*
f-fri-roots *f-final* as *dg* *composition-fri-roots* $\subseteq$
 $\{ \textit{composition-table} \}$
proof (*cases*
composition-fri-supported-root-composition-table-candidates *s* *fr*
f-fri-roots *f-final* as *dg* *composition-fri-roots* = $\{ \}$)
case *True*
then show *?thesis*
by (*intro* *exI*[*of* - $\square$]) *simp*
next
case *False*
then obtain *composition-table0* **where** *table0*:
composition-table0 $\in$
composition-fri-supported-root-composition-table-candidates *s* *fr*
f-fri-roots *f-final* as *dg* *composition-fri-roots*
by *blast*
then obtain *final-state0* **where** *state0*:
composition-fri-supported-root-composition-table-candidate-state *s* *fr*
f-fri-roots *f-final* as *dg* *composition-fri-roots* *composition-table0*
final-state0
using *composition-fri-supported-root-composition-table-candidates-iff-state*
by *blast*
have *subset*:
composition-fri-supported-root-composition-table-candidates *s* *fr*
f-fri-roots *f-final* as *dg* *composition-fri-roots* $\subseteq$
 $\{ \textit{composition-table0} \}$
proof
fix *composition-table*
assume *table*:
composition-table $\in$
composition-fri-supported-root-composition-table-candidates *s* *fr*
f-fri-roots *f-final* as *dg* *composition-fri-roots*
then obtain *final-state* **where** *state*:
composition-fri-supported-root-composition-table-candidate-state *s* *fr*
f-fri-roots *f-final* as *dg* *composition-fri-roots* *composition-table*
final-state
using *composition-fri-supported-root-composition-table-candidates-iff-state*
by *blast*

```

have ext: merkle-hash-extends final-state u
  by (rule common-ext[OF state])
have ext0: merkle-hash-extends final-state0 u
  by (rule common-ext[OF state0])
have clean-pair:
  ¬ merkle-hash-value-conflict final-state final-state0 ∧
  ¬ hash-map-output-collision
    (merkle-hash-state-merge final-state final-state0)
  by (rule common-clean-merkle-extension-imp-pairwise-merkle-clean
    [OF ext ext0 clean])
have compatible:
  merkle-hash-maps-compatible final-state final-state0
  using clean-pair
  unfolding merkle-hash-maps-compatible-iff-no-value-conflict by simp
have clean-merge:
  ¬ hash-map-output-collision
    (merkle-hash-state-merge final-state final-state0)
  using clean-pair by simp
have composition-table = composition-table0
  by (rule
    composition-fri-supported-root-composition-table-candidate-states-merkle-compatible-unique-if-clean-merge
    [OF compatible clean-merge state state0])
then show composition-table ∈ {composition-table0}
  by simp
qed
then show ?thesis
  by blast
qed

lemma composition-fri-supported-root-composition-table-candidate-states-distinct-imp-merkle-conflict-or-merge-collis
assumes candidate:
  composition-fri-supported-root-composition-table-candidate-state s fr
  f-fri-roots f-final as dg composition-fri-roots composition-table
  final-state
and candidate':
  composition-fri-supported-root-composition-table-candidate-state s fr
  f-fri-roots f-final as dg composition-fri-roots composition-table'
  final-state'
and distinct-tables: composition-table ≠ composition-table'
shows
  merkle-hash-value-conflict final-state final-state' ∨
  hash-map-output-collision
    (merkle-hash-state-merge final-state final-state')
proof (rule ccontr)
assume no-bad:
  ¬ (merkle-hash-value-conflict final-state final-state' ∨
    hash-map-output-collision
      (merkle-hash-state-merge final-state final-state'))
have compatible:

```

```

merkle-hash-maps-compatible final-state final-state'
using no-bad unfolding merkle-hash-maps-compatible-iff-no-value-conflict
by simp
have clean:
   $\neg$  hash-map-output-collision
    (merkle-hash-state-merge final-state final-state')
using no-bad by simp
have composition-table = composition-table'
by (rule
  composition-fri-supported-root-composition-table-candidate-states-merkle-compatible-unique-if-clean-merge
  [OF compatible clean candidate candidate'])
then show False
  using distinct-tables by contradiction
qed

lemma composition-fri-supported-root-composition-table-candidates-not-singleton-imp-pairwise-merkle-bad:
assumes not-unique:
   $\neg$  ( $\exists$  composition-table.
    composition-fri-supported-root-composition-table-candidates s fr
      f-fri-roots f-final as dg composition-fri-roots  $\subseteq$ 
        {composition-table})
shows
   $\exists$  composition-table final-state composition-table' final-state'.
    composition-fri-supported-root-composition-table-candidate-state s fr
      f-fri-roots f-final as dg composition-fri-roots composition-table
        final-state  $\wedge$ 
    composition-fri-supported-root-composition-table-candidate-state s fr
      f-fri-roots f-final as dg composition-fri-roots composition-table'
        final-state'  $\wedge$ 
    composition-table  $\neq$  composition-table'  $\wedge$ 
    (merkle-hash-value-conflict final-state final-state'  $\vee$ 
      hash-map-output-collision
        (merkle-hash-state-merge final-state final-state'))

proof -
let ?C =
  composition-fri-supported-root-composition-table-candidates s fr
    f-fri-roots f-final as dg composition-fri-roots
have nonempty: ?C  $\neq$  {}
using not-unique by auto
then obtain composition-table where table: composition-table  $\in$  ?C
by blast
have  $\neg$  ?C  $\subseteq$  {composition-table}
using not-unique by blast
then obtain composition-table' where table':
  composition-table'  $\in$  ?C
and distinct-tables: composition-table  $\neq$  composition-table'
by auto
from table obtain final-state where candidate:
  composition-fri-supported-root-composition-table-candidate-state s fr

```

*f-fri-roots f-final as dg composition-fri-roots composition-table
 final-state*
using *composition-fri-supported-root-composition-table-candidates-iff-state*
by *blast*
from *table'* **obtain** *final-state'* **where** *candidate'*:
composition-fri-supported-root-composition-table-candidate-state s fr
f-fri-roots f-final as dg composition-fri-roots composition-table'
final-state'
using *composition-fri-supported-root-composition-table-candidates-iff-state*
by *blast*
have *bad*:
merkle-hash-value-conflict final-state final-state' ∨
hash-map-output-collision
(merkle-hash-state-merge final-state final-state')
by (rule
composition-fri-supported-root-composition-table-candidate-states-distinct-imp-merkle-conflict-or-merge-collision
[OF candidate candidate' distinct-tables])
show *?thesis*
using *candidate candidate' distinct-tables bad* **by** *blast*
qed

definition *composition-fri-supported-root-pairwise-merkle-bad*

$:: (f, 'a) \text{ protocol-channel-scheme} \Rightarrow$
 $f \Rightarrow f \text{ list} \Rightarrow f \Rightarrow f \text{ list} \Rightarrow f \Rightarrow$
 $f \text{ list} \Rightarrow \text{bool}$

where

composition-fri-supported-root-pairwise-merkle-bad s fr f-fri-roots
f-final as dg composition-fri-roots $\longleftrightarrow$
($\exists$ composition-table final-state composition-table' final-state'.
composition-fri-supported-root-composition-table-candidate-state s fr
f-fri-roots f-final as dg composition-fri-roots composition-table
final-state $\wedge$
composition-fri-supported-root-composition-table-candidate-state s fr
f-fri-roots f-final as dg composition-fri-roots composition-table'
final-state' $\wedge$
composition-table $\neq$ composition-table' $\wedge$
(merkle-hash-value-conflict final-state final-state' $\vee$
hash-map-output-collision
(merkle-hash-state-merge final-state final-state'))))

definition *composition-fri-supported-pairwise-merkle-bad*

$:: (f, 'a) \text{ protocol-channel-scheme} \Rightarrow \text{bool}$

where

composition-fri-supported-pairwise-merkle-bad s $\longleftrightarrow$
($\exists$ fr f-fri-roots f-final as dg composition-fri-roots.
composition-fri-supported-root-pairwise-merkle-bad s fr f-fri-roots
f-final as dg composition-fri-roots)

lemma *composition-fri-supported-root-pairwise-merkle-bad-extends-initial*:

assumes *bad*:
composition-fri-supported-root-pairwise-merkle-bad s fr f-fri-roots
f-final as dg composition-fri-roots
obtains *composition-table final-state composition-table' final-state'*
where
composition-fri-supported-root-composition-table-candidate-state s fr
f-fri-roots f-final as dg composition-fri-roots composition-table
final-state
composition-fri-supported-root-composition-table-candidate-state s fr
f-fri-roots f-final as dg composition-fri-roots composition-table'
final-state'
composition-table $\neq$ composition-table'
merkle-hash-value-conflict final-state final-state' $\vee$
hash-map-output-collision
(merkle-hash-state-merge final-state final-state')
s $\leq$ final-state
s $\leq$ final-state'
proof –
from *bad* **obtain** *composition-table final-state composition-table' final-state'*
where *candidate*:
composition-fri-supported-root-composition-table-candidate-state s fr
f-fri-roots f-final as dg composition-fri-roots composition-table
final-state
and *candidate'*:
composition-fri-supported-root-composition-table-candidate-state s fr
f-fri-roots f-final as dg composition-fri-roots composition-table'
final-state'
and *distinct*: *composition-table $\neq$ composition-table'*
and *pair-bad*:
merkle-hash-value-conflict final-state final-state' $\vee$
hash-map-output-collision
(merkle-hash-state-merge final-state final-state')
unfolding *composition-fri-supported-root-pairwise-merkle-bad-def*
by *blast*
have *ext*: *s $\leq$ final-state*
by (*rule*
composition-fri-supported-root-composition-table-candidate-state-hash-extends
[OF candidate])
have *ext'*: *s $\leq$ final-state'*
by (*rule*
composition-fri-supported-root-composition-table-candidate-state-hash-extends
[OF candidate'])
show *?thesis*
by (*rule that*[*OF candidate candidate' distinct pair-bad ext ext'*])
qed

definition *composition-fri-supported-root-pairwise-coupling-bad*

$:: ('f, 'a) \text{ protocol-channel-scheme} \Rightarrow$
 $'f \Rightarrow 'f \text{ list} \Rightarrow 'f \Rightarrow 'f \text{ list} \Rightarrow 'f \Rightarrow$

'f list $\Rightarrow$ bool
where
composition-fri-supported-root-pairwise-coupling-bad s fr f-fri-roots
f-final as dg composition-fri-roots $\longleftrightarrow$
($\exists$ composition-table final-state composition-table' final-state'.
composition-fri-supported-root-composition-table-candidate-state s fr
f-fri-roots f-final as dg composition-fri-roots composition-table
final-state $\wedge$
composition-fri-supported-root-composition-table-candidate-state s fr
f-fri-roots f-final as dg composition-fri-roots composition-table'
final-state' $\wedge$
composition-table $\neq$ composition-table' $\wedge$
merkle-hash-pairwise-coupling-bad final-state final-state')

definition *composition-fri-supported-pairwise-coupling-bad*

:: ('f, 'a) protocol-channel-scheme $\Rightarrow$ bool

where

composition-fri-supported-pairwise-coupling-bad s $\longleftrightarrow$
($\exists$ fr f-fri-roots f-final as dg composition-fri-roots.
composition-fri-supported-root-pairwise-coupling-bad s fr f-fri-roots
f-final as dg composition-fri-roots)

lemma *composition-fri-supported-root-pairwise-merkle-bad-imp-local-collision-or-coupling-bad:*

assumes *bad:*

composition-fri-supported-root-pairwise-merkle-bad s fr f-fri-roots
f-final as dg composition-fri-roots

shows

($\exists$ composition-table final-state composition-table' final-state'.
composition-fri-supported-root-composition-table-candidate-state s fr
f-fri-roots f-final as dg composition-fri-roots composition-table
final-state $\wedge$
composition-fri-supported-root-composition-table-candidate-state s fr
f-fri-roots f-final as dg composition-fri-roots composition-table'
final-state' $\wedge$
composition-table $\neq$ composition-table' $\wedge$
(hash-map-output-collision final-state $\vee$
hash-map-output-collision final-state')) $\vee$
composition-fri-supported-root-pairwise-coupling-bad s fr f-fri-roots
f-final as dg composition-fri-roots

proof –

from *bad* **obtain** *composition-table final-state composition-table' final-state'*

where *candidate:*

composition-fri-supported-root-composition-table-candidate-state s fr
f-fri-roots f-final as dg composition-fri-roots composition-table
final-state

and *candidate':*

composition-fri-supported-root-composition-table-candidate-state s fr
f-fri-roots f-final as dg composition-fri-roots composition-table'
final-state'

```

and distinct: composition-table  $\neq$  composition-table'
and pair-bad:
  merkle-hash-value-conflict final-state final-state'  $\vee$ 
  hash-map-output-collision
    (merkle-hash-state-merge final-state final-state')
unfolding composition-fri-supported-root-pairwise-merkle-bad-def
by blast
have decomp:
  hash-map-output-collision final-state  $\vee$ 
  hash-map-output-collision final-state'  $\vee$ 
  merkle-hash-pairwise-coupling-bad final-state final-state'
by (rule merkle-hash-pairwise-bad-imp-local-collision-or-coupling-bad
  [OF pair-bad])
then show ?thesis
proof
  assume hash-map-output-collision final-state
  then show ?thesis
    using candidate candidate' distinct by blast
next
  assume rest:
    hash-map-output-collision final-state'  $\vee$ 
    merkle-hash-pairwise-coupling-bad final-state final-state'
  then show ?thesis
  proof
    assume hash-map-output-collision final-state'
    then show ?thesis
      using candidate candidate' distinct by blast
  next
    assume merkle-hash-pairwise-coupling-bad final-state final-state'
    then have
      composition-fri-supported-root-pairwise-coupling-bad s fr f-fri-roots
      f-final as dg composition-fri-roots
      unfolding composition-fri-supported-root-pairwise-coupling-bad-def
      using candidate candidate' distinct by blast
      then show ?thesis by simp
    qed
  qed
qed

lemma composition-fri-supported-pairwise-merkle-bad-imp-local-collision-or-coupling-bad:
assumes bad: composition-fri-supported-pairwise-merkle-bad s
shows
  ( $\exists$  fr f-fri-roots f-final as dg composition-fri-roots
  composition-table final-state composition-table' final-state'.
  composition-fri-supported-root-composition-table-candidate-state s fr
  f-fri-roots f-final as dg composition-fri-roots composition-table
  final-state  $\wedge$ 
  composition-fri-supported-root-composition-table-candidate-state s fr
  f-fri-roots f-final as dg composition-fri-roots composition-table')

```

$final-state' \wedge$
 $composition-table \neq composition-table' \wedge$
 $(hash-map-output-collision\ final-state \vee$
 $hash-map-output-collision\ final-state')) \vee$
 $composition-fri-supported-pairwise-coupling-bad\ s$
proof –
from *bad* **obtain** *fr f-fri-roots f-final as dg composition-fri-roots* **where**
 $root-bad:$
 $composition-fri-supported-root-pairwise-merkle-bad\ s\ fr\ f-fri-roots$
 $f-final\ as\ dg\ composition-fri-roots$
unfolding $composition-fri-supported-pairwise-merkle-bad-def$ **by** *blast*
from
 $composition-fri-supported-root-pairwise-merkle-bad-imp-local-collision-or-coupling-bad$
 $[OF\ root-bad]$
show *?thesis*
unfolding $composition-fri-supported-pairwise-coupling-bad-def$ **by** *blast*
qed

lemma $composition-fri-supported-pairwise-merkle-bad-imp-supported-hash-collision-possible-or-coupling-bad:$
assumes *bad: composition-fri-supported-pairwise-merkle-bad s*
shows
 $supported-hash-output-collision-possible\ s \vee$
 $composition-fri-supported-pairwise-coupling-bad\ s$
proof –
have *decomp:*
 $(\exists\ fr\ f-fri-roots\ f-final\ as\ dg\ composition-fri-roots$
 $composition-table\ final-state\ composition-table'\ final-state'.$
 $composition-fri-supported-root-composition-table-candidate-state\ s\ fr$
 $f-fri-roots\ f-final\ as\ dg\ composition-fri-roots\ composition-table$
 $final-state \wedge$
 $composition-fri-supported-root-composition-table-candidate-state\ s\ fr$
 $f-fri-roots\ f-final\ as\ dg\ composition-fri-roots\ composition-table'$
 $final-state' \wedge$
 $composition-table \neq composition-table' \wedge$
 $(hash-map-output-collision\ final-state \vee$
 $hash-map-output-collision\ final-state')) \vee$
 $composition-fri-supported-pairwise-coupling-bad\ s$
by (*rule*
 $composition-fri-supported-pairwise-merkle-bad-imp-local-collision-or-coupling-bad$
 $[OF\ bad])$
then show *?thesis*
proof
assume *local:*
 $\exists\ fr\ f-fri-roots\ f-final\ as\ dg\ composition-fri-roots$
 $composition-table\ final-state\ composition-table'\ final-state'.$
 $composition-fri-supported-root-composition-table-candidate-state\ s\ fr$
 $f-fri-roots\ f-final\ as\ dg\ composition-fri-roots\ composition-table$
 $final-state \wedge$
 $composition-fri-supported-root-composition-table-candidate-state\ s\ fr$


```

    f-fri-roots f-final as dg composition-fri-roots composition-table'
    final-state' ∧
    composition-table ≠ composition-table' ∧
    (hash-map-output-collision final-state ∨
     hash-map-output-collision final-state')
then obtain fr f-fri-roots f-final as dg composition-fri-roots
    composition-table final-state composition-table' final-state' where
    candidate:
    composition-fri-supported-root-composition-table-candidate-state s fr
    f-fri-roots f-final as dg composition-fri-roots composition-table
    final-state
and candidate':
    composition-fri-supported-root-composition-table-candidate-state s fr
    f-fri-roots f-final as dg composition-fri-roots composition-table'
    final-state'
and collision:
    hash-map-output-collision final-state ∨
    hash-map-output-collision final-state'
by blast
from collision show ?thesis
proof
    assume collision-final: hash-map-output-collision final-state
    from candidate obtain result trace-table query-idxs trace-bs comp-bs final
    rest where
    outcome:
    Some (result, final-state) ∈ set-dist (execute verify-monad s)
    and bound:
    accepted-with-bound-tables s (Some (result, final-state))
    trace-table composition-table as query-idxs
    unfolding
    composition-fri-supported-root-composition-table-candidate-state-def
    by blast
    have supported-hash-output-collision-possible s
    by (rule
    accepted-with-bound-tables-hash-collision-imp-supported-hash-output-collision-possible
    [OF outcome bound collision-final])
    then show ?thesis by simp
next
    assume collision-final': hash-map-output-collision final-state'
    from candidate' obtain result' trace-table' query-idxs' trace-bs'
    comp-bs' final' rest' where
    outcome':
    Some (result', final-state') ∈ set-dist (execute verify-monad s)
    and bound':
    accepted-with-bound-tables s (Some (result', final-state'))
    trace-table' composition-table' as query-idxs'
    unfolding
    composition-fri-supported-root-composition-table-candidate-state-def
    by blast

```

```

have supported-hash-output-collision-possible s
  by (rule
    accepted-with-bound-tables-hash-collision-imp-supported-hash-output-collision-possible
    [OF outcome' bound' collision-final'])
  then show ?thesis by simp
qed
next
  assume composition-fri-supported-pairwise-coupling-bad s
  then show ?thesis by simp
qed
qed

```

definition *alpha-output-local-candidate-ambiguity-bad*
 $:: (f, 'a) \text{ protocol-channel-scheme} \Rightarrow$
 $(\text{unit list} \times (f, 'a) \text{ protocol-channel-scheme}) \text{ option} \Rightarrow \text{bool}$
where
 $\text{alpha-output-local-candidate-ambiguity-bad } s \text{ out} \longleftrightarrow$
 $(\exists \text{ trace-table composition-table alphas query-idxs}$
 $\text{ trace-table' composition-table' alphas' query-idxs'.$
 $\text{accepted-with-bound-tables } s \text{ out trace-table composition-table}$
 $\text{ alphas query-idxs} \wedge$
 $\text{accepted-with-bound-tables } s \text{ out trace-table' composition-table'}$
 $\text{ alphas' query-idxs'} \wedge$
 $\text{trace-table} \neq \text{trace-table'})$

definition *query-output-local-candidate-ambiguity-bad*
 $:: (f, 'a) \text{ protocol-channel-scheme} \Rightarrow$
 $(\text{unit list} \times (f, 'a) \text{ protocol-channel-scheme}) \text{ option} \Rightarrow \text{bool}$
where
 $\text{query-output-local-candidate-ambiguity-bad } s \text{ out} \longleftrightarrow$
 $\text{accepted-bound-table-ambiguity-bad } s \text{ out}$

definition *trace-fri-output-local-candidate-ambiguity-bad*
 $:: (f, 'a) \text{ protocol-channel-scheme} \Rightarrow$
 $(\text{unit list} \times (f, 'a) \text{ protocol-channel-scheme}) \text{ option} \Rightarrow \text{bool}$
where
 $\text{trace-fri-output-local-candidate-ambiguity-bad } s \text{ out} \longleftrightarrow$
 $(\exists \text{ trace-table composition-table alphas query-idxs}$
 $\text{ trace-table' composition-table' alphas' query-idxs'}$
 $\text{ trace-bs dg composition-bs.}$
 $\text{accepted-with-bound-tables } s \text{ out trace-table composition-table}$
 $\text{ alphas query-idxs} \wedge$
 $\text{accepted-with-bound-tables } s \text{ out trace-table' composition-table'}$
 $\text{ alphas' query-idxs'} \wedge$
 $\text{accepted-fri-challenges } s \text{ out trace-bs dg composition-bs} \wedge$
 $\text{trace-table} \neq \text{trace-table'})$

definition *composition-fri-output-local-candidate-ambiguity-bad*
 $:: (f, 'a) \text{ protocol-channel-scheme} \Rightarrow$

$(\text{unit list} \times ('f, 'a) \text{ protocol-channel-scheme}) \text{ option} \Rightarrow \text{bool}$
where
 $\text{composition-fri-output-local-candidate-ambiguity-bad } s \text{ out} \longleftrightarrow$
 $(\exists \text{ trace-table } \text{composition-table } \text{alphas } \text{query-idxs}$
 $\text{trace-table}' \text{ composition-table}' \text{ alphas}' \text{ query-idxs}'$
 $\text{trace-bs } dg \text{ composition-bs.}$
 $\text{accepted-with-bound-tables } s \text{ out trace-table } \text{composition-table}$
 $\text{alphas } \text{query-idxs} \wedge$
 $\text{accepted-with-bound-tables } s \text{ out trace-table}' \text{ composition-table}'$
 $\text{alphas}' \text{ query-idxs}' \wedge$
 $\text{accepted-fri-challenges } s \text{ out trace-bs } dg \text{ composition-bs} \wedge$
 $\text{composition-table} \neq \text{composition-table}')$

definition $\text{supported-output-local-candidate-ambiguity-bad}$
 $:: ('f, 'a) \text{ protocol-channel-scheme} \Rightarrow$
 $(\text{unit list} \times ('f, 'a) \text{ protocol-channel-scheme}) \text{ option} \Rightarrow \text{bool}$
where
 $\text{supported-output-local-candidate-ambiguity-bad } s \text{ out} \longleftrightarrow$
 $\alpha\text{-output-local-candidate-ambiguity-bad } s \text{ out} \vee$
 $\text{query-output-local-candidate-ambiguity-bad } s \text{ out} \vee$
 $\text{trace-fri-output-local-candidate-ambiguity-bad } s \text{ out} \vee$
 $\text{composition-fri-output-local-candidate-ambiguity-bad } s \text{ out}$

lemma $\alpha\text{-output-local-candidate-ambiguity-bad-imp-accepted-bound-table-ambiguity-bad}:$
assumes $\alpha\text{-output-local-candidate-ambiguity-bad } s \text{ out}$
shows $\text{accepted-bound-table-ambiguity-bad } s \text{ out}$
using assms
unfolding $\alpha\text{-output-local-candidate-ambiguity-bad-def}$
 $\text{accepted-bound-table-ambiguity-bad-def}$
by blast

lemma $\text{query-output-local-candidate-ambiguity-bad-imp-accepted-bound-table-ambiguity-bad}:$
assumes $\text{query-output-local-candidate-ambiguity-bad } s \text{ out}$
shows $\text{accepted-bound-table-ambiguity-bad } s \text{ out}$
using assms **unfolding** $\text{query-output-local-candidate-ambiguity-bad-def}$.

lemma $\text{trace-fri-output-local-candidate-ambiguity-bad-imp-accepted-bound-table-ambiguity-bad}:$
assumes $\text{trace-fri-output-local-candidate-ambiguity-bad } s \text{ out}$
shows $\text{accepted-bound-table-ambiguity-bad } s \text{ out}$
using assms
unfolding $\text{trace-fri-output-local-candidate-ambiguity-bad-def}$
 $\text{accepted-bound-table-ambiguity-bad-def}$
by blast

lemma $\text{composition-fri-output-local-candidate-ambiguity-bad-imp-accepted-bound-table-ambiguity-bad}:$
assumes $\text{composition-fri-output-local-candidate-ambiguity-bad } s \text{ out}$
shows $\text{accepted-bound-table-ambiguity-bad } s \text{ out}$
using assms
unfolding $\text{composition-fri-output-local-candidate-ambiguity-bad-def}$

accepted-bound-table-ambiguity-bad-def
by *blast*

lemma *supported-output-local-candidate-ambiguity-bad-imp-accepted-bound-table-ambiguity-bad:*
assumes *supported-output-local-candidate-ambiguity-bad s out*
shows *accepted-bound-table-ambiguity-bad s out*
using *assms*
unfolding *supported-output-local-candidate-ambiguity-bad-def*
using
alpha-output-local-candidate-ambiguity-bad-imp-accepted-bound-table-ambiguity-bad
query-output-local-candidate-ambiguity-bad-imp-accepted-bound-table-ambiguity-bad
trace-fri-output-local-candidate-ambiguity-bad-imp-accepted-bound-table-ambiguity-bad
composition-fri-output-local-candidate-ambiguity-bad-imp-accepted-bound-table-ambiguity-bad
by *blast*

lemma *supported-output-local-candidate-ambiguity-bad-imp-hash-map-output-collision-bad:*
assumes *supported-output-local-candidate-ambiguity-bad s out*
shows *hash-map-output-collision-bad s out*
by (*rule*
accepted-bound-table-ambiguity-bad-imp-hash-map-output-collision-bad)
(*rule*
supported-output-local-candidate-ambiguity-bad-imp-accepted-bound-table-ambiguity-bad
[*OF assms*])

lemma *supported-output-local-candidate-ambiguity-bad-mono-hash-map-output-collision-bad:*
wp-event verify-mono
(*supported-output-local-candidate-ambiguity-bad s*) $s \leq$
wp-event verify-mono (*hash-map-output-collision-bad s*) s
by (*rule wp-event-mono*)
(*rule*
supported-output-local-candidate-ambiguity-bad-imp-hash-map-output-collision-bad)

definition *supported-output-local-side-bad*
 $:: (f, 'a) \text{ protocol-channel-scheme} \Rightarrow$
 $(\text{unit list} \times (f, 'a) \text{ protocol-channel-scheme}) \text{ option} \Rightarrow \text{bool}$
where
supported-output-local-side-bad s out $\longleftrightarrow$
hash-map-output-collision-bad s out $\vee$
supported-output-local-candidate-ambiguity-bad s out

lemma *supported-output-local-side-bad-iff-hash-map-output-collision-bad:*
supported-output-local-side-bad s out $\longleftrightarrow$
hash-map-output-collision-bad s out
unfolding *supported-output-local-side-bad-def*
using
supported-output-local-candidate-ambiguity-bad-imp-hash-map-output-collision-bad
by *blast*

lemma *supported-output-local-side-bad-imp-accepted:*

assumes *supported-output-local-side-bad s out*
shows *accepted out*
using *assms*
unfolding *supported-output-local-side-bad-iff-hash-map-output-collision-bad*
by (*rule hash-map-output-collision-bad-imp-accepted*)

lemma *supported-output-local-side-bad-mono-hash-map-output-collision-bad:*
wp-event verify-monad (supported-output-local-side-bad s) s ≤
wp-event verify-monad (hash-map-output-collision-bad s) s
by (*rule wp-event-mono*)
(simp add: supported-output-local-side-bad-iff-hash-map-output-collision-bad)

end

end

theory *Soundness-Reductions-Side*
imports *Soundness-Reductions-Composition*
begin

Supported pairwise/output-local side-event reductions.

context *soundness*
begin

The following cross-continuation predicates are retained temporarily as a record of the earlier proof route. The refined route below does not charge same-key/different-output lazy-oracle continuations as bad events; it uses *supported-output-local-side-bad* instead.

definition *supported-pairwise-merkle-bad*
 $:: (f, 'a) \text{ protocol-channel-scheme} \Rightarrow \text{bool}$
where
supported-pairwise-merkle-bad s $\longleftrightarrow$
alpha-supported-pairwise-merkle-bad s $\vee$
query-supported-pairwise-merkle-bad s $\vee$
trace-fri-supported-pairwise-merkle-bad s $\vee$
composition-fri-supported-pairwise-merkle-bad s

definition *supported-pairwise-coupling-bad*
 $:: (f, 'a) \text{ protocol-channel-scheme} \Rightarrow \text{bool}$
where
supported-pairwise-coupling-bad s $\longleftrightarrow$
alpha-supported-pairwise-coupling-bad s $\vee$
query-supported-pairwise-coupling-bad s $\vee$
trace-fri-supported-pairwise-coupling-bad s $\vee$
composition-fri-supported-pairwise-coupling-bad s

lemma *supported-pairwise-merkle-bad-imp-supported-hash-collision-possible-or-coupling-bad:*
assumes *bad: supported-pairwise-merkle-bad s*
shows

supported-hash-output-collision-possible s $\vee$
supported-pairwise-coupling-bad s
using *assms*
unfolding *supported-pairwise-merkle-bad-def*
supported-pairwise-coupling-bad-def
using
alpha-supported-pairwise-merkle-bad-imp-supported-hash-collision-possible-or-coupling-bad
query-supported-pairwise-merkle-bad-imp-supported-hash-collision-possible-or-coupling-bad
trace-fri-supported-pairwise-merkle-bad-imp-supported-hash-collision-possible-or-coupling-bad
composition-fri-supported-pairwise-merkle-bad-imp-supported-hash-collision-possible-or-coupling-bad
by *blast*

lemma *no-supported-pairwise-merkle-bad-if-no-hash-collision-possible-no-coupling-bad:*
assumes *no-collision:* $\neg$ *supported-hash-output-collision-possible s*
and *no-coupling:* $\neg$ *supported-pairwise-coupling-bad s*
shows $\neg$ *supported-pairwise-merkle-bad s*
proof
assume *bad:* *supported-pairwise-merkle-bad s*
have *supported-hash-output-collision-possible s* $\vee$
supported-pairwise-coupling-bad s
by (*rule*
supported-pairwise-merkle-bad-imp-supported-hash-collision-possible-or-coupling-bad
[*OF bad*])
then show *False*
using *no-collision no-coupling* **by** *blast*
qed

lemma *no-alpha-supported-pairwise-merkle-bad-if-no-pairwise-side-bad:*
assumes *no-collision:* $\neg$ *supported-hash-output-collision-possible s*
and *no-coupling:* $\neg$ *supported-pairwise-coupling-bad s*
shows $\neg$ *alpha-supported-pairwise-merkle-bad s*
proof –
have $\neg$ *supported-pairwise-merkle-bad s*
by (*rule*
no-supported-pairwise-merkle-bad-if-no-hash-collision-possible-no-coupling-bad
[*OF no-collision no-coupling*])
then show *?thesis*
unfolding *supported-pairwise-merkle-bad-def* **by** *blast*
qed

lemma *no-query-supported-pairwise-merkle-bad-if-no-pairwise-side-bad:*
assumes *no-collision:* $\neg$ *supported-hash-output-collision-possible s*
and *no-coupling:* $\neg$ *supported-pairwise-coupling-bad s*
shows $\neg$ *query-supported-pairwise-merkle-bad s*
proof –
have $\neg$ *supported-pairwise-merkle-bad s*
by (*rule*
no-supported-pairwise-merkle-bad-if-no-hash-collision-possible-no-coupling-bad
[*OF no-collision no-coupling*])

then show *?thesis*
unfolding *supported-pairwise-merkle-bad-def* **by** *blast*
qed

lemma *no-trace-fri-supported-pairwise-merkle-bad-if-no-pairwise-side-bad*:
assumes *no-collision*: $\neg$ *supported-hash-output-collision-possible* *s*
and *no-coupling*: $\neg$ *supported-pairwise-coupling-bad* *s*
shows $\neg$ *trace-fri-supported-pairwise-merkle-bad* *s*
proof –
have $\neg$ *supported-pairwise-merkle-bad* *s*
by (*rule*
no-supported-pairwise-merkle-bad-if-no-hash-collision-possible-no-coupling-bad
[*OF no-collision no-coupling*])
then show *?thesis*
unfolding *supported-pairwise-merkle-bad-def* **by** *blast*
qed

lemma *no-composition-fri-supported-pairwise-merkle-bad-if-no-pairwise-side-bad*:
assumes *no-collision*: $\neg$ *supported-hash-output-collision-possible* *s*
and *no-coupling*: $\neg$ *supported-pairwise-coupling-bad* *s*
shows $\neg$ *composition-fri-supported-pairwise-merkle-bad* *s*
proof –
have $\neg$ *supported-pairwise-merkle-bad* *s*
by (*rule*
no-supported-pairwise-merkle-bad-if-no-hash-collision-possible-no-coupling-bad
[*OF no-collision no-coupling*])
then show *?thesis*
unfolding *supported-pairwise-merkle-bad-def* **by** *blast*
qed

definition *alpha-supported-pairwise-coupling-event*
 $:: (f, 'a) \text{ protocol-channel-scheme} \Rightarrow$
 $(\text{unit list} \times (f, 'a) \text{ protocol-channel-scheme}) \text{ option} \Rightarrow \text{bool}$
where
alpha-supported-pairwise-coupling-event *s out* $\longleftrightarrow$
 $(\exists \text{ result final-state fr f-fri-roots f-final trace-table}$
 $\text{trace-table' final-state'.$
 $\text{out} = \text{Some} (\text{result}, \text{final-state}) \wedge$
 $\text{alpha-header-supported-trace-table-candidate-state } s \text{ fr f-fri-roots}$
 $\text{f-final trace-table final-state} \wedge$
 $\text{alpha-header-supported-trace-table-candidate-state } s \text{ fr f-fri-roots}$
 $\text{f-final trace-table' final-state'} \wedge$
 $\text{trace-table} \neq \text{trace-table'} \wedge$
 $\text{merkle-hash-pairwise-coupling-bad final-state final-state'})$

definition *query-supported-pairwise-coupling-event*
 $:: (f, 'a) \text{ protocol-channel-scheme} \Rightarrow$
 $(\text{unit list} \times (f, 'a) \text{ protocol-channel-scheme}) \text{ option} \Rightarrow \text{bool}$
where

query-supported-pairwise-coupling-event s $out \longleftrightarrow$
 $(\exists \text{result final-state fr f-fri-roots f-final as dg}$
 $\quad \text{composition-fri-roots final trace-table composition-table}$
 $\quad \text{trace-table' composition-table' final-state'}. \wedge$
 $\text{out} = \text{Some (result, final-state)} \wedge$
 $\text{query-header-supported-table-candidate-state } s \text{ fr f-fri-roots f-final}$
 $\quad \text{as dg composition-fri-roots final trace-table composition-table}$
 $\quad \text{final-state} \wedge$
 $\text{query-header-supported-table-candidate-state } s \text{ fr f-fri-roots f-final}$
 $\quad \text{as dg composition-fri-roots final trace-table' composition-table'}$
 $\quad \text{final-state'} \wedge$
 $(\text{trace-table} \neq \text{trace-table'} \vee$
 $\quad \text{composition-table} \neq \text{composition-table'}) \wedge$
 $\text{merkle-hash-pairwise-coupling-bad final-state final-state'})$

definition *trace-fri-supported-pairwise-coupling-event*

$:: (f, 'a) \text{ protocol-channel-scheme} \Rightarrow$
 $(\text{unit list} \times (f, 'a) \text{ protocol-channel-scheme}) \text{ option} \Rightarrow \text{bool}$

where

$\text{trace-fri-supported-pairwise-coupling-event } s \text{ out} \longleftrightarrow$
 $(\exists \text{result final-state fr trace-table trace-table' final-state'}. \wedge$
 $\text{out} = \text{Some (result, final-state)} \wedge$
 $\text{trace-fri-supported-root-trace-table-candidate-state } s \text{ fr trace-table}$
 $\quad \text{final-state} \wedge$
 $\text{trace-fri-supported-root-trace-table-candidate-state } s \text{ fr trace-table'}$
 $\quad \text{final-state'} \wedge$
 $\text{trace-table} \neq \text{trace-table'} \wedge$
 $\text{merkle-hash-pairwise-coupling-bad final-state final-state'})$

definition *composition-fri-supported-pairwise-coupling-event*

$:: (f, 'a) \text{ protocol-channel-scheme} \Rightarrow$
 $(\text{unit list} \times (f, 'a) \text{ protocol-channel-scheme}) \text{ option} \Rightarrow \text{bool}$

where

$\text{composition-fri-supported-pairwise-coupling-event } s \text{ out} \longleftrightarrow$
 $(\exists \text{result final-state fr f-fri-roots f-final as dg}$
 $\quad \text{composition-fri-roots composition-table composition-table'}$
 $\quad \text{final-state'}. \wedge$
 $\text{out} = \text{Some (result, final-state)} \wedge$
 $\text{composition-fri-supported-root-composition-table-candidate-state } s \text{ fr}$
 $\quad \text{f-fri-roots f-final as dg composition-fri-roots composition-table}$
 $\quad \text{final-state} \wedge$
 $\text{composition-fri-supported-root-composition-table-candidate-state } s \text{ fr}$
 $\quad \text{f-fri-roots f-final as dg composition-fri-roots composition-table'}$
 $\quad \text{final-state'} \wedge$
 $\text{composition-table} \neq \text{composition-table'} \wedge$
 $\text{merkle-hash-pairwise-coupling-bad final-state final-state'})$

definition *supported-pairwise-coupling-event*

$:: (f, 'a) \text{ protocol-channel-scheme} \Rightarrow$

$(\text{unit list} \times ('f, 'a) \text{ protocol-channel-scheme}) \text{ option} \Rightarrow \text{bool}$
where
 $\text{supported-pairwise-coupling-event } s \text{ out} \longleftrightarrow$
 $\alpha\text{-supported-pairwise-coupling-event } s \text{ out} \vee$
 $\text{query-supported-pairwise-coupling-event } s \text{ out} \vee$
 $\text{trace-fri-supported-pairwise-coupling-event } s \text{ out} \vee$
 $\text{composition-fri-supported-pairwise-coupling-event } s \text{ out}$

definition *supported-pairwise-side-bad*
 $:: ('f, 'a) \text{ protocol-channel-scheme} \Rightarrow$
 $(\text{unit list} \times ('f, 'a) \text{ protocol-channel-scheme}) \text{ option} \Rightarrow \text{bool}$
where
 $\text{supported-pairwise-side-bad } s \text{ out} \longleftrightarrow$
 $\text{hash-map-output-collision-bad } s \text{ out} \vee$
 $\text{supported-pairwise-coupling-event } s \text{ out}$

lemma *alpha-supported-pairwise-coupling-event-imp-accepted:*
assumes $\alpha\text{-supported-pairwise-coupling-event } s \text{ out}$
shows accepted out
using *assms*
unfolding $\alpha\text{-supported-pairwise-coupling-event-def accepted-def}$
by *auto*

lemma *query-supported-pairwise-coupling-event-imp-accepted:*
assumes $\text{query-supported-pairwise-coupling-event } s \text{ out}$
shows accepted out
using *assms*
unfolding $\text{query-supported-pairwise-coupling-event-def accepted-def}$
by *auto*

lemma *trace-fri-supported-pairwise-coupling-event-imp-accepted:*
assumes $\text{trace-fri-supported-pairwise-coupling-event } s \text{ out}$
shows accepted out
using *assms*
unfolding $\text{trace-fri-supported-pairwise-coupling-event-def accepted-def}$
by *auto*

lemma *composition-fri-supported-pairwise-coupling-event-imp-accepted:*
assumes $\text{composition-fri-supported-pairwise-coupling-event } s \text{ out}$
shows accepted out
using *assms*
unfolding $\text{composition-fri-supported-pairwise-coupling-event-def accepted-def}$
by *auto*

lemma *supported-pairwise-coupling-event-imp-accepted:*
assumes $\text{supported-pairwise-coupling-event } s \text{ out}$
shows accepted out
using *assms*
unfolding $\text{supported-pairwise-coupling-event-def}$

using *alpha-supported-pairwise-coupling-event-imp-accepted*
query-supported-pairwise-coupling-event-imp-accepted
trace-fri-supported-pairwise-coupling-event-imp-accepted
composition-fri-supported-pairwise-coupling-event-imp-accepted
by *blast*

lemma *supported-pairwise-side-bad-imp-accepted*:
assumes *supported-pairwise-side-bad s out*
shows *accepted out*
using *assms*
unfolding *supported-pairwise-side-bad-def*
using *hash-map-output-collision-bad-imp-accepted*
supported-pairwise-coupling-event-imp-accepted
by *blast*

lemma *supported-pairwise-side-bad-mono-accepted*:
wp-event verify-monad (supported-pairwise-side-bad s) s ≤
wp-event verify-monad accepted s
by (rule *wp-event-mono*)
(rule *supported-pairwise-side-bad-imp-accepted*)

lemma *supported-pairwise-side-bad-union-bound*:
fixes *hash-error coupling-error :: prob*
assumes *hash-bound*:
wp-event verify-monad (hash-map-output-collision-bad s) s ≤
hash-error
and *coupling-bound*:
wp-event verify-monad (supported-pairwise-coupling-event s) s ≤
coupling-error
shows
wp-event verify-monad (supported-pairwise-side-bad s) s ≤
hash-error + coupling-error
proof –
have *wp-event verify-monad (supported-pairwise-side-bad s) s ≤*
wp-event verify-monad (hash-map-output-collision-bad s) s +
wp-event verify-monad (supported-pairwise-coupling-event s) s
unfolding *supported-pairwise-side-bad-def*
supported-pairwise-coupling-event-def
by (rule *wp-event-union-bound*)
also have *... ≤ hash-error + coupling-error*
by (intro *add-mono hash-bound coupling-bound*)
finally show *?thesis* .
qed

lemma *verifier-initial-hash-map-output-collision-bad-bound-concrete*:
wp-event verify-monad
(hash-map-output-collision-bad (verifier-initial-state tr))
(verifier-initial-state tr) ≤ concrete-merkle-binding-error
proof –

let $?s = \text{verifier-initial-state } tr$
have $\text{wp-event verify-monad } (\text{hash-map-output-collision-bad } ?s) \text{ } ?s \leq$
 $\text{wp-event verify-monad } (\text{hash-map-new-output-collision-bad } ?s) \text{ } ?s$
by (rule wp-event-mono)
 $(\text{rule hash-map-output-collision-bad-imp-new-collision-bad-if-initial-clean,}$
 $\text{simp, assumption})$
also have $\dots \leq \text{concrete-merkle-binding-error}$
by (rule $\text{verifier-initial-hash-map-new-output-collision-bad-bound-concrete}$)
finally show $?thesis$.
qed

lemma $\text{verifier-initial-supported-output-local-candidate-ambiguity-bad-bound-concrete:}$

$\text{wp-event verify-monad}$
 $(\text{supported-output-local-candidate-ambiguity-bad}$
 $(\text{verifier-initial-state } tr))$
 $(\text{verifier-initial-state } tr) \leq$
 $\text{concrete-merkle-binding-error}$
proof –
have $\text{wp-event verify-monad}$
 $(\text{supported-output-local-candidate-ambiguity-bad}$
 $(\text{verifier-initial-state } tr))$
 $(\text{verifier-initial-state } tr) \leq$
 $\text{wp-event verify-monad}$
 $(\text{hash-map-output-collision-bad } (\text{verifier-initial-state } tr))$
 $(\text{verifier-initial-state } tr)$
by (rule
 $\text{supported-output-local-candidate-ambiguity-bad-mono-hash-map-output-collision-bad})$
also have $\dots \leq \text{concrete-merkle-binding-error}$
by (rule $\text{verifier-initial-hash-map-output-collision-bad-bound-concrete}$)
finally show $?thesis$.
qed

lemma $\text{verifier-initial-supported-output-local-side-bad-bound-concrete:}$

$\text{wp-event verify-monad}$
 $(\text{supported-output-local-side-bad } (\text{verifier-initial-state } tr))$
 $(\text{verifier-initial-state } tr) \leq$
 $\text{concrete-merkle-binding-error}$
unfolding $\text{supported-output-local-side-bad-iff-hash-map-output-collision-bad}$
by (rule $\text{verifier-initial-hash-map-output-collision-bad-bound-concrete}$)

lemma $\text{verifier-initial-supported-pairwise-side-bad-bound-if-coupling-event-bound:}$

assumes coupling-bound:
 $\text{wp-event verify-monad}$
 $(\text{supported-pairwise-coupling-event } (\text{verifier-initial-state } tr))$
 $(\text{verifier-initial-state } tr) \leq \text{coupling-error}$
shows
 $\text{wp-event verify-monad}$
 $(\text{supported-pairwise-side-bad } (\text{verifier-initial-state } tr))$
 $(\text{verifier-initial-state } tr) \leq$

$\text{concrete-merkle-binding-error} + \text{coupling-error}$
by (*rule supported-pairwise-side-bad-union-bound*
[OF verifier-initial-hash-map-output-collision-bad-bound-concrete
coupling-bound])

lemma *supported-hash-output-collision-possible-imp-exists-supported-pairwise-side-bad:*
assumes *supported-hash-output-collision-possible s*
shows $\exists \text{out} \in \text{set-dist } (\text{execute verify-monad } s).$
supported-pairwise-side-bad s out
using *assms*
unfolding *supported-hash-output-collision-possible-def*
supported-pairwise-side-bad-def
by *blast*

lemma *alpha-supported-pairwise-coupling-bad-imp-exists-supported-pairwise-side-bad:*
assumes *alpha-bad: alpha-supported-pairwise-coupling-bad s*
shows $\exists \text{out} \in \text{set-dist } (\text{execute verify-monad } s).$
supported-pairwise-side-bad s out

proof —

from *alpha-bad* **obtain** *fr f-fri-roots f-final trace-table final-state*
trace-table' final-state' where
candidate:
alpha-header-supported-trace-table-candidate-state s fr f-fri-roots
f-final trace-table final-state
and *candidate':*
alpha-header-supported-trace-table-candidate-state s fr f-fri-roots
f-final trace-table' final-state'
and *distinct: trace-table $\neq$ trace-table'*
and *pair-bad:*
merkle-hash-pairwise-coupling-bad final-state final-state'
unfolding *alpha-supported-pairwise-coupling-bad-def*
alpha-header-supported-pairwise-coupling-bad-def
by *blast*
from *candidate* **obtain** *result composition-table as query-idxs dg*
composition-fri-roots final rest where
outcome:
Some (result, final-state) $\in$ set-dist (execute verify-monad s)
and *bound:*
accepted-with-bound-tables s (Some (result, final-state))
trace-table composition-table as query-idxs
unfolding *alpha-header-supported-trace-table-candidate-state-def* **by** *blast*
have *coupling:*
supported-pairwise-coupling-event s (Some (result, final-state))
unfolding *supported-pairwise-coupling-event-def*
alpha-supported-pairwise-coupling-event-def
using *candidate candidate' distinct pair-bad* **by** *blast*
have *side:*
supported-pairwise-side-bad s (Some (result, final-state))
unfolding *supported-pairwise-side-bad-def*

```

    using coupling by simp
show ?thesis
    using outcome side by blast
qed

lemma query-supported-pairwise-coupling-bad-imp-exists-supported-pairwise-side-bad:
  assumes query-bad: query-supported-pairwise-coupling-bad s
  shows  $\exists$  out  $\in$  set-dist (execute verify-monad s).
    supported-pairwise-side-bad s out
proof -
  from query-bad obtain fr f-fri-roots f-final as dg composition-fri-roots
    final trace-table composition-table final-state trace-table'
    composition-table' final-state' where
  candidate:
    query-header-supported-table-candidate-state s fr f-fri-roots f-final
    as dg composition-fri-roots final trace-table composition-table
    final-state
  and candidate':
    query-header-supported-table-candidate-state s fr f-fri-roots f-final
    as dg composition-fri-roots final trace-table' composition-table'
    final-state'
  and distinct:
    trace-table  $\neq$  trace-table'  $\vee$ 
    composition-table  $\neq$  composition-table'
  and pair-bad:
    merkle-hash-pairwise-coupling-bad final-state final-state'
  unfolding query-supported-pairwise-coupling-bad-def
    query-header-supported-pairwise-coupling-bad-def
  by blast
  from candidate obtain result query-idxs rest where
  outcome:
    Some (result, final-state)  $\in$  set-dist (execute verify-monad s)
  and bound:
    accepted-with-bound-tables s (Some (result, final-state))
    trace-table composition-table as query-idxs
  unfolding query-header-supported-table-candidate-state-def by blast
  have coupling:
    supported-pairwise-coupling-event s (Some (result, final-state))
  unfolding supported-pairwise-coupling-event-def
    query-supported-pairwise-coupling-event-def
  using candidate candidate' distinct pair-bad by blast
  have side:
    supported-pairwise-side-bad s (Some (result, final-state))
  unfolding supported-pairwise-side-bad-def
  using coupling by simp
show ?thesis
  using outcome side by blast
qed

```

lemma *trace-fri-supported-pairwise-coupling-bad-imp-exists-supported-pairwise-side-bad:*
assumes *trace-bad: trace-fri-supported-pairwise-coupling-bad s*
shows $\exists \text{out} \in \text{set-dist } (\text{execute verify-monad } s).$
supported-pairwise-side-bad s out

proof –
from *trace-bad* **obtain** *fr trace-table final-state trace-table' final-state'*
where *candidate:*
trace-fri-supported-root-trace-table-candidate-state s fr trace-table
final-state
and *candidate':*
trace-fri-supported-root-trace-table-candidate-state s fr trace-table'
final-state'
and *distinct: trace-table $\neq$ trace-table'*
and *pair-bad:*
merkle-hash-pairwise-coupling-bad final-state final-state'
unfolding *trace-fri-supported-pairwise-coupling-bad-def*
trace-fri-supported-root-pairwise-coupling-bad-def
by *blast*
from *candidate* **obtain** *result composition-table as query-idxs trace-bs dg*
comp-bs f-fri-roots f-final header-as composition-fri-roots final rest
where *outcome:*
Some (result, final-state) $\in$ set-dist (execute verify-monad s)
and *bound:*
accepted-with-bound-tables s (Some (result, final-state))
trace-table composition-table as query-idxs
unfolding *trace-fri-supported-root-trace-table-candidate-state-def*
by *blast*
have *coupling:*
supported-pairwise-coupling-event s (Some (result, final-state))
unfolding *supported-pairwise-coupling-event-def*
trace-fri-supported-pairwise-coupling-event-def
using *candidate candidate' distinct pair-bad* **by** *blast*
have *side:*
supported-pairwise-side-bad s (Some (result, final-state))
unfolding *supported-pairwise-side-bad-def*
using *coupling* **by** *simp*
show *?thesis*
using *outcome side* **by** *blast*

qed

lemma *composition-fri-supported-pairwise-coupling-bad-imp-exists-supported-pairwise-side-bad:*
assumes *composition-bad: composition-fri-supported-pairwise-coupling-bad s*
shows $\exists \text{out} \in \text{set-dist } (\text{execute verify-monad } s).$
supported-pairwise-side-bad s out

proof –
from *composition-bad* **obtain** *fr f-fri-roots f-final as dg*
composition-fri-roots composition-table final-state composition-table'
final-state' where
candidate:

```

    composition-fri-supported-root-composition-table-candidate-state s fr
      f-fri-roots f-final as dg composition-fri-roots composition-table
      final-state
  and candidate':
    composition-fri-supported-root-composition-table-candidate-state s fr
      f-fri-roots f-final as dg composition-fri-roots composition-table'
      final-state'
  and distinct: composition-table  $\neq$  composition-table'
  and pair-bad:
    merkle-hash-pairwise-coupling-bad final-state final-state'
  unfolding composition-fri-supported-pairwise-coupling-bad-def
    composition-fri-supported-root-pairwise-coupling-bad-def
  by blast
from candidate obtain result trace-table query-idxs trace-bs comp-bs final
  rest where
outcome:
  Some (result, final-state)  $\in$  set-dist (execute verify-monad s)
and bound:
  accepted-with-bound-tables s (Some (result, final-state))
  trace-table composition-table as query-idxs
unfolding
  composition-fri-supported-root-composition-table-candidate-state-def
by blast
have coupling:
  supported-pairwise-coupling-event s (Some (result, final-state))
unfolding supported-pairwise-coupling-event-def
  composition-fri-supported-pairwise-coupling-event-def
using candidate candidate' distinct pair-bad by blast
have side:
  supported-pairwise-side-bad s (Some (result, final-state))
unfolding supported-pairwise-side-bad-def
using coupling by simp
show ?thesis
  using outcome side by blast
qed

lemma supported-pairwise-coupling-bad-imp-exists-supported-pairwise-side-bad:
  assumes coupling: supported-pairwise-coupling-bad s
  shows  $\exists$  out  $\in$  set-dist (execute verify-monad s).
    supported-pairwise-side-bad s out
  using coupling
  unfolding supported-pairwise-coupling-bad-def
  using
    alpha-supported-pairwise-coupling-bad-imp-exists-supported-pairwise-side-bad
    query-supported-pairwise-coupling-bad-imp-exists-supported-pairwise-side-bad
    trace-fri-supported-pairwise-coupling-bad-imp-exists-supported-pairwise-side-bad
    composition-fri-supported-pairwise-coupling-bad-imp-exists-supported-pairwise-side-bad
  by blast

```

lemma *supported-pairwise-merkle-bad-imp-exists-supported-pairwise-side-bad:*
assumes *bad: supported-pairwise-merkle-bad s*
shows $\exists out \in \text{set-dist } (\text{execute verify-monad } s).$
supported-pairwise-side-bad s out
proof –
have *side:*
supported-hash-output-collision-possible s $\vee$
supported-pairwise-coupling-bad s
by (rule
supported-pairwise-merkle-bad-imp-supported-hash-collision-possible-or-coupling-bad
 $[OF \text{ bad}]$)
then show *?thesis*
proof
assume *supported-hash-output-collision-possible s*
then show *?thesis*
by (rule
supported-hash-output-collision-possible-imp-exists-supported-pairwise-side-bad)
next
assume *supported-pairwise-coupling-bad s*
then show *?thesis*
by (rule
supported-pairwise-coupling-bad-imp-exists-supported-pairwise-side-bad)
qed
qed

lemma *no-supported-pairwise-side-bad-on-support-imp-no-hash-collision-possible:*
assumes *no-side:*
 $\bigwedge out. out \in \text{set-dist } (\text{execute verify-monad } s) \implies$
 $\neg \text{supported-pairwise-side-bad } s \text{ out}$
shows $\neg \text{supported-hash-output-collision-possible } s$
proof
assume *possible: supported-hash-output-collision-possible s*
from *supported-hash-output-collision-possible-imp-exists-supported-pairwise-side-bad*
 $[OF \text{ possible}]$
obtain *out where out:*
 $out \in \text{set-dist } (\text{execute verify-monad } s)$
and *side: supported-pairwise-side-bad s out*
by *blast*
show *False*
using *no-side* $[OF \text{ out}]$ *side* **by** *contradiction*
qed

lemma *no-supported-pairwise-side-bad-on-support-imp-no-coupling-bad:*
assumes *no-side:*
 $\bigwedge out. out \in \text{set-dist } (\text{execute verify-monad } s) \implies$
 $\neg \text{supported-pairwise-side-bad } s \text{ out}$
shows $\neg \text{supported-pairwise-coupling-bad } s$
proof
assume *coupling: supported-pairwise-coupling-bad s*

from *supported-pairwise-coupling-bad-imp-exists-supported-pairwise-side-bad*
 [OF *coupling*]
obtain *out* **where** *out*:
 out $\in$ *set-dist* (*execute verify-monad s*)
and *side*: *supported-pairwise-side-bad s out*
by *blast*
show *False*
 using *no-side*[OF *out*] *side* **by** *contradiction*
qed

lemma *no-supported-pairwise-side-bad-on-support-imp-no-pairwise-side-bad*:
assumes *no-side*:
 $\bigwedge out. out \in \text{set-dist } (\text{execute verify-monad } s) \implies$
 $\neg \text{supported-pairwise-side-bad } s \text{ out}$
shows
 $\neg \text{supported-hash-output-collision-possible } s$
 $\neg \text{supported-pairwise-coupling-bad } s$
proof –
show $\neg \text{supported-hash-output-collision-possible } s$
 by (rule
 no-supported-pairwise-side-bad-on-support-imp-no-hash-collision-possible
 [OF *no-side*])
show $\neg \text{supported-pairwise-coupling-bad } s$
 by (rule
 no-supported-pairwise-side-bad-on-support-imp-no-coupling-bad
 [OF *no-side*])
qed

lemma *composition-fri-supported-root-candidate-unique-if-no-pairwise-merkle-bad*:
assumes *no-bad*:
 $\neg \text{composition-fri-supported-root-pairwise-merkle-bad } s \text{ fr } f\text{-fri-roots}$
 f-final as dg composition-fri-roots
shows
 $\exists \text{composition-table.}$
 composition-fri-supported-root-composition-table-candidates s fr
 f-fri-roots f-final as dg composition-fri-roots $\subseteq$
 {*composition-table*}
proof (rule *ccontr*)
assume *not-unique*:
 $\neg (\exists \text{composition-table.}$
 composition-fri-supported-root-composition-table-candidates s fr
 f-fri-roots f-final as dg composition-fri-roots $\subseteq$
 {*composition-table*})
have *composition-fri-supported-root-pairwise-merkle-bad s fr f-fri-roots*
 f-final as dg composition-fri-roots
using
 composition-fri-supported-root-composition-table-candidates-not-singleton-imp-pairwise-merkle-bad
 [OF *not-unique*]
unfolding *composition-fri-supported-root-pairwise-merkle-bad-def*

by blast
 then show False
 using no-bad by contradiction
 qed

lemma *composition-fri-supported-root-candidate-unique-if-no-global-pairwise-merkle-bad:*
 assumes no-bad: $\neg$ composition-fri-supported-pairwise-merkle-bad s
 shows
 $\exists$ composition-table.
 composition-fri-supported-root-composition-table-candidates s fr
 f-fri-roots f-final as dg composition-fri-roots $\subseteq$
 {composition-table}

proof (rule
 composition-fri-supported-root-candidate-unique-if-no-pairwise-merkle-bad)
 show
 $\neg$ composition-fri-supported-root-pairwise-merkle-bad s fr f-fri-roots
 f-final as dg composition-fri-roots
 using no-bad
 unfolding composition-fri-supported-pairwise-merkle-bad-def by blast
 qed

definition *composition-fri-supported-root-union-bad-sets*

$:: (f, 'a)$ protocol-channel-scheme $\Rightarrow$
 $(f \Rightarrow 'f \text{ list} \Rightarrow 'f \text{ list set}) \Rightarrow$
 $'f \Rightarrow 'f \text{ list} \Rightarrow 'f \Rightarrow 'f \text{ list} \Rightarrow 'f \Rightarrow$
 $'f \text{ list} \Rightarrow 'f \text{ list set}$

where

composition-fri-supported-root-union-bad-sets s bad-sets fr f-fri-roots
 f-final as dg composition-fri-roots =
 $\{ \text{comp-bs} \in \text{fri-challenge-space } (\text{ceil-log } (\text{to-nat dg} + 1)) .$
 $\exists$ composition-table $\in$
 composition-fri-supported-root-composition-table-candidates s fr
 f-fri-roots f-final as dg composition-fri-roots.
 $\text{comp-bs} \in \text{bad-sets dg composition-table} \}$

lemma *composition-fri-supported-root-union-bad-sets-subset:*
 composition-fri-supported-root-union-bad-sets s bad-sets fr f-fri-roots
 f-final as dg composition-fri-roots $\subseteq$
 fri-challenge-space (ceil-log (to-nat dg + 1))
 unfolding composition-fri-supported-root-union-bad-sets-def by auto

lemma *composition-fri-challenge-set-hit-imp-supported-root-union-list-set-hit:*
 assumes outcome: out $\in$ set-dist (execute verify-monad s)
 and hit: composition-fri-challenge-set-hit s bad-sets out
 shows

composition-fri-root-list-set-hit s
 (composition-fri-supported-root-union-bad-sets s bad-sets) out

proof –
 from hit obtain trace-table composition-table as query-idxs trace-bs dg comp-bs

where *bound*:
accepted-with-bound-tables s out trace-table composition-table as
query-idxs
and *challenges*: *accepted-fri-challenges s out trace-bs dg comp-bs*
and *comp-bs-bad*: *comp-bs ∈ bad-sets dg composition-table*
unfolding *composition-fri-challenge-set-hit-def* **by** *blast*
from *bound* **obtain** *result-b final-state-b fr f-fri-roots f-final dg-b*
composition-fri-roots final rest **where**
header-bound:
verifier-header-transcript s fr f-fri-roots f-final as dg-b
composition-fri-roots final rest
unfolding *accepted-with-bound-tables-def* **by** *blast*
from *challenges[unfolded accepted-fri-challenges-def]*
obtain *result-c final-state-c fr-c f-fl-c f-final-c as-c fl-c final-c*
query-state-c **where** *challenge-body*:
out = Some (result-c, final-state-c) ∧
verifier-header-transcript s fr-c (map snd f-fl-c) f-final-c as-c dg
(map snd fl-c) final-c (PTranscript query-state-c) ∧
PState query-state-c =
verifier-header-state s fr-c (map snd f-fl-c) f-final-c as-c dg
(map snd fl-c) final-c ∧
Some (result-c, final-state-c) ∈
set-dist (execute
(ntimes
(verifier-query-round-program fr-c f-fl-c f-final-c as-c fl-c
final-c)
rounds)
query-state-c) ∧
PQueryCounter query-state-c = PQueryCounter s ∧
trace-bs = map fst f-fl-c ∧
comp-bs = map fst fl-c ∧
(∀ i < length f-fl-c.
fmlookup (HashMap query-state-c)
(TraceFriChallenge (PTraceFriCounter s + i)
(foldl concat (concat (PState s) fr-c)
(take (Suc i) (map snd f-fl-c)))) =
Some (fst (f-fl-c ! i))) ∧
(∀ i < length fl-c.
fmlookup (HashMap query-state-c)
(CompositionFriChallenge (PCompositionFriCounter s + i)
(foldl concat
(concat
(foldl concat
(concat
(foldl concat (concat (PState s) fr-c)
(map snd f-fl-c))
f-final-c)
as-c)
dg)

```

      (take (Suc i) (map snd fl-c)))) =
      Some (fst (fl-c ! i)))
  by (elim exE)
have out-challenges: out = Some (result-c, final-state-c)
  using challenge-body by simp
have header-challenges:
  verifier-header-transcript s fr-c (map snd f-fl-c) f-final-c as-c dg
    (map snd fl-c) final-c (PTranscript query-state-c)
  using challenge-body by simp
have dg-eq: dg-b = dg
  using verifier-header-transcript-unique[OF header-challenges header-bound]
  by simp
have header:
  verifier-header-transcript s fr f-fri-roots f-final as dg
    composition-fri-roots final rest
  using header-bound dg-eq by simp
have candidate:
  composition-table ∈
    composition-fri-supported-root-composition-table-candidates s fr
      f-fri-roots f-final as dg composition-fri-roots
  unfolding composition-fri-supported-root-composition-table-candidates-def
  using outcome bound challenges header by blast
have comp-bs-space:
  comp-bs ∈ fri-challenge-space (ceil-log (to-nat dg + 1))
  by (rule accepted-fri-challenges-composition-space[OF challenges])
have comp-bs-union:
  comp-bs ∈
    composition-fri-supported-root-union-bad-sets s bad-sets fr
      f-fri-roots f-final as dg composition-fri-roots
  unfolding composition-fri-supported-root-union-bad-sets-def
  using comp-bs-space candidate comp-bs-bad by blast
show ?thesis
  unfolding composition-fri-root-list-set-hit-def
  using challenges header comp-bs-union by blast
qed

lemma wp-composition-fri-challenge-set-hit-bound-via-supported-root-union:
  fixes C :: prob
  assumes future: composition-fri-future-fresh s
  and union-bound:
     $\bigwedge_{fr} f-fri-roots\ f-final\ as\ dg\ composition-fri-roots.$ 
      nnreal
      (card
        (composition-fri-supported-root-union-bad-sets s bad-sets fr
          f-fri-roots f-final as dg composition-fri-roots)) /
      nnreal (CARD('f)  $\wedge$  ceil-log (to-nat dg + 1))  $\leq C$ 
  shows
    wp-event verify-monad (composition-fri-challenge-set-hit s bad-sets) s  $\leq C$ 

```

proof –
have *root-bound*:
 wp-event verify-monad
 (*composition-fri-root-list-set-hit s*
 (*composition-fri-supported-root-union-bad-sets s bad-sets*)) $s \leq C$
by (*rule wp-verify-monad-composition-fri-root-list-set-bound*[*OF future*])
 (*use composition-fri-supported-root-union-bad-sets-subset union-bound in*
 simp-all)
have *wp-event verify-monad* (*composition-fri-challenge-set-hit s bad-sets*) $s \leq$
 wp-event verify-monad
 (*composition-fri-root-list-set-hit s*
 (*composition-fri-supported-root-union-bad-sets s bad-sets*)) s
by (*rule wp-event-mono-on-support*)
 (*rule composition-fri-challenge-set-hit-imp-supported-root-union-list-set-hit*)
also have $\dots \leq C$
 by (*rule root-bound*)
finally show *?thesis* .
qed

lemma *composition-fri-supported-root-union-bad-sets-fraction-bound-if-unique-candidate*:

fixes $C :: \text{prob}$

assumes *unique*:

$\exists$ *composition-table*.

composition-fri-supported-root-composition-table-candidates s fr

f-fri-roots f-final as dg composition-fri-roots $\subseteq$

 {*composition-table*}

and *subset*:

$\bigwedge$ *composition-table. bad-sets dg composition-table* $\subseteq$

fri-challenge-space (ceil-log (to-nat dg + 1))

and *bound*:

$\bigwedge$ *composition-table*.

$\text{nnreal (card (bad-sets dg composition-table))} /$

$\text{nnreal (CARD('f) } \wedge \text{ ceil-log (to-nat dg + 1))} \leq C$

shows

nnreal

 (*card*

 (*composition-fri-supported-root-union-bad-sets s bad-sets fr*

f-fri-roots f-final as dg composition-fri-roots)) /

$\text{nnreal (CARD('f) } \wedge \text{ ceil-log (to-nat dg + 1))} \leq C$

proof –

from *unique* **obtain** *composition-table* **where** *candidates*:

composition-fri-supported-root-composition-table-candidates s fr

f-fri-roots f-final as dg composition-fri-roots $\subseteq$

 {*composition-table*}

by *blast*

have *union-subset*:

composition-fri-supported-root-union-bad-sets s bad-sets fr f-fri-roots

f-final as dg composition-fri-roots $\subseteq$

bad-sets dg composition-table

unfolding *composition-fri-supported-root-union-bad-sets-def*
using *candidates* **by** *auto*
have *finite-bad*: *finite* (*bad-sets dg composition-table*)
by (*rule finite-subset*[*OF subset finite-fri-challenge-space*])
have *card-le*:
card
(*composition-fri-supported-root-union-bad-sets s bad-sets fr*
f-fri-roots f-final as dg composition-fri-roots) $\leq$
card (*bad-sets dg composition-table*)
by (*rule card-mono*[*OF finite-bad union-subset*])
have *nnreal*
(*card*
(*composition-fri-supported-root-union-bad-sets s bad-sets fr*
f-fri-roots f-final as dg composition-fri-roots)) /
nnreal (*CARD*('f') $\wedge$ *ceil-log* (*to-nat dg + 1*)) $\leq$
nnreal (*card* (*bad-sets dg composition-table*)) /
nnreal (*CARD*('f') $\wedge$ *ceil-log* (*to-nat dg + 1*))
by (*rule nnreal-nat-divide-right-mono*[*OF card-le*])
also have ... $\leq C$
by (*rule bound*)
finally show ?thesis .
qed

definition *composition-fri-supported-root-partial-union-bad-sets*

$:: (f, 'a) \text{ protocol-channel-scheme} \Rightarrow$
 $(f \Rightarrow 'f \text{ list} \Rightarrow 'f \text{ list set}) \Rightarrow$
 $'f \Rightarrow 'f \text{ list} \Rightarrow 'f \Rightarrow 'f \text{ list} \Rightarrow 'f \Rightarrow$
 $'f \text{ list} \Rightarrow 'f \text{ list set}$

where

composition-fri-supported-root-partial-union-bad-sets s bad-sets fr
f-fri-roots f-final as dg composition-fri-roots =
 $\{ \text{comp-bs} \in \text{fri-challenge-space } (\text{ceil-log } (\text{to-nat dg} + 1)) .$
 $\exists \text{ composition-table} \in$
 $\text{composition-fri-supported-root-partial-composition-table-candidates}$
 $s \text{ fr } f\text{-fri-roots } f\text{-final as dg composition-fri-roots} .$
 $\text{comp-bs} \in \text{bad-sets dg composition-table} \}$

lemma *composition-fri-supported-root-partial-union-bad-sets-subset*:

composition-fri-supported-root-partial-union-bad-sets s bad-sets fr
f-fri-roots f-final as dg composition-fri-roots $\subseteq$
 $\text{fri-challenge-space } (\text{ceil-log } (\text{to-nat dg} + 1))$

unfolding *composition-fri-supported-root-partial-union-bad-sets-def* **by** *auto*

lemma *composition-fri-supported-root-partial-union-bad-sets-fraction-bound-if-unique-candidate*:

fixes *C* :: *prob*

assumes *unique*:

$\exists \text{ composition-table} .$

composition-fri-supported-root-partial-composition-table-candidates s
fr f-fri-roots f-final as dg composition-fri-roots $\subseteq$

```

      {composition-table}
and subset:
   $\bigwedge \text{composition-table. bad-sets dg composition-table} \subseteq$ 
    fri-challenge-space (ceil-log (to-nat dg + 1))
and bound:
   $\bigwedge \text{composition-table.}$ 
    nnreal (card (bad-sets dg composition-table)) /
    nnreal (CARD('f)  $\wedge$  ceil-log (to-nat dg + 1))  $\leq C$ 
shows
  nnreal
    (card
      (composition-fri-supported-root-partial-union-bad-sets s bad-sets fr
        f-fri-roots f-final as dg composition-fri-roots)) /
    nnreal (CARD('f)  $\wedge$  ceil-log (to-nat dg + 1))  $\leq C$ 
proof –
  from unique obtain composition-table where candidates:
    composition-fri-supported-root-partial-composition-table-candidates s fr
    f-fri-roots f-final as dg composition-fri-roots  $\subseteq$ 
      {composition-table}
  by blast
have union-subset:
  composition-fri-supported-root-partial-union-bad-sets s bad-sets fr
  f-fri-roots f-final as dg composition-fri-roots  $\subseteq$ 
    bad-sets dg composition-table
  unfolding composition-fri-supported-root-partial-union-bad-sets-def
  using candidates by auto
have finite-bad: finite (bad-sets dg composition-table)
  by (rule finite-subset[OF subset finite-fri-challenge-space])
have card-le:
  card
    (composition-fri-supported-root-partial-union-bad-sets s bad-sets fr
      f-fri-roots f-final as dg composition-fri-roots)  $\leq$ 
    card (bad-sets dg composition-table)
  by (rule card-mono[OF finite-bad union-subset])
have nnreal
  (card
    (composition-fri-supported-root-partial-union-bad-sets s bad-sets fr
      f-fri-roots f-final as dg composition-fri-roots)) /
  nnreal (CARD('f)  $\wedge$  ceil-log (to-nat dg + 1))  $\leq$ 
  nnreal (card (bad-sets dg composition-table)) /
  nnreal (CARD('f)  $\wedge$  ceil-log (to-nat dg + 1))
  by (rule nnreal-nat-divide-right-mono[OF card-le])
also have ...  $\leq C$ 
  by (rule bound)
finally show ?thesis .
qed

```

lemma composition-fri-challenge-set-hit-imp-partial-union-or-collision:
assumes outcome: out $\in$ set-dist (execute verify-monad s)

and *hit*: *composition-fri-challenge-set-hit s bad-sets out*
shows
composition-fri-root-list-set-hit s
(composition-fri-supported-root-partial-union-bad-sets s bad-sets) out $\vee$
hash-map-output-collision-bad s out
proof –
from *hit* **obtain** *trace-table composition-table as query-idxs trace-bs dg comp-bs*
where *bound*:
accepted-with-bound-tables s out trace-table composition-table as
query-idxs
and *challenges*: *accepted-fri-challenges s out trace-bs dg comp-bs*
and *trace-low*: *trace-table-low-degree trace-table*
and *not-low*:
 $\neg$ *composition-table-low-degree maxDegree composition-table*
and *comp-bs-bad*: *comp-bs* $\in$ *bad-sets dg composition-table*
unfolding *composition-fri-challenge-set-hit-def* **by** *blast*
from *accepted-with-bound-tables-imp-accepted[OF bound]*
obtain *result final-state* **where** *out-eq*: *out = Some (result, final-state)*
unfolding *accepted-def* **by** *(cases out) auto*
show *?thesis*
proof *(cases hash-map-output-collision final-state)*
case *True*
then have *hash-map-output-collision-bad s out*
unfolding *hash-map-output-collision-bad-def accepted-def out-eq* **by** *simp*
then show *?thesis* **by** *simp*
next
case *clean: False*
from *challenges* **obtain** *result-c final-state-c fr-c f-fl-c f-final-c as-c*
fl-c final-c query-state-c **where**
out-c: *out = Some (result-c, final-state-c)*
and
header-challenges:
verifier-header-transcript s fr-c (map snd f-fl-c) f-final-c as-c dg
(map snd fl-c) final-c (PTranscript query-state-c)
and *query-state-c*:
PState query-state-c =
verifier-header-state s fr-c (map snd f-fl-c) f-final-c as-c dg
(map snd fl-c) final-c
and *query-out-c*:
Some (result-c, final-state-c) $\in$
set-dist
(execute
(ntimes
(verifier-query-round-program fr-c f-fl-c f-final-c as-c fl-c
final-c) rounds)
query-state-c)
and *query-count-c*:
PQueryCounter query-state-c = PQueryCounter s
and *trace-eq-c*: *trace-bs = map fst f-fl-c*


```

and comp-eq-c: comp-bs = map fst fl-c
and trace-lookup-c:
   $\bigwedge i. i < \text{length } f\text{-fl-c} \implies$ 
    fmlookup (HashMap query-state-c)
      (TraceFriChallenge (PTraceFriCounter s + i)
        (foldl concat (concat (PState s) fr-c)
          (take (Suc i) (map snd f-fl-c)))) =
      Some (fst (f-fl-c ! i))
and comp-lookup-c:
   $\bigwedge i. i < \text{length } fl\text{-c} \implies$ 
    fmlookup (HashMap query-state-c)
      (CompositionFriChallenge (PCompositionFriCounter s + i)
        (foldl concat
          (concat
            (foldl concat
              (concat
                (foldl concat (concat (PState s) fr-c)
                  (map snd f-fl-c))
                f-final-c)
              as-c)
            dg)
          (take (Suc i) (map snd fl-c)))) =
      Some (fst (fl-c ! i))
unfolding accepted-fri-challenges-def by blast
from bound obtain fr-b f-fri-roots-b f-final-b dg-b
  composition-fri-roots-b final-b rest-b where
    header-bound:
      verifier-header-transcript s fr-b f-fri-roots-b f-final-b as dg-b
      composition-fri-roots-b final-b rest-b
and comp-nonempty:
  composition-fri-roots-b  $\neq []$ 
and comp-bind:
  merkle-root-binds-table (hd composition-fri-roots-b) composition-table
    final-state
unfolding accepted-with-bound-tables-def out-eq by blast
have dg-eq: dg-b = dg
using verifier-header-transcript-unique
  [OF header-bound header-challenges]
by simp
have header:
  verifier-header-transcript s fr-b f-fri-roots-b f-final-b as dg
  composition-fri-roots-b final-b rest-b
using header-bound dg-eq by simp
have outcome-some:
  Some (result, final-state)  $\in \text{set-dist } (\text{execute verify-monad } s)$ 
using outcome out-eq by simp
have bound-some:
  accepted-with-bound-tables s (Some (result, final-state)) trace-table
    composition-table as query-idxs

```

```

    using bound out-eq by simp
  have challenges-some:
    accepted-fri-challenges s (Some (result, final-state)) trace-bs dg
    comp-bs
  using challenges out-eq by simp
  have state:
    composition-fri-supported-root-composition-table-candidate-state s
    fr-b f-fri-roots-b f-final-b as dg composition-fri-roots-b
    composition-table final-state
  unfolding composition-fri-supported-root-composition-table-candidate-state-def
  using outcome-some bound-some challenges-some header comp-nonempty
    comp-bind
  by blast
  have candidate:
    composition-table ∈
    composition-fri-supported-root-partial-composition-table-candidates s
    fr-b f-fri-roots-b f-final-b as dg composition-fri-roots-b
  using
    composition-fri-supported-root-composition-table-candidate-state-imp-partial-candidate-state
    [OF state clean]
    composition-fri-supported-root-partial-composition-table-candidates-iff-state
  by blast
  have comp-bs-space:
    comp-bs ∈ fri-challenge-space (ceil-log (to-nat dg + 1))
  by (rule accepted-fri-challenges-composition-space[OF challenges])
  have comp-bs-union:
    comp-bs ∈
    composition-fri-supported-root-partial-union-bad-sets s bad-sets
    fr-b f-fri-roots-b f-final-b as dg composition-fri-roots-b
  unfolding composition-fri-supported-root-partial-union-bad-sets-def
  using comp-bs-space candidate comp-bs-bad by blast
  show ?thesis
  unfolding composition-fri-root-list-set-hit-def
  using challenges header comp-bs-union by blast
qed
qed

lemma wp-composition-fri-challenge-set-hit-bound-via-partial-union-or-collision:
  fixes C H :: prob
  assumes partial-bound:
    wp-event verify-monad
    (composition-fri-root-list-set-hit s
    (composition-fri-supported-root-partial-union-bad-sets s bad-sets))
    s ≤ C
  and collision-bound:
    wp-event verify-monad (hash-map-output-collision-bad s) s ≤ H
  shows
    wp-event verify-monad (composition-fri-challenge-set-hit s bad-sets) s ≤
    C + H

```

```

proof –
  have mono:
    wp-event verify-monad (composition-fri-challenge-set-hit s bad-sets) s ≤
      wp-event verify-monad
        (λout.
          composition-fri-root-list-set-hit s
            (composition-fri-supported-root-partial-union-bad-sets s bad-sets)
              out ∨
              hash-map-output-collision-bad s out) s
    by (rule wp-event-mono-on-support)
      (use composition-fri-challenge-set-hit-imp-partial-union-or-collision
        in blast)
  also have ... ≤ C + H
proof –
  have wp-event verify-monad
    (λout.
      composition-fri-root-list-set-hit s
        (composition-fri-supported-root-partial-union-bad-sets s bad-sets)
          out ∨
          hash-map-output-collision-bad s out) s ≤
    wp-event verify-monad
      (composition-fri-root-list-set-hit s
        (composition-fri-supported-root-partial-union-bad-sets s bad-sets))
      s +
      wp-event verify-monad (hash-map-output-collision-bad s) s
    by (rule wp-event-union-bound)
  also have ... ≤ C + H
    by (intro add-mono partial-bound collision-bound)
  finally show ?thesis .
qed
finally show ?thesis .
qed

lemma wp-composition-fri-challenge-set-hit-bound-via-supported-partial-union-or-collision:
  fixes C H :: prob
  assumes future: composition-fri-future-fresh s
  and union-bound:
    ∧fr f-fri-roots f-final as dg composition-fri-roots.
      nnreal
        (card
          (composition-fri-supported-root-partial-union-bad-sets s bad-sets
            fr f-fri-roots f-final as dg composition-fri-roots)) /
      nnreal (CARD('f) ^ ceil-log (to-nat dg + 1)) ≤ C
  and collision-bound:
    wp-event verify-monad (hash-map-output-collision-bad s) s ≤ H
  shows
    wp-event verify-monad (composition-fri-challenge-set-hit s bad-sets) s ≤
      C + H
proof –

```

```

have partial-bound:
  wp-event verify-monad
    (composition-fri-root-list-set-hit s
      (composition-fri-supported-root-partial-union-bad-sets s bad-sets))
     $s \leq C$ 
  by (rule wp-verify-monad-composition-fri-root-list-set-bound [OF future])
    (use composition-fri-supported-root-partial-union-bad-sets-subset
      union-bound in simp-all)
show ?thesis
  by (rule
    wp-composition-fri-challenge-set-hit-bound-via-partial-union-or-collision
      [OF partial-bound collision-bound])
qed

lemma wp-composition-fri-challenge-set-hit-bound-if-supported-partial-root-candidate-unique-or-collision:
  fixes  $H :: \text{prob}$ 
  assumes future: composition-fri-future-fresh s
  and unique:
     $\bigwedge \text{fr } f\text{-fri-roots } f\text{-final as } dg \text{ composition-fri-roots.}$ 
     $\exists \text{ composition-table.}$ 
    composition-fri-supported-root-partial-composition-table-candidates
       $s \text{ fr } f\text{-fri-roots } f\text{-final as } dg \text{ composition-fri-roots} \subseteq$ 
       $\{\text{composition-table}\}$ 
  and bounded: composition-fri-bad-challenge-sets-bounded bad-sets
  and collision-bound:
    wp-event verify-monad (hash-map-output-collision-bad s)  $s \leq H$ 
  shows
    wp-event verify-monad (composition-fri-challenge-set-hit s bad-sets)  $s \leq$ 
    composition-fri-error +  $H$ 
proof –
  have union-bound:
     $\bigwedge \text{fr } f\text{-fri-roots } f\text{-final as } dg \text{ composition-fri-roots.}$ 
    nnreal
    (card
      (composition-fri-supported-root-partial-union-bad-sets s bad-sets
        fr f-fri-roots f-final as dg composition-fri-roots)) /
    nnreal ( $\text{CARD}('f) \wedge \text{ceil-log (to-nat } dg + 1)) \leq$ 
    composition-fri-error
proof –
  fix fr f-fri-roots f-final as dg composition-fri-roots
  show nnreal
    (card
      (composition-fri-supported-root-partial-union-bad-sets s bad-sets
        fr f-fri-roots f-final as dg composition-fri-roots)) /
    nnreal ( $\text{CARD}('f) \wedge \text{ceil-log (to-nat } dg + 1)) \leq$ 
    composition-fri-error
  by (rule
    composition-fri-supported-root-partial-union-bad-sets-fraction-bound-if-unique-candidate
      [OF unique])

```

```

      (use bounded in
        ‹auto simp: composition-fri-bad-challenge-sets-bounded-def›)
qed
show ?thesis
  by (rule
    wp-composition-fri-challenge-set-hit-bound-via-supported-partial-union-or-collision
    [OF future union-bound collision-bound])
qed

definition trace-fri-bad-with-tables
:: ('f, 'a) protocol-channel-scheme  $\Rightarrow$ 
  (unit list  $\times$  ('f, 'a) protocol-channel-scheme) option  $\Rightarrow$  bool
where
  trace-fri-bad-with-tables s out  $\longleftrightarrow$ 
    accepted-with-fri-tables s out  $\wedge$  trace-fri-bad s out

definition composition-fri-bad-with-tables
:: ('f, 'a) protocol-channel-scheme  $\Rightarrow$ 
  (unit list  $\times$  ('f, 'a) protocol-channel-scheme) option  $\Rightarrow$  bool
where
  composition-fri-bad-with-tables s out  $\longleftrightarrow$ 
    accepted-with-fri-tables s out  $\wedge$  composition-fri-bad s out

definition trace-fri-bad-with-partial-openings
:: ('f, 'a) protocol-channel-scheme  $\Rightarrow$ 
  (unit list  $\times$  ('f, 'a) protocol-channel-scheme) option  $\Rightarrow$  bool
where
  trace-fri-bad-with-partial-openings s out  $\longleftrightarrow$ 
    trace-fri-bad s out  $\wedge$ 
    ( $\exists$  fr query-idxs trace-openings.
      accepted-with-partial-trace-openings s out fr query-idxs
        trace-openings)

definition composition-fri-bad-with-partial-openings
:: ('f, 'a) protocol-channel-scheme  $\Rightarrow$ 
  (unit list  $\times$  ('f, 'a) protocol-channel-scheme) option  $\Rightarrow$  bool
where
  composition-fri-bad-with-partial-openings s out  $\longleftrightarrow$ 
    composition-fri-bad s out  $\wedge$ 
    ( $\exists$  fr query-idxs trace-openings.
      accepted-with-partial-trace-openings s out fr query-idxs
        trace-openings)

end

end

theory Soundness-Reductions-Partial

```

```

imports Soundness-Reductions-Side
begin

```

Partial-opening bad-event definitions and accepted-execution partitions.

```

context soundness
begin

```

Reachable-state FRI interfaces for the partial-opening proof path. These predicates express the verifier-local FRI low-degree reductions over authenticated partial openings and deliberately avoid complete-table witnesses in their statement.

```

definition trace-fri-partial-opening-reduction-assumption
  :: ('f, 'a) protocol-channel-scheme  $\Rightarrow$  bool
where
  trace-fri-partial-opening-reduction-assumption  $s \longleftrightarrow$ 
    wp-event verify-monad (trace-fri-bad-with-partial-openings  $s$ )  $s \leq$ 
      trace-fri-error

```

```

definition composition-fri-partial-opening-reduction-assumption
  :: ('f, 'a) protocol-channel-scheme  $\Rightarrow$  bool
where
  composition-fri-partial-opening-reduction-assumption  $s \longleftrightarrow$ 
    wp-event verify-monad
      (composition-fri-bad-with-partial-openings  $s$ )  $s \leq$ 
        composition-fri-error

```

```

lemma trace-fri-bad-with-partial-openings-bound-from-partial-reduction:
assumes trace-fri-partial-opening-reduction-assumption  $s$ 
shows wp-event verify-monad (trace-fri-bad-with-partial-openings  $s$ )  $s \leq$ 
  trace-fri-error
using assms unfolding trace-fri-partial-opening-reduction-assumption-def
by simp

```

```

lemma composition-fri-bad-with-partial-openings-bound-from-partial-reduction:
assumes composition-fri-partial-opening-reduction-assumption  $s$ 
shows
  wp-event verify-monad
    (composition-fri-bad-with-partial-openings  $s$ )  $s \leq$ 
      composition-fri-error
using assms
unfolding composition-fri-partial-opening-reduction-assumption-def by simp

```

```

lemma trace-fri-bad-with-partial-openings-imp-trace-fri-bad:
assumes trace-fri-bad-with-partial-openings  $s$  out
shows trace-fri-bad  $s$  out
using assms unfolding trace-fri-bad-with-partial-openings-def by simp

```

```

lemma composition-fri-bad-with-partial-openings-imp-composition-fri-bad:
assumes composition-fri-bad-with-partial-openings  $s$  out
shows composition-fri-bad  $s$  out

```

using *assms* **unfolding** *composition-fri-bad-with-partial-openings-def* **by** *simp*

lemma *wp-trace-fri-bad-with-partial-openings-le-trace-fri-bad*:
wp-event verify-monad (trace-fri-bad-with-partial-openings s) s ≤
wp-event verify-monad (trace-fri-bad s) s
by (*rule wp-event-mono*)
(*rule trace-fri-bad-with-partial-openings-imp-trace-fri-bad*)

lemma *wp-composition-fri-bad-with-partial-openings-le-composition-fri-bad*:
wp-event verify-monad (composition-fri-bad-with-partial-openings s) s ≤
wp-event verify-monad (composition-fri-bad s) s
by (*rule wp-event-mono*)
(*rule composition-fri-bad-with-partial-openings-imp-composition-fri-bad*)

lemma *trace-fri-bad-with-tables-imp-partial-openings-on-support*:
assumes *support*: *out ∈ set-dist (execute verify-monad s)*
and *bad*: *trace-fri-bad-with-tables s out*
shows *trace-fri-bad-with-partial-openings s out*
proof –
have *tables*: *accepted-with-fri-tables s out*
using *bad* **unfolding** *trace-fri-bad-with-tables-def* **by** *simp*
have *accepted*: *accepted out*
by (*rule accepted-with-fri-tables-imp-accepted[OF tables]*)
then obtain *result final-state* **where** *out-eq*:
out = Some (result, final-state)
unfolding *accepted-def* **by** (*cases out*) *auto*
from *verify-monad-accepted-with-partial-trace-openings*
[OF support[unfolded out-eq]]
obtain *fr query-idxs trace-openings* **where** *partial*:
accepted-with-partial-trace-openings s
(Some (result, final-state)) fr query-idxs trace-openings
by *blast*
have *trace-bad*: *trace-fri-bad s out*
using *bad* **unfolding** *trace-fri-bad-with-tables-def* **by** *simp*
show *?thesis*
unfolding *trace-fri-bad-with-partial-openings-def out-eq*
using *trace-bad[unfolded out-eq]* *partial* **by** *blast*
qed

lemma *composition-fri-bad-with-tables-imp-partial-openings-on-support*:
assumes *support*: *out ∈ set-dist (execute verify-monad s)*
and *bad*: *composition-fri-bad-with-tables s out*
shows *composition-fri-bad-with-partial-openings s out*
proof –
have *tables*: *accepted-with-fri-tables s out*
using *bad* **unfolding** *composition-fri-bad-with-tables-def* **by** *simp*
have *accepted*: *accepted out*
by (*rule accepted-with-fri-tables-imp-accepted[OF tables]*)
then obtain *result final-state* **where** *out-eq*:

```

    out = Some (result, final-state)
    unfolding accepted-def by (cases out) auto
from verify-monad-accepted-with-partial-trace-openings
    [OF support[unfolded out-eq]]
obtain fr query-idxs trace-openings where partial:
    accepted-with-partial-trace-openings s
    (Some (result, final-state)) fr query-idxs trace-openings
by blast
have composition-bad: composition-fri-bad s out
    using bad unfolding composition-fri-bad-with-tables-def by simp
show ?thesis
    unfolding composition-fri-bad-with-partial-openings-def out-eq
    using composition-bad[unfolded out-eq] partial by blast
qed

lemma wp-trace-fri-bad-with-tables-le-partial-openings:
    wp-event verify-monad (trace-fri-bad-with-tables s) s ≤
    wp-event verify-monad (trace-fri-bad-with-partial-openings s) s
by (rule wp-event-mono-on-support)
    (rule trace-fri-bad-with-tables-imp-partial-openings-on-support)

lemma wp-composition-fri-bad-with-tables-le-partial-openings:
    wp-event verify-monad (composition-fri-bad-with-tables s) s ≤
    wp-event verify-monad (composition-fri-bad-with-partial-openings s) s
by (rule wp-event-mono-on-support)
    (rule composition-fri-bad-with-tables-imp-partial-openings-on-support)

definition soundness-bad-event-table-aware
  :: ('f, 'a) protocol-channel-scheme ⇒
    (unit list × ('f, 'a) protocol-channel-scheme) option ⇒ bool
where
    soundness-bad-event-table-aware s out ⟷
    composition-bad s out ∨
    trace-fri-bad-with-tables s out ∨
    composition-fri-bad-with-tables s out ∨
    query-bad s out

definition soundness-bad-event-partial-opening
  :: ('f, 'a) protocol-channel-scheme ⇒
    (unit list × ('f, 'a) protocol-channel-scheme) option ⇒ bool
where
    soundness-bad-event-partial-opening s out ⟷
    composition-bad s out ∨
    trace-fri-bad-with-partial-openings s out ∨
    composition-fri-bad-with-partial-openings s out ∨
    query-bad s out

definition soundness-bad-event-table-aware-with-merkle
  :: ('f, 'a) protocol-channel-scheme ⇒

```


$(\text{unit list} \times ('f, 'a) \text{ protocol-channel-scheme}) \text{ option} \Rightarrow \text{bool}$
where
 $\text{soundness-bad-event-table-aware-with-merkle } s \text{ out} \longleftrightarrow$
 $\text{fri-merkle-binding-bad } s \text{ out} \vee$
 $\text{soundness-bad-event-table-aware } s \text{ out}$

definition *soundness-bad-event-partial-opening-with-merkle*
 $:: ('f, 'a) \text{ protocol-channel-scheme} \Rightarrow$
 $(\text{unit list} \times ('f, 'a) \text{ protocol-channel-scheme}) \text{ option} \Rightarrow \text{bool}$
where
 $\text{soundness-bad-event-partial-opening-with-merkle } s \text{ out} \longleftrightarrow$
 $\text{fri-merkle-binding-bad } s \text{ out} \vee$
 $\text{soundness-bad-event-partial-opening } s \text{ out}$

definition *soundness-bad-event-partial-opening-with-initial-merkle*
 $:: ('f, 'a) \text{ protocol-channel-scheme} \Rightarrow$
 $(\text{unit list} \times ('f, 'a) \text{ protocol-channel-scheme}) \text{ option} \Rightarrow \text{bool}$
where
 $\text{soundness-bad-event-partial-opening-with-initial-merkle } s \text{ out} \longleftrightarrow$
 $\text{initial-merkle-binding-bad } s \text{ out} \vee$
 $\text{soundness-bad-event-partial-opening } s \text{ out}$

definition *soundness-bad-event-partial-opening-with-partial-merkle*
 $:: ('f, 'a) \text{ protocol-channel-scheme} \Rightarrow$
 $(\text{unit list} \times ('f, 'a) \text{ protocol-channel-scheme}) \text{ option} \Rightarrow \text{bool}$
where
 $\text{soundness-bad-event-partial-opening-with-partial-merkle } s \text{ out} \longleftrightarrow$
 $\text{partial-merkle-inconsistency-bad } s \text{ out} \vee$
 $\text{soundness-bad-event-partial-opening } s \text{ out}$

definition *composition-bad-with-partial-candidates*
 $:: ('f, 'a) \text{ protocol-channel-scheme} \Rightarrow$
 $(\text{unit list} \times ('f, 'a) \text{ protocol-channel-scheme}) \text{ option} \Rightarrow \text{bool}$
where
 $\text{composition-bad-with-partial-candidates } s \text{ out} \longleftrightarrow$
 $(\exists \text{ fr } f\text{-fri-roots } f\text{-final as dg composition-fri-roots final}$
 $\text{ trace-query-idxs trace-openings composition-query-idxs}$
 $\text{ composition-openings trace-table composition-table } f.$
 $\text{ accepted-with-partial-initial-openings } s \text{ out fr } f\text{-fri-roots } f\text{-final}$
 $\text{ as dg composition-fri-roots final trace-query-idxs trace-openings}$
 $\text{ composition-query-idxs composition-openings} \wedge$
 $\text{ partial-trace-table-candidate trace-table trace-openings} \wedge$
 $\text{ partial-composition-table-candidate composition-table}$
 $\text{ composition-openings} \wedge$
 $\text{ degree } f < \text{ clength} \wedge$
 $\text{ trace-table} = \text{ map } (\text{poly } f) \text{ eval-domain} \wedge$
 $\text{ violated-constraints } f \neq \{\}$
 $\text{ composition-table-low-degree maxDegree composition-table} \wedge$
 $\text{ all-queries-consistent trace-table composition-table as})$

definition *composition-degree-bad-with-partial-candidates*
 $:: ('f, 'a) \text{ protocol-channel-scheme} \Rightarrow$
 $(\text{unit list} \times ('f, 'a) \text{ protocol-channel-scheme}) \text{ option} \Rightarrow \text{bool}$
where
 $\text{composition-degree-bad-with-partial-candidates } s \text{ out} \longleftrightarrow$
 $(\exists \text{ fr } f\text{-fri-roots } f\text{-final as dg composition-fri-roots final}$
 $\text{ trace-query-idxs trace-openings composition-query-idxs}$
 $\text{ composition-openings trace-table composition-table } f.$
 $\text{ accepted-with-partial-initial-openings } s \text{ out fr } f\text{-fri-roots } f\text{-final}$
 $\text{ as dg composition-fri-roots final trace-query-idxs trace-openings}$
 $\text{ composition-query-idxs composition-openings } \wedge$
 $\text{ partial-trace-table-candidate trace-table trace-openings } \wedge$
 $\text{ partial-composition-table-candidate composition-table}$
 $\text{ composition-openings } \wedge$
 $\text{ degree } f < \text{clength} \wedge$
 $\text{ trace-table} = \text{map } (\text{poly } f) \text{ eval-domain } \wedge$
 $\text{ violated-constraints } f \neq \{\}$
 $\text{ composition-table-low-degree maxDegree composition-table } \wedge$
 $\text{ all-queries-consistent trace-table composition-table as } \wedge$
 $\neg \text{ common-denominator-degree-bounds } f \text{ as})$

definition *composition-randomization-bad-with-partial-candidates*
 $:: ('f, 'a) \text{ protocol-channel-scheme} \Rightarrow$
 $(\text{unit list} \times ('f, 'a) \text{ protocol-channel-scheme}) \text{ option} \Rightarrow \text{bool}$
where
 $\text{composition-randomization-bad-with-partial-candidates } s \text{ out} \longleftrightarrow$
 $(\exists \text{ fr } f\text{-fri-roots } f\text{-final as dg composition-fri-roots final}$
 $\text{ trace-query-idxs trace-openings composition-query-idxs}$
 $\text{ composition-openings trace-table composition-table } f.$
 $\text{ accepted-with-partial-initial-openings } s \text{ out fr } f\text{-fri-roots } f\text{-final}$
 $\text{ as dg composition-fri-roots final trace-query-idxs trace-openings}$
 $\text{ composition-query-idxs composition-openings } \wedge$
 $\text{ partial-trace-table-candidate trace-table trace-openings } \wedge$
 $\text{ partial-composition-table-candidate composition-table}$
 $\text{ composition-openings } \wedge$
 $\text{ degree } f < \text{clength} \wedge$
 $\text{ trace-table} = \text{map } (\text{poly } f) \text{ eval-domain } \wedge$
 $\text{ violated-constraints } f \neq \{\}$
 $\text{ composition-table-low-degree maxDegree composition-table } \wedge$
 $\text{ all-queries-consistent trace-table composition-table as } \wedge$
 $\text{ common-denominator-degree-bounds } f \text{ as } \wedge$
 $\text{ random-combination-common-denominator-hides-violations } f \text{ as})$

definition *trace-fri-bad-with-partial-candidates*
 $:: ('f, 'a) \text{ protocol-channel-scheme} \Rightarrow$
 $(\text{unit list} \times ('f, 'a) \text{ protocol-channel-scheme}) \text{ option} \Rightarrow \text{bool}$
where
 $\text{trace-fri-bad-with-partial-candidates } s \text{ out} \longleftrightarrow$

$(\exists \text{ fr f-fri-roots f-final as dg composition-fri-roots final}$
 $\text{ trace-query-idxs trace-openings composition-query-idxs}$
 $\text{ composition-openings trace-table.}$
 $\text{ accepted-with-partial-initial-openings s out fr f-fri-roots f-final}$
 $\text{ as dg composition-fri-roots final trace-query-idxs trace-openings}$
 $\text{ composition-query-idxs composition-openings } \wedge$
 $\text{ partial-trace-table-candidate trace-table trace-openings } \wedge$
 $\neg \text{ trace-table-low-degree trace-table})$

definition *composition-fri-bad-with-partial-candidates*

$:: (\text{'f}, \text{'a}) \text{ protocol-channel-scheme} \Rightarrow$
 $(\text{unit list} \times (\text{'f}, \text{'a}) \text{ protocol-channel-scheme}) \text{ option} \Rightarrow \text{bool}$

where

$\text{composition-fri-bad-with-partial-candidates s out} \longleftrightarrow$
 $(\exists \text{ fr f-fri-roots f-final as dg composition-fri-roots final}$
 $\text{ trace-query-idxs trace-openings composition-query-idxs}$
 $\text{ composition-openings trace-table composition-table.}$
 $\text{ accepted-with-partial-initial-openings s out fr f-fri-roots f-final}$
 $\text{ as dg composition-fri-roots final trace-query-idxs trace-openings}$
 $\text{ composition-query-idxs composition-openings } \wedge$
 $\text{ partial-trace-table-candidate trace-table trace-openings } \wedge$
 $\text{ partial-composition-table-candidate composition-table}$
 $\text{ composition-openings } \wedge$
 $\text{ trace-table-low-degree trace-table } \wedge$
 $\neg \text{ composition-table-low-degree maxDegree composition-table})$

definition *query-bad-with-partial-candidates*

$:: (\text{'f}, \text{'a}) \text{ protocol-channel-scheme} \Rightarrow$
 $(\text{unit list} \times (\text{'f}, \text{'a}) \text{ protocol-channel-scheme}) \text{ option} \Rightarrow \text{bool}$

where

$\text{query-bad-with-partial-candidates s out} \longleftrightarrow$
 $(\exists \text{ fr f-fri-roots f-final as dg composition-fri-roots final}$
 $\text{ trace-query-idxs trace-openings composition-query-idxs}$
 $\text{ composition-openings trace-table composition-table.}$
 $\text{ accepted-with-partial-initial-openings s out fr f-fri-roots f-final}$
 $\text{ as dg composition-fri-roots final trace-query-idxs trace-openings}$
 $\text{ composition-query-idxs composition-openings } \wedge$
 $\text{ partial-trace-table-candidate trace-table trace-openings } \wedge$
 $\text{ partial-composition-table-candidate composition-table}$
 $\text{ composition-openings } \wedge$
 $\text{ trace-table-low-degree trace-table } \wedge$
 $\text{ composition-table-low-degree maxDegree composition-table } \wedge$
 $\neg \text{ all-queries-consistent trace-table composition-table as})$

definition *soundness-bad-event-partial-candidate*

$:: (\text{'f}, \text{'a}) \text{ protocol-channel-scheme} \Rightarrow$
 $(\text{unit list} \times (\text{'f}, \text{'a}) \text{ protocol-channel-scheme}) \text{ option} \Rightarrow \text{bool}$

where

$\text{soundness-bad-event-partial-candidate s out} \longleftrightarrow$

composition-bad-with-partial-candidates s out $\vee$
trace-fri-bad-with-partial-candidates s out $\vee$
composition-fri-bad-with-partial-candidates s out $\vee$
query-bad-with-partial-candidates s out

definition *soundness-bad-event-partial-candidate-with-partial-merkle*

$:: ('f, 'a) \text{ protocol-channel-scheme} \Rightarrow$
 $(\text{unit list} \times ('f, 'a) \text{ protocol-channel-scheme}) \text{ option} \Rightarrow \text{bool}$

where

soundness-bad-event-partial-candidate-with-partial-merkle s out $\longleftrightarrow$
partial-merkle-inconsistency-bad s out $\vee$
soundness-bad-event-partial-candidate s out

definition *accepted-with-empty-composition-header-candidates*

$:: ('f, 'a) \text{ protocol-channel-scheme} \Rightarrow$
 $(\text{unit list} \times ('f, 'a) \text{ protocol-channel-scheme}) \text{ option} \Rightarrow$
 $'f \Rightarrow 'f \text{ list} \Rightarrow 'f \Rightarrow 'f \text{ list} \Rightarrow 'f \Rightarrow$
 $'f \Rightarrow \text{nat list} \Rightarrow 'f \text{ authenticated-opening list list} \Rightarrow$
 $'f \text{ list} \Rightarrow 'f \text{ list} \Rightarrow \text{bool}$

where

accepted-with-empty-composition-header-candidates s out fr f-fri-roots
f-final as dg final trace-query-idxs trace-openings trace-table
composition-table $\longleftrightarrow$
accepted out $\wedge$
 $(\exists \text{ rest.}$
 $\text{verifier-header-transcript s fr f-fri-roots f-final as dg []}$
 $\text{final rest}) \wedge$
accepted-with-partial-trace-openings s out fr trace-query-idxs
trace-openings $\wedge$
partial-trace-table-candidate trace-table trace-openings $\wedge$
*composition-table = replicate (scale * clength) final*

definition *composition-bad-with-empty-composition-header-candidates*

$:: ('f, 'a) \text{ protocol-channel-scheme} \Rightarrow$
 $(\text{unit list} \times ('f, 'a) \text{ protocol-channel-scheme}) \text{ option} \Rightarrow \text{bool}$

where

composition-bad-with-empty-composition-header-candidates s out $\longleftrightarrow$
 $(\exists \text{ fr f-fri-roots f-final as dg final trace-query-idxs}$
 $\text{trace-openings trace-table composition-table f.}$
accepted-with-empty-composition-header-candidates s out fr
f-fri-roots f-final as dg final trace-query-idxs trace-openings
trace-table composition-table $\wedge$
 $\text{degree f} < \text{clength} \wedge$
 $\text{trace-table} = \text{map (poly f) eval-domain} \wedge$
 $\text{violated-constraints f} \neq \{\}$ $\wedge$
composition-table-low-degree maxDegree composition-table $\wedge$
all-queries-consistent trace-table composition-table as)

definition *composition-degree-bad-with-empty-composition-header-candidates*

$:: ('f, 'a) \text{ protocol-channel-scheme} \Rightarrow$
 $(\text{unit list} \times ('f, 'a) \text{ protocol-channel-scheme}) \text{ option} \Rightarrow \text{bool}$

where

$\text{composition-degree-bad-with-empty-composition-header-candidates } s \text{ out} \longleftrightarrow$
 $(\exists \text{ fr f-fri-roots f-final as dg final trace-query-idxs}$
 $\text{ trace-openings trace-table composition-table } f.$
 $\text{accepted-with-empty-composition-header-candidates } s \text{ out fr}$
 $\text{ f-fri-roots f-final as dg final trace-query-idxs trace-openings}$
 $\text{ trace-table composition-table} \wedge$
 $\text{degree } f < \text{clength} \wedge$
 $\text{trace-table} = \text{map } (\text{poly } f) \text{ eval-domain} \wedge$
 $\text{violated-constraints } f \neq \{\}$
 $\text{composition-table-low-degree maxDegree composition-table} \wedge$
 $\text{all-queries-consistent trace-table composition-table as} \wedge$
 $\neg \text{common-denominator-degree-bounds } f \text{ as})$

definition *composition-randomization-bad-with-empty-composition-header-candidates*

$:: ('f, 'a) \text{ protocol-channel-scheme} \Rightarrow$
 $(\text{unit list} \times ('f, 'a) \text{ protocol-channel-scheme}) \text{ option} \Rightarrow \text{bool}$

where

$\text{composition-randomization-bad-with-empty-composition-header-candidates } s \text{ out} \longleftrightarrow$
 $(\exists \text{ fr f-fri-roots f-final as dg final trace-query-idxs}$
 $\text{ trace-openings trace-table composition-table } f.$
 $\text{accepted-with-empty-composition-header-candidates } s \text{ out fr}$
 $\text{ f-fri-roots f-final as dg final trace-query-idxs trace-openings}$
 $\text{ trace-table composition-table} \wedge$
 $\text{degree } f < \text{clength} \wedge$
 $\text{trace-table} = \text{map } (\text{poly } f) \text{ eval-domain} \wedge$
 $\text{violated-constraints } f \neq \{\}$
 $\text{composition-table-low-degree maxDegree composition-table} \wedge$
 $\text{all-queries-consistent trace-table composition-table as} \wedge$
 $\text{common-denominator-degree-bounds } f \text{ as} \wedge$
 $\text{random-combination-common-denominator-hides-violations } f \text{ as})$

definition *composition-alpha-bad-set-hit-with-empty-composition-header-candidates*

$:: ('f, 'a) \text{ protocol-channel-scheme} \Rightarrow$
 $(\text{unit list} \times ('f, 'a) \text{ protocol-channel-scheme}) \text{ option} \Rightarrow \text{bool}$

where

$\text{composition-alpha-bad-set-hit-with-empty-composition-header-candidates } s \text{ out} \longleftrightarrow$
 $(\exists \text{ fr f-fri-roots f-final as dg final trace-query-idxs}$
 $\text{ trace-openings trace-table composition-table.}$
 $\text{accepted-with-empty-composition-header-candidates } s \text{ out fr}$
 $\text{ f-fri-roots f-final as dg final trace-query-idxs trace-openings}$
 $\text{ trace-table composition-table} \wedge$
 $\text{as} \in \text{composition-trace-bad-alpha-space trace-table})$

definition *composition-alpha-bad-set-hit-with-partial-candidates*

$:: ('f, 'a) \text{ protocol-channel-scheme} \Rightarrow$
 $(\text{unit list} \times ('f, 'a) \text{ protocol-channel-scheme}) \text{ option} \Rightarrow \text{bool}$

where

composition-alpha-bad-set-hit-with-partial-candidates s out $\longleftrightarrow$
 $(\exists$ fr f -fri-roots f -final as dg *composition-fri-roots* final
trace-query-idxs *trace-openings* *composition-query-idxs*
composition-openings *trace-table*.
accepted-with-partial-initial-openings s out fr f -fri-roots f -final
as dg *composition-fri-roots* final *trace-query-idxs* *trace-openings*
composition-query-idxs *composition-openings* $\wedge$
partial-trace-table-candidate *trace-table* *trace-openings* $\wedge$
as $\in$ *composition-trace-bad-alpha-space* *trace-table*)

definition *partial-trace-opening-alpha-union-bad-sets*

$:: 'f$ *authenticated-opening* list list $\Rightarrow 'f$ list set

where

partial-trace-opening-alpha-union-bad-sets *trace-openings* =
 $(\bigcup$ *trace-table* $\in$ *partial-trace-table-candidates* *trace-openings*.
composition-trace-bad-alpha-space *trace-table*)

definition *composition-alpha-partial-opening-union-hit-with-empty-header*

$:: ('f, 'a)$ *protocol-channel-scheme* $\Rightarrow$
 $(\text{unit list} \times ('f, 'a)$ *protocol-channel-scheme*) *option* $\Rightarrow$ bool

where

composition-alpha-partial-opening-union-hit-with-empty-header s out $\longleftrightarrow$
 $(\exists$ fr f -fri-roots f -final as dg final *trace-query-idxs*
trace-openings *trace-table* *composition-table*.
accepted-with-empty-composition-header-candidates s out fr
 f -fri-roots f -final as dg final *trace-query-idxs* *trace-openings*
trace-table *composition-table* $\wedge$
as $\in$ *partial-trace-opening-alpha-union-bad-sets* *trace-openings*)

lemma *partial-trace-opening-alpha-union-bad-sets-subset-alpha-space:*

partial-trace-opening-alpha-union-bad-sets *trace-openings* $\subseteq$
alpha-space

unfolding *partial-trace-opening-alpha-union-bad-sets-def*

using *composition-trace-bad-alpha-space-subset-alpha-space* **by** *auto*

lemma *partial-trace-opening-alpha-union-bad-sets-finite:*

finite (*partial-trace-opening-alpha-union-bad-sets* *trace-openings*)

by (*rule* *finite-subset*

[*OF* *partial-trace-opening-alpha-union-bad-sets-subset-alpha-space*
finite-alpha-space])

lemma *partial-trace-opening-alpha-union-bad-sets-fraction-bound-if-unique-candidate:*

assumes *unique:*

$\exists$ *trace-table*.

partial-trace-table-candidates *trace-openings* $\subseteq \{ \text{trace-table} \}$

shows

nnreal

$(\text{card } (\text{partial-trace-opening-alpha-union-bad-sets } \text{trace-openings})) /$

$nnreal \ (card \ alpha-space) \leq \ composition-error-bound$
proof –
from *unique* **obtain** *trace-table* **where** *candidates:*
 $\partial partial-trace-table-candidates \ trace-openings \subseteq \{trace-table\}$
by *blast*
have *union-subset:*
 $\partial partial-trace-opening-alpha-union-bad-sets \ trace-openings \subseteq$
 $\partial \ composition-trace-bad-alpha-space \ trace-table$
unfolding *partial-trace-opening-alpha-union-bad-sets-def*
using *candidates* **by** *auto*
have *finite-bad:* $\partial finite \ (composition-trace-bad-alpha-space \ trace-table)$
by $(rule \ finite-subset$
 $\partial [OF \ composition-trace-bad-alpha-space-subset-alpha-space$
 $\partial \ finite-alpha-space])$
have $\partial card \ (partial-trace-opening-alpha-union-bad-sets \ trace-openings) \leq$
 $\partial card \ (composition-trace-bad-alpha-space \ trace-table)$
by $(rule \ card-mono[OF \ finite-bad \ union-subset])$
then have
 $\partial nnreal$
 $\partial (card \ (partial-trace-opening-alpha-union-bad-sets \ trace-openings)) /$
 $\partial nnreal \ (card \ alpha-space) \leq$
 $\partial nnreal \ (card \ (composition-trace-bad-alpha-space \ trace-table)) /$
 $\partial nnreal \ (card \ alpha-space)$
by $(rule \ nnreal-nat-divide-right-mono)$
also have $\partial \dots \leq \ composition-error-bound$
by $(rule \ composition-trace-bad-alpha-space-fraction-bound-alpha-space)$
finally show *?thesis* .
qed

lemma *composition-alpha-bad-set-hit-with-empty-header-imp-partial-opening-union-hit:*

assumes

$\partial \partial composition-alpha-bad-set-hit-with-empty-composition-header-candidates$
 $\partial s \ out$

shows

$\partial \partial composition-alpha-partial-opening-union-hit-with-empty-header \ s \ out$

proof –

obtain *fr f-fri-roots f-final as dg final trace-query-idxs*

$\partial \partial \partial trace-openings \ trace-table \ composition-table$ **where** *partial:*
 $\partial \partial \partial accepted-with-empty-composition-header-candidates \ s \ out \ fr$
 $\partial \partial \partial f-fri-roots \ f-final \ as \ dg \ final \ trace-query-idxs \ trace-openings$
 $\partial \partial \partial trace-table \ composition-table$

and *as-bad:* $\partial as \in \ composition-trace-bad-alpha-space \ trace-table$

using *assms*

unfolding

$\partial \partial \partial composition-alpha-bad-set-hit-with-empty-composition-header-candidates-def$

by *blast*

have *candidate:*

$\partial \partial trace-table \in \ partial-trace-table-candidates \ trace-openings$

using *partial*

unfolding *accepted-with-empty-composition-header-candidates-def*
partial-trace-table-candidates-def
by *simp*
have
as $\in$ *partial-trace-opening-alpha-union-bad-sets trace-openings*
unfolding *partial-trace-opening-alpha-union-bad-sets-def*
using *candidate as-bad* **by** *blast*
then show *?thesis*
unfolding *composition-alpha-partial-opening-union-hit-with-empty-header-def*
using *partial* **by** *blast*
qed

lemma *wp-composition-alpha-bad-set-hit-with-empty-header-le-partial-opening-union-hit:*
wp-event verify-monad
(composition-alpha-bad-set-hit-with-empty-composition-header-candidates s)
s $\leq$
wp-event verify-monad
(composition-alpha-partial-opening-union-hit-with-empty-header s) s
by *(rule wp-event-mono)*
(rule
composition-alpha-bad-set-hit-with-empty-header-imp-partial-opening-union-hit)

lemma *composition-alpha-partial-opening-union-hit-with-empty-header-imp-alpha-header-partial-union-on-support:*
assumes *support: out $\in$ set-dist (execute verify-monad s)*
and *hit:*
composition-alpha-partial-opening-union-hit-with-empty-header s out
shows
alpha-header-list-set-hit s
(alpha-header-supported-partial-union-bad-sets s
composition-trace-bad-alpha-space) out

proof —

obtain *fr f-fri-roots f-final as dg final trace-query-idxs*
trace-openings trace-table composition-table rest **where** *partial:*
accepted-with-empty-composition-header-candidates s out fr
f-fri-roots f-final as dg final trace-query-idxs trace-openings
trace-table composition-table
and *header:*
verifier-header-transcript s fr f-fri-roots f-final as dg []
final rest
and *trace-partial:*
accepted-with-partial-trace-openings s out fr trace-query-idxs
trace-openings
and *as-bad:*
as $\in$ partial-trace-opening-alpha-union-bad-sets trace-openings
using *hit*
unfolding *composition-alpha-partial-opening-union-hit-with-empty-header-def*
accepted-with-empty-composition-header-candidates-def
by *blast*
from *trace-partial* **obtain** *result final-state* **where** *out-eq:*


```

    out = Some (result, final-state)
    unfolding accepted-with-partial-trace-openings-def by blast
from verify-monad-accepted-transcript-shape[OF support[unfolded out-eq]]
obtain as' query-idxs where shape':
    accepted-transcript-shape s (Some (result, final-state)) as' query-idxs
by blast
from shape' obtain result' final-state' fr' f-fri-roots' f-final' dg'
    composition-fri-roots' final' rest' where out-shape:
    Some (result, final-state) = Some (result', final-state')
and header':
    verifier-header-transcript s fr' f-fri-roots' f-final' as' dg'
    composition-fri-roots' final' rest'
by (elim accepted-transcript-shape-header-query-extraction)
have as'-eq: as' = as
    using verifier-header-transcript-unique[OF header' header] by simp
have shape:
    accepted-transcript-shape s out as query-idxs
    using shape' as'-eq out-eq by simp
from as-bad obtain trace-table' where candidate':
    trace-table' ∈ partial-trace-table-candidates trace-openings
and as-bad':
    as ∈ composition-trace-bad-alpha-space trace-table'
    unfolding partial-trace-opening-alpha-union-bad-sets-def by blast
have candidate-header:
    trace-table' ∈
    alpha-header-supported-partial-trace-table-candidates s fr
    f-fri-roots f-final
proof –
    have partial-candidate':
    partial-trace-table-candidate trace-table' trace-openings
    using candidate'
    unfolding partial-trace-table-candidates-def by simp
    show ?thesis
    unfolding alpha-header-supported-partial-trace-table-candidates-def
    using support trace-partial partial-candidate' header by blast
qed
have as-union:
    as ∈
    alpha-header-supported-partial-union-bad-sets s
    composition-trace-bad-alpha-space fr f-fri-roots f-final
    unfolding alpha-header-supported-partial-union-bad-sets-def
    using candidate-header as-bad'
    composition-trace-bad-alpha-space-subset-alpha-space[of trace-table']
    by auto
    show ?thesis
    unfolding alpha-header-list-set-hit-def
proof (intro exI conjI)
    show accepted-transcript-shape s out as query-idxs
    by (rule shape)

```

```

next
  show  $out = \text{Some } (result, final\text{-}state)$ 
    by (rule out-eq)
next
  show
    verifier-header-transcript  $s$  fr  $f$ -fri-roots  $f$ -final  $as$   $dg$  []
    final rest
    by (rule header)
next
  show
     $as \in$ 
    alpha-header-supported-partial-union-bad-sets  $s$ 
    composition-trace-bad-alpha-space fr  $f$ -fri-roots  $f$ -final
    by (rule as-union)
qed
qed

lemma wp-composition-alpha-partial-opening-union-hit-with-empty-header-le-alpha-header-partial-union:
  wp-event verify-monad
    (composition-alpha-partial-opening-union-hit-with-empty-header  $s$ )  $s \leq$ 
  wp-event verify-monad
    (alpha-header-list-set-hit  $s$ 
    (alpha-header-supported-partial-union-bad-sets  $s$ 
    composition-trace-bad-alpha-space))  $s$ 
  by (rule wp-event-mono-on-support)
  (rule
    composition-alpha-partial-opening-union-hit-with-empty-header-imp-alpha-header-partial-union-on-support)

definition trace-fri-bad-with-empty-composition-header-candidates
  :: ( $f$ ,  $a$ ) protocol-channel-scheme  $\Rightarrow$ 
    (unit list  $\times$  ( $f$ ,  $a$ ) protocol-channel-scheme) option  $\Rightarrow$  bool
where
  trace-fri-bad-with-empty-composition-header-candidates  $s$  out  $\longleftrightarrow$ 
    ( $\exists$  fr  $f$ -fri-roots  $f$ -final  $as$   $dg$  final trace-query-idxs
    trace-openings trace-table composition-table.
    accepted-with-empty-composition-header-candidates  $s$  out fr
     $f$ -fri-roots  $f$ -final  $as$   $dg$  final trace-query-idxs trace-openings
    trace-table composition-table  $\wedge$ 
     $\neg$  trace-table-low-degree trace-table)

definition query-bad-with-empty-composition-header-candidates
  :: ( $f$ ,  $a$ ) protocol-channel-scheme  $\Rightarrow$ 
    (unit list  $\times$  ( $f$ ,  $a$ ) protocol-channel-scheme) option  $\Rightarrow$  bool
where
  query-bad-with-empty-composition-header-candidates  $s$  out  $\longleftrightarrow$ 
    ( $\exists$  fr  $f$ -fri-roots  $f$ -final  $as$   $dg$  final trace-query-idxs
    trace-openings trace-table composition-table.
    accepted-with-empty-composition-header-candidates  $s$  out fr
     $f$ -fri-roots  $f$ -final  $as$   $dg$  final trace-query-idxs trace-openings

```

$\text{trace-table composition-table} \wedge$
 $\text{trace-table-low-degree trace-table} \wedge$
 $\text{composition-table-low-degree maxDegree composition-table} \wedge$
 $\neg \text{all-queries-consistent trace-table composition-table as}$

definition *soundness-bad-event-partial-candidate-empty-header*
 $:: (\text{'f}, \text{'a}) \text{protocol-channel-scheme} \Rightarrow$
 $(\text{unit list} \times (\text{'f}, \text{'a}) \text{protocol-channel-scheme}) \text{option} \Rightarrow \text{bool}$

where

$\text{soundness-bad-event-partial-candidate-empty-header } s \text{ out} \longleftrightarrow$
 $\text{composition-bad-with-empty-composition-header-candidates } s \text{ out} \vee$
 $\text{trace-fri-bad-with-empty-composition-header-candidates } s \text{ out} \vee$
 $\text{query-bad-with-empty-composition-header-candidates } s \text{ out}$

definition *soundness-bad-event-table-aware-with-pairwise-side*
 $:: (\text{'f}, \text{'a}) \text{protocol-channel-scheme} \Rightarrow$
 $(\text{unit list} \times (\text{'f}, \text{'a}) \text{protocol-channel-scheme}) \text{option} \Rightarrow \text{bool}$

where

$\text{soundness-bad-event-table-aware-with-pairwise-side } s \text{ out} \longleftrightarrow$
 $\text{supported-pairwise-side-bad } s \text{ out} \vee$
 $\text{soundness-bad-event-table-aware-with-merkle } s \text{ out}$

definition *soundness-bad-event-table-aware-with-output-local-side*
 $:: (\text{'f}, \text{'a}) \text{protocol-channel-scheme} \Rightarrow$
 $(\text{unit list} \times (\text{'f}, \text{'a}) \text{protocol-channel-scheme}) \text{option} \Rightarrow \text{bool}$

where

$\text{soundness-bad-event-table-aware-with-output-local-side } s \text{ out} \longleftrightarrow$
 $\text{supported-output-local-side-bad } s \text{ out} \vee$
 $\text{soundness-bad-event-table-aware-with-merkle } s \text{ out}$

lemma *soundness-bad-event-table-aware-imp-with-merkle:*
assumes *soundness-bad-event-table-aware* $s \text{ out}$
shows *soundness-bad-event-table-aware-with-merkle* $s \text{ out}$
using *assms* **unfolding** *soundness-bad-event-table-aware-with-merkle-def* **by** *simp*

lemma *soundness-bad-event-partial-opening-imp-with-merkle:*
assumes *soundness-bad-event-partial-opening* $s \text{ out}$
shows *soundness-bad-event-partial-opening-with-merkle* $s \text{ out}$
using *assms*
unfolding *soundness-bad-event-partial-opening-with-merkle-def* **by** *simp*

lemma *initial-merkle-binding-bad-imp-partial-opening-with-initial-merkle:*
assumes *initial-merkle-binding-bad* $s \text{ out}$
shows *soundness-bad-event-partial-opening-with-initial-merkle* $s \text{ out}$
using *assms*
unfolding *soundness-bad-event-partial-opening-with-initial-merkle-def*
by *simp*

lemma *soundness-bad-event-partial-opening-imp-with-initial-merkle:*

assumes *soundness-bad-event-partial-opening s out*
shows *soundness-bad-event-partial-opening-with-initial-merkle s out*
using *assms*
unfolding *soundness-bad-event-partial-opening-with-initial-merkle-def*
by *simp*

lemma *partial-merkle-inconsistency-bad-imp-partial-opening-with-partial-merkle:*
assumes *partial-merkle-inconsistency-bad s out*
shows *soundness-bad-event-partial-opening-with-partial-merkle s out*
using *assms*
unfolding *soundness-bad-event-partial-opening-with-partial-merkle-def*
by *simp*

lemma *soundness-bad-event-partial-opening-imp-with-partial-merkle:*
assumes *soundness-bad-event-partial-opening s out*
shows *soundness-bad-event-partial-opening-with-partial-merkle s out*
using *assms*
unfolding *soundness-bad-event-partial-opening-with-partial-merkle-def*
by *simp*

lemma *soundness-bad-event-partial-candidate-imp-with-partial-merkle:*
assumes *soundness-bad-event-partial-candidate s out*
shows *soundness-bad-event-partial-candidate-with-partial-merkle s out*
using *assms*
unfolding *soundness-bad-event-partial-candidate-with-partial-merkle-def*
by *simp*

lemma *partial-merkle-inconsistency-bad-imp-partial-candidate-with-partial-merkle:*
assumes *partial-merkle-inconsistency-bad s out*
shows *soundness-bad-event-partial-candidate-with-partial-merkle s out*
using *assms*
unfolding *soundness-bad-event-partial-candidate-with-partial-merkle-def*
by *simp*

lemma *accepted-with-partial-initial-openings-alpha-length:*
assumes
 accepted-with-partial-initial-openings s out fr f-fri-roots f-final as dg
 composition-fri-roots final trace-query-idxs trace-openings
 composition-query-idxs composition-openings
shows *length as = length spec*
proof –
 obtain *rest* **where**
 verifier-header-transcript s fr f-fri-roots f-final as dg
 composition-fri-roots final rest
 using *assms*
 unfolding *accepted-with-partial-initial-openings-def* **by** *blast*
 then show *?thesis*
 using *verifier-header-transcript-shapes(3)* **by** *blast*
qed

```

lemma composition-bad-with-partial-candidates-split:
  assumes false-statement:  $\neg$  exists-valid-trace
    and bad: composition-bad-with-partial-candidates s out
  shows
    composition-degree-bad-with-partial-candidates s out  $\vee$ 
    composition-randomization-bad-with-partial-candidates s out
proof –
  obtain fr f-fri-roots f-final as dg composition-fri-roots final
    trace-query-idxs trace-openings composition-query-idxs
    composition-openings trace-table composition-table f
  where partial:
    accepted-with-partial-initial-openings s out fr f-fri-roots f-final
    as dg composition-fri-roots final trace-query-idxs trace-openings
    composition-query-idxs composition-openings
  and trace-candidate:
    partial-trace-table-candidate trace-table trace-openings
  and composition-candidate:
    partial-composition-table-candidate composition-table
    composition-openings
  and deg-f: degree f < clength
  and trace-table: trace-table = map (poly f) eval-domain
  and violated: violated-constraints f  $\neq$  {}
  and composition-low:
    composition-table-low-degree maxDegree composition-table
  and all-queries:
    all-queries-consistent trace-table composition-table as
  using bad
  unfolding composition-bad-with-partial-candidates-def by blast
show ?thesis
proof (cases common-denominator-degree-bounds f as)
  case False
  then have composition-degree-bad-with-partial-candidates s out
    unfolding composition-degree-bad-with-partial-candidates-def
    using partial trace-candidate composition-candidate deg-f trace-table
    violated composition-low all-queries
    by blast
  then show ?thesis by blast
next
  case True
  note bounds = True
  have len-as: length as = length spec
    by (rule accepted-with-partial-initial-openings-alpha-length
      [OF partial])
  have hides:
    random-combination-common-denominator-hides-violations f as
    by (rule random-combination-common-denominator-hides-from-queries
      [OF len-as false-statement deg-f trace-table composition-low
        all-queries bounds])

```

have *composition-randomization-bad-with-partial-candidates s out*
unfolding *composition-randomization-bad-with-partial-candidates-def*
using *partial trace-candidate composition-candidate deg-f trace-table*
violated composition-low all-queries bounds hides
by *blast*
then show *?thesis* **by** *blast*
qed
qed

lemma *composition-degree-bad-with-partial-candidates-false:*
 $\neg$ *composition-degree-bad-with-partial-candidates s out*
using *common-denominator-degree-bounds-from-spec-degree-wellformed*
 $[OF \text{ spec-degree-wellformed-from-spec-query-margin}]$
unfolding *composition-degree-bad-with-partial-candidates-def* **by** *blast*

lemma *composition-randomization-bad-with-partial-candidates-imp-alpha-hit:*
assumes *bad: composition-randomization-bad-with-partial-candidates s out*
shows *composition-alpha-bad-set-hit-with-partial-candidates s out*
proof –
obtain *fr f-fri-roots f-final as dg composition-fri-roots final*
trace-query-idxs trace-openings composition-query-idxs
composition-openings trace-table composition-table f
where *partial:*
accepted-with-partial-initial-openings s out fr f-fri-roots f-final
as dg composition-fri-roots final trace-query-idxs trace-openings
composition-query-idxs composition-openings
and *trace-candidate:*
partial-trace-table-candidate trace-table trace-openings
and *deg-f: degree f < clength*
and *trace-table: trace-table = map (poly f) eval-domain*
and *violated: violated-constraints f $\neq$ {}*
and *bounds: common-denominator-degree-bounds f as*
and *hides:*
random-combination-common-denominator-hides-violations f as
using *bad*
unfolding *composition-randomization-bad-with-partial-candidates-def*
by *blast*
have *as-in:*
as $\in$ common-denominator-hiding-alpha-space f
using *accepted-with-partial-initial-openings-alpha-length[OF partial]*
hides
unfolding *common-denominator-hiding-alpha-space-def alpha-space-def*
by *simp*
have *witness-eq: low-degree-trace-witness trace-table = f*
by *(rule low-degree-trace-witness-eq[OF deg-f trace-table])*
have *as $\in$ composition-trace-bad-alpha-space trace-table*
using *deg-f trace-table violated bounds as-in*
unfolding *composition-trace-bad-alpha-space-def witness-eq*
by *simp*

```

then show ?thesis
  unfolding composition-alpha-bad-set-hit-with-partial-candidates-def
  using partial trace-candidate by blast
qed

lemma composition-alpha-bad-set-hit-with-partial-candidates-imp-alpha-header-partial-union-on-support:
  assumes support:  $out \in \text{set-dist}$  (execute verify-monad s)
  and hit: composition-alpha-bad-set-hit-with-partial-candidates s out
  shows
    alpha-header-list-set-hit s
    (alpha-header-supported-partial-union-bad-sets s
     composition-trace-bad-alpha-space) out
proof -
  obtain fr f-fri-roots f-final as dg composition-fri-roots final
    trace-query-idxs trace-openings composition-query-idxs
    composition-openings trace-table rest
  where partial:
    accepted-with-partial-initial-openings s out fr f-fri-roots f-final
    as dg composition-fri-roots final trace-query-idxs trace-openings
    composition-query-idxs composition-openings
  and header:
    verifier-header-transcript s fr f-fri-roots f-final as dg
    composition-fri-roots final rest
  and trace-partial:
    accepted-with-partial-trace-openings s out fr trace-query-idxs
    trace-openings
  and trace-candidate:
    partial-trace-table-candidate trace-table trace-openings
  and as-bad:
    as  $\in$  composition-trace-bad-alpha-space trace-table
  using hit
  unfolding composition-alpha-bad-set-hit-with-partial-candidates-def
    accepted-with-partial-initial-openings-def
  by blast
  from trace-partial obtain result final-state where out-eq:
    out = Some (result, final-state)
  unfolding accepted-with-partial-trace-openings-def by blast
  from verify-monad-accepted-transcript-shape[OF support[unfolded out-eq]]
  obtain as' query-idxs where shape':
    accepted-transcript-shape s (Some (result, final-state)) as' query-idxs
  by blast
  from shape' obtain result' final-state' fr' f-fri-roots' f-final' dg'
    composition-fri-roots' final' rest' where out-shape:
    Some (result, final-state) = Some (result', final-state')
  and header':
    verifier-header-transcript s fr' f-fri-roots' f-final' as' dg'
    composition-fri-roots' final' rest'
  by (elim accepted-transcript-shape-header-query-extraction)
  have as'-eq: as' = as

```

```

using verifier-header-transcript-unique[OF header' header] by simp
have shape:
  accepted-transcript-shape s out as query-idxs
  using shape' as'-eq out-eq by simp
have candidate-header:
  trace-table ∈
    alpha-header-supported-partial-trace-table-candidates s fr
    f-fri-roots f-final
  unfolding alpha-header-supported-partial-trace-table-candidates-def
  using support trace-partial trace-candidate header by blast
have as-union:
  as ∈
    alpha-header-supported-partial-union-bad-sets s
    composition-trace-bad-alpha-space fr f-fri-roots f-final
  unfolding alpha-header-supported-partial-union-bad-sets-def
  using candidate-header as-bad
    composition-trace-bad-alpha-space-subset-alpha-space[of trace-table]
  by auto
show ?thesis
  unfolding alpha-header-list-set-hit-def
  using shape out-eq header as-union by blast
qed

lemma wp-composition-randomization-bad-with-partial-candidates-le-alpha-header-partial-union:
  wp-event verify-monad
    (composition-randomization-bad-with-partial-candidates s) s ≤
  wp-event verify-monad
    (alpha-header-list-set-hit s
      (alpha-header-supported-partial-union-bad-sets s
        composition-trace-bad-alpha-space)) s
  by (rule wp-event-mono-on-support)
  (rule
    composition-alpha-bad-set-hit-with-partial-candidates-imp-alpha-header-partial-union-on-support
    [OF - composition-randomization-bad-with-partial-candidates-imp-alpha-hit])

lemma accepted-partial-candidate-partition:
  assumes false-statement: ¬ exists-valid-trace
  and partial:
    accepted-with-partial-initial-openings s out fr f-fri-roots f-final
    as dg composition-fri-roots final trace-query-idxs trace-openings
    composition-query-idxs composition-openings
  and trace-candidate:
    partial-trace-table-candidate trace-table trace-openings
  and composition-candidate:
    partial-composition-table-candidate composition-table
    composition-openings
  shows soundness-bad-event-partial-candidate s out
proof (cases trace-table-low-degree trace-table)
  case False

```



```

have trace-fri-bad-with-partial-candidates s out
  unfolding trace-fri-bad-with-partial-candidates-def
  using partial trace-candidate False by blast
then show ?thesis
  unfolding soundness-bad-event-partial-candidate-def by blast
next
case True
note trace-low = True
show ?thesis
proof (cases composition-table-low-degree maxDegree composition-table)
  case False
  have composition-fri-bad-with-partial-candidates s out
    unfolding composition-fri-bad-with-partial-candidates-def
    using partial trace-candidate composition-candidate trace-low False
    by blast
  then show ?thesis
    unfolding soundness-bad-event-partial-candidate-def by blast
next
case True
note composition-low = True
show ?thesis
proof (cases all-queries-consistent trace-table composition-table as)
  case False
  have query-bad-with-partial-candidates s out
    unfolding query-bad-with-partial-candidates-def
    using partial trace-candidate composition-candidate trace-low
      composition-low False
    by blast
  then show ?thesis
    unfolding soundness-bad-event-partial-candidate-def by blast
next
case True
note all-queries = True
from trace-low obtain f where
  deg-f: degree f < clength
  and trace-table-eq: trace-table = map (poly f) eval-domain
  unfolding trace-table-low-degree-def by blast
have violated: violated-constraints f ≠ {}
  by (rule false-statement-violated-constraints
    [OF false-statement deg-f])
have composition-bad-with-partial-candidates s out
  unfolding composition-bad-with-partial-candidates-def
  using partial trace-candidate composition-candidate deg-f
    trace-table-eq violated composition-low all-queries
  by blast
then show ?thesis
  unfolding soundness-bad-event-partial-candidate-def by blast
qed
qed

```

qed

lemma *constant-composition-table-low-degree:*

*composition-table-low-degree maxDegree (replicate (scale * clength) final)*

proof –

have *map (poly [:final:]) eval-domain =*
*replicate (scale * clength) final*

proof –

have *map (poly [:final:]) eval-domain =*
replicate (length eval-domain) final

proof (*rule nth-equalityI*)

show *length (map (poly [:final:]) eval-domain) =*
length (replicate (length eval-domain) final)

by *simp*

next

fix *i*

assume *i < length (map (poly [:final:]) eval-domain)*

then show *map (poly [:final:]) eval-domain ! i =*
replicate (length eval-domain) final ! i

by *simp*

qed

then show *?thesis*

using *eval-domain-length* **by** (*simp add: mult.commute*)

qed

then show *?thesis*

unfolding *composition-table-low-degree-def*

by (*intro exI[of - [:final:]]*) *simp*

qed

lemma *accepted-with-empty-composition-header-candidates-alpha-length:*

assumes

accepted-with-empty-composition-header-candidates s out fr
f-fri-roots f-final as dg final trace-query-idxs trace-openings
trace-table composition-table

shows *length as = length spec*

proof –

obtain *rest* **where**

verifier-header-transcript s fr f-fri-roots f-final as dg []
final rest

using *assms*

unfolding *accepted-with-empty-composition-header-candidates-def* **by** *blast*

then show *?thesis*

using *verifier-header-transcript-shapes(3)* **by** *blast*

qed

lemma *composition-bad-with-empty-composition-header-candidates-split:*

assumes *false-statement: $\neg$ exists-valid-trace*

and *bad: composition-bad-with-empty-composition-header-candidates s out*

shows

composition-degree-bad-with-empty-composition-header-candidates s out $\vee$
composition-randomization-bad-with-empty-composition-header-candidates s out
proof –
obtain *fr f-fri-roots f-final as dg final trace-query-idxs*
trace-openings trace-table composition-table f
where *partial*:
accepted-with-empty-composition-header-candidates s out fr
f-fri-roots f-final as dg final trace-query-idxs trace-openings
trace-table composition-table
and *deg-f*: *degree f < clength*
and *trace-table*: *trace-table = map (poly f) eval-domain*
and *violated*: *violated-constraints f $\neq$ {}*
and *composition-low*:
composition-table-low-degree maxDegree composition-table
and *all-queries*:
all-queries-consistent trace-table composition-table as
using *bad*
unfolding *composition-bad-with-empty-composition-header-candidates-def*
by *blast*
show *?thesis*
proof (*cases common-denominator-degree-bounds f as*)
case *False*
then have
composition-degree-bad-with-empty-composition-header-candidates s out
unfolding *composition-degree-bad-with-empty-composition-header-candidates-def*
using *partial deg-f trace-table violated composition-low all-queries*
by *blast*
then show *?thesis* **by** *blast*
next
case *True*
note *bounds = True*
have *len-as: length as = length spec*
by (*rule accepted-with-empty-composition-header-candidates-alpha-length*
[OF partial])
have *hides*:
random-combination-common-denominator-hides-violations f as
by (*rule random-combination-common-denominator-hides-from-queries*
[OF len-as false-statement deg-f trace-table composition-low
all-queries bounds])
have
composition-randomization-bad-with-empty-composition-header-candidates
s out
unfolding
composition-randomization-bad-with-empty-composition-header-candidates-def
using *partial deg-f trace-table violated composition-low all-queries*
bounds hides
by *blast*
then show *?thesis* **by** *blast*
qed

qed

lemma *composition-degree-bad-with-empty-composition-header-candidates-false:*
 $\neg$ *composition-degree-bad-with-empty-composition-header-candidates s out*
 using *common-denominator-degree-bounds-from-spec-degree-wellformed*
 [*OF spec-degree-wellformed-from-spec-query-margin*]
 unfolding *composition-degree-bad-with-empty-composition-header-candidates-def*
 by *blast*

lemma *composition-degree-bad-with-empty-composition-header-candidates-bound:*
 wp-event verify-monad
 (*composition-degree-bad-with-empty-composition-header-candidates s*) $s \leq$
 0

proof —

have *composition-degree-bad-with-empty-composition-header-candidates s =*
 ($\lambda out :: (\text{unit list} \times ('f, 'a) \text{ protocol-channel-scheme}) \text{ option.}$
 False)
 by (*rule ext*)
 (*simp add:*
 composition-degree-bad-with-empty-composition-header-candidates-false)
 then show *?thesis*
 unfolding *wp-event-def wp-def dist-expect-def* **by** *simp*

qed

lemma *composition-randomization-bad-with-empty-composition-header-candidates-imp-alpha-hit:*
 assumes *bad:*

composition-randomization-bad-with-empty-composition-header-candidates
 s out

shows

composition-alpha-bad-set-hit-with-empty-composition-header-candidates
 s out

proof —

obtain *fr f-fri-roots f-final as dg final trace-query-idxs*
 trace-openings trace-table composition-table f

where *partial:*

accepted-with-empty-composition-header-candidates s out fr
 f-fri-roots f-final as dg final trace-query-idxs trace-openings
 trace-table composition-table

and *deg-f: degree f < clength*

and *trace-table: trace-table = map (poly f) eval-domain*

and *violated: violated-constraints f $\neq \{\}$*

and *bounds: common-denominator-degree-bounds f as*

and *hides:*

random-combination-common-denominator-hides-violations f as

using *bad*

unfolding

composition-randomization-bad-with-empty-composition-header-candidates-def

by *blast*

have *as-in:*

```

as ∈ common-denominator-hiding-alpha-space f
using accepted-with-empty-composition-header-candidates-alpha-length[OF partial]
  hides
unfolding common-denominator-hiding-alpha-space-def alpha-space-def
by simp
have witness-eq: low-degree-trace-witness trace-table = f
  by (rule low-degree-trace-witness-eq[OF deg-f trace-table])
have as ∈ composition-trace-bad-alpha-space trace-table
  using deg-f trace-table violated bounds as-in
  unfolding composition-trace-bad-alpha-space-def witness-eq
  by simp
then show ?thesis
  unfolding
    composition-alpha-bad-set-hit-with-empty-composition-header-candidates-def
  using partial by blast
qed

lemma wp-composition-randomization-bad-with-empty-composition-header-candidates-le-alpha-hit:
  wp-event verify-monad
    (composition-randomization-bad-with-empty-composition-header-candidates s)
  s ≤
  wp-event verify-monad
    (composition-alpha-bad-set-hit-with-empty-composition-header-candidates s)
  s
by (rule wp-event-mono)
(rule
  composition-randomization-bad-with-empty-composition-header-candidates-imp-alpha-hit)

lemma accepted-empty-composition-header-candidate-partition:
assumes false-statement: ¬ exists-valid-trace
and partial:
  accepted-with-empty-composition-header-candidates s out fr
  f-fri-roots f-final as dg final trace-query-idxs trace-openings
  trace-table composition-table
shows soundness-bad-event-partial-candidate-empty-header s out
proof (cases trace-table-low-degree trace-table)
case False
have trace-fri-bad-with-empty-composition-header-candidates s out
  unfolding trace-fri-bad-with-empty-composition-header-candidates-def
  using partial False by blast
then show ?thesis
  unfolding soundness-bad-event-partial-candidate-empty-header-def by blast
next
case True
note trace-low = True
have composition-low:
  composition-table-low-degree maxDegree composition-table
proof –
  have composition-table = replicate (scale * clength) final

```

```

    using partial
    unfolding accepted-with-empty-composition-header-candidates-def by simp
  then show ?thesis
    using constant-composition-table-low-degree by simp
qed
show ?thesis
proof (cases all-queries-consistent trace-table composition-table as)
  case False
  have query-bad-with-empty-composition-header-candidates s out
    unfolding query-bad-with-empty-composition-header-candidates-def
    using partial trace-low composition-low False by blast
  then show ?thesis
    unfolding soundness-bad-event-partial-candidate-empty-header-def
    by blast
next
  case True
  note all-queries = True
  from trace-low obtain f where
    deg-f: degree f < clength
    and trace-table-eq: trace-table = map (poly f) eval-domain
    unfolding trace-table-low-degree-def by blast
  have violated: violated-constraints f ≠ {}
    by (rule false-statement-violated-constraints
        [OF false-statement deg-f])
  have composition-bad-with-empty-composition-header-candidates s out
    unfolding composition-bad-with-empty-composition-header-candidates-def
    using partial deg-f trace-table-eq violated composition-low all-queries
    by blast
  then show ?thesis
    unfolding soundness-bad-event-partial-candidate-empty-header-def
    by blast
qed
qed

lemma fri-merkle-binding-bad-imp-table-aware-with-merkle:
  assumes fri-merkle-binding-bad s out
  shows soundness-bad-event-table-aware-with-merkle s out
  using assms unfolding soundness-bad-event-table-aware-with-merkle-def by simp

lemma fri-merkle-binding-bad-imp-partial-opening-with-merkle:
  assumes fri-merkle-binding-bad s out
  shows soundness-bad-event-partial-opening-with-merkle s out
  using assms
  unfolding soundness-bad-event-partial-opening-with-merkle-def by simp

lemma soundness-bad-event-table-aware-imp-partial-opening-on-support:
  assumes support: out ∈ set-dist (execute verify-monad s)
  and bad: soundness-bad-event-table-aware s out
  shows soundness-bad-event-partial-opening s out

```

```

    using bad
  proof (unfold soundness-bad-event-table-aware-def, elim disjE)
    assume composition-bad s out
    then show ?thesis
      unfolding soundness-bad-event-partial-opening-def by blast
  next
    assume trace-bad: trace-fri-bad-with-tables s out
    then have trace-fri-bad-with-partial-openings s out
      by (rule trace-fri-bad-with-tables-imp-partial-openings-on-support
        [OF support])
    then show ?thesis
      unfolding soundness-bad-event-partial-opening-def by blast
  next
    assume composition-bad: composition-fri-bad-with-tables s out
    then have composition-fri-bad-with-partial-openings s out
      by (rule composition-fri-bad-with-tables-imp-partial-openings-on-support
        [OF support])
    then show ?thesis
      unfolding soundness-bad-event-partial-opening-def by blast
  next
    assume query-bad s out
    then show ?thesis
      unfolding soundness-bad-event-partial-opening-def by blast
qed

```

lemma *soundness-bad-event-table-aware-with-merkle-imp-partial-opening-with-merkle-on-support:*
 assumes support: $out \in \text{set-dist } (\text{execute verify-monad } s)$
 and bad: *soundness-bad-event-table-aware-with-merkle s out*
 shows *soundness-bad-event-partial-opening-with-merkle s out*
 using bad
proof (unfold soundness-bad-event-table-aware-with-merkle-def, elim disjE)
 assume fri-merkle-binding-bad s out
 then show ?thesis
 by (rule fri-merkle-binding-bad-imp-partial-opening-with-merkle)
 next
 assume soundness-bad-event-table-aware s out
 then have soundness-bad-event-partial-opening s out
 by (rule soundness-bad-event-table-aware-imp-partial-opening-on-support
 [OF support])
 then show ?thesis
 by (rule soundness-bad-event-partial-opening-imp-with-merkle)
 qed

lemma *wp-soundness-bad-event-table-aware-le-partial-opening:*
 wp-event verify-monad (soundness-bad-event-table-aware s) $s \leq$
 wp-event verify-monad (soundness-bad-event-partial-opening s) s
by (rule wp-event-mono-on-support)
 (rule soundness-bad-event-table-aware-imp-partial-opening-on-support)

lemma *wp-soundness-bad-event-table-aware-with-merkle-le-partial-opening-with-merkle:*
wp-event verify-monad (soundness-bad-event-table-aware-with-merkle s) s ≤
wp-event verify-monad
(soundness-bad-event-partial-opening-with-merkle s) s
by (rule *wp-event-mono-on-support*)
(rule
soundness-bad-event-table-aware-with-merkle-imp-partial-opening-with-merkle-on-support)

lemma *soundness-bad-event-table-aware-with-merkle-imp-with-pairwise-side:*
assumes *soundness-bad-event-table-aware-with-merkle s out*
shows *soundness-bad-event-table-aware-with-pairwise-side s out*
using *assms*
unfolding *soundness-bad-event-table-aware-with-pairwise-side-def* **by** *simp*

lemma *supported-pairwise-side-bad-imp-table-aware-with-pairwise-side:*
assumes *supported-pairwise-side-bad s out*
shows *soundness-bad-event-table-aware-with-pairwise-side s out*
using *assms*
unfolding *soundness-bad-event-table-aware-with-pairwise-side-def* **by** *simp*

lemma *soundness-bad-event-table-aware-with-merkle-imp-with-output-local-side:*
assumes *soundness-bad-event-table-aware-with-merkle s out*
shows *soundness-bad-event-table-aware-with-output-local-side s out*
using *assms*
unfolding *soundness-bad-event-table-aware-with-output-local-side-def*
by *simp*

lemma *supported-output-local-side-bad-imp-table-aware-with-output-local-side:*
assumes *supported-output-local-side-bad s out*
shows *soundness-bad-event-table-aware-with-output-local-side s out*
using *assms*
unfolding *soundness-bad-event-table-aware-with-output-local-side-def*
by *simp*

lemma *accepted-with-fri-tables-table-aware-partition:*
assumes *false-statement: ¬ exists-valid-trace*
and *tables: accepted-with-fri-tables s out*
shows *soundness-bad-event-table-aware s out*
proof –
from *accepted-with-fri-tables-obtain-bound-tables[OF tables]*
obtain *trace-table composition-table as query-idxs* **where** *bound:*
accepted-with-bound-tables s out trace-table composition-table as
query-idxs
by *blast*
have *partition:*
composition-bad s out ∨ trace-fri-bad s out ∨
composition-fri-bad s out ∨ query-bad s out
by (rule *accepted-bound-table-partition[OF false-statement bound]*)
then show *?thesis*


```

proof
  assume composition-bad s out
  then show ?thesis
    unfolding soundness-bad-event-table-aware-def by blast
next
assume rest: trace-fri-bad s out  $\vee$ 
  composition-fri-bad s out  $\vee$  query-bad s out
then show ?thesis
proof
  assume trace-bad: trace-fri-bad s out
  then have trace-fri-bad-with-tables s out
    using tables unfolding trace-fri-bad-with-tables-def by simp
  then show ?thesis
    unfolding soundness-bad-event-table-aware-def by blast
next
assume rest': composition-fri-bad s out  $\vee$  query-bad s out
then show ?thesis
proof
  assume composition-bad: composition-fri-bad s out
  then have composition-fri-bad-with-tables s out
    using tables unfolding composition-fri-bad-with-tables-def by simp
  then show ?thesis
    unfolding soundness-bad-event-table-aware-def by blast
next
  assume query-bad s out
  then show ?thesis
    unfolding soundness-bad-event-table-aware-def by blast
qed
qed
qed
qed

lemma accepted-partition-soundness-bad-event-table-aware-with-merkle:
  assumes false-statement:  $\neg$  exists-valid-trace
  and acc: accepted out
  shows soundness-bad-event-table-aware-with-merkle s out
proof -
  from accepted-partition-fri-merkle-binding[OF acc]
  consider
    (merkle-bad) fri-merkle-binding-bad s out
  | (with-tables) accepted-with-fri-tables s out
  by blast
then show ?thesis
proof cases
  case merkle-bad
  then show ?thesis
    by (rule fri-merkle-binding-bad-imp-table-aware-with-merkle)
next
  case with-tables

```

```

have soundness-bad-event-table-aware s out
  by (rule accepted-with-fri-tables-table-aware-partition
    [OF false-statement with-tables])
then show ?thesis
  by (rule soundness-bad-event-table-aware-imp-with-merkle)
qed
qed

lemma accepted-partition-soundness-bad-event-table-aware-with-pairwise-side:
  assumes false-statement:  $\neg$  exists-valid-trace
  and acc: accepted out
shows soundness-bad-event-table-aware-with-pairwise-side s out
by (rule soundness-bad-event-table-aware-with-merkle-imp-with-pairwise-side)
  (rule accepted-partition-soundness-bad-event-table-aware-with-merkle
    [OF false-statement acc])

lemma accepted-partition-soundness-bad-event-table-aware-with-output-local-side:
  assumes false-statement:  $\neg$  exists-valid-trace
  and acc: accepted out
shows soundness-bad-event-table-aware-with-output-local-side s out
by (rule
  soundness-bad-event-table-aware-with-merkle-imp-with-output-local-side)
  (rule accepted-partition-soundness-bad-event-table-aware-with-merkle
    [OF false-statement acc])

lemma accepted-partition-soundness-bad-event-partial-opening-with-merkle:
  assumes false-statement:  $\neg$  exists-valid-trace
  and supp: out  $\in$  set-dist (execute verify-monad s)
  and acc: accepted out
shows soundness-bad-event-partial-opening-with-merkle s out
proof –
  have soundness-bad-event-table-aware-with-merkle s out
  by (rule accepted-partition-soundness-bad-event-table-aware-with-merkle
    [OF false-statement acc])
  then show ?thesis
  by (rule
    soundness-bad-event-table-aware-with-merkle-imp-partial-opening-with-merkle-on-support
    [OF supp])
qed

lemma accepted-partition-soundness-bad-event-partial-opening-with-initial-merkle:
  assumes false-statement:  $\neg$  exists-valid-trace
  and supp: out  $\in$  set-dist (execute verify-monad s)
  and acc: accepted out
shows soundness-bad-event-partial-opening-with-initial-merkle s out
proof –
  from acc obtain result final-state where out-eq:
    out = Some (result, final-state)
  unfolding accepted-def by (cases out) auto

```

```

from verify-monad-accepted-transcript-shape[OF supp[unfolded out-eq]]
obtain as query-idxs where shape:
  accepted-transcript-shape s (Some (result, final-state)) as query-idxs
by blast
show ?thesis
proof (cases
   $\exists$  trace-table composition-table.
    accepted-with-bound-tables s (Some (result, final-state))
    trace-table composition-table as query-idxs)
case False
have initial-merkle-binding-bad s out
  unfolding initial-merkle-binding-bad-def out-eq
  using shape False by blast
then show ?thesis
  by (rule initial-merkle-binding-bad-imp-partial-opening-with-initial-merkle)
next
case True
then obtain trace-table composition-table where bound:
  accepted-with-bound-tables s (Some (result, final-state))
  trace-table composition-table as query-idxs
by blast
have partial-bad:
  soundness-bad-event-partial-opening s out
proof –
  have partition:
    composition-bad s (Some (result, final-state))  $\vee$ 
    trace-fri-bad s (Some (result, final-state))  $\vee$ 
    composition-fri-bad s (Some (result, final-state))  $\vee$ 
    query-bad s (Some (result, final-state))
    by (rule accepted-bound-table-partition[OF false-statement bound])
  then show ?thesis
proof
  assume composition-bad s (Some (result, final-state))
  then show ?thesis
    unfolding soundness-bad-event-partial-opening-def out-eq by blast
next
assume rest:
  trace-fri-bad s (Some (result, final-state))  $\vee$ 
  composition-fri-bad s (Some (result, final-state))  $\vee$ 
  query-bad s (Some (result, final-state))
then show ?thesis
proof
  assume trace-bad:
    trace-fri-bad s (Some (result, final-state))
  from verify-monad-accepted-with-partial-trace-openings
    [OF supp[unfolded out-eq]]
  obtain fr query-idxs' trace-openings where partial:
    accepted-with-partial-trace-openings s
    (Some (result, final-state)) fr query-idxs' trace-openings

```

```

    by blast
  have trace-fri-bad-with-partial-openings s out
    unfolding trace-fri-bad-with-partial-openings-def out-eq
    using trace-bad partial by blast
  then show ?thesis
    unfolding soundness-bad-event-partial-opening-def by blast
next
assume rest':
  composition-fri-bad s (Some (result, final-state)) ∨
  query-bad s (Some (result, final-state))
then show ?thesis
proof
  assume composition-bad:
    composition-fri-bad s (Some (result, final-state))
  from verify-monad-accepted-with-partial-trace-openings
    [OF supp[unfolded out-eq]]
  obtain fr query-idxs' trace-openings where partial:
    accepted-with-partial-trace-openings s
    (Some (result, final-state)) fr query-idxs' trace-openings
  by blast
  have composition-fri-bad-with-partial-openings s out
    unfolding composition-fri-bad-with-partial-openings-def out-eq
    using composition-bad partial by blast
  then show ?thesis
    unfolding soundness-bad-event-partial-opening-def by blast
next
  assume query-bad s (Some (result, final-state))
  then show ?thesis
    unfolding soundness-bad-event-partial-opening-def out-eq
    by blast
qed
qed
qed
qed
then show ?thesis
  by (rule soundness-bad-event-partial-opening-imp-with-initial-merkle)
qed
qed

lemma accepted-partition-soundness-bad-event-partial-candidate-with-partial-merkle-if-header-nonempty:
  assumes false-statement:  $\neg$  exists-valid-trace
  and supp:  $out \in \text{set-dist } (\text{execute verify-monad } s)$ 
  and acc: accepted out
  and header:
    verifier-header-transcript s fr f-fri-roots f-final as dg
    composition-fri-roots final rest
  and comp-nonempty: composition-fri-roots  $\neq []$ 
  shows soundness-bad-event-partial-candidate-with-partial-merkle s out
proof -

```

```

from acc obtain result final-state where out-eq:
  out = Some (result, final-state)
  unfolding accepted-def by (cases out) auto
from verify-monad-partial-initial-candidates-or-partial-merkle-bad
  [OF supp[unfolded out-eq] header comp-nonempty]
consider
  (partial-merkle)
  partial-merkle-inconsistency-bad s (Some (result, final-state))
| (candidates) trace-query-idxs trace-openings composition-query-idxs
  composition-openings trace-table composition-table where
  accepted-with-partial-initial-openings s
  (Some (result, final-state)) fr f-fri-roots f-final as dg
  composition-fri-roots final trace-query-idxs trace-openings
  composition-query-idxs composition-openings
  partial-trace-table-candidate trace-table trace-openings
  partial-composition-table-candidate composition-table
  composition-openings
by blast
then show ?thesis
proof cases
  case partial-merkle
  then show ?thesis
    unfolding out-eq
    by (rule
      partial-merkle-inconsistency-bad-imp-partial-candidate-with-partial-merkle)
  next
  case (candidates trace-query-idxs trace-openings composition-query-idxs
    composition-openings trace-table composition-table)
  have bad:
    soundness-bad-event-partial-candidate s
    (Some (result, final-state))
    by (rule accepted-partial-candidate-partition
      [OF false-statement candidates(1) candidates(2) candidates(3)])
  then show ?thesis
    unfolding out-eq
    by (rule soundness-bad-event-partial-candidate-imp-with-partial-merkle)
  qed
qed

lemma accepted-partition-soundness-bad-event-partial-candidate-with-partial-merkle-or-empty-header:
assumes false-statement: ¬ exists-valid-trace
and supp: out ∈ set-dist (execute verify-monad s)
and acc: accepted out
and header:
  verifier-header-transcript s fr f-fri-roots f-final as dg
  composition-fri-roots final rest
shows
  soundness-bad-event-partial-candidate-with-partial-merkle s out ∨
  soundness-bad-event-partial-candidate-empty-header s out

```

```

proof (cases composition-fri-roots = [])
  case False
  then have comp-nonempty: composition-fri-roots  $\neq$  []
    by simp
  have soundness-bad-event-partial-candidate-with-partial-merkle s out
    by (rule
      accepted-partition-soundness-bad-event-partial-candidate-with-partial-merkle-if-header-nonempty
      [OF false-statement supp acc header comp-nonempty])
  then show ?thesis by simp
next
  case True
  note comp-empty = True
  from acc obtain result final-state where out-eq:
    out = Some (result, final-state)
    unfolding accepted-def by (cases out) auto
  show ?thesis
  proof (cases
    partial-merkle-inconsistency-bad s (Some (result, final-state)))
    case True
    then have soundness-bad-event-partial-candidate-with-partial-merkle s out
      unfolding out-eq
      by (rule
        partial-merkle-inconsistency-bad-imp-partial-candidate-with-partial-merkle)
    then show ?thesis by simp
  next
    case False
    from verify-monad-accepted-with-partial-trace-openings-for-header
      [OF supp[unfolded out-eq] header]
    obtain trace-query-idxs trace-openings where trace-partial:
      accepted-with-partial-trace-openings s
      (Some (result, final-state)) fr trace-query-idxs trace-openings
    by blast
    obtain trace-table where trace-candidate:
      partial-trace-table-candidate trace-table trace-openings
    using accepted-with-partial-trace-openings-candidate-if-no-partial-merkle-bad
      [OF trace-partial False]
    by blast
    have accepted-out: accepted (Some (result, final-state))
      by (rule accepted-with-partial-trace-openings-imp-accepted
        [OF trace-partial])
    let ?composition-table = replicate (scale * clength) final
    have empty-partial:
      accepted-with-empty-composition-header-candidates s
      (Some (result, final-state)) fr f-fri-roots f-final as dg final
      trace-query-idxs trace-openings trace-table ?composition-table
    unfolding accepted-with-empty-composition-header-candidates-def
    by (intro conjI exI[of - rest])
      (use accepted-out header comp-empty trace-partial trace-candidate
        in simp-all)

```

have *soundness-bad-event-partial-candidate-empty-header s*
 (Some (result, final-state))
by (rule *accepted-empty-composition-header-candidate-partition*
 [OF *false-statement empty-partial*])
then show *?thesis*
unfolding *out-eq* **by** *simp*
qed
qed

definition *trace-fri-with-tables-opening-algebraic-cover*

:: (*'f*, *'a*) *protocol-channel-scheme* $\Rightarrow$
 (*'f list* $\Rightarrow$ *'f list set*) $\Rightarrow$ *bool*

where

trace-fri-with-tables-opening-algebraic-cover s bad-sets $\longleftrightarrow$
 ($\forall$ *out trace-table composition-table as query-idxs opening-query-idxs*
trace-roots trace-bs trace-final dg composition-roots composition-bs
composition-final trace-round-layers composition-round-layers.
accepted-with-fri-tables s out $\longrightarrow$
accepted-with-bound-tables s out trace-table composition-table as
query-idxs $\longrightarrow$
accepted-fri-opening-transcript s out trace-roots trace-bs trace-final
dg composition-roots composition-bs composition-final opening-query-idxs
trace-round-layers composition-round-layers $\longrightarrow$
 $\neg$ *trace-table-low-degree trace-table* $\longrightarrow$
trace-bs $\in$ *bad-sets trace-table*)

definition *composition-fri-with-tables-opening-algebraic-cover*

:: (*'f*, *'a*) *protocol-channel-scheme* $\Rightarrow$
 (*'f* $\Rightarrow$ *'f list* $\Rightarrow$ *'f list set*) $\Rightarrow$ *bool*

where

composition-fri-with-tables-opening-algebraic-cover s bad-sets $\longleftrightarrow$
 ($\forall$ *out trace-table composition-table as query-idxs opening-query-idxs*
trace-roots trace-bs trace-final dg composition-roots composition-bs
composition-final trace-round-layers composition-round-layers.
accepted-with-fri-tables s out $\longrightarrow$
accepted-with-bound-tables s out trace-table composition-table as
query-idxs $\longrightarrow$
accepted-fri-opening-transcript s out trace-roots trace-bs trace-final
dg composition-roots composition-bs composition-final opening-query-idxs
trace-round-layers composition-round-layers $\longrightarrow$
trace-table-low-degree trace-table $\longrightarrow$
 $\neg$ *composition-table-low-degree maxDegree composition-table* $\longrightarrow$
composition-bs $\in$ *bad-sets dg composition-table*)

definition *trace-fri-with-tables-bad-sets*

:: (*'f*, *'a*) *protocol-channel-scheme* $\Rightarrow$ *'f list* $\Rightarrow$ *'f list set*

where

trace-fri-with-tables-bad-sets s trace-table =
 {*trace-bs*.

$\exists$ out composition-table as query-idxs opening-query-idxs
 trace-roots trace-final dg composition-roots composition-bs
 composition-final trace-round-layers composition-round-layers.
 accepted-with-fri-tables s out $\wedge$
 accepted-with-bound-tables s out trace-table composition-table as
 query-idxs $\wedge$
 accepted-fri-opening-transcript s out trace-roots trace-bs
 trace-final dg composition-roots composition-bs
 composition-final opening-query-idxs trace-round-layers
 composition-round-layers $\wedge$
 $\neg$ trace-table-low-degree trace-table}

definition *composition-fri-with-tables-bad-sets*

$:: ('f, 'a) \text{ protocol-channel-scheme} \Rightarrow 'f \Rightarrow 'f \text{ list} \Rightarrow 'f \text{ list set}$

where

composition-fri-with-tables-bad-sets s dg composition-table =
 {composition-bs.

$\exists$ out trace-table as query-idxs opening-query-idxs trace-roots
 trace-bs trace-final composition-roots composition-final
 trace-round-layers composition-round-layers.

accepted-with-fri-tables s out $\wedge$

accepted-with-bound-tables s out trace-table composition-table as
 query-idxs $\wedge$

accepted-fri-opening-transcript s out trace-roots trace-bs
 trace-final dg composition-roots composition-bs composition-final
 opening-query-idxs trace-round-layers composition-round-layers $\wedge$

trace-table-low-degree trace-table $\wedge$

$\neg$ composition-table-low-degree maxDegree composition-table}

lemma *trace-fri-with-tables-bad-sets-cover:*

trace-fri-with-tables-opening-algebraic-cover s

(*trace-fri-with-tables-bad-sets* s)

unfolding *trace-fri-with-tables-opening-algebraic-cover-def*

trace-fri-with-tables-bad-sets-def

by blast

lemma *composition-fri-with-tables-bad-sets-cover:*

composition-fri-with-tables-opening-algebraic-cover s

(*composition-fri-with-tables-bad-sets* s)

unfolding *composition-fri-with-tables-opening-algebraic-cover-def*

composition-fri-with-tables-bad-sets-def

by blast

lemma *trace-fri-with-tables-bad-sets-subset:*

trace-fri-with-tables-bad-sets s trace-table $\subseteq$

fri-challenge-space (ceil-log clength)

unfolding *trace-fri-with-tables-bad-sets-def*

using *accepted-fri-opening-transcript-challenges*

accepted-fri-challenges-trace-space

by blast

lemma *composition-fri-with-tables-bad-sets-subset*:
composition-fri-with-tables-bad-sets s dg composition-table $\subseteq$
fri-challenge-space (ceil-log (to-nat dg + 1))
unfolding composition-fri-with-tables-bad-sets-def
using accepted-fri-opening-transcript-challenges
accepted-fri-challenges-composition-space
by blast

lemma *trace-fri-with-tables-bad-sets-card-bound*:
card (trace-fri-with-tables-bad-sets s trace-table) $\leq$
CARD('f) ^ ceil-log clength

proof –
have card (trace-fri-with-tables-bad-sets s trace-table) $\leq$
card (fri-challenge-space (ceil-log clength))
by (rule card-mono[OF finite-fri-challenge-space])
(rule trace-fri-with-tables-bad-sets-subset)
also have ... = CARD('f) ^ ceil-log clength
by (rule card-fri-challenge-space)
finally show ?thesis .
qed

lemma *composition-fri-with-tables-bad-sets-card-bound*:
card (composition-fri-with-tables-bad-sets s dg composition-table) $\leq$
CARD('f) ^ ceil-log (to-nat dg + 1)

proof –
have card (composition-fri-with-tables-bad-sets s dg composition-table) $\leq$
card (fri-challenge-space (ceil-log (to-nat dg + 1)))
by (rule card-mono[OF finite-fri-challenge-space])
(rule composition-fri-with-tables-bad-sets-subset)
also have ... = CARD('f) ^ ceil-log (to-nat dg + 1)
by (rule card-fri-challenge-space)
finally show ?thesis .
qed

lemma *trace-fri-with-tables-bad-sets-fraction-le-one*:
nnreal (card (trace-fri-with-tables-bad-sets s trace-table)) /
nnreal (CARD('f) ^ ceil-log clength) ≤ 1

proof –
have nnreal (card (trace-fri-with-tables-bad-sets s trace-table)) /
nnreal (CARD('f) ^ ceil-log clength) $\leq$
nnreal (CARD('f) ^ ceil-log clength) /
nnreal (CARD('f) ^ ceil-log clength)
by (rule nnreal-nat-divide-right-mono)
(rule trace-fri-with-tables-bad-sets-card-bound)
also have ... = 1
by (rule nnreal-nat-divide-self) simp
finally show ?thesis .

qed

lemma *composition-fri-with-tables-bad-sets-fraction-le-one:*

*nnreal (card (composition-fri-with-tables-bad-sets s dg composition-table)) /
nnreal (CARD('f) ^ ceil-log (to-nat dg + 1)) ≤ 1*

proof –

have *nnreal*

*(card (composition-fri-with-tables-bad-sets s dg composition-table)) /
nnreal (CARD('f) ^ ceil-log (to-nat dg + 1)) ≤
nnreal (CARD('f) ^ ceil-log (to-nat dg + 1)) /
nnreal (CARD('f) ^ ceil-log (to-nat dg + 1))*

by *(rule nnreal-nat-divide-right-mono)*

(rule composition-fri-with-tables-bad-sets-card-bound)

also have *... = 1*

by *(rule nnreal-nat-divide-self) simp*

finally show *?thesis .*

qed

definition *trace-fri-bad-with-tables-challenge-cover*

*:: ('f, 'a) protocol-channel-scheme ⇒
('f list ⇒ 'f list set) ⇒ bool*

where

*trace-fri-bad-with-tables-challenge-cover s bad-sets ⟷
(∀ out ∈ set-dist (execute verify-monad s).
trace-fri-bad-with-tables s out ⟶
trace-fri-challenge-set-hit s bad-sets out)*

definition *composition-fri-bad-with-tables-challenge-cover*

*:: ('f, 'a) protocol-channel-scheme ⇒
('f ⇒ 'f list ⇒ 'f list set) ⇒ bool*

where

*composition-fri-bad-with-tables-challenge-cover s bad-sets ⟷
(∀ out ∈ set-dist (execute verify-monad s).
composition-fri-bad-with-tables s out ⟶
composition-fri-challenge-set-hit s bad-sets out)*

lemma *trace-fri-with-tables-opening-algebraic-cover-imp-bad-challenge-cover:*

assumes *cover:*

trace-fri-with-tables-opening-algebraic-cover s bad-sets

shows *trace-fri-bad-with-tables-challenge-cover s bad-sets*

unfolding *trace-fri-bad-with-tables-challenge-cover-def*

proof *(intro ballI impI)*

fix *out :: (unit list × ('f, 'a) protocol-channel-scheme) option*

assume *outcome: out ∈ set-dist (execute verify-monad s)*

and *bad: trace-fri-bad-with-tables s out*

then have *with-tables: accepted-with-fri-tables s out*

and *trace-bad: trace-fri-bad s out*

unfolding *trace-fri-bad-with-tables-def* **by** *simp-all*

from *trace-bad* **obtain** *trace-table composition-table as query-idxs* **where** *bound:*

```

    accepted-with-bound-tables s out trace-table composition-table as
      query-idxs
  and not-low:  $\neg$  trace-table-low-degree trace-table
  unfolding trace-fri-bad-def by blast
from accepted-with-bound-tables-imp-accepted[OF bound]
obtain result final-state where out-eq: out = Some (result, final-state)
  unfolding accepted-def by (cases out) auto
have verify-out:
  Some (result, final-state)  $\in$  set-dist (execute verify-monad s)
  using outcome out-eq by simp
from verify-monad-accepted-fri-opening-transcript[OF verify-out]
obtain trace-roots trace-bs trace-final dg composition-roots composition-bs
  composition-final query-idxs' trace-round-layers composition-round-layers
where openings:
  accepted-fri-opening-transcript s (Some (result, final-state))
  trace-roots trace-bs trace-final dg composition-roots composition-bs
  composition-final query-idxs' trace-round-layers
  composition-round-layers
by blast
have trace-bs-bad: trace-bs  $\in$  bad-sets trace-table
  using cover with-tables bound openings not-low out-eq
  unfolding trace-fri-with-tables-opening-algebraic-cover-def by blast
have challenges:
  accepted-fri-challenges s out trace-bs dg composition-bs
  using accepted-fri-opening-transcript-challenges[OF openings] out-eq
  by simp
show trace-fri-challenge-set-hit s bad-sets out
  unfolding trace-fri-challenge-set-hit-def
  using bound challenges not-low trace-bs-bad by blast
qed

lemma composition-fri-with-tables-opening-algebraic-cover-imp-bad-challenge-cover:
  assumes cover:
    composition-fri-with-tables-opening-algebraic-cover s bad-sets
  shows composition-fri-bad-with-tables-challenge-cover s bad-sets
  unfolding composition-fri-bad-with-tables-challenge-cover-def
proof (intro ballI impI)
  fix out :: (unit list  $\times$  ('f, 'a) protocol-channel-scheme) option
  assume outcome: out  $\in$  set-dist (execute verify-monad s)
  and bad: composition-fri-bad-with-tables s out
  then have with-tables: accepted-with-fri-tables s out
  and composition-bad: composition-fri-bad s out
  unfolding composition-fri-bad-with-tables-def by simp-all
from composition-bad obtain trace-table composition-table as query-idxs
  where bound:
    accepted-with-bound-tables s out trace-table composition-table as
      query-idxs
  and trace-low: trace-table-low-degree trace-table
  and not-low:  $\neg$  composition-table-low-degree maxDegree composition-table

```

```

    unfolding composition-fri-bad-def by blast
  from accepted-with-bound-tables-imp-accepted[OF bound]
  obtain result final-state where out-eq: out = Some (result, final-state)
    unfolding accepted-def by (cases out) auto
  have verify-out:
    Some (result, final-state) ∈ set-dist (execute verify-monad s)
    using outcome out-eq by simp
  from verify-monad-accepted-fri-opening-transcript[OF verify-out]
  obtain trace-roots trace-bs trace-final dg composition-roots composition-bs
    composition-final query-idxs' trace-round-layers composition-round-layers
  where openings:
    accepted-fri-opening-transcript s (Some (result, final-state))
    trace-roots trace-bs trace-final dg composition-roots composition-bs
    composition-final query-idxs' trace-round-layers
    composition-round-layers
  by blast
  have composition-bs-bad:
    composition-bs ∈ bad-sets dg composition-table
    using cover with-tables bound openings trace-low not-low out-eq
    unfolding composition-fri-with-tables-opening-algebraic-cover-def by blast
  have challenges:
    accepted-fri-challenges s out trace-bs dg composition-bs
    using accepted-fri-opening-transcript-challenges[OF openings] out-eq
    by simp
  show composition-fri-challenge-set-hit s bad-sets out
    unfolding composition-fri-challenge-set-hit-def
    using bound challenges trace-low not-low composition-bs-bad by blast
qed

lemma wp-trace-fri-challenge-set-hit-bound-if-root-candidate-unique:
  assumes future: trace-fri-future-fresh s
  and unique:
     $\bigwedge_{fr.} \exists \text{trace-table.}$ 
    trace-fri-root-trace-table-candidates s fr  $\subseteq \{\text{trace-table}\}$ 
  and bounded: trace-fri-bad-challenge-sets-bounded bad-sets
  shows wp-event verify-monad (trace-fri-challenge-set-hit s bad-sets) s  $\leq$ 
    trace-fri-error
proof -
  have union-bound:
     $\bigwedge_{fr.} \text{nnreal}$ 
    (card (trace-fri-root-union-bad-sets s bad-sets fr)) /
    nnreal (CARD('f)  $\wedge$  ceil-log clength)  $\leq$  trace-fri-error
  proof -
    fix fr
    show nnreal
      (card (trace-fri-root-union-bad-sets s bad-sets fr)) /
      nnreal (CARD('f)  $\wedge$  ceil-log clength)  $\leq$  trace-fri-error
    by (rule trace-fri-root-union-bad-sets-fraction-bound-if-unique-candidate

```

```

      [OF unique])
    (use bounded in
      ⟨auto simp: trace-fri-bad-challenge-sets-bounded-def⟩)
  qed
  show ?thesis
    by (rule wp-trace-fri-challenge-set-hit-bound-via-root-union
      [OF future union-bound])
  qed

lemma wp-trace-fri-challenge-set-hit-bound-if-supported-root-candidate-unique:
  assumes future: trace-fri-future-fresh s
  and unique:
     $\bigwedge fr.$ 
     $\exists trace-table.$ 
     $trace-fri-supported-root-trace-table-candidates\ s\ fr \subseteq \{trace-table\}$ 
  and bounded: trace-fri-bad-challenge-sets-bounded bad-sets
  shows wp-event verify-monad (trace-fri-challenge-set-hit s bad-sets) s  $\leq$ 
    trace-fri-error
  proof -
    have union-bound:
       $\bigwedge fr.$  nnreal
      (card (trace-fri-supported-root-union-bad-sets s bad-sets fr)) /
      nnreal (CARD('f) ^ ceil-log clength)  $\leq$  trace-fri-error
    proof -
      fix fr
      show nnreal
        (card (trace-fri-supported-root-union-bad-sets s bad-sets fr)) /
        nnreal (CARD('f) ^ ceil-log clength)  $\leq$  trace-fri-error
      by (rule
        trace-fri-supported-root-union-bad-sets-fraction-bound-if-unique-candidate
        [OF unique])
      (use bounded in
        ⟨auto simp: trace-fri-bad-challenge-sets-bounded-def⟩)
    qed
    show ?thesis
      by (rule wp-trace-fri-challenge-set-hit-bound-via-supported-root-union
        [OF future union-bound])
  qed

lemma wp-trace-fri-challenge-set-hit-bound-if-no-supported-pairwise-merkle-bad:
  assumes future: trace-fri-future-fresh s
  and no-bad:  $\neg trace-fri-supported-pairwise-merkle-bad\ s$ 
  and bounded: trace-fri-bad-challenge-sets-bounded bad-sets
  shows wp-event verify-monad (trace-fri-challenge-set-hit s bad-sets) s  $\leq$ 
    trace-fri-error
  proof (rule wp-trace-fri-challenge-set-hit-bound-if-supported-root-candidate-unique
    [OF future - bounded])
    fix fr

```

show
 $\exists \text{ trace-table.}$
 $\text{trace-fri-supported-root-trace-table-candidates } s \text{ fr} \subseteq$
 $\{\text{trace-table}\}$
by (rule
 $\text{trace-fri-supported-root-candidate-unique-if-no-global-pairwise-merkle-bad}$
 $[\text{OF no-bad}]$)
qed

lemma *wp-composition-fri-challenge-set-hit-bound-if-degree-candidate-unique:*
assumes *future: composition-fri-future-fresh s*
and *unique:*
 $\bigwedge dg.$
 $\exists \text{ composition-table.}$
 $\text{composition-fri-degree-table-candidates } s \text{ dg} \subseteq$
 $\{\text{composition-table}\}$
and *bounded: composition-fri-bad-challenge-sets-bounded bad-sets*
shows
 $\text{wp-event verify-monad } (\text{composition-fri-challenge-set-hit } s \text{ bad-sets}) \text{ } s \leq$
 $\text{composition-fri-error}$

proof –
have *union-bound:*
 $\bigwedge dg. \text{ nnreal}$
 $(\text{card } (\text{composition-fri-degree-union-bad-sets } s \text{ bad-sets } dg)) /$
 $\text{nnreal } (\text{CARD}('f) \wedge \text{ceil-log } (\text{to-nat } dg + 1)) \leq$
 $\text{composition-fri-error}$
proof –
fix *dg*
show *nnreal*
 $(\text{card } (\text{composition-fri-degree-union-bad-sets } s \text{ bad-sets } dg)) /$
 $\text{nnreal } (\text{CARD}('f) \wedge \text{ceil-log } (\text{to-nat } dg + 1)) \leq$
 $\text{composition-fri-error}$
by (rule
 $\text{composition-fri-degree-union-bad-sets-fraction-bound-if-unique-candidate}$
 $[\text{OF unique}]$)
(use *bounded* **in**
 $\langle \text{auto simp: composition-fri-bad-challenge-sets-bounded-def} \rangle$)
qed
show *?thesis*
by (rule *wp-composition-fri-challenge-set-hit-bound-via-degree-union*
 $[\text{OF future union-bound}]$)
qed

lemma *wp-composition-fri-challenge-set-hit-bound-if-supported-root-candidate-unique:*
assumes *future: composition-fri-future-fresh s*
and *unique:*
 $\bigwedge fr \text{ f-fri-roots f-final as dg composition-fri-roots.}$
 $\exists \text{ composition-table.}$
 $\text{composition-fri-supported-root-composition-table-candidates } s \text{ fr}$

```

      f-fri-roots f-final as dg composition-fri-roots  $\subseteq$ 
      {composition-table}
    and bounded: composition-fri-bad-challenge-sets-bounded bad-sets
  shows
    wp-event verify-monad (composition-fri-challenge-set-hit s bad-sets) s  $\leq$ 
      composition-fri-error
  proof -
    have union-bound:
       $\bigwedge$  fr f-fri-roots f-final as dg composition-fri-roots.
      nnreal
      (card
        (composition-fri-supported-root-union-bad-sets s bad-sets fr
          f-fri-roots f-final as dg composition-fri-roots)) /
      nnreal (CARD('f)  $\wedge$  ceil-log (to-nat dg + 1))  $\leq$ 
      composition-fri-error
    proof -
      fix fr f-fri-roots f-final as dg composition-fri-roots
      show nnreal
        (card
          (composition-fri-supported-root-union-bad-sets s bad-sets fr
            f-fri-roots f-final as dg composition-fri-roots)) /
        nnreal (CARD('f)  $\wedge$  ceil-log (to-nat dg + 1))  $\leq$ 
        composition-fri-error
      by (rule
        composition-fri-supported-root-union-bad-sets-fraction-bound-if-unique-candidate
        [OF unique])
      (use bounded in
         $\langle$ auto simp: composition-fri-bad-challenge-sets-bounded-def $\rangle$ )
    qed
  show ?thesis
    by (rule
      wp-composition-fri-challenge-set-hit-bound-via-supported-root-union
      [OF future union-bound])
  qed

lemma wp-composition-fri-challenge-set-hit-bound-if-no-supported-pairwise-merkle-bad:
  assumes future: composition-fri-future-fresh s
  and no-bad:  $\neg$  composition-fri-supported-pairwise-merkle-bad s
  and bounded: composition-fri-bad-challenge-sets-bounded bad-sets
  shows
    wp-event verify-monad (composition-fri-challenge-set-hit s bad-sets) s  $\leq$ 
      composition-fri-error
  proof (rule
    wp-composition-fri-challenge-set-hit-bound-if-supported-root-candidate-unique
    [OF future - bounded])
    fix fr f-fri-roots f-final as dg composition-fri-roots
    show
       $\exists$  composition-table.
      composition-fri-supported-root-composition-table-candidates s fr
  
```

$f\text{-fri-roots } f\text{-final as dg composition-fri-roots } \subseteq$
 $\{ \text{composition-table} \}$
by (rule
 $\text{composition-fri-supported-root-candidate-unique-if-no-global-pairwise-merkle-bad}$
 $[OF \text{ no-bad}]$)
qed

definition *trace-fri-opening-algebraic-cover*

$:: ('f, 'a) \text{ protocol-channel-scheme } \Rightarrow$
 $('f \text{ list } \Rightarrow 'f \text{ list set}) \Rightarrow \text{bool}$

where

$\text{trace-fri-opening-algebraic-cover } s \text{ bad-sets } \longleftrightarrow$
 $(\forall \text{ out trace-table composition-table as query-idxs opening-query-idxs}$
 $\text{ trace-roots trace-bs trace-final dg composition-roots composition-bs}$
 $\text{ composition-final trace-round-layers composition-round-layers.}$
 $\text{accepted-with-bound-tables } s \text{ out trace-table composition-table as query-idxs } \longrightarrow$
 $\text{accepted-fri-opening-transcript } s \text{ out trace-roots trace-bs trace-final}$
 $\text{ dg composition-roots composition-bs composition-final opening-query-idxs}$
 $\text{ trace-round-layers composition-round-layers } \longrightarrow$
 $\neg \text{ trace-table-low-degree trace-table } \longrightarrow$
 $\text{ trace-bs } \in \text{ bad-sets trace-table})$

definition *composition-fri-opening-algebraic-cover*

$:: ('f, 'a) \text{ protocol-channel-scheme } \Rightarrow$
 $('f \Rightarrow 'f \text{ list } \Rightarrow 'f \text{ list set}) \Rightarrow \text{bool}$

where

$\text{composition-fri-opening-algebraic-cover } s \text{ bad-sets } \longleftrightarrow$
 $(\forall \text{ out trace-table composition-table as query-idxs opening-query-idxs}$
 $\text{ trace-roots trace-bs trace-final dg composition-roots composition-bs}$
 $\text{ composition-final trace-round-layers composition-round-layers.}$
 $\text{accepted-with-bound-tables } s \text{ out trace-table composition-table as query-idxs } \longrightarrow$
 $\text{accepted-fri-opening-transcript } s \text{ out trace-roots trace-bs trace-final}$
 $\text{ dg composition-roots composition-bs composition-final opening-query-idxs}$
 $\text{ trace-round-layers composition-round-layers } \longrightarrow$
 $\text{ trace-table-low-degree trace-table } \longrightarrow$
 $\neg \text{ composition-table-low-degree maxDegree composition-table } \longrightarrow$
 $\text{ composition-bs } \in \text{ bad-sets dg composition-table})$

definition *trace-fri-opening-reduction-assumption*

$:: ('f, 'a) \text{ protocol-channel-scheme } \Rightarrow \text{bool}$

where

$\text{trace-fri-opening-reduction-assumption } s \longleftrightarrow$
 $(\exists \text{ bad-sets.}$
 $\text{ trace-fri-opening-algebraic-cover } s \text{ bad-sets } \wedge$
 $\text{ trace-fri-bad-challenge-sets-bounded bad-sets})$

definition *composition-fri-opening-reduction-assumption*

$:: ('f, 'a) \text{ protocol-channel-scheme } \Rightarrow \text{bool}$

where

composition-fri-opening-reduction-assumption s $\longleftrightarrow$
 $(\exists \text{ bad-sets.}$
composition-fri-opening-algebraic-cover s bad-sets $\wedge$
composition-fri-bad-challenge-sets-bounded bad-sets)

lemma *trace-fri-opening-reductionI*:
assumes *trace-fri-opening-algebraic-cover s bad-sets*
and *trace-fri-bad-challenge-sets-bounded bad-sets*
shows *trace-fri-opening-reduction-assumption s*
using *assms* **unfolding** *trace-fri-opening-reduction-assumption-def* **by** *blast*

lemma *trace-fri-opening-reductionE*:
assumes *trace-fri-opening-reduction-assumption s*
obtains *bad-sets*
where *trace-fri-opening-algebraic-cover s bad-sets*
and *trace-fri-bad-challenge-sets-bounded bad-sets*
using *assms* **unfolding** *trace-fri-opening-reduction-assumption-def* **by** *blast*

lemma *composition-fri-opening-reductionI*:
assumes *composition-fri-opening-algebraic-cover s bad-sets*
and *composition-fri-bad-challenge-sets-bounded bad-sets*
shows *composition-fri-opening-reduction-assumption s*
using *assms* **unfolding** *composition-fri-opening-reduction-assumption-def* **by** *blast*

lemma *composition-fri-opening-reductionE*:
assumes *composition-fri-opening-reduction-assumption s*
obtains *bad-sets*
where *composition-fri-opening-algebraic-cover s bad-sets*
and *composition-fri-bad-challenge-sets-bounded bad-sets*
using *assms* **unfolding** *composition-fri-opening-reduction-assumption-def*
by *blast*

lemma *trace-fri-multiround-bad-sets-bounded*:
assumes *frac*:
 $\wedge \text{trace-table.}$
 $\text{nnreal (card (trace-fri-multiround-bad-sets bad trace-table)) /}$
 $\text{nnreal (CARD('f) ^ ceil-log clength) } \leq \text{trace-fri-error}$
shows *trace-fri-bad-challenge-sets-bounded*
 $(\text{trace-fri-multiround-bad-sets bad})$
unfolding *trace-fri-bad-challenge-sets-bounded-def*
proof (*intro allI conjI*)
fix *trace-table*
show *trace-fri-multiround-bad-sets bad trace-table* $\subseteq$
 $\text{fri-challenge-space (ceil-log clength)}$
by (*rule trace-fri-multiround-bad-sets-subset*)
show $\text{nnreal (card (trace-fri-multiround-bad-sets bad trace-table)) /}$
 $\text{nnreal (CARD('f) ^ ceil-log clength) } \leq \text{trace-fri-error}$
by (*rule frac*)
qed

lemma *composition-fri-multiround-bad-sets-bounded*:
assumes *frac*:
 $\bigwedge dg$ *composition-table*.
 $nnreal$ (*card*
 $(composition-fri-multiround-bad-sets\ bad\ dg\ composition-table)) /$
 $nnreal$ ($CARD('f) \wedge \text{ceil-log } (to-nat\ dg + 1) \leq$
 $composition-fri-error$
shows *composition-fri-bad-challenge-sets-bounded*
 $(composition-fri-multiround-bad-sets\ bad)$
unfolding *composition-fri-bad-challenge-sets-bounded-def*
proof (*intro allI conjI*)
fix *dg composition-table*
show *composition-fri-multiround-bad-sets bad dg composition-table* $\subseteq$
 $fri-challenge-space\ (\text{ceil-log } (to-nat\ dg + 1))$
by (*rule composition-fri-multiround-bad-sets-subset*)
show $nnreal$ (*card*
 $(composition-fri-multiround-bad-sets\ bad\ dg\ composition-table)) /$
 $nnreal$ ($CARD('f) \wedge \text{ceil-log } (to-nat\ dg + 1) \leq$
 $composition-fri-error$
by (*rule frac*)
qed

lemma *trace-fri-opening-reduction-from-multiround-bad-sets*:
assumes *cover*:
 $trace-fri-opening-algebraic-cover\ s$
 $(trace-fri-multiround-bad-sets\ bad)$
and *frac*:
 $\bigwedge trace-table.$
 $nnreal$ (*card* ($trace-fri-multiround-bad-sets\ bad\ trace-table$)) /
 $nnreal$ ($CARD('f) \wedge \text{ceil-log } clength \leq trace-fri-error$
shows *trace-fri-opening-reduction-assumption s*
by (*rule trace-fri-opening-reductionI*
 $[OF\ cover\ trace-fri-multiround-bad-sets-bounded[OF\ frac]]$)

lemma *composition-fri-opening-reduction-from-multiround-bad-sets*:
assumes *cover*:
 $composition-fri-opening-algebraic-cover\ s$
 $(composition-fri-multiround-bad-sets\ bad)$
and *frac*:
 $\bigwedge dg$ *composition-table*.
 $nnreal$ (*card*
 $(composition-fri-multiround-bad-sets\ bad\ dg\ composition-table)) /$
 $nnreal$ ($CARD('f) \wedge \text{ceil-log } (to-nat\ dg + 1) \leq$
 $composition-fri-error$
shows *composition-fri-opening-reduction-assumption s*
by (*rule composition-fri-opening-reductionI*
 $[OF\ cover\ composition-fri-multiround-bad-sets-bounded[OF\ frac]]$)

end

end

theory *Soundness-Reductions-Opening*
 imports *Soundness-Reductions-Partial*
begin

FRI opening algebraic covers and final reduction bounds.

context *soundness*
begin

The multi-round bad-set constructors above isolate the proof-only part of the FRI reductions: once an algebraic cover and the corresponding error-fraction bound are available, the existing opening-reduction assumptions follow directly. The current transcript predicate records authenticated local openings, but it does not bind complete intermediate FRI layer tables. Thus the algebraic cover is still an external FRI/table-binding theorem target unless the protocol model is extended with such tables or an equivalent binding interface.

definition *trace-fri-bad-challenge-cover*
 $:: ('f, 'a) \text{ protocol-channel-scheme} \Rightarrow$
 $('f \text{ list} \Rightarrow 'f \text{ list set}) \Rightarrow \text{bool}$
 where
 $\text{trace-fri-bad-challenge-cover } s \text{ bad-sets} \longleftrightarrow$
 $(\forall \text{ out} \in \text{set-dist } (\text{execute verify-monad } s).$
 $\text{trace-fri-bad } s \text{ out} \longrightarrow$
 $\text{trace-fri-challenge-set-hit } s \text{ bad-sets out})$

definition *composition-fri-bad-challenge-cover*
 $:: ('f, 'a) \text{ protocol-channel-scheme} \Rightarrow$
 $('f \Rightarrow 'f \text{ list} \Rightarrow 'f \text{ list set}) \Rightarrow \text{bool}$
 where
 $\text{composition-fri-bad-challenge-cover } s \text{ bad-sets} \longleftrightarrow$
 $(\forall \text{ out} \in \text{set-dist } (\text{execute verify-monad } s).$
 $\text{composition-fri-bad } s \text{ out} \longrightarrow$
 $\text{composition-fri-challenge-set-hit } s \text{ bad-sets out})$

lemma *trace-fri-opening-algebraic-cover-imp-bad-challenge-cover:*

assumes *cover: trace-fri-opening-algebraic-cover s bad-sets*

shows *trace-fri-bad-challenge-cover s bad-sets*

unfolding *trace-fri-bad-challenge-cover-def*

proof (*intro ballI impI*)

fix *out* $:: (\text{unit list} \times ('f, 'a) \text{ protocol-channel-scheme}) \text{ option}$

assume *outcome: out* $\in \text{set-dist } (\text{execute verify-monad } s)$

and *bad: trace-fri-bad s out*

then obtain *trace-table composition-table as query-idxs* **where** *bound:*

accepted-with-bound-tables s out trace-table composition-table as query-idxs

```

and not-low:  $\neg$  trace-table-low-degree trace-table
unfolding trace-fri-bad-def by blast
from accepted-with-bound-tables-imp-accepted[OF bound]
obtain result final-state where out-eq: out = Some (result, final-state)
unfolding accepted-def by (cases out) auto
have verify-out:
  Some (result, final-state)  $\in$  set-dist (execute verify-monad s)
using outcome out-eq by simp
from verify-monad-accepted-fri-opening-transcript[OF verify-out]
obtain trace-roots trace-bs trace-final dg composition-roots composition-bs
  composition-final query-idxs' trace-round-layers composition-round-layers
where openings:
  accepted-fri-opening-transcript s (Some (result, final-state))
  trace-roots trace-bs trace-final dg composition-roots composition-bs
  composition-final query-idxs' trace-round-layers composition-round-layers
by blast
have trace-bs-bad: trace-bs  $\in$  bad-sets trace-table
using cover bound openings not-low out-eq
unfolding trace-fri-opening-algebraic-cover-def by blast
have challenges:
  accepted-fri-challenges s out trace-bs dg composition-bs
using accepted-fri-opening-transcript-challenges[OF openings] out-eq by simp
show trace-fri-challenge-set-hit s bad-sets out
unfolding trace-fri-challenge-set-hit-def
using bound challenges not-low trace-bs-bad by blast
qed

```

```

lemma composition-fri-opening-algebraic-cover-imp-bad-challenge-cover:
assumes cover: composition-fri-opening-algebraic-cover s bad-sets
shows composition-fri-bad-challenge-cover s bad-sets
unfolding composition-fri-bad-challenge-cover-def
proof (intro ballI impI)
fix out :: (unit list  $\times$  ('f, 'a) protocol-channel-scheme) option
assume outcome: out  $\in$  set-dist (execute verify-monad s)
and bad: composition-fri-bad s out
then obtain trace-table composition-table as query-idxs where bound:
  accepted-with-bound-tables s out trace-table composition-table as query-idxs
and trace-low: trace-table-low-degree trace-table
and not-low:  $\neg$  composition-table-low-degree maxDegree composition-table
unfolding composition-fri-bad-def by blast
from accepted-with-bound-tables-imp-accepted[OF bound]
obtain result final-state where out-eq: out = Some (result, final-state)
unfolding accepted-def by (cases out) auto
have verify-out:
  Some (result, final-state)  $\in$  set-dist (execute verify-monad s)
using outcome out-eq by simp
from verify-monad-accepted-fri-opening-transcript[OF verify-out]
obtain trace-roots trace-bs trace-final dg composition-roots composition-bs
  composition-final query-idxs' trace-round-layers composition-round-layers

```

where *openings*:
accepted-fri-opening-transcript s (Some (result, final-state))
trace-roots trace-bs trace-final dg composition-roots composition-bs
composition-final query-idxs' trace-round-layers composition-round-layers
by *blast*
have *composition-bs-bad: composition-bs ∈ bad-sets dg composition-table*
using *cover bound openings trace-low not-low out-eq*
unfolding *composition-fri-opening-algebraic-cover-def* **by** *blast*
have *challenges*:
accepted-fri-challenges s out trace-bs dg composition-bs
using *accepted-fri-opening-transcript-challenges[OF openings] out-eq* **by** *simp*
show *composition-fri-challenge-set-hit s bad-sets out*
unfolding *composition-fri-challenge-set-hit-def*
using *bound challenges trace-low not-low composition-bs-bad* **by** *blast*
qed

definition *trace-fri-challenge-freshness-assumption*
 $:: (f, 'a) \text{ protocol-channel-scheme} \Rightarrow \text{bool}$
where
trace-fri-challenge-freshness-assumption s $\longleftrightarrow$
($\forall$ bad-sets.
trace-fri-bad-challenge-sets-bounded bad-sets $\longrightarrow$
wp-event verify-monad (trace-fri-challenge-set-hit s bad-sets) s $\leq$
trace-fri-error)

definition *composition-fri-challenge-freshness-assumption*
 $:: (f, 'a) \text{ protocol-channel-scheme} \Rightarrow \text{bool}$
where
composition-fri-challenge-freshness-assumption s $\longleftrightarrow$
($\forall$ bad-sets.
composition-fri-bad-challenge-sets-bounded bad-sets $\longrightarrow$
wp-event verify-monad
(composition-fri-challenge-set-hit s bad-sets) s $\leq$
composition-fri-error)

definition *random-oracle-freshness-assumption*
 $:: (f, 'a) \text{ protocol-channel-scheme} \Rightarrow \text{bool}$
where
random-oracle-freshness-assumption s $\longleftrightarrow$
alpha-challenge-freshness-assumption s $\wedge$
query-index-freshness-assumption s $\wedge$
trace-fri-challenge-freshness-assumption s $\wedge$
composition-fri-challenge-freshness-assumption s

lemma *random-oracle-freshness-alpha*:
assumes *random-oracle-freshness-assumption s*
shows *alpha-challenge-freshness-assumption s*
using *assms* **unfolding** *random-oracle-freshness-assumption-def* **by** *simp*

```

lemma random-oracle-freshness-query:
  assumes random-oracle-freshness-assumption s
  shows query-index-freshness-assumption s
  using assms unfolding random-oracle-freshness-assumption-def by simp

lemma random-oracle-freshness-trace-fri:
  assumes random-oracle-freshness-assumption s
  shows trace-fri-challenge-freshness-assumption s
  using assms unfolding random-oracle-freshness-assumption-def by simp

lemma random-oracle-freshness-composition-fri:
  assumes random-oracle-freshness-assumption s
  shows composition-fri-challenge-freshness-assumption s
  using assms unfolding random-oracle-freshness-assumption-def by simp

lemma alpha-challenge-freshness-if-no-supported-pairwise-merkle-bad:
  assumes future: alpha-future-fresh s
  and no-bad:  $\neg$  alpha-supported-pairwise-merkle-bad s
  shows alpha-challenge-freshness-assumption s
  unfolding alpha-challenge-freshness-assumption-def
proof (intro allI impI)
  fix bad-sets :: 'f list  $\Rightarrow$  'f list set
  assume subset:
     $\forall$  trace-table. bad-sets trace-table  $\subseteq$  alpha-space
  and bounded:
     $\forall$  trace-table. nnreal (card (bad-sets trace-table)) /
       $\text{nnreal (card alpha-space)} \leq \text{composition-error-bound}$ 
  show wp-event verify-monad (alpha-bad-set-hit s bad-sets) s  $\leq$ 
    composition-error-bound
proof (rule wp-alpha-bad-set-hit-bound-via-supported-header-union
  [OF future])
  fix trace-table
  show bad-sets trace-table  $\subseteq$  alpha-space
    using subset by blast
next
  fix fr f-fri-roots f-final
  have unique:
     $\exists$  trace-table. alpha-header-supported-trace-table-candidates s fr f-fri-roots
      f-final  $\subseteq$  {trace-table}
  by (rule alpha-header-supported-candidate-unique-if-no-global-pairwise-merkle-bad
    [OF no-bad])
  show
     $\text{nnreal (card (alpha-header-supported-union-bad-sets s bad-sets fr f-fri-roots
      f-final)) / nnreal (card alpha-space)} \leq \text{composition-error-bound}$ 

```

by (rule alpha-header-supported-union-bad-sets-fraction-bound-if-unique-candidate
 [OF unique])
 (use subset bounded in blast)+
 qed
 qed

lemma trace-fri-challenge-freshness-if-no-supported-pairwise-merkle-bad:
 assumes future: trace-fri-future-fresh s
 and no-bad: $\neg$ trace-fri-supported-pairwise-merkle-bad s
 shows trace-fri-challenge-freshness-assumption s
 unfolding trace-fri-challenge-freshness-assumption-def
proof (intro allI impI)
 fix bad-sets :: 'f list $\Rightarrow$ 'f list set
 assume bounded: trace-fri-bad-challenge-sets-bounded bad-sets
 show wp-event verify-monad (trace-fri-challenge-set-hit s bad-sets) s $\leq$
 trace-fri-error
 by (rule
 wp-trace-fri-challenge-set-hit-bound-if-no-supported-pairwise-merkle-bad
 [OF future no-bad bounded])
 qed

lemma composition-fri-challenge-freshness-if-no-supported-pairwise-merkle-bad:
 assumes future: composition-fri-future-fresh s
 and no-bad: $\neg$ composition-fri-supported-pairwise-merkle-bad s
 shows composition-fri-challenge-freshness-assumption s
 unfolding composition-fri-challenge-freshness-assumption-def
proof (intro allI impI)
 fix bad-sets :: 'f $\Rightarrow$ 'f list $\Rightarrow$ 'f list set
 assume bounded: composition-fri-bad-challenge-sets-bounded bad-sets
 show
 wp-event verify-monad
 (composition-fri-challenge-set-hit s bad-sets) s $\leq$
 composition-fri-error
 by (rule
 wp-composition-fri-challenge-set-hit-bound-if-no-supported-pairwise-merkle-bad
 [OF future no-bad bounded])
 qed

lemma verifier-initial-alpha-challenge-freshness-if-no-supported-pairwise-merkle-bad:
 assumes no-bad:
 $\neg$ alpha-supported-pairwise-merkle-bad (verifier-initial-state tr)
 shows alpha-challenge-freshness-assumption (verifier-initial-state tr)
 by (rule alpha-challenge-freshness-if-no-supported-pairwise-merkle-bad)
 (use no-bad in simp-all)

lemma verifier-initial-trace-fri-challenge-freshness-if-no-supported-pairwise-merkle-bad:
 assumes no-bad:
 $\neg$ trace-fri-supported-pairwise-merkle-bad (verifier-initial-state tr)
 shows

trace-fri-challenge-freshness-assumption (*verifier-initial-state tr*)
by (*rule trace-fri-challenge-freshness-if-no-supported-pairwise-merkle-bad*)
(use no-bad in simp-all)

lemma *verifier-initial-composition-fri-challenge-freshness-if-no-supported-pairwise-merkle-bad*:
assumes *no-bad*:

$\neg$ *composition-fri-supported-pairwise-merkle-bad*
(verifier-initial-state tr)

shows

composition-fri-challenge-freshness-assumption
(verifier-initial-state tr)

by (*rule*

composition-fri-challenge-freshness-if-no-supported-pairwise-merkle-bad)
(use no-bad in simp-all)

lemma *alpha-challenge-freshness-if-no-pairwise-side-bad*:

assumes *future: alpha-future-fresh s*

and *no-collision: $\neg$ supported-hash-output-collision-possible s*

and *no-coupling: $\neg$ supported-pairwise-coupling-bad s*

shows *alpha-challenge-freshness-assumption s*

proof (*rule alpha-challenge-freshness-if-no-supported-pairwise-merkle-bad*
[OF future])

show $\neg$ *alpha-supported-pairwise-merkle-bad s*

by (*rule no-alpha-supported-pairwise-merkle-bad-if-no-pairwise-side-bad*
[OF no-collision no-coupling])

qed

lemma *query-index-freshness-if-no-pairwise-side-bad*:

assumes *future: query-future-fresh s*

and *raw-bound: query-index-raw-preimage-bound*

and *no-collision: $\neg$ supported-hash-output-collision-possible s*

and *no-coupling: $\neg$ supported-pairwise-coupling-bad s*

shows *query-index-freshness-assumption s*

proof (*rule query-index-freshness-if-no-query-supported-pairwise-merkle-bad*
[OF future raw-bound])

show $\neg$ *query-supported-pairwise-merkle-bad s*

by (*rule no-query-supported-pairwise-merkle-bad-if-no-pairwise-side-bad*
[OF no-collision no-coupling])

qed

lemma *trace-fri-challenge-freshness-if-no-pairwise-side-bad*:

assumes *future: trace-fri-future-fresh s*

and *no-collision: $\neg$ supported-hash-output-collision-possible s*

and *no-coupling: $\neg$ supported-pairwise-coupling-bad s*

shows *trace-fri-challenge-freshness-assumption s*

proof (*rule trace-fri-challenge-freshness-if-no-supported-pairwise-merkle-bad*
[OF future])

show $\neg$ *trace-fri-supported-pairwise-merkle-bad s*

by (*rule*

$no\text{-}trace\text{-}fri\text{-}supported\text{-}pairwise\text{-}merkle\text{-}bad\text{-}if\text{-}no\text{-}pairwise\text{-}side\text{-}bad$
 $[OF\ no\text{-}collision\ no\text{-}coupling])$
qed

lemma $composition\text{-}fri\text{-}challenge\text{-}freshness\text{-}if\text{-}no\text{-}pairwise\text{-}side\text{-}bad$:
assumes $future$: $composition\text{-}fri\text{-}future\text{-}fresh\ s$
and $no\text{-}collision$: $\neg supported\text{-}hash\text{-}output\text{-}collision\text{-}possible\ s$
and $no\text{-}coupling$: $\neg supported\text{-}pairwise\text{-}coupling\text{-}bad\ s$
shows $composition\text{-}fri\text{-}challenge\text{-}freshness\text{-}assumption\ s$
proof (rule
 $composition\text{-}fri\text{-}challenge\text{-}freshness\text{-}if\text{-}no\text{-}supported\text{-}pairwise\text{-}merkle\text{-}bad$
 $[OF\ future])$
show $\neg composition\text{-}fri\text{-}supported\text{-}pairwise\text{-}merkle\text{-}bad\ s$
by (rule
 $no\text{-}composition\text{-}fri\text{-}supported\text{-}pairwise\text{-}merkle\text{-}bad\text{-}if\text{-}no\text{-}pairwise\text{-}side\text{-}bad$
 $[OF\ no\text{-}collision\ no\text{-}coupling])$
qed

lemma $random\text{-}oracle\text{-}freshness\text{-}if\text{-}no\text{-}pairwise\text{-}side\text{-}bad$:
assumes $alpha\text{-}future$: $alpha\text{-}future\text{-}fresh\ s$
and $query\text{-}future$: $query\text{-}future\text{-}fresh\ s$
and $trace\text{-}future$: $trace\text{-}fri\text{-}future\text{-}fresh\ s$
and $composition\text{-}future$: $composition\text{-}fri\text{-}future\text{-}fresh\ s$
and $raw\text{-}bound$: $query\text{-}index\text{-}raw\text{-}preimage\text{-}bound$
and $no\text{-}collision$: $\neg supported\text{-}hash\text{-}output\text{-}collision\text{-}possible\ s$
and $no\text{-}coupling$: $\neg supported\text{-}pairwise\text{-}coupling\text{-}bad\ s$
shows $random\text{-}oracle\text{-}freshness\text{-}assumption\ s$
unfolding $random\text{-}oracle\text{-}freshness\text{-}assumption\text{-}def$
proof (intro conjI)
show $alpha\text{-}challenge\text{-}freshness\text{-}assumption\ s$
by (rule $alpha\text{-}challenge\text{-}freshness\text{-}if\text{-}no\text{-}pairwise\text{-}side\text{-}bad$
 $[OF\ alpha\text{-}future\ no\text{-}collision\ no\text{-}coupling])$
show $query\text{-}index\text{-}freshness\text{-}assumption\ s$
by (rule $query\text{-}index\text{-}freshness\text{-}if\text{-}no\text{-}pairwise\text{-}side\text{-}bad$
 $[OF\ query\text{-}future\ raw\text{-}bound\ no\text{-}collision\ no\text{-}coupling])$
show $trace\text{-}fri\text{-}challenge\text{-}freshness\text{-}assumption\ s$
by (rule $trace\text{-}fri\text{-}challenge\text{-}freshness\text{-}if\text{-}no\text{-}pairwise\text{-}side\text{-}bad$
 $[OF\ trace\text{-}future\ no\text{-}collision\ no\text{-}coupling])$
show $composition\text{-}fri\text{-}challenge\text{-}freshness\text{-}assumption\ s$
by (rule $composition\text{-}fri\text{-}challenge\text{-}freshness\text{-}if\text{-}no\text{-}pairwise\text{-}side\text{-}bad$
 $[OF\ composition\text{-}future\ no\text{-}collision\ no\text{-}coupling])$
qed

lemma $query\text{-}bad\text{-}bound\text{-}if\text{-}no\text{-}pairwise\text{-}side\text{-}bad$:
assumes $future$: $query\text{-}future\text{-}fresh\ s$
and $raw\text{-}bound$: $query\text{-}index\text{-}raw\text{-}preimage\text{-}bound$
and $no\text{-}collision$: $\neg supported\text{-}hash\text{-}output\text{-}collision\text{-}possible\ s$
and $no\text{-}coupling$: $\neg supported\text{-}pairwise\text{-}coupling\text{-}bad\ s$
shows $wp\text{-}event\ verify\text{-}monad\ (query\text{-}bad\ s)\ s \leq$

$nnreal\ rounds * query_error_bound$
proof –
have *query-fresh*: *query-index-freshness-assumption s*
by (rule *query-index-freshness-if-no-pairwise-side-bad*
 $[OF\ future\ raw_bound\ no_collision\ no_coupling]$)
show *?thesis*
by (rule *query-bad-bound-from-phase4B* $[OF\ query_fresh]$)
qed

lemma *query-bad-bound-if-no-pairwise-side-event-on-support*:
assumes *future*: *query-future-fresh s*
and *raw-bound*: *query-index-raw-preimage-bound*
and *no-side*:
 $\bigwedge out. out \in set_dist\ (execute\ verify_monad\ s) \implies$
 $\neg supported_pairwise_side_bad\ s\ out$
shows *wp-event verify-monad (query-bad s) s* $\leq$
 $nnreal\ rounds * query_error_bound$
proof –
have *no-collision*: $\neg supported_hash_output_collision_possible\ s$
by (rule
 $no_supported_pairwise_side_bad_on_support_imp_no_hash_collision_possible$
 $[OF\ no_side]$)
have *no-coupling*: $\neg supported_pairwise_coupling_bad\ s$
by (rule *no-supported-pairwise-side-bad-on-support-imp-no-coupling-bad*
 $[OF\ no_side]$)
show *?thesis*
by (rule *query-bad-bound-if-no-pairwise-side-bad*
 $[OF\ future\ raw_bound\ no_collision\ no_coupling]$)
qed

lemma *query-bad-bound-if-no-pairwise-side-event-on-support-from-sampler-wellformed*:
assumes *future*: *query-future-fresh s*
and *no-side*:
 $\bigwedge out. out \in set_dist\ (execute\ verify_monad\ s) \implies$
 $\neg supported_pairwise_side_bad\ s\ out$
shows *wp-event verify-monad (query-bad s) s* $\leq$
 $nnreal\ rounds * query_error_bound$
by (rule *query-bad-bound-if-no-pairwise-side-event-on-support*
 $[OF\ future\ query_index_raw_preimage_bound_from_sampler_wellformed$
 $no_side]$)

lemma *composition-bad-bound-if-no-pairwise-side-bad*:
assumes *wf*: *spec-degree-wellformed*
and *future*: *alpha-future-fresh s*
and *no-collision*: $\neg supported_hash_output_collision_possible\ s$
and *no-coupling*: $\neg supported_pairwise_coupling_bad\ s$
shows *wp-event verify-monad (composition-bad s) s* $\leq$
 $composition_error_bound$
proof –

```

have alpha-fresh: alpha-challenge-freshness-assumption s
  by (rule alpha-challenge-freshness-if-no-pairwise-side-bad
    [OF future no-collision no-coupling])
show ?thesis
  by (rule composition-bad-bound-from-phase4A[OF wf alpha-fresh])
qed

lemma composition-bad-bound-if-no-pairwise-side-event-on-support:
  assumes wf: spec-degree-wellformed
  and future: alpha-future-fresh s
  and no-side:
     $\bigwedge out. out \in \text{set-dist } (\text{execute verify-monad } s) \implies$ 
     $\neg \text{supported-pairwise-side-bad } s \text{ out}$ 
  shows wp-event verify-monad (composition-bad s) s  $\leq$ 
    composition-error-bound
proof -
  have no-collision:  $\neg \text{supported-hash-output-collision-possible } s$ 
    by (rule
      no-supported-pairwise-side-bad-on-support-imp-no-hash-collision-possible
      [OF no-side])
  have no-coupling:  $\neg \text{supported-pairwise-coupling-bad } s$ 
    by (rule no-supported-pairwise-side-bad-on-support-imp-no-coupling-bad
      [OF no-side])
  show ?thesis
    by (rule composition-bad-bound-if-no-pairwise-side-bad
      [OF wf future no-collision no-coupling])
qed

lemma verifier-initial-random-oracle-freshness-if-uniform-sampler-and-no-pairwise-side-bad:
  — The range premise states that to-nat enumerates exactly the field indices used by the query
  sampler; the divisibility premise makes the modulo/index projection uniform over query-sample-space.
  assumes range-eq: range to-nat = {0..size}
  and dvd: query-sample-space-size dvd size
  and no-collision:
     $\neg \text{supported-hash-output-collision-possible } (\text{verifier-initial-state } tr)$ 
  and no-coupling:
     $\neg \text{supported-pairwise-coupling-bad } (\text{verifier-initial-state } tr)$ 
  shows random-oracle-freshness-assumption (verifier-initial-state tr)
proof -
  have raw-bound: query-index-raw-preimage-bound
    by (rule query-index-raw-preimage-bound-if-uniform-range
      [OF range-eq dvd])
  show ?thesis
    by (rule random-oracle-freshness-if-no-pairwise-side-bad
      [OF - - - raw-bound no-collision no-coupling])
    simp-all
qed

```

The remaining soundness interfaces have the following status without changing the pro-

TOCOL. The predicate *initial-merkle-binding-no-bad* is a cryptographic/model boundary excluding accepted executions whose local Merkle openings cannot be extended to full committed tables. The predicate *random-oracle-freshness-assumption* is a random-oracle model boundary: its component projections are deterministic, but deriving freshness itself would require assumptions about the hash oracle, and deriving domain separation internally would require protocol labels that are not present in the current formalization. The two FRI reduction interfaces are proof obligations rather than protocol changes. Each packages an opening-transcript algebraic cover with a bounded bad-challenge-set statement. The lemmas below derive the verifier bad-event bounds directly from those opening-based interfaces.

lemma *trace-fri-bad-bound-from-opening-reduction*:
assumes *reduction*: *trace-fri-opening-reduction-assumption* *s*
and *fresh*: *trace-fri-challenge-freshness-assumption* *s*
shows *wp-event verify-monad* (*trace-fri-bad* *s*) $s \leq$ *trace-fri-error*
proof –
from *reduction* **obtain** *bad-sets* **where** *opening-cover*:
trace-fri-opening-algebraic-cover *s* *bad-sets*
and *bounded*:
trace-fri-bad-challenge-sets-bounded *bad-sets*
by (rule *trace-fri-opening-reductionE*)
have *cover*: *trace-fri-bad-challenge-cover* *s* *bad-sets*
by (rule *trace-fri-opening-algebraic-cover-imp-bad-challenge-cover*
[*OF* *opening-cover*])
have *wp-event verify-monad* (*trace-fri-bad* *s*) $s \leq$
wp-event verify-monad (*trace-fri-challenge-set-hit* *s* *bad-sets*) *s*
by (rule *wp-event-mono-on-support*)
(use *cover* in $\langle$ *auto simp*: *trace-fri-bad-challenge-cover-def* $\rangle$)
also have ... $\leq$ *trace-fri-error*
using *fresh* *bounded*
unfolding *trace-fri-challenge-freshness-assumption-def* **by** *blast*
finally show *?thesis* .
qed

lemma *composition-fri-bad-bound-from-opening-reduction*:
assumes *reduction*: *composition-fri-opening-reduction-assumption* *s*
and *fresh*: *composition-fri-challenge-freshness-assumption* *s*
shows *wp-event verify-monad* (*composition-fri-bad* *s*) $s \leq$
composition-fri-error
proof –
from *reduction* **obtain** *bad-sets* **where** *opening-cover*:
composition-fri-opening-algebraic-cover *s* *bad-sets*
and *bounded*:
composition-fri-bad-challenge-sets-bounded *bad-sets*
by (rule *composition-fri-opening-reductionE*)
have *cover*: *composition-fri-bad-challenge-cover* *s* *bad-sets*
by (rule *composition-fri-opening-algebraic-cover-imp-bad-challenge-cover*
[*OF* *opening-cover*])
have *wp-event verify-monad* (*composition-fri-bad* *s*) $s \leq$

wp-event verify-monad
 (composition-fri-challenge-set-hit s bad-sets) s
by (rule wp-event-mono-on-support)
 (use cover in $\langle$ auto simp: composition-fri-bad-challenge-cover-def $\rangle$)
also have ... $\leq$ composition-fri-error
 using fresh bounded
 unfolding composition-fri-challenge-freshness-assumption-def **by** blast
finally show ?thesis .
qed

lemma trace-fri-bad-with-partial-openings-bound-from-opening-reduction:
assumes reduction: trace-fri-opening-reduction-assumption s
and fresh: trace-fri-challenge-freshness-assumption s
shows wp-event verify-monad (trace-fri-bad-with-partial-openings s) $s \leq$
 trace-fri-error
proof –
have wp-event verify-monad (trace-fri-bad-with-partial-openings s) $s \leq$
 wp-event verify-monad (trace-fri-bad s) s
by (rule wp-trace-fri-bad-with-partial-openings-le-trace-fri-bad)
also have ... $\leq$ trace-fri-error
by (rule trace-fri-bad-bound-from-opening-reduction
 [OF reduction fresh])
finally show ?thesis .
qed

lemma composition-fri-bad-with-partial-openings-bound-from-opening-reduction:
assumes reduction: composition-fri-opening-reduction-assumption s
and fresh: composition-fri-challenge-freshness-assumption s
shows
 wp-event verify-monad
 (composition-fri-bad-with-partial-openings s) $s \leq$
 composition-fri-error
proof –
have wp-event verify-monad
 (composition-fri-bad-with-partial-openings s) $s \leq$
 wp-event verify-monad (composition-fri-bad s) s
by (rule
 wp-composition-fri-bad-with-partial-openings-le-composition-fri-bad)
also have ... $\leq$ composition-fri-error
by (rule composition-fri-bad-bound-from-opening-reduction
 [OF reduction fresh])
finally show ?thesis .
qed

lemma trace-fri-with-tables-bad-sets-bounded-if-error-ge-one:
assumes error-ge-one: $1 \leq$ trace-fri-error
shows trace-fri-bad-challenge-sets-bounded
 (trace-fri-with-tables-bad-sets s)
unfolding trace-fri-bad-challenge-sets-bounded-def

```

proof (intro allI conjI)
  fix trace-table
  show trace-fri-with-tables-bad-sets s trace-table  $\subseteq$ 
    fri-challenge-space (ceil-log clength)
  by (rule trace-fri-with-tables-bad-sets-subset)
  show nnreal (card (trace-fri-with-tables-bad-sets s trace-table)) /
    nnreal (CARD('f) ^ ceil-log clength)  $\leq$  trace-fri-error
  by (rule order-trans
    [OF trace-fri-with-tables-bad-sets-fraction-le-one error-ge-one])
qed

lemma composition-fri-with-tables-bad-sets-bounded-if-error-ge-one:
  assumes error-ge-one:  $1 \leq$  composition-fri-error
  shows composition-fri-bad-challenge-sets-bounded
    (composition-fri-with-tables-bad-sets s)
  unfolding composition-fri-bad-challenge-sets-bounded-def
proof (intro allI conjI)
  fix dg composition-table
  show composition-fri-with-tables-bad-sets s dg composition-table  $\subseteq$ 
    fri-challenge-space (ceil-log (to-nat dg + 1))
  by (rule composition-fri-with-tables-bad-sets-subset)
  show nnreal
    (card (composition-fri-with-tables-bad-sets s dg composition-table)) /
    nnreal (CARD('f) ^ ceil-log (to-nat dg + 1))  $\leq$ 
    composition-fri-error
  by (rule order-trans
    [OF composition-fri-with-tables-bad-sets-fraction-le-one error-ge-one])
qed

lemma trace-fri-bad-with-tables-bound-from-cover:
  assumes cover:
    trace-fri-with-tables-opening-algebraic-cover s bad-sets
  and bounded: trace-fri-bad-challenge-sets-bounded bad-sets
  and fresh: trace-fri-challenge-freshness-assumption s
  shows wp-event verify-monad (trace-fri-bad-with-tables s) s  $\leq$ 
    trace-fri-error
proof –
  have challenge-cover:
    trace-fri-bad-with-tables-challenge-cover s bad-sets
  by (rule trace-fri-with-tables-opening-algebraic-cover-imp-bad-challenge-cover
    [OF cover])
  have wp-event verify-monad (trace-fri-bad-with-tables s) s  $\leq$ 
    wp-event verify-monad (trace-fri-challenge-set-hit s bad-sets) s
  by (rule wp-event-mono-on-support)
    (use challenge-cover in
      ⟨auto simp: trace-fri-bad-with-tables-challenge-cover-def⟩)
  also have ...  $\leq$  trace-fri-error
  using fresh bounded
  unfolding trace-fri-challenge-freshness-assumption-def by blast

```

finally show ?thesis .
qed

lemma *composition-fri-bad-with-tables-bound-from-cover:*

assumes *cover:*

composition-fri-with-tables-opening-algebraic-cover s bad-sets

and *bounded: composition-fri-bad-challenge-sets-bounded bad-sets*

and *fresh: composition-fri-challenge-freshness-assumption s*

shows *wp-event verify-monad (composition-fri-bad-with-tables s) s ≤ composition-fri-error*

proof –

have *challenge-cover:*

composition-fri-bad-with-tables-challenge-cover s bad-sets

by (rule

composition-fri-with-tables-opening-algebraic-cover-imp-bad-challenge-cover
[OF *cover*])

have *wp-event verify-monad (composition-fri-bad-with-tables s) s ≤*

wp-event verify-monad

(composition-fri-challenge-set-hit s bad-sets) s

by (rule *wp-event-mono-on-support*)

(use *challenge-cover in*

⟨*auto simp: composition-fri-bad-with-tables-challenge-cover-def*⟩)

also have ... ≤ *composition-fri-error*

using *fresh bounded*

unfolding *composition-fri-challenge-freshness-assumption-def* by blast

finally show ?thesis .

qed

definition *trace-fri-with-tables-opening-reduction-assumption*

:: (*f, 'a*) *protocol-channel-scheme* ⇒ bool

where

trace-fri-with-tables-opening-reduction-assumption s ⇔

(∃ *bad-sets.*

trace-fri-with-tables-opening-algebraic-cover s bad-sets ∧

trace-fri-bad-challenge-sets-bounded bad-sets)

definition *composition-fri-with-tables-opening-reduction-assumption*

:: (*f, 'a*) *protocol-channel-scheme* ⇒ bool

where

composition-fri-with-tables-opening-reduction-assumption s ⇔

(∃ *bad-sets.*

composition-fri-with-tables-opening-algebraic-cover s bad-sets ∧

composition-fri-bad-challenge-sets-bounded bad-sets)

lemma *trace-fri-with-tables-opening-reductionE:*

assumes *trace-fri-with-tables-opening-reduction-assumption s*

obtains *bad-sets*

where *trace-fri-with-tables-opening-algebraic-cover s bad-sets*

and *trace-fri-bad-challenge-sets-bounded bad-sets*

```

using assms
unfolding trace-fri-with-tables-opening-reduction-assumption-def by blast

lemma composition-fri-with-tables-opening-reductionE:
  assumes composition-fri-with-tables-opening-reduction-assumption s
  obtains bad-sets
  where composition-fri-with-tables-opening-algebraic-cover s bad-sets
    and composition-fri-bad-challenge-sets-bounded bad-sets
  using assms
  unfolding composition-fri-with-tables-opening-reduction-assumption-def
  by blast

lemma trace-fri-bad-with-tables-bound-from-opening-reduction:
  assumes reduction:
    trace-fri-with-tables-opening-reduction-assumption s
    and fresh: trace-fri-challenge-freshness-assumption s
  shows wp-event verify-monad (trace-fri-bad-with-tables s) s ≤
    trace-fri-error
proof –
  from reduction obtain bad-sets where cover:
    trace-fri-with-tables-opening-algebraic-cover s bad-sets
    and bounded: trace-fri-bad-challenge-sets-bounded bad-sets
    by (rule trace-fri-with-tables-opening-reductionE)
  show ?thesis
    by (rule trace-fri-bad-with-tables-bound-from-cover
      [OF cover bounded fresh])
qed

lemma composition-fri-bad-with-tables-bound-from-opening-reduction:
  assumes reduction:
    composition-fri-with-tables-opening-reduction-assumption s
    and fresh: composition-fri-challenge-freshness-assumption s
  shows wp-event verify-monad (composition-fri-bad-with-tables s) s ≤
    composition-fri-error
proof –
  from reduction obtain bad-sets where cover:
    composition-fri-with-tables-opening-algebraic-cover s bad-sets
    and bounded: composition-fri-bad-challenge-sets-bounded bad-sets
    by (rule composition-fri-with-tables-opening-reductionE)
  show ?thesis
    by (rule composition-fri-bad-with-tables-bound-from-cover
      [OF cover bounded fresh])
qed

end

end

```



```

theory Soundness-Reductions
  imports Soundness-Reductions-Opening
begin

```

Compatibility umbrella for soundness reduction lemmas.

```

end

```

```

theory Soundness-Staged-Query-Prefix
  imports Soundness-Reductions Staged-Security-Experiment
begin

```

Staged query-prefix replay and prefix receive decomposition lemmas.

```

context soundness
begin

```

The deterministic partition above reduces soundness to four probability bounds. The bounds themselves require additional cryptographic/algebraic interfaces: random-linear-combination soundness, query-sampling freshness, and FRI low-degree soundness. Those assumptions are kept in the separate bounded-soundness locale below, rather than hidden as unproved lemmas in this base locale.

```

lemma soundness-bound-component-accounting:
  soundness-bound =
    composition-error-bound + trace-fri-error + composition-fri-error +
    nnreal rounds * query-error-bound
unfolding soundness-bound-def by simp

```

```

lemma soundness-bound-with-merkle-component-accounting:
  soundness-bound-with-merkle =
    merkle-binding-error + composition-error-bound + trace-fri-error +
    composition-fri-error + nnreal rounds * query-error-bound
unfolding soundness-bound-with-merkle-def soundness-bound-def
by (simp add: algebra-simps)

```

```

lemma wp-event-finite-union-bound:
  fixes B :: 'i  $\Rightarrow$  prob
  assumes finite: finite I
  and bounds:  $\bigwedge i. i \in I \implies \text{wp-event } m \ (E \ i) \ s \leq B \ i$ 
  shows
     $\text{wp-event } m \ (\lambda \text{out}. \exists i \in I. E \ i \ \text{out}) \ s \leq (\sum i \in I. B \ i)$ 
  using finite bounds
proof (induction I)
  case empty
  show ?case
    unfolding wp-event-def wp-def dist-expect-def by simp
next
  case (insert i I)
  have split:

```

```

wp-event m ( $\lambda out. \exists j \in \text{insert } i \ I. E \ j \ out$ ) s  $\leq$ 
  wp-event m ( $E \ i$ ) s +
  wp-event m ( $\lambda out. \exists j \in I. E \ j \ out$ ) s
proof -
  have wp-event m ( $\lambda out. \exists j \in \text{insert } i \ I. E \ j \ out$ ) s  $\leq$ 
    wp-event m ( $\lambda out. E \ i \ out \vee (\exists j \in I. E \ j \ out)$ ) s
    by (rule wp-event-mono) auto
  also have ...  $\leq$ 
    wp-event m ( $E \ i$ ) s +
    wp-event m ( $\lambda out. \exists j \in I. E \ j \ out$ ) s
    by (rule wp-event-union-bound)
  finally show ?thesis .
qed
have tail:
  wp-event m ( $\lambda out. \exists j \in I. E \ j \ out$ ) s  $\leq$  ( $\sum j \in I. B \ j$ )
  by (rule insert.IH) (use insert.premis in blast)
have wp-event m ( $E \ i$ ) s +
  wp-event m ( $\lambda out. \exists j \in I. E \ j \ out$ ) s  $\leq$ 
  B i + ( $\sum j \in I. B \ j$ )
  by (intro add-mono tail insert.premis) simp
also have ... = ( $\sum j \in \text{insert } i \ I. B \ j$ )
  using insert.hyps by simp
finally show ?case
  by (rule order-trans[OF split])
qed

lemma hash-extends-verifier-state-from-adversary-right:
  assumes ext: s  $\leq$  t
  shows s  $\leq$  verifier-state-from-adversary t tr
  using ext
  unfolding verifier-state-from-adversary-def less-eq-hash-ext-def
  less-eq-fmap-def
  by simp

lemma checked-staged-security-with-data-state-supported-output-local-side-bad-bound-controlled:
  assumes wf: staged-budget-wellformed budgets
  and controlled: staged-adversary-controlled budgets A
  shows
  wp-event (checked-staged-security-experiment-with-data-state A)
    (staged-security-with-data-state-verifier-event
      supported-output-local-side-bad)
  adversary-initial-state  $\leq$ 
  hash-collision-budget-value 0
    (staged-attacker-query-budget budgets +
      staged-challenge-query-budget + verifier-hash-query-budget)
  unfolding supported-output-local-side-bad-iff-hash-map-output-collision-bad
  by (rule
    checked-staged-security-with-data-state-hash-map-output-collision-bad-bound-controlled
    [OF wf controlled])

```

lemma *checked-staged-security-with-data-state-supported-pairwise-side-bad-bound-controlled:*
fixes *coupling-error :: prob*
assumes *wf: staged-budget-wellformed budgets*
and *controlled: staged-adversary-controlled budgets A*
and *coupling-bound:*
 wp-event (checked-staged-security-experiment-with-data-state A)
 (staged-security-with-data-state-verifier-event
 supported-pairwise-coupling-event)
 adversary-initial-state ≤ coupling-error
shows
 wp-event (checked-staged-security-experiment-with-data-state A)
 (staged-security-with-data-state-verifier-event
 supported-pairwise-side-bad)
 adversary-initial-state ≤
 hash-collision-budget-value 0
 (staged-attacker-query-budget budgets +
 staged-challenge-query-budget + verifier-hash-query-budget) +
 coupling-error
proof –
let *?M = checked-staged-security-experiment-with-data-state A*
let *?E = staged-security-with-data-state-verifier-event*
have *side-eq:*
 ?E supported-pairwise-side-bad =
 (λout. ?E hash-map-output-collision-bad out ∨
 ?E supported-pairwise-coupling-event out)
unfolding *staged-security-with-data-state-verifier-event-def*
 supported-pairwise-side-bad-def supported-pairwise-coupling-event-def
by *(rule ext) (auto split: option.splits prod.splits)*
have *wp-event ?M (?E supported-pairwise-side-bad)*
 adversary-initial-state ≤
 wp-event ?M (?E hash-map-output-collision-bad)
 adversary-initial-state +
 wp-event ?M (?E supported-pairwise-coupling-event)
 adversary-initial-state
unfolding *side-eq*
by *(rule wp-event-union-bound)*
also have *... ≤*
 hash-collision-budget-value 0
 (staged-attacker-query-budget budgets +
 staged-challenge-query-budget + verifier-hash-query-budget) +
 coupling-error
by *(intro add-mono coupling-bound*
 checked-staged-security-with-data-state-hash-map-output-collision-bad-bound-controlled
 [OF wf controlled])
finally show *?thesis .*
qed

lemma *checked-staged-accepted-imp-transcript-pre-or-new-hit:*

```

assumes support:
  out ∈
    set-dist
      (execute (checked-staged-security-experiment-with-data-state A)
        adversary-initial-state)
  and acc: accepted out
shows
  staged-security-with-data-state-transcript-pre-hit out ∨
  staged-security-with-data-state-transcript-new-hit out
proof (cases out)
case None
then show ?thesis
  using acc unfolding accepted-def by simp
next
case (Some packed)
then obtain data attacker-state result final-state where out-eq:
  out = Some (((data, attacker-state), result), final-state)
  by (cases packed, auto split: prod.splits)
let ?tr = staged-proof-transcript data
let ?s = verifier-state-from-adversary attacker-state ?tr
have verifier:
  Some (result, final-state) ∈ set-dist (execute verify-monad ?s)
  using checked-staged-security-experiment-with-data-state-outcomeE
    [OF support[unfolded out-eq]]
  by blast
have split:
  hash-map-output-values ?s ∩ set ?tr ≠ {} ∨
  hash-map-new-output-hit (set ?tr) ?s final-state
  by (rule verify-monad-trace-root-target-preexisting-or-new[OF verifier])
    simp
then show ?thesis
  unfolding out-eq
    staged-security-with-data-state-transcript-pre-hit-def
    staged-security-with-data-state-transcript-new-hit-def
  by simp
qed

lemma checked-staged-acceptance-transcript-hit-union-bound:
  wp-event (checked-staged-security-experiment-with-data-state A) accepted
    adversary-initial-state ≤
  wp-event (checked-staged-security-experiment-with-data-state A)
    staged-security-with-data-state-transcript-pre-hit
    adversary-initial-state +
  wp-event (checked-staged-security-experiment-with-data-state A)
    staged-security-with-data-state-transcript-new-hit
    adversary-initial-state
proof –
let ?M = checked-staged-security-experiment-with-data-state A
let ?pre = staged-security-with-data-state-transcript-pre-hit

```

```

let ?new = staged-security-with-data-state-transcript-new-hit
have wp-event ?M accepted adversary-initial-state ≤
  wp-event ?M (λout. ?pre out ∨ ?new out) adversary-initial-state
by (rule wp-event-mono-on-support)
  (rule checked-staged-accepted-imp-transcript-pre-or-new-hit)
also have ... ≤
  wp-event ?M ?pre adversary-initial-state +
  wp-event ?M ?new adversary-initial-state
by (rule wp-event-union-bound)
finally show ?thesis .
qed

```

definition *staged-security-with-data-state-query-partial-header-key-hit*
 $:: (((\text{'f staged-proof-data} \times \text{'f protocol-channel}) \times \text{unit list}) \times$
 $\text{'f protocol-channel}) \text{option} \Rightarrow \text{bool}$

where

```

staged-security-with-data-state-query-partial-header-key-hit out  $\longleftrightarrow$ 
  (case out of
    None  $\Rightarrow$  False
  | Some (((data, attacker-state), -), final-state)  $\Rightarrow$ 
    (let s =
      verifier-state-from-adversary attacker-state
      (staged-proof-transcript data);
      B =
        query-header-supported-partial-union-good-sets s
        query-sampling-success-space
        (staged-trace-root data)
        (staged-trace-fri-roots data)
        (staged-trace-final data)
        (staged-alphas data)
        (staged-degree data)
        (staged-composition-fri-roots data)
        (staged-composition-final data)
    in
      ( $\exists i$  raw.  $i < \text{rounds} \wedge$ 
        fmlookup (HashMap final-state)
          (QueryIndexChallenge i
            (state-after-query-chunks
              (staged-query-start-hash data)
              (staged-query-chunks data) i)) =
          Some raw  $\wedge$ 
          index (to-nat raw)  $\in B$ )))

```

definition *staged-security-with-data-state-query-bad-hit-at*

```

 $:: \text{nat} \Rightarrow$ 
  (((\text{'f staged-proof-data}  $\times$  \text{'f protocol-channel})  $\times$  \text{unit list})  $\times$ 
    \text{'f protocol-channel}) \text{option}  $\Rightarrow$  \text{bool}

```

where

```

staged-security-with-data-state-query-bad-hit-at i out  $\longleftrightarrow$ 

```

```

(case out of
  None  $\Rightarrow$  False
| Some (((data, attacker-state), result), final-state)  $\Rightarrow$ 
  (let s =
    verifier-state-from-adversary attacker-state
    (staged-proof-transcript data)
  in
    Some (result, final-state)  $\in$  set-dist (execute verify-monad s)  $\wedge$ 
    query-index-round-set-hit-at s query-sampling-success-space i
    (Some (result, final-state))))

lemma staged-security-with-data-state-query-bad-hit-imp-round-hit:
  assumes bad: staged-security-with-data-state-query-bad-hit out
  shows  $\exists i \in \{..<\text{rounds}\}$ .
    staged-security-with-data-state-query-bad-hit-at i out
proof (cases out)
  case None
  then show ?thesis
    using bad unfolding staged-security-with-data-state-query-bad-hit-def
    by simp
next
  case (Some packed)
  then obtain data attacker-state result final-state where out-eq:
    out = Some (((data, attacker-state), result), final-state)
    by (cases packed, auto split: prod.splits)
  let ?s =
    verifier-state-from-adversary attacker-state
    (staged-proof-transcript data)
  from bad[unfolded out-eq
    staged-security-with-data-state-query-bad-hit-def Let-def]
  have verifier:
    Some (result, final-state)  $\in$  set-dist (execute verify-monad ?s)
    and query-bad: query-bad ?s (Some (result, final-state))
    by simp-all
  from query-bad-imp-query-index-round-set-hit[OF query-bad]
  obtain i where i-bound: i < rounds
    and hit-at:
      query-index-round-set-hit-at ?s query-sampling-success-space i
      (Some (result, final-state))
  unfolding query-index-round-set-hit-def by blast
  have staged-security-with-data-state-query-bad-hit-at i out
    unfolding out-eq staged-security-with-data-state-query-bad-hit-at-def
    Let-def
    using verifier hit-at by simp
  then show ?thesis
    using i-bound by blast
qed

```

lemma checked-staged-security-with-data-state-query-bad-hit-bound-from-rounds:

```

assumes round-bound:
   $\bigwedge i. i < \text{rounds} \implies$ 
    wp-event (checked-staged-security-experiment-with-data-state A)
      (staged-security-with-data-state-query-bad-hit-at i)
      adversary-initial-state  $\leq C i$ 
shows
  wp-event (checked-staged-security-experiment-with-data-state A)
    staged-security-with-data-state-query-bad-hit
    adversary-initial-state  $\leq (\sum i < \text{rounds}. C i)$ 
proof –
  have wp-event (checked-staged-security-experiment-with-data-state A)
    staged-security-with-data-state-query-bad-hit
    adversary-initial-state  $\leq$ 
  wp-event (checked-staged-security-experiment-with-data-state A)
    ( $\lambda \text{out}. \exists i \in \{.. < \text{rounds}\}.$ 
      staged-security-with-data-state-query-bad-hit-at i out)
    adversary-initial-state
  by (rule wp-event-mono)
    (rule staged-security-with-data-state-query-bad-hit-imp-round-hit)
  also have ...  $\leq (\sum i < \text{rounds}. C i)$ 
  by (rule wp-event-finite-UN-bound)
    (simp-all add: round-bound)
  finally show ?thesis .
qed

lemma checked-staged-security-with-data-state-query-bad-imp-partial-header-key-hit-or-collision-on-support:
  assumes wf: staged-budget-wellformed budgets
  and controlled: staged-adversary-controlled budgets A
  and support:
    out  $\in$ 
      set-dist
      (execute
        (checked-staged-security-experiment-with-data-state A)
        adversary-initial-state)
  and bad: staged-security-with-data-state-query-bad-hit out
  shows
    staged-security-with-data-state-query-partial-header-key-hit out  $\vee$ 
    staged-security-with-data-state-verifier-event
    hash-map-output-collision-bad out
  proof (cases out)
  case None
  then show ?thesis
    using bad unfolding staged-security-with-data-state-query-bad-hit-def
    by simp
  next
  case (Some result-pack)
  then obtain data attacker-state result final-state where out-eq:
    out = Some (((data, attacker-state), result), final-state)
  by (cases result-pack, auto split: prod.splits)

```

```

let ?s =
  verifier-state-from-adversary attacker-state
    (staged-proof-transcript data)
have support-some:
  Some (((data, attacker-state), result), final-state) ∈
    set-dist
      (execute
        (checked-staged-security-experiment-with-data-state A)
        adversary-initial-state)
  using support unfolding out-eq .
from checked-staged-security-experiment-with-data-state-outcomeE
  [OF support-some]
have builder:
  Some (data, attacker-state) ∈
    set-dist
      (execute (checked-staged-transcript-program A)
        adversary-initial-state)
  and verifier:
    Some (result, final-state) ∈ set-dist (execute verify-monad ?s)
  by blast+
have staged-header:
  verifier-header-transcript ?s
    (staged-trace-root data)
    (staged-trace-fri-roots data)
    (staged-trace-final data)
    (staged-alphas data)
    (staged-degree data)
    (staged-composition-fri-roots data)
    (staged-composition-final data)
    (List.concat (staged-query-chunks data))
  by (rule checked-staged-transcript-program-outcome-header-transcript
    [OF wf controlled builder])
from bad[unfolded out-eq
  staged-security-with-data-state-query-bad-hit-def Let-def]
have query-bad: query-bad ?s (Some (result, final-state))
  by simp
from query-bad-imp-query-index-round-set-hit[OF query-bad]
obtain i trace-table composition-table as query-idxs where
  bound:
    accepted-with-bound-tables ?s (Some (result, final-state))
    trace-table composition-table as query-idxs
  and i-bound: i < rounds
  and query-hit:
    query-idxs ! i ∈
      query-sampling-success-space trace-table composition-table as
  unfolding query-index-round-set-hit-def
    query-index-round-set-hit-at-def
  by blast
have shape:

```



```

    accepted-transcript-shape ?s (Some (result, final-state))
    as query-idxs
  using bound
  unfolding accepted-with-bound-tables-def accepted-with-tables-def
  by simp
from checked-staged-security-with-data-state-accepted-shape-query-keys
  [OF wf controlled support-some shape]
obtain raw-idxs where
  as-eq: as = staged-alphas data
  and len-raw: length raw-idxs = rounds
  and query-idxs-eq:
    query-idxs = map (λraw. index (to-nat raw)) raw-idxs
  and lookup:
     $\bigwedge j. j < rounds \implies$ 
    fmlookup (HashMap final-state)
      (QueryIndexChallenge j
        (state-after-query-chunks
          (staged-query-start-hash data)
          (staged-query-chunks data) j)) =
      Some (raw-idxs ! j)
  by blast
show ?thesis
proof (cases hash-map-output-collision final-state)
  case True
  then have
    staged-security-with-data-state-verifier-event
    hash-map-output-collision-bad
    (Some (((data, attacker-state), result), final-state))
  unfolding staged-security-with-data-state-verifier-event-def
    hash-map-output-collision-bad-def accepted-def
  by simp
  then show ?thesis
    unfolding out-eq by simp
next
  case clean: False
  have idx-sample:
    query-idxs ! i ∈ query-sample-space
  proof -
    have len-query: length query-idxs = rounds
      using query-idxs-eq len-raw by simp
    have query-idxs ! i ∈ set query-idxs
      by (rule nth-mem) (use i-bound len-query in simp)
    then show ?thesis
      by (rule accepted-with-bound-tables-query-sample-space[OF bound])
  qed
from bound obtain fr f-fri-roots f-final dg composition-fri-roots final
  rest where
  header':
    verifier-header-transcript ?s fr f-fri-roots f-final as dg

```

```

    composition-fri-roots final rest
and trace-bind:
    merkle-root-binds-table fr trace-table final-state
and comp-nonempty: composition-fri-roots  $\neq$  []
and comp-bind:
    merkle-root-binds-table (hd composition-fri-roots) composition-table
    final-state
unfolding accepted-with-bound-tables-def by blast
have eqs:
  fr = staged-trace-root data  $\wedge$ 
  f-fri-roots = staged-trace-fri-roots data  $\wedge$ 
  f-final = staged-trace-final data  $\wedge$ 
  as = staged-alphas data  $\wedge$ 
  dg = staged-degree data  $\wedge$ 
  composition-fri-roots = staged-composition-fri-roots data  $\wedge$ 
  final = staged-composition-final data  $\wedge$ 
  rest = List.concat (staged-query-chunks data)
using verifier-header-transcript-unique[OF header' staged-header]
by simp
have state:
  query-header-supported-table-candidate-state ?s
    (staged-trace-root data)
    (staged-trace-fri-roots data)
    (staged-trace-final data)
    (staged-alphas data)
    (staged-degree data)
    (staged-composition-fri-roots data)
    (staged-composition-final data)
    trace-table composition-table final-state
unfolding query-header-supported-table-candidate-state-def
by (intro exI[of - result] exI[of - query-idxs]
    exI[of - List.concat (staged-query-chunks data)]))
    (use verifier bound staged-header trace-bind comp-nonempty comp-bind
    eqs in simp)
have candidate:
  (trace-table, composition-table)  $\in$ 
    query-header-supported-partial-table-candidates ?s
      (staged-trace-root data)
      (staged-trace-fri-roots data)
      (staged-trace-final data)
      (staged-alphas data)
      (staged-degree data)
      (staged-composition-fri-roots data)
      (staged-composition-final data)
using
  query-header-supported-table-candidate-state-imp-partial-candidate-state
  [OF state clean]
  query-header-supported-partial-table-candidates-iff-state
by blast

```

```

have raw-hit:
  index (to-nat (raw-idxs ! i)) ∈
    query-header-supported-partial-union-good-sets ?s
    query-sampling-success-space
    (staged-trace-root data)
    (staged-trace-fri-roots data)
    (staged-trace-final data)
    (staged-alphas data)
    (staged-degree data)
    (staged-composition-fri-roots data)
    (staged-composition-final data)
proof -
  have raw-idx-eq: index (to-nat (raw-idxs ! i)) = query-idxs ! i
    using query-idxs-eq len-raw i-bound by simp
  show ?thesis
    unfolding query-header-supported-partial-union-good-sets-def
      raw-idx-eq
    using idx-sample candidate query-hit as-eq by blast
qed
have lookup-i:
  fmlookup (HashMap final-state)
    (QueryIndexChallenge i
      (state-after-query-chunks
        (staged-query-start-hash data)
        (staged-query-chunks data) i)) =
    Some (raw-idxs ! i)
  by (rule lookup[OF i-bound])
show ?thesis
  unfolding out-eq
    staged-security-with-data-state-query-partial-header-key-hit-def
    Let-def
  using i-bound lookup-i raw-hit by auto
qed
qed

lemma checked-staged-security-with-data-state-query-bad-bound-from-partial-header-key-hit-or-collision:
  fixes C K :: prob
  assumes wf: staged-budget-wellformed budgets
  and controlled: staged-adversary-controlled budgets A
  and partial-header-key-bound:
    wp-event (checked-staged-security-experiment-with-data-state A)
      staged-security-with-data-state-query-partial-header-key-hit
      adversary-initial-state ≤ C
  and collision-bound:
    wp-event (checked-staged-security-experiment-with-data-state A)
      (staged-security-with-data-state-verifier-event
        hash-map-output-collision-bad)
      adversary-initial-state ≤ K
shows

```

```

    wp-event (checked-staged-security-experiment-with-data-state A)
      staged-security-with-data-state-query-bad-hit
      adversary-initial-state  $\leq C + K$ 
  proof -
    let ?M = checked-staged-security-experiment-with-data-state A
    have wp-event ?M staged-security-with-data-state-query-bad-hit
      adversary-initial-state  $\leq$ 
      wp-event ?M
      ( $\lambda$ out.
        staged-security-with-data-state-query-partial-header-key-hit out  $\vee$ 
        staged-security-with-data-state-verifier-event
        hash-map-output-collision-bad out)
      adversary-initial-state
  proof (rule wp-event-mono-on-support)
    fix out
    assume support:
      out  $\in$  set-dist (execute ?M adversary-initial-state)
    and bad: staged-security-with-data-state-query-bad-hit out
    show
      staged-security-with-data-state-query-partial-header-key-hit out  $\vee$ 
      staged-security-with-data-state-verifier-event
      hash-map-output-collision-bad out
    by (rule
      checked-staged-security-with-data-state-query-bad-imp-partial-header-key-hit-or-collision-on-support
      [OF wf controlled support bad])
  qed
  also have ...  $\leq$ 
    wp-event ?M staged-security-with-data-state-query-partial-header-key-hit
      adversary-initial-state +
    wp-event ?M
      (staged-security-with-data-state-verifier-event
      hash-map-output-collision-bad)
      adversary-initial-state
    by (rule wp-event-union-bound)
  also have ...  $\leq C + K$ 
    by (intro add-mono partial-header-key-bound collision-bound)
  finally show ?thesis .
qed

lemma checked-staged-security-with-data-state-query-bad-verifier-event-bound-from-partial-header-key-hit-or-collision
  fixes C K :: prob
  assumes wf: staged-budget-wellformed budgets
  and controlled: staged-adversary-controlled budgets A
  and partial-header-key-bound:
    wp-event (checked-staged-security-experiment-with-data-state A)
      staged-security-with-data-state-query-partial-header-key-hit
      adversary-initial-state  $\leq C$ 
  and collision-bound:
    wp-event (checked-staged-security-experiment-with-data-state A)

```

(staged-security-with-data-state-verifier-event
 hash-map-output-collision-bad)
 adversary-initial-state $\leq K$

shows

wp-event (checked-staged-security-experiment-with-data-state A)
 (staged-security-with-data-state-verifier-event query-bad)
 adversary-initial-state $\leq C + K$

proof –

have wp-event (checked-staged-security-experiment-with-data-state A)
 (staged-security-with-data-state-verifier-event query-bad)
 adversary-initial-state $\leq$
 wp-event (checked-staged-security-experiment-with-data-state A)
 staged-security-with-data-state-query-bad-hit
 adversary-initial-state
by (rule checked-staged-security-with-data-state-query-bad-verifier-event-bound
 [OF order-refl])
also have ... $\leq C + K$
by (rule
 checked-staged-security-with-data-state-query-bad-bound-from-partial-header-key-hit-or-collision
 [OF wf controlled partial-header-key-bound collision-bound])
finally show ?thesis .

qed

lemma checked-staged-security-with-data-state-partial-header-key-hit-bound-from-index-set:

assumes cover:

$\bigwedge$ data attacker-state.
 query-header-supported-partial-union-good-sets
 (verifier-state-from-adversary attacker-state
 (staged-proof-transcript data))
 query-sampling-success-space
 (staged-trace-root data)
 (staged-trace-fri-roots data)
 (staged-trace-final data)
 (staged-alphas data)
 (staged-degree data)
 (staged-composition-fri-roots data)
 (staged-composition-final data) $\subseteq B$

and index-bound:

wp-event (checked-staged-security-experiment-with-data A)
 (staged-security-with-data-query-index-set-hit B)
 adversary-initial-state $\leq C$

shows

wp-event (checked-staged-security-experiment-with-data-state A)
 staged-security-with-data-state-query-partial-header-key-hit
 adversary-initial-state $\leq C$

proof –

let ?projected =
 λ out. case out of None $\Rightarrow$
 staged-security-with-data-query-index-set-hit B None

```

| Some (((data, -), result), t) ⇒
  staged-security-with-data-query-index-set-hit B
  (Some ((data, result), t))
have event-le:
  wp-event (checked-staged-security-experiment-with-data-state A)
    staged-security-with-data-state-query-partial-header-key-hit
    adversary-initial-state ≤
  wp-event (checked-staged-security-experiment-with-data-state A)
    ?projected adversary-initial-state
proof (rule wp-event-mono)
  fix out
  assume hit:
    staged-security-with-data-state-query-partial-header-key-hit out
  show ?projected out
  proof (cases out)
  case None
  then show ?thesis
    using hit
    unfolding
      staged-security-with-data-state-query-partial-header-key-hit-def
    by simp
  next
  case (Some packed)
  then obtain data attacker-state result final-state where out-eq:
    out = Some (((data, attacker-state), result), final-state)
  by (cases packed, auto split: prod.splits)
  let ?s =
    verifier-state-from-adversary attacker-state
    (staged-proof-transcript data)
  let ?B =
    query-header-supported-partial-union-good-sets ?s
    query-sampling-success-space
    (staged-trace-root data)
    (staged-trace-fri-roots data)
    (staged-trace-final data)
    (staged-alphas data)
    (staged-degree data)
    (staged-composition-fri-roots data)
    (staged-composition-final data)
  from hit[unfolded out-eq
    staged-security-with-data-state-query-partial-header-key-hit-def
    Let-def]
  obtain i raw where i-bound: i < rounds
  and lookup:
    fmlookup (HashMap final-state)
      (QueryIndexChallenge i
        (state-after-query-chunks
          (staged-query-start-hash data)
          (staged-query-chunks data) i)) =

```

```

    Some raw
    and raw-hit: index (to-nat raw) ∈ ?B
    by auto
    have raw-in-B: index (to-nat raw) ∈ B
    proof -
      have ?B ⊆ B
      by (rule cover)
      then show ?thesis
      using raw-hit by blast
    qed
    show ?thesis
      unfolding out-eq staged-security-with-data-query-index-set-hit-def
      using i-bound lookup raw-in-B by auto
    qed
  qed
  also have
    wp-event (checked-staged-security-experiment-with-data-state A)
      ?projected adversary-initial-state =
    wp-event (checked-staged-security-experiment-with-data A)
      (staged-security-with-data-query-index-set-hit B)
      adversary-initial-state
    using checked-staged-security-experiment-with-data-event-from-data-state
      [of A staged-security-with-data-query-index-set-hit B]
    by simp
  also have ... ≤ C
    by (rule index-bound)
  finally show ?thesis .
  qed

```

lemma *checked-staged-security-with-data-state-partial-header-key-hit-bound-from-index-set-controlled:*

```

assumes wf: staged-budget-wellformed budgets
and controlled: staged-adversary-controlled budgets A
and cover:
  ∧ data attacker-state.
    query-header-supported-partial-union-good-sets
      (verifier-state-from-adversary attacker-state
        (staged-proof-transcript data))
    query-sampling-success-space
      (staged-trace-root data)
      (staged-trace-fri-roots data)
      (staged-trace-final data)
      (staged-alphas data)
      (staged-degree data)
      (staged-composition-fri-roots data)
      (staged-composition-final data) ⊆ B

```

shows

```

wp-event (checked-staged-security-experiment-with-data-state A)
  staged-security-with-data-state-query-partial-header-key-hit
  adversary-initial-state ≤

```

staged-phase-target-error (*query-index-raw-preimage* B)
 (*staged-attacker-query-budget* *budgets* +
staged-challenge-query-budget)
by (*rule*
checked-staged-security-with-data-state-partial-header-key-hit-bound-from-index-set
 [*OF cover*])
 (*rule checked-staged-security-with-data-query-index-set-hit-bound*
 [*OF wf controlled*])

lemma *checked-staged-security-with-data-state-query-bad-verifier-event-bound-from-partial-fixed-query-error-cover*:
assumes *wf*: *staged-budget-wellformed* *budgets*
and *controlled*: *staged-adversary-controlled* *budgets* A
and *cover*:
 $\bigwedge$ *data* *attacker-state*.
query-header-supported-partial-union-good-sets
 (*verifier-state-from-adversary* *attacker-state*
 (*staged-proof-transcript* *data*))
query-sampling-success-space
 (*staged-trace-root* *data*)
 (*staged-trace-fri-roots* *data*)
 (*staged-trace-final* *data*)
 (*staged-alphas* *data*)
 (*staged-degree* *data*)
 (*staged-composition-fri-roots* *data*)
 (*staged-composition-final* *data*) $\subseteq B$
and *raw-bound*: *query-index-raw-preimage-bound*
and *subset*: $B \subseteq \text{query-sample-space}$
and *frac*:
 $\text{nnreal} (\text{card } B) / \text{nnreal} (\text{card } \text{query-sample-space}) \leq$
query-error-bound
shows
wp-event (*checked-staged-security-experiment-with-data-state* A)
 (*staged-security-with-data-state-verifier-event* *query-bad*)
adversary-initial-state $\leq$
 nnreal
 (*staged-attacker-query-budget* *budgets* +
staged-challenge-query-budget) * *query-error-bound* +
hash-collision-budget-value 0
 (*staged-attacker-query-budget* *budgets* +
staged-challenge-query-budget + *verifier-hash-query-budget*)
proof –
have *partial-key-bound*:
wp-event (*checked-staged-security-experiment-with-data-state* A)
staged-security-with-data-state-query-partial-header-key-hit
adversary-initial-state $\leq$
 nnreal
 (*staged-attacker-query-budget* *budgets* +
staged-challenge-query-budget) * *query-error-bound*
proof –

have *wp-event* (*checked-staged-security-experiment-with-data-state A*)
staged-security-with-data-state-query-partial-header-key-hit
adversary-initial-state $\leq$
staged-phase-target-error (*query-index-raw-preimage B*)
(staged-attacker-query-budget budgets +
staged-challenge-query-budget)
by (*rule*
checked-staged-security-with-data-state-partial-header-key-hit-bound-from-index-set-controlled
[OF wf controlled cover])
also have ... $\leq$
nnreal
(staged-attacker-query-budget budgets +
*staged-challenge-query-budget) * query-error-bound*
by (*rule staged-phase-query-index-target-error-query-bound*
[OF raw-bound subset frac])
finally show ?thesis .
qed
have *collision-bound*:
wp-event (*checked-staged-security-experiment-with-data-state A*)
(staged-security-with-data-state-verifier-event
hash-map-output-collision-bad)
adversary-initial-state $\leq$
hash-collision-budget-value 0
(staged-attacker-query-budget budgets +
staged-challenge-query-budget + verifier-hash-query-budget)
by (*rule*
checked-staged-security-with-data-state-hash-map-output-collision-bad-bound-controlled
[OF wf controlled])
show ?thesis
by (*rule*
checked-staged-security-with-data-state-query-bad-verifier-event-bound-from-partial-header-key-hit-or-collision
[OF wf controlled partial-key-bound collision-bound])
qed

lemma *staged-dynamic-partial-query-set-subset*:
query-header-supported-partial-union-good-sets
(verifier-state-from-adversary attacker-state
(staged-proof-transcript data))
query-sampling-success-space
(staged-trace-root data)
(staged-trace-fri-roots data)
(staged-trace-final data)
(staged-alphas data)
(staged-degree data)
(staged-composition-fri-roots data)
(staged-composition-final data) $\subseteq$ query-sample-space
by (*rule query-header-supported-partial-union-good-sets-subset*)

lemma *staged-dynamic-partial-query-set-fraction-bound-if-unique-candidate*:

assumes *unique*:

$\exists$ *trace-table composition-table*.
 query-header-supported-partial-table-candidates
 (*verifier-state-from-adversary attacker-state*
 (*staged-proof-transcript data*))
 (*staged-trace-root data*)
 (*staged-trace-fri-roots data*)
 (*staged-trace-final data*)
 (*staged-alphas data*)
 (*staged-degree data*)
 (*staged-composition-fri-roots data*)
 (*staged-composition-final data*) $\subseteq$
 $\{(trace-table, composition-table)\}$

shows

nnreal
 (*card*
 (*query-header-supported-partial-union-good-sets*
 (*verifier-state-from-adversary attacker-state*
 (*staged-proof-transcript data*))
 query-sampling-success-space
 (*staged-trace-root data*)
 (*staged-trace-fri-roots data*)
 (*staged-trace-final data*)
 (*staged-alphas data*)
 (*staged-degree data*)
 (*staged-composition-fri-roots data*)
 (*staged-composition-final data*))) /
 nnreal (card query-sample-space) $\leq$ query-error-bound
 by (rule
 query-header-supported-partial-union-good-sets-fraction-bound-if-unique-candidate
 [*OF unique query-sampling-success-space-subset*
 query-sampling-success-space-fraction-bound-query-sample-space])

lemma *staged-dynamic-partial-query-set-target-error-bound-if-unique-candidate*:

assumes *raw-bound*: *query-index-raw-preimage-bound*

and *unique*:

$\exists$ *trace-table composition-table*.
 query-header-supported-partial-table-candidates
 (*verifier-state-from-adversary attacker-state*
 (*staged-proof-transcript data*))
 (*staged-trace-root data*)
 (*staged-trace-fri-roots data*)
 (*staged-trace-final data*)
 (*staged-alphas data*)
 (*staged-degree data*)
 (*staged-composition-fri-roots data*)
 (*staged-composition-final data*) $\subseteq$
 $\{(trace-table, composition-table)\}$

shows

staged-phase-target-error
 (*query-index-raw-preimage*
 (*query-header-supported-partial-union-good-sets*
 (*verifier-state-from-adversary attacker-state*
 (*staged-proof-transcript data*)))
query-sampling-success-space
 (*staged-trace-root data*)
 (*staged-trace-fri-roots data*)
 (*staged-trace-final data*)
 (*staged-alphas data*)
 (*staged-degree data*)
 (*staged-composition-fri-roots data*)
 (*staged-composition-final data*))) $q \leq$
nnreal $q * \text{query-error-bound}$
by (*rule staged-phase-query-index-target-error-query-bound*
 [*OF raw-bound staged-dynamic-partial-query-set-subset*
staged-dynamic-partial-query-set-fraction-bound-if-unique-candidate
 [*OF unique*]])

lemma *checked-staged-query-prefix-receive-bad-index-imp-new-output-hit:*
assumes *wf: staged-budget-wellformed budgets*
and *controlled: staged-adversary-controlled budgets A*
and *i-bound: $i \leq \text{rounds}$*
and *absent: query-index-raw-preimage-absent B s*
and *out:*
Some (raw, t) ∈
set-dist
(execute
(checked-staged-query-challenge-prefix-program A i $\gg$
*(λ -. *receive-query-index-challenge*)) s)*
and *bad: index (to-nat raw) ∈ B*
shows
hash-new-output-hit-event (query-index-raw-preimage B) s
(Some (raw, t))

proof –
obtain *prefix-out u where prefix:*
Some (prefix-out, u) ∈
set-dist
(execute (checked-staged-query-challenge-prefix-program A i) s)
and *recv:*
Some (raw, t) ∈ set-dist (execute receive-query-index-challenge u)
using *out by (auto elim!: set-dist-bindE)*
have *unchecked-prefix:*
Some
((spp-degree prefix-out, spp-composition-final prefix-out,
spp-query-chunks prefix-out), u) ∈
set-dist (execute (staged-query-challenge-prefix-program A i) s)
by (*rule checked-staged-query-challenge-prefix-outcome-imp-staged-query-prefix*
 [*OF prefix*])

```

have prefix-ext:  $s \leq u$ 
using hash-target-program-staged-query-challenge-prefix-program
  [OF uf controlled i-bound, of query-index-raw-preimage B]
unchecked-prefix
unfolding hash-target-program-def hash-extension-preserving-def
by blast
have recv-out:
   $u \leq t \wedge$ 
  fmlookup (HashMap t)
    (QueryIndexChallenge (PQueryCounter u) (PState u)) = Some raw
using receive-query-index-challenge-outcome[OF recv] by blast
have raw-in: raw  $\in$  query-index-raw-preimage B
using bad unfolding query-index-raw-preimage-def by simp
show ?thesis
proof (cases
  fmlookup (HashMap s)
    (QueryIndexChallenge (PQueryCounter u) (PState u)))
case None
then have hash-map-new-output-hit (query-index-raw-preimage B) s t
  unfolding hash-map-new-output-hit-def
  using recv-out raw-in by blast
then show ?thesis
  unfolding hash-new-output-hit-event-def by simp
next
case (Some old-raw)
have old-lookup-t:
  fmlookup (HashMap t)
    (QueryIndexChallenge (PQueryCounter u) (PState u)) =
    Some old-raw
by (rule hash-extension-lookup[OF Some])
(rule hash-ext-trans[OF prefix-ext conjunct1[OF recv-out]])
then have old-eq: old-raw = raw
  using recv-out by simp
have old-raw  $\notin$  query-index-raw-preimage B
  using absent Some
  unfolding query-index-raw-preimage-absent-def by blast
then show ?thesis
  using old-eq raw-in by simp
qed
qed

```

definition *checked-staged-query-prefix-with-state*

```

:: 'f staged-adversary  $\Rightarrow$  nat  $\Rightarrow$ 
  (('f staged-query-prefix-data  $\times$  'f protocol-channel),
   'f protocol-channel) state-monad
where
  checked-staged-query-prefix-with-state A i =
    checked-staged-query-challenge-prefix-program A i  $\gg=$ 
    ( $\lambda$ prefix. get  $\gg=$  ( $\lambda$ prefix-state.

```

return (prefix, prefix-state)))

definition *checked-staged-query-prefix-receive-with-state*

*:: 'f staged-adversary $\Rightarrow$ nat $\Rightarrow$
 (((('f staged-query-prefix-data $\times$ 'f protocol-channel) $\times$ 'f),
 'f protocol-channel) state-monad*

where

*checked-staged-query-prefix-receive-with-state A i =
 checked-staged-query-prefix-with-state A i $\gg$
 (λ prefix-pack. receive-query-index-challenge $\gg$ (λ raw.
 return (prefix-pack, raw)))*

lemma *checked-staged-query-program-decomp:*

assumes *i-bound: i < n*

shows

*checked-staged-query-program A trace-roots composition-roots start n =
 checked-staged-query-program A trace-roots composition-roots start i $\gg$
 (λ prefix-chunks.
 receive-query-index-challenge $\gg$ (λ raw.
 let idx = index (to-nat raw) in
 query-opening-stage A (start + i) raw $\gg$ (λ chunk.
 assert
 (verifier-query-round-chunk idx trace-roots composition-roots
 chunk) $\gg$ (λ -.
 record-staged-messages chunk $\gg$ (λ -.
 checked-staged-query-program A trace-roots composition-roots
 (Suc (start + i)) (n - Suc i) $\gg$ (λ suffix-chunks.
 return (prefix-chunks @ chunk # suffix-chunks))))))*

using *i-bound*

proof (*induction i arbitrary: start n*)

case *0*

then obtain *m where n-eq: n = Suc m*

by (*cases n*) *simp-all*

show *?case*

unfolding *n-eq*

by (*simp add: sm-bind-assoc Let-def*)

next

case (*Suc i*)

then obtain *m where n-eq: n = Suc m*

by (*cases n*) *simp-all*

have *i-lt-m: i < m*

using *Suc.prem*s **unfolding** *n-eq* **by** *simp*

have *IH:*

*checked-staged-query-program A trace-roots composition-roots
 (Suc start) m =
 checked-staged-query-program A trace-roots composition-roots
 (Suc start) i $\gg$
 (λ prefix-chunks.
 receive-query-index-challenge $\gg$ (λ raw.*

```

    let idx = index (to-nat raw) in
    query-opening-stage A (Suc start + i) raw  $\gg$  (λchunk.
    assert
      (verifier-query-round-chunk idx trace-roots composition-roots
       chunk)  $\gg$  (λ-.
    record-staged-messages chunk  $\gg$  (λ-.
    checked-staged-query-program A trace-roots composition-roots
      (Suc (Suc start + i)) (m - Suc i)  $\gg$  (λsuffix-chunks.
    return (prefix-chunks @ chunk # suffix-chunks))))))
  by (rule Suc.IH[OF i-lt-m])
show ?case
  unfolding n-eq
  by (simp add: sm-bind-assoc Let-def IH)
qed

```

```

lemma get-bind-const:
  (get  $\gg$  (λ-. m)) = m
proof transfer
  fix m
  show dist-bind dist-get (λ-. m) = m
proof
  fix s
  show dist-bind dist-get (λ-. m) s = m s
  apply (rule dist-inject[THEN iffD1])
  unfolding dist-bind.rep-eq dist-get-def bind-cont-map-def
  apply (simp add: o-def map-fun-def dist-delta-dist)
  apply (subst map-bind-delta-left-state[where f=λs. Some (s, s)])
  by (auto split: option.splits prod.splits)
qed
qed

```

```

lemma get-bind-return-state-support:
  Some ((x, s), s) ∈
    set-dist (execute (get  $\gg$  (λt. return (x, t))) s)
proof -
  have get-out: Some (s, s) ∈ set-dist (execute get s)
  unfolding set-dist-def get.rep-eq dist-get-def dist-delta-dist
    delta-map-def
  by simp
  have ret-out:
    Some ((x, s), s) ∈ set-dist (execute (return (x, s)) s)
  unfolding set-dist-def return.rep-eq dist-return-def dist-delta-dist
    delta-map-def
  by simp
  show ?thesis
  by (rule-tac x=s and t=s in set-dist-bindI)
    (rule get-out, rule ret-out)
qed

```

lemma *get-bind-return-state-outcome*:
assumes *outcome*:
Some ((x-out, captured), final) ∈
set-dist (execute (get $\gg$ ($\lambda t.$ return (x, t))) s)
shows $x\text{-out} = x \wedge \text{captured} = s \wedge \text{final} = s$
proof –
from *outcome* **obtain** $y\ u$ **where** *get-out*:
Some (y, u) ∈ set-dist (execute get s)
and *ret-out*:
Some ((x-out, captured), final) ∈
set-dist (execute (return (x, y)) u)
by (*auto elim!:* *set-dist-bindE*)
have $y\text{-}u$: $y = s \wedge u = s$
using *get-out*
unfolding *set-dist-def get.rep-eq dist-get-def dist-delta-dist*
delta-map-def
by *auto*
have *captured-final*: $x\text{-out} = x \wedge \text{captured} = y \wedge \text{final} = u$
using *ret-out*
unfolding *set-dist-def return.rep-eq dist-return-def dist-delta-dist*
delta-map-def
by *auto*
show *?thesis*
using $y\text{-}u$ *captured-final* **by** *simp*
qed

lemma *checked-staged-query-prefix-with-state-outcome-state*:
assumes *outcome*:
Some ((prefix, prefix-state), u) ∈
set-dist (execute (checked-staged-query-prefix-with-state A i) s)
shows $u = \text{prefix-state}$
proof –
show *?thesis*
using *outcome*
unfolding *checked-staged-query-prefix-with-state-def*
by (*auto dest!:* *get-bind-return-state-outcome*
elim!: *set-dist-bindE*)
qed

lemma *checked-staged-query-prefix-with-state-outcome-canonical*:
assumes *outcome*:
Some ((prefix, prefix-state), u) ∈
set-dist (execute (checked-staged-query-prefix-with-state A i) s)
shows
Some ((prefix, prefix-state), prefix-state) ∈
set-dist (execute (checked-staged-query-prefix-with-state A i) s)
using *outcome* *checked-staged-query-prefix-with-state-outcome-state*[*OF outcome*]
by *simp*

lemma *checked-staged-query-prefix-receive-with-state-prefix-support*:
assumes *outcome*:
Some (((prefix, prefix-state), raw), raw-state) ∈
set-dist
(execute (checked-staged-query-prefix-receive-with-state A i) s)
shows
Some ((prefix, prefix-state), prefix-state) ∈
set-dist (execute (checked-staged-query-prefix-with-state A i) s)
proof –
from *outcome* **obtain** *u* **where** *prefix-u*:
Some ((prefix, prefix-state), u) ∈
set-dist (execute (checked-staged-query-prefix-with-state A i) s)
unfolding *checked-staged-query-prefix-receive-with-state-def*
by (*auto elim!: set-dist-bindE*)
show *?thesis*
by (*rule checked-staged-query-prefix-with-state-outcome-canonical*
[OF prefix-u])
qed

lemma *checked-staged-query-prefix-receive-with-state-receive-support*:
assumes *outcome*:
Some (((prefix, prefix-state), raw), raw-state) ∈
set-dist
(execute (checked-staged-query-prefix-receive-with-state A i) s)
shows
Some (raw, raw-state) ∈
set-dist (execute receive-query-index-challenge prefix-state)
proof –
from *outcome* **obtain** *u* **where** *prefix-u*:
Some ((prefix, prefix-state), u) ∈
set-dist (execute (checked-staged-query-prefix-with-state A i) s)
and *recv-u*:
Some (raw, raw-state) ∈
set-dist (execute receive-query-index-challenge u)
unfolding *checked-staged-query-prefix-receive-with-state-def*
by (*auto elim!: set-dist-bindE*)
have *u = prefix-state*
by (*rule checked-staged-query-prefix-with-state-outcome-state*
[OF prefix-u])
then show *?thesis*
using *recv-u* **by** *simp*
qed

lemma *checked-staged-query-challenge-prefix-program-outcomeE*:
assumes *outcome*:
Some (prefix, prefix-state) ∈
set-dist
(execute (checked-staged-query-challenge-prefix-program A i)
adversary-initial-state)

obtains *fr s1 s2 trace-roots trace-bs s3 trace-final s4 s5 as s6 dg*
s7 s8 s9 composition-roots composition-bs s10 composition-final s11
query-start query-chunks **where**
Some (fr, s1) ∈
set-dist (execute (trace-root-stage A) adversary-initial-state)
Some ((), s2) ∈ set-dist (execute (record-staged-message fr) s1)
Some ((trace-roots, trace-bs), s3) ∈
set-dist
(execute
(staged-trace-fri-program A 0 (ceil-log clength) []) s2)
Some (trace-final, s4) ∈
set-dist (execute (trace-final-stage A trace-bs) s3)
Some ((), s5) ∈
set-dist (execute (record-staged-message trace-final) s4)
Some (as, s6) ∈
set-dist (execute (staged-alpha-program (length spec)) s5)
Some (dg, s7) ∈ set-dist (execute (degree-stage A as) s6)
Some ((), s8) ∈
set-dist (execute (record-staged-message dg) s7)
Some ((), s9) ∈
set-dist
(execute
(assert (ceil-log (to-nat dg + 1) ≤ ceil-log (maxDegree + 1)))
s8)
Some ((composition-roots, composition-bs), s10) ∈
set-dist
(execute
(staged-composition-fri-program A dg 0
(ceil-log (to-nat dg + 1)) []) s9)
Some (composition-final, s11) ∈
set-dist
(execute (composition-final-stage A dg composition-bs) s10)
Some ((), query-start) ∈
set-dist (execute (record-staged-message composition-final) s11)
Some (query-chunks, prefix-state) ∈
set-dist
(execute
(checked-staged-query-program A trace-roots composition-roots
0 i)
query-start)
prefix =
(|sqp-trace-root = fr,
sqp-trace-fri-roots = trace-roots,
sqp-trace-fri-challenges = trace-bs,
sqp-trace-final = trace-final,
sqp-alphas = as,
sqp-degree = dg,
sqp-composition-fri-roots = composition-roots,
sqp-composition-fri-challenges = composition-bs,

```

      sqp-composition-final = composition-final,
      sqp-query-chunks = query-chunks)
using outcome
unfolding checked-staged-query-challenge-prefix-program-def
      staged-alpha-prefix-program-def Let-def
by (auto elim!: set-dist-bindE intro: that split: prod.splits)

```

definition *checked-staged-after-query-prefix-receive*

```

:: 'f staged-adversary ⇒ nat ⇒
  (('f staged-query-prefix-data × 'f protocol-channel) × 'f) ⇒
  ('f staged-proof-data, 'f protocol-channel) state-monad
where
  checked-staged-after-query-prefix-receive A i prefix-pack =
    (case prefix-pack of ((prefix, -), raw) ⇒
      do {
        let idx = index (to-nat raw);
        chunk ← query-opening-stage A i raw;
        assert
          (verifier-query-round-chunk idx
            (sqp-trace-fri-roots prefix)
            (sqp-composition-fri-roots prefix) chunk);
        record-staged-messages chunk;
        suffix-chunks ←
          checked-staged-query-program A
            (sqp-trace-fri-roots prefix)
            (sqp-composition-fri-roots prefix)
            (Suc i) (rounds - Suc i);
        return
          (|staged-trace-root = sqp-trace-root prefix,
            staged-trace-fri-roots = sqp-trace-fri-roots prefix,
            staged-trace-fri-challenges =
              sqp-trace-fri-challenges prefix,
            staged-trace-final = sqp-trace-final prefix,
            staged-alphas = sqp-alphas prefix,
            staged-degree = sqp-degree prefix,
            staged-composition-fri-roots =
              sqp-composition-fri-roots prefix,
            staged-composition-fri-challenges =
              sqp-composition-fri-challenges prefix,
            staged-composition-final = sqp-composition-final prefix,
            staged-query-chunks =
              sqp-query-chunks prefix @ chunk # suffix-chunks|)
      })

```

definition *checked-staged-after-query-prefix-receive-with-verifier*

```

:: 'f staged-adversary ⇒ nat ⇒
  (('f staged-query-prefix-data × 'f protocol-channel) × 'f) ⇒
  (((('f staged-proof-data × 'f protocol-channel) × unit list),
    'f protocol-channel) state-monad

```

where

checked-staged-after-query-prefix-receive-with-verifier A i prefix-pack =
checked-staged-after-query-prefix-receive A i prefix-pack $\gg$
 $(\lambda data.$
 $get \gg (\lambda s.$
 put
 $(verifier-state-from-adversary s$
 $(staged-proof-transcript data)) \gg$
 $(\lambda.$
 $verify-monad \gg$
 $(\lambda result. return ((data, s), result))))))$

lemma *checked-staged-after-query-prefix-receive-with-verifier-outcome:*

assumes *outcome:*

Some (((data, attacker-state), result), final-state) ∈
set-dist
(execute
(checked-staged-after-query-prefix-receive-with-verifier A i
prefix-pack)
s)

shows

Some (data, attacker-state) ∈
set-dist
(execute
(checked-staged-after-query-prefix-receive A i prefix-pack) s)
Some (result, final-state) ∈
set-dist
(execute verify-monad
(verifier-state-from-adversary attacker-state
(staged-proof-transcript data))))

proof –

from *outcome[unfolded*

checked-staged-after-query-prefix-receive-with-verifier-def]

obtain *data' attacker-state' where data':*

Some (data', attacker-state') ∈
set-dist
(execute
(checked-staged-after-query-prefix-receive A i prefix-pack) s)

and *rest:*

Some (((data, attacker-state), result), final-state) ∈
set-dist
(execute
(get $\gg$
($\lambda sa.$
put
(verifier-state-from-adversary sa
(staged-proof-transcript data')) $\gg$
($\lambda.$
verify-monad $\gg$

```

      (λr. return ((data', sa), r))))
    attacker-state')
  by (rule set-dist-bindE) (rule that)
from rest obtain saved-state state-after-get where get-out:
  Some (saved-state, state-after-get) ∈
    set-dist (execute get attacker-state')
and after-get:
  Some (((data, attacker-state), result), final-state) ∈
    set-dist
      (execute
        (put
          (verifier-state-from-adversary saved-state
            (staged-proof-transcript data'))) >>=
          (λ-.
            verify-monad >>=
              (λr. return ((data', saved-state), r))))
        state-after-get)
  by (rule set-dist-bindE) (rule that)
from get-out have saved-state-eq: saved-state = attacker-state'
  and state-after-get-eq: state-after-get = attacker-state'
  by simp-all
from after-get[unfolded saved-state-eq state-after-get-eq]
obtain u verifier-start where put-out:
  Some (u, verifier-start) ∈
    set-dist
      (execute
        (put
          (verifier-state-from-adversary attacker-state'
            (staged-proof-transcript data')))
        attacker-state')
  and after-put:
  Some (((data, attacker-state), result), final-state) ∈
    set-dist
      (execute
        (verify-monad >>=
          (λr. return ((data', attacker-state'), r)))
        verifier-start)
  by (rule set-dist-bindE) (rule that)
from put-out have verifier-start-eq:
  verifier-start =
    verifier-state-from-adversary attacker-state'
    (staged-proof-transcript data')
  by simp
from after-put[unfolded verifier-start-eq]
obtain result' final-state' where verifier:
  Some (result', final-state') ∈
    set-dist
      (execute verify-monad
        (verifier-state-from-adversary attacker-state'

```

```

      (staged-proof-transcript data'))))
and returned:
  Some (((data, attacker-state), result), final-state) ∈
    set-dist
    (execute (return ((data', attacker-state'), result'))
      final-state')
by (elim set-dist-bindE)
from returned have data-eq: data = data'
and attacker-state-eq: attacker-state = attacker-state'
and result-eq: result = result'
and final-state-eq: final-state = final-state'
by simp-all
show
  Some (data, attacker-state) ∈
    set-dist
    (execute
      (checked-staged-after-query-prefix-receive A i prefix-pack) s)
using data' unfolding data-eq attacker-state-eq by simp
show
  Some (result, final-state) ∈
    set-dist
    (execute verify-monad
      (verifier-state-from-adversary attacker-state
        (staged-proof-transcript data)))
using verifier
unfolding data-eq attacker-state-eq result-eq final-state-eq
by simp
qed

```

lemma *checked-staged-security-experiment-with-data-state-outcomeI*:

assumes *builder*:

```

  Some (data, attacker-state) ∈
    set-dist
    (execute (checked-staged-transcript-program A)
      adversary-initial-state)

```

and *verifier*:

```

  Some (result, final-state) ∈
    set-dist
    (execute verify-monad
      (verifier-state-from-adversary attacker-state
        (staged-proof-transcript data)))

```

shows

```

  Some (((data, attacker-state), result), final-state) ∈
    set-dist
    (execute
      (checked-staged-security-experiment-with-data-state A)
      adversary-initial-state)

```

unfolding *checked-staged-security-experiment-with-data-state-def*

apply (rule-tac *x=data and t=attacker-state in set-dist-bindI*)

```

  apply (rule builder)
  apply (rule-tac x=attacker-state and t=attacker-state in set-dist-bindI)
  apply simp
  apply (rule-tac x=() and
    t=verifier-state-from-adversary attacker-state
    (staged-proof-transcript data) in set-dist-bindI)
  apply simp
  apply (rule-tac x=result and t=final-state in set-dist-bindI)
  apply (rule verifier)
  apply simp
done

```

lemma *checked-staged-transcript-program-query-prefix-receive-decomp:*

assumes *i-bound: i < rounds*

shows

checked-staged-transcript-program A =
checked-staged-query-prefix-receive-with-state A i $\gg$
checked-staged-after-query-prefix-receive A i

proof –

have *query-decomp:*

$\bigwedge$ *trace-roots composition-roots.*
checked-staged-query-program A trace-roots composition-roots 0 rounds =
checked-staged-query-program A trace-roots composition-roots 0 i $\gg$
 $(\lambda$ *prefix-chunks.*
receive-query-index-challenge $\gg$ $(\lambda$ *raw.*
let idx = index (to-nat raw) in
query-opening-stage A (0 + i) raw $\gg$ $(\lambda$ *chunk.*
assert
(verifier-query-round-chunk idx trace-roots composition-roots
chunk) $\gg$ $(\lambda$ -.
record-staged-messages chunk $\gg$ $(\lambda$ -.
checked-staged-query-program A trace-roots composition-roots
(Suc (0 + i)) (rounds - Suc i) $\gg$ $(\lambda$ *suffix-chunks.*
return (prefix-chunks @ chunk # suffix-chunks))))))

by (rule *checked-staged-query-program-decomp*[*OF i-bound*])

show *?thesis*

unfolding *checked-staged-transcript-program-def*
checked-staged-query-prefix-receive-with-state-def
checked-staged-query-prefix-with-state-def
checked-staged-query-challenge-prefix-program-def
staged-alpha-prefix-program-def
checked-staged-after-query-prefix-receive-def

apply (*simp add: sm-bind-assoc Let-def split-def query-decomp*
get-bind-const)

done

qed

lemma *checked-staged-transcript-program-query-prefix-receive-cont-support:*

assumes *i-bound: i < rounds*

and *outcome*:
Some (*data*, *attacker-state*) $\in$
set-dist
(execute (checked-staged-transcript-program A)
adversary-initial-state)
obtains *prefix prefix-state raw raw-state* **where**
Some (((*prefix*, *prefix-state*), *raw*), *raw-state*) $\in$
set-dist
(execute (checked-staged-query-prefix-receive-with-state A i)
adversary-initial-state)
Some (*data*, *attacker-state*) $\in$
set-dist
(execute
(checked-staged-after-query-prefix-receive A i
((prefix, prefix-state), raw))
raw-state)
proof –
have *decomp*:
checked-staged-transcript-program A =
checked-staged-query-prefix-receive-with-state A i $\gg$
checked-staged-after-query-prefix-receive A i
by (*rule checked-staged-transcript-program-query-prefix-receive-decomp*
[OF i-bound])
from *outcome[unfolded decomp]*
obtain *x raw-state* **where** *head*:
Some (*x*, *raw-state*) $\in$
set-dist
(execute (checked-staged-query-prefix-receive-with-state A i)
adversary-initial-state)
and *tail*:
Some (*data*, *attacker-state*) $\in$
set-dist
(execute
(checked-staged-after-query-prefix-receive A i x)
raw-state)
by (*auto elim!:* *set-dist-bindE*)
obtain *prefix-pack raw* **where** *x-eq*: *x = (prefix-pack, raw)*
by (*cases x*) *simp*
obtain *prefix prefix-state* **where** *prefix-pack-eq*:
prefix-pack = (prefix, prefix-state)
by (*cases prefix-pack*) *simp*
show *?thesis*
proof (*rule that[of prefix prefix-state raw raw-state]*)
show *Some* (((*prefix*, *prefix-state*), *raw*), *raw-state*) $\in$
set-dist
(execute (checked-staged-query-prefix-receive-with-state A i)
adversary-initial-state)
using *head unfolding x-eq prefix-pack-eq* **by** *simp*
show *Some* (*data*, *attacker-state*) $\in$

```

    set-dist
    (execute
      (checked-staged-after-query-prefix-receive A i
        ((prefix, prefix-state), raw))
      raw-state)
  using tail unfolding x-eq prefix-pack-eq by simp
qed
qed

lemma checked-staged-security-experiment-with-data-state-query-prefix-receive-decomp:
  assumes i-bound:  $i < \text{rounds}$ 
  shows
    checked-staged-security-experiment-with-data-state A =
      checked-staged-query-prefix-receive-with-state A i  $\gg$ 
      checked-staged-after-query-prefix-receive-with-verifier A i
  unfolding checked-staged-security-experiment-with-data-state-def
    checked-staged-transcript-program-query-prefix-receive-decomp[OF i-bound]
    checked-staged-after-query-prefix-receive-with-verifier-def
  by (simp add: sm-bind-assoc)

definition staged-query-prefix-start-hash
  :: 'f staged-query-prefix-data  $\Rightarrow$  'f
  where
    staged-query-prefix-start-hash prefix =
      concat
        (foldl concat
          (concat
            (foldl concat
              (concat
                (foldl concat
                  (concat 0 (sqp-trace-root prefix))
                  (sqp-trace-fri-roots prefix))
                  (sqp-trace-final prefix))
                  (sqp-alphas prefix))
                  (sqp-degree prefix))
                  (sqp-composition-fri-roots prefix))
                  (sqp-composition-final prefix)
            )
          )
        )

lemma checked-staged-query-prefix-with-state-query-chunks-length:
  assumes wf: staged-budget-wellformed budgets
  and controlled: staged-adversary-controlled budgets A
  and i-bound:  $i < \text{rounds}$ 
  and support:
    Some ((prefix, prefix-state), prefix-state)  $\in$ 
      set-dist
        (execute (checked-staged-query-prefix-with-state A i)
          adversary-initial-state)
  shows length (sqp-query-chunks prefix) = i
proof –

```



```

from support have prefix-out:
  Some (prefix, prefix-state) ∈
    set-dist
      (execute (checked-staged-query-challenge-prefix-program A i)
        adversary-initial-state)
  unfolding checked-staged-query-prefix-with-state-def
  by (auto elim!: set-dist-bindE)
from prefix-out obtain fr s1 s2 trace-roots trace-bs s3 trace-final s4
  s5 as s6 dg s7 s8 s9 composition-roots composition-bs s10
  composition-final s11 query-start query-chunks where
  query-out:
    Some (query-chunks, prefix-state) ∈
      set-dist
        (execute
          (checked-staged-query-program A trace-roots composition-roots
            0 i) s11)
  and prefix-eq:
    prefix =
      (|spp-trace-root = fr,
        spp-trace-fri-roots = trace-roots,
        spp-trace-fri-challenges = trace-bs,
        spp-trace-final = trace-final,
        spp-alphas = as,
        spp-degree = dg,
        spp-composition-fri-roots = composition-roots,
        spp-composition-fri-challenges = composition-bs,
        spp-composition-final = composition-final,
        spp-query-chunks = query-chunks|)
  using prefix-out
  by (elim checked-staged-query-challenge-prefix-program-outcomeE)
have bound: 0 + i ≤ length (query-opening-budgets budgets)
  using wf i-bound unfolding staged-budget-wellformed-def by simp
from checked-staged-query-program-outcome-with-raws
  [OF controlled bound query-out]
obtain raw-idxs query-idxs where len: length query-chunks = i
  by blast
then show ?thesis
  unfolding prefix-eq by simp
qed

```

```

lemma checked-staged-query-prefix-with-state-alignment:
  assumes wf: staged-budget-wellformed budgets
  and controlled: staged-adversary-controlled budgets A
  and i-bound: i < rounds
  and support:
    Some ((prefix, prefix-state), prefix-state) ∈
      set-dist
        (execute (checked-staged-query-prefix-with-state A i)
          adversary-initial-state)

```

shows

$PQueryCounter\ prefix\text{-}state = i \wedge$
 $PState\ prefix\text{-}state =$
 $state\text{-}after\text{-}query\text{-}chunks$
 $(staged\text{-}query\text{-}prefix\text{-}start\text{-}hash\ prefix)$
 $(sqp\text{-}query\text{-}chunks\ prefix)\ i \wedge$
 $length\ (sqp\text{-}query\text{-}chunks\ prefix) = i$

proof –

from *support* **have** *prefix-out*:

$Some\ (prefix,\ prefix\text{-}state) \in$
 $set\text{-}dist$
 $(execute\ (checked\text{-}staged\text{-}query\text{-}challenge\text{-}prefix\text{-}program\ A\ i)$
 $adversary\text{-}initial\text{-}state)$

unfolding *checked-staged-query-prefix-with-state-def*

by (*auto elim!:* *set-dist-bindE*)

from *prefix-out* **obtain** *fr s1 s2 trace-roots trace-bs s3 trace-final s4*

s5 as s6 dg s7 s8 s9 composition-roots composition-bs s10

composition-final s11 query-start query-chunks **where**

root-out:

$Some\ (fr,\ s1) \in$
 $set\text{-}dist\ (execute\ (trace\text{-}root\text{-}stage\ A)\ adversary\text{-}initial\text{-}state)$

and *record-root*:

$Some\ ((),\ s2) \in set\text{-}dist\ (execute\ (record\text{-}staged\text{-}message\ fr)\ s1)$

and *trace-out*:

$Some\ ((trace\text{-}roots,\ trace\text{-}bs),\ s3) \in$
 $set\text{-}dist$
 $(execute$
 $(staged\text{-}trace\text{-}fri\text{-}program\ A\ 0\ (ceil\text{-}log\ clength)\ [])\ s2)$

and *final-out*:

$Some\ (trace\text{-}final,\ s4) \in$
 $set\text{-}dist\ (execute\ (trace\text{-}final\text{-}stage\ A\ trace\text{-}bs)\ s3)$

and *record-final*:

$Some\ ((),\ s5) \in$
 $set\text{-}dist\ (execute\ (record\text{-}staged\text{-}message\ trace\text{-}final)\ s4)$

and *alpha-out*:

$Some\ (as,\ s6) \in$
 $set\text{-}dist\ (execute\ (staged\text{-}alpha\text{-}program\ (length\ spec))\ s5)$

and *degree-out*:

$Some\ (dg,\ s7) \in set\text{-}dist\ (execute\ (degree\text{-}stage\ A\ as)\ s6)$

and *record-degree*:

$Some\ ((),\ s8) \in set\text{-}dist\ (execute\ (record\text{-}staged\text{-}message\ dg)\ s7)$

and *assert-out*:

$Some\ ((),\ s9) \in$
 $set\text{-}dist$
 $(execute$
 $(assert$
 $(ceil\text{-}log\ (to\text{-}nat\ dg + 1) \leq$
 $ceil\text{-}log\ (maxDegree + 1)))\ s8)$

and *composition-out*:

```

Some ((composition-roots, composition-bs), s10) ∈
  set-dist
    (execute
      (staged-composition-fri-program A dg 0
        (ceil-log (to-nat dg + 1)) [] s9)
    and composition-final-out:
      Some (composition-final, s11) ∈
        set-dist
          (execute (composition-final-stage A dg composition-bs) s10)
    and record-composition-final:
      Some ((), query-start) ∈
        set-dist (execute (record-staged-message composition-final) s11)
    and query-out:
      Some (query-chunks, prefix-state) ∈
        set-dist
          (execute
            (checked-staged-query-program A trace-roots composition-roots
              0 i)
            query-start)
    and prefix-eq:
      prefix =
        (|sqp-trace-root = fr,
          sqp-trace-fri-roots = trace-roots,
          sqp-trace-fri-challenges = trace-bs,
          sqp-trace-final = trace-final,
          sqp-alphas = as,
          sqp-degree = dg,
          sqp-composition-fri-roots = composition-roots,
          sqp-composition-fri-challenges = composition-bs,
          sqp-composition-final = composition-final,
          sqp-query-chunks = query-chunks|)
    using prefix-out
    by (elim checked-staged-query-challenge-prefix-program-outcomeE)
have root-controlled:
  controlled-ro-program (trace-root-budget budgets) (trace-root-stage A)
  using controlled unfolding staged-adversary-controlled-def by simp
have trace-final-controlled:
  controlled-ro-program (trace-final-budget budgets)
    (trace-final-stage A trace-bs)
  using controlled unfolding staged-adversary-controlled-def by blast
have degree-controlled:
  controlled-ro-program (degree-budget budgets) (degree-stage A as)
  using controlled unfolding staged-adversary-controlled-def by blast
have composition-final-controlled:
  controlled-ro-program (composition-final-budget budgets)
    (composition-final-stage A dg composition-bs)
  using controlled unfolding staged-adversary-controlled-def by blast
have root-state: PState s1 = 0
  using controlled-stage-outcome-fields[OF root-controlled root-out]

```

```

  by simp
have s2-eq:
  s2 = s1(
    PState := concat (PState s1) fr,
    PTranscript := PTranscript s1 @ [fr])
  by (rule record-staged-message-outcome[OF record-root])
have trace-bound:
  0 + ceil-log clength ≤ length (trace-fri-budgets budgets)
  using wf unfolding staged-budget-wellformed-def by simp
have trace-alignment:
  PState s3 = foldl concat (PState s2) trace-roots
  using staged-trace-fri-program-alignment
    [OF controlled trace-bound trace-out]
  by simp
have state-s3:
  PState s3 = foldl concat (concat 0 fr) trace-roots
  using root-state trace-alignment unfolding s2-eq by simp
have state-s4: PState s4 = PState s3
  using controlled-stage-outcome-fields
    [OF trace-final-controlled final-out]
  by simp
have s5-eq:
  s5 = s4(
    PState := concat (PState s4) trace-final,
    PTranscript := PTranscript s4 @ [trace-final])
  by (rule record-staged-message-outcome[OF record-final])
have alpha-alignment:
  PState s6 = foldl concat (PState s5) as
  using staged-alpha-program-alignment[OF alpha-out] by simp
have state-s6:
  PState s6 =
    foldl concat
      (concat (foldl concat (concat 0 fr) trace-roots) trace-final)
    as
  using state-s3 state-s4 alpha-alignment unfolding s5-eq by simp
have state-s7: PState s7 = PState s6
  using controlled-stage-outcome-fields[OF degree-controlled degree-out]
  by simp
have s8-eq:
  s8 = s7(
    PState := concat (PState s7) dg,
    PTranscript := PTranscript s7 @ [dg])
  by (rule record-staged-message-outcome[OF record-degree])
have s9-eq: s9 = s8
  using assert-out unfolding assert-def
  by (cases
    ceil-log (to-nat dg + 1) ≤ ceil-log (maxDegree + 1))
    (auto simp: throw-no-outcome)
have round-bound:

```

```

ceil-log (to-nat dg + 1) ≤ ceil-log (maxDegree + 1)
using assert-out unfolding assert-def
by (cases
  ceil-log (to-nat dg + 1) ≤ ceil-log (maxDegree + 1))
  (auto simp; throw-no-outcome)
have composition-bound:
  0 + ceil-log (to-nat dg + 1) ≤
    length (composition-fri-budgets budgets)
  using wf round-bound unfolding staged-budget-wellformed-def by simp
have composition-alignment:
  PState s10 = foldl concat (PState s9) composition-roots
  using staged-composition-fri-program-alignment
    [OF controlled composition-bound composition-out]
  by simp
have state-s10:
  PState s10 =
    foldl concat
      (concat
        (foldl concat
          (concat (foldl concat (concat 0 fr) trace-roots) trace-final)
          as)
        dg)
        composition-roots
  using state-s6 state-s7 composition-alignment
  unfolding s8-eq s9-eq by simp
have state-s11: PState s11 = PState s10
  using controlled-stage-outcome-fields
    [OF composition-final-controlled composition-final-out]
  by simp
have query-start-eq:
  query-start = s11(
    PState := concat (PState s11) composition-final,
    PTranscript := PTranscript s11 @ [composition-final])
  by (rule record-staged-message-outcome [OF record-composition-final])
have start-state:
  PState query-start = staged-query-prefix-start-hash prefix
  unfolding prefix-eq staged-query-prefix-start-hash-def
  using state-s10 state-s11 unfolding query-start-eq by simp
have query-counter-start: PQueryCounter query-start = 0
proof –
  have root-counter:
    PQueryCounter s1 = PQueryCounter adversary-initial-state
    using controlled-stage-outcome-fields [OF root-controlled root-out]
    by simp
  have trace-counter:
    PQueryCounter s3 = PQueryCounter s2
    using staged-trace-fri-program-alignment
      [OF controlled trace-bound trace-out]
    by simp

```

```

have final-counter: PQueryCounter s4 = PQueryCounter s3
  using controlled-stage-outcome-fields
    [OF trace-final-controlled final-out]
  by simp
have alpha-counter: PQueryCounter s6 = PQueryCounter s5
  using staged-alpha-program-alignment[OF alpha-out] by simp
have degree-counter: PQueryCounter s7 = PQueryCounter s6
  using controlled-stage-outcome-fields[OF degree-controlled degree-out]
  by simp
have comp-counter: PQueryCounter s10 = PQueryCounter s9
  using staged-composition-fri-program-alignment
    [OF controlled composition-bound composition-out]
  by simp
have comp-final-counter: PQueryCounter s11 = PQueryCounter s10
  using controlled-stage-outcome-fields
    [OF composition-final-controlled composition-final-out]
  by simp
show ?thesis
  using root-counter trace-counter final-counter alpha-counter
    degree-counter comp-counter comp-final-counter
  unfolding s2-eq s5-eq s8-eq s9-eq query-start-eq by simp
qed
have query-bound:  $0 + i \leq \text{length } (\text{query-opening-budgets budgets})$ 
  using wf i-bound unfolding staged-budget-wellformed-def by simp
from checked-staged-query-program-outcome-with-raws
  [OF controlled query-bound query-out]
obtain raw-idxs query-idxs where
  len:  $\text{length } \text{query-chunks} = i$ 
  and state:
    PState prefix-state =
      state-after-query-chunks (PState query-start) query-chunks i
  and counter:
    PQueryCounter prefix-state = PQueryCounter query-start + i
  by blast
show ?thesis
  using start-state query-counter-start state counter len
  unfolding prefix-eq by simp
qed

lemma checked-staged-query-program-prefix-receive-support:
assumes i-bound:  $i < n$ 
and outcome:
  Some (chunks, t) ∈
    set-dist
      (execute
        (checked-staged-query-program A trace-roots composition-roots
          start n) s)
obtains prefix-chunks prefix-state raw raw-state chunk chunk-state
  record-state suffix-chunks

```

where

Some (*prefix-chunks*, *prefix-state*) ∈
set-dist
 (*execute*
 (*checked-staged-query-program* *A* *trace-roots* *composition-roots*
start i) *s*)
Some (*raw*, *raw-state*) ∈
set-dist (*execute* *receive-query-index-challenge* *prefix-state*)
Some (*chunk*, *chunk-state*) ∈
set-dist
 (*execute* (*query-opening-stage* *A* (*start* + *i*) *raw*) *raw-state*)
verifier-query-round-chunk (*index* (*to-nat* *raw*))
trace-roots *composition-roots* *chunk*
Some (*()*, *record-state*) ∈
set-dist (*execute* (*record-staged-messages* *chunk*) *chunk-state*)
Some (*suffix-chunks*, *t*) ∈
set-dist
 (*execute*
 (*checked-staged-query-program* *A* *trace-roots* *composition-roots*
 (*Suc* (*start* + *i*)) (*n* − *Suc i*) *record-state*)
chunks = *prefix-chunks* @ *chunk* # *suffix-chunks*

proof −

have *decomp*:

checked-staged-query-program *A* *trace-roots* *composition-roots* *start n* =
checked-staged-query-program *A* *trace-roots* *composition-roots* *start i* ≫=
 (λ*prefix-chunks*.
receive-query-index-challenge ≫= (λ*raw*.
let *idx* = *index* (*to-nat* *raw*) *in*
query-opening-stage *A* (*start* + *i*) *raw* ≫= (λ*chunk*.
assert
 (*verifier-query-round-chunk* *idx* *trace-roots* *composition-roots*
chunk) ≫= (λ-.
record-staged-messages *chunk* ≫= (λ-.
checked-staged-query-program *A* *trace-roots* *composition-roots*
 (*Suc* (*start* + *i*)) (*n* − *Suc i*) ≫= (λ*suffix-chunks*.
return (*prefix-chunks* @ *chunk* # *suffix-chunks*))))))

by (*rule checked-staged-query-program-decomp*[*OF i-bound*])

from *outcome*[*unfolded decomp*]

obtain *prefix-chunks* *prefix-state* *raw* *raw-state* *chunk* *chunk-state*

record-state *suffix-chunks* **where**

prefix-out:

Some (*prefix-chunks*, *prefix-state*) ∈
set-dist
 (*execute*
 (*checked-staged-query-program* *A* *trace-roots* *composition-roots*
start i) *s*)

and *raw-out*:

Some (*raw*, *raw-state*) ∈
set-dist (*execute* *receive-query-index-challenge* *prefix-state*)

and *chunk-out*:
Some (chunk, chunk-state) ∈
set-dist
(execute (query-opening-stage A (start + i) raw) raw-state)
and *chunk-shape*:
verifier-query-round-chunk (index (to-nat raw))
trace-roots composition-roots chunk
and *record-out*:
Some ((), record-state) ∈
set-dist (execute (record-staged-messages chunk) chunk-state)
and *suffix-out*:
Some (suffix-chunks, t) ∈
set-dist
(execute
(checked-staged-query-program A trace-roots composition-roots
(Suc (start + i)) (n - Suc i) record-state)
and *chunks-eq*: *chunks = prefix-chunks @ chunk # suffix-chunks*
by (*auto simp: assert-def throw-no-outcome elim!: set-dist-bindE*
split: if-splits)
show *?thesis*
by (*rule that[OF prefix-out raw-out chunk-out chunk-shape record-out*
suffix-out chunks-eq])
qed

lemma *state-after-query-chunks-append-prefix*:
assumes *len-prefix: length prefix = i*
shows
state-after-query-chunks st (prefix @ suffix) i =
state-after-query-chunks st prefix i
using *len-prefix*
unfolding *state-after-query-chunks-def* **by** *simp*

lemma *checked-staged-query-program-prefix-receive-state-alignment*:
assumes *controlled: staged-adversary-controlled budgets A*
and *bound: start + n ≤ length (query-opening-budgets budgets)*
and *i-bound: i < n*
and *outcome*:
Some (chunks, t) ∈
set-dist
(execute
(checked-staged-query-program A trace-roots composition-roots
start n) s)
obtains *prefix-chunks prefix-state raw raw-state chunk chunk-state*
record-state suffix-chunks
where
Some (prefix-chunks, prefix-state) ∈
set-dist
(execute
(checked-staged-query-program A trace-roots composition-roots


```

      start i) s)
Some (raw, raw-state) ∈
  set-dist (execute receive-query-index-challenge prefix-state)
Some (chunk, chunk-state) ∈
  set-dist
    (execute (query-opening-stage A (start + i) raw) raw-state)
verifier-query-round-chunk (index (to-nat raw))
  trace-roots composition-roots chunk
Some ((), record-state) ∈
  set-dist (execute (record-staged-messages chunk) chunk-state)
Some (suffix-chunks, t) ∈
  set-dist
    (execute
      (checked-staged-query-program A trace-roots composition-roots
        (Suc (start + i)) (n - Suc i)) record-state)
chunks = prefix-chunks @ chunk # suffix-chunks
PQueryCounter prefix-state = PQueryCounter s + i
PState prefix-state = state-after-query-chunks (PState s) chunks i
proof -
from checked-staged-query-program-prefix-receive-support
  [OF i-bound outcome]
obtain prefix-chunks prefix-state raw raw-state chunk chunk-state
  record-state suffix-chunks where
  prefix-out:
    Some (prefix-chunks, prefix-state) ∈
      set-dist
        (execute
          (checked-staged-query-program A trace-roots composition-roots
            start i) s)
and raw-out:
    Some (raw, raw-state) ∈
      set-dist (execute receive-query-index-challenge prefix-state)
and chunk-out:
    Some (chunk, chunk-state) ∈
      set-dist
        (execute (query-opening-stage A (start + i) raw) raw-state)
and chunk-shape:
    verifier-query-round-chunk (index (to-nat raw))
      trace-roots composition-roots chunk
and record-out:
    Some ((), record-state) ∈
      set-dist (execute (record-staged-messages chunk) chunk-state)
and suffix-out:
    Some (suffix-chunks, t) ∈
      set-dist
        (execute
          (checked-staged-query-program A trace-roots composition-roots
            (Suc (start + i)) (n - Suc i)) record-state)
and chunks-eq: chunks = prefix-chunks @ chunk # suffix-chunks

```

```

    by (rule checked-staged-query-program-prefix-receive-support
        [OF i-bound outcome])
  have prefix-bound: start + i ≤ length (query-opening-budgets budgets)
    using bound i-bound by simp
  from checked-staged-query-program-outcome-with-raws
    [OF controlled prefix-bound prefix-out]
  obtain raw-prefix query-prefix where
    len-prefix: length prefix-chunks = i
  and state-prefix:
    PState prefix-state =
      state-after-query-chunks (PState s) prefix-chunks i
  and counter-prefix:
    PQueryCounter prefix-state = PQueryCounter s + i
  by blast
  have state-full:
    PState prefix-state = state-after-query-chunks (PState s) chunks i
  using state-prefix len-prefix
  unfolding chunks-eq
  by (simp add: state-after-query-chunks-append-prefix)
  show ?thesis
    by (rule that[OF prefix-out raw-out chunk-out chunk-shape record-out
        suffix-out chunks-eq counter-prefix state-full])
qed

```

lemma *checked-staged-transcript-program-query-prefix-receive-support:*

```

  assumes wf: staged-budget-wellformed budgets
  and controlled: staged-adversary-controlled budgets A
  and i-bound: i < rounds
  and outcome:
    Some (data, attacker-state) ∈
      set-dist
        (execute (checked-staged-transcript-program A)
          adversary-initial-state)
  obtains prefix prefix-state raw raw-state
  where
    Some (((prefix, prefix-state), raw), raw-state) ∈
      set-dist
        (execute (checked-staged-query-prefix-receive-with-state A i)
          adversary-initial-state)
  sqp-trace-root prefix = staged-trace-root data
  sqp-trace-fri-roots prefix = staged-trace-fri-roots data
  sqp-trace-fri-challenges prefix =
    staged-trace-fri-challenges data
  sqp-trace-final prefix = staged-trace-final data
  sqp-alphas prefix = staged-alphas data
  sqp-degree prefix = staged-degree data
  sqp-composition-fri-roots prefix =
    staged-composition-fri-roots data
  sqp-composition-fri-challenges prefix =

```

```

    staged-composition-fri-challenges data
  sqp-composition-final prefix = staged-composition-final data
  sqp-query-chunks prefix = take i (staged-query-chunks data)
  PQueryCounter prefix-state = i
  PState prefix-state =
    state-after-query-chunks
    (staged-query-start-hash data) (staged-query-chunks data) i
  prefix-state ≤ attacker-state
  raw-state ≤ attacker-state
  fmlookup (HashMap raw-state)
    (QueryIndexChallenge i
    (state-after-query-chunks
    (staged-query-start-hash data) (staged-query-chunks data) i)) =
    Some raw
proof –
from outcome obtain fr s1 s2 trace-roots trace-bs s3 trace-final s4 s5
  as s6 dg s7 s8 s9 composition-roots composition-bs s10
  composition-final s11 query-start query-chunks where
  root-out:
    Some (fr, s1) ∈
      set-dist (execute (trace-root-stage A) adversary-initial-state)
  and record-root:
    Some ((), s2) ∈ set-dist (execute (record-staged-message fr) s1)
  and trace-out:
    Some ((trace-roots, trace-bs), s3) ∈
      set-dist
      (execute
      (staged-trace-fri-program A 0 (ceil-log clength) []) s2)
  and final-out:
    Some (trace-final, s4) ∈
      set-dist (execute (trace-final-stage A trace-bs) s3)
  and record-final:
    Some ((), s5) ∈
      set-dist (execute (record-staged-message trace-final) s4)
  and alpha-out:
    Some (as, s6) ∈
      set-dist (execute (staged-alpha-program (length spec)) s5)
  and degree-out:
    Some (dg, s7) ∈ set-dist (execute (degree-stage A as) s6)
  and record-degree:
    Some ((), s8) ∈ set-dist (execute (record-staged-message dg) s7)
  and assert-out:
    Some ((), s9) ∈
      set-dist
      (execute
      (assert
      (ceil-log (to-nat dg + 1) ≤
      ceil-log (maxDegree + 1))) s8)
  and composition-out:

```

```

Some ((composition-roots, composition-bs), s10) ∈
  set-dist
    (execute
      (staged-composition-fri-program A dg 0
        (ceil-log (to-nat dg + 1)) [] s9)
    and composition-final-out:
      Some (composition-final, s11) ∈
        set-dist
          (execute (composition-final-stage A dg composition-bs) s10)
    and record-composition-final:
      Some ((), query-start) ∈
        set-dist (execute (record-staged-message composition-final) s11)
    and query-out:
      Some (query-chunks, attacker-state) ∈
        set-dist
          (execute
            (checked-staged-query-program A trace-roots composition-roots
              0 rounds)
            query-start)
    and data-eq:
      data =
        (|staged-trace-root = fr,
          staged-trace-fri-roots = trace-roots,
          staged-trace-fri-challenges = trace-bs,
          staged-trace-final = trace-final,
          staged-alphas = as,
          staged-degree = dg,
          staged-composition-fri-roots = composition-roots,
          staged-composition-fri-challenges = composition-bs,
          staged-composition-final = composition-final,
          staged-query-chunks = query-chunks|)
    unfolding checked-staged-transcript-program-def Let-def
    by (auto elim!: set-dist-bindE split: prod.splits)
  have assert-out-Suc:
    Some ((), s9) ∈
      set-dist
        (execute
          (assert
            (ceil-log (Suc (to-nat dg)) ≤
              ceil-log (Suc maxDegree))) s8)
    unfolding Suc-eq-plus1
    by (rule assert-out)
  have composition-out-Suc:
    Some ((composition-roots, composition-bs), s10) ∈
      set-dist
        (execute
          (staged-composition-fri-program A dg 0
            (ceil-log (Suc (to-nat dg)))) [] s9)
    unfolding Suc-eq-plus1

```

```

by (rule composition-out)
have query-bound:  $0 + \text{rounds} \leq \text{length } (\text{query-opening-budgets } \text{budgets})$ 
using wf unfolding staged-budget-wellformed-def by simp
from checked-staged-query-program-prefix-receive-state-alignment
  [OF controlled query-bound i-bound query-out]
obtain prefix-chunks prefix-state raw raw-state chunk chunk-state
  record-state suffix-chunks where
  prefix-query-out:
    Some (prefix-chunks, prefix-state)  $\in$ 
      set-dist
        (execute
          (checked-staged-query-program A trace-roots composition-roots
            0 i) query-start)
and raw-out:
    Some (raw, raw-state)  $\in$ 
      set-dist (execute receive-query-index-challenge prefix-state)
and chunk-out:
    Some (chunk, chunk-state)  $\in$ 
      set-dist
        (execute (query-opening-stage A (0 + i) raw) raw-state)
and chunk-shape:
    verifier-query-round-chunk (index (to-nat raw))
      trace-roots composition-roots chunk
and record-out:
    Some ((), record-state)  $\in$ 
      set-dist (execute (record-staged-messages chunk) chunk-state)
and suffix-out:
    Some (suffix-chunks, attacker-state)  $\in$ 
      set-dist
        (execute
          (checked-staged-query-program A trace-roots composition-roots
            (Suc (0 + i)) (rounds - Suc i)) record-state)
and chunks-eq: query-chunks = prefix-chunks @ chunk # suffix-chunks
and counter-prefix:
    PQueryCounter prefix-state = PQueryCounter query-start + i
and state-prefix:
    PState prefix-state =
      state-after-query-chunks (PState query-start) query-chunks i
by (rule checked-staged-query-program-prefix-receive-state-alignment
  [OF controlled query-bound i-bound query-out])
let ?prefix =
  ( $\sqcap$ sqp-trace-root = fr,
    sqp-trace-fri-roots = trace-roots,
    sqp-trace-fri-challenges = trace-bs,
    sqp-trace-final = trace-final,
    sqp-alphas = as,
    sqp-degree = dg,
    sqp-composition-fri-roots = composition-roots,
    sqp-composition-fri-challenges = composition-bs,

```

$sqp\text{-}composition\text{-}final = composition\text{-}final,$
 $sqp\text{-}query\text{-}chunks = prefix\text{-}chunks)$
have $\alpha\text{-}prefix\text{-}out$:
 $Some ((fr, trace\text{-}roots, trace\text{-}bs, trace\text{-}final), s5) \in$
 $set\text{-}dist$
 $(execute (staged\text{-}\alpha\text{-}prefix\text{-}program A) adversary\text{-}initial\text{-}state)$
proof –
have $after\text{-}record\text{-}final$:
 $Some ((fr, trace\text{-}roots, trace\text{-}bs, trace\text{-}final), s5) \in$
 $set\text{-}dist$
 $(execute$
 $(record\text{-}staged\text{-}message trace\text{-}final \gg=$
 $(\lambda\text{-}. return (fr, trace\text{-}roots, trace\text{-}bs, trace\text{-}final))))$
 $s4)$
by ($rule\text{-}tac x=()$ **and** $t=s5$ **in** $set\text{-}dist\text{-}bindI$)
 $(rule\ record\text{-}final, simp)$
have $after\text{-}final$:
 $Some ((fr, trace\text{-}roots, trace\text{-}bs, trace\text{-}final), s5) \in$
 $set\text{-}dist$
 $(execute$
 $(trace\text{-}final\text{-}stage A trace\text{-}bs \gg=$
 $(\lambda trace\text{-}final.$
 $record\text{-}staged\text{-}message trace\text{-}final \gg=$
 $(\lambda\text{-}. return (fr, trace\text{-}roots, trace\text{-}bs, trace\text{-}final))))))$
 $s3)$
by ($rule\text{-}tac x=trace\text{-}final$ **and** $t=s4$ **in** $set\text{-}dist\text{-}bindI$)
 $(rule\ final\text{-}out, rule\ after\text{-}record\text{-}final)$
have $after\text{-}trace$:
 $Some ((fr, trace\text{-}roots, trace\text{-}bs, trace\text{-}final), s5) \in$
 $set\text{-}dist$
 $(execute$
 $(staged\text{-}trace\text{-}fri\text{-}program A 0 (ceil\text{-}log\ clength) [] \gg=$
 $(\lambda (trace\text{-}roots, trace\text{-}bs).$
 $trace\text{-}final\text{-}stage A trace\text{-}bs \gg=$
 $(\lambda trace\text{-}final.$
 $record\text{-}staged\text{-}message trace\text{-}final \gg=$
 $(\lambda\text{-}. return$
 $(fr, trace\text{-}roots, trace\text{-}bs, trace\text{-}final))))))$
 $s2)$
apply ($rule\text{-}tac x=(trace\text{-}roots, trace\text{-}bs)$ **and** $t=s3$
in $set\text{-}dist\text{-}bindI$)
apply ($rule\ trace\text{-}out$)
apply ($simp\ add: after\text{-}final$)
done
have $after\text{-}record\text{-}root$:
 $Some ((fr, trace\text{-}roots, trace\text{-}bs, trace\text{-}final), s5) \in$
 $set\text{-}dist$
 $(execute$
 $(record\text{-}staged\text{-}message fr \gg=$

```

    (λ-. staged-trace-fri-program A 0 (ceil-log clength) [] >=
      (λ(trace-roots, trace-bs).
        trace-final-stage A trace-bs >=
        (λtrace-final.
          record-staged-message trace-final >=
          (λ-. return
            (fr, trace-roots, trace-bs, trace-final))))))
  s1)
  by (rule-tac x=() and t=s2 in set-dist-bindI)
    (rule record-root, rule after-trace)
  show ?thesis
    unfolding staged-alpha-prefix-program-def
    by (rule-tac x=fr and t=s1 in set-dist-bindI)
      (rule root-out, rule after-record-root)
  qed
  have prefix-program-out:
    Some (?prefix, prefix-state) ∈
      set-dist
        (execute
          (checked-staged-query-challenge-prefix-program A i)
          adversary-initial-state)
  proof -
    have after-prefix-query:
      Some (?prefix, prefix-state) ∈
        set-dist
          (execute
            (checked-staged-query-program A trace-roots composition-roots
              0 i >=
              (λquery-chunks.
                return
                  (sqp-trace-root = fr,
                   sqp-trace-fri-roots = trace-roots,
                   sqp-trace-fri-challenges = trace-bs,
                   sqp-trace-final = trace-final,
                   sqp-alphas = as,
                   sqp-degree = dg,
                   sqp-composition-fri-roots = composition-roots,
                   sqp-composition-fri-challenges = composition-bs,
                   sqp-composition-final = composition-final,
                   sqp-query-chunks = query-chunks))))
            query-start)
    by (rule-tac x=prefix-chunks and t=prefix-state in set-dist-bindI)
      (rule prefix-query-out, simp)
  have after-record-composition-final:
    Some (?prefix, prefix-state) ∈
      set-dist
        (execute
          (record-staged-message composition-final >=
            (λ-.

```

```

checked-staged-query-program A trace-roots composition-roots
  0 i  $\gg$ 
( $\lambda$ query-chunks.
  return
    ( $\downarrow$ sqp-trace-root = fr,
      sqp-trace-fri-roots = trace-roots,
      sqp-trace-fri-challenges = trace-bs,
      sqp-trace-final = trace-final,
      sqp-alphas = as,
      sqp-degree = dg,
      sqp-composition-fri-roots = composition-roots,
      sqp-composition-fri-challenges = composition-bs,
      sqp-composition-final = composition-final,
      sqp-query-chunks = query-chunks $\downarrow$ ))))
s11)
by (rule-tac x=() and t=query-start in set-dist-bindI)
  (rule record-composition-final, rule after-prefix-query)
have after-composition-final:
  Some (?prefix, prefix-state)  $\in$ 
    set-dist
    (execute
      (composition-final-stage A dg composition-bs  $\gg$ 
        ( $\lambda$ composition-final.
          record-staged-message composition-final  $\gg$ 
            ( $\lambda$ -.
              checked-staged-query-program A trace-roots
                composition-roots 0 i  $\gg$ 
                  ( $\lambda$ query-chunks.
                    return
                      ( $\downarrow$ sqp-trace-root = fr,
                        sqp-trace-fri-roots = trace-roots,
                        sqp-trace-fri-challenges = trace-bs,
                        sqp-trace-final = trace-final,
                        sqp-alphas = as,
                        sqp-degree = dg,
                        sqp-composition-fri-roots = composition-roots,
                        sqp-composition-fri-challenges = composition-bs,
                        sqp-composition-final = composition-final,
                        sqp-query-chunks = query-chunks $\downarrow$ ))))))
s10)
by (rule-tac x=composition-final and t=s11 in set-dist-bindI)
  (rule composition-final-out, rule after-record-composition-final)
have after-composition:
  Some (?prefix, prefix-state)  $\in$ 
    set-dist
    (execute
      (staged-composition-fri-program A dg 0
        (ceil-log (Suc (to-nat dg)))  $\square$   $\gg$ 
          ( $\lambda$ (composition-roots, composition-bs).

```



```

composition-final-stage A dg composition-bs >>=
(λcomposition-final.
  record-staged-message composition-final >>=
  (λ-.
    checked-staged-query-program A trace-roots
      composition-roots 0 i >>=
    (λquery-chunks.
      return
        (|spp-trace-root = fr,
          spp-trace-fri-roots = trace-roots,
          spp-trace-fri-challenges = trace-bs,
          spp-trace-final = trace-final,
          spp-alphas = as,
          spp-degree = dg,
          spp-composition-fri-roots = composition-roots,
          spp-composition-fri-challenges = composition-bs,
          spp-composition-final = composition-final,
          spp-query-chunks = query-chunks|))))))
s9)
apply (rule-tac x=(composition-roots, composition-bs) and t=s10
  in set-dist-bindI)
apply (rule composition-out-Suc)
apply (simp add: after-composition-final)
done
have after-assert:
  Some (?prefix, prefix-state) ∈
    set-dist
    (execute
      (assert (ceil-log (Suc (to-nat dg)) ≤
        ceil-log (Suc maxDegree)) >>=
      (λ-.
        staged-composition-fri-program A dg 0
          (ceil-log (Suc (to-nat dg))) [] >>=
        (λ(composition-roots, composition-bs).
          composition-final-stage A dg composition-bs >>=
          (λcomposition-final.
            record-staged-message composition-final >>=
            (λ-.
              checked-staged-query-program A trace-roots
                composition-roots 0 i >>=
              (λquery-chunks.
                return
                  (|spp-trace-root = fr,
                    spp-trace-fri-roots = trace-roots,
                    spp-trace-fri-challenges = trace-bs,
                    spp-trace-final = trace-final,
                    spp-alphas = as,
                    spp-degree = dg,
                    spp-composition-fri-roots = composition-roots,

```

```

      sqp-composition-fri-challenges = composition-bs,
      sqp-composition-final = composition-final,
      sqp-query-chunks = query-chunks)))))))))
s8)
apply (rule-tac x=() and t=s9 in set-dist-bindI)
  apply (rule assert-out-Suc)
  apply (simp add: after-composition)
  done
have after-record-degree:
  Some (?prefix, prefix-state) ∈
    set-dist
    (execute
      (record-staged-message dg ≫=
        (λ-.
          assert (ceil-log (Suc (to-nat dg)) ≤
            ceil-log (Suc maxDegree)) ≫=
          (λ-.
            staged-composition-fri-program A dg 0
            (ceil-log (Suc (to-nat dg))) [] ≫=
            (λ(composition-roots, composition-bs).
              composition-final-stage A dg composition-bs ≫=
              (λcomposition-final.
                record-staged-message composition-final ≫=
              (λ-.
                checked-staged-query-program A trace-roots
                composition-roots 0 i ≫=
              (λquery-chunks.
                return
                  (|sqp-trace-root = fr,
                    sqp-trace-fri-roots = trace-roots,
                    sqp-trace-fri-challenges = trace-bs,
                    sqp-trace-final = trace-final,
                    sqp-alphas = as,
                    sqp-degree = dg,
                    sqp-composition-fri-roots = composition-roots,
                    sqp-composition-fri-challenges = composition-bs,
                    sqp-composition-final = composition-final,
                    sqp-query-chunks = query-chunks)))))))))))))
s7)
apply (rule-tac x=() and t=s8 in set-dist-bindI)
  apply (rule record-degree)
  apply (simp add: after-assert)
  done
have after-degree:
  Some (?prefix, prefix-state) ∈
    set-dist
    (execute
      (degree-stage A as ≫=
        (λdg.

```

```

record-staged-message dg >>=
(λ-.
  assert (ceil-log (Suc (to-nat dg)) ≤
    ceil-log (Suc maxDegree)) >>=
(λ-.
  staged-composition-fri-program A dg 0
    (ceil-log (Suc (to-nat dg))) [] >>=
(λ(composition-roots, composition-bs).
  composition-final-stage A dg composition-bs >>=
(λcomposition-final.
  record-staged-message composition-final >>=
(λ-.
  checked-staged-query-program A trace-roots
    composition-roots 0 i >>=
(λquery-chunks.
  return
    (|sqp-trace-root = fr,
      sqp-trace-fri-roots = trace-roots,
      sqp-trace-fri-challenges = trace-bs,
      sqp-trace-final = trace-final,
      sqp-alphas = as,
      sqp-degree = dg,
      sqp-composition-fri-roots = composition-roots,
      sqp-composition-fri-challenges = composition-bs,
      sqp-composition-final = composition-final,
      sqp-query-chunks = query-chunks|)))))))))
s6)
apply (rule-tac x=dg and t=s7 in set-dist-bindI)
apply (rule degree-out)
apply (simp add: after-record-degree)
done
have after-alpha:
  Some (?prefix, prefix-state) ∈
    set-dist
    (execute
      (staged-alpha-program (length spec) >>=
        (λas.
          degree-stage A as >>=
            (λdg.
              record-staged-message dg >>=
                (λ-.
                  assert (ceil-log (Suc (to-nat dg)) ≤
                    ceil-log (Suc maxDegree)) >>=
                    (λ-.
                      staged-composition-fri-program A dg 0
                        (ceil-log (Suc (to-nat dg))) [] >>=
                        (λ(composition-roots, composition-bs).
                          composition-final-stage A dg composition-bs >>=
                          (λcomposition-final.

```

```

record-staged-message composition-final  $\gg$ 
( $\lambda$ -.
  checked-staged-query-program A trace-roots
  composition-roots 0 i  $\gg$ 
( $\lambda$ query-chunks.
  return
    ( $\lambda$ sqp-trace-root = fr,
      sqp-trace-fri-roots = trace-roots,
      sqp-trace-fri-challenges = trace-bs,
      sqp-trace-final = trace-final,
      sqp-alphas = as,
      sqp-degree = dg,
      sqp-composition-fri-roots = composition-roots,
      sqp-composition-fri-challenges = composition-bs,
      sqp-composition-final = composition-final,
      sqp-query-chunks = query-chunks)))))))))
s5)
apply (rule-tac x=as and t=s6 in set-dist-bindI)
apply (rule alpha-out)
apply (simp add: after-degree)
done
show ?thesis
unfolding checked-staged-query-challenge-prefix-program-def Let-def
apply (rule-tac x=(fr, trace-roots, trace-bs, trace-final) and t=s5
in set-dist-bindI)
apply (rule alpha-prefix-out)
apply (simp add: after-alpha)
done
qed
have prefix-with-state-out:
  Some ((?prefix, prefix-state), prefix-state)  $\in$ 
    set-dist
      (execute (checked-staged-query-prefix-with-state A i)
        adversary-initial-state)
unfolding checked-staged-query-prefix-with-state-def
by (rule set-dist-bindI[where x=?prefix and t=prefix-state,
  OF prefix-program-out get-bind-return-state-support])
have prefix-receive-out:
  Some (((?prefix, prefix-state), raw), raw-state)  $\in$ 
    set-dist
      (execute (checked-staged-query-prefix-receive-with-state A i)
        adversary-initial-state)
unfolding checked-staged-query-prefix-receive-with-state-def
by (intro set-dist-bindI[OF prefix-with-state-out]
  set-dist-bindI[OF raw-out])
  simp
have root-controlled:
  controlled-ro-program (trace-root-budget budgets) (trace-root-stage A)
  using controlled unfolding staged-adversary-controlled-def by simp

```

```

have trace-final-controlled:
  controlled-ro-program (trace-final-budget budgets)
    (trace-final-stage A trace-bs)
  using controlled unfolding staged-adversary-controlled-def by blast
have degree-controlled:
  controlled-ro-program (degree-budget budgets) (degree-stage A as)
  using controlled unfolding staged-adversary-controlled-def by blast
have composition-final-controlled:
  controlled-ro-program (composition-final-budget budgets)
    (composition-final-stage A dg composition-bs)
  using controlled unfolding staged-adversary-controlled-def by blast
have root-state: PState s1 = 0
  using controlled-stage-outcome-fields[OF root-controlled root-out]
  by simp
have s2-eq:
  s2 = s1(
    PState := concat (PState s1) fr,
    PTranscript := PTranscript s1 @ [fr])
  by (rule record-staged-message-outcome[OF record-root])
have trace-bound:
   $0 + \text{ceil-log } \text{clength} \leq \text{length } (\text{trace-fri-budgets } \text{budgets})$ 
  using wf unfolding staged-budget-wellformed-def by simp
have trace-alignment:
  PState s3 = foldl concat (PState s2) trace-roots
  using staged-trace-fri-program-alignment
    [OF controlled trace-bound trace-out]
  by simp
have state-s3:
  PState s3 = foldl concat (concat 0 fr) trace-roots
  using root-state trace-alignment unfolding s2-eq by simp
have state-s4: PState s4 = PState s3
  using controlled-stage-outcome-fields
    [OF trace-final-controlled final-out]
  by simp
have s5-eq:
  s5 = s4(
    PState := concat (PState s4) trace-final,
    PTranscript := PTranscript s4 @ [trace-final])
  by (rule record-staged-message-outcome[OF record-final])
have alpha-alignment:
  PState s6 = foldl concat (PState s5) as
  using staged-alpha-program-alignment[OF alpha-out] by simp
have state-s6:
  PState s6 =
    foldl concat
      (concat (foldl concat (concat 0 fr) trace-roots) trace-final)
      as
  using state-s3 state-s4 alpha-alignment unfolding s5-eq by simp
have state-s7: PState s7 = PState s6

```

```

using controlled-stage-outcome-fields[OF degree-controlled degree-out]
by simp
have s8-eq:
  s8 = s7(
    PState := concat (PState s7) dg,
    PTranscript := PTranscript s7 @ [dg])
  by (rule record-staged-message-outcome[OF record-degree])
have s9-eq: s9 = s8
  using assert-out unfolding assert-def
  by (cases
    ceil-log (to-nat dg + 1) ≤ ceil-log (maxDegree + 1))
    (auto simp: throw-no-outcome)
have round-bound:
  ceil-log (to-nat dg + 1) ≤ ceil-log (maxDegree + 1)
  using assert-out unfolding assert-def
  by (cases
    ceil-log (to-nat dg + 1) ≤ ceil-log (maxDegree + 1))
    (auto simp: throw-no-outcome)
have composition-bound:
  0 + ceil-log (to-nat dg + 1) ≤
    length (composition-fri-budgets budgets)
  using wf round-bound unfolding staged-budget-wellformed-def by simp
have composition-alignment:
  PState s10 = foldl concat (PState s9) composition-roots
  using staged-composition-fri-program-alignment
    [OF controlled composition-bound composition-out]
  by simp
have state-s10:
  PState s10 =
    foldl concat
      (concat
        (foldl concat
          (concat (foldl concat (concat 0 fr) trace-roots) trace-final)
          as)
        dg)
        composition-roots
  using state-s6 state-s7 composition-alignment
  unfolding s8-eq s9-eq by simp
have state-s11: PState s11 = PState s10
  using controlled-stage-outcome-fields
    [OF composition-final-controlled composition-final-out]
  by simp
have query-start-eq:
  query-start = s11(
    PState := concat (PState s11) composition-final,
    PTranscript := PTranscript s11 @ [composition-final])
  by (rule record-staged-message-outcome[OF record-composition-final])
have state-query-start:
  PState query-start =

```

```

concat
  (foldl concat
    (concat
      (foldl concat
        (concat (foldl concat (concat 0 fr) trace-roots) trace-final)
        as)
      dg)
    composition-roots)
  composition-final
using state-s10 state-s11 unfolding query-start-eq by simp
have start-state:
  PState query-start = staged-query-start-hash data
unfolding data-eq staged-query-start-hash-def
  staged-composition-fri-start-hash-def staged-trace-fri-start-hash-def
using state-query-start by simp
have query-counter-start: PQueryCounter query-start = 0
proof –
  have root-fields:
    PQueryCounter s1 = PQueryCounter adversary-initial-state
    using controlled-stage-outcome-fields[OF root-controlled root-out]
    by simp
  have trace-counters:
    PQueryCounter s3 = PQueryCounter s2
    using staged-trace-fri-program-alignment
    [OF controlled trace-bound trace-out]
    by simp
  have final-counter: PQueryCounter s4 = PQueryCounter s3
    using controlled-stage-outcome-fields
    [OF trace-final-controlled final-out]
    by simp
  have alpha-counter: PQueryCounter s6 = PQueryCounter s5
    using staged-alpha-program-alignment[OF alpha-out] by simp
  have degree-counter: PQueryCounter s7 = PQueryCounter s6
    using controlled-stage-outcome-fields[OF degree-controlled degree-out]
    by simp
  have comp-counter: PQueryCounter s10 = PQueryCounter s9
    using staged-composition-fri-program-alignment
    [OF controlled composition-bound composition-out]
    by simp
  have comp-final-counter: PQueryCounter s11 = PQueryCounter s10
    using controlled-stage-outcome-fields
    [OF composition-final-controlled composition-final-out]
    by simp
  show ?thesis
    using root-fields trace-counters final-counter alpha-counter
    degree-counter comp-counter comp-final-counter
    unfolding s2-eq s5-eq s8-eq s9-eq query-start-eq by simp
qed
have prefix-counter: PQueryCounter prefix-state = i

```

```

using counter-prefix query-counter-start by simp
have prefix-state-eq:
  PState prefix-state =
    state-after-query-chunks
      (staged-query-start-hash data) (staged-query-chunks data) i
  using state-prefix start-state unfolding data-eq by simp
have prefix-query-bound:  $0 + i \leq \text{length } (\text{query-opening-budgets budgets})$ 
  using query-bound i-bound by simp
have prefix-len:  $\text{length prefix-chunks} = i$ 
proof –
  from checked-staged-query-program-outcome-with-raws
    [OF controlled prefix-query-bound prefix-query-out]
  show ?thesis
    by blast
qed
have prefix-chunks-eq:
  prefix-chunks = take i query-chunks
  using chunks-eq prefix-len by simp
have receive-lookup:
  fmlookup (HashMap raw-state)
    (QueryIndexChallenge i
      (state-after-query-chunks
        (staged-query-start-hash data) (staged-query-chunks data) i)) =
    Some raw
  using receive-query-index-challenge-outcome[OF raw-out]
    prefix-counter prefix-state-eq
  by simp
have stage-controlled:
  controlled-ro-program (query-opening-budgets budgets ! i)
    (query-opening-stage A i raw)
  using controlled wf i-bound
  unfolding staged-adversary-controlled-def staged-budget-wellformed-def
  by simp
have raw-chunk-ext:  $\text{raw-state} \leq \text{chunk-state}$ 
proof –
  have chunk-out-i:
    Some (chunk, chunk-state) ∈
      set-dist (execute (query-opening-stage A i raw) raw-state)
  using chunk-out by simp
  show ?thesis
    using controlled-ro-program-extension[OF stage-controlled] chunk-out-i
    unfolding hash-extension-preserving-def by blast
qed
have prefix-raw-ext:  $\text{prefix-state} \leq \text{raw-state}$ 
  by (rule receive-query-index-challenge-extends[OF raw-out])
have record-state-eq:
  record-state = chunk-state[
    PState := foldl concat (PState chunk-state) chunk,
    PTranscript := PTranscript chunk-state @ chunk]

```



```

  by (rule record-staged-messages-outcome[OF record-out])
have chunk-record-ext: chunk-state ≤ record-state
  unfolding record-state-eq less-eq-hash-ext-def less-eq-fmap-def by simp
have suffix-bound:
  Suc (0 + i) + (rounds - Suc i) ≤
    length (query-opening-budgets budgets)
  using query-bound i-bound by simp
have suffix-target:
  hash-target-program {}
    (sum-list
      (take (rounds - Suc i)
        (drop (Suc (0 + i)) (query-opening-budgets budgets))) +
      (rounds - Suc i))
    (checked-staged-query-program A trace-roots composition-roots
      (Suc (0 + i)) (rounds - Suc i))
  by (rule hash-target-program-checked-staged-query-program
    [OF controlled suffix-bound])
have suffix-ext: record-state ≤ attacker-state
  using hash-target-program-extension[OF suffix-target] suffix-out
  unfolding hash-extension-preserving-def by blast
have raw-ext: raw-state ≤ attacker-state
  by (rule hash-ext-trans[OF raw-chunk-ext])
    (rule hash-ext-trans[OF chunk-record-ext suffix-ext])
have prefix-ext: prefix-state ≤ attacker-state
  by (rule hash-ext-trans[OF prefix-raw-ext raw-ext])
show ?thesis
  by (rule that[OF prefix-receive-out])
    (use data-eq prefix-chunks-eq prefix-counter prefix-state-eq prefix-ext
      raw-ext receive-lookup in simp-all)
qed

lemma checked-staged-transcript-program-query-prefix-receive-final-lookup-support:
  assumes wf: staged-budget-wellformed budgets
  and controlled: staged-adversary-controlled budgets A
  and i-bound: i < rounds
  and outcome:
    Some (data, attacker-state) ∈
      set-dist
        (execute (checked-staged-transcript-program A)
          adversary-initial-state)
  and verifier:
    Some (result, final-state) ∈
      set-dist
        (execute verify-monad
          (verifier-state-from-adversary attacker-state
            (staged-proof-transcript data)))
  and final-lookup:
    fmllookup (HashMap final-state)
      (QueryIndexChallenge i

```

```

      (state-after-query-chunks
       (staged-query-start-hash data) (staged-query-chunks data) i)) =
    Some raw
obtains prefix prefix-state raw-state where
  Some (((prefix, prefix-state), raw), raw-state) ∈
    set-dist
    (execute (checked-staged-query-prefix-receive-with-state A i)
             adversary-initial-state)
proof –
  let ?s =
    verifier-state-from-adversary attacker-state
    (staged-proof-transcript data)
  show ?thesis
proof (rule checked-staged-transcript-program-query-prefix-receive-support
        [OF wf controlled i-bound outcome])
  fix prefix prefix-state raw' raw-state
  assume prefix-receive:
    Some (((prefix, prefix-state), raw'), raw-state) ∈
      set-dist
      (execute (checked-staged-query-prefix-receive-with-state A i)
               adversary-initial-state)
  and raw-ext: raw-state ≤ attacker-state
  and raw-lookup:
    fmlookup (HashMap raw-state)
    (QueryIndexChallenge i
     (state-after-query-chunks
      (staged-query-start-hash data) (staged-query-chunks data) i)) =
    Some raw'
  have raw-ext-s: raw-state ≤ ?s
  by (rule hash-extends-verifier-state-from-adversary-right[OF raw-ext])
  have s-ext-final: ?s ≤ final-state
  by (rule verify-monad-hash-extends[OF verifier])
  have raw-ext-final: raw-state ≤ final-state
  by (rule hash-ext-trans[OF raw-ext-s s-ext-final])
  have final-lookup-raw':
    fmlookup (HashMap final-state)
    (QueryIndexChallenge i
     (state-after-query-chunks
      (staged-query-start-hash data) (staged-query-chunks data) i)) =
    Some raw'
  by (rule hash-extension-lookup[OF raw-lookup raw-ext-final])
  have raw-eq: raw' = raw
  using final-lookup final-lookup-raw' by simp
  have prefix-receive-raw:
    Some (((prefix, prefix-state), raw), raw-state) ∈
      set-dist
      (execute (checked-staged-query-prefix-receive-with-state A i)
               adversary-initial-state)
  using prefix-receive raw-eq by simp

```

```

show ?thesis
  by (rule that[OF prefix-receive-raw])
qed
qed

lemma checked-staged-transcript-program-query-prefix-receive-final-lookup-support-with-alphas:
assumes wf: staged-budget-wellformed budgets
and controlled: staged-adversary-controlled budgets A
and i-bound: i < rounds
and outcome:
  Some (data, attacker-state) ∈
    set-dist
      (execute (checked-staged-transcript-program A)
        adversary-initial-state)
and verifier:
  Some (result, final-state) ∈
    set-dist
      (execute verify-monad
        (verifier-state-from-adversary attacker-state
          (staged-proof-transcript data))))
and final-lookup:
  fmlookup (HashMap final-state)
    (QueryIndexChallenge i
      (state-after-query-chunks
        (staged-query-start-hash data) (staged-query-chunks data) i)) =
    Some raw
obtains prefix prefix-state raw-state where
  Some (((prefix, prefix-state), raw), raw-state) ∈
    set-dist
      (execute (checked-staged-query-prefix-receive-with-state A i)
        adversary-initial-state)
  sqp-alphas prefix = staged-alphas data
proof –
let ?s =
  verifier-state-from-adversary attacker-state
    (staged-proof-transcript data)
show ?thesis
proof (rule checked-staged-transcript-program-query-prefix-receive-support
  [OF wf controlled i-bound outcome])
fix prefix prefix-state raw' raw-state
assume prefix-receive:
  Some (((prefix, prefix-state), raw'), raw-state) ∈
    set-dist
      (execute (checked-staged-query-prefix-receive-with-state A i)
        adversary-initial-state)
and alphas-eq: sqp-alphas prefix = staged-alphas data
and raw-ext: raw-state ≤ attacker-state
and raw-lookup:
  fmlookup (HashMap raw-state)

```

```

      (QueryIndexChallenge i
       (state-after-query-chunks
        (staged-query-start-hash data) (staged-query-chunks data) i)) =
      Some raw'
have raw-ext-s: raw-state ≤ ?s
  by (rule hash-extends-verifier-state-from-adversary-right[OF raw-ext])
have s-ext-final: ?s ≤ final-state
  by (rule verify-monad-hash-extends[OF verifier])
have raw-ext-final: raw-state ≤ final-state
  by (rule hash-ext-trans[OF raw-ext-s s-ext-final])
have final-lookup-raw':
  fmllookup (HashMap final-state)
  (QueryIndexChallenge i
   (state-after-query-chunks
    (staged-query-start-hash data) (staged-query-chunks data) i)) =
  Some raw'
  by (rule hash-extension-lookup[OF raw-lookup raw-ext-final])
have raw-eq: raw' = raw
  using final-lookup final-lookup-raw' by simp
have prefix-receive-raw:
  Some (((prefix, prefix-state), raw), raw-state) ∈
  set-dist
  (execute (checked-staged-query-prefix-receive-with-state A i)
   adversary-initial-state)
  using prefix-receive raw-eq by simp
show ?thesis
  by (rule that[OF prefix-receive-raw alphas-eq])
qed
qed

end

end

```

```

theory Soundness-Staged-Query
  imports Soundness-Staged-Query-Prefix
begin

```

Dynamic staged query target events and probability bounds.

```

context soundness
begin

```

```

definition checked-staged-query-prefix-dynamic-index-hit
  :: ('f staged-query-prefix-data ⇒ 'f protocol-channel ⇒
      nat set) ⇒
      (((('f staged-query-prefix-data × 'f protocol-channel) × 'f) ×
        'f protocol-channel) option ⇒ bool
where

```

checked-staged-query-prefix-dynamic-index-hit B $out \longleftrightarrow$
 (case out of
 None $\Rightarrow$ False
 | Some (((*prefix*, *prefix-state*), *raw*), -) $\Rightarrow$
 index (to-nat *raw*) $\in B$ *prefix* *prefix-state*)

definition *staged-partial-query-target*
 :: 'f staged-proof-data $\Rightarrow$ 'f protocol-channel $\Rightarrow$ nat set
where

staged-partial-query-target *data* *attacker-state* =
 query-header-supported-partial-union-good-sets
 (*verifier-state-from-adversary* *attacker-state*
 (*staged-proof-transcript* *data*))
 query-sampling-success-space
 (*staged-trace-root* *data*)
 (*staged-trace-fri-roots* *data*)
 (*staged-trace-final* *data*)
 (*staged-alphas* *data*)
 (*staged-degree* *data*)
 (*staged-composition-fri-roots* *data*)
 (*staged-composition-final* *data*)

definition *staged-query-prefix-matches-data*
 :: nat $\Rightarrow$ 'f staged-query-prefix-data $\Rightarrow$ 'f staged-proof-data $\Rightarrow$
 bool

where
staged-query-prefix-matches-data i *prefix* *data* $\longleftrightarrow$
 sqp-trace-root *prefix* = *staged-trace-root* *data* $\wedge$
 sqp-trace-fri-roots *prefix* = *staged-trace-fri-roots* *data* $\wedge$
 sqp-trace-fri-challenges *prefix* =
 staged-trace-fri-challenges *data* $\wedge$
 sqp-trace-final *prefix* = *staged-trace-final* *data* $\wedge$
 sqp-alphas *prefix* = *staged-alphas* *data* $\wedge$
 sqp-degree *prefix* = *staged-degree* *data* $\wedge$
 sqp-composition-fri-roots *prefix* =
 staged-composition-fri-roots *data* $\wedge$
 sqp-composition-fri-challenges *prefix* =
 staged-composition-fri-challenges *data* $\wedge$
 sqp-composition-final *prefix* =
 staged-composition-final *data* $\wedge$
 sqp-query-chunks *prefix* = take i (*staged-query-chunks* *data*)

lemma *staged-query-start-hash-of-prefix-completion*:
assumes *match*: *staged-query-prefix-matches-data* i *prefix* *data*
shows *staged-query-start-hash* *data* =
 staged-query-prefix-start-hash *prefix*
using *match*
unfolding *staged-query-prefix-matches-data-def*
 staged-query-start-hash-def *staged-composition-fri-start-hash-def*

staged-trace-fri-start-hash-def staged-query-prefix-start-hash-def
by *simp*

definition *staged-query-prefix-partial-query-target*

$:: 'f \text{ staged-query-prefix-data} \Rightarrow 'f \text{ protocol-channel} \Rightarrow$
 nat set

where

staged-query-prefix-partial-query-target *i prefix prefix-state* =
 $\{idx. \exists \text{ data attacker-state.}$
 $\text{staged-query-prefix-matches-data } i \text{ prefix data} \wedge$
 $\text{prefix-state} \leq \text{attacker-state} \wedge$
 $idx \in \text{staged-partial-query-target data attacker-state}\}$

definition *staged-query-prefix-committed-query-target*

$:: 'f \text{ staged-query-prefix-data} \Rightarrow 'f \text{ protocol-channel} \Rightarrow \text{nat set}$

where

staged-query-prefix-committed-query-target *prefix prefix-state* =
 $\text{query-header-committed-union-good-sets prefix-state}$
 $\text{query-sampling-success-space}$
 $(\text{sqp-trace-root prefix})$
 $(\text{sqp-alphas prefix})$
 $(\text{sqp-composition-fri-roots prefix})$

definition *staged-query-prefix-length-committed-table-candidates*

$:: 'f \text{ staged-query-prefix-data} \Rightarrow 'f \text{ protocol-channel} \Rightarrow$
 $('f \text{ list} \times 'f \text{ list}) \text{ set}$

where

staged-query-prefix-length-committed-table-candidates *prefix prefix-state* =
 $\{(\text{trace-table, composition-table}).$
 $\text{merkle-root-binds-table } (\text{sqp-trace-root prefix}) \text{ trace-table}$
 $\text{prefix-state} \wedge$
 $\text{sqp-composition-fri-roots prefix} \neq [] \wedge$
 $\text{merkle-root-binds-table } (\text{hd } (\text{sqp-composition-fri-roots prefix}))$
 $\text{composition-table prefix-state} \wedge$
 $\text{length trace-table} = \text{clength} * \text{scale} \wedge$
 $\text{length composition-table} = \text{clength} * \text{scale}\}$

definition *staged-query-prefix-length-committed-query-target*

$:: 'f \text{ staged-query-prefix-data} \Rightarrow 'f \text{ protocol-channel} \Rightarrow \text{nat set}$

where

staged-query-prefix-length-committed-query-target *prefix prefix-state* =
 $\{idx \in \text{query-sample-space.}$
 $\exists \text{ trace-table composition-table.}$
 $(\text{trace-table, composition-table}) \in$
 $\text{staged-query-prefix-length-committed-table-candidates prefix}$
 $\text{prefix-state} \wedge$
 $idx \in \text{query-sampling-success-space trace-table}$
 $\text{composition-table } (\text{sqp-alphas prefix})\}$

definition *staged-query-prefix-clean-length-committed-query-target*

$:: 'f \text{ staged-query-prefix-data} \Rightarrow 'f \text{ protocol-channel} \Rightarrow \text{nat set}$

where

staged-query-prefix-clean-length-committed-query-target *prefix*
prefix-state =
 (if *hash-map-output-collision* *prefix-state* then {}
 else *staged-query-prefix-length-committed-query-target* *prefix*
prefix-state)

definition *checked-staged-query-prefix-hash-map-output-collision*

$:: ((('f \text{ staged-query-prefix-data} \times 'f \text{ protocol-channel}) \times 'f) \times 'f \text{ protocol-channel}) \text{ option} \Rightarrow \text{bool}$

where

checked-staged-query-prefix-hash-map-output-collision *out* $\longleftrightarrow$
 (case *out* of
 None $\Rightarrow$ *False*
 | *Some* (((-, *prefix-state*), -), -) $\Rightarrow$
 hash-map-output-collision *prefix-state*)

definition *staged-query-prefix-committed-roots-clean*

$:: 'f \text{ staged-query-prefix-data} \Rightarrow 'f \text{ protocol-channel} \Rightarrow \text{bool}$

where

staged-query-prefix-committed-roots-clean *prefix* *prefix-state* $\longleftrightarrow$
 $\neg \text{merkle-root-binding-collision } (\text{sqp-trace-root } \text{prefix})$
prefix-state $\wedge$
 $(\forall \text{rt. } \text{sqp-composition-fri-roots } \text{prefix} \neq [] \longrightarrow$
 $\text{rt} = \text{hd } (\text{sqp-composition-fri-roots } \text{prefix}) \longrightarrow$
 $\neg \text{merkle-root-binding-collision } \text{rt } \text{prefix-state})$

definition *staged-query-prefix-clean-committed-query-target*

$:: 'f \text{ staged-query-prefix-data} \Rightarrow 'f \text{ protocol-channel} \Rightarrow \text{nat set}$

where

staged-query-prefix-clean-committed-query-target *prefix* *prefix-state* =
 (if *staged-query-prefix-committed-roots-clean* *prefix* *prefix-state*
 then *staged-query-prefix-committed-query-target* *prefix* *prefix-state*
 else {})

definition *checked-staged-query-prefix-committed-root-collision*

$:: ((('f \text{ staged-query-prefix-data} \times 'f \text{ protocol-channel}) \times 'f) \times 'f \text{ protocol-channel}) \text{ option} \Rightarrow \text{bool}$

where

checked-staged-query-prefix-committed-root-collision *out* $\longleftrightarrow$
 (case *out* of
 None $\Rightarrow$ *False*
 | *Some* (((*prefix*, *prefix-state*), -), -) $\Rightarrow$
 $\neg \text{staged-query-prefix-committed-roots-clean } \text{prefix } \text{prefix-state}$)

definition *staged-security-with-data-state-query-committed-prefix-hit*

$:: 'f \text{ staged-adversary} \Rightarrow$

$((('f \text{ staged-proof-data} \times 'f \text{ protocol-channel}) \times \text{unit list}) \times$
 $'f \text{ protocol-channel}) \text{ option} \Rightarrow \text{bool}$

where

$\text{staged-security-with-data-state-query-committed-prefix-hit } A \text{ out} \longleftrightarrow$
 $(\text{case out of}$
 $\text{None} \Rightarrow \text{False}$
 $| \text{Some } (((\text{data}, \text{attacker-state}), -), -) \Rightarrow$
 $(\exists i \text{ prefix prefix-state raw raw-state.}$
 $i < \text{rounds} \wedge$
 $\text{Some } (\text{data}, \text{attacker-state}) \in$
 set-dist
 $(\text{execute } (\text{checked-staged-transcript-program } A)$
 $\text{adversary-initial-state}) \wedge$
 $\text{Some } (((\text{prefix}, \text{prefix-state}), \text{raw}), \text{raw-state}) \in$
 set-dist
 $(\text{execute } (\text{checked-staged-query-prefix-receive-with-state } A \ i)$
 $\text{adversary-initial-state}) \wedge$
 $\text{checked-staged-query-prefix-dynamic-index-hit}$
 $(\lambda \text{prefix prefix-state.}$
 $\text{staged-query-prefix-committed-query-target prefix prefix-state})$
 $(\text{Some } (((\text{prefix}, \text{prefix-state}), \text{raw}), \text{raw-state}))))))$

definition $\text{staged-security-with-data-state-query-length-committed-prefix-hit}$

$:: 'f \text{ staged-adversary} \Rightarrow$
 $((('f \text{ staged-proof-data} \times 'f \text{ protocol-channel}) \times \text{unit list}) \times$
 $'f \text{ protocol-channel}) \text{ option} \Rightarrow \text{bool}$

where

$\text{staged-security-with-data-state-query-length-committed-prefix-hit } A \text{ out} \longleftrightarrow$
 $(\text{case out of}$
 $\text{None} \Rightarrow \text{False}$
 $| \text{Some } (((\text{data}, \text{attacker-state}), -), -) \Rightarrow$
 $(\exists i \text{ prefix prefix-state raw raw-state.}$
 $i < \text{rounds} \wedge$
 $\text{Some } (\text{data}, \text{attacker-state}) \in$
 set-dist
 $(\text{execute } (\text{checked-staged-transcript-program } A)$
 $\text{adversary-initial-state}) \wedge$
 $\text{Some } (((\text{prefix}, \text{prefix-state}), \text{raw}), \text{raw-state}) \in$
 set-dist
 $(\text{execute } (\text{checked-staged-query-prefix-receive-with-state } A \ i)$
 $\text{adversary-initial-state}) \wedge$
 $\text{checked-staged-query-prefix-dynamic-index-hit}$
 $\text{staged-query-prefix-length-committed-query-target}$
 $(\text{Some } (((\text{prefix}, \text{prefix-state}), \text{raw}), \text{raw-state}))))))$

definition $\text{staged-security-with-data-state-query-tree-output-hit}$

$:: 'f \text{ staged-adversary} \Rightarrow$
 $((('f \text{ staged-proof-data} \times 'f \text{ protocol-channel}) \times \text{unit list}) \times$
 $'f \text{ protocol-channel}) \text{ option} \Rightarrow \text{bool}$

where

staged-security-with-data-state-query-tree-output-hit A $out \longleftrightarrow$
 (case out of
 None $\Rightarrow$ False
 | Some (((data, attacker-state), result), final-state) $\Rightarrow$
 (let s =
 verifier-state-from-adversary attacker-state
 (staged-proof-transcript data)
 in
 ($\exists$ i prefix prefix-state raw raw-state trace-table
 composition-table as query-idxs trace-tree composition-tree.
 i < rounds $\wedge$
 Some (data, attacker-state) $\in$
 set-dist
 (execute (checked-staged-transcript-program A)
 adversary-initial-state) $\wedge$
 Some (result, final-state) $\in$
 set-dist (execute verify-monad s) $\wedge$
 accepted-with-bound-tables s (Some (result, final-state))
 trace-table composition-table as query-idxs $\wedge$
 fmlookup (HashMap final-state)
 (QueryIndexChallenge i
 (state-after-query-chunks
 (staged-query-start-hash data)
 (staged-query-chunks data) i)) =
 Some raw $\wedge$
 index (to-nat raw) $\in$
 query-sampling-success-space trace-table composition-table
 (staged-alphas data) $\wedge$
 Some (((prefix, prefix-state), raw), raw-state) $\in$
 set-dist
 (execute (checked-staged-query-prefix-receive-with-state A i)
 adversary-initial-state) $\wedge$
 sqp-composition-fri-roots prefix $\neq \square$ $\wedge$
 created-tree trace-table trace-tree final-state $\wedge$
 sqp-trace-root prefix = value trace-tree $\wedge$
 created-tree composition-table composition-tree final-state $\wedge$
 hd (sqp-composition-fri-roots prefix) = value composition-tree $\wedge$
 hash-map-new-output-hit
 (set-tree trace-tree $\cup$ set-tree composition-tree)
 prefix-state final-state)))

lemma *staged-partial-query-target-subset-prefix-completion:*

assumes match: staged-query-prefix-matches-data i prefix data

and ext: prefix-state $\leq$ attacker-state

shows

staged-partial-query-target data attacker-state $\subseteq$

staged-query-prefix-partial-query-target i prefix prefix-state

unfolding staged-query-prefix-partial-query-target-def

using *match ext* **by** *blast*

lemma *staged-query-prefix-partial-query-target-subset:*

staged-query-prefix-partial-query-target i prefix prefix-state $\subseteq$
query-sample-space

unfolding *staged-query-prefix-partial-query-target-def*
staged-partial-query-target-def

using *query-header-supported-partial-union-good-sets-subset* **by** *blast*

lemma *staged-query-prefix-committed-query-target-subset:*

staged-query-prefix-committed-query-target prefix prefix-state $\subseteq$
query-sample-space

unfolding *staged-query-prefix-committed-query-target-def*
by (rule *query-header-committed-union-good-sets-subset*)

lemma *staged-query-prefix-committed-query-targetI:*

assumes *candidate:*

(*trace-table*, *composition-table*) $\in$
query-header-committed-table-candidates prefix-state
(*sqp-trace-root prefix*) (*sqp-composition-fri-roots prefix*)

and *hit:*

idx $\in$ *query-sampling-success-space trace-table composition-table*
(*sqp-alphas prefix*)

shows *idx* $\in$ *staged-query-prefix-committed-query-target prefix prefix-state*

proof –

have *idx-space:* *idx* $\in$ *query-sample-space*

using *query-sampling-success-space-subset hit* **by** *blast*

show *?thesis*

unfolding *staged-query-prefix-committed-query-target-def*
query-header-committed-union-good-sets-def

using *candidate hit idx-space* **by** *blast*

qed

lemma *staged-query-prefix-committed-query-target-fraction-bound-if-unique-candidate:*

assumes *unique:*

$\exists$ *trace-table composition-table.*
query-header-committed-table-candidates prefix-state
(*sqp-trace-root prefix*)
(*sqp-composition-fri-roots prefix*) $\subseteq$
 $\{(trace-table, composition-table)\}$

shows

nnreal
(*card*
(*staged-query-prefix-committed-query-target prefix prefix-state*)) /
nnreal (card query-sample-space) $\leq$ *query-error-bound*

unfolding *staged-query-prefix-committed-query-target-def*

by (rule

query-header-committed-union-good-sets-fraction-bound-if-unique-candidate
[*OF unique query-sampling-success-space-subset*

query-sampling-success-space-fraction-bound-query-sample-space])

lemma *staged-query-prefix-committed-query-target-fraction-bound-if-no-root-collisions:*

assumes *trace-clean:*

¬ *merkle-root-binding-collision* (*sqp-trace-root prefix*)
prefix-state

and *comp-clean:*

∧ *rt. sqp-composition-fri-roots prefix* ≠ [] ⇒
rt = hd (sqp-composition-fri-roots prefix) ⇒
 ¬ *merkle-root-binding-collision rt prefix-state*

shows

nnreal
 (*card*
 (*staged-query-prefix-committed-query-target prefix prefix-state*)) /
nnreal (card query-sample-space) ≤ *query-error-bound*

proof —

have *unique:*

∃ *trace-table composition-table.*
query-header-committed-table-candidates prefix-state
 (*sqp-trace-root prefix*)
 (*sqp-composition-fri-roots prefix*) ⊆
 {(*trace-table, composition-table*)}

by (*rule*

query-header-committed-table-candidates-subset-singleton-if-no-root-collisions
 [*OF trace-clean comp-clean*])

show *?thesis*

by (*rule*

staged-query-prefix-committed-query-target-fraction-bound-if-unique-candidate
 [*OF unique*])

qed

lemma *staged-query-prefix-length-committed-query-target-subset:*

staged-query-prefix-length-committed-query-target prefix prefix-state
 ⊆ *query-sample-space*

unfolding *staged-query-prefix-length-committed-query-target-def* **by** *auto*

lemma *staged-query-prefix-length-committed-query-targetI:*

assumes *candidate:*

(*trace-table, composition-table*) ∈
staged-query-prefix-length-committed-table-candidates prefix prefix-state

and *hit:*

idx ∈ *query-sampling-success-space trace-table composition-table*
 (*sqp-alphas prefix*)

shows

idx ∈ *staged-query-prefix-length-committed-query-target prefix*
prefix-state

proof —

have *idx-space:* *idx* ∈ *query-sample-space*

using *query-sampling-success-space-subset hit* **by** *blast*

```

show ?thesis
  unfolding staged-query-prefix-length-committed-query-target-def
  using candidate hit idx-space by blast
qed

lemma staged-query-prefix-clean-length-committed-query-target-subset:
  staged-query-prefix-clean-length-committed-query-target prefix prefix-state
     $\subseteq$  query-sample-space
  unfolding staged-query-prefix-clean-length-committed-query-target-def
  using staged-query-prefix-length-committed-query-target-subset
  by simp

lemma staged-query-prefix-length-committed-query-target-subset-committed:
  staged-query-prefix-length-committed-query-target prefix prefix-state
     $\subseteq$  staged-query-prefix-committed-query-target prefix prefix-state
  unfolding staged-query-prefix-length-committed-query-target-def
    staged-query-prefix-length-committed-table-candidates-def
    staged-query-prefix-committed-query-target-def
    query-header-committed-union-good-sets-def
    query-header-committed-table-candidates-def
  by auto

lemma staged-query-prefix-length-committed-table-candidates-subset-singleton-if-no-hash-collision:
  assumes clean:  $\neg$  hash-map-output-collision prefix-state
  shows
     $\exists$  trace-table composition-table.
      staged-query-prefix-length-committed-table-candidates prefix
        prefix-state  $\subseteq$   $\{(trace-table, composition-table)\}$ 
  proof (cases
    staged-query-prefix-length-committed-table-candidates prefix
    prefix-state =  $\{\}$ )
  case True
  then show ?thesis
    by (intro exI[of -]  $\square$  exI[of -]  $\square$ ) simp
next
  case False
  then obtain pair where pair:
    pair  $\in$ 
      staged-query-prefix-length-committed-table-candidates prefix
      prefix-state
  by auto
  obtain trace-table composition-table where pair-eq:
    pair = (trace-table, composition-table)
  by (cases pair)
  then have candidate:
    (trace-table, composition-table)  $\in$ 
      staged-query-prefix-length-committed-table-candidates prefix
      prefix-state
  using pair by simp

```

```

have trace-bind:
  merkle-root-binds-table (sqp-trace-root prefix) trace-table
    prefix-state
  using candidate
  unfolding staged-query-prefix-length-committed-table-candidates-def
  by simp
have comp-nonempty: sqp-composition-fri-roots prefix  $\neq []$ 
  using candidate
  unfolding staged-query-prefix-length-committed-table-candidates-def
  by simp
have comp-bind:
  merkle-root-binds-table (hd (sqp-composition-fri-roots prefix))
    composition-table prefix-state
  using candidate
  unfolding staged-query-prefix-length-committed-table-candidates-def
  by simp
have trace-len: length trace-table = clength * scale
  using candidate
  unfolding staged-query-prefix-length-committed-table-candidates-def
  by simp
have comp-len: length composition-table = clength * scale
  using candidate
  unfolding staged-query-prefix-length-committed-table-candidates-def
  by simp
have trace-nonempty: trace-table  $\neq []$ 
  using trace-len eval-domain-size-pos by auto
have comp-nonempty-table: composition-table  $\neq []$ 
  using comp-len eval-domain-size-pos by auto
have subset:
  staged-query-prefix-length-committed-table-candidates prefix
    prefix-state  $\subseteq \{(trace-table, composition-table)\}$ 
proof
  fix pair
  assume pair-in:
    pair  $\in$ 
      staged-query-prefix-length-committed-table-candidates prefix
        prefix-state
  obtain trace-table' composition-table' where pair-eq:
    pair = (trace-table', composition-table')
  by (cases pair)
  have trace-bind':
    merkle-root-binds-table (sqp-trace-root prefix) trace-table'
      prefix-state
  using pair-in unfolding pair-eq
    staged-query-prefix-length-committed-table-candidates-def
  by simp
  have comp-bind':
    merkle-root-binds-table (hd (sqp-composition-fri-roots prefix))
      composition-table' prefix-state

```

```

using pair-in unfolding pair-eq
  staged-query-prefix-length-committed-table-candidates-def
by simp
have trace-len':
  length trace-table' = clength * scale
using pair-in unfolding pair-eq
  staged-query-prefix-length-committed-table-candidates-def
by simp
have comp-len':
  length composition-table' = clength * scale
using pair-in unfolding pair-eq
  staged-query-prefix-length-committed-table-candidates-def
by simp
have trace-eq: trace-table' = trace-table
by (rule merkle-root-binds-same-length-tables-unique-if-no-hash-collision
  [OF trace-bind' trace-bind - clean])
  (simp add: trace-len' trace-len)
have comp-eq: composition-table' = composition-table
by (rule merkle-root-binds-same-length-tables-unique-if-no-hash-collision
  [OF comp-bind' comp-bind - clean])
  (simp add: comp-len' comp-len)
show pair ∈ {(trace-table, composition-table)}
  unfolding pair-eq trace-eq comp-eq by simp
qed
show ?thesis
  using subset by blast
qed

lemma staged-query-prefix-length-committed-query-target-fraction-bound-if-no-hash-collision:
assumes clean:  $\neg$  hash-map-output-collision prefix-state
shows
  nnreal
    (card
      (staged-query-prefix-length-committed-query-target prefix
        prefix-state)) /
    nnreal (card query-sample-space)  $\leq$  query-error-bound
proof –
from
  staged-query-prefix-length-committed-table-candidates-subset-singleton-if-no-hash-collision
  [OF clean, of prefix]
obtain trace-table composition-table where candidates:
  staged-query-prefix-length-committed-table-candidates prefix
  prefix-state  $\subseteq$  {(trace-table, composition-table)}
by blast
have union-subset:
  staged-query-prefix-length-committed-query-target prefix prefix-state
   $\subseteq$ 
  query-sampling-success-space trace-table composition-table
  (sqp-alphas prefix)

```

```

unfolding staged-query-prefix-length-committed-query-target-def
using candidates by auto
have finite-good:
  finite
    (query-sampling-success-space trace-table composition-table
      (sqp-alphas prefix))
  by (rule finite-subset[OF query-sampling-success-space-subset
    finite-query-sample-space])
have card-le:
  card
    (staged-query-prefix-length-committed-query-target prefix
      prefix-state) ≤
  card
    (query-sampling-success-space trace-table composition-table
      (sqp-alphas prefix))
  by (rule card-mono[OF finite-good union-subset])
have nnreal
  (card
    (staged-query-prefix-length-committed-query-target prefix
      prefix-state)) /
  nnreal (card query-sample-space) ≤
  nnreal
    (card
      (query-sampling-success-space trace-table composition-table
        (sqp-alphas prefix))) /
  nnreal (card query-sample-space)
  by (rule nnreal-nat-divide-right-mono[OF card-le])
also have ... ≤ query-error-bound
  by (rule query-sampling-success-space-fraction-bound-query-sample-space)
finally show ?thesis .
qed

lemma staged-query-prefix-clean-length-committed-query-target-fraction-bound:
shows
  nnreal
    (card
      (staged-query-prefix-clean-length-committed-query-target prefix
        prefix-state)) /
  nnreal (card query-sample-space) ≤ query-error-bound
proof (cases hash-map-output-collision prefix-state)
case True
then show ?thesis
  unfolding staged-query-prefix-clean-length-committed-query-target-def
  by simp
next
case False
have bound:
  nnreal
    (card

```

```

      (staged-query-prefix-length-committed-query-target prefix
       prefix-state)) /
    nreal (card query-sample-space) ≤ query-error-bound
  by (rule
      staged-query-prefix-length-committed-query-target-fraction-bound-if-no-hash-collision
      [OF False])
  then show ?thesis
  using False
  unfolding staged-query-prefix-clean-length-committed-query-target-def
  by simp
qed

```

lemma *checked-staged-query-prefix-length-committed-hit-imp-clean-hit-or-hash-collision:*

```

  assumes hit:
    checked-staged-query-prefix-dynamic-index-hit
    staged-query-prefix-length-committed-query-target out
  shows
    checked-staged-query-prefix-dynamic-index-hit
    staged-query-prefix-clean-length-committed-query-target out ∨
    checked-staged-query-prefix-hash-map-output-collision out
  proof (cases out)
    case None
    then show ?thesis
    using hit
    unfolding checked-staged-query-prefix-dynamic-index-hit-def by simp
  next
    case (Some packed)
    then obtain prefix prefix-state raw raw-state where out-eq:
      out = Some (((prefix, prefix-state), raw), raw-state)
      by (cases packed, auto split: prod.splits)
    show ?thesis
    proof (cases hash-map-output-collision prefix-state)
      case True
      then have checked-staged-query-prefix-hash-map-output-collision out
        unfolding out-eq checked-staged-query-prefix-hash-map-output-collision-def
        by simp
      then show ?thesis by simp
    next
      case False
      have
        checked-staged-query-prefix-dynamic-index-hit
        staged-query-prefix-clean-length-committed-query-target out
        using hit False
        unfolding out-eq checked-staged-query-prefix-dynamic-index-hit-def
        staged-query-prefix-clean-length-committed-query-target-def
        by simp
      then show ?thesis by simp
    qed
  qed

```


lemma *staged-query-prefix-clean-committed-query-target-subset*:
staged-query-prefix-clean-committed-query-target prefix prefix-state
 $\subseteq$ *query-sample-space*
unfolding *staged-query-prefix-clean-committed-query-target-def*
using *staged-query-prefix-committed-query-target-subset*[of *prefix prefix-state*]
by *simp*

lemma *staged-query-prefix-clean-committed-query-target-subset-committed*:
staged-query-prefix-clean-committed-query-target prefix prefix-state
 $\subseteq$ *staged-query-prefix-committed-query-target prefix prefix-state*
unfolding *staged-query-prefix-clean-committed-query-target-def* **by** *auto*

lemma *staged-query-prefix-clean-committed-query-target-fraction-bound*:
shows *nnreal*
(card
(staged-query-prefix-clean-committed-query-target prefix prefix-state)) /
nnreal (card query-sample-space) ≤ query-error-bound
proof (*cases*
staged-query-prefix-committed-roots-clean prefix prefix-state)
case *True*
then have *trace-clean*:
 $\neg$ *merkle-root-binding-collision (sqp-trace-root prefix)*
prefix-state
and *comp-clean*:
 $\bigwedge rt. \text{sqp-composition-fri-roots } prefix \neq [] \implies$
 $rt = \text{hd } (\text{sqp-composition-fri-roots } prefix) \implies$
 $\neg \text{merkle-root-binding-collision } rt \text{ } prefix\text{-state}$
unfolding *staged-query-prefix-committed-roots-clean-def* **by** *blast+*
show *?thesis*
unfolding *staged-query-prefix-clean-committed-query-target-def*
using *True*
staged-query-prefix-committed-query-target-fraction-bound-if-no-root-collisions
 $[OF \text{ } trace\text{-clean } comp\text{-clean}]$
by *simp*
next
case *False*
then show *?thesis*
unfolding *staged-query-prefix-clean-committed-query-target-def* **by** *simp*
qed

lemma *checked-staged-query-prefix-committed-hit-imp-clean-hit-or-root-collision*:
assumes *hit*:
checked-staged-query-prefix-dynamic-index-hit
 $(\lambda prefix \text{ } prefix\text{-state}.$
staged-query-prefix-committed-query-target prefix prefix-state) out
shows
checked-staged-query-prefix-dynamic-index-hit
staged-query-prefix-clean-committed-query-target out $\vee$

```

    checked-staged-query-prefix-committed-root-collision out
proof (cases out)
  case None
  then show ?thesis
    using hit
    unfolding checked-staged-query-prefix-dynamic-index-hit-def by simp
next
  case (Some packed)
  then obtain prefix prefix-state raw raw-state where out-eq:
    out = Some (((prefix, prefix-state), raw), raw-state)
    by (cases packed, auto split: prod.splits)
  show ?thesis
  proof (cases
    staged-query-prefix-committed-roots-clean prefix prefix-state)
    case True
    have clean-hit:
      checked-staged-query-prefix-dynamic-index-hit
      staged-query-prefix-clean-committed-query-target out
    using hit True
    unfolding out-eq checked-staged-query-prefix-dynamic-index-hit-def
      staged-query-prefix-clean-committed-query-target-def
    by simp
    then show ?thesis by simp
  next
  case False
  then have
    checked-staged-query-prefix-committed-root-collision out
    unfolding out-eq checked-staged-query-prefix-committed-root-collision-def
    by simp
  then show ?thesis by simp
qed
qed

```

lemma staged-partial-query-target-subset:
 staged-partial-query-target data attacker-state $\subseteq$
 query-sample-space
unfolding staged-partial-query-target-def
by (rule query-header-supported-partial-union-good-sets-subset)

lemma staged-partial-query-target-fraction-bound-if-unique-candidate:
assumes unique:
 $\exists$ trace-table composition-table.
 query-header-supported-partial-table-candidates
 (verifier-state-from-adversary attacker-state
 (staged-proof-transcript data))
 (staged-trace-root data)
 (staged-trace-fri-roots data)
 (staged-trace-final data)
 (staged-alphas data)

```

    (staged-degree data)
    (staged-composition-fri-roots data)
    (staged-composition-final data)  $\subseteq$ 
    {(trace-table, composition-table)}
shows
  nnreal (card (staged-partial-query-target data attacker-state)) /
  nnreal (card query-sample-space)  $\leq$  query-error-bound
unfolding staged-partial-query-target-def
by (rule
  query-header-supported-partial-union-good-sets-fraction-bound-if-unique-candidate
  [OF unique query-sampling-success-space-subset
  query-sampling-success-space-fraction-bound-query-sample-space])

definition checked-staged-query-prefix-fixed-transcript-target-hit
:: 'f staged-proof-data  $\Rightarrow$  'f protocol-channel  $\Rightarrow$ 
  (((('f staged-query-prefix-data  $\times$  'f protocol-channel)  $\times$  'f)  $\times$ 
  'f protocol-channel) option  $\Rightarrow$  bool
where
  checked-staged-query-prefix-fixed-transcript-target-hit data
  attacker-state out  $\longleftrightarrow$ 
  checked-staged-query-prefix-dynamic-index-hit
  ( $\lambda$  -. staged-partial-query-target data attacker-state) out

lemma checked-staged-query-prefix-dynamic-index-hit-bound:
fixes C :: prob
assumes raw-bound: query-index-raw-preimage-bound
and prefix-bound:
   $\bigwedge$  prefix prefix-state.
  Some ((prefix, prefix-state), prefix-state)  $\in$ 
  set-dist
  (execute (checked-staged-query-prefix-with-state A i) s)  $\Rightarrow$ 
  query-future-fresh prefix-state  $\wedge$ 
  B prefix prefix-state  $\subseteq$  query-sample-space  $\wedge$ 
  nnreal (card (B prefix prefix-state)) /
  nnreal (card query-sample-space)  $\leq$  C
shows
  wp-event (checked-staged-query-prefix-receive-with-state A i)
  (checked-staged-query-prefix-dynamic-index-hit B) s  $\leq$  C
unfolding checked-staged-query-prefix-receive-with-state-def
proof (rule wp-event-bind-bound-by-cont)
show  $\neg$  checked-staged-query-prefix-dynamic-index-hit B None
unfolding checked-staged-query-prefix-dynamic-index-hit-def by simp
next
fix prefix-pack prefix-state
assume prefix-support:
  Some (prefix-pack, prefix-state)  $\in$ 
  set-dist
  (execute (checked-staged-query-prefix-with-state A i) s)
obtain prefix captured-state where prefix-pack-eq:

```

```

    prefix-pack = (prefix, captured-state)
  by (cases prefix-pack) simp
have captured-eq: captured-state = prefix-state
  using checked-staged-query-prefix-with-state-outcome-state
    [OF prefix-support[unfolded prefix-pack-eq]]
  by simp
have support':
  Some ((prefix, prefix-state), prefix-state) ∈
    set-dist
      (execute (checked-staged-query-prefix-with-state A i) s)
  using checked-staged-query-prefix-with-state-outcome-canonical
    [OF prefix-support[unfolded prefix-pack-eq]]
    captured-eq
  by simp
from prefix-bound[OF support'] have future:
  query-future-fresh prefix-state
and subset:
  B prefix prefix-state ⊆ query-sample-space
and frac:
  nnreal (card (B prefix prefix-state)) /
    nnreal (card query-sample-space) ≤ C
  by blast+
have receive-bound:
  wp-event
    (receive-query-index-challenge ≫ (λraw.
      return (prefix-pack, raw)))
    (checked-staged-query-prefix-dynamic-index-hit B) prefix-state ≤
    C
proof -
  let ?P =
    λout. case out of None ⇒ False
      | Some (raw, -) ⇒ index (to-nat raw) ∈ B prefix prefix-state
  have cont-eq:
    wp-event
      (receive-query-index-challenge ≫ (λraw.
        return (prefix-pack, raw)))
      (checked-staged-query-prefix-dynamic-index-hit B) prefix-state =
      wp-event receive-query-index-challenge ?P prefix-state
  unfolding wp-event-def
  apply (subst wp-bind)
  apply (rule arg-cong[where
    f = λQ. wp receive-query-index-challenge Q prefix-state])
  apply (rule ext)
  apply (simp add: wp-return prefix-pack-eq captured-eq
    checked-staged-query-prefix-dynamic-index-hit-def
    split: option.splits prod.splits)
  done
show ?thesis
  unfolding cont-eq

```

by (rule order-trans
 [OF wp-receive-query-index-challenge-fresh-index-set-raw-bound
 [OF future raw-bound subset] frac])
 qed
 show wp-event
 (receive-query-index-challenge $\gg$ (λ raw.
 return (prefix-pack, raw)))
 (checked-staged-query-prefix-dynamic-index-hit B) prefix-state $\leq$
 C
 by (rule receive-bound)
 qed
lemma checked-staged-query-prefix-fixed-transcript-target-hit-bound:
 fixes C :: prob
 assumes raw-bound: query-index-raw-preimage-bound
 and prefix-bound:
 $\bigwedge$ prefix prefix-state.
 Some ((prefix, prefix-state), prefix-state) $\in$
 set-dist
 (execute (checked-staged-query-prefix-with-state A i) s) $\implies$
 query-future-fresh prefix-state
 and subset: staged-partial-query-target data attacker-state $\subseteq$
 query-sample-space
 and frac:
 nnreal (card (staged-partial-query-target data attacker-state)) /
 nnreal (card query-sample-space) $\leq$ C
 shows
 wp-event (checked-staged-query-prefix-receive-with-state A i)
 (checked-staged-query-prefix-fixed-transcript-target-hit data
 attacker-state) s $\leq$ C
 unfolding checked-staged-query-prefix-fixed-transcript-target-hit-def
 by (rule checked-staged-query-prefix-dynamic-index-hit-bound
 [OF raw-bound])
 (use prefix-bound subset frac in auto)

lemma checked-staged-query-prefix-committed-target-hit-bound:
 assumes raw-bound: query-index-raw-preimage-bound
 and prefix-bound:
 $\bigwedge$ prefix prefix-state.
 Some ((prefix, prefix-state), prefix-state) $\in$
 set-dist
 (execute (checked-staged-query-prefix-with-state A i) s) $\implies$
 query-future-fresh prefix-state $\wedge$
 $\neg$ merkle-root-binding-collision (sqp-trace-root prefix)
 prefix-state $\wedge$
 ($\forall$ rt. sqp-composition-fri-roots prefix $\neq$ [] $\longrightarrow$
 rt = hd (sqp-composition-fri-roots prefix) $\longrightarrow$
 $\neg$ merkle-root-binding-collision rt prefix-state)
 shows

```

wp-event (checked-staged-query-prefix-receive-with-state A i)
  (checked-staged-query-prefix-dynamic-index-hit
    (λprefix prefix-state.
      staged-query-prefix-committed-query-target prefix prefix-state))
  s ≤ query-error-bound
proof (rule checked-staged-query-prefix-dynamic-index-hit-bound
  [OF raw-bound])
fix prefix prefix-state
assume support:
  Some ((prefix, prefix-state), prefix-state) ∈
    set-dist
    (execute (checked-staged-query-prefix-with-state A i) s)
from prefix-bound[OF support]
have future: query-future-fresh prefix-state
and trace-clean:
  ¬ merkle-root-binding-collision (sqp-trace-root prefix)
    prefix-state
and comp-clean:
  ∧ rt. sqp-composition-fri-roots prefix ≠ [] ⇒
    rt = hd (sqp-composition-fri-roots prefix) ⇒
    ¬ merkle-root-binding-collision rt prefix-state
by blast+
show
  query-future-fresh prefix-state ∧
  staged-query-prefix-committed-query-target prefix prefix-state
    ⊆ query-sample-space ∧
  nnreal
    (card
      (staged-query-prefix-committed-query-target prefix prefix-state)) /
  nnreal (card query-sample-space) ≤ query-error-bound
using future
  staged-query-prefix-committed-query-target-subset[of prefix prefix-state]
  staged-query-prefix-committed-query-target-fraction-bound-if-no-root-collisions
    [OF trace-clean comp-clean]
by blast
qed

```

definition *checked-staged-query-prefix-dynamic-index-prehit*

```

:: ('f staged-query-prefix-data ⇒ 'f protocol-channel ⇒
  nat set) ⇒
  (((('f staged-query-prefix-data × 'f protocol-channel) × 'f) ×
    'f protocol-channel) option ⇒ bool

```

where

```

checked-staged-query-prefix-dynamic-index-prehit B out ⇔
  (case out of
    None ⇒ False
  | Some (((prefix, prefix-state), -), -) ⇒
    (∃ raw ∈ query-index-raw-preimage (B prefix prefix-state).
      fmlookup (HashMap prefix-state)

```

(*QueryIndexChallenge* (*PQueryCounter prefix-state*)
(PState prefix-state)) = *Some raw*)

definition *checked-staged-query-prefix-dynamic-index-prehit-relation*

$:: 'f \text{ staged-adversary} \Rightarrow \text{nat} \Rightarrow 'f \text{ protocol-channel} \Rightarrow$
 $('f \text{ staged-query-prefix-data} \Rightarrow 'f \text{ protocol-channel} \Rightarrow$
 $\text{nat set}) \Rightarrow$
 $'f \text{ protocol-hash-input} \Rightarrow 'f \Rightarrow \text{bool}$

where

checked-staged-query-prefix-dynamic-index-prehit-relation A i s B x y
 $\longleftrightarrow$
 $(\exists \text{ prefix prefix-state.}$
 $\text{Some } ((\text{prefix, prefix-state}), \text{prefix-state}) \in$
 set-dist
 $(\text{execute } (\text{checked-staged-query-prefix-with-state A i}) s) \wedge$
 $x = \text{QueryIndexChallenge } (\text{PQueryCounter prefix-state})$
 $(\text{PState prefix-state}) \wedge$
 $y \in \text{query-index-raw-preimage } (B \text{ prefix prefix-state}))$

lemma *checked-staged-query-prefix-dynamic-index-prehit-imp-relation-hit:*

assumes *empty: HashMap s = fmempty*

and *prefix-support:*

$\text{Some } ((\text{prefix, prefix-state}), \text{prefix-state}) \in$
 set-dist
 $(\text{execute } (\text{checked-staged-query-prefix-with-state A i}) s)$

and *receive-support:*

$\text{Some } (((\text{prefix, prefix-state}), \text{raw}), \text{raw-state}) \in$
 set-dist
 $(\text{execute } (\text{checked-staged-query-prefix-receive-with-state A i}) s)$

and *prehit:*

checked-staged-query-prefix-dynamic-index-prehit B
 $(\text{Some } (((\text{prefix, prefix-state}), \text{raw}), \text{raw-state}))$

shows

hash-relation-hit-event
 $(\text{checked-staged-query-prefix-dynamic-index-prehit-relation A i s B})$
 $s (\text{Some } (((\text{prefix, prefix-state}), \text{raw}), \text{raw-state}))$

proof —

from *prehit* **obtain** *old-raw* **where** *old-raw-in:*

$\text{old-raw} \in \text{query-index-raw-preimage } (B \text{ prefix prefix-state})$

and *lookup-prefix:*

$\text{fmlookup } (\text{HashMap prefix-state})$
 $(\text{QueryIndexChallenge } (\text{PQueryCounter prefix-state})$
 $(\text{PState prefix-state})) = \text{Some old-raw}$

unfolding *checked-staged-query-prefix-dynamic-index-prehit-def*

by *auto*

have *recv:*

$\text{Some } (\text{raw, raw-state}) \in$
 $\text{set-dist } (\text{execute receive-query-index-challenge prefix-state})$

proof —

```

from receive-support obtain u where
  prefix-u:
    Some ((prefix, prefix-state), u) ∈
      set-dist
        (execute (checked-staged-query-prefix-with-state A i s) s)
  and recv-u:
    Some (raw, raw-state) ∈
      set-dist (execute receive-query-index-challenge u)
  unfolding checked-staged-query-prefix-receive-with-state-def
  by (auto elim!: set-dist-bindE)
have u = prefix-state
  by (rule checked-staged-query-prefix-with-state-outcome-state
    [OF prefix-u])
then show ?thesis
  using recv-u by simp
qed
have prefix-ext: prefix-state ≤ raw-state
  by (rule receive-query-index-challenge-extends[OF recv])
have lookup-raw-state:
  fmlookup (HashMap raw-state)
    (QueryIndexChallenge (PQueryCounter prefix-state)
      (PState prefix-state)) = Some old-raw
  by (rule hash-extension-lookup[OF lookup-prefix prefix-ext])
have lookup-initial:
  fmlookup (HashMap s)
    (QueryIndexChallenge (PQueryCounter prefix-state)
      (PState prefix-state)) = None
  using empty by simp
have rel:
  checked-staged-query-prefix-dynamic-index-prehit-relation A i s B
    (QueryIndexChallenge (PQueryCounter prefix-state)
      (PState prefix-state)) old-raw
  unfolding
    checked-staged-query-prefix-dynamic-index-prehit-relation-def
  using prefix-support old-raw-in by blast
have hit:
  hash-relation-hit
    (checked-staged-query-prefix-dynamic-index-prehit-relation A i s B)
    s raw-state
  unfolding hash-relation-hit-def
  by (intro exI[of -
    QueryIndexChallenge (PQueryCounter prefix-state)
      (PState prefix-state)]
    exI[of - old-raw] conjI)
    (use lookup-initial lookup-raw-state rel in simp-all)
show ?thesis
  unfolding hash-relation-hit-event-def using hit by simp
qed

```



```

lemma checked-staged-query-prefix-dynamic-index-prehit-bound-by-relation-hit:
  assumes empty:  $\text{HashMap } s = \text{fmempty}$ 
  shows
     $\text{wp-event } (\text{checked-staged-query-prefix-receive-with-state } A \ i)$ 
       $(\text{checked-staged-query-prefix-dynamic-index-prehit } B) \ s \leq$ 
       $\text{wp-event } (\text{checked-staged-query-prefix-receive-with-state } A \ i)$ 
       $(\text{hash-relation-hit-event } (\text{checked-staged-query-prefix-dynamic-index-prehit-relation } A \ i \ s \ B)$ 
       $s) \ s$ 
proof (rule wp-event-mono-on-support)
  fix out
  assume support:
     $out \in$ 
     $\text{set-dist}$ 
     $(\text{execute } (\text{checked-staged-query-prefix-receive-with-state } A \ i) \ s)$ 
  and prehit: checked-staged-query-prefix-dynamic-index-prehit B out
  show
    hash-relation-hit-event
     $(\text{checked-staged-query-prefix-dynamic-index-prehit-relation } A \ i \ s \ B)$ 
     $s \ out$ 
proof (cases out)
  case None
  then show ?thesis
    using prehit
    unfolding checked-staged-query-prefix-dynamic-index-prehit-def
    by simp
  next
  case (Some packed)
  then obtain prefix prefix-state raw raw-state where out-eq:
     $out = \text{Some } (((\text{prefix}, \text{prefix-state}), \text{raw}), \text{raw-state})$ 
    by (cases packed, auto split: prod.splits)
  have receive-support:
     $\text{Some } (((\text{prefix}, \text{prefix-state}), \text{raw}), \text{raw-state}) \in$ 
     $\text{set-dist}$ 
     $(\text{execute } (\text{checked-staged-query-prefix-receive-with-state } A \ i) \ s)$ 
    using support unfolding out-eq .
  have prefix-support:
     $\text{Some } ((\text{prefix}, \text{prefix-state}), \text{prefix-state}) \in$ 
     $\text{set-dist}$ 
     $(\text{execute } (\text{checked-staged-query-prefix-with-state } A \ i) \ s)$ 
  proof –
  from receive-support obtain u where
    prefix-u:
     $\text{Some } ((\text{prefix}, \text{prefix-state}), u) \in$ 
     $\text{set-dist}$ 
     $(\text{execute } (\text{checked-staged-query-prefix-with-state } A \ i) \ s)$ 
    unfolding checked-staged-query-prefix-receive-with-state-def
    by (auto elim!: set-dist-bindE)
  show ?thesis

```

```

    by (rule checked-staged-query-prefix-with-state-outcome-canonical
        [OF prefix-u])
qed
show ?thesis
  unfolding out-eq
  by (rule checked-staged-query-prefix-dynamic-index-prehit-imp-relation-hit
      [OF empty prefix-support receive-support])
    (use prehit out-eq in simp)
qed
qed

lemma checked-staged-query-prefix-dynamic-index-prehit-bound-from-relation-program:
  assumes empty: HashMap s = fmempty
  and relation-program:
    hash-relation-program
    (checked-staged-query-prefix-dynamic-index-prehit-relation A i s B)
    fiber-bound query-bound
    (checked-staged-query-prefix-receive-with-state A i)
  shows
    wp-event (checked-staged-query-prefix-receive-with-state A i)
    (checked-staged-query-prefix-dynamic-index-prehit B) s ≤
    staged-phase-relation-error fiber-bound query-bound
proof -
  have prehit-le-relation:
    wp-event (checked-staged-query-prefix-receive-with-state A i)
    (checked-staged-query-prefix-dynamic-index-prehit B) s ≤
    wp-event (checked-staged-query-prefix-receive-with-state A i)
    (hash-relation-hit-event
      (checked-staged-query-prefix-dynamic-index-prehit-relation A i s B)
      s) s
  by (rule
      checked-staged-query-prefix-dynamic-index-prehit-bound-by-relation-hit
      [OF empty])
  also have ... ≤ staged-phase-relation-error fiber-bound query-bound
  using relation-program
  unfolding hash-relation-program-def hash-relation-budget-def
    staged-phase-relation-error-def
  by blast
  finally show ?thesis .
qed

lemma hash-map-preserving-get:
  hash-map-preserving get
  unfolding hash-map-preserving-def by simp

lemma hash-relation-program-get:
  hash-relation-program R b 0 get
  by (rule hash-relation-program-zero)
    (rule hash-map-preserving-get)

```

lemma *hash-relation-program-modify-preserves-hash-map*:
assumes *bump*:
 $\bigwedge s. \text{HashMap } (\text{bump } s) = \text{HashMap } s$
shows *hash-relation-program* *R* *b* 0 (*modify bump*)
by (*rule hash-relation-program-zero*)
(*rule hash-map-preserving-modify* [*OF bump*])

lemma *hash-relation-program-receive-counted-tagged-random-field-element*:
assumes *bump*:
 $\bigwedge s. \text{HashMap } (\text{bump } s) = \text{HashMap } s$
and *fibers*: $\bigwedge x. \text{card } \{y. R \ x \ y\} \leq b$
shows
hash-relation-program *R* *b* 1
(*protocol-receive-counted-tagged-random-field-element counter bump tag*)

proof –
have *modify-return*:
hash-relation-program *R* *b* (0 + 0)
(*modify bump* $\gg=$ ($\lambda-. \text{return } r$))
for *r* :: 'f
by (*rule hash-relation-program-bind*)
(*rule hash-relation-program-modify-preserves-hash-map* [*OF bump*],
rule hash-relation-program-zero [*OF hash-map-preserving-return*])

have *hash-tail*:
hash-relation-program *R* *b* (1 + (0 + 0))
(*hash* (*tag* (*counter s*) (*PState s*)) $\gg=$
($\lambda r. \text{modify bump } \gg= (\lambda-. \text{return } r)$))
for *s*
by (*rule hash-relation-program-bind*)
(*rule hash-relation-program-hash* [*OF fibers*], *rule modify-return*)

have *hash-relation-program* *R* *b* (0 + (1 + (0 + 0)))
(*get* $\gg=$
($\lambda s. \text{hash } (\text{tag } (\text{counter } s) (\text{PState } s)) \gg=$
($\lambda r. \text{modify bump } \gg= (\lambda-. \text{return } r)$)))
by (*rule hash-relation-program-bind*)
(*rule hash-relation-program-get*, *rule hash-tail*)

then show *?thesis*
unfolding *protocol-receive-counted-tagged-random-field-element-def*
by *simp*

qed

lemma *hash-relation-program-receive-query-index-challenge*:
assumes *fibers*: $\bigwedge x. \text{card } \{y. R \ x \ y\} \leq b$
shows *hash-relation-program* *R* *b* 1
receive-query-index-challenge
unfolding *receive-query-index-challenge-def*
by (*rule hash-relation-program-receive-counted-tagged-random-field-element*
[*OF - fibers*])

simp

lemma *hash-map-preserving-get-bind-return-pair:*
hash-map-preserving (*get* $\gg=$ ($\lambda s. \text{return } (x, s)$))
unfolding *hash-map-preserving-def*
by (*auto elim!*: *set-dist-bindE*)

lemma *hash-map-preserving-imp-hash-range-budget-zero:*
fixes *m* :: (*'r*, (*'f*, *'a*) *protocol-channel-scheme*) *state-monad*
assumes *preserving*: *hash-map-preserving m*
shows *hash-range-budget 0 m*
unfolding *hash-range-budget-def*
proof (*intro allI impI*)
fix *s y t*
assume *Some* (*y*, *t*) $\in$ *set-dist* (*execute m s*)
then have *HashMap t = HashMap s*
using *preserving* **unfolding** *hash-map-preserving-def* **by** *blast*
then show
card (*hash-map-output-values t*) $\leq$
card (*hash-map-output-values s*) + 0
by (*simp add*: *hash-map-output-values-def*)
qed

lemma *hash-map-preserving-imp-hash-collision-budget-zero-scheme:*
fixes *m* :: (*'r*, (*'f*, *'a*) *protocol-channel-scheme*) *state-monad*
assumes *preserving*: *hash-map-preserving m*
shows *hash-collision-budget 0 m*
unfolding *hash-collision-budget-def*
proof (*intro allI impI*)
fix *s* :: (*'f*, *'a*) *protocol-channel-scheme*
assume *clean*: $\neg$ *hash-map-output-collision s*
have *no-event*:
 $\bigwedge \text{out}. \text{out} \in \text{set-dist } (\text{execute } m \text{ } s) \implies$
 $\neg \text{hash-new-collision-event } s \text{ } \text{out}$
proof –
fix *out*
assume *support*: *out* $\in$ *set-dist* (*execute m s*)
show $\neg$ *hash-new-collision-event s out*
proof (*cases out*)
case *None*
then show *?thesis*
unfolding *hash-new-collision-event-def* **by** *simp*
next
case (*Some xt*)
then obtain *x t* **where** *xt*: *xt* = (*x*, *t*)
by (*cases xt*) *simp*
have *same*: *HashMap t = HashMap s*
using *preserving support Some xt*
unfolding *hash-map-preserving-def* **by** *blast*

```

show ?thesis
  using clean same Some xt
  unfolding hash-new-collision-event-def
    hash-map-new-output-collision-def hash-map-output-collision-def
  by simp
qed
qed
have no-event-dom:
   $\bigwedge out. out \in \text{dom} (\text{dist} (\text{execute } m \ s)) \implies$ 
   $\neg \text{hash-new-collision-event } s \ out$ 
  using no-event unfolding set-dist-def by blast
have wp-event m (hash-new-collision-event s) s = 0
  unfolding wp-event-def wp-def dist-expect-def
  by (intro sum.neutral) (simp add: no-event-dom)
then show
  wp-event m (hash-new-collision-event s) s  $\leq$ 
  hash-collision-budget-value
  (card (hash-map-output-values s)) 0
  unfolding hash-collision-budget-value-def by simp
qed

lemma checked-staged-query-prefix-dynamic-index-prehit-relation-fiber-bound:
assumes fibers:
   $\bigwedge x. \text{card}$ 
   $(\bigcup \{ \text{query-index-raw-preimage } (B \ \text{prefix} \ \text{prefix-state}) \mid$ 
   $\text{prefix} \ \text{prefix-state}.$ 
   $\text{Some } ((\text{prefix}, \ \text{prefix-state}), \ \text{prefix-state}) \in$ 
   $\text{set-dist}$ 
   $(\text{execute } (\text{checked-staged-query-prefix-with-state } A \ i) \ s) \wedge$ 
   $x = \text{QueryIndexChallenge } (P\text{QueryCounter } \text{prefix-state})$ 
   $(P\text{State } \text{prefix-state}) \}) \leq b$ 

shows
  card
   $\{ y. \text{checked-staged-query-prefix-dynamic-index-prehit-relation}$ 
   $A \ i \ s \ B \ x \ y \} \leq b$ 

proof –
let ?U =
   $\bigcup \{ \text{query-index-raw-preimage } (B \ \text{prefix} \ \text{prefix-state}) \mid$ 
   $\text{prefix} \ \text{prefix-state}.$ 
   $\text{Some } ((\text{prefix}, \ \text{prefix-state}), \ \text{prefix-state}) \in$ 
   $\text{set-dist}$ 
   $(\text{execute } (\text{checked-staged-query-prefix-with-state } A \ i) \ s) \wedge$ 
   $x = \text{QueryIndexChallenge } (P\text{QueryCounter } \text{prefix-state})$ 
   $(P\text{State } \text{prefix-state}) \}$ 

have subset:
   $\{ y. \text{checked-staged-query-prefix-dynamic-index-prehit-relation}$ 
   $A \ i \ s \ B \ x \ y \} \subseteq ?U$ 
  unfolding checked-staged-query-prefix-dynamic-index-prehit-relation-def
  by blast

```

```

have finite-U: finite ?U
  by simp
have card
  {y. checked-staged-query-prefix-dynamic-index-prehit-relation
   A i s B x y} ≤ card ?U
  by (rule card-mono[OF finite-U subset])
also have ... ≤ b
  by (rule fibers)
finally show ?thesis .
qed

lemma checked-staged-query-prefix-dynamic-index-prehit-relation-fiber-bound-size:
  card
  {y. checked-staged-query-prefix-dynamic-index-prehit-relation
   A i s B x y} ≤ size
proof -
  have subset:
    {y. checked-staged-query-prefix-dynamic-index-prehit-relation
     A i s B x y} ⊆ (UNIV :: 'f set)
  by simp
  have finite-field: finite (UNIV :: 'f set)
  by simp
  have card
    {y. checked-staged-query-prefix-dynamic-index-prehit-relation
     A i s B x y} ≤ card (UNIV :: 'f set)
  by (rule card-mono[OF finite-field subset])
  also have ... = size
  using size-card by simp
  finally show ?thesis .
qed

lemma hash-relation-program-checked-staged-query-prefix-with-state:
  assumes wf: staged-budget-wellformed budgets
  and controlled: staged-adversary-controlled budgets A
  and i-bound: i ≤ rounds
  and fibers:  $\bigwedge x. \text{card } \{y. R \ x \ y\} \leq b$ 
  shows
    hash-relation-program R b
      (staged-query-search-queries budgets i)
      (checked-staged-query-prefix-with-state A i)
  unfolding checked-staged-query-prefix-with-state-def
proof -
  have prefix:
    hash-relation-program R b
      (staged-query-search-queries budgets i)
      (checked-staged-query-challenge-prefix-program A i)
  by (rule hash-relation-program-checked-staged-query-challenge-prefix-program
      [OF wf controlled i-bound fibers])
  have tail:

```

```

 $\wedge$ prefix. hash-relation-program  $R$   $b$  0
  (get  $\gg$  ( $\lambda$ prefix-state. return (prefix, prefix-state)))
by (rule hash-relation-program-zero)
  (rule hash-map-preserving-get-bind-return-pair)
have hash-relation-program  $R$   $b$ 
  (staged-query-search-queries budgets  $i + 0$ )
  (checked-staged-query-challenge-prefix-program  $A$   $i$   $\gg$ 
    ( $\lambda$ prefix. get  $\gg$  ( $\lambda$ prefix-state.
      return (prefix, prefix-state)))))
by (rule hash-relation-program-bind)
  (rule prefix, rule tail)
then show hash-relation-program  $R$   $b$ 
  (staged-query-search-queries budgets  $i$ )
  (checked-staged-query-challenge-prefix-program  $A$   $i$   $\gg$ 
    ( $\lambda$ prefix. get  $\gg$  ( $\lambda$ prefix-state.
      return (prefix, prefix-state)))))
by simp
qed

lemma hash-relation-program-checked-staged-query-prefix-receive-with-state:
  assumes wf: staged-budget-wellformed budgets
  and controlled: staged-adversary-controlled budgets  $A$ 
  and i-bound:  $i \leq \text{rounds}$ 
  and fibers:  $\wedge x. \text{card } \{y. R \ x \ y\} \leq b$ 
shows
  hash-relation-program  $R$   $b$ 
    (staged-query-search-queries budgets  $i + 1$ )
    (checked-staged-query-prefix-receive-with-state  $A$   $i$ )
unfolding checked-staged-query-prefix-receive-with-state-def
proof –
  have prefix:
    hash-relation-program  $R$   $b$ 
      (staged-query-search-queries budgets  $i$ )
      (checked-staged-query-prefix-with-state  $A$   $i$ )
  by (rule hash-relation-program-checked-staged-query-prefix-with-state
    [OF wf controlled i-bound fibers])
  have receive:
     $\wedge$ prefix-pack. hash-relation-program  $R$   $b$  1
      (receive-query-index-challenge  $\gg$  ( $\lambda$ raw.
        return (prefix-pack, raw)))
  proof –
    fix prefix-pack
    have hash-relation-program  $R$   $b$  ( $1 + 0$ )
      (receive-query-index-challenge  $\gg$  ( $\lambda$ raw.
        return (prefix-pack, raw)))
    by (rule hash-relation-program-bind)
      (rule hash-relation-program-receive-query-index-challenge[OF fibers],
        rule hash-relation-program-zero[OF hash-map-preserving-return])
    then show hash-relation-program  $R$   $b$  1

```

```

    (receive-query-index-challenge  $\gg$  ( $\lambda$ raw.
      return (prefix-pack, raw)))
  by simp
qed
show hash-relation-program  $R$   $b$ 
  (staged-query-search-queries budgets  $i + 1$ )
  (checked-staged-query-prefix-with-state  $A$   $i$   $\gg$ 
    ( $\lambda$ prefix-pack. receive-query-index-challenge  $\gg$ 
      ( $\lambda$ raw. return (prefix-pack, raw)))))
  by (rule hash-relation-program-bind)
    (rule prefix, rule receive)
qed

lemma hash-range-budget-checked-staged-query-prefix-with-state:
  assumes wf: staged-budget-wellformed budgets
    and controlled: staged-adversary-controlled budgets  $A$ 
    and i-bound:  $i \leq \text{rounds}$ 
  shows
    hash-range-budget
      (staged-query-search-queries budgets  $i$ )
      (checked-staged-query-prefix-with-state  $A$   $i$ )
  unfolding checked-staged-query-prefix-with-state-def
proof -
  have prefix:
    hash-range-budget
      (staged-query-search-queries budgets  $i$ )
      (checked-staged-query-challenge-prefix-program  $A$   $i$ )
  by (rule hash-range-budget-checked-staged-query-challenge-prefix-program
    [OF wf controlled i-bound])
  have tail:
     $\bigwedge$ prefix. hash-range-budget 0
      (get  $\gg$  ( $\lambda$ prefix-state. return (prefix, prefix-state)))
  by (rule hash-map-preserving-imp-hash-range-budget-zero)
    (rule hash-map-preserving-get-bind-return-pair)
  have hash-range-budget
    (staged-query-search-queries budgets  $i + 0$ )
    (checked-staged-query-challenge-prefix-program  $A$   $i$   $\gg$ 
      ( $\lambda$ prefix. get  $\gg$  ( $\lambda$ prefix-state.
        return (prefix, prefix-state)))))
  by (rule hash-range-budget-bind)
    (rule prefix, rule tail)
  then show hash-range-budget
    (staged-query-search-queries budgets  $i$ )
    (checked-staged-query-challenge-prefix-program  $A$   $i$   $\gg$ 
      ( $\lambda$ prefix. get  $\gg$  ( $\lambda$ prefix-state.
        return (prefix, prefix-state)))))
  by simp
qed

```


lemma *hash-collision-budget-checked-staged-query-prefix-with-state:*
assumes *wf: staged-budget-wellformed budgets*
and *controlled: staged-adversary-controlled budgets A*
and *i-bound: $i \leq \text{rounds}$*
shows
hash-collision-budget
(staged-query-search-queries budgets i)
(checked-staged-query-prefix-with-state A i)
unfolding *checked-staged-query-prefix-with-state-def*
proof –
have *prefix-range:*
hash-range-budget
(staged-query-search-queries budgets i)
(checked-staged-query-challenge-prefix-program A i)
by (rule *hash-range-budget-checked-staged-query-challenge-prefix-program*
 $[OF\ wf\ controlled\ i-bound]$)
have *prefix-collision:*
hash-collision-budget
(staged-query-search-queries budgets i)
(checked-staged-query-challenge-prefix-program A i)
by (rule *hash-collision-budget-checked-staged-query-challenge-prefix-program*
 $[OF\ wf\ controlled\ i-bound]$)
have *tail-range:*
 $\bigwedge \text{prefix. hash-range-budget } 0$
 $(\text{get} \gg (\lambda \text{prefix-state. return } (\text{prefix}, \text{prefix-state})))$
by (rule *hash-map-preserving-imp-hash-range-budget-zero*)
(rule *hash-map-preserving-get-bind-return-pair*)
have *tail-collision:*
 $\bigwedge \text{prefix. hash-collision-budget } 0$
 $(\text{get} \gg (\lambda \text{prefix-state. return } (\text{prefix}, \text{prefix-state})))$
by (rule *hash-map-preserving-imp-hash-collision-budget-zero-scheme*)
(rule *hash-map-preserving-get-bind-return-pair*)
have *hash-collision-budget*
(staged-query-search-queries budgets $i + 0$)
(checked-staged-query-challenge-prefix-program A i $\gg$
 $(\lambda \text{prefix. get} \gg (\lambda \text{prefix-state.}$
 $\text{return } (\text{prefix}, \text{prefix-state}))))$
by (rule *hash-collision-budget-bind*)
(rule *prefix-range*, rule *prefix-collision*, rule *tail-range*,
rule *tail-collision*)
then show *hash-collision-budget*
(staged-query-search-queries budgets i)
(checked-staged-query-challenge-prefix-program A i $\gg$
 $(\lambda \text{prefix. get} \gg (\lambda \text{prefix-state.}$
 $\text{return } (\text{prefix}, \text{prefix-state}))))$
by *simp*
qed

lemma *hash-range-budget-checked-staged-query-prefix-receive-with-state:*

assumes *wf*: staged-budget-wellformed budgets
and *controlled*: staged-adversary-controlled budgets *A*
and *i-bound*: $i \leq \text{rounds}$
shows
hash-range-budget
(*staged-query-search-queries* budgets $i + 1$)
(*checked-staged-query-prefix-receive-with-state* *A* *i*)
unfolding *checked-staged-query-prefix-receive-with-state-def*
proof –
have *prefix*:
hash-range-budget
(*staged-query-search-queries* budgets *i*)
(*checked-staged-query-prefix-with-state* *A* *i*)
by (rule *hash-range-budget-checked-staged-query-prefix-with-state*
[*OF wf controlled i-bound*])
have *receive*:
 $\wedge \text{prefix-pack. hash-range-budget } 1$
(*receive-query-index-challenge* $\gg$ ($\lambda \text{raw.}$
return (*prefix-pack*, *raw*)))
proof –
fix *prefix-pack*
have *hash-range-budget* ($1 + 0$)
(*receive-query-index-challenge* $\gg$ ($\lambda \text{raw.}$
return (*prefix-pack*, *raw*)))
by (rule *hash-range-budget-bind*)
(rule *hash-range-budget-receive-query-index-challenge*,
rule *hash-range-budget-return*)
then show *hash-range-budget* 1
(*receive-query-index-challenge* $\gg$ ($\lambda \text{raw.}$
return (*prefix-pack*, *raw*)))
by *simp*
qed
show *hash-range-budget*
(*staged-query-search-queries* budgets $i + 1$)
(*checked-staged-query-prefix-with-state* *A* *i* $\gg$
($\lambda \text{prefix-pack. receive-query-index-challenge } \gg$
($\lambda \text{raw. return (prefix-pack, raw)}$))))
by (rule *hash-range-budget-bind*)
(rule *prefix*, rule *receive*)
qed
lemma *hash-collision-budget-checked-staged-query-prefix-receive-with-state*:
assumes *wf*: staged-budget-wellformed budgets
and *controlled*: staged-adversary-controlled budgets *A*
and *i-bound*: $i \leq \text{rounds}$
shows
hash-collision-budget
(*staged-query-search-queries* budgets $i + 1$)
(*checked-staged-query-prefix-receive-with-state* *A* *i*)

unfolding *checked-staged-query-prefix-recv-with-state-def*

proof –

have *prefix-range*:

hash-range-budget
 (*staged-query-search-queries* *budgets* *i*)
 (*checked-staged-query-prefix-with-state* *A* *i*)

by (*rule* *hash-range-budget-checked-staged-query-prefix-with-state*
 [*OF wf controlled i-bound*])

have *prefix-collision*:

hash-collision-budget
 (*staged-query-search-queries* *budgets* *i*)
 (*checked-staged-query-prefix-with-state* *A* *i*)

by (*rule* *hash-collision-budget-checked-staged-query-prefix-with-state*
 [*OF wf controlled i-bound*])

have *receive-range*:

$\wedge$ *prefix-pack*. *hash-range-budget* 1
 (*receive-query-index-challenge* $\gg$ (λ *raw*.
 return (*prefix-pack*, *raw*)))

proof –

fix *prefix-pack*

have *hash-range-budget* (*1 + 0*)
 (*receive-query-index-challenge* $\gg$ (λ *raw*.
 return (*prefix-pack*, *raw*)))

by (*rule* *hash-range-budget-bind*)
 (*rule* *hash-range-budget-receive-query-index-challenge*,
 rule *hash-range-budget-return*)

then show *hash-range-budget* 1
 (*receive-query-index-challenge* $\gg$ (λ *raw*.
 return (*prefix-pack*, *raw*)))

by *simp*

qed

have *receive-collision*:

$\wedge$ *prefix-pack*. *hash-collision-budget* 1
 (*receive-query-index-challenge* $\gg$ (λ *raw*.
 return (*prefix-pack*, *raw*)))

proof –

fix *prefix-pack*

have *hash-collision-budget* (*1 + 0*)
 (*receive-query-index-challenge* $\gg$ (λ *raw*.
 return (*prefix-pack*, *raw*)))

by (*rule* *hash-collision-budget-bind*)
 (*rule* *hash-range-budget-receive-query-index-challenge*,
 rule *hash-collision-budget-receive-query-index-challenge*,
 rule *hash-range-budget-return*, *rule* *hash-collision-budget-return*)

then show *hash-collision-budget* 1
 (*receive-query-index-challenge* $\gg$ (λ *raw*.
 return (*prefix-pack*, *raw*)))

by *simp*

qed

```

show hash-collision-budget
  (staged-query-search-queries budgets  $i + 1$ )
  (checked-staged-query-prefix-with-state  $A\ i \gg=$ 
    ( $\lambda$ prefix-pack. receive-query-index-challenge  $\gg=$ 
      ( $\lambda$ raw. return (prefix-pack, raw))))
by (rule hash-collision-budget-bind)
  (rule prefix-range, rule prefix-collision, rule receive-range,
    rule receive-collision)
qed

lemma hash-map-output-collision-mono:
  assumes collision: hash-map-output-collision  $s$ 
    and ext:  $s \leq t$ 
  shows hash-map-output-collision  $t$ 
proof –
  obtain  $x\ y\ z$  where neq:  $x \neq y$ 
    and x-lookup: fmlookup (HashMap  $s$ )  $x = \text{Some } z$ 
    and y-lookup: fmlookup (HashMap  $s$ )  $y = \text{Some } z$ 
    using collision unfolding hash-map-output-collision-def by blast
  have x-lookup-t: fmlookup (HashMap  $t$ )  $x = \text{Some } z$ 
    by (rule hash-extension-lookup[OF x-lookup ext])
  have y-lookup-t: fmlookup (HashMap  $t$ )  $y = \text{Some } z$ 
    by (rule hash-extension-lookup[OF y-lookup ext])
  show ?thesis
    by (rule hash-map-output-collisionI[OF neq x-lookup-t y-lookup-t])
qed

lemma checked-staged-query-prefix-hash-map-output-collision-imp-new-collision-on-support:
  assumes support:
     $out \in \text{set-dist}$ 
    (execute (checked-staged-query-prefix-receive-with-state  $A\ i$ )
      adversary-initial-state)
  and collision:
    checked-staged-query-prefix-hash-map-output-collision  $out$ 
  shows hash-new-collision-event adversary-initial-state  $out$ 
proof (cases  $out$ )
  case None
  then show ?thesis
    using collision
    unfolding checked-staged-query-prefix-hash-map-output-collision-def
    by simp
next
  case (Some packed)
  then obtain prefix prefix-state raw raw-state where out-eq:
     $out = \text{Some } (((\text{prefix}, \text{prefix-state}), \text{raw}), \text{raw-state})$ 
  by (cases packed) auto
  have receive-support:
     $\text{Some } (((\text{prefix}, \text{prefix-state}), \text{raw}), \text{raw-state}) \in \text{set-dist}$ 

```

```

      (execute (checked-staged-query-prefix-receive-with-state A i)
        adversary-initial-state)
    using support unfolding out-eq .
  have recv:
    Some (raw, raw-state) ∈
      set-dist (execute receive-query-index-challenge prefix-state)
  by (rule
    checked-staged-query-prefix-receive-with-state-receive-support
    [OF receive-support])
  have prefix-collision: hash-map-output-collision prefix-state
    using collision unfolding out-eq
      checked-staged-query-prefix-hash-map-output-collision-def by simp
  have raw-ext: prefix-state ≤ raw-state
    by (rule receive-query-index-challenge-extends[OF recv])
  have raw-collision: hash-map-output-collision raw-state
    by (rule hash-map-output-collision-mono[OF prefix-collision raw-ext])
  show ?thesis
    unfolding out-eq hash-new-collision-event-def
      hash-map-new-output-collision-def
    using raw-collision by simp
qed

lemma checked-staged-query-prefix-hash-map-output-collision-bound:
  assumes wf: staged-budget-wellformed budgets
    and controlled: staged-adversary-controlled budgets A
    and i-bound: i ≤ rounds
  shows
    wp-event (checked-staged-query-prefix-receive-with-state A i)
      checked-staged-query-prefix-hash-map-output-collision
      adversary-initial-state ≤
      hash-collision-budget-value 0
      (staged-query-search-queries budgets i + 1)
proof -
  let ?M = checked-staged-query-prefix-receive-with-state A i
  let ?q = staged-query-search-queries budgets i + 1
  have event-le:
    wp-event ?M checked-staged-query-prefix-hash-map-output-collision
      adversary-initial-state ≤
    wp-event ?M (hash-new-collision-event adversary-initial-state)
      adversary-initial-state
  by (rule wp-event-mono-on-support)
  (rule
    checked-staged-query-prefix-hash-map-output-collision-imp-new-collision-on-support)
  have collision-budget:
    hash-collision-budget ?q ?M
  by (rule hash-collision-budget-checked-staged-query-prefix-receive-with-state
    [OF wf controlled i-bound])
  have new-collision-bound:
    wp-event ?M (hash-new-collision-event adversary-initial-state)

```

```

      adversary-initial-state ≤
      hash-collision-budget-value
      (card (hash-map-output-values adversary-initial-state)) ?q
using collision-budget adversary-initial-state-no-output-collision
unfolding hash-collision-budget-def by blast
have wp-event ?M checked-staged-query-prefix-hash-map-output-collision
      adversary-initial-state ≤
      hash-collision-budget-value
      (card (hash-map-output-values adversary-initial-state)) ?q
by (rule order-trans[OF event-le new-collision-bound])
then show ?thesis
by simp
qed

lemma checked-staged-query-prefix-partial-target-prehit-bound:
assumes wf: staged-budget-wellformed budgets
and controlled: staged-adversary-controlled budgets A
and i-bound: i ≤ rounds
shows
  wp-event (checked-staged-query-prefix-receive-with-state A i)
    (checked-staged-query-prefix-dynamic-index-prehit
      (staged-query-prefix-partial-query-target i))
    adversary-initial-state ≤
    staged-phase-relation-error size
    (staged-query-search-queries budgets i + 1)
proof (rule checked-staged-query-prefix-dynamic-index-prehit-bound-from-relation-program)
show HashMap adversary-initial-state = fmempty
by (simp add: adversary-initial-state-def)
show hash-relation-program
  (checked-staged-query-prefix-dynamic-index-prehit-relation A i
    adversary-initial-state
    (staged-query-prefix-partial-query-target i))
  size (staged-query-search-queries budgets i + 1)
  (checked-staged-query-prefix-receive-with-state A i)
by (rule hash-relation-program-checked-staged-query-prefix-receive-with-state
  [OF wf controlled i-bound])
  (rule
    checked-staged-query-prefix-dynamic-index-prehit-relation-fiber-bound-size)
qed

lemma checked-staged-query-prefix-committed-target-prehit-bound:
assumes wf: staged-budget-wellformed budgets
and controlled: staged-adversary-controlled budgets A
and i-bound: i ≤ rounds
shows
  wp-event (checked-staged-query-prefix-receive-with-state A i)
    (checked-staged-query-prefix-dynamic-index-prehit
      (λprefix prefix-state.
        staged-query-prefix-committed-query-target prefix prefix-state))

```

$\text{adversary-initial-state} \leq$
 $\text{staged-phase-relation-error size}$
 $(\text{staged-query-search-queries budgets } i + 1)$
proof (rule *checked-staged-query-prefix-dynamic-index-prehit-bound-from-relation-program*)
show $\text{HashMap adversary-initial-state} = \text{fmempty}$
by (simp add: *adversary-initial-state-def*)
show *hash-relation-program*
 $(\text{checked-staged-query-prefix-dynamic-index-prehit-relation } A \ i$
 $\text{adversary-initial-state}$
 $(\lambda \text{prefix prefix-state.}$
 $\text{staged-query-prefix-committed-query-target prefix prefix-state}))$
 $\text{size } (\text{staged-query-search-queries budgets } i + 1)$
 $(\text{checked-staged-query-prefix-receive-with-state } A \ i)$
by (rule *hash-relation-program-checked-staged-query-prefix-receive-with-state*
 $[\text{OF wf controlled } i\text{-bound}]$)
 $(\text{rule}$
 $\text{checked-staged-query-prefix-dynamic-index-prehit-relation-fiber-bound-size})$
qed

lemma *checked-staged-query-prefix-clean-committed-target-prehit-bound:*
assumes *wf: staged-budget-wellformed budgets*
and *controlled: staged-adversary-controlled budgets A*
and *i-bound: $i \leq \text{rounds}$*
shows
 $\text{wp-event } (\text{checked-staged-query-prefix-receive-with-state } A \ i)$
 $(\text{checked-staged-query-prefix-dynamic-index-prehit}$
 $\text{staged-query-prefix-clean-committed-query-target})$
 $\text{adversary-initial-state} \leq$
 $\text{staged-phase-relation-error size}$
 $(\text{staged-query-search-queries budgets } i + 1)$
proof (rule *checked-staged-query-prefix-dynamic-index-prehit-bound-from-relation-program*)
show $\text{HashMap adversary-initial-state} = \text{fmempty}$
by (simp add: *adversary-initial-state-def*)
show *hash-relation-program*
 $(\text{checked-staged-query-prefix-dynamic-index-prehit-relation } A \ i$
 $\text{adversary-initial-state}$
 $\text{staged-query-prefix-clean-committed-query-target})$
 $\text{size } (\text{staged-query-search-queries budgets } i + 1)$
 $(\text{checked-staged-query-prefix-receive-with-state } A \ i)$
by (rule *hash-relation-program-checked-staged-query-prefix-receive-with-state*
 $[\text{OF wf controlled } i\text{-bound}]$)
 $(\text{rule}$
 $\text{checked-staged-query-prefix-dynamic-index-prehit-relation-fiber-bound-size})$
qed

lemma *checked-staged-query-prefix-clean-length-committed-target-prehit-bound:*
assumes *wf: staged-budget-wellformed budgets*
and *controlled: staged-adversary-controlled budgets A*
and *i-bound: $i \leq \text{rounds}$*

shows

wp-event (*checked-staged-query-prefix-receive-with-state A i*)
 (*checked-staged-query-prefix-dynamic-index-prehit*
staged-query-prefix-clean-length-committed-query-target)
adversary-initial-state $\leq$
staged-phase-relation-error size
(staged-query-search-queries budgets i + 1)

proof (*rule checked-staged-query-prefix-dynamic-index-prehit-bound-from-relation-program*)

show *HashMap adversary-initial-state = fmempty*

by (*simp add: adversary-initial-state-def*)

show *hash-relation-program*

(checked-staged-query-prefix-dynamic-index-prehit-relation A i
adversary-initial-state
staged-query-prefix-clean-length-committed-query-target)
size (staged-query-search-queries budgets i + 1)

(checked-staged-query-prefix-receive-with-state A i)

by (*rule hash-relation-program-checked-staged-query-prefix-receive-with-state*
[OF wf controlled i-bound])

(rule
checked-staged-query-prefix-dynamic-index-prehit-relation-fiber-bound-size)

qed

definition *checked-staged-query-prefix-dynamic-index-fresh-output-hit*

$:: ('f \text{ staged-query-prefix-data} \Rightarrow 'f \text{ protocol-channel} \Rightarrow$
 $\text{nat set}) \Rightarrow$
 $((('f \text{ staged-query-prefix-data} \times 'f \text{ protocol-channel}) \times 'f) \times$
 $'f \text{ protocol-channel}) \text{ option} \Rightarrow \text{bool}$

where

checked-staged-query-prefix-dynamic-index-fresh-output-hit B out $\longleftrightarrow$
(case out of
None $\Rightarrow$ *False*
 $| \text{Some } (((\text{prefix}, \text{prefix-state}), \text{raw}), \text{raw-state}) \Rightarrow$
hash-new-output-hit-event
(query-index-raw-preimage (B prefix prefix-state))
prefix-state (Some (raw, raw-state)))

lemma *checked-staged-query-prefix-dynamic-index-hit-imp-prehit-or-fresh:*

assumes *absent:*

$\neg$ *checked-staged-query-prefix-dynamic-index-prehit B*
(Some (((prefix, prefix-state), raw), raw-state))

and *prefix-support:*

Some ((prefix, prefix-state), prefix-state) $\in$
set-dist
(execute (checked-staged-query-prefix-with-state A i) s)

and *receive-support:*

Some (((prefix, prefix-state), raw), raw-state) $\in$
set-dist
(execute (checked-staged-query-prefix-receive-with-state A i) s)

and *hit:*


```

    checked-staged-query-prefix-dynamic-index-hit B
      (Some (((prefix, prefix-state), raw), raw-state))
  shows
    hash-new-output-hit-event
      (query-index-raw-preimage (B prefix prefix-state)) prefix-state
      (Some (raw, raw-state))
  proof -
    have raw-in:
      raw ∈ query-index-raw-preimage (B prefix prefix-state)
    using hit unfolding checked-staged-query-prefix-dynamic-index-hit-def
      query-index-raw-preimage-def
    by simp
  have no-lookup:
    fmllookup (HashMap prefix-state)
      (QueryIndexChallenge (PQueryCounter prefix-state)
        (PState prefix-state)) = None
  proof (cases
    fmllookup (HashMap prefix-state)
      (QueryIndexChallenge (PQueryCounter prefix-state)
        (PState prefix-state)))
    case None
    then show ?thesis .
  next
    case (Some old-raw)
    have old-lookup-prefix:
      fmllookup (HashMap prefix-state)
        (QueryIndexChallenge (PQueryCounter prefix-state)
          (PState prefix-state)) = Some old-raw
    using Some by simp
    have old-in: old-raw ∈ query-index-raw-preimage
      (B prefix prefix-state)
    proof -
      have old-idx:
        index (to-nat old-raw) ∈ B prefix prefix-state
    proof -
      have raw-idx:
        index (to-nat raw) ∈ B prefix prefix-state
      using hit
      unfolding checked-staged-query-prefix-dynamic-index-hit-def
      by simp
    have recv:
      Some (raw, raw-state) ∈
        set-dist
          (execute receive-query-index-challenge prefix-state)
      by (rule
        checked-staged-query-prefix-receive-with-state-receive-support
          [OF receive-support])
    have recv-lookup:
      fmllookup (HashMap raw-state)

```

```

    (QueryIndexChallenge (PQueryCounter prefix-state)
      (PState prefix-state)) = Some raw
  using receive-query-index-challenge-outcome[OF recv] by blast
have ext: prefix-state ≤ raw-state
  using receive-query-index-challenge-extends[OF recv] .
have old-lookup-raw-state:
  fmlookup (HashMap raw-state)
    (QueryIndexChallenge (PQueryCounter prefix-state)
      (PState prefix-state)) = Some old-raw
  by (rule hash-extension-lookup[OF old-lookup-prefix ext])
have old-raw-eq: old-raw = raw
  using recv-lookup old-lookup-raw-state by simp
show ?thesis
  using raw-idx unfolding old-raw-eq .
qed
show ?thesis
  using old-idx unfolding query-index-raw-preimage-def by simp
qed
have prehit:
  checked-staged-query-prefix-dynamic-index-prehit B
    (Some (((prefix, prefix-state), raw), raw-state))
  using old-lookup-prefix old-in
  by (auto simp: checked-staged-query-prefix-dynamic-index-prehit-def)
then show ?thesis
  using absent by contradiction
qed
from receive-support have recv:
  Some (raw, raw-state) ∈
    set-dist (execute receive-query-index-challenge prefix-state)
  by (rule
    checked-staged-query-prefix-receive-with-state-receive-support)
have recv-lookup:
  fmlookup (HashMap raw-state)
    (QueryIndexChallenge (PQueryCounter prefix-state)
      (PState prefix-state)) = Some raw
  using receive-query-index-challenge-outcome[OF recv] by blast
have new-hit:
  hash-map-new-output-hit
    (query-index-raw-preimage (B prefix prefix-state))
    prefix-state raw-state
  unfolding hash-map-new-output-hit-def
  using no-lookup recv-lookup raw-in by blast
show ?thesis
  unfolding hash-new-output-hit-event-def using new-hit by simp
qed

lemma checked-staged-query-prefix-dynamic-index-hit-bound-by-prehit-and-fresh:
  wp-event (checked-staged-query-prefix-receive-with-state A i)
    (checked-staged-query-prefix-dynamic-index-hit B) s ≤

```

```

wp-event (checked-staged-query-prefix-receive-with-state A i)
  (checked-staged-query-prefix-dynamic-index-prehit B) s +
wp-event (checked-staged-query-prefix-receive-with-state A i)
  (checked-staged-query-prefix-dynamic-index-fresh-output-hit B) s
proof -
  let ?M = checked-staged-query-prefix-receive-with-state A i
  let ?hit = checked-staged-query-prefix-dynamic-index-hit B
  let ?pre = checked-staged-query-prefix-dynamic-index-prehit B
  let ?fresh = checked-staged-query-prefix-dynamic-index-fresh-output-hit B
  have wp-event ?M ?hit s ≤ wp-event ?M (λout. ?pre out ∨ ?fresh out) s
  proof (rule wp-event-mono-on-support)
    fix out
    assume support: out ∈ set-dist (execute ?M s)
    and hit: ?hit out
    show ?pre out ∨ ?fresh out
    proof (cases out)
      case None
      then show ?thesis
        using hit unfolding checked-staged-query-prefix-dynamic-index-hit-def
        by simp
    next
      case (Some packed)
      then obtain prefix prefix-state raw raw-state where out-eq:
        out = Some (((prefix, prefix-state), raw), raw-state)
        by (cases packed, auto split: prod.splits)
      show ?thesis
      proof (cases ?pre out)
        case True
        then show ?thesis by simp
      next
        case pre-absent: False
        have prefix-support:
          Some ((prefix, prefix-state), prefix-state) ∈
            set-dist
              (execute (checked-staged-query-prefix-with-state A i) s)
        by (rule
          checked-staged-query-prefix-receive-with-state-prefix-support
            [OF support[unfolded out-eq]])
        have fresh:
          hash-new-output-hit-event
            (query-index-raw-preimage (B prefix prefix-state))
            prefix-state (Some (raw, raw-state))
        by (rule checked-staged-query-prefix-dynamic-index-hit-imp-prehit-or-fresh
          [OF - prefix-support support[unfolded out-eq]
            hit[unfolded out-eq]])
        (use pre-absent out-eq in simp)
      then show ?thesis
        unfolding out-eq
        checked-staged-query-prefix-dynamic-index-fresh-output-hit-def

```

```

      by simp
    qed
  qed
  qed
  also have ... ≤ wp-event ?M ?pre s + wp-event ?M ?fresh s
    by (rule wp-event-union-bound)
  finally show ?thesis .
qed

lemma wp-receive-query-index-challenge-new-output-hit-bound:
  fixes B :: 'f set
  and s :: ('f, 'a) protocol-channel-scheme
  shows
    wp-event receive-query-index-challenge
      (hash-new-output-hit-event B s) s ≤ nnreal (card B) / nnreal size
  proof (cases
    fmlookup (HashMap s)
      (QueryIndexChallenge (PQueryCounter s) (PState s)))
  case None
  have event-le:
    wp-event receive-query-index-challenge
      (hash-new-output-hit-event B s) s ≤
    wp-event receive-query-index-challenge
      (λout. case out of None ⇒ False | Some (raw, -) ⇒ raw ∈ B) s
  proof (rule wp-event-mono-on-support)
    fix out
    assume support: out ∈ set-dist (execute receive-query-index-challenge s)
    and hit: hash-new-output-hit-event B s out
    show (case out of None ⇒ False | Some (raw, -) ⇒ raw ∈ B)
  proof (cases out)
    case None
    then show ?thesis
      using hit unfolding hash-new-output-hit-event-def by simp
  next
    case (Some packed)
    then obtain raw t where packed-eq: packed = (raw, t)
      by (cases packed) simp
    have out':
      Some (raw, t) ∈
      set-dist (execute receive-query-index-challenge s)
      using support unfolding Some packed-eq .
    have lookup:
      fmlookup (HashMap t)
      (QueryIndexChallenge (PQueryCounter s) (PState s)) = Some raw
      using receive-query-index-challenge-outcome[OF out'] by blast
    have new-hit: hash-map-new-output-hit B s t
      using hit unfolding Some packed-eq hash-new-output-hit-event-def
      by simp
    then show ?thesis

```

```

proof –
  from new-hit obtain  $x\ y$  where none-x:
     $\text{fmlookup } (\text{HashMap } s) \ x = \text{None}$ 
    and some-x:  $\text{fmlookup } (\text{HashMap } t) \ x = \text{Some } y$ 
    and y-in:  $y \in B$ 
    unfolding hash-map-new-output-hit-def by blast
  let  $?q = \text{QueryIndexChallenge } (P\text{QueryCounter } s) \ (P\text{State } s)$ 
  have  $\text{raw} \in B$ 
  proof (cases  $x = ?q$ )
    case True
    then have  $\text{Some } y = \text{Some } \text{raw}$ 
    using some-x lookup by simp
    then show ?thesis
    using y-in by simp
  next
    case False
    have  $\text{fmlookup } (\text{HashMap } t) \ x = \text{fmlookup } (\text{HashMap } s) \ x$ 
    by (rule receive-query-index-challenge-preserves-other-lookup
      [OF out' False])
    then show ?thesis
    using none-x some-x by simp
  qed
  then show ?thesis
  unfolding Some packed-eq by simp
qed
qed
qed
also have  $\dots = \text{nnreal } (\text{card } B) / \text{nnreal } \text{size}$ 
by (rule wp-receive-query-index-challenge-fresh-set [OF None])
finally show ?thesis .
next
  case (Some old)
  have old-lookup:
     $\text{fmlookup } (\text{HashMap } s)$ 
     $(\text{QueryIndexChallenge } (P\text{QueryCounter } s) \ (P\text{State } s)) = \text{Some } \text{old}$ 
    using Some by simp
  have no-new-on-support:
     $\bigwedge \text{out}. \text{out} \in \text{set-dist } (\text{execute receive-query-index-challenge } s) \implies$ 
     $\neg \text{hash-new-output-hit-event } B \ s \ \text{out}$ 
  proof –
    fix out
    assume support:  $\text{out} \in \text{set-dist } (\text{execute receive-query-index-challenge } s)$ 
    show  $\neg \text{hash-new-output-hit-event } B \ s \ \text{out}$ 
    proof (cases out)
      case None
      then show ?thesis
      unfolding hash-new-output-hit-event-def by simp
    next
      case (Some packed)

```

```

then obtain raw t where packed-eq: packed = (raw, t)
  by (cases packed) simp
have out':
  Some (raw, t) ∈
    set-dist (execute receive-query-index-challenge s)
  using support unfolding Some packed-eq .
have ext: s ≤ t
  by (rule receive-query-index-challenge-extends[OF out'])
show ?thesis
proof
  assume bad: hash-new-output-hit-event B s out
  have bad':
     $\exists x y. \text{fmlookup } (\text{HashMap } s) \ x = \text{None} \wedge$ 
     $\text{fmlookup } (\text{HashMap } t) \ x = \text{Some } y \wedge y \in B$ 
  using bad
  unfolding Some packed-eq hash-new-output-hit-event-def
    hash-map-new-output-hit-def
  by simp
then obtain x y where none-x:
  fmlookup (HashMap s) x = None
  and some-x: fmlookup (HashMap t) x = Some y
  and y-in: y ∈ B
  by blast
let ?q = QueryIndexChallenge (PQueryCounter s) (PState s)
show False
proof (cases x = ?q)
  case True
    then show ?thesis
    using none-x old-lookup by simp
  next
    case False
    have fmlookup (HashMap t) x = fmlookup (HashMap s) x
    by (rule receive-query-index-challenge-preserves-other-lookup
      [OF out' False])
    then show ?thesis
    using none-x some-x by simp
  qed
qed
qed
qed
have wp-event receive-query-index-challenge
  (hash-new-output-hit-event B s) s ≤
  wp-event receive-query-index-challenge (λ-. False) s
by (rule wp-event-mono-on-support)
  (use no-new-on-support in blast)
also have ... = 0
  unfolding wp-event-def wp-def dist-expect-def by simp
also have 0 ≤ nnreal (card B) / nnreal size
by simp

```

finally show ?thesis .
qed

lemma *checked-staged-query-prefix-dynamic-index-fresh-output-hit-bound*:
fixes $C :: \text{prob}$
assumes *raw-bound*: *query-index-raw-preimage-bound*
and *prefix-bound*:
 $\bigwedge \text{prefix prefix-state.}$
 $\text{Some } ((\text{prefix}, \text{prefix-state}), \text{prefix-state}) \in$
 set-dist
 $(\text{execute } (\text{checked-staged-query-prefix-with-state } A \ i) \ s) \implies$
 $B \ \text{prefix} \ \text{prefix-state} \subseteq \text{query-sample-space} \wedge$
 $\text{nnreal } (\text{card } (B \ \text{prefix} \ \text{prefix-state})) /$
 $\text{nnreal } (\text{card } \text{query-sample-space}) \leq C$
shows
 $\text{wp-event } (\text{checked-staged-query-prefix-receive-with-state } A \ i)$
 $(\text{checked-staged-query-prefix-dynamic-index-fresh-output-hit } B) \ s \leq C$
unfolding *checked-staged-query-prefix-receive-with-state-def*
proof (rule *wp-event-bind-bound-by-cont*)
show
 $\neg \text{checked-staged-query-prefix-dynamic-index-fresh-output-hit } B \ \text{None}$
unfolding *checked-staged-query-prefix-dynamic-index-fresh-output-hit-def*
by *simp*
next
fix *prefix-pack prefix-state*
assume *prefix-support*:
 $\text{Some } (\text{prefix-pack}, \text{prefix-state}) \in$
 set-dist
 $(\text{execute } (\text{checked-staged-query-prefix-with-state } A \ i) \ s)$
obtain *prefix captured-state* **where** *prefix-pack-eq*:
 $\text{prefix-pack} = (\text{prefix}, \text{captured-state})$
by (cases *prefix-pack*) *simp*
have *captured-eq*: *captured-state* = *prefix-state*
using *checked-staged-query-prefix-with-state-outcome-state*
 $[OF \ \text{prefix-support}[\text{unfolded } \text{prefix-pack-eq}]]$
by *simp*
have *support'*:
 $\text{Some } ((\text{prefix}, \text{prefix-state}), \text{prefix-state}) \in$
 set-dist
 $(\text{execute } (\text{checked-staged-query-prefix-with-state } A \ i) \ s)$
using *checked-staged-query-prefix-with-state-outcome-canonical*
 $[OF \ \text{prefix-support}[\text{unfolded } \text{prefix-pack-eq}]]$
 captured-eq
by *simp*
from *prefix-bound*[*OF support'*] **have** *subset*:
 $B \ \text{prefix} \ \text{prefix-state} \subseteq \text{query-sample-space}$
and *frac*:
 $\text{nnreal } (\text{card } (B \ \text{prefix} \ \text{prefix-state})) /$
 $\text{nnreal } (\text{card } \text{query-sample-space}) \leq C$

```

    by blast+
  have raw-frac:
    nnreal (card (query-index-raw-preimage (B prefix prefix-state))) /
      nnreal size ≤ C
  proof -
    have nnreal (card (query-index-raw-preimage (B prefix prefix-state))) /
      nnreal size ≤
      nnreal (card (B prefix prefix-state)) /
      nnreal (card query-sample-space)
    using raw-bound subset unfolding query-index-raw-preimage-bound-def
    by blast
    also have ... ≤ C
    by (rule frac)
    finally show ?thesis .
  qed
  have cont-eq:
    wp-event
      (receive-query-index-challenge ≫= (λraw.
        return (prefix-pack, raw)))
      (checked-staged-query-prefix-dynamic-index-fresh-output-hit B)
      prefix-state =
    wp-event receive-query-index-challenge
      (hash-new-output-hit-event
        (query-index-raw-preimage (B prefix prefix-state)) prefix-state)
      prefix-state
    unfolding wp-event-def
    apply (subst wp-bind)
    apply (rule arg-cong[where
      f=λQ. wp receive-query-index-challenge Q prefix-state])
    apply (rule ext)
    apply (simp add: wp-return prefix-pack-eq captured-eq
      checked-staged-query-prefix-dynamic-index-fresh-output-hit-def
      hash-new-output-hit-event-def
      split: option.splits prod.splits)
    done
  show wp-event
    (receive-query-index-challenge ≫= (λraw.
      return (prefix-pack, raw)))
    (checked-staged-query-prefix-dynamic-index-fresh-output-hit B)
    prefix-state ≤ C
    unfolding cont-eq
    by (rule order-trans[
      OF wp-receive-query-index-challenge-new-output-hit-bound raw-frac])
  qed

lemma checked-staged-query-prefix-dynamic-index-hit-bound-by-prehit-and-query-error:
  fixes C :: prob
  assumes raw-bound: query-index-raw-preimage-bound
  and prefix-bound:

```


$\wedge \text{prefix prefix-state.}$
 $\text{Some } ((\text{prefix}, \text{prefix-state}), \text{prefix-state}) \in$
 set-dist
 $(\text{execute } (\text{checked-staged-query-prefix-with-state } A \ i) \ s) \implies$
 $B \text{ prefix prefix-state} \subseteq \text{query-sample-space} \wedge$
 $\text{nnreal } (\text{card } (B \text{ prefix prefix-state})) /$
 $\text{nnreal } (\text{card } \text{query-sample-space}) \leq C$
shows
 $\text{wp-event } (\text{checked-staged-query-prefix-receive-with-state } A \ i)$
 $(\text{checked-staged-query-prefix-dynamic-index-hit } B) \ s \leq$
 $\text{wp-event } (\text{checked-staged-query-prefix-receive-with-state } A \ i)$
 $(\text{checked-staged-query-prefix-dynamic-index-prehit } B) \ s + C$
proof –
have $\text{wp-event } (\text{checked-staged-query-prefix-receive-with-state } A \ i)$
 $(\text{checked-staged-query-prefix-dynamic-index-hit } B) \ s \leq$
 $\text{wp-event } (\text{checked-staged-query-prefix-receive-with-state } A \ i)$
 $(\text{checked-staged-query-prefix-dynamic-index-prehit } B) \ s +$
 $\text{wp-event } (\text{checked-staged-query-prefix-receive-with-state } A \ i)$
 $(\text{checked-staged-query-prefix-dynamic-index-fresh-output-hit } B) \ s$
by (rule checked-staged-query-prefix-dynamic-index-hit-bound-by-prehit-and-fresh)
also have $\dots \leq$
 $\text{wp-event } (\text{checked-staged-query-prefix-receive-with-state } A \ i)$
 $(\text{checked-staged-query-prefix-dynamic-index-prehit } B) \ s + C$
by (intro add-mono order-refl
checked-staged-query-prefix-dynamic-index-fresh-output-hit-bound
[OF raw-bound prefix-bound])
finally show ?thesis .
qed

lemma query-sample-space-fraction-le-one:
assumes subset: $B \subseteq \text{query-sample-space}$
shows $\text{nnreal } (\text{card } B) / \text{nnreal } (\text{card } \text{query-sample-space}) \leq (1::\text{prob})$
proof –
have finite-B: finite B
by (rule finite-subset[OF subset finite-query-sample-space])
have card-le: $\text{card } B \leq \text{card } \text{query-sample-space}$
by (rule card-mono[OF finite-query-sample-space subset])
have $\text{nnreal } (\text{card } B) / \text{nnreal } (\text{card } \text{query-sample-space}) \leq$
 $\text{nnreal } (\text{card } \text{query-sample-space}) / \text{nnreal } (\text{card } \text{query-sample-space})$
by (rule nnreal-nat-divide-right-mono[OF card-le])
also have $\dots = (1::\text{prob})$
by (rule nnreal-nat-divide-self) (simp add: query-sample-space-size-pos)
finally show ?thesis .
qed

lemma checked-staged-query-prefix-partial-target-hit-bound-by-prehit-plus-one:
assumes raw-bound: query-index-raw-preimage-bound
shows
 $\text{wp-event } (\text{checked-staged-query-prefix-receive-with-state } A \ i)$

```

    (checked-staged-query-prefix-dynamic-index-hit
      (staged-query-prefix-partial-query-target i)) s ≤
wp-event (checked-staged-query-prefix-receive-with-state A i)
    (checked-staged-query-prefix-dynamic-index-prehit
      (staged-query-prefix-partial-query-target i)) s + 1
proof (rule
  checked-staged-query-prefix-dynamic-index-hit-bound-by-prehit-and-query-error
  [OF raw-bound])
fix prefix prefix-state
assume support:
  Some ((prefix, prefix-state), prefix-state) ∈
    set-dist (execute (checked-staged-query-prefix-with-state A i) s)
have subset:
  staged-query-prefix-partial-query-target i prefix prefix-state ⊆
    query-sample-space
by (rule staged-query-prefix-partial-query-target-subset)
have frac:
  nnreal
    (card (staged-query-prefix-partial-query-target i prefix prefix-state)) /
    nnreal (card query-sample-space) ≤ (1::prob)
by (rule query-sample-space-fraction-le-one[OF subset])
show staged-query-prefix-partial-query-target i prefix prefix-state ⊆
  query-sample-space ∧
  nnreal
    (card (staged-query-prefix-partial-query-target i prefix prefix-state)) /
    nnreal (card query-sample-space) ≤ (1::prob)
using subset frac by blast
qed

```

lemma checked-staged-query-prefix-partial-target-hit-bound-by-relation-plus-one:

assumes raw-bound: query-index-raw-preimage-bound
and wf: staged-budget-wellformed budgets
and controlled: staged-adversary-controlled budgets A
and i-bound: $i \leq \text{rounds}$

shows

wp-event (checked-staged-query-prefix-receive-with-state A i)
 (checked-staged-query-prefix-dynamic-index-hit
 (staged-query-prefix-partial-query-target i))
 adversary-initial-state ≤
 staged-phase-relation-error size
 (staged-query-search-queries budgets i + 1) + 1

proof –

have wp-event (checked-staged-query-prefix-receive-with-state A i)
 (checked-staged-query-prefix-dynamic-index-hit
 (staged-query-prefix-partial-query-target i))
 adversary-initial-state ≤
 wp-event (checked-staged-query-prefix-receive-with-state A i)
 (checked-staged-query-prefix-dynamic-index-prehit
 (staged-query-prefix-partial-query-target i))

adversary-initial-state + 1
by (rule *checked-staged-query-prefix-partial-target-hit-bound-by-prehit-plus-one*
[OF raw-bound])
also have ... $\leq$
staged-phase-relation-error size
(staged-query-search-queries budgets i + 1) + 1
by (intro *add-mono order-refl*
checked-staged-query-prefix-partial-target-prehit-bound
[OF wf controlled i-bound])
finally show ?thesis .
qed

lemma *checked-staged-query-prefix-committed-target-hit-bound-by-relation-and-query-error:*

assumes *raw-bound: query-index-raw-preimage-bound*

and *wf: staged-budget-wellformed budgets*

and *controlled: staged-adversary-controlled budgets A*

and *i-bound: i ≤ rounds*

and *prefix-clean:*

$\bigwedge \text{prefix prefix-state.}$

Some ((prefix, prefix-state), prefix-state) ∈

set-dist

(execute (checked-staged-query-prefix-with-state A i)

adversary-initial-state) ⇒

$\neg$ *merkle-root-binding-collision (sqp-trace-root prefix)*

prefix-state ∧

$(\forall \text{rt. sqp-composition-fri-roots prefix} \neq [] \longrightarrow$

rt = hd (sqp-composition-fri-roots prefix) ⇒

$\neg$ *merkle-root-binding-collision rt prefix-state)*

shows

wp-event (checked-staged-query-prefix-receive-with-state A i)

(checked-staged-query-prefix-dynamic-index-hit

(λprefix prefix-state.

staged-query-prefix-committed-query-target prefix prefix-state))

adversary-initial-state ≤

staged-phase-relation-error size

(staged-query-search-queries budgets i + 1) + query-error-bound

proof —

have *hit-bound:*

wp-event (checked-staged-query-prefix-receive-with-state A i)

(checked-staged-query-prefix-dynamic-index-hit

(λprefix prefix-state.

staged-query-prefix-committed-query-target prefix prefix-state))

adversary-initial-state ≤

wp-event (checked-staged-query-prefix-receive-with-state A i)

(checked-staged-query-prefix-dynamic-index-prehit

(λprefix prefix-state.

staged-query-prefix-committed-query-target prefix prefix-state))

adversary-initial-state + query-error-bound

proof (rule *checked-staged-query-prefix-dynamic-index-hit-bound-by-prehit-and-query-error*

$[OF \text{ raw-bound}]$
fix $prefix \text{ prefix-state}$
assume $support$:
 $Some ((prefix, prefix-state), prefix-state) \in$
 $set-dist$
 $(execute (checked-staged-query-prefix-with-state A i)$
 $adversary-initial-state)$
from $prefix-clean[OF support]$
have $trace-clean$:
 $\neg merkle-root-binding-collision (sqp-trace-root prefix)$
 $prefix-state$
and $comp-clean$:
 $\bigwedge rt. sqp-composition-fri-roots prefix \neq [] \implies$
 $rt = hd (sqp-composition-fri-roots prefix) \implies$
 $\neg merkle-root-binding-collision rt prefix-state$
by $blast+$
show
 $(\lambda prefix \text{ prefix-state}.$
 $staged-query-prefix-committed-query-target prefix prefix-state)$
 $prefix \text{ prefix-state} \subseteq query-sample-space \wedge$
 $nnreal$
 $(card$
 $((\lambda prefix \text{ prefix-state}.$
 $staged-query-prefix-committed-query-target prefix prefix-state)$
 $prefix \text{ prefix-state})) /$
 $nnreal (card query-sample-space) \leq query-error-bound$
using
 $staged-query-prefix-committed-query-target-subset[of prefix \text{ prefix-state}]$
 $staged-query-prefix-committed-query-target-fraction-bound-if-no-root-collisions$
 $[OF trace-clean comp-clean]$
by $blast$
qed
also have $\dots \leq$
 $staged-phase-relation-error \text{ size}$
 $(staged-query-search-queries \text{ budgets } i + 1) + query-error-bound$
by $(intro add-mono order-refl$
 $checked-staged-query-prefix-committed-target-prehit-bound$
 $[OF wf \text{ controlled } i-bound])$
finally show $?thesis$.
qed

lemma $checked-staged-query-prefix-clean-committed-target-hit-bound$:
assumes $raw-bound$: $query-index-raw-preimage-bound$
and wf : $staged-budget-wellformed \text{ budgets}$
and $controlled$: $staged-adversary-controlled \text{ budgets } A$
and $i-bound$: $i \leq rounds$
shows
 $wp-event (checked-staged-query-prefix-receive-with-state A i)$
 $(checked-staged-query-prefix-dynamic-index-hit$

$\text{staged-query-prefix-clean-committed-query-target})$
 $\text{adversary-initial-state} \leq$
 $\text{staged-phase-relation-error size}$
 $(\text{staged-query-search-queries budgets } i + 1) + \text{query-error-bound}$

proof –

have *hit-bound*:

$\text{wp-event } (\text{checked-staged-query-prefix-receive-with-state } A \ i)$
 $(\text{checked-staged-query-prefix-dynamic-index-hit}$
 $\text{staged-query-prefix-clean-committed-query-target})$
 $\text{adversary-initial-state} \leq$
 $\text{wp-event } (\text{checked-staged-query-prefix-receive-with-state } A \ i)$
 $(\text{checked-staged-query-prefix-dynamic-index-prehit}$
 $\text{staged-query-prefix-clean-committed-query-target})$
 $\text{adversary-initial-state} + \text{query-error-bound}$

proof (rule
 $\text{checked-staged-query-prefix-dynamic-index-hit-bound-by-prehit-and-query-error}$
 $[\text{OF raw-bound}]$)

fix *prefix prefix-state*

assume *support*:

$\text{Some } ((\text{prefix}, \text{prefix-state}), \text{prefix-state}) \in$
 set-dist
 $(\text{execute } (\text{checked-staged-query-prefix-with-state } A \ i)$
 $\text{adversary-initial-state})$

have *subset*:

$\text{staged-query-prefix-clean-committed-query-target prefix prefix-state}$
 $\subseteq \text{query-sample-space}$

by (rule $\text{staged-query-prefix-clean-committed-query-target-subset}$)

have *frac*:

nnreal
 $(\text{card}$
 $(\text{staged-query-prefix-clean-committed-query-target prefix}$
 $\text{prefix-state})) /$
 $\text{nnreal } (\text{card query-sample-space}) \leq \text{query-error-bound}$

by (rule $\text{staged-query-prefix-clean-committed-query-target-fraction-bound}$)

show $\text{staged-query-prefix-clean-committed-query-target prefix prefix-state}$
 $\subseteq \text{query-sample-space} \wedge$

nnreal
 $(\text{card}$
 $(\text{staged-query-prefix-clean-committed-query-target prefix}$
 $\text{prefix-state})) /$
 $\text{nnreal } (\text{card query-sample-space}) \leq \text{query-error-bound}$

using *subset frac* **by** *blast*

qed

also have $\dots \leq$
 $\text{staged-phase-relation-error size}$
 $(\text{staged-query-search-queries budgets } i + 1) + \text{query-error-bound}$

by (intro *add-mono order-refl*
 $\text{checked-staged-query-prefix-clean-committed-target-prehit-bound}$
 $[\text{OF wf controlled } i\text{-bound}]$)

finally show ?thesis .
qed

lemma *checked-staged-query-prefix-committed-target-hit-bound-by-clean-and-collision:*

wp-event (checked-staged-query-prefix-receive-with-state A i)
(checked-staged-query-prefix-dynamic-index-hit
staged-query-prefix-committed-query-target) s ≤
wp-event (checked-staged-query-prefix-receive-with-state A i)
(checked-staged-query-prefix-dynamic-index-hit
staged-query-prefix-clean-committed-query-target) s +
wp-event (checked-staged-query-prefix-receive-with-state A i)
checked-staged-query-prefix-committed-root-collision s

proof —

let ?M = *checked-staged-query-prefix-receive-with-state A i*
let ?hit =
checked-staged-query-prefix-dynamic-index-hit
staged-query-prefix-committed-query-target
let ?clean =
checked-staged-query-prefix-dynamic-index-hit
staged-query-prefix-clean-committed-query-target
let ?collision = *checked-staged-query-prefix-committed-root-collision*
have *wp-event ?M ?hit s ≤ wp-event ?M (λout. ?clean out ∨ ?collision out) s*
by (rule *wp-event-mono*)
(rule *checked-staged-query-prefix-committed-hit-imp-clean-hit-or-root-collision*)
also have ... ≤ *wp-event ?M ?clean s + wp-event ?M ?collision s*
by (rule *wp-event-union-bound*)
finally show ?thesis .

qed

lemma *checked-staged-query-prefix-committed-target-hit-bound-by-relation-query-error-and-collision:*

assumes *raw-bound: query-index-raw-preimage-bound*
and *wf: staged-budget-wellformed budgets*
and *controlled: staged-adversary-controlled budgets A*
and *i-bound: i ≤ rounds*

shows

wp-event (checked-staged-query-prefix-receive-with-state A i)
(checked-staged-query-prefix-dynamic-index-hit
staged-query-prefix-committed-query-target)
adversary-initial-state ≤
staged-phase-relation-error size
(staged-query-search-queries budgets i + 1) +
query-error-bound +
wp-event (checked-staged-query-prefix-receive-with-state A i)
checked-staged-query-prefix-committed-root-collision
adversary-initial-state

proof —

have *split:*
wp-event (checked-staged-query-prefix-receive-with-state A i)
(checked-staged-query-prefix-dynamic-index-hit

```

    staged-query-prefix-committed-query-target)
  adversary-initial-state ≤
wp-event (checked-staged-query-prefix-receive-with-state A i)
  (checked-staged-query-prefix-dynamic-index-hit
    staged-query-prefix-clean-committed-query-target)
  adversary-initial-state +
wp-event (checked-staged-query-prefix-receive-with-state A i)
  checked-staged-query-prefix-committed-root-collision
  adversary-initial-state
by (rule
  checked-staged-query-prefix-committed-target-hit-bound-by-clean-and-collision)
have clean-bound:
wp-event (checked-staged-query-prefix-receive-with-state A i)
  (checked-staged-query-prefix-dynamic-index-hit
    staged-query-prefix-clean-committed-query-target)
  adversary-initial-state ≤
  staged-phase-relation-error size
  (staged-query-search-queries budgets i + 1) + query-error-bound
by (rule checked-staged-query-prefix-clean-committed-target-hit-bound
  [OF raw-bound wf controlled i-bound])
have tail-bound:
wp-event (checked-staged-query-prefix-receive-with-state A i)
  (checked-staged-query-prefix-dynamic-index-hit
    staged-query-prefix-clean-committed-query-target)
  adversary-initial-state +
wp-event (checked-staged-query-prefix-receive-with-state A i)
  checked-staged-query-prefix-committed-root-collision
  adversary-initial-state ≤
  staged-phase-relation-error size
  (staged-query-search-queries budgets i + 1) +
  query-error-bound +
wp-event (checked-staged-query-prefix-receive-with-state A i)
  checked-staged-query-prefix-committed-root-collision
  adversary-initial-state
using clean-bound by (simp add: add-mono)
show ?thesis
by (rule order-trans[OF split tail-bound])
qed

```

lemma *checked-staged-query-prefix-clean-length-committed-target-hit-bound:*

assumes raw-bound: query-index-raw-preimage-bound
and wf: staged-budget-wellformed budgets
and controlled: staged-adversary-controlled budgets A
and i-bound: $i \leq \text{rounds}$

shows

```

wp-event (checked-staged-query-prefix-receive-with-state A i)
  (checked-staged-query-prefix-dynamic-index-hit
    staged-query-prefix-clean-length-committed-query-target)
  adversary-initial-state ≤

```

staged-phase-relation-error size
(staged-query-search-queries budgets $i + 1$) + query-error-bound
proof –
have *hit-bound*:
wp-event (checked-staged-query-prefix-receive-with-state A i)
(checked-staged-query-prefix-dynamic-index-hit
staged-query-prefix-clean-length-committed-query-target)
adversary-initial-state ≤
wp-event (checked-staged-query-prefix-receive-with-state A i)
(checked-staged-query-prefix-dynamic-index-prehit
staged-query-prefix-clean-length-committed-query-target)
adversary-initial-state + query-error-bound
proof (*rule*
checked-staged-query-prefix-dynamic-index-hit-bound-by-prehit-and-query-error
[OF raw-bound])
fix *prefix prefix-state*
assume *support*:
Some ((prefix, prefix-state), prefix-state) ∈
set-dist
(execute (checked-staged-query-prefix-with-state A i)
adversary-initial-state)
have *subset*:
staged-query-prefix-clean-length-committed-query-target prefix
prefix-state ⊆ query-sample-space
by (*rule staged-query-prefix-clean-length-committed-query-target-subset*)
have *frac*:
nnreal
(card
(staged-query-prefix-clean-length-committed-query-target prefix
prefix-state)) /
nnreal (card query-sample-space) ≤ query-error-bound
by (*rule staged-query-prefix-clean-length-committed-query-target-fraction-bound*)
show *staged-query-prefix-clean-length-committed-query-target prefix*
prefix-state ⊆ query-sample-space ∧
nnreal
(card
(staged-query-prefix-clean-length-committed-query-target prefix
prefix-state)) /
nnreal (card query-sample-space) ≤ query-error-bound
using *subset frac by blast*
qed
also have ... ≤
staged-phase-relation-error size
(staged-query-search-queries budgets $i + 1$) + query-error-bound
by (*intro add-mono order-refl*
checked-staged-query-prefix-clean-length-committed-target-prehit-bound
[OF wf controlled i-bound])
finally show *?thesis* .
qed

lemma *checked-staged-query-prefix-length-committed-target-hit-bound-by-clean-and-collision:*

wp-event (checked-staged-query-prefix-receive-with-state A i)
(checked-staged-query-prefix-dynamic-index-hit
staged-query-prefix-length-committed-query-target) s ≤
wp-event (checked-staged-query-prefix-receive-with-state A i)
(checked-staged-query-prefix-dynamic-index-hit
staged-query-prefix-clean-length-committed-query-target) s +
wp-event (checked-staged-query-prefix-receive-with-state A i)
checked-staged-query-prefix-hash-map-output-collision s

proof –

let *?M = checked-staged-query-prefix-receive-with-state A i*
let *?hit =*
checked-staged-query-prefix-dynamic-index-hit
staged-query-prefix-length-committed-query-target
let *?clean =*
checked-staged-query-prefix-dynamic-index-hit
staged-query-prefix-clean-length-committed-query-target
let *?collision = checked-staged-query-prefix-hash-map-output-collision*
have *wp-event ?M ?hit s ≤ wp-event ?M (λout. ?clean out ∨ ?collision out) s*
by (rule *wp-event-mono*)
(rule
checked-staged-query-prefix-length-committed-hit-imp-clean-hit-or-hash-collision)
also have *... ≤ wp-event ?M ?clean s + wp-event ?M ?collision s*
by (rule *wp-event-union-bound*)
finally show *?thesis .*

qed

lemma *checked-staged-query-prefix-length-committed-target-hit-bound-by-relation-query-error-and-collision:*

assumes *raw-bound: query-index-raw-preimage-bound*
and *wf: staged-budget-wellformed budgets*
and *controlled: staged-adversary-controlled budgets A*
and *i-bound: i ≤ rounds*

shows

wp-event (checked-staged-query-prefix-receive-with-state A i)
(checked-staged-query-prefix-dynamic-index-hit
staged-query-prefix-length-committed-query-target)
adversary-initial-state ≤
staged-phase-relation-error size
(staged-query-search-queries budgets i + 1) +
query-error-bound +
wp-event (checked-staged-query-prefix-receive-with-state A i)
checked-staged-query-prefix-hash-map-output-collision
adversary-initial-state

proof –

have *split:*
wp-event (checked-staged-query-prefix-receive-with-state A i)
(checked-staged-query-prefix-dynamic-index-hit
staged-query-prefix-length-committed-query-target)

$\text{adversary-initial-state} \leq$
 $\text{wp-event } (\text{checked-staged-query-prefix-receive-with-state } A \ i)$
 $(\text{checked-staged-query-prefix-dynamic-index-hit}$
 $\quad \text{staged-query-prefix-clean-length-committed-query-target})$
 $\text{adversary-initial-state} +$
 $\text{wp-event } (\text{checked-staged-query-prefix-receive-with-state } A \ i)$
 $\text{checked-staged-query-prefix-hash-map-output-collision}$
 $\text{adversary-initial-state}$
by (rule
 $\text{checked-staged-query-prefix-length-committed-target-hit-bound-by-clean-and-collision})$
have *clean-bound*:
 $\text{wp-event } (\text{checked-staged-query-prefix-receive-with-state } A \ i)$
 $(\text{checked-staged-query-prefix-dynamic-index-hit}$
 $\quad \text{staged-query-prefix-clean-length-committed-query-target})$
 $\text{adversary-initial-state} \leq$
 $\text{staged-phase-relation-error size}$
 $(\text{staged-query-search-queries budgets } i + 1) + \text{query-error-bound}$
by (rule
 $\text{checked-staged-query-prefix-clean-length-committed-target-hit-bound}$
 $[\text{OF raw-bound wf controlled } i\text{-bound}])$
have *tail-bound*:
 $\text{wp-event } (\text{checked-staged-query-prefix-receive-with-state } A \ i)$
 $(\text{checked-staged-query-prefix-dynamic-index-hit}$
 $\quad \text{staged-query-prefix-clean-length-committed-query-target})$
 $\text{adversary-initial-state} +$
 $\text{wp-event } (\text{checked-staged-query-prefix-receive-with-state } A \ i)$
 $\text{checked-staged-query-prefix-hash-map-output-collision}$
 $\text{adversary-initial-state} \leq$
 $\text{staged-phase-relation-error size}$
 $(\text{staged-query-search-queries budgets } i + 1) +$
 $\text{query-error-bound} +$
 $\text{wp-event } (\text{checked-staged-query-prefix-receive-with-state } A \ i)$
 $\text{checked-staged-query-prefix-hash-map-output-collision}$
 $\text{adversary-initial-state}$
using *clean-bound* **by** (*simp add: add-mono*)
show ?thesis
by (rule *order-trans[OF split tail-bound]*)
qed

lemma *checked-staged-query-prefix-length-committed-target-hit-bound*:

assumes *raw-bound*: *query-index-raw-preimage-bound*
and *wf*: *staged-budget-wellformed budgets*
and *controlled*: *staged-adversary-controlled budgets A*
and *i-bound*: $i \leq \text{rounds}$

shows

$\text{wp-event } (\text{checked-staged-query-prefix-receive-with-state } A \ i)$
 $(\text{checked-staged-query-prefix-dynamic-index-hit}$
 $\quad \text{staged-query-prefix-length-committed-query-target})$
 $\text{adversary-initial-state} \leq$

```

    staged-phase-relation-error size
      (staged-query-search-queries budgets  $i + 1$ ) +
    query-error-bound +
    hash-collision-budget-value 0
    (staged-query-search-queries budgets  $i + 1$ )
  proof -
  have hit-bound:
    wp-event (checked-staged-query-prefix-receive-with-state A i)
      (checked-staged-query-prefix-dynamic-index-hit
        staged-query-prefix-length-committed-query-target)
    adversary-initial-state  $\leq$ 
    staged-phase-relation-error size
      (staged-query-search-queries budgets  $i + 1$ ) +
    query-error-bound +
    wp-event (checked-staged-query-prefix-receive-with-state A i)
      checked-staged-query-prefix-hash-map-output-collision
      adversary-initial-state
  by (rule
    checked-staged-query-prefix-length-committed-target-hit-bound-by-relation-query-error-and-collision
    [OF raw-bound wf controlled i-bound])
  have collision-bound:
    wp-event (checked-staged-query-prefix-receive-with-state A i)
      checked-staged-query-prefix-hash-map-output-collision
    adversary-initial-state  $\leq$ 
    hash-collision-budget-value 0
    (staged-query-search-queries budgets  $i + 1$ )
  by (rule checked-staged-query-prefix-hash-map-output-collision-bound
    [OF wf controlled i-bound])
  have tail:
    staged-phase-relation-error size
      (staged-query-search-queries budgets  $i + 1$ ) +
    query-error-bound +
    wp-event (checked-staged-query-prefix-receive-with-state A i)
      checked-staged-query-prefix-hash-map-output-collision
      adversary-initial-state  $\leq$ 
    staged-phase-relation-error size
      (staged-query-search-queries budgets  $i + 1$ ) +
    query-error-bound +
    hash-collision-budget-value 0
    (staged-query-search-queries budgets  $i + 1$ )
  using collision-bound by (intro add-mono order-refl)
  show ?thesis
  by (rule order-trans[OF hit-bound tail])
qed

end

end

```

```

theory Soundness-Staged-Transcript
  imports Soundness-Staged-Query
begin

```

Transcript-level staged query events and their transfer to verifier executions.

```

context soundness
begin

```

```

definition staged-transcript-query-partial-header-key-hit
  :: ('f staged-proof-data × 'f protocol-channel) option ⇒ bool
where
  staged-transcript-query-partial-header-key-hit out ⟷
    (case out of
      None ⇒ False
    | Some (data, attacker-state) ⇒
      (let s =
          verifier-state-from-adversary attacker-state
            (staged-proof-transcript data);
        B =
          query-header-supported-partial-union-good-sets s
            query-sampling-success-space
              (staged-trace-root data)
              (staged-trace-fri-roots data)
              (staged-trace-final data)
              (staged-alphas data)
              (staged-degree data)
              (staged-composition-fri-roots data)
              (staged-composition-final data)
        in
          (∃ i raw. i < rounds ∧
            fmllookup (HashMap attacker-state)
              (QueryIndexChallenge i
                (state-after-query-chunks
                  (staged-query-start-hash data)
                  (staged-query-chunks data) i)) =
              Some raw ∧
            index (to-nat raw) ∈ B)))

```

```

lemma staged-transcript-query-partial-header-key-hit-alt:
  staged-transcript-query-partial-header-key-hit out ⟷
    (case out of
      None ⇒ False
    | Some (data, attacker-state) ⇒
      (∃ i raw. i < rounds ∧
        fmllookup (HashMap attacker-state)
          (QueryIndexChallenge i
            (state-after-query-chunks
              (staged-query-start-hash data)

```

$(\text{staged-query-chunks data } i)) =$
 $\text{Some raw} \wedge$
 $\text{index (to-nat raw)} \in$
 $\text{staged-partial-query-target data attacker-state}))$
unfolding $\text{staged-transcript-query-partial-header-key-hit-def}$
 $\text{staged-partial-query-target-def Let-def}$
by ($\text{simp split: option.splits prod.splits}$)

definition $\text{staged-transcript-query-partial-header-key-hit-at}$
 $:: \text{nat} \Rightarrow ('f \text{ staged-proof-data} \times 'f \text{ protocol-channel}) \text{ option} \Rightarrow$
 bool

where

$\text{staged-transcript-query-partial-header-key-hit-at } i \text{ out} \longleftrightarrow$
 $(\text{case out of}$
 $\text{None} \Rightarrow \text{False}$
 $| \text{Some (data, attacker-state)} \Rightarrow$
 $(\exists \text{raw. } i < \text{rounds} \wedge$
 $\text{fmlookup (HashMap attacker-state)}$
 $(\text{QueryIndexChallenge } i$
 $(\text{state-after-query-chunks}$
 $(\text{staged-query-start-hash data})$
 $(\text{staged-query-chunks data } i)) =$
 $\text{Some raw} \wedge$
 $\text{index (to-nat raw)} \in$
 $\text{staged-partial-query-target data attacker-state}))$

lemma $\text{checked-staged-transcript-program-query-prefix-receive-lookup-support:}$

assumes $\text{wf: staged-budget-wellformed budgets}$

and $\text{controlled: staged-adversary-controlled budgets } A$

and $\text{i-bound: } i < \text{rounds}$

and outcome:

$\text{Some (data, attacker-state)} \in$
 set-dist
 $(\text{execute (checked-staged-transcript-program } A)$
 $\text{adversary-initial-state})$

obtains $\text{prefix prefix-state raw raw-state}$

where

$\text{Some (((prefix, prefix-state), raw), raw-state)} \in$
 set-dist
 $(\text{execute (checked-staged-query-prefix-receive-with-state } A \ i)$
 $\text{adversary-initial-state})$
 $\text{raw-state} \leq \text{attacker-state}$
 $\text{fmlookup (HashMap raw-state)}$
 $(\text{QueryIndexChallenge } i$
 $(\text{state-after-query-chunks}$
 $(\text{staged-query-start-hash data}) (\text{staged-query-chunks data } i)) =$
 Some raw

proof ($\text{rule checked-staged-transcript-program-query-prefix-receive-support}$
 $[\text{OF wf controlled i-bound outcome}]$)

```

fix prefix prefix-state raw raw-state
assume prefix-receive:
  Some (((prefix, prefix-state), raw), raw-state) ∈
    set-dist
    (execute (checked-staged-query-prefix-receive-with-state A i)
      adversary-initial-state)
and raw-ext: raw-state ≤ attacker-state
and lookup:
  fmlookup (HashMap raw-state)
    (QueryIndexChallenge i
      (state-after-query-chunks
        (staged-query-start-hash data) (staged-query-chunks data) i)) =
    Some raw
show ?thesis
  by (rule that[OF prefix-receive raw-ext lookup])
qed

lemma checked-staged-security-with-data-state-accepted-shape-query-prefix-receive-index:
assumes wf: staged-budget-wellformed budgets
and controlled: staged-adversary-controlled budgets A
and support:
  Some (((data, attacker-state), result), final-state) ∈
    set-dist
    (execute (checked-staged-security-experiment-with-data-state A)
      adversary-initial-state)
and shape:
  accepted-transcript-shape
    (verifier-state-from-adversary attacker-state
      (staged-proof-transcript data))
    (Some (result, final-state)) as query-idxs
and i-bound: i < rounds
obtains prefix prefix-state raw raw-state
where
  Some (((prefix, prefix-state), raw), raw-state) ∈
    set-dist
    (execute (checked-staged-query-prefix-receive-with-state A i)
      adversary-initial-state)
  index (to-nat raw) = query-idxs ! i
proof –
  let ?s =
    verifier-state-from-adversary attacker-state
    (staged-proof-transcript data)
from checked-staged-security-experiment-with-data-state-outcomeE
  [OF support]
have builder:
  Some (data, attacker-state) ∈
    set-dist
    (execute (checked-staged-transcript-program A)
      adversary-initial-state)

```

and *verifier*:
Some (result, final-state) ∈ set-dist (execute verify-monad ?s)
by *blast+*
from *checked-staged-security-with-data-state-accepted-shape-query-keys*
[OF wf controlled support shape]
obtain *raw-idxs* **where**
len-raw: length raw-idxs = rounds
and *query-idxs-eq*:
query-idxs = map (λraw. index (to-nat raw)) raw-idxs
and *lookup-final*:
 $\bigwedge j. j < rounds \implies$
fmlookup (HashMap final-state)
(QueryIndexChallenge j
(state-after-query-chunks
(staged-query-start-hash data)
(staged-query-chunks data) j)) =
Some (raw-idxs ! j)
by *blast*
from *checked-staged-transcript-program-query-prefix-receive-lookup-support*
[OF wf controlled i-bound builder]
obtain *prefix prefix-state raw raw-state* **where**
prefix-receive:
Some (((prefix, prefix-state), raw), raw-state) ∈
set-dist
(execute (checked-staged-query-prefix-receive-with-state A i)
adversary-initial-state)
and *raw-ext: raw-state ≤ attacker-state*
and *lookup-raw*:
fmlookup (HashMap raw-state)
(QueryIndexChallenge i
(state-after-query-chunks
(staged-query-start-hash data) (staged-query-chunks data) i)) =
Some raw
by *blast*
have *attacker-ext-s: attacker-state ≤ ?s*
proof –
have *attacker-state ≤ attacker-state*
unfolding *less-eq-hash-ext-def less-eq-fmap-def* **by** *simp*
then show *?thesis*
by *(rule hash-extends-verifier-state-from-adversary-right)*
qed
have *s-ext-final: ?s ≤ final-state*
by *(rule verify-monad-hash-extends[OF verifier])*
have *raw-ext-final: raw-state ≤ final-state*
by *(rule hash-ext-trans[OF raw-ext])*
(rule hash-ext-trans[OF attacker-ext-s s-ext-final])
have *lookup-final-raw*:
fmlookup (HashMap final-state)
(QueryIndexChallenge i

```

    (state-after-query-chunks
      (staged-query-start-hash data) (staged-query-chunks data) i)) =
  Some raw
  by (rule hash-extension-lookup[OF lookup-raw raw-ext-final])
have raw-eq: raw-idxs ! i = raw
  using lookup-final[OF i-bound] lookup-final-raw by simp
have idx-eq: index (to-nat raw) = query-idxs ! i
  using query-idxs-eq len-raw i-bound raw-eq by simp
show ?thesis
  by (rule that[OF prefix-receive idx-eq])
qed

lemma checked-staged-transcript-program-query-prefix-receive-match-support:
  assumes wf: staged-budget-wellformed budgets
    and controlled: staged-adversary-controlled budgets A
    and i-bound: i < rounds
    and outcome:
      Some (data, attacker-state) ∈
        set-dist
          (execute (checked-staged-transcript-program A)
            adversary-initial-state)
  obtains prefix prefix-state raw raw-state
  where
    Some (((prefix, prefix-state), raw), raw-state) ∈
      set-dist
        (execute (checked-staged-query-prefix-receive-with-state A i)
          adversary-initial-state)
    sqp-trace-root prefix = staged-trace-root data
    sqp-trace-fri-roots prefix = staged-trace-fri-roots data
    sqp-trace-fri-challenges prefix =
      staged-trace-fri-challenges data
    sqp-trace-final prefix = staged-trace-final data
    sqp-alphas prefix = staged-alphas data
    sqp-degree prefix = staged-degree data
    sqp-composition-fri-roots prefix =
      staged-composition-fri-roots data
    sqp-composition-fri-challenges prefix =
      staged-composition-fri-challenges data
    sqp-composition-final prefix = staged-composition-final data
    sqp-query-chunks prefix = take i (staged-query-chunks data)
    prefix-state ≤ attacker-state
    raw-state ≤ attacker-state
    fmlookup (HashMap raw-state)
      (QueryIndexChallenge i
        (state-after-query-chunks
          (staged-query-start-hash data) (staged-query-chunks data) i)) =
      Some raw
  proof (rule checked-staged-transcript-program-query-prefix-receive-support
    [OF wf controlled i-bound outcome])

```



```

fix prefix prefix-state raw raw-state
assume prefix-receive:
  Some (((prefix, prefix-state), raw), raw-state) ∈
    set-dist
    (execute (checked-staged-query-prefix-receive-with-state A i)
      adversary-initial-state)
and root-eq: sqp-trace-root prefix = staged-trace-root data
and trace-roots-eq:
  sqp-trace-fri-roots prefix = staged-trace-fri-roots data
and trace-bs-eq:
  sqp-trace-fri-challenges prefix =
    staged-trace-fri-challenges data
and trace-final-eq: sqp-trace-final prefix = staged-trace-final data
and alphas-eq: sqp-alphas prefix = staged-alphas data
and degree-eq: sqp-degree prefix = staged-degree data
and comp-roots-eq:
  sqp-composition-fri-roots prefix =
    staged-composition-fri-roots data
and comp-bs-eq:
  sqp-composition-fri-challenges prefix =
    staged-composition-fri-challenges data
and comp-final-eq:
  sqp-composition-final prefix = staged-composition-final data
and query-chunks-eq:
  sqp-query-chunks prefix = take i (staged-query-chunks data)
and prefix-ext: prefix-state ≤ attacker-state
and raw-ext: raw-state ≤ attacker-state
and lookup:
  fmlookup (HashMap raw-state)
    (QueryIndexChallenge i
      (state-after-query-chunks
        (staged-query-start-hash data) (staged-query-chunks data) i)) =
    Some raw
show ?thesis
by (rule that[OF prefix-receive root-eq trace-roots-eq trace-bs-eq
  trace-final-eq alphas-eq degree-eq comp-roots-eq comp-bs-eq
  comp-final-eq query-chunks-eq prefix-ext raw-ext lookup])
qed

```

```

lemma checked-staged-transcript-query-hit-at-prefix-receive-support:
assumes wf: staged-budget-wellformed budgets
and controlled: staged-adversary-controlled budgets A
and outcome:
  Some (data, attacker-state) ∈
    set-dist
    (execute (checked-staged-transcript-program A)
      adversary-initial-state)
and hit:
  staged-transcript-query-partial-header-key-hit-at i

```

```

    (Some (data, attacker-state))
  shows
    ∃ prefix prefix-state raw raw-state.
      Some (((prefix, prefix-state), raw), raw-state) ∈
        set-dist
          (execute (checked-staged-query-prefix-receive-with-state A i)
            adversary-initial-state) ∧
        checked-staged-query-prefix-fixed-transcript-target-hit
          data attacker-state
        (Some (((prefix, prefix-state), raw), raw-state))
  proof -
    from hit obtain raw-hit where
      i-bound: i < rounds
    and lookup-attacker:
      fmlookup (HashMap attacker-state)
        (QueryIndexChallenge i
          (state-after-query-chunks
            (staged-query-start-hash data) (staged-query-chunks data) i)) =
        Some raw-hit
    and raw-hit-target:
      index (to-nat raw-hit) ∈ staged-partial-query-target data attacker-state
    unfolding staged-transcript-query-partial-header-key-hit-at-def
    by auto
  from checked-staged-transcript-program-query-prefix-receive-lookup-support
    [OF wf controlled i-bound outcome]
  obtain prefix prefix-state raw raw-state where
    prefix-receive:
      Some (((prefix, prefix-state), raw), raw-state) ∈
        set-dist
          (execute (checked-staged-query-prefix-receive-with-state A i)
            adversary-initial-state)
    and raw-ext: raw-state ≤ attacker-state
    and lookup-raw:
      fmlookup (HashMap raw-state)
        (QueryIndexChallenge i
          (state-after-query-chunks
            (staged-query-start-hash data) (staged-query-chunks data) i)) =
        Some raw
    by (rule checked-staged-transcript-program-query-prefix-receive-lookup-support
      [OF wf controlled i-bound outcome])
  have lookup-attacker-raw:
    fmlookup (HashMap attacker-state)
      (QueryIndexChallenge i
        (state-after-query-chunks
          (staged-query-start-hash data) (staged-query-chunks data) i)) =
    Some raw
  by (rule hash-extension-lookup[OF lookup-raw raw-ext])
  have raw-eq: raw = raw-hit
  using lookup-attacker lookup-attacker-raw by simp

```

```

have fixed-hit:
  checked-staged-query-prefix-fixed-transcript-target-hit
    data attacker-state
    (Some (((prefix, prefix-state), raw), raw-state))
using raw-hit-target unfolding raw-eq
  checked-staged-query-prefix-fixed-transcript-target-hit-def
  checked-staged-query-prefix-dynamic-index-hit-def
by simp
show ?thesis
  using prefix-receive fixed-hit by blast
qed

lemma checked-staged-transcript-query-hit-at-prefix-completion-receive-support:
assumes wf: staged-budget-wellformed budgets
and controlled: staged-adversary-controlled budgets A
and outcome:
  Some (data, attacker-state) ∈
    set-dist
    (execute (checked-staged-transcript-program A)
      adversary-initial-state)
and hit:
  staged-transcript-query-partial-header-key-hit-at i
    (Some (data, attacker-state))
shows
  ∃ prefix prefix-state raw raw-state.
    Some (((prefix, prefix-state), raw), raw-state) ∈
      set-dist
      (execute (checked-staged-query-prefix-receive-with-state A i)
        adversary-initial-state) ∧
      checked-staged-query-prefix-dynamic-index-hit
        (staged-query-prefix-partial-query-target i)
        (Some (((prefix, prefix-state), raw), raw-state))
proof –
from hit obtain raw-hit where
  i-bound: i < rounds
and lookup-attacker:
  fmlookup (HashMap attacker-state)
    (QueryIndexChallenge i
      (state-after-query-chunks
        (staged-query-start-hash data) (staged-query-chunks data i))) =
    Some raw-hit
and raw-hit-target:
  index (to-nat raw-hit) ∈ staged-partial-query-target data attacker-state
unfolding staged-transcript-query-partial-header-key-hit-at-def
by auto
from checked-staged-transcript-program-query-prefix-receive-match-support
  [OF wf controlled i-bound outcome]
obtain prefix prefix-state raw raw-state where
  prefix-receive:

```

```

    Some (((prefix, prefix-state), raw), raw-state) ∈
      set-dist
        (execute (checked-staged-query-prefix-receive-with-state A i)
          adversary-initial-state)
  and root-eq: sqp-trace-root prefix = staged-trace-root data
  and trace-roots-eq:
    sqp-trace-fri-roots prefix = staged-trace-fri-roots data
  and trace-bs-eq:
    sqp-trace-fri-challenges prefix =
      staged-trace-fri-challenges data
  and trace-final-eq:
    sqp-trace-final prefix = staged-trace-final data
  and alphas-eq: sqp-alphas prefix = staged-alphas data
  and degree-eq: sqp-degree prefix = staged-degree data
  and comp-roots-eq:
    sqp-composition-fri-roots prefix =
      staged-composition-fri-roots data
  and comp-bs-eq:
    sqp-composition-fri-challenges prefix =
      staged-composition-fri-challenges data
  and comp-final-eq:
    sqp-composition-final prefix = staged-composition-final data
  and query-chunks-eq:
    sqp-query-chunks prefix = take i (staged-query-chunks data)
  and prefix-ext: prefix-state ≤ attacker-state
  and raw-ext: raw-state ≤ attacker-state
  and lookup-raw:
    fmlookup (HashMap raw-state)
      (QueryIndexChallenge i
        (state-after-query-chunks
          (staged-query-start-hash data) (staged-query-chunks data) i)) =
      Some raw
  by (rule checked-staged-transcript-program-query-prefix-receive-match-support
    [OF wf controlled i-bound outcome])
  have lookup-attacker-raw:
    fmlookup (HashMap attacker-state)
      (QueryIndexChallenge i
        (state-after-query-chunks
          (staged-query-start-hash data) (staged-query-chunks data) i)) =
    Some raw
  by (rule hash-extension-lookup[OF lookup-raw raw-ext])
  have raw-eq: raw = raw-hit
  using lookup-attacker lookup-attacker-raw by simp
  have match:
    staged-query-prefix-matches-data i prefix data
  unfolding staged-query-prefix-matches-data-def
  using root-eq trace-roots-eq trace-bs-eq trace-final-eq alphas-eq
    degree-eq comp-roots-eq comp-bs-eq comp-final-eq query-chunks-eq
  by simp

```

have *prefix-target*:
index (*to-nat raw*) $\in$
staged-query-prefix-partial-query-target *i* *prefix* *prefix-state*
unfolding *raw-eq*
using *raw-hit-target*
staged-partial-query-target-subset-prefix-completion [*OF match prefix-ext*]
by *blast*
have *prefix-hit*:
checked-staged-query-prefix-dynamic-index-hit
(*staged-query-prefix-partial-query-target* *i*)
(*Some* (((*prefix*, *prefix-state*), *raw*), *raw-state*))
using *prefix-target*
unfolding *checked-staged-query-prefix-dynamic-index-hit-def*
by *simp*
show *?thesis*
using *prefix-receive prefix-hit* **by** *blast*
qed

lemma *checked-staged-after-query-prefix-receive-partial-hit-imp-prefix-hit*:

assumes *wf*: *staged-budget-wellformed* *budgets*
and *controlled*: *staged-adversary-controlled* *budgets A*
and *i-bound*: *i* < *rounds*
and *head-support*:
Some (((*prefix*, *prefix-state*), *raw*), *raw-state*) $\in$
set-dist
(*execute* (*checked-staged-query-prefix-receive-with-state A i*)
adversary-initial-state)
and *cont-support*:
Some (*data*, *attacker-state*) $\in$
set-dist
(*execute*
(*checked-staged-after-query-prefix-receive A i*
((*prefix*, *prefix-state*), *raw*))
raw-state)
and *hit*:
staged-transcript-query-partial-header-key-hit-at i
(*Some* (*data*, *attacker-state*))

shows

checked-staged-query-prefix-dynamic-index-hit
(*staged-query-prefix-partial-query-target* *i*)
(*Some* (((*prefix*, *prefix-state*), *raw*), *raw-state*))

proof –

have *prefix-support*:
Some ((*prefix*, *prefix-state*), *prefix-state*) $\in$
set-dist
(*execute* (*checked-staged-query-prefix-with-state A i*)
adversary-initial-state)
by (*rule checked-staged-query-prefix-receive-with-state-prefix-support*
[*OF head-support*])

```

have recv:
  Some (raw, raw-state) ∈
    set-dist (execute receive-query-index-challenge prefix-state)
by (rule checked-staged-query-prefix-receive-with-state-receive-support
  [OF head-support])
from checked-staged-query-prefix-with-state-alignment
  [OF wf controlled i-bound prefix-support]
have prefix-counter: PQueryCounter prefix-state = i
and prefix-state:
  PState prefix-state =
    state-after-query-chunks
      (staged-query-prefix-start-hash prefix)
      (sqp-query-chunks prefix) i
and prefix-len: length (sqp-query-chunks prefix) = i
by blast+
from cont-support obtain chunk chunk-state record-state suffix-chunks where
  chunk-out:
    Some (chunk, chunk-state) ∈
      set-dist (execute (query-opening-stage A i raw) raw-state)
and record-out:
  Some ((), record-state) ∈
    set-dist (execute (record-staged-messages chunk) chunk-state)
and suffix-out:
  Some (suffix-chunks, attacker-state) ∈
    set-dist
      (execute
        (checked-staged-query-program A
          (sqp-trace-fri-roots prefix)
          (sqp-composition-fri-roots prefix)
          (Suc i) (rounds - Suc i))
        record-state)
and data-eq:
  data =
    (staged-trace-root = sqp-trace-root prefix,
     staged-trace-fri-roots = sqp-trace-fri-roots prefix,
     staged-trace-fri-challenges = sqp-trace-fri-challenges prefix,
     staged-trace-final = sqp-trace-final prefix,
     staged-alphas = sqp-alphas prefix,
     staged-degree = sqp-degree prefix,
     staged-composition-fri-roots = sqp-composition-fri-roots prefix,
     staged-composition-fri-challenges =
       sqp-composition-fri-challenges prefix,
     staged-composition-final = sqp-composition-final prefix,
     staged-query-chunks =
       sqp-query-chunks prefix @ chunk # suffix-chunks)
unfolding checked-staged-after-query-prefix-receive-def Let-def
by (auto elim!: set-dist-bindE simp: assert-def throw-no-outcome
  split: if-splits)
have match: staged-query-prefix-matches-data i prefix data

```

```

unfolding staged-query-prefix-matches-data-def data-eq
using prefix-len by simp
have start-hash:
  staged-query-start-hash data = staged-query-prefix-start-hash prefix
  by (rule staged-query-start-hash-of-prefix-completion[OF match])
have prefix-key-state:
  PState prefix-state =
    state-after-query-chunks
    (staged-query-start-hash data) (staged-query-chunks data) i
using prefix-state prefix-len start-hash
unfolding data-eq
by (simp add: state-after-query-chunks-append-prefix)
have receive-lookup:
  fmlookup (HashMap raw-state)
    (QueryIndexChallenge i
      (state-after-query-chunks
        (staged-query-start-hash data) (staged-query-chunks data) i)) =
    Some raw
using receive-query-index-challenge-outcome[OF recv]
  prefix-counter prefix-key-state
by simp
have prefix-raw-ext: prefix-state  $\leq$  raw-state
  by (rule receive-query-index-challenge-extends[OF recv])
have stage-controlled:
  controlled-ro-program (query-opening-budgets budgets ! i)
    (query-opening-stage A i raw)
using controlled wf i-bound
unfolding staged-adversary-controlled-def staged-budget-wellformed-def
by simp
have raw-chunk-ext: raw-state  $\leq$  chunk-state
using controlled-ro-program-extension[OF stage-controlled] chunk-out
unfolding hash-extension-preserving-def
by blast
have record-state-eq:
  record-state = chunk-state[
    PState := foldl concat (PState chunk-state) chunk,
    PTranscript := PTranscript chunk-state @ chunk]
  by (rule record-staged-messages-outcome[OF record-out])
have chunk-record-ext: chunk-state  $\leq$  record-state
unfolding record-state-eq less-eq-hash-ext-def less-eq-fmap-def
by simp
have suffix-bound:
  Suc i + (rounds - Suc i)  $\leq$  length (query-opening-budgets budgets)
using wf i-bound unfolding staged-budget-wellformed-def by simp
have suffix-target:
  hash-target-program {}
    (sum-list
      (take (rounds - Suc i)
        (drop (Suc i) (query-opening-budgets budgets)))) +

```

```

    (rounds - Suc i))
  (checked-staged-query-program A
   (sqp-trace-fri-roots prefix)
   (sqp-composition-fri-roots prefix)
   (Suc i) (rounds - Suc i))
  by (rule hash-target-program-checked-staged-query-program
      [OF controlled suffix-bound])
  have suffix-ext: record-state ≤ attacker-state
    using hash-target-program-extension[OF suffix-target] suffix-out
    unfolding hash-extension-preserving-def
    by blast
  have raw-ext: raw-state ≤ attacker-state
    by (rule hash-ext-trans[OF raw-chunk-ext])
      (rule hash-ext-trans[OF chunk-record-ext suffix-ext])
  have prefix-ext: prefix-state ≤ attacker-state
    by (rule hash-ext-trans[OF prefix-raw-ext raw-ext])
  from hit obtain raw-hit where
    lookup-attacker:
      fmlookup (HashMap attacker-state)
        (QueryIndexChallenge i
         (state-after-query-chunks
          (staged-query-start-hash data) (staged-query-chunks data) i)) =
        Some raw-hit
    and raw-hit-target:
      index (to-nat raw-hit) ∈
        staged-partial-query-target data attacker-state
    unfolding staged-transcript-query-partial-header-key-hit-at-def
    by auto
  have lookup-attacker-raw:
    fmlookup (HashMap attacker-state)
      (QueryIndexChallenge i
       (state-after-query-chunks
        (staged-query-start-hash data) (staged-query-chunks data) i)) =
    Some raw
    by (rule hash-extension-lookup[OF receive-lookup raw-ext])
  have raw-eq: raw = raw-hit
    using lookup-attacker lookup-attacker-raw by simp
  have prefix-target:
    index (to-nat raw) ∈
      staged-query-prefix-partial-query-target i prefix prefix-state
    unfolding raw-eq
    using raw-hit-target
      staged-partial-query-target-subset-prefix-completion[OF match prefix-ext]
    by blast
  show ?thesis
    using prefix-target
    unfolding checked-staged-query-prefix-dynamic-index-hit-def
    by simp
qed

```


lemma *checked-staged-transcript-program-query-prefix-receive-compact-support*:

assumes *wf*: *staged-budget-wellformed budgets*
and *controlled*: *staged-adversary-controlled budgets A*
and *i-bound*: $i < \text{rounds}$
and *outcome*:
 Some (*data*, *attacker-state*) $\in$
 set-dist
 (*execute* (*checked-staged-transcript-program A*)
 adversary-initial-state)

obtains *prefix prefix-state raw raw-state*
where
 Some (((*prefix*, *prefix-state*), *raw*), *raw-state*) $\in$
 set-dist
 (*execute* (*checked-staged-query-prefix-receive-with-state A i*)
 adversary-initial-state)
 sqp-trace-root prefix = staged-trace-root data
 sqp-alphas prefix = staged-alphas data
 sqp-composition-fri-roots prefix =
 staged-composition-fri-roots data
 prefix-state $\leq$ attacker-state
 raw-state $\leq$ attacker-state
 fmlookup (HashMap raw-state)
 (*QueryIndexChallenge i*
 (*state-after-query-chunks*
 (*staged-query-start-hash data*) (*staged-query-chunks data*) *i*)) =
 Some raw

proof (*rule checked-staged-transcript-program-query-prefix-receive-match-support*
 [*OF wf controlled i-bound outcome*])
fix *prefix prefix-state raw raw-state*
assume *prefix-receive*:
 Some (((*prefix*, *prefix-state*), *raw*), *raw-state*) $\in$
 set-dist
 (*execute* (*checked-staged-query-prefix-receive-with-state A i*)
 adversary-initial-state)
and *root-eq*: *sqp-trace-root prefix = staged-trace-root data*
and *alphas-eq*: *sqp-alphas prefix = staged-alphas data*
and *comp-roots-eq*:
 sqp-composition-fri-roots prefix =
 staged-composition-fri-roots data
and *prefix-ext*: *prefix-state $\leq$ attacker-state*
and *raw-ext*: *raw-state $\leq$ attacker-state*
and *lookup*:
 fmlookup (HashMap raw-state)
 (*QueryIndexChallenge i*
 (*state-after-query-chunks*
 (*staged-query-start-hash data*) (*staged-query-chunks data*) *i*)) =
 Some raw

show *?thesis*

by (rule that[OF prefix-receive root-eq alphas-eq comp-roots-eq
 prefix-ext raw-ext lookup])

qed

lemma *checked-staged-after-query-prefix-receive-extends:*
assumes *wf: staged-budget-wellformed budgets*
and *controlled: staged-adversary-controlled budgets A*
and *i-bound: i < rounds*
and *data-out:*
Some (data, attacker-state) ∈
set-dist
(execute
(checked-staged-after-query-prefix-receive A i
((prefix, prefix-state), raw))
raw-state)
shows *raw-state ≤ attacker-state*

proof –

from *data-out* **obtain** *chunk chunk-state record-state suffix-chunks* **where**
chunk-out:
Some (chunk, chunk-state) ∈
set-dist (execute (query-opening-stage A i raw) raw-state)
and *record-out:*
Some ((), record-state) ∈
set-dist (execute (record-staged-messages chunk) chunk-state)
and *suffix-out:*
Some (suffix-chunks, attacker-state) ∈
set-dist
(execute
(checked-staged-query-program A
(sqp-trace-fri-roots prefix)
(sqp-composition-fri-roots prefix)
(Suc i) (rounds – Suc i))
record-state)
unfolding *checked-staged-after-query-prefix-receive-def Let-def*
by (auto elim!: *set-dist-bindE simp: assert-def throw-no-outcome*
split: if-splits)

have *stage-controlled:*
controlled-ro-program (query-opening-budgets budgets ! i)
(query-opening-stage A i raw)
using *controlled wf i-bound*
unfolding *staged-adversary-controlled-def staged-budget-wellformed-def*
by *simp*
have *raw-chunk-ext: raw-state ≤ chunk-state*
using *controlled-ro-program-extension[OF stage-controlled] chunk-out*
unfolding *hash-extension-preserving-def*
by *blast*

have *record-state-eq:*
record-state = chunk-state[
PState := foldl concat (PState chunk-state) chunk,

```

    PTranscript := PTranscript chunk-state @ chunk)
  by (rule record-staged-messages-outcome[OF record-out])
have chunk-record-ext: chunk-state ≤ record-state
  unfolding record-state-eq less-eq-hash-ext-def less-eq-fmap-def
  by simp
have suffix-bound:
  Suc i + (rounds - Suc i) ≤ length (query-opening-budgets budgets)
  using wf i-bound unfolding staged-budget-wellformed-def by simp
have suffix-target:
  hash-target-program {}
  (sum-list
    (take (rounds - Suc i)
      (drop (Suc i) (query-opening-budgets budgets))) +
    (rounds - Suc i))
  (checked-staged-query-program A
    (spp-trace-fri-roots prefix)
    (spp-composition-fri-roots prefix)
    (Suc i) (rounds - Suc i))
  by (rule hash-target-program-checked-staged-query-program
    [OF controlled suffix-bound])
have suffix-ext: record-state ≤ attacker-state
  using hash-target-program-extension[OF suffix-target] suffix-out
  unfolding hash-extension-preserving-def
  by blast
show ?thesis
  by (rule hash-ext-trans[OF raw-chunk-ext])
    (rule hash-ext-trans[OF chunk-record-ext suffix-ext])
qed

```

lemma *checked-staged-after-query-prefix-receive-with-verifier-query-bad-at-imp-length-prefix-hit-or-tree-output:*
assumes *wf: staged-budget-wellformed budgets*
and *controlled: staged-adversary-controlled budgets A*
and *i-bound: i < rounds*
and *head-support:*
Some (((prefix, prefix-state), raw), raw-state) ∈
set-dist
(execute (checked-staged-query-prefix-receive-with-state A i)
adversary-initial-state)
and *cont-support:*
out ∈
set-dist
(execute
(checked-staged-after-query-prefix-receive-with-verifier A i
((prefix, prefix-state), raw))
raw-state)
and *hit:*
staged-security-with-data-state-query-bad-hit-at i out
shows
checked-staged-query-prefix-dynamic-index-hit

```

    staged-query-prefix-length-committed-query-target
    (Some (((prefix, prefix-state), raw), raw-state))  $\vee$ 
    staged-security-with-data-state-query-tree-output-hit A out
proof (cases out)
  case None
  then show ?thesis
    using hit unfolding staged-security-with-data-state-query-bad-hit-at-def
    by simp
next
  case (Some packed)
  then obtain data attacker-state result final-state where out-eq:
    out = Some (((data, attacker-state), result), final-state)
    by (cases packed, auto split: prod.splits)
  let ?s =
    verifier-state-from-adversary attacker-state
    (staged-proof-transcript data)
  have data-out':
    Some (data, attacker-state)  $\in$ 
    set-dist
    (execute
      (checked-staged-after-query-prefix-receive A i
        ((prefix, prefix-state), raw))
      raw-state)
    by (rule
      checked-staged-after-query-prefix-receive-with-verifier-outcome(1)
      [OF cont-support[unfolded out-eq]])
  have verifier:
    Some (result, final-state)  $\in$  set-dist (execute verify-monad ?s)
    by (rule
      checked-staged-after-query-prefix-receive-with-verifier-outcome(2)
      [OF cont-support[unfolded out-eq]])
  from hit[unfolded out-eq]
    staged-security-with-data-state-query-bad-hit-at-def Let-def]
  have hit-at:
    query-index-round-set-hit-at ?s query-sampling-success-space i
    (Some (result, final-state))
    by simp
  from hit-at obtain trace-table composition-table as query-idxs where
    bound:
      accepted-with-bound-tables ?s (Some (result, final-state))
      trace-table composition-table as query-idxs
  and query-hit:
    query-idxs ! i  $\in$ 
    query-sampling-success-space trace-table composition-table as
  unfolding query-index-round-set-hit-at-def by blast
  have transcript-decomp:
    checked-staged-transcript-program A =
    checked-staged-query-prefix-receive-with-state A i  $\gg$ 
    checked-staged-after-query-prefix-receive A i

```

```

by (rule checked-staged-transcript-program-query-prefix-receive-decomp
  [OF i-bound])
have builder:
  Some (data, attacker-state) ∈
    set-dist
      (execute (checked-staged-transcript-program A)
        adversary-initial-state)
unfolding transcript-decomp
by (rule-tac x=((prefix, prefix-state), raw) and t=raw-state
  in set-dist-bindI)
  (rule head-support, rule data-out')
have security-decomp:
  checked-staged-security-experiment-with-data-state A =
  checked-staged-query-prefix-receive-with-state A i >>=
  checked-staged-after-query-prefix-receive-with-verifier A i
by (rule
  checked-staged-security-experiment-with-data-state-query-prefix-receive-decomp
  [OF i-bound])
have support-some:
  Some (((data, attacker-state), result), final-state) ∈
    set-dist
      (execute (checked-staged-security-experiment-with-data-state A)
        adversary-initial-state)
unfolding security-decomp out-eq[symmetric]
by (rule-tac x=((prefix, prefix-state), raw) and t=raw-state
  in set-dist-bindI)
  (rule head-support, rule cont-support)
have shape:
  accepted-transcript-shape ?s (Some (result, final-state))
  as query-idxs
using bound
unfolding accepted-with-bound-tables-def accepted-with-tables-def
by simp
from checked-staged-security-with-data-state-accepted-shape-query-keys
  [OF wf controlled support-some shape]
obtain final-raw-idxs where
  as-eq: as = staged-alphas data
and len-final-raw: length final-raw-idxs = rounds
and query-idxs-eq:
  query-idxs = map (λraw. index (to-nat raw)) final-raw-idxs
and lookup-final:
  ∧j. j < rounds ⇒
    fmlookup (HashMap final-state)
      (QueryIndexChallenge j
        (state-after-query-chunks
          (staged-query-start-hash data)
          (staged-query-chunks data) j)) =
      Some (final-raw-idxs ! j)
by blast

```

have *prefix-support*:
Some ((*prefix*, *prefix-state*), *prefix-state*) $\in$
set-dist
(*execute* (*checked-staged-query-prefix-with-state* *A* *i*)
adversary-initial-state)
by (*rule* *checked-staged-query-prefix-receive-with-state-prefix-support*
[*OF* *head-support*])
have *recv*:
Some (*raw*, *raw-state*) $\in$
set-dist (*execute* *receive-query-index-challenge* *prefix-state*)
by (*rule* *checked-staged-query-prefix-receive-with-state-receive-support*
[*OF* *head-support*])
from *checked-staged-query-prefix-with-state-alignment*
[*OF* *wf* *controlled* *i-bound* *prefix-support*]
have *prefix-counter*: *PQueryCounter* *prefix-state* = *i*
and *prefix-state-hash*:
PState *prefix-state* =
state-after-query-chunks
(*staged-query-prefix-start-hash* *prefix*)
(*sqp-query-chunks* *prefix*) *i*
and *prefix-len*: *length* (*sqp-query-chunks* *prefix*) = *i*
by *blast*+
from *data-out'* **obtain** *chunk* *chunk-state* *record-state* *suffix-chunks* **where**
chunk-out:
Some (*chunk*, *chunk-state*) $\in$
set-dist (*execute* (*query-opening-stage* *A* *i* *raw*) *raw-state*)
and *record-out*:
Some ((), *record-state*) $\in$
set-dist (*execute* (*record-staged-messages* *chunk*) *chunk-state*)
and *suffix-out*:
Some (*suffix-chunks*, *attacker-state*) $\in$
set-dist
(*execute*
(*checked-staged-query-program* *A*
(*sqp-trace-fri-roots* *prefix*)
(*sqp-composition-fri-roots* *prefix*)
(*Suc* *i*) (*rounds* - *Suc* *i*))
record-state)
and *data-eq*:
data =
(*staged-trace-root* = *sqp-trace-root* *prefix*,
staged-trace-fri-roots = *sqp-trace-fri-roots* *prefix*,
staged-trace-fri-challenges = *sqp-trace-fri-challenges* *prefix*,
staged-trace-final = *sqp-trace-final* *prefix*,
staged-alphas = *sqp-alphas* *prefix*,
staged-degree = *sqp-degree* *prefix*,
staged-composition-fri-roots = *sqp-composition-fri-roots* *prefix*,
staged-composition-fri-challenges =
sqp-composition-fri-challenges *prefix*,

```

    staged-composition-final = sqp-composition-final prefix,
    staged-query-chunks =
      sqp-query-chunks prefix @ chunk # suffix-chunks)
  unfolding checked-staged-after-query-prefix-receive-def Let-def
  by (auto elim!: set-dist-bindE simp: assert-def throw-no-outcome
    split: if-splits)
have match: staged-query-prefix-matches-data i prefix data
  unfolding staged-query-prefix-matches-data-def data-eq
  using prefix-len by simp
have start-hash:
  staged-query-start-hash data = staged-query-prefix-start-hash prefix
  by (rule staged-query-start-hash-of-prefix-completion[OF match])
have prefix-key-state:
  PState prefix-state =
    state-after-query-chunks
      (staged-query-start-hash data) (staged-query-chunks data) i
  using prefix-state-hash prefix-len start-hash
  unfolding data-eq
  by (simp add: state-after-query-chunks-append-prefix)
have receive-lookup:
  fmllookup (HashMap raw-state)
    (QueryIndexChallenge i
      (state-after-query-chunks
        (staged-query-start-hash data) (staged-query-chunks data) i)) =
    Some raw
  using receive-query-index-challenge-outcome[OF recv]
  prefix-counter prefix-key-state
  by simp
have raw-ext-attacker: raw-state ≤ attacker-state
  by (rule checked-staged-after-query-prefix-receive-extends
    [OF wf controlled i-bound data-out])
have attacker-ext-s: attacker-state ≤ ?s
proof –
  have attacker-state ≤ attacker-state
  unfolding less-eq-hash-ext-def less-eq-fmap-def by simp
  then show ?thesis
  by (rule hash-extends-verifier-state-from-adversary-right)
qed
have s-ext-final: ?s ≤ final-state
  by (rule verify-monad-hash-extends[OF verifier])
have raw-ext-final: raw-state ≤ final-state
  by (rule hash-ext-trans[OF raw-ext-attacker])
  (rule hash-ext-trans[OF attacker-ext-s s-ext-final])
have lookup-final-raw:
  fmllookup (HashMap final-state)
    (QueryIndexChallenge i
      (state-after-query-chunks
        (staged-query-start-hash data) (staged-query-chunks data) i)) =
    Some raw

```

```

    by (rule hash-extension-lookup[OF receive-lookup raw-ext-final])
  have raw-eq: final-raw-idxs ! i = raw
    using lookup-final[OF i-bound] lookup-final-raw by simp
  have raw-hit-actual:
    index (to-nat raw) ∈
      query-sampling-success-space trace-table composition-table
        (staged-alphas data)
  proof -
    have index (to-nat (final-raw-idxs ! i)) = query-idxs ! i
      using query-idxs-eq len-final-raw i-bound by simp
    then show ?thesis
      using query-hit as-eq raw-eq by simp
  qed
  have prefix-ext-raw: prefix-state ≤ raw-state
    by (rule receive-query-index-challenge-extends[OF recv])
  have prefix-ext-final: prefix-state ≤ final-state
    by (rule hash-ext-trans[OF prefix-ext-raw raw-ext-final])
  have staged-header:
    verifier-header-transcript ?s
      (staged-trace-root data)
      (staged-trace-fri-roots data)
      (staged-trace-final data)
      (staged-alphas data)
      (staged-degree data)
      (staged-composition-fri-roots data)
      (staged-composition-final data)
      (List.concat (staged-query-chunks data))
    by (rule checked-staged-transcript-program-outcome-header-transcript
      [OF wf controlled builder])
  from accepted-with-bound-tables-initial-roots-pullback-or-new-tree-output-hit
    [OF prefix-ext-final bound]
  obtain fr f-fri-roots f-final dg composition-fri-roots final rest
    trace-tree composition-tree where
    header:
      verifier-header-transcript ?s fr f-fri-roots f-final as dg
      composition-fri-roots final rest
  and comp-nonempty: composition-fri-roots ≠ []
  and trace-created:
    created-tree trace-table trace-tree final-state
  and trace-root: fr = value trace-tree
  and comp-created:
    created-tree composition-table composition-tree final-state
  and comp-root:
    hd composition-fri-roots = value composition-tree
  and split:
    (merkle-root-binds-table fr trace-table prefix-state ∧
      merkle-root-binds-table (hd composition-fri-roots)
        composition-table prefix-state) ∨
    hash-map-new-output-hit

```



```

    (set-tree trace-tree  $\cup$  set-tree composition-tree)
    prefix-state final-state
  by blast
have eqs:
  fr = staged-trace-root data  $\wedge$ 
  f-fri-roots = staged-trace-fri-roots data  $\wedge$ 
  f-final = staged-trace-final data  $\wedge$ 
  as = staged-alphas data  $\wedge$ 
  dg = staged-degree data  $\wedge$ 
  composition-fri-roots = staged-composition-fri-roots data  $\wedge$ 
  final = staged-composition-final data  $\wedge$ 
  rest = List.concat (staged-query-chunks data)
using verifier-header-transcript-unique[OF header staged-header]
by simp
have trace-len: length trace-table = clength * scale
  using bound unfolding accepted-with-bound-tables-def
  accepted-with-tables-def by simp
have comp-len: length composition-table = clength * scale
  using bound unfolding accepted-with-bound-tables-def
  accepted-with-tables-def by simp
from split show ?thesis
proof
  assume binds:
    merkle-root-binds-table fr trace-table prefix-state  $\wedge$ 
    merkle-root-binds-table (hd composition-fri-roots)
    composition-table prefix-state
  have candidate:
    (trace-table, composition-table)  $\in$ 
    staged-query-prefix-length-committed-table-candidates prefix
    prefix-state
  unfolding staged-query-prefix-length-committed-table-candidates-def
  using binds comp-nonempty eqs data-eq trace-len comp-len by simp
  have target:
    index (to-nat raw)  $\in$ 
    staged-query-prefix-length-committed-query-target prefix
    prefix-state
  proof -
    have raw-hit-prefix:
      index (to-nat raw)  $\in$ 
      query-sampling-success-space trace-table composition-table
      (sqp-alphas prefix)
    using raw-hit-actual data-eq by simp
  show ?thesis
    by (rule staged-query-prefix-length-committed-query-targetI
        [OF candidate raw-hit-prefix])
  qed
then show ?thesis
  unfolding checked-staged-query-prefix-dynamic-index-hit-def
  by simp

```

```

next
  assume tree-hit:
    hash-map-new-output-hit
      (set-tree trace-tree  $\cup$  set-tree composition-tree)
      prefix-state final-state
  have comp-nonempty-prefix:
    sqp-composition-fri-roots prefix  $\neq []$ 
    using comp-nonempty eqs data-eq by simp
  have trace-root-prefix:
    sqp-trace-root prefix = value trace-tree
    using trace-root eqs data-eq by simp
  have comp-root-prefix:
    hd (sqp-composition-fri-roots prefix) = value composition-tree
    using comp-root eqs data-eq by simp
  have tree-event-body:
     $\exists i$  prefix prefix-state raw raw-state trace-table
      composition-table as query-idxs trace-tree composition-tree.
    i < rounds  $\wedge$ 
    Some (data, attacker-state)  $\in$ 
      set-dist
        (execute (checked-staged-transcript-program A)
          adversary-initial-state)  $\wedge$ 
    Some (result, final-state)  $\in$ 
      set-dist (execute verify-monad ?s)  $\wedge$ 
    accepted-with-bound-tables ?s (Some (result, final-state))
      trace-table composition-table as query-idxs  $\wedge$ 
    fmlookup (HashMap final-state)
      (QueryIndexChallenge i
        (state-after-query-chunks
          (staged-query-start-hash data)
          (staged-query-chunks data) i)) =
      Some raw  $\wedge$ 
    index (to-nat raw)  $\in$ 
      query-sampling-success-space trace-table composition-table
        (staged-alphas data)  $\wedge$ 
    Some (((prefix, prefix-state), raw), raw-state)  $\in$ 
      set-dist
        (execute
          (checked-staged-query-prefix-receive-with-state A i)
          adversary-initial-state)  $\wedge$ 
    sqp-composition-fri-roots prefix  $\neq []$   $\wedge$ 
    created-tree trace-table trace-tree final-state  $\wedge$ 
    sqp-trace-root prefix = value trace-tree  $\wedge$ 
    created-tree composition-table composition-tree final-state  $\wedge$ 
    hd (sqp-composition-fri-roots prefix) =
      value composition-tree  $\wedge$ 
    hash-map-new-output-hit
      (set-tree trace-tree  $\cup$  set-tree composition-tree)
      prefix-state final-state

```

```

apply (rule-tac x=i in exI)
apply (rule-tac x=prefix in exI)
apply (rule-tac x=prefix-state in exI)
apply (rule-tac x=raw in exI)
apply (rule-tac x=raw-state in exI)
apply (rule-tac x=trace-table in exI)
apply (rule-tac x=composition-table in exI)
apply (rule-tac x=as in exI)
apply (rule-tac x=query-idxs in exI)
apply (rule-tac x=trace-tree in exI)
apply (rule-tac x=composition-tree in exI)
using i-bound builder verifier bound lookup-final-raw raw-hit-actual
  head-support comp-nonempty-prefix trace-created trace-root-prefix
  comp-created comp-root-prefix tree-hit
by blast
have tree-event:
  staged-security-with-data-state-query-tree-output-hit A out
using tree-event-body
unfolding out-eq
  staged-security-with-data-state-query-tree-output-hit-def Let-def
by simp
then show ?thesis by simp
qed
qed

```

lemma *checked-staged-security-with-data-state-query-bad-hit-at-bound-by-length-prefix-and-tree:*

```

assumes raw-bound: query-index-raw-preimage-bound
and wf: staged-budget-wellformed budgets
and controlled: staged-adversary-controlled budgets A
and i-bound: i < rounds

```

shows

```

wp-event (checked-staged-security-experiment-with-data-state A)
  (staged-security-with-data-state-query-bad-hit-at i)
adversary-initial-state ≤
  staged-phase-relation-error size
  (staged-query-search-queries budgets i + 1) +
  query-error-bound +
  hash-collision-budget-value 0
  (staged-query-search-queries budgets i + 1) +
wp-event (checked-staged-security-experiment-with-data-state A)
  (staged-security-with-data-state-query-tree-output-hit A)
  adversary-initial-state

```

proof –

```

let ?M = checked-staged-security-experiment-with-data-state A
let ?Head = checked-staged-query-prefix-receive-with-state A i
let ?Cont =
  checked-staged-after-query-prefix-receive-with-verifier A i
let ?Prefix =
  checked-staged-query-prefix-dynamic-index-hit

```

```

    staged-query-prefix-length-committed-query-target
let ?Tree = staged-security-with-data-state-query-tree-output-hit A
let ?Bad = staged-security-with-data-state-query-bad-hit-at i
let ?BadNoTree =  $\lambda out. ?Bad\ out \wedge \neg ?Tree\ out$ 
have split:
    wp-event ?M ?Bad adversary-initial-state  $\leq$ 
    wp-event ?M ?BadNoTree adversary-initial-state +
    wp-event ?M ?Tree adversary-initial-state
proof -
    have wp-event ?M ?Bad adversary-initial-state  $\leq$ 
    wp-event ?M ( $\lambda out. ?BadNoTree\ out \vee ?Tree\ out$ )
    adversary-initial-state
    by (rule wp-event-mono) blast
    also have ...  $\leq$ 
    wp-event ?M ?BadNoTree adversary-initial-state +
    wp-event ?M ?Tree adversary-initial-state
    by (rule wp-event-union-bound)
    finally show ?thesis .
qed
have prefix-bound:
    wp-event ?Head ?Prefix adversary-initial-state  $\leq$ 
    staged-phase-relation-error size
    (staged-query-search-queries budgets  $i + 1$ ) +
    query-error-bound +
    hash-collision-budget-value 0
    (staged-query-search-queries budgets  $i + 1$ )
    by (rule checked-staged-query-prefix-length-committed-target-hit-bound
    [OF raw-bound wf controlled])
    (use i-bound in simp)
have bad-no-tree-bound:
    wp-event ?M ?BadNoTree adversary-initial-state  $\leq$ 
    staged-phase-relation-error size
    (staged-query-search-queries budgets  $i + 1$ ) +
    query-error-bound +
    hash-collision-budget-value 0
    (staged-query-search-queries budgets  $i + 1$ )
unfolding
    checked-staged-security-experiment-with-data-state-query-prefix-receive-decomp
    [OF i-bound]
proof (rule wp-event-bind-bound-by-head-event[OF prefix-bound])
    show ?BadNoTree None  $\implies$  ?Prefix None
    unfolding staged-security-with-data-state-query-bad-hit-at-def
    checked-staged-query-prefix-dynamic-index-hit-def
    by simp
next
fix x t out
assume head:
    Some (x, t)  $\in$  set-dist (execute ?Head adversary-initial-state)
    and cont: out  $\in$  set-dist (execute (?Cont x) t)

```

```

    and bad-no-tree: ?BadNoTree out
  show ?Prefix (Some (x, t))
  proof (cases x)
    case x-eq: (Pair prefix-pack raw)
    then obtain prefix prefix-state where prefix-pack-eq:
      prefix-pack = (prefix, prefix-state)
      by (cases prefix-pack) simp
    have head':
      Some (((prefix, prefix-state), raw), t) ∈
        set-dist (execute ?Head adversary-initial-state)
    using head unfolding x-eq prefix-pack-eq .
    have cont':
      out ∈
        set-dist
          (execute
            (checked-staged-after-query-prefix-receive-with-verifier A i
              ((prefix, prefix-state), raw))
            t)
    using cont unfolding x-eq prefix-pack-eq .
  from checked-staged-after-query-prefix-receive-with-verifier-query-bad-at-imp-length-prefix-hit-or-tree-output
    [OF wf controlled i-bound head' cont']
  have ?Prefix (Some (((prefix, prefix-state), raw), t)) ∨
    ?Tree out
    using bad-no-tree by simp
  then show ?thesis
    using bad-no-tree unfolding x-eq prefix-pack-eq by simp
qed
qed
have wp-event ?M ?Bad adversary-initial-state ≤
  (staged-phase-relation-error size
    (staged-query-search-queries budgets i + 1) +
    query-error-bound +
    hash-collision-budget-value 0
    (staged-query-search-queries budgets i + 1)) +
  wp-event ?M ?Tree adversary-initial-state
  by (rule order-trans[OF split])
  (intro add-mono bad-no-tree-bound order-refl)
then show ?thesis
  by (simp add: algebra-simps)
qed

lemma checked-staged-security-with-data-state-query-bad-hit-at-no-tree-bound-by-length-prefix:
  assumes raw-bound: query-index-raw-preimage-bound
  and wf: staged-budget-wellformed budgets
  and controlled: staged-adversary-controlled budgets A
  and i-bound: i < rounds
  shows
    wp-event (checked-staged-security-experiment-with-data-state A)
      (λout.

```

```

    staged-security-with-data-state-query-bad-hit-at  $i$  out  $\wedge$ 
     $\neg$  staged-security-with-data-state-query-tree-output-hit  $A$  out)
  adversary-initial-state  $\leq$ 
  staged-phase-relation-error size
    (staged-query-search-queries budgets  $i + 1$ ) +
  query-error-bound +
  hash-collision-budget-value 0
    (staged-query-search-queries budgets  $i + 1$ )
proof –
  let ?Head = checked-staged-query-prefix-receive-with-state  $A$   $i$ 
  let ?Cont =
    checked-staged-after-query-prefix-receive-with-verifier  $A$   $i$ 
  let ?Prefix =
    checked-staged-query-prefix-dynamic-index-hit
    staged-query-prefix-length-committed-query-target
  let ?Tree = staged-security-with-data-state-query-tree-output-hit  $A$ 
  let ?Bad = staged-security-with-data-state-query-bad-hit-at  $i$ 
  let ?BadNoTree =  $\lambda$ out. ?Bad out  $\wedge \neg$  ?Tree out
  have prefix-bound:
    wp-event ?Head ?Prefix adversary-initial-state  $\leq$ 
    staged-phase-relation-error size
      (staged-query-search-queries budgets  $i + 1$ ) +
    query-error-bound +
    hash-collision-budget-value 0
      (staged-query-search-queries budgets  $i + 1$ )
  by (rule checked-staged-query-prefix-length-committed-target-hit-bound
    [OF raw-bound wf controlled])
    (use i-bound in simp)
show ?thesis
unfolding
  checked-staged-security-experiment-with-data-state-query-prefix-receive-decomp
  [OF i-bound]
proof (rule wp-event-bind-bound-by-head-event[OF prefix-bound])
show ?BadNoTree None  $\implies$  ?Prefix None
unfolding staged-security-with-data-state-query-bad-hit-at-def
  checked-staged-query-prefix-dynamic-index-hit-def
by simp
next
fix  $x$   $t$  out
assume head:
  Some  $(x, t) \in \text{set-dist (execute ?Head adversary-initial-state)}$ 
and cont: out  $\in \text{set-dist (execute (?Cont } x) t)}$ 
and bad-no-tree: ?BadNoTree out
show ?Prefix (Some  $(x, t)$ )
proof (cases  $x$ )
case  $x$ -eq: (Pair prefix-pack raw)
then obtain prefix prefix-state where prefix-pack-eq:
  prefix-pack = (prefix, prefix-state)
by (cases prefix-pack) simp

```

```

have head':
  Some (((prefix, prefix-state), raw), t) ∈
    set-dist (execute ?Head adversary-initial-state)
  using head unfolding x-eq prefix-pack-eq .
have cont':
  out ∈
    set-dist
      (execute
        (checked-staged-after-query-prefix-receive-with-verifier A i
          ((prefix, prefix-state), raw))
        t)
  using cont unfolding x-eq prefix-pack-eq .
from checked-staged-after-query-prefix-receive-with-verifier-query-bad-at-imp-length-prefix-hit-or-tree-output
  [OF wf controlled i-bound head' cont']
have ?Prefix (Some (((prefix, prefix-state), raw), t)) ∨
  ?Tree out
  using bad-no-tree by simp
then show ?thesis
  using bad-no-tree unfolding x-eq prefix-pack-eq by simp
qed
qed
qed

```

lemma *checked-staged-security-with-data-state-query-bad-hit-bound-by-length-prefixes-and-tree*:

assumes *raw-bound: query-index-raw-preimage-bound*

and *wf: staged-budget-wellformed budgets*

and *controlled: staged-adversary-controlled budgets A*

shows

wp-event (*checked-staged-security-experiment-with-data-state A*)
staged-security-with-data-state-query-bad-hit
adversary-initial-state ≤
 ($\sum i < \text{rounds.}$
 staged-phase-relation-error size
 (*staged-query-search-queries budgets i + 1*) +
 query-error-bound +
 hash-collision-budget-value 0
 (*staged-query-search-queries budgets i + 1*)) +
wp-event (*checked-staged-security-experiment-with-data-state A*)
 (*staged-security-with-data-state-query-tree-output-hit A*)
adversary-initial-state

proof –

let ?*M* = *checked-staged-security-experiment-with-data-state A*

let ?*Tree* = *staged-security-with-data-state-query-tree-output-hit A*

let ?*NoTree* =

λi *out.*

staged-security-with-data-state-query-bad-hit-at i out ∧

 ¬ ?*Tree out*

let ?*E* =

$\lambda i.$

staged-phase-relation-error size
(staged-query-search-queries budgets $i + 1$) +
query-error-bound +
hash-collision-budget-value 0
(staged-query-search-queries budgets $i + 1$)
have *split*:
wp-event ?M staged-security-with-data-state-query-bad-hit
adversary-initial-state $\leq$
wp-event ?M ($\lambda out. \exists i \in \{..<rounds\}. ?NoTree i out$)
adversary-initial-state +
wp-event ?M ?Tree adversary-initial-state
proof –
have *wp-event ?M staged-security-with-data-state-query-bad-hit*
adversary-initial-state $\leq$
wp-event ?M
($\lambda out. (\exists i \in \{..<rounds\}. ?NoTree i out) \vee ?Tree out$)
adversary-initial-state
proof (rule *wp-event-mono*)
fix *out*
assume *bad: staged-security-with-data-state-query-bad-hit out*
from *staged-security-with-data-state-query-bad-hit-imp-round-hit[OF bad]*
obtain *i where i-bound: $i \in \{..<rounds\}$*
and *hit: staged-security-with-data-state-query-bad-hit-at i out*
by *blast*
show *($\exists i \in \{..<rounds\}. ?NoTree i out$) $\vee$?Tree out*
proof (cases ?Tree out)
case *True*
then show *?thesis* **by** *simp*
next
case *False*
then show *?thesis*
using *i-bound hit* **by** *blast*
qed
qed
also have *... $\leq$*
wp-event ?M ($\lambda out. \exists i \in \{..<rounds\}. ?NoTree i out$)
adversary-initial-state +
wp-event ?M ?Tree adversary-initial-state
by (rule *wp-event-union-bound*)
finally show *?thesis* .
qed
have *no-tree-bound*:
wp-event ?M ($\lambda out. \exists i \in \{..<rounds\}. ?NoTree i out$)
adversary-initial-state $\leq$ ($\sum i < rounds. ?E i$)
proof (rule *wp-event-finite-union-bound*)
show *finite $\{..<rounds\}$*
by *simp*
next
fix *i*

assume $i\text{-bound}: i \in \{..<\text{rounds}\}$
show $wp\text{-event } ?M \text{ } (?NoTree \ i) \text{ adversary-initial-state} \leq ?E \ i$
by (rule
checked-staged-security-with-data-state-query-bad-hit-at-no-tree-bound-by-length-prefix
 $[OF \text{ raw-bound } wf \text{ controlled}]$)
 (use $i\text{-bound}$ **in** *simp*)
qed
have $wp\text{-event } ?M \text{ staged-security-with-data-state-query-bad-hit}$
 $\text{adversary-initial-state} \leq$
 $(\sum i<\text{rounds}. ?E \ i) + wp\text{-event } ?M \text{ } ?Tree \text{ adversary-initial-state}$
by (rule *order-trans* $[OF \text{ split}]$)
 (intro *add-mono no-tree-bound order-refl*)
then show $?thesis$.
qed

lemma *checked-staged-security-with-data-state-query-bad-hit-bound-by-length-prefixes-and-tree-from-budgets:*
assumes $wf: \text{staged-budget-wellformed budgets}$
and $\text{controlled}: \text{staged-adversary-controlled budgets } A$
shows
 $wp\text{-event } (\text{checked-staged-security-experiment-with-data-state } A)$
 $\text{staged-security-with-data-state-query-bad-hit}$
 $\text{adversary-initial-state} \leq$
 $(\sum i<\text{rounds}.$
 $\text{staged-phase-relation-error size}$
 $(\text{staged-query-search-queries budgets } i + 1) +$
 $\text{query-error-bound} +$
 $\text{hash-collision-budget-value } 0$
 $(\text{staged-query-search-queries budgets } i + 1)) +$
 $wp\text{-event } (\text{checked-staged-security-experiment-with-data-state } A)$
 $(\text{staged-security-with-data-state-query-tree-output-hit } A)$
 $\text{adversary-initial-state}$

proof –
have $\text{raw-bound}: \text{query-index-raw-preimage-bound}$
by (rule *query-index-raw-preimage-bound-from-sampler-wellformed*)
show $?thesis$
by (rule
checked-staged-security-with-data-state-query-bad-hit-bound-by-length-prefixes-and-tree
 $[OF \text{ raw-bound } wf \text{ controlled}]$)
qed

lemma *checked-staged-security-with-data-state-query-bad-verifier-event-bound-by-length-prefixes-and-tree:*
assumes $\text{raw-bound}: \text{query-index-raw-preimage-bound}$
and $wf: \text{staged-budget-wellformed budgets}$
and $\text{controlled}: \text{staged-adversary-controlled budgets } A$
shows
 $wp\text{-event } (\text{checked-staged-security-experiment-with-data-state } A)$
 $(\text{staged-security-with-data-state-verifier-event query-bad})$
 $\text{adversary-initial-state} \leq$
 $(\sum i<\text{rounds}.$

staged-phase-relation-error size
(staged-query-search-queries budgets $i + 1$) +
query-error-bound +
hash-collision-budget-value 0
(staged-query-search-queries budgets $i + 1$)) +
wp-event (checked-staged-security-experiment-with-data-state A)
(staged-security-with-data-state-query-tree-output-hit A)
adversary-initial-state
proof –
have *wp-event (checked-staged-security-experiment-with-data-state A)*
(staged-security-with-data-state-verifier-event query-bad)
adversary-initial-state $\leq$
wp-event (checked-staged-security-experiment-with-data-state A)
staged-security-with-data-state-query-bad-hit
adversary-initial-state
by (rule *checked-staged-security-with-data-state-query-bad-verifier-event-bound*)
simp
also have ... $\leq$
 $(\sum i < \text{rounds.}$
staged-phase-relation-error size
(staged-query-search-queries budgets $i + 1$) +
query-error-bound +
hash-collision-budget-value 0
(staged-query-search-queries budgets $i + 1$)) +
wp-event (checked-staged-security-experiment-with-data-state A)
(staged-security-with-data-state-query-tree-output-hit A)
adversary-initial-state
by (rule
checked-staged-security-with-data-state-query-bad-hit-bound-by-length-prefixes-and-tree
[OF raw-bound wf controlled])
finally show ?thesis .
qed

lemma *checked-staged-security-with-data-state-query-bad-verifier-event-bound-by-length-prefixes-and-tree-from-budget*
assumes *wf: staged-budget-wellformed budgets*
and *controlled: staged-adversary-controlled budgets A*
shows
wp-event (checked-staged-security-experiment-with-data-state A)
(staged-security-with-data-state-verifier-event query-bad)
adversary-initial-state $\leq$
 $(\sum i < \text{rounds.}$
staged-phase-relation-error size
(staged-query-search-queries budgets $i + 1$) +
query-error-bound +
hash-collision-budget-value 0
(staged-query-search-queries budgets $i + 1$)) +
wp-event (checked-staged-security-experiment-with-data-state A)
(staged-security-with-data-state-query-tree-output-hit A)
adversary-initial-state

proof –

- have** *raw-bound*: *query-index-raw-preimage-bound*
- by** (rule *query-index-raw-preimage-bound-from-sampler-wellformed*)
- show** *?thesis*
- by** (rule *checked-staged-security-with-data-state-query-bad-verifier-event-bound-by-length-prefixes-and-tree* [OF *raw-bound* *wf* *controlled*])

qed

lemma *checked-staged-transcript-query-partial-header-key-hit-at-bound-from-prefix-receive*:

- assumes** *wf*: *staged-budget-wellformed* *budgets*
- and** *controlled*: *staged-adversary-controlled* *budgets* *A*
- and** *i-bound*: $i < \text{rounds}$
- and** *prefix-bound*:
- wp-event* (*checked-staged-query-prefix-receive-with-state* *A* *i*)
- (*checked-staged-query-prefix-dynamic-index-hit*
- (*staged-query-prefix-partial-query-target* *i*))
- adversary-initial-state* $\leq C$
- shows**
- wp-event* (*checked-staged-transcript-program* *A*)
- (*staged-transcript-query-partial-header-key-hit-at* *i*)
- adversary-initial-state* $\leq C$
- unfolding** *checked-staged-transcript-program-query-prefix-receive-decomp*
- [OF *i-bound*]

proof (rule *wp-event-bind-bound-by-head-event*[OF *prefix-bound*])

show

- staged-transcript-query-partial-header-key-hit-at* *i* *None* $\implies$
- checked-staged-query-prefix-dynamic-index-hit*
- (*staged-query-prefix-partial-query-target* *i*) *None*
- unfolding** *staged-transcript-query-partial-header-key-hit-at-def*
- checked-staged-query-prefix-dynamic-index-hit-def*
- by** *simp*

next

- fix** *x t out*
- assume** *head*:
- Some* (*x, t*) $\in$
- set-dist*
- (*execute* (*checked-staged-query-prefix-receive-with-state* *A* *i*)
- adversary-initial-state*)
- and** *cont*:
- out* $\in$
- set-dist*
- (*execute* (*checked-staged-after-query-prefix-receive* *A* *i* *x*) *t*)
- and** *hit*: *staged-transcript-query-partial-header-key-hit-at* *i* *out*

show

- checked-staged-query-prefix-dynamic-index-hit*
- (*staged-query-prefix-partial-query-target* *i*) (*Some* (*x, t*))

proof (*cases* *x*)

- case** *x-eq*: (*Pair* *prefix-pack* *raw*)

```

then obtain prefix prefix-state where prefix-pack-eq:
  prefix-pack = (prefix, prefix-state)
  by (cases prefix-pack) simp
show ?thesis
proof (cases out)
  case None
  then show ?thesis
    using hit
    unfolding staged-transcript-query-partial-header-key-hit-at-def
    by simp
  next
  case (Some packed)
  then obtain data attacker-state where out-eq:
    out = Some (data, attacker-state)
    by (cases packed) simp
  have head':
    Some (((prefix, prefix-state), raw), t) ∈
      set-dist
        (execute (checked-staged-query-prefix-receive-with-state A i)
          adversary-initial-state)
    using head unfolding x-eq prefix-pack-eq .
  have cont':
    Some (data, attacker-state) ∈
      set-dist
        (execute
          (checked-staged-after-query-prefix-receive A i
            ((prefix, prefix-state), raw))
          t)
    using cont unfolding x-eq prefix-pack-eq out-eq .
  have hit':
    staged-transcript-query-partial-header-key-hit-at i
      (Some (data, attacker-state))
    using hit unfolding out-eq .
  have prefix-hit:
    checked-staged-query-prefix-dynamic-index-hit
      (staged-query-prefix-partial-query-target i)
      (Some (((prefix, prefix-state), raw), t))
    by (rule
      checked-staged-after-query-prefix-receive-partial-hit-imp-prefix-hit
      [OF wf controlled i-bound head' cont' hit'])
  show ?thesis
    using prefix-hit unfolding x-eq prefix-pack-eq .
qed
qed
qed

lemma checked-staged-transcript-query-bound-table-hit-at-committed-prefix-receive-support:
  assumes wf: staged-budget-wellformed budgets
  and controlled: staged-adversary-controlled budgets A

```

and *outcome*:
Some (data, attacker-state) ∈
set-dist
(execute (checked-staged-transcript-program A)
adversary-initial-state)
and *i-bound*: *i < rounds*
and *lookup-attacker*:
fmlookup (HashMap attacker-state)
(QueryIndexChallenge i
(state-after-query-chunks
(staged-query-start-hash data) (staged-query-chunks data) i)) =
Some raw-hit
and *raw-hit-target*:
index (to-nat raw-hit) ∈
query-sampling-success-space trace-table composition-table
(staged-alphas data)
and *comp-nonempty*:
staged-composition-fri-roots data ≠ []
shows
 $\exists$ *prefix prefix-state raw raw-state.*
Some (((prefix, prefix-state), raw), raw-state) ∈
set-dist
(execute (checked-staged-query-prefix-receive-with-state A i)
adversary-initial-state) ∧
(merkle-root-binds-table (sqp-trace-root prefix) trace-table
prefix-state →
merkle-root-binds-table (hd (sqp-composition-fri-roots prefix))
composition-table prefix-state →
checked-staged-query-prefix-dynamic-index-hit
(λprefix prefix-state.
staged-query-prefix-committed-query-target prefix prefix-state)
(Some (((prefix, prefix-state), raw), raw-state)))
proof –
from *checked-staged-transcript-program-query-prefix-receive-compact-support*
[OF wf controlled i-bound outcome]
obtain *prefix prefix-state raw raw-state where*
prefix-receive:
Some (((prefix, prefix-state), raw), raw-state) ∈
set-dist
(execute (checked-staged-query-prefix-receive-with-state A i)
adversary-initial-state)
and *root-eq*: *sqp-trace-root prefix = staged-trace-root data*
and *alphas-eq*: *sqp-alphas prefix = staged-alphas data*
and *comp-roots-eq*:
sqp-composition-fri-roots prefix =
staged-composition-fri-roots data
and *prefix-ext*: *prefix-state ≤ attacker-state*
and *raw-ext*: *raw-state ≤ attacker-state*
and *lookup-raw*:

```

    fmlookup (HashMap raw-state)
      (QueryIndexChallenge i
        (state-after-query-chunks
          (staged-query-start-hash data) (staged-query-chunks data) i)) =
        Some raw
  by (rule checked-staged-transcript-program-query-prefix-receive-compact-support
    [OF wf controlled i-bound outcome])
  have lookup-attacker-raw:
    fmlookup (HashMap attacker-state)
      (QueryIndexChallenge i
        (state-after-query-chunks
          (staged-query-start-hash data) (staged-query-chunks data) i)) =
        Some raw
  by (rule hash-extension-lookup[OF lookup-raw raw-ext])
  have raw-eq: raw = raw-hit
  using lookup-attacker lookup-attacker-raw by simp
  have comp-nonempty-prefix:
    sqp-composition-fri-roots prefix ≠ []
  using comp-nonempty comp-roots-eq by simp
  have hit-if-bound:
    merkle-root-binds-table (sqp-trace-root prefix) trace-table
      prefix-state ⇒
    merkle-root-binds-table (hd (sqp-composition-fri-roots prefix))
      composition-table prefix-state ⇒
    checked-staged-query-prefix-dynamic-index-hit
      (λprefix prefix-state.
        staged-query-prefix-committed-query-target prefix prefix-state)
      (Some (((prefix, prefix-state), raw), raw-state))
  proof -
    assume trace-bind:
      merkle-root-binds-table (sqp-trace-root prefix) trace-table
        prefix-state
    assume comp-bind:
      merkle-root-binds-table (hd (sqp-composition-fri-roots prefix))
        composition-table prefix-state
    have candidate:
      (trace-table, composition-table) ∈
        query-header-committed-table-candidates prefix-state
          (sqp-trace-root prefix)
          (sqp-composition-fri-roots prefix)
    unfolding query-header-committed-table-candidates-def
    using trace-bind comp-nonempty-prefix comp-bind by simp
    have raw-target-prefix:
      index (to-nat raw) ∈
        query-sampling-success-space trace-table composition-table
          (sqp-alphas prefix)
    using raw-hit-target unfolding raw-eq alphas-eq .
    have target:
      index (to-nat raw) ∈

```

```

    staged-query-prefix-committed-query-target prefix prefix-state
  by (rule staged-query-prefix-committed-query-targetI
    [OF candidate raw-target-prefix])
show ?thesis
  unfolding checked-staged-query-prefix-dynamic-index-hit-def
  using target by simp
qed
show ?thesis
  using prefix-receive hit-if-bound by blast
qed

lemma checked-staged-transcript-query-bound-table-hit-at-length-committed-prefix-receive-support:
  assumes wf: staged-budget-wellformed budgets
  and controlled: staged-adversary-controlled budgets A
  and outcome:
    Some (data, attacker-state) ∈
      set-dist
        (execute (checked-staged-transcript-program A)
          adversary-initial-state)
  and i-bound: i < rounds
  and lookup-attacker:
    fmlookup (HashMap attacker-state)
      (QueryIndexChallenge i
        (state-after-query-chunks
          (staged-query-start-hash data) (staged-query-chunks data) i)) =
      Some raw-hit
  and raw-hit-target:
    index (to-nat raw-hit) ∈
      query-sampling-success-space trace-table composition-table
        (staged-alphas data)
  and comp-nonempty:
    staged-composition-fri-roots data ≠ []
  and trace-len: length trace-table = clength * scale
  and comp-len: length composition-table = clength * scale
shows
  ∃ prefix prefix-state raw raw-state.
    Some (((prefix, prefix-state), raw), raw-state) ∈
      set-dist
        (execute (checked-staged-query-prefix-receive-with-state A i)
          adversary-initial-state) ∧
    (merkle-root-binds-table (sqp-trace-root prefix) trace-table
      prefix-state →
      merkle-root-binds-table (hd (sqp-composition-fri-roots prefix))
        composition-table prefix-state →
      checked-staged-query-prefix-dynamic-index-hit
        staged-query-prefix-length-committed-query-target
        (Some (((prefix, prefix-state), raw), raw-state)))
proof -
  from checked-staged-transcript-program-query-prefix-receive-compact-support

```

[OF wf controlled i-bound outcome]

obtain *prefix prefix-state raw raw-state* **where**

prefix-receive:

Some (((*prefix, prefix-state*), *raw*), *raw-state*) ∈
set-dist
 (execute (*checked-staged-query-prefix-receive-with-state A i*)
adversary-initial-state)

and *root-eq: sqp-trace-root prefix = staged-trace-root data*

and *alphas-eq: sqp-alphas prefix = staged-alphas data*

and *comp-roots-eq:*

sqp-composition-fri-roots prefix =
staged-composition-fri-roots data

and *prefix-ext: prefix-state ≤ attacker-state*

and *raw-ext: raw-state ≤ attacker-state*

and *lookup-raw:*

fmlookup (HashMap raw-state)
 (*QueryIndexChallenge i*
 (*state-after-query-chunks*
 (*staged-query-start-hash data*) (*staged-query-chunks data*) *i*)) =
 Some *raw*

by (rule *checked-staged-transcript-program-query-prefix-receive-compact-support*
 [OF wf controlled i-bound outcome])

have *lookup-attacker-raw:*

fmlookup (HashMap attacker-state)
 (*QueryIndexChallenge i*
 (*state-after-query-chunks*
 (*staged-query-start-hash data*) (*staged-query-chunks data*) *i*)) =
 Some *raw*

by (rule *hash-extension-lookup*[OF *lookup-raw raw-ext*])

have *raw-eq: raw = raw-hit*

using *lookup-attacker lookup-attacker-raw* **by** *simp*

have *comp-nonempty-prefix:*

sqp-composition-fri-roots prefix ≠ []

using *comp-nonempty comp-roots-eq* **by** *simp*

have *hit-if-bound:*

merkle-root-binds-table (sqp-trace-root prefix) trace-table
prefix-state ⇒
merkle-root-binds-table (hd (sqp-composition-fri-roots prefix))
composition-table prefix-state ⇒
checked-staged-query-prefix-dynamic-index-hit
staged-query-prefix-length-committed-query-target
 (Some (((*prefix, prefix-state*), *raw*), *raw-state*))

proof –

assume *trace-bind:*

merkle-root-binds-table (sqp-trace-root prefix) trace-table
prefix-state

assume *comp-bind:*

merkle-root-binds-table (hd (sqp-composition-fri-roots prefix))
composition-table prefix-state


```

have candidate:
  (trace-table, composition-table) ∈
    staged-query-prefix-length-committed-table-candidates prefix
    prefix-state
unfolding staged-query-prefix-length-committed-table-candidates-def
using trace-bind comp-nonempty-prefix comp-bind trace-len comp-len
by simp
have raw-target-prefix:
  index (to-nat raw) ∈
    query-sampling-success-space trace-table composition-table
    (sqp-alphas prefix)
using raw-hit-target unfolding raw-eq alphas-eq .
have target:
  index (to-nat raw) ∈
    staged-query-prefix-length-committed-query-target prefix
    prefix-state
by (rule staged-query-prefix-length-committed-query-targetI
    [OF candidate raw-target-prefix])
show ?thesis
unfolding checked-staged-query-prefix-dynamic-index-hit-def
using target by simp
qed
show ?thesis
using prefix-receive hit-if-bound by blast
qed

lemma checked-staged-bound-tables-initial-roots-late-output-at-query-prefix-support:
assumes wf: staged-budget-wellformed budgets
and controlled: staged-adversary-controlled budgets A
and outcome:
  Some (data, attacker-state) ∈
    set-dist
    (execute (checked-staged-transcript-program A)
    adversary-initial-state)
and verifier:
  Some (result, final-state) ∈
    set-dist
    (execute verify-monad
    (verifier-state-from-adversary attacker-state
    (staged-proof-transcript data))))
and i-bound: i < rounds
and bound:
  accepted-with-bound-tables
  (verifier-state-from-adversary attacker-state
  (staged-proof-transcript data))
  (Some (result, final-state))
  trace-table composition-table as query-idxs
shows
  ∃ prefix prefix-state raw raw-state.

```

```

Some (((prefix, prefix-state), raw), raw-state) ∈
  set-dist
  (execute (checked-staged-query-prefix-receive-with-state A i)
    adversary-initial-state) ∧
sqp-composition-fri-roots prefix ≠ [] ∧
merkle-root-binds-table (sqp-trace-root prefix) trace-table
  final-state ∧
merkle-root-binds-table (hd (sqp-composition-fri-roots prefix))
  composition-table final-state ∧
(((∃ input.
  fmlookup (HashMap prefix-state) input =
    Some (sqp-trace-root prefix)) ∧
(∃ input.
  fmlookup (HashMap prefix-state) input =
    Some (hd (sqp-composition-fri-roots prefix)))) ∨
hash-map-new-output-hit
  {sqp-trace-root prefix, hd (sqp-composition-fri-roots prefix)}
  prefix-state final-state)
proof —
let ?s =
  verifier-state-from-adversary attacker-state
  (staged-proof-transcript data)
from checked-staged-transcript-program-query-prefix-receive-compact-support
  [OF wf controlled i-bound outcome]
obtain prefix prefix-state raw raw-state where
  prefix-receive:
    Some (((prefix, prefix-state), raw), raw-state) ∈
      set-dist
      (execute (checked-staged-query-prefix-receive-with-state A i)
        adversary-initial-state)
and root-eq: sqp-trace-root prefix = staged-trace-root data
and comp-roots-eq:
  sqp-composition-fri-roots prefix =
    staged-composition-fri-roots data
and prefix-ext: prefix-state ≤ attacker-state
and raw-ext: raw-state ≤ attacker-state
and lookup-raw:
  fmlookup (HashMap raw-state)
    (QueryIndexChallenge i
      (state-after-query-chunks
        (staged-query-start-hash data) (staged-query-chunks data) i)) =
    Some raw
by (rule checked-staged-transcript-program-query-prefix-receive-compact-support
  [OF wf controlled i-bound outcome])
have prefix-ext-s: prefix-state ≤ ?s
by (rule hash-extends-verifier-state-from-adversary-right
  [OF prefix-ext])
have s-ext-final: ?s ≤ final-state
by (rule verify-monad-hash-extends[OF verifier])

```

```

have prefix-ext-final: prefix-state ≤ final-state
  by (rule hash-ext-trans[OF prefix-ext-s s-ext-final])
have staged-header:
  verifier-header-transcript ?s
    (staged-trace-root data)
    (staged-trace-fri-roots data)
    (staged-trace-final data)
    (staged-alphas data)
    (staged-degree data)
    (staged-composition-fri-roots data)
    (staged-composition-final data)
    (List.concat (staged-query-chunks data))
  by (rule checked-staged-transcript-program-outcome-header-transcript
    [OF wf controlled outcome])
from accepted-with-bound-tables-initial-roots-late-output-preexisting-or-union-hit
  [OF prefix-ext-final bound]
obtain fr f-fri-roots f-final dg composition-fri-roots final rest where
  header:
    verifier-header-transcript ?s fr f-fri-roots f-final as dg
    composition-fri-roots final rest
and comp-nonempty: composition-fri-roots ≠ []
and trace-bind:
  merkle-root-binds-table fr trace-table final-state
and comp-bind:
  merkle-root-binds-table (hd composition-fri-roots) composition-table
    final-state
and late:
  ((∃ input. fmlookup (HashMap prefix-state) input = Some fr) ∧
    (∃ input.
      fmlookup (HashMap prefix-state) input =
        Some (hd composition-fri-roots))) ∨
    hash-map-new-output-hit {fr, hd composition-fri-roots}
      prefix-state final-state
by blast
have eqs:
  fr = staged-trace-root data ∧
  f-fri-roots = staged-trace-fri-roots data ∧
  f-final = staged-trace-final data ∧
  as = staged-alphas data ∧
  dg = staged-degree data ∧
  composition-fri-roots = staged-composition-fri-roots data ∧
  final = staged-composition-final data ∧
  rest = List.concat (staged-query-chunks data)
using verifier-header-transcript-unique[OF header staged-header]
by simp
have comp-nonempty-prefix: sqp-composition-fri-roots prefix ≠ []
  using comp-nonempty comp-roots-eq eqs by simp
have trace-bind-prefix:
  merkle-root-binds-table (sqp-trace-root prefix) trace-table

```

```

    final-state
  using trace-bind root-eq eqs by simp
have comp-bind-prefix:
  merkle-root-binds-table (hd (sqp-composition-fri-roots prefix))
    composition-table final-state
  using comp-bind comp-roots-eq eqs by simp
have late-prefix:
  (( $\exists$  input.
    fmlookup (HashMap prefix-state) input =
      Some (sqp-trace-root prefix))  $\wedge$ 
    ( $\exists$  input.
      fmlookup (HashMap prefix-state) input =
        Some (hd (sqp-composition-fri-roots prefix))))  $\vee$ 
    hash-map-new-output-hit
    {sqp-trace-root prefix, hd (sqp-composition-fri-roots prefix)}
    prefix-state final-state
  using late root-eq comp-roots-eq eqs by simp
show ?thesis
  apply (rule-tac x=prefix in exI)
  apply (rule-tac x=prefix-state in exI)
  apply (rule-tac x=raw in exI)
  apply (rule-tac x=raw-state in exI)
  using prefix-receive comp-nonempty-prefix trace-bind-prefix
    comp-bind-prefix late-prefix
  by blast
qed

lemma checked-staged-bound-tables-initial-roots-pullback-at-query-prefix-support:
  assumes wf: staged-budget-wellformed budgets
  and controlled: staged-adversary-controlled budgets A
  and outcome:
    Some (data, attacker-state)  $\in$ 
      set-dist
        (execute (checked-staged-transcript-program A)
          adversary-initial-state)
  and verifier:
    Some (result, final-state)  $\in$ 
      set-dist
        (execute verify-monad
          (verifier-state-from-adversary attacker-state
            (staged-proof-transcript data)))
  and i-bound:  $i < \text{rounds}$ 
  and bound:
    accepted-with-bound-tables
      (verifier-state-from-adversary attacker-state
        (staged-proof-transcript data))
      (Some (result, final-state))
    trace-table composition-table as query-idxs
  and query-hit:

```

query-idxs ! i ∈
query-sampling-success-space trace-table composition-table as
shows
 $\exists$ *prefix prefix-state raw raw-state trace-tree composition-tree.*
Some (((prefix, prefix-state), raw), raw-state) ∈
set-dist
(execute (checked-staged-query-prefix-receive-with-state A i)
adversary-initial-state) ∧
sqp-alphas prefix = staged-alphas data ∧
fmlookup (HashMap final-state)
(QueryIndexChallenge i
(state-after-query-chunks
(staged-query-start-hash data) (staged-query-chunks data) i)) =
Some raw ∧
index (to-nat raw) ∈
query-sampling-success-space trace-table composition-table
(staged-alphas data) ∧
sqp-composition-fri-roots prefix ≠ [] ∧
created-tree trace-table trace-tree final-state ∧
sqp-trace-root prefix = value trace-tree ∧
created-tree composition-table composition-tree final-state ∧
hd (sqp-composition-fri-roots prefix) = value composition-tree ∧
((merkle-root-binds-table (sqp-trace-root prefix) trace-table
prefix-state ∧
merkle-root-binds-table (hd (sqp-composition-fri-roots prefix))
composition-table prefix-state) ∨
hash-map-new-output-hit
(set-tree trace-tree ∪ set-tree composition-tree)
prefix-state final-state)
proof –
let *?s =*
verifier-state-from-adversary attacker-state
(staged-proof-transcript data)
from *checked-staged-transcript-program-query-prefix-receive-compact-support*
[OF wf controlled i-bound outcome]
obtain *prefix prefix-state raw raw-state where*
prefix-receive:
Some (((prefix, prefix-state), raw), raw-state) ∈
set-dist
(execute (checked-staged-query-prefix-receive-with-state A i)
adversary-initial-state)
and *root-eq: sqp-trace-root prefix = staged-trace-root data*
and *alphas-eq: sqp-alphas prefix = staged-alphas data*
and *comp-roots-eq:*
sqp-composition-fri-roots prefix =
staged-composition-fri-roots data
and *prefix-ext: prefix-state ≤ attacker-state*
and *raw-ext: raw-state ≤ attacker-state*
and *lookup-raw:*

```

    fmlookup (HashMap raw-state)
      (QueryIndexChallenge i
        (state-after-query-chunks
          (staged-query-start-hash data) (staged-query-chunks data) i)) =
        Some raw
  by (rule checked-staged-transcript-program-query-prefix-receive-compact-support
    [OF wf controlled i-bound outcome])
  have prefix-ext-s: prefix-state ≤ ?s
  by (rule hash-extends-verifier-state-from-adversary-right
    [OF prefix-ext])
  have s-ext-final: ?s ≤ final-state
  by (rule verify-monad-hash-extends[OF verifier])
  have prefix-ext-final: prefix-state ≤ final-state
  by (rule hash-ext-trans[OF prefix-ext-s s-ext-final])
  have raw-ext-s: raw-state ≤ ?s
  by (rule hash-extends-verifier-state-from-adversary-right
    [OF raw-ext])
  have raw-ext-final: raw-state ≤ final-state
  by (rule hash-ext-trans[OF raw-ext-s s-ext-final])
  have lookup-final-raw:
    fmlookup (HashMap final-state)
      (QueryIndexChallenge i
        (state-after-query-chunks
          (staged-query-start-hash data) (staged-query-chunks data) i)) =
        Some raw
  by (rule hash-extension-lookup[OF lookup-raw raw-ext-final])
  have staged-header:
    verifier-header-transcript ?s
      (staged-trace-root data)
      (staged-trace-fri-roots data)
      (staged-trace-final data)
      (staged-alphas data)
      (staged-degree data)
      (staged-composition-fri-roots data)
      (staged-composition-final data)
      (List.concat (staged-query-chunks data))
  by (rule checked-staged-transcript-program-outcome-header-transcript
    [OF wf controlled outcome])
  from accepted-with-bound-tables-initial-roots-pullback-or-new-tree-output-hit
    [OF prefix-ext-final bound]
  obtain fr f-fri-roots f-final dg composition-fri-roots final rest
    trace-tree composition-tree where
    header:
      verifier-header-transcript ?s fr f-fri-roots f-final as dg
      composition-fri-roots final rest
  and comp-nonempty: composition-fri-roots ≠ []
  and trace-created:
    created-tree trace-table trace-tree final-state
  and trace-root: fr = value trace-tree

```

and *comp-created*:
created-tree composition-table composition-tree final-state
and *comp-root*:
hd composition-fri-roots = value composition-tree
and *split*:
(merkle-root-binds-table fr trace-table prefix-state ∧
merkle-root-binds-table (hd composition-fri-roots)
composition-table prefix-state) ∨
hash-map-new-output-hit
(set-tree trace-tree ∪ set-tree composition-tree)
prefix-state final-state
by *blast*
have *eqs*:
fr = staged-trace-root data ∧
f-fri-roots = staged-trace-fri-roots data ∧
f-final = staged-trace-final data ∧
as = staged-alphas data ∧
dg = staged-degree data ∧
composition-fri-roots = staged-composition-fri-roots data ∧
final = staged-composition-final data ∧
rest = List.concat (staged-query-chunks data)
using *verifier-header-transcript-unique*[*OF header staged-header*]
by *simp*
have *support-some*:
Some (((data, attacker-state), result), final-state) ∈
set-dist
(execute
(checked-staged-security-experiment-with-data-state A)
adversary-initial-state)
by (*rule checked-staged-security-experiment-with-data-state-outcomeI*
[*OF outcome verifier*])
have *shape*:
accepted-transcript-shape ?s (Some (result, final-state))
as query-idxs
using *bound*
unfolding *accepted-with-bound-tables-def accepted-with-tables-def*
by *simp*
from *checked-staged-security-with-data-state-accepted-shape-query-keys*
[*OF wf controlled support-some shape*]
obtain *final-raw-idxs* **where**
as-eq: as = staged-alphas data
and *len-final-raw: length final-raw-idxs = rounds*
and *query-idxs-eq*:
query-idxs = map (λraw. index (to-nat raw)) final-raw-idxs
and *lookup-final*:
 $\bigwedge j. j < \text{rounds} \implies$
fmlookup (HashMap final-state)
(QueryIndexChallenge j
(state-after-query-chunks

```

      (staged-query-start-hash data)
      (staged-query-chunks data) j)) =
    Some (final-raw-idxs ! j)
  by blast
  have raw-eq: final-raw-idxs ! i = raw
    using lookup-final[OF i-bound] lookup-final-raw by simp
  have raw-hit:
    index (to-nat raw) ∈
      query-sampling-success-space trace-table composition-table
      (staged-alphas data)
  proof -
    have index (to-nat (final-raw-idxs ! i)) = query-idxs ! i
      using query-idxs-eq len-final-raw i-bound by simp
    then show ?thesis
      using query-hit as-eq raw-eq by simp
  qed
  have comp-nonempty-prefix: sqp-composition-fri-roots prefix ≠ []
    using comp-nonempty comp-roots-eq eqs by simp
  have trace-root-prefix:
    sqp-trace-root prefix = value trace-tree
    using root-eq trace-root eqs by simp
  have comp-root-prefix:
    hd (sqp-composition-fri-roots prefix) = value composition-tree
    using comp-root comp-roots-eq eqs by simp
  have split-prefix:
    ((merkle-root-binds-table (sqp-trace-root prefix) trace-table
      prefix-state ∧
      merkle-root-binds-table (hd (sqp-composition-fri-roots prefix))
      composition-table prefix-state) ∨
      hash-map-new-output-hit
      (set-tree trace-tree ∪ set-tree composition-tree)
      prefix-state final-state)
    using split root-eq comp-roots-eq eqs by simp
  show ?thesis
    apply (rule-tac x=prefix in exI)
    apply (rule-tac x=prefix-state in exI)
    apply (rule-tac x=raw in exI)
    apply (rule-tac x=raw-state in exI)
    apply (rule-tac x=trace-tree in exI)
    apply (rule-tac x=composition-tree in exI)
    using prefix-receive alphas-eq lookup-final-raw raw-hit
      comp-nonempty-prefix trace-created trace-root-prefix comp-created
      comp-root-prefix split-prefix
    by blast
  qed

lemma checked-staged-transcript-query-bound-table-hit-or-tree-output-at-length-committed-prefix-receive-support:
  assumes wf: staged-budget-wellformed budgets
  and controlled: staged-adversary-controlled budgets A

```


and *outcome*:
Some (data, attacker-state) ∈
set-dist
(execute (checked-staged-transcript-program A)
adversary-initial-state)
and *verifier*:
Some (result, final-state) ∈
set-dist
(execute verify-monad
(verifier-state-from-adversary attacker-state
(staged-proof-transcript data))))
and *i-bound*: *i < rounds*
and *bound*:
accepted-with-bound-tables
(verifier-state-from-adversary attacker-state
(staged-proof-transcript data))
(Some (result, final-state))
trace-table composition-table as query-idxs
and *query-hit*:
query-idxs ! i ∈
query-sampling-success-space trace-table composition-table as
shows
 $\exists$ *prefix prefix-state raw raw-state trace-tree composition-tree.*
Some (((prefix, prefix-state), raw), raw-state) ∈
set-dist
(execute (checked-staged-query-prefix-receive-with-state A i)
adversary-initial-state) ∧
fmlookup (HashMap final-state)
(QueryIndexChallenge i
(state-after-query-chunks
(staged-query-start-hash data) (staged-query-chunks data) i)) =
Some raw ∧
index (to-nat raw) ∈
query-sampling-success-space trace-table composition-table
(staged-alphas data) ∧
sqp-composition-fri-roots prefix ≠ [] ∧
created-tree trace-table trace-tree final-state ∧
sqp-trace-root prefix = value trace-tree ∧
created-tree composition-table composition-tree final-state ∧
hd (sqp-composition-fri-roots prefix) = value composition-tree ∧
(checked-staged-query-prefix-dynamic-index-hit
staged-query-prefix-length-committed-query-target
(Some (((prefix, prefix-state), raw), raw-state)) ∨
hash-map-new-output-hit
(set-tree trace-tree ∪ set-tree composition-tree)
prefix-state final-state)
proof –
from *checked-staged-bound-tables-initial-roots-pullback-at-query-prefix-support*
[OF wf controlled outcome verifier i-bound bound query-hit]

obtain *prefix prefix-state raw raw-state trace-tree composition-tree* **where**
prefix-receive:
Some (((prefix, prefix-state), raw), raw-state) ∈
set-dist
(execute (checked-staged-query-prefix-receive-with-state A i)
adversary-initial-state)
and *alphas-eq:* *sqp-alphas prefix = staged-alphas data*
and *lookup-final:*
fmlookup (HashMap final-state)
(QueryIndexChallenge i
(state-after-query-chunks
(staged-query-start-hash data) (staged-query-chunks data) i)) =
Some raw
and *raw-hit:*
index (to-nat raw) ∈
query-sampling-success-space trace-table composition-table
(staged-alphas data)
and *comp-nonempty:* *sqp-composition-fri-roots prefix ≠ []*
and *trace-created:* *created-tree trace-table trace-tree final-state*
and *trace-root:* *sqp-trace-root prefix = value trace-tree*
and *comp-created:*
created-tree composition-table composition-tree final-state
and *comp-root:*
hd (sqp-composition-fri-roots prefix) = value composition-tree
and *split:*
(merkle-root-binds-table (sqp-trace-root prefix) trace-table
prefix-state ∧
merkle-root-binds-table (hd (sqp-composition-fri-roots prefix))
composition-table prefix-state) ∨
hash-map-new-output-hit
(set-tree trace-tree ∪ set-tree composition-tree)
prefix-state final-state
by *blast*
have *trace-len:* *length trace-table = clength * scale*
using *bound unfolding accepted-with-bound-tables-def*
accepted-with-tables-def **by** *simp*
have *comp-len:* *length composition-table = clength * scale*
using *bound unfolding accepted-with-bound-tables-def*
accepted-with-tables-def **by** *simp*
have *prefix-or-tree:*
checked-staged-query-prefix-dynamic-index-hit
staged-query-prefix-length-committed-query-target
(Some (((prefix, prefix-state), raw), raw-state)) ∨
hash-map-new-output-hit
(set-tree trace-tree ∪ set-tree composition-tree)
prefix-state final-state
proof —
from *split* **show** *?thesis*
proof

```

assume binds:
  merkle-root-binds-table (sqp-trace-root prefix) trace-table
  prefix-state  $\wedge$ 
  merkle-root-binds-table (hd (sqp-composition-fri-roots prefix))
  composition-table prefix-state
then have candidate:
  (trace-table, composition-table)  $\in$ 
    staged-query-prefix-length-committed-table-candidates prefix
    prefix-state
unfolding staged-query-prefix-length-committed-table-candidates-def
using comp-nonempty trace-len comp-len by simp
have target:
  index (to-nat raw)  $\in$ 
    staged-query-prefix-length-committed-query-target prefix
    prefix-state
proof –
  have raw-hit-prefix:
    index (to-nat raw)  $\in$ 
      query-sampling-success-space trace-table composition-table
      (sqp-alphas prefix)
    using raw-hit alphas-eq by simp
  show ?thesis
    by (rule staged-query-prefix-length-committed-query-targetI
      [OF candidate raw-hit-prefix])
  qed
then show ?thesis
  unfolding checked-staged-query-prefix-dynamic-index-hit-def
  by simp
next
assume
  hash-map-new-output-hit
  (set-tree trace-tree  $\cup$  set-tree composition-tree)
  prefix-state final-state
  then show ?thesis by simp
qed
qed
show ?thesis
  apply (rule-tac x=prefix in exI)
  apply (rule-tac x=prefix-state in exI)
  apply (rule-tac x=raw in exI)
  apply (rule-tac x=raw-state in exI)
  apply (rule-tac x=trace-tree in exI)
  apply (rule-tac x=composition-tree in exI)
  using prefix-receive lookup-final raw-hit comp-nonempty trace-created
    trace-root comp-created comp-root prefix-or-tree
  by blast
qed

lemma checked-staged-transcript-query-bound-table-hit-or-tree-output-at-committed-prefix-receive-support:

```

assumes *wf*: staged-budget-wellformed budgets
and *controlled*: staged-adversary-controlled budgets *A*
and *outcome*:
Some (*data*, *attacker-state*) $\in$
set-dist
(execute (checked-staged-transcript-program A)
adversary-initial-state)
and *verifier*:
Some (*result*, *final-state*) $\in$
set-dist
(execute verify-monad
(verifier-state-from-adversary attacker-state
(staged-proof-transcript data)))
and *i-bound*: *i* < rounds
and *lookup-attacker*:
fmlookup (HashMap attacker-state)
(QueryIndexChallenge i
(state-after-query-chunks
(staged-query-start-hash data) (staged-query-chunks data) i)) =
Some raw-hit
and *raw-hit-target*:
index (to-nat raw-hit) $\in$
query-sampling-success-space trace-table composition-table
(staged-alphas data)
and *bound*:
accepted-with-bound-tables
(verifier-state-from-adversary attacker-state
(staged-proof-transcript data))
(Some (result, final-state))
trace-table composition-table as query-idxs
shows
 $\exists$ *prefix prefix-state raw raw-state trace-tree composition-tree.*
Some (((prefix, prefix-state), raw), raw-state) $\in$
set-dist
(execute (checked-staged-query-prefix-receive-with-state A i)
adversary-initial-state) $\wedge$
fmlookup (HashMap final-state)
(QueryIndexChallenge i
(state-after-query-chunks
(staged-query-start-hash data) (staged-query-chunks data) i)) =
Some raw $\wedge$
index (to-nat raw) $\in$
query-sampling-success-space trace-table composition-table
(staged-alphas data) $\wedge$
sqp-composition-fri-roots prefix $\neq []$ $\wedge$
created-tree trace-table trace-tree final-state $\wedge$
sqp-trace-root prefix = value trace-tree $\wedge$
created-tree composition-table composition-tree final-state $\wedge$
hd (sqp-composition-fri-roots prefix) = value composition-tree $\wedge$

```

    (checked-staged-query-prefix-dynamic-index-hit
      (λprefix prefix-state.
        staged-query-prefix-committed-query-target prefix prefix-state)
      (Some (((prefix, prefix-state), raw), raw-state)) ∨
      hash-map-new-output-hit
      (set-tree trace-tree ∪ set-tree composition-tree)
      prefix-state final-state)
  proof -
  let ?s =
    verifier-state-from-adversary attacker-state
    (staged-proof-transcript data)
  from checked-staged-transcript-program-query-prefix-receive-compact-support
    [OF wf controlled i-bound outcome]
  obtain prefix prefix-state raw raw-state where
    prefix-receive:
      Some (((prefix, prefix-state), raw), raw-state) ∈
      set-dist
      (execute (checked-staged-query-prefix-receive-with-state A i)
        adversary-initial-state)
  and root-eq: sqp-trace-root prefix = staged-trace-root data
  and alphas-eq: sqp-alphas prefix = staged-alphas data
  and comp-roots-eq:
    sqp-composition-fri-roots prefix =
      staged-composition-fri-roots data
  and prefix-ext: prefix-state ≤ attacker-state
  and raw-ext: raw-state ≤ attacker-state
  and lookup-raw:
    fmlookup (HashMap raw-state)
      (QueryIndexChallenge i
        (state-after-query-chunks
          (staged-query-start-hash data) (staged-query-chunks data) i)) =
      Some raw
  by (rule checked-staged-transcript-program-query-prefix-receive-compact-support
    [OF wf controlled i-bound outcome])
  have lookup-attacker-raw:
    fmlookup (HashMap attacker-state)
      (QueryIndexChallenge i
        (state-after-query-chunks
          (staged-query-start-hash data) (staged-query-chunks data) i)) =
      Some raw
  by (rule hash-extension-lookup[OF lookup-raw raw-ext])
  have raw-eq: raw = raw-hit
  using lookup-attacker lookup-attacker-raw by simp
  have prefix-ext-s: prefix-state ≤ ?s
  by (rule hash-extends-verifier-state-from-adversary-right
    [OF prefix-ext])
  have s-ext-final: ?s ≤ final-state
  by (rule verify-monad-hash-extends[OF verifier])
  have prefix-ext-final: prefix-state ≤ final-state

```

```

by (rule hash-ext-trans[OF prefix-ext-s s-ext-final])
have staged-header:
  verifier-header-transcript ?s
    (staged-trace-root data)
    (staged-trace-fri-roots data)
    (staged-trace-final data)
    (staged-alphas data)
    (staged-degree data)
    (staged-composition-fri-roots data)
    (staged-composition-final data)
    (List.concat (staged-query-chunks data))
by (rule checked-staged-transcript-program-outcome-header-transcript
  [OF wf controlled outcome])
from accepted-with-bound-tables-initial-roots-pullback-or-new-tree-output-hit
  [OF prefix-ext-final bound]
obtain fr f-fri-roots f-final dg composition-fri-roots final rest
  trace-tree composition-tree where
  header:
    verifier-header-transcript ?s fr f-fri-roots f-final as dg
    composition-fri-roots final rest
and comp-nonempty: composition-fri-roots  $\neq []$ 
and trace-created:
  created-tree trace-table trace-tree final-state
and trace-root: fr = value trace-tree
and comp-created:
  created-tree composition-table composition-tree final-state
and comp-root:
  hd composition-fri-roots = value composition-tree
and split:
  (merkle-root-binds-table fr trace-table prefix-state  $\wedge$ 
    merkle-root-binds-table (hd composition-fri-roots)
    composition-table prefix-state)  $\vee$ 
  hash-map-new-output-hit
  (set-tree trace-tree  $\cup$  set-tree composition-tree)
  prefix-state final-state
by blast
have eqs:
  fr = staged-trace-root data  $\wedge$ 
  f-fri-roots = staged-trace-fri-roots data  $\wedge$ 
  f-final = staged-trace-final data  $\wedge$ 
  as = staged-alphas data  $\wedge$ 
  dg = staged-degree data  $\wedge$ 
  composition-fri-roots = staged-composition-fri-roots data  $\wedge$ 
  final = staged-composition-final data  $\wedge$ 
  rest = List.concat (staged-query-chunks data)
using verifier-header-transcript-unique[OF header staged-header]
by simp
have split-prefix:
  (merkle-root-binds-table (spp-trace-root prefix) trace-table

```

```

    prefix-state  $\wedge$ 
    merkle-root-binds-table (hd (sqp-composition-fri-roots prefix))
    composition-table prefix-state)  $\vee$ 
    hash-map-new-output-hit
    (set-tree trace-tree  $\cup$  set-tree composition-tree)
    prefix-state final-state
  using split root-eq comp-roots-eq eqs by simp
have comp-nonempty-prefix:
  sqp-composition-fri-roots prefix  $\neq$  []
  using comp-nonempty comp-roots-eq eqs by simp
have raw-hit-actual:
  index (to-nat raw)  $\in$ 
    query-sampling-success-space trace-table composition-table
    (staged-alphas data)
  using raw-hit-target unfolding raw-eq .
have lookup-initial-row:
  fmllookup (HashMap ?s)
    (QueryIndexChallenge i
      (state-after-query-chunks
        (staged-query-start-hash data) (staged-query-chunks data) i)) =
    Some raw
  using lookup-attacker-row by simp
have lookup-final-row:
  fmllookup (HashMap final-state)
    (QueryIndexChallenge i
      (state-after-query-chunks
        (staged-query-start-hash data) (staged-query-chunks data) i)) =
    Some raw
  by (rule hash-extension-lookup[OF lookup-initial-row s-ext-final])
have trace-root-prefix:
  sqp-trace-root prefix = value trace-tree
  using root-eq trace-root eqs by simp
have comp-root-prefix:
  hd (sqp-composition-fri-roots prefix) = value composition-tree
  using comp-root comp-roots-eq eqs by simp
have conclusion:
  checked-staged-query-prefix-dynamic-index-hit
    ( $\lambda$ prefix prefix-state.
      staged-query-prefix-committed-query-target prefix prefix-state)
    (Some (((prefix, prefix-state), raw), raw-state))  $\vee$ 
  hash-map-new-output-hit
    (set-tree trace-tree  $\cup$  set-tree composition-tree)
    prefix-state final-state
proof -
  from split-prefix show ?thesis
proof
  assume prefix-binds:
    merkle-root-binds-table (sqp-trace-root prefix) trace-table
    prefix-state  $\wedge$ 

```

```

    merkle-root-binds-table (hd (sqp-composition-fri-roots prefix))
    composition-table prefix-state
  then have trace-bind:
    merkle-root-binds-table (sqp-trace-root prefix) trace-table
    prefix-state
  and comp-bind:
    merkle-root-binds-table (hd (sqp-composition-fri-roots prefix))
    composition-table prefix-state
  by simp-all
have candidate:
  (trace-table, composition-table) ∈
  query-header-committed-table-candidates prefix-state
  (sqp-trace-root prefix)
  (sqp-composition-fri-roots prefix)
  unfolding query-header-committed-table-candidates-def
  using trace-bind comp-nonempty-prefix comp-bind by simp
have raw-target-prefix:
  index (to-nat raw) ∈
  query-sampling-success-space trace-table composition-table
  (sqp-alphas prefix)
  using raw-hit-target unfolding raw-eq alphas-eq .
have target:
  index (to-nat raw) ∈
  staged-query-prefix-committed-query-target prefix prefix-state
  by (rule staged-query-prefix-committed-query-targetI
    [OF candidate raw-target-prefix])
then show ?thesis
  unfolding checked-staged-query-prefix-dynamic-index-hit-def by simp
qed
next
  assume
    hash-map-new-output-hit
    (set-tree trace-tree ∪ set-tree composition-tree)
    prefix-state final-state
  then show ?thesis by simp
qed
qed
show ?thesis
  apply (rule-tac x=prefix in exI)
  apply (rule-tac x=prefix-state in exI)
  apply (rule-tac x=raw in exI)
  apply (rule-tac x=raw-state in exI)
  apply (rule-tac x=trace-tree in exI)
  apply (rule-tac x=composition-tree in exI)
  using prefix-receive lookup-final-raw raw-hit-actual
    comp-nonempty-prefix trace-created trace-root-prefix comp-created
    comp-root-prefix conclusion
  by blast
qed

```


lemma *checked-staged-security-with-data-state-query-bad-imp-committed-prefix-hit-or-tree-output-on-support:*
assumes *wf: staged-budget-wellformed budgets*
and *controlled: staged-adversary-controlled budgets A*
and *support:*
 out $\in$
 set-dist
 (*execute*
 (*checked-staged-security-experiment-with-data-state A*)
 adversary-initial-state)
and *bad: staged-security-with-data-state-query-bad-hit out*
shows
 staged-security-with-data-state-query-committed-prefix-hit A out $\vee$
 staged-security-with-data-state-query-tree-output-hit A out
proof (*cases out*)
case *None*
then show *?thesis*
 using *bad unfolding staged-security-with-data-state-query-bad-hit-def*
 by *simp*
next
case (*Some result-pack*)
then obtain *data attacker-state result final-state* **where** *out-eq:*
 out = Some (((data, attacker-state), result), final-state)
 by (*cases result-pack, auto split: prod.splits*)
let *?s =*
 verifier-state-from-adversary attacker-state
 (*staged-proof-transcript data*)
have *support-some:*
 Some (((data, attacker-state), result), final-state) $\in$
 set-dist
 (*execute*
 (*checked-staged-security-experiment-with-data-state A*)
 adversary-initial-state)
 using *support unfolding out-eq .*
from *checked-staged-security-experiment-with-data-state-outcomeE*
 [*OF support-some*]
have *builder:*
 Some (data, attacker-state) $\in$
 set-dist
 (*execute (checked-staged-transcript-program A)*
 adversary-initial-state)
 and *verifier:*
 Some (result, final-state) $\in$ set-dist (execute verify-monad ?s)
 by *blast+*
from *bad[unfolded out-eq*
 staged-security-with-data-state-query-bad-hit-def Let-def]
have *query-bad: query-bad ?s (Some (result, final-state))*
 by *simp*
from *query-bad-imp-query-index-round-set-hit[OF query-bad]*
obtain *i trace-table composition-table as query-idxs* **where**

```

bound:
  accepted-with-bound-tables ?s (Some (result, final-state))
  trace-table composition-table as query-idxs
and i-bound: i < rounds
and query-hit:
  query-idxs ! i ∈
    query-sampling-success-space trace-table composition-table as
unfolding query-index-round-set-hit-def
  query-index-round-set-hit-at-def
by blast
have shape:
  accepted-transcript-shape ?s (Some (result, final-state))
  as query-idxs
using bound
unfolding accepted-with-bound-tables-def accepted-with-tables-def
by simp
from checked-staged-security-with-data-state-accepted-shape-query-keys
  [OF wf controlled support-some shape]
obtain final-raw-idxs where
  as-eq: as = staged-alphas data
  and len-final-raw: length final-raw-idxs = rounds
  and query-idxs-eq:
    query-idxs = map (λraw. index (to-nat raw)) final-raw-idxs
  and lookup-final:
     $\bigwedge j. j < rounds \implies$ 
      fmllookup (HashMap final-state)
        (QueryIndexChallenge j
          (state-after-query-chunks
            (staged-query-start-hash data)
            (staged-query-chunks data) j)) =
        Some (final-raw-idxs ! j)
  by blast
from checked-staged-transcript-program-outcome-query-lookups
  [OF wf controlled builder]
obtain raw-idxs where
  len-raw: length raw-idxs = rounds
  and lookup-attacker-all:
     $\forall j < rounds.$ 
      fmllookup (HashMap attacker-state)
        (QueryIndexChallenge j
          (state-after-query-chunks
            (staged-query-start-hash data)
            (staged-query-chunks data) j)) =
        Some (raw-idxs ! j)
  by blast
have s-ext-final: ?s ≤ final-state
  by (rule verify-monad-hash-extends[OF verifier])
have lookup-initial-i:
  fmllookup (HashMap ?s)

```

```

    (QueryIndexChallenge i
      (state-after-query-chunks
        (staged-query-start-hash data) (staged-query-chunks data) i)) =
    Some (raw-idxs ! i)
  using lookup-attacker-all i-bound by simp
have lookup-final-i-from-initial:
  fmlookup (HashMap final-state)
    (QueryIndexChallenge i
      (state-after-query-chunks
        (staged-query-start-hash data) (staged-query-chunks data) i)) =
    Some (raw-idxs ! i)
  by (rule hash-extension-lookup[OF lookup-initial-i s-ext-final])
have raw-eq: final-raw-idxs ! i = raw-idxs ! i
  using lookup-final[OF i-bound] lookup-final-i-from-initial by simp
have raw-hit-target:
  index (to-nat (raw-idxs ! i)) ∈
    query-sampling-success-space trace-table composition-table
      (staged-alphas data)
proof -
  have index (to-nat (final-raw-idxs ! i)) = query-idxs ! i
    using query-idxs-eq len-final-raw i-bound by simp
  then show ?thesis
    using query-hit as-eq raw-eq by simp
qed
have lookup-attacker-i:
  fmlookup (HashMap attacker-state)
    (QueryIndexChallenge i
      (state-after-query-chunks
        (staged-query-start-hash data) (staged-query-chunks data) i)) =
    Some (raw-idxs ! i)
  using lookup-attacker-all i-bound by simp
from
  checked-staged-transcript-query-bound-table-hit-or-tree-output-at-committed-prefix-receive-support
    [OF wf controlled builder verifier i-bound lookup-attacker-i
      raw-hit-target bound]
obtain prefix prefix-state raw raw-state trace-tree composition-tree where
  prefix-receive:
    Some (((prefix, prefix-state), raw), raw-state) ∈
      set-dist
        (execute (checked-staged-query-prefix-receive-with-state A i)
          adversary-initial-state)
and lookup-final-raw:
  fmlookup (HashMap final-state)
    (QueryIndexChallenge i
      (state-after-query-chunks
        (staged-query-start-hash data) (staged-query-chunks data) i)) =
    Some raw
and raw-hit-actual:
  index (to-nat raw) ∈

```

```

    query-sampling-success-space trace-table composition-table
    (staged-alphas data)
and comp-nonempty: sqp-composition-fri-roots prefix ≠ []
and trace-created:
    created-tree trace-table trace-tree final-state
and trace-root: sqp-trace-root prefix = value trace-tree
and comp-created:
    created-tree composition-table composition-tree final-state
and comp-root:
    hd (sqp-composition-fri-roots prefix) = value composition-tree
and split:
    checked-staged-query-prefix-dynamic-index-hit
    (λprefix prefix-state.
      staged-query-prefix-committed-query-target prefix prefix-state)
    (Some (((prefix, prefix-state), raw), raw-state)) ∨
    hash-map-new-output-hit
    (set-tree trace-tree ∪ set-tree composition-tree)
    prefix-state final-state
by blast
from split show ?thesis
proof
  assume prefix-hit:
    checked-staged-query-prefix-dynamic-index-hit
    (λprefix prefix-state.
      staged-query-prefix-committed-query-target prefix prefix-state)
    (Some (((prefix, prefix-state), raw), raw-state))
  have committed-body:
    ∃ i prefix prefix-state raw raw-state.
      i < rounds ∧
      Some (data, attacker-state) ∈
        set-dist
        (execute (checked-staged-transcript-program A)
          adversary-initial-state) ∧
      Some (((prefix, prefix-state), raw), raw-state) ∈
        set-dist
        (execute
          (checked-staged-query-prefix-receive-with-state A i)
          adversary-initial-state) ∧
      checked-staged-query-prefix-dynamic-index-hit
      staged-query-prefix-committed-query-target
      (Some (((prefix, prefix-state), raw), raw-state))
  apply (rule-tac x=i in exI)
  apply (rule-tac x=prefix in exI)
  apply (rule-tac x=prefix-state in exI)
  apply (rule-tac x=raw in exI)
  apply (rule-tac x=raw-state in exI)
  using i-bound builder prefix-receive prefix-hit
  by simp
then show ?thesis

```

```

unfolding out-eq
  staged-security-with-data-state-query-committed-prefix-hit-def
by simp
next
  assume tree-hit:
    hash-map-new-output-hit
      (set-tree trace-tree  $\cup$  set-tree composition-tree)
    prefix-state final-state
  have tree-body:
     $\exists i$  prefix prefix-state raw raw-state trace-table composition-table
      as query-idxs trace-tree composition-tree.
     $i < \text{rounds} \wedge$ 
    Some (data, attacker-state)  $\in$ 
      set-dist
        (execute (checked-staged-transcript-program A)
          adversary-initial-state)  $\wedge$ 
    Some (result, final-state)  $\in$  set-dist (execute verify-monad ?s)  $\wedge$ 
    accepted-with-bound-tables ?s (Some (result, final-state))
    trace-table composition-table as query-idxs  $\wedge$ 
    fmllookup (HashMap final-state)
      (QueryIndexChallenge i
        (state-after-query-chunks
          (staged-query-start-hash data)
          (staged-query-chunks data) i)) =
    Some raw  $\wedge$ 
    index (to-nat raw)  $\in$ 
      query-sampling-success-space trace-table composition-table
        (staged-alphas data)  $\wedge$ 
    Some (((prefix, prefix-state), raw), raw-state)  $\in$ 
      set-dist
        (execute
          (checked-staged-query-prefix-receive-with-state A i)
          adversary-initial-state)  $\wedge$ 
    sqp-composition-fri-roots prefix  $\neq []$   $\wedge$ 
    created-tree trace-table trace-tree final-state  $\wedge$ 
    sqp-trace-root prefix = value trace-tree  $\wedge$ 
    created-tree composition-table composition-tree final-state  $\wedge$ 
    hd (sqp-composition-fri-roots prefix) =
      value composition-tree  $\wedge$ 
    hash-map-new-output-hit
      (set-tree trace-tree  $\cup$  set-tree composition-tree)
    prefix-state final-state
  apply (rule-tac x=i in exI)
  apply (rule-tac x=prefix in exI)
  apply (rule-tac x=prefix-state in exI)
  apply (rule-tac x=raw in exI)
  apply (rule-tac x=raw-state in exI)
  apply (rule-tac x=trace-table in exI)
  apply (rule-tac x=composition-table in exI)

```

```

    apply (rule-tac x=as in exI)
    apply (rule-tac x=query-idxs in exI)
    apply (rule-tac x=trace-tree in exI)
    apply (rule-tac x=composition-tree in exI)
    using i-bound builder verifier bound lookup-final-raw raw-hit-actual
           prefix-receive comp-nonempty trace-created trace-root comp-created
           comp-root tree-hit
    by blast
  then show ?thesis
    unfolding out-eq
      staged-security-with-data-state-query-tree-output-hit-def Let-def
    by simp
qed
qed

lemma checked-staged-security-with-data-state-query-bad-imp-length-committed-prefix-hit-or-tree-output-on-support:
  assumes wf: staged-budget-wellformed budgets
  and controlled: staged-adversary-controlled budgets A
  and support:
    out ∈
      set-dist
        (execute
          (checked-staged-security-experiment-with-data-state A)
          adversary-initial-state)
  and bad: staged-security-with-data-state-query-bad-hit out
  shows
    staged-security-with-data-state-query-length-committed-prefix-hit A out ∨
    staged-security-with-data-state-query-tree-output-hit A out
proof (cases out)
  case None
  then show ?thesis
    using bad unfolding staged-security-with-data-state-query-bad-hit-def
    by simp
next
  case (Some result-pack)
  then obtain data attacker-state result final-state where out-eq:
    out = Some (((data, attacker-state), result), final-state)
  by (cases result-pack, auto split: prod.splits)
  let ?s =
    verifier-state-from-adversary attacker-state
      (staged-proof-transcript data)
  have support-some:
    Some (((data, attacker-state), result), final-state) ∈
      set-dist
        (execute
          (checked-staged-security-experiment-with-data-state A)
          adversary-initial-state)
  using support unfolding out-eq .
from checked-staged-security-experiment-with-data-state-outcomeE

```

```

[OF support-some]
have builder:
  Some (data, attacker-state) ∈
    set-dist
      (execute (checked-staged-transcript-program A)
        adversary-initial-state)
and verifier:
  Some (result, final-state) ∈ set-dist (execute verify-monad ?s)
by blast+
from bad[unfolded out-eq
  staged-security-with-data-state-query-bad-hit-def Let-def]
have query-bad: query-bad ?s (Some (result, final-state))
by simp
from query-bad-imp-query-index-round-set-hit[OF query-bad]
obtain i trace-table composition-table as query-idxs where
  bound:
    accepted-with-bound-tables ?s (Some (result, final-state))
    trace-table composition-table as query-idxs
and i-bound: i < rounds
and query-hit:
  query-idxs ! i ∈
    query-sampling-success-space trace-table composition-table as
unfolding query-index-round-set-hit-def
  query-index-round-set-hit-at-def
by blast
from
  checked-staged-transcript-query-bound-table-hit-or-tree-output-at-length-committed-prefix-receive-support
  [OF wf controlled builder verifier i-bound bound query-hit]
obtain prefix prefix-state raw raw-state trace-tree composition-tree where
  prefix-receive:
    Some (((prefix, prefix-state), raw), raw-state) ∈
      set-dist
        (execute (checked-staged-query-prefix-receive-with-state A i)
          adversary-initial-state)
and lookup-final-raw:
  fmlookup (HashMap final-state)
    (QueryIndexChallenge i
      (state-after-query-chunks
        (staged-query-start-hash data) (staged-query-chunks data) i)) =
    Some raw
and raw-hit-actual:
  index (to-nat raw) ∈
    query-sampling-success-space trace-table composition-table
    (staged-alphas data)
and comp-nonempty: sqp-composition-fri-roots prefix ≠ []
and trace-created:
  created-tree trace-table trace-tree final-state
and trace-root: sqp-trace-root prefix = value trace-tree
and comp-created:

```

```

    created-tree composition-table composition-tree final-state
and comp-root:
    hd (sqp-composition-fri-roots prefix) = value composition-tree
and split:
    checked-staged-query-prefix-dynamic-index-hit
    staged-query-prefix-length-committed-query-target
    (Some (((prefix, prefix-state), raw), raw-state)) ∨
    hash-map-new-output-hit
    (set-tree trace-tree ∪ set-tree composition-tree)
    prefix-state final-state
by blast
from split show ?thesis
proof
  assume prefix-hit:
    checked-staged-query-prefix-dynamic-index-hit
    staged-query-prefix-length-committed-query-target
    (Some (((prefix, prefix-state), raw), raw-state))
  have length-body:
    ∃ i prefix prefix-state raw raw-state.
    i < rounds ∧
    Some (data, attacker-state) ∈
    set-dist
    (execute (checked-staged-transcript-program A)
    adversary-initial-state) ∧
    Some (((prefix, prefix-state), raw), raw-state) ∈
    set-dist
    (execute
    (checked-staged-query-prefix-receive-with-state A i)
    adversary-initial-state) ∧
    checked-staged-query-prefix-dynamic-index-hit
    staged-query-prefix-length-committed-query-target
    (Some (((prefix, prefix-state), raw), raw-state))
  apply (rule-tac x=i in exI)
  apply (rule-tac x=prefix in exI)
  apply (rule-tac x=prefix-state in exI)
  apply (rule-tac x=raw in exI)
  apply (rule-tac x=raw-state in exI)
  using i-bound builder prefix-receive prefix-hit
  by simp
  then show ?thesis
  unfolding out-eq
    staged-security-with-data-state-query-length-committed-prefix-hit-def
  by simp
next
  assume tree-hit:
    hash-map-new-output-hit
    (set-tree trace-tree ∪ set-tree composition-tree)
    prefix-state final-state
  have tree-body:

```



```

∃ i prefix prefix-state raw raw-state trace-table composition-table
  as query-idxs trace-tree composition-tree.
i < rounds ∧
Some (data, attacker-state) ∈
  set-dist
    (execute (checked-staged-transcript-program A)
      adversary-initial-state) ∧
Some (result, final-state) ∈ set-dist (execute verify-monad ?s) ∧
accepted-with-bound-tables ?s (Some (result, final-state))
  trace-table composition-table as query-idxs ∧
fmllookup (HashMap final-state)
  (QueryIndexChallenge i
    (state-after-query-chunks
      (staged-query-start-hash data)
      (staged-query-chunks data) i)) =
    Some raw ∧
index (to-nat raw) ∈
  query-sampling-success-space trace-table composition-table
    (staged-alphas data) ∧
Some (((prefix, prefix-state), raw), raw-state) ∈
  set-dist
    (execute
      (checked-staged-query-prefix-receive-with-state A i)
      adversary-initial-state) ∧
sqp-composition-fri-roots prefix ≠ [] ∧
created-tree trace-table trace-tree final-state ∧
sqp-trace-root prefix = value trace-tree ∧
created-tree composition-table composition-tree final-state ∧
hd (sqp-composition-fri-roots prefix) =
  value composition-tree ∧
hash-map-new-output-hit
  (set-tree trace-tree ∪ set-tree composition-tree)
  prefix-state final-state
apply (rule-tac x=i in exI)
apply (rule-tac x=prefix in exI)
apply (rule-tac x=prefix-state in exI)
apply (rule-tac x=raw in exI)
apply (rule-tac x=raw-state in exI)
apply (rule-tac x=trace-table in exI)
apply (rule-tac x=composition-table in exI)
apply (rule-tac x=as in exI)
apply (rule-tac x=query-idxs in exI)
apply (rule-tac x=trace-tree in exI)
apply (rule-tac x=composition-tree in exI)
using i-bound builder verifier bound lookup-final-raw raw-hit-actual
  prefix-receive comp-nonempty trace-created trace-root comp-created
  comp-root tree-hit
by blast
then show ?thesis

```

unfolding *out-eq*
staged-security-with-data-state-query-tree-output-hit-def Let-def
by *simp*
qed
qed

lemma *checked-staged-security-with-data-state-query-bad-bound-from-committed-prefix-hit-or-tree-output:*

assumes *wf: staged-budget-wellformed budgets*

and *controlled: staged-adversary-controlled budgets A*

shows

wp-event (checked-staged-security-experiment-with-data-state A)
staged-security-with-data-state-query-bad-hit adversary-initial-state ≤
wp-event (checked-staged-security-experiment-with-data-state A)
(staged-security-with-data-state-query-committed-prefix-hit A)
adversary-initial-state +
wp-event (checked-staged-security-experiment-with-data-state A)
(staged-security-with-data-state-query-tree-output-hit A)
adversary-initial-state

proof —

let *?M = checked-staged-security-experiment-with-data-state A*

let *?committed =*

staged-security-with-data-state-query-committed-prefix-hit A

let *?tree = staged-security-with-data-state-query-tree-output-hit A*

have *wp-event ?M staged-security-with-data-state-query-bad-hit*

adversary-initial-state ≤

wp-event ?M (λout. ?committed out ∨ ?tree out)

adversary-initial-state

by *(rule wp-event-mono-on-support)*

(rule checked-staged-security-with-data-state-query-bad-imp-committed-prefix-hit-or-tree-output-on-support
[OF wf controlled])

also have *... ≤*

wp-event ?M ?committed adversary-initial-state +

wp-event ?M ?tree adversary-initial-state

by *(rule wp-event-union-bound)*

finally show *?thesis .*

qed

lemma *checked-staged-security-with-data-state-query-bad-bound-from-length-committed-prefix-hit-or-tree-output:*

assumes *wf: staged-budget-wellformed budgets*

and *controlled: staged-adversary-controlled budgets A*

shows

wp-event (checked-staged-security-experiment-with-data-state A)
staged-security-with-data-state-query-bad-hit adversary-initial-state ≤
wp-event (checked-staged-security-experiment-with-data-state A)
(staged-security-with-data-state-query-length-committed-prefix-hit A)
adversary-initial-state +
wp-event (checked-staged-security-experiment-with-data-state A)
(staged-security-with-data-state-query-tree-output-hit A)
adversary-initial-state

```

proof –
  let ?M = checked-staged-security-experiment-with-data-state A
  let ?committed =
    staged-security-with-data-state-query-length-committed-prefix-hit A
  let ?tree = staged-security-with-data-state-query-tree-output-hit A
  have wp-event ?M staged-security-with-data-state-query-bad-hit
    adversary-initial-state ≤
    wp-event ?M (λout. ?committed out ∨ ?tree out)
    adversary-initial-state
  by (rule wp-event-mono-on-support)
    (rule
      checked-staged-security-with-data-state-query-bad-imp-length-committed-prefix-hit-or-tree-output-on-support
      [OF wf controlled])
  also have ... ≤
    wp-event ?M ?committed adversary-initial-state +
    wp-event ?M ?tree adversary-initial-state
  by (rule wp-event-union-bound)
  finally show ?thesis .
qed

end

end

```

```

theory Soundness-Bound-Table-Partial-Openings
  imports Soundness-Partial-Merkle
begin

```

Bridges from complete Merkle-bound table witnesses to partial authenticated openings. This proof layer is kept separate from `Soundness_Partial_Merkle` to avoid growing the base Merkle/event theory.

```

context soundness
begin

```

```

definition partial-composition-openings-inconsistent-bad
  :: ('f, 'a) protocol-channel-scheme ⇒
    (unit list × ('f, 'a) protocol-channel-scheme) option ⇒ bool
where
  partial-composition-openings-inconsistent-bad s out ⇔
    (∃ composition-root query-idxs composition-openings result final-state
      i opn opn'.
      out = Some (result, final-state) ∧
      accepted-with-partial-composition-openings s out composition-root
        query-idxs composition-openings ∧
      i < rounds ∧
      opn ∈ set (composition-openings ! i) ∧
      opn' ∈ set (composition-openings ! i) ∧
      opening-index opn = opening-index opn' ∧

```

opening-value opn ≠ opening-value opn')

lemma *partial-composition-openings-inconsistent-bad-imp-partial-merkle-inconsistency-bad:*

assumes *bad: partial-composition-openings-inconsistent-bad s out*

shows *partial-merkle-inconsistency-bad s out*

proof –

obtain *composition-root query-idxs composition-openings result final-state*

i opn opn' where

out: out = Some (result, final-state)

and *partial:*

accepted-with-partial-composition-openings s out composition-root

query-idxs composition-openings

and *i-bound: i < rounds*

and *opn-in: opn ∈ set (composition-openings ! i)*

and *opn'-in: opn' ∈ set (composition-openings ! i)*

and *same-index: opening-index opn = opening-index opn'*

and *diff-value: opening-value opn ≠ opening-value opn'*

using *bad*

unfolding *partial-composition-openings-inconsistent-bad-def* **by** *blast*

have *table:*

*partial-authenticated-table composition-root (scale * clength)*

(composition-openings ! i) final-state

proof –

obtain *result' final-state' where*

partial-out: out = Some (result', final-state')

and *tables:*

∀ i < rounds.

*partial-authenticated-table composition-root (scale * clength)*

(composition-openings ! i) final-state'

using *partial*

unfolding *accepted-with-partial-composition-openings-def* **by** *auto*

have *final-state-eq: final-state' = final-state*

using *out partial-out* **by** *simp*

show *?thesis*

using *tables i-bound* **unfolding** *final-state-eq* **by** *simp*

qed

have *opn-auth: authenticated-opening-in final-state opn*

using *table opn-in* **unfolding** *partial-authenticated-table-def* **by** *blast*

have *opn'-auth: authenticated-opening-in final-state opn'*

using *table opn'-in* **unfolding** *partial-authenticated-table-def* **by** *blast*

have *same-root: opening-root opn = opening-root opn'*

proof –

have *root-opn: opening-root opn = composition-root*

using *table opn-in* **unfolding** *partial-authenticated-table-def* **by** *simp*

have *root-opn': opening-root opn' = composition-root*

using *table opn'-in* **unfolding** *partial-authenticated-table-def* **by** *simp*

show *?thesis*

using *root-opn root-opn'* **by** *simp*

qed

```

have same-length: opening-length opn = opening-length opn'
proof -
  have length-opn: opening-length opn = scale * clength
    using table opn-in unfolding partial-authenticated-table-def by simp
  have length-opn': opening-length opn' = scale * clength
    using table opn'-in unfolding partial-authenticated-table-def by simp
  show ?thesis
    using length-opn length-opn' by simp
qed
have accepted-out: accepted out
  using partial
  unfolding accepted-with-partial-composition-openings-def by simp
show ?thesis
  unfolding partial-merkle-inconsistency-bad-def
  using accepted-out out opn-auth opn'-auth same-root same-length
    same-index diff-value
  by blast
qed

lemma partial-composition-openings-inconsistent-bad-imp-hash-map-output-collision-bad:
  assumes partial-composition-openings-inconsistent-bad s out
  shows hash-map-output-collision-bad s out
  by (rule partial-merkle-inconsistency-bad-imp-hash-map-output-collision-bad)
  (rule
    partial-composition-openings-inconsistent-bad-imp-partial-merkle-inconsistency-bad
    [OF assms])

lemma partial-composition-openings-inconsistent-bad-mono-hash-map-output-collision-bad:
  wp-event verify-monad (partial-composition-openings-inconsistent-bad s) s
  ≤ wp-event verify-monad (hash-map-output-collision-bad s) s
  by (rule wp-event-mono)
  (rule partial-composition-openings-inconsistent-bad-imp-hash-map-output-collision-bad)

lemma partial-composition-openings-inconsistent-bad-bound-from-hash-map-output-collision:
  fixes H :: prob
  assumes wp-event verify-monad (hash-map-output-collision-bad s) s ≤ H
  shows
    wp-event verify-monad
      (partial-composition-openings-inconsistent-bad s) s ≤ H
  by (rule order-trans
    [OF
      partial-composition-openings-inconsistent-bad-mono-hash-map-output-collision-bad
      assms])

lemma table-agrees-with-authenticated-openings-from-maps:
  assumes len-table: length table = len
  and indices: map opening-index openings = idxs
  and value-map: map opening-value openings = map (!) table idxs
  and idx-bound:  $\bigwedge idx. idx \in \text{set } idxs \implies idx < len$ 

```

```

shows table-agrees-with-authenticated-openings table len openings
proof –
  have opening-props:
     $\forall \text{opn} \in \text{set openings.}$ 
     $\text{opening-index } \text{opn} < \text{len} \wedge$ 
     $\text{table ! opening-index } \text{opn} = \text{opening-value } \text{opn}$ 
  proof
    fix opn
    assume opn-in:  $\text{opn} \in \text{set openings}$ 
    then obtain k where k-bound:  $k < \text{length openings}$ 
    and opn-eq:  $\text{opn} = \text{openings ! } k$ 
    by (metis in-set-conv-nth)
    have idxs-len:  $\text{length idxs} = \text{length openings}$ 
    using arg-cong[OF indices, of length] by simp
    have idx-eq:  $\text{opening-index } \text{opn} = \text{idxs ! } k$ 
    using indices k-bound idxs-len opn-eq
    by (metis length-map nth-map)
    have value-eq:  $\text{opening-value } \text{opn} = \text{table ! (idxs ! } k)$ 
    proof –
      have  $\text{opening-value } \text{opn} = \text{map opening-value openings ! } k$ 
      using k-bound opn-eq by simp
      also have  $\dots = \text{map (! table) idxs ! } k$ 
      using value-map by simp
      also have  $\dots = \text{table ! (idxs ! } k)$ 
      using k-bound idxs-len by simp
      finally show ?thesis .
    qed
    have idx-in:  $\text{idxs ! } k \in \text{set idxs}$ 
    by (rule nth-mem) (use k-bound idxs-len in simp)
    show  $\text{opening-index } \text{opn} < \text{len} \wedge$ 
     $\text{table ! opening-index } \text{opn} = \text{opening-value } \text{opn}$ 
    using idx-bound[OF idx-in] idx-eq value-eq by simp
  qed
show ?thesis
  unfolding table-agrees-with-authenticated-openings-def
  using len-table opening-props by simp
qed

```

```

lemma accepted-with-bound-tables-partial-opening-witnesses:
  assumes bound:
    accepted-with-bound-tables s out trace-table composition-table as
    query-idxs
  obtains result final-state fr f-fri-roots f-final dg composition-fri-roots
    final rest trace-openings composition-openings where
    out = Some (result, final-state)
    verifier-header-transcript s fr f-fri-roots f-final as dg
    composition-fri-roots final rest
    composition-fri-roots  $\neq \square$ 
    accepted-with-partial-trace-openings s out fr query-idxs

```

```

    trace-openings
  accepted-with-partial-composition-openings s out
    (hd composition-fri-roots) query-idxs composition-openings
  partial-trace-table-candidate trace-table trace-openings
  partial-composition-table-candidate composition-table
    composition-openings
  merkle-root-binds-table fr trace-table final-state
  merkle-root-binds-table (hd composition-fri-roots) composition-table
    final-state
proof –
from bound obtain result final-state fr f-fri-roots f-final dg
  composition-fri-roots final rest where out-eq:
    out = Some (result, final-state)
and header:
  verifier-header-transcript s fr f-fri-roots f-final as dg
    composition-fri-roots final rest
and comp-nonempty: composition-fri-roots ≠ []
and trace-bind:
  merkle-root-binds-table fr trace-table final-state
and comp-bind:
  merkle-root-binds-table (hd composition-fri-roots) composition-table
    final-state
unfolding accepted-with-bound-tables-def by blast
have accepted-out: accepted out
  by (rule accepted-with-bound-tables-imp-accepted[OF bound])
have trace-len-mult: length trace-table = scale * clength
  using accepted-with-bound-tables-shapes(1)[OF bound]
  by (simp add: mult.commute)
have comp-len-mult: length composition-table = scale * clength
  using accepted-with-bound-tables-shapes(2)[OF bound]
  by (simp add: mult.commute)
have query-len: length query-idxs = rounds
  by (rule accepted-with-bound-tables-shapes(4)[OF bound])
have trace-exists:
   $\bigwedge i. i < \text{rounds} \implies$ 
   $\exists \text{openings.}$ 
    map opening-index openings = powers-scaled (query-idxs ! i)  $\wedge$ 
    map opening-value openings =
      map ((! trace-table) (powers-scaled (query-idxs ! i)))  $\wedge$ 
    partial-authenticated-table fr (scale * clength) openings
      final-state
proof –
fix i
assume i-bound: i < rounds
have idx-in: query-idxs ! i ∈ set query-idxs
  by (rule nth-mem) (use i-bound query-len in simp)
have idx-sample: query-idxs ! i ∈ query-sample-space
  by (rule accepted-with-bound-tables-query-sample-space[OF bound idx-in])
have idxs-bound:

```

```

 $\bigwedge idx. idx \in \text{set } (\text{powers-scaled } (\text{query-idxs } ! i)) \implies$ 
   $idx < \text{scale} * \text{clength}$ 
using query-sample-space-powers-scaled-bound[OF idx-sample]
by (simp add: mult.commute)
show
 $\exists \text{openings.}$ 
   $\text{map opening-index openings} = \text{powers-scaled } (\text{query-idxs } ! i) \wedge$ 
   $\text{map opening-value openings} =$ 
     $\text{map } (! \text{ trace-table}) (\text{powers-scaled } (\text{query-idxs } ! i)) \wedge$ 
     $\text{partial-authenticated-table fr } (\text{scale} * \text{clength}) \text{ openings}$ 
     $\text{final-state}$ 
proof (rule merkle-root-binds-table-authenticated-openings[
  OF trace-bind trace-len-mult idxs-bound])
fix openings
assume index-map:
   $\text{map opening-index openings} = \text{powers-scaled } (\text{query-idxs } ! i)$ 
and value-map:
   $\text{map opening-value openings} =$ 
     $\text{map } (! \text{ trace-table}) (\text{powers-scaled } (\text{query-idxs } ! i))$ 
and partial:
   $\text{partial-authenticated-table fr } (\text{scale} * \text{clength}) \text{ openings}$ 
   $\text{final-state}$ 
show ?thesis
by (intro exI[of - openings] conjI index-map value-map partial)
qed
qed
let ?trace-opening =  $\lambda i. \text{SOME openings.}$ 
   $\text{map opening-index openings} = \text{powers-scaled } (\text{query-idxs } ! i) \wedge$ 
   $\text{map opening-value openings} =$ 
     $\text{map } (! \text{ trace-table}) (\text{powers-scaled } (\text{query-idxs } ! i)) \wedge$ 
     $\text{partial-authenticated-table fr } (\text{scale} * \text{clength}) \text{ openings final-state}$ 
define trace-openings where
   $\text{trace-openings} = \text{map } ?\text{trace-opening } [0..<\text{rounds}]$ 
have trace-openings-len:  $\text{length trace-openings} = \text{rounds}$ 
unfolding trace-openings-def by simp
have trace-openings-props:
 $\bigwedge i. i < \text{rounds} \implies$ 
   $\text{map opening-index } (\text{trace-openings } ! i) =$ 
     $\text{powers-scaled } (\text{query-idxs } ! i) \wedge$ 
   $\text{map opening-value } (\text{trace-openings } ! i) =$ 
     $\text{map } (! \text{ trace-table}) (\text{powers-scaled } (\text{query-idxs } ! i)) \wedge$ 
     $\text{partial-authenticated-table fr } (\text{scale} * \text{clength})$ 
     $(\text{trace-openings } ! i) \text{ final-state}$ 
proof -
fix i
assume i-bound:  $i < \text{rounds}$ 
have some-prop:
   $\text{map opening-index } (? \text{trace-opening } i) = \text{powers-scaled } (\text{query-idxs } ! i) \wedge$ 
   $\text{map opening-value } (? \text{trace-opening } i) =$ 

```



```

    map (!) trace-table) (powers-scaled (query-idxs ! i)) ∧
    partial-authenticated-table fr (scale * clength)
    (?trace-opening i) final-state
  by (rule someI-ex[OF trace-exists[OF i-bound]])
show
  map opening-index (trace-openings ! i) =
    powers-scaled (query-idxs ! i) ∧
  map opening-value (trace-openings ! i) =
    map (!) trace-table) (powers-scaled (query-idxs ! i)) ∧
    partial-authenticated-table fr (scale * clength)
    (trace-openings ! i) final-state
  using some-prop i-bound unfolding trace-openings-def by simp
qed
have comp-exists:
  ∧i. i < rounds ⇒
    ∃ openings.
      map opening-index openings =
        [query-idxs ! i,
         fri-sibling-index (scale * clength) (query-idxs ! i)] ∧
      map opening-value openings =
        map (!) composition-table)
        [query-idxs ! i,
         fri-sibling-index (scale * clength) (query-idxs ! i)] ∧
      partial-authenticated-table (hd composition-fri-roots)
      (scale * clength) openings final-state
proof -
  fix i
  assume i-bound: i < rounds
  have idx-in: query-idxs ! i ∈ set query-idxs
    by (rule nth-mem) (use i-bound query-len in simp)
  have idx-bound: query-idxs ! i < scale * clength
    using accepted-with-bound-tables-shapes(5)[OF bound idx-in]
    by (simp add: mult.commute)
  have sibling-bound:
    fri-sibling-index (scale * clength) (query-idxs ! i) <
      scale * clength
    unfolding fri-sibling-index-def
    using eval-domain-size-pos by (simp add: mult.commute)
  have idxs-bound:
    ∧idx. idx ∈
      set [query-idxs ! i,
         fri-sibling-index (scale * clength) (query-idxs ! i)] ⇒
      idx < scale * clength
  proof -
    fix idx
    assume idx ∈
      set [query-idxs ! i,
         fri-sibling-index (scale * clength) (query-idxs ! i)]
    then have idx = query-idxs ! i ∨

```

```

    idx = fri-sibling-index (scale * clength) (query-idxs ! i)
  by simp
then show idx < scale * clength
  using idx-bound sibling-bound by blast
qed
show
  ∃ openings.
    map opening-index openings =
      [query-idxs ! i,
       fri-sibling-index (scale * clength) (query-idxs ! i)] ∧
    map opening-value openings =
      map (!) composition-table
      [query-idxs ! i,
       fri-sibling-index (scale * clength) (query-idxs ! i)] ∧
    partial-authenticated-table (hd composition-fri-roots)
      (scale * clength) openings final-state
proof (rule merkle-root-binds-table-authenticated-openings[
  OF comp-bind comp-len-mult idxs-bound])
fix openings
assume index-map:
  map opening-index openings =
    [query-idxs ! i,
     fri-sibling-index (scale * clength) (query-idxs ! i)]
and value-map:
  map opening-value openings =
    map (!) composition-table
    [query-idxs ! i,
     fri-sibling-index (scale * clength) (query-idxs ! i)]
and partial:
  partial-authenticated-table (hd composition-fri-roots)
    (scale * clength) openings final-state
show ?thesis
  by (intro exI[of - openings] conjI index-map value-map partial)
qed
qed
let ?composition-opening = λi. SOME openings.
  map opening-index openings =
    [query-idxs ! i,
     fri-sibling-index (scale * clength) (query-idxs ! i)] ∧
  map opening-value openings =
    map (!) composition-table
    [query-idxs ! i,
     fri-sibling-index (scale * clength) (query-idxs ! i)] ∧
  partial-authenticated-table (hd composition-fri-roots)
    (scale * clength) openings final-state
define composition-openings where
  composition-openings = map ?composition-opening [0..<rounds]
have comp-openings-len: length composition-openings = rounds
unfolding composition-openings-def by simp

```

```

have comp-openings-props:
   $\bigwedge i. i < \text{rounds} \implies$ 
    map opening-index (composition-openings ! i) =
      [query-idxs ! i,
        fri-sibling-index (scale * clength) (query-idxs ! i)]  $\wedge$ 
    map opening-value (composition-openings ! i) =
      map (!) composition-table
        [query-idxs ! i,
          fri-sibling-index (scale * clength) (query-idxs ! i)]  $\wedge$ 
    partial-authenticated-table (hd composition-fri-roots)
      (scale * clength) (composition-openings ! i) final-state
proof –
  fix i
  assume i-bound: i < rounds
  have some-prop:
    map opening-index (?composition-opening i) =
      [query-idxs ! i,
        fri-sibling-index (scale * clength) (query-idxs ! i)]  $\wedge$ 
    map opening-value (?composition-opening i) =
      map (!) composition-table
        [query-idxs ! i,
          fri-sibling-index (scale * clength) (query-idxs ! i)]  $\wedge$ 
    partial-authenticated-table (hd composition-fri-roots)
      (scale * clength) (?composition-opening i) final-state
  by (rule someI-ex[OF comp-exists[OF i-bound]])
  show
    map opening-index (composition-openings ! i) =
      [query-idxs ! i,
        fri-sibling-index (scale * clength) (query-idxs ! i)]  $\wedge$ 
    map opening-value (composition-openings ! i) =
      map (!) composition-table
        [query-idxs ! i,
          fri-sibling-index (scale * clength) (query-idxs ! i)]  $\wedge$ 
    partial-authenticated-table (hd composition-fri-roots)
      (scale * clength) (composition-openings ! i) final-state
    using some-prop i-bound unfolding composition-openings-def by simp
qed
have trace-partial:
  accepted-with-partial-trace-openings s out fr query-idxs
    trace-openings
proof –
  have indices:
     $\bigwedge i. i < \text{rounds} \implies$ 
      map opening-index (trace-openings ! i) =
        powers-scaled (query-idxs ! i)
    using trace-openings-props by simp
  have partials:
     $\bigwedge i. i < \text{rounds} \implies$ 
      partial-authenticated-table fr (scale * clength)

```

```

      (trace-openings ! i) final-state
    using trace-openings-props by simp
  show ?thesis
    unfolding accepted-with-partial-trace-openings-def
    by (intro conjI exI[of - result] exI[of - final-state])
      (use accepted-out out-eq query-len trace-openings-len indices partials
        in simp-all)
  qed
  have comp-partial:
    accepted-with-partial-composition-openings s out
      (hd composition-fri-roots) query-idxs composition-openings
  proof -
    have indices:
       $\bigwedge i. i < \text{rounds} \implies$ 
      map opening-index (composition-openings ! i) =
        [query-idxs ! i,
          fri-sibling-index (scale * clength) (query-idxs ! i)]
    using comp-openings-props by simp
    have partials:
       $\bigwedge i. i < \text{rounds} \implies$ 
      partial-authenticated-table (hd composition-fri-roots)
        (scale * clength) (composition-openings ! i) final-state
    using comp-openings-props by simp
    show ?thesis
      unfolding accepted-with-partial-composition-openings-def
      by (intro conjI exI[of - result] exI[of - final-state])
        (use accepted-out out-eq query-len comp-openings-len indices partials
          in simp-all)
  qed
  have trace-candidate:
    partial-trace-table-candidate trace-table trace-openings
  proof -
    have agrees:
       $\bigwedge i. i < \text{rounds} \implies$ 
      table-agrees-with-authenticated-openings trace-table
        (scale * clength) (trace-openings ! i)
    proof -
      fix i
      assume i-bound: i < rounds
      have idx-in: query-idxs ! i  $\in$  set query-idxs
        by (rule nth-mem) (use i-bound query-len in simp)
      have idx-sample: query-idxs ! i  $\in$  query-sample-space
        by (rule accepted-with-bound-tables-query-sample-space[
          OF bound idx-in])
      have idxs-bound:
         $\bigwedge \text{idx}. \text{idx} \in \text{set (powers-scaled (query-idxs ! i))} \implies$ 
        idx < scale * clength
      using query-sample-space-powers-scaled-bound[OF idx-sample]
      by (simp add: mult.commute)
    qed
  qed

```

```

show
  table-agrees-with-authenticated-openings trace-table
    (scale * clength) (trace-openings ! i)
proof -
  have index-map:
    map opening-index (trace-openings ! i) =
      powers-scaled (query-idxs ! i)
  using trace-openings-props[OF i-bound] by blast
  have value-map:
    map opening-value (trace-openings ! i) =
      map (!) trace-table (powers-scaled (query-idxs ! i))
  using trace-openings-props[OF i-bound] by blast
  show ?thesis
    by (rule table-agrees-with-authenticated-openings-from-maps[
      OF trace-len-mult index-map value-map idxs-bound])
qed
qed
show ?thesis
  unfolding partial-trace-table-candidate-def
  by (intro conjI allI impI)
    (use trace-len-mult trace-openings-len agrees in simp-all)
qed
have comp-candidate:
  partial-composition-table-candidate composition-table
    composition-openings
proof -
  have agrees:
     $\bigwedge i. i < \text{rounds} \implies$ 
    table-agrees-with-authenticated-openings composition-table
      (scale * clength) (composition-openings ! i)
  proof -
    fix i
    assume i-bound: i < rounds
    have idx-in: query-idxs ! i  $\in$  set query-idxs
      by (rule nth-mem) (use i-bound query-len in simp)
    have idx-bound: query-idxs ! i < scale * clength
      using accepted-with-bound-tables-shapes(5)[OF bound idx-in]
      by (simp add: mult.commute)
    have sibling-bound:
      fri-sibling-index (scale * clength) (query-idxs ! i) <
        scale * clength
    unfolding fri-sibling-index-def
    using eval-domain-size-pos by (simp add: mult.commute)
    have idxs-bound:
       $\bigwedge idx. idx \in$ 
      set [query-idxs ! i,
        fri-sibling-index (scale * clength) (query-idxs ! i)]  $\implies$ 
      idx < scale * clength
  proof -

```

```

fix idx
assume idx ∈
  set [query-idxs ! i,
    fri-sibling-index (scale * clength) (query-idxs ! i)]
then have idx = query-idxs ! i ∨
  idx = fri-sibling-index (scale * clength) (query-idxs ! i)
by simp
then show idx < scale * clength
  using idx-bound sibling-bound by blast
qed
show
  table-agrees-with-authenticated-openings composition-table
    (scale * clength) (composition-openings ! i)
proof –
  have index-map:
    map opening-index (composition-openings ! i) =
      [query-idxs ! i,
        fri-sibling-index (scale * clength) (query-idxs ! i)]
    using comp-openings-props[OF i-bound] by blast
  have value-map:
    map opening-value (composition-openings ! i) =
      map (!) composition-table
        [query-idxs ! i,
          fri-sibling-index (scale * clength) (query-idxs ! i)]
    using comp-openings-props[OF i-bound] by blast
  show ?thesis
    by (rule table-agrees-with-authenticated-openings-from-maps[
      OF comp-len-mult index-map value-map idxs-bound])
qed
qed
show ?thesis
  unfolding partial-composition-table-candidate-def
  by (intro conjI allI impI)
    (use comp-len-mult comp-openings-len agrees in simp-all)
qed
show ?thesis
  by (rule that[OF out-eq header comp-nonempty trace-partial comp-partial
    trace-candidate comp-candidate trace-bind comp-bind])
qed

lemma accepted-with-bound-tables-partial-opening-witnesses-Some:
assumes bound:
  accepted-with-bound-tables s (Some (result, final-state))
  trace-table composition-table as query-idxs
obtains fr f-fri-roots f-final dg composition-fri-roots final rest
  trace-openings composition-openings where
  verifier-header-transcript s fr f-fri-roots f-final as dg
  composition-fri-roots final rest
  composition-fri-roots ≠ []

```

```

accepted-with-partial-trace-openings s (Some (result, final-state))
  fr query-idxs trace-openings
accepted-with-partial-composition-openings s
  (Some (result, final-state)) (hd composition-fri-roots) query-idxs
  composition-openings
partial-trace-table-candidate trace-table trace-openings
partial-composition-table-candidate composition-table
  composition-openings
merkle-root-binds-table fr trace-table final-state
merkle-root-binds-table (hd composition-fri-roots) composition-table
  final-state
proof –
show ?thesis
proof (rule accepted-with-bound-tables-partial-opening-witnesses[OF bound])
fix result' final-state' fr f-fri-roots f-final dg
  composition-fri-roots final rest trace-openings composition-openings
assume out-eq:
  Some (result, final-state) = Some (result', final-state')
and header:
  verifier-header-transcript s fr f-fri-roots f-final as dg
  composition-fri-roots final rest
and comp-nonempty: composition-fri-roots ≠ []
and trace-partial:
  accepted-with-partial-trace-openings s
    (Some (result, final-state)) fr query-idxs trace-openings
and comp-partial:
  accepted-with-partial-composition-openings s
    (Some (result, final-state)) (hd composition-fri-roots) query-idxs
    composition-openings
and trace-candidate:
  partial-trace-table-candidate trace-table trace-openings
and comp-candidate:
  partial-composition-table-candidate composition-table
    composition-openings
and trace-bind:
  merkle-root-binds-table fr trace-table final-state'
and comp-bind:
  merkle-root-binds-table (hd composition-fri-roots) composition-table
    final-state'
have final-state-eq: final-state' = final-state
using out-eq by simp
have trace-bind-outer:
  merkle-root-binds-table fr trace-table final-state
using trace-bind final-state-eq by simp
have comp-bind-outer:
  merkle-root-binds-table (hd composition-fri-roots) composition-table
    final-state
using comp-bind final-state-eq by simp
show ?thesis

```

```

    by (rule that[OF header comp-nonempty trace-partial comp-partial
      trace-candidate comp-candidate trace-bind-outer comp-bind-outer])
  qed
qed
end
end

```

```

theory Soundness-Staged-Partial-Query
imports
  Soundness-Staged-Query-Prefix
  Soundness-Bound-Table-Partial-Openings
begin

```

Partial-opening query events for staged soundness.

This layer records query consistency directly from authenticated openings. It deliberately avoids the stronger, generally false claim that sampled openings determine unique complete trace and composition tables.

```

context soundness
begin

```

lemma *checked-staged-security-with-data-state-accepted-shape-query-chunks*:

```

assumes wf: staged-budget-wellformed budgets
and controlled: staged-adversary-controlled budgets A
and outcome:
  Some (((data, attacker-state), result), final-state) ∈
    set-dist
    (execute
      (checked-staged-security-experiment-with-data-state A)
      adversary-initial-state)
and shape:
  accepted-transcript-shape
  (verifier-state-from-adversary attacker-state
    (staged-proof-transcript data))
  (Some (result, final-state)) as query-idxs
shows
  as = staged-alphas data ∧
  (∃ raw-idxs.
    length raw-idxs = rounds ∧
    query-idxs = map (λraw. index (to-nat raw)) raw-idxs ∧
    (∀ i < rounds.
      verifier-query-round-chunk (query-idxs ! i)
      (staged-trace-fri-roots data)
      (staged-composition-fri-roots data)
      (staged-query-chunks data ! i)) ∧
    (∀ i < rounds.
      fmllookup (HashMap final-state)

```



```

      (QueryIndexChallenge i
        (state-after-query-chunks
          (staged-query-start-hash data)
          (staged-query-chunks data) i)) =
        Some (raw-idxs ! i)) ∧
      (∀ idx ∈ set query-idxs. idx < clength * scale))
proof –
  let ?s =
    verifier-state-from-adversary attacker-state
      (staged-proof-transcript data)
from checked-staged-security-experiment-with-data-state-outcomeE
  [OF outcome]
have builder:
  Some (data, attacker-state) ∈
    set-dist
      (execute (checked-staged-transcript-program A)
        adversary-initial-state)
by blast
have staged-header:
  verifier-header-transcript ?s
    (staged-trace-root data)
    (staged-trace-fri-roots data)
    (staged-trace-final data)
    (staged-alphas data)
    (staged-degree data)
    (staged-composition-fri-roots data)
    (staged-composition-final data)
    (List.concat (staged-query-chunks data))
by (rule checked-staged-transcript-program-outcome-header-transcript
  [OF wf controlled builder])
from accepted-transcript-shape-query-chunksE[OF shape]
obtain result' final-state' fr f-fri-roots f-final dg
  composition-fri-roots final rest raw-idxs query-chunks trailing
where out-eq:
  Some (result, final-state) = Some (result', final-state')
and header:
  verifier-header-transcript ?s fr f-fri-roots f-final as dg
  composition-fri-roots final rest
and len-raw: length raw-idxs = rounds
and query-idxs-eq:
  query-idxs = map (λraw. index (to-nat raw)) raw-idxs
and len-chunks: length query-chunks = rounds
and concat-chunks: List.concat query-chunks @ trailing = rest
and chunk-shape:
  ∧ i. i < rounds ⇒
    verifier-query-round-chunk (query-idxs ! i)
    f-fri-roots composition-fri-roots (query-chunks ! i)
and lookup:
  ∧ i. i < rounds ⇒

```

```

    fmlookup (HashMap final-state')
      (QueryIndexChallenge (PQueryCounter ?s + i)
        (state-after-query-chunks
          (verifier-header-state ?s fr f-fri-roots f-final as dg
            composition-fri-roots final)
          query-chunks i)) =
        Some (raw-idxs ! i)
  and idx-bound:
     $\forall \text{idx} \in \text{set query-idxs. idx} < \text{clength} * \text{scale}$ 
  by blast
  have header-eq:
    fr = staged-trace-root data  $\wedge$ 
    f-fri-roots = staged-trace-fri-roots data  $\wedge$ 
    f-final = staged-trace-final data  $\wedge$ 
    as = staged-alphas data  $\wedge$ 
    dg = staged-degree data  $\wedge$ 
    composition-fri-roots = staged-composition-fri-roots data  $\wedge$ 
    final = staged-composition-final data  $\wedge$ 
    rest = List.concat (staged-query-chunks data)
  using verifier-header-transcript-unique[OF staged-header header]
  by simp
  have final-state-eq: final-state' = final-state
  using out-eq by simp
  have header-state-eq:
    verifier-header-state ?s fr f-fri-roots f-final as dg
      composition-fri-roots final =
      staged-query-start-hash data
  using header-eq
  unfolding verifier-header-state-def verifier-header-messages-def
    staged-query-start-hash-def staged-composition-fri-start-hash-def
    staged-trace-fri-start-hash-def
  by simp
  from checked-staged-transcript-program-query-chunks-match-verifier-lengths
    [OF builder]
  obtain staged-query-idxs where
    staged-match:
      staged-query-chunks-match-verifier-lengths data staged-query-idxs
  by blast
  have query-idxs-len: length query-idxs = rounds
  using query-idxs-eq len-raw by simp
  have staged-match-query-idxs:
    staged-query-chunks-match-verifier-lengths data query-idxs
  by (rule staged-query-chunks-match-verifier-lengths-transfer
    [OF staged-match query-idxs-len])
  have parser-chunk-len:
     $\wedge i. i < \text{rounds} \implies$ 
    length (query-chunks ! i) =
      verifier-query-round-transcript-length (query-idxs ! i)
      (staged-trace-fri-roots data)

```

```

      (staged-composition-fri-roots data)
proof -
  fix i
  assume i-bound: i < rounds
  show
    length (query-chunks ! i) =
      verifier-query-round-transcript-length (query-idxs ! i)
        (staged-trace-fri-roots data)
        (staged-composition-fri-roots data)
  using verifier-query-round-chunk-length[OF chunk-shape[OF i-bound]]
    header-eq by simp
qed
have concat-staged:
  List.concat query-chunks @ trailing =
    List.concat (staged-query-chunks data)
  using concat-chunks header-eq by simp
have query-chunks-eq:
  query-chunks = staged-query-chunks data ∧ trailing = []
  by (rule query-chunks-eq-staged-if-matching-lengths
    [OF len-chunks concat-staged parser-chunk-len
      staged-match-query-idxs])
have chunk-shape-staged:
  ∧i. i < rounds ⇒
    verifier-query-round-chunk (query-idxs ! i)
      (staged-trace-fri-roots data)
      (staged-composition-fri-roots data)
      (staged-query-chunks data ! i)
  using chunk-shape header-eq query-chunks-eq by simp
have lookup-staged:
  ∧i. i < rounds ⇒
    fmlookup (HashMap final-state)
      (QueryIndexChallenge i
        (state-after-query-chunks
          (staged-query-start-hash data)
          (staged-query-chunks data) i)) =
      Some (raw-idxs ! i)
proof -
  fix i
  assume i-bound: i < rounds
  have lookup-i:
    fmlookup (HashMap final-state')
      (QueryIndexChallenge (PQueryCounter ?s + i)
        (state-after-query-chunks
          (verifier-header-state ?s fr f-fri-roots f-final as dg
            composition-fri-roots final)
          query-chunks i)) =
      Some (raw-idxs ! i)
  by (rule lookup[OF i-bound])
show

```

```

    fmlookup (HashMap final-state)
      (QueryIndexChallenge i
        (state-after-query-chunks
          (staged-query-start-hash data)
          (staged-query-chunks data) i)) =
    Some (raw-idxs ! i)
  using lookup-i final-state-eq header-state-eq query-chunks-eq by simp
qed
show ?thesis
  using header-eq len-raw query-idxs-eq chunk-shape-staged lookup-staged
    idx-bound
  by blast
qed

lemma checked-staged-security-with-data-state-query-bad-round-chunkE:
  assumes wf: staged-budget-wellformed budgets
  and controlled: staged-adversary-controlled budgets A
  and support:
    Some (((data, attacker-state), result), final-state) ∈
      set-dist
        (execute
          (checked-staged-security-experiment-with-data-state A)
          adversary-initial-state)
  and bad:
    staged-security-with-data-state-query-bad-hit
      (Some (((data, attacker-state), result), final-state))
  obtains i trace-table composition-table query-idxs raw-idxs where
    i < rounds
    accepted-with-bound-tables
      (verifier-state-from-adversary attacker-state
        (staged-proof-transcript data))
      (Some (result, final-state))
    trace-table composition-table (staged-alphas data) query-idxs
    query-idxs ! i ∈
      query-sampling-success-space trace-table composition-table
        (staged-alphas data)
    length raw-idxs = rounds
    query-idxs = map (λraw. index (to-nat raw)) raw-idxs
    verifier-query-round-chunk (query-idxs ! i)
      (staged-trace-fri-roots data)
      (staged-composition-fri-roots data)
      (staged-query-chunks data ! i)
    fmlookup (HashMap final-state)
      (QueryIndexChallenge i
        (state-after-query-chunks
          (staged-query-start-hash data) (staged-query-chunks data) i)) =
    Some (raw-idxs ! i)
proof -
  let ?s =

```

```

    verifier-state-from-adversary attacker-state
      (staged-proof-transcript data)
from bad[unfolded staged-security-with-data-state-query-bad-hit-def
    Let-def]
have query-bad: query-bad ?s (Some (result, final-state))
  by simp
from query-bad-imp-query-index-round-set-hit[OF query-bad]
obtain i trace-table composition-table as query-idxs where
  bound:
    accepted-with-bound-tables ?s (Some (result, final-state))
    trace-table composition-table as query-idxs
and i-bound: i < rounds
and query-hit:
  query-idxs ! i ∈
    query-sampling-success-space trace-table composition-table as
unfolding query-index-round-set-hit-def
  query-index-round-set-hit-at-def
by blast
have shape:
  accepted-transcript-shape ?s (Some (result, final-state))
    as query-idxs
using bound
unfolding accepted-with-bound-tables-def accepted-with-tables-def
by simp
from checked-staged-security-with-data-state-accepted-shape-query-chunks
  [OF wf controlled support shape]
obtain raw-idxs where
  as-eq: as = staged-alphas data
and len-raw: length raw-idxs = rounds
and query-idxs-eq:
  query-idxs = map (λraw. index (to-nat raw)) raw-idxs
and chunk-shape:
   $\bigwedge j. j < rounds \implies$ 
    verifier-query-round-chunk (query-idxs ! j)
      (staged-trace-fri-roots data)
      (staged-composition-fri-roots data)
      (staged-query-chunks data ! j)
and lookup:
   $\bigwedge j. j < rounds \implies$ 
    fmllookup (HashMap final-state)
      (QueryIndexChallenge j
        (state-after-query-chunks
          (staged-query-start-hash data)
          (staged-query-chunks data) j)) =
      Some (raw-idxs ! j)
by blast
have bound-staged:
  accepted-with-bound-tables ?s (Some (result, final-state))
    trace-table composition-table (staged-alphas data) query-idxs

```

```

    using bound as-eq by simp
  have query-hit-staged:
    query-idxs ! i ∈
      query-sampling-success-space trace-table composition-table
        (staged-alphas data)
    using query-hit as-eq by simp
  show ?thesis
  by (rule that[OF i-bound bound-staged query-hit-staged len-raw
    query-idxs-eq chunk-shape[OF i-bound] lookup[OF i-bound]])
qed

```

lemma *ntimes-outcome-decomp-at:*

```

  assumes outcome:
    Some (results, t) ∈ set-dist (execute (ntimes m n) s)
  and i-bound: i < n
  shows
    ∃ prefix x suffix s-i s-suc.
      Some (prefix, s-i) ∈ set-dist (execute (ntimes m i) s) ∧
      Some (x, s-suc) ∈ set-dist (execute m s-i) ∧
      Some (suffix, t) ∈
        set-dist (execute (ntimes m (n - Suc i)) s-suc) ∧
      results = prefix @ x # suffix
  using outcome i-bound
proof (induction n arbitrary: i s results t)
  case 0
  then show ?case by simp
next
  case (Suc n)
  from Suc.prem1(1) obtain x results' s1 where
    head: Some (x, s1) ∈ set-dist (execute m s)
  and tail:
    Some (results', t) ∈ set-dist (execute (ntimes m n) s1)
  and results-eq: results = x # results'
  by (auto elim!: set-dist-bindE)
  show ?case
proof (cases i)
  case 0
  have prefix0:
    Some ([], s) ∈ set-dist (execute (ntimes m i) s)
  using 0 by simp
  have suffix0:
    Some (results', t) ∈
      set-dist (execute (ntimes m (Suc n - Suc i)) s1)
  using tail 0 by simp
  show ?thesis
  using prefix0 head suffix0 results-eq
  by (intro exI[of - []] exI[of - x] exI[of - results']
    exI[of - s] exI[of - s1]) simp
next

```

```

case (Suc j)
have j-bound:  $j < n$ 
  using Suc.premis(2) Suc by simp
from Suc.IH[OF tail j-bound]
obtain prefix y suffix s-j s-suc where
  prefix:
    Some (prefix, s-j)  $\in$  set-dist (execute (ntimes m j) s1)
  and round: Some (y, s-suc)  $\in$  set-dist (execute m s-j)
  and suffix:
    Some (suffix, t)  $\in$ 
      set-dist (execute (ntimes m (n - Suc j)) s-suc)
  and results'-eq: results' = prefix @ y # suffix
  by blast
have prefix-suc:
  Some (x # prefix, s-j)  $\in$ 
    set-dist (execute (ntimes m (Suc j)) s)
  unfolding ntimes.simps
  apply (rule set-dist-bindI[OF head])
  apply (rule set-dist-bindI[OF prefix])
  apply simp
  done
have n-minus:  $\text{Suc } n - \text{Suc } (\text{Suc } j) = n - \text{Suc } j$ 
  by simp
have suffix-suc:
  Some (suffix, t)  $\in$ 
    set-dist (execute (ntimes m (Suc n - Suc i)) s-suc)
  using suffix n-minus Suc by simp
show ?thesis
  using prefix-suc round suffix-suc results-eq results'-eq Suc
  by (intro exI[of - x # prefix] exI[of - y] exI[of - suffix]
    exI[of - s-j] exI[of - s-suc]) simp
qed
qed

```

definition partial-query-round-consistent

```

:: 'f authenticated-opening list list  $\Rightarrow$ 
  'f authenticated-opening list list  $\Rightarrow$  'f list  $\Rightarrow$  nat  $\Rightarrow$ 
  nat  $\Rightarrow$  bool

```

where

```

partial-query-round-consistent trace-openings composition-openings as i idx
 $\longleftrightarrow$ 
   $i < \text{rounds} \wedge$ 
   $\text{idx} \in \text{query-sample-space} \wedge$ 
   $\text{length } \text{trace-openings} = \text{rounds} \wedge$ 
   $\text{length } \text{composition-openings} = \text{rounds} \wedge$ 
   $\text{map } \text{opening-index } (\text{trace-openings } ! i) = \text{powers-scaled } \text{idx} \wedge$ 
   $\text{map } \text{opening-index } (\text{composition-openings } ! i) =$ 
     $[\text{idx}, \text{fri-sibling-index } (\text{scale} * \text{clength}) \text{ idx}] \wedge$ 
   $\text{composition-openings } ! i \neq [] \wedge$ 

```

$\text{opening-value } ((\text{composition-openings } ! i) ! 0) =$
 $\text{cp-eval as } (\text{map opening-value } (\text{trace-openings } ! i))$
 $(h \wedge \text{idx} * \text{shift})$

definition *partial-query-success-indices-at*

$:: 'f \text{ authenticated-opening list list} \Rightarrow$
 $'f \text{ authenticated-opening list list} \Rightarrow 'f \text{ list} \Rightarrow \text{nat} \Rightarrow$
 nat set

where

$\text{partial-query-success-indices-at trace-openings composition-openings as } i =$
 $\{ \text{idx} \in \text{query-sample-space.}$
 $\text{partial-query-round-consistent trace-openings composition-openings as}$
 $i \text{ idx} \}$

lemma *partial-query-success-indices-at-subset:*

$\text{partial-query-success-indices-at trace-openings composition-openings as } i$
 $\subseteq \text{query-sample-space}$

unfolding *partial-query-success-indices-at-def* **by** *auto*

lemma *partial-query-success-indices-at-subset-query-sampling-success-space:*

assumes *trace-candidate:*

$\text{partial-trace-table-candidate trace-table trace-openings}$

and *comp-candidate:*

$\text{partial-composition-table-candidate composition-table}$
 $\text{composition-openings}$

and *trace-low:* $\text{trace-table-low-degree trace-table}$

and *comp-low:* $\text{composition-table-low-degree maxDegree composition-table}$

and *not-all:*

$\neg \text{all-queries-consistent trace-table composition-table as}$

shows

$\text{partial-query-success-indices-at trace-openings composition-openings as } i$
 $\subseteq \text{query-sampling-success-space trace-table composition-table as}$

proof

fix *idx*

assume *idx-in:*

$\text{idx} \in \text{partial-query-success-indices-at trace-openings}$
 $\text{composition-openings as } i$

have *round:*

$\text{partial-query-round-consistent trace-openings composition-openings as}$
 $i \text{ idx}$

using *idx-in* **unfolding** *partial-query-success-indices-at-def* **by** *simp*

have *i-bound:* $i < \text{rounds}$

using *round* **unfolding** *partial-query-round-consistent-def* **by** *simp*

have *idx-sample:* $\text{idx} \in \text{query-sample-space}$

using *round* **unfolding** *partial-query-round-consistent-def* **by** *simp*

have *trace-indices:*

$\text{map opening-index } (\text{trace-openings } ! i) = \text{powers-scaled idx}$

using *round* **unfolding** *partial-query-round-consistent-def* **by** *simp*

have *comp-indices:*


```

map opening-index (composition-openings ! i) =
  [idx, fri-sibling-index (scale * clength) idx]
using round unfolding partial-query-round-consistent-def by simp
have comp-value-opened:
  opening-value ((composition-openings ! i) ! 0) =
    cp-eval as (map opening-value (trace-openings ! i))
      (h ^ idx * shift)
using round unfolding partial-query-round-consistent-def by simp
have trace-len: length trace-table = scale * clength
using trace-candidate unfolding partial-trace-table-candidate-def
by simp
have comp-len: length composition-table = scale * clength
using comp-candidate unfolding partial-composition-table-candidate-def
by simp
have trace-values:
  map (! trace-table) (powers-scaled idx) =
    map opening-value (trace-openings ! i)
by (rule partial-trace-table-candidate-query-values
  [OF trace-candidate i-bound trace-indices])
have comp-value:
  composition-table ! idx =
    opening-value ((composition-openings ! i) ! 0)
by (rule partial-composition-table-candidate-query-value
  [OF comp-candidate i-bound comp-indices])
have idx-domain: idx < clength * scale
by (rule query-sample-space-less-domain [OF idx-sample])
have powers-bound:
  ∀ j ∈ set (powers-scaled idx). j < length trace-table
using query-sample-space-powers-scaled-bound [OF idx-sample] trace-len
by (simp add: mult.commute)
have consistent:
  query-consistent-at trace-table composition-table as idx
unfolding query-consistent-at-def
using idx-domain trace-len comp-len powers-bound trace-values comp-value
  comp-value-opened
by (simp add: mult.commute)
show idx ∈ query-sampling-success-space trace-table composition-table as
unfolding query-sampling-success-space-def query-agreement-indices-def
using idx-sample consistent trace-low comp-low not-all by simp
qed

```

```

lemma partial-query-round-consistent-imp-query-consistent-at:
assumes trace-candidate:
  partial-trace-table-candidate trace-table trace-openings
and comp-candidate:
  partial-composition-table-candidate composition-table
  composition-openings
and round:
  partial-query-round-consistent trace-openings composition-openings

```

```

    as i idx
  shows query-consistent-at trace-table composition-table as idx
proof -
  have i-bound:  $i < \text{rounds}$ 
    using round unfolding partial-query-round-consistent-def by simp
  have idx-sample:  $\text{idx} \in \text{query-sample-space}$ 
    using round unfolding partial-query-round-consistent-def by simp
  have trace-indices:
    map opening-index (trace-openings ! i) = powers-scaled idx
    using round unfolding partial-query-round-consistent-def by simp
  have comp-indices:
    map opening-index (composition-openings ! i) =
      [idx, fri-sibling-index (scale * clength) idx]
    using round unfolding partial-query-round-consistent-def by simp
  have comp-value-opened:
    opening-value ((composition-openings ! i) ! 0) =
      cp-eval as (map opening-value (trace-openings ! i))
        ( $h \wedge \text{idx} * \text{shift}$ )
    using round unfolding partial-query-round-consistent-def by simp
  have trace-len:  $\text{length trace-table} = \text{scale} * \text{clength}$ 
    using trace-candidate unfolding partial-trace-table-candidate-def
    by simp
  have comp-len:  $\text{length composition-table} = \text{scale} * \text{clength}$ 
    using comp-candidate unfolding partial-composition-table-candidate-def
    by simp
  have trace-values:
    map ((! trace-table) (powers-scaled idx)) =
      map opening-value (trace-openings ! i)
    by (rule partial-trace-table-candidate-query-values
        [OF trace-candidate i-bound trace-indices])
  have comp-value:
    composition-table ! idx =
      opening-value ((composition-openings ! i) ! 0)
    by (rule partial-composition-table-candidate-query-value
        [OF comp-candidate i-bound comp-indices])
  have idx-domain:  $\text{idx} < \text{clength} * \text{scale}$ 
    by (rule query-sample-space-less-domain [OF idx-sample])
  have powers-bound:
     $\forall j \in \text{set (powers-scaled idx)}. j < \text{length trace-table}$ 
    using query-sample-space-powers-scaled-bound [OF idx-sample] trace-len
    by (simp add: mult.commute)
  show ?thesis
    unfolding query-consistent-at-def
    using idx-domain trace-len comp-len powers-bound trace-values comp-value
      comp-value-opened
    by (simp add: mult.commute)
qed

```

lemma *partial-query-success-indices-at-fraction-bound-if-candidate-low-degree:*

```

assumes trace-candidate:
  partial-trace-table-candidate trace-table trace-openings
and comp-candidate:
  partial-composition-table-candidate composition-table
  composition-openings
and trace-low: trace-table-low-degree trace-table
and comp-low: composition-table-low-degree maxDegree composition-table
and not-all:
   $\neg$  all-queries-consistent trace-table composition-table as
shows
  nnreal
    (card
      (partial-query-success-indices-at trace-openings composition-openings
        as i)) /
    nnreal (card query-sample-space)  $\leq$  query-error-bound
proof –
  have subset:
    partial-query-success-indices-at trace-openings composition-openings as i
       $\subseteq$  query-sampling-success-space trace-table composition-table as
  by (rule partial-query-success-indices-at-subset-query-sampling-success-space
    [OF trace-candidate comp-candidate trace-low comp-low not-all])
  have finite-success:
    finite (query-sampling-success-space trace-table composition-table as)
  by (rule finite-subset[OF query-sampling-success-space-subset
    finite-query-sample-space])
  have card-le:
    card
      (partial-query-success-indices-at trace-openings composition-openings
        as i)  $\leq$ 
      card (query-sampling-success-space trace-table composition-table as)
  by (rule card-mono[OF finite-success subset])
  have nnreal
    (card
      (partial-query-success-indices-at trace-openings
        composition-openings as i)) /
    nnreal (card query-sample-space)  $\leq$ 
    nnreal (card (query-sampling-success-space trace-table
      composition-table as)) /
    nnreal (card query-sample-space)
  by (rule nnreal-nat-divide-right-mono[OF card-le])
  also have ...  $\leq$  query-error-bound
  by (rule query-sampling-success-space-fraction-bound-query-sample-space)
  finally show ?thesis .
qed

```

```

lemma partial-query-round-consistentI:
assumes i-bound: i < rounds
and idx-sample: idx  $\in$  query-sample-space
and trace-len: length trace-openings = rounds

```

```

and comp-len: length composition-openings = rounds
and trace-indices:
  map opening-index (trace-openings ! i) = powers-scaled idx
and comp-indices:
  map opening-index (composition-openings ! i) =
    [idx, fri-sibling-index (scale * clength) idx]
and comp-value:
  opening-value ((composition-openings ! i) ! 0) =
    cp-eval as (map opening-value (trace-openings ! i))
    (h ^ idx * shift)
shows
  partial-query-round-consistent trace-openings composition-openings as
    i idx
proof -
have comp-round-nonempty: composition-openings ! i ≠ []
proof -
have length (map opening-index (composition-openings ! i)) =
  length [idx, fri-sibling-index (scale * clength) idx]
using arg-cong[OF comp-indices, of length] by simp
then have length (composition-openings ! i) = 2
by simp
then show ?thesis by auto
qed
show ?thesis
unfolding partial-query-round-consistent-def
using i-bound idx-sample trace-len comp-len trace-indices comp-indices
  comp-round-nonempty comp-value
by simp
qed

lemma partial-query-success-indices-atI:
assumes partial-query-round-consistent trace-openings composition-openings
  as i idx
shows
  idx ∈ partial-query-success-indices-at trace-openings
    composition-openings as i
using assms unfolding partial-query-success-indices-at-def
  partial-query-round-consistent-def by simp

definition partial-query-openings-consistent
  :: 'f authenticated-opening list ⇒
    'f authenticated-opening list ⇒ 'f list ⇒ nat ⇒ bool
where
  partial-query-openings-consistent trace-openings composition-openings as idx
    ⇔
    idx ∈ query-sample-space ∧
    map opening-index trace-openings = powers-scaled idx ∧
    map opening-index composition-openings =
      [idx, fri-sibling-index (scale * clength) idx] ∧

```

$\text{composition-openings} \neq [] \wedge$
 $\text{opening-value } (\text{composition-openings} ! 0) =$
 $\text{cp-eval as } (\text{map opening-value trace-openings}) (h \wedge \text{idx} * \text{shift})$

lemma *partial-query-openings-consistentI*:

assumes *idx-sample*: $\text{idx} \in \text{query-sample-space}$

and *trace-indices*:

$\text{map opening-index trace-openings} = \text{powers-scaled idx}$

and *comp-indices*:

$\text{map opening-index composition-openings} =$
 $[\text{idx}, \text{fri-sibling-index } (\text{scale} * \text{clength}) \text{idx}]$

and *comp-value*:

$\text{opening-value } (\text{composition-openings} ! 0) =$
 $\text{cp-eval as } (\text{map opening-value trace-openings}) (h \wedge \text{idx} * \text{shift})$

shows

$\text{partial-query-openings-consistent trace-openings composition-openings}$
 as idx

proof –

have *comp-nonempty*: $\text{composition-openings} \neq []$

proof –

have $\text{length } (\text{map opening-index composition-openings}) =$
 $\text{length } [\text{idx}, \text{fri-sibling-index } (\text{scale} * \text{clength}) \text{idx}]$

using *arg-cong*[*OF comp-indices, of length*] **by** *simp*

then have $\text{length } \text{composition-openings} = 2$

by *simp*

then show *?thesis* **by** *auto*

qed

show *?thesis*

unfolding *partial-query-openings-consistent-def*

using *idx-sample trace-indices comp-indices comp-nonempty comp-value*

by *simp*

qed

lemma *partial-query-round-consistent-single-roundI*:

assumes *i-bound*: $i < \text{rounds}$

and *consistent*:

$\text{partial-query-openings-consistent trace-openings composition-openings}$
 as idx

shows

$\text{partial-query-round-consistent}$
 $((\text{replicate rounds } []) [i := \text{trace-openings}])$
 $((\text{replicate rounds } []) [i := \text{composition-openings}])$
 $\text{as } i \text{ idx}$

proof –

have *idx-sample*: $\text{idx} \in \text{query-sample-space}$

using *consistent* **unfolding** *partial-query-openings-consistent-def* **by** *simp*

have *trace-indices*:

$\text{map opening-index trace-openings} = \text{powers-scaled idx}$

using *consistent* **unfolding** *partial-query-openings-consistent-def* **by** *simp*

```

have comp-indices:
  map opening-index composition-openings =
    [idx, fri-sibling-index (scale * clength) idx]
  using consistent unfolding partial-query-openings-consistent-def by simp
have comp-value:
  opening-value (composition-openings ! 0) =
    cp-eval as (map opening-value trace-openings) (h ^ idx * shift)
  using consistent unfolding partial-query-openings-consistent-def by simp
have comp-nonempty: composition-openings ≠ []
  using consistent unfolding partial-query-openings-consistent-def by simp
have comp-round-value:
  opening-value
    ((((replicate rounds []) [i := composition-openings]) ! i) ! 0) =
    cp-eval as
      (map opening-value
        ((((replicate rounds []) [i := trace-openings]) ! i))
        (h ^ idx * shift))
  using i-bound comp-value by simp
show ?thesis
  unfolding partial-query-round-consistent-def
  using i-bound idx-sample trace-indices comp-indices comp-nonempty
    comp-round-value
  by simp
qed

```

```

lemma powers-scaled-hd:
  hd (powers-scaled idx) = idx
  unfolding powers-scaled-def
  using powers-pos by (simp add: upt-conv-Cons)

```

```

lemma partial-query-openings-consistent-index-unique:
  assumes left:
    partial-query-openings-consistent trace-openings
    composition-openings as idx
  and right:
    partial-query-openings-consistent trace-openings
    composition-openings as idx'
  shows idx = idx'
proof –
  have powers-scaled idx = powers-scaled idx'
    using left right
    unfolding partial-query-openings-consistent-def by simp
  then have hd (powers-scaled idx) = hd (powers-scaled idx')
    by simp
  then show ?thesis
    by (simp add: powers-scaled-hd)
qed

```

```

lemma partial-query-openings-consistent-indices-card-le-one:

```

```

card
  {idx ∈ query-sample-space.
   partial-query-openings-consistent trace-openings composition-openings
   as idx} ≤ 1
proof —
let ?S =
  {idx ∈ query-sample-space.
   partial-query-openings-consistent trace-openings composition-openings
   as idx}
have finite-S: finite ?S
  by simp
have unique:  $\bigwedge idx\ idx'. idx \in ?S \implies idx' \in ?S \implies idx = idx'$ 
  by (rule partial-query-openings-consistent-index-unique) auto
show ?thesis
proof (cases  $\exists idx. idx \in ?S$ )
case False
  have empty: ?S = {}
  proof (rule equals0I)
    fix idx
    assume idx ∈ ?S
    then show False
      using False by blast
  qed
  show ?thesis
    by (subst empty, simp)
next
case True
  then obtain idx where idx-in: idx ∈ ?S
    by blast
  have subset-single: ?S ⊆ {idx}
    using idx-in unique by blast
  have card ?S ≤ card {idx}
    by (rule card-mono) (use subset-single in simp-all)
  then show ?thesis
    by simp
qed
qed

lemma partial-query-openings-consistent-imp-round-consistent-list-update:
  assumes i-bound: i < rounds
  and consistent:
    partial-query-openings-consistent trace-openings composition-openings
    as idx
  shows
    partial-query-round-consistent
      ((replicate rounds []) [i := trace-openings])
      ((replicate rounds []) [i := composition-openings])
      as i idx
proof —

```

```

have idx-sample:  $idx \in \text{query-sample-space}$ 
  using consistent
  unfolding partial-query-openings-consistent-def by simp
have trace-indices:
  map opening-index
    (((replicate rounds []) [ $i := \text{trace-openings}$ ]) !  $i$ ) =
    powers-scaled idx
  using consistent i-bound
  unfolding partial-query-openings-consistent-def by simp
have comp-indices:
  map opening-index
    (((replicate rounds []) [ $i := \text{composition-openings}$ ]) !  $i$ ) =
    [idx, fri-sibling-index ( $\text{scale} * \text{clength}$ ) idx]
  using consistent i-bound
  unfolding partial-query-openings-consistent-def by simp
have comp-value:
  opening-value
    ((((replicate rounds []) [ $i := \text{composition-openings}$ ]) !  $i$ ) ! 0) =
    cp-eval as
      (map opening-value
        (((replicate rounds []) [ $i := \text{trace-openings}$ ]) !  $i$ ))
      ( $h \wedge idx * \text{shift}$ )
  using consistent i-bound
  unfolding partial-query-openings-consistent-def by simp
show ?thesis
  by (rule partial-query-round-consistentI
    [OF i-bound idx-sample])
    (use i-bound trace-indices comp-indices comp-value in simp-all)
qed

lemma partial-query-openings-consistent-indices-subset-round-success-list-update:
  assumes i-bound:  $i < \text{rounds}$ 
  shows
    { $idx \in \text{query-sample-space}$ .
      partial-query-openings-consistent trace-openings composition-openings
      as idx}
     $\subseteq$ 
    partial-query-success-indices-at
      ((replicate rounds []) [ $i := \text{trace-openings}$ ])
      ((replicate rounds []) [ $i := \text{composition-openings}$ ])
      as i
  unfolding partial-query-success-indices-at-def
  using partial-query-openings-consistent-imp-round-consistent-list-update
    [OF i-bound]
  by blast

lemma partial-query-openings-consistent-indices-fraction-bound-if-candidate-low-degree:
  assumes i-bound:  $i < \text{rounds}$ 
  and trace-candidate:

```



```

    partial-trace-table-candidate trace-table
      ((replicate rounds []) [i := trace-openings])
  and comp-candidate:
    partial-composition-table-candidate composition-table
      ((replicate rounds []) [i := composition-openings])
  and trace-low: trace-table-low-degree trace-table
  and comp-low: composition-table-low-degree maxDegree composition-table
  and not-all:
    ¬ all-queries-consistent trace-table composition-table as
shows
  nnreal
    (card
      {idx ∈ query-sample-space.
        partial-query-openings-consistent trace-openings
          composition-openings as idx}) /
    nnreal (card query-sample-space) ≤ query-error-bound
proof -
  let ?trace-openings = (replicate rounds []) [i := trace-openings]
  let ?composition-openings =
    (replicate rounds []) [i := composition-openings]
  let ?flat =
    {idx ∈ query-sample-space.
      partial-query-openings-consistent trace-openings composition-openings
        as idx}
  let ?round =
    partial-query-success-indices-at ?trace-openings
      ?composition-openings as i
  have subset: ?flat ⊆ ?round
  by (rule
    partial-query-openings-consistent-indices-subset-round-success-list-update
      [OF i-bound])
  have finite-round: finite ?round
  by (rule finite-subset[OF partial-query-success-indices-at-subset
    finite-query-sample-space])
  have card-le: card ?flat ≤ card ?round
  by (rule card-mono[OF finite-round subset])
  have nnreal (card ?flat) / nnreal (card query-sample-space) ≤
    nnreal (card ?round) / nnreal (card query-sample-space)
  by (rule nnreal-nat-divide-right-mono[OF card-le])
  also have ... ≤ query-error-bound
  by (rule partial-query-success-indices-at-fraction-bound-if-candidate-low-degree
    [OF trace-candidate comp-candidate trace-low comp-low not-all])
  finally show ?thesis .
qed

lemma partial-trace-table-candidate-single-roundI:
  assumes i-bound: i < rounds
  and len-table: length trace-table = scale * clength
  and agrees:

```

```

    table-agrees-with-authenticated-openings trace-table
      (scale * clength) trace-openings
  shows
    partial-trace-table-candidate trace-table
      ((replicate rounds []) [i := trace-openings])
  unfolding partial-trace-table-candidate-def
proof (intro conjI allI impI)
  show length trace-table = scale * clength
    by (rule len-table)
next
  show length ((replicate rounds []) [i := trace-openings]) = rounds
    using i-bound by simp
next
  fix j
  assume j-bound: j < rounds
  show
    table-agrees-with-authenticated-openings trace-table
      (scale * clength)
      (((replicate rounds []) [i := trace-openings]) ! j)
proof (cases j = i)
  case True
  then show ?thesis
    using agrees i-bound by simp
next
  case False
  then show ?thesis
    using j-bound len-table
    unfolding table-agrees-with-authenticated-openings-def
    by simp
qed
qed

lemma partial-composition-table-candidate-single-roundI:
  assumes i-bound: i < rounds
  and len-table: length composition-table = scale * clength
  and agrees:
    table-agrees-with-authenticated-openings composition-table
      (scale * clength) composition-openings
  shows
    partial-composition-table-candidate composition-table
      ((replicate rounds []) [i := composition-openings])
  unfolding partial-composition-table-candidate-def
proof (intro conjI allI impI)
  show length composition-table = scale * clength
    by (rule len-table)
next
  show length ((replicate rounds []) [i := composition-openings]) =
    rounds
    using i-bound by simp

```

```

next
  fix j
  assume j-bound:  $j < \text{rounds}$ 
  show
    table-agrees-with-authenticated-openings composition-table
      (scale * clength)
      (((replicate rounds []) [i := composition-openings]) ! j)
  proof (cases j = i)
    case True
      then show ?thesis
        using agrees i-bound by simp
    next
      case False
      then show ?thesis
        using j-bound len-table
        unfolding table-agrees-with-authenticated-openings-def
        by simp
  qed
qed

lemma partial-query-openings-consistent-indices-fraction-bound-if-tables-agree-low-degree:
  assumes i-bound:  $i < \text{rounds}$ 
  and trace-len:  $\text{length trace-table} = \text{scale} * \text{clength}$ 
  and comp-len:  $\text{length composition-table} = \text{scale} * \text{clength}$ 
  and trace-agrees:
    table-agrees-with-authenticated-openings trace-table
      (scale * clength) trace-openings
  and comp-agrees:
    table-agrees-with-authenticated-openings composition-table
      (scale * clength) composition-openings
  and trace-low: trace-table-low-degree trace-table
  and comp-low: composition-table-low-degree maxDegree composition-table
  and not-all:
     $\neg \text{all-queries-consistent trace-table composition-table as}$ 
  shows
    nnreal
      (card
        {idx  $\in$  query-sample-space.
          partial-query-openings-consistent trace-openings
            composition-openings as idx}) /
      nnreal (card query-sample-space)  $\leq$  query-error-bound
  proof -
    have trace-candidate:
      partial-trace-table-candidate trace-table
        ((replicate rounds []) [i := trace-openings])
    by (rule partial-trace-table-candidate-single-roundI
      [OF i-bound trace-len trace-agrees])
    have comp-candidate:
      partial-composition-table-candidate composition-table

```

```

    ((replicate rounds []) [i := composition-openings])
  by (rule partial-composition-table-candidate-single-roundI
      [OF i-bound comp-len comp-agrees])
show ?thesis
  by (rule
      partial-query-openings-consistent-indices-fraction-bound-if-candidate-low-degree
      [OF i-bound trace-candidate comp-candidate trace-low comp-low
         not-all])
qed

```

lemma *table-agrees-with-authenticated-openings-values*:

```

  assumes agrees:
    table-agrees-with-authenticated-openings table len openings
  and indices: map opening-index openings = idxs
  shows map (!) table idxs = map opening-value openings
proof (rule nth-equalityI)
  show length (map (!) table) idxs = length (map opening-value openings)
    by (simp add: indices[symmetric])
next
  fix i
  assume i-bound: i < length (map (!) table) idxs
  have i-openings: i < length openings
    using i-bound by (simp add: indices[symmetric])
  have opn-in: openings ! i ∈ set openings
    by (rule nth-mem [OF i-openings])
  have table-value:
    table ! opening-index (openings ! i) = opening-value (openings ! i)
    using agrees opn-in
  unfolding table-agrees-with-authenticated-openings-def by auto
  have idxs ! i = opening-index (openings ! i)
    using i-bound by (simp add: indices[symmetric])
  then show map (!) table idxs ! i = map opening-value openings ! i
    using table-value i-bound i-openings by simp
qed

```

lemma *partial-authenticated-table-agrees-with-bound-table-if-clean*:

```

  assumes bind: merkle-root-binds-table rt table final-state
  and len-table: length table = scale * clength
  and partial:
    partial-authenticated-table rt (scale * clength) openings final-state
  and clean: ¬ hash-map-output-collision final-state
  shows
    table-agrees-with-authenticated-openings table (scale * clength)
    openings
proof –
  obtain n where len-pow: clength * scale = 2 ^ n
    using eval-domain-length-power by blast
  have table-pow: length table = 2 ^ n
    using len-table len-pow by (simp add: mult.commute)

```

```

have opening-agrees:
   $\forall \text{opn} \in \text{set openings.}$ 
    opening-index  $\text{opn} < \text{scale} * \text{clength} \wedge$ 
    table ! opening-index  $\text{opn} = \text{opening-value opn}$ 
proof
  fix opn
  assume opn-in:  $\text{opn} \in \text{set openings}$ 
  have root: opening-root  $\text{opn} = \text{rt}$ 
    using partial opn-in unfolding partial-authenticated-table-def
    by blast
  have len: opening-length  $\text{opn} = \text{length table}$ 
    using partial opn-in len-table
    unfolding partial-authenticated-table-def by simp
  have auth: authenticated-opening-in final-state  $\text{opn}$ 
    using partial opn-in unfolding partial-authenticated-table-def
    by blast
  have agreement:
    opening-index  $\text{opn} < \text{length table} \wedge$ 
    table ! opening-index  $\text{opn} = \text{opening-value opn}$ 
    by (rule merkle-bound-table-agrees-with-authenticated-opening-if-clean
      [OF bind table-pow auth root len clean])
  then show
    opening-index  $\text{opn} < \text{scale} * \text{clength} \wedge$ 
    table ! opening-index  $\text{opn} = \text{opening-value opn}$ 
    using len-table by simp
qed
show ?thesis
  unfolding table-agrees-with-authenticated-openings-def
  using len-table opening-agrees by simp
qed

lemma partial-query-openings-consistent-from-sampling-success:
assumes trace-bind:
  merkle-root-binds-table fr trace-table final-state
and comp-bind:
  merkle-root-binds-table composition-root composition-table
  final-state
and trace-len: length trace-table = scale * clength
and comp-len: length composition-table = scale * clength
and trace-auth:
  partial-authenticated-table fr (scale * clength) trace-openings
  final-state
and comp-auth:
  partial-authenticated-table composition-root (scale * clength)
  composition-openings final-state
and clean:  $\neg \text{hash-map-output-collision final-state}$ 
and trace-indices:
  map opening-index trace-openings = powers-scaled idx
and comp-indices:

```

```

    map opening-index composition-openings =
      [idx, fri-sibling-index (scale * clength) idx]
  and success:
    idx ∈ query-sampling-success-space trace-table composition-table as
  shows
    partial-query-openings-consistent trace-openings composition-openings
      as idx
  proof -
    have idx-sample: idx ∈ query-sample-space
      using success
      unfolding query-sampling-success-space-def query-agreement-indices-def
      by (auto split: if-splits)
    have consistent: query-consistent-at trace-table composition-table as idx
      using success
      unfolding query-sampling-success-space-def query-agreement-indices-def
      by (auto split: if-splits)
    have trace-agrees:
      table-agrees-with-authenticated-openings trace-table
        (scale * clength) trace-openings
      by (rule partial-authenticated-table-agrees-with-bound-table-if-clean
        [OF trace-bind trace-len trace-auth clean])
    have comp-agrees:
      table-agrees-with-authenticated-openings composition-table
        (scale * clength) composition-openings
      by (rule partial-authenticated-table-agrees-with-bound-table-if-clean
        [OF comp-bind comp-len comp-auth clean])
    have trace-values:
      map ((! trace-table) (powers-scaled idx)) =
        map opening-value trace-openings
      by (rule table-agrees-with-authenticated-openings-values
        [OF trace-agrees trace-indices])
    have comp-len-openings: length composition-openings = 2
      using arg-cong[OF comp-indices, of length] by simp
    have first-in: composition-openings ! 0 ∈ set composition-openings
      by (rule nth-mem) (use comp-len-openings in simp)
    have first-index: opening-index (composition-openings ! 0) = idx
  proof -
    have map opening-index composition-openings ! 0 = idx
      using comp-indices by simp
    then show ?thesis
      using comp-len-openings by simp
  qed
  have composition-value:
    composition-table ! idx = opening-value (composition-openings ! 0)
    using comp-agrees first-in first-index
    unfolding table-agrees-with-authenticated-openings-def by auto
  have opened-value:
    opening-value (composition-openings ! 0) =
      cp-eval as (map opening-value trace-openings) (h ^ idx * shift)

```

```

    using consistent trace-values composition-value
    unfolding query-consistent-at-def by simp
show ?thesis
by (rule partial-query-openings-consistentI
    [OF idx-sample trace-indices comp-indices opened-value])
qed

lemma partial-query-round-consistent-from-sampling-success:
  assumes trace-partial:
    accepted-with-partial-trace-openings s out fr query-idxs
    trace-openings
  and comp-partial:
    accepted-with-partial-composition-openings s out composition-root
    query-idxs composition-openings
  and trace-candidate:
    partial-trace-table-candidate trace-table trace-openings
  and comp-candidate:
    partial-composition-table-candidate composition-table
    composition-openings
  and i-bound:  $i < \text{rounds}$ 
  and success:
    query-idxs !  $i \in$ 
    query-sampling-success-space trace-table composition-table as
  shows
    partial-query-round-consistent trace-openings composition-openings as  $i$ 
    (query-idxs !  $i$ )
proof -
  let ?idx = query-idxs !  $i$ 
  have idx-sample: ?idx  $\in$  query-sample-space
  using success
  unfolding query-sampling-success-space-def query-agreement-indices-def
  by (auto split: if-splits)
  have consistent: query-consistent-at trace-table composition-table as ?idx
  using success
  unfolding query-sampling-success-space-def query-agreement-indices-def
  by (auto split: if-splits)
  have trace-len: length trace-openings = rounds
  by (rule accepted-with-partial-trace-openings-shapes(2)
      [OF trace-partial])
  have comp-len: length composition-openings = rounds
  by (rule accepted-with-partial-composition-openings-shapes(2)
      [OF comp-partial])
  have trace-indices:
    map opening-index (trace-openings !  $i$ ) = powers-scaled ?idx
  by (rule accepted-with-partial-trace-openings-shapes(3)
      [OF trace-partial i-bound])
  have comp-indices:
    map opening-index (composition-openings !  $i$ ) =
    [?idx, fri-sibling-index (scale * clength) ?idx]

```

```

by (rule accepted-with-partial-composition-openings-shapes(3)
  [OF comp-partial i-bound])
have trace-values:
  map (! trace-table) (powers-scaled ?idx) =
    map opening-value (trace-openings ! i)
by (rule partial-trace-table-candidate-query-values
  [OF trace-candidate i-bound trace-indices])
have composition-value:
  composition-table ! ?idx =
    opening-value ((composition-openings ! i) ! 0)
by (rule partial-composition-table-candidate-query-value
  [OF comp-candidate i-bound comp-indices])
have opened-value:
  opening-value ((composition-openings ! i) ! 0) =
    cp-eval as (map opening-value (trace-openings ! i))
    (h ^ ?idx * shift)
using consistent trace-values composition-value
unfolding query-consistent-at-def by simp
show ?thesis
by (rule partial-query-round-consistentI
  [OF i-bound idx-sample trace-len comp-len trace-indices comp-indices
    opened-value])

```

qed

definition *query-header-supported-partial-opening-witnesses*

```

:: (f, 'a) protocol-channel-scheme ⇒
  f ⇒ 'f list ⇒ 'f ⇒ 'f list ⇒ 'f ⇒
  'f list ⇒ 'f ⇒
  ('f authenticated-opening list ×
   'f authenticated-opening list) set

```

where

```

query-header-supported-partial-opening-witnesses s fr f-fri-roots
  f-final as dg composition-fri-roots final =
  {(trace-openings, composition-openings).
   composition-fri-roots ≠ [] ∧
   (∃ result final-state rest.
    Some (result, final-state) ∈ set-dist (execute verify-monad s) ∧
    partial-authenticated-table fr (scale * clength) trace-openings
      final-state ∧
    partial-authenticated-table (hd composition-fri-roots)
      (scale * clength) composition-openings final-state ∧
    verifier-header-transcript s fr f-fri-roots f-final as dg
      composition-fri-roots final rest)}}

```

lemma *query-header-supported-partial-opening-witnessesE*:

assumes

```

(trace-openings, composition-openings) ∈
  query-header-supported-partial-opening-witnesses s fr f-fri-roots
  f-final as dg composition-fri-roots final

```


obtains *result final-state rest* **where**
composition-fri-roots $\neq []$
Some (*result*, *final-state*) $\in$ *set-dist* (*execute verify-monad s*)
partial-authenticated-table *fr* (*scale * clength*) *trace-openings*
final-state
partial-authenticated-table (*hd composition-fri-roots*)
(*scale * clength*) *composition-openings final-state*
verifier-header-transcript s fr f-fri-roots f-final as dg
composition-fri-roots final rest
using *assms*
unfolding *query-header-supported-partial-opening-witnesses-def*
by *blast*

definition *query-header-supported-partial-opening-success-indices-at*

$:: (f, 'a) \text{ protocol-channel-scheme} \Rightarrow$
 $'f \Rightarrow 'f \text{ list} \Rightarrow 'f \Rightarrow 'f \text{ list} \Rightarrow 'f \Rightarrow$
 $'f \text{ list} \Rightarrow 'f \Rightarrow \text{nat} \Rightarrow \text{nat set}$

where

query-header-supported-partial-opening-success-indices-at s fr
f-fri-roots f-final as dg composition-fri-roots final i =
 $\{idx \in \text{query-sample-space}.$
 $\exists \text{ trace-openings composition-openings}.$
 $(\text{trace-openings}, \text{composition-openings}) \in$
 $\text{query-header-supported-partial-opening-witnesses } s \text{ fr}$
 $f\text{-fri-roots } f\text{-final as dg composition-fri-roots final} \wedge$
 $\text{partial-query-openings-consistent trace-openings composition-openings}$
 $\text{as } idx\}$

lemma *query-header-supported-partial-opening-success-indices-at-subset:*

query-header-supported-partial-opening-success-indices-at s fr
f-fri-roots f-final as dg composition-fri-roots final i
 $\subseteq \text{query-sample-space}$

unfolding *query-header-supported-partial-opening-success-indices-at-def*
by *auto*

lemma *query-header-supported-partial-opening-success-indices-atE:*

assumes

idx $\in$ *query-header-supported-partial-opening-success-indices-at s fr*
f-fri-roots f-final as dg composition-fri-roots final i

obtains *trace-openings composition-openings* **where**

idx $\in$ *query-sample-space*
 $(\text{trace-openings}, \text{composition-openings}) \in$
 $\text{query-header-supported-partial-opening-witnesses } s \text{ fr}$
 $f\text{-fri-roots } f\text{-final as dg composition-fri-roots final}$
 $\text{partial-query-openings-consistent trace-openings composition-openings}$
 $\text{as } idx$

using *assms*

unfolding *query-header-supported-partial-opening-success-indices-at-def*
by *blast*

lemma *query-header-supported-partial-opening-success-indices-at-UN:*

query-header-supported-partial-opening-success-indices-at s fr
f-fri-roots f-final as dg composition-fri-roots final i =
 $(\bigcup (\text{trace-openings}, \text{composition-openings}) \in$
query-header-supported-partial-opening-witnesses s fr
f-fri-roots f-final as dg composition-fri-roots final.
 $\{\text{idx} \in \text{query-sample-space}.$
partial-query-openings-consistent trace-openings
composition-openings as idx}\})

unfolding *query-header-supported-partial-opening-success-indices-at-def*
by *auto*

lemma *query-header-supported-partial-opening-success-indices-at-subset-if-witnesses:*

assumes *cover:*

$\bigwedge \text{trace-openings composition-openings}.$
 $(\text{trace-openings}, \text{composition-openings}) \in$
query-header-supported-partial-opening-witnesses s fr
f-fri-roots f-final as dg composition-fri-roots final $\implies$
 $\{\text{idx} \in \text{query-sample-space}.$
partial-query-openings-consistent trace-openings
composition-openings as idx\} \subseteq B

shows

query-header-supported-partial-opening-success-indices-at s fr
f-fri-roots f-final as dg composition-fri-roots final i $\subseteq B$

unfolding *query-header-supported-partial-opening-success-indices-at-UN*
using *cover* **by** *auto*

lemma *query-header-supported-partial-opening-success-indices-atI-from-bound-tables:*

assumes *outcome:*

Some (result, final-state) $\in$ set-dist (execute verify-monad s)

and *header:*

verifier-header-transcript s fr f-fri-roots f-final as dg
composition-fri-roots final rest

and *comp-nonempty:* *composition-fri-roots $\neq []$*

and *trace-bind:*

merkle-root-binds-table fr trace-table final-state

and *comp-bind:*

merkle-root-binds-table (hd composition-fri-roots) composition-table
final-state

and *trace-len:* *length trace-table = scale * clength*

and *comp-len:* *length composition-table = scale * clength*

and *trace-auth:*

*partial-authenticated-table fr (scale * clength) trace-openings*
final-state

and *comp-auth:*

partial-authenticated-table (hd composition-fri-roots)
*(scale * clength) composition-openings final-state*

and *clean:* $\neg \text{hash-map-output-collision final-state}$

```

and trace-indices:
  map opening-index trace-openings = powers-scaled idx
and comp-indices:
  map opening-index composition-openings =
    [idx, fri-sibling-index (scale * clength) idx]
and success:
  idx ∈ query-sampling-success-space trace-table composition-table as
shows
  idx ∈
    query-header-supported-partial-opening-success-indices-at s fr
    f-fri-roots f-final as dg composition-fri-roots final i
proof –
  have consistent:
    partial-query-openings-consistent trace-openings composition-openings
    as idx
  by (rule partial-query-openings-consistent-from-sampling-success
    [OF trace-bind comp-bind trace-len comp-len trace-auth comp-auth
    clean trace-indices comp-indices success])
  have witness:
    (trace-openings, composition-openings) ∈
      query-header-supported-partial-opening-witnesses s fr f-fri-roots
      f-final as dg composition-fri-roots final
  unfolding query-header-supported-partial-opening-witnesses-def
  using comp-nonempty outcome trace-auth comp-auth header
  by (auto split: prod.splits)
  have idx-sample: idx ∈ query-sample-space
  using consistent
  unfolding partial-query-openings-consistent-def by simp
  show ?thesis
  unfolding query-header-supported-partial-opening-success-indices-at-def
  by (intro CollectI conjI exI[of - trace-openings]
    exI[of - composition-openings])
    (use idx-sample witness consistent in simp-all)
qed

lemma query-header-supported-partial-opening-success-indices-atI-from-partials:
assumes outcome:
  Some (result, final-state) ∈ set-dist (execute verify-monad s)
and trace-partial:
  accepted-with-partial-trace-openings s
  (Some (result, final-state)) fr query-idxs trace-openings
and comp-partial:
  accepted-with-partial-composition-openings s
  (Some (result, final-state)) composition-root query-idxs
  composition-openings
and trace-candidate:
  partial-trace-table-candidate trace-table trace-openings
and comp-candidate:
  partial-composition-table-candidate composition-table

```

```

    composition-openings
and header:
    verifier-header-transcript s fr f-fri-roots f-final as dg
    composition-fri-roots final rest
and comp-nonempty: composition-fri-roots  $\neq$  []
and root-eq: composition-root = hd composition-fri-roots
and i-bound: i < rounds
and success:
    query-idxs ! i  $\in$ 
    query-sampling-success-space trace-table composition-table as
shows
    query-idxs ! i  $\in$ 
    query-header-supported-partial-opening-success-indices-at s fr
    f-fri-roots f-final as dg composition-fri-roots final i
proof –
    let ?trace-openings = trace-openings ! i
    let ?composition-openings = composition-openings ! i
    have consistent-round:
        partial-query-round-consistent trace-openings composition-openings as
        i (query-idxs ! i)
    by (rule partial-query-round-consistent-from-sampling-success
        [OF trace-partial comp-partial trace-candidate comp-candidate
        i-bound success])
    have consistent:
        partial-query-openings-consistent ?trace-openings
        ?composition-openings as (query-idxs ! i)
    unfolding partial-query-openings-consistent-def
    using consistent-round
    unfolding partial-query-round-consistent-def
    by simp
    have trace-table:
        partial-authenticated-table fr (scale * clength) ?trace-openings
        final-state
    using trace-partial i-bound
    unfolding accepted-with-partial-trace-openings-def by blast
    have comp-table:
        partial-authenticated-table (hd composition-fri-roots)
        (scale * clength) ?composition-openings final-state
    using comp-partial i-bound root-eq
    unfolding accepted-with-partial-composition-openings-def by blast
    have witness:
        (?trace-openings, ?composition-openings)  $\in$ 
        query-header-supported-partial-opening-witnesses s fr f-fri-roots
        f-final as dg composition-fri-roots final
    unfolding query-header-supported-partial-opening-witnesses-def
    using comp-nonempty outcome trace-table comp-table header
    by auto
    have idx-sample:
        query-idxs ! i  $\in$  query-sample-space

```

```

    using consistent unfolding partial-query-openings-consistent-def by simp
show ?thesis
    unfolding query-header-supported-partial-opening-success-indices-at-def
    by (intro CollectI conjI exI[of - ?trace-openings]
        exI[of - ?composition-openings])
    (use idx-sample witness consistent in simp-all)
qed

lemma partial-query-round-consistent-from-all-queries-consistent:
  assumes trace-partial:
    accepted-with-partial-trace-openings s
    (Some (result, final-state)) fr query-idxs trace-openings
  and comp-partial:
    accepted-with-partial-composition-openings s
    (Some (result, final-state)) composition-root query-idxs
    composition-openings
  and trace-candidate:
    partial-trace-table-candidate trace-table trace-openings
  and comp-candidate:
    partial-composition-table-candidate composition-table
    composition-openings
  and i-bound:  $i < \text{rounds}$ 
  and idx-sample:  $\text{query-idxs} \vdash i \in \text{query-sample-space}$ 
  and all-queries:
    all-queries-consistent trace-table composition-table as
  shows
    partial-query-round-consistent trace-openings composition-openings as
    i (query-idxs  $\vdash$  i)
proof -
  let ?idx = query-idxs  $\vdash$  i
  have trace-len:  $\text{length trace-openings} = \text{rounds}$ 
  using trace-partial unfolding accepted-with-partial-trace-openings-def
  by blast
  have comp-len:  $\text{length composition-openings} = \text{rounds}$ 
  using comp-partial
  unfolding accepted-with-partial-composition-openings-def by blast
  have trace-indices:
    map opening-index (trace-openings  $\vdash$  i) = powers-scaled ?idx
  using trace-partial i-bound
  unfolding accepted-with-partial-trace-openings-def by blast
  have comp-indices:
    map opening-index (composition-openings  $\vdash$  i) =
    [?idx, fri-sibling-index (scale * clength) ?idx]
  using comp-partial i-bound
  unfolding accepted-with-partial-composition-openings-def by blast
  have qc:
    query-consistent-at trace-table composition-table as ?idx
  using all-queries idx-sample unfolding all-queries-consistent-def
  by simp

```

```

have trace-values:
  map (! trace-table) (powers-scaled ?idx) =
    map opening-value (trace-openings ! i)
by (rule partial-trace-table-candidate-query-values
      [OF trace-candidate i-bound trace-indices])
have comp-value:
  composition-table ! ?idx =
    opening-value ((composition-openings ! i) ! 0)
by (rule partial-composition-table-candidate-query-value
      [OF comp-candidate i-bound comp-indices])
have opened-value:
  opening-value ((composition-openings ! i) ! 0) =
    cp-eval as (map opening-value (trace-openings ! i))
      (h ^ ?idx * shift)
using qc trace-values comp-value unfolding query-consistent-at-def
by simp
show ?thesis
by (rule partial-query-round-consistentI
      [OF i-bound idx-sample trace-len comp-len trace-indices
        comp-indices opened-value])
qed

```

lemma query-header-supported-partial-opening-success-indices-atI-from-partials-consistent:

```

assumes outcome:
  Some (result, final-state) ∈ set-dist (execute verify-monad s)
and trace-partial:
  accepted-with-partial-trace-openings s
  (Some (result, final-state)) fr query-idxs trace-openings
and comp-partial:
  accepted-with-partial-composition-openings s
  (Some (result, final-state)) composition-root query-idxs
  composition-openings
and trace-candidate:
  partial-trace-table-candidate trace-table trace-openings
and comp-candidate:
  partial-composition-table-candidate composition-table
  composition-openings
and header:
  verifier-header-transcript s fr f-fri-roots f-final as dg
  composition-fri-roots final rest
and comp-nonempty: composition-fri-roots ≠ []
and root-eq: composition-root = hd composition-fri-roots
and i-bound: i < rounds
and idx-sample: query-idxs ! i ∈ query-sample-space
and all-queries:
  all-queries-consistent trace-table composition-table as
shows
  query-idxs ! i ∈
  query-header-supported-partial-opening-success-indices-at s fr

```

f -fri-roots f -final as dg composition-fri-roots final i
proof –
 let $?trace-openings = trace-openings ! i$
 let $?composition-openings = composition-openings ! i$
 have consistent-round:
 partial-query-round-consistent trace-openings composition-openings as
 $i (query-idxs ! i)$
 by (rule partial-query-round-consistent-from-all-queries-consistent
 [OF trace-partial comp-partial trace-candidate comp-candidate
 i -bound idx -sample all-queries])
 have consistent:
 partial-query-openings-consistent $?trace-openings$
 $?composition-openings$ as $(query-idxs ! i)$
 unfolding partial-query-openings-consistent-def
 using consistent-round
 unfolding partial-query-round-consistent-def
 by simp
 have trace-table:
 partial-authenticated-table fr $(scale * clength) ?trace-openings$
 final-state
 using trace-partial i -bound
 unfolding accepted-with-partial-trace-openings-def by blast
 have comp-table:
 partial-authenticated-table $(hd\ composition-fri-roots)$
 $(scale * clength) ?composition-openings$ final-state
 using comp-partial i -bound root-eq
 unfolding accepted-with-partial-composition-openings-def by blast
 have witness:
 $(?trace-openings, ?composition-openings) \in$
 query-header-supported-partial-opening-witnesses s fr f -fri-roots
 f -final as dg composition-fri-roots final
 unfolding query-header-supported-partial-opening-witnesses-def
 using comp-nonempty outcome trace-table comp-table header
 by auto
 show $?thesis$
 unfolding query-header-supported-partial-opening-success-indices-at-def
 by (intro CollectI conjI exI[of - $?trace-openings$]
 exI[of - $?composition-openings$])
 (use idx -sample witness consistent in simp-all)
qed

definition staged-security-with-data-state-query-partial-opening-hit-at
 $:: nat \Rightarrow$
 $((('f\ staged-proof-data \times 'f\ protocol-channel) \times unit\ list) \times$
 $'f\ protocol-channel)\ option \Rightarrow bool$
where
 staged-security-with-data-state-query-partial-opening-hit-at i out
 $\longleftrightarrow$
 (case out of

```

None ⇒ False
| Some (((data, attacker-state), -), final-state) ⇒
  (let s =
    verifier-state-from-adversary attacker-state
      (staged-proof-transcript data);
   B =
    query-header-supported-partial-opening-success-indices-at s
      (staged-trace-root data)
      (staged-trace-fri-roots data)
      (staged-trace-final data)
      (staged-alphas data)
      (staged-degree data)
      (staged-composition-fri-roots data)
      (staged-composition-final data)
   i
  in
  (∃ raw.
    i < rounds ∧
    fmllookup (HashMap final-state)
      (QueryIndexChallenge i
        (state-after-query-chunks
          (staged-query-start-hash data)
          (staged-query-chunks data) i)) =
      Some raw ∧
    index (to-nat raw) ∈ B)))

```

definition *staged-security-with-data-state-query-partial-opening-hit*
 $:: (((f \text{ staged-proof-data} \times f \text{ protocol-channel}) \times \text{unit list}) \times$
 $f \text{ protocol-channel}) \text{ option} \Rightarrow \text{bool}$

where

```

staged-security-with-data-state-query-partial-opening-hit out  $\longleftrightarrow$ 
(∃ i < rounds.
  staged-security-with-data-state-query-partial-opening-hit-at i out)

```

lemma *staged-security-with-data-state-query-partial-opening-hit-atD:*
assumes

staged-security-with-data-state-query-partial-opening-hit-at i out

shows $i < \text{rounds}$

using *assms*

unfolding *staged-security-with-data-state-query-partial-opening-hit-at-def*
by (*cases out; auto split: prod.splits*)

lemma *staged-security-with-data-state-query-partial-opening-hit-atE:*
assumes

staged-security-with-data-state-query-partial-opening-hit-at i
(Some (((data, attacker-state), result), final-state))

obtains *raw* **where**

$i < \text{rounds}$

fmllookup (HashMap final-state)


```

    (QueryIndexChallenge i
      (state-after-query-chunks
        (staged-query-start-hash data)
        (staged-query-chunks data) i)) =
    Some raw
  index (to-nat raw) ∈
    query-header-supported-partial-opening-success-indices-at
      (verifier-state-from-adversary attacker-state
        (staged-proof-transcript data))
      (staged-trace-root data)
      (staged-trace-fri-roots data)
      (staged-trace-final data)
      (staged-alphas data)
      (staged-degree data)
      (staged-composition-fri-roots data)
      (staged-composition-final data)
      i
  using assms
  unfolding staged-security-with-data-state-query-partial-opening-hit-at-def
    Let-def
  by auto

lemma staged-security-with-data-state-query-partial-opening-hitI:
  assumes i < rounds
    and staged-security-with-data-state-query-partial-opening-hit-at i out
  shows staged-security-with-data-state-query-partial-opening-hit out
  using assms
  unfolding staged-security-with-data-state-query-partial-opening-hit-def
  by blast

lemma staged-security-with-data-state-query-partial-opening-hit-imp-round-hit:
  assumes hit: staged-security-with-data-state-query-partial-opening-hit out
  shows  $\exists i \in \{..<\text{rounds}\}.$ 
    staged-security-with-data-state-query-partial-opening-hit-at i out
  using assms
  unfolding staged-security-with-data-state-query-partial-opening-hit-def
  by blast

lemma checked-staged-security-with-data-state-query-partial-opening-hit-bound-from-rounds:
  assumes round-bound:
     $\bigwedge i. i < \text{rounds} \implies$ 
      wp-event (checked-staged-security-experiment-with-data-state A)
        (staged-security-with-data-state-query-partial-opening-hit-at i)
        adversary-initial-state  $\leq C i$ 
  shows
    wp-event (checked-staged-security-experiment-with-data-state A)
      staged-security-with-data-state-query-partial-opening-hit
      adversary-initial-state  $\leq (\sum i < \text{rounds}. C i)$ 
proof –

```

```

have wp-event (checked-staged-security-experiment-with-data-state A)
  staged-security-with-data-state-query-partial-opening-hit
  adversary-initial-state ≤
wp-event (checked-staged-security-experiment-with-data-state A)
  (λout. ∃ i ∈ {.. $\text{rounds}$ }.
    staged-security-with-data-state-query-partial-opening-hit-at i out)
  adversary-initial-state
by (rule wp-event-mono)
  (rule staged-security-with-data-state-query-partial-opening-hit-imp-round-hit)
also have ... ≤ ( $\sum$  i<rounds. C i)
by (rule wp-event-finite-UN-bound)
  (simp-all add: round-bound)
finally show ?thesis .
qed

lemma checked-staged-security-with-data-state-partial-opening-hit-bound-from-index-set:
assumes cover:
   $\bigwedge$ data attacker-state i.
    i < rounds  $\implies$ 
    query-header-supported-partial-opening-success-indices-at
      (verifier-state-from-adversary attacker-state
        (staged-proof-transcript data))
      (staged-trace-root data)
      (staged-trace-fri-roots data)
      (staged-trace-final data)
      (staged-alphas data)
      (staged-degree data)
      (staged-composition-fri-roots data)
      (staged-composition-final data)
      i  $\subseteq$  B
and index-bound:
  wp-event (checked-staged-security-experiment-with-data A)
    (staged-security-with-data-query-index-set-hit B)
    adversary-initial-state ≤ C
shows
  wp-event (checked-staged-security-experiment-with-data-state A)
    staged-security-with-data-state-query-partial-opening-hit
    adversary-initial-state ≤ C
proof –
let ?projected =
  λout. case out of None  $\implies$ 
    staged-security-with-data-query-index-set-hit B None
  | Some (((data, -), result), t)  $\implies$ 
    staged-security-with-data-query-index-set-hit B
      (Some ((data, result), t))
have event-le:
  wp-event (checked-staged-security-experiment-with-data-state A)
    staged-security-with-data-state-query-partial-opening-hit
    adversary-initial-state ≤

```

```

wp-event (checked-staged-security-experiment-with-data-state A)
  ?projected adversary-initial-state
proof (rule wp-event-mono)
  fix out
  assume hit:
    staged-security-with-data-state-query-partial-opening-hit out
  show ?projected out
  proof (cases out)
  case None
  then show ?thesis
    using hit
    unfolding staged-security-with-data-state-query-partial-opening-hit-def
      staged-security-with-data-state-query-partial-opening-hit-at-def
    by simp
  next
  case (Some packed)
  then obtain data attacker-state result final-state where out-eq:
    out = Some (((data, attacker-state), result), final-state)
    by (cases packed, auto split: prod.splits)
  let ?s =
    verifier-state-from-adversary attacker-state
    (staged-proof-transcript data)
  let ?B =
     $\lambda i.$  query-header-supported-partial-opening-success-indices-at ?s
      (staged-trace-root data)
      (staged-trace-fri-roots data)
      (staged-trace-final data)
      (staged-alphas data)
      (staged-degree data)
      (staged-composition-fri-roots data)
      (staged-composition-final data)
      i
  from hit[unfolded out-eq
    staged-security-with-data-state-query-partial-opening-hit-def
    staged-security-with-data-state-query-partial-opening-hit-at-def
    Let-def]
  obtain i raw where i-bound:  $i < \text{rounds}$ 
  and lookup:
    fmlookup (HashMap final-state)
      (QueryIndexChallenge i
        (state-after-query-chunks
          (staged-query-start-hash data)
          (staged-query-chunks data) i)) =
      Some raw
  and raw-hit:  $\text{index } (\text{to-nat raw}) \in ?B \ i$ 
  by auto
  have raw-in-B:  $\text{index } (\text{to-nat raw}) \in B$ 
  proof -
    have ?B  $i \subseteq B$ 

```

```

      by (rule cover[OF i-bound])
    then show ?thesis
      using raw-hit by blast
  qed
show ?thesis
  unfolding out-eq staged-security-with-data-query-index-set-hit-def
  using i-bound lookup raw-in-B by auto
qed
qed
also have
  wp-event (checked-staged-security-experiment-with-data-state A)
    ?projected adversary-initial-state =
  wp-event (checked-staged-security-experiment-with-data A)
    (staged-security-with-data-query-index-set-hit B)
    adversary-initial-state
  using checked-staged-security-experiment-with-data-event-from-data-state
    [of A staged-security-with-data-query-index-set-hit B]
  by simp
also have ...  $\leq$  C
  by (rule index-bound)
finally show ?thesis .
qed

```

lemma *checked-staged-security-with-data-state-partial-opening-hit-bound-from-index-set-controlled:*

assumes wf: staged-budget-wellformed budgets

and controlled: staged-adversary-controlled budgets A

and cover:

$\bigwedge$ data attacker-state i.

$i < \text{rounds} \implies$

query-header-supported-partial-opening-success-indices-at

(verifier-state-from-adversary attacker-state

(staged-proof-transcript data))

(staged-trace-root data)

(staged-trace-fri-roots data)

(staged-trace-final data)

(staged-alphas data)

(staged-degree data)

(staged-composition-fri-roots data)

(staged-composition-final data)

$i \subseteq B$

shows

wp-event (checked-staged-security-experiment-with-data-state A)

staged-security-with-data-state-query-partial-opening-hit

adversary-initial-state $\leq$

staged-phase-target-error (query-index-raw-preimage B)

(staged-attacker-query-budget budgets +

staged-challenge-query-budget)

proof (rule

checked-staged-security-with-data-state-partial-opening-hit-bound-from-index-set)

```

fix data attacker-state  $i$ 
assume  $i$ -bound:  $i < \text{rounds}$ 
show
  query-header-supported-partial-opening-success-indices-at
    (verifier-state-from-adversary attacker-state
      (staged-proof-transcript data))
    (staged-trace-root data)
    (staged-trace-fri-roots data)
    (staged-trace-final data)
    (staged-alphas data)
    (staged-degree data)
    (staged-composition-fri-roots data)
    (staged-composition-final data)
     $i \subseteq B$ 
  by (rule cover[OF  $i$ -bound])
next
show
  wp-event (checked-staged-security-experiment-with-data  $A$ )
    (staged-security-with-data-query-index-set-hit  $B$ )
  adversary-initial-state  $\leq$ 
  staged-phase-target-error (query-index-raw-preimage  $B$ )
    (staged-attacker-query-budget budgets +
      staged-challenge-query-budget)
  by (rule checked-staged-security-with-data-query-index-set-hit-bound
    [OF wf controlled])
qed

lemma checked-staged-security-with-data-state-partial-opening-hit-bound-from-fixed-query-error-cover:
assumes wf: staged-budget-wellformed budgets
and controlled: staged-adversary-controlled budgets  $A$ 
and cover:
 $\bigwedge$  data attacker-state  $i$ .
   $i < \text{rounds} \implies$ 
  query-header-supported-partial-opening-success-indices-at
    (verifier-state-from-adversary attacker-state
      (staged-proof-transcript data))
    (staged-trace-root data)
    (staged-trace-fri-roots data)
    (staged-trace-final data)
    (staged-alphas data)
    (staged-degree data)
    (staged-composition-fri-roots data)
    (staged-composition-final data)
     $i \subseteq B$ 
and raw-bound: query-index-raw-preimage-bound
and subset:  $B \subseteq \text{query-sample-space}$ 
and frac:
   $\text{nnreal}(\text{card } B) / \text{nnreal}(\text{card } \text{query-sample-space}) \leq$ 
  query-error-bound

```

shows

$$\begin{aligned} & \text{wp-event } (\text{checked-staged-security-experiment-with-data-state } A) \\ & \text{staged-security-with-data-state-query-partial-opening-hit} \\ & \text{adversary-initial-state} \leq \\ & \text{nnreal} \\ & (\text{staged-attacker-query-budget budgets} + \\ & \text{staged-challenge-query-budget}) * \text{query-error-bound} \end{aligned}$$
proof –
have *target*:

$$\begin{aligned} & \text{wp-event } (\text{checked-staged-security-experiment-with-data-state } A) \\ & \text{staged-security-with-data-state-query-partial-opening-hit} \\ & \text{adversary-initial-state} \leq \\ & \text{staged-phase-target-error } (\text{query-index-raw-preimage } B) \\ & (\text{staged-attacker-query-budget budgets} + \\ & \text{staged-challenge-query-budget}) \end{aligned}$$
by (rule

$$\text{checked-staged-security-with-data-state-partial-opening-hit-bound-from-index-set-controlled}$$

$$[\text{OF wf controlled cover}])$$
also have ... $\leq$

$$\begin{aligned} & \text{nnreal} \\ & (\text{staged-attacker-query-budget budgets} + \\ & \text{staged-challenge-query-budget}) * \text{query-error-bound} \end{aligned}$$
by (rule *staged-phase-query-index-target-error-query-bound*

$$[\text{OF raw-bound subset frac}])$$
finally show *?thesis* .
qed

lemma *checked-staged-security-with-data-state-query-bad-imp-partial-opening-hit-on-support*:
assumes *wf*: *staged-budget-wellformed budgets*
and *controlled*: *staged-adversary-controlled budgets A*
and *support*:

$$\begin{aligned} & \text{Some } (((\text{data}, \text{attacker-state}), \text{result}), \text{final-state}) \in \\ & \text{set-dist} \\ & (\text{execute} \\ & (\text{checked-staged-security-experiment-with-data-state } A) \\ & \text{adversary-initial-state}) \end{aligned}$$
and *bad*:

$$\begin{aligned} & \text{staged-security-with-data-state-query-bad-hit} \\ & (\text{Some } (((\text{data}, \text{attacker-state}), \text{result}), \text{final-state})) \end{aligned}$$
shows

$$\begin{aligned} & \text{staged-security-with-data-state-query-partial-opening-hit} \\ & (\text{Some } (((\text{data}, \text{attacker-state}), \text{result}), \text{final-state})) \end{aligned}$$
proof –
let *?s* =

$$\begin{aligned} & \text{verifier-state-from-adversary attacker-state} \\ & (\text{staged-proof-transcript data}) \end{aligned}$$
from *checked-staged-security-experiment-with-data-state-outcomeE*

$$[\text{OF support}]$$
obtain *builder* **where** *builder*:

```

    Some (data, attacker-state) ∈
      set-dist
        (execute (checked-staged-transcript-program A)
          adversary-initial-state)
  and verifier:
    Some (result, final-state) ∈ set-dist (execute verify-monad ?s)
  by blast
have staged-header:
  verifier-header-transcript ?s
    (staged-trace-root data)
    (staged-trace-fri-roots data)
    (staged-trace-final data)
    (staged-alphas data)
    (staged-degree data)
    (staged-composition-fri-roots data)
    (staged-composition-final data)
    (List.concat (staged-query-chunks data))
  by (rule checked-staged-transcript-program-outcome-header-transcript
    [OF wf controlled builder])
show ?thesis
proof (rule checked-staged-security-with-data-state-query-bad-round-chunkE
  [OF wf controlled support bad])
  fix i trace-table composition-table query-idxs raw-idxs
  assume i-bound: i < rounds
  and bound:
    accepted-with-bound-tables ?s (Some (result, final-state))
    trace-table composition-table (staged-alphas data) query-idxs
  and success:
    query-idxs ! i ∈
      query-sampling-success-space trace-table composition-table
        (staged-alphas data)
  and len-raw: length raw-idxs = rounds
  and query-idxs-eq:
    query-idxs = map (λraw. index (to-nat raw)) raw-idxs
  and query-chunk:
    verifier-query-round-chunk (query-idxs ! i)
      (staged-trace-fri-roots data)
      (staged-composition-fri-roots data)
      (staged-query-chunks data ! i)
  and lookup:
    fmllookup (HashMap final-state)
      (QueryIndexChallenge i
        (state-after-query-chunks
          (staged-query-start-hash data) (staged-query-chunks data) i)) =
      Some (raw-idxs ! i)
  show ?thesis
proof (rule accepted-with-bound-tables-partial-opening-witnesses-Some
  [OF bound])
  fix fr f-fri-roots f-final dg composition-fri-roots final rest

```

```

    trace-openings composition-openings
assume header:
    verifier-header-transcript ?s fr f-fri-roots f-final
    (staged-alphas data) dg composition-fri-roots final rest
and comp-nonempty: composition-fri-roots  $\neq$  []
and trace-partial:
    accepted-with-partial-trace-openings ?s
    (Some (result, final-state)) fr query-idxs trace-openings
and comp-partial:
    accepted-with-partial-composition-openings ?s
    (Some (result, final-state)) (hd composition-fri-roots)
    query-idxs composition-openings
and trace-candidate:
    partial-trace-table-candidate trace-table trace-openings
and comp-candidate:
    partial-composition-table-candidate composition-table
    composition-openings
and trace-bind:
    merkle-root-binds-table fr trace-table final-state
and comp-bind:
    merkle-root-binds-table (hd composition-fri-roots)
    composition-table final-state
have header-eq:
    fr = staged-trace-root data  $\wedge$ 
    f-fri-roots = staged-trace-fri-roots data  $\wedge$ 
    f-final = staged-trace-final data  $\wedge$ 
    dg = staged-degree data  $\wedge$ 
    composition-fri-roots = staged-composition-fri-roots data  $\wedge$ 
    final = staged-composition-final data  $\wedge$ 
    rest = List.concat (staged-query-chunks data)
using verifier-header-transcript-unique[OF staged-header header]
by simp
have idx-in-header-set:
    query-idxs ! i  $\in$ 
    query-header-supported-partial-opening-success-indices-at ?s fr
    f-fri-roots f-final (staged-alphas data) dg
    composition-fri-roots final i
by (rule
    query-header-supported-partial-opening-success-indices-atI-from-partials
    [OF verifier trace-partial comp-partial trace-candidate
    comp-candidate header comp-nonempty refl i-bound success])
have idx-in-staged-set:
    query-idxs ! i  $\in$ 
    query-header-supported-partial-opening-success-indices-at ?s
    (staged-trace-root data)
    (staged-trace-fri-roots data)
    (staged-trace-final data)
    (staged-alphas data)
    (staged-degree data)

```



```

      (staged-composition-fri-roots data)
      (staged-composition-final data)
    i
  using idx-in-header-set header-eq by simp
have raw-idx:
  index (to-nat (raw-idxs ! i)) = query-idxs ! i
  using query-idxs-eq len-raw i-bound by simp
have hit-at:
  staged-security-with-data-state-query-partial-opening-hit-at i
    (Some (((data, attacker-state), result), final-state))
  unfolding
    staged-security-with-data-state-query-partial-opening-hit-at-def
  using i-bound lookup idx-in-staged-set raw-idx
  by (simp add: Let-def)
show ?thesis
  by (rule staged-security-with-data-state-query-partial-opening-hitI
    [OF i-bound hit-at])
qed
qed
qed

lemma checked-staged-security-with-data-state-query-bad-bound-from-partial-opening-hit:
  assumes wf: staged-budget-wellformed budgets
  and controlled: staged-adversary-controlled budgets A
  shows
    wp-event (checked-staged-security-experiment-with-data-state A)
      staged-security-with-data-state-query-bad-hit
      adversary-initial-state ≤
    wp-event (checked-staged-security-experiment-with-data-state A)
      staged-security-with-data-state-query-partial-opening-hit
      adversary-initial-state
proof (rule wp-event-mono-on-support)
  fix out
  assume support:
    out ∈
    set-dist
    (execute
      (checked-staged-security-experiment-with-data-state A)
      adversary-initial-state)
  and bad: staged-security-with-data-state-query-bad-hit out
  show staged-security-with-data-state-query-partial-opening-hit out
proof (cases out)
  case None
  then show ?thesis
    using bad unfolding staged-security-with-data-state-query-bad-hit-def
    by simp
next
  case (Some packed)
  then obtain data attacker-state result final-state where out-eq:

```

```

    out = Some (((data, attacker-state), result), final-state)
  by (cases packed, auto split: prod.splits)
show ?thesis
  unfolding out-eq
  by (rule
    checked-staged-security-with-data-state-query-bad-imp-partial-opening-hit-on-support
    [OF wf controlled])
    (use support bad out-eq in simp-all)
qed
qed

lemma checked-staged-security-with-data-state-query-bad-verifier-event-bound-from-partial-opening-hit:
assumes wf: staged-budget-wellformed budgets
and controlled: staged-adversary-controlled budgets A
and partial-bound:
  wp-event (checked-staged-security-experiment-with-data-state A)
  staged-security-with-data-state-query-partial-opening-hit
  adversary-initial-state  $\leq Q$ 
shows
  wp-event (checked-staged-security-experiment-with-data-state A)
  (staged-security-with-data-state-verifier-event query-bad)
  adversary-initial-state  $\leq Q$ 
proof -
have query-hit-bound:
  wp-event (checked-staged-security-experiment-with-data-state A)
  staged-security-with-data-state-query-bad-hit
  adversary-initial-state  $\leq Q$ 
by (rule order.trans
  [OF checked-staged-security-with-data-state-query-bad-bound-from-partial-opening-hit
  [OF wf controlled] partial-bound])
show ?thesis
by (rule checked-staged-security-with-data-state-query-bad-verifier-event-bound
  [OF query-hit-bound])
qed

lemma checked-staged-security-with-data-state-query-bad-bound-from-partial-opening-fixed-query-error-cover:
assumes wf: staged-budget-wellformed budgets
and controlled: staged-adversary-controlled budgets A
and cover:
 $\bigwedge$  data attacker-state  $i$ .
   $i < \text{rounds} \implies$ 
  query-header-supported-partial-opening-success-indices-at
    (verifier-state-from-adversary attacker-state
      (staged-proof-transcript data))
    (staged-trace-root data)
    (staged-trace-fri-roots data)
    (staged-trace-final data)
    (staged-alphas data)
    (staged-degree data)

```

(staged-composition-fri-roots data)
 (staged-composition-final data)
 $i \subseteq B$
and raw-bound: query-index-raw-preimage-bound
and subset: $B \subseteq \text{query-sample-space}$
and frac:
 $\text{nnreal } (\text{card } B) / \text{nnreal } (\text{card } \text{query-sample-space}) \leq$
 query-error-bound
shows
 $\text{wp-event } (\text{checked-staged-security-experiment-with-data-state } A)$
 $\text{staged-security-with-data-state-query-bad-hit}$
 $\text{adversary-initial-state} \leq$
 nnreal
 $(\text{staged-attacker-query-budget budgets} +$
 $\text{staged-challenge-query-budget}) * \text{query-error-bound}$
proof –
have $\text{wp-event } (\text{checked-staged-security-experiment-with-data-state } A)$
 $\text{staged-security-with-data-state-query-bad-hit}$
 $\text{adversary-initial-state} \leq$
 $\text{wp-event } (\text{checked-staged-security-experiment-with-data-state } A)$
 $\text{staged-security-with-data-state-query-partial-opening-hit}$
 $\text{adversary-initial-state}$
by (rule
 $\text{checked-staged-security-with-data-state-query-bad-bound-from-partial-opening-hit}$
 $[\text{OF wf controlled}]$)
also have ... $\leq$
 nnreal
 $(\text{staged-attacker-query-budget budgets} +$
 $\text{staged-challenge-query-budget}) * \text{query-error-bound}$
by (rule
 $\text{checked-staged-security-with-data-state-partial-opening-hit-bound-from-fixed-query-error-cover}$
 $[\text{OF wf controlled cover raw-bound subset frac}]$)
finally show ?thesis .
qed

lemma verifier-query-round-program-authenticated-initial-openings:
fixes $s \ t :: ('f, 'a) \text{protocol-channel-scheme}$
assumes fl-eq: $\text{fl} = (b, \text{composition-root}) \# \text{fl-tail}$
and outcome:
 $\text{Some } ((), t) \in$
 set-dist
 $(\text{execute}$
 $(\text{verifier-query-round-program fr f-fl f-final as fl final}) \ s)$
obtains raw idx trace-leaves trace-openings composition-openings **where**
 $\text{idx} = \text{index } (\text{to-nat raw})$
 $\text{map opening-value trace-openings} = \text{trace-leaves}$
 $\text{map opening-index trace-openings} = \text{powers-scaled idx}$
 $\text{partial-authenticated-table fr } (\text{scale} * \text{clength}) \text{ trace-openings } t$
 $\text{partial-authenticated-table composition-root } (\text{scale} * \text{clength})$

```

    composition-openings t
  map opening-index composition-openings =
    [idx, fri-sibling-index (scale * clength) idx]
proof -
from outcome[unfolded verifier-query-round-program-alt-def]
obtain raw s0 where tail:
  Some ((), t) ∈
    set-dist
      (execute
        (verifier-query-round-after-index-program
          fr f-fl f-final as fl final raw) s0)
by (auto elim!: set-dist-bindE)
show ?thesis
proof (rule verifier-query-round-after-index-authenticated-trace-openings
  [OF tail])
fix idx trace-leaves trace-openings
assume idx-eq: idx = index (to-nat raw)
and trace-values:
  map opening-value trace-openings = trace-leaves
and trace-indices:
  map opening-index trace-openings = powers-scaled idx
and trace-table:
  partial-authenticated-table fr (scale * clength) trace-openings t
show ?thesis
proof (rule
  verifier-query-round-after-index-authenticated-composition-openings
  [OF fl-eq tail])
fix idx' composition-openings
assume idx'-eq: idx' = index (to-nat raw)
and comp-table:
  partial-authenticated-table composition-root (scale * clength)
  composition-openings t
and comp-indices:
  map opening-index composition-openings =
    [idx', fri-sibling-index (scale * clength) idx']
have idx'-idx: idx' = idx
using idx-eq idx'-eq by simp
show ?thesis
by (rule that[OF idx-eq trace-values trace-indices trace-table
  comp-table])
  (use comp-indices idx'-idx in simp)
qed
qed
qed

lemma verifier-query-round-program-authenticated-initial-openings-with-lookup:
fixes s t :: ('f, 'a) protocol-channel-scheme
assumes fl-eq: fl = (b, composition-root) # fl-tail
and outcome:

```

```

    Some (( ), t) ∈
      set-dist
        (execute
          (verifier-query-round-program fr f-fl f-final as fl final) s)
obtains raw idx trace-leaves trace-openings composition-openings where
  idx = index (to-nat raw)
  map opening-value trace-openings = trace-leaves
  map opening-index trace-openings = powers-scaled idx
  partial-authenticated-table fr (scale * clength) trace-openings t
  partial-authenticated-table composition-root (scale * clength)
    composition-openings t
  map opening-index composition-openings =
    [idx, fri-sibling-index (scale * clength) idx]
  fmlookup (HashMap t)
    (QueryIndexChallenge (PQueryCounter s) (PState s)) = Some raw
proof –
from outcome[unfolded verifier-query-round-program-alt-def]
obtain raw s0 where rand:
  Some (raw, s0) ∈
    set-dist (execute receive-query-index-challenge s)
and tail:
  Some (( ), t) ∈
    set-dist
      (execute
        (verifier-query-round-after-index-program
          fr f-fl f-final as fl final raw) s0)
by (auto elim!: set-dist-bindE)
have lookup-s0:
  fmlookup (HashMap s0)
    (QueryIndexChallenge (PQueryCounter s) (PState s)) = Some raw
using receive-query-index-challenge-outcome[OF rand] by simp
have lookup-t:
  fmlookup (HashMap t)
    (QueryIndexChallenge (PQueryCounter s) (PState s)) = Some raw
using verifier-query-round-after-index-program-preserves-query-lookup
  [OF tail, of PQueryCounter s PState s] lookup-s0
by simp
show ?thesis
proof (rule verifier-query-round-after-index-authenticated-trace-openings
  [OF tail])
fix idx trace-leaves trace-openings
assume idx-eq: idx = index (to-nat raw)
and trace-values:
  map opening-value trace-openings = trace-leaves
and trace-indices:
  map opening-index trace-openings = powers-scaled idx
and trace-table:
  partial-authenticated-table fr (scale * clength) trace-openings t
show ?thesis

```

```

proof (rule
  verifier-query-round-after-index-authenticated-composition-openings
  [OF fl-eq tail])
fix idx' composition-openings
assume idx'-eq: idx' = index (to-nat raw)
and comp-table:
  partial-authenticated-table composition-root (scale * clength)
  composition-openings t
and comp-indices:
  map opening-index composition-openings =
  [idx', fri-sibling-index (scale * clength) idx']
have idx'-idx: idx' = idx
using idx-eq idx'-eq by simp
show ?thesis
by (rule that[OF idx-eq trace-values trace-indices trace-table
  comp-table - lookup-t])
  (use comp-indices idx'-idx in simp)
qed
qed
qed

lemma verifier-query-round-program-authenticated-initial-openings-with-chunk:
fixes s t :: ('f, 'a) protocol-channel-scheme
assumes fl-eq: fl = (b, composition-root) # fl-tail
and outcome:
  Some (( ), t) ∈
  set-dist
  (execute
    (verifier-query-round-program fr f-fl f-final as fl final) s)
obtains raw idx chunk trace-leaves trace-openings composition-openings
where
  idx = index (to-nat raw)
  verifier-query-round-chunk idx (map snd f-fl) (map snd fl) chunk
  PTranscript s = chunk @ PTranscript t
  PState t = foldl concat (PState s) chunk
  s ≤ t
  PQueryCounter t = Suc (PQueryCounter s)
  map opening-value trace-openings = trace-leaves
  map opening-index trace-openings = powers-scaled idx
  partial-authenticated-table fr (scale * clength) trace-openings t
  partial-authenticated-table composition-root (scale * clength)
  composition-openings t
  map opening-index composition-openings =
  [idx, fri-sibling-index (scale * clength) idx]
  fmlookup (HashMap t)
  (QueryIndexChallenge (PQueryCounter s) (PState s)) = Some raw
proof –
show ?thesis
proof (rule verifier-query-round-program-outcome[OF outcome])

```

```

fix raw0 idx0 chunk
assume idx0-eq: idx0 = index (to-nat raw0)
  and chunk-shape:
    verifier-query-round-chunk idx0 (map snd f-fl) (map snd fl)
    chunk
  and transcript:
    PTranscript s = chunk @ PTranscript t
  and state:
    PState t = foldl concat (PState s) chunk
  and ext: s ≤ t
  and counter:
    PQueryCounter t = Suc (PQueryCounter s)
  and lookup0:
    fmlookup (HashMap t)
      (QueryIndexChallenge (PQueryCounter s) (PState s)) = Some raw0
show ?thesis
proof (rule
  verifier-query-round-program-authenticated-initial-openings-with-lookup
  [OF fl-eq outcome])
fix raw idx trace-leaves trace-openings composition-openings
assume idx-eq: idx = index (to-nat raw)
  and trace-values:
    map opening-value trace-openings = trace-leaves
  and trace-indices:
    map opening-index trace-openings = powers-scaled idx
  and trace-table:
    partial-authenticated-table fr (scale * clength)
    trace-openings t
  and comp-table:
    partial-authenticated-table composition-root (scale * clength)
    composition-openings t
  and comp-indices:
    map opening-index composition-openings =
      [idx, fri-sibling-index (scale * clength) idx]
  and lookup:
    fmlookup (HashMap t)
      (QueryIndexChallenge (PQueryCounter s) (PState s)) = Some raw
have raw-eq: raw = raw0
  using lookup lookup0 by simp
have idx-idx0: idx = idx0
  using idx-eq idx0-eq raw-eq by simp
show ?thesis
  by (rule that[OF idx-eq - transcript state ext counter
    trace-values trace-indices trace-table comp-table comp-indices
    lookup])
    (use chunk-shape idx-idx0 in simp)
qed
qed
qed

```

lemma *ntimes-verifier-query-rounds-authenticated-initial-openings-at:*
fixes $s\ t :: ('f, 'a)\ \text{protocol-channel-scheme}$
assumes $\text{fl-eg: } \text{fl} = (b, \text{composition-root}) \# \text{fl-tail}$
and *outcome:*
 $\text{Some } (\text{results}, t) \in$
 set-dist
 $(\text{execute}$
 $(\text{ntimes}$
 $(\text{verifier-query-round-program } \text{fr } f\text{-fl } f\text{-final } \text{as } \text{fl } \text{final})\ n)$
 $s)$
and *i-bound:* $i < n$
obtains $\text{raw } \text{idx } \text{trace-leaves } \text{trace-openings } \text{composition-openings}$ **where**
 $\text{idx} = \text{index } (\text{to-nat } \text{raw})$
 $\text{map } \text{opening-value } \text{trace-openings} = \text{trace-leaves}$
 $\text{map } \text{opening-index } \text{trace-openings} = \text{powers-scaled } \text{idx}$
 $\text{partial-authenticated-table } \text{fr } (\text{scale} * \text{clength})\ \text{trace-openings } t$
 $\text{partial-authenticated-table } \text{composition-root } (\text{scale} * \text{clength})$
 $\text{composition-openings } t$
 $\text{map } \text{opening-index } \text{composition-openings} =$
 $[\text{idx}, \text{fri-sibling-index } (\text{scale} * \text{clength})\ \text{idx}]$
proof $-$
have
 $\exists \text{raw } \text{idx } \text{trace-leaves } \text{trace-openings } \text{composition-openings}.$
 $\text{idx} = \text{index } (\text{to-nat } \text{raw}) \wedge$
 $\text{map } \text{opening-value } \text{trace-openings} = \text{trace-leaves} \wedge$
 $\text{map } \text{opening-index } \text{trace-openings} = \text{powers-scaled } \text{idx} \wedge$
 $\text{partial-authenticated-table } \text{fr } (\text{scale} * \text{clength})\ \text{trace-openings } t \wedge$
 $\text{partial-authenticated-table } \text{composition-root } (\text{scale} * \text{clength})$
 $\text{composition-openings } t \wedge$
 $\text{map } \text{opening-index } \text{composition-openings} =$
 $[\text{idx}, \text{fri-sibling-index } (\text{scale} * \text{clength})\ \text{idx}]$
using *outcome i-bound*
proof (*induction n arbitrary: i s results t*)
case 0
then show *?case*
by *simp*
next
case ($\text{Suc } n$)
from $\text{Suc.prem}(1)$ **obtain** $u\ \text{results}'\ s1$ **where**
head:
 $\text{Some } (u, s1) \in$
 set-dist
 $(\text{execute}$
 $(\text{verifier-query-round-program } \text{fr } f\text{-fl } f\text{-final } \text{as } \text{fl } \text{final})\ s)$
and *tail:*
 $\text{Some } (\text{results}', t) \in$
 set-dist
 $(\text{execute}$


```

      (ntimes
        (verifier-query-round-program fr f-fl f-final as fl final)
        n) s1)
  by (auto elim!: set-dist-bindE)
have u-eq: u = ()
  by (cases u) simp
show ?case
proof (cases i)
  case 0
  show ?thesis
proof (rule verifier-query-round-program-authenticated-initial-openings
  [OF fl-eq head[unfolded u-eq]])
  fix raw idx trace-leaves trace-openings composition-openings
  assume idx-eq: idx = index (to-nat raw)
  and trace-values:
    map opening-value trace-openings = trace-leaves
  and trace-indices:
    map opening-index trace-openings = powers-scaled idx
  and trace-table-s1:
    partial-authenticated-table fr (scale * clength)
    trace-openings s1
  and comp-table-s1:
    partial-authenticated-table composition-root (scale * clength)
    composition-openings s1
  and comp-indices:
    map opening-index composition-openings =
      [idx, fri-sibling-index (scale * clength) idx]
  from ntimes-verifier-query-rounds-outcome[OF tail]
  obtain raw-idxs query-idxs query-chunks where
    s1-t: s1 ≤ t
  by (elim exE conjE)
  have trace-table-t:
    partial-authenticated-table fr (scale * clength)
    trace-openings t
  by (rule partial-authenticated-table-mono[OF trace-table-s1 s1-t])
  have comp-table-t:
    partial-authenticated-table composition-root (scale * clength)
    composition-openings t
  by (rule partial-authenticated-table-mono[OF comp-table-s1 s1-t])
show ?thesis
  apply (rule exI[where x=raw])
  apply (rule exI[where x=idx])
  apply (rule exI[where x=trace-leaves])
  apply (rule exI[where x=trace-openings])
  apply (rule exI[where x=composition-openings])
  apply (intro conjI)
    apply (rule idx-eq)
    apply (rule trace-values)
    apply (rule trace-indices)

```

```

      apply (rule trace-table-t)
    apply (rule comp-table-t)
    apply (rule comp-indices)
  done
qed
next
case (Suc j)
have j-bound:  $j < n$ 
  using Suc.premis(2)  $\langle i = \text{Suc } j \rangle$  by simp
from Suc.IH[OF tail j-bound] show ?thesis .
qed
qed
then obtain raw idx trace-leaves trace-openings composition-openings where
  idx-eq:  $\text{idx} = \text{index } (\text{to-nat raw})$ 
and trace-values:
  map opening-value trace-openings = trace-leaves
and trace-indices:
  map opening-index trace-openings = powers-scaled idx
and trace-table:
  partial-authenticated-table fr (scale * clength) trace-openings t
and comp-table:
  partial-authenticated-table composition-root (scale * clength)
  composition-openings t
and comp-indices:
  map opening-index composition-openings =
    [idx, fri-sibling-index (scale * clength) idx]
by (elim exE conjE)
show ?thesis
by (rule that[OF idx-eq trace-values trace-indices trace-table
  comp-table comp-indices])
qed

lemma accepted-with-bound-tables-partial-opening-header-witness-at:
  assumes outcome:
    Some (result, final-state)  $\in \text{set-dist } (\text{execute verify-monad } s)$ 
  and header:
    verifier-header-transcript s fr f-fri-roots f-final as dg
    composition-fri-roots final rest
  and bound:
    accepted-with-bound-tables s (Some (result, final-state))
    trace-table composition-table as query-idxs
  and i-bound:  $i < \text{rounds}$ 
  obtains trace-openings composition-openings where
    (trace-openings, composition-openings)  $\in$ 
    query-header-supported-partial-opening-witnesses s fr f-fri-roots
    f-final as dg composition-fri-roots final
proof -
  show ?thesis
proof (rule accepted-with-bound-tables-partial-opening-witnesses-Some[OF bound])

```

```

fix fr' f-fri-roots' f-final' dg' composition-fri-roots' final'
  rest' trace-openingss composition-openingss
assume
  header':
    verifier-header-transcript s fr' f-fri-roots' f-final' as dg'
      composition-fri-roots' final' rest'
and comp-nonempty': composition-fri-roots'  $\neq$  []
and trace-partial:
  accepted-with-partial-trace-openings s (Some (result, final-state))
    fr' query-idxs trace-openingss
and comp-partial:
  accepted-with-partial-composition-openings s
    (Some (result, final-state)) (hd composition-fri-roots')
    query-idxs composition-openingss
have eqs:
  fr' = fr  $\wedge$ 
  f-fri-roots' = f-fri-roots  $\wedge$ 
  f-final' = f-final  $\wedge$ 
  dg' = dg  $\wedge$ 
  composition-fri-roots' = composition-fri-roots  $\wedge$ 
  final' = final  $\wedge$ 
  rest' = rest
  using verifier-header-transcript-unique[OF header' header] by simp
have trace-table-i:
  partial-authenticated-table fr (scale * clength)
    (trace-openingss ! i) final-state
  using trace-partial i-bound eqs
  unfolding accepted-with-partial-trace-openings-def by blast
have comp-table-i:
  partial-authenticated-table (hd composition-fri-roots) (scale * clength)
    (composition-openingss ! i) final-state
  using comp-partial i-bound eqs
  unfolding accepted-with-partial-composition-openings-def by simp
have witness:
  (trace-openingss ! i, composition-openingss ! i)  $\in$ 
    query-header-supported-partial-opening-witnesses s fr f-fri-roots
      f-final as dg composition-fri-roots final
  unfolding query-header-supported-partial-opening-witnesses-def
  using outcome header trace-table-i comp-table-i comp-nonempty' eqs
  by auto
show ?thesis
  by (rule that[OF witness])
qed
qed

lemma checked-actual-alpha-prefix-support-partial-opening-header-witness-at:
fixes prefix prefix-state data attacker-state result final-state
defines
  s  $\equiv$ 

```

verifier-state-from-adversary attacker-state
(staged-proof-transcript data)
assumes *support*:
Some (((((prefix, prefix-state), data), attacker-state),
result), final-state) ∈
set-dist
(execute
(checked-staged-security-experiment-with-actual-alpha-prefix-data-state
A)
adversary-initial-state)
and *header*:
verifier-header-transcript s fr f-fri-roots f-final as dg
composition-fri-roots final rest
and *bound*:
accepted-with-bound-tables s (Some (result, final-state))
trace-table composition-table as query-idxs
and *i-bound*: *i < rounds*
obtains *trace-openings composition-openings* **where**
(trace-openings, composition-openings) ∈
query-header-supported-partial-opening-witnesses s fr f-fri-roots
f-final as dg composition-fri-roots final
proof –
have *verify-out*:
Some (result, final-state) ∈ set-dist (execute verify-monad s)
using *support*
unfolding *checked-staged-security-experiment-with-actual-alpha-prefix-data-state-def*
s-def
by *(auto elim!: set-dist-bindE split: prod.splits)*
show *?thesis*
by *(rule accepted-with-bound-tables-partial-opening-header-witness-at*
[OF verify-out header bound i-bound that])
qed

lemma *checked-actual-alpha-prefix-branch-subtree-verifier-partial-opening-header-witness-at*:
fixes *prefix prefix-state data attacker-state result final-state*
defines
s ≡
verifier-state-from-adversary attacker-state
(staged-proof-transcript data)
assumes *support*:
Some (((((p, prefix-state), data), attacker-state),
result), final-state) ∈
set-dist
(execute
(checked-staged-security-experiment-with-actual-alpha-prefix-data-state
A)
adversary-initial-state)
and *hit*:
checked-staged-security-with-actual-alpha-prefix-branch-tree-subtree-verifier-output-hit

```

    (Some (((((prefix, prefix-state), data), attacker-state),
      result), final-state))
  and i-bound: i < rounds
obtain as dg composition-fri-roots final rest trace-table composition-table
  query-idxs trace-openings composition-openings where
  accepted-with-bound-tables s (Some (result, final-state))
  trace-table composition-table as query-idxs
  verifier-header-transcript s
  (staged-trace-root data) (staged-trace-fri-roots data)
  (staged-trace-final data) as dg composition-fri-roots final rest
  (trace-openings, composition-openings) ∈
  query-header-supported-partial-opening-witnesses s
  (staged-trace-root data) (staged-trace-fri-roots data)
  (staged-trace-final data) as dg composition-fri-roots final
proof –
obtain trace-table composition-table as query-idxs dg
  composition-fri-roots final rest trace-tree composition-tree where
  verify-out:
    Some (result, final-state) ∈ set-dist (execute verify-monad s)
  and bound:
    accepted-with-bound-tables s (Some (result, final-state))
    trace-table composition-table as query-idxs
  and header:
    verifier-header-transcript s
    (staged-trace-root data) (staged-trace-fri-roots data)
    (staged-trace-final data) as dg composition-fri-roots final rest
  using hit
  unfolding
    checked-staged-security-with-actual-alpha-prefix-branch-tree-subtree-verifier-output-hit-def
    s-def Let-def
  by auto
obtain trace-openings composition-openings where witness:
  (trace-openings, composition-openings) ∈
  query-header-supported-partial-opening-witnesses s
  (staged-trace-root data) (staged-trace-fri-roots data)
  (staged-trace-final data) as dg composition-fri-roots final
  by (rule accepted-with-bound-tables-partial-opening-header-witness-at
    [OF verify-out header bound i-bound])
show ?thesis
  by (rule that[OF bound header witness])
qed

definition
  checked-actual-alpha-prefix-branch-subtree-verifier-partial-opening-witness-hit
  :: ((((((f × 'f list × 'f list × 'f) ×
    'f protocol-channel) × 'f staged-proof-data) ×
    'f protocol-channel) × unit list) ×
    'f protocol-channel) option ⇒ bool
where

```

```

checked-actual-alpha-prefix-branch-subtree-verifier-partial-opening-witness-hit
  out  $\longleftrightarrow$ 
  (case out of
    None  $\Rightarrow$  False
  | Some (((((prefix, prefix-state), data), attacker-state),
    result), final-state)  $\Rightarrow$ 
    (let s =
      verifier-state-from-adversary attacker-state
      (staged-proof-transcript data)
    in  $\exists$  as dg composition-fri-roots final rest trace-table
      composition-table query-idxs trace-openings composition-openings.
      accepted-with-bound-tables s (Some (result, final-state))
      trace-table composition-table as query-idxs  $\wedge$ 
      verifier-header-transcript s
      (staged-trace-root data) (staged-trace-fri-roots data)
      (staged-trace-final data) as dg composition-fri-roots final rest  $\wedge$ 
      (trace-openings, composition-openings)  $\in$ 
      query-header-supported-partial-opening-witnesses s
      (staged-trace-root data) (staged-trace-fri-roots data)
      (staged-trace-final data) as dg composition-fri-roots final))

lemma checked-actual-alpha-prefix-branch-subtree-verifier-imp-partial-opening-witness-hit-on-support:
  assumes support:
    out  $\in$ 
    set-dist
    (execute
      (checked-staged-security-experiment-with-actual-alpha-prefix-data-state
        A)
      adversary-initial-state)
  and hit:
    checked-staged-security-with-actual-alpha-prefix-branch-tree-subtree-verifier-output-hit
    out
  shows
    checked-actual-alpha-prefix-branch-subtree-verifier-partial-opening-witness-hit
    out
proof (cases out)
case None
then show ?thesis
  using hit
  unfolding
    checked-staged-security-with-actual-alpha-prefix-branch-tree-subtree-verifier-output-hit-def
  by simp
next
case (Some result-pack)
then obtain prefix prefix-state data attacker-state result final-state where
  out-eq:
    out = Some (((((prefix, prefix-state), data), attacker-state),
      result), final-state)
  by (cases result-pack, auto split: prod.splits)

```

```

let ?s =
  verifier-state-from-adversary attacker-state
  (staged-proof-transcript data)
have support-some:
  Some (((((prefix, prefix-state), data), attacker-state),
    result), final-state) ∈
  set-dist
  (execute
    (checked-staged-security-experiment-with-actual-alpha-prefix-data-state
      A)
    adversary-initial-state)
using support unfolding out-eq .
have hit-some:
  checked-staged-security-with-actual-alpha-prefix-branch-tree-subtree-verifier-output-hit
  (Some (((((prefix, prefix-state), data), attacker-state),
    result), final-state))
using hit unfolding out-eq .
obtain as dg composition-fri-roots final rest trace-table composition-table
  query-idxs trace-openings composition-openings where
  bound:
    accepted-with-bound-tables ?s (Some (result, final-state))
    trace-table composition-table as query-idxs
and header:
  verifier-header-transcript ?s
  (staged-trace-root data) (staged-trace-fri-roots data)
  (staged-trace-final data) as dg composition-fri-roots final rest
and witness:
  (trace-openings, composition-openings) ∈
  query-header-supported-partial-opening-witnesses ?s
  (staged-trace-root data) (staged-trace-fri-roots data)
  (staged-trace-final data) as dg composition-fri-roots final
proof -
  obtain trace-table composition-table as query-idxs dg
    composition-fri-roots final rest trace-tree composition-tree where
  verify-out:
    Some (result, final-state) ∈ set-dist (execute verify-monad ?s)
  and bound:
    accepted-with-bound-tables ?s (Some (result, final-state))
    trace-table composition-table as query-idxs
  and header:
    verifier-header-transcript ?s
    (staged-trace-root data) (staged-trace-fri-roots data)
    (staged-trace-final data) as dg composition-fri-roots final rest
  using hit-some
  unfolding
    checked-staged-security-with-actual-alpha-prefix-branch-tree-subtree-verifier-output-hit-def
    Let-def
  by auto
  obtain trace-openings composition-openings where witness:

```

```

    (trace-openings, composition-openings) ∈
      query-header-supported-partial-opening-witnesses ?s
      (staged-trace-root data) (staged-trace-fri-roots data)
      (staged-trace-final data) as dg composition-fri-roots final
  by (rule accepted-with-bound-tables-partial-opening-header-witness-at
      [OF verify-out header bound rounds-positive])
show ?thesis
  by (rule that[OF bound header witness])
qed
show ?thesis
  unfolding out-eq
    checked-actual-alpha-prefix-branch-subtree-verifier-partial-opening-witness-hit-def
    Let-def
  apply simp
  apply (rule exI[where x=as])
  apply (intro conjI)
  apply (rule exI[where x=trace-table])
  apply (rule exI[where x=composition-table])
  apply (rule exI[where x=query-idxs])
  apply (rule bound)
  apply (rule exI[where x=dg])
  apply (rule exI[where x=composition-fri-roots])
  apply (rule exI[where x=final])
  apply (intro conjI)
  apply (rule exI[where x=rest])
  apply (rule header)
  apply (rule exI[where x=trace-openings])
  apply (rule exI[where x=composition-openings])
  apply (rule witness)
done
qed

end

end

```

```

theory Soundness-Staged-Partial-Query-Branch
  imports Soundness-Staged-Partial-Query
begin

```

Branch-specific probability bridges for the partial-opening query layer. This keeps branch/tree-output integration material out of the size-sensitive base partial-query theory.

```

context soundness
begin

```

```

definition checked-staged-security-with-actual-alpha-prefix-query-partial-opening-hit
  :: ((((((f × 'f list × 'f list × 'f) ×

```


$'f \text{ protocol-channel}) \times 'f \text{ staged-proof-data}) \times$
 $'f \text{ protocol-channel}) \times \text{unit list}) \times$
 $'f \text{ protocol-channel}) \text{ option} \Rightarrow \text{bool}$

where

$\text{checked-staged-security-with-actual-alpha-prefix-query-partial-opening-hit}$
 $\text{out} \longleftrightarrow$
 $(\text{case out of}$
 $\text{None} \Rightarrow \text{False}$
 $| \text{Some } (x, t) \Rightarrow$
 $\text{staged-security-with-data-state-query-partial-opening-hit}$
 $(\text{Some } (((\text{snd } (\text{fst } (\text{fst } x)), \text{snd } (\text{fst } x)), \text{snd } x), t)))$

lemma *checked-staged-security-with-actual-alpha-prefix-query-partial-opening-hitE*:

assumes *hit*:

$\text{checked-staged-security-with-actual-alpha-prefix-query-partial-opening-hit}$
 $(\text{Some } ((((\text{prefix}, \text{prefix-state}), \text{data}), \text{attacker-state}), \text{result}),$
 $\text{final-state}))$

obtains *i* *raw* *trace-openings* *composition-openings* **where**

$i < \text{rounds}$
 $\text{fmlookup } (\text{HashMap } \text{final-state})$
 $(\text{QueryIndexChallenge } i$
 $(\text{state-after-query-chunks}$
 $(\text{staged-query-start-hash } \text{data})$
 $(\text{staged-query-chunks } \text{data}) \ i)) =$
 Some raw
 $\text{index } (\text{to-nat raw}) \in \text{query-sample-space}$
 $(\text{trace-openings}, \text{composition-openings}) \in$
 $\text{query-header-supported-partial-opening-witnesses}$
 $(\text{verifier-state-from-adversary } \text{attacker-state}$
 $(\text{staged-proof-transcript } \text{data}))$
 $(\text{staged-trace-root } \text{data})$
 $(\text{staged-trace-fri-roots } \text{data})$
 $(\text{staged-trace-final } \text{data})$
 $(\text{staged-alphas } \text{data})$
 $(\text{staged-degree } \text{data})$
 $(\text{staged-composition-fri-roots } \text{data})$
 $(\text{staged-composition-final } \text{data})$
 $\text{partial-query-openings-consistent } \text{trace-openings } \text{composition-openings}$
 $(\text{staged-alphas } \text{data}) (\text{index } (\text{to-nat raw}))$

proof –

have *data-hit*:

$\text{staged-security-with-data-state-query-partial-opening-hit}$
 $(\text{Some } (((\text{data}, \text{attacker-state}), \text{result}), \text{final-state}))$

using *hit*

unfolding

$\text{checked-staged-security-with-actual-alpha-prefix-query-partial-opening-hit-def}$

by *simp*

then obtain *i* **where** *i*-bound: $i < \text{rounds}$

and *hit-at*:

```

    staged-security-with-data-state-query-partial-opening-hit-at i
    (Some (((data, attacker-state), result), final-state))
unfolding staged-security-with-data-state-query-partial-opening-hit-def
by blast
from staged-security-with-data-state-query-partial-opening-hit-atE
    [OF hit-at]
obtain raw where lookup:
    fmlookup (HashMap final-state)
    (QueryIndexChallenge i
    (state-after-query-chunks
    (staged-query-start-hash data)
    (staged-query-chunks data) i)) =
    Some raw
and raw-hit:
    index (to-nat raw) ∈
    query-header-supported-partial-opening-success-indices-at
    (verifier-state-from-adversary attacker-state
    (staged-proof-transcript data))
    (staged-trace-root data)
    (staged-trace-fri-roots data)
    (staged-trace-final data)
    (staged-alphas data)
    (staged-degree data)
    (staged-composition-fri-roots data)
    (staged-composition-final data)
    i
by blast
from query-header-supported-partial-opening-success-indices-atE
    [OF raw-hit]
obtain trace-openings composition-openings where idx-sample:
    index (to-nat raw) ∈ query-sample-space
and witness:
    (trace-openings, composition-openings) ∈
    query-header-supported-partial-opening-witnesses
    (verifier-state-from-adversary attacker-state
    (staged-proof-transcript data))
    (staged-trace-root data)
    (staged-trace-fri-roots data)
    (staged-trace-final data)
    (staged-alphas data)
    (staged-degree data)
    (staged-composition-fri-roots data)
    (staged-composition-final data)
and consistent:
    partial-query-openings-consistent trace-openings composition-openings
    (staged-alphas data) (index (to-nat raw))
by blast
show ?thesis
by (rule that[OF i-bound lookup idx-sample witness consistent])

```

qed

lemma *checked-staged-security-with-actual-alpha-prefix-query-partial-opening-hit-projection:*

```
wp-event
  (checked-staged-security-experiment-with-actual-alpha-prefix-data-state A)
  checked-staged-security-with-actual-alpha-prefix-query-partial-opening-hit
  adversary-initial-state =
wp-event (checked-staged-security-experiment-with-data-state A)
  staged-security-with-data-state-query-partial-opening-hit
  adversary-initial-state
```

proof –

```
let ?project =
  λx. ((snd (fst (fst x)), snd (fst x)), snd x)
```

have

```
wp-event (checked-staged-security-experiment-with-data-state A)
  staged-security-with-data-state-query-partial-opening-hit
  adversary-initial-state =
wp-event
  (checked-staged-security-experiment-with-actual-alpha-prefix-data-state A
    ≫ (λx. return (?project x)))
  staged-security-with-data-state-query-partial-opening-hit
  adversary-initial-state
```

using

```
checked-staged-security-experiment-with-actual-alpha-prefix-data-state-projection
[of A]
```

by *simp*

also have ... =

```
wp-event
  (checked-staged-security-experiment-with-actual-alpha-prefix-data-state A)
  (λout. case out of
    None ⇒ staged-security-with-data-state-query-partial-opening-hit None
  | Some (x, t) ⇒
    staged-security-with-data-state-query-partial-opening-hit
      (Some (?project x, t)))
  adversary-initial-state
by (rule wp-event-bind-return-map)
```

also have ... =

```
wp-event
  (checked-staged-security-experiment-with-actual-alpha-prefix-data-state A)
  checked-staged-security-with-actual-alpha-prefix-query-partial-opening-hit
  adversary-initial-state
```

unfolding

```
checked-staged-security-with-actual-alpha-prefix-query-partial-opening-hit-def
staged-security-with-data-state-query-partial-opening-hit-def
staged-security-with-data-state-query-partial-opening-hit-at-def
```

by (rule arg-cong[**where**

```
f = λQ. wp-event
  (checked-staged-security-experiment-with-actual-alpha-prefix-data-state A)
  Q adversary-initial-state])
```

```

      (rule ext, simp split: option.splits prod.splits)
    finally show ?thesis
      by simp
  qed

lemma checked-staged-security-with-actual-alpha-prefix-query-partial-opening-hit-bound-from-fixed-query-error-cover:
  assumes wf: staged-budget-wellformed budgets
  and controlled: staged-adversary-controlled budgets A
  and cover:
     $\bigwedge$ data attacker-state i.
      i < rounds  $\implies$ 
        query-header-supported-partial-opening-success-indices-at
          (verifier-state-from-adversary attacker-state
            (staged-proof-transcript data))
            (staged-trace-root data)
            (staged-trace-fri-roots data)
            (staged-trace-final data)
            (staged-alphas data)
            (staged-degree data)
            (staged-composition-fri-roots data)
            (staged-composition-final data)
          i  $\subseteq$  B
  and raw-bound: query-index-raw-preimage-bound
  and subset: B  $\subseteq$  query-sample-space
  and frac:
    nnreal (card B) / nnreal (card query-sample-space)  $\leq$ 
      query-error-bound
  shows
    wp-event
      (checked-staged-security-experiment-with-actual-alpha-prefix-data-state A)
      checked-staged-security-with-actual-alpha-prefix-query-partial-opening-hit
      adversary-initial-state  $\leq$ 
        nnreal
          (staged-attacker-query-budget budgets +
            staged-challenge-query-budget) * query-error-bound
  proof -
    have data-bound:
      wp-event (checked-staged-security-experiment-with-data-state A)
        staged-security-with-data-state-query-partial-opening-hit
        adversary-initial-state  $\leq$ 
          nnreal
            (staged-attacker-query-budget budgets +
              staged-challenge-query-budget) * query-error-bound
    by (rule
      checked-staged-security-with-data-state-partial-opening-hit-bound-from-fixed-query-error-cover
      [OF wf controlled cover raw-bound subset frac])
  show ?thesis
    unfolding
      checked-staged-security-with-actual-alpha-prefix-query-partial-opening-hit-projection

```

by (rule data-bound)
qed

lemma *checked-staged-security-with-actual-alpha-prefix-support-imp-data-state-support-branch:*

assumes *support:*

*Some (((((prefix, prefix-state), data), attacker-state), result),
final-state) ∈
set-dist
(execute
(checked-staged-security-experiment-with-actual-alpha-prefix-data-state
A)
adversary-initial-state)*

shows

*Some (((data, attacker-state), result), final-state) ∈
set-dist
(execute (checked-staged-security-experiment-with-data-state A)
adversary-initial-state)*

proof –

let *?project* =

$\lambda x. ((\text{snd } (\text{fst } (\text{fst } x)), \text{snd } (\text{fst } x)), \text{snd } x)$

have *projected:*

*Some (((data, attacker-state), result), final-state) ∈
set-dist
(execute
(checked-staged-security-experiment-with-actual-alpha-prefix-data-state
A $\gg$ ($\lambda x. \text{return } (?project\ x)$))
adversary-initial-state)*

apply (rule set-dist-bindI[OF *support*])

apply *simp*

done

show *?thesis*

using *projected*

*checked-staged-security-experiment-with-actual-alpha-prefix-data-state-projection
[of A]*

by *simp*

qed

lemma *checked-staged-security-with-actual-alpha-prefix-bound-tables-all-queries-imp-query-partial-opening-hit-on-sup*

assumes *wf: staged-budget-wellformed budgets*

and *controlled: staged-adversary-controlled budgets A*

and *support:*

*Some (((((prefix, prefix-state), data), attacker-state), result),
final-state) ∈
set-dist
(execute
(checked-staged-security-experiment-with-actual-alpha-prefix-data-state
A)
adversary-initial-state)*

and *bound:*

```

    accepted-with-bound-tables
      (verifier-state-from-adversary attacker-state
       (staged-proof-transcript data))
      (Some (result, final-state)) trace-table composition-table as
      query-idxs
  and header:
    verifier-header-transcript
      (verifier-state-from-adversary attacker-state
       (staged-proof-transcript data))
      (staged-trace-root data) (staged-trace-fri-roots data)
      (staged-trace-final data) as dg composition-fri-roots final rest
  and all-queries:
    all-queries-consistent trace-table composition-table as
  shows
    checked-staged-security-with-actual-alpha-prefix-query-partial-opening-hit
      (Some (((((prefix, prefix-state), data), attacker-state), result),
              final-state)))
  proof —
  let ?s =
    verifier-state-from-adversary attacker-state
    (staged-proof-transcript data)
  have data-support:
    Some (((data, attacker-state), result), final-state) ∈
    set-dist
      (execute (checked-staged-security-experiment-with-data-state A)
               adversary-initial-state)
  by (rule
      checked-staged-security-with-actual-alpha-prefix-support-imp-data-state-support-branch
      [OF support])
  from checked-staged-security-experiment-with-data-state-outcomeE
  [OF data-support]
  obtain builder where builder:
    Some (data, attacker-state) ∈
    set-dist
      (execute (checked-staged-transcript-program A)
               adversary-initial-state)
  and verifier:
    Some (result, final-state) ∈ set-dist (execute verify-monad ?s)
  by blast
  have staged-header:
    verifier-header-transcript ?s
      (staged-trace-root data)
      (staged-trace-fri-roots data)
      (staged-trace-final data)
      (staged-alphas data)
      (staged-degree data)
      (staged-composition-fri-roots data)
      (staged-composition-final data)
      (List.concat (staged-query-chunks data))

```

```

  by (rule checked-staged-transcript-program-outcome-header-transcript
    [OF wf controlled builder])
have header-eq:
  as = staged-alphas data ∧
  dg = staged-degree data ∧
  composition-fri-roots = staged-composition-fri-roots data ∧
  final = staged-composition-final data ∧
  rest = List.concat (staged-query-chunks data)
  using verifier-header-transcript-unique[OF staged-header header]
  by simp
have shape:
  accepted-transcript-shape ?s (Some (result, final-state)) as query-idxs
  using bound
  unfolding accepted-with-bound-tables-def accepted-with-tables-def
  by simp
from checked-staged-security-with-data-state-accepted-shape-query-chunks
  [OF wf controlled data-support shape]
obtain raw-idxs where
  as-eq: as = staged-alphas data
  and len-raw: length raw-idxs = rounds
  and query-idxs-eq:
    query-idxs = map (λraw. index (to-nat raw)) raw-idxs
  and lookup:
    ∧ i. i < rounds ⇒
      fmllookup (HashMap final-state)
        (QueryIndexChallenge i
          (state-after-query-chunks
            (staged-query-start-hash data)
            (staged-query-chunks data) i)) =
        Some (raw-idxs ! i)
  by blast
have bound-staged:
  accepted-with-bound-tables ?s (Some (result, final-state))
  trace-table composition-table (staged-alphas data) query-idxs
  using bound as-eq by simp
have all-queries-staged:
  all-queries-consistent trace-table composition-table
    (staged-alphas data)
  using all-queries as-eq by simp
have i-bound: 0 < rounds
  by (rule rounds-positive)
have idx-sample:
  query-idxs ! 0 ∈ query-sample-space
proof -
  have len-query-idxs: length query-idxs = rounds
    by (rule accepted-with-bound-tables-shapes(4)[OF bound-staged])
  have query-idxs ! 0 ∈ set query-idxs
    by (rule nth-mem) (use i-bound len-query-idxs in simp)
  then show ?thesis

```

```

    by (rule accepted-with-bound-tables-query-sample-space
        [OF bound-staged])
qed
show ?thesis
proof (rule accepted-with-bound-tables-partial-opening-witnesses-Some
    [OF bound-staged])
  fix fr f-fri-roots f-final dg' composition-fri-roots' final'
    rest' trace-openings composition-openings
  assume header':
    verifier-header-transcript ?s fr f-fri-roots f-final
      (staged-alphas data) dg' composition-fri-roots' final' rest'
  and comp-nonempty: composition-fri-roots'  $\neq$  []
  and trace-partial:
    accepted-with-partial-trace-openings ?s
      (Some (result, final-state)) fr query-idxs trace-openings
  and comp-partial:
    accepted-with-partial-composition-openings ?s
      (Some (result, final-state)) (hd composition-fri-roots')
      query-idxs composition-openings
  and trace-candidate:
    partial-trace-table-candidate trace-table trace-openings
  and comp-candidate:
    partial-composition-table-candidate composition-table
      composition-openings
  have header-eq':
    fr = staged-trace-root data  $\wedge$ 
    f-fri-roots = staged-trace-fri-roots data  $\wedge$ 
    f-final = staged-trace-final data  $\wedge$ 
    dg' = staged-degree data  $\wedge$ 
    composition-fri-roots' = staged-composition-fri-roots data  $\wedge$ 
    final' = staged-composition-final data  $\wedge$ 
    rest' = List.concat (staged-query-chunks data)
  using verifier-header-transcript-unique[OF staged-header header']
  by simp
  have idx-in:
    query-idxs ! 0  $\in$ 
    query-header-supported-partial-opening-success-indices-at ?s fr
      f-fri-roots f-final (staged-alphas data) dg'
      composition-fri-roots' final' 0
  by (rule
    query-header-supported-partial-opening-success-indices-atI-from-partials-consistent
    [OF verifier trace-partial comp-partial trace-candidate
      comp-candidate header' comp-nonempty refl i-bound idx-sample
      all-queries-staged])
  have idx-in-staged:
    query-idxs ! 0  $\in$ 
    query-header-supported-partial-opening-success-indices-at ?s
      (staged-trace-root data)
      (staged-trace-fri-roots data)

```



```

    (staged-trace-final data)
    (staged-alphas data)
    (staged-degree data)
    (staged-composition-fri-roots data)
    (staged-composition-final data)
    0
  using idx-in header-eq' by simp
have raw-idx:
  index (to-nat (raw-idxs ! 0)) = query-idxs ! 0
  using query-idxs-eq len-raw i-bound by simp
have hit-at:
  staged-security-with-data-state-query-partial-opening-hit-at 0
  (Some (((data, attacker-state), result), final-state))
  unfolding staged-security-with-data-state-query-partial-opening-hit-at-def
  using i-bound lookup[OF i-bound] idx-in-staged raw-idx
  by (simp add: Let-def)
have hit:
  staged-security-with-data-state-query-partial-opening-hit
  (Some (((data, attacker-state), result), final-state))
  by (rule staged-security-with-data-state-query-partial-opening-hitI
    [OF i-bound hit-at])
show ?thesis
  unfolding
    checked-staged-security-with-actual-alpha-prefix-query-partial-opening-hit-def
  using hit by simp
qed
qed

definition checked-actual-alpha-prefix-branch-subtree-verifier-partial-opening-witness-all-queries-hit
:: ((((((f × 'f list × 'f list × 'f) ×
  'f protocol-channel) × 'f staged-proof-data) ×
  'f protocol-channel) × unit list) ×
  'f protocol-channel) option ⇒ bool
where
  checked-actual-alpha-prefix-branch-subtree-verifier-partial-opening-witness-all-queries-hit
  out ⇔
  (case out of
    None ⇒ False
  | Some (((((prefix, prefix-state), data), attacker-state),
    result), final-state) ⇒
    (let s =
      verifier-state-from-adversary attacker-state
      (staged-proof-transcript data)
    in ∃ as dg composition-fri-roots final rest trace-table
      composition-table query-idxs trace-openings composition-openings.
      accepted-with-bound-tables s (Some (result, final-state))
      trace-table composition-table as query-idxs ∧
      verifier-header-transcript s
      (staged-trace-root data) (staged-trace-fri-roots data))

```

(staged-trace-final data) as dg composition-fri-roots final rest $\wedge$
 (trace-openings, composition-openings) $\in$
 query-header-supported-partial-opening-witnesses s
 (staged-trace-root data) (staged-trace-fri-roots data)
 (staged-trace-final data) as dg composition-fri-roots final $\wedge$
 all-queries-consistent trace-table composition-table as))

lemma *checked-actual-alpha-prefix-branch-subtree-verifier-partial-opening-witness-all-queries-hit-imp-query-hit-on-su*

assumes *wf: staged-budget-wellformed budgets*

and *controlled: staged-adversary-controlled budgets A*

and *support:*

out $\in$

set-dist

(*execute*

(*checked-staged-security-experiment-with-actual-alpha-prefix-data-state*

A)

adversary-initial-state)

and *hit:*

checked-actual-alpha-prefix-branch-subtree-verifier-partial-opening-witness-all-queries-hit

out

shows

checked-staged-security-with-actual-alpha-prefix-query-partial-opening-hit

out

proof (*cases out*)

case *None*

then show *?thesis*

using *hit*

unfolding

checked-actual-alpha-prefix-branch-subtree-verifier-partial-opening-witness-all-queries-hit-def

by *simp*

next

case (*Some result-pack*)

then obtain *prefix prefix-state data attacker-state result final-state* **where**

out-eq:

out = *Some* (((((*prefix*, *prefix-state*), *data*), *attacker-state*),

result), *final-state*)

by (*cases result-pack*, *auto split: prod.splits*)

let *?s* =

verifier-state-from-adversary attacker-state

(*staged-proof-transcript data*)

have *support-some:*

Some (((((*prefix*, *prefix-state*), *data*), *attacker-state*),

result), *final-state*) $\in$

set-dist

(*execute*

(*checked-staged-security-experiment-with-actual-alpha-prefix-data-state*

A)

adversary-initial-state)

using *support* **unfolding** *out-eq* .

from *hit* **obtain** *as dg composition-fri-roots final rest trace-table*
composition-table query-idxs trace-openings composition-openings **where**
bound:
accepted-with-bound-tables ?s (Some (result, final-state))
trace-table composition-table as query-idxs
and *header:*
verifier-header-transcript ?s
(staged-trace-root data) (staged-trace-fri-roots data)
(staged-trace-final data) as dg composition-fri-roots final rest
and *all-queries:*
all-queries-consistent trace-table composition-table as
unfolding *out-eq*
checked-actual-alpha-prefix-branch-subtree-verifier-partial-opening-witness-all-queries-hit-def
Let-def
by *auto*
show *?thesis*
unfolding *out-eq*
by (rule
checked-staged-security-with-actual-alpha-prefix-bound-tables-all-queries-imp-query-partial-opening-hit-on-support
[OF wf controlled support-some bound header all-queries])
qed

lemma *checked-actual-alpha-prefix-branch-subtree-verifier-partial-opening-witness-all-queries-hit-bound-from-query-h*
assumes *wf: staged-budget-wellformed budgets*
and *controlled: staged-adversary-controlled budgets A*
and *query-bound:*
wp-event
(checked-staged-security-experiment-with-actual-alpha-prefix-data-state A)
checked-staged-security-with-actual-alpha-prefix-query-partial-opening-hit
adversary-initial-state $\leq Q$
shows
wp-event
(checked-staged-security-experiment-with-actual-alpha-prefix-data-state A)
checked-actual-alpha-prefix-branch-subtree-verifier-partial-opening-witness-all-queries-hit
adversary-initial-state $\leq Q$

proof –
have
wp-event
(checked-staged-security-experiment-with-actual-alpha-prefix-data-state A)
checked-actual-alpha-prefix-branch-subtree-verifier-partial-opening-witness-all-queries-hit
adversary-initial-state $\leq$
wp-event
(checked-staged-security-experiment-with-actual-alpha-prefix-data-state A)
checked-staged-security-with-actual-alpha-prefix-query-partial-opening-hit
adversary-initial-state
by (rule *wp-event-mono-on-support*)
(i rule
checked-actual-alpha-prefix-branch-subtree-verifier-partial-opening-witness-all-queries-hit-imp-query-hit-on-support
[OF wf controlled])

then show *?thesis*
by (rule order-trans[*OF - query-bound*])
qed

definition *checked-actual-alpha-prefix-branch-subtree-verifier-partial-opening-witness-query-hit*

$$\begin{aligned} &:: ((((((f \times 'f \text{ list} \times 'f \text{ list} \times 'f) \times \\ &\quad 'f \text{ protocol-channel}) \times 'f \text{ staged-proof-data}) \times \\ &\quad 'f \text{ protocol-channel}) \times \text{unit list}) \times \\ &\quad 'f \text{ protocol-channel}) \text{ option} \Rightarrow \text{bool} \end{aligned}$$

where

checked-actual-alpha-prefix-branch-subtree-verifier-partial-opening-witness-query-hit
 $out \longleftrightarrow$
checked-actual-alpha-prefix-branch-subtree-verifier-partial-opening-witness-hit
 $out \wedge$
checked-staged-security-with-actual-alpha-prefix-query-partial-opening-hit
 out

lemma *checked-actual-alpha-prefix-branch-subtree-verifier-partial-opening-witness-query-hit-imp-query-hit*:

assumes

checked-actual-alpha-prefix-branch-subtree-verifier-partial-opening-witness-query-hit
 out

shows *checked-staged-security-with-actual-alpha-prefix-query-partial-opening-hit*

out

using *assms*

unfolding

checked-actual-alpha-prefix-branch-subtree-verifier-partial-opening-witness-query-hit-def

by *simp*

lemma *checked-actual-alpha-prefix-branch-subtree-verifier-partial-opening-witness-query-hit-bound-from-query-hit*:

assumes *query-bound*:

wp-event

(*checked-staged-security-experiment-with-actual-alpha-prefix-data-state A*)

checked-staged-security-with-actual-alpha-prefix-query-partial-opening-hit

adversary-initial-state $\leq Q$

shows

wp-event

(*checked-staged-security-experiment-with-actual-alpha-prefix-data-state A*)

checked-actual-alpha-prefix-branch-subtree-verifier-partial-opening-witness-query-hit

adversary-initial-state $\leq Q$

by (rule order.trans[*OF - query-bound*])

(rule *wp-event-mono*,

rule

checked-actual-alpha-prefix-branch-subtree-verifier-partial-opening-witness-query-hit-imp-query-hit)

lemma *checked-actual-alpha-prefix-branch-subtree-verifier-partial-opening-witness-query-hit-bound-from-fixed-query-e*

assumes *wf*: *staged-budget-wellformed budgets*

and *controlled*: *staged-adversary-controlled budgets A*

and *cover*:

$\bigwedge \text{data attacker-state } i.$

```

i < rounds  $\implies$ 
query-header-supported-partial-opening-success-indices-at
  (verifier-state-from-adversary attacker-state
    (staged-proof-transcript data))
  (staged-trace-root data)
  (staged-trace-fri-roots data)
  (staged-trace-final data)
  (staged-alphas data)
  (staged-degree data)
  (staged-composition-fri-roots data)
  (staged-composition-final data)
i  $\subseteq$  B
and raw-bound: query-index-raw-preimage-bound
and subset: B  $\subseteq$  query-sample-space
and frac:
  nnreal (card B) / nnreal (card query-sample-space)  $\leq$ 
  query-error-bound
shows
  wp-event
  (checked-staged-security-experiment-with-actual-alpha-prefix-data-state A)
  checked-actual-alpha-prefix-branch-subtree-verifier-partial-opening-witness-query-hit
  adversary-initial-state  $\leq$ 
  nnreal
  (staged-attacker-query-budget budgets +
    staged-challenge-query-budget) * query-error-bound
proof –
have query-bound:
  wp-event
  (checked-staged-security-experiment-with-actual-alpha-prefix-data-state A)
  checked-staged-security-with-actual-alpha-prefix-query-partial-opening-hit
  adversary-initial-state  $\leq$ 
  nnreal
  (staged-attacker-query-budget budgets +
    staged-challenge-query-budget) * query-error-bound
by (rule
  checked-staged-security-with-actual-alpha-prefix-query-partial-opening-hit-bound-from-fixed-query-error-cover
  [OF wf controlled cover raw-bound subset frac])
show ?thesis
by (rule
  checked-actual-alpha-prefix-branch-subtree-verifier-partial-opening-witness-query-hit-bound-from-query-hit
  [OF query-bound])
qed

definition checked-actual-alpha-prefix-branch-subtree-verifier-partial-opening-witness-without-query-hit
:: (((((('f  $\times$  'f list  $\times$  'f list  $\times$  'f)  $\times$ 
'f protocol-channel)  $\times$  'f staged-proof-data)  $\times$ 
'f protocol-channel)  $\times$  unit list)  $\times$ 
'f protocol-channel) option  $\Rightarrow$  bool
where

```

checked-actual-alpha-prefix-branch-subtree-verifier-partial-opening-witness-without-query-hit
 $out \longleftrightarrow$
checked-actual-alpha-prefix-branch-subtree-verifier-partial-opening-witness-hit
 $out \wedge$
 $\neg$ *checked-staged-security-with-actual-alpha-prefix-query-partial-opening-hit*
 out

lemma *checked-actual-alpha-prefix-branch-subtree-verifier-partial-opening-witness-hit-imp-query-or-without-query:*

assumes

checked-actual-alpha-prefix-branch-subtree-verifier-partial-opening-witness-hit
 out

shows

checked-actual-alpha-prefix-branch-subtree-verifier-partial-opening-witness-query-hit
 $out \vee$
checked-actual-alpha-prefix-branch-subtree-verifier-partial-opening-witness-without-query-hit
 out

using *assms*

unfolding

checked-actual-alpha-prefix-branch-subtree-verifier-partial-opening-witness-query-hit-def
checked-actual-alpha-prefix-branch-subtree-verifier-partial-opening-witness-without-query-hit-def
by *blast*

lemma *checked-actual-alpha-prefix-branch-subtree-verifier-partial-opening-witness-hit-bound-from-query-and-without-*

assumes *query-bound:*

wp-event
 $($ *checked-staged-security-experiment-with-actual-alpha-prefix-data-state*
 $A)$
checked-actual-alpha-prefix-branch-subtree-verifier-partial-opening-witness-query-hit
 $adversary-initial-state \leq Q$

and *without-bound:*

wp-event
 $($ *checked-staged-security-experiment-with-actual-alpha-prefix-data-state*
 $A)$
checked-actual-alpha-prefix-branch-subtree-verifier-partial-opening-witness-without-query-hit
 $adversary-initial-state \leq W$

shows

wp-event
 $($ *checked-staged-security-experiment-with-actual-alpha-prefix-data-state* $A)$
checked-actual-alpha-prefix-branch-subtree-verifier-partial-opening-witness-hit
 $adversary-initial-state \leq Q + W$

proof $-$

let $?M =$

checked-staged-security-experiment-with-actual-alpha-prefix-data-state A

let $?Witness =$

checked-actual-alpha-prefix-branch-subtree-verifier-partial-opening-witness-hit

let $?Query =$

checked-actual-alpha-prefix-branch-subtree-verifier-partial-opening-witness-query-hit

let $?Without =$

checked-actual-alpha-prefix-branch-subtree-verifier-partial-opening-witness-without-query-hit

```

have wp-event ?M ?Witness adversary-initial-state ≤
  wp-event ?M (λout. ?Query out ∨ ?Without out)
  adversary-initial-state
by (rule wp-event-mono)
  (rule
    checked-actual-alpha-prefix-branch-subtree-verifier-partial-opening-witness-hit-imp-query-or-without-query)
also have ... ≤
  wp-event ?M ?Query adversary-initial-state +
  wp-event ?M ?Without adversary-initial-state
by (rule wp-event-union-bound)
also have ... ≤ Q + W
by (intro add-mono query-bound without-bound)
finally show ?thesis .
qed

```

lemma *checked-actual-alpha-prefix-branch-subtree-verifier-output-hit-bound-from-partial-opening-witness-hit:*
assumes *witness-bound:*

```

  wp-event
  (checked-staged-security-experiment-with-actual-alpha-prefix-data-state A)
  checked-actual-alpha-prefix-branch-subtree-verifier-partial-opening-witness-hit
  adversary-initial-state ≤ W

```

shows

```

  wp-event
  (checked-staged-security-experiment-with-actual-alpha-prefix-data-state A)
  checked-staged-security-with-actual-alpha-prefix-branch-tree-subtree-verifier-output-hit
  adversary-initial-state ≤ W

```

proof –

have

```

  wp-event
  (checked-staged-security-experiment-with-actual-alpha-prefix-data-state A)
  checked-staged-security-with-actual-alpha-prefix-branch-tree-subtree-verifier-output-hit
  adversary-initial-state ≤
  wp-event
  (checked-staged-security-experiment-with-actual-alpha-prefix-data-state A)
  checked-actual-alpha-prefix-branch-subtree-verifier-partial-opening-witness-hit
  adversary-initial-state
by (rule wp-event-mono-on-support)
  (rule
    checked-actual-alpha-prefix-branch-subtree-verifier-imp-partial-opening-witness-hit-on-support)

```

also have ... ≤ W

by (rule witness-bound)

finally show ?thesis .

qed

definition *checked-actual-alpha-prefix-branch-subtree-verifier-partial-opening-witness-without-all-queries-hit*

```

:: (((((( 'f × 'f list × 'f list × 'f) ×
  'f protocol-channel) × 'f staged-proof-data) ×
  'f protocol-channel) × unit list) ×
  'f protocol-channel) option ⇒ bool

```

where

checked-actual-alpha-prefix-branch-subtree-verifier-partial-opening-witness-without-all-queries-hit
 out $\longleftrightarrow$
 checked-actual-alpha-prefix-branch-subtree-verifier-partial-opening-witness-hit
 out $\wedge$
 $\neg$ *checked-actual-alpha-prefix-branch-subtree-verifier-partial-opening-witness-all-queries-hit*
 out

lemma *checked-actual-alpha-prefix-branch-subtree-verifier-partial-opening-witness-hit-imp-all-queries-or-without-all-q*

assumes

checked-actual-alpha-prefix-branch-subtree-verifier-partial-opening-witness-hit
 out

shows

checked-actual-alpha-prefix-branch-subtree-verifier-partial-opening-witness-all-queries-hit
 out $\vee$
 checked-actual-alpha-prefix-branch-subtree-verifier-partial-opening-witness-without-all-queries-hit
 out

using *assms*

unfolding

checked-actual-alpha-prefix-branch-subtree-verifier-partial-opening-witness-without-all-queries-hit-def
by *blast*

lemma *checked-actual-alpha-prefix-branch-subtree-verifier-partial-opening-witness-hit-bound-from-all-queries-and-wit*

assumes *all-queries-bound*:

wp-event
 (*checked-staged-security-experiment-with-actual-alpha-prefix-data-state*
 A)
 checked-actual-alpha-prefix-branch-subtree-verifier-partial-opening-witness-all-queries-hit
 adversary-initial-state $\leq Q$

and *without-bound*:

wp-event
 (*checked-staged-security-experiment-with-actual-alpha-prefix-data-state*
 A)
 checked-actual-alpha-prefix-branch-subtree-verifier-partial-opening-witness-without-all-queries-hit
 adversary-initial-state $\leq W$

shows

wp-event
 (*checked-staged-security-experiment-with-actual-alpha-prefix-data-state A*)
 checked-actual-alpha-prefix-branch-subtree-verifier-partial-opening-witness-hit
 adversary-initial-state $\leq Q + W$

proof –

let *?M* =

checked-staged-security-experiment-with-actual-alpha-prefix-data-state A

let *?Witness* =

checked-actual-alpha-prefix-branch-subtree-verifier-partial-opening-witness-hit

let *?All* =

checked-actual-alpha-prefix-branch-subtree-verifier-partial-opening-witness-all-queries-hit

let *?Without* =

checked-actual-alpha-prefix-branch-subtree-verifier-partial-opening-witness-without-all-queries-hit

have $wp\text{-event } ?M \text{ ?Witness adversary-initial-state} \leq$
 $wp\text{-event } ?M (\lambda out. \text{ ?All out } \vee \text{ ?Without out})$
 $adversary\text{-initial-state}$
by (rule $wp\text{-event-mono}$)
(rule
 $checked\text{-actual-alpha-prefix-branch-subtree-verifier-partial-opening-witness-hit-imp-all-queries-or-without-all-queries}$)
also have $\dots \leq$
 $wp\text{-event } ?M \text{ ?All adversary-initial-state} +$
 $wp\text{-event } ?M \text{ ?Without adversary-initial-state}$
by (rule $wp\text{-event-union-bound}$)
also have $\dots \leq Q + W$
by (intro $add\text{-mono all-queries-bound without-bound}$)
finally show $?thesis$.
qed

lemma $checked\text{-actual-alpha-prefix-branch-subtree-verifier-output-hit-bound-from-all-queries-and-without-all-queries}$:
assumes $all\text{-queries-bound}$:
 $wp\text{-event}$
 $(checked\text{-staged-security-experiment-with-actual-alpha-prefix-data-state}$
 $A)$
 $checked\text{-actual-alpha-prefix-branch-subtree-verifier-partial-opening-witness-all-queries-hit}$
 $adversary\text{-initial-state} \leq Q$
and $without\text{-bound}$:
 $wp\text{-event}$
 $(checked\text{-staged-security-experiment-with-actual-alpha-prefix-data-state}$
 $A)$
 $checked\text{-actual-alpha-prefix-branch-subtree-verifier-partial-opening-witness-without-all-queries-hit}$
 $adversary\text{-initial-state} \leq W$
shows
 $wp\text{-event}$
 $(checked\text{-staged-security-experiment-with-actual-alpha-prefix-data-state } A)$
 $checked\text{-staged-security-with-actual-alpha-prefix-branch-tree-subtree-verifier-output-hit}$
 $adversary\text{-initial-state} \leq Q + W$
proof –
have $witness\text{-bound}$:
 $wp\text{-event}$
 $(checked\text{-staged-security-experiment-with-actual-alpha-prefix-data-state } A)$
 $checked\text{-actual-alpha-prefix-branch-subtree-verifier-partial-opening-witness-hit}$
 $adversary\text{-initial-state} \leq Q + W$
by (rule
 $checked\text{-actual-alpha-prefix-branch-subtree-verifier-partial-opening-witness-hit-bound-from-all-queries-and-without-all-queries}$
 $[OF all\text{-queries-bound without-bound}])$
show $?thesis$
by (rule
 $checked\text{-actual-alpha-prefix-branch-subtree-verifier-output-hit-bound-from-partial-opening-witness-hit}$
 $[OF witness\text{-bound}])$
qed

lemma $checked\text{-actual-alpha-prefix-branch-subtree-verifier-partial-opening-witness-without-all-queries-hit-imp-fri-or-q}$

```

assumes support:
  out ∈
    set-dist
      (execute
        (checked-staged-security-experiment-with-actual-alpha-prefix-data-state
          A)
        adversary-initial-state)
and hit:
  checked-actual-alpha-prefix-branch-subtree-verifier-partial-opening-witness-without-all-queries-hit
    out
shows
  checked-staged-security-with-actual-alpha-prefix-verifier-event
    trace-fri-bad-with-partial-openings out ∨
  checked-staged-security-with-actual-alpha-prefix-verifier-event
    composition-fri-bad-with-partial-openings out ∨
  checked-staged-security-with-actual-alpha-prefix-verifier-event
    query-bad out
proof (cases out)
  case None
  then show ?thesis
    using hit
    unfolding
      checked-actual-alpha-prefix-branch-subtree-verifier-partial-opening-witness-without-all-queries-hit-def
      checked-actual-alpha-prefix-branch-subtree-verifier-partial-opening-witness-hit-def
    by simp
next
  case (Some result-pack)
  then obtain prefix prefix-state data attacker-state result final-state where
    out-eq:
      out = Some (((((prefix, prefix-state), data), attacker-state),
        result), final-state)
    by (cases result-pack, auto split: prod.splits)
  let ?s =
    verifier-state-from-adversary attacker-state
      (staged-proof-transcript data)
  have support-some:
    Some (((((prefix, prefix-state), data), attacker-state),
      result), final-state) ∈
      set-dist
        (execute
          (checked-staged-security-experiment-with-actual-alpha-prefix-data-state
            A)
          adversary-initial-state)
    using support unfolding out-eq .
  have witness-hit:
    checked-actual-alpha-prefix-branch-subtree-verifier-partial-opening-witness-hit
      out
    using hit
    unfolding

```

```

checked-actual-alpha-prefix-branch-subtree-verifier-partial-opening-witness-without-all-queries-hit-def
by simp
have not-all-hit:
  ¬
    checked-actual-alpha-prefix-branch-subtree-verifier-partial-opening-witness-all-queries-hit
    out
using hit
unfolding
checked-actual-alpha-prefix-branch-subtree-verifier-partial-opening-witness-without-all-queries-hit-def
by simp
from witness-hit obtain as dg composition-fri-roots final rest trace-table
  composition-table query-idxs trace-openings composition-openings where
bound:
  accepted-with-bound-tables ?s (Some (result, final-state))
  trace-table composition-table as query-idxs
and header:
  verifier-header-transcript ?s
  (staged-trace-root data) (staged-trace-fri-roots data)
  (staged-trace-final data) as dg composition-fri-roots final rest
and witness:
  (trace-openings, composition-openings) ∈
  query-header-supported-partial-opening-witnesses ?s
  (staged-trace-root data) (staged-trace-fri-roots data)
  (staged-trace-final data) as dg composition-fri-roots final
unfolding out-eq
checked-actual-alpha-prefix-branch-subtree-verifier-partial-opening-witness-hit-def
Let-def
by auto
have not-all-queries:
  ¬ all-queries-consistent trace-table composition-table as
proof
assume all-queries:
  all-queries-consistent trace-table composition-table as
have
  checked-actual-alpha-prefix-branch-subtree-verifier-partial-opening-witness-all-queries-hit
  out
unfolding out-eq
checked-actual-alpha-prefix-branch-subtree-verifier-partial-opening-witness-all-queries-hit-def
Let-def
apply simp
apply (rule exI[where x=as])
apply (rule exI[where x=dg])
apply (rule exI[where x=composition-fri-roots])
apply (rule exI[where x=final])
apply (rule exI[where x=rest])
apply (rule exI[where x=trace-table])
apply (rule exI[where x=composition-table])
apply (intro conjI)
  apply (rule exI[where x=query-idxs])

```

```

    apply (rule bound)
    apply (rule header)
    apply (rule exI[where x=trace-openings])
    apply (rule exI[where x=composition-openings])
    apply (rule witness)
    apply (rule all-queries)
  done
then show False
  using not-all-hit by contradiction
qed
show ?thesis
proof (cases trace-table-low-degree trace-table)
  case False
  obtain fr f-fri-roots f-final dg' composition-fri-roots' final'
    rest' trace-openings' composition-openings' where
    trace-partial:
    accepted-with-partial-trace-openings ?s
    (Some (result, final-state)) fr query-idxs trace-openings'
  by (rule accepted-with-bound-tables-partial-opening-witnesses-Some
    [OF bound]) blast
have trace-bad:
  trace-fri-bad-with-partial-openings ?s
  (Some (result, final-state))
  unfolding trace-fri-bad-with-partial-openings-def trace-fri-bad-def
  using bound False trace-partial by blast
then show ?thesis
  unfolding out-eq
  checked-staged-security-with-actual-alpha-prefix-verifier-event-def
  Let-def
  by simp
next
case trace-low: True
show ?thesis
proof (cases
  composition-table-low-degree maxDegree composition-table)
  case False
  obtain fr f-fri-roots f-final dg' composition-fri-roots' final'
    rest' trace-openings' composition-openings' where
    trace-partial:
    accepted-with-partial-trace-openings ?s
    (Some (result, final-state)) fr query-idxs trace-openings'
  by (rule accepted-with-bound-tables-partial-opening-witnesses-Some
    [OF bound]) blast
have comp-bad:
  composition-fri-bad-with-partial-openings ?s
  (Some (result, final-state))
  unfolding composition-fri-bad-with-partial-openings-def
  composition-fri-bad-def
  using bound trace-low False trace-partial by blast

```

```

then show ?thesis
  unfolding out-eq
    checked-staged-security-with-actual-alpha-prefix-verifier-event-def
    Let-def
  by simp
next
case composition-low: True
have query-bad:
  query-bad ?s (Some (result, final-state))
  unfolding query-bad-def
  using bound trace-low composition-low not-all-queries by blast
then show ?thesis
  unfolding out-eq
    checked-staged-security-with-actual-alpha-prefix-verifier-event-def
    Let-def
  by simp
qed
qed
qed

lemma checked-actual-alpha-prefix-branch-subtree-verifier-partial-opening-witness-without-all-queries-hit-bound-from
assumes trace-bound:
  wp-event
    (checked-staged-security-experiment-with-actual-alpha-prefix-data-state
      A)
    (checked-staged-security-with-actual-alpha-prefix-verifier-event
      trace-fri-bad-with-partial-openings)
    adversary-initial-state  $\leq T$ 
and composition-bound:
  wp-event
    (checked-staged-security-experiment-with-actual-alpha-prefix-data-state
      A)
    (checked-staged-security-with-actual-alpha-prefix-verifier-event
      composition-fri-bad-with-partial-openings)
    adversary-initial-state  $\leq C$ 
and query-bound:
  wp-event
    (checked-staged-security-experiment-with-actual-alpha-prefix-data-state
      A)
    (checked-staged-security-with-actual-alpha-prefix-verifier-event
      query-bad)
    adversary-initial-state  $\leq Q$ 
shows
  wp-event
    (checked-staged-security-experiment-with-actual-alpha-prefix-data-state A)
  checked-actual-alpha-prefix-branch-subtree-verifier-partial-opening-witness-without-all-queries-hit
  adversary-initial-state  $\leq T + C + Q$ 
proof –
  let ?M =

```

```

    checked-staged-security-experiment-with-actual-alpha-prefix-data-state A
  let ?Without =
    checked-actual-alpha-prefix-branch-subtree-verifier-partial-opening-witness-without-all-queries-hit
  let ?Trace =
    checked-staged-security-with-actual-alpha-prefix-verifier-event
    trace-fri-bad-with-partial-openings
  let ?Composition =
    checked-staged-security-with-actual-alpha-prefix-verifier-event
    composition-fri-bad-with-partial-openings
  let ?Query =
    checked-staged-security-with-actual-alpha-prefix-verifier-event query-bad
  have wp-event ?M ?Without adversary-initial-state ≤
    wp-event ?M (λout. ?Trace out ∨ (?Composition out ∨ ?Query out))
    adversary-initial-state
  by (rule wp-event-mono-on-support)
  (rule
    checked-actual-alpha-prefix-branch-subtree-verifier-partial-opening-witness-without-all-queries-hit-imp-fri-or-query)
  also have ... ≤
    wp-event ?M ?Trace adversary-initial-state +
    wp-event ?M (λout. ?Composition out ∨ ?Query out)
    adversary-initial-state
  by (rule wp-event-union-bound)
  also have ... ≤
    wp-event ?M ?Trace adversary-initial-state +
    (wp-event ?M ?Composition adversary-initial-state +
    wp-event ?M ?Query adversary-initial-state)
  by (intro add-mono order-refl wp-event-union-bound)
  also have ... ≤ T + (C + Q)
  by (intro add-mono trace-bound composition-bound query-bound)
  also have ... = T + C + Q
  by (simp add: add.assoc)
  finally show ?thesis .
qed

```

```

lemma checked-actual-alpha-prefix-branch-subtree-verifier-output-hit-bound-from-query-hit-partial-fri-and-query:
  assumes wf: staged-budget-wellformed budgets
  and controlled: staged-adversary-controlled budgets A
  and query-hit-bound:
    wp-event
      (checked-staged-security-experiment-with-actual-alpha-prefix-data-state
        A)
    checked-staged-security-with-actual-alpha-prefix-query-partial-opening-hit
    adversary-initial-state ≤ QH
  and trace-bound:
    wp-event
      (checked-staged-security-experiment-with-actual-alpha-prefix-data-state
        A)
      (checked-staged-security-with-actual-alpha-prefix-verifier-event
        trace-fri-bad-with-partial-openings)

```

$adversary-initial-state \leq T$
and *composition-bound*:
wp-event
 (checked-staged-security-experiment-with-actual-alpha-prefix-data-state
 A)
 (checked-staged-security-with-actual-alpha-prefix-verifier-event
 composition-fri-bad-with-partial-openings)
 $adversary-initial-state \leq C$
and *query-bound*:
wp-event
 (checked-staged-security-experiment-with-actual-alpha-prefix-data-state
 A)
 (checked-staged-security-with-actual-alpha-prefix-verifier-event
 query-bad)
 $adversary-initial-state \leq Q$
shows
wp-event
 (checked-staged-security-experiment-with-actual-alpha-prefix-data-state *A*)
 checked-staged-security-with-actual-alpha-prefix-branch-tree-subtree-verifier-output-hit
 $adversary-initial-state \leq QH + (T + C + Q)$
proof –
have *all-queries-bound*:
wp-event
 (checked-staged-security-experiment-with-actual-alpha-prefix-data-state *A*)
 checked-actual-alpha-prefix-branch-subtree-verifier-partial-opening-witness-all-queries-hit
 $adversary-initial-state \leq QH$
by (rule
 checked-actual-alpha-prefix-branch-subtree-verifier-partial-opening-witness-all-queries-hit-bound-from-query-hit
 [OF *wf controlled query-hit-bound*])
have *without-bound*:
wp-event
 (checked-staged-security-experiment-with-actual-alpha-prefix-data-state *A*)
 checked-actual-alpha-prefix-branch-subtree-verifier-partial-opening-witness-without-all-queries-hit
 $adversary-initial-state \leq T + C + Q$
by (rule
 checked-actual-alpha-prefix-branch-subtree-verifier-partial-opening-witness-without-all-queries-hit-bound-from-par
 [OF *trace-bound composition-bound query-bound*])
show *?thesis*
by (rule
 checked-actual-alpha-prefix-branch-subtree-verifier-output-hit-bound-from-all-queries-and-without-all-queries
 [OF *all-queries-bound without-bound*])
qed

lemma *checked-actual-alpha-prefix-branch-subtree-verifier-output-hit-bound-from-partial-opening-witness-query-and-u*
assumes *query-bound*:
wp-event
 (checked-staged-security-experiment-with-actual-alpha-prefix-data-state
 A)
 checked-actual-alpha-prefix-branch-subtree-verifier-partial-opening-witness-query-hit

$adversary-initial-state \leq Q$
and *without-bound*:
wp-event
 (checked-staged-security-experiment-with-actual-alpha-prefix-data-state A)
 checked-actual-alpha-prefix-branch-subtree-verifier-partial-opening-witness-without-query-hit
 $adversary-initial-state \leq W$
shows
wp-event
 (checked-staged-security-experiment-with-actual-alpha-prefix-data-state A)
 checked-staged-security-with-actual-alpha-prefix-branch-tree-subtree-verifier-output-hit
 $adversary-initial-state \leq Q + W$
proof –
have *witness-bound*:
wp-event
 (checked-staged-security-experiment-with-actual-alpha-prefix-data-state A)
 checked-actual-alpha-prefix-branch-subtree-verifier-partial-opening-witness-hit
 $adversary-initial-state \leq Q + W$
by (rule
 checked-actual-alpha-prefix-branch-subtree-verifier-partial-opening-witness-hit-bound-from-query-and-without-que
 [OF query-bound without-bound])
show *?thesis*
by (rule
 checked-actual-alpha-prefix-branch-subtree-verifier-output-hit-bound-from-partial-opening-witness-hit
 [OF witness-bound])
qed

lemma *checked-actual-alpha-prefix-branch-subtree-verifier-output-hit-bound-from-partial-opening-fixed-query-cover-and*
assumes *wf*: staged-budget-wellformed budgets
and *controlled*: staged-adversary-controlled budgets A
and *cover*:
 $\bigwedge data$ attacker-state i .
 $i < rounds \implies$
 query-header-supported-partial-opening-success-indices-at
 (verifier-state-from-adversary attacker-state
 (staged-proof-transcript data))
 (staged-trace-root data)
 (staged-trace-fri-roots data)
 (staged-trace-final data)
 (staged-alphas data)
 (staged-degree data)
 (staged-composition-fri-roots data)
 (staged-composition-final data)
 $i \subseteq B$
and *raw-bound*: query-index-raw-preimage-bound
and *subset*: $B \subseteq query-sample-space$
and *frac*:
 $nnreal (card B) / nnreal (card query-sample-space) \leq$
 query-error-bound

and *without-bound*:
 wp-event
 (*checked-staged-security-experiment-with-actual-alpha-prefix-data-state*
 A)
 checked-actual-alpha-prefix-branch-subtree-verifier-partial-opening-witness-without-query-hit
 adversary-initial-state $\leq$ *W*
shows
 wp-event
 (*checked-staged-security-experiment-with-actual-alpha-prefix-data-state A*)
 checked-staged-security-with-actual-alpha-prefix-branch-tree-subtree-verifier-output-hit
 adversary-initial-state $\leq$
 nnreal
 (*staged-attacker-query-budget budgets* +
 staged-challenge-query-budget) * *query-error-bound* + *W*
proof –
 have *query-bound*:
 wp-event
 (*checked-staged-security-experiment-with-actual-alpha-prefix-data-state A*)
 checked-actual-alpha-prefix-branch-subtree-verifier-partial-opening-witness-query-hit
 adversary-initial-state $\leq$
 nnreal
 (*staged-attacker-query-budget budgets* +
 staged-challenge-query-budget) * *query-error-bound*
 by (rule
 checked-actual-alpha-prefix-branch-subtree-verifier-partial-opening-witness-query-hit-bound-from-fixed-query-error
 [*OF wf controlled cover raw-bound subset frac*])
 show *?thesis*
 by (rule
 checked-actual-alpha-prefix-branch-subtree-verifier-output-hit-bound-from-partial-opening-witness-query-and-without-bound
 [*OF query-bound without-bound*])
qed

lemma *checked-staged-security-with-data-state-composition-bad-bound-from-branch-split-witness-and-transcript-budget*
assumes *wf*: *staged-budget-wellformed budgets*
and *controlled*: *staged-adversary-controlled budgets A*
and *root-pre-bound*:
 wp-event
 (*checked-staged-security-experiment-with-actual-alpha-prefix-data-state*
 A)
 checked-staged-security-with-actual-alpha-prefix-branch-transcript-root-preverifier-output-hit
 adversary-initial-state $\leq$ *RP*
and *subtree-pre-bound*:
 wp-event
 (*checked-staged-security-experiment-with-actual-alpha-prefix-data-state*
 A)
 checked-staged-security-with-actual-alpha-prefix-branch-tree-subtree-preverifier-output-hit
 adversary-initial-state $\leq$ *SP*
and *subtree-verifier-witness-bound*:
 wp-event

(checked-staged-security-experiment-with-actual-alpha-prefix-data-state
A)
checked-actual-alpha-prefix-branch-subtree-verifier-partial-opening-witness-hit
adversary-initial-state $\leq$ *WV*

shows

wp-event (checked-staged-security-experiment-with-data-state A)
(staged-security-with-data-state-verifier-event composition-bad)
adversary-initial-state $\leq$
composition-error-bound +
staged-phase-relation-error size
(staged-alpha-search-queries budgets 0) +
hash-collision-budget-value 0
(staged-alpha-search-queries budgets 0) +
(RP + staged-concrete-transcript-target-error-bound + SP + WV)

proof –

have *subtree-verifier-bound*:

wp-event
(checked-staged-security-experiment-with-actual-alpha-prefix-data-state
A)
checked-staged-security-with-actual-alpha-prefix-branch-tree-subtree-verifier-output-hit
adversary-initial-state $\leq$ *WV*

by (rule
checked-actual-alpha-prefix-branch-subtree-verifier-output-hit-bound-from-partial-opening-witness-hit
[OF subtree-verifier-witness-bound]))

show *?thesis*

by (rule
checked-staged-security-with-data-state-composition-bad-bound-from-branch-split-side-events-and-transcript-budg
[OF wf controlled root-pre-bound subtree-pre-bound
subtree-verifier-bound]))

qed

lemma *checked-staged-security-with-data-state-composition-bad-bound-from-branch-split-query-hit-partial-fri-and-que*

assumes *wf*: *staged-budget-wellformed budgets*

and *controlled*: *staged-adversary-controlled budgets A*

and *subtree-pre-bound*:

wp-event
(checked-staged-security-experiment-with-actual-alpha-prefix-data-state
A)
checked-staged-security-with-actual-alpha-prefix-branch-tree-subtree-preverifier-output-hit
adversary-initial-state $\leq$ *SP*

and *query-hit-bound*:

wp-event
(checked-staged-security-experiment-with-actual-alpha-prefix-data-state
A)
checked-staged-security-with-actual-alpha-prefix-query-partial-opening-hit
adversary-initial-state $\leq$ *QH*

and *trace-bound*:

wp-event
(checked-staged-security-experiment-with-actual-alpha-prefix-data-state

A)
 (checked-staged-security-with-actual-alpha-prefix-verifier-event
 trace-fri-bad-with-partial-openings)
 adversary-initial-state $\leq T$
and composition-bound:
 wp-event
 (checked-staged-security-experiment-with-actual-alpha-prefix-data-state
 A)
 (checked-staged-security-with-actual-alpha-prefix-verifier-event
 composition-fri-bad-with-partial-openings)
 adversary-initial-state $\leq C$
and query-bound:
 wp-event
 (checked-staged-security-experiment-with-actual-alpha-prefix-data-state
 A)
 (checked-staged-security-with-actual-alpha-prefix-verifier-event
 query-bad)
 adversary-initial-state $\leq Q$
shows
 wp-event (checked-staged-security-experiment-with-data-state A)
 (staged-security-with-data-state-verifier-event composition-bad)
 adversary-initial-state $\leq$
 composition-error-bound +
 staged-phase-relation-error size
 (staged-alpha-search-queries budgets 0) +
 hash-collision-budget-value 0
 (staged-alpha-search-queries budgets 0) +
 (staged-phase-relation-error size
 (staged-attacker-query-budget budgets + staged-challenge-query-budget) +
 staged-concrete-transcript-target-error-bound + SP +
 (QH + (T + C + Q)))
proof –
have subtree-verifier-bound:
 wp-event
 (checked-staged-security-experiment-with-actual-alpha-prefix-data-state
 A)
 checked-staged-security-with-actual-alpha-prefix-branch-tree-subtree-verifier-output-hit
 adversary-initial-state $\leq QH + (T + C + Q)$
by (rule
 checked-actual-alpha-prefix-branch-subtree-verifier-output-hit-bound-from-query-hit-partial-fri-and-query
 [OF wf controlled query-hit-bound trace-bound composition-bound
 query-bound])
show ?thesis
by (rule
 checked-staged-security-with-data-state-composition-bad-bound-from-branch-split-side-events-and-transcript-budg
 [OF wf controlled subtree-pre-bound subtree-verifier-bound])
qed
end

end

theory *Staged-Security-Experiment-Composition-Partial*

imports

Staged-Security-Experiment-Composition-Branch

Soundness-Reductions-Composition

begin

Partial-candidate composition reductions for the staged security experiment.

This layer avoids treating verifier-side Merkle subtree recomputation as a rare event. Instead, it lifts the verifier-local reduction from composition alpha badness to a partial-header candidate event plus hash collision.

context *soundness*

begin

definition *checked-staged-security-with-actual-alpha-prefix-partial-header-bad-set-hit*

$$\begin{aligned} &:: ('f \text{ list} \Rightarrow 'f \text{ list set}) \Rightarrow \\ &\quad ((((((('f \times 'f \text{ list} \times 'f \text{ list} \times 'f) \times \\ &\quad \quad 'f \text{ protocol-channel}) \times 'f \text{ staged-proof-data}) \times \\ &\quad \quad 'f \text{ protocol-channel}) \times \text{unit list}) \times \\ &\quad \quad 'f \text{ protocol-channel}) \text{ option} \Rightarrow \text{bool} \end{aligned}$$

where

checked-staged-security-with-actual-alpha-prefix-partial-header-bad-set-hit

bad-sets out $\longleftrightarrow$

(case out of

None $\Rightarrow$ *False*

| *Some* (*x*, *final-state*) $\Rightarrow$

$\quad$ (let *packed* = *fst* (*fst* *x*);

$\quad$ *data* = *snd* *packed*;

$\quad$ *attacker-state* = *snd* (*fst* *x*);

$\quad$ *result* = *snd* *x*;

$\quad$ *s* =

$\quad$ *verifier-state-from-adversary attacker-state*

$\quad$ (*staged-proof-transcript data*)

$\quad$ in *alpha-header-list-set-hit s*

$\quad$ (*alpha-header-supported-partial-union-bad-sets s bad-sets*)

$\quad$ (*Some* (*result*, *final-state*))))

definition *checked-staged-security-with-actual-alpha-prefix-partial-header-prequery-hit*

$$\begin{aligned} &:: ('f \text{ list} \Rightarrow 'f \text{ list set}) \Rightarrow \\ &\quad ((((((('f \times 'f \text{ list} \times 'f \text{ list} \times 'f) \times \\ &\quad \quad 'f \text{ protocol-channel}) \times 'f \text{ staged-proof-data}) \times \\ &\quad \quad 'f \text{ protocol-channel}) \times \text{unit list}) \times \\ &\quad \quad 'f \text{ protocol-channel}) \text{ option} \Rightarrow \text{bool} \end{aligned}$$

where

checked-staged-security-with-actual-alpha-prefix-partial-header-prequery-hit

bad-sets out $\longleftrightarrow$

```

(case out of
  None  $\Rightarrow$  False
| Some (x, final-state)  $\Rightarrow$ 
  (let packed = fst (fst x);
    prefix-state = snd (fst packed);
    data = snd packed;
    attacker-state = snd (fst x);
    result = snd x;
    s =
      verifier-state-from-adversary attacker-state
      (staged-proof-transcript data)
  in alpha-header-list-set-hit s
    (alpha-header-supported-partial-union-bad-sets s bad-sets)
    (Some (result, final-state))  $\wedge$ 
    alpha-vector-prequeried-in-state prefix-state
    (staged-alphas data) (length spec)))

```

definition *checked-staged-security-with-actual-alpha-prefix-partial-header-fresh-hit*

```

:: ('f list  $\Rightarrow$  'f list set)  $\Rightarrow$ 
  ((((((('f  $\times$  'f list  $\times$  'f list  $\times$  'f)  $\times$ 
    'f protocol-channel)  $\times$  'f staged-proof-data)  $\times$ 
    'f protocol-channel)  $\times$  unit list)  $\times$ 
    'f protocol-channel) option  $\Rightarrow$  bool

```

where

```

checked-staged-security-with-actual-alpha-prefix-partial-header-fresh-hit
  bad-sets out  $\longleftrightarrow$ 
checked-staged-security-with-actual-alpha-prefix-partial-header-bad-set-hit
  bad-sets out  $\wedge$ 
(case out of
  None  $\Rightarrow$  False
| Some (x, -)  $\Rightarrow$ 
  (let packed = fst (fst x);
    prefix-state = snd (fst packed);
    data = snd packed
  in  $\neg$  alpha-vector-prequeried-in-state prefix-state
    (staged-alphas data) (length spec)))

```

definition *checked-staged-security-with-actual-alpha-prefix-partial-header-candidate-drift*

```

:: ('f list  $\Rightarrow$  'f list set)  $\Rightarrow$ 
  ((((((('f  $\times$  'f list  $\times$  'f list  $\times$  'f)  $\times$ 
    'f protocol-channel)  $\times$  'f staged-proof-data)  $\times$ 
    'f protocol-channel)  $\times$  unit list)  $\times$ 
    'f protocol-channel) option  $\Rightarrow$  bool

```

where

```

checked-staged-security-with-actual-alpha-prefix-partial-header-candidate-drift
  bad-sets out  $\longleftrightarrow$ 
checked-staged-security-with-actual-alpha-prefix-partial-header-fresh-hit
  bad-sets out  $\wedge$ 
 $\neg$  checked-staged-security-with-actual-alpha-prefix-bad-set-hit

```

bad-sets out

definition *checked-staged-security-with-actual-alpha-prefix-composition-candidate-drift*

$:: ((((((f \times 'f \text{ list} \times 'f \text{ list} \times 'f) \times$
 $\quad 'f \text{ protocol-channel}) \times 'f \text{ staged-proof-data}) \times$
 $\quad 'f \text{ protocol-channel}) \times \text{unit list}) \times$
 $\quad 'f \text{ protocol-channel}) \text{ option} \Rightarrow \text{bool}$

where

checked-staged-security-with-actual-alpha-prefix-composition-candidate-drift
 $\text{out} \longleftrightarrow$
checked-staged-security-with-actual-alpha-prefix-verifier-event
composition-randomization-bad out $\wedge$
checked-staged-security-with-actual-alpha-prefix-partial-header-candidate-drift
composition-trace-bad-alpha-space out

definition *staged-alpha-prefix-partial-header-prequery-hit*

$:: 'f \text{ staged-adversary} \Rightarrow ('f \text{ list} \Rightarrow 'f \text{ list set}) \Rightarrow$
 $((('f \times 'f \text{ list} \times 'f \text{ list} \times 'f) \times$
 $\quad 'f \text{ protocol-channel}) \text{ option} \Rightarrow \text{bool}$

where

staged-alpha-prefix-partial-header-prequery-hit A bad-sets out
 $\longleftrightarrow$
 $(\text{case out of}$
 $\quad \text{None} \Rightarrow \text{False}$
 $\mid \text{Some (prefix, prefix-state)} \Rightarrow$
 $\quad (\exists \text{ data attacker-state result final-state.}$
 $\quad \quad \text{Some } ((((\text{prefix, prefix-state}), \text{data}), \text{attacker-state}),$
 $\quad \quad \text{result}), \text{final-state}) \in$
 $\quad \text{set-dist}$
 $\quad (\text{execute}$
 $\quad \quad (\text{checked-staged-security-experiment-with-actual-alpha-prefix-data-state}$
 $\quad \quad \text{A})$
 $\quad \text{adversary-initial-state}) \wedge$
 $\quad \text{checked-staged-security-with-actual-alpha-prefix-partial-header-prequery-hit}$
 $\quad \text{bad-sets}$
 $\quad (\text{Some } (((((\text{prefix, prefix-state}), \text{data}), \text{attacker-state}),$
 $\quad \quad \text{result}), \text{final-state}))))$

definition *staged-alpha-prefix-partial-header-prequery-relation*

$:: 'f \text{ staged-adversary} \Rightarrow 'f \text{ protocol-channel} \Rightarrow$
 $('f \text{ list} \Rightarrow 'f \text{ list set}) \Rightarrow$
 $'f \text{ protocol-hash-input} \Rightarrow 'f \Rightarrow \text{bool}$

where

staged-alpha-prefix-partial-header-prequery-relation A s bad-sets x y
 $\longleftrightarrow$
 $(\exists \text{ prefix prefix-state data attacker-state result final-state } i.$
 $\quad \text{Some } (((((\text{prefix, prefix-state}), \text{data}), \text{attacker-state}), \text{result}),$
 $\quad \quad \text{final-state}) \in$
 $\quad \text{set-dist}$

```

      (execute
        (checked-staged-security-experiment-with-actual-alpha-prefix-data-state
          A) s) ∧
checked-staged-security-with-actual-alpha-prefix-partial-header-prequery-hit
  bad-sets
  (Some (((((prefix, prefix-state), data), attacker-state), result),
    final-state)) ∧
  i < length spec ∧
  i < length (staged-alphas data) ∧
  x = alpha-challenge-key-at prefix-state i (staged-alphas data) ∧
  y = staged-alphas data ! i)

```

lemma *staged-alpha-prefix-partial-header-prequery-relation-fiber-bound-size:*

```

card
  {y. staged-alpha-prefix-partial-header-prequery-relation A s bad-sets x y}
≤ size

```

proof –

have *subset:*

```

  {y. staged-alpha-prefix-partial-header-prequery-relation A s bad-sets x y}
  ⊆ (UNIV :: 'f set)

```

by *simp*

have *finite-field: finite (UNIV :: 'f set)*

by *simp*

have *card*

```

  {y. staged-alpha-prefix-partial-header-prequery-relation A s bad-sets x y}
  ≤ card (UNIV :: 'f set)

```

by (rule *card-mono[OF finite-field subset]*)

also have ... = *size*

using *size-card* **by** *simp*

finally show *?thesis* .

qed

lemma *staged-alpha-prefix-partial-header-prequery-hit-imp-relation-hit:*

assumes *empty: HashMap s = fmempty*

and *support:*

```

  Some (prefix, prefix-state) ∈
  set-dist (execute (staged-alpha-prefix-program A) s)

```

and *hit:*

```

  staged-alpha-prefix-partial-header-prequery-hit A bad-sets
  (Some (prefix, prefix-state))

```

and *init: s = adversary-initial-state*

shows

```

hash-relation-hit-event
  (staged-alpha-prefix-partial-header-prequery-relation A s bad-sets)
  s (Some (prefix, prefix-state))

```

proof –

from *hit* **obtain** *data attacker-state result final-state* **where** *full-support:*

```

  Some (((((prefix, prefix-state), data), attacker-state), result),
  final-state) ∈

```

```

    set-dist
    (execute
      (checked-staged-security-experiment-with-actual-alpha-prefix-data-state
        A)
      adversary-initial-state)
  and prehit:
    checked-staged-security-with-actual-alpha-prefix-partial-header-prequery-hit
    bad-sets
    (Some (((prefix, prefix-state), data), attacker-state), result),
    final-state))
  unfolding staged-alpha-prefix-partial-header-prequery-hit-def by auto
from prehit obtain i where
  i-bound: i < length spec
  and i-len: i < length (staged-alphas data)
  and lookup:
    fmlookup (HashMap prefix-state)
      (alpha-challenge-key-at prefix-state i (staged-alphas data)) =
      Some (staged-alphas data ! i)
  unfolding
    checked-staged-security-with-actual-alpha-prefix-partial-header-prequery-hit-def
    alpha-vector-prequeried-in-state-def
  by (auto simp: Let-def)
have initial-none:
  fmlookup (HashMap s)
    (alpha-challenge-key-at prefix-state i (staged-alphas data)) = None
  using empty by simp
have relation:
  staged-alpha-prefix-partial-header-prequery-relation A s bad-sets
    (alpha-challenge-key-at prefix-state i (staged-alphas data))
    (staged-alphas data ! i)
  unfolding staged-alpha-prefix-partial-header-prequery-relation-def
  using full-support prehit i-bound i-len init by blast
have
  hash-relation-hit
    (staged-alpha-prefix-partial-header-prequery-relation A s bad-sets)
    s prefix-state
  unfolding hash-relation-hit-def
  by (intro exI[of - alpha-challenge-key-at prefix-state i
    (staged-alphas data)]
    exI[of - staged-alphas data ! i] conjI)
    (use initial-none lookup relation in simp-all)
then show ?thesis
  unfolding hash-relation-hit-event-def by simp
qed

lemma staged-alpha-prefix-partial-header-prequery-hit-bound-by-relation-hit:
  assumes init: s = adversary-initial-state
  shows
    wp-event (staged-alpha-prefix-program A)

```



```

      (staged-alpha-prefix-partial-header-prequery-hit A bad-sets) s ≤
wp-event (staged-alpha-prefix-program A)
      (hash-relation-hit-event
        (staged-alpha-prefix-partial-header-prequery-relation A s bad-sets) s)
s
proof (rule wp-event-mono-on-support)
  fix out
  assume support:
    out ∈ set-dist (execute (staged-alpha-prefix-program A) s)
  and hit:
    staged-alpha-prefix-partial-header-prequery-hit A bad-sets out
  show
    hash-relation-hit-event
      (staged-alpha-prefix-partial-header-prequery-relation A s bad-sets) s
    out
  proof (cases out)
    case None
    then show ?thesis
      using hit unfolding staged-alpha-prefix-partial-header-prequery-hit-def
      by simp
  next
    case (Some packed)
    then obtain prefix prefix-state where out-eq:
      out = Some (prefix, prefix-state)
    by (cases packed) simp
  show ?thesis
    unfolding out-eq
    by (rule staged-alpha-prefix-partial-header-prequery-hit-imp-relation-hit
      [OF - - - init])
    (use support hit out-eq init in
      ⟨simp-all add: adversary-initial-state-def⟩)
  qed
qed

lemma staged-alpha-prefix-partial-header-prequery-hit-bound:
  assumes wf: staged-budget-wellformed budgets
  and controlled: staged-adversary-controlled budgets A
  shows
    wp-event (staged-alpha-prefix-program A)
      (staged-alpha-prefix-partial-header-prequery-hit A bad-sets)
      adversary-initial-state ≤
      staged-phase-relation-error size
      (staged-alpha-search-queries budgets 0)
proof -
  have prehit-le-relation:
    wp-event (staged-alpha-prefix-program A)
      (staged-alpha-prefix-partial-header-prequery-hit A bad-sets)
      adversary-initial-state ≤
      wp-event (staged-alpha-prefix-program A)

```

```

    (hash-relation-hit-event
      (staged-alpha-prefix-partial-header-prequery-relation A
        adversary-initial-state bad-sets) adversary-initial-state)
    adversary-initial-state
  by (rule staged-alpha-prefix-partial-header-prequery-hit-bound-by-relation-hit)
    simp
also have ... ≤
  staged-phase-relation-error size
  (staged-alpha-search-queries budgets 0)
proof -
  have program:
    hash-relation-program
    (staged-alpha-prefix-partial-header-prequery-relation A
      adversary-initial-state bad-sets)
    size (staged-alpha-search-queries budgets 0)
    (staged-alpha-prefix-program A)
  by (rule hash-relation-program-staged-alpha-prefix-program
    [OF wf controlled])
    (rule
      staged-alpha-prefix-partial-header-prequery-relation-fiber-bound-size)
  show ?thesis
    using program
    unfolding hash-relation-program-def hash-relation-budget-def
      staged-phase-relation-error-def
    by blast
qed
finally show ?thesis .
qed

```

lemma *checked-staged-security-experiment-with-actual-alpha-prefix-data-stateI:*

```

assumes prefix:
  Some (prefix, prefix-state) ∈
    set-dist
      (execute (staged-alpha-prefix-program A) adversary-initial-state)
and after:
  Some (data, attacker-state) ∈
    set-dist
      (execute (checked-staged-after-alpha-prefix-program A prefix)
        prefix-state)
and verifier:
  Some (result, final-state) ∈
    set-dist
      (execute verify-monad
        (verifier-state-from-adversary attacker-state
          (staged-proof-transcript data)))
shows
  Some (((((prefix, prefix-state), data), attacker-state), result),
    final-state) ∈
    set-dist

```

```

      (execute
        (checked-staged-security-experiment-with-actual-alpha-prefix-data-state
          A)
        adversary-initial-state)
proof –
  have transcript:
    Some (((prefix, prefix-state), data), attacker-state) ∈
      set-dist
        (execute (checked-staged-transcript-with-alpha-prefix-program A)
          adversary-initial-state)
  unfolding checked-staged-transcript-with-alpha-prefix-program-def
  apply (rule-tac x=prefix and t=prefix-state in set-dist-bindI)
  apply (rule prefix)
  apply (rule-tac x=prefix-state and t=prefix-state in set-dist-bindI)
  apply simp
  apply (rule-tac x=data and t=attacker-state in set-dist-bindI)
  apply (rule after)
  apply simp
  done
show ?thesis
  unfolding
    checked-staged-security-experiment-with-actual-alpha-prefix-data-state-def
  apply (rule-tac x=((prefix, prefix-state), data) and
    t=attacker-state in set-dist-bindI)
  apply (rule transcript)
  apply (rule-tac x=attacker-state and t=attacker-state in set-dist-bindI)
  apply simp
  apply (rule-tac x=() and
    t=verifier-state-from-adversary attacker-state
    (staged-proof-transcript data) in set-dist-bindI)
  apply simp
  apply (rule-tac x=result and t=final-state in set-dist-bindI)
  apply (rule verifier)
  apply simp
  done
qed

lemma checked-staged-security-with-actual-alpha-prefix-partial-header-prequery-hit-bound-from-prefix:
shows
  wp-event
    (checked-staged-security-experiment-with-actual-alpha-prefix-data-state
      A)
    (checked-staged-security-with-actual-alpha-prefix-partial-header-prequery-hit
      bad-sets)
  adversary-initial-state ≤
  wp-event (staged-alpha-prefix-program A)
    (staged-alpha-prefix-partial-header-prequery-hit A bad-sets)
  adversary-initial-state
unfolding checked-staged-security-experiment-with-actual-alpha-prefix-data-state-def

```

```

checked-staged-transcript-with-alpha-prefix-program-def
sm-bind-assoc
apply (rule wp-event-bind-bound-by-head-event
  [where  $P = \text{staged-alpha-prefix-partial-header-prequery-hit } A \text{ bad-sets}$ ])
apply simp
apply (simp add:
  checked-staged-security-with-actual-alpha-prefix-partial-header-prequery-hit-def
  staged-alpha-prefix-partial-header-prequery-hit-def)
apply (auto simp:
  checked-staged-security-with-actual-alpha-prefix-partial-header-prequery-hit-def
  staged-alpha-prefix-partial-header-prequery-hit-def Let-def
  elim!: set-dist-bindE
  intro!: set-dist-bindI
  checked-staged-security-experiment-with-actual-alpha-prefix-data-stateI
  split: option.splits prod.splits)
apply (rule-tac  $x=bi$  in  $exI$ )
apply (rule-tac  $x=bj$  in  $exI$ )
apply (rule-tac  $x=bk$  in  $exI$ )
apply (rule-tac  $x=x2a$  in  $exI$ )
apply (intro conjI)
apply (rule checked-staged-security-experiment-with-actual-alpha-prefix-data-stateI)
apply assumption
apply assumption
apply assumption
apply assumption
apply assumption
done

```

lemma *checked-staged-security-with-actual-alpha-prefix-partial-header-prequery-hit-bound:*
assumes wf : *staged-budget-wellformed budgets*
and *controlled*: *staged-adversary-controlled budgets A*
shows

```

wp-event
  (checked-staged-security-experiment-with-actual-alpha-prefix-data-state
    A)
  (checked-staged-security-with-actual-alpha-prefix-partial-header-prequery-hit
    bad-sets)
  adversary-initial-state  $\leq$ 
  staged-phase-relation-error size
  (staged-alpha-search-queries budgets 0)

```

proof —

have

```

wp-event
  (checked-staged-security-experiment-with-actual-alpha-prefix-data-state A)
  (checked-staged-security-with-actual-alpha-prefix-partial-header-prequery-hit
    bad-sets)
  adversary-initial-state  $\leq$ 
wp-event (staged-alpha-prefix-program A)
  (staged-alpha-prefix-partial-header-prequery-hit A bad-sets)

```

```

    adversary-initial-state
  by (rule
    checked-staged-security-with-actual-alpha-prefix-partial-header-prequery-hit-bound-from-prefix)
  also have ... ≤
    staged-phase-relation-error size
    (staged-alpha-search-queries budgets 0)
  by (rule staged-alpha-prefix-partial-header-prequery-hit-bound
    [OF wf controlled])
  finally show ?thesis .
qed

```

Vector-valued, prefix-state-dependent accounting for future alpha challenge keys. The bad vector set may be selected from the concrete verifier/header state reached after the alpha prefix; the bound is derived from the existing relation-budget interface for the prefix program, not from a freshness assumption.

lemma *checked-staged-security-with-actual-alpha-prefix-vector-alpha-prequery-accounting:*

```

  assumes wf: staged-budget-wellformed budgets
    and controlled: staged-adversary-controlled budgets A
  shows

```

```

  wp-event
    (checked-staged-security-experiment-with-actual-alpha-prefix-data-state
      A)
    (checked-staged-security-with-actual-alpha-prefix-partial-header-prequery-hit
      bad-sets)
  adversary-initial-state ≤
    staged-phase-relation-error size
    (staged-alpha-search-queries budgets 0)
  by (rule
    checked-staged-security-with-actual-alpha-prefix-partial-header-prequery-hit-bound
    [OF wf controlled])

```

lemma *checked-staged-security-with-actual-alpha-prefix-partial-header-bad-set-hit-imp-fresh-or-prequery:*

```

  assumes
    checked-staged-security-with-actual-alpha-prefix-partial-header-bad-set-hit
    bad-sets out
  shows
    checked-staged-security-with-actual-alpha-prefix-partial-header-fresh-hit
    bad-sets out ∨
    checked-staged-security-with-actual-alpha-prefix-partial-header-prequery-hit
    bad-sets out
  using assms
  unfolding
    checked-staged-security-with-actual-alpha-prefix-partial-header-bad-set-hit-def
    checked-staged-security-with-actual-alpha-prefix-partial-header-fresh-hit-def
    checked-staged-security-with-actual-alpha-prefix-partial-header-prequery-hit-def
  by (auto simp: Let-def split: option.splits prod.splits)

```

lemma *checked-staged-security-with-actual-alpha-prefix-partial-header-bad-set-hit-bound-from-fresh-and-prequery:*
 assumes *fresh-bound:*

wp-event
 (checked-staged-security-experiment-with-actual-alpha-prefix-data-state
 A)
 (checked-staged-security-with-actual-alpha-prefix-partial-header-fresh-hit
 bad-sets)
adversary-initial-state $\leq F$
and *prequery-bound*:
wp-event
 (checked-staged-security-experiment-with-actual-alpha-prefix-data-state
 A)
 (checked-staged-security-with-actual-alpha-prefix-partial-header-prequery-hit
 bad-sets)
adversary-initial-state $\leq P$
shows
wp-event
 (checked-staged-security-experiment-with-actual-alpha-prefix-data-state
 A)
 (checked-staged-security-with-actual-alpha-prefix-partial-header-bad-set-hit
 bad-sets)
adversary-initial-state $\leq F + P$
proof –
let *?M* =
 checked-staged-security-experiment-with-actual-alpha-prefix-data-state *A*
let *?Bad* =
 checked-staged-security-with-actual-alpha-prefix-partial-header-bad-set-hit
 bad-sets
let *?Fresh* =
 checked-staged-security-with-actual-alpha-prefix-partial-header-fresh-hit
 bad-sets
let *?Pre* =
 checked-staged-security-with-actual-alpha-prefix-partial-header-prequery-hit
 bad-sets
have *wp-event ?M ?Bad adversary-initial-state* $\leq$
 wp-event ?M ($\lambda out. ?Fresh out \vee ?Pre out$) *adversary-initial-state*
by (rule *wp-event-mono*)
 (rule
 checked-staged-security-with-actual-alpha-prefix-partial-header-bad-set-hit-imp-fresh-or-prequery)
also have ... $\leq$
 wp-event ?M ?Fresh adversary-initial-state +
 wp-event ?M ?Pre adversary-initial-state
by (rule *wp-event-union-bound*)
also have ... $\leq F + P$
by (intro *add-mono fresh-bound prequery-bound*)
finally show *?thesis* .
qed

lemma *checked-staged-security-with-actual-alpha-prefix-partial-header-bad-set-hit-bound-from-fresh-and-budgets*:
assumes *wf*: *staged-budget-wellformed budgets*
and *controlled*: *staged-adversary-controlled budgets A*

and *fresh-bound*:
wp-event
 (*checked-staged-security-experiment-with-actual-alpha-prefix-data-state*
A)
 (*checked-staged-security-with-actual-alpha-prefix-partial-header-fresh-hit*
bad-sets)
adversary-initial-state $\leq F$
shows
wp-event
 (*checked-staged-security-experiment-with-actual-alpha-prefix-data-state*
A)
 (*checked-staged-security-with-actual-alpha-prefix-partial-header-bad-set-hit*
bad-sets)
adversary-initial-state $\leq$
F + staged-phase-relation-error size
 (*staged-alpha-search-queries budgets 0*)
proof –
have *prequery-bound*:
wp-event
 (*checked-staged-security-experiment-with-actual-alpha-prefix-data-state A*)
 (*checked-staged-security-with-actual-alpha-prefix-partial-header-prequery-hit*
bad-sets)
adversary-initial-state $\leq$
staged-phase-relation-error size
 (*staged-alpha-search-queries budgets 0*)
by (rule
checked-staged-security-with-actual-alpha-prefix-vector-alpha-prequery-accounting
 [OF wf controlled])
show *?thesis*
by (rule
checked-staged-security-with-actual-alpha-prefix-partial-header-bad-set-hit-bound-from-fresh-and-prequery
 [OF fresh-bound prequery-bound])
qed

lemma *checked-staged-security-with-actual-alpha-prefix-partial-header-fresh-hit-imp-prefix-or-drift*:
assumes
checked-staged-security-with-actual-alpha-prefix-partial-header-fresh-hit
bad-sets out
shows
checked-staged-security-with-actual-alpha-prefix-bad-set-hit bad-sets
out $\vee$
checked-staged-security-with-actual-alpha-prefix-partial-header-candidate-drift
bad-sets out
using *assms*
unfolding
checked-staged-security-with-actual-alpha-prefix-partial-header-candidate-drift-def
by *blast*

lemma *checked-staged-security-with-actual-alpha-prefix-partial-header-fresh-hit-bound-from-prefix-and-drift*:

assumes *prefix-bound*:
 wp-event
 (*checked-staged-security-experiment-with-actual-alpha-prefix-data-state*
 A)
 (*checked-staged-security-with-actual-alpha-prefix-bad-set-hit*
 bad-sets)
 adversary-initial-state $\leq P$
and *drift-bound*:
 wp-event
 (*checked-staged-security-experiment-with-actual-alpha-prefix-data-state*
 A)
 (*checked-staged-security-with-actual-alpha-prefix-partial-header-candidate-drift*
 bad-sets)
 adversary-initial-state $\leq D$
shows
 wp-event
 (*checked-staged-security-experiment-with-actual-alpha-prefix-data-state*
 A)
 (*checked-staged-security-with-actual-alpha-prefix-partial-header-fresh-hit*
 bad-sets)
 adversary-initial-state $\leq P + D$
proof –
 let *?M* =
 checked-staged-security-experiment-with-actual-alpha-prefix-data-state A
 let *?Fresh* =
 checked-staged-security-with-actual-alpha-prefix-partial-header-fresh-hit
 bad-sets
 let *?Prefix* =
 checked-staged-security-with-actual-alpha-prefix-bad-set-hit bad-sets
 let *?Drift* =
 checked-staged-security-with-actual-alpha-prefix-partial-header-candidate-drift
 bad-sets
 have *wp-event ?M ?Fresh adversary-initial-state* $\leq$
 wp-event ?M (λ out. ?Prefix out $\vee$?Drift out)
 adversary-initial-state
 by (*rule wp-event-mono*)
 (*rule*
 checked-staged-security-with-actual-alpha-prefix-partial-header-fresh-hit-imp-prefix-or-drift)
 also have ... $\leq$
 wp-event ?M ?Prefix adversary-initial-state +
 wp-event ?M ?Drift adversary-initial-state
 by (*rule wp-event-union-bound*)
 also have ... $\leq P + D$
 by (*intro add-mono prefix-bound drift-bound*)
 finally show *?thesis* .
qed

lemma *checked-staged-security-with-actual-alpha-prefix-composition-candidate-drift-imp-branch-tree-on-support*:
 assumes *wf*: *staged-budget-wellformed budgets*

and *controlled*: staged-adversary-controlled budgets A
and *support*:
 $out \in$
 $set-dist$
 $(execute$
 $(checked-staged-security-experiment-with-actual-alpha-prefix-data-state$
 $A)$
 $adversary-initial-state)$
and *drift*:
 $checked-staged-security-with-actual-alpha-prefix-composition-candidate-drift$
 out
shows
 $checked-staged-security-with-actual-alpha-prefix-branch-tree-output-hit$
 out
proof —
have *random-bad*:
 $checked-staged-security-with-actual-alpha-prefix-verifier-event$
 $composition-randomization-bad out$
using *drift*
unfolding
 $checked-staged-security-with-actual-alpha-prefix-composition-candidate-drift-def$
by *simp*
have *not-prefix*:
 $\neg checked-staged-security-with-actual-alpha-prefix-bad-set-hit$
 $composition-trace-bad-alpha-space out$
using *drift*
unfolding
 $checked-staged-security-with-actual-alpha-prefix-composition-candidate-drift-def$
 $checked-staged-security-with-actual-alpha-prefix-partial-header-candidate-drift-def$
by *simp*
have *split*:
 $checked-staged-security-with-actual-alpha-prefix-bad-set-hit$
 $composition-trace-bad-alpha-space out \vee$
 $checked-staged-security-with-actual-alpha-prefix-branch-tree-output-hit$
 out
by (*rule*
 $checked-staged-security-with-actual-alpha-prefix-composition-randomization-bad-imp-prefix-or-branch-tree-on-sup$
 $[OF wf controlled support random-bad])$
then show *?thesis*
using *not-prefix* **by** *blast*
qed

lemma *checked-staged-security-with-actual-alpha-prefix-composition-candidate-drift-bound-from-branch-tree*:
assumes *wf*: staged-budget-wellformed budgets
and *controlled*: staged-adversary-controlled budgets A
and *branch-tree-bound*:
 $wp-event$
 $(checked-staged-security-experiment-with-actual-alpha-prefix-data-state$
 $A)$

checked-staged-security-with-actual-alpha-prefix-branch-tree-output-hit
adversary-initial-state $\leq T$

shows

wp-event
 (*checked-staged-security-experiment-with-actual-alpha-prefix-data-state*
A)
checked-staged-security-with-actual-alpha-prefix-composition-candidate-drift
adversary-initial-state $\leq T$

proof –

let $?M =$

checked-staged-security-experiment-with-actual-alpha-prefix-data-state A
have *wp-event* $?M$
checked-staged-security-with-actual-alpha-prefix-composition-candidate-drift
adversary-initial-state $\leq$
wp-event $?M$
checked-staged-security-with-actual-alpha-prefix-branch-tree-output-hit
adversary-initial-state
by (rule *wp-event-mono-on-support*)
 (rule
checked-staged-security-with-actual-alpha-prefix-composition-candidate-drift-imp-branch-tree-on-support
 [OF *wf* controlled])
then show $?thesis$
by (rule *order-trans*[OF - *branch-tree-bound*])
qed

lemma *checked-staged-security-with-actual-alpha-prefix-partial-header-bad-set-hit-bound-from-prefix-drift-and-budgets*

assumes *wf*: *staged-budget-wellformed* *budgets*

and *controlled*: *staged-adversary-controlled* *budgets A*

and *prefix-bound*:

wp-event
 (*checked-staged-security-experiment-with-actual-alpha-prefix-data-state*
A)
 (*checked-staged-security-with-actual-alpha-prefix-bad-set-hit*
bad-sets)
adversary-initial-state $\leq P$

and *drift-bound*:

wp-event
 (*checked-staged-security-experiment-with-actual-alpha-prefix-data-state*
A)
 (*checked-staged-security-with-actual-alpha-prefix-partial-header-candidate-drift*
bad-sets)
adversary-initial-state $\leq D$

shows

wp-event
 (*checked-staged-security-experiment-with-actual-alpha-prefix-data-state*
A)
 (*checked-staged-security-with-actual-alpha-prefix-partial-header-bad-set-hit*
bad-sets)
adversary-initial-state $\leq$

$P + D +$
staged-phase-relation-error size
(staged-alpha-search-queries budgets 0)

proof –

have *fresh-bound*:

wp-event
(checked-staged-security-experiment-with-actual-alpha-prefix-data-state A)
(checked-staged-security-with-actual-alpha-prefix-partial-header-fresh-hit
bad-sets)
adversary-initial-state $\leq P + D$

by (rule
checked-staged-security-with-actual-alpha-prefix-partial-header-fresh-hit-bound-from-prefix-and-drift
[OF prefix-bound drift-bound])

have

wp-event
(checked-staged-security-experiment-with-actual-alpha-prefix-data-state A)
(checked-staged-security-with-actual-alpha-prefix-partial-header-bad-set-hit
bad-sets)
adversary-initial-state $\leq$
 $(P + D) +$
staged-phase-relation-error size
(staged-alpha-search-queries budgets 0)

by (rule
checked-staged-security-with-actual-alpha-prefix-partial-header-bad-set-hit-bound-from-fresh-and-budgets
[OF wf controlled fresh-bound])

then show *?thesis*

by (*simp add: add.assoc*)

qed

lemma *checked-staged-security-with-actual-alpha-prefix-composition-randomization-bad-imp-partial-header-or-collision*

assumes *support*:

out $\in$
set-dist
(execute
(checked-staged-security-experiment-with-actual-alpha-prefix-data-state
A)
adversary-initial-state)

and *bad*:

checked-staged-security-with-actual-alpha-prefix-verifier-event
composition-randomization-bad out

shows

checked-staged-security-with-actual-alpha-prefix-partial-header-bad-set-hit
composition-trace-bad-alpha-space out $\vee$
checked-staged-security-with-actual-alpha-prefix-verifier-event
hash-map-output-collision-bad out

proof (*cases out*)

case *None*

then show *?thesis*

using *bad*

```

    unfolding checked-staged-security-with-actual-alpha-prefix-verifier-event-def
  by simp
next
case (Some result-pack)
then obtain prefix prefix-state data attacker-state result final-state
where out-eq:
  out =
    Some (((((prefix, prefix-state), data), attacker-state),
      result), final-state)
  by (cases result-pack, auto split: prod.splits)
let ?s =
  verifier-state-from-adversary attacker-state
  (staged-proof-transcript data)
have verify-out:
  Some (result, final-state) ∈ set-dist (execute verify-monad ?s)
using support
unfolding out-eq
  checked-staged-security-experiment-with-actual-alpha-prefix-data-state-def
by (auto elim!: set-dist-bindE split: prod.splits)
have random-bad:
  composition-randomization-bad ?s (Some (result, final-state))
using bad
unfolding out-eq
  checked-staged-security-with-actual-alpha-prefix-verifier-event-def
  Let-def
by simp
have alpha-bad:
  alpha-bad-set-hit ?s composition-trace-bad-alpha-space
  (Some (result, final-state))
by (rule composition-randomization-bad-imp-alpha-bad-set-hit
  [OF random-bad])
have split:
  alpha-header-list-set-hit ?s
  (alpha-header-supported-partial-union-bad-sets ?s
    composition-trace-bad-alpha-space)
  (Some (result, final-state)) ∨
  hash-map-output-collision-bad ?s (Some (result, final-state))
by (rule alpha-bad-set-hit-imp-partial-union-or-collision
  [OF verify-out alpha-bad
    composition-trace-bad-alpha-space-subset-alpha-space])
then show ?thesis
proof
  assume partial:
    alpha-header-list-set-hit ?s
    (alpha-header-supported-partial-union-bad-sets ?s
      composition-trace-bad-alpha-space)
    (Some (result, final-state))
  have
    checked-staged-security-with-actual-alpha-prefix-partial-header-bad-set-hit

```

```

    composition-trace-bad-alpha-space out
  unfolding out-eq
    checked-staged-security-with-actual-alpha-prefix-partial-header-bad-set-hit-def
    Let-def
  using partial by simp
  then show ?thesis by simp
next
assume collision:
  hash-map-output-collision-bad ?s (Some (result, final-state))
have
  checked-staged-security-with-actual-alpha-prefix-verifier-event
  hash-map-output-collision-bad out
  unfolding out-eq
    checked-staged-security-with-actual-alpha-prefix-verifier-event-def
    Let-def
  using collision by simp
  then show ?thesis by simp
qed
qed

lemma checked-staged-security-with-actual-alpha-prefix-composition-randomization-bad-imp-prefix-prequery-drift-or-collision
  assumes support:
    out ∈
    set-dist
    (execute
      (checked-staged-security-experiment-with-actual-alpha-prefix-data-state
        A)
      adversary-initial-state)
  and bad:
    checked-staged-security-with-actual-alpha-prefix-verifier-event
    composition-randomization-bad out
  shows
    checked-staged-security-with-actual-alpha-prefix-bad-set-hit
    composition-trace-bad-alpha-space out ∨
    checked-staged-security-with-actual-alpha-prefix-partial-header-prequery-hit
    composition-trace-bad-alpha-space out ∨
    checked-staged-security-with-actual-alpha-prefix-composition-candidate-drift
    out ∨
    checked-staged-security-with-actual-alpha-prefix-verifier-event
    hash-map-output-collision-bad out
proof —
  have partial-or-collision:
    checked-staged-security-with-actual-alpha-prefix-partial-header-bad-set-hit
    composition-trace-bad-alpha-space out ∨
    checked-staged-security-with-actual-alpha-prefix-verifier-event
    hash-map-output-collision-bad out
  by (rule
    checked-staged-security-with-actual-alpha-prefix-composition-randomization-bad-imp-partial-header-or-collision-or-collision
    [OF support bad])

```

```

then show ?thesis
proof
  assume partial:
    checked-staged-security-with-actual-alpha-prefix-partial-header-bad-set-hit
    composition-trace-bad-alpha-space out
  have fresh-or-prequery:
    checked-staged-security-with-actual-alpha-prefix-partial-header-fresh-hit
    composition-trace-bad-alpha-space out ∨
    checked-staged-security-with-actual-alpha-prefix-partial-header-prequery-hit
    composition-trace-bad-alpha-space out
  by (rule
    checked-staged-security-with-actual-alpha-prefix-partial-header-bad-set-hit-imp-fresh-or-prequery
    [OF partial])
then show ?thesis
proof
  assume fresh:
    checked-staged-security-with-actual-alpha-prefix-partial-header-fresh-hit
    composition-trace-bad-alpha-space out
  have prefix-or-drift:
    checked-staged-security-with-actual-alpha-prefix-bad-set-hit
    composition-trace-bad-alpha-space out ∨
    checked-staged-security-with-actual-alpha-prefix-partial-header-candidate-drift
    composition-trace-bad-alpha-space out
  by (rule
    checked-staged-security-with-actual-alpha-prefix-partial-header-fresh-hit-imp-prefix-or-drift
    [OF fresh])
then show ?thesis
proof
  assume
    checked-staged-security-with-actual-alpha-prefix-bad-set-hit
    composition-trace-bad-alpha-space out
  then show ?thesis by simp
next
  assume drift:
    checked-staged-security-with-actual-alpha-prefix-partial-header-candidate-drift
    composition-trace-bad-alpha-space out
  have
    checked-staged-security-with-actual-alpha-prefix-composition-candidate-drift
    out
  unfolding
    checked-staged-security-with-actual-alpha-prefix-composition-candidate-drift-def
  using bad drift by simp
  then show ?thesis by simp
qed
next
  assume
    checked-staged-security-with-actual-alpha-prefix-partial-header-prequery-hit
    composition-trace-bad-alpha-space out
  then show ?thesis by simp

```

```

qed
next
assume
  checked-staged-security-with-actual-alpha-prefix-verifier-event
  hash-map-output-collision-bad out
then show ?thesis by simp
qed
qed

lemma checked-staged-security-with-actual-alpha-prefix-composition-randomization-bad-bound-from-prefix-prequery-d
assumes prefix-bound:
  wp-event
    (checked-staged-security-experiment-with-actual-alpha-prefix-data-state
      A)
    (checked-staged-security-with-actual-alpha-prefix-bad-set-hit
      composition-trace-bad-alpha-space)
  adversary-initial-state  $\leq P$ 
and prequery-bound:
  wp-event
    (checked-staged-security-experiment-with-actual-alpha-prefix-data-state
      A)
    (checked-staged-security-with-actual-alpha-prefix-partial-header-prequery-hit
      composition-trace-bad-alpha-space)
  adversary-initial-state  $\leq Q$ 
and drift-bound:
  wp-event
    (checked-staged-security-experiment-with-actual-alpha-prefix-data-state
      A)
    checked-staged-security-with-actual-alpha-prefix-composition-candidate-drift
  adversary-initial-state  $\leq D$ 
and collision-bound:
  wp-event
    (checked-staged-security-experiment-with-actual-alpha-prefix-data-state
      A)
    (checked-staged-security-with-actual-alpha-prefix-verifier-event
      hash-map-output-collision-bad)
  adversary-initial-state  $\leq C$ 
shows
  wp-event
    (checked-staged-security-experiment-with-actual-alpha-prefix-data-state
      A)
    (checked-staged-security-with-actual-alpha-prefix-verifier-event
      composition-randomization-bad)
  adversary-initial-state  $\leq P + Q + D + C$ 
proof -
let ?M =
  checked-staged-security-experiment-with-actual-alpha-prefix-data-state A
let ?Prefix =
  checked-staged-security-with-actual-alpha-prefix-bad-set-hit

```

```

    composition-trace-bad-alpha-space
  let ?Prequery =
    checked-staged-security-with-actual-alpha-prefix-partial-header-prequery-hit
    composition-trace-bad-alpha-space
  let ?Drift =
    checked-staged-security-with-actual-alpha-prefix-composition-candidate-drift
  let ?Collision =
    checked-staged-security-with-actual-alpha-prefix-verifier-event
    hash-map-output-collision-bad
  let ?Random =
    checked-staged-security-with-actual-alpha-prefix-verifier-event
    composition-randomization-bad
  have wp-event ?M ?Random adversary-initial-state ≤
    wp-event ?M
    (λout. ?Prefix out ∨ ?Prequery out ∨ ?Drift out ∨
     ?Collision out)
    adversary-initial-state
  by (rule wp-event-mono-on-support)
  (rule
    checked-staged-security-with-actual-alpha-prefix-composition-randomization-bad-imp-prefix-prequery-drift-or-collision)
  also have ... ≤
    wp-event ?M ?Prefix adversary-initial-state +
    wp-event ?M ?Prequery adversary-initial-state +
    wp-event ?M ?Drift adversary-initial-state +
    wp-event ?M ?Collision adversary-initial-state
  by (rule wp-event-union-bound4)
  also have ... ≤ P + Q + D + C
  by (intro add-mono prefix-bound prequery-bound drift-bound
    collision-bound)
  finally show ?thesis .
qed

lemma checked-staged-security-with-actual-alpha-prefix-composition-randomization-bad-bound-from-partial-header-and-collision-bound:
  assumes partial-bound:
    wp-event
      (checked-staged-security-experiment-with-actual-alpha-prefix-data-state
        A)
      (checked-staged-security-with-actual-alpha-prefix-partial-header-bad-set-hit
        composition-trace-bad-alpha-space)
      adversary-initial-state ≤ P
  and collision-bound:
    wp-event
      (checked-staged-security-experiment-with-actual-alpha-prefix-data-state
        A)
      (checked-staged-security-with-actual-alpha-prefix-verifier-event
        hash-map-output-collision-bad)
      adversary-initial-state ≤ C
  shows
    wp-event

```



```

    (checked-staged-security-experiment-with-actual-alpha-prefix-data-state
      A)
    (checked-staged-security-with-actual-alpha-prefix-verifier-event
      composition-randomization-bad)
    adversary-initial-state  $\leq P + C$ 
  proof -
    let ?M =
      checked-staged-security-experiment-with-actual-alpha-prefix-data-state A
    let ?Partial =
      checked-staged-security-with-actual-alpha-prefix-partial-header-bad-set-hit
      composition-trace-bad-alpha-space
    let ?Collision =
      checked-staged-security-with-actual-alpha-prefix-verifier-event
      hash-map-output-collision-bad
    let ?Random =
      checked-staged-security-with-actual-alpha-prefix-verifier-event
      composition-randomization-bad
    have wp-event ?M ?Random adversary-initial-state  $\leq$ 
      wp-event ?M ( $\lambda$ out. ?Partial out  $\vee$  ?Collision out)
      adversary-initial-state
    by (rule wp-event-mono-on-support)
      (rule
        checked-staged-security-with-actual-alpha-prefix-composition-randomization-bad-imp-partial-header-or-collision-o
    also have ...  $\leq$ 
      wp-event ?M ?Partial adversary-initial-state +
      wp-event ?M ?Collision adversary-initial-state
    by (rule wp-event-union-bound)
    also have ...  $\leq P + C$ 
    by (intro add-mono partial-bound collision-bound)
    finally show ?thesis .
  qed

  lemma checked-staged-security-with-data-state-composition-bad-bound-from-actual-alpha-prefix-partial-header-and-co
  assumes partial-bound:
    wp-event
      (checked-staged-security-experiment-with-actual-alpha-prefix-data-state
        A)
      (checked-staged-security-with-actual-alpha-prefix-partial-header-bad-set-hit
        composition-trace-bad-alpha-space)
      adversary-initial-state  $\leq P$ 
  and collision-bound:
    wp-event
      (checked-staged-security-experiment-with-actual-alpha-prefix-data-state
        A)
      (checked-staged-security-with-actual-alpha-prefix-verifier-event
        hash-map-output-collision-bad)
      adversary-initial-state  $\leq C$ 
  shows
    wp-event (checked-staged-security-experiment-with-data-state A)

```

```

      (staged-security-with-data-state-verifier-event composition-bad)
      adversary-initial-state  $\leq P + C$ 
proof –
  let ?M =
    checked-staged-security-experiment-with-actual-alpha-prefix-data-state A
  let ?degree =
    checked-staged-security-with-actual-alpha-prefix-verifier-event
    composition-degree-bad
  let ?random =
    checked-staged-security-with-actual-alpha-prefix-verifier-event
    composition-randomization-bad
  let ?bad =
    checked-staged-security-with-actual-alpha-prefix-verifier-event
    composition-bad
  have projected:
    wp-event (checked-staged-security-experiment-with-data-state A)
      (staged-security-with-data-state-verifier-event composition-bad)
      adversary-initial-state =
        wp-event ?M ?bad adversary-initial-state
  by (rule
    checked-staged-security-with-data-state-verifier-event-actual-alpha-prefix-projection)
  have event-eq: ?bad = ( $\lambda$ out. ?degree out  $\vee$  ?random out)
  by (rule ext)
    (auto simp:
      checked-staged-security-with-actual-alpha-prefix-verifier-event-def
      composition-bad-def split: option.splits prod.splits)
  have degree-false: ?degree = ( $\lambda$ out. False)
  by (rule ext)
    (auto simp:
      checked-staged-security-with-actual-alpha-prefix-verifier-event-def
      composition-degree-bad-false[OF spec-degree-wellformed-from-spec-query-margin]
      split: option.splits prod.splits)
  have degree-bound:
    wp-event ?M ?degree adversary-initial-state  $\leq 0$ 
  unfolding degree-false wp-event-def wp-def dist-expect-def by simp
  have random-bound:
    wp-event ?M ?random adversary-initial-state  $\leq P + C$ 
  by (rule
    checked-staged-security-with-actual-alpha-prefix-composition-randomization-bad-bound-from-partial-header-and-c
    [OF partial-bound collision-bound])
  have wp-event ?M ?bad adversary-initial-state  $\leq$ 
    wp-event ?M ?degree adversary-initial-state +
    wp-event ?M ?random adversary-initial-state
  unfolding event-eq by (rule wp-event-union-bound)
  also have ...  $\leq 0 + (P + C)$ 
  by (intro add-mono degree-bound random-bound)
  finally show ?thesis
  unfolding projected by simp
qed

```

lemma *checked-staged-security-with-actual-alpha-prefix-hash-map-output-collision-bad-bound-controlled:*

assumes *wf: staged-budget-wellformed budgets*

and *controlled: staged-adversary-controlled budgets A*

shows

wp-event

(*checked-staged-security-experiment-with-actual-alpha-prefix-data-state A*)

(*checked-staged-security-with-actual-alpha-prefix-verifier-event*
hash-map-output-collision-bad)

adversary-initial-state $\leq$

hash-collision-budget-value 0

(*staged-attacker-query-budget budgets* +
staged-challenge-query-budget + *verifier-hash-query-budget*)

proof –

have *data-bound:*

wp-event (*checked-staged-security-experiment-with-data-state A*)

(*staged-security-with-data-state-verifier-event*
hash-map-output-collision-bad)

adversary-initial-state $\leq$

hash-collision-budget-value 0

(*staged-attacker-query-budget budgets* +
staged-challenge-query-budget + *verifier-hash-query-budget*)

by (*rule*

checked-staged-security-with-data-state-hash-map-output-collision-bad-bound-controlled
[OF wf controlled])

then show *?thesis*

using

checked-staged-security-with-data-state-verifier-event-actual-alpha-prefix-projection
[of A hash-map-output-collision-bad]

by *simp*

qed

lemma *checked-staged-security-with-data-state-composition-bad-bound-from-actual-alpha-prefix-partial-header-and-co*

assumes *wf: staged-budget-wellformed budgets*

and *controlled: staged-adversary-controlled budgets A*

and *partial-bound:*

wp-event

(*checked-staged-security-experiment-with-actual-alpha-prefix-data-state*
A)

(*checked-staged-security-with-actual-alpha-prefix-partial-header-bad-set-hit*
composition-trace-bad-alpha-space)

adversary-initial-state $\leq P$

shows

wp-event (*checked-staged-security-experiment-with-data-state A*)

(*staged-security-with-data-state-verifier-event composition-bad*)

adversary-initial-state $\leq$

P +

hash-collision-budget-value 0

(*staged-attacker-query-budget budgets* +

staged-challenge-query-budget + verifier-hash-query-budget)

proof –

have *collision-bound*:

wp-event
(checked-staged-security-experiment-with-actual-alpha-prefix-data-state A)
(checked-staged-security-with-actual-alpha-prefix-verifier-event
hash-map-output-collision-bad)
adversary-initial-state ≤
hash-collision-budget-value 0
(staged-attacker-query-budget budgets +
staged-challenge-query-budget + verifier-hash-query-budget)

by (rule
checked-staged-security-with-actual-alpha-prefix-hash-map-output-collision-bad-bound-controlled
[OF wf controlled])

show *?thesis*

by (rule
checked-staged-security-with-data-state-composition-bad-bound-from-actual-alpha-prefix-partial-header-and-collision-bound
[OF partial-bound collision-bound])

qed

lemma *checked-staged-security-with-data-state-composition-bad-bound-from-partial-header-candidate-drift-and-budget*

assumes *wf: staged-budget-wellformed budgets*

and *controlled: staged-adversary-controlled budgets A*

and *drift-bound*:

wp-event
(checked-staged-security-experiment-with-actual-alpha-prefix-data-state
A)
(checked-staged-security-with-actual-alpha-prefix-partial-header-candidate-drift
composition-trace-bad-alpha-space)
adversary-initial-state ≤ D

shows

wp-event (checked-staged-security-experiment-with-data-state A)
(staged-security-with-data-state-verifier-event composition-bad)
adversary-initial-state ≤
composition-error-bound +
staged-phase-relation-error size
(staged-alpha-search-queries budgets 0) +
hash-collision-budget-value 0
(staged-alpha-search-queries budgets 0) +
D +
staged-phase-relation-error size
(staged-alpha-search-queries budgets 0) +
hash-collision-budget-value 0
(staged-attacker-query-budget budgets +
staged-challenge-query-budget + verifier-hash-query-budget)

proof –

let *?Prefix =*
composition-error-bound +
staged-phase-relation-error size

```

      (staged-alpha-search-queries budgets 0) +
      hash-collision-budget-value 0
      (staged-alpha-search-queries budgets 0)
have transcript-bound:
  wp-event (checked-staged-transcript-with-alpha-prefix-program A)
    (checked-staged-transcript-actual-alpha-prefix-bad-set-hit
      composition-trace-bad-alpha-space)
    adversary-initial-state ≤ ?Prefix
by (rule
      checked-staged-transcript-with-alpha-prefix-actual-bad-set-hit-bound-from-budgets
      [OF wf controlled])
have prefix-bound:
  wp-event
    (checked-staged-security-experiment-with-actual-alpha-prefix-data-state A)
    (checked-staged-security-with-actual-alpha-prefix-bad-set-hit
      composition-trace-bad-alpha-space)
    adversary-initial-state ≤ ?Prefix
by (rule
      checked-staged-security-with-actual-alpha-prefix-bad-set-hit-bound-from-transcript
      [OF transcript-bound])
have partial-bound:
  wp-event
    (checked-staged-security-experiment-with-actual-alpha-prefix-data-state A)
    (checked-staged-security-with-actual-alpha-prefix-partial-header-bad-set-hit
      composition-trace-bad-alpha-space)
    adversary-initial-state ≤
      ?Prefix + D +
      staged-phase-relation-error size
      (staged-alpha-search-queries budgets 0)
by (rule
      checked-staged-security-with-actual-alpha-prefix-partial-header-bad-set-hit-bound-from-prefix-drift-and-budgets
      [OF wf controlled prefix-bound drift-bound])
have
  wp-event (checked-staged-security-experiment-with-data-state A)
    (staged-security-with-data-state-verifier-event composition-bad)
    adversary-initial-state ≤
      (?Prefix + D +
        staged-phase-relation-error size
        (staged-alpha-search-queries budgets 0)) +
      hash-collision-budget-value 0
      (staged-attacker-query-budget budgets +
        staged-challenge-query-budget + verifier-hash-query-budget)
by (rule
      checked-staged-security-with-data-state-composition-bad-bound-from-actual-alpha-prefix-partial-header-and-control
      [OF wf controlled partial-bound])
then show ?thesis
by (simp add: add.assoc)
qed

```

lemma *checked-staged-security-with-data-state-composition-bad-bound-from-partial-prequery-branch-tree-and-budgets*

assumes *wf*: staged-budget-wellformed budgets

and *controlled*: staged-adversary-controlled budgets *A*

and *branch-tree-bound*:

wp-event

(*checked-staged-security-experiment-with-actual-alpha-prefix-data-state*
A)

checked-staged-security-with-actual-alpha-prefix-branch-tree-output-hit
adversary-initial-state $\leq T$

shows

wp-event (*checked-staged-security-experiment-with-data-state* *A*)

(*staged-security-with-data-state-verifier-event composition-bad*)

adversary-initial-state $\leq$

composition-error-bound +

staged-phase-relation-error size

(*staged-alpha-search-queries budgets 0*) +

hash-collision-budget-value 0

(*staged-alpha-search-queries budgets 0*) +

staged-phase-relation-error size

(*staged-alpha-search-queries budgets 0*) +

T +

hash-collision-budget-value 0

(*staged-attacker-query-budget budgets* +

staged-challenge-query-budget + *verifier-hash-query-budget*)

proof –

let *?M* =

checked-staged-security-experiment-with-actual-alpha-prefix-data-state A

let *?Prefix* =

composition-error-bound +

staged-phase-relation-error size

(*staged-alpha-search-queries budgets 0*) +

hash-collision-budget-value 0

(*staged-alpha-search-queries budgets 0*)

let *?Pre* =

staged-phase-relation-error size

(*staged-alpha-search-queries budgets 0*)

let *?Coll* =

hash-collision-budget-value 0

(*staged-attacker-query-budget budgets* +

staged-challenge-query-budget + *verifier-hash-query-budget*)

have *transcript-bound*:

wp-event (*checked-staged-transcript-with-alpha-prefix-program A*)

(*checked-staged-transcript-actual-alpha-prefix-bad-set-hit*

composition-trace-bad-alpha-space)

adversary-initial-state $\leq$ *?Prefix*

by (*rule*

checked-staged-transcript-with-alpha-prefix-actual-bad-set-hit-bound-from-budgets

[*OF wf controlled*])

have *prefix-bound*:

```

wp-event ?M
  (checked-staged-security-with-actual-alpha-prefix-bad-set-hit
   composition-trace-bad-alpha-space)
  adversary-initial-state ≤ ?Prefix
by (rule
  checked-staged-security-with-actual-alpha-prefix-bad-set-hit-bound-from-transcript
  [OF transcript-bound])
have prequery-bound:
  wp-event ?M
    (checked-staged-security-with-actual-alpha-prefix-partial-header-prequery-hit
     composition-trace-bad-alpha-space)
    adversary-initial-state ≤ ?Pre
  by (rule
    checked-staged-security-with-actual-alpha-prefix-vector-alpha-prequery-accounting
    [OF wf controlled])
have drift-bound:
  wp-event ?M
    checked-staged-security-with-actual-alpha-prefix-composition-candidate-drift
    adversary-initial-state ≤ T
  by (rule
    checked-staged-security-with-actual-alpha-prefix-composition-candidate-drift-bound-from-branch-tree
    [OF wf controlled branch-tree-bound])
have collision-bound:
  wp-event ?M
    (checked-staged-security-with-actual-alpha-prefix-verifier-event
     hash-map-output-collision-bad)
    adversary-initial-state ≤ ?Coll
  by (rule
    checked-staged-security-with-actual-alpha-prefix-hash-map-output-collision-bad-bound-controlled
    [OF wf controlled])
let ?degree =
  checked-staged-security-with-actual-alpha-prefix-verifier-event
  composition-degree-bad
let ?random =
  checked-staged-security-with-actual-alpha-prefix-verifier-event
  composition-randomization-bad
let ?bad =
  checked-staged-security-with-actual-alpha-prefix-verifier-event
  composition-bad
have projected:
  wp-event (checked-staged-security-experiment-with-data-state A)
    (staged-security-with-data-state-verifier-event composition-bad)
    adversary-initial-state =
  wp-event ?M ?bad adversary-initial-state
  by (rule
    checked-staged-security-with-data-state-verifier-event-actual-alpha-prefix-projection)
have event-eq: ?bad = (λout. ?degree out ∨ ?random out)
  by (rule ext)
  (auto simp:

```

```

    checked-staged-security-with-actual-alpha-prefix-verifier-event-def
    composition-bad-def split: option.splits prod.splits)
have degree-false: ?degree = ( $\lambda$ out. False)
by (rule ext)
  (auto simp:
    checked-staged-security-with-actual-alpha-prefix-verifier-event-def
    composition-degree-bad-false[OF spec-degree-wellformed-from-spec-query-margin]
    split: option.splits prod.splits)
have degree-bound:
  wp-event ?M ?degree adversary-initial-state  $\leq$  0
  unfolding degree-false wp-event-def wp-def dist-expect-def by simp
have random-bound:
  wp-event ?M ?random adversary-initial-state  $\leq$ 
    ?Prefix + ?Pre + T + ?Coll
by (rule
  checked-staged-security-with-actual-alpha-prefix-composition-randomization-bad-bound-from-prefix-prequery-drift-
    [OF prefix-bound prequery-bound drift-bound collision-bound])
have wp-event ?M ?bad adversary-initial-state  $\leq$ 
  wp-event ?M ?degree adversary-initial-state +
  wp-event ?M ?random adversary-initial-state
  unfolding event-eq by (rule wp-event-union-bound)
also have ...  $\leq$  0 + (?Prefix + ?Pre + T + ?Coll)
  by (intro add-mono degree-bound random-bound)
finally show ?thesis
  unfolding projected by (simp add: add.assoc)
qed

end

end

```

theory Soundness-Staged-Candidate-Query

imports

Soundness-Staged-Query

Soundness-Staged-Partial-Query

begin

Candidate-indexed dynamic query-prefix targets.

These targets are narrower than the old global partial-opening fixed cover: they are parameterized by one fixed trace/composition candidate pair and the authenticated openings extracted for that pair. This is the shape needed for the composition candidate-drift query case, where the target must be chosen before the query-index challenge is sampled.

context soundness

begin

definition staged-query-prefix-candidate-opening-query-target

$::$ 'f authenticated-opening list list $\Rightarrow$

'f authenticated-opening list list $\Rightarrow$

$\text{'f list} \Rightarrow \text{nat} \Rightarrow$
 $\text{'f staged-query-prefix-data} \Rightarrow \text{'f protocol-channel} \Rightarrow \text{nat set}$
where
 $\text{staged-query-prefix-candidate-opening-query-target trace-openings}$
 $\text{composition-openings as } i \text{ prefix prefix-state} =$
 $\text{partial-query-success-indices-at trace-openings composition-openings as } i$

definition *staged-query-prefix-candidate-pair-query-target*
 $:: \text{'f list} \Rightarrow \text{'f list} \Rightarrow \text{'f list} \Rightarrow$
 $\text{'f staged-query-prefix-data} \Rightarrow \text{'f protocol-channel} \Rightarrow \text{nat set}$
where
 $\text{staged-query-prefix-candidate-pair-query-target trace-table}$
 $\text{composition-table as prefix prefix-state} =$
 $\text{query-sampling-success-space trace-table composition-table as}$

definition *staged-query-prefix-candidate-opening-query-target-from-prefix*
 $:: \text{'f authenticated-opening list list} \Rightarrow$
 $\text{'f authenticated-opening list list} \Rightarrow$
 $\text{nat} \Rightarrow \text{'f staged-query-prefix-data} \Rightarrow$
 $\text{'f protocol-channel} \Rightarrow \text{nat set}$
where
 $\text{staged-query-prefix-candidate-opening-query-target-from-prefix}$
 $\text{trace-openings composition-openings } i \text{ prefix prefix-state} =$
 $\text{partial-query-success-indices-at trace-openings composition-openings}$
 $(\text{sqp-alphas prefix}) \text{ } i$

definition *staged-query-prefix-candidate-pair-query-target-from-prefix*
 $:: \text{'f list} \Rightarrow \text{'f list} \Rightarrow \text{'f staged-query-prefix-data} \Rightarrow$
 $\text{'f protocol-channel} \Rightarrow \text{nat set}$
where
 $\text{staged-query-prefix-candidate-pair-query-target-from-prefix trace-table}$
 $\text{composition-table prefix prefix-state} =$
 $\text{query-sampling-success-space trace-table composition-table}$
 $(\text{sqp-alphas prefix})$

definition *checked-staged-security-with-actual-alpha-prefix-candidate-pair-query-hit*
where
 $\text{checked-staged-security-with-actual-alpha-prefix-candidate-pair-query-hit } A$
 $\text{trace-table composition-table out} \longleftrightarrow$
 $(\text{case out of}$
 $\text{None} \Rightarrow \text{False}$
 $| \text{Some } (((((\text{alpha-prefix}, \text{alpha-prefix-state}), \text{data}), \text{attacker-state}), \text{result}),$
 $\text{final-state}) \Rightarrow$
 $(\exists i \text{ prefix prefix-state raw raw-state.}$
 $i < \text{rounds} \wedge$
 $\text{Some } (\text{data}, \text{attacker-state}) \in$
 set-dist
 $(\text{execute } (\text{checked-staged-transcript-program } A)$
 $\text{adversary-initial-state}) \wedge$

$\text{Some } (((\text{prefix}, \text{prefix-state}), \text{raw}), \text{raw-state}) \in$
 set-dist
 $(\text{execute } (\text{checked-staged-query-prefix-receive-with-state } A \ i)$
 $\text{adversary-initial-state}) \wedge$
 $\text{checked-staged-query-prefix-dynamic-index-hit}$
 $(\text{staged-query-prefix-candidate-pair-query-target } \text{trace-table}$
 $\text{composition-table } (\text{staged-alphas } \text{data}))$
 $(\text{Some } (((\text{prefix}, \text{prefix-state}), \text{raw}), \text{raw-state}))))$

definition *checked-staged-security-with-actual-alpha-prefix-candidate-opening-query-hit*
where

$\text{checked-staged-security-with-actual-alpha-prefix-candidate-opening-query-hit } A$
 $\text{trace-openings } \text{composition-openings } \text{as } i \text{ out } \longleftrightarrow$
 $(\text{case out of}$
 $\text{None} \Rightarrow \text{False}$
 $| \text{Some } (((((\text{alpha-prefix}, \text{alpha-prefix-state}), \text{data}), \text{attacker-state}), \text{result}),$
 $\text{final-state}) \Rightarrow$
 $\text{as} = \text{staged-alphas } \text{data} \wedge$
 $i < \text{rounds} \wedge$
 $(\exists \text{prefix } \text{prefix-state } \text{raw } \text{raw-state}.$
 $\text{Some } (\text{data}, \text{attacker-state}) \in$
 set-dist
 $(\text{execute } (\text{checked-staged-transcript-program } A)$
 $\text{adversary-initial-state}) \wedge$
 $\text{Some } (((\text{prefix}, \text{prefix-state}), \text{raw}), \text{raw-state}) \in$
 set-dist
 $(\text{execute } (\text{checked-staged-query-prefix-receive-with-state } A \ i)$
 $\text{adversary-initial-state}) \wedge$
 $\text{checked-staged-query-prefix-dynamic-index-hit}$
 $(\text{staged-query-prefix-candidate-opening-query-target}$
 $\text{trace-openings } \text{composition-openings } \text{as } i)$
 $(\text{Some } (((\text{prefix}, \text{prefix-state}), \text{raw}), \text{raw-state}))))$

lemma *staged-query-prefix-candidate-opening-query-target-subset:*
 $\text{staged-query-prefix-candidate-opening-query-target } \text{trace-openings}$
 $\text{composition-openings } \text{as } i \text{ prefix } \text{prefix-state} \subseteq \text{query-sample-space}$
unfolding *staged-query-prefix-candidate-opening-query-target-def*
by (*rule partial-query-success-indices-at-subset*)

lemma *staged-query-prefix-candidate-pair-query-target-subset:*
 $\text{staged-query-prefix-candidate-pair-query-target } \text{trace-table}$
 $\text{composition-table } \text{as } \text{prefix } \text{prefix-state} \subseteq \text{query-sample-space}$
unfolding *staged-query-prefix-candidate-pair-query-target-def*
by (*rule query-sampling-success-space-subset*)

lemma *staged-query-prefix-candidate-opening-query-target-from-prefix-subset:*
 $\text{staged-query-prefix-candidate-opening-query-target-from-prefix}$
 $\text{trace-openings } \text{composition-openings } i \text{ prefix } \text{prefix-state} \subseteq$
 $\text{query-sample-space}$

unfolding *staged-query-prefix-candidate-opening-query-target-from-prefix-def*
by (rule *partial-query-success-indices-at-subset*)

lemma *staged-query-prefix-candidate-pair-query-target-from-prefix-subset:*

staged-query-prefix-candidate-pair-query-target-from-prefix trace-table
composition-table prefix prefix-state $\subseteq$ query-sample-space

unfolding *staged-query-prefix-candidate-pair-query-target-from-prefix-def*
by (rule *query-sampling-success-space-subset*)

lemma *staged-query-prefix-candidate-opening-query-target-subset-candidate-pair:*

assumes *trace-candidate:*

partial-trace-table-candidate trace-table trace-openings

and *comp-candidate:*

partial-composition-table-candidate composition-table

composition-openings

and *trace-low: trace-table-low-degree trace-table*

and *comp-low: composition-table-low-degree maxDegree composition-table*

and *not-all:*

$\neg$ *all-queries-consistent trace-table composition-table as*

shows

staged-query-prefix-candidate-opening-query-target trace-openings

composition-openings as i prefix prefix-state $\subseteq$

staged-query-prefix-candidate-pair-query-target trace-table

composition-table as prefix prefix-state

unfolding *staged-query-prefix-candidate-opening-query-target-def*

staged-query-prefix-candidate-pair-query-target-def

by (rule *partial-query-success-indices-at-subset-query-sampling-success-space*
 $[OF \text{ trace-candidate comp-candidate trace-low comp-low not-all}]$)

lemma *staged-query-prefix-candidate-opening-query-target-from-prefix-subset-candidate-pair-from-prefix:*

assumes *trace-candidate:*

partial-trace-table-candidate trace-table trace-openings

and *comp-candidate:*

partial-composition-table-candidate composition-table

composition-openings

and *trace-low: trace-table-low-degree trace-table*

and *comp-low: composition-table-low-degree maxDegree composition-table*

and *not-all:*

$\neg$ *all-queries-consistent trace-table composition-table*

(sqp-alphas prefix)

shows

staged-query-prefix-candidate-opening-query-target-from-prefix

trace-openings composition-openings i prefix prefix-state $\subseteq$

staged-query-prefix-candidate-pair-query-target-from-prefix trace-table

composition-table prefix prefix-state

unfolding *staged-query-prefix-candidate-opening-query-target-from-prefix-def*

staged-query-prefix-candidate-pair-query-target-from-prefix-def

by (rule *partial-query-success-indices-at-subset-query-sampling-success-space*
 $[OF \text{ trace-candidate comp-candidate trace-low comp-low not-all}]$)

lemma *checked-staged-query-prefix-dynamic-index-hit-mono*:
assumes *subset*: $\bigwedge \text{prefix prefix-state}. A \text{ prefix prefix-state} \subseteq B \text{ prefix prefix-state}$
and *hit*:
checked-staged-query-prefix-dynamic-index-hit A
(Some (((prefix, prefix-state), raw), raw-state))
shows
checked-staged-query-prefix-dynamic-index-hit B
(Some (((prefix, prefix-state), raw), raw-state))
using *subset hit*
unfolding *checked-staged-query-prefix-dynamic-index-hit-def* **by** *auto*

lemma *partial-trace-table-candidate-single-round-from-all-rounds*:
assumes *cand*: *partial-trace-table-candidate trace-table trace-openingsss*
and *i-bound*: $i < \text{rounds}$
shows
partial-trace-table-candidate trace-table
((replicate rounds []) [i := trace-openingsss ! i])
proof –
have *len-table*: $\text{length trace-table} = \text{scale} * \text{clength}$
using *cand* **unfolding** *partial-trace-table-candidate-def* **by** *simp*
have *agrees*:
table-agrees-with-authenticated-openings trace-table
*(scale * clength) (trace-openingsss ! i)*
using *cand i-bound* **unfolding** *partial-trace-table-candidate-def* **by** *blast*
show *?thesis*
by (rule *partial-trace-table-candidate-single-roundI*
 $[OF \text{ i-bound len-table agrees}]$)
qed

lemma *partial-composition-table-candidate-single-round-from-all-rounds*:
assumes *cand*:
partial-composition-table-candidate composition-table
composition-openingsss
and *i-bound*: $i < \text{rounds}$
shows
partial-composition-table-candidate composition-table
((replicate rounds []) [i := composition-openingsss ! i])
proof –
have *len-table*: $\text{length composition-table} = \text{scale} * \text{clength}$
using *cand* **unfolding** *partial-composition-table-candidate-def* **by** *simp*
have *agrees*:
table-agrees-with-authenticated-openings composition-table
*(scale * clength) (composition-openingsss ! i)*
using *cand i-bound* **unfolding** *partial-composition-table-candidate-def*
by *blast*
show *?thesis*
by (rule *partial-composition-table-candidate-single-roundI*)

[OF *i-bound len-table agrees*])

qed

lemma *staged-query-prefix-candidate-opening-query-target-single-roundI*:
assumes *i-bound*: $i < \text{rounds}$
and *consistent*:
partial-query-openings-consistent trace-openings composition-openings
as idx
shows
idx $\in$
staged-query-prefix-candidate-opening-query-target
 $((\text{replicate rounds } []) [i := \text{trace-openings}])$
 $((\text{replicate rounds } []) [i := \text{composition-openings}])$
as i prefix prefix-state
unfolding *staged-query-prefix-candidate-opening-query-target-def*
by (rule *partial-query-success-indices-atI*,
rule *partial-query-round-consistent-single-roundI*
[OF *i-bound consistent*])

lemma *checked-staged-query-prefix-candidate-opening-target-single-round-hitI*:
assumes *i-bound*: $i < \text{rounds}$
and *consistent*:
partial-query-openings-consistent trace-openings composition-openings
as (index (to-nat raw))
shows
checked-staged-query-prefix-dynamic-index-hit
(staged-query-prefix-candidate-opening-query-target
 $((\text{replicate rounds } []) [i := \text{trace-openings}])$
 $((\text{replicate rounds } []) [i := \text{composition-openings}])$
as i)
(Some (((prefix, prefix-state), raw), raw-state))

proof –
have *index (to-nat raw)* $\in$
staged-query-prefix-candidate-opening-query-target
 $((\text{replicate rounds } []) [i := \text{trace-openings}])$
 $((\text{replicate rounds } []) [i := \text{composition-openings}])$
as i prefix prefix-state
by (rule *staged-query-prefix-candidate-opening-query-target-single-roundI*
[OF *i-bound consistent*])
then show *?thesis*
unfolding *checked-staged-query-prefix-dynamic-index-hit-def* **by** *simp*
qed

lemma *checked-staged-security-with-actual-alpha-prefix-candidate-opening-query-hitI*:
assumes *as-eq*: $as = \text{staged-alphas data}$
and *i-bound*: $i < \text{rounds}$
and *builder*:
Some (data, attacker-state) $\in$
set-dist

```

      (execute (checked-staged-transcript-program A)
        adversary-initial-state)
    and prefix-receive:
      Some (((prefix, prefix-state), raw), raw-state) ∈
        set-dist
        (execute (checked-staged-query-prefix-receive-with-state A i)
          adversary-initial-state)
    and target-hit:
      checked-staged-query-prefix-dynamic-index-hit
      (staged-query-prefix-candidate-opening-query-target
        trace-openings composition-openings as i)
      (Some (((prefix, prefix-state), raw), raw-state))
  shows
    checked-staged-security-with-actual-alpha-prefix-candidate-opening-query-hit A
    trace-openings composition-openings as i
    (Some (((((alpha-prefix, alpha-prefix-state), data), attacker-state), result),
      final-state))
  proof -
    have body:
      ∃ prefix prefix-state raw raw-state.
      Some (data, attacker-state) ∈
        set-dist
        (execute (checked-staged-transcript-program A)
          adversary-initial-state) ∧
      Some (((prefix, prefix-state), raw), raw-state) ∈
        set-dist
        (execute (checked-staged-query-prefix-receive-with-state A i)
          adversary-initial-state) ∧
      checked-staged-query-prefix-dynamic-index-hit
      (staged-query-prefix-candidate-opening-query-target
        trace-openings composition-openings as i)
      (Some (((prefix, prefix-state), raw), raw-state))
    by (intro exI[of - prefix] exI[of - prefix-state]
      exI[of - raw] exI[of - raw-state])
      (use builder prefix-receive target-hit in simp)
    show ?thesis
      unfolding
        checked-staged-security-with-actual-alpha-prefix-candidate-opening-query-hit-def
      using as-eq i-bound body by simp
  qed

```

lemma *checked-staged-security-with-actual-alpha-prefix-candidate-opening-query-hitI-from-raw:*
assumes *as-eq*: *as = staged-alphas data*
and *i-bound*: *i < rounds*
and *builder*:
 Some (data, attacker-state) ∈
 set-dist
 (execute (checked-staged-transcript-program A)
 adversary-initial-state)

and *prefix-receive*:
Some (((*prefix*, *prefix-state*), *raw*), *raw-state*) ∈
set-dist
(execute (checked-staged-query-prefix-receive-with-state A i)
adversary-initial-state)
and *consistent*:
partial-query-openings-consistent trace-openings composition-openings
as (index (to-nat raw))
shows
checked-staged-security-with-actual-alpha-prefix-candidate-opening-query-hit A
((replicate rounds []) [i := trace-openings])
((replicate rounds []) [i := composition-openings])
as i
(Some (((((alpha-prefix, alpha-prefix-state), data), attacker-state), result),
final-state)))
proof –
have *target-hit*:
checked-staged-query-prefix-dynamic-index-hit
(staged-query-prefix-candidate-opening-query-target
((replicate rounds []) [i := trace-openings])
((replicate rounds []) [i := composition-openings])
as i)
(Some (((prefix, prefix-state), raw), raw-state))
by (*rule checked-staged-query-prefix-candidate-opening-target-single-round-hitI*
[OF i-bound consistent])
show *?thesis*
by (*rule*
checked-staged-security-with-actual-alpha-prefix-candidate-opening-query-hitI
[OF as-eq i-bound builder prefix-receive target-hit])
qed

lemma *checked-staged-security-with-actual-alpha-prefix-candidate-opening-query-hitI-from-prefix-receive*:

assumes *i-bound: i < rounds*

and *builder*:

Some (*data*, *attacker-state*) ∈
set-dist

(execute (checked-staged-transcript-program A)
adversary-initial-state)

and *prefix-receive*:

Some (((*prefix*, *prefix-state*), *raw*), *raw-state*) ∈
set-dist

(execute (checked-staged-query-prefix-receive-with-state A i)
adversary-initial-state)

and *consistent*:

partial-query-openings-consistent trace-openings composition-openings
(staged-alphas data) (index (to-nat raw))

shows

checked-staged-security-with-actual-alpha-prefix-candidate-opening-query-hit A
((replicate rounds []) [i := trace-openings])

((*replicate rounds* []) [*i* := *composition-openings*])
 (*staged-alphas data*) *i*
 (*Some* ((((*alpha-prefix*, *alpha-prefix-state*), *data*), *attacker-state*), *result*),
 final-state))
 by (*rule*
 checked-staged-security-with-actual-alpha-prefix-candidate-opening-query-hitI-from-raw
 [*OF refl i-bound builder prefix-receive consistent*])

definition *checked-staged-security-with-actual-alpha-prefix-candidate-opening-query-hit-from-prefix*
where

checked-staged-security-with-actual-alpha-prefix-candidate-opening-query-hit-from-prefix A
trace-openings composition-openings i out $\longleftrightarrow$
 (case out of
 None $\Rightarrow$ *False*
 | *Some* ((((*alpha-prefix*, *alpha-prefix-state*), *data*), *attacker-state*), *result*),
 final-state) $\Rightarrow$
 i < *rounds* $\wedge$
 ($\exists$ *prefix prefix-state raw raw-state*.
 Some (*data*, *attacker-state*) $\in$
 set-dist
 (*execute* (*checked-staged-transcript-program A*)
 adversary-initial-state) $\wedge$
 Some (((*prefix*, *prefix-state*), *raw*), *raw-state*) $\in$
 set-dist
 (*execute* (*checked-staged-query-prefix-receive-with-state A i*)
 adversary-initial-state) $\wedge$
 checked-staged-query-prefix-dynamic-index-hit
 (*staged-query-prefix-candidate-opening-query-target-from-prefix*
 trace-openings composition-openings i)
 (*Some* (((*prefix*, *prefix-state*), *raw*), *raw-state*))))))

definition *checked-staged-security-with-actual-alpha-prefix-candidate-pair-query-hit-from-prefix*
where

checked-staged-security-with-actual-alpha-prefix-candidate-pair-query-hit-from-prefix A
trace-table composition-table i out $\longleftrightarrow$
 (case out of
 None $\Rightarrow$ *False*
 | *Some* ((((*alpha-prefix*, *alpha-prefix-state*), *data*), *attacker-state*), *result*),
 final-state) $\Rightarrow$
 i < *rounds* $\wedge$
 ($\exists$ *prefix prefix-state raw raw-state*.
 Some (*data*, *attacker-state*) $\in$
 set-dist
 (*execute* (*checked-staged-transcript-program A*)
 adversary-initial-state) $\wedge$
 Some (((*prefix*, *prefix-state*), *raw*), *raw-state*) $\in$
 set-dist
 (*execute* (*checked-staged-query-prefix-receive-with-state A i*)
 adversary-initial-state) $\wedge$

checked-staged-query-prefix-dynamic-index-hit
 (staged-query-prefix-candidate-pair-query-target-from-prefix
 trace-table composition-table)
 (Some (((prefix, prefix-state), raw), raw-state)))

lemma *checked-staged-security-with-actual-alpha-prefix-candidate-opening-query-hit-from-prefixI*:
assumes *i-bound*: $i < \text{rounds}$
and *builder*:
 Some (data, attacker-state) $\in$
 set-dist
 (execute (checked-staged-transcript-program A)
 adversary-initial-state)
and *prefix-receive*:
 Some (((prefix, prefix-state), raw), raw-state) $\in$
 set-dist
 (execute (checked-staged-query-prefix-receive-with-state A i)
 adversary-initial-state)
and *target-hit*:
checked-staged-query-prefix-dynamic-index-hit
 (staged-query-prefix-candidate-opening-query-target-from-prefix
 trace-openings composition-openings i)
 (Some (((prefix, prefix-state), raw), raw-state))
shows
checked-staged-security-with-actual-alpha-prefix-candidate-opening-query-hit-from-prefix A
trace-openings composition-openings i
 (Some (((((alpha-prefix, alpha-prefix-state), data), attacker-state), result),
 final-state))

proof –
have *body*:
 $\exists$ prefix prefix-state raw raw-state.
 Some (data, attacker-state) $\in$
 set-dist
 (execute (checked-staged-transcript-program A)
 adversary-initial-state) $\wedge$
 Some (((prefix, prefix-state), raw), raw-state) $\in$
 set-dist
 (execute (checked-staged-query-prefix-receive-with-state A i)
 adversary-initial-state) $\wedge$
checked-staged-query-prefix-dynamic-index-hit
 (staged-query-prefix-candidate-opening-query-target-from-prefix
 trace-openings composition-openings i)
 (Some (((prefix, prefix-state), raw), raw-state))
by (intro exI[of - prefix] exI[of - prefix-state]
 exI[of - raw] exI[of - raw-state])
 (use builder prefix-receive target-hit in simp)
show ?thesis
unfolding
checked-staged-security-with-actual-alpha-prefix-candidate-opening-query-hit-from-prefix-def
using *i-bound body* **by** simp

qed

lemma *checked-staged-security-with-actual-alpha-prefix-candidate-opening-query-hit-from-prefixE*:

assumes *hit*:

checked-staged-security-with-actual-alpha-prefix-candidate-opening-query-hit-from-prefix A
trace-openings composition-openings i
(*Some* (((*alpha-prefix*, *alpha-prefix-state*), *data*), *attacker-state*),
result), *final-state*))

obtains *prefix prefix-state raw raw-state* **where**

i < *rounds*

Some (*data*, *attacker-state*) ∈

set-dist

(*execute* (*checked-staged-transcript-program A*)
adversary-initial-state)

Some (((*prefix*, *prefix-state*), *raw*), *raw-state*) ∈

set-dist

(*execute* (*checked-staged-query-prefix-receive-with-state A i*)
adversary-initial-state)

checked-staged-query-prefix-dynamic-index-hit

(*staged-query-prefix-candidate-opening-query-target-from-prefix*
trace-openings composition-openings i)

(*Some* (((*prefix*, *prefix-state*), *raw*), *raw-state*))

using *hit*

proof –

have *body*:

i < *rounds* ∧

(∃ *prefix prefix-state raw raw-state*.

Some (*data*, *attacker-state*) ∈

set-dist

(*execute* (*checked-staged-transcript-program A*)
adversary-initial-state) ∧

Some (((*prefix*, *prefix-state*), *raw*), *raw-state*) ∈

set-dist

(*execute* (*checked-staged-query-prefix-receive-with-state A i*)
adversary-initial-state) ∧

checked-staged-query-prefix-dynamic-index-hit

(*staged-query-prefix-candidate-opening-query-target-from-prefix*
trace-openings composition-openings i)

(*Some* (((*prefix*, *prefix-state*), *raw*), *raw-state*)))

using *hit*

unfolding

checked-staged-security-with-actual-alpha-prefix-candidate-opening-query-hit-from-prefix-def

by *simp*

then obtain *prefix prefix-state raw raw-state* **where**

i-bound: *i* < *rounds*

and *builder*:

Some (*data*, *attacker-state*) ∈

set-dist

(*execute* (*checked-staged-transcript-program A*)

adversary-initial-state)
and *prefix-receive*:
Some (((prefix, prefix-state), raw), raw-state) ∈
set-dist
(execute (checked-staged-query-prefix-receive-with-state A i)
adversary-initial-state)
and *target-hit*:
checked-staged-query-prefix-dynamic-index-hit
(staged-query-prefix-candidate-opening-query-target-from-prefix
trace-openings composition-openings i)
(Some (((prefix, prefix-state), raw), raw-state))
by *blast*
show *?thesis*
by *(rule that[OF i-bound builder prefix-receive target-hit])*
qed

lemma *checked-staged-security-with-actual-alpha-prefix-candidate-pair-query-hit-from-prefixI*:
assumes *i-bound: i < rounds*
and *builder*:
Some (data, attacker-state) ∈
set-dist
(execute (checked-staged-transcript-program A)
adversary-initial-state)
and *prefix-receive*:
Some (((prefix, prefix-state), raw), raw-state) ∈
set-dist
(execute (checked-staged-query-prefix-receive-with-state A i)
adversary-initial-state)
and *target-hit*:
checked-staged-query-prefix-dynamic-index-hit
(staged-query-prefix-candidate-pair-query-target-from-prefix
trace-table composition-table)
(Some (((prefix, prefix-state), raw), raw-state))
shows
checked-staged-security-with-actual-alpha-prefix-candidate-pair-query-hit-from-prefix A
trace-table composition-table i
(Some ((((((alpha-prefix, alpha-prefix-state), data), attacker-state), result),
final-state)))
proof –
have *body*:
 $\exists$ *prefix prefix-state raw raw-state.*
Some (data, attacker-state) ∈
set-dist
(execute (checked-staged-transcript-program A)
adversary-initial-state) ∧
Some (((prefix, prefix-state), raw), raw-state) ∈
set-dist
(execute (checked-staged-query-prefix-receive-with-state A i)
adversary-initial-state) ∧

```

checked-staged-query-prefix-dynamic-index-hit
  (staged-query-prefix-candidate-pair-query-target-from-prefix
   trace-table composition-table)
  (Some (((prefix, prefix-state), raw), raw-state))
by (intro exI[of - prefix] exI[of - prefix-state]
    exI[of - raw] exI[of - raw-state])
  (use builder prefix-receive target-hit in simp)
show ?thesis
unfolding
  checked-staged-security-with-actual-alpha-prefix-candidate-pair-query-hit-from-prefix-def
using i-bound body by simp
qed

```

lemma *checked-staged-security-with-actual-alpha-prefix-candidate-pair-query-hit-from-prefixE*:

assumes *hit*:

```

checked-staged-security-with-actual-alpha-prefix-candidate-pair-query-hit-from-prefix A
trace-table composition-table i
(Some (((alpha-prefix, alpha-prefix-state), data), attacker-state),
 result), final-state))

```

obtains *prefix prefix-state raw raw-state* **where**

i < *rounds*

Some (*data*, *attacker-state*) ∈

set-dist

```

(execute (checked-staged-transcript-program A)
 adversary-initial-state)

```

Some (((*prefix*, *prefix-state*), *raw*), *raw-state*) ∈

set-dist

```

(execute (checked-staged-query-prefix-receive-with-state A i)
 adversary-initial-state)

```

checked-staged-query-prefix-dynamic-index-hit

```

(staged-query-prefix-candidate-pair-query-target-from-prefix
 trace-table composition-table)

```

(*Some* (((*prefix*, *prefix-state*), *raw*), *raw-state*))

using *hit*

proof –

have *body*:

i < *rounds* ∧

(∃ *prefix prefix-state raw raw-state*.

Some (*data*, *attacker-state*) ∈

set-dist

```

(execute (checked-staged-transcript-program A)
 adversary-initial-state) ∧

```

Some (((*prefix*, *prefix-state*), *raw*), *raw-state*) ∈

set-dist

```

(execute (checked-staged-query-prefix-receive-with-state A i)
 adversary-initial-state) ∧

```

checked-staged-query-prefix-dynamic-index-hit

```

(staged-query-prefix-candidate-pair-query-target-from-prefix
 trace-table composition-table)

```

```

      (Some (((prefix, prefix-state), raw), raw-state)))
using hit
unfolding
  checked-staged-security-with-actual-alpha-prefix-candidate-pair-query-hit-from-prefix-def
by simp
then obtain prefix prefix-state raw raw-state where
  i-bound:  $i < \text{rounds}$ 
and builder:
  Some (data, attacker-state)  $\in$ 
    set-dist
      (execute (checked-staged-transcript-program A)
        adversary-initial-state)
and prefix-receive:
  Some (((prefix, prefix-state), raw), raw-state)  $\in$ 
    set-dist
      (execute (checked-staged-query-prefix-receive-with-state A i)
        adversary-initial-state)
and target-hit:
  checked-staged-query-prefix-dynamic-index-hit
    (staged-query-prefix-candidate-pair-query-target-from-prefix
      trace-table composition-table)
    (Some (((prefix, prefix-state), raw), raw-state))
by blast
show ?thesis
by (rule that[OF i-bound builder prefix-receive target-hit])
qed

```

lemma checked-staged-security-with-actual-alpha-prefix-candidate-opening-query-hit-from-prefix-imp-candidate-pair-q

```

assumes hit:
  checked-staged-security-with-actual-alpha-prefix-candidate-opening-query-hit-from-prefix A
  trace-openings composition-openings i
  (Some (((((alpha-prefix, alpha-prefix-state), data), attacker-state),
    result), final-state))
and trace-candidate:
  partial-trace-table-candidate trace-table trace-openings
and comp-candidate:
  partial-composition-table-candidate composition-table
  composition-openings
and trace-low: trace-table-low-degree trace-table
and comp-low: composition-table-low-degree maxDegree composition-table
and not-all:
   $\bigwedge \text{prefix prefix-state.}$ 
  Some ((prefix, prefix-state), prefix-state)  $\in$ 
    set-dist
      (execute (checked-staged-query-prefix-with-state A i)
        adversary-initial-state)  $\implies$ 
   $\neg$  all-queries-consistent trace-table composition-table
    (sqp-alphas prefix)
shows

```

checked-staged-security-with-actual-alpha-prefix-candidate-pair-query-hit-from-prefix A
trace-table composition-table i
(Some (((((alpha-prefix, alpha-prefix-state), data), attacker-state),
result), final-state)))

proof –

from *checked-staged-security-with-actual-alpha-prefix-candidate-opening-query-hit-from-prefixE*
[OF hit]

obtain *prefix prefix-state raw raw-state* **where**
i-bound: i < rounds
and *builder:*
Some (data, attacker-state) ∈
set-dist
(execute (checked-staged-transcript-program A)
adversary-initial-state)
and *prefix-receive:*
Some (((prefix, prefix-state), raw), raw-state) ∈
set-dist
(execute (checked-staged-query-prefix-receive-with-state A i)
adversary-initial-state)
and *target-hit:*
checked-staged-query-prefix-dynamic-index-hit
(staged-query-prefix-candidate-opening-query-target-from-prefix
trace-openings composition-openings i)
(Some (((prefix, prefix-state), raw), raw-state)))
by *blast*

have *prefix-support:*
Some ((prefix, prefix-state), prefix-state) ∈
set-dist
(execute (checked-staged-query-prefix-with-state A i)
adversary-initial-state)
by *(rule checked-staged-query-prefix-receive-with-state-prefix-support*
[OF prefix-receive])

have *pair-target-hit:*
checked-staged-query-prefix-dynamic-index-hit
(staged-query-prefix-candidate-pair-query-target-from-prefix trace-table
composition-table)
(Some (((prefix, prefix-state), raw), raw-state)))

proof –

have *subset:*
staged-query-prefix-candidate-opening-query-target-from-prefix
trace-openings composition-openings i prefix prefix-state ⊆
staged-query-prefix-candidate-pair-query-target-from-prefix trace-table
composition-table prefix prefix-state
by *(rule*
staged-query-prefix-candidate-opening-query-target-from-prefix-subset-candidate-pair-from-prefix
[OF trace-candidate comp-candidate trace-low comp-low
not-all[OF prefix-support]])

show *?thesis*
using *target-hit subset*

```

    unfolding checked-staged-query-prefix-dynamic-index-hit-def by auto
qed
show ?thesis
  by (rule
    checked-staged-security-with-actual-alpha-prefix-candidate-pair-query-hit-from-prefixI
    [OF i-bound builder prefix-receive pair-target-hit])
qed

lemma checked-staged-query-prefix-candidate-opening-target-from-prefix-single-round-hitI:
  assumes i-bound:  $i < \text{rounds}$ 
  and consistent:
    partial-query-openings-consistent trace-openings composition-openings
    (sqp-alphas prefix) (index (to-nat raw))
  shows
    checked-staged-query-prefix-dynamic-index-hit
    (staged-query-prefix-candidate-opening-query-target-from-prefix
      ((replicate rounds []) [i := trace-openings])
      ((replicate rounds []) [i := composition-openings])
      i)
    (Some (((prefix, prefix-state), raw), raw-state))
proof -
  have index (to-nat raw)  $\in$ 
    staged-query-prefix-candidate-opening-query-target-from-prefix
    ((replicate rounds []) [i := trace-openings])
    ((replicate rounds []) [i := composition-openings])
    i prefix prefix-state
  unfolding staged-query-prefix-candidate-opening-query-target-from-prefix-def
  by (rule partial-query-success-indices-atI,
    rule partial-query-round-consistent-single-roundI
      [OF i-bound consistent])
  then show ?thesis
    unfolding checked-staged-query-prefix-dynamic-index-hit-def by simp
qed

lemma checked-staged-security-with-actual-alpha-prefix-candidate-opening-query-hit-from-prefixI-from-prefix-receive:
  assumes i-bound:  $i < \text{rounds}$ 
  and builder:
    Some (data, attacker-state)  $\in$ 
      set-dist
      (execute (checked-staged-transcript-program A)
        adversary-initial-state)
  and prefix-receive:
    Some (((prefix, prefix-state), raw), raw-state)  $\in$ 
      set-dist
      (execute (checked-staged-query-prefix-receive-with-state A i)
        adversary-initial-state)
  and alphas-eq: sqp-alphas prefix = staged-alphas data
  and consistent:
    partial-query-openings-consistent trace-openings composition-openings

```

```

      (staged-alphas data) (index (to-nat raw))
shows
  checked-staged-security-with-actual-alpha-prefix-candidate-opening-query-hit-from-prefix A
    ((replicate rounds []) [i := trace-openings])
    ((replicate rounds []) [i := composition-openings])
    i
    (Some (((alpha-prefix, alpha-prefix-state), data), attacker-state), result),
    final-state))
proof —
  have consistent-prefix:
    partial-query-openings-consistent trace-openings composition-openings
      (sqp-alphas prefix) (index (to-nat raw))
  using consistent alphas-eq by simp
  have target-hit:
    checked-staged-query-prefix-dynamic-index-hit
      (staged-query-prefix-candidate-opening-query-target-from-prefix
        ((replicate rounds []) [i := trace-openings])
        ((replicate rounds []) [i := composition-openings])
        i)
      (Some ((prefix, prefix-state), raw), raw-state))
  by (rule
    checked-staged-query-prefix-candidate-opening-target-from-prefix-single-round-hitI
    [OF i-bound consistent-prefix])
  show ?thesis
  by (rule
    checked-staged-security-with-actual-alpha-prefix-candidate-opening-query-hit-from-prefixI
    [OF i-bound builder prefix-receive target-hit])
qed

lemma checked-staged-query-prefix-candidate-pair-target-hitI:
assumes hit:
  index (to-nat raw) ∈
    query-sampling-success-space trace-table composition-table as
shows
  checked-staged-query-prefix-dynamic-index-hit
    (staged-query-prefix-candidate-pair-query-target trace-table
      composition-table as)
    (Some (((prefix, prefix-state), raw), raw-state))
using hit
unfolding checked-staged-query-prefix-dynamic-index-hit-def
  staged-query-prefix-candidate-pair-query-target-def
by simp

lemma checked-staged-security-with-actual-alpha-prefix-candidate-pair-query-hitI:
assumes i-bound: i < rounds
and builder:
  Some (data, attacker-state) ∈
    set-dist
    (execute (checked-staged-transcript-program A)

```


adversary-initial-state)
and *prefix-receive*:
Some (((prefix, prefix-state), raw), raw-state) ∈
set-dist
(execute (checked-staged-query-prefix-receive-with-state A i)
adversary-initial-state)
and *target-hit*:
checked-staged-query-prefix-dynamic-index-hit
(staged-query-prefix-candidate-pair-query-target trace-table
composition-table (staged-alphas data))
(Some (((prefix, prefix-state), raw), raw-state))
shows
checked-staged-security-with-actual-alpha-prefix-candidate-pair-query-hit A
trace-table composition-table
(Some (((((alpha-prefix, alpha-prefix-state), data), attacker-state), result),
final-state))
proof –
have *body*:
 $\exists i$ *prefix prefix-state raw raw-state.*
i < rounds ∧
Some (data, attacker-state) ∈
set-dist
(execute (checked-staged-transcript-program A)
adversary-initial-state) ∧
Some (((prefix, prefix-state), raw), raw-state) ∈
set-dist
(execute (checked-staged-query-prefix-receive-with-state A i)
adversary-initial-state) ∧
checked-staged-query-prefix-dynamic-index-hit
(staged-query-prefix-candidate-pair-query-target trace-table
composition-table (staged-alphas data))
(Some (((prefix, prefix-state), raw), raw-state))
by (*intro exI[of - i] exI[of - prefix] exI[of - prefix-state]*
exI[of - raw] exI[of - raw-state])
(use i-bound builder prefix-receive target-hit in simp)
show *?thesis*
unfolding
checked-staged-security-with-actual-alpha-prefix-candidate-pair-query-hit-def
using *body by simp*
qed

lemma *checked-staged-security-with-actual-alpha-prefix-candidate-opening-query-hit-imp-candidate-pair-query-hit*:
assumes *hit*:
checked-staged-security-with-actual-alpha-prefix-candidate-opening-query-hit A
trace-openings composition-openings as i
(Some (((((alpha-prefix, alpha-prefix-state), data), attacker-state),
result), final-state))
and *trace-candidate*:
partial-trace-table-candidate trace-table trace-openings

and *comp-candidate*:
partial-composition-table-candidate composition-table
composition-openings
and *trace-low*: *trace-table-low-degree trace-table*
and *comp-low*: *composition-table-low-degree maxDegree composition-table*
and *not-all*:
 $\neg$ *all-queries-consistent trace-table composition-table as*
shows
checked-staged-security-with-actual-alpha-prefix-candidate-pair-query-hit A
trace-table composition-table
(Some (((alpha-prefix, alpha-prefix-state), data), attacker-state),
result), final-state))
proof –
obtain *prefix prefix-state raw raw-state* **where**
as-eq: *as = staged-alphas data*
and *i-bound*: *i < rounds*
and *builder*:
Some (data, attacker-state) ∈
set-dist
(execute (checked-staged-transcript-program A)
adversary-initial-state)
and *prefix-receive*:
Some (((prefix, prefix-state), raw), raw-state) ∈
set-dist
(execute (checked-staged-query-prefix-receive-with-state A i)
adversary-initial-state)
and *target-hit*:
checked-staged-query-prefix-dynamic-index-hit
(staged-query-prefix-candidate-opening-query-target
trace-openings composition-openings as i)
(Some (((prefix, prefix-state), raw), raw-state))
using *hit*
unfolding
checked-staged-security-with-actual-alpha-prefix-candidate-opening-query-hit-def
by *auto*
have *pair-target-hit*:
checked-staged-query-prefix-dynamic-index-hit
(staged-query-prefix-candidate-pair-query-target trace-table
composition-table as)
(Some (((prefix, prefix-state), raw), raw-state))
by (rule *checked-staged-query-prefix-dynamic-index-hit-mono*
 $[OF - target-hit]$)
(rule
staged-query-prefix-candidate-opening-query-target-subset-candidate-pair
 $[OF trace-candidate comp-candidate trace-low comp-low not-all]$)
have *pair-target-hit'*:
checked-staged-query-prefix-dynamic-index-hit
(staged-query-prefix-candidate-pair-query-target trace-table
composition-table (staged-alphas data))

```

    (Some (((prefix, prefix-state), raw), raw-state))
  using pair-target-hit as-eq by simp
show ?thesis
  by (rule checked-staged-security-with-actual-alpha-prefix-candidate-pair-query-hitI
      [OF i-bound builder prefix-receive pair-target-hit])
qed

lemma checked-staged-security-with-actual-alpha-prefix-candidate-pair-query-hitI-from-raw:
  assumes i-bound: i < rounds
  and builder:
    Some (data, attacker-state) ∈
      set-dist
        (execute (checked-staged-transcript-program A)
          adversary-initial-state)
  and prefix-receive:
    Some (((prefix, prefix-state), raw), raw-state) ∈
      set-dist
        (execute (checked-staged-query-prefix-receive-with-state A i)
          adversary-initial-state)
  and target-raw:
    index (to-nat raw) ∈
      query-sampling-success-space trace-table composition-table
        (staged-alphas data)
  shows
    checked-staged-security-with-actual-alpha-prefix-candidate-pair-query-hit A
      trace-table composition-table
      (Some (((((alpha-prefix, alpha-prefix-state), data), attacker-state), result),
        final-state))
proof -
  have target-hit:
    checked-staged-query-prefix-dynamic-index-hit
      (staged-query-prefix-candidate-pair-query-target trace-table
        composition-table (staged-alphas data))
      (Some (((prefix, prefix-state), raw), raw-state))
  by (rule checked-staged-query-prefix-candidate-pair-target-hitI
      [OF target-raw])
  show ?thesis
  by (rule
      checked-staged-security-with-actual-alpha-prefix-candidate-pair-query-hitI
      [OF i-bound builder prefix-receive target-hit])
qed

lemma checked-staged-security-with-actual-alpha-prefix-candidate-pair-query-hitE:
  assumes hit:
    checked-staged-security-with-actual-alpha-prefix-candidate-pair-query-hit A
      trace-table composition-table
      (Some (((((alpha-prefix, alpha-prefix-state), data), attacker-state), result),
        final-state))
  obtains i prefix prefix-state raw raw-state where

```

```

i < rounds
Some (data, attacker-state) ∈
  set-dist
    (execute (checked-staged-transcript-program A)
      adversary-initial-state)
Some (((prefix, prefix-state), raw), raw-state) ∈
  set-dist
    (execute (checked-staged-query-prefix-receive-with-state A i)
      adversary-initial-state)
checked-staged-query-prefix-dynamic-index-hit
  (staged-query-prefix-candidate-pair-query-target trace-table
    composition-table (staged-alphas data))
  (Some (((prefix, prefix-state), raw), raw-state))
using hit
proof –
have body:
  ∃ i prefix prefix-state raw raw-state.
  i < rounds ∧
  Some (data, attacker-state) ∈
    set-dist
      (execute (checked-staged-transcript-program A)
        adversary-initial-state) ∧
  Some (((prefix, prefix-state), raw), raw-state) ∈
    set-dist
      (execute (checked-staged-query-prefix-receive-with-state A i)
        adversary-initial-state) ∧
  checked-staged-query-prefix-dynamic-index-hit
    (staged-query-prefix-candidate-pair-query-target trace-table
      composition-table (staged-alphas data))
    (Some (((prefix, prefix-state), raw), raw-state))
using hit
unfolding
  checked-staged-security-with-actual-alpha-prefix-candidate-pair-query-hit-def
by simp
then obtain i prefix prefix-state raw raw-state where
  i-bound: i < rounds
and builder:
  Some (data, attacker-state) ∈
    set-dist
      (execute (checked-staged-transcript-program A)
        adversary-initial-state)
and prefix-receive:
  Some (((prefix, prefix-state), raw), raw-state) ∈
    set-dist
      (execute (checked-staged-query-prefix-receive-with-state A i)
        adversary-initial-state)
and target-hit:
  checked-staged-query-prefix-dynamic-index-hit
    (staged-query-prefix-candidate-pair-query-target trace-table

```

```

      composition-table (staged-alphas data))
    (Some (((prefix, prefix-state), raw), raw-state))
  by blast
show ?thesis
  by (rule that[OF i-bound builder prefix-receive target-hit])
qed

lemma checked-staged-security-with-actual-alpha-prefix-candidate-pair-query-hit-rawE:
assumes hit:
  checked-staged-security-with-actual-alpha-prefix-candidate-pair-query-hit A
  trace-table composition-table
  (Some (((((alpha-prefix, alpha-prefix-state), data), attacker-state), result),
    final-state))
obtains i prefix prefix-state raw raw-state where
  i < rounds
  Some (data, attacker-state) ∈
    set-dist
      (execute (checked-staged-transcript-program A)
        adversary-initial-state)
  Some (((prefix, prefix-state), raw), raw-state) ∈
    set-dist
      (execute (checked-staged-query-prefix-receive-with-state A i)
        adversary-initial-state)
  index (to-nat raw) ∈
    query-sampling-success-space trace-table composition-table
      (staged-alphas data)
proof –
from checked-staged-security-with-actual-alpha-prefix-candidate-pair-query-hitE
  [OF hit]
obtain i prefix prefix-state raw raw-state where
  i-bound: i < rounds
and builder:
  Some (data, attacker-state) ∈
    set-dist
      (execute (checked-staged-transcript-program A)
        adversary-initial-state)
and prefix-receive:
  Some (((prefix, prefix-state), raw), raw-state) ∈
    set-dist
      (execute (checked-staged-query-prefix-receive-with-state A i)
        adversary-initial-state)
and target-hit:
  checked-staged-query-prefix-dynamic-index-hit
    (staged-query-prefix-candidate-pair-query-target trace-table
      composition-table (staged-alphas data))
    (Some (((prefix, prefix-state), raw), raw-state))
  by blast
have raw-hit:
  index (to-nat raw) ∈

```

```

    query-sampling-success-space trace-table composition-table
    (staged-alphas data)
using target-hit
unfolding checked-staged-query-prefix-dynamic-index-hit-def
    staged-query-prefix-candidate-pair-query-target-def
by simp
show ?thesis
    by (rule that[OF i-bound builder prefix-receive raw-hit])
qed

```

lemma staged-query-prefix-candidate-pair-query-target-fraction-bound:

assumes trace-low: trace-table-low-degree trace-table
and comp-low: composition-table-low-degree maxDegree composition-table
and not-all:
 $\neg$ all-queries-consistent trace-table composition-table as

shows
 $\text{nnreal } (\text{card } (\text{staged-query-prefix-candidate-pair-query-target trace-table composition-table as prefix prefix-state})) / \text{nnreal } (\text{card query-sample-space}) \leq \text{query-error-bound}$

unfolding staged-query-prefix-candidate-pair-query-target-def
by (rule query-sampling-success-space-fraction-bound-query-sample-space)

lemma staged-query-prefix-candidate-opening-query-target-fraction-bound:

assumes trace-candidate:
 partial-trace-table-candidate trace-table trace-openings
and comp-candidate:
 partial-composition-table-candidate composition-table
 composition-openings
and trace-low: trace-table-low-degree trace-table
and comp-low: composition-table-low-degree maxDegree composition-table
and not-all:
 $\neg$ all-queries-consistent trace-table composition-table as

shows
 $\text{nnreal } (\text{card } (\text{staged-query-prefix-candidate-opening-query-target trace-openings composition-openings as } i \text{ prefix prefix-state})) / \text{nnreal } (\text{card query-sample-space}) \leq \text{query-error-bound}$

unfolding staged-query-prefix-candidate-opening-query-target-def
by (rule partial-query-success-indices-at-fraction-bound-if-candidate-low-degree [OF trace-candidate comp-candidate trace-low comp-low not-all])

lemma staged-query-prefix-candidate-opening-query-target-from-prefix-fraction-bound:

assumes trace-candidate:
 partial-trace-table-candidate trace-table trace-openings
and comp-candidate:
 partial-composition-table-candidate composition-table

composition-openings
and *trace-low: trace-table-low-degree trace-table*
and *comp-low: composition-table-low-degree maxDegree composition-table*
and *not-all:*
 $\neg$ *all-queries-consistent trace-table composition-table*
(sqp-alphas prefix)
shows
nnreal
(card
(staged-query-prefix-candidate-opening-query-target-from-prefix
trace-openings composition-openings i prefix prefix-state)) /
nnreal (card query-sample-space) $\leq$ query-error-bound
unfolding *staged-query-prefix-candidate-opening-query-target-from-prefix-def*
by *(rule partial-query-success-indices-at-fraction-bound-if-candidate-low-degree*
[OF trace-candidate comp-candidate trace-low comp-low not-all])

lemma *staged-query-prefix-candidate-pair-query-target-from-prefix-fraction-bound:*
assumes *trace-low: trace-table-low-degree trace-table*
and *comp-low: composition-table-low-degree maxDegree composition-table*
and *not-all:*
 $\neg$ *all-queries-consistent trace-table composition-table*
(sqp-alphas prefix)
shows
nnreal
(card
(staged-query-prefix-candidate-pair-query-target-from-prefix
trace-table composition-table prefix prefix-state)) /
nnreal (card query-sample-space) $\leq$ query-error-bound
unfolding *staged-query-prefix-candidate-pair-query-target-from-prefix-def*
by *(rule query-sampling-success-space-fraction-bound-query-sample-space)*

lemma *checked-staged-query-prefix-candidate-opening-target-prehit-bound:*
assumes *wf: staged-budget-wellformed budgets*
and *controlled: staged-adversary-controlled budgets A*
and *i-bound: $i \leq$ rounds*
shows
wp-event (checked-staged-query-prefix-receive-with-state A i)
(checked-staged-query-prefix-dynamic-index-prehit
(staged-query-prefix-candidate-opening-query-target
trace-openings composition-openings as i))
adversary-initial-state $\leq$
staged-phase-relation-error size
(staged-query-search-queries budgets i + 1)
proof *(rule checked-staged-query-prefix-dynamic-index-prehit-bound-from-relation-program)*
show *HashMap adversary-initial-state = fmempty*
by *(simp add: adversary-initial-state-def)*
show *hash-relation-program*
(checked-staged-query-prefix-dynamic-index-prehit-relation A i
adversary-initial-state

(staged-query-prefix-candidate-opening-query-target
 trace-openings composition-openings as i))
 size (staged-query-search-queries budgets $i + 1$)
 (checked-staged-query-prefix-receive-with-state A i)
by (rule hash-relation-program-checked-staged-query-prefix-receive-with-state
 [OF wf controlled i-bound])
 (rule
 checked-staged-query-prefix-dynamic-index-prehit-relation-fiber-bound-size)
qed

lemma checked-staged-query-prefix-candidate-opening-target-hit-bound:

assumes raw-bound: query-index-raw-preimage-bound
and wf: staged-budget-wellformed budgets
and controlled: staged-adversary-controlled budgets A
and i-bound: $i \leq \text{rounds}$
and trace-candidate:
 partial-trace-table-candidate trace-table trace-openings
and comp-candidate:
 partial-composition-table-candidate composition-table
 composition-openings
and trace-low: trace-table-low-degree trace-table
and comp-low: composition-table-low-degree maxDegree composition-table
and not-all:
 $\neg$ all-queries-consistent trace-table composition-table as

shows

wp-event (checked-staged-query-prefix-receive-with-state A i)
 (checked-staged-query-prefix-dynamic-index-hit
 (staged-query-prefix-candidate-opening-query-target
 trace-openings composition-openings as i))
 adversary-initial-state $\leq$
 staged-phase-relation-error size
 (staged-query-search-queries budgets $i + 1$) + query-error-bound

proof –

have hit-bound:

wp-event (checked-staged-query-prefix-receive-with-state A i)
 (checked-staged-query-prefix-dynamic-index-hit
 (staged-query-prefix-candidate-opening-query-target
 trace-openings composition-openings as i))
 adversary-initial-state $\leq$
 wp-event (checked-staged-query-prefix-receive-with-state A i)
 (checked-staged-query-prefix-dynamic-index-prehit
 (staged-query-prefix-candidate-opening-query-target
 trace-openings composition-openings as i))
 adversary-initial-state + query-error-bound

proof (rule

checked-staged-query-prefix-dynamic-index-hit-bound-by-prehit-and-query-error
 [OF raw-bound])

fix prefix prefix-state

assume support:


```

Some ((prefix, prefix-state), prefix-state) ∈
  set-dist
  (execute (checked-staged-query-prefix-with-state A i)
    adversary-initial-state)
have subset:
  staged-query-prefix-candidate-opening-query-target trace-openings
    composition-openings as i prefix prefix-state ⊆
    query-sample-space
by (rule staged-query-prefix-candidate-opening-query-target-subset)
have frac:
  nnreal
  (card
    (staged-query-prefix-candidate-opening-query-target
      trace-openings composition-openings as i prefix prefix-state)) /
  nnreal (card query-sample-space) ≤ query-error-bound
by (rule staged-query-prefix-candidate-opening-query-target-fraction-bound
  [OF trace-candidate comp-candidate trace-low comp-low not-all])
show
  staged-query-prefix-candidate-opening-query-target trace-openings
    composition-openings as i prefix prefix-state ⊆
    query-sample-space ∧
  nnreal
  (card
    (staged-query-prefix-candidate-opening-query-target
      trace-openings composition-openings as i prefix prefix-state)) /
  nnreal (card query-sample-space) ≤ query-error-bound
using subset frac by blast
qed
have prehit-bound:
  wp-event (checked-staged-query-prefix-receive-with-state A i)
    (checked-staged-query-prefix-dynamic-index-prehit
      (staged-query-prefix-candidate-opening-query-target
        trace-openings composition-openings as i))
  adversary-initial-state ≤
  staged-phase-relation-error size
    (staged-query-search-queries budgets i + 1)
by (rule checked-staged-query-prefix-candidate-opening-target-prehit-bound
  [OF wf controlled i-bound])
have tail:
  wp-event (checked-staged-query-prefix-receive-with-state A i)
    (checked-staged-query-prefix-dynamic-index-prehit
      (staged-query-prefix-candidate-opening-query-target
        trace-openings composition-openings as i))
  adversary-initial-state + query-error-bound ≤
  staged-phase-relation-error size
    (staged-query-search-queries budgets i + 1) + query-error-bound
by (intro add-mono prehit-bound order-refl)
show ?thesis
by (rule order-trans[OF hit-bound tail])

```

qed

lemma *checked-staged-query-prefix-candidate-opening-target-from-prefix-prehit-bound:*

assumes *wf: staged-budget-wellformed budgets*

and *controlled: staged-adversary-controlled budgets A*

and *i-bound: $i \leq \text{rounds}$*

shows

wp-event (checked-staged-query-prefix-receive-with-state A i)
(checked-staged-query-prefix-dynamic-index-prehit
(staged-query-prefix-candidate-opening-query-target-from-prefix
trace-openings composition-openings i))

adversary-initial-state $\leq$

staged-phase-relation-error size

(staged-query-search-queries budgets i + 1)

proof (rule *checked-staged-query-prefix-dynamic-index-prehit-bound-from-relation-program*)

show *HashMap adversary-initial-state = fmempty*

by (simp add: *adversary-initial-state-def*)

show *hash-relation-program*

(checked-staged-query-prefix-dynamic-index-prehit-relation A i
adversary-initial-state

(staged-query-prefix-candidate-opening-query-target-from-prefix
trace-openings composition-openings i))

size (staged-query-search-queries budgets i + 1)

(checked-staged-query-prefix-receive-with-state A i)

by (rule *hash-relation-program-checked-staged-query-prefix-receive-with-state*
 $[OF\ wf\ controlled\ i-bound]$)

(rule

checked-staged-query-prefix-dynamic-index-prehit-relation-fiber-bound-size)

qed

lemma *checked-staged-query-prefix-candidate-opening-target-from-prefix-hit-bound:*

assumes *raw-bound: query-index-raw-preimage-bound*

and *wf: staged-budget-wellformed budgets*

and *controlled: staged-adversary-controlled budgets A*

and *i-bound: $i \leq \text{rounds}$*

and *trace-candidate:*

partial-trace-table-candidate trace-table trace-openings

and *comp-candidate:*

partial-composition-table-candidate composition-table

composition-openings

and *trace-low: trace-table-low-degree trace-table*

and *comp-low: composition-table-low-degree maxDegree composition-table*

and *not-all:*

$\bigwedge_{\text{prefix}} \text{prefix-state.}$

Some ((prefix, prefix-state), prefix-state) $\in$

set-dist

(execute (checked-staged-query-prefix-with-state A i)

adversary-initial-state) $\implies$

$\neg$ *all-queries-consistent trace-table composition-table*

(sqp-alphas prefix)
shows
 wp-event (checked-staged-query-prefix-receive-with-state A i)
 (checked-staged-query-prefix-dynamic-index-hit
 (staged-query-prefix-candidate-opening-query-target-from-prefix
 trace-openings composition-openings i))
 adversary-initial-state $\leq$
 staged-phase-relation-error size
 (staged-query-search-queries budgets i + 1) + query-error-bound
proof –
have hit-bound:
 wp-event (checked-staged-query-prefix-receive-with-state A i)
 (checked-staged-query-prefix-dynamic-index-hit
 (staged-query-prefix-candidate-opening-query-target-from-prefix
 trace-openings composition-openings i))
 adversary-initial-state $\leq$
 wp-event (checked-staged-query-prefix-receive-with-state A i)
 (checked-staged-query-prefix-dynamic-index-prehit
 (staged-query-prefix-candidate-opening-query-target-from-prefix
 trace-openings composition-openings i))
 adversary-initial-state + query-error-bound
proof (rule
 checked-staged-query-prefix-dynamic-index-hit-bound-by-prehit-and-query-error
 [OF raw-bound])
fix prefix prefix-state
assume support:
 Some ((prefix, prefix-state), prefix-state) $\in$
 set-dist
 (execute (checked-staged-query-prefix-with-state A i)
 adversary-initial-state)
have subset:
 staged-query-prefix-candidate-opening-query-target-from-prefix
 trace-openings composition-openings i prefix prefix-state $\subseteq$
 query-sample-space
by (rule staged-query-prefix-candidate-opening-query-target-from-prefix-subset)
have frac:
 nnreal
 (card
 (staged-query-prefix-candidate-opening-query-target-from-prefix
 trace-openings composition-openings i prefix prefix-state)) /
 nnreal (card query-sample-space) $\leq$ query-error-bound
by (rule
 staged-query-prefix-candidate-opening-query-target-from-prefix-fraction-bound
 [OF trace-candidate comp-candidate trace-low comp-low not-all[OF support]])
show
 staged-query-prefix-candidate-opening-query-target-from-prefix
 trace-openings composition-openings i prefix prefix-state $\subseteq$
 query-sample-space $\wedge$
 nnreal

```

    (card
      (staged-query-prefix-candidate-opening-query-target-from-prefix
        trace-openings composition-openings i prefix prefix-state)) /
    nnreal (card query-sample-space) ≤ query-error-bound
  using subset frac by blast
qed
have prehit-bound:
  wp-event (checked-staged-query-prefix-receive-with-state A i)
    (checked-staged-query-prefix-dynamic-index-prehit
      (staged-query-prefix-candidate-opening-query-target-from-prefix
        trace-openings composition-openings i))
  adversary-initial-state ≤
  staged-phase-relation-error size
  (staged-query-search-queries budgets i + 1)
by (rule
  checked-staged-query-prefix-candidate-opening-target-from-prefix-prehit-bound
  [OF wf controlled i-bound])
have tail:
  wp-event (checked-staged-query-prefix-receive-with-state A i)
    (checked-staged-query-prefix-dynamic-index-prehit
      (staged-query-prefix-candidate-opening-query-target-from-prefix
        trace-openings composition-openings i))
  adversary-initial-state + query-error-bound ≤
  staged-phase-relation-error size
  (staged-query-search-queries budgets i + 1) + query-error-bound
by (intro add-mono prehit-bound order-refl)
show ?thesis
by (rule order-trans[OF hit-bound tail])
qed

lemma checked-staged-query-prefix-candidate-pair-target-prehit-bound:
  assumes wf: staged-budget-wellformed budgets
  and controlled: staged-adversary-controlled budgets A
  and i-bound: i ≤ rounds
  shows
    wp-event (checked-staged-query-prefix-receive-with-state A i)
      (checked-staged-query-prefix-dynamic-index-prehit
        (staged-query-prefix-candidate-pair-query-target trace-table
          composition-table as))
    adversary-initial-state ≤
    staged-phase-relation-error size
    (staged-query-search-queries budgets i + 1)
proof (rule checked-staged-query-prefix-dynamic-index-prehit-bound-from-relation-program)
  show HashMap adversary-initial-state = fmempty
  by (simp add: adversary-initial-state-def)
  show hash-relation-program
  (checked-staged-query-prefix-dynamic-index-prehit-relation A i
    adversary-initial-state
    (staged-query-prefix-candidate-pair-query-target trace-table

```

composition-table as))
 size (staged-query-search-queries budgets $i + 1$)
 (checked-staged-query-prefix-receive-with-state $A\ i$)
by (rule hash-relation-program-checked-staged-query-prefix-receive-with-state
 [OF wf controlled i-bound])
 (rule
 checked-staged-query-prefix-dynamic-index-prehit-relation-fiber-bound-size)
qed

lemma checked-staged-query-prefix-candidate-pair-target-hit-bound:

assumes raw-bound: query-index-raw-preimage-bound
and wf: staged-budget-wellformed budgets
and controlled: staged-adversary-controlled budgets A
and i-bound: $i \leq \text{rounds}$
and trace-low: trace-table-low-degree trace-table
and comp-low: composition-table-low-degree maxDegree composition-table
and not-all:

$\neg$ all-queries-consistent trace-table composition-table as

shows

wp-event (checked-staged-query-prefix-receive-with-state $A\ i$)
 (checked-staged-query-prefix-dynamic-index-hit
 (staged-query-prefix-candidate-pair-query-target trace-table
 composition-table as))
 adversary-initial-state $\leq$
 staged-phase-relation-error size
 (staged-query-search-queries budgets $i + 1$) + query-error-bound

proof –

have hit-bound:

wp-event (checked-staged-query-prefix-receive-with-state $A\ i$)
 (checked-staged-query-prefix-dynamic-index-hit
 (staged-query-prefix-candidate-pair-query-target trace-table
 composition-table as))
 adversary-initial-state $\leq$
 wp-event (checked-staged-query-prefix-receive-with-state $A\ i$)
 (checked-staged-query-prefix-dynamic-index-prehit
 (staged-query-prefix-candidate-pair-query-target trace-table
 composition-table as))
 adversary-initial-state + query-error-bound

proof (rule

checked-staged-query-prefix-dynamic-index-hit-bound-by-prehit-and-query-error
 [OF raw-bound])

fix prefix prefix-state

assume support:

Some ((prefix, prefix-state), prefix-state) $\in$
 set-dist

(execute (checked-staged-query-prefix-with-state $A\ i$)
 adversary-initial-state)

have subset:

staged-query-prefix-candidate-pair-query-target trace-table

```

      composition-table as prefix prefix-state  $\subseteq$ 
      query-sample-space
    by (rule staged-query-prefix-candidate-pair-query-target-subset)
  have frac:
    nnreal
    (card
      (staged-query-prefix-candidate-pair-query-target trace-table
        composition-table as prefix prefix-state)) /
    nnreal (card query-sample-space)  $\leq$  query-error-bound
  by (rule staged-query-prefix-candidate-pair-query-target-fraction-bound
    [OF trace-low comp-low not-all])
  show
    staged-query-prefix-candidate-pair-query-target trace-table
      composition-table as prefix prefix-state  $\subseteq$ 
      query-sample-space  $\wedge$ 
    nnreal
    (card
      (staged-query-prefix-candidate-pair-query-target trace-table
        composition-table as prefix prefix-state)) /
    nnreal (card query-sample-space)  $\leq$  query-error-bound
  using subset frac by blast
qed
have prehit-bound:
  wp-event (checked-staged-query-prefix-receive-with-state A i)
    (checked-staged-query-prefix-dynamic-index-prehit
      (staged-query-prefix-candidate-pair-query-target trace-table
        composition-table as))
  adversary-initial-state  $\leq$ 
  staged-phase-relation-error size
    (staged-query-search-queries budgets i + 1)
  by (rule checked-staged-query-prefix-candidate-pair-target-prehit-bound
    [OF wf controlled i-bound])
have tail:
  wp-event (checked-staged-query-prefix-receive-with-state A i)
    (checked-staged-query-prefix-dynamic-index-prehit
      (staged-query-prefix-candidate-pair-query-target trace-table
        composition-table as))
  adversary-initial-state + query-error-bound  $\leq$ 
  staged-phase-relation-error size
    (staged-query-search-queries budgets i + 1) + query-error-bound
  by (intro add-mono prehit-bound order-refl)
show ?thesis
  by (rule order-trans[OF hit-bound tail])
qed

lemma checked-staged-query-prefix-candidate-pair-target-from-prefix-prehit-bound:
  assumes wf: staged-budget-wellformed budgets
  and controlled: staged-adversary-controlled budgets A
  and i-bound: i  $\leq$  rounds

```

shows
wp-event (*checked-staged-query-prefix-receive-with-state A i*)
 (*checked-staged-query-prefix-dynamic-index-prehit*
 (*staged-query-prefix-candidate-pair-query-target-from-prefix*
trace-table composition-table))
adversary-initial-state $\leq$
staged-phase-relation-error size
 (*staged-query-search-queries budgets i + 1*)
proof (*rule checked-staged-query-prefix-dynamic-index-prehit-bound-from-relation-program*)
show *HashMap adversary-initial-state = fmempty*
by (*simp add: adversary-initial-state-def*)
show *hash-relation-program*
 (*checked-staged-query-prefix-dynamic-index-prehit-relation A i*
adversary-initial-state
 (*staged-query-prefix-candidate-pair-query-target-from-prefix*
trace-table composition-table))
size (*staged-query-search-queries budgets i + 1*)
 (*checked-staged-query-prefix-receive-with-state A i*)
by (*rule hash-relation-program-checked-staged-query-prefix-receive-with-state*
 [*OF wf controlled i-bound*])
 (*rule*
checked-staged-query-prefix-dynamic-index-prehit-relation-fiber-bound-size)
qed

lemma *checked-staged-query-prefix-candidate-pair-target-from-prefix-hit-bound:*

assumes *raw-bound: query-index-raw-preimage-bound*
and *wf: staged-budget-wellformed budgets*
and *controlled: staged-adversary-controlled budgets A*
and *i-bound: i $\leq$ rounds*
and *trace-low: trace-table-low-degree trace-table*
and *comp-low: composition-table-low-degree maxDegree composition-table*
and *not-all:*
 $\bigwedge \text{prefix prefix-state.}$
Some ((prefix, prefix-state), prefix-state) $\in$
set-dist
 (*execute* (*checked-staged-query-prefix-with-state A i*)
adversary-initial-state) $\implies$
 $\neg$ *all-queries-consistent trace-table composition-table*
 (*sqp-alphas prefix*)

shows

wp-event (*checked-staged-query-prefix-receive-with-state A i*)
 (*checked-staged-query-prefix-dynamic-index-hit*
 (*staged-query-prefix-candidate-pair-query-target-from-prefix*
trace-table composition-table))
adversary-initial-state $\leq$
staged-phase-relation-error size
 (*staged-query-search-queries budgets i + 1*) + *query-error-bound*

proof —

have *hit-bound:*

```

wp-event (checked-staged-query-prefix-receive-with-state A i)
  (checked-staged-query-prefix-dynamic-index-hit
    (staged-query-prefix-candidate-pair-query-target-from-prefix
      trace-table composition-table))
adversary-initial-state ≤
wp-event (checked-staged-query-prefix-receive-with-state A i)
  (checked-staged-query-prefix-dynamic-index-prehit
    (staged-query-prefix-candidate-pair-query-target-from-prefix
      trace-table composition-table))
adversary-initial-state + query-error-bound
proof (rule
  checked-staged-query-prefix-dynamic-index-hit-bound-by-prehit-and-query-error
  [OF raw-bound])
fix prefix prefix-state
assume support:
  Some ((prefix, prefix-state), prefix-state) ∈
    set-dist
    (execute (checked-staged-query-prefix-with-state A i)
      adversary-initial-state)
have subset:
  staged-query-prefix-candidate-pair-query-target-from-prefix
    trace-table composition-table prefix prefix-state ⊆
    query-sample-space
by (rule staged-query-prefix-candidate-pair-query-target-from-prefix-subset)
have frac:
  nnreal
    (card
      (staged-query-prefix-candidate-pair-query-target-from-prefix
        trace-table composition-table prefix prefix-state)) /
    nnreal (card query-sample-space) ≤ query-error-bound
by (rule
  staged-query-prefix-candidate-pair-query-target-from-prefix-fraction-bound)
  (use trace-low comp-low not-all[OF support] in simp-all)
show
  staged-query-prefix-candidate-pair-query-target-from-prefix
    trace-table composition-table prefix prefix-state ⊆
    query-sample-space ∧
  nnreal
    (card
      (staged-query-prefix-candidate-pair-query-target-from-prefix
        trace-table composition-table prefix prefix-state)) /
    nnreal (card query-sample-space) ≤ query-error-bound
using subset frac by blast
qed
have prehit-bound:
  wp-event (checked-staged-query-prefix-receive-with-state A i)
    (checked-staged-query-prefix-dynamic-index-prehit
      (staged-query-prefix-candidate-pair-query-target-from-prefix
        trace-table composition-table))

```


$adversary-initial-state \leq$
 $staged-phase-relation-error\ size$
 $(staged-query-search-queries\ budgets\ i + 1)$
by (rule
 $checked-staged-query-prefix-candidate-pair-target-from-prefix-prehit-bound$
 $[OF\ wf\ controlled\ i-bound]$)
have *tail*:
 $wp-event\ (checked-staged-query-prefix-receive-with-state\ A\ i)$
 $(checked-staged-query-prefix-dynamic-index-prehit$
 $(staged-query-prefix-candidate-pair-query-target-from-prefix$
 $trace-table\ composition-table))$
 $adversary-initial-state + query-error-bound \leq$
 $staged-phase-relation-error\ size$
 $(staged-query-search-queries\ budgets\ i + 1) + query-error-bound$
by (intro *add-mono prehit-bound order-refl*)
show *?thesis*
by (rule *order-trans*[*OF hit-bound tail*])
qed

lemma *checked-staged-query-prefix-candidate-pair-target-from-prefix-hit-bound-from-budgets*:
assumes *wf*: *staged-budget-wellformed budgets*
and *controlled*: *staged-adversary-controlled budgets A*
and *i-bound*: $i < rounds$
and *trace-low*: *trace-table-low-degree trace-table*
and *comp-low*: *composition-table-low-degree maxDegree composition-table*
and *not-all*:
 $\bigwedge prefix\ prefix-state.$
 $Some\ ((prefix,\ prefix-state),\ prefix-state) \in$
 $set-dist$
 $(execute\ (checked-staged-query-prefix-with-state\ A\ i)$
 $adversary-initial-state) \implies$
 $\neg\ all-queries-consistent\ trace-table\ composition-table$
 $(sqp-alphas\ prefix)$
shows
 $wp-event\ (checked-staged-query-prefix-receive-with-state\ A\ i)$
 $(checked-staged-query-prefix-dynamic-index-hit$
 $(staged-query-prefix-candidate-pair-query-target-from-prefix$
 $trace-table\ composition-table))$
 $adversary-initial-state \leq$
 $staged-phase-relation-error\ size$
 $(staged-query-search-queries\ budgets\ i + 1) + query-error-bound$
proof –
have *raw-bound*: *query-index-raw-preimage-bound*
by (rule *query-index-raw-preimage-bound-from-sampler-wellformed*)
show *?thesis*
by (rule
 $checked-staged-query-prefix-candidate-pair-target-from-prefix-hit-bound$
 $[OF\ raw-bound\ wf\ controlled - trace-low\ comp-low\ not-all]$)
 $(use\ i-bound\ in\ simp)$

qed

end

end

theory *Staged-Security-Experiment-Composition-Query-Residual*

imports

Staged-Security-Experiment-Composition-Partial

Soundness-Staged-Partial-Query-Branch

Soundness-Staged-Candidate-Query

begin

Composition-specific residual split after the alpha-prequery accounting.

The broad branch-subtree witness event is not rare: honest verifier-side Merkle path re-computation can produce such witnesses. This layer therefore splits the actual remaining composition drift event itself into a part covered by the sampled partial-opening query event and a residual part that is still to be discharged by the Merkle/candidate proof.

context *soundness*

begin

The existing committed query-prefix targets are based on *query-sampling-success-space*. They are suitable for ordinary query inconsistency, but not directly for composition randomization witnesses: *composition-bad-context* already requires all sampled queries to be consistent. Hence the query-sampling success space of the actual composition witness is empty, and the composition-drift query case needs a narrower candidate-indexed target rather than the global partial-opening fixed cover.

lemma *composition-bad-context-query-sampling-success-space-empty:*

assumes

composition-bad-context s out trace-table composition-table as query-idxs f

shows *query-sampling-success-space trace-table composition-table as = {}*

proof –

have *all-queries: all-queries-consistent trace-table composition-table as*

using *assms* **unfolding** *composition-bad-context-def* **by** *blast*

then show *?thesis*

unfolding *query-sampling-success-space-def* **by** *simp*

qed

lemma *composition-randomization-bad-witness-query-sampling-success-space-empty:*

assumes *composition-randomization-bad s out*

obtains *trace-table composition-table as query-idxs f* **where**

composition-bad-context s out trace-table composition-table as query-idxs f

common-denominator-degree-bounds f as

random-combination-common-denominator-hides-violations f as

query-sampling-success-space trace-table composition-table as = {}

using *assms*

unfolding *composition-randomization-bad-def*

by (metis composition-bad-context-query-sampling-success-space-empty)

lemma *composition-trace-bad-alpha-spaceE*:

assumes *bad*: $as \in \text{composition-trace-bad-alpha-space trace-table}$

obtains *f* **where**

$f = \text{low-degree-trace-witness trace-table}$

$\text{degree } f < \text{clength}$

$\text{trace-table} = \text{map } (\text{poly } f) \text{ eval-domain}$

$\text{violated-constraints } f \neq \{\}$

$as \in \text{common-denominator-hiding-alpha-space } f$

$\text{common-denominator-degree-bounds } f \text{ as}$

$\text{trace-table-low-degree trace-table}$

proof –

let $?f = \text{low-degree-trace-witness trace-table}$

have *deg*: $\text{degree } ?f < \text{clength}$

and *trace-table*: $\text{trace-table} = \text{map } (\text{poly } ?f) \text{ eval-domain}$

and *violated*: $\text{violated-constraints } ?f \neq \{\}$

and *alpha*: $as \in \text{common-denominator-hiding-alpha-space } ?f$

and *degree-bounds*: $\text{common-denominator-degree-bounds } ?f \text{ as}$

using *bad*

unfolding *composition-trace-bad-alpha-space-def Let-def*

by (auto split: if-splits)

have *trace-low*: $\text{trace-table-low-degree trace-table}$

unfolding *trace-table-low-degree-def*

using *deg trace-table* **by** *blast*

show *?thesis*

by (rule that[*OF refl deg trace-table violated alpha degree-bounds trace-low*])

qed

lemma *composition-trace-bad-alpha-space-trace-low-degree*:

assumes $as \in \text{composition-trace-bad-alpha-space trace-table}$

shows $\text{trace-table-low-degree trace-table}$

using *assms* **by** (elim *composition-trace-bad-alpha-spaceE*)

definition *checked-staged-security-with-actual-alpha-prefix-composition-candidate-drift-query-hit*

$:: ((((((f \times 'f \text{ list} \times 'f \text{ list} \times 'f) \times$
 $\quad 'f \text{ protocol-channel}) \times 'f \text{ staged-proof-data}) \times$
 $\quad 'f \text{ protocol-channel}) \times \text{unit list}) \times$
 $\quad 'f \text{ protocol-channel}) \text{ option} \Rightarrow \text{bool}$

where

checked-staged-security-with-actual-alpha-prefix-composition-candidate-drift-query-hit

$\text{out} \longleftrightarrow$

checked-staged-security-with-actual-alpha-prefix-composition-candidate-drift

$\text{out} \wedge$

checked-staged-security-with-actual-alpha-prefix-query-partial-opening-hit

out

definition *checked-staged-security-with-actual-alpha-prefix-composition-candidate-drift-without-query-hit*

$$:: ((((((f \times 'f \text{ list} \times 'f \text{ list} \times 'f) \times$$

$$'f \text{ protocol-channel}) \times 'f \text{ staged-proof-data}) \times$$

$$'f \text{ protocol-channel}) \times \text{unit list}) \times$$

$$'f \text{ protocol-channel}) \text{ option} \Rightarrow \text{bool}$$

where

$$\text{checked-staged-security-with-actual-alpha-prefix-composition-candidate-drift-without-query-hit}$$

$$\text{out} \longleftrightarrow$$

$$\text{checked-staged-security-with-actual-alpha-prefix-composition-candidate-drift}$$

$$\text{out} \wedge$$

$$\neg \text{checked-staged-security-with-actual-alpha-prefix-query-partial-opening-hit}$$

$$\text{out}$$

lemma *checked-staged-security-with-actual-alpha-prefix-composition-candidate-drift-without-query-hit-imp-drift:*

assumes

$$\text{checked-staged-security-with-actual-alpha-prefix-composition-candidate-drift-without-query-hit}$$

$$\text{out}$$

shows

$$\text{checked-staged-security-with-actual-alpha-prefix-composition-candidate-drift}$$

$$\text{out}$$

using *assms*

unfolding

$$\text{checked-staged-security-with-actual-alpha-prefix-composition-candidate-drift-without-query-hit-def}$$

by *simp*

lemma *checked-staged-security-with-actual-alpha-prefix-composition-candidate-drift-without-query-hit-imp-not-query-*

assumes

$$\text{checked-staged-security-with-actual-alpha-prefix-composition-candidate-drift-without-query-hit}$$

$$\text{out}$$

shows

$$\neg \text{checked-staged-security-with-actual-alpha-prefix-query-partial-opening-hit}$$

$$\text{out}$$

using *assms*

unfolding

$$\text{checked-staged-security-with-actual-alpha-prefix-composition-candidate-drift-without-query-hit-def}$$

by *simp*

lemma *checked-staged-security-with-actual-alpha-prefix-composition-candidate-drift-without-query-hit-imp-randomiza-*

assumes

$$\text{checked-staged-security-with-actual-alpha-prefix-composition-candidate-drift-without-query-hit}$$

$$\text{out}$$

shows

$$\text{checked-staged-security-with-actual-alpha-prefix-verifier-event}$$

$$\text{composition-randomization-bad out}$$

using *assms*

unfolding

$$\text{checked-staged-security-with-actual-alpha-prefix-composition-candidate-drift-without-query-hit-def}$$

$$\text{checked-staged-security-with-actual-alpha-prefix-composition-candidate-drift-def}$$

by *simp*

lemma *checked-staged-security-with-actual-alpha-prefix-composition-candidate-drift-without-query-hit-imp-partial-head*
assumes
 checked-staged-security-with-actual-alpha-prefix-composition-candidate-drift-without-query-hit
 out
shows
 checked-staged-security-with-actual-alpha-prefix-partial-header-candidate-drift
 composition-trace-bad-alpha-space out
using *assms*
unfolding
 checked-staged-security-with-actual-alpha-prefix-composition-candidate-drift-without-query-hit-def
 checked-staged-security-with-actual-alpha-prefix-composition-candidate-drift-def
by *simp*

lemma *checked-staged-security-with-actual-alpha-prefix-composition-candidate-drift-without-query-hit-imp-partial-head*
assumes
 checked-staged-security-with-actual-alpha-prefix-composition-candidate-drift-without-query-hit
 out
shows
 checked-staged-security-with-actual-alpha-prefix-partial-header-fresh-hit
 composition-trace-bad-alpha-space out
using *assms*
unfolding
 checked-staged-security-with-actual-alpha-prefix-composition-candidate-drift-without-query-hit-def
 checked-staged-security-with-actual-alpha-prefix-composition-candidate-drift-def
 checked-staged-security-with-actual-alpha-prefix-partial-header-candidate-drift-def
by *simp*

lemma *checked-staged-security-with-actual-alpha-prefix-composition-candidate-drift-without-query-hit-imp-not-prefix*
assumes
 checked-staged-security-with-actual-alpha-prefix-composition-candidate-drift-without-query-hit
 out
shows
 $\neg$ *checked-staged-security-with-actual-alpha-prefix-bad-set-hit*
 composition-trace-bad-alpha-space out
using *assms*
unfolding
 checked-staged-security-with-actual-alpha-prefix-composition-candidate-drift-without-query-hit-def
 checked-staged-security-with-actual-alpha-prefix-composition-candidate-drift-def
 checked-staged-security-with-actual-alpha-prefix-partial-header-candidate-drift-def
by *simp*

lemma *checked-staged-security-with-actual-alpha-prefix-composition-candidate-drift-without-query-hitE*:
assumes *hit*:
 checked-staged-security-with-actual-alpha-prefix-composition-candidate-drift-without-query-hit
 (Some (((((prefix, prefix-state), data), attacker-state), result),
 final-state)))
obtains
 alpha-header-list-set-hit
 (verifier-state-from-adversary attacker-state

```

    (staged-proof-transcript data))
  (alpha-header-supported-partial-union-bad-sets
    (verifier-state-from-adversary attacker-state
      (staged-proof-transcript data))
    composition-trace-bad-alpha-space)
  (Some (result, final-state))
  ⊢ alpha-vector-prequeried-in-state prefix-state
    (staged-alphas data) (length spec)
  ⊢ checked-staged-security-with-actual-alpha-prefix-bad-set-hit
    composition-trace-bad-alpha-space
    (Some (((((prefix, prefix-state), data), attacker-state), result),
      final-state))
  ⊢ checked-staged-security-with-actual-alpha-prefix-query-partial-opening-hit
    (Some (((((prefix, prefix-state), data), attacker-state), result),
      final-state))
  checked-staged-security-with-actual-alpha-prefix-verifier-event
    composition-randomization-bad
    (Some (((((prefix, prefix-state), data), attacker-state), result),
      final-state))
using hit
unfolding
  checked-staged-security-with-actual-alpha-prefix-composition-candidate-drift-without-query-hit-def
  checked-staged-security-with-actual-alpha-prefix-composition-candidate-drift-def
  checked-staged-security-with-actual-alpha-prefix-partial-header-candidate-drift-def
  checked-staged-security-with-actual-alpha-prefix-partial-header-fresh-hit-def
  checked-staged-security-with-actual-alpha-prefix-partial-header-bad-set-hit-def
  Let-def
by auto

```

lemma *checked-staged-security-with-actual-alpha-prefix-composition-candidate-drift-without-query-hit-imp-branch-tree*

assumes *wf*: staged-budget-wellformed budgets

and *controlled*: staged-adversary-controlled budgets *A*

and *support*:

```

  out ∈
    set-dist
      (execute
        (checked-staged-security-experiment-with-actual-alpha-prefix-data-state
          A)
        adversary-initial-state)

```

and *hit*:

```

  checked-staged-security-with-actual-alpha-prefix-composition-candidate-drift-without-query-hit
  out

```

shows

```

  checked-staged-security-with-actual-alpha-prefix-branch-tree-output-hit
  out

```

by (rule

```

  checked-staged-security-with-actual-alpha-prefix-composition-candidate-drift-imp-branch-tree-on-support
  [OF wf controlled support])

```

(rule

checked-staged-security-with-actual-alpha-prefix-composition-candidate-drift-without-query-hit-imp-drift
 $[OF \text{ hit}]$)

lemma *checked-staged-security-with-actual-alpha-prefix-support-imp-data-state-support*:

assumes *support*:

Some (((((prefix, prefix-state), data), attacker-state), result),
 final-state) $\in$
set-dist
 (execute
 (checked-staged-security-experiment-with-actual-alpha-prefix-data-state
 A)
 adversary-initial-state)

shows

Some (((data, attacker-state), result), final-state) $\in$
set-dist
 (execute (checked-staged-security-experiment-with-data-state A)
 adversary-initial-state)

proof –

let ?project =

$\lambda x. ((\text{snd } (\text{fst } (\text{fst } x)), \text{snd } (\text{fst } x)), \text{snd } x)$

have *projected*:

Some (((data, attacker-state), result), final-state) $\in$
set-dist
 (execute
 (checked-staged-security-experiment-with-actual-alpha-prefix-data-state
 A $\gg (\lambda x. \text{return } (?project\ x))$)
 adversary-initial-state)

apply (rule *set-dist-bindI*[*OF support*])

apply *simp*

done

then show *?thesis*

using

checked-staged-security-experiment-with-actual-alpha-prefix-data-state-projection
 $[of\ A]$

by *simp*

qed

lemma *checked-staged-security-with-actual-alpha-prefix-composition-randomization-bad-imp-query-partial-opening-hi*

assumes *wf*: *staged-budget-wellformed budgets*

and *controlled*: *staged-adversary-controlled budgets A*

and *support*:

Some (((((prefix, prefix-state), data), attacker-state), result),
 final-state) $\in$
set-dist
 (execute
 (checked-staged-security-experiment-with-actual-alpha-prefix-data-state
 A)
 adversary-initial-state)

and *bad*:

```

    checked-staged-security-with-actual-alpha-prefix-verifier-event
    composition-randomization-bad
    (Some (((((prefix, prefix-state), data), attacker-state), result),
            final-state))
  shows
    checked-staged-security-with-actual-alpha-prefix-query-partial-opening-hit
    (Some (((((prefix, prefix-state), data), attacker-state), result),
            final-state))
  proof -
  let ?s =
    verifier-state-from-adversary attacker-state
    (staged-proof-transcript data)
  have data-support:
    Some (((data, attacker-state), result), final-state) ∈
    set-dist
    (execute (checked-staged-security-experiment-with-data-state A)
             adversary-initial-state)
  by (rule
      checked-staged-security-with-actual-alpha-prefix-support-imp-data-state-support
      [OF support])
  from checked-staged-security-experiment-with-data-state-outcomeE
  [OF data-support]
  obtain builder verifier where
    builder:
      Some (data, attacker-state) ∈
      set-dist
      (execute (checked-staged-transcript-program A)
               adversary-initial-state)
    and verifier:
      Some (result, final-state) ∈ set-dist (execute verify-monad ?s)
  by blast
  have staged-header:
    verifier-header-transcript ?s
    (staged-trace-root data)
    (staged-trace-fri-roots data)
    (staged-trace-final data)
    (staged-alphas data)
    (staged-degree data)
    (staged-composition-fri-roots data)
    (staged-composition-final data)
    (List.concat (staged-query-chunks data))
  by (rule checked-staged-transcript-program-outcome-header-transcript
      [OF wf controlled builder])
  from bad obtain trace-table composition-table as query-idxs f where
    ctr:
      composition-bad-context ?s (Some (result, final-state))
      trace-table composition-table as query-idxs f
  unfolding checked-staged-security-with-actual-alpha-prefix-verifier-event-def
    composition-randomization-bad-def Let-def

```



```

  by auto
have bound:
  accepted-with-bound-tables ?s (Some (result, final-state))
  trace-table composition-table as query-idxs
  using ctx unfolding composition-bad-context-def by blast
have all-queries:
  all-queries-consistent trace-table composition-table as
  using ctx unfolding composition-bad-context-def by blast
have shape:
  accepted-transcript-shape ?s (Some (result, final-state)) as query-idxs
  using bound
  unfolding accepted-with-bound-tables-def accepted-with-tables-def
  by simp
from checked-staged-security-with-data-state-accepted-shape-query-chunks
  [OF wf controlled data-support shape]
obtain raw-idxs where
  as-eq: as = staged-alphas data
  and len-raw: length raw-idxs = rounds
  and query-idxs-eq:
    query-idxs = map (λraw. index (to-nat raw)) raw-idxs
  and lookup:
    ∧ i. i < rounds ⇒
      fmllookup (HashMap final-state)
        (QueryIndexChallenge i
          (state-after-query-chunks
            (staged-query-start-hash data)
            (staged-query-chunks data) i)) =
        Some (raw-idxs ! i)
  and idx-bound:
    ∀ idx ∈ set query-idxs. idx < clength * scale
  by blast
have bound-staged:
  accepted-with-bound-tables ?s (Some (result, final-state))
  trace-table composition-table (staged-alphas data) query-idxs
  using bound as-eq by simp
have all-queries-staged:
  all-queries-consistent trace-table composition-table
    (staged-alphas data)
  using all-queries as-eq by simp
have i-bound: 0 < rounds
  by (rule rounds-positive)
have idx-sample:
  query-idxs ! 0 ∈ query-sample-space
proof -
  have len-query-idxs: length query-idxs = rounds
    by (rule accepted-with-bound-tables-shapes(4)[OF bound-staged])
  have query-idxs ! 0 ∈ set query-idxs
    by (rule nth-mem) (use i-bound len-query-idxs in simp)
  then show ?thesis

```

```

    by (rule accepted-with-bound-tables-query-sample-space
        [OF bound-staged])
qed
show ?thesis
proof (rule accepted-with-bound-tables-partial-opening-witnesses-Some
    [OF bound-staged])
fix fr f-fri-roots f-final dg composition-fri-roots final rest
    trace-openings composition-openings
assume header:
    verifier-header-transcript ?s fr f-fri-roots f-final
    (staged-alphas data) dg composition-fri-roots final rest
and comp-nonempty: composition-fri-roots  $\neq$  []
and trace-partial:
    accepted-with-partial-trace-openings ?s
    (Some (result, final-state)) fr query-idxs trace-openings
and comp-partial:
    accepted-with-partial-composition-openings ?s
    (Some (result, final-state)) (hd composition-fri-roots)
    query-idxs composition-openings
and trace-candidate:
    partial-trace-table-candidate trace-table trace-openings
and comp-candidate:
    partial-composition-table-candidate composition-table
    composition-openings
have header-eq:
    fr = staged-trace-root data  $\wedge$ 
    f-fri-roots = staged-trace-fri-roots data  $\wedge$ 
    f-final = staged-trace-final data  $\wedge$ 
    dg = staged-degree data  $\wedge$ 
    composition-fri-roots = staged-composition-fri-roots data  $\wedge$ 
    final = staged-composition-final data  $\wedge$ 
    rest = List.concat (staged-query-chunks data)
using verifier-header-transcript-unique[OF staged-header header]
by simp
have idx-in:
    query-idxs ! 0  $\in$ 
    query-header-supported-partial-opening-success-indices-at ?s fr
    f-fri-roots f-final (staged-alphas data) dg
    composition-fri-roots final 0
by (rule
    query-header-supported-partial-opening-success-indices-atI-from-partials-consistent
    [OF verifier trace-partial comp-partial trace-candidate
        comp-candidate header comp-nonempty refl i-bound idx-sample
        all-queries-staged])
have idx-in-staged:
    query-idxs ! 0  $\in$ 
    query-header-supported-partial-opening-success-indices-at ?s
    (staged-trace-root data)
    (staged-trace-fri-roots data)

```

```

    (staged-trace-final data)
    (staged-alphas data)
    (staged-degree data)
    (staged-composition-fri-roots data)
    (staged-composition-final data)
    0
  using idx-in header-eq by simp
have raw-idx:
  index (to-nat (raw-idxs ! 0)) = query-idxs ! 0
  using query-idxs-eq len-raw i-bound by simp
have hit-at:
  staged-security-with-data-state-query-partial-opening-hit-at 0
  (Some (((data, attacker-state), result), final-state))
  unfolding staged-security-with-data-state-query-partial-opening-hit-at-def
  using i-bound lookup[OF i-bound] idx-in-staged raw-idx
  by (simp add: Let-def)
have hit:
  staged-security-with-data-state-query-partial-opening-hit
  (Some (((data, attacker-state), result), final-state))
  by (rule staged-security-with-data-state-query-partial-opening-hitI
    [OF i-bound hit-at])
show ?thesis
  unfolding
    checked-staged-security-with-actual-alpha-prefix-query-partial-opening-hit-def
  using hit by simp
qed
qed

lemma checked-staged-security-with-actual-alpha-prefix-composition-candidate-drift-without-query-hit-false-on-support
  assumes wf: staged-budget-wellformed budgets
  and controlled: staged-adversary-controlled budgets A
  and support:
    out ∈
    set-dist
    (execute
      (checked-staged-security-experiment-with-actual-alpha-prefix-data-state
        A)
      adversary-initial-state)
  and hit:
    checked-staged-security-with-actual-alpha-prefix-composition-candidate-drift-without-query-hit
    out
  shows False
proof (cases out)
case None
then show ?thesis
  using hit
  unfolding
    checked-staged-security-with-actual-alpha-prefix-composition-candidate-drift-without-query-hit-def
    checked-staged-security-with-actual-alpha-prefix-composition-candidate-drift-def

```

```

    checked-staged-security-with-actual-alpha-prefix-verifier-event-def
  by simp
next
case (Some packed)
then obtain prefix prefix-state data attacker-state result final-state where
  out-eq:
    out =
      Some (((((prefix, prefix-state), data), attacker-state), result),
            final-state)
  by (cases packed, auto split: prod.splits)
have support-some:
  Some (((((prefix, prefix-state), data), attacker-state), result),
        final-state) ∈
  set-dist
    (execute
      (checked-staged-security-experiment-with-actual-alpha-prefix-data-state
        A)
      adversary-initial-state)
  using support unfolding out-eq .
have random-bad:
  checked-staged-security-with-actual-alpha-prefix-verifier-event
  composition-randomization-bad
  (Some (((((prefix, prefix-state), data), attacker-state), result),
        final-state))
  using hit unfolding out-eq
  checked-staged-security-with-actual-alpha-prefix-composition-candidate-drift-without-query-hit-def
  checked-staged-security-with-actual-alpha-prefix-composition-candidate-drift-def
  by simp
have query-hit:
  checked-staged-security-with-actual-alpha-prefix-query-partial-opening-hit
  (Some (((((prefix, prefix-state), data), attacker-state), result),
        final-state))
  by (rule
    checked-staged-security-with-actual-alpha-prefix-composition-randomization-bad-imp-query-partial-opening-hit-on
    [OF wf controlled support-some random-bad])
have not-query:
  ¬ checked-staged-security-with-actual-alpha-prefix-query-partial-opening-hit
  (Some (((((prefix, prefix-state), data), attacker-state), result),
        final-state))
  using hit unfolding out-eq
  checked-staged-security-with-actual-alpha-prefix-composition-candidate-drift-without-query-hit-def
  by simp
show ?thesis
  using query-hit not-query by contradiction
qed

lemma checked-staged-security-with-actual-alpha-prefix-composition-candidate-drift-without-query-hit-bound:
  assumes wf: staged-budget-wellformed budgets
  and controlled: staged-adversary-controlled budgets A

```

shows
wp-event
 (*checked-staged-security-experiment-with-actual-alpha-prefix-data-state*
A)
checked-staged-security-with-actual-alpha-prefix-composition-candidate-drift-without-query-hit
adversary-initial-state ≤ 0

proof —
have
wp-event
 (*checked-staged-security-experiment-with-actual-alpha-prefix-data-state*
A)
checked-staged-security-with-actual-alpha-prefix-composition-candidate-drift-without-query-hit
adversary-initial-state $\leq$
wp-event
 (*checked-staged-security-experiment-with-actual-alpha-prefix-data-state*
A)
 ($\lambda\cdot$. *False*)
adversary-initial-state
by (rule *wp-event-mono-on-support*)
 (use
checked-staged-security-with-actual-alpha-prefix-composition-candidate-drift-without-query-hit-false-on-support
 [OF *wf controlled*] **in** *blast*)
also have ... = 0
unfolding *wp-event-def wp-def dist-expect-def* **by** *simp*
finally show ?thesis .
qed

lemma *checked-staged-security-with-actual-alpha-prefix-composition-candidate-drift-imp-query-or-without-query:*
assumes
checked-staged-security-with-actual-alpha-prefix-composition-candidate-drift
out
shows
checked-staged-security-with-actual-alpha-prefix-composition-candidate-drift-query-hit
out $\vee$
checked-staged-security-with-actual-alpha-prefix-composition-candidate-drift-without-query-hit
out
using *assms*
unfolding
checked-staged-security-with-actual-alpha-prefix-composition-candidate-drift-query-hit-def
checked-staged-security-with-actual-alpha-prefix-composition-candidate-drift-without-query-hit-def
by *blast*

lemma *checked-staged-security-with-actual-alpha-prefix-composition-candidate-drift-query-hit-imp-query-hit:*
assumes
checked-staged-security-with-actual-alpha-prefix-composition-candidate-drift-query-hit
out
shows
checked-staged-security-with-actual-alpha-prefix-query-partial-opening-hit
out

```

using assms
unfolding
  checked-staged-security-with-actual-alpha-prefix-composition-candidate-drift-query-hit-def
by simp

lemma checked-staged-security-with-actual-alpha-prefix-composition-candidate-drift-query-hit-imp-branch-tree-on-sup
assumes wf: staged-budget-wellformed budgets
and controlled: staged-adversary-controlled budgets A
and support:
  out  $\in$ 
    set-dist
    (execute
      (checked-staged-security-experiment-with-actual-alpha-prefix-data-state
        A)
      adversary-initial-state)
and hit:
  checked-staged-security-with-actual-alpha-prefix-composition-candidate-drift-query-hit
    out
shows
  checked-staged-security-with-actual-alpha-prefix-branch-tree-output-hit
    out
proof –
have drift:
  checked-staged-security-with-actual-alpha-prefix-composition-candidate-drift
    out
using hit
unfolding
  checked-staged-security-with-actual-alpha-prefix-composition-candidate-drift-query-hit-def
by simp
show ?thesis
by (rule
  checked-staged-security-with-actual-alpha-prefix-composition-candidate-drift-imp-branch-tree-on-support
    [OF wf controlled support drift])
qed

lemma checked-staged-security-with-actual-alpha-prefix-composition-candidate-drift-query-hit-bound-from-branch-tree
assumes wf: staged-budget-wellformed budgets
and controlled: staged-adversary-controlled budgets A
and branch-tree-bound:
  wp-event
    (checked-staged-security-experiment-with-actual-alpha-prefix-data-state
      A)
    checked-staged-security-with-actual-alpha-prefix-branch-tree-output-hit
    adversary-initial-state  $\leq T$ 
shows
  wp-event
    (checked-staged-security-experiment-with-actual-alpha-prefix-data-state A)
    checked-staged-security-with-actual-alpha-prefix-composition-candidate-drift-query-hit
    adversary-initial-state  $\leq T$ 

```

proof –
have
wp-event
 (*checked-staged-security-experiment-with-actual-alpha-prefix-data-state A*)
checked-staged-security-with-actual-alpha-prefix-composition-candidate-drift-query-hit
adversary-initial-state $\leq$
wp-event
 (*checked-staged-security-experiment-with-actual-alpha-prefix-data-state A*)
checked-staged-security-with-actual-alpha-prefix-branch-tree-output-hit
adversary-initial-state
by (rule *wp-event-mono-on-support*)
 (rule
checked-staged-security-with-actual-alpha-prefix-composition-candidate-drift-query-hit-imp-branch-tree-on-support
 [OF *wf controlled*])
then show ?thesis
by (rule *order-trans*[OF - *branch-tree-bound*])
qed

lemma *checked-staged-security-with-actual-alpha-prefix-composition-candidate-driftE*:

assumes *hit*:

checked-staged-security-with-actual-alpha-prefix-composition-candidate-drift
 (*Some* ((((*prefix*, *prefix-state*), *data*), *attacker-state*), *result*),
final-state))

obtains *s* **where**

s = *verifier-state-from-adversary attacker-state*
 (*staged-proof-transcript data*)
composition-randomization-bad s (*Some* (*result*, *final-state*))
alpha-header-list-set-hit s
 (*alpha-header-supported-partial-union-bad-sets s*
composition-trace-bad-alpha-space)
 (*Some* (*result*, *final-state*))
 $\neg$ *staged-alphas data* $\in$
alpha-prefix-union-bad-sets prefix-state
composition-trace-bad-alpha-space (*fst prefix*)

proof –

let ?*s* =

verifier-state-from-adversary attacker-state
 (*staged-proof-transcript data*)

have *random-bad*:

composition-randomization-bad ?s (*Some* (*result*, *final-state*))

using *hit*

unfolding

checked-staged-security-with-actual-alpha-prefix-composition-candidate-drift-def
checked-staged-security-with-actual-alpha-prefix-verifier-event-def
Let-def

by *simp*

have *header-hit*:

alpha-header-list-set-hit ?s
 (*alpha-header-supported-partial-union-bad-sets ?s*)

```

      composition-trace-bad-alpha-space)
    (Some (result, final-state))
  using hit
  unfolding
    checked-staged-security-with-actual-alpha-prefix-composition-candidate-drift-def
    checked-staged-security-with-actual-alpha-prefix-partial-header-candidate-drift-def
    checked-staged-security-with-actual-alpha-prefix-partial-header-fresh-hit-def
    checked-staged-security-with-actual-alpha-prefix-partial-header-bad-set-hit-def
    Let-def
  by simp
  have not-prefix:
    ¬ staged-alphas data ∈
      alpha-prefix-union-bad-sets prefix-state
      composition-trace-bad-alpha-space (fst prefix)
  using hit
  unfolding
    checked-staged-security-with-actual-alpha-prefix-composition-candidate-drift-def
    checked-staged-security-with-actual-alpha-prefix-partial-header-candidate-drift-def
    checked-staged-security-with-actual-alpha-prefix-bad-set-hit-def
    Let-def
  by simp
  show ?thesis
  by (rule that[OF refl random-bad header-hit not-prefix])
qed

lemma checked-staged-security-with-actual-alpha-prefix-composition-candidate-drift-query-hitE:
  assumes hit:
    checked-staged-security-with-actual-alpha-prefix-composition-candidate-drift-query-hit
      (Some (((((prefix, prefix-state), data), attacker-state), result),
        final-state))
  obtains s where
    s = verifier-state-from-adversary attacker-state
      (staged-proof-transcript data)
    composition-randomization-bad s (Some (result, final-state))
    alpha-header-list-set-hit s
      (alpha-header-supported-partial-union-bad-sets s
        composition-trace-bad-alpha-space)
      (Some (result, final-state))
    ¬ staged-alphas data ∈
      alpha-prefix-union-bad-sets prefix-state
      composition-trace-bad-alpha-space (fst prefix)
    checked-staged-security-with-actual-alpha-prefix-query-partial-opening-hit
      (Some (((((prefix, prefix-state), data), attacker-state), result),
        final-state))
  proof -
  have drift:
    checked-staged-security-with-actual-alpha-prefix-composition-candidate-drift
      (Some (((((prefix, prefix-state), data), attacker-state), result),
        final-state))

```



```

using hit
unfolding
  checked-staged-security-with-actual-alpha-prefix-composition-candidate-drift-query-hit-def
by simp
have query-hit:
  checked-staged-security-with-actual-alpha-prefix-query-partial-opening-hit
    (Some ((((prefix, prefix-state), data), attacker-state), result),
      final-state))
using hit
unfolding
  checked-staged-security-with-actual-alpha-prefix-composition-candidate-drift-query-hit-def
by simp
from checked-staged-security-with-actual-alpha-prefix-composition-candidate-driftE
  [OF drift]
obtain s where
  s-def:
    s = verifier-state-from-adversary attacker-state
      (staged-proof-transcript data)
and random-bad:
    composition-randomization-bad s (Some (result, final-state))
and header-hit:
    alpha-header-list-set-hit s
      (alpha-header-supported-partial-union-bad-sets s
        composition-trace-bad-alpha-space)
      (Some (result, final-state))
and not-prefix:
     $\neg$  staged-alphas data  $\in$ 
      alpha-prefix-union-bad-sets prefix-state
      composition-trace-bad-alpha-space (fst prefix)
by blast
show ?thesis
by (rule that[OF s-def random-bad header-hit not-prefix query-hit])
qed

```

```

lemma checked-staged-security-with-actual-alpha-prefix-composition-candidate-drift-query-hit-supportedE:
assumes wf: staged-budget-wellformed budgets
and controlled: staged-adversary-controlled budgets A
and support:
  Some ((((prefix, prefix-state), data), attacker-state), result),
    final-state)  $\in$ 
  set-dist
    (execute
      (checked-staged-security-experiment-with-actual-alpha-prefix-data-state
        A)
      adversary-initial-state)
and hit:
  checked-staged-security-with-actual-alpha-prefix-composition-candidate-drift-query-hit
    (Some ((((prefix, prefix-state), data), attacker-state), result),
      final-state))

```

obtains s *trace-table* **where**

$s = \text{verifier-state-from-adversary attacker-state}$
 $(\text{staged-proof-transcript data})$
 $\text{composition-randomization-bad } s \text{ (Some (result, final-state))}$
 $\text{staged-alphas data} \in$
 $\text{alpha-header-supported-partial-union-bad-sets } s$
 $\text{composition-trace-bad-alpha-space}$
 $(\text{staged-trace-root data})$
 $(\text{staged-trace-fri-roots data})$
 $(\text{staged-trace-final data})$
 $\text{trace-table} \in$
 $\text{alpha-header-supported-partial-trace-table-candidates } s$
 $(\text{staged-trace-root data})$
 $(\text{staged-trace-fri-roots data})$
 $(\text{staged-trace-final data})$
 $\text{staged-alphas data} \in \text{composition-trace-bad-alpha-space trace-table}$
 $\neg \text{staged-alphas data} \in$
 $\text{alpha-prefix-union-bad-sets prefix-state}$
 $\text{composition-trace-bad-alpha-space (fst prefix)}$
 $\text{checked-staged-security-with-actual-alpha-prefix-query-partial-opening-hit}$
 $(\text{Some } (((((\text{prefix}, \text{prefix-state}), \text{data}), \text{attacker-state}), \text{result}),$
 $\text{final-state}))$

proof —

let $?s =$

$\text{verifier-state-from-adversary attacker-state}$
 $(\text{staged-proof-transcript data})$

from $\text{checked-staged-security-with-actual-alpha-prefix-composition-candidate-drift-query-hitE}$
 $[\text{OF hit}]$

have random-bad :

$\text{composition-randomization-bad } ?s \text{ (Some (result, final-state))}$

and header-list-hit :

$\text{alpha-header-list-set-hit } ?s$
 $(\text{alpha-header-supported-partial-union-bad-sets } ?s$
 $\text{composition-trace-bad-alpha-space})$
 $(\text{Some (result, final-state)})$

and not-prefix :

$\neg \text{staged-alphas data} \in$
 $\text{alpha-prefix-union-bad-sets prefix-state}$
 $\text{composition-trace-bad-alpha-space (fst prefix)}$

and query-hit :

$\text{checked-staged-security-with-actual-alpha-prefix-query-partial-opening-hit}$
 $(\text{Some } (((((\text{prefix}, \text{prefix-state}), \text{data}), \text{attacker-state}), \text{result}),$
 $\text{final-state}))$

by blast+

have data-support :

$\text{Some } (((\text{data}, \text{attacker-state}), \text{result}), \text{final-state}) \in$
 set-dist
 $(\text{execute } (\text{checked-staged-security-experiment-with-data-state } A)$
 $\text{adversary-initial-state})$

```

by (rule
  checked-staged-security-with-actual-alpha-prefix-support-imp-data-state-support
  [OF support])
from checked-staged-security-experiment-with-data-state-outcomeE
  [OF data-support]
obtain builder verifier where
  builder:
    Some (data, attacker-state) ∈
      set-dist
        (execute (checked-staged-transcript-program A)
          adversary-initial-state)
  and verifier:
    Some (result, final-state) ∈ set-dist (execute verify-monad ?s)
by blast
have staged-header:
  verifier-header-transcript ?s
    (staged-trace-root data)
    (staged-trace-fri-roots data)
    (staged-trace-final data)
    (staged-alphas data)
    (staged-degree data)
    (staged-composition-fri-roots data)
    (staged-composition-final data)
    (List.concat (staged-query-chunks data))
by (rule checked-staged-transcript-program-outcome-header-transcript
  [OF wf controlled builder])
from header-list-hit obtain as query-idxs fr f-fri-roots f-final dg
  composition-fri-roots final rest where
  shape: accepted-transcript-shape ?s (Some (result, final-state))
  as query-idxs
and header:
  verifier-header-transcript ?s fr f-fri-roots f-final as dg
  composition-fri-roots final rest
and as-bad:
  as ∈ alpha-header-supported-partial-union-bad-sets ?s
  composition-trace-bad-alpha-space fr f-fri-roots f-final
unfolding alpha-header-list-set-hit-def by blast
have header-eq:
  fr = staged-trace-root data ∧
  f-fri-roots = staged-trace-fri-roots data ∧
  f-final = staged-trace-final data ∧
  as = staged-alphas data ∧
  dg = staged-degree data ∧
  composition-fri-roots = staged-composition-fri-roots data ∧
  final = staged-composition-final data ∧
  rest = List.concat (staged-query-chunks data)
using verifier-header-transcript-unique[OF header staged-header]
by simp
have staged-bad:

```

```

    staged-alphas data ∈
      alpha-header-supported-partial-union-bad-sets ?s
      composition-trace-bad-alpha-space
      (staged-trace-root data)
      (staged-trace-fri-roots data)
      (staged-trace-final data)
    using as-bad header-eq by simp
  from staged-bad obtain trace-table where trace-candidate:
    trace-table ∈
      alpha-header-supported-partial-trace-table-candidates ?s
      (staged-trace-root data)
      (staged-trace-fri-roots data)
      (staged-trace-final data)
    and trace-bad:
      staged-alphas data ∈ composition-trace-bad-alpha-space trace-table
    unfolding alpha-header-supported-partial-union-bad-sets-def by blast
  show ?thesis
    by (rule that[OF refl random-bad staged-bad trace-candidate trace-bad
      not-prefix query-hit])
qed

lemma checked-staged-security-with-actual-alpha-prefix-composition-candidate-drift-query-hit-supported-openingsE:
  assumes wf: staged-budget-wellformed budgets
    and controlled: staged-adversary-controlled budgets A
    and support:
      Some (((((prefix, prefix-state), data), attacker-state), result),
        final-state) ∈
        set-dist
        (execute
          (checked-staged-security-experiment-with-actual-alpha-prefix-data-state
            A)
          adversary-initial-state)
    and hit:
      checked-staged-security-with-actual-alpha-prefix-composition-candidate-drift-query-hit
        (Some (((((prefix, prefix-state), data), attacker-state), result),
          final-state))
  obtains s trace-table i raw trace-openings composition-openings where
    s = verifier-state-from-adversary attacker-state
      (staged-proof-transcript data)
    composition-randomization-bad s (Some (result, final-state))
    trace-table ∈
      alpha-header-supported-partial-trace-table-candidates s
      (staged-trace-root data)
      (staged-trace-fri-roots data)
      (staged-trace-final data)
    staged-alphas data ∈ composition-trace-bad-alpha-space trace-table
    ¬ staged-alphas data ∈
      alpha-prefix-union-bad-sets prefix-state
      composition-trace-bad-alpha-space (fst prefix)

```

```

i < rounds
fmlookup (HashMap final-state)
  (QueryIndexChallenge i
    (state-after-query-chunks
      (staged-query-start-hash data)
      (staged-query-chunks data) i)) =
  Some raw
index (to-nat raw) ∈ query-sample-space
(trace-openings, composition-openings) ∈
  query-header-supported-partial-opening-witnesses s
    (staged-trace-root data)
    (staged-trace-fri-roots data)
    (staged-trace-final data)
    (staged-alphas data)
    (staged-degree data)
    (staged-composition-fri-roots data)
    (staged-composition-final data)
partial-query-openings-consistent trace-openings composition-openings
  (staged-alphas data) (index (to-nat raw))
proof –
from checked-staged-security-with-actual-alpha-prefix-composition-candidate-drift-query-hit-supportedE
  [OF wf controlled support hit]
obtain s trace-table where
  s-def:
    s = verifier-state-from-adversary attacker-state
      (staged-proof-transcript data)
and random-bad:
    composition-randomization-bad s (Some (result, final-state))
and trace-candidate:
    trace-table ∈
      alpha-header-supported-partial-trace-table-candidates s
        (staged-trace-root data)
        (staged-trace-fri-roots data)
        (staged-trace-final data)
and trace-bad:
    staged-alphas data ∈ composition-trace-bad-alpha-space trace-table
and not-prefix:
    ¬ staged-alphas data ∈
      alpha-prefix-union-bad-sets prefix-state
      composition-trace-bad-alpha-space (fst prefix)
and query-hit:
    checked-staged-security-with-actual-alpha-prefix-query-partial-opening-hit
      (Some (((((prefix, prefix-state), data), attacker-state), result),
        final-state))
by blast
from checked-staged-security-with-actual-alpha-prefix-query-partial-opening-hitE
  [OF query-hit]
obtain i raw trace-openings composition-openings where
  i-bound: i < rounds

```

```

and lookup:
  fmlookup (HashMap final-state)
    (QueryIndexChallenge i
      (state-after-query-chunks
        (staged-query-start-hash data)
        (staged-query-chunks data) i)) =
    Some raw
and idx-sample: index (to-nat raw) ∈ query-sample-space
and witness:
  (trace-openings, composition-openings) ∈
    query-header-supported-partial-opening-witnesses
      (verifier-state-from-adversary attacker-state
        (staged-proof-transcript data))
      (staged-trace-root data)
      (staged-trace-fri-roots data)
      (staged-trace-final data)
      (staged-alphas data)
      (staged-degree data)
      (staged-composition-fri-roots data)
      (staged-composition-final data)
and consistent:
  partial-query-openings-consistent trace-openings composition-openings
    (staged-alphas data) (index (to-nat raw))
by blast
have witness-s:
  (trace-openings, composition-openings) ∈
    query-header-supported-partial-opening-witnesses s
      (staged-trace-root data)
      (staged-trace-fri-roots data)
      (staged-trace-final data)
      (staged-alphas data)
      (staged-degree data)
      (staged-composition-fri-roots data)
      (staged-composition-final data)
using witness s-def by simp
show ?thesis
by (rule that[OF s-def random-bad trace-candidate trace-bad not-prefix
      i-bound lookup idx-sample witness-s consistent])
qed

definition checked-staged-security-with-actual-alpha-prefix-composition-candidate-opening-query-hit
  :: ('f staged-adversary ⇒
    (((((( 'f × 'f list × 'f list × 'f) ×
      'f protocol-channel) × 'f staged-proof-data) ×
      'f protocol-channel) × unit list) ×
      'f protocol-channel) option ⇒ bool)

where
  checked-staged-security-with-actual-alpha-prefix-composition-candidate-opening-query-hit
    A out  $\longleftrightarrow$ 

```

(case out of
 None $\Rightarrow$ False
 | Some (((((alpha-prefix, alpha-prefix-state), data), attacker-state),
 result), final-state) $\Rightarrow$
 ($\exists i$ trace-openings composition-openings.
 checked-staged-security-with-actual-alpha-prefix-candidate-opening-query-hit A
 ((replicate rounds []) [i := trace-openings])
 ((replicate rounds []) [i := composition-openings])
 (staged-alphas data) i
 (Some (((((alpha-prefix, alpha-prefix-state), data),
 attacker-state), result), final-state))))))

lemma checked-staged-security-with-actual-alpha-prefix-composition-candidate-opening-query-hitI:

assumes component:

checked-staged-security-with-actual-alpha-prefix-candidate-opening-query-hit A
 ((replicate rounds []) [i := trace-openings])
 ((replicate rounds []) [i := composition-openings])
 (staged-alphas data) i
 (Some (((((alpha-prefix, alpha-prefix-state), data), attacker-state), result),
 final-state))

shows

checked-staged-security-with-actual-alpha-prefix-composition-candidate-opening-query-hit A
 (Some (((((alpha-prefix, alpha-prefix-state), data), attacker-state), result),
 final-state))

proof —

have

$\exists j$ trace-openings' composition-openings'.
 checked-staged-security-with-actual-alpha-prefix-candidate-opening-query-hit A
 ((replicate rounds []) [j := trace-openings'])
 ((replicate rounds []) [j := composition-openings'])
 (staged-alphas data) j
 (Some (((((alpha-prefix, alpha-prefix-state), data), attacker-state), result),
 final-state))

by (intro exI[of - i] exI[of - trace-openings]
 exI[of - composition-openings])
 (rule component)

then show ?thesis

unfolding

checked-staged-security-with-actual-alpha-prefix-composition-candidate-opening-query-hit-def

by simp

qed

lemma checked-staged-security-with-actual-alpha-prefix-composition-candidate-drift-query-hit-bound-from-query-hit:

assumes query-bound:

wp-event
 (checked-staged-security-experiment-with-actual-alpha-prefix-data-state A)
 checked-staged-security-with-actual-alpha-prefix-query-partial-opening-hit
 adversary-initial-state $\leq Q$

shows

wp-event
 (*checked-staged-security-experiment-with-actual-alpha-prefix-data-state A*)
checked-staged-security-with-actual-alpha-prefix-composition-candidate-drift-query-hit
adversary-initial-state $\leq Q$
by (rule *order.trans[OF - query-bound]*)
 (rule *wp-event-mono*,
 rule
checked-staged-security-with-actual-alpha-prefix-composition-candidate-drift-query-hit-imp-query-hit)

lemma *checked-staged-security-with-actual-alpha-prefix-composition-candidate-drift-bound-from-query-and-without-q*
assumes *query-bound*:

wp-event
 (*checked-staged-security-experiment-with-actual-alpha-prefix-data-state A*)
checked-staged-security-with-actual-alpha-prefix-composition-candidate-drift-query-hit
adversary-initial-state $\leq Q$

and *without-bound*:

wp-event
 (*checked-staged-security-experiment-with-actual-alpha-prefix-data-state A*)
checked-staged-security-with-actual-alpha-prefix-composition-candidate-drift-without-query-hit
adversary-initial-state $\leq W$

shows

wp-event
 (*checked-staged-security-experiment-with-actual-alpha-prefix-data-state A*)
checked-staged-security-with-actual-alpha-prefix-composition-candidate-drift
adversary-initial-state $\leq Q + W$

proof –

let *?M* =
checked-staged-security-experiment-with-actual-alpha-prefix-data-state A
let *?Drift* =
checked-staged-security-with-actual-alpha-prefix-composition-candidate-drift
let *?Query* =
checked-staged-security-with-actual-alpha-prefix-composition-candidate-drift-query-hit
let *?Without* =
checked-staged-security-with-actual-alpha-prefix-composition-candidate-drift-without-query-hit
have *wp-event ?M ?Drift adversary-initial-state* $\leq$
wp-event ?M ($\lambda out. ?Query\ out \vee ?Without\ out$)
adversary-initial-state
by (rule *wp-event-mono*)
 (rule
checked-staged-security-with-actual-alpha-prefix-composition-candidate-drift-imp-query-or-without-query)
also have ... $\leq$
wp-event ?M ?Query adversary-initial-state +
wp-event ?M ?Without adversary-initial-state
by (rule *wp-event-union-bound*)
also have ... $\leq Q + W$
by (*intro add-mono query-bound without-bound*)
finally show *?thesis* .

qed

lemma *checked-staged-security-with-data-state-composition-bad-bound-from-composition-candidate-drift-and-budgets:*

assumes *wf: staged-budget-wellformed budgets*

and *controlled: staged-adversary-controlled budgets A*

and *drift-bound:*

wp-event

*(checked-staged-security-experiment-with-actual-alpha-prefix-data-state
A)*

*checked-staged-security-with-actual-alpha-prefix-composition-candidate-drift
adversary-initial-state $\leq D$*

shows

wp-event (checked-staged-security-experiment-with-data-state A)

(staged-security-with-data-state-verifier-event composition-bad)

adversary-initial-state $\leq$

composition-error-bound +

staged-phase-relation-error size

(staged-alpha-search-queries budgets 0) +

hash-collision-budget-value 0

(staged-alpha-search-queries budgets 0) +

staged-phase-relation-error size

(staged-alpha-search-queries budgets 0) +

D +

hash-collision-budget-value 0

(staged-attacker-query-budget budgets +

staged-challenge-query-budget + verifier-hash-query-budget)

proof –

let *?M =*

checked-staged-security-experiment-with-actual-alpha-prefix-data-state A

let *?Prefix =*

composition-error-bound +

staged-phase-relation-error size

(staged-alpha-search-queries budgets 0) +

hash-collision-budget-value 0

(staged-alpha-search-queries budgets 0)

let *?Pre =*

staged-phase-relation-error size

(staged-alpha-search-queries budgets 0)

let *?Coll =*

hash-collision-budget-value 0

(staged-attacker-query-budget budgets +

staged-challenge-query-budget + verifier-hash-query-budget)

have *transcript-bound:*

wp-event (checked-staged-transcript-with-alpha-prefix-program A)

(checked-staged-transcript-actual-alpha-prefix-bad-set-hit

composition-trace-bad-alpha-space)

adversary-initial-state $\leq$?Prefix

by *(rule*

checked-staged-transcript-with-alpha-prefix-actual-bad-set-hit-bound-from-budgets

```

    [OF wf controlled])
have prefix-bound:
  wp-event ?M
  (checked-staged-security-with-actual-alpha-prefix-bad-set-hit
   composition-trace-bad-alpha-space)
  adversary-initial-state ≤ ?Prefix
by (rule
  checked-staged-security-with-actual-alpha-prefix-bad-set-hit-bound-from-transcript
  [OF transcript-bound])
have prequery-bound:
  wp-event ?M
  (checked-staged-security-with-actual-alpha-prefix-partial-header-prequery-hit
   composition-trace-bad-alpha-space)
  adversary-initial-state ≤ ?Pre
by (rule
  checked-staged-security-with-actual-alpha-prefix-vector-alpha-prequery-accounting
  [OF wf controlled])
have collision-bound:
  wp-event ?M
  (checked-staged-security-with-actual-alpha-prefix-verifier-event
   hash-map-output-collision-bad)
  adversary-initial-state ≤ ?Coll
by (rule
  checked-staged-security-with-actual-alpha-prefix-hash-map-output-collision-bad-bound-controlled
  [OF wf controlled])
let ?degree =
  checked-staged-security-with-actual-alpha-prefix-verifier-event
  composition-degree-bad
let ?random =
  checked-staged-security-with-actual-alpha-prefix-verifier-event
  composition-randomization-bad
let ?bad =
  checked-staged-security-with-actual-alpha-prefix-verifier-event
  composition-bad
have projected:
  wp-event (checked-staged-security-experiment-with-data-state A)
  (staged-security-with-data-state-verifier-event composition-bad)
  adversary-initial-state =
  wp-event ?M ?bad adversary-initial-state
by (rule
  checked-staged-security-with-data-state-verifier-event-actual-alpha-prefix-projection)
have event-eq: ?bad = (λout. ?degree out ∨ ?random out)
by (rule ext)
  (auto simp:
  checked-staged-security-with-actual-alpha-prefix-verifier-event-def
  composition-bad-def split: option.splits prod.splits)
have degree-false: ?degree = (λout. False)
by (rule ext)
  (auto simp:

```

```

    checked-staged-security-with-actual-alpha-prefix-verifier-event-def
    composition-degree-bad-false[OF spec-degree-wellformed-from-spec-query-margin]
    split: option.splits prod.splits)
have degree-bound:
  wp-event ?M ?degree adversary-initial-state ≤ 0
  unfolding degree-false wp-event-def wp-def dist-expect-def by simp
have random-bound:
  wp-event ?M ?random adversary-initial-state ≤
    ?Prefix + ?Pre + D + ?Coll
  by (rule
    checked-staged-security-with-actual-alpha-prefix-composition-randomization-bad-bound-from-prefix-prequery-drift-
    [OF prefix-bound prequery-bound drift-bound collision-bound])
have wp-event ?M ?bad adversary-initial-state ≤
  wp-event ?M ?degree adversary-initial-state +
  wp-event ?M ?random adversary-initial-state
  unfolding event-eq by (rule wp-event-union-bound)
also have ... ≤ 0 + (?Prefix + ?Pre + D + ?Coll)
  by (intro add-mono degree-bound random-bound)
finally show ?thesis
  unfolding projected by (simp add: add.assoc)
qed

lemma checked-staged-security-with-data-state-composition-bad-bound-from-partial-opening-hit-and-budgets:
fixes query-error :: prob
assumes wf: staged-budget-wellformed budgets
and controlled: staged-adversary-controlled budgets A
and partial-opening-bound:
  wp-event (checked-staged-security-experiment-with-data-state A)
    staged-security-with-data-state-query-partial-opening-hit
    adversary-initial-state ≤ query-error
shows
  wp-event (checked-staged-security-experiment-with-data-state A)
    (staged-security-with-data-state-verifier-event composition-bad)
    adversary-initial-state ≤
    composition-error-bound +
    staged-phase-relation-error size
    (staged-alpha-search-queries budgets 0) +
    hash-collision-budget-value 0
    (staged-alpha-search-queries budgets 0) +
    staged-phase-relation-error size
    (staged-alpha-search-queries budgets 0) +
    (query-error + 0) +
    hash-collision-budget-value 0
    (staged-attacker-query-budget budgets +
    staged-challenge-query-budget + verifier-hash-query-budget)
proof -
let ?M =
  checked-staged-security-experiment-with-actual-alpha-prefix-data-state A
have query-bound:

```

wp-event ?M
checked-staged-security-with-actual-alpha-prefix-query-partial-opening-hit
adversary-initial-state ≤ query-error
unfolding
checked-staged-security-with-actual-alpha-prefix-query-partial-opening-hit-projection
by (rule *partial-opening-bound*)
have *drift-query-bound*:
wp-event ?M
checked-staged-security-with-actual-alpha-prefix-composition-candidate-drift-query-hit
adversary-initial-state ≤ query-error
by (rule
checked-staged-security-with-actual-alpha-prefix-composition-candidate-drift-query-hit-bound-from-query-hit
[OF *query-bound*])
have *without-bound*:
wp-event ?M
checked-staged-security-with-actual-alpha-prefix-composition-candidate-drift-without-query-hit
adversary-initial-state ≤ 0
by (rule
checked-staged-security-with-actual-alpha-prefix-composition-candidate-drift-without-query-hit-bound
[OF *wf controlled*])
have *drift-bound*:
wp-event ?M
checked-staged-security-with-actual-alpha-prefix-composition-candidate-drift
adversary-initial-state ≤ query-error + 0
by (rule
checked-staged-security-with-actual-alpha-prefix-composition-candidate-drift-bound-from-query-and-without-query
[OF *drift-query-bound without-bound*])
show *?thesis*
by (rule
checked-staged-security-with-data-state-composition-bad-bound-from-composition-candidate-drift-and-budgets
[OF *wf controlled drift-bound*])
qed

lemma *checked-staged-security-with-data-state-composition-bad-bound-from-partial-query-residual-and-budgets*:
assumes *wf: staged-budget-wellformed budgets*
and *controlled: staged-adversary-controlled budgets A*
and *cover:*
 $\bigwedge \text{data attacker-state } i.$
 $i < \text{rounds} \implies$
query-header-supported-partial-opening-success-indices-at
(*verifier-state-from-adversary attacker-state*
(*staged-proof-transcript data*))
(*staged-trace-root data*)
(*staged-trace-fri-roots data*)
(*staged-trace-final data*)
(*staged-alphas data*)
(*staged-degree data*)
(*staged-composition-fri-roots data*)
(*staged-composition-final data*)

$i \subseteq B$
and *raw-bound*: *query-index-raw-preimage-bound*
and *subset*: $B \subseteq \text{query-sample-space}$
and *frac*:
 $\text{nnreal}(\text{card } B) / \text{nnreal}(\text{card } \text{query-sample-space}) \leq$
 query-error-bound
and *residual-bound*:
 wp-event
 $(\text{checked-staged-security-experiment-with-actual-alpha-prefix-data-state}$
 $A)$
 $\text{checked-staged-security-with-actual-alpha-prefix-composition-candidate-drift-without-query-hit}$
 $\text{adversary-initial-state} \leq R$
shows
 $\text{wp-event}(\text{checked-staged-security-experiment-with-data-state } A)$
 $(\text{staged-security-with-data-state-verifier-event composition-bad})$
 $\text{adversary-initial-state} \leq$
 $\text{composition-error-bound} +$
 $\text{staged-phase-relation-error size}$
 $(\text{staged-alpha-search-queries budgets } 0) +$
 $\text{hash-collision-budget-value } 0$
 $(\text{staged-alpha-search-queries budgets } 0) +$
 $\text{staged-phase-relation-error size}$
 $(\text{staged-alpha-search-queries budgets } 0) +$
 $(\text{nnreal}$
 $(\text{staged-attacker-query-budget budgets} +$
 $\text{staged-challenge-query-budget}) * \text{query-error-bound} + R) +$
 $\text{hash-collision-budget-value } 0$
 $(\text{staged-attacker-query-budget budgets} +$
 $\text{staged-challenge-query-budget} + \text{verifier-hash-query-budget})$
proof –
let $?M =$
 $\text{checked-staged-security-experiment-with-actual-alpha-prefix-data-state } A$
let $?QueryCost =$
 nnreal
 $(\text{staged-attacker-query-budget budgets} +$
 $\text{staged-challenge-query-budget}) * \text{query-error-bound}$
have *query-bound*:
 $\text{wp-event } ?M$
 $\text{checked-staged-security-with-actual-alpha-prefix-query-partial-opening-hit}$
 $\text{adversary-initial-state} \leq ?QueryCost$
by (rule
 $\text{checked-staged-security-with-actual-alpha-prefix-query-partial-opening-hit-bound-from-fixed-query-error-cover}$
 $[\text{OF wf controlled cover raw-bound subset frac}]$)
have *drift-query-bound*:
 $\text{wp-event } ?M$
 $\text{checked-staged-security-with-actual-alpha-prefix-composition-candidate-drift-query-hit}$
 $\text{adversary-initial-state} \leq ?QueryCost$
by (rule
 $\text{checked-staged-security-with-actual-alpha-prefix-composition-candidate-drift-query-hit-bound-from-query-hit}$

$[OF \text{ query-bound}]$)
have *drift-bound*:
 $wp\text{-event } ?M$
 $checked\text{-staged}\text{-security}\text{-with}\text{-actual}\text{-alpha}\text{-prefix}\text{-composition}\text{-candidate}\text{-drift}$
 $adversary\text{-initial}\text{-state} \leq ?QueryCost + R$
by (rule
 $checked\text{-staged}\text{-security}\text{-with}\text{-actual}\text{-alpha}\text{-prefix}\text{-composition}\text{-candidate}\text{-drift}\text{-bound}\text{-from}\text{-query}\text{-and}\text{-without}\text{-query}$
 $[OF \text{ drift}\text{-query}\text{-bound residual}\text{-bound}]$)
have
 $wp\text{-event } (checked\text{-staged}\text{-security}\text{-experiment}\text{-with}\text{-data}\text{-state } A)$
 $(staged\text{-security}\text{-with}\text{-data}\text{-state}\text{-verifier}\text{-event } composition\text{-bad})$
 $adversary\text{-initial}\text{-state} \leq$
 $composition\text{-error}\text{-bound} +$
 $staged\text{-phase}\text{-relation}\text{-error } size$
 $(staged\text{-alpha}\text{-search}\text{-queries } budgets \ 0) +$
 $hash\text{-collision}\text{-budget}\text{-value } 0$
 $(staged\text{-alpha}\text{-search}\text{-queries } budgets \ 0) +$
 $staged\text{-phase}\text{-relation}\text{-error } size$
 $(staged\text{-alpha}\text{-search}\text{-queries } budgets \ 0) +$
 $(?QueryCost + R) +$
 $hash\text{-collision}\text{-budget}\text{-value } 0$
 $(staged\text{-attacker}\text{-query}\text{-budget } budgets +$
 $staged\text{-challenge}\text{-query}\text{-budget} + verifier\text{-hash}\text{-query}\text{-budget})$
by (rule
 $checked\text{-staged}\text{-security}\text{-with}\text{-data}\text{-state}\text{-composition}\text{-bad}\text{-bound}\text{-from}\text{-composition}\text{-candidate}\text{-drift}\text{-and}\text{-budgets}$
 $[OF \text{ wf controlled drift}\text{-bound}]$)
then show *?thesis*
by (*simp add: add.assoc*)
qed

lemma *checked-staged-security-with-data-state-composition-bad-bound-from-partial-query-and-budgets*:

assumes *wf: staged-budget-wellformed budgets*
and *controlled: staged-adversary-controlled budgets A*
and *cover:*
 $\bigwedge data \text{ attacker}\text{-state } i.$
 $i < rounds \implies$
 $query\text{-header}\text{-supported}\text{-partial}\text{-opening}\text{-success}\text{-indices}\text{-at}$
 $(verifier\text{-state}\text{-from}\text{-adversary } attacker\text{-state}$
 $(staged\text{-proof}\text{-transcript } data))$
 $(staged\text{-trace}\text{-root } data)$
 $(staged\text{-trace}\text{-fri}\text{-roots } data)$
 $(staged\text{-trace}\text{-final } data)$
 $(staged\text{-alphas } data)$
 $(staged\text{-degree } data)$
 $(staged\text{-composition}\text{-fri}\text{-roots } data)$
 $(staged\text{-composition}\text{-final } data)$
 $i \subseteq B$
and *raw-bound: query-index-raw-preimage-bound*
and *subset: $B \subseteq query\text{-sample}\text{-space}$*

and *frac*:

$$\text{nnreal } (\text{card } B) / \text{nnreal } (\text{card } \text{query-sample-space}) \leq \text{query-error-bound}$$

shows

$$\begin{aligned} & \text{wp-event } (\text{checked-staged-security-experiment-with-data-state } A) \\ & (\text{staged-security-with-data-state-verifier-event composition-bad}) \\ & \text{adversary-initial-state} \leq \\ & \text{composition-error-bound} + \\ & \text{staged-phase-relation-error size} \\ & (\text{staged-alpha-search-queries budgets } 0) + \\ & \text{hash-collision-budget-value } 0 \\ & (\text{staged-alpha-search-queries budgets } 0) + \\ & \text{staged-phase-relation-error size} \\ & (\text{staged-alpha-search-queries budgets } 0) + \\ & \text{nnreal} \\ & (\text{staged-attacker-query-budget budgets} + \\ & \quad \text{staged-challenge-query-budget}) * \text{query-error-bound} + \\ & \text{hash-collision-budget-value } 0 \\ & (\text{staged-attacker-query-budget budgets} + \\ & \quad \text{staged-challenge-query-budget} + \text{verifier-hash-query-budget}) \end{aligned}$$

proof –
have *residual-bound*:

$$\begin{aligned} & \text{wp-event} \\ & (\text{checked-staged-security-experiment-with-actual-alpha-prefix-data-state} \\ & \quad A) \\ & \text{checked-staged-security-with-actual-alpha-prefix-composition-candidate-drift-without-query-hit} \\ & \text{adversary-initial-state} \leq 0 \end{aligned}$$

by (rule

$$\text{checked-staged-security-with-actual-alpha-prefix-composition-candidate-drift-without-query-hit-bound}$$

$$[\text{OF wf controlled}])$$

have

$$\begin{aligned} & \text{wp-event } (\text{checked-staged-security-experiment-with-data-state } A) \\ & (\text{staged-security-with-data-state-verifier-event composition-bad}) \\ & \text{adversary-initial-state} \leq \\ & \text{composition-error-bound} + \\ & \text{staged-phase-relation-error size} \\ & (\text{staged-alpha-search-queries budgets } 0) + \\ & \text{hash-collision-budget-value } 0 \\ & (\text{staged-alpha-search-queries budgets } 0) + \\ & \text{staged-phase-relation-error size} \\ & (\text{staged-alpha-search-queries budgets } 0) + \\ & (\text{nnreal} \\ & \quad (\text{staged-attacker-query-budget budgets} + \\ & \quad \quad \text{staged-challenge-query-budget}) * \text{query-error-bound} + 0) + \\ & \text{hash-collision-budget-value } 0 \\ & (\text{staged-attacker-query-budget budgets} + \\ & \quad \text{staged-challenge-query-budget} + \text{verifier-hash-query-budget}) \end{aligned}$$

by (rule

$$\text{checked-staged-security-with-data-state-composition-bad-bound-from-partial-query-residual-and-budgets}$$

```

    [OF wf controlled cover raw-bound subset frac residual-bound])
  then show ?thesis
    by simp
qed

end

end

```

```

theory Soundness-Query-Opening-Extraction
imports Soundness-Partial-Merkle
begin

```

Low-level verifier query extraction lemmas used by staged soundness.

This layer is deliberately kept below the staged event definitions: it only talks about verifier query rounds, authenticated openings, transcript chunks, and random-oracle lookups.

```

context soundness
begin

```

lemma *query-opening-fri-layer-opening-step-partial-authenticated-table-first-value:*

```

fixes s t :: ('f, 'a) protocol-channel-scheme
assumes len-pos: 0 < len
and idx-bound: idx < len
and outcome:
  Some (out, t) ∈
    set-dist
      (execute (fri-layer-opening-step (b, rt) (idx, x, len, pw)) s)
obtains openings where
  partial-authenticated-table rt len openings t
  map opening-index openings = [idx, (idx + len div 2) mod len]
  length openings = 2
  opening-value (openings ! 0) = x

```

proof –

```

from outcome[unfolded fri-layer-opening-step-unfold-finish]
obtain xp s1 xp-path s2 xn s3 xn-path s4 where
  read-xp: Some (xp, s1) ∈ set-dist (execute read s)
and read-xp-path:
  Some (xp-path, s2) ∈
    set-dist (execute (ntimes read (floor-log len)) s1)
and read-xn: Some (xn, s3) ∈ set-dist (execute read s2)
and read-xn-path:
  Some (xn-path, s4) ∈
    set-dist (execute (ntimes read (floor-log len)) s3)
and finish:
  Some (out, t) ∈
    set-dist
      (execute
        (fri-layer-opening-finish b rt idx x len pw xp xp-path xn

```



```

       $xn\text{-path}$ )  $s_4$ )
    by (auto elim!: set-dist-bindE)
  have  $xp\text{-path-length}$ :  $\text{length } xp\text{-path} = \text{floor-log } len$ 
    using  $ntimes\text{-read-any-outcome}[OF \text{read-}xp\text{-path}]$  by simp
  have  $xn\text{-path-length}$ :  $\text{length } xn\text{-path} = \text{floor-log } len$ 
    using  $ntimes\text{-read-any-outcome}[OF \text{read-}xn\text{-path}]$  by simp
  have  $sibling\text{-bound}$ :  $(idx + len \text{ div } 2) \bmod len < len$ 
    using  $len\text{-pos}$  by simp
  have  $xp\text{-x}$ :  $xp = x$ 
    using  $finish$ 
  unfolding  $fri\text{-layer-opening-finish-def}$   $assert\text{-def}$ 
  by (auto simp:  $throw\text{-no-outcome}$  elim!: set-dist-bindE split: if-splits)
from  $fri\text{-layer-opening-finish-authenticated-openings}$ 
  [ $OF \text{idx-bound}$   $sibling\text{-bound}$   $xp\text{-path-length}$   $xn\text{-path-length}$   $finish$ ]
obtain  $xp\text{-opening}$   $xn\text{-opening}$  where
   $xp\text{-eq}$ :
     $xp\text{-opening} =$ 
      ( $\langle opening\text{-root} = rt,$ 
         $opening\text{-length} = len,$ 
         $opening\text{-index} = idx,$ 
         $opening\text{-value} = xp,$ 
         $opening\text{-path} = xp\text{-path} \rangle$ )
  and  $xn\text{-eq}$ :
     $xn\text{-opening} =$ 
      ( $\langle opening\text{-root} = rt,$ 
         $opening\text{-length} = len,$ 
         $opening\text{-index} = (idx + len \text{ div } 2) \bmod len,$ 
         $opening\text{-value} = xn,$ 
         $opening\text{-path} = xn\text{-path} \rangle$ )
  and  $xp\text{-auth}$ :  $authenticated\text{-opening-in } t \text{ } xp\text{-opening}$ 
  and  $xn\text{-auth}$ :  $authenticated\text{-opening-in } t \text{ } xn\text{-opening}$ 
  by blast
let  $?openings = [xp\text{-opening}, xn\text{-opening}]$ 
have  $table$ :  $partial\text{-authenticated-table } rt \text{ } len \text{ } ?openings \text{ } t$ 
  unfolding  $partial\text{-authenticated-table-def}$ 
  using  $xp\text{-eq}$   $xn\text{-eq}$   $xp\text{-auth}$   $xn\text{-auth}$  by simp
have  $indices$ :
   $map \text{opening-index } ?openings = [idx, (idx + len \text{ div } 2) \bmod len]$ 
  using  $xp\text{-eq}$   $xn\text{-eq}$  by simp
have  $first\text{-value}$ :  $opening\text{-value } (?openings ! 0) = x$ 
  using  $xp\text{-eq}$   $xp\text{-x}$  by simp
show  $?thesis$ 
  by (rule that[ $OF \text{table indices - first-value}$ ]) simp
qed

```

lemma $query\text{-opening-ntimes-outcome-decomp-at}$:

assumes $outcome$:

$Some (results, t) \in set\text{-dist } (execute (ntimes \text{ } m \text{ } n) \text{ } s)$

and $i\text{-bound}$: $i < n$

shows
 $\exists \text{prefix } x \text{ suffix } s\text{-}i \text{ s-suc.}$
 $\text{Some } (\text{prefix}, s\text{-}i) \in \text{set-dist } (\text{execute } (\text{ntimes } m \text{ } i) \text{ } s) \wedge$
 $\text{Some } (x, s\text{-}suc) \in \text{set-dist } (\text{execute } m \text{ } s\text{-}i) \wedge$
 $\text{Some } (\text{suffix}, t) \in$
 $\text{set-dist } (\text{execute } (\text{ntimes } m \text{ } (n - \text{Suc } i)) \text{ } s\text{-}suc) \wedge$
 $\text{results} = \text{prefix} @ x \# \text{suffix}$
using *outcome i-bound*
proof (*induction n arbitrary: i s results t*)
case 0
then show ?*case* **by** *simp*
next
case (*Suc n*)
from *Suc.prem*s(1) **obtain** *x results' s1* **where**
 $\text{head: Some } (x, s1) \in \text{set-dist } (\text{execute } m \text{ } s)$
and tail:
 $\text{Some } (\text{results}', t) \in \text{set-dist } (\text{execute } (\text{ntimes } m \text{ } n) \text{ } s1)$
and results-eq: $\text{results} = x \# \text{results}'$
by (*auto elim!: set-dist-bindE*)
show ?*case*
proof (*cases i*)
case 0
have *prefix0:*
 $\text{Some } ([], s) \in \text{set-dist } (\text{execute } (\text{ntimes } m \text{ } i) \text{ } s)$
using 0 **by** *simp*
have *suffix0:*
 $\text{Some } (\text{results}', t) \in$
 $\text{set-dist } (\text{execute } (\text{ntimes } m \text{ } (\text{Suc } n - \text{Suc } i)) \text{ } s1)$
using *tail 0* **by** *simp*
show ?*thesis*
using *prefix0 head suffix0 results-eq*
by (*intro exI[of - []] exI[of - x] exI[of - results']*
 $\text{exI[of - s] exI[of - s1]] \text{ simp}$)
next
case (*Suc j*)
have *j-bound:* $j < n$
using *Suc.prem*s(2) *Suc* **by** *simp*
from *Suc.IH*[*OF tail j-bound*]
obtain *prefix y suffix s-j s-suc* **where**
prefix:
 $\text{Some } (\text{prefix}, s\text{-}j) \in \text{set-dist } (\text{execute } (\text{ntimes } m \text{ } j) \text{ } s1)$
and round: $\text{Some } (y, s\text{-}suc) \in \text{set-dist } (\text{execute } m \text{ } s\text{-}j)$
and suffix:
 $\text{Some } (\text{suffix}, t) \in$
 $\text{set-dist } (\text{execute } (\text{ntimes } m \text{ } (n - \text{Suc } j)) \text{ } s\text{-}suc)$
and results'-eq: $\text{results}' = \text{prefix} @ y \# \text{suffix}$
by *blast*
have *prefix-suc:*
 $\text{Some } (x \# \text{prefix}, s\text{-}j) \in$

```

    set-dist (execute (ntimes m (Suc j)) s)
  unfolding ntimes.simps
  apply (rule set-dist-bindI[OF head])
  apply (rule set-dist-bindI[OF prefix])
  apply simp
  done
have n-minus: Suc n - Suc (Suc j) = n - Suc j
  by simp
have suffix-suc:
  Some (suffix, t) ∈
    set-dist (execute (ntimes m (Suc n - Suc i)) s-suc)
  using suffix n-minus Suc by simp
show ?thesis
  using prefix-suc round suffix-suc results-eq results'-eq Suc
  by (intro exI[of - x # prefix] exI[of - y] exI[of - suffix]
    exI[of - s-j] exI[of - s-suc]) simp
qed
qed

lemma query-opening-verifier-query-round-program-authenticated-with-lookup:
  fixes s t :: ('f, 'a) protocol-channel-scheme
  assumes fl-eq: fl = (b, composition-root) # fl-tail
  and outcome:
    Some ((), t) ∈
      set-dist
        (execute
          (verifier-query-round-program fr f-fl f-final as fl final) s)
  obtains raw idx trace-leaves trace-openings composition-openings where
    idx = index (to-nat raw)
    map opening-value trace-openings = trace-leaves
    map opening-index trace-openings = powers-scaled idx
    partial-authenticated-table fr (scale * clength) trace-openings t
    partial-authenticated-table composition-root (scale * clength)
      composition-openings t
    map opening-index composition-openings =
      [idx, fri-sibling-index (scale * clength) idx]
    fmlookup (HashMap t)
      (QueryIndexChallenge (PQueryCounter s) (PState s)) = Some raw
proof -
  from outcome[unfolded verifier-query-round-program-alt-def]
  obtain raw s0 where rand:
    Some (raw, s0) ∈
      set-dist (execute receive-query-index-challenge s)
  and tail:
    Some ((), t) ∈
      set-dist
        (execute
          (verifier-query-round-after-index-program
            fr f-fl f-final as fl final raw) s0)

```

```

    by (auto elim!: set-dist-bindE)
  have lookup-s0:
    fmlookup (HashMap s0)
      (QueryIndexChallenge (PQueryCounter s) (PState s)) = Some raw
    using receive-query-index-challenge-outcome[OF rand] by simp
  have lookup-t:
    fmlookup (HashMap t)
      (QueryIndexChallenge (PQueryCounter s) (PState s)) = Some raw
    using verifier-query-round-after-index-program-preserves-query-lookup
      [OF tail, of PQueryCounter s PState s] lookup-s0
    by simp
  show ?thesis
proof (rule verifier-query-round-after-index-authenticated-trace-openings
  [OF tail])
  fix idx trace-leaves trace-openings
  assume idx-eq: idx = index (to-nat raw)
  and trace-values:
    map opening-value trace-openings = trace-leaves
  and trace-indices:
    map opening-index trace-openings = powers-scaled idx
  and trace-table:
    partial-authenticated-table fr (scale * clength)
      trace-openings t
  show ?thesis
proof (rule
  verifier-query-round-after-index-authenticated-composition-openings
  [OF fl-eq tail])
  fix idx' composition-openings
  assume idx'-eq: idx' = index (to-nat raw)
  and comp-table:
    partial-authenticated-table composition-root (scale * clength)
      composition-openings t
  and comp-indices:
    map opening-index composition-openings =
      [idx', fri-sibling-index (scale * clength) idx']
  have idx'-idx: idx' = idx
    using idx-eq idx'-eq by simp
  show ?thesis
  by (rule that[OF idx-eq trace-values trace-indices trace-table
    comp-table - lookup-t])
    (use comp-indices idx'-idx in simp)
qed
qed
qed

```

lemma *query-opening-verifier-query-round-program-authenticated-with-chunk:*
fixes $s\ t :: ('f, 'a)\ \text{protocol-channel-scheme}$
assumes $fl\text{-eq}: fl = (b, \text{composition-root}) \# fl\text{-tail}$
and *outcome:*

```

    Some (( ), t) ∈
      set-dist
        (execute
          (verifier-query-round-program fr f-fl f-final as fl final) s)
obtains raw idx chunk trace-leaves trace-openings composition-openings
where
  idx = index (to-nat raw)
  verifier-query-round-chunk idx (map snd f-fl) (map snd fl) chunk
  PTranscript s = chunk @ PTranscript t
  PState t = foldl concat (PState s) chunk
  s ≤ t
  PQueryCounter t = Suc (PQueryCounter s)
  map opening-value trace-openings = trace-leaves
  map opening-index trace-openings = powers-scaled idx
  partial-authenticated-table fr (scale * clength) trace-openings t
  partial-authenticated-table composition-root (scale * clength)
    composition-openings t
  map opening-index composition-openings =
    [idx, fri-sibling-index (scale * clength) idx]
  fmllookup (HashMap t)
    (QueryIndexChallenge (PQueryCounter s) (PState s)) = Some raw
proof –
  show ?thesis
proof (rule verifier-query-round-program-outcome[OF outcome])
  fix raw0 idx0 chunk
  assume idx0-eq: idx0 = index (to-nat raw0)
  and chunk-shape:
    verifier-query-round-chunk idx0 (map snd f-fl) (map snd fl)
      chunk
  and transcript:
    PTranscript s = chunk @ PTranscript t
  and state:
    PState t = foldl concat (PState s) chunk
  and ext: s ≤ t
  and counter:
    PQueryCounter t = Suc (PQueryCounter s)
  and lookup0:
    fmllookup (HashMap t)
      (QueryIndexChallenge (PQueryCounter s) (PState s)) = Some raw0
  show ?thesis
proof (rule
  query-opening-verifier-query-round-program-authenticated-with-lookup
    [OF fl-eq outcome])
  fix raw idx trace-leaves trace-openings composition-openings
  assume idx-eq: idx = index (to-nat raw)
  and trace-values:
    map opening-value trace-openings = trace-leaves
  and trace-indices:
    map opening-index trace-openings = powers-scaled idx

```

```

and trace-table:
  partial-authenticated-table fr (scale * clength)
  trace-openings t
and comp-table:
  partial-authenticated-table composition-root (scale * clength)
  composition-openings t
and comp-indices:
  map opening-index composition-openings =
    [idx, fri-sibling-index (scale * clength) idx]
and lookup:
  fmlookup (HashMap t)
  (QueryIndexChallenge (PQueryCounter s) (PState s)) = Some raw
have raw-eq: raw = raw0
  using lookup lookup0 by simp
have idx-idx0: idx = idx0
  using idx-eq idx0-eq raw-eq by simp
show ?thesis
  by (rule that[OF idx-eq - transcript state ext counter
    trace-values trace-indices trace-table comp-table comp-indices
    lookup])
    (use chunk-shape idx-idx0 in simp)
qed
qed
qed

lemma query-opening-ntimes-prefix-state-alignment:
fixes s t :: ('f, 'a) protocol-channel-scheme
assumes outcome:
  Some (prefix, t) ∈
    set-dist
      (execute
        (ntimes
          (verifier-query-round-program fr f-fl f-final as fl final) i)
        s)
obtains prefix-chunks where
  length prefix-chunks = i
  PState t = state-after-query-chunks (PState s) prefix-chunks i
  PQueryCounter t = PQueryCounter s + i
  s ≤ t
proof –
from ntimes-verifier-query-rounds-outcome[OF outcome]
obtain raw-idxs query-idxs prefix-chunks where
  len-chunks: length prefix-chunks = i
and state:
  PState t = state-after-query-chunks (PState s) prefix-chunks i
and ext: s ≤ t
and counter: PQueryCounter t = PQueryCounter s + i
by blast
show ?thesis

```

by (rule that[OF len-chunks state counter ext])
qed

lemma *query-opening-ntimes-prefix-transcript-state-alignment*:
fixes $s\ t :: ('f, 'a)\ \text{protocol-channel-scheme}$
assumes *outcome*:
Some (prefix, t) ∈
set-dist
(execute
(ntimes
(verifier-query-round-program fr f-fl f-final as fl final) i)
s)
obtains *prefix-query-idxs prefix-chunks* **where**
length prefix-query-idxs = i
length prefix-chunks = i
PTranscript s = List.concat prefix-chunks @ PTranscript t
PState t = state-after-query-chunks (PState s) prefix-chunks i
PQueryCounter t = PQueryCounter s + i
s ≤ t
 $\bigwedge j. j < i \implies$
verifier-query-round-chunk (prefix-query-idxs ! j)
(map snd f-fl) (map snd fl) (prefix-chunks ! j)
proof –
from *ntimes-verifier-query-rounds-outcome[OF outcome]*
obtain *raw-idxs query-idxs prefix-chunks* **where**
len-raw-idxs: length raw-idxs = i
and *query-idxs-eq*:
query-idxs = map (λraw. index (to-nat raw)) raw-idxs
and
len-chunks: length prefix-chunks = i
and *transcript*:
PTranscript s = List.concat prefix-chunks @ PTranscript t
and *state*:
PState t = state-after-query-chunks (PState s) prefix-chunks i
and *ext: s ≤ t*
and *counter: PQueryCounter t = PQueryCounter s + i*
and *rounds*:
 $\bigwedge j. j < i \implies$
verifier-query-round-chunk (query-idxs ! j)
(map snd f-fl) (map snd fl) (prefix-chunks ! j)
by *blast*
have *len-idxs: length query-idxs = i*
using *len-raw-idxs query-idxs-eq* **by** *simp*
show *?thesis*
by (rule that[OF len-idxs len-chunks transcript state counter ext])
(use rounds in blast)
qed

lemma *query-opening-ntimes-suffix-extends*:

fixes $s\ t :: ('f, 'a) \text{ protocol-channel-scheme}$
assumes outcome :
 $\text{Some } (\text{suffix}, t) \in$
 set-dist
 $(\text{execute}$
 $(\text{ntimes}$
 $(\text{verifier-query-round-program } \text{fr } f\text{-fl } f\text{-final } \text{as } \text{fl } \text{final})\ n)$
 $s)$
shows $s \leq t$
proof –
from $\text{ntimes-verifier-query-rounds-outcome}[OF\ \text{outcome}]$
show $?thesis$
by blast
qed

lemma $\text{query-opening-ntimes-suffix-transcript-state-alignment}$:
fixes $s\ t :: ('f, 'a) \text{ protocol-channel-scheme}$
assumes outcome :
 $\text{Some } (\text{suffix}, t) \in$
 set-dist
 $(\text{execute}$
 $(\text{ntimes}$
 $(\text{verifier-query-round-program } \text{fr } f\text{-fl } f\text{-final } \text{as } \text{fl } \text{final})\ n)$
 $s)$
obtains $\text{suffix-query-idxs } \text{suffix-chunks}$ **where**
 $\text{length } \text{suffix-query-idxs} = n$
 $\text{length } \text{suffix-chunks} = n$
 $P\text{Transcript } s = \text{List.concat } \text{suffix-chunks} @ P\text{Transcript } t$
 $P\text{State } t = \text{state-after-query-chunks } (P\text{State } s) \text{ suffix-chunks } n$
 $P\text{QueryCounter } t = P\text{QueryCounter } s + n$
 $s \leq t$
 $\bigwedge j. j < n \implies$
 $\text{verifier-query-round-chunk } (\text{suffix-query-idxs } ! j)$
 $(\text{map } \text{snd } f\text{-fl}) (\text{map } \text{snd } \text{fl}) (\text{suffix-chunks } ! j)$

proof –
from $\text{ntimes-verifier-query-rounds-outcome}[OF\ \text{outcome}]$
obtain $\text{raw-idxs } \text{query-idxs } \text{suffix-chunks}$ **where**
 $\text{len-raw-idxs}: \text{length } \text{raw-idxs} = n$
and query-idxs-eq :
 $\text{query-idxs} = \text{map } (\lambda \text{raw}. \text{index } (\text{to-nat } \text{raw})) \text{ raw-idxs}$
and $\text{len-chunks}: \text{length } \text{suffix-chunks} = n$
and transcript :
 $P\text{Transcript } s = \text{List.concat } \text{suffix-chunks} @ P\text{Transcript } t$
and state :
 $P\text{State } t = \text{state-after-query-chunks } (P\text{State } s) \text{ suffix-chunks } n$
and $\text{ext}: s \leq t$
and $\text{counter}: P\text{QueryCounter } t = P\text{QueryCounter } s + n$
and rounds :
 $\bigwedge j. j < n \implies$


```

    verifier-query-round-chunk (query-idxs ! j)
      (map snd f-fl) (map snd fl) (suffix-chunks ! j)
  by blast
have len-idxs: length query-idxs = n
  using len-raw-idxs query-idxs-eq by simp
show ?thesis
  by (rule that[OF len-idxs len-chunks transcript state counter ext])
    (use rounds in blast)
qed

lemma query-opening-ntimes-authenticated-initial-openings-at:
  fixes s t :: ('f, 'a) protocol-channel-scheme
  assumes fl-eq: fl = (b, composition-root) # fl-tail
  and outcome:
    Some (results, t) ∈
      set-dist
        (execute
          (ntimes
            (verifier-query-round-program fr f-fl f-final as fl final) n)
          s)
  and i-bound: i < n
obtain raw idx prefix-chunks trace-openings composition-openings where
  idx = index (to-nat raw)
  length prefix-chunks = i
  map opening-index trace-openings = powers-scaled idx
  partial-authenticated-table fr (scale * clength) trace-openings t
  partial-authenticated-table composition-root (scale * clength)
    composition-openings t
  map opening-index composition-openings =
    [idx, fri-sibling-index (scale * clength) idx]
  fmlookup (HashMap t)
    (QueryIndexChallenge (PQueryCounter s + i)
      (state-after-query-chunks (PState s) prefix-chunks i)) = Some raw
proof -
  from query-opening-ntimes-outcome-decomp-at[OF outcome i-bound]
obtain prefix round suffix s-i s-suc where
  prefix:
    Some (prefix, s-i) ∈
      set-dist
        (execute
          (ntimes
            (verifier-query-round-program fr f-fl f-final as fl final) i)
          s)
  and round:
    Some (round, s-suc) ∈
      set-dist
        (execute
          (verifier-query-round-program fr f-fl f-final as fl final) s-i)
  and suffix:

```

```

    Some (suffix, t) ∈
      set-dist
        (execute
          (ntimes
            (verifier-query-round-program fr f-fl f-final as fl final)
            (n - Suc i))
          s-suc)
  by blast
have round-unit: round = ()
  by (cases round) simp
show ?thesis
proof (rule
  query-opening-verifier-query-round-program-authenticated-with-chunk
  [OF fl-eq round[unfolded round-unit]])
fix raw idx chunk trace-leaves trace-openings composition-openings
assume idx-eq: idx = index (to-nat raw)
and chunk-shape:
  verifier-query-round-chunk idx (map snd f-fl) (map snd fl)
  chunk
and transcript: PTranscript s-i = chunk @ PTranscript s-suc
and state-suc: PState s-suc = foldl concat (PState s-i) chunk
and ext-round: s-i ≤ s-suc
and counter-suc: PQueryCounter s-suc = Suc (PQueryCounter s-i)
and trace-values: map opening-value trace-openings = trace-leaves
and trace-indices:
  map opening-index trace-openings = powers-scaled idx
and trace-table-suc:
  partial-authenticated-table fr (scale * clength)
  trace-openings s-suc
and comp-table-suc:
  partial-authenticated-table composition-root (scale * clength)
  composition-openings s-suc
and comp-indices:
  map opening-index composition-openings =
    [idx, fri-sibling-index (scale * clength) idx]
and lookup-suc:
  fmllookup (HashMap s-suc)
  (QueryIndexChallenge (PQueryCounter s-i) (PState s-i)) =
    Some raw
show ?thesis
proof (rule query-opening-ntimes-prefix-state-alignment[OF prefix])
fix prefix-chunks
assume len-chunks: length prefix-chunks = i
and state-i:
  PState s-i =
    state-after-query-chunks (PState s) prefix-chunks i
and counter-i: PQueryCounter s-i = PQueryCounter s + i
have suffix-ext: s-suc ≤ t
  by (rule query-opening-ntimes-suffix-extends[OF suffix])

```

```

have trace-table-t:
  partial-authenticated-table fr (scale * clength)
  trace-openings t
by (rule
  partial-authenticated-table-mono[OF trace-table-suc
  suffix-ext])
have comp-table-t:
  partial-authenticated-table composition-root (scale * clength)
  composition-openings t
by (rule
  partial-authenticated-table-mono[OF comp-table-suc suffix-ext])
have lookup-t:
  fmllookup (HashMap t)
  (QueryIndexChallenge (PQueryCounter s + i)
  (state-after-query-chunks (PState s) prefix-chunks i)) =
  Some raw
using hash-extension-lookup[OF lookup-suc suffix-ext]
unfolding counter-i state-i .
show ?thesis
by (rule that[OF idx-eq len-chunks trace-indices trace-table-t
  comp-table-t comp-indices lookup-t])
qed
qed
qed
end
end

```

theory Soundness-Query-Opening-Consistency

imports

Soundness-Query-Opening-Extraction

Soundness-Staged-Partial-Query

begin

Verifier-local query-opening consistency facts. These lemmas package the composition check enforced by a successful query round together with the authenticated openings extracted from that round.

context soundness

begin

lemma verifier-query-round-after-index-authenticated-openings-consistent:

fixes $s\ t :: ('f, 'a)\ \text{protocol-channel-scheme}$

assumes $fl\text{-eq}: fl = (b, \text{composition-root}) \# fl\text{-tail}$

and outcome:

$\text{Some } ((), t) \in$

set-dist

$(\text{execute}$

```

      (verifier-query-round-after-index-program
       fr f-fl f-final as fl final raw) s)
obtains idx trace-openings composition-openings where
  idx = index (to-nat raw)
  map opening-index trace-openings = powers-scaled idx
  partial-authenticated-table fr (scale * clength) trace-openings t
  partial-authenticated-table composition-root (scale * clength)
    composition-openings t
  map opening-index composition-openings =
    [idx, fri-sibling-index (scale * clength) idx]
  partial-query-openings-consistent trace-openings composition-openings
    as idx
proof –
  let ?idx = index (to-nat raw)
  from outcome obtain fv s1 f-out s2 s3 c-out s4 where
    query-decommit:
      Some (fv, s1) ∈
        set-dist (execute (mmap (check-decommit-on-query fr ?idx)) s)
    and trace-fri:
      Some (f-out, s2) ∈
        set-dist (execute
          (mfold (?idx, hd fv, clength * scale, 1)
            (receive-query-commits f-fl)) s1)
    and assert-trace:
      Some ((), s3) ∈
        set-dist
          (execute
            (assert (case f-out of (-, f-x, -, -) ⇒ f-x = f-final)) s2)
    and comp-fri:
      Some (c-out, s4) ∈
        set-dist (execute
          (mfold (?idx, cp-eval as fv (h ^ ?idx * shift),
            clength * scale, 1)
            (receive-query-commits fl)) s3)
    and assert-comp:
      Some ((), t) ∈
        set-dist
          (execute
            (assert (case c-out of (-, x, -, -) ⇒ x = final)) s4)
    unfolding verifier-query-round-after-index-program-def
    by (auto simp: Let-def split: prod.splits elim!: set-dist-bindE)
  have idx-sample: ?idx ∈ query-sample-space
    using index-less-query-sample-space
    unfolding query-sample-space-def by simp
  from check-decommit-on-query-authenticated-openings
    [OF idx-sample query-decommit]
  obtain trace-openings where
    trace-values: map opening-value trace-openings = fv
    and trace-indices:

```

```

    map opening-index trace-openings = powers-scaled ?idx
and trace-table-s1:
    partial-authenticated-table fr (scale * clength) trace-openings s1
by blast
from receive-query-commits-layers-outcome[OF trace-fri]
obtain trace-layer-chunks trace-fri-chunk where s1-s2:  $s1 \leq s2$ 
by blast
have s3-eq:  $s3 = s2$ 
    using assert-trace unfolding assert-def
    by (cases case f-out of  $(-, f-x, -, -) \Rightarrow f-x = f-final$ )
        (auto simp: throw-no-outcome)
have comp-steps:
    receive-query-commits fl =
    fri-layer-opening-step (b, composition-root) #
    receive-query-commits fl-tail
    unfolding fl-eq receive-query-commits-def fri-layer-opening-step-def
    by simp
from comp-fri[unfolded comp-steps]
obtain out1 s-head where
    head:
    Some (out1, s-head)  $\in$ 
    set-dist
    (execute
    (fri-layer-opening-step (b, composition-root)
    (?idx, cp-eval as fv ( $h \wedge ?idx * shift$ ),
    clength * scale, 1)) s3)
and tail:
    Some (c-out, s4)  $\in$ 
    set-dist (execute (mfold out1 (receive-query-commits fl-tail))
    s-head)
    by (auto elim!: set-dist-bindE)
have len-pos:  $0 < clength * scale$ 
    using eval-domain-nontrivial by linarith
have idx-bound:  $?idx < clength * scale$ 
    by (rule index-less-domain)
from query-opening-fri-layer-opening-step-partial-authenticated-table-first-value
    [OF len-pos idx-bound head]
obtain composition-openings where
    comp-table-head:
    partial-authenticated-table composition-root (clength * scale)
    composition-openings s-head
and comp-indices0:
    map opening-index composition-openings =
    [ $?idx$ ,  $(?idx + (clength * scale) \text{ div } 2) \bmod (clength * scale)$ ]
and comp-len: length composition-openings = 2
and comp-value:
    opening-value (composition-openings ! 0) =
    cp-eval as fv ( $h \wedge ?idx * shift$ )
by blast

```

```

from fri-layer-opening-step-outcome[OF head]
have s3-s-head:  $s3 \leq s\text{-head}$ 
  by blast
obtain i1 x1 len1 pw1 where out1-eq: out1 = (i1, x1, len1, pw1)
  by (cases out1)
from receive-query-commits-layers-outcome[OF tail[unfolded out1-eq]]
obtain tail-layer-chunks tail-chunk where s-head-s4:  $s\text{-head} \leq s4$ 
  by blast
have t-eq:  $t = s4$ 
  using assert-comp unfolding assert-def
  by (cases case c-out of ( $\_, x, \_, \_ \Rightarrow x = \text{final}$ )
    (auto simp: throw-no-outcome))
have s1-t:  $s1 \leq t$ 
  using s1-s2 s3-s-head s-head-s4 unfolding s3-eq t-eq
  by (meson hash-ext-trans)
have trace-table-t:
  partial-authenticated-table fr ( $\text{scale} * \text{clength}$ ) trace-openings t
  by (rule partial-authenticated-table-mono[OF trace-table-s1 s1-t])
have comp-table-t:
  partial-authenticated-table composition-root ( $\text{scale} * \text{clength}$ )
  composition-openings t
  using partial-authenticated-table-mono[OF comp-table-head s-head-s4]
  unfolding t-eq by (simp add: mult commute)
have comp-indices:
  map opening-index composition-openings =
    [?idx, fri-sibling-index ( $\text{scale} * \text{clength}$ ) ?idx]
  using comp-indices0
  unfolding fri-sibling-index-def
  by (simp add: mult commute)
have consistent:
  partial-query-openings-consistent trace-openings composition-openings
  as ?idx
  by (rule partial-query-openings-consistentI
    [OF idx-sample trace-indices comp-indices])
    (use comp-value trace-values in simp)
show ?thesis
  by (rule that[OF refl trace-indices trace-table-t comp-table-t
    comp-indices consistent])
qed

```

```

lemma verifier-query-round-after-index-empty-composition-consistent:
fixes  $s\ t :: ('f, 'a)\ \text{protocol-channel-scheme}$ 
assumes fl-empty:  $\text{fl} = []$ 
and outcome:
  Some ( $()$ ,  $t$ )  $\in$ 
    set-dist
    (execute
      (verifier-query-round-after-index-program
        fr f-fl f-final as fl final raw)  $s$ )

```

obtains *idx trace-openings* **where**
idx = *index* (*to-nat raw*)
map opening-index trace-openings = *powers-scaled idx*
partial-authenticated-table fr (*scale * clength*) *trace-openings t*
cp-eval as (*map opening-value trace-openings*) (*h ^ idx * shift*) =
final
proof –
let *?idx* = *index* (*to-nat raw*)
from *outcome* **obtain** *fv s1 f-out s2 s3 c-out s4* **where**
query-decommit:
Some (*fv, s1*) ∈
set-dist (*execute* (*mmap* (*check-decommit-on-query fr ?idx*)) *s*)
and *trace-fri:*
Some (*f-out, s2*) ∈
set-dist (*execute*
(*mfold* (*?idx, hd fv, clength * scale, 1*)
(*receive-query-commits f-fl*)) *s1*)
and *assert-trace:*
Some (*()*, *s3*) ∈
set-dist
(*execute*
(*assert* (*case f-out of* (*-, f-x, -, -*) ⇒ *f-x = f-final*)) *s2*)
and *comp-fri:*
Some (*c-out, s4*) ∈
set-dist (*execute*
(*mfold* (*?idx, cp-eval as fv* (*h ^ ?idx * shift*),
*clength * scale, 1*)
(*receive-query-commits fl*)) *s3*)
and *assert-comp:*
Some (*()*, *t*) ∈
set-dist
(*execute*
(*assert* (*case c-out of* (*-, x, -, -*) ⇒ *x = final*)) *s4*)
unfolding *verifier-query-round-after-index-program-def*
by (*auto simp: Let-def split: prod.splits elim!: set-dist-bindE*)
have *idx-sample: ?idx* ∈ *query-sample-space*
using *index-less-query-sample-space*
unfolding *query-sample-space-def* **by** *simp*
from *check-decommit-on-query-authenticated-openings*
[*OF idx-sample query-decommit*]
obtain *trace-openings* **where**
trace-values: map opening-value trace-openings = *fv*
and *trace-indices:*
map opening-index trace-openings = *powers-scaled ?idx*
and *trace-table-s1:*
partial-authenticated-table fr (*scale * clength*) *trace-openings s1*
by *blast*
from *receive-query-commits-layers-outcome*[*OF trace-fri*]
obtain *trace-layer-chunks trace-fri-chunk* **where** *s1-s2: s1* ≤ *s2*

```

    by blast
  have s3-eq: s3 = s2
    using assert-trace unfolding assert-def
    by (cases case f-out of (-, f-x, -, -)  $\Rightarrow$  f-x = f-final)
      (auto simp: throw-no-outcome)
  have comp-out:
    c-out =
      (?idx, cp-eval as fv (h  $\wedge$  ?idx * shift), clength * scale, 1)  $\wedge$ 
      s4 = s3
    using comp-fri fl-empty
    unfolding receive-query-commits-def by simp
  have final-eq:
    cp-eval as fv (h  $\wedge$  ?idx * shift) = final
  proof (rule ccontr)
    assume  $\neg$  cp-eval as fv (h  $\wedge$  ?idx * shift) = final
    then have Some ((), t)  $\in$  set-dist (execute throw s3)
      using assert-comp comp-out unfolding assert-def by simp
    then show False
      by (simp add: throw-no-outcome)
  qed
  have t-eq: t = s4
    using assert-comp comp-out final-eq unfolding assert-def by simp
  have t-s2: t = s2
    using s3-eq t-eq comp-out by simp
  have s1-t: s1  $\leq$  t
    using s1-s2 t-s2 by simp
  have trace-table-t:
    partial-authenticated-table fr (scale * clength) trace-openings t
    by (rule partial-authenticated-table-mono[OF trace-table-s1 s1-t])
  have final-eq':
    cp-eval as (map opening-value trace-openings) (h  $\wedge$  ?idx * shift) =
      final
    using final-eq trace-values by simp
  show ?thesis
    by (rule that[OF refl trace-indices trace-table-t final-eq'])
qed

```

lemma verifier-query-round-program-authenticated-openings-consistent-with-lookup:

```

  fixes s t :: ('f, 'a) protocol-channel-scheme
  assumes fl-eq: fl = (b, composition-root) # fl-tail
  and outcome:
    Some ((), t)  $\in$ 
      set-dist
        (execute
          (verifier-query-round-program fr f-fl f-final as fl final) s)
  obtains raw idx trace-openings composition-openings where
    idx = index (to-nat raw)
    map opening-index trace-openings = powers-scaled idx
    partial-authenticated-table fr (scale * clength) trace-openings t

```



```

    partial-authenticated-table composition-root (scale * clength)
      composition-openings t
    map opening-index composition-openings =
      [idx, fri-sibling-index (scale * clength) idx]
    partial-query-openings-consistent trace-openings composition-openings
      as idx
    fmlookup (HashMap t)
      (QueryIndexChallenge (PQueryCounter s) (PState s)) = Some raw
  proof -
  from outcome[unfolded verifier-query-round-program-alt-def]
  obtain raw s0 where rand:
    Some (raw, s0) ∈
      set-dist (execute receive-query-index-challenge s)
  and tail:
    Some ((), t) ∈
      set-dist
        (execute
          (verifier-query-round-after-index-program
            fr f-fl f-final as fl final raw) s0)
  by (auto elim!: set-dist-bindE)
  have lookup-s0:
    fmlookup (HashMap s0)
      (QueryIndexChallenge (PQueryCounter s) (PState s)) = Some raw
  using receive-query-index-challenge-outcome[OF rand] by simp
  have lookup-t:
    fmlookup (HashMap t)
      (QueryIndexChallenge (PQueryCounter s) (PState s)) = Some raw
  using verifier-query-round-after-index-program-preserves-query-lookup
    [OF tail, of PQueryCounter s PState s] lookup-s0
  by simp
  show ?thesis
  proof (rule verifier-query-round-after-index-authenticated-openings-consistent
    [OF fl-eq tail])
  fix idx trace-openings composition-openings
  assume idx-eq: idx = index (to-nat raw)
  and trace-indices:
    map opening-index trace-openings = powers-scaled idx
  and trace-table:
    partial-authenticated-table fr (scale * clength) trace-openings t
  and comp-table:
    partial-authenticated-table composition-root (scale * clength)
      composition-openings t
  and comp-indices:
    map opening-index composition-openings =
      [idx, fri-sibling-index (scale * clength) idx]
  and consistent:
    partial-query-openings-consistent trace-openings
      composition-openings as idx
  show ?thesis

```

```

    by (rule that[OF idx-eq trace-indices trace-table comp-table
        comp-indices consistent lookup-t])
qed
qed

lemma verifier-query-round-program-empty-composition-consistent-with-lookup:
  fixes s t :: ('f, 'a) protocol-channel-scheme
  assumes fl-empty: fl = []
  and outcome:
    Some (((), t) ∈
      set-dist
        (execute
          (verifier-query-round-program fr f-fl f-final as fl final) s)
  obtains raw idx trace-openings where
    idx = index (to-nat raw)
    map opening-index trace-openings = powers-scaled idx
    partial-authenticated-table fr (scale * clength) trace-openings t
    cp-eval as (map opening-value trace-openings) (h ^ idx * shift) =
      final
    fmlookup (HashMap t)
      (QueryIndexChallenge (PQueryCounter s) (PState s)) = Some raw
proof -
  from outcome[unfolded verifier-query-round-program-alt-def]
  obtain raw s0 where rand:
    Some (raw, s0) ∈
      set-dist (execute receive-query-index-challenge s)
  and tail:
    Some (((), t) ∈
      set-dist
        (execute
          (verifier-query-round-after-index-program
            fr f-fl f-final as fl final raw) s0)
  by (auto elim!: set-dist-bindE)
  have lookup-s0:
    fmlookup (HashMap s0)
      (QueryIndexChallenge (PQueryCounter s) (PState s)) = Some raw
  using receive-query-index-challenge-outcome[OF rand] by simp
  have lookup-t:
    fmlookup (HashMap t)
      (QueryIndexChallenge (PQueryCounter s) (PState s)) = Some raw
  using verifier-query-round-after-index-program-preserves-query-lookup
    [OF tail, of PQueryCounter s PState s] lookup-s0
  by simp
show ?thesis
proof (rule verifier-query-round-after-index-empty-composition-consistent
  [OF fl-empty tail])
  fix idx trace-openings
  assume idx-eq: idx = index (to-nat raw)
  and trace-indices:

```

```

    map opening-index trace-openings = powers-scaled idx
and trace-table:
    partial-authenticated-table fr (scale * clength) trace-openings t
and consistent:
    cp-eval as (map opening-value trace-openings) (h ^ idx * shift) =
      final
show ?thesis
  by (rule that[OF idx-eq trace-indices trace-table consistent
    lookup-t])
qed
qed

```

lemma *ntimes-verifier-query-rounds-empty-composition-consistent*:

fixes $s\ t :: ('f, 'a)\ \text{protocol-channel-scheme}$

assumes *fl-empty*: $fl = []$

and *outcome*:

Some (results, t) $\in$

set-dist

(*execute*

(*ntimes*

(*verifier-query-round-program* fr f-fl f-final as fl final) n)

s)

obtains *raw-idxs query-idxs trace-openings* **where**

length raw-idxs = n

query-idxs = map ($\lambda raw.\ index\ (to\text{-}nat\ raw)$) *raw-idxs*

length trace-openings = n

$\bigwedge i.\ i < n \implies$

map opening-index (*trace-openings* ! i) =

powers-scaled (*query-idxs* ! i)

$\bigwedge i.\ i < n \implies$

partial-authenticated-table fr (scale * clength)

(*trace-openings* ! i) t

$\bigwedge i.\ i < n \implies$

cp-eval as (*map opening-value* (*trace-openings* ! i))

(h ^ (*query-idxs* ! i) * shift) = final

proof –

have

$\exists$ *raw-idxs query-idxs trace-openings*.

length raw-idxs = n $\wedge$

query-idxs = map ($\lambda raw.\ index\ (to\text{-}nat\ raw)$) *raw-idxs* $\wedge$

length trace-openings = n $\wedge$

($\forall i < n.$

map opening-index (*trace-openings* ! i) =

powers-scaled (*query-idxs* ! i)) $\wedge$

($\forall i < n.$

partial-authenticated-table fr (scale * clength)

(*trace-openings* ! i) t) $\wedge$

($\forall i < n.$

cp-eval as (*map opening-value* (*trace-openings* ! i))

```

      (h ^ (query-idxs ! i) * shift) = final)
using outcome
proof (induction n arbitrary: s results t)
  case 0
  then show ?case
    by (intro exI[of - []]) simp
next
case (Suc n)
from Suc.premis obtain u results' s1 where
  head:
    Some (u, s1) ∈
      set-dist
        (execute
          (verifier-query-round-program fr f-fl f-final as fl final) s)
  and tail:
    Some (results', t) ∈
      set-dist
        (execute
          (ntimes
            (verifier-query-round-program fr f-fl f-final as fl final)
            n) s1)
  by (auto elim!: set-dist-bindE)
have u-eq: u = ()
  by (cases u) simp
from verifier-query-round-program-empty-composition-consistent-with-lookup
  [OF fl-empty head[unfolded u-eq]]
obtain raw idx openings where
  idx-eq: idx = index (to-nat raw)
  and idx-openings: map opening-index openings = powers-scaled idx
  and table-s1:
    partial-authenticated-table fr (scale * clength) openings s1
  and consistent-head:
    cp-eval as (map opening-value openings) (h ^ idx * shift) =
      final
  by blast
from Suc.IH[OF tail] obtain raw-tail idx-tail openings-tail where
  len-raw-tail: length raw-tail = n
  and idx-tail-eq:
    idx-tail = map (λraw. index (to-nat raw)) raw-tail
  and len-openings-tail: length openings-tail = n
  and tail-indices:
    ∀ i < n.
      map opening-index (openings-tail ! i) =
        powers-scaled (idx-tail ! i)
  and tail-tables:
    ∀ i < n.
      partial-authenticated-table fr (scale * clength)
        (openings-tail ! i) t
  and tail-consistent:

```

```

   $\forall i < n.$ 
     $cp\_eval\ as\ (map\ opening\_value\ (openings\_tail\ !\ i))$ 
     $(h \wedge (idx\_tail\ !\ i) * shift) = final$ 
  by blast
have s1-t:  $s1 \leq t$ 
  using ntimes-verifier-query-rounds-outcome[OF tail] by blast
have table-t:
  partial-authenticated-table fr (scale * clength) openings t
  by (rule partial-authenticated-table-mono[OF table-s1 s1-t])
let ?rows = raw # raw-tail
let ?idxs = idx # idx-tail
let ?openings = openings # openings-tail
have idxs-eq:
  ?idxs = map ( $\lambda raw. index\ (to\_nat\ raw)$ ) ?rows
  using idx-eq idx-tail-eq by simp
have indices-all:
   $\forall i < Suc\ n.$ 
    map opening-index (?openings ! i) =
    powers-scaled (?idxs ! i)
proof (intro allI impI)
  fix i
  assume i-bound:  $i < Suc\ n$ 
  show map opening-index (?openings ! i) =
    powers-scaled (?idxs ! i)
  proof (cases i)
    case 0
    then show ?thesis
      using idx-openings by simp
  next
    case (Suc j)
    then show ?thesis
      using i-bound tail-indices by simp
  qed
qed
have tables-all:
   $\forall i < Suc\ n.$ 
    partial-authenticated-table fr (scale * clength)
    (?openings ! i) t
proof (intro allI impI)
  fix i
  assume i-bound:  $i < Suc\ n$ 
  show partial-authenticated-table fr (scale * clength)
    (?openings ! i) t
  proof (cases i)
    case 0
    then show ?thesis
      using table-t by simp
  next
    case (Suc j)

```

```

    then show ?thesis
      using i-bound tail-tables by simp
  qed
qed
have consistent-all:
   $\forall i < \text{Suc } n.$ 
    cp-eval as (map opening-value (?openings ! i))
      ( $h \wedge (?idxs ! i) * \text{shift}$ ) = final
proof (intro allI impI)
  fix i
  assume i-bound:  $i < \text{Suc } n$ 
  show cp-eval as (map opening-value (?openings ! i))
    ( $h \wedge (?idxs ! i) * \text{shift}$ ) = final
  proof (cases i)
    case 0
    then show ?thesis
      using consistent-head by simp
  next
    case (Suc j)
    then show ?thesis
      using i-bound tail-consistent by simp
  qed
qed
show ?case
  apply (intro exI[of - ?rows] exI[of - ?idxs]
    exI[of - ?openings] conjI)
  apply (simp add: len-row-tail)
  apply (rule idxs-eq)
  apply (simp add: len-openings-tail)
  apply (rule indices-all)
  apply (rule tables-all)
  apply (rule consistent-all)
  done
qed
then obtain raw-idxs query-idxs trace-openings where
  len-row: length raw-idxs = n
  and query-idxs-eq:
    query-idxs = map ( $\lambda \text{raw}. \text{index } (\text{to-nat } \text{raw})$ ) raw-idxs
  and len-trace: length trace-openings = n
  and indices:
     $\forall i < n.$ 
      map opening-index (trace-openings ! i) =
        powers-scaled (query-idxs ! i)
  and tables:
     $\forall i < n.$ 
      partial-authenticated-table fr (scale * clength)
        (trace-openings ! i) t
  and consistent:
     $\forall i < n.$ 

```

```

      cp-eval as (map opening-value (trace-openings ! i))
      (h ^ (query-idxs ! i) * shift) = final
    by blast
  show ?thesis
  by (rule that[OF len-raw query-idxs-eq len-trace])
    (use indices tables consistent in auto)
qed

lemma ntimes-verifier-query-rounds-empty-composition-consistent-with-lookup:
fixes s t :: ('f, 'a) protocol-channel-scheme
assumes fl-empty: fl = []
and outcome:
  Some (results, t) ∈
    set-dist
      (execute
        (ntimes
          (verifier-query-round-program fr f-fl f-final as fl final) n)
        s)
obtains raw-idxs query-idxs query-chunks trace-openings where
  length raw-idxs = n
  query-idxs = map (λraw. index (to-nat raw)) raw-idxs
  length query-chunks = n
  length trace-openings = n
  ∧ i < n ⇒
    fmlookup (HashMap t)
      (QueryIndexChallenge (PQueryCounter s + i)
        (state-after-query-chunks (PState s) query-chunks i)) =
      Some (raw-idxs ! i)
  ∧ i < n ⇒
    map opening-index (trace-openings ! i) =
      powers-scaled (query-idxs ! i)
  ∧ i < n ⇒
    partial-authenticated-table fr (scale * clength)
      (trace-openings ! i) t
  ∧ i < n ⇒
    cp-eval as (map opening-value (trace-openings ! i))
      (h ^ (query-idxs ! i) * shift) = final
proof –
have
  ∃ raw-idxs query-idxs query-chunks trace-openings.
    length raw-idxs = n ∧
    query-idxs = map (λraw. index (to-nat raw)) raw-idxs ∧
    length query-chunks = n ∧
    length trace-openings = n ∧
    (∀ i < n.
      fmlookup (HashMap t)
        (QueryIndexChallenge (PQueryCounter s + i)
          (state-after-query-chunks (PState s) query-chunks i)) =
        Some (raw-idxs ! i)) ∧

```

```

(∀ i < n.
  map opening-index (trace-openings ! i) =
    powers-scaled (query-idxs ! i)) ∧
(∀ i < n.
  partial-authenticated-table fr (scale * clength)
    (trace-openings ! i) t) ∧
(∀ i < n.
  cp-eval as (map opening-value (trace-openings ! i))
    (h ^ (query-idxs ! i) * shift) = final)
using outcome
proof (induction n arbitrary: s results t)
case 0
then show ?case
  by (intro exI[of - []]) simp
next
case (Suc n)
from Suc.premis obtain u results' s1 where
  head:
    Some (u, s1) ∈
      set-dist
        (execute
          (verifier-query-round-program fr f-fl f-final as fl final) s)
  and tail:
    Some (results', t) ∈
      set-dist
        (execute
          (ntimes
            (verifier-query-round-program fr f-fl f-final as fl final)
            n) s1)
  by (auto elim!: set-dist-bindE)
have u-eq: u = ()
  by (cases u) simp
from verifier-query-round-program-empty-composition-consistent-with-lookup
  [OF fl-empty head[unfolded u-eq]]
obtain raw idx openings where
  idx-eq: idx = index (to-nat raw)
  and idx-openings: map opening-index openings = powers-scaled idx
  and table-s1:
    partial-authenticated-table fr (scale * clength) openings s1
  and consistent-head:
    cp-eval as (map opening-value openings) (h ^ idx * shift) =
      final
  and lookup-s1:
    fmlookup (HashMap s1)
      (QueryIndexChallenge (PQueryCounter s) (PState s)) = Some raw
  by blast
from verifier-query-round-program-outcome[OF head[unfolded u-eq]]
obtain raw' idx' chunk where
  idx'-eq: idx' = index (to-nat raw')

```



```

and st-s1: PState s1 = foldl concat (PState s) chunk
and query-count-s1: PQueryCounter s1 = Suc (PQueryCounter s)
and lookup-s1':
  fmlookup (HashMap s1)
  (QueryIndexChallenge (PQueryCounter s) (PState s)) = Some raw'
by blast
have raw'-eq: raw' = raw
  using lookup-s1 lookup-s1' by simp
from Suc.IH[OF tail] obtain raw-tail idx-tail chunk-tail openings-tail
where len-raw-tail: length raw-tail = n
and idx-tail-eq:
  idx-tail = map (λraw. index (to-nat raw)) raw-tail
and len-chunk-tail: length chunk-tail = n
and len-openings-tail: length openings-tail = n
and lookup-tail:
   $\forall i < n.$ 
  fmlookup (HashMap t)
  (QueryIndexChallenge (PQueryCounter s1 + i)
   (state-after-query-chunks (PState s1) chunk-tail i)) =
  Some (raw-tail ! i)
and tail-indices:
   $\forall i < n.$ 
  map opening-index (openings-tail ! i) =
  powers-scaled (idx-tail ! i)
and tail-tables:
   $\forall i < n.$ 
  partial-authenticated-table fr (scale * clength)
  (openings-tail ! i) t
and tail-consistent:
   $\forall i < n.$ 
  cp-eval as (map opening-value (openings-tail ! i))
  (h ^ (idx-tail ! i) * shift) = final
by blast
have s1-t: s1 ≤ t
  using ntimes-verifier-query-rounds-outcome[OF tail] by blast
have table-t:
  partial-authenticated-table fr (scale * clength) openings t
  by (rule partial-authenticated-table-mono[OF table-s1 s1-t])
have lookup-head-t:
  fmlookup (HashMap t)
  (QueryIndexChallenge (PQueryCounter s) (PState s)) = Some raw
  using hash-extension-lookup[OF lookup-s1 s1-t] .
let ?raws = raw # raw-tail
let ?idxs = idx # idx-tail
let ?chunks = chunk # chunk-tail
let ?openings = openings # openings-tail
have idxs-eq:
  ?idxs = map (λraw. index (to-nat raw)) ?raws
  using idx-eq idx-tail-eq by simp

```

```

have state-shift:
   $\wedge i. \text{state-after-query-chunks } (PState\ s) \ ?chunks\ (Suc\ i) =$ 
     $\text{state-after-query-chunks } (PState\ s1) \ \text{chunk-tail } i$ 
  using st-s1 unfolding state-after-query-chunks-def by simp
have lookup-all:
   $\forall i < Suc\ n.$ 
     $\text{fmlookup } (HashMap\ t)$ 
       $(QueryIndexChallenge\ (PQueryCounter\ s + i)$ 
         $(\text{state-after-query-chunks } (PState\ s) \ ?chunks\ i)) =$ 
         $Some\ (?raws\ !\ i)$ 
proof (intro allI impI)
  fix i
  assume i-bound:  $i < Suc\ n$ 
  show  $\text{fmlookup } (HashMap\ t)$ 
     $(QueryIndexChallenge\ (PQueryCounter\ s + i)$ 
       $(\text{state-after-query-chunks } (PState\ s) \ ?chunks\ i)) =$ 
       $Some\ (?raws\ !\ i)$ 
  proof (cases i)
    case 0
    then show ?thesis
      using lookup-head-t unfolding state-after-query-chunks-def by simp
  next
    case  $(Suc\ j)$ 
    then show ?thesis
      using i-bound lookup-tail query-count-s1 state-shift[of j] by simp
  qed
qed
have indices-all:
   $\forall i < Suc\ n.$ 
     $\text{map opening-index } (?openings\ !\ i) =$ 
     $\text{powers-scaled } (?idxs\ !\ i)$ 
proof (intro allI impI)
  fix i
  assume i-bound:  $i < Suc\ n$ 
  show  $\text{map opening-index } (?openings\ !\ i) =$ 
     $\text{powers-scaled } (?idxs\ !\ i)$ 
  proof (cases i)
    case 0
    then show ?thesis
      using idx-openings by simp
  next
    case  $(Suc\ j)$ 
    then show ?thesis
      using i-bound tail-indices by simp
  qed
qed
have tables-all:
   $\forall i < Suc\ n.$ 
     $\text{partial-authenticated-table } fr\ (scale * \text{clength})$ 

```

```

      (?openings ! i) t
proof (intro allI impI)
  fix i
  assume i-bound:  $i < \text{Suc } n$ 
  show partial-authenticated-table fr (scale * clength)
    (?openings ! i) t
proof (cases i)
  case 0
  then show ?thesis
    using table-t by simp
next
  case (Suc j)
  then show ?thesis
    using i-bound tail-tables by simp
qed
qed
have consistent-all:
   $\forall i < \text{Suc } n.$ 
    cp-eval as (map opening-value (?openings ! i))
      ( $h \wedge (?idxs ! i) * \text{shift}$ ) = final
proof (intro allI impI)
  fix i
  assume i-bound:  $i < \text{Suc } n$ 
  show cp-eval as (map opening-value (?openings ! i))
    ( $h \wedge (?idxs ! i) * \text{shift}$ ) = final
proof (cases i)
  case 0
  then show ?thesis
    using consistent-head by simp
next
  case (Suc j)
  then show ?thesis
    using i-bound tail-consistent by simp
qed
qed
show ?case
  by (intro exI[of - ?rows] exI[of - ?idxs] exI[of - ?chunks]
    exI[of - ?openings] conjI)
    (use len-raw-tail idxs-eq len-chunk-tail len-openings-tail
      lookup-all indices-all tables-all consistent-all in simp-all)
qed
then obtain raw-idxs query-idxs query-chunks trace-openings where
  len-raw: length raw-idxs = n
  and query-idxs-eq:
    query-idxs = map ( $\lambda \text{raw}. \text{index } (\text{to-nat } \text{raw})$ ) raw-idxs
  and len-chunks: length query-chunks = n
  and len-trace: length trace-openings = n
  and lookup:
     $\forall i < n.$ 

```

```

    fmlookup (HashMap t)
      (QueryIndexChallenge (PQueryCounter s + i)
        (state-after-query-chunks (PState s) query-chunks i)) =
    Some (raw-idxs ! i)
  and indices:
    ∀ i < n.
      map opening-index (trace-openings ! i) =
        powers-scaled (query-idxs ! i)
  and tables:
    ∀ i < n.
      partial-authenticated-table fr (scale * clength)
        (trace-openings ! i) t
  and consistent:
    ∀ i < n.
      cp-eval as (map opening-value (trace-openings ! i))
        (h ^ (query-idxs ! i) * shift) = final
  by blast
  show ?thesis
  by (rule that[OF len-raw query-idxs-eq len-chunks len-trace])
    (use lookup indices tables consistent in auto)
qed

lemma ntimes-verifier-query-rounds-empty-composition-first-consistent:
  fixes s t :: ('f, 'a) protocol-channel-scheme
  assumes fl-empty: fl = []
  and outcome:
    Some (results, t) ∈
      set-dist
        (execute
          (ntimes
            (verifier-query-round-program fr f-fl f-final as fl final)
            rounds)
          s)
  obtains raw trace-openings where
    map opening-index trace-openings =
      powers-scaled (index (to-nat raw))
    partial-authenticated-table fr (scale * clength) trace-openings t
    cp-eval as (map opening-value trace-openings)
      (h ^ index (to-nat raw) * shift) = final
    fmlookup (HashMap t)
      (QueryIndexChallenge (PQueryCounter s) (PState s)) = Some raw
proof –
  from outcome obtain u results' s1 where
    head:
      Some (u, s1) ∈
        set-dist
          (execute
            (verifier-query-round-program fr f-fl f-final as fl final) s)
  and tail:

```

```

    Some (results', t) ∈
      set-dist
        (execute
          (ntimes
            (verifier-query-round-program fr f-fl f-final as fl final)
            (rounds - Suc 0))
          s1)
  using rounds-positive
  by (cases rounds) (auto elim!: set-dist-bindE)
  have u-eq: u = ()
  by (cases u) simp
  from verifier-query-round-program-empty-composition-consistent-with-lookup
    [OF fl-empty head[unfolded u-eq]]
  obtain raw idx trace-openings where idx-eq:
    idx = index (to-nat raw)
  and trace-indices:
    map opening-index trace-openings = powers-scaled idx
  and trace-table-s1:
    partial-authenticated-table fr (scale * clength) trace-openings s1
  and consistent:
    cp-eval as (map opening-value trace-openings) (h ^ idx * shift) =
      final
  and lookup-s1:
    fmlookup (HashMap s1)
    (QueryIndexChallenge (PQueryCounter s) (PState s)) = Some raw
  by blast
  have s1-t: s1 ≤ t
  using ntimes-verifier-query-rounds-outcome[OF tail] by blast
  have trace-table-t:
    partial-authenticated-table fr (scale * clength) trace-openings t
  by (rule partial-authenticated-table-mono[OF trace-table-s1 s1-t])
  have lookup-t:
    fmlookup (HashMap t)
    (QueryIndexChallenge (PQueryCounter s) (PState s)) = Some raw
  using hash-extension-lookup[OF lookup-s1 s1-t] .
  show ?thesis
  by (rule that[of trace-openings raw])
    (use idx-eq trace-indices trace-table-t consistent lookup-t in simp-all)
qed

```

lemma *ntimes-verifier-query-rounds-authenticated-openings-consistent-at:*
fixes $s\ t :: ('f, 'a)\ \text{protocol-channel-scheme}$
assumes $fl\text{-eq}: fl = (b, \text{composition-root}) \# fl\text{-tail}$
and *outcome:*
 Some (results, t) ∈
 set-dist
 (execute
 (ntimes
 (verifier-query-round-program fr f-fl f-final as fl final) n)

```

      s)
    and i-bound:  $i < n$ 
  obtains raw idx prefix-chunks trace-openings composition-openings where
    idx = index (to-nat raw)
    length prefix-chunks = i
    map opening-index trace-openings = powers-scaled idx
    partial-authenticated-table fr (scale * clength) trace-openings t
    partial-authenticated-table composition-root (scale * clength)
      composition-openings t
    map opening-index composition-openings =
      [idx, fri-sibling-index (scale * clength) idx]
    partial-query-openings-consistent trace-openings composition-openings
      as idx
    fmlookup (HashMap t)
      (QueryIndexChallenge (PQueryCounter s + i)
        (state-after-query-chunks (PState s) prefix-chunks i)) = Some raw
  proof -
  from query-opening-ntimes-outcome-decomp-at[OF outcome i-bound]
  obtain prefix round suffix s-i s-suc where
    prefix:
      Some (prefix, s-i) ∈
        set-dist
          (execute
            (ntimes
              (verifier-query-round-program fr f-fl f-final as fl final) i)
            s)
    and round:
      Some (round, s-suc) ∈
        set-dist
          (execute
            (verifier-query-round-program fr f-fl f-final as fl final) s-i)
    and suffix:
      Some (suffix, t) ∈
        set-dist
          (execute
            (ntimes
              (verifier-query-round-program fr f-fl f-final as fl final)
              (n - Suc i))
            s-suc)
  by blast
  have round-unit: round = ()
  by (cases round) simp
  show ?thesis
  proof (rule
    verifier-query-round-program-authenticated-openings-consistent-with-lookup
      [OF fl-eq round[unfolded round-unit]])
    fix raw idx trace-openings composition-openings
    assume idx-eq: idx = index (to-nat raw)
    and trace-indices:

```

```

    map opening-index trace-openings = powers-scaled idx
and trace-table-suc:
    partial-authenticated-table fr (scale * clength)
    trace-openings s-suc
and comp-table-suc:
    partial-authenticated-table composition-root (scale * clength)
    composition-openings s-suc
and comp-indices:
    map opening-index composition-openings =
    [idx, fri-sibling-index (scale * clength) idx]
and consistent:
    partial-query-openings-consistent trace-openings
    composition-openings as idx
and lookup-suc:
    fmlookup (HashMap s-suc)
    (QueryIndexChallenge (PQueryCounter s-i) (PState s-i)) =
    Some raw
show ?thesis
proof (rule query-opening-ntimes-prefix-state-alignment[OF prefix])
fix prefix-chunks
assume len-chunks: length prefix-chunks = i
and state-i:
    PState s-i =
    state-after-query-chunks (PState s) prefix-chunks i
and counter-i: PQueryCounter s-i = PQueryCounter s + i
have suffix-ext: s-suc ≤ t
by (rule query-opening-ntimes-suffix-extends[OF suffix])
have trace-table-t:
    partial-authenticated-table fr (scale * clength)
    trace-openings t
by (rule
    partial-authenticated-table-mono[OF trace-table-suc suffix-ext])
have comp-table-t:
    partial-authenticated-table composition-root (scale * clength)
    composition-openings t
by (rule
    partial-authenticated-table-mono[OF comp-table-suc suffix-ext])
have lookup-t:
    fmlookup (HashMap t)
    (QueryIndexChallenge (PQueryCounter s + i)
    (state-after-query-chunks (PState s) prefix-chunks i)) =
    Some raw
using hash-extension-lookup[OF lookup-suc suffix-ext]
unfolding counter-i state-i .
show ?thesis
by (rule that[OF idx-eq len-chunks trace-indices trace-table-t
    comp-table-t comp-indices consistent lookup-t])
qed
qed

```

qed

lemma *ntimes-verifier-query-rounds-empty-composition-consistent-at:*

fixes $s\ t :: ('f, 'a)\ \text{protocol-channel-scheme}$

assumes *fl-empty*: $fl = []$

and *outcome*:

$\text{Some } (results, t) \in$

set-dist

$(\text{execute}$

$(\text{ntimes}$

$(\text{verifier-query-round-program } fr\ f\text{-}fl\ f\text{-}final\ \text{as } fl\ final)\ n)$

$s)$

and *i-bound*: $i < n$

obtains *raw idx prefix-chunks trace-openings* **where**

$idx = \text{index } (to\text{-}nat\ raw)$

$\text{length } prefix\text{-}chunks = i$

$\text{map } opening\text{-}index\ trace\text{-}openings = \text{powers-scaled } idx$

$\text{partial-authenticated-table } fr\ (scale * \text{clength})\ trace\text{-}openings\ t$

$\text{cp-eval } as\ (\text{map } opening\text{-}value\ trace\text{-}openings)\ (h \wedge idx * shift) =$

final

$\text{fmlookup } (HashMap\ t)$

$(\text{QueryIndexChallenge } (PQueryCounter\ s + i)$

$(\text{state-after-query-chunks } (PState\ s)\ prefix\text{-}chunks\ i)) = \text{Some } raw$

proof –

from *query-opening-ntimes-outcome-decomp-at*[*OF outcome i-bound*]

obtain *prefix round suffix s-i s-suc* **where**

prefix:

$\text{Some } (prefix, s\text{-}i) \in$

set-dist

$(\text{execute}$

$(\text{ntimes}$

$(\text{verifier-query-round-program } fr\ f\text{-}fl\ f\text{-}final\ \text{as } fl\ final)\ i)$

$s)$

and *round*:

$\text{Some } (round, s\text{-}suc) \in$

set-dist

$(\text{execute}$

$(\text{verifier-query-round-program } fr\ f\text{-}fl\ f\text{-}final\ \text{as } fl\ final)\ s\text{-}i)$

and *suffix*:

$\text{Some } (suffix, t) \in$

set-dist

$(\text{execute}$

$(\text{ntimes}$

$(\text{verifier-query-round-program } fr\ f\text{-}fl\ f\text{-}final\ \text{as } fl\ final)$

$(n - Suc\ i))$

$s\text{-}suc)$

by *blast*

have *round-unit*: $round = ()$

by $(\text{cases } round)\ \text{simp}$


```

show ?thesis
proof (rule
  verifier-query-round-program-empty-composition-consistent-with-lookup
  [OF fl-empty round[unfolded round-unit]])
fix raw idx trace-openings
assume idx-eq: idx = index (to-nat raw)
and trace-indices:
  map opening-index trace-openings = powers-scaled idx
and trace-table-suc:
  partial-authenticated-table fr (scale * clength)
  trace-openings s-suc
and consistent:
  cp-eval as (map opening-value trace-openings) (h ^ idx * shift) =
  final
and lookup-suc:
  fmlookup (HashMap s-suc)
  (QueryIndexChallenge (PQueryCounter s-i) (PState s-i)) =
  Some raw
show ?thesis
proof (rule query-opening-ntimes-prefix-state-alignment[OF prefix])
fix prefix-chunks
assume len-chunks: length prefix-chunks = i
and state-i:
  PState s-i =
  state-after-query-chunks (PState s) prefix-chunks i
and counter-i: PQueryCounter s-i = PQueryCounter s + i
have suffix-ext: s-suc ≤ t
by (rule query-opening-ntimes-suffix-extends[OF suffix])
have trace-table-t:
  partial-authenticated-table fr (scale * clength)
  trace-openings t
by (rule
  partial-authenticated-table-mono[OF trace-table-suc suffix-ext])
have lookup-t:
  fmlookup (HashMap t)
  (QueryIndexChallenge (PQueryCounter s + i)
  (state-after-query-chunks (PState s) prefix-chunks i)) =
  Some raw
using hash-extension-lookup[OF lookup-suc suffix-ext]
unfolding counter-i state-i .
show ?thesis
by (rule that[OF idx-eq len-chunks trace-indices trace-table-t
  consistent lookup-t])

qed
qed
qed

lemma ntimes-verifier-query-rounds-selected-authenticated-chunks-at:
fixes s t :: ('f, 'a) protocol-channel-scheme

```

```

assumes fl-eq: fl = (b, composition-root) # fl-tail
and outcome:
  Some (results, t) ∈
    set-dist
      (execute
        (ntimes
          (verifier-query-round-program fr f-fl f-final as fl final) n)
        s)
and i-bound: i < n
obtains raw idx prefix-query-idxs prefix-chunks chunk suffix-query-idxs
  suffix-chunks trace-openings composition-openings where
idx = index (to-nat raw)
length prefix-query-idxs = i
length prefix-chunks = i
length suffix-query-idxs = n - Suc i
length suffix-chunks = n - Suc i
verifier-query-round-chunk idx (map snd f-fl) (map snd fl) chunk
∧j. j < i ⇒
  verifier-query-round-chunk (prefix-query-idxs ! j)
    (map snd f-fl) (map snd fl) (prefix-chunks ! j)
∧j. j < n - Suc i ⇒
  verifier-query-round-chunk (suffix-query-idxs ! j)
    (map snd f-fl) (map snd fl) (suffix-chunks ! j)
PTranscript s =
  List.concat (prefix-chunks @ chunk # suffix-chunks) @ PTranscript t
map opening-index trace-openings = powers-scaled idx
partial-authenticated-table fr (scale * clength) trace-openings t
partial-authenticated-table composition-root (scale * clength)
  composition-openings t
map opening-index composition-openings =
  [idx, fri-sibling-index (scale * clength) idx]
partial-query-openings-consistent trace-openings composition-openings
  as idx
fmlookup (HashMap t)
  (QueryIndexChallenge (PQueryCounter s + i)
    (state-after-query-chunks (PState s) prefix-chunks i)) = Some raw
proof -
from query-opening-ntimes-outcome-decomp-at[OF outcome i-bound]
obtain prefix round suffix s-i s-suc where
prefix:
  Some (prefix, s-i) ∈
    set-dist
      (execute
        (ntimes
          (verifier-query-round-program fr f-fl f-final as fl final) i)
        s)
and round:
  Some (round, s-suc) ∈
    set-dist

```

```

      (execute
        (verifier-query-round-program fr f-fl f-final as fl final) s-i)
and suffix:
  Some (suffix, t) ∈
    set-dist
      (execute
        (ntimes
          (verifier-query-round-program fr f-fl f-final as fl final)
          (n - Suc i))
        s-suc)
by blast
have round-unit: round = ()
  by (cases round) simp
show ?thesis
proof (rule query-opening-ntimes-prefix-transcript-state-alignment
  [OF prefix])
fix prefix-query-idxs prefix-chunks
assume len-prefix: length prefix-query-idxs = i
  and len-prefix-chunks: length prefix-chunks = i
  and transcript-prefix:
    PTranscript s = List.concat prefix-chunks @ PTranscript s-i
  and state-i:
    PState s-i =
      state-after-query-chunks (PState s) prefix-chunks i
  and counter-i: PQueryCounter s-i = PQueryCounter s + i
  and prefix-ext: s ≤ s-i
  and prefix-rounds:
    ∧j. j < i ⇒
      verifier-query-round-chunk (prefix-query-idxs ! j)
      (map snd f-fl) (map snd fl) (prefix-chunks ! j)
show ?thesis
proof (rule
  query-opening-verifier-query-round-program-authenticated-with-chunk
  [OF fl-eq round[unfolded round-unit]])
fix raw idx chunk trace-leaves trace-openings composition-openings
assume idx-eq: idx = index (to-nat raw)
  and chunk-shape:
    verifier-query-round-chunk idx (map snd f-fl) (map snd fl)
    chunk
  and transcript-round:
    PTranscript s-i = chunk @ PTranscript s-suc
  and state-suc:
    PState s-suc = foldl concat (PState s-i) chunk
  and round-ext: s-i ≤ s-suc
  and counter-suc: PQueryCounter s-suc = Suc (PQueryCounter s-i)
  and trace-values:
    map opening-value trace-openings = trace-leaves
  and trace-indices:
    map opening-index trace-openings = powers-scaled idx

```

```

and trace-table-suc:
  partial-authenticated-table fr (scale * clength)
  trace-openings s-suc
and comp-table-suc:
  partial-authenticated-table composition-root (scale * clength)
  composition-openings s-suc
and comp-indices:
  map opening-index composition-openings =
    [idx, fri-sibling-index (scale * clength) idx]
and lookup-suc:
  fmlookup (HashMap s-suc)
  (QueryIndexChallenge (PQueryCounter s-i) (PState s-i)) =
    Some raw
show ?thesis
proof (rule verifier-query-round-program-authenticated-openings-consistent-with-lookup
  [OF fl-eq round[unfolded round-unit]])
fix raw' idx' trace-openings' composition-openings'
assume idx'-eq: idx' = index (to-nat raw')
  and trace-indices':
    map opening-index trace-openings' = powers-scaled idx'
  and trace-table-suc':
    partial-authenticated-table fr (scale * clength)
    trace-openings' s-suc
  and comp-table-suc':
    partial-authenticated-table composition-root (scale * clength)
    composition-openings' s-suc
  and comp-indices':
    map opening-index composition-openings' =
      [idx', fri-sibling-index (scale * clength) idx']
  and consistent':
    partial-query-openings-consistent trace-openings'
    composition-openings' as idx'
  and lookup-suc':
    fmlookup (HashMap s-suc)
    (QueryIndexChallenge (PQueryCounter s-i) (PState s-i)) =
      Some raw'
have raw'-eq: raw' = raw
  using lookup-suc lookup-suc' by simp
have idx'-idx: idx' = idx
  using idx'-eq idx-eq raw'-eq by simp
have suffix-ext: s-suc ≤ t
  by (rule query-opening-ntimes-suffix-extends[OF suffix])
have trace-table-t:
  partial-authenticated-table fr (scale * clength)
  trace-openings' t
  by (rule partial-authenticated-table-mono
    [OF trace-table-suc' suffix-ext])
have comp-table-t:
  partial-authenticated-table composition-root (scale * clength)

```

```

    composition-openings' t
  by (rule partial-authenticated-table-mono
    [OF comp-table-suc' suffix-ext])
have lookup-t:
  fmlookup (HashMap t)
    (QueryIndexChallenge (PQueryCounter s + i)
      (state-after-query-chunks (PState s) prefix-chunks i)) =
    Some raw
  using hash-extension-lookup[OF lookup-suc suffix-ext]
  unfolding counter-i state-i .
show ?thesis
proof (rule query-opening-ntimes-suffix-transcript-state-alignment
  [OF suffix])
fix suffix-query-idxs suffix-chunks
assume len-suffix-idxs:
  length suffix-query-idxs = n - Suc i
and len-suffix-chunks:
  length suffix-chunks = n - Suc i
and transcript-suffix:
  PTranscript s-suc =
    List.concat suffix-chunks @ PTranscript t
and suffix-state:
  PState t =
    state-after-query-chunks (PState s-suc) suffix-chunks
    (n - Suc i)
and suffix-counter:
  PQueryCounter t = PQueryCounter s-suc + (n - Suc i)
and suffix-ext2: s-suc ≤ t
and suffix-rounds:
  ∧j. j < n - Suc i ⇒
    verifier-query-round-chunk (suffix-query-idxs ! j)
    (map snd f-fl) (map snd fl) (suffix-chunks ! j)
have transcript-all:
  PTranscript s =
    List.concat (prefix-chunks @ chunk # suffix-chunks) @
    PTranscript t
  using transcript-prefix transcript-round transcript-suffix
  by simp
show ?thesis
  by (rule that[OF idx-eq len-prefix len-prefix-chunks
    len-suffix-idxs len-suffix-chunks chunk-shape - -
    transcript-all - trace-table-t comp-table-t - - lookup-t])
    (use trace-indices' comp-indices' consistent' idx'-idx
      prefix-rounds suffix-rounds in simp-all)
qed
qed
qed
qed
qed

```

lemma *ntimes-verifier-query-rounds-aligned-authenticated-openings-consistent*:

fixes $s\ t :: ('f, 'a) \text{ protocol-channel-scheme}$

assumes $fl\text{-eq}: fl = (b, \text{composition-root}) \# fl\text{-tail}$

and *outcome*:

$\text{Some } (results, t) \in$

set-dist

$(\text{execute}$

$(\text{ntimes}$

$(\text{verifier-query-round-program } fr\ f\text{-}fl\ f\text{-}final\ \text{as } fl\ final)$

$\text{rounds})$

$s)$

obtains $query\text{-idxs}\ trace\text{-openings}\ composition\text{-openings}$ **where**

$\text{length } query\text{-idxs} = \text{rounds}$

$\text{length } trace\text{-openings} = \text{rounds}$

$\text{length } composition\text{-openings} = \text{rounds}$

$\bigwedge i. i < \text{rounds} \implies$

$\text{map } opening\text{-index } (trace\text{-openings } !\ i) =$

$\text{powers-scaled } (query\text{-idxs } !\ i)$

$\bigwedge i. i < \text{rounds} \implies$

$\text{partial-authenticated-table } fr\ (\text{scale} * \text{clength})$

$(trace\text{-openings } !\ i)\ t$

$\bigwedge i. i < \text{rounds} \implies$

$\text{partial-authenticated-table } composition\text{-root } (\text{scale} * \text{clength})$

$(composition\text{-openings } !\ i)\ t$

$\bigwedge i. i < \text{rounds} \implies$

$\text{map } opening\text{-index } (composition\text{-openings } !\ i) =$

$[query\text{-idxs } !\ i,$

$fri\text{-sibling-index } (\text{scale} * \text{clength})\ (query\text{-idxs } !\ i)]$

$\bigwedge i. i < \text{rounds} \implies$

$\text{partial-query-round-consistent } trace\text{-openings}\ composition\text{-openings}\ \text{as } i$

$(query\text{-idxs } !\ i)$

proof $(\text{cases } \text{rounds} = 0)$

case *True*

show *?thesis*

by $(\text{rule that}[of\ \square\ \square\ \square])\ (\text{use } \text{True}\ \text{in}\ \text{simp-all})$

next

case *nonzero: False*

let *?P* =

$\lambda i\ w. \text{map } opening\text{-index } (fst\ (snd\ w)) = \text{powers-scaled } (fst\ w) \wedge$

$\text{partial-authenticated-table } fr\ (\text{scale} * \text{clength})\ (fst\ (snd\ w))\ t \wedge$

$\text{partial-authenticated-table } composition\text{-root } (\text{scale} * \text{clength})$

$(snd\ (snd\ w))\ t \wedge$

$\text{map } opening\text{-index } (snd\ (snd\ w)) =$

$[fst\ w, fri\text{-sibling-index } (\text{scale} * \text{clength})\ (fst\ w)] \wedge$

$\text{partial-query-openings-consistent } (fst\ (snd\ w))\ (snd\ (snd\ w))$

$\text{as } (fst\ w)$

let *?wit* = $\lambda i. \text{SOME } w. ?P\ i\ w$

let *?idx* = $\lambda i. fst\ (?wit\ i)$

```

let ?trace-openings = λi. fst (snd (?wit i))
let ?composition-openings = λi. snd (snd (?wit i))
let ?query-idxs = map ?idx [0..<rounds]
let ?trace-tables = map ?trace-openings [0..<rounds]
let ?comp-tables = map ?composition-openings [0..<rounds]
have ex-round:
  ∧i. i < rounds ⇒
    ∃ w. ?P i w
proof -
  fix i
  assume i-bound: i < rounds
  show ∃ w. ?P i w
  proof (rule ntimes-verifier-query-rounds-selected-authenticated-chunks-at
    [OF fl-eq outcome i-bound])
    fix raw idx prefix-query-idxs prefix-chunks chunk suffix-query-idxs
      suffix-chunks trace-openings composition-openings
    assume idx-eq: idx = index (to-nat raw)
    and trace-indices:
      map opening-index trace-openings = powers-scaled idx
    and trace-table:
      partial-authenticated-table fr (scale * clength) trace-openings t
    and comp-table:
      partial-authenticated-table composition-root (scale * clength)
        composition-openings t
    and comp-indices:
      map opening-index composition-openings =
        [idx, fri-sibling-index (scale * clength) idx]
    and consistent:
      partial-query-openings-consistent trace-openings
        composition-openings as idx
    show ?thesis
    by (intro exI[of - (idx, trace-openings, composition-openings)]
      conjI)
      (use idx-eq trace-indices trace-table comp-table comp-indices
        consistent in simp-all)
  qed
qed
have round-props:
  ∧i. i < rounds ⇒
    map opening-index (?trace-tables ! i) =
      powers-scaled (?query-idxs ! i) ∧
    partial-authenticated-table fr (scale * clength)
      (?trace-tables ! i) t ∧
    partial-authenticated-table composition-root (scale * clength)
      (?comp-tables ! i) t ∧
    map opening-index (?comp-tables ! i) =
      [?query-idxs ! i,
        fri-sibling-index (scale * clength) (?query-idxs ! i)] ∧
    partial-query-round-consistent ?trace-tables ?comp-tables as i

```

```

    (?query-idxs ! i)
proof –
  fix i
  assume i-bound:  $i < \text{rounds}$ 
  have chosen:  $?P\ i\ (?wit\ i)$ 
    by (rule someI-ex[OF ex-round[OF i-bound]])
  have nths:
    ?query-idxs ! i = ?idx i
    ?trace-tables ! i = ?trace-openings i
    ?comp-tables ! i = ?composition-openings i
    using i-bound by simp-all
  have lengths:
    length ?trace-tables = rounds
    length ?comp-tables = rounds
    by simp-all
  show
    map opening-index (?trace-tables ! i) =
      powers-scaled (?query-idxs ! i)  $\wedge$ 
      partial-authenticated-table fr (scale * clength)
      (?trace-tables ! i) t  $\wedge$ 
      partial-authenticated-table composition-root (scale * clength)
      (?comp-tables ! i) t  $\wedge$ 
      map opening-index (?comp-tables ! i) =
      [?query-idxs ! i,
       fri-sibling-index (scale * clength) (?query-idxs ! i)]  $\wedge$ 
      partial-query-round-consistent ?trace-tables ?comp-tables as i
      (?query-idxs ! i)
    using chosen i-bound nths lengths
    unfolding partial-query-round-consistent-def
      partial-query-openings-consistent-def
    by simp
  qed
show ?thesis
  by (rule that[of ?query-idxs ?trace-tables ?comp-tables])
    (use round-props in simp-all)
qed

end

end

```

```

theory Soundness-Partial-Initial-Aligned
  imports Soundness-Query-Opening-Consistency
begin

```

Aligned partial initial-opening evidence.

The predicate *accepted-with-partial-initial-openings* records trace and composition openings with separate query-index lists. For query soundness bounds this theory packages

the stronger verifier-local fact that both opening families come from the same query-round execution and therefore use the same sampled query indices.

context *soundness*

begin

definition *accepted-with-partial-initial-openings-aligned*

$:: ('f, 'a) \text{ protocol-channel-scheme} \Rightarrow$
 $(\text{unit list} \times ('f, 'a) \text{ protocol-channel-scheme}) \text{ option} \Rightarrow$
 $'f \Rightarrow 'f \text{ list} \Rightarrow 'f \Rightarrow 'f \text{ list} \Rightarrow 'f \Rightarrow$
 $'f \text{ list} \Rightarrow 'f \Rightarrow \text{nat list} \Rightarrow$
 $'f \text{ authenticated-opening list list} \Rightarrow$
 $'f \text{ authenticated-opening list list} \Rightarrow \text{bool}$

where

accepted-with-partial-initial-openings-aligned s out fr f-fri-roots
f-final as dg composition-fri-roots final query-idxs trace-openings
composition-openings $\longleftrightarrow$
accepted out $\wedge$
composition-fri-roots $\neq [] \wedge$
 $(\exists \text{ rest.}$
verifier-header-transcript s fr f-fri-roots f-final as dg
composition-fri-roots final rest) $\wedge$
accepted-with-partial-trace-openings s out fr query-idxs
trace-openings $\wedge$
accepted-with-partial-composition-openings s out
 $(\text{hd } \text{composition-fri-roots}) \text{ query-idxs } \text{composition-openings}$

lemma *accepted-with-partial-initial-openings-aligned-imp-unaligned:*

assumes

accepted-with-partial-initial-openings-aligned s out fr f-fri-roots
f-final as dg composition-fri-roots final query-idxs trace-openings
composition-openings

shows

accepted-with-partial-initial-openings s out fr f-fri-roots f-final as dg
composition-fri-roots final query-idxs trace-openings query-idxs
composition-openings

using *assms*

unfolding *accepted-with-partial-initial-openings-aligned-def*

accepted-with-partial-initial-openings-def

by *blast*

lemma *accepted-with-partial-initial-openings-aligned-shapes:*

assumes

accepted-with-partial-initial-openings-aligned s out fr f-fri-roots
f-final as dg composition-fri-roots final query-idxs trace-openings
composition-openings

shows *composition-fri-roots* $\neq []$

and $\exists \text{ rest. } \text{verifier-header-transcript s fr f-fri-roots f-final as dg}$
composition-fri-roots final rest

and *accepted-with-partial-trace-openings s out fr query-idxs*

```

    trace-openings
  and accepted-with-partial-composition-openings s out
    (hd composition-fri-roots) query-idxs composition-openings
  using assms
  unfolding accepted-with-partial-initial-openings-aligned-def by blast+

lemma accepted-with-partial-initial-openings-aligned-candidates-if-no-partial-merkle-bad:
  assumes partial:
    accepted-with-partial-initial-openings-aligned s
    (Some (result, final-state)) fr f-fri-roots f-final as dg
    composition-fri-roots final query-idxs trace-openings
    composition-openings
  and no-bad:
    ¬ partial-merkle-inconsistency-bad s
    (Some (result, final-state))
  shows ∃ trace-table composition-table.
    partial-trace-table-candidate trace-table trace-openings ∧
    partial-composition-table-candidate composition-table
    composition-openings
proof -
  have unaligned:
    accepted-with-partial-initial-openings s
    (Some (result, final-state)) fr f-fri-roots f-final as dg
    composition-fri-roots final query-idxs trace-openings query-idxs
    composition-openings
  by (rule accepted-with-partial-initial-openings-aligned-imp-unaligned
    [OF partial])
  show ?thesis
  by (rule
    accepted-with-partial-initial-openings-candidates-if-no-partial-merkle-bad
    [OF unaligned no-bad])
qed

lemma verify-monad-accepted-with-partial-initial-openings-aligned:
  fixes s :: ('f, 'a) protocol-channel-scheme
  assumes outcome:
    Some (result, final-state) ∈ set-dist (execute verify-monad s)
  and header0:
    verifier-header-transcript s fr f-fri-roots f-final as dg
    composition-fri-roots final rest
  and comp-nonempty: composition-fri-roots ≠ []
  obtains query-idxs trace-openings composition-openings where
    accepted-with-partial-initial-openings-aligned s
    (Some (result, final-state)) fr f-fri-roots f-final as dg
    composition-fri-roots final query-idxs trace-openings
    composition-openings
  ∧ i. i < rounds ⇒
    partial-query-round-consistent trace-openings composition-openings as i
    (query-idxs ! i)

```

proof –

from *verify-monad-header-extraction*[*OF outcome*]

obtain *fr' f-fl f-final' as' dg' fl final' query-state* **where**

header:

verifier-header-transcript s fr' (map snd f-fl) f-final' as' dg'
(map snd fl) final' (PTranscript query-state)

and *query-out*:

Some (result, final-state) ∈
set-dist
(execute
(ntimes
(verifier-query-round-program fr' f-fl f-final' as' fl
final') rounds)
query-state)

by *metis*

have *eqs*:

fr' = fr ∧
map snd f-fl = f-fri-roots ∧
f-final' = f-final ∧
as' = as ∧
dg' = dg ∧
map snd fl = composition-fri-roots ∧
final' = final ∧
PTranscript query-state = rest

using *verifier-header-transcript-unique*[*OF header header0*] **by** *simp*

have *fl-nonempty*: *fl ≠ []*

using *comp-nonempty eqs* **by** (*cases fl*) *simp-all*

then obtain *b composition-root fl-tail* **where** *fl-eq*:

fl = (b, composition-root) # fl-tail

by (*cases fl*) *auto*

have *root-eq*: *composition-root = hd composition-fri-roots*

proof –

have *composition-root # map snd fl-tail = composition-fri-roots*

using *eqs unfolding fl-eq* **by** *simp*

then have *hd composition-fri-roots = composition-root*

by (*metis list.sel(1)*)

then show *?thesis*

by *simp*

qed

obtain *query-idxs trace-openings composition-openings* **where**

len-query-idxs: *length query-idxs = rounds*

and *len-trace*: *length trace-openings = rounds*

and *len-comp*: *length composition-openings = rounds*

and *trace-indices*:

$\bigwedge i. i < rounds \implies$
 $\text{map opening-index (trace-openings ! } i) =$
 $\text{powers-scaled (query-idxs ! } i)$

and *trace-tables*:

$\bigwedge i. i < rounds \implies$

```

    partial-authenticated-table fr' (scale * clength)
      (trace-openings ! i) final-state
  and comp-tables:
     $\bigwedge i. i < \text{rounds} \implies$ 
      partial-authenticated-table composition-root (scale * clength)
        (composition-openings ! i) final-state
  and comp-indices:
     $\bigwedge i. i < \text{rounds} \implies$ 
      map opening-index (composition-openings ! i) =
        [query-idxs ! i,
         fri-sibling-index (scale * clength) (query-idxs ! i)]
  and consistent:
     $\bigwedge i. i < \text{rounds} \implies$ 
      partial-query-round-consistent trace-openings composition-openings
        as' i (query-idxs ! i)
  proof (rule
    ntimes-verifier-query-rounds-aligned-authenticated-openings-consistent
      [OF fl-eq query-out])
  fix query-idxs trace-openings composition-openings
  assume len-query-idxs: length query-idxs = rounds
  and len-trace: length trace-openings = rounds
  and len-comp: length composition-openings = rounds
  and trace-indices:
     $\bigwedge i. i < \text{rounds} \implies$ 
      map opening-index (trace-openings ! i) =
        powers-scaled (query-idxs ! i)
  and trace-tables:
     $\bigwedge i. i < \text{rounds} \implies$ 
      partial-authenticated-table fr' (scale * clength)
        (trace-openings ! i) final-state
  and comp-tables:
     $\bigwedge i. i < \text{rounds} \implies$ 
      partial-authenticated-table composition-root (scale * clength)
        (composition-openings ! i) final-state
  and comp-indices:
     $\bigwedge i. i < \text{rounds} \implies$ 
      map opening-index (composition-openings ! i) =
        [query-idxs ! i,
         fri-sibling-index (scale * clength) (query-idxs ! i)]
  and consistent:
     $\bigwedge i. i < \text{rounds} \implies$ 
      partial-query-round-consistent trace-openings composition-openings
        as' i (query-idxs ! i)
  show ?thesis
  by (rule that[OF len-query-idxs len-trace len-comp trace-indices
    trace-tables comp-tables comp-indices consistent])
qed
have consistent-as:
   $\bigwedge i. i < \text{rounds} \implies$ 

```

```

    partial-query-round-consistent trace-openings composition-openings as i
    (query-idxs ! i)
  using consistent eqs by simp
have trace-partial:
  accepted-with-partial-trace-openings s
  (Some (result, final-state)) fr query-idxs trace-openings
proof -
  have all-indices:
     $\forall i < \text{rounds}.$ 
    map opening-index (trace-openings ! i) =
      powers-scaled (query-idxs ! i)
  using trace-indices by blast
have all-tables:
   $\forall i < \text{rounds}.$ 
  partial-authenticated-table fr (scale * clength)
  (trace-openings ! i) final-state
  using trace-tables eqs by simp
show ?thesis
  unfolding accepted-with-partial-trace-openings-def accepted-def
  by (intro conjI exI[of - result] exI[of - final-state])
  (use len-query-idxs len-trace all-indices all-tables in simp-all)
qed
have comp-partial:
  accepted-with-partial-composition-openings s
  (Some (result, final-state)) (hd composition-fri-roots) query-idxs
  composition-openings
proof -
  have all-indices:
     $\forall i < \text{rounds}.$ 
    map opening-index (composition-openings ! i) =
      [query-idxs ! i,
       fri-sibling-index (scale * clength) (query-idxs ! i)]
  using comp-indices by blast
have all-tables:
   $\forall i < \text{rounds}.$ 
  partial-authenticated-table (hd composition-fri-roots)
  (scale * clength) (composition-openings ! i) final-state
  using comp-tables root-eq by simp
show ?thesis
  unfolding accepted-with-partial-composition-openings-def accepted-def
  by (intro conjI exI[of - result] exI[of - final-state])
  (use len-query-idxs len-comp all-indices all-tables in simp-all)
qed
have aligned:
  accepted-with-partial-initial-openings-aligned s
  (Some (result, final-state)) fr f-fri-roots f-final as dg
  composition-fri-roots final query-idxs trace-openings
  composition-openings
proof -

```

```

have accepted-out: accepted (Some (result, final-state))
  using trace-partial
  by (rule accepted-with-partial-trace-openings-imp-accepted)
show ?thesis
unfolding accepted-with-partial-initial-openings-aligned-def
by (intro conjI exI[of - rest])
    (use accepted-out comp-nonempty header0 trace-partial comp-partial in
      simp-all)
qed
show ?thesis
  by (rule that[OF aligned consistent-as])
qed

end

end

```

```

theory Soundness-Aligned-Partial-Query
  imports Soundness-Partial-Initial-Aligned
begin

```

Aligned partial-candidate query events.

This layer records the query-bad shape needed by the staged prefix-target proof: one shared verifier query-index list for trace and composition openings, together with the local composition equation checked in each query round. It is downstream of the staged query-opening extraction layer to avoid import cycles with the core deterministic reductions.

```

context soundness
begin

```

definition *accepted-with-partial-initial-openings-aligned-consistent*

```

:: ('f, 'a) protocol-channel-scheme  $\Rightarrow$ 
  (unit list  $\times$  ('f, 'a) protocol-channel-scheme) option  $\Rightarrow$ 
  'f  $\Rightarrow$  'f list  $\Rightarrow$  'f  $\Rightarrow$  'f list  $\Rightarrow$  'f  $\Rightarrow$ 
  'f list  $\Rightarrow$  'f  $\Rightarrow$  nat list  $\Rightarrow$ 
  'f authenticated-opening list list  $\Rightarrow$ 
  'f authenticated-opening list list  $\Rightarrow$  bool

```

where

```

accepted-with-partial-initial-openings-aligned-consistent s out fr
  f-fri-roots f-final as dg composition-fri-roots final query-idxs
  trace-openings composition-openings  $\longleftrightarrow$ 
accepted-with-partial-initial-openings-aligned s out fr f-fri-roots
  f-final as dg composition-fri-roots final query-idxs trace-openings
  composition-openings  $\wedge$ 
  ( $\forall i < \text{rounds}.$ 
    partial-query-round-consistent trace-openings composition-openings as i
      (query-idxs ! i))

```

definition *query-bad-with-aligned-partial-candidates*
 $:: ('f, 'a) \text{ protocol-channel-scheme} \Rightarrow$
 $(\text{unit list} \times ('f, 'a) \text{ protocol-channel-scheme}) \text{ option} \Rightarrow \text{bool}$

where

query-bad-with-aligned-partial-candidates $s \text{ out} \longleftrightarrow$
 $(\exists \text{ fr } f\text{-fri-roots } f\text{-final as dg composition-fri-roots final}$
 $\text{ query-idxs trace-openings composition-openings trace-table}$
 $\text{ composition-table.}$
 $\text{ accepted-with-partial-initial-openings-aligned-consistent } s \text{ out fr}$
 $\text{ f-fri-roots f-final as dg composition-fri-roots final query-idxs}$
 $\text{ trace-openings composition-openings} \wedge$
 $\text{ partial-trace-table-candidate trace-table trace-openings} \wedge$
 $\text{ partial-composition-table-candidate composition-table}$
 $\text{ composition-openings} \wedge$
 $\text{ trace-table-low-degree trace-table} \wedge$
 $\text{ composition-table-low-degree maxDegree composition-table} \wedge$
 $\neg \text{ all-queries-consistent trace-table composition-table as})$

lemma *verify-monad-accepted-with-partial-initial-openings-aligned-consistent:*

fixes $s :: ('f, 'a) \text{ protocol-channel-scheme}$

assumes *outcome:*

$\text{Some (result, final-state)} \in \text{set-dist (execute verify-monad } s)$

and *header0:*

$\text{verifier-header-transcript } s \text{ fr } f\text{-fri-roots } f\text{-final as dg}$
 $\text{ composition-fri-roots final rest}$

and *comp-nonempty:* $\text{composition-fri-roots} \neq []$

obtains *query-idxs trace-openings composition-openings* **where**

$\text{accepted-with-partial-initial-openings-aligned-consistent } s$
 $(\text{Some (result, final-state)}) \text{ fr } f\text{-fri-roots } f\text{-final as dg}$
 $\text{ composition-fri-roots final query-idxs trace-openings}$
 $\text{ composition-openings}$

proof –

from *verify-monad-accepted-with-partial-initial-openings-aligned*

$[OF \text{ outcome header0 comp-nonempty}]$

obtain *query-idxs trace-openings composition-openings* **where** *aligned:*

$\text{accepted-with-partial-initial-openings-aligned } s$
 $(\text{Some (result, final-state)}) \text{ fr } f\text{-fri-roots } f\text{-final as dg}$
 $\text{ composition-fri-roots final query-idxs trace-openings}$
 $\text{ composition-openings}$

and *consistent:*

$\bigwedge i. i < \text{rounds} \implies$
 $\text{partial-query-round-consistent trace-openings composition-openings as } i$
 $(\text{query-idxs ! } i)$

by *blast*

have *all-consistent:*

$\forall i < \text{rounds.}$
 $\text{partial-query-round-consistent trace-openings composition-openings as } i$
 $(\text{query-idxs ! } i)$

```

using consistent by blast
have partial:
  accepted-with-partial-initial-openings-aligned-consistent s
    (Some (result, final-state)) fr f-fri-roots f-final as dg
    composition-fri-roots final query-idxs trace-openings
    composition-openings
  unfolding accepted-with-partial-initial-openings-aligned-consistent-def
  using aligned all-consistent by simp
show ?thesis
  by (rule that[OF partial])
qed

definition soundness-bad-event-aligned-partial-candidate
  :: (f, 'a) protocol-channel-scheme  $\Rightarrow$ 
    (unit list  $\times$  (f, 'a) protocol-channel-scheme) option  $\Rightarrow$  bool
where
  soundness-bad-event-aligned-partial-candidate s out  $\longleftrightarrow$ 
    composition-bad-with-partial-candidates s out  $\vee$ 
    trace-fri-bad-with-partial-candidates s out  $\vee$ 
    composition-fri-bad-with-partial-candidates s out  $\vee$ 
    query-bad-with-aligned-partial-candidates s out

definition soundness-bad-event-aligned-partial-candidate-with-partial-merkle
  :: (f, 'a) protocol-channel-scheme  $\Rightarrow$ 
    (unit list  $\times$  (f, 'a) protocol-channel-scheme) option  $\Rightarrow$  bool
where
  soundness-bad-event-aligned-partial-candidate-with-partial-merkle s out  $\longleftrightarrow$ 
    partial-merkle-inconsistency-bad s out  $\vee$ 
    soundness-bad-event-aligned-partial-candidate s out

lemma accepted-with-partial-initial-openings-aligned-consistent-imp-aligned:
assumes
  accepted-with-partial-initial-openings-aligned-consistent s out fr
  f-fri-roots f-final as dg composition-fri-roots final query-idxs
  trace-openings composition-openings
shows
  accepted-with-partial-initial-openings-aligned s out fr f-fri-roots
  f-final as dg composition-fri-roots final query-idxs trace-openings
  composition-openings
using assms
unfolding accepted-with-partial-initial-openings-aligned-consistent-def
by simp

lemma accepted-with-partial-initial-openings-aligned-consistent-imp-unaligned:
assumes
  accepted-with-partial-initial-openings-aligned-consistent s out fr
  f-fri-roots f-final as dg composition-fri-roots final query-idxs
  trace-openings composition-openings
shows

```


accepted-with-partial-initial-openings s out fr f-fri-roots f-final as dg
composition-fri-roots final query-idxs trace-openings query-idxs
composition-openings
by (rule *accepted-with-partial-initial-openings-aligned-imp-unaligned*)
(rule *accepted-with-partial-initial-openings-aligned-consistent-imp-aligned*
[*OF assms*])

lemma *accepted-with-partial-initial-openings-aligned-consistent-success-index:*

assumes *partial:*

accepted-with-partial-initial-openings-aligned-consistent s out fr
f-fri-roots f-final as dg composition-fri-roots final query-idxs
trace-openings composition-openings

and *i-bound: i < rounds*

shows

query-idxs ! i ∈
partial-query-success-indices-at trace-openings composition-openings
as i

using *partial i-bound*

unfolding *accepted-with-partial-initial-openings-aligned-consistent-def*

partial-query-success-indices-at-def partial-query-round-consistent-def

by *simp*

lemma *query-bad-with-aligned-partial-candidates-imp-query-bad-with-partial-candidates:*

assumes *query-bad-with-aligned-partial-candidates s out*

shows *query-bad-with-partial-candidates s out*

proof —

from *assms obtain* *fr f-fri-roots f-final as dg composition-fri-roots*
final query-idxs trace-openings composition-openings trace-table
composition-table **where** *partial:*

accepted-with-partial-initial-openings-aligned-consistent s out fr
f-fri-roots f-final as dg composition-fri-roots final query-idxs
trace-openings composition-openings

and *trace-candidate:*

partial-trace-table-candidate trace-table trace-openings

and *comp-candidate:*

partial-composition-table-candidate composition-table
composition-openings

and *trace-low: trace-table-low-degree trace-table*

and *comp-low:*

composition-table-low-degree maxDegree composition-table

and *not-all:*

$\neg$ *all-queries-consistent trace-table composition-table as*

unfolding *query-bad-with-aligned-partial-candidates-def* **by** *blast*

have *unaligned:*

accepted-with-partial-initial-openings s out fr f-fri-roots f-final as dg
composition-fri-roots final query-idxs trace-openings query-idxs
composition-openings

by (rule

accepted-with-partial-initial-openings-aligned-consistent-imp-unaligned

```

    [OF partial])
show ?thesis
  unfolding query-bad-with-partial-candidates-def
  using unaligned trace-candidate comp-candidate trace-low comp-low not-all
  by blast
qed

lemma soundness-bad-event-aligned-partial-candidate-imp-partial-candidate:
  assumes soundness-bad-event-aligned-partial-candidate s out
  shows soundness-bad-event-partial-candidate s out
  using assms query-bad-with-aligned-partial-candidates-imp-query-bad-with-partial-candidates
  unfolding soundness-bad-event-aligned-partial-candidate-def
    soundness-bad-event-partial-candidate-def
  by blast

lemma soundness-bad-event-aligned-partial-candidate-with-partial-merkle-imp-partial-candidate-with-partial-merkle:
  assumes
    soundness-bad-event-aligned-partial-candidate-with-partial-merkle s out
  shows soundness-bad-event-partial-candidate-with-partial-merkle s out
  using assms soundness-bad-event-aligned-partial-candidate-imp-partial-candidate
  unfolding soundness-bad-event-aligned-partial-candidate-with-partial-merkle-def
    soundness-bad-event-partial-candidate-with-partial-merkle-def
  by blast

lemma accepted-aligned-partial-candidate-partition:
  assumes false-statement:  $\neg$  exists-valid-trace
  and partial:
    accepted-with-partial-initial-openings-aligned-consistent s out fr
    f-fri-roots f-final as dg composition-fri-roots final query-idxs
    trace-openings composition-openings
  and trace-candidate:
    partial-trace-table-candidate trace-table trace-openings
  and composition-candidate:
    partial-composition-table-candidate composition-table
    composition-openings
  shows soundness-bad-event-aligned-partial-candidate s out
proof (cases trace-table-low-degree trace-table)
case False
have trace-fri-bad-with-partial-candidates s out
  unfolding trace-fri-bad-with-partial-candidates-def
  using accepted-with-partial-initial-openings-aligned-consistent-imp-unaligned
    [OF partial]
    trace-candidate False by blast
then show ?thesis
  unfolding soundness-bad-event-aligned-partial-candidate-def by blast
next
case True
note trace-low = True
show ?thesis

```

```

proof (cases composition-table-low-degree maxDegree composition-table)
  case False
  have composition-fri-bad-with-partial-candidates s out
    unfolding composition-fri-bad-with-partial-candidates-def
    using accepted-with-partial-initial-openings-aligned-consistent-imp-unaligned
      [OF partial]
      trace-candidate composition-candidate trace-low False
    by blast
  then show ?thesis
    unfolding soundness-bad-event-aligned-partial-candidate-def by blast
next
  case True
  note composition-low = True
  show ?thesis
  proof (cases all-queries-consistent trace-table composition-table as)
    case False
    have query-bad-with-aligned-partial-candidates s out
      unfolding query-bad-with-aligned-partial-candidates-def
      using partial trace-candidate composition-candidate trace-low
        composition-low False
      by blast
    then show ?thesis
      unfolding soundness-bad-event-aligned-partial-candidate-def by blast
  next
  case True
  note all-queries = True
  from trace-low obtain f where
    deg-f: degree f < clength
    and trace-table-eq: trace-table = map (poly f) eval-domain
    unfolding trace-table-low-degree-def by blast
  have violated: violated-constraints f ≠ {}
    by (rule false-statement-violated-constraints
      [OF false-statement deg-f])
  have composition-bad-with-partial-candidates s out
    unfolding composition-bad-with-partial-candidates-def
    using accepted-with-partial-initial-openings-aligned-consistent-imp-unaligned
      [OF partial]
      trace-candidate composition-candidate deg-f trace-table-eq
      violated composition-low all-queries
    by blast
  then show ?thesis
    unfolding soundness-bad-event-aligned-partial-candidate-def by blast
  qed
qed
qed

```

```

lemma soundness-bad-event-aligned-partial-candidate-union-bound:
  fixes s :: ('f, 'a) protocol-channel-scheme
  assumes comp-bound:

```

```

    wp-event verify-monad
      (composition-bad-with-partial-candidates s) s ≤
      composition-error
  and trace-fri-bound:
    wp-event verify-monad
      (trace-fri-bad-with-partial-candidates s) s ≤
      trace-fri-error'
  and comp-fri-bound:
    wp-event verify-monad
      (composition-fri-bad-with-partial-candidates s) s ≤
      composition-fri-error'
  and query-bound:
    wp-event verify-monad
      (query-bad-with-aligned-partial-candidates s) s ≤
      query-error
  shows
    wp-event verify-monad
      (soundness-bad-event-aligned-partial-candidate s) s ≤
      composition-error + trace-fri-error' + composition-fri-error' +
      query-error
  proof -
    have wp-event verify-monad
      (soundness-bad-event-aligned-partial-candidate s) s ≤
      wp-event verify-monad
        (composition-bad-with-partial-candidates s) s +
      wp-event verify-monad
        (trace-fri-bad-with-partial-candidates s) s +
      wp-event verify-monad
        (composition-fri-bad-with-partial-candidates s) s +
      wp-event verify-monad
        (query-bad-with-aligned-partial-candidates s) s
    unfolding soundness-bad-event-aligned-partial-candidate-def
    by (rule wp-event-union-bound4)
    also have ... ≤
      composition-error + trace-fri-error' + composition-fri-error' +
      query-error
    by (intro add-mono comp-bound trace-fri-bound comp-fri-bound
      query-bound)
    finally show ?thesis .
  qed

  lemma soundness-bad-event-aligned-partial-candidate-with-partial-merkle-union-bound:
    fixes s :: ('f, 'a) protocol-channel-scheme
    assumes merkle-bound:
      wp-event verify-monad (partial-merkle-inconsistency-bad s) s ≤
      merkle-error
    and comp-bound:
      wp-event verify-monad
        (composition-bad-with-partial-candidates s) s ≤

```

```

    composition-error
  and trace-fri-bound:
    wp-event verify-monad
      (trace-fri-bad-with-partial-candidates s) s ≤
      trace-fri-error'
  and comp-fri-bound:
    wp-event verify-monad
      (composition-fri-bad-with-partial-candidates s) s ≤
      composition-fri-error'
  and query-bound:
    wp-event verify-monad
      (query-bad-with-aligned-partial-candidates s) s ≤
      query-error
  shows
    wp-event verify-monad
      (soundness-bad-event-aligned-partial-candidate-with-partial-merkle s) s
      ≤
      merkle-error + composition-error + trace-fri-error' +
      composition-fri-error' + query-error
  proof -
    have partial-bound:
      wp-event verify-monad
        (soundness-bad-event-aligned-partial-candidate s) s ≤
        composition-error + trace-fri-error' + composition-fri-error' +
        query-error
    by (rule soundness-bad-event-aligned-partial-candidate-union-bound
        [OF comp-bound trace-fri-bound comp-fri-bound query-bound])
    have wp-event verify-monad
      (soundness-bad-event-aligned-partial-candidate-with-partial-merkle s) s
      ≤
      wp-event verify-monad (partial-merkle-inconsistency-bad s) s +
      wp-event verify-monad
        (soundness-bad-event-aligned-partial-candidate s) s
    unfolding
      soundness-bad-event-aligned-partial-candidate-with-partial-merkle-def
    by (rule wp-event-union-bound)
    also have ... ≤
      merkle-error +
      (composition-error + trace-fri-error' + composition-fri-error' +
      query-error)
    by (intro add-mono merkle-bound partial-bound)
    finally show ?thesis
    by (simp add: algebra-simps)
  qed

lemma checked-staged-security-with-data-state-aligned-partial-candidate-with-partial-merkle-union-bound:
  fixes merkle-error composition-error trace-fri-error'
    composition-fri-error' query-error :: prob
  assumes merkle-bound:

```

$wp\text{-}event$ (*checked-staged-security-experiment-with-data-state* A)
 (*staged-security-with-data-state-verifier-event*
partial-merkle-inconsistency-bad)
 $adversary\text{-}initial\text{-}state \leq merkle\text{-}error$

and *comp-bound*:

$wp\text{-}event$ (*checked-staged-security-experiment-with-data-state* A)
 (*staged-security-with-data-state-verifier-event*
composition-bad-with-partial-candidates)
 $adversary\text{-}initial\text{-}state \leq composition\text{-}error$

and *trace-fri-bound*:

$wp\text{-}event$ (*checked-staged-security-experiment-with-data-state* A)
 (*staged-security-with-data-state-verifier-event*
trace-fri-bad-with-partial-candidates)
 $adversary\text{-}initial\text{-}state \leq trace\text{-}fri\text{-}error'$

and *comp-fri-bound*:

$wp\text{-}event$ (*checked-staged-security-experiment-with-data-state* A)
 (*staged-security-with-data-state-verifier-event*
composition-fri-bad-with-partial-candidates)
 $adversary\text{-}initial\text{-}state \leq composition\text{-}fri\text{-}error'$

and *query-bound*:

$wp\text{-}event$ (*checked-staged-security-experiment-with-data-state* A)
 (*staged-security-with-data-state-verifier-event*
query-bad-with-aligned-partial-candidates)
 $adversary\text{-}initial\text{-}state \leq query\text{-}error$

shows

$wp\text{-}event$ (*checked-staged-security-experiment-with-data-state* A)
 (*staged-security-with-data-state-verifier-event*
soundness-bad-event-aligned-partial-candidate-with-partial-merkle)
 $adversary\text{-}initial\text{-}state \leq$
 $merkle\text{-}error + composition\text{-}error + trace\text{-}fri\text{-}error' +$
 $composition\text{-}fri\text{-}error' + query\text{-}error$

proof –

let $?M = checked\text{-}staged\text{-}security\text{-}experiment\text{-}with\text{-}data\text{-}state$ A
let $?E = staged\text{-}security\text{-}with\text{-}data\text{-}state\text{-}verifier\text{-}event$
let $?merkle = ?E$ *partial-merkle-inconsistency-bad*
let $?comp = ?E$ *composition-bad-with-partial-candidates*
let $?trace = ?E$ *trace-fri-bad-with-partial-candidates*
let $?comp\text{-}fri = ?E$ *composition-fri-bad-with-partial-candidates*
let $?query = ?E$ *query-bad-with-aligned-partial-candidates*
let $?partial = ?E$ *soundness-bad-event-aligned-partial-candidate*
let $?with\text{-}merkle =$
 $?E$ *soundness-bad-event-aligned-partial-candidate-with-partial-merkle*

have *partial-bound*:

$wp\text{-}event$ $?M$ $?partial$ $adversary\text{-}initial\text{-}state \leq$
 $composition\text{-}error + trace\text{-}fri\text{-}error' + composition\text{-}fri\text{-}error' +$
 $query\text{-}error$

proof –

have $wp\text{-}event$ $?M$ $?partial$ $adversary\text{-}initial\text{-}state \leq$
 $wp\text{-}event$ $?M$

```

    (λout. ?comp out ∨ ?trace out ∨ ?comp-fri out ∨
      ?query out)
    adversary-initial-state
  by (rule wp-event-mono)
  (auto simp: staged-security-with-data-state-verifier-event-def
    soundness-bad-event-aligned-partial-candidate-def
    split: option.splits prod.splits)
also have ... ≤
  wp-event ?M ?comp adversary-initial-state +
  wp-event ?M ?trace adversary-initial-state +
  wp-event ?M ?comp-fri adversary-initial-state +
  wp-event ?M ?query adversary-initial-state
  by (rule wp-event-union-bound4)
also have ... ≤
  composition-error + trace-fri-error' + composition-fri-error' +
  query-error
  by (intro add-mono comp-bound trace-fri-bound comp-fri-bound
    query-bound)
finally show ?thesis .
qed
have wp-event ?M ?with-merkle adversary-initial-state ≤
  wp-event ?M (λout. ?merkle out ∨ ?partial out)
  adversary-initial-state
  by (rule wp-event-mono)
  (auto simp: staged-security-with-data-state-verifier-event-def
    soundness-bad-event-aligned-partial-candidate-with-partial-merkle-def
    split: option.splits prod.splits)
also have ... ≤
  wp-event ?M ?merkle adversary-initial-state +
  wp-event ?M ?partial adversary-initial-state
  by (rule wp-event-union-bound)
also have ... ≤
  merkle-error +
  (composition-error + trace-fri-error' + composition-fri-error' +
    query-error)
  by (intro add-mono merkle-bound partial-bound)
finally show ?thesis
  by (simp add: algebra-simps)
qed

lemma query-bad-with-aligned-partial-candidates-success-indexE:
  assumes bad: query-bad-with-aligned-partial-candidates s out
    and i-bound: i < rounds
  obtains fr f-fri-roots f-final as dg composition-fri-roots final
    query-idxs trace-openings composition-openings trace-table
    composition-table where
    accepted-with-partial-initial-openings-aligned-consistent s out fr
    f-fri-roots f-final as dg composition-fri-roots final query-idxs
    trace-openings composition-openings

```

partial-trace-table-candidate trace-table trace-openings
partial-composition-table-candidate composition-table
composition-openings
trace-table-low-degree trace-table
composition-table-low-degree maxDegree composition-table
 $\neg$ *all-queries-consistent trace-table composition-table as*
query-idxs ! i ∈
partial-query-success-indices-at trace-openings composition-openings
as i
proof –
from *bad* **obtain** *fr f-fri-roots f-final as dg composition-fri-roots*
final query-idxs trace-openings composition-openings trace-table
composition-table **where** *partial:*
accepted-with-partial-initial-openings-aligned-consistent s out fr
f-fri-roots f-final as dg composition-fri-roots final query-idxs
trace-openings composition-openings
and *trace-candidate:*
partial-trace-table-candidate trace-table trace-openings
and *comp-candidate:*
partial-composition-table-candidate composition-table
composition-openings
and *trace-low: trace-table-low-degree trace-table*
and *comp-low:*
composition-table-low-degree maxDegree composition-table
and *not-all:*
 $\neg$ *all-queries-consistent trace-table composition-table as*
unfolding *query-bad-with-aligned-partial-candidates-def* **by** *blast*
have *success:*
query-idxs ! i ∈
partial-query-success-indices-at trace-openings composition-openings
as i
by (*rule*
accepted-with-partial-initial-openings-aligned-consistent-success-index
[OF partial i-bound])
show *?thesis*
by (*rule that[OF partial trace-candidate comp-candidate trace-low comp-low*
not-all success])
qed

lemma *query-bad-with-aligned-partial-candidates-target-fractionE:*
assumes *bad: query-bad-with-aligned-partial-candidates s out*
and *i-bound: i < rounds*
obtains *fr f-fri-roots f-final as dg composition-fri-roots final*
query-idxs trace-openings composition-openings trace-table
composition-table **where**
accepted-with-partial-initial-openings-aligned-consistent s out fr
f-fri-roots f-final as dg composition-fri-roots final query-idxs
trace-openings composition-openings
query-idxs ! i ∈

partial-query-success-indices-at trace-openings composition-openings
as i
nnreal
(card
(partial-query-success-indices-at trace-openings composition-openings
as i)) /
nnreal (card query-sample-space) ≤ query-error-bound
proof –
from *query-bad-with-aligned-partial-candidates-success-indexE*
[OF bad i-bound]
obtain *fr f-fri-roots f-final as dg composition-fri-roots final*
query-idxs trace-openings composition-openings trace-table
composition-table where partial:
accepted-with-partial-initial-openings-aligned-consistent s out fr
f-fri-roots f-final as dg composition-fri-roots final query-idxs
trace-openings composition-openings
and *trace-candidate:*
partial-trace-table-candidate trace-table trace-openings
and *comp-candidate:*
partial-composition-table-candidate composition-table
composition-openings
and *trace-low: trace-table-low-degree trace-table*
and *comp-low:*
composition-table-low-degree maxDegree composition-table
and *not-all:*
¬ all-queries-consistent trace-table composition-table as
and *success:*
query-idxs ! i ∈
partial-query-success-indices-at trace-openings composition-openings
as i
by *blast*
have *fraction:*
nnreal
(card
(partial-query-success-indices-at trace-openings composition-openings
as i)) /
nnreal (card query-sample-space) ≤ query-error-bound
by *(rule partial-query-success-indices-at-fraction-bound-if-candidate-low-degree*
[OF trace-candidate comp-candidate trace-low comp-low not-all])
show *?thesis*
by *(rule that[OF partial success fraction])*
qed

lemma *query-bad-with-aligned-partial-candidates-miss-disagreements:*
assumes *bad: query-bad-with-aligned-partial-candidates s out*
shows *∃ trace-table composition-table as query-idxs.*
query-samples-miss-disagreements trace-table composition-table as
query-idxs
proof –

```

from bad obtain fr f-fri-roots f-final as dg composition-fri-roots
  final query-idxs trace-openings composition-openings trace-table
  composition-table where partial:
    accepted-with-partial-initial-openings-aligned-consistent s out fr
      f-fri-roots f-final as dg composition-fri-roots final query-idxs
      trace-openings composition-openings
and trace-candidate:
  partial-trace-table-candidate trace-table trace-openings
and comp-candidate:
  partial-composition-table-candidate composition-table
  composition-openings
and trace-low: trace-table-low-degree trace-table
and comp-low:
  composition-table-low-degree maxDegree composition-table
and not-all:
   $\neg$  all-queries-consistent trace-table composition-table as
unfolding query-bad-with-aligned-partial-candidates-def by blast
have aligned:
  accepted-with-partial-initial-openings-aligned s out fr f-fri-roots
  f-final as dg composition-fri-roots final query-idxs trace-openings
  composition-openings
by (rule accepted-with-partial-initial-openings-aligned-consistent-imp-aligned
  [OF partial])
have trace-partial:
  accepted-with-partial-trace-openings s out fr query-idxs
  trace-openings
using accepted-with-partial-initial-openings-aligned-shapes(3)
  [OF aligned] .
have len-query: length query-idxs = rounds
  using accepted-with-partial-trace-openings-shapes(1) [OF trace-partial] .
have nonempty-disagreements:
  query-disagreement-indices trace-table composition-table as  $\neq \{\}$ 
using not-all
unfolding query-disagreement-indices-def bad-query-indices-def
  all-queries-consistent-def
by auto
have subset-agreement:
  set query-idxs  $\subseteq$ 
  query-agreement-indices trace-table composition-table as
proof
  fix idx
  assume idx-in: idx  $\in$  set query-idxs
  then obtain i where i-len: i < length query-idxs
    and idx-eq: query-idxs ! i = idx
    by (auto simp: in-set-conv-nth)
  have i-bound: i < rounds
    using i-len len-query by simp
  have success:
    query-idxs ! i  $\in$ 

```

```

    partial-query-success-indices-at trace-openings composition-openings
    as i
  by (rule
    accepted-with-partial-initial-openings-aligned-consistent-success-index
    [OF partial i-bound])
  have success-space:
    query-idxs ! i ∈
    query-sampling-success-space trace-table composition-table as
  using success
    partial-query-success-indices-at-subset-query-sampling-success-space
    [OF trace-candidate comp-candidate trace-low comp-low not-all,
    of i]
  by blast
  then show idx ∈
    query-agreement-indices trace-table composition-table as
  using idx-eq trace-low comp-low not-all
  unfolding query-sampling-success-space-def by simp
qed
have
  query-samples-miss-disagreements trace-table composition-table as
  query-idxs
  unfolding query-samples-miss-disagreements-def
  using nonempty-disagreements subset-agreement by simp
then show ?thesis by blast
qed

end

end

```

```

theory Soundness-Aligned-Partial-Query-Transcript
imports
  Soundness-Aligned-Partial-Query
  Soundness-Staged-Partial-Query
begin

```

Transcript-tied aligned partial-query events.

The aligned partial-opening predicate records that trace and composition openings use the same sampled query indices. For probabilistic query bounds we additionally need those indices to be the verifier-derived Fiat-Shamir query indices. This layer adds that transcript replay fact without changing the protocol model.

```

context soundness
begin

```

```

definition accepted-with-partial-initial-openings-aligned-transcript-consistent
:: ('f, 'a) protocol-channel-scheme ⇒
  (unit list × ('f, 'a) protocol-channel-scheme) option ⇒
  'f ⇒ 'f list ⇒ 'f ⇒ 'f list ⇒ 'f ⇒

```

$'f \text{ list} \Rightarrow 'f \Rightarrow \text{nat list} \Rightarrow$
 $'f \text{ authenticated-opening list list} \Rightarrow$
 $'f \text{ authenticated-opening list list} \Rightarrow \text{bool}$

where

$\text{accepted-with-partial-initial-openings-aligned-transcript-consistent } s \text{ out}$
 $\text{fr } f\text{-fri-roots } f\text{-final as dg composition-fri-roots final query-idxs}$
 $\text{trace-openings composition-openings} \longleftrightarrow$
 $\text{accepted-with-partial-initial-openings-aligned-consistent } s \text{ out fr}$
 $f\text{-fri-roots } f\text{-final as dg composition-fri-roots final query-idxs}$
 $\text{trace-openings composition-openings} \wedge$
 $\text{accepted-transcript-shape } s \text{ out as query-idxs}$

definition *query-bad-with-aligned-transcript-partial-candidates*

$:: ('f, 'a) \text{ protocol-channel-scheme} \Rightarrow$
 $(\text{unit list} \times ('f, 'a) \text{ protocol-channel-scheme}) \text{ option} \Rightarrow \text{bool}$

where

$\text{query-bad-with-aligned-transcript-partial-candidates } s \text{ out} \longleftrightarrow$
 $(\exists \text{ fr } f\text{-fri-roots } f\text{-final as dg composition-fri-roots final}$
 $\text{query-idxs trace-openings composition-openings trace-table}$
 $\text{composition-table.}$
 $\text{accepted-with-partial-initial-openings-aligned-transcript-consistent } s$
 $\text{out fr } f\text{-fri-roots } f\text{-final as dg composition-fri-roots final}$
 $\text{query-idxs trace-openings composition-openings} \wedge$
 $\text{partial-trace-table-candidate trace-table trace-openings} \wedge$
 $\text{partial-composition-table-candidate composition-table}$
 $\text{composition-openings} \wedge$
 $\text{trace-table-low-degree trace-table} \wedge$
 $\text{composition-table-low-degree maxDegree composition-table} \wedge$
 $\neg \text{all-queries-consistent trace-table composition-table as})$

definition *soundness-bad-event-aligned-transcript-partial-candidate*

$:: ('f, 'a) \text{ protocol-channel-scheme} \Rightarrow$
 $(\text{unit list} \times ('f, 'a) \text{ protocol-channel-scheme}) \text{ option} \Rightarrow \text{bool}$

where

$\text{soundness-bad-event-aligned-transcript-partial-candidate } s \text{ out} \longleftrightarrow$
 $\text{composition-bad-with-partial-candidates } s \text{ out} \vee$
 $\text{trace-fri-bad-with-partial-candidates } s \text{ out} \vee$
 $\text{composition-fri-bad-with-partial-candidates } s \text{ out} \vee$
 $\text{query-bad-with-aligned-transcript-partial-candidates } s \text{ out}$

definition *soundness-bad-event-aligned-transcript-partial-candidate-with-partial-merkle*

$:: ('f, 'a) \text{ protocol-channel-scheme} \Rightarrow$
 $(\text{unit list} \times ('f, 'a) \text{ protocol-channel-scheme}) \text{ option} \Rightarrow \text{bool}$

where

$\text{soundness-bad-event-aligned-transcript-partial-candidate-with-partial-merkle}$
 $s \text{ out} \longleftrightarrow$
 $\text{partial-merkle-inconsistency-bad } s \text{ out} \vee$
 $\text{soundness-bad-event-aligned-transcript-partial-candidate } s \text{ out}$

lemma *accepted-with-partial-initial-openings-aligned-transcript-consistent-imp-aligned:*
assumes

accepted-with-partial-initial-openings-aligned-transcript-consistent s out
fr f-fri-roots f-final as dg composition-fri-roots final query-idxs
trace-openings composition-openings

shows

accepted-with-partial-initial-openings-aligned-consistent s out fr
f-fri-roots f-final as dg composition-fri-roots final query-idxs
trace-openings composition-openings

using *assms*

unfolding

accepted-with-partial-initial-openings-aligned-transcript-consistent-def

by *simp*

lemma *accepted-with-partial-initial-openings-aligned-transcript-consistent-imp-shape:*

assumes

accepted-with-partial-initial-openings-aligned-transcript-consistent s out
fr f-fri-roots f-final as dg composition-fri-roots final query-idxs
trace-openings composition-openings

shows *accepted-transcript-shape s out as query-idxs*

using *assms*

unfolding

accepted-with-partial-initial-openings-aligned-transcript-consistent-def

by *simp*

lemma *query-bad-with-aligned-transcript-partial-candidates-imp-aligned:*

assumes *query-bad-with-aligned-transcript-partial-candidates s out*

shows *query-bad-with-aligned-partial-candidates s out*

proof —

from *assms* **obtain** *fr f-fri-roots f-final as dg composition-fri-roots*
final query-idxs trace-openings composition-openings trace-table
composition-table **where** *partial:*

accepted-with-partial-initial-openings-aligned-transcript-consistent s
out fr f-fri-roots f-final as dg composition-fri-roots final
query-idxs trace-openings composition-openings

and *trace-candidate:*

partial-trace-table-candidate trace-table trace-openings

and *comp-candidate:*

partial-composition-table-candidate composition-table
composition-openings

and *trace-low: trace-table-low-degree trace-table*

and *comp-low:*

composition-table-low-degree maxDegree composition-table

and *not-all:*

$\neg$ *all-queries-consistent trace-table composition-table as*

unfolding *query-bad-with-aligned-transcript-partial-candidates-def*

by *blast*

have *aligned:*

accepted-with-partial-initial-openings-aligned-consistent s out fr

f-fri-roots f-final as dg composition-fri-roots final query-idxs
trace-openings composition-openings
by (rule
accepted-with-partial-initial-openings-aligned-transcript-consistent-imp-aligned
[OF partial])
show ?thesis
unfolding *query-bad-with-aligned-partial-candidates-def*
using *aligned trace-candidate comp-candidate trace-low comp-low not-all*
by *blast*
qed

lemma *query-bad-with-aligned-transcript-partial-candidates-miss-disagreements:*
assumes *query-bad-with-aligned-transcript-partial-candidates s out*
shows $\exists$ *trace-table composition-table as query-idxs.*
query-samples-miss-disagreements trace-table composition-table as
query-idxs
by (rule *query-bad-with-aligned-partial-candidates-miss-disagreements*)
(rule *query-bad-with-aligned-transcript-partial-candidates-imp-aligned*
[OF assms])

lemma *soundness-bad-event-aligned-transcript-partial-candidate-imp-aligned:*
assumes *soundness-bad-event-aligned-transcript-partial-candidate s out*
shows *soundness-bad-event-aligned-partial-candidate s out*
using *assms query-bad-with-aligned-transcript-partial-candidates-imp-aligned*
unfolding *soundness-bad-event-aligned-transcript-partial-candidate-def*
soundness-bad-event-aligned-partial-candidate-def
by *blast*

lemma *soundness-bad-event-aligned-transcript-partial-candidate-with-partial-merkle-imp-aligned:*
assumes
soundness-bad-event-aligned-transcript-partial-candidate-with-partial-merkle
s out
shows
soundness-bad-event-aligned-partial-candidate-with-partial-merkle s out
using *assms soundness-bad-event-aligned-transcript-partial-candidate-imp-aligned*
unfolding
soundness-bad-event-aligned-transcript-partial-candidate-with-partial-merkle-def
soundness-bad-event-aligned-partial-candidate-with-partial-merkle-def
by *blast*

lemma *accepted-aligned-transcript-partial-candidate-partition:*
assumes *false-statement: $\neg$ exists-valid-trace*
and *partial:*
accepted-with-partial-initial-openings-aligned-transcript-consistent s
out fr f-fri-roots f-final as dg composition-fri-roots final
query-idxs trace-openings composition-openings
and *trace-candidate:*
partial-trace-table-candidate trace-table trace-openings
and *composition-candidate:*

```

    partial-composition-table-candidate composition-table
    composition-openings
shows soundness-bad-event-aligned-transcript-partial-candidate s out
proof (cases trace-table-low-degree trace-table)
case False
have aligned:
    accepted-with-partial-initial-openings-aligned-consistent s out fr
    f-fri-roots f-final as dg composition-fri-roots final query-idxs
    trace-openings composition-openings
by (rule
    accepted-with-partial-initial-openings-aligned-transcript-consistent-imp-aligned
    [OF partial])
have trace-fri-bad-with-partial-candidates s out
unfolding trace-fri-bad-with-partial-candidates-def
using accepted-with-partial-initial-openings-aligned-consistent-imp-unaligned
    [OF aligned]
    trace-candidate False by blast
then show ?thesis
unfolding soundness-bad-event-aligned-transcript-partial-candidate-def
by blast
next
case True
note trace-low = True
show ?thesis
proof (cases composition-table-low-degree maxDegree composition-table)
case False
have aligned:
    accepted-with-partial-initial-openings-aligned-consistent s out fr
    f-fri-roots f-final as dg composition-fri-roots final query-idxs
    trace-openings composition-openings
by (rule
    accepted-with-partial-initial-openings-aligned-transcript-consistent-imp-aligned
    [OF partial])
have composition-fri-bad-with-partial-candidates s out
unfolding composition-fri-bad-with-partial-candidates-def
using accepted-with-partial-initial-openings-aligned-consistent-imp-unaligned
    [OF aligned]
    trace-candidate composition-candidate trace-low False
by blast
then show ?thesis
unfolding soundness-bad-event-aligned-transcript-partial-candidate-def
by blast
next
case True
note composition-low = True
show ?thesis
proof (cases all-queries-consistent trace-table composition-table as)
case False
have query-bad-with-aligned-transcript-partial-candidates s out

```

```

unfolding query-bad-with-aligned-transcript-partial-candidates-def
using partial trace-candidate composition-candidate trace-low
      composition-low False
by blast
then show ?thesis
  unfolding soundness-bad-event-aligned-transcript-partial-candidate-def
  by blast
next
case True
note all-queries = True
from trace-low obtain f where
  deg-f: degree f < clength
  and trace-table-eq: trace-table = map (poly f) eval-domain
  unfolding trace-table-low-degree-def by blast
have violated: violated-constraints f ≠ {}
  by (rule false-statement-violated-constraints
    [OF false-statement deg-f])
have aligned:
  accepted-with-partial-initial-openings-aligned-consistent s out fr
  f-fri-roots f-final as dg composition-fri-roots final query-idxs
  trace-openings composition-openings
  by (rule
    accepted-with-partial-initial-openings-aligned-transcript-consistent-imp-aligned
    [OF partial])
have composition-bad-with-partial-candidates s out
  unfolding composition-bad-with-partial-candidates-def
  using accepted-with-partial-initial-openings-aligned-consistent-imp-unaligned
    [OF aligned]
  trace-candidate composition-candidate deg-f trace-table-eq
  violated composition-low all-queries
  by blast
then show ?thesis
  unfolding soundness-bad-event-aligned-transcript-partial-candidate-def
  by blast
qed
qed
qed

```

```

lemma soundness-bad-event-aligned-transcript-partial-candidate-union-bound:
fixes s :: ('f, 'a) protocol-channel-scheme
assumes comp-bound:
  wp-event verify-monad
  (composition-bad-with-partial-candidates s) s ≤
  composition-error
and trace-fri-bound:
  wp-event verify-monad
  (trace-fri-bad-with-partial-candidates s) s ≤
  trace-fri-error'
and comp-fri-bound:

```



```

wp-event verify-monad
  (composition-fri-bad-with-partial-candidates s) s ≤
  composition-fri-error'
and query-bound:
wp-event verify-monad
  (query-bad-with-aligned-transcript-partial-candidates s) s ≤
  query-error
shows
wp-event verify-monad
  (soundness-bad-event-aligned-transcript-partial-candidate s) s ≤
  composition-error + trace-fri-error' + composition-fri-error' +
  query-error
proof —
have wp-event verify-monad
  (soundness-bad-event-aligned-transcript-partial-candidate s) s ≤
wp-event verify-monad
  (composition-bad-with-partial-candidates s) s +
wp-event verify-monad
  (trace-fri-bad-with-partial-candidates s) s +
wp-event verify-monad
  (composition-fri-bad-with-partial-candidates s) s +
wp-event verify-monad
  (query-bad-with-aligned-transcript-partial-candidates s) s
unfolding soundness-bad-event-aligned-transcript-partial-candidate-def
by (rule wp-event-union-bound4)
also have ... ≤
  composition-error + trace-fri-error' + composition-fri-error' +
  query-error
by (intro add-mono comp-bound trace-fri-bound comp-fri-bound
  query-bound)
finally show ?thesis .
qed

lemma soundness-bad-event-aligned-transcript-partial-candidate-with-partial-merkle-union-bound:
fixes s :: ('f, 'a) protocol-channel-scheme
assumes merkle-bound:
  wp-event verify-monad (partial-merkle-inconsistency-bad s) s ≤
  merkle-error
and comp-bound:
  wp-event verify-monad
    (composition-bad-with-partial-candidates s) s ≤
    composition-error
and trace-fri-bound:
  wp-event verify-monad
    (trace-fri-bad-with-partial-candidates s) s ≤
    trace-fri-error'
and comp-fri-bound:
  wp-event verify-monad
    (composition-fri-bad-with-partial-candidates s) s ≤

```

```

    composition-fri-error'
  and query-bound:
    wp-event verify-monad
      (query-bad-with-aligned-transcript-partial-candidates s) s ≤
    query-error
  shows
    wp-event verify-monad
      (soundness-bad-event-aligned-transcript-partial-candidate-with-partial-merkle s) s
    ≤
    merkle-error + composition-error + trace-fri-error' +
    composition-fri-error' + query-error
  proof -
    have partial-bound:
      wp-event verify-monad
        (soundness-bad-event-aligned-transcript-partial-candidate s) s ≤
        composition-error + trace-fri-error' + composition-fri-error' +
        query-error
    by (rule
        soundness-bad-event-aligned-transcript-partial-candidate-union-bound
        [OF comp-bound trace-fri-bound comp-fri-bound query-bound])
    have wp-event verify-monad
      (soundness-bad-event-aligned-transcript-partial-candidate-with-partial-merkle s) s
    ≤
    wp-event verify-monad (partial-merkle-inconsistency-bad s) s +
    wp-event verify-monad
      (soundness-bad-event-aligned-transcript-partial-candidate s) s
    unfolding
      soundness-bad-event-aligned-transcript-partial-candidate-with-partial-merkle-def
    by (rule wp-event-union-bound)
    also have ... ≤
      merkle-error +
      (composition-error + trace-fri-error' + composition-fri-error' +
      query-error)
    by (intro add-mono merkle-bound partial-bound)
    finally show ?thesis
    by (simp add: algebra-simps)
  qed

lemma checked-staged-security-with-data-state-aligned-transcript-partial-candidate-union-bound:
  fixes composition-error trace-fri-error' composition-fri-error'
  query-error :: prob
  assumes comp-bound:
    wp-event (checked-staged-security-experiment-with-data-state A)
      (staged-security-with-data-state-verifier-event
        composition-bad-with-partial-candidates)
      adversary-initial-state ≤ composition-error
  and trace-fri-bound:
    wp-event (checked-staged-security-experiment-with-data-state A)
      (staged-security-with-data-state-verifier-event

```

```

    trace-fri-bad-with-partial-candidates)
  adversary-initial-state ≤ trace-fri-error'
and comp-fri-bound:
  wp-event (checked-staged-security-experiment-with-data-state A)
    (staged-security-with-data-state-verifier-event
      composition-fri-bad-with-partial-candidates)
  adversary-initial-state ≤ composition-fri-error'
and query-bound:
  wp-event (checked-staged-security-experiment-with-data-state A)
    (staged-security-with-data-state-verifier-event
      query-bad-with-aligned-transcript-partial-candidates)
  adversary-initial-state ≤ query-error
shows
  wp-event (checked-staged-security-experiment-with-data-state A)
    (staged-security-with-data-state-verifier-event
      soundness-bad-event-aligned-transcript-partial-candidate)
  adversary-initial-state ≤
    composition-error + trace-fri-error' + composition-fri-error' +
    query-error
proof –
  let ?M = checked-staged-security-experiment-with-data-state A
  let ?E = staged-security-with-data-state-verifier-event
  let ?comp = ?E composition-bad-with-partial-candidates
  let ?trace = ?E trace-fri-bad-with-partial-candidates
  let ?comp-fri = ?E composition-fri-bad-with-partial-candidates
  let ?query = ?E query-bad-with-aligned-transcript-partial-candidates
  let ?partial = ?E soundness-bad-event-aligned-transcript-partial-candidate
  have wp-event ?M ?partial adversary-initial-state ≤
    wp-event ?M
      (λout. ?comp out ∨ ?trace out ∨ ?comp-fri out ∨
        ?query out)
    adversary-initial-state
  by (rule wp-event-mono)
    (auto simp: staged-security-with-data-state-verifier-event-def
      soundness-bad-event-aligned-transcript-partial-candidate-def
      split: option.splits prod.splits)
  also have ... ≤
    wp-event ?M ?comp adversary-initial-state +
    wp-event ?M ?trace adversary-initial-state +
    wp-event ?M ?comp-fri adversary-initial-state +
    wp-event ?M ?query adversary-initial-state
  by (rule wp-event-union-bound4)
  also have ... ≤
    composition-error + trace-fri-error' + composition-fri-error' +
    query-error
  by (intro add-mono comp-bound trace-fri-bound comp-fri-bound
    query-bound)
  finally show ?thesis .
qed

```

lemma *checked-staged-security-with-data-state-aligned-transcript-partial-candidate-with-partial-merkle-component-un*
fixes *merkle-error composition-error trace-fri-error'*
composition-fri-error' query-error :: prob
assumes *merkle-bound*:
 wp-event (checked-staged-security-experiment-with-data-state A)
 (*staged-security-with-data-state-verifier-event*
 partial-merkle-inconsistency-bad)
 adversary-initial-state ≤ merkle-error
and *comp-bound*:
 wp-event (checked-staged-security-experiment-with-data-state A)
 (*staged-security-with-data-state-verifier-event*
 composition-bad-with-partial-candidates)
 adversary-initial-state ≤ composition-error
and *trace-fri-bound*:
 wp-event (checked-staged-security-experiment-with-data-state A)
 (*staged-security-with-data-state-verifier-event*
 trace-fri-bad-with-partial-candidates)
 adversary-initial-state ≤ trace-fri-error'
and *comp-fri-bound*:
 wp-event (checked-staged-security-experiment-with-data-state A)
 (*staged-security-with-data-state-verifier-event*
 composition-fri-bad-with-partial-candidates)
 adversary-initial-state ≤ composition-fri-error'
and *query-bound*:
 wp-event (checked-staged-security-experiment-with-data-state A)
 (*staged-security-with-data-state-verifier-event*
 query-bad-with-aligned-transcript-partial-candidates)
 adversary-initial-state ≤ query-error
shows
 wp-event (checked-staged-security-experiment-with-data-state A)
 (*staged-security-with-data-state-verifier-event*
 soundness-bad-event-aligned-transcript-partial-candidate-with-partial-merkle)
 adversary-initial-state ≤
 merkle-error + composition-error + trace-fri-error' +
 composition-fri-error' + query-error
proof –
have *partial-bound*:
 wp-event (checked-staged-security-experiment-with-data-state A)
 (*staged-security-with-data-state-verifier-event*
 soundness-bad-event-aligned-transcript-partial-candidate)
 adversary-initial-state ≤
 composition-error + trace-fri-error' + composition-fri-error' +
 query-error
by (rule
 checked-staged-security-with-data-state-aligned-transcript-partial-candidate-union-bound
 [*OF comp-bound trace-fri-bound comp-fri-bound query-bound*])
let *?M = checked-staged-security-experiment-with-data-state A*
let *?E = staged-security-with-data-state-verifier-event*

```

let ?merkle = ?E partial-merkle-inconsistency-bad
let ?partial = ?E soundness-bad-event-aligned-transcript-partial-candidate
let ?with-merkle =
  ?E soundness-bad-event-aligned-transcript-partial-candidate-with-partial-merkle
have wp-event ?M ?with-merkle adversary-initial-state ≤
  wp-event ?M (λout. ?merkle out ∨ ?partial out)
  adversary-initial-state
by (rule wp-event-mono)
  (auto simp: staged-security-with-data-state-verifier-event-def
    soundness-bad-event-aligned-transcript-partial-candidate-with-partial-merkle-def
    split: option.splits prod.splits)
also have ... ≤
  wp-event ?M ?merkle adversary-initial-state +
  wp-event ?M ?partial adversary-initial-state
by (rule wp-event-union-bound)
also have ... ≤
  merkle-error +
  (composition-error + trace-fri-error' + composition-fri-error' +
    query-error)
by (intro add-mono merkle-bound partial-bound)
finally show ?thesis
by (simp add: algebra-simps)
qed

```

lemma checked-staged-security-with-data-state-aligned-transcript-partial-candidate-with-partial-merkle-union-bound:

```

fixes merkle-error composition-error trace-fri-error'
  composition-fri-error' query-error :: prob
assumes merkle-bound:
  wp-event (checked-staged-security-experiment-with-data-state A)
  (staged-security-with-data-state-verifier-event
    partial-merkle-inconsistency-bad)
  adversary-initial-state ≤ merkle-error
and partial-bound:
  wp-event (checked-staged-security-experiment-with-data-state A)
  (staged-security-with-data-state-verifier-event
    soundness-bad-event-aligned-transcript-partial-candidate)
  adversary-initial-state ≤
  composition-error + trace-fri-error' + composition-fri-error' +
  query-error

```

shows

```

wp-event (checked-staged-security-experiment-with-data-state A)
(staged-security-with-data-state-verifier-event
  soundness-bad-event-aligned-transcript-partial-candidate-with-partial-merkle)
adversary-initial-state ≤
merkle-error + composition-error + trace-fri-error' +
composition-fri-error' + query-error

```

proof –

```

let ?M = checked-staged-security-experiment-with-data-state A
let ?E = staged-security-with-data-state-verifier-event

```

```

let ?merkle = ?E partial-merkle-inconsistency-bad
let ?partial = ?E soundness-bad-event-aligned-transcript-partial-candidate
have wp-event ?M
  (?E soundness-bad-event-aligned-transcript-partial-candidate-with-partial-merkle)
  adversary-initial-state ≤
  wp-event ?M (λout. ?merkle out ∨ ?partial out)
  adversary-initial-state
by (rule wp-event-mono)
  (auto simp: staged-security-with-data-state-verifier-event-def
    soundness-bad-event-aligned-transcript-partial-candidate-with-partial-merkle-def
    split: option.splits prod.splits)
also have ... ≤
  wp-event ?M ?merkle adversary-initial-state +
  wp-event ?M ?partial adversary-initial-state
by (rule wp-event-union-bound)
also have ... ≤
  merkle-error +
  (composition-error + trace-fri-error' + composition-fri-error' +
    query-error)
by (intro add-mono merkle-bound partial-bound)
finally show ?thesis
by (simp add: algebra-simps)
qed

```

```

lemma checked-staged-security-with-data-state-aligned-transcript-query-bad-imp-partial-opening-hit-on-support:
assumes wf: staged-budget-wellformed budgets
and controlled: staged-adversary-controlled budgets A
and support:
  Some (((data, attacker-state), result), final-state) ∈
  set-dist
  (execute
    (checked-staged-security-experiment-with-data-state A)
    adversary-initial-state)
and bad:
  staged-security-with-data-state-verifier-event
  query-bad-with-aligned-transcript-partial-candidates
  (Some (((data, attacker-state), result), final-state))
shows
  staged-security-with-data-state-query-partial-opening-hit
  (Some (((data, attacker-state), result), final-state))
proof –
let ?s =
  verifier-state-from-adversary attacker-state
  (staged-proof-transcript data)
from checked-staged-security-experiment-with-data-state-outcomeE
  [OF support]
obtain builder where builder:
  Some (data, attacker-state) ∈
  set-dist

```

```

      (execute (checked-staged-transcript-program A)
        adversary-initial-state)
and verifier:
  Some (result, final-state) ∈ set-dist (execute verify-monad ?s)
by blast
have staged-header:
  verifier-header-transcript ?s
    (staged-trace-root data)
    (staged-trace-fri-roots data)
    (staged-trace-final data)
    (staged-alphas data)
    (staged-degree data)
    (staged-composition-fri-roots data)
    (staged-composition-final data)
    (List.concat (staged-query-chunks data))
by (rule checked-staged-transcript-program-outcome-header-transcript
  [OF wf controlled builder])
from bad obtain fr f-fri-roots f-final as dg composition-fri-roots
  final query-idxs trace-openings composition-openings trace-table
  composition-table where partial:
  accepted-with-partial-initial-openings-aligned-transcript-consistent ?s
    (Some (result, final-state)) fr f-fri-roots f-final as dg
    composition-fri-roots final query-idxs trace-openings
    composition-openings
and trace-candidate:
  partial-trace-table-candidate trace-table trace-openings
and comp-candidate:
  partial-composition-table-candidate composition-table
    composition-openings
and trace-low: trace-table-low-degree trace-table
and comp-low:
  composition-table-low-degree maxDegree composition-table
and not-all:
  ¬ all-queries-consistent trace-table composition-table as
unfolding staged-security-with-data-state-verifier-event-def
  query-bad-with-aligned-transcript-partial-candidates-def
by simp blast
have aligned:
  accepted-with-partial-initial-openings-aligned-consistent ?s
    (Some (result, final-state)) fr f-fri-roots f-final as dg
    composition-fri-roots final query-idxs trace-openings
    composition-openings
by (rule
  accepted-with-partial-initial-openings-aligned-transcript-consistent-imp-aligned
  [OF partial])
have shape:
  accepted-transcript-shape ?s (Some (result, final-state)) as
    query-idxs
by (rule

```

```

    accepted-with-partial-initial-openings-aligned-transcript-consistent-imp-shape
    [OF partial])
from checked-staged-security-with-data-state-accepted-shape-query-chunks
    [OF wf controlled support shape]
obtain raw-idxs where
    as-eq: as = staged-alphas data
    and len-raw: length raw-idxs = rounds
    and query-idxs-eq:
        query-idxs = map (λraw. index (to-nat raw)) raw-idxs
    and lookup:
        ∀ i < rounds.
            fmllookup (HashMap final-state)
                (QueryIndexChallenge i
                 (state-after-query-chunks
                  (staged-query-start-hash data)
                  (staged-query-chunks data) i)) =
                Some (raw-idxs ! i)
    by blast
have aligned0:
    accepted-with-partial-initial-openings-aligned ?s
    (Some (result, final-state)) fr f-fri-roots f-final as dg
    composition-fri-roots final query-idxs trace-openings
    composition-openings
    by (rule accepted-with-partial-initial-openings-aligned-consistent-imp-aligned
        [OF aligned])
from accepted-with-partial-initial-openings-aligned-shapes[OF aligned0]
obtain rest where header:
    verifier-header-transcript ?s fr f-fri-roots f-final as dg
    composition-fri-roots final rest
    by blast
have comp-nonempty: composition-fri-roots ≠ []
    using accepted-with-partial-initial-openings-aligned-shapes(1)
    [OF aligned0] .
have trace-partial:
    accepted-with-partial-trace-openings ?s
    (Some (result, final-state)) fr query-idxs trace-openings
    using accepted-with-partial-initial-openings-aligned-shapes(3)
    [OF aligned0] .
have comp-partial:
    accepted-with-partial-composition-openings ?s
    (Some (result, final-state)) (hd composition-fri-roots) query-idxs
    composition-openings
    using accepted-with-partial-initial-openings-aligned-shapes(4)
    [OF aligned0] .
have header-eq:
    fr = staged-trace-root data ∧
    f-fri-roots = staged-trace-fri-roots data ∧
    f-final = staged-trace-final data ∧
    as = staged-alphas data ∧

```



```

  dg = staged-degree data ∧
  composition-fri-roots = staged-composition-fri-roots data ∧
  final = staged-composition-final data ∧
  rest = List.concat (staged-query-chunks data)
using verifier-header-transcript-unique[OF header staged-header]
by simp
let ?i = 0
have i-bound: ?i < rounds
  using rounds-positive by simp
have round-consistent:
  partial-query-round-consistent trace-openings composition-openings as
    ?i (query-idxs ! ?i)
  using aligned i-bound
  unfolding accepted-with-partial-initial-openings-aligned-consistent-def
  by simp
have opening-consistent:
  partial-query-openings-consistent (trace-openings ! ?i)
    (composition-openings ! ?i) as (query-idxs ! ?i)
  using round-consistent
  unfolding partial-query-round-consistent-def
    partial-query-openings-consistent-def
  by simp
have idx-sample: query-idxs ! ?i ∈ query-sample-space
  using round-consistent
  unfolding partial-query-round-consistent-def by simp
have trace-table:
  partial-authenticated-table fr (scale * clength)
    (trace-openings ! ?i) final-state
  using trace-partial i-bound
  unfolding accepted-with-partial-trace-openings-def by blast
have comp-table:
  partial-authenticated-table (hd composition-fri-roots)
    (scale * clength) (composition-openings ! ?i) final-state
  using comp-partial i-bound
  unfolding accepted-with-partial-composition-openings-def by blast
have witness:
  (trace-openings ! ?i, composition-openings ! ?i) ∈
    query-header-supported-partial-opening-witnesses ?s fr f-fri-roots
    f-final as dg composition-fri-roots final
  unfolding query-header-supported-partial-opening-witnesses-def
  using comp-nonempty verifier trace-table comp-table header by blast
have idx-in-header-set:
  query-idxs ! ?i ∈
    query-header-supported-partial-opening-success-indices-at ?s fr
    f-fri-roots f-final as dg composition-fri-roots final ?i
  unfolding query-header-supported-partial-opening-success-indices-at-def
  using idx-sample witness opening-consistent by blast
have idx-in-staged-set:
  query-idxs ! ?i ∈

```

```

    query-header-supported-partial-opening-success-indices-at ?s
      (staged-trace-root data)
      (staged-trace-fri-roots data)
      (staged-trace-final data)
      (staged-alphas data)
      (staged-degree data)
      (staged-composition-fri-roots data)
      (staged-composition-final data)
      ?i
  using idx-in-header-set header-eq by simp
have raw-idx:
  index (to-nat (raw-idxs ! ?i)) = query-idxs ! ?i
  using query-idxs-eq len-raw i-bound by simp
have lookup-i:
  fmlookup (HashMap final-state)
    (QueryIndexChallenge ?i
      (state-after-query-chunks
        (staged-query-start-hash data)
        (staged-query-chunks data) ?i)) =
    Some (raw-idxs ! ?i)
  using lookup i-bound by simp
have hit-at:
  staged-security-with-data-state-query-partial-opening-hit-at ?i
    (Some (((data, attacker-state), result), final-state))
  unfolding staged-security-with-data-state-query-partial-opening-hit-at-def
  using i-bound lookup-i idx-in-staged-set raw-idx
  by (simp add: Let-def)
show ?thesis
  by (rule staged-security-with-data-state-query-partial-opening-hitI
    [OF i-bound hit-at])
qed

lemma checked-staged-security-with-data-state-aligned-transcript-query-bad-bound-from-partial-opening-hit:
  assumes wf: staged-budget-wellformed budgets
  and controlled: staged-adversary-controlled budgets A
  shows
    wp-event (checked-staged-security-experiment-with-data-state A)
      (staged-security-with-data-state-verifier-event
        query-bad-with-aligned-transcript-partial-candidates)
      adversary-initial-state ≤
    wp-event (checked-staged-security-experiment-with-data-state A)
      staged-security-with-data-state-query-partial-opening-hit
      adversary-initial-state
proof (rule wp-event-mono-on-support)
  fix out
  assume support:
    out ∈
      set-dist
      (execute

```

```

    (checked-staged-security-experiment-with-data-state A)
    adversary-initial-state)
and bad:
  staged-security-with-data-state-verifier-event
  query-bad-with-aligned-transcript-partial-candidates out
show staged-security-with-data-state-query-partial-opening-hit out
proof (cases out)
  case None
  then show ?thesis
    using bad
    unfolding staged-security-with-data-state-verifier-event-def by simp
next
case (Some packed)
then obtain data attacker-state result final-state where out-eq:
  out = Some (((data, attacker-state), result), final-state)
  by (cases packed, auto split: prod.splits)
show ?thesis
  unfolding out-eq
  by (rule
    checked-staged-security-with-data-state-aligned-transcript-query-bad-imp-partial-opening-hit-on-support
    [OF wf controlled])
  (use support bad out-eq in simp-all)
qed
qed

lemma selected-query-openings-align-with-replayed-query-key:
  assumes i-bound:  $i < \text{rounds}$ 
  and len-raw:  $\text{length raw-idxs} = \text{rounds}$ 
  and query-idxs-eq:
    query-idxs =  $\text{map } (\lambda \text{raw. index (to-nat raw)}) \text{ raw-idxs}$ 
  and selected-idx:  $\text{idx} = \text{index (to-nat raw)}$ 
  and selected-lookup:
    fmlookup (HashMap final-state)
      (QueryIndexChallenge k prefix-state) = Some raw
  and replay-lookup:
    fmlookup (HashMap final-state)
      (QueryIndexChallenge k prefix-state) = Some (raw-idxs ! i)
  and trace-indices:
    map opening-index trace-openings = powers-scaled idx
  and comp-indices:
    map opening-index composition-openings =
      [idx, fri-sibling-index (scale * clength) idx]
  and consistent:
    partial-query-openings-consistent trace-openings
      composition-openings as idx
shows
  idx = query-idxs ! i
  map opening-index trace-openings = powers-scaled (query-idxs ! i)
  map opening-index composition-openings =

```

```

    [query-idxs ! i,
     fri-sibling-index (scale * clength) (query-idxs ! i)]
  partial-query-openings-consistent trace-openings composition-openings
  as (query-idxs ! i)
proof -
  have raw-eq: raw = raw-idxs ! i
    using selected-lookup replay-lookup by simp
  have idx-eq: idx = query-idxs ! i
    using selected-idx raw-eq query-idxs-eq len-raw i-bound by simp
  show idx = query-idxs ! i
    by (rule idx-eq)
  show map opening-index trace-openings = powers-scaled (query-idxs ! i)
    using trace-indices idx-eq by simp
  show map opening-index composition-openings =
    [query-idxs ! i,
     fri-sibling-index (scale * clength) (query-idxs ! i)]
    using comp-indices idx-eq by simp
  show partial-query-openings-consistent trace-openings composition-openings
    as (query-idxs ! i)
    using consistent idx-eq by simp
qed

```

lemma *selected-query-openings-align-with-replayed-query-index:*

```

assumes i-bound: i < rounds
and len-raw: length raw-idxs = rounds
and query-idxs-eq:
  query-idxs = map (λraw. index (to-nat raw)) raw-idxs
and selected-idx: idx = index (to-nat raw)
and selected-lookup:
  fmlookup (HashMap final-state)
    (QueryIndexChallenge i prefix-state) = Some raw
and replay-lookup:
  fmlookup (HashMap final-state)
    (QueryIndexChallenge i prefix-state) = Some (raw-idxs ! i)
and trace-indices:
  map opening-index trace-openings = powers-scaled idx
and comp-indices:
  map opening-index composition-openings =
    [idx, fri-sibling-index (scale * clength) idx]
and consistent:
  partial-query-openings-consistent trace-openings
    composition-openings as idx

```

shows

```

  idx = query-idxs ! i
  map opening-index trace-openings = powers-scaled (query-idxs ! i)
  map opening-index composition-openings =
    [query-idxs ! i,
     fri-sibling-index (scale * clength) (query-idxs ! i)]
  partial-query-openings-consistent trace-openings composition-openings

```

$as (query_idxs ! i)$
by (*rule selected-query-openings-align-with-replayed-query-key*
 $[OF\ i\text{-bound}\ len\text{-raw}\ query_idxs\text{-eq}\ selected_idx\ selected_lookup$
 $replay_lookup\ trace_indices\ comp_indices\ consistent]) +$

lemma *ntimes-verifier-query-rounds-selected-authenticated-chunks-aligned-with-replay-at:*
assumes *fl-eq:* $fl = (b, composition_root) \# fl_tail$
and *outcome:*
 $Some (results, final_state) \in$
 set_dist
 $(execute$
 $(ntimes$
 $(verifier_query_round_program\ fr\ f_fl\ f_final\ as\ fl\ final)$
 $rounds)$
 $query_state)$
and *i-bound:* $i < rounds$
and *len-raw:* $length\ raw_idxs = rounds$
and *query-idxs-eq:*
 $query_idxs = map (\lambda raw. index\ (to_nat\ raw))\ raw_idxs$
and *len-query-chunks:* $length\ query_chunks = rounds$
and *transcript-query:*
 $PTranscript\ query_state =$
 $List.concat\ query_chunks\ @\ PTranscript\ final_state$
and *query-chunk:*
 $\bigwedge i. i < rounds \implies$
 $verifier_query_round_chunk\ (query_idxs ! i)$
 $(map\ snd\ f_fl)\ (map\ snd\ fl)\ (query_chunks ! i)$
and *replay-lookup:*
 $\bigwedge i. i < rounds \implies$
 $fmlookup\ (HashMap\ final_state)$
 $(QueryIndexChallenge\ (PQueryCounter\ query_state + i)$
 $(state_after_query_chunks$
 $(PState\ query_state)\ query_chunks\ i)) =$
 $Some\ (raw_idxs ! i)$
obtains *trace-openings composition-openings where*
 $map\ opening_index\ trace_openings = powers_scaled\ (query_idxs ! i)$
 $partial_authenticated_table\ fr\ (scale * clength)\ trace_openings$
 $final_state$
 $partial_authenticated_table\ composition_root\ (scale * clength)$
 $composition_openings\ final_state$
 $map\ opening_index\ composition_openings =$
 $[query_idxs ! i,$
 $fri_sibling_index\ (scale * clength)\ (query_idxs ! i)]$
 $partial_query_openings_consistent\ trace_openings\ composition_openings$
 $as\ (query_idxs ! i)$
proof (*rule ntimes-verifier-query-rounds-selected-authenticated-chunks-at*
 $[OF\ fl_eq\ outcome\ i\text{-bound}])$
fix *raw idx prefix-query-idxs prefix-chunks chunk suffix-query-idxs*
 $suffix_chunks\ trace_openings\ composition_openings$

```

assume selected-idx: idx = index (to-nat raw)
and len-prefix-chunks: length prefix-chunks = i
and len-suffix-chunks: length suffix-chunks = rounds - Suc i
and chunk-shape:
  verifier-query-round-chunk idx (map snd f-fl) (map snd fl) chunk
and prefix-rounds:
   $\bigwedge j. j < i \implies$ 
    verifier-query-round-chunk (prefix-query-idxs ! j)
      (map snd f-fl) (map snd fl) (prefix-chunks ! j)
and suffix-rounds:
   $\bigwedge j. j < \text{rounds} - \text{Suc } i \implies$ 
    verifier-query-round-chunk (suffix-query-idxs ! j)
      (map snd f-fl) (map snd fl) (suffix-chunks ! j)
and selected-transcript:
  PTranscript query-state =
    List.concat (prefix-chunks @ chunk # suffix-chunks) @
      PTranscript final-state
and trace-indices:
  map opening-index trace-openings = powers-scaled idx
and trace-table:
  partial-authenticated-table fr (scale * clength) trace-openings
    final-state
and comp-table:
  partial-authenticated-table composition-root (scale * clength)
    composition-openings final-state
and comp-indices:
  map opening-index composition-openings =
    [idx, fri-sibling-index (scale * clength) idx]
and consistent:
  partial-query-openings-consistent trace-openings
    composition-openings as idx
and selected-lookup:
  fmlookup (HashMap final-state)
    (QueryIndexChallenge (PQueryCounter query-state + i)
      (state-after-query-chunks (PState query-state) prefix-chunks i)) =
    Some raw
let ?selected-chunks = prefix-chunks @ chunk # suffix-chunks
have len-selected-chunks: length ?selected-chunks = rounds
using len-prefix-chunks len-suffix-chunks i-bound by simp
have selected-chunk-lens:
   $\bigwedge j. j < \text{rounds} \implies$ 
    length (?selected-chunks ! j) =
      verifier-query-round-transcript-length (query-idxs ! j)
        (map snd f-fl) (map snd fl)
proof -
  fix j
  assume j-bound: j < rounds
  show
    length (?selected-chunks ! j) =

```

```

    verifier-query-round-transcript-length (query-idxs ! j)
      (map snd f-fl) (map snd fl)
  proof (cases j < i)
  case True
  have nth-eq: ?selected-chunks ! j = prefix-chunks ! j
    using True len-prefix-chunks by (simp add: nth-append)
  have
    length (prefix-chunks ! j) =
      verifier-query-round-transcript-length
        (prefix-query-idxs ! j) (map snd f-fl) (map snd fl)
    by (rule verifier-query-round-chunk-length[OF prefix-rounds[OF True]])
  then show ?thesis
    using nth-eq
    by (simp add: verifier-query-round-transcript-length-index-irrelevant)
next
case False
show ?thesis
proof (cases j = i)
case True
have nth-eq: ?selected-chunks ! j = chunk
  using True len-prefix-chunks by (simp add: nth-append)
have
  length chunk =
    verifier-query-round-transcript-length idx
      (map snd f-fl) (map snd fl)
  by (rule verifier-query-round-chunk-length[OF chunk-shape])
then show ?thesis
  using nth-eq
  by (simp add:
    verifier-query-round-transcript-length-index-irrelevant)
next
case False
let ?k = j - Suc i
have ge: Suc i ≤ j
  using <¬ j < i> False by linarith
have k-bound: ?k < rounds - Suc i
  using ge j-bound by linarith
have nth-eq: ?selected-chunks ! j = suffix-chunks ! ?k
  using ge len-prefix-chunks by (simp add: nth-append)
have
  length (suffix-chunks ! ?k) =
    verifier-query-round-transcript-length
      (suffix-query-idxs ! ?k) (map snd f-fl) (map snd fl)
  by (rule verifier-query-round-chunk-length
    [OF suffix-rounds[OF k-bound]])
then show ?thesis
  using nth-eq
  by (simp add:
    verifier-query-round-transcript-length-index-irrelevant)

```

```

    qed
  qed
  qed
  have query-chunk-lens:
     $\bigwedge j. j < \text{rounds} \implies$ 
     $\text{length } (\text{query-chunks } ! j) =$ 
     $\text{verifier-query-round-transcript-length } (\text{query-idxs } ! j)$ 
     $(\text{map snd f-fl}) (\text{map snd fl})$ 
  using query-chunk verifier-query-round-chunk-length by blast
  have concat-eq:  $\text{List.concat } ?\text{selected-chunks} = \text{List.concat } \text{query-chunks}$ 
  using selected-transcript transcript-query by simp
  have chunks-eq:  $?selected-chunks = \text{query-chunks}$ 
  proof -
    have same-lens:
       $\bigwedge j. j < \text{length } ?\text{selected-chunks} \implies$ 
       $\text{length } (?selected-chunks ! j) = \text{length } (\text{query-chunks } ! j)$ 
    using selected-chunk-lens query-chunk-lens len-selected-chunks by simp
  have concat-eq-empty:
     $\text{List.concat } ?\text{selected-chunks} @ [] = \text{List.concat } \text{query-chunks}$ 
  using concat-eq by simp
  have len-eq:  $\text{length } ?\text{selected-chunks} = \text{length } \text{query-chunks}$ 
  using len-selected-chunks len-query-chunks by simp
  have  $?selected-chunks = \text{query-chunks} \wedge ([] :: 'f \text{ list}) = []$ 
  by (rule concat-append-eq-concat-same-chunk-lengths
    [OF len-eq same-lens concat-eq-empty])
  then show ?thesis
  by simp
  qed
  have prefix-state-eq:
     $\text{state-after-query-chunks } (P\text{State } \text{query-state}) \text{ prefix-chunks } i =$ 
     $\text{state-after-query-chunks } (P\text{State } \text{query-state}) \text{ query-chunks } i$ 
  proof -
    have
       $\text{state-after-query-chunks } (P\text{State } \text{query-state}) ?\text{selected-chunks } i =$ 
       $\text{state-after-query-chunks } (P\text{State } \text{query-state}) \text{ prefix-chunks } i$ 
    by (rule state-after-query-chunks-append-prefix[OF len-prefix-chunks])
  then show ?thesis
  using chunks-eq by simp
  qed
  have selected-lookup-global:
     $\text{fmlookup } (\text{HashMap } \text{final-state})$ 
     $(\text{QueryIndexChallenge } (P\text{QueryCounter } \text{query-state} + i)$ 
     $(\text{state-after-query-chunks } (P\text{State } \text{query-state}) \text{ query-chunks } i)) =$ 
     $\text{Some raw}$ 
  using selected-lookup prefix-state-eq by simp
  have aligned:
     $\text{idx} = \text{query-idxs } ! i$ 
     $\text{map opening-index trace-openings} = \text{powers-scaled } (\text{query-idxs } ! i)$ 
     $\text{map opening-index composition-openings} =$ 

```



```

    [query-idxs ! i,
     fri-sibling-index (scale * clength) (query-idxs ! i)]
  partial-query-openings-consistent trace-openings composition-openings
  as (query-idxs ! i)
  by (rule selected-query-openings-align-with-replayed-query-key
      [OF i-bound len-raw query-idxs-eq selected-idx
       selected-lookup-global replay-lookup[OF i-bound] trace-indices
       comp-indices consistent])+)
  show ?thesis
  by (rule that)
      (use aligned trace-table comp-table in simp-all)
qed

```

lemma *ntimes-verifier-query-rounds-aligned-authenticated-openings-consistent-with-replay:*

assumes *fl-eq*: $fl = (b, \text{composition-root}) \# fl\text{-tail}$

and *outcome*:

Some (results, final-state) ∈
set-dist
(execute
(ntimes
(verifier-query-round-program fr f-fl f-final as fl final)
rounds)
query-state)

and *len-raw*: $\text{length raw-idxs} = \text{rounds}$

and *query-idxs-eq*:

query-idxs = map (λraw. index (to-nat raw)) raw-idxs

and *len-query-chunks*: $\text{length query-chunks} = \text{rounds}$

and *transcript-query*:

PTranscript query-state =
List.concat query-chunks @ PTranscript final-state

and *query-chunk*:

$\bigwedge i. i < \text{rounds} \implies$
verifier-query-round-chunk (query-idxs ! i)
(map snd f-fl) (map snd fl) (query-chunks ! i)

and *replay-lookup*:

$\bigwedge i. i < \text{rounds} \implies$
fmlookup (HashMap final-state)
(QueryIndexChallenge (PQueryCounter query-state + i)
(state-after-query-chunks
(PState query-state) query-chunks i)) =
Some (raw-idxs ! i)

obtains *trace-openings composition-openings where*

length trace-openings = rounds

length composition-openings = rounds

$\bigwedge i. i < \text{rounds} \implies$

map opening-index (trace-openings ! i) =
powers-scaled (query-idxs ! i)

$\bigwedge i. i < \text{rounds} \implies$

*partial-authenticated-table fr (scale * clength)*

```

    (trace-openings ! i) final-state
   $\wedge i. i < \text{rounds} \implies$ 
    partial-authenticated-table composition-root (scale * clength)
    (composition-openings ! i) final-state
   $\wedge i. i < \text{rounds} \implies$ 
    map opening-index (composition-openings ! i) =
      [query-idxs ! i,
       fri-sibling-index (scale * clength) (query-idxs ! i)]
   $\wedge i. i < \text{rounds} \implies$ 
    partial-query-round-consistent trace-openings composition-openings as i
    (query-idxs ! i)
proof -
let ?P =
   $\lambda i w.$ 
    map opening-index (fst w) = powers-scaled (query-idxs ! i)  $\wedge$ 
    partial-authenticated-table fr (scale * clength) (fst w)
    final-state  $\wedge$ 
    partial-authenticated-table composition-root (scale * clength)
    (snd w) final-state  $\wedge$ 
    map opening-index (snd w) =
      [query-idxs ! i,
       fri-sibling-index (scale * clength) (query-idxs ! i)]  $\wedge$ 
    query-idxs ! i  $\in$  query-sample-space  $\wedge$ 
    snd w  $\neq \square$   $\wedge$ 
    opening-value (snd w ! 0) =
      cp-eval as (map opening-value (fst w))
      (h  $\wedge$  query-idxs ! i * shift)
have ex-round:  $\wedge i. i < \text{rounds} \implies \exists w. ?P i w$ 
proof -
  fix i
  assume i-bound:  $i < \text{rounds}$ 
  show  $\exists w. ?P i w$ 
  proof (rule
    ntimes-verifier-query-rounds-selected-authenticated-chunks-aligned-with-replay-at
    [OF fl-eq outcome i-bound len-raw query-idxs-eq len-query-chunks
     transcript-query query-chunk replay-lookup])
  fix trace-openings composition-openings
  assume trace-indices:
    map opening-index trace-openings =
      powers-scaled (query-idxs ! i)
  and trace-table:
    partial-authenticated-table fr (scale * clength) trace-openings
    final-state
  and comp-table:
    partial-authenticated-table composition-root (scale * clength)
    composition-openings final-state
  and comp-indices:
    map opening-index composition-openings =
      [query-idxs ! i,

```

```

      fri-sibling-index (scale * clength) (query-idxs ! i)]
    and consistent:
      partial-query-openings-consistent trace-openings
      composition-openings as (query-idxs ! i)
    have idx-sample: query-idxs ! i ∈ query-sample-space
      using consistent unfolding partial-query-openings-consistent-def
      by simp
    have comp-nonempty: composition-openings ≠ []
      using consistent unfolding partial-query-openings-consistent-def
      by simp
    have comp-value:
      opening-value (composition-openings ! 0) =
        cp-eval as (map opening-value trace-openings)
        (h ^ query-idxs ! i * shift)
      using consistent unfolding partial-query-openings-consistent-def
      by simp
    show ?thesis
      by (intro exI[of - (trace-openings, composition-openings)]
          conjI)
        (use trace-indices trace-table comp-table comp-indices consistent
            idx-sample comp-nonempty comp-value in simp-all)
  qed
qed
let ?wit = λi. SOME w. ?P i w
let ?trace-openings = map (λi. fst (?wit i)) [0..<rounds]
let ?composition-openings = map (λi. snd (?wit i)) [0..<rounds]
have round-props: ∧i. i < rounds ⇒ ?P i (?wit i)
  by (rule someI-ex[OF ex-round])
have consistent:
  ∧i. i < rounds ⇒
    partial-query-round-consistent ?trace-openings ?composition-openings as
      i (query-idxs ! i)
proof -
  fix i
  assume i-bound: i < rounds
  have chosen: ?P i (?wit i)
    by (rule round-props[OF i-bound])
  have nth:
    ?trace-openings ! i = fst (?wit i)
    ?composition-openings ! i = snd (?wit i)
    using i-bound by simp-all
  have lengths:
    length ?trace-openings = rounds
    length ?composition-openings = rounds
    by simp-all
  have idx-sample: query-idxs ! i ∈ query-sample-space
    using chosen by blast
  have trace-indices0:
    map opening-index (fst (?wit i)) =

```

```

    powers-scaled (query-idxs ! i)
  using chosen by blast
have comp-indices0:
  map opening-index (snd (?wit i)) =
    [query-idxs ! i,
     fri-sibling-index (scale * clength) (query-idxs ! i)]
  using chosen by blast
have comp-value0:
  opening-value (snd (?wit i) ! 0) =
    cp-eval as (map opening-value (fst (?wit i)))
    (h ^ query-idxs ! i * shift)
  using chosen by blast
have trace-indices:
  map opening-index (?trace-openings ! i) =
    powers-scaled (query-idxs ! i)
  using trace-indices0 nths by simp
have comp-indices:
  map opening-index (?composition-openings ! i) =
    [query-idxs ! i,
     fri-sibling-index (scale * clength) (query-idxs ! i)]
  using comp-indices0 nths by simp
have comp-value:
  opening-value ((?composition-openings ! i) ! 0) =
    cp-eval as (map opening-value (?trace-openings ! i))
    (h ^ query-idxs ! i * shift)
  using comp-value0 nths by simp
show
  partial-query-round-consistent ?trace-openings ?composition-openings as
    i (query-idxs ! i)
  by (rule partial-query-round-consistentI
      [OF i-bound idx-sample lengths trace-indices comp-indices comp-value])
qed
show ?thesis
proof (rule that[of ?trace-openings ?composition-openings])
  show length ?trace-openings = rounds
    by simp
  show length ?composition-openings = rounds
    by simp
  show  $\bigwedge i. i < rounds \implies$ 
    map opening-index (?trace-openings ! i) =
      powers-scaled (query-idxs ! i)
  proof -
    fix i
    assume i-bound:  $i < rounds$ 
    have chosen:  $?P\ i\ (?wit\ i)$ 
      by (rule round-props[OF i-bound])
    have nth:  $?trace-openings\ !\ i = fst\ (?wit\ i)$ 
      using i-bound by simp
    have trace-indices0:

```

```

    map opening-index (fst (?wit i)) =
      powers-scaled (query-idxs ! i)
    using chosen by blast
  show map opening-index (?trace-openings ! i) =
    powers-scaled (query-idxs ! i)
    using trace-indices0 nth by simp
qed
show  $\bigwedge i. i < \text{rounds} \implies$ 
  partial-authenticated-table fr (scale * clength)
  (?trace-openings ! i) final-state
proof -
  fix i
  assume i-bound:  $i < \text{rounds}$ 
  have chosen:  $?P\ i\ (?wit\ i)$ 
    by (rule round-props[OF i-bound])
  have nth:  $?trace-openings\ !\ i = \text{fst}\ (?wit\ i)$ 
    using i-bound by simp
  have trace-table0:
    partial-authenticated-table fr (scale * clength)
    (fst (?wit i)) final-state
    using chosen by blast
  show partial-authenticated-table fr (scale * clength)
    (?trace-openings ! i) final-state
    using trace-table0 nth by simp
qed
show  $\bigwedge i. i < \text{rounds} \implies$ 
  partial-authenticated-table composition-root (scale * clength)
  (?composition-openings ! i) final-state
proof -
  fix i
  assume i-bound:  $i < \text{rounds}$ 
  have chosen:  $?P\ i\ (?wit\ i)$ 
    by (rule round-props[OF i-bound])
  have nth:  $?composition-openings\ !\ i = \text{snd}\ (?wit\ i)$ 
    using i-bound by simp
  have comp-table0:
    partial-authenticated-table composition-root (scale * clength)
    (snd (?wit i)) final-state
    using chosen by blast
  show partial-authenticated-table composition-root (scale * clength)
    (?composition-openings ! i) final-state
    using comp-table0 nth by simp
qed
show  $\bigwedge i. i < \text{rounds} \implies$ 
  map opening-index (?composition-openings ! i) =
    [query-idxs ! i,
     fri-sibling-index (scale * clength) (query-idxs ! i)]
proof -
  fix i

```

```

assume i-bound:  $i < \text{rounds}$ 
have chosen:  $?P\ i\ (?wit\ i)$ 
  by (rule round-props[OF i-bound])
have nth:  $?composition-openings\ !\ i = \text{snd}\ (?wit\ i)$ 
  using i-bound by simp
have comp-indices0:
  map opening-index (snd (?wit i)) =
    [query-idxs ! i,
     fri-sibling-index (scale * clength) (query-idxs ! i)]
  using chosen by blast
show map opening-index ( $?composition-openings\ !\ i$ ) =
  [query-idxs ! i,
   fri-sibling-index (scale * clength) (query-idxs ! i)]
  using comp-indices0 nth by simp
qed
show  $\bigwedge i. i < \text{rounds} \implies$ 
  partial-query-round-consistent ?trace-openings ?composition-openings as
    i (query-idxs ! i)
  by (rule consistent)
qed
qed

lemma query-rounds-replay-accepted-with-partial-initial-openings-aligned-consistent:
assumes header0:
  verifier-header-transcript s fr f-fri-roots f-final as dg
  composition-fri-roots final rest
and comp-nonempty: composition-fri-roots  $\neq []$ 
and fl-eq: fl = (b, composition-root) # fl-tail
and f-roots: map snd f-fl = f-fri-roots
and comp-roots: map snd fl = composition-fri-roots
and query-out:
  Some (result, final-state)  $\in$ 
    set-dist
      (execute
        (ntimes
          (verifier-query-round-program fr f-fl f-final as fl final)
            rounds)
          query-state)
and len-raw: length raw-idxs = rounds
and query-idxs-eq:
  query-idxs = map ( $\lambda \text{raw}. \text{index}\ (\text{to-nat}\ \text{raw})$ ) raw-idxs
and len-query-chunks: length query-chunks = rounds
and transcript-query:
  PTranscript query-state =
    List.concat query-chunks @ PTranscript final-state
and query-chunk:
   $\bigwedge i. i < \text{rounds} \implies$ 
    verifier-query-round-chunk (query-idxs ! i)
      (map snd f-fl) (map snd fl) (query-chunks ! i)

```

```

and replay-lookup:
   $\bigwedge i. i < \text{rounds} \implies$ 
    fmllookup (HashMap final-state)
      (QueryIndexChallenge (PQueryCounter query-state + i)
        (state-after-query-chunks (PState query-state) query-chunks i)) =
        Some (raw-idxs ! i)
obtains trace-openings composition-openings where
  accepted-with-partial-initial-openings-aligned-consistent s
    (Some (result, final-state)) fr f-fri-roots f-final as dg
    composition-fri-roots final query-idxs trace-openings
    composition-openings
proof –
  have root-eq: composition-root = hd composition-fri-roots
proof –
  have composition-root # map snd fl-tail = composition-fri-roots
    using comp-roots unfolding fl-eq by simp
  then have hd composition-fri-roots = composition-root
    by (metis list.sel(1))
  then show ?thesis by simp
qed
show ?thesis
proof (rule
  ntimes-verifier-query-rounds-aligned-authenticated-openings-consistent-with-replay
    [OF fl-eq query-out len-raw query-idxs-eq len-query-chunks
      transcript-query query-chunk replay-lookup])
fix trace-openings composition-openings
assume len-trace: length trace-openings = rounds
and len-comp: length composition-openings = rounds
and trace-indices:
   $\bigwedge i. i < \text{rounds} \implies$ 
    map opening-index (trace-openings ! i) =
    powers-scaled (query-idxs ! i)
and trace-tables:
   $\bigwedge i. i < \text{rounds} \implies$ 
    partial-authenticated-table fr (scale * clength)
      (trace-openings ! i) final-state
and comp-tables:
   $\bigwedge i. i < \text{rounds} \implies$ 
    partial-authenticated-table composition-root (scale * clength)
      (composition-openings ! i) final-state
and comp-indices:
   $\bigwedge i. i < \text{rounds} \implies$ 
    map opening-index (composition-openings ! i) =
    [query-idxs ! i,
      fri-sibling-index (scale * clength) (query-idxs ! i)]
and consistent:
   $\bigwedge i. i < \text{rounds} \implies$ 
    partial-query-round-consistent trace-openings composition-openings
      as i (query-idxs ! i)

```

```

have len-query-idxs: length query-idxs = rounds
  using len-raw query-idxs-eq by simp
have trace-partial:
  accepted-with-partial-trace-openings s
    (Some (result, final-state)) fr query-idxs trace-openings
proof –
  have all-indices:
     $\forall i < \text{rounds}.$ 
    map opening-index (trace-openings ! i) =
      powers-scaled (query-idxs ! i)
  using trace-indices by blast
have all-tables:
   $\forall i < \text{rounds}.$ 
  partial-authenticated-table fr (scale * clength)
    (trace-openings ! i) final-state
  using trace-tables by blast
show ?thesis
  unfolding accepted-with-partial-trace-openings-def accepted-def
  by (intro conjI exI[of - result] exI[of - final-state])
    (use len-query-idxs len-trace all-indices all-tables in simp-all)
qed
have comp-partial:
  accepted-with-partial-composition-openings s
    (Some (result, final-state)) (hd composition-fri-roots) query-idxs
    composition-openings
proof –
  have all-indices:
     $\forall i < \text{rounds}.$ 
    map opening-index (composition-openings ! i) =
      [query-idxs ! i,
       fri-sibling-index (scale * clength) (query-idxs ! i)]
  using comp-indices by blast
have all-tables:
   $\forall i < \text{rounds}.$ 
  partial-authenticated-table (hd composition-fri-roots)
    (scale * clength) (composition-openings ! i) final-state
  using comp-tables root-eq by simp
show ?thesis
  unfolding accepted-with-partial-composition-openings-def accepted-def
  by (intro conjI exI[of - result] exI[of - final-state])
    (use len-query-idxs len-comp all-indices all-tables in simp-all)
qed
have aligned:
  accepted-with-partial-initial-openings-aligned s
    (Some (result, final-state)) fr f-fri-roots f-final as dg
    composition-fri-roots final query-idxs trace-openings
    composition-openings
proof –
  have accepted-out: accepted (Some (result, final-state))

```



```

    using trace-partial
    by (rule accepted-with-partial-trace-openings-imp-accepted)
show ?thesis
    unfolding accepted-with-partial-initial-openings-aligned-def
    by (intro conjI exI[of - rest])
        (use accepted-out comp-nonempty header0 trace-partial comp-partial
            in simp-all)
qed
have all-consistent:
   $\forall i < \text{rounds}.$ 
    partial-query-round-consistent trace-openings composition-openings as  $i$ 
    (query-idxs !  $i$ )
    using consistent by blast
have aligned-consistent:
  accepted-with-partial-initial-openings-aligned-consistent  $s$ 
  (Some (result, final-state)) fr  $f$ -fri-roots  $f$ -final as  $dg$ 
  composition-fri-roots final query-idxs trace-openings
  composition-openings
  unfolding accepted-with-partial-initial-openings-aligned-consistent-def
  using aligned all-consistent by simp
show ?thesis
  by (rule that[OF aligned-consistent])
qed
qed

lemma query-rounds-replay-accepted-with-partial-initial-openings-aligned-transcript-consistent:
  assumes header0:
    verifier-header-transcript  $s$  fr  $f$ -fri-roots  $f$ -final as  $dg$ 
    composition-fri-roots final rest
  and comp-nonempty: composition-fri-roots  $\neq []$ 
  and fl-eq:  $fl = (b, \text{composition-root}) \# fl\text{-tail}$ 
  and f-roots:  $\text{map snd } f\text{-fl} = f\text{-fri-roots}$ 
  and comp-roots:  $\text{map snd } fl = \text{composition-fri-roots}$ 
  and query-state-transcript:  $P\text{Transcript } \text{query-state} = \text{rest}$ 
  and query-state-state:
     $P\text{State } \text{query-state} =$ 
    verifier-header-state  $s$  fr  $f$ -fri-roots  $f$ -final as  $dg$ 
    composition-fri-roots final
  and query-state-counter:  $P\text{QueryCounter } \text{query-state} = P\text{QueryCounter } s$ 
  and query-out:
    Some (result, final-state)  $\in$ 
    set-dist
    (execute
      (ntimes
        (verifier-query-round-program fr  $f\text{-fl}$   $f\text{-final}$  as  $fl$  final)
        rounds)
      query-state)
  and len-raw:  $\text{length raw-idxs} = \text{rounds}$ 
  and query-idxs-eq:

```

```

    query-idxs = map (λraw. index (to-nat raw)) raw-idxs
and len-query-chunks: length query-chunks = rounds
and transcript-query:
    PTranscript query-state =
      List.concat query-chunks @ PTranscript final-state
and query-chunk:
    ∧ i. i < rounds ⇒
      verifier-query-round-chunk (query-idxs ! i)
      (map snd f-fl) (map snd fl) (query-chunks ! i)
and replay-lookup:
    ∧ i. i < rounds ⇒
      fmlookup (HashMap final-state)
      (QueryIndexChallenge (PQueryCounter query-state + i)
      (state-after-query-chunks (PState query-state) query-chunks i)) =
      Some (raw-idxs ! i)
obtains trace-openings composition-openings where
  accepted-with-partial-initial-openings-aligned-transcript-consistent s
  (Some (result, final-state)) fr f-fri-roots f-final as dg
  composition-fri-roots final query-idxs trace-openings
  composition-openings
proof –
from query-rounds-replay-accepted-with-partial-initial-openings-aligned-consistent
  [OF header0 comp-nonempty fl-eq f-roots comp-roots query-out
   len-raw query-idxs-eq len-query-chunks transcript-query
   query-chunk replay-lookup]
obtain trace-openings composition-openings where aligned:
  accepted-with-partial-initial-openings-aligned-consistent s
  (Some (result, final-state)) fr f-fri-roots f-final as dg
  composition-fri-roots final query-idxs trace-openings
  composition-openings
by blast
have transcript-rest:
  List.concat query-chunks @ PTranscript final-state = rest
using transcript-query query-state-transcript by simp
have chunks:
  ∀ i < rounds.
    verifier-query-round-chunk (query-idxs ! i)
    f-fri-roots composition-fri-roots (query-chunks ! i)
using query-chunk f-roots comp-roots by simp
have lookups:
  ∀ i < rounds.
    fmlookup (HashMap final-state)
    (QueryIndexChallenge (PQueryCounter s + i)
    (state-after-query-chunks
    (verifier-header-state s fr f-fri-roots f-final as dg
    composition-fri-roots final)
    query-chunks i)) =
    Some (raw-idxs ! i)
using replay-lookup query-state-state query-state-counter by simp

```

```

have idx-bounds:
   $\forall idx \in \text{set query-idxs}. idx < \text{clength} * \text{scale}$ 
proof
  fix idx
  assume idx  $\in \text{set query-idxs}$ 
  then obtain raw where raw: raw  $\in \text{set raw-idxs}$ 
    and idx: idx = index (to-nat raw)
    unfolding query-idxs-eq by auto
  have index (to-nat raw) < query-sample-space-size
    by (rule index-less-query-sample-space)
  also have ...  $\leq \text{clength} * \text{scale}$ 
    by (rule query-sample-space-size-le-domain)
  finally show idx < clength * scale
    using idx by simp
qed
have derived:
  verifier-query-indices-derived s (Some (result, final-state))
    (verifier-header-state s fr f-fri-roots f-final as dg
      composition-fri-roots final)
    rest f-fri-roots composition-fri-roots query-idxs
  unfolding verifier-query-indices-derived-def
  by (intro exI[of - result] exI[of - final-state]
      exI[of - raw-idxs] exI[of - query-chunks]
      exI[of - PTranscript final-state] conjI)
    (use len-raw query-idxs-eq len-query-chunks transcript-rest chunks
      lookups idx-bounds in auto)
have shape:
  accepted-transcript-shape s (Some (result, final-state)) as query-idxs
  unfolding accepted-transcript-shape-def
  by (intro exI[of - result] exI[of - final-state] exI[of - fr]
      exI[of - f-fri-roots] exI[of - f-final] exI[of - dg]
      exI[of - composition-fri-roots] exI[of - final]
      exI[of - rest] conjI)
    (use header0 derived in simp-all)
have tied:
  accepted-with-partial-initial-openings-aligned-transcript-consistent s
    (Some (result, final-state)) fr f-fri-roots f-final as dg
    composition-fri-roots final query-idxs trace-openings
    composition-openings
  unfolding
    accepted-with-partial-initial-openings-aligned-transcript-consistent-def
  using aligned shape by simp
show ?thesis
  by (rule that[OF tied])
qed

end

end

```

theory *Soundness-Aligned-Partial-Query-Prefix*

imports

Soundness-Aligned-Partial-Query-Transcript

Soundness-Staged-Candidate-Query

Soundness-Staged-Transcript

begin

Bridge aligned partial-query evidence to prefix-local staged query targets.

The lemmas here are deterministic: they say that, once a verifier query index is replayed to a prefix receive and the prefix alphas agree with the accepted transcript, the aligned partial-candidate query evidence produces a hit in the existing prefix-local target. Probability bounds remain in the staged prefix-target layer.

context *soundness*

begin

definition *checked-staged-security-experiment-with-query-prefix-data-state*

where

checked-staged-security-experiment-with-query-prefix-data-state $A\ i =$
checked-staged-query-prefix-receive-with-state $A\ i \gg=$
 $(\lambda \text{prefix-out. get } \gg=$
 $(\lambda \text{prefix-state.}$
 $\text{checked-staged-after-query-prefix-receive-with-verifier } A\ i$
 $\text{prefix-out } \gg=$
 $(\lambda \text{out. return } ((\text{prefix-out}, \text{prefix-state}), \text{out}))))$

lemma *checked-staged-security-experiment-with-query-prefix-data-state-projection:*

assumes *i-bound*: $i < \text{rounds}$

shows

checked-staged-security-experiment-with-query-prefix-data-state $A\ i \gg=$
 $(\lambda \text{packed. return } (\text{snd } \text{packed})) =$
checked-staged-security-experiment-with-data-state A

unfolding

checked-staged-security-experiment-with-query-prefix-data-state-def
checked-staged-security-experiment-with-data-state-query-prefix-receive-decomp
 $[OF\ i\text{-bound}]$

by (*simp add: sm-bind-assoc get-bind-const*)

lemma *checked-staged-security-experiment-with-query-prefix-data-state-projection-event:*

assumes *i-bound*: $i < \text{rounds}$

shows

wp-event (*checked-staged-security-experiment-with-data-state* A) E
adversary-initial-state =
wp-event
 $(\text{checked-staged-security-experiment-with-query-prefix-data-state } A\ i)$
 $(\lambda \text{out. case out of}$
 $\text{None } \Rightarrow E\ \text{None}$
 $| \text{Some } (\text{packed}, t) \Rightarrow E\ (\text{Some } (\text{snd } \text{packed}, t)))$

```

      adversary-initial-state
proof –
  have
    wp-event (checked-staged-security-experiment-with-data-state A) E
      adversary-initial-state =
    wp-event
      (checked-staged-security-experiment-with-query-prefix-data-state A i  $\gg$ 
        ( $\lambda$ packed. return (snd packed)))
      E adversary-initial-state
  using
    checked-staged-security-experiment-with-query-prefix-data-state-projection
      [OF i-bound, of A]
  by simp
also have ... =
  wp-event
    (checked-staged-security-experiment-with-query-prefix-data-state A i)
    ( $\lambda$ out. case out of
      None  $\Rightarrow$  E None
    | Some (packed, t)  $\Rightarrow$  E (Some (snd packed, t)))
    adversary-initial-state
  by (rule wp-event-bind-return-map)
finally show ?thesis .
qed

```

lemma checked-staged-security-experiment-with-query-prefix-data-state-imp-data-state-support:

assumes i-bound: $i < \text{rounds}$

and outcome:

Some (((((prefix, prefix-state), raw), raw-state),
 ((data, attacker-state), result)), final-state)
 $\in$
 set-dist
 (execute
 (checked-staged-security-experiment-with-query-prefix-data-state A i)
 adversary-initial-state)

shows

Some (((data, attacker-state), result), final-state) $\in$
 set-dist
 (execute (checked-staged-security-experiment-with-data-state A)
 adversary-initial-state)

proof –

from outcome **obtain** t **where** prefix-receive:

Some (((prefix, prefix-state), raw), t) $\in$
 set-dist
 (execute (checked-staged-query-prefix-receive-with-state A i)
 adversary-initial-state)

and t-eq: $t = \text{raw-state}$

and cont:

Some (((data, attacker-state), result), final-state) $\in$
 set-dist

```

      (execute
        (checked-staged-after-query-prefix-receive-with-verifier A i
          ((prefix, prefix-state), raw))
        raw-state)
    unfolding checked-staged-security-experiment-with-query-prefix-data-state-def
  by (auto elim!: set-dist-bindE split: prod.splits)
have data-out:
  Some (data, attacker-state) ∈
    set-dist
      (execute
        (checked-staged-after-query-prefix-receive A i
          ((prefix, prefix-state), raw))
        raw-state)
  by (rule checked-staged-after-query-prefix-receive-with-verifier-outcome(1)
    [OF cont])
have verifier:
  Some (result, final-state) ∈
    set-dist
      (execute verify-monad
        (verifier-state-from-adversary attacker-state
          (staged-proof-transcript data)))
  by (rule checked-staged-after-query-prefix-receive-with-verifier-outcome(2)
    [OF cont])
have builder:
  Some (data, attacker-state) ∈
    set-dist
      (execute (checked-staged-transcript-program A)
        adversary-initial-state)
  unfolding checked-staged-transcript-program-query-prefix-receive-decomp
    [OF i-bound]
  apply (rule-tac x=((prefix, prefix-state), raw) and
    t=raw-state in set-dist-bindI)
  apply (rule prefix-receive[unfolded t-eq])
  apply (rule data-out)
  done
show ?thesis
  by (rule checked-staged-security-experiment-with-data-state-outcomeI
    [OF builder verifier])
qed

```

lemma *checked-staged-security-experiment-with-query-prefix-data-state-bound-by-prefix:*
assumes *prefix-bound:*
 wp-event (checked-staged-query-prefix-receive-with-state A i) P
 adversary-initial-state ≤ B
and *none-imp:* *Q None ⇒ P None*
and *cont-imp:*
 $\bigwedge \text{prefix-out } t \text{ out.}$
 Some (prefix-out, t) ∈
 set-dist

$(\text{execute } (\text{checked-staged-query-prefix-receive-with-state } A \ i) \text{ adversary-initial-state}) \implies$
 $\text{out} \in$
 set-dist
 $(\text{execute}$
 $(\text{get} \gg=$
 $(\lambda \text{prefix-state.}$
 $\text{checked-staged-after-query-prefix-receive-with-verifier } A \ i$
 $\text{prefix-out} \gg=$
 $(\lambda \text{result. return } ((\text{prefix-out}, \text{prefix-state}), \text{result}))))$
 $t) \implies$
 $Q \ \text{out} \implies P \ (\text{Some } (\text{prefix-out}, t))$
shows
 wp-event
 $(\text{checked-staged-security-experiment-with-query-prefix-data-state } A \ i)$
 $Q \ \text{adversary-initial-state} \leq B$
unfolding $\text{checked-staged-security-experiment-with-query-prefix-data-state-def}$
by $(\text{rule wp-event-bind-bound-by-head-event}$
 $[\text{OF prefix-bound none-imp cont-imp}])$

definition $\text{checked-staged-security-with-query-prefix-dynamic-index-hit}$

where

$\text{checked-staged-security-with-query-prefix-dynamic-index-hit } T \ \text{out} \longleftrightarrow$
 $(\text{case out of}$
 $\text{None} \Rightarrow \text{False}$
 $| \text{Some } (((((\text{prefix-pack}, \text{raw}), \text{raw-state}), \text{result}), \text{final-state})) \Rightarrow$
 $\text{checked-staged-query-prefix-dynamic-index-hit } T$
 $(\text{Some } ((\text{prefix-pack}, \text{raw}), \text{raw-state})))$

definition $\text{checked-staged-security-with-query-prefix-candidate-pair-from-prefix-hit}$

where

$\text{checked-staged-security-with-query-prefix-candidate-pair-from-prefix-hit}$
 $\text{trace-table composition-table out} \longleftrightarrow$
 $\text{checked-staged-security-with-query-prefix-dynamic-index-hit}$
 $(\text{staged-query-prefix-candidate-pair-query-target-from-prefix}$
 $\text{trace-table composition-table}) \ \text{out}$

lemma $\text{checked-staged-security-with-query-prefix-dynamic-index-hit-bound:}$

assumes prefix-bound:

$\text{wp-event } (\text{checked-staged-query-prefix-receive-with-state } A \ i)$
 $(\text{checked-staged-query-prefix-dynamic-index-hit } T)$
 $\text{adversary-initial-state} \leq B$

shows

wp-event
 $(\text{checked-staged-security-experiment-with-query-prefix-data-state } A \ i)$
 $(\text{checked-staged-security-with-query-prefix-dynamic-index-hit } T)$
 $\text{adversary-initial-state} \leq B$

proof $(\text{rule}$

$\text{checked-staged-security-experiment-with-query-prefix-data-state-bound-by-prefix}$

```

    [OF prefix-bound])
  show
    checked-staged-security-with-query-prefix-dynamic-index-hit T None  $\implies$ 
      checked-staged-query-prefix-dynamic-index-hit T None
  unfolding checked-staged-security-with-query-prefix-dynamic-index-hit-def
  by simp
next
  fix prefix-out t out
  assume head:
    Some (prefix-out, t)  $\in$ 
      set-dist
        (execute (checked-staged-query-prefix-receive-with-state A i)
          adversary-initial-state)
  and cont:
    out  $\in$ 
      set-dist
        (execute
          (get  $\gg$ 
            ( $\lambda$ prefix-state.
              checked-staged-after-query-prefix-receive-with-verifier A i
                prefix-out  $\gg$ 
                ( $\lambda$ result. return ((prefix-out, prefix-state), result))))
          t)
  assume hit:
    checked-staged-security-with-query-prefix-dynamic-index-hit T out
  show
    checked-staged-query-prefix-dynamic-index-hit T
      (Some (prefix-out, t))
  using cont hit
  unfolding checked-staged-security-with-query-prefix-dynamic-index-hit-def
  by (auto elim!: set-dist-bindE split: option.splits prod.splits)
qed

```

```

lemma checked-staged-query-prefix-dynamic-index-hit-bound-from-fraction:
  assumes wf: staged-budget-wellformed budgets
  and controlled: staged-adversary-controlled budgets A
  and i-bound:  $i < \text{rounds}$ 
  and target-bound:
     $\bigwedge \text{prefix prefix-state.}$ 
    Some ((prefix, prefix-state), prefix-state)  $\in$ 
      set-dist
        (execute (checked-staged-query-prefix-with-state A i)
          adversary-initial-state)  $\implies$ 
    T prefix prefix-state  $\subseteq$  query-sample-space  $\wedge$ 
    nnreal (card (T prefix prefix-state)) /
      nnreal (card query-sample-space)  $\leq C$ 
  shows
    wp-event (checked-staged-query-prefix-receive-with-state A i)
      (checked-staged-query-prefix-dynamic-index-hit T)

```


$adversary-initial-state \leq$
 $staged-phase-relation-error\ size$
 $(staged-query-search-queries\ budgets\ i + 1) + C$

proof –

have *raw-bound*: *query-index-raw-preimage-bound*
by (rule *query-index-raw-preimage-bound-from-sampler-wellformed*)

have *hit-bound*:
 $wp-event\ (checked-staged-query-prefix-receive-with-state\ A\ i)$
 $(checked-staged-query-prefix-dynamic-index-hit\ T)$
 $adversary-initial-state \leq$
 $wp-event\ (checked-staged-query-prefix-receive-with-state\ A\ i)$
 $(checked-staged-query-prefix-dynamic-index-prehit\ T)$
 $adversary-initial-state + C$
by (rule
 $checked-staged-query-prefix-dynamic-index-hit-bound-by-prehit-and-query-error$
 $[OF\ raw-bound\ target-bound])$)

have *prehit-bound*:
 $wp-event\ (checked-staged-query-prefix-receive-with-state\ A\ i)$
 $(checked-staged-query-prefix-dynamic-index-prehit\ T)$
 $adversary-initial-state \leq$
 $staged-phase-relation-error\ size$
 $(staged-query-search-queries\ budgets\ i + 1)$

proof (rule *checked-staged-query-prefix-dynamic-index-prehit-bound-from-relation-program*)

show $HashMap\ adversary-initial-state = fmempty$
by (*simp* *add*: *adversary-initial-state-def*)

show *hash-relation-program*
 $(checked-staged-query-prefix-dynamic-index-prehit-relation\ A\ i)$
 $adversary-initial-state\ T)$
 $size\ (staged-query-search-queries\ budgets\ i + 1)$
 $(checked-staged-query-prefix-receive-with-state\ A\ i)$
by (rule *hash-relation-program-checked-staged-query-prefix-receive-with-state*
 $[OF\ wf\ controlled])$
 $(use\ i-bound\ in\ simp-all,$
 $rule\ checked-staged-query-prefix-dynamic-index-prehit-relation-fiber-bound-size)$

qed

show *?thesis*
by (rule *order-trans* $[OF\ hit-bound])$
 $(intro\ add-mono\ prehit-bound\ order-refl)$

qed

lemma *checked-staged-security-with-query-prefix-dynamic-index-hit-bound-from-fraction*:
assumes *wf*: *staged-budget-wellformed* *budgets*
and *controlled*: *staged-adversary-controlled* *budgets* *A*
and *i-bound*: $i < rounds$
and *target-bound*:
 $\bigwedge prefix\ prefix-state.$
 $Some\ ((prefix,\ prefix-state),\ prefix-state) \in$
 $set-dist$
 $(execute\ (checked-staged-query-prefix-with-state\ A\ i))$

$\text{adversary-initial-state}) \implies$
 $T \text{ prefix prefix-state} \subseteq \text{query-sample-space} \wedge$
 $\text{nnreal} (\text{card} (T \text{ prefix prefix-state})) /$
 $\text{nnreal} (\text{card} \text{ query-sample-space}) \leq C$
shows
wp-event
 $(\text{checked-staged-security-experiment-with-query-prefix-data-state } A \ i)$
 $(\text{checked-staged-security-with-query-prefix-dynamic-index-hit } T)$
 $\text{adversary-initial-state} \leq$
 $\text{staged-phase-relation-error size}$
 $(\text{staged-query-search-queries budgets } i + 1) + C$
by $(\text{rule checked-staged-security-with-query-prefix-dynamic-index-hit-bound},$
 $\text{rule checked-staged-query-prefix-dynamic-index-hit-bound-from-fraction}$
 $[\text{OF wf controlled i-bound target-bound}])$

lemma *checked-staged-security-with-query-prefix-candidate-opening-target-from-prefix-hit-bound:*

assumes *raw-bound: query-index-raw-preimage-bound*
and *wf: staged-budget-wellformed budgets*
and *controlled: staged-adversary-controlled budgets A*
and *i-bound: $i \leq \text{rounds}$*
and *trace-candidate:*
 $\text{partial-trace-table-candidate trace-table trace-openings}$
and *comp-candidate:*
 $\text{partial-composition-table-candidate composition-table}$
 $\text{composition-openings}$
and *trace-low: trace-table-low-degree trace-table*
and *comp-low:*
 $\text{composition-table-low-degree maxDegree composition-table}$
and *not-all:*
 $\bigwedge \text{prefix prefix-state.}$
 $\text{Some } ((\text{prefix}, \text{prefix-state}), \text{prefix-state}) \in$
 set-dist
 $(\text{execute } (\text{checked-staged-query-prefix-with-state } A \ i)$
 $\text{adversary-initial-state}) \implies$
 $\neg \text{all-queries-consistent trace-table composition-table}$
 $(\text{sqp-alphas prefix})$

shows

wp-event
 $(\text{checked-staged-security-experiment-with-query-prefix-data-state } A \ i)$
 $(\text{checked-staged-security-with-query-prefix-dynamic-index-hit}$
 $(\text{staged-query-prefix-candidate-opening-query-target-from-prefix}$
 $\text{trace-openings composition-openings } i))$
 $\text{adversary-initial-state} \leq$
 $\text{staged-phase-relation-error size}$
 $(\text{staged-query-search-queries budgets } i + 1) + \text{query-error-bound}$
by $(\text{rule checked-staged-security-with-query-prefix-dynamic-index-hit-bound},$
 $\text{rule checked-staged-query-prefix-candidate-opening-target-from-prefix-hit-bound}$
 $[\text{OF raw-bound wf controlled i-bound trace-candidate comp-candidate}$
 $\text{trace-low comp-low not-all}])$

lemma *checked-staged-security-with-query-prefix-candidate-opening-target-from-prefix-hit-bound-from-budgets:*

assumes *wf: staged-budget-wellformed budgets*

and *controlled: staged-adversary-controlled budgets A*

and *i-bound: $i < \text{rounds}$*

and *trace-candidate:*

partial-trace-table-candidate trace-table trace-openings

and *comp-candidate:*

partial-composition-table-candidate composition-table

composition-openings

and *trace-low: trace-table-low-degree trace-table*

and *comp-low:*

composition-table-low-degree maxDegree composition-table

and *not-all:*

$\bigwedge \text{prefix prefix-state.}$

Some ((prefix, prefix-state), prefix-state) $\in$

set-dist

(execute (checked-staged-query-prefix-with-state A i)

adversary-initial-state) $\implies$

$\neg$ *all-queries-consistent trace-table composition-table*

(sqp-alphas prefix)

shows

wp-event

(checked-staged-security-experiment-with-query-prefix-data-state A i)

(checked-staged-security-with-query-prefix-dynamic-index-hit

(staged-query-prefix-candidate-opening-query-target-from-prefix

trace-openings composition-openings i))

adversary-initial-state $\leq$

staged-phase-relation-error size

(staged-query-search-queries budgets i + 1) + query-error-bound

proof –

have *raw-bound: query-index-raw-preimage-bound*

by *(rule query-index-raw-preimage-bound-from-sampler-wellformed)*

show *?thesis*

by *(rule*

checked-staged-security-with-query-prefix-candidate-opening-target-from-prefix-hit-bound

[OF raw-bound wf controlled - trace-candidate comp-candidate trace-low

comp-low not-all])

(use i-bound in simp)

qed

lemma *checked-staged-security-with-query-prefix-candidate-pair-target-from-prefix-hit-bound:*

assumes *raw-bound: query-index-raw-preimage-bound*

and *wf: staged-budget-wellformed budgets*

and *controlled: staged-adversary-controlled budgets A*

and *i-bound: $i \leq \text{rounds}$*

and *trace-low: trace-table-low-degree trace-table*

and *comp-low:*

composition-table-low-degree maxDegree composition-table

and not-all:
 $\bigwedge \text{prefix prefix-state.}$
 $\text{Some } ((\text{prefix}, \text{prefix-state}), \text{prefix-state}) \in$
 set-dist
 $(\text{execute } (\text{checked-staged-query-prefix-with-state } A \ i)$
 $\text{adversary-initial-state}) \implies$
 $\neg \text{all-queries-consistent trace-table composition-table}$
 $(\text{sqp-alphas prefix})$
shows
 wp-event
 $(\text{checked-staged-security-experiment-with-query-prefix-data-state } A \ i)$
 $(\text{checked-staged-security-with-query-prefix-dynamic-index-hit}$
 $(\text{staged-query-prefix-candidate-pair-query-target-from-prefix}$
 $\text{trace-table composition-table}))$
 $\text{adversary-initial-state} \leq$
 $\text{staged-phase-relation-error size}$
 $(\text{staged-query-search-queries budgets } i + 1) + \text{query-error-bound}$
by (rule checked-staged-security-with-query-prefix-dynamic-index-hit-bound,
rule checked-staged-query-prefix-candidate-pair-target-from-prefix-hit-bound
[OF raw-bound wf controlled i-bound trace-low comp-low not-all])

lemma checked-staged-security-with-query-prefix-candidate-pair-target-from-prefix-hit-bound-from-budgets:

assumes wf: staged-budget-wellformed budgets
and controlled: staged-adversary-controlled budgets A
and i-bound: $i < \text{rounds}$
and trace-low: trace-table-low-degree trace-table
and comp-low:
 $\text{composition-table-low-degree maxDegree composition-table}$
and not-all:
 $\bigwedge \text{prefix prefix-state.}$
 $\text{Some } ((\text{prefix}, \text{prefix-state}), \text{prefix-state}) \in$
 set-dist
 $(\text{execute } (\text{checked-staged-query-prefix-with-state } A \ i)$
 $\text{adversary-initial-state}) \implies$
 $\neg \text{all-queries-consistent trace-table composition-table}$
 $(\text{sqp-alphas prefix})$

shows

wp-event
 $(\text{checked-staged-security-experiment-with-query-prefix-data-state } A \ i)$
 $(\text{checked-staged-security-with-query-prefix-dynamic-index-hit}$
 $(\text{staged-query-prefix-candidate-pair-query-target-from-prefix}$
 $\text{trace-table composition-table}))$
 $\text{adversary-initial-state} \leq$
 $\text{staged-phase-relation-error size}$
 $(\text{staged-query-search-queries budgets } i + 1) + \text{query-error-bound}$

proof –

have raw-bound: query-index-raw-preimage-bound
by (rule query-index-raw-preimage-bound-from-sampler-wellformed)
show ?thesis

by (rule
 checked-staged-security-with-query-prefix-candidate-pair-target-from-prefix-hit-bound
 [*OF raw-bound wf controlled - trace-low comp-low not-all*])
 (use *i-bound* in *simp*)
 qed

lemma *checked-staged-security-with-query-prefix-candidate-pair-from-prefix-hit-bound-from-budgets*:
 assumes *wf*: *staged-budget-wellformed budgets*
 and *controlled*: *staged-adversary-controlled budgets A*
 and *i-bound*: *i < rounds*
 and *trace-low*: *trace-table-low-degree trace-table*
 and *comp-low*:
 composition-table-low-degree maxDegree composition-table
 and *not-all*:
 $\bigwedge \text{prefix prefix-state.}$
 Some ((prefix, prefix-state), prefix-state) ∈
 set-dist
 (*execute (checked-staged-query-prefix-with-state A i)*
 adversary-initial-state) $\implies$
 $\neg$ *all-queries-consistent trace-table composition-table*
 (*sqp-alphas prefix*)
 shows
 wp-event
 (*checked-staged-security-experiment-with-query-prefix-data-state A i*)
 (*checked-staged-security-with-query-prefix-candidate-pair-from-prefix-hit*
 trace-table composition-table)
 adversary-initial-state $\leq$
 staged-phase-relation-error size
 (*staged-query-search-queries budgets i + 1*) + *query-error-bound*
 unfolding
 checked-staged-security-with-query-prefix-candidate-pair-from-prefix-hit-def
 by (rule
 checked-staged-security-with-query-prefix-candidate-pair-target-from-prefix-hit-bound-from-budgets
 [*OF wf controlled i-bound trace-low comp-low not-all*])

lemma *powers-scaled-eq-imp-eq*:
 assumes *powers-scaled idx = powers-scaled idx'*
 shows *idx = idx'*
 proof –
 have *left: powers-scaled idx ! 0 = idx*
 unfolding *powers-scaled-def* using *powers-pos* by *simp*
 have *right: powers-scaled idx' ! 0 = idx'*
 unfolding *powers-scaled-def* using *powers-pos* by *simp*
 show ?thesis
 using *assms left right* by *metis*
 qed

lemma *partial-query-success-indices-at-card-le-one*:
card (partial-query-success-indices-at trace-openings

```

      composition-openings as i) ≤ Suc 0
proof –
  have finite-target:
    finite (partial-query-success-indices-at trace-openings
      composition-openings as i)
  by (rule finite-subset[OF partial-query-success-indices-at-subset
    finite-query-sample-space])
  have pairwise:
    ∀ idx ∈ partial-query-success-indices-at trace-openings
      composition-openings as i.
    ∀ idx' ∈ partial-query-success-indices-at trace-openings
      composition-openings as i.
      idx = idx'
  proof (intro ballI)
    fix idx idx'
    assume idx-in:
      idx ∈ partial-query-success-indices-at trace-openings
        composition-openings as i
    and idx'-in:
      idx' ∈ partial-query-success-indices-at trace-openings
        composition-openings as i
    have idx-indices:
      map opening-index (trace-openings ! i) = powers-scaled idx
    using idx-in
    unfolding partial-query-success-indices-at-def
      partial-query-round-consistent-def
    by simp
    have idx'-indices:
      map opening-index (trace-openings ! i) = powers-scaled idx'
    using idx'-in
    unfolding partial-query-success-indices-at-def
      partial-query-round-consistent-def
    by simp
    have powers-scaled idx = powers-scaled idx'
    using idx-indices idx'-indices by simp
    then show idx = idx'
    by (rule powers-scaled-eq-imp-eq)
  qed
  show ?thesis
  using card-le-Suc0-iff-eq[OF finite-target] pairwise by simp
qed

```

```

lemma prob-divide-right-mono:
  assumes (a::prob) ≤ b
  shows a / c ≤ b / c
  using assms
  apply transfer
  by (simp add: divide-right-mono)

```

lemma *partial-query-success-indices-at-singleton-fraction-bound:*

```

nnreal
  (card
    (partial-query-success-indices-at trace-openings composition-openings
      as i)) /
nnreal (card query-sample-space) ≤
(1::prob) / nnreal (card query-sample-space)
proof –
have card-le:
  nnreal
    (card
      (partial-query-success-indices-at trace-openings composition-openings
        as i)) ≤ (1::prob)
using partial-query-success-indices-at-card-le-one
by simp
show ?thesis
by (rule prob-divide-right-mono[OF card-le])
qed

```

lemma *staged-query-prefix-candidate-opening-query-target-from-prefix-singleton-fraction-bound:*

```

nnreal
  (card
    (staged-query-prefix-candidate-opening-query-target-from-prefix
      trace-openings composition-openings i prefix prefix-state)) /
nnreal (card query-sample-space) ≤
(1::prob) / nnreal (card query-sample-space)
unfolding staged-query-prefix-candidate-opening-query-target-from-prefix-def
by (rule partial-query-success-indices-at-singleton-fraction-bound)

```

lemma *checked-staged-query-prefix-candidate-opening-target-from-prefix-singleton-hit-bound:*

```

assumes raw-bound: query-index-raw-preimage-bound
and wf: staged-budget-wellformed budgets
and controlled: staged-adversary-controlled budgets A
and i-bound: i ≤ rounds
shows
  wp-event (checked-staged-query-prefix-receive-with-state A i)
    (checked-staged-query-prefix-dynamic-index-hit
      (staged-query-prefix-candidate-opening-query-target-from-prefix
        trace-openings composition-openings i))
  adversary-initial-state ≤
  staged-phase-relation-error size
    (staged-query-search-queries budgets i + 1) +
  (1::prob) / nnreal (card query-sample-space)

```

proof –

```

have hit-bound:
  wp-event (checked-staged-query-prefix-receive-with-state A i)
    (checked-staged-query-prefix-dynamic-index-hit
      (staged-query-prefix-candidate-opening-query-target-from-prefix
        trace-openings composition-openings i))

```

```

    adversary-initial-state ≤
    wp-event (checked-staged-query-prefix-receive-with-state A i)
    (checked-staged-query-prefix-dynamic-index-prehit
    (staged-query-prefix-candidate-opening-query-target-from-prefix
    trace-openings composition-openings i))
    adversary-initial-state +
    (1::prob) / nnreal (card query-sample-space)
proof (rule checked-staged-query-prefix-dynamic-index-hit-bound-by-prehit-and-query-error
    [OF raw-bound])
fix prefix prefix-state
assume
    Some ((prefix, prefix-state), prefix-state) ∈
    set-dist
    (execute (checked-staged-query-prefix-with-state A i)
    adversary-initial-state)
show
    staged-query-prefix-candidate-opening-query-target-from-prefix
    trace-openings composition-openings i prefix prefix-state ⊆
    query-sample-space ∧
    nnreal
    (card
    (staged-query-prefix-candidate-opening-query-target-from-prefix
    trace-openings composition-openings i prefix prefix-state)) /
    nnreal (card query-sample-space)
    ≤ (1::prob) / nnreal (card query-sample-space)
using
    staged-query-prefix-candidate-opening-query-target-from-prefix-subset
    staged-query-prefix-candidate-opening-query-target-from-prefix-singleton-fraction-bound
by blast
qed
have prehit-bound:
    wp-event (checked-staged-query-prefix-receive-with-state A i)
    (checked-staged-query-prefix-dynamic-index-prehit
    (staged-query-prefix-candidate-opening-query-target-from-prefix
    trace-openings composition-openings i))
    adversary-initial-state ≤
    staged-phase-relation-error size
    (staged-query-search-queries budgets i + 1)
by (rule checked-staged-query-prefix-candidate-opening-target-from-prefix-prehit-bound
    [OF wf controlled i-bound])
have tail:
    wp-event (checked-staged-query-prefix-receive-with-state A i)
    (checked-staged-query-prefix-dynamic-index-prehit
    (staged-query-prefix-candidate-opening-query-target-from-prefix
    trace-openings composition-openings i))
    adversary-initial-state +
    (1::prob) / nnreal (card query-sample-space) ≤
    staged-phase-relation-error size
    (staged-query-search-queries budgets i + 1) +

```



```

      (1::prob) / nnreal (card query-sample-space)
    by (intro add-mono prehit-bound order-refl)
  show ?thesis
    by (rule order-trans[OF hit-bound tail])
qed

```

lemma *checked-staged-security-with-query-prefix-candidate-opening-target-from-prefix-singleton-hit-bound:*
assumes *raw-bound: query-index-raw-preimage-bound*
and *wf: staged-budget-wellformed budgets*
and *controlled: staged-adversary-controlled budgets A*
and *i-bound: $i \leq \text{rounds}$*
shows
wp-event
 (*checked-staged-security-experiment-with-query-prefix-data-state A i*)
 (*checked-staged-security-with-query-prefix-dynamic-index-hit*
 (*staged-query-prefix-candidate-opening-query-target-from-prefix*
 trace-openings composition-openings i))
adversary-initial-state $\leq$
staged-phase-relation-error size
 (*staged-query-search-queries budgets i + 1*) +
 (1::prob) / nnreal (card query-sample-space)
by (rule *checked-staged-security-with-query-prefix-dynamic-index-hit-bound*,
 rule
 checked-staged-query-prefix-candidate-opening-target-from-prefix-singleton-hit-bound
 [OF *raw-bound wf controlled i-bound*])

lemma *checked-staged-security-with-query-prefix-candidate-opening-target-from-prefix-singleton-hit-bound-from-budget:*
assumes *wf: staged-budget-wellformed budgets*
and *controlled: staged-adversary-controlled budgets A*
and *i-bound: $i < \text{rounds}$*
shows
wp-event
 (*checked-staged-security-experiment-with-query-prefix-data-state A i*)
 (*checked-staged-security-with-query-prefix-dynamic-index-hit*
 (*staged-query-prefix-candidate-opening-query-target-from-prefix*
 trace-openings composition-openings i))
adversary-initial-state $\leq$
staged-phase-relation-error size
 (*staged-query-search-queries budgets i + 1*) +
 (1::prob) / nnreal (card query-sample-space)
proof –
have *raw-bound: query-index-raw-preimage-bound*
by (rule *query-index-raw-preimage-bound-from-sampler-wellformed*)
show ?thesis
by (rule
 checked-staged-security-with-query-prefix-candidate-opening-target-from-prefix-singleton-hit-bound
 [OF *raw-bound wf controlled*])
 (use *i-bound* **in** *simp*)
qed

definition *checked-staged-security-with-actual-query-prefix-candidate-opening-hit*
where

checked-staged-security-with-actual-query-prefix-candidate-opening-hit
 $\text{trace-openings } \text{composition-openings } i \text{ out} \longleftrightarrow$
checked-staged-security-with-query-prefix-dynamic-index-hit
 $(\text{staged-query-prefix-candidate-opening-query-target-from-prefix}$
 $\text{trace-openings } \text{composition-openings } i) \text{ out}$

lemma *checked-staged-security-with-actual-query-prefix-candidate-opening-hitE*:

assumes *hit*:

checked-staged-security-with-actual-query-prefix-candidate-opening-hit
 $\text{trace-openings } \text{composition-openings } i$
 $(\text{Some } ((((\text{prefix}, \text{prefix-state}), \text{raw}), \text{raw-state}),$
 $((\text{data}, \text{attacker-state}), \text{result})), \text{final-state}))$

obtains

$\text{index } (\text{to-nat } \text{raw}) \in$
 $\text{staged-query-prefix-candidate-opening-query-target-from-prefix}$
 $\text{trace-openings } \text{composition-openings } i \text{ prefix } \text{prefix-state}$

using *hit*

unfolding

checked-staged-security-with-actual-query-prefix-candidate-opening-hit-def
checked-staged-security-with-query-prefix-dynamic-index-hit-def
checked-staged-query-prefix-dynamic-index-hit-def

by *simp*

lemma *checked-staged-security-with-actual-query-prefix-candidate-opening-hit-bound*:

assumes *raw-bound*: *query-index-raw-preimage-bound*

and *wf*: *staged-budget-wellformed budgets*

and *controlled*: *staged-adversary-controlled budgets A*

and *i-bound*: $i \leq \text{rounds}$

shows

wp-event
 $(\text{checked-staged-security-experiment-with-query-prefix-data-state } A \ i)$
 $(\text{checked-staged-security-with-actual-query-prefix-candidate-opening-hit}$
 $\text{trace-openings } \text{composition-openings } i)$
 $\text{adversary-initial-state} \leq$
 $\text{staged-phase-relation-error size}$
 $(\text{staged-query-search-queries budgets } i + 1) +$
 $(1::\text{prob}) / \text{nnreal } (\text{card } \text{query-sample-space})$

unfolding

checked-staged-security-with-actual-query-prefix-candidate-opening-hit-def

by *(rule*

checked-staged-security-with-query-prefix-candidate-opening-target-from-prefix-singleton-hit-bound
 $[\text{OF } \text{raw-bound } \text{wf } \text{controlled } i\text{-bound}])$

lemma *checked-staged-security-with-actual-query-prefix-candidate-opening-hit-bound-from-budgets*:

assumes *wf*: *staged-budget-wellformed budgets*

and *controlled*: *staged-adversary-controlled budgets A*

and *i-bound*: $i < \text{rounds}$
shows
wp-event
 (checked-staged-security-experiment-with-query-prefix-data-state $A\ i$)
 (checked-staged-security-with-actual-query-prefix-candidate-opening-hit
 trace-openings composition-openings i)
adversary-initial-state $\leq$
staged-phase-relation-error size
 (staged-query-search-queries *budgets i + 1*) +
 (1::prob) / nnreal (card *query-sample-space*)
unfolding
checked-staged-security-with-actual-query-prefix-candidate-opening-hit-def
by (rule
checked-staged-security-with-query-prefix-candidate-opening-target-from-prefix-singleton-hit-bound-from-budgets
 [OF *wf controlled i-bound*])

lemma *checked-staged-security-with-data-state-bound-by-actual-query-prefix-candidate-opening-hit*:

assumes *raw-bound*: *query-index-raw-preimage-bound*
and *wf*: *staged-budget-wellformed budgets*
and *controlled*: *staged-adversary-controlled budgets A*
and *i-bound*: $i < \text{rounds}$
and *event-imp*:
 $\bigwedge \text{out.}$
 (case out of
 None $\Rightarrow E$ None
 | Some (packed, t) $\Rightarrow E$ (Some (snd packed, t))) $\Rightarrow$
checked-staged-security-with-actual-query-prefix-candidate-opening-hit
trace-openings composition-openings i out

shows

wp-event (checked-staged-security-experiment-with-data-state A) E
adversary-initial-state $\leq$
staged-phase-relation-error size
 (staged-query-search-queries *budgets i + 1*) +
 (1::prob) / nnreal (card *query-sample-space*)

proof –

have *projection*:
wp-event (checked-staged-security-experiment-with-data-state A) E
adversary-initial-state =
wp-event
 (checked-staged-security-experiment-with-query-prefix-data-state $A\ i$)
 ($\lambda \text{out. case out of}$
 None $\Rightarrow E$ None
 | Some (packed, t) $\Rightarrow E$ (Some (snd packed, t)))
adversary-initial-state
by (rule
checked-staged-security-experiment-with-query-prefix-data-state-projection-event
 [OF *i-bound*])
have *lifted-bound*:
wp-event

```

    (checked-staged-security-experiment-with-query-prefix-data-state A i)
  (λout. case out of
    None ⇒ E None
  | Some (packed, t) ⇒ E (Some (snd packed, t)))
  adversary-initial-state ≤
  wp-event
    (checked-staged-security-experiment-with-query-prefix-data-state A i)
    (checked-staged-security-with-actual-query-prefix-candidate-opening-hit
      trace-openings composition-openings i)
    adversary-initial-state
  by (rule wp-event-mono) (use event-imp in blast)
have fixed-bound:
  wp-event
    (checked-staged-security-experiment-with-query-prefix-data-state A i)
    (checked-staged-security-with-actual-query-prefix-candidate-opening-hit
      trace-openings composition-openings i)
    adversary-initial-state ≤
    staged-phase-relation-error size
      (staged-query-search-queries budgets i + 1) +
      (1::prob) / nnreal (card query-sample-space)
  by (rule
    checked-staged-security-with-actual-query-prefix-candidate-opening-hit-bound
      [OF raw-bound wf controlled])
    (use i-bound in simp)
show ?thesis
  unfolding projection
  by (rule order-trans[OF lifted-bound fixed-bound])
qed

```

lemma *checked-staged-security-with-data-state-bound-by-actual-query-prefix-candidate-opening-hit-from-budgets:*

assumes wf: staged-budget-wellformed budgets

and controlled: staged-adversary-controlled budgets A

and i-bound: $i < \text{rounds}$

and event-imp:

$\bigwedge \text{out.}$

(case out of

None ⇒ E None

| Some (packed, t) ⇒ E (Some (snd packed, t))) ⇒

checked-staged-security-with-actual-query-prefix-candidate-opening-hit

trace-openings composition-openings i out

shows

wp-event (checked-staged-security-experiment-with-data-state A) E

adversary-initial-state ≤

staged-phase-relation-error size

(staged-query-search-queries budgets i + 1) +

(1::prob) / nnreal (card query-sample-space)

proof –

have raw-bound: query-index-raw-preimage-bound

by (rule query-index-raw-preimage-bound-from-sampler-wellformed)

```

show ?thesis
  by (rule
    checked-staged-security-with-data-state-bound-by-actual-query-prefix-candidate-opening-hit
    [OF raw-bound wf controlled i-bound event-imp])
qed

lemma checked-staged-security-with-query-prefix-data-state-bound-by-actual-query-prefix-candidate-opening-hit:
assumes raw-bound: query-index-raw-preimage-bound
and wf: staged-budget-wellformed budgets
and controlled: staged-adversary-controlled budgets A
and i-bound:  $i < \text{rounds}$ 
and event-imp:
   $\bigwedge \text{out. } E \text{ out} \implies$ 
    checked-staged-security-with-actual-query-prefix-candidate-opening-hit
    trace-openings composition-openings i out
shows
  wp-event
    (checked-staged-security-experiment-with-query-prefix-data-state A i)
    E adversary-initial-state  $\leq$ 
    staged-phase-relation-error size
    (staged-query-search-queries budgets  $i + 1$ ) +
    (1::prob) / nnreal (card query-sample-space)
proof -
have event-le:
  wp-event
    (checked-staged-security-experiment-with-query-prefix-data-state A i)
    E adversary-initial-state  $\leq$ 
    wp-event
      (checked-staged-security-experiment-with-query-prefix-data-state A i)
      (checked-staged-security-with-actual-query-prefix-candidate-opening-hit
        trace-openings composition-openings i)
      adversary-initial-state
by (rule wp-event-mono) (rule event-imp)
have fixed-bound:
  wp-event
    (checked-staged-security-experiment-with-query-prefix-data-state A i)
    (checked-staged-security-with-actual-query-prefix-candidate-opening-hit
      trace-openings composition-openings i)
    adversary-initial-state  $\leq$ 
    staged-phase-relation-error size
    (staged-query-search-queries budgets  $i + 1$ ) +
    (1::prob) / nnreal (card query-sample-space)
by (rule
    checked-staged-security-with-actual-query-prefix-candidate-opening-hit-bound
    [OF raw-bound wf controlled])
    (use i-bound in simp)
show ?thesis
by (rule order-trans[OF event-le fixed-bound])
qed

```

lemma *checked-staged-security-with-query-prefix-data-state-bound-by-actual-query-prefix-candidate-opening-hit-from-*
assumes *wf: staged-budget-wellformed budgets*
and *controlled: staged-adversary-controlled budgets A*
and *i-bound: i < rounds*
and *event-imp:*
 $\bigwedge \text{out. } E \text{ out} \implies$
checked-staged-security-with-actual-query-prefix-candidate-opening-hit
trace-openings composition-openings i out
shows
wp-event
(checked-staged-security-experiment-with-query-prefix-data-state A i)
E adversary-initial-state ≤
staged-phase-relation-error size
(staged-query-search-queries budgets i + 1) +
(1::prob) / nnreal (card query-sample-space)
proof –
have *raw-bound: query-index-raw-preimage-bound*
by *(rule query-index-raw-preimage-bound-from-sampler-wellformed)*
show *?thesis*
by *(rule*
checked-staged-security-with-query-prefix-data-state-bound-by-actual-query-prefix-candidate-opening-hit
[OF raw-bound wf controlled i-bound event-imp])
qed

lemma *checked-staged-query-prefix-with-state-alpha-prefix-support:*
assumes *outcome:*
Some ((prefix, prefix-state), prefix-state) ∈
set-dist
(execute (checked-staged-query-prefix-with-state A i)
adversary-initial-state)
obtains *alpha-prefix alpha-prefix-state where*
alpha-prefix =
(sqp-trace-root prefix, sqp-trace-fri-roots prefix,
sqp-trace-fri-challenges prefix, sqp-trace-final prefix)
Some (alpha-prefix, alpha-prefix-state) ∈
set-dist
(execute (staged-alpha-prefix-program A) adversary-initial-state)
proof –
from *outcome obtain prefix-out prefix-state' where prefix-program-out:*
Some (prefix-out, prefix-state') ∈
set-dist
(execute (checked-staged-query-challenge-prefix-program A i)
adversary-initial-state)
and *prefix-eq0: prefix-out = prefix*
unfolding *checked-staged-query-prefix-with-state-def*
by *(auto elim!: set-dist-bindE dest!: get-bind-return-state-outcome)*
obtain *fr s1 s2 trace-roots trace-bs s3 trace-final s4 s5 as*
s6 dg s7 s8 s9 composition-roots composition-bs s10

composition-final s11 query-start query-chunks where
fr-out:
Some (fr, s1) ∈
set-dist (execute (trace-root-stage A) adversary-initial-state)
and *record-fr:*
Some ((), s2) ∈ set-dist (execute (record-staged-message fr) s1)
and *trace-fri:*
Some ((trace-roots, trace-bs), s3) ∈
set-dist
(execute
(staged-trace-fri-program A 0 (ceil-log clength) []) s2)
and *trace-final-out:*
Some (trace-final, s4) ∈
set-dist (execute (trace-final-stage A trace-bs) s3)
and *record-trace-final:*
Some ((), s5) ∈
set-dist (execute (record-staged-message trace-final) s4)
and *alpha-out:*
Some (as, s6) ∈
set-dist (execute (staged-alpha-program (length spec)) s5)
and *degree-out:*
Some (dg, s7) ∈ set-dist (execute (degree-stage A as) s6)
and *record-degree:*
Some ((), s8) ∈
set-dist (execute (record-staged-message dg) s7)
and *assert-degree:*
Some ((), s9) ∈
set-dist
(execute
(assert (ceil-log (to-nat dg + 1) ≤
ceil-log (maxDegree + 1))) s8)
and *comp-fri:*
Some ((composition-roots, composition-bs), s10) ∈
set-dist
(execute
(staged-composition-fri-program A dg 0
(ceil-log (to-nat dg + 1)) []) s9)
and *comp-final:*
Some (composition-final, s11) ∈
set-dist
(execute (composition-final-stage A dg composition-bs) s10)
and *record-comp-final:*
Some ((), query-start) ∈
set-dist (execute (record-staged-message composition-final) s11)
and *query-prefix:*
Some (query-chunks, prefix-state') ∈
set-dist
(execute
(checked-staged-query-program A trace-roots composition-roots

```

    0 i) query-start)
and prefix-out-eq:
  prefix-out =
    (|sqp-trace-root = fr,
     sqp-trace-fri-roots = trace-roots,
     sqp-trace-fri-challenges = trace-bs,
     sqp-trace-final = trace-final,
     sqp-alphas = as,
     sqp-degree = dg,
     sqp-composition-fri-roots = composition-roots,
     sqp-composition-fri-challenges = composition-bs,
     sqp-composition-final = composition-final,
     sqp-query-chunks = query-chunks)
  by (rule checked-staged-query-challenge-prefix-program-outcomeE[
    OF prefix-program-out])
have prefix-eq:
  prefix =
    (|sqp-trace-root = fr,
     sqp-trace-fri-roots = trace-roots,
     sqp-trace-fri-challenges = trace-bs,
     sqp-trace-final = trace-final,
     sqp-alphas = as,
     sqp-degree = dg,
     sqp-composition-fri-roots = composition-roots,
     sqp-composition-fri-challenges = composition-bs,
     sqp-composition-final = composition-final,
     sqp-query-chunks = query-chunks)
  using prefix-eq0 prefix-out-eq by simp
let ?alpha-prefix = (fr, trace-roots, trace-bs, trace-final)
have alpha-prefix-out:
  Some (?alpha-prefix, s5) ∈
    set-dist
    (execute (staged-alpha-prefix-program A) adversary-initial-state)
proof –
  have after-record-final:
    Some (?alpha-prefix, s5) ∈
      set-dist
      (execute
        (record-staged-message trace-final  $\gg$ 
          (λ-. return (fr, trace-roots, trace-bs, trace-final))))
        s4)
  by (rule-tac x=() and t=s5 in set-dist-bindI)
    (rule record-trace-final, simp)
have after-final:
  Some (?alpha-prefix, s5) ∈
    set-dist
    (execute
      (trace-final-stage A trace-bs  $\gg$ 
        (λtrace-final.

```



```

      record-staged-message trace-final >>=
      (λ-. return (fr, trace-roots, trace-bs, trace-final))))
    s3)
  by (rule-tac x=trace-final and t=s4 in set-dist-bindI)
    (rule trace-final-out, rule after-record-final)
have after-trace:
  Some (?alpha-prefix, s5) ∈
    set-dist
    (execute
      (staged-trace-fri-program A 0 (ceil-log clength) [] >>=
        (λ(trace-roots, trace-bs).
          trace-final-stage A trace-bs >>=
            (λtrace-final.
              record-staged-message trace-final >>=
                (λ-. return
                  (fr, trace-roots, trace-bs, trace-final))))))
      s2)
  apply (rule-tac x=(trace-roots, trace-bs) and t=s3
    in set-dist-bindI)
  apply (rule trace-fri)
  apply (simp add: after-final)
done
have after-record-root:
  Some (?alpha-prefix, s5) ∈
    set-dist
    (execute
      (record-staged-message fr >>=
        (λ-. staged-trace-fri-program A 0 (ceil-log clength) [] >>=
          (λ(trace-roots, trace-bs).
            trace-final-stage A trace-bs >>=
              (λtrace-final.
                record-staged-message trace-final >>=
                  (λ-. return
                    (fr, trace-roots, trace-bs, trace-final))))))
      s1)
  by (rule-tac x=() and t=s2 in set-dist-bindI)
    (rule record-fr, rule after-trace)
show ?thesis
  unfolding staged-alpha-prefix-program-def
  apply (rule-tac x=fr and t=s1 in set-dist-bindI)
  apply (rule fr-out)
  apply (rule after-record-root)
done
qed
show ?thesis
  by (rule that[OF - alpha-prefix-out]) (simp add: prefix-eq)
qed

```

lemma *checked-staged-security-with-query-prefix-data-state-alpha-prefix-support:*

assumes *outcome*:

Some (((((*prefix*, *prefix-state*), *raw*), *raw-state*), *result*), *final-state*)
∈
set-dist
(*execute*
(*checked-staged-security-experiment-with-query-prefix-data-state A i*)
adversary-initial-state)

obtains *alpha-prefix alpha-prefix-state* **where**

alpha-prefix =
(*sqp-trace-root prefix*, *sqp-trace-fri-roots prefix*,
sqp-trace-fri-challenges prefix, *sqp-trace-final prefix*)
Some (*alpha-prefix*, *alpha-prefix-state*) ∈
set-dist
(*execute* (*staged-alpha-prefix-program A*) *adversary-initial-state*)

proof –

from *outcome* **obtain** *t* **where** *prefix-receive*:

Some (((*prefix*, *prefix-state*), *raw*), *t*) ∈
set-dist
(*execute* (*checked-staged-query-prefix-receive-with-state A i*)
adversary-initial-state)

and *t-eq*: *t* = *raw-state*

unfolding *checked-staged-security-experiment-with-query-prefix-data-state-def*

by (*auto elim!*: *set-dist-bindE*)

have *prefix-support*:

Some ((*prefix*, *prefix-state*), *prefix-state*) ∈
set-dist
(*execute* (*checked-staged-query-prefix-with-state A i*)
adversary-initial-state)
by (*rule checked-staged-query-prefix-receive-with-state-prefix-support*)
(*use prefix-receive t-eq in simp*)

show *?thesis*

by (*rule*
checked-staged-query-prefix-with-state-alpha-prefix-support
[*OF prefix-support*])
(*rule that*)

qed

lemma *checked-staged-security-with-query-prefix-data-state-alphas-eq*:

assumes *outcome*:

Some (((((*prefix*, *prefix-state*), *raw*), *raw-state*),
((*data*, *attacker-state*), *result*)), *final-state*)
∈
set-dist
(*execute*
(*checked-staged-security-experiment-with-query-prefix-data-state A i*)
adversary-initial-state)

shows *staged-alphas data* = *sqp-alphas prefix*

using *outcome*

unfolding

checked-staged-security-experiment-with-query-prefix-data-state-def
checked-staged-after-query-prefix-receive-with-verifier-def
checked-staged-after-query-prefix-receive-def
by (auto elim!: set-dist-bindE split: prod.splits)

definition *checked-staged-security-with-query-prefix-authenticated-opening-hit*
where

checked-staged-security-with-query-prefix-authenticated-opening-hit
trace-openings composition-openings i out $\longleftrightarrow$
 (case out of
 None $\Rightarrow$ False
 | Some (((((prefix, prefix-state), raw), raw-state),
 ((data, attacker-state), result)), final-state) $\Rightarrow$
 staged-composition-fri-roots data $\neq \square$ $\wedge$
 partial-authenticated-table (staged-trace-root data)
 (scale * clength) trace-openings final-state $\wedge$
 partial-authenticated-table (hd (staged-composition-fri-roots data))
 (scale * clength) composition-openings final-state $\wedge$
 partial-query-openings-consistent trace-openings composition-openings
 (staged-alphas data) (index (to-nat raw)))

definition *checked-staged-security-with-query-prefix-prefix-authenticated-opening-hit*
where

checked-staged-security-with-query-prefix-prefix-authenticated-opening-hit
trace-openings composition-openings i out $\longleftrightarrow$
 (case out of
 None $\Rightarrow$ False
 | Some (((((prefix, prefix-state), raw), raw-state),
 ((data, attacker-state), result)), final-state) $\Rightarrow$
 staged-composition-fri-roots data $\neq \square$ $\wedge$
 partial-authenticated-table (staged-trace-root data)
 (scale * clength) trace-openings prefix-state $\wedge$
 partial-authenticated-table (hd (staged-composition-fri-roots data))
 (scale * clength) composition-openings prefix-state $\wedge$
 partial-query-openings-consistent trace-openings composition-openings
 (staged-alphas data) (index (to-nat raw)))

definition *checked-staged-security-with-query-prefix-opening-path-output-hit*
where

checked-staged-security-with-query-prefix-opening-path-output-hit
trace-openings composition-openings out $\longleftrightarrow$
 (case out of
 None $\Rightarrow$ False
 | Some (((((prefix, prefix-state), raw), raw-state),
 ((data, attacker-state), result)), final-state) $\Rightarrow$
 ($\exists$ opening $\in$ set trace-openings $\cup$ set composition-openings.
 hash-map-new-output-hit
 (merkle-path-target-roots final-state
 (opening-root opening))

(opening-length opening)
 (opening-index opening)
 (opening-value opening)
 (opening-path opening))
 prefix-state final-state))

lemma *checked-staged-security-with-query-prefix-opening-path-output-hitE:*

assumes *hit:*

checked-staged-security-with-query-prefix-opening-path-output-hit
trace-openings composition-openings out

obtains *prefix prefix-state raw raw-state data attacker-state result*
final-state opn **where**

out = Some (((((prefix, prefix-state), raw), raw-state),
((data, attacker-state), result)), final-state)

opn ∈ set trace-openings ∪ set composition-openings

hash-map-new-output-hit

(merkle-path-target-roots final-state

(opening-root opn)

(opening-length opn)

(opening-index opn)

(opening-value opn)

(opening-path opn))

prefix-state final-state

finite

(merkle-path-target-roots final-state

(opening-root opn)

(opening-length opn)

(opening-index opn)

(opening-value opn)

(opening-path opn))

card

(merkle-path-target-roots final-state

(opening-root opn)

(opening-length opn)

(opening-index opn)

(opening-value opn)

(opening-path opn)) ≤ Suc (length (opening-path opn))

using *hit*

unfolding *checked-staged-security-with-query-prefix-opening-path-output-hit-def*

by *(cases out)*

(auto intro: that card-merkle-path-target-roots-le split: prod.splits)

lemma *checked-staged-security-experiment-with-query-prefix-data-state-prefix-extends-final:*

assumes *wf: staged-budget-wellformed budgets*

and *controlled: staged-adversary-controlled budgets A*

and *i-bound: i < rounds*

and *outcome:*

Some (((((prefix, prefix-state), raw), raw-state),

((data, attacker-state), result)), final-state)

```

    ∈
    set-dist
      (execute
        (checked-staged-security-experiment-with-query-prefix-data-state A i)
        adversary-initial-state)
  shows prefix-state ≤ final-state
proof —
  from outcome obtain t where head-support:
    Some (((prefix, prefix-state), raw), t) ∈
      set-dist
        (execute (checked-staged-query-prefix-receive-with-state A i)
          adversary-initial-state)
  and t-eq: t = raw-state
  and cont-support:
    Some (((data, attacker-state), result), final-state) ∈
      set-dist
        (execute
          (checked-staged-after-query-prefix-receive-with-verifier A i
            ((prefix, prefix-state), raw))
          raw-state)
  unfolding checked-staged-security-experiment-with-query-prefix-data-state-def
  by (auto elim!: set-dist-bindE split: prod.splits)
  have head:
    Some (((prefix, prefix-state), raw), raw-state) ∈
      set-dist
        (execute (checked-staged-query-prefix-receive-with-state A i)
          adversary-initial-state)
  using head-support t-eq by simp
  have recv:
    Some (raw, raw-state) ∈
      set-dist (execute receive-query-index-challenge prefix-state)
  by (rule checked-staged-query-prefix-receive-with-state-receive-support
    [OF head])
  have prefix-ext-raw: prefix-state ≤ raw-state
  by (rule receive-query-index-challenge-extends[OF recv])
  have data-out:
    Some (data, attacker-state) ∈
      set-dist
        (execute
          (checked-staged-after-query-prefix-receive A i
            ((prefix, prefix-state), raw))
          raw-state)
  by (rule
    checked-staged-after-query-prefix-receive-with-verifier-outcome(1)
    [OF cont-support])
  have verifier-out:
    Some (result, final-state) ∈
      set-dist
        (execute verify-monad

```

```

      (verifier-state-from-adversary attacker-state
       (staged-proof-transcript data)))
    by (rule
        checked-staged-after-query-prefix-receive-with-verifier-outcome(2)
        [OF cont-support])
  have raw-ext-attacker: raw-state ≤ attacker-state
    by (rule checked-staged-after-query-prefix-receive-extends
        [OF wf controlled i-bound data-out])
  have attacker-ext-verifier:
    attacker-state ≤
      verifier-state-from-adversary attacker-state
      (staged-proof-transcript data)
  proof -
    have attacker-state ≤ attacker-state
      unfolding less-eq-hash-ext-def less-eq-fmap-def by simp
    then show ?thesis
      by (rule hash-extends-verifier-state-from-adversary-right)
  qed
  have verifier-ext-final:
    verifier-state-from-adversary attacker-state
      (staged-proof-transcript data) ≤
      final-state
    by (rule verify-monad-hash-extends[OF verifier-out])
  have raw-ext-final: raw-state ≤ final-state
    by (rule hash-ext-trans[OF raw-ext-attacker])
      (rule hash-ext-trans[OF attacker-ext-verifier verifier-ext-final])
  show ?thesis
    by (rule hash-ext-trans[OF prefix-ext-raw raw-ext-final])
  qed

lemma checked-staged-security-with-query-prefix-authenticated-opening-hit-imp-prefix-or-path-output-hit:
  assumes ext: prefix-state ≤ final-state
  and hit:
    checked-staged-security-with-query-prefix-authenticated-opening-hit
    trace-openings composition-openings i
    (Some (((((prefix, prefix-state), raw), raw-state),
      ((data, attacker-state), result)), final-state))
  shows
    checked-staged-security-with-query-prefix-prefix-authenticated-opening-hit
    trace-openings composition-openings i
    (Some (((((prefix, prefix-state), raw), raw-state),
      ((data, attacker-state), result)), final-state)) ∨
    checked-staged-security-with-query-prefix-opening-path-output-hit
    trace-openings composition-openings
    (Some (((((prefix, prefix-state), raw), raw-state),
      ((data, attacker-state), result)), final-state))
  proof -
    have comp-nonempty: staged-composition-fri-roots data ≠ []
    and trace-table:

```

```

    partial-authenticated-table (staged-trace-root data)
      (scale * clength) trace-openings final-state
and comp-table:
    partial-authenticated-table (hd (staged-composition-fri-roots data))
      (scale * clength) composition-openings final-state
and consistent:
    partial-query-openings-consistent trace-openings composition-openings
      (staged-alphas data) (index (to-nat raw))
using hit
unfolding checked-staged-security-with-query-prefix-authenticated-opening-hit-def
by simp-all
have trace-pullback:
    partial-authenticated-table (staged-trace-root data)
      (scale * clength) trace-openings prefix-state ∨
    (∃ opn ∈ set trace-openings.
      hash-map-new-output-hit
        (merkle-path-target-roots final-state (staged-trace-root data)
          (scale * clength)
          (opening-index opn)
          (opening-value opn)
          (opening-path opn)) prefix-state final-state)
    by (rule partial-authenticated-table-pullback-or-new-output-hit
      [OF ext trace-table])
then show ?thesis
proof
  assume trace-prefix:
    partial-authenticated-table (staged-trace-root data)
      (scale * clength) trace-openings prefix-state
  have comp-pullback:
    partial-authenticated-table (hd (staged-composition-fri-roots data))
      (scale * clength) composition-openings prefix-state ∨
    (∃ opn ∈ set composition-openings.
      hash-map-new-output-hit
        (merkle-path-target-roots final-state
          (hd (staged-composition-fri-roots data))
          (scale * clength)
          (opening-index opn)
          (opening-value opn)
          (opening-path opn)) prefix-state final-state)
    by (rule partial-authenticated-table-pullback-or-new-output-hit
      [OF ext comp-table])
  then show ?thesis
proof
  assume comp-prefix:
    partial-authenticated-table (hd (staged-composition-fri-roots data))
      (scale * clength) composition-openings prefix-state
  then have
    checked-staged-security-with-query-prefix-prefix-authenticated-opening-hit
      trace-openings composition-openings i

```

```

    (Some (((((prefix, prefix-state), raw), raw-state),
      ((data, attacker-state), result)), final-state))
using comp-nonempty trace-prefix consistent
unfolding
  checked-staged-security-with-query-prefix-prefix-authenticated-opening-hit-def
by simp
then show ?thesis by simp
next
assume comp-path:
   $\exists \text{opn} \in \text{set composition-openings.}$ 
  hash-map-new-output-hit
    (merkle-path-target-roots final-state
      (hd (staged-composition-fri-roots data))
      (scale * clength)
      (opening-index opn)
      (opening-value opn)
      (opening-path opn)) prefix-state final-state
then obtain opn where opn-in:  $\text{opn} \in \text{set composition-openings}$ 
and path-hit:
  hash-map-new-output-hit
    (merkle-path-target-roots final-state
      (hd (staged-composition-fri-roots data))
      (scale * clength)
      (opening-index opn)
      (opening-value opn)
      (opening-path opn)) prefix-state final-state
by blast
have root-eq:
  opening-root opn = hd (staged-composition-fri-roots data)
using comp-table opn-in
unfolding partial-authenticated-table-def by blast
have len-eq: opening-length opn = scale * clength
using comp-table opn-in
unfolding partial-authenticated-table-def by blast
have
  checked-staged-security-with-query-prefix-opening-path-output-hit
  trace-openings composition-openings
  (Some (((((prefix, prefix-state), raw), raw-state),
    ((data, attacker-state), result)), final-state))
proof –
  have path-hit':
    hash-map-new-output-hit
      (merkle-path-target-roots final-state
        (opening-root opn)
        (opening-length opn)
        (opening-index opn)
        (opening-value opn)
        (opening-path opn)) prefix-state final-state
    using path-hit root-eq len-eq by simp

```



```

show ?thesis
  unfolding
    checked-staged-security-with-query-prefix-opening-path-output-hit-def
    using opn-in path-hit' by auto
qed
then show ?thesis by simp
qed
next
assume trace-path:
   $\exists$  opn  $\in$  set trace-openings.
    hash-map-new-output-hit
      (merkle-path-target-roots final-state (staged-trace-root data)
        (scale * clength)
        (opening-index opn)
        (opening-value opn)
        (opening-path opn)) prefix-state final-state
then obtain opn where opn-in: opn  $\in$  set trace-openings
and path-hit:
  hash-map-new-output-hit
    (merkle-path-target-roots final-state (staged-trace-root data)
      (scale * clength)
      (opening-index opn)
      (opening-value opn)
      (opening-path opn)) prefix-state final-state
by blast
have root-eq: opening-root opn = staged-trace-root data
using trace-table opn-in
unfolding partial-authenticated-table-def by blast
have len-eq: opening-length opn = scale * clength
using trace-table opn-in
unfolding partial-authenticated-table-def by blast
have
  checked-staged-security-with-query-prefix-opening-path-output-hit
  trace-openings composition-openings
  (Some (((((prefix, prefix-state), raw), raw-state),
    ((data, attacker-state), result)), final-state))
proof -
have path-hit':
  hash-map-new-output-hit
    (merkle-path-target-roots final-state
      (opening-root opn)
      (opening-length opn)
      (opening-index opn)
      (opening-value opn)
      (opening-path opn)) prefix-state final-state
using path-hit root-eq len-eq by simp
show ?thesis
unfolding checked-staged-security-with-query-prefix-opening-path-output-hit-def
using opn-in path-hit' by auto

```

```

qed
then show ?thesis by simp
qed
qed

lemma checked-staged-security-with-query-prefix-authenticated-opening-hit-imp-prefix-or-path-output-hit-on-support:
  assumes wf: staged-budget-wellformed budgets
  and controlled: staged-adversary-controlled budgets A
  and i-bound: i < rounds
  and support:
    out ∈
      set-dist
        (execute
          (checked-staged-security-experiment-with-query-prefix-data-state A i)
          adversary-initial-state)
  and hit:
    checked-staged-security-with-query-prefix-authenticated-opening-hit
    trace-openings composition-openings i out
  shows
    checked-staged-security-with-query-prefix-prefix-authenticated-opening-hit
    trace-openings composition-openings i out ∨
    checked-staged-security-with-query-prefix-opening-path-output-hit
    trace-openings composition-openings out
  proof (cases out)
  case None
  then show ?thesis
    using hit
    unfolding checked-staged-security-with-query-prefix-authenticated-opening-hit-def
    by simp
  next
  case (Some packed)
  then obtain prefix prefix-state raw raw-state data attacker-state result
    final-state where out-eq:
    out =
      Some (((((prefix, prefix-state), raw), raw-state),
        ((data, attacker-state), result)), final-state)
    by (cases packed, auto split: prod.splits)
  have ext: prefix-state ≤ final-state
    by (rule
      checked-staged-security-experiment-with-query-prefix-data-state-prefix-extends-final
      [OF wf controlled i-bound])
    (use support out-eq in simp)
  show ?thesis
    unfolding out-eq
    by (rule
      checked-staged-security-with-query-prefix-authenticated-opening-hit-imp-prefix-or-path-output-hit
      [OF ext])
    (use hit out-eq in simp)
qed

```

lemma *checked-staged-security-with-query-prefix-authenticated-opening-hit-le-actual-query-prefix-hit*:

assumes *i-bound*: $i < \text{rounds}$

shows

wp-event

(*checked-staged-security-experiment-with-query-prefix-data-state* *A* *i*)

(*checked-staged-security-with-query-prefix-authenticated-opening-hit*
trace-openings composition-openings *i*)

adversary-initial-state $\leq$

wp-event

(*checked-staged-security-experiment-with-query-prefix-data-state* *A* *i*)

(*checked-staged-security-with-actual-query-prefix-candidate-opening-hit*

((*replicate rounds* []) [*i* := *trace-openings*])

((*replicate rounds* []) [*i* := *composition-openings*])

i)

adversary-initial-state

proof (rule *wp-event-mono-on-support*)

fix *out*

assume *support*:

out $\in$

set-dist

(*execute*

(*checked-staged-security-experiment-with-query-prefix-data-state* *A* *i*)

adversary-initial-state)

and *hit*:

checked-staged-security-with-query-prefix-authenticated-opening-hit

trace-openings composition-openings *i* *out*

show

checked-staged-security-with-actual-query-prefix-candidate-opening-hit

((*replicate rounds* []) [*i* := *trace-openings*])

((*replicate rounds* []) [*i* := *composition-openings*]) *i* *out*

proof (*cases out*)

case *None*

then show *?thesis*

using *hit*

unfolding

checked-staged-security-with-query-prefix-authenticated-opening-hit-def

by *simp*

next

case (*Some packed*)

then obtain *prefix prefix-state raw raw-state data attacker-state result*

final-state **where** *out-eq*:

out =

Some (((((*prefix*, *prefix-state*), *raw*), *raw-state*),

((*data*, *attacker-state*), *result*)), *final-state*)

by (*cases packed*, *auto split: prod.splits*)

have *alphas-eq*: *staged-alphas data* = *sqp-alphas prefix*

by (rule *checked-staged-security-with-query-prefix-data-state-alphas-eq*)

(*use support out-eq in simp*)

```

have consistent:
  partial-query-openings-consistent trace-openings composition-openings
    (sqp-alphas prefix) (index (to-nat raw))
using hit alphas-eq
unfolding out-eq
  checked-staged-security-with-query-prefix-authenticated-opening-hit-def
by simp
have target-hit:
  checked-staged-query-prefix-dynamic-index-hit
    (staged-query-prefix-candidate-opening-query-target-from-prefix
      ((replicate rounds []) [i := trace-openings])
      ((replicate rounds []) [i := composition-openings])
      i)
    (Some (((prefix, prefix-state), raw), raw-state))
by (rule
  checked-staged-query-prefix-candidate-opening-target-from-prefix-single-round-hitI)
  (use i-bound consistent in simp-all)
show ?thesis
  unfolding out-eq
    checked-staged-security-with-actual-query-prefix-candidate-opening-hit-def
    checked-staged-security-with-query-prefix-dynamic-index-hit-def
  using target-hit by simp
qed
qed

lemma checked-staged-security-with-query-prefix-authenticated-opening-hit-bound:
assumes raw-bound: query-index-raw-preimage-bound
and wf: staged-budget-wellformed budgets
and controlled: staged-adversary-controlled budgets A
and i-bound: i < rounds
shows
  wp-event
    (checked-staged-security-experiment-with-query-prefix-data-state A i)
    (checked-staged-security-with-query-prefix-authenticated-opening-hit
      trace-openings composition-openings i)
  adversary-initial-state ≤
  staged-phase-relation-error size
    (staged-query-search-queries budgets i + 1) +
    (1::prob) / nnreal (card query-sample-space)

proof –
have actual-bound:
  wp-event
    (checked-staged-security-experiment-with-query-prefix-data-state A i)
    (checked-staged-security-with-actual-query-prefix-candidate-opening-hit
      ((replicate rounds []) [i := trace-openings])
      ((replicate rounds []) [i := composition-openings])
      i)
  adversary-initial-state ≤
  staged-phase-relation-error size

```

```

      (staged-query-search-queries budgets  $i + 1$ ) +
      (1::prob) / nnreal (card query-sample-space)
    by (rule checked-staged-security-with-actual-query-prefix-candidate-opening-hit-bound
        [OF raw-bound wf controlled])
      (use i-bound in simp)
  show ?thesis
  by (rule order-trans[
      OF checked-staged-security-with-query-prefix-authenticated-opening-hit-le-actual-query-prefix-hit
      actual-bound])
      (use i-bound in simp)
qed

```

lemma *checked-staged-security-with-query-prefix-authenticated-opening-hit-bound-from-budgets:*

assumes *wf: staged-budget-wellformed budgets*
and *controlled: staged-adversary-controlled budgets A*
and *i-bound: $i < \text{rounds}$*

shows

wp-event
 (checked-staged-security-experiment-with-query-prefix-data-state $A \ i$)
 (checked-staged-security-with-query-prefix-authenticated-opening-hit
 trace-openings composition-openings i)
adversary-initial-state $\leq$
staged-phase-relation-error size
 (staged-query-search-queries budgets $i + 1$) +
 (1::prob) / nnreal (card query-sample-space)

proof –

have *raw-bound: query-index-raw-preimage-bound*
by (rule query-index-raw-preimage-bound-from-sampler-wellformed)
show ?thesis
by (rule
 checked-staged-security-with-query-prefix-authenticated-opening-hit-bound
 [OF raw-bound wf controlled i-bound])

qed

lemma *checked-staged-security-with-query-prefix-data-state-bound-by-query-prefix-authenticated-opening-hit:*

assumes *raw-bound: query-index-raw-preimage-bound*
and *wf: staged-budget-wellformed budgets*
and *controlled: staged-adversary-controlled budgets A*
and *i-bound: $i < \text{rounds}$*

and *event-imp:*

$\bigwedge \text{out. } E \text{ out} \implies$
 checked-staged-security-with-query-prefix-authenticated-opening-hit
 trace-openings composition-openings i out

shows

wp-event
 (checked-staged-security-experiment-with-query-prefix-data-state $A \ i$)
E adversary-initial-state $\leq$
staged-phase-relation-error size
 (staged-query-search-queries budgets $i + 1$) +

```

      (1::prob) / nnreal (card query-sample-space)
proof –
  have event-le:
    wp-event
      (checked-staged-security-experiment-with-query-prefix-data-state A i)
      E adversary-initial-state ≤
    wp-event
      (checked-staged-security-experiment-with-query-prefix-data-state A i)
      (checked-staged-security-with-query-prefix-authenticated-opening-hit
        trace-openings composition-openings i)
      adversary-initial-state
  by (rule wp-event-mono) (rule event-imp)
have fixed-bound:
  wp-event
    (checked-staged-security-experiment-with-query-prefix-data-state A i)
    (checked-staged-security-with-query-prefix-authenticated-opening-hit
      trace-openings composition-openings i)
    adversary-initial-state ≤
    staged-phase-relation-error size
      (staged-query-search-queries budgets i + 1) +
      (1::prob) / nnreal (card query-sample-space)
  by (rule checked-staged-security-with-query-prefix-authenticated-opening-hit-bound
    [OF raw-bound wf controlled i-bound])
show ?thesis
  by (rule order-trans[OF event-le fixed-bound])
qed

```

lemma checked-staged-security-with-query-prefix-data-state-bound-by-query-prefix-authenticated-opening-hit-from-bud

```

assumes wf: staged-budget-wellformed budgets
and controlled: staged-adversary-controlled budgets A
and i-bound: i < rounds
and event-imp:
   $\bigwedge \text{out. } E \text{ out} \implies$ 
    checked-staged-security-with-query-prefix-authenticated-opening-hit
      trace-openings composition-openings i out

```

shows

```

  wp-event
    (checked-staged-security-experiment-with-query-prefix-data-state A i)
    E adversary-initial-state ≤
    staged-phase-relation-error size
      (staged-query-search-queries budgets i + 1) +
      (1::prob) / nnreal (card query-sample-space)

```

proof –

```

have raw-bound: query-index-raw-preimage-bound
by (rule query-index-raw-preimage-bound-from-sampler-wellformed)
show ?thesis
by (rule
  checked-staged-security-with-query-prefix-data-state-bound-by-query-prefix-authenticated-opening-hit
    [OF raw-bound wf controlled i-bound event-imp])

```

qed

lemma *checked-staged-security-with-query-prefix-prefix-authenticated-opening-hit-le-actual-query-prefix-hit:*

assumes *i-bound: i < rounds*

shows

wp-event

(*checked-staged-security-experiment-with-query-prefix-data-state A i*)
 (*checked-staged-security-with-query-prefix-prefix-authenticated-opening-hit*
trace-openings composition-openings i)
adversary-initial-state ≤

wp-event

(*checked-staged-security-experiment-with-query-prefix-data-state A i*)
 (*checked-staged-security-with-actual-query-prefix-candidate-opening-hit*
 ((*replicate rounds []*) [*i := trace-openings*])
 ((*replicate rounds []*) [*i := composition-openings*])
i)
adversary-initial-state

proof (rule *wp-event-mono-on-support*)

fix *out*

assume *support:*

out ∈
set-dist
 (*execute*
 (*checked-staged-security-experiment-with-query-prefix-data-state A i*)
adversary-initial-state)

and *hit:*

checked-staged-security-with-query-prefix-prefix-authenticated-opening-hit
trace-openings composition-openings i out

show

checked-staged-security-with-actual-query-prefix-candidate-opening-hit
 ((*replicate rounds []*) [*i := trace-openings*])
 ((*replicate rounds []*) [*i := composition-openings*]) *i out*

proof (cases *out*)

case *None*

then show *?thesis*

using *hit*

unfolding

checked-staged-security-with-query-prefix-prefix-authenticated-opening-hit-def

by *simp*

next

case (*Some packed*)

then obtain *prefix prefix-state raw raw-state data attacker-state result*
final-state **where** *out-eq:*

out =
Some (((((prefix, prefix-state), raw), raw-state),
((data, attacker-state), result)), final-state)

by (*cases packed, auto split: prod.splits*)

have *alphas-eq: staged-alphas data = sqp-alphas prefix*

by (rule *checked-staged-security-with-query-prefix-data-state-alphas-eq*)

```

    (use support out-eq in simp)
have consistent:
  partial-query-openings-consistent trace-openings composition-openings
    (sqp-alphas prefix) (index (to-nat raw))
using hit alphas-eq
unfolding out-eq
  checked-staged-security-with-query-prefix-prefix-authenticated-opening-hit-def
by simp
have target-hit:
  checked-staged-query-prefix-dynamic-index-hit
    (staged-query-prefix-candidate-opening-query-target-from-prefix
      ((replicate rounds []) [i := trace-openings])
      ((replicate rounds []) [i := composition-openings])
      i)
    (Some (((prefix, prefix-state), raw), raw-state))
by (rule
  checked-staged-query-prefix-candidate-opening-target-from-prefix-single-round-hitI)
  (use i-bound consistent in simp-all)
show ?thesis
unfolding out-eq
  checked-staged-security-with-actual-query-prefix-candidate-opening-hit-def
  checked-staged-security-with-query-prefix-dynamic-index-hit-def
using target-hit by simp
qed
qed

lemma checked-staged-security-with-query-prefix-prefix-authenticated-opening-hit-bound:
assumes raw-bound: query-index-raw-preimage-bound
and wf: staged-budget-wellformed budgets
and controlled: staged-adversary-controlled budgets A
and i-bound: i < rounds
shows
  wp-event
    (checked-staged-security-experiment-with-query-prefix-data-state A i)
    (checked-staged-security-with-query-prefix-prefix-authenticated-opening-hit
      trace-openings composition-openings i)
  adversary-initial-state ≤
  staged-phase-relation-error size
    (staged-query-search-queries budgets i + 1) +
    (1::prob) / nnreal (card query-sample-space)
proof –
have actual-bound:
  wp-event
    (checked-staged-security-experiment-with-query-prefix-data-state A i)
    (checked-staged-security-with-actual-query-prefix-candidate-opening-hit
      ((replicate rounds []) [i := trace-openings])
      ((replicate rounds []) [i := composition-openings])
      i)
  adversary-initial-state ≤

```



```

    staged-phase-relation-error size
      (staged-query-search-queries budgets  $i + 1$ ) +
      (1::prob) / nnreal (card query-sample-space)
  by (rule checked-staged-security-with-actual-query-prefix-candidate-opening-hit-bound
      [OF raw-bound wf controlled])
      (use i-bound in simp)
  show ?thesis
  by (rule order-trans[
      OF checked-staged-security-with-query-prefix-prefix-authenticated-opening-hit-le-actual-query-prefix-hit
      actual-bound])
      (use i-bound in simp)
qed

```

lemma *checked-staged-security-with-query-prefix-prefix-authenticated-opening-hit-bound-from-budgets:*

assumes *wf: staged-budget-wellformed budgets*
and *controlled: staged-adversary-controlled budgets A*
and *i-bound: $i < \text{rounds}$*

shows

wp-event
 (checked-staged-security-experiment-with-query-prefix-data-state $A\ i$)
 (checked-staged-security-with-query-prefix-prefix-authenticated-opening-hit
 trace-openings composition-openings i)
adversary-initial-state $\leq$
staged-phase-relation-error size
 (staged-query-search-queries budgets $i + 1$) +
 (1::prob) / nnreal (card query-sample-space)

proof –

have *raw-bound: query-index-raw-preimage-bound*
by (rule query-index-raw-preimage-bound-from-sampler-wellformed)
show ?thesis
by (rule
 checked-staged-security-with-query-prefix-prefix-authenticated-opening-hit-bound
 [OF raw-bound wf controlled i-bound])

qed

lemma *checked-staged-security-with-query-prefix-authenticated-opening-hit-bound-from-prefix-and-path-output:*

assumes *wf: staged-budget-wellformed budgets*
and *controlled: staged-adversary-controlled budgets A*
and *i-bound: $i < \text{rounds}$*
and *prefix-bound:*

wp-event
 (checked-staged-security-experiment-with-query-prefix-data-state $A\ i$)
 (checked-staged-security-with-query-prefix-prefix-authenticated-opening-hit
 trace-openings composition-openings i)
adversary-initial-state $\leq P$

and *path-bound:*

wp-event
 (checked-staged-security-experiment-with-query-prefix-data-state $A\ i$)
 (checked-staged-security-with-query-prefix-opening-path-output-hit

$\text{trace-openings composition-openings})$
 $\text{adversary-initial-state} \leq \text{path-error}$

shows

wp-event
 $(\text{checked-staged-security-experiment-with-query-prefix-data-state } A \ i)$
 $(\text{checked-staged-security-with-query-prefix-authenticated-opening-hit}$
 $\text{trace-openings composition-openings } i)$
 $\text{adversary-initial-state} \leq P + \text{path-error}$

proof –

have *event-le*:

wp-event
 $(\text{checked-staged-security-experiment-with-query-prefix-data-state } A \ i)$
 $(\text{checked-staged-security-with-query-prefix-authenticated-opening-hit}$
 $\text{trace-openings composition-openings } i)$
 $\text{adversary-initial-state} \leq$
 wp-event
 $(\text{checked-staged-security-experiment-with-query-prefix-data-state } A \ i)$
 $(\lambda \text{out.}$
 $\text{checked-staged-security-with-query-prefix-prefix-authenticated-opening-hit}$
 $\text{trace-openings composition-openings } i \ \text{out} \ \vee$
 $\text{checked-staged-security-with-query-prefix-opening-path-output-hit}$
 $\text{trace-openings composition-openings } \text{out})$
 $\text{adversary-initial-state}$

by (rule *wp-event-mono-on-support*)

(rule
checked-staged-security-with-query-prefix-authenticated-opening-hit-imp-prefix-or-path-output-hit-on-support
[OF *wf controlled i-bound*])

have *union-bound*:

wp-event
 $(\text{checked-staged-security-experiment-with-query-prefix-data-state } A \ i)$
 $(\lambda \text{out.}$
 $\text{checked-staged-security-with-query-prefix-prefix-authenticated-opening-hit}$
 $\text{trace-openings composition-openings } i \ \text{out} \ \vee$
 $\text{checked-staged-security-with-query-prefix-opening-path-output-hit}$
 $\text{trace-openings composition-openings } \text{out})$
 $\text{adversary-initial-state} \leq$
 wp-event
 $(\text{checked-staged-security-experiment-with-query-prefix-data-state } A \ i)$
 $(\text{checked-staged-security-with-query-prefix-prefix-authenticated-opening-hit}$
 $\text{trace-openings composition-openings } i)$
 $\text{adversary-initial-state} +$
 wp-event
 $(\text{checked-staged-security-experiment-with-query-prefix-data-state } A \ i)$
 $(\text{checked-staged-security-with-query-prefix-opening-path-output-hit}$
 $\text{trace-openings composition-openings})$
 $\text{adversary-initial-state}$

by (rule *wp-event-union-bound*)

have *sum-bound*:

wp-event

```

    (checked-staged-security-experiment-with-query-prefix-data-state A i)
    (checked-staged-security-with-query-prefix-prefix-authenticated-opening-hit
      trace-openings composition-openings i)
    adversary-initial-state +
  wp-event
    (checked-staged-security-experiment-with-query-prefix-data-state A i)
    (checked-staged-security-with-query-prefix-opening-path-output-hit
      trace-openings composition-openings)
    adversary-initial-state  $\leq P + \text{path-error}$ 
  by (intro add-mono prefix-bound path-bound)
show ?thesis
  by (rule order-trans[OF event-le])
    (rule order-trans[OF union-bound sum-bound])
qed

lemma checked-staged-security-with-query-prefix-authenticated-opening-hit-bound-from-path-output:
assumes raw-bound: query-index-raw-preimage-bound
and wf: staged-budget-wellformed budgets
and controlled: staged-adversary-controlled budgets A
and i-bound:  $i < \text{rounds}$ 
and path-bound:
  wp-event
    (checked-staged-security-experiment-with-query-prefix-data-state A i)
    (checked-staged-security-with-query-prefix-opening-path-output-hit
      trace-openings composition-openings)
    adversary-initial-state  $\leq \text{path-error}$ 
shows
  wp-event
    (checked-staged-security-experiment-with-query-prefix-data-state A i)
    (checked-staged-security-with-query-prefix-prefix-authenticated-opening-hit
      trace-openings composition-openings i)
    adversary-initial-state  $\leq$ 
      staged-phase-relation-error size
      (staged-query-search-queries budgets  $i + 1$ ) +
       $(1::\text{prob}) / \text{nnreal} (\text{card query-sample-space}) + \text{path-error}$ 
proof -
have prefix-bound:
  wp-event
    (checked-staged-security-experiment-with-query-prefix-data-state A i)
    (checked-staged-security-with-query-prefix-prefix-authenticated-opening-hit
      trace-openings composition-openings i)
    adversary-initial-state  $\leq$ 
      staged-phase-relation-error size
      (staged-query-search-queries budgets  $i + 1$ ) +
       $(1::\text{prob}) / \text{nnreal} (\text{card query-sample-space})$ 
  by (rule checked-staged-security-with-query-prefix-prefix-authenticated-opening-hit-bound
    [OF raw-bound wf controlled i-bound])
show ?thesis
  by (rule

```

checked-staged-security-with-query-prefix-authenticated-opening-hit-bound-from-prefix-and-path-output
[OF wf controlled i-bound prefix-bound path-bound])

qed

lemma *accepted-with-partial-initial-openings-aligned-consistent-prefix-target-hit*:
assumes *partial*:
accepted-with-partial-initial-openings-aligned-consistent s out fr
f-fri-roots f-final as dg composition-fri-roots final query-idxs
trace-openings composition-openings
and *i-bound*: $i < \text{rounds}$
and *idx-eq*: $\text{query-idxs} \ ! \ i = \text{index} \ (\text{to-nat } \text{raw})$
and *alphas-eq*: $\text{as} = \text{sqp-alphas } \text{prefix}$
shows
checked-staged-query-prefix-dynamic-index-hit
(staged-query-prefix-candidate-opening-query-target-from-prefix
trace-openings composition-openings i)
(Some (((prefix, prefix-state), raw), raw-state))

proof –
have *success*:
query-idxs ! i ∈
partial-query-success-indices-at trace-openings composition-openings
as i
by (rule
accepted-with-partial-initial-openings-aligned-consistent-success-index
[OF partial i-bound])
have $\text{index} \ (\text{to-nat } \text{raw}) \in$
staged-query-prefix-candidate-opening-query-target-from-prefix
trace-openings composition-openings i prefix prefix-state
using *success idx-eq alphas-eq*
unfolding *staged-query-prefix-candidate-opening-query-target-from-prefix-def*
by *simp*
then show *?thesis*
unfolding *checked-staged-query-prefix-dynamic-index-hit-def* **by** *simp*

qed

lemma *query-bad-with-aligned-partial-candidates-prefix-target-hitE*:
assumes *bad*: *query-bad-with-aligned-partial-candidates s out*
and *i-bound*: $i < \text{rounds}$
and *idx-eq*: $\text{query-idxs} \ ! \ i = \text{index} \ (\text{to-nat } \text{raw})$
and *alphas-eq*: $\text{as} = \text{sqp-alphas } \text{prefix}$
and *partial*:
accepted-with-partial-initial-openings-aligned-consistent s out fr
f-fri-roots f-final as dg composition-fri-roots final query-idxs
trace-openings composition-openings
and *trace-candidate*:
partial-trace-table-candidate trace-table trace-openings
and *comp-candidate*:
partial-composition-table-candidate composition-table
composition-openings

and *trace-low*: *trace-table-low-degree trace-table*
and *comp-low*:
 composition-table-low-degree maxDegree composition-table
and *not-all*:
 $\neg$ *all-queries-consistent trace-table composition-table as*
obtains
 checked-staged-query-prefix-dynamic-index-hit
 (*staged-query-prefix-candidate-opening-query-target-from-prefix*
 trace-openings composition-openings i)
 (*Some* (((*prefix*, *prefix-state*), *raw*), *raw-state*))
 nnreal
 (*card*
 (*staged-query-prefix-candidate-opening-query-target-from-prefix*
 trace-openings composition-openings i prefix prefix-state)) /
 nnreal (*card query-sample-space*) $\leq$ *query-error-bound*
proof –
 have *hit*:
 checked-staged-query-prefix-dynamic-index-hit
 (*staged-query-prefix-candidate-opening-query-target-from-prefix*
 trace-openings composition-openings i)
 (*Some* (((*prefix*, *prefix-state*), *raw*), *raw-state*))
 by (*rule*
 accepted-with-partial-initial-openings-aligned-consistent-prefix-target-hit
 [*OF partial i-bound idx-eq alphas-eq*])
 have *not-all-prefix*:
 $\neg$ *all-queries-consistent trace-table composition-table*
 (*sqp-alphas prefix*)
 using *not-all alphas-eq* **by** *simp*
 have *fraction*:
 nnreal
 (*card*
 (*staged-query-prefix-candidate-opening-query-target-from-prefix*
 trace-openings composition-openings i prefix prefix-state)) /
 nnreal (*card query-sample-space*) $\leq$ *query-error-bound*
 by (*rule*
 staged-query-prefix-candidate-opening-query-target-from-prefix-fraction-bound
 [*OF trace-candidate comp-candidate trace-low comp-low not-all-prefix*])
 show *?thesis*
 by (*rule that*[*OF hit fraction*])
qed

lemma *accepted-with-partial-initial-openings-aligned-consistent-candidate-opening-hit-from-prefix*:
assumes *partial*:
 accepted-with-partial-initial-openings-aligned-consistent s out fr
 f-fri-roots f-final as dg composition-fri-roots final query-idxs
 trace-openings composition-openings
and *i-bound*: $i < \text{rounds}$
and *idx-eq*: *query-idxs* ! $i = \text{index (to-nat raw)}$
and *alphas-eq*: *as* = *sqp-alphas prefix*

and *builder*:
Some (data, attacker-state) ∈
set-dist
(execute (checked-staged-transcript-program A)
adversary-initial-state)
and *prefix-receive*:
Some (((prefix, prefix-state), raw), raw-state) ∈
set-dist
(execute (checked-staged-query-prefix-receive-with-state A i)
adversary-initial-state)
shows
checked-staged-security-with-actual-alpha-prefix-candidate-opening-query-hit-from-prefix
A trace-openings composition-openings i
(Some (((((alpha-prefix, alpha-prefix-state), data), attacker-state),
result), final-state))
proof –
have *target-hit*:
checked-staged-query-prefix-dynamic-index-hit
(staged-query-prefix-candidate-opening-query-target-from-prefix
trace-openings composition-openings i)
(Some (((prefix, prefix-state), raw), raw-state))
by (rule
accepted-with-partial-initial-openings-aligned-consistent-prefix-target-hit
[OF partial i-bound idx-eq alphas-eq])
show *?thesis*
by (rule
checked-staged-security-with-actual-alpha-prefix-candidate-opening-query-hit-from-prefixI
[OF i-bound builder prefix-receive target-hit])
qed

lemma *accepted-with-partial-initial-openings-aligned-transcript-consistent-candidate-opening-hit-from-prefix*:
assumes *partial*:
accepted-with-partial-initial-openings-aligned-transcript-consistent s
out fr f-fri-roots f-final as dg composition-fri-roots final
query-idxs trace-openings composition-openings
and *i-bound*: *i < rounds*
and *idx-eq*: *query-idxs ! i = index (to-nat raw)*
and *alphas-eq*: *as = sqp-alphas prefix*
and *builder*:
Some (data, attacker-state) ∈
set-dist
(execute (checked-staged-transcript-program A)
adversary-initial-state)
and *prefix-receive*:
Some (((prefix, prefix-state), raw), raw-state) ∈
set-dist
(execute (checked-staged-query-prefix-receive-with-state A i)
adversary-initial-state)
shows

checked-staged-security-with-actual-alpha-prefix-candidate-opening-query-hit-from-prefix
A trace-openings composition-openings i
(Some (((((alpha-prefix, alpha-prefix-state), data), attacker-state),
result), final-state)))
proof –
have *aligned*:
accepted-with-partial-initial-openings-aligned-consistent s out fr
f-fri-roots f-final as dg composition-fri-roots final query-idxs
trace-openings composition-openings
by (rule
accepted-with-partial-initial-openings-aligned-transcript-consistent-imp-aligned
[OF partial])
show *?thesis*
by (rule
accepted-with-partial-initial-openings-aligned-consistent-candidate-opening-hit-from-prefix
[OF aligned i-bound idx-eq alphas-eq builder prefix-receive])
qed

lemma *query-bad-with-aligned-partial-candidates-candidate-opening-hit-from-prefixE*:
assumes *bad: query-bad-with-aligned-partial-candidates s out*
and *i-bound: i < rounds*
and *idx-eq: query-idxs ! i = index (to-nat raw)*
and *alphas-eq: as = sqp-alphas prefix*
and *partial*:
accepted-with-partial-initial-openings-aligned-consistent s out fr
f-fri-roots f-final as dg composition-fri-roots final query-idxs
trace-openings composition-openings
and *builder*:
Some (data, attacker-state) ∈
set-dist
(execute (checked-staged-transcript-program A)
adversary-initial-state)
and *prefix-receive*:
Some (((prefix, prefix-state), raw), raw-state) ∈
set-dist
(execute (checked-staged-query-prefix-receive-with-state A i)
adversary-initial-state)
obtains
checked-staged-security-with-actual-alpha-prefix-candidate-opening-query-hit-from-prefix
A trace-openings composition-openings i
(Some (((((alpha-prefix, alpha-prefix-state), data), attacker-state),
result), final-state)))
proof –
have *hit*:
checked-staged-security-with-actual-alpha-prefix-candidate-opening-query-hit-from-prefix
A trace-openings composition-openings i
(Some (((((alpha-prefix, alpha-prefix-state), data), attacker-state),
result), final-state)))
by (rule

$\text{accepted-with-partial-initial-openings-aligned-consistent-candidate-opening-hit-from-prefix}$
 $[OF \text{ partial } i\text{-bound } idx\text{-eq } alphas\text{-eq } builder \text{ prefix-receive}]$
show ?thesis
by (rule that[OF hit])
qed

lemma *query-bad-with-aligned-transcript-partial-candidates-candidate-opening-hit-from-prefixE*:
assumes *bad*: *query-bad-with-aligned-transcript-partial-candidates* *s* *out*
and *i-bound*: $i < \text{rounds}$
and *idx-eq*: *query-idxs* ! $i = \text{index } (\text{to-nat } \text{raw})$
and *alphas-eq*: *as* = *sqp-alphas prefix*
and *partial*:
 $\text{accepted-with-partial-initial-openings-aligned-transcript-consistent } s$
 $\text{out fr } f\text{-fri-roots } f\text{-final } as \text{ dg composition-fri-roots final}$
 $\text{query-idxs trace-openings composition-openings}$
and *builder*:
 $\text{Some } (data, \text{attacker-state}) \in$
 set-dist
 $(\text{execute } (\text{checked-staged-transcript-program } A)$
 $\text{adversary-initial-state})$
and *prefix-receive*:
 $\text{Some } (((\text{prefix}, \text{prefix-state}), \text{raw}), \text{raw-state}) \in$
 set-dist
 $(\text{execute } (\text{checked-staged-query-prefix-receive-with-state } A \ i)$
 $\text{adversary-initial-state})$
obtains
 $\text{checked-staged-security-with-actual-alpha-prefix-candidate-opening-query-hit-from-prefix}$
 $A \text{ trace-openings composition-openings } i$
 $(\text{Some } (((((\text{alpha-prefix}, \text{alpha-prefix-state}), data), \text{attacker-state}),$
 $\text{result}), \text{final-state})))$
proof –
have *hit*:
 $\text{checked-staged-security-with-actual-alpha-prefix-candidate-opening-query-hit-from-prefix}$
 $A \text{ trace-openings composition-openings } i$
 $(\text{Some } (((((\text{alpha-prefix}, \text{alpha-prefix-state}), data), \text{attacker-state}),$
 $\text{result}), \text{final-state})))$
by (rule
 $\text{accepted-with-partial-initial-openings-aligned-transcript-consistent-candidate-opening-hit-from-prefix}$
 $[OF \text{ partial } i\text{-bound } idx\text{-eq } alphas\text{-eq } builder \text{ prefix-receive}]$)
show ?thesis
by (rule that[OF hit])
qed

lemma *partial-candidate-query-witness-candidate-opening-hit-from-prefix*:
assumes *partial*:
 $\text{accepted-with-partial-initial-openings } s \text{ out fr } f\text{-fri-roots } f\text{-final } as$
 $\text{dg composition-fri-roots final trace-query-idxs trace-openings}$
 $\text{composition-query-idxs composition-openings}$
and *trace-candidate*:

partial-trace-table-candidate trace-table trace-openings
and *comp-candidate:*
partial-composition-table-candidate composition-table
composition-openings
and *trace-low: trace-table-low-degree trace-table*
and *comp-low:*
composition-table-low-degree maxDegree composition-table
and *not-all:*
 $\neg$ *all-queries-consistent trace-table composition-table as*
and *i-bound: $i < \text{rounds}$*
and *alphas-eq: $as = \text{sqp-alphas prefix}$*
and *consistent:*
partial-query-openings-consistent (trace-openings ! i)
(composition-openings ! i) as (index (to-nat raw))
and *builder:*
Some (data, attacker-state) $\in$
set-dist
(execute (checked-staged-transcript-program A)
adversary-initial-state)
and *prefix-receive:*
Some (((prefix, prefix-state), raw), raw-state) $\in$
set-dist
(execute (checked-staged-query-prefix-receive-with-state A i)
adversary-initial-state)
shows
checked-staged-security-with-actual-alpha-prefix-candidate-opening-query-hit-from-prefix
A ((replicate rounds []) [i := trace-openings ! i])
((replicate rounds []) [i := composition-openings ! i]) i
(Some (((((alpha-prefix, alpha-prefix-state), data), attacker-state),
result), final-state)))
proof –
have *target-hit:*
checked-staged-query-prefix-dynamic-index-hit
(staged-query-prefix-candidate-opening-query-target-from-prefix
((replicate rounds []) [i := trace-openings ! i])
((replicate rounds []) [i := composition-openings ! i])
i)
(Some (((prefix, prefix-state), raw), raw-state)))
by (rule
checked-staged-query-prefix-candidate-opening-target-from-prefix-single-round-hitI)
(use i-bound consistent alphas-eq in simp-all)
show *?thesis*
by (rule
checked-staged-security-with-actual-alpha-prefix-candidate-opening-query-hit-from-prefixI
[OF i-bound builder prefix-receive target-hit])
qed

lemma *query-bad-with-partial-candidates-witness-candidate-opening-hit-from-prefix:*
assumes *bad: query-bad-with-partial-candidates s out*

and *partial*:
accepted-with-partial-initial-openings s out fr f-fri-roots f-final as
dg composition-fri-roots final trace-query-idxs trace-openings
composition-query-idxs composition-openings
and *trace-candidate*:
partial-trace-table-candidate trace-table trace-openings
and *comp-candidate*:
partial-composition-table-candidate composition-table
composition-openings
and *trace-low*: *trace-table-low-degree trace-table*
and *comp-low*:
composition-table-low-degree maxDegree composition-table
and *not-all*:
 $\neg$ *all-queries-consistent trace-table composition-table as*
and *i-bound*: *i < rounds*
and *alphas-eq*: *as = sqp-alphas prefix*
and *consistent*:
partial-query-openings-consistent (trace-openings ! i)
(composition-openings ! i) as (index (to-nat raw))
and *builder*:
Some (data, attacker-state) ∈
set-dist
(execute (checked-staged-transcript-program A)
adversary-initial-state)
and *prefix-receive*:
Some (((prefix, prefix-state), raw), raw-state) ∈
set-dist
(execute (checked-staged-query-prefix-receive-with-state A i)
adversary-initial-state)
shows
checked-staged-security-with-actual-alpha-prefix-candidate-opening-query-hit-from-prefix
A ((replicate rounds []) [i := trace-openings ! i])
((replicate rounds []) [i := composition-openings ! i]) i
(Some (((((alpha-prefix, alpha-prefix-state), data), attacker-state),
result), final-state))
by (*rule partial-candidate-query-witness-candidate-opening-hit-from-prefix*
[OF partial trace-candidate comp-candidate trace-low comp-low not-all
i-bound alphas-eq consistent builder prefix-receive])

end

end

theory *Staged-Security-Experiment-Composition-Query-Bridge*

imports

Staged-Security-Experiment-Composition-Query-Residual
Soundness-Query-Opening-Consistency
Soundness-Bound-Table-Partial-Openings

begin

Narrow bridge from the composition-drift query residual to the candidate-opening query target. This is kept separate from the residual split theory because the surrounding staged soundness layers are already heavy to consolidate.

context *soundness*

begin

lemma *partial-query-openings-consistent-synthetic:*

assumes *idx-sample*: $idx \in \text{query-sample-space}$

obtains *trace-openings composition-openings* **where**

partial-query-openings-consistent trace-openings composition-openings
as *idx*

proof –

let *?trace-openings* =

map ($\lambda j.$

$\langle \langle$ *opening-root* = 0,

opening-length = *scale* * *clength*,

opening-index = *j*,

opening-value = 0,

opening-path = $\langle \rangle$ $\rangle$) (*powers-scaled idx*)

let *?composition-openings* =

$\langle \langle$ *opening-root* = 0,

opening-length = *scale* * *clength*,

opening-index = *idx*,

opening-value =

cp-eval as (*map opening-value ?trace-openings*) (*h* $\wedge$ *idx* * *shift*),

opening-path = $\langle \rangle$ $\rangle$,

$\langle \langle$ *opening-root* = 0,

opening-length = *scale* * *clength*,

opening-index = *fri-sibling-index* (*scale* * *clength*) *idx*,

opening-value = 0,

opening-path = $\langle \rangle$ $\rangle$]

have *trace-indices*:

map opening-index ?trace-openings = *powers-scaled idx*

by (*simp add: o-def*)

have *comp-indices*:

map opening-index ?composition-openings =

[*idx*, *fri-sibling-index* (*scale* * *clength*) *idx*]

by *simp*

have *consistent*:

partial-query-openings-consistent ?trace-openings

?composition-openings as idx

by (*rule partial-query-openings-consistentI*

[*OF idx-sample trace-indices comp-indices*])

simp

show *?thesis*

by (*rule that[OF consistent]*)

qed

lemma *checked-staged-security-with-actual-alpha-prefix-composition-candidate-opening-query-hitI-from-prefix-receive*

assumes *i-bound*: $i < \text{rounds}$

and *builder*:

Some (*data*, *attacker-state*) $\in$
set-dist
(execute (checked-staged-transcript-program A)
adversary-initial-state)

and *prefix-receive*:

Some (((*prefix*, *prefix-state*), *raw*), *raw-state*) $\in$
set-dist
(execute (checked-staged-query-prefix-receive-with-state A i)
adversary-initial-state)

shows

checked-staged-security-with-actual-alpha-prefix-composition-candidate-opening-query-hit
A
(Some ((((((alpha-prefix, alpha-prefix-state), data), attacker-state),
result), final-state)))

proof –

obtain *trace-openings composition-openings* **where** *consistent*:

partial-query-openings-consistent trace-openings composition-openings
(staged-alphas data) (index (to-nat raw))

proof (*rule partial-query-openings-consistent-synthetic*)

show *index (to-nat raw) $\in$ query-sample-space*

using *index-less-query-sample-space*

unfolding *query-sample-space-def* **by** *simp*

qed

have *component*:

checked-staged-security-with-actual-alpha-prefix-candidate-opening-query-hit
A
((replicate rounds []) [i := trace-openings])
((replicate rounds []) [i := composition-openings])
(staged-alphas data) i
(Some ((((((alpha-prefix, alpha-prefix-state), data), attacker-state),
result), final-state)))

by (*rule*

checked-staged-security-with-actual-alpha-prefix-candidate-opening-query-hitI-from-prefix-receive
[OF i-bound builder prefix-receive consistent])

show *?thesis*

by (*rule*

checked-staged-security-with-actual-alpha-prefix-composition-candidate-opening-query-hitI
[OF component])

qed

definition *checked-staged-security-with-actual-alpha-prefix-authenticated-candidate-opening-query-hit*

$:: ('f \text{ staged-adversary} \Rightarrow$
 $(((((('f \times 'f \text{ list} \times 'f \text{ list} \times 'f) \times$
 $'f \text{ protocol-channel}) \times 'f \text{ staged-proof-data}) \times$

$\text{'f protocol-channel) } \times \text{ unit list) } \times$
 $\text{'f protocol-channel) option } \Rightarrow \text{ bool)}$

where

$\text{checked-staged-security-with-actual-alpha-prefix-authenticated-candidate-opening-query-hit}$
 $A \text{ out } \longleftrightarrow$
 $(\text{case out of}$
 $\text{None } \Rightarrow \text{ False}$
 $| \text{Some } (((((\text{alpha-prefix, alpha-prefix-state}), \text{data}), \text{attacker-state}),$
 $\text{result}), \text{final-state}) \Rightarrow$
 $\text{staged-composition-fri-roots data } \neq [] \wedge$
 $(\exists i \text{ trace-openings composition-openings.}$
 $\text{partial-authenticated-table (staged-trace-root data)}$
 $\text{(scale * clength) trace-openings final-state } \wedge$
 $\text{partial-authenticated-table (hd (staged-composition-fri-roots data))}$
 $\text{(scale * clength) composition-openings final-state } \wedge$
 $\text{checked-staged-security-with-actual-alpha-prefix-candidate-opening-query-hit A}$
 $((\text{replicate rounds } []) [i := \text{trace-openings}])$
 $((\text{replicate rounds } []) [i := \text{composition-openings}])$
 $(\text{staged-alphas data}) i$
 $(\text{Some } (((((\text{alpha-prefix, alpha-prefix-state}), \text{data}), \text{attacker-state}),$
 $\text{result}), \text{final-state}))))))$

lemma *checked-staged-security-with-actual-alpha-prefix-authenticated-candidate-opening-query-hitI:*

assumes *comp-nonempty: staged-composition-fri-roots data* $\neq []$

and *trace-auth:*

$\text{partial-authenticated-table (staged-trace-root data)}$
 $\text{(scale * clength) trace-openings final-state}$

and *comp-auth:*

$\text{partial-authenticated-table (hd (staged-composition-fri-roots data))}$
 $\text{(scale * clength) composition-openings final-state}$

and *component:*

$\text{checked-staged-security-with-actual-alpha-prefix-candidate-opening-query-hit A}$
 $((\text{replicate rounds } []) [i := \text{trace-openings}])$
 $((\text{replicate rounds } []) [i := \text{composition-openings}])$
 $(\text{staged-alphas data}) i$
 $(\text{Some } (((((\text{alpha-prefix, alpha-prefix-state}), \text{data}), \text{attacker-state}),$
 $\text{result}), \text{final-state}))))$

shows

$\text{checked-staged-security-with-actual-alpha-prefix-authenticated-candidate-opening-query-hit}$
 A
 $(\text{Some } (((((\text{alpha-prefix, alpha-prefix-state}), \text{data}), \text{attacker-state}),$
 $\text{result}), \text{final-state}))))$

proof –

have *witness:*

$\exists j \text{ trace-openings' composition-openings'.
 $\text{partial-authenticated-table (staged-trace-root data)}$
 $\text{(scale * clength) trace-openings' final-state } \wedge$
 $\text{partial-authenticated-table (hd (staged-composition-fri-roots data))}$
 $\text{(scale * clength) composition-openings' final-state } \wedge$$

```

checked-staged-security-with-actual-alpha-prefix-candidate-opening-query-hit A
  ((replicate rounds []) [j := trace-openings])
  ((replicate rounds []) [j := composition-openings])
  (staged-alphas data) j
  (Some (((alpha-prefix, alpha-prefix-state), data), attacker-state),
    result), final-state))
by (intro exI[of - i] exI[of - trace-openings]
    exI[of - composition-openings] conjI)
  (use trace-auth comp-auth component in simp-all)
show ?thesis
unfolding
  checked-staged-security-with-actual-alpha-prefix-authenticated-candidate-opening-query-hit-def
using comp-nonempty witness by simp
qed

lemma checked-staged-security-with-actual-alpha-prefix-authenticated-candidate-opening-query-hit-imp-compact:
  assumes hit:
    checked-staged-security-with-actual-alpha-prefix-authenticated-candidate-opening-query-hit
    A out
  shows
    checked-staged-security-with-actual-alpha-prefix-composition-candidate-opening-query-hit
    A out
proof (cases out)
case None
then show ?thesis
  using hit
  unfolding
    checked-staged-security-with-actual-alpha-prefix-authenticated-candidate-opening-query-hit-def
  by simp
next
case (Some packed)
then obtain alpha-prefix alpha-prefix-state data attacker-state result
  final-state where out-eq:
    out =
      Some (((alpha-prefix, alpha-prefix-state), data), attacker-state),
        result), final-state)
  by (cases packed, auto split: prod.splits)
from hit[unfolded out-eq]
  checked-staged-security-with-actual-alpha-prefix-authenticated-candidate-opening-query-hit-def]
obtain i trace-openings composition-openings where component:
  checked-staged-security-with-actual-alpha-prefix-candidate-opening-query-hit A
  ((replicate rounds []) [i := trace-openings])
  ((replicate rounds []) [i := composition-openings])
  (staged-alphas data) i
  (Some (((alpha-prefix, alpha-prefix-state), data), attacker-state),
    result), final-state))
by auto
show ?thesis
unfolding out-eq

```

by (rule
 checked-staged-security-with-actual-alpha-prefix-composition-candidate-opening-query-hitI
 [*OF component*])
 qed

lemma *checked-staged-security-with-actual-alpha-prefix-authenticated-candidate-opening-query-hit-le-compact*:
 wp-event
 (*checked-staged-security-experiment-with-actual-alpha-prefix-data-state A*)
 (*checked-staged-security-with-actual-alpha-prefix-authenticated-candidate-opening-query-hit*
 A)
 adversary-initial-state ≤
 wp-event
 (*checked-staged-security-experiment-with-actual-alpha-prefix-data-state A*)
 (*checked-staged-security-with-actual-alpha-prefix-composition-candidate-opening-query-hit*
 A)
 adversary-initial-state
 by (rule wp-event-mono)
 (rule
 checked-staged-security-with-actual-alpha-prefix-authenticated-candidate-opening-query-hit-imp-compact)

lemma *checked-staged-security-with-actual-alpha-prefix-authenticated-candidate-opening-query-hitI-from-current-open*
assumes *wf*: *staged-budget-wellformed budgets*
and *controlled*: *staged-adversary-controlled budgets A*
and *comp-nonempty*: *staged-composition-fri-roots data* ≠ []
and *i-bound*: *i* < *rounds*
and *builder*:
 Some (*data*, *attacker-state*) ∈
 set-dist
 (*execute* (*checked-staged-transcript-program A*)
 adversary-initial-state)
and *verifier*:
 Some (*result*, *final-state*) ∈
 set-dist
 (*execute verify-monad*
 (*verifier-state-from-adversary attacker-state*
 (*staged-proof-transcript data*))))
and *final-lookup*:
 fmlookup (*HashMap final-state*)
 (*QueryIndexChallenge i*
 (*state-after-query-chunks*
 (*staged-query-start-hash data*) (*staged-query-chunks data i*))) =
 Some raw
and *trace-auth*:
 partial-authenticated-table (*staged-trace-root data*)
 (*scale * clength*) *trace-openings final-state*
and *comp-auth*:
 partial-authenticated-table (*hd* (*staged-composition-fri-roots data*))
 (*scale * clength*) *composition-openings final-state*
and *consistent*:

partial-query-openings-consistent trace-openings composition-openings
 (staged-alphas data) (index (to-nat raw))

shows

checked-staged-security-with-actual-alpha-prefix-authenticated-candidate-opening-query-hit
A
 (Some (((((alpha-prefix, alpha-prefix-state), data), attacker-state),
 result), final-state))

proof –

from *checked-staged-transcript-program-query-prefix-receive-final-lookup-support*
 [OF wf controlled i-bound builder verifier final-lookup]

obtain *prefix prefix-state raw-state* **where** *prefix-receive*:
 Some (((prefix, prefix-state), raw), raw-state) ∈
 set-dist
 (execute (checked-staged-query-prefix-receive-with-state A i)
 adversary-initial-state)

by *blast*

have *component*:
checked-staged-security-with-actual-alpha-prefix-candidate-opening-query-hit A
 ((replicate rounds []) [i := trace-openings])
 ((replicate rounds []) [i := composition-openings])
 (staged-alphas data) i
 (Some (((((alpha-prefix, alpha-prefix-state), data), attacker-state),
 result), final-state))

by (rule
checked-staged-security-with-actual-alpha-prefix-candidate-opening-query-hitI-from-prefix-receive
 [OF i-bound builder prefix-receive consistent])

show *?thesis*

by (rule
checked-staged-security-with-actual-alpha-prefix-authenticated-candidate-opening-query-hitI
 [OF comp-nonempty trace-auth comp-auth component])

qed

definition *checked-staged-security-with-actual-alpha-prefix-authenticated-candidate-opening-query-hit-from-prefix*
 $:: ('f \text{ staged-adversary} \Rightarrow$
 $(((((('f \times 'f \text{ list} \times 'f \text{ list} \times 'f) \times$
 $'f \text{ protocol-channel}) \times 'f \text{ staged-proof-data}) \times$
 $'f \text{ protocol-channel}) \times \text{unit list}) \times$
 $'f \text{ protocol-channel}) \text{ option} \Rightarrow \text{bool})$

where

checked-staged-security-with-actual-alpha-prefix-authenticated-candidate-opening-query-hit-from-prefix
A out $\longleftrightarrow$
 (case out of
 None $\Rightarrow$ False
 | Some (((((alpha-prefix, alpha-prefix-state), data), attacker-state),
 result), final-state) $\Rightarrow$
 staged-composition-fri-roots data $\neq [] \wedge$
 $(\exists i \text{ trace-openings composition-openings.}$
 partial-authenticated-table (staged-trace-root data)
 (scale * clength) trace-openings final-state $\wedge$

partial-authenticated-table (*hd* (*staged-composition-fri-roots data*))
 (*scale * clength*) *composition-openings final-state* $\wedge$
checked-staged-security-with-actual-alpha-prefix-candidate-opening-query-hit-from-prefix
A
 ((*replicate rounds* []) [*i := trace-openings*])
 ((*replicate rounds* []) [*i := composition-openings*])
i
 (Some (((((*alpha-prefix*, *alpha-prefix-state*), *data*),
attacker-state), *result*), *final-state*))))

lemma *checked-staged-security-with-actual-alpha-prefix-authenticated-candidate-opening-query-hit-from-prefixI*:

assumes *comp-nonempty*: *staged-composition-fri-roots data* $\neq$ []

and *trace-auth*:

partial-authenticated-table (*staged-trace-root data*)
 (*scale * clength*) *trace-openings final-state*

and *comp-auth*:

partial-authenticated-table (*hd* (*staged-composition-fri-roots data*))
 (*scale * clength*) *composition-openings final-state*

and *component*:

checked-staged-security-with-actual-alpha-prefix-candidate-opening-query-hit-from-prefix
A
 ((*replicate rounds* []) [*i := trace-openings*])
 ((*replicate rounds* []) [*i := composition-openings*])
i
 (Some (((((*alpha-prefix*, *alpha-prefix-state*), *data*), *attacker-state*),
result), *final-state*))

shows

checked-staged-security-with-actual-alpha-prefix-authenticated-candidate-opening-query-hit-from-prefix
A
 (Some (((((*alpha-prefix*, *alpha-prefix-state*), *data*), *attacker-state*),
result), *final-state*))

proof —

have *witness*:

$\exists j$ *trace-openings'* *composition-openings'*.

partial-authenticated-table (*staged-trace-root data*)
 (*scale * clength*) *trace-openings' final-state* $\wedge$

partial-authenticated-table (*hd* (*staged-composition-fri-roots data*))
 (*scale * clength*) *composition-openings' final-state* $\wedge$

checked-staged-security-with-actual-alpha-prefix-candidate-opening-query-hit-from-prefix
A

((*replicate rounds* []) [*j := trace-openings'*])

((*replicate rounds* []) [*j := composition-openings'*])

j

(Some (((((*alpha-prefix*, *alpha-prefix-state*), *data*), *attacker-state*),
result), *final-state*))

by (*intro exI*[*of - i*] *exI*[*of - trace-openings*]

exI[*of - composition-openings*] *conjI*)

(*use trace-auth comp-auth component in simp-all*)

show *?thesis*

unfolding
checked-staged-security-with-actual-alpha-prefix-authenticated-candidate-opening-query-hit-from-prefix-def
using *comp-nonempty* **witness** **by** *simp*
qed

lemma *checked-staged-security-with-actual-alpha-prefix-authenticated-candidate-opening-query-hit-from-prefixI-from-*
assumes *wf: staged-budget-wellformed budgets*
and *controlled: staged-adversary-controlled budgets A*
and *comp-nonempty: staged-composition-fri-roots data ≠ []*
and *i-bound: i < rounds*
and *builder:*
Some (data, attacker-state) ∈
set-dist
(execute (checked-staged-transcript-program A)
adversary-initial-state)
and *verifier:*
Some (result, final-state) ∈
set-dist
(execute verify-monad
(verifier-state-from-adversary attacker-state
(staged-proof-transcript data))))
and *final-lookup:*
fmlookup (HashMap final-state)
(QueryIndexChallenge i
(state-after-query-chunks
(staged-query-start-hash data) (staged-query-chunks data) i)) =
Some raw
and *trace-auth:*
partial-authenticated-table (staged-trace-root data)
*(scale * clength) trace-openings final-state*
and *comp-auth:*
partial-authenticated-table (hd (staged-composition-fri-roots data))
*(scale * clength) composition-openings final-state*
and *consistent:*
partial-query-openings-consistent trace-openings composition-openings
(staged-alphas data) (index (to-nat raw))
shows
checked-staged-security-with-actual-alpha-prefix-authenticated-candidate-opening-query-hit-from-prefix
A
(Some (((((alpha-prefix, alpha-prefix-state), data), attacker-state),
result), final-state)))
proof –
from *checked-staged-transcript-program-query-prefix-receive-final-lookup-support-with-alphas*
[OF wf controlled i-bound builder verifier final-lookup]
obtain *prefix prefix-state raw-state* **where** *prefix-receive:*
Some (((prefix, prefix-state), raw), raw-state) ∈
set-dist
(execute (checked-staged-query-prefix-receive-with-state A i)
adversary-initial-state)

```

and alphas-eq: sqp-alphas prefix = staged-alphas data
by blast
have component:
  checked-staged-security-with-actual-alpha-prefix-candidate-opening-query-hit-from-prefix A
    ((replicate rounds []) [i := trace-openings])
    ((replicate rounds []) [i := composition-openings])
    i
    (Some (((((alpha-prefix, alpha-prefix-state), data), attacker-state),
      result), final-state))
by (rule
  checked-staged-security-with-actual-alpha-prefix-candidate-opening-query-hit-from-prefixI-from-prefix-receive
    [OF i-bound builder prefix-receive alphas-eq consistent])
show ?thesis
by (rule
  checked-staged-security-with-actual-alpha-prefix-authenticated-candidate-opening-query-hit-from-prefixI
    [OF comp-nonempty trace-auth comp-auth component])
qed

lemma selected-query-chunks-match-verifier-lengths:
assumes i-bound: i < rounds
and len-prefix-chunks: length prefix-chunks = i
and len-suffix-chunks: length suffix-chunks = rounds - Suc i
and chunk-shape:
  verifier-query-round-chunk idx trace-roots composition-roots chunk
and prefix-rounds:
   $\bigwedge j. j < i \implies$ 
    verifier-query-round-chunk (prefix-query-idxs ! j)
    trace-roots composition-roots (prefix-chunks ! j)
and suffix-rounds:
   $\bigwedge j. j < \text{rounds} - \text{Suc } i \implies$ 
    verifier-query-round-chunk (suffix-query-idxs ! j)
    trace-roots composition-roots (suffix-chunks ! j)
shows
  length (prefix-chunks @ chunk # suffix-chunks) = rounds
   $\bigwedge j. j < \text{rounds} \implies$ 
    length ((prefix-chunks @ chunk # suffix-chunks) ! j) =
      verifier-query-round-transcript-length (query-idxs ! j)
      trace-roots composition-roots
proof -
  show length (prefix-chunks @ chunk # suffix-chunks) = rounds
    using len-prefix-chunks len-suffix-chunks i-bound by simp
next
  fix j
  assume j-bound: j < rounds
  let ?chunks = prefix-chunks @ chunk # suffix-chunks
  show
    length (?chunks ! j) =
      verifier-query-round-transcript-length (query-idxs ! j)
      trace-roots composition-roots

```

```

proof (cases  $j < i$ )
  case True
  have nth-eq:  $?chunks ! j = prefix-chunks ! j$ 
    using True len-prefix-chunks by (simp add: nth-append)
  have
    length (prefix-chunks !  $j$ ) =
      verifier-query-round-transcript-length
        (prefix-query-idxs !  $j$ ) trace-roots composition-roots
    by (rule verifier-query-round-chunk-length [OF prefix-rounds [OF True]])
  then show ?thesis
    using nth-eq
    by (simp add: verifier-query-round-transcript-length-index-irrelevant)
next
  case False
  note not-lt = False
  show ?thesis
  proof (cases  $j = i$ )
    case True
    have nth-eq:  $?chunks ! j = chunk$ 
      using True len-prefix-chunks by (simp add: nth-append)
    have
      length chunk =
        verifier-query-round-transcript-length idx trace-roots
          composition-roots
      by (rule verifier-query-round-chunk-length [OF chunk-shape])
    then show ?thesis
      using nth-eq
      by (simp add: verifier-query-round-transcript-length-index-irrelevant)
    next
    case False
    note not-eq = False
    let  $?k = j - Suc\ i$ 
    have ge:  $Suc\ i \leq j$ 
      using not-lt not-eq by linarith
    have k-bound:  $?k < rounds - Suc\ i$ 
      using ge j-bound by linarith
    have nth-eq:  $?chunks ! j = suffix-chunks ! ?k$ 
      using ge len-prefix-chunks by (simp add: nth-append)
    have
      length (suffix-chunks !  $?k$ ) =
        verifier-query-round-transcript-length
          (suffix-query-idxs !  $?k$ ) trace-roots composition-roots
      by (rule verifier-query-round-chunk-length
        [OF suffix-rounds [OF k-bound]])
    then show ?thesis
      using nth-eq
      by (simp add: verifier-query-round-transcript-length-index-irrelevant)
  qed
qed

```

qed

lemma *selected-query-chunks-eq-staged*:

assumes *builder*:

Some (data, attacker-state) ∈

set-dist

(execute (checked-staged-transcript-program A)

adversary-initial-state)

and *query-idxs-len*: *length query-idxs = rounds*

and *chunks-len*: *length query-chunks = rounds*

and *concat-eq*:

List.concat query-chunks @ trailing =

List.concat (staged-query-chunks data)

and *parser-chunk-len*:

$\bigwedge j. j < rounds \implies$

length (query-chunks ! j) =

verifier-query-round-transcript-length (query-idxs ! j)

(staged-trace-fri-roots data)

(staged-composition-fri-roots data)

shows *query-chunks = staged-query-chunks data ∧ trailing = []*

proof –

from *checked-staged-transcript-program-query-chunks-match-verifier-lengths*

[OF builder]

obtain *staged-query-idxs* **where** *staged-match*:

staged-query-chunks-match-verifier-lengths data staged-query-idxs

by *blast*

have *staged-match-query-idxs*:

staged-query-chunks-match-verifier-lengths data query-idxs

by *(rule staged-query-chunks-match-verifier-lengths-transfer*

[OF staged-match query-idxs-len])

show *?thesis*

by *(rule query-chunks-eq-staged-if-matching-lengths*

[OF chunks-len concat-eq parser-chunk-len

staged-match-query-idxs])

qed

lemma *selected-query-prefix-state-eq-staged*:

assumes *len-prefix-chunks*: *length prefix-chunks = i*

and *chunks-eq*:

prefix-chunks @ chunk # suffix-chunks = staged-query-chunks data

and *query-state-hash*: *PState query-state = staged-query-start-hash data*

shows

state-after-query-chunks (PState query-state) prefix-chunks i =

state-after-query-chunks

(staged-query-start-hash data) (staged-query-chunks data) i

proof –

have

state-after-query-chunks (staged-query-start-hash data)

(prefix-chunks @ chunk # suffix-chunks) i =

```

    state-after-query-chunks (staged-query-start-hash data)
    prefix-chunks i
  by (rule state-after-query-chunks-append-prefix[OF len-prefix-chunks])
  then show ?thesis
  using chunks-eq query-state-hash by simp
qed

```

definition *checked-staged-security-with-actual-alpha-prefix-header-authenticated-candidate-opening-query-hit*

$$:: ('f \text{ staged-adversary} \Rightarrow$$

$$((((('f \times 'f \text{ list} \times 'f \text{ list} \times 'f) \times$$

$$'f \text{ protocol-channel}) \times 'f \text{ staged-proof-data}) \times$$

$$'f \text{ protocol-channel}) \times \text{unit list}) \times$$

$$'f \text{ protocol-channel}) \text{ option} \Rightarrow \text{bool})$$

where

checked-staged-security-with-actual-alpha-prefix-header-authenticated-candidate-opening-query-hit
 $A \text{ out} \longleftrightarrow$
 (case out of
 None $\Rightarrow$ False
 | Some (((((alpha-prefix, alpha-prefix-state), data), attacker-state),
 result), final-state) $\Rightarrow$
 (let s =
 verifier-state-from-adversary attacker-state
 (staged-proof-transcript data)
 in $\exists i \text{ trace-openings composition-openings.}$
 (trace-openings, composition-openings) $\in$
 query-header-supported-partial-opening-witnesses s
 (staged-trace-root data)
 (staged-trace-fri-roots data)
 (staged-trace-final data)
 (staged-alphas data)
 (staged-degree data)
 (staged-composition-fri-roots data)
 (staged-composition-final data) $\wedge$
 checked-staged-security-with-actual-alpha-prefix-candidate-opening-query-hit A
 ((replicate rounds []) [i := trace-openings])
 ((replicate rounds []) [i := composition-openings])
 (staged-alphas data) i
 (Some (((((alpha-prefix, alpha-prefix-state), data),
 attacker-state), result), final-state))))))

lemma *checked-staged-security-with-actual-alpha-prefix-header-authenticated-candidate-opening-query-hitI:*

assumes *witness:*

(trace-openings, composition-openings) $\in$
 query-header-supported-partial-opening-witnesses
 (verifier-state-from-adversary attacker-state
 (staged-proof-transcript data))
 (staged-trace-root data)
 (staged-trace-fri-roots data)
 (staged-trace-final data)

```

      (staged-alphas data)
      (staged-degree data)
      (staged-composition-fri-roots data)
      (staged-composition-final data)
    and component:
      checked-staged-security-with-actual-alpha-prefix-candidate-opening-query-hit A
      ((replicate rounds []) [i := trace-openings])
      ((replicate rounds []) [i := composition-openings])
      (staged-alphas data) i
      (Some (((alpha-prefix, alpha-prefix-state), data), attacker-state),
        result), final-state))
  shows
    checked-staged-security-with-actual-alpha-prefix-header-authenticated-candidate-opening-query-hit
      A
      (Some (((alpha-prefix, alpha-prefix-state), data), attacker-state),
        result), final-state))
  proof -
    have ex:
      ∃ j trace-openings' composition-openings'.
        (trace-openings', composition-openings') ∈
          query-header-supported-partial-opening-witnesses
            (verifier-state-from-adversary attacker-state
              (staged-proof-transcript data))
            (staged-trace-root data)
            (staged-trace-fri-roots data)
            (staged-trace-final data)
            (staged-alphas data)
            (staged-degree data)
            (staged-composition-fri-roots data)
            (staged-composition-final data) ∧
        checked-staged-security-with-actual-alpha-prefix-candidate-opening-query-hit A
          ((replicate rounds []) [j := trace-openings'])
          ((replicate rounds []) [j := composition-openings'])
          (staged-alphas data) j
          (Some (((alpha-prefix, alpha-prefix-state), data), attacker-state),
            result), final-state))
    by (intro exI[of - i] exI[of - trace-openings]
      exI[of - composition-openings] conjI)
      (use witness component in simp-all)
  show ?thesis
  unfolding
    checked-staged-security-with-actual-alpha-prefix-header-authenticated-candidate-opening-query-hit-def
    Let-def
  using ex by simp
qed

lemma checked-staged-security-with-actual-alpha-prefix-header-authenticated-candidate-opening-query-hit-imp-compa
  assumes hit:
    checked-staged-security-with-actual-alpha-prefix-header-authenticated-candidate-opening-query-hit

```

```

    A out
  shows
    checked-staged-security-with-actual-alpha-prefix-composition-candidate-opening-query-hit
    A out
proof (cases out)
  case None
  then show ?thesis
    using hit
    unfolding
      checked-staged-security-with-actual-alpha-prefix-header-authenticated-candidate-opening-query-hit-def
    by simp
next
  case (Some packed)
  then obtain alpha-prefix alpha-prefix-state data attacker-state result
    final-state where out-eq:
    out =
      Some (((((alpha-prefix, alpha-prefix-state), data), attacker-state),
        result), final-state)
    by (cases packed, auto split: prod.splits)
  from hit[unfolded out-eq
    checked-staged-security-with-actual-alpha-prefix-header-authenticated-candidate-opening-query-hit-def
    Let-def]
  obtain i trace-openings composition-openings where component:
    checked-staged-security-with-actual-alpha-prefix-candidate-opening-query-hit A
    ((replicate rounds []) [i := trace-openings])
    ((replicate rounds []) [i := composition-openings])
    (staged-alphas data) i
    (Some (((((alpha-prefix, alpha-prefix-state), data), attacker-state),
      result), final-state))
  by auto
  show ?thesis
    unfolding out-eq
    by (rule
      checked-staged-security-with-actual-alpha-prefix-composition-candidate-opening-query-hitI
      [OF component])
qed

```

```

lemma checked-staged-security-with-actual-alpha-prefix-header-authenticated-candidate-opening-query-hitE:
assumes hit:
  checked-staged-security-with-actual-alpha-prefix-header-authenticated-candidate-opening-query-hit
  A
  (Some (((((alpha-prefix, alpha-prefix-state), data), attacker-state),
    result), final-state))
obtains i trace-openings composition-openings where
  (trace-openings, composition-openings) ∈
  query-header-supported-partial-opening-witnesses
  (verifier-state-from-adversary attacker-state
    (staged-proof-transcript data))
  (staged-trace-root data)

```



```

    (staged-trace-fri-roots data)
    (staged-trace-final data)
    (staged-alphas data)
    (staged-degree data)
    (staged-composition-fri-roots data)
    (staged-composition-final data)
checked-staged-security-with-actual-alpha-prefix-candidate-opening-query-hit
A
((replicate rounds []) [i := trace-openings])
((replicate rounds []) [i := composition-openings])
(staged-alphas data) i
(Some (((((alpha-prefix, alpha-prefix-state), data), attacker-state),
result), final-state))
proof –
from hit[unfolded
checked-staged-security-with-actual-alpha-prefix-header-authenticated-candidate-opening-query-hit-def
Let-def]
obtain i trace-openings composition-openings where witness:
(trace-openings, composition-openings) ∈
query-header-supported-partial-opening-witnesses
(verifier-state-from-adversary attacker-state
(staged-proof-transcript data))
(staged-trace-root data)
(staged-trace-fri-roots data)
(staged-trace-final data)
(staged-alphas data)
(staged-degree data)
(staged-composition-fri-roots data)
(staged-composition-final data)
and component:
checked-staged-security-with-actual-alpha-prefix-candidate-opening-query-hit
A
((replicate rounds []) [i := trace-openings])
((replicate rounds []) [i := composition-openings])
(staged-alphas data) i
(Some (((((alpha-prefix, alpha-prefix-state), data), attacker-state),
result), final-state))
by auto
show ?thesis
by (rule that[OF witness component])
qed

lemma checked-staged-security-with-actual-alpha-prefix-header-authenticated-candidate-opening-query-hit-le-compact
wp-event
(checked-staged-security-experiment-with-actual-alpha-prefix-data-state A)
(checked-staged-security-with-actual-alpha-prefix-header-authenticated-candidate-opening-query-hit
A)
adversary-initial-state ≤
wp-event

```

(checked-staged-security-experiment-with-actual-alpha-prefix-data-state A)
 (checked-staged-security-with-actual-alpha-prefix-composition-candidate-opening-query-hit
 A)
 adversary-initial-state
by (rule wp-event-mono)
 (rule
 checked-staged-security-with-actual-alpha-prefix-header-authenticated-candidate-opening-query-hit-imp-compact)

definition checked-staged-security-with-actual-alpha-prefix-header-authenticated-candidate-opening-query-hit-from-prefix
 $:: ('f \text{ staged-adversary} \Rightarrow$
 $(((((('f \times 'f \text{ list} \times 'f \text{ list} \times 'f) \times$
 $'f \text{ protocol-channel}) \times 'f \text{ staged-proof-data}) \times$
 $'f \text{ protocol-channel}) \times \text{unit list}) \times$
 $'f \text{ protocol-channel}) \text{ option} \Rightarrow \text{bool})$

where

checked-staged-security-with-actual-alpha-prefix-header-authenticated-candidate-opening-query-hit-from-prefix
 $A \text{ out} \longleftrightarrow$
 (case out of
 $\text{None} \Rightarrow \text{False}$
 $| \text{Some} (((((\alpha\text{-prefix}, \alpha\text{-prefix-state}), \text{data}), \text{attacker-state}),$
 $\text{result}), \text{final-state}) \Rightarrow$
 (let $s =$
 $\text{verifier-state-from-adversary attacker-state}$
 $(\text{staged-proof-transcript data})$
 in $\exists i \text{ trace-openings composition-openings.}$
 $(\text{trace-openings}, \text{composition-openings}) \in$
 $\text{query-header-supported-partial-opening-witnesses } s$
 $(\text{staged-trace-root data})$
 $(\text{staged-trace-fri-roots data})$
 $(\text{staged-trace-final data})$
 $(\text{staged-alphas data})$
 $(\text{staged-degree data})$
 $(\text{staged-composition-fri-roots data})$
 $(\text{staged-composition-final data}) \wedge$
 checked-staged-security-with-actual-alpha-prefix-candidate-opening-query-hit-from-prefix
 A
 $((\text{replicate rounds } []) [i := \text{trace-openings}])$
 $((\text{replicate rounds } []) [i := \text{composition-openings}])$
 i
 $(\text{Some} (((((\alpha\text{-prefix}, \alpha\text{-prefix-state}), \text{data}),$
 $\text{attacker-state}), \text{result}), \text{final-state}))))$

lemma checked-staged-security-with-actual-alpha-prefix-header-authenticated-candidate-opening-query-hit-from-prefix
assumes witness:

$(\text{trace-openings}, \text{composition-openings}) \in$
 $\text{query-header-supported-partial-opening-witnesses}$
 $(\text{verifier-state-from-adversary attacker-state}$
 $(\text{staged-proof-transcript data}))$
 $(\text{staged-trace-root data})$

```

      (staged-trace-fri-roots data)
      (staged-trace-final data)
      (staged-alphas data)
      (staged-degree data)
      (staged-composition-fri-roots data)
      (staged-composition-final data)
    and component:
      checked-staged-security-with-actual-alpha-prefix-candidate-opening-query-hit-from-prefix A
      ((replicate rounds []) [i := trace-openings])
      ((replicate rounds []) [i := composition-openings])
      i
      (Some (((alpha-prefix, alpha-prefix-state), data), attacker-state),
        result), final-state))
  shows
    checked-staged-security-with-actual-alpha-prefix-header-authenticated-candidate-opening-query-hit-from-prefix
      A
      (Some (((alpha-prefix, alpha-prefix-state), data), attacker-state),
        result), final-state))
  proof -
    have ex:
      ∃ j trace-openings' composition-openings'.
        (trace-openings', composition-openings') ∈
          query-header-supported-partial-opening-witnesses
            (verifier-state-from-adversary attacker-state
              (staged-proof-transcript data))
            (staged-trace-root data)
            (staged-trace-fri-roots data)
            (staged-trace-final data)
            (staged-alphas data)
            (staged-degree data)
            (staged-composition-fri-roots data)
            (staged-composition-final data) ∧
        checked-staged-security-with-actual-alpha-prefix-candidate-opening-query-hit-from-prefix A
          ((replicate rounds []) [j := trace-openings'])
          ((replicate rounds []) [j := composition-openings'])
      j
      (Some (((alpha-prefix, alpha-prefix-state), data), attacker-state),
        result), final-state))
    by (intro exI[of - i] exI[of - trace-openings]
      exI[of - composition-openings] conjI)
      (use witness component in simp-all)
  show ?thesis
  unfolding
    checked-staged-security-with-actual-alpha-prefix-header-authenticated-candidate-opening-query-hit-from-prefix-def
    Let-def
  using ex by simp
qed

```

lemma *checked-staged-security-with-actual-alpha-prefix-header-authenticated-candidate-opening-query-hit-from-prefix*

assumes *hit*:

checked-staged-security-with-actual-alpha-prefix-header-authenticated-candidate-opening-query-hit-from-prefix

A

(*Some* (((((*alpha-prefix*, *alpha-prefix-state*), *data*), *attacker-state*),
 result), *final-state*))

obtains *i trace-openings composition-openings* **where**

(*trace-openings*, *composition-openings*) ∈

query-header-supported-partial-opening-witnesses

(*verifier-state-from-adversary attacker-state*

 (*staged-proof-transcript data*))

 (*staged-trace-root data*)

 (*staged-trace-fri-roots data*)

 (*staged-trace-final data*)

 (*staged-alphas data*)

 (*staged-degree data*)

 (*staged-composition-fri-roots data*)

 (*staged-composition-final data*)

checked-staged-security-with-actual-alpha-prefix-candidate-opening-query-hit-from-prefix

A

((*replicate rounds* []) [*i := trace-openings*])

((*replicate rounds* []) [*i := composition-openings*])

i

(*Some* (((((*alpha-prefix*, *alpha-prefix-state*), *data*), *attacker-state*),
 result), *final-state*))

proof —

from *hit*[*unfolded*

checked-staged-security-with-actual-alpha-prefix-header-authenticated-candidate-opening-query-hit-from-prefix-def

Let-def]

obtain *i trace-openings composition-openings* **where** *witness*:

(*trace-openings*, *composition-openings*) ∈

query-header-supported-partial-opening-witnesses

(*verifier-state-from-adversary attacker-state*

 (*staged-proof-transcript data*))

 (*staged-trace-root data*)

 (*staged-trace-fri-roots data*)

 (*staged-trace-final data*)

 (*staged-alphas data*)

 (*staged-degree data*)

 (*staged-composition-fri-roots data*)

 (*staged-composition-final data*)

and *component*:

checked-staged-security-with-actual-alpha-prefix-candidate-opening-query-hit-from-prefix

A

((*replicate rounds* []) [*i := trace-openings*])

((*replicate rounds* []) [*i := composition-openings*])

i

(*Some* (((((*alpha-prefix*, *alpha-prefix-state*), *data*), *attacker-state*),
 result), *final-state*))

by *auto*

show *?thesis*
by (rule that[*OF witness component*])
qed

lemma *checked-staged-security-with-actual-alpha-prefix-header-authenticated-candidate-opening-query-hit-from-prefix*
assumes *hit*:
checked-staged-security-with-actual-alpha-prefix-header-authenticated-candidate-opening-query-hit-from-prefix
A
(Some (((((alpha-prefix, alpha-prefix-state), data), attacker-state), result), final-state))
obtains *i trace-openings composition-openings query-prefix query-prefix-state*
raw raw-state **where**
(trace-openings, composition-openings) ∈
query-header-supported-partial-opening-witnesses
(verifier-state-from-adversary attacker-state
(staged-proof-transcript data))
(staged-trace-root data)
(staged-trace-fri-roots data)
(staged-trace-final data)
(staged-alphas data)
(staged-degree data)
(staged-composition-fri-roots data)
(staged-composition-final data)
i < rounds
Some (data, attacker-state) ∈
set-dist
(execute (checked-staged-transcript-program A)
adversary-initial-state)
Some (((query-prefix, query-prefix-state), raw), raw-state) ∈
set-dist
(execute (checked-staged-query-prefix-receive-with-state A i)
adversary-initial-state)
checked-staged-query-prefix-dynamic-index-hit
(staged-query-prefix-candidate-opening-query-target-from-prefix
((replicate rounds []) [i := trace-openings])
((replicate rounds []) [i := composition-openings])
i)
(Some (((query-prefix, query-prefix-state), raw), raw-state))

proof –
from *checked-staged-security-with-actual-alpha-prefix-header-authenticated-candidate-opening-query-hit-from-prefix*
[OF hit]
obtain *i trace-openings composition-openings* **where** *witness*:
(trace-openings, composition-openings) ∈
query-header-supported-partial-opening-witnesses
(verifier-state-from-adversary attacker-state
(staged-proof-transcript data))
(staged-trace-root data)
(staged-trace-fri-roots data)
(staged-trace-final data)

```

    (staged-alphas data)
    (staged-degree data)
    (staged-composition-fri-roots data)
    (staged-composition-final data)
  and component:
    checked-staged-security-with-actual-alpha-prefix-candidate-opening-query-hit-from-prefix
    A
    ((replicate rounds []) [i := trace-openings])
    ((replicate rounds []) [i := composition-openings])
    i
    (Some (((((alpha-prefix, alpha-prefix-state), data), attacker-state),
      result), final-state))
  by blast
from checked-staged-security-with-actual-alpha-prefix-candidate-opening-query-hit-from-prefixE
  [OF component]
obtain query-prefix query-prefix-state raw raw-state where
  i-bound: i < rounds
  and builder:
    Some (data, attacker-state) ∈
      set-dist
      (execute (checked-staged-transcript-program A)
        adversary-initial-state)
  and prefix-receive:
    Some (((query-prefix, query-prefix-state), raw), raw-state) ∈
      set-dist
      (execute (checked-staged-query-prefix-receive-with-state A i)
        adversary-initial-state)
  and target-hit:
    checked-staged-query-prefix-dynamic-index-hit
    (staged-query-prefix-candidate-opening-query-target-from-prefix
      ((replicate rounds []) [i := trace-openings])
      ((replicate rounds []) [i := composition-openings])
      i)
    (Some (((query-prefix, query-prefix-state), raw), raw-state))
  by blast
show ?thesis
  by (rule that[OF witness i-bound builder prefix-receive target-hit])
qed

```

lemma *checked-staged-security-with-actual-alpha-prefix-composition-candidate-drift-query-hit-imp-header-authenticat*
assumes *wf: staged-budget-wellformed budgets*
and controlled: *staged-adversary-controlled budgets A*
and support:
*Some (((((prefix, prefix-state), data), attacker-state), result),
 final-state) ∈
 set-dist
 (execute
 (checked-staged-security-experiment-with-actual-alpha-prefix-data-state
 A)*

```

      adversary-initial-state)
and hit:
  checked-staged-security-with-actual-alpha-prefix-composition-candidate-drift-query-hit
  (Some (((((prefix, prefix-state), data), attacker-state), result),
    final-state))
shows
  checked-staged-security-with-actual-alpha-prefix-header-authenticated-candidate-opening-query-hit
  A
  (Some ((((((prefix, prefix-state), data), attacker-state), result),
    final-state))
proof -
let ?s =
  verifier-state-from-adversary attacker-state
  (staged-proof-transcript data)
show ?thesis
proof (rule
  checked-staged-security-with-actual-alpha-prefix-composition-candidate-drift-query-hit-supported-openingsE
  [OF wf controlled support hit])
fix s trace-table i raw trace-openings composition-openings
assume s-def: s = ?s
and i-bound: i < rounds
and final-lookup:
  fmlookup (HashMap final-state)
  (QueryIndexChallenge i
    (state-after-query-chunks
      (staged-query-start-hash data)
      (staged-query-chunks data) i)) =
    Some raw
and witness:
  (trace-openings, composition-openings) ∈
  query-header-supported-partial-opening-witnesses s
  (staged-trace-root data)
  (staged-trace-fri-roots data)
  (staged-trace-final data)
  (staged-alphas data)
  (staged-degree data)
  (staged-composition-fri-roots data)
  (staged-composition-final data)
and consistent:
  partial-query-openings-consistent trace-openings composition-openings
  (staged-alphas data) (index (to-nat raw))
have data-support:
  Some (((data, attacker-state), result), final-state) ∈
  set-dist
  (execute (checked-staged-security-experiment-with-data-state A)
    adversary-initial-state)
by (rule
  checked-staged-security-with-actual-alpha-prefix-support-imp-data-state-support
  [OF support])

```

```

from checked-staged-security-experiment-with-data-state-outcomeE
  [OF data-support]
obtain builder verifier where
  builder:
    Some (data, attacker-state) ∈
      set-dist
      (execute (checked-staged-transcript-program A)
        adversary-initial-state)
  and verifier:
    Some (result, final-state) ∈ set-dist (execute verify-monad ?s)
  by blast
from checked-staged-transcript-program-query-prefix-receive-final-lookup-support
  [OF wf controlled i-bound builder verifier final-lookup]
obtain query-prefix query-prefix-state raw-state where prefix-receive:
  Some (((query-prefix, query-prefix-state), raw), raw-state) ∈
    set-dist
    (execute (checked-staged-query-prefix-receive-with-state A i)
      adversary-initial-state)
  by blast
have component:
  checked-staged-security-with-actual-alpha-prefix-candidate-opening-query-hit
    A
    (((replicate rounds []) [i := trace-openings])
      (((replicate rounds []) [i := composition-openings])
        (staged-alphas data) i
          (Some (((((prefix, prefix-state), data), attacker-state), result),
            final-state)))
        by (rule
          checked-staged-security-with-actual-alpha-prefix-candidate-opening-query-hitI-from-prefix-receive
            [OF i-bound builder prefix-receive consistent])
        show ?thesis
          by (rule
            checked-staged-security-with-actual-alpha-prefix-header-authenticated-candidate-opening-query-hitI
              [OF - component])
            (use witness s-def in simp)
          qed
        qed
      )
    )
lemma checked-staged-security-with-actual-alpha-prefix-composition-candidate-drift-query-hit-le-header-authenticated
assumes wf: staged-budget-wellformed budgets
and controlled: staged-adversary-controlled budgets A
shows
  wp-event
    (checked-staged-security-experiment-with-actual-alpha-prefix-data-state A)
    checked-staged-security-with-actual-alpha-prefix-composition-candidate-drift-query-hit
    adversary-initial-state ≤
  wp-event
    (checked-staged-security-experiment-with-actual-alpha-prefix-data-state A)
    (checked-staged-security-with-actual-alpha-prefix-header-authenticated-candidate-opening-query-hit

```



```

    A)
    adversary-initial-state
proof (rule wp-event-mono-on-support)
fix out
assume support:
    out ∈
    set-dist
    (execute
    (checked-staged-security-experiment-with-actual-alpha-prefix-data-state
    A)
    adversary-initial-state)
and hit:
    checked-staged-security-with-actual-alpha-prefix-composition-candidate-drift-query-hit
    out
show
    checked-staged-security-with-actual-alpha-prefix-header-authenticated-candidate-opening-query-hit
    A out
proof (cases out)
case None
then show ?thesis
using hit
unfolding
    checked-staged-security-with-actual-alpha-prefix-composition-candidate-drift-query-hit-def
    checked-staged-security-with-actual-alpha-prefix-composition-candidate-drift-def
    checked-staged-security-with-actual-alpha-prefix-verifier-event-def
by simp
next
case (Some packed)
then obtain prefix prefix-state data attacker-state result final-state
where out-eq:
    out =
    Some (((((prefix, prefix-state), data), attacker-state), result),
    final-state)
by (cases packed, auto split: prod.splits)
show ?thesis
unfolding out-eq
by (rule
    checked-staged-security-with-actual-alpha-prefix-composition-candidate-drift-query-hit-imp-header-authenticated-c
    [OF wf controlled])
    (use support hit out-eq in simp-all)
qed
qed

lemma checked-staged-security-with-actual-alpha-prefix-composition-candidate-drift-query-hit-imp-header-authenticated-c
assumes wf: staged-budget-wellformed budgets
and controlled: staged-adversary-controlled budgets A
and support:
    Some (((((prefix, prefix-state), data), attacker-state), result),
    final-state) ∈

```

```

    set-dist
      (execute
        (checked-staged-security-experiment-with-actual-alpha-prefix-data-state
          A)
        adversary-initial-state)
  and hit:
    checked-staged-security-with-actual-alpha-prefix-composition-candidate-drift-query-hit
      (Some (((prefix, prefix-state), data), attacker-state), result),
      final-state))
  shows
    checked-staged-security-with-actual-alpha-prefix-header-authenticated-candidate-opening-query-hit-from-prefix
      A
      (Some (((((prefix, prefix-state), data), attacker-state), result),
        final-state))
  proof -
  let ?s =
    verifier-state-from-adversary attacker-state
    (staged-proof-transcript data)
  show ?thesis
  proof (rule
    checked-staged-security-with-actual-alpha-prefix-composition-candidate-drift-query-hit-supported-openingsE
      [OF wf controlled support hit])
  fix s trace-table i raw trace-openings composition-openings
  assume s-def: s = ?s
  and i-bound: i < rounds
  and final-lookup:
    fmlookup (HashMap final-state)
      (QueryIndexChallenge i
        (state-after-query-chunks
          (staged-query-start-hash data)
          (staged-query-chunks data) i)) =
        Some raw
  and witness:
    (trace-openings, composition-openings) ∈
      query-header-supported-partial-opening-witnesses s
      (staged-trace-root data)
      (staged-trace-fri-roots data)
      (staged-trace-final data)
      (staged-alphas data)
      (staged-degree data)
      (staged-composition-fri-roots data)
      (staged-composition-final data)
  and consistent:
    partial-query-openings-consistent trace-openings composition-openings
      (staged-alphas data) (index (to-nat raw))
  have data-support:
    Some (((data, attacker-state), result), final-state) ∈
      set-dist
        (execute (checked-staged-security-experiment-with-data-state A)

```

```

    adversary-initial-state)
  by (rule
    checked-staged-security-with-actual-alpha-prefix-support-imp-data-state-support
    [OF support])
from checked-staged-security-experiment-with-data-state-outcomeE
  [OF data-support]
obtain builder verifier where
  builder:
    Some (data, attacker-state) ∈
      set-dist
        (execute (checked-staged-transcript-program A)
          adversary-initial-state)
  and verifier:
    Some (result, final-state) ∈ set-dist (execute verify-monad ?s)
  by blast
from checked-staged-transcript-program-query-prefix-receive-final-lookup-support-with-alphas
  [OF wf controlled i-bound builder verifier final-lookup]
obtain query-prefix query-prefix-state raw-state where prefix-receive:
  Some (((query-prefix, query-prefix-state), raw), raw-state) ∈
    set-dist
      (execute (checked-staged-query-prefix-receive-with-state A i)
        adversary-initial-state)
  and alphas-eq: sgp-alphas query-prefix = staged-alphas data
  by blast
have component:
  checked-staged-security-with-actual-alpha-prefix-candidate-opening-query-hit-from-prefix
  A
  ((replicate rounds []) [i := trace-openings])
  ((replicate rounds []) [i := composition-openings])
  i
  (Some (((((prefix, prefix-state), data), attacker-state), result),
    final-state))
  by (rule
    checked-staged-security-with-actual-alpha-prefix-candidate-opening-query-hit-from-prefixI-from-prefix-receive
    [OF i-bound builder prefix-receive alphas-eq consistent])
show ?thesis
  by (rule
    checked-staged-security-with-actual-alpha-prefix-header-authenticated-candidate-opening-query-hit-from-prefixI
    [OF - component])
  (use witness s-def in simp)
qed
qed

lemma checked-staged-security-with-actual-alpha-prefix-composition-candidate-drift-query-hit-le-header-authenticated
  assumes wf: staged-budget-wellformed budgets
  and controlled: staged-adversary-controlled budgets A
  shows
    wp-event
      (checked-staged-security-experiment-with-actual-alpha-prefix-data-state A)

```

```

    checked-staged-security-with-actual-alpha-prefix-composition-candidate-drift-query-hit
    adversary-initial-state ≤
    wp-event
    (checked-staged-security-experiment-with-actual-alpha-prefix-data-state A)
    (checked-staged-security-with-actual-alpha-prefix-header-authenticated-candidate-opening-query-hit-from-prefix
    A)
    adversary-initial-state
proof (rule wp-event-mono-on-support)
fix out
assume support:
    out ∈
    set-dist
    (execute
    (checked-staged-security-experiment-with-actual-alpha-prefix-data-state
    A)
    adversary-initial-state)
and hit:
    checked-staged-security-with-actual-alpha-prefix-composition-candidate-drift-query-hit
    out
show
    checked-staged-security-with-actual-alpha-prefix-header-authenticated-candidate-opening-query-hit-from-prefix
    A out
proof (cases out)
case None
then show ?thesis
using hit
unfolding
    checked-staged-security-with-actual-alpha-prefix-composition-candidate-drift-query-hit-def
    checked-staged-security-with-actual-alpha-prefix-composition-candidate-drift-def
    checked-staged-security-with-actual-alpha-prefix-verifier-event-def
by simp
next
case (Some packed)
then obtain prefix prefix-state data attacker-state result final-state
where out-eq:
    out =
    Some (((((prefix, prefix-state), data), attacker-state), result),
    final-state)
by (cases packed, auto split: prod.splits)
show ?thesis
unfolding out-eq
by (rule
    checked-staged-security-with-actual-alpha-prefix-composition-candidate-drift-query-hit-imp-header-authenticated
    [OF wf controlled])
    (use support hit out-eq in simp-all)
qed
qed

```

lemma checked-staged-security-with-actual-alpha-prefix-composition-candidate-drift-query-hit-bound-from-header-auth

assumes *wf*: staged-budget-wellformed budgets
and *controlled*: staged-adversary-controlled budgets *A*
and *header-bound*:
 wp-event
 (*checked-staged-security-experiment-with-actual-alpha-prefix-data-state*
 A)
 (*checked-staged-security-with-actual-alpha-prefix-header-authenticated-candidate-opening-query-hit*
 A)
 adversary-initial-state $\leq$ (*Q*::*prob*)
shows
 wp-event
 (*checked-staged-security-experiment-with-actual-alpha-prefix-data-state A*)
 checked-staged-security-with-actual-alpha-prefix-composition-candidate-drift-query-hit
 adversary-initial-state $\leq$ *Q*
by (*rule order.trans*
 [*OF*
 checked-staged-security-with-actual-alpha-prefix-composition-candidate-drift-query-hit-le-header-authenticated-can
 header-bound])
 (*use wf controlled in simp-all*)

lemma *checked-staged-security-with-actual-alpha-prefix-composition-candidate-drift-query-hit-bound-from-header-authenticated-candidate-opening-query-hit*
assumes *wf*: staged-budget-wellformed budgets
and *controlled*: staged-adversary-controlled budgets *A*
and *header-bound*:
 wp-event
 (*checked-staged-security-experiment-with-actual-alpha-prefix-data-state*
 A)
 (*checked-staged-security-with-actual-alpha-prefix-header-authenticated-candidate-opening-query-hit-from-prefix*
 A)
 adversary-initial-state $\leq$ (*Q*::*prob*)
shows
 wp-event
 (*checked-staged-security-experiment-with-actual-alpha-prefix-data-state A*)
 checked-staged-security-with-actual-alpha-prefix-composition-candidate-drift-query-hit
 adversary-initial-state $\leq$ *Q*
by (*rule order.trans*
 [*OF*
 checked-staged-security-with-actual-alpha-prefix-composition-candidate-drift-query-hit-le-header-authenticated-can
 header-bound])
 (*use wf controlled in simp-all*)

lemma *checked-staged-security-with-actual-alpha-prefix-composition-candidate-drift-bound-from-header-authenticated-candidate-opening-query-hit*
assumes *wf*: staged-budget-wellformed budgets
and *controlled*: staged-adversary-controlled budgets *A*
and *header-bound*:
 wp-event
 (*checked-staged-security-experiment-with-actual-alpha-prefix-data-state*
 A)
 (*checked-staged-security-with-actual-alpha-prefix-header-authenticated-candidate-opening-query-hit*

$A)$
 $\text{adversary-initial-state} \leq (Q::\text{prob})$
and *without-bound*:
 wp-event
 $(\text{checked-staged-security-experiment-with-actual-alpha-prefix-data-state}$
 $A)$
 $\text{checked-staged-security-with-actual-alpha-prefix-composition-candidate-drift-without-query-hit}$
 $\text{adversary-initial-state} \leq (W::\text{prob})$
shows
 wp-event
 $(\text{checked-staged-security-experiment-with-actual-alpha-prefix-data-state } A)$
 $\text{checked-staged-security-with-actual-alpha-prefix-composition-candidate-drift}$
 $\text{adversary-initial-state} \leq Q + W$
proof (rule
 $\text{checked-staged-security-with-actual-alpha-prefix-composition-candidate-drift-bound-from-query-and-without-query})$
show
 wp-event
 $(\text{checked-staged-security-experiment-with-actual-alpha-prefix-data-state } A)$
 $\text{checked-staged-security-with-actual-alpha-prefix-composition-candidate-drift-query-hit}$
 $\text{adversary-initial-state} \leq Q$
by (rule
 $\text{checked-staged-security-with-actual-alpha-prefix-composition-candidate-drift-query-hit-bound-from-header-authen}$
 $[OF \text{ wf controlled header-bound}]$)
show
 wp-event
 $(\text{checked-staged-security-experiment-with-actual-alpha-prefix-data-state } A)$
 $\text{checked-staged-security-with-actual-alpha-prefix-composition-candidate-drift-without-query-hit}$
 $\text{adversary-initial-state} \leq W$
by (rule *without-bound*)
qed

lemma $\text{checked-staged-security-with-actual-alpha-prefix-composition-candidate-drift-query-hit-alpha-bad-trace-openings}$
assumes $\text{wf: staged-budget-wellformed budgets}$
and $\text{controlled: staged-adversary-controlled budgets } A$
and *support*:
 $\text{Some } (((((\text{prefix}, \text{prefix-state}), \text{data}), \text{attacker-state}), \text{result}),$
 $\text{final-state}) \in$
 set-dist
 $(\text{execute}$
 $(\text{checked-staged-security-experiment-with-actual-alpha-prefix-data-state}$
 $A)$
 $\text{adversary-initial-state})$
and *hit*:
 $\text{checked-staged-security-with-actual-alpha-prefix-composition-candidate-drift-query-hit}$
 $(\text{Some } (((((\text{prefix}, \text{prefix-state}), \text{data}), \text{attacker-state}), \text{result}),$
 $\text{final-state}))$
obtains $s \text{ trace-table witness-out query-idxs trace-openings}$
 $\text{witness-as witness-dg witness-composition-fri-roots witness-final}$
 witness-rest **where**

```

s = verifier-state-from-adversary attacker-state
      (staged-proof-transcript data)
witness-out ∈ set-dist (execute verify-monad s)
accepted-with-partial-trace-openings s witness-out
      (staged-trace-root data) query-idxs trace-openings
partial-trace-table-candidate trace-table trace-openings
verifier-header-transcript s
      (staged-trace-root data)
      (staged-trace-fri-roots data)
      (staged-trace-final data)
      witness-as witness-dg witness-composition-fri-roots witness-final
      witness-rest
witness-as = staged-alphas data
staged-alphas data ∈ composition-trace-bad-alpha-space trace-table
¬ staged-alphas data ∈
      alpha-prefix-union-bad-sets prefix-state
      composition-trace-bad-alpha-space (fst prefix)
proof –
  let ?s =
    verifier-state-from-adversary attacker-state
      (staged-proof-transcript data)
  have data-support:
    Some (((data, attacker-state), result), final-state) ∈
      set-dist
      (execute (checked-staged-security-experiment-with-data-state A)
        adversary-initial-state)
  by (rule
    checked-staged-security-with-actual-alpha-prefix-support-imp-data-state-support
    [OF support])
from checked-staged-security-experiment-with-data-state-outcomeE
  [OF data-support]
obtain builder where
  builder:
    Some (data, attacker-state) ∈
      set-dist
      (execute (checked-staged-transcript-program A)
        adversary-initial-state)
  by blast
have staged-header:
  verifier-header-transcript ?s
    (staged-trace-root data)
    (staged-trace-fri-roots data)
    (staged-trace-final data)
    (staged-alphas data)
    (staged-degree data)
    (staged-composition-fri-roots data)
    (staged-composition-final data)
    (List.concat (staged-query-chunks data))
  by (rule checked-staged-transcript-program-outcome-header-transcript

```

```

    [OF wf controlled builder])
show ?thesis
proof (rule
  checked-staged-security-with-actual-alpha-prefix-composition-candidate-drift-query-hit-supportedE
  [OF wf controlled support hit])
fix s trace-table
assume s-def:
  s = verifier-state-from-adversary attacker-state
    (staged-proof-transcript data)
and trace-candidate:
  trace-table ∈
    alpha-header-supported-partial-trace-table-candidates s
    (staged-trace-root data)
    (staged-trace-fri-roots data)
    (staged-trace-final data)
and trace-bad:
  staged-alphas data ∈ composition-trace-bad-alpha-space trace-table
and not-prefix:
  ¬ staged-alphas data ∈
    alpha-prefix-union-bad-sets prefix-state
    composition-trace-bad-alpha-space (fst prefix)
show ?thesis
proof (rule alpha-header-supported-partial-trace-table-candidatesE
  [OF trace-candidate])
fix witness-out query-idxs trace-openings witness-as witness-dg
  witness-composition-fri-roots witness-final witness-rest
assume witness-out:
  witness-out ∈ set-dist (execute verify-monad s)
and partial:
  accepted-with-partial-trace-openings s witness-out
    (staged-trace-root data) query-idxs trace-openings
and partial-candidate:
  partial-trace-table-candidate trace-table trace-openings
and header:
  verifier-header-transcript s
    (staged-trace-root data)
    (staged-trace-fri-roots data)
    (staged-trace-final data)
  witness-as witness-dg witness-composition-fri-roots
  witness-final witness-rest
have staged-header-s:
  verifier-header-transcript s
    (staged-trace-root data)
    (staged-trace-fri-roots data)
    (staged-trace-final data)
    (staged-alphas data)
    (staged-degree data)
    (staged-composition-fri-roots data)
    (staged-composition-final data)

```



```

      (List.concat (staged-query-chunks data))
    using staged-header by (simp add: s-def[symmetric])
  have as-eq: witness-as = staged-alphas data
    using verifier-header-transcript-unique[OF staged-header-s header]
    by simp
  show ?thesis
    by (rule that[OF s-def witness-out partial partial-candidate
      header as-eq trace-bad not-prefix])
qed
qed
qed

lemma checked-staged-security-with-actual-alpha-prefix-composition-candidate-drift-query-hit-bound-table-witnessE:
  assumes wf: staged-budget-wellformed budgets
  and controlled: staged-adversary-controlled budgets A
  and support:
    Some (((((prefix, prefix-state), data), attacker-state), result),
      final-state) ∈
    set-dist
      (execute
        (checked-staged-security-experiment-with-actual-alpha-prefix-data-state
          A)
        adversary-initial-state)
  and hit:
    checked-staged-security-with-actual-alpha-prefix-composition-candidate-drift-query-hit
      (Some (((((prefix, prefix-state), data), attacker-state), result),
        final-state))
  obtains s trace-table composition-table query-idxs f bound-trace-openings
    bound-composition-openings where
    s = verifier-state-from-adversary attacker-state
      (staged-proof-transcript data)
    composition-bad-context s (Some (result, final-state)) trace-table
      composition-table (staged-alphas data) query-idxs f
    common-denominator-degree-bounds f (staged-alphas data)
    random-combination-common-denominator-hides-violations f
      (staged-alphas data)
    partial-trace-table-candidate trace-table bound-trace-openings
    partial-composition-table-candidate composition-table
      bound-composition-openings
    trace-table-low-degree trace-table
    composition-table-low-degree maxDegree composition-table
    all-queries-consistent trace-table composition-table
      (staged-alphas data)
    ¬ staged-alphas data ∈
      alpha-prefix-union-bad-sets prefix-state
      composition-trace-bad-alpha-space (fst prefix)
  proof -
  from checked-staged-security-with-actual-alpha-prefix-composition-candidate-drift-query-hit-supportedE
    [OF wf controlled support hit]

```

obtain *s trace-table* **where**
s-def:
s = *verifier-state-from-adversary attacker-state*
(staged-proof-transcript data)
and *random-bad:*
composition-randomization-bad s (Some (result, final-state))
and *not-prefix:*
 $\neg$ *staged-alphas data* $\in$
alpha-prefix-union-bad-sets prefix-state
composition-trace-bad-alpha-space (fst prefix)
by *blast*
from *random-bad*
obtain *trace-table'* *composition-table as query-idxs f* **where** *bad-context:*
composition-bad-context s (Some (result, final-state)) trace-table'
composition-table as query-idxs f
and *bounds: common-denominator-degree-bounds f as*
and *hides:*
random-combination-common-denominator-hides-violations f as
unfolding *composition-randomization-bad-def* **by** *blast*
have *bound-as:*
accepted-with-bound-tables s (Some (result, final-state)) trace-table'
composition-table as query-idxs
using *bad-context unfolding composition-bad-context-def* **by** *blast*
from *bound-as* **obtain** *fr f-fri-roots f-final dg composition-fri-roots*
final rest **where** *header-as:*
verifier-header-transcript s fr f-fri-roots f-final as dg
composition-fri-roots final rest
unfolding *accepted-with-bound-tables-def* **by** *blast*
have *data-support:*
Some (((data, attacker-state), result), final-state) $\in$
set-dist
(execute (checked-staged-security-experiment-with-data-state A)
adversary-initial-state)
by *(rule*
checked-staged-security-with-actual-alpha-prefix-support-imp-data-state-support
[OF support])
from *checked-staged-security-experiment-with-data-state-outcomeE*
[OF data-support]
obtain *builder verifier* **where** *builder:*
Some (data, attacker-state) $\in$
set-dist
(execute (checked-staged-transcript-program A)
adversary-initial-state)
and *verifier:*
Some (result, final-state) $\in$
set-dist (execute verify-monad
(verifier-state-from-adversary attacker-state
(staged-proof-transcript data)))
by *blast*

```

have staged-header:
  verifier-header-transcript s
    (staged-trace-root data)
    (staged-trace-fri-roots data)
    (staged-trace-final data)
    (staged-alphas data)
    (staged-degree data)
    (staged-composition-fri-roots data)
    (staged-composition-final data)
    (List.concat (staged-query-chunks data))
unfolding s-def
by (rule checked-staged-transcript-program-outcome-header-transcript
      [OF wf controlled builder])
have as-eq: as = staged-alphas data
  using verifier-header-transcript-unique[OF header-as staged-header]
  by simp
have bad-context-staged:
  composition-bad-context s (Some (result, final-state)) trace-table'
    composition-table (staged-alphas data) query-idxs f
  using bad-context as-eq by simp
have bounds-staged:
  common-denominator-degree-bounds f (staged-alphas data)
  using bounds as-eq by simp
have hides-staged:
  random-combination-common-denominator-hides-violations f
    (staged-alphas data)
  using hides as-eq by simp
have bound:
  accepted-with-bound-tables s (Some (result, final-state)) trace-table'
    composition-table (staged-alphas data) query-idxs
  using bad-context-staged unfolding composition-bad-context-def by blast
from accepted-with-bound-tables-partial-opening-witnesses-Some[OF bound]
obtain fr f-fri-roots f-final dg composition-fri-roots final rest
  bound-trace-openings bound-composition-openings where
    trace-candidate:
      partial-trace-table-candidate trace-table' bound-trace-openings
  and comp-candidate:
      partial-composition-table-candidate composition-table
      bound-composition-openings
  by blast
have trace-low: trace-table-low-degree trace-table'
proof –
  have deg-f: degree f < clength
    and trace-table-eq: trace-table' = map (poly f) eval-domain
    using bad-context-staged unfolding composition-bad-context-def by blast+
  show ?thesis
    unfolding trace-table-low-degree-def
    using deg-f trace-table-eq by blast
qed

```

```

have comp-low:
  composition-table-low-degree maxDegree composition-table
  using bad-context-staged unfolding composition-bad-context-def by blast
have all-queries:
  all-queries-consistent trace-table' composition-table
  (staged-alphas data)
  using bad-context-staged unfolding composition-bad-context-def by blast
show ?thesis
  by (rule that[OF s-def bad-context-staged bounds-staged hides-staged
    trace-candidate comp-candidate trace-low comp-low all-queries
    not-prefix])
qed

lemma checked-staged-security-with-actual-alpha-prefix-composition-candidate-drift-query-hit-aligned-openingsE:
assumes wf: staged-budget-wellformed budgets
and controlled: staged-adversary-controlled budgets A
and support:
  Some (((((prefix, prefix-state), data), attacker-state), result),
    final-state) ∈
    set-dist
    (execute
      (checked-staged-security-experiment-with-actual-alpha-prefix-data-state
        A)
      adversary-initial-state)
and hit:
  checked-staged-security-with-actual-alpha-prefix-composition-candidate-drift-query-hit
    (Some (((((prefix, prefix-state), data), attacker-state), result),
      final-state))
obtains s trace-table composition-table query-idxs f i raw
  header-trace-openings header-composition-openings
  bound-trace-openings bound-composition-openings where
s = verifier-state-from-adversary attacker-state
  (staged-proof-transcript data)
  composition-bad-context s (Some (result, final-state)) trace-table
  composition-table (staged-alphas data) query-idxs f
i < rounds
  fmlookup (HashMap final-state)
  (QueryIndexChallenge i
    (state-after-query-chunks
      (staged-query-start-hash data)
      (staged-query-chunks data) i)) =
    Some raw
  index (to-nat raw) = query-idxs ! i
  (header-trace-openings, header-composition-openings) ∈
  query-header-supported-partial-opening-witnesses s
    (staged-trace-root data)
    (staged-trace-fri-roots data)
    (staged-trace-final data)
    (staged-alphas data)

```

```

    (staged-degree data)
    (staged-composition-fri-roots data)
    (staged-composition-final data)
  partial-query-openings-consistent header-trace-openings
    header-composition-openings (staged-alphas data) (query-idxs ! i)
  partial-trace-table-candidate trace-table bound-trace-openings
  partial-composition-table-candidate composition-table
    bound-composition-openings
  all-queries-consistent trace-table composition-table
    (staged-alphas data)
  ¬ staged-alphas data ∈
    alpha-prefix-union-bad-sets prefix-state
    composition-trace-bad-alpha-space (fst prefix)
proof –
  let ?s =
    verifier-state-from-adversary attacker-state
    (staged-proof-transcript data)
  show ?thesis
proof (rule
  checked-staged-security-with-actual-alpha-prefix-composition-candidate-drift-query-hit-supported-openingsE
  [OF wf controlled support hit])
  fix s' trace-table' i raw header-trace-openings
    header-composition-openings
  assume s'-def: s' = ?s
  and i-bound: i < rounds
  and final-lookup:
    fmlookup (HashMap final-state)
      (QueryIndexChallenge i
        (state-after-query-chunks
          (staged-query-start-hash data)
          (staged-query-chunks data) i)) =
        Some raw
  and header-witness:
    (header-trace-openings, header-composition-openings) ∈
      query-header-supported-partial-opening-witnesses s'
      (staged-trace-root data)
      (staged-trace-fri-roots data)
      (staged-trace-final data)
      (staged-alphas data)
      (staged-degree data)
      (staged-composition-fri-roots data)
      (staged-composition-final data)
  and header-consistent:
    partial-query-openings-consistent header-trace-openings
      header-composition-openings (staged-alphas data)
      (index (to-nat raw))
  show ?thesis
proof (rule
  checked-staged-security-with-actual-alpha-prefix-composition-candidate-drift-query-hit-bound-table-witnessE

```

```

    [OF wf controlled support hit])
fix s trace-table composition-table query-idxs f bound-trace-openings
    bound-composition-openings
assume s-def: s = ?s
and bad-context:
    composition-bad-context s (Some (result, final-state)) trace-table
    composition-table (staged-alphas data) query-idxs f
and trace-candidate:
    partial-trace-table-candidate trace-table bound-trace-openings
and comp-candidate:
    partial-composition-table-candidate composition-table
    bound-composition-openings
and all-queries:
    all-queries-consistent trace-table composition-table
    (staged-alphas data)
and not-prefix:
    ¬ staged-alphas data ∈
    alpha-prefix-union-bad-sets prefix-state
    composition-trace-bad-alpha-space (fst prefix)
have data-support:
    Some (((data, attacker-state), result), final-state) ∈
    set-dist
    (execute (checked-staged-security-experiment-with-data-state A)
    adversary-initial-state)
by (rule
    checked-staged-security-with-actual-alpha-prefix-support-imp-data-state-support
    [OF support])
have bound:
    accepted-with-bound-tables s (Some (result, final-state)) trace-table
    composition-table (staged-alphas data) query-idxs
using bad-context unfolding composition-bad-context-def by blast
have shape:
    accepted-transcript-shape s (Some (result, final-state))
    (staged-alphas data) query-idxs
using bound unfolding accepted-with-bound-tables-def
    accepted-with-tables-def by simp
have shape-s:
    accepted-transcript-shape ?s (Some (result, final-state))
    (staged-alphas data) query-idxs
using shape s-def by simp
from checked-staged-security-with-data-state-accepted-shape-query-chunks
    [OF wf controlled data-support shape-s]
obtain raw-idxs where
    len-raw: length raw-idxs = rounds
and query-idxs-eq:
    query-idxs = map (λraw. index (to-nat raw)) raw-idxs
and lookup:
     $\bigwedge j. j < rounds \implies$ 
    fmllookup (HashMap final-state)

```

```

      (QueryIndexChallenge j
        (state-after-query-chunks
          (staged-query-start-hash data)
          (staged-query-chunks data) j)) =
      Some (raw-idxs ! j)
    by blast
  have raw-eq: raw = raw-idxs ! i
    using final-lookup lookup[OF i-bound] by simp
  have idx-eq: index (to-nat raw) = query-idxs ! i
    using raw-eq query-idxs-eq len-raw i-bound by simp
  have header-witness-s:
    (header-trace-openings, header-composition-openings) ∈
      query-header-supported-partial-opening-witnesses s
      (staged-trace-root data)
      (staged-trace-fri-roots data)
      (staged-trace-final data)
      (staged-alphas data)
      (staged-degree data)
      (staged-composition-fri-roots data)
      (staged-composition-final data)
    using header-witness s'-def s-def by simp
  have header-consistent-idx:
    partial-query-openings-consistent header-trace-openings
      header-composition-openings (staged-alphas data) (query-idxs ! i)
    using header-consistent idx-eq by simp
  show ?thesis
    by (rule that[OF s-def bad-context i-bound final-lookup idx-eq
      header-witness-s header-consistent-idx trace-candidate
      comp-candidate all-queries not-prefix])
qed
qed
qed

lemma checked-staged-security-with-actual-alpha-prefix-composition-candidate-drift-query-hit-authenticated-witnessE
assumes wf: staged-budget-wellformed budgets
and controlled: staged-adversary-controlled budgets A
and support:
  Some (((((prefix, prefix-state), data), attacker-state), result),
    final-state) ∈
  set-dist
    (execute
      (checked-staged-security-experiment-with-actual-alpha-prefix-data-state
        A)
      adversary-initial-state)
and hit:
  checked-staged-security-with-actual-alpha-prefix-composition-candidate-drift-query-hit
    (Some (((((prefix, prefix-state), data), attacker-state), result),
      final-state))
obtains s trace-table composition-table query-idxs f i raw

```

```

    header-trace-openings header-composition-openings
    bound-trace-openings bound-composition-openings
    witness-result witness-final-state witness-rest where
s = verifier-state-from-adversary attacker-state
  (staged-proof-transcript data)
composition-bad-context s (Some (result, final-state)) trace-table
  composition-table (staged-alphas data) query-idxs f
i < rounds
index (to-nat raw) = query-idxs ! i
Some (witness-result, witness-final-state) ∈
  set-dist (execute verify-monad s)
partial-authenticated-table (staged-trace-root data)
  (scale * clength) header-trace-openings witness-final-state
partial-authenticated-table (hd (staged-composition-fri-roots data))
  (scale * clength) header-composition-openings witness-final-state
verifier-header-transcript s
  (staged-trace-root data)
  (staged-trace-fri-roots data)
  (staged-trace-final data)
  (staged-alphas data)
  (staged-degree data)
  (staged-composition-fri-roots data)
  (staged-composition-final data)
  witness-rest
partial-query-openings-consistent header-trace-openings
  header-composition-openings (staged-alphas data) (query-idxs ! i)
partial-trace-table-candidate trace-table bound-trace-openings
partial-composition-table-candidate composition-table
  bound-composition-openings
all-queries-consistent trace-table composition-table
  (staged-alphas data)
¬ staged-alphas data ∈
  alpha-prefix-union-bad-sets prefix-state
  composition-trace-bad-alpha-space (fst prefix)
proof –
show ?thesis
proof (rule
  checked-staged-security-with-actual-alpha-prefix-composition-candidate-drift-query-hit-aligned-openingsE
  [OF wf controlled support hit])
fix s trace-table composition-table query-idxs f i raw
  header-trace-openings header-composition-openings
  bound-trace-openings bound-composition-openings
assume s-def:
  s = verifier-state-from-adversary attacker-state
    (staged-proof-transcript data)
and bad-context:
  composition-bad-context s (Some (result, final-state))
    trace-table composition-table (staged-alphas data) query-idxs f
and i-bound: i < rounds

```



```

and idx-eq: index (to-nat raw) = query-idxs ! i
and header-witness:
  (header-trace-openings, header-composition-openings) ∈
    query-header-supported-partial-opening-witnesses s
    (staged-trace-root data)
    (staged-trace-fri-roots data)
    (staged-trace-final data)
    (staged-alphas data)
    (staged-degree data)
    (staged-composition-fri-roots data)
    (staged-composition-final data)
and header-consistent:
  partial-query-openings-consistent header-trace-openings
  header-composition-openings (staged-alphas data) (query-idxs ! i)
and trace-candidate:
  partial-trace-table-candidate trace-table bound-trace-openings
and comp-candidate:
  partial-composition-table-candidate composition-table
  bound-composition-openings
and all-queries:
  all-queries-consistent trace-table composition-table
  (staged-alphas data)
and not-prefix:
  ¬ staged-alphas data ∈
    alpha-prefix-union-bad-sets prefix-state
    composition-trace-bad-alpha-space (fst prefix)
show ?thesis
proof (rule query-header-supported-partial-opening-witnessesE
  [OF header-witness])
fix witness-result witness-final-state witness-rest
assume witness-out:
  Some (witness-result, witness-final-state) ∈
    set-dist (execute verify-monad s)
and trace-auth:
  partial-authenticated-table (staged-trace-root data)
  (scale * clength) header-trace-openings witness-final-state
and comp-auth:
  partial-authenticated-table (hd (staged-composition-fri-roots data))
  (scale * clength) header-composition-openings
  witness-final-state
and header:
  verifier-header-transcript s
  (staged-trace-root data)
  (staged-trace-fri-roots data)
  (staged-trace-final data)
  (staged-alphas data)
  (staged-degree data)
  (staged-composition-fri-roots data)
  (staged-composition-final data)

```

```

    witness-rest
  show ?thesis
  by (rule that[OF s-def bad-context i-bound idx-eq witness-out
    trace-auth comp-auth header header-consistent trace-candidate
    comp-candidate all-queries not-prefix])
qed
qed
qed

lemma checked-staged-security-with-actual-alpha-prefix-composition-candidate-drift-query-hit-current-openingsE:
  assumes wf: staged-budget-wellformed budgets
  and controlled: staged-adversary-controlled budgets A
  and support:
    Some (((((prefix, prefix-state), data), attacker-state), result),
      final-state) ∈
    set-dist
    (execute
      (checked-staged-security-experiment-with-actual-alpha-prefix-data-state
        A)
      adversary-initial-state)
  and hit:
    checked-staged-security-with-actual-alpha-prefix-composition-candidate-drift-query-hit
    (Some (((((prefix, prefix-state), data), attacker-state), result),
      final-state))
  obtains s trace-table composition-table query-idxs f i raw
    trace-openings composition-openings
    trace-openingss composition-openingss where
    s = verifier-state-from-adversary attacker-state
      (staged-proof-transcript data)
    composition-bad-context s (Some (result, final-state)) trace-table
    composition-table (staged-alphas data) query-idxs f
    i < rounds
    fmlookup (HashMap final-state)
      (QueryIndexChallenge i
        (state-after-query-chunks
          (staged-query-start-hash data)
          (staged-query-chunks data) i)) =
      Some raw
    index (to-nat raw) = query-idxs ! i
    staged-composition-fri-roots data ≠ []
    partial-authenticated-table (staged-trace-root data)
      (scale * clength) trace-openings final-state
    partial-authenticated-table (hd (staged-composition-fri-roots data))
      (scale * clength) composition-openings final-state
    partial-query-openings-consistent trace-openings composition-openings
      (staged-alphas data) (query-idxs ! i)
    partial-trace-table-candidate trace-table trace-openingss
    partial-composition-table-candidate composition-table
      composition-openingss

```

```

    trace-openings = trace-openingsss ! i
    composition-openings = composition-openingsss ! i
    all-queries-consistent trace-table composition-table
      (staged-alphas data)
    ¬ staged-alphas data ∈
      alpha-prefix-union-bad-sets prefix-state
      composition-trace-bad-alpha-space (fst prefix)
proof –
  show ?thesis
  proof (rule
    checked-staged-security-with-actual-alpha-prefix-composition-candidate-drift-query-hit-aligned-openingsE
    [OF wf controlled support hit])
  fix s trace-table composition-table query-idxs f i raw
    header-trace-openings header-composition-openings
    bound-trace-openings bound-composition-openings
  assume s-def:
    s = verifier-state-from-adversary attacker-state
      (staged-proof-transcript data)
  and bad-context:
    composition-bad-context s (Some (result, final-state))
    trace-table composition-table (staged-alphas data) query-idxs f
  and i-bound: i < rounds
  and final-lookup:
    fmlookup (HashMap final-state)
      (QueryIndexChallenge i
        (state-after-query-chunks
          (staged-query-start-hash data)
          (staged-query-chunks data) i)) =
      Some raw
  and idx-eq: index (to-nat raw) = query-idxs ! i
  and all-queries:
    all-queries-consistent trace-table composition-table
      (staged-alphas data)
  and not-prefix:
    ¬ staged-alphas data ∈
      alpha-prefix-union-bad-sets prefix-state
      composition-trace-bad-alpha-space (fst prefix)
  let ?s =
    verifier-state-from-adversary attacker-state
      (staged-proof-transcript data)
  have data-support:
    Some (((data, attacker-state), result), final-state) ∈
      set-dist
        (execute (checked-staged-security-experiment-with-data-state A)
          adversary-initial-state)
  by (rule
    checked-staged-security-with-actual-alpha-prefix-support-imp-data-state-support
    [OF support])
  from checked-staged-security-experiment-with-data-state-outcomeE

```

```

[OF data-support]
obtain builder verifier where
  builder:
    Some (data, attacker-state) ∈
      set-dist
        (execute (checked-staged-transcript-program A)
          adversary-initial-state)
  and verifier:
    Some (result, final-state) ∈ set-dist (execute verify-monad ?s)
  by blast
have staged-header:
  verifier-header-transcript ?s
    (staged-trace-root data)
    (staged-trace-fri-roots data)
    (staged-trace-final data)
    (staged-alphas data)
    (staged-degree data)
    (staged-composition-fri-roots data)
    (staged-composition-final data)
    (List.concat (staged-query-chunks data))
  by (rule checked-staged-transcript-program-outcome-header-transcript
    [OF wf controlled builder])
have bound:
  accepted-with-bound-tables s (Some (result, final-state))
    trace-table composition-table (staged-alphas data) query-idxs
  using bad-context unfolding composition-bad-context-def by blast
show ?thesis
proof (rule accepted-with-bound-tables-partial-opening-witnesses-Some
  [OF bound])
  fix fr f-fri-roots f-final dg composition-fri-roots final rest
    trace-openingss composition-openingss
  assume header:
    verifier-header-transcript s fr f-fri-roots f-final
      (staged-alphas data) dg composition-fri-roots final rest
  and comp-nonempty: composition-fri-roots ≠ []
  and trace-partial:
    accepted-with-partial-trace-openings s
      (Some (result, final-state)) fr query-idxs trace-openingss
  and comp-partial:
    accepted-with-partial-composition-openings s
      (Some (result, final-state)) (hd composition-fri-roots)
      query-idxs composition-openingss
  and trace-candidate:
    partial-trace-table-candidate trace-table trace-openingss
  and comp-candidate:
    partial-composition-table-candidate composition-table
      composition-openingss
have header-s:
  verifier-header-transcript ?s fr f-fri-roots f-final

```

```

    (staged-alphas data) dg composition-fri-roots final rest
using header s-def by simp
have header-eq:
  fr = staged-trace-root data ∧
  f-fri-roots = staged-trace-fri-roots data ∧
  f-final = staged-trace-final data ∧
  dg = staged-degree data ∧
  composition-fri-roots = staged-composition-fri-roots data ∧
  final = staged-composition-final data ∧
  rest = List.concat (staged-query-chunks data)
using verifier-header-transcript-unique[OF staged-header header-s]
by simp
have query-len: length query-idxs = rounds
  by (rule accepted-with-bound-tables-shapes(4)[OF bound])
have idx-mem: query-idxs ! i ∈ set query-idxs
  by (rule nth-mem) (use i-bound query-len in simp)
have idx-sample: query-idxs ! i ∈ query-sample-space
  by (rule accepted-with-bound-tables-query-sample-space
    [OF bound idx-mem])
have consistent-round:
  partial-query-round-consistent trace-openingss
    composition-openingss (staged-alphas data) i (query-idxs ! i)
  by (rule partial-query-round-consistent-from-all-queries-consistent
    [OF trace-partial comp-partial trace-candidate comp-candidate
      i-bound idx-sample all-queries])
have consistent:
  partial-query-openings-consistent (trace-openingss ! i)
    (composition-openingss ! i) (staged-alphas data) (query-idxs ! i)
  using consistent-round
  unfolding partial-query-round-consistent-def
    partial-query-openings-consistent-def
  by simp
have trace-auth:
  partial-authenticated-table (staged-trace-root data)
    (scale * clength) (trace-openingss ! i) final-state
  using trace-partial header-eq i-bound
  unfolding accepted-with-partial-trace-openings-def by blast
have comp-auth:
  partial-authenticated-table
    (hd (staged-composition-fri-roots data)) (scale * clength)
    (composition-openingss ! i) final-state
  using comp-partial header-eq i-bound
  unfolding accepted-with-partial-composition-openings-def by simp
have comp-nonempty-staged:
  staged-composition-fri-roots data ≠ []
  using comp-nonempty header-eq by simp
show ?thesis
  by (rule that[OF s-def bad-context i-bound final-lookup idx-eq
    comp-nonempty-staged trace-auth comp-auth consistent

```

```

    trace-candidate comp-candidate refl refl all-queries not-prefix])
qed
qed
qed

lemma checked-staged-security-with-actual-alpha-prefix-composition-candidate-drift-query-hit-imp-authenticated-candidate-drift-query-hit
assumes wf: staged-budget-wellformed budgets
and controlled: staged-adversary-controlled budgets A
and support:
  Some (((((prefix, prefix-state), data), attacker-state), result),
    final-state) ∈
  set-dist
    (execute
      (checked-staged-security-experiment-with-actual-alpha-prefix-data-state
        A)
      adversary-initial-state)
and hit:
  checked-staged-security-with-actual-alpha-prefix-composition-candidate-drift-query-hit
    (Some (((((prefix, prefix-state), data), attacker-state), result),
      final-state))
shows
  checked-staged-security-with-actual-alpha-prefix-authenticated-candidate-opening-query-hit
    A
    (Some (((((prefix, prefix-state), data), attacker-state), result),
      final-state))
proof —
let ?s =
  verifier-state-from-adversary attacker-state
    (staged-proof-transcript data)
have data-support:
  Some (((data, attacker-state), result), final-state) ∈
  set-dist
    (execute (checked-staged-security-experiment-with-data-state A)
      adversary-initial-state)
by (rule
  checked-staged-security-with-actual-alpha-prefix-support-imp-data-state-support
    [OF support])
from checked-staged-security-experiment-with-data-state-outcomeE
  [OF data-support]
obtain builder verifier where
  builder:
    Some (data, attacker-state) ∈
    set-dist
      (execute (checked-staged-transcript-program A)
        adversary-initial-state)
and verifier:
  Some (result, final-state) ∈ set-dist (execute verify-monad ?s)
by blast
show ?thesis

```

proof (rule
checked-staged-security-with-actual-alpha-prefix-composition-candidate-drift-query-hit-current-openingsE
 [OF wf controlled support hit])
fix *s* trace-table composition-table query-idxs *f* *i* raw trace-openings
 composition-openings trace-openingss composition-openingsss
assume *i-bound*: *i* < rounds
and *final-lookup*:
fmlookup (HashMap *final-state*)
 (QueryIndexChallenge *i*
 (state-after-query-chunks
 (staged-query-start-hash *data*)
 (staged-query-chunks *data*) *i*)) =
 Some raw
and *idx-eq*: index (to-nat raw) = query-idxs ! *i*
and *comp-nonempty*: staged-composition-fri-roots *data* ≠ []
and *trace-auth*:
 partial-authenticated-table (staged-trace-root *data*)
 (scale * clength) trace-openings *final-state*
and *comp-auth*:
 partial-authenticated-table
 (hd (staged-composition-fri-roots *data*)) (scale * clength)
 composition-openings *final-state*
and *consistent*:
 partial-query-openings-consistent trace-openings
 composition-openings (staged-alphas *data*) (query-idxs ! *i*)
have *consistent-raw*:
 partial-query-openings-consistent trace-openings composition-openings
 (staged-alphas *data*) (index (to-nat raw))
using *consistent idx-eq* **by** *simp*
show ?thesis
by (rule
checked-staged-security-with-actual-alpha-prefix-authenticated-candidate-opening-query-hitI-from-current-openingsE
 [OF wf controlled comp-nonempty i-bound builder verifier
 final-lookup trace-auth comp-auth consistent-raw])
qed
qed

lemma *checked-staged-security-with-actual-alpha-prefix-composition-candidate-drift-query-hit-le-authenticated-candidate-opening-query-hit*
assumes *wf*: staged-budget-wellformed budgets
and *controlled*: staged-adversary-controlled budgets *A*
shows
wp-event
 (checked-staged-security-experiment-with-actual-alpha-prefix-data-state *A*)
 checked-staged-security-with-actual-alpha-prefix-composition-candidate-drift-query-hit
 adversary-initial-state ≤
wp-event
 (checked-staged-security-experiment-with-actual-alpha-prefix-data-state *A*)
 (checked-staged-security-with-actual-alpha-prefix-authenticated-candidate-opening-query-hit
A)

```

      adversary-initial-state
proof (rule wp-event-mono-on-support)
  fix out
  assume support:
    out ∈
      set-dist
      (execute
        (checked-staged-security-experiment-with-actual-alpha-prefix-data-state
          A)
        adversary-initial-state)
  and hit:
    checked-staged-security-with-actual-alpha-prefix-composition-candidate-drift-query-hit
    out
  show
    checked-staged-security-with-actual-alpha-prefix-authenticated-candidate-opening-query-hit
    A out
proof (cases out)
  case None
  then show ?thesis
    using hit
    unfolding
      checked-staged-security-with-actual-alpha-prefix-composition-candidate-drift-query-hit-def
      checked-staged-security-with-actual-alpha-prefix-composition-candidate-drift-def
      checked-staged-security-with-actual-alpha-prefix-verifier-event-def
    by simp
  next
  case (Some packed)
  then obtain prefix prefix-state data attacker-state result final-state
  where out-eq:
    out =
      Some (((((prefix, prefix-state), data), attacker-state), result),
        final-state)
  by (cases packed, auto split: prod.splits)
  show ?thesis
    unfolding out-eq
    by (rule
      checked-staged-security-with-actual-alpha-prefix-composition-candidate-drift-query-hit-imp-authenticated-candid
      [OF wf controlled])
    (use support hit out-eq in simp-all)
  qed
qed

lemma checked-staged-security-with-actual-alpha-prefix-composition-candidate-drift-query-hit-bound-from-authenticat
assumes wf: staged-budget-wellformed budgets
and controlled: staged-adversary-controlled budgets A
and authenticated-bound:
  wp-event
  (checked-staged-security-experiment-with-actual-alpha-prefix-data-state
    A)

```


(checked-staged-security-with-actual-alpha-prefix-authenticated-candidate-opening-query-hit
A)
adversary-initial-state $\leq$ (*Q::prob*)
shows
wp-event
(checked-staged-security-experiment-with-actual-alpha-prefix-data-state A)
checked-staged-security-with-actual-alpha-prefix-composition-candidate-drift-query-hit
adversary-initial-state $\leq$ *Q*
by (*rule order.trans*
[*OF*
checked-staged-security-with-actual-alpha-prefix-composition-candidate-drift-query-hit-le-authenticated-candidate-
authenticated-bound])
(use wf controlled in simp-all)

lemma *checked-staged-security-with-actual-alpha-prefix-composition-candidate-drift-query-hit-imp-authenticated-cand*
assumes *wf: staged-budget-wellformed budgets*
and *controlled: staged-adversary-controlled budgets A*
and *support:*
Some (((((prefix, prefix-state), data), attacker-state), result),
final-state) ∈
set-dist
(execute
(checked-staged-security-experiment-with-actual-alpha-prefix-data-state
A)
adversary-initial-state)
and *hit:*
checked-staged-security-with-actual-alpha-prefix-composition-candidate-drift-query-hit
(Some (((((prefix, prefix-state), data), attacker-state), result),
final-state))
shows
checked-staged-security-with-actual-alpha-prefix-authenticated-candidate-opening-query-hit-from-prefix
A
(Some (((((prefix, prefix-state), data), attacker-state), result),
final-state))
proof –
let *?s =*
verifier-state-from-adversary attacker-state
(staged-proof-transcript data)
have *data-support:*
Some (((data, attacker-state), result), final-state) ∈
set-dist
(execute (checked-staged-security-experiment-with-data-state A)
adversary-initial-state)
by (*rule*
checked-staged-security-with-actual-alpha-prefix-support-imp-data-state-support
[*OF support*])
from *checked-staged-security-experiment-with-data-state-outcomeE*
[*OF data-support*]
obtain *builder verifier where*

```

builder:
  Some (data, attacker-state) ∈
    set-dist
      (execute (checked-staged-transcript-program A)
        adversary-initial-state)
and verifier:
  Some (result, final-state) ∈ set-dist (execute verify-monad ?s)
by blast
show ?thesis
proof (rule
  checked-staged-security-with-actual-alpha-prefix-composition-candidate-drift-query-hit-current-openingsE
  [OF wf controlled support hit])
fix s trace-table composition-table query-idxs f i raw trace-openings
  composition-openings trace-openingss composition-openingsss
assume i-bound: i < rounds
and final-lookup:
  fmlookup (HashMap final-state)
    (QueryIndexChallenge i
      (state-after-query-chunks
        (staged-query-start-hash data)
        (staged-query-chunks data) i)) =
    Some raw
and idx-eq: index (to-nat raw) = query-idxs ! i
and comp-nonempty: staged-composition-fri-roots data ≠ []
and trace-auth:
  partial-authenticated-table (staged-trace-root data)
    (scale * clength) trace-openings final-state
and comp-auth:
  partial-authenticated-table
    (hd (staged-composition-fri-roots data)) (scale * clength)
    composition-openings final-state
and consistent:
  partial-query-openings-consistent trace-openings
    composition-openings (staged-alphas data) (query-idxs ! i)
have consistent-raw:
  partial-query-openings-consistent trace-openings composition-openings
    (staged-alphas data) (index (to-nat raw))
using consistent idx-eq by simp
show ?thesis
by (rule
  checked-staged-security-with-actual-alpha-prefix-authenticated-candidate-opening-query-hit-from-prefixI-from-cu
  [OF wf controlled comp-nonempty i-bound builder verifier
    final-lookup trace-auth comp-auth consistent-raw])
qed
qed

lemma checked-staged-security-with-actual-alpha-prefix-composition-candidate-drift-query-hit-le-authenticated-candid
  assumes wf: staged-budget-wellformed budgets
  and controlled: staged-adversary-controlled budgets A

```

shows

wp-event

(*checked-staged-security-experiment-with-actual-alpha-prefix-data-state A*)

checked-staged-security-with-actual-alpha-prefix-composition-candidate-drift-query-hit

adversary-initial-state $\leq$

wp-event

(*checked-staged-security-experiment-with-actual-alpha-prefix-data-state A*)

(*checked-staged-security-with-actual-alpha-prefix-authenticated-candidate-opening-query-hit-from-prefix A*)

adversary-initial-state

proof (*rule wp-event-mono-on-support*)

fix *out*

assume *support*:

out $\in$

set-dist

(*execute*

(*checked-staged-security-experiment-with-actual-alpha-prefix-data-state*

A)

adversary-initial-state)

and *hit*:

checked-staged-security-with-actual-alpha-prefix-composition-candidate-drift-query-hit

out

show

checked-staged-security-with-actual-alpha-prefix-authenticated-candidate-opening-query-hit-from-prefix

A out

proof (*cases out*)

case *None*

then show *?thesis*

using *hit*

unfolding

checked-staged-security-with-actual-alpha-prefix-composition-candidate-drift-query-hit-def

checked-staged-security-with-actual-alpha-prefix-composition-candidate-drift-def

checked-staged-security-with-actual-alpha-prefix-verifier-event-def

by *simp*

next

case (*Some packed*)

then obtain *prefix prefix-state data attacker-state result final-state*

where *out-eq*:

out =

Some (((((*prefix, prefix-state*), *data*), *attacker-state*), *result*),
final-state)

by (*cases packed, auto split: prod.splits*)

show *?thesis*

unfolding *out-eq*

by (*rule*

checked-staged-security-with-actual-alpha-prefix-composition-candidate-drift-query-hit-imp-authenticated-candid

[*OF wf controlled*])

(*use support hit out-eq in simp-all*)

qed

qed

lemma *checked-staged-security-with-actual-alpha-prefix-composition-candidate-drift-query-hit-bound-from-authenticat*

assumes *wf: staged-budget-wellformed budgets*

and *controlled: staged-adversary-controlled budgets A*

and *authenticated-bound:*

wp-event

*(checked-staged-security-experiment-with-actual-alpha-prefix-data-state
A)*

*(checked-staged-security-with-actual-alpha-prefix-authenticated-candidate-opening-query-hit-from-prefix
A)*

adversary-initial-state ≤ (Q::prob)

shows

wp-event

(checked-staged-security-experiment-with-actual-alpha-prefix-data-state A)

checked-staged-security-with-actual-alpha-prefix-composition-candidate-drift-query-hit

adversary-initial-state ≤ Q

by *(rule order.trans*

[OF

*checked-staged-security-with-actual-alpha-prefix-composition-candidate-drift-query-hit-le-authenticated-candidate-
authenticated-bound])*

(use wf controlled in simp-all)

lemma *checked-staged-security-with-actual-alpha-prefix-aligned-transcript-query-bad-imp-candidate-opening-hit-from-*

assumes *wf: staged-budget-wellformed budgets*

and *controlled: staged-adversary-controlled budgets A*

and *support:*

*Some (((((alpha-prefix, alpha-prefix-state), data), attacker-state),
result), final-state) ∈*

set-dist

(execute

*(checked-staged-security-experiment-with-actual-alpha-prefix-data-state
A)*

adversary-initial-state)

and *bad:*

checked-staged-security-with-actual-alpha-prefix-verifier-event

query-bad-with-aligned-transcript-partial-candidates

*(Some (((((alpha-prefix, alpha-prefix-state), data),
attacker-state), result), final-state))*

obtains *i trace-openings composition-openings where*

i < rounds

checked-staged-security-with-actual-alpha-prefix-candidate-opening-query-hit-from-prefix

A trace-openings composition-openings i

*(Some (((((alpha-prefix, alpha-prefix-state), data), attacker-state),
result), final-state))*

proof *—*

let *?s =*

verifier-state-from-adversary attacker-state

(staged-proof-transcript data)

```

have data-support:
  Some (((data, attacker-state), result), final-state) ∈
    set-dist
      (execute (checked-staged-security-experiment-with-data-state A)
        adversary-initial-state)
  by (rule
    checked-staged-security-with-actual-alpha-prefix-support-imp-data-state-support
    [OF support])
from checked-staged-security-experiment-with-data-state-outcomeE
  [OF data-support]
obtain builder verifier where
  builder:
    Some (data, attacker-state) ∈
      set-dist
        (execute (checked-staged-transcript-program A)
          adversary-initial-state)
  and verifier:
    Some (result, final-state) ∈ set-dist (execute verify-monad ?s)
  by blast
from bad obtain fr f-fri-roots f-final as dg composition-fri-roots
  final query-idxs trace-openings composition-openings trace-table
  composition-table where partial:
    accepted-with-partial-initial-openings-aligned-transcript-consistent
      ?s (Some (result, final-state)) fr f-fri-roots f-final as dg
      composition-fri-roots final query-idxs trace-openings
      composition-openings
  unfolding checked-staged-security-with-actual-alpha-prefix-verifier-event-def
    query-bad-with-aligned-transcript-partial-candidates-def
  by auto
have shape:
  accepted-transcript-shape ?s (Some (result, final-state)) as
    query-idxs
  by (rule
    accepted-with-partial-initial-openings-aligned-transcript-consistent-imp-shape
    [OF partial])
from checked-staged-security-with-data-state-accepted-shape-query-chunks
  [OF wf controlled data-support shape]
obtain raw-idxs where
  as-eq: as = staged-alphas data
  and len-raw: length raw-idxs = rounds
  and query-idxs-eq:
    query-idxs = map (λraw. index (to-nat raw)) raw-idxs
  and lookup:
    ∀ i < rounds.
      fmlookup (HashMap final-state)
        (QueryIndexChallenge i
          (state-after-query-chunks
            (staged-query-start-hash data)
            (staged-query-chunks data) i)) =

```

```

    Some (raw-idxs ! i)
  by blast
have zero-bound: 0 < rounds
  by (rule rounds-positive)
let ?raw = raw-idxs ! 0
have lookup0:
  fmllookup (HashMap final-state)
    (QueryIndexChallenge 0
      (state-after-query-chunks
        (staged-query-start-hash data)
        (staged-query-chunks data) 0)) =
    Some ?raw
  using lookup zero-bound by simp
from checked-staged-transcript-program-query-prefix-receive-final-lookup-support-with-alphas
  [OF wf controlled zero-bound builder verifier lookup0]
obtain prefix prefix-state raw-state where prefix-receive:
  Some (((prefix, prefix-state), ?raw), raw-state) ∈
  set-dist
    (execute (checked-staged-query-prefix-receive-with-state A 0)
      adversary-initial-state)
  and prefix-alphas: sqp-alphas prefix = staged-alphas data
  by blast
have idx-eq: query-idxs ! 0 = index (to-nat ?raw)
  using query-idxs-eq len-raw zero-bound by simp
have alphas-eq: as = sqp-alphas prefix
  using as-eq prefix-alphas by simp
have hit:
  checked-staged-security-with-actual-alpha-prefix-candidate-opening-query-hit-from-prefix
  A trace-openings composition-openings 0
  (Some (((((alpha-prefix, alpha-prefix-state), data), attacker-state),
    result), final-state))
  by (rule
    accepted-with-partial-initial-openings-aligned-transcript-consistent-candidate-opening-hit-from-prefix
    [OF partial zero-bound idx-eq alphas-eq builder prefix-receive])
show ?thesis
  by (rule that[OF zero-bound hit])
qed

```

lemma *checked-staged-security-with-actual-alpha-prefix-composition-candidate-drift-bound-from-authenticated-and-wf*
assumes *wf: staged-budget-wellformed budgets*
and *controlled: staged-adversary-controlled budgets A*
and *authenticated-bound:*
wp-event
 (checked-staged-security-experiment-with-actual-alpha-prefix-data-state
 A)
 (checked-staged-security-with-actual-alpha-prefix-authenticated-candidate-opening-query-hit
 A)
 adversary-initial-state ≤ (Q::prob)
and *without-bound:*

wp-event
 (*checked-staged-security-experiment-with-actual-alpha-prefix-data-state*
A)
checked-staged-security-with-actual-alpha-prefix-composition-candidate-drift-without-query-hit
adversary-initial-state $\leq$ (*W::prob*)
shows
wp-event
 (*checked-staged-security-experiment-with-actual-alpha-prefix-data-state* *A*)
checked-staged-security-with-actual-alpha-prefix-composition-candidate-drift
adversary-initial-state $\leq$ *Q* + *W*
proof (rule
checked-staged-security-with-actual-alpha-prefix-composition-candidate-drift-bound-from-query-and-without-query)
show
wp-event
 (*checked-staged-security-experiment-with-actual-alpha-prefix-data-state* *A*)
checked-staged-security-with-actual-alpha-prefix-composition-candidate-drift-query-hit
adversary-initial-state $\leq$ *Q*
by (rule
checked-staged-security-with-actual-alpha-prefix-composition-candidate-drift-query-hit-bound-from-authenticated
 [*OF wf controlled authenticated-bound*])
show
wp-event
 (*checked-staged-security-experiment-with-actual-alpha-prefix-data-state* *A*)
checked-staged-security-with-actual-alpha-prefix-composition-candidate-drift-without-query-hit
adversary-initial-state $\leq$ *W*
by (rule *without-bound*)
qed

lemma *checked-staged-security-with-actual-alpha-prefix-composition-candidate-drift-bound-from-authenticated-from-p*
assumes *wf*: *staged-budget-wellformed* *budgets*
and *controlled*: *staged-adversary-controlled* *budgets* *A*
and *authenticated-bound*:
wp-event
 (*checked-staged-security-experiment-with-actual-alpha-prefix-data-state*
A)
 (*checked-staged-security-with-actual-alpha-prefix-authenticated-candidate-opening-query-hit-from-prefix*
A)
adversary-initial-state $\leq$ (*Q::prob*)
and *without-bound*:
wp-event
 (*checked-staged-security-experiment-with-actual-alpha-prefix-data-state*
A)
checked-staged-security-with-actual-alpha-prefix-composition-candidate-drift-without-query-hit
adversary-initial-state $\leq$ (*W::prob*)
shows
wp-event
 (*checked-staged-security-experiment-with-actual-alpha-prefix-data-state* *A*)
checked-staged-security-with-actual-alpha-prefix-composition-candidate-drift
adversary-initial-state $\leq$ *Q* + *W*

proof (rule
checked-staged-security-with-actual-alpha-prefix-composition-candidate-drift-bound-from-query-and-without-query)
show
wp-event
(checked-staged-security-experiment-with-actual-alpha-prefix-data-state A)
checked-staged-security-with-actual-alpha-prefix-composition-candidate-drift-query-hit
adversary-initial-state $\leq Q$
by (rule
checked-staged-security-with-actual-alpha-prefix-composition-candidate-drift-query-hit-bound-from-authenticated-j
[OF wf controlled authenticated-bound])
show
wp-event
(checked-staged-security-experiment-with-actual-alpha-prefix-data-state A)
checked-staged-security-with-actual-alpha-prefix-composition-candidate-drift-without-query-hit
adversary-initial-state $\leq W$
by (rule *without-bound*)
qed

lemma *checked-staged-security-with-actual-alpha-prefix-composition-candidate-drift-query-hit-imp-candidate-opening*
assumes *wf: staged-budget-wellformed budgets*
and *controlled: staged-adversary-controlled budgets A*
and *support:*
Some (((((prefix, prefix-state), data), attacker-state), result),
final-state) ∈
set-dist
(execute
(checked-staged-security-experiment-with-actual-alpha-prefix-data-state
A)
adversary-initial-state)
and *hit:*
checked-staged-security-with-actual-alpha-prefix-composition-candidate-drift-query-hit
(Some (((((prefix, prefix-state), data), attacker-state), result),
final-state))
shows
checked-staged-security-with-actual-alpha-prefix-composition-candidate-opening-query-hit
A
(Some (((((prefix, prefix-state), data), attacker-state), result),
final-state))

proof –
let *?s =*
verifier-state-from-adversary attacker-state
(staged-proof-transcript data)
show *?thesis*
proof (rule
checked-staged-security-with-actual-alpha-prefix-composition-candidate-drift-query-hit-supported-openingsE
[OF wf controlled support hit])
fix *s trace-table i raw trace-openings composition-openings*
assume *i-bound: i < rounds*
and *final-lookup:*


```

fmlookup (HashMap final-state)
  (QueryIndexChallenge i
    (state-after-query-chunks
      (staged-query-start-hash data)
      (staged-query-chunks data) i)) =
  Some raw
and consistent:
  partial-query-openings-consistent trace-openings composition-openings
    (staged-alphas data) (index (to-nat raw))
have data-support:
  Some (((data, attacker-state), result), final-state) ∈
    set-dist
      (execute (checked-staged-security-experiment-with-data-state A)
        adversary-initial-state)
by (rule
  checked-staged-security-with-actual-alpha-prefix-support-imp-data-state-support
    [OF support])
from checked-staged-security-experiment-with-data-state-outcomeE
  [OF data-support]
obtain builder verifier where
  builder:
    Some (data, attacker-state) ∈
      set-dist
        (execute (checked-staged-transcript-program A)
          adversary-initial-state)
and verifier:
    Some (result, final-state) ∈ set-dist (execute verify-monad ?s)
by blast
from checked-staged-transcript-program-query-prefix-receive-final-lookup-support
  [OF wf controlled i-bound builder verifier final-lookup]
obtain query-prefix query-prefix-state raw-state where prefix-receive:
  Some (((query-prefix, query-prefix-state), raw), raw-state) ∈
    set-dist
      (execute (checked-staged-query-prefix-receive-with-state A i)
        adversary-initial-state)
by blast
have component:
  checked-staged-security-with-actual-alpha-prefix-candidate-opening-query-hit
    A
    ((replicate rounds []) [i := trace-openings])
    ((replicate rounds []) [i := composition-openings])
    (staged-alphas data) i
    (Some (((((prefix, prefix-state), data), attacker-state), result),
      final-state))
by (rule
  checked-staged-security-with-actual-alpha-prefix-candidate-opening-query-hitI-from-prefix-receive
    [OF i-bound builder prefix-receive consistent])
show ?thesis
by (rule

```

```

    checked-staged-security-with-actual-alpha-prefix-composition-candidate-opening-query-hitI
    [OF component])
qed
qed

lemma checked-staged-security-with-actual-alpha-prefix-composition-candidate-drift-query-hit-le-candidate-opening-q
assumes wf: staged-budget-wellformed budgets
    and controlled: staged-adversary-controlled budgets A
shows
    wp-event
    (checked-staged-security-experiment-with-actual-alpha-prefix-data-state A)
    checked-staged-security-with-actual-alpha-prefix-composition-candidate-drift-query-hit
    adversary-initial-state ≤
    wp-event
    (checked-staged-security-experiment-with-actual-alpha-prefix-data-state A)
    (checked-staged-security-with-actual-alpha-prefix-composition-candidate-opening-query-hit
    A)
    adversary-initial-state
proof (rule wp-event-mono-on-support)
fix out
assume support:
    out ∈
    set-dist
    (execute
    (checked-staged-security-experiment-with-actual-alpha-prefix-data-state
    A)
    adversary-initial-state)
and hit:
    checked-staged-security-with-actual-alpha-prefix-composition-candidate-drift-query-hit
    out
show
    checked-staged-security-with-actual-alpha-prefix-composition-candidate-opening-query-hit
    A out
proof (cases out)
case None
then show ?thesis
using hit
unfolding
    checked-staged-security-with-actual-alpha-prefix-composition-candidate-drift-query-hit-def
    checked-staged-security-with-actual-alpha-prefix-composition-candidate-drift-def
    checked-staged-security-with-actual-alpha-prefix-verifier-event-def
by simp
next
case (Some packed)
then obtain prefix prefix-state data attacker-state result final-state
where out-eq:
    out =
    Some (((((prefix, prefix-state), data), attacker-state), result),
    final-state)

```

```

    by (cases packed, auto split: prod.splits)
show ?thesis
  unfolding out-eq
  by (rule
    checked-staged-security-with-actual-alpha-prefix-composition-candidate-drift-query-hit-imp-candidate-opening-qu
      [OF wf controlled])
    (use support hit out-eq in simp-all)
qed
qed

lemma checked-staged-security-with-data-state-composition-bad-bound-from-authenticated-query-and-budgets:
  assumes wf: staged-budget-wellformed budgets
  and controlled: staged-adversary-controlled budgets A
  and authenticated-bound:
    wp-event
      (checked-staged-security-experiment-with-actual-alpha-prefix-data-state
        A)
      (checked-staged-security-with-actual-alpha-prefix-authenticated-candidate-opening-query-hit
        A)
      adversary-initial-state  $\leq Q$ 
  shows
    wp-event (checked-staged-security-experiment-with-data-state A)
      (staged-security-with-data-state-verifier-event composition-bad)
      adversary-initial-state  $\leq$ 
        composition-error-bound +
        staged-phase-relation-error size
        (staged-alpha-search-queries budgets 0) +
        hash-collision-budget-value 0
        (staged-alpha-search-queries budgets 0) +
        staged-phase-relation-error size
        (staged-alpha-search-queries budgets 0) +
        Q +
        hash-collision-budget-value 0
        (staged-attacker-query-budget budgets +
          staged-challenge-query-budget + verifier-hash-query-budget)
  proof -
  have without-bound:
    wp-event
      (checked-staged-security-experiment-with-actual-alpha-prefix-data-state
        A)
      checked-staged-security-with-actual-alpha-prefix-composition-candidate-drift-without-query-hit
      adversary-initial-state  $\leq 0$ 
  by (rule
    checked-staged-security-with-actual-alpha-prefix-composition-candidate-drift-without-query-hit-bound
      [OF wf controlled])
  have drift-bound:
    wp-event
      (checked-staged-security-experiment-with-actual-alpha-prefix-data-state A)
      checked-staged-security-with-actual-alpha-prefix-composition-candidate-drift

```

$adversary-initial-state \leq Q + 0$
by (rule
checked-staged-security-with-actual-alpha-prefix-composition-candidate-drift-bound-from-authenticated-and-without-bound
 $[OF \text{ wf controlled authenticated-bound without-bound}]$)
have *comp-bound*:
 $wp\text{-event } (checked\text{-staged-security-experiment-with-data-state } A)$
 $(staged\text{-security-with-data-state-verifier-event composition-bad})$
 $adversary-initial-state \leq$
 $composition\text{-error-bound} +$
 $staged\text{-phase-relation-error size}$
 $(staged\text{-alpha-search-queries budgets } 0) +$
 $hash\text{-collision-budget-value } 0$
 $(staged\text{-alpha-search-queries budgets } 0) +$
 $staged\text{-phase-relation-error size}$
 $(staged\text{-alpha-search-queries budgets } 0) +$
 $(Q + 0) +$
 $hash\text{-collision-budget-value } 0$
 $(staged\text{-attacker-query-budget budgets } +$
 $staged\text{-challenge-query-budget } + \text{ verifier-hash-query-budget})$
by (rule
checked-staged-security-with-data-state-composition-bad-bound-from-composition-candidate-drift-and-budgets
 $[OF \text{ wf controlled drift-bound}]$)
then show *?thesis*
by *simp*
qed

lemma *checked-staged-security-with-data-state-composition-bad-bound-from-header-authenticated-query-from-prefix-a*
assumes *wf: staged-budget-wellformed budgets*
and *controlled: staged-adversary-controlled budgets A*
and *header-bound*:
 $wp\text{-event}$
 $(checked\text{-staged-security-experiment-with-actual-alpha-prefix-data-state}$
 $A)$
 $(checked\text{-staged-security-with-actual-alpha-prefix-header-authenticated-candidate-opening-query-hit-from-prefix}$
 $A)$
 $adversary-initial-state \leq Q$
shows
 $wp\text{-event } (checked\text{-staged-security-experiment-with-data-state } A)$
 $(staged\text{-security-with-data-state-verifier-event composition-bad})$
 $adversary-initial-state \leq$
 $composition\text{-error-bound} +$
 $staged\text{-phase-relation-error size}$
 $(staged\text{-alpha-search-queries budgets } 0) +$
 $hash\text{-collision-budget-value } 0$
 $(staged\text{-alpha-search-queries budgets } 0) +$
 $staged\text{-phase-relation-error size}$
 $(staged\text{-alpha-search-queries budgets } 0) +$
 $Q +$
 $hash\text{-collision-budget-value } 0$

$(\text{staged-attacker-query-budget budgets} + \text{staged-challenge-query-budget} + \text{verifier-hash-query-budget})$
proof –
have *without-bound*:
wp-event
 $(\text{checked-staged-security-experiment-with-actual-alpha-prefix-data-state } A)$
 $\text{checked-staged-security-with-actual-alpha-prefix-composition-candidate-drift-without-query-hit}$
 $\text{adversary-initial-state} \leq 0$
by (rule
 $\text{checked-staged-security-with-actual-alpha-prefix-composition-candidate-drift-without-query-hit-bound}$
 $[OF \text{ wf controlled}]$)
have *drift-bound*:
wp-event
 $(\text{checked-staged-security-experiment-with-actual-alpha-prefix-data-state } A)$
 $\text{checked-staged-security-with-actual-alpha-prefix-composition-candidate-drift}$
 $\text{adversary-initial-state} \leq Q + 0$
proof (rule
 $\text{checked-staged-security-with-actual-alpha-prefix-composition-candidate-drift-bound-from-query-and-without-query}$)
show
wp-event
 $(\text{checked-staged-security-experiment-with-actual-alpha-prefix-data-state } A)$
 $\text{checked-staged-security-with-actual-alpha-prefix-composition-candidate-drift-query-hit}$
 $\text{adversary-initial-state} \leq Q$
by (rule
 $\text{checked-staged-security-with-actual-alpha-prefix-composition-candidate-drift-query-hit-bound-from-header-authen}$
 $[OF \text{ wf controlled header-bound}]$)
show
wp-event
 $(\text{checked-staged-security-experiment-with-actual-alpha-prefix-data-state } A)$
 $\text{checked-staged-security-with-actual-alpha-prefix-composition-candidate-drift-without-query-hit}$
 $\text{adversary-initial-state} \leq 0$
by (rule *without-bound*)
qed
have *comp-bound*:
wp-event $(\text{checked-staged-security-experiment-with-data-state } A)$
 $(\text{staged-security-with-data-state-verifier-event composition-bad})$
 $\text{adversary-initial-state} \leq$
 $\text{composition-error-bound} +$
 $\text{staged-phase-relation-error size}$
 $(\text{staged-alpha-search-queries budgets } 0) +$
 $\text{hash-collision-budget-value } 0$
 $(\text{staged-alpha-search-queries budgets } 0) +$
 $\text{staged-phase-relation-error size}$
 $(\text{staged-alpha-search-queries budgets } 0) +$
 $(Q + 0) +$
 $\text{hash-collision-budget-value } 0$

(staged-attacker-query-budget budgets +
 staged-challenge-query-budget + verifier-hash-query-budget)
 by (rule
 checked-staged-security-with-data-state-composition-bad-bound-from-composition-candidate-drift-and-budgets
 [OF wf controlled drift-bound])
 then show ?thesis
 by simp
 qed

lemma *checked-staged-security-with-data-state-composition-bad-bound-from-authenticated-query-from-prefix-and-bud*

assumes wf: staged-budget-wellformed budgets

and controlled: staged-adversary-controlled budgets A

and authenticated-bound:

wp-event

(checked-staged-security-experiment-with-actual-alpha-prefix-data-state

A)

(checked-staged-security-with-actual-alpha-prefix-authenticated-candidate-opening-query-hit-from-prefix

A)

adversary-initial-state $\leq Q$

shows

wp-event (checked-staged-security-experiment-with-data-state A)

(staged-security-with-data-state-verifier-event composition-bad)

adversary-initial-state $\leq$

composition-error-bound +

staged-phase-relation-error size

(staged-alpha-search-queries budgets 0) +

hash-collision-budget-value 0

(staged-alpha-search-queries budgets 0) +

staged-phase-relation-error size

(staged-alpha-search-queries budgets 0) +

Q +

hash-collision-budget-value 0

(staged-attacker-query-budget budgets +

staged-challenge-query-budget + verifier-hash-query-budget)

proof –

have without-bound:

wp-event

(checked-staged-security-experiment-with-actual-alpha-prefix-data-state

A)

checked-staged-security-with-actual-alpha-prefix-composition-candidate-drift-without-query-hit

adversary-initial-state ≤ 0

by (rule

checked-staged-security-with-actual-alpha-prefix-composition-candidate-drift-without-query-hit-bound

[OF wf controlled])

have drift-bound:

wp-event

(checked-staged-security-experiment-with-actual-alpha-prefix-data-state A)

checked-staged-security-with-actual-alpha-prefix-composition-candidate-drift

adversary-initial-state $\leq Q + 0$

```

    by (rule
      checked-staged-security-with-actual-alpha-prefix-composition-candidate-drift-bound-from-authenticated-from-prefix
      [OF wf controlled authenticated-bound without-bound])
  have comp-bound:
    wp-event (checked-staged-security-experiment-with-data-state A)
      (staged-security-with-data-state-verifier-event composition-bad)
      adversary-initial-state ≤
      composition-error-bound +
      staged-phase-relation-error size
      (staged-alpha-search-queries budgets 0) +
      hash-collision-budget-value 0
      (staged-alpha-search-queries budgets 0) +
      staged-phase-relation-error size
      (staged-alpha-search-queries budgets 0) +
      (Q + 0) +
      hash-collision-budget-value 0
      (staged-attacker-query-budget budgets +
        staged-challenge-query-budget + verifier-hash-query-budget)
  by (rule
    checked-staged-security-with-data-state-composition-bad-bound-from-composition-candidate-drift-and-budgets
    [OF wf controlled drift-bound])
  then show ?thesis
  by simp
qed

end

end

```

```

theory Staged-Security-Experiment-Composition-Query-Augmented
imports Staged-Security-Experiment-Composition-Query-Bridge
begin

```

Augmented query-prefix events for the composition query side.

These events live in the query-prefix augmented experiment, so the sampled query challenge is part of the event output. This is the correct setting for applying fixed-witness query-prefix bounds. The fixed-witness event below deliberately does not existentially choose openings after the query challenge; the remaining public-proof work is to derive such fixed witnesses from pre-query Merkle/candidate evidence or charge failures to existing side events.

```

context soundness
begin

```

```

lemma checked-staged-security-with-data-state-query-prefix-continuationE:
assumes i-bound:  $i < \text{rounds}$ 
and support:
  Some (((data, attacker-state), result), final-state) ∈

```

```

    set-dist
      (execute (checked-staged-security-experiment-with-data-state A)
        adversary-initial-state)
obtains prefix prefix-state raw raw-state where
  Some (((prefix, prefix-state), raw), raw-state) ∈
    set-dist
      (execute
        (checked-staged-query-prefix-receive-with-state A i)
        adversary-initial-state)
  Some (((data, attacker-state), result), final-state) ∈
    set-dist
      (execute
        (checked-staged-after-query-prefix-receive-with-verifier A i
          ((prefix, prefix-state), raw))
        raw-state)
proof –
  have decomp:
    checked-staged-security-experiment-with-data-state A =
    checked-staged-query-prefix-receive-with-state A i >>=
    checked-staged-after-query-prefix-receive-with-verifier A i
  by (rule
    checked-staged-security-experiment-with-data-state-query-prefix-receive-decomp
    [OF i-bound])
from support[unfolded decomp]
obtain x raw-state where prefix-receive:
  Some (x, raw-state) ∈
    set-dist
      (execute
        (checked-staged-query-prefix-receive-with-state A i)
        adversary-initial-state)
  and continuation:
    Some (((data, attacker-state), result), final-state) ∈
      set-dist
        (execute
          (checked-staged-after-query-prefix-receive-with-verifier A i x)
          raw-state)
  by (auto elim!: set-dist-bindE)
obtain prefix-pack raw where x-eq: x = (prefix-pack, raw)
  by (cases x) simp
obtain prefix prefix-state where prefix-pack-eq:
  prefix-pack = (prefix, prefix-state)
  by (cases prefix-pack) simp
show ?thesis
  by (rule that[of prefix prefix-state raw raw-state])
    (use prefix-receive continuation x-eq prefix-pack-eq in simp-all)
qed

```

definition checked-staged-security-with-query-prefix-header-authenticated-candidate-opening-hit
where

checked-staged-security-with-query-prefix-header-authenticated-candidate-opening-hit
trace-openings composition-openings i out $\longleftrightarrow$
 (case out of
 None $\Rightarrow$ False
 | Some (((((prefix, prefix-state), raw), raw-state),
 ((data, attacker-state), result)), final-state) $\Rightarrow$
 staged-composition-fri-roots data $\neq \square$ $\wedge$
 (trace-openings, composition-openings) $\in$
 query-header-supported-partial-opening-witnesses
 (verifier-state-from-adversary attacker-state
 (staged-proof-transcript data))
 (staged-trace-root data)
 (staged-trace-fri-roots data)
 (staged-trace-final data)
 (staged-alphas data)
 (staged-degree data)
 (staged-composition-fri-roots data)
 (staged-composition-final data) $\wedge$
 checked-staged-security-with-actual-query-prefix-candidate-opening-hit
 ((replicate rounds $\square$) [i := trace-openings])
 ((replicate rounds $\square$) [i := composition-openings])
 i out)

lemma *checked-staged-security-with-query-prefix-header-authenticated-candidate-opening-hit-imp-actual:*

assumes *hit:*

checked-staged-security-with-query-prefix-header-authenticated-candidate-opening-hit
trace-openings composition-openings i out

shows

checked-staged-security-with-actual-query-prefix-candidate-opening-hit
 ((replicate rounds $\square$) [i := trace-openings])
 ((replicate rounds $\square$) [i := composition-openings])
 i out

using *hit*

unfolding

checked-staged-security-with-query-prefix-header-authenticated-candidate-opening-hit-def
by (cases out) (auto split: prod.splits)

lemma *checked-staged-security-with-query-prefix-header-authenticated-candidate-opening-hitE:*

assumes *hit:*

checked-staged-security-with-query-prefix-header-authenticated-candidate-opening-hit
trace-openings composition-openings i
 (Some (((((prefix, prefix-state), raw), raw-state),
 ((data, attacker-state), result)), final-state))

obtains

staged-composition-fri-roots data $\neq \square$
 (trace-openings, composition-openings) $\in$
 query-header-supported-partial-opening-witnesses
 (verifier-state-from-adversary attacker-state
 (staged-proof-transcript data))

```

    (staged-trace-root data)
    (staged-trace-fri-roots data)
    (staged-trace-final data)
    (staged-alphas data)
    (staged-degree data)
    (staged-composition-fri-roots data)
    (staged-composition-final data)
  index (to-nat raw) ∈
    staged-query-prefix-candidate-opening-query-target-from-prefix
    ((replicate rounds []) [i := trace-openings])
    ((replicate rounds []) [i := composition-openings])
    i prefix prefix-state
proof —
  have comp-nonempty:
    staged-composition-fri-roots data ≠ []
  and witness:
    (trace-openings, composition-openings) ∈
      query-header-supported-partial-opening-witnesses
      (verifier-state-from-adversary attacker-state
        (staged-proof-transcript data))
      (staged-trace-root data)
      (staged-trace-fri-roots data)
      (staged-trace-final data)
      (staged-alphas data)
      (staged-degree data)
      (staged-composition-fri-roots data)
      (staged-composition-final data)
  and actual:
    checked-staged-security-with-actual-query-prefix-candidate-opening-hit
    ((replicate rounds []) [i := trace-openings])
    ((replicate rounds []) [i := composition-openings])
    i
    (Some (((((prefix, prefix-state), raw), raw-state),
      ((data, attacker-state), result)), final-state))
  using hit
  unfolding
    checked-staged-security-with-query-prefix-header-authenticated-candidate-opening-hit-def
  by simp-all
from checked-staged-security-with-actual-query-prefix-candidate-opening-hitE
  [OF actual]
have target:
  index (to-nat raw) ∈
    staged-query-prefix-candidate-opening-query-target-from-prefix
    ((replicate rounds []) [i := trace-openings])
    ((replicate rounds []) [i := composition-openings])
    i prefix prefix-state
  .
show ?thesis
  by (rule that[OF comp-nonempty witness target])

```

qed

lemma *checked-staged-security-with-query-prefix-header-authenticated-candidate-opening-hit-prefix-facts:*

assumes *support:*

*Some (((((prefix, prefix-state), raw), raw-state),
 ((data, attacker-state), result)), final-state) ∈
 set-dist
 (execute
 (checked-staged-security-experiment-with-query-prefix-data-state A i)
 adversary-initial-state)*

and *hit:*

*checked-staged-security-with-query-prefix-header-authenticated-candidate-opening-hit
 trace-openings composition-openings i
 (Some (((((prefix, prefix-state), raw), raw-state),
 ((data, attacker-state), result)), final-state))*

shows

*staged-alphas data = sqp-alphas prefix
 staged-trace-root data = sqp-trace-root prefix
 staged-composition-fri-roots data = sqp-composition-fri-roots prefix
 index (to-nat raw) ∈
 staged-query-prefix-candidate-opening-query-target-from-prefix
 ((replicate rounds []) [i := trace-openings])
 ((replicate rounds []) [i := composition-openings])
 i prefix prefix-state*

proof —

show *alphas: staged-alphas data = sqp-alphas prefix*

by (rule *checked-staged-security-with-query-prefix-data-state-alphas-eq*)
 (rule *support*)

from *hit* **show** *target:*

*index (to-nat raw) ∈
 staged-query-prefix-candidate-opening-query-target-from-prefix
 ((replicate rounds []) [i := trace-openings])
 ((replicate rounds []) [i := composition-openings])
 i prefix prefix-state*

by (rule
checked-staged-security-with-query-prefix-header-authenticated-candidate-opening-hitE)

from *support* **show** *staged-trace-root data = sqp-trace-root prefix*

unfolding *checked-staged-security-experiment-with-query-prefix-data-state-def
 checked-staged-after-query-prefix-receive-with-verifier-def
 checked-staged-after-query-prefix-receive-def*

by (auto elim!: *set-dist-bindE* split: *prod.splits*)

from *support* **show**

*staged-composition-fri-roots data = sqp-composition-fri-roots prefix
 unfolding checked-staged-security-experiment-with-query-prefix-data-state-def
 checked-staged-after-query-prefix-receive-with-verifier-def
 checked-staged-after-query-prefix-receive-def*

by (auto elim!: *set-dist-bindE* split: *prod.splits*)

qed

The next wrapper lets downstream composition proofs conjoin their own side-condition

with the fixed authenticated-opening query-prefix hit, without changing the probability bound. This is deliberately fixed-witness: the trace and composition openings are chosen before the query challenge is sampled.

definition *checked-staged-security-with-query-prefix-fixed-header-component*

where

checked-staged-security-with-query-prefix-fixed-header-component
 $E \text{ trace-openings composition-openings } i \text{ out } \longleftrightarrow$
 $E \text{ out } \wedge$
checked-staged-security-with-query-prefix-header-authenticated-candidate-opening-hit
 $\text{trace-openings composition-openings } i \text{ out}$

definition *checked-staged-security-with-query-prefix-fixed-current-authenticated-component*

where

checked-staged-security-with-query-prefix-fixed-current-authenticated-component
 $E \text{ trace-openings composition-openings } i \text{ out } \longleftrightarrow$
checked-staged-security-with-query-prefix-fixed-header-component
 $E \text{ trace-openings composition-openings } i \text{ out } \wedge$
checked-staged-security-with-query-prefix-authenticated-opening-hit
 $\text{trace-openings composition-openings } i \text{ out}$

definition *checked-staged-security-with-query-prefix-header-witness-not-current*

where

checked-staged-security-with-query-prefix-header-witness-not-current
 $E \text{ trace-openings composition-openings } i \text{ out } \longleftrightarrow$
checked-staged-security-with-query-prefix-fixed-header-component
 $E \text{ trace-openings composition-openings } i \text{ out } \wedge$
 $\neg \text{checked-staged-security-with-query-prefix-authenticated-opening-hit}$
 $\text{trace-openings composition-openings } i \text{ out}$

definition *checked-staged-security-with-query-prefix-header-witness-path-output-hit*

where

checked-staged-security-with-query-prefix-header-witness-path-output-hit
 $E \text{ trace-openings composition-openings } i \text{ out } \longleftrightarrow$
(case out of
 $\text{None} \Rightarrow \text{False}$
| $\text{Some } (((((\text{prefix}, \text{prefix-state}), \text{raw}), \text{raw-state}), ((\text{data}, \text{attacker-state}), \text{result})), \text{final-state}) \Rightarrow$
 $E \text{ out } \wedge$
 $\text{staged-composition-fri-roots data} \neq [] \wedge$
 $(\exists \text{witness-result witness-state rest.}$
 $\text{let } s = \text{verifier-state-from-adversary attacker-state}$
 $(\text{staged-proof-transcript data}) \text{ in}$
 $\text{Some } (\text{witness-result}, \text{witness-state}) \in$
 $\text{set-dist } (\text{execute verify-monad } s) \wedge$
 $\text{verifier-header-transcript } s$
 $(\text{staged-trace-root data})$
 $(\text{staged-trace-fri-roots data})$
 $(\text{staged-trace-final data})$
 $(\text{staged-alphas data})$

(staged-degree data)
 (staged-composition-fri-roots data)
 (staged-composition-final data)
 rest $\wedge$
 (($\exists$ opn $\in$ set trace-openings.
 hash-map-new-output-hit
 (merkle-path-target-roots witness-state
 (staged-trace-root data)
 (scale * clength)
 (opening-index opn)
 (opening-value opn)
 (opening-path opn))
 s witness-state) $\vee$
 ($\exists$ opn $\in$ set composition-openings.
 hash-map-new-output-hit
 (merkle-path-target-roots witness-state
 (hd (staged-composition-fri-roots data))
 (scale * clength)
 (opening-index opn)
 (opening-value opn)
 (opening-path opn))
 s witness-state))))

definition *checked-staged-security-with-query-prefix-header-witness-transcript-hit*

where

checked-staged-security-with-query-prefix-header-witness-transcript-hit
 E out $\longleftrightarrow$
 (case out of
 None $\Rightarrow$ False
 | Some (((((prefix, prefix-state), raw), raw-state),
 ((data, attacker-state), result)), final-state) $\Rightarrow$
 E out $\wedge$
 ($\exists$ witness-result witness-state.
 let s = verifier-state-from-adversary attacker-state
 (staged-proof-transcript data) in
 Some (witness-result, witness-state) $\in$
 set-dist (execute verify-monad s) $\wedge$
 (hash-map-output-values s $\cap$
 set (staged-proof-transcript data) $\neq$ {} $\vee$
 hash-map-new-output-hit (set (staged-proof-transcript data))
 s witness-state)))

definition *checked-staged-security-with-query-prefix-header-witness-transcript-pre-hit*

where

checked-staged-security-with-query-prefix-header-witness-transcript-pre-hit
 E out $\longleftrightarrow$
 (case out of
 None $\Rightarrow$ False
 | Some (((((prefix, prefix-state), raw), raw-state),

$((data, attacker-state), result)), final-state) \Rightarrow$
 $E \text{ out} \wedge$
 $hash-map-output-values$
 $(verifier-state-from-adversary\ attacker-state$
 $(staged-proof-transcript\ data)) \cap$
 $set\ (staged-proof-transcript\ data) \neq \{\}$

definition *checked-staged-security-with-query-prefix-header-witness-transcript-new-hit*
where

$checked-staged-security-with-query-prefix-header-witness-transcript-new-hit$
 $E \text{ out} \longleftrightarrow$
 $(case\ out\ of$
 $None \Rightarrow False$
 $| Some\ (((((prefix, prefix-state), raw), raw-state),$
 $((data, attacker-state), result)), final-state) \Rightarrow$
 $E \text{ out} \wedge$
 $(\exists\ witness-result\ witness-state.$
 $let\ s = verifier-state-from-adversary\ attacker-state$
 $(staged-proof-transcript\ data)\ in$
 $Some\ (witness-result, witness-state) \in$
 $set-dist\ (execute\ verify-monad\ s) \wedge$
 $hash-map-new-output-hit\ (set\ (staged-proof-transcript\ data))$
 $s\ witness-state))$

lemma *checked-staged-security-with-query-prefix-header-witness-transcript-hit-imp-pre-or-new:*

assumes *hit:*

$checked-staged-security-with-query-prefix-header-witness-transcript-hit$
 $E \text{ out}$

shows

$checked-staged-security-with-query-prefix-header-witness-transcript-pre-hit$
 $E \text{ out} \vee$
 $checked-staged-security-with-query-prefix-header-witness-transcript-new-hit$
 $E \text{ out}$

using *hit*

unfolding

$checked-staged-security-with-query-prefix-header-witness-transcript-hit-def$
 $checked-staged-security-with-query-prefix-header-witness-transcript-pre-hit-def$
 $checked-staged-security-with-query-prefix-header-witness-transcript-new-hit-def$

by $(cases\ out)\ (fastforce\ split:\ prod.splits) +$

lemma *checked-staged-security-with-query-prefix-header-witness-transcript-pre-hit-imp-data-state-pre-hit:*

assumes *hit:*

$checked-staged-security-with-query-prefix-header-witness-transcript-pre-hit$
 E
 $(Some\ (((((prefix, prefix-state), raw), raw-state),$
 $((data, attacker-state), result)), final-state))$

shows

$staged-security-with-data-state-transcript-pre-hit$
 $(Some\ (((data, attacker-state), result), final-state))$

```

using hit
unfolding
  checked-staged-security-with-query-prefix-header-witness-transcript-pre-hit-def
  staged-security-with-data-state-transcript-pre-hit-def
by simp

lemma checked-staged-security-with-query-prefix-header-witness-transcript-pre-hit-bound-from-data-state-pre-hit:
assumes i-bound: i < rounds
and pre-bound:
  wp-event (checked-staged-security-experiment-with-data-state A)
  staged-security-with-data-state-transcript-pre-hit
  adversary-initial-state ≤ P
shows
  wp-event
    (checked-staged-security-experiment-with-query-prefix-data-state A i)
    (checked-staged-security-with-query-prefix-header-witness-transcript-pre-hit
      E)
    adversary-initial-state ≤ P
proof –
have projection:
  wp-event (checked-staged-security-experiment-with-data-state A)
  staged-security-with-data-state-transcript-pre-hit
  adversary-initial-state =
  wp-event
    (checked-staged-security-experiment-with-query-prefix-data-state A i)
    ( $\lambda$ out. case out of
      None ⇒ staged-security-with-data-state-transcript-pre-hit None
      | Some (packed, t) ⇒
        staged-security-with-data-state-transcript-pre-hit
          (Some (snd packed, t)))
    adversary-initial-state
by (rule
  checked-staged-security-experiment-with-query-prefix-data-state-projection-event
  [OF i-bound])
have event-le:
  wp-event
    (checked-staged-security-experiment-with-query-prefix-data-state A i)
    (checked-staged-security-with-query-prefix-header-witness-transcript-pre-hit
      E)
    adversary-initial-state ≤
  wp-event
    (checked-staged-security-experiment-with-query-prefix-data-state A i)
    ( $\lambda$ out. case out of
      None ⇒ staged-security-with-data-state-transcript-pre-hit None
      | Some (packed, t) ⇒
        staged-security-with-data-state-transcript-pre-hit
          (Some (snd packed, t)))
    adversary-initial-state
proof (rule wp-event-mono)

```

```

fix out
assume hit:
  checked-staged-security-with-query-prefix-header-witness-transcript-pre-hit
  E out
show
  (case out of
    None  $\Rightarrow$  staged-security-with-data-state-transcript-pre-hit None
  | Some (packed, t)  $\Rightarrow$ 
    staged-security-with-data-state-transcript-pre-hit
      (Some (snd packed, t)))
proof (cases out)
  case None
  then show ?thesis
    using hit
    unfolding
      checked-staged-security-with-query-prefix-header-witness-transcript-pre-hit-def
    by simp
next
  case (Some packed)
  then obtain prefix prefix-state raw raw-state data attacker-state result
    final-state where out-eq:
    out =
      Some (((((prefix, prefix-state), raw), raw-state),
        ((data, attacker-state), result)), final-state)
    by (cases packed) (auto split: prod.splits)
  show ?thesis
    using
      checked-staged-security-with-query-prefix-header-witness-transcript-pre-hit-imp-data-state-pre-hit
      [OF hit[unfolded out-eq]]
    unfolding out-eq by simp
  qed
qed
show ?thesis
  by (rule order-trans[OF event-le])
  (use pre-bound projection in simp)
qed

lemma checked-staged-security-with-query-prefix-header-witness-path-output-hitE:
assumes hit:
  checked-staged-security-with-query-prefix-header-witness-path-output-hit
  E trace-openings composition-openings i
  (Some (((((prefix, prefix-state), raw), raw-state),
    ((data, attacker-state), result)), final-state))
and that:
   $\bigwedge$  witness-result witness-state rest.
  E (Some (((((prefix, prefix-state), raw), raw-state),
    ((data, attacker-state), result)), final-state))  $\Rightarrow$ 
  staged-composition-fri-roots data  $\neq$  []  $\Rightarrow$ 
  Some (witness-result, witness-state)  $\in$ 

```



```

    set-dist
      (execute verify-monad
        (verifier-state-from-adversary attacker-state
          (staged-proof-transcript data)))  $\impl$ 
    verifier-header-transcript
      (verifier-state-from-adversary attacker-state
        (staged-proof-transcript data))
      (staged-trace-root data)
      (staged-trace-fri-roots data)
      (staged-trace-final data)
      (staged-alphas data)
      (staged-degree data)
      (staged-composition-fri-roots data)
      (staged-composition-final data)
    rest  $\impl$ 
    (( $\exists$  opn  $\in$  set trace-openings.
      hash-map-new-output-hit
        (merkle-path-target-roots witness-state
          (staged-trace-root data)
          (scale * clength)
          (opening-index opn)
          (opening-value opn)
          (opening-path opn))
        (verifier-state-from-adversary attacker-state
          (staged-proof-transcript data))
        witness-state)  $\vee$ 
      ( $\exists$  opn  $\in$  set composition-openings.
        hash-map-new-output-hit
          (merkle-path-target-roots witness-state
            (hd (staged-composition-fri-roots data))
            (scale * clength)
            (opening-index opn)
            (opening-value opn)
            (opening-path opn))
          (verifier-state-from-adversary attacker-state
            (staged-proof-transcript data))
          witness-state))  $\impl$ 
    thesis
  shows thesis
proof -
  have E:
    E (Some (((((prefix, prefix-state), raw), raw-state),
      ((data, attacker-state), result)), final-state))
  and nonempty:
    staged-composition-fri-roots data  $\neq$  []
  using hit
  unfolding checked-staged-security-with-query-prefix-header-witness-path-output-hit-def
  by simp-all
from hit obtain witness-result witness-state rest where

```

```

witness:
  Some (witness-result, witness-state) ∈
    set-dist
      (execute verify-monad
        (verifier-state-from-adversary attacker-state
          (staged-proof-transcript data)))
and header:
  verifier-header-transcript
    (verifier-state-from-adversary attacker-state
      (staged-proof-transcript data))
    (staged-trace-root data)
    (staged-trace-fri-roots data)
    (staged-trace-final data)
    (staged-alphas data)
    (staged-degree data)
    (staged-composition-fri-roots data)
    (staged-composition-final data)
    rest
and path-hit:
  ((∃ opn ∈ set trace-openings.
    hash-map-new-output-hit
      (merkle-path-target-roots witness-state
        (staged-trace-root data)
        (scale * clength)
        (opening-index opn)
        (opening-value opn)
        (opening-path opn))
      (verifier-state-from-adversary attacker-state
        (staged-proof-transcript data))
      witness-state) ∨
    (∃ opn ∈ set composition-openings.
      hash-map-new-output-hit
        (merkle-path-target-roots witness-state
          (hd (staged-composition-fri-roots data))
          (scale * clength)
          (opening-index opn)
          (opening-value opn)
          (opening-path opn))
        (verifier-state-from-adversary attacker-state
          (staged-proof-transcript data))
        witness-state))
  unfolding checked-staged-security-with-query-prefix-header-witness-path-output-hit-def
  by (auto simp: Let-def; blast)
show ?thesis
  by (rule that[OF E nonempty witness header path-hit])
qed

lemma checked-staged-security-with-query-prefix-header-witness-path-output-imp-transcript-hit:
  assumes hit:

```

```

checked-staged-security-with-query-prefix-header-witness-path-output-hit
  E trace-openings composition-openings i out
shows
  checked-staged-security-with-query-prefix-header-witness-transcript-hit
  E out
proof (cases out)
  case None
  then show ?thesis
    using hit
    unfolding checked-staged-security-with-query-prefix-header-witness-path-output-hit-def
      checked-staged-security-with-query-prefix-header-witness-transcript-hit-def
    by simp
next
  case (Some packed)
  then obtain prefix prefix-state raw raw-state data attacker-state result
    final-state where out-eq:
    out =
      Some (((((prefix, prefix-state), raw), raw-state),
        ((data, attacker-state), result)), final-state)
    by (cases packed) (auto split: prod.splits)
  let ?s =
    verifier-state-from-adversary attacker-state
    (staged-proof-transcript data)
  from hit[unfolded out-eq]
  obtain witness-result witness-state rest where
    E:
    E (Some (((((prefix, prefix-state), raw), raw-state),
      ((data, attacker-state), result)), final-state))
    and witness:
    Some (witness-result, witness-state) ∈
    set-dist (execute verify-monad ?s)
    by (rule
      checked-staged-security-with-query-prefix-header-witness-path-output-hitE)
  have tr-eq: PTranscript ?s = staged-proof-transcript data
    by simp
  have transcript-hit:
    hash-map-output-values ?s ∩ set (staged-proof-transcript data)
    ≠ {} ∨
    hash-map-new-output-hit (set (staged-proof-transcript data))
    ?s witness-state
    by (rule verify-monad-trace-root-target-preexisting-or-new
      [OF witness tr-eq])
  show ?thesis
    unfolding out-eq
    checked-staged-security-with-query-prefix-header-witness-transcript-hit-def
    using E witness transcript-hit by (auto simp: Let-def)
qed

```

lemma *checked-staged-security-with-query-prefix-fixed-header-component-imp-header:*

assumes *hit*:

checked-staged-security-with-query-prefix-fixed-header-component

E trace-openings composition-openings i out

shows

checked-staged-security-with-query-prefix-header-authenticated-candidate-opening-hit

trace-openings composition-openings i out

using *hit*

unfolding *checked-staged-security-with-query-prefix-fixed-header-component-def*

by *simp*

lemma *checked-staged-security-with-query-prefix-fixed-current-authenticated-component-imp-current*:

assumes *hit*:

checked-staged-security-with-query-prefix-fixed-current-authenticated-component

E trace-openings composition-openings i out

shows

checked-staged-security-with-query-prefix-authenticated-opening-hit

trace-openings composition-openings i out

using *hit*

unfolding

checked-staged-security-with-query-prefix-fixed-current-authenticated-component-def

by *simp*

lemma *checked-staged-security-with-query-prefix-fixed-header-component-split-current*:

assumes *hit*:

checked-staged-security-with-query-prefix-fixed-header-component

E trace-openings composition-openings i out

shows

checked-staged-security-with-query-prefix-fixed-current-authenticated-component

E trace-openings composition-openings i out $\vee$

checked-staged-security-with-query-prefix-header-witness-not-current

E trace-openings composition-openings i out

using *hit*

unfolding

checked-staged-security-with-query-prefix-fixed-current-authenticated-component-def

checked-staged-security-with-query-prefix-header-witness-not-current-def

by *blast*

lemma *checked-staged-security-with-query-prefix-fixed-header-componentE*:

assumes *hit*:

checked-staged-security-with-query-prefix-fixed-header-component

E trace-openings composition-openings i

(*Some* (((((*prefix*, *prefix-state*), *raw*), *raw-state*),

((*data*, *attacker-state*), *result*)), *final-state*))

obtains

E (*Some* (((((*prefix*, *prefix-state*), *raw*), *raw-state*),

((*data*, *attacker-state*), *result*)), *final-state*))

staged-composition-fri-roots data $\neq []$

(*trace-openings*, *composition-openings*) $\in$

query-header-supported-partial-opening-witnesses

```

    (verifier-state-from-adversary attacker-state
      (staged-proof-transcript data))
    (staged-trace-root data)
    (staged-trace-fri-roots data)
    (staged-trace-final data)
    (staged-alphas data)
    (staged-degree data)
    (staged-composition-fri-roots data)
    (staged-composition-final data)
  index (to-nat raw) ∈
    staged-query-prefix-candidate-opening-query-target-from-prefix
      ((replicate rounds []) [i := trace-openings])
      ((replicate rounds []) [i := composition-openings])
    i prefix prefix-state
proof —
have E:
  E (Some (((((prefix, prefix-state), raw), raw-state),
    ((data, attacker-state), result)), final-state))
and header:
  checked-staged-security-with-query-prefix-header-authenticated-candidate-opening-hit
    trace-openings composition-openings i
  (Some (((((prefix, prefix-state), raw), raw-state),
    ((data, attacker-state), result)), final-state))
using hit
unfolding checked-staged-security-with-query-prefix-fixed-header-component-def
by simp-all
from header show ?thesis
proof (rule
  checked-staged-security-with-query-prefix-header-authenticated-candidate-opening-hitE)
assume comp-nonempty:
  staged-composition-fri-roots data ≠ []
and witness:
  (trace-openings, composition-openings) ∈
    query-header-supported-partial-opening-witnesses
      (verifier-state-from-adversary attacker-state
        (staged-proof-transcript data))
      (staged-trace-root data)
      (staged-trace-fri-roots data)
      (staged-trace-final data)
      (staged-alphas data)
      (staged-degree data)
      (staged-composition-fri-roots data)
      (staged-composition-final data)
and target:
  index (to-nat raw) ∈
    staged-query-prefix-candidate-opening-query-target-from-prefix
      ((replicate rounds []) [i := trace-openings])
      ((replicate rounds []) [i := composition-openings])
    i prefix prefix-state

```

```

show ?thesis
  by (rule that[OF E comp-nonempty witness target])
qed
qed

lemma checked-staged-security-with-query-prefix-fixed-header-component-supportE:
assumes support:
  Some (((((prefix, prefix-state), raw), raw-state),
    ((data, attacker-state), result)), final-state) ∈
  set-dist
    (execute
      (checked-staged-security-experiment-with-query-prefix-data-state A i)
      adversary-initial-state)
and hit:
  checked-staged-security-with-query-prefix-fixed-header-component
  E trace-openings composition-openings i
  (Some (((((prefix, prefix-state), raw), raw-state),
    ((data, attacker-state), result)), final-state))
obtains
  E (Some (((((prefix, prefix-state), raw), raw-state),
    ((data, attacker-state), result)), final-state))
  Some (((prefix, prefix-state), raw), raw-state) ∈
  set-dist
    (execute
      (checked-staged-query-prefix-receive-with-state A i)
      adversary-initial-state)
  Some (((data, attacker-state), result), final-state) ∈
  set-dist
    (execute
      (checked-staged-after-query-prefix-receive-with-verifier A i
        ((prefix, prefix-state), raw))
      raw-state)
  staged-alphas data = sqp-alphas prefix
  staged-trace-root data = sqp-trace-root prefix
  staged-composition-fri-roots data = sqp-composition-fri-roots prefix
  staged-composition-fri-roots data ≠ []
  (trace-openings, composition-openings) ∈
  query-header-supported-partial-opening-witnesses
    (verifier-state-from-adversary attacker-state
      (staged-proof-transcript data))
    (staged-trace-root data)
    (staged-trace-fri-roots data)
    (staged-trace-final data)
    (staged-alphas data)
    (staged-degree data)
    (staged-composition-fri-roots data)
    (staged-composition-final data)
  index (to-nat raw) ∈
  staged-query-prefix-candidate-opening-query-target-from-prefix

```

```

    ((replicate rounds []) [i := trace-openings])
    ((replicate rounds []) [i := composition-openings])
    i prefix prefix-state
proof –
have prefix-receive:
  Some (((prefix, prefix-state), raw), raw-state) ∈
    set-dist
    (execute
      (checked-staged-query-prefix-receive-with-state A i)
      adversary-initial-state)
and continuation:
  Some (((data, attacker-state), result), final-state) ∈
    set-dist
    (execute
      (checked-staged-after-query-prefix-receive-with-verifier A i
        ((prefix, prefix-state), raw))
      raw-state)
using support
unfolding checked-staged-security-experiment-with-query-prefix-data-state-def
by (auto elim!: set-dist-bindE)
have E:
  E (Some (((((prefix, prefix-state), raw), raw-state),
    ((data, attacker-state), result)), final-state))
and header:
  checked-staged-security-with-query-prefix-header-authenticated-candidate-opening-hit
    trace-openings composition-openings i
    (Some (((((prefix, prefix-state), raw), raw-state),
      ((data, attacker-state), result)), final-state))
using hit
unfolding checked-staged-security-with-query-prefix-fixed-header-component-def
by simp-all
from header show ?thesis
proof (rule
  checked-staged-security-with-query-prefix-header-authenticated-candidate-opening-hitE)
assume comp-nonempty:
  staged-composition-fri-roots data ≠ []
and witness:
  (trace-openings, composition-openings) ∈
    query-header-supported-partial-opening-witnesses
      (verifier-state-from-adversary attacker-state
        (staged-proof-transcript data))
      (staged-trace-root data)
      (staged-trace-fri-roots data)
      (staged-trace-final data)
      (staged-alphas data)
      (staged-degree data)
      (staged-composition-fri-roots data)
      (staged-composition-final data)
and target:

```

```

    index (to-nat raw) ∈
      staged-query-prefix-candidate-opening-query-target-from-prefix
        ((replicate rounds []) [i := trace-openings])
        ((replicate rounds []) [i := composition-openings])
      i prefix prefix-state
  have alphas:
    staged-alphas data = sqp-alphas prefix
  by (rule checked-staged-security-with-query-prefix-data-state-alphas-eq)
    (rule support)
  have trace-root:
    staged-trace-root data = sqp-trace-root prefix
  and comp-roots:
    staged-composition-fri-roots data = sqp-composition-fri-roots prefix
  using support
  unfolding checked-staged-security-experiment-with-query-prefix-data-state-def
    checked-staged-after-query-prefix-receive-with-verifier-def
    checked-staged-after-query-prefix-receive-def
  by (auto elim!: set-dist-bindE split: prod.splits)
  show ?thesis
  by (rule that[OF E prefix-receive continuation alphas trace-root
    comp-roots comp-nonempty witness target])
qed
qed

lemma checked-staged-security-with-query-prefix-header-witness-not-current-imp-path-output-on-support:
  assumes wf: staged-budget-wellformed budgets
  and controlled: staged-adversary-controlled budgets A
  and i-bound: i < rounds
  and support:
    Some (((((prefix, prefix-state), raw), raw-state),
      ((data, attacker-state), result)), final-state) ∈
      set-dist
      (execute
        (checked-staged-security-experiment-with-query-prefix-data-state A i)
        adversary-initial-state)
  and hit:
    checked-staged-security-with-query-prefix-header-witness-not-current
      E trace-openings composition-openings i
    (Some (((((prefix, prefix-state), raw), raw-state),
      ((data, attacker-state), result)), final-state))
  shows
    checked-staged-security-with-query-prefix-header-witness-path-output-hit
      E trace-openings composition-openings i
    (Some (((((prefix, prefix-state), raw), raw-state),
      ((data, attacker-state), result)), final-state))
  proof -
  let ?out =
    Some (((((prefix, prefix-state), raw), raw-state),
      ((data, attacker-state), result)), final-state)

```



```

let ?s =
  verifier-state-from-adversary attacker-state
  (staged-proof-transcript data)
have fixed:
  checked-staged-security-with-query-prefix-fixed-header-component
    E trace-openings composition-openings i ?out
and not-current:
  ¬ checked-staged-security-with-query-prefix-authenticated-opening-hit
    trace-openings composition-openings i ?out
using hit
unfolding
  checked-staged-security-with-query-prefix-header-witness-not-current-def
by simp-all
have support-facts:
  E ?out ∧
  Some (((data, attacker-state), result), final-state) ∈
    set-dist
    (execute
      (checked-staged-after-query-prefix-receive-with-verifier A i
        ((prefix, prefix-state), raw))
      raw-state) ∧
  staged-alphas data = sqp-alphas prefix ∧
  staged-composition-fri-roots data ≠ [] ∧
  (trace-openings, composition-openings) ∈
    query-header-supported-partial-opening-witnesses ?s
    (staged-trace-root data)
    (staged-trace-fri-roots data)
    (staged-trace-final data)
    (staged-alphas data)
    (staged-degree data)
    (staged-composition-fri-roots data)
    (staged-composition-final data) ∧
  index (to-nat raw) ∈
    staged-query-prefix-candidate-opening-query-target-from-prefix
    ((replicate rounds []) [i := trace-openings])
    ((replicate rounds []) [i := composition-openings])
    i prefix prefix-state
proof (rule
  checked-staged-security-with-query-prefix-fixed-header-component-supportE
  [OF support fixed])
assume E: E ?out
and continuation:
  Some (((data, attacker-state), result), final-state) ∈
    set-dist
    (execute
      (checked-staged-after-query-prefix-receive-with-verifier A i
        ((prefix, prefix-state), raw))
      raw-state)
and alphas-eq: staged-alphas data = sqp-alphas prefix

```

```

and comp-nonempty: staged-composition-fri-roots data  $\neq \square$ 
and witness:
  (trace-openings, composition-openings)  $\in$ 
    query-header-supported-partial-opening-witnesses ?s
      (staged-trace-root data)
      (staged-trace-fri-roots data)
      (staged-trace-final data)
      (staged-alphas data)
      (staged-degree data)
      (staged-composition-fri-roots data)
      (staged-composition-final data)
and target:
  index (to-nat raw)  $\in$ 
    staged-query-prefix-candidate-opening-query-target-from-prefix
      ((replicate rounds  $\square$ ) [i := trace-openings])
      ((replicate rounds  $\square$ ) [i := composition-openings])
      i prefix prefix-state
show ?thesis
  using E continuation alphas-eq comp-nonempty witness target by simp
qed
have E: E ?out
  using support-facts by blast
have continuation:
  Some (((data, attacker-state), result), final-state)  $\in$ 
    set-dist
      (execute
        (checked-staged-after-query-prefix-receive-with-verifier A i
          ((prefix, prefix-state), raw))
        raw-state)
  using support-facts by blast
have alphas-eq: staged-alphas data = sqp-alphas prefix
  using support-facts by blast
have comp-nonempty: staged-composition-fri-roots data  $\neq \square$ 
  using support-facts by blast
have witness:
  (trace-openings, composition-openings)  $\in$ 
    query-header-supported-partial-opening-witnesses ?s
      (staged-trace-root data)
      (staged-trace-fri-roots data)
      (staged-trace-final data)
      (staged-alphas data)
      (staged-degree data)
      (staged-composition-fri-roots data)
      (staged-composition-final data)
  using support-facts by blast
have target:
  index (to-nat raw)  $\in$ 
    staged-query-prefix-candidate-opening-query-target-from-prefix
      ((replicate rounds  $\square$ ) [i := trace-openings])

```

```

      ((replicate rounds []) [i := composition-openings])
      i prefix prefix-state
    using support-facts by blast
  from checked-staged-after-query-prefix-receive-with-verifier-outcome(2)
    [OF continuation]
  have current-verifier:
    Some (result, final-state) ∈
      set-dist (execute verify-monad ?s)
    .
  have s-ext-final: ?s ≤ final-state
    by (rule verify-monad-hash-extends[OF current-verifier])
  from witness obtain witness-result witness-state rest where
    witness-out:
      Some (witness-result, witness-state) ∈
        set-dist (execute verify-monad ?s)
  and trace-table-witness:
    partial-authenticated-table (staged-trace-root data)
      (scale * clength) trace-openings witness-state
  and comp-table-witness:
    partial-authenticated-table (hd (staged-composition-fri-roots data))
      (scale * clength) composition-openings witness-state
  and header:
    verifier-header-transcript ?s
      (staged-trace-root data)
      (staged-trace-fri-roots data)
      (staged-trace-final data)
      (staged-alphas data)
      (staged-degree data)
      (staged-composition-fri-roots data)
      (staged-composition-final data)
      rest
    by (rule query-header-supported-partial-opening-witnessesE)
  have s-ext-witness: ?s ≤ witness-state
    by (rule verify-monad-hash-extends[OF witness-out])
  have consistent:
    partial-query-openings-consistent trace-openings composition-openings
      (staged-alphas data) (index (to-nat raw))
  proof -
    have target-success:
      index (to-nat raw) ∈
        partial-query-success-indices-at
          ((replicate rounds []) [i := trace-openings])
          ((replicate rounds []) [i := composition-openings])
          (sqp-alphas prefix) i
    using target
    unfolding staged-query-prefix-candidate-opening-query-target-from-prefix-def
    by simp
  then have round:
    partial-query-round-consistent

```

```

    ((replicate rounds []) [i := trace-openings])
    ((replicate rounds []) [i := composition-openings])
    (sqp-alphas prefix) i (index (to-nat raw))
  unfolding partial-query-success-indices-at-def by simp
have trace-nth:
  ((replicate rounds []) [i := trace-openings]) ! i = trace-openings
  using i-bound by simp
have comp-nth:
  ((replicate rounds []) [i := composition-openings]) ! i =
    composition-openings
  using i-bound by simp
have idx-sample: index (to-nat raw) ∈ query-sample-space
  using round unfolding partial-query-round-consistent-def by simp
have trace-indices-full:
  map opening-index
    (((replicate rounds []) [i := trace-openings]) ! i) =
    powers-scaled (index (to-nat raw))
  using round unfolding partial-query-round-consistent-def by blast
have comp-indices-full:
  map opening-index
    (((replicate rounds []) [i := composition-openings]) ! i) =
    [index (to-nat raw),
     fri-sibling-index (scale * clength) (index (to-nat raw))]
  using round unfolding partial-query-round-consistent-def by blast
have comp-nonempty-full:
  ((replicate rounds []) [i := composition-openings]) ! i ≠ []
  using round unfolding partial-query-round-consistent-def by blast
have comp-value-full:
  opening-value
    (((replicate rounds []) [i := composition-openings]) ! i ! 0) =
    cp-eval (sqp-alphas prefix)
    (map opening-value
      (((replicate rounds []) [i := trace-openings]) ! i))
    (h ^ index (to-nat raw) * shift)
  using round unfolding partial-query-round-consistent-def by blast
have
  partial-query-openings-consistent trace-openings composition-openings
    (sqp-alphas prefix) (index (to-nat raw))
  unfolding partial-query-openings-consistent-def
  using idx-sample trace-indices-full comp-indices-full comp-nonempty-full
    comp-value-full trace-nth comp-nth
  by simp
then show ?thesis
  using alphas-eq by simp
qed
have trace-pullback:
  partial-authenticated-table (staged-trace-root data)
    (scale * clength) trace-openings ?s ∨
  (∃ opn ∈ set trace-openings.

```

```

    hash-map-new-output-hit
      (merkle-path-target-roots witness-state)
      (staged-trace-root data)
      (scale * clength)
      (opening-index opn)
      (opening-value opn)
      (opening-path opn)) ?s witness-state)
  by (rule partial-authenticated-table-pullback-or-new-output-hit
      [OF s-ext-witness trace-table-witness])
then show ?thesis
proof
  assume trace-table-s:
    partial-authenticated-table (staged-trace-root data)
    (scale * clength) trace-openings ?s
  have comp-pullback:
    partial-authenticated-table (hd (staged-composition-fri-roots data))
    (scale * clength) composition-openings ?s ∨
    (∃ opn ∈ set composition-openings.
      hash-map-new-output-hit
        (merkle-path-target-roots witness-state)
        (hd (staged-composition-fri-roots data))
        (scale * clength)
        (opening-index opn)
        (opening-value opn)
        (opening-path opn)) ?s witness-state)
  by (rule partial-authenticated-table-pullback-or-new-output-hit
      [OF s-ext-witness comp-table-witness])
then show ?thesis
proof
  assume comp-table-s:
    partial-authenticated-table
      (hd (staged-composition-fri-roots data))
      (scale * clength) composition-openings ?s
  have trace-table-final:
    partial-authenticated-table (staged-trace-root data)
    (scale * clength) trace-openings final-state
  by (rule partial-authenticated-table-mono[OF trace-table-s s-ext-final])
  have comp-table-final:
    partial-authenticated-table
      (hd (staged-composition-fri-roots data))
      (scale * clength) composition-openings final-state
  by (rule partial-authenticated-table-mono[OF comp-table-s s-ext-final])
  have current:
    checked-staged-security-with-query-prefix-authenticated-opening-hit
      trace-openings composition-openings i ?out
  unfolding
    checked-staged-security-with-query-prefix-authenticated-opening-hit-def
  using comp-nonempty trace-table-final comp-table-final consistent
  by simp

```

```

then show ?thesis
  using not-current by contradiction
next
assume comp-path:
   $\exists \text{opn} \in \text{set composition-openings.}$ 
  hash-map-new-output-hit
    (merkle-path-target-roots witness-state
      (hd (staged-composition-fri-roots data)) (scale * clength)
      (opening-index opn) (opening-value opn) (opening-path opn))
    ?s witness-state
have body:
   $\exists \text{witness-result witness-state rest.}$ 
  let s = ?s in
  Some (witness-result, witness-state)  $\in$ 
    set-dist (execute verify-monad s)  $\wedge$ 
  verifier-header-transcript s
    (staged-trace-root data)
    (staged-trace-fri-roots data)
    (staged-trace-final data)
    (staged-alphas data)
    (staged-degree data)
    (staged-composition-fri-roots data)
    (staged-composition-final data)
    rest  $\wedge$ 
  (( $\exists \text{opn} \in \text{set trace-openings.}$ 
    hash-map-new-output-hit
      (merkle-path-target-roots witness-state
        (staged-trace-root data)
        (scale * clength)
        (opening-index opn)
        (opening-value opn)
        (opening-path opn))
      s witness-state)  $\vee$ 
    ( $\exists \text{opn} \in \text{set composition-openings.}$ 
      hash-map-new-output-hit
        (merkle-path-target-roots witness-state
          (hd (staged-composition-fri-roots data))
          (scale * clength)
          (opening-index opn)
          (opening-value opn)
          (opening-path opn))
        s witness-state))
  by (intro exI[of - witness-result] exI[of - witness-state]
    exI[of - rest])
    (simp add: witness-out header comp-path)
show ?thesis
unfolding
  checked-staged-security-with-query-prefix-header-witness-path-output-hit-def
using E comp-nonempty body by simp

```

```

qed
next
assume trace-path:
  ∃ opn ∈ set trace-openings.
    hash-map-new-output-hit
      (merkle-path-target-roots witness-state
        (staged-trace-root data) (scale * clength)
        (opening-index opn) (opening-value opn) (opening-path opn))
      ?s witness-state
have body:
  ∃ witness-result witness-state rest.
    let s = ?s in
    Some (witness-result, witness-state) ∈
      set-dist (execute verify-monad s) ∧
    verifier-header-transcript s
      (staged-trace-root data)
      (staged-trace-fri-roots data)
      (staged-trace-final data)
      (staged-alphas data)
      (staged-degree data)
      (staged-composition-fri-roots data)
      (staged-composition-final data)
      rest ∧
    ((∃ opn ∈ set trace-openings.
      hash-map-new-output-hit
        (merkle-path-target-roots witness-state
          (staged-trace-root data)
          (scale * clength)
          (opening-index opn)
          (opening-value opn)
          (opening-path opn))
        s witness-state) ∨
      (∃ opn ∈ set composition-openings.
        hash-map-new-output-hit
          (merkle-path-target-roots witness-state
            (hd (staged-composition-fri-roots data))
            (scale * clength)
            (opening-index opn)
            (opening-value opn)
            (opening-path opn))
          s witness-state))
    by (intro exI[of - witness-result] exI[of - witness-state]
      exI[of - rest])
      (simp add: witness-out header trace-path)
show ?thesis
unfolding
  checked-staged-security-with-query-prefix-header-witness-path-output-hit-def
using E comp-nonempty body by simp
qed

```

qed

lemma *checked-staged-security-with-query-prefix-fixed-current-authenticated-component-imp-prefix-or-path-output-on*

assumes *wf: staged-budget-wellformed budgets*

and *controlled: staged-adversary-controlled budgets A*

and *i-bound: i < rounds*

and *support:*

out ∈

set-dist

(*execute*

(*checked-staged-security-experiment-with-query-prefix-data-state A i*)

adversary-initial-state)

and *hit:*

checked-staged-security-with-query-prefix-fixed-current-authenticated-component

E trace-openings composition-openings i out

shows

checked-staged-security-with-query-prefix-prefix-authenticated-opening-hit

trace-openings composition-openings i out ∨

checked-staged-security-with-query-prefix-opening-path-output-hit

trace-openings composition-openings out

by (*rule*

checked-staged-security-with-query-prefix-authenticated-opening-hit-imp-prefix-or-path-output-hit-on-support

[*OF wf controlled i-bound support*])

(*rule*

checked-staged-security-with-query-prefix-fixed-current-authenticated-component-imp-current

[*OF hit*])

lemma *checked-staged-security-with-query-prefix-fixed-header-component-bound:*

assumes *raw-bound: query-index-raw-preimage-bound*

and *wf: staged-budget-wellformed budgets*

and *controlled: staged-adversary-controlled budgets A*

and *i-bound: i < rounds*

shows

wp-event

(*checked-staged-security-experiment-with-query-prefix-data-state A i*)

(*checked-staged-security-with-query-prefix-fixed-header-component*

E trace-openings composition-openings i)

adversary-initial-state ≤

staged-phase-relation-error size

(*staged-query-search-queries budgets i + 1*) +

(*1::prob*) / *nnreal (card query-sample-space)*

proof –

have *event-le:*

wp-event

(*checked-staged-security-experiment-with-query-prefix-data-state A i*)

(*checked-staged-security-with-query-prefix-fixed-header-component*

E trace-openings composition-openings i)

adversary-initial-state ≤

wp-event


```

    (checked-staged-security-experiment-with-query-prefix-data-state A i)
    (checked-staged-security-with-actual-query-prefix-candidate-opening-hit
      ((replicate rounds []) [i := trace-openings])
      ((replicate rounds []) [i := composition-openings])
    i)
  adversary-initial-state
proof (rule wp-event-mono)
fix out
assume
  checked-staged-security-with-query-prefix-fixed-header-component
  E trace-openings composition-openings i out
then have
  checked-staged-security-with-query-prefix-header-authenticated-candidate-opening-hit
  trace-openings composition-openings i out
by (rule
  checked-staged-security-with-query-prefix-fixed-header-component-imp-header)
then show
  checked-staged-security-with-actual-query-prefix-candidate-opening-hit
  ((replicate rounds []) [i := trace-openings])
  ((replicate rounds []) [i := composition-openings])
  i out
by (rule
  checked-staged-security-with-query-prefix-header-authenticated-candidate-opening-hit-imp-actual)
qed
have actual-bound:
  wp-event
  (checked-staged-security-experiment-with-query-prefix-data-state A i)
  (checked-staged-security-with-actual-query-prefix-candidate-opening-hit
    ((replicate rounds []) [i := trace-openings])
    ((replicate rounds []) [i := composition-openings])
  i)
  adversary-initial-state ≤
  staged-phase-relation-error size
  (staged-query-search-queries budgets i + 1) +
  (1::prob) / nnreal (card query-sample-space)
by (rule checked-staged-security-with-actual-query-prefix-candidate-opening-hit-bound
  [OF raw-bound wf controlled])
  (use i-bound in simp)
show ?thesis
by (rule order-trans[OF event-le actual-bound])
qed

lemma checked-staged-security-with-query-prefix-fixed-header-component-bound-from-budgets:
assumes wf: staged-budget-wellformed budgets
and controlled: staged-adversary-controlled budgets A
and i-bound: i < rounds
shows
  wp-event
  (checked-staged-security-experiment-with-query-prefix-data-state A i)

```

```

    (checked-staged-security-with-query-prefix-fixed-header-component
      E trace-openings composition-openings i)
    adversary-initial-state ≤
    staged-phase-relation-error size
    (staged-query-search-queries budgets i + 1) +
    (1::prob) / nnreal (card query-sample-space)
proof –
  have raw-bound: query-index-raw-preimage-bound
  by (rule query-index-raw-preimage-bound-from-sampler-wellformed)
  show ?thesis
  by (rule
    checked-staged-security-with-query-prefix-fixed-header-component-bound
    [OF raw-bound wf controlled i-bound])
qed

lemma checked-staged-security-with-query-prefix-fixed-current-authenticated-component-bound:
assumes raw-bound: query-index-raw-preimage-bound
and wf: staged-budget-wellformed budgets
and controlled: staged-adversary-controlled budgets A
and i-bound: i < rounds
shows
  wp-event
  (checked-staged-security-experiment-with-query-prefix-data-state A i)
  (checked-staged-security-with-query-prefix-fixed-current-authenticated-component
    E trace-openings composition-openings i)
  adversary-initial-state ≤
  staged-phase-relation-error size
  (staged-query-search-queries budgets i + 1) +
  (1::prob) / nnreal (card query-sample-space)
proof –
  have event-le:
    wp-event
    (checked-staged-security-experiment-with-query-prefix-data-state A i)
    (checked-staged-security-with-query-prefix-fixed-current-authenticated-component
      E trace-openings composition-openings i)
    adversary-initial-state ≤
  wp-event
  (checked-staged-security-experiment-with-query-prefix-data-state A i)
  (checked-staged-security-with-query-prefix-authenticated-opening-hit
    trace-openings composition-openings i)
  adversary-initial-state
  by (rule wp-event-mono)
  (rule
    checked-staged-security-with-query-prefix-fixed-current-authenticated-component-imp-current)
have current-bound:
  wp-event
  (checked-staged-security-experiment-with-query-prefix-data-state A i)
  (checked-staged-security-with-query-prefix-authenticated-opening-hit
    trace-openings composition-openings i)

```

adversary-initial-state $\leq$
staged-phase-relation-error size
 (*staged-query-search-queries budgets* $i + 1$) +
 ($1::\text{prob}$) / *nnreal* (*card query-sample-space*)
by (*rule checked-staged-security-with-query-prefix-authenticated-opening-hit-bound*
 [*OF raw-bound wf controlled i-bound*])
show ?thesis
by (*rule order-trans* [*OF event-le current-bound*])
qed

lemma *checked-staged-security-with-query-prefix-fixed-current-authenticated-component-bound-from-budgets:*

assumes *wf: staged-budget-wellformed budgets*
and *controlled: staged-adversary-controlled budgets A*
and *i-bound: i < rounds*

shows

wp-event
 (*checked-staged-security-experiment-with-query-prefix-data-state A i*)
 (*checked-staged-security-with-query-prefix-fixed-current-authenticated-component*
E trace-openings composition-openings i)
adversary-initial-state $\leq$
staged-phase-relation-error size
 (*staged-query-search-queries budgets* $i + 1$) +
 ($1::\text{prob}$) / *nnreal* (*card query-sample-space*)

proof –

have *raw-bound: query-index-raw-preimage-bound*
by (*rule query-index-raw-preimage-bound-from-sampler-wellformed*)
show ?thesis
by (*rule*
checked-staged-security-with-query-prefix-fixed-current-authenticated-component-bound
 [*OF raw-bound wf controlled i-bound*])

qed

lemma *checked-staged-security-with-query-prefix-fixed-header-component-imp-current-or-witness-path-output-on-supp*

assumes *wf: staged-budget-wellformed budgets*
and *controlled: staged-adversary-controlled budgets A*
and *i-bound: i < rounds*

and *support:*

out $\in$
set-dist
 (*execute*
 (*checked-staged-security-experiment-with-query-prefix-data-state A i*)
adversary-initial-state)

and *hit:*

checked-staged-security-with-query-prefix-fixed-header-component
E trace-openings composition-openings i out

shows

checked-staged-security-with-query-prefix-fixed-current-authenticated-component
E trace-openings composition-openings i out $\vee$
checked-staged-security-with-query-prefix-header-witness-path-output-hit

```

      E trace-openings composition-openings i out
proof (cases out)
  case None
  then have False
    using hit
    unfolding
      checked-staged-security-with-query-prefix-fixed-header-component-def
      checked-staged-security-with-query-prefix-header-authenticated-candidate-opening-hit-def
    by simp
  then show ?thesis by simp
next
  case (Some packed)
  obtain prefix prefix-state raw raw-state data attacker-state result final-state
  where out-eq:
    out =
      Some (((((prefix, prefix-state), raw), raw-state),
        ((data, attacker-state), result)), final-state)
    using Some by (cases packed) (auto split: prod.splits)
  from checked-staged-security-with-query-prefix-fixed-header-component-split-current
    [OF hit]
  show ?thesis
proof
  assume current:
    checked-staged-security-with-query-prefix-fixed-current-authenticated-component
    E trace-openings composition-openings i out
  then show ?thesis by simp
next
  assume not-current:
    checked-staged-security-with-query-prefix-header-witness-not-current
    E trace-openings composition-openings i out
  have support':
    Some (((((prefix, prefix-state), raw), raw-state),
      ((data, attacker-state), result)), final-state) ∈
    set-dist
      (execute
        (checked-staged-security-experiment-with-query-prefix-data-state A i)
        adversary-initial-state)
  using support out-eq by simp
  have not-current':
    checked-staged-security-with-query-prefix-header-witness-not-current
    E trace-openings composition-openings i
    (Some (((((prefix, prefix-state), raw), raw-state),
      ((data, attacker-state), result)), final-state))
  using not-current out-eq by simp
  have path:
    checked-staged-security-with-query-prefix-header-witness-path-output-hit
    E trace-openings composition-openings i
    (Some (((((prefix, prefix-state), raw), raw-state),
      ((data, attacker-state), result)), final-state))

```

```

    by (rule
      checked-staged-security-with-query-prefix-header-witness-not-current-imp-path-output-on-support
      [OF wf controlled i-bound support' not-current])
  then show ?thesis
    using out-eq by simp
qed
qed

lemma checked-staged-security-with-query-prefix-fixed-header-component-bound-from-current-and-witness-path-output-on-support
  assumes wf: staged-budget-wellformed budgets
  and controlled: staged-adversary-controlled budgets A
  and i-bound:  $i < \text{rounds}$ 
  and current-bound:
    wp-event
    (checked-staged-security-experiment-with-query-prefix-data-state A i)
    (checked-staged-security-with-query-prefix-fixed-current-authenticated-component
      E trace-openings composition-openings i)
    adversary-initial-state  $\leq P$ 
  and path-bound:
    wp-event
    (checked-staged-security-experiment-with-query-prefix-data-state A i)
    (checked-staged-security-with-query-prefix-header-witness-path-output-hit
      E trace-openings composition-openings i)
    adversary-initial-state  $\leq Q$ 
  shows
    wp-event
    (checked-staged-security-experiment-with-query-prefix-data-state A i)
    (checked-staged-security-with-query-prefix-fixed-header-component
      E trace-openings composition-openings i)
    adversary-initial-state  $\leq P + Q$ 
proof -
  have event-le:
    wp-event
    (checked-staged-security-experiment-with-query-prefix-data-state A i)
    (checked-staged-security-with-query-prefix-fixed-header-component
      E trace-openings composition-openings i)
    adversary-initial-state  $\leq$ 
    wp-event
    (checked-staged-security-experiment-with-query-prefix-data-state A i)
    ( $\lambda \text{out.}$ 
      checked-staged-security-with-query-prefix-fixed-current-authenticated-component
      E trace-openings composition-openings i out  $\vee$ 
      checked-staged-security-with-query-prefix-header-witness-path-output-hit
      E trace-openings composition-openings i out)
    adversary-initial-state
  by (rule wp-event-mono-on-support)
  (rule
    checked-staged-security-with-query-prefix-fixed-header-component-imp-current-or-witness-path-output-on-support
    [OF wf controlled i-bound])

```

have *union-bound*:
wp-event
 (*checked-staged-security-experiment-with-query-prefix-data-state* *A i*)
 ($\lambda out.$
 checked-staged-security-with-query-prefix-fixed-current-authenticated-component
 E trace-openings composition-openings i out $\vee$
 checked-staged-security-with-query-prefix-header-witness-path-output-hit
 E trace-openings composition-openings i out)
adversary-initial-state $\leq$
wp-event
 (*checked-staged-security-experiment-with-query-prefix-data-state* *A i*)
 (*checked-staged-security-with-query-prefix-fixed-current-authenticated-component*
 E trace-openings composition-openings i)
adversary-initial-state +
wp-event
 (*checked-staged-security-experiment-with-query-prefix-data-state* *A i*)
 (*checked-staged-security-with-query-prefix-header-witness-path-output-hit*
 E trace-openings composition-openings i)
adversary-initial-state
by (rule *wp-event-union-bound*)
have *sum-bound*:
wp-event
 (*checked-staged-security-experiment-with-query-prefix-data-state* *A i*)
 (*checked-staged-security-with-query-prefix-fixed-current-authenticated-component*
 E trace-openings composition-openings i)
adversary-initial-state +
wp-event
 (*checked-staged-security-experiment-with-query-prefix-data-state* *A i*)
 (*checked-staged-security-with-query-prefix-header-witness-path-output-hit*
 E trace-openings composition-openings i)
adversary-initial-state $\leq P + Q$
by (intro *add-mono current-bound path-bound*)
show *?thesis*
by (rule *order-trans*[*OF event-le*])
 (rule *order-trans*[*OF union-bound sum-bound*])
qed

lemma *checked-staged-security-with-query-prefix-fixed-header-component-imp-current-or-witness-transcript-hit-on-su*

assumes *wf*: *staged-budget-wellformed* *budgets*
and *controlled*: *staged-adversary-controlled* *budgets A*
and *i-bound*: *i < rounds*
and *support*:
 out $\in$
 set-dist
 (*execute*
 (*checked-staged-security-experiment-with-query-prefix-data-state* *A i*)
 adversary-initial-state)
and *hit*:
 checked-staged-security-with-query-prefix-fixed-header-component

E trace-openings composition-openings i out
shows
checked-staged-security-with-query-prefix-fixed-current-authenticated-component
E trace-openings composition-openings i out $\vee$
checked-staged-security-with-query-prefix-header-witness-transcript-hit
E out
proof –
have *split*:
checked-staged-security-with-query-prefix-fixed-current-authenticated-component
E trace-openings composition-openings i out $\vee$
checked-staged-security-with-query-prefix-header-witness-path-output-hit
E trace-openings composition-openings i out
by (rule
checked-staged-security-with-query-prefix-fixed-header-component-imp-current-or-witness-path-output-on-support
 $[OF\ wf\ controlled\ i-bound\ support\ hit])$
then show *?thesis*
proof
assume
checked-staged-security-with-query-prefix-fixed-current-authenticated-component
E trace-openings composition-openings i out
then show *?thesis* **by** *simp*
next
assume *path*:
checked-staged-security-with-query-prefix-header-witness-path-output-hit
E trace-openings composition-openings i out
have
checked-staged-security-with-query-prefix-header-witness-transcript-hit
E out
by (rule
checked-staged-security-with-query-prefix-header-witness-path-output-imp-transcript-hit
 $[OF\ path])$
then show *?thesis* **by** *simp*
qed
qed

lemma *checked-staged-security-with-query-prefix-fixed-header-component-bound-from-current-and-witness-transcript-*
assumes *wf: staged-budget-wellformed budgets*
and *controlled: staged-adversary-controlled budgets A*
and *i-bound: i < rounds*
and *current-bound:*
wp-event
(checked-staged-security-experiment-with-query-prefix-data-state A i)
(checked-staged-security-with-query-prefix-fixed-current-authenticated-component
E trace-openings composition-openings i)
adversary-initial-state $\leq P$
and *transcript-bound:*
wp-event
(checked-staged-security-experiment-with-query-prefix-data-state A i)
(checked-staged-security-with-query-prefix-header-witness-transcript-hit

$E)$
 $\text{adversary-initial-state} \leq Q$
shows
 wp-event
 $(\text{checked-staged-security-experiment-with-query-prefix-data-state } A \ i)$
 $(\text{checked-staged-security-with-query-prefix-fixed-header-component}$
 $\quad E \text{ trace-openings composition-openings } i)$
 $\text{adversary-initial-state} \leq P + Q$
proof –
have *event-le*:
 wp-event
 $(\text{checked-staged-security-experiment-with-query-prefix-data-state } A \ i)$
 $(\text{checked-staged-security-with-query-prefix-fixed-header-component}$
 $\quad E \text{ trace-openings composition-openings } i)$
 $\text{adversary-initial-state} \leq$
 wp-event
 $(\text{checked-staged-security-experiment-with-query-prefix-data-state } A \ i)$
 $(\lambda \text{out.}$
 $\quad \text{checked-staged-security-with-query-prefix-fixed-current-authenticated-component}$
 $\quad \quad E \text{ trace-openings composition-openings } i \text{ out } \vee$
 $\quad \text{checked-staged-security-with-query-prefix-header-witness-transcript-hit}$
 $\quad \quad E \text{ out})$
 $\text{adversary-initial-state}$
by (rule *wp-event-mono-on-support*)
 $(\text{rule}$
 $\quad \text{checked-staged-security-with-query-prefix-fixed-header-component-imp-current-or-witness-transcript-hit-on-support}$
 $\quad \quad [OF \text{ wf controlled } i\text{-bound}])$
have *union-bound*:
 wp-event
 $(\text{checked-staged-security-experiment-with-query-prefix-data-state } A \ i)$
 $(\lambda \text{out.}$
 $\quad \text{checked-staged-security-with-query-prefix-fixed-current-authenticated-component}$
 $\quad \quad E \text{ trace-openings composition-openings } i \text{ out } \vee$
 $\quad \text{checked-staged-security-with-query-prefix-header-witness-transcript-hit}$
 $\quad \quad E \text{ out})$
 $\text{adversary-initial-state} \leq$
 wp-event
 $(\text{checked-staged-security-experiment-with-query-prefix-data-state } A \ i)$
 $(\text{checked-staged-security-with-query-prefix-fixed-current-authenticated-component}$
 $\quad E \text{ trace-openings composition-openings } i)$
 $\text{adversary-initial-state} +$
 wp-event
 $(\text{checked-staged-security-experiment-with-query-prefix-data-state } A \ i)$
 $(\text{checked-staged-security-with-query-prefix-header-witness-transcript-hit}$
 $\quad E)$
 $\text{adversary-initial-state}$
by (rule *wp-event-union-bound*)
have *sum-bound*:
 wp-event

(checked-staged-security-experiment-with-query-prefix-data-state A i)
 (checked-staged-security-with-query-prefix-fixed-current-authenticated-component
 E trace-openings composition-openings i)
 adversary-initial-state +
 wp-event
 (checked-staged-security-experiment-with-query-prefix-data-state A i)
 (checked-staged-security-with-query-prefix-header-witness-transcript-hit
 E)
 adversary-initial-state $\leq P + Q$
by (intro add-mono current-bound transcript-bound)
show ?thesis
by (rule order-trans[OF event-le])
 (rule order-trans[OF union-bound sum-bound])
qed

lemma checked-staged-security-with-query-prefix-fixed-header-component-bound-from-current-pre-and-witness-new-hit

assumes wf: staged-budget-wellformed budgets

and controlled: staged-adversary-controlled budgets A

and i-bound: $i < \text{rounds}$

and current-bound:

wp-event

(checked-staged-security-experiment-with-query-prefix-data-state A i)

(checked-staged-security-with-query-prefix-fixed-current-authenticated-component
 E trace-openings composition-openings i)

adversary-initial-state $\leq P$

and pre-bound:

wp-event

(checked-staged-security-experiment-with-query-prefix-data-state A i)

(checked-staged-security-with-query-prefix-header-witness-transcript-pre-hit
 E)

adversary-initial-state $\leq R$

and new-bound:

wp-event

(checked-staged-security-experiment-with-query-prefix-data-state A i)

(checked-staged-security-with-query-prefix-header-witness-transcript-new-hit
 E)

adversary-initial-state $\leq Q$

shows

wp-event

(checked-staged-security-experiment-with-query-prefix-data-state A i)

(checked-staged-security-with-query-prefix-fixed-header-component
 E trace-openings composition-openings i)

adversary-initial-state $\leq P + R + Q$

proof –

have transcript-bound:

wp-event

(checked-staged-security-experiment-with-query-prefix-data-state A i)

(checked-staged-security-with-query-prefix-header-witness-transcript-hit
 E)

$adversary-initial-state \leq R + Q$
proof –
have *event-le*:
wp-event
 (checked-staged-security-experiment-with-query-prefix-data-state $A \ i$)
 (checked-staged-security-with-query-prefix-header-witness-transcript-hit
 E)
 $adversary-initial-state \leq$
wp-event
 (checked-staged-security-experiment-with-query-prefix-data-state $A \ i$)
 ($\lambda out.$
 checked-staged-security-with-query-prefix-header-witness-transcript-pre-hit
 $E \ out \vee$
 checked-staged-security-with-query-prefix-header-witness-transcript-new-hit
 $E \ out$)
 $adversary-initial-state$
by (rule *wp-event-mono*)
 (rule
 checked-staged-security-with-query-prefix-header-witness-transcript-hit-imp-pre-or-new)
have *union-bound*:
wp-event
 (checked-staged-security-experiment-with-query-prefix-data-state $A \ i$)
 ($\lambda out.$
 checked-staged-security-with-query-prefix-header-witness-transcript-pre-hit
 $E \ out \vee$
 checked-staged-security-with-query-prefix-header-witness-transcript-new-hit
 $E \ out$)
 $adversary-initial-state \leq$
wp-event
 (checked-staged-security-experiment-with-query-prefix-data-state $A \ i$)
 (checked-staged-security-with-query-prefix-header-witness-transcript-pre-hit
 E)
 $adversary-initial-state +$
wp-event
 (checked-staged-security-experiment-with-query-prefix-data-state $A \ i$)
 (checked-staged-security-with-query-prefix-header-witness-transcript-new-hit
 E)
 $adversary-initial-state$
by (rule *wp-event-union-bound*)
have *sum-bound*:
wp-event
 (checked-staged-security-experiment-with-query-prefix-data-state $A \ i$)
 (checked-staged-security-with-query-prefix-header-witness-transcript-pre-hit
 E)
 $adversary-initial-state +$
wp-event
 (checked-staged-security-experiment-with-query-prefix-data-state $A \ i$)
 (checked-staged-security-with-query-prefix-header-witness-transcript-new-hit
 E)

```

    adversary-initial-state  $\leq R + Q$ 
  by (intro add-mono pre-bound new-bound)
show ?thesis
  by (rule order-trans[OF event-le])
    (rule order-trans[OF union-bound sum-bound])
qed
have header-bound:
  wp-event
    (checked-staged-security-experiment-with-query-prefix-data-state A i)
    (checked-staged-security-with-query-prefix-fixed-header-component
      E trace-openings composition-openings i)
    adversary-initial-state  $\leq P + (R + Q)$ 
  by (rule
    checked-staged-security-with-query-prefix-fixed-header-component-bound-from-current-and-witness-transcript-hit
    [OF wf controlled i-bound current-bound transcript-bound])
then show ?thesis
  by (simp add: algebra-simps)
qed

lemma checked-staged-security-with-query-prefix-fixed-header-component-bound-from-witness-path-output:
  assumes raw-bound: query-index-raw-preimage-bound
  and wf: staged-budget-wellformed budgets
  and controlled: staged-adversary-controlled budgets A
  and i-bound:  $i < \text{rounds}$ 
  and path-bound:
    wp-event
      (checked-staged-security-experiment-with-query-prefix-data-state A i)
      (checked-staged-security-with-query-prefix-header-witness-path-output-hit
        E trace-openings composition-openings i)
      adversary-initial-state  $\leq Q$ 
  shows
    wp-event
      (checked-staged-security-experiment-with-query-prefix-data-state A i)
      (checked-staged-security-with-query-prefix-fixed-header-component
        E trace-openings composition-openings i)
      adversary-initial-state  $\leq$ 
      staged-phase-relation-error size
      (staged-query-search-queries budgets  $i + 1$ ) +
       $(1::\text{prob}) / \text{nnreal} (\text{card query-sample-space}) + Q$ 
proof -
  have current-bound:
    wp-event
      (checked-staged-security-experiment-with-query-prefix-data-state A i)
      (checked-staged-security-with-query-prefix-fixed-current-authenticated-component
        E trace-openings composition-openings i)
      adversary-initial-state  $\leq$ 
      staged-phase-relation-error size
      (staged-query-search-queries budgets  $i + 1$ ) +
       $(1::\text{prob}) / \text{nnreal} (\text{card query-sample-space})$ 

```

by (rule
 checked-staged-security-with-query-prefix-fixed-current-authenticated-component-bound
 [*OF raw-bound wf controlled i-bound*])
 show ?thesis
 by (rule
 checked-staged-security-with-query-prefix-fixed-header-component-bound-from-current-and-witness-path-output
 [*OF wf controlled i-bound current-bound path-bound*])
 qed

lemma *checked-staged-security-with-query-prefix-fixed-header-component-bound-from-witness-path-output-and-budget*
assumes *wf: staged-budget-wellformed budgets*
and *controlled: staged-adversary-controlled budgets A*
and *i-bound: i < rounds*
and *path-bound:*
 wp-event
 (*checked-staged-security-experiment-with-query-prefix-data-state A i*)
 (*checked-staged-security-with-query-prefix-header-witness-path-output-hit*
 E trace-openings composition-openings i)
 adversary-initial-state ≤ Q
shows
 wp-event
 (*checked-staged-security-experiment-with-query-prefix-data-state A i*)
 (*checked-staged-security-with-query-prefix-fixed-header-component*
 E trace-openings composition-openings i)
 adversary-initial-state ≤
 staged-phase-relation-error size
 (*staged-query-search-queries budgets i + 1*) +
 (*1::prob*) / *nnreal (card query-sample-space) + Q*
proof –
have *raw-bound: query-index-raw-preimage-bound*
by (rule *query-index-raw-preimage-bound-from-sampler-wellformed*)
show ?thesis
by (rule
 checked-staged-security-with-query-prefix-fixed-header-component-bound-from-witness-path-output
 [*OF raw-bound wf controlled i-bound path-bound*])
 qed

lemma *checked-staged-security-with-query-prefix-fixed-header-component-bound-from-witness-transcript-hit*
assumes *raw-bound: query-index-raw-preimage-bound*
and *wf: staged-budget-wellformed budgets*
and *controlled: staged-adversary-controlled budgets A*
and *i-bound: i < rounds*
and *transcript-bound:*
 wp-event
 (*checked-staged-security-experiment-with-query-prefix-data-state A i*)
 (*checked-staged-security-with-query-prefix-header-witness-transcript-hit*
 E)
 adversary-initial-state ≤ Q
shows

wp-event
 (checked-staged-security-experiment-with-query-prefix-data-state A i)
 (checked-staged-security-with-query-prefix-fixed-header-component
 E trace-openings composition-openings i)
 adversary-initial-state $\leq$
 staged-phase-relation-error size
 (staged-query-search-queries budgets $i + 1$) +
 (1::prob) / nnreal (card query-sample-space) + Q

proof –

have current-bound:

wp-event
 (checked-staged-security-experiment-with-query-prefix-data-state A i)
 (checked-staged-security-with-query-prefix-fixed-current-authenticated-component
 E trace-openings composition-openings i)
 adversary-initial-state $\leq$
 staged-phase-relation-error size
 (staged-query-search-queries budgets $i + 1$) +
 (1::prob) / nnreal (card query-sample-space)

by (rule
 checked-staged-security-with-query-prefix-fixed-current-authenticated-component-bound
 [OF raw-bound wf controlled i-bound])

show ?thesis

by (rule
 checked-staged-security-with-query-prefix-fixed-header-component-bound-from-current-and-witness-transcript-hit
 [OF wf controlled i-bound current-bound transcript-bound])

qed

lemma checked-staged-security-with-query-prefix-fixed-header-component-bound-from-witness-transcript-hit-and-budget

assumes wf: staged-budget-wellformed budgets

and controlled: staged-adversary-controlled budgets A

and i-bound: $i < \text{rounds}$

and transcript-bound:

wp-event
 (checked-staged-security-experiment-with-query-prefix-data-state A i)
 (checked-staged-security-with-query-prefix-header-witness-transcript-hit
 E)
 adversary-initial-state $\leq Q$

shows

wp-event
 (checked-staged-security-experiment-with-query-prefix-data-state A i)
 (checked-staged-security-with-query-prefix-fixed-header-component
 E trace-openings composition-openings i)
 adversary-initial-state $\leq$
 staged-phase-relation-error size
 (staged-query-search-queries budgets $i + 1$) +
 (1::prob) / nnreal (card query-sample-space) + Q

proof –

have raw-bound: query-index-raw-preimage-bound

by (rule query-index-raw-preimage-bound-from-sampler-wellformed)

show ?thesis
by (rule
checked-staged-security-with-query-prefix-fixed-header-component-bound-from-witness-transcript-hit
[OF raw-bound wf controlled i-bound transcript-bound])
qed

lemma *checked-staged-security-with-query-prefix-fixed-header-component-bound-from-data-state-pre-and-witness-new*
assumes raw-bound: *query-index-raw-preimage-bound*
and wf: *staged-budget-wellformed budgets*
and controlled: *staged-adversary-controlled budgets A*
and i-bound: $i < \text{rounds}$
and pre-bound:
wp-event (checked-staged-security-experiment-with-data-state A)
staged-security-with-data-state-transcript-pre-hit
adversary-initial-state $\leq R$
and new-bound:
wp-event
(checked-staged-security-experiment-with-query-prefix-data-state A i)
(checked-staged-security-with-query-prefix-header-witness-transcript-new-hit
E)
adversary-initial-state $\leq Q$
shows
wp-event
(checked-staged-security-experiment-with-query-prefix-data-state A i)
(checked-staged-security-with-query-prefix-fixed-header-component
E trace-openings composition-openings i)
adversary-initial-state $\leq$
staged-phase-relation-error size
(staged-query-search-queries budgets $i + 1$) +
(1::prob) / nnreal (card query-sample-space) + R + Q
proof –
have current-bound:
wp-event
(checked-staged-security-experiment-with-query-prefix-data-state A i)
(checked-staged-security-with-query-prefix-fixed-current-authenticated-component
E trace-openings composition-openings i)
adversary-initial-state $\leq$
staged-phase-relation-error size
(staged-query-search-queries budgets $i + 1$) +
(1::prob) / nnreal (card query-sample-space)
by (rule
checked-staged-security-with-query-prefix-fixed-current-authenticated-component-bound
[OF raw-bound wf controlled i-bound])
have pre-hit-bound:
wp-event
(checked-staged-security-experiment-with-query-prefix-data-state A i)
(checked-staged-security-with-query-prefix-header-witness-transcript-pre-hit
E)
adversary-initial-state $\leq R$

by (rule
 checked-staged-security-with-query-prefix-header-witness-transcript-pre-hit-bound-from-data-state-pre-hit
 [*OF i-bound pre-bound*])
 have header-bound:
 wp-event
 (*checked-staged-security-experiment-with-query-prefix-data-state A i*)
 (*checked-staged-security-with-query-prefix-fixed-header-component*
 E trace-openings composition-openings i)
 adversary-initial-state ≤
 (*staged-phase-relation-error size*
 (*staged-query-search-queries budgets i + 1*) +
 (*1::prob*) / *nnreal (card query-sample-space)*) + *R + Q*
 by (rule
 checked-staged-security-with-query-prefix-fixed-header-component-bound-from-current-pre-and-witness-new-hit
 [*OF wf controlled i-bound current-bound pre-hit-bound new-bound*])
 then show ?thesis
 by (simp add: *algebra-simps*)
 qed

lemma *checked-staged-security-with-data-state-bound-by-query-prefix-fixed-header-component*:
assumes *raw-bound: query-index-raw-preimage-bound*
and *wf: staged-budget-wellformed budgets*
and *controlled: staged-adversary-controlled budgets A*
and *i-bound: i < rounds*
and *event-imp:*
 $\bigwedge out.$
 (case out of
 None $\Rightarrow$ *E None*
 | Some (*packed, t*) $\Rightarrow$ *E (Some (snd packed, t))*) $\Longrightarrow$
 checked-staged-security-with-query-prefix-fixed-header-component
 C trace-openings composition-openings i out
shows
 wp-event (*checked-staged-security-experiment-with-data-state A*) *E*
 adversary-initial-state ≤
 staged-phase-relation-error size
 (*staged-query-search-queries budgets i + 1*) +
 (*1::prob*) / *nnreal (card query-sample-space)*
proof –
have *projection*:
 wp-event (*checked-staged-security-experiment-with-data-state A*) *E*
 adversary-initial-state =
 wp-event
 (*checked-staged-security-experiment-with-query-prefix-data-state A i*)
 ($\lambda out.$ case out of
 None $\Rightarrow$ *E None*
 | Some (*packed, t*) $\Rightarrow$ *E (Some (snd packed, t))*)
 adversary-initial-state
by (rule
 checked-staged-security-experiment-with-query-prefix-data-state-projection-event

[OF *i-bound*])
have *lifted-bound*:
 wp-event
 (*checked-staged-security-experiment-with-query-prefix-data-state A i*)
 ($\lambda out. \text{case out of}$
 $None \Rightarrow E None$
 $| Some (packed, t) \Rightarrow E (Some (snd\ packed, t))$)
 adversary-initial-state $\leq$
 wp-event
 (*checked-staged-security-experiment-with-query-prefix-data-state A i*)
 (*checked-staged-security-with-query-prefix-fixed-header-component*
 C trace-openings composition-openings i)
 adversary-initial-state
 by (rule *wp-event-mono*) (use *event-imp* in *blast*)
have *fixed-bound*:
 wp-event
 (*checked-staged-security-experiment-with-query-prefix-data-state A i*)
 (*checked-staged-security-with-query-prefix-fixed-header-component*
 C trace-openings composition-openings i)
 adversary-initial-state $\leq$
 staged-phase-relation-error size
 (*staged-query-search-queries budgets i + 1*) +
 ($1::prob$) / *nnreal (card query-sample-space)*
 by (rule *checked-staged-security-with-query-prefix-fixed-header-component-bound*
 [OF *raw-bound wf controlled i-bound*])
show *?thesis*
 unfolding *projection*
 by (rule *order-trans*[OF *lifted-bound fixed-bound*])
qed

lemma *checked-staged-security-with-data-state-bound-by-query-prefix-fixed-header-component-from-budgets*:

assumes *wf: staged-budget-wellformed budgets*

and *controlled: staged-adversary-controlled budgets A*

and *i-bound: i < rounds*

and *event-imp:*

$\bigwedge out.$

 (*case out of*

$None \Rightarrow E None$

$| Some (packed, t) \Rightarrow E (Some (snd\ packed, t)) \implies$

checked-staged-security-with-query-prefix-fixed-header-component

C trace-openings composition-openings i out

shows

wp-event (checked-staged-security-experiment-with-data-state A) E

adversary-initial-state $\leq$

staged-phase-relation-error size

 (*staged-query-search-queries budgets i + 1*) +

 ($1::prob$) / *nnreal (card query-sample-space)*

proof –

have *raw-bound: query-index-raw-preimage-bound*


```

    by (rule query-index-raw-preimage-bound-from-sampler-wellformed)
  show ?thesis
    by (rule
      checked-staged-security-with-data-state-bound-by-query-prefix-fixed-header-component
      [OF raw-bound wf controlled i-bound event-imp])
qed

lemma checked-staged-security-with-data-state-bound-by-query-prefix-fixed-header-component-from-witness-path-output
  assumes raw-bound: query-index-raw-preimage-bound
    and wf: staged-budget-wellformed budgets
    and controlled: staged-adversary-controlled budgets A
    and i-bound:  $i < \text{rounds}$ 
    and path-bound:
      wp-event
        (checked-staged-security-experiment-with-query-prefix-data-state A i)
        (checked-staged-security-with-query-prefix-header-witness-path-output-hit
          C trace-openings composition-openings i)
        adversary-initial-state  $\leq Q$ 
    and event-imp:
       $\bigwedge \text{out.}$ 
      (case out of
        None  $\Rightarrow E$  None
        | Some (packed, t)  $\Rightarrow E$  (Some (snd packed, t)))  $\Rightarrow$ 
        checked-staged-security-with-query-prefix-fixed-header-component
          C trace-openings composition-openings i out
  shows
    wp-event (checked-staged-security-experiment-with-data-state A) E
      adversary-initial-state  $\leq$ 
      staged-phase-relation-error size
        (staged-query-search-queries budgets  $i + 1$ ) +
        (1::prob) / nnreal (card query-sample-space) + Q
proof -
  have projection:
    wp-event (checked-staged-security-experiment-with-data-state A) E
      adversary-initial-state =
      wp-event
        (checked-staged-security-experiment-with-query-prefix-data-state A i)
        ( $\lambda \text{out. case out of}$ 
          None  $\Rightarrow E$  None
          | Some (packed, t)  $\Rightarrow E$  (Some (snd packed, t)))
        adversary-initial-state
    by (rule
      checked-staged-security-experiment-with-query-prefix-data-state-projection-event
      [OF i-bound])
  have lifted-bound:
    wp-event
      (checked-staged-security-experiment-with-query-prefix-data-state A i)
      ( $\lambda \text{out. case out of}$ 
        None  $\Rightarrow E$  None

```

```

| Some (packed, t)  $\Rightarrow$  E (Some (snd packed, t)))
adversary-initial-state  $\leq$ 
wp-event
  (checked-staged-security-experiment-with-query-prefix-data-state A i)
  (checked-staged-security-with-query-prefix-fixed-header-component
    C trace-openings composition-openings i)
  adversary-initial-state
by (rule wp-event-mono) (use event-imp in blast)
have fixed-bound:
wp-event
  (checked-staged-security-experiment-with-query-prefix-data-state A i)
  (checked-staged-security-with-query-prefix-fixed-header-component
    C trace-openings composition-openings i)
  adversary-initial-state  $\leq$ 
  staged-phase-relation-error size
  (staged-query-search-queries budgets i + 1) +
  (1::prob) / nnreal (card query-sample-space) + Q
by (rule
  checked-staged-security-with-query-prefix-fixed-header-component-bound-from-witness-path-output
  [OF raw-bound wf controlled i-bound path-bound])
show ?thesis
  unfolding projection
  by (rule order-trans[OF lifted-bound fixed-bound])
qed

lemma checked-staged-security-with-data-state-bound-by-query-prefix-fixed-header-component-from-witness-path-outp
assumes wf: staged-budget-wellformed budgets
and controlled: staged-adversary-controlled budgets A
and i-bound: i < rounds
and path-bound:
  wp-event
    (checked-staged-security-experiment-with-query-prefix-data-state A i)
    (checked-staged-security-with-query-prefix-header-witness-path-output-hit
      C trace-openings composition-openings i)
    adversary-initial-state  $\leq$  Q
and event-imp:
   $\bigwedge$  out.
  (case out of
    None  $\Rightarrow$  E None
  | Some (packed, t)  $\Rightarrow$  E (Some (snd packed, t)))  $\Longrightarrow$ 
    checked-staged-security-with-query-prefix-fixed-header-component
    C trace-openings composition-openings i out
shows
  wp-event (checked-staged-security-experiment-with-data-state A) E
  adversary-initial-state  $\leq$ 
  staged-phase-relation-error size
  (staged-query-search-queries budgets i + 1) +
  (1::prob) / nnreal (card query-sample-space) + Q
proof -

```

```

have raw-bound: query-index-raw-preimage-bound
  by (rule query-index-raw-preimage-bound-from-sampler-wellformed)
show ?thesis
  by (rule
    checked-staged-security-with-data-state-bound-by-query-prefix-fixed-header-component-from-witness-path-output
    [OF raw-bound wf controlled i-bound path-bound event-imp])
qed

lemma checked-staged-security-with-actual-alpha-prefix-data-state-bound-by-query-prefix-fixed-header-component:
assumes raw-bound: query-index-raw-preimage-bound
and wf: staged-budget-wellformed budgets
and controlled: staged-adversary-controlled budgets A
and i-bound:  $i < \text{rounds}$ 
and event-imp:
   $\bigwedge \text{out.}$ 
  (case out of
    None  $\Rightarrow D$  None
  | Some (packed, t)  $\Rightarrow D$  (Some (snd packed, t)))  $\Rightarrow$ 
    checked-staged-security-with-query-prefix-fixed-header-component
    C trace-openings composition-openings i out
shows
  wp-event
  (checked-staged-security-experiment-with-actual-alpha-prefix-data-state A)
  ( $\lambda \text{out.}$  case out of
    None  $\Rightarrow D$  None
  | Some (packed, t)  $\Rightarrow$ 
    D (Some (((snd (fst (fst packed))), snd (fst packed)), snd packed),
      t)))
  adversary-initial-state  $\leq$ 
  staged-phase-relation-error size
  (staged-query-search-queries budgets i + 1) +
  (1::prob) / nnreal (card query-sample-space)
proof –
let ?project =
   $\lambda x. ((\text{snd} (\text{fst} (\text{fst } x)), \text{snd} (\text{fst } x)), \text{snd } x)$ 
have projection:
  wp-event
  (checked-staged-security-experiment-with-actual-alpha-prefix-data-state A)
  ( $\lambda \text{out.}$  case out of
    None  $\Rightarrow D$  None
  | Some (packed, t)  $\Rightarrow D$  (Some (?project packed, t)))
  adversary-initial-state =
  wp-event
  (checked-staged-security-experiment-with-data-state A)
  D
  adversary-initial-state
proof –
have
  wp-event (checked-staged-security-experiment-with-data-state A)

```

```

    D adversary-initial-state =
wp-event
  (checked-staged-security-experiment-with-actual-alpha-prefix-data-state A
     $\gg$  ( $\lambda x. \text{return } (?project\ x)$ ))
    D adversary-initial-state
using
  checked-staged-security-experiment-with-actual-alpha-prefix-data-state-projection
  [of A]
by simp
also have ... =
wp-event
  (checked-staged-security-experiment-with-actual-alpha-prefix-data-state A)
  ( $\lambda out. \text{case out of}$ 
    None  $\Rightarrow$  D None
    | Some (packed, t)  $\Rightarrow$  D (Some (?project packed, t)))
    adversary-initial-state
by (rule wp-event-bind-return-map)
finally show ?thesis
by simp
qed
have data-bound:
wp-event (checked-staged-security-experiment-with-data-state A)
  D adversary-initial-state  $\leq$ 
  staged-phase-relation-error size
  (staged-query-search-queries budgets i + 1) +
  (1::prob) / nnreal (card query-sample-space)
by (rule
  checked-staged-security-with-data-state-bound-by-query-prefix-fixed-header-component
  [OF raw-bound wf controlled i-bound event-imp])
show ?thesis
unfolding projection
by (rule data-bound)
qed

lemma checked-staged-security-with-actual-alpha-prefix-data-state-bound-by-query-prefix-fixed-header-component-from
assumes wf: staged-budget-wellformed budgets
and controlled: staged-adversary-controlled budgets A
and i-bound: i < rounds
and event-imp:
 $\bigwedge out.$ 
  (case out of
    None  $\Rightarrow$  D None
    | Some (packed, t)  $\Rightarrow$  D (Some (snd packed, t)))  $\implies$ 
    checked-staged-security-with-query-prefix-fixed-header-component
    C trace-openings composition-openings i out
shows
wp-event
  (checked-staged-security-experiment-with-actual-alpha-prefix-data-state A)
  ( $\lambda out. \text{case out of}$ 

```

```

    None  $\Rightarrow$  D None
  | Some (packed, t)  $\Rightarrow$ 
    D (Some (((snd (fst (fst packed))), snd (fst packed))), snd packed),
      t)))
  adversary-initial-state  $\leq$ 
  staged-phase-relation-error size
  (staged-query-search-queries budgets i + 1) +
  (1::prob) / nnreal (card query-sample-space)
proof -
  have raw-bound: query-index-raw-preimage-bound
  by (rule query-index-raw-preimage-bound-from-sampler-wellformed)
  show ?thesis
  by (rule
    checked-staged-security-with-actual-alpha-prefix-data-state-bound-by-query-prefix-fixed-header-component
    [OF raw-bound wf controlled i-bound event-imp])
qed

lemma checked-staged-security-with-actual-alpha-prefix-data-state-bound-by-query-prefix-fixed-header-component-from
assumes raw-bound: query-index-raw-preimage-bound
and wf: staged-budget-wellformed budgets
and controlled: staged-adversary-controlled budgets A
and i-bound: i < rounds
and path-bound:
  wp-event
  (checked-staged-security-experiment-with-query-prefix-data-state A i)
  (checked-staged-security-with-query-prefix-header-witness-path-output-hit
    C trace-openings composition-openings i)
  adversary-initial-state  $\leq$  Q
and event-imp:
   $\bigwedge$  out.
  (case out of
    None  $\Rightarrow$  D None
  | Some (packed, t)  $\Rightarrow$  D (Some (snd packed, t)))  $\implies$ 
  checked-staged-security-with-query-prefix-fixed-header-component
  C trace-openings composition-openings i out
shows
  wp-event
  (checked-staged-security-experiment-with-actual-alpha-prefix-data-state A)
  ( $\lambda$  out. case out of
    None  $\Rightarrow$  D None
  | Some (packed, t)  $\Rightarrow$ 
    D (Some (((snd (fst (fst packed))), snd (fst packed))), snd packed),
      t)))
  adversary-initial-state  $\leq$ 
  staged-phase-relation-error size
  (staged-query-search-queries budgets i + 1) +
  (1::prob) / nnreal (card query-sample-space) + Q
proof -
  let ?project =

```

```

    λx. ((snd (fst (fst x)), snd (fst x)), snd x)
have projection:
  wp-event
    (checked-staged-security-experiment-with-actual-alpha-prefix-data-state A)
    (λout. case out of
      None ⇒ D None
    | Some (packed, t) ⇒ D (Some (?project packed, t)))
  adversary-initial-state =
  wp-event
    (checked-staged-security-experiment-with-data-state A)
    D
  adversary-initial-state
proof –
have
  wp-event (checked-staged-security-experiment-with-data-state A)
    D adversary-initial-state =
  wp-event
    (checked-staged-security-experiment-with-actual-alpha-prefix-data-state A
      ≫ (λx. return (?project x)))
    D adversary-initial-state
using
  checked-staged-security-experiment-with-actual-alpha-prefix-data-state-projection
  [of A]
by simp
also have ... =
  wp-event
    (checked-staged-security-experiment-with-actual-alpha-prefix-data-state A)
    (λout. case out of
      None ⇒ D None
    | Some (packed, t) ⇒ D (Some (?project packed, t)))
  adversary-initial-state
by (rule wp-event-bind-return-map)
finally show ?thesis
by simp
qed
have data-bound:
  wp-event (checked-staged-security-experiment-with-data-state A)
    D adversary-initial-state ≤
  staged-phase-relation-error size
    (staged-query-search-queries budgets i + 1) +
  (1::prob) / nnreal (card query-sample-space) + Q
by (rule
  checked-staged-security-with-data-state-bound-by-query-prefix-fixed-header-component-from-witness-path-output
  [OF raw-bound wf controlled i-bound path-bound event-imp])
show ?thesis
unfolding projection
by (rule data-bound)
qed

```

lemma *checked-staged-security-with-actual-alpha-prefix-data-state-bound-by-query-prefix-fixed-header-component-from-w*
assumes *wf: staged-budget-wellformed budgets*
and *controlled: staged-adversary-controlled budgets A*
and *i-bound: i < rounds*
and *path-bound:*
wp-event
(checked-staged-security-experiment-with-query-prefix-data-state A i)
(checked-staged-security-with-query-prefix-header-witness-path-output-hit
C trace-openings composition-openings i)
adversary-initial-state ≤ Q
and *event-imp:*
 $\bigwedge out.$
(case out of
None $\Rightarrow$ D None
| Some (packed, t) $\Rightarrow$ D (Some (snd packed, t))) $\Rightarrow$
checked-staged-security-with-query-prefix-fixed-header-component
C trace-openings composition-openings i out
shows
wp-event
(checked-staged-security-experiment-with-actual-alpha-prefix-data-state A)
($\lambda out.$ case out of
None $\Rightarrow$ D None
| Some (packed, t) $\Rightarrow$
D (Some (((snd (fst (fst packed))), snd (fst packed)), snd packed),
t)))
adversary-initial-state ≤
staged-phase-relation-error size
(staged-query-search-queries budgets i + 1) +
(1::prob) / nnreal (card query-sample-space) + Q
proof –
have *raw-bound: query-index-raw-preimage-bound*
by *(rule query-index-raw-preimage-bound-from-sampler-wellformed)*
show *?thesis*
by *(rule*
checked-staged-security-with-actual-alpha-prefix-data-state-bound-by-query-prefix-fixed-header-component-from-w
[OF raw-bound wf controlled i-bound path-bound event-imp])
qed

lemma *checked-staged-security-with-query-prefix-header-authenticated-candidate-opening-hit-bound:*
assumes *raw-bound: query-index-raw-preimage-bound*
and *wf: staged-budget-wellformed budgets*
and *controlled: staged-adversary-controlled budgets A*
and *i-bound: i < rounds*
shows
wp-event
(checked-staged-security-experiment-with-query-prefix-data-state A i)
(checked-staged-security-with-query-prefix-header-authenticated-candidate-opening-hit
trace-openings composition-openings i)
adversary-initial-state ≤

```

    staged-phase-relation-error size
      (staged-query-search-queries budgets  $i + 1$ ) +
      (1::prob) / nnreal (card query-sample-space)
  proof -
    have event-le:
      wp-event
        (checked-staged-security-experiment-with-query-prefix-data-state  $A$   $i$ )
        (checked-staged-security-with-query-prefix-header-authenticated-candidate-opening-hit
          trace-openings composition-openings  $i$ )
        adversary-initial-state  $\leq$ 
      wp-event
        (checked-staged-security-experiment-with-query-prefix-data-state  $A$   $i$ )
        (checked-staged-security-with-actual-query-prefix-candidate-opening-hit
          ((replicate rounds []) [ $i :=$  trace-openings])
          ((replicate rounds []) [ $i :=$  composition-openings])
           $i$ )
        adversary-initial-state
    by (rule wp-event-mono)
      (rule
        checked-staged-security-with-query-prefix-header-authenticated-candidate-opening-hit-imp-actual)
    have actual-bound:
      wp-event
        (checked-staged-security-experiment-with-query-prefix-data-state  $A$   $i$ )
        (checked-staged-security-with-actual-query-prefix-candidate-opening-hit
          ((replicate rounds []) [ $i :=$  trace-openings])
          ((replicate rounds []) [ $i :=$  composition-openings])
           $i$ )
        adversary-initial-state  $\leq$ 
        staged-phase-relation-error size
          (staged-query-search-queries budgets  $i + 1$ ) +
          (1::prob) / nnreal (card query-sample-space)
    by (rule checked-staged-security-with-actual-query-prefix-candidate-opening-hit-bound
      [OF raw-bound wf controlled])
      (use i-bound in simp)
    show ?thesis
    by (rule order-trans[OF event-le actual-bound])
  qed

```

lemma *checked-staged-security-with-query-prefix-header-authenticated-candidate-opening-hit-bound-from-budgets:*
assumes *wf: staged-budget-wellformed budgets*
and *controlled: staged-adversary-controlled budgets A*
and *i-bound: $i < \text{rounds}$*
shows
 wp-event
 (checked-staged-security-experiment-with-query-prefix-data-state A i)
 (checked-staged-security-with-query-prefix-header-authenticated-candidate-opening-hit
 trace-openings composition-openings i)
 adversary-initial-state $\leq$
 staged-phase-relation-error size


```

      (staged-query-search-queries budgets i + 1) +
      (1::prob) / nnreal (card query-sample-space)
proof —
  have raw-bound: query-index-raw-preimage-bound
    by (rule query-index-raw-preimage-bound-from-sampler-wellformed)
  show ?thesis
    by (rule
      checked-staged-security-with-query-prefix-header-authenticated-candidate-opening-hit-bound
      [OF raw-bound wf controlled i-bound])
qed

end

end

```

```

theory Staged-Security-Experiment-Composition-Query-Prefix-Witness
  imports Staged-Security-Experiment-Composition-Query-Augmented
begin

```

Witness-level bridge for the composition query-prefix path.

The existing query-prefix bounds are fixed-witness bounds: the trace and composition openings must already be selected before the query challenge is sampled. This layer records the exact local bridge from such fixed witness data to the augmented query-prefix component event. The remaining public soundness work is to derive these fixed witnesses from prefix Merkle/candidate evidence, or charge failures to existing Merkle/collision side events.

```

context soundness
begin

```

```

lemma partial-query-openings-consistent-canonical-trace-index:
  assumes consistent:
    partial-query-openings-consistent trace-openings composition-openings
    as idx
  shows trace-openings ≠ []
    and idx = opening-index (hd trace-openings)
proof —
  have trace-indices:
    map opening-index trace-openings = powers-scaled idx
    using consistent unfolding partial-query-openings-consistent-def by simp
  have powers-nonempty: powers-scaled idx ≠ []
    unfolding powers-scaled-def using powers-pos by simp
  then show trace-nonempty: trace-openings ≠ []
    using trace-indices by auto
  have hd (map opening-index trace-openings) = hd (powers-scaled idx)
    using trace-indices by simp
  then have opening-index (hd trace-openings) = hd (powers-scaled idx)
    using trace-nonempty by (simp add: hd-map)

```

then show $idx = \text{opening-index } (hd \text{ trace-openings})$
using powers-scaled-hd **by** simp
qed

lemma *partial-query-success-indices-at-subset-canonical-trace-index:*
partial-query-success-indices-at trace-openings composition-openings as i
 $\subseteq \{ \text{opening-index } (hd \text{ (trace-openings ! i)}) \}$

proof
fix idx
assume $idx\text{-in}$:
 $idx \in \text{partial-query-success-indices-at trace-openings}$
composition-openings as i
have trace-indices :
 $\text{map opening-index } (trace-openings ! i) = \text{powers-scaled } idx$
using $idx\text{-in}$
unfolding $\text{partial-query-success-indices-at-def}$
 $\text{partial-query-round-consistent-def}$
by simp
have trace-nonempty : $\text{trace-openings ! i} \neq []$
proof –
have $\text{powers-scaled } idx \neq []$
unfolding powers-scaled-def **using** powers-pos **by** simp
then show $?thesis$
using trace-indices **by** auto
qed
have $hd (\text{map opening-index } (trace-openings ! i)) = hd (\text{powers-scaled } idx)$
using trace-indices **by** simp
then have $\text{opening-index } (hd (\text{trace-openings ! i})) = hd (\text{powers-scaled } idx)$
using trace-nonempty **by** $(\text{simp add: hd-map})$
then show $idx \in \{ \text{opening-index } (hd (\text{trace-openings ! i})) \}$
using powers-scaled-hd **by** simp
qed

lemma *staged-query-prefix-candidate-opening-query-target-from-prefix-subset-canonical-trace-index:*
staged-query-prefix-candidate-opening-query-target-from-prefix
trace-openings composition-openings i prefix prefix-state $\subseteq$
 $\{ \text{opening-index } (hd (\text{trace-openings ! i})) \}$
unfolding $\text{staged-query-prefix-candidate-opening-query-target-from-prefix-def}$
by $(\text{rule partial-query-success-indices-at-subset-canonical-trace-index})$

definition *query-header-supported-partial-opening-canonical-indices*
 $:: ('f, 'a) \text{ protocol-channel-scheme} \Rightarrow$
 $'f \Rightarrow 'f \text{ list} \Rightarrow 'f \Rightarrow 'f \text{ list} \Rightarrow 'f \Rightarrow$
 $'f \text{ list} \Rightarrow 'f \Rightarrow \text{nat set}$

where
 $\text{query-header-supported-partial-opening-canonical-indices } s \text{ fr}$
 $f\text{-fri-roots } f\text{-final as } dg \text{ composition-fri-roots final} =$
 $\{ \text{opening-index } (hd \text{ trace-openings}) \mid$
 $\text{trace-openings composition-openings.}$

$(\text{trace-openings}, \text{composition-openings}) \in$
 $\text{query-header-supported-partial-opening-witnesses } s \text{ fr}$
 $f\text{-fri-roots } f\text{-final as dg composition-fri-roots final } \wedge$
 $\text{trace-openings} \neq []$

definition *query-header-supported-partial-opening-canonical-query-indices*

$:: ('f, 'a) \text{ protocol-channel-scheme} \Rightarrow$
 $'f \Rightarrow 'f \text{ list} \Rightarrow 'f \Rightarrow 'f \text{ list} \Rightarrow 'f \Rightarrow$
 $'f \text{ list} \Rightarrow 'f \Rightarrow \text{nat set}$

where

$\text{query-header-supported-partial-opening-canonical-query-indices } s \text{ fr}$
 $f\text{-fri-roots } f\text{-final as dg composition-fri-roots final} =$
 $\text{query-header-supported-partial-opening-canonical-indices } s \text{ fr}$
 $f\text{-fri-roots } f\text{-final as dg composition-fri-roots final} \cap$
 $\text{query-sample-space}$

lemma *query-header-supported-partial-opening-canonical-query-indices-subset:*

$\text{query-header-supported-partial-opening-canonical-query-indices } s \text{ fr}$
 $f\text{-fri-roots } f\text{-final as dg composition-fri-roots final} \subseteq$
 $\text{query-sample-space}$

unfolding *query-header-supported-partial-opening-canonical-query-indices-def*
by *blast*

lemma *query-header-supported-partial-opening-canonical-query-indices-iff:*

$\text{id}x \in \text{query-header-supported-partial-opening-canonical-query-indices } s \text{ fr}$
 $f\text{-fri-roots } f\text{-final as dg composition-fri-roots final} \longleftrightarrow$
 $\text{id}x \in \text{query-sample-space} \wedge$
 $\text{id}x \in \text{query-header-supported-partial-opening-canonical-indices } s \text{ fr}$
 $f\text{-fri-roots } f\text{-final as dg composition-fri-roots final}$

unfolding *query-header-supported-partial-opening-canonical-query-indices-def*
by *blast*

lemma *finite-query-header-supported-partial-opening-canonical-query-indices[simp]:*

finite
 $(\text{query-header-supported-partial-opening-canonical-query-indices } s \text{ fr}$
 $f\text{-fri-roots } f\text{-final as dg composition-fri-roots final})$
by $(\text{rule finite-subset}$
 $[OF \text{ query-header-supported-partial-opening-canonical-query-indices-subset}$
 $\text{finite-query-sample-space}])$

lemma *card-query-header-supported-partial-opening-canonical-query-indices-le:*

card
 $(\text{query-header-supported-partial-opening-canonical-query-indices } s \text{ fr}$
 $f\text{-fri-roots } f\text{-final as dg composition-fri-roots final})$
 $\leq \text{card query-sample-space}$
by (rule card-mono)
 $(\text{rule finite-query-sample-space},$
 $\text{rule query-header-supported-partial-opening-canonical-query-indices-subset})$

lemma *query-header-supported-partial-opening-canonical-query-indices-eq-query-sample-space-if-all-indices-witnessed*

assumes *witnessed*:

$\bigwedge idx. idx \in \text{query-sample-space} \implies$
 $\exists \text{trace-openings } \text{composition-openings}.$
 $(\text{trace-openings}, \text{composition-openings}) \in$
 $\text{query-header-supported-partial-opening-witnesses } s \text{ fr}$
 $f\text{-fri-roots } f\text{-final as dg composition-fri-roots final} \wedge$
 $\text{trace-openings} \neq [] \wedge$
 $\text{opening-index } (\text{hd } \text{trace-openings}) = idx$

shows

query-header-supported-partial-opening-canonical-query-indices s fr
f-fri-roots f-final as dg composition-fri-roots final =
query-sample-space

proof

show

query-header-supported-partial-opening-canonical-query-indices s fr
f-fri-roots f-final as dg composition-fri-roots final $\subseteq$
query-sample-space

by (rule *query-header-supported-partial-opening-canonical-query-indices-subset*)

next

show

query-sample-space $\subseteq$
query-header-supported-partial-opening-canonical-query-indices s fr
f-fri-roots f-final as dg composition-fri-roots final

proof

fix *idx*

assume *idx-sample*: *idx $\in$ query-sample-space*

then obtain *trace-openings composition-openings* **where** *witness*:

$(\text{trace-openings}, \text{composition-openings}) \in$
 $\text{query-header-supported-partial-opening-witnesses } s \text{ fr}$
 $f\text{-fri-roots } f\text{-final as dg composition-fri-roots final}$

and *nonempty*: *trace-openings $\neq []$*

and *idx-eq*: *opening-index (hd trace-openings) = idx*

using *witnessed* **by** *blast*

have *idx-canonical*:

idx $\in$
query-header-supported-partial-opening-canonical-indices s fr
f-fri-roots f-final as dg composition-fri-roots final

unfolding *query-header-supported-partial-opening-canonical-indices-def*

using *witness nonempty idx-eq* **by** *blast*

show

idx $\in$
query-header-supported-partial-opening-canonical-query-indices s fr
f-fri-roots f-final as dg composition-fri-roots final

unfolding *query-header-supported-partial-opening-canonical-query-indices-def*

using *idx-sample idx-canonical* **by** *simp*

qed

qed

lemma *table-agrees-with-authenticated-openings-from-partial-authenticated-table-if-no-partial-merkle-bad*:

assumes *table*:

partial-authenticated-table *rt* (*scale* * *clength*) *openings* *final-state*

and *outcome*:

Some (*result*, *final-state*) $\in$ *set-dist* (*execute verify-monad* *s*)

and *no-bad*:

$\neg$ *partial-merkle-inconsistency-bad* *s* (*Some* (*result*, *final-state*))

shows $\exists$ *table*.

table-agrees-with-authenticated-openings *table* (*scale* * *clength*)

openings

proof –

have *accepted-out*: *accepted* (*Some* (*result*, *final-state*))

unfolding *accepted-def* **by** *simp*

have *idx-bound*:

$\bigwedge \text{opn. } \text{opn} \in \text{set openings} \implies$

opening-index *opn* < *scale* * *clength*

proof –

fix *opn*

assume *opn-in*: *opn* $\in$ *set openings*

have *auth*: *authenticated-opening-in* *final-state* *opn*

using *table opn-in* **unfolding** *partial-authenticated-table-def* **by** *blast*

have *len*: *opening-length* *opn* = *scale* * *clength*

using *table opn-in* **unfolding** *partial-authenticated-table-def* **by** *blast*

show *opening-index* *opn* < *scale* * *clength*

using *auth len* **unfolding** *authenticated-opening-in-def* **by** *simp*

qed

have *consistent*:

$\bigwedge \text{opn opn'}. \text{opn} \in \text{set openings} \implies$

opn' $\in$ *set openings* $\implies$

opening-index *opn* = *opening-index* *opn'* $\implies$

opening-value *opn* = *opening-value* *opn'*

proof (*rule ccontr*)

fix *opn opn'*

assume *opn-in*: *opn* $\in$ *set openings*

and *opn'-in*: *opn'* $\in$ *set openings*

and *same-index*: *opening-index* *opn* = *opening-index* *opn'*

and *neg*: *opening-value* *opn* $\neq$ *opening-value* *opn'*

have *auth*: *authenticated-opening-in* *final-state* *opn*

using *table opn-in* **unfolding** *partial-authenticated-table-def* **by** *blast*

have *auth'*: *authenticated-opening-in* *final-state* *opn'*

using *table opn'-in* **unfolding** *partial-authenticated-table-def* **by** *blast*

have *root*: *opening-root* *opn* = *opening-root* *opn'*

using *table opn-in opn'-in* **unfolding** *partial-authenticated-table-def*

by *auto*

have *len*: *opening-length* *opn* = *opening-length* *opn'*

using *table opn-in opn'-in* **unfolding** *partial-authenticated-table-def*

by *auto*

have *bad*:

partial-merkle-inconsistency-bad *s* (*Some* (*result*, *final-state*))

```

    unfolding partial-merkle-inconsistency-bad-def
  by (intro conjI exI[of - result] exI[of - final-state]
      exI[of - opn] exI[of - opn'])
    (use accepted-out auth auth' root len same-index neq in simp-all)
  then show False
    using no-bad by contradiction
qed
show ?thesis
  by (rule table-agrees-with-authenticated-openings-consistent-exists
      [OF idx-bound consistent])
qed

lemma partial-trace-table-candidate-single-witness-if-no-partial-merkle-bad:
  assumes auth:
    partial-authenticated-table fr (scale * clength) trace-openings
    final-state
  and outcome:
    Some (result, final-state) ∈ set-dist (execute verify-monad s)
  and no-bad:
    ¬ partial-merkle-inconsistency-bad s (Some (result, final-state))
  and i-bound: i < rounds
  shows ∃ trace-table.
    partial-trace-table-candidate trace-table
    ((replicate rounds []) [i := trace-openings])
  proof -
    obtain trace-table where agrees:
      table-agrees-with-authenticated-openings trace-table (scale * clength)
      trace-openings
    using
      table-agrees-with-authenticated-openings-from-partial-authenticated-table-if-no-partial-merkle-bad
      [OF auth outcome no-bad]
    by blast
  have len: length trace-table = scale * clength
    using agrees unfolding table-agrees-with-authenticated-openings-def
    by simp
  have candidate:
    partial-trace-table-candidate trace-table
    ((replicate rounds []) [i := trace-openings])
  by (rule partial-trace-table-candidate-single-roundI
      [OF i-bound len agrees])
  then show ?thesis by blast
qed

lemma partial-composition-table-candidate-single-witness-if-no-partial-merkle-bad:
  assumes auth:
    partial-authenticated-table composition-root (scale * clength)
    composition-openings final-state
  and outcome:
    Some (result, final-state) ∈ set-dist (execute verify-monad s)

```

and *no-bad*:
 $\neg$ *partial-merkle-inconsistency-bad* *s* (*Some* (*result*, *final-state*))
and *i-bound*: *i* < *rounds*
shows $\exists$ *composition-table*.
partial-composition-table-candidate *composition-table*
((*replicate* *rounds* []) [*i* := *composition-openings*])
proof –
obtain *composition-table* **where** *agrees*:
table-agrees-with-authenticated-openings *composition-table*
(scale * *clength*) *composition-openings*
using
table-agrees-with-authenticated-openings-from-partial-authenticated-table-if-no-partial-merkle-bad
[*OF* *auth* *outcome* *no-bad*]
by *blast*
have *len*: *length* *composition-table* = *scale* * *clength*
using *agrees* **unfolding** *table-agrees-with-authenticated-openings-def*
by *simp*
have *candidate*:
partial-composition-table-candidate *composition-table*
((*replicate* *rounds* []) [*i* := *composition-openings*])
by (*rule* *partial-composition-table-candidate-single-roundI*
[*OF* *i-bound* *len* *agrees*])
then show ?*thesis* **by** *blast*
qed

lemma *query-header-supported-partial-opening-witness-single-round-candidates-or-partial-merkle-bad*:

assumes *witness*:
(*trace-openings*, *composition-openings*) $\in$
query-header-supported-partial-opening-witnesses *s* *fr* *f-fri-roots*
f-final *as* *dg* *composition-fri-roots* *final*
and *i-bound*: *i* < *rounds*

shows
($\exists$ *result* *final-state*.
partial-merkle-inconsistency-bad *s* (*Some* (*result*, *final-state*))) $\vee$
($\exists$ *trace-table* *composition-table*.
partial-trace-table-candidate *trace-table*
((*replicate* *rounds* []) [*i* := *trace-openings*]) $\wedge$
partial-composition-table-candidate *composition-table*
((*replicate* *rounds* []) [*i* := *composition-openings*]))

proof –
from *query-header-supported-partial-opening-witnessesE*[*OF* *witness*]
obtain *result* *final-state* *rest* **where** *outcome*:
Some (*result*, *final-state*) $\in$ *set-dist* (*execute* *verify-monad* *s*)
and *trace-auth*:
partial-authenticated-table *fr* (scale * *clength*) *trace-openings*
final-state
and *comp-auth*:
partial-authenticated-table (*hd* *composition-fri-roots*)
(scale * *clength*) *composition-openings* *final-state*

```

    by blast
  show ?thesis
proof (cases
  partial-merkle-inconsistency-bad s (Some (result, final-state)))
  case True
  then show ?thesis by blast
next
  case False
  obtain trace-table where trace-candidate:
    partial-trace-table-candidate trace-table
    ((replicate rounds []) [i := trace-openings])
  using partial-trace-table-candidate-single-witness-if-no-partial-merkle-bad
    [OF trace-auth outcome False i-bound]
  by blast
  obtain composition-table where comp-candidate:
    partial-composition-table-candidate composition-table
    ((replicate rounds []) [i := composition-openings])
  using
    partial-composition-table-candidate-single-witness-if-no-partial-merkle-bad
    [OF comp-auth outcome False i-bound]
  by blast
  show ?thesis
  using trace-candidate comp-candidate by blast
qed
qed

lemma query-header-supported-partial-opening-witness-success-indices-fraction-bound-if-low-degree-candidates:
  assumes i-bound: i < rounds
  and trace-candidate:
    partial-trace-table-candidate trace-table
    ((replicate rounds []) [i := trace-openings])
  and comp-candidate:
    partial-composition-table-candidate composition-table
    ((replicate rounds []) [i := composition-openings])
  and trace-low: trace-table-low-degree trace-table
  and comp-low: composition-table-low-degree maxDegree composition-table
  and not-all:
    ¬ all-queries-consistent trace-table composition-table as
  shows
    nnreal
      (card
        {idx ∈ query-sample-space.
          partial-query-openings-consistent trace-openings
            composition-openings as idx}) /
      nnreal (card query-sample-space) ≤ query-error-bound
  by (rule
    partial-query-openings-consistent-indices-fraction-bound-if-candidate-low-degree
    [OF i-bound trace-candidate comp-candidate trace-low comp-low not-all])

```


lemma *accepted-with-partial-trace-openings-update-round:*
assumes *partial:*
 accepted-with-partial-trace-openings s (Some (result, final-state))
 fr query-idxs trace-openings
and *i-bound: i < rounds*
and *consistent:*
 partial-query-openings-consistent trace-openings-i
 composition-openings-i as (query-idxs ! i)
and *auth:*
 *partial-authenticated-table fr (scale * clength) trace-openings-i*
 final-state
shows
 accepted-with-partial-trace-openings s (Some (result, final-state))
 fr query-idxs (trace-openings[i := trace-openings-i])
proof –
have *len-query: length query-idxs = rounds*
 using *accepted-with-partial-trace-openings-shapes(1)[OF partial]* .
have *len-trace: length trace-openings = rounds*
 using *accepted-with-partial-trace-openings-shapes(2)[OF partial]* .
have *trace-indices-i:*
 map opening-index trace-openings-i = powers-scaled (query-idxs ! i)
 using *consistent unfolding partial-query-openings-consistent-def by simp*
have *indices:*
 $\forall j < rounds.$
 map opening-index ((trace-openings[i := trace-openings-i]) ! j) =
 powers-scaled (query-idxs ! j)
proof (*intro allI impI*)
 fix *j*
 assume *j-bound: j < rounds*
 show
 map opening-index ((trace-openings[i := trace-openings-i]) ! j) =
 powers-scaled (query-idxs ! j)
 proof (*cases j = i*)
 case *True*
 then show *?thesis*
 using *i-bound len-trace trace-indices-i by simp*
 next
 case *False*
 then show *?thesis*
 using *accepted-with-partial-trace-openings-shapes(3)[OF partial j-bound]*
 len-trace j-bound
 by *simp*
 qed
qed
have *tables:*
 $\forall j < rounds.$
 *partial-authenticated-table fr (scale * clength)*
 ((trace-openings[i := trace-openings-i]) ! j) final-state
proof (*intro allI impI*)

```

fix j
assume j-bound: j < rounds
show
  partial-authenticated-table fr (scale * clength)
    ((trace-openings[i := trace-openings-i] ! j) final-state)
proof (cases j = i)
  case True
  then show ?thesis
    using i-bound len-trace auth by simp
next
  case False
  then show ?thesis
    using partial j-bound len-trace
    unfolding accepted-with-partial-trace-openings-def
    by simp
qed
qed
show ?thesis
  unfolding accepted-with-partial-trace-openings-def accepted-def
  by (intro conjI exI[of - result] exI[of - final-state])
    (use len-query len-trace indices tables in simp-all)
qed

lemma accepted-with-partial-composition-openings-update-round:
assumes partial:
  accepted-with-partial-composition-openings s
  (Some (result, final-state)) composition-root query-idxs
  composition-openings
and i-bound: i < rounds
and consistent:
  partial-query-openings-consistent trace-openings-i
  composition-openings-i as (query-idxs ! i)
and auth:
  partial-authenticated-table composition-root (scale * clength)
  composition-openings-i final-state
shows
  accepted-with-partial-composition-openings s
  (Some (result, final-state)) composition-root query-idxs
  (composition-openings[i := composition-openings-i])
proof –
  have len-query: length query-idxs = rounds
    using accepted-with-partial-composition-openings-shapes(1)[OF partial] .
  have len-comp: length composition-openings = rounds
    using accepted-with-partial-composition-openings-shapes(2)[OF partial] .
  have comp-indices-i:
    map opening-index composition-openings-i =
      [query-idxs ! i,
        fri-sibling-index (scale * clength) (query-idxs ! i)]
    using consistent unfolding partial-query-openings-consistent-def by simp

```

```

have indices:
   $\forall j < \text{rounds}.$ 
    map opening-index
       $((\text{composition-openings}[i := \text{composition-openings-}i]) ! j) =$ 
       $[\text{query-idxs} ! j,$ 
        fri-sibling-index (scale * clength) (query-idxs ! j)]
proof (intro allI impI)
  fix j
  assume j-bound:  $j < \text{rounds}$ 
  show
    map opening-index
       $((\text{composition-openings}[i := \text{composition-openings-}i]) ! j) =$ 
       $[\text{query-idxs} ! j,$ 
        fri-sibling-index (scale * clength) (query-idxs ! j)]
  proof (cases j = i)
    case True
    then show ?thesis
      using i-bound len-comp comp-indices-i by simp
    next
    case False
    then show ?thesis
      using accepted-with-partial-composition-openings-shapes(3)
      [OF partial j-bound]
      len-comp j-bound
      by simp
  qed
qed
have tables:
   $\forall j < \text{rounds}.$ 
    partial-authenticated-table composition-root (scale * clength)
       $((\text{composition-openings}[i := \text{composition-openings-}i]) ! j)$ 
      final-state
proof (intro allI impI)
  fix j
  assume j-bound:  $j < \text{rounds}$ 
  show
    partial-authenticated-table composition-root (scale * clength)
       $((\text{composition-openings}[i := \text{composition-openings-}i]) ! j)$ 
      final-state
  proof (cases j = i)
    case True
    then show ?thesis
      using i-bound len-comp auth by simp
    next
    case False
    then show ?thesis
      using partial j-bound len-comp
      unfolding accepted-with-partial-composition-openings-def
      by simp

```

```

qed
qed
show ?thesis
  unfolding accepted-with-partial-composition-openings-def accepted-def
  by (intro conjI exI[of - result] exI[of - final-state])
    (use len-query len-comp indices tables in simp-all)
qed

lemma accepted-with-partial-initial-openings-update-round:
  assumes partial:
    accepted-with-partial-initial-openings s
    (Some (result, final-state)) fr f-fri-roots f-final as dg
    composition-fri-roots final trace-query-idxs trace-openings
    composition-query-idxs composition-openings
  and i-bound: i < rounds
  and same-idx: trace-query-idxs ! i = composition-query-idxs ! i
  and consistent:
    partial-query-openings-consistent trace-openings-i
    composition-openings-i as (trace-query-idxs ! i)
  and trace-auth:
    partial-authenticated-table fr (scale * clength) trace-openings-i
    final-state
  and comp-auth:
    partial-authenticated-table (hd composition-fri-roots)
    (scale * clength) composition-openings-i final-state
  shows
    accepted-with-partial-initial-openings s
    (Some (result, final-state)) fr f-fri-roots f-final as dg
    composition-fri-roots final trace-query-idxs
    (trace-openings[i := trace-openings-i]) composition-query-idxs
    (composition-openings[i := composition-openings-i])
proof -
  have comp-nonempty: composition-fri-roots ≠ []
  using accepted-with-partial-initial-openings-shapes(1)[OF partial] .
  have header:
    ∃ rest. verifier-header-transcript s fr f-fri-roots f-final as dg
    composition-fri-roots final rest
  using accepted-with-partial-initial-openings-shapes(2)[OF partial] .
  have trace-partial:
    accepted-with-partial-trace-openings s
    (Some (result, final-state)) fr trace-query-idxs trace-openings
  using accepted-with-partial-initial-openings-shapes(3)[OF partial] .
  have comp-partial:
    accepted-with-partial-composition-openings s
    (Some (result, final-state)) (hd composition-fri-roots)
    composition-query-idxs composition-openings
  using accepted-with-partial-initial-openings-shapes(4)[OF partial] .
  have trace-updated:
    accepted-with-partial-trace-openings s

```

```

    (Some (result, final-state)) fr trace-query-idxs
    (trace-openings[i := trace-openings-i])
  by (rule accepted-with-partial-trace-openings-update-round
      [OF trace-partial i-bound consistent trace-auth])
  have comp-consistent:
    partial-query-openings-consistent trace-openings-i
    composition-openings-i as (composition-query-idxs ! i)
  using consistent same-idx by simp
  have comp-updated:
    accepted-with-partial-composition-openings s
    (Some (result, final-state)) (hd composition-fri-roots)
    composition-query-idxs
    (composition-openings[i := composition-openings-i])
  by (rule accepted-with-partial-composition-openings-update-round
      [OF comp-partial i-bound comp-consistent comp-auth])
  have accepted-out: accepted (Some (result, final-state))
  by (rule accepted-with-partial-trace-openings-imp-accepted
      [OF trace-updated])
  show ?thesis
  unfolding accepted-with-partial-initial-openings-def
  using accepted-out comp-nonempty header trace-updated comp-updated
  by simp
qed

```

lemma *accepted-with-partial-initial-openings-update-round-candidates-if-no-partial-merkle-bad:*

```

  assumes partial:
    accepted-with-partial-initial-openings s
    (Some (result, final-state)) fr f-fri-roots f-final as dg
    composition-fri-roots final trace-query-idxs trace-openings
    composition-query-idxs composition-openings
  and no-bad:
    ¬ partial-merkle-inconsistency-bad s (Some (result, final-state))
  and i-bound: i < rounds
  and same-idx: trace-query-idxs ! i = composition-query-idxs ! i
  and consistent:
    partial-query-openings-consistent trace-openings-i
    composition-openings-i as (trace-query-idxs ! i)
  and trace-auth:
    partial-authenticated-table fr (scale * clength) trace-openings-i
    final-state
  and comp-auth:
    partial-authenticated-table (hd composition-fri-roots)
    (scale * clength) composition-openings-i final-state
  shows
    ∃ trace-table composition-table.
    partial-trace-table-candidate trace-table
    (trace-openings[i := trace-openings-i]) ∧
    partial-composition-table-candidate composition-table
    (composition-openings[i := composition-openings-i])

```

proof –

have *updated*:

accepted-with-partial-initial-openings s
(*Some (result, final-state)*) *fr f-fri-roots f-final as dg*
composition-fri-roots final trace-query-idxs
(*trace-openings[i := trace-openings-i]*) *composition-query-idxs*
(*composition-openings[i := composition-openings-i]*)
by (rule *accepted-with-partial-initial-openings-update-round*
[*OF partial i-bound same-idx consistent trace-auth comp-auth*])

show *?thesis*

by (rule *accepted-with-partial-initial-openings-candidates-if-no-partial-merkle-bad*
[*OF updated no-bad*])

qed

lemma *query-header-supported-partial-opening-success-indices-at-subset-canonical-indices*:

query-header-supported-partial-opening-success-indices-at s fr
f-fri-roots f-final as dg composition-fri-roots final i $\subseteq$
query-header-supported-partial-opening-canonical-indices s fr
f-fri-roots f-final as dg composition-fri-roots final

proof

fix *idx*

assume *idx-in*:

idx $\in$ *query-header-supported-partial-opening-success-indices-at s fr*
f-fri-roots f-final as dg composition-fri-roots final i

then obtain *trace-openings composition-openings* **where** *witness*:

(*trace-openings, composition-openings*) $\in$
query-header-supported-partial-opening-witnesses s fr
f-fri-roots f-final as dg composition-fri-roots final

and *consistent*:

partial-query-openings-consistent trace-openings composition-openings
as idx

by (rule *query-header-supported-partial-opening-success-indices-atE*)

have *nonempty*: *trace-openings* $\neq []$

and *idx-eq*: *idx = opening-index (hd trace-openings)*

by (rule *partial-query-openings-consistent-canonical-trace-index*
[*OF consistent*])+

show

idx $\in$ *query-header-supported-partial-opening-canonical-indices s fr*
f-fri-roots f-final as dg composition-fri-roots final

unfolding *query-header-supported-partial-opening-canonical-indices-def*

using *witness nonempty idx-eq* **by** *blast*

qed

lemma *query-header-supported-partial-opening-success-indices-at-subset-canonical-query-indices*:

query-header-supported-partial-opening-success-indices-at s fr
f-fri-roots f-final as dg composition-fri-roots final i $\subseteq$
query-header-supported-partial-opening-canonical-query-indices s fr
f-fri-roots f-final as dg composition-fri-roots final

proof

```

fix idx
assume idx-in:
  idx  $\in$  query-header-supported-partial-opening-success-indices-at s fr
    f-fri-roots f-final as dg composition-fri-roots final i
have idx-canonical:
  idx  $\in$  query-header-supported-partial-opening-canonical-indices s fr
    f-fri-roots f-final as dg composition-fri-roots final
using idx-in
  query-header-supported-partial-opening-success-indices-at-subset-canonical-indices
by blast
have idx-sample: idx  $\in$  query-sample-space
using idx-in query-header-supported-partial-opening-success-indices-at-subset
by blast
show
  idx  $\in$  query-header-supported-partial-opening-canonical-query-indices s fr
    f-fri-roots f-final as dg composition-fri-roots final
unfolding query-header-supported-partial-opening-canonical-query-indices-def
using idx-canonical idx-sample by simp
qed

lemma query-header-supported-partial-opening-success-indices-at-fraction-bound-if-canonical-query-indices:
assumes canonical-bound:
  nnreal
    (card
      (query-header-supported-partial-opening-canonical-query-indices
        s fr f-fri-roots f-final as dg composition-fri-roots final)) /
    nnreal (card query-sample-space)  $\leq$  query-error-bound
shows
  nnreal
    (card
      (query-header-supported-partial-opening-success-indices-at s fr
        f-fri-roots f-final as dg composition-fri-roots final i)) /
    nnreal (card query-sample-space)  $\leq$  query-error-bound
proof –
have subset:
  query-header-supported-partial-opening-success-indices-at s fr
    f-fri-roots f-final as dg composition-fri-roots final i  $\subseteq$ 
  query-header-supported-partial-opening-canonical-query-indices s fr
    f-fri-roots f-final as dg composition-fri-roots final
by (rule
  query-header-supported-partial-opening-success-indices-at-subset-canonical-query-indices)
have finite-canonical:
  finite
    (query-header-supported-partial-opening-canonical-query-indices s fr
      f-fri-roots f-final as dg composition-fri-roots final)
by simp
have card-le:
  card
    (query-header-supported-partial-opening-success-indices-at s fr

```

$f\text{-fri-roots } f\text{-final as dg composition-fri-roots final } i) \leq$
 card
 $(\text{query-header-supported-partial-opening-canonical-query-indices } s \text{ fr}$
 $f\text{-fri-roots } f\text{-final as dg composition-fri-roots final})$
 $\text{by (rule card-mono[OF finite-canonical subset])}$
have
 nnreal
 $(\text{card}$
 $(\text{query-header-supported-partial-opening-success-indices-at } s \text{ fr}$
 $f\text{-fri-roots } f\text{-final as dg composition-fri-roots final } i)) /$
 $\text{nnreal (card query-sample-space)} \leq$
 nnreal
 $(\text{card}$
 $(\text{query-header-supported-partial-opening-canonical-query-indices } s \text{ fr}$
 $f\text{-fri-roots } f\text{-final as dg composition-fri-roots final})) /$
 $\text{nnreal (card query-sample-space)}$
 $\text{by (rule nnreal-nat-divide-right-mono[OF card-le])}$
also have $\dots \leq \text{query-error-bound}$
 $\text{by (rule canonical-bound)}$
finally show $?thesis$.
qed

lemma *query-header-supported-partial-opening-success-indices-at-subset-if-canonical-indices:*
assumes *canonical-cover:*
 $\text{query-header-supported-partial-opening-canonical-indices } s \text{ fr}$
 $f\text{-fri-roots } f\text{-final as dg composition-fri-roots final } \subseteq B$
shows
 $\text{query-header-supported-partial-opening-success-indices-at } s \text{ fr}$
 $f\text{-fri-roots } f\text{-final as dg composition-fri-roots final } i \subseteq B$
by $(\text{rule order-trans}$
 $[\text{OF}$
 $\text{query-header-supported-partial-opening-success-indices-at-subset-canonical-indices}$
 $\text{canonical-cover}])$

lemma *query-header-supported-partial-opening-success-indices-at-subset-if-canonical-query-indices:*
assumes *canonical-cover:*
 $\text{query-header-supported-partial-opening-canonical-query-indices } s \text{ fr}$
 $f\text{-fri-roots } f\text{-final as dg composition-fri-roots final } \subseteq B$
shows
 $\text{query-header-supported-partial-opening-success-indices-at } s \text{ fr}$
 $f\text{-fri-roots } f\text{-final as dg composition-fri-roots final } i \subseteq B$
by $(\text{rule order-trans}$
 $[\text{OF}$
 $\text{query-header-supported-partial-opening-success-indices-at-subset-canonical-query-indices}$
 $\text{canonical-cover}])$

lemma *staged-query-header-supported-partial-opening-success-indices-at-subset-if-canonical-indices:*
assumes *canonical-cover:*
 $\text{query-header-supported-partial-opening-canonical-indices}$

(verifier-state-from-adversary attacker-state
 (staged-proof-transcript data))
 (staged-trace-root data)
 (staged-trace-fri-roots data)
 (staged-trace-final data)
 (staged-alphas data)
 (staged-degree data)
 (staged-composition-fri-roots data)
 (staged-composition-final data) $\subseteq B$

shows

query-header-supported-partial-opening-success-indices-at
 (verifier-state-from-adversary attacker-state
 (staged-proof-transcript data))
 (staged-trace-root data)
 (staged-trace-fri-roots data)
 (staged-trace-final data)
 (staged-alphas data)
 (staged-degree data)
 (staged-composition-fri-roots data)
 (staged-composition-final data)
 $i \subseteq B$

by (rule

query-header-supported-partial-opening-success-indices-at-subset-if-canonical-indices
 [OF canonical-cover])

lemma staged-query-header-supported-partial-opening-success-indices-at-subset-if-canonical-query-indices:

assumes canonical-cover:

query-header-supported-partial-opening-canonical-query-indices
 (verifier-state-from-adversary attacker-state
 (staged-proof-transcript data))
 (staged-trace-root data)
 (staged-trace-fri-roots data)
 (staged-trace-final data)
 (staged-alphas data)
 (staged-degree data)
 (staged-composition-fri-roots data)
 (staged-composition-final data) $\subseteq B$

shows

query-header-supported-partial-opening-success-indices-at
 (verifier-state-from-adversary attacker-state
 (staged-proof-transcript data))
 (staged-trace-root data)
 (staged-trace-fri-roots data)
 (staged-trace-final data)
 (staged-alphas data)
 (staged-degree data)
 (staged-composition-fri-roots data)
 (staged-composition-final data)
 $i \subseteq B$

by (rule
query-header-supported-partial-opening-success-indices-at-subset-if-canonical-query-indices
[OF canonical-cover])

lemma *checked-staged-security-with-data-state-partial-opening-hit-bound-from-canonical-query-error-cover:*

assumes *wf: staged-budget-wellformed budgets*

and *controlled: staged-adversary-controlled budgets A*

and *cover:*

$\wedge$ *data attacker-state.*

query-header-supported-partial-opening-canonical-query-indices

(verifier-state-from-adversary attacker-state

(staged-proof-transcript data))

(staged-trace-root data)

(staged-trace-fri-roots data)

(staged-trace-final data)

(staged-alphas data)

(staged-degree data)

(staged-composition-fri-roots data)

(staged-composition-final data) $\subseteq B$

and *subset: $B \subseteq \text{query-sample-space}$*

and *frac:*

nnreal (card B) / nnreal (card query-sample-space) $\leq$

query-error-bound

shows

wp-event (checked-staged-security-experiment-with-data-state A)

staged-security-with-data-state-query-partial-opening-hit

adversary-initial-state $\leq$

nnreal

(staged-attacker-query-budget budgets +

*staged-challenge-query-budget) * query-error-bound*

proof —

have *raw-bound: query-index-raw-preimage-bound*

by (rule *query-index-raw-preimage-bound-from-sampler-wellformed*)

show *?thesis*

proof (rule

checked-staged-security-with-data-state-partial-opening-hit-bound-from-fixed-query-error-cover

[OF wf controlled - raw-bound subset frac])

fix *data attacker-state i*

assume *i < rounds*

show

query-header-supported-partial-opening-success-indices-at

(verifier-state-from-adversary attacker-state

(staged-proof-transcript data))

(staged-trace-root data)

(staged-trace-fri-roots data)

(staged-trace-final data)

(staged-alphas data)

(staged-degree data)

(staged-composition-fri-roots data)

```

    (staged-composition-final data)
     $i \subseteq B$ 
  by (rule
    staged-query-header-supported-partial-opening-success-indices-at-subset-if-canonical-query-indices
    [OF cover])
qed
qed

lemma checked-staged-security-with-data-state-query-bad-bound-from-canonical-query-error-cover:
assumes wf: staged-budget-wellformed budgets
and controlled: staged-adversary-controlled budgets A
and cover:
   $\bigwedge$  data attacker-state.
    query-header-supported-partial-opening-canonical-query-indices
      (verifier-state-from-adversary attacker-state
        (staged-proof-transcript data))
      (staged-trace-root data)
      (staged-trace-fri-roots data)
      (staged-trace-final data)
      (staged-alphas data)
      (staged-degree data)
      (staged-composition-fri-roots data)
      (staged-composition-final data)  $\subseteq B$ 
and subset:  $B \subseteq \text{query-sample-space}$ 
and frac:
   $\text{nnreal} (\text{card } B) / \text{nnreal} (\text{card query-sample-space}) \leq$ 
    query-error-bound
shows
  wp-event (checked-staged-security-experiment-with-data-state A)
    staged-security-with-data-state-query-bad-hit
    adversary-initial-state  $\leq$ 
  nnreal
    (staged-attacker-query-budget budgets +
      staged-challenge-query-budget) * query-error-bound
proof –
have partial-bound:
  wp-event (checked-staged-security-experiment-with-data-state A)
    staged-security-with-data-state-query-partial-opening-hit
    adversary-initial-state  $\leq$ 
  nnreal
    (staged-attacker-query-budget budgets +
      staged-challenge-query-budget) * query-error-bound
by (rule
  checked-staged-security-with-data-state-partial-opening-hit-bound-from-canonical-query-error-cover
  [OF wf controlled cover subset frac])
show ?thesis
by (rule order.trans
  [OF checked-staged-security-with-data-state-query-bad-bound-from-partial-opening-hit
  [OF wf controlled] partial-bound])

```

qed

lemma *checked-staged-query-prefix-candidate-opening-target-from-prefix-canonical-indexD*:

assumes *i-bound*: $i < \text{rounds}$

and *hit*:

checked-staged-query-prefix-dynamic-index-hit
 (*staged-query-prefix-candidate-opening-query-target-from-prefix*
 ((*replicate rounds* []) [$i := \text{trace-openings}$])
 ((*replicate rounds* []) [$i := \text{composition-openings}$])
i)
 (Some (((*prefix*, *prefix-state*), *raw*), *raw-state*))

shows *index* (*to-nat raw*) = *opening-index* (*hd trace-openings*)

proof –

have *idx-in*:

index (*to-nat raw*) ∈
staged-query-prefix-candidate-opening-query-target-from-prefix
 ((*replicate rounds* []) [$i := \text{trace-openings}$])
 ((*replicate rounds* []) [$i := \text{composition-openings}$])
i prefix prefix-state

using *hit* **unfolding** *checked-staged-query-prefix-dynamic-index-hit-def*

by *simp*

have *singleton*:

staged-query-prefix-candidate-opening-query-target-from-prefix
 ((*replicate rounds* []) [$i := \text{trace-openings}$])
 ((*replicate rounds* []) [$i := \text{composition-openings}$])
i prefix prefix-state ⊆
 {*opening-index*
 (*hd* (((*replicate rounds* []) [$i := \text{trace-openings}$]) ! *i*))}

by (*rule*

staged-query-prefix-candidate-opening-query-target-from-prefix-subset-canonical-trace-index)

have *index* (*to-nat raw*) =

opening-index
 (*hd* (((*replicate rounds* []) [$i := \text{trace-openings}$]) ! *i*))

using *idx-in singleton* **by** *auto*

then show *?thesis*

using *i-bound* **by** *simp*

qed

lemma *checked-staged-security-with-query-prefix-fixed-header-component-canonical-indexD*:

assumes *i-bound*: $i < \text{rounds}$

and *hit*:

checked-staged-security-with-query-prefix-fixed-header-component
E trace-openings composition-openings i
 (Some ((((((*prefix*, *prefix-state*), *raw*), *raw-state*),
 ((*data*, *attacker-state*), *result*)), *final-state*))

shows *index* (*to-nat raw*) = *opening-index* (*hd trace-openings*)

proof –

from *checked-staged-security-with-query-prefix-fixed-header-componentE*[*OF hit*]

have *target*:

```

  index (to-nat raw) ∈
    staged-query-prefix-candidate-opening-query-target-from-prefix
      ((replicate rounds []) [i := trace-openings])
      ((replicate rounds []) [i := composition-openings])
      i prefix prefix-state
  by blast
have singleton:
  staged-query-prefix-candidate-opening-query-target-from-prefix
    ((replicate rounds []) [i := trace-openings])
    ((replicate rounds []) [i := composition-openings])
    i prefix prefix-state ⊆
    {opening-index
      (hd (((replicate rounds []) [i := trace-openings]) ! i))}
  by (rule
    staged-query-prefix-candidate-opening-query-target-from-prefix-subset-canonical-trace-index)
have index (to-nat raw) =
  opening-index
    (hd (((replicate rounds []) [i := trace-openings]) ! i))
  using target singleton by auto
then show ?thesis
  using i-bound by simp
qed

lemma checked-staged-security-with-query-prefix-fixed-header-component-trace-openings-nonempty:
  assumes i-bound: i < rounds
  and hit:
    checked-staged-security-with-query-prefix-fixed-header-component
      E trace-openings composition-openings i
      (Some (((prefix, prefix-state), raw), raw-state),
        ((data, attacker-state), result)), final-state))
  shows trace-openings ≠ []
proof -
  from checked-staged-security-with-query-prefix-fixed-header-componentE[OF hit]
  have target:
    index (to-nat raw) ∈
      staged-query-prefix-candidate-opening-query-target-from-prefix
        ((replicate rounds []) [i := trace-openings])
        ((replicate rounds []) [i := composition-openings])
        i prefix prefix-state
    by blast
  then have round:
    partial-query-round-consistent
      ((replicate rounds []) [i := trace-openings])
      ((replicate rounds []) [i := composition-openings])
      (sqp-alphas prefix) i (index (to-nat raw))
  unfolding staged-query-prefix-candidate-opening-query-target-from-prefix-def
    partial-query-success-indices-at-def
  by simp
have trace-indices:

```

```

map opening-index
  (((replicate rounds []) [i := trace-openings]) ! i) =
  powers-scaled (index (to-nat raw))
using round unfolding partial-query-round-consistent-def by blast
have powers-scaled (index (to-nat raw)) ≠ []
unfolding powers-scaled-def using powers-pos by simp
then have ((replicate rounds []) [i := trace-openings]) ! i ≠ []
using trace-indices by auto
then show ?thesis
using i-bound by simp
qed

```

lemma *checked-staged-security-with-query-prefix-header-authenticated-candidate-opening-hit-canonical-query-indexD*:

assumes *i-bound*: $i < \text{rounds}$

and *hit*:

checked-staged-security-with-query-prefix-header-authenticated-candidate-opening-hit
trace-openings composition-openings i
(Some (((prefix, prefix-state), raw), raw-state),
((data, attacker-state), result)), final-state))

shows

index (to-nat raw) ∈
query-header-supported-partial-opening-canonical-query-indices
(verifier-state-from-adversary attacker-state
(staged-proof-transcript data))
(staged-trace-root data)
(staged-trace-fri-roots data)
(staged-trace-final data)
(staged-alphas data)
(staged-degree data)
(staged-composition-fri-roots data)
(staged-composition-final data)

proof —

from *checked-staged-security-with-query-prefix-header-authenticated-candidate-opening-hitE*
[OF hit]

have *witness*:

(trace-openings, composition-openings) ∈
query-header-supported-partial-opening-witnesses
(verifier-state-from-adversary attacker-state
(staged-proof-transcript data))
(staged-trace-root data)
(staged-trace-fri-roots data)
(staged-trace-final data)
(staged-alphas data)
(staged-degree data)
(staged-composition-fri-roots data)
(staged-composition-final data)

and *target*:

index (to-nat raw) ∈
staged-query-prefix-candidate-opening-query-target-from-prefix

```

      ((replicate rounds []) [i := trace-openings])
      ((replicate rounds []) [i := composition-openings])
      i prefix prefix-state
    by blast+
  have idx-sample: index (to-nat raw) ∈ query-sample-space
    using target staged-query-prefix-candidate-opening-query-target-from-prefix-subset
    by blast
  have round:
    partial-query-round-consistent
      ((replicate rounds []) [i := trace-openings])
      ((replicate rounds []) [i := composition-openings])
      (sqp-alphas prefix) i (index (to-nat raw))
    using target
    unfolding staged-query-prefix-candidate-opening-query-target-from-prefix-def
      partial-query-success-indices-at-def
    by simp
  have trace-indices:
    map opening-index
      (((replicate rounds []) [i := trace-openings]) ! i) =
      powers-scaled (index (to-nat raw))
    using round unfolding partial-query-round-consistent-def by blast
  have trace-nonempty: trace-openings ≠ []
  proof -
    have powers-scaled (index (to-nat raw)) ≠ []
      unfolding powers-scaled-def using powers-pos by simp
    then have ((replicate rounds []) [i := trace-openings]) ! i ≠ []
      using trace-indices by auto
    then show ?thesis
      using i-bound by simp
  qed
  have singleton:
    staged-query-prefix-candidate-opening-query-target-from-prefix
      ((replicate rounds []) [i := trace-openings])
      ((replicate rounds []) [i := composition-openings])
      i prefix prefix-state ⊆
      {opening-index
        (hd (((replicate rounds []) [i := trace-openings]) ! i))}
    by (rule
      staged-query-prefix-candidate-opening-query-target-from-prefix-subset-canonical-trace-index)
  have idx-eq:
    index (to-nat raw) = opening-index (hd trace-openings)
    using target singleton i-bound by auto
  have idx-canonical:
    index (to-nat raw) ∈
      query-header-supported-partial-opening-canonical-indices
        (verifier-state-from-adversary attacker-state
          (staged-proof-transcript data))
        (staged-trace-root data)
        (staged-trace-fri-roots data)

```

```

    (staged-trace-final data)
    (staged-alphas data)
    (staged-degree data)
    (staged-composition-fri-roots data)
    (staged-composition-final data)
  unfolding query-header-supported-partial-opening-canonical-indices-def
  using witness trace-nonempty idx-eq by blast
show ?thesis
  unfolding query-header-supported-partial-opening-canonical-query-indices-def
  using idx-sample idx-canonical by simp
qed

lemma checked-staged-security-with-query-prefix-fixed-header-component-canonical-query-indexD:
  assumes i-bound:  $i < \text{rounds}$ 
  and hit:
    checked-staged-security-with-query-prefix-fixed-header-component
     $E$  trace-openings composition-openings  $i$ 
    (Some (((((prefix, prefix-state), raw), raw-state),
      ((data, attacker-state), result)), final-state))
  shows
    index (to-nat raw)  $\in$ 
    query-header-supported-partial-opening-canonical-query-indices
    (verifier-state-from-adversary attacker-state
      (staged-proof-transcript data))
    (staged-trace-root data)
    (staged-trace-fri-roots data)
    (staged-trace-final data)
    (staged-alphas data)
    (staged-degree data)
    (staged-composition-fri-roots data)
    (staged-composition-final data)
proof -
  have header:
    checked-staged-security-with-query-prefix-header-authenticated-candidate-opening-hit
    trace-openings composition-openings  $i$ 
    (Some (((((prefix, prefix-state), raw), raw-state),
      ((data, attacker-state), result)), final-state))
  using hit
  unfolding checked-staged-security-with-query-prefix-fixed-header-component-def
  by simp
show ?thesis
  by (rule
    checked-staged-security-with-query-prefix-header-authenticated-candidate-opening-hit-canonical-query-indexD
    [OF i-bound header])
qed

lemma checked-staged-security-with-query-prefix-header-authenticated-candidate-opening-hitI-from-dynamic-hit:
  assumes witness:
    (trace-openings, composition-openings)  $\in$ 

```



```

    query-header-supported-partial-opening-witnesses
      (verifier-state-from-adversary attacker-state
        (staged-proof-transcript data))
      (staged-trace-root data)
      (staged-trace-fri-roots data)
      (staged-trace-final data)
      (staged-alphas data)
      (staged-degree data)
      (staged-composition-fri-roots data)
      (staged-composition-final data)
  and target-hit:
    checked-staged-query-prefix-dynamic-index-hit
      (staged-query-prefix-candidate-opening-query-target-from-prefix
        ((replicate rounds []) [i := trace-openings])
        ((replicate rounds []) [i := composition-openings])
        i)
      (Some (((prefix, prefix-state), raw), raw-state))
  shows
    checked-staged-security-with-query-prefix-header-authenticated-candidate-opening-hit
      trace-openings composition-openings i
      (Some (((((prefix, prefix-state), raw), raw-state),
        ((data, attacker-state), result)), final-state))
  proof -
  from query-header-supported-partial-opening-witnessesE[OF witness]
  have comp-nonempty: staged-composition-fri-roots data ≠ []
  by blast
  have actual:
    checked-staged-security-with-actual-query-prefix-candidate-opening-hit
      ((replicate rounds []) [i := trace-openings])
      ((replicate rounds []) [i := composition-openings])
      i
      (Some (((((prefix, prefix-state), raw), raw-state),
        ((data, attacker-state), result)), final-state))
  using target-hit
  unfolding checked-staged-security-with-actual-query-prefix-candidate-opening-hit-def
    checked-staged-security-with-query-prefix-dynamic-index-hit-def
  by simp
  show ?thesis
  unfolding
    checked-staged-security-with-query-prefix-header-authenticated-candidate-opening-hit-def
  using comp-nonempty witness actual by simp
qed

lemma checked-staged-security-with-query-prefix-fixed-header-componentI-from-dynamic-hit:
  assumes event:
    E (Some (((((prefix, prefix-state), raw), raw-state),
      ((data, attacker-state), result)), final-state))
  and witness:
    (trace-openings, composition-openings) ∈

```

query-header-supported-partial-opening-witnesses
 (verifier-state-from-adversary attacker-state
 (staged-proof-transcript data))
 (staged-trace-root data)
 (staged-trace-fri-roots data)
 (staged-trace-final data)
 (staged-alphas data)
 (staged-degree data)
 (staged-composition-fri-roots data)
 (staged-composition-final data)
and target-hit:
 checked-staged-query-prefix-dynamic-index-hit
 (staged-query-prefix-candidate-opening-query-target-from-prefix
 ((replicate rounds []) [i := trace-openings])
 ((replicate rounds []) [i := composition-openings])
 i)
 (Some (((prefix, prefix-state), raw), raw-state))
shows
 checked-staged-security-with-query-prefix-fixed-header-component
 E trace-openings composition-openings i
 (Some (((((prefix, prefix-state), raw), raw-state),
 ((data, attacker-state), result)), final-state))
unfolding checked-staged-security-with-query-prefix-fixed-header-component-def
by (intro conjI event
 checked-staged-security-with-query-prefix-header-authenticated-candidate-opening-hitI-from-dynamic-hit
 [OF witness target-hit])
lemma checked-staged-security-with-query-prefix-fixed-header-component-bound-from-budgets:
assumes wf: staged-budget-wellformed budgets
and controlled: staged-adversary-controlled budgets A
and i-bound: i < rounds
shows
 wp-event
 (checked-staged-security-experiment-with-query-prefix-data-state A i)
 (checked-staged-security-with-query-prefix-fixed-header-component
 E trace-openings composition-openings i)
 adversary-initial-state ≤
 staged-phase-relation-error size
 (staged-query-search-queries budgets i + 1) +
 (1::prob) / nnreal (card query-sample-space)
proof –
have raw-bound: query-index-raw-preimage-bound
by (rule query-index-raw-preimage-bound-from-sampler-wellformed)
show ?thesis
by (rule
 checked-staged-security-with-query-prefix-fixed-header-component-bound
 [OF raw-bound wf controlled i-bound])
qed

lemma *checked-staged-security-with-query-prefix-header-authenticated-candidate-opening-hit-bound-from-budgets:*

assumes *wf: staged-budget-wellformed budgets*

and *controlled: staged-adversary-controlled budgets A*

and *i-bound: i < rounds*

shows

wp-event

(checked-staged-security-experiment-with-query-prefix-data-state A i)

*(checked-staged-security-with-query-prefix-header-authenticated-candidate-opening-hit
trace-openings composition-openings i)*

adversary-initial-state ≤

staged-phase-relation-error size

(staged-query-search-queries budgets i + 1) +

(1::prob) / nnreal (card query-sample-space)

proof —

have *raw-bound: query-index-raw-preimage-bound*

by *(rule query-index-raw-preimage-bound-from-sampler-wellformed)*

show *?thesis*

by *(rule*

*checked-staged-security-with-query-prefix-header-authenticated-candidate-opening-hit-bound
[OF raw-bound wf controlled i-bound])*

qed

lemma *checked-staged-query-prefix-candidate-opening-target-from-prefix-hit-bound-from-budgets:*

assumes *wf: staged-budget-wellformed budgets*

and *controlled: staged-adversary-controlled budgets A*

and *i-bound: i < rounds*

and *trace-candidate:*

partial-trace-table-candidate trace-table trace-openings

and *comp-candidate:*

partial-composition-table-candidate composition-table

composition-openings

and *trace-low: trace-table-low-degree trace-table*

and *comp-low: composition-table-low-degree maxDegree composition-table*

and *not-all:*

∧ prefix prefix-state.

Some ((prefix, prefix-state), prefix-state) ∈

set-dist

(execute (checked-staged-query-prefix-with-state A i)

adversary-initial-state) ⇒

¬ all-queries-consistent trace-table composition-table

(sqp-alphas prefix)

shows

wp-event (checked-staged-query-prefix-receive-with-state A i)

(checked-staged-query-prefix-dynamic-index-hit

(staged-query-prefix-candidate-opening-query-target-from-prefix

trace-openings composition-openings i))

adversary-initial-state ≤

staged-phase-relation-error size

(staged-query-search-queries budgets i + 1) + query-error-bound

```

proof –
  have raw-bound: query-index-raw-preimage-bound
    by (rule query-index-raw-preimage-bound-from-sampler-wellformed)
  show ?thesis
    by (rule
      checked-staged-query-prefix-candidate-opening-target-from-prefix-hit-bound
      [OF raw-bound wf controlled - trace-candidate comp-candidate
       trace-low comp-low not-all])
    (use i-bound in simp)

```

qed

end

end

```

theory Soundness-Partial-Merkle-Cross
  imports Soundness-Partial-Merkle
begin

```

Cross-state Merkle facts for partial authenticated openings.

The broad query-header candidate relation can be supported by different verifier executions and therefore different final hash-map states. These lemmas isolate the reusable binding fact available in the current model: two authenticated openings for the same root, length, and index cannot carry different values across compatible cleanly-merged states.

```

context soundness
begin

```

definition *partial-openings-cross-cover-all-indices*

```

:: 'f authenticated-opening list list  $\Rightarrow$ 
   'f authenticated-opening list list  $\Rightarrow$  bool

```

where

```

partial-openings-cross-cover-all-indices openings openings'  $\longleftrightarrow$ 
  ( $\forall j < \text{scale} * \text{clength}.$ 
    $\exists i < \text{rounds}. \exists i' < \text{rounds}.$ 
    $\exists \text{opn} \in \text{set } (\text{openings} ! i).$ 
    $\exists \text{opn}' \in \text{set } (\text{openings}' ! i').$ 
    $\text{opening-index } \text{opn} = j \wedge$ 
    $\text{opening-index } \text{opn}' = j)$ 

```

lemma *partial-openings-cross-cover-all-indicesD*:

assumes

```

partial-openings-cross-cover-all-indices openings openings'
and  $j < \text{scale} * \text{clength}$ 

```

obtains $i \ i' \ \text{opn} \ \text{opn}'$ **where**

```

i < rounds
i' < rounds

```

$opn \in \text{set } (\text{openings} ! i)$
 $opn' \in \text{set } (\text{openings}' ! i')$
 $\text{opening-index } opn = j$
 $\text{opening-index } opn' = j$
using *assms*
unfolding *partial-openings-cross-cover-all-indices-def* **by** *blast*

lemma *partial-openings-cross-cover-all-indices-sym*:
assumes *partial-openings-cross-cover-all-indices openings openings'*
shows *partial-openings-cross-cover-all-indices openings' openings*
using *assms*
unfolding *partial-openings-cross-cover-all-indices-def* **by** *blast*

lemma *not-partial-openings-cross-cover-all-indicesE*:
assumes $\neg \text{partial-openings-cross-cover-all-indices } openings \ openings'$
obtains *j* **where**
 $j < \text{scale} * \text{clength}$
 $\bigwedge i \ i' \ opn \ opn'. \ i < \text{rounds} \implies i' < \text{rounds} \implies$
 $\quad \text{opn} \in \text{set } (\text{openings} ! i) \implies$
 $\quad \text{opn}' \in \text{set } (\text{openings}' ! i') \implies$
 $\quad \text{opening-index } opn = j \implies \text{opening-index } opn' \neq j$
using *assms*
unfolding *partial-openings-cross-cover-all-indices-def* **by** *blast*

lemma *partial-openings-cross-cover-all-indices-imp-left-cover*:
assumes *cover*:
 $\text{partial-openings-cross-cover-all-indices } openings \ openings'$
and *j-bound*: $j < \text{scale} * \text{clength}$
shows $\exists i < \text{rounds}. \exists opn \in \text{set } (\text{openings} ! i).$
 $\text{opening-index } opn = j$
using *partial-openings-cross-cover-all-indicesD[OF cover j-bound]* **by** *blast*

lemma *partial-openings-cross-cover-all-indices-imp-right-cover*:
assumes *cover*:
 $\text{partial-openings-cross-cover-all-indices } openings \ openings'$
and *j-bound*: $j < \text{scale} * \text{clength}$
shows $\exists i < \text{rounds}. \exists opn \in \text{set } (\text{openings}' ! i).$
 $\text{opening-index } opn = j$
using *partial-openings-cross-cover-all-indicesD[OF cover j-bound]* **by** *blast*

lemma *cross-sparse-openings-obtain-unopened-index*:
assumes *rounds-len*: $\text{length } openings = \text{rounds}$
and *sparse*: $\text{length } (\text{List.concat } openings) < \text{scale} * \text{clength}$
obtains *j* **where**
 $j < \text{scale} * \text{clength}$
 $\bigwedge i \ opn. \ i < \text{rounds} \implies opn \in \text{set } (\text{openings} ! i) \implies$
 $\quad \text{opening-index } opn \neq j$
proof –
let $?N = \text{scale} * \text{clength}$

```

let ?I = opening-index ' set (List.concat openings)
have finite-I: finite ?I
  by simp
have card-I-le: card ?I ≤ length (List.concat openings)
proof -
  have card ?I ≤ card (set (List.concat openings))
    by (rule card-image-le) simp
  also have ... ≤ length (List.concat openings)
    by (rule card-length)
  finally show ?thesis .
qed
have not-all: ¬ {0..N} ⊆ ?I
proof
  assume subset: {0..<?N} ⊆ ?I
  have ?N = card {0..<?N}
    by simp
  also have ... ≤ card ?I
    by (rule card-mono[OF finite-I subset])
  also have ... ≤ length (List.concat openings)
    by (rule card-I-le)
  finally show False
    using sparse by simp
qed
from not-all obtain j where j-in: j ∈ {0..<?N} and j-notin: j ∉ ?I
  by blast
then have j-bound: j &lt; ?N
  by simp
have unopened:
  ∧ i opn. i &lt; rounds ⇒ opn ∈ set (openings ! i) ⇒
    opening-index opn ≠ j
proof -
  fix i opn
  assume i-bound: i &lt; rounds
    and opn-in: opn ∈ set (openings ! i)
  have openings ! i ∈ set openings
    using i-bound rounds-len by simp
  then have opn ∈ set (List.concat openings)
    using opn-in by auto
  then show opening-index opn ≠ j
    using j-notin by auto
qed
show ?thesis
  by (rule that[OF j-bound unopened])
qed

lemma partial-openings-cross-cover-all-indices-impossible-if-left-sparse:
  assumes rounds-len: length openings = rounds
    and sparse: length (List.concat openings) &lt; scale * clength
  shows ¬ partial-openings-cross-cover-all-indices openings openings'
</pre

```

```

proof
  assume cover:
    partial-openings-cross-cover-all-indices openings openings'
  from cross-sparse-openings-obtain-unopened-index[OF rounds-len sparse]
  obtain j where j-bound: j < scale * clength
  and unopened:
     $\bigwedge i \text{ opn. } i < \text{rounds} \implies \text{opn} \in \text{set } (\text{openings} ! i) \implies$ 
    opening-index opn  $\neq j$ 
  by blast
  from partial-openings-cross-cover-all-indices-imp-left-cover
    [OF cover j-bound]
  obtain i opn where i-bound: i < rounds
  and opn-in: opn  $\in \text{set } (\text{openings} ! i)$ 
  and opn-idx: opening-index opn = j
  by blast
  show False
  using unopened[OF i-bound opn-in] opn-idx by simp
qed

lemma partial-openings-cross-cover-all-indices-impossible-if-right-sparse:
  assumes rounds-len: length openings' = rounds
  and sparse: length (List.concat openings') < scale * clength
  shows  $\neg \text{partial-openings-cross-cover-all-indices openings openings'}$ 
proof –
  have not-sym:
     $\neg \text{partial-openings-cross-cover-all-indices openings' openings}$ 
  by (rule partial-openings-cross-cover-all-indices-impossible-if-left-sparse
    [OF rounds-len sparse])
  show ?thesis
proof
  assume cover:
    partial-openings-cross-cover-all-indices openings openings'
  have partial-openings-cross-cover-all-indices openings' openings
    by (rule partial-openings-cross-cover-all-indices-sym[OF cover])
  then show False
    using not-sym by contradiction
  qed
qed

lemma merkle-path-bound-merkle-mono:
  assumes bound: merkle-path-bound rt len idx v path s
  and ext: merkle-hash-extends s t
  shows merkle-path-bound rt len idx v path t
  using bound
proof (induction path arbitrary: rt len idx)
  case Nil
  then show ?case
    using ext unfolding merkle-hash-extends-def by simp
next

```

```

case (Cons sibling path)
show ?case
proof (cases idx < len div 2)
  case True
  then obtain child where
    child:
      merkle-path-bound child (len div 2) idx v path s
    and lookup:
      fmlookup (HashMap s) (MerkleNode child sibling) = Some rt
    using Cons.premis by auto
  have child-t:
    merkle-path-bound child (len div 2) idx v path t
    by (rule Cons.IH[OF child])
  have lookup-t:
    fmlookup (HashMap t) (MerkleNode child sibling) = Some rt
    using ext lookup unfolding merkle-hash-extends-def by simp
  show ?thesis
    using True child-t lookup-t
    by (auto intro!: exI[of - child])
next
  case False
  then obtain child where
    child:
      merkle-path-bound child (len div 2) (idx - len div 2)
      v path s
    and lookup:
      fmlookup (HashMap s) (MerkleNode sibling child) = Some rt
    using Cons.premis by auto
  have child-t:
    merkle-path-bound child (len div 2) (idx - len div 2)
    v path t
    by (rule Cons.IH[OF child])
  have lookup-t:
    fmlookup (HashMap t) (MerkleNode sibling child) = Some rt
    using ext lookup unfolding merkle-hash-extends-def by simp
  show ?thesis
    using False child-t lookup-t
    by (auto intro!: exI[of - child])
qed
qed

```

```

lemma authenticated-opening-in-merkle-mono:
  assumes opn-auth: authenticated-opening-in s opn
  and ext: merkle-hash-extends s t
  shows authenticated-opening-in t opn
  using opn-auth merkle-path-bound-merkle-mono[OF - ext]
  unfolding authenticated-opening-in-def by blast

```

```

lemma inconsistent-authenticated-openings-imp-merkle-conflict-or-merge-collision:

```


assumes *first: authenticated-opening-in s opening*
and *second: authenticated-opening-in t opening'*
and *same-root: opening-root opening = opening-root opening'*
and *same-length: opening-length opening = opening-length opening'*
and *same-index: opening-index opening = opening-index opening'*
and *different-value: opening-value opening $\neq$ opening-value opening'*
shows
merkle-hash-value-conflict s t $\vee$
hash-map-output-collision (merkle-hash-state-merge s t)
proof (*cases merkle-hash-value-conflict s t*)
case *True*
then show *?thesis* **by** *simp*
next
case *no-conflict: False*
have *compatible: merkle-hash-maps-compatible s t*
using *no-conflict*
unfolding *merkle-hash-maps-compatible-iff-no-value-conflict .*
let *?u = merkle-hash-state-merge s t*
have *s-ext: merkle-hash-extends s ?u*
by (*rule merkle-hash-state-merge-extends-left-if-compatible*
[OF compatible])
have *t-ext: merkle-hash-extends t ?u*
by (*rule merkle-hash-state-merge-extends-right*)
have *first-u: authenticated-opening-in ?u opening*
by (*rule authenticated-opening-in-merkle-mono[OF first s-ext]*)
have *second-u: authenticated-opening-in ?u opening'*
by (*rule authenticated-opening-in-merkle-mono[OF second t-ext]*)
have *collision: hash-map-output-collision ?u*
by (*rule inconsistent-authenticated-openings-imp-hash-collision*
[OF first-u second-u same-root same-length same-index
different-value])
then show *?thesis* **by** *simp*
qed

lemma *authenticated-openings-values-eq-if-no-merkle-conflict-clean-merge:*
assumes *first: authenticated-opening-in s opening*
and *second: authenticated-opening-in t opening'*
and *same-root: opening-root opening = opening-root opening'*
and *same-length: opening-length opening = opening-length opening'*
and *same-index: opening-index opening = opening-index opening'*
and *no-conflict: $\neg$ merkle-hash-value-conflict s t*
and *clean-merge:*
 $\neg$ hash-map-output-collision (merkle-hash-state-merge s t)
shows *opening-value opening = opening-value opening'*
proof (*rule ccontr*)
assume *different:*
opening-value opening $\neq$ opening-value opening'
have
merkle-hash-value-conflict s t $\vee$

```

      hash-map-output-collision (merkle-hash-state-merge s t)
    by (rule
      inconsistent-authenticated-openings-imp-merkle-conflict-or-merge-collision
      [OF first second same-root same-length same-index different])
    then show False
      using no-conflict clean-merge by blast
  qed

```

lemma *partial-authenticated-tables-values-eq-if-no-merkle-conflict-clean-merge:*

assumes *table:*

partial-authenticated-table *rt len openings s*

and *table':*

partial-authenticated-table *rt len openings' t*

and *opn-in:* *opn* $\in$ *set openings*

and *opn'-in:* *opn'* $\in$ *set openings'*

and *same-index:* *opening-index opn* = *opening-index opn'*

and *no-conflict:* $\neg$ *merkle-hash-value-conflict s t*

and *clean-merge:*

$\neg$ *hash-map-output-collision (merkle-hash-state-merge s t)*

shows *opening-value opn* = *opening-value opn'*

proof –

have *first:* *authenticated-opening-in s opn*

using *table opn-in unfolding partial-authenticated-table-def* **by** *blast*

have *second:* *authenticated-opening-in t opn'*

using *table' opn'-in unfolding partial-authenticated-table-def* **by** *blast*

have *root:* *opening-root opn* = *rt*

using *table opn-in unfolding partial-authenticated-table-def* **by** *blast*

have *root':* *opening-root opn'* = *rt*

using *table' opn'-in unfolding partial-authenticated-table-def* **by** *blast*

have *same-root:* *opening-root opn* = *opening-root opn'*

using *root root'* **by** *simp*

have *len:* *opening-length opn* = *len*

using *table opn-in unfolding partial-authenticated-table-def* **by** *blast*

have *len':* *opening-length opn'* = *len*

using *table' opn'-in unfolding partial-authenticated-table-def* **by** *blast*

have *same-length:* *opening-length opn* = *opening-length opn'*

using *len len'* **by** *simp*

show *?thesis*

by (rule *authenticated-openings-values-eq-if-no-merkle-conflict-clean-merge*

[OF *first second same-root same-length same-index no-conflict clean-merge*])

qed

lemma *partial-trace-openings-values-eq-if-no-merkle-conflict-clean-merge:*

assumes *partial:*

accepted-with-partial-trace-openings s

(*Some (result, final-state)*) *fr query-idxs trace-openings*

and *partial':*

accepted-with-partial-trace-openings s'

```

    (Some (result', final-state')) fr query-idxs' trace-openings'
  and i-bound: i < rounds
  and i'-bound: i' < rounds
  and opn-in: opn ∈ set (trace-openings ! i)
  and opn'-in: opn' ∈ set (trace-openings' ! i')
  and same-index: opening-index opn = opening-index opn'
  and no-conflict:
    ¬ merkle-hash-value-conflict final-state final-state'
  and clean-merge:
    ¬ hash-map-output-collision
      (merkle-hash-state-merge final-state final-state')
  shows opening-value opn = opening-value opn'
proof -
  have table:
    partial-authenticated-table fr (scale * clength)
      (trace-openings ! i) final-state
    using partial i-bound
    unfolding accepted-with-partial-trace-openings-def by blast
  have table':
    partial-authenticated-table fr (scale * clength)
      (trace-openings' ! i') final-state'
    using partial' i'-bound
    unfolding accepted-with-partial-trace-openings-def by blast
  show ?thesis
    by (rule
      partial-authenticated-tables-values-eq-if-no-merkle-conflict-clean-merge
      [OF table table' opn-in opn'-in same-index no-conflict
        clean-merge])

```

qed

lemma *partial-composition-openings-values-eq-if-no-merkle-conflict-clean-merge:*

```

  assumes partial:
    accepted-with-partial-composition-openings s
    (Some (result, final-state)) composition-root query-idxs
    composition-openings
  and partial':
    accepted-with-partial-composition-openings s'
    (Some (result', final-state')) composition-root query-idxs'
    composition-openings'
  and i-bound: i < rounds
  and i'-bound: i' < rounds
  and opn-in: opn ∈ set (composition-openings ! i)
  and opn'-in: opn' ∈ set (composition-openings' ! i')
  and same-index: opening-index opn = opening-index opn'
  and no-conflict:
    ¬ merkle-hash-value-conflict final-state final-state'
  and clean-merge:
    ¬ hash-map-output-collision
      (merkle-hash-state-merge final-state final-state')

```

```

shows opening-value opn = opening-value opn'
proof –
  have table:
    partial-authenticated-table composition-root (scale * clength)
      (composition-openings ! i) final-state
    using partial i-bound
    unfolding accepted-with-partial-composition-openings-def by blast
  have table':
    partial-authenticated-table composition-root (scale * clength)
      (composition-openings' ! i') final-state'
    using partial' i'-bound
    unfolding accepted-with-partial-composition-openings-def by blast
  show ?thesis
  by (rule
    partial-authenticated-tables-values-eq-if-no-merkle-conflict-clean-merge
    [OF table table' opn-in opn'-in same-index no-conflict
      clean-merge])
qed

```

lemma *partial-trace-table-candidates-eq-if-cross-openings-cover-clean-merge*:

```

assumes partial:
  accepted-with-partial-trace-openings s
  (Some (result, final-state)) fr query-idxs trace-openings
and partial':
  accepted-with-partial-trace-openings s'
  (Some (result', final-state')) fr query-idxs' trace-openings'
and cand: partial-trace-table-candidate trace-table trace-openings
and cand':
  partial-trace-table-candidate trace-table' trace-openings'
and cover:
   $\bigwedge j. j < \text{scale} * \text{clength} \implies$ 
     $\exists i < \text{rounds}. \exists i' < \text{rounds}.$ 
     $\exists \text{opn} \in \text{set } (\text{trace-openings} ! i).$ 
     $\exists \text{opn}' \in \text{set } (\text{trace-openings}' ! i').$ 
     $\text{opening-index } \text{opn} = j \wedge$ 
     $\text{opening-index } \text{opn}' = j$ 
and no-conflict:
   $\neg \text{merkle-hash-value-conflict } \text{final-state } \text{final-state}'$ 
and clean-merge:
   $\neg \text{hash-map-output-collision}$ 
  (merkle-hash-state-merge final-state final-state')
shows trace-table = trace-table'
proof (rule nth-equalityI)
  show length trace-table = length trace-table'
  using cand cand' unfolding partial-trace-table-candidate-def by simp
next
  fix j
  assume j-bound: j < length trace-table
  have len: length trace-table = scale * clength

```

```

    using cand unfolding partial-trace-table-candidate-def by simp
  from cover[OF j-bound[unfolded len]]
  obtain i i' opn opn' where
    i-bound: i < rounds
    and i'-bound: i' < rounds
    and opn-in: opn ∈ set (trace-openings ! i)
    and opn'-in: opn' ∈ set (trace-openings' ! i')
    and opn-idx: opening-index opn = j
    and opn'-idx: opening-index opn' = j
  by blast
  have trace-val: trace-table ! j = opening-value opn
    using partial-trace-table-candidateD(3)[OF cand i-bound opn-in]
    opn-idx by simp
  have trace'-val: trace-table' ! j = opening-value opn'
    using partial-trace-table-candidateD(3)[OF cand' i'-bound opn'-in]
    opn'-idx by simp
  have opening-values:
    opening-value opn = opening-value opn'
  by (rule partial-trace-openings-values-eq-if-no-merkle-conflict-clean-merge
    [OF partial partial' i-bound i'-bound opn-in opn'-in - no-conflict
      clean-merge])
    (use opn-idx opn'-idx in simp)
  show trace-table ! j = trace-table' ! j
    using trace-val trace'-val opening-values by simp
qed

```

lemma *partial-trace-table-candidates-eq-if-cross-cover-clean-merge*:

```

  assumes partial:
    accepted-with-partial-trace-openings s
    (Some (result, final-state)) fr query-idxs trace-openings
  and partial':
    accepted-with-partial-trace-openings s'
    (Some (result', final-state')) fr query-idxs' trace-openings'
  and cand: partial-trace-table-candidate trace-table trace-openings
  and cand':
    partial-trace-table-candidate trace-table' trace-openings'
  and cover:
    partial-openings-cross-cover-all-indices trace-openings
    trace-openings'
  and no-conflict:
    ¬ merkle-hash-value-conflict final-state final-state'
  and clean-merge:
    ¬ hash-map-output-collision
    (merkle-hash-state-merge final-state final-state')
  shows trace-table = trace-table'

```

```

proof (rule
  partial-trace-table-candidates-eq-if-cross-openings-cover-clean-merge
  [OF partial partial' cand cand' - no-conflict clean-merge])
  fix j

```

```

assume  $j < \text{scale} * \text{clength}$ 
from partial-openings-cross-cover-all-indicesD[OF cover this]
obtain  $i \ i' \ \text{opn} \ \text{opn}'$  where
   $i < \text{rounds} \ i' < \text{rounds}$ 
   $\text{opn} \in \text{set} \ (\text{trace-openings} \ ! \ i)$ 
   $\text{opn}' \in \text{set} \ (\text{trace-openings}' \ ! \ i')$ 
   $\text{opening-index} \ \text{opn} = j$ 
   $\text{opening-index} \ \text{opn}' = j$ 
by blast
then show
   $\exists i < \text{rounds}. \exists i' < \text{rounds}.$ 
   $\exists \text{opn} \in \text{set} \ (\text{trace-openings} \ ! \ i).$ 
   $\exists \text{opn}' \in \text{set} \ (\text{trace-openings}' \ ! \ i').$ 
   $\text{opening-index} \ \text{opn} = j \wedge \text{opening-index} \ \text{opn}' = j$ 
by blast
qed

lemma partial-composition-table-candidates-eq-if-cross-openings-cover-clean-merge:
assumes partial:
  accepted-with-partial-composition-openings s
   $(\text{Some} \ (\text{result}, \ \text{final-state})) \ \text{composition-root} \ \text{query-idxs}$ 
  composition-openings
and partial':
  accepted-with-partial-composition-openings s'
   $(\text{Some} \ (\text{result}', \ \text{final-state}')) \ \text{composition-root} \ \text{query-idxs}'$ 
  composition-openings'
and cand:
  partial-composition-table-candidate composition-table
  composition-openings
and cand':
  partial-composition-table-candidate composition-table'
  composition-openings'
and cover:
   $\bigwedge j. j < \text{scale} * \text{clength} \implies$ 
   $\exists i < \text{rounds}. \exists i' < \text{rounds}.$ 
   $\exists \text{opn} \in \text{set} \ (\text{composition-openings} \ ! \ i).$ 
   $\exists \text{opn}' \in \text{set} \ (\text{composition-openings}' \ ! \ i').$ 
   $\text{opening-index} \ \text{opn} = j \wedge$ 
   $\text{opening-index} \ \text{opn}' = j$ 
and no-conflict:
   $\neg \text{merkle-hash-value-conflict} \ \text{final-state} \ \text{final-state}'$ 
and clean-merge:
   $\neg \text{hash-map-output-collision}$ 
   $(\text{merkle-hash-state-merge} \ \text{final-state} \ \text{final-state}')$ 
shows  $\text{composition-table} = \text{composition-table}'$ 
proof (rule nth-equalityI)
show  $\text{length} \ \text{composition-table} = \text{length} \ \text{composition-table}'$ 
using cand cand'
unfolding partial-composition-table-candidate-def by simp

```

```

next
  fix j
  assume j-bound:  $j < \text{length composition-table}$ 
  have len:  $\text{length composition-table} = \text{scale} * \text{clength}$ 
    using and unfolding partial-composition-table-candidate-def by simp
  from cover[OF j-bound[unfolded len]]
  obtain  $i\ i'\ \text{opn}\ \text{opn}'$  where
     $i\text{-bound}: i < \text{rounds}$ 
    and  $i'\text{-bound}: i' < \text{rounds}$ 
    and  $\text{opn-in}: \text{opn} \in \text{set}(\text{composition-openings} ! i)$ 
    and  $\text{opn}'\text{-in}: \text{opn}' \in \text{set}(\text{composition-openings}' ! i')$ 
    and  $\text{opn-idx}: \text{opening-index opn} = j$ 
    and  $\text{opn}'\text{-idx}: \text{opening-index opn}' = j$ 
    by blast
  have composition-val:
     $\text{composition-table} ! j = \text{opening-value opn}$ 
    using partial-composition-table-candidateD( $\beta$ )[OF cand i-bound opn-in]
     $\text{opn-idx}$  by simp
  have composition'-val:
     $\text{composition-table}' ! j = \text{opening-value opn}'$ 
    using partial-composition-table-candidateD( $\beta$ )[OF cand' i'-bound opn'-in]
     $\text{opn}'\text{-idx}$  by simp
  have opening-values:
     $\text{opening-value opn} = \text{opening-value opn}'$ 
    by (rule
      partial-composition-openings-values-eq-if-no-merkle-conflict-clean-merge
      [OF partial partial' i-bound i'-bound opn-in opn'-in - no-conflict
        clean-merge])
      (use  $\text{opn-idx}\ \text{opn}'\text{-idx}$  in simp)
  show  $\text{composition-table} ! j = \text{composition-table}' ! j$ 
    using composition-val composition'-val opening-values by simp
qed

```

lemma *partial-composition-table-candidates-eq-if-cross-cover-clean-merge*:

```

assumes partial:
  accepted-with-partial-composition-openings s
  (Some (result, final-state)) composition-root query-idxs
  composition-openings
and partial':
  accepted-with-partial-composition-openings s'
  (Some (result', final-state')) composition-root query-idxs'
  composition-openings'
and cand:
  partial-composition-table-candidate composition-table
  composition-openings
and cand':
  partial-composition-table-candidate composition-table'
  composition-openings'
and cover:

```

```

    partial-openings-cross-cover-all-indices composition-openings
    composition-openings'
  and no-conflict:
    ¬ merkle-hash-value-conflict final-state final-state'
  and clean-merge:
    ¬ hash-map-output-collision
    (merkle-hash-state-merge final-state final-state')
  shows composition-table = composition-table'
proof (rule
  partial-composition-table-candidates-eq-if-cross-openings-cover-clean-merge
  [OF partial partial' cand cand' - no-conflict clean-merge])
fix j
assume j < scale * clength
from partial-openings-cross-cover-all-indicesD[OF cover this]
obtain i i' opn opn' where
  i < rounds i' < rounds
  opn ∈ set (composition-openings ! i)
  opn' ∈ set (composition-openings' ! i')
  opening-index opn = j
  opening-index opn' = j
by blast
then show
  ∃ i < rounds. ∃ i' < rounds.
  ∃ opn ∈ set (composition-openings ! i).
  ∃ opn' ∈ set (composition-openings' ! i').
  opening-index opn = j ∧ opening-index opn' = j
by blast
qed

end

end

```

```

theory Soundness-Query-Header-Cross-Candidate
imports
  Soundness-Reductions-Composition
  Soundness-Partial-Merkle-Cross
begin

```

Witnessed query-header partial candidates.

The public query-header candidate set hides the authenticated opening witnesses behind existentials. This layer exposes those witnesses so that the cross-state partial-Merkle binding lemmas can be applied before reducing back to the old candidate-set interface.

```

context soundness
begin

```

```

definition query-header-supported-partial-table-candidate-witness
  :: ('f, 'a) protocol-channel-scheme ⇒

```


$'f \Rightarrow 'f \text{ list} \Rightarrow 'f \Rightarrow 'f \text{ list} \Rightarrow 'f \Rightarrow$
 $'f \text{ list} \Rightarrow 'f \Rightarrow 'f \text{ list} \Rightarrow 'f \text{ list} \Rightarrow$
 $('f, 'a) \text{ protocol-channel-scheme} \Rightarrow$
 $'f \text{ authenticated-opening list list} \Rightarrow$
 $'f \text{ authenticated-opening list list} \Rightarrow \text{bool}$

where

query-header-supported-partial-table-candidate-witness s fr f-fri-roots
f-final as dg composition-fri-roots final trace-table
composition-table final-state trace-openings composition-openings
 $\longleftrightarrow$
composition-fri-roots $\neq [] \wedge$
 $(\exists \text{result trace-query-idxs composition-query-idxs rest.}$
 $\text{Some (result, final-state)} \in \text{set-dist (execute verify-monad s)} \wedge$
 $\text{accepted-with-partial-trace-openings s}$
 $(\text{Some (result, final-state)}) \text{ fr trace-query-idxs}$
 $\text{trace-openings} \wedge$
 $\text{partial-trace-table-candidate trace-table trace-openings} \wedge$
 $\text{accepted-with-partial-composition-openings s}$
 $(\text{Some (result, final-state)}) (\text{hd composition-fri-roots})$
 $\text{composition-query-idxs composition-openings} \wedge$
 $\text{partial-composition-table-candidate composition-table}$
 $\text{composition-openings} \wedge$
 $\text{verifier-header-transcript s fr f-fri-roots f-final as dg}$
 $\text{composition-fri-roots final rest})$

lemma *query-header-supported-partial-table-candidate-witness-imp-state:*

assumes

query-header-supported-partial-table-candidate-witness s fr f-fri-roots
f-final as dg composition-fri-roots final trace-table
composition-table final-state trace-openings composition-openings

shows

query-header-supported-partial-table-candidate-state s fr f-fri-roots
f-final as dg composition-fri-roots final trace-table
composition-table final-state

using *assms*

unfolding *query-header-supported-partial-table-candidate-witness-def*

query-header-supported-partial-table-candidate-state-def

by *blast*

lemma *query-header-supported-partial-table-candidate-state-obtain-witness:*

assumes

query-header-supported-partial-table-candidate-state s fr f-fri-roots
f-final as dg composition-fri-roots final trace-table
composition-table final-state

obtains *trace-openings composition-openings* **where**

query-header-supported-partial-table-candidate-witness s fr f-fri-roots
f-final as dg composition-fri-roots final trace-table
composition-table final-state trace-openings composition-openings

using *assms*

unfolding *query-header-supported-partial-table-candidate-witness-def*
query-header-supported-partial-table-candidate-state-def
by *blast*

lemma *query-header-supported-partial-table-candidate-witness-imp-candidate:*

assumes

query-header-supported-partial-table-candidate-witness s fr f-fri-roots
f-final as dg composition-fri-roots final trace-table
composition-table final-state trace-openings composition-openings

shows

$(\text{trace-table}, \text{composition-table}) \in$
query-header-supported-partial-table-candidates s fr f-fri-roots
f-final as dg composition-fri-roots final

using *query-header-supported-partial-table-candidate-witness-imp-state*
 $[OF \text{ assms}]$

query-header-supported-partial-table-candidates-iff-state

by *blast*

lemma *query-header-supported-partial-table-candidates-obtain-witness:*

assumes

$(\text{trace-table}, \text{composition-table}) \in$
query-header-supported-partial-table-candidates s fr f-fri-roots
f-final as dg composition-fri-roots final

obtains *final-state trace-openings composition-openings* **where**

query-header-supported-partial-table-candidate-witness s fr f-fri-roots
f-final as dg composition-fri-roots final trace-table
composition-table final-state trace-openings composition-openings

proof —

from *assms* **obtain** *final-state* **where** *state:*

query-header-supported-partial-table-candidate-state s fr f-fri-roots
f-final as dg composition-fri-roots final trace-table
composition-table final-state

using *query-header-supported-partial-table-candidates-iff-state* **by** *blast*

from *query-header-supported-partial-table-candidate-state-obtain-witness*

$[OF \text{ state}]$

obtain *trace-openings composition-openings* **where** *witness:*

query-header-supported-partial-table-candidate-witness s fr f-fri-roots
f-final as dg composition-fri-roots final trace-table
composition-table final-state trace-openings composition-openings

by *blast*

show *?thesis*

by $(\text{rule that}[OF \text{ witness}])$

qed

lemma *query-header-supported-partial-table-candidate-witness-shapes:*

assumes *witness:*

query-header-supported-partial-table-candidate-witness s fr f-fri-roots
f-final as dg composition-fri-roots final trace-table
composition-table final-state trace-openings composition-openings

shows

partial-trace-table-candidate trace-table trace-openings

partial-composition-table-candidate composition-table composition-openings

using *witness*

unfolding *query-header-supported-partial-table-candidate-witness-def*

by *blast+*

lemma *query-header-supported-partial-table-candidate-witness-eq-if-cross-cover-clean-merge:*

assumes *witness:*

query-header-supported-partial-table-candidate-witness s fr f-fri-roots

f-final as dg composition-fri-roots final trace-table

composition-table final-state trace-openings composition-openings

and *witness':*

query-header-supported-partial-table-candidate-witness s fr f-fri-roots

f-final as dg composition-fri-roots final trace-table'

composition-table' final-state' trace-openings' composition-openings'

and *trace-cover:*

partial-openings-cross-cover-all-indices trace-openings

trace-openings'

and *composition-cover:*

partial-openings-cross-cover-all-indices composition-openings

composition-openings'

and *no-conflict:*

$\neg$ *merkle-hash-value-conflict final-state final-state'*

and *clean-merge:*

$\neg$ *hash-map-output-collision*

(merkle-hash-state-merge final-state final-state')

shows *trace-table = trace-table' $\wedge$ composition-table = composition-table'*

proof –

from *witness* **obtain** *result trace-query-idxs composition-query-idxs rest* **where**

trace-partial:

accepted-with-partial-trace-openings s

(Some (result, final-state)) fr trace-query-idxs trace-openings

and *comp-partial:*

accepted-with-partial-composition-openings s

(Some (result, final-state)) (hd composition-fri-roots)

composition-query-idxs composition-openings

and *trace-cand:*

partial-trace-table-candidate trace-table trace-openings

and *comp-cand:*

partial-composition-table-candidate composition-table

composition-openings

unfolding *query-header-supported-partial-table-candidate-witness-def*

by *blast*

from *witness'* **obtain** *result' trace-query-idxs' composition-query-idxs' rest'*

where *trace-partial':*

accepted-with-partial-trace-openings s

(Some (result', final-state')) fr trace-query-idxs'

trace-openings'

```

and comp-partial':
  accepted-with-partial-composition-openings s
  (Some (result', final-state')) (hd composition-fri-roots)
  composition-query-idxs' composition-openings'
and trace-cand':
  partial-trace-table-candidate trace-table' trace-openings'
and comp-cand':
  partial-composition-table-candidate composition-table'
  composition-openings'
unfolding query-header-supported-partial-table-candidate-witness-def
by blast
have trace-eq: trace-table = trace-table'
by (rule partial-trace-table-candidates-eq-if-cross-cover-clean-merge
  [OF trace-partial trace-partial' trace-cand trace-cand' trace-cover
    no-conflict clean-merge])
have comp-eq: composition-table = composition-table'
by (rule partial-composition-table-candidates-eq-if-cross-cover-clean-merge
  [OF comp-partial comp-partial' comp-cand comp-cand'
    composition-cover no-conflict clean-merge])
show ?thesis
  using trace-eq comp-eq by simp
qed

lemma query-header-supported-partial-table-candidates-subset-singleton-if-pairwise-cross-cover-clean-merge:
assumes pairwise:
   $\bigwedge$  trace-table composition-table final-state trace-openings
  composition-openings trace-table' composition-table' final-state'
  trace-openings' composition-openings'.
  query-header-supported-partial-table-candidate-witness s fr
  f-fri-roots f-final as dg composition-fri-roots final trace-table
  composition-table final-state trace-openings composition-openings
 $\implies$ 
  query-header-supported-partial-table-candidate-witness s fr
  f-fri-roots f-final as dg composition-fri-roots final trace-table'
  composition-table' final-state' trace-openings' composition-openings'
 $\implies$ 
  partial-openings-cross-cover-all-indices trace-openings
  trace-openings'  $\wedge$ 
  partial-openings-cross-cover-all-indices composition-openings
  composition-openings'  $\wedge$ 
   $\neg$  merkle-hash-value-conflict final-state final-state'  $\wedge$ 
   $\neg$  hash-map-output-collision
  (merkle-hash-state-merge final-state final-state')
shows
   $\exists$  trace-table composition-table.
  query-header-supported-partial-table-candidates s fr f-fri-roots
  f-final as dg composition-fri-roots final  $\subseteq$ 
   $\{(trace-table, composition-table)\}$ 
proof (cases

```

```

    query-header-supported-partial-table-candidates s fr f-fri-roots
      f-final as dg composition-fri-roots final = {}
  case True
  then show ?thesis
    by (intro exI[of -] exI[of -]) simp
next
case False
then obtain pair0 where pair0-in:
  pair0 ∈
    query-header-supported-partial-table-candidates s fr f-fri-roots
      f-final as dg composition-fri-roots final
  by blast
then obtain trace-table0 composition-table0 where pair0-eq:
  pair0 = (trace-table0, composition-table0)
  by (cases pair0)
have cand0:
  (trace-table0, composition-table0) ∈
    query-header-supported-partial-table-candidates s fr f-fri-roots
      f-final as dg composition-fri-roots final
  using pair0-in unfolding pair0-eq .
from query-header-supported-partial-table-candidates-obtain-witness[OF cand0]
obtain final-state0 trace-openings0 composition-openings0 where witness0:
  query-header-supported-partial-table-candidate-witness s fr f-fri-roots
    f-final as dg composition-fri-roots final trace-table0
    composition-table0 final-state0 trace-openings0
    composition-openings0
  by blast
have subset:
  query-header-supported-partial-table-candidates s fr f-fri-roots
    f-final as dg composition-fri-roots final ⊆
    {(trace-table0, composition-table0)}
proof
  fix pair
  assume pair-in:
    pair ∈
      query-header-supported-partial-table-candidates s fr f-fri-roots
        f-final as dg composition-fri-roots final
  obtain trace-table composition-table where pair-eq:
    pair = (trace-table, composition-table)
    by (cases pair)
  from query-header-supported-partial-table-candidates-obtain-witness
    [OF pair-in[unfolded pair-eq]]
  obtain final-state trace-openings composition-openings where witness:
    query-header-supported-partial-table-candidate-witness s fr
      f-fri-roots f-final as dg composition-fri-roots final trace-table
      composition-table final-state trace-openings composition-openings
    by blast
  have pairwise-facts:
    partial-openings-cross-cover-all-indices trace-openings

```

```

    trace-openings0
  partial-openings-cross-cover-all-indices composition-openings
    composition-openings0
  ¬ merkle-hash-value-conflict final-state final-state0
  ¬ hash-map-output-collision
    (merkle-hash-state-merge final-state final-state0)
  using pairwise[OF witness witness0] by simp-all
have eq:
  trace-table = trace-table0 ∧ composition-table = composition-table0
by (rule
  query-header-supported-partial-table-candidate-witness-eq-if-cross-cover-clean-merge
  [OF witness witness0 pairwise-facts])
show pair ∈ {(trace-table0, composition-table0)}
  using pair-eq eq by simp
qed
then show ?thesis
  by blast
qed

lemma query-header-supported-partial-table-candidates-not-singleton-imp-cross-cover-or-merkle-bad:
  assumes not-unique:
    ¬ (∃ trace-table composition-table.
      query-header-supported-partial-table-candidates s fr f-fri-roots
        f-final as dg composition-fri-roots final ⊆
        {(trace-table, composition-table)})
  shows
    ∃ trace-table composition-table final-state trace-openings
      composition-openings trace-table' composition-table' final-state'
      trace-openings' composition-openings'.
      query-header-supported-partial-table-candidate-witness s fr
        f-fri-roots f-final as dg composition-fri-roots final trace-table
        composition-table final-state trace-openings composition-openings ∧
      query-header-supported-partial-table-candidate-witness s fr
        f-fri-roots f-final as dg composition-fri-roots final trace-table'
        composition-table' final-state' trace-openings'
        composition-openings' ∧
      (trace-table ≠ trace-table' ∨
        composition-table ≠ composition-table') ∧
      (¬ partial-openings-cross-cover-all-indices trace-openings
        trace-openings' ∨
        ¬ partial-openings-cross-cover-all-indices composition-openings
        composition-openings' ∨
        merkle-hash-value-conflict final-state final-state' ∨
        hash-map-output-collision
        (merkle-hash-state-merge final-state final-state'))
proof -
  let ?C =
    query-header-supported-partial-table-candidates s fr f-fri-roots
      f-final as dg composition-fri-roots final

```

```

have nonempty:  $?C \neq \{\}$ 
  using not-unique by auto
then obtain pair where pair-in:  $pair \in ?C$ 
  by blast
obtain trace-table composition-table where pair-eq:
  pair = (trace-table, composition-table)
  by (cases pair)
have cand:
  (trace-table, composition-table)  $\in ?C$ 
  using pair-in unfolding pair-eq .
have not-subset:  $\neg ?C \subseteq \{(trace-table, composition-table)\}$ 
  using not-unique by blast
then obtain pair' where pair'-in:  $pair' \in ?C$ 
  and pair'-not:  $pair' \notin \{(trace-table, composition-table)\}$ 
  by auto
obtain trace-table' composition-table' where pair'-eq:
  pair' = (trace-table', composition-table')
  by (cases pair')
have cand':
  (trace-table', composition-table')  $\in ?C$ 
  using pair'-in unfolding pair'-eq .
have distinct:
  trace-table  $\neq$  trace-table'  $\vee$ 
  composition-table  $\neq$  composition-table'
  using pair'-not unfolding pair'-eq by auto
from query-header-supported-partial-table-candidates-obtain-witness[OF cand]
obtain final-state trace-openings composition-openings where witness:
  query-header-supported-partial-table-candidate-witness s fr
  f-fri-roots f-final as dg composition-fri-roots final trace-table
  composition-table final-state trace-openings composition-openings
  by blast
from query-header-supported-partial-table-candidates-obtain-witness[OF cand']
obtain final-state' trace-openings' composition-openings' where witness':
  query-header-supported-partial-table-candidate-witness s fr
  f-fri-roots f-final as dg composition-fri-roots final trace-table'
  composition-table' final-state' trace-openings'
  composition-openings'
  by blast
have side:
   $\neg$  partial-openings-cross-cover-all-indices trace-openings
    trace-openings'  $\vee$ 
   $\neg$  partial-openings-cross-cover-all-indices composition-openings
    composition-openings'  $\vee$ 
  merkle-hash-value-conflict final-state final-state'  $\vee$ 
  hash-map-output-collision
    (merkle-hash-state-merge final-state final-state')
proof (rule ccontr)
assume no-side:
   $\neg$  ( $\neg$  partial-openings-cross-cover-all-indices trace-openings

```

```

    trace-openings'  $\vee$ 
 $\neg$  partial-openings-cross-cover-all-indices composition-openings
    composition-openings'  $\vee$ 
    merkle-hash-value-conflict final-state final-state'  $\vee$ 
    hash-map-output-collision
    (merkle-hash-state-merge final-state final-state'))
have eq:
    trace-table = trace-table'  $\wedge$ 
    composition-table = composition-table'
by (rule
    query-header-supported-partial-table-candidate-witness-eq-if-cross-cover-clean-merge
    [OF witness witness'])
    (use no-side in auto)
then show False
    using distinct by simp
qed
show ?thesis
    using witness witness' distinct side by blast
qed

```

definition query-header-supported-partial-pairwise-cross-or-merkle-bad

```

:: ('f, 'a) protocol-channel-scheme  $\Rightarrow$ 
    'f  $\Rightarrow$  'f list  $\Rightarrow$  'f  $\Rightarrow$  'f list  $\Rightarrow$  'f  $\Rightarrow$ 
    'f list  $\Rightarrow$  'f  $\Rightarrow$  bool

```

where

```

query-header-supported-partial-pairwise-cross-or-merkle-bad s fr
    f-fri-roots f-final as dg composition-fri-roots final  $\longleftrightarrow$ 
    ( $\exists$  trace-table composition-table final-state trace-openings
        composition-openings trace-table' composition-table' final-state'
        trace-openings' composition-openings'.
    query-header-supported-partial-table-candidate-witness s fr
        f-fri-roots f-final as dg composition-fri-roots final trace-table
        composition-table final-state trace-openings composition-openings  $\wedge$ 
    query-header-supported-partial-table-candidate-witness s fr
        f-fri-roots f-final as dg composition-fri-roots final trace-table'
        composition-table' final-state' trace-openings'
        composition-openings'  $\wedge$ 
    (trace-table  $\neq$  trace-table'  $\vee$ 
        composition-table  $\neq$  composition-table')  $\wedge$ 
    ( $\neg$  partial-openings-cross-cover-all-indices trace-openings
        trace-openings'  $\vee$ 
 $\neg$  partial-openings-cross-cover-all-indices composition-openings
        composition-openings'  $\vee$ 
        merkle-hash-value-conflict final-state final-state'  $\vee$ 
        hash-map-output-collision
        (merkle-hash-state-merge final-state final-state'))))

```

definition query-supported-partial-pairwise-cross-or-merkle-bad

```

:: ('f, 'a) protocol-channel-scheme  $\Rightarrow$  bool

```


where

query-supported-partial-pairwise-cross-or-merkle-bad $s \longleftrightarrow$
($\exists$ *fr* *f-fri-roots* *f-final* as *dg composition-fri-roots final*.
query-header-supported-partial-pairwise-cross-or-merkle-bad *s fr*
f-fri-roots f-final as *dg composition-fri-roots final*)

lemma *query-header-supported-partial-pairwise-cross-or-merkle-bad-if-not-singleton*:

assumes *not-unique*:

$\neg (\exists$ *trace-table composition-table*.
query-header-supported-partial-table-candidates *s fr f-fri-roots*
f-final as *dg composition-fri-roots final* $\subseteq$
 $\{(trace-table, composition-table)\})$

shows

query-header-supported-partial-pairwise-cross-or-merkle-bad *s fr*
f-fri-roots f-final as *dg composition-fri-roots final*

unfolding *query-header-supported-partial-pairwise-cross-or-merkle-bad-def*

using

query-header-supported-partial-table-candidates-not-singleton-imp-cross-cover-or-merkle-bad
[*OF not-unique*]

by *blast*

lemma *query-header-supported-partial-table-candidates-subset-singleton-if-no-cross-or-merkle-bad*:

assumes *no-bad*:

$\neg$ *query-header-supported-partial-pairwise-cross-or-merkle-bad* *s fr*
f-fri-roots f-final as *dg composition-fri-roots final*

shows

$\exists$ *trace-table composition-table*.
query-header-supported-partial-table-candidates *s fr f-fri-roots*
f-final as *dg composition-fri-roots final* $\subseteq$
 $\{(trace-table, composition-table)\}$

proof (rule *ccontr*)

assume *not-unique*:

$\neg (\exists$ *trace-table composition-table*.
query-header-supported-partial-table-candidates *s fr f-fri-roots*
f-final as *dg composition-fri-roots final* $\subseteq$
 $\{(trace-table, composition-table)\})$

then have

query-header-supported-partial-pairwise-cross-or-merkle-bad *s fr*
f-fri-roots f-final as *dg composition-fri-roots final*

unfolding *query-header-supported-partial-pairwise-cross-or-merkle-bad-def*

using

query-header-supported-partial-table-candidates-not-singleton-imp-cross-cover-or-merkle-bad
by *blast*

then show *False*

using *no-bad* **by** *contradiction*

qed

lemma *query-supported-partial-pairwise-cross-or-merkle-badD*:

assumes

query-supported-partial-pairwise-cross-or-merkle-bad s
obtains *fr f-fri-roots f-final as dg composition-fri-roots final* **where**
query-header-supported-partial-pairwise-cross-or-merkle-bad s fr
f-fri-roots f-final as dg composition-fri-roots final
using *assms*
unfolding *query-supported-partial-pairwise-cross-or-merkle-bad-def* **by** *blast*

lemma *query-header-supported-partial-table-candidates-subset-singleton-if-no-global-cross-or-merkle-bad:*
assumes *no-bad:*
 $\neg$ *query-supported-partial-pairwise-cross-or-merkle-bad s*
shows
 $\exists$ *trace-table composition-table.*
query-header-supported-partial-table-candidates s fr f-fri-roots
f-final as dg composition-fri-roots final $\subseteq$
 $\{(trace-table, composition-table)\}$
proof (rule
query-header-supported-partial-table-candidates-subset-singleton-if-no-cross-or-merkle-bad)
show
 $\neg$ *query-header-supported-partial-pairwise-cross-or-merkle-bad s fr*
f-fri-roots f-final as dg composition-fri-roots final
using *no-bad*
unfolding *query-supported-partial-pairwise-cross-or-merkle-bad-def* **by** *blast*
qed

lemma *query-header-supported-partial-union-good-sets-fraction-bound-if-no-cross-or-merkle-bad:*
assumes *no-bad:*
 $\neg$ *query-header-supported-partial-pairwise-cross-or-merkle-bad s fr*
f-fri-roots f-final as dg composition-fri-roots final
and *subset:*
 $\bigwedge$ *trace-table composition-table as.*
good-sets trace-table composition-table as $\subseteq$ *query-sample-space*
and *bounded:*
 $\bigwedge$ *trace-table composition-table as.*
 $nnreal (card (good-sets trace-table composition-table as)) /$
 $nnreal (card query-sample-space) \leq query-error-bound$
shows
 $nnreal$
 $(card$
 $(query-header-supported-partial-union-good-sets s good-sets fr$
 $f-fri-roots f-final as dg composition-fri-roots final)) /$
 $nnreal (card query-sample-space) \leq query-error-bound$
proof –
have *unique:*
 $\exists$ *trace-table composition-table.*
query-header-supported-partial-table-candidates s fr f-fri-roots
f-final as dg composition-fri-roots final $\subseteq$
 $\{(trace-table, composition-table)\}$
by (rule
query-header-supported-partial-table-candidates-subset-singleton-if-no-cross-or-merkle-bad

```

    [OF no-bad])
  show ?thesis
  by (rule
    query-header-supported-partial-union-good-sets-fraction-bound-if-unique-candidate
    [OF unique subset bounded])
qed

lemma query-header-supported-partial-union-good-sets-fraction-bound-if-no-global-cross-or-merkle-bad:
  assumes no-bad:
     $\neg$  query-supported-partial-pairwise-cross-or-merkle-bad s
  and subset:
     $\bigwedge$  trace-table composition-table as.
      good-sets trace-table composition-table as  $\subseteq$  query-sample-space
  and bounded:
     $\bigwedge$  trace-table composition-table as.
      nnreal (card (good-sets trace-table composition-table as)) /
      nnreal (card query-sample-space)  $\leq$  query-error-bound
  shows
    nnreal
      (card
        (query-header-supported-partial-union-good-sets s good-sets fr
          f-fri-roots f-final as dg composition-fri-roots final)) /
      nnreal (card query-sample-space)  $\leq$  query-error-bound
  proof (rule
    query-header-supported-partial-union-good-sets-fraction-bound-if-no-cross-or-merkle-bad
    [OF - subset bounded])
  show
     $\neg$  query-header-supported-partial-pairwise-cross-or-merkle-bad s fr
      f-fri-roots f-final as dg composition-fri-roots final
  using no-bad
  unfolding query-supported-partial-pairwise-cross-or-merkle-bad-def by blast
qed

lemma wp-query-index-round-set-hit-bound-if-no-global-cross-or-merkle-bad:
  fixes H :: prob
  assumes future: query-future-fresh s
  and raw-bound: query-index-raw-preimage-bound
  and no-bad:
     $\neg$  query-supported-partial-pairwise-cross-or-merkle-bad s
  and subset:
     $\bigwedge$  trace-table composition-table as.
      good-sets trace-table composition-table as  $\subseteq$  query-sample-space
  and bounded:
     $\bigwedge$  trace-table composition-table as.
      nnreal (card (good-sets trace-table composition-table as)) /
      nnreal (card query-sample-space)  $\leq$  query-error-bound
  and collision-bound:
    wp-event verify-monad (hash-map-output-collision-bad s) s  $\leq$  H
  shows

```

```

wp-event verify-monad (query-index-round-set-hit s good-sets) s ≤
  nnreal rounds * query-error-bound + H
proof (rule
  wp-query-index-round-set-hit-bound-via-supported-partial-union-or-collision
  [OF future raw-bound - collision-bound])
fix fr f-fri-roots f-final as dg composition-fri-roots final
show
  nnreal
  (card
    (query-header-supported-partial-union-good-sets s good-sets fr
      f-fri-roots f-final as dg composition-fri-roots final)) /
  nnreal (card query-sample-space) ≤ query-error-bound
by (rule
  query-header-supported-partial-union-good-sets-fraction-bound-if-no-global-cross-or-merkle-bad
  [OF no-bad subset bounded])
qed

end

end

```

```

theory Staged-Security-Experiment-Composition-Query-Header-Rounds
imports
  Staged-Security-Experiment-Composition-Query-Prefix-Witness
  Soundness-Query-Header-Cross-Candidate
begin

```

Per-round decomposition for the header-authenticated composition query event.

The global event existentially ranges over the query round and authenticated opening witnesses. This layer peels off the finite round existential. The remaining witness selection problem is deliberately left explicit: later proofs must either choose prefix-fixed witnesses before the query challenge or charge failures to Merkle/collision side events.

```

context soundness
begin

```

```

definition staged-query-prefix-header-supported-query-target
  :: 'f staged-query-prefix-data ⇒ 'f protocol-channel ⇒ nat set
where
  staged-query-prefix-header-supported-query-target prefix prefix-state =
    query-header-supported-partial-union-good-sets prefix-state
    query-sampling-success-space
    (sqp-trace-root prefix)
    (sqp-trace-fri-roots prefix)
    (sqp-trace-final prefix)
    (sqp-alphas prefix)
    (sqp-degree prefix)
    (sqp-composition-fri-roots prefix)
    (sqp-composition-final prefix)

```

lemma *staged-query-prefix-header-supported-query-target-subset:*
staged-query-prefix-header-supported-query-target prefix prefix-state
 $\subseteq$ *query-sample-space*
unfolding *staged-query-prefix-header-supported-query-target-def*
by (rule *query-header-supported-partial-union-good-sets-subset*)

lemma *staged-query-prefix-header-supported-query-target-fraction-bound-if-no-cross-or-merkle-bad:*
assumes *no-bad:*
 $\neg$ *query-supported-partial-pairwise-cross-or-merkle-bad prefix-state*
shows
 $\text{nnreal } (\text{card } (\text{staged-query-prefix-header-supported-query-target prefix prefix-state})) / \text{nnreal } (\text{card } \text{query-sample-space}) \leq \text{query-error-bound}$
unfolding *staged-query-prefix-header-supported-query-target-def*
by (rule *query-header-supported-partial-union-good-sets-fraction-bound-if-no-global-cross-or-merkle-bad*
 $[OF \text{ no-bad } \text{query-sampling-success-space-subset}$
 $\text{query-sampling-success-space-fraction-bound-query-sample-space}]$)

lemma *checked-staged-query-prefix-header-supported-query-target-prehit-bound:*
assumes *wf: staged-budget-wellformed budgets*
and *controlled: staged-adversary-controlled budgets A*
and *i-bound: $i \leq \text{rounds}$*

shows
 $\text{wp-event } (\text{checked-staged-query-prefix-receive-with-state } A \ i)$
 $(\text{checked-staged-query-prefix-dynamic-index-prehit}$
 $\text{staged-query-prefix-header-supported-query-target})$
 $\text{adversary-initial-state} \leq$
 $\text{staged-phase-relation-error size}$
 $(\text{staged-query-search-queries budgets } i + 1)$

proof (rule *checked-staged-query-prefix-dynamic-index-prehit-bound-from-relation-program*)

show $\text{HashMap adversary-initial-state} = \text{fmempty}$
by (simp add: *adversary-initial-state-def*)

show *hash-relation-program*
 $(\text{checked-staged-query-prefix-dynamic-index-prehit-relation } A \ i$
 $\text{adversary-initial-state}$
 $\text{staged-query-prefix-header-supported-query-target})$
 $\text{size } (\text{staged-query-search-queries budgets } i + 1)$
 $(\text{checked-staged-query-prefix-receive-with-state } A \ i)$
by (rule *hash-relation-program-checked-staged-query-prefix-receive-with-state*
 $[OF \text{ wf controlled } i\text{-bound}]$)
(rule
 $\text{checked-staged-query-prefix-dynamic-index-prehit-relation-fiber-bound-size})$

qed

lemma *checked-staged-query-prefix-header-supported-query-target-hit-bound-from-budgets:*

assumes *wf*: staged-budget-wellformed budgets
and *controlled*: staged-adversary-controlled budgets *A*
and *i-bound*: $i < \text{rounds}$
and *no-bad*:
 $\bigwedge \text{prefix prefix-state.}$
 $\text{Some } ((\text{prefix}, \text{prefix-state}), \text{prefix-state}) \in$
 set-dist
 $(\text{execute } (\text{checked-staged-query-prefix-with-state } A \ i)$
 $\text{adversary-initial-state}) \implies$
 $\neg \text{query-supported-partial-pairwise-cross-or-merkle-bad}$
 prefix-state
shows
 $\text{wp-event } (\text{checked-staged-query-prefix-receive-with-state } A \ i)$
 $(\text{checked-staged-query-prefix-dynamic-index-hit}$
 $\text{staged-query-prefix-header-supported-query-target})$
 $\text{adversary-initial-state} \leq$
 $\text{staged-phase-relation-error size}$
 $(\text{staged-query-search-queries budgets } i + 1) + \text{query-error-bound}$
proof –
have *raw-bound*: query-index-raw-preimage-bound
by (rule query-index-raw-preimage-bound-from-sampler-wellformed)
have *hit-bound*:
 $\text{wp-event } (\text{checked-staged-query-prefix-receive-with-state } A \ i)$
 $(\text{checked-staged-query-prefix-dynamic-index-hit}$
 $\text{staged-query-prefix-header-supported-query-target})$
 $\text{adversary-initial-state} \leq$
 $\text{wp-event } (\text{checked-staged-query-prefix-receive-with-state } A \ i)$
 $(\text{checked-staged-query-prefix-dynamic-index-prehit}$
 $\text{staged-query-prefix-header-supported-query-target})$
 $\text{adversary-initial-state} + \text{query-error-bound}$
proof (rule
checked-staged-query-prefix-dynamic-index-hit-bound-by-prehit-and-query-error
[OF raw-bound])
fix *prefix prefix-state*
assume *support*:
 $\text{Some } ((\text{prefix}, \text{prefix-state}), \text{prefix-state}) \in$
 set-dist
 $(\text{execute } (\text{checked-staged-query-prefix-with-state } A \ i)$
 $\text{adversary-initial-state})$
show
 $\text{staged-query-prefix-header-supported-query-target prefix prefix-state}$
 $\subseteq \text{query-sample-space} \wedge$
 nnreal
 $(\text{card}$
 $(\text{staged-query-prefix-header-supported-query-target prefix}$
 $\text{prefix-state})) /$
 $\text{nnreal } (\text{card query-sample-space}) \leq \text{query-error-bound}$
using
staged-query-prefix-header-supported-query-target-subset

```

    [of prefix prefix-state]
    staged-query-prefix-header-supported-query-target-fraction-bound-if-no-cross-or-merkle-bad
    [of prefix-state prefix, OF no-bad[OF support]]
  by blast
qed
have prehit-bound:
  wp-event (checked-staged-query-prefix-recv-with-state A i)
    (checked-staged-query-prefix-dynamic-index-prehit
     staged-query-prefix-header-supported-query-target)
  adversary-initial-state ≤
  staged-phase-relation-error size
  (staged-query-search-queries budgets i + 1)
by (rule
   checked-staged-query-prefix-header-supported-query-target-prehit-bound
   [OF wf controlled])
  (use i-bound in simp)
show ?thesis
by (rule order-trans[OF hit-bound])
  (intro add-mono prehit-bound order-refl)
qed

definition checked-staged-security-with-actual-alpha-prefix-header-authenticated-candidate-opening-query-hit-from-pr
:: nat ⇒ 'f staged-adversary ⇒
  ((((((('f × 'f list × 'f list × 'f) ×
    'f protocol-channel) × 'f staged-proof-data) ×
    'f protocol-channel) × unit list) ×
    'f protocol-channel) option ⇒ bool

where
  checked-staged-security-with-actual-alpha-prefix-header-authenticated-candidate-opening-query-hit-from-prefix-at
  i A out ⟷
  (case out of
   None ⇒ False
  | Some (((((alpha-prefix, alpha-prefix-state), data), attacker-state),
    result), final-state) ⇒
    (let s =
      verifier-state-from-adversary attacker-state
      (staged-proof-transcript data)
    in ∃ trace-openings composition-openings.
      (trace-openings, composition-openings) ∈
      query-header-supported-partial-opening-witnesses s
      (staged-trace-root data)
      (staged-trace-fri-roots data)
      (staged-trace-final data)
      (staged-alphas data)
      (staged-degree data)
      (staged-composition-fri-roots data)
      (staged-composition-final data) ∧
      checked-staged-security-with-actual-alpha-prefix-candidate-opening-query-hit-from-prefix
      A

```

$((\text{replicate rounds } [] [i := \text{trace-openings}])$
 $((\text{replicate rounds } [] [i := \text{composition-openings}])$
 i
 $(\text{Some } (((((\alpha\text{-prefix}, \alpha\text{-prefix-state}), \text{data}),$
 $\text{attacker-state}), \text{result}), \text{final-state}))))$

lemma *checked-staged-security-with-actual-alpha-prefix-header-authenticated-candidate-opening-query-hit-from-prefix*

assumes *witness:*

$(\text{trace-openings}, \text{composition-openings}) \in$
 $\text{query-header-supported-partial-opening-witnesses}$
 $(\text{verifier-state-from-adversary } \text{attacker-state}$
 $(\text{staged-proof-transcript } \text{data}))$
 $(\text{staged-trace-root } \text{data})$
 $(\text{staged-trace-fri-roots } \text{data})$
 $(\text{staged-trace-final } \text{data})$
 $(\text{staged-alphas } \text{data})$
 $(\text{staged-degree } \text{data})$
 $(\text{staged-composition-fri-roots } \text{data})$
 $(\text{staged-composition-final } \text{data})$

and *component:*

$\text{checked-staged-security-with-actual-alpha-prefix-candidate-opening-query-hit-from-prefix } A$
 $((\text{replicate rounds } [] [i := \text{trace-openings}])$
 $((\text{replicate rounds } [] [i := \text{composition-openings}])$
 i
 $(\text{Some } (((((\alpha\text{-prefix}, \alpha\text{-prefix-state}), \text{data}), \text{attacker-state}),$
 $\text{result}), \text{final-state}))))$

shows

$\text{checked-staged-security-with-actual-alpha-prefix-header-authenticated-candidate-opening-query-hit-from-prefix-at}$
 $i A$
 $(\text{Some } (((((\alpha\text{-prefix}, \alpha\text{-prefix-state}), \text{data}), \text{attacker-state}),$
 $\text{result}), \text{final-state}))))$

unfolding

$\text{checked-staged-security-with-actual-alpha-prefix-header-authenticated-candidate-opening-query-hit-from-prefix-at-de}$
 Let-def

using *witness component*

by $(\text{auto split: option.splits prod.splits})$

lemma *checked-staged-security-with-actual-alpha-prefix-header-authenticated-candidate-opening-query-hit-from-prefix*

assumes *hit:*

$\text{checked-staged-security-with-actual-alpha-prefix-header-authenticated-candidate-opening-query-hit-from-prefix}$
 $A \text{ out}$

shows

$\exists i < \text{rounds}.$

$\text{checked-staged-security-with-actual-alpha-prefix-header-authenticated-candidate-opening-query-hit-from-prefix-at}$
 $i A \text{ out}$

proof (cases out)

case *None*

then show *?thesis*

using *hit*


```

unfolding
  checked-staged-security-with-actual-alpha-prefix-header-authenticated-candidate-opening-query-hit-from-prefix-def
by simp
next
case (Some packed)
then obtain alpha-prefix alpha-prefix-state data attacker-state result
  final-state where out-eq:
  out =
    Some (((((alpha-prefix, alpha-prefix-state), data), attacker-state),
      result), final-state)
  by (cases packed) (auto split: prod.splits)
from checked-staged-security-with-actual-alpha-prefix-header-authenticated-candidate-opening-query-hit-from-prefix-def
  [OF hit[unfolded out-eq]]
obtain i trace-openings composition-openings where witness:
  (trace-openings, composition-openings) ∈
    query-header-supported-partial-opening-witnesses
    (verifier-state-from-adversary attacker-state
      (staged-proof-transcript data))
    (staged-trace-root data)
    (staged-trace-fri-roots data)
    (staged-trace-final data)
    (staged-alphas data)
    (staged-degree data)
    (staged-composition-fri-roots data)
    (staged-composition-final data)
and component:
  checked-staged-security-with-actual-alpha-prefix-candidate-opening-query-hit-from-prefix A
  ((replicate rounds []) [i := trace-openings])
  ((replicate rounds []) [i := composition-openings])
  i
  (Some (((((alpha-prefix, alpha-prefix-state), data), attacker-state),
    result), final-state))
by blast
have i-bound: i < rounds
using component
unfolding
  checked-staged-security-with-actual-alpha-prefix-candidate-opening-query-hit-from-prefix-def
by (simp split: option.splits prod.splits)
have round-hit:
  checked-staged-security-with-actual-alpha-prefix-header-authenticated-candidate-opening-query-hit-from-prefix-at
  i A out
unfolding out-eq
by (rule
  checked-staged-security-with-actual-alpha-prefix-header-authenticated-candidate-opening-query-hit-from-prefix-at
  [OF witness component])
show ?thesis
using i-bound round-hit by blast
qed

```

lemma *checked-staged-security-with-actual-alpha-prefix-header-authenticated-candidate-opening-query-hit-from-prefix*
assumes *hit*:
checked-staged-security-with-actual-alpha-prefix-header-authenticated-candidate-opening-query-hit-from-prefix-at
i A
(Some (((((alpha-prefix, alpha-prefix-state), data), attacker-state),
result), final-state)))
obtains *trace-openings composition-openings* **where**
(trace-openings, composition-openings) ∈
query-header-supported-partial-opening-witnesses
(verifier-state-from-adversary attacker-state
(staged-proof-transcript data))
(staged-trace-root data)
(staged-trace-fri-roots data)
(staged-trace-final data)
(staged-alphas data)
(staged-degree data)
(staged-composition-fri-roots data)
(staged-composition-final data)
checked-staged-security-with-actual-alpha-prefix-candidate-opening-query-hit-from-prefix
A
((replicate rounds []) [i := trace-openings])
((replicate rounds []) [i := composition-openings])
i
(Some (((((alpha-prefix, alpha-prefix-state), data), attacker-state),
result), final-state)))
using *hit*
unfolding
checked-staged-security-with-actual-alpha-prefix-header-authenticated-candidate-opening-query-hit-from-prefix-at-de
Let-def
by (*auto split: option.splits prod.splits*)

lemma *checked-staged-security-with-actual-alpha-prefix-header-authenticated-candidate-opening-query-hit-from-prefix*
assumes *hit*:
checked-staged-security-with-actual-alpha-prefix-header-authenticated-candidate-opening-query-hit-from-prefix-at
i A
(Some (((((alpha-prefix, alpha-prefix-state), data), attacker-state),
result), final-state)))
and *i-bound: i < rounds*
shows
(∃ result' final-state'.
partial-merkle-inconsistency-bad
(verifier-state-from-adversary attacker-state
(staged-proof-transcript data))
(Some (result', final-state')))) ∨
(∃ trace-openings composition-openings trace-table composition-table.
(trace-openings, composition-openings) ∈
query-header-supported-partial-opening-witnesses
(verifier-state-from-adversary attacker-state
(staged-proof-transcript data))

```

    (staged-trace-root data)
    (staged-trace-fri-roots data)
    (staged-trace-final data)
    (staged-alphas data)
    (staged-degree data)
    (staged-composition-fri-roots data)
    (staged-composition-final data) ∧
checked-staged-security-with-actual-alpha-prefix-candidate-opening-query-hit-from-prefix
A
((replicate rounds []) [i := trace-openings])
((replicate rounds []) [i := composition-openings])
i
(Some (((alpha-prefix, alpha-prefix-state), data), attacker-state),
  result), final-state)) ∧
partial-trace-table-candidate trace-table
((replicate rounds []) [i := trace-openings]) ∧
partial-composition-table-candidate composition-table
((replicate rounds []) [i := composition-openings]))
proof –
  let ?s =
    verifier-state-from-adversary attacker-state
    (staged-proof-transcript data)
  from
    checked-staged-security-with-actual-alpha-prefix-header-authenticated-candidate-opening-query-hit-from-prefix-atE
    [OF hit]
  obtain trace-openings composition-openings where witness:
    (trace-openings, composition-openings) ∈
    query-header-supported-partial-opening-witnesses ?s
    (staged-trace-root data)
    (staged-trace-fri-roots data)
    (staged-trace-final data)
    (staged-alphas data)
    (staged-degree data)
    (staged-composition-fri-roots data)
    (staged-composition-final data)
  and component:
    checked-staged-security-with-actual-alpha-prefix-candidate-opening-query-hit-from-prefix
    A
    ((replicate rounds []) [i := trace-openings])
    ((replicate rounds []) [i := composition-openings])
    i
    (Some (((alpha-prefix, alpha-prefix-state), data), attacker-state),
      result), final-state))
  by blast
from query-header-supported-partial-opening-witness-single-round-candidates-or-partial-merkle-bad
  [OF witness i-bound]
show ?thesis
  using witness component by blast
qed

```

lemma *checked-staged-security-with-actual-alpha-prefix-header-authenticated-candidate-opening-query-hit-from-prefix*

assumes *hit*:

checked-staged-security-with-actual-alpha-prefix-header-authenticated-candidate-opening-query-hit-from-prefix-at

i A

(*Some* ((((((*alpha-prefix*, *alpha-prefix-state*), *data*), *attacker-state*),
result), *final-state*)))

and *i-bound*: $i < \text{rounds}$

and *trace-low*:

$\bigwedge \text{trace-openings composition-openings trace-table}$
composition-table.

(*trace-openings*, *composition-openings*) $\in$
query-header-supported-partial-opening-witnesses
(*verifier-state-from-adversary attacker-state*
(*staged-proof-transcript data*))
(*staged-trace-root data*)
(*staged-trace-fri-roots data*)
(*staged-trace-final data*)
(*staged-alphas data*)
(*staged-degree data*)
(*staged-composition-fri-roots data*)
(*staged-composition-final data*) $\implies$

partial-trace-table-candidate trace-table

((*replicate rounds []*) [*i := trace-openings*]) $\implies$

partial-composition-table-candidate composition-table

((*replicate rounds []*) [*i := composition-openings*]) $\implies$

trace-table-low-degree trace-table

and *comp-low*:

$\bigwedge \text{trace-openings composition-openings trace-table}$
composition-table.

(*trace-openings*, *composition-openings*) $\in$
query-header-supported-partial-opening-witnesses
(*verifier-state-from-adversary attacker-state*
(*staged-proof-transcript data*))
(*staged-trace-root data*)
(*staged-trace-fri-roots data*)
(*staged-trace-final data*)
(*staged-alphas data*)
(*staged-degree data*)
(*staged-composition-fri-roots data*)
(*staged-composition-final data*) $\implies$

partial-trace-table-candidate trace-table

((*replicate rounds []*) [*i := trace-openings*]) $\implies$

partial-composition-table-candidate composition-table

((*replicate rounds []*) [*i := composition-openings*]) $\implies$

composition-table-low-degree maxDegree composition-table

and *not-all*:

$\bigwedge \text{trace-openings composition-openings trace-table}$
composition-table prefix prefix-state.

```

(trace-openings, composition-openings) ∈
  query-header-supported-partial-opening-witnesses
    (verifier-state-from-adversary attacker-state
      (staged-proof-transcript data))
    (staged-trace-root data)
    (staged-trace-fri-roots data)
    (staged-trace-final data)
    (staged-alphas data)
    (staged-degree data)
    (staged-composition-fri-roots data)
    (staged-composition-final data) ⇒
partial-trace-table-candidate trace-table
  ((replicate rounds []) [i := trace-openings]) ⇒
partial-composition-table-candidate composition-table
  ((replicate rounds []) [i := composition-openings]) ⇒
Some ((prefix, prefix-state), prefix-state) ∈
  set-dist
    (execute (checked-staged-query-prefix-with-state A i)
      adversary-initial-state) ⇒
¬ all-queries-consistent trace-table composition-table
  (sqp-alphas prefix)
shows
  (∃ result' final-state'.
    partial-merkle-inconsistency-bad
      (verifier-state-from-adversary attacker-state
        (staged-proof-transcript data))
      (Some (result', final-state'))) ∨
  (∃ trace-table composition-table.
    checked-staged-security-with-actual-alpha-prefix-candidate-pair-query-hit-from-prefix
      A trace-table composition-table i
      (Some (((alpha-prefix, alpha-prefix-state), data),
        attacker-state), result), final-state)))
proof –
  let ?s =
    verifier-state-from-adversary attacker-state
      (staged-proof-transcript data)
  from
    checked-staged-security-with-actual-alpha-prefix-header-authenticated-candidate-opening-query-hit-from-prefix-at-ca
      [OF hit i-bound]
  show ?thesis
  proof
    assume
      ∃ result' final-state'.
        partial-merkle-inconsistency-bad ?s (Some (result', final-state'))
    then show ?thesis by simp
  next
    assume candidates:
      ∃ trace-openings composition-openings trace-table
        composition-table.

```

```

(trace-openings, composition-openings) ∈
  query-header-supported-partial-opening-witnesses ?s
    (staged-trace-root data)
    (staged-trace-fri-roots data)
    (staged-trace-final data)
    (staged-alphas data)
    (staged-degree data)
    (staged-composition-fri-roots data)
    (staged-composition-final data) ∧
checked-staged-security-with-actual-alpha-prefix-candidate-opening-query-hit-from-prefix
A
((replicate rounds []) [i := trace-openings])
((replicate rounds []) [i := composition-openings])
i
(Some (((alpha-prefix, alpha-prefix-state), data),
  attacker-state), result), final-state)) ∧
partial-trace-table-candidate trace-table
((replicate rounds []) [i := trace-openings]) ∧
partial-composition-table-candidate composition-table
((replicate rounds []) [i := composition-openings])
then obtain trace-openings composition-openings trace-table
composition-table where witness:
(trace-openings, composition-openings) ∈
  query-header-supported-partial-opening-witnesses ?s
    (staged-trace-root data)
    (staged-trace-fri-roots data)
    (staged-trace-final data)
    (staged-alphas data)
    (staged-degree data)
    (staged-composition-fri-roots data)
    (staged-composition-final data)
and opening-hit:
checked-staged-security-with-actual-alpha-prefix-candidate-opening-query-hit-from-prefix
A
((replicate rounds []) [i := trace-openings])
((replicate rounds []) [i := composition-openings])
i
(Some (((alpha-prefix, alpha-prefix-state), data),
  attacker-state), result), final-state))
and trace-candidate:
partial-trace-table-candidate trace-table
((replicate rounds []) [i := trace-openings])
and comp-candidate:
partial-composition-table-candidate composition-table
((replicate rounds []) [i := composition-openings])
by blast
have pair-hit:
checked-staged-security-with-actual-alpha-prefix-candidate-pair-query-hit-from-prefix
A trace-table composition-table i

```

```

    (Some (((alpha-prefix, alpha-prefix-state), data),
      attacker-state), result), final-state))
  by (rule
    checked-staged-security-with-actual-alpha-prefix-candidate-opening-query-hit-from-prefix-imp-candidate-pair-que
    [OF opening-hit trace-candidate comp-candidate
      trace-low[OF witness trace-candidate comp-candidate]
      comp-low[OF witness trace-candidate comp-candidate]])
    (rule not-all[OF witness trace-candidate comp-candidate])
  then show ?thesis by blast
qed
qed

lemma checked-staged-security-with-actual-alpha-prefix-header-authenticated-candidate-opening-query-hit-from-prefix-imp-candidate-pair-que
  fixes Q :: nat  $\Rightarrow$  prob
  assumes round-bound:
     $\bigwedge i. i < \text{rounds} \implies$ 
      wp-event
        (checked-staged-security-experiment-with-actual-alpha-prefix-data-state A)
        (checked-staged-security-with-actual-alpha-prefix-header-authenticated-candidate-opening-query-hit-from-prefix-imp-candidate-pair-que
          i A)
        adversary-initial-state  $\leq$  Q i
  shows
    wp-event
      (checked-staged-security-experiment-with-actual-alpha-prefix-data-state A)
      (checked-staged-security-with-actual-alpha-prefix-header-authenticated-candidate-opening-query-hit-from-prefix-imp-candidate-pair-que
        A)
      adversary-initial-state  $\leq$  ( $\sum i < \text{rounds}. Q i$ )
proof -
  have event-le:
    wp-event
      (checked-staged-security-experiment-with-actual-alpha-prefix-data-state A)
      (checked-staged-security-with-actual-alpha-prefix-header-authenticated-candidate-opening-query-hit-from-prefix-imp-candidate-pair-que
        A)
      adversary-initial-state  $\leq$ 
    wp-event
      (checked-staged-security-experiment-with-actual-alpha-prefix-data-state A)
      ( $\lambda \text{out}. \exists i \in \{.. < \text{rounds}\}. \text{checked-staged-security-with-actual-alpha-prefix-header-authenticated-candidate-opening-query-hit-from-prefix-imp-candidate-pair-que } i A \text{ out}$ )
      adversary-initial-state
  by (rule wp-event-mono)
  (use
    checked-staged-security-with-actual-alpha-prefix-header-authenticated-candidate-opening-query-hit-from-prefix-imp-candidate-pair-que
    in blast)
  have union-bound:
    wp-event
      (checked-staged-security-experiment-with-actual-alpha-prefix-data-state A)
      ( $\lambda \text{out}. \exists i \in \{.. < \text{rounds}\}. \text{checked-staged-security-with-actual-alpha-prefix-header-authenticated-candidate-opening-query-hit-from-prefix-imp-candidate-pair-que } i A \text{ out}$ )
      adversary-initial-state

```

```

      i A out)
    adversary-initial-state ≤
    (∑ i<rounds. Q i)
  by (rule wp-event-finite-UN-bound) (simp-all add: round-bound)
show ?thesis
  by (rule order-trans[OF event-le union-bound])
qed

lemma checked-staged-security-with-data-state-composition-bad-bound-from-header-authenticated-query-from-prefix-rou
fixes Q :: nat ⇒ prob
assumes wf: staged-budget-wellformed budgets
and controlled: staged-adversary-controlled budgets A
and round-bound:
  ∧ i. i < rounds ⇒
    wp-event
      (checked-staged-security-experiment-with-actual-alpha-prefix-data-state
        A)
      (checked-staged-security-with-actual-alpha-prefix-header-authenticated-candidate-opening-query-hit-from-prefix-rou
        i A)
      adversary-initial-state ≤ Q i
shows
  wp-event (checked-staged-security-experiment-with-data-state A)
    (staged-security-with-data-state-verifier-event composition-bad)
    adversary-initial-state ≤
    composition-error-bound +
    staged-phase-relation-error size
    (staged-alpha-search-queries budgets 0) +
    hash-collision-budget-value 0
    (staged-alpha-search-queries budgets 0) +
    staged-phase-relation-error size
    (staged-alpha-search-queries budgets 0) +
    (∑ i<rounds. Q i) +
    hash-collision-budget-value 0
    (staged-attacker-query-budget budgets +
      staged-challenge-query-budget + verifier-hash-query-budget)
proof -
  have header-bound:
    wp-event
      (checked-staged-security-experiment-with-actual-alpha-prefix-data-state
        A)
      (checked-staged-security-with-actual-alpha-prefix-header-authenticated-candidate-opening-query-hit-from-prefix-rou
        A)
      adversary-initial-state ≤ (∑ i<rounds. Q i)
  by (rule
    checked-staged-security-with-actual-alpha-prefix-header-authenticated-candidate-opening-query-hit-from-prefix-rou
    [OF round-bound])
show ?thesis
  by (rule
    checked-staged-security-with-data-state-composition-bad-bound-from-header-authenticated-query-from-prefix-and-rou

```


[*OF wf controlled header-bound*])
qed

end

end

theory *Staged-Security-Experiment-Composition-Query-Classification*
imports *Staged-Security-Experiment-Composition-Query-Prefix-Witness*
begin

Local classification lemmas for current-final-state partial-opening evidence. These lemmas deliberately live downstream of the fixed-witness bridge to avoid growing the size-sensitive prefix-witness theory.

context *soundness*
begin

lemma *trace-fri-bad-with-updated-partial-candidatesI*:
assumes *updated*:
 accepted-with-partial-initial-openings s
 (*Some (result, final-state)*) *fr f-fri-roots f-final as dg*
 composition-fri-roots final trace-query-idxs trace-openings
 composition-query-idxs composition-openings
and *trace-candidate*:
 partial-trace-table-candidate trace-table trace-openings
and *not-low*: $\neg$ *trace-table-low-degree trace-table*
shows *trace-fri-bad-with-partial-candidates s*
 (*Some (result, final-state)*)
using *assms*
unfolding *trace-fri-bad-with-partial-candidates-def* **by** *blast*

lemma *composition-fri-bad-with-updated-partial-candidatesI*:
assumes *updated*:
 accepted-with-partial-initial-openings s
 (*Some (result, final-state)*) *fr f-fri-roots f-final as dg*
 composition-fri-roots final trace-query-idxs trace-openings
 composition-query-idxs composition-openings
and *trace-candidate*:
 partial-trace-table-candidate trace-table trace-openings
and *comp-candidate*:
 partial-composition-table-candidate composition-table
 composition-openings
and *trace-low*: *trace-table-low-degree trace-table*
and *not-comp-low*:
 $\neg$ *composition-table-low-degree maxDegree composition-table*
shows *composition-fri-bad-with-partial-candidates s*
 (*Some (result, final-state)*)
using *assms*

unfolding *composition-fri-bad-with-partial-candidates-def* **by** *blast*

lemma *query-bad-with-updated-partial-candidatesI:*

assumes *updated:*

accepted-with-partial-initial-openings s

(Some (result, final-state)) fr f-fri-roots f-final as dg

composition-fri-roots final trace-query-idxs trace-openings

composition-query-idxs composition-openings

and *trace-candidate:*

partial-trace-table-candidate trace-table trace-openings

and *comp-candidate:*

partial-composition-table-candidate composition-table

composition-openings

and *trace-low: trace-table-low-degree trace-table*

and *comp-low:*

composition-table-low-degree maxDegree composition-table

and *not-all: $\neg$ all-queries-consistent trace-table composition-table as*

shows *query-bad-with-partial-candidates s (Some (result, final-state))*

using *assms*

unfolding *query-bad-with-partial-candidates-def* **by** *blast*

lemma *accepted-with-partial-initial-openings-update-round-candidate-classification:*

assumes *partial:*

accepted-with-partial-initial-openings s

(Some (result, final-state)) fr f-fri-roots f-final as dg

composition-fri-roots final trace-query-idxs trace-openings

composition-query-idxs composition-openings

and *no-bad:*

$\neg$ partial-merkle-inconsistency-bad s (Some (result, final-state))

and *i-bound: $i < \text{rounds}$*

and *same-idx: trace-query-idxs ! i = composition-query-idxs ! i*

and *consistent:*

partial-query-openings-consistent trace-openings-i

composition-openings-i as (trace-query-idxs ! i)

and *trace-auth:*

*partial-authenticated-table fr (scale * clength) trace-openings-i*

final-state

and *comp-auth:*

partial-authenticated-table (hd composition-fri-roots)

*(scale * clength) composition-openings-i final-state*

shows

trace-fri-bad-with-partial-candidates s (Some (result, final-state)) $\vee$

composition-fri-bad-with-partial-candidates s

(Some (result, final-state)) $\vee$

query-bad-with-partial-candidates s (Some (result, final-state)) $\vee$

($\exists$ trace-table composition-table.

partial-trace-table-candidate trace-table

(trace-openings[i := trace-openings-i]) $\wedge$

partial-composition-table-candidate composition-table

```

      (composition-openings[i := composition-openings-i]) ∧
      trace-table-low-degree trace-table ∧
      composition-table-low-degree maxDegree composition-table ∧
      all-queries-consistent trace-table composition-table as)
proof –
  let ?trace-openings = trace-openings[i := trace-openings-i]
  let ?composition-openings =
    composition-openings[i := composition-openings-i]
  have updated:
    accepted-with-partial-initial-openings s
    (Some (result, final-state)) fr f-fri-roots f-final as dg
    composition-fri-roots final trace-query-idxs ?trace-openings
    composition-query-idxs ?composition-openings
  by (rule accepted-with-partial-initial-openings-update-round
    [OF partial i-bound same-idx consistent trace-auth comp-auth])
  obtain trace-table composition-table where trace-candidate:
    partial-trace-table-candidate trace-table ?trace-openings
  and comp-candidate:
    partial-composition-table-candidate composition-table
    ?composition-openings
  using accepted-with-partial-initial-openings-candidates-if-no-partial-merkle-bad
    [OF updated no-bad]
  by blast
  show ?thesis
  proof (cases trace-table-low-degree trace-table)
  case False
  then have trace-fri-bad-with-partial-candidates s
    (Some (result, final-state))
  by (rule trace-fri-bad-with-updated-partial-candidatesI
    [OF updated trace-candidate])
  then show ?thesis by simp
  next
  case trace-low: True
  show ?thesis
  proof (cases composition-table-low-degree maxDegree composition-table)
  case False
  then have composition-fri-bad-with-partial-candidates s
    (Some (result, final-state))
  by (rule composition-fri-bad-with-updated-partial-candidatesI
    [OF updated trace-candidate comp-candidate trace-low])
  then show ?thesis by simp
  next
  case comp-low: True
  show ?thesis
  proof (cases all-queries-consistent trace-table composition-table as)
  case False
  then have query-bad-with-partial-candidates s
    (Some (result, final-state))
  by (rule query-bad-with-updated-partial-candidatesI

```

```

      [OF updated trace-candidate comp-candidate trace-low comp-low])
    then show ?thesis by simp
  next
    case True
    then show ?thesis
      using trace-candidate comp-candidate trace-low comp-low by blast
  qed
qed
qed
qed

```

lemma *accepted-with-partial-initial-openings-aligned-consistent-update-round-candidate-classification:*

assumes *partial:*

accepted-with-partial-initial-openings-aligned-consistent s
(Some (result, final-state)) fr f-fri-roots f-final as dg
composition-fri-roots final query-idxs trace-openings
composition-openings

and *no-bad:*

$\neg$ *partial-merkle-inconsistency-bad s (Some (result, final-state))*

and *i-bound:* $i < \text{rounds}$

and *consistent:*

partial-query-openings-consistent trace-openings-i
composition-openings-i as (query-idxs ! i)

and *trace-auth:*

*partial-authenticated-table fr (scale * clength) trace-openings-i*
final-state

and *comp-auth:*

partial-authenticated-table (hd composition-fri-roots)
*(scale * clength) composition-openings-i final-state*

shows

trace-fri-bad-with-partial-candidates s (Some (result, final-state)) $\vee$
composition-fri-bad-with-partial-candidates s
(Some (result, final-state)) $\vee$
query-bad-with-partial-candidates s (Some (result, final-state)) $\vee$
 $(\exists \text{ trace-table composition-table.}$
partial-trace-table-candidate trace-table
 $(\text{trace-openings}[i := \text{trace-openings-}i]) \wedge$
partial-composition-table-candidate composition-table
 $(\text{composition-openings}[i := \text{composition-openings-}i]) \wedge$
trace-table-low-degree trace-table $\wedge$
composition-table-low-degree maxDegree composition-table $\wedge$
all-queries-consistent trace-table composition-table as)

proof —

have *aligned:*

accepted-with-partial-initial-openings-aligned s
(Some (result, final-state)) fr f-fri-roots f-final as dg
composition-fri-roots final query-idxs trace-openings
composition-openings

using *partial*

```

unfolding accepted-with-partial-initial-openings-aligned-consistent-def
by simp
have unaligned:
  accepted-with-partial-initial-openings s
  (Some (result, final-state)) fr f-fri-roots f-final as dg
  composition-fri-roots final query-idxs trace-openings query-idxs
  composition-openings
by (rule accepted-with-partial-initial-openings-aligned-imp-unaligned
  [OF aligned])
show ?thesis
by (rule
  accepted-with-partial-initial-openings-update-round-candidate-classification
  [OF unaligned no-bad i-bound - consistent trace-auth comp-auth])
  simp
qed

lemma accepted-with-partial-initial-openings-aligned-transcript-consistent-update-round-candidate-classification:
assumes partial:
  accepted-with-partial-initial-openings-aligned-transcript-consistent s
  (Some (result, final-state)) fr f-fri-roots f-final as dg
  composition-fri-roots final query-idxs trace-openings
  composition-openings
and no-bad:
   $\neg$  partial-merkle-inconsistency-bad s (Some (result, final-state))
and i-bound: i < rounds
and consistent:
  partial-query-openings-consistent trace-openings-i
  composition-openings-i as (query-idxs ! i)
and trace-auth:
  partial-authenticated-table fr (scale * clength) trace-openings-i
  final-state
and comp-auth:
  partial-authenticated-table (hd composition-fri-roots)
  (scale * clength) composition-openings-i final-state
shows
  trace-fri-bad-with-partial-candidates s (Some (result, final-state))  $\vee$ 
  composition-fri-bad-with-partial-candidates s
  (Some (result, final-state))  $\vee$ 
  query-bad-with-partial-candidates s (Some (result, final-state))  $\vee$ 
  ( $\exists$  trace-table composition-table.
  partial-trace-table-candidate trace-table
  (trace-openings[i := trace-openings-i])  $\wedge$ 
  partial-composition-table-candidate composition-table
  (composition-openings[i := composition-openings-i])  $\wedge$ 
  trace-table-low-degree trace-table  $\wedge$ 
  composition-table-low-degree maxDegree composition-table  $\wedge$ 
  all-queries-consistent trace-table composition-table as)
proof —
have aligned:

```

```

    accepted-with-partial-initial-openings-aligned-consistent s
      (Some (result, final-state)) fr f-fri-roots f-final as dg
      composition-fri-roots final query-idxs trace-openings
      composition-openings
  by (rule
    accepted-with-partial-initial-openings-aligned-transcript-consistent-imp-aligned
    [OF partial])
  show ?thesis
  by (rule
    accepted-with-partial-initial-openings-aligned-consistent-update-round-candidate-classification
    [OF aligned no-bad i-bound consistent trace-auth comp-auth])
qed

end

end

```

```

theory Staged-Security-Experiment-Composition-Query-Current
  imports Staged-Security-Experiment-Composition-Query-Classification
begin

```

Current-execution projection for the composition query branch.

The broader header-supported query witnesses are existential over possible verifier executions. This layer only reasons about openings authenticated in the actual sampled verifier final state, so the sampled query lookup is available and the event projects to the existing data-state partial-opening query event.

```

context soundness
begin

```

```

lemma checked-staged-security-experiment-with-query-prefix-data-state-raw-final-lookup:

```

```

  assumes wf: staged-budget-wellformed budgets
  and controlled: staged-adversary-controlled budgets A
  and i-bound: i < rounds
  and outcome:
    Some (((((prefix, prefix-state), raw), raw-state),
      ((data, attacker-state), result)), final-state) ∈
    set-dist
    (execute
      (checked-staged-security-experiment-with-query-prefix-data-state A i)
      adversary-initial-state)

```

```

  shows
    fmlookup (HashMap final-state)
      (QueryIndexChallenge i
        (state-after-query-chunks
          (staged-query-start-hash data) (staged-query-chunks data) i)) =
    Some raw

```

```

proof –
  have head:

```

```

Some (((prefix, prefix-state), raw), raw-state) ∈
  set-dist
    (execute (checked-staged-query-prefix-receive-with-state A i)
      adversary-initial-state)
and cont:
Some (((data, attacker-state), result), final-state) ∈
  set-dist
    (execute
      (checked-staged-after-query-prefix-receive-with-verifier A i
        ((prefix, prefix-state), raw))
      raw-state)
using outcome
unfolding checked-staged-security-experiment-with-query-prefix-data-state-def
by (auto elim!: set-dist-bindE)
have recv:
Some (raw, raw-state) ∈
  set-dist (execute receive-query-index-challenge prefix-state)
by (rule checked-staged-query-prefix-receive-with-state-receive-support
  [OF head])
have raw-lookup:
fmlookup (HashMap raw-state)
  (QueryIndexChallenge (PQueryCounter prefix-state) (PState prefix-state)) =
  Some raw
using receive-query-index-challenge-outcome[OF recv] by blast
have data-out:
Some (data, attacker-state) ∈
  set-dist
    (execute
      (checked-staged-after-query-prefix-receive A i
        ((prefix, prefix-state), raw))
      raw-state)
by (rule
  checked-staged-after-query-prefix-receive-with-verifier-outcome(1)
  [OF cont])
have verifier-out:
Some (result, final-state) ∈
  set-dist
    (execute verify-monad
      (verifier-state-from-adversary attacker-state
        (staged-proof-transcript data)))
by (rule
  checked-staged-after-query-prefix-receive-with-verifier-outcome(2)
  [OF cont])
have raw-ext-attacker: raw-state ≤ attacker-state
by (rule checked-staged-after-query-prefix-receive-extends
  [OF wf controlled i-bound data-out])
have attacker-ext-verifier:
attacker-state ≤
  verifier-state-from-adversary attacker-state

```

```

    (staged-proof-transcript data)
proof –
  have attacker-state ≤ attacker-state
    unfolding less-eq-hash-ext-def less-eq-fmap-def by simp
  then show ?thesis
    by (rule hash-extends-verifier-state-from-adversary-right)
qed
have verifier-ext-final:
  verifier-state-from-adversary attacker-state
    (staged-proof-transcript data) ≤
  final-state
  by (rule verify-monad-hash-extends[OF verifier-out])
have raw-ext-final: raw-state ≤ final-state
  by (rule hash-ext-trans[OF raw-ext-attacker])
    (rule hash-ext-trans[OF attacker-ext-verifier verifier-ext-final])
have prefix-support:
  Some ((prefix, prefix-state), prefix-state) ∈
    set-dist
    (execute (checked-staged-query-prefix-with-state A i)
      adversary-initial-state)
  by (rule checked-staged-query-prefix-receive-with-state-prefix-support
    [OF head])
have prefix-alignment:
  PQueryCounter prefix-state = i ∧
  PState prefix-state =
    state-after-query-chunks
    (staged-query-prefix-start-hash prefix)
    (sqp-query-chunks prefix) i ∧
  length (sqp-query-chunks prefix) = i
  by (rule checked-staged-query-prefix-with-state-alignment
    [OF wf controlled i-bound prefix-support])
from cont obtain chunk chunk-state assert-state record-state suffix-chunks
where data-eq:
  data =
    (staged-trace-root = sqp-trace-root prefix,
      staged-trace-fri-roots = sqp-trace-fri-roots prefix,
      staged-trace-fri-challenges = sqp-trace-fri-challenges prefix,
      staged-trace-final = sqp-trace-final prefix,
      staged-alphas = sqp-alphas prefix,
      staged-degree = sqp-degree prefix,
      staged-composition-fri-roots = sqp-composition-fri-roots prefix,
      staged-composition-fri-challenges =
        sqp-composition-fri-challenges prefix,
      staged-composition-final = sqp-composition-final prefix,
      staged-query-chunks = sqp-query-chunks prefix @ chunk # suffix-chunks)
  unfolding checked-staged-after-query-prefix-receive-with-verifier-def
    checked-staged-after-query-prefix-receive-def
  by (auto elim!: set-dist-bindE split: prod.splits)
have start-hash-eq:

```



```

    staged-query-start-hash data = staged-query-prefix-start-hash prefix
unfolding data-eq staged-query-start-hash-def
    staged-query-prefix-start-hash-def
    staged-composition-fri-start-hash-def
    staged-trace-fri-start-hash-def
by simp
have chunks-state-eq:
    state-after-query-chunks
    (staged-query-start-hash data) (staged-query-chunks data) i =
    state-after-query-chunks
    (staged-query-prefix-start-hash prefix) (sqp-query-chunks prefix) i
proof -
have len-prefix: length (sqp-query-chunks prefix) = i
    using prefix-alignment by simp
have
    state-after-query-chunks
    (staged-query-start-hash data) (staged-query-chunks data) i =
    state-after-query-chunks
    (staged-query-prefix-start-hash prefix)
    (sqp-query-chunks prefix @ chunk # suffix-chunks) i
    using start-hash-eq unfolding data-eq by simp
also have ... =
    state-after-query-chunks
    (staged-query-prefix-start-hash prefix) (sqp-query-chunks prefix) i
    by (rule state-after-query-chunks-append-prefix[OF len-prefix])
finally show ?thesis .
qed
have fields:
    PQueryCounter prefix-state = i
    PState prefix-state =
    state-after-query-chunks
    (staged-query-start-hash data) (staged-query-chunks data) i
    using prefix-alignment chunks-state-eq by simp-all
have raw-lookup':
    fmlookup (HashMap raw-state)
    (QueryIndexChallenge i
    (state-after-query-chunks
    (staged-query-start-hash data) (staged-query-chunks data) i)) =
    Some raw
    using raw-lookup fields by simp
show ?thesis
    by (rule hash-extension-lookup[OF raw-lookup' raw-ext-final])
qed

definition checked-staged-security-with-query-prefix-current-query-partial-opening-hit-at
where
    checked-staged-security-with-query-prefix-current-query-partial-opening-hit-at
    i out  $\longleftrightarrow$ 
    ( $\exists$  trace-openings composition-openings.

```

checked-staged-security-with-query-prefix-authenticated-opening-hit
trace-openings composition-openings i out)

definition *checked-staged-security-with-query-prefix-current-query-partial-opening-hit*
where

checked-staged-security-with-query-prefix-current-query-partial-opening-hit
out $\longleftrightarrow$
 $(\exists i < \text{rounds}.$
checked-staged-security-with-query-prefix-current-query-partial-opening-hit-at
i out)

lemma *checked-staged-security-with-query-prefix-current-query-partial-opening-hit-atI*:
assumes

checked-staged-security-with-query-prefix-authenticated-opening-hit
trace-openings composition-openings i out

shows

checked-staged-security-with-query-prefix-current-query-partial-opening-hit-at
i out

using *assms*

unfolding

checked-staged-security-with-query-prefix-current-query-partial-opening-hit-at-def
by *blast*

lemma *checked-staged-security-with-query-prefix-current-query-partial-opening-hitI*:
assumes *i < rounds*

and

checked-staged-security-with-query-prefix-current-query-partial-opening-hit-at
i out

shows

checked-staged-security-with-query-prefix-current-query-partial-opening-hit
out

using *assms*

unfolding

checked-staged-security-with-query-prefix-current-query-partial-opening-hit-def
by *blast*

definition *staged-query-prefix-authenticated-opening-target*
 $:: 'f \text{ staged-query-prefix-data} \Rightarrow 'f \text{ protocol-channel} \Rightarrow \text{nat set}$

where

staged-query-prefix-authenticated-opening-target prefix prefix-state =
 $\{idx \in \text{query-sample-space}.$
 $\exists (\text{trace-openings} :: 'f \text{ authenticated-opening list})$
 $(\text{composition-openings} :: 'f \text{ authenticated-opening list}).$
 $\text{sqp-composition-fri-roots prefix} \neq [] \wedge$
 $\text{partial-authenticated-table (sqp-trace-root prefix)}$
 $(\text{scale} * \text{clength}) \text{ trace-openings prefix-state} \wedge$
 $\text{partial-authenticated-table (hd (sqp-composition-fri-roots prefix))}$
 $(\text{scale} * \text{clength}) \text{ composition-openings prefix-state} \wedge$
 $\text{partial-query-openings-consistent trace-openings composition-openings}$

$(sqp\text{-}alphas\ prefix)\ idx\}$

definition *staged-query-prefix-prefix-authenticated-trace-indices*

$:: 'f\ staged\text{-}query\text{-}prefix\text{-}data \Rightarrow 'f\ protocol\text{-}channel \Rightarrow nat\ set$

where

staged-query-prefix-prefix-authenticated-trace-indices prefix prefix-state =

$\{idx \in query\text{-}sample\text{-}space.$

$\exists opn :: 'f\ authenticated\text{-}opening.$

$authenticated\text{-}opening\text{-}in\ prefix\text{-}state\ opn \wedge$

$opening\text{-}root\ opn = sqp\text{-}trace\text{-}root\ prefix \wedge$

$opening\text{-}length\ opn = scale * clength \wedge$

$opening\text{-}index\ opn = idx\}$

lemma *staged-query-prefix-prefix-authenticated-opening-target-subset:*

staged-query-prefix-prefix-authenticated-opening-target prefix prefix-state

$\subseteq query\text{-}sample\text{-}space$

unfolding *staged-query-prefix-prefix-authenticated-opening-target-def*

by *blast*

lemma *staged-query-prefix-prefix-authenticated-opening-target-eq-query-sample-space-if-all-indices-witnessed:*

assumes *witnessed:*

$\bigwedge idx. idx \in query\text{-}sample\text{-}space \implies$

$\exists trace\text{-}openings\ composition\text{-}openings.$

$sqp\text{-}composition\text{-}fri\text{-}roots\ prefix \neq [] \wedge$

$partial\text{-}authenticated\text{-}table\ (sqp\text{-}trace\text{-}root\ prefix)$

$(scale * clength)\ trace\text{-}openings\ prefix\text{-}state \wedge$

$partial\text{-}authenticated\text{-}table$

$(hd\ (sqp\text{-}composition\text{-}fri\text{-}roots\ prefix))$

$(scale * clength)\ composition\text{-}openings\ prefix\text{-}state \wedge$

$partial\text{-}query\text{-}openings\text{-}consistent\ trace\text{-}openings$

$composition\text{-}openings\ (sqp\text{-}alphas\ prefix)\ idx$

shows

staged-query-prefix-prefix-authenticated-opening-target prefix

prefix-state = query-sample-space

proof

show

staged-query-prefix-prefix-authenticated-opening-target prefix

prefix-state $\subseteq$ query-sample-space

by (rule *staged-query-prefix-prefix-authenticated-opening-target-subset*)

next

show

query-sample-space $\subseteq$

staged-query-prefix-prefix-authenticated-opening-target prefix

prefix-state

proof

fix *idx*

assume *idx-sample: idx $\in$ query-sample-space*

then obtain *trace-openings composition-openings* **where**

sqp-composition-fri-roots prefix $\neq []$

```

    partial-authenticated-table (sqp-trace-root prefix)
      (scale * clength) trace-openings prefix-state
  partial-authenticated-table
    (hd (sqp-composition-fri-roots prefix))
    (scale * clength) composition-openings prefix-state
  partial-query-openings-consistent trace-openings composition-openings
    (sqp-alphas prefix) idx
  using witnessed by blast
then show
  idx ∈
    staged-query-prefix-prefix-authenticated-opening-target prefix
    prefix-state
  unfolding staged-query-prefix-prefix-authenticated-opening-target-def
  using idx-sample by blast
qed
qed

lemma staged-query-prefix-prefix-authenticated-opening-target-subset-trace-indices:
  staged-query-prefix-prefix-authenticated-opening-target prefix prefix-state
  ⊆
  staged-query-prefix-prefix-authenticated-trace-indices prefix prefix-state
proof
  fix idx
  assume idx-in:
    idx ∈
      staged-query-prefix-prefix-authenticated-opening-target prefix
      prefix-state
  then obtain trace-openings composition-openings where
    idx-sample: idx ∈ query-sample-space
  and trace-table:
    partial-authenticated-table (sqp-trace-root prefix)
      (scale * clength) trace-openings prefix-state
  and consistent:
    partial-query-openings-consistent trace-openings
      composition-openings (sqp-alphas prefix) idx
  unfolding staged-query-prefix-prefix-authenticated-opening-target-def
  by blast
  have trace-indices:
    map opening-index trace-openings = powers-scaled idx
  using consistent unfolding partial-query-openings-consistent-def
  by simp
  have trace-nonempty: trace-openings ≠ []
proof -
  have powers-scaled idx ≠ []
  unfolding powers-scaled-def using powers-pos by simp
  then show ?thesis
  using trace-indices by auto
qed
have hd-idx: opening-index (hd trace-openings) = idx

```

```

proof –
  have hd (map opening-index trace-openings) = hd (powers-scaled idx)
    using trace-indices by simp
  then have opening-index (hd trace-openings) = hd (powers-scaled idx)
    using trace-nonempty by (simp add: hd-map)
  then show ?thesis
    using powers-scaled-hd by simp
qed
have hd-in: hd trace-openings ∈ set trace-openings
  using trace-nonempty by simp
have auth:
  authenticated-opening-in prefix-state (hd trace-openings)
  using trace-table hd-in unfolding partial-authenticated-table-def
  by blast
have root:
  opening-root (hd trace-openings) = sqp-trace-root prefix
  using trace-table hd-in unfolding partial-authenticated-table-def
  by blast
have len:
  opening-length (hd trace-openings) = scale * clength
  using trace-table hd-in unfolding partial-authenticated-table-def
  by blast
show
  idx ∈
    staged-query-prefix-prefix-authenticated-trace-indices prefix
    prefix-state
  unfolding staged-query-prefix-prefix-authenticated-trace-indices-def
  by (intro CollectI conjI exI[of - hd trace-openings])
    (use idx-sample auth root len hd-idx in simp-all)
qed

lemma staged-query-prefix-prefix-authenticated-trace-indices-subset:
  staged-query-prefix-prefix-authenticated-trace-indices prefix prefix-state
  ⊆ query-sample-space
unfolding staged-query-prefix-prefix-authenticated-trace-indices-def
by blast

lemma staged-query-prefix-prefix-authenticated-trace-indices-eq-query-sample-space-if-all-indices-witnessed:
assumes witnessed:
   $\bigwedge idx. idx \in query\_sample\_space \implies$ 
   $\exists opn :: 'f \text{ authenticated-opening.}$ 
    authenticated-opening-in prefix-state opn ∧
    opening-root opn = sqp-trace-root prefix ∧
    opening-length opn = scale * clength ∧
    opening-index opn = idx
shows
  staged-query-prefix-prefix-authenticated-trace-indices prefix
  prefix-state = query-sample-space
proof

```

```

show
  staged-query-prefix-prefix-authenticated-trace-indices prefix
  prefix-state  $\subseteq$  query-sample-space
by (rule staged-query-prefix-prefix-authenticated-trace-indices-subset)
next
show
  query-sample-space  $\subseteq$ 
    staged-query-prefix-prefix-authenticated-trace-indices prefix
    prefix-state
unfolding staged-query-prefix-prefix-authenticated-trace-indices-def
using witnessed by blast
qed

lemma finite-staged-query-prefix-prefix-authenticated-trace-indices[simp]:
  finite
  (staged-query-prefix-prefix-authenticated-trace-indices prefix
   prefix-state)
by (rule finite-subset
  [OF staged-query-prefix-prefix-authenticated-trace-indices-subset
   finite-query-sample-space])

lemma staged-query-prefix-prefix-authenticated-opening-target-fraction-bound-if-trace-indices:
assumes trace-frac:
  nnreal
  (card
    (staged-query-prefix-prefix-authenticated-trace-indices prefix
     prefix-state)) /
  nnreal (card query-sample-space)  $\leq$  query-error-bound
shows
  nnreal
  (card
    (staged-query-prefix-prefix-authenticated-opening-target prefix
     prefix-state)) /
  nnreal (card query-sample-space)  $\leq$  query-error-bound
proof –
have card-le:
  card
  (staged-query-prefix-prefix-authenticated-opening-target prefix
   prefix-state)  $\leq$ 
  card
  (staged-query-prefix-prefix-authenticated-trace-indices prefix
   prefix-state)
by (rule card-mono)
  (simp,
   rule staged-query-prefix-prefix-authenticated-opening-target-subset-trace-indices)
have
  nnreal
  (card
    (staged-query-prefix-prefix-authenticated-opening-target prefix

```

```

      prefix-state)) /
    nnreal (card query-sample-space) ≤
  nnreal
    (card
      (staged-query-prefix-prefix-authenticated-trace-indices prefix
        prefix-state)) /
    nnreal (card query-sample-space)
  by (rule nnreal-nat-divide-right-mono[OF card-le])
also have ... ≤ query-error-bound
  by (rule trace-frac)
finally show ?thesis .
qed

```

lemma *checked-staged-query-prefix-prefix-authenticated-target-hit-bound-from-trace-indices:*

assumes *wf: staged-budget-wellformed budgets*

and *controlled: staged-adversary-controlled budgets A*

and *i-bound: i < rounds*

and *trace-frac:*

$\bigwedge \text{prefix prefix-state.}$

Some ((prefix, prefix-state), prefix-state) ∈
set-dist

(execute (checked-staged-query-prefix-with-state A i)
adversary-initial-state) ⇒

nnreal

(card

(staged-query-prefix-prefix-authenticated-trace-indices prefix
prefix-state)) /

nnreal (card query-sample-space) ≤ query-error-bound

shows

wp-event (checked-staged-query-prefix-receive-with-state A i)

(checked-staged-query-prefix-dynamic-index-hit

staged-query-prefix-prefix-authenticated-opening-target)

adversary-initial-state ≤

staged-phase-relation-error size

(staged-query-search-queries budgets i + 1) + query-error-bound

proof –

have *raw-bound: query-index-raw-preimage-bound*

by (rule *query-index-raw-preimage-bound-from-sampler-wellformed*)

have *hit-bound:*

wp-event (checked-staged-query-prefix-receive-with-state A i)

(checked-staged-query-prefix-dynamic-index-hit

staged-query-prefix-prefix-authenticated-opening-target)

adversary-initial-state ≤

wp-event (checked-staged-query-prefix-receive-with-state A i)

(checked-staged-query-prefix-dynamic-index-prehit

staged-query-prefix-prefix-authenticated-opening-target)

adversary-initial-state + query-error-bound

proof (rule

checked-staged-query-prefix-dynamic-index-hit-bound-by-prehit-and-query-error

```

[OF raw-bound])
fix prefix prefix-state
assume support:
  Some ((prefix, prefix-state), prefix-state) ∈
    set-dist
    (execute (checked-staged-query-prefix-with-state A i)
      adversary-initial-state)
have subset:
  staged-query-prefix-prefix-authenticated-opening-target prefix
  prefix-state ⊆ query-sample-space
by (rule staged-query-prefix-prefix-authenticated-opening-target-subset)
have frac:
  nnreal
  (card
    (staged-query-prefix-prefix-authenticated-opening-target prefix
      prefix-state)) /
  nnreal (card query-sample-space) ≤ query-error-bound
by (rule
  staged-query-prefix-prefix-authenticated-opening-target-fraction-bound-if-trace-indices)
  (rule trace-frac[OF support])
show
  staged-query-prefix-prefix-authenticated-opening-target prefix
  prefix-state ⊆ query-sample-space ∧
  nnreal
  (card
    (staged-query-prefix-prefix-authenticated-opening-target prefix
      prefix-state)) /
  nnreal (card query-sample-space) ≤ query-error-bound
using subset frac by blast
qed
have prehit-bound:
  wp-event (checked-staged-query-prefix-receive-with-state A i)
  (checked-staged-query-prefix-dynamic-index-prehit
    staged-query-prefix-prefix-authenticated-opening-target)
  adversary-initial-state ≤
  staged-phase-relation-error size
  (staged-query-search-queries budgets i + 1)
proof (rule
  checked-staged-query-prefix-dynamic-index-prehit-bound-from-relation-program)
show HashMap adversary-initial-state = fmempty
by (simp add: adversary-initial-state-def)
show hash-relation-program
  (checked-staged-query-prefix-dynamic-index-prehit-relation A i
    adversary-initial-state
    staged-query-prefix-prefix-authenticated-opening-target)
  size (staged-query-search-queries budgets i + 1)
  (checked-staged-query-prefix-receive-with-state A i)
by (rule hash-relation-program-checked-staged-query-prefix-receive-with-state
  [OF wf controlled])

```



```

      (use i-bound in simp-all,
       rule checked-staged-query-prefix-dynamic-index-prehit-relation-fiber-bound-size)
qed
show ?thesis
  by (rule order-trans[OF hit-bound])
    (intro add-mono prehit-bound order-refl)
qed

definition checked-staged-security-with-query-prefix-current-prefix-authenticated-hit-at
where
  checked-staged-security-with-query-prefix-current-prefix-authenticated-hit-at
    i out  $\longleftrightarrow$ 
    ( $\exists$  (trace-openings :: 'f authenticated-opening list)
     (composition-openings :: 'f authenticated-opening list).
     checked-staged-security-with-query-prefix-prefix-authenticated-opening-hit
       trace-openings composition-openings i out)

definition checked-staged-security-with-query-prefix-current-path-output-hit-at
where
  checked-staged-security-with-query-prefix-current-path-output-hit-at
    (i::nat) out  $\longleftrightarrow$ 
    ( $\exists$  (trace-openings :: 'f authenticated-opening list)
     (composition-openings :: 'f authenticated-opening list).
     checked-staged-security-with-query-prefix-opening-path-output-hit
       trace-openings composition-openings out)

lemma checked-staged-security-with-query-prefix-current-path-output-hit-at-imp-lower:
assumes hit:
  checked-staged-security-with-query-prefix-current-path-output-hit-at
    i out
shows
   $\exists$  (trace-openings :: 'f authenticated-opening list)
    (composition-openings :: 'f authenticated-opening list).
    checked-staged-security-with-query-prefix-opening-path-output-hit
      trace-openings composition-openings out
using hit
unfolding
  checked-staged-security-with-query-prefix-current-path-output-hit-at-def
by assumption

lemma checked-staged-security-with-query-prefix-current-path-output-hit-atE:
assumes hit:
  checked-staged-security-with-query-prefix-current-path-output-hit-at
    i out
obtains prefix prefix-state raw raw-state data attacker-state result
  final-state rt len idx val pth where
  out = Some (((((prefix, prefix-state), raw), raw-state),
    ((data, attacker-state), result)), final-state)
  hash-map-new-output-hit

```

```

    (merkle-path-target-roots final-state rt len idx val pth)
    prefix-state final-state
    finite (merkle-path-target-roots final-state rt len idx val pth)
    card (merkle-path-target-roots final-state rt len idx val pth) ≤
      Suc (length pth)
proof –
from hit have lower-ex:
  ∃ (trace-openings :: 'f authenticated-opening list)
    (composition-openings :: 'f authenticated-opening list).
    checked-staged-security-with-query-prefix-opening-path-output-hit
      trace-openings composition-openings out
unfolding
  checked-staged-security-with-query-prefix-current-path-output-hit-at-def
by assumption
then show ?thesis
proof
  fix trace-openings
  assume composition-ex:
    ∃ composition-openings.
      checked-staged-security-with-query-prefix-opening-path-output-hit
        trace-openings composition-openings out
  then show ?thesis
  proof
    fix composition-openings
    assume lower:
      checked-staged-security-with-query-prefix-opening-path-output-hit
        trace-openings composition-openings out
    then show ?thesis
    proof (rule checked-staged-security-with-query-prefix-opening-path-output-hitE)
      fix prefix prefix-state raw raw-state data attacker-state result
        final-state opn
      assume out-eq:
        out = Some (((((prefix, prefix-state), raw), raw-state),
          ((data, attacker-state), result)), final-state)
      assume new-output:
        hash-map-new-output-hit
          (merkle-path-target-roots final-state
            (opening-root opn)
            (opening-length opn)
            (opening-index opn)
            (opening-value opn)
            (opening-path opn))
          prefix-state final-state
      assume finite-roots:
        finite
          (merkle-path-target-roots final-state
            (opening-root opn)
            (opening-length opn)
            (opening-index opn)

```

```

      (opening-value opn)
      (opening-path opn))
assume card-roots:
  card
  (merkle-path-target-roots final-state
   (opening-root opn)
   (opening-length opn)
   (opening-index opn)
   (opening-value opn)
   (opening-path opn)) ≤
  Suc (length (opening-path opn))
show thesis
  by (rule that
    [of prefix prefix-state raw raw-state data attacker-state result
     final-state opening-root opn opening-length opn
     opening-index opn opening-value opn opening-path opn])
    (rule out-eq, rule new-output, rule finite-roots, rule card-roots))
qed
qed
qed
qed

lemma checked-staged-security-with-query-prefix-current-prefix-authenticated-hit-at-imp-dynamic-target:
assumes wf: staged-budget-wellformed budgets
and controlled: staged-adversary-controlled budgets A
and i-bound: i < rounds
and support:
  out ∈
  set-dist
  (execute
   (checked-staged-security-experiment-with-query-prefix-data-state A i)
   adversary-initial-state)
and hit:
  checked-staged-security-with-query-prefix-current-prefix-authenticated-hit-at
  i out
shows
  checked-staged-security-with-query-prefix-dynamic-index-hit
  staged-query-prefix-prefix-authenticated-opening-target out
proof (cases out)
case None
then show ?thesis
  using hit
  unfolding
  checked-staged-security-with-query-prefix-current-prefix-authenticated-hit-at-def
  checked-staged-security-with-query-prefix-prefix-authenticated-opening-hit-def
  by simp
next
case (Some packed)
then obtain prefix prefix-state raw raw-state data attacker-state result

```

```

    final-state where out-eq:
out =
    Some (((((prefix, prefix-state), raw), raw-state),
      ((data, attacker-state), result)), final-state)
by (cases packed, auto split: prod.splits)
have support-out:
    Some (((((prefix, prefix-state), raw), raw-state),
      ((data, attacker-state), result)), final-state) ∈
    set-dist
    (execute
      (checked-staged-security-experiment-with-query-prefix-data-state A i)
      adversary-initial-state)
using support out-eq by simp
have head:
    Some (((prefix, prefix-state), raw), raw-state) ∈
    set-dist
    (execute (checked-staged-query-prefix-receive-with-state A i)
      adversary-initial-state)
and cont:
    Some (((data, attacker-state), result), final-state) ∈
    set-dist
    (execute
      (checked-staged-after-query-prefix-receive-with-verifier A i
        ((prefix, prefix-state), raw))
      raw-state)
using support-out
unfolding checked-staged-security-experiment-with-query-prefix-data-state-def
by (auto elim!: set-dist-bindE)
have prefix-support:
    Some ((prefix, prefix-state), prefix-state) ∈
    set-dist
    (execute (checked-staged-query-prefix-with-state A i)
      adversary-initial-state)
by (rule checked-staged-query-prefix-receive-with-state-prefix-support
      [OF head])
have prefix-alignment:
    PQueryCounter prefix-state = i ∧
    PState prefix-state =
      state-after-query-chunks
      (staged-query-prefix-start-hash prefix)
      (sqp-query-chunks prefix) i ∧
    length (sqp-query-chunks prefix) = i
by (rule checked-staged-query-prefix-with-state-alignment
      [OF wf controlled i-bound prefix-support])
from cont obtain chunk chunk-state assert-state record-state suffix-chunks
where data-eq:
    data =
      (|staged-trace-root = sqp-trace-root prefix,
        staged-trace-fri-roots = sqp-trace-fri-roots prefix,

```

```

    staged-trace-fri-challenges = sqp-trace-fri-challenges prefix,
    staged-trace-final = sqp-trace-final prefix,
    staged-alphas = sqp-alphas prefix,
    staged-degree = sqp-degree prefix,
    staged-composition-fri-roots = sqp-composition-fri-roots prefix,
    staged-composition-fri-challenges =
      sqp-composition-fri-challenges prefix,
    staged-composition-final = sqp-composition-final prefix,
    staged-query-chunks = sqp-query-chunks prefix @ chunk # suffix-chunks)
unfolding checked-staged-after-query-prefix-receive-with-verifier-def
  checked-staged-after-query-prefix-receive-def
by (auto elim!: set-dist-bindE split: prod.splits)
from hit obtain trace-openings composition-openings where prefix-hit:
  checked-staged-security-with-query-prefix-prefix-authenticated-opening-hit
  trace-openings composition-openings i out
unfolding
  checked-staged-security-with-query-prefix-current-prefix-authenticated-hit-at-def
by blast
have idx-in:
  index (to-nat raw) ∈
    staged-query-prefix-prefix-authenticated-opening-target prefix prefix-state
using prefix-hit index-less-query-sample-space
unfolding out-eq
  checked-staged-security-with-query-prefix-prefix-authenticated-opening-hit-def
  staged-query-prefix-prefix-authenticated-opening-target-def
  query-sample-space-def
  data-eq
by auto
show ?thesis
unfolding out-eq
  checked-staged-security-with-query-prefix-dynamic-index-hit-def
  checked-staged-query-prefix-dynamic-index-hit-def
using idx-in by simp
qed

lemma checked-staged-security-with-query-prefix-current-prefix-authenticated-hit-at-le-dynamic-target:
assumes wf: staged-budget-wellformed budgets
and controlled: staged-adversary-controlled budgets A
and i-bound: i < rounds
shows
  wp-event
    (checked-staged-security-experiment-with-query-prefix-data-state A i)
    (checked-staged-security-with-query-prefix-current-prefix-authenticated-hit-at i)
  adversary-initial-state ≤
  wp-event
    (checked-staged-security-experiment-with-query-prefix-data-state A i)
    (checked-staged-security-with-query-prefix-dynamic-index-hit
      staged-query-prefix-prefix-authenticated-opening-target)
  adversary-initial-state

```

by (*rule wp-event-mono-on-support*)

(*rule*

checked-staged-security-with-query-prefix-current-prefix-authenticated-hit-at-imp-dynamic-target
[*OF wf controlled i-bound*])

lemma *checked-staged-security-with-query-prefix-current-prefix-authenticated-hit-at-bound-from-dynamic-target:*

assumes *wf: staged-budget-wellformed budgets*

and *controlled: staged-adversary-controlled budgets A*

and *i-bound: i < rounds*

and *prefix-bound:*

wp-event (checked-staged-query-prefix-receive-with-state A i)

(checked-staged-query-prefix-dynamic-index-hit

staged-query-prefix-prefix-authenticated-opening-target)

adversary-initial-state ≤ B

shows

wp-event

(checked-staged-security-experiment-with-query-prefix-data-state A i)

(checked-staged-security-with-query-prefix-current-prefix-authenticated-hit-at i)

adversary-initial-state ≤ B

proof —

have *lifted:*

wp-event

(checked-staged-security-experiment-with-query-prefix-data-state A i)

(checked-staged-security-with-query-prefix-dynamic-index-hit

staged-query-prefix-prefix-authenticated-opening-target)

adversary-initial-state ≤ B

by (*rule checked-staged-security-with-query-prefix-dynamic-index-hit-bound*

[*OF prefix-bound*])

show *?thesis*

by (*rule order-trans*[

OF checked-staged-security-with-query-prefix-current-prefix-authenticated-hit-at-le-dynamic-target
lifted])

(*use wf controlled i-bound in simp-all*)

qed

lemma *checked-staged-security-with-query-prefix-current-prefix-authenticated-hit-at-bound-from-trace-indices:*

assumes *wf: staged-budget-wellformed budgets*

and *controlled: staged-adversary-controlled budgets A*

and *i-bound: i < rounds*

and *trace-frac:*

∧ prefix prefix-state.

Some ((prefix, prefix-state), prefix-state) ∈

set-dist

(execute (checked-staged-query-prefix-with-state A i)

adversary-initial-state) ⇒

nnreal

(card

(staged-query-prefix-prefix-authenticated-trace-indices prefix

prefix-state)) /

$nnreal \text{ (card query-sample-space)} \leq \text{query-error-bound}$

shows

wp-event
(checked-staged-security-experiment-with-query-prefix-data-state A i)
(checked-staged-security-with-query-prefix-current-prefix-authenticated-hit-at i)
adversary-initial-state $\leq$
staged-phase-relation-error size
(staged-query-search-queries budgets i + 1) + query-error-bound

proof (rule
checked-staged-security-with-query-prefix-current-prefix-authenticated-hit-at-bound-from-dynamic-target
[OF wf controlled i-bound])

show

wp-event (checked-staged-query-prefix-receive-with-state A i)
(checked-staged-query-prefix-dynamic-index-hit
staged-query-prefix-prefix-authenticated-opening-target)
adversary-initial-state $\leq$
staged-phase-relation-error size
(staged-query-search-queries budgets i + 1) + query-error-bound

by (rule
checked-staged-query-prefix-prefix-authenticated-target-hit-bound-from-trace-indices
[OF wf controlled i-bound trace-frac])

qed

lemma *checked-staged-security-with-query-prefix-current-query-partial-opening-hit-at-imp-prefix-or-path-on-support:*
assumes *wf: staged-budget-wellformed budgets*
and *controlled: staged-adversary-controlled budgets A*
and *i-bound: i < rounds*
and *support:*
out $\in$
set-dist
(execute
(checked-staged-security-experiment-with-query-prefix-data-state A i)
adversary-initial-state)
and *hit:*
checked-staged-security-with-query-prefix-current-query-partial-opening-hit-at
i out

shows

checked-staged-security-with-query-prefix-current-prefix-authenticated-hit-at
i out $\vee$
checked-staged-security-with-query-prefix-current-path-output-hit-at
i out

proof –

from *hit* **obtain** *trace-openings composition-openings* **where** *current:*
checked-staged-security-with-query-prefix-authenticated-opening-hit
trace-openings composition-openings i out

unfolding
checked-staged-security-with-query-prefix-current-query-partial-opening-hit-at-def

by *blast*

have *split:*

```

checked-staged-security-with-query-prefix-prefix-authenticated-opening-hit
  trace-openings composition-openings i out ∨
checked-staged-security-with-query-prefix-opening-path-output-hit
  trace-openings composition-openings out
by (rule
  checked-staged-security-with-query-prefix-authenticated-opening-hit-imp-prefix-or-path-output-hit-on-support
  [OF wf controlled i-bound support current])
then show ?thesis
unfolding
  checked-staged-security-with-query-prefix-current-prefix-authenticated-hit-at-def
  checked-staged-security-with-query-prefix-current-path-output-hit-at-def
by blast
qed

lemma checked-staged-security-with-query-prefix-current-query-partial-opening-hit-at-bound-from-prefix-and-path:
  assumes wf: staged-budget-wellformed budgets
  and controlled: staged-adversary-controlled budgets A
  and i-bound: i < rounds
  and prefix-bound:
    wp-event
    (checked-staged-security-experiment-with-query-prefix-data-state A i)
    (checked-staged-security-with-query-prefix-current-prefix-authenticated-hit-at i)
    adversary-initial-state ≤ P
  and path-bound:
    wp-event
    (checked-staged-security-experiment-with-query-prefix-data-state A i)
    (checked-staged-security-with-query-prefix-current-path-output-hit-at i)
    adversary-initial-state ≤ Q
  shows
    wp-event
    (checked-staged-security-experiment-with-query-prefix-data-state A i)
    (checked-staged-security-with-query-prefix-current-query-partial-opening-hit-at i)
    adversary-initial-state ≤ P + Q
proof -
  have event-le:
    wp-event
    (checked-staged-security-experiment-with-query-prefix-data-state A i)
    (checked-staged-security-with-query-prefix-current-query-partial-opening-hit-at i)
    adversary-initial-state ≤
    wp-event
    (checked-staged-security-experiment-with-query-prefix-data-state A i)
    (λout.
      checked-staged-security-with-query-prefix-current-prefix-authenticated-hit-at
        i out ∨
      checked-staged-security-with-query-prefix-current-path-output-hit-at
        i out)
    adversary-initial-state
  by (rule wp-event-mono-on-support)
  (rule

```


checked-staged-security-with-query-prefix-current-query-partial-opening-hit-at-imp-prefix-or-path-on-support
 [OF wf controlled i-bound])
have union-bound:
 wp-event
 (checked-staged-security-experiment-with-query-prefix-data-state A i)
 (λout.
 checked-staged-security-with-query-prefix-current-prefix-authenticated-hit-at
 i out ∨
 checked-staged-security-with-query-prefix-current-path-output-hit-at
 i out)
 adversary-initial-state ≤
 wp-event
 (checked-staged-security-experiment-with-query-prefix-data-state A i)
 (checked-staged-security-with-query-prefix-current-prefix-authenticated-hit-at i)
 adversary-initial-state +
 wp-event
 (checked-staged-security-experiment-with-query-prefix-data-state A i)
 (checked-staged-security-with-query-prefix-current-path-output-hit-at i)
 adversary-initial-state
 by (rule wp-event-union-bound)
have sum-bound:
 wp-event
 (checked-staged-security-experiment-with-query-prefix-data-state A i)
 (checked-staged-security-with-query-prefix-current-prefix-authenticated-hit-at i)
 adversary-initial-state +
 wp-event
 (checked-staged-security-experiment-with-query-prefix-data-state A i)
 (checked-staged-security-with-query-prefix-current-path-output-hit-at i)
 adversary-initial-state ≤ P + Q
 by (intro add-mono prefix-bound path-bound)
show ?thesis
 by (rule order-trans[OF event-le])
 (rule order-trans[OF union-bound sum-bound])
qed

definition staged-security-with-data-state-current-query-partial-opening-hit-at

:: nat ⇒
 (((f staged-proof-data × f protocol-channel) × unit list) ×
 f protocol-channel) option ⇒ bool

where

staged-security-with-data-state-current-query-partial-opening-hit-at i out
 ⇔
 (case out of
 None ⇒ False
 | Some (((data, attacker-state), result), final-state) ⇒
 (let s =
 verifier-state-from-adversary attacker-state
 (staged-proof-transcript data)
 in

$(\exists \text{ raw trace-openings composition-openings rest.}$
 $i < \text{rounds} \wedge$
 $\text{fmlookup (HashMap final-state)}$
 $(\text{QueryIndexChallenge } i$
 $(\text{state-after-query-chunks}$
 $(\text{staged-query-start-hash data})$
 $(\text{staged-query-chunks data } i)) =$
 $\text{Some raw} \wedge$
 $\text{staged-composition-fri-roots data} \neq [] \wedge$
 $\text{Some (result, final-state)} \in$
 $\text{set-dist (execute verify-monad s)} \wedge$
 $\text{partial-authenticated-table (staged-trace-root data)}$
 $(\text{scale} * \text{clength}) \text{ trace-openings final-state} \wedge$
 $\text{partial-authenticated-table}$
 $(\text{hd (staged-composition-fri-roots data)})$
 $(\text{scale} * \text{clength}) \text{ composition-openings final-state} \wedge$
 $\text{verifier-header-transcript s}$
 $(\text{staged-trace-root data})$
 $(\text{staged-trace-fri-roots data})$
 $(\text{staged-trace-final data})$
 $(\text{staged-alphas data})$
 $(\text{staged-degree data})$
 $(\text{staged-composition-fri-roots data})$
 $(\text{staged-composition-final data})$
 $\text{rest} \wedge$
 $\text{partial-query-openings-consistent trace-openings}$
 $\text{composition-openings (staged-alphas data)}$
 $(\text{index (to-nat raw)})))))$

definition *staged-security-with-data-state-current-query-partial-opening-hit*
 $:: (((\text{'f staged-proof-data} \times \text{'f protocol-channel}) \times \text{unit list}) \times$
 $\text{'f protocol-channel}) \text{ option} \Rightarrow \text{bool}$

where

staged-security-with-data-state-current-query-partial-opening-hit out

$\longleftrightarrow$

$(\exists i < \text{rounds.}$
 $\text{staged-security-with-data-state-current-query-partial-opening-hit-at}$
 $i \text{ out})$

lemma *staged-security-with-data-state-current-query-partial-opening-hit-at-aligned-transcript-classification-on-support*

assumes *wf: staged-budget-wellformed budgets*

and *controlled: staged-adversary-controlled budgets A*

and *support:*

$\text{Some } (((\text{data, attacker-state}), \text{result}), \text{final-state}) \in$
 set-dist

$(\text{execute}$

$(\text{checked-staged-security-experiment-with-data-state } A)$

$\text{adversary-initial-state})$

and *partial:*

```

    accepted-with-partial-initial-openings-aligned-transcript-consistent
      (verifier-state-from-adversary attacker-state
        (staged-proof-transcript data))
      (Some (result, final-state)) fr f-fri-roots f-final as dg
      composition-fri-roots final query-idxs trace-openings
      composition-openings
  and no-bad:
    ¬ partial-merkle-inconsistency-bad
      (verifier-state-from-adversary attacker-state
        (staged-proof-transcript data))
      (Some (result, final-state))
  and hit:
    staged-security-with-data-state-current-query-partial-opening-hit-at i
      (Some (((data, attacker-state), result), final-state))
shows
  trace-fri-bad-with-partial-candidates
    (verifier-state-from-adversary attacker-state
      (staged-proof-transcript data))
    (Some (result, final-state)) ∨
  composition-fri-bad-with-partial-candidates
    (verifier-state-from-adversary attacker-state
      (staged-proof-transcript data))
    (Some (result, final-state)) ∨
  query-bad-with-partial-candidates
    (verifier-state-from-adversary attacker-state
      (staged-proof-transcript data))
    (Some (result, final-state)) ∨
  (∃ trace-table composition-table trace-openings-i composition-openings-i.
    partial-trace-table-candidate trace-table
      (trace-openings[i := trace-openings-i]) ∧
    partial-composition-table-candidate composition-table
      (composition-openings[i := composition-openings-i]) ∧
    trace-table-low-degree trace-table ∧
    composition-table-low-degree maxDegree composition-table ∧
    all-queries-consistent trace-table composition-table as)
proof —
  let ?s =
    verifier-state-from-adversary attacker-state
      (staged-proof-transcript data)
  from checked-staged-security-experiment-with-data-state-outcomeE
    [OF support]
  obtain builder where builder:
    Some (data, attacker-state) ∈
      set-dist
        (execute (checked-staged-transcript-program A)
          adversary-initial-state)
  by blast
  have staged-header:
    verifier-header-transcript ?s

```

```

    (staged-trace-root data)
    (staged-trace-fri-roots data)
    (staged-trace-final data)
    (staged-alphas data)
    (staged-degree data)
    (staged-composition-fri-roots data)
    (staged-composition-final data)
    (List.concat (staged-query-chunks data))
  by (rule checked-staged-transcript-program-outcome-header-transcript
      [OF wf controlled builder])
  have shape:
    accepted-transcript-shape ?s (Some (result, final-state)) as
      query-idxs
  by (rule
      accepted-with-partial-initial-openings-aligned-transcript-consistent-imp-shape
      [OF partial])
  from checked-staged-security-with-data-state-accepted-shape-query-chunks
    [OF wf controlled support shape]
  obtain raw-idxs where as-eq: as = staged-alphas data
  and len-raw: length raw-idxs = rounds
  and query-idxs-eq:
    query-idxs = map (λraw. index (to-nat raw)) raw-idxs
  and lookup:
    ∀ j < rounds.
      fmllookup (HashMap final-state)
        (QueryIndexChallenge j
          (state-after-query-chunks
            (staged-query-start-hash data)
            (staged-query-chunks data) j)) =
        Some (raw-idxs ! j)
  by blast
  from partial have aligned:
    accepted-with-partial-initial-openings-aligned-consistent ?s
      (Some (result, final-state)) fr f-fri-roots f-final as dg
      composition-fri-roots final query-idxs trace-openings
      composition-openings
  by (rule
      accepted-with-partial-initial-openings-aligned-transcript-consistent-imp-aligned)
  have aligned0:
    accepted-with-partial-initial-openings-aligned ?s
      (Some (result, final-state)) fr f-fri-roots f-final as dg
      composition-fri-roots final query-idxs trace-openings
      composition-openings
  by (rule
      accepted-with-partial-initial-openings-aligned-consistent-imp-aligned
      [OF aligned])
  from accepted-with-partial-initial-openings-aligned-shapes [OF aligned0]
  obtain rest where header:
    verifier-header-transcript ?s fr f-fri-roots f-final as dg

```

```

    composition-fri-roots final rest
  by blast
have header-eq:
  fr = staged-trace-root data ∧
  f-fri-roots = staged-trace-fri-roots data ∧
  f-final = staged-trace-final data ∧
  as = staged-alphas data ∧
  dg = staged-degree data ∧
  composition-fri-roots = staged-composition-fri-roots data ∧
  final = staged-composition-final data ∧
  rest = List.concat (staged-query-chunks data)
using verifier-header-transcript-unique[OF header staged-header]
by simp
from hit obtain raw trace-openings-i composition-openings-i rest' where
  i-bound: i < rounds
and final-lookup:
  fmlookup (HashMap final-state)
    (QueryIndexChallenge i
      (state-after-query-chunks
        (staged-query-start-hash data)
        (staged-query-chunks data) i)) =
    Some raw
and trace-auth:
  partial-authenticated-table (staged-trace-root data)
    (scale * clength) trace-openings-i final-state
and comp-auth:
  partial-authenticated-table
    (hd (staged-composition-fri-roots data)) (scale * clength)
    composition-openings-i final-state
and consistent-raw:
  partial-query-openings-consistent trace-openings-i
    composition-openings-i (staged-alphas data) (index (to-nat raw))
unfolding staged-security-with-data-state-current-query-partial-opening-hit-at-def
  Let-def
by auto
have raw-eq: raw = raw-idxs ! i
  using final-lookup lookup i-bound by simp
have idx-eq: index (to-nat raw) = query-idxs ! i
  using raw-eq query-idxs-eq len-raw i-bound by simp
have consistent:
  partial-query-openings-consistent trace-openings-i
    composition-openings-i as (query-idxs ! i)
  using consistent-raw idx-eq header-eq by simp
have trace-auth':
  partial-authenticated-table fr (scale * clength) trace-openings-i
    final-state
  using trace-auth header-eq by simp
have comp-auth':
  partial-authenticated-table (hd composition-fri-roots)

```

```

    (scale * clength) composition-openings-i final-state
using comp-auth header-eq by simp
have classified:
  trace-fri-bad-with-partial-candidates ?s
    (Some (result, final-state)) ∨
  composition-fri-bad-with-partial-candidates ?s
    (Some (result, final-state)) ∨
  query-bad-with-partial-candidates ?s (Some (result, final-state)) ∨
  (∃ trace-table composition-table.
    partial-trace-table-candidate trace-table
      (trace-openings[i := trace-openings-i]) ∧
    partial-composition-table-candidate composition-table
      (composition-openings[i := composition-openings-i]) ∧
    trace-table-low-degree trace-table ∧
    composition-table-low-degree maxDegree composition-table ∧
    all-queries-consistent trace-table composition-table as)
by (rule
  accepted-with-partial-initial-openings-aligned-transcript-consistent-update-round-candidate-classification
    [OF partial no-bad i-bound consistent trace-auth' comp-auth'])
then show ?thesis
by blast
qed

lemma staged-security-with-data-state-current-query-partial-opening-hit-at-imp-query-prefix-current-on-support:
assumes wf: staged-budget-wellformed budgets
and controlled: staged-adversary-controlled budgets A
and i-bound: i < rounds
and support:
  Some (((((prefix, prefix-state), raw), raw-state),
    ((data, attacker-state), result)), final-state) ∈
  set-dist
    (execute
      (checked-staged-security-experiment-with-query-prefix-data-state A i)
      adversary-initial-state)
and hit:
  staged-security-with-data-state-current-query-partial-opening-hit-at i
    (Some (((data, attacker-state), result), final-state))
shows
  checked-staged-security-with-query-prefix-current-query-partial-opening-hit-at
    i
    (Some (((((prefix, prefix-state), raw), raw-state),
      ((data, attacker-state), result)), final-state))
proof –
let ?s =
  verifier-state-from-adversary attacker-state
    (staged-proof-transcript data)
have hit':
  ∃ raw' trace-openings composition-openings rest.
    i < rounds ∧

```

```

fmlookup (HashMap final-state)
  (QueryIndexChallenge i
    (state-after-query-chunks
      (staged-query-start-hash data)
      (staged-query-chunks data) i)) =
Some raw' ∧
staged-composition-fri-roots data ≠ [] ∧
Some (result, final-state) ∈ set-dist (execute verify-monad ?s) ∧
partial-authenticated-table (staged-trace-root data)
  (scale * clength) trace-openings final-state ∧
partial-authenticated-table
  (hd (staged-composition-fri-roots data)) (scale * clength)
  composition-openings final-state ∧
verifier-header-transcript ?s
  (staged-trace-root data)
  (staged-trace-fri-roots data)
  (staged-trace-final data)
  (staged-alphas data)
  (staged-degree data)
  (staged-composition-fri-roots data)
  (staged-composition-final data)
  rest ∧
partial-query-openings-consistent trace-openings
  composition-openings (staged-alphas data) (index (to-nat raw'))
using hit
unfolding staged-security-with-data-state-current-query-partial-opening-hit-at-def
  Let-def
by simp
from hit'
obtain raw' trace-openings composition-openings rest where
  final-lookup':
    fmlookup (HashMap final-state)
      (QueryIndexChallenge i
        (state-after-query-chunks
          (staged-query-start-hash data)
          (staged-query-chunks data) i)) =
      Some raw'
and comp-nonempty: staged-composition-fri-roots data ≠ []
and trace-auth:
  partial-authenticated-table (staged-trace-root data)
    (scale * clength) trace-openings final-state
and comp-auth:
  partial-authenticated-table
    (hd (staged-composition-fri-roots data)) (scale * clength)
    composition-openings final-state
and consistent':
  partial-query-openings-consistent trace-openings
    composition-openings (staged-alphas data) (index (to-nat raw'))
by blast

```

```

have final-lookup:
  fmlookup (HashMap final-state)
    (QueryIndexChallenge i
      (state-after-query-chunks
        (staged-query-start-hash data) (staged-query-chunks data) i)) =
      Some raw
by (rule
      checked-staged-security-experiment-with-query-prefix-data-state-raw-final-lookup
      [OF wf controlled i-bound support])
have raw-eq: raw' = raw
  using final-lookup' final-lookup by simp
have current:
  checked-staged-security-with-query-prefix-authenticated-opening-hit
    trace-openings composition-openings i
    (Some ((((prefix, prefix-state), raw), raw-state),
      ((data, attacker-state), result)), final-state))
  unfolding checked-staged-security-with-query-prefix-authenticated-opening-hit-def
  using comp-nonempty trace-auth comp-auth consistent' raw-eq by simp
show ?thesis
  by (rule
    checked-staged-security-with-query-prefix-current-query-partial-opening-hit-atI
    [OF current])
qed

lemma checked-staged-security-with-data-state-current-query-partial-opening-hit-at-le-query-prefix-current:
assumes wf: staged-budget-wellformed budgets
and controlled: staged-adversary-controlled budgets A
and i-bound: i < rounds
shows
  wp-event
    (checked-staged-security-experiment-with-data-state A)
    (staged-security-with-data-state-current-query-partial-opening-hit-at i)
    adversary-initial-state ≤
  wp-event
    (checked-staged-security-experiment-with-query-prefix-data-state A i)
    (checked-staged-security-with-query-prefix-current-query-partial-opening-hit-at i)
    adversary-initial-state
proof –
let ?projected =
  λout. case out of
    None ⇒
      staged-security-with-data-state-current-query-partial-opening-hit-at
        i None
    | Some (packed, t) ⇒
      staged-security-with-data-state-current-query-partial-opening-hit-at
        i (Some (snd packed, t))
have projection:
  wp-event
    (checked-staged-security-experiment-with-data-state A)

```



```

    (staged-security-with-data-state-current-query-partial-opening-hit-at i)
    adversary-initial-state =
wp-event
    (checked-staged-security-experiment-with-query-prefix-data-state A i)
    ?projected
    adversary-initial-state
by (rule
    checked-staged-security-experiment-with-query-prefix-data-state-projection-event
    [OF i-bound])
have event-le:
wp-event
    (checked-staged-security-experiment-with-query-prefix-data-state A i)
    ?projected
    adversary-initial-state ≤
wp-event
    (checked-staged-security-experiment-with-query-prefix-data-state A i)
    (checked-staged-security-with-query-prefix-current-query-partial-opening-hit-at i)
    adversary-initial-state
proof (rule wp-event-mono-on-support)
fix out
assume support:
    out ∈
    set-dist
    (execute
    (checked-staged-security-experiment-with-query-prefix-data-state A i)
    adversary-initial-state)
    and hit: ?projected out
show
    checked-staged-security-with-query-prefix-current-query-partial-opening-hit-at
    i out
proof (cases out)
case None
then show ?thesis
    using hit
    unfolding
    staged-security-with-data-state-current-query-partial-opening-hit-at-def
    by simp
next
case (Some packed)
then obtain prefix prefix-state raw raw-state data attacker-state result
    final-state where out-eq:
    out =
    Some (((((prefix, prefix-state), raw), raw-state),
    ((data, attacker-state), result)), final-state)
    by (cases packed) (auto split: prod.splits)
show ?thesis
    unfolding out-eq
    by (rule
    staged-security-with-data-state-current-query-partial-opening-hit-at-imp-query-prefix-current-on-support

```

```

      [OF wf controlled i-bound])
    (use support hit out-eq in simp-all)
  qed
qed
show ?thesis
  unfolding projection
  by (rule event-le)
qed

lemma checked-staged-security-with-data-state-current-query-partial-opening-hit-bound-from-query-prefix-rounds:
  assumes round-bound:
     $\bigwedge i. i < \text{rounds} \implies$ 
      wp-event
      (checked-staged-security-experiment-with-query-prefix-data-state A i)
      (checked-staged-security-with-query-prefix-current-query-partial-opening-hit-at i)
      adversary-initial-state  $\leq C i$ 
  and wf: staged-budget-wellformed budgets
  and controlled: staged-adversary-controlled budgets A
  shows
    wp-event
    (checked-staged-security-experiment-with-data-state A)
    staged-security-with-data-state-current-query-partial-opening-hit
    adversary-initial-state  $\leq (\sum i < \text{rounds}. C i)$ 
proof -
  have round-le:
     $\bigwedge i. i < \text{rounds} \implies$ 
      wp-event
      (checked-staged-security-experiment-with-data-state A)
      (staged-security-with-data-state-current-query-partial-opening-hit-at i)
      adversary-initial-state  $\leq C i$ 
  proof -
    fix i
    assume i-bound:  $i < \text{rounds}$ 
    show
      wp-event
      (checked-staged-security-experiment-with-data-state A)
      (staged-security-with-data-state-current-query-partial-opening-hit-at i)
      adversary-initial-state  $\leq C i$ 
    by (rule order-trans
      [OF
        checked-staged-security-with-data-state-current-query-partial-opening-hit-at-le-query-prefix-current
        round-bound[OF i-bound]])
      (use wf controlled i-bound in simp-all)
  qed
  have event-le:
    wp-event
    (checked-staged-security-experiment-with-data-state A)
    staged-security-with-data-state-current-query-partial-opening-hit
    adversary-initial-state  $\leq$ 

```

```

wp-event
  (checked-staged-security-experiment-with-data-state A)
  ( $\lambda out. \exists i \in \{..<rounds\}.$ 
    staged-security-with-data-state-current-query-partial-opening-hit-at
      i out)
  adversary-initial-state
by (rule wp-event-mono)
  (auto simp: staged-security-with-data-state-current-query-partial-opening-hit-def)
also have ...  $\leq (\sum i<rounds. C i)$ 
  by (rule wp-event-finite-UN-bound)
    (simp-all add: round-le)
finally show ?thesis .
qed

lemma staged-security-with-data-state-current-query-partial-opening-hit-at-imp-partial-opening-hit-at:
  assumes hit:
    staged-security-with-data-state-current-query-partial-opening-hit-at i
      out
  shows
    staged-security-with-data-state-query-partial-opening-hit-at i out
proof (cases out)
case None
then show ?thesis
  using hit
  unfolding
    staged-security-with-data-state-current-query-partial-opening-hit-at-def
    staged-security-with-data-state-query-partial-opening-hit-at-def
  by simp
next
case (Some packed)
then obtain data attacker-state result final-state where out-eq:
  out = Some (((data, attacker-state), result), final-state)
  by (cases packed, auto split: prod.splits)
let ?s =
  verifier-state-from-adversary attacker-state
    (staged-proof-transcript data)
have hit':
 $\exists raw$  trace-openings composition-openings rest.
  i < rounds  $\wedge$ 
  fmlookup (HashMap final-state)
    (QueryIndexChallenge i
      (state-after-query-chunks
        (staged-query-start-hash data)
        (staged-query-chunks data) i)) =
  Some raw  $\wedge$ 
  staged-composition-fri-roots data  $\neq \square$   $\wedge$ 
  Some (result, final-state)  $\in$  set-dist (execute verify-monad ?s)  $\wedge$ 
  partial-authenticated-table (staged-trace-root data)
    (scale * clength) trace-openings final-state  $\wedge$ 

```

```

partial-authenticated-table
  (hd (staged-composition-fri-roots data)) (scale * clength)
  composition-openings final-state ∧
verifier-header-transcript ?s
  (staged-trace-root data)
  (staged-trace-fri-roots data)
  (staged-trace-final data)
  (staged-alphas data)
  (staged-degree data)
  (staged-composition-fri-roots data)
  (staged-composition-final data)
  rest ∧
partial-query-openings-consistent trace-openings
  composition-openings (staged-alphas data) (index (to-nat raw))
using hit
unfolding out-eq
  staged-security-with-data-state-current-query-partial-opening-hit-at-def
  Let-def
by simp
from hit'
obtain raw trace-openings composition-openings rest where
  i-bound: i < rounds
and lookup:
  fmlookup (HashMap final-state)
    (QueryIndexChallenge i
      (state-after-query-chunks
        (staged-query-start-hash data)
        (staged-query-chunks data) i)) =
    Some raw
and comp-nonempty: staged-composition-fri-roots data ≠ []
and verifier:
  Some (result, final-state) ∈ set-dist (execute verify-monad ?s)
and trace-auth:
  partial-authenticated-table (staged-trace-root data)
    (scale * clength) trace-openings final-state
and comp-auth:
  partial-authenticated-table
    (hd (staged-composition-fri-roots data)) (scale * clength)
    composition-openings final-state
and header:
  verifier-header-transcript ?s
    (staged-trace-root data)
    (staged-trace-fri-roots data)
    (staged-trace-final data)
    (staged-alphas data)
    (staged-degree data)
    (staged-composition-fri-roots data)
    (staged-composition-final data)
    rest

```

```

and consistent:
  partial-query-openings-consistent trace-openings
  composition-openings (staged-alphas data) (index (to-nat raw))
by blast
have idx-sample: index (to-nat raw) ∈ query-sample-space
using consistent
unfolding partial-query-openings-consistent-def by simp
have witness:
  (trace-openings, composition-openings) ∈
    query-header-supported-partial-opening-witnesses ?s
    (staged-trace-root data)
    (staged-trace-fri-roots data)
    (staged-trace-final data)
    (staged-alphas data)
    (staged-degree data)
    (staged-composition-fri-roots data)
    (staged-composition-final data)
unfolding query-header-supported-partial-opening-witnesses-def
using comp-nonempty verifier trace-auth comp-auth header
by blast
have success:
  index (to-nat raw) ∈
    query-header-supported-partial-opening-success-indices-at ?s
    (staged-trace-root data)
    (staged-trace-fri-roots data)
    (staged-trace-final data)
    (staged-alphas data)
    (staged-degree data)
    (staged-composition-fri-roots data)
    (staged-composition-final data)
    i
unfolding query-header-supported-partial-opening-success-indices-at-def
by (intro CollectI conjI exI[of - trace-openings]
      exI[of - composition-openings])
      (use idx-sample witness consistent in simp-all)
show ?thesis
unfolding out-eq staged-security-with-data-state-query-partial-opening-hit-at-def
  Let-def
using i-bound lookup success by simp
qed

lemma staged-security-with-data-state-current-query-partial-opening-hit-imp-partial-opening-hit:
assumes hit:
  staged-security-with-data-state-current-query-partial-opening-hit out
shows staged-security-with-data-state-query-partial-opening-hit out
proof —
from hit obtain i where i-bound: i < rounds
and round-hit:
  staged-security-with-data-state-current-query-partial-opening-hit-at

```

i out
unfolding *staged-security-with-data-state-current-query-partial-opening-hit-def*
by *blast*
have
staged-security-with-data-state-query-partial-opening-hit-at i out
by (rule
staged-security-with-data-state-current-query-partial-opening-hit-at-imp-partial-opening-hit-at
[OF round-hit])
then show *?thesis*
by (rule *staged-security-with-data-state-query-partial-opening-hitI*
[OF i-bound])
qed

lemma *staged-security-with-data-state-current-query-partial-opening-hitI:*

assumes *i < rounds*

and

staged-security-with-data-state-current-query-partial-opening-hit-at
i out

shows

staged-security-with-data-state-current-query-partial-opening-hit out

using *assms*

unfolding *staged-security-with-data-state-current-query-partial-opening-hit-def*

by *blast*

lemma *checked-staged-security-with-data-state-aligned-transcript-query-bad-imp-current-partial-opening-hit-on-suppo*

assumes *wf: staged-budget-wellformed budgets*

and *controlled: staged-adversary-controlled budgets A*

and *support:*

Some (((data, attacker-state), result), final-state) ∈
set-dist
(execute
(checked-staged-security-experiment-with-data-state A)
adversary-initial-state)

and *bad:*

staged-security-with-data-state-verifier-event
query-bad-with-aligned-transcript-partial-candidates
(Some (((data, attacker-state), result), final-state))

shows

staged-security-with-data-state-current-query-partial-opening-hit
(Some (((data, attacker-state), result), final-state))

proof –

let *?s =*

verifier-state-from-adversary attacker-state
(staged-proof-transcript data)

from *checked-staged-security-experiment-with-data-state-outcomeE*

[OF support]

obtain *builder* **where** *builder:*

Some (data, attacker-state) ∈
set-dist

```

      (execute (checked-staged-transcript-program A)
        adversary-initial-state)
    and verifier:
      Some (result, final-state) ∈ set-dist (execute verify-monad ?s)
    by blast
  have staged-header:
    verifier-header-transcript ?s
    (staged-trace-root data)
    (staged-trace-fri-roots data)
    (staged-trace-final data)
    (staged-alphas data)
    (staged-degree data)
    (staged-composition-fri-roots data)
    (staged-composition-final data)
    (List.concat (staged-query-chunks data))
  by (rule checked-staged-transcript-program-outcome-header-transcript
    [OF wf controlled builder])
  from bad obtain fr f-fri-roots f-final as dg composition-fri-roots
    final query-idxs trace-openings composition-openings trace-table
    composition-table where partial:
    accepted-with-partial-initial-openings-aligned-transcript-consistent ?s
    (Some (result, final-state)) fr f-fri-roots f-final as dg
    composition-fri-roots final query-idxs trace-openings
    composition-openings
  unfolding staged-security-with-data-state-verifier-event-def
    query-bad-with-aligned-transcript-partial-candidates-def
  by simp blast
  have aligned:
    accepted-with-partial-initial-openings-aligned-consistent ?s
    (Some (result, final-state)) fr f-fri-roots f-final as dg
    composition-fri-roots final query-idxs trace-openings
    composition-openings
  by (rule
    accepted-with-partial-initial-openings-aligned-transcript-consistent-imp-aligned
    [OF partial])
  have shape:
    accepted-transcript-shape ?s (Some (result, final-state)) as
    query-idxs
  by (rule
    accepted-with-partial-initial-openings-aligned-transcript-consistent-imp-shape
    [OF partial])
  from checked-staged-security-with-data-state-accepted-shape-query-chunks
    [OF wf controlled support shape]
  obtain raw-idxs where
    as-eq: as = staged-alphas data
  and len-raw: length raw-idxs = rounds
  and query-idxs-eq:
    query-idxs = map (λraw. index (to-nat raw)) raw-idxs
  and lookup:

```

```

     $\forall i < \text{rounds}.$ 
     $\text{fmlookup } (\text{HashMap } \text{final-state})$ 
       $(\text{QueryIndexChallenge } i$ 
         $(\text{state-after-query-chunks}$ 
           $(\text{staged-query-start-hash } \text{data})$ 
           $(\text{staged-query-chunks } \text{data}) \ i)) =$ 
         $\text{Some } (\text{raw-idxs } ! \ i)$ 
    by blast
have aligned0:
  accepted-with-partial-initial-openings-aligned ?s
     $(\text{Some } (\text{result}, \text{final-state})) \text{ fr } f\text{-fri-roots } f\text{-final as } dg$ 
    composition-fri-roots final query-idxs trace-openings
    composition-openings
  by (rule
    accepted-with-partial-initial-openings-aligned-consistent-imp-aligned
    [OF aligned])
from accepted-with-partial-initial-openings-aligned-shapes[OF aligned0]
obtain rest where header:
  verifier-header-transcript ?s fr f-fri-roots f-final as dg
  composition-fri-roots final rest
  by blast
have comp-nonempty: composition-fri-roots  $\neq []$ 
  using accepted-with-partial-initial-openings-aligned-shapes(1)
    [OF aligned0] .
have trace-partial:
  accepted-with-partial-trace-openings ?s
     $(\text{Some } (\text{result}, \text{final-state})) \text{ fr } \text{query-idxs } \text{trace-openings}$ 
  using accepted-with-partial-initial-openings-aligned-shapes(3)
    [OF aligned0] .
have comp-partial:
  accepted-with-partial-composition-openings ?s
     $(\text{Some } (\text{result}, \text{final-state})) \text{ (hd } \text{composition-fri-roots}) \text{ query-idxs}$ 
    composition-openings
  using accepted-with-partial-initial-openings-aligned-shapes(4)
    [OF aligned0] .
have header-eq:
  fr = staged-trace-root data  $\wedge$ 
  f-fri-roots = staged-trace-fri-roots data  $\wedge$ 
  f-final = staged-trace-final data  $\wedge$ 
  as = staged-alphas data  $\wedge$ 
  dg = staged-degree data  $\wedge$ 
  composition-fri-roots = staged-composition-fri-roots data  $\wedge$ 
  final = staged-composition-final data  $\wedge$ 
  rest = List.concat (staged-query-chunks data)
  using verifier-header-transcript-unique[OF header staged-header]
  by simp
let ?i = 0
have i-bound: ?i < rounds
  using rounds-positive by simp

```



```

have round-consistent:
  partial-query-round-consistent trace-openings composition-openings as
    ?i (query-idxs ! ?i)
using aligned i-bound
unfolding accepted-with-partial-initial-openings-aligned-consistent-def
by simp
have opening-consistent:
  partial-query-openings-consistent (trace-openings ! ?i)
    (composition-openings ! ?i) as (query-idxs ! ?i)
using round-consistent
unfolding partial-query-round-consistent-def
  partial-query-openings-consistent-def
by simp
have trace-table:
  partial-authenticated-table fr (scale * clength)
    (trace-openings ! ?i) final-state
using trace-partial i-bound
unfolding accepted-with-partial-trace-openings-def by blast
have comp-table:
  partial-authenticated-table (hd composition-fri-roots)
    (scale * clength) (composition-openings ! ?i) final-state
using comp-partial i-bound
unfolding accepted-with-partial-composition-openings-def by blast
have raw-idx:
  index (to-nat (raw-idxs ! ?i)) = query-idxs ! ?i
using query-idxs-eq len-raw i-bound by simp
have lookup-i:
  fmlookup (HashMap final-state)
    (QueryIndexChallenge ?i
      (state-after-query-chunks
        (staged-query-start-hash data) (staged-query-chunks data) ?i)) =
    Some (raw-idxs ! ?i)
using lookup i-bound by simp
have exists-hit:
  ∃ raw trace-openings' composition-openings' rest'.
    ?i < rounds ∧
    fmlookup (HashMap final-state)
      (QueryIndexChallenge ?i
        (state-after-query-chunks
          (staged-query-start-hash data) (staged-query-chunks data) ?i)) =
      Some raw ∧
    staged-composition-fri-roots data ≠ [] ∧
    Some (result, final-state) ∈ set-dist (execute verify-monad ?s) ∧
    partial-authenticated-table (staged-trace-root data)
      (scale * clength) trace-openings' final-state ∧
    partial-authenticated-table (hd (staged-composition-fri-roots data))
      (scale * clength) composition-openings' final-state ∧
    verifier-header-transcript ?s
      (staged-trace-root data)

```

```

    (staged-trace-fri-roots data)
    (staged-trace-final data)
    (staged-alphas data)
    (staged-degree data)
    (staged-composition-fri-roots data)
    (staged-composition-final data)
    rest'  $\wedge$ 
    partial-query-openings-consistent trace-openings'
    composition-openings' (staged-alphas data) (index (to-nat raw))
  apply (rule exI[where x = raw-idxs ! ?i])
  apply (rule exI[where x = trace-openings ! ?i])
  apply (rule exI[where x = composition-openings ! ?i])
  apply (rule exI[where x = List.concat (staged-query-chunks data)])
  using i-bound lookup-i comp-nonempty verifier trace-table comp-table
    opening-consistent raw-idx header-eq staged-header
  by simp
have hit-at:
  staged-security-with-data-state-current-query-partial-opening-hit-at ?i
  (Some (((data, attacker-state), result), final-state))
unfolding
  staged-security-with-data-state-current-query-partial-opening-hit-at-def
  Let-def
  using exists-hit by simp
show ?thesis
  by (rule staged-security-with-data-state-current-query-partial-opening-hitI
    [OF i-bound hit-at])
qed

```

lemma *checked-staged-security-with-data-state-aligned-transcript-query-bad-bound-from-current-partial-opening-hit:*
assumes wf: staged-budget-wellformed budgets
and controlled: staged-adversary-controlled budgets A
shows
 wp-event (checked-staged-security-experiment-with-data-state A)
 (staged-security-with-data-state-verifier-event
 query-bad-with-aligned-transcript-partial-candidates)
 adversary-initial-state $\leq$
 wp-event (checked-staged-security-experiment-with-data-state A)
 staged-security-with-data-state-current-query-partial-opening-hit
 adversary-initial-state
proof (rule wp-event-mono-on-support)
fix out
assume support:
 out $\in$
 set-dist
 (execute
 (checked-staged-security-experiment-with-data-state A)
 adversary-initial-state)
and bad:
 staged-security-with-data-state-verifier-event

```

      query-bad-with-aligned-transcript-partial-candidates out
show staged-security-with-data-state-current-query-partial-opening-hit out
proof (cases out)
  case None
  then show ?thesis
    using bad
    unfolding staged-security-with-data-state-verifier-event-def
      query-bad-with-aligned-transcript-partial-candidates-def
    by simp
next
  case (Some packed)
  then obtain data attacker-state result final-state where out-eq:
    out = Some (((data, attacker-state), result), final-state)
    by (cases packed, auto split: prod.splits)
  show ?thesis
    unfolding out-eq
    by (rule
      checked-staged-security-with-data-state-aligned-transcript-query-bad-imp-current-partial-opening-hit-on-support
      [OF wf controlled])
      (use support bad out-eq in simp-all)
qed
qed

lemma checked-staged-security-with-actual-alpha-prefix-composition-candidate-drift-query-hit-imp-data-state-current-
assumes wf: staged-budget-wellformed budgets
and controlled: staged-adversary-controlled budgets A
and support:
  Some (((((alpha-prefix, alpha-prefix-state), data), attacker-state),
    result), final-state) ∈
  set-dist
    (execute
      (checked-staged-security-experiment-with-actual-alpha-prefix-data-state
        A)
      adversary-initial-state)
and hit:
  checked-staged-security-with-actual-alpha-prefix-composition-candidate-drift-query-hit
    (Some (((((alpha-prefix, alpha-prefix-state), data), attacker-state),
      result), final-state))
shows
  staged-security-with-data-state-current-query-partial-opening-hit
    (Some (((data, attacker-state), result), final-state))
proof –
  let ?s =
    verifier-state-from-adversary attacker-state
    (staged-proof-transcript data)
  have data-support:
    Some (((data, attacker-state), result), final-state) ∈
    set-dist
      (execute (checked-staged-security-experiment-with-data-state A)

```

```

    adversary-initial-state)
  by (rule
    checked-staged-security-with-actual-alpha-prefix-support-imp-data-state-support
    [OF support])
  from checked-staged-security-experiment-with-data-state-outcomeE
    [OF data-support]
  obtain builder verifier where
    builder:
      Some (data, attacker-state) ∈
        set-dist
          (execute (checked-staged-transcript-program A)
            adversary-initial-state)
    and verifier:
      Some (result, final-state) ∈ set-dist (execute verify-monad ?s)
  by blast
  have header:
    verifier-header-transcript ?s
      (staged-trace-root data)
      (staged-trace-fri-roots data)
      (staged-trace-final data)
      (staged-alphas data)
      (staged-degree data)
      (staged-composition-fri-roots data)
      (staged-composition-final data)
      (List.concat (staged-query-chunks data))
  by (rule checked-staged-transcript-program-outcome-header-transcript
    [OF wf controlled builder])
  show ?thesis
  proof (rule
    checked-staged-security-with-actual-alpha-prefix-composition-candidate-drift-query-hit-current-openingsE
    [OF wf controlled support hit])
    fix s trace-table composition-table query-idxs f i raw trace-openings
      composition-openings trace-openingsss composition-openingsss
    assume s-def:
      s = verifier-state-from-adversary attacker-state
        (staged-proof-transcript data)
    and i-bound: i < rounds
    and final-lookup:
      fmlookup (HashMap final-state)
        (QueryIndexChallenge i
          (state-after-query-chunks
            (staged-query-start-hash data)
            (staged-query-chunks data) i)) =
        Some raw
    and idx-eq: index (to-nat raw) = query-idxs ! i
    and comp-nonempty: staged-composition-fri-roots data ≠ []
    and trace-auth:
      partial-authenticated-table (staged-trace-root data)
        (scale * clength) trace-openings final-state

```

```

and comp-auth:
  partial-authenticated-table
    (hd (staged-composition-fri-roots data)) (scale * clength)
    composition-openings final-state
and consistent:
  partial-query-openings-consistent trace-openings
    composition-openings (staged-alphas data) (query-idxs ! i)
have consistent-raw:
  partial-query-openings-consistent trace-openings composition-openings
    (staged-alphas data) (index (to-nat raw))
using consistent idx-eq by simp
have round-hit:
  staged-security-with-data-state-current-query-partial-opening-hit-at i
    (Some (((data, attacker-state), result), final-state))
unfolding
  staged-security-with-data-state-current-query-partial-opening-hit-at-def
  Let-def
apply simp
by (intro exI[of - raw] exI[of - trace-openings]
  exI[of - composition-openings]
  exI[of - List.concat (staged-query-chunks data)] conjI)
  (use i-bound final-lookup comp-nonempty verifier trace-auth
  comp-auth header consistent-raw in simp-all)
show
  staged-security-with-data-state-current-query-partial-opening-hit
    (Some (((data, attacker-state), result), final-state))
by (rule
  staged-security-with-data-state-current-query-partial-opening-hitI
  [OF i-bound round-hit])
qed
qed

lemma checked-staged-security-with-actual-alpha-prefix-composition-candidate-drift-query-hit-le-data-state-current-po
assumes wf: staged-budget-wellformed budgets
and controlled: staged-adversary-controlled budgets A
shows
  wp-event
    (checked-staged-security-experiment-with-actual-alpha-prefix-data-state A)
    checked-staged-security-with-actual-alpha-prefix-composition-candidate-drift-query-hit
    adversary-initial-state ≤
  wp-event
    (checked-staged-security-experiment-with-data-state A)
    staged-security-with-data-state-current-query-partial-opening-hit
    adversary-initial-state
proof –
let ?project =
  λx. ((snd (fst (fst x)), snd (fst x)), snd x)
let ?projected =
  λout. case out of

```

```

  None  $\Rightarrow$  False
| Some (packed, t)  $\Rightarrow$ 
  staged-security-with-data-state-current-query-partial-opening-hit
  (Some (?project packed, t))
have event-le:
  wp-event
  (checked-staged-security-experiment-with-actual-alpha-prefix-data-state A)
  checked-staged-security-with-actual-alpha-prefix-composition-candidate-drift-query-hit
  adversary-initial-state  $\leq$ 
  wp-event
  (checked-staged-security-experiment-with-actual-alpha-prefix-data-state A)
  ?projected adversary-initial-state
proof (rule wp-event-mono-on-support)
fix out
assume support:
  out  $\in$ 
  set-dist
  (execute
    (checked-staged-security-experiment-with-actual-alpha-prefix-data-state
      A)
    adversary-initial-state)
and hit:
  checked-staged-security-with-actual-alpha-prefix-composition-candidate-drift-query-hit
  out
show ?projected out
proof (cases out)
case None
then show ?thesis
  using hit
  unfolding
  checked-staged-security-with-actual-alpha-prefix-composition-candidate-drift-query-hit-def
  checked-staged-security-with-actual-alpha-prefix-composition-candidate-drift-def
  checked-staged-security-with-actual-alpha-prefix-verifier-event-def
  by simp
next
case (Some packed)
then obtain alpha-prefix alpha-prefix-state data attacker-state result
  final-state where out-eq:
  out =
    Some (((((alpha-prefix, alpha-prefix-state), data),
      attacker-state), result), final-state)
  by (cases packed, auto split: prod.splits)
have partial-hit:
  staged-security-with-data-state-current-query-partial-opening-hit
  (Some (((data, attacker-state), result), final-state))
  by (rule
    checked-staged-security-with-actual-alpha-prefix-composition-candidate-drift-query-hit-imp-data-state-current-p
    [OF wf controlled])
  (use support hit out-eq in simp-all)

```

```

    show ?thesis
      unfolding out-eq
      using partial-hit by simp
  qed
qed
have projection:
  wp-event
    (checked-staged-security-experiment-with-actual-alpha-prefix-data-state A)
    ?projected adversary-initial-state =
  wp-event
    (checked-staged-security-experiment-with-data-state A)
    staged-security-with-data-state-current-query-partial-opening-hit
    adversary-initial-state
proof -
  have
    wp-event (checked-staged-security-experiment-with-data-state A)
      staged-security-with-data-state-current-query-partial-opening-hit
      adversary-initial-state =
    wp-event
      (checked-staged-security-experiment-with-actual-alpha-prefix-data-state A
         $\gg (\lambda x. \text{return } (?project\ x))$ )
      staged-security-with-data-state-current-query-partial-opening-hit
      adversary-initial-state
  using
    checked-staged-security-experiment-with-actual-alpha-prefix-data-state-projection
    [of A]
  by simp
also have ... =
  wp-event
    (checked-staged-security-experiment-with-actual-alpha-prefix-data-state A)
    ?projected adversary-initial-state
  by (subst wp-event-bind-return-map)
    (simp add:
      staged-security-with-data-state-current-query-partial-opening-hit-def
      staged-security-with-data-state-current-query-partial-opening-hit-at-def)
  finally show ?thesis
    by simp
qed
show ?thesis
  by (rule order-trans[OF event-le])
    (simp add: projection)
qed

lemma checked-staged-security-with-actual-alpha-prefix-composition-candidate-drift-query-hit-bound-from-data-state-
  assumes wf: staged-budget-wellformed budgets
  and controlled: staged-adversary-controlled budgets A
  and partial-bound:
    wp-event
      (checked-staged-security-experiment-with-data-state A)

```

$\text{staged-security-with-data-state-current-query-partial-opening-hit}$
 $\text{adversary-initial-state} \leq (Q::\text{prob})$

shows

wp-event
 $(\text{checked-staged-security-experiment-with-actual-alpha-prefix-data-state } A)$
 $\text{checked-staged-security-with-actual-alpha-prefix-composition-candidate-drift-query-hit}$
 $\text{adversary-initial-state} \leq Q$

by (rule *order.trans*
 $[OF$
 $\text{checked-staged-security-with-actual-alpha-prefix-composition-candidate-drift-query-hit-le-data-state-current-parti}$
 $\text{partial-bound}]$)
 (use *wf controlled* **in** *simp-all*)

lemma *checked-staged-security-with-actual-alpha-prefix-composition-candidate-drift-query-hit-bound-from-query-prefix*

assumes *wf*: *staged-budget-wellformed* budgets
and *controlled*: *staged-adversary-controlled* budgets *A*
and *round-bound*:
 $\bigwedge i. i < \text{rounds} \implies$
 wp-event
 $(\text{checked-staged-security-experiment-with-query-prefix-data-state } A \ i)$
 $(\text{checked-staged-security-with-query-prefix-current-query-partial-opening-hit-at } i)$
 $\text{adversary-initial-state} \leq C \ i$

shows

wp-event
 $(\text{checked-staged-security-experiment-with-actual-alpha-prefix-data-state } A)$
 $\text{checked-staged-security-with-actual-alpha-prefix-composition-candidate-drift-query-hit}$
 $\text{adversary-initial-state} \leq (\sum i < \text{rounds}. C \ i)$

proof –

have *current-bound*:

wp-event
 $(\text{checked-staged-security-experiment-with-data-state } A)$
 $\text{staged-security-with-data-state-current-query-partial-opening-hit}$
 $\text{adversary-initial-state} \leq (\sum i < \text{rounds}. C \ i)$

by (rule
 $\text{checked-staged-security-with-data-state-current-query-partial-opening-hit-bound-from-query-prefix-rounds}$
 $[OF \text{ round-bound } wf \text{ controlled}]$)

show *?thesis*

by (rule
 $\text{checked-staged-security-with-actual-alpha-prefix-composition-candidate-drift-query-hit-bound-from-data-state-curr}$
 $[OF \text{ wf controlled } \text{current-bound}]$)

qed

lemma *checked-staged-security-with-actual-alpha-prefix-composition-candidate-drift-query-hit-bound-from-prefix-target*

assumes *wf*: *staged-budget-wellformed* budgets
and *controlled*: *staged-adversary-controlled* budgets *A*
and *prefix-bound*:
 $\bigwedge i. i < \text{rounds} \implies$
 $\text{wp-event } (\text{checked-staged-query-prefix-receive-with-state } A \ i)$
 $(\text{checked-staged-query-prefix-dynamic-index-hit}$

$\text{staged-query-prefix-prefix-authenticated-opening-target})$
 $\text{adversary-initial-state} \leq P \ i$
and path-bound :
 $\bigwedge i. i < \text{rounds} \implies$
 wp-event
 $(\text{checked-staged-security-experiment-with-query-prefix-data-state } A \ i)$
 $(\text{checked-staged-security-with-query-prefix-current-path-output-hit-at } i)$
 $\text{adversary-initial-state} \leq Q \ i$
shows
 wp-event
 $(\text{checked-staged-security-experiment-with-actual-alpha-prefix-data-state } A)$
 $\text{checked-staged-security-with-actual-alpha-prefix-composition-candidate-drift-query-hit}$
 $\text{adversary-initial-state} \leq (\sum i < \text{rounds}. P \ i + Q \ i)$
proof (rule
 $\text{checked-staged-security-with-actual-alpha-prefix-composition-candidate-drift-query-hit-bound-from-query-prefix-curr}$
 $[\text{OF wf controlled}])$
fix i
assume $i\text{-bound}$: $i < \text{rounds}$
have prefix-hit-bound :
 wp-event
 $(\text{checked-staged-security-experiment-with-query-prefix-data-state } A \ i)$
 $(\text{checked-staged-security-with-query-prefix-current-prefix-authenticated-hit-at } i)$
 $\text{adversary-initial-state} \leq P \ i$
by (rule
 $\text{checked-staged-security-with-query-prefix-current-prefix-authenticated-hit-at-bound-from-dynamic-target}$
 $[\text{OF wf controlled } i\text{-bound } \text{prefix-bound}[\text{OF } i\text{-bound}]])$
show
 wp-event
 $(\text{checked-staged-security-experiment-with-query-prefix-data-state } A \ i)$
 $(\text{checked-staged-security-with-query-prefix-current-query-partial-opening-hit-at } i)$
 $\text{adversary-initial-state} \leq P \ i + Q \ i$
by (rule
 $\text{checked-staged-security-with-query-prefix-current-query-partial-opening-hit-at-bound-from-prefix-and-path}$
 $[\text{OF wf controlled } i\text{-bound } \text{prefix-hit-bound } \text{path-bound}[\text{OF } i\text{-bound}]])$
qed

lemma $\text{checked-staged-security-with-actual-alpha-prefix-composition-candidate-drift-query-hit-bound-from-trace-indices}$
assumes wf : $\text{staged-budget-wellformed}$ budgets
and controlled : $\text{staged-adversary-controlled}$ $\text{budgets } A$
and trace-frac :
 $\bigwedge i \text{ prefix } \text{prefix-state}.$
 $i < \text{rounds} \implies$
 $\text{Some } ((\text{prefix}, \text{prefix-state}), \text{prefix-state}) \in$
 set-dist
 $(\text{execute } (\text{checked-staged-query-prefix-with-state } A \ i)$
 $\text{adversary-initial-state}) \implies$
 nnreal
 $(\text{card}$
 $(\text{staged-query-prefix-prefix-authenticated-trace-indices } \text{prefix}$

$\text{prefix-state})) /$
 $\text{nnreal (card query-sample-space)} \leq \text{query-error-bound}$
and *path-bound*:
 $\bigwedge i. i < \text{rounds} \implies$
 wp-event
 $(\text{checked-staged-security-experiment-with-query-prefix-data-state } A \ i)$
 $(\text{checked-staged-security-with-query-prefix-current-path-output-hit-at } i)$
 $\text{adversary-initial-state} \leq Q \ i$
shows
 wp-event
 $(\text{checked-staged-security-experiment-with-actual-alpha-prefix-data-state } A)$
 $\text{checked-staged-security-with-actual-alpha-prefix-composition-candidate-drift-query-hit}$
 $\text{adversary-initial-state} \leq$
 $(\sum i < \text{rounds}.$
 $\text{staged-phase-relation-error size}$
 $(\text{staged-query-search-queries budgets } i + 1) +$
 $\text{query-error-bound} + Q \ i)$
proof (rule
 $\text{checked-staged-security-with-actual-alpha-prefix-composition-candidate-drift-query-hit-bound-from-query-prefix-curr}$
 $[\text{OF wf controlled}])$
fix *i*
assume *i-bound*: $i < \text{rounds}$
have *prefix-hit-bound*:
 wp-event
 $(\text{checked-staged-security-experiment-with-query-prefix-data-state } A \ i)$
 $(\text{checked-staged-security-with-query-prefix-current-prefix-authenticated-hit-at } i)$
 $\text{adversary-initial-state} \leq$
 $\text{staged-phase-relation-error size}$
 $(\text{staged-query-search-queries budgets } i + 1) + \text{query-error-bound}$
by (rule
 $\text{checked-staged-security-with-query-prefix-current-prefix-authenticated-hit-at-bound-from-trace-indices}$
 $[\text{OF wf controlled } i\text{-bound}])$
 $(\text{use trace-frac}[\text{OF } i\text{-bound}] \text{ in blast})$
show
 wp-event
 $(\text{checked-staged-security-experiment-with-query-prefix-data-state } A \ i)$
 $(\text{checked-staged-security-with-query-prefix-current-query-partial-opening-hit-at } i)$
 $\text{adversary-initial-state} \leq$
 $\text{staged-phase-relation-error size}$
 $(\text{staged-query-search-queries budgets } i + 1) +$
 $\text{query-error-bound} + Q \ i$
by (rule
 $\text{checked-staged-security-with-query-prefix-current-query-partial-opening-hit-at-bound-from-prefix-and-path}$
 $[\text{OF wf controlled } i\text{-bound } \text{prefix-hit-bound } \text{path-bound}[\text{OF } i\text{-bound}]])$
qed

lemma *checked-staged-security-with-data-state-composition-bad-bound-from-trace-indices-path-and-budgets*:
assumes *wf*: *staged-budget-wellformed budgets*
and *controlled*: *staged-adversary-controlled budgets A*

and *trace-frac*:
 $\bigwedge i$ *prefix* *prefix-state*.
 $i < \text{rounds} \implies$
Some ((*prefix*, *prefix-state*), *prefix-state*) $\in$
set-dist
(*execute* (*checked-staged-query-prefix-with-state* *A* *i*)
adversary-initial-state) $\implies$
nnreal
(*card*
(*staged-query-prefix-prefix-authenticated-trace-indices* *prefix*
prefix-state)) /
nnreal (*card* *query-sample-space*) $\leq$ *query-error-bound*
and *path-bound*:
 $\bigwedge i. i < \text{rounds} \implies$
wp-event
(*checked-staged-security-experiment-with-query-prefix-data-state* *A* *i*)
(*checked-staged-security-with-query-prefix-current-path-output-hit-at* *i*)
adversary-initial-state $\leq$ *Q* *i*
shows
wp-event (*checked-staged-security-experiment-with-data-state* *A*)
(*staged-security-with-data-state-verifier-event* *composition-bad*)
adversary-initial-state $\leq$
composition-error-bound +
staged-phase-relation-error *size*
(*staged-alpha-search-queries* *budgets* 0) +
hash-collision-budget-value 0
(*staged-alpha-search-queries* *budgets* 0) +
staged-phase-relation-error *size*
(*staged-alpha-search-queries* *budgets* 0) +
($\sum i < \text{rounds}.$
staged-phase-relation-error *size*
(*staged-query-search-queries* *budgets* *i* + 1) +
query-error-bound + *Q* *i*) +
hash-collision-budget-value 0
(*staged-attacker-query-budget* *budgets* +
staged-challenge-query-budget + *verifier-hash-query-budget*)
proof –
let ?*Q* =
($\sum i < \text{rounds}.$
staged-phase-relation-error *size*
(*staged-query-search-queries* *budgets* *i* + 1) +
query-error-bound + *Q* *i*)
have *query-bound*:
wp-event
(*checked-staged-security-experiment-with-actual-alpha-prefix-data-state* *A*)
checked-staged-security-with-actual-alpha-prefix-composition-candidate-drift-query-hit
adversary-initial-state $\leq$?*Q*
by (*rule*
checked-staged-security-with-actual-alpha-prefix-composition-candidate-drift-query-hit-bound-from-trace-indices-a

$[OF \text{ wf controlled trace-frac path-bound}]$)
have *without-bound*:
wp-event
 $(\text{checked-staged-security-experiment-with-actual-alpha-prefix-data-state } A)$
 $\text{checked-staged-security-with-actual-alpha-prefix-composition-candidate-drift-without-query-hit}$
 $\text{adversary-initial-state} \leq 0$
by (rule
 $\text{checked-staged-security-with-actual-alpha-prefix-composition-candidate-drift-without-query-hit-bound}$
 $[OF \text{ wf controlled}]$)
have *drift-bound*:
wp-event
 $(\text{checked-staged-security-experiment-with-actual-alpha-prefix-data-state } A)$
 $\text{checked-staged-security-with-actual-alpha-prefix-composition-candidate-drift}$
 $\text{adversary-initial-state} \leq ?Q + 0$
by (rule
 $\text{checked-staged-security-with-actual-alpha-prefix-composition-candidate-drift-bound-from-query-and-without-query}$
 $[OF \text{ query-bound without-bound}]$)
have *comp-bound*:
wp-event $(\text{checked-staged-security-experiment-with-data-state } A)$
 $(\text{staged-security-with-data-state-verifier-event composition-bad})$
 $\text{adversary-initial-state} \leq$
 $\text{composition-error-bound} +$
 $\text{staged-phase-relation-error size}$
 $(\text{staged-alpha-search-queries budgets } 0) +$
 $\text{hash-collision-budget-value } 0$
 $(\text{staged-alpha-search-queries budgets } 0) +$
 $\text{staged-phase-relation-error size}$
 $(\text{staged-alpha-search-queries budgets } 0) +$
 $(?Q + 0) +$
 $\text{hash-collision-budget-value } 0$
 $(\text{staged-attacker-query-budget budgets} +$
 $\text{staged-challenge-query-budget} + \text{verifier-hash-query-budget})$
by (rule
 $\text{checked-staged-security-with-data-state-composition-bad-bound-from-composition-candidate-drift-and-budgets}$
 $[OF \text{ wf controlled drift-bound}]$)
then show *?thesis*
by *simp*
qed

lemma *checked-staged-security-with-actual-alpha-prefix-composition-candidate-drift-query-hit-imp-data-state-partial-*
assumes *wf: staged-budget-wellformed budgets*
and *controlled: staged-adversary-controlled budgets A*
and *support:*
 $\text{Some } (((((\text{alpha-prefix}, \text{alpha-prefix-state}), \text{data}), \text{attacker-state}),$
 $\text{result}), \text{final-state}) \in$
 set-dist
 $(\text{execute}$
 $(\text{checked-staged-security-experiment-with-actual-alpha-prefix-data-state}$

```

      A)
      adversary-initial-state)
and hit:
  checked-staged-security-with-actual-alpha-prefix-composition-candidate-drift-query-hit
    (Some (((alpha-prefix, alpha-prefix-state), data), attacker-state),
      result), final-state))
shows
  staged-security-with-data-state-query-partial-opening-hit
    (Some (((data, attacker-state), result), final-state))
proof –
let ?s =
  verifier-state-from-adversary attacker-state
    (staged-proof-transcript data)
have data-support:
  Some (((data, attacker-state), result), final-state) ∈
    set-dist
      (execute (checked-staged-security-experiment-with-data-state A)
        adversary-initial-state)
by (rule
  checked-staged-security-with-actual-alpha-prefix-support-imp-data-state-support
    [OF support])
from checked-staged-security-experiment-with-data-state-outcomeE
  [OF data-support]
obtain builder verifier where
  builder:
    Some (data, attacker-state) ∈
      set-dist
        (execute (checked-staged-transcript-program A)
          adversary-initial-state)
and verifier:
    Some (result, final-state) ∈ set-dist (execute verify-monad ?s)
by blast
have header:
  verifier-header-transcript ?s
    (staged-trace-root data)
    (staged-trace-fri-roots data)
    (staged-trace-final data)
    (staged-alphas data)
    (staged-degree data)
    (staged-composition-fri-roots data)
    (staged-composition-final data)
    (List.concat (staged-query-chunks data))
by (rule checked-staged-transcript-program-outcome-header-transcript
  [OF wf controlled builder])
show ?thesis
proof (rule
  checked-staged-security-with-actual-alpha-prefix-composition-candidate-drift-query-hit-current-openingsE
    [OF wf controlled support hit])
fix s trace-table composition-table query-idxs f i raw trace-openings

```

```

    composition-openings trace-openingss composition-openingsss
assume s-def:
    s = verifier-state-from-adversary attacker-state
      (staged-proof-transcript data)
and i-bound: i < rounds
and final-lookup:
    fmllookup (HashMap final-state)
      (QueryIndexChallenge i
        (state-after-query-chunks
          (staged-query-start-hash data)
          (staged-query-chunks data) i)) =
      Some raw
and idx-eq: index (to-nat raw) = query-idxs ! i
and comp-nonempty: staged-composition-fri-roots data ≠ []
and trace-auth:
    partial-authenticated-table (staged-trace-root data)
      (scale * clength) trace-openings final-state
and comp-auth:
    partial-authenticated-table
      (hd (staged-composition-fri-roots data)) (scale * clength)
      composition-openings final-state
and consistent:
    partial-query-openings-consistent trace-openings
      composition-openings (staged-alphas data) (query-idxs ! i)
have consistent-raw:
    partial-query-openings-consistent trace-openings composition-openings
      (staged-alphas data) (index (to-nat raw))
    using consistent idx-eq by simp
have idx-sample: index (to-nat raw) ∈ query-sample-space
    using consistent-raw
    unfolding partial-query-openings-consistent-def by simp
have witness:
    (trace-openings, composition-openings) ∈
      query-header-supported-partial-opening-witnesses ?s
      (staged-trace-root data)
      (staged-trace-fri-roots data)
      (staged-trace-final data)
      (staged-alphas data)
      (staged-degree data)
      (staged-composition-fri-roots data)
      (staged-composition-final data)
    unfolding query-header-supported-partial-opening-witnesses-def
    using comp-nonempty verifier trace-auth comp-auth header
    by auto
have success:
    index (to-nat raw) ∈
      query-header-supported-partial-opening-success-indices-at ?s
      (staged-trace-root data)
      (staged-trace-fri-roots data)

```

```

    (staged-trace-final data)
    (staged-alphas data)
    (staged-degree data)
    (staged-composition-fri-roots data)
    (staged-composition-final data)
  i
unfolding query-header-supported-partial-opening-success-indices-at-def
by (intro CollectI conjI exI[of - trace-openings]
    exI[of - composition-openings])
    (use idx-sample witness consistent-row in simp-all)
have round-hit:
  staged-security-with-data-state-query-partial-opening-hit-at i
    (Some (((data, attacker-state), result), final-state))
unfolding staged-security-with-data-state-query-partial-opening-hit-at-def
  Let-def
using i-bound final-lookup success by simp
show
  staged-security-with-data-state-query-partial-opening-hit
    (Some (((data, attacker-state), result), final-state))
by (rule staged-security-with-data-state-query-partial-opening-hitI
    [OF i-bound round-hit])
qed
qed

lemma checked-staged-security-with-actual-alpha-prefix-composition-candidate-drift-query-hit-le-data-state-partial-op
assumes wf: staged-budget-wellformed budgets
and controlled: staged-adversary-controlled budgets A
shows
  wp-event
    (checked-staged-security-experiment-with-actual-alpha-prefix-data-state A)
    checked-staged-security-with-actual-alpha-prefix-composition-candidate-drift-query-hit
    adversary-initial-state ≤
  wp-event
    (checked-staged-security-experiment-with-data-state A)
    staged-security-with-data-state-query-partial-opening-hit
    adversary-initial-state
proof -
let ?project =
  λx. ((snd (fst (fst x)), snd (fst x)), snd x)
let ?projected =
  λout. case out of
    None ⇒ False
  | Some (packed, t) ⇒
    staged-security-with-data-state-query-partial-opening-hit
      (Some (?project packed, t))
have event-le:
  wp-event
    (checked-staged-security-experiment-with-actual-alpha-prefix-data-state A)
    checked-staged-security-with-actual-alpha-prefix-composition-candidate-drift-query-hit

```

```

    adversary-initial-state ≤
wp-event
  (checked-staged-security-experiment-with-actual-alpha-prefix-data-state A)
  ?projected adversary-initial-state
proof (rule wp-event-mono-on-support)
fix out
assume support:
  out ∈
    set-dist
    (execute
      (checked-staged-security-experiment-with-actual-alpha-prefix-data-state
        A)
      adversary-initial-state)
and hit:
  checked-staged-security-with-actual-alpha-prefix-composition-candidate-drift-query-hit
  out
show ?projected out
proof (cases out)
case None
then show ?thesis
  using hit
  unfolding
    checked-staged-security-with-actual-alpha-prefix-composition-candidate-drift-query-hit-def
    checked-staged-security-with-actual-alpha-prefix-composition-candidate-drift-def
    checked-staged-security-with-actual-alpha-prefix-verifier-event-def
  by simp
next
case (Some packed)
then obtain alpha-prefix alpha-prefix-state data attacker-state result
  final-state where out-eq:
    out =
      Some (((((alpha-prefix, alpha-prefix-state), data),
        attacker-state), result), final-state)
  by (cases packed, auto split: prod.splits)
have partial-hit:
  staged-security-with-data-state-query-partial-opening-hit
  (Some (((data, attacker-state), result), final-state))
  by (rule
    checked-staged-security-with-actual-alpha-prefix-composition-candidate-drift-query-hit-imp-data-state-partial-imp
    [OF wf controlled])
  (use support hit out-eq in simp-all)
show ?thesis
  unfolding out-eq
  using partial-hit by simp
qed
qed
have projection:
  wp-event
  (checked-staged-security-experiment-with-actual-alpha-prefix-data-state A)

```



```

    ?projected adversary-initial-state =
wp-event
  (checked-staged-security-experiment-with-data-state A)
  staged-security-with-data-state-query-partial-opening-hit
  adversary-initial-state
proof –
  have
    wp-event (checked-staged-security-experiment-with-data-state A)
      staged-security-with-data-state-query-partial-opening-hit
      adversary-initial-state =
    wp-event
      (checked-staged-security-experiment-with-actual-alpha-prefix-data-state A
         $\gg (\lambda x. \text{return } (?project\ x))$ )
      staged-security-with-data-state-query-partial-opening-hit
      adversary-initial-state
  using
    checked-staged-security-experiment-with-actual-alpha-prefix-data-state-projection
    [of A]
  by simp
also have ... =
  wp-event
    (checked-staged-security-experiment-with-actual-alpha-prefix-data-state A)
    ?projected adversary-initial-state
  by (subst wp-event-bind-return-map)
    (simp add: staged-security-with-data-state-query-partial-opening-hit-def
      staged-security-with-data-state-query-partial-opening-hit-at-def)
finally show ?thesis
  by simp
qed
show ?thesis
  by (rule order-trans[OF event-le])
    (simp add: projection)
qed

lemma checked-staged-security-with-actual-alpha-prefix-composition-candidate-drift-query-hit-bound-from-data-state-
assumes wf: staged-budget-wellformed budgets
and controlled: staged-adversary-controlled budgets A
and partial-bound:
  wp-event
    (checked-staged-security-experiment-with-data-state A)
    staged-security-with-data-state-query-partial-opening-hit
    adversary-initial-state  $\leq (Q::prob)$ 
shows
  wp-event
    (checked-staged-security-experiment-with-actual-alpha-prefix-data-state A)
    checked-staged-security-with-actual-alpha-prefix-composition-candidate-drift-query-hit
    adversary-initial-state  $\leq Q$ 
by (rule order.trans
  [OF

```

```

    checked-staged-security-with-actual-alpha-prefix-composition-candidate-drift-query-hit-le-data-state-partial-openin
    partial-bound])
  (use wf controlled in simp-all)

end

end

theory Staged-Security-Experiment-Composition-Query-Binding-Gap
imports
  Staged-Security-Experiment-Composition-Query-Header-Rounds
  Staged-Security-Experiment-Composition-Query-Current
begin

Candidate-binding gap for the query-prefix composition route.
The dynamic header-supported query target is small only for candidates that are already
visible in the prefix oracle state. This layer isolates the remaining proof obligation: if a
later authenticated candidate pair is not in that prefix candidate set, the gap must be
charged to Merkle/collision side events in the next proof layer.

context soundness
begin

lemma checked-staged-query-prefix-candidate-pair-hit-imp-header-supported-target-hit:
assumes candidate:
  (trace-table, composition-table) ∈
  query-header-supported-partial-table-candidates prefix-state
  (sqp-trace-root prefix)
  (sqp-trace-fri-roots prefix)
  (sqp-trace-final prefix)
  (sqp-alphas prefix)
  (sqp-degree prefix)
  (sqp-composition-fri-roots prefix)
  (sqp-composition-final prefix)
and hit:
  checked-staged-query-prefix-dynamic-index-hit
  (staged-query-prefix-candidate-pair-query-target-from-prefix
   trace-table composition-table)
  (Some (((prefix, prefix-state), raw), raw-state))
shows
  checked-staged-query-prefix-dynamic-index-hit
  staged-query-prefix-header-supported-query-target
  (Some (((prefix, prefix-state), raw), raw-state))
proof –
have subset:
  staged-query-prefix-candidate-pair-query-target-from-prefix
  trace-table composition-table prefix prefix-state ⊆
  staged-query-prefix-header-supported-query-target prefix prefix-state
unfolding

```

```

    staged-query-prefix-candidate-pair-query-target-from-prefix-def
    staged-query-prefix-header-supported-query-target-def
    query-header-supported-partial-union-good-sets-def
    using candidate query-sampling-success-space-subset by auto
    show ?thesis
    using hit subset
    unfolding checked-staged-query-prefix-dynamic-index-hit-def
    by auto
  qed

```

definition *query-header-supported-single-round-partial-table-candidates*

```

:: ('f, 'a) protocol-channel-scheme =>
  'f => 'f list => 'f => 'f list => 'f =>
  'f list => 'f => nat => ('f list × 'f list) set

```

where

```

query-header-supported-single-round-partial-table-candidates s fr
  f-fri-roots f-final as dg composition-fri-roots final i =
  {(trace-table, composition-table).
    composition-fri-roots ≠ [] ∧
    (∃ trace-openings composition-openings rest.
      partial-authenticated-table fr (scale * clength)
        trace-openings s ∧
      partial-authenticated-table (hd composition-fri-roots)
        (scale * clength) composition-openings s ∧
      partial-trace-table-candidate trace-table
        ((replicate rounds []) [i := trace-openings]) ∧
      partial-composition-table-candidate composition-table
        ((replicate rounds []) [i := composition-openings]) ∧
      verifier-header-transcript s fr f-fri-roots f-final as dg
        composition-fri-roots final rest)}}

```

lemma *query-header-supported-single-round-partial-table-candidatesI:*

```

assumes comp-nonempty: composition-fri-roots ≠ []
and trace-auth:
  partial-authenticated-table fr (scale * clength)
    trace-openings s
and comp-auth:
  partial-authenticated-table (hd composition-fri-roots)
    (scale * clength) composition-openings s
and trace-candidate:
  partial-trace-table-candidate trace-table
    ((replicate rounds []) [i := trace-openings])
and comp-candidate:
  partial-composition-table-candidate composition-table
    ((replicate rounds []) [i := composition-openings])
and header:
  verifier-header-transcript s fr f-fri-roots f-final as dg
    composition-fri-roots final rest
shows

```

$(\text{trace-table}, \text{composition-table}) \in$
 $\text{query-header-supported-single-round-partial-table-candidates } s \text{ fr}$
 $f\text{-fri-roots } f\text{-final as dg composition-fri-roots final } i$
unfolding $\text{query-header-supported-single-round-partial-table-candidates-def}$
using assms by blast

lemma $\text{query-header-supported-single-round-partial-table-candidate-notin-contradicts:}$

assumes notin:
 $(\text{trace-table}, \text{composition-table}) \notin$
 $\text{query-header-supported-single-round-partial-table-candidates } s \text{ fr}$
 $f\text{-fri-roots } f\text{-final as dg composition-fri-roots final } i$
and $\text{comp-nonempty: composition-fri-roots} \neq []$
and trace-auth:
 $\text{partial-authenticated-table fr (scale * clength)}$
 $\text{trace-openings } s$
and comp-auth:
 $\text{partial-authenticated-table (hd composition-fri-roots)}$
 $(\text{scale * clength}) \text{ composition-openings } s$
and trace-candidate:
 $\text{partial-trace-table-candidate trace-table}$
 $((\text{replicate rounds } []) [i := \text{trace-openings}])$
and comp-candidate:
 $\text{partial-composition-table-candidate composition-table}$
 $((\text{replicate rounds } []) [i := \text{composition-openings}])$
and header:
 $\text{verifier-header-transcript } s \text{ fr } f\text{-fri-roots } f\text{-final as dg}$
 $\text{composition-fri-roots final rest}$
shows False
using notin
 $\text{query-header-supported-single-round-partial-table-candidatesI}$
 $[OF \text{ comp-nonempty trace-auth comp-auth trace-candidate}$
 $\text{comp-candidate header}]$
by contradiction

definition $\text{staged-query-prefix-single-round-header-supported-query-target}$

$:: \text{nat} \Rightarrow 'f \text{ staged-query-prefix-data} \Rightarrow$
 $'f \text{ protocol-channel} \Rightarrow \text{nat set}$

where

$\text{staged-query-prefix-single-round-header-supported-query-target } i \text{ prefix}$
 $\text{prefix-state} =$
 $\{idx.$
 $\exists \text{ trace-table composition-table.}$
 $(\text{trace-table}, \text{composition-table}) \in$
 $\text{query-header-supported-single-round-partial-table-candidates}$
 prefix-state
 $(\text{sqp-trace-root prefix})$
 $(\text{sqp-trace-fri-roots prefix})$
 $(\text{sqp-trace-final prefix})$
 $(\text{sqp-alphas prefix})$

(sqp-degree prefix)
 (sqp-composition-fri-roots prefix)
 (sqp-composition-final prefix)
 i ∧
 idx ∈
 staged-query-prefix-candidate-pair-query-target-from-prefix
 trace-table composition-table prefix prefix-state}

lemma *checked-staged-query-prefix-candidate-pair-hit-imp-single-round-header-target-hit:*

assumes *candidate:*

(trace-table, composition-table) ∈
 query-header-supported-single-round-partial-table-candidates
 prefix-state
 (sqp-trace-root prefix)
 (sqp-trace-fri-roots prefix)
 (sqp-trace-final prefix)
 (sqp-alphas prefix)
 (sqp-degree prefix)
 (sqp-composition-fri-roots prefix)
 (sqp-composition-final prefix)
 i

and *hit:*

checked-staged-query-prefix-dynamic-index-hit
 (staged-query-prefix-candidate-pair-query-target-from-prefix
 trace-table composition-table)
 (Some (((prefix, prefix-state), raw), raw-state))

shows

checked-staged-query-prefix-dynamic-index-hit
 (staged-query-prefix-single-round-header-supported-query-target i)
 (Some (((prefix, prefix-state), raw), raw-state))

proof —

have *idx-in:*

index (to-nat raw) ∈
 staged-query-prefix-candidate-pair-query-target-from-prefix
 trace-table composition-table prefix prefix-state

using *hit* **unfolding** *checked-staged-query-prefix-dynamic-index-hit-def*

by *simp*

have

index (to-nat raw) ∈
 staged-query-prefix-single-round-header-supported-query-target i
 prefix prefix-state

unfolding *staged-query-prefix-single-round-header-supported-query-target-def*

by (intro CollectI exI[of - trace-table])

exI[of - composition-table])

(use candidate *idx-in* in *simp*)

then show *?thesis*

unfolding *checked-staged-query-prefix-dynamic-index-hit-def*

by *simp*

qed

lemma *query-header-supported-partial-table-candidatesI-from-partials:*
assumes *comp-nonempty: composition-fri-roots $\neq \square$*
and *outcome:*
Some (result, final-state) $\in$ set-dist (execute verify-monad s)
and *trace-partial:*
accepted-with-partial-trace-openings s
(Some (result, final-state)) fr trace-query-idxs trace-openings
and *trace-candidate:*
partial-trace-table-candidate trace-table trace-openings
and *comp-partial:*
accepted-with-partial-composition-openings s
(Some (result, final-state)) (hd composition-fri-roots)
composition-query-idxs composition-openings
and *comp-candidate:*
partial-composition-table-candidate composition-table
composition-openings
and *header:*
verifier-header-transcript s fr f-fri-roots f-final as dg
composition-fri-roots final rest
shows
(trace-table, composition-table) $\in$
query-header-supported-partial-table-candidates s fr f-fri-roots
f-final as dg composition-fri-roots final
proof –
have *body:*
composition-fri-roots $\neq \square$ $\wedge$
($\exists$ out trace-query-idxs composition-query-idxs trace-openings
composition-openings rest.
out $\in$ set-dist (execute verify-monad s) $\wedge$
accepted-with-partial-trace-openings s out fr trace-query-idxs
trace-openings $\wedge$
partial-trace-table-candidate trace-table trace-openings $\wedge$
accepted-with-partial-composition-openings s out
(hd composition-fri-roots) composition-query-idxs
composition-openings $\wedge$
partial-composition-table-candidate composition-table
composition-openings $\wedge$
verifier-header-transcript s fr f-fri-roots f-final as dg
composition-fri-roots final rest)
using *comp-nonempty outcome trace-partial trace-candidate comp-partial*
comp-candidate header
by *blast*
show *?thesis*
unfolding *query-header-supported-partial-table-candidates-def*
using *body by simp*
qed

lemma *query-header-supported-partial-table-candidate-notin-contradicts-partials:*

assumes *notin*:
 (trace-table, composition-table) $\notin$
 query-header-supported-partial-table-candidates *s* fr *f-fri-roots*
f-final as *dg* composition-fri-roots *final*
and *comp-nonempty*: composition-fri-roots $\neq \square$
and *outcome*:
 Some (result, final-state) $\in$ set-dist (execute verify-monad *s*)
and *trace-partial*:
 accepted-with-partial-trace-openings *s*
 (Some (result, final-state)) fr trace-query-idxs trace-openings
and *trace-candidate*:
 partial-trace-table-candidate trace-table trace-openings
and *comp-partial*:
 accepted-with-partial-composition-openings *s*
 (Some (result, final-state)) (hd composition-fri-roots)
 composition-query-idxs composition-openings
and *comp-candidate*:
 partial-composition-table-candidate composition-table
 composition-openings
and *header*:
 verifier-header-transcript *s* fr *f-fri-roots* *f-final* as *dg*
 composition-fri-roots *final* *rest*
shows *False*
using *notin*
 query-header-supported-partial-table-candidatesI-from-partials
 [OF *comp-nonempty* *outcome* *trace-partial* *trace-candidate* *comp-partial*
comp-candidate *header*]
by *contradiction*

lemma *query-header-supported-partial-table-candidate-notin-no-accepted-partials*:

assumes *notin*:
 (trace-table, composition-table) $\notin$
 query-header-supported-partial-table-candidates *s* fr *f-fri-roots*
f-final as *dg* composition-fri-roots *final*
and *comp-nonempty*: composition-fri-roots $\neq \square$
and *trace-candidate*:
 partial-trace-table-candidate trace-table trace-openings
and *comp-candidate*:
 partial-composition-table-candidate composition-table
 composition-openings
shows
 $\neg (\exists \text{result final-state trace-query-idxs composition-query-idxs rest.}$
 Some (result, final-state) $\in$ set-dist (execute verify-monad *s*) $\wedge$
 accepted-with-partial-trace-openings *s*
 (Some (result, final-state)) fr trace-query-idxs trace-openings $\wedge$
 accepted-with-partial-composition-openings *s*
 (Some (result, final-state)) (hd composition-fri-roots)
 composition-query-idxs composition-openings $\wedge$
 verifier-header-transcript *s* fr *f-fri-roots* *f-final* as *dg*

composition-fri-roots final rest)

proof

assume

$\exists$ *result final-state trace-query-idxs composition-query-idxs rest.*
Some (result, final-state) ∈ set-dist (execute verify-monad s) ∧
accepted-with-partial-trace-openings s
(Some (result, final-state)) fr trace-query-idxs trace-openings ∧
accepted-with-partial-composition-openings s
(Some (result, final-state)) (hd composition-fri-roots)
composition-query-idxs composition-openings ∧
verifier-header-transcript s fr f-fri-roots f-final as dg
composition-fri-roots final rest

then obtain *result final-state trace-query-idxs composition-query-idxs rest*

where *outcome:*
Some (result, final-state) ∈ set-dist (execute verify-monad s)
and *trace-partial:*
accepted-with-partial-trace-openings s
(Some (result, final-state)) fr trace-query-idxs trace-openings
and *comp-partial:*
accepted-with-partial-composition-openings s
(Some (result, final-state)) (hd composition-fri-roots)
composition-query-idxs composition-openings
and *header:*
verifier-header-transcript s fr f-fri-roots f-final as dg
composition-fri-roots final rest

by *blast*

show *False*

by (*rule query-header-supported-partial-table-candidate-notin-contradicts-partials*
[OF notin comp-nonempty outcome trace-partial trace-candidate
comp-partial comp-candidate header])

qed

lemma *accepted-with-partial-trace-openings-single-round-update-other-empty:*

assumes *partial:*

accepted-with-partial-trace-openings s out fr query-idxs
((replicate rounds []) [i := trace-openings])

and *j-bound: j < rounds*

and *j-ne: j ≠ i*

shows *powers-scaled (query-idxs ! j) = []*

proof —

have *indices:*

map opening-index (((replicate rounds []) [i := trace-openings]) ! j) =
powers-scaled (query-idxs ! j)

using *accepted-with-partial-trace-openings-shapes(3)[OF partial j-bound]*

.

have *((replicate rounds []) [i := trace-openings]) ! j = []*

using *j-bound j-ne* **by** *simp*

then show *?thesis*

using *indices* **by** *simp*

qed

lemma *accepted-with-partial-composition-openings-single-round-update-other-false:*

assumes *partial:*

accepted-with-partial-composition-openings s out composition-root

query-idxs ((replicate rounds []) [i := composition-openings])

and *j-bound: j < rounds*

and *j-ne: j ≠ i*

shows *False*

proof –

have *indices:*

map opening-index (((replicate rounds []) [i := composition-openings]) ! j) =

*[query-idxs ! j, fri-sibling-index (scale * clength) (query-idxs ! j)]*

using

accepted-with-partial-composition-openings-shapes(3)

[OF partial j-bound]

.

have *((replicate rounds []) [i := composition-openings]) ! j = []*

using *j-bound j-ne* **by** *simp*

then show *False*

using *indices* **by** *simp*

qed

lemma *query-header-supported-partial-opening-witnesses-pullback-or-path-output:*

assumes *witness:*

(trace-openings, composition-openings) ∈

query-header-supported-partial-opening-witnesses s fr f-fri-roots

f-final as dg composition-fri-roots final

shows

*(partial-authenticated-table fr (scale * clength) trace-openings s ∧*

*partial-authenticated-table (hd composition-fri-roots) (scale * clength)*

composition-openings s) ∨

(∃ witness-state. ∃ opn ∈ set trace-openings.

hash-map-new-output-hit

*(merkle-path-target-roots witness-state fr (scale * clength)*

(opening-index opn) (opening-value opn) (opening-path opn))

s witness-state) ∨

(∃ witness-state. ∃ opn ∈ set composition-openings.

hash-map-new-output-hit

(merkle-path-target-roots witness-state (hd composition-fri-roots)

*(scale * clength) (opening-index opn) (opening-value opn)*

(opening-path opn))

s witness-state)

proof –

from *witness* **obtain** *result witness-state rest* **where** *outcome:*

Some (result, witness-state) ∈ set-dist (execute verify-monad s)

and *trace-table:*

*partial-authenticated-table fr (scale * clength) trace-openings*

witness-state

and *comp-table*:
partial-authenticated-table (*hd composition-fri-roots*)
*(scale * clength) composition-openings witness-state*
by (*elim query-header-supported-partial-opening-witnessesE*)
have *ext*: *s ≤ witness-state*
by (*rule verify-monad-hash-extends[OF outcome]*)
have *trace-pull*:
*partial-authenticated-table fr (scale * clength) trace-openings s ∨*
(∃ opn ∈ set trace-openings.
hash-map-new-output-hit
*(merkle-path-target-roots witness-state fr (scale * clength)*
(opening-index opn) (opening-value opn) (opening-path opn))
s witness-state)
by (*rule partial-authenticated-table-pullback-or-new-output-hit*
[OF ext trace-table])
have *comp-pull*:
*partial-authenticated-table (hd composition-fri-roots) (scale * clength)*
composition-openings s ∨
(∃ opn ∈ set composition-openings.
hash-map-new-output-hit
(merkle-path-target-roots witness-state (hd composition-fri-roots)
*(scale * clength) (opening-index opn) (opening-value opn)*
(opening-path opn))
s witness-state)
by (*rule partial-authenticated-table-pullback-or-new-output-hit*
[OF ext comp-table])
from *trace-pull comp-pull* **show** *?thesis* **by** *blast*
qed

lemma *query-header-supported-single-round-candidate-from-witness-or-path-output*:

assumes *witness*:
(trace-openings, composition-openings) ∈
query-header-supported-partial-opening-witnesses s fr f-fri-roots
f-final as dg composition-fri-roots final
and *trace-candidate*:
partial-trace-table-candidate trace-table
((replicate rounds []) [i := trace-openings])
and *comp-candidate*:
partial-composition-table-candidate composition-table
((replicate rounds []) [i := composition-openings])
shows
(trace-table, composition-table) ∈
query-header-supported-single-round-partial-table-candidates s fr
f-fri-roots f-final as dg composition-fri-roots final i ∨
(∃ witness-state. ∃ opn ∈ set trace-openings.
hash-map-new-output-hit
*(merkle-path-target-roots witness-state fr (scale * clength)*
(opening-index opn) (opening-value opn) (opening-path opn))
s witness-state) ∨

```

( $\exists$  witness-state.  $\exists$  opn  $\in$  set composition-openings.
  hash-map-new-output-hit
    (merkle-path-target-roots witness-state (hd composition-fri-roots)
      (scale * clength) (opening-index opn) (opening-value opn)
      (opening-path opn))
    s witness-state)
proof –
from witness obtain result witness-state rest where comp-nonempty:
  composition-fri-roots  $\neq$  []
and header:
  verifier-header-transcript s fr f-fri-roots f-final as dg
    composition-fri-roots final rest
by (elim query-header-supported-partial-opening-witnessesE)
have pull:
  (partial-authenticated-table fr (scale * clength) trace-openings s  $\wedge$ 
    partial-authenticated-table (hd composition-fri-roots) (scale * clength)
      composition-openings s)  $\vee$ 
  ( $\exists$  witness-state.  $\exists$  opn  $\in$  set trace-openings.
    hash-map-new-output-hit
      (merkle-path-target-roots witness-state fr (scale * clength)
        (opening-index opn) (opening-value opn) (opening-path opn))
      s witness-state)  $\vee$ 
  ( $\exists$  witness-state.  $\exists$  opn  $\in$  set composition-openings.
    hash-map-new-output-hit
      (merkle-path-target-roots witness-state (hd composition-fri-roots)
        (scale * clength) (opening-index opn) (opening-value opn)
        (opening-path opn))
      s witness-state)
by (rule query-header-supported-partial-opening-witnesses-pullback-or-path-output
  [OF witness])
then show ?thesis
proof
assume auths:
  partial-authenticated-table fr (scale * clength) trace-openings s  $\wedge$ 
    partial-authenticated-table (hd composition-fri-roots) (scale * clength)
      composition-openings s
have candidate:
  (trace-table, composition-table)  $\in$ 
    query-header-supported-single-round-partial-table-candidates s fr
      f-fri-roots f-final as dg composition-fri-roots final i
by (rule query-header-supported-single-round-partial-table-candidatesI
  [OF comp-nonempty - - trace-candidate comp-candidate header])
  (use auths in simp-all)
then show ?thesis by simp
next
assume
  ( $\exists$  witness-state.  $\exists$  opn  $\in$  set trace-openings.
    hash-map-new-output-hit
      (merkle-path-target-roots witness-state fr (scale * clength)

```

```

      (opening-index opn) (opening-value opn) (opening-path opn))
    s witness-state) ∨
  (∃ witness-state. ∃ opn ∈ set composition-openings.
    hash-map-new-output-hit
      (merkle-path-target-roots witness-state (hd composition-fri-roots)
        (scale * clength) (opening-index opn) (opening-value opn)
        (opening-path opn))
      s witness-state)
  then show ?thesis by simp
qed
qed

lemma query-header-supported-single-round-candidate-notin-imp-path-output:
  assumes witness:
    (trace-openings, composition-openings) ∈
      query-header-supported-partial-opening-witnesses s fr f-fri-roots
      f-final as dg composition-fri-roots final
  and trace-candidate:
    partial-trace-table-candidate trace-table
      ((replicate rounds []) [i := trace-openings])
  and comp-candidate:
    partial-composition-table-candidate composition-table
      ((replicate rounds []) [i := composition-openings])
  and notin:
    (trace-table, composition-table) ∉
      query-header-supported-single-round-partial-table-candidates s fr
      f-fri-roots f-final as dg composition-fri-roots final i
  shows
    (∃ witness-state. ∃ opn ∈ set trace-openings.
      hash-map-new-output-hit
        (merkle-path-target-roots witness-state fr (scale * clength)
          (opening-index opn) (opening-value opn) (opening-path opn))
        s witness-state) ∨
    (∃ witness-state. ∃ opn ∈ set composition-openings.
      hash-map-new-output-hit
        (merkle-path-target-roots witness-state (hd composition-fri-roots)
          (scale * clength) (opening-index opn) (opening-value opn)
          (opening-path opn))
        s witness-state)
  using query-header-supported-single-round-candidate-from-witness-or-path-output
    [OF witness trace-candidate comp-candidate] notin
  by blast

definition checked-staged-security-with-query-prefix-header-supported-query-target-hit-at
where
  checked-staged-security-with-query-prefix-header-supported-query-target-hit-at
    i out ⟷
  (case out of
    None ⇒ False

```

| *Some* (((((*prefix*, *prefix-state*), *raw*), *raw-state*), *continuation*),
final-state) $\Rightarrow$
checked-staged-query-prefix-dynamic-index-hit
staged-query-prefix-header-supported-query-target
(*Some* (((*prefix*, *prefix-state*), *raw*), *raw-state*)))

definition *checked-staged-security-with-query-prefix-candidate-binding-gap-at*
where

checked-staged-security-with-query-prefix-candidate-binding-gap-at
trace-openings composition-openings trace-table composition-table i out
 $\longleftrightarrow$
(*case out of*
None $\Rightarrow$ *False*
| *Some* (((((*prefix*, *prefix-state*), *raw*), *raw-state*),
((*data*, *attacker-state*), *result*)), *final-state*) $\Rightarrow$
checked-staged-security-with-query-prefix-header-authenticated-candidate-opening-hit
trace-openings composition-openings i out $\wedge$
partial-trace-table-candidate trace-table
((*replicate rounds* []) [*i* := *trace-openings*]) $\wedge$
partial-composition-table-candidate composition-table
((*replicate rounds* []) [*i* := *composition-openings*]) $\wedge$
(*trace-table*, *composition-table*) $\notin$
query-header-supported-partial-table-candidates prefix-state
(*sqp-trace-root prefix*)
(*sqp-trace-fri-roots prefix*)
(*sqp-trace-final prefix*)
(*sqp-alphas prefix*)
(*sqp-degree prefix*)
(*sqp-composition-fri-roots prefix*)
(*sqp-composition-final prefix*))

definition *checked-staged-security-with-query-prefix-candidate-binding-gap-side*
where

checked-staged-security-with-query-prefix-candidate-binding-gap-side
trace-openings composition-openings trace-table composition-table i out
 $\longleftrightarrow$
(*case out of*
None $\Rightarrow$ *False*
| *Some* (((((*prefix*, *prefix-state*), *raw*), *raw-state*),
((*data*, *attacker-state*), *result*)), *final-state*) $\Rightarrow$
partial-trace-table-candidate trace-table
((*replicate rounds* []) [*i* := *trace-openings*]) $\wedge$
partial-composition-table-candidate composition-table
((*replicate rounds* []) [*i* := *composition-openings*]) $\wedge$
(*trace-table*, *composition-table*) $\notin$
query-header-supported-partial-table-candidates prefix-state
(*sqp-trace-root prefix*)
(*sqp-trace-fri-roots prefix*)
(*sqp-trace-final prefix*)

(sqp-alphas prefix)
 (sqp-degree prefix)
 (sqp-composition-fri-roots prefix)
 (sqp-composition-final prefix))

definition *checked-staged-security-with-query-prefix-prefix-candidate-binding-gap-at*
where

checked-staged-security-with-query-prefix-prefix-candidate-binding-gap-at
trace-openings composition-openings trace-table composition-table i out
 $\longleftrightarrow$
checked-staged-security-with-query-prefix-candidate-binding-gap-side
trace-openings composition-openings trace-table composition-table i out $\wedge$
checked-staged-security-with-query-prefix-prefix-authenticated-opening-hit
trace-openings composition-openings i out

lemma *checked-staged-security-with-query-prefix-prefix-candidate-binding-gap-imp-current-prefix-authenticated:*
assumes *gap:*

checked-staged-security-with-query-prefix-prefix-candidate-binding-gap-at
trace-openings composition-openings trace-table composition-table i out

shows

checked-staged-security-with-query-prefix-current-prefix-authenticated-hit-at
i out

using *gap*

unfolding

checked-staged-security-with-query-prefix-prefix-candidate-binding-gap-at-def
checked-staged-security-with-query-prefix-current-prefix-authenticated-hit-at-def

by *blast*

lemma *checked-staged-security-with-query-prefix-prefix-candidate-binding-gap-bound-from-dynamic-target:*

assumes *wf: staged-budget-wellformed budgets*

and *controlled: staged-adversary-controlled budgets A*

and *i-bound: i < rounds*

and *prefix-bound:*

wp-event (checked-staged-query-prefix-receive-with-state A i)
(checked-staged-query-prefix-dynamic-index-hit
staged-query-prefix-prefix-authenticated-opening-target)
adversary-initial-state $\leq$ B

shows

wp-event
(checked-staged-security-experiment-with-query-prefix-data-state A i)
(checked-staged-security-with-query-prefix-prefix-candidate-binding-gap-at
trace-openings composition-openings trace-table composition-table i)
adversary-initial-state $\leq$ B

proof —

have *event-le:*

wp-event
(checked-staged-security-experiment-with-query-prefix-data-state A i)
(checked-staged-security-with-query-prefix-prefix-candidate-binding-gap-at
trace-openings composition-openings trace-table composition-table i)

```

    adversary-initial-state ≤
wp-event
  (checked-staged-security-experiment-with-query-prefix-data-state A i)
  (checked-staged-security-with-query-prefix-current-prefix-authenticated-hit-at
    i)
  adversary-initial-state
by (rule wp-event-mono)
  (rule
    checked-staged-security-with-query-prefix-prefix-candidate-binding-gap-imp-current-prefix-authenticated)
have current-bound:
  wp-event
    (checked-staged-security-experiment-with-query-prefix-data-state A i)
    (checked-staged-security-with-query-prefix-current-prefix-authenticated-hit-at
      i)
    adversary-initial-state ≤ B
  by (rule
    checked-staged-security-with-query-prefix-current-prefix-authenticated-hit-at-bound-from-dynamic-target
      [OF wf controlled i-bound prefix-bound])
show ?thesis
  by (rule order-trans[OF event-le current-bound])
qed

lemma checked-staged-security-with-query-prefix-prefix-candidate-binding-gap-bound-from-trace-indices:
  assumes wf: staged-budget-wellformed budgets
  and controlled: staged-adversary-controlled budgets A
  and i-bound: i < rounds
  and trace-frac:
     $\bigwedge \text{prefix prefix-state.}$ 
    Some ((prefix, prefix-state), prefix-state) ∈
      set-dist
        (execute (checked-staged-query-prefix-with-state A i)
          adversary-initial-state)  $\implies$ 
nnreal
  (card
    (staged-query-prefix-prefix-authenticated-trace-indices prefix
      prefix-state)) /
    nnreal (card query-sample-space) ≤ query-error-bound
  shows
    wp-event
      (checked-staged-security-experiment-with-query-prefix-data-state A i)
      (checked-staged-security-with-query-prefix-prefix-candidate-binding-gap-at
        trace-openings composition-openings trace-table composition-table i)
      adversary-initial-state ≤
        staged-phase-relation-error size
          (staged-query-search-queries budgets i + 1) + query-error-bound
proof -
  have current-bound:
    wp-event
      (checked-staged-security-experiment-with-query-prefix-data-state A i)

```

```

    (checked-staged-security-with-query-prefix-current-prefix-authenticated-hit-at
      i)
    adversary-initial-state ≤
    staged-phase-relation-error size
    (staged-query-search-queries budgets i + 1) + query-error-bound
  by (rule
    checked-staged-security-with-query-prefix-current-prefix-authenticated-hit-at-bound-from-trace-indices
    [OF wf controlled i-bound trace-frac])
  have event-le:
    wp-event
    (checked-staged-security-experiment-with-query-prefix-data-state A i)
    (checked-staged-security-with-query-prefix-prefix-candidate-binding-gap-at
      trace-openings composition-openings trace-table composition-table i)
    adversary-initial-state ≤
    wp-event
    (checked-staged-security-experiment-with-query-prefix-data-state A i)
    (checked-staged-security-with-query-prefix-current-prefix-authenticated-hit-at
      i)
    adversary-initial-state
  by (rule wp-event-mono)
  (rule
    checked-staged-security-with-query-prefix-prefix-candidate-binding-gap-imp-current-prefix-authenticated)
  show ?thesis
  by (rule order-trans[OF event-le current-bound])
qed

lemma checked-staged-security-with-query-prefix-candidate-binding-gap-iff-fixed-header-component:
  checked-staged-security-with-query-prefix-candidate-binding-gap-at
    trace-openings composition-openings trace-table composition-table i out
   $\longleftrightarrow$ 
  checked-staged-security-with-query-prefix-fixed-header-component
    (checked-staged-security-with-query-prefix-candidate-binding-gap-side
      trace-openings composition-openings trace-table composition-table i)
    trace-openings composition-openings i out
unfolding
  checked-staged-security-with-query-prefix-candidate-binding-gap-at-def
  checked-staged-security-with-query-prefix-candidate-binding-gap-side-def
  checked-staged-security-with-query-prefix-fixed-header-component-def
by (cases out) (auto split: prod.splits)

lemma checked-staged-security-with-query-prefix-header-authenticated-candidate-opening-hit-imp-header-target-or-bin
assumes hit:
  checked-staged-security-with-query-prefix-header-authenticated-candidate-opening-hit
    trace-openings composition-openings i
  (Some (((((prefix, prefix-state), raw), raw-state),
    ((data, attacker-state), result)), final-state))
and trace-candidate:
  partial-trace-table-candidate trace-table
    ((replicate rounds []) [i := trace-openings])

```


and *comp-candidate*:
partial-composition-table-candidate composition-table
((replicate rounds []) [i := composition-openings])
and *trace-low*: *trace-table-low-degree trace-table*
and *comp-low*: *composition-table-low-degree maxDegree composition-table*
and *not-all*:
 $\neg$ *all-queries-consistent trace-table composition-table*
(sqp-alphas prefix)
shows
checked-staged-security-with-query-prefix-header-supported-query-target-hit-at
i
(Some (((((prefix, prefix-state), raw), raw-state),
((data, attacker-state), result)), final-state)) $\vee$
checked-staged-security-with-query-prefix-candidate-binding-gap-at
trace-openings composition-openings trace-table composition-table i
(Some (((((prefix, prefix-state), raw), raw-state),
((data, attacker-state), result)), final-state))
proof –
from *hit* **have** *opening-target*:
index (to-nat raw) $\in$
staged-query-prefix-candidate-opening-query-target-from-prefix
((replicate rounds []) [i := trace-openings])
((replicate rounds []) [i := composition-openings])
i prefix prefix-state
by (rule
checked-staged-security-with-query-prefix-header-authenticated-candidate-opening-hitE)
have *opening-subset*:
staged-query-prefix-candidate-opening-query-target-from-prefix
((replicate rounds []) [i := trace-openings])
((replicate rounds []) [i := composition-openings])
i prefix prefix-state $\subseteq$
staged-query-prefix-candidate-pair-query-target-from-prefix trace-table
composition-table prefix prefix-state
by (rule
staged-query-prefix-candidate-opening-query-target-from-prefix-subset-candidate-pair-from-prefix
 $[OF \text{ trace-candidate comp-candidate trace-low comp-low not-all}]$)
have *pair-target-hit*:
checked-staged-query-prefix-dynamic-index-hit
(staged-query-prefix-candidate-pair-query-target-from-prefix
trace-table composition-table)
(Some (((prefix, prefix-state), raw), raw-state))
using *opening-target opening-subset*
unfolding *checked-staged-query-prefix-dynamic-index-hit-def* **by** *auto*
show *?thesis*
proof (cases
(trace-table, composition-table) $\in$
query-header-supported-partial-table-candidates prefix-state
(sqp-trace-root prefix)
(sqp-trace-fri-roots prefix)

```

      (sqp-trace-final prefix)
      (sqp-alphas prefix)
      (sqp-degree prefix)
      (sqp-composition-fri-roots prefix)
      (sqp-composition-final prefix))
  case True
  have header-target:
    checked-staged-query-prefix-dynamic-index-hit
    staged-query-prefix-header-supported-query-target
    (Some (((prefix, prefix-state), raw), raw-state))
  by (rule
      checked-staged-query-prefix-candidate-pair-hit-imp-header-supported-target-hit
      [OF True pair-target-hit])
  have
    checked-staged-security-with-query-prefix-header-supported-query-target-hit-at
    i
    (Some (((((prefix, prefix-state), raw), raw-state),
      ((data, attacker-state), result)), final-state))
  unfolding
    checked-staged-security-with-query-prefix-header-supported-query-target-hit-at-def
  using header-target by simp
  then show ?thesis by simp
next
case False
have
  checked-staged-security-with-query-prefix-candidate-binding-gap-at
  trace-openings composition-openings trace-table composition-table i
  (Some (((((prefix, prefix-state), raw), raw-state),
    ((data, attacker-state), result)), final-state))
  unfolding
    checked-staged-security-with-query-prefix-candidate-binding-gap-at-def
  using hit trace-candidate comp-candidate False by simp
  then show ?thesis by simp
qed
qed

lemma checked-staged-security-with-query-prefix-candidate-binding-gap-imp-header-authenticated:
  assumes gap:
    checked-staged-security-with-query-prefix-candidate-binding-gap-at
    trace-openings composition-openings trace-table composition-table i out
  shows
    checked-staged-security-with-query-prefix-header-authenticated-candidate-opening-hit
    trace-openings composition-openings i out
  using gap
  unfolding checked-staged-security-with-query-prefix-candidate-binding-gap-at-def
  by (cases out) (auto split: prod.splits)

lemma checked-staged-security-with-query-prefix-candidate-binding-gap-current-imp-prefix-gap-or-opening-path-output:
  assumes wf: staged-budget-wellformed budgets

```

and *controlled*: staged-adversary-controlled budgets A
and *i-bound*: $i < \text{rounds}$
and *support*:
 $out \in$
 set-dist
 (*execute*
 (*checked-staged-security-experiment-with-query-prefix-data-state* A i)
 adversary-initial-state)
and *current*:
 checked-staged-security-with-query-prefix-fixed-current-authenticated-component
 (*checked-staged-security-with-query-prefix-candidate-binding-gap-side*
 trace-openings composition-openings trace-table composition-table i)
 trace-openings composition-openings i out
shows
 checked-staged-security-with-query-prefix-prefix-candidate-binding-gap-at
 trace-openings composition-openings trace-table composition-table i $out \vee$
 checked-staged-security-with-query-prefix-opening-path-output-hit
 trace-openings composition-openings out
proof —
 have *side*:
 checked-staged-security-with-query-prefix-candidate-binding-gap-side
 trace-openings composition-openings trace-table composition-table i out
 and *current-hit*:
 checked-staged-security-with-query-prefix-authenticated-opening-hit
 trace-openings composition-openings i out
 using *current*
 unfolding
 checked-staged-security-with-query-prefix-fixed-current-authenticated-component-def
 checked-staged-security-with-query-prefix-fixed-header-component-def
 by *simp-all*
 have *split*:
 checked-staged-security-with-query-prefix-prefix-authenticated-opening-hit
 trace-openings composition-openings i $out \vee$
 checked-staged-security-with-query-prefix-opening-path-output-hit
 trace-openings composition-openings out
 by (*rule*
 checked-staged-security-with-query-prefix-fixed-current-authenticated-component-imp-prefix-or-path-output-on-sup
 [*OF wf controlled i-bound support current*])
then show *?thesis*
proof
 assume *prefix*:
 checked-staged-security-with-query-prefix-prefix-authenticated-opening-hit
 trace-openings composition-openings i out
 have
 checked-staged-security-with-query-prefix-prefix-candidate-binding-gap-at
 trace-openings composition-openings trace-table composition-table i out
 unfolding
 checked-staged-security-with-query-prefix-prefix-candidate-binding-gap-at-def
 using *side prefix* **by** *simp*

```

    then show ?thesis by simp
next
assume
  checked-staged-security-with-query-prefix-opening-path-output-hit
  trace-openings composition-openings out
  then show ?thesis by simp
qed
qed

lemma checked-staged-security-with-query-prefix-candidate-binding-gap-imp-current-or-path-output-on-support:
  assumes wf: staged-budget-wellformed budgets
  and controlled: staged-adversary-controlled budgets A
  and i-bound: i < rounds
  and support:
    out ∈
      set-dist
      (execute
        (checked-staged-security-experiment-with-query-prefix-data-state A i)
        adversary-initial-state)
  and gap:
    checked-staged-security-with-query-prefix-candidate-binding-gap-at
    trace-openings composition-openings trace-table composition-table i out
  shows
    checked-staged-security-with-query-prefix-fixed-current-authenticated-component
    (checked-staged-security-with-query-prefix-candidate-binding-gap-side
      trace-openings composition-openings trace-table composition-table i)
    trace-openings composition-openings i out ∨
    checked-staged-security-with-query-prefix-header-witness-path-output-hit
    (checked-staged-security-with-query-prefix-candidate-binding-gap-side
      trace-openings composition-openings trace-table composition-table i)
    trace-openings composition-openings i out
proof -
  have fixed:
    checked-staged-security-with-query-prefix-fixed-header-component
    (checked-staged-security-with-query-prefix-candidate-binding-gap-side
      trace-openings composition-openings trace-table composition-table i)
    trace-openings composition-openings i out
  using gap
  unfolding
    checked-staged-security-with-query-prefix-candidate-binding-gap-iff-fixed-header-component
  .
  show ?thesis
  by (rule
    checked-staged-security-with-query-prefix-fixed-header-component-imp-current-or-witness-path-output-on-support
    [OF wf controlled i-bound support fixed])
qed

lemma checked-staged-security-with-query-prefix-candidate-binding-gap-imp-prefix-gap-or-path-outputs-on-support:
  assumes wf: staged-budget-wellformed budgets

```

and *controlled*: staged-adversary-controlled budgets A
and *i-bound*: $i < \text{rounds}$
and *support*:
 $\text{out} \in$
 set-dist
 (*execute*
 (*checked-staged-security-experiment-with-query-prefix-data-state* A i)
 adversary-initial-state)
and *gap*:
 checked-staged-security-with-query-prefix-candidate-binding-gap-at
 trace-openings composition-openings trace-table composition-table i out
shows
 checked-staged-security-with-query-prefix-prefix-candidate-binding-gap-at
 trace-openings composition-openings trace-table composition-table i $\text{out} \vee$
 checked-staged-security-with-query-prefix-opening-path-output-hit
 trace-openings composition-openings $\text{out} \vee$
 checked-staged-security-with-query-prefix-header-witness-path-output-hit
 (*checked-staged-security-with-query-prefix-candidate-binding-gap-side*
 trace-openings composition-openings trace-table composition-table i)
 trace-openings composition-openings i out
proof –
 have *split*:
 checked-staged-security-with-query-prefix-fixed-current-authenticated-component
 (*checked-staged-security-with-query-prefix-candidate-binding-gap-side*
 trace-openings composition-openings trace-table composition-table i)
 trace-openings composition-openings i $\text{out} \vee$
 checked-staged-security-with-query-prefix-header-witness-path-output-hit
 (*checked-staged-security-with-query-prefix-candidate-binding-gap-side*
 trace-openings composition-openings trace-table composition-table i)
 trace-openings composition-openings i out
 by (*rule*
 checked-staged-security-with-query-prefix-candidate-binding-gap-imp-current-or-path-output-on-support
 [*OF wf controlled i-bound support gap*])
then show *?thesis*
proof
 assume *current*:
 checked-staged-security-with-query-prefix-fixed-current-authenticated-component
 (*checked-staged-security-with-query-prefix-candidate-binding-gap-side*
 trace-openings composition-openings trace-table composition-table i)
 trace-openings composition-openings i out
 have
 checked-staged-security-with-query-prefix-prefix-candidate-binding-gap-at
 trace-openings composition-openings trace-table composition-table i $\text{out} \vee$
 checked-staged-security-with-query-prefix-opening-path-output-hit
 trace-openings composition-openings out
 by (*rule*
 checked-staged-security-with-query-prefix-candidate-binding-gap-current-imp-prefix-gap-or-opening-path-output-on-support
 [*OF wf controlled i-bound support current*])
then show *?thesis* **by** *blast*

```

next
  assume
    checked-staged-security-with-query-prefix-header-witness-path-output-hit
      (checked-staged-security-with-query-prefix-candidate-binding-gap-side
        trace-openings composition-openings trace-table composition-table i)
      trace-openings composition-openings i out
  then show ?thesis by simp
qed
qed

lemma checked-staged-security-with-query-prefix-candidate-binding-gap-bound-from-prefix-target-and-path-outputs:
  assumes wf: staged-budget-wellformed budgets
  and controlled: staged-adversary-controlled budgets A
  and i-bound:  $i < \text{rounds}$ 
  and prefix-bound:
    wp-event (checked-staged-query-prefix-receive-with-state A i)
      (checked-staged-query-prefix-dynamic-index-hit
        staged-query-prefix-prefix-authenticated-opening-target)
      adversary-initial-state  $\leq P$ 
  and opening-path-bound:
    wp-event
      (checked-staged-security-experiment-with-query-prefix-data-state A i)
      (checked-staged-security-with-query-prefix-opening-path-output-hit
        trace-openings composition-openings)
      adversary-initial-state  $\leq Q$ 
  and witness-path-bound:
    wp-event
      (checked-staged-security-experiment-with-query-prefix-data-state A i)
      (checked-staged-security-with-query-prefix-header-witness-path-output-hit
        (checked-staged-security-with-query-prefix-candidate-binding-gap-side
          trace-openings composition-openings trace-table composition-table i)
        trace-openings composition-openings i)
      adversary-initial-state  $\leq R$ 
  shows
    wp-event
      (checked-staged-security-experiment-with-query-prefix-data-state A i)
      (checked-staged-security-with-query-prefix-candidate-binding-gap-at
        trace-openings composition-openings trace-table composition-table i)
      adversary-initial-state  $\leq P + Q + R$ 
proof -
  let ?m =
    checked-staged-security-experiment-with-query-prefix-data-state A i
  let ?prefix =
    checked-staged-security-with-query-prefix-prefix-candidate-binding-gap-at
      trace-openings composition-openings trace-table composition-table i
  let ?open-path =
    checked-staged-security-with-query-prefix-opening-path-output-hit
      trace-openings composition-openings
  let ?witness-path =

```

checked-staged-security-with-query-prefix-header-witness-path-output-hit
 (*checked-staged-security-with-query-prefix-candidate-binding-gap-side*
 trace-openings composition-openings trace-table composition-table i)
trace-openings composition-openings i

have *event-le*:
wp-event ?m
 (*checked-staged-security-with-query-prefix-candidate-binding-gap-at*
 trace-openings composition-openings trace-table composition-table i)
adversary-initial-state $\leq$
wp-event ?m
 ($\lambda out. ?prefix\ out \vee ?open-path\ out \vee ?witness-path\ out$)
adversary-initial-state
by (rule *wp-event-mono-on-support*)
 (rule
 checked-staged-security-with-query-prefix-candidate-binding-gap-imp-prefix-gap-or-path-outputs-on-support
 [*OF wf controlled i-bound*])

have *union-le*:
wp-event ?m
 ($\lambda out. ?prefix\ out \vee ?open-path\ out \vee ?witness-path\ out$)
adversary-initial-state $\leq$
wp-event ?m ?prefix adversary-initial-state +
 (*wp-event ?m ?open-path adversary-initial-state* +
 wp-event ?m ?witness-path adversary-initial-state)

proof –
have
wp-event ?m
 ($\lambda out. ?prefix\ out \vee ?open-path\ out \vee ?witness-path\ out$)
adversary-initial-state $\leq$
wp-event ?m ?prefix adversary-initial-state +
wp-event ?m
 ($\lambda out. ?open-path\ out \vee ?witness-path\ out$)
adversary-initial-state
by (rule *wp-event-union-bound*)

also have
 ... $\leq$
wp-event ?m ?prefix adversary-initial-state +
 (*wp-event ?m ?open-path adversary-initial-state* +
 wp-event ?m ?witness-path adversary-initial-state)
by (intro *add-mono order-refl wp-event-union-bound*)

finally show *?thesis* .

qed

have *prefix-gap-bound*:
wp-event ?m ?prefix adversary-initial-state $\leq P$
by (rule
 checked-staged-security-with-query-prefix-prefix-candidate-binding-gap-bound-from-dynamic-target
 [*OF wf controlled i-bound prefix-bound*])

have *sum-bound*:
wp-event ?m ?prefix adversary-initial-state +
 (*wp-event ?m ?open-path adversary-initial-state* +

$wp\text{-}event\ ?m\ ?witness\text{-}path\ adversary\text{-}initial\text{-}state) \leq P + Q + R$
proof –
have *path-sum-bound*:
 $wp\text{-}event\ ?m\ ?open\text{-}path\ adversary\text{-}initial\text{-}state +$
 $wp\text{-}event\ ?m\ ?witness\text{-}path\ adversary\text{-}initial\text{-}state \leq Q + R$
by (rule *add-mono*[*OF opening-path-bound witness-path-bound*])
have
 $wp\text{-}event\ ?m\ ?prefix\ adversary\text{-}initial\text{-}state +$
 $(wp\text{-}event\ ?m\ ?open\text{-}path\ adversary\text{-}initial\text{-}state +$
 $wp\text{-}event\ ?m\ ?witness\text{-}path\ adversary\text{-}initial\text{-}state) \leq P + (Q + R)$
by (rule *add-mono*[*OF prefix-gap-bound path-sum-bound*])
then show *?thesis*
by (*simp add: add.assoc*)
qed
show *?thesis*
by (rule *order-trans*[*OF event-le order-trans*[*OF union-le sum-bound*]])
qed

lemma *checked-staged-security-with-query-prefix-candidate-binding-gap-bound-from-budgets*:
assumes *wf: staged-budget-wellformed budgets*
and *controlled: staged-adversary-controlled budgets A*
and *i-bound: i < rounds*
shows
 $wp\text{-}event$
 $(checked\text{-}staged\text{-}security\text{-}experiment\text{-}with\text{-}query\text{-}prefix\text{-}data\text{-}state\ A\ i)$
 $(checked\text{-}staged\text{-}security\text{-}with\text{-}query\text{-}prefix\text{-}candidate\text{-}binding\text{-}gap\text{-}at$
 $trace\text{-}openings\ composition\text{-}openings\ trace\text{-}table\ composition\text{-}table\ i)$
 $adversary\text{-}initial\text{-}state \leq$
 $staged\text{-}phase\text{-}relation\text{-}error\ size$
 $(staged\text{-}query\text{-}search\text{-}queries\ budgets\ i + 1) +$
 $(1::prob) / nnreal\ (card\ query\text{-}sample\text{-}space)$
proof –
have *event-le*:
 $wp\text{-}event$
 $(checked\text{-}staged\text{-}security\text{-}experiment\text{-}with\text{-}query\text{-}prefix\text{-}data\text{-}state\ A\ i)$
 $(checked\text{-}staged\text{-}security\text{-}with\text{-}query\text{-}prefix\text{-}candidate\text{-}binding\text{-}gap\text{-}at$
 $trace\text{-}openings\ composition\text{-}openings\ trace\text{-}table\ composition\text{-}table\ i)$
 $adversary\text{-}initial\text{-}state \leq$
 $wp\text{-}event$
 $(checked\text{-}staged\text{-}security\text{-}experiment\text{-}with\text{-}query\text{-}prefix\text{-}data\text{-}state\ A\ i)$
 $(checked\text{-}staged\text{-}security\text{-}with\text{-}query\text{-}prefix\text{-}header\text{-}authenticated\text{-}candidate\text{-}opening\text{-}hit$
 $trace\text{-}openings\ composition\text{-}openings\ i)$
 $adversary\text{-}initial\text{-}state$
by (rule *wp-event-mono*)
(rule
 $checked\text{-}staged\text{-}security\text{-}with\text{-}query\text{-}prefix\text{-}candidate\text{-}binding\text{-}gap\text{-}imp\text{-}header\text{-}authenticated$)
have *header-bound*:
 $wp\text{-}event$
 $(checked\text{-}staged\text{-}security\text{-}experiment\text{-}with\text{-}query\text{-}prefix\text{-}data\text{-}state\ A\ i)$


```

    (checked-staged-security-with-query-prefix-header-authenticated-candidate-opening-hit
      trace-openings composition-openings i)
  adversary-initial-state ≤
  staged-phase-relation-error size
  (staged-query-search-queries budgets i + 1) +
  (1::prob) / nnreal (card query-sample-space)
by (rule
  checked-staged-security-with-query-prefix-header-authenticated-candidate-opening-hit-bound-from-budgets
  [OF wf controlled i-bound])
show ?thesis
by (rule order-trans[OF event-le header-bound])
qed

```

definition *checked-staged-security-with-actual-alpha-prefix-header-supported-query-target-hit-at*

```

:: nat ⇒ 'f staged-adversary ⇒
  ((((((f × 'f list × 'f list × 'f) ×
    'f protocol-channel) × 'f staged-proof-data) ×
    'f protocol-channel) × unit list) ×
    'f protocol-channel) option ⇒ bool

```

where

```

checked-staged-security-with-actual-alpha-prefix-header-supported-query-target-hit-at
  i A out ⟷
  (case out of
    None ⇒ False
  | Some (((alpha-prefix, alpha-prefix-state), data), attacker-state),
    result), final-state) ⇒
  (∃ prefix prefix-state raw raw-state.
    Some (data, attacker-state) ∈
      set-dist
        (execute (checked-staged-transcript-program A)
          adversary-initial-state) ∧
    Some (((prefix, prefix-state), raw), raw-state) ∈
      set-dist
        (execute (checked-staged-query-prefix-receive-with-state A i)
          adversary-initial-state) ∧
    checked-staged-query-prefix-dynamic-index-hit
      staged-query-prefix-header-supported-query-target
      (Some (((prefix, prefix-state), raw), raw-state))))

```

definition *checked-staged-security-with-actual-alpha-prefix-single-round-header-supported-query-target-hit-at*

```

:: nat ⇒ 'f staged-adversary ⇒
  ((((((f × 'f list × 'f list × 'f) ×
    'f protocol-channel) × 'f staged-proof-data) ×
    'f protocol-channel) × unit list) ×
    'f protocol-channel) option ⇒ bool

```

where

```

checked-staged-security-with-actual-alpha-prefix-single-round-header-supported-query-target-hit-at
  i A out ⟷
  (case out of

```

$None \Rightarrow False$
 $| Some (((alpha\text{-}prefix, alpha\text{-}prefix\text{-}state), data), attacker\text{-}state),$
 $result), final\text{-}state) \Rightarrow$
 $(\exists prefix\ prefix\text{-}state\ raw\ raw\text{-}state.$
 $Some (data, attacker\text{-}state) \in$
 $set\text{-}dist$
 $(execute (checked\text{-}staged\text{-}transcript\text{-}program\ A)$
 $adversary\text{-}initial\text{-}state) \wedge$
 $Some (((prefix, prefix\text{-}state), raw), raw\text{-}state) \in$
 $set\text{-}dist$
 $(execute (checked\text{-}staged\text{-}query\text{-}prefix\text{-}receive\text{-}with\text{-}state\ A\ i)$
 $adversary\text{-}initial\text{-}state) \wedge$
 $checked\text{-}staged\text{-}query\text{-}prefix\text{-}dynamic\text{-}index\text{-}hit$
 $(staged\text{-}query\text{-}prefix\text{-}single\text{-}round\text{-}header\text{-}supported\text{-}query\text{-}target\ i)$
 $(Some (((prefix, prefix\text{-}state), raw), raw\text{-}state))))$

The preceding actual-alpha event is a diagnostic compatibility predicate: it existentially ranges over query-prefix branches. It should not be bounded directly from query-prefix target budgets unless a separate projection lemma is proved. Bounded public-route arguments should keep the event at the query-prefix data-state layer, or use an explicitly aligned actual-alpha predicate whose query-prefix output is fixed by the run.

definition *checked-staged-security-with-actual-alpha-prefix-query-prefix-candidate-binding-gap-at*

$:: nat \Rightarrow 'f\ staged\text{-}adversary \Rightarrow$
 $(((((('f \times 'f\ list \times 'f\ list \times 'f) \times$
 $'f\ protocol\text{-}channel) \times 'f\ staged\text{-}proof\text{-}data) \times$
 $'f\ protocol\text{-}channel) \times unit\ list) \times$
 $'f\ protocol\text{-}channel) option \Rightarrow bool$

where

$checked\text{-}staged\text{-}security\text{-}with\text{-}actual\text{-}alpha\text{-}prefix\text{-}query\text{-}prefix\text{-}candidate\text{-}binding\text{-}gap\text{-}at$
 $i\ A\ out \longleftrightarrow$
 $(case\ out\ of$
 $None \Rightarrow False$
 $| Some (((alpha\text{-}prefix, alpha\text{-}prefix\text{-}state), data), attacker\text{-}state),$
 $result), final\text{-}state) \Rightarrow$
 $(\exists trace\text{-}table\ composition\text{-}table\ prefix\ prefix\text{-}state\ raw\ raw\text{-}state.$
 $Some (data, attacker\text{-}state) \in$
 $set\text{-}dist$
 $(execute (checked\text{-}staged\text{-}transcript\text{-}program\ A)$
 $adversary\text{-}initial\text{-}state) \wedge$
 $Some (((prefix, prefix\text{-}state), raw), raw\text{-}state) \in$
 $set\text{-}dist$
 $(execute (checked\text{-}staged\text{-}query\text{-}prefix\text{-}receive\text{-}with\text{-}state\ A\ i)$
 $adversary\text{-}initial\text{-}state) \wedge$
 $checked\text{-}staged\text{-}query\text{-}prefix\text{-}dynamic\text{-}index\text{-}hit$
 $(staged\text{-}query\text{-}prefix\text{-}candidate\text{-}pair\text{-}query\text{-}target\text{-}from\text{-}prefix$
 $trace\text{-}table\ composition\text{-}table)$
 $(Some (((prefix, prefix\text{-}state), raw), raw\text{-}state)) \wedge$
 $(trace\text{-}table, composition\text{-}table) \notin$
 $query\text{-}header\text{-}supported\text{-}partial\text{-}table\text{-}candidates\ prefix\text{-}state$

(sqp-trace-root prefix)
 (sqp-trace-fri-roots prefix)
 (sqp-trace-final prefix)
 (sqp-alphas prefix)
 (sqp-degree prefix)
 (sqp-composition-fri-roots prefix)
 (sqp-composition-final prefix)))

definition *checked-staged-security-with-actual-alpha-prefix-query-prefix-single-round-candidate-binding-gap-at*

:: nat $\Rightarrow$ 'f staged-adversary $\Rightarrow$
 ((((((('f $\times$ 'f list $\times$ 'f list $\times$ 'f) $\times$
 'f protocol-channel) $\times$ 'f staged-proof-data) $\times$
 'f protocol-channel) $\times$ unit list) $\times$
 'f protocol-channel) option $\Rightarrow$ bool

where

checked-staged-security-with-actual-alpha-prefix-query-prefix-single-round-candidate-binding-gap-at

i A out $\longleftrightarrow$

(case out of

None $\Rightarrow$ False

| Some (((((alpha-prefix, alpha-prefix-state), data), attacker-state),
result), final-state) $\Rightarrow$

($\exists$ trace-table composition-table prefix prefix-state raw raw-state.

Some (data, attacker-state) $\in$

set-dist

(execute (checked-staged-transcript-program A)

adversary-initial-state) $\wedge$

Some (((prefix, prefix-state), raw), raw-state) $\in$

set-dist

(execute (checked-staged-query-prefix-receive-with-state A i)

adversary-initial-state) $\wedge$

checked-staged-query-prefix-dynamic-index-hit

(staged-query-prefix-candidate-pair-query-target-from-prefix
trace-table composition-table)

(Some (((prefix, prefix-state), raw), raw-state)) $\wedge$

(trace-table, composition-table) $\notin$

query-header-supported-single-round-partial-table-candidates

prefix-state

(sqp-trace-root prefix)

(sqp-trace-fri-roots prefix)

(sqp-trace-final prefix)

(sqp-alphas prefix)

(sqp-degree prefix)

(sqp-composition-fri-roots prefix)

(sqp-composition-final prefix)

i))

definition *checked-staged-security-with-actual-alpha-prefix-query-prefix-single-round-candidate-binding-gap-witness-a*

:: nat $\Rightarrow$ 'f staged-adversary $\Rightarrow$
 ((((((('f $\times$ 'f list $\times$ 'f list $\times$ 'f) $\times$

$'f \text{ protocol-channel}) \times 'f \text{ staged-proof-data}) \times$
 $'f \text{ protocol-channel}) \times \text{unit list}) \times$
 $'f \text{ protocol-channel}) \text{ option} \Rightarrow \text{bool}$

where

$\text{checked-staged-security-with-actual-alpha-prefix-query-prefix-single-round-candidate-binding-gap-witness-at}$
 $i \ A \ \text{out} \longleftrightarrow$
 $(\text{case out of}$
 $\text{None} \Rightarrow \text{False}$
 $| \text{Some } (((\text{alpha-prefix}, \text{alpha-prefix-state}), \text{data}), \text{attacker-state}),$
 $\text{result}), \text{final-state}) \Rightarrow$
 $(\exists \text{ trace-openings composition-openings trace-table composition-table}$
 $\text{prefix prefix-state raw raw-state.}$
 $\text{Some } (\text{data}, \text{attacker-state}) \in$
 set-dist
 $(\text{execute } (\text{checked-staged-transcript-program } A)$
 $\text{adversary-initial-state}) \wedge$
 $\text{Some } (((\text{prefix}, \text{prefix-state}), \text{raw}), \text{raw-state}) \in$
 set-dist
 $(\text{execute } (\text{checked-staged-query-prefix-receive-with-state } A \ i)$
 $\text{adversary-initial-state}) \wedge$
 $(\text{trace-openings}, \text{composition-openings}) \in$
 $\text{query-header-supported-partial-opening-witnesses}$
 $(\text{verifier-state-from-adversary attacker-state}$
 $(\text{staged-proof-transcript data}))$
 $(\text{staged-trace-root data})$
 $(\text{staged-trace-fri-roots data})$
 $(\text{staged-trace-final data})$
 $(\text{staged-alphas data})$
 $(\text{staged-degree data})$
 $(\text{staged-composition-fri-roots data})$
 $(\text{staged-composition-final data}) \wedge$
 $\text{partial-trace-table-candidate trace-table}$
 $((\text{replicate rounds } []) [i := \text{trace-openings}]) \wedge$
 $\text{partial-composition-table-candidate composition-table}$
 $((\text{replicate rounds } []) [i := \text{composition-openings}]) \wedge$
 $\text{checked-staged-query-prefix-dynamic-index-hit}$
 $(\text{staged-query-prefix-candidate-pair-query-target-from-prefix}$
 $\text{trace-table composition-table})$
 $(\text{Some } (((\text{prefix}, \text{prefix-state}), \text{raw}), \text{raw-state})) \wedge$
 $(\text{trace-table}, \text{composition-table}) \notin$
 $\text{query-header-supported-single-round-partial-table-candidates}$
 prefix-state
 $(\text{sqp-trace-root prefix})$
 $(\text{sqp-trace-fri-roots prefix})$
 $(\text{sqp-trace-final prefix})$
 $(\text{sqp-alphas prefix})$
 $(\text{sqp-degree prefix})$
 $(\text{sqp-composition-fri-roots prefix})$
 $(\text{sqp-composition-final prefix})$

i))

This witness variant has the same diagnostic status: the authenticated openings are carried, but the query-prefix branch is still existential in the actual-alpha output. Use the same-run query-prefix witness-gap predicates for probability bounds.

lemma *checked-staged-security-with-actual-alpha-prefix-query-prefix-single-round-candidate-binding-gap-witness-imp-*
assumes *gap:*
checked-staged-security-with-actual-alpha-prefix-query-prefix-single-round-candidate-binding-gap-witness-at
i A out
shows
checked-staged-security-with-actual-alpha-prefix-query-prefix-single-round-candidate-binding-gap-at
i A out
proof (*cases out*)
case *None*
then show *?thesis*
using *gap*
unfolding
checked-staged-security-with-actual-alpha-prefix-query-prefix-single-round-candidate-binding-gap-witness-at-def
by *simp*
next
case (*Some packed*)
then obtain *alpha-prefix alpha-prefix-state data attacker-state result*
final-state where out-eq:
out =
Some (((((alpha-prefix, alpha-prefix-state), data), attacker-state),
result), final-state)
by (*cases packed*) (*auto split: prod.splits*)
from *gap* **obtain** *trace-openings composition-openings trace-table*
composition-table prefix prefix-state raw raw-state where builder:
Some (data, attacker-state) ∈
set-dist
(execute (checked-staged-transcript-program A)
adversary-initial-state)
and *prefix-receive:*
Some (((prefix, prefix-state), raw), raw-state) ∈
set-dist
(execute (checked-staged-query-prefix-receive-with-state A i)
adversary-initial-state)
and *target-hit:*
checked-staged-query-prefix-dynamic-index-hit
(staged-query-prefix-candidate-pair-query-target-from-prefix
trace-table composition-table)
(Some (((prefix, prefix-state), raw), raw-state))
and *notin:*
(trace-table, composition-table) ∉
query-header-supported-single-round-partial-table-candidates
prefix-state
(sqp-trace-root prefix)
(sqp-trace-fri-roots prefix)

```

      (sqp-trace-final prefix)
      (sqp-alphas prefix)
      (sqp-degree prefix)
      (sqp-composition-fri-roots prefix)
      (sqp-composition-final prefix)
    i
  unfolding out-eq
  checked-staged-security-with-actual-alpha-prefix-query-prefix-single-round-candidate-binding-gap-witness-at-def
  by auto
  have body:
  ∃ trace-table composition-table prefix prefix-state raw raw-state.
    Some (data, attacker-state) ∈
      set-dist
        (execute (checked-staged-transcript-program A)
          adversary-initial-state) ∧
    Some (((prefix, prefix-state), raw), raw-state) ∈
      set-dist
        (execute (checked-staged-query-prefix-receive-with-state A i)
          adversary-initial-state) ∧
    checked-staged-query-prefix-dynamic-index-hit
      (staged-query-prefix-candidate-pair-query-target-from-prefix
        trace-table composition-table)
      (Some (((prefix, prefix-state), raw), raw-state)) ∧
    (trace-table, composition-table) ∉
      query-header-supported-single-round-partial-table-candidates
        prefix-state
        (sqp-trace-root prefix)
        (sqp-trace-fri-roots prefix)
        (sqp-trace-final prefix)
        (sqp-alphas prefix)
        (sqp-degree prefix)
        (sqp-composition-fri-roots prefix)
        (sqp-composition-final prefix)
    i
  by (intro exI[of - trace-table] exI[of - composition-table]
    exI[of - prefix] exI[of - prefix-state]
    exI[of - raw] exI[of - raw-state])
    (use builder prefix-receive target-hit notin in simp)
  show ?thesis
  unfolding out-eq
  checked-staged-security-with-actual-alpha-prefix-query-prefix-single-round-candidate-binding-gap-at-def
  using body by simp
qed

```

lemma *checked-staged-security-with-actual-alpha-prefix-query-prefix-single-round-candidate-binding-gap-witnessE*:
assumes *gap*:
checked-staged-security-with-actual-alpha-prefix-query-prefix-single-round-candidate-binding-gap-witness-at
i A
(Some ((((((alpha-prefix, alpha-prefix-state), data), attacker-state),

result), *final-state*))
obtains *trace-openings composition-openings trace-table composition-table*
prefix prefix-state raw raw-state **where**
Some (data, attacker-state) ∈
set-dist
(execute (checked-staged-transcript-program A)
adversary-initial-state)
Some (((prefix, prefix-state), raw), raw-state) ∈
set-dist
(execute (checked-staged-query-prefix-receive-with-state A i)
adversary-initial-state)
(trace-openings, composition-openings) ∈
query-header-supported-partial-opening-witnesses
(verifier-state-from-adversary attacker-state
(staged-proof-transcript data))
(staged-trace-root data)
(staged-trace-fri-roots data)
(staged-trace-final data)
(staged-alphas data)
(staged-degree data)
(staged-composition-fri-roots data)
(staged-composition-final data)
partial-trace-table-candidate trace-table
((replicate rounds []) [i := trace-openings])
partial-composition-table-candidate composition-table
((replicate rounds []) [i := composition-openings])
checked-staged-query-prefix-dynamic-index-hit
(staged-query-prefix-candidate-pair-query-target-from-prefix
trace-table composition-table)
(Some (((prefix, prefix-state), raw), raw-state))
(trace-table, composition-table) ∉
query-header-supported-single-round-partial-table-candidates
prefix-state
(sqp-trace-root prefix)
(sqp-trace-fri-roots prefix)
(sqp-trace-final prefix)
(sqp-alphas prefix)
(sqp-degree prefix)
(sqp-composition-fri-roots prefix)
(sqp-composition-final prefix)
i
proof –
have *body*:
 $\exists$ *trace-openings composition-openings trace-table composition-table*
prefix prefix-state raw raw-state.
Some (data, attacker-state) ∈
set-dist
(execute (checked-staged-transcript-program A)
adversary-initial-state) ∧

Some (((*prefix*, *prefix-state*), *raw*), *raw-state*) ∈
set-dist
 (execute (*checked-staged-query-prefix-receive-with-state A i*)
adversary-initial-state) ∧
 (*trace-openings*, *composition-openings*) ∈
query-header-supported-partial-opening-witnesses
 (*verifier-state-from-adversary attacker-state*
 (*staged-proof-transcript data*))
 (*staged-trace-root data*)
 (*staged-trace-fri-roots data*)
 (*staged-trace-final data*)
 (*staged-alphas data*)
 (*staged-degree data*)
 (*staged-composition-fri-roots data*)
 (*staged-composition-final data*) ∧
partial-trace-table-candidate trace-table
 ((*replicate rounds []*) [*i := trace-openings*]) ∧
partial-composition-table-candidate composition-table
 ((*replicate rounds []*) [*i := composition-openings*]) ∧
checked-staged-query-prefix-dynamic-index-hit
 (*staged-query-prefix-candidate-pair-query-target-from-prefix*
trace-table composition-table)
 (*Some* (((*prefix*, *prefix-state*), *raw*), *raw-state*)) ∧
 (*trace-table*, *composition-table*) ∉
query-header-supported-single-round-partial-table-candidates
prefix-state
 (*sqp-trace-root prefix*)
 (*sqp-trace-fri-roots prefix*)
 (*sqp-trace-final prefix*)
 (*sqp-alphas prefix*)
 (*sqp-degree prefix*)
 (*sqp-composition-fri-roots prefix*)
 (*sqp-composition-final prefix*)
i
using *gap*
unfolding
checked-staged-security-with-actual-alpha-prefix-query-prefix-single-round-candidate-binding-gap-witness-at-def
by *simp*
then obtain *trace-openings composition-openings trace-table composition-table*
prefix prefix-state raw raw-state where
Some (*data*, *attacker-state*) ∈
set-dist
 (execute (*checked-staged-transcript-program A*)
adversary-initial-state)
Some (((*prefix*, *prefix-state*), *raw*), *raw-state*) ∈
set-dist
 (execute (*checked-staged-query-prefix-receive-with-state A i*)
adversary-initial-state)
 (*trace-openings*, *composition-openings*) ∈


```

query-header-supported-partial-opening-witnesses
  (verifier-state-from-adversary attacker-state
    (staged-proof-transcript data))
  (staged-trace-root data)
  (staged-trace-fri-roots data)
  (staged-trace-final data)
  (staged-alphas data)
  (staged-degree data)
  (staged-composition-fri-roots data)
  (staged-composition-final data)
partial-trace-table-candidate trace-table
  ((replicate rounds []) [i := trace-openings])
partial-composition-table-candidate composition-table
  ((replicate rounds []) [i := composition-openings])
checked-staged-query-prefix-dynamic-index-hit
  (staged-query-prefix-candidate-pair-query-target-from-prefix
    trace-table composition-table)
  (Some (((prefix, prefix-state), raw), raw-state))
(trace-table, composition-table)  $\notin$ 
query-header-supported-single-round-partial-table-candidates
  prefix-state
  (sqp-trace-root prefix)
  (sqp-trace-fri-roots prefix)
  (sqp-trace-final prefix)
  (sqp-alphas prefix)
  (sqp-degree prefix)
  (sqp-composition-fri-roots prefix)
  (sqp-composition-final prefix)
i
by blast
then show ?thesis
by (rule that)
qed

definition checked-staged-security-with-actual-alpha-prefix-query-prefix-candidate-binding-gap-witness-at
  :: nat  $\Rightarrow$  'f staged-adversary  $\Rightarrow$ 
    ((((((f  $\times$  'f list  $\times$  'f list  $\times$  'f)  $\times$ 
      'f protocol-channel)  $\times$  'f staged-proof-data)  $\times$ 
      'f protocol-channel)  $\times$  unit list)  $\times$ 
      'f protocol-channel) option  $\Rightarrow$  bool

where
  checked-staged-security-with-actual-alpha-prefix-query-prefix-candidate-binding-gap-witness-at
    i A out  $\longleftrightarrow$ 
    (case out of
      None  $\Rightarrow$  False
      | Some (((((alpha-prefix, alpha-prefix-state), data), attacker-state),
        result), final-state)  $\Rightarrow$ 
        ( $\exists$  trace-openings composition-openings trace-table composition-table
          prefix prefix-state raw raw-state.)

```

Some (*data*, *attacker-state*) ∈
set-dist
 (*execute* (*checked-staged-transcript-program* *A*)
adversary-initial-state) ∧
Some (((*prefix*, *prefix-state*), *raw*), *raw-state*) ∈
set-dist
 (*execute* (*checked-staged-query-prefix-receive-with-state* *A i*)
adversary-initial-state) ∧
 (*trace-openings*, *composition-openings*) ∈
query-header-supported-partial-opening-witnesses
 (*verifier-state-from-adversary* *attacker-state*
 (*staged-proof-transcript* *data*))
 (*staged-trace-root* *data*)
 (*staged-trace-fri-roots* *data*)
 (*staged-trace-final* *data*)
 (*staged-alphas* *data*)
 (*staged-degree* *data*)
 (*staged-composition-fri-roots* *data*)
 (*staged-composition-final* *data*) ∧
partial-trace-table-candidate *trace-table*
 ((*replicate* *rounds []*) [*i := trace-openings*]) ∧
partial-composition-table-candidate *composition-table*
 ((*replicate* *rounds []*) [*i := composition-openings*]) ∧
checked-staged-query-prefix-dynamic-index-hit
 (*staged-query-prefix-candidate-pair-query-target-from-prefix*
trace-table composition-table)
 (*Some* (((*prefix*, *prefix-state*), *raw*), *raw-state*)) ∧
 (*trace-table*, *composition-table*) ∉
query-header-supported-partial-table-candidates *prefix-state*
 (*sqp-trace-root* *prefix*)
 (*sqp-trace-fri-roots* *prefix*)
 (*sqp-trace-final* *prefix*)
 (*sqp-alphas* *prefix*)
 (*sqp-degree* *prefix*)
 (*sqp-composition-fri-roots* *prefix*)
 (*sqp-composition-final* *prefix*)))

lemma *checked-staged-security-with-actual-alpha-prefix-query-prefix-candidate-binding-gap-witness-imp-gap:*

assumes *gap:*

checked-staged-security-with-actual-alpha-prefix-query-prefix-candidate-binding-gap-witness-at
i A out

shows

checked-staged-security-with-actual-alpha-prefix-query-prefix-candidate-binding-gap-at
i A out

using *gap*

proof (*cases out*)

case *None*

then show *?thesis*

using *gap*

```

unfolding
  checked-staged-security-with-actual-alpha-prefix-query-prefix-candidate-binding-gap-witness-at-def
by simp
next
case (Some packed)
then obtain alpha-prefix alpha-prefix-state data attacker-state result
  final-state where out-eq:
  out =
    Some (((((alpha-prefix, alpha-prefix-state), data), attacker-state),
      result), final-state)
by (cases packed) (auto split: prod.splits)
have gap-body:
   $\exists$  trace-openings composition-openings trace-table composition-table
    prefix prefix-state raw raw-state.
  Some (data, attacker-state)  $\in$ 
    set-dist
      (execute (checked-staged-transcript-program A)
        adversary-initial-state)  $\wedge$ 
  Some (((prefix, prefix-state), raw), raw-state)  $\in$ 
    set-dist
      (execute (checked-staged-query-prefix-receive-with-state A i)
        adversary-initial-state)  $\wedge$ 
  (trace-openings, composition-openings)  $\in$ 
    query-header-supported-partial-opening-witnesses
      (verifier-state-from-adversary attacker-state
        (staged-proof-transcript data))
      (staged-trace-root data)
      (staged-trace-fri-roots data)
      (staged-trace-final data)
      (staged-alphas data)
      (staged-degree data)
      (staged-composition-fri-roots data)
      (staged-composition-final data)  $\wedge$ 
  partial-trace-table-candidate trace-table
    ((replicate rounds []) [i := trace-openings])  $\wedge$ 
  partial-composition-table-candidate composition-table
    ((replicate rounds []) [i := composition-openings])  $\wedge$ 
  checked-staged-query-prefix-dynamic-index-hit
    (staged-query-prefix-candidate-pair-query-target-from-prefix
      trace-table composition-table)
    (Some (((prefix, prefix-state), raw), raw-state))  $\wedge$ 
  (trace-table, composition-table)  $\notin$ 
    query-header-supported-partial-table-candidates prefix-state
      (sqp-trace-root prefix)
      (sqp-trace-fri-roots prefix)
      (sqp-trace-final prefix)
      (sqp-alphas prefix)
      (sqp-degree prefix)
      (sqp-composition-fri-roots prefix)

```

```

      (sqp-composition-final prefix)
using gap
unfolding out-eq
      checked-staged-security-with-actual-alpha-prefix-query-prefix-candidate-binding-gap-witness-at-def
by simp
from gap-body
obtain trace-openings composition-openings trace-table composition-table
  prefix prefix-state raw raw-state where builder:
    Some (data, attacker-state) ∈
      set-dist
      (execute (checked-staged-transcript-program A)
        adversary-initial-state)
and prefix-receive:
  Some (((prefix, prefix-state), raw), raw-state) ∈
    set-dist
    (execute (checked-staged-query-prefix-receive-with-state A i)
      adversary-initial-state)
and target-hit:
  checked-staged-query-prefix-dynamic-index-hit
  (staged-query-prefix-candidate-pair-query-target-from-prefix
    trace-table composition-table)
  (Some (((prefix, prefix-state), raw), raw-state))
and notin:
  (trace-table, composition-table) ∉
    query-header-supported-partial-table-candidates prefix-state
    (sqp-trace-root prefix)
    (sqp-trace-fri-roots prefix)
    (sqp-trace-final prefix)
    (sqp-alphas prefix)
    (sqp-degree prefix)
    (sqp-composition-fri-roots prefix)
    (sqp-composition-final prefix)
by blast
have body:
  ∃ trace-table composition-table prefix prefix-state raw raw-state.
  Some (data, attacker-state) ∈
    set-dist
    (execute (checked-staged-transcript-program A)
      adversary-initial-state) ∧
  Some (((prefix, prefix-state), raw), raw-state) ∈
    set-dist
    (execute (checked-staged-query-prefix-receive-with-state A i)
      adversary-initial-state) ∧
  checked-staged-query-prefix-dynamic-index-hit
  (staged-query-prefix-candidate-pair-query-target-from-prefix
    trace-table composition-table)
  (Some (((prefix, prefix-state), raw), raw-state)) ∧
  (trace-table, composition-table) ∉
    query-header-supported-partial-table-candidates prefix-state

```

```

    (sqp-trace-root prefix)
    (sqp-trace-fri-roots prefix)
    (sqp-trace-final prefix)
    (sqp-alphas prefix)
    (sqp-degree prefix)
    (sqp-composition-fri-roots prefix)
    (sqp-composition-final prefix)
  by (intro exI[of - trace-table] exI[of - composition-table]
      exI[of - prefix] exI[of - prefix-state]
      exI[of - raw] exI[of - raw-state])
    (use builder prefix-receive target-hit notin in simp)
show ?thesis
unfolding out-eq
  checked-staged-security-with-actual-alpha-prefix-query-prefix-candidate-binding-gap-at-def
using body by simp
qed

```

lemma *checked-staged-security-with-actual-alpha-prefix-query-prefix-candidate-binding-gap-witnessE*:

assumes *gap*:

checked-staged-security-with-actual-alpha-prefix-query-prefix-candidate-binding-gap-witness-at
i A
(Some (((((alpha-prefix, alpha-prefix-state), data), attacker-state),
result), final-state)))

obtains *trace-openings composition-openings trace-table composition-table*

prefix prefix-state raw raw-state **where**

Some (data, attacker-state) ∈

set-dist

(execute (checked-staged-transcript-program A)
adversary-initial-state)

Some (((prefix, prefix-state), raw), raw-state) ∈

set-dist

(execute (checked-staged-query-prefix-receive-with-state A i)
adversary-initial-state)

(trace-openings, composition-openings) ∈

query-header-supported-partial-opening-witnesses

(verifier-state-from-adversary attacker-state

(staged-proof-transcript data))

(staged-trace-root data)

(staged-trace-fri-roots data)

(staged-trace-final data)

(staged-alphas data)

(staged-degree data)

(staged-composition-fri-roots data)

(staged-composition-final data)

partial-trace-table-candidate trace-table

((replicate rounds []) [i := trace-openings])

partial-composition-table-candidate composition-table

((replicate rounds []) [i := composition-openings])

checked-staged-query-prefix-dynamic-index-hit

(staged-query-prefix-candidate-pair-query-target-from-prefix
trace-table composition-table)
(Some (((prefix, prefix-state), raw), raw-state))
(trace-table, composition-table) ∉
query-header-supported-partial-table-candidates prefix-state
(sqp-trace-root prefix)
(sqp-trace-fri-roots prefix)
(sqp-trace-final prefix)
(sqp-alphas prefix)
(sqp-degree prefix)
(sqp-composition-fri-roots prefix)
(sqp-composition-final prefix)

proof –

have *body*:

$\exists$ *trace-openings composition-openings trace-table composition-table*
prefix prefix-state raw raw-state.
Some (data, attacker-state) ∈
set-dist
(execute (checked-staged-transcript-program A)
adversary-initial-state) ∧
Some (((prefix, prefix-state), raw), raw-state) ∈
set-dist
(execute (checked-staged-query-prefix-receive-with-state A i)
adversary-initial-state) ∧
(trace-openings, composition-openings) ∈
query-header-supported-partial-opening-witnesses
(verifier-state-from-adversary attacker-state
(staged-proof-transcript data))
(staged-trace-root data)
(staged-trace-fri-roots data)
(staged-trace-final data)
(staged-alphas data)
(staged-degree data)
(staged-composition-fri-roots data)
(staged-composition-final data) ∧
partial-trace-table-candidate trace-table
((replicate rounds []) [i := trace-openings]) ∧
partial-composition-table-candidate composition-table
((replicate rounds []) [i := composition-openings]) ∧
checked-staged-query-prefix-dynamic-index-hit
(staged-query-prefix-candidate-pair-query-target-from-prefix
trace-table composition-table)
(Some (((prefix, prefix-state), raw), raw-state)) ∧
(trace-table, composition-table) ∉
query-header-supported-partial-table-candidates prefix-state
(sqp-trace-root prefix)
(sqp-trace-fri-roots prefix)
(sqp-trace-final prefix)
(sqp-alphas prefix)

```

      (sqp-degree prefix)
      (sqp-composition-fri-roots prefix)
      (sqp-composition-final prefix)
    using gap
    unfolding
      checked-staged-security-with-actual-alpha-prefix-query-prefix-candidate-binding-gap-witness-at-def
    by simp
  then obtain trace-openings composition-openings trace-table composition-table
    prefix prefix-state raw raw-state where
    Some (data, attacker-state) ∈
      set-dist
        (execute (checked-staged-transcript-program A)
          adversary-initial-state)
    Some (((prefix, prefix-state), raw), raw-state) ∈
      set-dist
        (execute (checked-staged-query-prefix-receive-with-state A i)
          adversary-initial-state)
    (trace-openings, composition-openings) ∈
      query-header-supported-partial-opening-witnesses
        (verifier-state-from-adversary attacker-state
          (staged-proof-transcript data))
        (staged-trace-root data)
        (staged-trace-fri-roots data)
        (staged-trace-final data)
        (staged-alphas data)
        (staged-degree data)
        (staged-composition-fri-roots data)
        (staged-composition-final data)
    partial-trace-table-candidate trace-table
      ((replicate rounds []) [i := trace-openings])
    partial-composition-table-candidate composition-table
      ((replicate rounds []) [i := composition-openings])
    checked-staged-query-prefix-dynamic-index-hit
      (staged-query-prefix-candidate-pair-query-target-from-prefix
        trace-table composition-table)
      (Some (((prefix, prefix-state), raw), raw-state))
    (trace-table, composition-table) ∉
      query-header-supported-partial-table-candidates prefix-state
        (sqp-trace-root prefix)
        (sqp-trace-fri-roots prefix)
        (sqp-trace-final prefix)
        (sqp-alphas prefix)
        (sqp-degree prefix)
        (sqp-composition-fri-roots prefix)
        (sqp-composition-final prefix)
  by blast
  then show ?thesis
  by (rule that)
qed

```

lemma *checked-staged-security-with-actual-alpha-prefix-candidate-pair-query-hit-from-prefix-imp-single-round-header*

assumes *hit*:

checked-staged-security-with-actual-alpha-prefix-candidate-pair-query-hit-from-prefix

A trace-table composition-table i

*(Some (((((alpha-prefix, alpha-prefix-state), data), attacker-state),
result), final-state)))*

shows

checked-staged-security-with-actual-alpha-prefix-single-round-header-supported-query-target-hit-at

i A

*(Some (((((alpha-prefix, alpha-prefix-state), data), attacker-state),
result), final-state))) ∨*

checked-staged-security-with-actual-alpha-prefix-query-prefix-single-round-candidate-binding-gap-at

i A

*(Some (((((alpha-prefix, alpha-prefix-state), data), attacker-state),
result), final-state)))*

proof —

from

checked-staged-security-with-actual-alpha-prefix-candidate-pair-query-hit-from-prefixE

[*OF hit*]

obtain *prefix prefix-state raw raw-state* **where**

builder:

Some (data, attacker-state) ∈

set-dist

*(execute (checked-staged-transcript-program A)
adversary-initial-state)*

and *prefix-receive*:

Some (((prefix, prefix-state), raw), raw-state) ∈

set-dist

*(execute (checked-staged-query-prefix-receive-with-state A i)
adversary-initial-state)*

and *target-hit*:

checked-staged-query-prefix-dynamic-index-hit

*(staged-query-prefix-candidate-pair-query-target-from-prefix
trace-table composition-table)*

(Some (((prefix, prefix-state), raw), raw-state))

by (*rule*

checked-staged-security-with-actual-alpha-prefix-candidate-pair-query-hit-from-prefixE

[*OF hit*])

show *?thesis*

proof (*cases*

(trace-table, composition-table) ∈

query-header-supported-single-round-partial-table-candidates

prefix-state

(sqp-trace-root prefix)

(sqp-trace-fri-roots prefix)

(sqp-trace-final prefix)

(sqp-alphas prefix)

(sqp-degree prefix)


```

      (sqp-composition-fri-roots prefix)
      (sqp-composition-final prefix)
    i)
  case True
  have single-round-hit:
    checked-staged-query-prefix-dynamic-index-hit
      (staged-query-prefix-single-round-header-supported-query-target i)
      (Some (((prefix, prefix-state), raw), raw-state))
  by (rule
      checked-staged-query-prefix-candidate-pair-hit-imp-single-round-header-target-hit
      [OF True target-hit])
  have
    checked-staged-security-with-actual-alpha-prefix-single-round-header-supported-query-target-hit-at
      i A
      (Some (((((alpha-prefix, alpha-prefix-state), data), attacker-state),
        result), final-state))
  proof -
    have inner:
      ∃ prefix prefix-state raw raw-state.
        Some (data, attacker-state) ∈
          set-dist
            (execute (checked-staged-transcript-program A)
              adversary-initial-state) ∧
        Some (((prefix, prefix-state), raw), raw-state) ∈
          set-dist
            (execute (checked-staged-query-prefix-receive-with-state A i)
              adversary-initial-state) ∧
        checked-staged-query-prefix-dynamic-index-hit
          (staged-query-prefix-single-round-header-supported-query-target i)
          (Some (((prefix, prefix-state), raw), raw-state))
    by (intro exI[of - prefix] exI[of - prefix-state]
        exI[of - raw] exI[of - raw-state])
        (use builder prefix-receive single-round-hit in simp)
    show ?thesis
      unfolding
        checked-staged-security-with-actual-alpha-prefix-single-round-header-supported-query-target-hit-at-def
        using inner by simp
  qed
  then show ?thesis by simp
next
case False
have
  checked-staged-security-with-actual-alpha-prefix-query-prefix-single-round-candidate-binding-gap-at
    i A
    (Some (((((alpha-prefix, alpha-prefix-state), data), attacker-state),
      result), final-state))
  proof -
    have inner:
      ∃ trace-table composition-table prefix prefix-state raw raw-state.

```

```

Some (data, attacker-state) ∈
  set-dist
    (execute (checked-staged-transcript-program A)
      adversary-initial-state) ∧
Some (((prefix, prefix-state), raw), raw-state) ∈
  set-dist
    (execute (checked-staged-query-prefix-receive-with-state A i)
      adversary-initial-state) ∧
checked-staged-query-prefix-dynamic-index-hit
  (staged-query-prefix-candidate-pair-query-target-from-prefix
    trace-table composition-table)
  (Some (((prefix, prefix-state), raw), raw-state)) ∧
(trace-table, composition-table) ∉
  query-header-supported-single-round-partial-table-candidates
    prefix-state
    (sqp-trace-root prefix)
    (sqp-trace-fri-roots prefix)
    (sqp-trace-final prefix)
    (sqp-alphas prefix)
    (sqp-degree prefix)
    (sqp-composition-fri-roots prefix)
    (sqp-composition-final prefix)
  i
by (intro exI[of - trace-table] exI[of - composition-table]
  exI[of - prefix] exI[of - prefix-state]
  exI[of - raw] exI[of - raw-state])
  (use builder prefix-receive target-hit False in simp)
show ?thesis
  unfolding
    checked-staged-security-with-actual-alpha-prefix-query-prefix-single-round-candidate-binding-gap-at-def
  using inner by simp
qed
then show ?thesis by simp
qed
qed

lemma checked-staged-security-with-actual-alpha-prefix-candidate-pair-query-hit-from-prefix-with-witness-imp-single-
assumes hit:
  checked-staged-security-with-actual-alpha-prefix-candidate-pair-query-hit-from-prefix
  A trace-table composition-table i
  (Some (((((alpha-prefix, alpha-prefix-state), data), attacker-state),
    result), final-state))
and witness:
  (trace-openings, composition-openings) ∈
  query-header-supported-partial-opening-witnesses
    (verifier-state-from-adversary attacker-state
      (staged-proof-transcript data))
    (staged-trace-root data)
    (staged-trace-fri-roots data)

```

```

    (staged-trace-final data)
    (staged-alphas data)
    (staged-degree data)
    (staged-composition-fri-roots data)
    (staged-composition-final data)
  and trace-candidate:
    partial-trace-table-candidate trace-table
    ((replicate rounds []) [i := trace-openings])
  and comp-candidate:
    partial-composition-table-candidate composition-table
    ((replicate rounds []) [i := composition-openings])
shows
  checked-staged-security-with-actual-alpha-prefix-single-round-header-supported-query-target-hit-at
  i A
  (Some (((((alpha-prefix, alpha-prefix-state), data), attacker-state),
    result), final-state)) ∨
  checked-staged-security-with-actual-alpha-prefix-query-prefix-single-round-candidate-binding-gap-witness-at
  i A
  (Some (((((alpha-prefix, alpha-prefix-state), data), attacker-state),
    result), final-state))
proof -
  from
    checked-staged-security-with-actual-alpha-prefix-candidate-pair-query-hit-from-prefixE
    [OF hit]
  obtain prefix prefix-state raw raw-state where
    builder:
      Some (data, attacker-state) ∈
        set-dist
        (execute (checked-staged-transcript-program A)
          adversary-initial-state)
  and prefix-receive:
    Some (((prefix, prefix-state), raw), raw-state) ∈
      set-dist
      (execute (checked-staged-query-prefix-receive-with-state A i)
        adversary-initial-state)
  and target-hit:
    checked-staged-query-prefix-dynamic-index-hit
    (staged-query-prefix-candidate-pair-query-target-from-prefix
      trace-table composition-table)
    (Some (((prefix, prefix-state), raw), raw-state))
  by (rule
    checked-staged-security-with-actual-alpha-prefix-candidate-pair-query-hit-from-prefixE
    [OF hit])
show ?thesis
proof (cases
  (trace-table, composition-table) ∈
  query-header-supported-single-round-partial-table-candidates
  prefix-state
  (sqp-trace-root prefix)

```

```

    (sqp-trace-fri-roots prefix)
    (sqp-trace-final prefix)
    (sqp-alphas prefix)
    (sqp-degree prefix)
    (sqp-composition-fri-roots prefix)
    (sqp-composition-final prefix)
  i)
case True
have single-round-hit:
  checked-staged-query-prefix-dynamic-index-hit
  (staged-query-prefix-single-round-header-supported-query-target i)
  (Some (((prefix, prefix-state), raw), raw-state))
by (rule
  checked-staged-query-prefix-candidate-pair-hit-imp-single-round-header-target-hit
  [OF True target-hit])
have
  checked-staged-security-with-actual-alpha-prefix-single-round-header-supported-query-target-hit-at
  i A
  (Some (((((alpha-prefix, alpha-prefix-state), data), attacker-state),
    result), final-state))
proof -
  have inner:
    ∃ prefix prefix-state raw raw-state.
      Some (data, attacker-state) ∈
        set-dist
        (execute (checked-staged-transcript-program A)
          adversary-initial-state) ∧
      Some (((prefix, prefix-state), raw), raw-state) ∈
        set-dist
        (execute (checked-staged-query-prefix-receive-with-state A i)
          adversary-initial-state) ∧
      checked-staged-query-prefix-dynamic-index-hit
      (staged-query-prefix-single-round-header-supported-query-target i)
      (Some (((prefix, prefix-state), raw), raw-state))
  by (intro exI[of - prefix] exI[of - prefix-state]
    exI[of - raw] exI[of - raw-state])
    (use builder prefix-receive single-round-hit in simp)
  show ?thesis
  unfolding
    checked-staged-security-with-actual-alpha-prefix-single-round-header-supported-query-target-hit-at-def
  using inner by simp
qed
then show ?thesis by simp
next
case False
have
  checked-staged-security-with-actual-alpha-prefix-query-prefix-single-round-candidate-binding-gap-witness-at
  i A
  (Some (((((alpha-prefix, alpha-prefix-state), data), attacker-state),

```

$result), final-state))$
proof –
have *inner*:
 $\exists$ *trace-openings composition-openings trace-table composition-table*
prefix prefix-state raw raw-state.
Some (data, attacker-state) ∈
set-dist
(execute (checked-staged-transcript-program A)
adversary-initial-state) ∧
Some (((prefix, prefix-state), raw), raw-state) ∈
set-dist
(execute (checked-staged-query-prefix-receive-with-state A i)
adversary-initial-state) ∧
(trace-openings, composition-openings) ∈
query-header-supported-partial-opening-witnesses
(verifier-state-from-adversary attacker-state
(staged-proof-transcript data))
(staged-trace-root data)
(staged-trace-fri-roots data)
(staged-trace-final data)
(staged-alphas data)
(staged-degree data)
(staged-composition-fri-roots data)
(staged-composition-final data) ∧
partial-trace-table-candidate trace-table
((replicate rounds []) [i := trace-openings]) ∧
partial-composition-table-candidate composition-table
((replicate rounds []) [i := composition-openings]) ∧
checked-staged-query-prefix-dynamic-index-hit
(staged-query-prefix-candidate-pair-query-target-from-prefix
trace-table composition-table)
(Some (((prefix, prefix-state), raw), raw-state)) ∧
(trace-table, composition-table) ∉
query-header-supported-single-round-partial-table-candidates
prefix-state
(sqp-trace-root prefix)
(sqp-trace-fri-roots prefix)
(sqp-trace-final prefix)
(sqp-alphas prefix)
(sqp-degree prefix)
(sqp-composition-fri-roots prefix)
(sqp-composition-final prefix)
i
by (*intro exI[of - trace-openings] exI[of - composition-openings]*
exI[of - trace-table] exI[of - composition-table]
exI[of - prefix] exI[of - prefix-state]
exI[of - raw] exI[of - raw-state])
(use builder prefix-receive witness trace-candidate comp-candidate
target-hit False in simp)

```

show ?thesis
  unfolding
    checked-staged-security-with-actual-alpha-prefix-query-prefix-single-round-candidate-binding-gap-witness-at-def
  using inner by simp
qed
then show ?thesis by simp
qed
qed

lemma checked-staged-security-with-actual-alpha-prefix-candidate-pair-query-hit-from-prefix-with-witness-imp-header
assumes hit:
  checked-staged-security-with-actual-alpha-prefix-candidate-pair-query-hit-from-prefix
  A trace-table composition-table i
  (Some (((((alpha-prefix, alpha-prefix-state), data), attacker-state),
    result), final-state))
and witness:
  (trace-openings, composition-openings) ∈
  query-header-supported-partial-opening-witnesses
  (verifier-state-from-adversary attacker-state
    (staged-proof-transcript data))
  (staged-trace-root data)
  (staged-trace-fri-roots data)
  (staged-trace-final data)
  (staged-alphas data)
  (staged-degree data)
  (staged-composition-fri-roots data)
  (staged-composition-final data)
and trace-candidate:
  partial-trace-table-candidate trace-table
  ((replicate rounds []) [i := trace-openings])
and comp-candidate:
  partial-composition-table-candidate composition-table
  ((replicate rounds []) [i := composition-openings])
shows
  checked-staged-security-with-actual-alpha-prefix-header-supported-query-target-hit-at
  i A
  (Some (((((alpha-prefix, alpha-prefix-state), data), attacker-state),
    result), final-state)) ∨
  checked-staged-security-with-actual-alpha-prefix-query-prefix-candidate-binding-gap-witness-at
  i A
  (Some (((((alpha-prefix, alpha-prefix-state), data), attacker-state),
    result), final-state))
proof –
from
  checked-staged-security-with-actual-alpha-prefix-candidate-pair-query-hit-from-prefixE
  [OF hit]
obtain prefix prefix-state raw raw-state where
  builder:
    Some (data, attacker-state) ∈

```

```

    set-dist
    (execute (checked-staged-transcript-program A)
      adversary-initial-state)
and prefix-receive:
  Some (((prefix, prefix-state), raw), raw-state) ∈
    set-dist
    (execute (checked-staged-query-prefix-receive-with-state A i)
      adversary-initial-state)
and target-hit:
  checked-staged-query-prefix-dynamic-index-hit
  (staged-query-prefix-candidate-pair-query-target-from-prefix
    trace-table composition-table)
  (Some (((prefix, prefix-state), raw), raw-state))
by (rule
  checked-staged-security-with-actual-alpha-prefix-candidate-pair-query-hit-from-prefixE
  [OF hit])
show ?thesis
proof (cases
  (trace-table, composition-table) ∈
    query-header-supported-partial-table-candidates prefix-state
    (sqp-trace-root prefix)
    (sqp-trace-fri-roots prefix)
    (sqp-trace-final prefix)
    (sqp-alphas prefix)
    (sqp-degree prefix)
    (sqp-composition-fri-roots prefix)
    (sqp-composition-final prefix))
  case True
  have header-hit:
    checked-staged-query-prefix-dynamic-index-hit
    staged-query-prefix-header-supported-query-target
    (Some (((prefix, prefix-state), raw), raw-state))
    by (rule
      checked-staged-query-prefix-candidate-pair-hit-imp-header-supported-target-hit
      [OF True target-hit])
  have
    checked-staged-security-with-actual-alpha-prefix-header-supported-query-target-hit-at
    i A
    (Some (((((alpha-prefix, alpha-prefix-state), data), attacker-state),
      result), final-state))
  proof –
  have inner:
    ∃ prefix prefix-state raw raw-state.
    Some (data, attacker-state) ∈
      set-dist
      (execute (checked-staged-transcript-program A)
        adversary-initial-state) ∧
    Some (((prefix, prefix-state), raw), raw-state) ∈
      set-dist

```

```

      (execute (checked-staged-query-prefix-receive-with-state A i)
        adversary-initial-state) ∧
    checked-staged-query-prefix-dynamic-index-hit
    staged-query-prefix-header-supported-query-target
    (Some (((prefix, prefix-state), raw), raw-state))
  by (intro exI[of - prefix] exI[of - prefix-state]
    exI[of - raw] exI[of - raw-state])
    (use builder prefix-receive header-hit in simp)
show ?thesis
  unfolding
    checked-staged-security-with-actual-alpha-prefix-header-supported-query-target-hit-at-def
  using inner by simp
qed
then show ?thesis by simp
next
case False
have
  checked-staged-security-with-actual-alpha-prefix-query-prefix-candidate-binding-gap-witness-at
    i A
    (Some (((((alpha-prefix, alpha-prefix-state), data), attacker-state),
      result), final-state))
proof -
  have inner:
    ∃ trace-openings composition-openings trace-table composition-table
      prefix prefix-state raw raw-state.
    Some (data, attacker-state) ∈
      set-dist
        (execute (checked-staged-transcript-program A)
          adversary-initial-state) ∧
    Some (((prefix, prefix-state), raw), raw-state) ∈
      set-dist
        (execute (checked-staged-query-prefix-receive-with-state A i)
          adversary-initial-state) ∧
    (trace-openings, composition-openings) ∈
      query-header-supported-partial-opening-witnesses
        (verifier-state-from-adversary attacker-state
          (staged-proof-transcript data))
        (staged-trace-root data)
        (staged-trace-fri-roots data)
        (staged-trace-final data)
        (staged-alphas data)
        (staged-degree data)
        (staged-composition-fri-roots data)
        (staged-composition-final data) ∧
    partial-trace-table-candidate trace-table
      ((replicate rounds []) [i := trace-openings]) ∧
    partial-composition-table-candidate composition-table
      ((replicate rounds []) [i := composition-openings]) ∧
    checked-staged-query-prefix-dynamic-index-hit

```



```

    (staged-query-prefix-candidate-pair-query-target-from-prefix
     trace-table composition-table)
    (Some (((prefix, prefix-state), raw), raw-state))  $\wedge$ 
    (trace-table, composition-table)  $\notin$ 
    query-header-supported-partial-table-candidates prefix-state
    (sqp-trace-root prefix)
    (sqp-trace-fri-roots prefix)
    (sqp-trace-final prefix)
    (sqp-alphas prefix)
    (sqp-degree prefix)
    (sqp-composition-fri-roots prefix)
    (sqp-composition-final prefix)
  by (intro exI[of - trace-openings] exI[of - composition-openings]
      exI[of - trace-table] exI[of - composition-table]
      exI[of - prefix] exI[of - prefix-state]
      exI[of - raw] exI[of - raw-state])
    (use builder prefix-receive witness trace-candidate comp-candidate
     target-hit False in simp)
show ?thesis
  unfolding
    checked-staged-security-with-actual-alpha-prefix-query-prefix-candidate-binding-gap-witness-at-def
  using inner by simp
qed
then show ?thesis by simp
qed
qed

lemma checked-staged-security-with-actual-alpha-prefix-header-authenticated-candidate-opening-query-hit-from-prefix-at
  assumes hit:
    checked-staged-security-with-actual-alpha-prefix-header-authenticated-candidate-opening-query-hit-from-prefix-at
    i A
    (Some ((((((alpha-prefix, alpha-prefix-state), data), attacker-state),
      result), final-state)))
  and i-bound: i < rounds
  and trace-low:
     $\wedge$  trace-openings composition-openings trace-table
    composition-table.
    (trace-openings, composition-openings)  $\in$ 
    query-header-supported-partial-opening-witnesses
    (verifier-state-from-adversary attacker-state
     (staged-proof-transcript data))
    (staged-trace-root data)
    (staged-trace-fri-roots data)
    (staged-trace-final data)
    (staged-alphas data)
    (staged-degree data)
    (staged-composition-fri-roots data)
    (staged-composition-final data)  $\implies$ 
    partial-trace-table-candidate trace-table

```

$((\text{replicate rounds } []) [i := \text{trace-openings}]) \implies$
 $\text{partial-composition-table-candidate composition-table}$
 $((\text{replicate rounds } []) [i := \text{composition-openings}]) \implies$
 $\text{trace-table-low-degree trace-table}$
and comp-low:
 $\bigwedge \text{trace-openings composition-openings trace-table}$
 $\text{composition-table.}$
 $(\text{trace-openings, composition-openings}) \in$
 $\text{query-header-supported-partial-opening-witnesses}$
 $(\text{verifier-state-from-adversary attacker-state}$
 $(\text{staged-proof-transcript data}))$
 $(\text{staged-trace-root data})$
 $(\text{staged-trace-fri-roots data})$
 $(\text{staged-trace-final data})$
 $(\text{staged-alphas data})$
 $(\text{staged-degree data})$
 $(\text{staged-composition-fri-roots data})$
 $(\text{staged-composition-final data}) \implies$
 $\text{partial-trace-table-candidate trace-table}$
 $((\text{replicate rounds } []) [i := \text{trace-openings}]) \implies$
 $\text{partial-composition-table-candidate composition-table}$
 $((\text{replicate rounds } []) [i := \text{composition-openings}]) \implies$
 $\text{composition-table-low-degree maxDegree composition-table}$
and not-all:
 $\bigwedge \text{trace-openings composition-openings trace-table}$
 $\text{composition-table prefix prefix-state.}$
 $(\text{trace-openings, composition-openings}) \in$
 $\text{query-header-supported-partial-opening-witnesses}$
 $(\text{verifier-state-from-adversary attacker-state}$
 $(\text{staged-proof-transcript data}))$
 $(\text{staged-trace-root data})$
 $(\text{staged-trace-fri-roots data})$
 $(\text{staged-trace-final data})$
 $(\text{staged-alphas data})$
 $(\text{staged-degree data})$
 $(\text{staged-composition-fri-roots data})$
 $(\text{staged-composition-final data}) \implies$
 $\text{partial-trace-table-candidate trace-table}$
 $((\text{replicate rounds } []) [i := \text{trace-openings}]) \implies$
 $\text{partial-composition-table-candidate composition-table}$
 $((\text{replicate rounds } []) [i := \text{composition-openings}]) \implies$
 $\text{Some } ((\text{prefix, prefix-state}), \text{prefix-state}) \in$
 set-dist
 $(\text{execute } (\text{checked-staged-query-prefix-with-state } A \ i)$
 $\text{adversary-initial-state}) \implies$
 $\neg \text{all-queries-consistent trace-table composition-table}$
 $(\text{sqp-alphas prefix})$
shows
 $(\exists \text{result' final-state'.$

```

    partial-merkle-inconsistency-bad
      (verifier-state-from-adversary attacker-state
        (staged-proof-transcript data))
      (Some (result', final-state')) ∨
checked-staged-security-with-actual-alpha-prefix-header-supported-query-target-hit-at
  i A
  (Some (((((alpha-prefix, alpha-prefix-state), data), attacker-state),
    result), final-state)) ∨
checked-staged-security-with-actual-alpha-prefix-query-prefix-candidate-binding-gap-witness-at
  i A
  (Some (((((alpha-prefix, alpha-prefix-state), data), attacker-state),
    result), final-state))
proof –
  let ?s =
    verifier-state-from-adversary attacker-state
      (staged-proof-transcript data)
  from
    checked-staged-security-with-actual-alpha-prefix-header-authenticated-candidate-opening-query-hit-from-prefix-at-ca
      [OF hit i-bound]
  show ?thesis
  proof
    assume
      ∃ result' final-state'.
        partial-merkle-inconsistency-bad ?s (Some (result', final-state'))
    then show ?thesis by simp
  next
    assume candidates:
      ∃ trace-openings composition-openings trace-table
        composition-table.
      (trace-openings, composition-openings) ∈
        query-header-supported-partial-opening-witnesses ?s
        (staged-trace-root data)
        (staged-trace-fri-roots data)
        (staged-trace-final data)
        (staged-alphas data)
        (staged-degree data)
        (staged-composition-fri-roots data)
        (staged-composition-final data) ∧
checked-staged-security-with-actual-alpha-prefix-candidate-opening-query-hit-from-prefix
  A
  ((replicate rounds []) [i := trace-openings])
  ((replicate rounds []) [i := composition-openings])
  i
  (Some (((((alpha-prefix, alpha-prefix-state), data),
    attacker-state), result), final-state)) ∧
partial-trace-table-candidate trace-table
  ((replicate rounds []) [i := trace-openings]) ∧
partial-composition-table-candidate composition-table
  ((replicate rounds []) [i := composition-openings])

```

then obtain *trace-openings composition-openings trace-table*
composition-table **where** *witness*:
(trace-openings, composition-openings) ∈
query-header-supported-partial-opening-witnesses ?s
(staged-trace-root data)
(staged-trace-fri-roots data)
(staged-trace-final data)
(staged-alphas data)
(staged-degree data)
(staged-composition-fri-roots data)
(staged-composition-final data)
and *opening-hit*:
checked-staged-security-with-actual-alpha-prefix-candidate-opening-query-hit-from-prefix
A
((replicate rounds []) [i := trace-openings])
((replicate rounds []) [i := composition-openings])
i
(Some (((((alpha-prefix, alpha-prefix-state), data),
attacker-state), result), final-state)))
and *trace-candidate*:
partial-trace-table-candidate trace-table
((replicate rounds []) [i := trace-openings])
and *comp-candidate*:
partial-composition-table-candidate composition-table
((replicate rounds []) [i := composition-openings])
by *blast*
have *pair-hit*:
checked-staged-security-with-actual-alpha-prefix-candidate-pair-query-hit-from-prefix
A trace-table composition-table i
(Some (((((alpha-prefix, alpha-prefix-state), data),
attacker-state), result), final-state)))
by *(rule*
checked-staged-security-with-actual-alpha-prefix-candidate-opening-query-hit-from-prefix-imp-candidate-pair-que
[OF opening-hit trace-candidate comp-candidate
trace-low[OF witness trace-candidate comp-candidate]
comp-low[OF witness trace-candidate comp-candidate]])
(rule not-all[OF witness trace-candidate comp-candidate])
have
checked-staged-security-with-actual-alpha-prefix-header-supported-query-target-hit-at
i A
(Some (((((alpha-prefix, alpha-prefix-state), data),
attacker-state), result), final-state))) ∨
checked-staged-security-with-actual-alpha-prefix-query-prefix-candidate-binding-gap-witness-at
i A
(Some (((((alpha-prefix, alpha-prefix-state), data),
attacker-state), result), final-state)))
by *(rule*
checked-staged-security-with-actual-alpha-prefix-candidate-pair-query-hit-from-prefix-with-witness-imp-header-ta
[OF pair-hit witness trace-candidate comp-candidate])

then show *?thesis* by simp
 qed
 qed

lemma *checked-staged-security-with-actual-alpha-prefix-header-authenticated-candidate-opening-query-hit-from-prefix-at*

assumes *hit*:

checked-staged-security-with-actual-alpha-prefix-header-authenticated-candidate-opening-query-hit-from-prefix-at

i A

(Some (((((alpha-prefix, alpha-prefix-state), data), attacker-state),
 result), final-state))

and *i-bound*: $i < \text{rounds}$

and *trace-low*:

$\bigwedge \text{trace-openings composition-openings trace-table}$
 $\text{composition-table.}$

(trace-openings, composition-openings) $\in$
 query-header-supported-partial-opening-witnesses
 (verifier-state-from-adversary attacker-state
 (staged-proof-transcript data))
 (staged-trace-root data)
 (staged-trace-fri-roots data)
 (staged-trace-final data)
 (staged-alphas data)
 (staged-degree data)
 (staged-composition-fri-roots data)
 (staged-composition-final data) $\implies$

partial-trace-table-candidate trace-table

((replicate rounds []) [i := trace-openings]) $\implies$

partial-composition-table-candidate composition-table

((replicate rounds []) [i := composition-openings]) $\implies$

trace-table-low-degree trace-table

and *comp-low*:

$\bigwedge \text{trace-openings composition-openings trace-table}$
 $\text{composition-table.}$

(trace-openings, composition-openings) $\in$
 query-header-supported-partial-opening-witnesses
 (verifier-state-from-adversary attacker-state
 (staged-proof-transcript data))
 (staged-trace-root data)
 (staged-trace-fri-roots data)
 (staged-trace-final data)
 (staged-alphas data)
 (staged-degree data)
 (staged-composition-fri-roots data)
 (staged-composition-final data) $\implies$

partial-trace-table-candidate trace-table

((replicate rounds []) [i := trace-openings]) $\implies$

partial-composition-table-candidate composition-table

((replicate rounds []) [i := composition-openings]) $\implies$

composition-table-low-degree maxDegree composition-table

and not-all:

$\bigwedge$ *trace-openings composition-openings trace-table*
composition-table prefix prefix-state.
(trace-openings, composition-openings) ∈
query-header-supported-partial-opening-witnesses
(verifier-state-from-adversary attacker-state
(staged-proof-transcript data))
(staged-trace-root data)
(staged-trace-fri-roots data)
(staged-trace-final data)
(staged-alphas data)
(staged-degree data)
(staged-composition-fri-roots data)
(staged-composition-final data) ⇒
partial-trace-table-candidate trace-table
((replicate rounds []) [i := trace-openings]) ⇒
partial-composition-table-candidate composition-table
((replicate rounds []) [i := composition-openings]) ⇒
Some ((prefix, prefix-state), prefix-state) ∈
set-dist
(execute (checked-staged-query-prefix-with-state A i)
adversary-initial-state) ⇒
 $\neg$ *all-queries-consistent trace-table composition-table*
(sqp-alphas prefix)

shows

$(\exists \text{result}' \text{final-state}'.$
partial-merkle-inconsistency-bad
(verifier-state-from-adversary attacker-state
(staged-proof-transcript data))
(Some (result', final-state')) ∨
checked-staged-security-with-actual-alpha-prefix-single-round-header-supported-query-target-hit-at
i A
(Some (((((alpha-prefix, alpha-prefix-state), data), attacker-state),
result), final-state))) ∨
checked-staged-security-with-actual-alpha-prefix-query-prefix-single-round-candidate-binding-gap-witness-at
i A
(Some (((((alpha-prefix, alpha-prefix-state), data), attacker-state),
result), final-state)))

proof —

let ?s =

verifier-state-from-adversary attacker-state
(staged-proof-transcript data)

from

checked-staged-security-with-actual-alpha-prefix-header-authenticated-candidate-opening-query-hit-from-prefix-at-ca
[OF hit i-bound]

show ?thesis

proof

assume

$\exists \text{result}' \text{final-state}'.$

```

    partial-merkle-inconsistency-bad ?s (Some (result', final-state'))
  then show ?thesis by simp
next
assume candidates:
  ∃ trace-openings composition-openings trace-table
    composition-table.
  (trace-openings, composition-openings) ∈
    query-header-supported-partial-opening-witnesses ?s
    (staged-trace-root data)
    (staged-trace-fri-roots data)
    (staged-trace-final data)
    (staged-alphas data)
    (staged-degree data)
    (staged-composition-fri-roots data)
    (staged-composition-final data) ∧
  checked-staged-security-with-actual-alpha-prefix-candidate-opening-query-hit-from-prefix
  A
  ((replicate rounds []) [i := trace-openings])
  ((replicate rounds []) [i := composition-openings])
  i
  (Some (((alpha-prefix, alpha-prefix-state), data),
    attacker-state), result), final-state)) ∧
  partial-trace-table-candidate trace-table
  ((replicate rounds []) [i := trace-openings]) ∧
  partial-composition-table-candidate composition-table
  ((replicate rounds []) [i := composition-openings])
then obtain trace-openings composition-openings trace-table
  composition-table where witness:
  (trace-openings, composition-openings) ∈
    query-header-supported-partial-opening-witnesses ?s
    (staged-trace-root data)
    (staged-trace-fri-roots data)
    (staged-trace-final data)
    (staged-alphas data)
    (staged-degree data)
    (staged-composition-fri-roots data)
    (staged-composition-final data)
and opening-hit:
  checked-staged-security-with-actual-alpha-prefix-candidate-opening-query-hit-from-prefix
  A
  ((replicate rounds []) [i := trace-openings])
  ((replicate rounds []) [i := composition-openings])
  i
  (Some (((alpha-prefix, alpha-prefix-state), data),
    attacker-state), result), final-state))
and trace-candidate:
  partial-trace-table-candidate trace-table
  ((replicate rounds []) [i := trace-openings])
and comp-candidate:

```

```

    partial-composition-table-candidate composition-table
      ((replicate rounds []) [i := composition-openings])
  by blast
have pair-hit:
  checked-staged-security-with-actual-alpha-prefix-candidate-pair-query-hit-from-prefix
    A trace-table composition-table i
    (Some (((((alpha-prefix, alpha-prefix-state), data),
      attacker-state), result), final-state))
  by (rule
    checked-staged-security-with-actual-alpha-prefix-candidate-opening-query-hit-from-prefix-imp-candidate-pair-que
      [OF opening-hit trace-candidate comp-candidate
        trace-low[OF witness trace-candidate comp-candidate]
        comp-low[OF witness trace-candidate comp-candidate]])
    (rule not-all[OF witness trace-candidate comp-candidate])
have
  checked-staged-security-with-actual-alpha-prefix-single-round-header-supported-query-target-hit-at
    i A
    (Some (((((alpha-prefix, alpha-prefix-state), data),
      attacker-state), result), final-state)) ∨
  checked-staged-security-with-actual-alpha-prefix-query-prefix-single-round-candidate-binding-gap-witness-at
    i A
    (Some (((((alpha-prefix, alpha-prefix-state), data),
      attacker-state), result), final-state))
  by (rule
    checked-staged-security-with-actual-alpha-prefix-candidate-pair-query-hit-from-prefix-with-witness-imp-single-rou
      [OF pair-hit witness trace-candidate comp-candidate])
  then show ?thesis by simp
qed
qed

lemma checked-staged-security-with-actual-alpha-prefix-candidate-pair-query-hit-from-prefix-imp-header-target-or-bin
  assumes hit:
    checked-staged-security-with-actual-alpha-prefix-candidate-pair-query-hit-from-prefix
      A trace-table composition-table i
      (Some (((((alpha-prefix, alpha-prefix-state), data), attacker-state),
        result), final-state))
  shows
    checked-staged-security-with-actual-alpha-prefix-header-supported-query-target-hit-at
      i A
      (Some (((((alpha-prefix, alpha-prefix-state), data), attacker-state),
        result), final-state)) ∨
    checked-staged-security-with-actual-alpha-prefix-query-prefix-candidate-binding-gap-at
      i A
      (Some (((((alpha-prefix, alpha-prefix-state), data), attacker-state),
        result), final-state))
proof –
  from
    checked-staged-security-with-actual-alpha-prefix-candidate-pair-query-hit-from-prefixE
      [OF hit]

```


obtain *prefix prefix-state raw raw-state* **where**
i-bound: i < rounds
and *builder:*
Some (data, attacker-state) ∈
set-dist
(execute (checked-staged-transcript-program A)
adversary-initial-state)
and *prefix-receive:*
Some (((prefix, prefix-state), raw), raw-state) ∈
set-dist
(execute (checked-staged-query-prefix-receive-with-state A i)
adversary-initial-state)
and *target-hit:*
checked-staged-query-prefix-dynamic-index-hit
(staged-query-prefix-candidate-pair-query-target-from-prefix
trace-table composition-table)
(Some (((prefix, prefix-state), raw), raw-state))
by (rule
checked-staged-security-with-actual-alpha-prefix-candidate-pair-query-hit-from-prefixE
[OF hit])
show *?thesis*
proof (cases
(trace-table, composition-table) ∈
query-header-supported-partial-table-candidates prefix-state
(sqp-trace-root prefix)
(sqp-trace-fri-roots prefix)
(sqp-trace-final prefix)
(sqp-alphas prefix)
(sqp-degree prefix)
(sqp-composition-fri-roots prefix)
(sqp-composition-final prefix))
case *True*
have *header-hit:*
checked-staged-query-prefix-dynamic-index-hit
staged-query-prefix-header-supported-query-target
(Some (((prefix, prefix-state), raw), raw-state))
by (rule
checked-staged-query-prefix-candidate-pair-hit-imp-header-supported-target-hit
[OF True target-hit])
have
checked-staged-security-with-actual-alpha-prefix-header-supported-query-target-hit-at
i A
(Some ((((((alpha-prefix, alpha-prefix-state), data), attacker-state),
result), final-state))
proof –
have *inner:*
∃ prefix prefix-state raw raw-state.
Some (((prefix, prefix-state), raw), raw-state) ∈
set-dist

```

      (execute (checked-staged-query-prefix-receive-with-state A i)
        adversary-initial-state) ∧
    checked-staged-query-prefix-dynamic-index-hit
    staged-query-prefix-header-supported-query-target
    (Some (((prefix, prefix-state), raw), raw-state))
  by (intro exI[of - prefix] exI[of - prefix-state]
    exI[of - raw] exI[of - raw-state])
    (use prefix-receive header-hit in simp)
show ?thesis
  unfolding
    checked-staged-security-with-actual-alpha-prefix-header-supported-query-target-hit-at-def
  using builder inner by simp
qed
then show ?thesis by simp
next
case False
have
  checked-staged-security-with-actual-alpha-prefix-query-prefix-candidate-binding-gap-at
  i A
  (Some (((((alpha-prefix, alpha-prefix-state), data), attacker-state),
    result), final-state))
proof -
  have inner:
    ∃ trace-table composition-table prefix prefix-state raw raw-state.
      Some (((prefix, prefix-state), raw), raw-state) ∈
        set-dist
        (execute (checked-staged-query-prefix-receive-with-state A i)
          adversary-initial-state) ∧
      checked-staged-query-prefix-dynamic-index-hit
      (staged-query-prefix-candidate-pair-query-target-from-prefix
        trace-table composition-table)
      (Some (((prefix, prefix-state), raw), raw-state)) ∧
      (trace-table, composition-table) ∉
        query-header-supported-partial-table-candidates prefix-state
        (sqp-trace-root prefix)
        (sqp-trace-fri-roots prefix)
        (sqp-trace-final prefix)
        (sqp-alphas prefix)
        (sqp-degree prefix)
        (sqp-composition-fri-roots prefix)
        (sqp-composition-final prefix)
  by (intro exI[of - trace-table] exI[of - composition-table]
    exI[of - prefix] exI[of - prefix-state]
    exI[of - raw] exI[of - raw-state])
    (use prefix-receive target-hit False in simp)
show ?thesis
  unfolding
    checked-staged-security-with-actual-alpha-prefix-query-prefix-candidate-binding-gap-at-def
  using builder inner by simp

```

qed
 then show ?thesis by simp
 qed
 qed

lemma *checked-staged-security-with-actual-alpha-prefix-header-authenticated-candidate-opening-query-hit-from-prefix-at*

assumes *hit*:

checked-staged-security-with-actual-alpha-prefix-header-authenticated-candidate-opening-query-hit-from-prefix-at
i A

(Some (((((alpha-prefix, alpha-prefix-state), data), attacker-state),
 result), final-state))

and *i-bound*: $i < \text{rounds}$

and *trace-low*:

$\bigwedge \text{trace-openings composition-openings trace-table}$
composition-table.

(trace-openings, composition-openings) $\in$
query-header-supported-partial-opening-witnesses
 (verifier-state-from-adversary attacker-state
 (staged-proof-transcript data))
 (staged-trace-root data)
 (staged-trace-fri-roots data)
 (staged-trace-final data)
 (staged-alphas data)
 (staged-degree data)
 (staged-composition-fri-roots data)
 (staged-composition-final data) $\implies$

partial-trace-table-candidate trace-table

((replicate rounds []) [i := trace-openings]) $\implies$

partial-composition-table-candidate composition-table

((replicate rounds []) [i := composition-openings]) $\implies$

trace-table-low-degree trace-table

and *comp-low*:

$\bigwedge \text{trace-openings composition-openings trace-table}$
composition-table.

(trace-openings, composition-openings) $\in$
query-header-supported-partial-opening-witnesses
 (verifier-state-from-adversary attacker-state
 (staged-proof-transcript data))
 (staged-trace-root data)
 (staged-trace-fri-roots data)
 (staged-trace-final data)
 (staged-alphas data)
 (staged-degree data)
 (staged-composition-fri-roots data)
 (staged-composition-final data) $\implies$

partial-trace-table-candidate trace-table

((replicate rounds []) [i := trace-openings]) $\implies$

partial-composition-table-candidate composition-table

((replicate rounds []) [i := composition-openings]) $\implies$

composition-table-low-degree maxDegree composition-table
and not-all:

$\wedge$ *trace-openings composition-openings trace-table*
composition-table prefix prefix-state.
(trace-openings, composition-openings) $\in$
query-header-supported-partial-opening-witnesses
(verifier-state-from-adversary attacker-state
(staged-proof-transcript data))
(staged-trace-root data)
(staged-trace-fri-roots data)
(staged-trace-final data)
(staged-alphas data)
(staged-degree data)
(staged-composition-fri-roots data)
(staged-composition-final data) $\implies$
partial-trace-table-candidate trace-table
((replicate rounds []) [i := trace-openings]) $\implies$
partial-composition-table-candidate composition-table
((replicate rounds []) [i := composition-openings]) $\implies$
Some ((prefix, prefix-state), prefix-state) $\in$
set-dist
(execute (checked-staged-query-prefix-with-state A i)
adversary-initial-state) $\implies$
 $\neg$ *all-queries-consistent trace-table composition-table*
(sqp-alphas prefix)

shows

$(\exists \text{result}' \text{ final-state'}. \text{partial-merkle-inconsistency-bad}$
(verifier-state-from-adversary attacker-state
(staged-proof-transcript data))
(Some (result', final-state')) $\vee$
checked-staged-security-with-actual-alpha-prefix-header-supported-query-target-hit-at
i A
(Some (((((alpha-prefix, alpha-prefix-state), data), attacker-state),
result), final-state))) $\vee$
checked-staged-security-with-actual-alpha-prefix-query-prefix-candidate-binding-gap-at
i A
(Some (((((alpha-prefix, alpha-prefix-state), data), attacker-state),
result), final-state)))

proof –

have base:

$(\exists \text{result}' \text{ final-state'}. \text{partial-merkle-inconsistency-bad}$
(verifier-state-from-adversary attacker-state
(staged-proof-transcript data))
(Some (result', final-state')) $\vee$
 $(\exists \text{trace-table composition-table}.$
checked-staged-security-with-actual-alpha-prefix-candidate-pair-query-hit-from-prefix
A trace-table composition-table i

```

      (Some (((alpha-prefix, alpha-prefix-state), data),
        attacker-state), result), final-state)))
  by (rule
    checked-staged-security-with-actual-alpha-prefix-header-authenticated-candidate-opening-query-hit-from-prefix-at-
    [OF hit i-bound trace-low comp-low not-all])
  then show ?thesis
proof
  assume
     $\exists$  result' final-state'.
    partial-merkle-inconsistency-bad
    (verifier-state-from-adversary attacker-state
      (staged-proof-transcript data))
    (Some (result', final-state'))
  then show ?thesis by simp
next
  assume
     $\exists$  trace-table composition-table.
    checked-staged-security-with-actual-alpha-prefix-candidate-pair-query-hit-from-prefix
    A trace-table composition-table i
    (Some (((alpha-prefix, alpha-prefix-state), data),
      attacker-state), result), final-state))
  then obtain trace-table composition-table where pair-hit:
    checked-staged-security-with-actual-alpha-prefix-candidate-pair-query-hit-from-prefix
    A trace-table composition-table i
    (Some (((alpha-prefix, alpha-prefix-state), data),
      attacker-state), result), final-state))
  by blast
  have
    checked-staged-security-with-actual-alpha-prefix-header-supported-query-target-hit-at
    i A
    (Some (((alpha-prefix, alpha-prefix-state), data),
      attacker-state), result), final-state))  $\vee$ 
    checked-staged-security-with-actual-alpha-prefix-query-prefix-candidate-binding-gap-at
    i A
    (Some (((alpha-prefix, alpha-prefix-state), data),
      attacker-state), result), final-state))
  by (rule
    checked-staged-security-with-actual-alpha-prefix-candidate-pair-query-hit-from-prefix-imp-header-target-or-binding-gap-at
    [OF pair-hit])
  then show ?thesis by simp
qed
qed
end
end

```

theory *Staged-Security-Experiment-Composition-Query-Single-Round*

imports *Staged-Security-Experiment-Composition-Query-Binding-Gap*
begin

Same-run single-round candidate binding for the composition query branch.

The actual-alpha diagnostic events in the previous layer intentionally record enough data to relate a candidate pair to a query-prefix target, but they are too broad for Merkle path-output accounting because the query-prefix branch is existential. This layer states the single-round binding gap directly over the query-prefix data-state experiment, where prefix state, sampled query, verifier continuation, and authenticated openings are from the same run.

context *soundness*
begin

definition *checked-staged-security-with-query-prefix-single-round-header-supported-query-target-hit-at*
where

checked-staged-security-with-query-prefix-single-round-header-supported-query-target-hit-at
 $i \text{ out} \longleftrightarrow$
 (case out of
 None $\Rightarrow$ False
 | Some (((((prefix, prefix-state), raw), raw-state), continuation),
 final-state) $\Rightarrow$
 checked-staged-query-prefix-dynamic-index-hit
 (staged-query-prefix-single-round-header-supported-query-target i)
 (Some (((prefix, prefix-state), raw), raw-state)))

lemma *checked-staged-security-with-query-prefix-single-round-header-supported-query-target-hit-at-iff-dynamic:*
checked-staged-security-with-query-prefix-single-round-header-supported-query-target-hit-at

$i \text{ out} \longleftrightarrow$
checked-staged-security-with-query-prefix-dynamic-index-hit
 (staged-query-prefix-single-round-header-supported-query-target i) out

unfolding

checked-staged-security-with-query-prefix-single-round-header-supported-query-target-hit-at-def
checked-staged-security-with-query-prefix-dynamic-index-hit-def
by (cases out) (auto split: prod.splits)

lemma *checked-staged-security-with-query-prefix-single-round-header-supported-query-target-hit-at-bound:*

assumes *prefix-bound:*

wp-event (checked-staged-query-prefix-receive-with-state A i)
 (checked-staged-query-prefix-dynamic-index-hit
 (staged-query-prefix-single-round-header-supported-query-target i))
adversary-initial-state $\leq B$

shows

wp-event
 (checked-staged-security-experiment-with-query-prefix-data-state A i)
 (checked-staged-security-with-query-prefix-single-round-header-supported-query-target-hit-at
 i)
adversary-initial-state $\leq B$

unfolding

checked-staged-security-with-query-prefix-single-round-header-supported-query-target-hit-at-iff-dynamic
by (*rule checked-staged-security-with-query-prefix-dynamic-index-hit-bound*
 [OF *prefix-bound*])

lemma *staged-query-prefix-candidate-pair-query-target-from-prefix-empty-if-all-queries-consistent:*

assumes *all-queries:*

all-queries-consistent trace-table composition-table (sqp-alphas prefix)

shows

staged-query-prefix-candidate-pair-query-target-from-prefix trace-table
composition-table prefix prefix-state = {}

using *all-queries*

unfolding *staged-query-prefix-candidate-pair-query-target-from-prefix-def*

query-sampling-success-space-def

by *simp*

lemma *checked-staged-query-prefix-candidate-pair-hit-from-prefix-false-if-all-queries-consistent:*

assumes *all-queries:*

all-queries-consistent trace-table composition-table (sqp-alphas prefix)

shows

$\neg$ *checked-staged-query-prefix-dynamic-index-hit*
(staged-query-prefix-candidate-pair-query-target-from-prefix trace-table
composition-table)
(Some (((prefix, prefix-state), raw), raw-state))

using

staged-query-prefix-candidate-pair-query-target-from-prefix-empty-if-all-queries-consistent
 [OF *all-queries, of prefix-state*]

unfolding *checked-staged-query-prefix-dynamic-index-hit-def*

by *simp*

definition *checked-staged-security-with-query-prefix-single-round-candidate-binding-gap-side*

where

checked-staged-security-with-query-prefix-single-round-candidate-binding-gap-side

trace-openings composition-openings trace-table composition-table i out

$\longleftrightarrow$

(*case out of*

None $\Rightarrow$ *False*

| *Some (((((prefix, prefix-state), raw), raw-state),*

((data, attacker-state), result)), final-state) $\Rightarrow$

partial-trace-table-candidate trace-table

((replicate rounds []) [i := trace-openings]) $\wedge$

partial-composition-table-candidate composition-table

((replicate rounds []) [i := composition-openings]) $\wedge$

(trace-table, composition-table) $\notin$

query-header-supported-single-round-partial-table-candidates

prefix-state

(sqp-trace-root prefix)

(sqp-trace-fri-roots prefix)

(sqp-trace-final prefix)

(sqp-alphas prefix)

(sqp-degree prefix)
 (sqp-composition-fri-roots prefix)
 (sqp-composition-final prefix)
 i)

definition *checked-staged-security-with-query-prefix-single-round-candidate-binding-gap-at*
where

checked-staged-security-with-query-prefix-single-round-candidate-binding-gap-at
trace-openings composition-openings trace-table composition-table i out
 $\longleftrightarrow$
 (case out of
 None $\Rightarrow$ False
 | Some (((((prefix, prefix-state), raw), raw-state),
 ((data, attacker-state), result)), final-state) $\Rightarrow$
 checked-staged-security-with-query-prefix-header-authenticated-candidate-opening-hit
 trace-openings composition-openings i out $\wedge$
 partial-trace-table-candidate trace-table
 ((replicate rounds []) [i := trace-openings]) $\wedge$
 partial-composition-table-candidate composition-table
 ((replicate rounds []) [i := composition-openings]) $\wedge$
 (trace-table, composition-table) $\notin$
 query-header-supported-single-round-partial-table-candidates
 prefix-state
 (sqp-trace-root prefix)
 (sqp-trace-fri-roots prefix)
 (sqp-trace-final prefix)
 (sqp-alphas prefix)
 (sqp-degree prefix)
 (sqp-composition-fri-roots prefix)
 (sqp-composition-final prefix)
 i)

definition *checked-staged-security-with-query-prefix-prefix-single-round-candidate-binding-gap-at*
where

checked-staged-security-with-query-prefix-prefix-single-round-candidate-binding-gap-at
trace-openings composition-openings trace-table composition-table i out
 $\longleftrightarrow$
checked-staged-security-with-query-prefix-single-round-candidate-binding-gap-side
trace-openings composition-openings trace-table composition-table i out $\wedge$
checked-staged-security-with-query-prefix-prefix-authenticated-opening-hit
trace-openings composition-openings i out

lemma *checked-staged-security-with-query-prefix-single-round-candidate-binding-gap-iff-fixed-header-component:*

checked-staged-security-with-query-prefix-single-round-candidate-binding-gap-at
trace-openings composition-openings trace-table composition-table i out
 $\longleftrightarrow$
checked-staged-security-with-query-prefix-fixed-header-component
 (checked-staged-security-with-query-prefix-single-round-candidate-binding-gap-side
 trace-openings composition-openings trace-table composition-table i)

trace-openings composition-openings i out
unfolding
checked-staged-security-with-query-prefix-single-round-candidate-binding-gap-at-def
checked-staged-security-with-query-prefix-single-round-candidate-binding-gap-side-def
checked-staged-security-with-query-prefix-fixed-header-component-def
by (*cases out*) (*auto split: prod.splits*)

lemma *checked-staged-security-with-query-prefix-prefix-single-round-candidate-binding-gap-imp-current-prefix-authen*
assumes *gap:*
checked-staged-security-with-query-prefix-prefix-single-round-candidate-binding-gap-at
trace-openings composition-openings trace-table composition-table i out
shows
checked-staged-security-with-query-prefix-current-prefix-authenticated-hit-at
i out
using *gap*
unfolding
checked-staged-security-with-query-prefix-prefix-single-round-candidate-binding-gap-at-def
checked-staged-security-with-query-prefix-current-prefix-authenticated-hit-at-def
by *blast*

lemma *checked-staged-security-with-query-prefix-prefix-single-round-candidate-binding-gap-bound-from-dynamic-targ*
assumes *wf: staged-budget-wellformed budgets*
and *controlled: staged-adversary-controlled budgets A*
and *i-bound: i < rounds*
and *prefix-bound:*
wp-event (checked-staged-query-prefix-receive-with-state A i)
(checked-staged-query-prefix-dynamic-index-hit
staged-query-prefix-prefix-authenticated-opening-target)
adversary-initial-state ≤ B
shows
wp-event
(checked-staged-security-experiment-with-query-prefix-data-state A i)
(checked-staged-security-with-query-prefix-prefix-single-round-candidate-binding-gap-at
trace-openings composition-openings trace-table composition-table i)
adversary-initial-state ≤ B
proof –
have *event-le:*
wp-event
(checked-staged-security-experiment-with-query-prefix-data-state A i)
(checked-staged-security-with-query-prefix-prefix-single-round-candidate-binding-gap-at
trace-openings composition-openings trace-table composition-table i)
adversary-initial-state ≤
wp-event
(checked-staged-security-experiment-with-query-prefix-data-state A i)
(checked-staged-security-with-query-prefix-current-prefix-authenticated-hit-at
i)
adversary-initial-state
by (*rule wp-event-mono*)
(rule

checked-staged-security-with-query-prefix-prefix-single-round-candidate-binding-gap-imp-current-prefix-authenticat
have *current-bound*:
wp-event
(checked-staged-security-experiment-with-query-prefix-data-state A i)
(checked-staged-security-with-query-prefix-current-prefix-authenticated-hit-at
i)
adversary-initial-state ≤ B
by (rule
checked-staged-security-with-query-prefix-current-prefix-authenticated-hit-at-bound-from-dynamic-target
[OF wf controlled i-bound prefix-bound])
show *?thesis*
by (rule *order-trans[OF event-le current-bound]*)
qed

lemma *checked-staged-security-with-query-prefix-header-authenticated-candidate-opening-hit-imp-single-round-header*
assumes *hit*:

checked-staged-security-with-query-prefix-header-authenticated-candidate-opening-hit
trace-openings composition-openings i
(Some (((((prefix, prefix-state), raw), raw-state),
((data, attacker-state), result)), final-state))

and *trace-candidate*:

partial-trace-table-candidate trace-table
((replicate rounds []) [i := trace-openings])

and *comp-candidate*:

partial-composition-table-candidate composition-table
((replicate rounds []) [i := composition-openings])

and *trace-low*: *trace-table-low-degree trace-table*

and *comp-low*: *composition-table-low-degree maxDegree composition-table*

and *not-all*:

$\neg$ *all-queries-consistent trace-table composition-table*
(sqp-alphas prefix)

shows

checked-staged-security-with-query-prefix-single-round-header-supported-query-target-hit-at
i
(Some (((((prefix, prefix-state), raw), raw-state),
((data, attacker-state), result)), final-state)) ∨
checked-staged-security-with-query-prefix-single-round-candidate-binding-gap-at
trace-openings composition-openings trace-table composition-table i
(Some (((((prefix, prefix-state), raw), raw-state),
((data, attacker-state), result)), final-state))

proof –

from *hit* **have** *opening-target*:

index (to-nat raw) ∈
staged-query-prefix-candidate-opening-query-target-from-prefix
((replicate rounds []) [i := trace-openings])
((replicate rounds []) [i := composition-openings])
i prefix prefix-state

by (rule

checked-staged-security-with-query-prefix-header-authenticated-candidate-opening-hitE)

```

have opening-subset:
  staged-query-prefix-candidate-opening-query-target-from-prefix
    ((replicate rounds []) [i := trace-openings])
    ((replicate rounds []) [i := composition-openings])
  i prefix prefix-state  $\subseteq$ 
  staged-query-prefix-candidate-pair-query-target-from-prefix trace-table
  composition-table prefix prefix-state
by (rule
  staged-query-prefix-candidate-opening-query-target-from-prefix-subset-candidate-pair-from-prefix
    [OF trace-candidate comp-candidate trace-low comp-low not-all])
have pair-target-hit:
  checked-staged-query-prefix-dynamic-index-hit
    (staged-query-prefix-candidate-pair-query-target-from-prefix
      trace-table composition-table)
    (Some (((prefix, prefix-state), raw), raw-state))
using opening-target opening-subset
unfolding checked-staged-query-prefix-dynamic-index-hit-def by auto
show ?thesis
proof (cases
  (trace-table, composition-table)  $\in$ 
  query-header-supported-single-round-partial-table-candidates
    prefix-state
    (sqp-trace-root prefix)
    (sqp-trace-fri-roots prefix)
    (sqp-trace-final prefix)
    (sqp-alphas prefix)
    (sqp-degree prefix)
    (sqp-composition-fri-roots prefix)
    (sqp-composition-final prefix)
  i)
case True
have target:
  checked-staged-query-prefix-dynamic-index-hit
    (staged-query-prefix-single-round-header-supported-query-target i)
    (Some (((prefix, prefix-state), raw), raw-state))
by (rule
  checked-staged-query-prefix-candidate-pair-hit-imp-single-round-header-target-hit
    [OF True pair-target-hit])
have
  checked-staged-security-with-query-prefix-single-round-header-supported-query-target-hit-at
    i
    (Some ((((((prefix, prefix-state), raw), raw-state),
      ((data, attacker-state), result)), final-state))
unfolding
  checked-staged-security-with-query-prefix-single-round-header-supported-query-target-hit-at-def
using target by simp
then show ?thesis by simp
next
case False

```

```

have
  checked-staged-security-with-query-prefix-single-round-candidate-binding-gap-at
    trace-openings composition-openings trace-table composition-table i
    (Some (((((prefix, prefix-state), raw), raw-state),
      ((data, attacker-state), result)), final-state))
unfolding
  checked-staged-security-with-query-prefix-single-round-candidate-binding-gap-at-def
using hit trace-candidate comp-candidate False by simp
then show ?thesis by simp
qed
qed

lemma checked-staged-security-with-query-prefix-single-round-candidate-binding-gap-current-imp-prefix-gap-or-openings
assumes wf: staged-budget-wellformed budgets
and controlled: staged-adversary-controlled budgets A
and i-bound: i < rounds
and support:
  out ∈
    set-dist
    (execute
      (checked-staged-security-experiment-with-query-prefix-data-state A i)
      adversary-initial-state)
and current:
  checked-staged-security-with-query-prefix-fixed-current-authenticated-component
    (checked-staged-security-with-query-prefix-single-round-candidate-binding-gap-side
      trace-openings composition-openings trace-table composition-table i)
  trace-openings composition-openings i out
shows
  checked-staged-security-with-query-prefix-prefix-single-round-candidate-binding-gap-at
    trace-openings composition-openings trace-table composition-table i out ∨
  checked-staged-security-with-query-prefix-opening-path-output-hit
    trace-openings composition-openings out
proof —
have side:
  checked-staged-security-with-query-prefix-single-round-candidate-binding-gap-side
    trace-openings composition-openings trace-table composition-table i out
and current-hit:
  checked-staged-security-with-query-prefix-authenticated-opening-hit
    trace-openings composition-openings i out
using current
unfolding
  checked-staged-security-with-query-prefix-fixed-current-authenticated-component-def
  checked-staged-security-with-query-prefix-fixed-header-component-def
by simp-all
have split:
  checked-staged-security-with-query-prefix-prefix-authenticated-opening-hit
    trace-openings composition-openings i out ∨
  checked-staged-security-with-query-prefix-opening-path-output-hit
    trace-openings composition-openings out

```

by (rule
 checked-staged-security-with-query-prefix-fixed-current-authenticated-component-imp-prefix-or-path-output-on-sup
 [*OF wf controlled i-bound support current*])
 then show ?thesis
 proof
 assume *prefix*:
 checked-staged-security-with-query-prefix-prefix-authenticated-opening-hit
 trace-openings composition-openings i out
 have
 checked-staged-security-with-query-prefix-prefix-single-round-candidate-binding-gap-at
 trace-openings composition-openings trace-table composition-table i out
 unfolding
 checked-staged-security-with-query-prefix-prefix-single-round-candidate-binding-gap-at-def
 using *side prefix by simp*
 then show ?thesis by *simp*
 next
 assume
 checked-staged-security-with-query-prefix-opening-path-output-hit
 trace-openings composition-openings out
 then show ?thesis by *simp*
 qed
 qed

lemma *checked-staged-security-with-query-prefix-single-round-candidate-binding-gap-imp-current-or-path-output-on-sup*
assumes *wf: staged-budget-wellformed budgets*
and *controlled: staged-adversary-controlled budgets A*
and *i-bound: i < rounds*
and *support:*
 out ∈
 set-dist
 (*execute*
 (*checked-staged-security-experiment-with-query-prefix-data-state A i*)
 adversary-initial-state)
and *gap:*
 checked-staged-security-with-query-prefix-single-round-candidate-binding-gap-at
 trace-openings composition-openings trace-table composition-table i out
shows
 checked-staged-security-with-query-prefix-fixed-current-authenticated-component
 (*checked-staged-security-with-query-prefix-single-round-candidate-binding-gap-side*
 trace-openings composition-openings trace-table composition-table i)
 trace-openings composition-openings i out ∨
 checked-staged-security-with-query-prefix-header-witness-path-output-hit
 (*checked-staged-security-with-query-prefix-single-round-candidate-binding-gap-side*
 trace-openings composition-openings trace-table composition-table i)
 trace-openings composition-openings i out
proof —
have *fixed:*
 checked-staged-security-with-query-prefix-fixed-header-component
 (*checked-staged-security-with-query-prefix-single-round-candidate-binding-gap-side*

```

    trace-openings composition-openings trace-table composition-table i)
  trace-openings composition-openings i out
using gap
unfolding
  checked-staged-security-with-query-prefix-single-round-candidate-binding-gap-iff-fixed-header-component
.
show ?thesis
by (rule
  checked-staged-security-with-query-prefix-fixed-header-component-imp-current-or-witness-path-output-on-support
  [OF wf controlled i-bound support fixed])
qed

lemma checked-staged-security-with-query-prefix-single-round-candidate-binding-gap-imp-prefix-gap-or-path-outputs-c
assumes wf: staged-budget-wellformed budgets
and controlled: staged-adversary-controlled budgets A
and i-bound: i < rounds
and support:
  out ∈
  set-dist
  (execute
    (checked-staged-security-experiment-with-query-prefix-data-state A i)
    adversary-initial-state)
and gap:
  checked-staged-security-with-query-prefix-single-round-candidate-binding-gap-at
  trace-openings composition-openings trace-table composition-table i out
shows
  checked-staged-security-with-query-prefix-prefix-single-round-candidate-binding-gap-at
  trace-openings composition-openings trace-table composition-table i out ∨
  checked-staged-security-with-query-prefix-opening-path-output-hit
  trace-openings composition-openings out ∨
  checked-staged-security-with-query-prefix-header-witness-path-output-hit
  (checked-staged-security-with-query-prefix-single-round-candidate-binding-gap-side
    trace-openings composition-openings trace-table composition-table i)
  trace-openings composition-openings i out
proof —
have split:
  checked-staged-security-with-query-prefix-fixed-current-authenticated-component
  (checked-staged-security-with-query-prefix-single-round-candidate-binding-gap-side
    trace-openings composition-openings trace-table composition-table i)
  trace-openings composition-openings i out ∨
  checked-staged-security-with-query-prefix-header-witness-path-output-hit
  (checked-staged-security-with-query-prefix-single-round-candidate-binding-gap-side
    trace-openings composition-openings trace-table composition-table i)
  trace-openings composition-openings i out
by (rule
  checked-staged-security-with-query-prefix-single-round-candidate-binding-gap-imp-current-or-path-output-on-suppp
  [OF wf controlled i-bound support gap])
then show ?thesis
proof

```

```

assume current:
  checked-staged-security-with-query-prefix-fixed-current-authenticated-component
    (checked-staged-security-with-query-prefix-single-round-candidate-binding-gap-side
      trace-openings composition-openings trace-table composition-table i)
    trace-openings composition-openings i out
have
  checked-staged-security-with-query-prefix-prefix-single-round-candidate-binding-gap-at
    trace-openings composition-openings trace-table composition-table i out  $\vee$ 
  checked-staged-security-with-query-prefix-opening-path-output-hit
    trace-openings composition-openings out
by (rule
  checked-staged-security-with-query-prefix-single-round-candidate-binding-gap-current-imp-prefix-gap-or-opening
    [OF wf controlled i-bound support current])
then show ?thesis by blast
next
assume
  checked-staged-security-with-query-prefix-header-witness-path-output-hit
    (checked-staged-security-with-query-prefix-single-round-candidate-binding-gap-side
      trace-openings composition-openings trace-table composition-table i)
    trace-openings composition-openings i out
then show ?thesis by simp
qed
qed

lemma checked-staged-security-with-query-prefix-single-round-candidate-binding-gap-bound-from-prefix-target-and-pa
assumes wf: staged-budget-wellformed budgets
and controlled: staged-adversary-controlled budgets A
and i-bound: i < rounds
and prefix-bound:
  wp-event (checked-staged-query-prefix-receive-with-state A i)
  (checked-staged-query-prefix-dynamic-index-hit
    staged-query-prefix-prefix-authenticated-opening-target)
  adversary-initial-state  $\leq P$ 
and opening-path-bound:
  wp-event
    (checked-staged-security-experiment-with-query-prefix-data-state A i)
    (checked-staged-security-with-query-prefix-opening-path-output-hit
      trace-openings composition-openings)
    adversary-initial-state  $\leq Q$ 
and witness-path-bound:
  wp-event
    (checked-staged-security-experiment-with-query-prefix-data-state A i)
    (checked-staged-security-with-query-prefix-header-witness-path-output-hit
      (checked-staged-security-with-query-prefix-single-round-candidate-binding-gap-side
        trace-openings composition-openings trace-table composition-table i)
      trace-openings composition-openings i)
    adversary-initial-state  $\leq R$ 
shows
  wp-event

```

```

    (checked-staged-security-experiment-with-query-prefix-data-state A i)
    (checked-staged-security-with-query-prefix-single-round-candidate-binding-gap-at
      trace-openings composition-openings trace-table composition-table i)
    adversary-initial-state  $\leq P + Q + R$ 
proof –
  let ?m =
    checked-staged-security-experiment-with-query-prefix-data-state A i
  let ?prefix =
    checked-staged-security-with-query-prefix-prefix-single-round-candidate-binding-gap-at
      trace-openings composition-openings trace-table composition-table i
  let ?open-path =
    checked-staged-security-with-query-prefix-opening-path-output-hit
      trace-openings composition-openings
  let ?witness-path =
    checked-staged-security-with-query-prefix-header-witness-path-output-hit
      (checked-staged-security-with-query-prefix-single-round-candidate-binding-gap-side
        trace-openings composition-openings trace-table composition-table i)
      trace-openings composition-openings i
  have event-le:
    wp-event ?m
      (checked-staged-security-with-query-prefix-single-round-candidate-binding-gap-at
        trace-openings composition-openings trace-table composition-table i)
      adversary-initial-state  $\leq$ 
    wp-event ?m
      ( $\lambda$ out. ?prefix out  $\vee$  ?open-path out  $\vee$  ?witness-path out)
      adversary-initial-state
  by (rule wp-event-mono-on-support)
    (rule
      checked-staged-security-with-query-prefix-single-round-candidate-binding-gap-imp-prefix-gap-or-path-outputs-on-s
        [OF wf controlled i-bound])
  have union-le:
    wp-event ?m
      ( $\lambda$ out. ?prefix out  $\vee$  ?open-path out  $\vee$  ?witness-path out)
      adversary-initial-state  $\leq$ 
    wp-event ?m ?prefix adversary-initial-state +
    (wp-event ?m ?open-path adversary-initial-state +
      wp-event ?m ?witness-path adversary-initial-state)
  proof –
  have
    wp-event ?m
      ( $\lambda$ out. ?prefix out  $\vee$  ?open-path out  $\vee$  ?witness-path out)
      adversary-initial-state  $\leq$ 
    wp-event ?m ?prefix adversary-initial-state +
    wp-event ?m
      ( $\lambda$ out. ?open-path out  $\vee$  ?witness-path out)
      adversary-initial-state
  by (rule wp-event-union-bound)
  also have
    ...  $\leq$ 

```



```

    wp-event ?m ?prefix adversary-initial-state +
    (wp-event ?m ?open-path adversary-initial-state +
     wp-event ?m ?witness-path adversary-initial-state)
  by (intro add-mono order-refl wp-event-union-bound)
  finally show ?thesis .
qed
have prefix-gap-bound:
  wp-event ?m ?prefix adversary-initial-state ≤ P
  by (rule
    checked-staged-security-with-query-prefix-prefix-single-round-candidate-binding-gap-bound-from-dynamic-target
    [OF wf controlled i-bound prefix-bound])
have sum-bound:
  wp-event ?m ?prefix adversary-initial-state +
  (wp-event ?m ?open-path adversary-initial-state +
   wp-event ?m ?witness-path adversary-initial-state) ≤ P + Q + R
proof -
  have path-sum-bound:
    wp-event ?m ?open-path adversary-initial-state +
    wp-event ?m ?witness-path adversary-initial-state ≤ Q + R
    by (rule add-mono[OF opening-path-bound witness-path-bound])
  have
    wp-event ?m ?prefix adversary-initial-state +
    (wp-event ?m ?open-path adversary-initial-state +
     wp-event ?m ?witness-path adversary-initial-state) ≤ P + (Q + R)
    by (rule add-mono[OF prefix-gap-bound path-sum-bound])
  then show ?thesis
    by (simp add: add.assoc)
qed
show ?thesis
  by (rule order-trans[OF event-le order-trans[OF union-le sum-bound]])
qed

lemma checked-staged-security-with-query-prefix-single-round-candidate-binding-gap-bound-from-witness-path-output
  assumes wf: staged-budget-wellformed budgets
  and controlled: staged-adversary-controlled budgets A
  and i-bound: i < rounds
  and path-bound:
    wp-event
      (checked-staged-security-experiment-with-query-prefix-data-state A i)
      (checked-staged-security-with-query-prefix-header-witness-path-output-hit
       (checked-staged-security-with-query-prefix-single-round-candidate-binding-gap-side
        trace-openings composition-openings trace-table composition-table i)
       trace-openings composition-openings i)
      adversary-initial-state ≤ Q
  shows
    wp-event
      (checked-staged-security-experiment-with-query-prefix-data-state A i)
      (checked-staged-security-with-query-prefix-single-round-candidate-binding-gap-at
       trace-openings composition-openings trace-table composition-table i)

```

$\text{adversary-initial-state} \leq$
 $\text{staged-phase-relation-error size}$
 $(\text{staged-query-search-queries budgets } i + 1) +$
 $(1::\text{prob}) / \text{nnreal} (\text{card query-sample-space}) + Q$

proof –

have *event-eq*:

$\text{checked-staged-security-with-query-prefix-single-round-candidate-binding-gap-at}$
 $\text{trace-openings composition-openings trace-table composition-table } i =$
 $\text{checked-staged-security-with-query-prefix-fixed-header-component}$
 $(\text{checked-staged-security-with-query-prefix-single-round-candidate-binding-gap-side}$
 $\text{trace-openings composition-openings trace-table composition-table } i)$
 $\text{trace-openings composition-openings } i$

by (rule *ext*)

(rule

$\text{checked-staged-security-with-query-prefix-single-round-candidate-binding-gap-iff-fixed-header-component})$

show *?thesis*

unfolding *event-eq*

by (rule

$\text{checked-staged-security-with-query-prefix-fixed-header-component-bound-from-witness-path-output-and-budgets}$
 $[OF \text{ wf controlled } i\text{-bound path-bound}])$

qed

lemma *checked-staged-security-with-query-prefix-single-round-candidate-binding-gap-bound-from-budgets*:

assumes *wf*: *staged-budget-wellformed budgets*

and *controlled*: *staged-adversary-controlled budgets A*

and *i-bound*: $i < \text{rounds}$

shows

wp-event

$(\text{checked-staged-security-experiment-with-query-prefix-data-state } A \ i)$
 $(\text{checked-staged-security-with-query-prefix-single-round-candidate-binding-gap-at}$
 $\text{trace-openings composition-openings trace-table composition-table } i)$
 $\text{adversary-initial-state} \leq$
 $\text{staged-phase-relation-error size}$
 $(\text{staged-query-search-queries budgets } i + 1) +$
 $(1::\text{prob}) / \text{nnreal} (\text{card query-sample-space})$

proof –

have *event-eq*:

$\text{checked-staged-security-with-query-prefix-single-round-candidate-binding-gap-at}$
 $\text{trace-openings composition-openings trace-table composition-table } i =$
 $\text{checked-staged-security-with-query-prefix-fixed-header-component}$
 $(\text{checked-staged-security-with-query-prefix-single-round-candidate-binding-gap-side}$
 $\text{trace-openings composition-openings trace-table composition-table } i)$
 $\text{trace-openings composition-openings } i$

by (rule *ext*)

(rule

$\text{checked-staged-security-with-query-prefix-single-round-candidate-binding-gap-iff-fixed-header-component})$

show *?thesis*

unfolding *event-eq*

by (rule

checked-staged-security-with-query-prefix-fixed-header-component-bound-from-budgets
 $[OF \text{ wf controlled } i\text{-bound}]$)

qed

lemma *checked-staged-security-with-query-prefix-header-authenticated-candidate-opening-hit-bound-from-target-and-f*

assumes *wf: staged-budget-wellformed budgets*

and *controlled: staged-adversary-controlled budgets A*

and *i-bound: i < rounds*

and *trace-candidate:*

partial-trace-table-candidate trace-table

$((\text{replicate rounds } []) [i := \text{trace-openings}])$

and *comp-candidate:*

partial-composition-table-candidate composition-table

$((\text{replicate rounds } []) [i := \text{composition-openings}])$

and *trace-low: trace-table-low-degree trace-table*

and *comp-low: composition-table-low-degree maxDegree composition-table*

and *not-all:*

$\bigwedge \text{prefix prefix-state raw raw-state data attacker-state result}$
final-state.

Some (((((prefix, prefix-state), raw), raw-state),
((data, attacker-state), result)), final-state) ∈

set-dist

(execute

(checked-staged-security-experiment-with-query-prefix-data-state
A i)

adversary-initial-state) ⇒

$\neg \text{all-queries-consistent trace-table composition-table}$

(sqp-alphas prefix)

and *target-bound:*

wp-event

(checked-staged-security-experiment-with-query-prefix-data-state A i)

(checked-staged-security-with-query-prefix-single-round-header-supported-query-target-hit-at
i)

adversary-initial-state ≤ T

shows

wp-event

(checked-staged-security-experiment-with-query-prefix-data-state A i)

(checked-staged-security-with-query-prefix-header-authenticated-candidate-opening-hit
trace-openings composition-openings i)

adversary-initial-state ≤

T +

(staged-phase-relation-error size

(staged-query-search-queries budgets i + 1) +

(1::prob) / nnreal (card query-sample-space))

proof –

let *?m =*

checked-staged-security-experiment-with-query-prefix-data-state A i

let *?target =*

checked-staged-security-with-query-prefix-single-round-header-supported-query-target-hit-at

```

i
let ?gap =
  checked-staged-security-with-query-prefix-single-round-candidate-binding-gap-at
    trace-openings composition-openings trace-table composition-table i
have event-le:
  wp-event ?m
    (checked-staged-security-with-query-prefix-header-authenticated-candidate-opening-hit
      trace-openings composition-openings i)
    adversary-initial-state ≤
    wp-event ?m ( $\lambda out. ?target\ out \vee ?gap\ out$ )
    adversary-initial-state
proof (rule wp-event-mono-on-support)
fix out
assume support:
  out ∈ set-dist (execute ?m adversary-initial-state)
and hit:
  checked-staged-security-with-query-prefix-header-authenticated-candidate-opening-hit
    trace-openings composition-openings i out
show  $?target\ out \vee ?gap\ out$ 
proof (cases out)
case None
then show ?thesis
  using hit
  unfolding
    checked-staged-security-with-query-prefix-header-authenticated-candidate-opening-hit-def
  by simp
next
case (Some packed)
then obtain prefix prefix-state raw raw-state data attacker-state
  result final-state where out-eq:
    out =
      Some (((((prefix, prefix-state), raw), raw-state),
        ((data, attacker-state), result)), final-state)
  by (cases packed) (auto split: prod.splits)
have not-all':
   $\neg$  all-queries-consistent trace-table composition-table
    (sqp-alphas prefix)
  by (rule not-all)
    (use support out-eq in simp)
show ?thesis
  unfolding out-eq
  by (rule
    checked-staged-security-with-query-prefix-header-authenticated-candidate-opening-hit-imp-single-round-header-
      [OF - trace-candidate comp-candidate trace-low comp-low not-all'])
    (use hit out-eq in simp)
qed
qed
have union-le:
  wp-event ?m ( $\lambda out. ?target\ out \vee ?gap\ out$ )

```

```

    adversary-initial-state ≤
    wp-event ?m ?target adversary-initial-state +
    wp-event ?m ?gap adversary-initial-state
  by (rule wp-event-union-bound)
have gap-bound:
  wp-event ?m ?gap adversary-initial-state ≤
    staged-phase-relation-error size
    (staged-query-search-queries budgets i + 1) +
    (1::prob) / nnreal (card query-sample-space)
  by (rule
    checked-staged-security-with-query-prefix-single-round-candidate-binding-gap-bound-from-budgets
    [OF wf controlled i-bound])
have sum-bound:
  wp-event ?m ?target adversary-initial-state +
  wp-event ?m ?gap adversary-initial-state ≤
  T +
  (staged-phase-relation-error size
    (staged-query-search-queries budgets i + 1) +
    (1::prob) / nnreal (card query-sample-space))
  by (rule add-mono[OF target-bound gap-bound])
show ?thesis
  by (rule order-trans[OF event-le order-trans[OF union-le sum-bound]])
qed

```

lemma *checked-staged-security-with-query-prefix-verifier-unsupported-witness-gap-imp-header-witness-path-output:*

```

assumes witness:
  (trace-openings, composition-openings) ∈
  query-header-supported-partial-opening-witnesses
  (verifier-state-from-adversary attacker-state
    (staged-proof-transcript data))
  (staged-trace-root data)
  (staged-trace-fri-roots data)
  (staged-trace-final data)
  (staged-alphas data)
  (staged-degree data)
  (staged-composition-fri-roots data)
  (staged-composition-final data)
and trace-candidate:
  partial-trace-table-candidate trace-table
  ((replicate rounds []) [i := trace-openings])
and comp-candidate:
  partial-composition-table-candidate composition-table
  ((replicate rounds []) [i := composition-openings])
and notin-prefix:
  (trace-table, composition-table) ∉
  query-header-supported-single-round-partial-table-candidates
  prefix-state
  (sqp-trace-root prefix)
  (sqp-trace-fri-roots prefix)

```

```

    (sqp-trace-final prefix)
    (sqp-alphas prefix)
    (sqp-degree prefix)
    (sqp-composition-fri-roots prefix)
    (sqp-composition-final prefix)
  i
and notin-verifier:
  (trace-table, composition-table) ∉
  query-header-supported-single-round-partial-table-candidates
  (verifier-state-from-adversary attacker-state
    (staged-proof-transcript data))
  (staged-trace-root data)
  (staged-trace-fri-roots data)
  (staged-trace-final data)
  (staged-alphas data)
  (staged-degree data)
  (staged-composition-fri-roots data)
  (staged-composition-final data)
  i
shows
  checked-staged-security-with-query-prefix-header-witness-path-output-hit
  (checked-staged-security-with-query-prefix-single-round-candidate-binding-gap-side
    trace-openings composition-openings trace-table composition-table i)
  trace-openings composition-openings i
  (Some (((((prefix, prefix-state), raw), raw-state),
    ((data, attacker-state), result)), final-state))
proof –
  let ?s =
    verifier-state-from-adversary attacker-state
    (staged-proof-transcript data)
  from query-header-supported-partial-opening-witnessesE[OF witness]
  obtain witness-result witness-state rest where comp-nonempty:
    staged-composition-fri-roots data ≠ []
  and outcome:
    Some (witness-result, witness-state) ∈
    set-dist (execute verify-monad ?s)
  and trace-auth:
    partial-authenticated-table (staged-trace-root data)
    (scale * clength) trace-openings witness-state
  and comp-auth:
    partial-authenticated-table (hd (staged-composition-fri-roots data))
    (scale * clength) composition-openings witness-state
  and header:
    verifier-header-transcript ?s
    (staged-trace-root data)
    (staged-trace-fri-roots data)
    (staged-trace-final data)
    (staged-alphas data)
    (staged-degree data)

```

```

    (staged-composition-fri-roots data)
    (staged-composition-final data)
    rest
  by blast
have ext: ?s ≤ witness-state
  by (rule verify-monad-hash-extends[OF outcome])
have trace-pull:
  partial-authenticated-table (staged-trace-root data)
    (scale * clength) trace-openings ?s ∨
  (∃ opn ∈ set trace-openings.
    hash-map-new-output-hit
      (merkle-path-target-roots witness-state
        (staged-trace-root data) (scale * clength)
        (opening-index opn) (opening-value opn) (opening-path opn))
      ?s witness-state)
  by (rule partial-authenticated-table-pullback-or-new-output-hit
    [OF ext trace-auth])
have comp-pull:
  partial-authenticated-table (hd (staged-composition-fri-roots data))
    (scale * clength) composition-openings ?s ∨
  (∃ opn ∈ set composition-openings.
    hash-map-new-output-hit
      (merkle-path-target-roots witness-state
        (hd (staged-composition-fri-roots data)) (scale * clength)
        (opening-index opn) (opening-value opn) (opening-path opn))
      ?s witness-state)
  by (rule partial-authenticated-table-pullback-or-new-output-hit
    [OF ext comp-auth])
have path:
  (∃ opn ∈ set trace-openings.
    hash-map-new-output-hit
      (merkle-path-target-roots witness-state
        (staged-trace-root data) (scale * clength)
        (opening-index opn) (opening-value opn) (opening-path opn))
      ?s witness-state) ∨
  (∃ opn ∈ set composition-openings.
    hash-map-new-output-hit
      (merkle-path-target-roots witness-state
        (hd (staged-composition-fri-roots data)) (scale * clength)
        (opening-index opn) (opening-value opn) (opening-path opn))
      ?s witness-state)
proof (rule ccontr)
  assume not-path: ¬ ?thesis
  have trace-on-s:
    partial-authenticated-table (staged-trace-root data)
      (scale * clength) trace-openings ?s
  using trace-pull not-path by blast
  have comp-on-s:
    partial-authenticated-table (hd (staged-composition-fri-roots data))

```

```

    (scale * clength) composition-openings ?s
using comp-pull not-path by blast
show False
by (rule
  query-header-supported-single-round-partial-table-candidate-notin-contradicts
  [OF notin-verifier comp-nonempty trace-on-s comp-on-s
    trace-candidate comp-candidate header])
qed
have side:
  checked-staged-security-with-query-prefix-single-round-candidate-binding-gap-side
  trace-openings composition-openings trace-table composition-table i
  (Some (((((prefix, prefix-state), raw), raw-state),
    ((data, attacker-state), result)), final-state))
unfolding
  checked-staged-security-with-query-prefix-single-round-candidate-binding-gap-side-def
using trace-candidate comp-candidate notin-prefix by simp
show ?thesis
unfolding
  checked-staged-security-with-query-prefix-header-witness-path-output-hit-def
using side comp-nonempty outcome header path
apply (simp add: Let-def)
apply (intro exI[of - witness-result] exI[of - witness-state]
  exI[of - rest])
apply auto
done
qed

lemma checked-staged-security-with-query-prefix-witness-gap-side-imp-verifier-supported-or-header-witness-path-outp
assumes witness:
  (trace-openings, composition-openings) ∈
  query-header-supported-partial-opening-witnesses
  (verifier-state-from-adversary attacker-state
    (staged-proof-transcript data))
  (staged-trace-root data)
  (staged-trace-fri-roots data)
  (staged-trace-final data)
  (staged-alphas data)
  (staged-degree data)
  (staged-composition-fri-roots data)
  (staged-composition-final data)
and side:
  checked-staged-security-with-query-prefix-single-round-candidate-binding-gap-side
  trace-openings composition-openings trace-table composition-table i
  (Some (((((prefix, prefix-state), raw), raw-state),
    ((data, attacker-state), result)), final-state))
shows
  (trace-table, composition-table) ∈
  query-header-supported-single-round-partial-table-candidates
  (verifier-state-from-adversary attacker-state

```



```

    (staged-proof-transcript data))
  (staged-trace-root data)
  (staged-trace-fri-roots data)
  (staged-trace-final data)
  (staged-alphas data)
  (staged-degree data)
  (staged-composition-fri-roots data)
  (staged-composition-final data)
  i ∨
checked-staged-security-with-query-prefix-header-witness-path-output-hit
  (checked-staged-security-with-query-prefix-single-round-candidate-binding-gap-side
    trace-openings composition-openings trace-table composition-table i)
  trace-openings composition-openings i
  (Some (((((prefix, prefix-state), raw), raw-state),
    ((data, attacker-state), result)), final-state))
proof (cases
  (trace-table, composition-table) ∈
    query-header-supported-single-round-partial-table-candidates
    (verifier-state-from-adversary attacker-state
      (staged-proof-transcript data))
    (staged-trace-root data)
    (staged-trace-fri-roots data)
    (staged-trace-final data)
    (staged-alphas data)
    (staged-degree data)
    (staged-composition-fri-roots data)
    (staged-composition-final data)
    i)
case True
then show ?thesis by simp
next
case notin-verifier: False
from side have trace-candidate:
  partial-trace-table-candidate trace-table
  ((replicate rounds []) [i := trace-openings])
and comp-candidate:
  partial-composition-table-candidate composition-table
  ((replicate rounds []) [i := composition-openings])
and notin-prefix:
  (trace-table, composition-table) ∉
    query-header-supported-single-round-partial-table-candidates
    prefix-state
    (sqp-trace-root prefix)
    (sqp-trace-fri-roots prefix)
    (sqp-trace-final prefix)
    (sqp-alphas prefix)
    (sqp-degree prefix)
    (sqp-composition-fri-roots prefix)
    (sqp-composition-final prefix)

```

i
unfolding
checked-staged-security-with-query-prefix-single-round-candidate-binding-gap-side-def
by *simp-all*
have
checked-staged-security-with-query-prefix-header-witness-path-output-hit
(checked-staged-security-with-query-prefix-single-round-candidate-binding-gap-side
trace-openings composition-openings trace-table composition-table i)
trace-openings composition-openings i
(Some (((((prefix, prefix-state), raw), raw-state),
((data, attacker-state), result)), final-state))
by *(rule*
checked-staged-security-with-query-prefix-verifier-unsupported-witness-gap-imp-header-witness-path-output
[OF witness trace-candidate comp-candidate notin-prefix notin-verifier])
then show *?thesis* **by** *simp*
qed

definition *checked-staged-security-with-query-prefix-single-round-candidate-binding-gap-witness-side*
where

checked-staged-security-with-query-prefix-single-round-candidate-binding-gap-witness-side
trace-openings composition-openings trace-table composition-table i out
 $\longleftrightarrow$
(case out of
None $\Rightarrow$ False
| Some (((((prefix, prefix-state), raw), raw-state),
((data, attacker-state), result)), final-state) $\Rightarrow$
checked-staged-security-with-query-prefix-single-round-candidate-binding-gap-side
trace-openings composition-openings trace-table composition-table i
out $\wedge$
(trace-openings, composition-openings) $\in$
query-header-supported-partial-opening-witnesses
(verifier-state-from-adversary attacker-state
(staged-proof-transcript data))
(staged-trace-root data)
(staged-trace-fri-roots data)
(staged-trace-final data)
(staged-alphas data)
(staged-degree data)
(staged-composition-fri-roots data)
(staged-composition-final data))

definition *checked-staged-security-with-query-prefix-single-round-candidate-pair-witness-gap-at*
where

checked-staged-security-with-query-prefix-single-round-candidate-pair-witness-gap-at
trace-openings composition-openings trace-table composition-table i out
 $\longleftrightarrow$
checked-staged-security-with-query-prefix-dynamic-index-hit
(staged-query-prefix-candidate-pair-query-target-from-prefix
trace-table composition-table) out $\wedge$

checked-staged-security-with-query-prefix-single-round-candidate-binding-gap-witness-side
trace-openings composition-openings trace-table composition-table i out

lemma *checked-staged-security-with-query-prefix-single-round-candidate-pair-witness-gap-at-imp-dynamic-hit:*

assumes *gap:*

checked-staged-security-with-query-prefix-single-round-candidate-pair-witness-gap-at
trace-openings composition-openings trace-table composition-table i out

shows

checked-staged-security-with-query-prefix-dynamic-index-hit
(staged-query-prefix-candidate-pair-query-target-from-prefix
trace-table composition-table) out

using *gap*

unfolding

checked-staged-security-with-query-prefix-single-round-candidate-pair-witness-gap-at-def

by *simp*

lemma *checked-staged-security-with-query-prefix-single-round-candidate-pair-witness-gap-at-imp-witness-side:*

assumes *gap:*

checked-staged-security-with-query-prefix-single-round-candidate-pair-witness-gap-at
trace-openings composition-openings trace-table composition-table i out

shows

checked-staged-security-with-query-prefix-single-round-candidate-binding-gap-witness-side
trace-openings composition-openings trace-table composition-table i out

using *gap*

unfolding

checked-staged-security-with-query-prefix-single-round-candidate-pair-witness-gap-at-def

by *simp*

definition *checked-staged-security-with-query-prefix-single-round-candidate-binding-gap-witness-at*

where

checked-staged-security-with-query-prefix-single-round-candidate-binding-gap-witness-at
trace-openings composition-openings trace-table composition-table i out

$\longleftrightarrow$

checked-staged-security-with-query-prefix-single-round-candidate-binding-gap-at
trace-openings composition-openings trace-table composition-table i out $\wedge$

checked-staged-security-with-query-prefix-single-round-candidate-binding-gap-witness-side
trace-openings composition-openings trace-table composition-table i out

lemma *checked-staged-security-with-query-prefix-single-round-candidate-binding-gap-witness-at-imp-gap:*

assumes *hit:*

checked-staged-security-with-query-prefix-single-round-candidate-binding-gap-witness-at
trace-openings composition-openings trace-table composition-table i out

shows

checked-staged-security-with-query-prefix-single-round-candidate-binding-gap-at
trace-openings composition-openings trace-table composition-table i out

using *hit*

unfolding

checked-staged-security-with-query-prefix-single-round-candidate-binding-gap-witness-at-def

by *simp*

lemma *checked-staged-security-with-query-prefix-single-round-candidate-binding-gap-witness-at-imp-witness-side:*
assumes *hit:*

checked-staged-security-with-query-prefix-single-round-candidate-binding-gap-witness-at
trace-openings composition-openings trace-table composition-table i out

shows

checked-staged-security-with-query-prefix-single-round-candidate-binding-gap-witness-side
trace-openings composition-openings trace-table composition-table i out

using *hit*

unfolding

checked-staged-security-with-query-prefix-single-round-candidate-binding-gap-witness-at-def
by *simp*

lemma *checked-staged-security-with-query-prefix-single-round-candidate-binding-gap-witness-at-bound-from-budgets:*

assumes *wf: staged-budget-wellformed budgets*

and *controlled: staged-adversary-controlled budgets A*

and *i-bound: i < rounds*

shows

wp-event

(checked-staged-security-experiment-with-query-prefix-data-state A i)

(checked-staged-security-with-query-prefix-single-round-candidate-binding-gap-witness-at
trace-openings composition-openings trace-table composition-table i)

adversary-initial-state ≤

staged-phase-relation-error size

(staged-query-search-queries budgets i + 1) +

(1::prob) / nnreal (card query-sample-space)

proof —

have *event-le:*

wp-event

(checked-staged-security-experiment-with-query-prefix-data-state A i)

(checked-staged-security-with-query-prefix-single-round-candidate-binding-gap-witness-at
trace-openings composition-openings trace-table composition-table i)

adversary-initial-state ≤

wp-event

(checked-staged-security-experiment-with-query-prefix-data-state A i)

(checked-staged-security-with-query-prefix-single-round-candidate-binding-gap-at
trace-openings composition-openings trace-table composition-table i)

adversary-initial-state

by *(rule wp-event-mono)*

(rule

checked-staged-security-with-query-prefix-single-round-candidate-binding-gap-witness-at-imp-gap)

have *gap-bound:*

wp-event

(checked-staged-security-experiment-with-query-prefix-data-state A i)

(checked-staged-security-with-query-prefix-single-round-candidate-binding-gap-at
trace-openings composition-openings trace-table composition-table i)

adversary-initial-state ≤

staged-phase-relation-error size

(staged-query-search-queries budgets i + 1) +

```

    (1::prob) / nnreal (card query-sample-space)
  by (rule
    checked-staged-security-with-query-prefix-single-round-candidate-binding-gap-bound-from-budgets
    [OF wf controlled i-bound])
  show ?thesis
  by (rule order-trans[OF event-le gap-bound])
qed

```

lemma *checked-staged-security-with-query-prefix-single-round-candidate-binding-gap-witness-at-bound-from-witness-side*
assumes *side-bound*:

```

  wp-event
  (checked-staged-security-experiment-with-query-prefix-data-state A i)
  (checked-staged-security-with-query-prefix-single-round-candidate-binding-gap-witness-side
    trace-openings composition-openings trace-table composition-table i)
  adversary-initial-state ≤ S

```

shows

```

  wp-event
  (checked-staged-security-experiment-with-query-prefix-data-state A i)
  (checked-staged-security-with-query-prefix-single-round-candidate-binding-gap-witness-at
    trace-openings composition-openings trace-table composition-table i)
  adversary-initial-state ≤ S

```

by (rule order-trans[OF - side-bound])

```

  (rule wp-event-mono,
  rule checked-staged-security-with-query-prefix-single-round-candidate-binding-gap-witness-at-imp-witness-side)

```

definition *checked-staged-security-with-query-prefix-single-round-verifier-supported-gap-witness-side*
where

```

checked-staged-security-with-query-prefix-single-round-verifier-supported-gap-witness-side
  trace-openings composition-openings trace-table composition-table i out
  ⇔
  (case out of
    None ⇒ False
  | Some (((prefix, prefix-state), raw), raw-state),
    ((data, attacker-state), result), final-state) ⇒
    checked-staged-security-with-query-prefix-single-round-candidate-binding-gap-witness-side
      trace-openings composition-openings trace-table composition-table i
      out ∧
    (trace-table, composition-table) ∈
      query-header-supported-single-round-partial-table-candidates
        (verifier-state-from-adversary attacker-state
          (staged-proof-transcript data))
        (staged-trace-root data)
        (staged-trace-fri-roots data)
        (staged-trace-final data)
        (staged-alphas data)
        (staged-degree data)
        (staged-composition-fri-roots data)
        (staged-composition-final data)
    i)

```

lemma *checked-staged-security-with-query-prefix-single-round-candidate-binding-gap-witness-side-imp-supported-or-p*

assumes *hit*:

checked-staged-security-with-query-prefix-single-round-candidate-binding-gap-witness-side
trace-openings composition-openings trace-table composition-table i out

shows

checked-staged-security-with-query-prefix-single-round-verifier-supported-gap-witness-side
trace-openings composition-openings trace-table composition-table i out $\vee$
checked-staged-security-with-query-prefix-header-witness-path-output-hit
 (*checked-staged-security-with-query-prefix-single-round-candidate-binding-gap-side*
trace-openings composition-openings trace-table composition-table i)
trace-openings composition-openings i out

proof (cases out)

case *None*

then show *?thesis*

using *hit*

unfolding

checked-staged-security-with-query-prefix-single-round-candidate-binding-gap-witness-side-def

by *simp*

next

case (Some packed)

then obtain *prefix prefix-state raw raw-state data attacker-state result*
final-state where out-eq:

out =
 Some (((((prefix, prefix-state), raw), raw-state),
 ((data, attacker-state), result)), final-state)

by (cases packed) (auto split: prod.splits)

from *hit have* *side*:

checked-staged-security-with-query-prefix-single-round-candidate-binding-gap-side
trace-openings composition-openings trace-table composition-table i
out

and *witness*:

(*trace-openings, composition-openings*) $\in$
query-header-supported-partial-opening-witnesses
 (*verifier-state-from-adversary attacker-state*
 (*staged-proof-transcript data*))
 (*staged-trace-root data*)
 (*staged-trace-fri-roots data*)
 (*staged-trace-final data*)
 (*staged-alphas data*)
 (*staged-degree data*)
 (*staged-composition-fri-roots data*)
 (*staged-composition-final data*)

unfolding *out-eq*

checked-staged-security-with-query-prefix-single-round-candidate-binding-gap-witness-side-def

by *simp-all*

have *split*:

(*trace-table, composition-table*) $\in$
query-header-supported-single-round-partial-table-candidates

```

    (verifier-state-from-adversary attacker-state
      (staged-proof-transcript data))
    (staged-trace-root data)
    (staged-trace-fri-roots data)
    (staged-trace-final data)
    (staged-alphas data)
    (staged-degree data)
    (staged-composition-fri-roots data)
    (staged-composition-final data)
    i ∨
checked-staged-security-with-query-prefix-header-witness-path-output-hit
  (checked-staged-security-with-query-prefix-single-round-candidate-binding-gap-side
    trace-openings composition-openings trace-table composition-table i)
  trace-openings composition-openings i out
unfolding out-eq
by (rule
  checked-staged-security-with-query-prefix-witness-gap-side-imp-verifier-supported-or-header-witness-path-output
    [OF witness side[unfolded out-eq]])
then show ?thesis
proof
  assume supported:
    (trace-table, composition-table) ∈
      query-header-supported-single-round-partial-table-candidates
        (verifier-state-from-adversary attacker-state
          (staged-proof-transcript data))
        (staged-trace-root data)
        (staged-trace-fri-roots data)
        (staged-trace-final data)
        (staged-alphas data)
        (staged-degree data)
        (staged-composition-fri-roots data)
        (staged-composition-final data)
        i
  have
    checked-staged-security-with-query-prefix-single-round-verifier-supported-gap-witness-side
      trace-openings composition-openings trace-table composition-table i out
  unfolding out-eq
    checked-staged-security-with-query-prefix-single-round-verifier-supported-gap-witness-side-def
  using hit[unfolded out-eq] supported by simp
then show ?thesis by simp
next
  assume
    checked-staged-security-with-query-prefix-header-witness-path-output-hit
      (checked-staged-security-with-query-prefix-single-round-candidate-binding-gap-side
        trace-openings composition-openings trace-table composition-table i)
      trace-openings composition-openings i out
  then show ?thesis by simp
qed
qed

```

lemma *checked-staged-security-with-query-prefix-single-round-candidate-binding-gap-witness-side-bound-from-support*

assumes *supported-bound*:

wp-event

(*checked-staged-security-experiment-with-query-prefix-data-state A i*)

(*checked-staged-security-with-query-prefix-single-round-verifier-supported-gap-witness-side*
trace-openings composition-openings trace-table composition-table i)

adversary-initial-state $\leq S$

and *path-bound*:

wp-event

(*checked-staged-security-experiment-with-query-prefix-data-state A i*)

(*checked-staged-security-with-query-prefix-header-witness-path-output-hit*

(*checked-staged-security-with-query-prefix-single-round-candidate-binding-gap-side*
trace-openings composition-openings trace-table composition-table i)

trace-openings composition-openings i)

adversary-initial-state $\leq P$

shows

wp-event

(*checked-staged-security-experiment-with-query-prefix-data-state A i*)

(*checked-staged-security-with-query-prefix-single-round-candidate-binding-gap-witness-side*
trace-openings composition-openings trace-table composition-table i)

adversary-initial-state $\leq S + P$

proof –

let *?m* =

checked-staged-security-experiment-with-query-prefix-data-state A i

let *?W* =

checked-staged-security-with-query-prefix-single-round-candidate-binding-gap-witness-side
trace-openings composition-openings trace-table composition-table i

let *?S* =

checked-staged-security-with-query-prefix-single-round-verifier-supported-gap-witness-side
trace-openings composition-openings trace-table composition-table i

let *?P* =

checked-staged-security-with-query-prefix-header-witness-path-output-hit

(*checked-staged-security-with-query-prefix-single-round-candidate-binding-gap-side*

trace-openings composition-openings trace-table composition-table i)

trace-openings composition-openings i

have *event-le*:

wp-event ?m ?W adversary-initial-state $\leq$

wp-event ?m ($\lambda out. ?S out \vee ?P out$) *adversary-initial-state*

by (rule *wp-event-mono*)

(rule

checked-staged-security-with-query-prefix-single-round-candidate-binding-gap-witness-side-imp-supported-or-path

have *union-le*:

wp-event ?m ($\lambda out. ?S out \vee ?P out$) *adversary-initial-state* $\leq$

wp-event ?m ?S adversary-initial-state +

wp-event ?m ?P adversary-initial-state

by (rule *wp-event-union-bound*)

have *sum-le*:

wp-event ?m ?S adversary-initial-state +


```

    wp-event ?m ?P adversary-initial-state ≤ S + P
  by (rule add-mono[OF supported-bound path-bound])
show ?thesis
  by (rule order-trans[OF event-le order-trans[OF union-le sum-le]])
qed

```

lemma *checked-staged-security-with-query-prefix-single-round-candidate-binding-gap-witness-side-bound-from-supported*
assumes *supported-bound*:

```

  wp-event
  (checked-staged-security-experiment-with-query-prefix-data-state A i)
  (checked-staged-security-with-query-prefix-single-round-verifier-supported-gap-witness-side
    trace-openings composition-openings trace-table composition-table i)
  adversary-initial-state ≤ S

```

and *transcript-bound*:

```

  wp-event
  (checked-staged-security-experiment-with-query-prefix-data-state A i)
  (checked-staged-security-with-query-prefix-header-witness-transcript-hit
    (checked-staged-security-with-query-prefix-single-round-candidate-binding-gap-side
      trace-openings composition-openings trace-table composition-table i))
  adversary-initial-state ≤ P

```

shows

```

  wp-event
  (checked-staged-security-experiment-with-query-prefix-data-state A i)
  (checked-staged-security-with-query-prefix-single-round-candidate-binding-gap-witness-side
    trace-openings composition-openings trace-table composition-table i)
  adversary-initial-state ≤ S + P

```

proof –

let *?m* =

```

  checked-staged-security-experiment-with-query-prefix-data-state A i

```

let *?side* =

```

  checked-staged-security-with-query-prefix-single-round-candidate-binding-gap-side
  trace-openings composition-openings trace-table composition-table i

```

have *path-bound*:

```

  wp-event ?m
  (checked-staged-security-with-query-prefix-header-witness-path-output-hit
    ?side trace-openings composition-openings i)
  adversary-initial-state ≤ P

```

proof –

have

```

  wp-event ?m
  (checked-staged-security-with-query-prefix-header-witness-path-output-hit
    ?side trace-openings composition-openings i)
  adversary-initial-state ≤

```

```

  wp-event ?m

```

```

  (checked-staged-security-with-query-prefix-header-witness-transcript-hit
    ?side)

```

```

  adversary-initial-state

```

by (rule *wp-event-mono*)

```

  (rule

```

```

      checked-staged-security-with-query-prefix-header-witness-path-output-imp-transcript-hit)
    then show ?thesis
      by (rule order-trans[OF - transcript-bound])
  qed
show ?thesis
  by (rule
    checked-staged-security-with-query-prefix-single-round-candidate-binding-gap-witness-side-bound-from-supported-
      [OF supported-bound path-bound])
qed

lemma checked-staged-security-with-query-prefix-single-round-candidate-binding-gap-witness-side-bound-from-support
assumes supported-bound:
  wp-event
    (checked-staged-security-experiment-with-query-prefix-data-state A i)
    (checked-staged-security-with-query-prefix-single-round-verifier-supported-gap-witness-side
      trace-openings composition-openings trace-table composition-table i)
    adversary-initial-state  $\leq S$ 
and pre-bound:
  wp-event
    (checked-staged-security-experiment-with-query-prefix-data-state A i)
    (checked-staged-security-with-query-prefix-header-witness-transcript-pre-hit
      (checked-staged-security-with-query-prefix-single-round-candidate-binding-gap-side
        trace-openings composition-openings trace-table composition-table i))
    adversary-initial-state  $\leq R$ 
and new-bound:
  wp-event
    (checked-staged-security-experiment-with-query-prefix-data-state A i)
    (checked-staged-security-with-query-prefix-header-witness-transcript-new-hit
      (checked-staged-security-with-query-prefix-single-round-candidate-binding-gap-side
        trace-openings composition-openings trace-table composition-table i))
    adversary-initial-state  $\leq Q$ 
shows
  wp-event
    (checked-staged-security-experiment-with-query-prefix-data-state A i)
    (checked-staged-security-with-query-prefix-single-round-candidate-binding-gap-witness-side
      trace-openings composition-openings trace-table composition-table i)
    adversary-initial-state  $\leq S + (R + Q)$ 
proof -
  let ?m =
    checked-staged-security-experiment-with-query-prefix-data-state A i
  let ?side =
    checked-staged-security-with-query-prefix-single-round-candidate-binding-gap-side
      trace-openings composition-openings trace-table composition-table i
  have transcript-bound:
    wp-event ?m
      (checked-staged-security-with-query-prefix-header-witness-transcript-hit
        ?side)
    adversary-initial-state  $\leq R + Q$ 
proof -

```

```

have event-le:
  wp-event ?m
  (checked-staged-security-with-query-prefix-header-witness-transcript-hit
   ?side)
  adversary-initial-state ≤
  wp-event ?m
  (λout.
   checked-staged-security-with-query-prefix-header-witness-transcript-pre-hit
    ?side out ∨
   checked-staged-security-with-query-prefix-header-witness-transcript-new-hit
    ?side out)
  adversary-initial-state
by (rule wp-event-mono)
  (rule
   checked-staged-security-with-query-prefix-header-witness-transcript-hit-imp-pre-or-new)
have union-le:
  wp-event ?m
  (λout.
   checked-staged-security-with-query-prefix-header-witness-transcript-pre-hit
    ?side out ∨
   checked-staged-security-with-query-prefix-header-witness-transcript-new-hit
    ?side out)
  adversary-initial-state ≤
  wp-event ?m
  (checked-staged-security-with-query-prefix-header-witness-transcript-pre-hit
   ?side)
  adversary-initial-state +
  wp-event ?m
  (checked-staged-security-with-query-prefix-header-witness-transcript-new-hit
   ?side)
  adversary-initial-state
by (rule wp-event-union-bound)
have sum-le:
  wp-event ?m
  (checked-staged-security-with-query-prefix-header-witness-transcript-pre-hit
   ?side)
  adversary-initial-state +
  wp-event ?m
  (checked-staged-security-with-query-prefix-header-witness-transcript-new-hit
   ?side)
  adversary-initial-state ≤ R + Q
by (rule add-mono[OF pre-bound new-bound])
show ?thesis
by (rule order-trans[OF event-le order-trans[OF union-le sum-le]])
qed
have bound:
  wp-event ?m
  (checked-staged-security-with-query-prefix-single-round-candidate-binding-gap-witness-side
   trace-openings composition-openings trace-table composition-table i)

```

$adversary-initial-state \leq S + (R + Q)$
by (rule
checked-staged-security-with-query-prefix-single-round-candidate-binding-gap-witness-side-bound-from-supported-
[OF supported-bound transcript-bound])
then show *?thesis* .
qed

lemma *checked-staged-security-with-query-prefix-single-round-candidate-binding-gap-witness-at-bound-from-supported-*
assumes *supported-bound*:
wp-event
(checked-staged-security-experiment-with-query-prefix-data-state A i)
(checked-staged-security-with-query-prefix-single-round-verifier-supported-gap-witness-side
trace-openings composition-openings trace-table composition-table i)
adversary-initial-state $\leq S$
and *pre-bound*:
wp-event
(checked-staged-security-experiment-with-query-prefix-data-state A i)
(checked-staged-security-with-query-prefix-header-witness-transcript-pre-hit
(checked-staged-security-with-query-prefix-single-round-candidate-binding-gap-side
trace-openings composition-openings trace-table composition-table i))
adversary-initial-state $\leq R$
and *new-bound*:
wp-event
(checked-staged-security-experiment-with-query-prefix-data-state A i)
(checked-staged-security-with-query-prefix-header-witness-transcript-new-hit
(checked-staged-security-with-query-prefix-single-round-candidate-binding-gap-side
trace-openings composition-openings trace-table composition-table i))
adversary-initial-state $\leq Q$
shows
wp-event
(checked-staged-security-experiment-with-query-prefix-data-state A i)
(checked-staged-security-with-query-prefix-single-round-candidate-binding-gap-witness-at
trace-openings composition-openings trace-table composition-table i)
adversary-initial-state $\leq S + (R + Q)$
proof –
have *side-bound*:
wp-event
(checked-staged-security-experiment-with-query-prefix-data-state A i)
(checked-staged-security-with-query-prefix-single-round-candidate-binding-gap-witness-side
trace-openings composition-openings trace-table composition-table i)
adversary-initial-state $\leq S + (R + Q)$
by (rule
checked-staged-security-with-query-prefix-single-round-candidate-binding-gap-witness-side-bound-from-supported-
[OF supported-bound pre-bound new-bound])
show *?thesis*
by (rule
checked-staged-security-with-query-prefix-single-round-candidate-binding-gap-witness-at-bound-from-witness-side
[OF side-bound])
qed

definition *checked-staged-security-with-actual-alpha-prefix-partial-merkle-inconsistency-exists*
where

checked-staged-security-with-actual-alpha-prefix-partial-merkle-inconsistency-exists
 $out \longleftrightarrow$
 (case out of
 None $\Rightarrow$ False
 | Some (((((alpha-prefix, alpha-prefix-state), data), attacker-state),
 result), final-state) $\Rightarrow$
 ($\exists$ result' final-state'.
 partial-merkle-inconsistency-bad
 (verifier-state-from-adversary attacker-state
 (staged-proof-transcript data))
 (Some (result', final-state')))))

lemma *checked-staged-security-with-actual-alpha-prefix-header-authenticated-candidate-opening-query-hit-from-prefix-at*
assumes *hit*:

checked-staged-security-with-actual-alpha-prefix-header-authenticated-candidate-opening-query-hit-from-prefix-at
 $i \ A$
 (Some (((((alpha-prefix, alpha-prefix-state), data), attacker-state),
 result), final-state))

and *i-bound*: $i < \text{rounds}$

and *trace-low*:

$\bigwedge$ *trace-openings composition-openings trace-table*
composition-table.
 (trace-openings, composition-openings) $\in$
query-header-supported-partial-opening-witnesses
 (verifier-state-from-adversary attacker-state
 (staged-proof-transcript data))
 (staged-trace-root data)
 (staged-trace-fri-roots data)
 (staged-trace-final data)
 (staged-alphas data)
 (staged-degree data)
 (staged-composition-fri-roots data)
 (staged-composition-final data) $\implies$
partial-trace-table-candidate trace-table
 ((replicate rounds []) [$i := \text{trace-openings}$]) $\implies$
partial-composition-table-candidate composition-table
 ((replicate rounds []) [$i := \text{composition-openings}$]) $\implies$
trace-table-low-degree trace-table

and *comp-low*:

$\bigwedge$ *trace-openings composition-openings trace-table*
composition-table.
 (trace-openings, composition-openings) $\in$
query-header-supported-partial-opening-witnesses
 (verifier-state-from-adversary attacker-state
 (staged-proof-transcript data))
 (staged-trace-root data)

(staged-trace-fri-roots data)
 (staged-trace-final data)
 (staged-alphas data)
 (staged-degree data)
 (staged-composition-fri-roots data)
 (staged-composition-final data) $\implies$
 partial-trace-table-candidate trace-table
 ((replicate rounds []) [i := trace-openings]) $\implies$
 partial-composition-table-candidate composition-table
 ((replicate rounds []) [i := composition-openings]) $\implies$
 composition-table-low-degree maxDegree composition-table

and not-all:

$\wedge$ trace-openings composition-openings trace-table
 composition-table prefix prefix-state.
 (trace-openings, composition-openings) $\in$
 query-header-supported-partial-opening-witnesses
 (verifier-state-from-adversary attacker-state
 (staged-proof-transcript data))
 (staged-trace-root data)
 (staged-trace-fri-roots data)
 (staged-trace-final data)
 (staged-alphas data)
 (staged-degree data)
 (staged-composition-fri-roots data)
 (staged-composition-final data) $\implies$
 partial-trace-table-candidate trace-table
 ((replicate rounds []) [i := trace-openings]) $\implies$
 partial-composition-table-candidate composition-table
 ((replicate rounds []) [i := composition-openings]) $\implies$
 Some ((prefix, prefix-state), prefix-state) $\in$
 set-dist
 (execute (checked-staged-query-prefix-with-state A i)
 adversary-initial-state) $\implies$
 $\neg$ all-queries-consistent trace-table composition-table
 (sqp-alphas prefix)

shows

checked-staged-security-with-actual-alpha-prefix-partial-merkle-inconsistency-exists
 (Some (((((alpha-prefix, alpha-prefix-state), data), attacker-state),
 result), final-state)) $\vee$
 checked-staged-security-with-actual-alpha-prefix-single-round-header-supported-query-target-hit-at
 i A
 (Some (((((alpha-prefix, alpha-prefix-state), data), attacker-state),
 result), final-state)) $\vee$
 checked-staged-security-with-actual-alpha-prefix-query-prefix-single-round-candidate-binding-gap-witness-at
 i A
 (Some (((((alpha-prefix, alpha-prefix-state), data), attacker-state),
 result), final-state))

using

checked-staged-security-with-actual-alpha-prefix-header-authenticated-candidate-opening-query-hit-from-prefix-at-im

[OF hit i-bound trace-low comp-low not-all]

unfolding

checked-staged-security-with-actual-alpha-prefix-partial-merkle-inconsistency-exists-def

by *simp*

lemma *checked-staged-security-with-actual-alpha-prefix-header-authenticated-candidate-opening-query-hit-from-prefix-at*

assumes *split*:

$\bigwedge out.$

$out \in$

set-dist

(*execute*

(*checked-staged-security-experiment-with-actual-alpha-prefix-data-state*

A)

adversary-initial-state) $\implies$

checked-staged-security-with-actual-alpha-prefix-header-authenticated-candidate-opening-query-hit-from-prefix-at

i A out $\implies$

checked-staged-security-with-actual-alpha-prefix-partial-merkle-inconsistency-exists

out $\vee$

checked-staged-security-with-actual-alpha-prefix-single-round-header-supported-query-target-hit-at

i A out $\vee$

checked-staged-security-with-actual-alpha-prefix-query-prefix-single-round-candidate-binding-gap-witness-at

i A out

and *partial-merkle-bound*:

wp-event

(*checked-staged-security-experiment-with-actual-alpha-prefix-data-state*

A)

checked-staged-security-with-actual-alpha-prefix-partial-merkle-inconsistency-exists

adversary-initial-state $\leq P$

and *target-bound*:

wp-event

(*checked-staged-security-experiment-with-actual-alpha-prefix-data-state*

A)

(*checked-staged-security-with-actual-alpha-prefix-single-round-header-supported-query-target-hit-at*

i A)

adversary-initial-state $\leq Q$

and *gap-bound*:

wp-event

(*checked-staged-security-experiment-with-actual-alpha-prefix-data-state*

A)

(*checked-staged-security-with-actual-alpha-prefix-query-prefix-single-round-candidate-binding-gap-witness-at*

i A)

adversary-initial-state $\leq R$

shows

wp-event

(*checked-staged-security-experiment-with-actual-alpha-prefix-data-state* *A*)

(*checked-staged-security-with-actual-alpha-prefix-header-authenticated-candidate-opening-query-hit-from-prefix-at*

i A)

adversary-initial-state $\leq P + Q + R$

proof –

```

let ?m =
  checked-staged-security-experiment-with-actual-alpha-prefix-data-state A
let ?pm =
  checked-staged-security-with-actual-alpha-prefix-partial-merkle-inconsistency-exists
let ?target =
  checked-staged-security-with-actual-alpha-prefix-single-round-header-supported-query-target-hit-at
    i A
let ?gap =
  checked-staged-security-with-actual-alpha-prefix-query-prefix-single-round-candidate-binding-gap-witness-at
    i A
have event-le:
  wp-event ?m
  (checked-staged-security-with-actual-alpha-prefix-header-authenticated-candidate-opening-query-hit-from-prefix-at
    i A)
  adversary-initial-state  $\leq$ 
  wp-event ?m ( $\lambda out. ?pm out \vee ?target out \vee ?gap out$ )
  adversary-initial-state
by (rule wp-event-mono-on-support)
  (rule split)
have union-le:
  wp-event ?m ( $\lambda out. ?pm out \vee ?target out \vee ?gap out$ )
  adversary-initial-state  $\leq$ 
  wp-event ?m ?pm adversary-initial-state +
  (wp-event ?m ?target adversary-initial-state +
  wp-event ?m ?gap adversary-initial-state)
proof -
have
  wp-event ?m ( $\lambda out. ?pm out \vee ?target out \vee ?gap out$ )
  adversary-initial-state  $\leq$ 
  wp-event ?m ?pm adversary-initial-state +
  wp-event ?m ( $\lambda out. ?target out \vee ?gap out$ )
  adversary-initial-state
by (rule wp-event-union-bound)
also have
  ...  $\leq$ 
  wp-event ?m ?pm adversary-initial-state +
  (wp-event ?m ?target adversary-initial-state +
  wp-event ?m ?gap adversary-initial-state)
by (intro add-mono order-refl wp-event-union-bound)
finally show ?thesis .
qed
have sum-bound:
  wp-event ?m ?pm adversary-initial-state +
  (wp-event ?m ?target adversary-initial-state +
  wp-event ?m ?gap adversary-initial-state)  $\leq P + Q + R$ 
proof -
have
  wp-event ?m ?target adversary-initial-state +
  wp-event ?m ?gap adversary-initial-state  $\leq Q + R$ 

```



```

    by (rule add-mono[OF target-bound gap-bound])
  then have
    wp-event ?m ?pm adversary-initial-state +
      (wp-event ?m ?target adversary-initial-state +
        wp-event ?m ?gap adversary-initial-state)  $\leq P + (Q + R)$ 
    by (rule add-mono[OF partial-merkle-bound])
  then show ?thesis
    by (simp add: add.assoc)
qed
show ?thesis
  by (rule order-trans[OF event-le order-trans[OF union-le sum-bound]])
qed

end

end

```

```

theory Staged-Security-Experiment-Composition-Query-Actual-Run
imports Staged-Security-Experiment-Composition-Query-Single-Round
begin

```

Same-run authenticated-opening split for the composition query branch.

This layer keeps the verifier continuation from the staged experiment fixed, so partial-Merkle inconsistency is expressed as the actual verifier event for the sampled run.

```

context soundness
begin

```

```

definition checked-staged-security-with-actual-alpha-prefix-partial-merkle-inconsistency-bad
where
  checked-staged-security-with-actual-alpha-prefix-partial-merkle-inconsistency-bad
    out  $\longleftrightarrow$ 
    checked-staged-security-with-actual-alpha-prefix-verifier-event
      partial-merkle-inconsistency-bad out

```

```

lemma checked-staged-security-with-actual-alpha-prefix-authenticated-candidate-opening-query-hit-from-prefix-imp-acc

```

```

assumes support:

```

```

  Some (((((alpha-prefix, alpha-prefix-state), data), attacker-state),
    result), final-state)  $\in$ 
  set-dist
    (execute
      (checked-staged-security-experiment-with-actual-alpha-prefix-data-state
        A)
      adversary-initial-state)

```

```

and hit:

```

```

  checked-staged-security-with-actual-alpha-prefix-authenticated-candidate-opening-query-hit-from-prefix
    A
  (Some (((((alpha-prefix, alpha-prefix-state), data), attacker-state),
    result), final-state))

```

and *trace-low*:

$\bigwedge i$ *trace-openings composition-openings trace-table*
composition-table.
 $i < \text{rounds} \implies$
partial-trace-table-candidate trace-table
 $((\text{replicate rounds } []) [i := \text{trace-openings}]) \implies$
partial-composition-table-candidate composition-table
 $((\text{replicate rounds } []) [i := \text{composition-openings}]) \implies$
trace-table-low-degree trace-table

and *comp-low*:

$\bigwedge i$ *trace-openings composition-openings trace-table*
composition-table.
 $i < \text{rounds} \implies$
partial-trace-table-candidate trace-table
 $((\text{replicate rounds } []) [i := \text{trace-openings}]) \implies$
partial-composition-table-candidate composition-table
 $((\text{replicate rounds } []) [i := \text{composition-openings}]) \implies$
composition-table-low-degree maxDegree composition-table

and *not-all*:

$\bigwedge i$ *trace-openings composition-openings trace-table*
composition-table prefix prefix-state.
 $i < \text{rounds} \implies$
partial-trace-table-candidate trace-table
 $((\text{replicate rounds } []) [i := \text{trace-openings}]) \implies$
partial-composition-table-candidate composition-table
 $((\text{replicate rounds } []) [i := \text{composition-openings}]) \implies$
 $\text{Some } ((\text{prefix}, \text{prefix-state}), \text{prefix-state}) \in$
set-dist
 $(\text{execute } (\text{checked-staged-query-prefix-with-state } A \ i)$
adversary-initial-state) $\implies$
 $\neg \text{all-queries-consistent trace-table composition-table}$
 $(\text{sqp-alphas prefix})$

shows

checked-staged-security-with-actual-alpha-prefix-partial-merkle-inconsistency-bad
 $(\text{Some } (((((\text{alpha-prefix}, \text{alpha-prefix-state}), \text{data}), \text{attacker-state}),$
result), *final-state*)) $\vee$
 $(\exists i$ *trace-table composition-table*.
checked-staged-security-with-actual-alpha-prefix-candidate-pair-query-hit-from-prefix
 A *trace-table composition-table* i
 $(\text{Some } (((((\text{alpha-prefix}, \text{alpha-prefix-state}), \text{data}), \text{attacker-state}),$
result), *final-state*)))

proof –

let *?out* =
 $\text{Some } (((((\text{alpha-prefix}, \text{alpha-prefix-state}), \text{data}), \text{attacker-state}),$
result), *final-state*)
let *?vout* = $\text{Some } (\text{result}, \text{final-state})$
let *?s* =
verifier-state-from-adversary attacker-state
 $(\text{staged-proof-transcript data})$

```

have data-support:
  Some (((data, attacker-state), result), final-state) ∈
    set-dist
      (execute (checked-staged-security-experiment-with-data-state A)
        adversary-initial-state)
  by (rule
    checked-staged-security-with-actual-alpha-prefix-support-imp-data-state-support
    [OF support])
from checked-staged-security-experiment-with-data-state-outcomeE
  [OF data-support]
obtain builder verifier where
  verifier:
    Some (result, final-state) ∈ set-dist (execute verify-monad ?s)
  by blast
from hit obtain i trace-openings composition-openings where
  comp-nonempty: staged-composition-fri-roots data ≠ []
  and trace-auth:
    partial-authenticated-table (staged-trace-root data)
      (scale * clength) trace-openings final-state
  and comp-auth:
    partial-authenticated-table (hd (staged-composition-fri-roots data))
      (scale * clength) composition-openings final-state
  and component:
    checked-staged-security-with-actual-alpha-prefix-candidate-opening-query-hit-from-prefix
      A
      ((replicate rounds []) [i := trace-openings])
      ((replicate rounds []) [i := composition-openings])
      i ?out
  unfolding
    checked-staged-security-with-actual-alpha-prefix-authenticated-candidate-opening-query-hit-from-prefix-def
  by auto
have i-bound: i < rounds
  using component
  unfolding
    checked-staged-security-with-actual-alpha-prefix-candidate-opening-query-hit-from-prefix-def
  by (simp split: option.splits prod.splits)
show ?thesis
proof (cases partial-merkle-inconsistency-bad ?s ?vout)
  case True
  then show ?thesis
    unfolding
      checked-staged-security-with-actual-alpha-prefix-partial-merkle-inconsistency-bad-def
      checked-staged-security-with-actual-alpha-prefix-verifier-event-def
    by (simp add: Let-def)
next
  case no-bad: False
  obtain trace-table where trace-candidate:
    partial-trace-table-candidate trace-table
    ((replicate rounds []) [i := trace-openings])

```

```

using
  partial-trace-table-candidate-single-witness-if-no-partial-merkle-bad
  [OF trace-auth verifier no-bad i-bound]
by blast
obtain composition-table where comp-candidate:
  partial-composition-table-candidate composition-table
  ((replicate rounds []) [i := composition-openings])
using
  partial-composition-table-candidate-single-witness-if-no-partial-merkle-bad
  [OF comp-auth verifier no-bad i-bound]
by blast
have pair-hit:
  checked-staged-security-with-actual-alpha-prefix-candidate-pair-query-hit-from-prefix
  A trace-table composition-table i ?out
by (rule
  checked-staged-security-with-actual-alpha-prefix-candidate-opening-query-hit-from-prefix-imp-candidate-pair-que
  [OF component trace-candidate comp-candidate
  trace-low[OF i-bound trace-candidate comp-candidate]
  comp-low[OF i-bound trace-candidate comp-candidate]])
  (rule not-all[OF i-bound trace-candidate comp-candidate])
then show ?thesis
  using i-bound by blast
qed
qed

lemma checked-staged-security-with-actual-alpha-prefix-authenticated-candidate-opening-query-hit-from-prefix-imp-ac
assumes support:
  Some (((((alpha-prefix, alpha-prefix-state), data), attacker-state),
  result), final-state) ∈
  set-dist
  (execute
  (checked-staged-security-experiment-with-actual-alpha-prefix-data-state
  A)
  adversary-initial-state)
and hit:
  checked-staged-security-with-actual-alpha-prefix-authenticated-candidate-opening-query-hit-from-prefix
  A
  (Some (((((alpha-prefix, alpha-prefix-state), data), attacker-state),
  result), final-state))
and trace-low:
  ∧ i trace-openings composition-openings trace-table
  composition-table.
  i < rounds ⇒
  partial-trace-table-candidate trace-table
  ((replicate rounds []) [i := trace-openings]) ⇒
  partial-composition-table-candidate composition-table
  ((replicate rounds []) [i := composition-openings]) ⇒
  trace-table-low-degree trace-table
and comp-low:

```

$\wedge i$ trace-openings composition-openings trace-table
 composition-table.
 $i < \text{rounds} \implies$
 partial-trace-table-candidate trace-table
 $((\text{replicate rounds } []) [i := \text{trace-openings}]) \implies$
 partial-composition-table-candidate composition-table
 $((\text{replicate rounds } []) [i := \text{composition-openings}]) \implies$
 composition-table-low-degree maxDegree composition-table
and not-all:
 $\wedge i$ trace-openings composition-openings trace-table
 composition-table prefix prefix-state.
 $i < \text{rounds} \implies$
 partial-trace-table-candidate trace-table
 $((\text{replicate rounds } []) [i := \text{trace-openings}]) \implies$
 partial-composition-table-candidate composition-table
 $((\text{replicate rounds } []) [i := \text{composition-openings}]) \implies$
 Some ((prefix, prefix-state), prefix-state) $\in$
 set-dist
 (execute (checked-staged-query-prefix-with-state A i)
 adversary-initial-state) $\implies$
 $\neg$ all-queries-consistent trace-table composition-table
 (sqp-alphas prefix)
shows
 checked-staged-security-with-actual-alpha-prefix-partial-merkle-inconsistency-bad
 (Some (((((alpha-prefix, alpha-prefix-state), data), attacker-state),
 result), final-state)) $\vee$
 $(\exists i < \text{rounds}.$
 checked-staged-security-with-actual-alpha-prefix-single-round-header-supported-query-target-hit-at
 i A
 (Some (((((alpha-prefix, alpha-prefix-state), data), attacker-state),
 result), final-state)) $\vee$
 checked-staged-security-with-actual-alpha-prefix-query-prefix-single-round-candidate-binding-gap-at
 i A
 (Some (((((alpha-prefix, alpha-prefix-state), data), attacker-state),
 result), final-state))))
proof –
have split:
 checked-staged-security-with-actual-alpha-prefix-partial-merkle-inconsistency-bad
 (Some (((((alpha-prefix, alpha-prefix-state), data), attacker-state),
 result), final-state)) $\vee$
 $(\exists i$ trace-table composition-table.
 checked-staged-security-with-actual-alpha-prefix-candidate-pair-query-hit-from-prefix
 A trace-table composition-table i
 (Some (((((alpha-prefix, alpha-prefix-state), data), attacker-state),
 result), final-state))))
by (rule
 checked-staged-security-with-actual-alpha-prefix-authenticated-candidate-opening-query-hit-from-prefix-imp-actua
 [OF support hit trace-low comp-low not-all])
then show ?thesis

```

proof
  assume
    checked-staged-security-with-actual-alpha-prefix-partial-merkle-inconsistency-bad
    (Some (((alpha-prefix, alpha-prefix-state), data), attacker-state),
      result), final-state))
  then show ?thesis by simp
next
  assume
     $\exists i$  trace-table composition-table.
    checked-staged-security-with-actual-alpha-prefix-candidate-pair-query-hit-from-prefix
    A trace-table composition-table i
    (Some (((alpha-prefix, alpha-prefix-state), data), attacker-state),
      result), final-state))
  then obtain i trace-table composition-table where pair-hit:
    checked-staged-security-with-actual-alpha-prefix-candidate-pair-query-hit-from-prefix
    A trace-table composition-table i
    (Some (((alpha-prefix, alpha-prefix-state), data), attacker-state),
      result), final-state))
  by blast
  have i-bound: i < rounds
  by (rule
    checked-staged-security-with-actual-alpha-prefix-candidate-pair-query-hit-from-prefixE
    [OF pair-hit])
  have
    checked-staged-security-with-actual-alpha-prefix-single-round-header-supported-query-target-hit-at
    i A
    (Some (((alpha-prefix, alpha-prefix-state), data), attacker-state),
      result), final-state))  $\vee$ 
    checked-staged-security-with-actual-alpha-prefix-query-prefix-single-round-candidate-binding-gap-at
    i A
    (Some (((alpha-prefix, alpha-prefix-state), data), attacker-state),
      result), final-state))
  by (rule
    checked-staged-security-with-actual-alpha-prefix-candidate-pair-query-hit-from-prefix-imp-single-round-header-target-hit-at
    [OF pair-hit])
  then show ?thesis
  using i-bound by blast
qed
qed

lemma checked-staged-security-with-actual-alpha-prefix-authenticated-candidate-opening-query-hit-from-prefix-bound
fixes PM :: prob and T G :: nat  $\Rightarrow$  prob
assumes split:
   $\bigwedge out.$ 
  out  $\in$ 
    set-dist
    (execute
      (checked-staged-security-experiment-with-actual-alpha-prefix-data-state
        A)

```

$\text{adversary-initial-state}) \implies$
checked-staged-security-with-actual-alpha-prefix-authenticated-candidate-opening-query-hit-from-prefix
 $A \text{ out} \implies$
checked-staged-security-with-actual-alpha-prefix-partial-merkle-inconsistency-bad
 $\text{out} \vee$
 $(\exists i < \text{rounds}.$
checked-staged-security-with-actual-alpha-prefix-single-round-header-supported-query-target-hit-at
 $i \ A \ \text{out} \vee$
checked-staged-security-with-actual-alpha-prefix-query-prefix-single-round-candidate-binding-gap-at
 $i \ A \ \text{out})$
and *partial-merkle-bound*:
 wp-event
 $(\text{checked-staged-security-experiment-with-actual-alpha-prefix-data-state}$
 $A)$
checked-staged-security-with-actual-alpha-prefix-partial-merkle-inconsistency-bad
 $\text{adversary-initial-state} \leq PM$
and *target-bound*:
 $\bigwedge i. i < \text{rounds} \implies$
 wp-event
 $(\text{checked-staged-security-experiment-with-actual-alpha-prefix-data-state}$
 $A)$
 $(\text{checked-staged-security-with-actual-alpha-prefix-single-round-header-supported-query-target-hit-at}$
 $i \ A)$
 $\text{adversary-initial-state} \leq T \ i$
and *gap-bound*:
 $\bigwedge i. i < \text{rounds} \implies$
 wp-event
 $(\text{checked-staged-security-experiment-with-actual-alpha-prefix-data-state}$
 $A)$
 $(\text{checked-staged-security-with-actual-alpha-prefix-query-prefix-single-round-candidate-binding-gap-at}$
 $i \ A)$
 $\text{adversary-initial-state} \leq G \ i$
shows
 wp-event
 $(\text{checked-staged-security-experiment-with-actual-alpha-prefix-data-state } A)$
 $(\text{checked-staged-security-with-actual-alpha-prefix-authenticated-candidate-opening-query-hit-from-prefix}$
 $A)$
 $\text{adversary-initial-state} \leq$
 $PM + (\sum i < \text{rounds}. T \ i + G \ i)$
proof –
let $?M =$
 $\text{checked-staged-security-experiment-with-actual-alpha-prefix-data-state } A$
let $?auth =$
 $\text{checked-staged-security-with-actual-alpha-prefix-authenticated-candidate-opening-query-hit-from-prefix}$
 A
let $?pm =$
 $\text{checked-staged-security-with-actual-alpha-prefix-partial-merkle-inconsistency-bad}$
let $?round =$
 $\lambda i \ \text{out}.$

checked-staged-security-with-actual-alpha-prefix-single-round-header-supported-query-target-hit-at
 $i \ A \ out \vee$
checked-staged-security-with-actual-alpha-prefix-query-prefix-single-round-candidate-binding-gap-at
 $i \ A \ out$
have *event-le*:
 $wp\text{-}event \ ?M \ ?auth \ adversary\text{-}initial\text{-}state \leq$
 $wp\text{-}event \ ?M \ (\lambda out. \ ?pm \ out \vee (\exists i \in \{..<rounds\}. \ ?round \ i \ out))$
 $adversary\text{-}initial\text{-}state$
by (rule *wp-event-mono-on-support*)
 (use *split* **in** *auto*)
have *union-le*:
 $wp\text{-}event \ ?M \ (\lambda out. \ ?pm \ out \vee (\exists i \in \{..<rounds\}. \ ?round \ i \ out))$
 $adversary\text{-}initial\text{-}state \leq$
 $wp\text{-}event \ ?M \ ?pm \ adversary\text{-}initial\text{-}state +$
 $wp\text{-}event \ ?M \ (\lambda out. \ \exists i \in \{..<rounds\}. \ ?round \ i \ out)$
 $adversary\text{-}initial\text{-}state$
by (rule *wp-event-union-bound*)
have *round-bound*:
 $\bigwedge i. \ i \in \{..<rounds\} \implies$
 $wp\text{-}event \ ?M \ (?round \ i) \ adversary\text{-}initial\text{-}state \leq T \ i + G \ i$
proof –
fix *i*
assume *i-bound*: $i \in \{..<rounds\}$
have
 $wp\text{-}event \ ?M \ (?round \ i) \ adversary\text{-}initial\text{-}state \leq$
 $wp\text{-}event \ ?M$
 (*checked-staged-security-with-actual-alpha-prefix-single-round-header-supported-query-target-hit-at*
 $i \ A$)
 $adversary\text{-}initial\text{-}state +$
 $wp\text{-}event \ ?M$
 (*checked-staged-security-with-actual-alpha-prefix-query-prefix-single-round-candidate-binding-gap-at*
 $i \ A$)
 $adversary\text{-}initial\text{-}state$
by (rule *wp-event-union-bound*)
also have $\dots \leq T \ i + G \ i$
by (intro *add-mono target-bound gap-bound*)
 (use *i-bound* **in** *simp-all*)
finally show $wp\text{-}event \ ?M \ (?round \ i) \ adversary\text{-}initial\text{-}state \leq T \ i + G \ i .$
qed
have *rounds-bound*:
 $wp\text{-}event \ ?M \ (\lambda out. \ \exists i \in \{..<rounds\}. \ ?round \ i \ out)$
 $adversary\text{-}initial\text{-}state \leq (\sum i<rounds. \ T \ i + G \ i)$
by (rule *wp-event-finite-union-bound*)
 (*simp-all* add: *round-bound*)
have
 $wp\text{-}event \ ?M \ (\lambda out. \ ?pm \ out \vee (\exists i \in \{..<rounds\}. \ ?round \ i \ out))$
 $adversary\text{-}initial\text{-}state \leq PM + (\sum i<rounds. \ T \ i + G \ i)$
by (rule *order-trans[OF union-le]*)
 (intro *add-mono partial-merkle-bound rounds-bound*)

then show *?thesis*
by (rule order-trans[*OF event-le*])
qed

lemma *checked-staged-security-with-data-state-composition-bad-bound-from-actual-single-round-components-and-bud.*

fixes *PM :: prob* **and** *T G :: nat $\Rightarrow$ prob*

assumes *wf: staged-budget-wellformed budgets*

and *controlled: staged-adversary-controlled budgets A*

and *split:*

$\bigwedge out.$

out $\in$

set-dist

(*execute*

(*checked-staged-security-experiment-with-actual-alpha-prefix-data-state*

A)

adversary-initial-state) $\Rightarrow$

checked-staged-security-with-actual-alpha-prefix-authenticated-candidate-opening-query-hit-from-prefix

A out $\Rightarrow$

checked-staged-security-with-actual-alpha-prefix-partial-merkle-inconsistency-bad

out $\vee$

($\exists i < rounds.$

checked-staged-security-with-actual-alpha-prefix-single-round-header-supported-query-target-hit-at

i A out $\vee$

checked-staged-security-with-actual-alpha-prefix-query-prefix-single-round-candidate-binding-gap-at

i A out)

and *partial-merkle-bound:*

wp-event

(*checked-staged-security-experiment-with-actual-alpha-prefix-data-state*

A)

checked-staged-security-with-actual-alpha-prefix-partial-merkle-inconsistency-bad

adversary-initial-state $\leq PM$

and *target-bound:*

$\bigwedge i. i < rounds \Rightarrow$

wp-event

(*checked-staged-security-experiment-with-actual-alpha-prefix-data-state*

A)

(*checked-staged-security-with-actual-alpha-prefix-single-round-header-supported-query-target-hit-at*

i A)

adversary-initial-state $\leq T i$

and *gap-bound:*

$\bigwedge i. i < rounds \Rightarrow$

wp-event

(*checked-staged-security-experiment-with-actual-alpha-prefix-data-state*

A)

(*checked-staged-security-with-actual-alpha-prefix-query-prefix-single-round-candidate-binding-gap-at*

i A)

adversary-initial-state $\leq G i$

shows

wp-event (checked-staged-security-experiment-with-data-state A)

```

    (staged-security-with-data-state-verifier-event composition-bad)
    adversary-initial-state ≤
    composition-error-bound +
    staged-phase-relation-error size
    (staged-alpha-search-queries budgets 0) +
    hash-collision-budget-value 0
    (staged-alpha-search-queries budgets 0) +
    staged-phase-relation-error size
    (staged-alpha-search-queries budgets 0) +
    (PM + (∑ i<rounds. T i + G i)) +
    hash-collision-budget-value 0
    (staged-attacker-query-budget budgets +
    staged-challenge-query-budget + verifier-hash-query-budget)
proof –
  have authenticated-bound:
    wp-event
    (checked-staged-security-experiment-with-actual-alpha-prefix-data-state A)
    (checked-staged-security-with-actual-alpha-prefix-authenticated-candidate-opening-query-hit-from-prefix
    A)
    adversary-initial-state ≤ PM + (∑ i<rounds. T i + G i)
  by (rule
    checked-staged-security-with-actual-alpha-prefix-authenticated-candidate-opening-query-hit-from-prefix-bound-from
    [OF split partial-merkle-bound target-bound gap-bound])
  show ?thesis
  by (rule
    checked-staged-security-with-data-state-composition-bad-bound-from-authenticated-query-from-prefix-and-budgets
    [OF wf controlled authenticated-bound])
qed

end

end

```

theory *Soundness-Staged-Bounds*

imports

Soundness-Staged-Transcript
Soundness-Staged-Partial-Query
Soundness-Staged-Partial-Query-Branch
Staged-Security-Experiment-Composition-Partial
Staged-Security-Experiment-Composition-Query-Residual
Staged-Security-Experiment-Composition-Query-Actual-Run

begin

Staged soundness bad-event bounds and accepted-execution partitions.

context *soundness*

begin

lemma *checked-staged-security-with-data-state-query-bad-verifier-event-bound-from-query-bad-rounds:*

assumes *wf*: staged-budget-wellformed budgets
and *controlled*: staged-adversary-controlled budgets *A*
and *round-bound*:
 $\bigwedge i. i < \text{rounds} \implies$
 $\text{wp-event } (\text{checked-staged-security-experiment-with-data-state } A)$
 $(\text{staged-security-with-data-state-query-bad-hit-at } i)$
 $\text{adversary-initial-state} \leq C \ i$
shows
 $\text{wp-event } (\text{checked-staged-security-experiment-with-data-state } A)$
 $(\text{staged-security-with-data-state-verifier-event } \text{query-bad})$
 $\text{adversary-initial-state} \leq (\sum i < \text{rounds}. C \ i)$
proof –
have $\text{wp-event } (\text{checked-staged-security-experiment-with-data-state } A)$
 $(\text{staged-security-with-data-state-verifier-event } \text{query-bad})$
 $\text{adversary-initial-state} \leq$
 $\text{wp-event } (\text{checked-staged-security-experiment-with-data-state } A)$
 $\text{staged-security-with-data-state-query-bad-hit}$
 $\text{adversary-initial-state}$
by (rule *checked-staged-security-with-data-state-query-bad-verifier-event-bound*)
 simp
also have $\dots \leq (\sum i < \text{rounds}. C \ i)$
by (rule
 $\text{checked-staged-security-with-data-state-query-bad-hit-bound-from-rounds}$)
 $(\text{rule } \text{round-bound})$
finally show *?thesis* .
qed

lemma *checked-staged-security-with-data-state-query-bad-verifier-event-bound-from-committed-prefix-hit-or-tree-output*

fixes *committed-error tree-error* :: prob
assumes *wf*: staged-budget-wellformed budgets
and *controlled*: staged-adversary-controlled budgets *A*
and *committed-bound*:
 $\text{wp-event } (\text{checked-staged-security-experiment-with-data-state } A)$
 $(\text{staged-security-with-data-state-query-committed-prefix-hit } A)$
 $\text{adversary-initial-state} \leq \text{committed-error}$
and *tree-bound*:
 $\text{wp-event } (\text{checked-staged-security-experiment-with-data-state } A)$
 $(\text{staged-security-with-data-state-query-tree-output-hit } A)$
 $\text{adversary-initial-state} \leq \text{tree-error}$
shows
 $\text{wp-event } (\text{checked-staged-security-experiment-with-data-state } A)$
 $(\text{staged-security-with-data-state-verifier-event } \text{query-bad})$
 $\text{adversary-initial-state} \leq \text{committed-error} + \text{tree-error}$

proof –
have $\text{wp-event } (\text{checked-staged-security-experiment-with-data-state } A)$
 $(\text{staged-security-with-data-state-verifier-event } \text{query-bad})$
 $\text{adversary-initial-state} \leq$
 $\text{wp-event } (\text{checked-staged-security-experiment-with-data-state } A)$
 $\text{staged-security-with-data-state-query-bad-hit}$

```

    adversary-initial-state
  by (rule checked-staged-security-with-data-state-query-bad-verifier-event-bound)
    simp
also have ... ≤
  wp-event (checked-staged-security-experiment-with-data-state A)
    (staged-security-with-data-state-query-committed-prefix-hit A)
    adversary-initial-state +
  wp-event (checked-staged-security-experiment-with-data-state A)
    (staged-security-with-data-state-query-tree-output-hit A)
    adversary-initial-state
  by (rule
    checked-staged-security-with-data-state-query-bad-bound-from-committed-prefix-hit-or-tree-output
    [OF wf controlled])
also have ... ≤ committed-error + tree-error
  by (intro add-mono committed-bound tree-bound)
finally show ?thesis .
qed

lemma checked-staged-security-with-data-state-query-bad-verifier-event-bound-from-length-committed-prefix-hit-or-tree-output
  fixes committed-error tree-error :: prob
  assumes wf: staged-budget-wellformed budgets
  and controlled: staged-adversary-controlled budgets A
  and committed-bound:
    wp-event (checked-staged-security-experiment-with-data-state A)
      (staged-security-with-data-state-query-length-committed-prefix-hit A)
      adversary-initial-state ≤ committed-error
  and tree-bound:
    wp-event (checked-staged-security-experiment-with-data-state A)
      (staged-security-with-data-state-query-tree-output-hit A)
      adversary-initial-state ≤ tree-error
  shows
    wp-event (checked-staged-security-experiment-with-data-state A)
      (staged-security-with-data-state-verifier-event query-bad)
      adversary-initial-state ≤ committed-error + tree-error
proof -
  have wp-event (checked-staged-security-experiment-with-data-state A)
    (staged-security-with-data-state-verifier-event query-bad)
    adversary-initial-state ≤
  wp-event (checked-staged-security-experiment-with-data-state A)
    staged-security-with-data-state-query-bad-hit
    adversary-initial-state
  by (rule checked-staged-security-with-data-state-query-bad-verifier-event-bound)
    simp
also have ... ≤
  wp-event (checked-staged-security-experiment-with-data-state A)
    (staged-security-with-data-state-query-length-committed-prefix-hit A)
    adversary-initial-state +
  wp-event (checked-staged-security-experiment-with-data-state A)
    (staged-security-with-data-state-query-tree-output-hit A)
    adversary-initial-state

```

```

    adversary-initial-state
  by (rule
    checked-staged-security-with-data-state-query-bad-bound-from-length-committed-prefix-hit-or-tree-output
    [OF wf controlled])
  also have ...  $\leq$  committed-error + tree-error
    by (intro add-mono committed-bound tree-bound)
  finally show ?thesis .
qed

lemma staged-security-with-data-state-query-tree-output-hit-imp-partial-header-key-hit-or-collision:
  assumes wf: staged-budget-wellformed budgets
    and controlled: staged-adversary-controlled budgets A
    and tree-hit:
      staged-security-with-data-state-query-tree-output-hit A out
  shows
    staged-security-with-data-state-query-partial-header-key-hit out  $\vee$ 
    staged-security-with-data-state-verifier-event
    hash-map-output-collision-bad out
proof (cases out)
  case None
  then show ?thesis
    using tree-hit
    unfolding staged-security-with-data-state-query-tree-output-hit-def
    by simp
next
  case (Some packed)
  then obtain data attacker-state result final-state where out-eq:
    out = Some (((data, attacker-state), result), final-state)
    by (cases packed, auto split: prod.splits)
  let ?s =
    verifier-state-from-adversary attacker-state
    (staged-proof-transcript data)
  have tree-body:
     $\exists i$  prefix prefix-state raw raw-state trace-table composition-table
      as query-idxs trace-tree composition-tree.
     $i < \text{rounds} \wedge$ 
    Some (data, attacker-state)  $\in$ 
      set-dist
        (execute (checked-staged-transcript-program A)
          adversary-initial-state)  $\wedge$ 
    Some (result, final-state)  $\in$  set-dist (execute verify-monad ?s)  $\wedge$ 
    accepted-with-bound-tables ?s (Some (result, final-state))
    trace-table composition-table as query-idxs  $\wedge$ 
    fmlookup (HashMap final-state)
      (QueryIndexChallenge i
        (state-after-query-chunks
          (staged-query-start-hash data) (staged-query-chunks data) i)) =
    Some raw  $\wedge$ 
    index (to-nat raw)  $\in$ 

```

```

    query-sampling-success-space trace-table composition-table
      (staged-alphas data)  $\wedge$ 
    Some (((prefix, prefix-state), raw), raw-state)  $\in$ 
      set-dist
        (execute (checked-staged-query-prefix-receive-with-state A i)
          adversary-initial-state)  $\wedge$ 
    sqp-composition-fri-roots prefix  $\neq$  []  $\wedge$ 
    created-tree trace-table trace-tree final-state  $\wedge$ 
    sqp-trace-root prefix = value trace-tree  $\wedge$ 
    created-tree composition-table composition-tree final-state  $\wedge$ 
    hd (sqp-composition-fri-roots prefix) = value composition-tree  $\wedge$ 
    hash-map-new-output-hit
      (set-tree trace-tree  $\cup$  set-tree composition-tree)
      prefix-state final-state
  using tree-hit
  unfolding out-eq
    staged-security-with-data-state-query-tree-output-hit-def Let-def
  by simp
from tree-body
obtain i prefix prefix-state raw raw-state trace-table composition-table
  as query-idxs trace-tree composition-tree where
  i-bound:  $i < \text{rounds}$ 
and builder:
  Some (data, attacker-state)  $\in$ 
    set-dist
      (execute (checked-staged-transcript-program A)
        adversary-initial-state)
and verifier:
  Some (result, final-state)  $\in$ 
    set-dist (execute verify-monad ?s)
and bound:
  accepted-with-bound-tables ?s (Some (result, final-state))
  trace-table composition-table as query-idxs
and lookup-final-raw:
  fmlookup (HashMap final-state)
    (QueryIndexChallenge i
      (state-after-query-chunks
        (staged-query-start-hash data) (staged-query-chunks data) i)) =
    Some raw
and raw-hit:
  index (to-nat raw)  $\in$ 
    query-sampling-success-space trace-table composition-table
      (staged-alphas data)
  by blast
have accepted-out: accepted (Some (result, final-state))
  unfolding accepted-def by simp
show ?thesis
proof (cases hash-map-output-collision final-state)
  case True

```

```

then show ?thesis
  unfolding out-eq staged-security-with-data-state-verifier-event-def
    hash-map-output-collision-bad-def
  using accepted-out by simp
next
case clean: False
have staged-header:
  verifier-header-transcript ?s
    (staged-trace-root data)
    (staged-trace-fri-roots data)
    (staged-trace-final data)
    (staged-alphas data)
    (staged-degree data)
    (staged-composition-fri-roots data)
    (staged-composition-final data)
    (List.concat (staged-query-chunks data))
  by (rule checked-staged-transcript-program-outcome-header-transcript
    [OF wf controlled builder])
from bound obtain fr f-fri-roots f-final dg composition-fri-roots final
  rest where
    header:
      verifier-header-transcript ?s fr f-fri-roots f-final as dg
        composition-fri-roots final rest
  and trace-bind:
    merkle-root-binds-table fr trace-table final-state
  and comp-nonempty: composition-fri-roots  $\neq []$ 
  and comp-bind:
    merkle-root-binds-table (hd composition-fri-roots)
      composition-table final-state
  unfolding accepted-with-bound-tables-def by blast
have eqs:
  fr = staged-trace-root data  $\wedge$ 
  f-fri-roots = staged-trace-fri-roots data  $\wedge$ 
  f-final = staged-trace-final data  $\wedge$ 
  as = staged-alphas data  $\wedge$ 
  dg = staged-degree data  $\wedge$ 
  composition-fri-roots = staged-composition-fri-roots data  $\wedge$ 
  final = staged-composition-final data  $\wedge$ 
  rest = List.concat (staged-query-chunks data)
  using verifier-header-transcript-unique[OF header staged-header]
  by simp
have state:
  query-header-supported-table-candidate-state ?s
    (staged-trace-root data)
    (staged-trace-fri-roots data)
    (staged-trace-final data)
    (staged-alphas data)
    (staged-degree data)
    (staged-composition-fri-roots data)

```

```

    (staged-composition-final data)
    trace-table composition-table final-state
unfolding query-header-supported-table-candidate-state-def
by (intro exI[of - result] exI[of - query-idxs]
    exI[of - List.concat (staged-query-chunks data)])
    (use verifier bound staged-header trace-bind comp-nonempty comp-bind
    eqs in simp)
have candidate:
  (trace-table, composition-table) ∈
    query-header-supported-partial-table-candidates ?s
    (staged-trace-root data)
    (staged-trace-fri-roots data)
    (staged-trace-final data)
    (staged-alphas data)
    (staged-degree data)
    (staged-composition-fri-roots data)
    (staged-composition-final data)
using
  query-header-supported-table-candidate-state-imp-partial-candidate-state
  [OF state clean]
  query-header-supported-partial-table-candidates-iff-state
by blast
have raw-partial-hit:
  index (to-nat raw) ∈
    query-header-supported-partial-union-good-sets ?s
    query-sampling-success-space
    (staged-trace-root data)
    (staged-trace-fri-roots data)
    (staged-trace-final data)
    (staged-alphas data)
    (staged-degree data)
    (staged-composition-fri-roots data)
    (staged-composition-final data)
proof —
  have raw-in-sample:
    index (to-nat raw) ∈ query-sample-space
    using raw-hit query-sampling-success-space-subset by blast
  show ?thesis
  unfolding query-header-supported-partial-union-good-sets-def
    using candidate raw-hit raw-in-sample by blast
qed
then show ?thesis
  unfolding out-eq
    staged-security-with-data-state-query-partial-header-key-hit-def
    Let-def
  using i-bound lookup-final-raw by auto
qed
qed

```


lemma *staged-security-with-data-state-query-tree-output-hit-bound-from-partial-header-key-hit-or-collision:*

assumes *wf: staged-budget-wellformed budgets*

and *controlled: staged-adversary-controlled budgets A*

shows

wp-event (checked-staged-security-experiment-with-data-state A)
(staged-security-with-data-state-query-tree-output-hit A)
adversary-initial-state ≤
wp-event (checked-staged-security-experiment-with-data-state A)
staged-security-with-data-state-query-partial-header-key-hit
adversary-initial-state +
wp-event (checked-staged-security-experiment-with-data-state A)
(staged-security-with-data-state-verifier-event
hash-map-output-collision-bad)
adversary-initial-state

proof –

let *?M = checked-staged-security-experiment-with-data-state A*

let *?partial = staged-security-with-data-state-query-partial-header-key-hit*

let *?collision =*

staged-security-with-data-state-verifier-event
hash-map-output-collision-bad

have *wp-event ?M (staged-security-with-data-state-query-tree-output-hit A)*

adversary-initial-state ≤

wp-event ?M (λout. ?partial out ∨ ?collision out)

adversary-initial-state

by *(rule wp-event-mono)*

(rule

staged-security-with-data-state-query-tree-output-hit-imp-partial-header-key-hit-or-collision
[OF wf controlled])

also have *... ≤*

wp-event ?M ?partial adversary-initial-state +

wp-event ?M ?collision adversary-initial-state

by *(rule wp-event-union-bound)*

finally show *?thesis .*

qed

lemma *checked-staged-security-with-data-state-query-bad-verifier-event-bound-by-length-prefixes-partial-header-and-collision-error :: prob*

fixes *partial-error collision-error :: prob*

assumes *raw-bound: query-index-raw-preimage-bound*

and *wf: staged-budget-wellformed budgets*

and *controlled: staged-adversary-controlled budgets A*

and *partial-bound:*

wp-event (checked-staged-security-experiment-with-data-state A)
staged-security-with-data-state-query-partial-header-key-hit
adversary-initial-state ≤ partial-error

and *collision-bound:*

wp-event (checked-staged-security-experiment-with-data-state A)
(staged-security-with-data-state-verifier-event
hash-map-output-collision-bad)
adversary-initial-state ≤ collision-error

shows

wp-event (checked-staged-security-experiment-with-data-state A)
(staged-security-with-data-state-verifier-event query-bad)
adversary-initial-state $\leq$
 $(\sum i < \text{rounds}.$
 staged-phase-relation-error size
 (staged-query-search-queries budgets i + 1) +
 query-error-bound +
 hash-collision-budget-value 0
 (staged-query-search-queries budgets i + 1)) +
(partial-error + collision-error)

proof –

let *?M* = *checked-staged-security-experiment-with-data-state A*
let *?Tree* = *staged-security-with-data-state-query-tree-output-hit A*
let *?PrefixSum* =
 $(\sum i < \text{rounds}.$
 staged-phase-relation-error size
 (staged-query-search-queries budgets i + 1) +
 query-error-bound +
 hash-collision-budget-value 0
 (staged-query-search-queries budgets i + 1))

have *tree-bound*:

wp-event ?M ?Tree adversary-initial-state $\leq$
partial-error + collision-error

proof –

have *wp-event ?M ?Tree adversary-initial-state* $\leq$
wp-event ?M staged-security-with-data-state-query-partial-header-key-hit
adversary-initial-state +
wp-event ?M
(staged-security-with-data-state-verifier-event
 hash-map-output-collision-bad)
adversary-initial-state
by (rule
 staged-security-with-data-state-query-tree-output-hit-bound-from-partial-header-key-hit-or-collision
 [*OF wf controlled*])
also have ... $\leq$ *partial-error + collision-error*
by (intro *add-mono partial-bound collision-bound*)
finally show *?thesis* .

qed

have *wp-event ?M*

(staged-security-with-data-state-verifier-event query-bad)
adversary-initial-state $\leq$
?PrefixSum + wp-event ?M ?Tree adversary-initial-state

by (rule

checked-staged-security-with-data-state-query-bad-verifier-event-bound-by-length-prefixes-and-tree
 [*OF raw-bound wf controlled*])

also have ... $\leq$ *?PrefixSum + (partial-error + collision-error)*

by (intro *add-mono order-refl tree-bound*)

finally show *?thesis* .

qed

lemma *checked-staged-security-with-data-state-query-bad-verifier-event-bound-by-length-prefixes-partial-header-and-collision-bound*

assumes *raw-bound: query-index-raw-preimage-bound*

and *wf: staged-budget-wellformed budgets*

and *controlled: staged-adversary-controlled budgets A*

and *partial-bound:*

wp-event (checked-staged-security-experiment-with-data-state A)

staged-security-with-data-state-query-partial-header-key-hit

adversary-initial-state ≤ partial-error

shows

wp-event (checked-staged-security-experiment-with-data-state A)

(staged-security-with-data-state-verifier-event query-bad)

adversary-initial-state ≤

($\sum i < \text{rounds}$.

staged-phase-relation-error size

(staged-query-search-queries budgets i + 1) +

query-error-bound +

hash-collision-budget-value 0

(staged-query-search-queries budgets i + 1)) +

(partial-error +

hash-collision-budget-value 0

(staged-attacker-query-budget budgets +

staged-challenge-query-budget + verifier-hash-query-budget))

proof –

have *collision-bound:*

wp-event (checked-staged-security-experiment-with-data-state A)

(staged-security-with-data-state-verifier-event

hash-map-output-collision-bad)

adversary-initial-state ≤

hash-collision-budget-value 0

(staged-attacker-query-budget budgets +

staged-challenge-query-budget + verifier-hash-query-budget)

by (rule

checked-staged-security-with-data-state-hash-map-output-collision-bad-bound-controlled

[OF wf controlled])

show *?thesis*

by (rule

checked-staged-security-with-data-state-query-bad-verifier-event-bound-by-length-prefixes-partial-header-and-collision-bound

[OF raw-bound wf controlled partial-bound collision-bound])

qed

lemma *checked-staged-security-with-data-state-query-bad-verifier-event-bound-by-length-prefixes-partial-header-and-collision-bound*

assumes *wf: staged-budget-wellformed budgets*

and *controlled: staged-adversary-controlled budgets A*

and *partial-bound:*

wp-event (checked-staged-security-experiment-with-data-state A)

staged-security-with-data-state-query-partial-header-key-hit

adversary-initial-state ≤ partial-error

shows

wp-event (checked-staged-security-experiment-with-data-state A)
(staged-security-with-data-state-verifier-event query-bad)
adversary-initial-state $\leq$
 $(\sum_{i < \text{rounds.}}$
staged-phase-relation-error size
(staged-query-search-queries budgets i + 1) +
query-error-bound +
hash-collision-budget-value 0
(staged-query-search-queries budgets i + 1)) +
(partial-error +
hash-collision-budget-value 0
(staged-attacker-query-budget budgets +
staged-challenge-query-budget + verifier-hash-query-budget))

proof –

have *raw-bound: query-index-raw-preimage-bound*
by *(rule query-index-raw-preimage-bound-from-sampler-wellformed)*
show *?thesis*
by *(rule*
checked-staged-security-with-data-state-query-bad-verifier-event-bound-by-length-prefixes-partial-header-and-control
[OF raw-bound wf controlled partial-bound])

qed

lemma *checked-staged-security-with-data-state-partial-trace-openings-inconsistent-bad-bound-controlled:*

assumes *wf: staged-budget-wellformed budgets*
and *controlled: staged-adversary-controlled budgets A*
shows

wp-event (checked-staged-security-experiment-with-data-state A)
(staged-security-with-data-state-verifier-event
partial-trace-openings-inconsistent-bad)
adversary-initial-state $\leq$
hash-collision-budget-value 0
(staged-attacker-query-budget budgets +
staged-challenge-query-budget + verifier-hash-query-budget)

proof –

have *wp-event (checked-staged-security-experiment-with-data-state A)*
(staged-security-with-data-state-verifier-event
partial-trace-openings-inconsistent-bad)
adversary-initial-state $\leq$
wp-event (checked-staged-security-experiment-with-data-state A)
(staged-security-with-data-state-verifier-event
hash-map-output-collision-bad)
adversary-initial-state
by *(rule wp-event-mono)*
(auto simp: staged-security-with-data-state-verifier-event-def
intro: partial-trace-openings-inconsistent-bad-imp-hash-map-output-collision-bad
split: option.splits prod.splits)
also have *... $\leq$*
hash-collision-budget-value 0

(staged-attacker-query-budget budgets +
 staged-challenge-query-budget + verifier-hash-query-budget)
 by (rule
 checked-staged-security-with-data-state-hash-map-output-collision-bad-bound-controlled
 [OF wf controlled])
 finally show ?thesis .
 qed

lemma *checked-staged-security-with-data-state-partial-composition-openings-inconsistent-bad-bound-controlled:*
assumes wf: staged-budget-wellformed budgets
and controlled: staged-adversary-controlled budgets A
shows

wp-event (checked-staged-security-experiment-with-data-state A)
 (staged-security-with-data-state-verifier-event
 partial-composition-openings-inconsistent-bad)
 adversary-initial-state ≤
 hash-collision-budget-value 0
 (staged-attacker-query-budget budgets +
 staged-challenge-query-budget + verifier-hash-query-budget)

proof –

have wp-event (checked-staged-security-experiment-with-data-state A)
 (staged-security-with-data-state-verifier-event
 partial-composition-openings-inconsistent-bad)
 adversary-initial-state ≤
 wp-event (checked-staged-security-experiment-with-data-state A)
 (staged-security-with-data-state-verifier-event
 hash-map-output-collision-bad)
 adversary-initial-state
by (rule wp-event-mono)
 (auto simp: staged-security-with-data-state-verifier-event-def
 intro:
 partial-composition-openings-inconsistent-bad-imp-hash-map-output-collision-bad
 split: option.splits prod.splits)
also have ... ≤
 hash-collision-budget-value 0
 (staged-attacker-query-budget budgets +
 staged-challenge-query-budget + verifier-hash-query-budget)
by (rule
 checked-staged-security-with-data-state-hash-map-output-collision-bad-bound-controlled
 [OF wf controlled])
 finally show ?thesis .
 qed

lemma *checked-staged-security-with-data-state-partial-merkle-inconsistency-bad-bound-controlled:*
assumes wf: staged-budget-wellformed budgets
and controlled: staged-adversary-controlled budgets A
shows

wp-event (checked-staged-security-experiment-with-data-state A)
 (staged-security-with-data-state-verifier-event

```

    partial-merkle-inconsistency-bad)
  adversary-initial-state ≤
  hash-collision-budget-value 0
  (staged-attacker-query-budget budgets +
   staged-challenge-query-budget + verifier-hash-query-budget)
proof –
  have wp-event (checked-staged-security-experiment-with-data-state A)
    (staged-security-with-data-state-verifier-event
     partial-merkle-inconsistency-bad)
  adversary-initial-state ≤
  wp-event (checked-staged-security-experiment-with-data-state A)
    (staged-security-with-data-state-verifier-event
     hash-map-output-collision-bad)
  adversary-initial-state
  by (rule wp-event-mono)
    (auto simp: staged-security-with-data-state-verifier-event-def
     intro: partial-merkle-inconsistency-bad-imp-hash-map-output-collision-bad
     split: option.splits prod.splits)
  also have ... ≤
    hash-collision-budget-value 0
    (staged-attacker-query-budget budgets +
     staged-challenge-query-budget + verifier-hash-query-budget)
  by (rule
    checked-staged-security-with-data-state-hash-map-output-collision-bad-bound-controlled
    [OF wf controlled])
  finally show ?thesis .
qed

```

lemma *checked-staged-security-with-actual-alpha-prefix-partial-merkle-inconsistency-bad-bound-controlled:*
assumes wf: staged-budget-wellformed budgets
and controlled: staged-adversary-controlled budgets A
shows

```

  wp-event
    (checked-staged-security-experiment-with-actual-alpha-prefix-data-state A)
  checked-staged-security-with-actual-alpha-prefix-partial-merkle-inconsistency-bad
  adversary-initial-state ≤
  hash-collision-budget-value 0
  (staged-attacker-query-budget budgets +
   staged-challenge-query-budget + verifier-hash-query-budget)

```

proof –

have event-eq:

```

  wp-event
    (checked-staged-security-experiment-with-actual-alpha-prefix-data-state A)
  checked-staged-security-with-actual-alpha-prefix-partial-merkle-inconsistency-bad
  adversary-initial-state =
  wp-event (checked-staged-security-experiment-with-data-state A)
    (staged-security-with-data-state-verifier-event
     partial-merkle-inconsistency-bad)
  adversary-initial-state

```

```

unfolding
  checked-staged-security-with-actual-alpha-prefix-partial-merkle-inconsistency-bad-def
using
  checked-staged-security-with-data-state-verifier-event-actual-alpha-prefix-projection
  [of A partial-merkle-inconsistency-bad]
by simp
show ?thesis
  unfolding event-eq
  by (rule
    checked-staged-security-with-data-state-partial-merkle-inconsistency-bad-bound-controlled
    [OF wf controlled])
qed

lemma checked-staged-security-with-data-state-supported-output-local-side-bad-bound-controlled:
assumes wf: staged-budget-wellformed budgets
and controlled: staged-adversary-controlled budgets A
shows
  wp-event (checked-staged-security-experiment-with-data-state A)
  (staged-security-with-data-state-verifier-event
    supported-output-local-side-bad)
  adversary-initial-state  $\leq$ 
  hash-collision-budget-value 0
  (staged-attacker-query-budget budgets +
    staged-challenge-query-budget + verifier-hash-query-budget)
proof –
  have wp-event (checked-staged-security-experiment-with-data-state A)
  (staged-security-with-data-state-verifier-event
    supported-output-local-side-bad)
  adversary-initial-state  $\leq$ 
  wp-event (checked-staged-security-experiment-with-data-state A)
  (staged-security-with-data-state-verifier-event
    hash-map-output-collision-bad)
  adversary-initial-state
  by (rule wp-event-mono)
  (auto simp: staged-security-with-data-state-verifier-event-def
    supported-output-local-side-bad-iff-hash-map-output-collision-bad
    split: option.splits prod.splits)
  also have ...  $\leq$ 
  hash-collision-budget-value 0
  (staged-attacker-query-budget budgets +
    staged-challenge-query-budget + verifier-hash-query-budget)
  by (rule
    checked-staged-security-with-data-state-hash-map-output-collision-bad-bound-controlled
    [OF wf controlled])
  finally show ?thesis .
qed

lemma checked-staged-security-with-data-state-supported-pairwise-side-bad-bound-from-coupling:
fixes coupling-error :: prob

```

```

assumes wf: staged-budget-wellformed budgets
and controlled: staged-adversary-controlled budgets A
and coupling-bound:
  wp-event (checked-staged-security-experiment-with-data-state A)
    (staged-security-with-data-state-verifier-event
      supported-pairwise-coupling-event)
  adversary-initial-state  $\leq$  coupling-error
shows
  wp-event (checked-staged-security-experiment-with-data-state A)
    (staged-security-with-data-state-verifier-event
      supported-pairwise-side-bad)
  adversary-initial-state  $\leq$ 
    hash-collision-budget-value 0
    (staged-attacker-query-budget budgets +
      staged-challenge-query-budget + verifier-hash-query-budget) +
    coupling-error
proof –
  let ?M = checked-staged-security-experiment-with-data-state A
  let ?E = staged-security-with-data-state-verifier-event
  have event-eq:
    ?E supported-pairwise-side-bad =
      ( $\lambda$ out. ?E hash-map-output-collision-bad out  $\vee$ 
        ?E supported-pairwise-coupling-event out)
  by (rule ext)
    (auto simp: staged-security-with-data-state-verifier-event-def
      supported-pairwise-side-bad-def
      split: option.splits prod.splits)
  have wp-event ?M (?E supported-pairwise-side-bad)
    adversary-initial-state  $\leq$ 
    wp-event ?M (?E hash-map-output-collision-bad)
      adversary-initial-state +
    wp-event ?M (?E supported-pairwise-coupling-event)
      adversary-initial-state
  unfolding event-eq by (rule wp-event-union-bound)
  also have ...  $\leq$ 
    hash-collision-budget-value 0
    (staged-attacker-query-budget budgets +
      staged-challenge-query-budget + verifier-hash-query-budget) +
    coupling-error
  by (intro add-mono coupling-bound
    checked-staged-security-with-data-state-hash-map-output-collision-bad-bound-controlled
    [OF wf controlled])
  finally show ?thesis .
qed

```

```

lemma checked-staged-security-with-data-state-query-bad-verifier-event-bound-from-committed-prefix-partial-header-a
fixes committed-error partial-error collision-error :: prob
assumes wf: staged-budget-wellformed budgets
and controlled: staged-adversary-controlled budgets A

```


and *committed-bound*:
 $wp\text{-event}$ (*checked-staged-security-experiment-with-data-state* A)
 $(\text{staged-security-with-data-state-query-committed-prefix-hit } A)$
 $\text{adversary-initial-state} \leq \text{committed-error}$
and *partial-bound*:
 $wp\text{-event}$ (*checked-staged-security-experiment-with-data-state* A)
 $\text{staged-security-with-data-state-query-partial-header-key-hit}$
 $\text{adversary-initial-state} \leq \text{partial-error}$
and *collision-bound*:
 $wp\text{-event}$ (*checked-staged-security-experiment-with-data-state* A)
 $(\text{staged-security-with-data-state-verifier-event}$
 $\text{hash-map-output-collision-bad})$
 $\text{adversary-initial-state} \leq \text{collision-error}$
shows
 $wp\text{-event}$ (*checked-staged-security-experiment-with-data-state* A)
 $(\text{staged-security-with-data-state-verifier-event query-bad})$
 $\text{adversary-initial-state} \leq$
 $\text{committed-error} + (\text{partial-error} + \text{collision-error})$
proof –
let $?M = \text{checked-staged-security-experiment-with-data-state } A$
have *tree-bound*:
 $wp\text{-event } ?M$ (*staged-security-with-data-state-query-tree-output-hit* A)
 $\text{adversary-initial-state} \leq \text{partial-error} + \text{collision-error}$
proof –
have $wp\text{-event } ?M$
 $(\text{staged-security-with-data-state-query-tree-output-hit } A)$
 $\text{adversary-initial-state} \leq$
 $wp\text{-event } ?M$ *staged-security-with-data-state-query-partial-header-key-hit*
 $\text{adversary-initial-state} +$
 $wp\text{-event } ?M$
 $(\text{staged-security-with-data-state-verifier-event}$
 $\text{hash-map-output-collision-bad})$
 $\text{adversary-initial-state}$
by (rule
 $\text{staged-security-with-data-state-query-tree-output-hit-bound-from-partial-header-key-hit-or-collision}$
 $[OF \text{ wf controlled}]$)
also have $\dots \leq \text{partial-error} + \text{collision-error}$
by (intro *add-mono partial-bound collision-bound*)
finally show $?thesis$.
qed
show $?thesis$
by (rule
 $\text{checked-staged-security-with-data-state-query-bad-verifier-event-bound-from-committed-prefix-hit-or-tree-output}$
 $[OF \text{ wf controlled committed-bound tree-bound}]$)
qed

lemma *checked-staged-security-with-data-state-query-bad-verifier-event-bound-from-partial-opening-fixed-query-error-*
assumes *wf*: *staged-budget-wellformed budgets*
and *controlled*: *staged-adversary-controlled budgets* A

and *cover*:
 $\bigwedge \text{data attacker-state } i.$
 $i < \text{rounds} \implies$
query-header-supported-partial-opening-success-indices-at
 $(\text{verifier-state-from-adversary attacker-state}$
 $(\text{staged-proof-transcript data}))$
 $(\text{staged-trace-root data})$
 $(\text{staged-trace-fri-roots data})$
 $(\text{staged-trace-final data})$
 $(\text{staged-alphas data})$
 $(\text{staged-degree data})$
 $(\text{staged-composition-fri-roots data})$
 $(\text{staged-composition-final data})$
 $i \subseteq B$
and *raw-bound*: *query-index-raw-preimage-bound*
and *subset*: $B \subseteq \text{query-sample-space}$
and *frac*:
 $\text{nnreal}(\text{card } B) / \text{nnreal}(\text{card query-sample-space}) \leq$
 query-error-bound
shows
 $\text{wp-event}(\text{checked-staged-security-experiment-with-data-state } A)$
 $(\text{staged-security-with-data-state-verifier-event query-bad})$
 $\text{adversary-initial-state} \leq$
 nnreal
 $(\text{staged-attacker-query-budget budgets} +$
 $\text{staged-challenge-query-budget}) * \text{query-error-bound}$
by (*rule checked-staged-security-with-data-state-query-bad-verifier-event-bound*)
 $(\text{rule}$
 $\text{checked-staged-security-with-data-state-query-bad-bound-from-partial-opening-fixed-query-error-cover}$
 $[\text{OF wf controlled cover raw-bound subset frac}])$

lemma *staged-transcript-query-partial-header-key-hit-iff-round-hit*:
 $\text{staged-transcript-query-partial-header-key-hit out} \longleftrightarrow$
 $(\exists i \in \{..<\text{rounds}\}.$
 $\text{staged-transcript-query-partial-header-key-hit-at } i \text{ out})$
unfolding *staged-transcript-query-partial-header-key-hit-alt*
 $\text{staged-transcript-query-partial-header-key-hit-at-def}$
by (*auto split: option.splits prod.splits*)

lemma *checked-staged-transcript-query-partial-header-key-hit-bound-from-rounds*:
assumes *round-bound*:
 $\bigwedge i. i < \text{rounds} \implies$
 $\text{wp-event}(\text{checked-staged-transcript-program } A)$
 $(\text{staged-transcript-query-partial-header-key-hit-at } i)$
 $\text{adversary-initial-state} \leq C i$
shows
 $\text{wp-event}(\text{checked-staged-transcript-program } A)$
 $\text{staged-transcript-query-partial-header-key-hit}$
 $\text{adversary-initial-state} \leq (\sum i < \text{rounds}. C i)$

```

proof –
  have event-eq:
    staged-transcript-query-partial-header-key-hit =
      ( $\lambda out. \exists i \in \{..<rounds\}.$ 
        staged-transcript-query-partial-header-key-hit-at i out)
  by (rule ext)
    (simp add:
      staged-transcript-query-partial-header-key-hit-iff-round-hit)
  show ?thesis
  unfolding event-eq
  by (rule wp-event-finite-union-bound)
    (simp-all add: round-bound)
qed

lemma checked-staged-security-with-data-state-partial-header-key-hit-bound-from-dynamic-transcript:
  assumes wf: staged-budget-wellformed budgets
  and controlled: staged-adversary-controlled budgets A
  and transcript-bound:
    wp-event (checked-staged-transcript-program A)
    staged-transcript-query-partial-header-key-hit
    adversary-initial-state  $\leq C$ 
  shows
    wp-event (checked-staged-security-experiment-with-data-state A)
    staged-security-with-data-state-query-partial-header-key-hit
    adversary-initial-state  $\leq C$ 
  unfolding checked-staged-security-experiment-with-data-state-def
proof (rule wp-event-bind-bound-by-head-event[OF transcript-bound])
  show
    staged-security-with-data-state-query-partial-header-key-hit None  $\implies$ 
    staged-transcript-query-partial-header-key-hit None
  unfolding staged-security-with-data-state-query-partial-header-key-hit-def
    staged-transcript-query-partial-header-key-hit-def
  by simp
next
  fix data attacker-state out
  assume builder:
    Some (data, attacker-state) \in
      set-dist
      (execute (checked-staged-transcript-program A)
        adversary-initial-state)
  and cont:
    out \in
      set-dist
      (execute
        (get  $\gg=$ 
          ( $\lambda s.$  put
            (verifier-state-from-adversary s
              (staged-proof-transcript data))  $\gg=$ 
            ( $\lambda -. \text{verify-monad}$   $\gg=$ 

```

```

      (λresult. return ((data, s), result))))
    attacker-state)
and hit:
  staged-security-with-data-state-query-partial-header-key-hit out
from checked-staged-transcript-program-outcome-query-lookups
  [OF wf controlled builder]
obtain raw-idxs query-idxs where
  len-raw: length raw-idxs = rounds
and lookup-attacker:
  ∀ j < rounds.
    fmlookup (HashMap attacker-state)
      (QueryIndexChallenge j
        (state-after-query-chunks
          (staged-query-start-hash data)
          (staged-query-chunks data) j)) =
      Some (raw-idxs ! j)
by blast
from hit obtain data' attacker-state' result final-state i raw where
  out-eq:
    out = Some (((data', attacker-state'), result), final-state)
and i-bound: i < rounds
and lookup-final:
  fmlookup (HashMap final-state)
    (QueryIndexChallenge i
      (state-after-query-chunks
        (staged-query-start-hash data')
        (staged-query-chunks data') i)) =
    Some raw
and raw-hit:
  index (to-nat raw) ∈
    query-header-supported-partial-union-good-sets
      (verifier-state-from-adversary attacker-state'
        (staged-proof-transcript data'))
    query-sampling-success-space
      (staged-trace-root data')
      (staged-trace-fri-roots data')
      (staged-trace-final data')
      (staged-alphas data')
      (staged-degree data')
      (staged-composition-fri-roots data')
      (staged-composition-final data')
unfolding staged-security-with-data-state-query-partial-header-key-hit-def
by (auto split: option.splits prod.splits)
from cont[unfolded out-eq] obtain verifier-state verifier-result where
  verifier:
    Some (verifier-result, final-state) ∈
      set-dist
        (execute verify-monad
          (verifier-state-from-adversary attacker-state

```

```

      (staged-proof-transcript data)))
and data'-eq: data' = data
and attacker-state'-eq: attacker-state' = attacker-state
by (auto elim!: set-dist-bindE)
have ext:
  verifier-state-from-adversary attacker-state
    (staged-proof-transcript data) ≤ final-state
by (rule verify-monad-hash-extends[OF verifier])
have lookup-initial:
  fmllookup
    (HashMap
      (verifier-state-from-adversary attacker-state
        (staged-proof-transcript data)))
    (QueryIndexChallenge i
      (state-after-query-chunks
        (staged-query-start-hash data)
        (staged-query-chunks data) i)) =
    Some (raw-idxs ! i)
using lookup-attacker i-bound by simp
have lookup-final-from-initial:
  fmllookup (HashMap final-state)
    (QueryIndexChallenge i
      (state-after-query-chunks
        (staged-query-start-hash data)
        (staged-query-chunks data) i)) =
    Some (raw-idxs ! i)
by (rule hash-extension-lookup[OF lookup-initial ext])
have raw-eq: raw = raw-idxs ! i
using lookup-final lookup-final-from-initial data'-eq by simp
have lookup-attacker-i:
  fmllookup (HashMap attacker-state)
    (QueryIndexChallenge i
      (state-after-query-chunks
        (staged-query-start-hash data)
        (staged-query-chunks data) i)) =
    Some (raw-idxs ! i)
using lookup-attacker i-bound by simp
have raw-idxs-hit:
  index (to-nat (raw-idxs ! i)) ∈
    query-header-supported-partial-union-good-sets
      (verifier-state-from-adversary attacker-state
        (staged-proof-transcript data))
    query-sampling-success-space
      (staged-trace-root data)
      (staged-trace-fri-roots data)
      (staged-trace-final data)
      (staged-alphas data)
      (staged-degree data)
      (staged-composition-fri-roots data)

```

```

    (staged-composition-final data)
  using raw-hit raw-eq data'-eq attacker-state'-eq by simp
show staged-transcript-query-partial-header-key-hit
  (Some (data, attacker-state))
  unfolding staged-transcript-query-partial-header-key-hit-def Let-def
  using i-bound lookup-attacker-i raw-idxs-hit by auto
qed

definition checked-staged-event-soundness-bound
  :: prob  $\Rightarrow$  prob  $\Rightarrow$  prob  $\Rightarrow$  prob
  where
    checked-staged-event-soundness-bound merkle-error composition-error
    query-error =
      merkle-error + composition-error + trace-fri-error +
      composition-fri-error + query-error

lemma soundness-bad-event-union-bound:
  fixes s :: ('f, 'a) protocol-channel-scheme
  assumes comp-bound:
    wp-event verify-monad (composition-bad s) s  $\leq$  composition-error-bound
  and trace-fri-bound:
    wp-event verify-monad (trace-fri-bad s) s  $\leq$  trace-fri-error
  and comp-fri-bound:
    wp-event verify-monad (composition-fri-bad s) s  $\leq$  composition-fri-error
  and query-bound:
    wp-event verify-monad (query-bad s) s  $\leq$  nnreal rounds * query-error-bound
  shows wp-event verify-monad (soundness-bad-event s) s  $\leq$  soundness-bound
proof -
  have wp-event verify-monad (soundness-bad-event s) s  $\leq$ 
    wp-event verify-monad (composition-bad s) s +
    wp-event verify-monad (trace-fri-bad s) s +
    wp-event verify-monad (composition-fri-bad s) s +
    wp-event verify-monad (query-bad s) s
  unfolding soundness-bad-event-def
  by (rule wp-event-union-bound4)
  also have ...  $\leq$ 
    composition-error-bound + trace-fri-error + composition-fri-error +
    nnreal rounds * query-error-bound
  by (intro add-mono comp-bound trace-fri-bound comp-fri-bound query-bound)
  also have ... = soundness-bound
  by (simp add: soundness-bound-component-accounting)
  finally show ?thesis .
qed

```

```

lemma soundness-bad-event-table-aware-union-bound:
  fixes s :: ('f, 'a) protocol-channel-scheme
  assumes comp-bound:
    wp-event verify-monad (composition-bad s) s  $\leq$  composition-error-bound
  and trace-fri-bound:

```

$wp\text{-event verify-monad } (trace\text{-fri-bad-with-tables } s) \ s \leq$
 $trace\text{-fri-error}$
and $comp\text{-fri-bound}$:
 $wp\text{-event verify-monad } (composition\text{-fri-bad-with-tables } s) \ s \leq$
 $composition\text{-fri-error}$
and $query\text{-bound}$:
 $wp\text{-event verify-monad } (query\text{-bad } s) \ s \leq nnreal \ rounds * query\text{-error-bound}$
shows
 $wp\text{-event verify-monad } (soundness\text{-bad-event-table-aware } s) \ s \leq$
 $soundness\text{-bound}$
proof –
have $wp\text{-event verify-monad } (soundness\text{-bad-event-table-aware } s) \ s \leq$
 $wp\text{-event verify-monad } (composition\text{-bad } s) \ s +$
 $wp\text{-event verify-monad } (trace\text{-fri-bad-with-tables } s) \ s +$
 $wp\text{-event verify-monad } (composition\text{-fri-bad-with-tables } s) \ s +$
 $wp\text{-event verify-monad } (query\text{-bad } s) \ s$
unfolding $soundness\text{-bad-event-table-aware-def}$
by (rule $wp\text{-event-union-bound}_4$)
also have $\dots \leq$
 $composition\text{-error-bound} + trace\text{-fri-error} + composition\text{-fri-error} +$
 $nnreal \ rounds * query\text{-error-bound}$
by (intro $add\text{-mono comp-bound trace-fri-bound comp-fri-bound query-bound}$)
also have $\dots = soundness\text{-bound}$
by (simp add: $soundness\text{-bound-component-accounting}$)
finally show $?thesis$.
qed

lemma $soundness\text{-bad-event-partial-opening-union-bound}$:
fixes $s :: ('f, 'a) \text{ protocol-channel-scheme}$
assumes $comp\text{-bound}$:
 $wp\text{-event verify-monad } (composition\text{-bad } s) \ s \leq composition\text{-error-bound}$
and $trace\text{-fri-bound}$:
 $wp\text{-event verify-monad } (trace\text{-fri-bad-with-partial-openings } s) \ s \leq$
 $trace\text{-fri-error}$
and $comp\text{-fri-bound}$:
 $wp\text{-event verify-monad}$
 $(composition\text{-fri-bad-with-partial-openings } s) \ s \leq$
 $composition\text{-fri-error}$
and $query\text{-bound}$:
 $wp\text{-event verify-monad } (query\text{-bad } s) \ s \leq nnreal \ rounds * query\text{-error-bound}$
shows
 $wp\text{-event verify-monad } (soundness\text{-bad-event-partial-opening } s) \ s \leq$
 $soundness\text{-bound}$
proof –
have $wp\text{-event verify-monad } (soundness\text{-bad-event-partial-opening } s) \ s \leq$
 $wp\text{-event verify-monad } (composition\text{-bad } s) \ s +$
 $wp\text{-event verify-monad } (trace\text{-fri-bad-with-partial-openings } s) \ s +$
 $wp\text{-event verify-monad}$
 $(composition\text{-fri-bad-with-partial-openings } s) \ s +$

wp-event verify-monad (query-bad s) s
unfolding soundness-bad-event-partial-opening-def
 by (rule wp-event-union-bound4)
also have ... $\leq$
 composition-error-bound + trace-fri-error + composition-fri-error +
 nnreal rounds * query-error-bound
 by (intro add-mono comp-bound trace-fri-bound comp-fri-bound query-bound)
also have ... = soundness-bound
 by (simp add: soundness-bound-component-accounting)
finally show ?thesis .
qed

lemma soundness-bad-event-table-aware-with-merkle-union-bound:

fixes s :: ('f, 'a) protocol-channel-scheme

assumes merkle-bound:

wp-event verify-monad (fri-merkle-binding-bad s) s $\leq$
 merkle-binding-error

and comp-bound:

wp-event verify-monad (composition-bad s) s $\leq$
 composition-error-bound

and trace-fri-bound:

wp-event verify-monad (trace-fri-bad-with-tables s) s $\leq$
 trace-fri-error

and comp-fri-bound:

wp-event verify-monad (composition-fri-bad-with-tables s) s $\leq$
 composition-fri-error

and query-bound:

wp-event verify-monad (query-bad s) s $\leq$
 nnreal rounds * query-error-bound

shows

wp-event verify-monad
 (soundness-bad-event-table-aware-with-merkle s) s $\leq$
 soundness-bound-with-merkle

proof –

have table-bound:

wp-event verify-monad (soundness-bad-event-table-aware s) s $\leq$
 soundness-bound

by (rule soundness-bad-event-table-aware-union-bound
 [OF comp-bound trace-fri-bound comp-fri-bound query-bound])

have wp-event verify-monad

(soundness-bad-event-table-aware-with-merkle s) s $\leq$
 wp-event verify-monad (fri-merkle-binding-bad s) s +
 wp-event verify-monad (soundness-bad-event-table-aware s) s

unfolding soundness-bad-event-table-aware-with-merkle-def

by (rule wp-event-union-bound)

also have ... $\leq$ merkle-binding-error + soundness-bound

by (intro add-mono merkle-bound table-bound)

also have ... = soundness-bound-with-merkle

unfolding soundness-bound-with-merkle-def **by** (simp add: algebra-simps)

finally show ?thesis .
qed

lemma *soundness-bad-event-partial-opening-with-merkle-union-bound:*

fixes $s :: ('f, 'a) \text{ protocol-channel-scheme}$

assumes *merkle-bound:*

$\text{wp-event verify-monad (fri-merkle-binding-bad } s) s \leq$
 $\text{merkle-binding-error}$

and *comp-bound:*

$\text{wp-event verify-monad (composition-bad } s) s \leq$
 $\text{composition-error-bound}$

and *trace-fri-bound:*

$\text{wp-event verify-monad (trace-fri-bad-with-partial-openings } s) s \leq$
 trace-fri-error

and *comp-fri-bound:*

$\text{wp-event verify-monad}$
 $\text{(composition-fri-bad-with-partial-openings } s) s \leq$
 $\text{composition-fri-error}$

and *query-bound:*

$\text{wp-event verify-monad (query-bad } s) s \leq$
 $\text{nnreal rounds * query-error-bound}$

shows

$\text{wp-event verify-monad}$
 $\text{(soundness-bad-event-partial-opening-with-merkle } s) s \leq$
 $\text{soundness-bound-with-merkle}$

proof —

have *partial-bound:*

$\text{wp-event verify-monad (soundness-bad-event-partial-opening } s) s \leq$
 soundness-bound

by (rule *soundness-bad-event-partial-opening-union-bound*
[OF *comp-bound trace-fri-bound comp-fri-bound query-bound*])

have *wp-event verify-monad*

$\text{(soundness-bad-event-partial-opening-with-merkle } s) s \leq$
 $\text{wp-event verify-monad (fri-merkle-binding-bad } s) s +$
 $\text{wp-event verify-monad (soundness-bad-event-partial-opening } s) s$

unfolding *soundness-bad-event-partial-opening-with-merkle-def*

by (rule *wp-event-union-bound*)

also have $\dots \leq \text{merkle-binding-error} + \text{soundness-bound}$

by (intro *add-mono merkle-bound partial-bound*)

also have $\dots = \text{soundness-bound-with-merkle}$

unfolding *soundness-bound-with-merkle-def* **by** (*simp add: algebra-simps*)

finally show ?thesis .

qed

lemma *soundness-bad-event-partial-opening-with-initial-merkle-union-bound:*

fixes $s :: ('f, 'a) \text{ protocol-channel-scheme}$

assumes *merkle-bound:*

$\text{wp-event verify-monad (initial-merkle-binding-bad } s) s \leq$
 $\text{merkle-binding-error}$

and *comp-bound*:
 $wp\text{-event verify-monad } (composition\text{-bad } s) \ s \leq$
 $composition\text{-error-bound}$
and *trace-fri-bound*:
 $wp\text{-event verify-monad } (trace\text{-fri-bad-with-partial-openings } s) \ s \leq$
 $trace\text{-fri-error}$
and *comp-fri-bound*:
 $wp\text{-event verify-monad}$
 $(composition\text{-fri-bad-with-partial-openings } s) \ s \leq$
 $composition\text{-fri-error}$
and *query-bound*:
 $wp\text{-event verify-monad } (query\text{-bad } s) \ s \leq$
 $nnreal \ rounds * query\text{-error-bound}$
shows
 $wp\text{-event verify-monad}$
 $(soundness\text{-bad-event-partial-opening-with-initial-merkle } s) \ s \leq$
 $soundness\text{-bound-with-merkle}$
proof –
have *partial-bound*:
 $wp\text{-event verify-monad } (soundness\text{-bad-event-partial-opening } s) \ s \leq$
 $soundness\text{-bound}$
by (rule *soundness-bad-event-partial-opening-union-bound*
 $[OF \ comp\text{-bound} \ trace\text{-fri-bound} \ comp\text{-fri-bound} \ query\text{-bound}]$)
have *wp-event verify-monad*
 $(soundness\text{-bad-event-partial-opening-with-initial-merkle } s) \ s \leq$
 $wp\text{-event verify-monad } (initial\text{-merkle-binding-bad } s) \ s +$
 $wp\text{-event verify-monad } (soundness\text{-bad-event-partial-opening } s) \ s$
unfolding *soundness-bad-event-partial-opening-with-initial-merkle-def*
by (rule *wp-event-union-bound*)
also have $\dots \leq merkle\text{-binding-error} + soundness\text{-bound}$
by (intro *add-mono merkle-bound partial-bound*)
also have $\dots = soundness\text{-bound-with-merkle}$
unfolding *soundness-bound-with-merkle-def* **by** (*simp add: algebra-simps*)
finally show *?thesis* .
qed

lemma *soundness-bad-event-partial-opening-with-partial-merkle-union-bound*:
fixes $s :: ('f, 'a) \text{ protocol-channel-scheme}$
assumes *merkle-bound*:
 $wp\text{-event verify-monad } (partial\text{-merkle-inconsistency-bad } s) \ s \leq$
 $merkle\text{-binding-error}$
and *comp-bound*:
 $wp\text{-event verify-monad } (composition\text{-bad } s) \ s \leq$
 $composition\text{-error-bound}$
and *trace-fri-bound*:
 $wp\text{-event verify-monad } (trace\text{-fri-bad-with-partial-openings } s) \ s \leq$
 $trace\text{-fri-error}$
and *comp-fri-bound*:
 $wp\text{-event verify-monad}$

$(\text{composition-fri-bad-with-partial-openings } s) \ s \leq$
 $\text{composition-fri-error}$
and query-bound:
 $\text{wp-event verify-monad } (\text{query-bad } s) \ s \leq$
 $\text{nnreal rounds} * \text{query-error-bound}$
shows
 $\text{wp-event verify-monad}$
 $(\text{soundness-bad-event-partial-opening-with-partial-merkle } s) \ s \leq$
 $\text{soundness-bound-with-merkle}$
proof –
have partial-bound:
 $\text{wp-event verify-monad } (\text{soundness-bad-event-partial-opening } s) \ s \leq$
 soundness-bound
by (rule $\text{soundness-bad-event-partial-opening-union-bound}$
 $[\text{OF comp-bound trace-fri-bound comp-fri-bound query-bound}]$)
have $\text{wp-event verify-monad}$
 $(\text{soundness-bad-event-partial-opening-with-partial-merkle } s) \ s \leq$
 $\text{wp-event verify-monad } (\text{partial-merkle-inconsistency-bad } s) \ s +$
 $\text{wp-event verify-monad } (\text{soundness-bad-event-partial-opening } s) \ s$
unfolding $\text{soundness-bad-event-partial-opening-with-partial-merkle-def}$
by (rule $\text{wp-event-union-bound}$)
also have $\dots \leq \text{merkle-binding-error} + \text{soundness-bound}$
by (intro $\text{add-mono merkle-bound partial-bound}$)
also have $\dots = \text{soundness-bound-with-merkle}$
unfolding $\text{soundness-bound-with-merkle-def}$ **by** (simp $\text{add: algebra-simps}$)
finally show $?thesis$.
qed

lemma $\text{soundness-bad-event-partial-candidate-union-bound:}$
fixes $s :: ('f, 'a) \text{ protocol-channel-scheme}$
assumes comp-bound:
 $\text{wp-event verify-monad}$
 $(\text{composition-bad-with-partial-candidates } s) \ s \leq$
 $\text{composition-error-bound}$
and trace-fri-bound:
 $\text{wp-event verify-monad}$
 $(\text{trace-fri-bad-with-partial-candidates } s) \ s \leq$
 trace-fri-error
and comp-fri-bound:
 $\text{wp-event verify-monad}$
 $(\text{composition-fri-bad-with-partial-candidates } s) \ s \leq$
 $\text{composition-fri-error}$
and query-bound:
 $\text{wp-event verify-monad}$
 $(\text{query-bad-with-partial-candidates } s) \ s \leq$
 $\text{nnreal rounds} * \text{query-error-bound}$
shows
 $\text{wp-event verify-monad } (\text{soundness-bad-event-partial-candidate } s) \ s \leq$
 soundness-bound

proof –

have $wp\text{-event}\ verify\text{-monad}\ (soundness\text{-bad}\text{-event}\text{-partial}\text{-candidate}\ s)\ s \leq$
 $wp\text{-event}\ verify\text{-monad}$
 $(composition\text{-bad}\text{-with}\text{-partial}\text{-candidates}\ s)\ s +$
 $wp\text{-event}\ verify\text{-monad}$
 $(trace\text{-fri}\text{-bad}\text{-with}\text{-partial}\text{-candidates}\ s)\ s +$
 $wp\text{-event}\ verify\text{-monad}$
 $(composition\text{-fri}\text{-bad}\text{-with}\text{-partial}\text{-candidates}\ s)\ s +$
 $wp\text{-event}\ verify\text{-monad}\ (query\text{-bad}\text{-with}\text{-partial}\text{-candidates}\ s)\ s$
unfolding $soundness\text{-bad}\text{-event}\text{-partial}\text{-candidate}\text{-def}$
by $(rule\ wp\text{-event}\text{-union}\text{-bound}_4)$
also have $\dots \leq composition\text{-error}\text{-bound} + trace\text{-fri}\text{-error} +$
 $composition\text{-fri}\text{-error} + nnreal\ rounds * query\text{-error}\text{-bound}$
by $(intro\ add\text{-mono}\ comp\text{-bound}\ trace\text{-fri}\text{-bound}\ comp\text{-fri}\text{-bound}$
 $query\text{-bound})$
also have $\dots = soundness\text{-bound}$
unfolding $soundness\text{-bound}\text{-def}$ **by** $(simp\ add:\ algebra\text{-simps})$
finally show $?thesis$.

qed

lemma $soundness\text{-bad}\text{-event}\text{-partial}\text{-candidate}\text{-with}\text{-partial}\text{-merkle}\text{-union}\text{-bound}$:

fixes $s :: ('f, 'a)\ protocol\text{-channel}\text{-scheme}$

assumes $merkle\text{-bound}$:

$wp\text{-event}\ verify\text{-monad}\ (partial\text{-merkle}\text{-inconsistency}\text{-bad}\ s)\ s \leq$
 $merkle\text{-binding}\text{-error}$

and $comp\text{-bound}$:

$wp\text{-event}\ verify\text{-monad}$
 $(composition\text{-bad}\text{-with}\text{-partial}\text{-candidates}\ s)\ s \leq$
 $composition\text{-error}\text{-bound}$

and $trace\text{-fri}\text{-bound}$:

$wp\text{-event}\ verify\text{-monad}$
 $(trace\text{-fri}\text{-bad}\text{-with}\text{-partial}\text{-candidates}\ s)\ s \leq$
 $trace\text{-fri}\text{-error}$

and $comp\text{-fri}\text{-bound}$:

$wp\text{-event}\ verify\text{-monad}$
 $(composition\text{-fri}\text{-bad}\text{-with}\text{-partial}\text{-candidates}\ s)\ s \leq$
 $composition\text{-fri}\text{-error}$

and $query\text{-bound}$:

$wp\text{-event}\ verify\text{-monad}$
 $(query\text{-bad}\text{-with}\text{-partial}\text{-candidates}\ s)\ s \leq$
 $nnreal\ rounds * query\text{-error}\text{-bound}$

shows

$wp\text{-event}\ verify\text{-monad}$
 $(soundness\text{-bad}\text{-event}\text{-partial}\text{-candidate}\text{-with}\text{-partial}\text{-merkle}\ s)\ s \leq$
 $soundness\text{-bound}\text{-with}\text{-merkle}$

proof –

have $partial\text{-bound}$:

$wp\text{-event}\ verify\text{-monad}\ (soundness\text{-bad}\text{-event}\text{-partial}\text{-candidate}\ s)\ s \leq$
 $soundness\text{-bound}$

by (rule soundness-bad-event-partial-candidate-union-bound
 [OF comp-bound trace-fri-bound comp-fri-bound query-bound])
 have wp-event verify-monad
 (soundness-bad-event-partial-candidate-with-partial-merkle s) s ≤
 wp-event verify-monad (partial-merkle-inconsistency-bad s) s +
 wp-event verify-monad (soundness-bad-event-partial-candidate s) s
 unfolding soundness-bad-event-partial-candidate-with-partial-merkle-def
 by (rule wp-event-union-bound)
 also have ... ≤ merkle-binding-error + soundness-bound
 by (intro add-mono merkle-bound partial-bound)
 also have ... = soundness-bound-with-merkle
 unfolding soundness-bound-with-merkle-def by (simp add: algebra-simps)
 finally show ?thesis .
 qed

lemma checked-staged-security-with-data-state-partial-candidate-with-partial-merkle-union-bound:
fixes merkle-error composition-error trace-fri-error'
 composition-fri-error' query-error :: prob
assumes merkle-bound:
 wp-event (checked-staged-security-experiment-with-data-state A)
 (staged-security-with-data-state-verifier-event
 partial-merkle-inconsistency-bad)
 adversary-initial-state ≤ merkle-error
and comp-bound:
 wp-event (checked-staged-security-experiment-with-data-state A)
 (staged-security-with-data-state-verifier-event
 composition-bad-with-partial-candidates)
 adversary-initial-state ≤ composition-error
and trace-fri-bound:
 wp-event (checked-staged-security-experiment-with-data-state A)
 (staged-security-with-data-state-verifier-event
 trace-fri-bad-with-partial-candidates)
 adversary-initial-state ≤ trace-fri-error'
and comp-fri-bound:
 wp-event (checked-staged-security-experiment-with-data-state A)
 (staged-security-with-data-state-verifier-event
 composition-fri-bad-with-partial-candidates)
 adversary-initial-state ≤ composition-fri-error'
and query-bound:
 wp-event (checked-staged-security-experiment-with-data-state A)
 (staged-security-with-data-state-verifier-event
 query-bad-with-partial-candidates)
 adversary-initial-state ≤ query-error
shows
 wp-event (checked-staged-security-experiment-with-data-state A)
 (staged-security-with-data-state-verifier-event
 soundness-bad-event-partial-candidate-with-partial-merkle)
 adversary-initial-state ≤
 merkle-error + composition-error + trace-fri-error' +

```

    composition-fri-error' + query-error
proof –
  let ?M = checked-staged-security-experiment-with-data-state A
  let ?E = staged-security-with-data-state-verifier-event
  let ?merkle = ?E partial-merkle-inconsistency-bad
  let ?comp = ?E composition-bad-with-partial-candidates
  let ?trace = ?E trace-fri-bad-with-partial-candidates
  let ?comp-fri = ?E composition-fri-bad-with-partial-candidates
  let ?query = ?E query-bad-with-partial-candidates
  let ?partial = ?E soundness-bad-event-partial-candidate
  let ?with-merkle =
    ?E soundness-bad-event-partial-candidate-with-partial-merkle
have partial-bound:
  wp-event ?M ?partial adversary-initial-state ≤
    composition-error + trace-fri-error' + composition-fri-error' +
    query-error
proof –
  have wp-event ?M ?partial adversary-initial-state ≤
    wp-event ?M
      (λout. ?comp out ∨ ?trace out ∨ ?comp-fri out ∨
        ?query out)
      adversary-initial-state
  by (rule wp-event-mono)
    (auto simp: staged-security-with-data-state-verifier-event-def
      soundness-bad-event-partial-candidate-def
      split: option.splits prod.splits)
  also have ... ≤
    wp-event ?M ?comp adversary-initial-state +
    wp-event ?M ?trace adversary-initial-state +
    wp-event ?M ?comp-fri adversary-initial-state +
    wp-event ?M ?query adversary-initial-state
  by (rule wp-event-union-bound4)
  also have ... ≤
    composition-error + trace-fri-error' + composition-fri-error' +
    query-error
  by (intro add-mono comp-bound trace-fri-bound comp-fri-bound
    query-bound)
  finally show ?thesis .
qed
have wp-event ?M ?with-merkle adversary-initial-state ≤
  wp-event ?M (λout. ?merkle out ∨ ?partial out)
    adversary-initial-state
  by (rule wp-event-mono)
    (auto simp: staged-security-with-data-state-verifier-event-def
      soundness-bad-event-partial-candidate-with-partial-merkle-def
      split: option.splits prod.splits)
  also have ... ≤
    wp-event ?M ?merkle adversary-initial-state +
    wp-event ?M ?partial adversary-initial-state

```

```

    by (rule wp-event-union-bound)
  also have ... ≤
    merkle-error +
    (composition-error + trace-fri-error' + composition-fri-error' +
     query-error)
  by (intro add-mono merkle-bound partial-bound)
  finally show ?thesis
    by (simp add: algebra-simps)
qed

lemma soundness-bad-event-partial-candidate-empty-header-union-bound:
  fixes s :: ('f, 'a) protocol-channel-scheme
  assumes comp-bound:
    wp-event verify-monad
    (composition-bad-with-empty-composition-header-candidates s) s ≤
    composition-error-bound
  and trace-fri-bound:
    wp-event verify-monad
    (trace-fri-bad-with-empty-composition-header-candidates s) s ≤
    trace-fri-error
  and query-bound:
    wp-event verify-monad
    (query-bad-with-empty-composition-header-candidates s) s ≤
    nnreal rounds * query-error-bound
  shows
    wp-event verify-monad
    (soundness-bad-event-partial-candidate-empty-header s) s ≤
    composition-error-bound + trace-fri-error +
    nnreal rounds * query-error-bound
proof -
  have wp-event verify-monad
    (soundness-bad-event-partial-candidate-empty-header s) s ≤
    wp-event verify-monad
    (λout.
      composition-bad-with-empty-composition-header-candidates s out ∨
      trace-fri-bad-with-empty-composition-header-candidates s out ∨
      query-bad-with-empty-composition-header-candidates s out ∨
      False) s
  unfolding soundness-bad-event-partial-candidate-empty-header-def
  by simp
  also have ... ≤
    wp-event verify-monad
    (composition-bad-with-empty-composition-header-candidates s) s +
    wp-event verify-monad
    (trace-fri-bad-with-empty-composition-header-candidates s) s +
    wp-event verify-monad
    (query-bad-with-empty-composition-header-candidates s) s +
    wp-event verify-monad (λ-. False) s
  by (rule wp-event-union-bound4)

```

also have ... =

$$\begin{aligned} & \text{wp-event verify-monad} \\ & \quad (\text{composition-bad-with-empty-composition-header-candidates } s) \ s + \\ & \text{wp-event verify-monad} \\ & \quad (\text{trace-fri-bad-with-empty-composition-header-candidates } s) \ s + \\ & \text{wp-event verify-monad} \\ & \quad (\text{query-bad-with-empty-composition-header-candidates } s) \ s \\ & \text{unfolding wp-event-def wp-def dist-expect-def by simp} \end{aligned}$$
also have ... $\leq$

$$\begin{aligned} & \text{composition-error-bound} + \text{trace-fri-error} + \\ & \quad \text{nnreal rounds} * \text{query-error-bound} \\ & \text{by (intro add-mono comp-bound trace-fri-bound query-bound)} \end{aligned}$$
finally show ?thesis .
qed

lemma composition-bad-with-empty-composition-header-candidates-union-bound:
fixes $s :: ('f, 'a) \text{ protocol-channel-scheme}$
assumes false-statement: $\neg \text{exists-valid-trace}$
and degree-bound:

$$\begin{aligned} & \text{wp-event verify-monad} \\ & \quad (\text{composition-degree-bad-with-empty-composition-header-candidates } s) \ s \\ & \leq \text{degree-error} \end{aligned}$$
and randomization-bound:

$$\begin{aligned} & \text{wp-event verify-monad} \\ & \quad (\text{composition-randomization-bad-with-empty-composition-header-candidates} \\ & \quad \quad s) \ s \leq \text{randomization-error} \end{aligned}$$
shows

$$\begin{aligned} & \text{wp-event verify-monad} \\ & \quad (\text{composition-bad-with-empty-composition-header-candidates } s) \ s \leq \\ & \quad \text{degree-error} + \text{randomization-error} \end{aligned}$$
proof –
have wp-event verify-monad

$$\begin{aligned} & \quad (\text{composition-bad-with-empty-composition-header-candidates } s) \ s \leq \\ & \text{wp-event verify-monad} \\ & \quad (\lambda \text{out.} \\ & \quad \quad \text{composition-degree-bad-with-empty-composition-header-candidates} \\ & \quad \quad \quad s \ \text{out} \ \vee \\ & \quad \quad \text{composition-randomization-bad-with-empty-composition-header-candidates} \\ & \quad \quad \quad s \ \text{out}) \ s \end{aligned}$$
by (rule wp-event-mono)
(rule composition-bad-with-empty-composition-header-candidates-split
[OF false-statement])
also have ... $\leq$

$$\begin{aligned} & \text{wp-event verify-monad} \\ & \quad (\text{composition-degree-bad-with-empty-composition-header-candidates } s) \ s + \\ & \text{wp-event verify-monad} \\ & \quad (\text{composition-randomization-bad-with-empty-composition-header-candidates} \\ & \quad \quad s) \ s \end{aligned}$$
by (rule wp-event-union-bound)

also have $\dots \leq \text{degree-error} + \text{randomization-error}$
 by (intro add-mono degree-bound randomization-bound)
 finally show ?thesis .
 qed

lemma *soundness-bad-event-partial-candidate-empty-header-split-union-bound*:
 fixes $s :: ('f, 'a) \text{ protocol-channel-scheme}$
 assumes *false-statement*: $\neg \text{exists-valid-trace}$
 and *degree-bound*:
 $\text{wp-event verify-monad}$
 $(\text{composition-degree-bad-with-empty-composition-header-candidates } s) \ s \leq \text{degree-error}$
 and *randomization-bound*:
 $\text{wp-event verify-monad}$
 $(\text{composition-randomization-bad-with-empty-composition-header-candidates } s) \ s \leq \text{randomization-error}$
 and *trace-fri-bound*:
 $\text{wp-event verify-monad}$
 $(\text{trace-fri-bad-with-empty-composition-header-candidates } s) \ s \leq \text{trace-fri-error}$
 and *query-bound*:
 $\text{wp-event verify-monad}$
 $(\text{query-bad-with-empty-composition-header-candidates } s) \ s \leq \text{nnreal rounds} * \text{query-error-bound}$
 shows
 $\text{wp-event verify-monad}$
 $(\text{soundness-bad-event-partial-candidate-empty-header } s) \ s \leq \text{degree-error} + \text{randomization-error} + \text{trace-fri-error} + \text{nnreal rounds} * \text{query-error-bound}$

proof –
 have $\text{wp-event verify-monad}$
 $(\text{soundness-bad-event-partial-candidate-empty-header } s) \ s \leq$
 $\text{wp-event verify-monad}$
 $(\lambda \text{out.}$
 $\text{composition-degree-bad-with-empty-composition-header-candidates } s \ \text{out} \vee$
 $\text{composition-randomization-bad-with-empty-composition-header-candidates } s \ \text{out} \vee$
 $\text{trace-fri-bad-with-empty-composition-header-candidates } s \ \text{out} \vee$
 $\text{query-bad-with-empty-composition-header-candidates } s \ \text{out}) \ s$

proof (rule *wp-event-mono*)
 fix *out*
 assume *empty*:
 $\text{soundness-bad-event-partial-candidate-empty-header } s \ \text{out}$
 then consider
 $(\text{comp}) \ \text{composition-bad-with-empty-composition-header-candidates } s \ \text{out}$
 | $(\text{trace}) \ \text{trace-fri-bad-with-empty-composition-header-candidates } s \ \text{out}$
 | $(\text{query}) \ \text{query-bad-with-empty-composition-header-candidates } s \ \text{out}$
 unfolding *soundness-bad-event-partial-candidate-empty-header-def*

```

    by blast
  then show
    composition-degree-bad-with-empty-composition-header-candidates
      s out  $\vee$ 
    composition-randomization-bad-with-empty-composition-header-candidates
      s out  $\vee$ 
    trace-fri-bad-with-empty-composition-header-candidates s out  $\vee$ 
    query-bad-with-empty-composition-header-candidates s out
  proof cases
    case comp
    then show ?thesis
      using composition-bad-with-empty-composition-header-candidates-split
        [OF false-statement comp]
      by blast
  qed blast+
qed
also have ...  $\leq$ 
  wp-event verify-monad
    (composition-degree-bad-with-empty-composition-header-candidates s) s +
  wp-event verify-monad
    (composition-randomization-bad-with-empty-composition-header-candidates
      s) s +
  wp-event verify-monad
    (trace-fri-bad-with-empty-composition-header-candidates s) s +
  wp-event verify-monad
    (query-bad-with-empty-composition-header-candidates s) s
  by (rule wp-event-union-bound4)
also have ...  $\leq$ 
  degree-error + randomization-error + trace-fri-error +
  nnreal rounds * query-error-bound
  by (intro add-mono degree-bound randomization-bound trace-fri-bound
    query-bound)
finally show ?thesis .
qed

lemma soundness-bad-event-partial-candidate-empty-header-reduced-union-bound:
  fixes s :: ('f, 'a) protocol-channel-scheme
  assumes false-statement:  $\neg$  exists-valid-trace
  and randomization-bound:
    wp-event verify-monad
      (composition-randomization-bad-with-empty-composition-header-candidates
        s) s  $\leq$  composition-error-bound
  and trace-fri-bound:
    wp-event verify-monad
      (trace-fri-bad-with-empty-composition-header-candidates s) s  $\leq$ 
      trace-fri-error
  and query-bound:
    wp-event verify-monad
      (query-bad-with-empty-composition-header-candidates s) s  $\leq$ 

```

$nnreal\ rounds * query\ error\ bound$
shows
 $wp\ event\ verify\ monad$
 $(soundness\ bad\ event\ partial\ candidate\ empty\ header\ s)\ s \leq$
 $composition\ error\ bound + trace\ fri\ error +$
 $nnreal\ rounds * query\ error\ bound$
proof –
have $split\ bound$:
 $wp\ event\ verify\ monad$
 $(soundness\ bad\ event\ partial\ candidate\ empty\ header\ s)\ s \leq$
 $0 + composition\ error\ bound + trace\ fri\ error +$
 $nnreal\ rounds * query\ error\ bound$
by (rule $soundness\ bad\ event\ partial\ candidate\ empty\ header\ split\ union\ bound$
 $[OF\ false\ statement$
 $composition\ degree\ bad\ with\ empty\ composition\ header\ candidates\ bound$
 $randomization\ bound\ trace\ fri\ bound\ query\ bound]$)
then show $?thesis$ **by** $simp$
qed

lemma $soundness\ bad\ event\ partial\ candidate\ empty\ header\ bound\ from\ alpha\ hit$:
fixes $s :: ('f, 'a)\ protocol\ channel\ scheme$
assumes $false\ statement: \neg\ exists\ valid\ trace$
and $alpha\ bound$:
 $wp\ event\ verify\ monad$
 $(composition\ alpha\ bad\ set\ hit\ with\ empty\ composition\ header\ candidates$
 $s)\ s \leq composition\ error\ bound$
and $trace\ fri\ bound$:
 $wp\ event\ verify\ monad$
 $(trace\ fri\ bad\ with\ empty\ composition\ header\ candidates\ s)\ s \leq$
 $trace\ fri\ error$
and $query\ bound$:
 $wp\ event\ verify\ monad$
 $(query\ bad\ with\ empty\ composition\ header\ candidates\ s)\ s \leq$
 $nnreal\ rounds * query\ error\ bound$
shows
 $wp\ event\ verify\ monad$
 $(soundness\ bad\ event\ partial\ candidate\ empty\ header\ s)\ s \leq$
 $composition\ error\ bound + trace\ fri\ error +$
 $nnreal\ rounds * query\ error\ bound$
proof –
have $randomization\ bound$:
 $wp\ event\ verify\ monad$
 $(composition\ randomization\ bad\ with\ empty\ composition\ header\ candidates$
 $s)\ s \leq composition\ error\ bound$
proof –
have $wp\ event\ verify\ monad$
 $(composition\ randomization\ bad\ with\ empty\ composition\ header\ candidates$
 $s)\ s \leq$
 $wp\ event\ verify\ monad$

```

      (composition-alpha-bad-set-hit-with-empty-composition-header-candidates
       s) s
    by (rule
        wp-composition-randomization-bad-with-empty-composition-header-candidates-le-alpha-hit)
  also have ... ≤ composition-error-bound
    by (rule alpha-bound)
  finally show ?thesis .
qed
show ?thesis
  by (rule soundness-bad-event-partial-candidate-empty-header-reduced-union-bound
      [OF false-statement randomization-bound trace-fri-bound query-bound])
qed

lemma soundness-bad-event-partial-candidate-empty-header-bound-from-partial-opening-alpha-union-hit:
  fixes s :: ('f, 'a) protocol-channel-scheme
  assumes false-statement: ¬ exists-valid-trace
  and alpha-bound:
    wp-event verify-monad
    (composition-alpha-partial-opening-union-hit-with-empty-header s) s ≤
    composition-error-bound
  and trace-fri-bound:
    wp-event verify-monad
    (trace-fri-bad-with-empty-composition-header-candidates s) s ≤
    trace-fri-error
  and query-bound:
    wp-event verify-monad
    (query-bad-with-empty-composition-header-candidates s) s ≤
    nnreal rounds * query-error-bound
  shows
    wp-event verify-monad
    (soundness-bad-event-partial-candidate-empty-header s) s ≤
    composition-error-bound + trace-fri-error +
    nnreal rounds * query-error-bound
proof -
  have alpha-hit-bound:
    wp-event verify-monad
    (composition-alpha-bad-set-hit-with-empty-composition-header-candidates
     s) s ≤ composition-error-bound
  proof -
    have wp-event verify-monad
      (composition-alpha-bad-set-hit-with-empty-composition-header-candidates
       s) s ≤
      wp-event verify-monad
      (composition-alpha-partial-opening-union-hit-with-empty-header s) s
    by (rule
        wp-composition-alpha-bad-set-hit-with-empty-header-le-partial-opening-union-hit)
    also have ... ≤ composition-error-bound
      by (rule alpha-bound)
    finally show ?thesis .
  qed

```

```

qed
show ?thesis
  by (rule soundness-bad-event-partial-candidate-empty-header-bound-from-alpha-hit
      [OF false-statement alpha-hit-bound trace-fri-bound query-bound])
qed

lemma soundness-bad-event-partial-candidate-empty-header-bound-from-alpha-header-partial-union-hit:
  fixes s :: ('f, 'a) protocol-channel-scheme
  assumes false-statement:  $\neg$  exists-valid-trace
  and alpha-bound:
    wp-event verify-monad
      (alpha-header-list-set-hit s
        (alpha-header-supported-partial-union-bad-sets s
          composition-trace-bad-alpha-space)) s  $\leq$ 
      composition-error-bound
  and trace-fri-bound:
    wp-event verify-monad
      (trace-fri-bad-with-empty-composition-header-candidates s) s  $\leq$ 
      trace-fri-error
  and query-bound:
    wp-event verify-monad
      (query-bad-with-empty-composition-header-candidates s) s  $\leq$ 
      nnreal rounds * query-error-bound
  shows
    wp-event verify-monad
      (soundness-bad-event-partial-candidate-empty-header s) s  $\leq$ 
      composition-error-bound + trace-fri-error +
      nnreal rounds * query-error-bound
proof -
  have partial-alpha-bound:
    wp-event verify-monad
      (composition-alpha-partial-opening-union-hit-with-empty-header s) s  $\leq$ 
      composition-error-bound
  proof -
    have wp-event verify-monad
      (composition-alpha-partial-opening-union-hit-with-empty-header s) s
       $\leq$ 
      wp-event verify-monad
        (alpha-header-list-set-hit s
          (alpha-header-supported-partial-union-bad-sets s
            composition-trace-bad-alpha-space)) s
      by (rule
        wp-composition-alpha-partial-opening-union-hit-with-empty-header-le-alpha-header-partial-union)
    also have ...  $\leq$  composition-error-bound
      by (rule alpha-bound)
    finally show ?thesis .
  qed
show ?thesis
  by (rule

```

soundness-bad-event-partial-candidate-empty-header-bound-from-partial-opening-alpha-union-hit
[OF false-statement partial-alpha-bound trace-fri-bound query-bound])

qed

lemma *checked-staged-security-with-actual-alpha-prefix-partial-candidate-randomization-imp-partial-header-bad-set-h*
assumes *support*:
 out ∈
 set-dist
 (*execute*
 (*checked-staged-security-experiment-with-actual-alpha-prefix-data-state*
 A)
 adversary-initial-state)
and *hit*:
 checked-staged-security-with-actual-alpha-prefix-verifier-event
 composition-randomization-bad-with-partial-candidates out
shows
 checked-staged-security-with-actual-alpha-prefix-partial-header-bad-set-hit
 composition-trace-bad-alpha-space out
proof (*cases out*)
case *None*
then show *?thesis*
 using *hit*
 unfolding *checked-staged-security-with-actual-alpha-prefix-verifier-event-def*
 by *simp*
next
case (*Some result-pack*)
then obtain *prefix prefix-state data attacker-state result final-state*
where *out-eq*:
 out =
 Some (((((prefix, prefix-state), data), attacker-state),
 result), final-state)
 by (*cases result-pack, auto split: prod.splits*)
let *?s* =
 verifier-state-from-adversary attacker-state
 (*staged-proof-transcript data*)
have *verifier*:
 Some (result, final-state) ∈ set-dist (execute verify-monad ?s)
 using *support*
 unfolding *out-eq*
 checked-staged-security-experiment-with-actual-alpha-prefix-data-state-def
 by (*auto elim!: set-dist-bindE split: prod.splits*)
have *local-random*:
 composition-randomization-bad-with-partial-candidates ?s
 (*Some (result, final-state)*)
 using *hit*
 unfolding *out-eq*
 checked-staged-security-with-actual-alpha-prefix-verifier-event-def
 Let-def
 by *simp*

have *local-hit*:
composition-alpha-bad-set-hit-with-partial-candidates ?s
(Some (result, final-state))
by (rule *composition-randomization-bad-with-partial-candidates-imp-alpha-hit*
[OF local-random])
have *partial-header*:
alpha-header-list-set-hit ?s
(alpha-header-supported-partial-union-bad-sets ?s
composition-trace-bad-alpha-space)
(Some (result, final-state))
by (rule
composition-alpha-bad-set-hit-with-partial-candidates-imp-alpha-header-partial-union-on-support
[OF verifier local-hit])
show *?thesis*
unfolding *out-eq*
checked-staged-security-with-actual-alpha-prefix-partial-header-bad-set-hit-def
Let-def
using *partial-header* **by** *simp*
qed

lemma *checked-staged-security-with-actual-alpha-prefix-partial-candidate-randomization-le-partial-header-bad-set-hit*:
wp-event
(checked-staged-security-experiment-with-actual-alpha-prefix-data-state A)
(checked-staged-security-with-actual-alpha-prefix-verifier-event
composition-randomization-bad-with-partial-candidates)
adversary-initial-state ≤
wp-event
(checked-staged-security-experiment-with-actual-alpha-prefix-data-state A)
(checked-staged-security-with-actual-alpha-prefix-partial-header-bad-set-hit
composition-trace-bad-alpha-space)
adversary-initial-state
by (rule *wp-event-mono-on-support*)
(rule
checked-staged-security-with-actual-alpha-prefix-partial-candidate-randomization-imp-partial-header-bad-set-hit-on

lemma *checked-staged-security-with-actual-alpha-prefix-partial-candidate-composition-randomization-bound-from-pre*:
assumes *wf*: *staged-budget-wellformed budgets*
and *controlled*: *staged-adversary-controlled budgets A*
and *prefix-bound*:
wp-event
(checked-staged-security-experiment-with-actual-alpha-prefix-data-state
A)
(checked-staged-security-with-actual-alpha-prefix-bad-set-hit
composition-trace-bad-alpha-space)
adversary-initial-state ≤ P
and *drift-bound*:
wp-event
(checked-staged-security-experiment-with-actual-alpha-prefix-data-state
A)

(checked-staged-security-with-actual-alpha-prefix-partial-header-candidate-drift
 composition-trace-bad-alpha-space)
 adversary-initial-state $\leq D$

shows

wp-event
 (checked-staged-security-experiment-with-actual-alpha-prefix-data-state A)
 (checked-staged-security-with-actual-alpha-prefix-verifier-event
 composition-randomization-bad-with-partial-candidates)
 adversary-initial-state $\leq$
 $P + D +$
 staged-phase-relation-error size
 (staged-alpha-search-queries budgets 0)

proof –

have partial-header-bound:

wp-event
 (checked-staged-security-experiment-with-actual-alpha-prefix-data-state A)
 (checked-staged-security-with-actual-alpha-prefix-partial-header-bad-set-hit
 composition-trace-bad-alpha-space)
 adversary-initial-state $\leq$
 $P + D +$
 staged-phase-relation-error size
 (staged-alpha-search-queries budgets 0)

by (rule
 checked-staged-security-with-actual-alpha-prefix-partial-header-bad-set-hit-bound-from-prefix-drift-and-budgets
 [OF wf controlled prefix-bound drift-bound])

have wp-event

(checked-staged-security-experiment-with-actual-alpha-prefix-data-state A)
 (checked-staged-security-with-actual-alpha-prefix-verifier-event
 composition-randomization-bad-with-partial-candidates)
 adversary-initial-state $\leq$
 wp-event
 (checked-staged-security-experiment-with-actual-alpha-prefix-data-state A)
 (checked-staged-security-with-actual-alpha-prefix-partial-header-bad-set-hit
 composition-trace-bad-alpha-space)
 adversary-initial-state

by (rule
 checked-staged-security-with-actual-alpha-prefix-partial-candidate-randomization-le-partial-header-bad-set-hit)

also have ... $\leq$

$P + D +$
 staged-phase-relation-error size
 (staged-alpha-search-queries budgets 0)

by (rule partial-header-bound)

finally show ?thesis .

qed

lemma checked-staged-security-with-data-state-partial-candidate-composition-randomization-bound-from-prefix-drift-c

assumes wf: staged-budget-wellformed budgets

and controlled: staged-adversary-controlled budgets A

and prefix-bound:

wp-event
 (checked-staged-security-experiment-with-actual-alpha-prefix-data-state
 A)
 (checked-staged-security-with-actual-alpha-prefix-bad-set-hit
 composition-trace-bad-alpha-space)
 adversary-initial-state $\leq P$

and *drift-bound*:

wp-event
 (checked-staged-security-experiment-with-actual-alpha-prefix-data-state
 A)
 (checked-staged-security-with-actual-alpha-prefix-partial-header-candidate-drift
 composition-trace-bad-alpha-space)
 adversary-initial-state $\leq D$

shows

wp-event (checked-staged-security-experiment-with-data-state *A*)
 (staged-security-with-data-state-verifier-event
 composition-randomization-bad-with-partial-candidates)
 adversary-initial-state $\leq$
 $P + D +$
 staged-phase-relation-error size
 (staged-alpha-search-queries budgets 0)

proof –

have *projected*:

wp-event (checked-staged-security-experiment-with-data-state *A*)
 (staged-security-with-data-state-verifier-event
 composition-randomization-bad-with-partial-candidates)
 adversary-initial-state =
wp-event
 (checked-staged-security-experiment-with-actual-alpha-prefix-data-state *A*)
 (checked-staged-security-with-actual-alpha-prefix-verifier-event
 composition-randomization-bad-with-partial-candidates)
 adversary-initial-state

by (rule
 checked-staged-security-with-data-state-verifier-event-actual-alpha-prefix-projection)

show *?thesis*

unfolding *projected*

by (rule
 checked-staged-security-with-actual-alpha-prefix-partial-candidate-composition-randomization-bound-from-prefix-c
 [OF *wf* controlled prefix-bound drift-bound])

qed

lemma *checked-staged-security-with-data-state-partial-candidate-composition-bad-bound-from-prefix-drift-and-budgets*

assumes *false-statement*: $\neg$ exists-valid-trace

and *wf*: staged-budget-wellformed budgets

and *controlled*: staged-adversary-controlled budgets *A*

and *prefix-bound*:

wp-event
 (checked-staged-security-experiment-with-actual-alpha-prefix-data-state
 A)

```

    (checked-staged-security-with-actual-alpha-prefix-bad-set-hit
     composition-trace-bad-alpha-space)
    adversary-initial-state  $\leq P$ 
and drift-bound:
  wp-event
    (checked-staged-security-experiment-with-actual-alpha-prefix-data-state
     A)
    (checked-staged-security-with-actual-alpha-prefix-partial-header-candidate-drift
     composition-trace-bad-alpha-space)
    adversary-initial-state  $\leq D$ 
shows
  wp-event (checked-staged-security-experiment-with-data-state A)
    (staged-security-with-data-state-verifier-event
     composition-bad-with-partial-candidates)
    adversary-initial-state  $\leq$ 
     $P + D +$ 
    staged-phase-relation-error size
    (staged-alpha-search-queries budgets 0)
proof -
  let ?M = checked-staged-security-experiment-with-data-state A
  let ?E = staged-security-with-data-state-verifier-event
  let ?degree = ?E composition-degree-bad-with-partial-candidates
  let ?random = ?E composition-randomization-bad-with-partial-candidates
  let ?bad = ?E composition-bad-with-partial-candidates
  have degree-false: ?degree = ( $\lambda$ out. False)
  by (rule ext)
    (auto simp: staged-security-with-data-state-verifier-event-def
     composition-degree-bad-with-partial-candidates-false
     split: option.splits prod.splits)
  have degree-bound:
    wp-event ?M ?degree adversary-initial-state  $\leq 0$ 
  unfolding degree-false wp-event-def wp-def dist-expect-def by simp
  have random-bound:
    wp-event ?M ?random adversary-initial-state  $\leq$ 
     $P + D +$ 
    staged-phase-relation-error size
    (staged-alpha-search-queries budgets 0)
  by (rule
    checked-staged-security-with-data-state-partial-candidate-composition-randomization-bound-from-prefix-drift-and-
    [OF wf controlled prefix-bound drift-bound])
  have wp-event ?M ?bad adversary-initial-state  $\leq$ 
    wp-event ?M ( $\lambda$ out. ?degree out  $\vee$  ?random out)
    adversary-initial-state
  by (rule wp-event-mono)
    (auto simp: staged-security-with-data-state-verifier-event-def
     dest: composition-bad-with-partial-candidates-split
     [OF false-statement]
     split: option.splits prod.splits)
  also have ...  $\leq$ 

```

```

    wp-event ?M ?degree adversary-initial-state +
    wp-event ?M ?random adversary-initial-state
  by (rule wp-event-union-bound)
also have ... ≤
  0 + (P + D +
    staged-phase-relation-error size
    (staged-alpha-search-queries budgets 0))
  by (intro add-mono degree-bound random-bound)
finally show ?thesis
  by simp
qed

```

lemma *checked-staged-security-with-data-state-partial-candidate-composition-bad-bound-from-drift-and-budgets:*

```

assumes false-statement:  $\neg$  exists-valid-trace
and wf: staged-budget-wellformed budgets
and controlled: staged-adversary-controlled budgets A
and drift-bound:
  wp-event
    (checked-staged-security-experiment-with-actual-alpha-prefix-data-state
     A)
    (checked-staged-security-with-actual-alpha-prefix-partial-header-candidate-drift
     composition-trace-bad-alpha-space)
    adversary-initial-state  $\leq$  D

```

shows

```

  wp-event (checked-staged-security-experiment-with-data-state A)
    (staged-security-with-data-state-verifier-event
     composition-bad-with-partial-candidates)
  adversary-initial-state  $\leq$ 
    composition-error-bound +
    staged-phase-relation-error size
    (staged-alpha-search-queries budgets 0) +
    hash-collision-budget-value 0
    (staged-alpha-search-queries budgets 0) +
    D +
    staged-phase-relation-error size
    (staged-alpha-search-queries budgets 0)

```

proof –

```

let ?P =
  composition-error-bound +
  staged-phase-relation-error size
  (staged-alpha-search-queries budgets 0) +
  hash-collision-budget-value 0
  (staged-alpha-search-queries budgets 0)

```

have transcript-bound:

```

  wp-event (checked-staged-transcript-with-alpha-prefix-program A)
    (checked-staged-transcript-actual-alpha-prefix-bad-set-hit
     composition-trace-bad-alpha-space)
  adversary-initial-state  $\leq$  ?P
by (rule

```

```

      checked-staged-transcript-with-alpha-prefix-actual-bad-set-hit-bound-from-budgets
      [OF wf controlled])
have prefix-bound:
  wp-event
    (checked-staged-security-experiment-with-actual-alpha-prefix-data-state
     A)
    (checked-staged-security-with-actual-alpha-prefix-bad-set-hit
     composition-trace-bad-alpha-space)
    adversary-initial-state  $\leq$  ?P
  by (rule
    checked-staged-security-with-actual-alpha-prefix-bad-set-hit-bound-from-transcript
    [OF transcript-bound])
have
  wp-event (checked-staged-security-experiment-with-data-state A)
    (staged-security-with-data-state-verifier-event
     composition-bad-with-partial-candidates)
    adversary-initial-state  $\leq$ 
    ?P + D +
    staged-phase-relation-error size
    (staged-alpha-search-queries budgets 0)
  by (rule
    checked-staged-security-with-data-state-partial-candidate-composition-bad-bound-from-prefix-drift-and-budgets
    [OF false-statement wf controlled prefix-bound drift-bound])
then show ?thesis
  by (simp add: add.assoc)
qed

lemma checked-staged-security-with-actual-alpha-prefix-empty-header-partial-alpha-hit-imp-partial-header-bad-set-hit
assumes support:
  out  $\in$ 
  set-dist
  (execute
   (checked-staged-security-experiment-with-actual-alpha-prefix-data-state
    A)
   adversary-initial-state)
and hit:
  checked-staged-security-with-actual-alpha-prefix-verifier-event
  composition-alpha-partial-opening-union-hit-with-empty-header out
shows
  checked-staged-security-with-actual-alpha-prefix-partial-header-bad-set-hit
  composition-trace-bad-alpha-space out
proof (cases out)
case None
then show ?thesis
  using hit
  unfolding checked-staged-security-with-actual-alpha-prefix-verifier-event-def
  by simp
next
case (Some result-pack)

```

```

then obtain prefix prefix-state data attacker-state result final-state
where out-eq:
  out =
    Some (((((prefix, prefix-state), data), attacker-state),
      result), final-state)
by (cases result-pack, auto split: prod.splits)
let ?s =
  verifier-state-from-adversary attacker-state
  (staged-proof-transcript data)
have verifier:
  Some (result, final-state)  $\in$  set-dist (execute verify-monad ?s)
using support
unfolding out-eq
  checked-staged-security-experiment-with-actual-alpha-prefix-data-state-def
by (auto elim!: set-dist-bindE split: prod.splits)
have local-hit:
  composition-alpha-partial-opening-union-hit-with-empty-header ?s
  (Some (result, final-state))
using hit
unfolding out-eq
  checked-staged-security-with-actual-alpha-prefix-verifier-event-def
  Let-def
by simp
have partial-header:
  alpha-header-list-set-hit ?s
  (alpha-header-supported-partial-union-bad-sets ?s
    composition-trace-bad-alpha-space)
  (Some (result, final-state))
by (rule
  composition-alpha-partial-opening-union-hit-with-empty-header-imp-alpha-header-partial-union-on-support
    [OF verifier local-hit])
show ?thesis
unfolding out-eq
  checked-staged-security-with-actual-alpha-prefix-partial-header-bad-set-hit-def
  Let-def
using partial-header by simp
qed

```

```

lemma checked-staged-security-with-actual-alpha-prefix-empty-header-partial-alpha-hit-le-partial-header-bad-set-hit:
  wp-event
  (checked-staged-security-experiment-with-actual-alpha-prefix-data-state A)
  (checked-staged-security-with-actual-alpha-prefix-verifier-event
    composition-alpha-partial-opening-union-hit-with-empty-header)
  adversary-initial-state  $\leq$ 
  wp-event
  (checked-staged-security-experiment-with-actual-alpha-prefix-data-state A)
  (checked-staged-security-with-actual-alpha-prefix-partial-header-bad-set-hit
    composition-trace-bad-alpha-space)
  adversary-initial-state

```

by (rule wp-event-mono-on-support)
 (rule
 checked-staged-security-with-actual-alpha-prefix-empty-header-partial-alpha-hit-imp-partial-header-bad-set-hit-on-s

lemma checked-staged-security-with-actual-alpha-prefix-empty-header-partial-alpha-hit-bound-from-prefix-drift-and-b

assumes wf: staged-budget-wellformed budgets

and controlled: staged-adversary-controlled budgets A

and prefix-bound:

wp-event

(checked-staged-security-experiment-with-actual-alpha-prefix-data-state A)

(checked-staged-security-with-actual-alpha-prefix-bad-set-hit
 composition-trace-bad-alpha-space)

adversary-initial-state $\leq P$

and drift-bound:

wp-event

(checked-staged-security-experiment-with-actual-alpha-prefix-data-state A)

(checked-staged-security-with-actual-alpha-prefix-partial-header-candidate-drift
 composition-trace-bad-alpha-space)

adversary-initial-state $\leq D$

shows

wp-event

(checked-staged-security-experiment-with-actual-alpha-prefix-data-state A)

(checked-staged-security-with-actual-alpha-prefix-verifier-event
 composition-alpha-partial-opening-union-hit-with-empty-header)

adversary-initial-state $\leq$

$P + D +$

staged-phase-relation-error size

(staged-alpha-search-queries budgets 0)

proof –

have partial-header-bound:

wp-event

(checked-staged-security-experiment-with-actual-alpha-prefix-data-state A)

(checked-staged-security-with-actual-alpha-prefix-partial-header-bad-set-hit
 composition-trace-bad-alpha-space)

adversary-initial-state $\leq$

$P + D +$

staged-phase-relation-error size

(staged-alpha-search-queries budgets 0)

by (rule

checked-staged-security-with-actual-alpha-prefix-partial-header-bad-set-hit-bound-from-prefix-drift-and-budgets
 [OF wf controlled prefix-bound drift-bound])

have wp-event

(checked-staged-security-experiment-with-actual-alpha-prefix-data-state A)

(checked-staged-security-with-actual-alpha-prefix-verifier-event
 composition-alpha-partial-opening-union-hit-with-empty-header)

adversary-initial-state $\leq$

wp-event

(checked-staged-security-experiment-with-actual-alpha-prefix-data-state A)
 (checked-staged-security-with-actual-alpha-prefix-partial-header-bad-set-hit
 composition-trace-bad-alpha-space)
 adversary-initial-state
 by (rule
 checked-staged-security-with-actual-alpha-prefix-empty-header-partial-alpha-hit-le-partial-header-bad-set-hit)
 also have ... $\leq$
 $P + D +$
 staged-phase-relation-error size
 (staged-alpha-search-queries budgets 0)
 by (rule partial-header-bound)
 finally show ?thesis .
 qed

lemma checked-staged-security-with-data-state-empty-header-partial-alpha-hit-bound-from-prefix-drift-and-budgets:

assumes wf: staged-budget-wellformed budgets

and controlled: staged-adversary-controlled budgets A

and prefix-bound:

wp-event

(checked-staged-security-experiment-with-actual-alpha-prefix-data-state
 A)

(checked-staged-security-with-actual-alpha-prefix-bad-set-hit
 composition-trace-bad-alpha-space)

adversary-initial-state $\leq P$

and drift-bound:

wp-event

(checked-staged-security-experiment-with-actual-alpha-prefix-data-state
 A)

(checked-staged-security-with-actual-alpha-prefix-partial-header-candidate-drift
 composition-trace-bad-alpha-space)

adversary-initial-state $\leq D$

shows

wp-event (checked-staged-security-experiment-with-data-state A)

(staged-security-with-data-state-verifier-event
 composition-alpha-partial-opening-union-hit-with-empty-header)

adversary-initial-state $\leq$

$P + D +$

staged-phase-relation-error size

(staged-alpha-search-queries budgets 0)

proof –

have projected:

wp-event (checked-staged-security-experiment-with-data-state A)

(staged-security-with-data-state-verifier-event
 composition-alpha-partial-opening-union-hit-with-empty-header)

adversary-initial-state =

wp-event

(checked-staged-security-experiment-with-actual-alpha-prefix-data-state A)

(checked-staged-security-with-actual-alpha-prefix-verifier-event
 composition-alpha-partial-opening-union-hit-with-empty-header)

adversary-initial-state
by (rule
checked-staged-security-with-data-state-verifier-event-actual-alpha-prefix-projection)
show *?thesis*
unfolding *projected*
by (rule
checked-staged-security-with-actual-alpha-prefix-empty-header-partial-alpha-hit-bound-from-prefix-drift-and-budg
[OF wf controlled prefix-bound drift-bound])
qed

lemma *checked-staged-security-with-data-state-empty-header-composition-randomization-bound-from-prefix-drift-and*
assumes *wf: staged-budget-wellformed budgets*
and *controlled: staged-adversary-controlled budgets A*
and *prefix-bound:*
wp-event
(checked-staged-security-experiment-with-actual-alpha-prefix-data-state
A)
(checked-staged-security-with-actual-alpha-prefix-bad-set-hit
composition-trace-bad-alpha-space)
adversary-initial-state ≤ P
and *drift-bound:*
wp-event
(checked-staged-security-experiment-with-actual-alpha-prefix-data-state
A)
(checked-staged-security-with-actual-alpha-prefix-partial-header-candidate-drift
composition-trace-bad-alpha-space)
adversary-initial-state ≤ D
shows
wp-event (checked-staged-security-experiment-with-data-state A)
(staged-security-with-data-state-verifier-event
composition-randomization-bad-with-empty-composition-header-candidates)
adversary-initial-state ≤
P + D +
staged-phase-relation-error size
(staged-alpha-search-queries budgets 0)
proof –
have *alpha-bound:*
wp-event (checked-staged-security-experiment-with-data-state A)
(staged-security-with-data-state-verifier-event
composition-alpha-partial-opening-union-hit-with-empty-header)
adversary-initial-state ≤
P + D +
staged-phase-relation-error size
(staged-alpha-search-queries budgets 0)
by (rule
checked-staged-security-with-data-state-empty-header-partial-alpha-hit-bound-from-prefix-drift-and-budg
[OF wf controlled prefix-bound drift-bound])
have *wp-event (checked-staged-security-experiment-with-data-state A)*
(staged-security-with-data-state-verifier-event

composition-randomization-bad-with-empty-composition-header-candidates)
adversary-initial-state $\leq$
wp-event (*checked-staged-security-experiment-with-data-state A*)
(*staged-security-with-data-state-verifier-event*
composition-alpha-partial-opening-union-hit-with-empty-header)
adversary-initial-state
by (*rule wp-event-mono*)
(*auto simp: staged-security-with-data-state-verifier-event-def*
intro:
composition-alpha-bad-set-hit-with-empty-header-imp-partial-opening-union-hit
composition-randomization-bad-with-empty-composition-header-candidates-imp-alpha-hit
split: option.splits prod.splits)
also have ... $\leq$
P + D +
staged-phase-relation-error size
(*staged-alpha-search-queries budgets 0*)
by (*rule alpha-bound*)
finally show ?thesis .
qed

lemma *checked-staged-security-with-data-state-empty-header-bad-event-bound-from-prefix-drift-trace-fri-and-query:*

assumes *false-statement: $\neg$ exists-valid-trace*
and *wf: staged-budget-wellformed budgets*
and *controlled: staged-adversary-controlled budgets A*
and *prefix-bound:*
wp-event
(*checked-staged-security-experiment-with-actual-alpha-prefix-data-state*
A)
(*checked-staged-security-with-actual-alpha-prefix-bad-set-hit*
composition-trace-bad-alpha-space)
adversary-initial-state $\leq P$
and *drift-bound:*
wp-event
(*checked-staged-security-experiment-with-actual-alpha-prefix-data-state*
A)
(*checked-staged-security-with-actual-alpha-prefix-partial-header-candidate-drift*
composition-trace-bad-alpha-space)
adversary-initial-state $\leq D$
and *trace-fri-bound:*
wp-event (*checked-staged-security-experiment-with-data-state A*)
(*staged-security-with-data-state-verifier-event*
trace-fri-bad-with-empty-composition-header-candidates)
adversary-initial-state $\leq$ *trace-fri-error*
and *query-bound:*
wp-event (*checked-staged-security-experiment-with-data-state A*)
(*staged-security-with-data-state-verifier-event*
query-bad-with-empty-composition-header-candidates)
adversary-initial-state $\leq$ *empty-query-error*
shows

```

wp-event (checked-staged-security-experiment-with-data-state A)
  (staged-security-with-data-state-verifier-event
    soundness-bad-event-partial-candidate-empty-header)
adversary-initial-state ≤
P + D +
staged-phase-relation-error size
(staged-alpha-search-queries budgets 0) +
trace-fri-error + empty-query-error
proof –
  let ?M = checked-staged-security-experiment-with-data-state A
  let ?E = staged-security-with-data-state-verifier-event
  let ?degree =
    ?E composition-degree-bad-with-empty-composition-header-candidates
  let ?random =
    ?E composition-randomization-bad-with-empty-composition-header-candidates
  let ?trace =
    ?E trace-fri-bad-with-empty-composition-header-candidates
  let ?query =
    ?E query-bad-with-empty-composition-header-candidates
  let ?bad = ?E soundness-bad-event-partial-candidate-empty-header
  have degree-false: ?degree = (λout. False)
    by (rule ext)
      (auto simp: staged-security-with-data-state-verifier-event-def
        composition-degree-bad-with-empty-composition-header-candidates-false
        split: option.splits prod.splits)
  have degree-bound:
    wp-event ?M ?degree adversary-initial-state ≤ 0
    unfolding degree-false wp-event-def wp-def dist-expect-def by simp
  have random-bound:
    wp-event ?M ?random adversary-initial-state ≤
      P + D +
      staged-phase-relation-error size
      (staged-alpha-search-queries budgets 0)
    by (rule
      checked-staged-security-with-data-state-empty-header-composition-randomization-bound-from-prefix-drift-and-buo
      [OF wf controlled prefix-bound drift-bound])
  have wp-event ?M ?bad adversary-initial-state ≤
    wp-event ?M
      (λout. ?degree out ∨ ?random out ∨ ?trace out ∨
        ?query out)
      adversary-initial-state
  proof (rule wp-event-mono)
    fix out
    assume bad: ?bad out
    show ?degree out ∨ ?random out ∨ ?trace out ∨ ?query out
    proof (cases out)
      case None
      then show ?thesis
        using bad

```

```

    unfolding staged-security-with-data-state-verifier-event-def
    by simp
next
case (Some packed)
then obtain data attacker-state result final-state where out-eq:
  out = Some (((data, attacker-state), result), final-state)
  by (cases packed, auto split: prod.splits)
let ?s =
  verifier-state-from-adversary attacker-state
  (staged-proof-transcript data)
have local-bad:
  soundness-bad-event-partial-candidate-empty-header ?s
  (Some (result, final-state))
using bad
unfolding out-eq staged-security-with-data-state-verifier-event-def
by simp
then consider
  (comp)
    composition-bad-with-empty-composition-header-candidates ?s
    (Some (result, final-state))
  | (trace)
    trace-fri-bad-with-empty-composition-header-candidates ?s
    (Some (result, final-state))
  | (query)
    query-bad-with-empty-composition-header-candidates ?s
    (Some (result, final-state))
  unfolding soundness-bad-event-partial-candidate-empty-header-def
  by blast
then show ?thesis
proof cases
case comp
then have comp-split:
  composition-degree-bad-with-empty-composition-header-candidates ?s
  (Some (result, final-state)) ∨
  composition-randomization-bad-with-empty-composition-header-candidates
  ?s (Some (result, final-state))
  by (rule
    composition-bad-with-empty-composition-header-candidates-split
    [OF false-statement])
then show ?thesis
  unfolding out-eq staged-security-with-data-state-verifier-event-def
  using comp-split by (auto split: option.splits prod.splits)
next
case trace
then show ?thesis
  unfolding out-eq staged-security-with-data-state-verifier-event-def
  by simp
next
case query

```

```

    then show ?thesis
      unfolding out-eq staged-security-with-data-state-verifier-event-def
      by simp
    qed
  qed
qed
also have ... ≤
  wp-event ?M ?degree adversary-initial-state +
  wp-event ?M ?random adversary-initial-state +
  wp-event ?M ?trace adversary-initial-state +
  wp-event ?M ?query adversary-initial-state
  by (rule wp-event-union-bound4)
also have ... ≤
  0 +
  (P + D +
    staged-phase-relation-error size
    (staged-alpha-search-queries budgets 0)) +
  trace-fri-error + empty-query-error
  by (intro add-mono degree-bound random-bound trace-fri-bound query-bound)
finally show ?thesis
  by (simp add: algebra-simps)
qed

lemma checked-staged-security-with-data-state-empty-header-bad-event-bound-from-drift-trace-fri-and-query:
  assumes false-statement:  $\neg$  exists-valid-trace
  and wf: staged-budget-wellformed budgets
  and controlled: staged-adversary-controlled budgets A
  and drift-bound:
    wp-event
      (checked-staged-security-experiment-with-actual-alpha-prefix-data-state
        A)
      (checked-staged-security-with-actual-alpha-prefix-partial-header-candidate-drift
        composition-trace-bad-alpha-space)
      adversary-initial-state ≤ D
  and trace-fri-bound:
    wp-event (checked-staged-security-experiment-with-data-state A)
      (staged-security-with-data-state-verifier-event
        trace-fri-bad-with-empty-composition-header-candidates)
      adversary-initial-state ≤ trace-fri-error
  and query-bound:
    wp-event (checked-staged-security-experiment-with-data-state A)
      (staged-security-with-data-state-verifier-event
        query-bad-with-empty-composition-header-candidates)
      adversary-initial-state ≤ nnreal rounds * query-error-bound
  shows
    wp-event (checked-staged-security-experiment-with-data-state A)
      (staged-security-with-data-state-verifier-event
        soundness-bad-event-partial-candidate-empty-header)
      adversary-initial-state ≤

```

```

    composition-error-bound +
    staged-phase-relation-error size
    (staged-alpha-search-queries budgets 0) +
    hash-collision-budget-value 0
    (staged-alpha-search-queries budgets 0) +
    D +
    staged-phase-relation-error size
    (staged-alpha-search-queries budgets 0) +
    trace-fri-error + nnreal rounds * query-error-bound
proof -
  let ?P =
    composition-error-bound +
    staged-phase-relation-error size
    (staged-alpha-search-queries budgets 0) +
    hash-collision-budget-value 0
    (staged-alpha-search-queries budgets 0)
  have transcript-bound:
    wp-event (checked-staged-transcript-with-alpha-prefix-program A)
    (checked-staged-transcript-actual-alpha-prefix-bad-set-hit
     composition-trace-bad-alpha-space)
    adversary-initial-state ≤ ?P
  by (rule
    checked-staged-transcript-with-alpha-prefix-actual-bad-set-hit-bound-from-budgets
    [OF wf controlled])
  have prefix-bound:
    wp-event
    (checked-staged-security-experiment-with-actual-alpha-prefix-data-state
     A)
    (checked-staged-security-with-actual-alpha-prefix-bad-set-hit
     composition-trace-bad-alpha-space)
    adversary-initial-state ≤ ?P
  by (rule
    checked-staged-security-with-actual-alpha-prefix-bad-set-hit-bound-from-transcript
    [OF transcript-bound])
  have
    wp-event (checked-staged-security-experiment-with-data-state A)
    (staged-security-with-data-state-verifier-event
     soundness-bad-event-partial-candidate-empty-header)
    adversary-initial-state ≤
    ?P + D +
    staged-phase-relation-error size
    (staged-alpha-search-queries budgets 0) +
    trace-fri-error + nnreal rounds * query-error-bound
  by (rule
    checked-staged-security-with-data-state-empty-header-bad-event-bound-from-prefix-drift-trace-fri-and-query
    [OF false-statement wf controlled prefix-bound drift-bound
     trace-fri-bound query-bound])
  then show ?thesis
  by (simp add: add.assoc)

```

qed

lemma *soundness-bad-event-table-aware-with-pairwise-side-union-bound:*
fixes $s :: ('f, 'a) \text{ protocol-channel-scheme}$
and $\text{pairwise-side-error} :: \text{prob}$
assumes *side-bound:*
 $\text{wp-event verify-monad (supported-pairwise-side-bad } s) s \leq$
 $\text{pairwise-side-error}$
and *merkle-bound:*
 $\text{wp-event verify-monad (fri-merkle-binding-bad } s) s \leq$
 $\text{merkle-binding-error}$
and *comp-bound:*
 $\text{wp-event verify-monad (composition-bad } s) s \leq \text{composition-error-bound}$
and *trace-fri-bound:*
 $\text{wp-event verify-monad (trace-fri-bad-with-tables } s) s \leq$
 trace-fri-error
and *comp-fri-bound:*
 $\text{wp-event verify-monad (composition-fri-bad-with-tables } s) s \leq$
 $\text{composition-fri-error}$
and *query-bound:*
 $\text{wp-event verify-monad (query-bad } s) s \leq \text{nnreal rounds} * \text{query-error-bound}$
shows
 $\text{wp-event verify-monad}$
 $(\text{soundness-bad-event-table-aware-with-pairwise-side } s) s \leq$
 $\text{pairwise-side-error} + \text{soundness-bound-with-merkle}$
proof –
have *merkle-table-bound:*
 $\text{wp-event verify-monad}$
 $(\text{soundness-bad-event-table-aware-with-merkle } s) s \leq$
 $\text{soundness-bound-with-merkle}$
by (rule *soundness-bad-event-table-aware-with-merkle-union-bound*
 $[\text{OF merkle-bound comp-bound trace-fri-bound comp-fri-bound}$
 $\text{query-bound}]$)
have $\text{wp-event verify-monad}$
 $(\text{soundness-bad-event-table-aware-with-pairwise-side } s) s \leq$
 $\text{wp-event verify-monad (supported-pairwise-side-bad } s) s +$
 $\text{wp-event verify-monad}$
 $(\text{soundness-bad-event-table-aware-with-merkle } s) s$
unfolding *soundness-bad-event-table-aware-with-pairwise-side-def*
by (rule *wp-event-union-bound*)
also have $\dots \leq \text{pairwise-side-error} + \text{soundness-bound-with-merkle}$
by (intro *add-mono side-bound merkle-table-bound*)
finally show *?thesis* .
qed

lemma *soundness-bad-event-table-aware-with-output-local-side-union-bound:*
fixes $s :: ('f, 'a) \text{ protocol-channel-scheme}$
and $\text{output-local-side-error} :: \text{prob}$
assumes *side-bound:*

$wp\text{-event verify-monad (supported-output-local-side-bad } s) s \leq$
 $output\text{-local-side-error}$
and merkle-bound:
 $wp\text{-event verify-monad (fri-merkle-binding-bad } s) s \leq$
 $merkle\text{-binding-error}$
and comp-bound:
 $wp\text{-event verify-monad (composition-bad } s) s \leq composition\text{-error-bound}$
and trace-fri-bound:
 $wp\text{-event verify-monad (trace-fri-bad-with-tables } s) s \leq$
 $trace\text{-fri-error}$
and comp-fri-bound:
 $wp\text{-event verify-monad (composition-fri-bad-with-tables } s) s \leq$
 $composition\text{-fri-error}$
and query-bound:
 $wp\text{-event verify-monad (query-bad } s) s \leq$
 $nnreal\ rounds * query\text{-error-bound}$
shows
 $wp\text{-event verify-monad}$
 $(soundness\text{-bad-event-table-aware-with-output-local-side } s) s \leq$
 $output\text{-local-side-error} + soundness\text{-bound-with-merkle}$
proof –
have merkle-table-bound:
 $wp\text{-event verify-monad}$
 $(soundness\text{-bad-event-table-aware-with-merkle } s) s \leq$
 $soundness\text{-bound-with-merkle}$
by (rule soundness-bad-event-table-aware-with-merkle-union-bound
 $[OF\ merkle\text{-bound comp-bound trace-fri-bound comp-fri-bound}$
 $query\text{-bound}]$)
have $wp\text{-event verify-monad}$
 $(soundness\text{-bad-event-table-aware-with-output-local-side } s) s \leq$
 $wp\text{-event verify-monad (supported-output-local-side-bad } s) s +$
 $wp\text{-event verify-monad}$
 $(soundness\text{-bad-event-table-aware-with-merkle } s) s$
unfolding soundness-bad-event-table-aware-with-output-local-side-def
by (rule wp-event-union-bound)
also have $\dots \leq$
 $output\text{-local-side-error} + soundness\text{-bound-with-merkle}$
by (intro add-mono side-bound merkle-table-bound)
finally show ?thesis .
qed

lemma soundness-from-bad-event-partition:
fixes $s :: ('f, 'a)\ protocol\text{-channel-scheme}$
assumes false-statement: $\neg\ exists\text{-valid-trace}$
and partition:
 $\bigwedge out :: (unit\ list \times ('f, 'a)\ protocol\text{-channel-scheme})\ option.$
 $out \in set\text{-dist (execute verify-monad } s) \implies$
 $accepted\ out \implies soundness\text{-bad-event } s\ out$
and comp-bound: $wp\text{-event verify-monad (composition-bad } s) s \leq composition\text{-error-bound}$

and *trace-fri-bound*: *wp-event verify-monad (trace-fri-bad s) s ≤ trace-fri-error*
and *comp-fri-bound*:
 wp-event verify-monad (composition-fri-bad s) s ≤ composition-fri-error
and *query-bound*:
 *wp-event verify-monad (query-bad s) s ≤ nreal rounds * query-error-bound*
shows *wp-event verify-monad accepted s ≤ soundness-bound*
proof –
 have *bad-event-bound*:
 wp-event verify-monad (soundness-bad-event s) s ≤ soundness-bound
 by (rule *soundness-bad-event-union-bound*
 [*OF comp-bound trace-fri-bound comp-fri-bound query-bound*])
 have *accepted-subset*:
 $\bigwedge out :: (unit\ list \times ('f, 'a)\ protocol-channel-scheme)\ option.$
 $out \in set-dist\ (execute\ verify-monad\ s) \implies accepted\ out \implies$
 $soundness-bad-event\ s\ out$
 proof –
 fix *out* :: $(unit\ list \times ('f, 'a)\ protocol-channel-scheme)\ option$
 assume *supp*: $out \in set-dist\ (execute\ verify-monad\ s)$
 assume *accepted out*
 then show *soundness-bad-event s out*
 by (rule *partition[OF supp]*)
 qed
 have *wp-event verify-monad accepted s ≤*
 wp-event verify-monad (soundness-bad-event s) s
 by (rule *wp-event-mono-on-support*) (rule *accepted-subset*)
 also have ... $\leq soundness-bound$
 by (rule *bad-event-bound*)
 finally show *?thesis* .
qed

lemma *soundness-from-table-aware-bad-event-partition*:
fixes *s* :: $('f, 'a)\ protocol-channel-scheme$
assumes *false-statement*: $\neg exists-valid-trace$
and *partition*:
 $\bigwedge out :: (unit\ list \times ('f, 'a)\ protocol-channel-scheme)\ option.$
 $out \in set-dist\ (execute\ verify-monad\ s) \implies$
 $accepted\ out \implies$
 $soundness-bad-event-table-aware-with-merkle\ s\ out$
and *merkle-bound*:
 wp-event verify-monad (fri-merkle-binding-bad s) s ≤
 merkle-binding-error
and *comp-bound*:
 wp-event verify-monad (composition-bad s) s ≤
 composition-error-bound
and *trace-fri-bound*:
 wp-event verify-monad (trace-fri-bad-with-tables s) s ≤
 trace-fri-error
and *comp-fri-bound*:
 wp-event verify-monad (composition-fri-bad-with-tables s) s ≤

composition-fri-error
and *query-bound*:
wp-event verify-monad (query-bad s) s ≤
*nnreal rounds * query-error-bound*
shows *wp-event verify-monad accepted s ≤ soundness-bound-with-merkle*
proof –
have *bad-event-bound*:
wp-event verify-monad
(soundness-bad-event-table-aware-with-merkle s) s ≤
soundness-bound-with-merkle
by (rule *soundness-bad-event-table-aware-with-merkle-union-bound*
[OF merkle-bound comp-bound trace-fri-bound comp-fri-bound
query-bound])
have *accepted-subset*:
 $\bigwedge out :: (\text{unit list} \times ('f, 'a) \text{ protocol-channel-scheme}) \text{ option.}$
 $out \in \text{set-dist } (\text{execute verify-monad } s) \implies \text{accepted } out \implies$
 $\text{soundness-bad-event-table-aware-with-merkle } s \text{ out}$
proof –
fix *out* :: (*unit list* × (*'f*, *'a*) *protocol-channel-scheme*) *option*
assume *supp*: *out* ∈ *set-dist (execute verify-monad s)*
assume *accepted out*
then show *soundness-bad-event-table-aware-with-merkle s out*
by (rule *partition[OF supp]*)
qed
have *wp-event verify-monad accepted s ≤*
wp-event verify-monad
(soundness-bad-event-table-aware-with-merkle s) s
by (rule *wp-event-mono-on-support*) (rule *accepted-subset*)
also have ... ≤ *soundness-bound-with-merkle*
by (rule *bad-event-bound*)
finally show ?thesis .
qed

lemma *soundness-from-partial-opening-bad-event-partition*:
fixes *s* :: (*'f*, *'a*) *protocol-channel-scheme*
and *B* :: *prob*
assumes *partition*:
 $\bigwedge out :: (\text{unit list} \times ('f, 'a) \text{ protocol-channel-scheme}) \text{ option.}$
 $out \in \text{set-dist } (\text{execute verify-monad } s) \implies$
 $\text{accepted } out \implies$
 $\text{soundness-bad-event-partial-opening-with-merkle } s \text{ out}$
and *bad-event-bound*:
wp-event verify-monad
(soundness-bad-event-partial-opening-with-merkle s) s ≤ B
shows *wp-event verify-monad accepted s ≤ B*
proof –
have *wp-event verify-monad accepted s ≤*
wp-event verify-monad
(soundness-bad-event-partial-opening-with-merkle s) s

```

    by (rule wp-event-mono-on-support) (rule partition)
  also have ...  $\leq$  B
    by (rule bad-event-bound)
  finally show ?thesis .
qed

lemma soundness-from-table-aware-bad-event-partition-with-pairwise-side:
  fixes s :: ('f, 'a) protocol-channel-scheme
    and pairwise-side-error :: prob
  assumes false-statement:  $\neg$  exists-valid-trace
    and partition:
     $\bigwedge$  out :: (unit list  $\times$  ('f, 'a) protocol-channel-scheme) option.
      out  $\in$  set-dist (execute verify-monad s)  $\implies$ 
        accepted out  $\implies$ 
          soundness-bad-event-table-aware-with-pairwise-side s out
    and side-bound:
      wp-event verify-monad (supported-pairwise-side-bad s) s  $\leq$ 
        pairwise-side-error
    and merkle-bound:
      wp-event verify-monad (fri-merkle-binding-bad s) s  $\leq$ 
        merkle-binding-error
    and comp-bound:
      wp-event verify-monad (composition-bad s) s  $\leq$ 
        composition-error-bound
    and trace-fri-bound:
      wp-event verify-monad (trace-fri-bad-with-tables s) s  $\leq$ 
        trace-fri-error
    and comp-fri-bound:
      wp-event verify-monad (composition-fri-bad-with-tables s) s  $\leq$ 
        composition-fri-error
    and query-bound:
      wp-event verify-monad (query-bad s) s  $\leq$ 
        nnreal rounds * query-error-bound
  shows
    wp-event verify-monad accepted s  $\leq$ 
      pairwise-side-error + soundness-bound-with-merkle
proof -
  have bad-event-bound:
    wp-event verify-monad
      (soundness-bad-event-table-aware-with-pairwise-side s) s  $\leq$ 
        pairwise-side-error + soundness-bound-with-merkle
  by (rule soundness-bad-event-table-aware-with-pairwise-side-union-bound
    [OF side-bound merkle-bound comp-bound trace-fri-bound comp-fri-bound
      query-bound])
  have accepted-subset:
     $\bigwedge$  out :: (unit list  $\times$  ('f, 'a) protocol-channel-scheme) option.
      out  $\in$  set-dist (execute verify-monad s)  $\implies$  accepted out  $\implies$ 
        soundness-bad-event-table-aware-with-pairwise-side s out
proof -

```

```

fix out :: (unit list × ('f, 'a) protocol-channel-scheme) option
assume supp: out ∈ set-dist (execute verify-monad s)
assume accepted out
then show soundness-bad-event-table-aware-with-pairwise-side s out
  by (rule partition[OF supp])
qed
have wp-event verify-monad accepted s ≤
  wp-event verify-monad
    (soundness-bad-event-table-aware-with-pairwise-side s) s
  by (rule wp-event-mono-on-support) (rule accepted-subset)
also have ... ≤ pairwise-side-error + soundness-bound-with-merkle
  by (rule bad-event-bound)
finally show ?thesis .
qed

lemma soundness-from-table-aware-bad-event-partition-with-output-local-side:
fixes s :: ('f, 'a) protocol-channel-scheme
and output-local-side-error :: prob
assumes false-statement: ¬ exists-valid-trace
and partition:
  ∧ out :: (unit list × ('f, 'a) protocol-channel-scheme) option.
  out ∈ set-dist (execute verify-monad s) ⇒
  accepted out ⇒
  soundness-bad-event-table-aware-with-output-local-side s out
and side-bound:
  wp-event verify-monad (supported-output-local-side-bad s) s ≤
  output-local-side-error
and merkle-bound:
  wp-event verify-monad (fri-merkle-binding-bad s) s ≤
  merkle-binding-error
and comp-bound:
  wp-event verify-monad (composition-bad s) s ≤
  composition-error-bound
and trace-fri-bound:
  wp-event verify-monad (trace-fri-bad-with-tables s) s ≤
  trace-fri-error
and comp-fri-bound:
  wp-event verify-monad (composition-fri-bad-with-tables s) s ≤
  composition-fri-error
and query-bound:
  wp-event verify-monad (query-bad s) s ≤
  nnreal rounds * query-error-bound
shows
  wp-event verify-monad accepted s ≤
  output-local-side-error + soundness-bound-with-merkle
proof –
have bad-event-bound:
  wp-event verify-monad
    (soundness-bad-event-table-aware-with-output-local-side s) s ≤

```

output-local-side-error + *soundness-bound-with-merkle*
by (rule
soundness-bad-event-table-aware-with-output-local-side-union-bound
[*OF side-bound merkle-bound comp-bound trace-fri-bound comp-fri-bound*
query-bound])
have *wp-event verify-monad accepted s* ≤
wp-event verify-monad
(*soundness-bad-event-table-aware-with-output-local-side s*) *s*
by (rule *wp-event-mono-on-support*) (rule *partition*)
also have ... ≤
output-local-side-error + *soundness-bound-with-merkle*
by (rule *bad-event-bound*)
finally show ?thesis .
qed

lemma *accepted-execution-partition-obligation*:
fixes *s* :: ('f, 'a) *protocol-channel-scheme*
assumes *false-statement*: ¬ *exists-valid-trace*
and *bind*: *initial-merkle-binding-no-bad s*
shows
 $\bigwedge out :: (unit\ list \times ('f, 'a)\ protocol-channel-scheme)\ option.$
 $out \in set-dist\ (execute\ verify-monad\ s) \implies$
 $accepted\ out \implies$
 $soundness-bad-event\ s\ out$

proof –
fix *out* :: (unit list × ('f, 'a) protocol-channel-scheme) option
assume *outcome*: *out* ∈ *set-dist* (*execute verify-monad s*)
and *acc*: *accepted out*
from *acc* **obtain** *result final-state* **where** *out-eq*: *out* = *Some (result, final-state)*
unfolding *accepted-def* **by** (*cases out*) *auto*
have *verify-out*:
Some (result, final-state) ∈ *set-dist* (*execute verify-monad s*)
using *outcome out-eq* **by** *simp*
from *verify-monad-merkle-bound-witnesses*[*OF bind verify-out*]
obtain *trace-table composition-table* as *query-idxs* **where** *bound*:
accepted-with-bound-tables s (*Some (result, final-state)*)
trace-table composition-table as *query-idxs*
by *blast*
have *composition-bad s* (*Some (result, final-state)*) ∨
trace-fri-bad s (*Some (result, final-state)*) ∨
composition-fri-bad s (*Some (result, final-state)*) ∨
query-bad s (*Some (result, final-state)*)
by (rule *accepted-bound-table-partition*[*OF false-statement bound*])
then show *soundness-bad-event s out*
unfolding *soundness-bad-event-def* **using** *out-eq* **by** *simp*
qed

lemma *accepted-execution-partition-obligation-partial-opening*:
fixes *s* :: ('f, 'a) *protocol-channel-scheme*

```

assumes false-statement:  $\neg$  exists-valid-trace
shows
   $\bigwedge out :: (\text{unit list} \times ('f, 'a) \text{ protocol-channel-scheme}) \text{ option.}$ 
   $out \in \text{set-dist } (\text{execute verify-monad } s) \implies$ 
   $\text{accepted } out \implies$ 
   $\text{soundness-bad-event-partial-opening-with-merkle } s \text{ out}$ 
by (rule accepted-partition-soundness-bad-event-partial-opening-with-merkle
  [OF false-statement])

```

end

end

```

theory Soundness-Staged-Partial-Merkle
  imports Soundness-Staged-Bounds
begin

```

Aggregator for staged partial-Merkle soundness infrastructure. The old public-path compatibility wrappers based on partial-Merkle event-bound premises were removed before publication; downstream active-route theories import this file only for the underlying staged bounds.

end

```

theory Soundness-Partial-Candidate-Blocker
  imports Soundness-Partial-Merkle
begin

```

Diagnostic lemmas for the remaining query-bound route.

Partial authenticated openings constrain only the opened indices. Without a separate Merkle-binding/completion theorem, the corresponding partial table candidate sets are not singleton in general: unopened positions can be changed while preserving all partial-opening evidence.

```

context soundness
begin

```

lemma *table-agrees-with-authenticated-openings-update-unopened*:

```

assumes agrees:
  table-agrees-with-authenticated-openings table len openings
and unopened:
   $\bigwedge opn. opn \in \text{set openings} \implies \text{opening-index } opn \neq j$ 

```

shows

```

  table-agrees-with-authenticated-openings (table[j := v]) len openings

```

proof –

```

  have len: len = length table
  using agrees unfolding table-agrees-with-authenticated-openings-def
  by simp

```

```

have vals:
   $\bigwedge \text{opn}. \text{opn} \in \text{set openings} \implies$ 
    opening-index opn < length table  $\wedge$ 
    table ! opening-index opn = opening-value opn
  using agrees unfolding table-agrees-with-authenticated-openings-def
  by blast
have updated-values:
   $\bigwedge \text{opn}. \text{opn} \in \text{set openings} \implies$ 
    opening-index opn < length (table[j := v])  $\wedge$ 
    (table[j := v]) ! opening-index opn = opening-value opn
proof -
  fix opn
  assume opn-in: opn  $\in$  set openings
  have idx-bound: opening-index opn < length table
    using vals[OF opn-in] by simp
  have idx-ne: opening-index opn  $\neq$  j
    by (rule unopened[OF opn-in])
  have (table[j := v]) ! opening-index opn =
    table ! opening-index opn
    using idx-bound idx-ne by simp
  then show
    opening-index opn < length (table[j := v])  $\wedge$ 
    (table[j := v]) ! opening-index opn = opening-value opn
    using vals[OF opn-in] by simp
qed
show ?thesis
  unfolding table-agrees-with-authenticated-openings-def
  using len updated-values by simp
qed

lemma table-agrees-with-authenticated-openings-eq-if-all-indices-opened:
  assumes agrees:
    table-agrees-with-authenticated-openings table len openings
  and agrees':
    table-agrees-with-authenticated-openings table' len openings
  and cover:
     $\bigwedge j. j < \text{len} \implies$ 
       $\exists \text{opn} \in \text{set openings}. \text{opening-index opn} = j$ 
  shows table = table'
proof (rule nth-equalityI)
  show length table = length table'
    using agrees agrees'
    unfolding table-agrees-with-authenticated-openings-def by simp
next
  fix j
  assume j-bound: j < length table
  have len-eq: length table = len
    using agrees unfolding table-agrees-with-authenticated-openings-def
    by simp

```

```

then obtain opn where opn-in: opn ∈ set openings
  and opn-idx: opening-index opn = j
  using cover j-bound by blast
have table-val: table ! j = opening-value opn
  using agrees opn-in opn-idx
  unfolding table-agrees-with-authenticated-openings-def by blast
have table'-val: table' ! j = opening-value opn
  using agrees' opn-in opn-idx
  unfolding table-agrees-with-authenticated-openings-def by blast
show table ! j = table' ! j
  using table-val table'-val by simp
qed

```

```

lemma partial-trace-table-candidate-update-unopened:
  assumes cand: partial-trace-table-candidate trace-table trace-openings
  and unopened:
     $\bigwedge i \text{ opn. } i < \text{rounds} \implies \text{opn} \in \text{set } (\text{trace-openings} ! i) \implies$ 
       $\text{opening-index opn} \neq j$ 
  shows
    partial-trace-table-candidate (trace-table[j := v]) trace-openings
proof -
  have len: length (trace-table[j := v]) = scale * clength
    using cand unfolding partial-trace-table-candidate-def by simp
  have rounds-len: length trace-openings = rounds
    using cand unfolding partial-trace-table-candidate-def by simp
  have agrees:
     $\bigwedge i. i < \text{rounds} \implies$ 
      table-agrees-with-authenticated-openings (trace-table[j := v])
      (scale * clength) (trace-openings ! i)
  proof -
    fix i
    assume i-bound: i < rounds
    have old:
      table-agrees-with-authenticated-openings trace-table
      (scale * clength) (trace-openings ! i)
      using cand i-bound unfolding partial-trace-table-candidate-def by blast
    show
      table-agrees-with-authenticated-openings (trace-table[j := v])
      (scale * clength) (trace-openings ! i)
      by (rule table-agrees-with-authenticated-openings-update-unopened
        [OF old])
        (use unopened [OF i-bound] in blast)
  qed
  show ?thesis
    unfolding partial-trace-table-candidate-def
    using len rounds-len agrees by simp
qed

```

```

lemma partial-trace-table-candidates-not-singleton-if-unopened:

```

```

assumes cand: partial-trace-table-candidate trace-table trace-openings
and j-bound:  $j < \text{scale} * \text{clength}$ 
and unopened:
   $\bigwedge i \text{ opn. } i < \text{rounds} \implies \text{opn} \in \text{set } (\text{trace-openings} ! i) \implies$ 
     $\text{opening-index } \text{opn} \neq j$ 
shows
   $\neg (\exists \text{table.}$ 
     $\text{partial-trace-table-candidates } \text{trace-openings} \subseteq \{\text{table}\})$ 
proof
assume unique:
   $\exists \text{table. } \text{partial-trace-table-candidates } \text{trace-openings} \subseteq \{\text{table}\}$ 
let  $? \text{trace-table}' = \text{trace-table}[j := \text{trace-table} ! j + 1]$ 
have cand-in:
   $\text{trace-table} \in \text{partial-trace-table-candidates } \text{trace-openings}$ 
using cand unfolding partial-trace-table-candidates-def by simp
have cand'-in:
   $? \text{trace-table}' \in \text{partial-trace-table-candidates } \text{trace-openings}$ 
using partial-trace-table-candidate-update-unopened [OF cand unopened]
unfolding partial-trace-table-candidates-def by simp
from unique obtain table where subset:
   $\text{partial-trace-table-candidates } \text{trace-openings} \subseteq \{\text{table}\}$ 
by blast
have trace-eq:  $\text{trace-table} = \text{table}$ 
using subset cand-in by blast
have trace'-eq:  $? \text{trace-table}' = \text{table}$ 
using subset cand'-in by blast
have len:  $\text{length } \text{trace-table} = \text{scale} * \text{clength}$ 
using cand unfolding partial-trace-table-candidate-def by simp
have changed:  $? \text{trace-table}' ! j \neq \text{trace-table} ! j$ 
using j-bound len by simp
show False
using changed trace-eq trace'-eq by simp
qed

lemma partial-trace-table-candidate-eq-if-all-indices-opened:
assumes cand: partial-trace-table-candidate trace-table trace-openings
and cand':
   $\text{partial-trace-table-candidate } \text{trace-table}' \text{ trace-openings}$ 
and cover:
   $\bigwedge j. j < \text{scale} * \text{clength} \implies$ 
     $\exists i < \text{rounds. } \exists \text{opn} \in \text{set } (\text{trace-openings} ! i).$ 
     $\text{opening-index } \text{opn} = j$ 
shows  $\text{trace-table} = \text{trace-table}'$ 
proof (rule nth-equalityI)
show  $\text{length } \text{trace-table} = \text{length } \text{trace-table}'$ 
using cand cand' unfolding partial-trace-table-candidate-def by simp
next
fix j
assume j-bound:  $j < \text{length } \text{trace-table}$ 

```



```

have len: length trace-table = scale * clength
  using cand unfolding partial-trace-table-candidate-def by simp
from cover[OF j-bound[unfolded len]]
obtain i opn where i-bound: i < rounds
  and opn-in: opn ∈ set (trace-openings ! i)
  and opn-idx: opening-index opn = j
  by blast
have table-val: trace-table ! j = opening-value opn
  using partial-trace-table-candidateD(?)[OF cand i-bound opn-in]
  opn-idx by simp
have table'-val: trace-table' ! j = opening-value opn
  using partial-trace-table-candidateD(?)[OF cand' i-bound opn-in]
  opn-idx by simp
show trace-table ! j = trace-table' ! j
  using table-val table'-val by simp
qed

lemma partial-trace-table-candidates-subset-singleton-if-all-indices-opened:
  assumes cand: partial-trace-table-candidate trace-table trace-openings
  and cover:
     $\bigwedge j. j < \text{scale} * \text{clength} \implies$ 
     $\exists i < \text{rounds}. \exists \text{opn} \in \text{set} (\text{trace-openings} ! i).$ 
     $\text{opening-index opn} = j$ 
  shows partial-trace-table-candidates trace-openings ⊆ {trace-table}
proof
  fix trace-table'
  assume trace-table' ∈ partial-trace-table-candidates trace-openings
  then have cand':
    partial-trace-table-candidate trace-table' trace-openings
  unfolding partial-trace-table-candidates-def by simp
  have trace-table' = trace-table
  using partial-trace-table-candidate-eq-if-all-indices-opened
    [OF cand' cand cover] .
  then show trace-table' ∈ {trace-table}
  by simp
qed

lemma partial-composition-table-candidate-update-unopened:
  assumes cand:
    partial-composition-table-candidate composition-table
    composition-openings
  and unopened:
     $\bigwedge i \text{ opn}. i < \text{rounds} \implies$ 
     $\text{opn} \in \text{set} (\text{composition-openings} ! i) \implies$ 
     $\text{opening-index opn} \neq j$ 
  shows
    partial-composition-table-candidate
    (composition-table[j := v]) composition-openings
proof –

```

```

have len: length (composition-table[j := v]) = scale * clength
  using cand unfolding partial-composition-table-candidate-def by simp
have rounds-len: length composition-openings = rounds
  using cand unfolding partial-composition-table-candidate-def by simp
have agrees:
   $\bigwedge i. i < rounds \implies$ 
    table-agrees-with-authenticated-openings
      (composition-table[j := v]) (scale * clength)
      (composition-openings ! i)
proof -
  fix i
  assume i-bound: i < rounds
  have old:
    table-agrees-with-authenticated-openings composition-table
      (scale * clength) (composition-openings ! i)
  using cand i-bound
  unfolding partial-composition-table-candidate-def by blast
  show
    table-agrees-with-authenticated-openings
      (composition-table[j := v]) (scale * clength)
      (composition-openings ! i)
  by (rule table-agrees-with-authenticated-openings-update-unopened
      [OF old])
    (use unopened[OF i-bound] in blast)
qed
show ?thesis
  unfolding partial-composition-table-candidate-def
  using len rounds-len agrees by simp
qed

lemma partial-composition-table-candidates-not-singleton-if-unopened:
  assumes cand:
    partial-composition-table-candidate composition-table
      composition-openings
  and j-bound: j < scale * clength
  and unopened:
     $\bigwedge i \text{ opn. } i < rounds \implies$ 
      opn  $\in$  set (composition-openings ! i)  $\implies$ 
      opening-index opn  $\neq$  j
  shows
     $\neg (\exists \text{ table.}$ 
      partial-composition-table-candidates composition-openings  $\subseteq$ 
      {table})
proof
  assume unique:
     $\exists \text{ table. } \text{partial-composition-table-candidates composition-openings}$ 
       $\subseteq$  {table}
  let ?composition-table' =
    composition-table[j := composition-table ! j + 1]

```

```

have cand-in:
  composition-table ∈
    partial-composition-table-candidates composition-openings
  using cand unfolding partial-composition-table-candidates-def by simp
have cand'-in:
  ?composition-table' ∈
    partial-composition-table-candidates composition-openings
  using partial-composition-table-candidate-update-unopened[OF cand unopened]
  unfolding partial-composition-table-candidates-def by simp
from unique obtain table where subset:
  partial-composition-table-candidates composition-openings ⊆
    {table}
  by blast
have comp-eq: composition-table = table
  using subset cand-in by blast
have comp'-eq: ?composition-table' = table
  using subset cand'-in by blast
have len: length composition-table = scale * clength
  using cand unfolding partial-composition-table-candidate-def by simp
have changed: ?composition-table' ! j ≠ composition-table ! j
  using j-bound len by simp
show False
  using changed comp-eq comp'-eq by simp
qed

```

lemma *partial-composition-table-candidate-eq-if-all-indices-opened:*

```

assumes cand:
  partial-composition-table-candidate composition-table
  composition-openings
and cand':
  partial-composition-table-candidate composition-table'
  composition-openings
and cover:
   $\bigwedge j. j < \text{scale} * \text{clength} \implies$ 
     $\exists i < \text{rounds}. \exists \text{opn} \in \text{set } (\text{composition-openings } ! i).$ 
     $\text{opening-index } \text{opn} = j$ 
shows composition-table = composition-table'
proof (rule nth-equalityI)
  show length composition-table = length composition-table'
    using cand cand'
    unfolding partial-composition-table-candidate-def by simp
next
  fix j
  assume j-bound: j < length composition-table
  have len: length composition-table = scale * clength
    using cand unfolding partial-composition-table-candidate-def by simp
  from cover[OF j-bound[unfolded len]]
  obtain i opn where i-bound: i < rounds
    and opn-in: opn ∈ set (composition-openings ! i)

```

```

    and opn-idx: opening-index opn = j
    by blast
  have table-val: composition-table ! j = opening-value opn
    using partial-composition-table-candidateD(3)[OF cand i-bound opn-in]
    opn-idx by simp
  have table'-val: composition-table' ! j = opening-value opn
    using partial-composition-table-candidateD(3)[OF cand' i-bound opn-in]
    opn-idx by simp
  show composition-table ! j = composition-table' ! j
    using table-val table'-val by simp
qed

lemma partial-composition-table-candidates-subset-singleton-if-all-indices-opened:
  assumes cand:
    partial-composition-table-candidate composition-table
    composition-openings
  and cover:
     $\bigwedge j. j < \text{scale} * \text{clength} \implies$ 
     $\exists i < \text{rounds}. \exists \text{opn} \in \text{set} (\text{composition-openings} ! i).$ 
    opening-index opn = j
  shows
    partial-composition-table-candidates composition-openings  $\subseteq$ 
    {composition-table}
proof
  fix composition-table'
  assume
    composition-table'  $\in$ 
    partial-composition-table-candidates composition-openings
  then have cand':
    partial-composition-table-candidate composition-table'
    composition-openings
  unfolding partial-composition-table-candidates-def by simp
  have composition-table' = composition-table
    using partial-composition-table-candidate-eq-if-all-indices-opened
    [OF cand' cand cover] .
  then show composition-table'  $\in$  {composition-table}
    by simp
qed

end

end

theory Soundness-Query-Partial-Candidate-Blocker
  imports
    Soundness-Query-Header-Cross-Candidate
    Soundness-Partial-Candidate-Blocker
begin

```

Exact query-header version of the partial-candidate non-uniqueness diagnostic. These lemmas show why the old singleton-candidate route cannot be applied to the broad partial-opening candidate relation.

context *soundness*
begin

lemma *sparse-openings-obtain-unopened-index*:
assumes *rounds-len*: $\text{length openings} = \text{rounds}$
and *sparse*: $\text{length (List.concat openings)} < \text{scale} * \text{clength}$
obtains *j* **where**
 $j < \text{scale} * \text{clength}$
 $\bigwedge i \text{ opn. } i < \text{rounds} \implies \text{opn} \in \text{set (openings ! } i) \implies$
 $\text{opening-index opn} \neq j$
proof –
let *?N* = $\text{scale} * \text{clength}$
let *?I* = $\text{opening-index 'set (List.concat openings)}$
have *finite-I*: *finite* *?I*
by *simp*
have *card-I-le*: $\text{card } ?I \leq \text{length (List.concat openings)}$
proof –
have $\text{card } ?I \leq \text{card (set (List.concat openings))}$
by (*rule card-image-le*) *simp*
also have $\dots \leq \text{length (List.concat openings)}$
by (*rule card-length*)
finally show *?thesis* .
qed
have *not-all*: $\neg \{0..<?N\} \subseteq ?I$
proof
assume *subset*: $\{0..<?N\} \subseteq ?I$
have *?N* = $\text{card } \{0..<?N\}$
by *simp*
also have $\dots \leq \text{card } ?I$
by (*rule card-mono[OF finite-I subset]*)
also have $\dots \leq \text{length (List.concat openings)}$
by (*rule card-I-le*)
finally show *False*
using *sparse* **by** *simp*
qed
from *not-all* **obtain** *j* **where** *j-in*: $j \in \{0..<?N\}$ **and** *j-notin*: $j \notin ?I$
by *blast*
then have *j-bound*: $j < ?N$
by *simp*
have *unopened*:
 $\bigwedge i \text{ opn. } i < \text{rounds} \implies \text{opn} \in \text{set (openings ! } i) \implies$
 $\text{opening-index opn} \neq j$
proof –
fix *i opn*
assume *i-bound*: $i < \text{rounds}$
and *opn-in*: $\text{opn} \in \text{set (openings ! } i)$

```

have openings ! i ∈ set openings
  using i-bound rounds-len by simp
then have opn ∈ set (List.concat openings)
  using opn-in by auto
then show opening-index opn ≠ j
  using j-notin by auto
qed
show ?thesis
  by (rule that[OF j-bound unopened])
qed

lemma accepted-with-partial-trace-openings-concat-length:
  assumes partial:
    accepted-with-partial-trace-openings s out fr query-idxs trace-openings
  shows length (List.concat trace-openings) = rounds * powers
proof -
  have len: length trace-openings = rounds
    by (rule accepted-with-partial-trace-openings-shapes(2)[OF partial])
  have entry-len:
     $\bigwedge xs. xs \in \text{set trace-openings} \implies \text{length } xs = \text{powers}$ 
  proof -
    fix xs
    assume xs-in: xs ∈ set trace-openings
    then obtain i where i-bound: i < length trace-openings
      and xs-eq: xs = trace-openings ! i
      by (auto simp: in-set-conv-nth)
    have idxs:
      map opening-index (trace-openings ! i) =
        powers-scaled (query-idxs ! i)
      by (rule accepted-with-partial-trace-openings-shapes(3)[OF partial])
        (use i-bound len in simp)
    have len-xs: length (map opening-index xs) =
      length (powers-scaled (query-idxs ! i))
      using idxs xs-eq by simp
    show length xs = powers
      using len-xs unfolding powers-scaled-def by simp
  qed
  have map-len: map length trace-openings = replicate rounds powers
  proof (rule nth-equalityI)
    show length (map length trace-openings) =
      length (replicate rounds powers)
      using len by simp
  next
    fix i
    assume i-bound: i < length (map length trace-openings)
    have trace-openings ! i ∈ set trace-openings
      by (rule nth-mem) (use i-bound in simp)
    then show
      map length trace-openings ! i = replicate rounds powers ! i

```

```

    using entry-len[of trace-openings ! i] i-bound len by simp
qed
show ?thesis
proof -
  have length (List.concat trace-openings) =
    sum-list (map length trace-openings)
    by (induction trace-openings) simp-all
  also have ... = sum-list (replicate rounds powers)
    using map-len by simp
  also have ... = rounds * powers
    by (induction rounds) simp-all
  finally show ?thesis .
qed
qed

lemma accepted-with-partial-composition-openings-concat-length:
  assumes partial:
    accepted-with-partial-composition-openings s out composition-root
    query-idxs composition-openings
  shows length (List.concat composition-openings) = rounds * 2
proof -
  have len: length composition-openings = rounds
    by (rule accepted-with-partial-composition-openings-shapes(2)
      [OF partial])
  have entry-len:
     $\bigwedge xs. xs \in \text{set } \text{composition-openings} \implies \text{length } xs = 2$ 
  proof -
    fix xs
    assume xs-in:  $xs \in \text{set } \text{composition-openings}$ 
    then obtain i where i-bound:  $i < \text{length } \text{composition-openings}$ 
      and xs-eq:  $xs = \text{composition-openings} ! i$ 
    by (auto simp: in-set-conv-nth)
    have idxs:
      map opening-index (composition-openings ! i) =
        [query-idxs ! i,
         fri-sibling-index (scale * clength) (query-idxs ! i)]
    by (rule accepted-with-partial-composition-openings-shapes(3)
      [OF partial])
      (use i-bound len in simp)
    have len-xs: length (map opening-index xs) = length
      [query-idxs ! i,
       fri-sibling-index (scale * clength) (query-idxs ! i)]
    using idxs xs-eq by simp
    show length xs = 2
    using len-xs by simp
  qed
  have map-len: map length composition-openings = replicate rounds 2
proof (rule nth-equalityI)
  show length (map length composition-openings) =

```

```

    length (replicate rounds 2)
    using len by simp
next
fix i
assume i-bound: i < length (map length composition-openings)
have composition-openings ! i ∈ set composition-openings
  by (rule nth-mem) (use i-bound in simp)
then show
  map length composition-openings ! i = replicate rounds 2 ! i
  using entry-len[of composition-openings ! i] i-bound len by simp
qed
show ?thesis
proof -
  have length (List.concat composition-openings) =
    sum-list (map length composition-openings)
    by (induction composition-openings) simp-all
  also have ... = sum-list (replicate rounds 2)
    using map-len by simp
  also have ... = rounds * 2
    by (induction rounds) simp-all
  finally show ?thesis .
qed
qed

lemma query-header-supported-partial-table-candidates-update-trace-unopened:
  assumes comp-nonempty: composition-fri-roots ≠ []
  and outcome:
    Some (result, final-state) ∈ set-dist (execute verify-monad s)
  and trace-partial:
    accepted-with-partial-trace-openings s
    (Some (result, final-state)) fr trace-query-idxs trace-openings
  and trace-candidate:
    partial-trace-table-candidate trace-table trace-openings
  and comp-partial:
    accepted-with-partial-composition-openings s
    (Some (result, final-state)) (hd composition-fri-roots)
    composition-query-idxs composition-openings
  and comp-candidate:
    partial-composition-table-candidate composition-table
    composition-openings
  and header:
    verifier-header-transcript s fr f-fri-roots f-final as dg
    composition-fri-roots final rest
  and unopened:
     $\bigwedge i \text{ opn. } i < \text{rounds} \implies \text{opn} \in \text{set } (\text{trace-openings ! } i) \implies$ 
    opening-index opn ≠ j
  shows
    (trace-table[j := v], composition-table) ∈
    query-header-supported-partial-table-candidates s fr f-fri-roots

```


f-final as dg composition-fri-roots final

proof –

have *trace-candidate'*:
partial-trace-table-candidate (*trace-table*[*j* := *v*]) *trace-openings*
by (*rule partial-trace-table-candidate-update-unopened*
[*OF trace-candidate unopened*])
show *?thesis*
unfolding *query-header-supported-partial-table-candidates-def*
using *comp-nonempty outcome trace-partial trace-candidate'*
comp-partial comp-candidate header
by *blast*

qed

lemma *query-header-supported-partial-table-candidates-not-singleton-if-trace-unopened*:
assumes *comp-nonempty: composition-fri-roots* $\neq \square$
and *outcome*:
Some (result, final-state) $\in$ *set-dist (execute verify-monad s)*
and *trace-partial*:
accepted-with-partial-trace-openings s
(Some (result, final-state)) fr trace-query-idxs trace-openings
and *trace-candidate*:
partial-trace-table-candidate trace-table trace-openings
and *comp-partial*:
accepted-with-partial-composition-openings s
(Some (result, final-state)) (hd composition-fri-roots)
composition-query-idxs composition-openings
and *comp-candidate*:
partial-composition-table-candidate composition-table
composition-openings
and *header*:
verifier-header-transcript s fr f-fri-roots f-final as dg
composition-fri-roots final rest
and *j-bound: j < scale * clength*
and *unopened*:
 $\bigwedge i \text{ opn. } i < \text{rounds} \implies \text{opn} \in \text{set } (\text{trace-openings } ! i) \implies$
opening-index opn $\neq j$

shows
 $\neg (\exists \text{ trace-table0 } \text{composition-table0.}$
query-header-supported-partial-table-candidates s fr f-fri-roots
f-final as dg composition-fri-roots final $\subseteq$
 $\{(\text{trace-table0}, \text{composition-table0})\})$

proof

assume *unique*:
 $\exists \text{ trace-table0 } \text{composition-table0.}$
query-header-supported-partial-table-candidates s fr f-fri-roots
f-final as dg composition-fri-roots final $\subseteq$
 $\{(\text{trace-table0}, \text{composition-table0})\}$

let *?trace-table' = trace-table*[*j* := *trace-table* ! *j* + 1]

have *candidate*:

```

(trace-table, composition-table) ∈
  query-header-supported-partial-table-candidates s fr f-fri-roots
  f-final as dg composition-fri-roots final
unfolding query-header-supported-partial-table-candidates-def
using comp-nonempty outcome trace-partial trace-candidate
  comp-partial comp-candidate header
by blast
have candidate':
  (?trace-table', composition-table) ∈
    query-header-supported-partial-table-candidates s fr f-fri-roots
    f-final as dg composition-fri-roots final
by (rule
  query-header-supported-partial-table-candidates-update-trace-unopened
  [OF comp-nonempty outcome trace-partial trace-candidate comp-partial
    comp-candidate header unopened])
from unique obtain trace-table0 composition-table0 where subset:
  query-header-supported-partial-table-candidates s fr f-fri-roots
  f-final as dg composition-fri-roots final ⊆
  {(trace-table0, composition-table0)}
by blast
have pair-eq:
  (trace-table, composition-table) = (trace-table0, composition-table0)
  using subset candidate by blast
have pair'-eq:
  (?trace-table', composition-table) = (trace-table0, composition-table0)
  using subset candidate' by blast
have len: length trace-table = scale * clength
  using trace-candidate unfolding partial-trace-table-candidate-def by simp
have changed: ?trace-table' ! j ≠ trace-table ! j
  using j-bound len by simp
show False
  using pair-eq pair'-eq changed by simp
qed

lemma query-header-supported-partial-table-candidates-not-singleton-if-trace-sparse:
assumes comp-nonempty: composition-fri-roots ≠ []
and outcome:
  Some (result, final-state) ∈ set-dist (execute verify-monad s)
and trace-partial:
  accepted-with-partial-trace-openings s
  (Some (result, final-state)) fr trace-query-idxs trace-openings
and trace-candidate:
  partial-trace-table-candidate trace-table trace-openings
and comp-partial:
  accepted-with-partial-composition-openings s
  (Some (result, final-state)) (hd composition-fri-roots)
  composition-query-idxs composition-openings
and comp-candidate:
  partial-composition-table-candidate composition-table

```

```

    composition-openings
  and header:
    verifier-header-transcript s fr f-fri-roots f-final as dg
    composition-fri-roots final rest
  and sparse: length (List.concat trace-openings) < scale * clength
shows
  ¬ (∃ trace-table0 composition-table0.
    query-header-supported-partial-table-candidates s fr f-fri-roots
    f-final as dg composition-fri-roots final ⊆
    {(trace-table0, composition-table0)})
proof -
  have rounds-len: length trace-openings = rounds
  using trace-candidate unfolding partial-trace-table-candidate-def by simp
  obtain j where j-bound: j < scale * clength
  and unopened:
    ∧ i opn. i < rounds ⇒ opn ∈ set (trace-openings ! i) ⇒
      opening-index opn ≠ j
  using sparse-openings-obtain-unopened-index[OF rounds-len sparse]
  by blast
  show ?thesis
  by (rule
    query-header-supported-partial-table-candidates-not-singleton-if-trace-unopened
    [OF comp-nonempty outcome trace-partial trace-candidate comp-partial
      comp-candidate header j-bound unopened])
qed

```

lemma *query-header-supported-partial-pairwise-cross-or-merkle-bad-if-trace-sparse:*

```

  assumes comp-nonempty: composition-fri-roots ≠ []
  and outcome:
    Some (result, final-state) ∈ set-dist (execute verify-monad s)
  and trace-partial:
    accepted-with-partial-trace-openings s
    (Some (result, final-state)) fr trace-query-idxs trace-openings
  and trace-candidate:
    partial-trace-table-candidate trace-table trace-openings
  and comp-partial:
    accepted-with-partial-composition-openings s
    (Some (result, final-state)) (hd composition-fri-roots)
    composition-query-idxs composition-openings
  and comp-candidate:
    partial-composition-table-candidate composition-table
    composition-openings
  and header:
    verifier-header-transcript s fr f-fri-roots f-final as dg
    composition-fri-roots final rest
  and sparse: length (List.concat trace-openings) < scale * clength
shows
  query-header-supported-partial-pairwise-cross-or-merkle-bad s fr
  f-fri-roots f-final as dg composition-fri-roots final

```

proof –

have *not-unique*:

¬ (∃ *trace-table0* *composition-table0*.

query-header-supported-partial-table-candidates *s* fr *f-fri-roots*

f-final as *dg* *composition-fri-roots* *final* ⊆

{(*trace-table0*, *composition-table0*)})

by (*rule*

query-header-supported-partial-table-candidates-not-singleton-if-trace-sparse

[*OF comp-nonempty outcome trace-partial trace-candidate comp-partial*

comp-candidate header sparse])

show *?thesis*

by (*rule*

query-header-supported-partial-pairwise-cross-or-merkle-bad-if-not-singleton

[*OF not-unique*])

qed

lemma *query-header-supported-partial-table-candidates-not-singleton-if-trace-query-sparse*:

assumes *comp-nonempty*: *composition-fri-roots* ≠ []

and *outcome*:

Some (*result*, *final-state*) ∈ *set-dist* (*execute verify-monad s*)

and *trace-partial*:

accepted-with-partial-trace-openings *s*

(*Some* (*result*, *final-state*)) fr *trace-query-idxs* *trace-openings*

and *trace-candidate*:

partial-trace-table-candidate *trace-table* *trace-openings*

and *comp-partial*:

accepted-with-partial-composition-openings *s*

(*Some* (*result*, *final-state*)) (*hd* *composition-fri-roots*)

composition-query-idxs *composition-openings*

and *comp-candidate*:

partial-composition-table-candidate *composition-table*

composition-openings

and *header*:

verifier-header-transcript *s* fr *f-fri-roots* *f-final* as *dg*

composition-fri-roots *final* *rest*

and *sparse*: *rounds* * *powers* < *scale* * *clength*

shows

¬ (∃ *trace-table0* *composition-table0*.

query-header-supported-partial-table-candidates *s* fr *f-fri-roots*

f-final as *dg* *composition-fri-roots* *final* ⊆

{(*trace-table0*, *composition-table0*)})

proof –

have *concat-sparse*:

length (*List.concat* *trace-openings*) < *scale* * *clength*

using *accepted-with-partial-trace-openings-concat-length*[*OF trace-partial*]

sparse **by** *simp*

show *?thesis*

by (*rule*

query-header-supported-partial-table-candidates-not-singleton-if-trace-sparse

[OF comp-nonempty outcome trace-partial trace-candidate comp-partial
comp-candidate header concat-sparse])

qed

lemma *query-header-supported-partial-pairwise-cross-or-merkle-bad-if-trace-query-sparse:*
assumes *comp-nonempty: composition-fri-roots $\neq \square$*
and *outcome:*
Some (result, final-state) $\in$ set-dist (execute verify-monad s)
and *trace-partial:*
accepted-with-partial-trace-openings s
(Some (result, final-state)) fr trace-query-idxs trace-openings
and *trace-candidate:*
partial-trace-table-candidate trace-table trace-openings
and *comp-partial:*
accepted-with-partial-composition-openings s
(Some (result, final-state)) (hd composition-fri-roots)
composition-query-idxs composition-openings
and *comp-candidate:*
partial-composition-table-candidate composition-table
composition-openings
and *header:*
verifier-header-transcript s fr f-fri-roots f-final as dg
composition-fri-roots final rest
and *sparse: rounds * powers < scale * clength*
shows
query-header-supported-partial-pairwise-cross-or-merkle-bad s fr
f-fri-roots f-final as dg composition-fri-roots final
proof –
have *not-unique:*
 $\neg (\exists \text{ trace-table0 } \text{composition-table0}.$
query-header-supported-partial-table-candidates s fr f-fri-roots
f-final as dg composition-fri-roots final $\subseteq$
{(trace-table0, composition-table0)}))
by (rule
query-header-supported-partial-table-candidates-not-singleton-if-trace-query-sparse
[OF comp-nonempty outcome trace-partial trace-candidate comp-partial
comp-candidate header sparse])
show *?thesis*
by (rule
query-header-supported-partial-pairwise-cross-or-merkle-bad-if-not-singleton
[OF not-unique])
qed

lemma *query-header-supported-partial-table-candidates-update-composition-unopened:*
assumes *comp-nonempty: composition-fri-roots $\neq \square$*
and *outcome:*
Some (result, final-state) $\in$ set-dist (execute verify-monad s)
and *trace-partial:*
accepted-with-partial-trace-openings s

```

    (Some (result, final-state)) fr trace-query-idxs trace-openings
and trace-candidate:
    partial-trace-table-candidate trace-table trace-openings
and comp-partial:
    accepted-with-partial-composition-openings s
    (Some (result, final-state)) (hd composition-fri-roots)
    composition-query-idxs composition-openings
and comp-candidate:
    partial-composition-table-candidate composition-table
    composition-openings
and header:
    verifier-header-transcript s fr f-fri-roots f-final as dg
    composition-fri-roots final rest
and unopened:
     $\bigwedge i \text{ opn. } i < \text{rounds} \implies$ 
     $\text{opn} \in \text{set } (\text{composition-openings} ! i) \implies$ 
     $\text{opening-index opn} \neq j$ 
shows
    (trace-table, composition-table[j := v])  $\in$ 
    query-header-supported-partial-table-candidates s fr f-fri-roots
    f-final as dg composition-fri-roots final
proof –
    have comp-candidate':
    partial-composition-table-candidate (composition-table[j := v])
    composition-openings
    by (rule partial-composition-table-candidate-update-unopened
    [OF comp-candidate unopened])
    show ?thesis
    unfolding query-header-supported-partial-table-candidates-def
    using comp-nonempty outcome trace-partial trace-candidate
    comp-partial comp-candidate' header
    by blast
qed

```

```

lemma query-header-supported-partial-table-candidates-not-singleton-if-composition-unopened:
assumes comp-nonempty: composition-fri-roots  $\neq []$ 
and outcome:
    Some (result, final-state)  $\in$  set-dist (execute verify-monad s)
and trace-partial:
    accepted-with-partial-trace-openings s
    (Some (result, final-state)) fr trace-query-idxs trace-openings
and trace-candidate:
    partial-trace-table-candidate trace-table trace-openings
and comp-partial:
    accepted-with-partial-composition-openings s
    (Some (result, final-state)) (hd composition-fri-roots)
    composition-query-idxs composition-openings
and comp-candidate:
    partial-composition-table-candidate composition-table

```

```

    composition-openings
and header:
    verifier-header-transcript s fr f-fri-roots f-final as dg
    composition-fri-roots final rest
and j-bound: j < scale * clength
and unopened:
     $\bigwedge i \text{ opn. } i < \text{rounds} \implies$ 
    opn  $\in \text{set } (\text{composition-openings } ! i) \implies$ 
    opening-index opn  $\neq j$ 
shows
     $\neg (\exists \text{ trace-table0 composition-table0.}$ 
    query-header-supported-partial-table-candidates s fr f-fri-roots
    f-final as dg composition-fri-roots final  $\subseteq$ 
     $\{(\text{trace-table0}, \text{composition-table0})\})$ 
proof
assume unique:
     $\exists \text{ trace-table0 composition-table0.}$ 
    query-header-supported-partial-table-candidates s fr f-fri-roots
    f-final as dg composition-fri-roots final  $\subseteq$ 
     $\{(\text{trace-table0}, \text{composition-table0})\}$ 
let ?composition-table' =
    composition-table[j := composition-table ! j + 1]
have candidate:
    (trace-table, composition-table)  $\in$ 
    query-header-supported-partial-table-candidates s fr f-fri-roots
    f-final as dg composition-fri-roots final
unfolding query-header-supported-partial-table-candidates-def
using comp-nonempty outcome trace-partial trace-candidate
    comp-partial comp-candidate header
by blast
have candidate':
    (trace-table, ?composition-table')  $\in$ 
    query-header-supported-partial-table-candidates s fr f-fri-roots
    f-final as dg composition-fri-roots final
by (rule
    query-header-supported-partial-table-candidates-update-composition-unopened
    [OF comp-nonempty outcome trace-partial trace-candidate comp-partial
    comp-candidate header unopened])
from unique obtain trace-table0 composition-table0 where subset:
    query-header-supported-partial-table-candidates s fr f-fri-roots
    f-final as dg composition-fri-roots final  $\subseteq$ 
     $\{(\text{trace-table0}, \text{composition-table0})\}$ 
by blast
have pair-eq:
    (trace-table, composition-table) = (trace-table0, composition-table0)
using subset candidate by blast
have pair'-eq:
    (trace-table, ?composition-table') =
    (trace-table0, composition-table0)

```

```

    using subset candidate' by blast
have len: length composition-table = scale * clength
    using comp-candidate
    unfolding partial-composition-table-candidate-def by simp
have changed: ?composition-table' ! j ≠ composition-table ! j
    using j-bound len by simp
show False
    using pair-eq pair'-eq changed by simp
qed

lemma query-header-supported-partial-table-candidates-not-singleton-if-composition-sparse:
  assumes comp-nonempty: composition-fri-roots ≠ []
  and outcome:
    Some (result, final-state) ∈ set-dist (execute verify-monad s)
  and trace-partial:
    accepted-with-partial-trace-openings s
    (Some (result, final-state)) fr trace-query-idxs trace-openings
  and trace-candidate:
    partial-trace-table-candidate trace-table trace-openings
  and comp-partial:
    accepted-with-partial-composition-openings s
    (Some (result, final-state)) (hd composition-fri-roots)
    composition-query-idxs composition-openings
  and comp-candidate:
    partial-composition-table-candidate composition-table
    composition-openings
  and header:
    verifier-header-transcript s fr f-fri-roots f-final as dg
    composition-fri-roots final rest
  and sparse:
    length (List.concat composition-openings) < scale * clength
  shows
    ¬ (∃ trace-table0 composition-table0.
      query-header-supported-partial-table-candidates s fr f-fri-roots
      f-final as dg composition-fri-roots final ⊆
      {(trace-table0, composition-table0)})
proof -
  have rounds-len: length composition-openings = rounds
  using comp-candidate
  unfolding partial-composition-table-candidate-def by simp
  obtain j where j-bound: j < scale * clength
  and unopened:
    ∧ i opn. i < rounds ⇒
      opn ∈ set (composition-openings ! i) ⇒
      opening-index opn ≠ j
  using sparse-openings-obtain-unopened-index[OF rounds-len sparse]
  by blast
  show ?thesis
  by (rule

```


query-header-supported-partial-table-candidates-not-singleton-if-composition-unopened
 [OF comp-nonempty outcome trace-partial trace-candidate comp-partial
 comp-candidate header j-bound unopened])

qed

lemma *query-header-supported-partial-pairwise-cross-or-merkle-bad-if-composition-sparse:*

assumes comp-nonempty: composition-fri-roots $\neq \square$
and outcome:
 Some (result, final-state) $\in$ set-dist (execute verify-monad s)
and trace-partial:
 accepted-with-partial-trace-openings s
 (Some (result, final-state)) fr trace-query-idxs trace-openings
and trace-candidate:
 partial-trace-table-candidate trace-table trace-openings
and comp-partial:
 accepted-with-partial-composition-openings s
 (Some (result, final-state)) (hd composition-fri-roots)
 composition-query-idxs composition-openings
and comp-candidate:
 partial-composition-table-candidate composition-table
 composition-openings
and header:
 verifier-header-transcript s fr f-fri-roots f-final as dg
 composition-fri-roots final rest
and sparse:
 length (List.concat composition-openings) < scale * clength
shows
query-header-supported-partial-pairwise-cross-or-merkle-bad s fr
 f-fri-roots f-final as dg composition-fri-roots final

proof –

have not-unique:
 $\neg (\exists \text{ trace-table0 } \text{ composition-table0}.$
query-header-supported-partial-table-candidates s fr f-fri-roots
 f-final as dg composition-fri-roots final $\subseteq$
 $\{(\text{trace-table0}, \text{composition-table0})\})$
by (rule
query-header-supported-partial-table-candidates-not-singleton-if-composition-sparse
 [OF comp-nonempty outcome trace-partial trace-candidate comp-partial
 comp-candidate header sparse])
show ?thesis
by (rule
query-header-supported-partial-pairwise-cross-or-merkle-bad-if-not-singleton
 [OF not-unique])

qed

lemma *query-header-supported-partial-table-candidates-not-singleton-if-composition-query-sparse:*

assumes comp-nonempty: composition-fri-roots $\neq \square$
and outcome:
 Some (result, final-state) $\in$ set-dist (execute verify-monad s)

and *trace-partial*:
accepted-with-partial-trace-openings s
(Some (result, final-state)) fr trace-query-idxs trace-openings
and *trace-candidate*:
partial-trace-table-candidate trace-table trace-openings
and *comp-partial*:
accepted-with-partial-composition-openings s
(Some (result, final-state)) (hd composition-fri-roots)
composition-query-idxs composition-openings
and *comp-candidate*:
partial-composition-table-candidate composition-table
composition-openings
and *header*:
verifier-header-transcript s fr f-fri-roots f-final as dg
composition-fri-roots final rest
and *sparse*: *rounds * 2 < scale * clength*
shows
 $\neg (\exists \text{ trace-table0 } \text{ composition-table0.}$
query-header-supported-partial-table-candidates s fr f-fri-roots
f-final as dg composition-fri-roots final \subseteq
\{(trace-table0, composition-table0)\})
proof –
have *concat-sparse*:
*length (List.concat composition-openings) < scale * clength*
using *accepted-with-partial-composition-openings-concat-length*
[OF comp-partial] sparse by simp
show *?thesis*
by (rule
query-header-supported-partial-table-candidates-not-singleton-if-composition-sparse
[OF comp-nonempty outcome trace-partial trace-candidate comp-partial
comp-candidate header concat-sparse])
qed

lemma *query-header-supported-partial-pairwise-cross-or-merkle-bad-if-composition-query-sparse*:
assumes *comp-nonempty*: *composition-fri-roots $\neq []$*
and *outcome*:
Some (result, final-state) $\in$ set-dist (execute verify-monad s)
and *trace-partial*:
accepted-with-partial-trace-openings s
(Some (result, final-state)) fr trace-query-idxs trace-openings
and *trace-candidate*:
partial-trace-table-candidate trace-table trace-openings
and *comp-partial*:
accepted-with-partial-composition-openings s
(Some (result, final-state)) (hd composition-fri-roots)
composition-query-idxs composition-openings
and *comp-candidate*:
partial-composition-table-candidate composition-table
composition-openings

```

and header:
  verifier-header-transcript s fr f-fri-roots f-final as dg
  composition-fri-roots final rest
and sparse: rounds * 2 < scale * clength
shows
  query-header-supported-partial-pairwise-cross-or-merkle-bad s fr
  f-fri-roots f-final as dg composition-fri-roots final
proof –
  have not-unique:
    ¬ (∃ trace-table0 composition-table0.
      query-header-supported-partial-table-candidates s fr f-fri-roots
      f-final as dg composition-fri-roots final ⊆
      {(trace-table0, composition-table0)})
  by (rule
    query-header-supported-partial-table-candidates-not-singleton-if-composition-query-sparse
    [OF comp-nonempty outcome trace-partial trace-candidate comp-partial
     comp-candidate header sparse])
  show ?thesis
  by (rule
    query-header-supported-partial-pairwise-cross-or-merkle-bad-if-not-singleton
    [OF not-unique])
qed

end

end

```

```

theory Soundness-Alpha-Header-Candidate-Blocker
imports
  Soundness-Query-Partial-Candidate-Blocker
  Staged-Security-Experiment-Composition
begin

```

Diagnostic lemmas for the alpha-header partial-candidate route.

The staged composition proof uses *alpha-header-supported-partial-trace-table-candidates*. This candidate set is trace-only, but it has the same sparse-opening obstruction as the query-header partial-candidate set: sampled authenticated openings do not determine unopened trace-table positions.

```

context soundness
begin

```

```

definition alpha-header-partial-candidate-not-prefix-bound
  :: 'f protocol-channel ⇒ ('f, 'a) protocol-channel-scheme ⇒
  'f ⇒ 'f list ⇒ 'f ⇒ bool

```

```

where
  alpha-header-partial-candidate-not-prefix-bound prefix-state s fr
  f-fri-roots f-final ⟷
  (∃ trace-table ∈

```

*alpha-header-supported-partial-trace-table-candidates s fr f-fri-roots
f-final.*
trace-table $\notin$ alpha-prefix-trace-table-candidates prefix-state fr)

lemma *alpha-header-partial-candidate-not-prefix-bound-iff-not-subset:*
*alpha-header-partial-candidate-not-prefix-bound prefix-state s fr
f-fri-roots f-final $\longleftrightarrow$*
 $\neg$ *alpha-header-supported-partial-trace-table-candidates s fr
f-fri-roots f-final $\subseteq$*
alpha-prefix-trace-table-candidates prefix-state fr
unfolding *alpha-header-partial-candidate-not-prefix-bound-def* **by** *blast*

lemma *alpha-header-supported-partial-union-bad-sets-subset-prefix-union-if-bound:*
assumes *bound:*
 $\neg$ *alpha-header-partial-candidate-not-prefix-bound prefix-state s fr
f-fri-roots f-final*
shows
*alpha-header-supported-partial-union-bad-sets s bad-sets fr f-fri-roots
f-final $\subseteq$ alpha-prefix-union-bad-sets prefix-state bad-sets fr*
using *bound*
unfolding *alpha-header-partial-candidate-not-prefix-bound-def
alpha-header-supported-partial-union-bad-sets-def
alpha-prefix-union-bad-sets-def*
by *blast*

lemma *alpha-header-partial-union-hit-not-prefix-hit-imp-not-prefix-bound:*
assumes *hit:*
as $\in$
*alpha-header-supported-partial-union-bad-sets s bad-sets fr
f-fri-roots f-final*
and *not-prefix:*
as $\notin$ alpha-prefix-union-bad-sets prefix-state bad-sets fr
shows
*alpha-header-partial-candidate-not-prefix-bound prefix-state s fr
f-fri-roots f-final*
proof (rule *ccontr*)
assume
 $\neg$ *alpha-header-partial-candidate-not-prefix-bound prefix-state s fr
f-fri-roots f-final*
then have
*alpha-header-supported-partial-union-bad-sets s bad-sets fr
f-fri-roots f-final $\subseteq$*
alpha-prefix-union-bad-sets prefix-state bad-sets fr
by (rule *alpha-header-supported-partial-union-bad-sets-subset-prefix-union-if-bound*)
then show *False*
using *hit not-prefix* **by** *blast*
qed

lemma *alpha-header-supported-partial-trace-table-candidates-update-unopened:*

```

assumes outcome:
  Some (result, final-state) ∈ set-dist (execute verify-monad s)
and partial:
  accepted-with-partial-trace-openings s
  (Some (result, final-state)) fr query-idxs trace-openings
and candidate:
  partial-trace-table-candidate trace-table trace-openings
and header:
  verifier-header-transcript s fr f-fri-roots f-final as dg
  composition-fri-roots final rest
and unopened:
   $\bigwedge i \text{ opn. } i < \text{rounds} \implies \text{opn} \in \text{set } (\text{trace-openings } ! i) \implies$ 
  opening-index opn  $\neq j$ 
shows
  trace-table[j := v] ∈
  alpha-header-supported-partial-trace-table-candidates s fr f-fri-roots
  f-final
proof –
  have candidate':
    partial-trace-table-candidate (trace-table[j := v]) trace-openings
  by (rule partial-trace-table-candidate-update-unopened
    [OF candidate unopened])
  show ?thesis
    unfolding alpha-header-supported-partial-trace-table-candidates-def
    using outcome partial candidate' header by blast
qed

lemma alpha-header-supported-partial-trace-table-candidates-not-singleton-if-trace-unopened:
assumes outcome:
  Some (result, final-state) ∈ set-dist (execute verify-monad s)
and partial:
  accepted-with-partial-trace-openings s
  (Some (result, final-state)) fr query-idxs trace-openings
and candidate:
  partial-trace-table-candidate trace-table trace-openings
and header:
  verifier-header-transcript s fr f-fri-roots f-final as dg
  composition-fri-roots final rest
and j-bound: j < scale * clength
and unopened:
   $\bigwedge i \text{ opn. } i < \text{rounds} \implies \text{opn} \in \text{set } (\text{trace-openings } ! i) \implies$ 
  opening-index opn  $\neq j$ 
shows
   $\neg (\exists \text{ trace-table0.}$ 
  alpha-header-supported-partial-trace-table-candidates s fr f-fri-roots
  f-final  $\subseteq \{\text{trace-table0}\})$ 
proof
assume unique:
   $\exists \text{ trace-table0.}$ 

```

```

    alpha-header-supported-partial-trace-table-candidates s fr f-fri-roots
    f-final  $\subseteq \{\text{trace-table0}\}$ 
  let ?trace-table' = trace-table[j := trace-table ! j + 1]
  have candidate-in:
    trace-table  $\in$ 
    alpha-header-supported-partial-trace-table-candidates s fr f-fri-roots
    f-final
  unfolding alpha-header-supported-partial-trace-table-candidates-def
  using outcome partial candidate header by blast
  have candidate'-in:
    ?trace-table'  $\in$ 
    alpha-header-supported-partial-trace-table-candidates s fr f-fri-roots
    f-final
  by (rule
    alpha-header-supported-partial-trace-table-candidates-update-unopened
    [OF outcome partial candidate header unopened])
  from unique obtain trace-table0 where subset:
    alpha-header-supported-partial-trace-table-candidates s fr f-fri-roots
    f-final  $\subseteq \{\text{trace-table0}\}$ 
  by blast
  have trace-eq: trace-table = trace-table0
  using subset candidate-in by blast
  have trace'-eq: ?trace-table' = trace-table0
  using subset candidate'-in by blast
  have len: length trace-table = scale * clength
  using candidate unfolding partial-trace-table-candidate-def by simp
  have changed: ?trace-table' ! j  $\neq$  trace-table ! j
  using j-bound len by simp
  show False
  using changed trace-eq trace'-eq by simp
qed

lemma alpha-header-supported-partial-trace-table-candidates-not-singleton-if-trace-sparse:
  assumes outcome:
    Some (result, final-state)  $\in$  set-dist (execute verify-monad s)
  and partial:
    accepted-with-partial-trace-openings s
    (Some (result, final-state)) fr query-idxs trace-openings
  and candidate:
    partial-trace-table-candidate trace-table trace-openings
  and header:
    verifier-header-transcript s fr f-fri-roots f-final as dg
    composition-fri-roots final rest
  and sparse: length (List.concat trace-openings) < scale * clength
  shows
     $\neg (\exists \text{trace-table0}.$ 
      alpha-header-supported-partial-trace-table-candidates s fr f-fri-roots
      f-final  $\subseteq \{\text{trace-table0}\})$ 
  proof -

```

```

have rounds-len: length trace-openings = rounds
  using candidate unfolding partial-trace-table-candidate-def by simp
obtain j where j-bound: j < scale * clength
and unopened:
   $\bigwedge i \text{ opn. } i < \text{rounds} \implies \text{opn} \in \text{set } (\text{trace-openings } ! i) \implies$ 
    opening-index opn  $\neq$  j
  using sparse-openings-obtain-unopened-index[OF rounds-len sparse]
  by blast
show ?thesis
  by (rule
    alpha-header-supported-partial-trace-table-candidates-not-singleton-if-trace-unopened
    [OF outcome partial candidate header j-bound unopened])
qed

end

end

```

```

theory Soundness-Alpha-Header-Cross-Candidate
imports
  Soundness-Alpha-Header-Candidate-Blocker
  Soundness-Partial-Merkle-Cross
begin

```

Witnessed alpha-header partial trace candidates.

This layer keeps verifier outcomes and authenticated opening witnesses explicit. It proves the trace-only analogue of the query-header cross-candidate decomposition: if witnessed candidates are not singleton, then either pairwise opening coverage fails or a Merkle/collision side condition occurs.

```

context soundness
begin

```

definition alpha-header-supported-partial-trace-table-candidate-witness

```

:: ('f, 'a) protocol-channel-scheme  $\Rightarrow$ 
  'f  $\Rightarrow$  'f list  $\Rightarrow$  'f  $\Rightarrow$  'f list  $\Rightarrow$ 
  ('f, 'a) protocol-channel-scheme  $\Rightarrow$ 
  'f authenticated-opening list list  $\Rightarrow$  bool

```

where

```

alpha-header-supported-partial-trace-table-candidate-witness s fr
  f-fri-roots f-final trace-table final-state trace-openings  $\longleftrightarrow$ 
  ( $\exists$  result query-idxs as dg composition-fri-roots final rest.
    Some (result, final-state)  $\in$  set-dist (execute verify-monad s)  $\wedge$ 
    accepted-with-partial-trace-openings s
    (Some (result, final-state)) fr query-idxs trace-openings  $\wedge$ 
    partial-trace-table-candidate trace-table trace-openings  $\wedge$ 
    verifier-header-transcript s fr f-fri-roots f-final as dg
    composition-fri-roots final rest)

```

definition *alpha-header-supported-witnessed-partial-trace-table-candidates*

$:: ('f, 'a) \text{ protocol-channel-scheme} \Rightarrow$
 $'f \Rightarrow 'f \text{ list} \Rightarrow 'f \Rightarrow 'f \text{ list set}$

where

alpha-header-supported-witnessed-partial-trace-table-candidates $s \text{ fr}$
 $f\text{-fri-roots } f\text{-final} =$
 $\{ \text{trace-table.}$
 $\exists \text{ final-state trace-openings.}$
 $\text{alpha-header-supported-partial-trace-table-candidate-witness } s \text{ fr}$
 $f\text{-fri-roots } f\text{-final trace-table final-state trace-openings} \}$

definition *alpha-header-supported-witnessed-partial-union-bad-sets*

$:: ('f, 'a) \text{ protocol-channel-scheme} \Rightarrow$
 $('f \text{ list} \Rightarrow 'f \text{ list set}) \Rightarrow$
 $'f \Rightarrow 'f \text{ list} \Rightarrow 'f \Rightarrow 'f \text{ list set}$

where

alpha-header-supported-witnessed-partial-union-bad-sets $s \text{ bad-sets fr}$
 $f\text{-fri-roots } f\text{-final} =$
 $\{ as \in \text{alpha-space.}$
 $\exists \text{ trace-table} \in$
 $\text{alpha-header-supported-witnessed-partial-trace-table-candidates } s \text{ fr}$
 $f\text{-fri-roots } f\text{-final.}$
 $as \in \text{bad-sets trace-table} \}$

lemma *alpha-header-supported-partial-trace-table-candidate-witness-imp-candidate:*

assumes

alpha-header-supported-partial-trace-table-candidate-witness $s \text{ fr}$
 $f\text{-fri-roots } f\text{-final trace-table final-state trace-openings}$

shows

$\text{trace-table} \in$
 $\text{alpha-header-supported-partial-trace-table-candidates } s \text{ fr } f\text{-fri-roots}$
 $f\text{-final}$

using *assms*

unfolding *alpha-header-supported-partial-trace-table-candidate-witness-def*

alpha-header-supported-partial-trace-table-candidates-def

by *blast*

lemma *alpha-header-supported-witnessed-candidates-subset-candidates:*

alpha-header-supported-witnessed-partial-trace-table-candidates $s \text{ fr}$
 $f\text{-fri-roots } f\text{-final} \subseteq$
 $\text{alpha-header-supported-partial-trace-table-candidates } s \text{ fr } f\text{-fri-roots}$
 $f\text{-final}$

unfolding *alpha-header-supported-witnessed-partial-trace-table-candidates-def*

using *alpha-header-supported-partial-trace-table-candidate-witness-imp-candidate*

by *blast*

lemma *alpha-header-supported-partial-trace-table-candidates-subset-witnessed-candidates:*

alpha-header-supported-partial-trace-table-candidates $s \text{ fr } f\text{-fri-roots}$
 $f\text{-final} \subseteq$

$\alpha\text{-header-supported-witnessed-partial-trace-table-candidates } s \text{ fr } f\text{-fri-roots } f\text{-final}$
unfolding $\alpha\text{-header-supported-partial-trace-table-candidates-def}$
 $\alpha\text{-header-supported-witnessed-partial-trace-table-candidates-def}$
 $\alpha\text{-header-supported-partial-trace-table-candidate-witness-def}$
 $\text{accepted-with-partial-trace-openings-def}$
by *blast*

lemma $\alpha\text{-header-supported-witnessed-candidates-eq-candidates:}$
 $\alpha\text{-header-supported-witnessed-partial-trace-table-candidates } s \text{ fr } f\text{-fri-roots } f\text{-final} =$
 $\alpha\text{-header-supported-partial-trace-table-candidates } s \text{ fr } f\text{-fri-roots } f\text{-final}$
using $\alpha\text{-header-supported-witnessed-candidates-subset-candidates}$
 $\alpha\text{-header-supported-partial-trace-table-candidates-subset-witnessed-candidates}$
by *blast*

lemma $\alpha\text{-header-supported-witnessed-union-bad-sets-subset-union-bad-sets:}$
 $\alpha\text{-header-supported-witnessed-partial-union-bad-sets } s \text{ bad-sets fr } f\text{-fri-roots } f\text{-final} \subseteq$
 $\alpha\text{-header-supported-partial-union-bad-sets } s \text{ bad-sets fr } f\text{-fri-roots } f\text{-final}$
unfolding $\alpha\text{-header-supported-witnessed-partial-union-bad-sets-def}$
 $\alpha\text{-header-supported-partial-union-bad-sets-def}$
using $\alpha\text{-header-supported-witnessed-candidates-subset-candidates}$
by *blast*

lemma $\alpha\text{-header-supported-union-bad-sets-subset-witnessed-union-bad-sets:}$
 $\alpha\text{-header-supported-partial-union-bad-sets } s \text{ bad-sets fr } f\text{-fri-roots } f\text{-final} \subseteq$
 $\alpha\text{-header-supported-witnessed-partial-union-bad-sets } s \text{ bad-sets fr } f\text{-fri-roots } f\text{-final}$
unfolding $\alpha\text{-header-supported-witnessed-partial-union-bad-sets-def}$
 $\alpha\text{-header-supported-partial-union-bad-sets-def}$
using $\alpha\text{-header-supported-partial-trace-table-candidates-subset-witnessed-candidates}$
by *blast*

lemma $\alpha\text{-header-supported-witnessed-union-bad-sets-eq-union-bad-sets:}$
 $\alpha\text{-header-supported-witnessed-partial-union-bad-sets } s \text{ bad-sets fr } f\text{-fri-roots } f\text{-final} =$
 $\alpha\text{-header-supported-partial-union-bad-sets } s \text{ bad-sets fr } f\text{-fri-roots } f\text{-final}$
using $\alpha\text{-header-supported-witnessed-union-bad-sets-subset-union-bad-sets}$
 $\alpha\text{-header-supported-union-bad-sets-subset-witnessed-union-bad-sets}$
by *blast*

lemma $\alpha\text{-header-partial-candidate-not-prefix-bound-obtain-witness:}$
assumes
 $\alpha\text{-header-partial-candidate-not-prefix-bound prefix-state } s \text{ fr}$

f-fri-roots f-final
obtains *trace-table final-state trace-openings* **where**
alpha-header-supported-partial-trace-table-candidate-witness s fr
f-fri-roots f-final trace-table final-state trace-openings
trace-table ∉ alpha-prefix-trace-table-candidates prefix-state fr
using *assms*
unfolding *alpha-header-partial-candidate-not-prefix-bound-def*
using *alpha-header-supported-partial-trace-table-candidates-subset-witnessed-candidates*
unfolding *alpha-header-supported-witnessed-partial-trace-table-candidates-def*
by *blast*

lemma *alpha-header-supported-witnessed-partial-trace-table-candidates-obtain-witness:*
assumes
trace-table ∈
alpha-header-supported-witnessed-partial-trace-table-candidates s fr
f-fri-roots f-final
obtains *final-state trace-openings* **where**
alpha-header-supported-partial-trace-table-candidate-witness s fr
f-fri-roots f-final trace-table final-state trace-openings
using *assms*
unfolding *alpha-header-supported-witnessed-partial-trace-table-candidates-def*
by *blast*

lemma *alpha-header-supported-partial-trace-table-candidate-witness-shape:*
assumes
alpha-header-supported-partial-trace-table-candidate-witness s fr
f-fri-roots f-final trace-table final-state trace-openings
shows *partial-trace-table-candidate trace-table trace-openings*
using *assms*
unfolding *alpha-header-supported-partial-trace-table-candidate-witness-def*
by *blast*

lemma *alpha-header-supported-partial-trace-table-candidate-witness-eq-if-cross-cover-clean-merge:*
assumes *witness:*
alpha-header-supported-partial-trace-table-candidate-witness s fr
f-fri-roots f-final trace-table final-state trace-openings
and *witness':*
alpha-header-supported-partial-trace-table-candidate-witness s fr
f-fri-roots f-final trace-table' final-state' trace-openings'
and *trace-cover:*
partial-openings-cross-cover-all-indices trace-openings
trace-openings'
and *no-conflict:*
 $\neg$ *merkle-hash-value-conflict final-state final-state'*
and *clean-merge:*
 $\neg$ *hash-map-output-collision*
(merkle-hash-state-merge final-state final-state')
shows *trace-table = trace-table'*
proof –

```

from witness obtain result query-idxs as dg composition-fri-roots final rest
where trace-partial:
  accepted-with-partial-trace-openings s
  (Some (result, final-state)) fr query-idxs trace-openings
and trace-cand:
  partial-trace-table-candidate trace-table trace-openings
unfolding alpha-header-supported-partial-trace-table-candidate-witness-def
by blast
from witness' obtain result' query-idxs' as' dg' composition-fri-roots'
  final' rest' where trace-partial':
    accepted-with-partial-trace-openings s
    (Some (result', final-state')) fr query-idxs' trace-openings'
and trace-cand':
  partial-trace-table-candidate trace-table' trace-openings'
unfolding alpha-header-supported-partial-trace-table-candidate-witness-def
by blast
show ?thesis
by (rule partial-trace-table-candidates-eq-if-cross-cover-clean-merge
  [OF trace-partial trace-partial' trace-cand trace-cand' trace-cover
    no-conflict clean-merge])

```

qed

lemma *alpha-header-supported-witnessed-partial-trace-table-candidates-subset-singleton-if-pairwise-cross-cover-clean-*
assumes *pairwise*:

```

   $\bigwedge \text{trace-table final-state trace-openings trace-table' final-state'}$ 
   $\text{trace-openings'}$ .
  alpha-header-supported-partial-trace-table-candidate-witness s fr
  f-fri-roots f-final trace-table final-state trace-openings
 $\implies$ 
  alpha-header-supported-partial-trace-table-candidate-witness s fr
  f-fri-roots f-final trace-table' final-state' trace-openings'
 $\implies$ 
  partial-openings-cross-cover-all-indices trace-openings
  trace-openings'  $\wedge$ 
   $\neg \text{merkle-hash-value-conflict final-state final-state'}$   $\wedge$ 
   $\neg \text{hash-map-output-collision}$ 
  (merkle-hash-state-merge final-state final-state')

```

shows

```

 $\exists \text{trace-table.}$ 
alpha-header-supported-witnessed-partial-trace-table-candidates s fr
f-fri-roots f-final  $\subseteq \{\text{trace-table}\}$ 

```

proof (*cases*

```

  alpha-header-supported-witnessed-partial-trace-table-candidates s fr
  f-fri-roots f-final = \{\})

```

case *True*

then show *?thesis*

```

  by (intro exI[of - \{\}] simp)

```

next

case *False*

then obtain *trace-table0* **where** *cand0*:
trace-table0 $\in$
alpha-header-supported-witnessed-partial-trace-table-candidates *s* *fr*
f-fri-roots *f-final*
by *blast*
from *alpha-header-supported-witnessed-partial-trace-table-candidates-obtain-witness*
 $[OF\ cand0]$
obtain *final-state0* *trace-openings0* **where** *witness0*:
alpha-header-supported-partial-trace-table-candidate-witness *s* *fr*
f-fri-roots *f-final* *trace-table0* *final-state0* *trace-openings0*
by *blast*
have *subset*:
alpha-header-supported-witnessed-partial-trace-table-candidates *s* *fr*
f-fri-roots *f-final* $\subseteq \{trace-table0\}$
proof
fix *trace-table*
assume *cand*:
trace-table $\in$
alpha-header-supported-witnessed-partial-trace-table-candidates *s* *fr*
f-fri-roots *f-final*
from *alpha-header-supported-witnessed-partial-trace-table-candidates-obtain-witness*
 $[OF\ cand]$
obtain *final-state* *trace-openings* **where** *witness*:
alpha-header-supported-partial-trace-table-candidate-witness *s* *fr*
f-fri-roots *f-final* *trace-table* *final-state* *trace-openings*
by *blast*
have *pairwise-facts*:
partial-openings-cross-cover-all-indices *trace-openings*
trace-openings0
 $\neg$ *merkle-hash-value-conflict* *final-state* *final-state0*
 $\neg$ *hash-map-output-collision*
(*merkle-hash-state-merge* *final-state* *final-state0*)
using *pairwise* $[OF\ witness\ witness0]$ **by** *simp-all*
have *trace-table* = *trace-table0*
by (*rule*
alpha-header-supported-partial-trace-table-candidate-witness-eq-if-cross-cover-clean-merge
 $[OF\ witness\ witness0\ pairwise-facts]$)
then show *trace-table* $\in \{trace-table0\}$
by *simp*
qed
then show *?thesis*
by *blast*
qed

lemma *alpha-header-supported-witnessed-partial-trace-table-candidates-not-singleton-imp-cross-cover-or-merkle-bad*:
assumes *not-unique*:
 $\neg (\exists\ trace-table.$
alpha-header-supported-witnessed-partial-trace-table-candidates *s* *fr*
f-fri-roots *f-final* $\subseteq \{trace-table\})$

shows

$\exists$ *trace-table final-state trace-openings trace-table' final-state'*
trace-openings'.
alpha-header-supported-partial-trace-table-candidate-witness s fr
f-fri-roots f-final trace-table final-state trace-openings $\wedge$
alpha-header-supported-partial-trace-table-candidate-witness s fr
f-fri-roots f-final trace-table' final-state' trace-openings' $\wedge$
trace-table $\neq$ *trace-table'* $\wedge$
 $(\neg$ *partial-openings-cross-cover-all-indices trace-openings*
trace-openings' $\vee$
merkle-hash-value-conflict final-state final-state' $\vee$
hash-map-output-collision
(merkle-hash-state-merge final-state final-state'))

proof –

let *?C* =
alpha-header-supported-witnessed-partial-trace-table-candidates s fr
f-fri-roots f-final
have *nonempty*: *?C* $\neq$ {}
using *not-unique* **by** *auto*
then obtain *trace-table* **where** *cand*: *trace-table* $\in$ *?C*
by *blast*
have *not-subset*: $\neg$ *?C* $\subseteq$ {*trace-table*}
using *not-unique* **by** *blast*
then obtain *trace-table'* **where** *cand'*: *trace-table'* $\in$ *?C*
and *distinct*: *trace-table* $\neq$ *trace-table'*
by *auto*
from *alpha-header-supported-witnessed-partial-trace-table-candidates-obtain-witness*
[*OF cand*]
obtain *final-state trace-openings* **where** *witness*:
alpha-header-supported-partial-trace-table-candidate-witness s fr
f-fri-roots f-final trace-table final-state trace-openings
by *blast*
from *alpha-header-supported-witnessed-partial-trace-table-candidates-obtain-witness*
[*OF cand*]
obtain *final-state' trace-openings'* **where** *witness'*:
alpha-header-supported-partial-trace-table-candidate-witness s fr
f-fri-roots f-final trace-table' final-state' trace-openings'
by *blast*
have *side*:
 $\neg$ *partial-openings-cross-cover-all-indices trace-openings*
trace-openings' $\vee$
merkle-hash-value-conflict final-state final-state' $\vee$
hash-map-output-collision
(merkle-hash-state-merge final-state final-state')
proof (*rule ccontr*)
assume *no-side*:
 $\neg$ $(\neg$ *partial-openings-cross-cover-all-indices trace-openings*
trace-openings' $\vee$
merkle-hash-value-conflict final-state final-state' $\vee$

```

    hash-map-output-collision
    (merkle-hash-state-merge final-state final-state'))
have trace-table = trace-table'
by (rule
    alpha-header-supported-partial-trace-table-candidate-witness-eq-if-cross-cover-clean-merge
    [OF witness witness'])
    (use no-side in auto)
then show False
    using distinct by simp
qed
show ?thesis
    using witness witness' distinct side by blast
qed

```

definition *alpha-header-supported-witnessed-partial-trace-pairwise-cross-or-merkle-bad*
 $:: (f, 'a) \text{ protocol-channel-scheme} \Rightarrow$
 $f \Rightarrow f \text{ list} \Rightarrow f \Rightarrow \text{bool}$

where

```

alpha-header-supported-witnessed-partial-trace-pairwise-cross-or-merkle-bad
  s fr f-fri-roots f-final  $\longleftrightarrow$ 
  ( $\exists$  trace-table final-state trace-openings trace-table' final-state'
    trace-openings'.
    alpha-header-supported-partial-trace-table-candidate-witness s fr
      f-fri-roots f-final trace-table final-state trace-openings  $\wedge$ 
    alpha-header-supported-partial-trace-table-candidate-witness s fr
      f-fri-roots f-final trace-table' final-state' trace-openings'  $\wedge$ 
    trace-table  $\neq$  trace-table'  $\wedge$ 
    ( $\neg$  partial-openings-cross-cover-all-indices trace-openings
      trace-openings'  $\vee$ 
    merkle-hash-value-conflict final-state final-state'  $\vee$ 
    hash-map-output-collision
    (merkle-hash-state-merge final-state final-state'))))

```

definition *alpha-supported-witnessed-partial-trace-pairwise-cross-or-merkle-bad*
 $:: (f, 'a) \text{ protocol-channel-scheme} \Rightarrow \text{bool}$

where

```

alpha-supported-witnessed-partial-trace-pairwise-cross-or-merkle-bad s
 $\longleftrightarrow$ 
  ( $\exists$  fr f-fri-roots f-final.
    alpha-header-supported-witnessed-partial-trace-pairwise-cross-or-merkle-bad
      s fr f-fri-roots f-final)

```

lemma *alpha-header-supported-witnessed-partial-trace-pairwise-cross-or-merkle-bad-if-not-singleton:*

assumes *not-unique:*

```

 $\neg (\exists \text{ trace-table.}$ 
  alpha-header-supported-witnessed-partial-trace-table-candidates s fr
    f-fri-roots f-final  $\subseteq \{\text{trace-table}\})$ 

```

shows

```

alpha-header-supported-witnessed-partial-trace-pairwise-cross-or-merkle-bad

```

$s \text{ fr } f\text{-fri-roots } f\text{-final}$
unfolding
 $\alpha\text{-header-supported-witnessed-partial-trace-pairwise-cross-or-merkle-bad-def}$
using
 $\alpha\text{-header-supported-witnessed-partial-trace-table-candidates-not-singleton-imp-cross-cover-or-merkle-bad}$
 $[OF \text{ not-unique}]$
by *blast*

lemma $\alpha\text{-header-supported-witnessed-partial-trace-table-candidates-subset-singleton-if-no-cross-or-merkle-bad}$:
assumes no-bad :
 $\neg \alpha\text{-header-supported-witnessed-partial-trace-pairwise-cross-or-merkle-bad}$
 $s \text{ fr } f\text{-fri-roots } f\text{-final}$
shows
 $\exists \text{ trace-table.}$
 $\alpha\text{-header-supported-witnessed-partial-trace-table-candidates } s \text{ fr }$
 $f\text{-fri-roots } f\text{-final} \subseteq \{\text{trace-table}\}$
proof (rule *ccontr*)
assume not-unique :
 $\neg (\exists \text{ trace-table.}$
 $\alpha\text{-header-supported-witnessed-partial-trace-table-candidates } s \text{ fr }$
 $f\text{-fri-roots } f\text{-final} \subseteq \{\text{trace-table}\})$
then have
 $\alpha\text{-header-supported-witnessed-partial-trace-pairwise-cross-or-merkle-bad}$
 $s \text{ fr } f\text{-fri-roots } f\text{-final}$
by (rule
 $\alpha\text{-header-supported-witnessed-partial-trace-pairwise-cross-or-merkle-bad-if-not-singleton})$
then show *False*
using no-bad **by** *contradiction*
qed

lemma $\alpha\text{-header-supported-partial-trace-table-candidates-subset-singleton-if-no-witnessed-cross-or-merkle-bad}$:
assumes no-bad :
 $\neg \alpha\text{-header-supported-witnessed-partial-trace-pairwise-cross-or-merkle-bad}$
 $s \text{ fr } f\text{-fri-roots } f\text{-final}$
shows
 $\exists \text{ trace-table.}$
 $\alpha\text{-header-supported-partial-trace-table-candidates } s \text{ fr } f\text{-fri-roots}$
 $f\text{-final} \subseteq \{\text{trace-table}\}$
proof –
have
 $\exists \text{ trace-table.}$
 $\alpha\text{-header-supported-witnessed-partial-trace-table-candidates } s \text{ fr }$
 $f\text{-fri-roots } f\text{-final} \subseteq \{\text{trace-table}\}$
by (rule
 $\alpha\text{-header-supported-witnessed-partial-trace-table-candidates-subset-singleton-if-no-cross-or-merkle-bad}$
 $[OF \text{ no-bad}])$
moreover have
 $\alpha\text{-header-supported-partial-trace-table-candidates } s \text{ fr } f\text{-fri-roots}$
 $f\text{-final} =$

α -header-supported-witnessed-partial-trace-table-candidates s fr
 f -fri-roots f -final
using α -header-supported-witnessed-candidates-eq-candidates
 $[of\ s\ fr\ f$ -fri-roots f -final]
by *simp*
then show *?thesis*
using *calculation* **by** *simp*
qed

lemma α -header-supported-partial-trace-table-candidates-subset-singleton-if-no-global-witnessed-cross-or-merkle-bad:
assumes *no-bad*:
 $\neg \alpha$ -supported-witnessed-partial-trace-pairwise-cross-or-merkle-bad
 s
shows
 $\exists$ *trace-table*.
 α -header-supported-partial-trace-table-candidates s fr f -fri-roots
 f -final $\subseteq \{trace\text{-}table\}$
proof (*rule*
 α -header-supported-partial-trace-table-candidates-subset-singleton-if-no-witnessed-cross-or-merkle-bad)
show
 $\neg \alpha$ -header-supported-witnessed-partial-trace-pairwise-cross-or-merkle-bad
 s fr f -fri-roots f -final
using *no-bad*
unfolding
 α -supported-witnessed-partial-trace-pairwise-cross-or-merkle-bad-def
by *blast*
qed

lemma α -header-supported-witnessed-partial-union-bad-sets-fraction-bound-if-no-cross-or-merkle-bad:
fixes $C :: prob$
assumes *no-bad*:
 $\neg \alpha$ -header-supported-witnessed-partial-trace-pairwise-cross-or-merkle-bad
 s fr f -fri-roots f -final
and *subset*: $\bigwedge trace\text{-}table. bad\text{-}sets\ trace\text{-}table \subseteq \alpha$ -space
and *bounded*:
 $\bigwedge trace\text{-}table.$
 $nnreal\ (card\ (bad\text{-}sets\ trace\text{-}table)) / nnreal\ (card\ \alpha$ -space) $\leq$
 C
shows
 $nnreal$
 $(card$
 $(\alpha$ -header-supported-witnessed-partial-union-bad-sets s *bad-sets*
 $fr\ f$ -fri-roots f -final)) /
 $nnreal\ (card\ \alpha$ -space) $\leq C$
proof –
have *unique*:
 $\exists$ *trace-table*.
 α -header-supported-witnessed-partial-trace-table-candidates s fr
 f -fri-roots f -final $\subseteq \{trace\text{-}table\}$


```

    by (rule
      alpha-header-supported-witnessed-partial-trace-table-candidates-subset-singleton-if-no-cross-or-merkle-bad
      [OF no-bad])
  from unique obtain trace-table where candidates:
    alpha-header-supported-witnessed-partial-trace-table-candidates s fr
    f-fri-roots f-final  $\subseteq$  {trace-table}
  by blast
  have union-subset:
    alpha-header-supported-witnessed-partial-union-bad-sets s bad-sets fr
    f-fri-roots f-final  $\subseteq$  bad-sets trace-table
  unfolding alpha-header-supported-witnessed-partial-union-bad-sets-def
  using candidates by auto
  have finite-bad: finite (bad-sets trace-table)
  by (rule finite-subset[OF subset finite-alpha-space])
  have card-le:
    card
    (alpha-header-supported-witnessed-partial-union-bad-sets s bad-sets fr
      f-fri-roots f-final)  $\leq$  card (bad-sets trace-table)
  by (rule card-mono[OF finite-bad union-subset])
  have nnreal
    (card
      (alpha-header-supported-witnessed-partial-union-bad-sets s bad-sets
        fr f-fri-roots f-final)) /
    nnreal (card alpha-space)  $\leq$ 
    nnreal (card (bad-sets trace-table)) / nnreal (card alpha-space)
  by (rule nnreal-nat-divide-right-mono[OF card-le])
  also have ...  $\leq C$ 
  by (rule bounded)
  finally show ?thesis .
qed

```

```

lemma alpha-header-supported-partial-union-bad-sets-fraction-bound-if-no-witnessed-cross-or-merkle-bad:
  fixes C :: prob
  assumes no-bad:
     $\neg$  alpha-header-supported-witnessed-partial-trace-pairwise-cross-or-merkle-bad
    s fr f-fri-roots f-final
  and subset:  $\bigwedge$  trace-table. bad-sets trace-table  $\subseteq$  alpha-space
  and bounded:
     $\bigwedge$  trace-table.
    nnreal (card (bad-sets trace-table)) / nnreal (card alpha-space)  $\leq$ 
    C
  shows
    nnreal
      (card
        (alpha-header-supported-partial-union-bad-sets s bad-sets fr
          f-fri-roots f-final)) /
      nnreal (card alpha-space)  $\leq C$ 
  using
    alpha-header-supported-witnessed-partial-union-bad-sets-fraction-bound-if-no-cross-or-merkle-bad

```

[OF no-bad subset bounded]

by (*simp add: alpha-header-supported-witnessed-union-bad-sets-eq-union-bad-sets*)

lemma *alpha-header-supported-witnessed-partial-union-bad-sets-fraction-bound-if-no-global-cross-or-merkle-bad*:

fixes $C :: \text{prob}$

assumes *no-bad*:

$\neg$ *alpha-supported-witnessed-partial-trace-pairwise-cross-or-merkle-bad*
 s

and *subset*: $\bigwedge \text{trace-table. bad-sets trace-table} \subseteq \text{alpha-space}$

and *bounded*:

$\bigwedge \text{trace-table.}$
 $\text{nnreal (card (bad-sets trace-table)) / nnreal (card alpha-space)} \leq$
 C

shows

nnreal
 $(\text{card}$
 $(\text{alpha-header-supported-witnessed-partial-union-bad-sets } s \text{ bad-sets}$
 $\text{fr } f\text{-fri-roots } f\text{-final})) /$
 $\text{nnreal (card alpha-space)}) \leq C$

proof –

have *no-header-bad*:

$\neg$ *alpha-header-supported-witnessed-partial-trace-pairwise-cross-or-merkle-bad*
 $s \text{ fr } f\text{-fri-roots } f\text{-final}$

using *no-bad*

unfolding

alpha-supported-witnessed-partial-trace-pairwise-cross-or-merkle-bad-def

by *blast*

show *?thesis*

by (*rule*
 alpha-header-supported-witnessed-partial-union-bad-sets-fraction-bound-if-no-cross-or-merkle-bad
 [OF *no-header-bad subset bounded*])

qed

lemma *alpha-header-supported-partial-union-bad-sets-fraction-bound-if-no-global-witnessed-cross-or-merkle-bad*:

fixes $C :: \text{prob}$

assumes *no-bad*:

$\neg$ *alpha-supported-witnessed-partial-trace-pairwise-cross-or-merkle-bad*
 s

and *subset*: $\bigwedge \text{trace-table. bad-sets trace-table} \subseteq \text{alpha-space}$

and *bounded*:

$\bigwedge \text{trace-table.}$
 $\text{nnreal (card (bad-sets trace-table)) / nnreal (card alpha-space)} \leq$
 C

shows

nnreal
 $(\text{card}$
 $(\text{alpha-header-supported-partial-union-bad-sets } s \text{ bad-sets fr}$
 $f\text{-fri-roots } f\text{-final})) /$
 $\text{nnreal (card alpha-space)}) \leq C$

using
alpha-header-supported-witnessed-partial-union-bad-sets-fraction-bound-if-no-global-cross-or-merkle-bad
 $[OF \text{ no-bad subset bounded}]$
by (*simp add: alpha-header-supported-witnessed-union-bad-sets-eq-union-bad-sets*)

lemma *wp-alpha-bad-set-hit-bound-if-no-global-witnessed-cross-or-merkle-bad:*
fixes $C \ H :: \text{prob}$
assumes *future: alpha-future-fresh s*
and *no-bad:*
 $\neg \text{alpha-supported-witnessed-partial-trace-pairwise-cross-or-merkle-bad } s$
and *subset:* $\bigwedge \text{trace-table. bad-sets trace-table} \subseteq \text{alpha-space}$
and *bounded:*
 $\bigwedge \text{trace-table. nnreal (card (bad-sets trace-table)) / nnreal (card alpha-space)} \leq C$
and *collision-bound:*
 $\text{wp-event verify-monad (hash-map-output-collision-bad } s) \ s \leq H$
shows $\text{wp-event verify-monad (alpha-bad-set-hit } s \text{ bad-sets)} \ s \leq C + H$

proof –
have *unique:*
 $\bigwedge \text{fr f-fri-roots f-final.} \exists \text{ trace-table.}$
 $\text{alpha-header-supported-partial-trace-table-candidates } s \text{ fr}$
 $\text{f-fri-roots f-final} \subseteq \{\text{trace-table}\}$
by (*rule*
 $\text{alpha-header-supported-partial-trace-table-candidates-subset-singleton-if-no-global-witnessed-cross-or-merkle-bad}$
 $[OF \text{ no-bad}]$)
show *?thesis*
by (*rule*
 $\text{wp-alpha-bad-set-hit-bound-if-supported-partial-header-candidate-unique-or-collision}$
 $[OF \text{ future unique subset bounded collision-bound}]$)

qed

lemma *composition-randomization-bad-bound-if-no-global-witnessed-cross-or-merkle-bad:*
fixes $H :: \text{prob}$
assumes *future: alpha-future-fresh s*
and *no-bad:*
 $\neg \text{alpha-supported-witnessed-partial-trace-pairwise-cross-or-merkle-bad } s$
and *collision-bound:*
 $\text{wp-event verify-monad (hash-map-output-collision-bad } s) \ s \leq H$
shows
 $\text{wp-event verify-monad (composition-randomization-bad } s) \ s \leq \text{composition-error-bound} + H$

proof –
have *unique:*
 $\bigwedge \text{fr f-fri-roots f-final.} \exists \text{ trace-table.}$

```

      alpha-header-supported-partial-trace-table-candidates s fr
      f-fri-roots f-final  $\subseteq \{ \text{trace-table} \}$ 
    by (rule
      alpha-header-supported-partial-trace-table-candidates-subset-singleton-if-no-global-witnessed-cross-or-merkle-bad
      [OF no-bad])
  show ?thesis
  by (rule
    composition-randomization-bad-bound-if-alpha-header-supported-partial-candidate-unique-or-collision
    [OF future unique collision-bound])
qed

lemma alpha-header-witness-candidate-in-prefix-if-pullback-clean-cover:
  assumes prefix-candidate:
    prefix-trace-table  $\in$ 
    alpha-prefix-trace-table-candidates prefix-state fr
  and witness:
    alpha-header-supported-partial-trace-table-candidate-witness s fr
    f-fri-roots f-final trace-table final-state trace-openings
  and pullback:
     $\bigwedge i. i < \text{rounds} \implies$ 
    partial-authenticated-table fr (scale * clength)
    (trace-openings ! i) prefix-state
  and cover:
     $\bigwedge j. j < \text{scale} * \text{clength} \implies$ 
     $\exists i < \text{rounds}. \exists \text{opn} \in \text{set } (\text{trace-openings} ! i).$ 
    opening-index opn = j
  and clean-prefix:  $\neg \text{hash-map-output-collision prefix-state}$ 
  shows
    trace-table  $\in$ 
    alpha-prefix-trace-table-candidates prefix-state fr
proof -
  have prefix-len:
    length prefix-trace-table = clength * scale
  using prefix-candidate
  unfolding alpha-prefix-trace-table-candidates-def by simp
  have prefix-bind:
    merkle-root-binds-table fr prefix-trace-table prefix-state
  using prefix-candidate
  unfolding alpha-prefix-trace-table-candidates-def by simp
  from witness obtain result query-idxs as dg composition-fri-roots final rest
  where trace-cand:
    partial-trace-table-candidate trace-table trace-openings
  unfolding alpha-header-supported-partial-trace-table-candidate-witness-def
  by blast
  have prefix-trace-cand:
    partial-trace-table-candidate prefix-trace-table trace-openings
proof -
  obtain n where len-pow: clength * scale =  $2^n$ 
  using eval-domain-length-power by blast

```

```

have table-pow: length prefix-trace-table = 2 ^ n
  using prefix-len len-pow by (simp add: mult.commute)
have rounds-len: length trace-openings = rounds
  using trace-cand unfolding partial-trace-table-candidate-def by simp
have agrees:
   $\bigwedge i. i < rounds \implies$ 
    table-agrees-with-authenticated-openings prefix-trace-table
      (scale * clength) (trace-openings ! i)
proof -
  fix i
  assume i-bound: i < rounds
  have table-i:
    partial-authenticated-table fr (scale * clength)
      (trace-openings ! i) prefix-state
  by (rule pullback[OF i-bound])
  have opening-agrees:
     $\forall \text{opn} \in \text{set } (\text{trace-openings} ! i).$ 
      opening-index opn < scale * clength  $\wedge$ 
      prefix-trace-table ! opening-index opn = opening-value opn
  proof
    fix opn
    assume opn-in: opn  $\in$  set (trace-openings ! i)
    have root: opening-root opn = fr
      using table-i opn-in unfolding partial-authenticated-table-def
    by blast
    have len: opening-length opn = length prefix-trace-table
      using table-i opn-in prefix-len
    unfolding partial-authenticated-table-def by simp
    have auth: authenticated-opening-in prefix-state opn
      using table-i opn-in unfolding partial-authenticated-table-def
    by blast
    have agreement:
      opening-index opn < length prefix-trace-table  $\wedge$ 
      prefix-trace-table ! opening-index opn = opening-value opn
    by (rule merkle-bound-table-agrees-with-authenticated-opening-if-clean
      [OF prefix-bind table-pow auth root len clean-prefix])
    then show
      opening-index opn < scale * clength  $\wedge$ 
      prefix-trace-table ! opening-index opn = opening-value opn
    using prefix-len by (simp add: mult.commute)
  qed
show
  table-agrees-with-authenticated-openings prefix-trace-table
    (scale * clength) (trace-openings ! i)
  unfolding table-agrees-with-authenticated-openings-def
  using prefix-len opening-agrees by simp
qed
show ?thesis
  unfolding partial-trace-table-candidate-def

```

using *prefix-len rounds-len agrees* by *simp*
 qed
 have *trace-eq*: *trace-table* = *prefix-trace-table*
 by (rule *partial-trace-table-candidate-eq-if-all-indices-opened*
 [OF *trace-cand prefix-trace-cand cover*])
 show ?thesis
 using *prefix-candidate unfolding trace-eq* .
 qed

lemma *alpha-header-not-prefix-witness-imp-no-prefix-candidate-or-pullback-gap*:
 assumes *not-prefix*:
 trace-table $\notin$ *alpha-prefix-trace-table-candidates prefix-state fr*
 and *witness*:
 alpha-header-supported-partial-trace-table-candidate-witness s fr
 f-fri-roots f-final trace-table final-state trace-openings
 and *clean-prefix*: $\neg$ *hash-map-output-collision prefix-state*
 and *pullback*:
 $\bigwedge i. i < \text{rounds} \implies$
 *partial-authenticated-table fr (scale * clength)*
 (*trace-openings* ! *i*) *prefix-state*
 and *cover*:
 $\bigwedge j. j < \text{scale} * \text{clength} \implies$
 $\exists i < \text{rounds}. \exists \text{opn} \in \text{set } (\text{trace-openings} ! i).$
 opening-index opn = *j*
 shows
 alpha-prefix-trace-table-candidates prefix-state fr = {}
proof (rule *ccontr*)
 assume *alpha-prefix-trace-table-candidates prefix-state fr* $\neq$ {}
 then obtain *prefix-trace-table* **where** *prefix-candidate*:
 prefix-trace-table $\in$
 alpha-prefix-trace-table-candidates prefix-state fr
 by *blast*
 have
 trace-table $\in$
 alpha-prefix-trace-table-candidates prefix-state fr
 by (rule *alpha-header-witness-candidate-in-prefix-if-pullback-clean-cover*
 [OF *prefix-candidate witness pullback cover clean-prefix*])
 then show *False*
 using *not-prefix* by *contradiction*
 qed

lemma *alpha-header-witness-opening-pullback-or-new-output-hit*:
 assumes *ext*: *prefix-state* $\leq$ *final-state*
 and *witness*:
 alpha-header-supported-partial-trace-table-candidate-witness s fr
 f-fri-roots f-final trace-table final-state trace-openings
 and *i-bound*: *i* < *rounds*
 and *opn-in*: *opn* $\in$ *set (trace-openings ! i)*
 shows

```

    authenticated-opening-in prefix-state opn ∨
    hash-map-new-output-hit
      (merkle-path-target-roots final-state fr (scale * clength)
        (opening-index opn) (opening-value opn) (opening-path opn))
    prefix-state final-state
  proof -
  from witness obtain result query-idxs as dg composition-fri-roots final rest
  where partial:
    accepted-with-partial-trace-openings s
      (Some (result, final-state)) fr query-idxs trace-openings
  unfolding alpha-header-supported-partial-trace-table-candidate-witness-def
  by blast
  have table-i:
    partial-authenticated-table fr (scale * clength)
      (trace-openings ! i) final-state
  using partial i-bound
  unfolding accepted-with-partial-trace-openings-def by blast
  show ?thesis
  by (rule partial-authenticated-table-opening-pullback-or-new-output-hit
    [OF ext table-i opn-in])
qed

lemma alpha-header-witness-openings-pullback-or-new-output-hit:
  assumes ext: prefix-state ≤ final-state
  and witness:
    alpha-header-supported-partial-trace-table-candidate-witness s fr
      f-fri-roots f-final trace-table final-state trace-openings
  shows
    (∀ i < rounds.
      partial-authenticated-table fr (scale * clength)
        (trace-openings ! i) prefix-state) ∨
    (∃ i < rounds. ∃ opn ∈ set (trace-openings ! i).
      hash-map-new-output-hit
        (merkle-path-target-roots final-state fr (scale * clength)
          (opening-index opn) (opening-value opn) (opening-path opn))
        prefix-state final-state)
  proof (cases
    ∃ i < rounds. ∃ opn ∈ set (trace-openings ! i).
      hash-map-new-output-hit
        (merkle-path-target-roots final-state fr (scale * clength)
          (opening-index opn) (opening-value opn) (opening-path opn))
        prefix-state final-state)
  case True
  then show ?thesis by simp
next
case False
have pullback-all:
  ∀ i < rounds.
    partial-authenticated-table fr (scale * clength)

```

```

      (trace-openings ! i) prefix-state
proof (intro allI impI)
  fix i
  assume i-bound: i < rounds
  from witness obtain result query-idxs as dg composition-fri-roots final rest
  where partial:
    accepted-with-partial-trace-openings s
    (Some (result, final-state)) fr query-idxs trace-openings
  unfolding alpha-header-supported-partial-trace-table-candidate-witness-def
  by blast
have table-i:
  partial-authenticated-table fr (scale * clength)
    (trace-openings ! i) final-state
  using partial i-bound
  unfolding accepted-with-partial-trace-openings-def by blast
show
  partial-authenticated-table fr (scale * clength)
    (trace-openings ! i) prefix-state
  unfolding partial-authenticated-table-def
proof
  fix opn
  assume opn-in: opn ∈ set (trace-openings ! i)
  have root-len:
    opening-root opn = fr ∧ opening-length opn = scale * clength
  using table-i opn-in unfolding partial-authenticated-table-def
  by blast
  have pullback:
    authenticated-opening-in prefix-state opn ∨
    hash-map-new-output-hit
      (merkle-path-target-roots final-state fr (scale * clength)
        (opening-index opn) (opening-value opn) (opening-path opn))
    prefix-state final-state
  by (rule alpha-header-witness-opening-pullback-or-new-output-hit
    [OF ext witness i-bound opn-in])
  have auth-prefix: authenticated-opening-in prefix-state opn
  using pullback False i-bound opn-in by blast
show
  opening-root opn = fr ∧
  opening-length opn = scale * clength ∧
  authenticated-opening-in prefix-state opn
  using root-len auth-prefix by simp
qed
qed
then show ?thesis by simp
qed

end

end

```


theory *Staged-Security-Experiment-Alpha-Header-Binding*

imports

Staged-Security-Experiment-Composition-Query-Residual

Soundness-Alpha-Header-Cross-Candidate

begin

Staged wrapper for the alpha-header candidate-binding diagnostic.

This does not bound the event. It isolates the remaining proof obligation: a partial-header alpha hit that is not already a prefix-fixed alpha hit must use a header-supported partial trace candidate that is not bound by the prefix Merkle evidence.

context *soundness*

begin

definition *checked-staged-security-with-actual-alpha-prefix-partial-header-candidate-not-prefix-bound*

$$\begin{aligned} &:: ('f \text{ list} \Rightarrow 'f \text{ list set}) \Rightarrow \\ &(((('f \times 'f \text{ list} \times 'f \text{ list} \times 'f) \times \\ &\quad 'f \text{ protocol-channel}) \times 'f \text{ staged-proof-data}) \times \\ &\quad 'f \text{ protocol-channel}) \times \text{unit list}) \times \\ &\quad 'f \text{ protocol-channel}) \text{ option} \Rightarrow \text{bool} \end{aligned}$$

where

checked-staged-security-with-actual-alpha-prefix-partial-header-candidate-not-prefix-bound

bad-sets out $\longleftrightarrow$

(*case out of*

None $\Rightarrow$ *False*

| *Some* (*x*, *final-state*) $\Rightarrow$

(*let* *packed* = *fst* (*fst* *x*);

prefix = *fst* (*fst* *packed*);

prefix-state = *snd* (*fst* *packed*);

data = *snd* *packed*;

attacker-state = *snd* (*fst* *x*);

result = *snd* *x*;

s =

verifier-state-from-adversary attacker-state

(*staged-proof-transcript data*)

in $\exists$ *as query-idxs fr f-fri-roots f-final dg composition-fri-roots*

final rest.

accepted-transcript-shape s (*Some* (*result*, *final-state*))

as query-idxs $\wedge$

verifier-header-transcript s fr f-fri-roots f-final as dg

composition-fri-roots final rest $\wedge$

as $\in$

alpha-header-supported-partial-union-bad-sets s bad-sets fr

f-fri-roots f-final $\wedge$

as $\notin$ *alpha-prefix-union-bad-sets prefix-state bad-sets fr* $\wedge$

alpha-header-partial-candidate-not-prefix-bound prefix-state s
fr f-fri-roots f-final))

lemma *checked-staged-security-with-actual-alpha-prefix-partial-header-candidate-not-prefix-boundI*:
assumes *s-eq*:

s =
verifier-state-from-adversary attacker-state
(staged-proof-transcript data)
and *shape*:
accepted-transcript-shape s (Some (result, final-state))
as query-idxs
and *header*:
verifier-header-transcript s fr f-fri-roots f-final as dg
composition-fri-roots final rest
and *hit*:
as ∈
alpha-header-supported-partial-union-bad-sets s bad-sets fr
f-fri-roots f-final
and *not-prefix*:
as ∉ *alpha-prefix-union-bad-sets prefix-state bad-sets fr*
and *not-bound*:
alpha-header-partial-candidate-not-prefix-bound prefix-state s fr
f-fri-roots f-final

shows

checked-staged-security-with-actual-alpha-prefix-partial-header-candidate-not-prefix-bound
bad-sets
(Some (((((prefix, prefix-state), data), attacker-state), result),
final-state)))

proof —

let *?s* =
verifier-state-from-adversary attacker-state
(staged-proof-transcript data)
have *shape'*:
accepted-transcript-shape ?s (Some (result, final-state))
as query-idxs
using *shape* **by** (*simp add: s-eq*)
have *header'*:
verifier-header-transcript ?s fr f-fri-roots f-final as dg
composition-fri-roots final rest
using *header* **by** (*simp add: s-eq*)
have *hit'*:
as ∈
alpha-header-supported-partial-union-bad-sets ?s bad-sets fr
f-fri-roots f-final
using *hit* **by** (*simp add: s-eq*)
have *not-bound'*:
alpha-header-partial-candidate-not-prefix-bound prefix-state ?s fr
f-fri-roots f-final
using *not-bound* **by** (*simp add: s-eq*)

show *?thesis*

unfolding

checked-staged-security-with-actual-alpha-prefix-partial-header-candidate-not-prefix-bound-def

```

  Let-def
  apply simp
  apply (rule exI[where x=as])
  apply (intro conjI)
  apply (rule exI[where x=query-idxs])
  apply (rule shape')
  apply (rule exI[where x=fr])
  apply (rule exI[where x=f-fri-roots])
  apply (rule exI[where x=f-final])
  apply (intro conjI)
  apply (rule exI[where x=dg])
  apply (rule exI[where x=composition-fri-roots])
  apply (rule exI[where x=final])
  apply (rule exI[where x=rest])
  apply (rule header')
  apply (rule hit')
  apply (rule not-prefix)
  apply (rule not-bound')
done
qed

lemma checked-staged-security-with-actual-alpha-prefix-support-prefix-extends-verifier-state:
  assumes wf: staged-budget-wellformed budgets
  and controlled: staged-adversary-controlled budgets A
  and support:
    Some (((((prefix, prefix-state), data), attacker-state), result),
      final-state) ∈
    set-dist
      (execute
        (checked-staged-security-experiment-with-actual-alpha-prefix-data-state
          A)
        adversary-initial-state)
  shows
    prefix-state ≤
    verifier-state-from-adversary attacker-state
    (staged-proof-transcript data)
proof -
  from support obtain trans-out where trans-out:
    Some (((prefix, prefix-state), data), attacker-state) ∈
    set-dist
      (execute (checked-staged-transcript-with-alpha-prefix-program A)
        adversary-initial-state)
  unfolding
    checked-staged-security-experiment-with-actual-alpha-prefix-data-state-def
  by (auto elim!: set-dist-bindE split: prod.splits)
  from checked-staged-transcript-with-alpha-prefix-program-support
    [OF wf controlled trans-out]
  have prefix-ext: prefix-state ≤ attacker-state
  by blast

```

```

show ?thesis
  by (rule
    alpha-prefix-hash-extends-verifier-state-from-adversary-right
    [OF prefix-ext])
qed

lemma checked-staged-security-with-actual-alpha-prefix-support-prefix-extends-verifier-final:
assumes wf: staged-budget-wellformed budgets
and controlled: staged-adversary-controlled budgets A
and support:
  Some (((((prefix, prefix-state), data), attacker-state), result),
    final-state) ∈
    set-dist
      (execute
        (checked-staged-security-experiment-with-actual-alpha-prefix-data-state
          A)
        adversary-initial-state)
and verifier-out:
  Some (result', witness-final-state) ∈
    set-dist
      (execute verify-monad
        (verifier-state-from-adversary attacker-state
          (staged-proof-transcript data)))
shows prefix-state ≤ witness-final-state
proof –
have prefix-s:
  prefix-state ≤
    verifier-state-from-adversary attacker-state
    (staged-proof-transcript data)
by (rule
  checked-staged-security-with-actual-alpha-prefix-support-prefix-extends-verifier-state
  [OF wf controlled support])
have s-final:
  verifier-state-from-adversary attacker-state
    (staged-proof-transcript data) ≤
    witness-final-state
by (rule verify-monad-hash-extends[OF verifier-out])
show ?thesis
by (rule hash-ext-trans[OF prefix-s s-final])
qed

lemma checked-staged-security-with-actual-alpha-prefix-not-prefix-bound-obtain-witness-final-ext:
assumes wf: staged-budget-wellformed budgets
and controlled: staged-adversary-controlled budgets A
and support:
  Some (((((prefix, prefix-state), data), attacker-state), result),
    final-state) ∈
    set-dist
      (execute

```

(checked-staged-security-experiment-with-actual-alpha-prefix-data-state
 A)
 adversary-initial-state)
and bound:
 alpha-header-partial-candidate-not-prefix-bound prefix-state
 (verifier-state-from-adversary attacker-state
 (staged-proof-transcript data))
 fr f-fri-roots f-final
obtains trace-table witness-final-state trace-openings **where**
 alpha-header-supported-partial-trace-table-candidate-witness
 (verifier-state-from-adversary attacker-state
 (staged-proof-transcript data))
 fr f-fri-roots f-final trace-table witness-final-state
 trace-openings
 trace-table $\notin$ alpha-prefix-trace-table-candidates prefix-state fr
 prefix-state $\leq$ witness-final-state
proof –
 let ?s =
 verifier-state-from-adversary attacker-state
 (staged-proof-transcript data)
from alpha-header-partial-candidate-not-prefix-bound-obtain-witness
 [OF bound]
obtain trace-table witness-final-state trace-openings **where** witness:
 alpha-header-supported-partial-trace-table-candidate-witness ?s fr
 f-fri-roots f-final trace-table witness-final-state
 trace-openings
and not-prefix:
 trace-table $\notin$ alpha-prefix-trace-table-candidates prefix-state fr
by blast
from witness **obtain** witness-result query-idxs as dg composition-fri-roots
 final rest **where** verifier-out:
 Some (witness-result, witness-final-state) $\in$
 set-dist (execute verify-monad ?s)
unfolding alpha-header-supported-partial-trace-table-candidate-witness-def
by blast
have prefix-final: prefix-state $\leq$ witness-final-state
by (rule
 checked-staged-security-with-actual-alpha-prefix-support-prefix-extends-verifier-final
 [OF wf controlled support verifier-out])
show ?thesis
by (rule that[OF witness not-prefix prefix-final])
qed

lemma checked-staged-security-with-actual-alpha-prefix-not-prefix-bound-decompose:
assumes wf: staged-budget-wellformed budgets
and controlled: staged-adversary-controlled budgets A
and support:
 Some (((((prefix, prefix-state), data), attacker-state), result),
 final-state) $\in$

```

    set-dist
      (execute
        (checked-staged-security-experiment-with-actual-alpha-prefix-data-state
          A)
        adversary-initial-state)
  and bound:
    alpha-header-partial-candidate-not-prefix-bound prefix-state
      (verifier-state-from-adversary attacker-state
        (staged-proof-transcript data))
    fr f-fri-roots f-final
  shows
    hash-map-output-collision prefix-state  $\vee$ 
    ( $\exists$  trace-table witness-final-state trace-openings.
      alpha-header-supported-partial-trace-table-candidate-witness
        (verifier-state-from-adversary attacker-state
          (staged-proof-transcript data))
      fr f-fri-roots f-final trace-table witness-final-state
      trace-openings  $\wedge$ 
      trace-table  $\notin$ 
        alpha-prefix-trace-table-candidates prefix-state fr  $\wedge$ 
      prefix-state  $\leq$  witness-final-state  $\wedge$ 
      (( $\exists j < \text{scale} * \text{clength}$ .
         $\forall i \text{ opn. } i < \text{rounds} \longrightarrow$ 
          opn  $\in \text{set } (\text{trace-openings } ! i) \longrightarrow$ 
          opening-index opn  $\neq j$ )  $\vee$ 
        alpha-prefix-trace-table-candidates prefix-state fr =  $\{\}$   $\vee$ 
        ( $\exists i < \text{rounds. } \exists \text{ opn} \in \text{set } (\text{trace-openings } ! i).$ 
          hash-map-new-output-hit
            (merkle-path-target-roots witness-final-state fr
              (scale * clength) (opening-index opn)
              (opening-value opn) (opening-path opn))
            prefix-state witness-final-state))))
  proof -
  let ?s =
    verifier-state-from-adversary attacker-state
      (staged-proof-transcript data)
  from checked-staged-security-with-actual-alpha-prefix-not-prefix-bound-obtain-witness-final-ext
    [OF wf controlled support bound]
  obtain trace-table witness-final-state trace-openings where witness:
    alpha-header-supported-partial-trace-table-candidate-witness ?s fr
    f-fri-roots f-final trace-table witness-final-state
    trace-openings
  and not-prefix:
    trace-table  $\notin$ 
      alpha-prefix-trace-table-candidates prefix-state fr
  and prefix-final: prefix-state  $\leq$  witness-final-state
  by blast
  show ?thesis
  proof (cases hash-map-output-collision prefix-state)

```

```

case True
then show ?thesis by simp
next
case clean-prefix: False
show ?thesis
proof (cases
   $\forall j < \text{scale} * \text{clength}.$ 
   $\exists i < \text{rounds}. \exists \text{opn} \in \text{set } (\text{trace-openings} ! i).$ 
   $\text{opening-index } \text{opn} = j$ )
case False
then have uncovered:
   $\exists j < \text{scale} * \text{clength}.$ 
   $\forall i \text{ opn}. i < \text{rounds} \longrightarrow$ 
   $\text{opn} \in \text{set } (\text{trace-openings} ! i) \longrightarrow$ 
   $\text{opening-index } \text{opn} \neq j$ 
by blast
show ?thesis
using witness not-prefix prefix-final uncovered by blast
next
case True
have pullback-or-new:
  ( $\forall i < \text{rounds}.$ 
     $\text{partial-authenticated-table fr } (\text{scale} * \text{clength})$ 
     $(\text{trace-openings} ! i) \text{ prefix-state}) \vee$ 
    ( $\exists i < \text{rounds}. \exists \text{opn} \in \text{set } (\text{trace-openings} ! i).$ 
      hash-map-new-output-hit
      (merkle-path-target-roots witness-final-state fr
         $(\text{scale} * \text{clength}) (\text{opening-index } \text{opn})$ 
         $(\text{opening-value } \text{opn}) (\text{opening-path } \text{opn}))$ 
      prefix-state witness-final-state)
    by (rule alpha-header-witness-openings-pullback-or-new-output-hit
      [OF prefix-final witness])
  from pullback-or-new show ?thesis
proof
assume pullback-all:
   $\forall i < \text{rounds}.$ 
   $\text{partial-authenticated-table fr } (\text{scale} * \text{clength})$ 
   $(\text{trace-openings} ! i) \text{ prefix-state}$ 
have no-prefix-candidate:
   $\text{alpha-prefix-trace-table-candidates prefix-state fr} = \{\}$ 
proof (rule
  alpha-header-not-prefix-witness-imp-no-prefix-candidate-or-pullback-gap
  [OF not-prefix witness clean-prefix])
fix i
assume  $i < \text{rounds}$ 
then show
   $\text{partial-authenticated-table fr } (\text{scale} * \text{clength})$ 
   $(i) \text{ prefix-state}$ 
using pullback-all by blast

```

```

next
  fix j
  assume j < scale * clength
  then show
     $\exists i < \text{rounds}. \exists \text{opn} \in \text{set } (\text{trace-openings } ! i).$ 
    opening-index opn = j
  using True by blast
qed
show ?thesis
  using witness not-prefix prefix-final no-prefix-candidate by blast
next
assume
   $\exists i < \text{rounds}. \exists \text{opn} \in \text{set } (\text{trace-openings } ! i).$ 
  hash-map-new-output-hit
  (merkle-path-target-roots witness-final-state fr
    (scale * clength) (opening-index opn)
    (opening-value opn) (opening-path opn))
  prefix-state witness-final-state
  then show ?thesis
  using witness not-prefix prefix-final by blast
qed
qed
qed
qed

definition checked-staged-security-with-actual-alpha-prefix-not-prefix-collision
:: ((((((f × 'f list × 'f list × 'f) ×
  'f protocol-channel) × 'f staged-proof-data) ×
  'f protocol-channel) × unit list) ×
  'f protocol-channel) option ⇒ bool
where
checked-staged-security-with-actual-alpha-prefix-not-prefix-collision out
 $\longleftrightarrow$ 
(case out of
  None ⇒ False
| Some (x, -) ⇒
  (let packed = fst (fst x);
    prefix-state = snd (fst packed)
  in hash-map-output-collision prefix-state))

lemma checked-staged-security-with-actual-alpha-prefix-not-prefix-collision-bound:
assumes wf: staged-budget-wellformed budgets
and controlled: staged-adversary-controlled budgets A
shows
  wp-event
  (checked-staged-security-experiment-with-actual-alpha-prefix-data-state A)
  checked-staged-security-with-actual-alpha-prefix-not-prefix-collision
  adversary-initial-state ≤
  hash-collision-budget-value 0 (staged-alpha-search-queries budgets 0)

```



```

proof —
  have projection:
    wp-event
      (checked-staged-security-experiment-with-actual-alpha-prefix-data-state A)
      checked-staged-security-with-actual-alpha-prefix-not-prefix-collision
      adversary-initial-state ≤
    wp-event
      (checked-staged-transcript-with-alpha-prefix-program A)
      checked-staged-transcript-actual-alpha-prefix-collision-hit
      adversary-initial-state
  unfolding
    checked-staged-security-experiment-with-actual-alpha-prefix-data-state-def
proof (rule wp-event-bind-bound-by-head-event)
show
  wp-event (checked-staged-transcript-with-alpha-prefix-program A)
    checked-staged-transcript-actual-alpha-prefix-collision-hit
    adversary-initial-state ≤
  wp-event (checked-staged-transcript-with-alpha-prefix-program A)
    checked-staged-transcript-actual-alpha-prefix-collision-hit
    adversary-initial-state
  by simp
next
show
  checked-staged-security-with-actual-alpha-prefix-not-prefix-collision
    None ⇒
  checked-staged-transcript-actual-alpha-prefix-collision-hit None
unfolding
  checked-staged-security-with-actual-alpha-prefix-not-prefix-collision-def
  checked-staged-transcript-actual-alpha-prefix-collision-hit-def
by simp
next
fix packed attacker-state out
assume cont:
  out ∈
    set-dist
      (execute
        (get ≥
          (λs. put
            (verifier-state-from-adversary s
              (staged-proof-transcript (snd packed)))) ≥
            (λ-. verify-monad ≥
              (λresult. return ((packed, s), result))))))
      attacker-state)
and hit:
  checked-staged-security-with-actual-alpha-prefix-not-prefix-collision
    out
obtain prefix prefix-state data where packed-eq:
  packed = ((prefix, prefix-state), data)
by (cases packed, auto split: prod.splits)

```

```

have hash-map-output-collision prefix-state
  using cont hit
  unfolding
    checked-staged-security-with-actual-alpha-prefix-not-prefix-collision-def
    packed-eq
  by (auto elim!: set-dist-bindE split: option.splits prod.splits)
then show
  checked-staged-transcript-actual-alpha-prefix-collision-hit
    (Some (packed, attacker-state))
  unfolding checked-staged-transcript-actual-alpha-prefix-collision-hit-def
    packed-eq
  by simp
qed
have transcript-bound:
  wp-event
    (checked-staged-transcript-with-alpha-prefix-program A)
    checked-staged-transcript-actual-alpha-prefix-collision-hit
    adversary-initial-state ≤
  wp-event (staged-alpha-prefix-program A)
    (hash-new-collision-event adversary-initial-state)
    adversary-initial-state
  by (rule checked-staged-transcript-with-alpha-prefix-collision-bound)
have collision-budget:
  hash-collision-budget (staged-alpha-search-queries budgets 0)
    (staged-alpha-prefix-program A)
  by (rule hash-collision-budget-staged-alpha-prefix-program
    [OF wf controlled])
have collision-bound:
  wp-event (staged-alpha-prefix-program A)
    (hash-new-collision-event adversary-initial-state)
    adversary-initial-state ≤
  hash-collision-budget-value 0 (staged-alpha-search-queries budgets 0)
proof –
  have
    wp-event (staged-alpha-prefix-program A)
      (hash-new-collision-event adversary-initial-state)
      adversary-initial-state ≤
    hash-collision-budget-value
      (card (hash-map-output-values adversary-initial-state))
      (staged-alpha-search-queries budgets 0)
    using collision-budget adversary-initial-state-no-output-collision
    unfolding hash-collision-budget-def by blast
  also have ... =
    hash-collision-budget-value 0 (staged-alpha-search-queries budgets 0)
  by simp
  finally show ?thesis .
qed
show ?thesis
  by (rule order-trans[OF projection])

```

(rule order-trans[OF transcript-bound collision-bound])
qed

definition *checked-staged-security-with-actual-alpha-prefix-not-prefix-uncovered-index*

$$\begin{aligned} &:: ((((((f \times 'f \text{ list} \times 'f \text{ list} \times 'f) \times \\ &\quad 'f \text{ protocol-channel}) \times 'f \text{ staged-proof-data}) \times \\ &\quad 'f \text{ protocol-channel}) \times \text{unit list}) \times \\ &\quad 'f \text{ protocol-channel}) \text{ option} \Rightarrow \text{bool} \end{aligned}$$

where

checked-staged-security-with-actual-alpha-prefix-not-prefix-uncovered-index

$$\begin{aligned} &\text{out} \longleftrightarrow \\ &(\text{case out of} \\ &\quad \text{None} \Rightarrow \text{False} \\ &| \text{Some } (x, -) \Rightarrow \\ &\quad (\text{let packed} = \text{fst } (\text{fst } x); \\ &\quad \quad \text{prefix-state} = \text{snd } (\text{fst } \text{packed}); \\ &\quad \quad \text{data} = \text{snd } \text{packed}; \\ &\quad \quad \text{attacker-state} = \text{snd } (\text{fst } x); \\ &\quad \quad s = \\ &\quad \quad \quad \text{verifier-state-from-adversary attacker-state} \\ &\quad \quad \quad (\text{staged-proof-transcript data}) \\ &\quad \text{in } \exists \text{fr } f\text{-fri-roots } f\text{-final trace-table witness-final-state} \\ &\quad \quad \text{trace-openings.} \\ &\quad \quad \text{alpha-header-supported-partial-trace-table-candidate-witness} \\ &\quad \quad s \text{ fr } f\text{-fri-roots } f\text{-final trace-table witness-final-state} \\ &\quad \quad \text{trace-openings} \wedge \\ &\quad \quad \text{trace-table} \notin \\ &\quad \quad \quad \text{alpha-prefix-trace-table-candidates prefix-state fr} \wedge \\ &\quad \quad \text{prefix-state} \leq \text{witness-final-state} \wedge \\ &\quad \quad (\exists j < \text{scale} * \text{clength.} \\ &\quad \quad \quad \forall i \text{ opn. } i < \text{rounds} \longrightarrow \\ &\quad \quad \quad \text{opn} \in \text{set } (\text{trace-openings ! } i) \longrightarrow \\ &\quad \quad \quad \text{opening-index opn} \neq j))) \end{aligned}$$

definition *checked-staged-security-with-actual-alpha-prefix-not-prefix-no-prefix-candidate*

$$\begin{aligned} &:: ((((((f \times 'f \text{ list} \times 'f \text{ list} \times 'f) \times \\ &\quad 'f \text{ protocol-channel}) \times 'f \text{ staged-proof-data}) \times \\ &\quad 'f \text{ protocol-channel}) \times \text{unit list}) \times \\ &\quad 'f \text{ protocol-channel}) \text{ option} \Rightarrow \text{bool} \end{aligned}$$

where

checked-staged-security-with-actual-alpha-prefix-not-prefix-no-prefix-candidate

$$\begin{aligned} &\text{out} \longleftrightarrow \\ &(\text{case out of} \\ &\quad \text{None} \Rightarrow \text{False} \\ &| \text{Some } (x, -) \Rightarrow \\ &\quad (\text{let packed} = \text{fst } (\text{fst } x); \\ &\quad \quad \text{prefix-state} = \text{snd } (\text{fst } \text{packed}); \\ &\quad \quad \text{data} = \text{snd } \text{packed}; \\ &\quad \quad \text{attacker-state} = \text{snd } (\text{fst } x); \end{aligned}$$

$s =$
 $\text{verifier-state-from-adversary } \text{attacker-state}$
 $(\text{staged-proof-transcript } \text{data})$
 $\text{in } \exists \text{fr } f\text{-fri-roots } f\text{-final } \text{trace-table } \text{witness-final-state}$
 trace-openings.
 $\text{alpha-header-supported-partial-trace-table-candidate-witness}$
 $s \text{ fr } f\text{-fri-roots } f\text{-final } \text{trace-table } \text{witness-final-state}$
 $\text{trace-openings } \wedge$
 $\text{trace-table } \notin$
 $\text{alpha-prefix-trace-table-candidates } \text{prefix-state } \text{fr } \wedge$
 $\text{prefix-state } \leq \text{witness-final-state } \wedge$
 $\text{alpha-prefix-trace-table-candidates } \text{prefix-state } \text{fr} = \{\}$

definition *checked-staged-security-with-actual-alpha-prefix-not-prefix-path-output*

$:: ((((((f \times 'f \text{ list } \times 'f \text{ list } \times 'f) \times$
 $'f \text{ protocol-channel}) \times 'f \text{ staged-proof-data}) \times$
 $'f \text{ protocol-channel}) \times \text{unit list}) \times$
 $'f \text{ protocol-channel}) \text{ option} \Rightarrow \text{bool}$

where

$\text{checked-staged-security-with-actual-alpha-prefix-not-prefix-path-output}$
 $\text{out} \longleftrightarrow$
 $(\text{case out of}$
 $\text{None} \Rightarrow \text{False}$
 $| \text{Some } (x, -) \Rightarrow$
 $(\text{let packed} = \text{fst } (\text{fst } x);$
 $\text{prefix-state} = \text{snd } (\text{fst } \text{packed});$
 $\text{data} = \text{snd } \text{packed};$
 $\text{attacker-state} = \text{snd } (\text{fst } x);$
 $s =$
 $\text{verifier-state-from-adversary } \text{attacker-state}$
 $(\text{staged-proof-transcript } \text{data})$
 $\text{in } \exists \text{fr } f\text{-fri-roots } f\text{-final } \text{trace-table } \text{witness-final-state}$
 trace-openings.
 $\text{alpha-header-supported-partial-trace-table-candidate-witness}$
 $s \text{ fr } f\text{-fri-roots } f\text{-final } \text{trace-table } \text{witness-final-state}$
 $\text{trace-openings } \wedge$
 $\text{trace-table } \notin$
 $\text{alpha-prefix-trace-table-candidates } \text{prefix-state } \text{fr } \wedge$
 $\text{prefix-state } \leq \text{witness-final-state } \wedge$
 $(\exists i < \text{rounds. } \exists \text{opn} \in \text{set } (\text{trace-openings } ! i).$
 $\text{hash-map-new-output-hit}$
 $(\text{merkle-path-target-roots } \text{witness-final-state } \text{fr}$
 $(\text{scale} * \text{clength}) (\text{opening-index } \text{opn})$
 $(\text{opening-value } \text{opn}) (\text{opening-path } \text{opn}))$
 $\text{prefix-state } \text{witness-final-state}))$

definition *checked-staged-security-with-actual-alpha-prefix-witnessed-cross-or-merkle-bad*

$:: ((((((f \times 'f \text{ list } \times 'f \text{ list } \times 'f) \times$
 $'f \text{ protocol-channel}) \times 'f \text{ staged-proof-data}) \times$

'f protocol-channel) × unit list) ×
'f protocol-channel) option ⇒ bool
where
checked-staged-security-with-actual-alpha-prefix-witnessed-cross-or-merkle-bad
out $\longleftrightarrow$
(case out of
None ⇒ False
| Some (x, -) ⇒
(let packed = fst (fst x);
data = snd packed;
attacker-state = snd (fst x);
s =
verifier-state-from-adversary attacker-state
(staged-proof-transcript data)
in alpha-supported-witnessed-partial-trace-pairwise-cross-or-merkle-bad
s))

definition *checked-staged-security-with-actual-alpha-prefix-witness-transcript-hit*
 $:: ((((((f \times 'f \text{ list} \times 'f \text{ list} \times 'f) \times$
 $\quad 'f \text{ protocol-channel}) \times 'f \text{ staged-proof-data}) \times$
 $\quad 'f \text{ protocol-channel}) \times \text{unit list}) \times$
 $\quad 'f \text{ protocol-channel}) \text{ option} \Rightarrow \text{bool}$

where
checked-staged-security-with-actual-alpha-prefix-witness-transcript-hit
out $\longleftrightarrow$
(case out of
None ⇒ False
| Some (x, -) ⇒
(let packed = fst (fst x);
data = snd packed;
attacker-state = snd (fst x);
s =
verifier-state-from-adversary attacker-state
(staged-proof-transcript data)
in ∃ witness-result witness-state.
Some (witness-result, witness-state) ∈
set-dist (execute verify-monad s) ∧
(hash-map-output-values s ∩
set (staged-proof-transcript data) ≠ {} ∨
hash-map-new-output-hit (set (staged-proof-transcript data))
s witness-state)))

definition *checked-staged-security-with-actual-alpha-prefix-witness-transcript-pre-hit*
 $:: ((((((f \times 'f \text{ list} \times 'f \text{ list} \times 'f) \times$
 $\quad 'f \text{ protocol-channel}) \times 'f \text{ staged-proof-data}) \times$
 $\quad 'f \text{ protocol-channel}) \times \text{unit list}) \times$
 $\quad 'f \text{ protocol-channel}) \text{ option} \Rightarrow \text{bool}$

where
checked-staged-security-with-actual-alpha-prefix-witness-transcript-pre-hit

```

    out  $\longleftrightarrow$ 
  (case out of
    None  $\Rightarrow$  False
  | Some (x, -)  $\Rightarrow$ 
    (let packed = fst (fst x);
      data = snd packed;
      attacker-state = snd (fst x);
      s =
        verifier-state-from-adversary attacker-state
          (staged-proof-transcript data)
      in hash-map-output-values s  $\cap$ 
        set (staged-proof-transcript data)  $\neq$  {}))

```

definition *checked-staged-security-with-actual-alpha-prefix-witness-transcript-new-hit*

```

:: ((((((f  $\times$  'f list  $\times$  'f list  $\times$  'f)  $\times$ 
  'f protocol-channel)  $\times$  'f staged-proof-data)  $\times$ 
  'f protocol-channel)  $\times$  unit list)  $\times$ 
  'f protocol-channel) option  $\Rightarrow$  bool

```

where

```

checked-staged-security-with-actual-alpha-prefix-witness-transcript-new-hit
  out  $\longleftrightarrow$ 
  (case out of
    None  $\Rightarrow$  False
  | Some (x, -)  $\Rightarrow$ 
    (let packed = fst (fst x);
      data = snd packed;
      attacker-state = snd (fst x);
      s =
        verifier-state-from-adversary attacker-state
          (staged-proof-transcript data)
      in  $\exists$  witness-result witness-state.
        Some (witness-result, witness-state)  $\in$ 
          set-dist (execute verify-monad s)  $\wedge$ 
          hash-map-new-output-hit (set (staged-proof-transcript data))
            s witness-state))

```

lemma *checked-staged-security-with-actual-alpha-prefix-witness-transcript-hit-imp-pre-or-new:*

assumes

```

  checked-staged-security-with-actual-alpha-prefix-witness-transcript-hit
  out

```

shows

```

  checked-staged-security-with-actual-alpha-prefix-witness-transcript-pre-hit
  out  $\vee$ 
  checked-staged-security-with-actual-alpha-prefix-witness-transcript-new-hit
  out

```

using *assms*

unfolding

```

  checked-staged-security-with-actual-alpha-prefix-witness-transcript-hit-def
  checked-staged-security-with-actual-alpha-prefix-witness-transcript-pre-hit-def

```

checked-staged-security-with-actual-alpha-prefix-witness-transcript-new-hit-def
by (*auto split: option.splits prod.splits*)

lemma *checked-staged-security-with-actual-alpha-prefix-not-prefix-path-output-imp-witness-transcript-hit:*
assumes

checked-staged-security-with-actual-alpha-prefix-not-prefix-path-output
out

shows

checked-staged-security-with-actual-alpha-prefix-witness-transcript-hit
out

proof (*cases out*)

case *None*

then show *?thesis*

using *assms*

unfolding

checked-staged-security-with-actual-alpha-prefix-not-prefix-path-output-def
checked-staged-security-with-actual-alpha-prefix-witness-transcript-hit-def

by *simp*

next

case (*Some packed*)

then obtain *prefix prefix-state data attacker-state result final-state*

where *out-eq:*

out =
Some (((((prefix, prefix-state), data), attacker-state), result),
final-state)

by (*cases packed, auto split: prod.splits*)

let *?s =*

verifier-state-from-adversary attacker-state
(staged-proof-transcript data)

from *assms obtain fr f-fri-roots f-final trace-table witness-final-state*

trace-openings where witness:

alpha-header-supported-partial-trace-table-candidate-witness ?s fr
f-fri-roots f-final trace-table witness-final-state trace-openings

unfolding *out-eq*

checked-staged-security-with-actual-alpha-prefix-not-prefix-path-output-def
Let-def

by (*auto split: option.splits prod.splits*)

from *witness obtain witness-result query-idxs as dg composition-fri-roots*

final rest where outcome:

Some (witness-result, witness-final-state) ∈
set-dist (execute verify-monad ?s)

unfolding *alpha-header-supported-partial-trace-table-candidate-witness-def*

by *blast*

have *tr-eq: PTranscript ?s = staged-proof-transcript data*

by *simp*

have *transcript-hit:*

hash-map-output-values ?s ∩ set (staged-proof-transcript data)
 $\neq \{\}$ $\vee$

hash-map-new-output-hit (set (staged-proof-transcript data))

```

      ?s witness-final-state
    by (rule verify-monad-trace-root-target-preexisting-or-new
        [OF outcome tr-eq])
  show ?thesis
    unfolding out-eq
      checked-staged-security-with-actual-alpha-prefix-witness-transcript-hit-def
      Let-def
    using outcome transcript-hit
    by (auto split: option.splits prod.splits)
qed

lemma checked-staged-security-with-actual-alpha-prefix-not-prefix-path-output-le-witness-transcript-hit:
  wp-event
  (checked-staged-security-experiment-with-actual-alpha-prefix-data-state A)
  checked-staged-security-with-actual-alpha-prefix-not-prefix-path-output
  adversary-initial-state ≤
  wp-event
  (checked-staged-security-experiment-with-actual-alpha-prefix-data-state A)
  checked-staged-security-with-actual-alpha-prefix-witness-transcript-hit
  adversary-initial-state
  by (rule wp-event-mono)
  (rule
    checked-staged-security-with-actual-alpha-prefix-not-prefix-path-output-imp-witness-transcript-hit)

lemma checked-staged-security-with-actual-alpha-prefix-not-prefix-no-prefix-candidate-imp-witness-transcript-hit:
  assumes
    checked-staged-security-with-actual-alpha-prefix-not-prefix-no-prefix-candidate
    out
  shows
    checked-staged-security-with-actual-alpha-prefix-witness-transcript-hit
    out
  proof (cases out)
  case None
  then show ?thesis
    using assms
    unfolding
      checked-staged-security-with-actual-alpha-prefix-not-prefix-no-prefix-candidate-def
      checked-staged-security-with-actual-alpha-prefix-witness-transcript-hit-def
    by simp
  next
  case (Some packed)
  then obtain prefix prefix-state data attacker-state result final-state
    where out-eq:
      out =
        Some (((((prefix, prefix-state), data), attacker-state), result),
              final-state)
    by (cases packed, auto split: prod.splits)
  let ?s =
    verifier-state-from-adversary attacker-state

```



```

    (staged-proof-transcript data)
from assms obtain fr f-fri-roots f-final trace-table witness-final-state
    trace-openings where witness:
    alpha-header-supported-partial-trace-table-candidate-witness ?s fr
    f-fri-roots f-final trace-table witness-final-state trace-openings
unfolding out-eq
    checked-staged-security-with-actual-alpha-prefix-not-prefix-no-prefix-candidate-def
    Let-def
by (auto split: option.splits prod.splits)
from witness obtain witness-result query-idxs as dg composition-fri-roots
    final rest where outcome:
    Some (witness-result, witness-final-state) ∈
    set-dist (execute verify-monad ?s)
unfolding alpha-header-supported-partial-trace-table-candidate-witness-def
by blast
have tr-eq: PTranscript ?s = staged-proof-transcript data
by simp
have transcript-hit:
    hash-map-output-values ?s ∩ set (staged-proof-transcript data)
    ≠ {} ∨
    hash-map-new-output-hit (set (staged-proof-transcript data))
    ?s witness-final-state
by (rule verify-monad-trace-root-target-preexisting-or-new
    [OF outcome tr-eq])
show ?thesis
unfolding out-eq
    checked-staged-security-with-actual-alpha-prefix-witness-transcript-hit-def
    Let-def
using outcome transcript-hit
by (auto split: option.splits prod.splits)
qed

lemma checked-staged-security-with-actual-alpha-prefix-not-prefix-no-prefix-candidate-le-witness-transcript-hit:
    wp-event
    (checked-staged-security-experiment-with-actual-alpha-prefix-data-state A)
    checked-staged-security-with-actual-alpha-prefix-not-prefix-no-prefix-candidate
    adversary-initial-state ≤
    wp-event
    (checked-staged-security-experiment-with-actual-alpha-prefix-data-state A)
    checked-staged-security-with-actual-alpha-prefix-witness-transcript-hit
    adversary-initial-state
by (rule wp-event-mono)
    (rule
    checked-staged-security-with-actual-alpha-prefix-not-prefix-no-prefix-candidate-imp-witness-transcript-hit)

lemma checked-staged-security-with-actual-alpha-prefix-not-prefix-uncovered-index-imp-witnessed-cross-or-merkle-bac
assumes
    checked-staged-security-with-actual-alpha-prefix-not-prefix-uncovered-index
    out

```

```

shows
  checked-staged-security-with-actual-alpha-prefix-witnessed-cross-or-merkle-bad
  out
proof (cases out)
case None
then show ?thesis
  using assms
  unfolding
    checked-staged-security-with-actual-alpha-prefix-not-prefix-uncovered-index-def
  by simp
next
case (Some packed)
then obtain prefix prefix-state data attacker-state result final-state
  where out-eq:
    out =
      Some (((((prefix, prefix-state), data), attacker-state), result),
        final-state)
  by (cases packed, auto split: prod.splits)
let ?s =
  verifier-state-from-adversary attacker-state
  (staged-proof-transcript data)
from assms obtain fr f-fri-roots f-final trace-table witness-final-state
  trace-openings j where witness:
    alpha-header-supported-partial-trace-table-candidate-witness ?s fr
    f-fri-roots f-final trace-table witness-final-state trace-openings
and j-bound: j < scale * clength
and unopened:
   $\bigwedge i \text{ opn. } i < \text{rounds} \implies$ 
   $\text{opn} \in \text{set } (\text{trace-openings } ! i) \implies$ 
   $\text{opening-index opn} \neq j$ 
unfolding out-eq
  checked-staged-security-with-actual-alpha-prefix-not-prefix-uncovered-index-def
  Let-def
by (auto split: option.splits prod.splits)
from witness obtain result' query-idxs as dg composition-fri-roots final rest
where outcome:
  Some (result', witness-final-state)  $\in$  set-dist (execute verify-monad ?s)
and partial:
  accepted-with-partial-trace-openings ?s
  (Some (result', witness-final-state)) fr query-idxs trace-openings
and candidate: partial-trace-table-candidate trace-table trace-openings
and header:
  verifier-header-transcript ?s fr f-fri-roots f-final as dg
  composition-fri-roots final rest
unfolding alpha-header-supported-partial-trace-table-candidate-witness-def
by blast
have not-singleton:
 $\neg (\exists \text{ trace-table0.}$ 
  alpha-header-supported-partial-trace-table-candidates ?s fr f-fri-roots

```

$f\text{-final} \subseteq \{\text{trace-table0}\})$
by (rule
 $\alpha\text{-header-supported-partial-trace-table-candidates-not-singleton-if-trace-unopened}$
 $[OF \text{ outcome partial candidate header j-bound unopened}]$)
have $\text{not-singleton-witnessed}$:
 $\neg (\exists \text{ trace-table0}.$
 $\alpha\text{-header-supported-witnessed-partial-trace-table-candidates } ?s \text{ fr}$
 $f\text{-fri-roots } f\text{-final} \subseteq \{\text{trace-table0}\})$
using not-singleton
by ($\text{simp add: } \alpha\text{-header-supported-witnessed-candidates-eq-candidates}$)
have
 $\alpha\text{-header-supported-witnessed-partial-trace-pairwise-cross-or-merkle-bad}$
 $?s \text{ fr } f\text{-fri-roots } f\text{-final}$
by (rule
 $\alpha\text{-header-supported-witnessed-partial-trace-pairwise-cross-or-merkle-bad-if-not-singleton}$
 $[OF \text{ not-singleton-witnessed}]$)
then have
 $\alpha\text{-supported-witnessed-partial-trace-pairwise-cross-or-merkle-bad } ?s$
unfolding $\alpha\text{-supported-witnessed-partial-trace-pairwise-cross-or-merkle-bad-def}$
by blast
then show $?thesis$
unfolding out-eq
 $\text{checked-staged-security-with-actual-alpha-prefix-witnessed-cross-or-merkle-bad-def}$
 Let-def
by simp
qed

lemma $\text{checked-staged-security-with-actual-alpha-prefix-not-prefix-uncovered-index-le-witnessed-cross-or-merkle-bad}$:
 wp-event
 $(\text{checked-staged-security-experiment-with-actual-alpha-prefix-data-state } A)$
 $\text{checked-staged-security-with-actual-alpha-prefix-not-prefix-uncovered-index}$
 $\text{adversary-initial-state} \leq$
 wp-event
 $(\text{checked-staged-security-experiment-with-actual-alpha-prefix-data-state } A)$
 $\text{checked-staged-security-with-actual-alpha-prefix-witnessed-cross-or-merkle-bad}$
 $\text{adversary-initial-state}$
by (rule wp-event-mono)
(rule
 $\text{checked-staged-security-with-actual-alpha-prefix-not-prefix-uncovered-index-imp-witnessed-cross-or-merkle-bad}$)

lemma $\text{checked-staged-security-with-actual-alpha-prefix-not-prefix-bound-imp-decomposition-on-support}$:
assumes $\text{wf: staged-budget-wellformed budgets}$
and $\text{controlled: staged-adversary-controlled budgets } A$
and support:
 $\text{out} \in$
 set-dist
 $(\text{execute}$
 $(\text{checked-staged-security-experiment-with-actual-alpha-prefix-data-state}$
 $A)$

```

      adversary-initial-state)
and hit:
  checked-staged-security-with-actual-alpha-prefix-partial-header-candidate-not-prefix-bound
  bad-sets out
shows
  checked-staged-security-with-actual-alpha-prefix-not-prefix-collision out ∨
  checked-staged-security-with-actual-alpha-prefix-not-prefix-uncovered-index
  out ∨
  checked-staged-security-with-actual-alpha-prefix-not-prefix-no-prefix-candidate
  out ∨
  checked-staged-security-with-actual-alpha-prefix-not-prefix-path-output
  out
proof (cases out)
case None
then show ?thesis
  using hit
  unfolding
    checked-staged-security-with-actual-alpha-prefix-partial-header-candidate-not-prefix-bound-def
  by simp
next
case (Some packed)
then obtain prefix prefix-state data attacker-state result final-state
  where out-eq:
    out =
      Some (((((prefix, prefix-state), data), attacker-state), result),
        final-state)
  by (cases packed, auto split: prod.splits)
let ?s =
  verifier-state-from-adversary attacker-state
  (staged-proof-transcript data)
have hit-body:
  ∃ as query-idxs fr f-fri-roots f-final dg composition-fri-roots
    final rest.
    accepted-transcript-shape ?s (Some (result, final-state))
    as query-idxs ∧
    verifier-header-transcript ?s fr f-fri-roots f-final as dg
    composition-fri-roots final rest ∧
  as ∈
    alpha-header-supported-partial-union-bad-sets ?s bad-sets fr
    f-fri-roots f-final ∧
  as ∉ alpha-prefix-union-bad-sets prefix-state bad-sets fr ∧
  alpha-header-partial-candidate-not-prefix-bound prefix-state ?s fr
  f-fri-roots f-final
using hit
unfolding out-eq
  checked-staged-security-with-actual-alpha-prefix-partial-header-candidate-not-prefix-bound-def
  Let-def
by simp
from hit-body obtain as query-idxs fr f-fri-roots f-final dg

```

```

    composition-fri-roots final rest where bound:
    alpha-header-partial-candidate-not-prefix-bound prefix-state ?s fr
    f-fri-roots f-final
by blast
have support-some:
    Some (((((prefix, prefix-state), data), attacker-state), result),
    final-state) ∈
    set-dist
    (execute
    (checked-staged-security-experiment-with-actual-alpha-prefix-data-state
    A)
    adversary-initial-state)
using support out-eq by simp
from
    checked-staged-security-with-actual-alpha-prefix-not-prefix-bound-decompose
    [OF wf controlled support-some bound]
show ?thesis
proof
    assume collision: hash-map-output-collision prefix-state
    have
    checked-staged-security-with-actual-alpha-prefix-not-prefix-collision
    out
    unfolding out-eq
    checked-staged-security-with-actual-alpha-prefix-not-prefix-collision-def
    Let-def
    using collision by simp
    then show ?thesis by simp
next
    assume witness-cases:
    ∃ trace-table witness-final-state trace-openings.
    alpha-header-supported-partial-trace-table-candidate-witness ?s fr
    f-fri-roots f-final trace-table witness-final-state
    trace-openings ∧
    trace-table ∉
    alpha-prefix-trace-table-candidates prefix-state fr ∧
    prefix-state ≤ witness-final-state ∧
    ((∃ j < scale * clength.
    ∀ i opn.
    i < rounds ⟶
    opn ∈ set (trace-openings ! i) ⟶
    opening-index opn ≠ j) ∨
    alpha-prefix-trace-table-candidates prefix-state fr = {} ∨
    (∃ i < rounds. ∃ opn ∈ set (trace-openings ! i).
    hash-map-new-output-hit
    (merkle-path-target-roots witness-final-state fr
    (scale * clength) (opening-index opn)
    (opening-value opn) (opening-path opn))
    prefix-state witness-final-state))
from witness-cases obtain trace-table witness-final-state trace-openings

```

```

where witness:
  alpha-header-supported-partial-trace-table-candidate-witness ?s fr
    f-fri-roots f-final trace-table witness-final-state
    trace-openings
and not-prefix:
  trace-table  $\notin$ 
    alpha-prefix-trace-table-candidates prefix-state fr
and ext: prefix-state  $\leq$  witness-final-state
and cases:
  ( $\exists j < \text{scale} * \text{clength}$ .
     $\forall i \text{ opn}$ .
       $i < \text{rounds} \longrightarrow$ 
         $\text{opn} \in \text{set } (\text{trace-openings} ! i) \longrightarrow$ 
           $\text{opening-index opn} \neq j) \vee$ 
    alpha-prefix-trace-table-candidates prefix-state fr = {}  $\vee$ 
    ( $\exists i < \text{rounds}$ .  $\exists \text{opn} \in \text{set } (\text{trace-openings} ! i)$ .
      hash-map-new-output-hit
        (merkle-path-target-roots witness-final-state fr
          (scale * clength) (opening-index opn)
          (opening-value opn) (opening-path opn))
        prefix-state witness-final-state)
  by blast
from cases show ?thesis
proof
  assume uncovered:
     $\exists j < \text{scale} * \text{clength}$ .
       $\forall i \text{ opn}$ .
         $i < \text{rounds} \longrightarrow$ 
           $\text{opn} \in \text{set } (\text{trace-openings} ! i) \longrightarrow$ 
             $\text{opening-index opn} \neq j$ 
  have body:
     $\exists \text{fr}' \text{ f-fri-roots}' \text{ f-final}' \text{ trace-table}'$ 
      witness-final-state' trace-openings'.
    alpha-header-supported-partial-trace-table-candidate-witness ?s
      fr' f-fri-roots' f-final' trace-table'
      witness-final-state' trace-openings'  $\wedge$ 
      trace-table'  $\notin$ 
        alpha-prefix-trace-table-candidates prefix-state fr'  $\wedge$ 
        prefix-state  $\leq$  witness-final-state'  $\wedge$ 
        ( $\exists j < \text{scale} * \text{clength}$ .
           $\forall i \text{ opn}$ .  $i < \text{rounds} \longrightarrow$ 
             $\text{opn} \in \text{set } (\text{trace-openings}' ! i) \longrightarrow$ 
               $\text{opening-index opn} \neq j$ )
  using witness not-prefix ext uncovered by blast
have
  checked-staged-security-with-actual-alpha-prefix-not-prefix-uncovered-index
    out
unfolding out-eq
  checked-staged-security-with-actual-alpha-prefix-not-prefix-uncovered-index-def

```

```

    Let-def
    using body by (simp split: prod.splits)
  then show ?thesis by simp
next
assume rest:
  alpha-prefix-trace-table-candidates prefix-state fr = {} ∨
  (∃ i < rounds. ∃ opn ∈ set (trace-openings ! i).
    hash-map-new-output-hit
      (merkle-path-target-roots witness-final-state fr
        (scale * clength) (opening-index opn)
        (opening-value opn) (opening-path opn))
      prefix-state witness-final-state)
then show ?thesis
proof
  assume no-prefix:
    alpha-prefix-trace-table-candidates prefix-state fr = {}
  have body:
    ∃ fr' f-fri-roots' f-final' trace-table'
      witness-final-state' trace-openings'.
    alpha-header-supported-partial-trace-table-candidate-witness ?s
      fr' f-fri-roots' f-final' trace-table'
      witness-final-state' trace-openings' ∧
    trace-table' ∉
      alpha-prefix-trace-table-candidates prefix-state fr' ∧
    prefix-state ≤ witness-final-state' ∧
    alpha-prefix-trace-table-candidates prefix-state fr' = {}
  using witness not-prefix ext no-prefix by blast
  have
    checked-staged-security-with-actual-alpha-prefix-not-prefix-no-prefix-candidate
      out
    unfolding out-eq
      checked-staged-security-with-actual-alpha-prefix-not-prefix-no-prefix-candidate-def
      Let-def
    using body by (simp split: prod.splits)
  then show ?thesis by simp
next
assume path:
  ∃ i < rounds. ∃ opn ∈ set (trace-openings ! i).
  hash-map-new-output-hit
    (merkle-path-target-roots witness-final-state fr
      (scale * clength) (opening-index opn)
      (opening-value opn) (opening-path opn))
  prefix-state witness-final-state
  have body:
    ∃ fr' f-fri-roots' f-final' trace-table'
      witness-final-state' trace-openings'.
    alpha-header-supported-partial-trace-table-candidate-witness ?s
      fr' f-fri-roots' f-final' trace-table'
      witness-final-state' trace-openings' ∧

```

```

    trace-table'  $\notin$ 
      alpha-prefix-trace-table-candidates prefix-state fr'  $\wedge$ 
      prefix-state  $\leq$  witness-final-state'  $\wedge$ 
      ( $\exists i < \text{rounds}. \exists \text{opn} \in \text{set } (\text{trace-openings}' ! i).$ 
        hash-map-new-output-hit
          (merkle-path-target-roots witness-final-state' fr'
            (scale * clength) (opening-index opn)
            (opening-value opn) (opening-path opn))
          prefix-state witness-final-state')
    using witness not-prefix ext path by blast
  have
    checked-staged-security-with-actual-alpha-prefix-not-prefix-path-output
      out
    unfolding out-eq
      checked-staged-security-with-actual-alpha-prefix-not-prefix-path-output-def
      Let-def
    using body by (simp split: prod.splits)
  then show ?thesis by simp
qed
qed
qed
qed

lemma checked-staged-security-with-actual-alpha-prefix-not-prefix-bound-union-bound:
  fixes C U N P :: prob
  assumes wf: staged-budget-wellformed budgets
  and controlled: staged-adversary-controlled budgets A
  and collision-bound:
    wp-event
      (checked-staged-security-experiment-with-actual-alpha-prefix-data-state
        A)
    checked-staged-security-with-actual-alpha-prefix-not-prefix-collision
    adversary-initial-state  $\leq$  C
  and uncovered-bound:
    wp-event
      (checked-staged-security-experiment-with-actual-alpha-prefix-data-state
        A)
    checked-staged-security-with-actual-alpha-prefix-not-prefix-uncovered-index
    adversary-initial-state  $\leq$  U
  and no-prefix-bound:
    wp-event
      (checked-staged-security-experiment-with-actual-alpha-prefix-data-state
        A)
    checked-staged-security-with-actual-alpha-prefix-not-prefix-no-prefix-candidate
    adversary-initial-state  $\leq$  N
  and path-bound:
    wp-event
      (checked-staged-security-experiment-with-actual-alpha-prefix-data-state
        A)

```


checked-staged-security-with-actual-alpha-prefix-not-prefix-path-output
adversary-initial-state $\leq P$

shows

wp-event
 (*checked-staged-security-experiment-with-actual-alpha-prefix-data-state*
A)
 (*checked-staged-security-with-actual-alpha-prefix-partial-header-candidate-not-prefix-bound*
bad-sets)
adversary-initial-state $\leq C + U + N + P$

proof –

let $?M =$
checked-staged-security-experiment-with-actual-alpha-prefix-data-state A
let $?B =$
checked-staged-security-with-actual-alpha-prefix-partial-header-candidate-not-prefix-bound
bad-sets
let $?C =$ *checked-staged-security-with-actual-alpha-prefix-not-prefix-collision*
let $?U =$
checked-staged-security-with-actual-alpha-prefix-not-prefix-uncovered-index
let $?N =$
checked-staged-security-with-actual-alpha-prefix-not-prefix-no-prefix-candidate
let $?P =$ *checked-staged-security-with-actual-alpha-prefix-not-prefix-path-output*
have *wp-event ?M ?B adversary-initial-state* $\leq$
wp-event ?M ($\lambda out. ?C \text{ out} \vee ?U \text{ out} \vee ?N \text{ out} \vee ?P \text{ out}$)
adversary-initial-state
by (rule *wp-event-mono-on-support*)
 (rule
checked-staged-security-with-actual-alpha-prefix-not-prefix-bound-imp-decomposition-on-support
 [OF *wf controlled*])
also have ... $\leq$
wp-event ?M ?C adversary-initial-state +
wp-event ?M ?U adversary-initial-state +
wp-event ?M ?N adversary-initial-state +
wp-event ?M ?P adversary-initial-state
by (rule *wp-event-union-bound4*)
also have ... $\leq C + U + N + P$
by (intro *add-mono collision-bound uncovered-bound no-prefix-bound*
path-bound)
finally show *?thesis* .

qed

lemma *checked-staged-security-with-actual-alpha-prefix-partial-header-candidate-drift-imp-not-prefix-bound-on-support*
assumes *wf: staged-budget-wellformed budgets*
and *controlled: staged-adversary-controlled budgets A*
and *support:*
out $\in$
set-dist
 (execute
 (*checked-staged-security-experiment-with-actual-alpha-prefix-data-state*
A)

```

      adversary-initial-state)
and drift:
  checked-staged-security-with-actual-alpha-prefix-partial-header-candidate-drift
  bad-sets out
shows
  checked-staged-security-with-actual-alpha-prefix-partial-header-candidate-not-prefix-bound
  bad-sets out
proof (cases out)
case None
then show ?thesis
using drift
unfolding
  checked-staged-security-with-actual-alpha-prefix-partial-header-candidate-drift-def
  checked-staged-security-with-actual-alpha-prefix-partial-header-fresh-hit-def
  checked-staged-security-with-actual-alpha-prefix-partial-header-bad-set-hit-def
by simp
next
case (Some packed-out)
then obtain prefix prefix-state data attacker-state result final-state
where out-eq:
  out =
    Some (((((prefix, prefix-state), data), attacker-state), result),
      final-state)
by (cases packed-out, auto split: prod.splits)
let ?s =
  verifier-state-from-adversary attacker-state
  (staged-proof-transcript data)
have partial-hit:
  checked-staged-security-with-actual-alpha-prefix-partial-header-bad-set-hit
  bad-sets out
using drift
unfolding
  checked-staged-security-with-actual-alpha-prefix-partial-header-candidate-drift-def
  checked-staged-security-with-actual-alpha-prefix-partial-header-fresh-hit-def
by simp
have not-prefix-event:
  ¬ checked-staged-security-with-actual-alpha-prefix-bad-set-hit
  bad-sets out
using drift
unfolding
  checked-staged-security-with-actual-alpha-prefix-partial-header-candidate-drift-def
by simp
have builder:
  Some (data, attacker-state) ∈
  set-dist
  (execute (checked-staged-transcript-program A)
    adversary-initial-state)
proof —
  have data-support:

```

```

Some (((data, attacker-state), result), final-state) ∈
  set-dist
  (execute (checked-staged-security-experiment-with-data-state A)
    adversary-initial-state)
by (rule
  checked-staged-security-with-actual-alpha-prefix-support-imp-data-state-support
  [OF support[unfolded out-eq]])
from checked-staged-security-experiment-with-data-state-outcomeE
  [OF data-support]
show ?thesis
  by blast
qed
have staged-header:
  verifier-header-transcript ?s
  (staged-trace-root data)
  (staged-trace-fri-roots data)
  (staged-trace-final data)
  (staged-alphas data)
  (staged-degree data)
  (staged-composition-fri-roots data)
  (staged-composition-final data)
  (List.concat (staged-query-chunks data))
  by (rule checked-staged-transcript-program-outcome-header-transcript
    [OF wf controlled builder])
have prefix-root-eq: staged-trace-root data = fst prefix
  using support out-eq
  unfolding checked-staged-security-experiment-with-actual-alpha-prefix-data-state-def
    checked-staged-transcript-with-alpha-prefix-program-def
    checked-staged-after-alpha-prefix-program-def Let-def
  by (auto elim!: set-dist-bindE split: prod.splits)
have alpha-header-hit:
  alpha-header-list-set-hit ?s
  (alpha-header-supported-partial-union-bad-sets ?s bad-sets)
  (Some (result, final-state))
  using partial-hit unfolding out-eq
    checked-staged-security-with-actual-alpha-prefix-partial-header-bad-set-hit-def
  by (simp add: Let-def)
from alpha-header-hit obtain as query-idxs fr f-fri-roots f-final dg
  composition-fri-roots final rest where shape:
  accepted-transcript-shape ?s (Some (result, final-state)) as query-idxs
and header:
  verifier-header-transcript ?s fr f-fri-roots f-final as dg
  composition-fri-roots final rest
and hit:
as ∈
  alpha-header-supported-partial-union-bad-sets ?s bad-sets fr
  f-fri-roots f-final
  unfolding alpha-header-list-set-hit-def by blast
have as-eq: as = staged-alphas data

```

```

using verifier-header-transcript-unique[OF staged-header header]
by simp
have fr-eq: fr = staged-trace-root data
  using verifier-header-transcript-unique[OF header staged-header]
  by simp
have not-prefix:
  as  $\notin$  alpha-prefix-union-bad-sets prefix-state bad-sets fr
proof
  assume as-prefix:
    as  $\in$  alpha-prefix-union-bad-sets prefix-state bad-sets fr
  have
    staged-alphas data  $\in$ 
      alpha-prefix-union-bad-sets prefix-state bad-sets (fst prefix)
    using as-prefix as-eq fr-eq prefix-root-eq by simp
  then show False
    using not-prefix-event unfolding out-eq
      checked-staged-security-with-actual-alpha-prefix-bad-set-hit-def
    by simp
qed
have not-bound:
  alpha-header-partial-candidate-not-prefix-bound prefix-state ?s fr
  f-fri-roots f-final
by (rule alpha-header-partial-union-hit-not-prefix-hit-imp-not-prefix-bound
  [OF hit not-prefix])
show ?thesis
  unfolding out-eq
by (rule
  checked-staged-security-with-actual-alpha-prefix-partial-header-candidate-not-prefix-boundI
  [OF refl shape header hit not-prefix not-bound])
qed

lemma checked-staged-security-with-actual-alpha-prefix-partial-header-candidate-drift-le-not-prefix-bound:
assumes wf: staged-budget-wellformed budgets
and controlled: staged-adversary-controlled budgets A
shows
  wp-event
    (checked-staged-security-experiment-with-actual-alpha-prefix-data-state A)
    (checked-staged-security-with-actual-alpha-prefix-partial-header-candidate-drift
      bad-sets)
  adversary-initial-state  $\leq$ 
  wp-event
    (checked-staged-security-experiment-with-actual-alpha-prefix-data-state A)
    (checked-staged-security-with-actual-alpha-prefix-partial-header-candidate-not-prefix-bound
      bad-sets)
  adversary-initial-state
by (rule wp-event-mono-on-support)
  (rule
  checked-staged-security-with-actual-alpha-prefix-partial-header-candidate-drift-imp-not-prefix-bound-on-support
  [OF wf controlled])

```

lemma *checked-staged-security-with-data-state-composition-bad-bound-from-candidate-not-prefix-and-budgets:*

assumes *wf: staged-budget-wellformed budgets*

and *controlled: staged-adversary-controlled budgets A*

and *not-prefix-bound:*

wp-event

(*checked-staged-security-experiment-with-actual-alpha-prefix-data-state*
A)

(*checked-staged-security-with-actual-alpha-prefix-partial-header-candidate-not-prefix-bound*
composition-trace-bad-alpha-space)

adversary-initial-state $\leq D$

shows

wp-event (*checked-staged-security-experiment-with-data-state A*)

(*staged-security-with-data-state-verifier-event composition-bad*)

adversary-initial-state $\leq$

composition-error-bound +

staged-phase-relation-error size

(*staged-alpha-search-queries budgets 0*) +

hash-collision-budget-value 0

(*staged-alpha-search-queries budgets 0*) +

D +

staged-phase-relation-error size

(*staged-alpha-search-queries budgets 0*) +

hash-collision-budget-value 0

(*staged-attacker-query-budget budgets* +

staged-challenge-query-budget + *verifier-hash-query-budget*)

proof –

have *drift-bound:*

wp-event

(*checked-staged-security-experiment-with-actual-alpha-prefix-data-state*
A)

(*checked-staged-security-with-actual-alpha-prefix-partial-header-candidate-drift*
composition-trace-bad-alpha-space)

adversary-initial-state $\leq D$

proof –

have *wp-event*

(*checked-staged-security-experiment-with-actual-alpha-prefix-data-state*
A)

(*checked-staged-security-with-actual-alpha-prefix-partial-header-candidate-drift*
composition-trace-bad-alpha-space)

adversary-initial-state $\leq$

wp-event

(*checked-staged-security-experiment-with-actual-alpha-prefix-data-state*
A)

(*checked-staged-security-with-actual-alpha-prefix-partial-header-candidate-not-prefix-bound*
composition-trace-bad-alpha-space)

adversary-initial-state

by (rule

checked-staged-security-with-actual-alpha-prefix-partial-header-candidate-drift-le-not-prefix-bound

```

    [OF wf controlled])
  also have ...  $\leq D$ 
    by (rule not-prefix-bound)
  finally show ?thesis .
qed
show ?thesis
  by (rule
    checked-staged-security-with-data-state-composition-bad-bound-from-partial-header-candidate-drift-and-budgets
    [OF wf controlled drift-bound])
qed

lemma checked-staged-security-with-data-state-composition-bad-bound-from-not-prefix-components-and-budgets:
fixes  $C\ U\ N\ P :: \text{prob}$ 
assumes wf: staged-budget-wellformed budgets
and controlled: staged-adversary-controlled budgets  $A$ 
and collision-bound:
  wp-event
    (checked-staged-security-experiment-with-actual-alpha-prefix-data-state
       $A$ )
    checked-staged-security-with-actual-alpha-prefix-not-prefix-collision
    adversary-initial-state  $\leq C$ 
and uncovered-bound:
  wp-event
    (checked-staged-security-experiment-with-actual-alpha-prefix-data-state
       $A$ )
    checked-staged-security-with-actual-alpha-prefix-not-prefix-uncovered-index
    adversary-initial-state  $\leq U$ 
and no-prefix-bound:
  wp-event
    (checked-staged-security-experiment-with-actual-alpha-prefix-data-state
       $A$ )
    checked-staged-security-with-actual-alpha-prefix-not-prefix-no-prefix-candidate
    adversary-initial-state  $\leq N$ 
and path-bound:
  wp-event
    (checked-staged-security-experiment-with-actual-alpha-prefix-data-state
       $A$ )
    checked-staged-security-with-actual-alpha-prefix-not-prefix-path-output
    adversary-initial-state  $\leq P$ 
shows
  wp-event (checked-staged-security-experiment-with-data-state  $A$ )
    (staged-security-with-data-state-verifier-event composition-bad)
    adversary-initial-state  $\leq$ 
    composition-error-bound +
    staged-phase-relation-error size
    (staged-alpha-search-queries budgets 0) +
    hash-collision-budget-value 0
    (staged-alpha-search-queries budgets 0) +
    ( $C + U + N + P$ ) +

```

staged-phase-relation-error size
(staged-alpha-search-queries budgets 0) +
hash-collision-budget-value 0
(staged-attacker-query-budget budgets +
staged-challenge-query-budget + verifier-hash-query-budget)

proof –

have *not-prefix-bound*:

wp-event
(checked-staged-security-experiment-with-actual-alpha-prefix-data-state
A)
(checked-staged-security-with-actual-alpha-prefix-partial-header-candidate-not-prefix-bound
composition-trace-bad-alpha-space)
adversary-initial-state ≤ C + U + N + P

by (rule
checked-staged-security-with-actual-alpha-prefix-not-prefix-bound-union-bound
[OF wf controlled collision-bound uncovered-bound no-prefix-bound
path-bound])

show *?thesis*

by (rule
checked-staged-security-with-data-state-composition-bad-bound-from-candidate-not-prefix-and-budgets
[OF wf controlled not-prefix-bound])

qed

lemma *checked-staged-security-with-data-state-composition-bad-bound-from-not-prefix-noncollision-components-and-*
fixes *U N P :: prob*

assumes *wf: staged-budget-wellformed budgets*

and *controlled: staged-adversary-controlled budgets A*

and *uncovered-bound*:

wp-event
(checked-staged-security-experiment-with-actual-alpha-prefix-data-state
A)
checked-staged-security-with-actual-alpha-prefix-not-prefix-uncovered-index
adversary-initial-state ≤ U

and *no-prefix-bound*:

wp-event
(checked-staged-security-experiment-with-actual-alpha-prefix-data-state
A)
checked-staged-security-with-actual-alpha-prefix-not-prefix-no-prefix-candidate
adversary-initial-state ≤ N

and *path-bound*:

wp-event
(checked-staged-security-experiment-with-actual-alpha-prefix-data-state
A)
checked-staged-security-with-actual-alpha-prefix-not-prefix-path-output
adversary-initial-state ≤ P

shows

wp-event (checked-staged-security-experiment-with-data-state A)
(staged-security-with-data-state-verifier-event composition-bad)
adversary-initial-state ≤

composition-error-bound +
staged-phase-relation-error size
(staged-alpha-search-queries budgets 0) +
hash-collision-budget-value 0
(staged-alpha-search-queries budgets 0) +
(hash-collision-budget-value 0
(staged-alpha-search-queries budgets 0) + U + N + P) +
staged-phase-relation-error size
(staged-alpha-search-queries budgets 0) +
hash-collision-budget-value 0
(staged-attacker-query-budget budgets +
staged-challenge-query-budget + verifier-hash-query-budget)
proof (rule
checked-staged-security-with-data-state-composition-bad-bound-from-not-prefix-components-and-budgets
[OF wf controlled - uncovered-bound no-prefix-bound path-bound])
show
wp-event
(checked-staged-security-experiment-with-actual-alpha-prefix-data-state
A)
checked-staged-security-with-actual-alpha-prefix-not-prefix-collision
adversary-initial-state ≤
hash-collision-budget-value 0 (staged-alpha-search-queries budgets 0)
by (rule
checked-staged-security-with-actual-alpha-prefix-not-prefix-collision-bound
[OF wf controlled])
qed

lemma *checked-staged-security-with-data-state-composition-bad-bound-from-cross-no-prefix-path-components-and-bud*
fixes *X N P :: prob*
assumes *wf: staged-budget-wellformed budgets*
and *controlled: staged-adversary-controlled budgets A*
and *cross-bound:*
wp-event
(checked-staged-security-experiment-with-actual-alpha-prefix-data-state
A)
checked-staged-security-with-actual-alpha-prefix-witnessed-cross-or-merkle-bad
adversary-initial-state ≤ X
and *no-prefix-bound:*
wp-event
(checked-staged-security-experiment-with-actual-alpha-prefix-data-state
A)
checked-staged-security-with-actual-alpha-prefix-not-prefix-no-prefix-candidate
adversary-initial-state ≤ N
and *path-bound:*
wp-event
(checked-staged-security-experiment-with-actual-alpha-prefix-data-state
A)
checked-staged-security-with-actual-alpha-prefix-not-prefix-path-output
adversary-initial-state ≤ P

shows

wp-event (*checked-staged-security-experiment-with-data-state A*)
 (*staged-security-with-data-state-verifier-event composition-bad*)
adversary-initial-state $\leq$
composition-error-bound +
staged-phase-relation-error size
 (*staged-alpha-search-queries budgets 0*) +
hash-collision-budget-value 0
 (*staged-alpha-search-queries budgets 0*) +
 (*hash-collision-budget-value 0*
 (*staged-alpha-search-queries budgets 0*) + $X + N + P$) +
staged-phase-relation-error size
 (*staged-alpha-search-queries budgets 0*) +
hash-collision-budget-value 0
 (*staged-attacker-query-budget budgets* +
staged-challenge-query-budget + *verifier-hash-query-budget*)

proof (*rule*

checked-staged-security-with-data-state-composition-bad-bound-from-not-prefix-noncollision-components-and-budget
 [*OF wf controlled - no-prefix-bound path-bound*])

show

wp-event
 (*checked-staged-security-experiment-with-actual-alpha-prefix-data-state*
A)
checked-staged-security-with-actual-alpha-prefix-not-prefix-uncovered-index
adversary-initial-state $\leq X$

by (*rule order-trans*

[*OF*
checked-staged-security-with-actual-alpha-prefix-not-prefix-uncovered-index-le-witnessed-cross-or-merkle-bad
cross-bound])

qed

lemma *checked-staged-security-with-data-state-composition-bad-bound-from-cross-no-prefix-transcript-components-and*

fixes $X N T :: \text{prob}$

assumes *wf: staged-budget-wellformed budgets*

and *controlled: staged-adversary-controlled budgets A*

and *cross-bound:*

wp-event
 (*checked-staged-security-experiment-with-actual-alpha-prefix-data-state*
A)
checked-staged-security-with-actual-alpha-prefix-witnessed-cross-or-merkle-bad
adversary-initial-state $\leq X$

and *no-prefix-bound:*

wp-event
 (*checked-staged-security-experiment-with-actual-alpha-prefix-data-state*
A)
checked-staged-security-with-actual-alpha-prefix-not-prefix-no-prefix-candidate
adversary-initial-state $\leq N$

and *transcript-bound:*

wp-event

(checked-staged-security-experiment-with-actual-alpha-prefix-data-state
 A)
 checked-staged-security-with-actual-alpha-prefix-witness-transcript-hit
 adversary-initial-state $\leq T$

shows

wp-event (checked-staged-security-experiment-with-data-state A)
 (staged-security-with-data-state-verifier-event composition-bad)
 adversary-initial-state $\leq$
 composition-error-bound +
 staged-phase-relation-error size
 (staged-alpha-search-queries budgets 0) +
 hash-collision-budget-value 0
 (staged-alpha-search-queries budgets 0) +
 (hash-collision-budget-value 0
 (staged-alpha-search-queries budgets 0) + X + N + T) +
 staged-phase-relation-error size
 (staged-alpha-search-queries budgets 0) +
 hash-collision-budget-value 0
 (staged-attacker-query-budget budgets +
 staged-challenge-query-budget + verifier-hash-query-budget)

proof (rule

checked-staged-security-with-data-state-composition-bad-bound-from-cross-no-prefix-path-components-and-budgets
 [OF wf controlled cross-bound no-prefix-bound])

show

wp-event
 (checked-staged-security-experiment-with-actual-alpha-prefix-data-state
 A)
 checked-staged-security-with-actual-alpha-prefix-not-prefix-path-output
 adversary-initial-state $\leq T$

by (rule order-trans

[OF
 checked-staged-security-with-actual-alpha-prefix-not-prefix-path-output-le-witness-transcript-hit
 transcript-bound])

qed

lemma checked-staged-security-with-actual-alpha-prefix-witness-transcript-hit-bound-from-pre-and-new:

assumes pre-bound:

wp-event
 (checked-staged-security-experiment-with-actual-alpha-prefix-data-state
 A)
 checked-staged-security-with-actual-alpha-prefix-witness-transcript-pre-hit
 adversary-initial-state $\leq P$

and new-bound:

wp-event
 (checked-staged-security-experiment-with-actual-alpha-prefix-data-state
 A)
 checked-staged-security-with-actual-alpha-prefix-witness-transcript-new-hit
 adversary-initial-state $\leq Q$

shows

$wp\text{-}event$
 $(checked\text{-}staged\text{-}security\text{-}experiment\text{-}with\text{-}actual\text{-}alpha\text{-}prefix\text{-}data\text{-}state$
 $A)$
 $checked\text{-}staged\text{-}security\text{-}with\text{-}actual\text{-}alpha\text{-}prefix\text{-}witness\text{-}transcript\text{-}hit$
 $adversary\text{-}initial\text{-}state \leq P + Q$

proof –

have $event\text{-}le$:

$wp\text{-}event$
 $(checked\text{-}staged\text{-}security\text{-}experiment\text{-}with\text{-}actual\text{-}alpha\text{-}prefix\text{-}data\text{-}state$
 $A)$
 $checked\text{-}staged\text{-}security\text{-}with\text{-}actual\text{-}alpha\text{-}prefix\text{-}witness\text{-}transcript\text{-}hit$
 $adversary\text{-}initial\text{-}state \leq$
 $wp\text{-}event$
 $(checked\text{-}staged\text{-}security\text{-}experiment\text{-}with\text{-}actual\text{-}alpha\text{-}prefix\text{-}data\text{-}state$
 $A)$
 $(\lambda out.$
 $checked\text{-}staged\text{-}security\text{-}with\text{-}actual\text{-}alpha\text{-}prefix\text{-}witness\text{-}transcript\text{-}pre\text{-}hit$
 $out \vee$
 $checked\text{-}staged\text{-}security\text{-}with\text{-}actual\text{-}alpha\text{-}prefix\text{-}witness\text{-}transcript\text{-}new\text{-}hit$
 $out)$
 $adversary\text{-}initial\text{-}state$

by (rule $wp\text{-}event\text{-}mono$)

(rule

$checked\text{-}staged\text{-}security\text{-}with\text{-}actual\text{-}alpha\text{-}prefix\text{-}witness\text{-}transcript\text{-}hit\text{-}imp\text{-}pre\text{-}or\text{-}new)$

have $union\text{-}bound$:

$wp\text{-}event$
 $(checked\text{-}staged\text{-}security\text{-}experiment\text{-}with\text{-}actual\text{-}alpha\text{-}prefix\text{-}data\text{-}state$
 $A)$
 $(\lambda out.$
 $checked\text{-}staged\text{-}security\text{-}with\text{-}actual\text{-}alpha\text{-}prefix\text{-}witness\text{-}transcript\text{-}pre\text{-}hit$
 $out \vee$
 $checked\text{-}staged\text{-}security\text{-}with\text{-}actual\text{-}alpha\text{-}prefix\text{-}witness\text{-}transcript\text{-}new\text{-}hit$
 $out)$
 $adversary\text{-}initial\text{-}state \leq$
 $wp\text{-}event$
 $(checked\text{-}staged\text{-}security\text{-}experiment\text{-}with\text{-}actual\text{-}alpha\text{-}prefix\text{-}data\text{-}state$
 $A)$
 $checked\text{-}staged\text{-}security\text{-}with\text{-}actual\text{-}alpha\text{-}prefix\text{-}witness\text{-}transcript\text{-}pre\text{-}hit$
 $adversary\text{-}initial\text{-}state +$
 $wp\text{-}event$
 $(checked\text{-}staged\text{-}security\text{-}experiment\text{-}with\text{-}actual\text{-}alpha\text{-}prefix\text{-}data\text{-}state$
 $A)$
 $checked\text{-}staged\text{-}security\text{-}with\text{-}actual\text{-}alpha\text{-}prefix\text{-}witness\text{-}transcript\text{-}new\text{-}hit$
 $adversary\text{-}initial\text{-}state$

by (rule $wp\text{-}event\text{-}union\text{-}bound$)

have $sum\text{-}bound$:

$wp\text{-}event$
 $(checked\text{-}staged\text{-}security\text{-}experiment\text{-}with\text{-}actual\text{-}alpha\text{-}prefix\text{-}data\text{-}state$
 $A)$

```

    checked-staged-security-with-actual-alpha-prefix-witness-transcript-pre-hit
    adversary-initial-state +
    wp-event
    ( checked-staged-security-experiment-with-actual-alpha-prefix-data-state
      A)
    checked-staged-security-with-actual-alpha-prefix-witness-transcript-new-hit
    adversary-initial-state ≤ P + Q
  by (intro add-mono pre-bound new-bound)
show ?thesis
  by (rule order-trans[OF event-le])
    (rule order-trans[OF union-bound sum-bound])
qed

lemma checked-staged-security-with-actual-alpha-prefix-witness-transcript-pre-hit-imp-data-state-pre-hit:
  assumes
    checked-staged-security-with-actual-alpha-prefix-witness-transcript-pre-hit
    (Some (((prefix, prefix-state), data), attacker-state), result),
    final-state))
  shows
    staged-security-with-data-state-transcript-pre-hit
    (Some (((data, attacker-state), result), final-state))
  using assms
  unfolding
    checked-staged-security-with-actual-alpha-prefix-witness-transcript-pre-hit-def
    staged-security-with-data-state-transcript-pre-hit-def
  by simp

lemma checked-staged-security-with-actual-alpha-prefix-witness-transcript-pre-hit-bound-from-data-state-pre-hit:
  assumes pre-bound:
    wp-event (checked-staged-security-experiment-with-data-state A)
    staged-security-with-data-state-transcript-pre-hit
    adversary-initial-state ≤ P
  shows
    wp-event
    (checked-staged-security-experiment-with-actual-alpha-prefix-data-state A)
    checked-staged-security-with-actual-alpha-prefix-witness-transcript-pre-hit
    adversary-initial-state ≤ P
proof -
  let ?project =
    λx. ((snd (fst (fst x)), snd (fst x)), snd x)
  have projection:
    wp-event (checked-staged-security-experiment-with-data-state A)
    staged-security-with-data-state-transcript-pre-hit
    adversary-initial-state =
    wp-event
    (checked-staged-security-experiment-with-actual-alpha-prefix-data-state A)
    (λout. case out of
      None ⇒ staged-security-with-data-state-transcript-pre-hit None
      | Some (packed, t) ⇒

```

```

    staged-security-with-data-state-transcript-pre-hit
    (Some (?project packed, t)))
  adversary-initial-state
proof –
  have
    wp-event (checked-staged-security-experiment-with-data-state A)
    staged-security-with-data-state-transcript-pre-hit
    adversary-initial-state =
    wp-event
    (checked-staged-security-experiment-with-actual-alpha-prefix-data-state A
     ≫ (λx. return (?project x)))
    staged-security-with-data-state-transcript-pre-hit
    adversary-initial-state
  using
    checked-staged-security-experiment-with-actual-alpha-prefix-data-state-projection
    [of A]
  by simp
also have ... =
  wp-event
  (checked-staged-security-experiment-with-actual-alpha-prefix-data-state A)
  (λout. case out of
    None ⇒ staged-security-with-data-state-transcript-pre-hit None
  | Some (packed, t) ⇒
    staged-security-with-data-state-transcript-pre-hit
    (Some (?project packed, t)))
  adversary-initial-state
  by (rule wp-event-bind-return-map)
finally show ?thesis .
qed
have event-le:
  wp-event
  (checked-staged-security-experiment-with-actual-alpha-prefix-data-state A)
  checked-staged-security-with-actual-alpha-prefix-witness-transcript-pre-hit
  adversary-initial-state ≤
  wp-event
  (checked-staged-security-experiment-with-actual-alpha-prefix-data-state A)
  (λout. case out of
    None ⇒ staged-security-with-data-state-transcript-pre-hit None
  | Some (packed, t) ⇒
    staged-security-with-data-state-transcript-pre-hit
    (Some (?project packed, t)))
  adversary-initial-state
proof (rule wp-event-mono)
  fix out
  assume hit:
    checked-staged-security-with-actual-alpha-prefix-witness-transcript-pre-hit
    out
  show
    (case out of

```

```

    None  $\Rightarrow$  staged-security-with-data-state-transcript-pre-hit None
  | Some (packed, t)  $\Rightarrow$ 
    staged-security-with-data-state-transcript-pre-hit
      (Some (?project packed, t)))
proof (cases out)
case None
then show ?thesis
  using hit
  unfolding
    checked-staged-security-with-actual-alpha-prefix-witness-transcript-pre-hit-def
  by simp
next
case (Some packed)
then obtain prefix prefix-state data attacker-state result final-state
where out-eq:
  out =
    Some (((((prefix, prefix-state), data), attacker-state), result),
      final-state)
  by (cases packed) (auto split: prod.splits)
show ?thesis
  using
    checked-staged-security-with-actual-alpha-prefix-witness-transcript-pre-hit-imp-data-state-pre-hit
    [OF hit[unfolded out-eq]]
  unfolding out-eq by simp
qed
qed
show ?thesis
  by (rule order-trans[OF event-le])
  (use pre-bound projection in simp)
qed

definition checked-staged-security-with-actual-alpha-prefix-sampled-transcript-hit
  :: ((((((f  $\times$  'f list  $\times$  'f list  $\times$  'f)  $\times$ 
    'f protocol-channel)  $\times$  'f staged-proof-data)  $\times$ 
    'f protocol-channel)  $\times$  unit list)  $\times$ 
    'f protocol-channel) option  $\Rightarrow$  bool
where
  checked-staged-security-with-actual-alpha-prefix-sampled-transcript-hit
    out  $\longleftrightarrow$ 
  (case out of
    None  $\Rightarrow$  False
  | Some (((((prefix, prefix-state), data), attacker-state), result),
    final-state)  $\Rightarrow$ 
    staged-security-with-data-state-transcript-pre-hit
      (Some (((data, attacker-state), result), final-state))  $\vee$ 
    staged-security-with-data-state-transcript-new-hit
      (Some (((data, attacker-state), result), final-state)))

```

lemma checked-staged-security-with-actual-alpha-prefix-sampled-transcript-hit-bound-from-data-state:

```

assumes pre-bound:
  wp-event (checked-staged-security-experiment-with-data-state A)
    staged-security-with-data-state-transcript-pre-hit
    adversary-initial-state  $\leq P$ 
and new-bound:
  wp-event (checked-staged-security-experiment-with-data-state A)
    staged-security-with-data-state-transcript-new-hit
    adversary-initial-state  $\leq R$ 
shows
  wp-event
    (checked-staged-security-experiment-with-actual-alpha-prefix-data-state A)
    checked-staged-security-with-actual-alpha-prefix-sampled-transcript-hit
    adversary-initial-state  $\leq P + R$ 
proof –
  let ?project =
     $\lambda x. ((\text{snd } (\text{fst } (\text{fst } x)), \text{snd } (\text{fst } x)), \text{snd } x)$ 
  let ?E =
     $\lambda out.$ 
      staged-security-with-data-state-transcript-pre-hit out  $\vee$ 
      staged-security-with-data-state-transcript-new-hit out
  let ?projected =
     $\lambda out. \text{case } out \text{ of}$ 
      None  $\Rightarrow ?E \text{ None}$ 
       $| \text{Some } (\text{packed}, t) \Rightarrow ?E (\text{Some } (?project \text{ packed}, t))$ 
  have projected-eq:
    ?projected =
      checked-staged-security-with-actual-alpha-prefix-sampled-transcript-hit
  unfolding
    checked-staged-security-with-actual-alpha-prefix-sampled-transcript-hit-def
    staged-security-with-data-state-transcript-pre-hit-def
    staged-security-with-data-state-transcript-new-hit-def
  by (rule ext) (auto split: option.splits prod.splits)
  have projection:
    wp-event (checked-staged-security-experiment-with-data-state A) ?E
      adversary-initial-state =
        wp-event
          (checked-staged-security-experiment-with-actual-alpha-prefix-data-state A)
          checked-staged-security-with-actual-alpha-prefix-sampled-transcript-hit
          adversary-initial-state
  proof –
  have
    wp-event (checked-staged-security-experiment-with-data-state A) ?E
      adversary-initial-state =
        wp-event
          (checked-staged-security-experiment-with-actual-alpha-prefix-data-state A
             $\gg (\lambda x. \text{return } (?project x))$ )
          ?E adversary-initial-state
  using
    checked-staged-security-experiment-with-actual-alpha-prefix-data-state-projection

```

```

    [of A]
  by simp
also have ... =
  wp-event
    (checked-staged-security-experiment-with-actual-alpha-prefix-data-state A)
    ?projected adversary-initial-state
  by (rule wp-event-bind-return-map)
finally show ?thesis
  unfolding projected-eq .
qed
have union-bound:
  wp-event (checked-staged-security-experiment-with-data-state A) ?E
    adversary-initial-state ≤
  wp-event (checked-staged-security-experiment-with-data-state A)
    staged-security-with-data-state-transcript-pre-hit
    adversary-initial-state +
  wp-event (checked-staged-security-experiment-with-data-state A)
    staged-security-with-data-state-transcript-new-hit
    adversary-initial-state
  by (rule wp-event-union-bound)
have
  wp-event (checked-staged-security-experiment-with-data-state A) ?E
    adversary-initial-state ≤ P + R
  by (rule order-trans[OF union-bound])
    (intro add-mono pre-bound new-bound)
then show ?thesis
  unfolding projection .
qed

lemma checked-staged-security-with-actual-alpha-prefix-some-support-imp-sampled-transcript-hit:
  assumes support:
    Some (((((prefix, prefix-state), data), attacker-state), result),
      final-state) ∈
    set-dist
      (execute
        (checked-staged-security-experiment-with-actual-alpha-prefix-data-state
          A)
        adversary-initial-state)
  shows
    checked-staged-security-with-actual-alpha-prefix-sampled-transcript-hit
      (Some (((((prefix, prefix-state), data), attacker-state), result),
        final-state))
proof -
  let ?s =
    verifier-state-from-adversary attacker-state
      (staged-proof-transcript data)
  have data-support:
    Some (((data, attacker-state), result), final-state) ∈
    set-dist

```



```

    (execute
      (checked-staged-security-experiment-with-data-state A)
      adversary-initial-state)
  by (rule
    checked-staged-security-with-actual-alpha-prefix-support-imp-data-state-support
    [OF support])
from checked-staged-security-experiment-with-data-state-outcomeE
  [OF data-support]
have verifier:
  Some (result, final-state) ∈ set-dist (execute verify-monad ?s)
  by blast
have transcript-split:
  hash-map-output-values ?s ∩ set (staged-proof-transcript data)
    ≠ {} ∨
  hash-map-new-output-hit (set (staged-proof-transcript data))
    ?s final-state
  by (rule verify-monad-trace-root-target-preexisting-or-new[OF verifier])
    simp
show ?thesis
  unfolding
    checked-staged-security-with-actual-alpha-prefix-sampled-transcript-hit-def
    staged-security-with-data-state-transcript-pre-hit-def
    staged-security-with-data-state-transcript-new-hit-def
  using transcript-split by simp
qed

```

lemma *checked-staged-security-with-actual-alpha-prefix-not-prefix-no-prefix-candidate-bound-from-sampled-transcript*

assumes *pre-bound*:

wp-event (checked-staged-security-experiment-with-data-state A)
staged-security-with-data-state-transcript-pre-hit
adversary-initial-state ≤ P

and *new-bound*:

wp-event (checked-staged-security-experiment-with-data-state A)
staged-security-with-data-state-transcript-new-hit
adversary-initial-state ≤ R

shows

wp-event
(checked-staged-security-experiment-with-actual-alpha-prefix-data-state A)
checked-staged-security-with-actual-alpha-prefix-not-prefix-no-prefix-candidate
adversary-initial-state ≤ P + R

proof –

have *event-le*:

wp-event
(checked-staged-security-experiment-with-actual-alpha-prefix-data-state A)
checked-staged-security-with-actual-alpha-prefix-not-prefix-no-prefix-candidate
adversary-initial-state ≤
wp-event
(checked-staged-security-experiment-with-actual-alpha-prefix-data-state A)
checked-staged-security-with-actual-alpha-prefix-sampled-transcript-hit

```

    adversary-initial-state
proof (rule wp-event-mono-on-support)
  fix out
  assume support:
    out ∈
      set-dist
      (execute
        (checked-staged-security-experiment-with-actual-alpha-prefix-data-state A)
        adversary-initial-state)
  and hit:
    checked-staged-security-with-actual-alpha-prefix-not-prefix-no-prefix-candidate
    out
show
  checked-staged-security-with-actual-alpha-prefix-sampled-transcript-hit
  out
proof (cases out)
  case None
  then show ?thesis
    using hit
    unfolding
      checked-staged-security-with-actual-alpha-prefix-not-prefix-no-prefix-candidate-def
    by simp
next
  case (Some packed)
  then obtain prefix prefix-state data attacker-state result final-state
  where out-eq:
    out =
      Some (((((prefix, prefix-state), data), attacker-state), result),
        final-state)
  by (cases packed) (auto split: prod.splits)
show ?thesis
  unfolding out-eq
  by (rule
    checked-staged-security-with-actual-alpha-prefix-some-support-imp-sampled-transcript-hit)
    (rule support[unfolded out-eq])
qed
qed
have transcript-bound:
  wp-event
    (checked-staged-security-experiment-with-actual-alpha-prefix-data-state A)
    checked-staged-security-with-actual-alpha-prefix-sampled-transcript-hit
    adversary-initial-state ≤ P + R
  by (rule
    checked-staged-security-with-actual-alpha-prefix-sampled-transcript-hit-bound-from-data-state
    [OF pre-bound new-bound])
show ?thesis
  by (rule order-trans[OF event-le transcript-bound])
qed

```

lemma *checked-staged-security-with-actual-alpha-prefix-not-prefix-path-output-bound-from-sampled-transcript:*

assumes *pre-bound:*

wp-event (checked-staged-security-experiment-with-data-state A)
staged-security-with-data-state-transcript-pre-hit
adversary-initial-state $\leq P$

and *new-bound:*

wp-event (checked-staged-security-experiment-with-data-state A)
staged-security-with-data-state-transcript-new-hit
adversary-initial-state $\leq R$

shows

wp-event
(checked-staged-security-experiment-with-actual-alpha-prefix-data-state A)
checked-staged-security-with-actual-alpha-prefix-not-prefix-path-output
adversary-initial-state $\leq P + R$

proof –

have *event-le:*

wp-event
(checked-staged-security-experiment-with-actual-alpha-prefix-data-state A)
checked-staged-security-with-actual-alpha-prefix-not-prefix-path-output
adversary-initial-state $\leq$
wp-event
(checked-staged-security-experiment-with-actual-alpha-prefix-data-state A)
checked-staged-security-with-actual-alpha-prefix-sampled-transcript-hit
adversary-initial-state

proof (rule *wp-event-mono-on-support*)

fix *out*

assume *support:*

out $\in$
set-dist
(execute
(checked-staged-security-experiment-with-actual-alpha-prefix-data-state A)
adversary-initial-state)

and *hit:*

checked-staged-security-with-actual-alpha-prefix-not-prefix-path-output
out

show

checked-staged-security-with-actual-alpha-prefix-sampled-transcript-hit
out

proof (cases *out*)

case *None*

then show *?thesis*

using *hit*

unfolding
checked-staged-security-with-actual-alpha-prefix-not-prefix-path-output-def

by *simp*

next

case (Some *packed*)

then obtain *prefix prefix-state data attacker-state result final-state*

where *out-eq:*

```

    out =
      Some (((((prefix, prefix-state), data), attacker-state), result),
            final-state)
  by (cases packed) (auto split: prod.splits)
show ?thesis
  unfolding out-eq
  by (rule
      checked-staged-security-with-actual-alpha-prefix-some-support-imp-sampled-transcript-hit)
    (rule support[unfolded out-eq])
qed
qed
have transcript-bound:
  wp-event
    (checked-staged-security-experiment-with-actual-alpha-prefix-data-state A)
    checked-staged-security-with-actual-alpha-prefix-sampled-transcript-hit
    adversary-initial-state  $\leq P + R$ 
  by (rule
      checked-staged-security-with-actual-alpha-prefix-sampled-transcript-hit-bound-from-data-state
        [OF pre-bound new-bound])
show ?thesis
  by (rule order-trans[OF event-le transcript-bound])
qed

lemma checked-staged-security-with-data-state-transcript-new-hit-bound:
  assumes wf: staged-budget-wellformed budgets
  and controlled: staged-adversary-controlled budgets A
  shows
    wp-event (checked-staged-security-experiment-with-data-state A)
      staged-security-with-data-state-transcript-new-hit
      adversary-initial-state  $\leq$  staged-concrete-transcript-target-error-bound
proof -
  let ?Builder = checked-staged-transcript-program A
  let ?Tail =
     $\lambda$ data s.
      wp-event verify-monad
        (hash-new-output-hit-event (set (staged-proof-transcript data))
          (verifier-state-from-adversary s (staged-proof-transcript data)))
        (verifier-state-from-adversary s (staged-proof-transcript data))
  have new-none[simp]:  $\neg$  staged-security-with-data-state-transcript-new-hit None
  unfolding staged-security-with-data-state-transcript-new-hit-def by simp
  have return-eq:
    wp (return ((data, s), result))
      ( $\lambda$ out.
        if staged-security-with-data-state-transcript-new-hit out then 1
        else 0)
    final-state =
      (if hash-map-new-output-hit (set (staged-proof-transcript data))
        (verifier-state-from-adversary s (staged-proof-transcript data))
        final-state then 1 else 0)

```

```

for data s result final-state
unfolding staged-security-with-data-state-transcript-new-hit-def
by (simp add: wpsimps)
have verify-post-eq:
  ( $\lambda r$ . case r of
    None  $\Rightarrow 0$ 
    | Some (result, final-state)  $\Rightarrow$ 
      wp (return ((data, s), result))
        ( $\lambda out$ .
          if staged-security-with-data-state-transcript-new-hit out
            then 1 else 0)
        final-state) =
  ( $\lambda r$ .
    if hash-new-output-hit-event (set (staged-proof-transcript data))
      (verifier-state-from-adversary s (staged-proof-transcript data)) r
        then 1 else 0)
for data s
proof (rule ext)
fix r
show (case r of
  None  $\Rightarrow 0$ 
  | Some (result, final-state)  $\Rightarrow$ 
    wp (return ((data, s), result))
      ( $\lambda out$ .
        if staged-security-with-data-state-transcript-new-hit out
          then 1 else 0)
      final-state) =
    (if hash-new-output-hit-event (set (staged-proof-transcript data))
      (verifier-state-from-adversary s (staged-proof-transcript data)) r
        then 1 else 0)
proof (cases r)
case None
then show ?thesis
  unfolding hash-new-output-hit-event-def by simp
next
case (Some packed)
then obtain result final-state where
  packed-eq: packed = (result, final-state)
by (cases packed) simp
show ?thesis
  using return-eq[of data s result final-state]
  unfolding Some packed-eq hash-new-output-hit-event-def
  by simp
qed
qed
have tail-eq:
  wp
    (get  $\gg=$  ( $\lambda s$ .
      put (verifier-state-from-adversary s

```

```

    (staged-proof-transcript data)) >>=
    (λ-. verify-monad >>=
      (λresult. return ((data, s), result))))))
  (λout.
    if staged-security-with-data-state-transcript-new-hit out then 1
    else 0)
  s =
  ?Tail data s
for data s
unfolding wp-event-def
by (simp add: wpsimps verify-post-eq)
have post-eq:
  (λout. case out of
    None ⇒ 0
    | Some (x, y) ⇒
      wp
      (get >>= (λs.
        put (verifier-state-from-adversary s
          (staged-proof-transcript x)) >>=
          (λ-. verify-monad >>=
            (λresult. return ((x, s), result))))))
      (λout.
        if staged-security-with-data-state-transcript-new-hit out
        then 1 else 0)
      y) =
  (λout. case out of
    None ⇒ 0
    | Some (data, s) ⇒ ?Tail data s)
by (rule ext) (simp add: tail-eq split: option.splits prod.splits)
have post-tail-unfold:
  (λout. case out of
    None ⇒ 0
    | Some (data, s) ⇒ ?Tail data s) =
  (λout. case out of
    None ⇒ 0
    | Some (data, s) ⇒
      wp verify-monad
      (λx. if hash-new-output-hit-event
        (set (staged-proof-transcript data))
        (verifier-state-from-adversary s
          (staged-proof-transcript data))
        x then 1 else 0)
      (verifier-state-from-adversary s
        (staged-proof-transcript data)))
by (rule ext) (simp add: wp-event-def split: option.splits prod.splits)
have event-eq:
  wp-event (checked-staged-security-experiment-with-data-state A)
  staged-security-with-data-state-transcript-new-hit
  adversary-initial-state =

```

```

wp ?Builder
  (λout. case out of
    None ⇒ 0
  | Some (data, s) ⇒ ?Tail data s)
adversary-initial-state
unfolding checked-staged-security-experiment-with-data-state-def
wp-event-def
by (simp add: wpsimps post-eq post-tail-unfold
  split: prod.splits option.splits)
have tail-bound:
  ∧out. out ∈ set-dist (execute ?Builder adversary-initial-state) ⇒
    (case out of None ⇒ 0 | Some (data, s) ⇒ ?Tail data s)
    ≤ staged-concrete-transcript-target-error-bound
proof -
fix out
assume support:
  out ∈ set-dist (execute ?Builder adversary-initial-state)
show (case out of None ⇒ 0 | Some (data, s) ⇒ ?Tail data s)
  ≤ staged-concrete-transcript-target-error-bound
proof (cases out)
case None
then show ?thesis
  unfolding staged-concrete-transcript-target-error-bound-def by simp
next
case (Some packed)
then obtain data s where packed-eq: packed = (data, s)
  by (cases packed) simp
have outcome:
  Some (data, s) ∈
    set-dist (execute ?Builder adversary-initial-state)
  using support Some packed-eq by simp
have verifier-bound:
  ?Tail data s ≤
    concrete-transcript-target-error (staged-proof-transcript data)
  using wp-verify-monad-hash-new-output-hit-bound
  [of set (staged-proof-transcript data)
    verifier-state-from-adversary s
    (staged-proof-transcript data)]
  unfolding concrete-transcript-target-error-def
  hash-target-budget-value-def
  by (simp add: mult.commute)
have target-bound:
  concrete-transcript-target-error (staged-proof-transcript data)
  ≤ staged-concrete-transcript-target-error-bound
proof -
have len:
  length (staged-proof-transcript data) ≤
    staged-proof-transcript-length-bound
  by (rule

```

```

    checked-staged-transcript-program-staged-proof-transcript-length-bound
    [OF wf controlled outcome])
  show ?thesis
    unfolding staged-concrete-transcript-target-error-bound-def
    by (rule concrete-transcript-target-error-bound-from-length[OF len])
qed
show ?thesis
  using Some packed-eq order-trans[OF verifier-bound target-bound]
  by simp
qed
qed
show ?thesis
  unfolding event-eq
  by (rule wp-le-const-on-support[OF tail-bound])
qed

lemma checked-staged-security-with-data-state-composition-bad-bound-from-cross-no-prefix-transcript-split-components
  fixes X N P Q :: prob
  assumes wf: staged-budget-wellformed budgets
  and controlled: staged-adversary-controlled budgets A
  and cross-bound:
    wp-event
    (checked-staged-security-experiment-with-actual-alpha-prefix-data-state
     A)
    checked-staged-security-with-actual-alpha-prefix-witnessed-cross-or-merkle-bad
    adversary-initial-state  $\leq$  X
  and no-prefix-bound:
    wp-event
    (checked-staged-security-experiment-with-actual-alpha-prefix-data-state
     A)
    checked-staged-security-with-actual-alpha-prefix-not-prefix-no-prefix-candidate
    adversary-initial-state  $\leq$  N
  and pre-bound:
    wp-event
    (checked-staged-security-experiment-with-actual-alpha-prefix-data-state
     A)
    checked-staged-security-with-actual-alpha-prefix-witness-transcript-pre-hit
    adversary-initial-state  $\leq$  P
  and new-bound:
    wp-event
    (checked-staged-security-experiment-with-actual-alpha-prefix-data-state
     A)
    checked-staged-security-with-actual-alpha-prefix-witness-transcript-new-hit
    adversary-initial-state  $\leq$  Q
  shows
    wp-event (checked-staged-security-experiment-with-data-state A)
    (staged-security-with-data-state-verifier-event composition-bad)
    adversary-initial-state  $\leq$ 
    composition-error-bound +

```


staged-phase-relation-error size
(staged-alpha-search-queries budgets 0) +
hash-collision-budget-value 0
(staged-alpha-search-queries budgets 0) +
(hash-collision-budget-value 0
(staged-alpha-search-queries budgets 0) + X + N + (P + Q)) +
staged-phase-relation-error size
(staged-alpha-search-queries budgets 0) +
hash-collision-budget-value 0
(staged-attacker-query-budget budgets +
staged-challenge-query-budget + verifier-hash-query-budget)
proof (rule
checked-staged-security-with-data-state-composition-bad-bound-from-cross-no-prefix-transcript-components-and-bud
[OF wf controlled cross-bound no-prefix-bound])
show
wp-event
(checked-staged-security-experiment-with-actual-alpha-prefix-data-state
A)
checked-staged-security-with-actual-alpha-prefix-witness-transcript-hit
adversary-initial-state ≤ P + Q
by (rule
checked-staged-security-with-actual-alpha-prefix-witness-transcript-hit-bound-from-pre-and-new
[OF pre-bound new-bound])
qed

lemma *checked-staged-security-with-data-state-composition-bad-bound-from-cross-no-prefix-data-pre-and-witness-new*
fixes *X N P Q :: prob*
assumes *wf: staged-budget-wellformed budgets*
and *controlled: staged-adversary-controlled budgets A*
and *cross-bound:*
wp-event
(checked-staged-security-experiment-with-actual-alpha-prefix-data-state
A)
checked-staged-security-with-actual-alpha-prefix-witnessed-cross-or-merkle-bad
adversary-initial-state ≤ X
and *no-prefix-bound:*
wp-event
(checked-staged-security-experiment-with-actual-alpha-prefix-data-state
A)
checked-staged-security-with-actual-alpha-prefix-not-prefix-no-prefix-candidate
adversary-initial-state ≤ N
and *data-pre-bound:*
wp-event (checked-staged-security-experiment-with-data-state A)
staged-security-with-data-state-transcript-pre-hit
adversary-initial-state ≤ P
and *new-bound:*
wp-event
(checked-staged-security-experiment-with-actual-alpha-prefix-data-state
A)

checked-staged-security-with-actual-alpha-prefix-witness-transcript-new-hit
adversary-initial-state $\leq Q$

shows

wp-event (*checked-staged-security-experiment-with-data-state* A)
 (*staged-security-with-data-state-verifier-event composition-bad*)
adversary-initial-state $\leq$
composition-error-bound +
staged-phase-relation-error size
 (*staged-alpha-search-queries budgets* 0) +
hash-collision-budget-value 0
 (*staged-alpha-search-queries budgets* 0) +
 (*hash-collision-budget-value* 0
 (*staged-alpha-search-queries budgets* 0) + $X + N + (P + Q)$) +
staged-phase-relation-error size
 (*staged-alpha-search-queries budgets* 0) +
hash-collision-budget-value 0
 (*staged-attacker-query-budget budgets* +
staged-challenge-query-budget + *verifier-hash-query-budget*)

proof (rule

checked-staged-security-with-data-state-composition-bad-bound-from-cross-no-prefix-transcript-split-components-and
 [*OF wf controlled cross-bound no-prefix-bound - new-bound*])

show

wp-event
 (*checked-staged-security-experiment-with-actual-alpha-prefix-data-state*
 A)
checked-staged-security-with-actual-alpha-prefix-witness-transcript-pre-hit
adversary-initial-state $\leq P$

by (rule

checked-staged-security-with-actual-alpha-prefix-witness-transcript-pre-hit-bound-from-data-state-pre-hit
 [*OF data-pre-bound*])

qed

lemma *checked-staged-security-with-data-state-composition-bad-bound-from-cross-and-transcript-components-and-bu*

fixes $X T :: \text{prob}$

assumes *wf*: *staged-budget-wellformed budgets*

and *controlled*: *staged-adversary-controlled budgets* A

and *cross-bound*:

wp-event
 (*checked-staged-security-experiment-with-actual-alpha-prefix-data-state*
 A)
checked-staged-security-with-actual-alpha-prefix-witnessed-cross-or-merkle-bad
adversary-initial-state $\leq X$

and *transcript-bound*:

wp-event
 (*checked-staged-security-experiment-with-actual-alpha-prefix-data-state*
 A)
checked-staged-security-with-actual-alpha-prefix-witness-transcript-hit
adversary-initial-state $\leq T$

shows

$$\begin{aligned}
& wp\text{-}event \ (checked\text{-}staged\text{-}security\text{-}experiment\text{-}with\text{-}data\text{-}state \ A) \\
& \ (staged\text{-}security\text{-}with\text{-}data\text{-}state\text{-}verifier\text{-}event \ composition\text{-}bad) \\
& adversary\text{-}initial\text{-}state \leq \\
& composition\text{-}error\text{-}bound + \\
& staged\text{-}phase\text{-}relation\text{-}error \ size \\
& \ (staged\text{-}\alpha\text{-}search\text{-}queries \ budgets \ 0) + \\
& hash\text{-}collision\text{-}budget\text{-}value \ 0 \\
& \ (staged\text{-}\alpha\text{-}search\text{-}queries \ budgets \ 0) + \\
& (hash\text{-}collision\text{-}budget\text{-}value \ 0 \\
& \ (staged\text{-}\alpha\text{-}search\text{-}queries \ budgets \ 0) + X + T + T) + \\
& staged\text{-}phase\text{-}relation\text{-}error \ size \\
& \ (staged\text{-}\alpha\text{-}search\text{-}queries \ budgets \ 0) + \\
& hash\text{-}collision\text{-}budget\text{-}value \ 0 \\
& \ (staged\text{-}attacker\text{-}query\text{-}budget \ budgets + \\
& \ \ \ staged\text{-}challenge\text{-}query\text{-}budget + verifier\text{-}hash\text{-}query\text{-}budget)
\end{aligned}$$

proof (rule

$checked\text{-}staged\text{-}security\text{-}with\text{-}data\text{-}state\text{-}composition\text{-}bad\text{-}bound\text{-}from\text{-}cross\text{-}no\text{-}prefix\text{-}transcript\text{-}components\text{-}and\text{-}bud$

$[OF \ wf \ controlled \ cross\text{-}bound - transcript\text{-}bound])$

show

$wp\text{-}event$

$(checked\text{-}staged\text{-}security\text{-}experiment\text{-}with\text{-}actual\text{-}\alpha\text{-}prefix\text{-}data\text{-}state$

$A)$

$checked\text{-}staged\text{-}security\text{-}with\text{-}actual\text{-}\alpha\text{-}prefix\text{-}not\text{-}prefix\text{-}no\text{-}prefix\text{-}candidate$

$adversary\text{-}initial\text{-}state \leq T$

by (rule $order\text{-}trans$

$[OF$

$checked\text{-}staged\text{-}security\text{-}with\text{-}actual\text{-}\alpha\text{-}prefix\text{-}not\text{-}prefix\text{-}no\text{-}prefix\text{-}candidate\text{-}le\text{-}witness\text{-}transcript\text{-}hit$

$transcript\text{-}bound])$

qed

lemma $checked\text{-}staged\text{-}security\text{-}with\text{-}data\text{-}state\text{-}composition\text{-}bad\text{-}bound\text{-}from\text{-}cross\text{-}data\text{-}pre\text{-}and\text{-}witness\text{-}new\text{-}compon$

fixes $X \ P \ Q :: prob$

assumes wf : $staged\text{-}budget\text{-}wellformed \ budgets$

and $controlled$: $staged\text{-}adversary\text{-}controlled \ budgets \ A$

and $cross\text{-}bound$:

$wp\text{-}event$

$(checked\text{-}staged\text{-}security\text{-}experiment\text{-}with\text{-}actual\text{-}\alpha\text{-}prefix\text{-}data\text{-}state$

$A)$

$checked\text{-}staged\text{-}security\text{-}with\text{-}actual\text{-}\alpha\text{-}prefix\text{-}witnessed\text{-}cross\text{-}or\text{-}merkle\text{-}bad$

$adversary\text{-}initial\text{-}state \leq X$

and $data\text{-}pre\text{-}bound$:

$wp\text{-}event \ (checked\text{-}staged\text{-}security\text{-}experiment\text{-}with\text{-}data\text{-}state \ A)$

$staged\text{-}security\text{-}with\text{-}data\text{-}state\text{-}transcript\text{-}pre\text{-}hit$

$adversary\text{-}initial\text{-}state \leq P$

and $new\text{-}bound$:

$wp\text{-}event$

$(checked\text{-}staged\text{-}security\text{-}experiment\text{-}with\text{-}actual\text{-}\alpha\text{-}prefix\text{-}data\text{-}state$

$A)$

$checked\text{-}staged\text{-}security\text{-}with\text{-}actual\text{-}\alpha\text{-}prefix\text{-}witness\text{-}transcript\text{-}new\text{-}hit$

```

    adversary-initial-state ≤ Q
  shows
    wp-event (checked-staged-security-experiment-with-data-state A)
      (staged-security-with-data-state-verifier-event composition-bad)
    adversary-initial-state ≤
      composition-error-bound +
      staged-phase-relation-error size
      (staged-alpha-search-queries budgets 0) +
      hash-collision-budget-value 0
      (staged-alpha-search-queries budgets 0) +
      (hash-collision-budget-value 0
        (staged-alpha-search-queries budgets 0) + X + (P + Q) + (P + Q)) +
      staged-phase-relation-error size
      (staged-alpha-search-queries budgets 0) +
      hash-collision-budget-value 0
      (staged-attacker-query-budget budgets +
        staged-challenge-query-budget + verifier-hash-query-budget)
  proof (rule
    checked-staged-security-with-data-state-composition-bad-bound-from-cross-and-transcript-components-and-budgets
    [OF wf controlled cross-bound])
  show
    wp-event
      (checked-staged-security-experiment-with-actual-alpha-prefix-data-state
        A)
    checked-staged-security-with-actual-alpha-prefix-witness-transcript-hit
    adversary-initial-state ≤ P + Q
  proof (rule
    checked-staged-security-with-actual-alpha-prefix-witness-transcript-hit-bound-from-pre-and-new
    [OF - new-bound])
  show
    wp-event
      (checked-staged-security-experiment-with-actual-alpha-prefix-data-state
        A)
    checked-staged-security-with-actual-alpha-prefix-witness-transcript-pre-hit
    adversary-initial-state ≤ P
  by (rule
    checked-staged-security-with-actual-alpha-prefix-witness-transcript-pre-hit-bound-from-data-state-pre-hit
    [OF data-pre-bound])
qed
qed

end

end

theory Staged-Security-Experiment-Composition-Query-Current-Alignment
  imports Staged-Security-Experiment-Composition-Query-Current
begin

```

Query-prefix alignment facts for the current-final-state composition-query route. This theory is intentionally narrow so that the already large current-query projection theory does not grow past the project size threshold.

context *soundness*
begin

lemma *checked-staged-security-experiment-with-query-prefix-data-state-outcomeI*:

assumes *prefix-receive*:

Some (((prefix, prefix-state), raw), raw-state) ∈
set-dist
(execute (checked-staged-query-prefix-receive-with-state A i)
adversary-initial-state)

and *after*:

Some (data, attacker-state) ∈
set-dist
(execute
(checked-staged-after-query-prefix-receive A i
((prefix, prefix-state), raw))
raw-state)

and *verifier*:

Some (result, final-state) ∈
set-dist
(execute verify-monad
(verifier-state-from-adversary attacker-state
(staged-proof-transcript data))))

shows

Some ((((((prefix, prefix-state), raw), raw-state),
((data, attacker-state), result)), final-state) ∈
set-dist
(execute
(checked-staged-security-experiment-with-query-prefix-data-state A i)
adversary-initial-state)

proof –

have *with-verifier*:

Some (((data, attacker-state), result), final-state) ∈
set-dist
(execute
(checked-staged-after-query-prefix-receive-with-verifier A i
((prefix, prefix-state), raw))
raw-state)

unfolding *checked-staged-after-query-prefix-receive-with-verifier-def*

apply (*rule-tac x=data and t=attacker-state in set-dist-bindI*)

apply (*rule after*)

apply (*rule-tac x=attacker-state and t=attacker-state in set-dist-bindI*)

apply *simp*

apply (*rule-tac x=() and*

t=verifier-state-from-adversary attacker-state
(staged-proof-transcript data) in set-dist-bindI)

apply *simp*

```

  apply (rule-tac x=result and t=final-state in set-dist-bindI)
  apply (rule verifier)
  apply simp
  done
show ?thesis
  unfolding checked-staged-security-experiment-with-query-prefix-data-state-def
  apply (rule set-dist-bindI[OF prefix-receive])
  apply (rule-tac x=raw-state and t=raw-state in set-dist-bindI)
  apply simp
  apply (rule-tac x=((data, attacker-state), result) and
    t=final-state in set-dist-bindI)
  apply (rule with-verifier)
  apply simp
  done
qed

```

lemma *checked-staged-after-query-prefix-receive-alphas:*

assumes *after:*

Some (data, attacker-state) ∈
set-dist
(execute
(checked-staged-after-query-prefix-receive A i
((prefix, prefix-state), raw))
raw-state)

shows *staged-alphas data = sqp-alphas prefix*

using *after*

unfolding *checked-staged-after-query-prefix-receive-def Let-def*

by (*auto elim!: set-dist-bindE split: prod.splits*)

lemma *checked-staged-security-with-actual-alpha-prefix-support-query-prefix-data-stateE:*

assumes *i-bound: i < rounds*

and *support:*

Some (((((alpha-prefix, alpha-prefix-state), data), attacker-state),
result), final-state) ∈
set-dist
(execute
(checked-staged-security-experiment-with-actual-alpha-prefix-data-state
A)
adversary-initial-state)

obtains *prefix prefix-state raw raw-state where*

Some (((((prefix, prefix-state), raw), raw-state),
((data, attacker-state), result)), final-state) ∈
set-dist
(execute
(checked-staged-security-experiment-with-query-prefix-data-state A i)
adversary-initial-state)

proof –

let *?s =*

verifier-state-from-adversary attacker-state

```

    (staged-proof-transcript data)
have data-support:
  Some (((data, attacker-state), result), final-state) ∈
    set-dist
      (execute (checked-staged-security-experiment-with-data-state A)
        adversary-initial-state)
  by (rule
    checked-staged-security-with-actual-alpha-prefix-support-imp-data-state-support
    [OF support])
from checked-staged-security-experiment-with-data-state-outcomeE
  [OF data-support]
obtain builder verifier where builder:
  Some (data, attacker-state) ∈
    set-dist
      (execute (checked-staged-transcript-program A)
        adversary-initial-state)
  and verifier:
  Some (result, final-state) ∈
    set-dist (execute verify-monad ?s)
  by blast
from checked-staged-transcript-program-query-prefix-receive-cont-support
  [OF i-bound builder]
obtain prefix prefix-state raw raw-state where prefix-receive:
  Some (((prefix, prefix-state), raw), raw-state) ∈
    set-dist
      (execute (checked-staged-query-prefix-receive-with-state A i)
        adversary-initial-state)
  and after:
  Some (data, attacker-state) ∈
    set-dist
      (execute
        (checked-staged-after-query-prefix-receive A i
          ((prefix, prefix-state), raw))
        raw-state)
  by blast
have qprefix-support:
  Some ((((((prefix, prefix-state), raw), raw-state),
    ((data, attacker-state), result)), final-state) ∈
    set-dist
      (execute
        (checked-staged-security-experiment-with-query-prefix-data-state A i)
        adversary-initial-state)
  by (rule checked-staged-security-experiment-with-query-prefix-data-state-outcomeI
    [OF prefix-receive after verifier])
show ?thesis
  by (rule that[OF qprefix-support])
qed

lemma checked-staged-security-with-actual-alpha-prefix-support-query-prefix-data-state-alphasE:

```

assumes *i-bound*: $i < \text{rounds}$
and *support*:
Some (((((alpha-prefix, alpha-prefix-state), data), attacker-state),
result), final-state) $\in$
set-dist
(*execute*
(*checked-staged-security-experiment-with-actual-alpha-prefix-data-state*
A)
adversary-initial-state)
obtains *prefix prefix-state raw raw-state* **where**
Some (((((prefix, prefix-state), raw), raw-state),
((data, attacker-state), result)), final-state) $\in$
set-dist
(*execute*
(*checked-staged-security-experiment-with-query-prefix-data-state A i*)
adversary-initial-state)
sqp-alphas prefix = staged-alphas data
proof –
let *?s* =
verifier-state-from-adversary attacker-state
(*staged-proof-transcript data*)
have *data-support*:
Some (((data, attacker-state), result), final-state) $\in$
set-dist
(*execute* (*checked-staged-security-experiment-with-data-state A*)
adversary-initial-state)
by (*rule*
checked-staged-security-with-actual-alpha-prefix-support-imp-data-state-support
[*OF support*])
from *checked-staged-security-experiment-with-data-state-outcomeE*
[*OF data-support*]
obtain *builder verifier* **where** *builder*:
Some (data, attacker-state) $\in$
set-dist
(*execute* (*checked-staged-transcript-program A*)
adversary-initial-state)
and *verifier*:
Some (result, final-state) $\in$
set-dist (*execute verify-monad ?s*)
by *blast*
from *checked-staged-transcript-program-query-prefix-receive-cont-support*
[*OF i-bound builder*]
obtain *prefix prefix-state raw raw-state* **where** *prefix-receive*:
Some (((prefix, prefix-state), raw), raw-state) $\in$
set-dist
(*execute* (*checked-staged-query-prefix-receive-with-state A i*)
adversary-initial-state)
and *after*:
Some (data, attacker-state) $\in$


```

    set-dist
    (execute
      (checked-staged-after-query-prefix-receive A i
        ((prefix, prefix-state), raw))
      raw-state)
  by blast
have qprefix-support:
  Some (((((prefix, prefix-state), raw), raw-state),
    ((data, attacker-state), result)), final-state) ∈
  set-dist
  (execute
    (checked-staged-security-experiment-with-query-prefix-data-state A i)
    adversary-initial-state)
  by (rule checked-staged-security-experiment-with-query-prefix-data-state-outcomeI
    [OF prefix-receive after verifier])
have alphas-eq: sqp-alphas prefix = staged-alphas data
  using checked-staged-after-query-prefix-receive-alphas[OF after]
  by simp
show ?thesis
  by (rule that[OF qprefix-support alphas-eq])
qed

lemma checked-staged-security-with-actual-alpha-prefix-candidate-pair-success-query-prefix-hitE:
  assumes i-bound: i < rounds
  and support:
    Some (((((alpha-prefix, alpha-prefix-state), data), attacker-state),
      result), final-state) ∈
    set-dist
    (execute
      (checked-staged-security-experiment-with-actual-alpha-prefix-data-state
        A)
      adversary-initial-state)
  and success:
     $\bigwedge$  prefix prefix-state raw raw-state.
    Some (((((prefix, prefix-state), raw), raw-state),
      ((data, attacker-state), result)), final-state) ∈
    set-dist
    (execute
      (checked-staged-security-experiment-with-query-prefix-data-state
        A i)
      adversary-initial-state)  $\implies$ 
    sqp-alphas prefix = staged-alphas data  $\implies$ 
    index (to-nat raw) ∈
    query-sampling-success-space trace-table composition-table
    (staged-alphas data)
  obtains prefix prefix-state raw raw-state where
    Some (((((prefix, prefix-state), raw), raw-state),
      ((data, attacker-state), result)), final-state) ∈
    set-dist

```

```

      (execute
        (checked-staged-security-experiment-with-query-prefix-data-state A i)
        adversary-initial-state)
    checked-staged-security-with-query-prefix-dynamic-index-hit
    (staged-query-prefix-candidate-pair-query-target-from-prefix
      trace-table composition-table)
    (Some (((((prefix, prefix-state), raw), raw-state),
      ((data, attacker-state), result)), final-state))
proof –
from checked-staged-security-with-actual-alpha-prefix-support-query-prefix-data-state-alphasE
  [OF i-bound support]
obtain prefix prefix-state raw raw-state where qprefix-support:
  Some (((((prefix, prefix-state), raw), raw-state),
    ((data, attacker-state), result)), final-state) ∈
  set-dist
  (execute
    (checked-staged-security-experiment-with-query-prefix-data-state A i)
    adversary-initial-state)
and alphas-eq: sqp-alphas prefix = staged-alphas data
by blast
have hit:
  checked-staged-security-with-query-prefix-dynamic-index-hit
  (staged-query-prefix-candidate-pair-query-target-from-prefix
    trace-table composition-table)
  (Some (((((prefix, prefix-state), raw), raw-state),
    ((data, attacker-state), result)), final-state))
using success[OF qprefix-support alphas-eq] alphas-eq
unfolding
  checked-staged-security-with-query-prefix-dynamic-index-hit-def
  checked-staged-query-prefix-dynamic-index-hit-def
  staged-query-prefix-candidate-pair-query-target-from-prefix-def
by simp
show ?thesis
by (rule that[OF qprefix-support hit])
qed

lemma checked-staged-actual-alpha-prefix-current-query-partial-opening-hit-at-imp-query-prefix-current-on-support:
assumes wf: staged-budget-wellformed budgets
and controlled: staged-adversary-controlled budgets A
and i-bound: i < rounds
and support:
  Some (((((alpha-prefix, alpha-prefix-state), data), attacker-state),
    result), final-state) ∈
  set-dist
  (execute
    (checked-staged-security-experiment-with-actual-alpha-prefix-data-state
      A)
    adversary-initial-state)
and hit:

```

staged-security-with-data-state-current-query-partial-opening-hit-at i
(Some (((data, attacker-state), result), final-state))
obtains *prefix prefix-state raw raw-state* **where**
Some (((((prefix, prefix-state), raw), raw-state),
((data, attacker-state), result)), final-state) ∈
set-dist
(execute
(checked-staged-security-experiment-with-query-prefix-data-state A i)
adversary-initial-state)
checked-staged-security-with-query-prefix-current-query-partial-opening-hit-at
i
(Some (((((prefix, prefix-state), raw), raw-state),
((data, attacker-state), result)), final-state))
proof –
let *?s =*
verifier-state-from-adversary attacker-state
(staged-proof-transcript data)
have *data-support:*
Some (((data, attacker-state), result), final-state) ∈
set-dist
(execute
(checked-staged-security-experiment-with-data-state A)
adversary-initial-state)
by (rule
checked-staged-security-with-actual-alpha-prefix-support-imp-data-state-support
[OF support])
from *checked-staged-security-experiment-with-data-state-outcomeE*
[OF data-support]
obtain *builder verifier* **where** *builder:*
Some (data, attacker-state) ∈
set-dist
(execute (checked-staged-transcript-program A)
adversary-initial-state)
and *verifier:*
Some (result, final-state) ∈ set-dist (execute verify-monad ?s)
by *blast*
have *decomp:*
checked-staged-transcript-program A =
checked-staged-query-prefix-receive-with-state A i ≫
checked-staged-after-query-prefix-receive A i
by (rule *checked-staged-transcript-program-query-prefix-receive-decomp*
[OF i-bound])
from *builder[unfolded decomp]*
obtain *prefix-out raw-state* **where** *prefix-receive:*
Some (prefix-out, raw-state) ∈
set-dist
(execute (checked-staged-query-prefix-receive-with-state A i)
adversary-initial-state)
and *after:*

```

    Some (data, attacker-state) ∈
      set-dist
        (execute
          (checked-staged-after-query-prefix-receive A i prefix-out)
          raw-state)
  by (elim set-dist-bindE)
obtain prefix prefix-state raw where prefix-out-eq:
  prefix-out = ((prefix, prefix-state), raw)
by (cases prefix-out) auto
have qprefix-support:
  Some (((((prefix, prefix-state), raw), raw-state),
    ((data, attacker-state), result)), final-state) ∈
    set-dist
      (execute
        (checked-staged-security-experiment-with-query-prefix-data-state A i)
        adversary-initial-state)
by (rule checked-staged-security-experiment-with-query-prefix-data-state-outcomeI)
    (use prefix-receive after verifier prefix-out-eq in simp-all)
have qprefix-hit:
  checked-staged-security-with-query-prefix-current-query-partial-opening-hit-at
    i
    (Some (((((prefix, prefix-state), raw), raw-state),
      ((data, attacker-state), result)), final-state))
by (rule
  staged-security-with-data-state-current-query-partial-opening-hit-at-imp-query-prefix-current-on-support
  [OF wf controlled i-bound qprefix-support hit])
show ?thesis
by (rule that[OF qprefix-support qprefix-hit])
qed

```

definition checked-staged-security-with-actual-alpha-prefix-current-query-partial-opening-hit-at

```

:: nat ⇒
  ((((((f × f list × f list × f) ×
    f protocol-channel) × f staged-proof-data) ×
    f protocol-channel) × unit list) ×
    f protocol-channel) option ⇒ bool

```

where

```

checked-staged-security-with-actual-alpha-prefix-current-query-partial-opening-hit-at
  i out ⇔
  (case out of
    None ⇒ False
  | Some (((((alpha-prefix, alpha-prefix-state), data), attacker-state),
    result), final-state) ⇒
    staged-security-with-data-state-current-query-partial-opening-hit-at i
      (Some (((data, attacker-state), result), final-state)))

```

definition checked-staged-security-with-actual-alpha-prefix-current-query-partial-opening-hit

```

:: ((((((f × f list × f list × f) ×
  f protocol-channel) × f staged-proof-data) ×

```

$\text{'f protocol-channel) } \times \text{ unit list) } \times$
 $\text{'f protocol-channel) option } \Rightarrow \text{ bool}$

where

$\text{checked-staged-security-with-actual-alpha-prefix-current-query-partial-opening-hit}$
 $\text{out} \longleftrightarrow$
 $(\exists i \in \{..<\text{rounds}\}.$
 $\text{checked-staged-security-with-actual-alpha-prefix-current-query-partial-opening-hit-at}$
 $i \text{ out})$

lemma $\text{checked-staged-security-with-actual-alpha-prefix-current-query-partial-opening-hit-at-imp-query-prefix-current}$

assumes $\text{wf: staged-budget-wellformed budgets}$

and $\text{controlled: staged-adversary-controlled budgets } A$

and $\text{i-bound: } i < \text{rounds}$

and support:

$\text{out} \in$
 set-dist
 $(\text{execute}$
 $(\text{checked-staged-security-experiment-with-actual-alpha-prefix-data-state}$
 $A)$
 $\text{adversary-initial-state})$

and hit:

$\text{checked-staged-security-with-actual-alpha-prefix-current-query-partial-opening-hit-at}$
 $i \text{ out}$

shows

$\exists \text{prefix prefix-state raw raw-state data attacker-state result final-state.}$
 $\text{Some } (((((\text{prefix, prefix-state}), \text{raw}), \text{raw-state}),$
 $((\text{data, attacker-state}), \text{result})), \text{final-state}) \in$
 set-dist
 $(\text{execute}$
 $(\text{checked-staged-security-experiment-with-query-prefix-data-state } A \ i)$
 $\text{adversary-initial-state}) \wedge$
 $\text{checked-staged-security-with-query-prefix-current-query-partial-opening-hit-at}$
 i
 $(\text{Some } (((((\text{prefix, prefix-state}), \text{raw}), \text{raw-state}),$
 $((\text{data, attacker-state}), \text{result})), \text{final-state}))$

proof (cases out)

case None

then show $?thesis$

using hit

unfolding

$\text{checked-staged-security-with-actual-alpha-prefix-current-query-partial-opening-hit-at-def}$

by simp

next

case (Some packed)

then obtain $\text{alpha-prefix alpha-prefix-state data attacker-state result}$
 $\text{final-state where out-eq:}$

$\text{out} =$
 $\text{Some } (((((\text{alpha-prefix, alpha-prefix-state}), \text{data}), \text{attacker-state}),$
 $\text{result}), \text{final-state})$

```

    by (cases packed) (auto split: prod.splits)
  have data-hit:
    staged-security-with-data-state-current-query-partial-opening-hit-at i
      (Some (((data, attacker-state), result), final-state))
  using hit
  unfolding out-eq
    checked-staged-security-with-actual-alpha-prefix-current-query-partial-opening-hit-at-def
  by simp
  have support-some:
    Some (((((alpha-prefix, alpha-prefix-state), data), attacker-state),
      result), final-state) ∈
    set-dist
      (execute
        (checked-staged-security-experiment-with-actual-alpha-prefix-data-state
          A)
        adversary-initial-state)
  using support out-eq by simp
  from checked-staged-actual-alpha-prefix-current-query-partial-opening-hit-at-imp-query-prefix-current-on-support
    [OF wf controlled i-bound support-some data-hit]
  obtain prefix prefix-state raw raw-state where qsupport:
    Some (((((prefix, prefix-state), raw), raw-state),
      ((data, attacker-state), result)), final-state) ∈
    set-dist
      (execute
        (checked-staged-security-experiment-with-query-prefix-data-state A i)
        adversary-initial-state)
  and qhit:
    checked-staged-security-with-query-prefix-current-query-partial-opening-hit-at
      i
      (Some (((((prefix, prefix-state), raw), raw-state),
        ((data, attacker-state), result)), final-state))
  by blast
  show ?thesis
  using qsupport qhit by blast
qed

lemma checked-staged-security-with-actual-alpha-prefix-current-query-partial-opening-hit-at-bound-from-query-prefix-
  assumes wf: staged-budget-wellformed budgets
  and controlled: staged-adversary-controlled budgets A
  and i-bound: i < rounds
  and query-prefix-bound:
    wp-event
      (checked-staged-security-experiment-with-query-prefix-data-state A i)
      (checked-staged-security-with-query-prefix-current-query-partial-opening-hit-at i)
      adversary-initial-state ≤ Q
  shows
    wp-event
      (checked-staged-security-experiment-with-actual-alpha-prefix-data-state A)
      (checked-staged-security-with-actual-alpha-prefix-current-query-partial-opening-hit-at i)

```

```

    adversary-initial-state ≤ Q
proof –
  let ?project =
    λx. ((snd (fst (fst x)), snd (fst x)), snd x)
  have projection:
    wp-event
      (checked-staged-security-experiment-with-actual-alpha-prefix-data-state A)
      (checked-staged-security-with-actual-alpha-prefix-current-query-partial-opening-hit-at i)
    adversary-initial-state =
    wp-event
      (checked-staged-security-experiment-with-data-state A)
      (staged-security-with-data-state-current-query-partial-opening-hit-at i)
    adversary-initial-state
proof –
  have
    wp-event
      (checked-staged-security-experiment-with-data-state A)
      (staged-security-with-data-state-current-query-partial-opening-hit-at i)
    adversary-initial-state =
    wp-event
      (checked-staged-security-experiment-with-actual-alpha-prefix-data-state A
        ≫ (λx. return (?project x)))
      (staged-security-with-data-state-current-query-partial-opening-hit-at i)
    adversary-initial-state
  using
    checked-staged-security-experiment-with-actual-alpha-prefix-data-state-projection
    [of A]
  by simp
also have ... =
  wp-event
    (checked-staged-security-experiment-with-actual-alpha-prefix-data-state A)
    (checked-staged-security-with-actual-alpha-prefix-current-query-partial-opening-hit-at i)
  adversary-initial-state
proof –
  have pred-eq:
    (λout. case out of
      None ⇒
        staged-security-with-data-state-current-query-partial-opening-hit-at
          i None
      | Some (x, t) ⇒
        staged-security-with-data-state-current-query-partial-opening-hit-at
          i (Some (?project x, t))) =
    checked-staged-security-with-actual-alpha-prefix-current-query-partial-opening-hit-at
      i
  by (rule ext)
  (simp add:
    checked-staged-security-with-actual-alpha-prefix-current-query-partial-opening-hit-at-def
    staged-security-with-data-state-current-query-partial-opening-hit-at-def
    split: option.splits prod.splits)

```

```

    show ?thesis
    by (subst wp-event-bind-return-map) (simp add: pred-eq)
qed
finally show ?thesis
  by simp
qed
have data-bound:
  wp-event
    (checked-staged-security-experiment-with-data-state A)
    (staged-security-with-data-state-current-query-partial-opening-hit-at i)
    adversary-initial-state  $\leq$  Q
  by (rule order-trans
    [OF
      checked-staged-security-with-data-state-current-query-partial-opening-hit-at-le-query-prefix-current
      query-prefix-bound])
    (use wf controlled i-bound in simp-all)
show ?thesis
  unfolding projection by (rule data-bound)
qed

lemma checked-staged-security-with-actual-alpha-prefix-current-query-partial-opening-hit-bound-from-query-prefix-current
  assumes wf: staged-budget-wellformed budgets
  and controlled: staged-adversary-controlled budgets A
  and round-bound:
     $\bigwedge i. i < \text{rounds} \implies$ 
    wp-event
      (checked-staged-security-experiment-with-query-prefix-data-state A i)
      (checked-staged-security-with-query-prefix-current-query-partial-opening-hit-at i)
      adversary-initial-state  $\leq$  C i
  shows
    wp-event
      (checked-staged-security-experiment-with-actual-alpha-prefix-data-state A)
      checked-staged-security-with-actual-alpha-prefix-current-query-partial-opening-hit
      adversary-initial-state  $\leq$  ( $\sum i < \text{rounds}. C i$ )
proof -
  have round-le:
     $\bigwedge i. i < \text{rounds} \implies$ 
    wp-event
      (checked-staged-security-experiment-with-actual-alpha-prefix-data-state A)
      (checked-staged-security-with-actual-alpha-prefix-current-query-partial-opening-hit-at i)
      adversary-initial-state  $\leq$  C i
  by (rule
    checked-staged-security-with-actual-alpha-prefix-current-query-partial-opening-hit-at-bound-from-query-prefix-current
    [OF wf controlled - round-bound])
  have
    wp-event
      (checked-staged-security-experiment-with-actual-alpha-prefix-data-state A)
      checked-staged-security-with-actual-alpha-prefix-current-query-partial-opening-hit
      adversary-initial-state  $\leq$ 

```



```

wp-event
  (checked-staged-security-experiment-with-actual-alpha-prefix-data-state A)
  (λout. ∃ i ∈ {.. $\text{rounds}$ }.
    checked-staged-security-with-actual-alpha-prefix-current-query-partial-opening-hit-at
      i out)
  adversary-initial-state
by (rule wp-event-mono)
  (simp add:
    checked-staged-security-with-actual-alpha-prefix-current-query-partial-opening-hit-def)
also have ... ≤ (∑ i<rounds. C i)
  by (rule wp-event-finite-UN-bound)
    (simp-all add: round-le)
finally show ?thesis .
qed

lemma checked-staged-security-with-actual-alpha-prefix-current-query-partial-opening-hit-projection:
  wp-event
    (checked-staged-security-experiment-with-actual-alpha-prefix-data-state A)
    checked-staged-security-with-actual-alpha-prefix-current-query-partial-opening-hit
    adversary-initial-state =
  wp-event
    (checked-staged-security-experiment-with-data-state A)
    staged-security-with-data-state-current-query-partial-opening-hit
    adversary-initial-state
proof -
  let ?project =
    λx. ((snd (fst (fst x)), snd (fst x)), snd x)
  have
    wp-event
      (checked-staged-security-experiment-with-data-state A)
      staged-security-with-data-state-current-query-partial-opening-hit
      adversary-initial-state =
    wp-event
      (checked-staged-security-experiment-with-actual-alpha-prefix-data-state A
        ≫ (λx. return (?project x)))
      staged-security-with-data-state-current-query-partial-opening-hit
      adversary-initial-state
  using
    checked-staged-security-experiment-with-actual-alpha-prefix-data-state-projection
    [of A]
  by simp
also have ... =
  wp-event
    (checked-staged-security-experiment-with-actual-alpha-prefix-data-state A)
    checked-staged-security-with-actual-alpha-prefix-current-query-partial-opening-hit
    adversary-initial-state
proof -
  have pred-eq:
    (λout. case out of

```

```

None  $\Rightarrow$ 
  staged-security-with-data-state-current-query-partial-opening-hit
  None
| Some (x, t)  $\Rightarrow$ 
  staged-security-with-data-state-current-query-partial-opening-hit
  (Some (?project x, t)) =
checked-staged-security-with-actual-alpha-prefix-current-query-partial-opening-hit
by (rule ext)
  (auto simp:
    checked-staged-security-with-actual-alpha-prefix-current-query-partial-opening-hit-def
    checked-staged-security-with-actual-alpha-prefix-current-query-partial-opening-hit-at-def
    staged-security-with-data-state-current-query-partial-opening-hit-def
    staged-security-with-data-state-current-query-partial-opening-hit-at-def
    split: option.splits prod.splits)
show ?thesis
  by (subst wp-event-bind-return-map) (simp add: pred-eq)
qed
finally show ?thesis
  by simp
qed

lemma checked-staged-security-with-data-state-current-query-partial-opening-hit-bound-from-actual-alpha-current-qu
assumes bound:
  wp-event
  (checked-staged-security-experiment-with-actual-alpha-prefix-data-state A)
  checked-staged-security-with-actual-alpha-prefix-current-query-partial-opening-hit
  adversary-initial-state  $\leq Q$ 
shows
  wp-event
  (checked-staged-security-experiment-with-data-state A)
  staged-security-with-data-state-current-query-partial-opening-hit
  adversary-initial-state  $\leq Q$ 
using bound
unfolding
  checked-staged-security-with-actual-alpha-prefix-current-query-partial-opening-hit-projection
by simp

lemma checked-staged-security-with-actual-alpha-prefix-composition-candidate-drift-query-hit-imp-current-query-part
assumes wf: staged-budget-wellformed budgets
and controlled: staged-adversary-controlled budgets A
and support:
  out  $\in$ 
  set-dist
  (execute
    (checked-staged-security-experiment-with-actual-alpha-prefix-data-state
      A)
    adversary-initial-state)
and hit:
  checked-staged-security-with-actual-alpha-prefix-composition-candidate-drift-query-hit

```

```

    out
  shows
    checked-staged-security-with-actual-alpha-prefix-current-query-partial-opening-hit
    out
proof (cases out)
  case None
  then show ?thesis
    using hit
    unfolding
      checked-staged-security-with-actual-alpha-prefix-composition-candidate-drift-query-hit-def
      checked-staged-security-with-actual-alpha-prefix-composition-candidate-drift-def
      checked-staged-security-with-actual-alpha-prefix-verifier-event-def
      checked-staged-security-with-actual-alpha-prefix-current-query-partial-opening-hit-def
      checked-staged-security-with-actual-alpha-prefix-current-query-partial-opening-hit-at-def
    by simp
next
  case (Some packed)
  then obtain alpha-prefix alpha-prefix-state data attacker-state result
    final-state where out-eq:
    out =
      Some (((((alpha-prefix, alpha-prefix-state), data), attacker-state),
        result), final-state)
    by (cases packed) (auto split: prod.splits)
  have support-some:
    Some (((((alpha-prefix, alpha-prefix-state), data), attacker-state),
      result), final-state) ∈
    set-dist
      (execute
        (checked-staged-security-experiment-with-actual-alpha-prefix-data-state
          A)
        adversary-initial-state)
    using support out-eq by simp
  have hit-some:
    checked-staged-security-with-actual-alpha-prefix-composition-candidate-drift-query-hit
      (Some (((((alpha-prefix, alpha-prefix-state), data), attacker-state),
        result), final-state))
    using hit out-eq by simp
  have data-hit:
    staged-security-with-data-state-current-query-partial-opening-hit
      (Some (((data, attacker-state), result), final-state))
    by (rule
      checked-staged-security-with-actual-alpha-prefix-composition-candidate-drift-query-hit-imp-data-state-current-par
      [OF wf controlled support-some hit-some])
  then obtain i where i-bound: i < rounds
  and round-hit:
    staged-security-with-data-state-current-query-partial-opening-hit-at i
      (Some (((data, attacker-state), result), final-state))
  unfolding staged-security-with-data-state-current-query-partial-opening-hit-def
  by blast

```

have *actual-round*:
checked-staged-security-with-actual-alpha-prefix-current-query-partial-opening-hit-at
i
(Some (((((alpha-prefix, alpha-prefix-state), data), attacker-state),
result), final-state)))
using *round-hit*
unfolding
checked-staged-security-with-actual-alpha-prefix-current-query-partial-opening-hit-at-def
by *simp*
show *?thesis*
unfolding *out-eq*
checked-staged-security-with-actual-alpha-prefix-current-query-partial-opening-hit-def
using *i-bound actual-round* **by** *blast*
qed

lemma *checked-staged-security-with-actual-alpha-prefix-composition-candidate-drift-query-hit-bound-from-actual-alpha*
assumes *wf: staged-budget-wellformed budgets*
and *controlled: staged-adversary-controlled budgets A*
and *current-query-bound:*
wp-event
(checked-staged-security-experiment-with-actual-alpha-prefix-data-state A)
checked-staged-security-with-actual-alpha-prefix-current-query-partial-opening-hit
adversary-initial-state ≤ Q
shows
wp-event
(checked-staged-security-experiment-with-actual-alpha-prefix-data-state A)
checked-staged-security-with-actual-alpha-prefix-composition-candidate-drift-query-hit
adversary-initial-state ≤ Q
by (*rule order-trans[*
OF - current-query-bound])
(*rule wp-event-mono-on-support,*
rule checked-staged-security-with-actual-alpha-prefix-composition-candidate-drift-query-hit-imp-current-query-part
[OF wf controlled])

lemma *checked-staged-security-with-actual-alpha-prefix-current-query-partial-opening-hit-at-aligned-transcript-classif*
assumes *wf: staged-budget-wellformed budgets*
and *controlled: staged-adversary-controlled budgets A*
and *support:*
Some (((((alpha-prefix, alpha-prefix-state), data), attacker-state),
result), final-state) ∈
set-dist
(execute
(checked-staged-security-experiment-with-actual-alpha-prefix-data-state
A)
adversary-initial-state)
and *partial:*
accepted-with-partial-initial-openings-aligned-transcript-consistent
(verifier-state-from-adversary attacker-state
(staged-proof-transcript data))

```

    (Some (result, final-state)) fr f-fri-roots f-final as dg
    composition-fri-roots final query-idxs trace-openings
    composition-openings
and no-bad:
  ¬ partial-merkle-inconsistency-bad
    (verifier-state-from-adversary attacker-state
      (staged-proof-transcript data))
    (Some (result, final-state))
and hit:
  checked-staged-security-with-actual-alpha-prefix-current-query-partial-opening-hit-at
    i
    (Some (((alpha-prefix, alpha-prefix-state), data), attacker-state),
      result), final-state))
shows
  trace-fri-bad-with-partial-candidates
    (verifier-state-from-adversary attacker-state
      (staged-proof-transcript data))
    (Some (result, final-state)) ∨
  composition-fri-bad-with-partial-candidates
    (verifier-state-from-adversary attacker-state
      (staged-proof-transcript data))
    (Some (result, final-state)) ∨
  query-bad-with-partial-candidates
    (verifier-state-from-adversary attacker-state
      (staged-proof-transcript data))
    (Some (result, final-state)) ∨
  (∃ trace-table composition-table trace-openings-i
    composition-openings-i.
    partial-trace-table-candidate trace-table
      (trace-openings[i := trace-openings-i]) ∧
    partial-composition-table-candidate composition-table
      (composition-openings[i := composition-openings-i]) ∧
    trace-table-low-degree trace-table ∧
    composition-table-low-degree maxDegree composition-table ∧
    all-queries-consistent trace-table composition-table as)
proof –
have data-support:
  Some (((data, attacker-state), result), final-state) ∈
    set-dist
      (execute
        (checked-staged-security-experiment-with-data-state A)
        adversary-initial-state)
by (rule
  checked-staged-security-with-actual-alpha-prefix-support-imp-data-state-support
  [OF support])
have data-hit:
  staged-security-with-data-state-current-query-partial-opening-hit-at i
    (Some (((data, attacker-state), result), final-state))
using hit

```

unfolding
checked-staged-security-with-actual-alpha-prefix-current-query-partial-opening-hit-at-def
by *simp*
show *?thesis*
by (rule
staged-security-with-data-state-current-query-partial-opening-hit-at-aligned-transcript-classification-on-support
[OF wf controlled data-support partial no-bad data-hit])
qed

lemma *checked-staged-security-with-actual-alpha-prefix-current-query-partial-opening-hit-aligned-transcript-classification-on-support*
assumes *wf: staged-budget-wellformed budgets*
and *controlled: staged-adversary-controlled budgets A*
and *support:*
Some (((((alpha-prefix, alpha-prefix-state), data), attacker-state),
result), final-state) ∈
set-dist
(execute
(checked-staged-security-experiment-with-actual-alpha-prefix-data-state
A)
adversary-initial-state)
and *partial:*
accepted-with-partial-initial-openings-aligned-transcript-consistent
(verifier-state-from-adversary attacker-state
(staged-proof-transcript data))
(Some (result, final-state)) fr f-fri-roots f-final as dg
composition-fri-roots final query-idxs trace-openings
composition-openings
and *no-bad:*
 $\neg$ *partial-merkle-inconsistency-bad*
(verifier-state-from-adversary attacker-state
(staged-proof-transcript data))
(Some (result, final-state))
and *hit:*
checked-staged-security-with-actual-alpha-prefix-current-query-partial-opening-hit
(Some (((((alpha-prefix, alpha-prefix-state), data), attacker-state),
result), final-state))
shows
trace-fri-bad-with-partial-candidates
(verifier-state-from-adversary attacker-state
(staged-proof-transcript data))
(Some (result, final-state)) ∨
composition-fri-bad-with-partial-candidates
(verifier-state-from-adversary attacker-state
(staged-proof-transcript data))
(Some (result, final-state)) ∨
query-bad-with-partial-candidates
(verifier-state-from-adversary attacker-state
(staged-proof-transcript data))
(Some (result, final-state)) ∨

```

(∃ i trace-table composition-table trace-openings-i
  composition-openings-i.
  i < rounds ∧
  partial-trace-table-candidate trace-table
    (trace-openings[i := trace-openings-i]) ∧
  partial-composition-table-candidate composition-table
    (composition-openings[i := composition-openings-i]) ∧
  trace-table-low-degree trace-table ∧
  composition-table-low-degree maxDegree composition-table ∧
  all-queries-consistent trace-table composition-table as)
proof –
from hit obtain i where i-bound: i < rounds
and round-hit:
  checked-staged-security-with-actual-alpha-prefix-current-query-partial-opening-hit-at
    i
    (Some (((alpha-prefix, alpha-prefix-state), data), attacker-state),
      result), final-state))
unfolding
  checked-staged-security-with-actual-alpha-prefix-current-query-partial-opening-hit-def
by blast
have classified:
  trace-fri-bad-with-partial-candidates
    (verifier-state-from-adversary attacker-state
      (staged-proof-transcript data))
    (Some (result, final-state)) ∨
  composition-fri-bad-with-partial-candidates
    (verifier-state-from-adversary attacker-state
      (staged-proof-transcript data))
    (Some (result, final-state)) ∨
  query-bad-with-partial-candidates
    (verifier-state-from-adversary attacker-state
      (staged-proof-transcript data))
    (Some (result, final-state)) ∨
  (∃ trace-table composition-table trace-openings-i
    composition-openings-i.
    partial-trace-table-candidate trace-table
      (trace-openings[i := trace-openings-i]) ∧
    partial-composition-table-candidate composition-table
      (composition-openings[i := composition-openings-i]) ∧
    trace-table-low-degree trace-table ∧
    composition-table-low-degree maxDegree composition-table ∧
    all-queries-consistent trace-table composition-table as)
by (rule
  checked-staged-security-with-actual-alpha-prefix-current-query-partial-opening-hit-at-aligned-transcript-classificat
    [OF wf controlled support partial no-bad round-hit])
then show ?thesis
using i-bound by blast
qed

```

lemma *checked-staged-actual-alpha-prefix-query-prefix-receive-alignment-from-final-lookup:*

assumes *wf: staged-budget-wellformed budgets*

and *controlled: staged-adversary-controlled budgets A*

and *i-bound: i < rounds*

and *support:*

Some (((alpha-prefix, alpha-prefix-state), data), attacker-state),
result), final-state) ∈
set-dist
(execute
(ichecked-staged-security-experiment-with-actual-alpha-prefix-data-state
A)
adversary-initial-state)

and *final-lookup:*

fmlookup (HashMap final-state)
(QueryIndexChallenge i
(state-after-query-chunks
(staged-query-start-hash data) (staged-query-chunks data) i)) =
Some raw

obtains *prefix prefix-state raw-state where*

Some (((prefix, prefix-state), raw), raw-state) ∈
set-dist
(execute (checked-staged-query-prefix-receive-with-state A i)
adversary-initial-state)

sqp-alphas prefix = staged-alphas data
sqp-query-chunks prefix = take i (staged-query-chunks data)
PQueryCounter prefix-state = i
PState prefix-state =
state-after-query-chunks
(staged-query-start-hash data) (staged-query-chunks data) i

proof –

let *?s =*

verifier-state-from-adversary attacker-state
(staged-proof-transcript data)

have *data-support:*

Some (((data, attacker-state), result), final-state) ∈
set-dist
(execute
(ichecked-staged-security-experiment-with-data-state A)
adversary-initial-state)

by (rule
checked-staged-security-with-actual-alpha-prefix-support-imp-data-state-support
[OF support])

from *checked-staged-security-experiment-with-data-state-outcomeE*
[OF data-support]

obtain *builder verifier where builder:*

Some (data, attacker-state) ∈
set-dist
(execute (checked-staged-transcript-program A)
adversary-initial-state)


```

and verifier:
  Some (result, final-state) ∈ set-dist (execute verify-monad ?s)
by blast
show ?thesis
proof (rule checked-staged-transcript-program-query-prefix-receive-support
  [OF wf controlled i-bound builder])
fix prefix prefix-state raw' raw-state
assume prefix-receive:
  Some (((prefix, prefix-state), raw'), raw-state) ∈
    set-dist
    (execute (checked-staged-query-prefix-receive-with-state A i)
      adversary-initial-state)
and alphas-eq: sqp-alphas prefix = staged-alphas data
and chunks-eq:
  sqp-query-chunks prefix = take i (staged-query-chunks data)
and counter-eq: PQueryCounter prefix-state = i
and state-eq:
  PState prefix-state =
    state-after-query-chunks
    (staged-query-start-hash data) (staged-query-chunks data) i
and raw-ext: raw-state ≤ attacker-state
and raw-lookup:
  fmlookup (HashMap raw-state)
    (QueryIndexChallenge i
      (state-after-query-chunks
        (staged-query-start-hash data) (staged-query-chunks data) i)) =
    Some raw'
have raw-ext-s: raw-state ≤ ?s
  by (rule hash-extends-verifier-state-from-adversary-right[OF raw-ext])
have s-ext-final: ?s ≤ final-state
  by (rule verify-monad-hash-extends[OF verifier])
have raw-ext-final: raw-state ≤ final-state
  by (rule hash-ext-trans[OF raw-ext-s s-ext-final])
have final-lookup-raw':
  fmlookup (HashMap final-state)
    (QueryIndexChallenge i
      (state-after-query-chunks
        (staged-query-start-hash data) (staged-query-chunks data) i)) =
    Some raw'
  by (rule hash-extension-lookup[OF raw-lookup raw-ext-final])
have raw-eq: raw' = raw
  using final-lookup final-lookup-raw' by simp
have prefix-receive-raw:
  Some (((prefix, prefix-state), raw), raw-state) ∈
    set-dist
    (execute (checked-staged-query-prefix-receive-with-state A i)
      adversary-initial-state)
  using prefix-receive raw-eq by simp
show ?thesis

```

```

    by (rule that[OF prefix-receive-raw alphas-eq chunks-eq counter-eq
        state-eq])
  qed
qed
end
end

```

```

theory Soundness-Conceptual-Table
imports Soundness-Partial-Merkle
begin

```

Conceptual Merkle tables.

This layer deliberately does not reconstruct a complete *created-tree* witness from sampled openings. Instead it records the value that is uniquely authenticated for a root, length, and index in a hash-map state. Values at unauthenticated indices are arbitrary Hilbert-choice defaults; later proofs must only use them after showing agreement with authenticated openings or charging disagreement to a Merkle/hash side event.

The intended prefix-fixed use is to build these tables from the query-prefix oracle state, before the query-index challenge is sampled. Later openings checked by the verifier are handled in a downstream layer via pullback or controlled collision/inconsistency events.

```

context soundness
begin

```

```

definition authenticated-value-at
  :: ('f, 'a) protocol-channel-scheme  $\Rightarrow$ 
    'f  $\Rightarrow$  nat  $\Rightarrow$  nat  $\Rightarrow$  'f  $\Rightarrow$  bool

```

where

```

  authenticated-value-at s r len i v  $\longleftrightarrow$ 
    ( $\exists$  opn :: 'f authenticated-opening.
      authenticated-opening-in s opn  $\wedge$ 
      opening-root opn = r  $\wedge$ 
      opening-length opn = len  $\wedge$ 
      opening-index opn = i  $\wedge$ 
      opening-value opn = v)

```

lemma authenticated-value-atI:

```

  assumes authenticated-opening-in s opn
    and opening-root opn = r
    and opening-length opn = len
    and opening-index opn = i
    and opening-value opn = v
  shows authenticated-value-at s r len i v
  using assms unfolding authenticated-value-at-def by blast

```

lemma authenticated-value-atE:

```

assumes authenticated-value-at s r len i v
obtains opn where
  authenticated-opening-in s opn
  opening-root opn = r
  opening-length opn = len
  opening-index opn = i
  opening-value opn = v
using assms unfolding authenticated-value-at-def by blast

lemma authenticated-value-at-unique-if-clean:
  assumes clean:  $\neg$  hash-map-output-collision s
    and v: authenticated-value-at s r len i v
    and v': authenticated-value-at s r len i v'
  shows v = v'
proof –
  from authenticated-value-atE[OF v] obtain opn where
    auth: authenticated-opening-in s opn
    and root: opening-root opn = r
    and len-eq: opening-length opn = len
    and idx: opening-index opn = i
    and val: opening-value opn = v
    by blast
  from authenticated-value-atE[OF v'] obtain opn' where
    auth': authenticated-opening-in s opn'
    and root': opening-root opn' = r
    and len-eq': opening-length opn' = len
    and idx': opening-index opn' = i
    and val': opening-value opn' = v'
    by blast
  show ?thesis
  proof (rule ccontr)
    assume v  $\neq$  v'
    then have diff: opening-value opn  $\neq$  opening-value opn'
      using val val' by simp
    have collision: hash-map-output-collision s
      by (rule inconsistent-authenticated-openings-imp-hash-collision
        [OF auth auth'])
      (use root root' len-eq len-eq' idx idx' diff in simp-all)
    then show False
      using clean by contradiction
  qed
qed

lemma authenticated-value-at-mono:
  assumes v: authenticated-value-at s r len i x
    and ext: s  $\leq$  t
  shows authenticated-value-at t r len i x
proof –
  from authenticated-value-atE[OF v] obtain opn where

```

```

    auth: authenticated-opening-in s opn
  and root: opening-root opn = r
  and len-eq: opening-length opn = len
  and idx: opening-index opn = i
  and val: opening-value opn = x
  by blast
have authenticated-opening-in t opn
  by (rule authenticated-opening-in-mono[OF auth ext])
then show ?thesis
  by (rule authenticated-value-atI[OF - root len-eq idx val])
qed

definition conceptual-opening-value
  :: ('f, 'a) protocol-channel-scheme  $\Rightarrow$  'f  $\Rightarrow$  nat  $\Rightarrow$  nat  $\Rightarrow$  'f
where
  conceptual-opening-value s r len i =
    (THE v. authenticated-value-at s r len i v)

lemma conceptual-opening-value-eq-if-authenticated:
  assumes clean:  $\neg$  hash-map-output-collision s
  and v: authenticated-value-at s r len i x
  shows conceptual-opening-value s r len i = x
  unfolding conceptual-opening-value-def
proof (rule the-equality)
  show authenticated-value-at s r len i x
  by (rule v)
next
  fix y
  assume authenticated-value-at s r len i y
  then show y = x
  using authenticated-value-at-unique-if-clean[OF clean - v] by blast
qed

definition conceptual-table
  :: ('f, 'a) protocol-channel-scheme  $\Rightarrow$  'f  $\Rightarrow$  nat  $\Rightarrow$  'f list
where
  conceptual-table s r len =
    map (conceptual-opening-value s r len) [0..definition conceptual-opening-value-with-default
  :: ('f, 'a) protocol-channel-scheme  $\Rightarrow$  'f  $\Rightarrow$  nat  $\Rightarrow$ 
    'f  $\Rightarrow$  nat  $\Rightarrow$  'f
where
  conceptual-opening-value-with-default s r len dflt i =
    (if  $\exists$  v. authenticated-value-at s r len i v
     then conceptual-opening-value s r len i
     else dflt)

definition conceptual-table-with-default

```

```

:: ('f, 'a) protocol-channel-scheme ⇒ 'f ⇒ nat ⇒
   'f list ⇒ 'f list
where
  conceptual-table-with-default s r len default-table =
    map (λi. conceptual-opening-value-with-default s r len
      (default-table ! i) i) [0..

```

```

     $\bigwedge i v. i < \text{len} \implies \text{authenticated-value-at } s \ r \ \text{len } i \ v \implies$ 
     $\text{default-table } ! i = v$ 
  shows conceptual-table-with-default s r len default-table = default-table
proof (rule nth-equalityI)
  show length (conceptual-table-with-default s r len default-table) =
    length default-table
  using len-eq by simp
next
fix i
assume i-bound:
  i < length (conceptual-table-with-default s r len default-table)
then have i-len: i < len
  by simp
show conceptual-table-with-default s r len default-table ! i =
  default-table ! i
proof (cases  $\exists v. \text{authenticated-value-at } s \ r \ \text{len } i \ v$ )
  case True
  then obtain v where v: authenticated-value-at s r len i v
    by blast
  have
    conceptual-table-with-default s r len default-table ! i =
    conceptual-opening-value-with-default s r len (default-table ! i) i
    by (rule conceptual-table-with-default-nth[OF i-len])
  also have ... = v
    by (rule conceptual-opening-value-with-default-eq-if-authenticated
      [OF clean v])
  also have ... = default-table ! i
    using agrees[OF i-len v] by simp
  finally show ?thesis .
next
case False
have
  conceptual-table-with-default s r len default-table ! i =
  conceptual-opening-value-with-default s r len (default-table ! i) i
  by (rule conceptual-table-with-default-nth[OF i-len])
  also have ... = default-table ! i
    by (rule
      conceptual-opening-value-with-default-eq-default-if-not-authenticated
      [OF False])
  finally show ?thesis .
qed
qed

lemma conceptual-table-agrees-with-authenticated-opening:
  assumes clean:  $\neg \text{hash-map-output-collision } s$ 
  and auth: authenticated-opening-in s opn
  and root: opening-root opn = r
  and len-eq: opening-length opn = len
  and idx-bound: opening-index opn < len

```

```

shows
  conceptual-table s r len ! opening-index opn = opening-value opn
proof –
  have v:
    authenticated-value-at s r len (opening-index opn)
    (opening-value opn)
    by (rule authenticated-value-atI[OF auth root len-eq refl refl])
  have conceptual-table s r len ! opening-index opn =
    conceptual-opening-value s r len (opening-index opn)
    by (rule conceptual-table-nth[OF idx-bound])
  also have ... = opening-value opn
    by (rule conceptual-opening-value-eq-if-authenticated[OF clean v])
  finally show ?thesis .
qed

lemma conceptual-table-agrees-with-partial-authenticated-table:
  assumes clean:  $\neg$  hash-map-output-collision s
    and table: partial-authenticated-table r len openings s
    and opn-in: opn  $\in$  set openings
  shows
    conceptual-table s r len ! opening-index opn = opening-value opn
proof –
  have auth: authenticated-opening-in s opn
    using table opn-in unfolding partial-authenticated-table-def by blast
  have root: opening-root opn = r
    using table opn-in unfolding partial-authenticated-table-def by blast
  have len-eq: opening-length opn = len
    using table opn-in unfolding partial-authenticated-table-def by blast
  have idx-bound: opening-index opn < len
    using auth len-eq unfolding authenticated-opening-in-def by simp
  show ?thesis
    by (rule conceptual-table-agrees-with-authenticated-opening
      [OF clean auth root len-eq idx-bound])
qed

end

end

```

```

theory Soundness-Conceptual-Query-Prefix
  imports Soundness-Conceptual-Table
begin

```

Prefix-fixed conceptual tables.

These tables are built from a staged query-prefix state, i.e. after the transcript prefix and earlier query rounds have been processed, but before the next query-index challenge is sampled. This is the state selection needed for later dynamic query-target bounds.

```

context soundness

```

begin

definition *query-prefix-trace-conceptual-table*
:: 'f staged-query-prefix-data $\Rightarrow$
('f, 'a) protocol-channel-scheme $\Rightarrow$ 'f list

where

query-prefix-trace-conceptual-table prefix s =
conceptual-table s (sqp-trace-root prefix) (scale * clength)

definition *query-prefix-composition-conceptual-table*
:: 'f staged-query-prefix-data $\Rightarrow$
('f, 'a) protocol-channel-scheme $\Rightarrow$ 'f list

where

query-prefix-composition-conceptual-table prefix s =
conceptual-table s (hd (sqp-composition-fri-roots prefix))
(scale * clength)

definition *query-prefix-trace-conceptual-table-with-default*
:: 'f staged-query-prefix-data $\Rightarrow$
('f, 'a) protocol-channel-scheme $\Rightarrow$ 'f list $\Rightarrow$ 'f list

where

query-prefix-trace-conceptual-table-with-default prefix s default-table =
conceptual-table-with-default s (sqp-trace-root prefix)
(scale * clength) default-table

definition *query-prefix-composition-conceptual-table-with-default*
:: 'f staged-query-prefix-data $\Rightarrow$
('f, 'a) protocol-channel-scheme $\Rightarrow$ 'f list $\Rightarrow$ 'f list

where

query-prefix-composition-conceptual-table-with-default prefix s
default-table =
conceptual-table-with-default s (hd (sqp-composition-fri-roots prefix))
(scale * clength) default-table

lemma *length-query-prefix-trace-conceptual-table[simp]*:
length (query-prefix-trace-conceptual-table prefix s) = scale * clength
unfolding query-prefix-trace-conceptual-table-def **by** simp

lemma *length-query-prefix-composition-conceptual-table[simp]*:
length (query-prefix-composition-conceptual-table prefix s) =
scale * clength
unfolding query-prefix-composition-conceptual-table-def **by** simp

lemma *length-query-prefix-trace-conceptual-table-with-default[simp]*:
length
(query-prefix-trace-conceptual-table-with-default prefix s
default-table) = scale * clength
unfolding query-prefix-trace-conceptual-table-with-default-def **by** simp

lemma *length-query-prefix-composition-conceptual-table-with-default*[simp]:
length
 (*query-prefix-composition-conceptual-table-with-default* *prefix s*
default-table) = *scale * clength*
unfolding *query-prefix-composition-conceptual-table-with-default-def*
by *simp*

lemma *query-prefix-trace-conceptual-table-agrees-with-authenticated-opening*:
assumes *clean*: $\neg$ *hash-map-output-collision s*
and *auth*: *authenticated-opening-in s opn*
and *root*: *opening-root opn = sqp-trace-root prefix*
and *len-eq*: *opening-length opn = scale * clength*
and *idx-bound*: *opening-index opn < scale * clength*
shows
query-prefix-trace-conceptual-table prefix s ! opening-index opn =
opening-value opn
unfolding *query-prefix-trace-conceptual-table-def*
by (*rule conceptual-table-agrees-with-authenticated-opening*
 [*OF clean auth root len-eq idx-bound*])

lemma *query-prefix-composition-conceptual-table-agrees-with-authenticated-opening*:
assumes *clean*: $\neg$ *hash-map-output-collision s*
and *auth*: *authenticated-opening-in s opn*
and *roots-nonempty*: *sqp-composition-fri-roots prefix* $\neq []$
and *root*: *opening-root opn = hd (sqp-composition-fri-roots prefix)*
and *len-eq*: *opening-length opn = scale * clength*
and *idx-bound*: *opening-index opn < scale * clength*
shows
query-prefix-composition-conceptual-table prefix s !
opening-index opn = opening-value opn
unfolding *query-prefix-composition-conceptual-table-def*
by (*rule conceptual-table-agrees-with-authenticated-opening*
 [*OF clean auth root len-eq idx-bound*])

lemma
query-prefix-trace-conceptual-table-with-default-agrees-with-authenticated-opening:
assumes *clean*: $\neg$ *hash-map-output-collision s*
and *auth*: *authenticated-opening-in s opn*
and *root*: *opening-root opn = sqp-trace-root prefix*
and *len-eq*: *opening-length opn = scale * clength*
and *idx-bound*: *opening-index opn < scale * clength*
shows
query-prefix-trace-conceptual-table-with-default prefix s
default-table ! opening-index opn =
opening-value opn
proof –
have *v*:
*authenticated-value-at s (sqp-trace-root prefix) (scale * clength)*
(opening-index opn) (opening-value opn)

```

    by (rule authenticated-value-atI[OF auth root len-eq refl refl])
  have
    query-prefix-trace-conceptual-table-with-default prefix s
      default-table ! opening-index opn =
      conceptual-opening-value-with-default s (sqp-trace-root prefix)
        (scale * clength) (default-table ! opening-index opn)
        (opening-index opn)
    unfolding query-prefix-trace-conceptual-table-with-default-def
    by (rule conceptual-table-with-default-nth[OF idx-bound])
  also have ... = opening-value opn
    by (rule conceptual-opening-value-with-default-eq-if-authenticated
        [OF clean v])
  finally show ?thesis .
qed

lemma
  query-prefix-composition-conceptual-table-with-default-agrees-with-authenticated-opening:
  assumes clean:  $\neg$  hash-map-output-collision s
    and auth: authenticated-opening-in s opn
    and root: opening-root opn = hd (sqp-composition-fri-roots prefix)
    and len-eq: opening-length opn = scale * clength
    and idx-bound: opening-index opn < scale * clength
  shows
    query-prefix-composition-conceptual-table-with-default prefix s
      default-table ! opening-index opn =
      opening-value opn
  proof -
    have v:
      authenticated-value-at s (hd (sqp-composition-fri-roots prefix))
        (scale * clength) (opening-index opn) (opening-value opn)
    by (rule authenticated-value-atI[OF auth root len-eq refl refl])
    have
      query-prefix-composition-conceptual-table-with-default prefix s
        default-table ! opening-index opn =
        conceptual-opening-value-with-default s
          (hd (sqp-composition-fri-roots prefix)) (scale * clength)
          (default-table ! opening-index opn) (opening-index opn)
      unfolding query-prefix-composition-conceptual-table-with-default-def
      by (rule conceptual-table-with-default-nth[OF idx-bound])
    also have ... = opening-value opn
      by (rule conceptual-opening-value-with-default-eq-if-authenticated
          [OF clean v])
    finally show ?thesis .
  qed

lemma query-prefix-trace-conceptual-table-agrees-with-partial-authenticated-table:
  assumes clean:  $\neg$  hash-map-output-collision s
    and table:
      partial-authenticated-table (sqp-trace-root prefix)

```

(scale * clength) openings s
and opn-in: opn ∈ set openings
shows
 query-prefix-trace-conceptual-table prefix s ! opening-index opn =
 opening-value opn
unfolding query-prefix-trace-conceptual-table-def
by (rule conceptual-table-agrees-with-partial-authenticated-table
 [OF clean table opn-in])

lemma

query-prefix-composition-conceptual-table-agrees-with-partial-authenticated-table:
assumes clean: ¬ hash-map-output-collision s
and table:
 partial-authenticated-table (hd (sqp-composition-fri-roots prefix))
 (scale * clength) openings s
and opn-in: opn ∈ set openings
shows
 query-prefix-composition-conceptual-table prefix s ! opening-index opn =
 opening-value opn
unfolding query-prefix-composition-conceptual-table-def
by (rule conceptual-table-agrees-with-partial-authenticated-table
 [OF clean table opn-in])

lemma

query-prefix-trace-conceptual-table-with-default-agrees-with-partial-authenticated-table:
assumes clean: ¬ hash-map-output-collision s
and table:
 partial-authenticated-table (sqp-trace-root prefix)
 (scale * clength) openings s
and opn-in: opn ∈ set openings
shows
 query-prefix-trace-conceptual-table-with-default prefix s
 default-table ! opening-index opn =
 opening-value opn

proof –

have auth: authenticated-opening-in s opn
using table opn-in **unfolding** partial-authenticated-table-def **by** blast
have root: opening-root opn = sqp-trace-root prefix
using table opn-in **unfolding** partial-authenticated-table-def **by** blast
have len-eq: opening-length opn = scale * clength
using table opn-in **unfolding** partial-authenticated-table-def **by** blast
have idx-bound: opening-index opn < scale * clength
using auth len-eq **unfolding** authenticated-opening-in-def **by** simp
show ?thesis
by (rule
 query-prefix-trace-conceptual-table-with-default-agrees-with-authenticated-opening
 [OF clean auth root len-eq idx-bound])

qed

lemma

query-prefix-composition-conceptual-table-with-default-agrees-with-partial-authenticated-table:

assumes *clean*: $\neg$ *hash-map-output-collision* *s*

and *table*:

partial-authenticated-table (*hd* (*sqp-composition-fri-roots* *prefix*))
*(scale * clength)* *openings* *s*

and *opn-in*: *opn* $\in$ *set* *openings*

shows

query-prefix-composition-conceptual-table-with-default *prefix* *s*
default-table ! *opening-index* *opn* =
opening-value *opn*

proof –

have *auth*: *authenticated-opening-in* *s* *opn*

using *table* *opn-in* **unfolding** *partial-authenticated-table-def* **by** *blast*

have *root*: *opening-root* *opn* = *hd* (*sqp-composition-fri-roots* *prefix*)

using *table* *opn-in* **unfolding** *partial-authenticated-table-def* **by** *blast*

have *len-eq*: *opening-length* *opn* = *scale* * *clength*

using *table* *opn-in* **unfolding** *partial-authenticated-table-def* **by** *blast*

have *idx-bound*: *opening-index* *opn* < *scale* * *clength*

using *auth* *len-eq* **unfolding** *authenticated-opening-in-def* **by** *simp*

show *?thesis*

by (*rule*

query-prefix-composition-conceptual-table-with-default-agrees-with-authenticated-opening
[*OF* *clean* *auth* *root* *len-eq* *idx-bound*])

qed

lemma *query-prefix-trace-conceptual-table-with-default-eq-default-if-agrees-with-authenticated-values:*

assumes *clean*: $\neg$ *hash-map-output-collision* *s*

and *len-eq*: *length* *default-table* = *scale* * *clength*

and *agrees*:

$\bigwedge i\ v. i < \text{scale} * \text{clength} \implies$
authenticated-value-at *s* (*sqp-trace-root* *prefix*)
*(scale * clength)* *i* *v* $\implies$
default-table ! *i* = *v*

shows

query-prefix-trace-conceptual-table-with-default *prefix* *s*
default-table = *default-table*

unfolding *query-prefix-trace-conceptual-table-with-default-def*

by (*rule* *conceptual-table-with-default-eq-if-agrees-with-authenticated-values*
[*OF* *clean* *len-eq* *agrees*])

lemma *query-prefix-composition-conceptual-table-with-default-eq-default-if-agrees-with-authenticated-values:*

assumes *clean*: $\neg$ *hash-map-output-collision* *s*

and *len-eq*: *length* *default-table* = *scale* * *clength*

and *agrees*:

$\bigwedge i\ v. i < \text{scale} * \text{clength} \implies$
authenticated-value-at *s* (*hd* (*sqp-composition-fri-roots* *prefix*))
*(scale * clength)* *i* *v* $\implies$
default-table ! *i* = *v*

shows
query-prefix-composition-conceptual-table-with-default prefix s
default-table = default-table
unfolding *query-prefix-composition-conceptual-table-with-default-def*
by (*rule conceptual-table-with-default-eq-if-agrees-with-authenticated-values*
[OF clean len-eq agrees])

lemma *query-prefix-trace-conceptual-table-partial-trace-candidate:*
assumes *clean: $\neg$ hash-map-output-collision s*
and *len: length trace-openings = rounds*
and *tables:*
 $\bigwedge i. i < \text{rounds} \implies$
partial-authenticated-table (spp-trace-root prefix)
*(scale * clength) (trace-openings ! i) s*
shows
partial-trace-table-candidate
(query-prefix-trace-conceptual-table prefix s) trace-openings
unfolding *partial-trace-table-candidate-def*
proof (*intro conjI allI impI*)
show *length (query-prefix-trace-conceptual-table prefix s) =*
*scale * clength*
by *simp*
next
show *length trace-openings = rounds*
by (*rule len*)
next
fix *i*
assume *i-bound: i < rounds*
show
table-agrees-with-authenticated-openings
*(query-prefix-trace-conceptual-table prefix s) (scale * clength)*
(trace-openings ! i)
unfolding *table-agrees-with-authenticated-openings-def*
proof (*intro conjI ballI*)
show *length (query-prefix-trace-conceptual-table prefix s) =*
*scale * clength*
by *simp*
next
fix *opn*
assume *opn-in: opn $\in$ set (trace-openings ! i)*
have *table: partial-authenticated-table (spp-trace-root prefix)*
*(scale * clength) (trace-openings ! i) s*
by (*rule tables[OF i-bound]*)
show *opening-index opn < scale * clength*
using *table opn-in*
unfolding *partial-authenticated-table-def authenticated-opening-in-def*
by *auto*
next
fix *opn*

```

assume opn-in: opn  $\in$  set (trace-openings ! i)
have table: partial-authenticated-table (sqp-trace-root prefix)
  (scale * clength) (trace-openings ! i) s
  by (rule tables[OF i-bound])
show
  query-prefix-trace-conceptual-table prefix s ! opening-index opn =
    opening-value opn
  by (rule
    query-prefix-trace-conceptual-table-agrees-with-partial-authenticated-table
    [OF clean table opn-in])
qed
qed

lemma query-prefix-trace-conceptual-table-with-default-partial-trace-candidate:
assumes clean:  $\neg$  hash-map-output-collision s
and len: length trace-openings = rounds
and tables:
   $\bigwedge i. i < \text{rounds} \implies$ 
    partial-authenticated-table (sqp-trace-root prefix)
    (scale * clength) (trace-openings ! i) s
shows
  partial-trace-table-candidate
    (query-prefix-trace-conceptual-table-with-default prefix s
     default-table) trace-openings
unfolding partial-trace-table-candidate-def
proof (intro conjI allI impI)
show
  length
    (query-prefix-trace-conceptual-table-with-default prefix s
     default-table) =
    scale * clength
  by simp
next
show length trace-openings = rounds
  by (rule len)
next
fix i
assume i-bound: i < rounds
show
  table-agrees-with-authenticated-openings
    (query-prefix-trace-conceptual-table-with-default prefix s
     default-table)
    (scale * clength) (trace-openings ! i)
unfolding table-agrees-with-authenticated-openings-def
proof (intro conjI ballI)
show
  length
    (query-prefix-trace-conceptual-table-with-default prefix s
     default-table) =

```

```

      scale * clength
    by simp
next
fix opn
assume opn-in: opn ∈ set (trace-openings ! i)
have table: partial-authenticated-table (sqp-trace-root prefix)
  (scale * clength) (trace-openings ! i) s
  by (rule tables[OF i-bound])
show opening-index opn < scale * clength
  using table opn-in
  unfolding partial-authenticated-table-def authenticated-opening-in-def
  by auto
next
fix opn
assume opn-in: opn ∈ set (trace-openings ! i)
have table: partial-authenticated-table (sqp-trace-root prefix)
  (scale * clength) (trace-openings ! i) s
  by (rule tables[OF i-bound])
show
  query-prefix-trace-conceptual-table-with-default prefix s
  default-table ! opening-index opn =
  opening-value opn
  by (rule
    query-prefix-trace-conceptual-table-with-default-agrees-with-partial-authenticated-table
    [OF clean table opn-in])
qed
qed

lemma query-prefix-composition-conceptual-table-partial-composition-candidate:
  assumes clean: ¬ hash-map-output-collision s
  and len: length composition-openings = rounds
  and tables:
    ∧ i. i < rounds ⇒
      partial-authenticated-table (hd (sqp-composition-fri-roots prefix))
      (scale * clength) (composition-openings ! i) s
  shows
    partial-composition-table-candidate
      (query-prefix-composition-conceptual-table prefix s)
      composition-openings
  unfolding partial-composition-table-candidate-def
proof (intro conjI allI impI)
  show
    length (query-prefix-composition-conceptual-table prefix s) =
      scale * clength
    by simp
next
  show length composition-openings = rounds
    by (rule len)
next

```

```

fix i
assume i-bound:  $i < \text{rounds}$ 
show
  table-agrees-with-authenticated-openings
    (query-prefix-composition-conceptual-table prefix s)
    (scale * clength) (composition-openings ! i)
  unfolding table-agrees-with-authenticated-openings-def
proof (intro conjI ballI)
  show
    length (query-prefix-composition-conceptual-table prefix s) =
      scale * clength
    by simp
next
  fix opn
  assume opn-in:  $\text{opn} \in \text{set } (\text{composition-openings} ! i)$ 
  have table:
    partial-authenticated-table (hd (sqp-composition-fri-roots prefix))
      (scale * clength) (composition-openings ! i) s
    by (rule tables[OF i-bound])
  show opening-index opn < scale * clength
    using table opn-in
    unfolding partial-authenticated-table-def authenticated-opening-in-def
    by auto
next
  fix opn
  assume opn-in:  $\text{opn} \in \text{set } (\text{composition-openings} ! i)$ 
  have table:
    partial-authenticated-table (hd (sqp-composition-fri-roots prefix))
      (scale * clength) (composition-openings ! i) s
    by (rule tables[OF i-bound])
  show
    query-prefix-composition-conceptual-table prefix s !
      opening-index opn = opening-value opn
    by (rule
      query-prefix-composition-conceptual-table-agrees-with-partial-authenticated-table
      [OF clean table opn-in])
qed
qed

lemma
  query-prefix-composition-conceptual-table-with-default-partial-composition-candidate:
assumes clean:  $\neg \text{hash-map-output-collision } s$ 
  and len: length composition-openings = rounds
  and tables:
     $\bigwedge i. i < \text{rounds} \implies$ 
      partial-authenticated-table (hd (sqp-composition-fri-roots prefix))
        (scale * clength) (composition-openings ! i) s
shows
  partial-composition-table-candidate

```



```

      (query-prefix-composition-conceptual-table-with-default prefix s
        default-table)
      composition-openings
unfolding partial-composition-table-candidate-def
proof (intro conjI allI impI)
show
  length
    (query-prefix-composition-conceptual-table-with-default prefix s
      default-table) =
    scale * clength
  by simp
next
show length composition-openings = rounds
  by (rule len)
next
fix i
assume i-bound: i < rounds
show
  table-agrees-with-authenticated-openings
    (query-prefix-composition-conceptual-table-with-default prefix s
      default-table)
    (scale * clength) (composition-openings ! i)
unfolding table-agrees-with-authenticated-openings-def
proof (intro conjI ballI)
show
  length
    (query-prefix-composition-conceptual-table-with-default prefix s
      default-table) =
    scale * clength
  by simp
next
fix opn
assume opn-in: opn ∈ set (composition-openings ! i)
have table:
  partial-authenticated-table (hd (sqp-composition-fri-roots prefix))
    (scale * clength) (composition-openings ! i) s
  by (rule tables[OF i-bound])
show opening-index opn < scale * clength
  using table opn-in
  unfolding partial-authenticated-table-def authenticated-opening-in-def
  by auto
next
fix opn
assume opn-in: opn ∈ set (composition-openings ! i)
have table:
  partial-authenticated-table (hd (sqp-composition-fri-roots prefix))
    (scale * clength) (composition-openings ! i) s
  by (rule tables[OF i-bound])
show

```

```

    query-prefix-composition-conceptual-table-with-default prefix s
      default-table ! opening-index opn =
        opening-value opn
  by (rule
    query-prefix-composition-conceptual-table-with-default-agrees-with-partial-authenticated-table
      [OF clean table opn-in])
qed
qed
end
end

```

```

theory Soundness-Conceptual-Query-Target
imports
  Soundness-Conceptual-Query-Prefix
  Soundness-Aligned-Partial-Query-Prefix
begin

```

Prefix-fixed query targets built from conceptual Merkle tables.

This layer deliberately uses only the query-prefix state. It therefore supports the dynamic query-index probability bound, but it does not claim that openings first authenticated after the query challenge agree with these tables. That later-opening bridge is a separate proof obligation.

```

context soundness
begin

```

```

definition staged-query-prefix-conceptual-query-target
  :: 'f staged-query-prefix-data  $\Rightarrow$  'f protocol-channel  $\Rightarrow$  nat set
where
  staged-query-prefix-conceptual-query-target prefix prefix-state =
    query-sampling-success-space
      (query-prefix-trace-conceptual-table prefix prefix-state)
      (query-prefix-composition-conceptual-table prefix prefix-state)
      (sqp-alphas prefix)

```

```

definition staged-query-prefix-conceptual-default-query-target
  :: 'f list  $\Rightarrow$  'f list  $\Rightarrow$ 
    'f staged-query-prefix-data  $\Rightarrow$  'f protocol-channel  $\Rightarrow$  nat set
where
  staged-query-prefix-conceptual-default-query-target trace-default
    composition-default prefix prefix-state =
    query-sampling-success-space
      (query-prefix-trace-conceptual-table-with-default prefix prefix-state
        trace-default)
      (query-prefix-composition-conceptual-table-with-default prefix
        prefix-state composition-default)
      (sqp-alphas prefix)

```

definition *staged-query-prefix-conceptual-empty-query-target*
 $:: 'f \text{ staged-query-prefix-data} \Rightarrow 'f \text{ protocol-channel} \Rightarrow \text{nat set}$
where
staged-query-prefix-conceptual-empty-query-target prefix prefix-state =
query-sampling-success-space
 (*query-prefix-trace-conceptual-table prefix prefix-state*)
 (*replicate (scale * clength) (sqp-composition-final prefix)*)
 (*sqp-alphas prefix*)

lemma *staged-query-prefix-conceptual-query-target-subset:*
staged-query-prefix-conceptual-query-target prefix prefix-state $\subseteq$
query-sample-space
unfolding *staged-query-prefix-conceptual-query-target-def*
by (*rule query-sampling-success-space-subset*)

lemma *staged-query-prefix-conceptual-default-query-target-subset:*
staged-query-prefix-conceptual-default-query-target trace-default
composition-default prefix prefix-state $\subseteq$
query-sample-space
unfolding *staged-query-prefix-conceptual-default-query-target-def*
by (*rule query-sampling-success-space-subset*)

lemma *staged-query-prefix-conceptual-empty-query-target-subset:*
staged-query-prefix-conceptual-empty-query-target prefix prefix-state
 $\subseteq$ *query-sample-space*
unfolding *staged-query-prefix-conceptual-empty-query-target-def*
by (*rule query-sampling-success-space-subset*)

lemma *staged-query-prefix-conceptual-query-target-fraction-bound:*
 nnreal
 (*card*
 (*staged-query-prefix-conceptual-query-target prefix prefix-state*)) /
 $\text{nnreal (card query-sample-space)} \leq \text{query-error-bound}$
unfolding *staged-query-prefix-conceptual-query-target-def*
by (*rule query-sampling-success-space-fraction-bound-query-sample-space*)

lemma *staged-query-prefix-conceptual-default-query-target-fraction-bound:*
 nnreal
 (*card*
 (*staged-query-prefix-conceptual-default-query-target trace-default*
composition-default prefix prefix-state)) /
 $\text{nnreal (card query-sample-space)} \leq \text{query-error-bound}$
unfolding *staged-query-prefix-conceptual-default-query-target-def*
by (*rule query-sampling-success-space-fraction-bound-query-sample-space*)

lemma *staged-query-prefix-conceptual-empty-query-target-fraction-bound:*
 nnreal
 (*card*

$(\text{staged-query-prefix-conceptual-empty-query-target } \text{prefix} \text{ prefix-state})) /$
 $\text{nnreal } (\text{card } \text{query-sample-space}) \leq \text{query-error-bound}$
unfolding $\text{staged-query-prefix-conceptual-empty-query-target-def}$
by $(\text{rule } \text{query-sampling-success-space-fraction-bound-query-sample-space})$

lemma $\text{query-prefix-trace-conceptual-table-with-default-eq-if-agrees:}$

assumes $\text{clean: } \neg \text{hash-map-output-collision } \text{prefix-state}$
and $\text{len: length trace-default} = \text{scale} * \text{clength}$
and agrees:
 $\bigwedge i v. i < \text{scale} * \text{clength} \implies$
 $\text{authenticated-value-at } \text{prefix-state} (\text{sqp-trace-root } \text{prefix})$
 $(\text{scale} * \text{clength}) i v \implies$
 $\text{trace-default} ! i = v$

shows

$\text{query-prefix-trace-conceptual-table-with-default } \text{prefix } \text{prefix-state}$
 $\text{trace-default} = \text{trace-default}$

unfolding $\text{query-prefix-trace-conceptual-table-with-default-def}$

by $(\text{rule } \text{conceptual-table-with-default-eq-if-agrees-with-authenticated-values} [\text{OF clean len agrees}])$

lemma $\text{query-prefix-composition-conceptual-table-with-default-eq-if-agrees:}$

assumes $\text{clean: } \neg \text{hash-map-output-collision } \text{prefix-state}$
and $\text{len: length composition-default} = \text{scale} * \text{clength}$
and agrees:
 $\bigwedge i v. i < \text{scale} * \text{clength} \implies$
 $\text{authenticated-value-at } \text{prefix-state}$
 $(\text{hd } (\text{sqp-composition-fri-roots } \text{prefix})) (\text{scale} * \text{clength}) i v \implies$
 $\text{composition-default} ! i = v$

shows

$\text{query-prefix-composition-conceptual-table-with-default } \text{prefix}$
 $\text{prefix-state } \text{composition-default} = \text{composition-default}$

unfolding $\text{query-prefix-composition-conceptual-table-with-default-def}$

by $(\text{rule } \text{conceptual-table-with-default-eq-if-agrees-with-authenticated-values} [\text{OF clean len agrees}])$

lemma

$\text{staged-query-prefix-candidate-pair-query-target-from-prefix-subset-conceptual-default-if-agrees:}$

assumes $\text{clean: } \neg \text{hash-map-output-collision } \text{prefix-state}$
and $\text{trace-len: length trace-table} = \text{scale} * \text{clength}$
and $\text{composition-len: length composition-table} = \text{scale} * \text{clength}$
and trace-agrees:
 $\bigwedge i v. i < \text{scale} * \text{clength} \implies$
 $\text{authenticated-value-at } \text{prefix-state} (\text{sqp-trace-root } \text{prefix})$
 $(\text{scale} * \text{clength}) i v \implies$
 $\text{trace-table} ! i = v$
and $\text{composition-agrees:}$
 $\bigwedge i v. i < \text{scale} * \text{clength} \implies$
 $\text{authenticated-value-at } \text{prefix-state}$

$(hd \ (sqp\text{-}composition\text{-}fri\text{-}roots \ prefix)) \ (scale * clength) \ i \ v \implies$
 $composition\text{-}table \ ! \ i = v$

shows

$staged\text{-}query\text{-}prefix\text{-}candidate\text{-}pair\text{-}query\text{-}target\text{-}from\text{-}prefix \ trace\text{-}table$
 $composition\text{-}table \ prefix \ prefix\text{-}state \subseteq$
 $staged\text{-}query\text{-}prefix\text{-}conceptual\text{-}default\text{-}query\text{-}target \ trace\text{-}table$
 $composition\text{-}table \ prefix \ prefix\text{-}state$

proof –

have $trace\text{-}eq$:

$query\text{-}prefix\text{-}trace\text{-}conceptual\text{-}table\text{-}with\text{-}default \ prefix \ prefix\text{-}state$
 $trace\text{-}table = trace\text{-}table$

by (rule $query\text{-}prefix\text{-}trace\text{-}conceptual\text{-}table\text{-}with\text{-}default\text{-}eq\text{-}if\text{-}agrees$
 $[OF \ clean \ trace\text{-}len \ trace\text{-}agrees])$

have $composition\text{-}eq$:

$query\text{-}prefix\text{-}composition\text{-}conceptual\text{-}table\text{-}with\text{-}default \ prefix$
 $prefix\text{-}state \ composition\text{-}table = composition\text{-}table$

by (rule
 $query\text{-}prefix\text{-}composition\text{-}conceptual\text{-}table\text{-}with\text{-}default\text{-}eq\text{-}if\text{-}agrees$
 $[OF \ clean \ composition\text{-}len \ composition\text{-}agrees])$

show $?thesis$

unfolding $staged\text{-}query\text{-}prefix\text{-}candidate\text{-}pair\text{-}query\text{-}target\text{-}from\text{-}prefix\text{-}def$
 $staged\text{-}query\text{-}prefix\text{-}conceptual\text{-}default\text{-}query\text{-}target\text{-}def$

using $trace\text{-}eq \ composition\text{-}eq$ **by** $simp$

qed

definition $query\text{-}prefix\text{-}trace\text{-}default\text{-}agrees\text{-}with\text{-}authenticated\text{-}values$

$:: 'f \ list \Rightarrow 'f \ staged\text{-}query\text{-}prefix\text{-}data \Rightarrow$
 $'f \ protocol\text{-}channel \Rightarrow bool$

where

$query\text{-}prefix\text{-}trace\text{-}default\text{-}agrees\text{-}with\text{-}authenticated\text{-}values \ trace\text{-}default$
 $prefix \ prefix\text{-}state \longleftrightarrow$
 $(\forall i \ v. \ i < scale * clength \longrightarrow$
 $authenticated\text{-}value\text{-}at \ prefix\text{-}state \ (sqp\text{-}trace\text{-}root \ prefix)$
 $(scale * clength) \ i \ v \longrightarrow$
 $trace\text{-}default \ ! \ i = v)$

definition $query\text{-}prefix\text{-}composition\text{-}default\text{-}agrees\text{-}with\text{-}authenticated\text{-}values$

$:: 'f \ list \Rightarrow 'f \ staged\text{-}query\text{-}prefix\text{-}data \Rightarrow$
 $'f \ protocol\text{-}channel \Rightarrow bool$

where

$query\text{-}prefix\text{-}composition\text{-}default\text{-}agrees\text{-}with\text{-}authenticated\text{-}values$
 $composition\text{-}default \ prefix \ prefix\text{-}state \longleftrightarrow$
 $(\forall i \ v. \ i < scale * clength \longrightarrow$
 $authenticated\text{-}value\text{-}at \ prefix\text{-}state$
 $(hd \ (sqp\text{-}composition\text{-}fri\text{-}roots \ prefix)) \ (scale * clength) \ i \ v \longrightarrow$
 $composition\text{-}default \ ! \ i = v)$

definition $query\text{-}prefix\text{-}candidate\text{-}defaults\text{-}agree\text{-}with\text{-}authenticated\text{-}values$

$:: 'f \ list \Rightarrow 'f \ list \Rightarrow 'f \ staged\text{-}query\text{-}prefix\text{-}data \Rightarrow$

'f protocol-channel $\Rightarrow$ bool
where
query-prefix-candidate-defaults-agree-with-authenticated-values
trace-default composition-default prefix prefix-state $\longleftrightarrow$
query-prefix-trace-default-agrees-with-authenticated-values trace-default
prefix prefix-state $\wedge$
query-prefix-composition-default-agrees-with-authenticated-values
composition-default prefix prefix-state

lemma *query-prefix-trace-default-agrees-with-authenticated-valuesD:*
assumes
query-prefix-trace-default-agrees-with-authenticated-values trace-default
prefix prefix-state
and *i < scale * clength*
and
authenticated-value-at prefix-state (sqp-trace-root prefix)
*(scale * clength) i v*
shows *trace-default ! i = v*
using *assms*
unfolding *query-prefix-trace-default-agrees-with-authenticated-values-def*
by *blast*

lemma *query-prefix-composition-default-agrees-with-authenticated-valuesD:*
assumes
query-prefix-composition-default-agrees-with-authenticated-values
composition-default prefix prefix-state
and *i < scale * clength*
and
authenticated-value-at prefix-state
*(hd (sqp-composition-fri-roots prefix)) (scale * clength) i v*
shows *composition-default ! i = v*
using *assms*
unfolding
query-prefix-composition-default-agrees-with-authenticated-values-def
by *blast*

lemma
staged-query-prefix-candidate-pair-query-target-from-prefix-subset-conceptual-default-if-prefix-agrees:
assumes *clean: $\neg$ hash-map-output-collision prefix-state*
and *trace-len: length trace-table = scale * clength*
and *composition-len: length composition-table = scale * clength*
and *agrees:*
query-prefix-candidate-defaults-agree-with-authenticated-values
trace-table composition-table prefix prefix-state
shows
staged-query-prefix-candidate-pair-query-target-from-prefix trace-table
composition-table prefix prefix-state $\subseteq$
staged-query-prefix-conceptual-default-query-target trace-table
composition-table prefix prefix-state

```

proof (rule
  staged-query-prefix-candidate-pair-query-target-from-prefix-subset-conceptual-default-if-agrees
  [OF clean trace-len composition-len])
fix i v
assume i-bound: i < scale * clength
and auth:
  authenticated-value-at prefix-state (sqp-trace-root prefix)
  (scale * clength) i v
have trace-agrees:
  query-prefix-trace-default-agrees-with-authenticated-values trace-table
  prefix prefix-state
using agrees
unfolding
  query-prefix-candidate-defaults-agree-with-authenticated-values-def
by simp
show trace-table ! i = v
by (rule
  query-prefix-trace-default-agrees-with-authenticated-valuesD
  [OF trace-agrees i-bound auth])
next
fix i v
assume i-bound: i < scale * clength
and auth:
  authenticated-value-at prefix-state
  (hd (sqp-composition-fri-roots prefix)) (scale * clength) i v
have composition-agrees:
  query-prefix-composition-default-agrees-with-authenticated-values
  composition-table prefix prefix-state
using agrees
unfolding
  query-prefix-candidate-defaults-agree-with-authenticated-values-def
by simp
show composition-table ! i = v
by (rule
  query-prefix-composition-default-agrees-with-authenticated-valuesD
  [OF composition-agrees i-bound auth])
qed

```

definition query-prefix-candidate-defaults-agree-at-query

:: 'f list $\Rightarrow$ 'f list $\Rightarrow$ nat $\Rightarrow$

'f staged-query-prefix-data $\Rightarrow$ 'f protocol-channel $\Rightarrow$ bool

where

query-prefix-candidate-defaults-agree-at-query trace-table

composition-table idx prefix prefix-state $\longleftrightarrow$

($\forall j \in \text{set } (\text{powers-scaled } \text{idx}).$

query-prefix-trace-conceptual-table-with-default prefix prefix-state

trace-table ! j = trace-table ! j) $\wedge$

query-prefix-composition-conceptual-table-with-default prefix prefix-state

composition-table ! idx = composition-table ! idx

lemma *query-prefix-candidate-pair-query-hit-imp-conceptual-default-if-agrees-at-query:*

assumes *hit:*

idx ∈

staged-query-prefix-candidate-pair-query-target-from-prefix trace-table

composition-table prefix prefix-state

and *trace-len:* *length trace-table = scale * clength*

and *composition-len:* *length composition-table = scale * clength*

and *agrees:*

query-prefix-candidate-defaults-agree-at-query trace-table

composition-table idx prefix prefix-state

and *default-trace-low:*

trace-table-low-degree

(query-prefix-trace-conceptual-table-with-default prefix prefix-state trace-table)

and *default-composition-low:*

composition-table-low-degree maxDegree

(query-prefix-composition-conceptual-table-with-default prefix prefix-state composition-table)

and *default-not-all:*

$\neg$ *all-queries-consistent*

(query-prefix-trace-conceptual-table-with-default prefix prefix-state trace-table)

(query-prefix-composition-conceptual-table-with-default prefix prefix-state composition-table)

(sqp-alphas prefix)

shows

idx ∈

staged-query-prefix-conceptual-default-query-target trace-table

composition-table prefix prefix-state

proof —

let *?trace-default =*

query-prefix-trace-conceptual-table-with-default prefix prefix-state trace-table

let *?composition-default =*

query-prefix-composition-conceptual-table-with-default prefix prefix-state composition-table

have *trace-len-eq:* *length trace-table = length ?trace-default*

using *trace-len* **by** *simp*

have *composition-len-eq:*

length composition-table = length ?composition-default

using *composition-len* **by** *simp*

have *trace-values:*

map (!) trace-table) (powers-scaled idx) =

map (!) ?trace-default) (powers-scaled idx)

proof (*rule nth-equalityI*)

show

length (map (!) trace-table) (powers-scaled idx) =

length (map (!) ?trace-default) (powers-scaled idx))


```

    by simp
next
fix k
assume k-bound:
  k < length (map (!) trace-table) (powers-scaled idx)
then have ps-bound: k < length (powers-scaled idx)
  by simp
have ps-mem: powers-scaled idx ! k ∈ set (powers-scaled idx)
  by (rule nth-mem[OF ps-bound])
have eq:
  ?trace-default ! (powers-scaled idx ! k) =
    trace-table ! (powers-scaled idx ! k)
  using agrees ps-mem
  unfolding query-prefix-candidate-defaults-agree-at-query-def
  by blast
show
  map (!) trace-table (powers-scaled idx) ! k =
    map (!) ?trace-default (powers-scaled idx) ! k
  using ps-bound eq by simp
qed
have composition-value:
  composition-table ! idx = ?composition-default ! idx
  using agrees
  unfolding query-prefix-candidate-defaults-agree-at-query-def
  by simp
have qc-iff:
  query-consistent-at trace-table composition-table (sqp-alphas prefix)
    idx ⟷
    query-consistent-at ?trace-default ?composition-default
      (sqp-alphas prefix) idx
  by (rule query-consistent-at-cong-opened-values
    [OF trace-len-eq composition-len-eq trace-values composition-value])
have qc:
  query-consistent-at trace-table composition-table (sqp-alphas prefix)
    idx
  using hit
  unfolding staged-query-prefix-candidate-pair-query-target-from-prefix-def
    query-sampling-success-space-def query-agreement-indices-def
  by (cases
    trace-table-low-degree trace-table ∧
    composition-table-low-degree maxDegree composition-table ∧
    ¬ all-queries-consistent trace-table composition-table
      (sqp-alphas prefix))
  auto
have query-idx: idx ∈ query-sample-space
  using hit query-sampling-success-space-subset
  unfolding staged-query-prefix-candidate-pair-query-target-from-prefix-def
  by blast
have default-qc:

```

```

  query-consistent-at ?trace-default ?composition-default
    (sqp-alphas prefix) idx
  using qc qc-iff by simp
show ?thesis
  using query-idx default-qc default-trace-low default-composition-low
    default-not-all
  unfolding staged-query-prefix-candidate-pair-query-target-from-prefix-def
    staged-query-prefix-conceptual-default-query-target-def
    query-sampling-success-space-def query-agreement-indices-def
  by auto
qed

lemma query-prefix-candidate-defaults-agree-at-query-from-prefix-authenticated-openings:
  assumes clean:  $\neg$  hash-map-output-collision prefix-state
  and i-bound:  $i < \text{rounds}$ 
  and trace-prefix:
    partial-authenticated-table (sqp-trace-root prefix)
      (scale * clength) trace-openings prefix-state
  and composition-prefix:
    partial-authenticated-table (hd (sqp-composition-fri-roots prefix))
      (scale * clength) composition-openings prefix-state
  and consistent:
    partial-query-openings-consistent trace-openings composition-openings
      (sqp-alphas prefix) idx
  and trace-candidate:
    partial-trace-table-candidate trace-table
      ((replicate rounds []) [i := trace-openings])
  and composition-candidate:
    partial-composition-table-candidate composition-table
      ((replicate rounds []) [i := composition-openings])
  shows
    query-prefix-candidate-defaults-agree-at-query trace-table
      composition-table idx prefix prefix-state
proof -
  let ?trace-openingsss = (replicate rounds []) [i := trace-openings]
  let ?composition-openingsss =
    (replicate rounds []) [i := composition-openings]
  have trace-agrees:
    table-agrees-with-authenticated-openings trace-table
      (scale * clength) trace-openings
proof -
  have all:
     $\forall ia < \text{rounds}.$ 
    table-agrees-with-authenticated-openings trace-table
      (scale * clength) (?trace-openingsss ! ia)
  using trace-candidate
  unfolding partial-trace-table-candidate-def by blast
  have at-i:
    table-agrees-with-authenticated-openings trace-table

```

```

    (scale * clength) (?trace-openings ! i)
  using all i-bound by blast
have ?trace-openings ! i = trace-openings
  using i-bound by simp
then show ?thesis
  using at-i by simp
qed
have composition-agrees:
  table-agrees-with-authenticated-openings composition-table
    (scale * clength) composition-openings
proof -
  have all:
     $\forall ia < rounds.$ 
    table-agrees-with-authenticated-openings composition-table
      (scale * clength) (?composition-openings ! ia)
  using composition-candidate
  unfolding partial-composition-table-candidate-def by blast
have at-i:
  table-agrees-with-authenticated-openings composition-table
    (scale * clength) (?composition-openings ! i)
  using all i-bound by blast
have ?composition-openings ! i = composition-openings
  using i-bound by simp
then show ?thesis
  using at-i by simp
qed
have trace-indices:
  map opening-index trace-openings = powers-scaled idx
  using consistent unfolding partial-query-openings-consistent-def by simp
have composition-indices:
  map opening-index composition-openings =
    [idx, fri-sibling-index (scale * clength) idx]
  using consistent unfolding partial-query-openings-consistent-def by simp
have composition-nonempty: composition-openings  $\neq$  []
  using consistent unfolding partial-query-openings-consistent-def by simp
have trace-point:
   $\bigwedge j. j \in \text{set (powers-scaled idx)} \implies$ 
  query-prefix-trace-conceptual-table-with-default prefix prefix-state
    trace-table ! j =
  trace-table ! j
proof -
  fix j
  assume j-in:  $j \in \text{set (powers-scaled idx)}$ 
  then obtain opn where opn-in:  $opn \in \text{set trace-openings}$ 
    and opn-idx: opening-index opn = j
  using trace-indices
  by (metis (mono-tags, lifting) in-set-conv-nth length-map nth-map)
have conceptual:
  query-prefix-trace-conceptual-table-with-default prefix prefix-state

```

```

    trace-table ! opening-index opn =
    opening-value opn
  by (rule
    query-prefix-trace-conceptual-table-with-default-agrees-with-partial-authenticated-table
    [OF clean trace-prefix opn-in])
  have table:
    trace-table ! opening-index opn = opening-value opn
    using trace-agrees opn-in
    unfolding table-agrees-with-authenticated-openings-def by blast
  show
    query-prefix-trace-conceptual-table-with-default prefix prefix-state
    trace-table ! j =
    trace-table ! j
    using conceptual table opn-idx by simp
qed
have composition-point:
  query-prefix-composition-conceptual-table-with-default prefix
  prefix-state composition-table ! idx =
  composition-table ! idx
proof -
  have opn0-in: composition-openings ! 0 ∈ set composition-openings
    using composition-nonempty by simp
  have opn0-idx: opening-index (composition-openings ! 0) = idx
    using composition-indices composition-nonempty
    by (cases composition-openings) simp-all
  have conceptual:
    query-prefix-composition-conceptual-table-with-default prefix
    prefix-state composition-table ! opening-index (composition-openings ! 0) =
    opening-value (composition-openings ! 0)
    by (rule
      query-prefix-composition-conceptual-table-with-default-agrees-with-partial-authenticated-table
      [OF clean composition-prefix opn0-in])
  have table:
    composition-table ! opening-index (composition-openings ! 0) =
    opening-value (composition-openings ! 0)
    using composition-agrees opn0-in
    unfolding table-agrees-with-authenticated-openings-def by blast
  show ?thesis
    using conceptual table opn0-idx by simp
qed
show ?thesis
  unfolding query-prefix-candidate-defaults-agree-at-query-def
  using trace-point composition-point by blast
qed

lemma query-prefix-current-openings-imp-conceptual-default-query-target:
  assumes clean: ¬ hash-map-output-collision prefix-state
  and i-bound: i < rounds
  and trace-prefix:

```

```

    partial-authenticated-table (sqp-trace-root prefix)
      (scale * clength) trace-openings prefix-state
and composition-prefix:
    partial-authenticated-table (hd (sqp-composition-fri-roots prefix))
      (scale * clength) composition-openings prefix-state
and consistent:
    partial-query-openings-consistent trace-openings composition-openings
      (sqp-alphas prefix) idx
and trace-candidate:
    partial-trace-table-candidate trace-table
      ((replicate rounds []) [i := trace-openings])
and composition-candidate:
    partial-composition-table-candidate composition-table
      ((replicate rounds []) [i := composition-openings])
and default-trace-low:
    trace-table-low-degree
      (query-prefix-trace-conceptual-table-with-default prefix prefix-state
        trace-table)
and default-composition-low:
    composition-table-low-degree maxDegree
      (query-prefix-composition-conceptual-table-with-default prefix
        prefix-state composition-table)
and default-not-all:
    ¬ all-queries-consistent
      (query-prefix-trace-conceptual-table-with-default prefix prefix-state
        trace-table)
      (query-prefix-composition-conceptual-table-with-default prefix
        prefix-state composition-table)
      (sqp-alphas prefix)
shows
    idx ∈
      staged-query-prefix-conceptual-default-query-target trace-table
        composition-table prefix prefix-state
proof –
let ?trace-openingsss = (replicate rounds []) [i := trace-openings]
let ?composition-openingsss =
  (replicate rounds []) [i := composition-openings]
let ?trace-default =
  query-prefix-trace-conceptual-table-with-default prefix prefix-state
    trace-table
let ?composition-default =
  query-prefix-composition-conceptual-table-with-default prefix
    prefix-state composition-table
have trace-len: length trace-table = scale * clength
  using trace-candidate unfolding partial-trace-table-candidate-def
  by simp
have composition-len: length composition-table = scale * clength
  using composition-candidate
  unfolding partial-composition-table-candidate-def by simp

```

```

have idx-sample: idx ∈ query-sample-space
  using consistent unfolding partial-query-openings-consistent-def by simp
have round:
  partial-query-round-consistent ?trace-openingsss ?composition-openingsss
    (sqp-alphas prefix) i idx
  using i-bound consistent
  unfolding partial-query-round-consistent-def
    partial-query-openings-consistent-def
  by simp
have candidate-consistent:
  query-consistent-at trace-table composition-table (sqp-alphas prefix)
    idx
  by (rule partial-query-round-consistent-imp-query-consistent-at
    [OF trace-candidate composition-candidate round])
have agrees:
  query-prefix-candidate-defaults-agree-at-query trace-table
    composition-table idx prefix prefix-state
  by (rule
    query-prefix-candidate-defaults-agree-at-query-from-prefix-authenticated-openings
    [OF clean i-bound trace-prefix composition-prefix consistent
      trace-candidate composition-candidate])
have trace-len-eq: length trace-table = length ?trace-default
  using trace-len by simp
have composition-len-eq:
  length composition-table = length ?composition-default
  using composition-len by simp
have trace-values-eq:
  map (! trace-table) (powers-scaled idx) =
    map (! ?trace-default) (powers-scaled idx)
proof (rule nth-equalityI)
  show
    length (map (! trace-table) (powers-scaled idx)) =
      length (map (! ?trace-default) (powers-scaled idx))
    by simp
next
  fix k
  assume k-bound:
    k < length (map (! trace-table) (powers-scaled idx))
  then have ps-bound: k < length (powers-scaled idx)
    by simp
  have ps-mem: powers-scaled idx ! k ∈ set (powers-scaled idx)
    by (rule nth-mem[OF ps-bound])
  have eq:
    ?trace-default ! (powers-scaled idx ! k) =
      trace-table ! (powers-scaled idx ! k)
    using agrees ps-mem
    unfolding query-prefix-candidate-defaults-agree-at-query-def
    by blast
  show

```

```

    map (!) trace-table) (powers-scaled idx) ! k =
    map (!) ?trace-default) (powers-scaled idx) ! k
    using ps-bound eq by simp
qed
have composition-value-eq:
  composition-table ! idx = ?composition-default ! idx
  using agrees
  unfolding query-prefix-candidate-defaults-agree-at-query-def by simp
have qc-iff:
  query-consistent-at trace-table composition-table (sqp-alphas prefix)
  idx  $\longleftrightarrow$ 
  query-consistent-at ?trace-default ?composition-default
  (sqp-alphas prefix) idx
  by (rule query-consistent-at-cong-opened-values
    [OF trace-len-eq composition-len-eq trace-values-eq
      composition-value-eq])
have default-consistent:
  query-consistent-at ?trace-default ?composition-default
  (sqp-alphas prefix) idx
  using candidate-consistent qc-iff by simp
show ?thesis
  unfolding staged-query-prefix-conceptual-default-query-target-def
  query-sampling-success-space-def query-agreement-indices-def
  using idx-sample default-consistent default-trace-low
  default-composition-low default-not-all by simp
qed

lemma trace-table-agrees-with-prefix-authenticated-values-if-final-merkle-bound:
  assumes ext: prefix-state  $\leq$  final-state
  and clean:  $\neg$  hash-map-output-collision final-state
  and bind:
    merkle-root-binds-table (sqp-trace-root prefix) trace-table
    final-state
  and len: length trace-table = scale * clength
  and i-bound: i < scale * clength
  and auth:
    authenticated-value-at prefix-state (sqp-trace-root prefix)
    (scale * clength) i v
  shows trace-table ! i = v
proof -
  from authenticated-value-atE[OF auth] obtain opn where
    opn-auth-prefix: authenticated-opening-in prefix-state opn
  and root: opening-root opn = sqp-trace-root prefix
  and opn-len: opening-length opn = scale * clength
  and idx: opening-index opn = i
  and val: opening-value opn = v
  by blast
  have opn-auth-final: authenticated-opening-in final-state opn
  by (rule authenticated-opening-in-mono[OF opn-auth-prefix ext])

```

obtain n **where** len-pow : $\text{length trace-table} = 2 \wedge n$
using $\text{len eval-domain-length-power}$ **by** ($\text{metis mult.commute}$)
have agrees :
 $\text{opening-index } \text{opn} < \text{length trace-table} \wedge$
 $\text{trace-table} ! \text{opening-index } \text{opn} = \text{opening-value } \text{opn}$
by ($\text{rule merkle-bound-table-agrees-with-authenticated-opening-if-clean}$
 $[\text{OF bind len-pow opn-auth-final root - clean}]$)
 $(\text{use len opn-len in simp})$
then show $?thesis$
using idx val **by** simp
qed

lemma $\text{composition-table-agrees-with-prefix-authenticated-values-if-final-merkle-bound}$:
assumes ext : $\text{prefix-state} \leq \text{final-state}$
and clean : $\neg \text{hash-map-output-collision final-state}$
and bind :
 $\text{merkle-root-binds-table} (\text{hd} (\text{sqp-composition-fri-roots prefix}))$
 $\text{composition-table final-state}$
and len : $\text{length composition-table} = \text{scale} * \text{clength}$
and i-bound : $i < \text{scale} * \text{clength}$
and auth :
 $\text{authenticated-value-at prefix-state}$
 $(\text{hd} (\text{sqp-composition-fri-roots prefix})) (\text{scale} * \text{clength}) i v$
shows $\text{composition-table} ! i = v$
proof –
from $\text{authenticated-value-atE}[\text{OF auth}]$ **obtain** opn **where**
 opn-auth-prefix : $\text{authenticated-opening-in prefix-state opn}$
and root :
 $\text{opening-root } \text{opn} = \text{hd} (\text{sqp-composition-fri-roots prefix})$
and opn-len : $\text{opening-length } \text{opn} = \text{scale} * \text{clength}$
and idx : $\text{opening-index } \text{opn} = i$
and val : $\text{opening-value } \text{opn} = v$
by blast
have opn-auth-final : $\text{authenticated-opening-in final-state opn}$
by ($\text{rule authenticated-opening-in-mono}[\text{OF opn-auth-prefix ext}]$)
obtain n **where** len-pow : $\text{length composition-table} = 2 \wedge n$
using $\text{len eval-domain-length-power}$ **by** ($\text{metis mult.commute}$)
have agrees :
 $\text{opening-index } \text{opn} < \text{length composition-table} \wedge$
 $\text{composition-table} ! \text{opening-index } \text{opn} = \text{opening-value } \text{opn}$
by ($\text{rule merkle-bound-table-agrees-with-authenticated-opening-if-clean}$
 $[\text{OF bind len-pow opn-auth-final root - clean}]$)
 $(\text{use len opn-len in simp})$
then show $?thesis$
using idx val **by** simp
qed

lemma
 $\text{staged-query-prefix-candidate-pair-query-target-from-prefix-subset-conceptual-default-if-final-merkle-bound}$:


```

assumes prefix-clean:  $\neg$  hash-map-output-collision prefix-state
and final-clean:  $\neg$  hash-map-output-collision final-state
and ext: prefix-state  $\leq$  final-state
and trace-bind:
  merkle-root-binds-table (sqp-trace-root prefix) trace-table
  final-state
and composition-bind:
  merkle-root-binds-table (hd (sqp-composition-fri-roots prefix))
  composition-table final-state
and trace-len: length trace-table = scale * clength
and composition-len: length composition-table = scale * clength
shows
  staged-query-prefix-candidate-pair-query-target-from-prefix trace-table
  composition-table prefix prefix-state  $\subseteq$ 
  staged-query-prefix-conceptual-default-query-target trace-table
  composition-table prefix prefix-state
proof (rule
  staged-query-prefix-candidate-pair-query-target-from-prefix-subset-conceptual-default-if-agrees
  [OF prefix-clean trace-len composition-len])
fix i v
assume i-bound: i < scale * clength
and auth:
  authenticated-value-at prefix-state (sqp-trace-root prefix)
  (scale * clength) i v
show trace-table ! i = v
by (rule
  trace-table-agrees-with-prefix-authenticated-values-if-final-merkle-bound
  [OF ext final-clean trace-bind trace-len i-bound auth])
next
fix i v
assume i-bound: i < scale * clength
and auth:
  authenticated-value-at prefix-state
  (hd (sqp-composition-fri-roots prefix)) (scale * clength) i v
show composition-table ! i = v
by (rule
  composition-table-agrees-with-prefix-authenticated-values-if-final-merkle-bound
  [OF ext final-clean composition-bind composition-len i-bound auth])
qed

lemma
staged-query-prefix-candidate-opening-query-target-from-prefix-subset-conceptual:
assumes clean:  $\neg$  hash-map-output-collision prefix-state
and trace-len: length trace-openings = rounds
and composition-len: length composition-openings = rounds
and trace-tables:
   $\bigwedge j. j < \text{rounds} \implies$ 
  partial-authenticated-table (sqp-trace-root prefix)
  (scale * clength) (trace-openings ! j) prefix-state

```

and *composition-tables*:
 $\bigwedge j. j < \text{rounds} \implies$
 $\text{partial-authenticated-table } (\text{hd } (\text{sqp-composition-fri-roots } \text{prefix}))$
 $(\text{scale} * \text{clength}) (\text{composition-openings } ! j) \text{ prefix-state}$
and *trace-low*:
 $\text{trace-table-low-degree}$
 $(\text{query-prefix-trace-conceptual-table } \text{prefix } \text{prefix-state})$
and *composition-low*:
 $\text{composition-table-low-degree } \text{maxDegree}$
 $(\text{query-prefix-composition-conceptual-table } \text{prefix } \text{prefix-state})$
and *not-all*:
 $\neg \text{all-queries-consistent}$
 $(\text{query-prefix-trace-conceptual-table } \text{prefix } \text{prefix-state})$
 $(\text{query-prefix-composition-conceptual-table } \text{prefix } \text{prefix-state})$
 $(\text{sqp-alphas } \text{prefix})$
shows
 $\text{staged-query-prefix-candidate-opening-query-target-from-prefix}$
 $\text{trace-openings } \text{composition-openings } i \text{ prefix } \text{prefix-state} \subseteq$
 $\text{staged-query-prefix-conceptual-query-target } \text{prefix } \text{prefix-state}$
proof –
have *trace-candidate*:
 $\text{partial-trace-table-candidate}$
 $(\text{query-prefix-trace-conceptual-table } \text{prefix } \text{prefix-state})$
 trace-openings
by (rule *query-prefix-trace-conceptual-table-partial-trace-candidate*
 $[\text{OF clean trace-len trace-tables}]$)
have *composition-candidate*:
 $\text{partial-composition-table-candidate}$
 $(\text{query-prefix-composition-conceptual-table } \text{prefix } \text{prefix-state})$
 $\text{composition-openings}$
by (rule
 $\text{query-prefix-composition-conceptual-table-partial-composition-candidate}$
 $[\text{OF clean composition-len composition-tables}]$)
show *?thesis*
unfolding *staged-query-prefix-candidate-opening-query-target-from-prefix-def*
 $\text{staged-query-prefix-conceptual-query-target-def}$
by (rule *partial-query-success-indices-at-subset-query-sampling-success-space*
 $[\text{OF trace-candidate composition-candidate trace-low composition-low}$
 $\text{not-all}]$)
qed

lemma
 $\text{staged-query-prefix-candidate-opening-query-target-from-prefix-subset-conceptual-default}$:
assumes *clean*: $\neg \text{hash-map-output-collision } \text{prefix-state}$
and *trace-len*: $\text{length } \text{trace-openings} = \text{rounds}$
and *composition-len*: $\text{length } \text{composition-openings} = \text{rounds}$
and *trace-tables*:
 $\bigwedge j. j < \text{rounds} \implies$
 $\text{partial-authenticated-table } (\text{sqp-trace-root } \text{prefix})$

```

    (scale * clength) (trace-openings ! j) prefix-state
and composition-tables:
   $\bigwedge j. j < \text{rounds} \implies$ 
    partial-authenticated-table (hd (sqp-composition-fri-roots prefix))
    (scale * clength) (composition-openings ! j) prefix-state
and trace-low:
  trace-table-low-degree
    (query-prefix-trace-conceptual-table-with-default prefix
      prefix-state trace-default)
and composition-low:
  composition-table-low-degree maxDegree
    (query-prefix-composition-conceptual-table-with-default prefix
      prefix-state composition-default)
and not-all:
   $\neg$  all-queries-consistent
    (query-prefix-trace-conceptual-table-with-default prefix
      prefix-state trace-default)
    (query-prefix-composition-conceptual-table-with-default prefix
      prefix-state composition-default)
    (sqp-alphas prefix)
shows
  staged-query-prefix-candidate-opening-query-target-from-prefix
  trace-openings composition-openings i prefix prefix-state  $\subseteq$ 
  staged-query-prefix-conceptual-default-query-target trace-default
  composition-default prefix prefix-state
proof —
have trace-candidate:
  partial-trace-table-candidate
    (query-prefix-trace-conceptual-table-with-default prefix prefix-state
      trace-default)
  trace-openings
by (rule
  query-prefix-trace-conceptual-table-with-default-partial-trace-candidate
  [OF clean trace-len trace-tables])
have composition-candidate:
  partial-composition-table-candidate
    (query-prefix-composition-conceptual-table-with-default prefix
      prefix-state composition-default)
  composition-openings
by (rule
  query-prefix-composition-conceptual-table-with-default-partial-composition-candidate
  [OF clean composition-len composition-tables])
show ?thesis
unfolding staged-query-prefix-candidate-opening-query-target-from-prefix-def
  staged-query-prefix-conceptual-default-query-target-def
by (rule partial-query-success-indices-at-subset-query-sampling-success-space
  [OF trace-candidate composition-candidate trace-low composition-low
  not-all])
qed

```

lemma *checked-staged-query-prefix-conceptual-query-target-hit-bound:*
fixes $C :: \text{prob}$
assumes *raw-bound: query-index-raw-preimage-bound*
and *prefix-bound:*
 $\bigwedge \text{prefix prefix-state.}$
 $\text{Some } ((\text{prefix}, \text{prefix-state}), \text{prefix-state}) \in$
 set-dist
 $(\text{execute } (\text{checked-staged-query-prefix-with-state } A \ i) \ s) \implies$
 $\text{query-future-fresh prefix-state}$
shows
 $\text{wp-event } (\text{checked-staged-query-prefix-receive-with-state } A \ i)$
 $(\text{checked-staged-query-prefix-dynamic-index-hit}$
 $\text{staged-query-prefix-conceptual-query-target}) \ s \leq$
 query-error-bound
proof (rule *checked-staged-query-prefix-dynamic-index-hit-bound*[OF *raw-bound*])
fix $\text{prefix prefix-state}$
assume *support:*
 $\text{Some } ((\text{prefix}, \text{prefix-state}), \text{prefix-state}) \in$
 set-dist
 $(\text{execute } (\text{checked-staged-query-prefix-with-state } A \ i) \ s)$
have *future: query-future-fresh prefix-state*
by (rule *prefix-bound*[OF *support*])
show
 $\text{query-future-fresh prefix-state} \wedge$
 $\text{staged-query-prefix-conceptual-query-target prefix prefix-state}$
 $\subseteq \text{query-sample-space} \wedge$
 nnreal
 $(\text{card}$
 $(\text{staged-query-prefix-conceptual-query-target prefix prefix-state})) /$
 $\text{nnreal } (\text{card } \text{query-sample-space}) \leq \text{query-error-bound}$
using *future*
 $\text{staged-query-prefix-conceptual-query-target-subset[of prefix prefix-state]}$
 $\text{staged-query-prefix-conceptual-query-target-fraction-bound}$
by *blast*
qed

lemma *checked-staged-query-prefix-conceptual-query-target-hit-bound-from-sampler:*
assumes *prefix-bound:*
 $\bigwedge \text{prefix prefix-state.}$
 $\text{Some } ((\text{prefix}, \text{prefix-state}), \text{prefix-state}) \in$
 set-dist
 $(\text{execute } (\text{checked-staged-query-prefix-with-state } A \ i) \ s) \implies$
 $\text{query-future-fresh prefix-state}$
shows
 $\text{wp-event } (\text{checked-staged-query-prefix-receive-with-state } A \ i)$
 $(\text{checked-staged-query-prefix-dynamic-index-hit}$
 $\text{staged-query-prefix-conceptual-query-target}) \ s \leq$
 query-error-bound

by (rule checked-staged-query-prefix-conceptual-query-target-hit-bound
 [OF query-index-raw-preimage-bound-from-sampler-wellformed
 prefix-bound])

lemma checked-staged-query-prefix-conceptual-default-query-target-hit-bound:

fixes $C :: \text{prob}$

assumes raw-bound: query-index-raw-preimage-bound

and prefix-bound:

$\bigwedge \text{prefix prefix-state.}$

Some ((prefix, prefix-state), prefix-state) $\in$
 set-dist

(execute (checked-staged-query-prefix-with-state A i) s) $\implies$
 query-future-fresh prefix-state

shows

wp-event (checked-staged-query-prefix-receive-with-state A i)

(checked-staged-query-prefix-dynamic-index-hit

(staged-query-prefix-conceptual-default-query-target trace-default
 composition-default)) $s \leq$

query-error-bound

proof (rule checked-staged-query-prefix-dynamic-index-hit-bound[OF raw-bound])

fix prefix prefix-state

assume support:

Some ((prefix, prefix-state), prefix-state) $\in$
 set-dist

(execute (checked-staged-query-prefix-with-state A i) s)

have future: query-future-fresh prefix-state

by (rule prefix-bound[OF support])

show

query-future-fresh prefix-state $\wedge$

staged-query-prefix-conceptual-default-query-target trace-default
 composition-default prefix prefix-state $\subseteq$ query-sample-space $\wedge$

nnreal

(card

(staged-query-prefix-conceptual-default-query-target trace-default
 composition-default prefix prefix-state)) /

nnreal (card query-sample-space) $\leq$ query-error-bound

using future

staged-query-prefix-conceptual-default-query-target-subset

staged-query-prefix-conceptual-default-query-target-fraction-bound

by blast

qed

lemma checked-staged-query-prefix-conceptual-default-query-target-hit-bound-from-sampler:

assumes prefix-bound:

$\bigwedge \text{prefix prefix-state.}$

Some ((prefix, prefix-state), prefix-state) $\in$
 set-dist

(execute (checked-staged-query-prefix-with-state A i) s) $\implies$
 query-future-fresh prefix-state

shows

wp-event (checked-staged-query-prefix-receive-with-state A i)
(checked-staged-query-prefix-dynamic-index-hit
(staged-query-prefix-conceptual-default-query-target trace-default
composition-default)) s ≤
query-error-bound

by *(rule*

checked-staged-query-prefix-conceptual-default-query-target-hit-bound
[OF query-index-raw-preimage-bound-from-sampler-wellformed
prefix-bound])

lemma *checked-staged-query-prefix-conceptual-empty-query-target-hit-bound:*

fixes *C :: prob*

assumes *raw-bound: query-index-raw-preimage-bound*

and *prefix-bound:*

∧ prefix prefix-state.

Some ((prefix, prefix-state), prefix-state) ∈
set-dist

(execute (checked-staged-query-prefix-with-state A i) s) ⇒
query-future-fresh prefix-state

shows

wp-event (checked-staged-query-prefix-receive-with-state A i)
(checked-staged-query-prefix-dynamic-index-hit
staged-query-prefix-conceptual-empty-query-target) s ≤
query-error-bound

proof *(rule checked-staged-query-prefix-dynamic-index-hit-bound[OF raw-bound])*

fix *prefix prefix-state*

assume *support:*

Some ((prefix, prefix-state), prefix-state) ∈
set-dist

(execute (checked-staged-query-prefix-with-state A i) s)

have *future: query-future-fresh prefix-state*

by *(rule prefix-bound[OF support])*

show

query-future-fresh prefix-state ∧
staged-query-prefix-conceptual-empty-query-target prefix prefix-state
⊆ query-sample-space ∧
nnreal

(card
(staged-query-prefix-conceptual-empty-query-target prefix
prefix-state)) /

nnreal (card query-sample-space) ≤ query-error-bound

using *future*

staged-query-prefix-conceptual-empty-query-target-subset

staged-query-prefix-conceptual-empty-query-target-fraction-bound

by *blast*

qed

lemma *checked-staged-query-prefix-conceptual-empty-query-target-hit-bound-from-sampler:*

assumes *prefix-bound*:

$\bigwedge \text{prefix prefix-state.}$

$\text{Some } ((\text{prefix}, \text{prefix-state}), \text{prefix-state}) \in$
 set-dist

$(\text{execute } (\text{checked-staged-query-prefix-with-state } A \ i) \ s) \implies$
 $\text{query-future-fresh prefix-state}$

shows

$\text{wp-event } (\text{checked-staged-query-prefix-receive-with-state } A \ i)$
 $(\text{checked-staged-query-prefix-dynamic-index-hit}$
 $\text{staged-query-prefix-conceptual-empty-query-target}) \ s \leq$
 query-error-bound

by (rule *checked-staged-query-prefix-conceptual-empty-query-target-hit-bound*
[*OF query-index-raw-preimage-bound-from-sampler-wellformed*
prefix-bound])

lemma *checked-staged-query-prefix-conceptual-query-target-prehit-bound*:

assumes *wf*: *staged-budget-wellformed budgets*

and *controlled*: *staged-adversary-controlled budgets A*

and *i-bound*: $i \leq \text{rounds}$

shows

$\text{wp-event } (\text{checked-staged-query-prefix-receive-with-state } A \ i)$
 $(\text{checked-staged-query-prefix-dynamic-index-prehit}$
 $\text{staged-query-prefix-conceptual-query-target})$
 $\text{adversary-initial-state} \leq$
 $\text{staged-phase-relation-error size}$
 $(\text{staged-query-search-queries budgets } i + 1)$

proof (rule *checked-staged-query-prefix-dynamic-index-prehit-bound-from-relation-program*)

show *HashMap adversary-initial-state = fmempty*

by (*simp add: adversary-initial-state-def*)

show *hash-relation-program*

$(\text{checked-staged-query-prefix-dynamic-index-prehit-relation } A \ i$
 $\text{adversary-initial-state staged-query-prefix-conceptual-query-target})$
 $\text{size } (\text{staged-query-search-queries budgets } i + 1)$
 $(\text{checked-staged-query-prefix-receive-with-state } A \ i)$

by (rule *hash-relation-program-checked-staged-query-prefix-receive-with-state*
[*OF wf controlled i-bound*])
(rule
checked-staged-query-prefix-dynamic-index-prehit-relation-fiber-bound-size)

qed

lemma *checked-staged-query-prefix-conceptual-default-query-target-prehit-bound*:

assumes *wf*: *staged-budget-wellformed budgets*

and *controlled*: *staged-adversary-controlled budgets A*

and *i-bound*: $i \leq \text{rounds}$

shows

$\text{wp-event } (\text{checked-staged-query-prefix-receive-with-state } A \ i)$
 $(\text{checked-staged-query-prefix-dynamic-index-prehit}$
 $(\text{staged-query-prefix-conceptual-default-query-target trace-default}$
 $\text{composition-default}))$

$\text{adversary-initial-state} \leq$
 $\text{staged-phase-relation-error size}$
 $(\text{staged-query-search-queries budgets } i + 1)$
proof (rule checked-staged-query-prefix-dynamic-index-prehit-bound-from-relation-program)
show $\text{HashMap adversary-initial-state} = \text{fmempty}$
by (simp add: adversary-initial-state-def)
show hash-relation-program
 $(\text{checked-staged-query-prefix-dynamic-index-prehit-relation } A \ i$
 $\text{adversary-initial-state}$
 $(\text{staged-query-prefix-conceptual-default-query-target trace-default}$
 $\text{composition-default}))$
 $\text{size } (\text{staged-query-search-queries budgets } i + 1)$
 $(\text{checked-staged-query-prefix-receive-with-state } A \ i)$
by (rule hash-relation-program-checked-staged-query-prefix-receive-with-state
 $[\text{OF wf controlled } i\text{-bound}]$)
(rule
 $\text{checked-staged-query-prefix-dynamic-index-prehit-relation-fiber-bound-size})$
qed

lemma checked-staged-query-prefix-conceptual-empty-query-target-prehit-bound:
assumes wf: staged-budget-wellformed budgets
and controlled: staged-adversary-controlled budgets A
and $i\text{-bound}$: $i \leq \text{rounds}$
shows
 $\text{wp-event } (\text{checked-staged-query-prefix-receive-with-state } A \ i)$
 $(\text{checked-staged-query-prefix-dynamic-index-prehit}$
 $\text{staged-query-prefix-conceptual-empty-query-target})$
 $\text{adversary-initial-state} \leq$
 $\text{staged-phase-relation-error size}$
 $(\text{staged-query-search-queries budgets } i + 1)$
proof (rule checked-staged-query-prefix-dynamic-index-prehit-bound-from-relation-program)
show $\text{HashMap adversary-initial-state} = \text{fmempty}$
by (simp add: adversary-initial-state-def)
show hash-relation-program
 $(\text{checked-staged-query-prefix-dynamic-index-prehit-relation } A \ i$
 $\text{adversary-initial-state}$
 $\text{staged-query-prefix-conceptual-empty-query-target})$
 $\text{size } (\text{staged-query-search-queries budgets } i + 1)$
 $(\text{checked-staged-query-prefix-receive-with-state } A \ i)$
by (rule hash-relation-program-checked-staged-query-prefix-receive-with-state
 $[\text{OF wf controlled } i\text{-bound}]$)
(rule
 $\text{checked-staged-query-prefix-dynamic-index-prehit-relation-fiber-bound-size})$
qed

lemma checked-staged-query-prefix-conceptual-query-target-hit-bound-from-budgets:
assumes wf: staged-budget-wellformed budgets
and controlled: staged-adversary-controlled budgets A
and $i\text{-bound}$: $i < \text{rounds}$

shows

wp-event (*checked-staged-query-prefix-receive-with-state A i*)
 (*checked-staged-query-prefix-dynamic-index-hit*
staged-query-prefix-conceptual-query-target)
adversary-initial-state $\leq$
staged-phase-relation-error size
 (*staged-query-search-queries budgets i + 1*) + *query-error-bound*

proof –

have *raw-bound: query-index-raw-preimage-bound*
by (*rule query-index-raw-preimage-bound-from-sampler-wellformed*)

have *hit-bound:*

wp-event (*checked-staged-query-prefix-receive-with-state A i*)
 (*checked-staged-query-prefix-dynamic-index-hit*
staged-query-prefix-conceptual-query-target)
adversary-initial-state $\leq$
wp-event (*checked-staged-query-prefix-receive-with-state A i*)
 (*checked-staged-query-prefix-dynamic-index-prehit*
staged-query-prefix-conceptual-query-target)
adversary-initial-state + *query-error-bound*

proof (*rule*

checked-staged-query-prefix-dynamic-index-hit-bound-by-prehit-and-query-error
 [*OF raw-bound*])

fix *prefix prefix-state*

assume

Some ((*prefix, prefix-state*), *prefix-state*) $\in$
set-dist
 (*execute* (*checked-staged-query-prefix-with-state A i*)
adversary-initial-state)

show

staged-query-prefix-conceptual-query-target prefix prefix-state
 $\subseteq$ *query-sample-space* $\wedge$
nnreal
 (*card*
 (*staged-query-prefix-conceptual-query-target prefix*
prefix-state)) /
nnreal (*card query-sample-space*) $\leq$ *query-error-bound*
using *staged-query-prefix-conceptual-query-target-subset*
staged-query-prefix-conceptual-query-target-fraction-bound
by *blast*

qed

have *prehit-bound:*

wp-event (*checked-staged-query-prefix-receive-with-state A i*)
 (*checked-staged-query-prefix-dynamic-index-prehit*
staged-query-prefix-conceptual-query-target)
adversary-initial-state $\leq$
staged-phase-relation-error size
 (*staged-query-search-queries budgets i + 1*)
by (*rule checked-staged-query-prefix-conceptual-query-target-prehit-bound*
 [*OF wf controlled*])

```

    (use i-bound in simp)
  show ?thesis
    by (rule order-trans[OF hit-bound])
      (intro add-mono prehit-bound order-refl)
qed

lemma checked-staged-query-prefix-conceptual-default-query-target-hit-bound-from-budgets:
  assumes wf: staged-budget-wellformed budgets
    and controlled: staged-adversary-controlled budgets A
    and i-bound: i < rounds
  shows
    wp-event (checked-staged-query-prefix-receive-with-state A i)
      (checked-staged-query-prefix-dynamic-index-hit
        (staged-query-prefix-conceptual-default-query-target trace-default
          composition-default))
    adversary-initial-state ≤
    staged-phase-relation-error size
      (staged-query-search-queries budgets i + 1) + query-error-bound
  proof -
    have raw-bound: query-index-raw-preimage-bound
      by (rule query-index-raw-preimage-bound-from-sampler-wellformed)
    have hit-bound:
      wp-event (checked-staged-query-prefix-receive-with-state A i)
        (checked-staged-query-prefix-dynamic-index-hit
          (staged-query-prefix-conceptual-default-query-target trace-default
            composition-default))
      adversary-initial-state ≤
      wp-event (checked-staged-query-prefix-receive-with-state A i)
        (checked-staged-query-prefix-dynamic-index-prehit
          (staged-query-prefix-conceptual-default-query-target trace-default
            composition-default))
      adversary-initial-state + query-error-bound
    proof (rule
      checked-staged-query-prefix-dynamic-index-hit-bound-by-prehit-and-query-error
        [OF raw-bound])
      fix prefix prefix-state
      assume
        Some ((prefix, prefix-state), prefix-state) ∈
          set-dist
            (execute (checked-staged-query-prefix-with-state A i)
              adversary-initial-state)
      show
        staged-query-prefix-conceptual-default-query-target trace-default
          composition-default prefix prefix-state ⊆ query-sample-space ∧
        nnreal
          (card
            (staged-query-prefix-conceptual-default-query-target trace-default
              composition-default prefix prefix-state)) /
        nnreal (card query-sample-space) ≤ query-error-bound
    qed
  qed

```

```

using staged-query-prefix-conceptual-default-query-target-subset
      staged-query-prefix-conceptual-default-query-target-fraction-bound
by blast
qed
have prehit-bound:
  wp-event (checked-staged-query-prefix-receive-with-state A i)
    (checked-staged-query-prefix-dynamic-index-prehit
      (staged-query-prefix-conceptual-default-query-target trace-default
        composition-default))
  adversary-initial-state  $\leq$ 
  staged-phase-relation-error size
    (staged-query-search-queries budgets i + 1)
by (rule
      checked-staged-query-prefix-conceptual-default-query-target-prehit-bound
      [OF wf controlled])
    (use i-bound in simp)
show ?thesis
by (rule order-trans[OF hit-bound])
    (intro add-mono prehit-bound order-refl)
qed

lemma checked-staged-query-prefix-conceptual-empty-query-target-hit-bound-from-budgets:
assumes wf: staged-budget-wellformed budgets
and controlled: staged-adversary-controlled budgets A
and i-bound: i < rounds
shows
  wp-event (checked-staged-query-prefix-receive-with-state A i)
    (checked-staged-query-prefix-dynamic-index-hit
      staged-query-prefix-conceptual-empty-query-target)
  adversary-initial-state  $\leq$ 
  staged-phase-relation-error size
    (staged-query-search-queries budgets i + 1) + query-error-bound
proof -
have raw-bound: query-index-raw-preimage-bound
by (rule query-index-raw-preimage-bound-from-sampler-wellformed)
have hit-bound:
  wp-event (checked-staged-query-prefix-receive-with-state A i)
    (checked-staged-query-prefix-dynamic-index-hit
      staged-query-prefix-conceptual-empty-query-target)
  adversary-initial-state  $\leq$ 
  wp-event (checked-staged-query-prefix-receive-with-state A i)
    (checked-staged-query-prefix-dynamic-index-prehit
      staged-query-prefix-conceptual-empty-query-target)
  adversary-initial-state + query-error-bound
proof (rule
  checked-staged-query-prefix-dynamic-index-hit-bound-by-prehit-and-query-error
  [OF raw-bound])
fix prefix prefix-state
assume

```

Some $((\text{prefix}, \text{prefix-state}), \text{prefix-state}) \in$
set-dist
 (execute (checked-staged-query-prefix-with-state A i)
 adversary-initial-state)
show
staged-query-prefix-conceptual-empty-query-target prefix prefix-state
 $\subseteq \text{query-sample-space} \wedge$
nnreal
 (card
 (staged-query-prefix-conceptual-empty-query-target *prefix*
prefix-state)) /
nnreal (card *query-sample-space*) $\leq$ *query-error-bound*
using *staged-query-prefix-conceptual-empty-query-target-subset*
staged-query-prefix-conceptual-empty-query-target-fraction-bound
by *blast*
qed
have *prehit-bound*:
wp-event (checked-staged-query-prefix-receive-with-state A i)
 (checked-staged-query-prefix-dynamic-index-prehit
 staged-query-prefix-conceptual-empty-query-target)
adversary-initial-state $\leq$
staged-phase-relation-error size
 (staged-query-search-queries budgets $i + 1$)
by (rule
 checked-staged-query-prefix-conceptual-empty-query-target-prehit-bound
 [OF *wf controlled*])
 (use *i-bound* **in** *simp*)
show *?thesis*
by (rule *order-trans*[OF *hit-bound*])
 (intro *add-mono prehit-bound order-refl*)
qed
definition *checked-staged-security-with-query-prefix-conceptual-target-hit*
where
checked-staged-security-with-query-prefix-conceptual-target-hit out $\longleftrightarrow$
checked-staged-security-with-query-prefix-dynamic-index-hit
staged-query-prefix-conceptual-query-target out
definition *checked-staged-security-with-query-prefix-conceptual-default-target-hit*
where
checked-staged-security-with-query-prefix-conceptual-default-target-hit
trace-default composition-default out $\longleftrightarrow$
checked-staged-security-with-query-prefix-dynamic-index-hit
 (staged-query-prefix-conceptual-default-query-target *trace-default*
composition-default) *out*
definition *checked-staged-security-with-query-prefix-conceptual-empty-target-hit*
where
checked-staged-security-with-query-prefix-conceptual-empty-target-hit out

$\longleftrightarrow$

checked-staged-security-with-query-prefix-dynamic-index-hit
staged-query-prefix-conceptual-empty-query-target out

lemma *checked-staged-security-with-query-prefix-conceptual-target-hit-bound:*

assumes *wf: staged-budget-wellformed budgets*

and *controlled: staged-adversary-controlled budgets A*

and *i-bound: i < rounds*

shows

wp-event

(*checked-staged-security-experiment-with-query-prefix-data-state A i*)

checked-staged-security-with-query-prefix-conceptual-target-hit

adversary-initial-state $\leq$

staged-phase-relation-error size

(*staged-query-search-queries budgets i + 1*) + *query-error-bound*

unfolding *checked-staged-security-with-query-prefix-conceptual-target-hit-def*

by (rule *checked-staged-security-with-query-prefix-dynamic-index-hit-bound*)

(rule *checked-staged-query-prefix-conceptual-query-target-hit-bound-from-budgets*
[*OF wf controlled i-bound*])

lemma *checked-staged-security-with-query-prefix-conceptual-default-target-hit-bound:*

assumes *wf: staged-budget-wellformed budgets*

and *controlled: staged-adversary-controlled budgets A*

and *i-bound: i < rounds*

shows

wp-event

(*checked-staged-security-experiment-with-query-prefix-data-state A i*)

(*checked-staged-security-with-query-prefix-conceptual-default-target-hit*

trace-default composition-default)

adversary-initial-state $\leq$

staged-phase-relation-error size

(*staged-query-search-queries budgets i + 1*) + *query-error-bound*

unfolding

checked-staged-security-with-query-prefix-conceptual-default-target-hit-def

by (rule *checked-staged-security-with-query-prefix-dynamic-index-hit-bound*)

(rule

checked-staged-query-prefix-conceptual-default-query-target-hit-bound-from-budgets
[*OF wf controlled i-bound*])

lemma *checked-staged-security-with-query-prefix-conceptual-empty-target-hit-bound:*

assumes *wf: staged-budget-wellformed budgets*

and *controlled: staged-adversary-controlled budgets A*

and *i-bound: i < rounds*

shows

wp-event

(*checked-staged-security-experiment-with-query-prefix-data-state A i*)

checked-staged-security-with-query-prefix-conceptual-empty-target-hit

adversary-initial-state $\leq$

staged-phase-relation-error size

```

    (staged-query-search-queries budgets  $i + 1$ ) + query-error-bound
  unfolding
    checked-staged-security-with-query-prefix-conceptual-empty-target-hit-def
  by (rule checked-staged-security-with-query-prefix-dynamic-index-hit-bound)
    (rule
      checked-staged-query-prefix-conceptual-empty-query-target-hit-bound-from-budgets
      [OF wf controlled i-bound])

end

end

theory Soundness-Conceptual-Query-Current
imports
  Soundness-Conceptual-Query-Target
  Staged-Security-Experiment-Composition-Query-Current
begin

Connect the existing prefix-authenticated query target to the conceptual prefix tables.
This is a deterministic bridge: it only uses openings that are already authenticated in the
query-prefix state, so the resulting target remains fixed before the query-index challenge
is sampled.

context soundness
begin

definition staged-query-prefix-prefix-authenticated-empty-opening-target
  :: 'f staged-query-prefix-data  $\Rightarrow$  'f protocol-channel  $\Rightarrow$  nat set
where
  staged-query-prefix-prefix-authenticated-empty-opening-target
    prefix prefix-state =
    {idx  $\in$  query-sample-space.
       $\exists$  (trace-openings :: 'f authenticated-opening list).
        partial-authenticated-table (sqp-trace-root prefix)
          (scale * clength) trace-openings prefix-state  $\wedge$ 
        map opening-index trace-openings = powers-scaled idx  $\wedge$ 
        cp-eval (sqp-alphas prefix) (map opening-value trace-openings)
          (h  $\wedge$  idx * shift) = sqp-composition-final prefix}

lemma staged-query-prefix-prefix-authenticated-empty-opening-target-subset:
  staged-query-prefix-prefix-authenticated-empty-opening-target
    prefix prefix-state  $\subseteq$  query-sample-space
unfolding staged-query-prefix-prefix-authenticated-empty-opening-target-def
by blast

definition checked-staged-security-with-query-prefix-prefix-authenticated-empty-opening-hit
where
  checked-staged-security-with-query-prefix-prefix-authenticated-empty-opening-hit
    trace-openings i out  $\longleftrightarrow$ 

```

```

(case out of
  None  $\Rightarrow$  False
| Some (((((prefix, prefix-state), raw), raw-state),
  ((data, attacker-state), result)), final-state)  $\Rightarrow$ 
  partial-authenticated-table (sqp-trace-root prefix)
    (scale * clength) trace-openings prefix-state  $\wedge$ 
  map opening-index trace-openings = powers-scaled (index (to-nat raw))  $\wedge$ 
  cp-eval (sqp-alphas prefix) (map opening-value trace-openings)
    (h  $\wedge$  index (to-nat raw) * shift) =
  sqp-composition-final prefix)

```

definition *checked-staged-security-with-query-prefix-empty-authenticated-opening-hit*
where

```

checked-staged-security-with-query-prefix-empty-authenticated-opening-hit
  trace-openings i out  $\longleftrightarrow$ 
(case out of
  None  $\Rightarrow$  False
| Some (((((prefix, prefix-state), raw), raw-state),
  ((data, attacker-state), result)), final-state)  $\Rightarrow$ 
  partial-authenticated-table (sqp-trace-root prefix)
    (scale * clength) trace-openings final-state  $\wedge$ 
  map opening-index trace-openings = powers-scaled (index (to-nat raw))  $\wedge$ 
  cp-eval (sqp-alphas prefix) (map opening-value trace-openings)
    (h  $\wedge$  index (to-nat raw) * shift) =
  sqp-composition-final prefix)

```

definition *checked-staged-security-with-query-prefix-empty-trace-path-output-hit*
where

```

checked-staged-security-with-query-prefix-empty-trace-path-output-hit
  trace-openings out  $\longleftrightarrow$ 
(case out of
  None  $\Rightarrow$  False
| Some (((((prefix, prefix-state), raw), raw-state),
  ((data, attacker-state), result)), final-state)  $\Rightarrow$ 
  ( $\exists$  opening  $\in$  set trace-openings.
    hash-map-new-output-hit
      (merkle-path-target-roots final-state
        (sqp-trace-root prefix)
        (scale * clength)
        (opening-index opening)
        (opening-value opening)
        (opening-path opening))
      prefix-state final-state))

```

definition *checked-staged-security-with-query-prefix-current-empty-prefix-authenticated-hit-at*
where

```

checked-staged-security-with-query-prefix-current-empty-prefix-authenticated-hit-at
  i out  $\longleftrightarrow$ 
  ( $\exists$  (trace-openings :: 'f authenticated-opening list).

```

checked-staged-security-with-query-prefix-prefix-authenticated-empty-opening-hit
trace-openings i out)

definition *checked-staged-security-with-query-prefix-current-empty-authenticated-hit-at*
where

checked-staged-security-with-query-prefix-current-empty-authenticated-hit-at
i out $\longleftrightarrow$
 $(\exists (trace-openings :: 'f \text{ authenticated-opening list}).$
checked-staged-security-with-query-prefix-empty-authenticated-opening-hit
trace-openings i out)

definition *checked-staged-security-with-query-prefix-current-empty-trace-path-output-hit-at*
where

checked-staged-security-with-query-prefix-current-empty-trace-path-output-hit-at
i out $\longleftrightarrow$
 $(\exists (trace-openings :: 'f \text{ authenticated-opening list}).$
checked-staged-security-with-query-prefix-empty-trace-path-output-hit
trace-openings out)

definition *checked-staged-security-with-query-prefix-conceptual-empty-context-residual-at*
where

checked-staged-security-with-query-prefix-conceptual-empty-context-residual-at
out $\longleftrightarrow$
 $(\text{case } out \text{ of}$
 $\text{None} \Rightarrow \text{False}$
 $| \text{Some } (((((prefix, prefix-state), raw), raw-state),$
 $((data, attacker-state), result)), final-state) \Rightarrow$
 $hash-map-output-collision \text{ prefix-state} \vee$
 $\neg \text{trace-table-low-degree}$
 $(query-prefix-trace-conceptual-table \text{ prefix prefix-state}) \vee$
 $all-queries-consistent$
 $(query-prefix-trace-conceptual-table \text{ prefix prefix-state})$
 $(\text{replicate } (scale * clength) (sqp-composition-final \text{ prefix}))$
 $(sqp-alphas \text{ prefix}))$

definition *checked-staged-security-with-query-prefix-conceptual-empty-context-residual-hit-at*
where

checked-staged-security-with-query-prefix-conceptual-empty-context-residual-hit-at
 $(i :: nat) \text{ out} \longleftrightarrow$
checked-staged-security-with-query-prefix-current-empty-prefix-authenticated-hit-at
i out $\wedge$
checked-staged-security-with-query-prefix-conceptual-empty-context-residual-at
out

lemma *checked-staged-security-with-query-prefix-current-empty-prefix-authenticated-hit-at-imp-dynamic-target:*
assumes *hit:*

checked-staged-security-with-query-prefix-current-empty-prefix-authenticated-hit-at
i out

shows


```

    checked-staged-security-with-query-prefix-dynamic-index-hit
    staged-query-prefix-prefix-authenticated-empty-opening-target out
proof (cases out)
  case None
  then show ?thesis
    using hit
    unfolding
      checked-staged-security-with-query-prefix-current-empty-prefix-authenticated-hit-at-def
      checked-staged-security-with-query-prefix-prefix-authenticated-empty-opening-hit-def
    by simp
next
  case (Some packed)
  then obtain prefix prefix-state raw raw-state data attacker-state result
    final-state where out-eq:
    out =
      Some (((((prefix, prefix-state), raw), raw-state),
        ((data, attacker-state), result)), final-state)
    by (cases packed) (auto split: prod.splits)
  from hit obtain trace-openings where prefix-hit:
    checked-staged-security-with-query-prefix-prefix-authenticated-empty-opening-hit
    trace-openings i out
  unfolding
    checked-staged-security-with-query-prefix-current-empty-prefix-authenticated-hit-at-def
  by blast
  have idx-in:
    index (to-nat raw) ∈
      staged-query-prefix-prefix-authenticated-empty-opening-target
      prefix prefix-state
  using prefix-hit index-less-query-sample-space
  unfolding out-eq
    checked-staged-security-with-query-prefix-prefix-authenticated-empty-opening-hit-def
    staged-query-prefix-prefix-authenticated-empty-opening-target-def
    query-sample-space-def
  by auto
  show ?thesis
    unfolding out-eq
      checked-staged-security-with-query-prefix-dynamic-index-hit-def
      checked-staged-query-prefix-dynamic-index-hit-def
    using idx-in by simp
qed

lemma checked-staged-security-with-query-prefix-empty-authenticated-opening-hit-imp-prefix-or-path-output-hit:
  assumes ext: prefix-state ≤ final-state
  and hit:
    checked-staged-security-with-query-prefix-empty-authenticated-opening-hit
    trace-openings i
    (Some (((((prefix, prefix-state), raw), raw-state),
      ((data, attacker-state), result)), final-state))
  shows

```

```

checked-staged-security-with-query-prefix-prefix-authenticated-empty-opening-hit
  trace-openings i
  (Some (((((prefix, prefix-state), raw), raw-state),
    ((data, attacker-state), result)), final-state)) ∨
checked-staged-security-with-query-prefix-empty-trace-path-output-hit
  trace-openings
  (Some (((((prefix, prefix-state), raw), raw-state),
    ((data, attacker-state), result)), final-state))
proof –
  have trace-table:
    partial-authenticated-table (sqp-trace-root prefix)
      (scale * clength) trace-openings final-state
  and trace-indices:
    map opening-index trace-openings = powers-scaled (index (to-nat raw))
  and consistent:
    cp-eval (sqp-alphas prefix) (map opening-value trace-openings)
      (h ^ index (to-nat raw) * shift) =
      sqp-composition-final prefix
  using hit
  unfolding
    checked-staged-security-with-query-prefix-empty-authenticated-opening-hit-def
  by simp-all
  have trace-pullback:
    partial-authenticated-table (sqp-trace-root prefix)
      (scale * clength) trace-openings prefix-state ∨
    (∃ opn ∈ set trace-openings.
      hash-map-new-output-hit
        (merkle-path-target-roots final-state (sqp-trace-root prefix)
          (scale * clength)
          (opening-index opn)
          (opening-value opn)
          (opening-path opn)) prefix-state final-state)
  by (rule partial-authenticated-table-pullback-or-new-output-hit
    [OF ext trace-table])
  then show ?thesis
  proof
    assume trace-prefix:
      partial-authenticated-table (sqp-trace-root prefix)
        (scale * clength) trace-openings prefix-state
    then have
      checked-staged-security-with-query-prefix-prefix-authenticated-empty-opening-hit
        trace-openings i
      (Some (((((prefix, prefix-state), raw), raw-state),
        ((data, attacker-state), result)), final-state))
    using trace-indices consistent
    unfolding
      checked-staged-security-with-query-prefix-prefix-authenticated-empty-opening-hit-def
    by simp
  then show ?thesis by simp

```

```

next
  assume
     $\exists \text{opn} \in \text{set trace-openings.}$ 
    hash-map-new-output-hit
      (merkle-path-target-roots final-state (sqp-trace-root prefix)
        (scale * clength)
        (opening-index opn)
        (opening-value opn)
        (opening-path opn)) prefix-state final-state
  then have
    checked-staged-security-with-query-prefix-empty-trace-path-output-hit
      trace-openings
      (Some (((prefix, prefix-state), raw), raw-state),
        ((data, attacker-state), result)), final-state))
  unfolding
    checked-staged-security-with-query-prefix-empty-trace-path-output-hit-def
  by simp
  then show ?thesis by simp
qed
qed

lemma checked-staged-security-with-query-prefix-empty-authenticated-opening-hit-imp-prefix-or-path-output-hit-on-su
  assumes wf: staged-budget-wellformed budgets
  and controlled: staged-adversary-controlled budgets A
  and i-bound: i < rounds
  and support:
    out  $\in$ 
      set-dist
      (execute
        (checked-staged-security-experiment-with-query-prefix-data-state A i)
        adversary-initial-state)
  and hit:
    checked-staged-security-with-query-prefix-empty-authenticated-opening-hit
      trace-openings i out
  shows
    checked-staged-security-with-query-prefix-prefix-authenticated-empty-opening-hit
      trace-openings i out  $\vee$ 
    checked-staged-security-with-query-prefix-empty-trace-path-output-hit
      trace-openings out
proof (cases out)
  case None
  then show ?thesis
    using hit
    unfolding
      checked-staged-security-with-query-prefix-empty-authenticated-opening-hit-def
    by simp
next
  case (Some packed)
  then obtain prefix prefix-state raw raw-state data attacker-state result

```

```

    final-state where out-eq:
out =
    Some (((((prefix, prefix-state), raw), raw-state),
      ((data, attacker-state), result)), final-state)
by (cases packed) (auto split: prod.splits)
have ext: prefix-state ≤ final-state
by (rule
    checked-staged-security-experiment-with-query-prefix-data-state-prefix-extends-final
    [OF wf controlled i-bound])
    (use support out-eq in simp)
show ?thesis
unfolding out-eq
by (rule
    checked-staged-security-with-query-prefix-empty-authenticated-opening-hit-imp-prefix-or-path-output-hit
    [OF ext])
    (use hit out-eq in simp)
qed

lemma checked-staged-security-with-query-prefix-current-empty-authenticated-hit-at-imp-prefix-or-path-output-hit-on-
assumes wf: staged-budget-wellformed budgets
and controlled: staged-adversary-controlled budgets A
and i-bound: i < rounds
and support:
    out ∈
    set-dist
    (execute
      (checked-staged-security-experiment-with-query-prefix-data-state A i)
      adversary-initial-state)
and hit:
    checked-staged-security-with-query-prefix-current-empty-authenticated-hit-at
    i out
shows
    checked-staged-security-with-query-prefix-current-empty-prefix-authenticated-hit-at
    i out ∨
    checked-staged-security-with-query-prefix-current-empty-trace-path-output-hit-at
    i out
proof –
from hit obtain trace-openings where trace-hit:
    checked-staged-security-with-query-prefix-empty-authenticated-opening-hit
    trace-openings i out
unfolding
    checked-staged-security-with-query-prefix-current-empty-authenticated-hit-at-def
by blast
have split:
    checked-staged-security-with-query-prefix-prefix-authenticated-empty-opening-hit
    trace-openings i out ∨
    checked-staged-security-with-query-prefix-empty-trace-path-output-hit
    trace-openings out
by (rule

```

```

    checked-staged-security-with-query-prefix-empty-authenticated-opening-hit-imp-prefix-or-path-output-hit-on-suppo
    [OF wf controlled i-bound support trace-hit])
  then show ?thesis
    unfolding
      checked-staged-security-with-query-prefix-current-empty-prefix-authenticated-hit-at-def
      checked-staged-security-with-query-prefix-current-empty-trace-path-output-hit-at-def
    by blast
qed

```

lemma *low-degree-all-queries-consistent-imp-composition-trace-bad-alpha-space:*

```

  assumes false-statement:  $\neg$  exists-valid-trace
    and len-as: length as = length spec
    and trace-low: trace-table-low-degree trace-table
    and composition-low:
      composition-table-low-degree maxDegree composition-table
    and all-queries:
      all-queries-consistent trace-table composition-table as
  shows  $as \in$  composition-trace-bad-alpha-space trace-table
proof -
  from trace-low obtain f where deg-f: degree f < clength
    and trace-table: trace-table = map (poly f) eval-domain
    unfolding trace-table-low-degree-def by blast
  have bounds: common-denominator-degree-bounds f as
    by (rule common-denominator-degree-bounds-from-spec-degree-wellformed
      [OF spec-degree-wellformed-from-spec-query-margin deg-f])
  have hides:
    random-combination-common-denominator-hides-violations f as
    by (rule random-combination-common-denominator-hides-from-queries
      [OF len-as false-statement deg-f trace-table composition-low
        all-queries bounds])
  have violated: violated-constraints f  $\neq$  {}
    by (rule false-statement-violated-constraints
      [OF false-statement deg-f])
  have as-common:  $as \in$  common-denominator-hiding-alpha-space f
    using len-as hides
    unfolding common-denominator-hiding-alpha-space-def alpha-space-def
    by simp
  have witness-eq: low-degree-trace-witness trace-table = f
    by (rule low-degree-trace-witness-eq[OF deg-f trace-table])
  show ?thesis
    using deg-f trace-table violated bounds as-common
    unfolding composition-trace-bad-alpha-space-def witness-eq
    by simp
qed

```

lemma *staged-query-prefix-authenticated-empty-opening-target-subset-conceptual-empty:*

```

  assumes clean:  $\neg$  hash-map-output-collision prefix-state
    and trace-low:
      trace-table-low-degree

```

```

    (query-prefix-trace-conceptual-table prefix prefix-state)
  and not-all:
    ¬ all-queries-consistent
    (query-prefix-trace-conceptual-table prefix prefix-state)
    (replicate (scale * clength) (sqp-composition-final prefix))
    (sqp-alphas prefix)
  shows
    staged-query-prefix-prefix-authenticated-empty-opening-target
      prefix prefix-state ⊆
    staged-query-prefix-conceptual-empty-query-target prefix prefix-state
proof
  fix idx
  assume idx-in:
    idx ∈
      staged-query-prefix-prefix-authenticated-empty-opening-target
      prefix prefix-state
  then obtain trace-openings where
    idx-sample: idx ∈ query-sample-space
  and trace-table:
    partial-authenticated-table (sqp-trace-root prefix)
    (scale * clength) trace-openings prefix-state
  and trace-indices:
    map opening-index trace-openings = powers-scaled idx
  and final-eq:
    cp-eval (sqp-alphas prefix) (map opening-value trace-openings)
    (h ^ idx * shift) = sqp-composition-final prefix
  unfolding staged-query-prefix-prefix-authenticated-empty-opening-target-def
  by blast
  let ?trace-table =
    query-prefix-trace-conceptual-table prefix prefix-state
  let ?composition-table =
    replicate (scale * clength) (sqp-composition-final prefix)
  have idx-bound: idx < clength * scale
    by (rule query-sample-space-less-domain[OF idx-sample])
  have trace-values:
    map (!) ?trace-table (powers-scaled idx) =
      map opening-value trace-openings
proof (rule nth-equalityI)
  show
    length (map (!) ?trace-table (powers-scaled idx)) =
      length (map opening-value trace-openings)
    using trace-indices by (metis length-map)
next
  fix k
  assume k-bound:
    k < length (map (!) ?trace-table (powers-scaled idx))
  then have ps-bound: k < length (powers-scaled idx)
    by simp
  have opn-bound: k < length trace-openings

```

```

    using trace-indices ps-bound by (metis length-map)
  have opn-in: trace-openings ! k ∈ set trace-openings
    by (rule nth-mem[OF opn-bound])
  have idx-eq:
    opening-index (trace-openings ! k) = powers-scaled idx ! k
    using trace-indices ps-bound opn-bound
    by (metis nth-map)
  have conceptual:
    ?trace-table ! opening-index (trace-openings ! k) =
      opening-value (trace-openings ! k)
    by (rule
      query-prefix-trace-conceptual-table-agrees-with-partial-authenticated-table
      [OF clean trace-table opn-in])
  show
    map (!) ?trace-table (powers-scaled idx) ! k =
      map opening-value trace-openings ! k
    using ps-bound opn-bound idx-eq conceptual by simp
qed
have query-consistent:
  query-consistent-at ?trace-table ?composition-table
    (sqp-alphas prefix) idx
proof -
  have composition-value:
    ?composition-table ! idx = sqp-composition-final prefix
    using idx-bound by (simp add: mult.commute)
  have cp-eq:
    cp-eval (sqp-alphas prefix)
      (map (!) ?trace-table (powers-scaled idx))
      (h ^ idx * shift) =
        sqp-composition-final prefix
    using trace-values final-eq by simp
  show ?thesis
    unfolding query-consistent-at-def
    using idx-bound query-sample-space-powers-scaled-bound[OF idx-sample]
      composition-value cp-eq by (simp add: mult.commute)
qed
show
  idx ∈
    staged-query-prefix-conceptual-empty-query-target prefix prefix-state
  unfolding staged-query-prefix-conceptual-empty-query-target-def
    query-sampling-success-space-def query-agreement-indices-def
  using idx-sample trace-low constant-composition-table-low-degree not-all
    query-consistent by simp
qed

lemma checked-staged-security-with-query-prefix-current-empty-prefix-authenticated-hit-imp-conceptual-empty-or-res
  assumes hit:
    checked-staged-security-with-query-prefix-current-empty-prefix-authenticated-hit-at
      i out

```

```

shows
  checked-staged-security-with-query-prefix-conceptual-empty-target-hit out  $\vee$ 
  checked-staged-security-with-query-prefix-conceptual-empty-context-residual-hit-at
    i out
proof (cases out)
  case None
  then show ?thesis
    using hit
    unfolding
      checked-staged-security-with-query-prefix-current-empty-prefix-authenticated-hit-at-def
      checked-staged-security-with-query-prefix-prefix-authenticated-empty-opening-hit-def
    by simp
next
  case (Some packed)
  then obtain prefix prefix-state raw raw-state data attacker-state result
    final-state where out-eq:
    out =
      Some (((((prefix, prefix-state), raw), raw-state),
        ((data, attacker-state), result)), final-state)
    by (cases packed) (auto split: prod.splits)
  show ?thesis
  proof (cases
     $\neg$  hash-map-output-collision prefix-state  $\wedge$ 
    trace-table-low-degree
    (query-prefix-trace-conceptual-table prefix prefix-state)  $\wedge$ 
     $\neg$  all-queries-consistent
    (query-prefix-trace-conceptual-table prefix prefix-state)
    (replicate (scale * clength) (sqp-composition-final prefix))
    (sqp-alphas prefix))
  case False
  then have
    checked-staged-security-with-query-prefix-conceptual-empty-context-residual-hit-at
      i out
  unfolding
    checked-staged-security-with-query-prefix-conceptual-empty-context-residual-hit-at-def
    checked-staged-security-with-query-prefix-conceptual-empty-context-residual-at-def
    out-eq
  using hit out-eq by simp
  then show ?thesis by simp
next
  case True
  then have clean:
     $\neg$  hash-map-output-collision prefix-state
  and trace-low:
    trace-table-low-degree
    (query-prefix-trace-conceptual-table prefix prefix-state)
  and not-all:
     $\neg$  all-queries-consistent
    (query-prefix-trace-conceptual-table prefix prefix-state)

```



```

      (replicate (scale * clength) (sqp-composition-final prefix))
      (sqp-alphas prefix)
    by blast+
  have prefix-hit:
    checked-staged-security-with-query-prefix-dynamic-index-hit
      staged-query-prefix-prefix-authenticated-empty-opening-target out
  by (rule
    checked-staged-security-with-query-prefix-current-empty-prefix-authenticated-hit-at-imp-dynamic-target
      [OF hit])
  have subset:
    staged-query-prefix-prefix-authenticated-empty-opening-target
      prefix prefix-state  $\subseteq$ 
    staged-query-prefix-conceptual-empty-query-target prefix prefix-state
  by (rule
    staged-query-prefix-prefix-authenticated-empty-opening-target-subset-conceptual-empty
      [OF clean trace-low not-all])
  have conceptual-hit:
    checked-staged-security-with-query-prefix-conceptual-empty-target-hit
      out
  using prefix-hit subset
  unfolding out-eq
    checked-staged-security-with-query-prefix-conceptual-empty-target-hit-def
    checked-staged-security-with-query-prefix-dynamic-index-hit-def
    checked-staged-query-prefix-dynamic-index-hit-def
  by auto
  then show ?thesis by simp
qed
qed

lemma checked-staged-security-with-query-prefix-current-empty-prefix-authenticated-hit-at-bound-from-conceptual-em
  assumes wf: staged-budget-wellformed budgets
  and controlled: staged-adversary-controlled budgets A
  and i-bound:  $i < \text{rounds}$ 
  and residual-bound:
    wp-event
      (checked-staged-security-experiment-with-query-prefix-data-state A i)
      (checked-staged-security-with-query-prefix-conceptual-empty-context-residual-hit-at
        i)
      adversary-initial-state  $\leq R$ 
  shows
    wp-event
      (checked-staged-security-experiment-with-query-prefix-data-state A i)
      (checked-staged-security-with-query-prefix-current-empty-prefix-authenticated-hit-at
        i)
      adversary-initial-state  $\leq$ 
      staged-phase-relation-error size
      (staged-query-search-queries budgets  $i + 1$ ) +
      query-error-bound + R
proof -

```

```

let ?M = checked-staged-security-experiment-with-query-prefix-data-state A i
let ?Conceptual =
  checked-staged-security-with-query-prefix-conceptual-empty-target-hit
let ?Residual =
  checked-staged-security-with-query-prefix-conceptual-empty-context-residual-hit-at
  i
have event-le:
  wp-event ?M
    (checked-staged-security-with-query-prefix-current-empty-prefix-authenticated-hit-at
     i)
    adversary-initial-state ≤
  wp-event ?M (λout. ?Conceptual out ∨ ?Residual out)
    adversary-initial-state
by (rule wp-event-mono)
  (rule
   checked-staged-security-with-query-prefix-current-empty-prefix-authenticated-hit-imp-conceptual-empty-or-residual)
have conceptual-bound:
  wp-event ?M ?Conceptual adversary-initial-state ≤
    staged-phase-relation-error size
    (staged-query-search-queries budgets i + 1) + query-error-bound
by (rule
   checked-staged-security-with-query-prefix-conceptual-empty-target-hit-bound
   [OF wf controlled i-bound])
have union-bound:
  wp-event ?M (λout. ?Conceptual out ∨ ?Residual out)
    adversary-initial-state ≤
  wp-event ?M ?Conceptual adversary-initial-state +
  wp-event ?M ?Residual adversary-initial-state
by (rule wp-event-union-bound)
have sum-bound:
  wp-event ?M ?Conceptual adversary-initial-state +
  wp-event ?M ?Residual adversary-initial-state ≤
    (staged-phase-relation-error size
     (staged-query-search-queries budgets i + 1) + query-error-bound) +
    R
by (intro add-mono conceptual-bound residual-bound)
show ?thesis
by (rule order-trans[OF event-le])
  (rule order-trans[OF union-bound sum-bound])
qed

```

lemma checked-staged-security-with-query-prefix-current-empty-authenticated-hit-at-bound-from-prefix-and-path:
assumes wf: staged-budget-wellformed budgets
and controlled: staged-adversary-controlled budgets A
and i-bound: i < rounds
and prefix-bound:
 wp-event
 (checked-staged-security-experiment-with-query-prefix-data-state A i)
 (checked-staged-security-with-query-prefix-current-empty-prefix-authenticated-hit-at

$i)$
 $\text{adversary-initial-state} \leq P$
and path-bound :
 wp-event
 $(\text{checked-staged-security-experiment-with-query-prefix-data-state } A \ i)$
 $(\text{checked-staged-security-with-query-prefix-current-empty-trace-path-output-hit-at}$
 $i)$
 $\text{adversary-initial-state} \leq T$
shows
 wp-event
 $(\text{checked-staged-security-experiment-with-query-prefix-data-state } A \ i)$
 $(\text{checked-staged-security-with-query-prefix-current-empty-authenticated-hit-at}$
 $i)$
 $\text{adversary-initial-state} \leq P + T$
proof –
let $?M = \text{checked-staged-security-experiment-with-query-prefix-data-state } A \ i$
let $?Prefix =$
 $\text{checked-staged-security-with-query-prefix-current-empty-prefix-authenticated-hit-at}$
 i
let $?Path =$
 $\text{checked-staged-security-with-query-prefix-current-empty-trace-path-output-hit-at}$
 i
have event-le :
 $\text{wp-event } ?M$
 $(\text{checked-staged-security-with-query-prefix-current-empty-authenticated-hit-at}$
 $i)$
 $\text{adversary-initial-state} \leq$
 $\text{wp-event } ?M (\lambda \text{out. } ?Prefix \text{ out} \vee ?Path \text{ out})$
 $\text{adversary-initial-state}$
by (rule $\text{wp-event-mono-on-support}$)
(rule
 $\text{checked-staged-security-with-query-prefix-current-empty-authenticated-hit-at-imp-prefix-or-path-output-hit-on-sup}$
 $[OF \text{ wf controlled } i\text{-bound}]$)
have union-bound :
 $\text{wp-event } ?M (\lambda \text{out. } ?Prefix \text{ out} \vee ?Path \text{ out})$
 $\text{adversary-initial-state} \leq$
 $\text{wp-event } ?M ?Prefix \text{ adversary-initial-state} +$
 $\text{wp-event } ?M ?Path \text{ adversary-initial-state}$
by (rule $\text{wp-event-union-bound}$)
have sum-bound :
 $\text{wp-event } ?M ?Prefix \text{ adversary-initial-state} +$
 $\text{wp-event } ?M ?Path \text{ adversary-initial-state} \leq P + T$
by (intro $\text{add-mono prefix-bound path-bound}$)
show $?thesis$
by (rule $\text{order-trans}[OF \text{ event-le}]$)
(rule $\text{order-trans}[OF \text{ union-bound sum-bound}]$)
qed

lemma $\text{staged-query-prefix-prefix-authenticated-opening-target-subset-conceptual}$:

```

assumes clean:  $\neg$  hash-map-output-collision prefix-state
and trace-low:
  trace-table-low-degree
  (query-prefix-trace-conceptual-table prefix prefix-state)
and composition-low:
  composition-table-low-degree maxDegree
  (query-prefix-composition-conceptual-table prefix prefix-state)
and not-all:
   $\neg$  all-queries-consistent
  (query-prefix-trace-conceptual-table prefix prefix-state)
  (query-prefix-composition-conceptual-table prefix prefix-state)
  (sqp-alphas prefix)
shows
  staged-query-prefix-prefix-authenticated-opening-target prefix
  prefix-state  $\subseteq$ 
  staged-query-prefix-conceptual-query-target prefix prefix-state
proof
fix idx
assume idx-in:
  idx  $\in$  staged-query-prefix-prefix-authenticated-opening-target prefix
  prefix-state
then obtain trace-openings composition-openings where
  trace-table:
    partial-authenticated-table (sqp-trace-root prefix)
    (scale * clength) trace-openings prefix-state
and composition-table:
    partial-authenticated-table (hd (sqp-composition-fri-roots prefix))
    (scale * clength) composition-openings prefix-state
and consistent:
    partial-query-openings-consistent trace-openings composition-openings
    (sqp-alphas prefix) idx
unfolding staged-query-prefix-prefix-authenticated-opening-target-def
by blast
let ?trace-openings =
  (replicate rounds []) [0 := trace-openings]
let ?composition-openings =
  (replicate rounds []) [0 := composition-openings]
have zero-bound: 0 < rounds
by (rule rounds-positive)
have trace-len: length ?trace-openings = rounds
by simp
have composition-len: length ?composition-openings = rounds
by simp
have trace-tables:
   $\bigwedge j. j < \text{rounds} \implies$ 
    partial-authenticated-table (sqp-trace-root prefix)
    (scale * clength) (?trace-openings ! j) prefix-state
proof –
fix j

```

```

assume j-bound:  $j < \text{rounds}$ 
show
  partial-authenticated-table (sqp-trace-root prefix)
    (scale * clength) (?trace-openings ! j) prefix-state
proof (cases j = 0)
  case True
  then show ?thesis
    using trace-table zero-bound by simp
next
  case False
  then have ?trace-openings ! j = []
    using j-bound by simp
  then show ?thesis
    unfolding partial-authenticated-table-def by simp
qed
qed
have composition-tables:
   $\bigwedge j. j < \text{rounds} \implies$ 
    partial-authenticated-table (hd (sqp-composition-fri-roots prefix))
      (scale * clength) (?composition-openings ! j) prefix-state
proof –
  fix j
  assume j-bound:  $j < \text{rounds}$ 
  show
    partial-authenticated-table (hd (sqp-composition-fri-roots prefix))
      (scale * clength) (?composition-openings ! j) prefix-state
  proof (cases j = 0)
  case True
  then show ?thesis
    using composition-table zero-bound by simp
next
  case False
  then have ?composition-openings ! j = []
    using j-bound by simp
  then show ?thesis
    unfolding partial-authenticated-table-def by simp
qed
qed
have opening-target:
  idx  $\in$ 
    staged-query-prefix-candidate-opening-query-target-from-prefix
      ?trace-openings ?composition-openings 0 prefix prefix-state
  unfolding staged-query-prefix-candidate-opening-query-target-from-prefix-def
  by (rule partial-query-success-indices-atI,
    rule partial-query-round-consistent-single-roundI
    [OF zero-bound consistent])
have target-subset:
  staged-query-prefix-candidate-opening-query-target-from-prefix
    ?trace-openings ?composition-openings 0 prefix prefix-state  $\subseteq$ 

```

```

    staged-query-prefix-conceptual-query-target prefix prefix-state
  by (rule
    staged-query-prefix-candidate-opening-query-target-from-prefix-subset-conceptual
    [OF clean trace-len composition-len trace-tables composition-tables
      trace-low composition-low not-all])
  show
    idx ∈ staged-query-prefix-conceptual-query-target prefix prefix-state
  using opening-target target-subset by blast
qed

lemma staged-query-prefix-prefix-authenticated-opening-target-fraction-bound-if-conceptual:
  assumes clean:  $\neg$  hash-map-output-collision prefix-state
  and trace-low:
    trace-table-low-degree
    (query-prefix-trace-conceptual-table prefix prefix-state)
  and composition-low:
    composition-table-low-degree maxDegree
    (query-prefix-composition-conceptual-table prefix prefix-state)
  and not-all:
     $\neg$  all-queries-consistent
    (query-prefix-trace-conceptual-table prefix prefix-state)
    (query-prefix-composition-conceptual-table prefix prefix-state)
    (sqp-alphas prefix)
  shows
    nnreal
      (card
        (staged-query-prefix-prefix-authenticated-opening-target prefix
          prefix-state)) /
      nnreal (card query-sample-space)  $\leq$  query-error-bound
proof -
  have subset:
    staged-query-prefix-prefix-authenticated-opening-target prefix
    prefix-state  $\subseteq$ 
    staged-query-prefix-conceptual-query-target prefix prefix-state
  by (rule
    staged-query-prefix-prefix-authenticated-opening-target-subset-conceptual
    [OF clean trace-low composition-low not-all])
  have finite-conceptual:
    finite (staged-query-prefix-conceptual-query-target prefix prefix-state)
  by (rule finite-subset
    [OF staged-query-prefix-conceptual-query-target-subset
      finite-query-sample-space])
  have card-le:
    card
      (staged-query-prefix-prefix-authenticated-opening-target prefix
        prefix-state)  $\leq$ 
    card (staged-query-prefix-conceptual-query-target prefix prefix-state)
  by (rule card-mono[OF finite-conceptual subset])
  have frac-le:

```

```

nnreal
  (card
    (staged-query-prefix-prefix-authenticated-opening-target prefix
      prefix-state)) /
  nnreal (card query-sample-space) ≤
nnreal
  (card
    (staged-query-prefix-conceptual-query-target prefix prefix-state)) /
  nnreal (card query-sample-space)
by (rule nnreal-nat-divide-right-mono[OF card-le])
show ?thesis
by (rule order-trans[OF frac-le
  staged-query-prefix-conceptual-query-target-fraction-bound])
qed

lemma staged-query-prefix-prefix-authenticated-opening-target-subset-conceptual-default:
assumes clean:  $\neg$  hash-map-output-collision prefix-state
and trace-low:
  trace-table-low-degree
  (query-prefix-trace-conceptual-table-with-default prefix
    prefix-state trace-default)
and composition-low:
  composition-table-low-degree maxDegree
  (query-prefix-composition-conceptual-table-with-default prefix
    prefix-state composition-default)
and not-all:
   $\neg$  all-queries-consistent
  (query-prefix-trace-conceptual-table-with-default prefix
    prefix-state trace-default)
  (query-prefix-composition-conceptual-table-with-default prefix
    prefix-state composition-default)
  (sqp-alphas prefix)
shows
  staged-query-prefix-prefix-authenticated-opening-target prefix
    prefix-state  $\subseteq$ 
  staged-query-prefix-conceptual-default-query-target trace-default
    composition-default prefix prefix-state
proof
fix idx
assume idx-in:
  idx  $\in$  staged-query-prefix-prefix-authenticated-opening-target prefix
    prefix-state
then obtain trace-openings composition-openings where
  trace-table:
    partial-authenticated-table (sqp-trace-root prefix)
    (scale * clength) trace-openings prefix-state
and composition-table:
    partial-authenticated-table (hd (sqp-composition-fri-roots prefix))
    (scale * clength) composition-openings prefix-state

```

and *consistent*:
partial-query-openings-consistent trace-openings composition-openings
(sqp-alphas prefix) idx
unfolding *staged-query-prefix-prefix-authenticated-opening-target-def*
by *blast*
let *?trace-openings* =
(replicate rounds []) [0 := trace-openings]
let *?composition-openings* =
(replicate rounds []) [0 := composition-openings]
have *zero-bound: 0 < rounds*
by *(rule rounds-positive)*
have *trace-len: length ?trace-openings = rounds*
by *simp*
have *composition-len: length ?composition-openings = rounds*
by *simp*
have *trace-tables:*
 $\bigwedge j. j < rounds \implies$
partial-authenticated-table (sqp-trace-root prefix)
*(scale * clength) (?trace-openings ! j) prefix-state*
proof –
fix *j*
assume *j-bound: j < rounds*
show
partial-authenticated-table (sqp-trace-root prefix)
*(scale * clength) (?trace-openings ! j) prefix-state*
proof (*cases j = 0*)
case *True*
then show *?thesis*
using *trace-table zero-bound by simp*
next
case *False*
then have *?trace-openings ! j = []*
using *j-bound by simp*
then show *?thesis*
unfolding *partial-authenticated-table-def by simp*
qed
qed
have *composition-tables:*
 $\bigwedge j. j < rounds \implies$
partial-authenticated-table (hd (sqp-composition-fri-roots prefix))
*(scale * clength) (?composition-openings ! j) prefix-state*
proof –
fix *j*
assume *j-bound: j < rounds*
show
partial-authenticated-table (hd (sqp-composition-fri-roots prefix))
*(scale * clength) (?composition-openings ! j) prefix-state*
proof (*cases j = 0*)
case *True*


```

    then show ?thesis
      using composition-table zero-bound by simp
  next
    case False
    then have ?composition-openings ! j = []
      using j-bound by simp
    then show ?thesis
      unfolding partial-authenticated-table-def by simp
  qed
qed
have opening-target:
  idx ∈
    staged-query-prefix-candidate-opening-query-target-from-prefix
    ?trace-openings ?composition-openings 0 prefix prefix-state
  unfolding staged-query-prefix-candidate-opening-query-target-from-prefix-def
  by (rule partial-query-success-indices-atI,
      rule partial-query-round-consistent-single-roundI
        [OF zero-bound consistent])
have target-subset:
  staged-query-prefix-candidate-opening-query-target-from-prefix
    ?trace-openings ?composition-openings 0 prefix prefix-state ⊆
  staged-query-prefix-conceptual-default-query-target trace-default
  composition-default prefix prefix-state
  by (rule
      staged-query-prefix-candidate-opening-query-target-from-prefix-subset-conceptual-default
        [OF clean trace-len composition-len trace-tables composition-tables
          trace-low composition-low not-all])
show
  idx ∈
    staged-query-prefix-conceptual-default-query-target trace-default
    composition-default prefix prefix-state
  using opening-target target-subset by blast
qed

definition checked-staged-security-with-query-prefix-conceptual-context-residual-at
where
  checked-staged-security-with-query-prefix-conceptual-context-residual-at
    out  $\longleftrightarrow$ 
    (case out of
      None  $\Rightarrow$  False
    | Some (((((prefix, prefix-state), raw), raw-state),
      ((data, attacker-state), result)), final-state)  $\Rightarrow$ 
      hash-map-output-collision prefix-state  $\vee$ 
       $\neg$  trace-table-low-degree
      (query-prefix-trace-conceptual-table prefix prefix-state)  $\vee$ 
       $\neg$  composition-table-low-degree maxDegree
      (query-prefix-composition-conceptual-table prefix prefix-state)  $\vee$ 
      all-queries-consistent
      (query-prefix-trace-conceptual-table prefix prefix-state)
    )

```

(*query-prefix-composition-conceptual-table* *prefix* *prefix-state*)
(*sqp-alphas* *prefix*))

definition *checked-staged-security-with-query-prefix-conceptual-default-context-residual-at*
where

checked-staged-security-with-query-prefix-conceptual-default-context-residual-at
trace-default *composition-default* *out* $\longleftrightarrow$
(case *out* of
None $\Rightarrow$ *False*
| *Some* ((((*prefix*, *prefix-state*), *raw*), *raw-state*),
((*data*, *attacker-state*), *result*)), *final-state*) $\Rightarrow$
hash-map-output-collision *prefix-state* $\vee$
 $\neg$ *trace-table-low-degree*
(*query-prefix-trace-conceptual-table-with-default* *prefix*
prefix-state *trace-default*) $\vee$
 $\neg$ *composition-table-low-degree* *maxDegree*
(*query-prefix-composition-conceptual-table-with-default* *prefix*
prefix-state *composition-default*) $\vee$
all-queries-consistent
(*query-prefix-trace-conceptual-table-with-default* *prefix*
prefix-state *trace-default*)
(*query-prefix-composition-conceptual-table-with-default* *prefix*
prefix-state *composition-default*)
(*sqp-alphas* *prefix*))

lemma *checked-staged-security-with-query-prefix-current-prefix-authenticated-hit-imp-conceptual-or-residual-on-support*

assumes *wf*: *staged-budget-wellformed* *budgets*

and *controlled*: *staged-adversary-controlled* *budgets* *A*

and *i-bound*: *i* < *rounds*

and *support*:

out $\in$
set-dist
(*execute*
(*checked-staged-security-experiment-with-query-prefix-data-state* *A* *i*)
adversary-initial-state)

and *hit*:

checked-staged-security-with-query-prefix-current-prefix-authenticated-hit-at
i *out*

shows

checked-staged-security-with-query-prefix-conceptual-target-hit *out* $\vee$
checked-staged-security-with-query-prefix-conceptual-context-residual-at
out

proof (cases *out*)

case *None*

then show *?thesis*

using *hit*

unfolding

checked-staged-security-with-query-prefix-current-prefix-authenticated-hit-at-def
checked-staged-security-with-query-prefix-prefix-authenticated-opening-hit-def

```

    by simp
next
case (Some packed)
then obtain prefix prefix-state raw raw-state data attacker-state result
    final-state where out-eg:
    out =
        Some (((((prefix, prefix-state), raw), raw-state),
            ((data, attacker-state), result)), final-state)
    by (cases packed) (auto split: prod.splits)
show ?thesis
proof (cases
    ¬ hash-map-output-collision prefix-state ∧
    trace-table-low-degree
    (query-prefix-trace-conceptual-table prefix prefix-state) ∧
    composition-table-low-degree maxDegree
    (query-prefix-composition-conceptual-table prefix prefix-state) ∧
    ¬ all-queries-consistent
    (query-prefix-trace-conceptual-table prefix prefix-state)
    (query-prefix-composition-conceptual-table prefix prefix-state)
    (sqp-alphas prefix))
case False
then have
    checked-staged-security-with-query-prefix-conceptual-context-residual-at
    out
    unfolding out-eg
    checked-staged-security-with-query-prefix-conceptual-context-residual-at-def
    by simp
then show ?thesis by simp
next
case True
then have clean:
    ¬ hash-map-output-collision prefix-state
    and trace-low:
        trace-table-low-degree
        (query-prefix-trace-conceptual-table prefix prefix-state)
    and composition-low:
        composition-table-low-degree maxDegree
        (query-prefix-composition-conceptual-table prefix prefix-state)
    and not-all:
        ¬ all-queries-consistent
        (query-prefix-trace-conceptual-table prefix prefix-state)
        (query-prefix-composition-conceptual-table prefix prefix-state)
        (sqp-alphas prefix)
    by blast+
have prefix-hit:
    checked-staged-security-with-query-prefix-dynamic-index-hit
    staged-query-prefix-prefix-authenticated-opening-target out
    by (rule
        checked-staged-security-with-query-prefix-current-prefix-authenticated-hit-at-imp-dynamic-target

```

```

    [OF wf controlled i-bound support hit])
have subset:
  staged-query-prefix-prefix-authenticated-opening-target prefix
    prefix-state  $\subseteq$ 
    staged-query-prefix-conceptual-query-target prefix prefix-state
by (rule
    staged-query-prefix-prefix-authenticated-opening-target-subset-conceptual
    [OF clean trace-low composition-low not-all])
have conceptual-hit:
  checked-staged-security-with-query-prefix-conceptual-target-hit out
using prefix-hit subset
unfolding out-eq
  checked-staged-security-with-query-prefix-conceptual-target-hit-def
  checked-staged-security-with-query-prefix-dynamic-index-hit-def
  checked-staged-query-prefix-dynamic-index-hit-def
by auto
then show ?thesis by simp
qed
qed

lemma checked-staged-security-with-query-prefix-current-prefix-authenticated-hit-at-bound-from-conceptual-residual:
assumes wf: staged-budget-wellformed budgets
and controlled: staged-adversary-controlled budgets A
and i-bound:  $i < \text{rounds}$ 
and residual-bound:
  wp-event
  (checked-staged-security-experiment-with-query-prefix-data-state A i)
  checked-staged-security-with-query-prefix-conceptual-context-residual-at
  adversary-initial-state  $\leq R$ 
shows
  wp-event
  (checked-staged-security-experiment-with-query-prefix-data-state A i)
  (checked-staged-security-with-query-prefix-current-prefix-authenticated-hit-at
  i)
  adversary-initial-state  $\leq$ 
  staged-phase-relation-error size
  (staged-query-search-queries budgets  $i + 1$ ) +
  query-error-bound + R
proof -
let ?M = checked-staged-security-experiment-with-query-prefix-data-state A i
let ?Conceptual =
  checked-staged-security-with-query-prefix-conceptual-target-hit
let ?Residual =
  checked-staged-security-with-query-prefix-conceptual-context-residual-at
have event-le:
  wp-event ?M
  (checked-staged-security-with-query-prefix-current-prefix-authenticated-hit-at
  i)
  adversary-initial-state  $\leq$ 

```

```

    wp-event ?M ( $\lambda$ out. ?Conceptual out  $\vee$  ?Residual out)
    adversary-initial-state
  by (rule wp-event-mono-on-support)
    (rule
      checked-staged-security-with-query-prefix-current-prefix-authenticated-hit-imp-conceptual-or-residual-on-support
      [OF wf controlled i-bound])
  have conceptual-bound:
    wp-event ?M ?Conceptual adversary-initial-state  $\leq$ 
      staged-phase-relation-error size
      (staged-query-search-queries budgets  $i + 1$ ) + query-error-bound
  by (rule
    checked-staged-security-with-query-prefix-conceptual-target-hit-bound
    [OF wf controlled i-bound])
  have union-bound:
    wp-event ?M ( $\lambda$ out. ?Conceptual out  $\vee$  ?Residual out)
    adversary-initial-state  $\leq$ 
    wp-event ?M ?Conceptual adversary-initial-state +
    wp-event ?M ?Residual adversary-initial-state
  by (rule wp-event-union-bound)
  have sum-bound:
    wp-event ?M ?Conceptual adversary-initial-state +
    wp-event ?M ?Residual adversary-initial-state  $\leq$ 
      (staged-phase-relation-error size
        (staged-query-search-queries budgets  $i + 1$ ) + query-error-bound) +
      R
  by (intro add-mono conceptual-bound residual-bound)
  show ?thesis
  by (rule order-trans[OF event-le])
    (rule order-trans[OF union-bound sum-bound])
qed

lemma checked-staged-security-with-query-prefix-current-prefix-authenticated-hit-imp-conceptual-default-or-residual-c
  assumes wf: staged-budget-wellformed budgets
  and controlled: staged-adversary-controlled budgets A
  and i-bound:  $i < \text{rounds}$ 
  and support:
    out  $\in$ 
    set-dist
    (execute
      (checked-staged-security-experiment-with-query-prefix-data-state A i)
      adversary-initial-state)
  and hit:
    checked-staged-security-with-query-prefix-current-prefix-authenticated-hit-at
    i out
  shows
    checked-staged-security-with-query-prefix-conceptual-default-target-hit
    trace-default composition-default out  $\vee$ 
    checked-staged-security-with-query-prefix-conceptual-default-context-residual-at
    trace-default composition-default out

```

```

proof (cases out)
  case None
  then show ?thesis
    using hit
    unfolding
      checked-staged-security-with-query-prefix-current-prefix-authenticated-hit-at-def
      checked-staged-security-with-query-prefix-prefix-authenticated-opening-hit-def
    by simp
next
  case (Some packed)
  then obtain prefix prefix-state raw raw-state data attacker-state result
    final-state where out-eq:
    out =
      Some (((((prefix, prefix-state), raw), raw-state),
        ((data, attacker-state), result)), final-state)
    by (cases packed) (auto split: prod.splits)
  show ?thesis
  proof (cases
     $\neg$  hash-map-output-collision prefix-state  $\wedge$ 
    trace-table-low-degree
    (query-prefix-trace-conceptual-table-with-default prefix
      prefix-state trace-default)  $\wedge$ 
    composition-table-low-degree maxDegree
    (query-prefix-composition-conceptual-table-with-default prefix
      prefix-state composition-default)  $\wedge$ 
     $\neg$  all-queries-consistent
    (query-prefix-trace-conceptual-table-with-default prefix
      prefix-state trace-default)
    (query-prefix-composition-conceptual-table-with-default prefix
      prefix-state composition-default)
    (sqp-alphas prefix))
  case False
  then have
    checked-staged-security-with-query-prefix-conceptual-default-context-residual-at
    trace-default composition-default out
  unfolding out-eq
    checked-staged-security-with-query-prefix-conceptual-default-context-residual-at-def
  by simp
  then show ?thesis by simp
next
  case True
  then have clean:
     $\neg$  hash-map-output-collision prefix-state
  and trace-low:
    trace-table-low-degree
    (query-prefix-trace-conceptual-table-with-default prefix
      prefix-state trace-default)
  and composition-low:
    composition-table-low-degree maxDegree

```

```

    (query-prefix-composition-conceptual-table-with-default prefix
     prefix-state composition-default)
  and not-all:
    ¬ all-queries-consistent
    (query-prefix-trace-conceptual-table-with-default prefix
     prefix-state trace-default)
    (query-prefix-composition-conceptual-table-with-default prefix
     prefix-state composition-default)
    (sqp-alphas prefix)
  by blast+
have prefix-hit:
  checked-staged-security-with-query-prefix-dynamic-index-hit
  staged-query-prefix-prefix-authenticated-opening-target out
  by (rule
    checked-staged-security-with-query-prefix-current-prefix-authenticated-hit-at-imp-dynamic-target
    [OF wf controlled i-bound support hit])
have subset:
  staged-query-prefix-prefix-authenticated-opening-target prefix
  prefix-state  $\subseteq$ 
  staged-query-prefix-conceptual-default-query-target trace-default
  composition-default prefix prefix-state
  by (rule
    staged-query-prefix-prefix-authenticated-opening-target-subset-conceptual-default
    [OF clean trace-low composition-low not-all])
have conceptual-hit:
  checked-staged-security-with-query-prefix-conceptual-default-target-hit
  trace-default composition-default out
  using prefix-hit subset
  unfolding out-eq
  checked-staged-security-with-query-prefix-conceptual-default-target-hit-def
  checked-staged-security-with-query-prefix-dynamic-index-hit-def
  checked-staged-query-prefix-dynamic-index-hit-def
  by auto
then show ?thesis by simp
qed
qed

lemma checked-staged-query-prefix-prefix-authenticated-target-hit-bound-from-conceptual:
  assumes wf: staged-budget-wellformed budgets
  and controlled: staged-adversary-controlled budgets A
  and i-bound:  $i < \text{rounds}$ 
  and conceptual-context:
     $\bigwedge \text{prefix prefix-state.}$ 
    Some ((prefix, prefix-state), prefix-state)  $\in$ 
      set-dist
      (execute (checked-staged-query-prefix-with-state A i)
       adversary-initial-state)  $\implies$ 
    ¬ hash-map-output-collision prefix-state  $\wedge$ 
    trace-table-low-degree

```

$(\text{query-prefix-trace-conceptual-table } \text{prefix } \text{prefix-state}) \wedge$
 $\text{composition-table-low-degree } \text{maxDegree}$
 $(\text{query-prefix-composition-conceptual-table } \text{prefix } \text{prefix-state}) \wedge$
 $\neg \text{all-queries-consistent}$
 $(\text{query-prefix-trace-conceptual-table } \text{prefix } \text{prefix-state})$
 $(\text{query-prefix-composition-conceptual-table } \text{prefix } \text{prefix-state})$
 $(\text{sqp-alphas } \text{prefix})$

shows

$\text{wp-event } (\text{checked-staged-query-prefix-receive-with-state } A \ i)$
 $(\text{checked-staged-query-prefix-dynamic-index-hit}$
 $\text{staged-query-prefix-prefix-authenticated-opening-target})$
 $\text{adversary-initial-state} \leq$
 $\text{staged-phase-relation-error size}$
 $(\text{staged-query-search-queries budgets } i + 1) + \text{query-error-bound}$

proof –

have *raw-bound*: $\text{query-index-raw-preimage-bound}$

by (rule $\text{query-index-raw-preimage-bound-from-sampler-wellformed}$)

have *hit-bound*:

$\text{wp-event } (\text{checked-staged-query-prefix-receive-with-state } A \ i)$
 $(\text{checked-staged-query-prefix-dynamic-index-hit}$
 $\text{staged-query-prefix-prefix-authenticated-opening-target})$
 $\text{adversary-initial-state} \leq$
 $\text{wp-event } (\text{checked-staged-query-prefix-receive-with-state } A \ i)$
 $(\text{checked-staged-query-prefix-dynamic-index-prehit}$
 $\text{staged-query-prefix-prefix-authenticated-opening-target})$
 $\text{adversary-initial-state} + \text{query-error-bound}$

proof (rule
 $\text{checked-staged-query-prefix-dynamic-index-hit-bound-by-prehit-and-query-error}$
 $[\text{OF raw-bound}]$)

fix $\text{prefix } \text{prefix-state}$

assume *support*:

$\text{Some } ((\text{prefix}, \text{prefix-state}), \text{prefix-state}) \in$
 set-dist
 $(\text{execute } (\text{checked-staged-query-prefix-with-state } A \ i)$
 $\text{adversary-initial-state})$

have *ctx*:

$\neg \text{hash-map-output-collision } \text{prefix-state} \wedge$
 $\text{trace-table-low-degree}$
 $(\text{query-prefix-trace-conceptual-table } \text{prefix } \text{prefix-state}) \wedge$
 $\text{composition-table-low-degree } \text{maxDegree}$
 $(\text{query-prefix-composition-conceptual-table } \text{prefix } \text{prefix-state}) \wedge$
 $\neg \text{all-queries-consistent}$
 $(\text{query-prefix-trace-conceptual-table } \text{prefix } \text{prefix-state})$
 $(\text{query-prefix-composition-conceptual-table } \text{prefix } \text{prefix-state})$
 $(\text{sqp-alphas } \text{prefix})$

by (rule $\text{conceptual-context}[\text{OF support}]$)

have *subset*:

$\text{staged-query-prefix-prefix-authenticated-opening-target } \text{prefix}$
 $\text{prefix-state} \subseteq \text{query-sample-space}$


```

    by (rule staged-query-prefix-prefix-authenticated-opening-target-subset)
  have frac:
    nnreal
      (card
        (staged-query-prefix-prefix-authenticated-opening-target prefix
          prefix-state)) /
      nnreal (card query-sample-space) ≤ query-error-bound
  using ctx
  by (intro
      staged-query-prefix-prefix-authenticated-opening-target-fraction-bound-if-conceptual)
    blast+
  show
    staged-query-prefix-prefix-authenticated-opening-target prefix
      prefix-state ⊆ query-sample-space ∧
    nnreal
      (card
        (staged-query-prefix-prefix-authenticated-opening-target prefix
          prefix-state)) /
      nnreal (card query-sample-space) ≤ query-error-bound
  using subset frac by blast
qed
have prehit-bound:
  wp-event (checked-staged-query-prefix-receive-with-state A i)
    (checked-staged-query-prefix-dynamic-index-prehit
      staged-query-prefix-prefix-authenticated-opening-target)
  adversary-initial-state ≤
    staged-phase-relation-error size
    (staged-query-search-queries budgets i + 1)
proof (rule
  checked-staged-query-prefix-dynamic-index-prehit-bound-from-relation-program)
show HashMap adversary-initial-state = fmempty
  by (simp add: adversary-initial-state-def)
show hash-relation-program
  (checked-staged-query-prefix-dynamic-index-prehit-relation A i
    adversary-initial-state
    staged-query-prefix-prefix-authenticated-opening-target)
  size (staged-query-search-queries budgets i + 1)
  (checked-staged-query-prefix-receive-with-state A i)
  by (rule hash-relation-program-checked-staged-query-prefix-receive-with-state
    [OF wf controlled])
    (use i-bound in simp-all,
      rule checked-staged-query-prefix-dynamic-index-prehit-relation-fiber-bound-size)
qed
show ?thesis
  by (rule order-trans[OF hit-bound])
    (intro add-mono prehit-bound order-refl)
qed
end

```

end

theory *Staged-Security-Experiment-Composition-Query-Current-Path*
imports *Staged-Security-Experiment-Composition-Query-Current*
begin

Structured current-query Merkle path-output events.

The broad *checked-staged-security-with-query-prefix-current-path-output-hit-at* event intentionally forgets why the path was relevant. This layer keeps the authenticated-opening root, length, and final-state authentication evidence, so later Merkle/candidate arguments can distinguish ordinary verifier hashing from candidate drift.

context *soundness*
begin

definition *checked-staged-security-with-query-prefix-transcript-hit*
where

checked-staged-security-with-query-prefix-transcript-hit out $\longleftrightarrow$
 (case out of
 None $\Rightarrow$ *False*
 | *Some* ((((((*prefix*, *prefix-state*), *raw*), *raw-state*),
 ((*data*, *attacker-state*), *result*)), *final-state*) $\Rightarrow$
 hash-map-output-values
 (*verifier-state-from-adversary attacker-state*
 (*staged-proof-transcript data*)) $\cap$
 set (*staged-proof-transcript data*) $\neq \{\}$ $\vee$
 hash-map-new-output-hit (*set* (*staged-proof-transcript data*))
 (*verifier-state-from-adversary attacker-state*
 (*staged-proof-transcript data*))
 final-state)

definition *checked-staged-security-with-query-prefix-current-trace-path-output-hit-at*
where

checked-staged-security-with-query-prefix-current-trace-path-output-hit-at
i out $\longleftrightarrow$
 (case out of
 None $\Rightarrow$ *False*
 | *Some* ((((((*prefix*, *prefix-state*), *raw*), *raw-state*),
 ((*data*, *attacker-state*), *result*)), *final-state*) $\Rightarrow$
 ($\exists$ *opn* :: 'f *authenticated-opening*.
 opening-root opn = *staged-trace-root data* $\wedge$
 opening-length opn = *scale* * *clength* $\wedge$
 authenticated-opening-in final-state opn $\wedge$
 hash-map-new-output-hit
 (*merkle-path-target-roots final-state*
 (*staged-trace-root data*)
 (*scale* * *clength*)
 (*opening-index opn*))

(opening-value opn)
 (opening-path opn))
 prefix-state final-state))

definition *checked-staged-security-with-query-prefix-current-composition-path-output-hit-at*
where

checked-staged-security-with-query-prefix-current-composition-path-output-hit-at
i out $\longleftrightarrow$
 (case out of
 None $\Rightarrow$ False
 | Some (((((prefix, prefix-state), raw), raw-state),
 ((data, attacker-state), result)), final-state) $\Rightarrow$
 staged-composition-fri-roots data $\neq \square$ $\wedge$
 ($\exists$ opn :: 'f authenticated-opening.
 opening-root opn = hd (staged-composition-fri-roots data) $\wedge$
 opening-length opn = scale * clength $\wedge$
 authenticated-opening-in final-state opn $\wedge$
 hash-map-new-output-hit
 (merkle-path-target-roots final-state
 (hd (staged-composition-fri-roots data))
 (scale * clength)
 (opening-index opn)
 (opening-value opn)
 (opening-path opn))
 prefix-state final-state))

lemma *checked-staged-security-with-query-prefix-structured-path-hit-imp-transcript-hit-on-support:*

assumes *i-bound*: $i < \text{rounds}$

and *support*:

out $\in$
 set-dist
 (execute
 (checked-staged-security-experiment-with-query-prefix-data-state A i)
 adversary-initial-state)

and *hit*:

checked-staged-security-with-query-prefix-current-trace-path-output-hit-at
i out $\vee$
checked-staged-security-with-query-prefix-current-composition-path-output-hit-at
i out

shows *checked-staged-security-with-query-prefix-transcript-hit out*

proof (cases out)

case None

then show ?thesis

using *hit*

unfolding

checked-staged-security-with-query-prefix-current-trace-path-output-hit-at-def
checked-staged-security-with-query-prefix-current-composition-path-output-hit-at-def

by *simp*

next

```

case (Some packed)
then obtain prefix prefix-state raw raw-state data attacker-state result
  final-state where out-eq:
  out =
    Some (((((prefix, prefix-state), raw), raw-state),
      ((data, attacker-state), result)), final-state)
  by (cases packed) (auto split: prod.splits)
have mapped-support:
  Some (((data, attacker-state), result), final-state) ∈
    set-dist
      (execute
        (checked-staged-security-experiment-with-query-prefix-data-state A i >>=
          (λpacked. return (snd packed)))
        adversary-initial-state)
  unfolding out-eq
  by (rule set-dist-bindI[OF support[unfolded out-eq]]) simp
have projection:
  checked-staged-security-experiment-with-query-prefix-data-state A i >>=
    (λpacked. return (snd packed)) =
    checked-staged-security-experiment-with-data-state A
  by (rule checked-staged-security-experiment-with-query-prefix-data-state-projection
    [OF i-bound])
have data-support:
  Some (((data, attacker-state), result), final-state) ∈
    set-dist
      (execute (checked-staged-security-experiment-with-data-state A)
        adversary-initial-state)
  using mapped-support unfolding projection .
let ?s =
  verifier-state-from-adversary attacker-state
    (staged-proof-transcript data)
from checked-staged-security-experiment-with-data-state-outcomeE
  [OF data-support]
have verifier:
  Some (result, final-state) ∈ set-dist (execute verify-monad ?s)
  by blast
have transcript-split:
  hash-map-output-values ?s ∩ set (staged-proof-transcript data)
    ≠ {} ∨
  hash-map-new-output-hit (set (staged-proof-transcript data))
    ?s final-state
  by (rule verify-monad-trace-root-target-preexisting-or-new[OF verifier])
    simp
show ?thesis
  unfolding out-eq checked-staged-security-with-query-prefix-transcript-hit-def
  using transcript-split by simp
qed

```

lemma checked-staged-security-with-query-prefix-transcript-hit-bound-from-data-state:

```

assumes i-bound:  $i < \text{rounds}$ 
and pre-bound:
  wp-event (checked-staged-security-experiment-with-data-state A)
    staged-security-with-data-state-transcript-pre-hit
    adversary-initial-state  $\leq P$ 
and new-bound:
  wp-event (checked-staged-security-experiment-with-data-state A)
    staged-security-with-data-state-transcript-new-hit
    adversary-initial-state  $\leq R$ 
shows
  wp-event
    (checked-staged-security-experiment-with-query-prefix-data-state A i)
    checked-staged-security-with-query-prefix-transcript-hit
    adversary-initial-state  $\leq P + R$ 
proof –
  let ?E =
     $\lambda out.$ 
      staged-security-with-data-state-transcript-pre-hit out  $\vee$ 
      staged-security-with-data-state-transcript-new-hit out
  let ?projected =
     $\lambda out.$  case out of
      None  $\Rightarrow ?E \text{ None}$ 
      | Some (packed, t)  $\Rightarrow ?E (\text{Some (snd packed, t)})$ 
  have projected-eq:
    ?projected = checked-staged-security-with-query-prefix-transcript-hit
  unfolding
    checked-staged-security-with-query-prefix-transcript-hit-def
    staged-security-with-data-state-transcript-pre-hit-def
    staged-security-with-data-state-transcript-new-hit-def
  by (rule ext) (auto split: option.splits prod.splits)
  have projection:
    wp-event (checked-staged-security-experiment-with-data-state A)
      ?E
      adversary-initial-state =
        wp-event
          (checked-staged-security-experiment-with-query-prefix-data-state A i)
          checked-staged-security-with-query-prefix-transcript-hit
          adversary-initial-state
  using
    checked-staged-security-experiment-with-query-prefix-data-state-projection-event
    [OF i-bound, of A ?E]
  unfolding projected-eq .
  have union-bound:
    wp-event (checked-staged-security-experiment-with-data-state A)
      ?E
      adversary-initial-state  $\leq$ 
      wp-event (checked-staged-security-experiment-with-data-state A)
        staged-security-with-data-state-transcript-pre-hit
        adversary-initial-state +

```

```

    wp-event (checked-staged-security-experiment-with-data-state A)
      staged-security-with-data-state-transcript-new-hit
      adversary-initial-state
  by (rule wp-event-union-bound)
have wp-event (checked-staged-security-experiment-with-data-state A)
  ?E
  adversary-initial-state  $\leq P + R$ 
by (rule order-trans[OF union-bound])
  (intro add-mono pre-bound new-bound)
then show ?thesis
  unfolding projection .
qed

lemma checked-staged-security-with-query-prefix-current-trace-path-output-hit-at-bound-from-transcript:
  assumes i-bound:  $i < \text{rounds}$ 
  and pre-bound:
    wp-event (checked-staged-security-experiment-with-data-state A)
      staged-security-with-data-state-transcript-pre-hit
      adversary-initial-state  $\leq P$ 
  and new-bound:
    wp-event (checked-staged-security-experiment-with-data-state A)
      staged-security-with-data-state-transcript-new-hit
      adversary-initial-state  $\leq R$ 
  shows
    wp-event
      (checked-staged-security-experiment-with-query-prefix-data-state A i)
      (checked-staged-security-with-query-prefix-current-trace-path-output-hit-at i)
      adversary-initial-state  $\leq P + R$ 
proof -
  have event-le:
    wp-event
      (checked-staged-security-experiment-with-query-prefix-data-state A i)
      (checked-staged-security-with-query-prefix-current-trace-path-output-hit-at i)
      adversary-initial-state  $\leq$ 
    wp-event
      (checked-staged-security-experiment-with-query-prefix-data-state A i)
      checked-staged-security-with-query-prefix-transcript-hit
      adversary-initial-state
  proof (rule wp-event-mono-on-support)
    fix out
    assume support:
      out  $\in$ 
      set-dist
      (execute
        (checked-staged-security-experiment-with-query-prefix-data-state A i)
        adversary-initial-state)
    and hit:
      checked-staged-security-with-query-prefix-current-trace-path-output-hit-at
      i out
  qed

```

```

show checked-staged-security-with-query-prefix-transcript-hit out
  by (rule
    checked-staged-security-with-query-prefix-structured-path-hit-imp-transcript-hit-on-support
    [OF i-bound support])
    (use hit in simp)
qed
have transcript-bound:
  wp-event
    (checked-staged-security-experiment-with-query-prefix-data-state A i)
    checked-staged-security-with-query-prefix-transcript-hit
    adversary-initial-state  $\leq P + R$ 
  by (rule
    checked-staged-security-with-query-prefix-transcript-hit-bound-from-data-state
    [OF i-bound pre-bound new-bound])
show ?thesis
  by (rule order-trans[OF event-le transcript-bound])
qed

lemma checked-staged-security-with-query-prefix-current-composition-path-output-hit-at-bound-from-transcript:
assumes i-bound:  $i < \text{rounds}$ 
and pre-bound:
  wp-event (checked-staged-security-experiment-with-data-state A)
  staged-security-with-data-state-transcript-pre-hit
  adversary-initial-state  $\leq P$ 
and new-bound:
  wp-event (checked-staged-security-experiment-with-data-state A)
  staged-security-with-data-state-transcript-new-hit
  adversary-initial-state  $\leq R$ 
shows
  wp-event
    (checked-staged-security-experiment-with-query-prefix-data-state A i)
    (checked-staged-security-with-query-prefix-current-composition-path-output-hit-at i)
    adversary-initial-state  $\leq P + R$ 
proof –
have event-le:
  wp-event
    (checked-staged-security-experiment-with-query-prefix-data-state A i)
    (checked-staged-security-with-query-prefix-current-composition-path-output-hit-at i)
    adversary-initial-state  $\leq$ 
  wp-event
    (checked-staged-security-experiment-with-query-prefix-data-state A i)
    checked-staged-security-with-query-prefix-transcript-hit
    adversary-initial-state
proof (rule wp-event-mono-on-support)
fix out
assume support:
  out  $\in$ 
    set-dist
    (execute

```

```

      (checked-staged-security-experiment-with-query-prefix-data-state A i)
      adversary-initial-state)
  and hit:
    checked-staged-security-with-query-prefix-current-composition-path-output-hit-at
    i out
  show checked-staged-security-with-query-prefix-transcript-hit out
  by (rule
    checked-staged-security-with-query-prefix-structured-path-hit-imp-transcript-hit-on-support
    [OF i-bound support])
    (use hit in simp)
qed
have transcript-bound:
  wp-event
  (checked-staged-security-experiment-with-query-prefix-data-state A i)
  checked-staged-security-with-query-prefix-transcript-hit
  adversary-initial-state  $\leq P + R$ 
  by (rule
    checked-staged-security-with-query-prefix-transcript-hit-bound-from-data-state
    [OF i-bound pre-bound new-bound])
  show ?thesis
  by (rule order-trans[OF event-le transcript-bound])
qed

lemma checked-staged-security-with-query-prefix-current-query-partial-opening-hit-at-imp-prefix-or-structured-path-or-
  assumes wf: staged-budget-wellformed budgets
  and controlled: staged-adversary-controlled budgets A
  and i-bound:  $i < \text{rounds}$ 
  and support:
    out  $\in$ 
    set-dist
    (execute
      (checked-staged-security-experiment-with-query-prefix-data-state A i)
      adversary-initial-state)
  and hit:
    checked-staged-security-with-query-prefix-current-query-partial-opening-hit-at
    i out
  shows
    checked-staged-security-with-query-prefix-current-prefix-authenticated-hit-at
    i out  $\vee$ 
    checked-staged-security-with-query-prefix-current-trace-path-output-hit-at
    i out  $\vee$ 
    checked-staged-security-with-query-prefix-current-composition-path-output-hit-at
    i out
proof (cases out)
case None
then show ?thesis
using hit
unfolding checked-staged-security-with-query-prefix-current-query-partial-opening-hit-at-def
checked-staged-security-with-query-prefix-authenticated-opening-hit-def

```



```

    by simp
next
case (Some packed)
then obtain prefix prefix-state raw raw-state data attacker-state result
    final-state where out-eq:
    out =
        Some (((((prefix, prefix-state), raw), raw-state),
            ((data, attacker-state), result)), final-state)
    by (cases packed) (auto split: prod.splits)
from hit obtain trace-openings composition-openings where current:
    checked-staged-security-with-query-prefix-authenticated-opening-hit
    trace-openings composition-openings i out
    unfolding checked-staged-security-with-query-prefix-current-query-partial-opening-hit-at-def
    by blast
have ext: prefix-state  $\leq$  final-state
    using support unfolding out-eq
    by (rule
        checked-staged-security-experiment-with-query-prefix-data-state-prefix-extends-final
        [OF wf controlled i-bound])
have comp-nonempty: staged-composition-fri-roots data  $\neq$  []
    and trace-table:
        partial-authenticated-table (staged-trace-root data)
        (scale * clength) trace-openings final-state
    and comp-table:
        partial-authenticated-table (hd (staged-composition-fri-roots data))
        (scale * clength) composition-openings final-state
    and consistent:
        partial-query-openings-consistent trace-openings composition-openings
        (staged-alphas data) (index (to-nat raw))
    using current unfolding out-eq
        checked-staged-security-with-query-prefix-authenticated-opening-hit-def
    by simp-all
have trace-pullback:
    partial-authenticated-table (staged-trace-root data)
    (scale * clength) trace-openings prefix-state  $\vee$ 
    ( $\exists$  opn  $\in$  set trace-openings.
        hash-map-new-output-hit
        (merkle-path-target-roots final-state (staged-trace-root data)
            (scale * clength)
            (opening-index opn)
            (opening-value opn)
            (opening-path opn)) prefix-state final-state)
    by (rule partial-authenticated-table-pullback-or-new-output-hit
        [OF ext trace-table])
then show ?thesis
proof
    assume trace-prefix:
        partial-authenticated-table (staged-trace-root data)
        (scale * clength) trace-openings prefix-state

```

```

have comp-pullback:
  partial-authenticated-table (hd (staged-composition-fri-roots data))
    (scale * clength) composition-openings prefix-state  $\vee$ 
  ( $\exists$  opn  $\in$  set composition-openings.
    hash-map-new-output-hit
      (merkle-path-target-roots final-state
        (hd (staged-composition-fri-roots data))
          (scale * clength)
            (opening-index opn)
              (opening-value opn)
                (opening-path opn)) prefix-state final-state)
  by (rule partial-authenticated-table-pullback-or-new-output-hit
    [OF ext comp-table])
then show ?thesis
proof
  assume comp-prefix:
    partial-authenticated-table (hd (staged-composition-fri-roots data))
      (scale * clength) composition-openings prefix-state
  have
    checked-staged-security-with-query-prefix-current-prefix-authenticated-hit-at
      i out
  unfolding out-eq
    checked-staged-security-with-query-prefix-current-prefix-authenticated-hit-at-def
    checked-staged-security-with-query-prefix-prefix-authenticated-opening-hit-def
  using comp-nonempty trace-prefix comp-prefix consistent by auto
  then show ?thesis by simp
next
  assume comp-path:
     $\exists$  opn  $\in$  set composition-openings.
      hash-map-new-output-hit
        (merkle-path-target-roots final-state
          (hd (staged-composition-fri-roots data))
            (scale * clength)
              (opening-index opn)
                (opening-value opn)
                  (opening-path opn)) prefix-state final-state)
  then obtain opn where opn-in: opn  $\in$  set composition-openings
  and path-hit:
    hash-map-new-output-hit
      (merkle-path-target-roots final-state
        (hd (staged-composition-fri-roots data))
          (scale * clength)
            (opening-index opn)
              (opening-value opn)
                (opening-path opn)) prefix-state final-state)
  by blast
  have auth: authenticated-opening-in final-state opn
  using comp-table opn-in
  unfolding partial-authenticated-table-def by blast

```

```

have root-eq:
  opening-root opn = hd (staged-composition-fri-roots data)
  using comp-table opn-in
  unfolding partial-authenticated-table-def by blast
have len-eq: opening-length opn = scale * clength
  using comp-table opn-in
  unfolding partial-authenticated-table-def by blast
have
  checked-staged-security-with-query-prefix-current-composition-path-output-hit-at
    i out
  unfolding out-eq
    checked-staged-security-with-query-prefix-current-composition-path-output-hit-at-def
  using comp-nonempty auth root-eq len-eq path-hit by auto
then show ?thesis by simp
qed
next
assume trace-path:
   $\exists$  opn  $\in$  set trace-openings.
    hash-map-new-output-hit
      (merkle-path-target-roots final-state (staged-trace-root data)
        (scale * clength)
        (opening-index opn)
        (opening-value opn)
        (opening-path opn)) prefix-state final-state
then obtain opn where opn-in: opn  $\in$  set trace-openings
and path-hit:
  hash-map-new-output-hit
    (merkle-path-target-roots final-state (staged-trace-root data)
      (scale * clength)
      (opening-index opn)
      (opening-value opn)
      (opening-path opn)) prefix-state final-state
  by blast
have auth: authenticated-opening-in final-state opn
  using trace-table opn-in
  unfolding partial-authenticated-table-def by blast
have root-eq: opening-root opn = staged-trace-root data
  using trace-table opn-in
  unfolding partial-authenticated-table-def by blast
have len-eq: opening-length opn = scale * clength
  using trace-table opn-in
  unfolding partial-authenticated-table-def by blast
have
  checked-staged-security-with-query-prefix-current-trace-path-output-hit-at
    i out
  unfolding out-eq
    checked-staged-security-with-query-prefix-current-trace-path-output-hit-at-def
  using auth root-eq len-eq path-hit by auto
then show ?thesis by simp

```

qed
qed

lemma *checked-staged-security-with-query-prefix-authenticated-opening-hit-imp-candidate-pair-or-structured-path-on-*

assumes *wf: staged-budget-wellformed budgets*

and *controlled: staged-adversary-controlled budgets A*

and *i-bound: i < rounds*

and *support:*

*Some (((((prefix, prefix-state), raw), raw-state),
((data, attacker-state), result)), final-state) ∈
set-dist
(execute
(checked-staged-security-experiment-with-query-prefix-data-state A i)
adversary-initial-state)*

and *hit:*

*checked-staged-security-with-query-prefix-authenticated-opening-hit
trace-openings composition-openings i
(Some (((((prefix, prefix-state), raw), raw-state),
((data, attacker-state), result)), final-state))*

and *trace-candidate:*

*partial-trace-table-candidate trace-table
((replicate rounds []) [i := trace-openings])*

and *comp-candidate:*

*partial-composition-table-candidate composition-table
((replicate rounds []) [i := composition-openings])*

and *trace-low: trace-table-low-degree trace-table*

and *comp-low: composition-table-low-degree maxDegree composition-table*

and *not-all:*

*¬ all-queries-consistent trace-table composition-table
(sqp-alphas prefix)*

shows

*checked-staged-security-with-query-prefix-candidate-pair-from-prefix-hit
trace-table composition-table
(Some (((((prefix, prefix-state), raw), raw-state),
((data, attacker-state), result)), final-state)) ∨
checked-staged-security-with-query-prefix-current-trace-path-output-hit-at
i
(Some (((((prefix, prefix-state), raw), raw-state),
((data, attacker-state), result)), final-state)) ∨
checked-staged-security-with-query-prefix-current-composition-path-output-hit-at
i
(Some (((((prefix, prefix-state), raw), raw-state),
((data, attacker-state), result)), final-state))*

proof –

let *?out =*

*Some (((((prefix, prefix-state), raw), raw-state),
((data, attacker-state), result)), final-state)*

have *ext: prefix-state ≤ final-state*

by *(rule*

```

    checked-staged-security-experiment-with-query-prefix-data-state-prefix-extends-final
    [OF wf controlled i-bound support])
have alphas-eq: staged-alphas data = sqp-alphas prefix
  by (rule checked-staged-security-with-query-prefix-data-state-alphas-eq
    [OF support])
have comp-nonempty: staged-composition-fri-roots data  $\neq$  []
and trace-table-final:
  partial-authenticated-table (staged-trace-root data)
  (scale * clength) trace-openings final-state
and comp-table-final:
  partial-authenticated-table (hd (staged-composition-fri-roots data))
  (scale * clength) composition-openings final-state
and consistent:
  partial-query-openings-consistent trace-openings composition-openings
  (staged-alphas data) (index (to-nat raw))
using hit
unfolding checked-staged-security-with-query-prefix-authenticated-opening-hit-def
by simp-all
have trace-pullback:
  partial-authenticated-table (staged-trace-root data)
  (scale * clength) trace-openings prefix-state  $\vee$ 
  ( $\exists$  opn  $\in$  set trace-openings.
    hash-map-new-output-hit
    (merkle-path-target-roots final-state (staged-trace-root data)
      (scale * clength) (opening-index opn) (opening-value opn)
      (opening-path opn)) prefix-state final-state)
  by (rule partial-authenticated-table-pullback-or-new-output-hit
    [OF ext trace-table-final])
then show ?thesis
proof
  assume trace-prefix:
    partial-authenticated-table (staged-trace-root data)
    (scale * clength) trace-openings prefix-state
  have comp-pullback:
    partial-authenticated-table (hd (staged-composition-fri-roots data))
    (scale * clength) composition-openings prefix-state  $\vee$ 
    ( $\exists$  opn  $\in$  set composition-openings.
      hash-map-new-output-hit
      (merkle-path-target-roots final-state
        (hd (staged-composition-fri-roots data))
        (scale * clength) (opening-index opn) (opening-value opn)
        (opening-path opn)) prefix-state final-state)
    by (rule partial-authenticated-table-pullback-or-new-output-hit
      [OF ext comp-table-final])
  then show ?thesis
proof
  assume comp-prefix:
    partial-authenticated-table (hd (staged-composition-fri-roots data))
    (scale * clength) composition-openings prefix-state

```

```

have opening-target:
  index (to-nat raw) ∈
    staged-query-prefix-candidate-opening-query-target-from-prefix
      ((replicate rounds []) [i := trace-openings])
      ((replicate rounds []) [i := composition-openings])
    i prefix prefix-state
unfolding staged-query-prefix-candidate-opening-query-target-from-prefix-def
by (rule partial-query-success-indices-atI,
      rule partial-query-round-consistent-single-roundI
        [OF i-bound])
    (use consistent alphas-eq in simp)
have opening-subset:
  staged-query-prefix-candidate-opening-query-target-from-prefix
    ((replicate rounds []) [i := trace-openings])
    ((replicate rounds []) [i := composition-openings])
    i prefix prefix-state ⊆
  staged-query-prefix-candidate-pair-query-target-from-prefix
    trace-table composition-table prefix prefix-state
by (rule
    staged-query-prefix-candidate-opening-query-target-from-prefix-subset-candidate-pair-from-prefix
      [OF trace-candidate comp-candidate trace-low comp-low not-all])
have pair-hit:
  checked-staged-security-with-query-prefix-candidate-pair-from-prefix-hit
    trace-table composition-table ?out
unfolding
  checked-staged-security-with-query-prefix-candidate-pair-from-prefix-hit-def
  checked-staged-security-with-query-prefix-dynamic-index-hit-def
  checked-staged-query-prefix-dynamic-index-hit-def
using opening-target opening-subset by auto
then show ?thesis by simp
next
assume comp-path:
  ∃ opn ∈ set composition-openings.
    hash-map-new-output-hit
      (merkle-path-target-roots final-state
        (hd (staged-composition-fri-roots data))
        (scale * clength) (opening-index opn) (opening-value opn)
        (opening-path opn)) prefix-state final-state
then obtain opn where opn-in: opn ∈ set composition-openings
and path-hit:
  hash-map-new-output-hit
    (merkle-path-target-roots final-state
      (hd (staged-composition-fri-roots data))
      (scale * clength) (opening-index opn) (opening-value opn)
      (opening-path opn)) prefix-state final-state
by blast
have auth: authenticated-opening-in final-state opn
using comp-table-final opn-in
unfolding partial-authenticated-table-def by blast

```

```

have root-eq:
  opening-root opn = hd (staged-composition-fri-roots data)
  using comp-table-final opn-in
  unfolding partial-authenticated-table-def by blast
have len-eq: opening-length opn = scale * clength
  using comp-table-final opn-in
  unfolding partial-authenticated-table-def by blast
have
  checked-staged-security-with-query-prefix-current-composition-path-output-hit-at
    i ?out
  unfolding
    checked-staged-security-with-query-prefix-current-composition-path-output-hit-at-def
  using comp-nonempty auth root-eq len-eq path-hit by auto
then show ?thesis by simp
qed
next
assume trace-path:
   $\exists opn \in \text{set trace-openings.}$ 
  hash-map-new-output-hit
    (merkle-path-target-roots final-state (staged-trace-root data)
      (scale * clength) (opening-index opn) (opening-value opn)
      (opening-path opn)) prefix-state final-state
then obtain opn where opn-in: opn ∈ set trace-openings
and path-hit:
  hash-map-new-output-hit
    (merkle-path-target-roots final-state (staged-trace-root data)
      (scale * clength) (opening-index opn) (opening-value opn)
      (opening-path opn)) prefix-state final-state
  by blast
have auth: authenticated-opening-in final-state opn
  using trace-table-final opn-in
  unfolding partial-authenticated-table-def by blast
have root-eq: opening-root opn = staged-trace-root data
  using trace-table-final opn-in
  unfolding partial-authenticated-table-def by blast
have len-eq: opening-length opn = scale * clength
  using trace-table-final opn-in
  unfolding partial-authenticated-table-def by blast
have
  checked-staged-security-with-query-prefix-current-trace-path-output-hit-at
    i ?out
  unfolding
    checked-staged-security-with-query-prefix-current-trace-path-output-hit-at-def
  using auth root-eq len-eq path-hit by auto
then show ?thesis by simp
qed
qed

```

definition *staged-query-prefix-inconsistent-candidate-pair-query-target*

where

staged-query-prefix-inconsistent-candidate-pair-query-target
trace-table composition-table prefix prefix-state =
(if all-queries-consistent trace-table composition-table
(sqp-alphas prefix)
then {}
else staged-query-prefix-candidate-pair-query-target-from-prefix
trace-table composition-table prefix prefix-state)

lemma *staged-query-prefix-inconsistent-candidate-pair-query-target-bound:*
assumes *trace-low: trace-table-low-degree trace-table*
and *comp-low:*
composition-table-low-degree maxDegree composition-table
shows
staged-query-prefix-inconsistent-candidate-pair-query-target
trace-table composition-table prefix prefix-state
 $\subseteq$ *query-sample-space* $\wedge$
nnreal
(card
(staged-query-prefix-inconsistent-candidate-pair-query-target
trace-table composition-table prefix prefix-state)) /
nnreal (card query-sample-space) $\leq$ query-error-bound

proof *(cases*
all-queries-consistent trace-table composition-table (sqp-alphas prefix))
case *True*
then show *?thesis*
unfolding *staged-query-prefix-inconsistent-candidate-pair-query-target-def*
by *simp*

next
case *False*
have *subset:*
staged-query-prefix-candidate-pair-query-target-from-prefix trace-table
composition-table prefix prefix-state $\subseteq$ query-sample-space
by *(rule staged-query-prefix-candidate-pair-query-target-from-prefix-subset)*
have *frac:*
nnreal
(card
(staged-query-prefix-candidate-pair-query-target-from-prefix
trace-table composition-table prefix prefix-state)) /
nnreal (card query-sample-space) $\leq$ query-error-bound
by *(rule*
staged-query-prefix-candidate-pair-query-target-from-prefix-fraction-bound
[OF trace-low comp-low False])
show *?thesis*
unfolding *staged-query-prefix-inconsistent-candidate-pair-query-target-def*
using *False subset frac* **by** *simp*

qed

lemma *checked-staged-security-with-query-prefix-inconsistent-candidate-pair-hit-bound:*

assumes *wf*: staged-budget-wellformed budgets
and *controlled*: staged-adversary-controlled budgets *A*
and *i-bound*: $i < \text{rounds}$
and *trace-low*: trace-table-low-degree trace-table
and *comp-low*:
 composition-table-low-degree maxDegree composition-table
shows
 wp-event
 (*checked-staged-security-experiment-with-query-prefix-data-state* *A i*)
 (*checked-staged-security-with-query-prefix-dynamic-index-hit*
 (*staged-query-prefix-inconsistent-candidate-pair-query-target*
 trace-table composition-table))
 adversary-initial-state $\leq$
 staged-phase-relation-error size
 (*staged-query-search-queries budgets i + 1*) + *query-error-bound*
by (*rule*
 checked-staged-security-with-query-prefix-dynamic-index-hit-bound-from-fraction
 [*OF wf controlled i-bound*])
 (*use*
 staged-query-prefix-inconsistent-candidate-pair-query-target-bound
 [*OF trace-low comp-low*] **in** *blast*)

lemma *checked-staged-security-with-query-prefix-candidate-pair-from-prefix-hit-imp-inconsistent-target*:

assumes *hit*:
 checked-staged-security-with-query-prefix-candidate-pair-from-prefix-hit
 trace-table composition-table
 (*Some* ((((*prefix*, *prefix-state*), *raw*), *raw-state*),
 ((*data*, *attacker-state*), *result*)), *final-state*))
and *not-all*:
 $\neg$ *all-queries-consistent trace-table composition-table*
 (*sqp-alphas prefix*)

shows
 checked-staged-security-with-query-prefix-dynamic-index-hit
 (*staged-query-prefix-inconsistent-candidate-pair-query-target*
 trace-table composition-table)
 (*Some* ((((*prefix*, *prefix-state*), *raw*), *raw-state*),
 ((*data*, *attacker-state*), *result*)), *final-state*))

using *hit not-all*

unfolding

checked-staged-security-with-query-prefix-candidate-pair-from-prefix-hit-def
 checked-staged-security-with-query-prefix-dynamic-index-hit-def
 checked-staged-query-prefix-dynamic-index-hit-def
 staged-query-prefix-inconsistent-candidate-pair-query-target-def

by *simp*

lemma *checked-staged-security-with-query-prefix-authenticated-opening-hit-bound-from-candidate-pair-and-structured*

assumes *wf*: staged-budget-wellformed budgets
and *controlled*: staged-adversary-controlled budgets *A*
and *i-bound*: $i < \text{rounds}$

and *trace-candidate*:
partial-trace-table-candidate *trace-table*
 $((\text{replicate rounds } []) [i := \text{trace-openings}])$
and *comp-candidate*:
partial-composition-table-candidate *composition-table*
 $((\text{replicate rounds } []) [i := \text{composition-openings}])$
and *trace-low*: *trace-table-low-degree* *trace-table*
and *comp-low*: *composition-table-low-degree* *maxDegree* *composition-table*
and *not-all*:
 $\bigwedge \text{prefix prefix-state raw raw-state data attacker-state result}$
 final-state.
 $\text{Some } (((((\text{prefix}, \text{prefix-state}), \text{raw}), \text{raw-state}),$
 $((\text{data}, \text{attacker-state}), \text{result})), \text{final-state}) \in$
 set-dist
 $(\text{execute}$
 $(\text{checked-staged-security-experiment-with-query-prefix-data-state}$
 $A \ i)$
 $\text{adversary-initial-state}) \implies$
 $\neg \text{all-queries-consistent trace-table composition-table}$
 $(\text{sqp-alphas prefix})$
and *pair-bound*:
 wp-event
 $(\text{checked-staged-security-experiment-with-query-prefix-data-state } A \ i)$
 $(\text{checked-staged-security-with-query-prefix-candidate-pair-from-prefix-hit}$
 $\text{trace-table composition-table})$
 $\text{adversary-initial-state} \leq P$
and *trace-path-bound*:
 wp-event
 $(\text{checked-staged-security-experiment-with-query-prefix-data-state } A \ i)$
 $(\text{checked-staged-security-with-query-prefix-current-trace-path-output-hit-at}$
 $i)$
 $\text{adversary-initial-state} \leq T$
and *composition-path-bound*:
 wp-event
 $(\text{checked-staged-security-experiment-with-query-prefix-data-state } A \ i)$
 $(\text{checked-staged-security-with-query-prefix-current-composition-path-output-hit-at}$
 $i)$
 $\text{adversary-initial-state} \leq C$
shows
 wp-event
 $(\text{checked-staged-security-experiment-with-query-prefix-data-state } A \ i)$
 $(\text{checked-staged-security-with-query-prefix-authenticated-opening-hit}$
 $\text{trace-openings composition-openings } i)$
 $\text{adversary-initial-state} \leq P + T + C$
proof –
let $?M =$
 $\text{checked-staged-security-experiment-with-query-prefix-data-state } A \ i$
let $?Pair =$
 $\text{checked-staged-security-with-query-prefix-candidate-pair-from-prefix-hit}$

```

    trace-table composition-table
let ?Trace =
    checked-staged-security-with-query-prefix-current-trace-path-output-hit-at
    i
let ?Comp =
    checked-staged-security-with-query-prefix-current-composition-path-output-hit-at
    i
have event-le:
    wp-event ?M
    (checked-staged-security-with-query-prefix-authenticated-opening-hit
     trace-openings composition-openings i)
    adversary-initial-state ≤
    wp-event ?M (λout. ?Pair out ∨ ?Trace out ∨ ?Comp out)
    adversary-initial-state
proof (rule wp-event-mono-on-support)
fix out
assume support:
    out ∈ set-dist (execute ?M adversary-initial-state)
and hit:
    checked-staged-security-with-query-prefix-authenticated-opening-hit
    trace-openings composition-openings i out
show ?Pair out ∨ ?Trace out ∨ ?Comp out
proof (cases out)
case None
then show ?thesis
    using hit
    unfolding
        checked-staged-security-with-query-prefix-authenticated-opening-hit-def
    by simp
next
case (Some packed)
then obtain prefix prefix-state raw raw-state data attacker-state result
    final-state where out-eq:
    out =
        Some (((((prefix, prefix-state), raw), raw-state),
                ((data, attacker-state), result)), final-state)
    by (cases packed) (auto split: prod.splits)
have support-some:
    Some (((((prefix, prefix-state), raw), raw-state),
            ((data, attacker-state), result)), final-state) ∈
    set-dist (execute ?M adversary-initial-state)
    using support out-eq by simp
show ?thesis
    unfolding out-eq
    by (rule
        checked-staged-security-with-query-prefix-authenticated-opening-hit-imp-candidate-pair-or-structured-path-on-s
        [OF wf controlled i-bound support-some])
    (use hit[unfolded out-eq] trace-candidate comp-candidate
     trace-low comp-low not-all[OF support-some] in simp-all)

```

```

qed
qed
have union-le:
  wp-event ?M ( $\lambda out. ?Pair\ out \vee ?Trace\ out \vee ?Comp\ out$ )
    adversary-initial-state  $\leq$ 
    wp-event ?M ?Pair adversary-initial-state +
    (wp-event ?M ?Trace adversary-initial-state +
    wp-event ?M ?Comp adversary-initial-state)
proof -
  have
    wp-event ?M ( $\lambda out. ?Pair\ out \vee ?Trace\ out \vee ?Comp\ out$ )
      adversary-initial-state  $\leq$ 
      wp-event ?M ?Pair adversary-initial-state +
      wp-event ?M ( $\lambda out. ?Trace\ out \vee ?Comp\ out$ )
        adversary-initial-state
    by (rule wp-event-union-bound)
  also have
    ...  $\leq$ 
    wp-event ?M ?Pair adversary-initial-state +
    (wp-event ?M ?Trace adversary-initial-state +
    wp-event ?M ?Comp adversary-initial-state)
    by (intro add-mono order-refl wp-event-union-bound)
  finally show ?thesis .
qed
have sum-bound:
  wp-event ?M ?Pair adversary-initial-state +
  (wp-event ?M ?Trace adversary-initial-state +
  wp-event ?M ?Comp adversary-initial-state)  $\leq P + T + C$ 
proof -
  have path-sum:
    wp-event ?M ?Trace adversary-initial-state +
    wp-event ?M ?Comp adversary-initial-state  $\leq T + C$ 
    by (rule add-mono[OF trace-path-bound composition-path-bound])
  have
    wp-event ?M ?Pair adversary-initial-state +
    (wp-event ?M ?Trace adversary-initial-state +
    wp-event ?M ?Comp adversary-initial-state)  $\leq P + (T + C)$ 
    by (rule add-mono[OF pair-bound path-sum])
  then show ?thesis
    by (simp add: add.assoc)
qed
show ?thesis
  by (rule order-trans[OF event-le order-trans[OF union-le sum-bound]])
qed

```

lemma *checked-staged-security-with-query-prefix-authenticated-opening-hit-bound-from-inconsistent-candidate-pair-an*
assumes *wf: staged-budget-wellformed budgets*
and *controlled: staged-adversary-controlled budgets A*
and *i-bound: i < rounds*

and *trace-candidate*:
partial-trace-table-candidate trace-table
((replicate rounds []) [i := trace-openings])
and *comp-candidate*:
partial-composition-table-candidate composition-table
((replicate rounds []) [i := composition-openings])
and *trace-low: trace-table-low-degree trace-table*
and *comp-low: composition-table-low-degree maxDegree composition-table*
and *not-all-on-hit*:
 $\bigwedge \text{prefix prefix-state raw raw-state data attacker-state result}$
 final-state.
 $\text{Some } (((((\text{prefix, prefix-state}), \text{raw}), \text{raw-state}),$
 $((\text{data, attacker-state}), \text{result})), \text{final-state}) \in$
 set-dist
 $(\text{execute}$
 $(\text{checked-staged-security-experiment-with-query-prefix-data-state}$
 $A \ i)$
 $\text{adversary-initial-state}) \implies$
 $\text{checked-staged-security-with-query-prefix-authenticated-opening-hit}$
 $\text{trace-openings composition-openings } i$
 $(\text{Some } (((((\text{prefix, prefix-state}), \text{raw}), \text{raw-state}),$
 $((\text{data, attacker-state}), \text{result})), \text{final-state})) \implies$
 $\neg \text{all-queries-consistent trace-table composition-table}$
 $(\text{sqp-alphas prefix})$
and *trace-path-bound*:
 wp-event
 $(\text{checked-staged-security-experiment-with-query-prefix-data-state } A \ i)$
 $(\text{checked-staged-security-with-query-prefix-current-trace-path-output-hit-at}$
 $i)$
 $\text{adversary-initial-state} \leq T$
and *composition-path-bound*:
 wp-event
 $(\text{checked-staged-security-experiment-with-query-prefix-data-state } A \ i)$
 $(\text{checked-staged-security-with-query-prefix-current-composition-path-output-hit-at}$
 $i)$
 $\text{adversary-initial-state} \leq C$
shows
 wp-event
 $(\text{checked-staged-security-experiment-with-query-prefix-data-state } A \ i)$
 $(\text{checked-staged-security-with-query-prefix-authenticated-opening-hit}$
 $\text{trace-openings composition-openings } i)$
 $\text{adversary-initial-state} \leq$
 $\text{staged-phase-relation-error size}$
 $(\text{staged-query-search-queries budgets } i + 1) +$
 $\text{query-error-bound} + T + C$
proof –
let $?M =$
 $\text{checked-staged-security-experiment-with-query-prefix-data-state } A \ i$
let $?Pair =$

```

checked-staged-security-with-query-prefix-dynamic-index-hit
  (staged-query-prefix-inconsistent-candidate-pair-query-target
   trace-table composition-table)
let ?Trace =
  checked-staged-security-with-query-prefix-current-trace-path-output-hit-at
  i
let ?Comp =
  checked-staged-security-with-query-prefix-current-composition-path-output-hit-at
  i
have event-le:
  wp-event ?M
  (checked-staged-security-with-query-prefix-authenticated-opening-hit
   trace-openings composition-openings i)
  adversary-initial-state ≤
  wp-event ?M (λout. ?Pair out ∨ ?Trace out ∨ ?Comp out)
  adversary-initial-state
proof (rule wp-event-mono-on-support)
fix out
assume support:
  out ∈ set-dist (execute ?M adversary-initial-state)
and hit:
  checked-staged-security-with-query-prefix-authenticated-opening-hit
  trace-openings composition-openings i out
show ?Pair out ∨ ?Trace out ∨ ?Comp out
proof (cases out)
case None
then show ?thesis
  using hit
  unfolding
    checked-staged-security-with-query-prefix-authenticated-opening-hit-def
  by simp
next
case (Some packed)
then obtain prefix prefix-state raw raw-state data attacker-state result
  final-state where out-eq:
    out =
      Some (((((prefix, prefix-state), raw), raw-state),
              ((data, attacker-state), result)), final-state)
  by (cases packed) (auto split: prod.splits)
have support-some:
  Some (((((prefix, prefix-state), raw), raw-state),
          ((data, attacker-state), result)), final-state) ∈
  set-dist (execute ?M adversary-initial-state)
  using support out-eq by simp
have not-all:
  ¬ all-queries-consistent trace-table composition-table
  (sqp-alphas prefix)
by (rule not-all-on-hit[OF support-some])
  (use hit out-eq in simp)

```

```

have split:
  checked-staged-security-with-query-prefix-candidate-pair-from-prefix-hit
    trace-table composition-table
      (Some (((prefix, prefix-state), raw), raw-state),
        ((data, attacker-state), result)), final-state)) ∨
  checked-staged-security-with-query-prefix-current-trace-path-output-hit-at
    i
      (Some (((prefix, prefix-state), raw), raw-state),
        ((data, attacker-state), result)), final-state)) ∨
  checked-staged-security-with-query-prefix-current-composition-path-output-hit-at
    i
      (Some (((prefix, prefix-state), raw), raw-state),
        ((data, attacker-state), result)), final-state))
by (rule
  checked-staged-security-with-query-prefix-authenticated-opening-hit-imp-candidate-pair-or-structured-path-on-s
    [OF wf controlled i-bound support-some])
  (use hit[unfolded out-eq] trace-candidate comp-candidate
    trace-low comp-low not-all in simp-all)
then show ?thesis
proof
  assume pair:
    checked-staged-security-with-query-prefix-candidate-pair-from-prefix-hit
      trace-table composition-table
        (Some (((prefix, prefix-state), raw), raw-state),
          ((data, attacker-state), result)), final-state))
  have ?Pair
    (Some (((prefix, prefix-state), raw), raw-state),
      ((data, attacker-state), result)), final-state))
  by (rule
    checked-staged-security-with-query-prefix-candidate-pair-from-prefix-hit-imp-inconsistent-target
      [OF pair not-all])
  then show ?thesis
    unfolding out-eq by simp
next
  assume
    checked-staged-security-with-query-prefix-current-trace-path-output-hit-at
      i
        (Some (((prefix, prefix-state), raw), raw-state),
          ((data, attacker-state), result)), final-state)) ∨
    checked-staged-security-with-query-prefix-current-composition-path-output-hit-at
      i
        (Some (((prefix, prefix-state), raw), raw-state),
          ((data, attacker-state), result)), final-state))
  then show ?thesis
    unfolding out-eq by simp
qed
qed
qed
have pair-bound:

```

```

wp-event ?M ?Pair adversary-initial-state ≤
  staged-phase-relation-error size
  (staged-query-search-queries budgets i + 1) + query-error-bound
by (rule
  checked-staged-security-with-query-prefix-inconsistent-candidate-pair-hit-bound
  [OF wf controlled i-bound trace-low comp-low])
have union-le:
  wp-event ?M (λout. ?Pair out ∨ ?Trace out ∨ ?Comp out)
    adversary-initial-state ≤
  wp-event ?M ?Pair adversary-initial-state +
  (wp-event ?M ?Trace adversary-initial-state +
  wp-event ?M ?Comp adversary-initial-state)
proof -
  have
    wp-event ?M (λout. ?Pair out ∨ ?Trace out ∨ ?Comp out)
      adversary-initial-state ≤
    wp-event ?M ?Pair adversary-initial-state +
    wp-event ?M (λout. ?Trace out ∨ ?Comp out)
      adversary-initial-state
  by (rule wp-event-union-bound)
  also have
    ... ≤
    wp-event ?M ?Pair adversary-initial-state +
    (wp-event ?M ?Trace adversary-initial-state +
    wp-event ?M ?Comp adversary-initial-state)
  by (intro add-mono order-refl wp-event-union-bound)
  finally show ?thesis .
qed
have sum-bound:
  wp-event ?M ?Pair adversary-initial-state +
  (wp-event ?M ?Trace adversary-initial-state +
  wp-event ?M ?Comp adversary-initial-state) ≤
  (staged-phase-relation-error size
  (staged-query-search-queries budgets i + 1) + query-error-bound) +
  (T + C)
by (intro add-mono pair-bound add-mono trace-path-bound
  composition-path-bound)
have rhs-eq:
  (staged-phase-relation-error size
  (staged-query-search-queries budgets i + 1) + query-error-bound) +
  (T + C) =
  staged-phase-relation-error size
  (staged-query-search-queries budgets i + 1) +
  query-error-bound + T + C
by (simp add: add.assoc)
have bound:
  wp-event ?M
  (checked-staged-security-with-query-prefix-authenticated-opening-hit
  trace-openings composition-openings i)

```


$adversary-initial-state \leq$
 $(staged-phase-relation-error\ size$
 $(staged-query-search-queries\ budgets\ i + 1) + query-error-bound) +$
 $(T + C)$
by (rule order-trans[*OF event-le order-trans[OF union-le sum-bound]*])
show ?thesis
using bound rhs-eq **by** simp
qed

lemma checked-staged-security-with-query-prefix-authenticated-opening-hit-bound-from-candidate-pair-budgets-and-st

assumes wf: staged-budget-wellformed budgets

and controlled: staged-adversary-controlled budgets *A*

and i-bound: $i < rounds$

and trace-candidate:

partial-trace-table-candidate trace-table

((replicate rounds []) [i := trace-openings])

and comp-candidate:

partial-composition-table-candidate composition-table

((replicate rounds []) [i := composition-openings])

and trace-low: trace-table-low-degree trace-table

and comp-low: composition-table-low-degree maxDegree composition-table

and not-all:

$\bigwedge prefix\ prefix-state.$

Some ((prefix, prefix-state), prefix-state) $\in$

set-dist

(execute (checked-staged-query-prefix-with-state *A i*)

adversary-initial-state) $\implies$

$\neg$ all-queries-consistent trace-table composition-table

(sqp-alphas prefix)

and trace-path-bound:

wp-event

(checked-staged-security-experiment-with-query-prefix-data-state *A i*)

(checked-staged-security-with-query-prefix-current-trace-path-output-hit-at

i)

adversary-initial-state $\leq T$

and composition-path-bound:

wp-event

(checked-staged-security-experiment-with-query-prefix-data-state *A i*)

(checked-staged-security-with-query-prefix-current-composition-path-output-hit-at

i)

adversary-initial-state $\leq C$

shows

wp-event

(checked-staged-security-experiment-with-query-prefix-data-state *A i*)

(checked-staged-security-with-query-prefix-authenticated-opening-hit

trace-openings composition-openings *i*)

adversary-initial-state $\leq$

staged-phase-relation-error size

(staged-query-search-queries budgets $i + 1$) +

$query-error-bound + T + C$

proof –

have *pair-bound*:

wp-event
(checked-staged-security-experiment-with-query-prefix-data-state A i)
(checked-staged-security-with-query-prefix-candidate-pair-from-prefix-hit
trace-table composition-table)
adversary-initial-state ≤
staged-phase-relation-error size
(staged-query-search-queries budgets i + 1) + query-error-bound

by (*rule*
checked-staged-security-with-query-prefix-candidate-pair-from-prefix-hit-bound-from-budgets
[OF wf controlled i-bound trace-low comp-low not-all]))

have *not-all-support*:

$\bigwedge prefix\ prefix-state\ raw\ raw-state\ data\ attacker-state\ result$
final-state.
Some (((((prefix, prefix-state), raw), raw-state),
((data, attacker-state), result)), final-state) ∈
set-dist
(execute
(checked-staged-security-experiment-with-query-prefix-data-state
A i)
adversary-initial-state) ⇒
 $\neg$ *all-queries-consistent trace-table composition-table*
(sqp-alphas prefix)

proof –

fix *prefix prefix-state raw raw-state data attacker-state result final-state*

assume *support*:

Some (((((prefix, prefix-state), raw), raw-state),
((data, attacker-state), result)), final-state) ∈
set-dist
(execute
(checked-staged-security-experiment-with-query-prefix-data-state
A i)
adversary-initial-state)

have *head*:

Some (((prefix, prefix-state), raw), raw-state) ∈
set-dist
(execute (checked-staged-query-prefix-receive-with-state A i)
adversary-initial-state)

using *support*

unfolding *checked-staged-security-experiment-with-query-prefix-data-state-def*

by (*auto elim! set-dist-bindE split prod.splits*)

have *prefix-support*:

Some ((prefix, prefix-state), prefix-state) ∈
set-dist
(execute (checked-staged-query-prefix-with-state A i)
adversary-initial-state)

by (*rule checked-staged-query-prefix-receive-with-state-prefix-support*

```

    [OF head])
  show
    ¬ all-queries-consistent trace-table composition-table
      (sqp-alphas prefix)
    by (rule not-all[OF prefix-support])
qed
show ?thesis
  by (rule
    checked-staged-security-with-query-prefix-authenticated-opening-hit-bound-from-candidate-pair-and-structured-path
    [OF wf controlled i-bound trace-candidate comp-candidate trace-low
      comp-low not-all-support pair-bound trace-path-bound
      composition-path-bound])
qed

lemma checked-staged-security-with-query-prefix-current-query-partial-opening-hit-at-bound-from-prefix-and-structured-path
  assumes wf: staged-budget-wellformed budgets
  and controlled: staged-adversary-controlled budgets A
  and i-bound: i < rounds
  and prefix-bound:
    wp-event
      (checked-staged-security-experiment-with-query-prefix-data-state A i)
      (checked-staged-security-with-query-prefix-current-prefix-authenticated-hit-at i)
      adversary-initial-state ≤ P
  and trace-path-bound:
    wp-event
      (checked-staged-security-experiment-with-query-prefix-data-state A i)
      (checked-staged-security-with-query-prefix-current-trace-path-output-hit-at i)
      adversary-initial-state ≤ T
  and composition-path-bound:
    wp-event
      (checked-staged-security-experiment-with-query-prefix-data-state A i)
      (checked-staged-security-with-query-prefix-current-composition-path-output-hit-at i)
      adversary-initial-state ≤ C
  shows
    wp-event
      (checked-staged-security-experiment-with-query-prefix-data-state A i)
      (checked-staged-security-with-query-prefix-current-query-partial-opening-hit-at i)
      adversary-initial-state ≤ P + T + C
proof -
  let ?M = checked-staged-security-experiment-with-query-prefix-data-state A i
  let ?Prefix =
    checked-staged-security-with-query-prefix-current-prefix-authenticated-hit-at i
  let ?Trace =
    checked-staged-security-with-query-prefix-current-trace-path-output-hit-at i
  let ?Composition =
    checked-staged-security-with-query-prefix-current-composition-path-output-hit-at i
  have event-le:
    wp-event ?M
      (checked-staged-security-with-query-prefix-current-query-partial-opening-hit-at i)

```

```

    adversary-initial-state ≤
    wp-event ?M (λout. ?Prefix out ∨ ?Trace out ∨ ?Composition out)
    adversary-initial-state
  by (rule wp-event-mono-on-support)
    (rule
      checked-staged-security-with-query-prefix-current-query-partial-opening-hit-at-imp-prefix-or-structured-path-on-su
      [OF wf controlled i-bound])
  have first-union:
    wp-event ?M (λout. ?Prefix out ∨ ?Trace out ∨ ?Composition out)
    adversary-initial-state ≤
    wp-event ?M ?Prefix adversary-initial-state +
    wp-event ?M (λout. ?Trace out ∨ ?Composition out)
    adversary-initial-state
  by (rule wp-event-union-bound)
  have second-union:
    wp-event ?M (λout. ?Trace out ∨ ?Composition out)
    adversary-initial-state ≤
    wp-event ?M ?Trace adversary-initial-state +
    wp-event ?M ?Composition adversary-initial-state
  by (rule wp-event-union-bound)
  have path-sum-bound:
    wp-event ?M (λout. ?Trace out ∨ ?Composition out)
    adversary-initial-state ≤ T + C
  by (rule order-trans[OF second-union])
    (intro add-mono trace-path-bound composition-path-bound)
  have sum-bound:
    wp-event ?M ?Prefix adversary-initial-state +
    wp-event ?M (λout. ?Trace out ∨ ?Composition out)
    adversary-initial-state ≤ P + (T + C)
  by (intro add-mono prefix-bound path-sum-bound)
  have union-sum-bound:
    wp-event ?M (λout. ?Prefix out ∨ ?Trace out ∨ ?Composition out)
    adversary-initial-state ≤ P + T + C
  by (rule order-trans[OF first-union])
    (use sum-bound in ⟨simp add: add.assoc⟩)
  show ?thesis
  by (rule order-trans[OF event-le])
    (rule union-sum-bound)
qed

```

lemma *checked-staged-security-with-actual-alpha-prefix-composition-candidate-drift-query-hit-bound-from-prefix-target*
assumes *wf: staged-budget-wellformed budgets*
and *controlled: staged-adversary-controlled budgets A*
and *prefix-bound:*
 $\bigwedge i. i < \text{rounds} \implies$
 wp-event (*checked-staged-query-prefix-receive-with-state A i*)
 (*checked-staged-query-prefix-dynamic-index-hit*
staged-query-prefix-prefix-authenticated-opening-target)
 adversary-initial-state ≤ P i

and *trace-path-bound*:
 $\bigwedge i. i < \text{rounds} \implies$
 wp-event
 $(\text{checked-staged-security-experiment-with-query-prefix-data-state } A \ i)$
 $(\text{checked-staged-security-with-query-prefix-current-trace-path-output-hit-at } i)$
 $\text{adversary-initial-state} \leq T \ i$
and *composition-path-bound*:
 $\bigwedge i. i < \text{rounds} \implies$
 wp-event
 $(\text{checked-staged-security-experiment-with-query-prefix-data-state } A \ i)$
 $(\text{checked-staged-security-with-query-prefix-current-composition-path-output-hit-at } i)$
 $\text{adversary-initial-state} \leq C \ i$
shows
 wp-event
 $(\text{checked-staged-security-experiment-with-actual-alpha-prefix-data-state } A)$
 $\text{checked-staged-security-with-actual-alpha-prefix-composition-candidate-drift-query-hit}$
 $\text{adversary-initial-state} \leq (\sum_{i < \text{rounds}} P \ i + T \ i + C \ i)$
proof (rule
 $\text{checked-staged-security-with-actual-alpha-prefix-composition-candidate-drift-query-hit-bound-from-query-prefix-curr}$
 $[\text{OF wf controlled}])$
fix i
assume $i\text{-bound}: i < \text{rounds}$
have prefix-hit-bound :
 wp-event
 $(\text{checked-staged-security-experiment-with-query-prefix-data-state } A \ i)$
 $(\text{checked-staged-security-with-query-prefix-current-prefix-authenticated-hit-at } i)$
 $\text{adversary-initial-state} \leq P \ i$
by (rule
 $\text{checked-staged-security-with-query-prefix-current-prefix-authenticated-hit-at-bound-from-dynamic-target}$
 $[\text{OF wf controlled } i\text{-bound } \text{prefix-bound}[\text{OF } i\text{-bound}]])$
show
 wp-event
 $(\text{checked-staged-security-experiment-with-query-prefix-data-state } A \ i)$
 $(\text{checked-staged-security-with-query-prefix-current-query-partial-opening-hit-at } i)$
 $\text{adversary-initial-state} \leq P \ i + T \ i + C \ i$
by (rule
 $\text{checked-staged-security-with-query-prefix-current-query-partial-opening-hit-at-bound-from-prefix-and-structured-p}$
 $[\text{OF wf controlled } i\text{-bound } \text{prefix-hit-bound } \text{trace-path-bound}[\text{OF } i\text{-bound}]$
 $\text{composition-path-bound}[\text{OF } i\text{-bound}]])$
qed

lemma $\text{checked-staged-security-with-actual-alpha-prefix-composition-candidate-drift-query-hit-bound-from-trace-indic}$
assumes $\text{wf}: \text{staged-budget-wellformed budgets}$
and $\text{controlled}: \text{staged-adversary-controlled budgets } A$
and trace-frac :
 $\bigwedge i \text{ prefix } \text{prefix-state}.$
 $i < \text{rounds} \implies$
 $\text{Some } ((\text{prefix}, \text{prefix-state}), \text{prefix-state}) \in$
 set-dist

$(\text{execute } (\text{checked-staged-query-prefix-with-state } A \ i) \text{ adversary-initial-state}) \implies$
 nnreal
 $(\text{card } (\text{staged-query-prefix-prefix-authenticated-trace-indices } \text{prefix} \text{ prefix-state})) /$
 $\text{nnreal } (\text{card } \text{query-sample-space}) \leq \text{query-error-bound}$
and *trace-path-bound*:
 $\bigwedge i. i < \text{rounds} \implies$
 wp-event
 $(\text{checked-staged-security-experiment-with-query-prefix-data-state } A \ i)$
 $(\text{checked-staged-security-with-query-prefix-current-trace-path-output-hit-at } i)$
 $\text{adversary-initial-state} \leq T \ i$
and *composition-path-bound*:
 $\bigwedge i. i < \text{rounds} \implies$
 wp-event
 $(\text{checked-staged-security-experiment-with-query-prefix-data-state } A \ i)$
 $(\text{checked-staged-security-with-query-prefix-current-composition-path-output-hit-at } i)$
 $\text{adversary-initial-state} \leq C \ i$
shows
 wp-event
 $(\text{checked-staged-security-experiment-with-actual-alpha-prefix-data-state } A)$
 $\text{checked-staged-security-with-actual-alpha-prefix-composition-candidate-drift-query-hit}$
 $\text{adversary-initial-state} \leq$
 $(\sum i < \text{rounds}.$
 $\text{staged-phase-relation-error size}$
 $(\text{staged-query-search-queries budgets } i + 1) +$
 $\text{query-error-bound} + T \ i + C \ i)$
proof (rule
 $\text{checked-staged-security-with-actual-alpha-prefix-composition-candidate-drift-query-hit-bound-from-prefix-target-and-}$
 $[\text{OF wf controlled - trace-path-bound composition-path-bound}])$
fix i
assume $i\text{-bound: } i < \text{rounds}$
show $\text{wp-event } (\text{checked-staged-query-prefix-receive-with-state } A \ i)$
 $(\text{checked-staged-query-prefix-dynamic-index-hit}$
 $\text{staged-query-prefix-prefix-authenticated-opening-target})$
 $\text{adversary-initial-state} \leq$
 $\text{staged-phase-relation-error size}$
 $(\text{staged-query-search-queries budgets } i + 1) + \text{query-error-bound}$
proof (rule
 $\text{checked-staged-query-prefix-prefix-authenticated-target-hit-bound-from-trace-indices}$
 $[\text{OF wf controlled } i\text{-bound}])$
fix $\text{prefix prefix-state}$
assume *support*:
 $\text{Some } ((\text{prefix}, \text{prefix-state}), \text{prefix-state}) \in$
 set-dist
 $(\text{execute } (\text{checked-staged-query-prefix-with-state } A \ i) \text{ adversary-initial-state})$
show

```

nnreal
  (card
    (staged-query-prefix-prefix-authenticated-trace-indices prefix
      prefix-state)) /
  nnreal (card query-sample-space) ≤ query-error-bound
by (rule trace-frac[OF i-bound support])
qed
qed

lemma checked-staged-security-with-data-state-composition-bad-bound-from-trace-indices-structured-paths-and-budget
  assumes wf: staged-budget-wellformed budgets
  and controlled: staged-adversary-controlled budgets A
  and trace-frac:
     $\bigwedge i$  prefix prefix-state.
     $i < \text{rounds} \implies$ 
    Some ((prefix, prefix-state), prefix-state) ∈
      set-dist
      (execute (checked-staged-query-prefix-with-state A i)
        adversary-initial-state)  $\implies$ 
    nnreal
      (card
        (staged-query-prefix-prefix-authenticated-trace-indices prefix
          prefix-state)) /
      nnreal (card query-sample-space) ≤ query-error-bound
  and trace-path-bound:
     $\bigwedge i.$   $i < \text{rounds} \implies$ 
    wp-event
      (checked-staged-security-experiment-with-query-prefix-data-state A i)
      (checked-staged-security-with-query-prefix-current-trace-path-output-hit-at i)
      adversary-initial-state ≤ T i
  and composition-path-bound:
     $\bigwedge i.$   $i < \text{rounds} \implies$ 
    wp-event
      (checked-staged-security-experiment-with-query-prefix-data-state A i)
      (checked-staged-security-with-query-prefix-current-composition-path-output-hit-at i)
      adversary-initial-state ≤ C i
  shows
    wp-event (checked-staged-security-experiment-with-data-state A)
      (staged-security-with-data-state-verifier-event composition-bad)
      adversary-initial-state ≤
      composition-error-bound +
      staged-phase-relation-error size
      (staged-alpha-search-queries budgets 0) +
      hash-collision-budget-value 0
      (staged-alpha-search-queries budgets 0) +
      staged-phase-relation-error size
      (staged-alpha-search-queries budgets 0) +
      ( $\sum i < \text{rounds}.$ 
        staged-phase-relation-error size

```

```

    (staged-query-search-queries budgets  $i + 1$ ) +
    query-error-bound +  $T i + C i$ ) +
    hash-collision-budget-value 0
    (staged-attacker-query-budget budgets +
    staged-challenge-query-budget + verifier-hash-query-budget)
proof -
  let ?Q =
    ( $\sum i < \text{rounds}$ .
      staged-phase-relation-error size
      (staged-query-search-queries budgets  $i + 1$ ) +
      query-error-bound +  $T i + C i$ )
  have query-bound:
    wp-event
    (checked-staged-security-experiment-with-actual-alpha-prefix-data-state A)
    checked-staged-security-with-actual-alpha-prefix-composition-candidate-drift-query-hit
    adversary-initial-state  $\leq$  ?Q
  by (rule
    checked-staged-security-with-actual-alpha-prefix-composition-candidate-drift-query-hit-bound-from-trace-indices-a
    [OF wf controlled trace-frac trace-path-bound composition-path-bound])
  have without-bound:
    wp-event
    (checked-staged-security-experiment-with-actual-alpha-prefix-data-state A)
    checked-staged-security-with-actual-alpha-prefix-composition-candidate-drift-without-query-hit
    adversary-initial-state  $\leq$  0
  by (rule
    checked-staged-security-with-actual-alpha-prefix-composition-candidate-drift-without-query-hit-bound
    [OF wf controlled])
  have drift-bound:
    wp-event
    (checked-staged-security-experiment-with-actual-alpha-prefix-data-state A)
    checked-staged-security-with-actual-alpha-prefix-composition-candidate-drift
    adversary-initial-state  $\leq$  ?Q + 0
  by (rule
    checked-staged-security-with-actual-alpha-prefix-composition-candidate-drift-bound-from-query-and-without-query
    [OF query-bound without-bound])
  have comp-bound:
    wp-event (checked-staged-security-experiment-with-data-state A)
    (staged-security-with-data-state-verifier-event composition-bad)
    adversary-initial-state  $\leq$ 
    composition-error-bound +
    staged-phase-relation-error size
    (staged-alpha-search-queries budgets 0) +
    hash-collision-budget-value 0
    (staged-alpha-search-queries budgets 0) +
    staged-phase-relation-error size
    (staged-alpha-search-queries budgets 0) +
    (?Q + 0) +
    hash-collision-budget-value 0
    (staged-attacker-query-budget budgets +

```



```

    staged-challenge-query-budget + verifier-hash-query-budget)
  by (rule
    checked-staged-security-with-data-state-composition-bad-bound-from-composition-candidate-drift-and-budgets
    [OF wf controlled drift-bound])
  then show ?thesis
  by simp
qed

end

end

```

theory *Soundness-Conceptual-Query-Empty-Current*

imports

Soundness-Query-Opening-Consistency
Soundness-Conceptual-Query-Current
Staged-Security-Experiment-Composition-Query-Current-Path
Soundness-Staged-Bounds

begin

Empty-composition current-query projection.

This layer connects the verifier-local empty-composition query extraction to the prefix-fixed conceptual empty query target infrastructure. It is kept separate from the non-empty current-query layer to avoid growing the already size-sensitive staged/query theories.

context *soundness*

begin

definition *staged-security-with-data-state-current-empty-query-opening-hit-at*

$:: \text{nat} \Rightarrow$
 $((('f \text{ staged-proof-data} \times 'f \text{ protocol-channel}) \times \text{unit list}) \times$
 $'f \text{ protocol-channel}) \text{ option} \Rightarrow \text{bool}$

where

staged-security-with-data-state-current-empty-query-opening-hit-at i out
 $\iff$
 (case out of
 None $\Rightarrow$ False
 | Some (((data, attacker-state), result), final-state) $\Rightarrow$
 (let s =
 verifier-state-from-adversary attacker-state
 (staged-proof-transcript data)
 in
 ($\exists$ raw trace-openings rest.
 $i < \text{rounds} \wedge$
 fmlookup (HashMap final-state)
 (QueryIndexChallenge i
 (state-after-query-chunks
 (staged-query-start-hash data))

$(\text{staged-query-chunks data } i)) =$
 $\text{Some raw} \wedge$
 $\text{staged-composition-fri-roots data} = [] \wedge$
 $\text{Some (result, final-state)} \in$
 $\text{set-dist (execute verify-monad s)} \wedge$
 $\text{partial-authenticated-table (staged-trace-root data)}$
 $(\text{scale} * \text{clength}) \text{ trace-openings final-state} \wedge$
 $\text{verifier-header-transcript s}$
 $(\text{staged-trace-root data})$
 $(\text{staged-trace-fri-roots data})$
 $(\text{staged-trace-final data})$
 $(\text{staged-alphas data})$
 $(\text{staged-degree data})$
 $[]$
 $(\text{staged-composition-final data})$
 $\text{rest} \wedge$
 $\text{map opening-index trace-openings} =$
 $\text{powers-scaled (index (to-nat raw))} \wedge$
 $\text{cp-eval (staged-alphas data) (map opening-value trace-openings)}$
 $(h \wedge \text{index (to-nat raw)} * \text{shift}) =$
 $\text{staged-composition-final data}))$

definition *staged-security-with-data-state-current-empty-query-opening-hit*
 $:: (((\text{'f staged-proof-data} \times \text{'f protocol-channel}) \times \text{unit list}) \times$
 $\text{'f protocol-channel}) \text{ option} \Rightarrow \text{bool}$

where

$\text{staged-security-with-data-state-current-empty-query-opening-hit out}$
 $\longleftrightarrow$
 $(\exists i < \text{rounds}.$
 $\text{staged-security-with-data-state-current-empty-query-opening-hit-at}$
 $i \text{ out})$

lemma *staged-security-with-data-state-current-empty-query-opening-hitI:*
assumes $i < \text{rounds}$
and $\text{staged-security-with-data-state-current-empty-query-opening-hit-at}$
 $i \text{ out}$
shows $\text{staged-security-with-data-state-current-empty-query-opening-hit out}$
using *assms*
unfolding *staged-security-with-data-state-current-empty-query-opening-hit-def*
by *blast*

lemma *staged-security-with-data-state-current-empty-query-opening-hit-atI:*
assumes $i\text{-bound: } i < \text{rounds}$
and *lookup:*
 $\text{fmlookup (HashMap final-state)}$
 $(\text{QueryIndexChallenge } i$
 $(\text{state-after-query-chunks}$
 $(\text{staged-query-start-hash data})$
 $(\text{staged-query-chunks data } i)) =$

```

    Some raw
  and comp-empty: staged-composition-fri-roots data = []
  and verifier:
    Some (result, final-state) ∈
      set-dist
        (execute verify-monad
          (verifier-state-from-adversary attacker-state
            (staged-proof-transcript data)))
  and trace-auth:
    partial-authenticated-table (staged-trace-root data)
      (scale * clength) trace-openings final-state
  and header:
    verifier-header-transcript
      (verifier-state-from-adversary attacker-state
        (staged-proof-transcript data))
      (staged-trace-root data)
      (staged-trace-fri-roots data)
      (staged-trace-final data)
      (staged-alphas data)
      (staged-degree data)
      []
      (staged-composition-final data)
    rest
  and trace-indices:
    map opening-index trace-openings =
      powers-scaled (index (to-nat raw))
  and consistent:
    cp-eval (staged-alphas data) (map opening-value trace-openings)
      (h ^ index (to-nat raw) * shift) =
        staged-composition-final data
  shows
    staged-security-with-data-state-current-empty-query-opening-hit-at i
      (Some (((data, attacker-state), result), final-state))
  unfolding staged-security-with-data-state-current-empty-query-opening-hit-at-def
  Let-def
  apply simp
  by (intro exI[of - raw] exI[of - trace-openings] exI[of - rest] conjI)
    (use assms in simp-all)

lemma empty-query-bad-header-alignment-on-support:
  assumes wf: staged-budget-wellformed budgets
  and controlled: staged-adversary-controlled budgets A
  and support:
    Some (((data, attacker-state), result), final-state) ∈
      set-dist
        (execute
          (checked-staged-security-experiment-with-data-state A)
            adversary-initial-state)
  and bad:

```

```

    staged-security-with-data-state-verifier-event
    query-bad-with-empty-composition-header-candidates
    (Some (((data, attacker-state), result), final-state))
obtains fr f-fri-roots f-final as dg final rest where
  verifier-header-transcript
    (verifier-state-from-adversary attacker-state
     (staged-proof-transcript data))
  fr f-fri-roots f-final as dg [] final rest
  fr = staged-trace-root data
  f-fri-roots = staged-trace-fri-roots data
  f-final = staged-trace-final data
  as = staged-alphas data
  dg = staged-degree data
  staged-composition-fri-roots data = []
  final = staged-composition-final data
  rest = List.concat (staged-query-chunks data)
  verifier-header-state
    (verifier-state-from-adversary attacker-state
     (staged-proof-transcript data))
  fr f-fri-roots f-final as dg [] final =
    staged-query-start-hash data
proof -
  let ?s =
    verifier-state-from-adversary attacker-state
    (staged-proof-transcript data)
from checked-staged-security-experiment-with-data-state-outcomeE
  [OF support]
obtain builder where builder:
  Some (data, attacker-state) ∈
    set-dist
    (execute (checked-staged-transcript-program A)
     adversary-initial-state)
  by blast
have staged-header:
  verifier-header-transcript ?s
    (staged-trace-root data)
    (staged-trace-fri-roots data)
    (staged-trace-final data)
    (staged-alphas data)
    (staged-degree data)
    (staged-composition-fri-roots data)
    (staged-composition-final data)
    (List.concat (staged-query-chunks data))
  by (rule checked-staged-transcript-program-outcome-header-transcript
    [OF wf controlled builder])
have bad-verifier:
  query-bad-with-empty-composition-header-candidates ?s
    (Some (result, final-state))
using bad

```

```

unfolding staged-security-with-data-state-verifier-event-def
by simp
from bad-verifier obtain fr f-fri-roots f-final as dg final
  trace-query-idxs trace-openings trace-table composition-table
where partial:
  accepted-with-empty-composition-header-candidates ?s
    (Some (result, final-state)) fr f-fri-roots f-final as dg final
    trace-query-idxs trace-openings trace-table composition-table
unfolding query-bad-with-empty-composition-header-candidates-def
by blast
from partial obtain rest where header:
  verifier-header-transcript ?s fr f-fri-roots f-final as dg []
  final rest
unfolding accepted-with-empty-composition-header-candidates-def
by blast
have header-eq:
  fr = staged-trace-root data ∧
  f-fri-roots = staged-trace-fri-roots data ∧
  f-final = staged-trace-final data ∧
  as = staged-alphas data ∧
  dg = staged-degree data ∧
  [] = staged-composition-fri-roots data ∧
  final = staged-composition-final data ∧
  rest = List.concat (staged-query-chunks data)
using verifier-header-transcript-unique[OF header staged-header]
by simp
have header-state:
  verifier-header-state ?s fr f-fri-roots f-final as dg [] final =
    staged-query-start-hash data
using header-eq
unfolding verifier-header-state-def verifier-header-messages-def
  staged-query-start-hash-def staged-composition-fri-start-hash-def
  staged-trace-fri-start-hash-def
by simp
show ?thesis
by (rule that[OF header])
  (use header-eq header-state in simp-all)
qed

```

```

lemma empty-query-bad-query-rounds-on-support:
assumes wf: staged-budget-wellformed budgets
and controlled: staged-adversary-controlled budgets A
and support:
  Some (((data, attacker-state), result), final-state) ∈
    set-dist
    (execute
      (checked-staged-security-experiment-with-data-state A)
      adversary-initial-state)
and bad:

```

```

    staged-security-with-data-state-verifier-event
    query-bad-with-empty-composition-header-candidates
    (Some (((data, attacker-state), result), final-state))
obtains f-fl query-state where
    PState query-state = staged-query-start-hash data
    PQueryCounter query-state = 0
    Some (result, final-state) ∈
    set-dist
    (execute
    (ntimes
    (verifier-query-round-program
    (staged-trace-root data)
    f-fl
    (staged-trace-final data)
    (staged-alphas data)
    []
    (staged-composition-final data))
    rounds)
    query-state)
proof –
    let ?s =
    verifier-state-from-adversary attacker-state
    (staged-proof-transcript data)
from checked-staged-security-experiment-with-data-state-outcomeE
    [OF support]
have verifier:
    Some (result, final-state) ∈ set-dist (execute verify-monad ?s)
by blast
from empty-query-bad-header-alignment-on-support
    [OF wf controlled support bad]
obtain fr f-fri-roots f-final as dg final rest where header:
    verifier-header-transcript ?s fr f-fri-roots f-final as dg []
    final rest
and fr-eq: fr = staged-trace-root data
and f-final-eq: f-final = staged-trace-final data
and as-eq: as = staged-alphas data
and final-eq: final = staged-composition-final data
and header-state:
    verifier-header-state ?s fr f-fri-roots f-final as dg [] final =
    staged-query-start-hash data
by blast
from verify-monad-supplied-empty-header-query-rounds[OF verifier header]
obtain f-fl query-state where
    query-state-state:
    PState query-state =
    verifier-header-state ?s fr f-fri-roots f-final as dg [] final
and query-counter: PQueryCounter query-state = PQueryCounter ?s
and query-out:
    Some (result, final-state) ∈

```

```

    set-dist
      (execute
        (ntimes
          (verifier-query-round-program fr f-fl f-final as [] final)
          rounds)
        query-state)
  by blast
show ?thesis
  by (rule that[of query-state f-fl])
    (use query-state-state query-counter query-out header-state fr-eq
      f-final-eq as-eq final-eq in simp-all)
qed

lemma staged-security-with-data-state-current-empty-query-opening-hit-at-imp-query-prefix-current-on-support:
assumes wf: staged-budget-wellformed budgets
and controlled: staged-adversary-controlled budgets A
and i-bound: i < rounds
and support:
  Some (((((prefix, prefix-state), raw), raw-state),
    ((data, attacker-state), result)), final-state) ∈
  set-dist
    (execute
      (checked-staged-security-experiment-with-query-prefix-data-state A i)
      adversary-initial-state)
and hit:
  staged-security-with-data-state-current-empty-query-opening-hit-at i
    (Some (((data, attacker-state), result), final-state))
shows
  checked-staged-security-with-query-prefix-current-empty-authenticated-hit-at
    i
    (Some (((((prefix, prefix-state), raw), raw-state),
      ((data, attacker-state), result)), final-state))
proof -
  let ?s =
    verifier-state-from-adversary attacker-state
      (staged-proof-transcript data)
  have hit':
    ∃ raw' trace-openings rest.
      i < rounds ∧
      fmlookup (HashMap final-state)
        (QueryIndexChallenge i
          (state-after-query-chunks
            (staged-query-start-hash data)
            (staged-query-chunks data) i)) =
        Some raw' ∧
      staged-composition-fri-roots data = [] ∧
      Some (result, final-state) ∈ set-dist (execute verify-monad ?s) ∧
      partial-authenticated-table (staged-trace-root data)
        (scale * clength) trace-openings final-state ∧

```

```

verifier-header-transcript ?s
  (staged-trace-root data)
  (staged-trace-fri-roots data)
  (staged-trace-final data)
  (staged-alphas data)
  (staged-degree data)
  []
  (staged-composition-final data)
  rest ∧
map opening-index trace-openings =
  powers-scaled (index (to-nat raw')) ∧
cp-eval (staged-alphas data) (map opening-value trace-openings)
  (h ^ index (to-nat raw') * shift) =
  staged-composition-final data
using hit
unfolding
  staged-security-with-data-state-current-empty-query-opening-hit-at-def
  Let-def
by simp
from hit' obtain raw' trace-openings rest where
  final-lookup':
    fmlookup (HashMap final-state)
      (QueryIndexChallenge i
        (state-after-query-chunks
          (staged-query-start-hash data)
          (staged-query-chunks data) i)) =
      Some raw'
and trace-auth:
  partial-authenticated-table (staged-trace-root data)
    (scale * clength) trace-openings final-state
and trace-indices:
  map opening-index trace-openings =
    powers-scaled (index (to-nat raw'))
and consistent:
  cp-eval (staged-alphas data) (map opening-value trace-openings)
    (h ^ index (to-nat raw') * shift) =
    staged-composition-final data
by blast
have final-lookup:
  fmlookup (HashMap final-state)
    (QueryIndexChallenge i
      (state-after-query-chunks
        (staged-query-start-hash data) (staged-query-chunks data) i)) =
    Some raw
by (rule
  checked-staged-security-experiment-with-query-prefix-data-state-raw-final-lookup
  [OF wf controlled i-bound support])
have raw-eq: raw' = raw
using final-lookup' final-lookup by simp

```



```

from support obtain chunk chunk-state assert-state record-state suffix-chunks
where data-eq:
  data =
    (|staged-trace-root = sqp-trace-root prefix,
     staged-trace-fri-roots = sqp-trace-fri-roots prefix,
     staged-trace-fri-challenges = sqp-trace-fri-challenges prefix,
     staged-trace-final = sqp-trace-final prefix,
     staged-alphas = sqp-alphas prefix,
     staged-degree = sqp-degree prefix,
     staged-composition-fri-roots = sqp-composition-fri-roots prefix,
     staged-composition-fri-challenges =
       sqp-composition-fri-challenges prefix,
     staged-composition-final = sqp-composition-final prefix,
     staged-query-chunks = sqp-query-chunks prefix @ chunk # suffix-chunks|)
unfolding checked-staged-security-experiment-with-query-prefix-data-state-def
  checked-staged-after-query-prefix-receive-with-verifier-def
  checked-staged-after-query-prefix-receive-def
by (auto elim!: set-dist-bindE split: prod.splits)
have current:
  checked-staged-security-with-query-prefix-empty-authenticated-opening-hit
  trace-openings i
  (Some (((((prefix, prefix-state), raw), raw-state),
    ((data, attacker-state), result)), final-state))
unfolding
  checked-staged-security-with-query-prefix-empty-authenticated-opening-hit-def
using trace-auth trace-indices consistent raw-eq data-eq by simp
show ?thesis
unfolding
  checked-staged-security-with-query-prefix-current-empty-authenticated-hit-at-def
using current by blast
qed

lemma checked-staged-security-with-data-state-current-empty-query-opening-hit-at-le-query-prefix-current:
assumes wf: staged-budget-wellformed budgets
and controlled: staged-adversary-controlled budgets A
and i-bound: i < rounds
shows
  wp-event
  (checked-staged-security-experiment-with-data-state A)
  (staged-security-with-data-state-current-empty-query-opening-hit-at i)
  adversary-initial-state ≤
  wp-event
  (checked-staged-security-experiment-with-query-prefix-data-state A i)
  (checked-staged-security-with-query-prefix-current-empty-authenticated-hit-at i)
  adversary-initial-state
proof –
let ?projected =
  λout. case out of
  None ⇒

```

```

    staged-security-with-data-state-current-empty-query-opening-hit-at
    i None
  | Some (packed, t) ⇒
    staged-security-with-data-state-current-empty-query-opening-hit-at
    i (Some (snd packed, t))
have projection:
  wp-event
  (checked-staged-security-experiment-with-data-state A)
  (staged-security-with-data-state-current-empty-query-opening-hit-at i)
  adversary-initial-state =
  wp-event
  (checked-staged-security-experiment-with-query-prefix-data-state A i)
  ?projected
  adversary-initial-state
by (rule
  checked-staged-security-experiment-with-query-prefix-data-state-projection-event
  [OF i-bound])
have event-le:
  wp-event
  (checked-staged-security-experiment-with-query-prefix-data-state A i)
  ?projected
  adversary-initial-state ≤
  wp-event
  (checked-staged-security-experiment-with-query-prefix-data-state A i)
  (checked-staged-security-with-query-prefix-current-empty-authenticated-hit-at i)
  adversary-initial-state
proof (rule wp-event-mono-on-support)
fix out
assume support:
  out ∈
  set-dist
  (execute
  (checked-staged-security-experiment-with-query-prefix-data-state A i)
  adversary-initial-state)
and hit: ?projected out
show
  checked-staged-security-with-query-prefix-current-empty-authenticated-hit-at
  i out
proof (cases out)
case None
then show ?thesis
  using hit
  unfolding
  staged-security-with-data-state-current-empty-query-opening-hit-at-def
  by simp
next
case (Some packed)
then obtain prefix prefix-state raw raw-state data attacker-state result
  final-state where out-eq:

```

```

    out =
      Some (((((prefix, prefix-state), raw), raw-state),
        ((data, attacker-state), result)), final-state)
    by (cases packed) (auto split: prod.splits)
show ?thesis
  unfolding out-eq
  by (rule
    staged-security-with-data-state-current-empty-query-opening-hit-at-imp-query-prefix-current-on-support
      [OF wf controlled i-bound])
    (use support hit out-eq in simp-all)
qed
qed
show ?thesis
  unfolding projection
  by (rule event-le)
qed

lemma empty-query-bad-imp-current-empty-opening-hit-at-zero-on-support:
  assumes wf: staged-budget-wellformed budgets
  and controlled: staged-adversary-controlled budgets A
  and support:
    Some (((data, attacker-state), result), final-state) ∈
      set-dist
        (execute
          (checked-staged-security-experiment-with-data-state A)
          adversary-initial-state)
  and bad:
    staged-security-with-data-state-verifier-event
      query-bad-with-empty-composition-header-candidates
      (Some (((data, attacker-state), result), final-state))
  shows
    staged-security-with-data-state-current-empty-query-opening-hit-at 0
      (Some (((data, attacker-state), result), final-state))
proof -
  let ?s =
    verifier-state-from-adversary attacker-state
      (staged-proof-transcript data)
  from checked-staged-security-experiment-with-data-state-outcomeE
    [OF support]
  have verifier:
    Some (result, final-state) ∈ set-dist (execute verify-monad ?s)
  by blast
  from empty-query-bad-header-alignment-on-support
    [OF wf controlled support bad]
  obtain rest where header:
    verifier-header-transcript ?s
      (staged-trace-root data)
      (staged-trace-fri-roots data)
      (staged-trace-final data)

```

```

    (staged-alphas data)
    (staged-degree data)
    []
    (staged-composition-final data)
    rest
  and comp-empty:
    staged-composition-fri-roots data = []
  by blast
from empty-query-bad-query-rounds-on-support
  [OF wf controlled support bad]
obtain f-fl query-state where query-state-state:
  PState query-state = staged-query-start-hash data
and query-counter:
  PQueryCounter query-state = 0
and query-out:
  Some (result, final-state) ∈
    set-dist
      (execute
        (ntimes
          (verifier-query-round-program
            (staged-trace-root data)
            f-fl
            (staged-trace-final data)
            (staged-alphas data)
            []
            (staged-composition-final data))
          rounds)
        query-state)
    by blast
from ntimes-verifier-query-rounds-empty-composition-first-consistent
  [OF refl query-out]
obtain raw trace-openings where trace-indices:
  map opening-index trace-openings =
    powers-scaled (index (to-nat raw))
and trace-table:
  partial-authenticated-table (staged-trace-root data)
  (scale * clength) trace-openings final-state
and consistent:
  cp-eval (staged-alphas data) (map opening-value trace-openings)
  (h ^ index (to-nat raw) * shift) =
    staged-composition-final data
and lookup:
  fmlookup (HashMap final-state)
  (QueryIndexChallenge (PQueryCounter query-state) (PState query-state)) =
    Some raw
  by blast
have zero-bound: 0 < rounds
  using rounds-positive by simp
have lookup0:

```

```

    fmlookup (HashMap final-state)
      (QueryIndexChallenge 0
        (state-after-query-chunks
          (staged-query-start-hash data)
          (staged-query-chunks data) 0)) =
      Some raw
  using lookup query-state-state query-counter
  unfolding state-after-query-chunks-def by simp
show ?thesis
by (rule
  staged-security-with-data-state-current-empty-query-opening-hit-atI
  [OF zero-bound lookup0 comp-empty verifier trace-table header
    trace-indices consistent])
qed

lemma checked-staged-security-with-data-state-empty-query-bad-le-current-empty-opening-hit-at-zero:
  assumes wf: staged-budget-wellformed budgets
  and controlled: staged-adversary-controlled budgets A
  shows
    wp-event
      (checked-staged-security-experiment-with-data-state A)
      (staged-security-with-data-state-verifier-event
        query-bad-with-empty-composition-header-candidates)
      adversary-initial-state ≤
    wp-event
      (checked-staged-security-experiment-with-data-state A)
      (staged-security-with-data-state-current-empty-query-opening-hit-at 0)
      adversary-initial-state
proof (rule wp-event-mono-on-support)
  fix out
  assume support:
    out ∈
    set-dist
      (execute
        (checked-staged-security-experiment-with-data-state A)
        adversary-initial-state)
  and bad:
    staged-security-with-data-state-verifier-event
      query-bad-with-empty-composition-header-candidates out
  show
    staged-security-with-data-state-current-empty-query-opening-hit-at 0 out
proof (cases out)
  case None
  then show ?thesis
  using bad
  unfolding staged-security-with-data-state-verifier-event-def
    query-bad-with-empty-composition-header-candidates-def
  by simp
next

```

```

case (Some packed)
then obtain data attacker-state result final-state where out-eq:
  out = Some (((data, attacker-state), result), final-state)
by (cases packed) (auto split: prod.splits)
show ?thesis
  unfolding out-eq
  by (rule empty-query-bad-imp-current-empty-opening-hit-at-zero-on-support
    [OF wf controlled])
    (use support bad out-eq in simp-all)
qed
qed

lemma checked-staged-security-with-data-state-current-empty-query-opening-hit-at-bound-from-prefix-and-path:
assumes wf: staged-budget-wellformed budgets
and controlled: staged-adversary-controlled budgets A
and i-bound: i < rounds
and prefix-bound:
  wp-event
    (checked-staged-security-experiment-with-query-prefix-data-state A i)
    (checked-staged-security-with-query-prefix-current-empty-prefix-authenticated-hit-at
      i)
    adversary-initial-state ≤ P
and path-bound:
  wp-event
    (checked-staged-security-experiment-with-query-prefix-data-state A i)
    (checked-staged-security-with-query-prefix-current-empty-trace-path-output-hit-at
      i)
    adversary-initial-state ≤ T
shows
  wp-event
    (checked-staged-security-experiment-with-data-state A)
    (staged-security-with-data-state-current-empty-query-opening-hit-at i)
    adversary-initial-state ≤ P + T
proof –
have current-le-prefix:
  wp-event
    (checked-staged-security-experiment-with-data-state A)
    (staged-security-with-data-state-current-empty-query-opening-hit-at i)
    adversary-initial-state ≤
  wp-event
    (checked-staged-security-experiment-with-query-prefix-data-state A i)
    (checked-staged-security-with-query-prefix-current-empty-authenticated-hit-at i)
    adversary-initial-state
by (rule
  checked-staged-security-with-data-state-current-empty-query-opening-hit-at-le-query-prefix-current
    [OF wf controlled i-bound])
have prefix-bound':
  wp-event
    (checked-staged-security-experiment-with-query-prefix-data-state A i)

```

(*checked-staged-security-with-query-prefix-current-empty-authenticated-hit-at* i)
 $\text{adversary-initial-state} \leq P + T$
by (rule
 checked-staged-security-with-query-prefix-current-empty-authenticated-hit-at-bound-from-prefix-and-path
 [*OF wf controlled i-bound prefix-bound path-bound*])
show ?thesis
 by (rule *order-trans*[*OF current-le-prefix prefix-bound*])
qed

lemma *checked-staged-security-with-data-state-current-empty-query-opening-hit-at-bound-from-conceptual-empty-residual-hit-at*
assumes *wf: staged-budget-wellformed budgets*
and *controlled: staged-adversary-controlled budgets A*
and *i-bound: $i < \text{rounds}$*
and *residual-bound:*
 wp-event
 (*checked-staged-security-experiment-with-query-prefix-data-state A i*)
 (*checked-staged-security-with-query-prefix-conceptual-empty-context-residual-hit-at*
 i)
 $\text{adversary-initial-state} \leq R$
and *path-bound:*
 wp-event
 (*checked-staged-security-experiment-with-query-prefix-data-state A i*)
 (*checked-staged-security-with-query-prefix-current-empty-trace-path-output-hit-at*
 i)
 $\text{adversary-initial-state} \leq T$
shows
 wp-event
 (*checked-staged-security-experiment-with-data-state A*)
 (*staged-security-with-data-state-current-empty-query-opening-hit-at i*)
 $\text{adversary-initial-state} \leq$
 staged-phase-relation-error size
 (*staged-query-search-queries budgets $i + 1$*) +
 query-error-bound + R + T
proof –
 have *prefix-bound:*
 wp-event
 (*checked-staged-security-experiment-with-query-prefix-data-state A i*)
 (*checked-staged-security-with-query-prefix-current-empty-prefix-authenticated-hit-at*
 i)
 $\text{adversary-initial-state} \leq$
 staged-phase-relation-error size
 (*staged-query-search-queries budgets $i + 1$*) +
 query-error-bound + R
 by (rule
 checked-staged-security-with-query-prefix-current-empty-prefix-authenticated-hit-at-bound-from-conceptual-empty-residual-hit-at
 [*OF wf controlled i-bound residual-bound*])
 have *current-bound:*
 wp-event
 (*checked-staged-security-experiment-with-data-state A*)

```

    (staged-security-with-data-state-current-empty-query-opening-hit-at i)
    adversary-initial-state ≤
    (staged-phase-relation-error size
      (staged-query-search-queries budgets i + 1) +
      query-error-bound + R) + T
  by (rule
    checked-staged-security-with-data-state-current-empty-query-opening-hit-at-bound-from-prefix-and-path
    [OF wf controlled i-bound prefix-bound path-bound])
  then show ?thesis
    by (simp add: algebra-simps)
qed

lemma checked-staged-security-with-query-prefix-current-empty-trace-path-output-hit-at-bound-from-transcript:
  assumes i-bound: i < rounds
  and pre-bound:
    wp-event (checked-staged-security-experiment-with-data-state A)
    staged-security-with-data-state-transcript-pre-hit
    adversary-initial-state ≤ P
  and new-bound:
    wp-event (checked-staged-security-experiment-with-data-state A)
    staged-security-with-data-state-transcript-new-hit
    adversary-initial-state ≤ R
  shows
    wp-event
      (checked-staged-security-experiment-with-query-prefix-data-state A i)
      (checked-staged-security-with-query-prefix-current-empty-trace-path-output-hit-at
        i)
      adversary-initial-state ≤ P + R
proof -
  have event-le:
    wp-event
      (checked-staged-security-experiment-with-query-prefix-data-state A i)
      (checked-staged-security-with-query-prefix-current-empty-trace-path-output-hit-at
        i)
      adversary-initial-state ≤
    wp-event
      (checked-staged-security-experiment-with-query-prefix-data-state A i)
      checked-staged-security-with-query-prefix-transcript-hit
      adversary-initial-state
  proof (rule wp-event-mono-on-support)
    fix out
    assume support:
      out ∈
      set-dist
      (execute
        (checked-staged-security-experiment-with-query-prefix-data-state A i)
        adversary-initial-state)
    and hit:
      checked-staged-security-with-query-prefix-current-empty-trace-path-output-hit-at

```



```

      i out
show checked-staged-security-with-query-prefix-transcript-hit out
proof (cases out)
  case None
  then show ?thesis
    using hit
    unfolding
      checked-staged-security-with-query-prefix-current-empty-trace-path-output-hit-at-def
      checked-staged-security-with-query-prefix-empty-trace-path-output-hit-def
    by simp
next
  case (Some packed)
  then obtain prefix prefix-state raw raw-state data attacker-state result
    final-state where out-eq:
    out =
      Some (((((prefix, prefix-state), raw), raw-state),
        ((data, attacker-state), result)), final-state)
    by (cases packed) (auto split: prod.splits)
  have mapped-support:
    Some (((data, attacker-state), result), final-state) ∈
      set-dist
        (execute
          (checked-staged-security-experiment-with-query-prefix-data-state A i >>=
            (λpacked. return (snd packed))))
          adversary-initial-state)
    unfolding out-eq
    by (rule set-dist-bindI[OF support[unfolded out-eq]]) simp
  have projection:
    checked-staged-security-experiment-with-query-prefix-data-state A i >>=
      (λpacked. return (snd packed)) =
      checked-staged-security-experiment-with-data-state A
    by (rule checked-staged-security-experiment-with-query-prefix-data-state-projection
      [OF i-bound])
  have data-support:
    Some (((data, attacker-state), result), final-state) ∈
      set-dist
        (execute (checked-staged-security-experiment-with-data-state A)
          adversary-initial-state)
    using mapped-support unfolding projection .
  let ?s =
    verifier-state-from-adversary attacker-state
      (staged-proof-transcript data)
  from checked-staged-security-experiment-with-data-state-outcomeE
    [OF data-support]
  have verifier:
    Some (result, final-state) ∈ set-dist (execute verify-monad ?s)
    by blast
  have transcript-split:
    hash-map-output-values ?s ∩ set (staged-proof-transcript data)

```

```

    ≠ {} ∨
    hash-map-new-output-hit (set (staged-proof-transcript data))
    ?s final-state
  by (rule verify-monad-trace-root-target-preexisting-or-new[OF verifier])
    simp
show ?thesis
  unfolding out-eq
    checked-staged-security-with-query-prefix-transcript-hit-def
  using transcript-split by simp
qed
qed
have transcript-bound:
  wp-event
    (checked-staged-security-experiment-with-query-prefix-data-state A i)
    checked-staged-security-with-query-prefix-transcript-hit
    adversary-initial-state ≤ P + R
  by (rule
    checked-staged-security-with-query-prefix-transcript-hit-bound-from-data-state
    [OF i-bound pre-bound new-bound])
show ?thesis
  by (rule order-trans[OF event-le transcript-bound])
qed

lemma checked-staged-security-with-data-state-current-empty-query-opening-hit-at-bound-from-conceptual-empty-residual-hit-at
  assumes wf: staged-budget-wellformed budgets
  and controlled: staged-adversary-controlled budgets A
  and i-bound: i < rounds
  and residual-bound:
    wp-event
      (checked-staged-security-experiment-with-query-prefix-data-state A i)
      (checked-staged-security-with-query-prefix-conceptual-empty-context-residual-hit-at i)
      adversary-initial-state ≤ E
  and pre-bound:
    wp-event (checked-staged-security-experiment-with-data-state A)
      staged-security-with-data-state-transcript-pre-hit
      adversary-initial-state ≤ P
  and new-bound:
    wp-event (checked-staged-security-experiment-with-data-state A)
      staged-security-with-data-state-transcript-new-hit
      adversary-initial-state ≤ R
  shows
    wp-event
      (checked-staged-security-experiment-with-data-state A)
      (staged-security-with-data-state-current-empty-query-opening-hit-at i)
      adversary-initial-state ≤
      staged-phase-relation-error size
      (staged-query-search-queries budgets i + 1) +
      query-error-bound + E + (P + R)

```

proof –

have *path-bound*:

wp-event

(*checked-staged-security-experiment-with-query-prefix-data-state A i*)

(*checked-staged-security-with-query-prefix-current-empty-trace-path-output-hit-at i*)

adversary-initial-state $\leq P + R$

by (*rule*

checked-staged-security-with-query-prefix-current-empty-trace-path-output-hit-at-bound-from-transcript
[*OF i-bound pre-bound new-bound*])

show *?thesis*

by (*rule*

checked-staged-security-with-data-state-current-empty-query-opening-hit-at-bound-from-conceptual-empty-residual
[*OF wf controlled i-bound residual-bound path-bound*])

qed

lemma *checked-staged-security-with-data-state-empty-header-bad-event-bound-from-prefix-drift-trace-fri-and-current-*

assumes *false-statement*: $\neg \text{exists-valid-trace}$

and *wf*: *staged-budget-wellformed budgets*

and *controlled*: *staged-adversary-controlled budgets A*

and *prefix-bound*:

wp-event

(*checked-staged-security-experiment-with-actual-alpha-prefix-data-state A*)

(*checked-staged-security-with-actual-alpha-prefix-bad-set-hit composition-trace-bad-alpha-space*)

adversary-initial-state $\leq P$

and *drift-bound*:

wp-event

(*checked-staged-security-experiment-with-actual-alpha-prefix-data-state A*)

(*checked-staged-security-with-actual-alpha-prefix-partial-header-candidate-drift composition-trace-bad-alpha-space*)

adversary-initial-state $\leq D$

and *trace-fri-bound*:

wp-event (*checked-staged-security-experiment-with-data-state A*)

(*staged-security-with-data-state-verifier-event trace-fri-bad-with-empty-composition-header-candidates*)

adversary-initial-state $\leq \text{trace-fri-error}$

and *current-empty-bound*:

wp-event (*checked-staged-security-experiment-with-data-state A*)

(*staged-security-with-data-state-current-empty-query-opening-hit-at 0*)

adversary-initial-state $\leq \text{empty-query-error}$

shows

wp-event (*checked-staged-security-experiment-with-data-state A*)

(*staged-security-with-data-state-verifier-event soundness-bad-event-partial-candidate-empty-header*)

adversary-initial-state $\leq$

$P + D +$

```

    staged-phase-relation-error size
    (staged-alpha-search-queries budgets 0) +
    trace-fri-error + empty-query-error
proof –
  have query-bound:
    wp-event (checked-staged-security-experiment-with-data-state A)
    (staged-security-with-data-state-verifier-event
     query-bad-with-empty-composition-header-candidates)
    adversary-initial-state ≤ empty-query-error
  by (rule order-trans[
    OF checked-staged-security-with-data-state-empty-query-bad-le-current-empty-opening-hit-at-zero[
      OF wf controlled] current-empty-bound])
  show ?thesis
  by (rule
    checked-staged-security-with-data-state-empty-header-bad-event-bound-from-prefix-drift-trace-fri-and-query
    [OF false-statement wf controlled prefix-bound drift-bound
     trace-fri-bound query-bound])
qed

end

end

```

theory *Soundness-Staged-Aligned-Partial-Merkle*

imports

Soundness-Staged-Partial-Merkle
Soundness-Aligned-Partial-Query-Transcript
Staged-Security-Experiment-Alpha-Header-Binding
Staged-Security-Experiment-Composition-Query-Current-Alignment
Soundness-Conceptual-Query-Empty-Current

begin

Staged partial-Merkle packaging that uses aligned nonempty-header query evidence. This downstream layer avoids importing verifier-query extraction machinery into the upstream deterministic reductions.

context *soundness*

begin

lemma *partial-merkle-inconsistency-bad-imp-aligned-partial-candidate-with-partial-merkle:*

assumes *partial-merkle-inconsistency-bad s out*

shows

soundness-bad-event-aligned-partial-candidate-with-partial-merkle s out

using *assms*

unfolding

soundness-bad-event-aligned-partial-candidate-with-partial-merkle-def

by *simp*

lemma *soundness-bad-event-aligned-partial-candidate-imp-with-partial-merkle:*

assumes *soundness-bad-event-aligned-partial-candidate s out*
shows
soundness-bad-event-aligned-partial-candidate-with-partial-merkle s out
using *assms*
unfolding
soundness-bad-event-aligned-partial-candidate-with-partial-merkle-def
by *simp*

lemma *accepted-partition-soundness-bad-event-aligned-partial-candidate-with-partial-merkle-if-header-nonempty:*

assumes *false-statement: $\neg$ exists-valid-trace*
and *supp: $out \in \text{set-dist } (\text{execute verify-monad } s)$*
and *acc: accepted out*
and *header:*
verifier-header-transcript s fr f-fri-roots f-final as dg
composition-fri-roots final rest
and *comp-nonempty: composition-fri-roots $\neq []$*
shows

soundness-bad-event-aligned-partial-candidate-with-partial-merkle s out

proof –

from *acc* **obtain** *result final-state* **where** *out-eq:*
out = Some (result, final-state)

unfolding *accepted-def* **by** *(cases out) auto*

from

verify-monad-accepted-with-partial-initial-openings-aligned-consistent
[OF supp[unfolded out-eq] header comp-nonempty]

obtain *query-idxs trace-openings composition-openings* **where** *partial:*

accepted-with-partial-initial-openings-aligned-consistent s
(Some (result, final-state)) fr f-fri-roots f-final as dg
composition-fri-roots final query-idxs trace-openings
composition-openings

by *blast*

show *?thesis*

proof *(cases*

partial-merkle-inconsistency-bad s (Some (result, final-state)))

case *True*

then show *?thesis*

unfolding *out-eq*

by *(rule*

partial-merkle-inconsistency-bad-imp-aligned-partial-candidate-with-partial-merkle)

next

case *False*

have *aligned:*

accepted-with-partial-initial-openings-aligned s
(Some (result, final-state)) fr f-fri-roots f-final as dg
composition-fri-roots final query-idxs trace-openings
composition-openings

by *(rule*

accepted-with-partial-initial-openings-aligned-consistent-imp-aligned
[OF partial])

obtain *trace-table composition-table* **where** *trace-candidate:*
partial-trace-table-candidate trace-table trace-openings
and *composition-candidate:*
partial-composition-table-candidate composition-table
composition-openings
using
accepted-with-partial-initial-openings-aligned-candidates-if-no-partial-merkle-bad
[OF aligned False]
by *blast*
have *bad:*
soundness-bad-event-aligned-partial-candidate s
(Some (result, final-state))
by *(rule accepted-aligned-partial-candidate-partition*
[OF false-statement partial trace-candidate composition-candidate])
then show *?thesis*
unfolding *out-eq*
by *(rule soundness-bad-event-aligned-partial-candidate-imp-with-partial-merkle)*
qed
qed

lemma *accepted-partition-soundness-bad-event-aligned-transcript-partial-candidate-with-partial-merkle-if-header-non*
assumes *false-statement: $\neg$ exists-valid-trace*
and *supp: out $\in$ set-dist (execute verify-monad s)*
and *acc: accepted out*
and *header:*
verifier-header-transcript s fr f-fri-roots f-final as dg
composition-fri-roots final rest
and *comp-nonempty: composition-fri-roots $\neq$ []*
shows
soundness-bad-event-aligned-transcript-partial-candidate-with-partial-merkle
s out

proof –
from *acc* **obtain** *result final-state* **where** *out-eq:*
out = Some (result, final-state)
unfolding *accepted-def* **by** *(cases out) auto*
from *verify-monad-supplied-header-transcript-shape*
[OF supp[unfolded out-eq] header]
obtain *query-idxs raw-idxs query-chunks* **where** *shape:*
accepted-transcript-shape s (Some (result, final-state)) as query-idxs
and *len-raw: length raw-idxs = rounds*
and *query-idxs-eq:*
query-idxs = map (λ raw. index (to-nat raw)) raw-idxs
and *len-query-chunks: length query-chunks = rounds*
and *transcript-rest:*
List.concat query-chunks @ PTranscript final-state = rest
and *query-chunk-header:*
 $\bigwedge i. i < rounds \implies$
verifier-query-round-chunk (query-idxs ! i)
f-fri-roots composition-fri-roots (query-chunks ! i)

```

and replay-lookup-header:
   $\wedge i. i < \text{rounds} \implies$ 
    fmlookup (HashMap final-state)
      (QueryIndexChallenge (PQueryCounter s + i)
        (state-after-query-chunks
          (verifier-header-state s fr f-fri-roots f-final as dg
            composition-fri-roots final)
            query-chunks i)) =
        Some (raw-idxs ! i)
  by blast
from verify-monad-supplied-header-query-rounds
  [OF supp[unfolded out-eq] header]
obtain f-fl fl query-state where f-roots:
  map snd f-fl = f-fri-roots
and comp-roots: map snd fl = composition-fri-roots
and query-transcript-state: PTranscript query-state = rest
and query-hash-state:
  PState query-state =
    verifier-header-state s fr f-fri-roots f-final as dg
    composition-fri-roots final
and query-counter: PQueryCounter query-state = PQueryCounter s
and query-out:
  Some (result, final-state) ∈
    set-dist
    (execute
      (ntimes
        (verifier-query-round-program fr f-fl f-final as fl final)
        rounds)
      query-state)
  by blast
have fl-nonempty: fl ≠ []
  using comp-nonempty comp-roots by (cases fl) simp-all
then obtain b composition-root fl-tail where fl-eq:
  fl = (b, composition-root) # fl-tail
  by (cases fl) auto
have transcript-query:
  PTranscript query-state =
    List.concat query-chunks @ PTranscript final-state
  using query-transcript-state transcript-rest by simp
have query-chunk:
   $\wedge i. i < \text{rounds} \implies$ 
    verifier-query-round-chunk (query-idxs ! i)
    (map snd f-fl) (map snd fl) (query-chunks ! i)
  using query-chunk-header f-roots comp-roots by simp
have replay-lookup:
   $\wedge i. i < \text{rounds} \implies$ 
    fmlookup (HashMap final-state)
      (QueryIndexChallenge (PQueryCounter query-state + i)
        (state-after-query-chunks (PState query-state) query-chunks i)) =

```

```

    Some (raw-idxs ! i)
using replay-lookup-header query-hash-state query-counter by simp
from
  query-rounds-replay-accepted-with-partial-initial-openings-aligned-transcript-consistent
  [OF header comp-nonempty fl-eq f-roots comp-roots
   query-transcript-state query-hash-state query-counter query-out
   len-raw query-idxs-eq len-query-chunks transcript-query query-chunk
   replay-lookup]
obtain trace-openings composition-openings where partial:
  accepted-with-partial-initial-openings-aligned-transcript-consistent s
  (Some (result, final-state)) fr f-fri-roots f-final as dg
  composition-fri-roots final query-idxs trace-openings
  composition-openings
by blast
show ?thesis
proof (cases
  partial-merkle-inconsistency-bad s (Some (result, final-state)))
case True
then show ?thesis
  unfolding out-eq
  soundness-bad-event-aligned-transcript-partial-candidate-with-partial-merkle-def
  by simp
next
case False
have aligned0:
  accepted-with-partial-initial-openings-aligned s
  (Some (result, final-state)) fr f-fri-roots f-final as dg
  composition-fri-roots final query-idxs trace-openings
  composition-openings
  using partial
  by (blast dest:
    accepted-with-partial-initial-openings-aligned-transcript-consistent-imp-aligned
    accepted-with-partial-initial-openings-aligned-consistent-imp-aligned)
obtain trace-table composition-table where trace-candidate:
  partial-trace-table-candidate trace-table trace-openings
and composition-candidate:
  partial-composition-table-candidate composition-table
  composition-openings
using
  accepted-with-partial-initial-openings-aligned-candidates-if-no-partial-merkle-bad
  [OF aligned0 False]
by blast
have bad:
  soundness-bad-event-aligned-transcript-partial-candidate s
  (Some (result, final-state))
  by (rule accepted-aligned-transcript-partial-candidate-partition
    [OF false-statement partial trace-candidate composition-candidate])
then show ?thesis
  unfolding out-eq

```



```

    by (simp add:
        soundness-bad-event-aligned-transcript-partial-candidate-with-partial-merkle-def)
qed
qed

lemma accepted-partition-soundness-bad-event-aligned-partial-candidate-with-partial-merkle-or-empty-header:
  assumes false-statement:  $\neg$  exists-valid-trace
    and supp:  $out \in \text{set-dist } (\text{execute verify-monad } s)$ 
    and acc: accepted out
    and header:
      verifier-header-transcript s fr f-fri-roots f-final as dg
      composition-fri-roots final rest
  shows
    soundness-bad-event-aligned-partial-candidate-with-partial-merkle s out  $\vee$ 
    soundness-bad-event-partial-candidate-empty-header s out
proof (cases composition-fri-roots = [])
case False
  then have comp-nonempty: composition-fri-roots  $\neq []$ 
    by simp
  have soundness-bad-event-aligned-partial-candidate-with-partial-merkle s
    out
    by (rule
        accepted-partition-soundness-bad-event-aligned-partial-candidate-with-partial-merkle-if-header-nonempty
        [OF false-statement supp acc header comp-nonempty])
  then show ?thesis by simp
next
case True
  note comp-empty = True
  from acc obtain result final-state where out-eq:
    out = Some (result, final-state)
    unfolding accepted-def by (cases out) auto
  show ?thesis
  proof (cases
      partial-merkle-inconsistency-bad s (Some (result, final-state)))
  case True
    then have
      soundness-bad-event-aligned-partial-candidate-with-partial-merkle s out
      unfolding out-eq
      by (rule
          partial-merkle-inconsistency-bad-imp-aligned-partial-candidate-with-partial-merkle)
    then show ?thesis by simp
  next
  case False
    from verify-monad-accepted-with-partial-trace-openings-for-header
      [OF supp[unfolded out-eq] header]
    obtain trace-query-idxs trace-openings where trace-partial:
      accepted-with-partial-trace-openings s
      (Some (result, final-state)) fr trace-query-idxs trace-openings
    by blast

```

```

obtain trace-table where trace-candidate:
  partial-trace-table-candidate trace-table trace-openings
using accepted-with-partial-trace-openings-candidate-if-no-partial-merkle-bad
  [OF trace-partial False]
by blast
have accepted-out: accepted (Some (result, final-state))
  by (rule accepted-with-partial-trace-openings-imp-accepted
    [OF trace-partial])
let ?composition-table = replicate (scale * clength) final
have empty-partial:
  accepted-with-empty-composition-header-candidates s
    (Some (result, final-state)) fr f-fri-roots f-final as dg final
    trace-query-idxs trace-openings trace-table ?composition-table
unfolding accepted-with-empty-composition-header-candidates-def
by (intro conjI exI [of - rest])
  (use accepted-out header comp-empty trace-partial trace-candidate
    in simp-all)
have soundness-bad-event-partial-candidate-empty-header s
  (Some (result, final-state))
  by (rule accepted-empty-composition-header-candidate-partition
    [OF false-statement empty-partial])
then show ?thesis
  unfolding out-eq by simp
qed
qed

lemma accepted-partition-soundness-bad-event-aligned-transcript-partial-candidate-with-partial-merkle-or-empty-header
assumes false-statement:  $\neg$  exists-valid-trace
and supp: out  $\in$  set-dist (execute verify-monad s)
and acc: accepted out
and header:
  verifier-header-transcript s fr f-fri-roots f-final as dg
  composition-fri-roots final rest
shows
  soundness-bad-event-aligned-transcript-partial-candidate-with-partial-merkle
    s out  $\vee$ 
    soundness-bad-event-partial-candidate-empty-header s out
proof (cases composition-fri-roots = [])
case False
then have comp-nonempty: composition-fri-roots  $\neq$  []
  by simp
have
  soundness-bad-event-aligned-transcript-partial-candidate-with-partial-merkle
    s out
  by (rule
    accepted-partition-soundness-bad-event-aligned-transcript-partial-candidate-with-partial-merkle-if-header-nonempty
    [OF false-statement supp acc header comp-nonempty])
then show ?thesis by simp
next

```

```

case True
note comp-empty = True
from acc obtain result final-state where out-eq:
  out = Some (result, final-state)
  unfolding accepted-def by (cases out) auto
show ?thesis
proof (cases
  partial-merkle-inconsistency-bad s (Some (result, final-state)))
case True
then have
  soundness-bad-event-aligned-transcript-partial-candidate-with-partial-merkle
  s out
  unfolding out-eq
  soundness-bad-event-aligned-transcript-partial-candidate-with-partial-merkle-def
  by simp
then show ?thesis by simp
next
case False
from verify-monad-accepted-with-partial-trace-openings-for-header
  [OF supp[unfolded out-eq] header]
obtain trace-query-idxs trace-openings where trace-partial:
  accepted-with-partial-trace-openings s
  (Some (result, final-state)) fr trace-query-idxs trace-openings
by blast
obtain trace-table where trace-candidate:
  partial-trace-table-candidate trace-table trace-openings
using accepted-with-partial-trace-openings-candidate-if-no-partial-merkle-bad
  [OF trace-partial False]
by blast
have accepted-out: accepted (Some (result, final-state))
by (rule accepted-with-partial-trace-openings-imp-accepted
  [OF trace-partial])
let ?composition-table = replicate (scale * clength) final
have empty-partial:
  accepted-with-empty-composition-header-candidates s
  (Some (result, final-state)) fr f-fri-roots f-final as dg final
  trace-query-idxs trace-openings trace-table ?composition-table
unfolding accepted-with-empty-composition-header-candidates-def
by (intro conjI exI[of - rest])
  (use accepted-out header comp-empty trace-partial trace-candidate
  in simp-all)
have soundness-bad-event-partial-candidate-empty-header s
  (Some (result, final-state))
by (rule accepted-empty-composition-header-candidate-partition
  [OF false-statement empty-partial])
then show ?thesis
  unfolding out-eq by simp
qed
qed

```

lemma *checked-staged-soundness-from-aligned-partial-candidate-and-empty-header-bounds:*
fixes *partial-candidate-error empty-header-error :: prob*
assumes *false-statement: $\neg$ exists-valid-trace*
and *partial-candidate-bound:*
 wp-event (checked-staged-security-experiment-with-data-state A)
 (staged-security-with-data-state-verifier-event
 soundness-bad-event-aligned-partial-candidate-with-partial-merkle)
 adversary-initial-state $\leq$ partial-candidate-error
and *empty-header-bound:*
 wp-event (checked-staged-security-experiment-with-data-state A)
 (staged-security-with-data-state-verifier-event
 soundness-bad-event-partial-candidate-empty-header)
 adversary-initial-state $\leq$ empty-header-error
shows *checked-staged-adversary-acceptance-probability A $\leq$*
 partial-candidate-error + empty-header-error
proof –
 let *?M = checked-staged-security-experiment-with-data-state A*
 let *?E = staged-security-with-data-state-verifier-event*
 let *?partial =*
 ?E soundness-bad-event-aligned-partial-candidate-with-partial-merkle
 let *?empty = ?E soundness-bad-event-partial-candidate-empty-header*
 let *?bad = λ out. ?partial out $\vee$?empty out*
 have *accepted-bad:*
 wp-event ?M accepted adversary-initial-state $\leq$
 wp-event ?M ?bad adversary-initial-state
 proof (rule *wp-event-mono-on-support*)
 fix *out*
 assume *support: out $\in$ set-dist (execute ?M adversary-initial-state)*
 and *acc: accepted out*
 show *?bad out*
 proof (cases *out*)
 case *None*
 then show *?thesis*
 using *acc unfolding accepted-def by simp*
 next
 case (Some *packed*)
 then obtain *data attacker-state result final-state* **where** *out-eq:*
 out = Some (((data, attacker-state), result), final-state)
 by (cases *packed*, auto *split: prod.splits*)
 let *?s =*
 verifier-state-from-adversary attacker-state
 (staged-proof-transcript data)
 have *verifier:*
 Some (result, final-state) $\in$ set-dist (execute verify-monad ?s)
 using *checked-staged-security-experiment-with-data-state-outcomeE*
 [OF support[unfolded out-eq]]
 by *blast*
 have *verifier-acc: accepted (Some (result, final-state))*

```

    unfolding accepted-def by simp
  from verify-monad-accepted-transcript-shape[OF verifier]
  obtain alphas query-idxs where shape:
    accepted-transcript-shape ?s (Some (result, final-state))
    alphas query-idxs
  by blast
  from shape obtain result' final-state' fr f-fri-roots f-final dg
    composition-fri-roots final rest where
    header:
      verifier-header-transcript ?s fr f-fri-roots f-final alphas dg
      composition-fri-roots final rest
  by (elim accepted-transcript-shape-header-query-extraction)
  have local-bad:
    soundness-bad-event-aligned-partial-candidate-with-partial-merkle ?s
      (Some (result, final-state))  $\vee$ 
    soundness-bad-event-partial-candidate-empty-header ?s
      (Some (result, final-state))
  by (rule
    accepted-partition-soundness-bad-event-aligned-partial-candidate-with-partial-merkle-or-empty-header
      [OF false-statement verifier verifier-acc header])
  then show ?thesis
    unfolding out-eq staged-security-with-data-state-verifier-event-def
    by simp
qed
qed
have bad-bound:
  wp-event ?M ?bad adversary-initial-state  $\leq$ 
  partial-candidate-error + empty-header-error
proof -
  have wp-event ?M ?bad adversary-initial-state  $\leq$ 
    wp-event ?M ?partial adversary-initial-state +
    wp-event ?M ?empty adversary-initial-state
  by (rule wp-event-union-bound)
  also have ...  $\leq$  partial-candidate-error + empty-header-error
  by (intro add-mono partial-candidate-bound empty-header-bound)
  finally show ?thesis .
qed
have staged-bound:
  wp-event (checked-staged-security-experiment A) accepted
    adversary-initial-state  $\leq$ 
    partial-candidate-error + empty-header-error
  unfolding checked-staged-security-experiment-acceptance-with-data-state
  by (rule order-trans[OF accepted-bad bad-bound])
show ?thesis
proof -
  have
    wp-event (checked-staged-security-experiment A)
      ( $\lambda$ out.  $\neg$  Option.is-none out) adversary-initial-state =
    wp-event (checked-staged-security-experiment A) accepted

```

```

    adversary-initial-state
  unfolding wp-event-def accepted-def
  by (rule arg-cong[where
    f= $\lambda Q. wp \text{ (checked-staged-security-experiment } A) \ Q$ 
    adversary-initial-state])
    (rule ext, simp)
  then show ?thesis
    unfolding checked-staged-adversary-acceptance-probability-def
    using staged-bound by simp
qed
qed

lemma checked-staged-soundness-from-aligned-transcript-partial-candidate-and-empty-header-bounds:
  fixes partial-candidate-error empty-header-error :: prob
  assumes false-statement:  $\neg \text{exists-valid-trace}$ 
  and partial-candidate-bound:
    wp-event (checked-staged-security-experiment-with-data-state A)
      (staged-security-with-data-state-verifier-event
        soundness-bad-event-aligned-transcript-partial-candidate-with-partial-merkle)
    adversary-initial-state  $\leq$  partial-candidate-error
  and empty-header-bound:
    wp-event (checked-staged-security-experiment-with-data-state A)
      (staged-security-with-data-state-verifier-event
        soundness-bad-event-partial-candidate-empty-header)
    adversary-initial-state  $\leq$  empty-header-error
  shows checked-staged-adversary-acceptance-probability A  $\leq$ 
    partial-candidate-error + empty-header-error
proof -
  let ?M = checked-staged-security-experiment-with-data-state A
  let ?E = staged-security-with-data-state-verifier-event
  let ?partial =
    ?E soundness-bad-event-aligned-transcript-partial-candidate-with-partial-merkle
  let ?empty = ?E soundness-bad-event-partial-candidate-empty-header
  let ?bad =  $\lambda out. ?partial \ out \vee ?empty \ out$ 
  have accepted-bad:
    wp-event ?M accepted adversary-initial-state  $\leq$ 
    wp-event ?M ?bad adversary-initial-state
  proof (rule wp-event-mono-on-support)
    fix out
    assume support:  $out \in \text{set-dist (execute ?M adversary-initial-state)}$ 
    and acc: accepted out
    show ?bad out
    proof (cases out)
      case None
      then show ?thesis
        using acc unfolding accepted-def by simp
    next
      case (Some packed)
      then obtain data attacker-state result final-state where out-eq:

```

```

    out = Some (((data, attacker-state), result), final-state)
  by (cases packed, auto split: prod.splits)
let ?s =
  verifier-state-from-adversary attacker-state
  (staged-proof-transcript data)
have verifier:
  Some (result, final-state) ∈ set-dist (execute verify-monad ?s)
  using checked-staged-security-experiment-with-data-state-outcomeE
  [OF support[unfolded out-eq]]
  by blast
have verifier-acc: accepted (Some (result, final-state))
  unfolding accepted-def by simp
from verify-monad-accepted-transcript-shape[OF verifier]
obtain alphas query-idxs where shape:
  accepted-transcript-shape ?s (Some (result, final-state))
  alphas query-idxs
  by blast
from shape obtain result' final-state' fr f-fri-roots f-final dg
  composition-fri-roots final rest where
  header:
    verifier-header-transcript ?s fr f-fri-roots f-final alphas dg
    composition-fri-roots final rest
  by (elim accepted-transcript-shape-header-query-extraction)
have local-bad:
  soundness-bad-event-aligned-transcript-partial-candidate-with-partial-merkle
    ?s (Some (result, final-state)) ∨
  soundness-bad-event-partial-candidate-empty-header ?s
    (Some (result, final-state))
  by (rule
    accepted-partition-soundness-bad-event-aligned-transcript-partial-candidate-with-partial-merkle-or-empty-header)
  [OF false-statement verifier verifier-acc header])
then show ?thesis
  unfolding out-eq staged-security-with-data-state-verifier-event-def
  by simp
qed
qed
have bad-bound:
  wp-event ?M ?bad adversary-initial-state ≤
  partial-candidate-error + empty-header-error
proof -
  have wp-event ?M ?bad adversary-initial-state ≤
    wp-event ?M ?partial adversary-initial-state +
    wp-event ?M ?empty adversary-initial-state
  by (rule wp-event-union-bound)
  also have ... ≤ partial-candidate-error + empty-header-error
  by (intro add-mono partial-candidate-bound empty-header-bound)
  finally show ?thesis .
qed
have staged-bound:

```

```

wp-event (checked-staged-security-experiment A) accepted
  adversary-initial-state ≤
    partial-candidate-error + empty-header-error
unfolding checked-staged-security-experiment-acceptance-with-data-state
by (rule order-trans[OF accepted-bad bad-bound])
show ?thesis
proof –
  have
    wp-event (checked-staged-security-experiment A)
      (λout. ¬ Option.is-none out) adversary-initial-state =
        wp-event (checked-staged-security-experiment A) accepted
          adversary-initial-state
    unfolding wp-event-def accepted-def
    by (rule arg-cong[where
      f=λQ. wp (checked-staged-security-experiment A) Q
      adversary-initial-state])
      (rule ext, simp)
    then show ?thesis
      unfolding checked-staged-adversary-acceptance-probability-def
      using staged-bound by simp
  qed
qed

lemma checked-staged-soundness-from-aligned-partial-candidate-components-and-empty-header-bound:
fixes merkle-error composition-error trace-fri-error'
  composition-fri-error' query-error empty-header-error :: prob
assumes false-statement: ¬ exists-valid-trace
and merkle-bound:
  wp-event (checked-staged-security-experiment-with-data-state A)
    (staged-security-with-data-state-verifier-event
      partial-merkle-inconsistency-bad)
    adversary-initial-state ≤ merkle-error
and comp-bound:
  wp-event (checked-staged-security-experiment-with-data-state A)
    (staged-security-with-data-state-verifier-event
      composition-bad-with-partial-candidates)
    adversary-initial-state ≤ composition-error
and trace-fri-bound:
  wp-event (checked-staged-security-experiment-with-data-state A)
    (staged-security-with-data-state-verifier-event
      trace-fri-bad-with-partial-candidates)
    adversary-initial-state ≤ trace-fri-error'
and comp-fri-bound:
  wp-event (checked-staged-security-experiment-with-data-state A)
    (staged-security-with-data-state-verifier-event
      composition-fri-bad-with-partial-candidates)
    adversary-initial-state ≤ composition-fri-error'
and query-bound:
  wp-event (checked-staged-security-experiment-with-data-state A)

```


(staged-security-with-data-state-verifier-event
query-bad-with-aligned-partial-candidates)
adversary-initial-state $\leq$ *query-error*
and *empty-header-bound*:
wp-event (checked-staged-security-experiment-with-data-state A)
(staged-security-with-data-state-verifier-event
soundness-bad-event-partial-candidate-empty-header)
adversary-initial-state $\leq$ *empty-header-error*
shows *checked-staged-adversary-acceptance-probability A* $\leq$
merkle-error + composition-error + trace-fri-error' +
composition-fri-error' + query-error + empty-header-error
proof –
have *partial-candidate-bound*:
wp-event (checked-staged-security-experiment-with-data-state A)
(staged-security-with-data-state-verifier-event
soundness-bad-event-aligned-partial-candidate-with-partial-merkle)
adversary-initial-state $\leq$
merkle-error + composition-error + trace-fri-error' +
composition-fri-error' + query-error
by (rule
checked-staged-security-with-data-state-aligned-partial-candidate-with-partial-merkle-union-bound
[OF merkle-bound comp-bound trace-fri-bound comp-fri-bound
query-bound])
have *bound*:
checked-staged-adversary-acceptance-probability A $\leq$
(merkle-error + composition-error + trace-fri-error' +
composition-fri-error' + query-error) + empty-header-error
by (rule
checked-staged-soundness-from-aligned-partial-candidate-and-empty-header-bounds
[OF false-statement partial-candidate-bound empty-header-bound])
then show *?thesis*
by (*simp add: algebra-simps*)
qed

lemma *checked-staged-soundness-from-aligned-transcript-partial-candidate-components-and-empty-header-bound*:
fixes *merkle-error composition-error trace-fri-error'*
composition-fri-error' query-error empty-header-error :: prob
assumes *false-statement: $\neg$ exists-valid-trace*
and *merkle-bound*:
wp-event (checked-staged-security-experiment-with-data-state A)
(staged-security-with-data-state-verifier-event
partial-merkle-inconsistency-bad)
adversary-initial-state $\leq$ *merkle-error*
and *comp-bound*:
wp-event (checked-staged-security-experiment-with-data-state A)
(staged-security-with-data-state-verifier-event
composition-bad-with-partial-candidates)
adversary-initial-state $\leq$ *composition-error*
and *trace-fri-bound*:

```

    wp-event (checked-staged-security-experiment-with-data-state A)
      (staged-security-with-data-state-verifier-event
        trace-fri-bad-with-partial-candidates)
      adversary-initial-state  $\leq$  trace-fri-error'
  and comp-fri-bound:
    wp-event (checked-staged-security-experiment-with-data-state A)
      (staged-security-with-data-state-verifier-event
        composition-fri-bad-with-partial-candidates)
      adversary-initial-state  $\leq$  composition-fri-error'
  and query-bound:
    wp-event (checked-staged-security-experiment-with-data-state A)
      (staged-security-with-data-state-verifier-event
        query-bad-with-aligned-transcript-partial-candidates)
      adversary-initial-state  $\leq$  query-error
  and empty-header-bound:
    wp-event (checked-staged-security-experiment-with-data-state A)
      (staged-security-with-data-state-verifier-event
        soundness-bad-event-partial-candidate-empty-header)
      adversary-initial-state  $\leq$  empty-header-error
  shows checked-staged-adversary-acceptance-probability A  $\leq$ 
    merkle-error + composition-error + trace-fri-error' +
    composition-fri-error' + query-error + empty-header-error
proof -
  have partial-candidate-bound:
    wp-event (checked-staged-security-experiment-with-data-state A)
      (staged-security-with-data-state-verifier-event
        soundness-bad-event-aligned-transcript-partial-candidate-with-partial-merkle)
      adversary-initial-state  $\leq$ 
        merkle-error + composition-error + trace-fri-error' +
        composition-fri-error' + query-error
  by (rule
    checked-staged-security-with-data-state-aligned-transcript-partial-candidate-with-partial-merkle-component-union
      [OF merkle-bound comp-bound trace-fri-bound comp-fri-bound
        query-bound])
  have bound:
    checked-staged-adversary-acceptance-probability A  $\leq$ 
      (merkle-error + composition-error + trace-fri-error' +
        composition-fri-error' + query-error) + empty-header-error
  by (rule
    checked-staged-soundness-from-aligned-transcript-partial-candidate-and-empty-header-bounds
      [OF false-statement partial-candidate-bound empty-header-bound])
  then show ?thesis
  by (simp add: algebra-simps)
qed

lemma checked-staged-soundness-from-aligned-partial-candidate-components-and-empty-header-components:
  fixes trace-fri-error' composition-fri-error' query-error P D :: prob
  assumes false-statement:  $\neg$  exists-valid-trace
  and wf: staged-budget-wellformed budgets

```

and *controlled: staged-adversary-controlled budgets* A
and *trace-fri-bound:*
 wp-event (*checked-staged-security-experiment-with-data-state* A)
 (*staged-security-with-data-state-verifier-event*
 trace-fri-bad-with-partial-candidates)
 adversary-initial-state $\leq$ *trace-fri-error'*
and *comp-fri-bound:*
 wp-event (*checked-staged-security-experiment-with-data-state* A)
 (*staged-security-with-data-state-verifier-event*
 composition-fri-bad-with-partial-candidates)
 adversary-initial-state $\leq$ *composition-fri-error'*
and *query-bound:*
 wp-event (*checked-staged-security-experiment-with-data-state* A)
 (*staged-security-with-data-state-verifier-event*
 query-bad-with-aligned-partial-candidates)
 adversary-initial-state $\leq$ *query-error*
and *prefix-bound:*
 wp-event
 (*checked-staged-security-experiment-with-actual-alpha-prefix-data-state*
 A)
 (*checked-staged-security-with-actual-alpha-prefix-bad-set-hit*
 composition-trace-bad-alpha-space)
 adversary-initial-state $\leq$ P
and *drift-bound:*
 wp-event
 (*checked-staged-security-experiment-with-actual-alpha-prefix-data-state*
 A)
 (*checked-staged-security-with-actual-alpha-prefix-partial-header-candidate-drift*
 composition-trace-bad-alpha-space)
 adversary-initial-state $\leq$ D
and *empty-trace-fri-bound:*
 wp-event (*checked-staged-security-experiment-with-data-state* A)
 (*staged-security-with-data-state-verifier-event*
 trace-fri-bad-with-empty-composition-header-candidates)
 adversary-initial-state $\leq$ *trace-fri-error*
and *empty-query-bound:*
 wp-event (*checked-staged-security-experiment-with-data-state* A)
 (*staged-security-with-data-state-verifier-event*
 query-bad-with-empty-composition-header-candidates)
 adversary-initial-state $\leq$ *nnreal rounds* * *query-error-bound*
shows *checked-staged-adversary-acceptance-probability* $A \leq$
 hash-collision-budget-value 0
 (*staged-attacker-query-budget* *budgets* +
 staged-challenge-query-budget + *verifier-hash-query-budget*) +
 $(P + D +$
 staged-phase-relation-error size
 (*staged-alpha-search-queries* *budgets* 0)) +
 trace-fri-error' + composition-fri-error' +
 query-error +

$(P + D +$
 $\text{staged-phase-relation-error size}$
 $(\text{staged-alpha-search-queries budgets } 0) +$
 $\text{trace-fri-error} + \text{nnreal rounds} * \text{query-error-bound})$
proof –
have *comp-bound*:
 $\text{wp-event } (\text{checked-staged-security-experiment-with-data-state } A)$
 $(\text{staged-security-with-data-state-verifier-event}$
 $\text{composition-bad-with-partial-candidates})$
 $\text{adversary-initial-state} \leq$
 $P + D +$
 $\text{staged-phase-relation-error size}$
 $(\text{staged-alpha-search-queries budgets } 0)$
by (rule
 $\text{checked-staged-security-with-data-state-partial-candidate-composition-bad-bound-from-prefix-drift-and-budgets}$
 $[\text{OF false-statement wf controlled prefix-bound drift-bound}])$
have *merkle-bound*:
 $\text{wp-event } (\text{checked-staged-security-experiment-with-data-state } A)$
 $(\text{staged-security-with-data-state-verifier-event}$
 $\text{partial-merkle-inconsistency-bad})$
 $\text{adversary-initial-state} \leq$
 $\text{hash-collision-budget-value } 0$
 $(\text{staged-attacker-query-budget budgets} +$
 $\text{staged-challenge-query-budget} + \text{verifier-hash-query-budget})$
by (rule
 $\text{checked-staged-security-with-data-state-partial-merkle-inconsistency-bad-bound-controlled}$
 $[\text{OF wf controlled}])$
have *empty-header-bound*:
 $\text{wp-event } (\text{checked-staged-security-experiment-with-data-state } A)$
 $(\text{staged-security-with-data-state-verifier-event}$
 $\text{soundness-bad-event-partial-candidate-empty-header})$
 $\text{adversary-initial-state} \leq$
 $P + D +$
 $\text{staged-phase-relation-error size}$
 $(\text{staged-alpha-search-queries budgets } 0) +$
 $\text{trace-fri-error} + \text{nnreal rounds} * \text{query-error-bound}$
by (rule
 $\text{checked-staged-security-with-data-state-empty-header-bad-event-bound-from-prefix-drift-trace-fri-and-query}$
 $[\text{OF false-statement wf controlled prefix-bound drift-bound}$
 $\text{empty-trace-fri-bound empty-query-bound}])$
have *bound*:
 $\text{checked-staged-adversary-acceptance-probability } A \leq$
 $\text{hash-collision-budget-value } 0$
 $(\text{staged-attacker-query-budget budgets} +$
 $\text{staged-challenge-query-budget} + \text{verifier-hash-query-budget}) +$
 $(P + D +$
 $\text{staged-phase-relation-error size}$
 $(\text{staged-alpha-search-queries budgets } 0)) +$
 $\text{trace-fri-error}' + \text{composition-fri-error}' +$

```

    query-error +
    (P + D +
      staged-phase-relation-error size
      (staged-alpha-search-queries budgets 0) +
      trace-fri-error + nnreal rounds * query-error-bound)
  by (rule
    checked-staged-soundness-from-aligned-partial-candidate-components-and-empty-header-bound
    [OF false-statement merkle-bound comp-bound trace-fri-bound
      comp-fri-bound query-bound empty-header-bound])
  then show ?thesis .
qed

lemma checked-staged-soundness-from-aligned-partial-candidate-components-and-current-empty-query:
  fixes trace-fri-error' composition-fri-error' query-error P D
  empty-query-error :: prob
  assumes false-statement:  $\neg$  exists-valid-trace
  and wf: staged-budget-wellformed budgets
  and controlled: staged-adversary-controlled budgets A
  and trace-fri-bound:
    wp-event (checked-staged-security-experiment-with-data-state A)
    (staged-security-with-data-state-verifier-event
      trace-fri-bad-with-partial-candidates)
    adversary-initial-state  $\leq$  trace-fri-error'
  and comp-fri-bound:
    wp-event (checked-staged-security-experiment-with-data-state A)
    (staged-security-with-data-state-verifier-event
      composition-fri-bad-with-partial-candidates)
    adversary-initial-state  $\leq$  composition-fri-error'
  and query-bound:
    wp-event (checked-staged-security-experiment-with-data-state A)
    (staged-security-with-data-state-verifier-event
      query-bad-with-aligned-partial-candidates)
    adversary-initial-state  $\leq$  query-error
  and prefix-bound:
    wp-event
    (checked-staged-security-experiment-with-actual-alpha-prefix-data-state
      A)
    (checked-staged-security-with-actual-alpha-prefix-bad-set-hit
      composition-trace-bad-alpha-space)
    adversary-initial-state  $\leq$  P
  and drift-bound:
    wp-event
    (checked-staged-security-experiment-with-actual-alpha-prefix-data-state
      A)
    (checked-staged-security-with-actual-alpha-prefix-partial-header-candidate-drift
      composition-trace-bad-alpha-space)
    adversary-initial-state  $\leq$  D
  and empty-trace-fri-bound:
    wp-event (checked-staged-security-experiment-with-data-state A)

```

(staged-security-with-data-state-verifier-event
trace-fri-bad-with-empty-composition-header-candidates)
adversary-initial-state $\leq$ *trace-fri-error*
and *current-empty-bound:*
wp-event (checked-staged-security-experiment-with-data-state A)
(staged-security-with-data-state-current-empty-query-opening-hit-at 0)
adversary-initial-state $\leq$ *empty-query-error*
shows *checked-staged-adversary-acceptance-probability A* $\leq$
hash-collision-budget-value 0
(staged-attacker-query-budget budgets +
staged-challenge-query-budget + verifier-hash-query-budget) +
(P + D +
staged-phase-relation-error size
(staged-alpha-search-queries budgets 0)) +
trace-fri-error' + composition-fri-error' +
query-error +
(P + D +
staged-phase-relation-error size
(staged-alpha-search-queries budgets 0) +
trace-fri-error + empty-query-error)
proof –
have *comp-bound:*
wp-event (checked-staged-security-experiment-with-data-state A)
(staged-security-with-data-state-verifier-event
composition-bad-with-partial-candidates)
adversary-initial-state $\leq$
P + D +
staged-phase-relation-error size
(staged-alpha-search-queries budgets 0)
by (rule
checked-staged-security-with-data-state-partial-candidate-composition-bad-bound-from-prefix-drift-and-budgets
[OF false-statement wf controlled prefix-bound drift-bound])
have *merkle-bound:*
wp-event (checked-staged-security-experiment-with-data-state A)
(staged-security-with-data-state-verifier-event
partial-merkle-inconsistency-bad)
adversary-initial-state $\leq$
hash-collision-budget-value 0
(staged-attacker-query-budget budgets +
staged-challenge-query-budget + verifier-hash-query-budget)
by (rule
checked-staged-security-with-data-state-partial-merkle-inconsistency-bad-bound-controlled
[OF wf controlled])
have *empty-header-bound:*
wp-event (checked-staged-security-experiment-with-data-state A)
(staged-security-with-data-state-verifier-event
soundness-bad-event-partial-candidate-empty-header)
adversary-initial-state $\leq$
P + D +

staged-phase-relation-error size
(staged-alpha-search-queries budgets 0) +
trace-fri-error + empty-query-error
by (rule
checked-staged-security-with-data-state-empty-header-bad-event-bound-from-prefix-drift-trace-fri-and-current-empty-header-bound
[OF false-statement wf controlled prefix-bound drift-bound
empty-trace-fri-bound current-empty-bound])
have bound:
checked-staged-adversary-acceptance-probability A ≤
hash-collision-budget-value 0
(staged-attacker-query-budget budgets +
staged-challenge-query-budget + verifier-hash-query-budget) +
(P + D +
staged-phase-relation-error size
(staged-alpha-search-queries budgets 0)) +
trace-fri-error' + composition-fri-error' +
query-error +
(P + D +
staged-phase-relation-error size
(staged-alpha-search-queries budgets 0) +
trace-fri-error + empty-query-error)
by (rule
checked-staged-soundness-from-aligned-partial-candidate-components-and-empty-header-bound
[OF false-statement merkle-bound comp-bound trace-fri-bound
comp-fri-bound query-bound empty-header-bound])
then show ?thesis .
qed

lemma *checked-staged-soundness-from-aligned-transcript-partial-candidate-components-and-empty-header-components*
fixes *trace-fri-error' composition-fri-error' query-error P D :: prob*
assumes *false-statement: ¬ exists-valid-trace*
and *wf: staged-budget-wellformed budgets*
and *controlled: staged-adversary-controlled budgets A*
and *trace-fri-bound:*
wp-event (checked-staged-security-experiment-with-data-state A)
(staged-security-with-data-state-verifier-event
trace-fri-bad-with-partial-candidates)
adversary-initial-state ≤ trace-fri-error'
and *comp-fri-bound:*
wp-event (checked-staged-security-experiment-with-data-state A)
(staged-security-with-data-state-verifier-event
composition-fri-bad-with-partial-candidates)
adversary-initial-state ≤ composition-fri-error'
and *query-bound:*
wp-event (checked-staged-security-experiment-with-data-state A)
(staged-security-with-data-state-verifier-event
query-bad-with-aligned-transcript-partial-candidates)
adversary-initial-state ≤ query-error
and *prefix-bound:*

wp-event
 (checked-staged-security-experiment-with-actual-alpha-prefix-data-state
 A)
 (checked-staged-security-with-actual-alpha-prefix-bad-set-hit
 composition-trace-bad-alpha-space)
 adversary-initial-state $\leq P$
and drift-bound:
wp-event
 (checked-staged-security-experiment-with-actual-alpha-prefix-data-state
 A)
 (checked-staged-security-with-actual-alpha-prefix-partial-header-candidate-drift
 composition-trace-bad-alpha-space)
 adversary-initial-state $\leq D$
and empty-trace-fri-bound:
wp-event (checked-staged-security-experiment-with-data-state *A*)
 (staged-security-with-data-state-verifier-event
 trace-fri-bad-with-empty-composition-header-candidates)
 adversary-initial-state $\leq$ trace-fri-error
and empty-query-bound:
wp-event (checked-staged-security-experiment-with-data-state *A*)
 (staged-security-with-data-state-verifier-event
 query-bad-with-empty-composition-header-candidates)
 adversary-initial-state $\leq$ nnreal rounds * query-error-bound
shows checked-staged-adversary-acceptance-probability *A* $\leq$
 hash-collision-budget-value 0
 (staged-attacker-query-budget budgets +
 staged-challenge-query-budget + verifier-hash-query-budget) +
 (*P* + *D* +
 staged-phase-relation-error size
 (staged-alpha-search-queries budgets 0)) +
 trace-fri-error' + composition-fri-error' +
 query-error +
 (*P* + *D* +
 staged-phase-relation-error size
 (staged-alpha-search-queries budgets 0) +
 trace-fri-error + nnreal rounds * query-error-bound)
proof –
have comp-bound:
wp-event (checked-staged-security-experiment-with-data-state *A*)
 (staged-security-with-data-state-verifier-event
 composition-bad-with-partial-candidates)
 adversary-initial-state $\leq$
P + *D* +
 staged-phase-relation-error size
 (staged-alpha-search-queries budgets 0)
by (rule
 checked-staged-security-with-data-state-partial-candidate-composition-bad-bound-from-prefix-drift-and-budgets
 [OF false-statement wf controlled prefix-bound drift-bound])
have merkle-bound:

$wp\text{-}event$ ($checked\text{-}staged\text{-}security\text{-}experiment\text{-}with\text{-}data\text{-}state$ A)
 ($staged\text{-}security\text{-}with\text{-}data\text{-}state\text{-}verifier\text{-}event$
 $partial\text{-}merkle\text{-}inconsistency\text{-}bad$)
 $adversary\text{-}initial\text{-}state \leq$
 $hash\text{-}collision\text{-}budget\text{-}value$ 0
 ($staged\text{-}attacker\text{-}query\text{-}budget$ $budgets$ +
 $staged\text{-}challenge\text{-}query\text{-}budget$ + $verifier\text{-}hash\text{-}query\text{-}budget$)
by ($rule$
 $checked\text{-}staged\text{-}security\text{-}with\text{-}data\text{-}state\text{-}partial\text{-}merkle\text{-}inconsistency\text{-}bad\text{-}bound\text{-}controlled$
 $[OF$ wf $controlled]$)
have $empty\text{-}header\text{-}bound$:
 $wp\text{-}event$ ($checked\text{-}staged\text{-}security\text{-}experiment\text{-}with\text{-}data\text{-}state$ A)
 ($staged\text{-}security\text{-}with\text{-}data\text{-}state\text{-}verifier\text{-}event$
 $soundness\text{-}bad\text{-}event\text{-}partial\text{-}candidate\text{-}empty\text{-}header$)
 $adversary\text{-}initial\text{-}state \leq$
 $P + D +$
 $staged\text{-}phase\text{-}relation\text{-}error$ $size$
 ($staged\text{-}alpha\text{-}search\text{-}queries$ $budgets$ 0) +
 $trace\text{-}fri\text{-}error$ + $nnreal$ $rounds * query\text{-}error\text{-}bound$
by ($rule$
 $checked\text{-}staged\text{-}security\text{-}with\text{-}data\text{-}state\text{-}empty\text{-}header\text{-}bad\text{-}event\text{-}bound\text{-}from\text{-}prefix\text{-}drift\text{-}trace\text{-}fri\text{-}and\text{-}query$
 $[OF$ $false\text{-}statement$ wf $controlled$ $prefix\text{-}bound$ $drift\text{-}bound$
 $empty\text{-}trace\text{-}fri\text{-}bound$ $empty\text{-}query\text{-}bound]$)
have $bound$:
 $checked\text{-}staged\text{-}adversary\text{-}acceptance\text{-}probability$ $A \leq$
 $hash\text{-}collision\text{-}budget\text{-}value$ 0
 ($staged\text{-}attacker\text{-}query\text{-}budget$ $budgets$ +
 $staged\text{-}challenge\text{-}query\text{-}budget$ + $verifier\text{-}hash\text{-}query\text{-}budget$) +
 ($P + D +$
 $staged\text{-}phase\text{-}relation\text{-}error$ $size$
 ($staged\text{-}alpha\text{-}search\text{-}queries$ $budgets$ 0)) +
 $trace\text{-}fri\text{-}error'$ + $composition\text{-}fri\text{-}error'$ +
 $query\text{-}error$ +
 ($P + D +$
 $staged\text{-}phase\text{-}relation\text{-}error$ $size$
 ($staged\text{-}alpha\text{-}search\text{-}queries$ $budgets$ 0) +
 $trace\text{-}fri\text{-}error$ + $nnreal$ $rounds * query\text{-}error\text{-}bound$)
by ($rule$
 $checked\text{-}staged\text{-}soundness\text{-}from\text{-}aligned\text{-}transcript\text{-}partial\text{-}candidate\text{-}components\text{-}and\text{-}empty\text{-}header\text{-}bound$
 $[OF$ $false\text{-}statement$ $merkle\text{-}bound$ $comp\text{-}bound$ $trace\text{-}fri\text{-}bound$
 $comp\text{-}fri\text{-}bound$ $query\text{-}bound$ $empty\text{-}header\text{-}bound]$)
then show $?thesis$.
qed

lemma $checked\text{-}staged\text{-}soundness\text{-}from\text{-}aligned\text{-}transcript\text{-}components\text{-}and\text{-}partial\text{-}opening\text{-}hit$:
fixes $trace\text{-}fri\text{-}error'$ $composition\text{-}fri\text{-}error'$ $query\text{-}error$ P $D :: prob$
assumes $false\text{-}statement$: $\neg$ $exists\text{-}valid\text{-}trace$
and wf : $staged\text{-}budget\text{-}wellformed$ $budgets$
and $controlled$: $staged\text{-}adversary\text{-}controlled$ $budgets$ A

and *trace-fri-bound*:
 $\text{wp-event } (\text{checked-staged-security-experiment-with-data-state } A)$
 $(\text{staged-security-with-data-state-verifier-event}$
 $\text{trace-fri-bad-with-partial-candidates})$
 $\text{adversary-initial-state} \leq \text{trace-fri-error}'$

and *comp-fri-bound*:
 $\text{wp-event } (\text{checked-staged-security-experiment-with-data-state } A)$
 $(\text{staged-security-with-data-state-verifier-event}$
 $\text{composition-fri-bad-with-partial-candidates})$
 $\text{adversary-initial-state} \leq \text{composition-fri-error}'$

and *partial-opening-bound*:
 $\text{wp-event } (\text{checked-staged-security-experiment-with-data-state } A)$
 $\text{staged-security-with-data-state-query-partial-opening-hit}$
 $\text{adversary-initial-state} \leq \text{query-error}$

and *prefix-bound*:
 wp-event
 $(\text{checked-staged-security-experiment-with-actual-alpha-prefix-data-state}$
 $A)$
 $(\text{checked-staged-security-with-actual-alpha-prefix-bad-set-hit}$
 $\text{composition-trace-bad-alpha-space})$
 $\text{adversary-initial-state} \leq P$

and *drift-bound*:
 wp-event
 $(\text{checked-staged-security-experiment-with-actual-alpha-prefix-data-state}$
 $A)$
 $(\text{checked-staged-security-with-actual-alpha-prefix-partial-header-candidate-drift}$
 $\text{composition-trace-bad-alpha-space})$
 $\text{adversary-initial-state} \leq D$

and *empty-trace-fri-bound*:
 $\text{wp-event } (\text{checked-staged-security-experiment-with-data-state } A)$
 $(\text{staged-security-with-data-state-verifier-event}$
 $\text{trace-fri-bad-with-empty-composition-header-candidates})$
 $\text{adversary-initial-state} \leq \text{trace-fri-error}$

and *empty-query-bound*:
 $\text{wp-event } (\text{checked-staged-security-experiment-with-data-state } A)$
 $(\text{staged-security-with-data-state-verifier-event}$
 $\text{query-bad-with-empty-composition-header-candidates})$
 $\text{adversary-initial-state} \leq \text{nnreal rounds} * \text{query-error-bound}$

shows $\text{checked-staged-adversary-acceptance-probability } A \leq$
 $\text{hash-collision-budget-value } 0$
 $(\text{staged-attacker-query-budget budgets} +$
 $\text{staged-challenge-query-budget} + \text{verifier-hash-query-budget}) +$
 $(P + D +$
 $\text{staged-phase-relation-error size}$
 $(\text{staged-alpha-search-queries budgets } 0)) +$
 $\text{trace-fri-error}' + \text{composition-fri-error}' +$
 $\text{query-error} +$
 $(P + D +$
 $\text{staged-phase-relation-error size}$

$(\text{staged-alpha-search-queries budgets } 0) +$
 $\text{trace-fri-error} + \text{nnreal rounds} * \text{query-error-bound})$
proof –
have *query-bound*:
 $\text{wp-event } (\text{checked-staged-security-experiment-with-data-state } A)$
 $(\text{staged-security-with-data-state-verifier-event}$
 $\text{query-bad-with-aligned-transcript-partial-candidates})$
 $\text{adversary-initial-state} \leq \text{query-error}$
by (rule *order-trans*[
 $OF \text{checked-staged-security-with-data-state-aligned-transcript-query-bad-bound-from-partial-opening-hit}$
 $\text{partial-opening-bound}]$)
 $(\text{use wf controlled in simp-all})$
show *?thesis*
by (rule
 $\text{checked-staged-soundness-from-aligned-transcript-partial-candidate-components-and-empty-header-components}$
 $[OF \text{false-statement wf controlled trace-fri-bound comp-fri-bound}$
 $\text{query-bound prefix-bound drift-bound empty-trace-fri-bound}$
 $\text{empty-query-bound}]$)
qed

lemma *checked-staged-soundness-from-aligned-transcript-components-and-current-partial-opening-hit*:

fixes *trace-fri-error'* *composition-fri-error'* *query-error* *P* *D* :: *prob*

assumes *false-statement*: $\neg \text{exists-valid-trace}$

and *wf*: *staged-budget-wellformed* *budgets*

and *controlled*: *staged-adversary-controlled* *budgets* *A*

and *trace-fri-bound*:

$\text{wp-event } (\text{checked-staged-security-experiment-with-data-state } A)$
 $(\text{staged-security-with-data-state-verifier-event}$
 $\text{trace-fri-bad-with-partial-candidates})$
 $\text{adversary-initial-state} \leq \text{trace-fri-error}'$

and *comp-fri-bound*:

$\text{wp-event } (\text{checked-staged-security-experiment-with-data-state } A)$
 $(\text{staged-security-with-data-state-verifier-event}$
 $\text{composition-fri-bad-with-partial-candidates})$
 $\text{adversary-initial-state} \leq \text{composition-fri-error}'$

and *current-query-bound*:

$\text{wp-event } (\text{checked-staged-security-experiment-with-data-state } A)$
 $\text{staged-security-with-data-state-current-query-partial-opening-hit}$
 $\text{adversary-initial-state} \leq \text{query-error}$

and *prefix-bound*:

wp-event
 $(\text{checked-staged-security-experiment-with-actual-alpha-prefix-data-state}$
 $A)$
 $(\text{checked-staged-security-with-actual-alpha-prefix-bad-set-hit}$
 $\text{composition-trace-bad-alpha-space})$
 $\text{adversary-initial-state} \leq P$

and *drift-bound*:

wp-event
 $(\text{checked-staged-security-experiment-with-actual-alpha-prefix-data-state}$

A)
 (checked-staged-security-with-actual-alpha-prefix-partial-header-candidate-drift
 composition-trace-bad-alpha-space)
 adversary-initial-state $\leq D$
and empty-trace-fri-bound:
 wp-event (checked-staged-security-experiment-with-data-state A)
 (staged-security-with-data-state-verifier-event
 trace-fri-bad-with-empty-composition-header-candidates)
 adversary-initial-state $\leq$ trace-fri-error
and empty-query-bound:
 wp-event (checked-staged-security-experiment-with-data-state A)
 (staged-security-with-data-state-verifier-event
 query-bad-with-empty-composition-header-candidates)
 adversary-initial-state $\leq$ nnreal rounds * query-error-bound
shows checked-staged-adversary-acceptance-probability A $\leq$
 hash-collision-budget-value 0
 (staged-attacker-query-budget budgets +
 staged-challenge-query-budget + verifier-hash-query-budget) +
 (P + D +
 staged-phase-relation-error size
 (staged-alpha-search-queries budgets 0)) +
 trace-fri-error' + composition-fri-error' +
 query-error +
 (P + D +
 staged-phase-relation-error size
 (staged-alpha-search-queries budgets 0) +
 trace-fri-error + nnreal rounds * query-error-bound)
proof –
have query-bound:
 wp-event (checked-staged-security-experiment-with-data-state A)
 (staged-security-with-data-state-verifier-event
 query-bad-with-aligned-transcript-partial-candidates)
 adversary-initial-state $\leq$ query-error
by (rule order-trans[
 OF checked-staged-security-with-data-state-aligned-transcript-query-bad-bound-from-current-partial-opening-hit
 current-query-bound])
 (use wf controlled **in** simp-all)
show ?thesis
by (rule
 checked-staged-soundness-from-aligned-transcript-partial-candidate-components-and-empty-header-components
 [OF false-statement wf controlled trace-fri-bound comp-fri-bound
 query-bound prefix-bound drift-bound empty-trace-fri-bound
 empty-query-bound])
qed

lemma checked-staged-soundness-from-aligned-transcript-components-and-current-query-prefix-rounds:
fixes trace-fri-error' composition-fri-error' P D :: prob
and C :: nat $\Rightarrow$ prob
assumes false-statement: $\neg$ exists-valid-trace

and *wf*: staged-budget-wellformed budgets
and *controlled*: staged-adversary-controlled budgets *A*
and *trace-fri-bound*:
 wp-event (*checked-staged-security-experiment-with-data-state A*)
 (*staged-security-with-data-state-verifier-event*
 trace-fri-bad-with-partial-candidates)
 adversary-initial-state $\leq$ *trace-fri-error'*
and *comp-fri-bound*:
 wp-event (*checked-staged-security-experiment-with-data-state A*)
 (*staged-security-with-data-state-verifier-event*
 composition-fri-bad-with-partial-candidates)
 adversary-initial-state $\leq$ *composition-fri-error'*
and *round-bound*:
 $\bigwedge i. i < \text{rounds} \implies$
 wp-event
 (*checked-staged-security-experiment-with-query-prefix-data-state A i*)
 (*checked-staged-security-with-query-prefix-current-query-partial-opening-hit-at i*)
 adversary-initial-state $\leq C i$
and *prefix-bound*:
 wp-event
 (*checked-staged-security-experiment-with-actual-alpha-prefix-data-state*
 A)
 (*checked-staged-security-with-actual-alpha-prefix-bad-set-hit*
 composition-trace-bad-alpha-space)
 adversary-initial-state $\leq P$
and *drift-bound*:
 wp-event
 (*checked-staged-security-experiment-with-actual-alpha-prefix-data-state*
 A)
 (*checked-staged-security-with-actual-alpha-prefix-partial-header-candidate-drift*
 composition-trace-bad-alpha-space)
 adversary-initial-state $\leq D$
and *empty-trace-fri-bound*:
 wp-event (*checked-staged-security-experiment-with-data-state A*)
 (*staged-security-with-data-state-verifier-event*
 trace-fri-bad-with-empty-composition-header-candidates)
 adversary-initial-state $\leq$ *trace-fri-error*
and *empty-query-bound*:
 wp-event (*checked-staged-security-experiment-with-data-state A*)
 (*staged-security-with-data-state-verifier-event*
 query-bad-with-empty-composition-header-candidates)
 adversary-initial-state $\leq$ *nnreal rounds * query-error-bound*
shows *checked-staged-adversary-acceptance-probability A* $\leq$
 hash-collision-budget-value 0
 (*staged-attacker-query-budget budgets* +
 staged-challenge-query-budget + *verifier-hash-query-budget*) +
 (*P* + *D* +
 staged-phase-relation-error size
 (*staged-alpha-search-queries budgets 0*)) +

$\text{trace-fri-error}' + \text{composition-fri-error}' +$
 $(\sum i < \text{rounds}. C\ i) +$
 $(P + D +$
 $\text{staged-phase-relation-error size}$
 $(\text{staged-alpha-search-queries budgets } 0) +$
 $\text{trace-fri-error} + \text{nnreal rounds} * \text{query-error-bound})$

proof –

have *current-query-bound*:

wp-event (*checked-staged-security-experiment-with-data-state A*)
staged-security-with-data-state-current-query-partial-opening-hit
adversary-initial-state $\leq (\sum i < \text{rounds}. C\ i)$

by (*rule*
checked-staged-security-with-data-state-current-query-partial-opening-hit-bound-from-query-prefix-rounds
 $[OF\ \text{round-bound}\ wf\ \text{controlled}]$)

show *?thesis*

by (*rule*
checked-staged-soundness-from-aligned-transcript-components-and-current-partial-opening-hit
 $[OF\ \text{false-statement}\ wf\ \text{controlled}\ \text{trace-fri-bound}\ \text{comp-fri-bound}$
 $\text{current-query-bound}\ \text{prefix-bound}\ \text{drift-bound}\ \text{empty-trace-fri-bound}$
 $\text{empty-query-bound}]$)

qed

lemma *checked-staged-soundness-from-aligned-transcript-components-and-partial-opening-hit-from-drift*:

fixes *trace-fri-error'* *composition-fri-error'* *query-error D* :: *prob*

assumes *false-statement*: $\neg \text{exists-valid-trace}$

and *wf*: *staged-budget-wellformed budgets*

and *controlled*: *staged-adversary-controlled budgets A*

and *trace-fri-bound*:

wp-event (*checked-staged-security-experiment-with-data-state A*)
(staged-security-with-data-state-verifier-event
trace-fri-bad-with-partial-candidates)
adversary-initial-state $\leq \text{trace-fri-error}'$

and *comp-fri-bound*:

wp-event (*checked-staged-security-experiment-with-data-state A*)
(staged-security-with-data-state-verifier-event
composition-fri-bad-with-partial-candidates)
adversary-initial-state $\leq \text{composition-fri-error}'$

and *partial-opening-bound*:

wp-event (*checked-staged-security-experiment-with-data-state A*)
staged-security-with-data-state-query-partial-opening-hit
adversary-initial-state $\leq \text{query-error}$

and *drift-bound*:

wp-event
(checked-staged-security-experiment-with-actual-alpha-prefix-data-state
A)
(checked-staged-security-with-actual-alpha-prefix-partial-header-candidate-drift
composition-trace-bad-alpha-space)
adversary-initial-state $\leq D$

and *empty-trace-fri-bound*:

wp_event (*checked-staged-security-experiment-with-data-state* A)
 (*staged-security-with-data-state-verifier-event*
trace-fri-bad-with-empty-composition-header-candidates)
 $adversary_initial_state \leq trace_fri_error$
and *empty-query-bound*:
 wp_event (*checked-staged-security-experiment-with-data-state* A)
 (*staged-security-with-data-state-verifier-event*
query-bad-with-empty-composition-header-candidates)
 $adversary_initial_state \leq nnreal\ rounds * query_error_bound$
shows *checked-staged-adversary-acceptance-probability* $A \leq$
hash-collision-budget-value 0
 (*staged-attacker-query-budget* *budgets* +
staged-challenge-query-budget + *verifier-hash-query-budget*) +
 (*composition-error-bound* +
staged-phase-relation-error *size*
 (*staged-alpha-search-queries* *budgets* 0)) +
hash-collision-budget-value 0
 (*staged-alpha-search-queries* *budgets* 0) +
 D +
staged-phase-relation-error *size*
 (*staged-alpha-search-queries* *budgets* 0)) +
trace-fri-error' + *composition-fri-error'* +
query-error +
 (*composition-error-bound* +
staged-phase-relation-error *size*
 (*staged-alpha-search-queries* *budgets* 0)) +
hash-collision-budget-value 0
 (*staged-alpha-search-queries* *budgets* 0) +
 D +
staged-phase-relation-error *size*
 (*staged-alpha-search-queries* *budgets* 0)) +
trace-fri-error + $nnreal\ rounds * query_error_bound$)
proof –
have *comp-bound*:
 wp_event (*checked-staged-security-experiment-with-data-state* A)
 (*staged-security-with-data-state-verifier-event*
composition-bad-with-partial-candidates)
 $adversary_initial_state \leq$
composition-error-bound +
staged-phase-relation-error *size*
 (*staged-alpha-search-queries* *budgets* 0)) +
hash-collision-budget-value 0
 (*staged-alpha-search-queries* *budgets* 0) +
 D +
staged-phase-relation-error *size*
 (*staged-alpha-search-queries* *budgets* 0))
by (*rule*
checked-staged-security-with-data-state-partial-candidate-composition-bad-bound-from-drift-and-budgets
 [*OF false-statement wf controlled drift-bound*])

have *query-bound*:
wp-event (*checked-staged-security-experiment-with-data-state A*)
(*staged-security-with-data-state-verifier-event*
query-bad-with-aligned-transcript-partial-candidates)
adversary-initial-state $\leq$ *query-error*
by (rule *order-trans*[
OF checked-staged-security-with-data-state-aligned-transcript-query-bad-bound-from-partial-opening-hit
partial-opening-bound])
(use wf controlled in simp-all)
have *merkle-bound*:
wp-event (*checked-staged-security-experiment-with-data-state A*)
(*staged-security-with-data-state-verifier-event*
partial-merkle-inconsistency-bad)
adversary-initial-state $\leq$
hash-collision-budget-value 0
(*staged-attacker-query-budget budgets* +
staged-challenge-query-budget + *verifier-hash-query-budget*)
by (rule
checked-staged-security-with-data-state-partial-merkle-inconsistency-bad-bound-controlled
[*OF wf controlled*])
have *empty-header-bound*:
wp-event (*checked-staged-security-experiment-with-data-state A*)
(*staged-security-with-data-state-verifier-event*
soundness-bad-event-partial-candidate-empty-header)
adversary-initial-state $\leq$
composition-error-bound +
staged-phase-relation-error size
(*staged-alpha-search-queries budgets 0*) +
hash-collision-budget-value 0
(*staged-alpha-search-queries budgets 0*) +
D +
staged-phase-relation-error size
(*staged-alpha-search-queries budgets 0*) +
trace-fri-error + *nnreal rounds* * *query-error-bound*
by (rule
checked-staged-security-with-data-state-empty-header-bad-event-bound-from-drift-trace-fri-and-query
[*OF false-statement wf controlled drift-bound empty-trace-fri-bound*
empty-query-bound])
show *?thesis*
by (rule
checked-staged-soundness-from-aligned-transcript-partial-candidate-components-and-empty-header-bound
[*OF false-statement merkle-bound comp-bound trace-fri-bound*
comp-fri-bound query-bound empty-header-bound])
qed

lemma *checked-staged-soundness-from-aligned-transcript-components-and-current-partial-opening-hit-from-drift*:
fixes *trace-fri-error'* *composition-fri-error'* *query-error D* :: *prob*
assumes *false-statement*: $\neg$ *exists-valid-trace*
and *wf*: *staged-budget-wellformed budgets*

and *controlled: staged-adversary-controlled budgets* A
and *trace-fri-bound:*
wp-event (checked-staged-security-experiment-with-data-state A
(staged-security-with-data-state-verifier-event
trace-fri-bad-with-partial-candidates)
adversary-initial-state $\leq$ *trace-fri-error'*
and *comp-fri-bound:*
wp-event (checked-staged-security-experiment-with-data-state A
(staged-security-with-data-state-verifier-event
composition-fri-bad-with-partial-candidates)
adversary-initial-state $\leq$ *composition-fri-error'*
and *current-query-bound:*
wp-event (checked-staged-security-experiment-with-data-state A
staged-security-with-data-state-current-query-partial-opening-hit
adversary-initial-state $\leq$ *query-error*
and *drift-bound:*
wp-event
(checked-staged-security-experiment-with-actual-alpha-prefix-data-state
A)
(checked-staged-security-with-actual-alpha-prefix-partial-header-candidate-drift
composition-trace-bad-alpha-space)
adversary-initial-state $\leq$ D
and *empty-trace-fri-bound:*
wp-event (checked-staged-security-experiment-with-data-state A
(staged-security-with-data-state-verifier-event
trace-fri-bad-with-empty-composition-header-candidates)
adversary-initial-state $\leq$ *trace-fri-error*
and *empty-query-bound:*
wp-event (checked-staged-security-experiment-with-data-state A
(staged-security-with-data-state-verifier-event
query-bad-with-empty-composition-header-candidates)
adversary-initial-state $\leq$ *nnreal rounds * query-error-bound*
shows *checked-staged-adversary-acceptance-probability* $A \leq$
hash-collision-budget-value 0
(staged-attacker-query-budget budgets +
staged-challenge-query-budget + verifier-hash-query-budget) +
(composition-error-bound +
staged-phase-relation-error size
(staged-alpha-search-queries budgets 0) +
hash-collision-budget-value 0
(staged-alpha-search-queries budgets 0) +
 $D +$
staged-phase-relation-error size
(staged-alpha-search-queries budgets 0)) +
trace-fri-error' + composition-fri-error' +
query-error +
(composition-error-bound +
staged-phase-relation-error size
(staged-alpha-search-queries budgets 0) +

hash-collision-budget-value 0
(staged-alpha-search-queries budgets 0) +
D +
staged-phase-relation-error size
(staged-alpha-search-queries budgets 0) +
*trace-fri-error + nreal rounds * query-error-bound)*

proof –

have *comp-bound*:

wp-event (checked-staged-security-experiment-with-data-state A)
(staged-security-with-data-state-verifier-event
composition-bad-with-partial-candidates)
adversary-initial-state ≤
composition-error-bound +
staged-phase-relation-error size
(staged-alpha-search-queries budgets 0) +
hash-collision-budget-value 0
(staged-alpha-search-queries budgets 0) +
D +
staged-phase-relation-error size
(staged-alpha-search-queries budgets 0)

by (rule
checked-staged-security-with-data-state-partial-candidate-composition-bad-bound-from-drift-and-budgets
[OF false-statement wf controlled drift-bound])

have *query-bound*:

wp-event (checked-staged-security-experiment-with-data-state A)
(staged-security-with-data-state-verifier-event
query-bad-with-aligned-transcript-partial-candidates)
adversary-initial-state ≤ query-error

by (rule *order-trans*[
OF checked-staged-security-with-data-state-aligned-transcript-query-bad-bound-from-current-partial-opening-hit
current-query-bound])
(use wf controlled in simp-all)

have *merkle-bound*:

wp-event (checked-staged-security-experiment-with-data-state A)
(staged-security-with-data-state-verifier-event
partial-merkle-inconsistency-bad)
adversary-initial-state ≤
hash-collision-budget-value 0
(staged-attacker-query-budget budgets +
staged-challenge-query-budget + verifier-hash-query-budget)

by (rule
checked-staged-security-with-data-state-partial-merkle-inconsistency-bad-bound-controlled
[OF wf controlled])

have *empty-header-bound*:

wp-event (checked-staged-security-experiment-with-data-state A)
(staged-security-with-data-state-verifier-event
soundness-bad-event-partial-candidate-empty-header)
adversary-initial-state ≤
composition-error-bound +

```

    staged-phase-relation-error size
      (staged-alpha-search-queries budgets 0) +
    hash-collision-budget-value 0
      (staged-alpha-search-queries budgets 0) +
    D +
    staged-phase-relation-error size
      (staged-alpha-search-queries budgets 0) +
    trace-fri-error + nnreal rounds * query-error-bound
  by (rule
    checked-staged-security-with-data-state-empty-header-bad-event-bound-from-drift-trace-fri-and-query
    [OF false-statement wf controlled drift-bound empty-trace-fri-bound
      empty-query-bound])
  show ?thesis
  by (rule
    checked-staged-soundness-from-aligned-transcript-partial-candidate-components-and-empty-header-bound
    [OF false-statement merkle-bound comp-bound trace-fri-bound
      comp-fri-bound query-bound empty-header-bound])
qed

lemma checked-staged-soundness-from-aligned-transcript-components-and-partial-opening-hit-from-not-prefix:
fixes trace-fri-error' composition-fri-error' query-error N :: prob
assumes false-statement:  $\neg$  exists-valid-trace
and wf: staged-budget-wellformed budgets
and controlled: staged-adversary-controlled budgets A
and trace-fri-bound:
  wp-event (checked-staged-security-experiment-with-data-state A)
    (staged-security-with-data-state-verifier-event
      trace-fri-bad-with-partial-candidates)
    adversary-initial-state  $\leq$  trace-fri-error'
and comp-fri-bound:
  wp-event (checked-staged-security-experiment-with-data-state A)
    (staged-security-with-data-state-verifier-event
      composition-fri-bad-with-partial-candidates)
    adversary-initial-state  $\leq$  composition-fri-error'
and partial-opening-bound:
  wp-event (checked-staged-security-experiment-with-data-state A)
    staged-security-with-data-state-query-partial-opening-hit
    adversary-initial-state  $\leq$  query-error
and not-prefix-bound:
  wp-event
    (checked-staged-security-experiment-with-actual-alpha-prefix-data-state
      A)
    (checked-staged-security-with-actual-alpha-prefix-partial-header-candidate-not-prefix-bound
      composition-trace-bad-alpha-space)
    adversary-initial-state  $\leq$  N
and empty-trace-fri-bound:
  wp-event (checked-staged-security-experiment-with-data-state A)
    (staged-security-with-data-state-verifier-event
      trace-fri-bad-with-empty-composition-header-candidates)

```

$\text{adversary-initial-state} \leq \text{trace-fri-error}$
and *empty-query-bound*:
 $\text{wp-event } (\text{checked-staged-security-experiment-with-data-state } A)$
 $(\text{staged-security-with-data-state-verifier-event}$
 $\text{query-bad-with-empty-composition-header-candidates})$
 $\text{adversary-initial-state} \leq \text{nnreal rounds} * \text{query-error-bound}$
shows *checked-staged-adversary-acceptance-probability* $A \leq$
 $\text{hash-collision-budget-value } 0$
 $(\text{staged-attacker-query-budget budgets} +$
 $\text{staged-challenge-query-budget} + \text{verifier-hash-query-budget}) +$
 $(\text{composition-error-bound} +$
 $\text{staged-phase-relation-error size}$
 $(\text{staged-alpha-search-queries budgets } 0) +$
 $\text{hash-collision-budget-value } 0$
 $(\text{staged-alpha-search-queries budgets } 0) +$
 $N +$
 $\text{staged-phase-relation-error size}$
 $(\text{staged-alpha-search-queries budgets } 0)) +$
 $\text{trace-fri-error}' + \text{composition-fri-error}' +$
 $\text{query-error} +$
 $(\text{composition-error-bound} +$
 $\text{staged-phase-relation-error size}$
 $(\text{staged-alpha-search-queries budgets } 0) +$
 $\text{hash-collision-budget-value } 0$
 $(\text{staged-alpha-search-queries budgets } 0) +$
 $N +$
 $\text{staged-phase-relation-error size}$
 $(\text{staged-alpha-search-queries budgets } 0) +$
 $\text{trace-fri-error} + \text{nnreal rounds} * \text{query-error-bound})$
proof –
have *drift-bound*:
 wp-event
 $(\text{checked-staged-security-experiment-with-actual-alpha-prefix-data-state}$
 $A)$
 $(\text{checked-staged-security-with-actual-alpha-prefix-partial-header-candidate-drift}$
 $\text{composition-trace-bad-alpha-space})$
 $\text{adversary-initial-state} \leq N$
proof –
have *wp-event*
 $(\text{checked-staged-security-experiment-with-actual-alpha-prefix-data-state}$
 $A)$
 $(\text{checked-staged-security-with-actual-alpha-prefix-partial-header-candidate-drift}$
 $\text{composition-trace-bad-alpha-space})$
 $\text{adversary-initial-state} \leq$
 wp-event
 $(\text{checked-staged-security-experiment-with-actual-alpha-prefix-data-state}$
 $A)$
 $(\text{checked-staged-security-with-actual-alpha-prefix-partial-header-candidate-not-prefix-bound}$
 $\text{composition-trace-bad-alpha-space})$

```

    adversary-initial-state
  by (rule
    checked-staged-security-with-actual-alpha-prefix-partial-header-candidate-drift-le-not-prefix-bound
    [OF wf controlled])
  also have ...  $\leq N$ 
  by (rule not-prefix-bound)
  finally show ?thesis .
qed
show ?thesis
  by (rule
    checked-staged-soundness-from-aligned-transcript-components-and-partial-opening-hit-from-drift
    [OF false-statement wf controlled trace-fri-bound comp-fri-bound
      partial-opening-bound drift-bound empty-trace-fri-bound
      empty-query-bound])
qed

lemma checked-staged-soundness-from-aligned-transcript-components-and-current-partial-opening-hit-from-not-prefix-bound
  fixes trace-fri-error' composition-fri-error' query-error N :: prob
  assumes false-statement:  $\neg$  exists-valid-trace
  and wf: staged-budget-wellformed budgets
  and controlled: staged-adversary-controlled budgets A
  and trace-fri-bound:
    wp-event (checked-staged-security-experiment-with-data-state A)
    (staged-security-with-data-state-verifier-event
      trace-fri-bad-with-partial-candidates)
    adversary-initial-state  $\leq$  trace-fri-error'
  and comp-fri-bound:
    wp-event (checked-staged-security-experiment-with-data-state A)
    (staged-security-with-data-state-verifier-event
      composition-fri-bad-with-partial-candidates)
    adversary-initial-state  $\leq$  composition-fri-error'
  and current-query-bound:
    wp-event (checked-staged-security-experiment-with-data-state A)
    staged-security-with-data-state-current-query-partial-opening-hit
    adversary-initial-state  $\leq$  query-error
  and not-prefix-bound:
    wp-event
    (checked-staged-security-experiment-with-actual-alpha-prefix-data-state
      A)
    (checked-staged-security-with-actual-alpha-prefix-partial-header-candidate-not-prefix-bound
      composition-trace-bad-alpha-space)
    adversary-initial-state  $\leq N$ 
  and empty-trace-fri-bound:
    wp-event (checked-staged-security-experiment-with-data-state A)
    (staged-security-with-data-state-verifier-event
      trace-fri-bad-with-empty-composition-header-candidates)
    adversary-initial-state  $\leq$  trace-fri-error
  and empty-query-bound:
    wp-event (checked-staged-security-experiment-with-data-state A)

```

*(staged-security-with-data-state-verifier-event
 query-bad-with-empty-composition-header-candidates)
 adversary-initial-state* $\leq$ *nnreal rounds * query-error-bound*
shows *checked-staged-adversary-acceptance-probability A* $\leq$
hash-collision-budget-value 0
(staged-attacker-query-budget budgets +
staged-challenge-query-budget + verifier-hash-query-budget) $+$
(composition-error-bound +
staged-phase-relation-error size
(staged-alpha-search-queries budgets 0) +
hash-collision-budget-value 0
(staged-alpha-search-queries budgets 0) +
N +
staged-phase-relation-error size
(staged-alpha-search-queries budgets 0)) $+$
trace-fri-error' + composition-fri-error' +
query-error +
(composition-error-bound +
staged-phase-relation-error size
(staged-alpha-search-queries budgets 0) +
hash-collision-budget-value 0
(staged-alpha-search-queries budgets 0) +
N +
staged-phase-relation-error size
(staged-alpha-search-queries budgets 0) +
*trace-fri-error + nnreal rounds * query-error-bound)*

proof $-$

have *drift-bound:*

wp-event
(checked-staged-security-experiment-with-actual-alpha-prefix-data-state
A)
(checked-staged-security-with-actual-alpha-prefix-partial-header-candidate-drift
composition-trace-bad-alpha-space)
adversary-initial-state $\leq$ *N*

proof $-$

have *wp-event*
(checked-staged-security-experiment-with-actual-alpha-prefix-data-state
A)
(checked-staged-security-with-actual-alpha-prefix-partial-header-candidate-drift
composition-trace-bad-alpha-space)
adversary-initial-state $\leq$

wp-event
(checked-staged-security-experiment-with-actual-alpha-prefix-data-state
A)
(checked-staged-security-with-actual-alpha-prefix-partial-header-candidate-not-prefix-bound
composition-trace-bad-alpha-space)
adversary-initial-state

by (rule
checked-staged-security-with-actual-alpha-prefix-partial-header-candidate-drift-le-not-prefix-bound

```

    [OF wf controlled])
  also have ... ≤ N
    by (rule not-prefix-bound)
  finally show ?thesis .
qed
show ?thesis
  by (rule
    checked-staged-soundness-from-aligned-transcript-components-and-current-partial-opening-hit-from-drift
    [OF false-statement wf controlled trace-fri-bound comp-fri-bound
      current-query-bound drift-bound empty-trace-fri-bound
      empty-query-bound])
qed

lemma checked-staged-soundness-from-aligned-transcript-components-and-current-query-prefix-rounds-from-not-prefix-bound
  fixes trace-fri-error' composition-fri-error' N :: prob
  and C :: nat ⇒ prob
  assumes false-statement: ¬ exists-valid-trace
  and wf: staged-budget-wellformed budgets
  and controlled: staged-adversary-controlled budgets A
  and trace-fri-bound:
    wp-event (checked-staged-security-experiment-with-data-state A)
      (staged-security-with-data-state-verifier-event
        trace-fri-bad-with-partial-candidates)
      adversary-initial-state ≤ trace-fri-error'
  and comp-fri-bound:
    wp-event (checked-staged-security-experiment-with-data-state A)
      (staged-security-with-data-state-verifier-event
        composition-fri-bad-with-partial-candidates)
      adversary-initial-state ≤ composition-fri-error'
  and round-bound:
    ∧ i. i < rounds ⇒
      wp-event
        (checked-staged-security-experiment-with-query-prefix-data-state A i)
        (checked-staged-security-with-query-prefix-current-query-partial-opening-hit-at i)
        adversary-initial-state ≤ C i
  and not-prefix-bound:
    wp-event
      (checked-staged-security-experiment-with-actual-alpha-prefix-data-state
        A)
      (checked-staged-security-with-actual-alpha-prefix-partial-header-candidate-not-prefix-bound
        composition-trace-bad-alpha-space)
      adversary-initial-state ≤ N
  and empty-trace-fri-bound:
    wp-event (checked-staged-security-experiment-with-data-state A)
      (staged-security-with-data-state-verifier-event
        trace-fri-bad-with-empty-composition-header-candidates)
      adversary-initial-state ≤ trace-fri-error
  and empty-query-bound:
    wp-event (checked-staged-security-experiment-with-data-state A)

```

*(staged-security-with-data-state-verifier-event
 query-bad-with-empty-composition-header-candidates)
 adversary-initial-state ≤ nnreal rounds * query-error-bound*
shows *checked-staged-adversary-acceptance-probability A ≤
 hash-collision-budget-value 0
 (staged-attacker-query-budget budgets +
 staged-challenge-query-budget + verifier-hash-query-budget) +
 (composition-error-bound +
 staged-phase-relation-error size
 (staged-alpha-search-queries budgets 0) +
 hash-collision-budget-value 0
 (staged-alpha-search-queries budgets 0) +
 N +
 staged-phase-relation-error size
 (staged-alpha-search-queries budgets 0)) +
 trace-fri-error' + composition-fri-error' +
 (∑ i<rounds. C i) +
 (composition-error-bound +
 staged-phase-relation-error size
 (staged-alpha-search-queries budgets 0) +
 hash-collision-budget-value 0
 (staged-alpha-search-queries budgets 0) +
 N +
 staged-phase-relation-error size
 (staged-alpha-search-queries budgets 0) +
 trace-fri-error + nnreal rounds * query-error-bound)*

proof –

have *current-query-bound:*
*wp-event (checked-staged-security-experiment-with-data-state A)
 staged-security-with-data-state-current-query-partial-opening-hit
 adversary-initial-state ≤ (∑ i<rounds. C i)*

by (rule
*checked-staged-security-with-data-state-current-query-partial-opening-hit-bound-from-query-prefix-rounds
 [OF round-bound wf controlled])*

show *?thesis*

by (rule
*checked-staged-soundness-from-aligned-transcript-components-and-current-partial-opening-hit-from-not-prefix
 [OF false-statement wf controlled trace-fri-bound comp-fri-bound
 current-query-bound not-prefix-bound empty-trace-fri-bound
 empty-query-bound])*

qed

lemma *checked-staged-soundness-from-aligned-transcript-components-and-prefix-target-path-bounds-from-not-prefix:*
fixes *trace-fri-error' composition-fri-error' N :: prob*
and *P Q :: nat ⇒ prob*
assumes *false-statement: ¬ exists-valid-trace*
and *wf: staged-budget-wellformed budgets*
and *controlled: staged-adversary-controlled budgets A*
and *trace-fri-bound:*

wp_event (*checked-staged-security-experiment-with-data-state* A)
 (*staged-security-with-data-state-verifier-event*
trace-fri-bad-with-partial-candidates)
adversary-initial-state $\leq$ *trace-fri-error'*
and *comp-fri-bound*:
 wp_event (*checked-staged-security-experiment-with-data-state* A)
 (*staged-security-with-data-state-verifier-event*
composition-fri-bad-with-partial-candidates)
adversary-initial-state $\leq$ *composition-fri-error'*
and *prefix-bound*:
 $\bigwedge i. i < rounds \implies$
 wp_event (*checked-staged-query-prefix-receive-with-state* A i)
 (*checked-staged-query-prefix-dynamic-index-hit*
staged-query-prefix-prefix-authenticated-opening-target)
adversary-initial-state $\leq P$ i
and *path-bound*:
 $\bigwedge i. i < rounds \implies$
 wp_event
 (*checked-staged-security-experiment-with-query-prefix-data-state* A i)
 (*checked-staged-security-with-query-prefix-current-path-output-hit-at* i)
adversary-initial-state $\leq Q$ i
and *not-prefix-bound*:
 wp_event
 (*checked-staged-security-experiment-with-actual-alpha-prefix-data-state*
 A)
 (*checked-staged-security-with-actual-alpha-prefix-partial-header-candidate-not-prefix-bound*
composition-trace-bad-alpha-space)
adversary-initial-state $\leq N$
and *empty-trace-fri-bound*:
 wp_event (*checked-staged-security-experiment-with-data-state* A)
 (*staged-security-with-data-state-verifier-event*
trace-fri-bad-with-empty-composition-header-candidates)
adversary-initial-state $\leq$ *trace-fri-error*
and *empty-query-bound*:
 wp_event (*checked-staged-security-experiment-with-data-state* A)
 (*staged-security-with-data-state-verifier-event*
query-bad-with-empty-composition-header-candidates)
adversary-initial-state $\leq nnreal$ *rounds* * *query-error-bound*
shows *checked-staged-adversary-acceptance-probability* $A \leq$
hash-collision-budget-value 0
 (*staged-attacker-query-budget* *budgets* +
staged-challenge-query-budget + *verifier-hash-query-budget*) +
 (*composition-error-bound* +
staged-phase-relation-error *size*
 (*staged-alpha-search-queries* *budgets* 0) +
hash-collision-budget-value 0
 (*staged-alpha-search-queries* *budgets* 0) +
 N +
staged-phase-relation-error *size*

```

    (staged-alpha-search-queries budgets 0)) +
    trace-fri-error' + composition-fri-error' +
    ( $\sum i < \text{rounds}. P\ i + Q\ i$ ) +
    (composition-error-bound +
     staged-phase-relation-error size
     (staged-alpha-search-queries budgets 0) +
     hash-collision-budget-value 0
     (staged-alpha-search-queries budgets 0) +
     N +
     staged-phase-relation-error size
     (staged-alpha-search-queries budgets 0) +
     trace-fri-error + nnreal rounds * query-error-bound)
  proof -
  have round-bound:
     $\bigwedge i. i < \text{rounds} \implies$ 
    wp-event
      (checked-staged-security-experiment-with-query-prefix-data-state A i)
      (checked-staged-security-with-query-prefix-current-query-partial-opening-hit-at i)
      adversary-initial-state  $\leq P\ i + Q\ i$ 
  proof -
  fix i
  assume i-bound:  $i < \text{rounds}$ 
  have prefix-hit-bound:
    wp-event
      (checked-staged-security-experiment-with-query-prefix-data-state A i)
      (checked-staged-security-with-query-prefix-current-prefix-authenticated-hit-at i)
      adversary-initial-state  $\leq P\ i$ 
  by (rule
      checked-staged-security-with-query-prefix-current-prefix-authenticated-hit-at-bound-from-dynamic-target
      [OF wf controlled i-bound prefix-bound[OF i-bound]])
  show
    wp-event
      (checked-staged-security-experiment-with-query-prefix-data-state A i)
      (checked-staged-security-with-query-prefix-current-query-partial-opening-hit-at i)
      adversary-initial-state  $\leq P\ i + Q\ i$ 
  by (rule
      checked-staged-security-with-query-prefix-current-query-partial-opening-hit-at-bound-from-prefix-and-path
      [OF wf controlled i-bound prefix-hit-bound path-bound[OF i-bound]])
  qed
  show ?thesis
  by (rule
      checked-staged-soundness-from-aligned-transcript-components-and-current-query-prefix-rounds-from-not-prefix
      [OF false-statement wf controlled trace-fri-bound comp-fri-bound
       round-bound not-prefix-bound empty-trace-fri-bound
       empty-query-bound])
  qed

lemma checked-staged-soundness-from-aligned-transcript-components-and-trace-index-path-bounds-from-not-prefix:
  fixes trace-fri-error' composition-fri-error' N :: prob

```

and $Q :: \text{nat} \Rightarrow \text{prob}$
assumes *false-statement*: $\neg \text{exists-valid-trace}$
and *wf*: *staged-budget-wellformed budgets*
and *controlled*: *staged-adversary-controlled budgets A*
and *trace-fri-bound*:
 wp-event (*checked-staged-security-experiment-with-data-state A*)
 (*staged-security-with-data-state-verifier-event*
 trace-fri-bad-with-partial-candidates)
 adversary-initial-state $\leq$ *trace-fri-error'*
and *comp-fri-bound*:
 wp-event (*checked-staged-security-experiment-with-data-state A*)
 (*staged-security-with-data-state-verifier-event*
 composition-fri-bad-with-partial-candidates)
 adversary-initial-state $\leq$ *composition-fri-error'*
and *trace-frac*:
 $\bigwedge i$ *prefix prefix-state.*
 $i < \text{rounds} \implies$
 Some ((*prefix, prefix-state*), *prefix-state*) $\in$
 set-dist
 (*execute* (*checked-staged-query-prefix-with-state A i*)
 adversary-initial-state) $\implies$
 nnreal
 (*card*
 (*staged-query-prefix-prefix-authenticated-trace-indices prefix*
 prefix-state)) /
 nnreal (*card query-sample-space*) $\leq$ *query-error-bound*
and *path-bound*:
 $\bigwedge i. i < \text{rounds} \implies$
 wp-event
 (*checked-staged-security-experiment-with-query-prefix-data-state A i*)
 (*checked-staged-security-with-query-prefix-current-path-output-hit-at i*)
 adversary-initial-state $\leq Q i$
and *not-prefix-bound*:
 wp-event
 (*checked-staged-security-experiment-with-actual-alpha-prefix-data-state*
 A)
 (*checked-staged-security-with-actual-alpha-prefix-partial-header-candidate-not-prefix-bound*
 composition-trace-bad-alpha-space)
 adversary-initial-state $\leq N$
and *empty-trace-fri-bound*:
 wp-event (*checked-staged-security-experiment-with-data-state A*)
 (*staged-security-with-data-state-verifier-event*
 trace-fri-bad-with-empty-composition-header-candidates)
 adversary-initial-state $\leq$ *trace-fri-error*
and *empty-query-bound*:
 wp-event (*checked-staged-security-experiment-with-data-state A*)
 (*staged-security-with-data-state-verifier-event*
 query-bad-with-empty-composition-header-candidates)
 adversary-initial-state $\leq$ *nnreal rounds * query-error-bound*

shows *checked-staged-adversary-acceptance-probability* $A \leq$
hash-collision-budget-value 0
 (*staged-attacker-query-budget* budgets +
staged-challenge-query-budget + *verifier-hash-query-budget*) +
 (*composition-error-bound* +
staged-phase-relation-error size
 (*staged-alpha-search-queries* budgets 0) +
hash-collision-budget-value 0
 (*staged-alpha-search-queries* budgets 0) +
N +
staged-phase-relation-error size
 (*staged-alpha-search-queries* budgets 0)) +
trace-fri-error' + *composition-fri-error'* +
 ($\sum i < \text{rounds}$.
staged-phase-relation-error size
 (*staged-query-search-queries* budgets $i + 1$) +
query-error-bound + $Q \ i$) +
 (*composition-error-bound* +
staged-phase-relation-error size
 (*staged-alpha-search-queries* budgets 0) +
hash-collision-budget-value 0
 (*staged-alpha-search-queries* budgets 0) +
N +
staged-phase-relation-error size
 (*staged-alpha-search-queries* budgets 0) +
trace-fri-error + $\text{nnreal rounds} * \text{query-error-bound}$)

proof –

have *round-bound*:

$\bigwedge i. i < \text{rounds} \implies$

wp-event

(*checked-staged-security-experiment-with-query-prefix-data-state* $A \ i$)
 (*checked-staged-security-with-query-prefix-current-query-partial-opening-hit-at* i)
adversary-initial-state $\leq$
staged-phase-relation-error size
 (*staged-query-search-queries* budgets $i + 1$) +
query-error-bound + $Q \ i$

proof –

fix i

assume *i-bound*: $i < \text{rounds}$

have *prefix-hit-bound*:

wp-event

(*checked-staged-security-experiment-with-query-prefix-data-state* $A \ i$)
 (*checked-staged-security-with-query-prefix-current-prefix-authenticated-hit-at* i)
adversary-initial-state $\leq$
staged-phase-relation-error size
 (*staged-query-search-queries* budgets $i + 1$) + *query-error-bound*

by (rule

checked-staged-security-with-query-prefix-current-prefix-authenticated-hit-at-bound-from-trace-indices
 [OF wf controlled *i-bound*])

```

    (use trace-frac[OF i-bound] in blast)
  show
    wp-event
      (checked-staged-security-experiment-with-query-prefix-data-state A i)
      (checked-staged-security-with-query-prefix-current-query-partial-opening-hit-at i)
      adversary-initial-state ≤
      staged-phase-relation-error size
      (staged-query-search-queries budgets i + 1) +
      query-error-bound + Q i
    by (rule
      checked-staged-security-with-query-prefix-current-query-partial-opening-hit-at-bound-from-prefix-and-path
      [OF wf controlled i-bound prefix-hit-bound path-bound[OF i-bound]])
  qed
  show ?thesis
    by (rule
      checked-staged-soundness-from-aligned-transcript-components-and-current-query-prefix-rounds-from-not-prefix-co
      [OF false-statement wf controlled trace-fri-bound comp-fri-bound
      round-bound not-prefix-bound empty-trace-fri-bound
      empty-query-bound])
  qed

lemma checked-staged-soundness-from-aligned-transcript-components-and-trace-index-path-bounds-from-not-prefix-co
fixes trace-fri-error' composition-fri-error' C U M R :: prob
and Q :: nat ⇒ prob
assumes false-statement: ¬ exists-valid-trace
and wf: staged-budget-wellformed budgets
and controlled: staged-adversary-controlled budgets A
and trace-fri-bound:
  wp-event (checked-staged-security-experiment-with-data-state A)
    (staged-security-with-data-state-verifier-event
      trace-fri-bad-with-partial-candidates)
  adversary-initial-state ≤ trace-fri-error'
and comp-fri-bound:
  wp-event (checked-staged-security-experiment-with-data-state A)
    (staged-security-with-data-state-verifier-event
      composition-fri-bad-with-partial-candidates)
  adversary-initial-state ≤ composition-fri-error'
and trace-frac:
  ∧ i prefix prefix-state.
  i < rounds ⇒
  Some ((prefix, prefix-state), prefix-state) ∈
  set-dist
    (execute (checked-staged-query-prefix-with-state A i)
      adversary-initial-state) ⇒
  nnreal
    (card
      (staged-query-prefix-prefix-authenticated-trace-indices prefix
        prefix-state)) /
  nnreal (card query-sample-space) ≤ query-error-bound

```

and *path-bound*:
 $\bigwedge i. i < \text{rounds} \implies$
 wp-event
 $(\text{checked-staged-security-experiment-with-query-prefix-data-state } A \ i)$
 $(\text{checked-staged-security-with-query-prefix-current-path-output-hit-at } i)$
 $\text{adversary-initial-state} \leq Q \ i$
and *collision-bound*:
 wp-event
 $(\text{checked-staged-security-experiment-with-actual-alpha-prefix-data-state } A)$
 $\text{checked-staged-security-with-actual-alpha-prefix-not-prefix-collision}$
 $\text{adversary-initial-state} \leq C$
and *uncovered-bound*:
 wp-event
 $(\text{checked-staged-security-experiment-with-actual-alpha-prefix-data-state } A)$
 $\text{checked-staged-security-with-actual-alpha-prefix-not-prefix-uncovered-index}$
 $\text{adversary-initial-state} \leq U$
and *no-prefix-bound*:
 wp-event
 $(\text{checked-staged-security-experiment-with-actual-alpha-prefix-data-state } A)$
 $\text{checked-staged-security-with-actual-alpha-prefix-not-prefix-no-prefix-candidate}$
 $\text{adversary-initial-state} \leq M$
and *not-prefix-path-bound*:
 wp-event
 $(\text{checked-staged-security-experiment-with-actual-alpha-prefix-data-state } A)$
 $\text{checked-staged-security-with-actual-alpha-prefix-not-prefix-path-output}$
 $\text{adversary-initial-state} \leq R$
and *empty-trace-fri-bound*:
 $\text{wp-event } (\text{checked-staged-security-experiment-with-data-state } A)$
 $(\text{staged-security-with-data-state-verifier-event}$
 $\text{trace-fri-bad-with-empty-composition-header-candidates})$
 $\text{adversary-initial-state} \leq \text{trace-fri-error}$
and *empty-query-bound*:
 $\text{wp-event } (\text{checked-staged-security-experiment-with-data-state } A)$
 $(\text{staged-security-with-data-state-verifier-event}$
 $\text{query-bad-with-empty-composition-header-candidates})$
 $\text{adversary-initial-state} \leq \text{nnreal rounds} * \text{query-error-bound}$
shows $\text{checked-staged-adversary-acceptance-probability } A \leq$
 $\text{hash-collision-budget-value } 0$
 $(\text{staged-attacker-query-budget budgets} +$
 $\text{staged-challenge-query-budget} + \text{verifier-hash-query-budget}) +$
 $(\text{composition-error-bound} +$
 $\text{staged-phase-relation-error size}$
 $(\text{staged-alpha-search-queries budgets } 0) +$
 $\text{hash-collision-budget-value } 0$
 $(\text{staged-alpha-search-queries budgets } 0) +$

$(C + U + M + R) +$
staged-phase-relation-error size
 $(\text{staged-alpha-search-queries budgets } 0)) +$
 $\text{trace-fri-error}' + \text{composition-fri-error}' +$
 $(\sum i < \text{rounds}.$
 $\text{staged-phase-relation-error size}$
 $(\text{staged-query-search-queries budgets } i + 1) +$
 $\text{query-error-bound} + Q i) +$
 $(\text{composition-error-bound} +$
 $\text{staged-phase-relation-error size}$
 $(\text{staged-alpha-search-queries budgets } 0) +$
 $\text{hash-collision-budget-value } 0$
 $(\text{staged-alpha-search-queries budgets } 0) +$
 $(C + U + M + R) +$
 $\text{staged-phase-relation-error size}$
 $(\text{staged-alpha-search-queries budgets } 0) +$
 $\text{trace-fri-error} + \text{nnreal rounds} * \text{query-error-bound})$

proof –

have *not-prefix-bound*:

wp-event
 $(\text{checked-staged-security-experiment-with-actual-alpha-prefix-data-state}$
 $A)$
 $(\text{checked-staged-security-with-actual-alpha-prefix-partial-header-candidate-not-prefix-bound}$
 $\text{composition-trace-bad-alpha-space})$
 $\text{adversary-initial-state} \leq C + U + M + R$

by (rule
 $\text{checked-staged-security-with-actual-alpha-prefix-not-prefix-bound-union-bound}$
 $[OF \text{ wf controlled collision-bound uncovered-bound no-prefix-bound}$
 $\text{not-prefix-path-bound}]$)

show *?thesis*

by (rule
 $\text{checked-staged-soundness-from-aligned-transcript-components-and-trace-index-path-bounds-from-not-prefix}$
 $[OF \text{ false-statement wf controlled trace-fri-bound comp-fri-bound}$
 $\text{trace-frac path-bound not-prefix-bound empty-trace-fri-bound}$
 $\text{empty-query-bound}]$)

qed

lemma *checked-staged-soundness-from-aligned-transcript-components-and-trace-index-path-bounds-from-not-prefix-no*

fixes $\text{trace-fri-error}' \text{ composition-fri-error}' U M R :: \text{prob}$

and $Q :: \text{nat} \Rightarrow \text{prob}$

assumes *false-statement*: $\neg \text{exists-valid-trace}$

and *wf*: *staged-budget-wellformed* *budgets*

and *controlled*: *staged-adversary-controlled* *budgets* *A*

and *trace-fri-bound*:

wp-event $(\text{checked-staged-security-experiment-with-data-state } A)$
 $(\text{staged-security-with-data-state-verifier-event}$
 $\text{trace-fri-bad-with-partial-candidates})$
 $\text{adversary-initial-state} \leq \text{trace-fri-error}'$

and *comp-fri-bound*:

$wp\text{-}event \ (checked\text{-}staged\text{-}security\text{-}experiment\text{-}with\text{-}data\text{-}state \ A)$
 $(staged\text{-}security\text{-}with\text{-}data\text{-}state\text{-}verifier\text{-}event$
 $\quad composition\text{-}fri\text{-}bad\text{-}with\text{-}partial\text{-}candidates)$
 $\quad adversary\text{-}initial\text{-}state \leq composition\text{-}fri\text{-}error'$
and $trace\text{-}frac$:
 $\bigwedge i \ prefix \ prefix\text{-}state.$
 $i < rounds \implies$
 $Some \ ((prefix, \ prefix\text{-}state), \ prefix\text{-}state) \in$
 $\quad set\text{-}dist$
 $\quad (execute \ (checked\text{-}staged\text{-}query\text{-}prefix\text{-}with\text{-}state \ A \ i)$
 $\quad \quad adversary\text{-}initial\text{-}state) \implies$
 $nnreal$
 $\quad (card$
 $\quad \quad (staged\text{-}query\text{-}prefix\text{-}prefix\text{-}authenticated\text{-}trace\text{-}indices \ prefix$
 $\quad \quad \quad prefix\text{-}state)) \ /$
 $\quad nnreal \ (card \ query\text{-}sample\text{-}space) \leq query\text{-}error\text{-}bound$
and $path\text{-}bound$:
 $\bigwedge i. \ i < rounds \implies$
 $wp\text{-}event$
 $\quad (checked\text{-}staged\text{-}security\text{-}experiment\text{-}with\text{-}query\text{-}prefix\text{-}data\text{-}state \ A \ i)$
 $\quad (checked\text{-}staged\text{-}security\text{-}with\text{-}query\text{-}prefix\text{-}current\text{-}path\text{-}output\text{-}hit\text{-}at \ i)$
 $\quad \quad adversary\text{-}initial\text{-}state \leq Q \ i$
and $uncovered\text{-}bound$:
 $wp\text{-}event$
 $\quad (checked\text{-}staged\text{-}security\text{-}experiment\text{-}with\text{-}actual\text{-}alpha\text{-}prefix\text{-}data\text{-}state$
 $\quad \quad A)$
 $\quad checked\text{-}staged\text{-}security\text{-}with\text{-}actual\text{-}alpha\text{-}prefix\text{-}not\text{-}prefix\text{-}uncovered\text{-}index$
 $\quad \quad adversary\text{-}initial\text{-}state \leq U$
and $no\text{-}prefix\text{-}bound$:
 $wp\text{-}event$
 $\quad (checked\text{-}staged\text{-}security\text{-}experiment\text{-}with\text{-}actual\text{-}alpha\text{-}prefix\text{-}data\text{-}state$
 $\quad \quad A)$
 $\quad checked\text{-}staged\text{-}security\text{-}with\text{-}actual\text{-}alpha\text{-}prefix\text{-}not\text{-}prefix\text{-}no\text{-}prefix\text{-}candidate$
 $\quad \quad adversary\text{-}initial\text{-}state \leq M$
and $not\text{-}prefix\text{-}path\text{-}bound$:
 $wp\text{-}event$
 $\quad (checked\text{-}staged\text{-}security\text{-}experiment\text{-}with\text{-}actual\text{-}alpha\text{-}prefix\text{-}data\text{-}state$
 $\quad \quad A)$
 $\quad checked\text{-}staged\text{-}security\text{-}with\text{-}actual\text{-}alpha\text{-}prefix\text{-}not\text{-}prefix\text{-}path\text{-}output$
 $\quad \quad adversary\text{-}initial\text{-}state \leq R$
and $empty\text{-}trace\text{-}fri\text{-}bound$:
 $wp\text{-}event \ (checked\text{-}staged\text{-}security\text{-}experiment\text{-}with\text{-}data\text{-}state \ A)$
 $\quad (staged\text{-}security\text{-}with\text{-}data\text{-}state\text{-}verifier\text{-}event$
 $\quad \quad trace\text{-}fri\text{-}bad\text{-}with\text{-}empty\text{-}composition\text{-}header\text{-}candidates)$
 $\quad \quad adversary\text{-}initial\text{-}state \leq trace\text{-}fri\text{-}error$
and $empty\text{-}query\text{-}bound$:
 $wp\text{-}event \ (checked\text{-}staged\text{-}security\text{-}experiment\text{-}with\text{-}data\text{-}state \ A)$
 $\quad (staged\text{-}security\text{-}with\text{-}data\text{-}state\text{-}verifier\text{-}event$
 $\quad \quad query\text{-}bad\text{-}with\text{-}empty\text{-}composition\text{-}header\text{-}candidates)$

$adversary-initial-state \leq nnreal\ rounds * query-error-bound$
shows *checked-staged-adversary-acceptance-probability* $A \leq$
hash-collision-budget-value 0
(staged-attacker-query-budget budgets +
staged-challenge-query-budget + verifier-hash-query-budget) +
(composition-error-bound +
staged-phase-relation-error size
(staged-alpha-search-queries budgets 0) +
hash-collision-budget-value 0
(staged-alpha-search-queries budgets 0) +
(hash-collision-budget-value 0
(staged-alpha-search-queries budgets 0) + U + M + R) +
staged-phase-relation-error size
(staged-alpha-search-queries budgets 0)) +
trace-fri-error' + composition-fri-error' +
 $(\sum i < rounds.$
staged-phase-relation-error size
(staged-query-search-queries budgets i + 1) +
query-error-bound + Q i) +
(composition-error-bound +
staged-phase-relation-error size
(staged-alpha-search-queries budgets 0) +
hash-collision-budget-value 0
(staged-alpha-search-queries budgets 0) +
(hash-collision-budget-value 0
(staged-alpha-search-queries budgets 0) + U + M + R) +
staged-phase-relation-error size
(staged-alpha-search-queries budgets 0) +
*trace-fri-error + nnreal rounds * query-error-bound)*
proof (rule
checked-staged-soundness-from-aligned-transcript-components-and-trace-index-path-bounds-from-not-prefix-compon
 $[OF\ false-statement\ wf\ controlled\ trace-fri-bound\ comp-fri-bound$
 $trace-frac\ path-bound - uncovered-bound\ no-prefix-bound$
 $not-prefix-path-bound\ empty-trace-fri-bound\ empty-query-bound])$
show
wp-event
(checked-staged-security-experiment-with-actual-alpha-prefix-data-state
 $A)$
checked-staged-security-with-actual-alpha-prefix-not-prefix-collision
adversary-initial-state $\leq$
hash-collision-budget-value 0 *(staged-alpha-search-queries budgets 0)*
by (rule
checked-staged-security-with-actual-alpha-prefix-not-prefix-collision-bound
 $[OF\ wf\ controlled])$
qed
lemma *checked-staged-soundness-from-aligned-transcript-components-and-trace-index-path-bounds-from-cross-no-pre*
fixes *trace-fri-error' composition-fri-error' X M R :: prob*
and $Q :: nat \Rightarrow prob$

assumes *false-statement*: $\neg \text{exists-valid-trace}$
and *wf*: *staged-budget-wellformed budgets*
and *controlled*: *staged-adversary-controlled budgets A*
and *trace-fri-bound*:
 wp-event (*checked-staged-security-experiment-with-data-state A*)
 (*staged-security-with-data-state-verifier-event*
 trace-fri-bad-with-partial-candidates)
 adversary-initial-state $\leq$ *trace-fri-error'*
and *comp-fri-bound*:
 wp-event (*checked-staged-security-experiment-with-data-state A*)
 (*staged-security-with-data-state-verifier-event*
 composition-fri-bad-with-partial-candidates)
 adversary-initial-state $\leq$ *composition-fri-error'*
and *trace-frac*:
 $\bigwedge i$ *prefix prefix-state*.
 i < *rounds* $\implies$
 Some ((*prefix, prefix-state*), *prefix-state*) $\in$
 set-dist
 (*execute* (*checked-staged-query-prefix-with-state A i*)
 adversary-initial-state) $\implies$
 nnreal
 (*card*
 (*staged-query-prefix-prefix-authenticated-trace-indices prefix*
 prefix-state)) /
 nnreal (*card query-sample-space*) $\leq$ *query-error-bound*
and *path-bound*:
 $\bigwedge i$. *i* < *rounds* $\implies$
 wp-event
 (*checked-staged-security-experiment-with-query-prefix-data-state A i*)
 (*checked-staged-security-with-query-prefix-current-path-output-hit-at i*)
 adversary-initial-state $\leq$ *Q i*
and *cross-bound*:
 wp-event
 (*checked-staged-security-experiment-with-actual-alpha-prefix-data-state*
 A)
 checked-staged-security-with-actual-alpha-prefix-witnessed-cross-or-merkle-bad
 adversary-initial-state $\leq$ *X*
and *no-prefix-bound*:
 wp-event
 (*checked-staged-security-experiment-with-actual-alpha-prefix-data-state*
 A)
 checked-staged-security-with-actual-alpha-prefix-not-prefix-no-prefix-candidate
 adversary-initial-state $\leq$ *M*
and *not-prefix-path-bound*:
 wp-event
 (*checked-staged-security-experiment-with-actual-alpha-prefix-data-state*
 A)
 checked-staged-security-with-actual-alpha-prefix-not-prefix-path-output
 adversary-initial-state $\leq$ *R*

and *empty-trace-fri-bound*:
wp-event (*checked-staged-security-experiment-with-data-state A*)
(staged-security-with-data-state-verifier-event
trace-fri-bad-with-empty-composition-header-candidates)
adversary-initial-state $\leq$ *trace-fri-error*
and *empty-query-bound*:
wp-event (*checked-staged-security-experiment-with-data-state A*)
(staged-security-with-data-state-verifier-event
query-bad-with-empty-composition-header-candidates)
adversary-initial-state $\leq$ *nnreal rounds * query-error-bound*
shows *checked-staged-adversary-acceptance-probability A* $\leq$
hash-collision-budget-value 0
(staged-attacker-query-budget budgets +
staged-challenge-query-budget + verifier-hash-query-budget) +
(composition-error-bound +
staged-phase-relation-error size
(staged-alpha-search-queries budgets 0) +
hash-collision-budget-value 0
(staged-alpha-search-queries budgets 0) +
(hash-collision-budget-value 0
(staged-alpha-search-queries budgets 0) + X + M + R) +
staged-phase-relation-error size
(staged-alpha-search-queries budgets 0)) +
trace-fri-error' + composition-fri-error' +
($\sum i < \text{rounds}$.
staged-phase-relation-error size
(staged-query-search-queries budgets i + 1) +
query-error-bound + Q i) +
(composition-error-bound +
staged-phase-relation-error size
(staged-alpha-search-queries budgets 0) +
hash-collision-budget-value 0
(staged-alpha-search-queries budgets 0) +
(hash-collision-budget-value 0
(staged-alpha-search-queries budgets 0) + X + M + R) +
staged-phase-relation-error size
(staged-alpha-search-queries budgets 0) +
*trace-fri-error + nnreal rounds * query-error-bound)*
proof (*rule*
checked-staged-soundness-from-aligned-transcript-components-and-trace-index-path-bounds-from-not-prefix-noncoll
[OF false-statement wf controlled trace-fri-bound comp-fri-bound
trace-frac path-bound - no-prefix-bound not-prefix-path-bound
empty-trace-fri-bound empty-query-bound])
show
wp-event
(checked-staged-security-experiment-with-actual-alpha-prefix-data-state
A)
checked-staged-security-with-actual-alpha-prefix-not-prefix-uncovered-index
adversary-initial-state $\leq$ *X*

```

by (rule order-trans
  [OF
    checked-staged-security-with-actual-alpha-prefix-not-prefix-uncovered-index-le-witnessed-cross-or-merkle-bad
    cross-bound])
qed

lemma checked-staged-soundness-from-aligned-transcript-components-and-trace-index-path-bounds-from-cross-no-pre
fixes trace-fri-error' composition-fri-error' X M P R :: prob
and Q :: nat  $\Rightarrow$  prob
assumes false-statement:  $\neg$  exists-valid-trace
and wf: staged-budget-wellformed budgets
and controlled: staged-adversary-controlled budgets A
and trace-fri-bound:
  wp-event (checked-staged-security-experiment-with-data-state A)
    (staged-security-with-data-state-verifier-event
      trace-fri-bad-with-partial-candidates)
    adversary-initial-state  $\leq$  trace-fri-error'
and comp-fri-bound:
  wp-event (checked-staged-security-experiment-with-data-state A)
    (staged-security-with-data-state-verifier-event
      composition-fri-bad-with-partial-candidates)
    adversary-initial-state  $\leq$  composition-fri-error'
and trace-frac:
   $\bigwedge i$  prefix prefix-state.
   $i < \text{rounds} \implies$ 
  Some ((prefix, prefix-state), prefix-state)  $\in$ 
    set-dist
    (execute (checked-staged-query-prefix-with-state A i)
      adversary-initial-state)  $\implies$ 
  nnreal
    (card
      (staged-query-prefix-prefix-authenticated-trace-indices prefix
        prefix-state)) /
  nnreal (card query-sample-space)  $\leq$  query-error-bound
and path-bound:
   $\bigwedge i. i < \text{rounds} \implies$ 
  wp-event
    (checked-staged-security-experiment-with-query-prefix-data-state A i)
    (checked-staged-security-with-query-prefix-current-path-output-hit-at i)
    adversary-initial-state  $\leq$  Q i
and cross-bound:
  wp-event
    (checked-staged-security-experiment-with-actual-alpha-prefix-data-state
      A)
    checked-staged-security-with-actual-alpha-prefix-witnessed-cross-or-merkle-bad
    adversary-initial-state  $\leq$  X
and no-prefix-bound:
  wp-event
    (checked-staged-security-experiment-with-actual-alpha-prefix-data-state

```

$A)$
checked-staged-security-with-actual-alpha-prefix-not-prefix-no-prefix-candidate
adversary-initial-state $\leq M$
and *transcript-pre-bound*:
wp-event
(checked-staged-security-experiment-with-actual-alpha-prefix-data-state
 $A)$
checked-staged-security-with-actual-alpha-prefix-witness-transcript-pre-hit
adversary-initial-state $\leq P$
and *transcript-new-bound*:
wp-event
(checked-staged-security-experiment-with-actual-alpha-prefix-data-state
 $A)$
checked-staged-security-with-actual-alpha-prefix-witness-transcript-new-hit
adversary-initial-state $\leq R$
and *empty-trace-fri-bound*:
wp-event (checked-staged-security-experiment-with-data-state A)
(staged-security-with-data-state-verifier-event
trace-fri-bad-with-empty-composition-header-candidates)
adversary-initial-state $\leq$ *trace-fri-error*
and *empty-query-bound*:
wp-event (checked-staged-security-experiment-with-data-state A)
(staged-security-with-data-state-verifier-event
query-bad-with-empty-composition-header-candidates)
adversary-initial-state $\leq$ *nnreal rounds * query-error-bound*
shows *checked-staged-adversary-acceptance-probability A* $\leq$
hash-collision-budget-value 0
(staged-attacker-query-budget budgets +
staged-challenge-query-budget + verifier-hash-query-budget) +
(composition-error-bound +
staged-phase-relation-error size
(staged-alpha-search-queries budgets 0) +
hash-collision-budget-value 0
(staged-alpha-search-queries budgets 0) +
(hash-collision-budget-value 0
(staged-alpha-search-queries budgets 0) + X + M + (P + R)) +
staged-phase-relation-error size
(staged-alpha-search-queries budgets 0)) +
trace-fri-error' + composition-fri-error' +
 $(\sum_{i < \text{rounds.}}$
staged-phase-relation-error size
(staged-query-search-queries budgets i + 1) +
query-error-bound + Q i) +
(composition-error-bound +
staged-phase-relation-error size
(staged-alpha-search-queries budgets 0) +
hash-collision-budget-value 0
(staged-alpha-search-queries budgets 0) +
(hash-collision-budget-value 0

$(\text{staged-alpha-search-queries budgets } 0) + X + M + (P + R)) +$
 $\text{staged-phase-relation-error size}$
 $(\text{staged-alpha-search-queries budgets } 0) +$
 $\text{trace-fri-error} + \text{nnreal rounds} * \text{query-error-bound})$
proof (rule
 $\text{checked-staged-soundness-from-aligned-transcript-components-and-trace-index-path-bounds-from-cross-no-prefix-path}$
 $[\text{OF false-statement wf controlled trace-fri-bound comp-fri-bound}$
 $\text{trace-frac path-bound cross-bound no-prefix-bound - empty-trace-fri-bound}$
 $\text{empty-query-bound}]$)
show
 wp-event
 $(\text{checked-staged-security-experiment-with-actual-alpha-prefix-data-state}$
 $A)$
 $\text{checked-staged-security-with-actual-alpha-prefix-not-prefix-path-output}$
 $\text{adversary-initial-state} \leq P + R$
by (rule order-trans
 $[\text{OF}$
 $\text{checked-staged-security-with-actual-alpha-prefix-not-prefix-path-output-le-witness-transcript-hit}]$)
(rule
 $\text{checked-staged-security-with-actual-alpha-prefix-witness-transcript-hit-bound-from-pre-and-new}$
 $[\text{OF transcript-pre-bound transcript-new-bound}]$)
qed

lemma $\text{checked-staged-soundness-from-aligned-transcript-components-and-trace-index-path-bounds-from-cross-no-prefix-path}$
fixes $\text{trace-fri-error}' \text{ composition-fri-error}' X M P R :: \text{prob}$
and $Q :: \text{nat} \Rightarrow \text{prob}$
assumes $\text{false-statement}: \neg \text{exists-valid-trace}$
and $\text{wf}: \text{staged-budget-wellformed budgets}$
and $\text{controlled}: \text{staged-adversary-controlled budgets } A$
and $\text{trace-fri-bound}:$
 $\text{wp-event } (\text{checked-staged-security-experiment-with-data-state } A)$
 $(\text{staged-security-with-data-state-verifier-event}$
 $\text{trace-fri-bad-with-partial-candidates})$
 $\text{adversary-initial-state} \leq \text{trace-fri-error}'$
and $\text{comp-fri-bound}:$
 $\text{wp-event } (\text{checked-staged-security-experiment-with-data-state } A)$
 $(\text{staged-security-with-data-state-verifier-event}$
 $\text{composition-fri-bad-with-partial-candidates})$
 $\text{adversary-initial-state} \leq \text{composition-fri-error}'$
and $\text{trace-frac}:$
 $\bigwedge i \text{ prefix prefix-state.}$
 $i < \text{rounds} \implies$
 $\text{Some } ((\text{prefix}, \text{prefix-state}), \text{prefix-state}) \in$
 set-dist
 $(\text{execute } (\text{checked-staged-query-prefix-with-state } A \ i)$
 $\text{adversary-initial-state}) \implies$
 nnreal
 $(\text{card}$
 $(\text{staged-query-prefix-prefix-authenticated-trace-indices prefix}$

$\text{prefix-state})) /$
 $\text{nnreal}(\text{card query-sample-space}) \leq \text{query-error-bound}$
and *path-bound*:
 $\bigwedge i. i < \text{rounds} \implies$
 wp-event
 $(\text{checked-staged-security-experiment-with-query-prefix-data-state } A \ i)$
 $(\text{checked-staged-security-with-query-prefix-current-path-output-hit-at } i)$
 $\text{adversary-initial-state} \leq Q \ i$
and *cross-bound*:
 wp-event
 $(\text{checked-staged-security-experiment-with-actual-alpha-prefix-data-state } A)$
 $\text{checked-staged-security-with-actual-alpha-prefix-witnessed-cross-or-merkle-bad}$
 $\text{adversary-initial-state} \leq X$
and *no-prefix-bound*:
 wp-event
 $(\text{checked-staged-security-experiment-with-actual-alpha-prefix-data-state } A)$
 $\text{checked-staged-security-with-actual-alpha-prefix-not-prefix-no-prefix-candidate}$
 $\text{adversary-initial-state} \leq M$
and *data-pre-bound*:
 $\text{wp-event}(\text{checked-staged-security-experiment-with-data-state } A)$
 $\text{staged-security-with-data-state-transcript-pre-hit}$
 $\text{adversary-initial-state} \leq P$
and *transcript-new-bound*:
 wp-event
 $(\text{checked-staged-security-experiment-with-actual-alpha-prefix-data-state } A)$
 $\text{checked-staged-security-with-actual-alpha-prefix-witness-transcript-new-hit}$
 $\text{adversary-initial-state} \leq R$
and *empty-trace-fri-bound*:
 $\text{wp-event}(\text{checked-staged-security-experiment-with-data-state } A)$
 $(\text{staged-security-with-data-state-verifier-event}$
 $\text{trace-fri-bad-with-empty-composition-header-candidates})$
 $\text{adversary-initial-state} \leq \text{trace-fri-error}$
and *empty-query-bound*:
 $\text{wp-event}(\text{checked-staged-security-experiment-with-data-state } A)$
 $(\text{staged-security-with-data-state-verifier-event}$
 $\text{query-bad-with-empty-composition-header-candidates})$
 $\text{adversary-initial-state} \leq \text{nnreal rounds} * \text{query-error-bound}$
shows $\text{checked-staged-adversary-acceptance-probability } A \leq$
 $\text{hash-collision-budget-value } 0$
 $(\text{staged-attacker-query-budget budgets} +$
 $\text{staged-challenge-query-budget} + \text{verifier-hash-query-budget}) +$
 $(\text{composition-error-bound} +$
 $\text{staged-phase-relation-error size}$
 $(\text{staged-alpha-search-queries budgets } 0) +$
 $\text{hash-collision-budget-value } 0$
 $(\text{staged-alpha-search-queries budgets } 0) +$

(hash-collision-budget-value 0
 (staged-alpha-search-queries budgets 0) + X + M + (P + R)) +
 staged-phase-relation-error size
 (staged-alpha-search-queries budgets 0)) +
 trace-fri-error' + composition-fri-error' +
 ($\sum i < \text{rounds}$.
 staged-phase-relation-error size
 (staged-query-search-queries budgets i + 1) +
 query-error-bound + Q i) +
 (composition-error-bound +
 staged-phase-relation-error size
 (staged-alpha-search-queries budgets 0) +
 hash-collision-budget-value 0
 (staged-alpha-search-queries budgets 0) +
 (hash-collision-budget-value 0
 (staged-alpha-search-queries budgets 0) + X + M + (P + R)) +
 staged-phase-relation-error size
 (staged-alpha-search-queries budgets 0) +
 trace-fri-error + nreal rounds * query-error-bound)

proof (rule
 checked-staged-soundness-from-aligned-transcript-components-and-trace-index-path-bounds-from-cross-no-prefix-tra
 [OF false-statement wf controlled trace-fri-bound comp-fri-bound
 trace-frac path-bound cross-bound no-prefix-bound - transcript-new-bound
 empty-trace-fri-bound empty-query-bound])

show
 wp-event
 (checked-staged-security-experiment-with-actual-alpha-prefix-data-state
 A)
 checked-staged-security-with-actual-alpha-prefix-witness-transcript-pre-hit
 adversary-initial-state $\leq$ P

by (rule
 checked-staged-security-with-actual-alpha-prefix-witness-transcript-pre-hit-bound-from-data-state-pre-hit
 [OF data-pre-bound])

qed

lemma checked-staged-soundness-from-aligned-transcript-components-and-trace-index-path-bounds-from-cross-data-p
fixes trace-fri-error' composition-fri-error' X P R :: prob
and Q :: nat $\Rightarrow$ prob
assumes false-statement: $\neg$ exists-valid-trace
and wf: staged-budget-wellformed budgets
and controlled: staged-adversary-controlled budgets A
and trace-fri-bound:
 wp-event (checked-staged-security-experiment-with-data-state A)
 (staged-security-with-data-state-verifier-event
 trace-fri-bad-with-partial-candidates)
 adversary-initial-state $\leq$ trace-fri-error'
and comp-fri-bound:
 wp-event (checked-staged-security-experiment-with-data-state A)
 (staged-security-with-data-state-verifier-event

composition-fri-bad-with-partial-candidates
adversary-initial-state $\leq$ *composition-fri-error'*
and *trace-frac*:
 $\bigwedge i$ *prefix prefix-state*.
 $i < \text{rounds} \implies$
Some ((*prefix, prefix-state*), *prefix-state*) $\in$
set-dist
(*execute* (*checked-staged-query-prefix-with-state A i*)
adversary-initial-state) $\implies$
nnreal
(*card*
(*staged-query-prefix-prefix-authenticated-trace-indices prefix*
prefix-state)) /
nnreal (*card query-sample-space*) $\leq$ *query-error-bound*
and *path-bound*:
 $\bigwedge i. i < \text{rounds} \implies$
wp-event
(*checked-staged-security-experiment-with-query-prefix-data-state A i*)
(*checked-staged-security-with-query-prefix-current-path-output-hit-at i*)
adversary-initial-state $\leq Q i$
and *cross-bound*:
wp-event
(*checked-staged-security-experiment-with-actual-alpha-prefix-data-state*
A)
checked-staged-security-with-actual-alpha-prefix-witnessed-cross-or-merkle-bad
adversary-initial-state $\leq X$
and *data-pre-bound*:
wp-event (*checked-staged-security-experiment-with-data-state A*)
staged-security-with-data-state-transcript-pre-hit
adversary-initial-state $\leq P$
and *transcript-new-bound*:
wp-event
(*checked-staged-security-experiment-with-actual-alpha-prefix-data-state*
A)
checked-staged-security-with-actual-alpha-prefix-witness-transcript-new-hit
adversary-initial-state $\leq R$
and *empty-trace-fri-bound*:
wp-event (*checked-staged-security-experiment-with-data-state A*)
(*staged-security-with-data-state-verifier-event*
trace-fri-bad-with-empty-composition-header-candidates)
adversary-initial-state $\leq$ *trace-fri-error*
and *empty-query-bound*:
wp-event (*checked-staged-security-experiment-with-data-state A*)
(*staged-security-with-data-state-verifier-event*
query-bad-with-empty-composition-header-candidates)
adversary-initial-state $\leq$ *nnreal rounds * query-error-bound*
shows *checked-staged-adversary-acceptance-probability A* $\leq$
hash-collision-budget-value 0
(*staged-attacker-query-budget budgets* +

staged-challenge-query-budget + verifier-hash-query-budget) +
(composition-error-bound +
staged-phase-relation-error size
(staged-alpha-search-queries budgets 0) +
hash-collision-budget-value 0
(staged-alpha-search-queries budgets 0) +
(hash-collision-budget-value 0
(staged-alpha-search-queries budgets 0) + X + (P + R) + (P + R)) +
staged-phase-relation-error size
(staged-alpha-search-queries budgets 0)) +
trace-fri-error' + composition-fri-error' +
($\sum i < \text{rounds}$.
staged-phase-relation-error size
(staged-query-search-queries budgets i + 1) +
query-error-bound + Q i) +
(composition-error-bound +
staged-phase-relation-error size
(staged-alpha-search-queries budgets 0) +
hash-collision-budget-value 0
(staged-alpha-search-queries budgets 0) +
(hash-collision-budget-value 0
(staged-alpha-search-queries budgets 0) + X + (P + R) + (P + R)) +
staged-phase-relation-error size
(staged-alpha-search-queries budgets 0) +
*trace-fri-error + nnreal rounds * query-error-bound)*
proof (rule
checked-staged-soundness-from-aligned-transcript-components-and-trace-index-path-bounds-from-cross-no-prefix-dat
[OF false-statement wf controlled trace-fri-bound comp-fri-bound
trace-frac path-bound cross-bound - data-pre-bound transcript-new-bound
empty-trace-fri-bound empty-query-bound])
show
wp-event
(checked-staged-security-experiment-with-actual-alpha-prefix-data-state
A)
checked-staged-security-with-actual-alpha-prefix-not-prefix-no-prefix-candidate
adversary-initial-state $\leq P + R$
proof (rule *order-trans*
[OF
checked-staged-security-with-actual-alpha-prefix-not-prefix-no-prefix-candidate-le-witness-transcript-hit])
have *pre-bound:*
wp-event
(checked-staged-security-experiment-with-actual-alpha-prefix-data-state
A)
checked-staged-security-with-actual-alpha-prefix-witness-transcript-pre-hit
adversary-initial-state $\leq P$
by (rule
checked-staged-security-with-actual-alpha-prefix-witness-transcript-pre-hit-bound-from-data-state-pre-hit
[OF data-pre-bound])
show

```

    wp-event
      (checked-staged-security-experiment-with-actual-alpha-prefix-data-state
       A)
      checked-staged-security-with-actual-alpha-prefix-witness-transcript-hit
      adversary-initial-state  $\leq P + R$ 
    by (rule
      checked-staged-security-with-actual-alpha-prefix-witness-transcript-hit-bound-from-pre-and-new
      [OF pre-bound transcript-new-bound])
  qed
qed
end
end

```

```

theory Soundness-Staged-Aligned-Current-Query
  imports Soundness-Staged-Aligned-Partial-Merkle
begin

```

Thin wrappers that expose the actual-alpha current-query event in the aligned partial-candidate soundness path. The underlying query reduction still lives in the partial-Merkle layer; this theory only uses the projection proved in the current-query alignment layer.

```

context soundness
begin

```

```

lemma checked-staged-soundness-from-aligned-transcript-components-and-actual-alpha-current-partial-opening-hit-fr
fixes trace-fri-error' composition-fri-error' query-error  $N :: \text{prob}$ 
assumes false-statement:  $\neg \text{exists-valid-trace}$ 
and wf: staged-budget-wellformed budgets
and controlled: staged-adversary-controlled budgets A
and trace-fri-bound:
  wp-event (checked-staged-security-experiment-with-data-state A)
  (staged-security-with-data-state-verifier-event
   trace-fri-bad-with-partial-candidates)
  adversary-initial-state  $\leq \text{trace-fri-error}'$ 
and comp-fri-bound:
  wp-event (checked-staged-security-experiment-with-data-state A)
  (staged-security-with-data-state-verifier-event
   composition-fri-bad-with-partial-candidates)
  adversary-initial-state  $\leq \text{composition-fri-error}'$ 
and current-query-bound:
  wp-event
    (checked-staged-security-experiment-with-actual-alpha-prefix-data-state
     A)
    checked-staged-security-with-actual-alpha-prefix-current-query-partial-opening-hit
    adversary-initial-state  $\leq \text{query-error}$ 
and not-prefix-bound:

```

wp-event
 (checked-staged-security-experiment-with-actual-alpha-prefix-data-state
 A)
 (checked-staged-security-with-actual-alpha-prefix-partial-header-candidate-not-prefix-bound
 composition-trace-bad-alpha-space)
 adversary-initial-state $\leq N$
and empty-trace-fri-bound:
 wp-event (checked-staged-security-experiment-with-data-state *A*)
 (staged-security-with-data-state-verifier-event
 trace-fri-bad-with-empty-composition-header-candidates)
 adversary-initial-state $\leq$ trace-fri-error
and empty-query-bound:
 wp-event (checked-staged-security-experiment-with-data-state *A*)
 (staged-security-with-data-state-verifier-event
 query-bad-with-empty-composition-header-candidates)
 adversary-initial-state $\leq$ nnreal rounds * query-error-bound
shows checked-staged-adversary-acceptance-probability *A* $\leq$
 hash-collision-budget-value 0
 (staged-attacker-query-budget budgets +
 staged-challenge-query-budget + verifier-hash-query-budget) +
 (composition-error-bound +
 staged-phase-relation-error size
 (staged-alpha-search-queries budgets 0) +
 hash-collision-budget-value 0
 (staged-alpha-search-queries budgets 0) +
N +
 staged-phase-relation-error size
 (staged-alpha-search-queries budgets 0)) +
 trace-fri-error' + composition-fri-error' +
 query-error +
 (composition-error-bound +
 staged-phase-relation-error size
 (staged-alpha-search-queries budgets 0) +
 hash-collision-budget-value 0
 (staged-alpha-search-queries budgets 0) +
N +
 staged-phase-relation-error size
 (staged-alpha-search-queries budgets 0) +
 trace-fri-error + nnreal rounds * query-error-bound)
proof –
have data-current-query-bound:
 wp-event (checked-staged-security-experiment-with-data-state *A*)
 staged-security-with-data-state-current-query-partial-opening-hit
 adversary-initial-state $\leq$ query-error
by (rule
 checked-staged-security-with-data-state-current-query-partial-opening-hit-bound-from-actual-alpha-current-query
 [OF current-query-bound])
show ?thesis
by (rule

checked-staged-soundness-from-aligned-transcript-components-and-current-partial-opening-hit-from-not-prefix
 [OF false-statement wf controlled trace-fri-bound comp-fri-bound
 data-current-query-bound not-prefix-bound empty-trace-fri-bound
 empty-query-bound])

qed

theorem *stark-soundness-from-aligned-partial-actual-alpha-current-query*:
fixes *trace-fri-error'* *composition-fri-error'* *query-error* *N* :: prob
assumes *false-statement*: $\neg$ *exists-valid-trace*
and *wf*: *staged-budget-wellformed* *budgets*
and *controlled*: *staged-adversary-controlled* *budgets* *A*
and *trace-fri-bound*:
 wp-event (*checked-staged-security-experiment-with-data-state* *A*)
 (*staged-security-with-data-state-verifier-event*
 trace-fri-bad-with-partial-candidates)
 adversary-initial-state $\leq$ *trace-fri-error'*
and *comp-fri-bound*:
 wp-event (*checked-staged-security-experiment-with-data-state* *A*)
 (*staged-security-with-data-state-verifier-event*
 composition-fri-bad-with-partial-candidates)
 adversary-initial-state $\leq$ *composition-fri-error'*
and *current-query-bound*:
 wp-event
 (*checked-staged-security-experiment-with-actual-alpha-prefix-data-state*
 A)
 checked-staged-security-with-actual-alpha-prefix-current-query-partial-opening-hit
 adversary-initial-state $\leq$ *query-error*
and *not-prefix-bound*:
 wp-event
 (*checked-staged-security-experiment-with-actual-alpha-prefix-data-state*
 A)
 (*checked-staged-security-with-actual-alpha-prefix-partial-header-candidate-not-prefix-bound*
 composition-trace-bad-alpha-space)
 adversary-initial-state $\leq$ *N*
and *empty-trace-fri-bound*:
 wp-event (*checked-staged-security-experiment-with-data-state* *A*)
 (*staged-security-with-data-state-verifier-event*
 trace-fri-bad-with-empty-composition-header-candidates)
 adversary-initial-state $\leq$ *trace-fri-error*
and *empty-query-bound*:
 wp-event (*checked-staged-security-experiment-with-data-state* *A*)
 (*staged-security-with-data-state-verifier-event*
 query-bad-with-empty-composition-header-candidates)
 adversary-initial-state $\leq$ *nnreal* *rounds* * *query-error-bound*
shows *checked-staged-adversary-acceptance-probability* *A* $\leq$
 hash-collision-budget-value *0*
 (*staged-attacker-query-budget* *budgets* +
 staged-challenge-query-budget + *verifier-hash-query-budget*) +
 (*composition-error-bound* +

staged-phase-relation-error size
 (*staged-alpha-search-queries budgets 0*) +
hash-collision-budget-value 0
 (*staged-alpha-search-queries budgets 0*) +
N +
staged-phase-relation-error size
 (*staged-alpha-search-queries budgets 0*) +
trace-fri-error' + composition-fri-error' +
query-error +
 (*composition-error-bound +*
staged-phase-relation-error size
 (*staged-alpha-search-queries budgets 0*) +
hash-collision-budget-value 0
 (*staged-alpha-search-queries budgets 0*) +
N +
staged-phase-relation-error size
 (*staged-alpha-search-queries budgets 0*) +
*trace-fri-error + nnreal rounds * query-error-bound*)
by (*rule*
checked-staged-soundness-from-aligned-transcript-components-and-actual-alpha-current-partial-opening-hit-from-n
 [*OF false-statement wf controlled trace-fri-bound comp-fri-bound*
current-query-bound not-prefix-bound empty-trace-fri-bound
empty-query-bound])

theorem *stark-soundness-from-aligned-partial-query-prefix-current-rounds:*
fixes *trace-fri-error' composition-fri-error' N :: prob*
and *C :: nat $\Rightarrow$ prob*
assumes *false-statement: $\neg$ exists-valid-trace*
and *wf: staged-budget-wellformed budgets*
and *controlled: staged-adversary-controlled budgets A*
and *trace-fri-bound:*
wp-event (checked-staged-security-experiment-with-data-state A)
(staged-security-with-data-state-verifier-event
trace-fri-bad-with-partial-candidates)
adversary-initial-state $\leq$ trace-fri-error'
and *comp-fri-bound:*
wp-event (checked-staged-security-experiment-with-data-state A)
(staged-security-with-data-state-verifier-event
composition-fri-bad-with-partial-candidates)
adversary-initial-state $\leq$ composition-fri-error'
and *round-bound:*
 $\bigwedge i. i < \text{rounds} \implies$
wp-event
(checked-staged-security-experiment-with-query-prefix-data-state A i)
(checked-staged-security-with-query-prefix-current-query-partial-opening-hit-at i)
adversary-initial-state $\leq$ C i
and *not-prefix-bound:*
wp-event
(checked-staged-security-experiment-with-actual-alpha-prefix-data-state

A)
 (checked-staged-security-with-actual-alpha-prefix-partial-header-candidate-not-prefix-bound
 composition-trace-bad-alpha-space)
 adversary-initial-state $\leq N$
and empty-trace-fri-bound:
 wp-event (checked-staged-security-experiment-with-data-state A)
 (staged-security-with-data-state-verifier-event
 trace-fri-bad-with-empty-composition-header-candidates)
 adversary-initial-state $\leq$ trace-fri-error
and empty-query-bound:
 wp-event (checked-staged-security-experiment-with-data-state A)
 (staged-security-with-data-state-verifier-event
 query-bad-with-empty-composition-header-candidates)
 adversary-initial-state $\leq$ nnreal rounds * query-error-bound
shows checked-staged-adversary-acceptance-probability A $\leq$
 hash-collision-budget-value 0
 (staged-attacker-query-budget budgets +
 staged-challenge-query-budget + verifier-hash-query-budget) +
 (composition-error-bound +
 staged-phase-relation-error size
 (staged-alpha-search-queries budgets 0) +
 hash-collision-budget-value 0
 (staged-alpha-search-queries budgets 0) +
 N +
 staged-phase-relation-error size
 (staged-alpha-search-queries budgets 0)) +
 trace-fri-error' + composition-fri-error' +
 ($\sum i < \text{rounds}. C i$) +
 (composition-error-bound +
 staged-phase-relation-error size
 (staged-alpha-search-queries budgets 0) +
 hash-collision-budget-value 0
 (staged-alpha-search-queries budgets 0) +
 N +
 staged-phase-relation-error size
 (staged-alpha-search-queries budgets 0) +
 trace-fri-error + nnreal rounds * query-error-bound)
proof –
have current-query-bound:
 wp-event
 (checked-staged-security-experiment-with-actual-alpha-prefix-data-state A)
 checked-staged-security-with-actual-alpha-prefix-current-query-partial-opening-hit
 adversary-initial-state $\leq$ ($\sum i < \text{rounds}. C i$)
by (rule
 checked-staged-security-with-actual-alpha-prefix-current-query-partial-opening-hit-bound-from-query-prefix-current
 [OF wf controlled round-bound])
show ?thesis
by (rule
 stark-soundness-from-aligned-partial-actual-alpha-current-query

[*OF false-statement wf controlled trace-fri-bound comp-fri-bound
current-query-bound not-prefix-bound empty-trace-fri-bound
empty-query-bound*])

qed

theorem *stark-soundness-from-aligned-partial-query-prefix-current-prefix-path:*

fixes *trace-fri-error' composition-fri-error' N :: prob*

and *P Q :: nat $\Rightarrow$ prob*

assumes *false-statement: $\neg$ exists-valid-trace*

and *wf: staged-budget-wellformed budgets*

and *controlled: staged-adversary-controlled budgets A*

and *trace-fri-bound:*

*wp-event (checked-staged-security-experiment-with-data-state A)
(staged-security-with-data-state-verifier-event
trace-fri-bad-with-partial-candidates)
adversary-initial-state $\leq$ trace-fri-error'*

and *comp-fri-bound:*

*wp-event (checked-staged-security-experiment-with-data-state A)
(staged-security-with-data-state-verifier-event
composition-fri-bad-with-partial-candidates)
adversary-initial-state $\leq$ composition-fri-error'*

and *prefix-bound:*

*$\bigwedge i. i < \text{rounds} \Rightarrow$
wp-event
(checked-staged-security-experiment-with-query-prefix-data-state A i)
(checked-staged-security-with-query-prefix-current-prefix-authenticated-hit-at i)
adversary-initial-state $\leq$ P i*

and *path-bound:*

*$\bigwedge i. i < \text{rounds} \Rightarrow$
wp-event
(checked-staged-security-experiment-with-query-prefix-data-state A i)
(checked-staged-security-with-query-prefix-current-path-output-hit-at i)
adversary-initial-state $\leq$ Q i*

and *not-prefix-bound:*

wp-event
(checked-staged-security-experiment-with-actual-alpha-prefix-data-state A)
(checked-staged-security-with-actual-alpha-prefix-partial-header-candidate-not-prefix-bound
composition-trace-bad-alpha-space)
adversary-initial-state $\leq$ N

and *empty-trace-fri-bound:*

*wp-event (checked-staged-security-experiment-with-data-state A)
(staged-security-with-data-state-verifier-event
trace-fri-bad-with-empty-composition-header-candidates)
adversary-initial-state $\leq$ trace-fri-error*

and *empty-query-bound:*

*wp-event (checked-staged-security-experiment-with-data-state A)
(staged-security-with-data-state-verifier-event
query-bad-with-empty-composition-header-candidates)*

$adversary-initial-state \leq n_{real} \text{ rounds} * query-error-bound$
shows $checked-staged-adversary-acceptance-probability A \leq$
 $hash-collision-budget-value 0$
 $(staged-attacker-query-budget budgets +$
 $staged-challenge-query-budget + verifier-hash-query-budget) +$
 $(composition-error-bound +$
 $staged-phase-relation-error size$
 $(staged-alpha-search-queries budgets 0) +$
 $hash-collision-budget-value 0$
 $(staged-alpha-search-queries budgets 0) +$
 $N +$
 $staged-phase-relation-error size$
 $(staged-alpha-search-queries budgets 0)) +$
 $trace-fri-error' + composition-fri-error' +$
 $(\sum i < rounds. P i + Q i) +$
 $(composition-error-bound +$
 $staged-phase-relation-error size$
 $(staged-alpha-search-queries budgets 0) +$
 $hash-collision-budget-value 0$
 $(staged-alpha-search-queries budgets 0) +$
 $N +$
 $staged-phase-relation-error size$
 $(staged-alpha-search-queries budgets 0) +$
 $trace-fri-error + n_{real} \text{ rounds} * query-error-bound)$
proof (rule stark-soundness-from-aligned-partial-query-prefix-current-rounds
 $[OF false-statement wf controlled trace-fri-bound comp-fri-bound])$
fix i
assume $i-bound: i < rounds$
show
 $wp-event$
 $(checked-staged-security-experiment-with-query-prefix-data-state A i)$
 $(checked-staged-security-with-query-prefix-current-query-partial-opening-hit-at i)$
 $adversary-initial-state \leq P i + Q i$
by (rule
 $checked-staged-security-with-query-prefix-current-query-partial-opening-hit-at-bound-from-prefix-and-path$
 $[OF wf controlled i-bound prefix-bound [OF i-bound]$
 $path-bound [OF i-bound]])$
qed (rule not-prefix-bound empty-trace-fri-bound empty-query-bound)+

theorem stark-soundness-from-aligned-partial-query-prefix-dynamic-target-path:
fixes $trace-fri-error' composition-fri-error' N :: prob$
and $P Q :: nat \Rightarrow prob$
assumes $false-statement: \neg exists-valid-trace$
and $wf: staged-budget-wellformed budgets$
and $controlled: staged-adversary-controlled budgets A$
and $trace-fri-bound:$
 $wp-event (checked-staged-security-experiment-with-data-state A)$
 $(staged-security-with-data-state-verifier-event$
 $trace-fri-bad-with-partial-candidates)$

$\text{adversary-initial-state} \leq \text{trace-fri-error}'$
and *comp-fri-bound*:
 $\text{wp-event } (\text{checked-staged-security-experiment-with-data-state } A)$
 $(\text{staged-security-with-data-state-verifier-event}$
 $\text{composition-fri-bad-with-partial-candidates})$
 $\text{adversary-initial-state} \leq \text{composition-fri-error}'$
and *prefix-target-bound*:
 $\bigwedge i. i < \text{rounds} \implies$
 wp-event
 $(\text{checked-staged-query-prefix-receive-with-state } A \ i)$
 $(\text{checked-staged-query-prefix-dynamic-index-hit}$
 $\text{staged-query-prefix-prefix-authenticated-opening-target})$
 $\text{adversary-initial-state} \leq P \ i$
and *path-bound*:
 $\bigwedge i. i < \text{rounds} \implies$
 wp-event
 $(\text{checked-staged-security-experiment-with-query-prefix-data-state } A \ i)$
 $(\text{checked-staged-security-with-query-prefix-current-path-output-hit-at } i)$
 $\text{adversary-initial-state} \leq Q \ i$
and *not-prefix-bound*:
 wp-event
 $(\text{checked-staged-security-experiment-with-actual-alpha-prefix-data-state}$
 $A)$
 $(\text{checked-staged-security-with-actual-alpha-prefix-partial-header-candidate-not-prefix-bound}$
 $\text{composition-trace-bad-alpha-space})$
 $\text{adversary-initial-state} \leq N$
and *empty-trace-fri-bound*:
 $\text{wp-event } (\text{checked-staged-security-experiment-with-data-state } A)$
 $(\text{staged-security-with-data-state-verifier-event}$
 $\text{trace-fri-bad-with-empty-composition-header-candidates})$
 $\text{adversary-initial-state} \leq \text{trace-fri-error}$
and *empty-query-bound*:
 $\text{wp-event } (\text{checked-staged-security-experiment-with-data-state } A)$
 $(\text{staged-security-with-data-state-verifier-event}$
 $\text{query-bad-with-empty-composition-header-candidates})$
 $\text{adversary-initial-state} \leq \text{nnreal rounds} * \text{query-error-bound}$
shows $\text{checked-staged-adversary-acceptance-probability } A \leq$
 $\text{hash-collision-budget-value } 0$
 $(\text{staged-attacker-query-budget budgets} +$
 $\text{staged-challenge-query-budget} + \text{verifier-hash-query-budget}) +$
 $(\text{composition-error-bound} +$
 $\text{staged-phase-relation-error size}$
 $(\text{staged-alpha-search-queries budgets } 0) +$
 $\text{hash-collision-budget-value } 0$
 $(\text{staged-alpha-search-queries budgets } 0) +$
 $N +$
 $\text{staged-phase-relation-error size}$
 $(\text{staged-alpha-search-queries budgets } 0)) +$
 $\text{trace-fri-error}' + \text{composition-fri-error}' +$

```

( $\sum_{i < \text{rounds}} P\ i + Q\ i$ ) +
(composition-error-bound +
 staged-phase-relation-error size
 (staged-alpha-search-queries budgets 0) +
 hash-collision-budget-value 0
 (staged-alpha-search-queries budgets 0) +
 N +
 staged-phase-relation-error size
 (staged-alpha-search-queries budgets 0) +
 trace-fri-error + nnreal rounds * query-error-bound)
proof -
  have prefix-bound:
     $\bigwedge i. i < \text{rounds} \implies$ 
      wp-event
      (checked-staged-security-experiment-with-query-prefix-data-state A i)
      (checked-staged-security-with-query-prefix-current-prefix-authenticated-hit-at i)
      adversary-initial-state  $\leq P\ i$ 
    by (rule
      checked-staged-security-with-query-prefix-current-prefix-authenticated-hit-at-bound-from-dynamic-target
      [OF wf controlled - prefix-target-bound])
    show ?thesis
    by (rule stark-soundness-from-aligned-partial-query-prefix-current-prefix-path
      [OF false-statement wf controlled trace-fri-bound comp-fri-bound
        prefix-bound path-bound not-prefix-bound empty-trace-fri-bound
        empty-query-bound])
qed

end

end

theory Soundness-Conceptual-Query-Current-Path
imports
  Soundness-Conceptual-Query-Current
  Staged-Security-Experiment-Composition-Query-Current-Path
begin

Integration of conceptual prefix-query targets with the existing structured current-query
path split.

context soundness
begin

definition checked-staged-security-with-query-prefix-conceptual-context-residual-hit-at
where
  checked-staged-security-with-query-prefix-conceptual-context-residual-hit-at
     $i\ \text{out} \longleftrightarrow$ 
    checked-staged-security-with-query-prefix-current-query-partial-opening-hit-at
     $i\ \text{out} \wedge$ 

```

checked-staged-security-with-query-prefix-conceptual-context-residual-at
out

definition *checked-staged-security-with-query-prefix-conceptual-default-context-residual-hit-at*
where

checked-staged-security-with-query-prefix-conceptual-default-context-residual-hit-at
trace-default composition-default i out $\longleftrightarrow$
checked-staged-security-with-query-prefix-current-query-partial-opening-hit-at
i out $\wedge$
checked-staged-security-with-query-prefix-conceptual-default-context-residual-at
trace-default composition-default out

definition *checked-staged-security-with-query-prefix-candidate-pair-default-agreement-gap*
where

checked-staged-security-with-query-prefix-candidate-pair-default-agreement-gap
trace-table composition-table out $\longleftrightarrow$
 (case out of
 None $\Rightarrow$ False
 | Some (((((prefix, prefix-state), raw), raw-state),
 ((data, attacker-state), result)), final-state) $\Rightarrow$
 $\neg$ query-prefix-candidate-defaults-agree-with-authenticated-values
 trace-table composition-table prefix prefix-state)

definition *checked-staged-security-with-query-prefix-candidate-pair-sampled-default-agreement-gap*
where

checked-staged-security-with-query-prefix-candidate-pair-sampled-default-agreement-gap
trace-table composition-table out $\longleftrightarrow$
 (case out of
 None $\Rightarrow$ False
 | Some (((((prefix, prefix-state), raw), raw-state),
 ((data, attacker-state), result)), final-state) $\Rightarrow$
 $\neg$ query-prefix-candidate-defaults-agree-at-query *trace-table*
 composition-table (index (to-nat raw)) prefix prefix-state)

definition *checked-staged-security-with-query-prefix-authenticated-candidate-hit-at*
where

checked-staged-security-with-query-prefix-authenticated-candidate-hit-at
trace-table composition-table i out $\longleftrightarrow$
 ($\exists$ *trace-openings composition-openings*.
checked-staged-security-with-query-prefix-authenticated-opening-hit
trace-openings composition-openings i out $\wedge$
partial-trace-table-candidate trace-table
 ((replicate rounds []) [i := *trace-openings*]) $\wedge$
partial-composition-table-candidate composition-table
 ((replicate rounds []) [i := *composition-openings*]))

lemma *checked-staged-security-with-query-prefix-candidate-pair-hit-imp-conceptual-default-or-agreement-gap*:
assumes *hit*:

checked-staged-security-with-query-prefix-candidate-pair-from-prefix-hit

```

    trace-table composition-table
    (Some (((((prefix, prefix-state), raw), raw-state),
      ((data, attacker-state), result)), final-state))
  and clean:  $\neg$  hash-map-output-collision prefix-state
  and trace-len: length trace-table = scale * clength
  and composition-len: length composition-table = scale * clength
shows
  checked-staged-security-with-query-prefix-conceptual-default-target-hit
  trace-table composition-table
  (Some (((((prefix, prefix-state), raw), raw-state),
    ((data, attacker-state), result)), final-state))  $\vee$ 
  checked-staged-security-with-query-prefix-candidate-pair-default-agreement-gap
  trace-table composition-table
  (Some (((((prefix, prefix-state), raw), raw-state),
    ((data, attacker-state), result)), final-state))
proof (cases
  query-prefix-candidate-defaults-agree-with-authenticated-values
  trace-table composition-table prefix prefix-state)
case True
have subset:
  staged-query-prefix-candidate-pair-query-target-from-prefix trace-table
  composition-table prefix prefix-state  $\subseteq$ 
  staged-query-prefix-conceptual-default-query-target trace-table
  composition-table prefix prefix-state
by (rule
  staged-query-prefix-candidate-pair-query-target-from-prefix-subset-conceptual-default-if-prefix-agrees
  [OF clean trace-len composition-len True])
have
  checked-staged-security-with-query-prefix-conceptual-default-target-hit
  trace-table composition-table
  (Some (((((prefix, prefix-state), raw), raw-state),
    ((data, attacker-state), result)), final-state))
using hit subset
unfolding
  checked-staged-security-with-query-prefix-candidate-pair-from-prefix-hit-def
  checked-staged-security-with-query-prefix-conceptual-default-target-hit-def
  checked-staged-security-with-query-prefix-dynamic-index-hit-def
  checked-staged-query-prefix-dynamic-index-hit-def
by auto
then show ?thesis by simp
next
case False
then have
  checked-staged-security-with-query-prefix-candidate-pair-default-agreement-gap
  trace-table composition-table
  (Some (((((prefix, prefix-state), raw), raw-state),
    ((data, attacker-state), result)), final-state))
unfolding
  checked-staged-security-with-query-prefix-candidate-pair-default-agreement-gap-def

```

by simp
 then show ?thesis by simp
 qed

lemma *checked-staged-security-with-query-prefix-candidate-pair-hit-imp-conceptual-default-or-sampled-agreement-gap*

assumes *hit*:

checked-staged-security-with-query-prefix-candidate-pair-from-prefix-hit
trace-table composition-table
 (Some (((((prefix, prefix-state), raw), raw-state),
 ((data, attacker-state), result)), final-state))

and *trace-len*: length *trace-table* = scale * clength

and *composition-len*: length *composition-table* = scale * clength

and *default-trace-low*:

trace-table-low-degree
 (query-prefix-trace-conceptual-table-with-default prefix prefix-state
 trace-table)

and *default-composition-low*:

composition-table-low-degree maxDegree
 (query-prefix-composition-conceptual-table-with-default prefix
 prefix-state composition-table)

and *default-not-all*:

¬ all-queries-consistent
 (query-prefix-trace-conceptual-table-with-default prefix prefix-state
 trace-table)
 (query-prefix-composition-conceptual-table-with-default prefix
 prefix-state composition-table)
 (sqp-alphas prefix)

shows

checked-staged-security-with-query-prefix-conceptual-default-target-hit
trace-table composition-table
 (Some (((((prefix, prefix-state), raw), raw-state),
 ((data, attacker-state), result)), final-state)) ∨
checked-staged-security-with-query-prefix-candidate-pair-sampled-default-agreement-gap
trace-table composition-table
 (Some (((((prefix, prefix-state), raw), raw-state),
 ((data, attacker-state), result)), final-state))

proof (cases

query-prefix-candidate-defaults-agree-at-query trace-table
composition-table (index (to-nat raw)) prefix prefix-state)

case *True*

have *raw-hit*:

index (to-nat raw) ∈
staged-query-prefix-candidate-pair-query-target-from-prefix trace-table
composition-table prefix prefix-state

using *hit*

unfolding

checked-staged-security-with-query-prefix-candidate-pair-from-prefix-hit-def
checked-staged-security-with-query-prefix-dynamic-index-hit-def
checked-staged-query-prefix-dynamic-index-hit-def

```

    by simp
  have default-raw-hit:
    index (to-nat raw) ∈
      staged-query-prefix-conceptual-default-query-target trace-table
      composition-table prefix prefix-state
  by (rule
    query-prefix-candidate-pair-query-hit-imp-conceptual-default-if-agrees-at-query
    [OF raw-hit trace-len composition-len True default-trace-low
      default-composition-low default-not-all])
  have
    checked-staged-security-with-query-prefix-conceptual-default-target-hit
      trace-table composition-table
      (Some (((((prefix, prefix-state), raw), raw-state),
        ((data, attacker-state), result)), final-state))
  unfolding
    checked-staged-security-with-query-prefix-conceptual-default-target-hit-def
    checked-staged-security-with-query-prefix-dynamic-index-hit-def
    checked-staged-query-prefix-dynamic-index-hit-def
  using default-raw-hit by simp
  then show ?thesis by simp
next
  case False
  then have
    checked-staged-security-with-query-prefix-candidate-pair-sampled-default-agreement-gap
      trace-table composition-table
      (Some (((((prefix, prefix-state), raw), raw-state),
        ((data, attacker-state), result)), final-state))
  unfolding
    checked-staged-security-with-query-prefix-candidate-pair-sampled-default-agreement-gap-def
  by simp
  then show ?thesis by simp
qed

lemma checked-staged-security-with-query-prefix-candidate-pair-hit-imp-conceptual-default-if-final-merkle-bound:
  assumes hit:
    checked-staged-security-with-query-prefix-candidate-pair-from-prefix-hit
      trace-table composition-table
      (Some (((((prefix, prefix-state), raw), raw-state),
        ((data, attacker-state), result)), final-state))
  and prefix-clean:  $\neg$  hash-map-output-collision prefix-state
  and final-clean:  $\neg$  hash-map-output-collision final-state
  and ext:  $\text{prefix-state} \leq \text{final-state}$ 
  and trace-bind:
    merkle-root-binds-table (sqp-trace-root prefix) trace-table
    final-state
  and composition-bind:
    merkle-root-binds-table (hd (sqp-composition-fri-roots prefix))
    composition-table final-state
  and trace-len:  $\text{length trace-table} = \text{scale} * \text{clength}$ 

```

and *composition-len*: $\text{length } \text{composition-table} = \text{scale} * \text{clength}$
shows
checked-staged-security-with-query-prefix-conceptual-default-target-hit
trace-table composition-table
(Some (((((prefix, prefix-state), raw), raw-state),
((data, attacker-state), result)), final-state))
proof –
have *subset*:
staged-query-prefix-candidate-pair-query-target-from-prefix trace-table
composition-table prefix prefix-state $\subseteq$
staged-query-prefix-conceptual-default-query-target trace-table
composition-table prefix prefix-state
by (rule
staged-query-prefix-candidate-pair-query-target-from-prefix-subset-conceptual-default-if-final-merkle-bound
[OF prefix-clean final-clean ext trace-bind composition-bind trace-len
composition-len])
show *?thesis*
using *hit subset*
unfolding
checked-staged-security-with-query-prefix-candidate-pair-from-prefix-hit-def
checked-staged-security-with-query-prefix-conceptual-default-target-hit-def
checked-staged-security-with-query-prefix-dynamic-index-hit-def
checked-staged-query-prefix-dynamic-index-hit-def
by *auto*
qed

lemma *checked-staged-security-with-query-prefix-authenticated-opening-hit-imp-conceptual-default-or-residual-or-stru*
assumes *wf*: *staged-budget-wellformed budgets*
and *controlled*: *staged-adversary-controlled budgets A*
and *i-bound*: $i < \text{rounds}$
and *support*:
Some (((((prefix, prefix-state), raw), raw-state),
((data, attacker-state), result)), final-state) $\in$
set-dist
(execute
(checked-staged-security-experiment-with-query-prefix-data-state A i)
adversary-initial-state)
and *hit*:
checked-staged-security-with-query-prefix-authenticated-opening-hit
trace-openings composition-openings i
(Some (((((prefix, prefix-state), raw), raw-state),
((data, attacker-state), result)), final-state))
and *trace-candidate*:
partial-trace-table-candidate trace-table
((replicate rounds []) [i := trace-openings])
and *comp-candidate*:
partial-composition-table-candidate composition-table
((replicate rounds []) [i := composition-openings])
shows


```

checked-staged-security-with-query-prefix-conceptual-default-target-hit
  trace-table composition-table
  (Some (((((prefix, prefix-state), raw), raw-state),
    ((data, attacker-state), result)), final-state)) ∨
checked-staged-security-with-query-prefix-conceptual-default-context-residual-hit-at
  trace-table composition-table i
  (Some (((((prefix, prefix-state), raw), raw-state),
    ((data, attacker-state), result)), final-state)) ∨
checked-staged-security-with-query-prefix-current-trace-path-output-hit-at
  i
  (Some (((((prefix, prefix-state), raw), raw-state),
    ((data, attacker-state), result)), final-state)) ∨
checked-staged-security-with-query-prefix-current-composition-path-output-hit-at
  i
  (Some (((((prefix, prefix-state), raw), raw-state),
    ((data, attacker-state), result)), final-state))
proof –
  let ?out =
    Some (((((prefix, prefix-state), raw), raw-state),
      ((data, attacker-state), result)), final-state)
  have current-hit:
    checked-staged-security-with-query-prefix-current-query-partial-opening-hit-at
      i ?out
  by (rule
    checked-staged-security-with-query-prefix-current-query-partial-opening-hit-atI
    [OF hit])
  show ?thesis
  proof (cases
    ¬ hash-map-output-collision prefix-state ∧
    trace-table-low-degree
    (query-prefix-trace-conceptual-table-with-default prefix
      prefix-state trace-table) ∧
    composition-table-low-degree maxDegree
    (query-prefix-composition-conceptual-table-with-default prefix
      prefix-state composition-table) ∧
    ¬ all-queries-consistent
    (query-prefix-trace-conceptual-table-with-default prefix
      prefix-state trace-table)
    (query-prefix-composition-conceptual-table-with-default prefix
      prefix-state composition-table)
    (sqp-alphas prefix))
  case False
  then have
    checked-staged-security-with-query-prefix-conceptual-default-context-residual-hit-at
      trace-table composition-table i ?out
  unfolding
    checked-staged-security-with-query-prefix-conceptual-default-context-residual-hit-at-def
    checked-staged-security-with-query-prefix-conceptual-default-context-residual-at-def
  using current-hit by simp

```

```

then show ?thesis by simp
next
case True
then have clean:
   $\neg$  hash-map-output-collision prefix-state
and default-trace-low:
  trace-table-low-degree
  (query-prefix-trace-conceptual-table-with-default prefix
   prefix-state trace-table)
and default-composition-low:
  composition-table-low-degree maxDegree
  (query-prefix-composition-conceptual-table-with-default prefix
   prefix-state composition-table)
and default-not-all:
   $\neg$  all-queries-consistent
  (query-prefix-trace-conceptual-table-with-default prefix
   prefix-state trace-table)
  (query-prefix-composition-conceptual-table-with-default prefix
   prefix-state composition-table)
  (sqp-alphas prefix)
by blast+
have ext: prefix-state  $\leq$  final-state
by (rule
  checked-staged-security-experiment-with-query-prefix-data-state-prefix-extends-final
  [OF wf controlled i-bound support])
have alphas-eq: staged-alphas data = sqp-alphas prefix
by (rule checked-staged-security-with-query-prefix-data-state-alphas-eq
  [OF support])
have trace-root-eq: staged-trace-root data = sqp-trace-root prefix
using support
unfolding checked-staged-security-experiment-with-query-prefix-data-state-def
  checked-staged-after-query-prefix-receive-with-verifier-def
  checked-staged-after-query-prefix-receive-def
by (auto elim!: set-dist-bindE split: prod.splits)
have composition-roots-eq:
  staged-composition-fri-roots data = sqp-composition-fri-roots prefix
using support
unfolding checked-staged-security-experiment-with-query-prefix-data-state-def
  checked-staged-after-query-prefix-receive-with-verifier-def
  checked-staged-after-query-prefix-receive-def
by (auto elim!: set-dist-bindE split: prod.splits)
have comp-nonempty: staged-composition-fri-roots data  $\neq$  []
and trace-table-final:
  partial-authenticated-table (staged-trace-root data)
  (scale * clength) trace-openings final-state
and comp-table-final:
  partial-authenticated-table (hd (staged-composition-fri-roots data))
  (scale * clength) composition-openings final-state
and consistent:

```

```

    partial-query-openings-consistent trace-openings composition-openings
      (staged-alphas data) (index (to-nat raw))
using hit
unfolding checked-staged-security-with-query-prefix-authenticated-opening-hit-def
by simp-all
have trace-pullback:
  partial-authenticated-table (staged-trace-root data)
    (scale * clength) trace-openings prefix-state ∨
    (∃ opn ∈ set trace-openings.
      hash-map-new-output-hit
        (merkle-path-target-roots final-state (staged-trace-root data)
          (scale * clength) (opening-index opn) (opening-value opn)
          (opening-path opn)) prefix-state final-state)
by (rule partial-authenticated-table-pullback-or-new-output-hit
      [OF ext trace-table-final])
then show ?thesis
proof
  assume trace-prefix-staged:
    partial-authenticated-table (staged-trace-root data)
      (scale * clength) trace-openings prefix-state
  have comp-pullback:
    partial-authenticated-table (hd (staged-composition-fri-roots data))
      (scale * clength) composition-openings prefix-state ∨
      (∃ opn ∈ set composition-openings.
        hash-map-new-output-hit
          (merkle-path-target-roots final-state
            (hd (staged-composition-fri-roots data))
            (scale * clength) (opening-index opn) (opening-value opn)
            (opening-path opn)) prefix-state final-state)
by (rule partial-authenticated-table-pullback-or-new-output-hit
      [OF ext comp-table-final])
then show ?thesis
proof
  assume comp-prefix-staged:
    partial-authenticated-table (hd (staged-composition-fri-roots data))
      (scale * clength) composition-openings prefix-state
  have trace-prefix:
    partial-authenticated-table (sqp-trace-root prefix)
      (scale * clength) trace-openings prefix-state
    using trace-prefix-staged trace-root-eq by simp
  have composition-prefix:
    partial-authenticated-table (hd (sqp-composition-fri-roots prefix))
      (scale * clength) composition-openings prefix-state
    using comp-prefix-staged composition-roots-eq by simp
  have consistent-prefix:
    partial-query-openings-consistent trace-openings
      composition-openings (sqp-alphas prefix) (index (to-nat raw))
    using consistent-alphas-eq by simp
  have target:

```

```

index (to-nat raw) ∈
  staged-query-prefix-conceptual-default-query-target trace-table
  composition-table prefix prefix-state
by (rule
  query-prefix-current-openings-imp-conceptual-default-query-target
  [OF clean i-bound trace-prefix composition-prefix
   consistent-prefix trace-candidate comp-candidate
   default-trace-low default-composition-low default-not-all])
have
  checked-staged-security-with-query-prefix-conceptual-default-target-hit
  trace-table composition-table ?out
unfolding
  checked-staged-security-with-query-prefix-conceptual-default-target-hit-def
  checked-staged-security-with-query-prefix-dynamic-index-hit-def
  checked-staged-query-prefix-dynamic-index-hit-def
using target by simp
then show ?thesis by simp
next
assume comp-path:
  ∃ opn ∈ set composition-openings.
  hash-map-new-output-hit
  (merkle-path-target-roots final-state
   (hd (staged-composition-fri-roots data))
   (scale * clength) (opening-index opn) (opening-value opn)
   (opening-path opn)) prefix-state final-state
then obtain opn where opn-in: opn ∈ set composition-openings
and path-hit:
  hash-map-new-output-hit
  (merkle-path-target-roots final-state
   (hd (staged-composition-fri-roots data))
   (scale * clength) (opening-index opn) (opening-value opn)
   (opening-path opn)) prefix-state final-state
by blast
have auth: authenticated-opening-in final-state opn
using comp-table-final opn-in
unfolding partial-authenticated-table-def by blast
have root-eq:
  opening-root opn = hd (staged-composition-fri-roots data)
using comp-table-final opn-in
unfolding partial-authenticated-table-def by blast
have len-eq: opening-length opn = scale * clength
using comp-table-final opn-in
unfolding partial-authenticated-table-def by blast
have
  checked-staged-security-with-query-prefix-current-composition-path-output-hit-at
  i ?out
unfolding
  checked-staged-security-with-query-prefix-current-composition-path-output-hit-at-def
using comp-nonempty auth root-eq len-eq path-hit by auto

```

```

    then show ?thesis by simp
qed
next
  assume trace-path:
     $\exists \text{opn} \in \text{set trace-openings.}$ 
    hash-map-new-output-hit
      (merkle-path-target-roots final-state (staged-trace-root data)
        (scale * clength) (opening-index opn) (opening-value opn)
        (opening-path opn)) prefix-state final-state
  then obtain opn where opn-in:  $\text{opn} \in \text{set trace-openings}$ 
  and path-hit:
    hash-map-new-output-hit
      (merkle-path-target-roots final-state (staged-trace-root data)
        (scale * clength) (opening-index opn) (opening-value opn)
        (opening-path opn)) prefix-state final-state
  by blast
  have auth: authenticated-opening-in final-state opn
  using trace-table-final opn-in
  unfolding partial-authenticated-table-def by blast
  have root-eq: opening-root opn = staged-trace-root data
  using trace-table-final opn-in
  unfolding partial-authenticated-table-def by blast
  have len-eq: opening-length opn = scale * clength
  using trace-table-final opn-in
  unfolding partial-authenticated-table-def by blast
  have
    checked-staged-security-with-query-prefix-current-trace-path-output-hit-at
      i ?out
  unfolding
    checked-staged-security-with-query-prefix-current-trace-path-output-hit-at-def
  using auth root-eq len-eq path-hit by auto
  then show ?thesis by simp
qed
qed
qed

lemma checked-staged-security-with-query-prefix-authenticated-candidate-hit-at-imp-conceptual-default-or-residual-or-
  assumes wf: staged-budget-wellformed budgets
  and controlled: staged-adversary-controlled budgets A
  and i-bound:  $i < \text{rounds}$ 
  and support:
    out  $\in$ 
    set-dist
      (execute
        (checked-staged-security-experiment-with-query-prefix-data-state A i)
        adversary-initial-state)
  and hit:
    checked-staged-security-with-query-prefix-authenticated-candidate-hit-at
      trace-table composition-table i out

```

shows

checked-staged-security-with-query-prefix-conceptual-default-target-hit
 trace-table composition-table out $\vee$
checked-staged-security-with-query-prefix-conceptual-default-context-residual-hit-at
 trace-table composition-table i out $\vee$
checked-staged-security-with-query-prefix-current-trace-path-output-hit-at
 i out $\vee$
checked-staged-security-with-query-prefix-current-composition-path-output-hit-at
 i out

proof (*cases out*)

case *None*

then show *?thesis*

using *hit*

unfolding

checked-staged-security-with-query-prefix-authenticated-candidate-hit-at-def

checked-staged-security-with-query-prefix-authenticated-opening-hit-def

by *simp*

next

case (*Some packed*)

then obtain *prefix prefix-state raw raw-state data attacker-state result*
 final-state **where** *out-eq*:

out =

Some ((((((*prefix, prefix-state*), *raw*), *raw-state*),
 ((*data, attacker-state*), *result*)), *final-state*)

by (*cases packed*) (*auto split: prod.splits*)

from *hit[unfolded out-eq]*

checked-staged-security-with-query-prefix-authenticated-candidate-hit-at-def

obtain *trace-openings composition-openings* **where** *opening-hit*:

checked-staged-security-with-query-prefix-authenticated-opening-hit

trace-openings composition-openings i

 (*Some* ((((((*prefix, prefix-state*), *raw*), *raw-state*),
 ((*data, attacker-state*), *result*)), *final-state*))

and *trace-candidate*:

partial-trace-table-candidate trace-table

 ((*replicate rounds []*) [*i := trace-openings*])

and *comp-candidate*:

partial-composition-table-candidate composition-table

 ((*replicate rounds []*) [*i := composition-openings*])

by *blast*

have *support'*:

Some ((((((*prefix, prefix-state*), *raw*), *raw-state*),

 ((*data, attacker-state*), *result*)), *final-state*) $\in$

set-dist

 (*execute*

 (*checked-staged-security-experiment-with-query-prefix-data-state A i*)

adversary-initial-state)

using *support* **unfolding** *out-eq* .

show *?thesis*

unfolding *out-eq*

by (rule
 checked-staged-security-with-query-prefix-authenticated-opening-hit-imp-conceptual-default-or-residual-or-structu
 [*OF wf controlled i-bound support' opening-hit trace-candidate*
 comp-candidate])
 qed

lemma *checked-staged-security-with-query-prefix-conceptual-context-residual-hit-at-imp-current:*
assumes
 checked-staged-security-with-query-prefix-conceptual-context-residual-hit-at
 i out
shows
 checked-staged-security-with-query-prefix-current-query-partial-opening-hit-at
 i out
using *assms*
unfolding
 checked-staged-security-with-query-prefix-conceptual-context-residual-hit-at-def
by *simp*

lemma *checked-staged-security-with-query-prefix-conceptual-context-residual-hit-at-imp-residual:*
assumes
 checked-staged-security-with-query-prefix-conceptual-context-residual-hit-at
 i out
shows
 checked-staged-security-with-query-prefix-conceptual-context-residual-at
 out
using *assms*
unfolding
 checked-staged-security-with-query-prefix-conceptual-context-residual-hit-at-def
by *simp*

definition *checked-staged-security-with-query-prefix-conceptual-context-prefix-collision-hit-at*
where
 checked-staged-security-with-query-prefix-conceptual-context-prefix-collision-hit-at
 i out $\longleftrightarrow$
 checked-staged-security-with-query-prefix-current-query-partial-opening-hit-at
 i out $\wedge$
 (case out of
 None $\Rightarrow$ False
 | Some (((((*prefix*, *prefix-state*), *raw*), *raw-state*),
 ((*data*, *attacker-state*), *result*)), *final-state*) $\Rightarrow$
 hash-map-output-collision prefix-state)

definition *checked-staged-security-with-query-prefix-conceptual-context-algebraic-residual-hit-at*
where
 checked-staged-security-with-query-prefix-conceptual-context-algebraic-residual-hit-at
 i out $\longleftrightarrow$
 checked-staged-security-with-query-prefix-current-query-partial-opening-hit-at
 i out $\wedge$
 (case out of

$None \Rightarrow False$
 $| \text{Some } (((((prefix, prefix-state), raw), raw-state),$
 $((data, attacker-state), result)), final-state) \Rightarrow$
 $\neg \text{trace-table-low-degree}$
 $(\text{query-prefix-trace-conceptual-table } prefix \text{ } prefix-state) \vee$
 $\neg \text{composition-table-low-degree } maxDegree$
 $(\text{query-prefix-composition-conceptual-table } prefix \text{ } prefix-state) \vee$
 $\text{all-queries-consistent}$
 $(\text{query-prefix-trace-conceptual-table } prefix \text{ } prefix-state)$
 $(\text{query-prefix-composition-conceptual-table } prefix \text{ } prefix-state)$
 $(\text{sqp-alphas } prefix))$

definition *checked-staged-security-with-query-prefix-conceptual-trace-low-degree-bad-at*
where

$\text{checked-staged-security-with-query-prefix-conceptual-trace-low-degree-bad-at}$
 $i \text{ out} \longleftrightarrow$
 $\text{checked-staged-security-with-query-prefix-conceptual-context-algebraic-residual-hit-at}$
 $i \text{ out} \wedge$
 $(\text{case out of}$
 $None \Rightarrow False$
 $| \text{Some } (((((prefix, prefix-state), raw), raw-state),$
 $((data, attacker-state), result)), final-state) \Rightarrow$
 $\neg \text{trace-table-low-degree}$
 $(\text{query-prefix-trace-conceptual-table } prefix \text{ } prefix-state))$

definition *checked-staged-security-with-query-prefix-conceptual-composition-low-degree-bad-at*
where

$\text{checked-staged-security-with-query-prefix-conceptual-composition-low-degree-bad-at}$
 $i \text{ out} \longleftrightarrow$
 $\text{checked-staged-security-with-query-prefix-conceptual-context-algebraic-residual-hit-at}$
 $i \text{ out} \wedge$
 $(\text{case out of}$
 $None \Rightarrow False$
 $| \text{Some } (((((prefix, prefix-state), raw), raw-state),$
 $((data, attacker-state), result)), final-state) \Rightarrow$
 $\neg \text{composition-table-low-degree } maxDegree$
 $(\text{query-prefix-composition-conceptual-table } prefix \text{ } prefix-state))$

definition *checked-staged-security-with-query-prefix-conceptual-alpha-bad-set-hit-at*
where

$\text{checked-staged-security-with-query-prefix-conceptual-alpha-bad-set-hit-at}$
 $i \text{ out} \longleftrightarrow$
 $\text{checked-staged-security-with-query-prefix-conceptual-context-algebraic-residual-hit-at}$
 $i \text{ out} \wedge$
 $(\text{case out of}$
 $None \Rightarrow False$
 $| \text{Some } (((((prefix, prefix-state), raw), raw-state),$
 $((data, attacker-state), result)), final-state) \Rightarrow$
 $\text{sqp-alphas } prefix \in$

composition-trace-bad-alpha-space
(query-prefix-trace-conceptual-table prefix prefix-state))

lemma *checked-staged-query-prefix-with-state-alphas-length:*

assumes *support:*

Some ((prefix, prefix-state), prefix-state) ∈
set-dist
(execute (checked-staged-query-prefix-with-state A i)
adversary-initial-state)

shows *length (sqp-alphas prefix) = length spec*

proof –

from *support* **have** *prefix-out:*

Some (prefix, prefix-state) ∈
set-dist
(execute (checked-staged-query-challenge-prefix-program A i)
adversary-initial-state)

unfolding *checked-staged-query-prefix-with-state-def*

by *(auto elim!: set-dist-bindE)*

from *prefix-out* **obtain** *fr s1 s2 trace-roots trace-bs s3 trace-final s4*

s5 as s6 dg s7 s8 s9 composition-roots composition-bs s10

composition-final s11 query-start query-chunks **where**

alpha-out:

Some (as, s6) ∈
set-dist (execute (staged-alpha-program (length spec)) s5)

and *prefix-eq:*

prefix =
(|sqp-trace-root = fr,
sqp-trace-fri-roots = trace-roots,
sqp-trace-fri-challenges = trace-bs,
sqp-trace-final = trace-final,
sqp-alphas = as,
sqp-degree = dg,
sqp-composition-fri-roots = composition-roots,
sqp-composition-fri-challenges = composition-bs,
sqp-composition-final = composition-final,
sqp-query-chunks = query-chunks)

using *prefix-out*

by *(elim checked-staged-query-challenge-prefix-program-outcomeE)*

have *length as = length spec*

using *staged-alpha-program-alignment[OF alpha-out]* **by** *simp*

then show *?thesis*

unfolding *prefix-eq* **by** *simp*

qed

lemma *checked-staged-security-with-query-prefix-data-state-prefix-support:*

assumes *support:*

Some (((((prefix, prefix-state), raw), raw-state),
((data, attacker-state), result)), final-state) ∈
set-dist

```

      (execute
        (checked-staged-security-experiment-with-query-prefix-data-state A i)
        adversary-initial-state)
    shows
      Some ((prefix, prefix-state), prefix-state) ∈
        set-dist
          (execute (checked-staged-query-prefix-with-state A i)
            adversary-initial-state)
  proof -
    from support have receive-support:
      Some (((prefix, prefix-state), raw), raw-state) ∈
        set-dist
          (execute (checked-staged-query-prefix-receive-with-state A i)
            adversary-initial-state)
    unfolding checked-staged-security-experiment-with-query-prefix-data-state-def
    by (auto elim!: set-dist-bindE split: prod.splits)
    then show ?thesis
      by (rule checked-staged-query-prefix-receive-with-state-prefix-support)
  qed

lemma checked-staged-security-with-query-prefix-conceptual-context-algebraic-residual-hit-at-cases-on-support:
  assumes false-statement:  $\neg$  exists-valid-trace
  and support:
    Some (((((prefix, prefix-state), raw), raw-state),
      ((data, attacker-state), result)), final-state) ∈
      set-dist
        (execute
          (checked-staged-security-experiment-with-query-prefix-data-state A i)
          adversary-initial-state)
  and hit:
    checked-staged-security-with-query-prefix-conceptual-context-algebraic-residual-hit-at
      i
      (Some (((((prefix, prefix-state), raw), raw-state),
        ((data, attacker-state), result)), final-state))
  shows
     $\neg$  trace-table-low-degree
      (query-prefix-trace-conceptual-table prefix prefix-state)  $\vee$ 
     $\neg$  composition-table-low-degree maxDegree
      (query-prefix-composition-conceptual-table prefix prefix-state)  $\vee$ 
    sqp-alphas prefix ∈
      composition-trace-bad-alpha-space
        (query-prefix-trace-conceptual-table prefix prefix-state)
  proof -
    let ?trace = query-prefix-trace-conceptual-table prefix prefix-state
    let ?composition =
      query-prefix-composition-conceptual-table prefix prefix-state
    from hit have residual:
       $\neg$  trace-table-low-degree ?trace  $\vee$ 
       $\neg$  composition-table-low-degree maxDegree ?composition  $\vee$ 

```

```

    all-queries-consistent ?trace ?composition (sqp-alphas prefix)
  unfolding
    checked-staged-security-with-query-prefix-conceptual-context-algebraic-residual-hit-at-def
  by simp
show ?thesis
proof (cases trace-table-low-degree ?trace)
  case False
  then show ?thesis by simp
next
  case trace-low: True
  show ?thesis
  proof (cases composition-table-low-degree maxDegree ?composition)
    case False
    then show ?thesis by simp
  next
    case composition-low: True
    from residual trace-low composition-low have all-queries:
      all-queries-consistent ?trace ?composition (sqp-alphas prefix)
    by simp
    have prefix-support:
      Some ((prefix, prefix-state), prefix-state) ∈
        set-dist
          (execute (checked-staged-query-prefix-with-state A i)
            adversary-initial-state)
    by (rule
      checked-staged-security-with-query-prefix-data-state-prefix-support
      [OF support])
    have alpha-len: length (sqp-alphas prefix) = length spec
    by (rule checked-staged-query-prefix-with-state-alphas-length
      [OF prefix-support])
    have
      sqp-alphas prefix ∈
        composition-trace-bad-alpha-space ?trace
    by (rule low-degree-all-queries-consistent-imp-composition-trace-bad-alpha-space
      [OF false-statement alpha-len trace-low composition-low
        all-queries])
    then show ?thesis by simp
  qed
qed
qed

lemma checked-staged-security-with-query-prefix-conceptual-context-algebraic-residual-hit-at-split-on-support:
  assumes false-statement:  $\neg$  exists-valid-trace
  and support:
    Some (((((prefix, prefix-state), raw), raw-state),
      ((data, attacker-state), result)), final-state) ∈
      set-dist
        (execute
          (checked-staged-security-experiment-with-query-prefix-data-state A i)

```

adversary-initial-state)
and *hit*:
checked-staged-security-with-query-prefix-conceptual-context-algebraic-residual-hit-at
i
(Some (((((prefix, prefix-state), raw), raw-state),
((data, attacker-state), result)), final-state))
shows
checked-staged-security-with-query-prefix-conceptual-trace-low-degree-bad-at
i
(Some (((((prefix, prefix-state), raw), raw-state),
((data, attacker-state), result)), final-state)) ∨
checked-staged-security-with-query-prefix-conceptual-composition-low-degree-bad-at
i
(Some (((((prefix, prefix-state), raw), raw-state),
((data, attacker-state), result)), final-state)) ∨
checked-staged-security-with-query-prefix-conceptual-alpha-bad-set-hit-at
i
(Some (((((prefix, prefix-state), raw), raw-state),
((data, attacker-state), result)), final-state))
using
checked-staged-security-with-query-prefix-conceptual-context-algebraic-residual-hit-at-cases-on-support
[OF false-statement support hit] hit
unfolding
checked-staged-security-with-query-prefix-conceptual-trace-low-degree-bad-at-def
checked-staged-security-with-query-prefix-conceptual-composition-low-degree-bad-at-def
checked-staged-security-with-query-prefix-conceptual-alpha-bad-set-hit-at-def
by *simp*

lemma *checked-staged-security-with-query-prefix-conceptual-context-algebraic-residual-hit-at-bound-from-split*:

assumes *false-statement*: $\neg \text{exists-valid-trace}$

and *trace-bound*:

wp-event

(checked-staged-security-experiment-with-query-prefix-data-state A i)

(checked-staged-security-with-query-prefix-conceptual-trace-low-degree-bad-at

i)

adversary-initial-state $\leq T$

and *composition-bound*:

wp-event

(checked-staged-security-experiment-with-query-prefix-data-state A i)

(checked-staged-security-with-query-prefix-conceptual-composition-low-degree-bad-at

i)

adversary-initial-state $\leq C$

and *alpha-bound*:

wp-event

(checked-staged-security-experiment-with-query-prefix-data-state A i)

(checked-staged-security-with-query-prefix-conceptual-alpha-bad-set-hit-at

i)

adversary-initial-state $\leq P$

shows

```

wp-event
  (checked-staged-security-experiment-with-query-prefix-data-state A i)
  (checked-staged-security-with-query-prefix-conceptual-context-algebraic-residual-hit-at
    i)
  adversary-initial-state  $\leq T + C + P$ 
proof –
  let ?M = checked-staged-security-experiment-with-query-prefix-data-state A i
  let ?Trace =
    checked-staged-security-with-query-prefix-conceptual-trace-low-degree-bad-at
      i
  let ?Composition =
    checked-staged-security-with-query-prefix-conceptual-composition-low-degree-bad-at
      i
  let ?Alpha =
    checked-staged-security-with-query-prefix-conceptual-alpha-bad-set-hit-at
      i
  have event-le:
    wp-event ?M
      (checked-staged-security-with-query-prefix-conceptual-context-algebraic-residual-hit-at
        i)
      adversary-initial-state  $\leq$ 
      wp-event ?M ( $\lambda out. ?Trace\ out \vee ?Composition\ out \vee ?Alpha\ out$ )
      adversary-initial-state
  proof (rule wp-event-mono-on-support)
  fix out
  assume support:
    out  $\in$  set-dist (execute ?M adversary-initial-state)
  and hit:
    checked-staged-security-with-query-prefix-conceptual-context-algebraic-residual-hit-at
      i out
  show ?Trace out  $\vee$  ?Composition out  $\vee$  ?Alpha out
  proof (cases out)
  case None
  then show ?thesis
    using hit
    unfolding
      checked-staged-security-with-query-prefix-conceptual-context-algebraic-residual-hit-at-def
    by simp
  next
  case (Some packed)
  then obtain prefix prefix-state raw raw-state data attacker-state result
    final-state where out-eq:
    out =
      Some (((((prefix, prefix-state), raw), raw-state),
        ((data, attacker-state), result)), final-state)
  by (cases packed) (auto split: prod.splits)
  have support':
    Some (((((prefix, prefix-state), raw), raw-state),
      ((data, attacker-state), result)), final-state)  $\in$ 

```

```

    set-dist (execute ?M adversary-initial-state)
    using support unfolding out-eq .
  have hit':
    checked-staged-security-with-query-prefix-conceptual-context-algebraic-residual-hit-at
      i
      (Some (((((prefix, prefix-state), raw), raw-state),
        ((data, attacker-state), result)), final-state))
    using hit unfolding out-eq .
  show ?thesis
    unfolding out-eq
    by (rule
      checked-staged-security-with-query-prefix-conceptual-context-algebraic-residual-hit-at-split-on-support
        [OF false-statement support' hit'])
  qed
  qed
  also have ... ≤
    wp-event ?M ?Trace adversary-initial-state +
    wp-event ?M ?Composition adversary-initial-state +
    wp-event ?M ?Alpha adversary-initial-state
  proof -
    have ... ≤
      wp-event ?M ?Trace
        adversary-initial-state +
      wp-event ?M (λout. ?Composition out ∨ ?Alpha out)
        adversary-initial-state
    by (rule wp-event-union-bound
      [where P = ?Trace
        and Q = λout. ?Composition out ∨ ?Alpha out])
    also have ... ≤
      wp-event ?M ?Trace adversary-initial-state +
      (wp-event ?M ?Composition adversary-initial-state +
        wp-event ?M ?Alpha adversary-initial-state)
    by (intro add-mono order-refl wp-event-union-bound)
    finally show ?thesis
      by (simp add: add.assoc)
  qed
  also have ... ≤ T + C + P
    by (intro add-mono trace-bound composition-bound alpha-bound)
  finally show ?thesis .
  qed

```

lemma *checked-staged-security-with-query-prefix-conceptual-context-residual-hit-at-split:*
assumes
checked-staged-security-with-query-prefix-conceptual-context-residual-hit-at
i out
shows
checked-staged-security-with-query-prefix-conceptual-context-prefix-collision-hit-at
i out ∨
checked-staged-security-with-query-prefix-conceptual-context-algebraic-residual-hit-at

```

    i out
using assms
unfolding
  checked-staged-security-with-query-prefix-conceptual-context-residual-hit-at-def
  checked-staged-security-with-query-prefix-conceptual-context-residual-at-def
  checked-staged-security-with-query-prefix-conceptual-context-prefix-collision-hit-at-def
  checked-staged-security-with-query-prefix-conceptual-context-algebraic-residual-hit-at-def
by (cases out) (auto split: prod.splits)

lemma checked-staged-security-with-query-prefix-conceptual-context-prefix-collision-hit-at-bound:
assumes wf: staged-budget-wellformed budgets
and controlled: staged-adversary-controlled budgets A
and i-bound: i < rounds
shows
  wp-event
    (checked-staged-security-experiment-with-query-prefix-data-state A i)
    (checked-staged-security-with-query-prefix-conceptual-context-prefix-collision-hit-at
      i)
    adversary-initial-state ≤
    hash-collision-budget-value 0
    (staged-query-search-queries budgets i + 1)
proof –
have i-le: i ≤ rounds
using i-bound by simp
have prefix-bound:
  wp-event (checked-staged-query-prefix-receive-with-state A i)
  checked-staged-query-prefix-hash-map-output-collision
  adversary-initial-state ≤
  hash-collision-budget-value 0
  (staged-query-search-queries budgets i + 1)
by (rule checked-staged-query-prefix-hash-map-output-collision-bound
  [OF wf controlled i-le])
show ?thesis
proof (rule
  checked-staged-security-experiment-with-query-prefix-data-state-bound-by-prefix
  [OF prefix-bound])
show
  checked-staged-security-with-query-prefix-conceptual-context-prefix-collision-hit-at
    i None ⇒
    checked-staged-query-prefix-hash-map-output-collision None
unfolding
  checked-staged-security-with-query-prefix-conceptual-context-prefix-collision-hit-at-def
  checked-staged-query-prefix-hash-map-output-collision-def
by simp
next
fix prefix-out t out
assume
  Some (prefix-out, t) ∈
  set-dist

```

```

      (execute (checked-staged-query-prefix-receive-with-state A i)
        adversary-initial-state)
    and cont:
    out ∈
      set-dist
      (execute
        (get ≫=
          (λprefix-state.
            checked-staged-after-query-prefix-receive-with-verifier A i
              prefix-out ≫=
                (λresult. return ((prefix-out, prefix-state), result))))
          t)
    and event:
    checked-staged-security-with-query-prefix-conceptual-context-prefix-collision-hit-at
      i out
  show
    checked-staged-query-prefix-hash-map-output-collision
      (Some (prefix-out, t))
  using cont event
  unfolding
    checked-staged-security-with-query-prefix-conceptual-context-prefix-collision-hit-at-def
    checked-staged-query-prefix-hash-map-output-collision-def
  by (auto elim!: set-dist-bindE split: option.splits prod.splits)
qed
qed

```

lemma *checked-staged-security-with-query-prefix-conceptual-default-context-residual-hit-at-imp-prefix-collision-on-sup*

assumes *default-context:*

```

  ∧prefix prefix-state.
  Some ((prefix, prefix-state), prefix-state) ∈
    set-dist
      (execute (checked-staged-query-prefix-with-state A i)
        adversary-initial-state) ⇒
  trace-table-low-degree
    (query-prefix-trace-conceptual-table-with-default prefix
      prefix-state trace-default) ∧
  composition-table-low-degree maxDegree
    (query-prefix-composition-conceptual-table-with-default prefix
      prefix-state composition-default) ∧
  ¬ all-queries-consistent
    (query-prefix-trace-conceptual-table-with-default prefix
      prefix-state trace-default)
    (query-prefix-composition-conceptual-table-with-default prefix
      prefix-state composition-default)
    (sqp-alphas prefix)

```

and *support:*

```

  out ∈
    set-dist
      (execute

```



```

      (checked-staged-security-experiment-with-query-prefix-data-state A i)
      adversary-initial-state)
and hit:
  checked-staged-security-with-query-prefix-conceptual-default-context-residual-hit-at
  trace-default composition-default i out
shows
  checked-staged-security-with-query-prefix-conceptual-context-prefix-collision-hit-at
  i out
proof (cases out)
case None
then show ?thesis
using hit
unfolding
  checked-staged-security-with-query-prefix-conceptual-default-context-residual-hit-at-def
  checked-staged-security-with-query-prefix-current-query-partial-opening-hit-at-def
  checked-staged-security-with-query-prefix-authenticated-opening-hit-def
by simp
next
case (Some packed)
then obtain prefix prefix-state raw raw-state data attacker-state result
  final-state where out-eq:
  out =
    Some (((((prefix, prefix-state), raw), raw-state),
      ((data, attacker-state), result)), final-state)
by (cases packed) (auto split: prod.splits)
have support':
  Some (((((prefix, prefix-state), raw), raw-state),
    ((data, attacker-state), result)), final-state) ∈
  set-dist
  (execute
    (checked-staged-security-experiment-with-query-prefix-data-state A i)
    adversary-initial-state)
using support unfolding out-eq .
have prefix-support:
  Some ((prefix, prefix-state), prefix-state) ∈
  set-dist
  (execute (checked-staged-query-prefix-with-state A i)
    adversary-initial-state)
by (rule
  checked-staged-security-with-query-prefix-data-state-prefix-support
  [OF support'])
have cctx-fact:
  trace-table-low-degree
  (query-prefix-trace-conceptual-table-with-default prefix prefix-state
    trace-default) ∧
  composition-table-low-degree maxDegree
  (query-prefix-composition-conceptual-table-with-default prefix
    prefix-state composition-default) ∧
  ¬ all-queries-consistent

```

```

    (query-prefix-trace-conceptual-table-with-default prefix prefix-state
      trace-default)
    (query-prefix-composition-conceptual-table-with-default prefix
      prefix-state composition-default)
    (sqp-alphas prefix)
  by (rule default-context[OF prefix-support])
from hit ctxt-fact have collision: hash-map-output-collision prefix-state
unfolding out-eq
  checked-staged-security-with-query-prefix-conceptual-default-context-residual-hit-at-def
  checked-staged-security-with-query-prefix-conceptual-default-context-residual-at-def
by auto
have current-hit:
  checked-staged-security-with-query-prefix-current-query-partial-opening-hit-at
  i out
using hit
unfolding
  checked-staged-security-with-query-prefix-conceptual-default-context-residual-hit-at-def
by simp
show ?thesis
unfolding
  checked-staged-security-with-query-prefix-conceptual-context-prefix-collision-hit-at-def
using current-hit collision out-eq by simp
qed

```

lemma *checked-staged-security-with-query-prefix-conceptual-default-context-residual-hit-at-bound-from-context:*

```

assumes wf: staged-budget-wellformed budgets
and controlled: staged-adversary-controlled budgets A
and i-bound: i < rounds
and default-context:
   $\bigwedge \text{prefix prefix-state.}$ 
  Some ((prefix, prefix-state), prefix-state)  $\in$ 
    set-dist
    (execute (checked-staged-query-prefix-with-state A i)
      adversary-initial-state)  $\implies$ 
    trace-table-low-degree
    (query-prefix-trace-conceptual-table-with-default prefix
      prefix-state trace-default)  $\wedge$ 
    composition-table-low-degree maxDegree
    (query-prefix-composition-conceptual-table-with-default prefix
      prefix-state composition-default)  $\wedge$ 
   $\neg$  all-queries-consistent
    (query-prefix-trace-conceptual-table-with-default prefix
      prefix-state trace-default)
    (query-prefix-composition-conceptual-table-with-default prefix
      prefix-state composition-default)
    (sqp-alphas prefix)

```

shows

```

  wp-event
  (checked-staged-security-experiment-with-query-prefix-data-state A i)

```

(checked-staged-security-with-query-prefix-conceptual-default-context-residual-hit-at
 trace-default composition-default i)
 adversary-initial-state $\leq$
 hash-collision-budget-value 0
 (staged-query-search-queries budgets $i + 1$)
proof –
 let ?M = checked-staged-security-experiment-with-query-prefix-data-state A i
have event-le:
 wp-event ?M
 (checked-staged-security-with-query-prefix-conceptual-default-context-residual-hit-at
 trace-default composition-default i)
 adversary-initial-state $\leq$
 wp-event ?M
 (checked-staged-security-with-query-prefix-conceptual-context-prefix-collision-hit-at
 i)
 adversary-initial-state
by (rule wp-event-mono-on-support)
 (rule
 checked-staged-security-with-query-prefix-conceptual-default-context-residual-hit-at-imp-prefix-collision-on-support
 [OF default-context])
also have ... $\leq$
 hash-collision-budget-value 0
 (staged-query-search-queries budgets $i + 1$)
by (rule
 checked-staged-security-with-query-prefix-conceptual-context-prefix-collision-hit-at-bound
 [OF wf controlled i-bound])
finally show ?thesis .
qed

lemma checked-staged-security-with-query-prefix-conceptual-context-residual-hit-at-bound-from-split:
assumes wf: staged-budget-wellformed budgets
and controlled: staged-adversary-controlled budgets A
and i-bound: $i < \text{rounds}$
and algebraic-bound:
 wp-event
 (checked-staged-security-experiment-with-query-prefix-data-state A i)
 (checked-staged-security-with-query-prefix-conceptual-context-algebraic-residual-hit-at
 i)
 adversary-initial-state $\leq R$
shows
 wp-event
 (checked-staged-security-experiment-with-query-prefix-data-state A i)
 (checked-staged-security-with-query-prefix-conceptual-context-residual-hit-at
 i)
 adversary-initial-state $\leq$
 hash-collision-budget-value 0
 (staged-query-search-queries budgets $i + 1$) + R
proof –
 let ?M = checked-staged-security-experiment-with-query-prefix-data-state A i

```

let ?Collision =
  checked-staged-security-with-query-prefix-conceptual-context-prefix-collision-hit-at
  i
let ?Algebraic =
  checked-staged-security-with-query-prefix-conceptual-context-algebraic-residual-hit-at
  i
have event-le:
  wp-event ?M
    (checked-staged-security-with-query-prefix-conceptual-context-residual-hit-at i)
    adversary-initial-state ≤
    wp-event ?M (λout. ?Collision out ∨ ?Algebraic out)
    adversary-initial-state
by (rule wp-event-mono)
    (rule
      checked-staged-security-with-query-prefix-conceptual-context-residual-hit-at-split)
have union-bound:
  wp-event ?M (λout. ?Collision out ∨ ?Algebraic out)
    adversary-initial-state ≤
    wp-event ?M ?Collision adversary-initial-state +
    wp-event ?M ?Algebraic adversary-initial-state
by (rule wp-event-union-bound)
have collision-bound:
  wp-event ?M ?Collision adversary-initial-state ≤
    hash-collision-budget-value 0
    (staged-query-search-queries budgets i + 1)
by (rule
    checked-staged-security-with-query-prefix-conceptual-context-prefix-collision-hit-at-bound
    [OF wf controlled i-bound])
have sum-bound:
  wp-event ?M ?Collision adversary-initial-state +
  wp-event ?M ?Algebraic adversary-initial-state ≤
    hash-collision-budget-value 0
    (staged-query-search-queries budgets i + 1) + R
by (intro add-mono collision-bound algebraic-bound)
show ?thesis
by (rule order-trans[OF event-le order-trans[OF union-bound sum-bound]])
qed

```

lemma checked-staged-security-with-query-prefix-conceptual-context-residual-hit-at-bound-from-current:

assumes current-bound:

```

  wp-event
    (checked-staged-security-experiment-with-query-prefix-data-state A i)
    (checked-staged-security-with-query-prefix-current-query-partial-opening-hit-at
      i)
  adversary-initial-state ≤ C

```

shows

```

  wp-event
    (checked-staged-security-experiment-with-query-prefix-data-state A i)
    (checked-staged-security-with-query-prefix-conceptual-context-residual-hit-at
      i)

```

i)
 $\text{adversary-initial-state} \leq C$
by (rule *order-trans*[*OF* - *current-bound*])
(rule *wp-event-mono*,
rule *checked-staged-security-with-query-prefix-conceptual-context-residual-hit-at-imp-current*)

lemma *checked-staged-security-with-query-prefix-current-query-partial-opening-hit-at-imp-conceptual-or-residual-or-s*

assumes *wf*: *staged-budget-wellformed* budgets

and *controlled*: *staged-adversary-controlled* budgets *A*

and *i-bound*: $i < \text{rounds}$

and *support*:

out $\in$

set-dist

(*execute*

(*checked-staged-security-experiment-with-query-prefix-data-state* *A* *i*)

adversary-initial-state)

and *hit*:

checked-staged-security-with-query-prefix-current-query-partial-opening-hit-at

i out

shows

checked-staged-security-with-query-prefix-conceptual-target-hit out $\vee$

checked-staged-security-with-query-prefix-conceptual-context-residual-hit-at

i out $\vee$

checked-staged-security-with-query-prefix-current-trace-path-output-hit-at

i out $\vee$

checked-staged-security-with-query-prefix-current-composition-path-output-hit-at

i out

proof —

have *split*:

checked-staged-security-with-query-prefix-current-prefix-authenticated-hit-at

i out $\vee$

checked-staged-security-with-query-prefix-current-trace-path-output-hit-at

i out $\vee$

checked-staged-security-with-query-prefix-current-composition-path-output-hit-at

i out

by (rule

checked-staged-security-with-query-prefix-current-query-partial-opening-hit-at-imp-prefix-or-structured-path-on-su

[*OF wf controlled i-bound support hit*])

then show *?thesis*

proof

assume *prefix*:

checked-staged-security-with-query-prefix-current-prefix-authenticated-hit-at

i out

have

checked-staged-security-with-query-prefix-conceptual-target-hit out $\vee$

checked-staged-security-with-query-prefix-conceptual-context-residual-at

out

by (rule

checked-staged-security-with-query-prefix-current-prefix-authenticated-hit-imp-conceptual-or-residual-on-support

```

    [OF wf controlled i-bound support prefix])
then show ?thesis
  unfolding
    checked-staged-security-with-query-prefix-conceptual-context-residual-hit-at-def
  using hit by blast
next
  assume
    checked-staged-security-with-query-prefix-current-trace-path-output-hit-at
    i out ∨
    checked-staged-security-with-query-prefix-current-composition-path-output-hit-at
    i out
  then show ?thesis by blast
qed
qed

lemma checked-staged-security-with-query-prefix-current-query-partial-opening-hit-at-imp-conceptual-default-or-residual-hit-at
assumes wf: staged-budget-wellformed budgets
and controlled: staged-adversary-controlled budgets A
and i-bound: i < rounds
and support:
  out ∈
  set-dist
  (execute
    (checked-staged-security-experiment-with-query-prefix-data-state A i)
    adversary-initial-state)
and hit:
  checked-staged-security-with-query-prefix-current-query-partial-opening-hit-at
  i out
shows
  checked-staged-security-with-query-prefix-conceptual-default-target-hit
  trace-default composition-default out ∨
  checked-staged-security-with-query-prefix-conceptual-default-context-residual-hit-at
  trace-default composition-default i out ∨
  checked-staged-security-with-query-prefix-current-trace-path-output-hit-at
  i out ∨
  checked-staged-security-with-query-prefix-current-composition-path-output-hit-at
  i out
proof —
  have split:
    checked-staged-security-with-query-prefix-current-prefix-authenticated-hit-at
    i out ∨
    checked-staged-security-with-query-prefix-current-trace-path-output-hit-at
    i out ∨
    checked-staged-security-with-query-prefix-current-composition-path-output-hit-at
    i out
  by (rule
    checked-staged-security-with-query-prefix-current-query-partial-opening-hit-at-imp-prefix-or-structured-path-on-success
    [OF wf controlled i-bound support hit])
then show ?thesis

```

```

proof
  assume prefix:
    checked-staged-security-with-query-prefix-current-prefix-authenticated-hit-at
      i out
  have
    checked-staged-security-with-query-prefix-conceptual-default-target-hit
      trace-default composition-default out  $\vee$ 
    checked-staged-security-with-query-prefix-conceptual-default-context-residual-at
      trace-default composition-default out
  by (rule
    checked-staged-security-with-query-prefix-current-prefix-authenticated-hit-imp-conceptual-default-or-residual-on-
      [OF wf controlled i-bound support prefix])
  then show ?thesis
    unfolding
      checked-staged-security-with-query-prefix-conceptual-default-context-residual-hit-at-def
    using hit by blast
  next
    assume
      checked-staged-security-with-query-prefix-current-trace-path-output-hit-at
        i out  $\vee$ 
      checked-staged-security-with-query-prefix-current-composition-path-output-hit-at
        i out
    then show ?thesis by blast
  qed
qed

lemma checked-staged-security-with-query-prefix-current-query-partial-opening-hit-at-bound-from-conceptual-residual
  assumes wf: staged-budget-wellformed budgets
  and controlled: staged-adversary-controlled budgets A
  and i-bound: i < rounds
  and residual-bound:
    wp-event
      (checked-staged-security-experiment-with-query-prefix-data-state A i)
      (checked-staged-security-with-query-prefix-conceptual-context-residual-hit-at
        i)
      adversary-initial-state  $\leq R$ 
  and trace-path-bound:
    wp-event
      (checked-staged-security-experiment-with-query-prefix-data-state A i)
      (checked-staged-security-with-query-prefix-current-trace-path-output-hit-at i)
      adversary-initial-state  $\leq T$ 
  and composition-path-bound:
    wp-event
      (checked-staged-security-experiment-with-query-prefix-data-state A i)
      (checked-staged-security-with-query-prefix-current-composition-path-output-hit-at i)
      adversary-initial-state  $\leq C$ 
  shows
    wp-event
      (checked-staged-security-experiment-with-query-prefix-data-state A i)

```

(checked-staged-security-with-query-prefix-current-query-partial-opening-hit-at i)
 adversary-initial-state $\leq$
 staged-phase-relation-error size
 (staged-query-search-queries budgets $i + 1$) +
 query-error-bound + $R + T + C$

proof –

let $?M =$ checked-staged-security-experiment-with-query-prefix-data-state $A\ i$
let $?Conceptual =$
 checked-staged-security-with-query-prefix-conceptual-target-hit
let $?Residual =$
 checked-staged-security-with-query-prefix-conceptual-context-residual-hit-at
 i
let $?Trace =$
 checked-staged-security-with-query-prefix-current-trace-path-output-hit-at i
let $?Composition =$
 checked-staged-security-with-query-prefix-current-composition-path-output-hit-at i

have event-le:
 wp-event $?M$
 (checked-staged-security-with-query-prefix-current-query-partial-opening-hit-at i)
 adversary-initial-state $\leq$
 wp-event $?M$
 ($\lambda out. ?Conceptual\ out \vee ?Residual\ out \vee ?Trace\ out \vee$
 $?Composition\ out$)
 adversary-initial-state
by (rule wp-event-mono-on-support)
 (rule
 checked-staged-security-with-query-prefix-current-query-partial-opening-hit-at-imp-conceptual-or-residual-or-struc
 [OF wf controlled i -bound])

have conceptual-bound:
 wp-event $?M\ ?Conceptual$ adversary-initial-state $\leq$
 staged-phase-relation-error size
 (staged-query-search-queries budgets $i + 1$) + query-error-bound
by (rule checked-staged-security-with-query-prefix-conceptual-target-hit-bound
 [OF wf controlled i -bound])

have union-bound:
 wp-event $?M$
 ($\lambda out. ?Conceptual\ out \vee ?Residual\ out \vee ?Trace\ out \vee$
 $?Composition\ out$)
 adversary-initial-state $\leq$
 wp-event $?M\ ?Conceptual$ adversary-initial-state +
 wp-event $?M\ ?Residual$ adversary-initial-state +
 wp-event $?M\ ?Trace$ adversary-initial-state +
 wp-event $?M\ ?Composition$ adversary-initial-state
by (rule wp-event-union-bound4)

have sum-bound:
 wp-event $?M\ ?Conceptual$ adversary-initial-state +
 wp-event $?M\ ?Residual$ adversary-initial-state +
 wp-event $?M\ ?Trace$ adversary-initial-state +
 wp-event $?M\ ?Composition$ adversary-initial-state $\leq$

(staged-phase-relation-error size
 (staged-query-search-queries budgets $i + 1$) + query-error-bound) +
 $R + T + C$
by (intro add-mono conceptual-bound residual-bound trace-path-bound
 composition-path-bound)
have event-bound:
 wp-event ?M
 (λ out. ?Conceptual out $\vee$?Residual out $\vee$?Trace out $\vee$
 ?Composition out)
 adversary-initial-state $\leq$
 (staged-phase-relation-error size
 (staged-query-search-queries budgets $i + 1$) + query-error-bound) +
 $R + T + C$
by (rule order-trans[OF union-bound sum-bound])
show ?thesis
by (rule order-trans[OF event-le])
 (use event-bound **in** $\langle$ simp add: algebra-simps $\rangle$)
qed

lemma checked-staged-security-with-query-prefix-current-query-partial-opening-hit-at-bound-from-conceptual-default-
assumes wf: staged-budget-wellformed budgets
and controlled: staged-adversary-controlled budgets A
and i-bound: $i < \text{rounds}$
and residual-bound:
 wp-event
 (checked-staged-security-experiment-with-query-prefix-data-state A i)
 (checked-staged-security-with-query-prefix-conceptual-default-context-residual-hit-at
 trace-default composition-default i)
 adversary-initial-state $\leq R$
and trace-path-bound:
 wp-event
 (checked-staged-security-experiment-with-query-prefix-data-state A i)
 (checked-staged-security-with-query-prefix-current-trace-path-output-hit-at i)
 adversary-initial-state $\leq T$
and composition-path-bound:
 wp-event
 (checked-staged-security-experiment-with-query-prefix-data-state A i)
 (checked-staged-security-with-query-prefix-current-composition-path-output-hit-at i)
 adversary-initial-state $\leq C$
shows
 wp-event
 (checked-staged-security-experiment-with-query-prefix-data-state A i)
 (checked-staged-security-with-query-prefix-current-query-partial-opening-hit-at i)
 adversary-initial-state $\leq$
 staged-phase-relation-error size
 (staged-query-search-queries budgets $i + 1$) +
 query-error-bound + $R + T + C$
proof –
let ?M = checked-staged-security-experiment-with-query-prefix-data-state A i

```

let ?Conceptual =
  checked-staged-security-with-query-prefix-conceptual-default-target-hit
  trace-default composition-default
let ?Residual =
  checked-staged-security-with-query-prefix-conceptual-default-context-residual-hit-at
  trace-default composition-default i
let ?Trace =
  checked-staged-security-with-query-prefix-current-trace-path-output-hit-at i
let ?Composition =
  checked-staged-security-with-query-prefix-current-composition-path-output-hit-at i
have event-le:
  wp-event ?M
  (checked-staged-security-with-query-prefix-current-query-partial-opening-hit-at i)
  adversary-initial-state  $\leq$ 
  wp-event ?M
  ( $\lambda out. ?Conceptual\ out \vee ?Residual\ out \vee ?Trace\ out \vee$ 
   ?Composition out)
  adversary-initial-state
by (rule wp-event-mono-on-support)
  (rule
   checked-staged-security-with-query-prefix-current-query-partial-opening-hit-at-imp-conceptual-default-or-residual-
   [OF wf controlled i-bound])
have conceptual-bound:
  wp-event ?M ?Conceptual adversary-initial-state  $\leq$ 
  staged-phase-relation-error size
  (staged-query-search-queries budgets i + 1) + query-error-bound
by (rule
  checked-staged-security-with-query-prefix-conceptual-default-target-hit-bound
  [OF wf controlled i-bound])
have union-bound:
  wp-event ?M
  ( $\lambda out. ?Conceptual\ out \vee ?Residual\ out \vee ?Trace\ out \vee$ 
   ?Composition out)
  adversary-initial-state  $\leq$ 
  wp-event ?M ?Conceptual adversary-initial-state +
  wp-event ?M ?Residual adversary-initial-state +
  wp-event ?M ?Trace adversary-initial-state +
  wp-event ?M ?Composition adversary-initial-state
by (rule wp-event-union-bound4)
have sum-bound:
  wp-event ?M ?Conceptual adversary-initial-state +
  wp-event ?M ?Residual adversary-initial-state +
  wp-event ?M ?Trace adversary-initial-state +
  wp-event ?M ?Composition adversary-initial-state  $\leq$ 
  (staged-phase-relation-error size
   (staged-query-search-queries budgets i + 1) + query-error-bound) +
  R + T + C
by (intro add-mono conceptual-bound residual-bound trace-path-bound
  composition-path-bound)

```

have *event-bound*:
wp-event ?*M*
 $(\lambda out. ?Conceptual\ out \vee ?Residual\ out \vee ?Trace\ out \vee$
 $?Composition\ out)$
adversary-initial-state $\leq$
 $(staged-phase-relation-error\ size$
 $(staged-query-search-queries\ budgets\ i + 1) + query-error-bound) +$
 $R + T + C$
by (rule *order-trans*[*OF union-bound sum-bound*])
show ?*thesis*
by (rule *order-trans*[*OF event-le*])
 $(use\ event-bound\ in\ \langle simp\ add:\ algebra-simps \rangle)$
qed

lemma *checked-staged-security-with-query-prefix-current-query-partial-opening-hit-at-bound-from-conceptual-default-*

assumes *wf*: *staged-budget-wellformed* *budgets*

and *controlled*: *staged-adversary-controlled* *budgets* *A*

and *i-bound*: $i < rounds$

and *default-context*:

$\bigwedge prefix\ prefix-state.$

Some $((prefix, prefix-state), prefix-state) \in$
set-dist

$(execute\ (checked-staged-query-prefix-with-state\ A\ i)$
 $adversary-initial-state) \implies$

trace-table-low-degree

$(query-prefix-trace-conceptual-table-with-default\ prefix$
 $prefix-state\ trace-default) \wedge$

composition-table-low-degree *maxDegree*

$(query-prefix-composition-conceptual-table-with-default\ prefix$
 $prefix-state\ composition-default) \wedge$

$\neg all-queries-consistent$

$(query-prefix-trace-conceptual-table-with-default\ prefix$
 $prefix-state\ trace-default)$

$(query-prefix-composition-conceptual-table-with-default\ prefix$
 $prefix-state\ composition-default)$

$(sqp-alphas\ prefix)$

and *trace-path-bound*:

wp-event

$(checked-staged-security-experiment-with-query-prefix-data-state\ A\ i)$

$(checked-staged-security-with-query-prefix-current-trace-path-output-hit-at\ i)$

adversary-initial-state $\leq T$

and *composition-path-bound*:

wp-event

$(checked-staged-security-experiment-with-query-prefix-data-state\ A\ i)$

$(checked-staged-security-with-query-prefix-current-composition-path-output-hit-at\ i)$

adversary-initial-state $\leq C$

shows

wp-event

$(checked-staged-security-experiment-with-query-prefix-data-state\ A\ i)$

(checked-staged-security-with-query-prefix-current-query-partial-opening-hit-at i)
adversary-initial-state $\leq$
staged-phase-relation-error size
(staged-query-search-queries budgets i + 1) +
query-error-bound +
hash-collision-budget-value 0
(staged-query-search-queries budgets i + 1) + T + C

proof –

have *residual-bound*:

wp-event
(checked-staged-security-experiment-with-query-prefix-data-state A i)
(checked-staged-security-with-query-prefix-conceptual-default-context-residual-hit-at
trace-default composition-default i)
adversary-initial-state $\leq$
hash-collision-budget-value 0
(staged-query-search-queries budgets i + 1)

by (rule
checked-staged-security-with-query-prefix-conceptual-default-context-residual-hit-at-bound-from-context
[OF wf controlled i-bound default-context])

have *bound*:

wp-event
(checked-staged-security-experiment-with-query-prefix-data-state A i)
(checked-staged-security-with-query-prefix-current-query-partial-opening-hit-at i)
adversary-initial-state $\leq$
staged-phase-relation-error size
(staged-query-search-queries budgets i + 1) +
query-error-bound +
hash-collision-budget-value 0
(staged-query-search-queries budgets i + 1) + T + C

by (rule
checked-staged-security-with-query-prefix-current-query-partial-opening-hit-at-bound-from-conceptual-default-residual
[OF wf controlled i-bound residual-bound trace-path-bound
composition-path-bound])

show *?thesis*

using *bound* **by** *simp*

qed

lemma *checked-staged-security-with-query-prefix-authenticated-candidate-hit-at-bound-from-conceptual-default-residual*

assumes *wf: staged-budget-wellformed budgets*

and *controlled: staged-adversary-controlled budgets A*

and *i-bound: i < rounds*

and *residual-bound*:

wp-event
(checked-staged-security-experiment-with-query-prefix-data-state A i)
(checked-staged-security-with-query-prefix-conceptual-default-context-residual-hit-at
trace-table composition-table i)
adversary-initial-state $\leq R$

and *trace-path-bound*:

wp-event

```

    (checked-staged-security-experiment-with-query-prefix-data-state A i)
    (checked-staged-security-with-query-prefix-current-trace-path-output-hit-at i)
    adversary-initial-state  $\leq T$ 
and composition-path-bound:
    wp-event
    (checked-staged-security-experiment-with-query-prefix-data-state A i)
    (checked-staged-security-with-query-prefix-current-composition-path-output-hit-at i)
    adversary-initial-state  $\leq C$ 
shows
    wp-event
    (checked-staged-security-experiment-with-query-prefix-data-state A i)
    (checked-staged-security-with-query-prefix-authenticated-candidate-hit-at
      trace-table composition-table i)
    adversary-initial-state  $\leq$ 
    staged-phase-relation-error size
    (staged-query-search-queries budgets  $i + 1$ ) +
    query-error-bound +  $R + T + C$ 
proof –
    let ?M = checked-staged-security-experiment-with-query-prefix-data-state A i
    let ?Conceptual =
      checked-staged-security-with-query-prefix-conceptual-default-target-hit
      trace-table composition-table
    let ?Residual =
      checked-staged-security-with-query-prefix-conceptual-default-context-residual-hit-at
      trace-table composition-table i
    let ?Trace =
      checked-staged-security-with-query-prefix-current-trace-path-output-hit-at i
    let ?Composition =
      checked-staged-security-with-query-prefix-current-composition-path-output-hit-at i
    have event-le:
      wp-event ?M
      (checked-staged-security-with-query-prefix-authenticated-candidate-hit-at
        trace-table composition-table i)
      adversary-initial-state  $\leq$ 
      wp-event ?M
      ( $\lambda$ out. ?Conceptual out  $\vee$  ?Residual out  $\vee$  ?Trace out  $\vee$ 
        ?Composition out)
      adversary-initial-state
    by (rule wp-event-mono-on-support)
      (rule
        checked-staged-security-with-query-prefix-authenticated-candidate-hit-at-imp-conceptual-default-or-residual-or-str
        [OF wf controlled i-bound])
    have conceptual-bound:
      wp-event ?M ?Conceptual adversary-initial-state  $\leq$ 
      staged-phase-relation-error size
      (staged-query-search-queries budgets  $i + 1$ ) + query-error-bound
    by (rule
      checked-staged-security-with-query-prefix-conceptual-default-target-hit-bound
      [OF wf controlled i-bound])

```

have *union-bound*:
 $wp\text{-}event\ ?M$
 $(\lambda out. ?Conceptual\ out \vee ?Residual\ out \vee ?Trace\ out \vee ?Composition\ out)$
 $adversary\text{-}initial\text{-}state \leq$
 $wp\text{-}event\ ?M\ ?Conceptual\ adversary\text{-}initial\text{-}state +$
 $wp\text{-}event\ ?M\ ?Residual\ adversary\text{-}initial\text{-}state +$
 $wp\text{-}event\ ?M\ ?Trace\ adversary\text{-}initial\text{-}state +$
 $wp\text{-}event\ ?M\ ?Composition\ adversary\text{-}initial\text{-}state$
by (*rule wp-event-union-bound4*)
have *sum-bound*:
 $wp\text{-}event\ ?M\ ?Conceptual\ adversary\text{-}initial\text{-}state +$
 $wp\text{-}event\ ?M\ ?Residual\ adversary\text{-}initial\text{-}state +$
 $wp\text{-}event\ ?M\ ?Trace\ adversary\text{-}initial\text{-}state +$
 $wp\text{-}event\ ?M\ ?Composition\ adversary\text{-}initial\text{-}state \leq$
 $(staged\text{-}phase\text{-}relation\text{-}error\ size$
 $(staged\text{-}query\text{-}search\text{-}queries\ budgets\ i + 1) + query\text{-}error\text{-}bound) +$
 $R + T + C$
by (*intro add-mono conceptual-bound residual-bound trace-path-bound composition-path-bound*)
have *event-bound*:
 $wp\text{-}event\ ?M$
 $(\lambda out. ?Conceptual\ out \vee ?Residual\ out \vee ?Trace\ out \vee ?Composition\ out)$
 $adversary\text{-}initial\text{-}state \leq$
 $(staged\text{-}phase\text{-}relation\text{-}error\ size$
 $(staged\text{-}query\text{-}search\text{-}queries\ budgets\ i + 1) + query\text{-}error\text{-}bound) +$
 $R + T + C$
by (*rule order-trans[OF union-bound sum-bound]*)
show *?thesis*
by (*rule order-trans[OF event-le]*)
 $(use\ event\text{-}bound\ in\ \langle simp\ add: algebra\text{-}simps \rangle)$
qed

lemma *checked-staged-security-with-query-prefix-authenticated-candidate-hit-at-bound-from-conceptual-default-context*
assumes *wf: staged-budget-wellformed budgets*
and *controlled: staged-adversary-controlled budgets A*
and *i-bound: i < rounds*
and *default-context:*
 $\bigwedge prefix\ prefix\text{-}state.$
 $Some\ ((prefix, prefix\text{-}state), prefix\text{-}state) \in$
 $set\text{-}dist$
 $(execute\ (checked\text{-}staged\text{-}query\text{-}prefix\text{-}with\text{-}state\ A\ i)$
 $adversary\text{-}initial\text{-}state) \implies$
 $trace\text{-}table\text{-}low\text{-}degree$
 $(query\text{-}prefix\text{-}trace\text{-}conceptual\text{-}table\text{-}with\text{-}default\ prefix$
 $prefix\text{-}state\ trace\text{-}table) \wedge$
 $composition\text{-}table\text{-}low\text{-}degree\ maxDegree$
 $(query\text{-}prefix\text{-}composition\text{-}conceptual\text{-}table\text{-}with\text{-}default\ prefix$

$\neg$ $\text{prefix-state composition-table}) \wedge$
 $\neg$ $\text{all-queries-consistent}$
 $(\text{query-prefix-trace-conceptual-table-with-default prefix}$
 $\text{prefix-state trace-table})$
 $(\text{query-prefix-composition-conceptual-table-with-default prefix}$
 $\text{prefix-state composition-table})$
 $(\text{sqp-alphas prefix})$
and trace-path-bound:
 wp-event
 $(\text{checked-staged-security-experiment-with-query-prefix-data-state } A \ i)$
 $(\text{checked-staged-security-with-query-prefix-current-trace-path-output-hit-at } i)$
 $\text{adversary-initial-state} \leq T$
and $\text{composition-path-bound:}$
 wp-event
 $(\text{checked-staged-security-experiment-with-query-prefix-data-state } A \ i)$
 $(\text{checked-staged-security-with-query-prefix-current-composition-path-output-hit-at } i)$
 $\text{adversary-initial-state} \leq C$
shows
 wp-event
 $(\text{checked-staged-security-experiment-with-query-prefix-data-state } A \ i)$
 $(\text{checked-staged-security-with-query-prefix-authenticated-candidate-hit-at}$
 $\text{trace-table composition-table } i)$
 $\text{adversary-initial-state} \leq$
 $\text{staged-phase-relation-error size}$
 $(\text{staged-query-search-queries budgets } i + 1) +$
 $\text{query-error-bound} +$
 $\text{hash-collision-budget-value } 0$
 $(\text{staged-query-search-queries budgets } i + 1) + T + C$
proof –
have residual-bound:
 wp-event
 $(\text{checked-staged-security-experiment-with-query-prefix-data-state } A \ i)$
 $(\text{checked-staged-security-with-query-prefix-conceptual-default-context-residual-hit-at}$
 $\text{trace-table composition-table } i)$
 $\text{adversary-initial-state} \leq$
 $\text{hash-collision-budget-value } 0$
 $(\text{staged-query-search-queries budgets } i + 1)$
by (rule
 $\text{checked-staged-security-with-query-prefix-conceptual-default-context-residual-hit-at-bound-from-context}$
 $[\text{OF wf controlled } i\text{-bound default-context}]$)
have bound:
 wp-event
 $(\text{checked-staged-security-experiment-with-query-prefix-data-state } A \ i)$
 $(\text{checked-staged-security-with-query-prefix-authenticated-candidate-hit-at}$
 $\text{trace-table composition-table } i)$
 $\text{adversary-initial-state} \leq$
 $\text{staged-phase-relation-error size}$
 $(\text{staged-query-search-queries budgets } i + 1) +$
 $\text{query-error-bound} +$

hash-collision-budget-value 0
(staged-query-search-queries budgets $i + 1$) + $T + C$
by (rule
checked-staged-security-with-query-prefix-authenticated-candidate-hit-at-bound-from-conceptual-default-residual-a
[OF wf controlled i-bound residual-bound trace-path-bound
composition-path-bound])
show ?thesis
using bound by simp
qed

lemma *checked-staged-security-with-query-prefix-current-query-partial-opening-hit-at-bound-from-conceptual-algebraic*

assumes *wf: staged-budget-wellformed budgets*

and *controlled: staged-adversary-controlled budgets A*

and *i-bound: $i < \text{rounds}$*

and *algebraic-residual-bound:*

wp-event

(checked-staged-security-experiment-with-query-prefix-data-state A i)

(checked-staged-security-with-query-prefix-conceptual-context-algebraic-residual-hit-at
i)

adversary-initial-state $\leq R$

and *trace-path-bound:*

wp-event

(checked-staged-security-experiment-with-query-prefix-data-state A i)

(checked-staged-security-with-query-prefix-current-trace-path-output-hit-at i)

adversary-initial-state $\leq T$

and *composition-path-bound:*

wp-event

(checked-staged-security-experiment-with-query-prefix-data-state A i)

(checked-staged-security-with-query-prefix-current-composition-path-output-hit-at i)

adversary-initial-state $\leq C$

shows

wp-event

(checked-staged-security-experiment-with-query-prefix-data-state A i)

(checked-staged-security-with-query-prefix-current-query-partial-opening-hit-at i)

adversary-initial-state $\leq$

staged-phase-relation-error size

(staged-query-search-queries budgets $i + 1$) +

query-error-bound +

hash-collision-budget-value 0

(staged-query-search-queries budgets $i + 1$) +

$R + T + C$

proof –

have *residual-bound:*

wp-event

(checked-staged-security-experiment-with-query-prefix-data-state A i)

(checked-staged-security-with-query-prefix-conceptual-context-residual-hit-at
i)

adversary-initial-state $\leq$

hash-collision-budget-value 0

(staged-query-search-queries budgets $i + 1$) + R
by (rule
 checked-staged-security-with-query-prefix-conceptual-context-residual-hit-at-bound-from-split
 [OF wf controlled i-bound algebraic-residual-bound])
have bound:
 wp-event
 (checked-staged-security-experiment-with-query-prefix-data-state A i)
 (checked-staged-security-with-query-prefix-current-query-partial-opening-hit-at i)
 adversary-initial-state $\leq$
 staged-phase-relation-error size
 (staged-query-search-queries budgets $i + 1$) +
 query-error-bound +
 (hash-collision-budget-value 0
 (staged-query-search-queries budgets $i + 1$) + R) + T + C
by (rule
 checked-staged-security-with-query-prefix-current-query-partial-opening-hit-at-bound-from-conceptual-residual-and
 [OF wf controlled i-bound residual-bound trace-path-bound
 composition-path-bound])
show ?thesis
using bound **by** (simp add: algebra-simps)
qed

lemma checked-staged-security-with-query-prefix-current-query-partial-opening-hit-at-bound-from-conceptual-residual-and
assumes false-statement: $\neg$ exists-valid-trace
and wf: staged-budget-wellformed budgets
and controlled: staged-adversary-controlled budgets A
and i-bound: $i < \text{rounds}$
and trace-low-degree-bound:
 wp-event
 (checked-staged-security-experiment-with-query-prefix-data-state A i)
 (checked-staged-security-with-query-prefix-conceptual-trace-low-degree-bad-at
 i)
 adversary-initial-state $\leq LT$
and composition-low-degree-bound:
 wp-event
 (checked-staged-security-experiment-with-query-prefix-data-state A i)
 (checked-staged-security-with-query-prefix-conceptual-composition-low-degree-bad-at
 i)
 adversary-initial-state $\leq LC$
and alpha-bound:
 wp-event
 (checked-staged-security-experiment-with-query-prefix-data-state A i)
 (checked-staged-security-with-query-prefix-conceptual-alpha-bad-set-hit-at
 i)
 adversary-initial-state $\leq A'$
and trace-path-bound:
 wp-event
 (checked-staged-security-experiment-with-query-prefix-data-state A i)
 (checked-staged-security-with-query-prefix-current-trace-path-output-hit-at i)

$\text{adversary-initial-state} \leq T$
and *composition-path-bound*:
 wp-event
 $(\text{checked-staged-security-experiment-with-query-prefix-data-state } A \ i)$
 $(\text{checked-staged-security-with-query-prefix-current-composition-path-output-hit-at } i)$
 $\text{adversary-initial-state} \leq C$
shows
 wp-event
 $(\text{checked-staged-security-experiment-with-query-prefix-data-state } A \ i)$
 $(\text{checked-staged-security-with-query-prefix-current-query-partial-opening-hit-at } i)$
 $\text{adversary-initial-state} \leq$
 $\text{staged-phase-relation-error size}$
 $(\text{staged-query-search-queries budgets } i + 1) +$
 $\text{query-error-bound} +$
 $\text{hash-collision-budget-value } 0$
 $(\text{staged-query-search-queries budgets } i + 1) +$
 $(LT + LC + A') + T + C$
proof –
have *residual-bound*:
 wp-event
 $(\text{checked-staged-security-experiment-with-query-prefix-data-state } A \ i)$
 $(\text{checked-staged-security-with-query-prefix-conceptual-context-algebraic-residual-hit-at } i)$
 $\text{adversary-initial-state} \leq LT + LC + A'$
by (rule
 $\text{checked-staged-security-with-query-prefix-conceptual-context-algebraic-residual-hit-at-bound-from-split}$
 $[OF \text{ false-statement trace-low-degree-bound}$
 $\text{composition-low-degree-bound alpha-bound}]$)
show *?thesis*
by (rule
 $\text{checked-staged-security-with-query-prefix-current-query-partial-opening-hit-at-bound-from-conceptual-algebraic-re}$
 $[OF \text{ wf controlled i-bound residual-bound trace-path-bound}$
 $\text{composition-path-bound}]$)
qed
lemma *checked-staged-security-with-actual-alpha-prefix-composition-candidate-drift-query-hit-bound-from-conceptual-*
assumes *wf: staged-budget-wellformed budgets*
and *controlled: staged-adversary-controlled budgets A*
and *residual-bound*:
 $\bigwedge i. i < \text{rounds} \implies$
 wp-event
 $(\text{checked-staged-security-experiment-with-query-prefix-data-state } A \ i)$
 $(\text{checked-staged-security-with-query-prefix-conceptual-context-residual-hit-at } i)$
 $\text{adversary-initial-state} \leq R \ i$
and *trace-path-bound*:
 $\bigwedge i. i < \text{rounds} \implies$
 wp-event
 $(\text{checked-staged-security-experiment-with-query-prefix-data-state } A \ i)$

$(\text{checked-staged-security-with-query-prefix-current-trace-path-output-hit-at } i)$
 $\text{adversary-initial-state} \leq T \ i$
and *composition-path-bound*:
 $\bigwedge i. i < \text{rounds} \implies$
 wp-event
 $(\text{checked-staged-security-experiment-with-query-prefix-data-state } A \ i)$
 $(\text{checked-staged-security-with-query-prefix-current-composition-path-output-hit-at } i)$
 $\text{adversary-initial-state} \leq C \ i$
shows
 wp-event
 $(\text{checked-staged-security-experiment-with-actual-alpha-prefix-data-state } A)$
 $\text{checked-staged-security-with-actual-alpha-prefix-composition-candidate-drift-query-hit}$
 $\text{adversary-initial-state} \leq$
 $(\sum i < \text{rounds}.$
 $\text{staged-phase-relation-error size}$
 $(\text{staged-query-search-queries budgets } i + 1) +$
 $\text{query-error-bound} + R \ i + T \ i + C \ i)$
proof (rule
 $\text{checked-staged-security-with-actual-alpha-prefix-composition-candidate-drift-query-hit-bound-from-query-prefix-curr}$
 $[OF \ \text{wf controlled}]$)
fix i
assume $i\text{-bound}: i < \text{rounds}$
show
 wp-event
 $(\text{checked-staged-security-experiment-with-query-prefix-data-state } A \ i)$
 $(\text{checked-staged-security-with-query-prefix-current-query-partial-opening-hit-at } i)$
 $\text{adversary-initial-state} \leq$
 $\text{staged-phase-relation-error size}$
 $(\text{staged-query-search-queries budgets } i + 1) +$
 $\text{query-error-bound} + R \ i + T \ i + C \ i$
by (rule
 $\text{checked-staged-security-with-query-prefix-current-query-partial-opening-hit-at-bound-from-conceptual-residual-and}$
 $[OF \ \text{wf controlled } i\text{-bound residual-bound}[OF \ i\text{-bound}]$
 $\text{trace-path-bound}[OF \ i\text{-bound}] \ \text{composition-path-bound}[OF \ i\text{-bound}]]$)
qed

lemma $\text{checked-staged-security-with-actual-alpha-prefix-composition-candidate-drift-query-hit-bound-from-conceptual-}$
assumes $\text{wf}: \text{staged-budget-wellformed budgets}$
and $\text{controlled}: \text{staged-adversary-controlled budgets } A$
and residual-bound :
 $\bigwedge i. i < \text{rounds} \implies$
 wp-event
 $(\text{checked-staged-security-experiment-with-query-prefix-data-state } A \ i)$
 $(\text{checked-staged-security-with-query-prefix-conceptual-default-context-residual-hit-at}$
 $\text{trace-default composition-default } i)$
 $\text{adversary-initial-state} \leq R \ i$
and trace-path-bound :
 $\bigwedge i. i < \text{rounds} \implies$
 wp-event

(checked-staged-security-experiment-with-query-prefix-data-state $A \ i$)
 (checked-staged-security-with-query-prefix-current-trace-path-output-hit-at i)
 adversary-initial-state $\leq T \ i$
and composition-path-bound:
 $\bigwedge i. i < \text{rounds} \implies$
 wp-event
 (checked-staged-security-experiment-with-query-prefix-data-state $A \ i$)
 (checked-staged-security-with-query-prefix-current-composition-path-output-hit-at i)
 adversary-initial-state $\leq C \ i$
shows
 wp-event
 (checked-staged-security-experiment-with-actual-alpha-prefix-data-state A)
 checked-staged-security-with-actual-alpha-prefix-composition-candidate-drift-query-hit
 adversary-initial-state $\leq$
 ($\sum i < \text{rounds}.$
 staged-phase-relation-error size
 (staged-query-search-queries budgets $i + 1$) +
 query-error-bound + $R \ i + T \ i + C \ i$)
proof (rule
 checked-staged-security-with-actual-alpha-prefix-composition-candidate-drift-query-hit-bound-from-query-prefix-curr
 [OF wf controlled])
fix i
assume i -bound: $i < \text{rounds}$
show
 wp-event
 (checked-staged-security-experiment-with-query-prefix-data-state $A \ i$)
 (checked-staged-security-with-query-prefix-current-query-partial-opening-hit-at i)
 adversary-initial-state $\leq$
 staged-phase-relation-error size
 (staged-query-search-queries budgets $i + 1$) +
 query-error-bound + $R \ i + T \ i + C \ i$
by (rule
 checked-staged-security-with-query-prefix-current-query-partial-opening-hit-at-bound-from-conceptual-default-resi
 [OF wf controlled i -bound residual-bound[OF i -bound]
 trace-path-bound[OF i -bound] composition-path-bound[OF i -bound]])
qed

lemma checked-staged-security-with-actual-alpha-prefix-composition-candidate-drift-query-hit-bound-from-conceptual-
assumes wf: staged-budget-wellformed budgets
and controlled: staged-adversary-controlled budgets A
and default-context:
 $\bigwedge i \text{ prefix prefix-state. } i < \text{rounds} \implies$
 Some ((prefix, prefix-state), prefix-state) $\in$
 set-dist
 (execute (checked-staged-query-prefix-with-state $A \ i$)
 adversary-initial-state) $\implies$
 trace-table-low-degree
 (query-prefix-trace-conceptual-table-with-default prefix
 prefix-state trace-default) $\wedge$

composition-table-low-degree maxDegree
 (query-prefix-composition-conceptual-table-with-default prefix
 prefix-state composition-default) $\wedge$
 $\neg$ all-queries-consistent
 (query-prefix-trace-conceptual-table-with-default prefix
 prefix-state trace-default)
 (query-prefix-composition-conceptual-table-with-default prefix
 prefix-state composition-default)
 (sqp-alphas prefix)
and trace-path-bound:
 $\bigwedge i. i < \text{rounds} \implies$
 wp-event
 (checked-staged-security-experiment-with-query-prefix-data-state A i)
 (checked-staged-security-with-query-prefix-current-trace-path-output-hit-at i)
 adversary-initial-state $\leq T i$
and composition-path-bound:
 $\bigwedge i. i < \text{rounds} \implies$
 wp-event
 (checked-staged-security-experiment-with-query-prefix-data-state A i)
 (checked-staged-security-with-query-prefix-current-composition-path-output-hit-at i)
 adversary-initial-state $\leq C i$
shows
 wp-event
 (checked-staged-security-experiment-with-actual-alpha-prefix-data-state A)
 checked-staged-security-with-actual-alpha-prefix-composition-candidate-drift-query-hit
 adversary-initial-state $\leq$
 ($\sum i < \text{rounds}.$
 staged-phase-relation-error size
 (staged-query-search-queries budgets i + 1) +
 query-error-bound +
 hash-collision-budget-value 0
 (staged-query-search-queries budgets i + 1) +
 T i + C i)
proof (rule
 checked-staged-security-with-actual-alpha-prefix-composition-candidate-drift-query-hit-bound-from-query-prefix-curr
 [OF wf controlled])
fix i
assume i-bound: i < rounds
show
 wp-event
 (checked-staged-security-experiment-with-query-prefix-data-state A i)
 (checked-staged-security-with-query-prefix-current-query-partial-opening-hit-at i)
 adversary-initial-state $\leq$
 staged-phase-relation-error size
 (staged-query-search-queries budgets i + 1) +
 query-error-bound +
 hash-collision-budget-value 0
 (staged-query-search-queries budgets i + 1) +
 T i + C i

by (rule
 checked-staged-security-with-query-prefix-current-query-partial-opening-hit-at-bound-from-conceptual-default-cont
 [OF wf controlled i-bound - trace-path-bound[OF i-bound]
 composition-path-bound[OF i-bound]])
 (rule default-context[OF i-bound])
 qed

lemma checked-staged-security-with-actual-alpha-prefix-composition-candidate-drift-query-hit-bound-from-conceptual-

assumes wf: staged-budget-wellformed budgets

and controlled: staged-adversary-controlled budgets A

and algebraic-residual-bound:

$\bigwedge i. i < \text{rounds} \implies$

wp-event

(checked-staged-security-experiment-with-query-prefix-data-state A i)

(checked-staged-security-with-query-prefix-conceptual-context-algebraic-residual-hit-at
 i)

adversary-initial-state $\leq R$ i

and trace-path-bound:

$\bigwedge i. i < \text{rounds} \implies$

wp-event

(checked-staged-security-experiment-with-query-prefix-data-state A i)

(checked-staged-security-with-query-prefix-current-trace-path-output-hit-at i)

adversary-initial-state $\leq T$ i

and composition-path-bound:

$\bigwedge i. i < \text{rounds} \implies$

wp-event

(checked-staged-security-experiment-with-query-prefix-data-state A i)

(checked-staged-security-with-query-prefix-current-composition-path-output-hit-at i)

adversary-initial-state $\leq C$ i

shows

wp-event

(checked-staged-security-experiment-with-actual-alpha-prefix-data-state A)

checked-staged-security-with-actual-alpha-prefix-composition-candidate-drift-query-hit

adversary-initial-state $\leq$

($\sum i < \text{rounds}.$

staged-phase-relation-error size

(staged-query-search-queries budgets i + 1) +

query-error-bound +

hash-collision-budget-value 0

(staged-query-search-queries budgets i + 1) +

R i + T i + C i)

proof (rule

checked-staged-security-with-actual-alpha-prefix-composition-candidate-drift-query-hit-bound-from-query-prefix-curr

[OF wf controlled])

fix i

assume i-bound: i < rounds

show

wp-event

(checked-staged-security-experiment-with-query-prefix-data-state A i)

(checked-staged-security-with-query-prefix-current-query-partial-opening-hit-at i)
adversary-initial-state $\leq$
staged-phase-relation-error size
(staged-query-search-queries budgets i + 1) +
query-error-bound +
hash-collision-budget-value 0
(staged-query-search-queries budgets i + 1) +
R i + T i + C i
by (rule
checked-staged-security-with-query-prefix-current-query-partial-opening-hit-at-bound-from-conceptual-algebraic-re
[OF wf controlled i-bound algebraic-residual-bound[OF i-bound]
trace-path-bound[OF i-bound] composition-path-bound[OF i-bound]]])
qed

lemma *checked-staged-security-with-actual-alpha-prefix-composition-candidate-drift-query-hit-bound-from-conceptual-*

assumes *false-statement: $\neg$ exists-valid-trace*

and *wf: staged-budget-wellformed budgets*

and *controlled: staged-adversary-controlled budgets A*

and *trace-low-degree-bound:*

$\bigwedge i. i < \text{rounds} \implies$

wp-event

(checked-staged-security-experiment-with-query-prefix-data-state A i)

(checked-staged-security-with-query-prefix-conceptual-trace-low-degree-bad-at i)

adversary-initial-state $\leq LT i$

and *composition-low-degree-bound:*

$\bigwedge i. i < \text{rounds} \implies$

wp-event

(checked-staged-security-experiment-with-query-prefix-data-state A i)

(checked-staged-security-with-query-prefix-conceptual-composition-low-degree-bad-at i)

adversary-initial-state $\leq LC i$

and *alpha-bound:*

$\bigwedge i. i < \text{rounds} \implies$

wp-event

(checked-staged-security-experiment-with-query-prefix-data-state A i)

(checked-staged-security-with-query-prefix-conceptual-alpha-bad-set-hit-at i)

adversary-initial-state $\leq P i$

and *trace-path-bound:*

$\bigwedge i. i < \text{rounds} \implies$

wp-event

(checked-staged-security-experiment-with-query-prefix-data-state A i)

(checked-staged-security-with-query-prefix-current-trace-path-output-hit-at i)

adversary-initial-state $\leq T i$

and *composition-path-bound:*

$\bigwedge i. i < \text{rounds} \implies$

wp-event

(checked-staged-security-experiment-with-query-prefix-data-state A i)

$(\text{checked-staged-security-with-query-prefix-current-composition-path-output-hit-at } i)$
 $\text{adversary-initial-state} \leq C \ i$

shows

wp-event
 $(\text{checked-staged-security-experiment-with-actual-alpha-prefix-data-state } A)$
 $\text{checked-staged-security-with-actual-alpha-prefix-composition-candidate-drift-query-hit}$
 $\text{adversary-initial-state} \leq$
 $(\sum_{i < \text{rounds.}}$
 $\text{staged-phase-relation-error size}$
 $(\text{staged-query-search-queries budgets } i + 1) +$
 $\text{query-error-bound} +$
 $\text{hash-collision-budget-value } 0$
 $(\text{staged-query-search-queries budgets } i + 1) +$
 $(LT \ i + LC \ i + P \ i) + T \ i + C \ i)$

proof (rule
 $\text{checked-staged-security-with-actual-alpha-prefix-composition-candidate-drift-query-hit-bound-from-query-prefix-curr}$
 $[OF \ \text{wf controlled}]$)

fix i

assume $i\text{-bound}: i < \text{rounds}$

show

wp-event
 $(\text{checked-staged-security-experiment-with-query-prefix-data-state } A \ i)$
 $(\text{checked-staged-security-with-query-prefix-current-query-partial-opening-hit-at } i)$
 $\text{adversary-initial-state} \leq$
 $\text{staged-phase-relation-error size}$
 $(\text{staged-query-search-queries budgets } i + 1) +$
 $\text{query-error-bound} +$
 $\text{hash-collision-budget-value } 0$
 $(\text{staged-query-search-queries budgets } i + 1) +$
 $(LT \ i + LC \ i + P \ i) + T \ i + C \ i$

by (rule
 $\text{checked-staged-security-with-query-prefix-current-query-partial-opening-hit-at-bound-from-conceptual-residual-spl}$
 $[OF \ \text{false-statement wf controlled } i\text{-bound}]$
 $\text{trace-low-degree-bound}[OF \ i\text{-bound}]$
 $\text{composition-low-degree-bound}[OF \ i\text{-bound}]$
 $\text{alpha-bound}[OF \ i\text{-bound}]$
 $\text{trace-path-bound}[OF \ i\text{-bound}]$
 $\text{composition-path-bound}[OF \ i\text{-bound}]])$

qed

lemma $\text{checked-staged-security-with-query-prefix-conceptual-context-residual-hit-at-trivial-bound}:$

wp-event
 $(\text{checked-staged-security-experiment-with-query-prefix-data-state } A \ i)$
 $(\text{checked-staged-security-with-query-prefix-conceptual-context-residual-hit-at}$
 $i)$
 $\text{adversary-initial-state} \leq 1$

by (rule wp-event-le-1)

lemma $\text{checked-staged-security-with-query-prefix-current-query-partial-opening-hit-at-bound-from-conceptual-trivial-r}$

assumes *wf*: staged-budget-wellformed budgets
and *controlled*: staged-adversary-controlled budgets *A*
and *i-bound*: $i < \text{rounds}$
and *trace-path-bound*:
 wp-event
 (*checked-staged-security-experiment-with-query-prefix-data-state* *A i*)
 (*checked-staged-security-with-query-prefix-current-trace-path-output-hit-at* *i*)
 adversary-initial-state $\leq T$
and *composition-path-bound*:
 wp-event
 (*checked-staged-security-experiment-with-query-prefix-data-state* *A i*)
 (*checked-staged-security-with-query-prefix-current-composition-path-output-hit-at* *i*)
 adversary-initial-state $\leq C$
shows
 wp-event
 (*checked-staged-security-experiment-with-query-prefix-data-state* *A i*)
 (*checked-staged-security-with-query-prefix-current-query-partial-opening-hit-at* *i*)
 adversary-initial-state $\leq$
 staged-phase-relation-error size
 (*staged-query-search-queries budgets* $i + 1$) +
 query-error-bound + 1 + *T* + *C*
by (rule
 checked-staged-security-with-query-prefix-current-query-partial-opening-hit-at-bound-from-conceptual-residual-and-
 [*OF wf controlled i-bound*
 checked-staged-security-with-query-prefix-conceptual-context-residual-hit-at-trivial-bound
 trace-path-bound composition-path-bound])

lemma *checked-staged-security-with-actual-alpha-prefix-composition-candidate-drift-query-hit-bound-from-conceptual-*
assumes *wf*: staged-budget-wellformed budgets
and *controlled*: staged-adversary-controlled budgets *A*
and *trace-path-bound*:
 $\bigwedge i. i < \text{rounds} \implies$
 wp-event
 (*checked-staged-security-experiment-with-query-prefix-data-state* *A i*)
 (*checked-staged-security-with-query-prefix-current-trace-path-output-hit-at* *i*)
 adversary-initial-state $\leq T i$
and *composition-path-bound*:
 $\bigwedge i. i < \text{rounds} \implies$
 wp-event
 (*checked-staged-security-experiment-with-query-prefix-data-state* *A i*)
 (*checked-staged-security-with-query-prefix-current-composition-path-output-hit-at* *i*)
 adversary-initial-state $\leq C i$
shows
 wp-event
 (*checked-staged-security-experiment-with-actual-alpha-prefix-data-state* *A*)
 checked-staged-security-with-actual-alpha-prefix-composition-candidate-drift-query-hit
 adversary-initial-state $\leq$
 $(\sum i < \text{rounds}.$
 staged-phase-relation-error size

$(\text{staged-query-search-queries budgets } i + 1) +$
 $\text{query-error-bound} + 1 + T i + C i)$

proof (rule
checked-staged-security-with-actual-alpha-prefix-composition-candidate-drift-query-hit-bound-from-conceptual-residual
 $[OF \text{ wf controlled - trace-path-bound composition-path-bound}]$)
fix i
assume $i < \text{rounds}$
show
wp-event
 $(\text{checked-staged-security-experiment-with-query-prefix-data-state } A \ i)$
 $(\text{checked-staged-security-with-query-prefix-conceptual-context-residual-hit-at } i)$
 $\text{adversary-initial-state} \leq 1$
by (rule
checked-staged-security-with-query-prefix-conceptual-context-residual-hit-at-trivial-bound)
qed

lemma *checked-staged-security-with-actual-alpha-prefix-composition-candidate-drift-query-hit-bound-from-conceptual-*
assumes *wf: staged-budget-wellformed budgets*
and *controlled: staged-adversary-controlled budgets A*
and *conceptual-context:*
 $\bigwedge i \text{ prefix prefix-state.}$
 $i < \text{rounds} \implies$
 $\text{Some } ((\text{prefix}, \text{prefix-state}), \text{prefix-state}) \in$
set-dist
 $(\text{execute } (\text{checked-staged-query-prefix-with-state } A \ i)$
 $\text{adversary-initial-state}) \implies$
 $\neg \text{hash-map-output-collision prefix-state} \wedge$
trace-table-low-degree
 $(\text{query-prefix-trace-conceptual-table prefix prefix-state}) \wedge$
composition-table-low-degree maxDegree
 $(\text{query-prefix-composition-conceptual-table prefix prefix-state}) \wedge$
 $\neg \text{all-queries-consistent}$
 $(\text{query-prefix-trace-conceptual-table prefix prefix-state})$
 $(\text{query-prefix-composition-conceptual-table prefix prefix-state})$
 $(\text{sqr-alphas prefix})$
and *trace-path-bound:*
 $\bigwedge i. i < \text{rounds} \implies$
wp-event
 $(\text{checked-staged-security-experiment-with-query-prefix-data-state } A \ i)$
 $(\text{checked-staged-security-with-query-prefix-current-trace-path-output-hit-at } i)$
 $\text{adversary-initial-state} \leq T \ i$
and *composition-path-bound:*
 $\bigwedge i. i < \text{rounds} \implies$
wp-event
 $(\text{checked-staged-security-experiment-with-query-prefix-data-state } A \ i)$
 $(\text{checked-staged-security-with-query-prefix-current-composition-path-output-hit-at } i)$
 $\text{adversary-initial-state} \leq C \ i$
shows
wp-event

```

    (checked-staged-security-experiment-with-actual-alpha-prefix-data-state A)
    checked-staged-security-with-actual-alpha-prefix-composition-candidate-drift-query-hit
    adversary-initial-state ≤
    (∑ i < rounds.
      staged-phase-relation-error size
      (staged-query-search-queries budgets i + 1) +
      query-error-bound + T i + C i)
proof (rule
  checked-staged-security-with-actual-alpha-prefix-composition-candidate-drift-query-hit-bound-from-prefix-target-and-
  [OF wf controlled - trace-path-bound composition-path-bound])
fix i
assume i-bound: i < rounds
show
  wp-event (checked-staged-query-prefix-receive-with-state A i)
  (checked-staged-query-prefix-dynamic-index-hit
    staged-query-prefix-prefix-authenticated-opening-target)
  adversary-initial-state ≤
  staged-phase-relation-error size
  (staged-query-search-queries budgets i + 1) + query-error-bound
by (rule
  checked-staged-query-prefix-prefix-authenticated-target-hit-bound-from-conceptual
  [OF wf controlled i-bound])
  (rule conceptual-context[OF i-bound])
qed

end

end

```

```

theory Soundness-Conceptual-Query-Current-Candidate
imports Soundness-Conceptual-Query-Current-Path
begin

```

Fixed-candidate projection for current-query openings.

This layer keeps the trace/composition candidate tables as fixed parameters while projecting the data-state current-opening event to the corresponding query-prefix event. It is the form needed by the conceptual/default query target bound: the target is fixed before the query-index challenge.

```

context soundness
begin

```

```

definition staged-security-with-data-state-current-query-partial-candidate-hit-at
where

```

```

  staged-security-with-data-state-current-query-partial-candidate-hit-at
  trace-table composition-table i out ↔
  (case out of
    None ⇒ False
  | Some (((data, attacker-state), result), final-state) ⇒

```

(let $s =$
 verifier-state-from-adversary *attacker-state*
 (*staged-proof-transcript* *data*)
 in $\exists$ *raw* *trace-openings* *composition-openings* *rest*.
 $i < \text{rounds} \wedge$
 fmlookup (*HashMap* *final-state*)
 (*QueryIndexChallenge* i
 (*state-after-query-chunks*
 (*staged-query-start-hash* *data*)
 (*staged-query-chunks* *data*) i)) =
 Some *raw* $\wedge$
 staged-composition-fri-roots *data* $\neq [] \wedge$
 Some (*result*, *final-state*) $\in$ *set-dist* (*execute* *verify-monad* s) $\wedge$
 partial-authenticated-table (*staged-trace-root* *data*)
 (*scale* * *clength*) *trace-openings* *final-state* $\wedge$
 partial-authenticated-table
 (*hd* (*staged-composition-fri-roots* *data*)) (*scale* * *clength*)
 composition-openings *final-state* $\wedge$
 verifier-header-transcript s
 (*staged-trace-root* *data*)
 (*staged-trace-fri-roots* *data*)
 (*staged-trace-final* *data*)
 (*staged-alphas* *data*)
 (*staged-degree* *data*)
 (*staged-composition-fri-roots* *data*)
 (*staged-composition-final* *data*)
 rest $\wedge$
 partial-query-openings-consistent *trace-openings*
 composition-openings (*staged-alphas* *data*) (*index* (*to-nat* *raw*)) $\wedge$
 partial-trace-table-candidate *trace-table*
 ((*replicate* *rounds* []) [$i := \text{trace-openings}$]) $\wedge$
 partial-composition-table-candidate *composition-table*
 ((*replicate* *rounds* []) [$i := \text{composition-openings}$]))))

definition *staged-security-with-data-state-current-query-some-partial-candidate-hit*
where

staged-security-with-data-state-current-query-some-partial-candidate-hit
out $\longleftrightarrow$
 ($\exists$ *trace-table* *composition-table* i .
 $i < \text{rounds} \wedge$
 staged-security-with-data-state-current-query-partial-candidate-hit-at
 trace-table *composition-table* i *out*)

lemma *staged-security-with-data-state-current-query-partial-candidate-hit-at-imp-query-prefix-authenticated-candidate*
assumes *wf*: *staged-budget-wellformed* *budgets*
and *controlled*: *staged-adversary-controlled* *budgets* A
and *i-bound*: $i < \text{rounds}$
and *support*:
 Some (((((*prefix*, *prefix-state*), *raw*), *raw-state*),

```

      ((data, attacker-state), result)), final-state) ∈
    set-dist
      (execute
        (checked-staged-security-experiment-with-query-prefix-data-state A i)
        adversary-initial-state)
  and hit:
    staged-security-with-data-state-current-query-partial-candidate-hit-at
      trace-table composition-table i
      (Some (((data, attacker-state), result), final-state))
  shows
    checked-staged-security-with-query-prefix-authenticated-candidate-hit-at
      trace-table composition-table i
      (Some (((((prefix, prefix-state), raw), raw-state),
        ((data, attacker-state), result)), final-state))
  proof -
  let ?s =
    verifier-state-from-adversary attacker-state
    (staged-proof-transcript data)
  from hit obtain raw' trace-openings composition-openings rest where
    final-lookup':
    fmlookup (HashMap final-state)
      (QueryIndexChallenge i
        (state-after-query-chunks
          (staged-query-start-hash data) (staged-query-chunks data) i)) =
      Some raw'
  and comp-nonempty: staged-composition-fri-roots data ≠ []
  and trace-auth:
    partial-authenticated-table (staged-trace-root data)
      (scale * clength) trace-openings final-state
  and comp-auth:
    partial-authenticated-table
      (hd (staged-composition-fri-roots data)) (scale * clength)
      composition-openings final-state
  and consistent':
    partial-query-openings-consistent trace-openings
      composition-openings (staged-alphas data) (index (to-nat raw'))
  and trace-candidate:
    partial-trace-table-candidate trace-table
      ((replicate rounds []) [i := trace-openings])
  and comp-candidate:
    partial-composition-table-candidate composition-table
      ((replicate rounds []) [i := composition-openings])
  unfolding
    staged-security-with-data-state-current-query-partial-candidate-hit-at-def
    Let-def
  by auto
  have final-lookup:
    fmlookup (HashMap final-state)
      (QueryIndexChallenge i

```

```

    (state-after-query-chunks
      (staged-query-start-hash data) (staged-query-chunks data) i)) =
  Some raw
by (rule
  checked-staged-security-experiment-with-query-prefix-data-state-raw-final-lookup
  [OF wf controlled i-bound support])
have raw-eq: raw' = raw
  using final-lookup' final-lookup by simp
have opening-hit:
  checked-staged-security-with-query-prefix-authenticated-opening-hit
  trace-openings composition-openings i
  (Some (((((prefix, prefix-state), raw), raw-state),
    ((data, attacker-state), result)), final-state))
  unfolding checked-staged-security-with-query-prefix-authenticated-opening-hit-def
  using comp-nonempty trace-auth comp-auth consistent' raw-eq by simp
show ?thesis
  unfolding checked-staged-security-with-query-prefix-authenticated-candidate-hit-at-def
  by (intro exI[of - trace-openings] exI[of - composition-openings]
    conjI opening-hit trace-candidate comp-candidate)
qed

lemma staged-security-with-data-state-current-query-partial-candidate-hit-at-le-query-prefix-authenticated-candidate:
  assumes wf: staged-budget-wellformed budgets
  and controlled: staged-adversary-controlled budgets A
  and i-bound: i < rounds
  shows
    wp-event
      (checked-staged-security-experiment-with-data-state A)
      (staged-security-with-data-state-current-query-partial-candidate-hit-at
        trace-table composition-table i)
      adversary-initial-state ≤
    wp-event
      (checked-staged-security-experiment-with-query-prefix-data-state A i)
      (checked-staged-security-with-query-prefix-authenticated-candidate-hit-at
        trace-table composition-table i)
      adversary-initial-state
proof -
  let ?projected =
    λout. case out of
      None ⇒
        staged-security-with-data-state-current-query-partial-candidate-hit-at
        trace-table composition-table i None
      | Some (packed, t) ⇒
        staged-security-with-data-state-current-query-partial-candidate-hit-at
        trace-table composition-table i (Some (snd packed, t))
  have projection:
    wp-event
      (checked-staged-security-experiment-with-data-state A)
      (staged-security-with-data-state-current-query-partial-candidate-hit-at

```

```

    trace-table composition-table i)
  adversary-initial-state =
wp-event
  (checked-staged-security-experiment-with-query-prefix-data-state A i)
  ?projected
  adversary-initial-state
by (rule
  checked-staged-security-experiment-with-query-prefix-data-state-projection-event
  [OF i-bound])
have event-le:
wp-event
  (checked-staged-security-experiment-with-query-prefix-data-state A i)
  ?projected
  adversary-initial-state ≤
wp-event
  (checked-staged-security-experiment-with-query-prefix-data-state A i)
  (checked-staged-security-with-query-prefix-authenticated-candidate-hit-at
    trace-table composition-table i)
  adversary-initial-state
proof (rule wp-event-mono-on-support)
fix out
assume support:
  out ∈
  set-dist
  (execute
    (checked-staged-security-experiment-with-query-prefix-data-state A i)
    adversary-initial-state)
  and hit: ?projected out
show
  checked-staged-security-with-query-prefix-authenticated-candidate-hit-at
    trace-table composition-table i out
proof (cases out)
case None
then show ?thesis
  using hit
  unfolding
    staged-security-with-data-state-current-query-partial-candidate-hit-at-def
  by simp
next
case (Some packed)
then obtain prefix prefix-state raw raw-state data attacker-state result
  final-state where out-eq:
    out =
      Some (((((prefix, prefix-state), raw), raw-state),
        ((data, attacker-state), result)), final-state)
  by (cases packed) (auto split: prod.splits)
show ?thesis
  unfolding out-eq
  by (rule

```

```

      staged-security-with-data-state-current-query-partial-candidate-hit-at-imp-query-prefix-authenticated-candidate
      [OF wf controlled i-bound])
      (use support hit out-eq in simp-all)
    qed
  qed
  show ?thesis
    unfolding projection
    by (rule event-le)
  qed

```

lemma *staged-security-with-data-state-current-query-partial-candidate-hit-at-bound-from-conceptual-default:*

```

assumes wf: staged-budget-wellformed budgets
and controlled: staged-adversary-controlled budgets A
and i-bound:  $i < \text{rounds}$ 
and residual-bound:
  wp-event
    (checked-staged-security-experiment-with-query-prefix-data-state A i)
    (checked-staged-security-with-query-prefix-conceptual-default-context-residual-hit-at
      trace-table composition-table i)
    adversary-initial-state  $\leq R$ 
and trace-path-bound:
  wp-event
    (checked-staged-security-experiment-with-query-prefix-data-state A i)
    (checked-staged-security-with-query-prefix-current-trace-path-output-hit-at i)
    adversary-initial-state  $\leq T$ 
and composition-path-bound:
  wp-event
    (checked-staged-security-experiment-with-query-prefix-data-state A i)
    (checked-staged-security-with-query-prefix-current-composition-path-output-hit-at i)
    adversary-initial-state  $\leq C$ 

```

shows

```

  wp-event
    (checked-staged-security-experiment-with-data-state A)
    (staged-security-with-data-state-current-query-partial-candidate-hit-at
      trace-table composition-table i)
    adversary-initial-state  $\leq$ 
    staged-phase-relation-error size
    (staged-query-search-queries budgets  $i + 1$ ) +
    query-error-bound +  $R + T + C$ 

```

proof –

```

have prefix-bound:
  wp-event
    (checked-staged-security-experiment-with-query-prefix-data-state A i)
    (checked-staged-security-with-query-prefix-authenticated-candidate-hit-at
      trace-table composition-table i)
    adversary-initial-state  $\leq$ 
    staged-phase-relation-error size
    (staged-query-search-queries budgets  $i + 1$ ) +
    query-error-bound +  $R + T + C$ 

```


by (rule
 checked-staged-security-with-query-prefix-authenticated-candidate-hit-at-bound-from-conceptual-default-residual-a
 [*OF wf controlled i-bound residual-bound trace-path-bound*
 composition-path-bound])
 show ?thesis
 by (rule *order-trans*
 [*OF*
 staged-security-with-data-state-current-query-partial-candidate-hit-at-le-query-prefix-authenticated-candidate
 prefix-bound])
 (use *wf controlled i-bound* in *simp-all*)
 qed

lemma *staged-security-with-data-state-current-query-partial-candidate-hit-at-bound-from-conceptual-default-context*:
assumes *wf: staged-budget-wellformed budgets*
and *controlled: staged-adversary-controlled budgets A*
and *i-bound: i < rounds*
and *default-context*:
 $\bigwedge \text{prefix prefix-state.}$
 Some ((prefix, prefix-state), prefix-state) ∈
 set-dist
 (*execute (checked-staged-query-prefix-with-state A i)*
 adversary-initial-state) $\implies$
 trace-table-low-degree
 (*query-prefix-trace-conceptual-table-with-default prefix*
 prefix-state trace-table) $\wedge$
 composition-table-low-degree maxDegree
 (*query-prefix-composition-conceptual-table-with-default prefix*
 prefix-state composition-table) $\wedge$
 $\neg$ *all-queries-consistent*
 (*query-prefix-trace-conceptual-table-with-default prefix*
 prefix-state trace-table)
 (*query-prefix-composition-conceptual-table-with-default prefix*
 prefix-state composition-table)
 (*sqp-alphas prefix*)
and *trace-path-bound*:
 wp-event
 (*checked-staged-security-experiment-with-query-prefix-data-state A i*)
 (*checked-staged-security-with-query-prefix-current-trace-path-output-hit-at i*)
 adversary-initial-state ≤ T
and *composition-path-bound*:
 wp-event
 (*checked-staged-security-experiment-with-query-prefix-data-state A i*)
 (*checked-staged-security-with-query-prefix-current-composition-path-output-hit-at i*)
 adversary-initial-state ≤ C
shows
 wp-event
 (*checked-staged-security-experiment-with-data-state A*)
 (*staged-security-with-data-state-current-query-partial-candidate-hit-at*
 trace-table composition-table i)

adversary-initial-state $\leq$
staged-phase-relation-error size
(staged-query-search-queries budgets i + 1) +
query-error-bound +
hash-collision-budget-value 0
(staged-query-search-queries budgets i + 1) + T + C

proof –

have *prefix-bound*:

wp-event
(checked-staged-security-experiment-with-query-prefix-data-state A i)
(checked-staged-security-with-query-prefix-authenticated-candidate-hit-at
trace-table composition-table i)
adversary-initial-state $\leq$
staged-phase-relation-error size
(staged-query-search-queries budgets i + 1) +
query-error-bound +
hash-collision-budget-value 0
(staged-query-search-queries budgets i + 1) + T + C

by (rule
checked-staged-security-with-query-prefix-authenticated-candidate-hit-at-bound-from-conceptual-default-context-an
[OF wf controlled i-bound default-context trace-path-bound
composition-path-bound])

show *?thesis*

by (rule *order-trans*
[OF
staged-security-with-data-state-current-query-partial-candidate-hit-at-le-query-prefix-authenticated-candidate
prefix-bound])
(use wf controlled i-bound in simp-all)

qed

lemma *checked-staged-security-with-actual-alpha-prefix-composition-candidate-drift-query-hit-imp-data-state-current-*

assumes *wf: staged-budget-wellformed budgets*

and *controlled: staged-adversary-controlled budgets A*

and *support:*
Some (((((alpha-prefix, alpha-prefix-state), data), attacker-state),
result), final-state) ∈
set-dist
(execute
(checked-staged-security-experiment-with-actual-alpha-prefix-data-state
A)
adversary-initial-state)

and *hit:*
checked-staged-security-with-actual-alpha-prefix-composition-candidate-drift-query-hit
(Some (((((alpha-prefix, alpha-prefix-state), data), attacker-state),
result), final-state))

obtains *trace-table composition-table i where*
i < rounds
staged-security-with-data-state-current-query-partial-candidate-hit-at
trace-table composition-table i

```

      (Some (((data, attacker-state), result), final-state))
proof –
  let ?s =
    verifier-state-from-adversary attacker-state
    (staged-proof-transcript data)
  have data-support:
    Some (((data, attacker-state), result), final-state) ∈
      set-dist
      (execute (checked-staged-security-experiment-with-data-state A)
        adversary-initial-state)
  by (rule
    checked-staged-security-with-actual-alpha-prefix-support-imp-data-state-support
    [OF support])
from checked-staged-security-experiment-with-data-state-outcomeE
  [OF data-support]
obtain builder verifier where
  builder:
    Some (data, attacker-state) ∈
      set-dist
      (execute (checked-staged-transcript-program A)
        adversary-initial-state)
  and verifier:
    Some (result, final-state) ∈ set-dist (execute verify-monad ?s)
  by blast
have header:
  verifier-header-transcript ?s
    (staged-trace-root data)
    (staged-trace-fri-roots data)
    (staged-trace-final data)
    (staged-alphas data)
    (staged-degree data)
    (staged-composition-fri-roots data)
    (staged-composition-final data)
    (List.concat (staged-query-chunks data))
  by (rule checked-staged-transcript-program-outcome-header-transcript
    [OF wf controlled builder])
show ?thesis
proof (rule
  checked-staged-security-with-actual-alpha-prefix-composition-candidate-drift-query-hit-current-openingsE
  [OF wf controlled support hit])
  fix s trace-table composition-table query-idxs f i raw trace-openings
    composition-openings trace-openingss composition-openingss
  assume s-def:
    s = verifier-state-from-adversary attacker-state
    (staged-proof-transcript data)
  and bad-context:
    composition-bad-context s (Some (result, final-state))
    trace-table composition-table (staged-alphas data) query-idxs f
  and i-bound: i < rounds

```

```

and final-lookup:
  fmlookup (HashMap final-state)
    (QueryIndexChallenge i
      (state-after-query-chunks
        (staged-query-start-hash data)
        (staged-query-chunks data i)) =
      Some raw
and idx-eq: index (to-nat raw) = query-idxs ! i
and comp-nonempty: staged-composition-fri-roots data  $\neq$  []
and trace-auth:
  partial-authenticated-table (staged-trace-root data)
    (scale * clength) trace-openings final-state
and comp-auth:
  partial-authenticated-table
    (hd (staged-composition-fri-roots data)) (scale * clength)
    composition-openings final-state
and consistent:
  partial-query-openings-consistent trace-openings
    composition-openings (staged-alphas data) (query-idxs ! i)
and trace-candidate-full:
  partial-trace-table-candidate trace-table trace-openings
and comp-candidate-full:
  partial-composition-table-candidate composition-table
    composition-openings
and trace-openings-eq: trace-openings = trace-openings ! i
and comp-openings-eq: composition-openings = composition-openings ! i
and all-queries:
  all-queries-consistent trace-table composition-table
    (staged-alphas data)
and not-alpha-prefix:
   $\neg$  staged-alphas data  $\in$ 
    alpha-prefix-union-bad-sets alpha-prefix-state
    composition-trace-bad-alpha-space (fst alpha-prefix)
have consistent-raw:
  partial-query-openings-consistent trace-openings composition-openings
    (staged-alphas data) (index (to-nat raw))
  using consistent idx-eq by simp
have trace-candidate:
  partial-trace-table-candidate trace-table
    ((replicate rounds []) [i := trace-openings])
  by (rule partial-trace-table-candidate-single-roundI)
    (use i-bound trace-candidate-full trace-openings-eq in
       $\langle$ auto simp: partial-trace-table-candidate-def $\rangle$ )
have comp-candidate:
  partial-composition-table-candidate composition-table
    ((replicate rounds []) [i := composition-openings])
  by (rule partial-composition-table-candidate-single-roundI)
    (use i-bound comp-candidate-full comp-openings-eq in
       $\langle$ auto simp: partial-composition-table-candidate-def $\rangle$ )

```

```

have round-hit:
  staged-security-with-data-state-current-query-partial-candidate-hit-at
    trace-table composition-table i
    (Some (((data, attacker-state), result), final-state))
unfolding
  staged-security-with-data-state-current-query-partial-candidate-hit-at-def
  Let-def
apply simp
by (intro exI[of - raw] exI[of - trace-openings]
      exI[of - composition-openings]
      exI[of - List.concat (staged-query-chunks data)] conjI)
  (use i-bound final-lookup comp-nonempty verifier trace-auth
      comp-auth header consistent-raw trace-candidate comp-candidate
      in simp-all)
show ?thesis
by (rule that[OF i-bound round-hit])
qed
qed

lemma checked-staged-security-with-actual-alpha-prefix-composition-candidate-drift-query-hit-imp-data-state-current-
assumes wf: staged-budget-wellformed budgets
and controlled: staged-adversary-controlled budgets A
and support:
  Some (((((alpha-prefix, alpha-prefix-state), data), attacker-state),
    result), final-state) ∈
  set-dist
  (execute
    (checked-staged-security-experiment-with-actual-alpha-prefix-data-state
      A)
    adversary-initial-state)
and hit:
  checked-staged-security-with-actual-alpha-prefix-composition-candidate-drift-query-hit
    (Some (((((alpha-prefix, alpha-prefix-state), data), attacker-state),
      result), final-state))
shows
  staged-security-with-data-state-current-query-some-partial-candidate-hit
    (Some (((data, attacker-state), result), final-state))
proof —
from
  checked-staged-security-with-actual-alpha-prefix-composition-candidate-drift-query-hit-imp-data-state-current-partia
    [OF wf controlled support hit]
obtain trace-table composition-table i where
  i-bound: i < rounds
and round-hit:
  staged-security-with-data-state-current-query-partial-candidate-hit-at
    trace-table composition-table i
    (Some (((data, attacker-state), result), final-state))
by blast
show ?thesis

```

unfolding
staged-security-with-data-state-current-query-some-partial-candidate-hit-def
by (*intro* *exI*[*of* - *trace-table*] *exI*[*of* - *composition-table*]
exI[*of* - *i*] *conjI* *i-bound* *round-hit*)
qed

lemma *checked-staged-security-with-actual-alpha-prefix-composition-candidate-drift-query-hit-le-data-state-current-so*
assumes *wf*: *staged-budget-wellformed* *budgets*
and *controlled*: *staged-adversary-controlled* *budgets* *A*
shows
wp-event
(*checked-staged-security-experiment-with-actual-alpha-prefix-data-state* *A*)
checked-staged-security-with-actual-alpha-prefix-composition-candidate-drift-query-hit
adversary-initial-state $\leq$
wp-event
(*checked-staged-security-experiment-with-data-state* *A*)
staged-security-with-data-state-current-query-some-partial-candidate-hit
adversary-initial-state

proof –
let *?project* =
 $\lambda x. ((\text{snd } (\text{fst } (\text{fst } x))), \text{snd } (\text{fst } x)), \text{snd } x)$
let *?projected* =
 $\lambda \text{out}. \text{case out of}$
None $\Rightarrow$ *False*
| *Some* (*packed*, *t*) $\Rightarrow$
staged-security-with-data-state-current-query-some-partial-candidate-hit
(*Some* (*?project* *packed*, *t*))

have *event-le*:
wp-event
(*checked-staged-security-experiment-with-actual-alpha-prefix-data-state* *A*)
checked-staged-security-with-actual-alpha-prefix-composition-candidate-drift-query-hit
adversary-initial-state $\leq$
wp-event
(*checked-staged-security-experiment-with-actual-alpha-prefix-data-state* *A*)
?projected *adversary-initial-state*

proof (*rule* *wp-event-mono-on-support*)
fix *out*
assume *support*:
 $\text{out} \in$
set-dist
(*execute*
(*checked-staged-security-experiment-with-actual-alpha-prefix-data-state*
A)
adversary-initial-state)
and *hit*:
checked-staged-security-with-actual-alpha-prefix-composition-candidate-drift-query-hit
 out

show *?projected* *out*
proof (*cases* *out*)

```

case None
then show ?thesis
  using hit
  unfolding
    checked-staged-security-with-actual-alpha-prefix-composition-candidate-drift-query-hit-def
    checked-staged-security-with-actual-alpha-prefix-composition-candidate-drift-def
    checked-staged-security-with-actual-alpha-prefix-verifier-event-def
  by simp
next
case (Some packed)
then obtain alpha-prefix alpha-prefix-state data attacker-state result
  final-state where out-eq:
    out =
      Some (((((alpha-prefix, alpha-prefix-state), data),
        attacker-state), result), final-state)
    by (cases packed, auto split: prod.splits)
have candidate-hit:
  staged-security-with-data-state-current-query-some-partial-candidate-hit
    (Some (((data, attacker-state), result), final-state))
  by (rule
    checked-staged-security-with-actual-alpha-prefix-composition-candidate-drift-query-hit-imp-data-state-current-s
    [OF wf controlled])
    (use support hit out-eq in simp-all)
show ?thesis
  unfolding out-eq
  using candidate-hit by simp
qed
qed
have projection:
  wp-event
    (checked-staged-security-experiment-with-actual-alpha-prefix-data-state A)
    ?projected adversary-initial-state =
  wp-event
    (checked-staged-security-experiment-with-data-state A)
    staged-security-with-data-state-current-query-some-partial-candidate-hit
    adversary-initial-state
proof –
have
  wp-event (checked-staged-security-experiment-with-data-state A)
    staged-security-with-data-state-current-query-some-partial-candidate-hit
    adversary-initial-state =
  wp-event
    (checked-staged-security-experiment-with-actual-alpha-prefix-data-state A
       $\gg (\lambda x. \text{return } (?project\ x))$ )
    staged-security-with-data-state-current-query-some-partial-candidate-hit
    adversary-initial-state
using
  checked-staged-security-experiment-with-actual-alpha-prefix-data-state-projection
  [of A]

```

```

    by simp
  also have ... =
    wp-event
      (checked-staged-security-experiment-with-actual-alpha-prefix-data-state A)
      ?projected adversary-initial-state
    by (subst wp-event-bind-return-map)
      (simp add:
        staged-security-with-data-state-current-query-some-partial-candidate-hit-def
        staged-security-with-data-state-current-query-partial-candidate-hit-at-def)
  finally show ?thesis
    by simp
qed
show ?thesis
  by (rule order-trans[OF event-le])
    (simp add: projection)
qed

end

end

```

```

theory Soundness-Staged-Aligned-Current-Query-Path
imports
  Soundness-Staged-Aligned-Current-Query
  Soundness-Conceptual-Query-Current-Candidate
begin

```

Current-query wrappers that use structured Merkle path-output events.

This layer keeps trace and composition authenticated-opening path outputs separate, which is the form needed for Merkle and candidate uniqueness arguments in the staged soundness proof.

```

context soundness
begin

```

```

theorem stark-soundness-from-aligned-partial-query-prefix-current-prefix-structured-paths:
fixes trace-fri-error' composition-fri-error' N :: prob
and P T C :: nat  $\Rightarrow$  prob
assumes false-statement:  $\neg$  exists-valid-trace
and wf: staged-budget-wellformed budgets
and controlled: staged-adversary-controlled budgets A
and trace-fri-bound:
  wp-event (checked-staged-security-experiment-with-data-state A)
    (staged-security-with-data-state-verifier-event
      trace-fri-bad-with-partial-candidates)
    adversary-initial-state  $\leq$  trace-fri-error'
and comp-fri-bound:
  wp-event (checked-staged-security-experiment-with-data-state A)
    (staged-security-with-data-state-verifier-event

```


composition-fri-bad-with-partial-candidates
adversary-initial-state $\leq$ *composition-fri-error'*
and *prefix-bound*:
 $\bigwedge i. i < \text{rounds} \implies$
wp-event
(checked-staged-security-experiment-with-query-prefix-data-state A i)
(checked-staged-security-with-query-prefix-current-prefix-authenticated-hit-at i)
adversary-initial-state $\leq P\ i$
and *trace-path-bound*:
 $\bigwedge i. i < \text{rounds} \implies$
wp-event
(checked-staged-security-experiment-with-query-prefix-data-state A i)
(checked-staged-security-with-query-prefix-current-trace-path-output-hit-at i)
adversary-initial-state $\leq T\ i$
and *composition-path-bound*:
 $\bigwedge i. i < \text{rounds} \implies$
wp-event
(checked-staged-security-experiment-with-query-prefix-data-state A i)
(checked-staged-security-with-query-prefix-current-composition-path-output-hit-at i)
adversary-initial-state $\leq C\ i$
and *not-prefix-bound*:
wp-event
(checked-staged-security-experiment-with-actual-alpha-prefix-data-state A)
(checked-staged-security-with-actual-alpha-prefix-partial-header-candidate-not-prefix-bound composition-trace-bad-alpha-space)
adversary-initial-state $\leq N$
and *empty-trace-fri-bound*:
wp-event (checked-staged-security-experiment-with-data-state A)
(staged-security-with-data-state-verifier-event trace-fri-bad-with-empty-composition-header-candidates)
adversary-initial-state $\leq \text{trace-fri-error}$
and *empty-query-bound*:
wp-event (checked-staged-security-experiment-with-data-state A)
(staged-security-with-data-state-verifier-event query-bad-with-empty-composition-header-candidates)
adversary-initial-state $\leq \text{nnreal rounds} * \text{query-error-bound}$
shows *checked-staged-adversary-acceptance-probability A* $\leq$
hash-collision-budget-value 0
(staged-attacker-query-budget budgets +
staged-challenge-query-budget + verifier-hash-query-budget) +
(composition-error-bound +
staged-phase-relation-error size
(staged-alpha-search-queries budgets 0) +
hash-collision-budget-value 0
(staged-alpha-search-queries budgets 0) +
N +
staged-phase-relation-error size
(staged-alpha-search-queries budgets 0)) +

$\text{trace-fri-error}' + \text{composition-fri-error}' +$
 $(\sum i < \text{rounds}. P\ i + T\ i + C\ i) +$
 $(\text{composition-error-bound} +$
 $\text{staged-phase-relation-error size}$
 $(\text{staged-alpha-search-queries budgets } 0) +$
 $\text{hash-collision-budget-value } 0$
 $(\text{staged-alpha-search-queries budgets } 0) +$
 $N +$
 $\text{staged-phase-relation-error size}$
 $(\text{staged-alpha-search-queries budgets } 0) +$
 $\text{trace-fri-error} + \text{nnreal rounds} * \text{query-error-bound})$

proof (rule stark-soundness-from-aligned-partial-query-prefix-current-rounds
 $[OF\ \text{false-statement}\ wf\ \text{controlled}\ \text{trace-fri-bound}\ \text{comp-fri-bound}]$)

fix i

assume $i\text{-bound}: i < \text{rounds}$

show

$wp\text{-event}$
 $(\text{checked-staged-security-experiment-with-query-prefix-data-state } A\ i)$
 $(\text{checked-staged-security-with-query-prefix-current-query-partial-opening-hit-at } i)$
 $\text{adversary-initial-state} \leq P\ i + T\ i + C\ i$

by (rule
 $\text{checked-staged-security-with-query-prefix-current-query-partial-opening-hit-at-bound-from-prefix-and-structured-p}$
 $[OF\ wf\ \text{controlled}\ i\text{-bound}\ \text{prefix-bound}[OF\ i\text{-bound}]$
 $\text{trace-path-bound}[OF\ i\text{-bound}]\ \text{composition-path-bound}[OF\ i\text{-bound}]]$)

qed (rule not-prefix-bound empty-trace-fri-bound empty-query-bound)+

theorem stark-soundness-from-aligned-partial-query-prefix-dynamic-target-structured-paths:

fixes $\text{trace-fri-error}'\ \text{composition-fri-error}'\ N :: \text{prob}$

and $P\ T\ C :: \text{nat} \Rightarrow \text{prob}$

assumes $\text{false-statement}: \neg \text{exists-valid-trace}$

and $wf: \text{staged-budget-wellformed budgets}$

and $\text{controlled}: \text{staged-adversary-controlled budgets } A$

and $\text{trace-fri-bound}:$
 $wp\text{-event } (\text{checked-staged-security-experiment-with-data-state } A)$
 $(\text{staged-security-with-data-state-verifier-event}$
 $\text{trace-fri-bad-with-partial-candidates})$
 $\text{adversary-initial-state} \leq \text{trace-fri-error}'$

and $\text{comp-fri-bound}:$
 $wp\text{-event } (\text{checked-staged-security-experiment-with-data-state } A)$
 $(\text{staged-security-with-data-state-verifier-event}$
 $\text{composition-fri-bad-with-partial-candidates})$
 $\text{adversary-initial-state} \leq \text{composition-fri-error}'$

and $\text{prefix-target-bound}:$
 $\bigwedge i. i < \text{rounds} \implies$
 $wp\text{-event}$
 $(\text{checked-staged-query-prefix-receive-with-state } A\ i)$
 $(\text{checked-staged-query-prefix-dynamic-index-hit}$
 $\text{staged-query-prefix-prefix-authenticated-opening-target})$
 $\text{adversary-initial-state} \leq P\ i$

and *trace-path-bound*:
 $\bigwedge i. i < \text{rounds} \implies$
 wp-event
 $(\text{checked-staged-security-experiment-with-query-prefix-data-state } A \ i)$
 $(\text{checked-staged-security-with-query-prefix-current-trace-path-output-hit-at } i)$
 $\text{adversary-initial-state} \leq T \ i$
and *composition-path-bound*:
 $\bigwedge i. i < \text{rounds} \implies$
 wp-event
 $(\text{checked-staged-security-experiment-with-query-prefix-data-state } A \ i)$
 $(\text{checked-staged-security-with-query-prefix-current-composition-path-output-hit-at } i)$
 $\text{adversary-initial-state} \leq C \ i$
and *not-prefix-bound*:
 wp-event
 $(\text{checked-staged-security-experiment-with-actual-alpha-prefix-data-state } A)$
 $(\text{checked-staged-security-with-actual-alpha-prefix-partial-header-candidate-not-prefix-bound composition-trace-bad-alpha-space})$
 $\text{adversary-initial-state} \leq N$
and *empty-trace-fri-bound*:
 $\text{wp-event } (\text{checked-staged-security-experiment-with-data-state } A)$
 $(\text{staged-security-with-data-state-verifier-event trace-fri-bad-with-empty-composition-header-candidates})$
 $\text{adversary-initial-state} \leq \text{trace-fri-error}$
and *empty-query-bound*:
 $\text{wp-event } (\text{checked-staged-security-experiment-with-data-state } A)$
 $(\text{staged-security-with-data-state-verifier-event query-bad-with-empty-composition-header-candidates})$
 $\text{adversary-initial-state} \leq \text{nnreal rounds} * \text{query-error-bound}$
shows $\text{checked-staged-adversary-acceptance-probability } A \leq$
 $\text{hash-collision-budget-value } 0$
 $(\text{staged-attacker-query-budget budgets} +$
 $\text{staged-challenge-query-budget} + \text{verifier-hash-query-budget}) +$
 $(\text{composition-error-bound} +$
 $\text{staged-phase-relation-error size}$
 $(\text{staged-alpha-search-queries budgets } 0) +$
 $\text{hash-collision-budget-value } 0$
 $(\text{staged-alpha-search-queries budgets } 0) +$
 $N +$
 $\text{staged-phase-relation-error size}$
 $(\text{staged-alpha-search-queries budgets } 0)) +$
 $\text{trace-fri-error}' + \text{composition-fri-error}' +$
 $(\sum i < \text{rounds}. P \ i + T \ i + C \ i) +$
 $(\text{composition-error-bound} +$
 $\text{staged-phase-relation-error size}$
 $(\text{staged-alpha-search-queries budgets } 0) +$
 $\text{hash-collision-budget-value } 0$
 $(\text{staged-alpha-search-queries budgets } 0) +$
 $N +$

$\text{staged-phase-relation-error size}$
 $(\text{staged-alpha-search-queries budgets } 0) +$
 $\text{trace-fri-error} + \text{nnreal rounds} * \text{query-error-bound}$

proof –

have *prefix-bound*:

$\bigwedge i. i < \text{rounds} \implies$

wp-event

$(\text{checked-staged-security-experiment-with-query-prefix-data-state } A \ i)$

$(\text{checked-staged-security-with-query-prefix-current-prefix-authenticated-hit-at } i)$

$\text{adversary-initial-state} \leq P \ i$

by (rule

checked-staged-security-with-query-prefix-current-prefix-authenticated-hit-at-bound-from-dynamic-target

$[\text{OF } \text{wf controlled} - \text{prefix-target-bound}])$

show *?thesis*

by (rule

stark-soundness-from-aligned-partial-query-prefix-current-prefix-structured-paths

$[\text{OF } \text{false-statement wf controlled trace-fri-bound comp-fri-bound}$

$\text{prefix-bound trace-path-bound composition-path-bound not-prefix-bound}$

$\text{empty-trace-fri-bound empty-query-bound}])$

qed

theorem *stark-soundness-from-aligned-partial-query-prefix-trace-index-structured-paths*:

fixes *trace-fri-error' composition-fri-error' N :: prob*

and *T C :: nat $\Rightarrow$ prob*

assumes *false-statement: $\neg$ exists-valid-trace*

and *wf: staged-budget-wellformed budgets*

and *controlled: staged-adversary-controlled budgets A*

and *trace-fri-bound*:

wp-event $(\text{checked-staged-security-experiment-with-data-state } A)$

$(\text{staged-security-with-data-state-verifier-event}$

$\text{trace-fri-bad-with-partial-candidates})$

$\text{adversary-initial-state} \leq \text{trace-fri-error}'$

and *comp-fri-bound*:

wp-event $(\text{checked-staged-security-experiment-with-data-state } A)$

$(\text{staged-security-with-data-state-verifier-event}$

$\text{composition-fri-bad-with-partial-candidates})$

$\text{adversary-initial-state} \leq \text{composition-fri-error}'$

and *trace-frac*:

$\bigwedge i \text{ prefix prefix-state.}$

$i < \text{rounds} \implies$

$\text{Some } ((\text{prefix}, \text{prefix-state}), \text{prefix-state}) \in$

set-dist

$(\text{execute } (\text{checked-staged-query-prefix-with-state } A \ i)$

$\text{adversary-initial-state}) \implies$

nnreal

$(\text{card}$

$(\text{staged-query-prefix-prefix-authenticated-trace-indices prefix}$

$\text{prefix-state})) /$

$\text{nnreal } (\text{card } \text{query-sample-space}) \leq \text{query-error-bound}$

and *trace-path-bound*:
 $\bigwedge i. i < \text{rounds} \implies$
 wp-event
 $(\text{checked-staged-security-experiment-with-query-prefix-data-state } A \ i)$
 $(\text{checked-staged-security-with-query-prefix-current-trace-path-output-hit-at } i)$
 $\text{adversary-initial-state} \leq T \ i$
and *composition-path-bound*:
 $\bigwedge i. i < \text{rounds} \implies$
 wp-event
 $(\text{checked-staged-security-experiment-with-query-prefix-data-state } A \ i)$
 $(\text{checked-staged-security-with-query-prefix-current-composition-path-output-hit-at } i)$
 $\text{adversary-initial-state} \leq C \ i$
and *not-prefix-bound*:
 wp-event
 $(\text{checked-staged-security-experiment-with-actual-alpha-prefix-data-state } A)$
 $(\text{checked-staged-security-with-actual-alpha-prefix-partial-header-candidate-not-prefix-bound composition-trace-bad-alpha-space})$
 $\text{adversary-initial-state} \leq N$
and *empty-trace-fri-bound*:
 $\text{wp-event } (\text{checked-staged-security-experiment-with-data-state } A)$
 $(\text{staged-security-with-data-state-verifier-event trace-fri-bad-with-empty-composition-header-candidates})$
 $\text{adversary-initial-state} \leq \text{trace-fri-error}$
and *empty-query-bound*:
 $\text{wp-event } (\text{checked-staged-security-experiment-with-data-state } A)$
 $(\text{staged-security-with-data-state-verifier-event query-bad-with-empty-composition-header-candidates})$
 $\text{adversary-initial-state} \leq \text{nnreal rounds} * \text{query-error-bound}$
shows $\text{checked-staged-adversary-acceptance-probability } A \leq$
 $\text{hash-collision-budget-value } 0$
 $(\text{staged-attacker-query-budget budgets} +$
 $\text{staged-challenge-query-budget} + \text{verifier-hash-query-budget}) +$
 $(\text{composition-error-bound} +$
 $\text{staged-phase-relation-error size}$
 $(\text{staged-alpha-search-queries budgets } 0) +$
 $\text{hash-collision-budget-value } 0$
 $(\text{staged-alpha-search-queries budgets } 0) +$
 $N +$
 $\text{staged-phase-relation-error size}$
 $(\text{staged-alpha-search-queries budgets } 0)) +$
 $\text{trace-fri-error}' + \text{composition-fri-error}' +$
 $(\sum i < \text{rounds}.$
 $\text{staged-phase-relation-error size}$
 $(\text{staged-query-search-queries budgets } i + 1) +$
 $\text{query-error-bound} + T \ i + C \ i) +$
 $(\text{composition-error-bound} +$
 $\text{staged-phase-relation-error size}$
 $(\text{staged-alpha-search-queries budgets } 0) +$

hash-collision-budget-value 0
(staged-alpha-search-queries budgets 0) +
N +
staged-phase-relation-error size
(staged-alpha-search-queries budgets 0) +
*trace-fri-error + nreal rounds * query-error-bound)*

proof (rule
stark-soundness-from-aligned-partial-query-prefix-dynamic-target-structured-paths
[OF false-statement wf controlled trace-fri-bound comp-fri-bound - trace-path-bound
composition-path-bound not-prefix-bound empty-trace-fri-bound empty-query-bound])
fix *i*
assume *i-bound: i < rounds*
show
wp-event (checked-staged-query-prefix-receive-with-state A i)
(checked-staged-query-prefix-dynamic-index-hit
staged-query-prefix-prefix-authenticated-opening-target)
adversary-initial-state ≤
staged-phase-relation-error size
(staged-query-search-queries budgets i + 1) + query-error-bound
by (rule
checked-staged-query-prefix-prefix-authenticated-target-hit-bound-from-trace-indices
[OF wf controlled i-bound])
(use trace-frac[OF i-bound] in blast)
qed

theorem *stark-soundness-from-aligned-partial-query-prefix-conceptual-structured-paths:*
fixes *trace-fri-error' composition-fri-error' N :: prob*
and *T C :: nat ⇒ prob*
assumes *false-statement: ¬ exists-valid-trace*
and *wf: staged-budget-wellformed budgets*
and *controlled: staged-adversary-controlled budgets A*
and *trace-fri-bound:*
wp-event (checked-staged-security-experiment-with-data-state A)
(staged-security-with-data-state-verifier-event
trace-fri-bad-with-partial-candidates)
adversary-initial-state ≤ trace-fri-error'
and *comp-fri-bound:*
wp-event (checked-staged-security-experiment-with-data-state A)
(staged-security-with-data-state-verifier-event
composition-fri-bad-with-partial-candidates)
adversary-initial-state ≤ composition-fri-error'
and *conceptual-context:*
 $\bigwedge i$ *prefix prefix-state.*
 $i < rounds \implies$
Some ((prefix, prefix-state), prefix-state) ∈
set-dist
(execute (checked-staged-query-prefix-with-state A i)
adversary-initial-state) ⇒
 $\neg$ *hash-map-output-collision prefix-state ∧*

trace-table-low-degree
 (query-prefix-trace-conceptual-table prefix prefix-state) $\wedge$
composition-table-low-degree maxDegree
 (query-prefix-composition-conceptual-table prefix prefix-state) $\wedge$
 $\neg$ all-queries-consistent
 (query-prefix-trace-conceptual-table prefix prefix-state)
 (query-prefix-composition-conceptual-table prefix prefix-state)
 (sqp-alphas prefix)
and trace-path-bound:
 $\bigwedge i. i < \text{rounds} \implies$
 wp-event
 (checked-staged-security-experiment-with-query-prefix-data-state A i)
 (checked-staged-security-with-query-prefix-current-trace-path-output-hit-at i)
 adversary-initial-state $\leq T$ i
and composition-path-bound:
 $\bigwedge i. i < \text{rounds} \implies$
 wp-event
 (checked-staged-security-experiment-with-query-prefix-data-state A i)
 (checked-staged-security-with-query-prefix-current-composition-path-output-hit-at i)
 adversary-initial-state $\leq C$ i
and not-prefix-bound:
 wp-event
 (checked-staged-security-experiment-with-actual-alpha-prefix-data-state A)
 (checked-staged-security-with-actual-alpha-prefix-partial-header-candidate-not-prefix-bound
 composition-trace-bad-alpha-space)
 adversary-initial-state $\leq N$
and empty-trace-fri-bound:
 wp-event (checked-staged-security-experiment-with-data-state A)
 (staged-security-with-data-state-verifier-event
 trace-fri-bad-with-empty-composition-header-candidates)
 adversary-initial-state $\leq$ trace-fri-error
and empty-query-bound:
 wp-event (checked-staged-security-experiment-with-data-state A)
 (staged-security-with-data-state-verifier-event
 query-bad-with-empty-composition-header-candidates)
 adversary-initial-state $\leq$ nnreal rounds * query-error-bound
shows checked-staged-adversary-acceptance-probability A $\leq$
 hash-collision-budget-value 0
 (staged-attacker-query-budget budgets +
 staged-challenge-query-budget + verifier-hash-query-budget) +
 (composition-error-bound +
 staged-phase-relation-error size
 (staged-alpha-search-queries budgets 0) +
 hash-collision-budget-value 0
 (staged-alpha-search-queries budgets 0) +
 N +
 staged-phase-relation-error size
 (staged-alpha-search-queries budgets 0)) +

$\text{trace-fri-error}' + \text{composition-fri-error}' +$
 $(\sum i < \text{rounds}.$
 $\quad \text{staged-phase-relation-error size}$
 $\quad (\text{staged-query-search-queries budgets } i + 1) +$
 $\quad \text{query-error-bound} + T i + C i) +$
 $(\text{composition-error-bound} +$
 $\quad \text{staged-phase-relation-error size}$
 $\quad (\text{staged-alpha-search-queries budgets } 0) +$
 $\quad \text{hash-collision-budget-value } 0$
 $\quad (\text{staged-alpha-search-queries budgets } 0) +$
 $\quad N +$
 $\quad \text{staged-phase-relation-error size}$
 $\quad (\text{staged-alpha-search-queries budgets } 0) +$
 $\quad \text{trace-fri-error} + \text{nnreal rounds} * \text{query-error-bound})$

proof (rule
 $\text{stark-soundness-from-aligned-partial-query-prefix-dynamic-target-structured-paths}$
 $[OF \text{ false-statement } wf \text{ controlled } \text{trace-fri-bound} \text{ comp-fri-bound} - \text{trace-path-bound}$
 $\text{composition-path-bound not-prefix-bound empty-trace-fri-bound empty-query-bound}]$)

fix i
assume $i\text{-bound}: i < \text{rounds}$
show
 $wp\text{-event } (\text{checked-staged-query-prefix-receive-with-state } A \ i)$
 $(\text{checked-staged-query-prefix-dynamic-index-hit}$
 $\quad \text{staged-query-prefix-prefix-authenticated-opening-target})$
 $\text{adversary-initial-state} \leq$
 $\text{staged-phase-relation-error size}$
 $(\text{staged-query-search-queries budgets } i + 1) + \text{query-error-bound}$

by (rule
 $\text{checked-staged-query-prefix-prefix-authenticated-target-hit-bound-from-conceptual}$
 $[OF \ wf \ \text{controlled } i\text{-bound}]$)
 $(\text{rule conceptual-context}[OF \ i\text{-bound}])$

qed

theorem $\text{stark-soundness-from-aligned-partial-query-prefix-conceptual-default-structured-paths}$:
fixes $\text{trace-fri-error}' \ \text{composition-fri-error}' \ N :: \text{prob}$
and $T \ C :: \text{nat} \Rightarrow \text{prob}$
and $\text{trace-default} \ \text{composition-default} :: 'f \ \text{list}$
assumes $\text{false-statement}: \neg \text{exists-valid-trace}$
and wf : $\text{staged-budget-wellformed budgets}$
and controlled : $\text{staged-adversary-controlled budgets } A$
and trace-fri-bound :
 $wp\text{-event } (\text{checked-staged-security-experiment-with-data-state } A)$
 $(\text{staged-security-with-data-state-verifier-event}$
 $\quad \text{trace-fri-bad-with-partial-candidates})$
 $\text{adversary-initial-state} \leq \text{trace-fri-error}'$
and comp-fri-bound :
 $wp\text{-event } (\text{checked-staged-security-experiment-with-data-state } A)$
 $(\text{staged-security-with-data-state-verifier-event}$
 $\quad \text{composition-fri-bad-with-partial-candidates})$

$adversary-initial-state \leq composition-fri-error'$
and *default-context*:
 $\bigwedge i \text{ prefix prefix-state.}$
 $i < rounds \implies$
 $Some ((prefix, prefix-state), prefix-state) \in$
 $set-dist$
 $(execute (checked-staged-query-prefix-with-state A i)$
 $adversary-initial-state) \implies$
 $trace-table-low-degree$
 $(query-prefix-trace-conceptual-table-with-default prefix$
 $prefix-state trace-default) \wedge$
 $composition-table-low-degree maxDegree$
 $(query-prefix-composition-conceptual-table-with-default prefix$
 $prefix-state composition-default) \wedge$
 $\neg all-queries-consistent$
 $(query-prefix-trace-conceptual-table-with-default prefix$
 $prefix-state trace-default)$
 $(query-prefix-composition-conceptual-table-with-default prefix$
 $prefix-state composition-default)$
 $(sqp-alphas prefix)$
and *trace-path-bound*:
 $\bigwedge i. i < rounds \implies$
 $wp-event$
 $(checked-staged-security-experiment-with-query-prefix-data-state A i)$
 $(checked-staged-security-with-query-prefix-current-trace-path-output-hit-at i)$
 $adversary-initial-state \leq T i$
and *composition-path-bound*:
 $\bigwedge i. i < rounds \implies$
 $wp-event$
 $(checked-staged-security-experiment-with-query-prefix-data-state A i)$
 $(checked-staged-security-with-query-prefix-current-composition-path-output-hit-at i)$
 $adversary-initial-state \leq C i$
and *not-prefix-bound*:
 $wp-event$
 $(checked-staged-security-experiment-with-actual-alpha-prefix-data-state$
 $A)$
 $(checked-staged-security-with-actual-alpha-prefix-partial-header-candidate-not-prefix-bound$
 $composition-trace-bad-alpha-space)$
 $adversary-initial-state \leq N$
and *empty-trace-fri-bound*:
 $wp-event (checked-staged-security-experiment-with-data-state A)$
 $(staged-security-with-data-state-verifier-event$
 $trace-fri-bad-with-empty-composition-header-candidates)$
 $adversary-initial-state \leq trace-fri-error$
and *empty-query-bound*:
 $wp-event (checked-staged-security-experiment-with-data-state A)$
 $(staged-security-with-data-state-verifier-event$
 $query-bad-with-empty-composition-header-candidates)$
 $adversary-initial-state \leq nnreal rounds * query-error-bound$

shows *checked-staged-adversary-acceptance-probability* $A \leq$
hash-collision-budget-value 0
 (*staged-attacker-query-budget* *budgets* +
staged-challenge-query-budget + *verifier-hash-query-budget*) +
 (*composition-error-bound* +
staged-phase-relation-error *size*
 (*staged-alpha-search-queries* *budgets* 0) +
hash-collision-budget-value 0
 (*staged-alpha-search-queries* *budgets* 0) +
N +
staged-phase-relation-error *size*
 (*staged-alpha-search-queries* *budgets* 0)) +
trace-fri-error' + *composition-fri-error'* +
 ($\sum i < \text{rounds}$.
staged-phase-relation-error *size*
 (*staged-query-search-queries* *budgets* $i + 1$) +
query-error-bound +
hash-collision-budget-value 0
 (*staged-query-search-queries* *budgets* $i + 1$) +
T i + *C i*) +
 (*composition-error-bound* +
staged-phase-relation-error *size*
 (*staged-alpha-search-queries* *budgets* 0) +
hash-collision-budget-value 0
 (*staged-alpha-search-queries* *budgets* 0) +
N +
staged-phase-relation-error *size*
 (*staged-alpha-search-queries* *budgets* 0) +
trace-fri-error + *nnreal rounds* * *query-error-bound*)
proof (*rule stark-soundness-from-aligned-partial-query-prefix-current-rounds*
 [*OF false-statement* *wf controlled trace-fri-bound comp-fri-bound - not-prefix-bound*
empty-trace-fri-bound empty-query-bound])
fix *i*
assume *i-bound*: $i < \text{rounds}$
show
wp-event
 (*checked-staged-security-experiment-with-query-prefix-data-state* *A i*)
 (*checked-staged-security-with-query-prefix-current-query-partial-opening-hit-at* *i*)
adversary-initial-state $\leq$
staged-phase-relation-error *size*
 (*staged-query-search-queries* *budgets* $i + 1$) +
query-error-bound +
hash-collision-budget-value 0
 (*staged-query-search-queries* *budgets* $i + 1$) +
T i + *C i*
by (*rule*
checked-staged-security-with-query-prefix-current-query-partial-opening-hit-at-bound-from-conceptual-default-cont
 [*OF wf controlled i-bound - trace-path-bound* [*OF i-bound*]
composition-path-bound [*OF i-bound*]])

```

      (rule default-context[OF i-bound])
qed
end
end

```

```

theory Soundness-Conceptual-Query-Current-Alpha-Clean
  imports Soundness-Staged-Aligned-Current-Query-Path
begin

```

Alpha-clean wrappers for the default-backed conceptual current-query route.

This layer derives the default-context non-consistency premise from the false statement, low degree of the fixed default tables, and the fact that the sampled alpha vector is not in the corresponding composition bad set.

```

context soundness
begin

```

lemma *checked-staged-security-with-query-prefix-conceptual-default-context-from-alpha-clean:*

```

assumes false-statement:  $\neg$  exists-valid-trace
and alphas-length:  $\text{length } (\text{sqp-alphas prefix}) = \text{length spec}$ 
and trace-low:
  trace-table-low-degree
  (query-prefix-trace-conceptual-table-with-default prefix prefix-state
   trace-default)
and composition-low:
  composition-table-low-degree maxDegree
  (query-prefix-composition-conceptual-table-with-default prefix
   prefix-state composition-default)
and alpha-clean:
  sqp-alphas prefix  $\notin$  composition-trace-bad-alpha-space
  (query-prefix-trace-conceptual-table-with-default prefix prefix-state
   trace-default)

```

shows

```

  trace-table-low-degree
  (query-prefix-trace-conceptual-table-with-default prefix prefix-state
   trace-default)  $\wedge$ 
  composition-table-low-degree maxDegree
  (query-prefix-composition-conceptual-table-with-default prefix
   prefix-state composition-default)  $\wedge$ 
 $\neg$  all-queries-consistent
  (query-prefix-trace-conceptual-table-with-default prefix prefix-state
   trace-default)
  (query-prefix-composition-conceptual-table-with-default prefix
   prefix-state composition-default)
  (sqp-alphas prefix)

```

```

proof (intro conjI trace-low composition-low notI)
  assume all-queries:

```

```

all-queries-consistent
  (query-prefix-trace-conceptual-table-with-default prefix prefix-state
   trace-default)
  (query-prefix-composition-conceptual-table-with-default prefix
   prefix-state composition-default)
  (sqp-alphas prefix)
have alpha-bad:
  sqp-alphas prefix  $\in$  composition-trace-bad-alpha-space
  (query-prefix-trace-conceptual-table-with-default prefix prefix-state
   trace-default)
  by (rule low-degree-all-queries-consistent-imp-composition-trace-bad-alpha-space
   [OF false-statement alphas-length trace-low composition-low all-queries])
show False
  using alpha-clean alpha-bad by simp
qed

lemma checked-staged-security-with-query-prefix-conceptual-default-context-residual-hit-at-bound-from-alpha-clean:
assumes false-statement:  $\neg$  exists-valid-trace
and wf: staged-budget-wellformed budgets
and controlled: staged-adversary-controlled budgets A
and i-bound: i < rounds
and default-alpha-clean:
   $\bigwedge$  prefix prefix-state.
  Some ((prefix, prefix-state), prefix-state)  $\in$ 
    set-dist
    (execute (checked-staged-query-prefix-with-state A i)
     adversary-initial-state)  $\implies$ 
    trace-table-low-degree
    (query-prefix-trace-conceptual-table-with-default prefix
     prefix-state trace-default)  $\wedge$ 
    composition-table-low-degree maxDegree
    (query-prefix-composition-conceptual-table-with-default prefix
     prefix-state composition-default)  $\wedge$ 
    sqp-alphas prefix  $\notin$  composition-trace-bad-alpha-space
    (query-prefix-trace-conceptual-table-with-default prefix
     prefix-state trace-default)
shows
  wp-event
  (checked-staged-security-experiment-with-query-prefix-data-state A i)
  (checked-staged-security-with-query-prefix-conceptual-default-context-residual-hit-at
   trace-default composition-default i)
  adversary-initial-state  $\leq$ 
  hash-collision-budget-value 0
  (staged-query-search-queries budgets i + 1)
proof (rule
  checked-staged-security-with-query-prefix-conceptual-default-context-residual-hit-at-bound-from-context
  [OF wf controlled i-bound])
fix prefix prefix-state
assume prefix-support:

```

Some ((*prefix*, *prefix-state*), *prefix-state*) ∈
set-dist
 (execute (*checked-staged-query-prefix-with-state* *A i*)
adversary-initial-state)
from *default-alpha-clean*[*OF prefix-support*] **have** *trace-low*:
trace-table-low-degree
 (query-prefix-trace-conceptual-table-with-default *prefix*
prefix-state trace-default)
and *composition-low*:
composition-table-low-degree maxDegree
 (query-prefix-composition-conceptual-table-with-default *prefix*
prefix-state composition-default)
and *alpha-clean*:
sqp-alphas prefix ∉ *composition-trace-bad-alpha-space*
 (query-prefix-trace-conceptual-table-with-default *prefix*
prefix-state trace-default)
by *blast+*
have *alphas-length*: *length (sqp-alphas prefix) = length spec*
by (rule *checked-staged-query-prefix-with-state-alphas-length*
 [*OF prefix-support*])
show
trace-table-low-degree
 (query-prefix-trace-conceptual-table-with-default *prefix prefix-state*
trace-default) ∧
composition-table-low-degree maxDegree
 (query-prefix-composition-conceptual-table-with-default *prefix*
prefix-state composition-default) ∧
 ¬ *all-queries-consistent*
 (query-prefix-trace-conceptual-table-with-default *prefix prefix-state*
trace-default)
 (query-prefix-composition-conceptual-table-with-default *prefix*
prefix-state composition-default)
 (*sqp-alphas prefix*)
by (rule
checked-staged-security-with-query-prefix-conceptual-default-context-from-alpha-clean
 [*OF false-statement alphas-length trace-low composition-low alpha-clean*])
qed

lemma *query-prefix-default-alpha-clean-from-alpha-prefix-union-clean*:
assumes *trace-eq*:
query-prefix-trace-conceptual-table-with-default prefix prefix-state
trace-default = trace-default
and *trace-candidate*:
trace-default ∈
alpha-prefix-trace-table-candidates prefix-state (sqp-trace-root prefix)
and *union-clean*:
sqp-alphas prefix ∉
alpha-prefix-union-bad-sets prefix-state composition-trace-bad-alpha-space
 (*sqp-trace-root prefix*)

shows

sqp-alphas prefix $\notin$ *composition-trace-bad-alpha-space*
 (*query-prefix-trace-conceptual-table-with-default prefix prefix-state*
trace-default)

proof

assume *alpha-bad*:

sqp-alphas prefix $\in$ *composition-trace-bad-alpha-space*
 (*query-prefix-trace-conceptual-table-with-default prefix prefix-state*
trace-default)

have *alpha-space*:

sqp-alphas prefix $\in$ *alpha-space*

using *alpha-bad composition-trace-bad-alpha-space-subset-alpha-space*

by *blast*

have

sqp-alphas prefix $\in$
alpha-prefix-union-bad-sets prefix-state composition-trace-bad-alpha-space
(sqp-trace-root prefix)

unfolding *alpha-prefix-union-bad-sets-def*

using *trace-candidate alpha-bad trace-eq alpha-space* **by** *auto*

then show *False*

using *union-clean* **by** *simp*

qed

lemma *checked-staged-security-with-query-prefix-conceptual-default-context-residual-hit-at-bound-from-alpha-prefix-u*

assumes *false-statement*: $\neg$ *exists-valid-trace*

and *wf*: *staged-budget-wellformed budgets*

and *controlled*: *staged-adversary-controlled budgets A*

and *i-bound*: *i* < *rounds*

and *default-prefix-clean*:

$\bigwedge$ *prefix prefix-state*.

Some ((prefix, prefix-state), prefix-state) $\in$
set-dist

(execute (checked-staged-query-prefix-with-state A i)
adversary-initial-state) $\implies$

query-prefix-trace-conceptual-table-with-default prefix prefix-state
trace-default = trace-default $\wedge$

query-prefix-composition-conceptual-table-with-default prefix
prefix-state composition-default = composition-default $\wedge$

trace-table-low-degree trace-default $\wedge$

composition-table-low-degree maxDegree composition-default $\wedge$
trace-default $\in$

alpha-prefix-trace-table-candidates prefix-state

(sqp-trace-root prefix) $\wedge$

sqp-alphas prefix $\notin$

alpha-prefix-union-bad-sets prefix-state

composition-trace-bad-alpha-space (sqp-trace-root prefix)

shows

wp-event

(checked-staged-security-experiment-with-query-prefix-data-state A i)

(checked-staged-security-with-query-prefix-conceptual-default-context-residual-hit-at
trace-default composition-default i)
adversary-initial-state $\leq$
hash-collision-budget-value 0
(staged-query-search-queries budgets i + 1)
proof (rule
checked-staged-security-with-query-prefix-conceptual-default-context-residual-hit-at-bound-from-alpha-clean
[OF false-statement wf controlled i-bound])
fix *prefix prefix-state*
assume *prefix-support*:
Some ((prefix, prefix-state), prefix-state) ∈
set-dist
(execute (checked-staged-query-prefix-with-state A i)
adversary-initial-state)
from *default-prefix-clean[OF prefix-support]* **have** *trace-eq*:
query-prefix-trace-conceptual-table-with-default prefix prefix-state
trace-default = trace-default
and *composition-eq*:
query-prefix-composition-conceptual-table-with-default prefix
prefix-state composition-default = composition-default
and *trace-low*: *trace-table-low-degree trace-default*
and *composition-low*:
composition-table-low-degree maxDegree composition-default
and *trace-candidate*:
trace-default ∈
alpha-prefix-trace-table-candidates prefix-state
(sqp-trace-root prefix)
and *union-clean*:
sqp-alphas prefix ∉
alpha-prefix-union-bad-sets prefix-state
composition-trace-bad-alpha-space (sqp-trace-root prefix)
by *blast+*
have *alpha-clean*:
sqp-alphas prefix ∉ composition-trace-bad-alpha-space
(query-prefix-trace-conceptual-table-with-default prefix prefix-state
trace-default)
by (rule *query-prefix-default-alpha-clean-from-alpha-prefix-union-clean*
[OF trace-eq trace-candidate union-clean])
show
trace-table-low-degree
(query-prefix-trace-conceptual-table-with-default prefix prefix-state
trace-default) ∧
composition-table-low-degree maxDegree
(query-prefix-composition-conceptual-table-with-default prefix
prefix-state composition-default) ∧
sqp-alphas prefix ∉ composition-trace-bad-alpha-space
(query-prefix-trace-conceptual-table-with-default prefix prefix-state
trace-default)
using *trace-eq composition-eq trace-low composition-low alpha-clean by simp*

qed

lemma *checked-staged-security-with-query-prefix-current-query-partial-opening-hit-at-bound-from-conceptual-default-*

assumes *false-statement*: $\neg \text{exists-valid-trace}$

and *wf*: *staged-budget-wellformed budgets*

and *controlled*: *staged-adversary-controlled budgets A*

and *i-bound*: $i < \text{rounds}$

and *default-alpha-clean*:

$\bigwedge \text{prefix prefix-state.}$

Some ((prefix, prefix-state), prefix-state) $\in$
set-dist

(execute (checked-staged-query-prefix-with-state A i)
adversary-initial-state) $\implies$

trace-table-low-degree

(query-prefix-trace-conceptual-table-with-default prefix
prefix-state trace-default) $\wedge$

composition-table-low-degree maxDegree

(query-prefix-composition-conceptual-table-with-default prefix
prefix-state composition-default) $\wedge$

sqp-alphas prefix $\notin$ composition-trace-bad-alpha-space

(query-prefix-trace-conceptual-table-with-default prefix
prefix-state trace-default)

and *trace-path-bound*:

wp-event

(checked-staged-security-experiment-with-query-prefix-data-state A i)

(checked-staged-security-with-query-prefix-current-trace-path-output-hit-at i)

adversary-initial-state $\leq T$

and *composition-path-bound*:

wp-event

(checked-staged-security-experiment-with-query-prefix-data-state A i)

(checked-staged-security-with-query-prefix-current-composition-path-output-hit-at i)

adversary-initial-state $\leq C$

shows

wp-event

(checked-staged-security-experiment-with-query-prefix-data-state A i)

(checked-staged-security-with-query-prefix-current-query-partial-opening-hit-at i)

adversary-initial-state $\leq$

staged-phase-relation-error size

(staged-query-search-queries budgets i + 1) +

query-error-bound +

hash-collision-budget-value 0

(staged-query-search-queries budgets i + 1) + T + C

proof –

have *residual-bound*:

wp-event

(checked-staged-security-experiment-with-query-prefix-data-state A i)

(checked-staged-security-with-query-prefix-conceptual-default-context-residual-hit-at
trace-default composition-default i)

adversary-initial-state $\leq$

hash-collision-budget-value 0
(staged-query-search-queries budgets $i + 1$)
by (rule
checked-staged-security-with-query-prefix-conceptual-default-context-residual-hit-at-bound-from-alpha-clean
[OF false-statement wf controlled i-bound default-alpha-clean])
have bound:
wp-event
(checked-staged-security-experiment-with-query-prefix-data-state A i)
(checked-staged-security-with-query-prefix-current-query-partial-opening-hit-at i)
adversary-initial-state $\leq$
staged-phase-relation-error size
(staged-query-search-queries budgets $i + 1$) +
query-error-bound +
hash-collision-budget-value 0
(staged-query-search-queries budgets $i + 1$) + T + C
by (rule
checked-staged-security-with-query-prefix-current-query-partial-opening-hit-at-bound-from-conceptual-default-resi
[OF wf controlled i-bound residual-bound trace-path-bound
composition-path-bound])
show ?thesis
using bound **by** simp
qed

lemma *checked-staged-security-with-query-prefix-current-query-partial-opening-hit-at-bound-from-conceptual-default-*

assumes *false-statement: $\neg$ exists-valid-trace*

and *wf: staged-budget-wellformed budgets*

and *controlled: staged-adversary-controlled budgets A*

and *i-bound: $i < \text{rounds}$*

and *default-prefix-clean:*

$\bigwedge \text{prefix prefix-state.}$

Some ((prefix, prefix-state), prefix-state) $\in$

set-dist

(execute (checked-staged-query-prefix-with-state A i)

adversary-initial-state) $\implies$

query-prefix-trace-conceptual-table-with-default prefix prefix-state

trace-default = trace-default $\wedge$

query-prefix-composition-conceptual-table-with-default prefix

prefix-state composition-default = composition-default $\wedge$

trace-table-low-degree trace-default $\wedge$

composition-table-low-degree maxDegree composition-default $\wedge$

trace-default $\in$

alpha-prefix-trace-table-candidates prefix-state

(sqp-trace-root prefix) $\wedge$

sqp-alphas prefix $\notin$

alpha-prefix-union-bad-sets prefix-state

composition-trace-bad-alpha-space (sqp-trace-root prefix)

and *trace-path-bound:*

wp-event

(checked-staged-security-experiment-with-query-prefix-data-state A i)

(checked-staged-security-with-query-prefix-current-trace-path-output-hit-at i)
 adversary-initial-state $\leq T$
and composition-path-bound:
 wp-event
 (checked-staged-security-experiment-with-query-prefix-data-state A i)
 (checked-staged-security-with-query-prefix-current-composition-path-output-hit-at i)
 adversary-initial-state $\leq C$
shows
 wp-event
 (checked-staged-security-experiment-with-query-prefix-data-state A i)
 (checked-staged-security-with-query-prefix-current-query-partial-opening-hit-at i)
 adversary-initial-state $\leq$
 staged-phase-relation-error size
 (staged-query-search-queries budgets $i + 1$) +
 query-error-bound +
 hash-collision-budget-value 0
 (staged-query-search-queries budgets $i + 1$) + $T + C$
proof –
have residual-bound:
 wp-event
 (checked-staged-security-experiment-with-query-prefix-data-state A i)
 (checked-staged-security-with-query-prefix-conceptual-default-context-residual-hit-at
 trace-default composition-default i)
 adversary-initial-state $\leq$
 hash-collision-budget-value 0
 (staged-query-search-queries budgets $i + 1$)
by (rule
 checked-staged-security-with-query-prefix-conceptual-default-context-residual-hit-at-bound-from-alpha-prefix-union
 [OF false-statement wf controlled i -bound default-prefix-clean])
have bound:
 wp-event
 (checked-staged-security-experiment-with-query-prefix-data-state A i)
 (checked-staged-security-with-query-prefix-current-query-partial-opening-hit-at i)
 adversary-initial-state $\leq$
 staged-phase-relation-error size
 (staged-query-search-queries budgets $i + 1$) +
 query-error-bound +
 hash-collision-budget-value 0
 (staged-query-search-queries budgets $i + 1$) + $T + C$
by (rule
 checked-staged-security-with-query-prefix-current-query-partial-opening-hit-at-bound-from-conceptual-default-residual
 [OF wf controlled i -bound residual-bound trace-path-bound
 composition-path-bound])
show ?thesis
using bound **by** simp
qed

lemma checked-staged-security-with-query-prefix-authenticated-candidate-hit-at-bound-from-conceptual-default-alpha-
assumes false-statement: $\neg$ exists-valid-trace

and *wf*: staged-budget-wellformed budgets
and *controlled*: staged-adversary-controlled budgets *A*
and *i-bound*: $i < \text{rounds}$
and *default-alpha-clean*:
 $\bigwedge \text{prefix prefix-state.}$
Some $((\text{prefix}, \text{prefix-state}), \text{prefix-state}) \in$
set-dist
 $(\text{execute } (\text{checked-staged-query-prefix-with-state } A \ i)$
 $\text{adversary-initial-state}) \implies$
trace-table-low-degree
 $(\text{query-prefix-trace-conceptual-table-with-default prefix}$
 $\text{prefix-state trace-table}) \wedge$
composition-table-low-degree maxDegree
 $(\text{query-prefix-composition-conceptual-table-with-default prefix}$
 $\text{prefix-state composition-table}) \wedge$
 $\text{sqp-alphas prefix} \notin \text{composition-trace-bad-alpha-space}$
 $(\text{query-prefix-trace-conceptual-table-with-default prefix}$
 $\text{prefix-state trace-table})$
and *trace-path-bound*:
wp-event
 $(\text{checked-staged-security-experiment-with-query-prefix-data-state } A \ i)$
 $(\text{checked-staged-security-with-query-prefix-current-trace-path-output-hit-at } i)$
 $\text{adversary-initial-state} \leq T$
and *composition-path-bound*:
wp-event
 $(\text{checked-staged-security-experiment-with-query-prefix-data-state } A \ i)$
 $(\text{checked-staged-security-with-query-prefix-current-composition-path-output-hit-at } i)$
 $\text{adversary-initial-state} \leq C$
shows
wp-event
 $(\text{checked-staged-security-experiment-with-query-prefix-data-state } A \ i)$
 $(\text{checked-staged-security-with-query-prefix-authenticated-candidate-hit-at}$
 $\text{trace-table composition-table } i)$
 $\text{adversary-initial-state} \leq$
 $\text{staged-phase-relation-error size}$
 $(\text{staged-query-search-queries budgets } i + 1) +$
 $\text{query-error-bound} +$
 $\text{hash-collision-budget-value } 0$
 $(\text{staged-query-search-queries budgets } i + 1) + T + C$
proof –
have *residual-bound*:
wp-event
 $(\text{checked-staged-security-experiment-with-query-prefix-data-state } A \ i)$
 $(\text{checked-staged-security-with-query-prefix-conceptual-default-context-residual-hit-at}$
 $\text{trace-table composition-table } i)$
 $\text{adversary-initial-state} \leq$
 $\text{hash-collision-budget-value } 0$
 $(\text{staged-query-search-queries budgets } i + 1)$
by (*rule*

checked-staged-security-with-query-prefix-conceptual-default-context-residual-hit-at-bound-from-alpha-clean
 [OF false-statement wf controlled i-bound default-alpha-clean])
have bound:
wp-event
 (checked-staged-security-experiment-with-query-prefix-data-state A i)
 (checked-staged-security-with-query-prefix-authenticated-candidate-hit-at
 trace-table composition-table i)
 adversary-initial-state $\leq$
 staged-phase-relation-error size
 (staged-query-search-queries budgets i + 1) +
 query-error-bound +
 hash-collision-budget-value 0
 (staged-query-search-queries budgets i + 1) + T + C
by (rule
 checked-staged-security-with-query-prefix-authenticated-candidate-hit-at-bound-from-conceptual-default-residual-a
 [OF wf controlled i-bound residual-bound trace-path-bound
 composition-path-bound])
show ?thesis
 using bound by simp
qed

lemma staged-security-with-data-state-current-query-partial-candidate-hit-at-bound-from-conceptual-default-alpha-cl
assumes false-statement: $\neg$ exists-valid-trace
and wf: staged-budget-wellformed budgets
and controlled: staged-adversary-controlled budgets A
and i-bound: i < rounds
and default-alpha-clean:
 $\bigwedge$ prefix prefix-state.
 Some ((prefix, prefix-state), prefix-state) $\in$
 set-dist
 (execute (checked-staged-query-prefix-with-state A i)
 adversary-initial-state) $\implies$
 trace-table-low-degree
 (query-prefix-trace-conceptual-table-with-default prefix
 prefix-state trace-table) $\wedge$
 composition-table-low-degree maxDegree
 (query-prefix-composition-conceptual-table-with-default prefix
 prefix-state composition-table) $\wedge$
 sqp-alphas prefix $\notin$ composition-trace-bad-alpha-space
 (query-prefix-trace-conceptual-table-with-default prefix
 prefix-state trace-table)
and trace-path-bound:
wp-event
 (checked-staged-security-experiment-with-query-prefix-data-state A i)
 (checked-staged-security-with-query-prefix-current-trace-path-output-hit-at i)
 adversary-initial-state $\leq$ T
and composition-path-bound:
wp-event
 (checked-staged-security-experiment-with-query-prefix-data-state A i)

(checked-staged-security-with-query-prefix-current-composition-path-output-hit-at i)
adversary-initial-state $\leq C$

shows

wp-event
(checked-staged-security-experiment-with-data-state A)
(staged-security-with-data-state-current-query-partial-candidate-hit-at
trace-table composition-table i)
adversary-initial-state $\leq$
staged-phase-relation-error size
(staged-query-search-queries budgets i + 1) +
query-error-bound +
hash-collision-budget-value 0
(staged-query-search-queries budgets i + 1) + T + C

proof –

have *prefix-bound*:

wp-event
(checked-staged-security-experiment-with-query-prefix-data-state A i)
(checked-staged-security-with-query-prefix-authenticated-candidate-hit-at
trace-table composition-table i)
adversary-initial-state $\leq$
staged-phase-relation-error size
(staged-query-search-queries budgets i + 1) +
query-error-bound +
hash-collision-budget-value 0
(staged-query-search-queries budgets i + 1) + T + C

by (rule
checked-staged-security-with-query-prefix-authenticated-candidate-hit-at-bound-from-conceptual-default-alpha-clean
[OF *false-statement wf controlled i-bound default-alpha-clean*
trace-path-bound composition-path-bound])

show *?thesis*

by (rule *order-trans*
[OF
staged-security-with-data-state-current-query-partial-candidate-hit-at-le-query-prefix-authenticated-candidate
prefix-bound])
(use wf controlled i-bound in simp-all)

qed

lemma *checked-staged-security-with-actual-alpha-prefix-composition-candidate-drift-query-hit-bound-from-conceptual-*

assumes *false-statement: $\neg$ exists-valid-trace*

and *wf: staged-budget-wellformed budgets*

and *controlled: staged-adversary-controlled budgets A*

and *default-alpha-clean:*

$\bigwedge i$ *prefix prefix-state. i < rounds $\implies$*
Some ((prefix, prefix-state), prefix-state) $\in$
set-dist
(execute (checked-staged-query-prefix-with-state A i)
adversary-initial-state) $\implies$
trace-table-low-degree
(query-prefix-trace-conceptual-table-with-default prefix

$\text{prefix-state trace-default}) \wedge$
 $\text{composition-table-low-degree maxDegree}$
 $(\text{query-prefix-composition-conceptual-table-with-default prefix}$
 $\text{prefix-state composition-default}) \wedge$
 $\text{sqp-alphas prefix} \notin \text{composition-trace-bad-alpha-space}$
 $(\text{query-prefix-trace-conceptual-table-with-default prefix}$
 $\text{prefix-state trace-default})$
and trace-path-bound:
 $\bigwedge i. i < \text{rounds} \implies$
 wp-event
 $(\text{checked-staged-security-experiment-with-query-prefix-data-state } A \ i)$
 $(\text{checked-staged-security-with-query-prefix-current-trace-path-output-hit-at } i)$
 $\text{adversary-initial-state} \leq T \ i$
and $\text{composition-path-bound:}$
 $\bigwedge i. i < \text{rounds} \implies$
 wp-event
 $(\text{checked-staged-security-experiment-with-query-prefix-data-state } A \ i)$
 $(\text{checked-staged-security-with-query-prefix-current-composition-path-output-hit-at } i)$
 $\text{adversary-initial-state} \leq C \ i$
shows
 wp-event
 $(\text{checked-staged-security-experiment-with-actual-alpha-prefix-data-state } A)$
 $\text{checked-staged-security-with-actual-alpha-prefix-composition-candidate-drift-query-hit}$
 $\text{adversary-initial-state} \leq$
 $(\sum i < \text{rounds.}$
 $\text{staged-phase-relation-error size}$
 $(\text{staged-query-search-queries budgets } i + 1) +$
 $\text{query-error-bound} +$
 $\text{hash-collision-budget-value } 0$
 $(\text{staged-query-search-queries budgets } i + 1) +$
 $T \ i + C \ i)$
proof (rule
 $\text{checked-staged-security-with-actual-alpha-prefix-composition-candidate-drift-query-hit-bound-from-query-prefix-curr}$
 $[\text{OF wf controlled}])$
fix i
assume $i\text{-bound: } i < \text{rounds}$
show
 wp-event
 $(\text{checked-staged-security-experiment-with-query-prefix-data-state } A \ i)$
 $(\text{checked-staged-security-with-query-prefix-current-query-partial-opening-hit-at } i)$
 $\text{adversary-initial-state} \leq$
 $\text{staged-phase-relation-error size}$
 $(\text{staged-query-search-queries budgets } i + 1) +$
 $\text{query-error-bound} +$
 $\text{hash-collision-budget-value } 0$
 $(\text{staged-query-search-queries budgets } i + 1) +$
 $T \ i + C \ i$
by (rule
 $\text{checked-staged-security-with-query-prefix-current-query-partial-opening-hit-at-bound-from-conceptual-default-alpha}$

$[OF \text{ false-statement } wf \text{ controlled } i\text{-bound} - \text{trace-path-bound}[OF \text{ } i\text{-bound}]$
 $\text{composition-path-bound}[OF \text{ } i\text{-bound}]]$)
 $(\text{rule default-alpha-clean}[OF \text{ } i\text{-bound}])$

qed

theorem stark-soundness-from-aligned-partial-query-prefix-conceptual-default-alpha-clean-structured-paths:

fixes $\text{trace-fri-error}' \text{ composition-fri-error}' N :: \text{prob}$

and $T \ C :: \text{nat} \Rightarrow \text{prob}$

and $\text{trace-default} \text{ composition-default} :: 'f \text{ list}$

assumes $\text{false-statement}: \neg \text{exists-valid-trace}$

and $wf: \text{staged-budget-wellformed budgets}$

and $\text{controlled}: \text{staged-adversary-controlled budgets } A$

and $\text{trace-fri-bound}:$

$\text{wp-event } (\text{checked-staged-security-experiment-with-data-state } A)$
 $(\text{staged-security-with-data-state-verifier-event}$
 $\text{trace-fri-bad-with-partial-candidates})$
 $\text{adversary-initial-state} \leq \text{trace-fri-error}'$

and $\text{comp-fri-bound}:$

$\text{wp-event } (\text{checked-staged-security-experiment-with-data-state } A)$
 $(\text{staged-security-with-data-state-verifier-event}$
 $\text{composition-fri-bad-with-partial-candidates})$
 $\text{adversary-initial-state} \leq \text{composition-fri-error}'$

and $\text{default-alpha-clean}:$

$\bigwedge i \text{ prefix prefix-state.}$
 $i < \text{rounds} \implies$
 $\text{Some } ((\text{prefix}, \text{prefix-state}), \text{prefix-state}) \in$
 set-dist
 $(\text{execute } (\text{checked-staged-query-prefix-with-state } A \ i)$
 $\text{adversary-initial-state}) \implies$
 $\text{trace-table-low-degree}$
 $(\text{query-prefix-trace-conceptual-table-with-default prefix}$
 $\text{prefix-state trace-default}) \wedge$
 $\text{composition-table-low-degree maxDegree}$
 $(\text{query-prefix-composition-conceptual-table-with-default prefix}$
 $\text{prefix-state composition-default}) \wedge$
 $\text{sqp-alphas prefix} \notin \text{composition-trace-bad-alpha-space}$
 $(\text{query-prefix-trace-conceptual-table-with-default prefix}$
 $\text{prefix-state trace-default})$

and $\text{trace-path-bound}:$

$\bigwedge i. i < \text{rounds} \implies$
 wp-event
 $(\text{checked-staged-security-experiment-with-query-prefix-data-state } A \ i)$
 $(\text{checked-staged-security-with-query-prefix-current-trace-path-output-hit-at } i)$
 $\text{adversary-initial-state} \leq T \ i$

and $\text{composition-path-bound}:$

$\bigwedge i. i < \text{rounds} \implies$
 wp-event
 $(\text{checked-staged-security-experiment-with-query-prefix-data-state } A \ i)$
 $(\text{checked-staged-security-with-query-prefix-current-composition-path-output-hit-at } i)$

$\text{adversary-initial-state} \leq C \ i$
and *not-prefix-bound*:
 wp-event
 $(\text{checked-staged-security-experiment-with-actual-alpha-prefix-data-state } A)$
 $(\text{checked-staged-security-with-actual-alpha-prefix-partial-header-candidate-not-prefix-bound composition-trace-bad-alpha-space})$
 $\text{adversary-initial-state} \leq N$
and *empty-trace-fri-bound*:
 $\text{wp-event } (\text{checked-staged-security-experiment-with-data-state } A)$
 $(\text{staged-security-with-data-state-verifier-event trace-fri-bad-with-empty-composition-header-candidates})$
 $\text{adversary-initial-state} \leq \text{trace-fri-error}$
and *empty-query-bound*:
 $\text{wp-event } (\text{checked-staged-security-experiment-with-data-state } A)$
 $(\text{staged-security-with-data-state-verifier-event query-bad-with-empty-composition-header-candidates})$
 $\text{adversary-initial-state} \leq \text{nnreal rounds} * \text{query-error-bound}$
shows $\text{checked-staged-adversary-acceptance-probability } A \leq$
 $\text{hash-collision-budget-value } 0$
 $(\text{staged-attacker-query-budget budgets} +$
 $\text{staged-challenge-query-budget} + \text{verifier-hash-query-budget}) +$
 $(\text{composition-error-bound} +$
 $\text{staged-phase-relation-error size}$
 $(\text{staged-alpha-search-queries budgets } 0) +$
 $\text{hash-collision-budget-value } 0$
 $(\text{staged-alpha-search-queries budgets } 0) +$
 $N +$
 $\text{staged-phase-relation-error size}$
 $(\text{staged-alpha-search-queries budgets } 0)) +$
 $\text{trace-fri-error}' + \text{composition-fri-error}' +$
 $(\sum i < \text{rounds.}$
 $\text{staged-phase-relation-error size}$
 $(\text{staged-query-search-queries budgets } i + 1) +$
 $\text{query-error-bound} +$
 $\text{hash-collision-budget-value } 0$
 $(\text{staged-query-search-queries budgets } i + 1) +$
 $T \ i + C \ i) +$
 $(\text{composition-error-bound} +$
 $\text{staged-phase-relation-error size}$
 $(\text{staged-alpha-search-queries budgets } 0) +$
 $\text{hash-collision-budget-value } 0$
 $(\text{staged-alpha-search-queries budgets } 0) +$
 $N +$
 $\text{staged-phase-relation-error size}$
 $(\text{staged-alpha-search-queries budgets } 0) +$
 $\text{trace-fri-error} + \text{nnreal rounds} * \text{query-error-bound})$
proof $(\text{rule stark-soundness-from-aligned-partial-query-prefix-current-rounds}$
 $[\text{OF false-statement wf controlled trace-fri-bound comp-fri-bound - not-prefix-bound}$


```

    empty-trace-fri-bound empty-query-bound])
fix i
assume i-bound: i < rounds
show
  wp-event
    (checked-staged-security-experiment-with-query-prefix-data-state A i)
    (checked-staged-security-with-query-prefix-current-query-partial-opening-hit-at i)
    adversary-initial-state ≤
    staged-phase-relation-error size
      (staged-query-search-queries budgets i + 1) +
    query-error-bound +
    hash-collision-budget-value 0
      (staged-query-search-queries budgets i + 1) +
    T i + C i
by (rule
  checked-staged-security-with-query-prefix-current-query-partial-opening-hit-at-bound-from-conceptual-default-alpha-prefix-union-clean-structured-path
    [OF false-statement wf controlled i-bound - trace-path-bound[OF i-bound]
      composition-path-bound[OF i-bound]])
  (rule default-alpha-clean[OF i-bound])
qed

theorem stark-soundness-from-aligned-partial-query-prefix-conceptual-default-alpha-prefix-union-clean-structured-path
fixes trace-fri-error' composition-fri-error' N :: prob
and T C :: nat ⇒ prob
and trace-default composition-default :: 'f list
assumes false-statement: ¬ exists-valid-trace
and wf: staged-budget-wellformed budgets
and controlled: staged-adversary-controlled budgets A
and trace-fri-bound:
  wp-event (checked-staged-security-experiment-with-data-state A)
    (staged-security-with-data-state-verifier-event
      trace-fri-bad-with-partial-candidates)
    adversary-initial-state ≤ trace-fri-error'
and comp-fri-bound:
  wp-event (checked-staged-security-experiment-with-data-state A)
    (staged-security-with-data-state-verifier-event
      composition-fri-bad-with-partial-candidates)
    adversary-initial-state ≤ composition-fri-error'
and default-prefix-clean:
  ∧ i prefix prefix-state.
    i < rounds ⇒
    Some ((prefix, prefix-state), prefix-state) ∈
      set-dist
        (execute (checked-staged-query-prefix-with-state A i)
          adversary-initial-state) ⇒
    query-prefix-trace-conceptual-table-with-default prefix prefix-state
      trace-default = trace-default ∧
    query-prefix-composition-conceptual-table-with-default prefix
      prefix-state composition-default = composition-default ∧

```

$\text{trace-table-low-degree trace-default} \wedge$
 $\text{composition-table-low-degree maxDegree composition-default} \wedge$
 $\text{trace-default} \in$
 $\text{alpha-prefix-trace-table-candidates prefix-state}$
 $(\text{sqp-trace-root prefix}) \wedge$
 $\text{sqp-alphas prefix} \notin$
 $\text{alpha-prefix-union-bad-sets prefix-state}$
 $\text{composition-trace-bad-alpha-space} (\text{sqp-trace-root prefix})$
and trace-path-bound:
 $\bigwedge i. i < \text{rounds} \implies$
 wp-event
 $(\text{checked-staged-security-experiment-with-query-prefix-data-state } A \ i)$
 $(\text{checked-staged-security-with-query-prefix-current-trace-path-output-hit-at } i)$
 $\text{adversary-initial-state} \leq T \ i$
and $\text{composition-path-bound:}$
 $\bigwedge i. i < \text{rounds} \implies$
 wp-event
 $(\text{checked-staged-security-experiment-with-query-prefix-data-state } A \ i)$
 $(\text{checked-staged-security-with-query-prefix-current-composition-path-output-hit-at } i)$
 $\text{adversary-initial-state} \leq C \ i$
and not-prefix-bound:
 wp-event
 $(\text{checked-staged-security-experiment-with-actual-alpha-prefix-data-state } A)$
 $(\text{checked-staged-security-with-actual-alpha-prefix-partial-header-candidate-not-prefix-bound}$
 $\text{composition-trace-bad-alpha-space})$
 $\text{adversary-initial-state} \leq N$
and $\text{empty-trace-fri-bound:}$
 $\text{wp-event} (\text{checked-staged-security-experiment-with-data-state } A)$
 $(\text{staged-security-with-data-state-verifier-event}$
 $\text{trace-fri-bad-with-empty-composition-header-candidates})$
 $\text{adversary-initial-state} \leq \text{trace-fri-error}$
and $\text{empty-query-bound:}$
 $\text{wp-event} (\text{checked-staged-security-experiment-with-data-state } A)$
 $(\text{staged-security-with-data-state-verifier-event}$
 $\text{query-bad-with-empty-composition-header-candidates})$
 $\text{adversary-initial-state} \leq \text{nnreal rounds} * \text{query-error-bound}$
shows $\text{checked-staged-adversary-acceptance-probability } A \leq$
 $\text{hash-collision-budget-value } 0$
 $(\text{staged-attacker-query-budget budgets} +$
 $\text{staged-challenge-query-budget} + \text{verifier-hash-query-budget}) +$
 $(\text{composition-error-bound} +$
 $\text{staged-phase-relation-error size}$
 $(\text{staged-alpha-search-queries budgets } 0) +$
 $\text{hash-collision-budget-value } 0$
 $(\text{staged-alpha-search-queries budgets } 0) +$
 $N +$
 $\text{staged-phase-relation-error size}$
 $(\text{staged-alpha-search-queries budgets } 0)) +$

```

    trace-fri-error' + composition-fri-error' +
    (∑ i < rounds.
      staged-phase-relation-error size
      (staged-query-search-queries budgets i + 1) +
      query-error-bound +
      hash-collision-budget-value 0
      (staged-query-search-queries budgets i + 1) +
      T i + C i) +
    (composition-error-bound +
      staged-phase-relation-error size
      (staged-alpha-search-queries budgets 0) +
      hash-collision-budget-value 0
      (staged-alpha-search-queries budgets 0) +
      N +
      staged-phase-relation-error size
      (staged-alpha-search-queries budgets 0) +
      trace-fri-error + nreal rounds * query-error-bound)
  proof (rule stark-soundness-from-aligned-partial-query-prefix-current-rounds
    [OF false-statement wf controlled trace-fri-bound comp-fri-bound - not-prefix-bound
      empty-trace-fri-bound empty-query-bound])
  fix i
  assume i-bound: i < rounds
  show
    wp-event
    (checked-staged-security-experiment-with-query-prefix-data-state A i)
    (checked-staged-security-with-query-prefix-current-query-partial-opening-hit-at i)
    adversary-initial-state ≤
    staged-phase-relation-error size
    (staged-query-search-queries budgets i + 1) +
    query-error-bound +
    hash-collision-budget-value 0
    (staged-query-search-queries budgets i + 1) +
    T i + C i
  by (rule
    checked-staged-security-with-query-prefix-current-query-partial-opening-hit-at-bound-from-conceptual-default-alpha-clean
    [OF false-statement wf controlled i-bound - trace-path-bound [OF i-bound]
      composition-path-bound [OF i-bound]])
    (rule default-prefix-clean [OF i-bound])
qed

end

end

theory Soundness-Staged-Aligned-Current-Empty
imports
  Soundness-Conceptual-Query-Current-Alpha-Clean
  Soundness-Staged-Aligned-Partial-Merkle

```

begin

Packaging for the aligned-transcript partial-candidate path with the current-empty query bound. This combines the current-query non-empty branch with the empty-composition branch introduced in the current-empty query layer, using a refined empty-header query bound in this downstream route.

context *soundness*

begin

definition *checked-staged-security-with-actual-alpha-prefix-relevant-partial-header-candidate-drift*

$$\begin{aligned} &:: ((((((f \times 'f \text{ list} \times 'f \text{ list} \times 'f) \times \\ &\quad 'f \text{ protocol-channel}) \times 'f \text{ staged-proof-data}) \times \\ &\quad 'f \text{ protocol-channel}) \times \text{unit list}) \times \\ &\quad 'f \text{ protocol-channel}) \text{ option} \Rightarrow \text{bool} \end{aligned}$$

where

$$\begin{aligned} &\text{checked-staged-security-with-actual-alpha-prefix-relevant-partial-header-candidate-drift} \\ &\text{out} \longleftrightarrow \\ &\text{checked-staged-security-with-actual-alpha-prefix-partial-header-candidate-drift} \\ &\quad \text{composition-trace-bad-alpha-space out} \wedge \\ &(\text{checked-staged-security-with-actual-alpha-prefix-verifier-event} \\ &\quad \text{composition-randomization-bad-with-partial-candidates out} \vee \\ &\quad \text{checked-staged-security-with-actual-alpha-prefix-verifier-event} \\ &\quad \text{composition-alpha-partial-opening-union-hit-with-empty-header out}) \end{aligned}$$

lemma *checked-staged-security-with-actual-alpha-prefix-partial-candidate-randomization-imp-prefix-prequery-or-relev*

assumes *support*:

$$\begin{aligned} &\text{out} \in \\ &\quad \text{set-dist} \\ &\quad (\text{execute} \\ &\quad \quad (\text{checked-staged-security-experiment-with-actual-alpha-prefix-data-state} \\ &\quad \quad \quad A) \\ &\quad \quad \text{adversary-initial-state}) \end{aligned}$$

and *bad*:

$$\begin{aligned} &\text{checked-staged-security-with-actual-alpha-prefix-verifier-event} \\ &\quad \text{composition-randomization-bad-with-partial-candidates out} \end{aligned}$$

shows

$$\begin{aligned} &\text{checked-staged-security-with-actual-alpha-prefix-bad-set-hit} \\ &\quad \text{composition-trace-bad-alpha-space out} \vee \\ &\text{checked-staged-security-with-actual-alpha-prefix-partial-header-prequery-hit} \\ &\quad \text{composition-trace-bad-alpha-space out} \vee \\ &\text{checked-staged-security-with-actual-alpha-prefix-relevant-partial-header-candidate-drift} \\ &\quad \text{out} \end{aligned}$$

proof —

have *partial*:

$$\begin{aligned} &\text{checked-staged-security-with-actual-alpha-prefix-partial-header-bad-set-hit} \\ &\quad \text{composition-trace-bad-alpha-space out} \end{aligned}$$

by (rule

$$\begin{aligned} &\text{checked-staged-security-with-actual-alpha-prefix-partial-candidate-randomization-imp-partial-header-bad-set-hit-o} \\ &\quad [OF \text{ support bad}] \end{aligned}$$

have *fresh-or-prequery*:
checked-staged-security-with-actual-alpha-prefix-partial-header-fresh-hit
composition-trace-bad-alpha-space out $\vee$
checked-staged-security-with-actual-alpha-prefix-partial-header-prequery-hit
composition-trace-bad-alpha-space out
by (rule
checked-staged-security-with-actual-alpha-prefix-partial-header-bad-set-hit-imp-fresh-or-prequery
[*OF partial*])
then show *?thesis*
proof
assume *fresh*:
checked-staged-security-with-actual-alpha-prefix-partial-header-fresh-hit
composition-trace-bad-alpha-space out
have *prefix-or-drift*:
checked-staged-security-with-actual-alpha-prefix-bad-set-hit
composition-trace-bad-alpha-space out $\vee$
checked-staged-security-with-actual-alpha-prefix-partial-header-candidate-drift
composition-trace-bad-alpha-space out
by (rule
checked-staged-security-with-actual-alpha-prefix-partial-header-fresh-hit-imp-prefix-or-drift
[*OF fresh*])
then show *?thesis*
proof
assume
checked-staged-security-with-actual-alpha-prefix-bad-set-hit
composition-trace-bad-alpha-space out
then show *?thesis* **by** *simp*
next
assume *drift*:
checked-staged-security-with-actual-alpha-prefix-partial-header-candidate-drift
composition-trace-bad-alpha-space out
have
checked-staged-security-with-actual-alpha-prefix-relevant-partial-header-candidate-drift
out
unfolding
checked-staged-security-with-actual-alpha-prefix-relevant-partial-header-candidate-drift-def
using *drift bad* **by** *simp*
then show *?thesis* **by** *simp*
qed
next
assume
checked-staged-security-with-actual-alpha-prefix-partial-header-prequery-hit
composition-trace-bad-alpha-space out
then show *?thesis* **by** *simp*
qed
qed

lemma *checked-staged-security-with-actual-alpha-prefix-empty-header-alpha-hit-imp-prefix-prequery-or-relevant-drift-*
assumes *support*:

```

    out ∈
      set-dist
        (execute
          (checked-staged-security-experiment-with-actual-alpha-prefix-data-state
            A)
          adversary-initial-state)
  and hit:
    checked-staged-security-with-actual-alpha-prefix-verifier-event
    composition-alpha-partial-opening-union-hit-with-empty-header out
  shows
    checked-staged-security-with-actual-alpha-prefix-bad-set-hit
    composition-trace-bad-alpha-space out ∨
    checked-staged-security-with-actual-alpha-prefix-partial-header-prequery-hit
    composition-trace-bad-alpha-space out ∨
    checked-staged-security-with-actual-alpha-prefix-relevant-partial-header-candidate-drift
    out
  proof —
  have partial:
    checked-staged-security-with-actual-alpha-prefix-partial-header-bad-set-hit
    composition-trace-bad-alpha-space out
  by (rule
    checked-staged-security-with-actual-alpha-prefix-empty-header-partial-alpha-hit-imp-partial-header-bad-set-hit-on-
    [OF support hit])
  have fresh-or-prequery:
    checked-staged-security-with-actual-alpha-prefix-partial-header-fresh-hit
    composition-trace-bad-alpha-space out ∨
    checked-staged-security-with-actual-alpha-prefix-partial-header-prequery-hit
    composition-trace-bad-alpha-space out
  by (rule
    checked-staged-security-with-actual-alpha-prefix-partial-header-bad-set-hit-imp-fresh-or-prequery
    [OF partial])
  then show ?thesis
  proof
  assume fresh:
    checked-staged-security-with-actual-alpha-prefix-partial-header-fresh-hit
    composition-trace-bad-alpha-space out
  have prefix-or-drift:
    checked-staged-security-with-actual-alpha-prefix-bad-set-hit
    composition-trace-bad-alpha-space out ∨
    checked-staged-security-with-actual-alpha-prefix-partial-header-candidate-drift
    composition-trace-bad-alpha-space out
  by (rule
    checked-staged-security-with-actual-alpha-prefix-partial-header-fresh-hit-imp-prefix-or-drift
    [OF fresh])
  then show ?thesis
  proof
  assume
    checked-staged-security-with-actual-alpha-prefix-bad-set-hit
    composition-trace-bad-alpha-space out

```

```

    then show ?thesis by simp
next
  assume drift:
    checked-staged-security-with-actual-alpha-prefix-partial-header-candidate-drift
    composition-trace-bad-alpha-space out
  have
    checked-staged-security-with-actual-alpha-prefix-relevant-partial-header-candidate-drift
    out
  unfolding
    checked-staged-security-with-actual-alpha-prefix-relevant-partial-header-candidate-drift-def
  using drift hit by simp
  then show ?thesis by simp
qed
next
  assume
    checked-staged-security-with-actual-alpha-prefix-partial-header-prequery-hit
    composition-trace-bad-alpha-space out
  then show ?thesis by simp
qed
qed

lemma checked-staged-security-with-actual-alpha-prefix-partial-candidate-randomization-bound-from-relevant-drift-and-
assumes wf: staged-budget-wellformed budgets
and controlled: staged-adversary-controlled budgets A
and drift-bound:
  wp-event
  (checked-staged-security-experiment-with-actual-alpha-prefix-data-state
   A)
  checked-staged-security-with-actual-alpha-prefix-relevant-partial-header-candidate-drift
  adversary-initial-state  $\leq D$ 
shows
  wp-event
  (checked-staged-security-experiment-with-actual-alpha-prefix-data-state A)
  (checked-staged-security-with-actual-alpha-prefix-verifier-event
   composition-randomization-bad-with-partial-candidates)
  adversary-initial-state  $\leq$ 
  composition-error-bound +
  staged-phase-relation-error size
  (staged-alpha-search-queries budgets 0) +
  hash-collision-budget-value 0
  (staged-alpha-search-queries budgets 0) +
  staged-phase-relation-error size
  (staged-alpha-search-queries budgets 0) +
  D
proof -
  let ?M =
    checked-staged-security-experiment-with-actual-alpha-prefix-data-state A
  let ?Prefix =
    checked-staged-security-with-actual-alpha-prefix-bad-set-hit

```

```

    composition-trace-bad-alpha-space
let ?Prequery =
    checked-staged-security-with-actual-alpha-prefix-partial-header-prequery-hit
    composition-trace-bad-alpha-space
let ?Drift =
    checked-staged-security-with-actual-alpha-prefix-relevant-partial-header-candidate-drift
let ?Random =
    checked-staged-security-with-actual-alpha-prefix-verifier-event
    composition-randomization-bad-with-partial-candidates
let ?P =
    composition-error-bound +
    staged-phase-relation-error size
    (staged-alpha-search-queries budgets 0) +
    hash-collision-budget-value 0
    (staged-alpha-search-queries budgets 0)
let ?Q =
    staged-phase-relation-error size
    (staged-alpha-search-queries budgets 0)
have transcript-bound:
    wp-event (checked-staged-transcript-with-alpha-prefix-program A)
    (checked-staged-transcript-actual-alpha-prefix-bad-set-hit
     composition-trace-bad-alpha-space)
    adversary-initial-state ≤ ?P
by (rule
    checked-staged-transcript-with-alpha-prefix-actual-bad-set-hit-bound-from-budgets
    [OF wf controlled])
have prefix-bound:
    wp-event ?M ?Prefix adversary-initial-state ≤ ?P
by (rule
    checked-staged-security-with-actual-alpha-prefix-bad-set-hit-bound-from-transcript
    [OF transcript-bound])
have prequery-bound:
    wp-event ?M ?Prequery adversary-initial-state ≤ ?Q
by (rule
    checked-staged-security-with-actual-alpha-prefix-vector-alpha-prequery-accounting
    [OF wf controlled])
have wp-event ?M ?Random adversary-initial-state ≤
    wp-event ?M
    (λout. ?Prefix out ∨ ?Prequery out ∨ ?Drift out ∨ False)
    adversary-initial-state
by (rule wp-event-mono-on-support)
    (use
    checked-staged-security-with-actual-alpha-prefix-partial-candidate-randomization-imp-prefix-prequery-or-relevant-
    [of - A] in blast)
also have ... ≤
    wp-event ?M ?Prefix adversary-initial-state +
    wp-event ?M ?Prequery adversary-initial-state +
    wp-event ?M ?Drift adversary-initial-state +
    wp-event ?M (λ-. False) adversary-initial-state

```



```

    by (rule wp-event-union-bound4)
  also have ...  $\leq ?P + ?Q + D + 0$ 
    by (intro add-mono prefix-bound prequery-bound drift-bound)
      (simp add: wp-event-def wp-def dist-expect-def)
  finally show ?thesis
    by (simp add: algebra-simps)
qed

lemma checked-staged-security-with-actual-alpha-prefix-empty-header-alpha-hit-bound-from-relevant-drift-and-budget.
  assumes wf: staged-budget-wellformed budgets
    and controlled: staged-adversary-controlled budgets A
    and drift-bound:
      wp-event
      (checked-staged-security-experiment-with-actual-alpha-prefix-data-state
        A)
      checked-staged-security-with-actual-alpha-prefix-relevant-partial-header-candidate-drift
      adversary-initial-state  $\leq D$ 
  shows
    wp-event
    (checked-staged-security-experiment-with-actual-alpha-prefix-data-state A)
    (checked-staged-security-with-actual-alpha-prefix-verifier-event
      composition-alpha-partial-opening-union-hit-with-empty-header)
    adversary-initial-state  $\leq$ 
    composition-error-bound +
    staged-phase-relation-error size
    (staged-alpha-search-queries budgets 0) +
    hash-collision-budget-value 0
    (staged-alpha-search-queries budgets 0) +
    staged-phase-relation-error size
    (staged-alpha-search-queries budgets 0) +
    D
proof -
  let ?M =
    checked-staged-security-experiment-with-actual-alpha-prefix-data-state A
  let ?Prefix =
    checked-staged-security-with-actual-alpha-prefix-bad-set-hit
    composition-trace-bad-alpha-space
  let ?Prequery =
    checked-staged-security-with-actual-alpha-prefix-partial-header-prequery-hit
    composition-trace-bad-alpha-space
  let ?Drift =
    checked-staged-security-with-actual-alpha-prefix-relevant-partial-header-candidate-drift
  let ?Hit =
    checked-staged-security-with-actual-alpha-prefix-verifier-event
    composition-alpha-partial-opening-union-hit-with-empty-header
  let ?P =
    composition-error-bound +
    staged-phase-relation-error size
    (staged-alpha-search-queries budgets 0) +

```

```

    hash-collision-budget-value 0
    (staged-alpha-search-queries budgets 0)
  let ?Q =
    staged-phase-relation-error size
    (staged-alpha-search-queries budgets 0)
  have transcript-bound:
    wp-event (checked-staged-transcript-with-alpha-prefix-program A)
    (checked-staged-transcript-actual-alpha-prefix-bad-set-hit
     composition-trace-bad-alpha-space)
    adversary-initial-state ≤ ?P
  by (rule
     checked-staged-transcript-with-alpha-prefix-actual-bad-set-hit-bound-from-budgets
     [OF wf controlled])
  have prefix-bound:
    wp-event ?M ?Prefix adversary-initial-state ≤ ?P
  by (rule
     checked-staged-security-with-actual-alpha-prefix-bad-set-hit-bound-from-transcript
     [OF transcript-bound])
  have prequery-bound:
    wp-event ?M ?Prequery adversary-initial-state ≤ ?Q
  by (rule
     checked-staged-security-with-actual-alpha-prefix-vector-alpha-prequery-accounting
     [OF wf controlled])
  have wp-event ?M ?Hit adversary-initial-state ≤
    wp-event ?M
    (λout. ?Prefix out ∨ ?Prequery out ∨ ?Drift out ∨ False)
    adversary-initial-state
  by (rule wp-event-mono-on-support)
  (use
   checked-staged-security-with-actual-alpha-prefix-empty-header-alpha-hit-imp-prefix-prequery-or-relevant-drift-on-
   [of - A] in blast)
  also have ... ≤
    wp-event ?M ?Prefix adversary-initial-state +
    wp-event ?M ?Prequery adversary-initial-state +
    wp-event ?M ?Drift adversary-initial-state +
    wp-event ?M (λ-. False) adversary-initial-state
  by (rule wp-event-union-bound4)
  also have ... ≤ ?P + ?Q + D + 0
  by (intro add-mono prefix-bound prequery-bound drift-bound)
  (simp add: wp-event-def wp-def dist-expect-def)
  finally show ?thesis
  by (simp add: algebra-simps)
qed

lemma checked-staged-security-with-data-state-partial-candidate-composition-bad-bound-from-relevant-drift-and-budg
  assumes false-statement: ¬ exists-valid-trace
  and wf: staged-budget-wellformed budgets
  and controlled: staged-adversary-controlled budgets A
  and drift-bound:

```

$wp\text{-}event$
 $(checked\text{-}staged\text{-}security\text{-}experiment\text{-}with\text{-}actual\text{-}alpha\text{-}prefix\text{-}data\text{-}state$
 $A)$
 $checked\text{-}staged\text{-}security\text{-}with\text{-}actual\text{-}alpha\text{-}prefix\text{-}relevant\text{-}partial\text{-}header\text{-}candidate\text{-}drift$
 $adversary\text{-}initial\text{-}state \leq D$

shows

$wp\text{-}event (checked\text{-}staged\text{-}security\text{-}experiment\text{-}with\text{-}data\text{-}state A)$
 $(staged\text{-}security\text{-}with\text{-}data\text{-}state\text{-}verifier\text{-}event$
 $composition\text{-}bad\text{-}with\text{-}partial\text{-}candidates)$
 $adversary\text{-}initial\text{-}state \leq$
 $composition\text{-}error\text{-}bound +$
 $staged\text{-}phase\text{-}relation\text{-}error\text{-}size$
 $(staged\text{-}alpha\text{-}search\text{-}queries\text{-}budgets\text{-}0) +$
 $hash\text{-}collision\text{-}budget\text{-}value\text{-}0$
 $(staged\text{-}alpha\text{-}search\text{-}queries\text{-}budgets\text{-}0) +$
 $staged\text{-}phase\text{-}relation\text{-}error\text{-}size$
 $(staged\text{-}alpha\text{-}search\text{-}queries\text{-}budgets\text{-}0) +$
 D

proof –

let $?M = checked\text{-}staged\text{-}security\text{-}experiment\text{-}with\text{-}data\text{-}state A$
let $?E = staged\text{-}security\text{-}with\text{-}data\text{-}state\text{-}verifier\text{-}event$
let $?degree = ?E\text{-}composition\text{-}degree\text{-}bad\text{-}with\text{-}partial\text{-}candidates$
let $?random = ?E\text{-}composition\text{-}randomization\text{-}bad\text{-}with\text{-}partial\text{-}candidates$
let $?bad = ?E\text{-}composition\text{-}bad\text{-}with\text{-}partial\text{-}candidates$
have $degree\text{-}false: ?degree = (\lambda out. False)$
by (rule ext)
 $(auto\text{-}simp: staged\text{-}security\text{-}with\text{-}data\text{-}state\text{-}verifier\text{-}event\text{-}def$
 $composition\text{-}degree\text{-}bad\text{-}with\text{-}partial\text{-}candidates\text{-}false$
 $split: option.splits\text{-}prod.splits)$

have $degree\text{-}bound:$

$wp\text{-}event\text{-}?M\text{-}?degree\text{-}adversary\text{-}initial\text{-}state \leq 0$
unfolding $degree\text{-}false\text{-}wp\text{-}event\text{-}def\text{-}wp\text{-}def\text{-}dist\text{-}expect\text{-}def$ **by** $simp$

have $projected:$

$wp\text{-}event\text{-}?M\text{-}?random\text{-}adversary\text{-}initial\text{-}state =$
 $wp\text{-}event$
 $(checked\text{-}staged\text{-}security\text{-}experiment\text{-}with\text{-}actual\text{-}alpha\text{-}prefix\text{-}data\text{-}state A)$
 $(checked\text{-}staged\text{-}security\text{-}with\text{-}actual\text{-}alpha\text{-}prefix\text{-}verifier\text{-}event$
 $composition\text{-}randomization\text{-}bad\text{-}with\text{-}partial\text{-}candidates)$
 $adversary\text{-}initial\text{-}state$

by (rule

$checked\text{-}staged\text{-}security\text{-}with\text{-}data\text{-}state\text{-}verifier\text{-}event\text{-}actual\text{-}alpha\text{-}prefix\text{-}projection)$

have $random\text{-}bound:$

$wp\text{-}event\text{-}?M\text{-}?random\text{-}adversary\text{-}initial\text{-}state \leq$
 $composition\text{-}error\text{-}bound +$
 $staged\text{-}phase\text{-}relation\text{-}error\text{-}size$
 $(staged\text{-}alpha\text{-}search\text{-}queries\text{-}budgets\text{-}0) +$
 $hash\text{-}collision\text{-}budget\text{-}value\text{-}0$
 $(staged\text{-}alpha\text{-}search\text{-}queries\text{-}budgets\text{-}0) +$
 $staged\text{-}phase\text{-}relation\text{-}error\text{-}size$

(staged-alpha-search-queries budgets 0) +
 D
unfolding projected
by (rule
 checked-staged-security-with-actual-alpha-prefix-partial-candidate-randomization-bound-from-relevant-drift-and-b
 [OF wf controlled drift-bound])
have wp-event ?M ?bad adversary-initial-state ≤
 wp-event ?M (λout. ?degree out ∨ ?random out)
 adversary-initial-state
by (rule wp-event-mono)
 (auto simp: staged-security-with-data-state-verifier-event-def
 dest: composition-bad-with-partial-candidates-split
 [OF false-statement]
 split: option.splits prod.splits)
also have ... ≤
 wp-event ?M ?degree adversary-initial-state +
 wp-event ?M ?random adversary-initial-state
by (rule wp-event-union-bound)
also have ... ≤
 0 +
 (composition-error-bound +
 staged-phase-relation-error size
 (staged-alpha-search-queries budgets 0) +
 hash-collision-budget-value 0
 (staged-alpha-search-queries budgets 0) +
 staged-phase-relation-error size
 (staged-alpha-search-queries budgets 0) +
 D)
by (intro add-mono degree-bound random-bound)
finally show ?thesis
by simp
qed

lemma checked-staged-security-with-data-state-empty-header-composition-randomization-bound-from-relevant-drift-a
assumes wf: staged-budget-wellformed budgets
and controlled: staged-adversary-controlled budgets A
and drift-bound:
 wp-event
 (checked-staged-security-experiment-with-actual-alpha-prefix-data-state
 A)
 checked-staged-security-with-actual-alpha-prefix-relevant-partial-header-candidate-drift
 adversary-initial-state ≤ D
shows
 wp-event (checked-staged-security-experiment-with-data-state A)
 (staged-security-with-data-state-verifier-event
 composition-randomization-bad-with-empty-composition-header-candidates)
 adversary-initial-state ≤
 composition-error-bound +
 staged-phase-relation-error size

```

    (staged-alpha-search-queries budgets 0) +
    hash-collision-budget-value 0
    (staged-alpha-search-queries budgets 0) +
    staged-phase-relation-error size
    (staged-alpha-search-queries budgets 0) +
    D
proof –
  let ?M = checked-staged-security-experiment-with-data-state A
  let ?random =
    staged-security-with-data-state-verifier-event
    composition-randomization-bad-with-empty-composition-header-candidates
  let ?alpha =
    staged-security-with-data-state-verifier-event
    composition-alpha-partial-opening-union-hit-with-empty-header
  have alpha-projected:
    wp-event ?M ?alpha adversary-initial-state =
      wp-event
      (checked-staged-security-experiment-with-actual-alpha-prefix-data-state A)
      (checked-staged-security-with-actual-alpha-prefix-verifier-event
        composition-alpha-partial-opening-union-hit-with-empty-header)
      adversary-initial-state
  by (rule
    checked-staged-security-with-data-state-verifier-event-actual-alpha-prefix-projection)
  have alpha-bound:
    wp-event ?M ?alpha adversary-initial-state ≤
      composition-error-bound +
      staged-phase-relation-error size
      (staged-alpha-search-queries budgets 0) +
      hash-collision-budget-value 0
      (staged-alpha-search-queries budgets 0) +
      staged-phase-relation-error size
      (staged-alpha-search-queries budgets 0) +
      D
  unfolding alpha-projected
  by (rule
    checked-staged-security-with-actual-alpha-prefix-empty-header-alpha-hit-bound-from-relevant-drift-and-budgets
    [OF wf controlled drift-bound])
  have wp-event ?M ?random adversary-initial-state ≤
    wp-event ?M ?alpha adversary-initial-state
  by (rule wp-event-mono)
    (auto simp: staged-security-with-data-state-verifier-event-def
      intro:
        composition-alpha-bad-set-hit-with-empty-header-imp-partial-opening-union-hit
        composition-randomization-bad-with-empty-composition-header-candidates-imp-alpha-hit
        split: option.splits prod.splits)
  also have ... ≤
    composition-error-bound +
    staged-phase-relation-error size
    (staged-alpha-search-queries budgets 0) +

```

```

    hash-collision-budget-value 0
    (staged-alpha-search-queries budgets 0) +
    staged-phase-relation-error size
    (staged-alpha-search-queries budgets 0) +
    D
  by (rule alpha-bound)
  finally show ?thesis .
qed

lemma checked-staged-security-with-data-state-empty-header-bad-event-bound-from-relevant-drift-trace-fri-and-query
  fixes trace-fri-error' :: prob
  assumes false-statement:  $\neg$  exists-valid-trace
  and wf: staged-budget-wellformed budgets
  and controlled: staged-adversary-controlled budgets A
  and drift-bound:
    wp-event
      (checked-staged-security-experiment-with-actual-alpha-prefix-data-state
        A)
      checked-staged-security-with-actual-alpha-prefix-relevant-partial-header-candidate-drift
      adversary-initial-state  $\leq$  D
  and trace-fri-bound:
    wp-event (checked-staged-security-experiment-with-data-state A)
      (staged-security-with-data-state-verifier-event
        trace-fri-bad-with-empty-composition-header-candidates)
      adversary-initial-state  $\leq$  trace-fri-error'
  and query-bound:
    wp-event (checked-staged-security-experiment-with-data-state A)
      (staged-security-with-data-state-verifier-event
        query-bad-with-empty-composition-header-candidates)
      adversary-initial-state  $\leq$  empty-query-error
  shows
    wp-event (checked-staged-security-experiment-with-data-state A)
      (staged-security-with-data-state-verifier-event
        soundness-bad-event-partial-candidate-empty-header)
      adversary-initial-state  $\leq$ 
      composition-error-bound +
      staged-phase-relation-error size
      (staged-alpha-search-queries budgets 0) +
      hash-collision-budget-value 0
      (staged-alpha-search-queries budgets 0) +
      staged-phase-relation-error size
      (staged-alpha-search-queries budgets 0) +
      D + trace-fri-error' + empty-query-error
proof -
  let ?M = checked-staged-security-experiment-with-data-state A
  let ?E = staged-security-with-data-state-verifier-event
  let ?degree =
    ?E composition-degree-bad-with-empty-composition-header-candidates
  let ?random =

```

```

    ?E composition-randomization-bad-with-empty-composition-header-candidates
  let ?trace =
    ?E trace-fri-bad-with-empty-composition-header-candidates
  let ?query =
    ?E query-bad-with-empty-composition-header-candidates
  let ?bad = ?E soundness-bad-event-partial-candidate-empty-header
  have degree-false: ?degree = (λout. False)
  by (rule ext)
    (auto simp: staged-security-with-data-state-verifier-event-def
      composition-degree-bad-with-empty-composition-header-candidates-false
      split: option.splits prod.splits)
  have degree-bound:
    wp-event ?M ?degree adversary-initial-state ≤ 0
  unfolding degree-false wp-event-def wp-def dist-expect-def by simp
  have random-bound:
    wp-event ?M ?random adversary-initial-state ≤
      composition-error-bound +
      staged-phase-relation-error size
      (staged-alpha-search-queries budgets 0) +
      hash-collision-budget-value 0
      (staged-alpha-search-queries budgets 0) +
      staged-phase-relation-error size
      (staged-alpha-search-queries budgets 0) +
      D
  by (rule
    checked-staged-security-with-data-state-empty-header-composition-randomization-bound-from-relevant-drift-and-l
    [OF wf controlled drift-bound])
  have wp-event ?M ?bad adversary-initial-state ≤
    wp-event ?M
      (λout. ?degree out ∨ ?random out ∨ ?trace out ∨
        ?query out)
      adversary-initial-state
  proof (rule wp-event-mono)
    fix out
    assume bad: ?bad out
    show ?degree out ∨ ?random out ∨ ?trace out ∨ ?query out
  proof (cases out)
    case None
    then show ?thesis
      using bad
      unfolding staged-security-with-data-state-verifier-event-def
      by simp
  next
    case (Some packed)
    then obtain data attacker-state result final-state where out-eq:
      out = Some (((data, attacker-state), result), final-state)
      by (cases packed, auto split: prod.splits)
    let ?s =
      verifier-state-from-adversary attacker-state

```

```

    (staged-proof-transcript data)
have local-bad:
  soundness-bad-event-partial-candidate-empty-header ?s
    (Some (result, final-state))
using bad
unfolding out-eq staged-security-with-data-state-verifier-event-def
by simp
then consider
  (comp)
    composition-bad-with-empty-composition-header-candidates ?s
      (Some (result, final-state))
  | (trace)
    trace-fri-bad-with-empty-composition-header-candidates ?s
      (Some (result, final-state))
  | (query)
    query-bad-with-empty-composition-header-candidates ?s
      (Some (result, final-state))
unfolding soundness-bad-event-partial-candidate-empty-header-def
by blast
then show ?thesis
proof cases
case comp
then have comp-split:
  composition-degree-bad-with-empty-composition-header-candidates ?s
    (Some (result, final-state))  $\vee$ 
  composition-randomization-bad-with-empty-composition-header-candidates
    ?s (Some (result, final-state))
by (rule
  composition-bad-with-empty-composition-header-candidates-split
  [OF false-statement])
then show ?thesis
  unfolding out-eq staged-security-with-data-state-verifier-event-def
  using comp-split by (auto split: option.splits prod.splits)
next
case trace
then show ?thesis
  unfolding out-eq staged-security-with-data-state-verifier-event-def
  by simp
next
case query
then show ?thesis
  unfolding out-eq staged-security-with-data-state-verifier-event-def
  by simp
qed
qed
qed
also have ...  $\leq$ 
  wp-event ?M ?degree adversary-initial-state +
  wp-event ?M ?random adversary-initial-state +

```



```

    wp-event ?M ?trace adversary-initial-state +
    wp-event ?M ?query adversary-initial-state
  by (rule wp-event-union-bound4)
also have ... ≤
  0 +
  (composition-error-bound +
   staged-phase-relation-error size
   (staged-alpha-search-queries budgets 0) +
   hash-collision-budget-value 0
   (staged-alpha-search-queries budgets 0) +
   staged-phase-relation-error size
   (staged-alpha-search-queries budgets 0) +
   D) +
  trace-fri-error' + empty-query-error
  by (intro add-mono degree-bound random-bound trace-fri-bound query-bound)
finally show ?thesis
  by (simp add: algebra-simps)
qed

```

```

lemma checked-staged-soundness-from-aligned-partial-candidate-components-and-current-empty-query-from-relevant-
fixes trace-fri-error' composition-fri-error' query-error D
    empty-query-error :: prob
assumes false-statement: ¬ exists-valid-trace
and wf: staged-budget-wellformed budgets
and controlled: staged-adversary-controlled budgets A
and trace-fri-bound:
  wp-event (checked-staged-security-experiment-with-data-state A)
    (staged-security-with-data-state-verifier-event
     trace-fri-bad-with-partial-candidates)
    adversary-initial-state ≤ trace-fri-error'
and comp-fri-bound:
  wp-event (checked-staged-security-experiment-with-data-state A)
    (staged-security-with-data-state-verifier-event
     composition-fri-bad-with-partial-candidates)
    adversary-initial-state ≤ composition-fri-error'
and query-bound:
  wp-event (checked-staged-security-experiment-with-data-state A)
    (staged-security-with-data-state-verifier-event
     query-bad-with-aligned-partial-candidates)
    adversary-initial-state ≤ query-error
and drift-bound:
  wp-event
    (checked-staged-security-experiment-with-actual-alpha-prefix-data-state
     A)
    checked-staged-security-with-actual-alpha-prefix-relevant-partial-header-candidate-drift
    adversary-initial-state ≤ D
and empty-trace-fri-bound:
  wp-event (checked-staged-security-experiment-with-data-state A)
    (staged-security-with-data-state-verifier-event

```

trace-fri-bad-with-empty-composition-header-candidates)
adversary-initial-state $\leq$ *trace-fri-error*
and *current-empty-bound*:
wp-event (*checked-staged-security-experiment-with-data-state A*)
(staged-security-with-data-state-current-empty-query-opening-hit-at 0)
adversary-initial-state $\leq$ *empty-query-error*
shows *checked-staged-adversary-acceptance-probability A* $\leq$
hash-collision-budget-value 0
(staged-attacker-query-budget budgets +
staged-challenge-query-budget + verifier-hash-query-budget) +
(composition-error-bound +
staged-phase-relation-error size
(staged-alpha-search-queries budgets 0) +
hash-collision-budget-value 0
(staged-alpha-search-queries budgets 0) +
staged-phase-relation-error size
(staged-alpha-search-queries budgets 0) +
D) +
trace-fri-error' + composition-fri-error' +
query-error +
(composition-error-bound +
staged-phase-relation-error size
(staged-alpha-search-queries budgets 0) +
hash-collision-budget-value 0
(staged-alpha-search-queries budgets 0) +
staged-phase-relation-error size
(staged-alpha-search-queries budgets 0) +
D + trace-fri-error + empty-query-error)
proof –
have *comp-bound*:
wp-event (*checked-staged-security-experiment-with-data-state A*)
(staged-security-with-data-state-verifier-event
composition-bad-with-partial-candidates)
adversary-initial-state $\leq$
composition-error-bound +
staged-phase-relation-error size
(staged-alpha-search-queries budgets 0) +
hash-collision-budget-value 0
(staged-alpha-search-queries budgets 0) +
staged-phase-relation-error size
(staged-alpha-search-queries budgets 0) +
D
by (*rule*
checked-staged-security-with-data-state-partial-candidate-composition-bad-bound-from-relevant-drift-and-budgets
[OF false-statement wf controlled drift-bound])
have *merkle-bound*:
wp-event (*checked-staged-security-experiment-with-data-state A*)
(staged-security-with-data-state-verifier-event
partial-merkle-inconsistency-bad)

adversary-initial-state $\leq$
hash-collision-budget-value 0
 (*staged-attacker-query-budget* *budgets* +
 staged-challenge-query-budget + *verifier-hash-query-budget*)
by (rule
 checked-staged-security-with-data-state-partial-merkle-inconsistency-bad-bound-controlled
 [*OF wf controlled*])
have *empty-query-bound*:
 wp-event (*checked-staged-security-experiment-with-data-state* *A*)
 (*staged-security-with-data-state-verifier-event*
 query-bad-with-empty-composition-header-candidates)
 adversary-initial-state $\leq$ *empty-query-error*
by (rule *order-trans*[
 OF checked-staged-security-with-data-state-empty-query-bad-le-current-empty-opening-hit-at-zero
 OF wf controlled] *current-empty-bound*])
have *empty-header-bound*:
 wp-event (*checked-staged-security-experiment-with-data-state* *A*)
 (*staged-security-with-data-state-verifier-event*
 soundness-bad-event-partial-candidate-empty-header)
 adversary-initial-state $\leq$
 composition-error-bound +
 staged-phase-relation-error *size*
 (*staged-alpha-search-queries* *budgets* 0) +
 hash-collision-budget-value 0
 (*staged-alpha-search-queries* *budgets* 0) +
 staged-phase-relation-error *size*
 (*staged-alpha-search-queries* *budgets* 0) +
 D + *trace-fri-error* + *empty-query-error*
by (rule
 checked-staged-security-with-data-state-empty-header-bad-event-bound-from-relevant-drift-trace-fri-and-query
 [*OF false-statement wf controlled drift-bound empty-trace-fri-bound*
 empty-query-bound])
show ?thesis
by (rule
 checked-staged-soundness-from-aligned-partial-candidate-components-and-empty-header-bound
 [*OF false-statement merkle-bound comp-bound trace-fri-bound*
 comp-fri-bound query-bound empty-header-bound])
qed

lemma *checked-staged-soundness-from-aligned-transcript-partial-candidate-components-and-current-empty-query*:
fixes *trace-fri-error'* *composition-fri-error'* *query-error* *P D*
empty-query-error :: *prob*
assumes *false-statement*: $\neg$ *exists-valid-trace*
and *wf*: *staged-budget-wellformed* *budgets*
and *controlled*: *staged-adversary-controlled* *budgets* *A*
and *trace-fri-bound*:
 wp-event (*checked-staged-security-experiment-with-data-state* *A*)
 (*staged-security-with-data-state-verifier-event*
 trace-fri-bad-with-partial-candidates)

$\text{adversary-initial-state} \leq \text{trace-fri-error}'$
and *comp-fri-bound*:
 $\text{wp-event } (\text{checked-staged-security-experiment-with-data-state } A)$
 $(\text{staged-security-with-data-state-verifier-event}$
 $\text{composition-fri-bad-with-partial-candidates})$
 $\text{adversary-initial-state} \leq \text{composition-fri-error}'$
and *query-bound*:
 $\text{wp-event } (\text{checked-staged-security-experiment-with-data-state } A)$
 $(\text{staged-security-with-data-state-verifier-event}$
 $\text{query-bad-with-aligned-transcript-partial-candidates})$
 $\text{adversary-initial-state} \leq \text{query-error}$
and *prefix-bound*:
 wp-event
 $(\text{checked-staged-security-experiment-with-actual-alpha-prefix-data-state}$
 $A)$
 $(\text{checked-staged-security-with-actual-alpha-prefix-bad-set-hit}$
 $\text{composition-trace-bad-alpha-space})$
 $\text{adversary-initial-state} \leq P$
and *drift-bound*:
 wp-event
 $(\text{checked-staged-security-experiment-with-actual-alpha-prefix-data-state}$
 $A)$
 $(\text{checked-staged-security-with-actual-alpha-prefix-partial-header-candidate-drift}$
 $\text{composition-trace-bad-alpha-space})$
 $\text{adversary-initial-state} \leq D$
and *empty-trace-fri-bound*:
 $\text{wp-event } (\text{checked-staged-security-experiment-with-data-state } A)$
 $(\text{staged-security-with-data-state-verifier-event}$
 $\text{trace-fri-bad-with-empty-composition-header-candidates})$
 $\text{adversary-initial-state} \leq \text{trace-fri-error}$
and *current-empty-bound*:
 $\text{wp-event } (\text{checked-staged-security-experiment-with-data-state } A)$
 $(\text{staged-security-with-data-state-current-empty-query-opening-hit-at } 0)$
 $\text{adversary-initial-state} \leq \text{empty-query-error}$
shows $\text{checked-staged-adversary-acceptance-probability } A \leq$
 $\text{hash-collision-budget-value } 0$
 $(\text{staged-attacker-query-budget budgets} +$
 $\text{staged-challenge-query-budget} + \text{verifier-hash-query-budget}) +$
 $(P + D +$
 $\text{staged-phase-relation-error size}$
 $(\text{staged-alpha-search-queries budgets } 0)) +$
 $\text{trace-fri-error}' + \text{composition-fri-error}' +$
 $\text{query-error} +$
 $(P + D +$
 $\text{staged-phase-relation-error size}$
 $(\text{staged-alpha-search-queries budgets } 0) +$
 $\text{trace-fri-error} + \text{empty-query-error})$
proof –
have *comp-bound*:

wp-event (checked-staged-security-experiment-with-data-state A)
(staged-security-with-data-state-verifier-event
composition-bad-with-partial-candidates)
adversary-initial-state $\leq$
P + D +
staged-phase-relation-error size
(staged-alpha-search-queries budgets 0)
by (rule
checked-staged-security-with-data-state-partial-candidate-composition-bad-bound-from-prefix-drift-and-budgets
[OF false-statement wf controlled prefix-bound drift-bound])
have *merkle-bound:*
wp-event (checked-staged-security-experiment-with-data-state A)
(staged-security-with-data-state-verifier-event
partial-merkle-inconsistency-bad)
adversary-initial-state $\leq$
hash-collision-budget-value 0
(staged-attacker-query-budget budgets +
staged-challenge-query-budget + verifier-hash-query-budget)
by (rule
checked-staged-security-with-data-state-partial-merkle-inconsistency-bad-bound-controlled
[OF wf controlled])
have *empty-header-bound:*
wp-event (checked-staged-security-experiment-with-data-state A)
(staged-security-with-data-state-verifier-event
soundness-bad-event-partial-candidate-empty-header)
adversary-initial-state $\leq$
P + D +
staged-phase-relation-error size
(staged-alpha-search-queries budgets 0) +
trace-fri-error + empty-query-error
by (rule
checked-staged-security-with-data-state-empty-header-bad-event-bound-from-prefix-drift-trace-fri-and-current-emp
[OF false-statement wf controlled prefix-bound drift-bound
empty-trace-fri-bound current-empty-bound])
have *bound:*
checked-staged-adversary-acceptance-probability A $\leq$
hash-collision-budget-value 0
(staged-attacker-query-budget budgets +
staged-challenge-query-budget + verifier-hash-query-budget) +
(P + D +
staged-phase-relation-error size
(staged-alpha-search-queries budgets 0)) +
trace-fri-error' + composition-fri-error' +
query-error +
(P + D +
staged-phase-relation-error size
(staged-alpha-search-queries budgets 0) +
trace-fri-error + empty-query-error)
by (rule

checked-staged-soundness-from-aligned-transcript-partial-candidate-components-and-empty-header-bound
 [OF *false-statement merkle-bound comp-bound trace-fri-bound*
comp-fri-bound query-bound empty-header-bound])
then show ?thesis .
qed

lemma *checked-staged-soundness-from-aligned-transcript-partial-candidate-components-and-current-empty-query-from*
fixes *trace-fri-error'* *composition-fri-error'* *query-error D*
empty-query-error :: prob
assumes *false-statement: $\neg$ exists-valid-trace*
and *wf: staged-budget-wellformed budgets*
and *controlled: staged-adversary-controlled budgets A*
and *trace-fri-bound:*
wp-event (checked-staged-security-experiment-with-data-state A)
(staged-security-with-data-state-verifier-event
trace-fri-bad-with-partial-candidates)
adversary-initial-state $\leq$ trace-fri-error'
and *comp-fri-bound:*
wp-event (checked-staged-security-experiment-with-data-state A)
(staged-security-with-data-state-verifier-event
composition-fri-bad-with-partial-candidates)
adversary-initial-state $\leq$ composition-fri-error'
and *query-bound:*
wp-event (checked-staged-security-experiment-with-data-state A)
(staged-security-with-data-state-verifier-event
query-bad-with-aligned-transcript-partial-candidates)
adversary-initial-state $\leq$ query-error
and *drift-bound:*
wp-event
(checked-staged-security-experiment-with-actual-alpha-prefix-data-state
A)
(checked-staged-security-with-actual-alpha-prefix-partial-header-candidate-drift
composition-trace-bad-alpha-space)
adversary-initial-state $\leq$ D
and *empty-trace-fri-bound:*
wp-event (checked-staged-security-experiment-with-data-state A)
(staged-security-with-data-state-verifier-event
trace-fri-bad-with-empty-composition-header-candidates)
adversary-initial-state $\leq$ trace-fri-error
and *current-empty-bound:*
wp-event (checked-staged-security-experiment-with-data-state A)
(staged-security-with-data-state-current-empty-query-opening-hit-at 0)
adversary-initial-state $\leq$ empty-query-error
shows *checked-staged-adversary-acceptance-probability A $\leq$*
hash-collision-budget-value 0
(staged-attacker-query-budget budgets +
staged-challenge-query-budget + verifier-hash-query-budget) +
(composition-error-bound +
staged-phase-relation-error size

(staged-alpha-search-queries budgets 0) +
 hash-collision-budget-value 0
 (staged-alpha-search-queries budgets 0) +
 D +
 staged-phase-relation-error size
 (staged-alpha-search-queries budgets 0)) +
 trace-fri-error' + composition-fri-error' +
 query-error +
 (composition-error-bound +
 staged-phase-relation-error size
 (staged-alpha-search-queries budgets 0) +
 hash-collision-budget-value 0
 (staged-alpha-search-queries budgets 0) +
 D +
 staged-phase-relation-error size
 (staged-alpha-search-queries budgets 0) +
 trace-fri-error + empty-query-error)
proof –
 let ?P =
 composition-error-bound +
 staged-phase-relation-error size
 (staged-alpha-search-queries budgets 0) +
 hash-collision-budget-value 0
 (staged-alpha-search-queries budgets 0)
have transcript-bound:
 wp-event (checked-staged-transcript-with-alpha-prefix-program A)
 (checked-staged-transcript-actual-alpha-prefix-bad-set-hit
 composition-trace-bad-alpha-space)
 adversary-initial-state ≤ ?P
by (rule
 checked-staged-transcript-with-alpha-prefix-actual-bad-set-hit-bound-from-budgets
 [OF wf controlled])
have prefix-bound:
 wp-event
 (checked-staged-security-experiment-with-actual-alpha-prefix-data-state
 A)
 (checked-staged-security-with-actual-alpha-prefix-bad-set-hit
 composition-trace-bad-alpha-space)
 adversary-initial-state ≤ ?P
by (rule
 checked-staged-security-with-actual-alpha-prefix-bad-set-hit-bound-from-transcript
 [OF transcript-bound])
show ?thesis
by (rule
 checked-staged-soundness-from-aligned-transcript-partial-candidate-components-and-current-empty-query
 [OF false-statement wf controlled trace-fri-bound comp-fri-bound
 query-bound prefix-bound drift-bound empty-trace-fri-bound
 current-empty-bound])
qed

lemma *checked-staged-soundness-from-aligned-partial-candidate-components-and-current-empty-query-from-drift:*
fixes *trace-fri-error'* *composition-fri-error'* *query-error D*
empty-query-error :: prob
assumes *false-statement: $\neg$ exists-valid-trace*
and *wf: staged-budget-wellformed budgets*
and *controlled: staged-adversary-controlled budgets A*
and *trace-fri-bound:*
wp-event (checked-staged-security-experiment-with-data-state A)
(staged-security-with-data-state-verifier-event
trace-fri-bad-with-partial-candidates)
adversary-initial-state $\leq$ trace-fri-error'
and *comp-fri-bound:*
wp-event (checked-staged-security-experiment-with-data-state A)
(staged-security-with-data-state-verifier-event
composition-fri-bad-with-partial-candidates)
adversary-initial-state $\leq$ composition-fri-error'
and *query-bound:*
wp-event (checked-staged-security-experiment-with-data-state A)
(staged-security-with-data-state-verifier-event
query-bad-with-aligned-partial-candidates)
adversary-initial-state $\leq$ query-error
and *drift-bound:*
wp-event
(checked-staged-security-experiment-with-actual-alpha-prefix-data-state
A)
(checked-staged-security-with-actual-alpha-prefix-partial-header-candidate-drift
composition-trace-bad-alpha-space)
adversary-initial-state $\leq D$
and *empty-trace-fri-bound:*
wp-event (checked-staged-security-experiment-with-data-state A)
(staged-security-with-data-state-verifier-event
trace-fri-bad-with-empty-composition-header-candidates)
adversary-initial-state $\leq$ trace-fri-error
and *current-empty-bound:*
wp-event (checked-staged-security-experiment-with-data-state A)
(staged-security-with-data-state-current-empty-query-opening-hit-at 0)
adversary-initial-state $\leq$ empty-query-error
shows *checked-staged-adversary-acceptance-probability A $\leq$*
hash-collision-budget-value 0
(staged-attacker-query-budget budgets +
staged-challenge-query-budget + verifier-hash-query-budget) +
(composition-error-bound +
staged-phase-relation-error size
(staged-alpha-search-queries budgets 0) +
hash-collision-budget-value 0
(staged-alpha-search-queries budgets 0) +
D +
staged-phase-relation-error size


```

    (staged-alpha-search-queries budgets 0)) +
    trace-fri-error' + composition-fri-error' +
    query-error +
    (composition-error-bound +
     staged-phase-relation-error size
     (staged-alpha-search-queries budgets 0) +
     hash-collision-budget-value 0
     (staged-alpha-search-queries budgets 0) +
     D +
     staged-phase-relation-error size
     (staged-alpha-search-queries budgets 0) +
     trace-fri-error + empty-query-error)
proof -
  let ?P =
    composition-error-bound +
    staged-phase-relation-error size
    (staged-alpha-search-queries budgets 0) +
    hash-collision-budget-value 0
    (staged-alpha-search-queries budgets 0)
  have transcript-bound:
    wp-event (checked-staged-transcript-with-alpha-prefix-program A)
    (checked-staged-transcript-actual-alpha-prefix-bad-set-hit
     composition-trace-bad-alpha-space)
    adversary-initial-state ≤ ?P
  by (rule
    checked-staged-transcript-with-alpha-prefix-actual-bad-set-hit-bound-from-budgets
    [OF wf controlled])
  have prefix-bound:
    wp-event
    (checked-staged-security-experiment-with-actual-alpha-prefix-data-state
     A)
    (checked-staged-security-with-actual-alpha-prefix-bad-set-hit
     composition-trace-bad-alpha-space)
    adversary-initial-state ≤ ?P
  by (rule
    checked-staged-security-with-actual-alpha-prefix-bad-set-hit-bound-from-transcript
    [OF transcript-bound])
  show ?thesis
  by (rule
    checked-staged-soundness-from-aligned-partial-candidate-components-and-current-empty-query
    [OF false-statement wf controlled trace-fri-bound comp-fri-bound
     query-bound prefix-bound drift-bound empty-trace-fri-bound
     current-empty-bound])
qed

lemma checked-staged-soundness-from-aligned-transcript-components-and-current-partial-opening-hit-from-drift-curr
fixes trace-fri-error' composition-fri-error' query-error D
  empty-query-error :: prob
assumes false-statement: ¬ exists-valid-trace

```

and *wf*: staged-budget-wellformed budgets
and *controlled*: staged-adversary-controlled budgets *A*
and *trace-fri-bound*:
 wp-event (*checked-staged-security-experiment-with-data-state A*)
 (*staged-security-with-data-state-verifier-event*
 trace-fri-bad-with-partial-candidates)
 adversary-initial-state $\leq$ *trace-fri-error'*
and *comp-fri-bound*:
 wp-event (*checked-staged-security-experiment-with-data-state A*)
 (*staged-security-with-data-state-verifier-event*
 composition-fri-bad-with-partial-candidates)
 adversary-initial-state $\leq$ *composition-fri-error'*
and *current-query-bound*:
 wp-event (*checked-staged-security-experiment-with-data-state A*)
 staged-security-with-data-state-current-query-partial-opening-hit
 adversary-initial-state $\leq$ *query-error*
and *drift-bound*:
 wp-event
 (*checked-staged-security-experiment-with-actual-alpha-prefix-data-state*
 A)
 (*checked-staged-security-with-actual-alpha-prefix-partial-header-candidate-drift*
 composition-trace-bad-alpha-space)
 adversary-initial-state $\leq$ *D*
and *empty-trace-fri-bound*:
 wp-event (*checked-staged-security-experiment-with-data-state A*)
 (*staged-security-with-data-state-verifier-event*
 trace-fri-bad-with-empty-composition-header-candidates)
 adversary-initial-state $\leq$ *trace-fri-error*
and *current-empty-bound*:
 wp-event (*checked-staged-security-experiment-with-data-state A*)
 (*staged-security-with-data-state-current-empty-query-opening-hit-at 0*)
 adversary-initial-state $\leq$ *empty-query-error*
shows *checked-staged-adversary-acceptance-probability A* $\leq$
hash-collision-budget-value 0
 (*staged-attacker-query-budget budgets* +
 staged-challenge-query-budget + *verifier-hash-query-budget*) +
 (*composition-error-bound* +
 staged-phase-relation-error size
 (*staged-alpha-search-queries budgets 0*) +
 hash-collision-budget-value 0
 (*staged-alpha-search-queries budgets 0*) +
 D +
 staged-phase-relation-error size
 (*staged-alpha-search-queries budgets 0*)) +
 trace-fri-error' + composition-fri-error' +
 query-error +
 (*composition-error-bound* +
 staged-phase-relation-error size
 (*staged-alpha-search-queries budgets 0*) +

hash-collision-budget-value 0
(staged-alpha-search-queries budgets 0) +
D +
staged-phase-relation-error size
(staged-alpha-search-queries budgets 0) +
trace-fri-error + empty-query-error)
proof –
have *query-bound*:
wp-event (checked-staged-security-experiment-with-data-state A)
(staged-security-with-data-state-verifier-event
query-bad-with-aligned-transcript-partial-candidates)
adversary-initial-state ≤ query-error
by (rule *order-trans*[
OF checked-staged-security-with-data-state-aligned-transcript-query-bad-bound-from-current-partial-opening-hit
current-query-bound])
(use wf controlled in simp-all)
show *?thesis*
by (rule
checked-staged-soundness-from-aligned-transcript-partial-candidate-components-and-current-empty-query-from-d
[OF false-statement wf controlled trace-fri-bound comp-fri-bound
query-bound drift-bound empty-trace-fri-bound current-empty-bound])
qed

lemma *checked-staged-soundness-from-aligned-transcript-components-and-current-partial-opening-hit-from-not-prefix*
fixes *trace-fri-error' composition-fri-error' query-error N*
empty-query-error :: prob
assumes *false-statement: ¬ exists-valid-trace*
and *wf: staged-budget-wellformed budgets*
and *controlled: staged-adversary-controlled budgets A*
and *trace-fri-bound*:
wp-event (checked-staged-security-experiment-with-data-state A)
(staged-security-with-data-state-verifier-event
trace-fri-bad-with-partial-candidates)
adversary-initial-state ≤ trace-fri-error'
and *comp-fri-bound*:
wp-event (checked-staged-security-experiment-with-data-state A)
(staged-security-with-data-state-verifier-event
composition-fri-bad-with-partial-candidates)
adversary-initial-state ≤ composition-fri-error'
and *current-query-bound*:
wp-event (checked-staged-security-experiment-with-data-state A)
staged-security-with-data-state-current-query-partial-opening-hit
adversary-initial-state ≤ query-error
and *not-prefix-bound*:
wp-event
(checked-staged-security-experiment-with-actual-alpha-prefix-data-state
A)
(checked-staged-security-with-actual-alpha-prefix-partial-header-candidate-not-prefix-bound
composition-trace-bad-alpha-space)

$\text{adversary-initial-state} \leq N$
and *empty-trace-fri-bound*:
 $\text{wp-event } (\text{checked-staged-security-experiment-with-data-state } A)$
 $(\text{staged-security-with-data-state-verifier-event}$
 $\text{trace-fri-bad-with-empty-composition-header-candidates})$
 $\text{adversary-initial-state} \leq \text{trace-fri-error}$
and *current-empty-bound*:
 $\text{wp-event } (\text{checked-staged-security-experiment-with-data-state } A)$
 $(\text{staged-security-with-data-state-current-empty-query-opening-hit-at } 0)$
 $\text{adversary-initial-state} \leq \text{empty-query-error}$
shows *checked-staged-adversary-acceptance-probability* $A \leq$
 $\text{hash-collision-budget-value } 0$
 $(\text{staged-attacker-query-budget budgets} +$
 $\text{staged-challenge-query-budget} + \text{verifier-hash-query-budget}) +$
 $(\text{composition-error-bound} +$
 $\text{staged-phase-relation-error size}$
 $(\text{staged-alpha-search-queries budgets } 0) +$
 $\text{hash-collision-budget-value } 0$
 $(\text{staged-alpha-search-queries budgets } 0) +$
 $N +$
 $\text{staged-phase-relation-error size}$
 $(\text{staged-alpha-search-queries budgets } 0)) +$
 $\text{trace-fri-error}' + \text{composition-fri-error}' +$
 $\text{query-error} +$
 $(\text{composition-error-bound} +$
 $\text{staged-phase-relation-error size}$
 $(\text{staged-alpha-search-queries budgets } 0) +$
 $\text{hash-collision-budget-value } 0$
 $(\text{staged-alpha-search-queries budgets } 0) +$
 $N +$
 $\text{staged-phase-relation-error size}$
 $(\text{staged-alpha-search-queries budgets } 0) +$
 $\text{trace-fri-error} + \text{empty-query-error})$
proof –
have *query-bound*:
 $\text{wp-event } (\text{checked-staged-security-experiment-with-data-state } A)$
 $(\text{staged-security-with-data-state-verifier-event}$
 $\text{query-bad-with-aligned-transcript-partial-candidates})$
 $\text{adversary-initial-state} \leq \text{query-error}$
by (rule *order-trans*[
 $OF \text{checked-staged-security-with-data-state-aligned-transcript-query-bad-bound-from-current-partial-opening-hit}$
 $\text{current-query-bound}]$)
 $(\text{use wf controlled in simp-all})$
have *drift-bound*:
 wp-event
 $(\text{checked-staged-security-experiment-with-actual-alpha-prefix-data-state}$
 $A)$
 $(\text{checked-staged-security-with-actual-alpha-prefix-partial-header-candidate-drift}$
 $\text{composition-trace-bad-alpha-space})$

$adversary-initial-state \leq N$
proof –
have *wp-event*
 (*checked-staged-security-experiment-with-actual-alpha-prefix-data-state*
 A)
 (*checked-staged-security-with-actual-alpha-prefix-partial-header-candidate-drift*
 composition-trace-bad-alpha-space)
 $adversary-initial-state \leq$
wp-event
 (*checked-staged-security-experiment-with-actual-alpha-prefix-data-state*
 A)
 (*checked-staged-security-with-actual-alpha-prefix-partial-header-candidate-not-prefix-bound*
 composition-trace-bad-alpha-space)
 $adversary-initial-state$
by (*rule*
 checked-staged-security-with-actual-alpha-prefix-partial-header-candidate-drift-le-not-prefix-bound
 [*OF wf controlled*])
also have $\dots \leq N$
by (*rule not-prefix-bound*)
finally show *?thesis* .
qed
have *prefix-bound*:
 wp-event
 (*checked-staged-security-experiment-with-actual-alpha-prefix-data-state*
 A)
 (*checked-staged-security-with-actual-alpha-prefix-bad-set-hit*
 composition-trace-bad-alpha-space)
 $adversary-initial-state \leq$
 $composition-error-bound +$
 $staged-phase-relation-error\ size$
 (*staged-alpha-search-queries budgets 0*) +
 $hash-collision-budget-value\ 0$
 (*staged-alpha-search-queries budgets 0*)
proof –
let *?P* =
 $composition-error-bound +$
 $staged-phase-relation-error\ size$
 (*staged-alpha-search-queries budgets 0*) +
 $hash-collision-budget-value\ 0$
 (*staged-alpha-search-queries budgets 0*)
have *transcript-bound*:
 wp-event (*checked-staged-transcript-with-alpha-prefix-program A*)
 (*checked-staged-transcript-actual-alpha-prefix-bad-set-hit*
 composition-trace-bad-alpha-space)
 $adversary-initial-state \leq ?P$
by (*rule*
 checked-staged-transcript-with-alpha-prefix-actual-bad-set-hit-bound-from-budgets
 [*OF wf controlled*])

```

show ?thesis
  by (rule
    checked-staged-security-with-actual-alpha-prefix-bad-set-hit-bound-from-transcript
    [OF transcript-bound])
qed
show ?thesis
  by (rule
    checked-staged-soundness-from-aligned-transcript-partial-candidate-components-and-current-empty-query
    [OF false-statement wf controlled trace-fri-bound comp-fri-bound
      query-bound prefix-bound drift-bound empty-trace-fri-bound
      current-empty-bound])
qed

lemma checked-staged-soundness-from-aligned-transcript-components-and-current-query-prefix-rounds-from-drift-cur
fixes trace-fri-error' composition-fri-error' D empty-query-error :: prob
and C :: nat  $\Rightarrow$  prob
assumes false-statement:  $\neg$  exists-valid-trace
and wf: staged-budget-wellformed budgets
and controlled: staged-adversary-controlled budgets A
and trace-fri-bound:
  wp-event (checked-staged-security-experiment-with-data-state A)
  (staged-security-with-data-state-verifier-event
    trace-fri-bad-with-partial-candidates)
  adversary-initial-state  $\leq$  trace-fri-error'
and comp-fri-bound:
  wp-event (checked-staged-security-experiment-with-data-state A)
  (staged-security-with-data-state-verifier-event
    composition-fri-bad-with-partial-candidates)
  adversary-initial-state  $\leq$  composition-fri-error'
and round-bound:
   $\bigwedge i. i < \text{rounds} \implies$ 
  wp-event
  (checked-staged-security-experiment-with-query-prefix-data-state A i)
  (checked-staged-security-with-query-prefix-current-query-partial-opening-hit-at i)
  adversary-initial-state  $\leq$  C i
and drift-bound:
  wp-event
  (checked-staged-security-experiment-with-actual-alpha-prefix-data-state
    A)
  (checked-staged-security-with-actual-alpha-prefix-partial-header-candidate-drift
    composition-trace-bad-alpha-space)
  adversary-initial-state  $\leq$  D
and empty-trace-fri-bound:
  wp-event (checked-staged-security-experiment-with-data-state A)
  (staged-security-with-data-state-verifier-event
    trace-fri-bad-with-empty-composition-header-candidates)
  adversary-initial-state  $\leq$  trace-fri-error
and current-empty-bound:
  wp-event (checked-staged-security-experiment-with-data-state A)

```

$(\text{staged-security-with-data-state-current-empty-query-opening-hit-at } 0)$
 $\text{adversary-initial-state} \leq \text{empty-query-error}$
shows $\text{checked-staged-adversary-acceptance-probability } A \leq$
 $\text{hash-collision-budget-value } 0$
 $(\text{staged-attacker-query-budget budgets} +$
 $\text{staged-challenge-query-budget} + \text{verifier-hash-query-budget}) +$
 $(\text{composition-error-bound} +$
 $\text{staged-phase-relation-error size}$
 $(\text{staged-alpha-search-queries budgets } 0) +$
 $\text{hash-collision-budget-value } 0$
 $(\text{staged-alpha-search-queries budgets } 0) +$
 $D +$
 $\text{staged-phase-relation-error size}$
 $(\text{staged-alpha-search-queries budgets } 0)) +$
 $\text{trace-fri-error}' + \text{composition-fri-error}' +$
 $(\sum i < \text{rounds}. C i) +$
 $(\text{composition-error-bound} +$
 $\text{staged-phase-relation-error size}$
 $(\text{staged-alpha-search-queries budgets } 0) +$
 $\text{hash-collision-budget-value } 0$
 $(\text{staged-alpha-search-queries budgets } 0) +$
 $D +$
 $\text{staged-phase-relation-error size}$
 $(\text{staged-alpha-search-queries budgets } 0) +$
 $\text{trace-fri-error} + \text{empty-query-error})$
proof –
have $\text{current-query-bound}$:
 $\text{wp-event } (\text{checked-staged-security-experiment-with-data-state } A)$
 $\text{staged-security-with-data-state-current-query-partial-opening-hit}$
 $\text{adversary-initial-state} \leq (\sum i < \text{rounds}. C i)$
by (rule
 $\text{checked-staged-security-with-data-state-current-query-partial-opening-hit-bound-from-query-prefix-rounds}$
 $[\text{OF round-bound wf controlled}])$
show $?thesis$
by (rule
 $\text{checked-staged-soundness-from-aligned-transcript-components-and-current-partial-opening-hit-from-drift-current}$
 $[\text{OF false-statement wf controlled trace-fri-bound comp-fri-bound}$
 $\text{current-query-bound drift-bound empty-trace-fri-bound}$
 $\text{current-empty-bound}])$
qed
lemma $\text{checked-staged-soundness-from-aligned-transcript-components-and-current-query-prefix-rounds-from-not-prefi}$
fixes $\text{trace-fri-error}' \text{ composition-fri-error}' N \text{ empty-query-error} :: \text{prob}$
and $C :: \text{nat} \Rightarrow \text{prob}$
assumes $\text{false-statement}: \neg \text{exists-valid-trace}$
and $\text{wf}: \text{staged-budget-wellformed budgets}$
and $\text{controlled}: \text{staged-adversary-controlled budgets } A$
and trace-fri-bound :
 $\text{wp-event } (\text{checked-staged-security-experiment-with-data-state } A)$

(staged-security-with-data-state-verifier-event
trace-fri-bad-with-partial-candidates)
adversary-initial-state $\leq$ *trace-fri-error'*
and *comp-fri-bound*:
wp-event (checked-staged-security-experiment-with-data-state A)
(staged-security-with-data-state-verifier-event
composition-fri-bad-with-partial-candidates)
adversary-initial-state $\leq$ *composition-fri-error'*
and *round-bound*:
 $\bigwedge i. i < \text{rounds} \implies$
wp-event
(checked-staged-security-experiment-with-query-prefix-data-state A i)
(checked-staged-security-with-query-prefix-current-query-partial-opening-hit-at i)
adversary-initial-state $\leq C i$
and *not-prefix-bound*:
wp-event
(checked-staged-security-experiment-with-actual-alpha-prefix-data-state
A)
(checked-staged-security-with-actual-alpha-prefix-partial-header-candidate-not-prefix-bound
composition-trace-bad-alpha-space)
adversary-initial-state $\leq N$
and *empty-trace-fri-bound*:
wp-event (checked-staged-security-experiment-with-data-state A)
(staged-security-with-data-state-verifier-event
trace-fri-bad-with-empty-composition-header-candidates)
adversary-initial-state $\leq$ *trace-fri-error*
and *current-empty-bound*:
wp-event (checked-staged-security-experiment-with-data-state A)
(staged-security-with-data-state-current-empty-query-opening-hit-at 0)
adversary-initial-state $\leq$ *empty-query-error*
shows *checked-staged-adversary-acceptance-probability A* $\leq$
hash-collision-budget-value 0
(staged-attacker-query-budget budgets +
staged-challenge-query-budget + verifier-hash-query-budget) +
(composition-error-bound +
staged-phase-relation-error size
(staged-alpha-search-queries budgets 0) +
hash-collision-budget-value 0
(staged-alpha-search-queries budgets 0) +
N +
staged-phase-relation-error size
(staged-alpha-search-queries budgets 0)) +
trace-fri-error' + composition-fri-error' +
 $(\sum i < \text{rounds}. C i) +$
(composition-error-bound +
staged-phase-relation-error size
(staged-alpha-search-queries budgets 0) +
hash-collision-budget-value 0
(staged-alpha-search-queries budgets 0) +

$N +$
staged-phase-relation-error size
(staged-alpha-search-queries budgets 0) +
trace-fri-error + empty-query-error)
proof –
have *current-query-bound*:
wp-event (checked-staged-security-experiment-with-data-state A)
staged-security-with-data-state-current-query-partial-opening-hit
adversary-initial-state $\leq (\sum i < \text{rounds}. C i)$
by (rule
checked-staged-security-with-data-state-current-query-partial-opening-hit-bound-from-query-prefix-rounds
[OF round-bound wf controlled])
show *?thesis*
by (rule
checked-staged-soundness-from-aligned-transcript-components-and-current-partial-opening-hit-from-not-prefix-curre
[OF false-statement wf controlled trace-fri-bound comp-fri-bound
current-query-bound not-prefix-bound empty-trace-fri-bound
current-empty-bound])
qed

lemma *checked-staged-soundness-from-aligned-transcript-components-and-trace-index-path-bounds-from-not-prefix-cu*
fixes *trace-fri-error' composition-fri-error' N empty-query-error :: prob*
and *Q :: nat $\Rightarrow$ prob*
assumes *false-statement: $\neg$ exists-valid-trace*
and *wf: staged-budget-wellformed budgets*
and *controlled: staged-adversary-controlled budgets A*
and *trace-fri-bound*:
wp-event (checked-staged-security-experiment-with-data-state A)
(staged-security-with-data-state-verifier-event
trace-fri-bad-with-partial-candidates)
adversary-initial-state $\leq$ trace-fri-error'
and *comp-fri-bound*:
wp-event (checked-staged-security-experiment-with-data-state A)
(staged-security-with-data-state-verifier-event
composition-fri-bad-with-partial-candidates)
adversary-initial-state $\leq$ composition-fri-error'
and *trace-frac*:
 $\bigwedge i$ *prefix prefix-state.*
i < rounds $\implies$
Some ((prefix, prefix-state), prefix-state) $\in$
set-dist
(execute (checked-staged-query-prefix-with-state A i)
adversary-initial-state) $\implies$
nnreal
(card
(staged-query-prefix-prefix-authenticated-trace-indices prefix
prefix-state)) /
nnreal (card query-sample-space) $\leq$ query-error-bound
and *path-bound*:

$\bigwedge i. i < \text{rounds} \implies$
 wp-event
 $(\text{checked-staged-security-experiment-with-query-prefix-data-state } A \ i)$
 $(\text{checked-staged-security-with-query-prefix-current-path-output-hit-at } i)$
 $\text{adversary-initial-state} \leq Q \ i$
and not-prefix-bound :
 wp-event
 $(\text{checked-staged-security-experiment-with-actual-alpha-prefix-data-state } A)$
 $(\text{checked-staged-security-with-actual-alpha-prefix-partial-header-candidate-not-prefix-bound composition-trace-bad-alpha-space})$
 $\text{adversary-initial-state} \leq N$
and $\text{empty-trace-fri-bound}$:
 $\text{wp-event } (\text{checked-staged-security-experiment-with-data-state } A)$
 $(\text{staged-security-with-data-state-verifier-event trace-fri-bad-with-empty-composition-header-candidates})$
 $\text{adversary-initial-state} \leq \text{trace-fri-error}$
and $\text{current-empty-bound}$:
 $\text{wp-event } (\text{checked-staged-security-experiment-with-data-state } A)$
 $(\text{staged-security-with-data-state-current-empty-query-opening-hit-at } 0)$
 $\text{adversary-initial-state} \leq \text{empty-query-error}$
shows $\text{checked-staged-adversary-acceptance-probability } A \leq$
 $\text{hash-collision-budget-value } 0$
 $(\text{staged-attacker-query-budget budgets} +$
 $\text{staged-challenge-query-budget} + \text{verifier-hash-query-budget}) +$
 $(\text{composition-error-bound} +$
 $\text{staged-phase-relation-error size}$
 $(\text{staged-alpha-search-queries budgets } 0) +$
 $\text{hash-collision-budget-value } 0$
 $(\text{staged-alpha-search-queries budgets } 0) +$
 $N +$
 $\text{staged-phase-relation-error size}$
 $(\text{staged-alpha-search-queries budgets } 0)) +$
 $\text{trace-fri-error}' + \text{composition-fri-error}' +$
 $(\sum i < \text{rounds}.$
 $\text{staged-phase-relation-error size}$
 $(\text{staged-query-search-queries budgets } i + 1) +$
 $\text{query-error-bound} + Q \ i) +$
 $(\text{composition-error-bound} +$
 $\text{staged-phase-relation-error size}$
 $(\text{staged-alpha-search-queries budgets } 0) +$
 $\text{hash-collision-budget-value } 0$
 $(\text{staged-alpha-search-queries budgets } 0) +$
 $N +$
 $\text{staged-phase-relation-error size}$
 $(\text{staged-alpha-search-queries budgets } 0) +$
 $\text{trace-fri-error} + \text{empty-query-error})$
proof –
have round-bound :

```

 $\bigwedge i. i < \text{rounds} \implies$ 
  wp-event
    (checked-staged-security-experiment-with-query-prefix-data-state  $A\ i$ )
    (checked-staged-security-with-query-prefix-current-query-partial-opening-hit-at  $i$ )
    adversary-initial-state  $\leq$ 
    staged-phase-relation-error size
      (staged-query-search-queries budgets  $i + 1$ ) +
      query-error-bound +  $Q\ i$ 
proof –
  fix  $i$ 
  assume  $i\text{-bound}$ :  $i < \text{rounds}$ 
  have  $\text{prefix-hit-bound}$ :
    wp-event
      (checked-staged-security-experiment-with-query-prefix-data-state  $A\ i$ )
      (checked-staged-security-with-query-prefix-current-prefix-authenticated-hit-at  $i$ )
      adversary-initial-state  $\leq$ 
      staged-phase-relation-error size
        (staged-query-search-queries budgets  $i + 1$ ) + query-error-bound
    by (rule
      checked-staged-security-with-query-prefix-current-prefix-authenticated-hit-at-bound-from-trace-indices
        [OF wf controlled  $i\text{-bound}$ ])
      (use trace-frac[OF  $i\text{-bound}$ ] in blast)
  show
    wp-event
      (checked-staged-security-experiment-with-query-prefix-data-state  $A\ i$ )
      (checked-staged-security-with-query-prefix-current-query-partial-opening-hit-at  $i$ )
      adversary-initial-state  $\leq$ 
      staged-phase-relation-error size
        (staged-query-search-queries budgets  $i + 1$ ) +
        query-error-bound +  $Q\ i$ 
    by (rule
      checked-staged-security-with-query-prefix-current-query-partial-opening-hit-at-bound-from-prefix-and-path
        [OF wf controlled  $i\text{-bound}$   $\text{prefix-hit-bound}$   $\text{path-bound}$ [OF  $i\text{-bound}$ ]])
  qed
show ?thesis
  by (rule
    checked-staged-soundness-from-aligned-transcript-components-and-current-query-prefix-rounds-from-not-prefix-cu
      [OF false-statement wf controlled trace-fri-bound comp-fri-bound
        round-bound not-prefix-bound empty-trace-fri-bound
        current-empty-bound])
  qed

lemma checked-staged-soundness-from-aligned-transcript-components-and-trace-index-structured-path-bounds-from-n
  fixes trace-fri-error' composition-fri-error'  $N$  empty-query-error :: prob
  and  $T\ C :: \text{nat} \Rightarrow \text{prob}$ 
assumes false-statement:  $\neg \text{exists-valid-trace}$ 
and wf: staged-budget-wellformed budgets
and controlled: staged-adversary-controlled budgets  $A$ 
and trace-fri-bound:

```

$wp\text{-}event \text{ (checked-staged-security-experiment-with-data-state } A)$
 $\text{ (staged-security-with-data-state-verifier-event}$
 $\text{ trace-fri-bad-with-partial-candidates)}$
 $adversary\text{-}initial\text{-}state \leq trace\text{-}fri\text{-}error'$
and $comp\text{-}fri\text{-}bound$:
 $wp\text{-}event \text{ (checked-staged-security-experiment-with-data-state } A)$
 $\text{ (staged-security-with-data-state-verifier-event}$
 $\text{ composition-fri-bad-with-partial-candidates)}$
 $adversary\text{-}initial\text{-}state \leq composition\text{-}fri\text{-}error'$
and $trace\text{-}frac$:
 $\bigwedge i \text{ prefix prefix-state.}$
 $i < rounds \implies$
 $Some ((prefix, prefix-state), prefix-state) \in$
 $set\text{-}dist$
 $\text{ (execute (checked-staged-query-prefix-with-state } A \ i)$
 $\text{ adversary-initial-state}) \implies$
 $nnreal$
 (card
 $\text{ (staged-query-prefix-prefix-authenticated-trace-indices prefix}$
 prefix-state)) /
 $nnreal \text{ (card query-sample-space)} \leq query\text{-}error\text{-}bound$
and $trace\text{-}path\text{-}bound$:
 $\bigwedge i. i < rounds \implies$
 $wp\text{-}event$
 $\text{ (checked-staged-security-experiment-with-query-prefix-data-state } A \ i)$
 $\text{ (checked-staged-security-with-query-prefix-current-trace-path-output-hit-at } i)$
 $adversary\text{-}initial\text{-}state \leq T \ i$
and $composition\text{-}path\text{-}bound$:
 $\bigwedge i. i < rounds \implies$
 $wp\text{-}event$
 $\text{ (checked-staged-security-experiment-with-query-prefix-data-state } A \ i)$
 $\text{ (checked-staged-security-with-query-prefix-current-composition-path-output-hit-at } i)$
 $adversary\text{-}initial\text{-}state \leq C \ i$
and $not\text{-}prefix\text{-}bound$:
 $wp\text{-}event$
 $\text{ (checked-staged-security-experiment-with-actual-alpha-prefix-data-state}$
 $\text{ } A)$
 $\text{ (checked-staged-security-with-actual-alpha-prefix-partial-header-candidate-not-prefix-bound}$
 $\text{ composition-trace-bad-alpha-space)}$
 $adversary\text{-}initial\text{-}state \leq N$
and $empty\text{-}trace\text{-}fri\text{-}bound$:
 $wp\text{-}event \text{ (checked-staged-security-experiment-with-data-state } A)$
 $\text{ (staged-security-with-data-state-verifier-event}$
 $\text{ trace-fri-bad-with-empty-composition-header-candidates)}$
 $adversary\text{-}initial\text{-}state \leq trace\text{-}fri\text{-}error$
and $current\text{-}empty\text{-}bound$:
 $wp\text{-}event \text{ (checked-staged-security-experiment-with-data-state } A)$
 $\text{ (staged-security-with-data-state-current-empty-query-opening-hit-at } 0)$
 $adversary\text{-}initial\text{-}state \leq empty\text{-}query\text{-}error$

shows *checked-staged-adversary-acceptance-probability* $A \leq$
hash-collision-budget-value 0
 (*staged-attacker-query-budget* budgets +
staged-challenge-query-budget + *verifier-hash-query-budget*) +
 (*composition-error-bound* +
staged-phase-relation-error size
 (*staged-alpha-search-queries* budgets 0) +
hash-collision-budget-value 0
 (*staged-alpha-search-queries* budgets 0) +
N +
staged-phase-relation-error size
 (*staged-alpha-search-queries* budgets 0)) +
trace-fri-error' + *composition-fri-error'* +
 ($\sum i < \text{rounds}$.
staged-phase-relation-error size
 (*staged-query-search-queries* budgets $i + 1$) +
query-error-bound + $T i + C i$) +
 (*composition-error-bound* +
staged-phase-relation-error size
 (*staged-alpha-search-queries* budgets 0) +
hash-collision-budget-value 0
 (*staged-alpha-search-queries* budgets 0) +
N +
staged-phase-relation-error size
 (*staged-alpha-search-queries* budgets 0) +
trace-fri-error + *empty-query-error*)

proof –

have *round-bound*:

$\bigwedge i. i < \text{rounds} \implies$

wp-event

(*checked-staged-security-experiment-with-query-prefix-data-state* $A i$)
 (*checked-staged-security-with-query-prefix-current-query-partial-opening-hit-at* i)
adversary-initial-state $\leq$
staged-phase-relation-error size
 (*staged-query-search-queries* budgets $i + 1$) +
query-error-bound + $T i + C i$

proof –

fix i

assume *i-bound*: $i < \text{rounds}$

have *prefix-hit-bound*:

wp-event

(*checked-staged-security-experiment-with-query-prefix-data-state* $A i$)
 (*checked-staged-security-with-query-prefix-current-prefix-authenticated-hit-at* i)
adversary-initial-state $\leq$
staged-phase-relation-error size
 (*staged-query-search-queries* budgets $i + 1$) + *query-error-bound*

by (rule

checked-staged-security-with-query-prefix-current-prefix-authenticated-hit-at-bound-from-trace-indices
 [OF wf controlled *i-bound*])

```

    (use trace-frac[OF i-bound] in blast)
show
  wp-event
    (checked-staged-security-experiment-with-query-prefix-data-state A i)
    (checked-staged-security-with-query-prefix-current-query-partial-opening-hit-at i)
    adversary-initial-state ≤
  staged-phase-relation-error size
    (staged-query-search-queries budgets i + 1) +
  query-error-bound + T i + C i
  by (rule
    checked-staged-security-with-query-prefix-current-query-partial-opening-hit-at-bound-from-prefix-and-structured-
      [OF wf controlled i-bound prefix-hit-bound trace-path-bound[OF i-bound]
        composition-path-bound[OF i-bound]])
qed
show ?thesis
  by (rule
    checked-staged-soundness-from-aligned-transcript-components-and-current-query-prefix-rounds-from-not-prefix-cv
      [OF false-statement wf controlled trace-fri-bound comp-fri-bound
        round-bound not-prefix-bound empty-trace-fri-bound
        current-empty-bound])
qed

lemma checked-staged-soundness-from-aligned-transcript-components-and-conceptual-structured-paths-from-not-prefix-cv
  fixes trace-fri-error' composition-fri-error' N empty-query-error :: prob
  and T C :: nat ⇒ prob
  assumes false-statement: ¬ exists-valid-trace
  and wf: staged-budget-wellformed budgets
  and controlled: staged-adversary-controlled budgets A
  and trace-fri-bound:
    wp-event (checked-staged-security-experiment-with-data-state A)
      (staged-security-with-data-state-verifier-event
        trace-fri-bad-with-partial-candidates)
      adversary-initial-state ≤ trace-fri-error'
  and comp-fri-bound:
    wp-event (checked-staged-security-experiment-with-data-state A)
      (staged-security-with-data-state-verifier-event
        composition-fri-bad-with-partial-candidates)
      adversary-initial-state ≤ composition-fri-error'
  and conceptual-context:
    ∧ i prefix prefix-state.
      i < rounds ⇒
        Some ((prefix, prefix-state), prefix-state) ∈
          set-dist
            (execute (checked-staged-query-prefix-with-state A i)
              adversary-initial-state) ⇒
        ¬ hash-map-output-collision prefix-state ∧
        trace-table-low-degree
          (query-prefix-trace-conceptual-table prefix prefix-state) ∧
        composition-table-low-degree maxDegree

```

$(\text{query-prefix-composition-conceptual-table } \text{prefix } \text{prefix-state}) \wedge$
 $\neg \text{all-queries-consistent}$
 $(\text{query-prefix-trace-conceptual-table } \text{prefix } \text{prefix-state})$
 $(\text{query-prefix-composition-conceptual-table } \text{prefix } \text{prefix-state})$
 $(\text{sqp-alphas } \text{prefix})$
and *trace-path-bound*:
 $\bigwedge i. i < \text{rounds} \implies$
 wp-event
 $(\text{checked-staged-security-experiment-with-query-prefix-data-state } A \ i)$
 $(\text{checked-staged-security-with-query-prefix-current-trace-path-output-hit-at } i)$
 $\text{adversary-initial-state} \leq T \ i$
and *composition-path-bound*:
 $\bigwedge i. i < \text{rounds} \implies$
 wp-event
 $(\text{checked-staged-security-experiment-with-query-prefix-data-state } A \ i)$
 $(\text{checked-staged-security-with-query-prefix-current-composition-path-output-hit-at } i)$
 $\text{adversary-initial-state} \leq C \ i$
and *not-prefix-bound*:
 wp-event
 $(\text{checked-staged-security-experiment-with-actual-alpha-prefix-data-state } A)$
 $(\text{checked-staged-security-with-actual-alpha-prefix-partial-header-candidate-not-prefix-bound } \text{composition-trace-bad-alpha-space})$
 $\text{adversary-initial-state} \leq N$
and *empty-trace-fri-bound*:
 $\text{wp-event } (\text{checked-staged-security-experiment-with-data-state } A)$
 $(\text{staged-security-with-data-state-verifier-event } \text{trace-fri-bad-with-empty-composition-header-candidates})$
 $\text{adversary-initial-state} \leq \text{trace-fri-error}$
and *current-empty-bound*:
 $\text{wp-event } (\text{checked-staged-security-experiment-with-data-state } A)$
 $(\text{staged-security-with-data-state-current-empty-query-opening-hit-at } 0)$
 $\text{adversary-initial-state} \leq \text{empty-query-error}$
shows $\text{checked-staged-adversary-acceptance-probability } A \leq$
 $\text{hash-collision-budget-value } 0$
 $(\text{staged-attacker-query-budget } \text{budgets} +$
 $\text{staged-challenge-query-budget} + \text{verifier-hash-query-budget}) +$
 $(\text{composition-error-bound} +$
 $\text{staged-phase-relation-error size}$
 $(\text{staged-alpha-search-queries } \text{budgets } 0) +$
 $\text{hash-collision-budget-value } 0$
 $(\text{staged-alpha-search-queries } \text{budgets } 0) +$
 $N +$
 $\text{staged-phase-relation-error size}$
 $(\text{staged-alpha-search-queries } \text{budgets } 0)) +$
 $\text{trace-fri-error}' + \text{composition-fri-error}' +$
 $(\sum i < \text{rounds.}$
 $\text{staged-phase-relation-error size}$
 $(\text{staged-query-search-queries } \text{budgets } i + 1) +$

$query-error-bound + T\ i + C\ i) +$
 $(composition-error-bound +$
 $staged-phase-relation-error\ size$
 $(staged-alpha-search-queries\ budgets\ 0) +$
 $hash-collision-budget-value\ 0$
 $(staged-alpha-search-queries\ budgets\ 0) +$
 $N +$
 $staged-phase-relation-error\ size$
 $(staged-alpha-search-queries\ budgets\ 0) +$
 $trace-fri-error + empty-query-error)$

proof –

have *round-bound*:

$\bigwedge i. i < rounds \implies$

wp-event

$(checked-staged-security-experiment-with-query-prefix-data-state\ A\ i)$
 $(checked-staged-security-with-query-prefix-current-query-partial-opening-hit-at\ i)$
 $adversary-initial-state \leq$
 $staged-phase-relation-error\ size$
 $(staged-query-search-queries\ budgets\ i + 1) +$
 $query-error-bound + T\ i + C\ i$

proof –

fix *i*

assume *i-bound*: $i < rounds$

have *prefix-target-bound*:

wp-event $(checked-staged-query-prefix-receive-with-state\ A\ i)$

$(checked-staged-query-prefix-dynamic-index-hit$
 $staged-query-prefix-prefix-authenticated-opening-target)$
 $adversary-initial-state \leq$
 $staged-phase-relation-error\ size$
 $(staged-query-search-queries\ budgets\ i + 1) + query-error-bound$

by (rule

$checked-staged-query-prefix-prefix-authenticated-target-hit-bound-from-conceptual$
 $[OF\ wf\ controlled\ i-bound])$
 $(rule\ conceptual-context[OF\ i-bound])$

have *prefix-bound*:

wp-event

$(checked-staged-security-experiment-with-query-prefix-data-state\ A\ i)$
 $(checked-staged-security-with-query-prefix-current-prefix-authenticated-hit-at\ i)$
 $adversary-initial-state \leq$
 $staged-phase-relation-error\ size$
 $(staged-query-search-queries\ budgets\ i + 1) + query-error-bound$

by (rule

$checked-staged-security-with-query-prefix-current-prefix-authenticated-hit-at-bound-from-dynamic-target$
 $[OF\ wf\ controlled\ i-bound\ prefix-target-bound])$

show

wp-event

$(checked-staged-security-experiment-with-query-prefix-data-state\ A\ i)$
 $(checked-staged-security-with-query-prefix-current-query-partial-opening-hit-at\ i)$
 $adversary-initial-state \leq$


```

    staged-phase-relation-error size
      (staged-query-search-queries budgets  $i + 1$ ) +
      query-error-bound +  $T\ i + C\ i$ 
  by (rule
    checked-staged-security-with-query-prefix-current-query-partial-opening-hit-at-bound-from-prefix-and-structured-
      [OF wf controlled i-bound prefix-bound trace-path-bound [OF i-bound]
        composition-path-bound [OF i-bound]])
qed
show ?thesis
  by (rule
    checked-staged-soundness-from-aligned-transcript-components-and-current-query-prefix-rounds-from-not-prefix-cv
      [OF false-statement wf controlled trace-fri-bound comp-fri-bound
        round-bound not-prefix-bound empty-trace-fri-bound
        current-empty-bound])
qed

lemma checked-staged-soundness-from-aligned-transcript-components-and-trace-index-path-bounds-from-cross-data-p
  fixes trace-fri-error' composition-fri-error'  $X\ P\ R$  empty-query-error :: prob
  and  $Q :: \text{nat} \Rightarrow \text{prob}$ 
  assumes false-statement:  $\neg \text{exists-valid-trace}$ 
  and wf: staged-budget-wellformed budgets
  and controlled: staged-adversary-controlled budgets  $A$ 
  and trace-fri-bound:
    wp-event (checked-staged-security-experiment-with-data-state  $A$ )
      (staged-security-with-data-state-verifier-event
        trace-fri-bad-with-partial-candidates)
      adversary-initial-state  $\leq$  trace-fri-error'
  and comp-fri-bound:
    wp-event (checked-staged-security-experiment-with-data-state  $A$ )
      (staged-security-with-data-state-verifier-event
        composition-fri-bad-with-partial-candidates)
      adversary-initial-state  $\leq$  composition-fri-error'
  and trace-frac:
     $\bigwedge i$  prefix prefix-state.
       $i < \text{rounds} \implies$ 
      Some ((prefix, prefix-state), prefix-state)  $\in$ 
        set-dist
          (execute (checked-staged-query-prefix-with-state  $A\ i$ )
            adversary-initial-state)  $\implies$ 
      nnreal
        (card
          (staged-query-prefix-prefix-authenticated-trace-indices prefix
            prefix-state)) /
      nnreal (card query-sample-space)  $\leq$  query-error-bound
  and path-bound:
     $\bigwedge i. i < \text{rounds} \implies$ 
    wp-event
      (checked-staged-security-experiment-with-query-prefix-data-state  $A\ i$ )
      (checked-staged-security-with-query-prefix-current-path-output-hit-at  $i$ )

```

$\text{adversary-initial-state} \leq Q \ i$
and *cross-bound*:
 wp-event
 $(\text{checked-staged-security-experiment-with-actual-alpha-prefix-data-state } A)$
 $\text{checked-staged-security-with-actual-alpha-prefix-witnessed-cross-or-merkle-bad}$
 $\text{adversary-initial-state} \leq X$
and *data-pre-bound*:
 $\text{wp-event } (\text{checked-staged-security-experiment-with-data-state } A)$
 $\text{staged-security-with-data-state-transcript-pre-hit}$
 $\text{adversary-initial-state} \leq P$
and *transcript-new-bound*:
 wp-event
 $(\text{checked-staged-security-experiment-with-actual-alpha-prefix-data-state } A)$
 $\text{checked-staged-security-with-actual-alpha-prefix-witness-transcript-new-hit}$
 $\text{adversary-initial-state} \leq R$
and *empty-trace-fri-bound*:
 $\text{wp-event } (\text{checked-staged-security-experiment-with-data-state } A)$
 $(\text{staged-security-with-data-state-verifier-event}$
 $\text{trace-fri-bad-with-empty-composition-header-candidates})$
 $\text{adversary-initial-state} \leq \text{trace-fri-error}$
and *current-empty-bound*:
 $\text{wp-event } (\text{checked-staged-security-experiment-with-data-state } A)$
 $(\text{staged-security-with-data-state-current-empty-query-opening-hit-at } 0)$
 $\text{adversary-initial-state} \leq \text{empty-query-error}$
shows $\text{checked-staged-adversary-acceptance-probability } A \leq$
 $\text{hash-collision-budget-value } 0$
 $(\text{staged-attacker-query-budget budgets} +$
 $\text{staged-challenge-query-budget} + \text{verifier-hash-query-budget}) +$
 $(\text{composition-error-bound} +$
 $\text{staged-phase-relation-error size}$
 $(\text{staged-alpha-search-queries budgets } 0) +$
 $\text{hash-collision-budget-value } 0$
 $(\text{staged-alpha-search-queries budgets } 0) +$
 $(\text{hash-collision-budget-value } 0$
 $(\text{staged-alpha-search-queries budgets } 0) + X + (P + R) + (P + R)) +$
 $\text{staged-phase-relation-error size}$
 $(\text{staged-alpha-search-queries budgets } 0)) +$
 $\text{trace-fri-error}' + \text{composition-fri-error}' +$
 $(\sum i < \text{rounds.}$
 $\text{staged-phase-relation-error size}$
 $(\text{staged-query-search-queries budgets } i + 1) +$
 $\text{query-error-bound} + Q \ i) +$
 $(\text{composition-error-bound} +$
 $\text{staged-phase-relation-error size}$
 $(\text{staged-alpha-search-queries budgets } 0) +$
 $\text{hash-collision-budget-value } 0$
 $(\text{staged-alpha-search-queries budgets } 0) +$

(hash-collision-budget-value 0
 (staged-alpha-search-queries budgets 0) + X + ($P + R$) + ($P + R$)) +
 staged-phase-relation-error size
 (staged-alpha-search-queries budgets 0) +
 trace-fri-error + empty-query-error)

proof –

have transcript-pre-bound:

wp-event
 (checked-staged-security-experiment-with-actual-alpha-prefix-data-state A)
 checked-staged-security-with-actual-alpha-prefix-witness-transcript-pre-hit
 adversary-initial-state $\leq P$

by (rule
 checked-staged-security-with-actual-alpha-prefix-witness-transcript-pre-hit-bound-from-data-state-pre-hit
 [OF data-pre-bound])

have witness-hit-bound:

wp-event
 (checked-staged-security-experiment-with-actual-alpha-prefix-data-state A)
 checked-staged-security-with-actual-alpha-prefix-witness-transcript-hit
 adversary-initial-state $\leq P + R$

by (rule
 checked-staged-security-with-actual-alpha-prefix-witness-transcript-hit-bound-from-pre-and-new
 [OF transcript-pre-bound transcript-new-bound])

have no-prefix-bound:

wp-event
 (checked-staged-security-experiment-with-actual-alpha-prefix-data-state A)
 checked-staged-security-with-actual-alpha-prefix-not-prefix-no-prefix-candidate
 adversary-initial-state $\leq P + R$

by (rule order-trans
 [OF
 checked-staged-security-with-actual-alpha-prefix-not-prefix-no-prefix-candidate-le-witness-transcript-hit
 witness-hit-bound])

have path-output-bound:

wp-event
 (checked-staged-security-experiment-with-actual-alpha-prefix-data-state A)
 checked-staged-security-with-actual-alpha-prefix-not-prefix-path-output
 adversary-initial-state $\leq P + R$

by (rule order-trans
 [OF
 checked-staged-security-with-actual-alpha-prefix-not-prefix-path-output-le-witness-transcript-hit
 witness-hit-bound])

have uncovered-bound:

wp-event
 (checked-staged-security-experiment-with-actual-alpha-prefix-data-state A)
 checked-staged-security-with-actual-alpha-prefix-not-prefix-uncovered-index
 adversary-initial-state $\leq X$

by (rule order-trans
 [OF
 checked-staged-security-with-actual-alpha-prefix-not-prefix-uncovered-index-le-witnessed-cross-or-merkle-bad
 cross-bound])

have *collision-bound*:

wp-event

(*checked-staged-security-experiment-with-actual-alpha-prefix-data-state A*)
checked-staged-security-with-actual-alpha-prefix-not-prefix-collision
adversary-initial-state $\leq$
hash-collision-budget-value 0 (*staged-alpha-search-queries budgets 0*)

by (*rule*

checked-staged-security-with-actual-alpha-prefix-not-prefix-collision-bound
[OF wf controlled])

have *not-prefix-bound*:

wp-event

(*checked-staged-security-experiment-with-actual-alpha-prefix-data-state A*)
(*checked-staged-security-with-actual-alpha-prefix-partial-header-candidate-not-prefix-bound*
composition-trace-bad-alpha-space)
adversary-initial-state $\leq$
hash-collision-budget-value 0 (*staged-alpha-search-queries budgets 0*) +
X + (P + R) + (P + R)

by (*rule*

checked-staged-security-with-actual-alpha-prefix-not-prefix-bound-union-bound
[OF wf controlled collision-bound uncovered-bound no-prefix-bound
path-output-bound])

show *?thesis*

by (*rule*

checked-staged-soundness-from-aligned-transcript-components-and-trace-index-path-bounds-from-not-prefix-current-bound
[OF false-statement wf controlled trace-fri-bound comp-fri-bound
trace-frac path-bound not-prefix-bound empty-trace-fri-bound
current-empty-bound])

qed

lemma *checked-staged-soundness-from-aligned-transcript-components-and-trace-index-structured-path-bounds-from-current-bound*

fixes *trace-fri-error'* *composition-fri-error'* *X P R empty-query-error :: prob*

and *T C :: nat $\Rightarrow$ prob*

assumes *false-statement: $\neg$ exists-valid-trace*

and *wf: staged-budget-wellformed budgets*

and *controlled: staged-adversary-controlled budgets A*

and *trace-fri-bound:*

wp-event (*checked-staged-security-experiment-with-data-state A*)
(*staged-security-with-data-state-verifier-event*
trace-fri-bad-with-partial-candidates)
adversary-initial-state $\leq$ *trace-fri-error'*

and *comp-fri-bound:*

wp-event (*checked-staged-security-experiment-with-data-state A*)
(*staged-security-with-data-state-verifier-event*
composition-fri-bad-with-partial-candidates)
adversary-initial-state $\leq$ *composition-fri-error'*

and *trace-frac:*

$\bigwedge i$ *prefix prefix-state.*
i < rounds $\implies$
Some ((prefix, prefix-state), prefix-state) $\in$

set-dist
 (execute (checked-staged-query-prefix-with-state A i)
 adversary-initial-state) $\implies$
nnreal
 (card
 (staged-query-prefix-prefix-authenticated-trace-indices $prefix$
 prefix-state)) /
nnreal (card query-sample-space) $\leq$ query-error-bound
and trace-path-bound:
 $\bigwedge i. i < rounds \implies$
wp-event
 (checked-staged-security-experiment-with-query-prefix-data-state A i)
 (checked-staged-security-with-query-prefix-current-trace-path-output-hit-at i)
 adversary-initial-state $\leq T$ i
and composition-path-bound:
 $\bigwedge i. i < rounds \implies$
wp-event
 (checked-staged-security-experiment-with-query-prefix-data-state A i)
 (checked-staged-security-with-query-prefix-current-composition-path-output-hit-at i)
 adversary-initial-state $\leq C$ i
and cross-bound:
wp-event
 (checked-staged-security-experiment-with-actual-alpha-prefix-data-state
 A)
 checked-staged-security-with-actual-alpha-prefix-witnessed-cross-or-merkle-bad
 adversary-initial-state $\leq X$
and data-pre-bound:
wp-event (checked-staged-security-experiment-with-data-state A)
 staged-security-with-data-state-transcript-pre-hit
 adversary-initial-state $\leq P$
and transcript-new-bound:
wp-event
 (checked-staged-security-experiment-with-actual-alpha-prefix-data-state
 A)
 checked-staged-security-with-actual-alpha-prefix-witness-transcript-new-hit
 adversary-initial-state $\leq R$
and empty-trace-fri-bound:
wp-event (checked-staged-security-experiment-with-data-state A)
 (staged-security-with-data-state-verifier-event
 trace-fri-bad-with-empty-composition-header-candidates)
 adversary-initial-state $\leq$ trace-fri-error
and current-empty-bound:
wp-event (checked-staged-security-experiment-with-data-state A)
 (staged-security-with-data-state-current-empty-query-opening-hit-at 0)
 adversary-initial-state $\leq$ empty-query-error
shows checked-staged-adversary-acceptance-probability $A \leq$
 hash-collision-budget-value 0
 (staged-attacker-query-budget budgets +
 staged-challenge-query-budget + verifier-hash-query-budget) +

(composition-error-bound +
 staged-phase-relation-error size
 (staged-alpha-search-queries budgets 0) +
 hash-collision-budget-value 0
 (staged-alpha-search-queries budgets 0) +
 (hash-collision-budget-value 0
 (staged-alpha-search-queries budgets 0) + $X + (P + R) + (P + R)$) +
 staged-phase-relation-error size
 (staged-alpha-search-queries budgets 0)) +
 trace-fri-error' + composition-fri-error' +
 ($\sum i < \text{rounds}$.
 staged-phase-relation-error size
 (staged-query-search-queries budgets $i + 1$) +
 query-error-bound + $T i + C i$) +
 (composition-error-bound +
 staged-phase-relation-error size
 (staged-alpha-search-queries budgets 0) +
 hash-collision-budget-value 0
 (staged-alpha-search-queries budgets 0) +
 (hash-collision-budget-value 0
 (staged-alpha-search-queries budgets 0) + $X + (P + R) + (P + R)$) +
 staged-phase-relation-error size
 (staged-alpha-search-queries budgets 0) +
 trace-fri-error + empty-query-error)

proof –

have transcript-pre-bound:

 wp-event
 (checked-staged-security-experiment-with-actual-alpha-prefix-data-state A)
 checked-staged-security-with-actual-alpha-prefix-witness-transcript-pre-hit
 adversary-initial-state $\leq P$

by (rule
 checked-staged-security-with-actual-alpha-prefix-witness-transcript-pre-hit-bound-from-data-state-pre-hit
 [OF data-pre-bound])

have witness-hit-bound:

 wp-event
 (checked-staged-security-experiment-with-actual-alpha-prefix-data-state A)
 checked-staged-security-with-actual-alpha-prefix-witness-transcript-hit
 adversary-initial-state $\leq P + R$

by (rule
 checked-staged-security-with-actual-alpha-prefix-witness-transcript-hit-bound-from-pre-and-new
 [OF transcript-pre-bound transcript-new-bound])

have no-prefix-bound:

 wp-event
 (checked-staged-security-experiment-with-actual-alpha-prefix-data-state A)
 checked-staged-security-with-actual-alpha-prefix-not-prefix-no-prefix-candidate
 adversary-initial-state $\leq P + R$

by (rule order-trans
 [OF
 checked-staged-security-with-actual-alpha-prefix-not-prefix-no-prefix-candidate-le-witness-transcript-hit

$witness\text{-}hit\text{-}bound])$
have *path-output-bound*:
wp-event
 (*checked-staged-security-experiment-with-actual-alpha-prefix-data-state A*)
checked-staged-security-with-actual-alpha-prefix-not-prefix-path-output
adversary-initial-state $\leq P + R$
by (rule *order-trans*
 [OF
checked-staged-security-with-actual-alpha-prefix-not-prefix-path-output-le-witness-transcript-hit
witness-hit-bound])
have *uncovered-bound*:
wp-event
 (*checked-staged-security-experiment-with-actual-alpha-prefix-data-state A*)
checked-staged-security-with-actual-alpha-prefix-not-prefix-uncovered-index
adversary-initial-state $\leq X$
by (rule *order-trans*
 [OF
checked-staged-security-with-actual-alpha-prefix-not-prefix-uncovered-index-le-witnessed-cross-or-merkle-bad
cross-bound])
have *collision-bound*:
wp-event
 (*checked-staged-security-experiment-with-actual-alpha-prefix-data-state A*)
checked-staged-security-with-actual-alpha-prefix-not-prefix-collision
adversary-initial-state $\leq$
hash-collision-budget-value 0 (*staged-alpha-search-queries budgets 0*)
by (rule
checked-staged-security-with-actual-alpha-prefix-not-prefix-collision-bound
 [OF *wf controlled*])
have *not-prefix-bound*:
wp-event
 (*checked-staged-security-experiment-with-actual-alpha-prefix-data-state A*)
 (*checked-staged-security-with-actual-alpha-prefix-partial-header-candidate-not-prefix-bound*
composition-trace-bad-alpha-space)
adversary-initial-state $\leq$
hash-collision-budget-value 0 (*staged-alpha-search-queries budgets 0*) +
 $X + (P + R) + (P + R)$
by (rule
checked-staged-security-with-actual-alpha-prefix-not-prefix-bound-union-bound
 [OF *wf controlled collision-bound uncovered-bound no-prefix-bound*
path-output-bound])
show *?thesis*
by (rule
checked-staged-soundness-from-aligned-transcript-components-and-trace-index-structured-path-bounds-from-not-p
 [OF *false-statement wf controlled trace-fri-bound comp-fri-bound*
trace-frac trace-path-bound composition-path-bound not-prefix-bound
empty-trace-fri-bound current-empty-bound])
qed

theorem *checked-staged-soundness-from-aligned-transcript-conceptual-default-alpha-prefix-union-clean-structured-pat*

```

fixes trace-fri-error' composition-fri-error' N empty-query-error :: prob
and T C :: nat ⇒ prob
and trace-default composition-default :: 'f list
assumes false-statement: ¬ exists-valid-trace
and wf: staged-budget-wellformed budgets
and controlled: staged-adversary-controlled budgets A
and trace-fri-bound:
  wp-event (checked-staged-security-experiment-with-data-state A)
    (staged-security-with-data-state-verifier-event
      trace-fri-bad-with-partial-candidates)
    adversary-initial-state ≤ trace-fri-error'
and comp-fri-bound:
  wp-event (checked-staged-security-experiment-with-data-state A)
    (staged-security-with-data-state-verifier-event
      composition-fri-bad-with-partial-candidates)
    adversary-initial-state ≤ composition-fri-error'
and default-prefix-clean:
  ∧ i prefix prefix-state.
    i < rounds ⇒
      Some ((prefix, prefix-state), prefix-state) ∈
        set-dist
          (execute (checked-staged-query-prefix-with-state A i)
            adversary-initial-state) ⇒
            query-prefix-trace-conceptual-table-with-default prefix prefix-state
              trace-default = trace-default ∧
            query-prefix-composition-conceptual-table-with-default prefix
              prefix-state composition-default = composition-default ∧
            trace-table-low-degree trace-default ∧
            composition-table-low-degree maxDegree composition-default ∧
            trace-default ∈
              alpha-prefix-trace-table-candidates prefix-state
                (sqp-trace-root prefix) ∧
            sqp-alphas prefix ∉
              alpha-prefix-union-bad-sets prefix-state
                composition-trace-bad-alpha-space (sqp-trace-root prefix)
and trace-path-bound:
  ∧ i. i < rounds ⇒
    wp-event
      (checked-staged-security-experiment-with-query-prefix-data-state A i)
      (checked-staged-security-with-query-prefix-current-trace-path-output-hit-at i)
      adversary-initial-state ≤ T i
and composition-path-bound:
  ∧ i. i < rounds ⇒
    wp-event
      (checked-staged-security-experiment-with-query-prefix-data-state A i)
      (checked-staged-security-with-query-prefix-current-composition-path-output-hit-at i)
      adversary-initial-state ≤ C i
and not-prefix-bound:
  wp-event

```


(checked-staged-security-experiment-with-actual-alpha-prefix-data-state A)
(checked-staged-security-with-actual-alpha-prefix-partial-header-candidate-not-prefix-bound composition-trace-bad-alpha-space)
adversary-initial-state $\leq N$
and *empty-trace-fri-bound:*
wp-event (checked-staged-security-experiment-with-data-state A)
(staged-security-with-data-state-verifier-event trace-fri-bad-with-empty-composition-header-candidates)
adversary-initial-state $\leq$ trace-fri-error
and *current-empty-bound:*
wp-event (checked-staged-security-experiment-with-data-state A)
(staged-security-with-data-state-current-empty-query-opening-hit-at 0)
adversary-initial-state $\leq$ empty-query-error
shows *checked-staged-adversary-acceptance-probability $A \leq$*
hash-collision-budget-value 0
(staged-attacker-query-budget budgets +
staged-challenge-query-budget + verifier-hash-query-budget) +
(composition-error-bound +
staged-phase-relation-error size
(staged-alpha-search-queries budgets 0) +
hash-collision-budget-value 0
(staged-alpha-search-queries budgets 0) +
N +
staged-phase-relation-error size
(staged-alpha-search-queries budgets 0)) +
trace-fri-error' + composition-fri-error' +
($\sum i < \text{rounds}$.
staged-phase-relation-error size
(staged-query-search-queries budgets $i + 1$) +
query-error-bound +
hash-collision-budget-value 0
(staged-query-search-queries budgets $i + 1$) +
 $T i + C i$) +
(composition-error-bound +
staged-phase-relation-error size
(staged-alpha-search-queries budgets 0) +
hash-collision-budget-value 0
(staged-alpha-search-queries budgets 0) +
N +
staged-phase-relation-error size
(staged-alpha-search-queries budgets 0) +
trace-fri-error + empty-query-error)
proof *(rule*
checked-staged-soundness-from-aligned-transcript-components-and-current-query-prefix-rounds-from-not-prefix-curr
[OF false-statement wf controlled trace-fri-bound comp-fri-bound - not-prefix-bound
empty-trace-fri-bound current-empty-bound])
fix *i*
assume *i-bound: $i < \text{rounds}$*

show

wp-event

(*checked-staged-security-experiment-with-query-prefix-data-state A i*)
 (*checked-staged-security-with-query-prefix-current-query-partial-opening-hit-at i*)
adversary-initial-state $\leq$
staged-phase-relation-error size
 (*staged-query-search-queries budgets i + 1*) +
query-error-bound +
hash-collision-budget-value 0
 (*staged-query-search-queries budgets i + 1*) +
T i + C i

by (*rule*

checked-staged-security-with-query-prefix-current-query-partial-opening-hit-at-bound-from-conceptual-default-alpha-prefix-union-clean-structured-path
 [*OF false-statement wf controlled i-bound - trace-path-bound[OF i-bound]*
composition-path-bound[OF i-bound]])
 (*rule default-prefix-clean[OF i-bound]*)

qed

theorem *checked-staged-soundness-from-aligned-transcript-conceptual-default-alpha-prefix-union-clean-structured-path*

fixes *trace-fri-error' composition-fri-error' D empty-query-error :: prob*

and *T C :: nat $\Rightarrow$ prob*

and *trace-default composition-default :: 'f list*

assumes *false-statement: $\neg$ exists-valid-trace*

and *wf: staged-budget-wellformed budgets*

and *controlled: staged-adversary-controlled budgets A*

and *trace-fri-bound:*

wp-event (*checked-staged-security-experiment-with-data-state A*)
 (*staged-security-with-data-state-verifier-event*
trace-fri-bad-with-partial-candidates)
adversary-initial-state $\leq$ *trace-fri-error'*

and *comp-fri-bound:*

wp-event (*checked-staged-security-experiment-with-data-state A*)
 (*staged-security-with-data-state-verifier-event*
composition-fri-bad-with-partial-candidates)
adversary-initial-state $\leq$ *composition-fri-error'*

and *default-prefix-clean:*

$\bigwedge i$ *prefix prefix-state.*

i < rounds $\implies$

Some ((prefix, prefix-state), prefix-state) $\in$

set-dist

(*execute* (*checked-staged-query-prefix-with-state A i*)

adversary-initial-state) $\implies$

query-prefix-trace-conceptual-table-with-default prefix prefix-state

trace-default = *trace-default* $\wedge$

query-prefix-composition-conceptual-table-with-default prefix

prefix-state composition-default = *composition-default* $\wedge$

trace-table-low-degree trace-default $\wedge$

composition-table-low-degree maxDegree composition-default $\wedge$

trace-default $\in$

$\alpha\text{-prefix-trace-table-candidates } \text{prefix-state}$
 $(\text{sqp-trace-root } \text{prefix}) \wedge$
 $\text{sqp-alphas } \text{prefix} \notin$
 $\alpha\text{-prefix-union-bad-sets } \text{prefix-state}$
 $\text{composition-trace-bad-alpha-space } (\text{sqp-trace-root } \text{prefix})$
and trace-path-bound:
 $\bigwedge i. i < \text{rounds} \implies$
 wp-event
 $(\text{checked-staged-security-experiment-with-query-prefix-data-state } A \ i)$
 $(\text{checked-staged-security-with-query-prefix-current-trace-path-output-hit-at } i)$
 $\text{adversary-initial-state} \leq T \ i$
and $\text{composition-path-bound:}$
 $\bigwedge i. i < \text{rounds} \implies$
 wp-event
 $(\text{checked-staged-security-experiment-with-query-prefix-data-state } A \ i)$
 $(\text{checked-staged-security-with-query-prefix-current-composition-path-output-hit-at } i)$
 $\text{adversary-initial-state} \leq C \ i$
and drift-bound:
 wp-event
 $(\text{checked-staged-security-experiment-with-actual-alpha-prefix-data-state } A)$
 $(\text{checked-staged-security-with-actual-alpha-prefix-partial-header-candidate-drift } \text{composition-trace-bad-alpha-space})$
 $\text{adversary-initial-state} \leq D$
and $\text{empty-trace-fri-bound:}$
 $\text{wp-event } (\text{checked-staged-security-experiment-with-data-state } A)$
 $(\text{staged-security-with-data-state-verifier-event } \text{trace-fri-bad-with-empty-composition-header-candidates})$
 $\text{adversary-initial-state} \leq \text{trace-fri-error}$
and $\text{current-empty-bound:}$
 $\text{wp-event } (\text{checked-staged-security-experiment-with-data-state } A)$
 $(\text{staged-security-with-data-state-current-empty-query-opening-hit-at } 0)$
 $\text{adversary-initial-state} \leq \text{empty-query-error}$
shows $\text{checked-staged-adversary-acceptance-probability } A \leq$
 $\text{hash-collision-budget-value } 0$
 $(\text{staged-attacker-query-budget } \text{budgets} +$
 $\text{staged-challenge-query-budget} + \text{verifier-hash-query-budget}) +$
 $(\text{composition-error-bound} +$
 $\text{staged-phase-relation-error size}$
 $(\text{staged-alpha-search-queries } \text{budgets } 0) +$
 $\text{hash-collision-budget-value } 0$
 $(\text{staged-alpha-search-queries } \text{budgets } 0) +$
 $D +$
 $\text{staged-phase-relation-error size}$
 $(\text{staged-alpha-search-queries } \text{budgets } 0)) +$
 $\text{trace-fri-error}' + \text{composition-fri-error}' +$
 $(\sum i < \text{rounds.}$
 $\text{staged-phase-relation-error size}$
 $(\text{staged-query-search-queries } \text{budgets } i + 1) +$

```

    query-error-bound +
    hash-collision-budget-value 0
    (staged-query-search-queries budgets i + 1) +
    T i + C i) +
  (composition-error-bound +
    staged-phase-relation-error size
    (staged-alpha-search-queries budgets 0) +
    hash-collision-budget-value 0
    (staged-alpha-search-queries budgets 0) +
    D +
    staged-phase-relation-error size
    (staged-alpha-search-queries budgets 0) +
    trace-fri-error + empty-query-error)
proof (rule
  checked-staged-soundness-from-aligned-transcript-components-and-current-query-prefix-rounds-from-drift-current-e
  [OF false-statement wf controlled trace-fri-bound comp-fri-bound - drift-bound
    empty-trace-fri-bound current-empty-bound])
fix i
assume i-bound: i < rounds
show
  wp-event
  (checked-staged-security-experiment-with-query-prefix-data-state A i)
  (checked-staged-security-with-query-prefix-current-query-partial-opening-hit-at i)
  adversary-initial-state ≤
  staged-phase-relation-error size
  (staged-query-search-queries budgets i + 1) +
  query-error-bound +
  hash-collision-budget-value 0
  (staged-query-search-queries budgets i + 1) +
  T i + C i
by (rule
  checked-staged-security-with-query-prefix-current-query-partial-opening-hit-at-bound-from-conceptual-default-alpha
  [OF false-statement wf controlled i-bound - trace-path-bound[OF i-bound]
    composition-path-bound[OF i-bound]])
  (rule default-prefix-clean[OF i-bound])
qed

end

end

theory Soundness-Alpha-Fixed-Candidate
imports Soundness-Conceptual-Query-Current-Alpha-Clean
begin

```

Fixed-trace-table alpha targets.

This layer records the sound probability component that is available when a trace table has already been fixed before the alpha challenge is sampled. It deliberately does not

claim that sampled Merkle openings determine such a table.

context *soundness*
begin

definition *staged-alpha-prefix-fixed-trace-prequery-hit*

$:: 'f \text{ list} \Rightarrow$
 $((('f \times 'f \text{ list} \times 'f \text{ list} \times 'f) \times$
 $'f \text{ protocol-channel}) \text{ option} \Rightarrow \text{bool})$

where

$\text{staged-alpha-prefix-fixed-trace-prequery-hit } \text{trace-table } \text{out} \longleftrightarrow$
 $(\text{case out of}$
 $\quad \text{None} \Rightarrow \text{False}$
 $\mid \text{Some } (-, \text{prefix-state}) \Rightarrow$
 $\quad (\exists \text{as} \in \text{composition-trace-bad-alpha-space trace-table.}$
 $\quad \text{alpha-vector-prequeried-in-state prefix-state as}$
 $\quad (\text{length spec})))$

definition *staged-alpha-prefix-fixed-trace-prequery-relation*

$:: 'f \text{ staged-adversary} \Rightarrow 'f \text{ protocol-channel} \Rightarrow$
 $'f \text{ list} \Rightarrow 'f \text{ protocol-hash-input} \Rightarrow 'f \Rightarrow \text{bool}$

where

$\text{staged-alpha-prefix-fixed-trace-prequery-relation } A \text{ s trace-table } x \text{ y}$
 $\longleftrightarrow$
 $(\exists \text{prefix prefix-state as } i.$
 $\quad \text{Some } (\text{prefix}, \text{prefix-state}) \in$
 $\quad \text{set-dist } (\text{execute } (\text{staged-alpha-prefix-program } A) \text{ s}) \wedge$
 $\quad \text{as} \in \text{composition-trace-bad-alpha-space trace-table} \wedge$
 $\quad i < \text{length spec} \wedge$
 $\quad i < \text{length as} \wedge$
 $\quad x = \text{alpha-challenge-key-at prefix-state } i \text{ as} \wedge$
 $\quad y = \text{as} ! i)$

definition *checked-staged-transcript-fixed-trace-alpha-bad-excluding-prequeried*

$:: 'f \text{ list} \Rightarrow$
 $(((((('f \times 'f \text{ list} \times 'f \text{ list} \times 'f) \times$
 $'f \text{ protocol-channel}) \times 'f \text{ staged-proof-data}) \times$
 $'f \text{ protocol-channel}) \text{ option} \Rightarrow$
 $\text{bool})$

where

$\text{checked-staged-transcript-fixed-trace-alpha-bad-excluding-prequeried}$
 $\text{trace-table } \text{out} \longleftrightarrow$
 $(\text{case out of}$
 $\quad \text{None} \Rightarrow \text{False}$
 $\mid \text{Some } (((\text{prefix}, \text{prefix-state}), \text{data}), -) \Rightarrow$
 $\quad \text{staged-alphas data} \in \text{composition-trace-bad-alpha-space trace-table} \wedge$
 $\quad \neg \text{alpha-vector-prequeried-in-state prefix-state}$
 $\quad (\text{staged-alphas data}) (\text{length spec}))$

definition *checked-staged-transcript-fixed-trace-alpha-prequery-hit*

```

:: 'f list ⇒
  (((('f × 'f list × 'f list × 'f) ×
    'f protocol-channel) × 'f staged-proof-data) ×
    'f protocol-channel) option ⇒
  bool
where
  checked-staged-transcript-fixed-trace-alpha-prequery-hit trace-table out
  ⇔
  (case out of
    None ⇒ False
  | Some (((prefix, prefix-state), data), -) ⇒
    staged-alphas data ∈ composition-trace-bad-alpha-space trace-table ∧
    alpha-vector-prequeried-in-state prefix-state
    (staged-alphas data) (length spec))

```

definition *checked-staged-transcript-fixed-trace-alpha-bad*

```

:: 'f list ⇒
  (((('f × 'f list × 'f list × 'f) ×
    'f protocol-channel) × 'f staged-proof-data) ×
    'f protocol-channel) option ⇒
  bool
where
  checked-staged-transcript-fixed-trace-alpha-bad trace-table out
  ⇔
  (case out of
    None ⇒ False
  | Some (((prefix, prefix-state), data), -) ⇒
    staged-alphas data ∈ composition-trace-bad-alpha-space trace-table)

```

lemma *staged-alpha-prefix-fixed-trace-prequery-hit-imp-relation-hit:*

```

assumes empty: HashMap s = fmempty
and support:
  Some (prefix, prefix-state) ∈
    set-dist (execute (staged-alpha-prefix-program A) s)
and hit:
  staged-alpha-prefix-fixed-trace-prequery-hit trace-table
    (Some (prefix, prefix-state))
shows
  hash-relation-hit-event
    (staged-alpha-prefix-fixed-trace-prequery-relation A s trace-table)
    s (Some (prefix, prefix-state))

```

proof –

```

from hit obtain as i where member:
  as ∈ composition-trace-bad-alpha-space trace-table
and i-bound: i < length spec
and i-len: i < length as
and lookup:
  fmlookup (HashMap prefix-state)
    (alpha-challenge-key-at prefix-state i as) = Some (as ! i)

```

```

unfolding staged-alpha-prefix-fixed-trace-prequery-hit-def
  alpha-vector-prequeried-in-state-def
by auto
have initial-none:
  fmlookup (HashMap s) (alpha-challenge-key-at prefix-state i as) =
    None
using empty by simp
have relation:
  staged-alpha-prefix-fixed-trace-prequery-relation A s trace-table
    (alpha-challenge-key-at prefix-state i as) (as ! i)
unfolding staged-alpha-prefix-fixed-trace-prequery-relation-def
using support member i-bound i-len by blast
have hit-relation:
  hash-relation-hit
    (staged-alpha-prefix-fixed-trace-prequery-relation A s trace-table)
    s prefix-state
unfolding hash-relation-hit-def
using initial-none lookup relation by blast
show ?thesis
unfolding hash-relation-hit-event-def using hit-relation by simp
qed

lemma staged-alpha-prefix-fixed-trace-prequery-hit-bound-by-relation-hit:
assumes empty: HashMap s = fmempty
shows
  wp-event (staged-alpha-prefix-program A)
    (staged-alpha-prefix-fixed-trace-prequery-hit trace-table) s ≤
  wp-event (staged-alpha-prefix-program A)
    (hash-relation-hit-event
      (staged-alpha-prefix-fixed-trace-prequery-relation A s trace-table)
      s) s
proof (rule wp-event-mono-on-support)
fix out
assume support: out ∈ set-dist (execute (staged-alpha-prefix-program A) s)
and hit: staged-alpha-prefix-fixed-trace-prequery-hit trace-table out
show
  hash-relation-hit-event
    (staged-alpha-prefix-fixed-trace-prequery-relation A s trace-table)
    s out
proof (cases out)
case None
then show ?thesis
using hit unfolding staged-alpha-prefix-fixed-trace-prequery-hit-def
by simp
next
case (Some result)
then obtain prefix prefix-state where out-eq:
  out = Some (prefix, prefix-state)
by (cases result) auto

```

```

show ?thesis
  unfolding out-eq
  by (rule staged-alpha-prefix-fixed-trace-prequery-hit-imp-relation-hit
    [OF empty])
    (use support hit out-eq in simp-all)
qed
qed

lemma staged-alpha-prefix-fixed-trace-prequery-relation-fiber-bound-size:
  card
    {y. staged-alpha-prefix-fixed-trace-prequery-relation A s trace-table x y}
    ≤ size
proof –
  have subset:
    {y. staged-alpha-prefix-fixed-trace-prequery-relation A s trace-table x y}
    ⊆ (UNIV :: 'f set)
  by simp
  have finite-field: finite (UNIV :: 'f set)
  by simp
  have card
    {y. staged-alpha-prefix-fixed-trace-prequery-relation A s trace-table x y}
    ≤ card (UNIV :: 'f set)
  by (rule card-mono[OF finite-field subset])
  also have ... = size
  using size-card by simp
  finally show ?thesis .
qed

lemma staged-alpha-prefix-fixed-trace-prequery-hit-bound:
  assumes wf: staged-budget-wellformed budgets
  and controlled: staged-adversary-controlled budgets A
  shows
    wp-event (staged-alpha-prefix-program A)
      (staged-alpha-prefix-fixed-trace-prequery-hit trace-table)
      adversary-initial-state ≤
      staged-phase-relation-error size
      (staged-alpha-search-queries budgets 0)
proof –
  have prehit-le-relation:
    wp-event (staged-alpha-prefix-program A)
      (staged-alpha-prefix-fixed-trace-prequery-hit trace-table)
      adversary-initial-state ≤
    wp-event (staged-alpha-prefix-program A)
      (hash-relation-hit-event
        (staged-alpha-prefix-fixed-trace-prequery-relation A
          adversary-initial-state trace-table)
        adversary-initial-state)
      adversary-initial-state
  by (rule staged-alpha-prefix-fixed-trace-prequery-hit-bound-by-relation-hit)

```



```

    (simp add: adversary-initial-state-def)
  also have ... ≤
    staged-phase-relation-error size
    (staged-alpha-search-queries budgets 0)
  proof -
    have program:
      hash-relation-program
      (staged-alpha-prefix-fixed-trace-prequery-relation A
        adversary-initial-state trace-table)
      size (staged-alpha-search-queries budgets 0)
      (staged-alpha-prefix-program A)
    by (rule hash-relation-program-staged-alpha-prefix-program
      [OF wf controlled])
      (rule staged-alpha-prefix-fixed-trace-prequery-relation-fiber-bound-size)
    show ?thesis
      using program
      unfolding hash-relation-program-def hash-relation-budget-def
      staged-phase-relation-error-def
      by blast
  qed
  finally show ?thesis .
qed

lemma checked-staged-after-alpha-prefix-fixed-trace-alpha-bad-excluding-prequeried-bound:
  shows
    wp-event (checked-staged-after-alpha-prefix-program A prefix)
      (λout. case out of
        None ⇒ False
      | Some (data, -) ⇒
        staged-alphas data ∈
          composition-trace-bad-alpha-space trace-table ∧
          ¬ alpha-vector-prequeried-in-state s
            (staged-alphas data) (length spec))
        s ≤ composition-error-bound
  proof -
    let ?B = composition-trace-bad-alpha-space trace-table
    have bound:
      wp-event (checked-staged-after-alpha-prefix-program A prefix)
        (λout. case out of
          None ⇒ False
        | Some (data, -) ⇒
          staged-alphas data ∈ ?B ∧
          ¬ alpha-vector-prequeried-in-state s
            (staged-alphas data) (length spec))
          s ≤ nnreal (card ?B) / nnreal (CARD('f) ^ length spec)
    by (rule
      checked-staged-after-alpha-prefix-program-alpha-list-bound-excluding-prequeried
      [OF composition-trace-bad-alpha-space-subset-alpha-space])
    also have ... ≤ composition-error-bound

```

```

using composition-trace-bad-alpha-space-fraction-bound-alpha-space
  [of trace-table]
by simp
finally show ?thesis .
qed

lemma checked-staged-transcript-fixed-trace-alpha-bad-excluding-prequeried-bound:
shows
  wp-event (checked-staged-transcript-with-alpha-prefix-program A)
    (checked-staged-transcript-fixed-trace-alpha-bad-excluding-prequeried
      trace-table)
    adversary-initial-state  $\leq$  composition-error-bound
unfolding checked-staged-transcript-with-alpha-prefix-program-def
proof (rule wp-event-bind-bound-by-cont)
show
   $\neg$  checked-staged-transcript-fixed-trace-alpha-bad-excluding-prequeried
    trace-table None
unfolding
  checked-staged-transcript-fixed-trace-alpha-bad-excluding-prequeried-def
by simp
next
fix prefix prefix-state
assume prefix-out:
  Some (prefix, prefix-state)  $\in$ 
    set-dist
    (execute (staged-alpha-prefix-program A) adversary-initial-state)
let ?Q =
  checked-staged-transcript-fixed-trace-alpha-bad-excluding-prequeried
    trace-table
have get-eq:
  wp-event
    (get  $\gg$ 
      ( $\lambda$ prefix-state'.
        checked-staged-after-alpha-prefix-program A prefix  $\gg$ 
          ( $\lambda$ data. return ((prefix, prefix-state'), data))))
  ?Q prefix-state =
  wp-event
    (checked-staged-after-alpha-prefix-program A prefix  $\gg$ 
      ( $\lambda$ data. return ((prefix, prefix-state), data)))
  ?Q
    prefix-state
unfolding wp-event-def by (simp add: wpsimps)
have return-eq:
  wp-event
    (checked-staged-after-alpha-prefix-program A prefix  $\gg$ 
      ( $\lambda$ data. return ((prefix, prefix-state), data)))
  ?Q prefix-state =
  wp-event (checked-staged-after-alpha-prefix-program A prefix)
    ( $\lambda$ out. case out of

```

```

    None  $\Rightarrow$  False
  | Some (data, -)  $\Rightarrow$ 
    staged-alphas data  $\in$ 
      composition-trace-bad-alpha-space trace-table  $\wedge$ 
       $\neg$  alpha-vector-prequeried-in-state prefix-state
      (staged-alphas data) (length spec))
    prefix-state
  apply (subst wp-event-bind-return-map)
  unfolding
    checked-staged-transcript-fixed-trace-alpha-bad-excluding-prequeried-def
  by simp
show
  wp-event
  (get  $\gg=$ 
    ( $\lambda$ prefix-state.
      checked-staged-after-alpha-prefix-program A prefix  $\gg=$ 
        ( $\lambda$ data. return ((prefix, prefix-state), data))))
  ?Q prefix-state  $\leq$  composition-error-bound
  unfolding get-eq return-eq
  by (rule
    checked-staged-after-alpha-prefix-fixed-trace-alpha-bad-excluding-prequeried-bound)
qed

lemma checked-staged-transcript-fixed-trace-alpha-prequery-bound:
  assumes wf: staged-budget-wellformed budgets
  and controlled: staged-adversary-controlled budgets A
  shows
    wp-event (checked-staged-transcript-with-alpha-prefix-program A)
      (checked-staged-transcript-fixed-trace-alpha-prequery-hit trace-table)
      adversary-initial-state  $\leq$ 
      staged-phase-relation-error size
      (staged-alpha-search-queries budgets 0)
  unfolding checked-staged-transcript-with-alpha-prefix-program-def
proof (rule wp-event-bind-bound-by-head-event)
  show
    wp-event (staged-alpha-prefix-program A)
      (staged-alpha-prefix-fixed-trace-prequery-hit trace-table)
      adversary-initial-state
     $\leq$  staged-phase-relation-error size
      (staged-alpha-search-queries budgets 0)
  by (rule staged-alpha-prefix-fixed-trace-prequery-hit-bound
    [OF wf controlled])
next
show
  checked-staged-transcript-fixed-trace-alpha-prequery-hit trace-table
  None  $\Rightarrow$ 
  staged-alpha-prefix-fixed-trace-prequery-hit trace-table None
  unfolding checked-staged-transcript-fixed-trace-alpha-prequery-hit-def
    staged-alpha-prefix-fixed-trace-prequery-hit-def

```

```

  by simp
next
fix prefix prefix-state out
assume prefix-out:
  Some (prefix, prefix-state) ∈
    set-dist
      (execute (staged-alpha-prefix-program A) adversary-initial-state)
and cont:
  out ∈
    set-dist
      (execute
        (get  $\gg$ 
          (λprefix-state.
            checked-staged-after-alpha-prefix-program A prefix  $\gg$ 
              (λdata. return ((prefix, prefix-state), data))))
        prefix-state)
and hit:
  checked-staged-transcript-fixed-trace-alpha-prequery-hit trace-table
  out
from hit cont show
  staged-alpha-prefix-fixed-trace-prequery-hit trace-table
  (Some (prefix, prefix-state))
unfolding checked-staged-transcript-fixed-trace-alpha-prequery-hit-def
  staged-alpha-prefix-fixed-trace-prequery-hit-def
by (auto elim!: set-dist-bindE split: option.splits prod.splits)
qed

```

lemma *checked-staged-transcript-fixed-trace-alpha-bad-split:*
assumes *hit:*
checked-staged-transcript-fixed-trace-alpha-bad trace-table out
shows
checked-staged-transcript-fixed-trace-alpha-bad-excluding-prequeried
trace-table out ∨
checked-staged-transcript-fixed-trace-alpha-prequery-hit trace-table out
using *hit*
unfolding *checked-staged-transcript-fixed-trace-alpha-bad-def*
checked-staged-transcript-fixed-trace-alpha-bad-excluding-prequeried-def
checked-staged-transcript-fixed-trace-alpha-prequery-hit-def
by (auto split: option.splits prod.splits)

lemma *checked-staged-transcript-fixed-trace-alpha-bad-bound:*
assumes *wf: staged-budget-wellformed budgets*
and *controlled: staged-adversary-controlled budgets A*
shows
wp-event (checked-staged-transcript-with-alpha-prefix-program A)
(checked-staged-transcript-fixed-trace-alpha-bad trace-table)
adversary-initial-state ≤
composition-error-bound +
staged-phase-relation-error size

```

      (staged-alpha-search-queries budgets 0)
proof -
  let ?M = checked-staged-transcript-with-alpha-prefix-program A
  let ?Fresh =
    checked-staged-transcript-fixed-trace-alpha-bad-excluding-prequeried
    trace-table
  let ?Pre =
    checked-staged-transcript-fixed-trace-alpha-prequery-hit trace-table
  have event-le:
    wp-event ?M
      (checked-staged-transcript-fixed-trace-alpha-bad trace-table)
      adversary-initial-state ≤
      wp-event ?M (λout. ?Fresh out ∨ ?Pre out)
      adversary-initial-state
  by (rule wp-event-mono) (rule checked-staged-transcript-fixed-trace-alpha-bad-split)
  also have ... ≤
    wp-event ?M ?Fresh adversary-initial-state +
    wp-event ?M ?Pre adversary-initial-state
  by (rule wp-event-union-bound)
  also have ... ≤
    composition-error-bound +
    staged-phase-relation-error size
    (staged-alpha-search-queries budgets 0)
  by (intro add-mono
    checked-staged-transcript-fixed-trace-alpha-bad-excluding-prequeried-bound
    checked-staged-transcript-fixed-trace-alpha-prequery-bound
    wf controlled)
  finally show ?thesis .
qed

end

end

```

```

theory Staged-Security-Experiment-Composition-Query-Canonical
  imports Staged-Security-Experiment-Composition-Query-Classification
begin

```

Canonical-index projections for the header-authenticated composition query bridge. These lemmas do not bound the broad existential event by themselves; they expose the canonical query-index cover that a later Merkle/candidate binding argument has to control.

In particular, the projection from the actual-alpha-prefix event to a query-prefix hit is support-level evidence. It is not by itself a probability reduction to the separately sampled query-prefix program: the authenticated opening witness is still selected existentially from verifier-reachable evidence. A sound probability bound needs either a cover fixed before the query challenge is sampled, or a Merkle/collision/partial-opening side event accounting for failure to select such a witness.

context *soundness*
begin

lemma *checked-staged-security-with-actual-alpha-prefix-header-authenticated-candidate-opening-query-hit-from-prefix*

assumes *hit*:

checked-staged-security-with-actual-alpha-prefix-header-authenticated-candidate-opening-query-hit-from-prefix
A
(Some (((((alpha-prefix, alpha-prefix-state), data), attacker-state),
result), final-state))

obtains *i trace-openings composition-openings query-prefix query-prefix-state*

raw raw-state where

(trace-openings, composition-openings) ∈
query-header-supported-partial-opening-witnesses
(verifier-state-from-adversary attacker-state
(staged-proof-transcript data))
(staged-trace-root data)
(staged-trace-fri-roots data)
(staged-trace-final data)
(staged-alphas data)
(staged-degree data)
(staged-composition-fri-roots data)
(staged-composition-final data)

i < rounds

Some (data, attacker-state) ∈

set-dist

(execute (checked-staged-transcript-program A)
adversary-initial-state)

Some (((query-prefix, query-prefix-state), raw), raw-state) ∈

set-dist

(execute (checked-staged-query-prefix-receive-with-state A i)
adversary-initial-state)

index (to-nat raw) ∈

query-header-supported-partial-opening-canonical-query-indices
(verifier-state-from-adversary attacker-state
(staged-proof-transcript data))
(staged-trace-root data)
(staged-trace-fri-roots data)
(staged-trace-final data)
(staged-alphas data)
(staged-degree data)
(staged-composition-fri-roots data)
(staged-composition-final data)

proof (rule

checked-staged-security-with-actual-alpha-prefix-header-authenticated-candidate-opening-query-hit-from-prefix-query
[OF hit])

fix *i trace-openings composition-openings query-prefix query-prefix-state*

raw raw-state

assume *witness*:

(trace-openings, composition-openings) ∈

```

query-header-supported-partial-opening-witnesses
  (verifier-state-from-adversary attacker-state
   (staged-proof-transcript data))
  (staged-trace-root data)
  (staged-trace-fri-roots data)
  (staged-trace-final data)
  (staged-alphas data)
  (staged-degree data)
  (staged-composition-fri-roots data)
  (staged-composition-final data)
and i-bound:  $i < \text{rounds}$ 
and builder:
  Some (data, attacker-state) ∈
    set-dist
    (execute (checked-staged-transcript-program A
      adversary-initial-state)
and prefix-receive:
  Some (((query-prefix, query-prefix-state), raw), raw-state) ∈
    set-dist
    (execute (checked-staged-query-prefix-receive-with-state A i
      adversary-initial-state)
and target-hit:
  checked-staged-query-prefix-dynamic-index-hit
  (staged-query-prefix-candidate-opening-query-target-from-prefix
   ((replicate rounds [] [i := trace-openings])
    (replicate rounds [] [i := composition-openings])
    i)
   (Some (((query-prefix, query-prefix-state), raw), raw-state))
have header-hit:
  checked-staged-security-with-query-prefix-header-authenticated-candidate-opening-hit
  trace-openings composition-openings i
  (Some ((((((query-prefix, query-prefix-state), raw), raw-state),
    ((data, attacker-state), result)), final-state))
by (rule
  checked-staged-security-with-query-prefix-header-authenticated-candidate-opening-hitI-from-dynamic-hit
  [OF witness target-hit])
have canonical:
  index (to-nat raw) ∈
  query-header-supported-partial-opening-canonical-query-indices
  (verifier-state-from-adversary attacker-state
   (staged-proof-transcript data))
  (staged-trace-root data)
  (staged-trace-fri-roots data)
  (staged-trace-final data)
  (staged-alphas data)
  (staged-degree data)
  (staged-composition-fri-roots data)
  (staged-composition-final data)
by (rule

```

checked-staged-security-with-query-prefix-header-authenticated-candidate-opening-hit-canonical-query-indexD
 [OF *i*-bound header-hit])
show ?thesis
 by (rule that[OF witness *i*-bound builder prefix-receive canonical])
qed

lemma *checked-staged-security-with-actual-alpha-prefix-header-authenticated-candidate-opening-query-hit-from-prefix*
assumes *hit*:
checked-staged-security-with-actual-alpha-prefix-header-authenticated-candidate-opening-query-hit-from-prefix
A
 (Some (((((alpha-prefix, alpha-prefix-state), data), attacker-state),
 result), final-state))
and *cover*:
query-header-supported-partial-opening-canonical-query-indices
 (verifier-state-from-adversary attacker-state
 (staged-proof-transcript data))
 (staged-trace-root data)
 (staged-trace-fri-roots data)
 (staged-trace-final data)
 (staged-alphas data)
 (staged-degree data)
 (staged-composition-fri-roots data)
 (staged-composition-final data) $\subseteq B$
obtains *i* query-prefix query-prefix-state raw raw-state **where**
i < rounds
 Some (data, attacker-state) $\in$
 set-dist
 (execute (checked-staged-transcript-program A)
 adversary-initial-state)
 Some (((query-prefix, query-prefix-state), raw), raw-state) $\in$
 set-dist
 (execute (checked-staged-query-prefix-receive-with-state A *i*)
 adversary-initial-state)
checked-staged-query-prefix-dynamic-index-hit
 (λ - . *B*)
 (Some (((query-prefix, query-prefix-state), raw), raw-state))
proof (rule
checked-staged-security-with-actual-alpha-prefix-header-authenticated-candidate-opening-query-hit-from-prefix-canon
 [OF *hit*])
fix *i* trace-openings composition-openings query-prefix query-prefix-state raw
 raw-state
assume *i*-bound: *i* < rounds
and builder:
 Some (data, attacker-state) $\in$
 set-dist
 (execute (checked-staged-transcript-program A)
 adversary-initial-state)
and prefix-receive:
 Some (((query-prefix, query-prefix-state), raw), raw-state) $\in$


```

    set-dist
      (execute (checked-staged-query-prefix-receive-with-state A i)
        adversary-initial-state)
  and canonical:
    index (to-nat raw) ∈
      query-header-supported-partial-opening-canonical-query-indices
        (verifier-state-from-adversary attacker-state
          (staged-proof-transcript data))
        (staged-trace-root data)
        (staged-trace-fri-roots data)
        (staged-trace-final data)
        (staged-alphas data)
        (staged-degree data)
        (staged-composition-fri-roots data)
        (staged-composition-final data)
  have hit-B:
    checked-staged-query-prefix-dynamic-index-hit
      (λ- -. B)
      (Some (((query-prefix, query-prefix-state), raw), raw-state))
  using canonical cover
  unfolding checked-staged-query-prefix-dynamic-index-hit-def
  by auto
  show ?thesis
  by (rule that[OF i-bound builder prefix-receive hit-B])
qed

lemma checked-staged-query-prefix-constant-index-hit-bound-from-budgets:
  assumes wf: staged-budget-wellformed budgets
  and controlled: staged-adversary-controlled budgets A
  and i-bound: i < rounds
  and subset: B ⊆ query-sample-space
  and frac:
    nnreal (card B) / nnreal (card query-sample-space) ≤
      query-error-bound
  shows
    wp-event (checked-staged-query-prefix-receive-with-state A i)
      (checked-staged-query-prefix-dynamic-index-hit (λ- -. B))
      adversary-initial-state ≤
        staged-phase-relation-error size
          (staged-query-search-queries budgets i + 1) + query-error-bound
  proof -
  have raw-bound: query-index-raw-preimage-bound
  by (rule query-index-raw-preimage-bound-from-sampler-wellformed)
  have hit-bound:
    wp-event (checked-staged-query-prefix-receive-with-state A i)
      (checked-staged-query-prefix-dynamic-index-hit (λ- -. B))
      adversary-initial-state ≤
        wp-event (checked-staged-query-prefix-receive-with-state A i)
          (checked-staged-query-prefix-dynamic-index-prehit (λ- -. B))

```

adversary-initial-state + query-error-bound

proof (rule
checked-staged-query-prefix-dynamic-index-hit-bound-by-prehit-and-query-error
[OF raw-bound])
fix *prefix prefix-state*
assume
Some ((prefix, prefix-state), prefix-state) ∈
set-dist
(execute (checked-staged-query-prefix-with-state A i)
adversary-initial-state)
show
(λ- -. B) prefix prefix-state ⊆ query-sample-space ∧
nnreal (card ((λ- -. B) prefix prefix-state)) /
nnreal (card query-sample-space) ≤ query-error-bound
using *subset frac by simp*
qed
have *prehit-bound*:
wp-event (checked-staged-query-prefix-receive-with-state A i)
(checked-staged-query-prefix-dynamic-index-prehit (λ- -. B))
adversary-initial-state ≤
staged-phase-relation-error size
(staged-query-search-queries budgets i + 1)
proof (rule
checked-staged-query-prefix-dynamic-index-prehit-bound-from-relation-program)
show *HashMap adversary-initial-state = fmempty*
by (*simp add: adversary-initial-state-def*)
show *hash-relation-program*
(checked-staged-query-prefix-dynamic-index-prehit-relation A i
adversary-initial-state (λ- -. B))
size (staged-query-search-queries budgets i + 1)
(checked-staged-query-prefix-receive-with-state A i)
by (rule *hash-relation-program-checked-staged-query-prefix-receive-with-state*
[OF wf controlled])
(use i-bound in simp-all,
rule checked-staged-query-prefix-dynamic-index-prehit-relation-fiber-bound-size)
qed
show *?thesis*
by (rule *order-trans[OF hit-bound]*)
(intro add-mono prehit-bound order-refl)
qed

lemma *checked-staged-security-with-query-prefix-constant-index-hit-bound-from-budgets*:
assumes *wf: staged-budget-wellformed budgets*
and *controlled: staged-adversary-controlled budgets A*
and *i-bound: i < rounds*
and *subset: B ⊆ query-sample-space*
and *frac*:
nnreal (card B) / nnreal (card query-sample-space) ≤
query-error-bound

shows

wp-event

(checked-staged-security-experiment-with-query-prefix-data-state A i)

(checked-staged-security-with-query-prefix-dynamic-index-hit
(λ - -. B))

adversary-initial-state $\leq$

staged-phase-relation-error size

(staged-query-search-queries budgets $i + 1$) + query-error-bound

by *(rule checked-staged-security-with-query-prefix-dynamic-index-hit-bound,*
rule checked-staged-query-prefix-constant-index-hit-bound-from-budgets
[OF wf controlled i-bound subset frac])

end

end

theory *Soundness*

imports

Soundness-Staged-Aligned-Current-Query-Path

Soundness-Staged-Aligned-Current-Empty

Soundness-Alpha-Fixed-Candidate

Staged-Security-Experiment-Composition-Query-Header-Rounds

Staged-Security-Experiment-Composition-Query-Canonical

Staged-Security-Experiment-Composition-Query-Current

begin

Aggregator for the soundness infrastructure used by the active public endpoint. The public soundness theorem is exported by `Soundness_FRI_Trace_Restricted_Query_Endpoint.thy`.

end

theory *Staged-Security-Experiment-Composition-Query-Single-Round-Witness-Bridge*

imports

Staged-Security-Experiment-Composition-Query-Single-Round

Staged-Security-Experiment-Composition-Query-Current-Alignment

Staged-Security-Experiment-Composition-Query-Current-Path

Staged-Security-Experiment-Alpha-Header-Binding

begin

Bridge from the broad actual-alpha diagnostic witness event to a fixed query-prefix candidate-pair witness event.

The bridge is deterministic: it extracts the concrete prefix branch and candidate pair already present in the diagnostic event. It does not bound the diagnostic event probabilistically; later proof layers must still avoid treating the broad existential diagnostic event as a final public-path event.

context *soundness*

begin

definition *checked-staged-security-with-actual-alpha-prefix-query-prefix-fixed-candidate-pair-witness-gap-same-run-at*
where

checked-staged-security-with-actual-alpha-prefix-query-prefix-fixed-candidate-pair-witness-gap-same-run-at
trace-openings composition-openings trace-table composition-table i A out
 $\longleftrightarrow$
 (case out of
 None $\Rightarrow$ False
 | Some (((alpha-prefix, alpha-prefix-state), data), attacker-state),
 result), final-state) $\Rightarrow$
 ($\exists$ prefix prefix-state raw raw-state.
 Some (((((prefix, prefix-state), raw), raw-state),
 ((data, attacker-state), result)), final-state) $\in$
 set-dist
 (execute
 (checked-staged-security-experiment-with-query-prefix-data-state
 A i)
 adversary-initial-state) $\wedge$
 checked-staged-security-with-query-prefix-single-round-candidate-pair-witness-gap-at
 trace-openings composition-openings trace-table composition-table i
 (Some (((((prefix, prefix-state), raw), raw-state),
 ((data, attacker-state), result)), final-state))))))

definition *checked-staged-security-with-actual-alpha-prefix-query-prefix-fixed-candidate-pair-witness-gap-same-run-exists-at*
where

checked-staged-security-with-actual-alpha-prefix-query-prefix-fixed-candidate-pair-witness-gap-same-run-exists-at
i A out $\longleftrightarrow$
 ($\exists$ trace-openings composition-openings trace-table composition-table.
 checked-staged-security-with-actual-alpha-prefix-query-prefix-fixed-candidate-pair-witness-gap-same-run-at
 trace-openings composition-openings trace-table composition-table i A
 out)

definition *checked-staged-security-with-query-prefix-single-round-candidate-pair-witness-gap-exists-at*
where

checked-staged-security-with-query-prefix-single-round-candidate-pair-witness-gap-exists-at
i out $\longleftrightarrow$
 ($\exists$ trace-openings composition-openings trace-table composition-table.
 checked-staged-security-with-query-prefix-single-round-candidate-pair-witness-gap-at
 trace-openings composition-openings trace-table composition-table i
 out)

definition *staged-security-with-data-state-query-prefix-fixed-candidate-pair-witness-gap-same-run-at*
where

staged-security-with-data-state-query-prefix-fixed-candidate-pair-witness-gap-same-run-at
trace-openings composition-openings trace-table composition-table i A out
 $\longleftrightarrow$
 (case out of
 None $\Rightarrow$ False
 | Some (((data, attacker-state), result), final-state) $\Rightarrow$

$(\exists \text{prefix prefix-state raw raw-state.}$
 $\text{Some } (((((\text{prefix, prefix-state}), \text{raw}), \text{raw-state}),$
 $((\text{data, attacker-state}), \text{result})), \text{final-state}) \in$
 set-dist
 $(\text{execute}$
 $(\text{checked-staged-security-experiment-with-query-prefix-data-state}$
 $A i)$
 $\text{adversary-initial-state}) \wedge$
 $\text{checked-staged-security-with-query-prefix-single-round-candidate-pair-witness-gap-at}$
 $\text{trace-openings composition-openings trace-table composition-table } i$
 $(\text{Some } (((((\text{prefix, prefix-state}), \text{raw}), \text{raw-state}),$
 $((\text{data, attacker-state}), \text{result})), \text{final-state}))))$

definition *staged-security-with-data-state-query-prefix-fixed-candidate-pair-witness-gap-same-run-exists-at*
where

$\text{staged-security-with-data-state-query-prefix-fixed-candidate-pair-witness-gap-same-run-exists-at}$
 $i A \text{ out} \longleftrightarrow$
 $(\exists \text{trace-openings composition-openings trace-table composition-table.}$
 $\text{staged-security-with-data-state-query-prefix-fixed-candidate-pair-witness-gap-same-run-at}$
 $\text{trace-openings composition-openings trace-table composition-table } i A$
 $\text{out})$

lemma *checked-staged-security-with-actual-alpha-prefix-query-prefix-fixed-candidate-pair-witness-gap-same-run-at-pr*

$\text{checked-staged-security-with-actual-alpha-prefix-query-prefix-fixed-candidate-pair-witness-gap-same-run-at}$
 $\text{trace-openings composition-openings trace-table composition-table } i A \text{ out}$
 $\longleftrightarrow$

$(\text{case out of}$
 $\text{None} \Rightarrow \text{False}$
 $| \text{Some } (((((\text{alpha-prefix, alpha-prefix-state}), \text{data}), \text{attacker-state}),$
 $\text{result}), \text{final-state}) \Rightarrow$
 $\text{staged-security-with-data-state-query-prefix-fixed-candidate-pair-witness-gap-same-run-at}$
 $\text{trace-openings composition-openings trace-table composition-table } i A$
 $(\text{Some } (((\text{data, attacker-state}), \text{result}), \text{final-state}))))$

unfolding

$\text{checked-staged-security-with-actual-alpha-prefix-query-prefix-fixed-candidate-pair-witness-gap-same-run-at-def}$
 $\text{staged-security-with-data-state-query-prefix-fixed-candidate-pair-witness-gap-same-run-at-def}$

by (auto split: option.splits prod.splits)

lemma *checked-staged-security-with-actual-alpha-prefix-query-prefix-fixed-candidate-pair-witness-gap-same-run-at-wp*

wp-event
 $(\text{checked-staged-security-experiment-with-actual-alpha-prefix-data-state } A)$
 $(\text{checked-staged-security-with-actual-alpha-prefix-query-prefix-fixed-candidate-pair-witness-gap-same-run-at}$
 $\text{trace-openings composition-openings trace-table composition-table } i A)$
 $\text{adversary-initial-state} =$
 wp-event
 $(\text{checked-staged-security-experiment-with-data-state } A)$
 $(\text{staged-security-with-data-state-query-prefix-fixed-candidate-pair-witness-gap-same-run-at}$
 $\text{trace-openings composition-openings trace-table composition-table } i A)$
 $\text{adversary-initial-state}$

```

proof –
  let ?project =
    λx. ((snd (fst (fst x)), snd (fst x)), snd x)
  have
    wp-event
      (checked-staged-security-experiment-with-data-state A)
      (staged-security-with-data-state-query-prefix-fixed-candidate-pair-witness-gap-same-run-at
        trace-openings composition-openings trace-table composition-table i A)
      adversary-initial-state =
    wp-event
      (checked-staged-security-experiment-with-actual-alpha-prefix-data-state A
        ≫ (λx. return (?project x)))
      (staged-security-with-data-state-query-prefix-fixed-candidate-pair-witness-gap-same-run-at
        trace-openings composition-openings trace-table composition-table i A)
      adversary-initial-state
  using
    checked-staged-security-experiment-with-actual-alpha-prefix-data-state-projection
    [of A]
  by simp
also have ... =
  wp-event
    (checked-staged-security-experiment-with-actual-alpha-prefix-data-state A)
    (checked-staged-security-with-actual-alpha-prefix-query-prefix-fixed-candidate-pair-witness-gap-same-run-at
      trace-openings composition-openings trace-table composition-table i A)
    adversary-initial-state
proof –
  have pred-eq:
    (λout. case out of
      None ⇒
        staged-security-with-data-state-query-prefix-fixed-candidate-pair-witness-gap-same-run-at
          trace-openings composition-openings trace-table composition-table i A
        None
      | Some (x, t) ⇒
        staged-security-with-data-state-query-prefix-fixed-candidate-pair-witness-gap-same-run-at
          trace-openings composition-openings trace-table composition-table i A
        (Some (?project x, t))) =
    checked-staged-security-with-actual-alpha-prefix-query-prefix-fixed-candidate-pair-witness-gap-same-run-at
      trace-openings composition-openings trace-table composition-table i A
  by (rule ext)
    (simp add:
      checked-staged-security-with-actual-alpha-prefix-query-prefix-fixed-candidate-pair-witness-gap-same-run-at-proje
      staged-security-with-data-state-query-prefix-fixed-candidate-pair-witness-gap-same-run-at-def
      split: option.splits prod.splits)
  show ?thesis
    by (subst wp-event-bind-return-map) (simp add: pred-eq)
qed
finally show ?thesis
  by simp
qed

```

lemma *checked-staged-security-with-actual-alpha-prefix-query-prefix-fixed-candidate-pair-witness-gap-same-run-at-bound*
assumes *bound*:

wp-event
 (*checked-staged-security-experiment-with-data-state A*)
 (*staged-security-with-data-state-query-prefix-fixed-candidate-pair-witness-gap-same-run-at*
trace-openings composition-openings trace-table composition-table i A)
adversary-initial-state $\leq B$

shows

wp-event
 (*checked-staged-security-experiment-with-actual-alpha-prefix-data-state A*)
 (*checked-staged-security-with-actual-alpha-prefix-query-prefix-fixed-candidate-pair-witness-gap-same-run-at*
trace-openings composition-openings trace-table composition-table i A)
adversary-initial-state $\leq B$

using *bound*

unfolding

checked-staged-security-with-actual-alpha-prefix-query-prefix-fixed-candidate-pair-witness-gap-same-run-at-wp-project
by *simp*

lemma *checked-staged-security-with-actual-alpha-prefix-query-prefix-fixed-candidate-pair-witness-gap-same-run-exists-at*
checked-staged-security-with-actual-alpha-prefix-query-prefix-fixed-candidate-pair-witness-gap-same-run-exists-at
i A out

$\longleftrightarrow$

(*case out of*

None $\Rightarrow$ *False*

| *Some* (((*alpha-prefix*, *alpha-prefix-state*), *data*), *attacker-state*),
result), *final-state*) $\Rightarrow$

staged-security-with-data-state-query-prefix-fixed-candidate-pair-witness-gap-same-run-exists-at
i A (*Some* (((*data*, *attacker-state*), *result*), *final-state*)))

unfolding

checked-staged-security-with-actual-alpha-prefix-query-prefix-fixed-candidate-pair-witness-gap-same-run-exists-at-def
staged-security-with-data-state-query-prefix-fixed-candidate-pair-witness-gap-same-run-exists-at-def

by (*auto simp*:

checked-staged-security-with-actual-alpha-prefix-query-prefix-fixed-candidate-pair-witness-gap-same-run-at-projecti
split: option.splits prod.splits)

lemma *checked-staged-security-with-actual-alpha-prefix-query-prefix-fixed-candidate-pair-witness-gap-same-run-exists*
wp-event

(*checked-staged-security-experiment-with-actual-alpha-prefix-data-state A*)

(*checked-staged-security-with-actual-alpha-prefix-query-prefix-fixed-candidate-pair-witness-gap-same-run-exists-at*
i A)

adversary-initial-state =

wp-event

(*checked-staged-security-experiment-with-data-state A*)

(*staged-security-with-data-state-query-prefix-fixed-candidate-pair-witness-gap-same-run-exists-at*
i A)

adversary-initial-state

proof –

let *?project* =

```

λx. ((snd (fst (fst x)), snd (fst x)), snd x)
have
  wp-event
    (checked-staged-security-experiment-with-data-state A)
  (staged-security-with-data-state-query-prefix-fixed-candidate-pair-witness-gap-same-run-exists-at
    i A)
  adversary-initial-state =
  wp-event
    (checked-staged-security-experiment-with-actual-alpha-prefix-data-state A
      ≫ (λx. return (?project x)))
  (staged-security-with-data-state-query-prefix-fixed-candidate-pair-witness-gap-same-run-exists-at
    i A)
  adversary-initial-state
using
  checked-staged-security-experiment-with-actual-alpha-prefix-data-state-projection
  [of A]
by simp
also have ... =
  wp-event
    (checked-staged-security-experiment-with-actual-alpha-prefix-data-state A)
  (checked-staged-security-with-actual-alpha-prefix-query-prefix-fixed-candidate-pair-witness-gap-same-run-exists-at
    i A)
  adversary-initial-state
proof -
have pred-eq:
  (λout. case out of
    None ⇒
      staged-security-with-data-state-query-prefix-fixed-candidate-pair-witness-gap-same-run-exists-at
        i A None
    | Some (x, t) ⇒
      staged-security-with-data-state-query-prefix-fixed-candidate-pair-witness-gap-same-run-exists-at
        i A (Some (?project x, t))) =
  checked-staged-security-with-actual-alpha-prefix-query-prefix-fixed-candidate-pair-witness-gap-same-run-exists-at
    i A
by (rule ext)
  (simp add:
    checked-staged-security-with-actual-alpha-prefix-query-prefix-fixed-candidate-pair-witness-gap-same-run-exists-at
    staged-security-with-data-state-query-prefix-fixed-candidate-pair-witness-gap-same-run-exists-at-def
    staged-security-with-data-state-query-prefix-fixed-candidate-pair-witness-gap-same-run-at-def
    split: option.splits prod.splits)
show ?thesis
by (subst wp-event-bind-return-map) (simp add: pred-eq)
qed
finally show ?thesis
by simp
qed

lemma checked-staged-security-with-query-prefix-single-round-candidate-pair-witness-gap-exists-at-le-same-run-data-
assumes i-bound: i < rounds

```


shows

wp-event
(*checked-staged-security-experiment-with-query-prefix-data-state A i*)
(*checked-staged-security-with-query-prefix-single-round-candidate-pair-witness-gap-exists-at i*)
adversary-initial-state $\leq$
wp-event
(*checked-staged-security-experiment-with-data-state A*)
(*staged-security-with-data-state-query-prefix-fixed-candidate-pair-witness-gap-same-run-exists-at i A*)
adversary-initial-state

proof –

let *?M* =
 checked-staged-security-experiment-with-query-prefix-data-state A i
let *?Gap* =
 checked-staged-security-with-query-prefix-single-round-candidate-pair-witness-gap-exists-at i
let *?DataGap* =
 staged-security-with-data-state-query-prefix-fixed-candidate-pair-witness-gap-same-run-exists-at i A
let *?projected* =
 $\lambda out. \text{case out of}$
 None $\Rightarrow$ *?DataGap None*
 | *Some (packed, t)* $\Rightarrow$ *?DataGap (Some (snd packed, t))*
have *projection*:
 wp-event (checked-staged-security-experiment-with-data-state A)
 ?DataGap adversary-initial-state =
 wp-event ?M ?projected adversary-initial-state
 by (rule *checked-staged-security-experiment-with-query-prefix-data-state-projection-event*
 [OF *i-bound*])

have *event-le*:

wp-event ?M ?Gap adversary-initial-state $\leq$
wp-event ?M ?projected adversary-initial-state

proof (rule *wp-event-mono-on-support*)

fix *out*

assume *support*: *out* $\in$ *set-dist (execute ?M adversary-initial-state)*

and *gap*: *?Gap out*

show *?projected out*

proof (*cases out*)

case *None*

then show *?thesis*

using *gap*

unfolding

checked-staged-security-with-query-prefix-single-round-candidate-pair-witness-gap-exists-at-def
 checked-staged-security-with-query-prefix-single-round-candidate-pair-witness-gap-at-def
 checked-staged-security-with-query-prefix-dynamic-index-hit-def

by *simp*

next

case (*Some packed*)

```

then obtain prefix prefix-state raw raw-state data attacker-state result
final-state where out-eq:
  out =
    Some (((((prefix, prefix-state), raw), raw-state),
      ((data, attacker-state), result)), final-state)
  by (cases packed) (auto split: prod.splits)
from gap obtain trace-openings composition-openings trace-table
composition-table where fixed-gap:
checked-staged-security-with-query-prefix-single-round-candidate-pair-witness-gap-at
trace-openings composition-openings trace-table composition-table i
out
unfolding
checked-staged-security-with-query-prefix-single-round-candidate-pair-witness-gap-exists-at-def
by blast
have qsupport:
  Some (((((prefix, prefix-state), raw), raw-state),
    ((data, attacker-state), result)), final-state) ∈
    set-dist (execute ?M adversary-initial-state)
  using support out-eq by simp
have fixed-gap-out:
checked-staged-security-with-query-prefix-single-round-candidate-pair-witness-gap-at
trace-openings composition-openings trace-table composition-table i
  (Some (((((prefix, prefix-state), raw), raw-state),
    ((data, attacker-state), result)), final-state))
  using fixed-gap out-eq by simp
have data-gap:
staged-security-with-data-state-query-prefix-fixed-candidate-pair-witness-gap-same-run-at
trace-openings composition-openings trace-table composition-table i A
  (Some (((data, attacker-state), result), final-state))
unfolding
staged-security-with-data-state-query-prefix-fixed-candidate-pair-witness-gap-same-run-at-def
using qsupport fixed-gap-out by auto
have projected-gap:
  ?DataGap (Some (((data, attacker-state), result), final-state))
unfolding
staged-security-with-data-state-query-prefix-fixed-candidate-pair-witness-gap-same-run-exists-at-def
using data-gap by blast
show ?thesis
  unfolding out-eq
  using projected-gap by simp
qed
qed
show ?thesis
  unfolding projection
  by (rule event-le)
qed

lemma checked-staged-security-with-actual-alpha-prefix-query-prefix-fixed-candidate-pair-witness-gap-same-run-atE:
  assumes gap:

```

checked-staged-security-with-actual-alpha-prefix-query-prefix-fixed-candidate-pair-witness-gap-same-run-at
trace-openings composition-openings trace-table composition-table i A
(Some (((((alpha-prefix, alpha-prefix-state), data), attacker-state),
result), final-state))
obtains *prefix prefix-state raw raw-state where*
Some (((((prefix, prefix-state), raw), raw-state),
((data, attacker-state), result)), final-state) ∈
set-dist
(execute
(checked-staged-security-experiment-with-query-prefix-data-state A i)
adversary-initial-state)
checked-staged-security-with-query-prefix-single-round-candidate-pair-witness-gap-at
trace-openings composition-openings trace-table composition-table i
(Some (((((prefix, prefix-state), raw), raw-state),
((data, attacker-state), result)), final-state))
using *gap*
unfolding
checked-staged-security-with-actual-alpha-prefix-query-prefix-fixed-candidate-pair-witness-gap-same-run-at-def
by (*auto split: prod.splits*)

lemma *checked-staged-security-with-actual-alpha-prefix-query-prefix-fixed-candidate-pair-witness-gap-same-run-atI:*
assumes *support:*
Some (((((prefix, prefix-state), raw), raw-state),
((data, attacker-state), result)), final-state) ∈
set-dist
(execute
(checked-staged-security-experiment-with-query-prefix-data-state A i)
adversary-initial-state)
and *gap:*
checked-staged-security-with-query-prefix-single-round-candidate-pair-witness-gap-at
trace-openings composition-openings trace-table composition-table i
(Some (((((prefix, prefix-state), raw), raw-state),
((data, attacker-state), result)), final-state))
shows
checked-staged-security-with-actual-alpha-prefix-query-prefix-fixed-candidate-pair-witness-gap-same-run-at
trace-openings composition-openings trace-table composition-table i A
(Some (((((alpha-prefix, alpha-prefix-state), data), attacker-state),
result), final-state))
unfolding
checked-staged-security-with-actual-alpha-prefix-query-prefix-fixed-candidate-pair-witness-gap-same-run-at-def
using *support gap by auto*

lemma *checked-staged-security-with-actual-alpha-prefix-query-prefix-fixed-candidate-pair-witness-gap-same-run-exists*
assumes *gap:*
checked-staged-security-with-actual-alpha-prefix-query-prefix-fixed-candidate-pair-witness-gap-same-run-at
trace-openings composition-openings trace-table composition-table i A out
shows
checked-staged-security-with-actual-alpha-prefix-query-prefix-fixed-candidate-pair-witness-gap-same-run-exists-at
i A out

unfolding

checked-staged-security-with-actual-alpha-prefix-query-prefix-fixed-candidate-pair-witness-gap-same-run-exists-at-def
using *gap* **by** *blast*

lemma *checked-staged-security-with-query-prefix-single-round-candidate-pair-witness-gap-exists-atI*:

assumes *gap*:

checked-staged-security-with-query-prefix-single-round-candidate-pair-witness-gap-at
trace-openings composition-openings trace-table composition-table i out

shows

checked-staged-security-with-query-prefix-single-round-candidate-pair-witness-gap-exists-at
i out

unfolding

checked-staged-security-with-query-prefix-single-round-candidate-pair-witness-gap-exists-at-def
using *gap* **by** *blast*

lemma *checked-staged-security-with-actual-alpha-prefix-query-prefix-fixed-candidate-pair-witness-gap-same-run-exists-at*

assumes *gap*:

checked-staged-security-with-actual-alpha-prefix-query-prefix-fixed-candidate-pair-witness-gap-same-run-exists-at
i A out

shows

checked-staged-security-with-actual-alpha-prefix-witness-transcript-hit
out

proof (*cases out*)

case *None*

then show *?thesis*

using *gap*

unfolding

checked-staged-security-with-actual-alpha-prefix-query-prefix-fixed-candidate-pair-witness-gap-same-run-exists-at-def
checked-staged-security-with-actual-alpha-prefix-query-prefix-fixed-candidate-pair-witness-gap-same-run-at-def

by *simp*

next

case (*Some packed*)

then obtain *alpha-prefix alpha-prefix-state data attacker-state result*

final-state **where** *out-eq*:

out =

Some (((((*alpha-prefix*, *alpha-prefix-state*), *data*), *attacker-state*),
result), *final-state*)

by (*cases packed*, *auto split: prod.splits*)

from *gap* **obtain** *trace-openings composition-openings trace-table*

composition-table prefix prefix-state raw raw-state **where** *support*:

Some (((((*prefix*, *prefix-state*), *raw*), *raw-state*),

((*data*, *attacker-state*), *result*)), *final-state*) ∈

set-dist

(*execute*

(*checked-staged-security-experiment-with-query-prefix-data-state*

A i)

adversary-initial-state)

and *fixed-gap*:

checked-staged-security-with-query-prefix-single-round-candidate-pair-witness-gap-at

```

    trace-openings composition-openings trace-table composition-table i
    (Some (((((prefix, prefix-state), raw), raw-state),
      ((data, attacker-state), result)), final-state))
unfolding out-eq
checked-staged-security-with-actual-alpha-prefix-query-prefix-fixed-candidate-pair-witness-gap-same-run-exists-at-d
by (auto elim!:
  checked-staged-security-with-actual-alpha-prefix-query-prefix-fixed-candidate-pair-witness-gap-same-run-atE)
have witness-side:
checked-staged-security-with-query-prefix-single-round-candidate-binding-gap-witness-side
  trace-openings composition-openings trace-table composition-table i
  (Some (((((prefix, prefix-state), raw), raw-state),
    ((data, attacker-state), result)), final-state))
by (rule
  checked-staged-security-with-query-prefix-single-round-candidate-pair-witness-gap-at-imp-witness-side
    [OF fixed-gap])
let ?s =
  verifier-state-from-adversary attacker-state
  (staged-proof-transcript data)
from witness-side have witness:
  (trace-openings, composition-openings) ∈
  query-header-supported-partial-opening-witnesses ?s
  (staged-trace-root data)
  (staged-trace-fri-roots data)
  (staged-trace-final data)
  (staged-alphas data)
  (staged-degree data)
  (staged-composition-fri-roots data)
  (staged-composition-final data)
unfolding
checked-staged-security-with-query-prefix-single-round-candidate-binding-gap-witness-side-def
by simp
from query-header-supported-partial-opening-witnessesE[OF witness]
obtain witness-result witness-state rest where outcome:
  Some (witness-result, witness-state) ∈
  set-dist (execute verify-monad ?s)
by blast
have tr-eq: PTranscript ?s = staged-proof-transcript data
by simp
have transcript-hit:
  hash-map-output-values ?s ∩ set (staged-proof-transcript data)
  ≠ {} ∨
  hash-map-new-output-hit (set (staged-proof-transcript data))
  ?s witness-state
by (rule verify-monad-trace-root-target-preexisting-or-new
  [OF outcome tr-eq])
show ?thesis
unfolding out-eq
checked-staged-security-with-actual-alpha-prefix-witness-transcript-hit-def
Let-def

```

using *outcome transcript-hit*
 by (*auto split: prod.splits*)
 qed

lemma *checked-staged-security-with-actual-alpha-prefix-query-prefix-fixed-candidate-pair-witness-gap-same-run-exists*

wp-event

(*checked-staged-security-experiment-with-actual-alpha-prefix-data-state A*)

(*checked-staged-security-with-actual-alpha-prefix-query-prefix-fixed-candidate-pair-witness-gap-same-run-exists-at*
i A)

adversary-initial-state $\leq$

wp-event

(*checked-staged-security-experiment-with-actual-alpha-prefix-data-state A*)

checked-staged-security-with-actual-alpha-prefix-witness-transcript-hit

adversary-initial-state

by (*rule wp-event-mono*)

(*rule*

checked-staged-security-with-actual-alpha-prefix-query-prefix-fixed-candidate-pair-witness-gap-same-run-exists-at-i

lemma *checked-staged-security-with-actual-alpha-prefix-query-prefix-fixed-candidate-pair-witness-gap-same-run-exists*

assumes *pre-bound*:

wp-event

(*checked-staged-security-experiment-with-actual-alpha-prefix-data-state*
A)

checked-staged-security-with-actual-alpha-prefix-witness-transcript-pre-hit

adversary-initial-state $\leq P$

and *new-bound*:

wp-event

(*checked-staged-security-experiment-with-actual-alpha-prefix-data-state*
A)

checked-staged-security-with-actual-alpha-prefix-witness-transcript-new-hit

adversary-initial-state $\leq Q$

shows

wp-event

(*checked-staged-security-experiment-with-actual-alpha-prefix-data-state A*)

(*checked-staged-security-with-actual-alpha-prefix-query-prefix-fixed-candidate-pair-witness-gap-same-run-exists-at*
i A)

adversary-initial-state $\leq P + Q$

by (*rule order-trans*

[*OF*

checked-staged-security-with-actual-alpha-prefix-query-prefix-fixed-candidate-pair-witness-gap-same-run-exists-at-

checked-staged-security-with-actual-alpha-prefix-witness-transcript-hit-bound-from-pre-and-new

[*OF pre-bound new-bound*]])

lemma *checked-staged-security-with-actual-alpha-prefix-query-prefix-fixed-candidate-pair-witness-gap-same-run-exists*

assumes *data-pre-bound*:

wp-event (*checked-staged-security-experiment-with-data-state A*)

staged-security-with-data-state-transcript-pre-hit

adversary-initial-state $\leq P$

and *new-bound*:

wp-event
 (*checked-staged-security-experiment-with-actual-alpha-prefix-data-state*
 A)
checked-staged-security-with-actual-alpha-prefix-witness-transcript-new-hit
adversary-initial-state $\leq Q$

shows

wp-event
 (*checked-staged-security-experiment-with-actual-alpha-prefix-data-state* *A*)
 (*checked-staged-security-with-actual-alpha-prefix-query-prefix-fixed-candidate-pair-witness-gap-same-run-exists-at*
 i A)
adversary-initial-state $\leq P + Q$

proof –

have *pre-bound*:

wp-event
 (*checked-staged-security-experiment-with-actual-alpha-prefix-data-state* *A*)
checked-staged-security-with-actual-alpha-prefix-witness-transcript-pre-hit
adversary-initial-state $\leq P$

by (rule
checked-staged-security-with-actual-alpha-prefix-witness-transcript-pre-hit-bound-from-data-state-pre-hit
 [OF *data-pre-bound*])

show *?thesis*

by (rule
checked-staged-security-with-actual-alpha-prefix-query-prefix-fixed-candidate-pair-witness-gap-same-run-exists-at
 [OF *pre-bound new-bound*])

qed

lemma *staged-security-with-data-state-query-prefix-fixed-candidate-pair-witness-gap-same-run-exists-at-bound-from-*

assumes *data-pre-bound*:

wp-event (*checked-staged-security-experiment-with-data-state* *A*)
staged-security-with-data-state-transcript-pre-hit
adversary-initial-state $\leq P$

and *new-bound*:

wp-event
 (*checked-staged-security-experiment-with-actual-alpha-prefix-data-state*
 A)
checked-staged-security-with-actual-alpha-prefix-witness-transcript-new-hit
adversary-initial-state $\leq Q$

shows

wp-event
 (*checked-staged-security-experiment-with-data-state* *A*)
 (*staged-security-with-data-state-query-prefix-fixed-candidate-pair-witness-gap-same-run-exists-at*
 i A)
adversary-initial-state $\leq P + Q$

using

checked-staged-security-with-actual-alpha-prefix-query-prefix-fixed-candidate-pair-witness-gap-same-run-exists-at-bo
 [OF *data-pre-bound new-bound*, of *i*]

unfolding

checked-staged-security-with-actual-alpha-prefix-query-prefix-fixed-candidate-pair-witness-gap-same-run-exists-at-wp

by *simp*

lemma *checked-staged-security-with-query-prefix-single-round-candidate-pair-witness-gap-exists-at-bound-from-data-p*

assumes *i-bound*: $i < \text{rounds}$

and *data-pre-bound*:

wp-event (*checked-staged-security-experiment-with-data-state* *A*)

staged-security-with-data-state-transcript-pre-hit

adversary-initial-state $\leq P$

and *new-bound*:

wp-event

(*checked-staged-security-experiment-with-actual-alpha-prefix-data-state* *A*)

checked-staged-security-with-actual-alpha-prefix-witness-transcript-new-hit

adversary-initial-state $\leq Q$

shows

wp-event

(*checked-staged-security-experiment-with-query-prefix-data-state* *A i*)

(*checked-staged-security-with-query-prefix-single-round-candidate-pair-witness-gap-exists-at* *i*)

adversary-initial-state $\leq P + Q$

by (*rule order-trans*[

OF checked-staged-security-with-query-prefix-single-round-candidate-pair-witness-gap-exists-at-le-same-run-data-
OF i-bound]

staged-security-with-data-state-query-prefix-fixed-candidate-pair-witness-gap-same-run-exists-at-bound-from-data-
OF data-pre-bound new-bound]])

lemma *checked-staged-security-with-actual-alpha-prefix-candidate-pair-query-hit-from-prefix-with-witness-imp-single-*

assumes *hit*:

checked-staged-security-with-actual-alpha-prefix-candidate-pair-query-hit-from-prefix

A trace-table composition-table i

(*Some* (((*alpha-prefix*, *alpha-prefix-state*), *data*), *attacker-state*),
result), *final-state*))

and *witness*:

(*trace-openings*, *composition-openings*) $\in$

query-header-supported-partial-opening-witnesses

(*verifier-state-from-adversary attacker-state*

(*staged-proof-transcript data*))

(*staged-trace-root data*)

(*staged-trace-fri-roots data*)

(*staged-trace-final data*)

(*staged-alphas data*)

(*staged-degree data*)

(*staged-composition-fri-roots data*)

(*staged-composition-final data*)

and *trace-candidate*:

partial-trace-table-candidate trace-table

((*replicate rounds* []) [*i* := *trace-openings*])

and *comp-candidate*:

partial-composition-table-candidate composition-table

((*replicate rounds* []) [*i* := *composition-openings*])

and *same-run-support*:

$\bigwedge \text{prefix prefix-state raw raw-state.}$
Some (*data*, *attacker-state*) $\in$
set-dist
 (*execute* (*checked-staged-transcript-program* *A*)
adversary-initial-state) $\implies$
Some (((*prefix*, *prefix-state*), *raw*), *raw-state*) $\in$
set-dist
 (*execute* (*checked-staged-query-prefix-receive-with-state* *A i*)
adversary-initial-state) $\implies$
checked-staged-query-prefix-dynamic-index-hit
 (*staged-query-prefix-candidate-pair-query-target-from-prefix*
trace-table composition-table)
 (*Some* (((*prefix*, *prefix-state*), *raw*), *raw-state*)) $\implies$
Some (((((*prefix*, *prefix-state*), *raw*), *raw-state*),
 ((*data*, *attacker-state*), *result*)), *final-state*) $\in$
set-dist
 (*execute*
 (*checked-staged-security-experiment-with-query-prefix-data-state*
A i)
adversary-initial-state)

shows

checked-staged-security-with-actual-alpha-prefix-single-round-header-supported-query-target-hit-at
i A
 (*Some* (((((*alpha-prefix*, *alpha-prefix-state*), *data*), *attacker-state*),
result), *final-state*)) $\vee$
checked-staged-security-with-actual-alpha-prefix-query-prefix-fixed-candidate-pair-witness-gap-same-run-at
trace-openings composition-openings trace-table composition-table i A
 (*Some* (((((*alpha-prefix*, *alpha-prefix-state*), *data*), *attacker-state*),
result), *final-state*))

proof —

from

checked-staged-security-with-actual-alpha-prefix-candidate-pair-query-hit-from-prefixE
 [*OF hit*]

obtain *prefix prefix-state raw raw-state* **where**

builder:

Some (*data*, *attacker-state*) $\in$
set-dist
 (*execute* (*checked-staged-transcript-program* *A*)
adversary-initial-state)

and *prefix-receive*:

Some (((*prefix*, *prefix-state*), *raw*), *raw-state*) $\in$
set-dist
 (*execute* (*checked-staged-query-prefix-receive-with-state* *A i*)
adversary-initial-state)

and *target-hit*:

checked-staged-query-prefix-dynamic-index-hit
 (*staged-query-prefix-candidate-pair-query-target-from-prefix*
trace-table composition-table)

```

    (Some (((prefix, prefix-state), raw), raw-state))
  by (rule
    checked-staged-security-with-actual-alpha-prefix-candidate-pair-query-hit-from-prefixE
    [OF hit])
  have support:
    Some (((((prefix, prefix-state), raw), raw-state),
      ((data, attacker-state), result)), final-state) ∈
    set-dist
      (execute
        (checked-staged-security-experiment-with-query-prefix-data-state
          A i)
        adversary-initial-state)
    by (rule same-run-support[OF builder prefix-receive target-hit])
  show ?thesis
  proof (cases
    (trace-table, composition-table) ∈
    query-header-supported-single-round-partial-table-candidates
    prefix-state
    (sqp-trace-root prefix)
    (sqp-trace-fri-roots prefix)
    (sqp-trace-final prefix)
    (sqp-alphas prefix)
    (sqp-degree prefix)
    (sqp-composition-fri-roots prefix)
    (sqp-composition-final prefix)
    i)
  case True
  have single-round-hit:
    checked-staged-query-prefix-dynamic-index-hit
    (staged-query-prefix-single-round-header-supported-query-target i)
    (Some (((prefix, prefix-state), raw), raw-state))
  by (rule
    checked-staged-query-prefix-candidate-pair-hit-imp-single-round-header-target-hit
    [OF True target-hit])
  have
    checked-staged-security-with-actual-alpha-prefix-single-round-header-supported-query-target-hit-at
    i A
    (Some (((((alpha-prefix, alpha-prefix-state), data), attacker-state),
      result), final-state))
  proof -
    have inner:
      ∃ prefix prefix-state raw raw-state.
      Some (data, attacker-state) ∈
      set-dist
        (execute (checked-staged-transcript-program A)
          adversary-initial-state) ∧
      Some (((prefix, prefix-state), raw), raw-state) ∈
      set-dist
        (execute (checked-staged-query-prefix-receive-with-state A i)

```

```

      adversary-initial-state) ∧
      checked-staged-query-prefix-dynamic-index-hit
      (staged-query-prefix-single-round-header-supported-query-target i)
      (Some (((prefix, prefix-state), raw), raw-state))
    by (intro exI[of - prefix] exI[of - prefix-state]
        exI[of - raw] exI[of - raw-state])
      (use builder prefix-receive single-round-hit in simp)
  show ?thesis
    unfolding
      checked-staged-security-with-actual-alpha-prefix-single-round-header-supported-query-target-hit-at-def
    using inner by simp
qed
then show ?thesis by simp
next
case False
have success:
  index (to-nat raw) ∈
    query-sampling-success-space trace-table composition-table
    (sqp-alphas prefix)
  using target-hit
  unfolding
    checked-staged-query-prefix-dynamic-index-hit-def
    staged-query-prefix-candidate-pair-query-target-from-prefix-def
  by simp
have fixed-gap:
  checked-staged-security-with-query-prefix-single-round-candidate-pair-witness-gap-at
  trace-openings composition-openings trace-table composition-table i
  (Some (((((prefix, prefix-state), raw), raw-state),
    ((data, attacker-state), result)), final-state))
proof -
  have dyn:
    checked-staged-security-with-query-prefix-dynamic-index-hit
    (staged-query-prefix-candidate-pair-query-target-from-prefix
      trace-table composition-table)
    (Some (((((prefix, prefix-state), raw), raw-state),
      ((data, attacker-state), result)), final-state))
  unfolding
    checked-staged-security-with-query-prefix-dynamic-index-hit-def
    checked-staged-query-prefix-dynamic-index-hit-def
    staged-query-prefix-candidate-pair-query-target-from-prefix-def
  using success by simp
  have side:
    checked-staged-security-with-query-prefix-single-round-candidate-binding-gap-witness-side
    trace-openings composition-openings trace-table composition-table i
    (Some (((((prefix, prefix-state), raw), raw-state),
      ((data, attacker-state), result)), final-state))
  unfolding
    checked-staged-security-with-query-prefix-single-round-candidate-binding-gap-witness-side-def
    checked-staged-security-with-query-prefix-single-round-candidate-binding-gap-side-def

```

```

    using witness trace-candidate comp-candidate False by simp
show ?thesis
  unfolding
    checked-staged-security-with-query-prefix-single-round-candidate-pair-witness-gap-at-def
  using dyn side by simp
qed
have
  checked-staged-security-with-actual-alpha-prefix-query-prefix-fixed-candidate-pair-witness-gap-same-run-at
    trace-openings composition-openings trace-table composition-table i A
    (Some (((alpha-prefix, alpha-prefix-state), data), attacker-state),
      result), final-state))
  by (rule
    checked-staged-security-with-actual-alpha-prefix-query-prefix-fixed-candidate-pair-witness-gap-same-run-atI
      [OF support fixed-gap])
then show ?thesis by simp
qed
qed

lemma checked-staged-security-with-actual-alpha-prefix-candidate-pair-success-with-witness-imp-single-round-header-
  assumes i-bound: i < rounds
  and support:
    Some (((alpha-prefix, alpha-prefix-state), data), attacker-state),
      result), final-state) ∈
    set-dist
      (execute
        (checked-staged-security-experiment-with-actual-alpha-prefix-data-state
          A)
        adversary-initial-state)
  and success:
     $\bigwedge$  prefix prefix-state raw raw-state.
    Some (((prefix, prefix-state), raw), raw-state),
      ((data, attacker-state), result)), final-state) ∈
    set-dist
      (execute
        (checked-staged-security-experiment-with-query-prefix-data-state
          A i)
        adversary-initial-state)  $\implies$ 
    sqp-alphas prefix = staged-alphas data  $\implies$ 
    index (to-nat raw) ∈
      query-sampling-success-space trace-table composition-table
        (staged-alphas data)
  and witness:
    (trace-openings, composition-openings) ∈
      query-header-supported-partial-opening-witnesses
        (verifier-state-from-adversary attacker-state
          (staged-proof-transcript data))
        (staged-trace-root data)
        (staged-trace-fri-roots data)
        (staged-trace-final data)

```

```

    (staged-alias data)
    (staged-degree data)
    (staged-composition-fri-roots data)
    (staged-composition-final data)
and trace-candidate:
    partial-trace-table-candidate trace-table
    ((replicate rounds []) [i := trace-openings])
and comp-candidate:
    partial-composition-table-candidate composition-table
    ((replicate rounds []) [i := composition-openings])
shows
checked-staged-security-with-actual-alpha-prefix-single-round-header-supported-query-target-hit-at
i A
(Some (((((alpha-prefix, alpha-prefix-state), data), attacker-state),
result), final-state)) ∨
checked-staged-security-with-actual-alpha-prefix-query-prefix-fixed-candidate-pair-witness-gap-same-run-at
trace-openings composition-openings trace-table composition-table i A
(Some (((((alpha-prefix, alpha-prefix-state), data), attacker-state),
result), final-state))
proof (rule
checked-staged-security-with-actual-alpha-prefix-candidate-pair-success-query-prefix-hitE
[OF i-bound support success])
fix prefix prefix-state raw raw-state
assume qsupport:
Some (((((prefix, prefix-state), raw), raw-state),
((data, attacker-state), result)), final-state) ∈
set-dist
(execute
checked-staged-security-experiment-with-query-prefix-data-state A i)
adversary-initial-state)
and qhit:
checked-staged-security-with-query-prefix-dynamic-index-hit
(staged-query-prefix-candidate-pair-query-target-from-prefix
trace-table composition-table)
(Some (((((prefix, prefix-state), raw), raw-state),
((data, attacker-state), result)), final-state))
have data-support:
Some (((data, attacker-state), result), final-state) ∈
set-dist
(execute (checked-staged-security-experiment-with-data-state A)
adversary-initial-state)
by (rule
checked-staged-security-experiment-with-query-prefix-data-state-imp-data-state-support
[OF i-bound qsupport])
from checked-staged-security-experiment-with-data-state-outcomeE
[OF data-support]
obtain builder where builder:
Some (data, attacker-state) ∈
set-dist

```

```

      (execute (checked-staged-transcript-program A)
        adversary-initial-state)
    by blast
  have prefix-receive:
    Some (((prefix, prefix-state), raw), raw-state) ∈
      set-dist
        (execute (checked-staged-query-prefix-receive-with-state A i)
          adversary-initial-state)
  using qsupport
  unfolding checked-staged-security-experiment-with-query-prefix-data-state-def
  by (auto elim!: set-dist-bindE split: prod.splits)
  have raw-pair-hit:
    checked-staged-query-prefix-dynamic-index-hit
      (staged-query-prefix-candidate-pair-query-target-from-prefix
        trace-table composition-table)
      (Some (((prefix, prefix-state), raw), raw-state))
  using qhit
  unfolding checked-staged-security-with-query-prefix-dynamic-index-hit-def
  by simp
  show ?thesis
  proof (cases
    (trace-table, composition-table) ∈
      query-header-supported-single-round-partial-table-candidates
        prefix-state
        (sqp-trace-root prefix)
        (sqp-trace-fri-roots prefix)
        (sqp-trace-final prefix)
        (sqp-alphas prefix)
        (sqp-degree prefix)
        (sqp-composition-fri-roots prefix)
        (sqp-composition-final prefix)
        i)
  case True
  have single-round-hit:
    checked-staged-query-prefix-dynamic-index-hit
      (staged-query-prefix-single-round-header-supported-query-target i)
      (Some (((prefix, prefix-state), raw), raw-state))
  by (rule
    checked-staged-query-prefix-candidate-pair-hit-imp-single-round-header-target-hit
    [OF True raw-pair-hit])
  have
    checked-staged-security-with-actual-alpha-prefix-single-round-header-supported-query-target-hit-at
      i A
      (Some (((((alpha-prefix, alpha-prefix-state), data), attacker-state),
        result), final-state))
  proof -
    have inner:
      ∃ prefix prefix-state raw raw-state.
        Some (data, attacker-state) ∈

```

```

    set-dist
      (execute (checked-staged-transcript-program A)
        adversary-initial-state) ∧
    Some (((prefix, prefix-state), raw), raw-state) ∈
    set-dist
      (execute (checked-staged-query-prefix-receive-with-state A i)
        adversary-initial-state) ∧
    checked-staged-query-prefix-dynamic-index-hit
      (staged-query-prefix-single-round-header-supported-query-target i)
      (Some (((prefix, prefix-state), raw), raw-state))
  by (intro exI[of - prefix] exI[of - prefix-state]
    exI[of - raw] exI[of - raw-state])
    (use builder prefix-receive single-round-hit in simp)
show ?thesis
  unfolding
    checked-staged-security-with-actual-alpha-prefix-single-round-header-supported-query-target-hit-at-def
  using inner by simp
qed
then show ?thesis by simp
next
case False
have side:
  checked-staged-security-with-query-prefix-single-round-candidate-binding-gap-witness-side
    trace-openings composition-openings trace-table composition-table i
    (Some (((((prefix, prefix-state), raw), raw-state),
      ((data, attacker-state), result)), final-state))
  unfolding
    checked-staged-security-with-query-prefix-single-round-candidate-binding-gap-witness-side-def
    checked-staged-security-with-query-prefix-single-round-candidate-binding-gap-side-def
  using witness trace-candidate comp-candidate False by simp
have fixed-gap:
  checked-staged-security-with-query-prefix-single-round-candidate-pair-witness-gap-at
    trace-openings composition-openings trace-table composition-table i
    (Some (((((prefix, prefix-state), raw), raw-state),
      ((data, attacker-state), result)), final-state))
  unfolding
    checked-staged-security-with-query-prefix-single-round-candidate-pair-witness-gap-at-def
  using qhit side by simp
have
  checked-staged-security-with-actual-alpha-prefix-query-prefix-fixed-candidate-pair-witness-gap-same-run-at
    trace-openings composition-openings trace-table composition-table i A
    (Some (((((alpha-prefix, alpha-prefix-state), data), attacker-state),
      result), final-state))
  by (rule
    checked-staged-security-with-actual-alpha-prefix-query-prefix-fixed-candidate-pair-witness-gap-same-run-atI
    [OF qsupport fixed-gap])
then show ?thesis by simp
qed
qed

```

lemma *checked-staged-security-with-query-prefix-header-authenticated-candidate-opening-hit-imp-single-round-header*

assumes *hit*:

checked-staged-security-with-query-prefix-header-authenticated-candidate-opening-hit

trace-openings composition-openings i

*(Some (((((prefix, prefix-state), raw), raw-state),
((data, attacker-state), result)), final-state))*

and *trace-candidate*:

partial-trace-table-candidate trace-table

((replicate rounds []) [i := trace-openings])

and *comp-candidate*:

partial-composition-table-candidate composition-table

((replicate rounds []) [i := composition-openings])

and *trace-low*: *trace-table-low-degree trace-table*

and *comp-low*: *composition-table-low-degree maxDegree composition-table*

and *not-all*:

$\neg$ *all-queries-consistent trace-table composition-table*

(sqp-alphas prefix)

shows

checked-staged-security-with-query-prefix-single-round-header-supported-query-target-hit-at

i

*(Some (((((prefix, prefix-state), raw), raw-state),
((data, attacker-state), result)), final-state)) $\vee$*

checked-staged-security-with-query-prefix-single-round-candidate-binding-gap-witness-at

trace-openings composition-openings trace-table composition-table i

*(Some (((((prefix, prefix-state), raw), raw-state),
((data, attacker-state), result)), final-state))*

proof —

from *hit* **obtain** *comp-nonempty witness opening-target*

where *comp-nonempty*:

staged-composition-fri-roots data $\neq$ []

and *witness*:

(trace-openings, composition-openings) $\in$

query-header-supported-partial-opening-witnesses

(verifier-state-from-adversary attacker-state

(staged-proof-transcript data))

(staged-trace-root data)

(staged-trace-fri-roots data)

(staged-trace-final data)

(staged-alphas data)

(staged-degree data)

(staged-composition-fri-roots data)

(staged-composition-final data)

and *opening-target*:

index (to-nat raw) $\in$

staged-query-prefix-candidate-opening-query-target-from-prefix

((replicate rounds []) [i := trace-openings])

((replicate rounds []) [i := composition-openings])

i prefix prefix-state


```

by (rule
  checked-staged-security-with-query-prefix-header-authenticated-candidate-opening-hitE)
have opening-subset:
  staged-query-prefix-candidate-opening-query-target-from-prefix
    ((replicate rounds []) [i := trace-openings])
    ((replicate rounds []) [i := composition-openings])
  i prefix prefix-state  $\subseteq$ 
  staged-query-prefix-candidate-pair-query-target-from-prefix trace-table
  composition-table prefix prefix-state
by (rule
  staged-query-prefix-candidate-opening-query-target-from-prefix-subset-candidate-pair-from-prefix
    [OF trace-candidate comp-candidate trace-low comp-low not-all])
have pair-target-hit:
  checked-staged-query-prefix-dynamic-index-hit
    (staged-query-prefix-candidate-pair-query-target-from-prefix
      trace-table composition-table)
    (Some (((prefix, prefix-state), raw), raw-state))
using opening-target opening-subset
unfolding checked-staged-query-prefix-dynamic-index-hit-def by auto
show ?thesis
proof (cases
  (trace-table, composition-table)  $\in$ 
  query-header-supported-single-round-partial-table-candidates
    prefix-state
    (sqp-trace-root prefix)
    (sqp-trace-fri-roots prefix)
    (sqp-trace-final prefix)
    (sqp-alphas prefix)
    (sqp-degree prefix)
    (sqp-composition-fri-roots prefix)
    (sqp-composition-final prefix)
  i)
case True
have target:
  checked-staged-query-prefix-dynamic-index-hit
    (staged-query-prefix-single-round-header-supported-query-target i)
    (Some (((prefix, prefix-state), raw), raw-state))
by (rule
  checked-staged-query-prefix-candidate-pair-hit-imp-single-round-header-target-hit
    [OF True pair-target-hit])
have
  checked-staged-security-with-query-prefix-single-round-header-supported-query-target-hit-at
    i
    (Some ((((((prefix, prefix-state), raw), raw-state),
      ((data, attacker-state), result)), final-state)))
unfolding
  checked-staged-security-with-query-prefix-single-round-header-supported-query-target-hit-at-def
using target by simp
then show ?thesis by simp

```

```

next
case False
have gap:
  checked-staged-security-with-query-prefix-single-round-candidate-binding-gap-at
  trace-openings composition-openings trace-table composition-table i
  (Some (((((prefix, prefix-state), raw), raw-state),
    ((data, attacker-state), result)), final-state))
  unfolding
    checked-staged-security-with-query-prefix-single-round-candidate-binding-gap-at-def
  using hit trace-candidate comp-candidate False by simp
have side:
  checked-staged-security-with-query-prefix-single-round-candidate-binding-gap-witness-side
  trace-openings composition-openings trace-table composition-table i
  (Some (((((prefix, prefix-state), raw), raw-state),
    ((data, attacker-state), result)), final-state))
  unfolding
    checked-staged-security-with-query-prefix-single-round-candidate-binding-gap-witness-side-def
    checked-staged-security-with-query-prefix-single-round-candidate-binding-gap-side-def
  using witness trace-candidate comp-candidate False by simp
have
  checked-staged-security-with-query-prefix-single-round-candidate-binding-gap-witness-at
  trace-openings composition-openings trace-table composition-table i
  (Some (((((prefix, prefix-state), raw), raw-state),
    ((data, attacker-state), result)), final-state))
  unfolding
    checked-staged-security-with-query-prefix-single-round-candidate-binding-gap-witness-at-def
  using gap side by simp
then show ?thesis by simp
qed
qed

```

```

lemma checked-staged-security-with-query-prefix-header-authenticated-candidate-opening-hit-bound-from-target-and-f
  assumes wf: staged-budget-wellformed budgets
  and controlled: staged-adversary-controlled budgets A
  and i-bound: i < rounds
  and trace-candidate:
    partial-trace-table-candidate trace-table
    ((replicate rounds []) [i := trace-openings])
  and comp-candidate:
    partial-composition-table-candidate composition-table
    ((replicate rounds []) [i := composition-openings])
  and trace-low: trace-table-low-degree trace-table
  and comp-low: composition-table-low-degree maxDegree composition-table
  and not-all:
     $\bigwedge \text{prefix prefix-state raw raw-state data attacker-state result}$ 
    final-state.
    Some (((((prefix, prefix-state), raw), raw-state),
      ((data, attacker-state), result)), final-state)  $\in$ 
    set-dist

```

```

      (execute
        (checked-staged-security-experiment-with-query-prefix-data-state
          A i)
        adversary-initial-state)  $\implies$ 
 $\neg$  all-queries-consistent trace-table composition-table
      (sqp-alphas prefix)
and target-bound:
  wp-event
    (checked-staged-security-experiment-with-query-prefix-data-state A i)
    (checked-staged-security-with-query-prefix-single-round-header-supported-query-target-hit-at
      i)
    adversary-initial-state  $\leq T$ 
shows
  wp-event
    (checked-staged-security-experiment-with-query-prefix-data-state A i)
    (checked-staged-security-with-query-prefix-header-authenticated-candidate-opening-hit
      trace-openings composition-openings i)
    adversary-initial-state  $\leq$ 
      T +
      (staged-phase-relation-error size
        (staged-query-search-queries budgets i + 1) +
        (1::prob) / nnreal (card query-sample-space))
proof –
  let ?m =
    checked-staged-security-experiment-with-query-prefix-data-state A i
  let ?target =
    checked-staged-security-with-query-prefix-single-round-header-supported-query-target-hit-at
      i
  let ?gap =
    checked-staged-security-with-query-prefix-single-round-candidate-binding-gap-witness-at
      trace-openings composition-openings trace-table composition-table i
  have event-le:
    wp-event ?m
      (checked-staged-security-with-query-prefix-header-authenticated-candidate-opening-hit
        trace-openings composition-openings i)
      adversary-initial-state  $\leq$ 
      wp-event ?m ( $\lambda$ out. ?target out  $\vee$  ?gap out)
      adversary-initial-state
  proof (rule wp-event-mono-on-support)
  fix out
  assume support:
    out  $\in$  set-dist (execute ?m adversary-initial-state)
  and hit:
    checked-staged-security-with-query-prefix-header-authenticated-candidate-opening-hit
      trace-openings composition-openings i out
  show ?target out  $\vee$  ?gap out
  proof (cases out)
  case None
  then show ?thesis

```

```

    using hit
    unfolding
      checked-staged-security-with-query-prefix-header-authenticated-candidate-opening-hit-def
    by simp
next
case (Some packed)
then obtain prefix prefix-state raw raw-state data attacker-state
  result final-state where out-eq:
  out =
    Some (((((prefix, prefix-state), raw), raw-state),
      ((data, attacker-state), result)), final-state)
  by (cases packed) (auto split: prod.splits)
have not-all':
   $\neg$  all-queries-consistent trace-table composition-table
    (sqp-alphas prefix)
  by (rule not-all)
    (use support out-eq in simp)
show ?thesis
  unfolding out-eq
  by (rule
    checked-staged-security-with-query-prefix-header-authenticated-candidate-opening-hit-imp-single-round-header-
      [OF - trace-candidate comp-candidate trace-low comp-low not-all'])
    (use hit out-eq in simp)
qed
qed
have union-le:
  wp-event ?m ( $\lambda$ out. ?target out  $\vee$  ?gap out)
    adversary-initial-state  $\leq$ 
  wp-event ?m ?target adversary-initial-state +
  wp-event ?m ?gap adversary-initial-state
  by (rule wp-event-union-bound)
have gap-bound:
  wp-event ?m ?gap adversary-initial-state  $\leq$ 
    staged-phase-relation-error size
    (staged-query-search-queries budgets  $i + 1$ ) +
    (1::prob) / nnreal (card query-sample-space)
  by (rule
    checked-staged-security-with-query-prefix-single-round-candidate-binding-gap-witness-at-bound-from-budgets
      [OF wf controlled i-bound])
have sum-bound:
  wp-event ?m ?target adversary-initial-state +
  wp-event ?m ?gap adversary-initial-state  $\leq$ 
  T +
  (staged-phase-relation-error size
    (staged-query-search-queries budgets  $i + 1$ ) +
    (1::prob) / nnreal (card query-sample-space))
  by (rule add-mono[OF target-bound gap-bound])
show ?thesis
  by (rule order-trans[OF event-le order-trans[OF union-le sum-bound]])

```

qed

lemma *checked-staged-security-with-query-prefix-single-round-candidate-pair-witness-gap-atI*:

assumes *success*:

index (*to-nat raw*) $\in$
query-sampling-success-space trace-table composition-table
(*sqp-alphas prefix*)

and *witness*:

(*trace-openings, composition-openings*) $\in$
query-header-supported-partial-opening-witnesses
(*verifier-state-from-adversary attacker-state*
(*staged-proof-transcript data*))
(*staged-trace-root data*)
(*staged-trace-fri-roots data*)
(*staged-trace-final data*)
(*staged-alphas data*)
(*staged-degree data*)
(*staged-composition-fri-roots data*)
(*staged-composition-final data*)

and *trace-candidate*:

partial-trace-table-candidate trace-table
(*(replicate rounds []) [i := trace-openings]*)

and *comp-candidate*:

partial-composition-table-candidate composition-table
(*(replicate rounds []) [i := composition-openings]*)

and *notin*:

(*trace-table, composition-table*) $\notin$
query-header-supported-single-round-partial-table-candidates
prefix-state
(*sqp-trace-root prefix*)
(*sqp-trace-fri-roots prefix*)
(*sqp-trace-final prefix*)
(*sqp-alphas prefix*)
(*sqp-degree prefix*)
(*sqp-composition-fri-roots prefix*)
(*sqp-composition-final prefix*)
i

shows

checked-staged-security-with-query-prefix-single-round-candidate-pair-witness-gap-at
trace-openings composition-openings trace-table composition-table i
(*Some* ((((*prefix, prefix-state*), *raw*), *raw-state*),
(*(data, attacker-state, result)*), *final-state*))

proof –

have *dyn*:

checked-staged-security-with-query-prefix-dynamic-index-hit
(*staged-query-prefix-candidate-pair-query-target-from-prefix*
trace-table composition-table)
(*Some* ((((*prefix, prefix-state*), *raw*), *raw-state*),
(*(data, attacker-state, result)*), *final-state*))

```

using success
unfolding
  checked-staged-security-with-query-prefix-dynamic-index-hit-def
  checked-staged-query-prefix-dynamic-index-hit-def
  staged-query-prefix-candidate-pair-query-target-from-prefix-def
by simp
have side:
  checked-staged-security-with-query-prefix-single-round-candidate-binding-gap-witness-side
  trace-openings composition-openings trace-table composition-table i
  (Some ((((prefix, prefix-state), raw), raw-state),
    ((data, attacker-state), result)), final-state))
unfolding
  checked-staged-security-with-query-prefix-single-round-candidate-binding-gap-witness-side-def
  checked-staged-security-with-query-prefix-single-round-candidate-binding-gap-side-def
using witness trace-candidate comp-candidate notin by simp
show ?thesis
unfolding
  checked-staged-security-with-query-prefix-single-round-candidate-pair-witness-gap-at-def
using dyn side by simp
qed

lemma checked-staged-security-with-query-prefix-single-round-candidate-pair-witness-gap-at-bound-from-budgets:
assumes wf: staged-budget-wellformed budgets
and controlled: staged-adversary-controlled budgets A
and i-bound: i < rounds
and trace-low: trace-table-low-degree trace-table
and comp-low:
  composition-table-low-degree maxDegree composition-table
and not-all:
   $\bigwedge \text{prefix prefix-state.}$ 
   $\text{Some } ((\text{prefix}, \text{prefix-state}), \text{prefix-state}) \in$ 
    set-dist
    (execute (checked-staged-query-prefix-with-state A i)
      adversary-initial-state)  $\implies$ 
     $\neg$  all-queries-consistent trace-table composition-table
    (sqp-alphas prefix)
shows
  wp-event
  (checked-staged-security-experiment-with-query-prefix-data-state A i)
  (checked-staged-security-with-query-prefix-single-round-candidate-pair-witness-gap-at
    trace-openings composition-openings trace-table composition-table i)
  adversary-initial-state  $\leq$ 
  staged-phase-relation-error size
  (staged-query-search-queries budgets i + 1) + query-error-bound
proof –
have event-le:
  wp-event
  (checked-staged-security-experiment-with-query-prefix-data-state A i)
  (checked-staged-security-with-query-prefix-single-round-candidate-pair-witness-gap-at

```

```

    trace-openings composition-openings trace-table composition-table i)
  adversary-initial-state ≤
wp-event
  (checked-staged-security-experiment-with-query-prefix-data-state A i)
  (checked-staged-security-with-query-prefix-dynamic-index-hit
    (staged-query-prefix-candidate-pair-query-target-from-prefix
      trace-table composition-table))
  adversary-initial-state
by (rule wp-event-mono)
  (rule
    checked-staged-security-with-query-prefix-single-round-candidate-pair-witness-gap-at-imp-dynamic-hit)
have target-bound:
wp-event
  (checked-staged-security-experiment-with-query-prefix-data-state A i)
  (checked-staged-security-with-query-prefix-dynamic-index-hit
    (staged-query-prefix-candidate-pair-query-target-from-prefix
      trace-table composition-table))
  adversary-initial-state ≤
  staged-phase-relation-error size
    (staged-query-search-queries budgets i + 1) + query-error-bound
by (rule
  checked-staged-security-with-query-prefix-candidate-pair-target-from-prefix-hit-bound-from-budgets
    [OF wf controlled i-bound trace-low comp-low not-all])
show ?thesis
by (rule order-trans[OF event-le target-bound])
qed

lemma checked-staged-security-with-actual-alpha-prefix-query-prefix-single-round-candidate-binding-gap-witnessE-can
assumes gap:
  checked-staged-security-with-actual-alpha-prefix-query-prefix-single-round-candidate-binding-gap-witness-at
    i A
  (Some (((((alpha-prefix, alpha-prefix-state), data), attacker-state),
    result), final-state))
obtains trace-openings composition-openings trace-table composition-table
  prefix prefix-state raw raw-state where
  Some (data, attacker-state) ∈
    set-dist
      (execute (checked-staged-transcript-program A)
        adversary-initial-state)
  Some (((prefix, prefix-state), raw), raw-state) ∈
    set-dist
      (execute (checked-staged-query-prefix-receive-with-state A i)
        adversary-initial-state)
  checked-staged-security-with-query-prefix-single-round-candidate-pair-witness-gap-at
    trace-openings composition-openings trace-table composition-table i
  (Some (((((prefix, prefix-state), raw), raw-state),
    ((data, attacker-state), result)), final-state))
proof –
from gap show ?thesis

```

proof (rule
checked-staged-security-with-actual-alpha-prefix-query-prefix-single-round-candidate-binding-gap-witnessE)
fix *trace-openings composition-openings trace-table composition-table*
prefix prefix-state raw raw-state
assume *builder*:
Some (data, attacker-state) ∈
set-dist
(execute (checked-staged-transcript-program A)
adversary-initial-state)
assume *prefix-receive*:
Some (((prefix, prefix-state), raw), raw-state) ∈
set-dist
(execute (checked-staged-query-prefix-receive-with-state A i)
adversary-initial-state)
assume *witness*:
(trace-openings, composition-openings) ∈
query-header-supported-partial-opening-witnesses
(verifier-state-from-adversary attacker-state
(staged-proof-transcript data))
(staged-trace-root data)
(staged-trace-fri-roots data)
(staged-trace-final data)
(staged-alphas data)
(staged-degree data)
(staged-composition-fri-roots data)
(staged-composition-final data)
assume *trace-candidate*:
partial-trace-table-candidate trace-table
((replicate rounds []) [i := trace-openings])
assume *comp-candidate*:
partial-composition-table-candidate composition-table
((replicate rounds []) [i := composition-openings])
assume *target-hit*:
checked-staged-query-prefix-dynamic-index-hit
(staged-query-prefix-candidate-pair-query-target-from-prefix
trace-table composition-table)
(Some (((prefix, prefix-state), raw), raw-state))
assume *notin*:
(trace-table, composition-table) ∉
query-header-supported-single-round-partial-table-candidates
prefix-state
(sqp-trace-root prefix)
(sqp-trace-fri-roots prefix)
(sqp-trace-final prefix)
(sqp-alphas prefix)
(sqp-degree prefix)
(sqp-composition-fri-roots prefix)
(sqp-composition-final prefix)
i


```

have dyn:
  checked-staged-security-with-query-prefix-dynamic-index-hit
    (staged-query-prefix-candidate-pair-query-target-from-prefix
      trace-table composition-table)
    (Some (((prefix, prefix-state), raw), raw-state),
      ((data, attacker-state), result)), final-state))
  unfolding checked-staged-security-with-query-prefix-dynamic-index-hit-def
  using target-hit by simp
have side:
  checked-staged-security-with-query-prefix-single-round-candidate-binding-gap-witness-side
    trace-openings composition-openings trace-table composition-table i
    (Some (((prefix, prefix-state), raw), raw-state),
      ((data, attacker-state), result)), final-state))
  unfolding
    checked-staged-security-with-query-prefix-single-round-candidate-binding-gap-witness-side-def
    checked-staged-security-with-query-prefix-single-round-candidate-binding-gap-side-def
  using witness trace-candidate comp-candidate notin by simp
have fixed-gap:
  checked-staged-security-with-query-prefix-single-round-candidate-pair-witness-gap-at
    trace-openings composition-openings trace-table composition-table i
    (Some (((prefix, prefix-state), raw), raw-state),
      ((data, attacker-state), result)), final-state))
  unfolding
    checked-staged-security-with-query-prefix-single-round-candidate-pair-witness-gap-at-def
  using dyn side by simp
show ?thesis
  by (rule that[OF builder prefix-receive fixed-gap])
qed
qed

lemma checked-staged-security-with-query-prefix-candidate-pair-from-prefix-hit-imp-single-round-header-target-or-fix
assumes pair-hit:
  checked-staged-security-with-query-prefix-candidate-pair-from-prefix-hit
    trace-table composition-table
    (Some (((prefix, prefix-state), raw), raw-state),
      ((data, attacker-state), result)), final-state))
and witness:
  (trace-openings, composition-openings) ∈
    query-header-supported-partial-opening-witnesses
    (verifier-state-from-adversary attacker-state
      (staged-proof-transcript data))
    (staged-trace-root data)
    (staged-trace-fri-roots data)
    (staged-trace-final data)
    (staged-alphas data)
    (staged-degree data)
    (staged-composition-fri-roots data)
    (staged-composition-final data)
and trace-candidate:

```

```

    partial-trace-table-candidate trace-table
      ((replicate rounds []) [i := trace-openings])
  and comp-candidate:
    partial-composition-table-candidate composition-table
      ((replicate rounds []) [i := composition-openings])
  shows
    checked-staged-security-with-query-prefix-single-round-header-supported-query-target-hit-at
      i
      (Some (((((prefix, prefix-state), raw), raw-state),
        ((data, attacker-state), result)), final-state)) ∨
    checked-staged-security-with-query-prefix-single-round-candidate-pair-witness-gap-at
      trace-openings composition-openings trace-table composition-table i
      (Some (((((prefix, prefix-state), raw), raw-state),
        ((data, attacker-state), result)), final-state))
  proof -
  let ?out =
    Some (((((prefix, prefix-state), raw), raw-state),
      ((data, attacker-state), result)), final-state)
  have raw-hit:
    checked-staged-query-prefix-dynamic-index-hit
      (staged-query-prefix-candidate-pair-query-target-from-prefix
        trace-table composition-table)
      (Some (((prefix, prefix-state), raw), raw-state))
  using pair-hit
  unfolding
    checked-staged-security-with-query-prefix-candidate-pair-from-prefix-hit-def
    checked-staged-security-with-query-prefix-dynamic-index-hit-def
  by simp
  show ?thesis
  proof (cases
    (trace-table, composition-table) ∈
    query-header-supported-single-round-partial-table-candidates
      prefix-state
      (sqp-trace-root prefix)
      (sqp-trace-fri-roots prefix)
      (sqp-trace-final prefix)
      (sqp-alphas prefix)
      (sqp-degree prefix)
      (sqp-composition-fri-roots prefix)
      (sqp-composition-final prefix)
    i)
  case True
  have target:
    checked-staged-query-prefix-dynamic-index-hit
      (staged-query-prefix-single-round-header-supported-query-target i)
      (Some (((prefix, prefix-state), raw), raw-state))
  by (rule
    checked-staged-query-prefix-candidate-pair-hit-imp-single-round-header-target-hit
    [OF True raw-hit])

```

```

have
  checked-staged-security-with-query-prefix-single-round-header-supported-query-target-hit-at
    i ?out
unfolding
  checked-staged-security-with-query-prefix-single-round-header-supported-query-target-hit-at-def
using target by simp
then show ?thesis by simp
next
case False
have success:
  index (to-nat raw) ∈
    query-sampling-success-space trace-table composition-table
      (sqp-alphas prefix)
using raw-hit
unfolding
  checked-staged-query-prefix-dynamic-index-hit-def
  staged-query-prefix-candidate-pair-query-target-from-prefix-def
by simp
have
  checked-staged-security-with-query-prefix-single-round-candidate-pair-witness-gap-at
    trace-openings composition-openings trace-table composition-table i
    ?out
by (rule
    checked-staged-security-with-query-prefix-single-round-candidate-pair-witness-gap-atI
    [OF success witness trace-candidate comp-candidate False])
then show ?thesis by simp
qed
qed

lemma checked-staged-security-with-query-prefix-authenticated-opening-hit-imp-single-round-header-target-or-fixed-w
assumes wf: staged-budget-wellformed budgets
and controlled: staged-adversary-controlled budgets A
and i-bound: i < rounds
and support:
  Some (((((prefix, prefix-state), raw), raw-state),
    ((data, attacker-state), result)), final-state) ∈
    set-dist
    (execute
      (checked-staged-security-experiment-with-query-prefix-data-state A i)
      adversary-initial-state)
and hit:
  checked-staged-security-with-query-prefix-authenticated-opening-hit
    trace-openings composition-openings i
    (Some (((((prefix, prefix-state), raw), raw-state),
      ((data, attacker-state), result)), final-state))
and witness:
  (trace-openings, composition-openings) ∈
    query-header-supported-partial-opening-witnesses
    (verifier-state-from-adversary attacker-state

```

```

    (staged-proof-transcript data))
  (staged-trace-root data)
  (staged-trace-fri-roots data)
  (staged-trace-final data)
  (staged-alphas data)
  (staged-degree data)
  (staged-composition-fri-roots data)
  (staged-composition-final data)
and trace-candidate:
  partial-trace-table-candidate trace-table
  ((replicate rounds []) [i := trace-openings])
and comp-candidate:
  partial-composition-table-candidate composition-table
  ((replicate rounds []) [i := composition-openings])
and trace-low: trace-table-low-degree trace-table
and comp-low: composition-table-low-degree maxDegree composition-table
and not-all:
  ¬ all-queries-consistent trace-table composition-table
  (sqp-alphas prefix)
shows
  checked-staged-security-with-query-prefix-single-round-header-supported-query-target-hit-at
  i
  (Some (((((prefix, prefix-state), raw), raw-state),
    ((data, attacker-state), result)), final-state)) ∨
  checked-staged-security-with-query-prefix-single-round-candidate-pair-witness-gap-at
  trace-openings composition-openings trace-table composition-table i
  (Some (((((prefix, prefix-state), raw), raw-state),
    ((data, attacker-state), result)), final-state)) ∨
  checked-staged-security-with-query-prefix-current-trace-path-output-hit-at
  i
  (Some (((((prefix, prefix-state), raw), raw-state),
    ((data, attacker-state), result)), final-state)) ∨
  checked-staged-security-with-query-prefix-current-composition-path-output-hit-at
  i
  (Some (((((prefix, prefix-state), raw), raw-state),
    ((data, attacker-state), result)), final-state))
proof —
let ?out =
  Some (((((prefix, prefix-state), raw), raw-state),
    ((data, attacker-state), result)), final-state)
have split:
  checked-staged-security-with-query-prefix-candidate-pair-from-prefix-hit
  trace-table composition-table ?out ∨
  checked-staged-security-with-query-prefix-current-trace-path-output-hit-at
  i ?out ∨
  checked-staged-security-with-query-prefix-current-composition-path-output-hit-at
  i ?out
by (rule
  checked-staged-security-with-query-prefix-authenticated-opening-hit-imp-candidate-pair-or-structured-path-on-sup

```

```

    [OF wf controlled i-bound support hit trace-candidate
      comp-candidate trace-low comp-low not-all])
then show ?thesis
proof
  assume pair:
    checked-staged-security-with-query-prefix-candidate-pair-from-prefix-hit
    trace-table composition-table ?out
  from
    checked-staged-security-with-query-prefix-candidate-pair-from-prefix-hit-imp-single-round-header-target-or-fixed-witness
    [OF pair witness trace-candidate comp-candidate]
  show ?thesis
    by blast
next
  assume
    checked-staged-security-with-query-prefix-current-trace-path-output-hit-at
    i ?out ∨
    checked-staged-security-with-query-prefix-current-composition-path-output-hit-at
    i ?out
  then show ?thesis by blast
qed
qed

lemma checked-staged-security-with-query-prefix-authenticated-opening-hit-bound-from-single-round-target-fixed-witness
fixes H G T C :: prob
assumes wf: staged-budget-wellformed budgets
and controlled: staged-adversary-controlled budgets A
and i-bound: i < rounds
and witness:
   $\bigwedge \text{prefix prefix-state raw raw-state data attacker-state result}$ 
  final-state.
  Some (((((prefix, prefix-state), raw), raw-state),
    ((data, attacker-state), result)), final-state) ∈
  set-dist
  (execute
    (checked-staged-security-experiment-with-query-prefix-data-state
      A i)
    adversary-initial-state) ⇒
  (trace-openings, composition-openings) ∈
  query-header-supported-partial-opening-witnesses
  (verifier-state-from-adversary attacker-state
    (staged-proof-transcript data))
  (staged-trace-root data)
  (staged-trace-fri-roots data)
  (staged-trace-final data)
  (staged-alphas data)
  (staged-degree data)
  (staged-composition-fri-roots data)
  (staged-composition-final data)
and trace-candidate:

```

partial-trace-table-candidate trace-table
 ((replicate rounds []) [i := trace-openings])
and *comp-candidate:*
partial-composition-table-candidate composition-table
 ((replicate rounds []) [i := composition-openings])
and *trace-low: trace-table-low-degree trace-table*
and *comp-low: composition-table-low-degree maxDegree composition-table*
and *not-all:*
 $\bigwedge \text{prefix prefix-state raw raw-state data attacker-state result}$
 final-state.
 $\text{Some } (((((\text{prefix}, \text{prefix-state}), \text{raw}), \text{raw-state}),$
 $((\text{data}, \text{attacker-state}), \text{result})), \text{final-state}) \in$
 set-dist
 $(\text{execute}$
 $(\text{checked-staged-security-experiment-with-query-prefix-data-state}$
 $A \ i)$
 $\text{adversary-initial-state}) \implies$
 $\neg \text{all-queries-consistent trace-table composition-table}$
 $(\text{sqp-alphas prefix})$
and *target-bound:*
 wp-event
 $(\text{checked-staged-security-experiment-with-query-prefix-data-state } A \ i)$
 $(\text{checked-staged-security-with-query-prefix-single-round-header-supported-query-target-hit-at}$
 $i)$
 $\text{adversary-initial-state} \leq H$
and *fixed-gap-bound:*
 wp-event
 $(\text{checked-staged-security-experiment-with-query-prefix-data-state } A \ i)$
 $(\text{checked-staged-security-with-query-prefix-single-round-candidate-pair-witness-gap-at}$
 $\text{trace-openings composition-openings trace-table composition-table } i)$
 $\text{adversary-initial-state} \leq G$
and *trace-path-bound:*
 wp-event
 $(\text{checked-staged-security-experiment-with-query-prefix-data-state } A \ i)$
 $(\text{checked-staged-security-with-query-prefix-current-trace-path-output-hit-at}$
 $i)$
 $\text{adversary-initial-state} \leq T$
and *composition-path-bound:*
 wp-event
 $(\text{checked-staged-security-experiment-with-query-prefix-data-state } A \ i)$
 $(\text{checked-staged-security-with-query-prefix-current-composition-path-output-hit-at}$
 $i)$
 $\text{adversary-initial-state} \leq C$
shows
 wp-event
 $(\text{checked-staged-security-experiment-with-query-prefix-data-state } A \ i)$
 $(\text{checked-staged-security-with-query-prefix-authenticated-opening-hit}$
 $\text{trace-openings composition-openings } i)$
 $\text{adversary-initial-state} \leq H + G + T + C$

```

proof –
  let ?M =
    checked-staged-security-experiment-with-query-prefix-data-state A i
  let ?Target =
    checked-staged-security-with-query-prefix-single-round-header-supported-query-target-hit-at
    i
  let ?Gap =
    checked-staged-security-with-query-prefix-single-round-candidate-pair-witness-gap-at
    trace-openings composition-openings trace-table composition-table i
  let ?Trace =
    checked-staged-security-with-query-prefix-current-trace-path-output-hit-at
    i
  let ?Comp =
    checked-staged-security-with-query-prefix-current-composition-path-output-hit-at
    i
  have event-le:
    wp-event ?M
    (checked-staged-security-with-query-prefix-authenticated-opening-hit
     trace-openings composition-openings i)
    adversary-initial-state ≤
    wp-event ?M
    (λout. ?Target out ∨ ?Gap out ∨ ?Trace out ∨ ?Comp out)
    adversary-initial-state
  proof (rule wp-event-mono-on-support)
  fix out
  assume support:
    out ∈ set-dist (execute ?M adversary-initial-state)
  and hit:
    checked-staged-security-with-query-prefix-authenticated-opening-hit
    trace-openings composition-openings i out
  show ?Target out ∨ ?Gap out ∨ ?Trace out ∨ ?Comp out
  proof (cases out)
  case None
  then show ?thesis
    using hit
    unfolding
      checked-staged-security-with-query-prefix-authenticated-opening-hit-def
    by simp
  next
  case (Some packed)
  then obtain prefix prefix-state raw raw-state data attacker-state result
    final-state where out-eq:
    out =
      Some (((((prefix, prefix-state), raw), raw-state),
        ((data, attacker-state), result)), final-state)
    by (cases packed) (auto split: prod.splits)
  have support-some:
    Some (((((prefix, prefix-state), raw), raw-state),
      ((data, attacker-state), result)), final-state) ∈

```

```

    set-dist (execute ?M adversary-initial-state)
  using support out-eq by simp
show ?thesis
  unfolding out-eq
  by (rule
    checked-staged-security-with-query-prefix-authenticated-opening-hit-imp-single-round-header-target-or-fixed-wit
      [OF wf controlled i-bound support-some - witness[OF support-some]
        trace-candidate comp-candidate trace-low comp-low
        not-all[OF support-some]])
    (use hit out-eq in simp)
qed
qed
have union-bound:
  wp-event ?M
    (λout. ?Target out ∨ ?Gap out ∨ ?Trace out ∨ ?Comp out)
    adversary-initial-state ≤
  wp-event ?M ?Target adversary-initial-state +
  wp-event ?M ?Gap adversary-initial-state +
  wp-event ?M ?Trace adversary-initial-state +
  wp-event ?M ?Comp adversary-initial-state
  by (rule wp-event-union-bound4)
have component-bound:
  wp-event ?M ?Target adversary-initial-state +
  wp-event ?M ?Gap adversary-initial-state +
  wp-event ?M ?Trace adversary-initial-state +
  wp-event ?M ?Comp adversary-initial-state ≤
  H + G + T + C
  by (intro add-mono target-bound fixed-gap-bound trace-path-bound
    composition-path-bound)
show ?thesis
  by (rule order-trans[OF event-le order-trans[OF union-bound
    component-bound]])
qed

lemma checked-staged-security-with-query-prefix-authenticated-opening-hit-bound-from-single-round-target-budgets-and-
  fixes H T C :: prob
  assumes wf: staged-budget-wellformed budgets
  and controlled: staged-adversary-controlled budgets A
  and i-bound: i < rounds
  and witness:
    ∧ prefix prefix-state raw raw-state data attacker-state result
      final-state.
    Some (((((prefix, prefix-state), raw), raw-state),
      ((data, attacker-state), result)), final-state) ∈
    set-dist
      (execute
        (checked-staged-security-experiment-with-query-prefix-data-state
          A i)
        adversary-initial-state) ⇒

```


(trace-openings, composition-openings) ∈
query-header-supported-partial-opening-witnesses
(verifier-state-from-adversary attacker-state
(staged-proof-transcript data))
(staged-trace-root data)
(staged-trace-fri-roots data)
(staged-trace-final data)
(staged-alphas data)
(staged-degree data)
(staged-composition-fri-roots data)
(staged-composition-final data)
and *trace-candidate:*
partial-trace-table-candidate trace-table
((replicate rounds []) [i := trace-openings])
and *comp-candidate:*
partial-composition-table-candidate composition-table
((replicate rounds []) [i := composition-openings])
and *trace-low: trace-table-low-degree trace-table*
and *comp-low: composition-table-low-degree maxDegree composition-table*
and *not-all:*
 $\bigwedge$ *prefix prefix-state.*
Some ((prefix, prefix-state), prefix-state) ∈
set-dist
(execute (checked-staged-query-prefix-with-state A i)
adversary-initial-state) ⇒
 $\neg$ *all-queries-consistent trace-table composition-table*
(sqp-alphas prefix)
and *target-bound:*
wp-event
(checked-staged-security-experiment-with-query-prefix-data-state A i)
(checked-staged-security-with-query-prefix-single-round-header-supported-query-target-hit-at
i)
adversary-initial-state ≤ H
and *trace-path-bound:*
wp-event
(checked-staged-security-experiment-with-query-prefix-data-state A i)
(checked-staged-security-with-query-prefix-current-trace-path-output-hit-at
i)
adversary-initial-state ≤ T
and *composition-path-bound:*
wp-event
(checked-staged-security-experiment-with-query-prefix-data-state A i)
(checked-staged-security-with-query-prefix-current-composition-path-output-hit-at
i)
adversary-initial-state ≤ C
shows
wp-event
(checked-staged-security-experiment-with-query-prefix-data-state A i)
(checked-staged-security-with-query-prefix-authenticated-opening-hit

```

    trace-openings composition-openings i)
  adversary-initial-state ≤
  H +
  (staged-phase-relation-error size
    (staged-query-search-queries budgets i + 1) + query-error-bound) +
  T + C
proof –
have fixed-gap-bound:
  wp-event
  (checked-staged-security-experiment-with-query-prefix-data-state A i)
  (checked-staged-security-with-query-prefix-single-round-candidate-pair-witness-gap-at
    trace-openings composition-openings trace-table composition-table i)
  adversary-initial-state ≤
  staged-phase-relation-error size
  (staged-query-search-queries budgets i + 1) + query-error-bound
by (rule
  checked-staged-security-with-query-prefix-single-round-candidate-pair-witness-gap-at-bound-from-budgets
  [OF wf controlled i-bound trace-low comp-low])
  (use not-all in blast)
have not-all-on-data-support:
  ∧ prefix prefix-state raw raw-state data attacker-state result
  final-state.
  Some (((((prefix, prefix-state), raw), raw-state),
    ((data, attacker-state), result)), final-state) ∈
  set-dist
  (execute
    (checked-staged-security-experiment-with-query-prefix-data-state
      A i)
    adversary-initial-state) ⇒
  ¬ all-queries-consistent trace-table composition-table
  (sqp-alphas prefix)
proof –
fix prefix prefix-state raw raw-state data attacker-state result
  final-state
assume support:
  Some (((((prefix, prefix-state), raw), raw-state),
    ((data, attacker-state), result)), final-state) ∈
  set-dist
  (execute
    (checked-staged-security-experiment-with-query-prefix-data-state
      A i)
    adversary-initial-state)
have receive-support:
  Some (((prefix, prefix-state), raw), raw-state) ∈
  set-dist
  (execute (checked-staged-query-prefix-receive-with-state A i)
    adversary-initial-state)
using support
unfolding checked-staged-security-experiment-with-query-prefix-data-state-def

```

```

    by (auto elim!: set-dist-bindE split: prod.splits)
  have prefix-support:
    Some ((prefix, prefix-state), prefix-state) ∈
      set-dist
        (execute (checked-staged-query-prefix-with-state A i)
          adversary-initial-state)
    by (rule checked-staged-query-prefix-receive-with-state-prefix-support
      [OF receive-support])
  show ¬ all-queries-consistent trace-table composition-table
    (sqp-alphas prefix)
    by (rule not-all[OF prefix-support])
qed
show ?thesis
by (rule
  checked-staged-security-with-query-prefix-authenticated-opening-hit-bound-from-single-round-target-fixed-witness-
  [OF wf controlled i-bound witness trace-candidate comp-candidate
    trace-low comp-low not-all-on-data-support target-bound fixed-gap-bound
    trace-path-bound composition-path-bound])
qed
end
end
end

```

```

theory Staged-Security-Experiment-Composition-Query-Aligned-Witness-Bridge
imports
  Staged-Security-Experiment-Composition-Query-Current-Alignment
  Staged-Security-Experiment-Composition-Query-Single-Round-Witness-Bridge
begin

```

Same-run bridge from actual-alpha support to the fixed query-prefix candidate-pair witness-gap event.

This layer avoids the broad diagnostic event: the sampled query index, prefix state, verifier output, and witness side facts all live on the same query-prefix data-state output.

```

context soundness
begin

```

```

lemma checked-staged-security-with-query-prefix-data-state-accepted-shape-query-index:
assumes wf: staged-budget-wellformed budgets
and controlled: staged-adversary-controlled budgets A
and i-bound:  $i < \text{rounds}$ 
and support:
  Some (((((prefix, prefix-state), raw), raw-state),
    ((data, attacker-state), result)), final-state) ∈
    set-dist
      (execute
        (checked-staged-security-experiment-with-query-prefix-data-state A i)
        adversary-initial-state)

```

```

and shape:
  accepted-transcript-shape
    (verifier-state-from-adversary attacker-state
      (staged-proof-transcript data))
    (Some (result, final-state)) as query-idxs
shows index (to-nat raw) = query-idxs ! i
proof –
let ?s =
  verifier-state-from-adversary attacker-state
    (staged-proof-transcript data)
have data-support:
  Some (((data, attacker-state), result), final-state) ∈
    set-dist
    (execute (checked-staged-security-experiment-with-data-state A)
      adversary-initial-state)
by (rule
  checked-staged-security-experiment-with-query-prefix-data-state-imp-data-state-support
    [OF i-bound support])
from checked-staged-security-experiment-with-data-state-outcomeE
  [OF data-support]
obtain verifier where verifier:
  Some (result, final-state) ∈ set-dist (execute verify-monad ?s)
by blast
from checked-staged-security-with-data-state-accepted-shape-query-chunks
  [OF wf controlled data-support shape]
obtain raw-idxs where len-raw: length raw-idxs = rounds
and query-idxs-eq:
  query-idxs = map (λraw. index (to-nat raw)) raw-idxs
and lookup-final:
   $\forall j < \text{rounds}.$ 
  fmlookup (HashMap final-state)
    (QueryIndexChallenge j
      (state-after-query-chunks
        (staged-query-start-hash data) (staged-query-chunks data) j)) =
    Some (raw-idxs ! j)
by blast
from support obtain t where prefix-receive:
  Some (((prefix, prefix-state), raw), t) ∈
    set-dist
    (execute (checked-staged-query-prefix-receive-with-state A i)
      adversary-initial-state)
and t-eq: t = raw-state
and cont:
  Some (((data, attacker-state), result), final-state) ∈
    set-dist
    (execute
      (checked-staged-after-query-prefix-receive-with-verifier A i
        ((prefix, prefix-state), raw))
        raw-state)

```

unfolding *checked-staged-security-experiment-with-query-prefix-data-state-def*
by (*auto elim!*: *set-dist-bindE split: prod.splits*)
have *data-out*:
Some (data, attacker-state) ∈
set-dist
(execute
(checked-staged-after-query-prefix-receive A i
((prefix, prefix-state), raw))
raw-state)
by (*rule checked-staged-after-query-prefix-receive-with-verifier-outcome(1)*
[OF cont])
have *prefix-support*:
Some ((prefix, prefix-state), prefix-state) ∈
set-dist
(execute (checked-staged-query-prefix-with-state A i)
adversary-initial-state)
by (*rule checked-staged-query-prefix-receive-with-state-prefix-support*)
(use prefix-receive t-eq in simp)
have *recv*:
Some (raw, raw-state) ∈
set-dist (execute receive-query-index-challenge prefix-state)
by (*rule checked-staged-query-prefix-receive-with-state-receive-support*)
(use prefix-receive t-eq in simp)
from *checked-staged-query-prefix-with-state-alignment*
[OF wf controlled i-bound prefix-support]
have *prefix-counter*: *PQueryCounter prefix-state = i*
and *prefix-state-hash*:
PState prefix-state =
state-after-query-chunks
(staged-query-prefix-start-hash prefix)
(sqp-query-chunks prefix) i
and *prefix-len*: *length (sqp-query-chunks prefix) = i*
by *blast+*
from *data-out* **obtain** *chunk chunk-state record-state suffix-chunks* **where**
chunk-out:
Some (chunk, chunk-state) ∈
set-dist (execute (query-opening-stage A i raw) raw-state)
and *record-out*:
Some ((), record-state) ∈
set-dist (execute (record-staged-messages chunk) chunk-state)
and *suffix-out*:
Some (suffix-chunks, attacker-state) ∈
set-dist
(execute
(checked-staged-query-program A
(sqp-trace-fri-roots prefix)
(sqp-composition-fri-roots prefix)
(Suc i) (rounds - Suc i))
record-state)

```

and data-eq:
  data =
    (staged-trace-root = sqp-trace-root prefix,
     staged-trace-fri-roots = sqp-trace-fri-roots prefix,
     staged-trace-fri-challenges = sqp-trace-fri-challenges prefix,
     staged-trace-final = sqp-trace-final prefix,
     staged-alphas = sqp-alphas prefix,
     staged-degree = sqp-degree prefix,
     staged-composition-fri-roots = sqp-composition-fri-roots prefix,
     staged-composition-fri-challenges =
       sqp-composition-fri-challenges prefix,
     staged-composition-final = sqp-composition-final prefix,
     staged-query-chunks =
       sqp-query-chunks prefix @ chunk # suffix-chunks)
  unfolding checked-staged-after-query-prefix-receive-def Let-def
  by (auto elim!: set-dist-bindE simp: assert-def throw-no-outcome
      split: if-splits)
have match: staged-query-prefix-matches-data i prefix data
  unfolding staged-query-prefix-matches-data-def data-eq
  using prefix-len by simp
have start-hash:
  staged-query-start-hash data = staged-query-prefix-start-hash prefix
  by (rule staged-query-start-hash-of-prefix-completion[OF match])
have prefix-key-state:
  PState prefix-state =
    state-after-query-chunks
    (staged-query-start-hash data) (staged-query-chunks data) i
  using prefix-state-hash prefix-len start-hash
  unfolding data-eq
  by (simp add: state-after-query-chunks-append-prefix)
have receive-lookup:
  fmlookup (HashMap raw-state)
    (QueryIndexChallenge i
     (state-after-query-chunks
      (staged-query-start-hash data) (staged-query-chunks data) i)) =
    Some raw
  using receive-query-index-challenge-outcome[OF recv]
  prefix-counter prefix-key-state
  by simp
have raw-ext-attacker: raw-state ≤ attacker-state
  by (rule checked-staged-after-query-prefix-receive-extends
      [OF wf controlled i-bound data-out])
have attacker-ext-s: attacker-state ≤ ?s
proof –
  have attacker-state ≤ attacker-state
  unfolding less-eq-hash-ext-def less-eq-fmap-def by simp
  then show ?thesis
  by (rule hash-extends-verifier-state-from-adversary-right)
qed

```

```

have s-ext-final: ?s ≤ final-state
  by (rule verify-monad-hash-extends[OF verifier])
have raw-ext-final: raw-state ≤ final-state
  by (rule hash-ext-trans[OF raw-ext-attacker])
    (rule hash-ext-trans[OF attacker-ext-s s-ext-final])
have lookup-final-raw:
  fmlookup (HashMap final-state)
    (QueryIndexChallenge i
      (state-after-query-chunks
        (staged-query-start-hash data) (staged-query-chunks data) i)) =
    Some raw
  by (rule hash-extension-lookup[OF receive-lookup raw-ext-final])
have raw-eq: raw-idxs ! i = raw
  using lookup-final[rule-format, OF i-bound] lookup-final-raw by simp
show ?thesis
  using query-idxs-eq len-raw i-bound raw-eq by simp
qed

```

lemma *checked-staged-security-with-actual-alpha-prefix-aligned-transcript-query-bad-imp-header-authenticated-candidate*

```

assumes wf: staged-budget-wellformed budgets
and controlled: staged-adversary-controlled budgets A
and support:
  Some ((((alpha-prefix, alpha-prefix-state), data), attacker-state),
    result), final-state) ∈
  set-dist
    (execute
      (checked-staged-security-experiment-with-actual-alpha-prefix-data-state
        A)
      adversary-initial-state)
and bad:
  checked-staged-security-with-actual-alpha-prefix-verifier-event
    query-bad-with-aligned-transcript-partial-candidates
    (Some ((((alpha-prefix, alpha-prefix-state), data),
      attacker-state), result), final-state))

```

shows

```

checked-staged-security-with-actual-alpha-prefix-header-authenticated-candidate-opening-query-hit-from-prefix
  A
  (Some ((((alpha-prefix, alpha-prefix-state), data), attacker-state),
    result), final-state))

```

proof –

```

let ?s =
  verifier-state-from-adversary attacker-state
    (staged-proof-transcript data)
have data-support:
  Some (((data, attacker-state), result), final-state) ∈
  set-dist
    (execute (checked-staged-security-experiment-with-data-state A)
      adversary-initial-state)
by (rule

```

```

    checked-staged-security-with-actual-alpha-prefix-support-imp-data-state-support
    [OF support])
from checked-staged-security-experiment-with-data-state-outcomeE
    [OF data-support]
obtain builder verifier where builder:
    Some (data, attacker-state) ∈
    set-dist
    (execute (checked-staged-transcript-program A)
    adversary-initial-state)
and verifier:
    Some (result, final-state) ∈ set-dist (execute verify-monad ?s)
by blast
have staged-header:
    verifier-header-transcript ?s
    (staged-trace-root data)
    (staged-trace-fri-roots data)
    (staged-trace-final data)
    (staged-alphas data)
    (staged-degree data)
    (staged-composition-fri-roots data)
    (staged-composition-final data)
    (List.concat (staged-query-chunks data))
by (rule checked-staged-transcript-program-outcome-header-transcript
    [OF wf controlled builder])
from bad obtain fr f-fri-roots f-final as dg composition-fri-roots
    final query-idxs trace-openings composition-openings trace-table
    composition-table where partial:
    accepted-with-partial-initial-openings-aligned-transcript-consistent
    ?s (Some (result, final-state)) fr f-fri-roots f-final as dg
    composition-fri-roots final query-idxs trace-openings
    composition-openings
unfolding checked-staged-security-with-actual-alpha-prefix-verifier-event-def
    query-bad-with-aligned-transcript-partial-candidates-def
by auto
have aligned:
    accepted-with-partial-initial-openings-aligned-consistent ?s
    (Some (result, final-state)) fr f-fri-roots f-final as dg
    composition-fri-roots final query-idxs trace-openings
    composition-openings
by (rule
    accepted-with-partial-initial-openings-aligned-transcript-consistent-imp-aligned
    [OF partial])
have shape:
    accepted-transcript-shape ?s (Some (result, final-state)) as
    query-idxs
by (rule
    accepted-with-partial-initial-openings-aligned-transcript-consistent-imp-shape
    [OF partial])
from checked-staged-security-with-data-state-accepted-shape-query-chunks

```



```

    [OF wf controlled data-support shape]
obtain raw-idxs where as-eq: as = staged-alphas data
and len-raw: length raw-idxs = rounds
and query-idxs-eq:
    query-idxs = map (λraw. index (to-nat raw)) raw-idxs
and lookup:
    ∀ i < rounds.
        fmllookup (HashMap final-state)
            (QueryIndexChallenge i
              (state-after-query-chunks
                (staged-query-start-hash data)
                (staged-query-chunks data) i)) =
            Some (raw-idxs ! i)
by blast
have aligned0:
    accepted-with-partial-initial-openings-aligned ?s
    (Some (result, final-state)) fr f-fri-roots f-final as dg
    composition-fri-roots final query-idxs trace-openings
    composition-openings
by (rule accepted-with-partial-initial-openings-aligned-consistent-imp-aligned
    [OF aligned])
from accepted-with-partial-initial-openings-aligned-shapes[OF aligned0]
obtain rest where header:
    verifier-header-transcript ?s fr f-fri-roots f-final as dg
    composition-fri-roots final rest
by blast
have comp-nonempty: composition-fri-roots ≠ []
using accepted-with-partial-initial-openings-aligned-shapes(1)
    [OF aligned0] .
have trace-partial:
    accepted-with-partial-trace-openings ?s
    (Some (result, final-state)) fr query-idxs trace-openings
using accepted-with-partial-initial-openings-aligned-shapes(3)
    [OF aligned0] .
have comp-partial:
    accepted-with-partial-composition-openings ?s
    (Some (result, final-state)) (hd composition-fri-roots) query-idxs
    composition-openings
using accepted-with-partial-initial-openings-aligned-shapes(4)
    [OF aligned0] .
have header-eq:
    fr = staged-trace-root data ∧
    f-fri-roots = staged-trace-fri-roots data ∧
    f-final = staged-trace-final data ∧
    as = staged-alphas data ∧
    dg = staged-degree data ∧
    composition-fri-roots = staged-composition-fri-roots data ∧
    final = staged-composition-final data ∧
    rest = List.concat (staged-query-chunks data)

```

```

using verifier-header-transcript-unique[OF header staged-header]
by simp
let ?i = 0
have i-bound: ?i < rounds
  using rounds-positive by simp
have round-consistent:
  partial-query-round-consistent trace-openings composition-openings as
    ?i (query-idxs ! ?i)
  using aligned i-bound
  unfolding accepted-with-partial-initial-openings-aligned-consistent-def
  by simp
have opening-consistent:
  partial-query-openings-consistent (trace-openings ! ?i)
    (composition-openings ! ?i) (staged-alphas data)
    (index (to-nat (raw-idxs ! ?i)))
  using round-consistent query-idxs-eq len-raw i-bound header-eq
  unfolding partial-query-round-consistent-def
    partial-query-openings-consistent-def
  by simp
have trace-table-auth:
  partial-authenticated-table fr (scale * clength)
    (trace-openings ! ?i) final-state
  using trace-partial i-bound
  unfolding accepted-with-partial-trace-openings-def by blast
have comp-table-auth:
  partial-authenticated-table (hd composition-fri-roots)
    (scale * clength) (composition-openings ! ?i) final-state
  using comp-partial i-bound
  unfolding accepted-with-partial-composition-openings-def by blast
have witness-header:
  (trace-openings ! ?i, composition-openings ! ?i) ∈
    query-header-supported-partial-opening-witnesses ?s fr f-fri-roots
    f-final as dg composition-fri-roots final
  unfolding query-header-supported-partial-opening-witnesses-def
  using comp-nonempty verifier trace-table-auth comp-table-auth header
  by blast
have witness:
  (trace-openings ! ?i, composition-openings ! ?i) ∈
    query-header-supported-partial-opening-witnesses ?s
      (staged-trace-root data)
      (staged-trace-fri-roots data)
      (staged-trace-final data)
      (staged-alphas data)
      (staged-degree data)
      (staged-composition-fri-roots data)
      (staged-composition-final data)
  using witness-header header-eq by simp
have lookup0:
  fmlookup (HashMap final-state)

```

```

    (QueryIndexChallenge ?i
      (state-after-query-chunks
        (staged-query-start-hash data)
        (staged-query-chunks data) ?i)) =
      Some (raw-idxs ! ?i)
  using lookup i-bound by simp
from checked-staged-transcript-program-query-prefix-receive-final-lookup-support-with-alphas
  [OF wf controlled i-bound builder verifier lookup0]
obtain prefix prefix-state raw-state where prefix-receive:
  Some (((prefix, prefix-state), raw-idxs ! ?i), raw-state) ∈
    set-dist
      (execute (checked-staged-query-prefix-receive-with-state A ?i)
        adversary-initial-state)
and alphas-eq: sqp-alphas prefix = staged-alphas data
by blast
have component:
  checked-staged-security-with-actual-alpha-prefix-candidate-opening-query-hit-from-prefix A
  ((replicate rounds []) [?i := trace-openings ! ?i])
  ((replicate rounds []) [?i := composition-openings ! ?i])
  ?i
  (Some (((((alpha-prefix, alpha-prefix-state), data), attacker-state),
    result), final-state))
by (rule
  checked-staged-security-with-actual-alpha-prefix-candidate-opening-query-hit-from-prefixI-from-prefix-receive
  [OF i-bound builder prefix-receive alphas-eq opening-consistent])
show ?thesis
by (rule
  checked-staged-security-with-actual-alpha-prefix-header-authenticated-candidate-opening-query-hit-from-prefixI
  [OF witness component])
qed

lemma checked-staged-security-with-actual-alpha-prefix-aligned-transcript-query-bad-imp-single-round-header-target-c
assumes wf: staged-budget-wellformed budgets
and controlled: staged-adversary-controlled budgets A
and support:
  Some (((((alpha-prefix, alpha-prefix-state), data), attacker-state),
    result), final-state) ∈
    set-dist
      (execute
        (checked-staged-security-experiment-with-actual-alpha-prefix-data-state
          A)
        adversary-initial-state)
and bad:
  checked-staged-security-with-actual-alpha-prefix-verifier-event
  query-bad-with-aligned-transcript-partial-candidates
  (Some (((((alpha-prefix, alpha-prefix-state), data),
    attacker-state), result), final-state))
shows
  checked-staged-security-with-actual-alpha-prefix-single-round-header-supported-query-target-hit-at

```

```

0 A
(Some (((((alpha-prefix, alpha-prefix-state), data), attacker-state),
  result), final-state)) ∨
checked-staged-security-with-actual-alpha-prefix-query-prefix-single-round-candidate-binding-gap-witness-at
0 A
(Some (((((alpha-prefix, alpha-prefix-state), data), attacker-state),
  result), final-state))
proof –
let ?s =
  verifier-state-from-adversary attacker-state
  (staged-proof-transcript data)
have data-support:
  Some ((data, attacker-state), result), final-state) ∈
  set-dist
  (execute (checked-staged-security-experiment-with-data-state A)
    adversary-initial-state)
by (rule
  checked-staged-security-with-actual-alpha-prefix-support-imp-data-state-support
  [OF support])
from checked-staged-security-experiment-with-data-state-outcomeE
  [OF data-support]
obtain builder verifier where builder:
  Some (data, attacker-state) ∈
  set-dist
  (execute (checked-staged-transcript-program A)
    adversary-initial-state)
and verifier:
  Some (result, final-state) ∈ set-dist (execute verify-monad ?s)
by blast
have staged-header:
  verifier-header-transcript ?s
  (staged-trace-root data)
  (staged-trace-fri-roots data)
  (staged-trace-final data)
  (staged-alphas data)
  (staged-degree data)
  (staged-composition-fri-roots data)
  (staged-composition-final data)
  (List.concat (staged-query-chunks data))
by (rule checked-staged-transcript-program-outcome-header-transcript
  [OF wf controlled builder])
from bad obtain fr f-fri-roots f-final as dg composition-fri-roots
  final query-idxs trace-openings composition-openings trace-table
  composition-table where partial:
  accepted-with-partial-initial-openings-aligned-transcript-consistent
  ?s (Some (result, final-state)) fr f-fri-roots f-final as dg
  composition-fri-roots final query-idxs trace-openings
  composition-openings
and trace-candidate:

```

```

    partial-trace-table-candidate trace-table trace-openings
and comp-candidate:
    partial-composition-table-candidate composition-table
    composition-openings
and trace-low: trace-table-low-degree trace-table
and comp-low:
    composition-table-low-degree maxDegree composition-table
and not-all:
    ¬ all-queries-consistent trace-table composition-table as
unfolding checked-staged-security-with-actual-alpha-prefix-verifier-event-def
    query-bad-with-aligned-transcript-partial-candidates-def
by auto
have aligned:
    accepted-with-partial-initial-openings-aligned-consistent ?s
    (Some (result, final-state)) fr f-fri-roots f-final as dg
    composition-fri-roots final query-idxs trace-openings
    composition-openings
by (rule
    accepted-with-partial-initial-openings-aligned-transcript-consistent-imp-aligned
    [OF partial])
have shape:
    accepted-transcript-shape ?s (Some (result, final-state)) as
    query-idxs
by (rule
    accepted-with-partial-initial-openings-aligned-transcript-consistent-imp-shape
    [OF partial])
from checked-staged-security-with-data-state-accepted-shape-query-chunks
    [OF wf controlled data-support shape]
obtain raw-idxs where as-eq: as = staged-alphas data
and len-raw: length raw-idxs = rounds
and query-idxs-eq:
    query-idxs = map (λraw. index (to-nat raw)) raw-idxs
and lookup:
    ∀ i < rounds.
    fmlookup (HashMap final-state)
    (QueryIndexChallenge i
    (state-after-query-chunks
    (staged-query-start-hash data)
    (staged-query-chunks data) i)) =
    Some (raw-idxs ! i)
by blast
have aligned0:
    accepted-with-partial-initial-openings-aligned ?s
    (Some (result, final-state)) fr f-fri-roots f-final as dg
    composition-fri-roots final query-idxs trace-openings
    composition-openings
by (rule accepted-with-partial-initial-openings-aligned-consistent-imp-aligned
    [OF aligned])
from accepted-with-partial-initial-openings-aligned-shapes[OF aligned0]

```

```

obtain rest where header:
  verifier-header-transcript ?s fr f-fri-roots f-final as dg
    composition-fri-roots final rest
by blast
have comp-nonempty: composition-fri-roots ≠ []
  using accepted-with-partial-initial-openings-aligned-shapes(1)
    [OF aligned0] .
have trace-partial:
  accepted-with-partial-trace-openings ?s
    (Some (result, final-state)) fr query-idxs trace-openings
  using accepted-with-partial-initial-openings-aligned-shapes(3)
    [OF aligned0] .
have comp-partial:
  accepted-with-partial-composition-openings ?s
    (Some (result, final-state)) (hd composition-fri-roots) query-idxs
    composition-openings
  using accepted-with-partial-initial-openings-aligned-shapes(4)
    [OF aligned0] .
have header-eq:
  fr = staged-trace-root data ∧
  f-fri-roots = staged-trace-fri-roots data ∧
  f-final = staged-trace-final data ∧
  as = staged-alphas data ∧
  dg = staged-degree data ∧
  composition-fri-roots = staged-composition-fri-roots data ∧
  final = staged-composition-final data ∧
  rest = List.concat (staged-query-chunks data)
  using verifier-header-transcript-unique[OF header staged-header]
  by simp
let ?i = 0
have i-bound: ?i < rounds
  using rounds-positive by simp
have round-consistent:
  partial-query-round-consistent trace-openings composition-openings as
    ?i (query-idxs ! ?i)
  using aligned i-bound
  unfolding accepted-with-partial-initial-openings-aligned-consistent-def
  by simp
have opening-consistent:
  partial-query-openings-consistent (trace-openings ! ?i)
    (composition-openings ! ?i) (staged-alphas data)
    (index (to-nat (raw-idxs ! ?i)))
  using round-consistent query-idxs-eq len-raw i-bound header-eq
  unfolding partial-query-round-consistent-def
    partial-query-openings-consistent-def
  by simp
have trace-table-auth:
  partial-authenticated-table fr (scale * clength)
    (trace-openings ! ?i) final-state

```

```

using trace-partial i-bound
unfolding accepted-with-partial-trace-openings-def by blast
have comp-table-auth:
  partial-authenticated-table (hd composition-fri-roots)
    (scale * clength) (composition-openings ! ?i) final-state
using comp-partial i-bound
unfolding accepted-with-partial-composition-openings-def by blast
have witness-header:
  (trace-openings ! ?i, composition-openings ! ?i) ∈
    query-header-supported-partial-opening-witnesses ?s fr f-fri-roots
    f-final as dg composition-fri-roots final
unfolding query-header-supported-partial-opening-witnesses-def
using comp-nonempty verifier trace-table-auth comp-table-auth header
by blast
have witness:
  (trace-openings ! ?i, composition-openings ! ?i) ∈
    query-header-supported-partial-opening-witnesses ?s
      (staged-trace-root data)
      (staged-trace-fri-roots data)
      (staged-trace-final data)
      (staged-alphas data)
      (staged-degree data)
      (staged-composition-fri-roots data)
      (staged-composition-final data)
using witness-header header-eq by simp
have lookup0:
  fmlookup (HashMap final-state)
    (QueryIndexChallenge ?i
      (state-after-query-chunks
        (staged-query-start-hash data)
        (staged-query-chunks data) ?i)) =
    Some (raw-idxs ! ?i)
using lookup i-bound by simp
from checked-staged-transcript-program-query-prefix-receive-final-lookup-support-with-alphas
  [OF wf controlled i-bound builder verifier lookup0]
obtain prefix prefix-state raw-state where prefix-receive:
  Some (((prefix, prefix-state), raw-idxs ! ?i), raw-state) ∈
    set-dist
      (execute (checked-staged-query-prefix-receive-with-state A ?i)
        adversary-initial-state)
and alphas-eq: sqp-alphas prefix = staged-alphas data
by blast
have trace-candidate-single:
  partial-trace-table-candidate trace-table
    ((replicate rounds []) [?i := trace-openings ! ?i])
by (rule partial-trace-table-candidate-single-round-from-all-rounds
    [OF trace-candidate i-bound])
have comp-candidate-single:
  partial-composition-table-candidate composition-table

```

```

    ((replicate rounds []) [?i := composition-openings ! ?i])
  by (rule partial-composition-table-candidate-single-round-from-all-rounds
      [OF comp-candidate i-bound])
  have opening-target:
    checked-staged-query-prefix-dynamic-index-hit
      (staged-query-prefix-candidate-opening-query-target-from-prefix
        ((replicate rounds []) [?i := trace-openings ! ?i])
        ((replicate rounds []) [?i := composition-openings ! ?i])
        ?i)
      (Some (((prefix, prefix-state), raw-idxs ! ?i), raw-state))
  by (rule checked-staged-query-prefix-candidate-opening-target-from-prefix-single-round-hitI
      [OF i-bound])
    (use opening-consistent alphas-eq in simp)
  have not-all-prefix:
    ¬ all-queries-consistent trace-table composition-table
      (sqp-alphas prefix)
    using not-all header-eq alphas-eq by simp
  have opening-subset:
    staged-query-prefix-candidate-opening-query-target-from-prefix
      ((replicate rounds []) [?i := trace-openings ! ?i])
      ((replicate rounds []) [?i := composition-openings ! ?i])
      ?i prefix prefix-state ⊆
    staged-query-prefix-candidate-pair-query-target-from-prefix trace-table
      composition-table prefix prefix-state
  by (rule
      staged-query-prefix-candidate-opening-query-target-from-prefix-subset-candidate-pair-from-prefix
      [OF trace-candidate-single comp-candidate-single trace-low comp-low
        not-all-prefix])
  have pair-target:
    checked-staged-query-prefix-dynamic-index-hit
      (staged-query-prefix-candidate-pair-query-target-from-prefix trace-table
        composition-table)
      (Some (((prefix, prefix-state), raw-idxs ! ?i), raw-state))
    using opening-target opening-subset
  unfolding checked-staged-query-prefix-dynamic-index-hit-def by auto
  have pair-hit:
    checked-staged-security-with-actual-alpha-prefix-candidate-pair-query-hit-from-prefix
      A trace-table composition-table ?i
      (Some (((((alpha-prefix, alpha-prefix-state), data), attacker-state),
        result), final-state))
  by (rule
      checked-staged-security-with-actual-alpha-prefix-candidate-pair-query-hit-from-prefixI
      [OF i-bound builder prefix-receive pair-target])
  show ?thesis
  by (rule
      checked-staged-security-with-actual-alpha-prefix-candidate-pair-query-hit-from-prefix-with-witness-imp-single-round
      [OF pair-hit witness trace-candidate-single comp-candidate-single])
qed

```


definition *checked-staged-security-with-query-prefix-single-round-candidate-binding-gap-witness-exists-at*
where

checked-staged-security-with-query-prefix-single-round-candidate-binding-gap-witness-exists-at
 $i \text{ out} \longleftrightarrow$
 $(\exists \text{ trace-openings composition-openings trace-table composition-table.}$
checked-staged-security-with-query-prefix-single-round-candidate-binding-gap-witness-at
trace-openings composition-openings trace-table composition-table i out)

definition *checked-staged-security-with-query-prefix-single-round-candidate-binding-gap-witness-current-exists-at*
where

checked-staged-security-with-query-prefix-single-round-candidate-binding-gap-witness-current-exists-at
 $i \text{ out} \longleftrightarrow$
checked-staged-security-with-query-prefix-current-query-partial-opening-hit-at
 $i \text{ out} \wedge$
checked-staged-security-with-query-prefix-single-round-candidate-binding-gap-witness-exists-at
 $i \text{ out}$

definition *checked-staged-security-with-query-prefix-single-round-header-supported-query-target-current-hit-at*
where

checked-staged-security-with-query-prefix-single-round-header-supported-query-target-current-hit-at
 $i \text{ out} \longleftrightarrow$
checked-staged-security-with-query-prefix-current-query-partial-opening-hit-at
 $i \text{ out} \wedge$
checked-staged-security-with-query-prefix-single-round-header-supported-query-target-hit-at
 $i \text{ out}$

lemma *checked-staged-security-with-query-prefix-single-round-candidate-binding-gap-witness-current-exists-bound-from-*

assumes *current-bound:*

wp-event
 $(\text{checked-staged-security-experiment-with-query-prefix-data-state } A \ i)$
 $(\text{checked-staged-security-with-query-prefix-current-query-partial-opening-hit-at}$
 $i)$
 $\text{adversary-initial-state} \leq C$

shows

wp-event
 $(\text{checked-staged-security-experiment-with-query-prefix-data-state } A \ i)$
 $(\text{checked-staged-security-with-query-prefix-single-round-candidate-binding-gap-witness-current-exists-at}$
 $i)$
 $\text{adversary-initial-state} \leq C$

by $(\text{rule order-trans}[\text{OF} - \text{current-bound}],$

$\text{rule wp-event-mono})$

$(\text{simp add:}$

$\text{checked-staged-security-with-query-prefix-single-round-candidate-binding-gap-witness-current-exists-at-def})$

lemma *checked-staged-security-with-query-prefix-single-round-header-supported-query-target-current-hit-bound-from-*

assumes *current-bound:*

wp-event
 $(\text{checked-staged-security-experiment-with-query-prefix-data-state } A \ i)$
 $(\text{checked-staged-security-with-query-prefix-current-query-partial-opening-hit-at}$

i)
adversary-initial-state $\leq C$
shows
wp-event
(checked-staged-security-experiment-with-query-prefix-data-state A i)
(checked-staged-security-with-query-prefix-single-round-header-supported-query-target-current-hit-at
i)
adversary-initial-state $\leq C$
by (*rule order-trans*[*OF* - *current-bound*],
rule wp-event-mono)
(simp add:
checked-staged-security-with-query-prefix-single-round-header-supported-query-target-current-hit-at-def)

lemma *checked-staged-security-with-query-prefix-data-state-aligned-transcript-query-bad-imp-single-round-component*
assumes *wf: staged-budget-wellformed budgets*
and *controlled: staged-adversary-controlled budgets A*
and *support:*
Some (((((prefix, prefix-state), raw), raw-state),
((data, attacker-state), result)), final-state) $\in$
set-dist
(execute
(checked-staged-security-experiment-with-query-prefix-data-state A 0)
adversary-initial-state)
and *bad:*
staged-security-with-data-state-verifier-event
query-bad-with-aligned-transcript-partial-candidates
(Some (((data, attacker-state), result), final-state))
shows
checked-staged-security-with-query-prefix-single-round-header-supported-query-target-current-hit-at
0
(Some (((((prefix, prefix-state), raw), raw-state),
((data, attacker-state), result)), final-state)) $\vee$
checked-staged-security-with-query-prefix-single-round-candidate-pair-witness-gap-exists-at
0
(Some (((((prefix, prefix-state), raw), raw-state),
((data, attacker-state), result)), final-state))

proof –
let *?s =*
verifier-state-from-adversary attacker-state
(staged-proof-transcript data)
let *?i = 0*
have *i-bound: ?i < rounds*
using *rounds-positive* **by** *simp*
have *data-support:*
Some (((data, attacker-state), result), final-state) $\in$
set-dist
(execute (checked-staged-security-experiment-with-data-state A)
adversary-initial-state)
by (*rule*

```

    checked-staged-security-experiment-with-query-prefix-data-state-imp-data-state-support
    [OF i-bound support])
from checked-staged-security-experiment-with-data-state-outcomeE
    [OF data-support]
obtain builder verifier where builder:
    Some (data, attacker-state) ∈
    set-dist
    (execute (checked-staged-transcript-program A)
    adversary-initial-state)
and verifier:
    Some (result, final-state) ∈ set-dist (execute verify-monad ?s)
by blast
have staged-header:
    verifier-header-transcript ?s
    (staged-trace-root data)
    (staged-trace-fri-roots data)
    (staged-trace-final data)
    (staged-alphas data)
    (staged-degree data)
    (staged-composition-fri-roots data)
    (staged-composition-final data)
    (List.concat (staged-query-chunks data))
by (rule checked-staged-transcript-program-outcome-header-transcript
    [OF wf controlled builder])
from bad obtain fr f-fri-roots f-final as dg composition-fri-roots
    final query-idxs trace-openings composition-openings trace-table
    composition-table where partial:
    accepted-with-partial-initial-openings-aligned-transcript-consistent
    ?s (Some (result, final-state)) fr f-fri-roots f-final as dg
    composition-fri-roots final query-idxs trace-openings
    composition-openings
and trace-candidate:
    partial-trace-table-candidate trace-table trace-openings
and comp-candidate:
    partial-composition-table-candidate composition-table
    composition-openings
and trace-low: trace-table-low-degree trace-table
and comp-low:
    composition-table-low-degree maxDegree composition-table
and not-all:
    ¬ all-queries-consistent trace-table composition-table as
unfolding staged-security-with-data-state-verifier-event-def
    query-bad-with-aligned-transcript-partial-candidates-def
by auto
have aligned:
    accepted-with-partial-initial-openings-aligned-consistent ?s
    (Some (result, final-state)) fr f-fri-roots f-final as dg
    composition-fri-roots final query-idxs trace-openings
    composition-openings

```

```

by (rule
  accepted-with-partial-initial-openings-aligned-transcript-consistent-imp-aligned
  [OF partial])
have shape:
  accepted-transcript-shape ?s (Some (result, final-state)) as
  query-idxs
by (rule
  accepted-with-partial-initial-openings-aligned-transcript-consistent-imp-shape
  [OF partial])
have index-eq: index (to-nat raw) = query-idxs ! ?i
by (rule
  checked-staged-security-with-query-prefix-data-state-accepted-shape-query-index
  [OF wf controlled i-bound support shape])
have aligned0:
  accepted-with-partial-initial-openings-aligned ?s
  (Some (result, final-state)) fr f-fri-roots f-final as dg
  composition-fri-roots final query-idxs trace-openings
  composition-openings
by (rule accepted-with-partial-initial-openings-aligned-consistent-imp-aligned
  [OF aligned])
from accepted-with-partial-initial-openings-aligned-shapes[OF aligned0]
obtain rest where header:
  verifier-header-transcript ?s fr f-fri-roots f-final as dg
  composition-fri-roots final rest
by blast
have comp-nonempty: composition-fri-roots ≠ []
using accepted-with-partial-initial-openings-aligned-shapes(1)
[OF aligned0] .
have trace-partial:
  accepted-with-partial-trace-openings ?s
  (Some (result, final-state)) fr query-idxs trace-openings
using accepted-with-partial-initial-openings-aligned-shapes(3)
[OF aligned0] .
have comp-partial:
  accepted-with-partial-composition-openings ?s
  (Some (result, final-state)) (hd composition-fri-roots) query-idxs
  composition-openings
using accepted-with-partial-initial-openings-aligned-shapes(4)
[OF aligned0] .
have header-eq:
  fr = staged-trace-root data ∧
  f-fri-roots = staged-trace-fri-roots data ∧
  f-final = staged-trace-final data ∧
  as = staged-alphas data ∧
  dg = staged-degree data ∧
  composition-fri-roots = staged-composition-fri-roots data ∧
  final = staged-composition-final data ∧
  rest = List.concat (staged-query-chunks data)
using verifier-header-transcript-unique[OF header staged-header]

```

```

by simp
have alphas-eq: staged-alphas data = sqp-alphas prefix
by (rule checked-staged-security-with-query-prefix-data-state-alphas-eq)
(rule support)
have round-consistent:
partial-query-round-consistent trace-openings composition-openings as
  ?i (query-idxs ! ?i)
using aligned i-bound
unfolding accepted-with-partial-initial-openings-aligned-consistent-def
by simp
have opening-consistent:
partial-query-openings-consistent (trace-openings ! ?i)
  (composition-openings ! ?i) (sqp-alphas prefix)
  (index (to-nat raw))
using round-consistent index-eq header-eq alphas-eq
unfolding partial-query-round-consistent-def
  partial-query-openings-consistent-def
by simp
have trace-table-auth:
partial-authenticated-table fr (scale * clength)
  (trace-openings ! ?i) final-state
using trace-partial i-bound
unfolding accepted-with-partial-trace-openings-def by blast
have comp-table-auth:
partial-authenticated-table (hd composition-fri-roots)
  (scale * clength) (composition-openings ! ?i) final-state
using comp-partial i-bound
unfolding accepted-with-partial-composition-openings-def by blast
have witness-header:
(trace-openings ! ?i, composition-openings ! ?i) ∈
  query-header-supported-partial-opening-witnesses ?s fr f-fri-roots
  f-final as dg composition-fri-roots final
unfolding query-header-supported-partial-opening-witnesses-def
using comp-nonempty verifier trace-table-auth comp-table-auth header
by blast
have witness:
(trace-openings ! ?i, composition-openings ! ?i) ∈
  query-header-supported-partial-opening-witnesses ?s
    (staged-trace-root data)
    (staged-trace-fri-roots data)
    (staged-trace-final data)
    (staged-alphas data)
    (staged-degree data)
    (staged-composition-fri-roots data)
    (staged-composition-final data)
using witness-header header-eq by simp
have trace-candidate-single:
partial-trace-table-candidate trace-table
  ((replicate rounds []) [?i := trace-openings ! ?i])

```

```

by (rule partial-trace-table-candidate-single-round-from-all-rounds
  [OF trace-candidate i-bound])
have comp-candidate-single:
  partial-composition-table-candidate composition-table
  ((replicate rounds []) [?i := composition-openings ! ?i])
by (rule partial-composition-table-candidate-single-round-from-all-rounds
  [OF comp-candidate i-bound])
have opening-target:
  checked-staged-query-prefix-dynamic-index-hit
  (staged-query-prefix-candidate-opening-query-target-from-prefix
    ((replicate rounds []) [?i := trace-openings ! ?i])
    ((replicate rounds []) [?i := composition-openings ! ?i])
    ?i)
  (Some (((prefix, prefix-state), raw), raw-state))
by (rule checked-staged-query-prefix-candidate-opening-target-from-prefix-single-round-hitI
  [OF i-bound opening-consistent])
have not-all-prefix:
  ¬ all-queries-consistent trace-table composition-table
  (sqp-alphas prefix)
using not-all header-eq alphas-eq by simp
have opening-subset:
  staged-query-prefix-candidate-opening-query-target-from-prefix
  ((replicate rounds []) [?i := trace-openings ! ?i])
  ((replicate rounds []) [?i := composition-openings ! ?i])
  ?i prefix prefix-state ⊆
  staged-query-prefix-candidate-pair-query-target-from-prefix trace-table
  composition-table prefix prefix-state
by (rule
  staged-query-prefix-candidate-opening-query-target-from-prefix-subset-candidate-pair-from-prefix
  [OF trace-candidate-single comp-candidate-single trace-low comp-low
    not-all-prefix])
have pair-target:
  checked-staged-query-prefix-dynamic-index-hit
  (staged-query-prefix-candidate-pair-query-target-from-prefix trace-table
    composition-table)
  (Some (((prefix, prefix-state), raw), raw-state))
using opening-target opening-subset
unfolding checked-staged-query-prefix-dynamic-index-hit-def by auto
have auth-hit:
  checked-staged-security-with-query-prefix-authenticated-opening-hit
  (trace-openings ! ?i) (composition-openings ! ?i) ?i
  (Some (((((prefix, prefix-state), raw), raw-state),
    ((data, attacker-state), result)), final-state))
unfolding
  checked-staged-security-with-query-prefix-authenticated-opening-hit-def
using comp-nonempty trace-table-auth comp-table-auth opening-consistent
  header-eq alphas-eq
by simp
show ?thesis

```

```

proof (cases
  (trace-table, composition-table) ∈
    query-header-supported-single-round-partial-table-candidates
    prefix-state
    (sqp-trace-root prefix)
    (sqp-trace-fri-roots prefix)
    (sqp-trace-final prefix)
    (sqp-alphas prefix)
    (sqp-degree prefix)
    (sqp-composition-fri-roots prefix)
    (sqp-composition-final prefix)
    ?i)
case True
have target-hit:
  checked-staged-query-prefix-dynamic-index-hit
  (staged-query-prefix-single-round-header-supported-query-target ?i)
  (Some (((prefix, prefix-state), raw), raw-state))
by (rule
  checked-staged-query-prefix-candidate-pair-hit-imp-single-round-header-target-hit
  [OF True pair-target])
have
  checked-staged-security-with-query-prefix-single-round-header-supported-query-target-current-hit-at
  ?i
  (Some (((((prefix, prefix-state), raw), raw-state),
    ((data, attacker-state), result)), final-state))
unfolding
  checked-staged-security-with-query-prefix-single-round-header-supported-query-target-current-hit-at-def
  checked-staged-security-with-query-prefix-single-round-header-supported-query-target-hit-at-def
using target-hit
  checked-staged-security-with-query-prefix-current-query-partial-opening-hit-atI
  [OF auth-hit]
by simp
then show ?thesis by simp
next
case False
have success-prefix:
  index (to-nat raw) ∈
    query-sampling-success-space trace-table composition-table
    (sqp-alphas prefix)
using pair-target
unfolding
  checked-staged-query-prefix-dynamic-index-hit-def
  staged-query-prefix-candidate-pair-query-target-from-prefix-def
by simp
have gap-witness:
  checked-staged-security-with-query-prefix-single-round-candidate-pair-witness-gap-at
  (trace-openings ! ?i) (composition-openings ! ?i)
  trace-table composition-table ?i
  (Some (((((prefix, prefix-state), raw), raw-state),

```

```

      ((data, attacker-state), result)), final-state))
  by (rule
    checked-staged-security-with-query-prefix-single-round-candidate-pair-witness-gap-atI
    [OF success-prefix witness trace-candidate-single
      comp-candidate-single False])
  have gap-exists:
    checked-staged-security-with-query-prefix-single-round-candidate-pair-witness-gap-exists-at
      ?i
      (Some (((((prefix, prefix-state), raw), raw-state),
        ((data, attacker-state), result)), final-state))
  by (rule
    checked-staged-security-with-query-prefix-single-round-candidate-pair-witness-gap-exists-atI
    [OF gap-witness])
  then show ?thesis
    using gap-exists by simp
qed
qed

lemma checked-staged-security-with-query-prefix-data-state-aligned-transcript-query-bad-extract-round0-authenticated
assumes wf: staged-budget-wellformed budgets
and controlled: staged-adversary-controlled budgets A
and support:
  Some (((((prefix, prefix-state), raw), raw-state),
    ((data, attacker-state), result)), final-state) ∈
  set-dist
  (execute
    (checked-staged-security-experiment-with-query-prefix-data-state A 0)
    adversary-initial-state)
and bad:
  staged-security-with-data-state-verifier-event
  query-bad-with-aligned-transcript-partial-candidates
  (Some (((data, attacker-state), result), final-state))
obtains trace-opening composition-opening trace-table composition-table where
  checked-staged-security-with-query-prefix-authenticated-opening-hit
  trace-opening composition-opening 0
  (Some (((((prefix, prefix-state), raw), raw-state),
    ((data, attacker-state), result)), final-state))
  (trace-opening, composition-opening) ∈
  query-header-supported-partial-opening-witnesses
  (verifier-state-from-adversary attacker-state
    (staged-proof-transcript data))
  (staged-trace-root data)
  (staged-trace-fri-roots data)
  (staged-trace-final data)
  (staged-alphas data)
  (staged-degree data)
  (staged-composition-fri-roots data)
  (staged-composition-final data)
  partial-trace-table-candidate trace-table

```



```

    ((replicate rounds []) [0 := trace-opening])
    partial-composition-table-candidate composition-table
    ((replicate rounds []) [0 := composition-opening])
    trace-table-low-degree trace-table
    composition-table-low-degree maxDegree composition-table
    ¬ all-queries-consistent trace-table composition-table
    (sqp-alphas prefix)
proof –
  let ?s =
    verifier-state-from-adversary attacker-state
    (staged-proof-transcript data)
  let ?i = 0
  have i-bound: ?i < rounds
    using rounds-positive by simp
  have data-support:
    Some (((data, attacker-state), result), final-state) ∈
      set-dist
      (execute (checked-staged-security-experiment-with-data-state A)
        adversary-initial-state)
    by (rule
      checked-staged-security-experiment-with-query-prefix-data-state-imp-data-state-support
      [OF i-bound support])
  from checked-staged-security-experiment-with-data-state-outcomeE
    [OF data-support]
  obtain builder verifier where builder:
    Some (data, attacker-state) ∈
      set-dist
      (execute (checked-staged-transcript-program A)
        adversary-initial-state)
  and verifier:
    Some (result, final-state) ∈ set-dist (execute verify-monad ?s)
  by blast
  have staged-header:
    verifier-header-transcript ?s
    (staged-trace-root data)
    (staged-trace-fri-roots data)
    (staged-trace-final data)
    (staged-alphas data)
    (staged-degree data)
    (staged-composition-fri-roots data)
    (staged-composition-final data)
    (List.concat (staged-query-chunks data))
  by (rule checked-staged-transcript-program-outcome-header-transcript
    [OF wf controlled builder])
  from bad obtain fr f-fri-roots f-final as dg composition-fri-roots
    final query-idxs trace-openings composition-openings trace-table
    composition-table where partial:
    accepted-with-partial-initial-openings-aligned-transcript-consistent
    ?s (Some (result, final-state)) fr f-fri-roots f-final as dg

```

```

    composition-fri-roots final query-idxs trace-openings
    composition-openings
and trace-candidate:
    partial-trace-table-candidate trace-table trace-openings
and comp-candidate:
    partial-composition-table-candidate composition-table
    composition-openings
and trace-low: trace-table-low-degree trace-table
and comp-low:
    composition-table-low-degree maxDegree composition-table
and not-all:
     $\neg$  all-queries-consistent trace-table composition-table as
unfolding staged-security-with-data-state-verifier-event-def
    query-bad-with-aligned-transcript-partial-candidates-def
by auto
have aligned:
    accepted-with-partial-initial-openings-aligned-consistent ?s
    (Some (result, final-state)) fr f-fri-roots f-final as dg
    composition-fri-roots final query-idxs trace-openings
    composition-openings
by (rule
    accepted-with-partial-initial-openings-aligned-transcript-consistent-imp-aligned
    [OF partial])
have aligned0:
    accepted-with-partial-initial-openings-aligned ?s
    (Some (result, final-state)) fr f-fri-roots f-final as dg
    composition-fri-roots final query-idxs trace-openings
    composition-openings
by (rule accepted-with-partial-initial-openings-aligned-consistent-imp-aligned
    [OF aligned])
from accepted-with-partial-initial-openings-aligned-shapes[OF aligned0]
obtain rest where header:
    verifier-header-transcript ?s fr f-fri-roots f-final as dg
    composition-fri-roots final rest
by blast
have comp-nonempty: composition-fri-roots  $\neq$  []
using accepted-with-partial-initial-openings-aligned-shapes(1)
    [OF aligned0] .
have trace-partial:
    accepted-with-partial-trace-openings ?s
    (Some (result, final-state)) fr query-idxs trace-openings
using accepted-with-partial-initial-openings-aligned-shapes(3)
    [OF aligned0] .
have comp-partial:
    accepted-with-partial-composition-openings ?s
    (Some (result, final-state)) (hd composition-fri-roots) query-idxs
    composition-openings
using accepted-with-partial-initial-openings-aligned-shapes(4)
    [OF aligned0] .

```

```

have header-eq:
  fr = staged-trace-root data ∧
  f-fri-roots = staged-trace-fri-roots data ∧
  f-final = staged-trace-final data ∧
  as = staged-alphas data ∧
  dg = staged-degree data ∧
  composition-fri-roots = staged-composition-fri-roots data ∧
  final = staged-composition-final data ∧
  rest = List.concat (staged-query-chunks data)
using verifier-header-transcript-unique[OF header staged-header]
by simp
have alphas-eq: staged-alphas data = sqp-alphas prefix
by (rule checked-staged-security-with-query-prefix-data-state-alphas-eq)
    (rule support)
have shape:
  accepted-transcript-shape ?s (Some (result, final-state)) as
    query-idxs
by (rule
    accepted-with-partial-initial-openings-aligned-transcript-consistent-imp-shape
    [OF partial])
have index-eq: index (to-nat raw) = query-idxs ! ?i
by (rule
    checked-staged-security-with-query-prefix-data-state-accepted-shape-query-index
    [OF wf controlled i-bound support shape])
have round-consistent:
  partial-query-round-consistent trace-openings composition-openings as
    ?i (query-idxs ! ?i)
using aligned i-bound
unfolding accepted-with-partial-initial-openings-aligned-consistent-def
by simp
have opening-consistent:
  partial-query-openings-consistent (trace-openings ! ?i)
    (composition-openings ! ?i) (sqp-alphas prefix)
    (index (to-nat raw))
using round-consistent index-eq header-eq alphas-eq
unfolding partial-query-round-consistent-def
  partial-query-openings-consistent-def
by simp
have trace-table-auth:
  partial-authenticated-table fr (scale * clength)
    (trace-openings ! ?i) final-state
using trace-partial i-bound
unfolding accepted-with-partial-trace-openings-def by blast
have comp-table-auth:
  partial-authenticated-table (hd composition-fri-roots)
    (scale * clength) (composition-openings ! ?i) final-state
using comp-partial i-bound
unfolding accepted-with-partial-composition-openings-def by blast
have witness-header:

```

```

(trace-openings ! ?i, composition-openings ! ?i) ∈
  query-header-supported-partial-opening-witnesses ?s fr f-fri-roots
  f-final as dg composition-fri-roots final
unfolding query-header-supported-partial-opening-witnesses-def
using comp-nonempty verifier trace-table-auth comp-table-auth header
by blast
have witness:
  (trace-openings ! ?i, composition-openings ! ?i) ∈
    query-header-supported-partial-opening-witnesses ?s
    (staged-trace-root data)
    (staged-trace-fri-roots data)
    (staged-trace-final data)
    (staged-alphas data)
    (staged-degree data)
    (staged-composition-fri-roots data)
    (staged-composition-final data)
using witness-header header-eq by simp
have trace-candidate-single:
  partial-trace-table-candidate trace-table
  ((replicate rounds []) [?i := trace-openings ! ?i])
by (rule partial-trace-table-candidate-single-round-from-all-rounds
    [OF trace-candidate i-bound])
have comp-candidate-single:
  partial-composition-table-candidate composition-table
  ((replicate rounds []) [?i := composition-openings ! ?i])
by (rule partial-composition-table-candidate-single-round-from-all-rounds
    [OF comp-candidate i-bound])
have auth-hit:
  checked-staged-security-with-query-prefix-authenticated-opening-hit
  (trace-openings ! ?i) (composition-openings ! ?i) ?i
  (Some (((((prefix, prefix-state), raw), raw-state),
    ((data, attacker-state), result)), final-state))
unfolding
  checked-staged-security-with-query-prefix-authenticated-opening-hit-def
using comp-nonempty trace-table-auth comp-table-auth opening-consistent
  header-eq alphas-eq
by simp
have not-all-prefix:
  ¬ all-queries-consistent trace-table composition-table
  (sqp-alphas prefix)
using not-all header-eq alphas-eq by simp
show ?thesis
by (rule that[OF auth-hit witness trace-candidate-single
  comp-candidate-single trace-low comp-low not-all-prefix])
qed

lemma checked-staged-security-with-data-state-aligned-transcript-query-bad-bound-from-query-prefix-single-round-con
fixes T Gap :: prob
assumes wf: staged-budget-wellformed budgets

```

and *controlled*: staged-adversary-controlled budgets A
and *target-bound*:
 wp-event
 (*checked-staged-security-experiment-with-query-prefix-data-state* A 0)
 (*checked-staged-security-with-query-prefix-single-round-header-supported-query-target-current-hit-at*
 0)
 adversary-initial-state $\leq T$
and *gap-bound*:
 wp-event
 (*checked-staged-security-experiment-with-query-prefix-data-state* A 0)
 (*checked-staged-security-with-query-prefix-single-round-candidate-pair-witness-gap-exists-at*
 0)
 adversary-initial-state $\leq \text{Gap}$
shows
 wp-event (*checked-staged-security-experiment-with-data-state* A)
 (*staged-security-with-data-state-verifier-event*
 query-bad-with-aligned-transcript-partial-candidates)
 adversary-initial-state $\leq T + \text{Gap}$
proof –
 let $?i = 0$
 let $?m =$
 checked-staged-security-experiment-with-query-prefix-data-state A $?i$
 let $?query =$
 staged-security-with-data-state-verifier-event
 query-bad-with-aligned-transcript-partial-candidates
 let $?projected =$
 $\lambda out. \text{case out of}$
 $\text{None} \Rightarrow ?query \text{ None}$
 $| \text{Some (packed, t)} \Rightarrow ?query (\text{Some (snd packed, t)})$
 let $?target =$
 checked-staged-security-with-query-prefix-single-round-header-supported-query-target-current-hit-at
 $?i$
 let $?gap =$
 checked-staged-security-with-query-prefix-single-round-candidate-pair-witness-gap-exists-at
 $?i$
 have *i-bound*: $?i < \text{rounds}$
 using *rounds-positive* **by** *simp*
 have *projection*:
 wp-event (*checked-staged-security-experiment-with-data-state* A)
 $?query$ *adversary-initial-state* =
 wp-event $?m$ $?projected$ *adversary-initial-state*
 by (*rule*
 checked-staged-security-experiment-with-query-prefix-data-state-projection-event
 [*OF i-bound*])
 have *event-le*:
 wp-event $?m$ $?projected$ *adversary-initial-state* $\leq$
 wp-event $?m$ ($\lambda out. ?target \text{ out} \vee ?gap \text{ out}$)
 adversary-initial-state
proof (*rule wp-event-mono-on-support*)

```

fix out
assume support: out  $\in$  set-dist (execute ?m adversary-initial-state)
  and projected: ?projected out
show ?target out  $\vee$  ?gap out
proof (cases out)
  case None
  then show ?thesis
    using projected
    unfolding staged-security-with-data-state-verifier-event-def by simp
next
  case (Some packed)
  then obtain prefix prefix-state raw raw-state data attacker-state
    result final-state where out-eq:
    out =
      Some (((((prefix, prefix-state), raw), raw-state),
        ((data, attacker-state), result)), final-state)
    by (cases packed) (auto split: prod.splits)
  have bad:
    staged-security-with-data-state-verifier-event
      query-bad-with-aligned-transcript-partial-candidates
      (Some (((data, attacker-state), result), final-state))
    using projected unfolding out-eq by simp
  show ?thesis
    unfolding out-eq
    by (rule
      checked-staged-security-with-query-prefix-data-state-aligned-transcript-query-bad-imp-single-round-components
      [OF wf controlled])
    (use support bad out-eq in simp-all)
  qed
qed
have union-bound:
  wp-event ?m ( $\lambda$ out. ?target out  $\vee$  ?gap out)
    adversary-initial-state  $\leq$ 
  wp-event ?m ?target adversary-initial-state +
  wp-event ?m ?gap adversary-initial-state
  by (rule wp-event-union-bound)
have sum-bound:
  wp-event ?m ?target adversary-initial-state +
  wp-event ?m ?gap adversary-initial-state  $\leq$  T + Gap
  by (rule add-mono[OF target-bound gap-bound])
show ?thesis
  unfolding projection
  by (rule order-trans[OF event-le order-trans[OF union-bound sum-bound]])
qed

lemma checked-staged-security-with-data-state-aligned-transcript-query-bad-bound-from-query-prefix-single-round-target
  fixes T Gap :: prob
  assumes wf: staged-budget-wellformed budgets
  and controlled: staged-adversary-controlled budgets A

```

and *target-bound*:
 wp-event
 (*checked-staged-security-experiment-with-query-prefix-data-state* A 0)
 (*checked-staged-security-with-query-prefix-single-round-header-supported-query-target-current-hit-at*
 0)
 adversary-initial-state $\leq T$
and *gap-bound*:
 wp-event
 (*checked-staged-security-experiment-with-query-prefix-data-state* A 0)
 (*checked-staged-security-with-query-prefix-single-round-candidate-pair-witness-gap-exists-at*
 0)
 adversary-initial-state $\leq \text{Gap}$
shows
 wp-event (*checked-staged-security-experiment-with-data-state* A)
 (*staged-security-with-data-state-verifier-event*
 query-bad-with-aligned-transcript-partial-candidates)
 adversary-initial-state $\leq T + \text{Gap}$
by (*rule*
 checked-staged-security-with-data-state-aligned-transcript-query-bad-bound-from-query-prefix-single-round-compon
 [*OF wf controlled target-bound gap-bound*])

lemma *checked-staged-security-with-data-state-aligned-transcript-query-bad-bound-from-query-prefix-single-round-tar*
fixes $T\ P\ Q :: \text{prob}$
assumes *wf*: *staged-budget-wellformed budgets*
and *controlled*: *staged-adversary-controlled budgets* A
and *target-bound*:
 wp-event
 (*checked-staged-security-experiment-with-query-prefix-data-state* A 0)
 (*checked-staged-security-with-query-prefix-single-round-header-supported-query-target-current-hit-at*
 0)
 adversary-initial-state $\leq T$
and *data-pre-bound*:
 wp-event (*checked-staged-security-experiment-with-data-state* A)
 staged-security-with-data-state-transcript-pre-hit
 adversary-initial-state $\leq P$
and *new-bound*:
 wp-event
 (*checked-staged-security-experiment-with-actual-alpha-prefix-data-state*
 A)
 checked-staged-security-with-actual-alpha-prefix-witness-transcript-new-hit
 adversary-initial-state $\leq Q$
shows
 wp-event (*checked-staged-security-experiment-with-data-state* A)
 (*staged-security-with-data-state-verifier-event*
 query-bad-with-aligned-transcript-partial-candidates)
 adversary-initial-state $\leq T + (P + Q)$
proof –
have *i-bound*: $0 < \text{rounds}$
using *rounds-positive* **by** *simp*

have *gap-bound*:
 wp-event
 (*checked-staged-security-experiment-with-query-prefix-data-state* A 0)
 (*checked-staged-security-with-query-prefix-single-round-candidate-pair-witness-gap-exists-at*
 0)
 adversary-initial-state $\leq P + Q$
by (*rule*
 checked-staged-security-with-query-prefix-single-round-candidate-pair-witness-gap-exists-at-bound-from-data-pre-
 [*OF i-bound data-pre-bound new-bound*])
show *?thesis*
by (*rule*
 checked-staged-security-with-data-state-aligned-transcript-query-bad-bound-from-query-prefix-single-round-target-
 [*OF wf controlled target-bound gap-bound*])
qed

lemma *checked-staged-security-with-data-state-aligned-transcript-query-bad-bound-from-header-authenticated-candidate*
assumes *wf*: *staged-budget-wellformed budgets*
and *controlled*: *staged-adversary-controlled budgets* A
and *header-bound*:
 wp-event
 (*checked-staged-security-experiment-with-actual-alpha-prefix-data-state*
 A)
 (*checked-staged-security-with-actual-alpha-prefix-header-authenticated-candidate-opening-query-hit-from-prefix*
 A)
 adversary-initial-state $\leq Q$
shows
 wp-event (*checked-staged-security-experiment-with-data-state* A)
 (*staged-security-with-data-state-verifier-event*
 query-bad-with-aligned-transcript-partial-candidates)
 adversary-initial-state $\leq Q$
proof –
let *?project* =
 $\lambda x. ((\text{snd } (\text{fst } (\text{fst } x))), \text{snd } (\text{fst } x)), \text{snd } x)$
let *?query* =
 staged-security-with-data-state-verifier-event
 query-bad-with-aligned-transcript-partial-candidates
let *?projected* =
 $\lambda \text{out}. \text{case out of}$
 $\text{None} \Rightarrow \text{False}$
 | $\text{Some } (\text{packed}, t) \Rightarrow ?\text{query } (\text{Some } (?project \text{ packed}, t))$
have *projection*:
 wp-event (*checked-staged-security-experiment-with-data-state* A)
 ?query adversary-initial-state =
 wp-event
 (*checked-staged-security-experiment-with-actual-alpha-prefix-data-state* A)
 ?projected adversary-initial-state
proof –
have
 wp-event (*checked-staged-security-experiment-with-data-state* A)


```

    ?query adversary-initial-state =
wp-event
  (checked-staged-security-experiment-with-actual-alpha-prefix-data-state A
     $\gg$  ( $\lambda x.$  return ( $?project\ x$ )))
    ?query adversary-initial-state
using
  checked-staged-security-experiment-with-actual-alpha-prefix-data-state-projection
  [of A]
by simp
also have ... =
wp-event
  (checked-staged-security-experiment-with-actual-alpha-prefix-data-state A)
  ?projected adversary-initial-state
by (subst wp-event-bind-return-map)
  (simp add: staged-security-with-data-state-verifier-event-def)
finally show ?thesis
by simp
qed
have event-le:
wp-event
  (checked-staged-security-experiment-with-actual-alpha-prefix-data-state A)
  ?projected adversary-initial-state  $\leq$ 
wp-event
  (checked-staged-security-experiment-with-actual-alpha-prefix-data-state A)
(checked-staged-security-with-actual-alpha-prefix-header-authenticated-candidate-opening-query-hit-from-prefix
  A)
  adversary-initial-state
proof (rule wp-event-mono-on-support)
fix out
assume support:
  out  $\in$ 
  set-dist
  (execute
    (checked-staged-security-experiment-with-actual-alpha-prefix-data-state
      A)
    adversary-initial-state)
and projected: ?projected out
show
checked-staged-security-with-actual-alpha-prefix-header-authenticated-candidate-opening-query-hit-from-prefix
  A out
proof (cases out)
case None
then show ?thesis
  using projected by simp
next
case (Some packed)
then obtain alpha-prefix alpha-prefix-state data attacker-state
  result final-state where out-eq:
  out =

```

```

      Some (((((alpha-prefix, alpha-prefix-state), data),
               attacker-state), result), final-state)
    by (cases packed, auto split: prod.splits)
  have bad:
    checked-staged-security-with-actual-alpha-prefix-verifier-event
      query-bad-with-aligned-transcript-partial-candidates out
    using projected unfolding out-eq
  by (simp add:
      checked-staged-security-with-actual-alpha-prefix-verifier-event-def
      staged-security-with-data-state-verifier-event-def Let-def)
  show ?thesis
    unfolding out-eq
  by (rule
      checked-staged-security-with-actual-alpha-prefix-aligned-transcript-query-bad-imp-header-authenticated-candidates
        [OF wf controlled])
    (use support bad out-eq in simp-all)
qed
qed
show ?thesis
  unfolding projection
  by (rule order-trans[OF event-le header-bound])
qed

lemma checked-staged-security-with-data-state-aligned-transcript-query-bad-bound-from-single-round-components:
  fixes T Gap :: prob
  assumes wf: staged-budget-wellformed budgets
  and controlled: staged-adversary-controlled budgets A
  and target-bound:
    wp-event
      (checked-staged-security-experiment-with-actual-alpha-prefix-data-state
        A)
    (checked-staged-security-with-actual-alpha-prefix-single-round-header-supported-query-target-hit-at
      0 A)
    adversary-initial-state ≤ T
  and gap-bound:
    wp-event
      (checked-staged-security-experiment-with-actual-alpha-prefix-data-state
        A)
    (checked-staged-security-with-actual-alpha-prefix-query-prefix-single-round-candidate-binding-gap-witness-at
      0 A)
    adversary-initial-state ≤ Gap
  shows
    wp-event (checked-staged-security-experiment-with-data-state A)
      (staged-security-with-data-state-verifier-event
        query-bad-with-aligned-transcript-partial-candidates)
      adversary-initial-state ≤ T + Gap
proof -
  let ?project =
    λx. ((snd (fst (fst x)), snd (fst x)), snd x)

```

```

let ?query =
  staged-security-with-data-state-verifier-event
  query-bad-with-aligned-transcript-partial-candidates
let ?projected =
  λout. case out of
    None ⇒ False
  | Some (packed, t) ⇒ ?query (Some (?project packed, t))
let ?target =
  checked-staged-security-with-actual-alpha-prefix-single-round-header-supported-query-target-hit-at
  0 A
let ?gap =
  checked-staged-security-with-actual-alpha-prefix-query-prefix-single-round-candidate-binding-gap-witness-at
  0 A
have projection:
  wp-event (checked-staged-security-experiment-with-data-state A)
  ?query adversary-initial-state =
  wp-event
  (checked-staged-security-experiment-with-actual-alpha-prefix-data-state A)
  ?projected adversary-initial-state
proof –
  have
    wp-event (checked-staged-security-experiment-with-data-state A)
    ?query adversary-initial-state =
    wp-event
    (checked-staged-security-experiment-with-actual-alpha-prefix-data-state A
     ≫ (λx. return (?project x)))
    ?query adversary-initial-state
  using
    checked-staged-security-experiment-with-actual-alpha-prefix-data-state-projection
    [of A]
  by simp
also have ... =
  wp-event
  (checked-staged-security-experiment-with-actual-alpha-prefix-data-state A)
  ?projected adversary-initial-state
  by (subst wp-event-bind-return-map)
  (simp add: staged-security-with-data-state-verifier-event-def)
finally show ?thesis
  by simp
qed
have event-le:
  wp-event
  (checked-staged-security-experiment-with-actual-alpha-prefix-data-state A)
  ?projected adversary-initial-state ≤
  wp-event
  (checked-staged-security-experiment-with-actual-alpha-prefix-data-state A)
  (λout. ?target out ∨ ?gap out)
  adversary-initial-state
proof (rule wp-event-mono-on-support)

```

```

fix out
assume support:
  out ∈
    set-dist
      (execute
        (checked-staged-security-experiment-with-actual-alpha-prefix-data-state
          A)
        adversary-initial-state)
  and projected: ?projected out
show ?target out ∨ ?gap out
proof (cases out)
  case None
  then show ?thesis
    using projected by simp
next
  case (Some packed)
  then obtain alpha-prefix alpha-prefix-state data attacker-state
    result final-state where out-eq:
    out =
      Some (((((alpha-prefix, alpha-prefix-state), data),
        attacker-state), result), final-state)
    by (cases packed, auto split: prod.splits)
  have bad:
    checked-staged-security-with-actual-alpha-prefix-verifier-event
      query-bad-with-aligned-transcript-partial-candidates out
    using projected unfolding out-eq
    by (simp add:
      checked-staged-security-with-actual-alpha-prefix-verifier-event-def
      staged-security-with-data-state-verifier-event-def Let-def)
  show ?thesis
    unfolding out-eq
    by (rule
      checked-staged-security-with-actual-alpha-prefix-aligned-transcript-query-bad-imp-single-round-header-target-on
      [OF wf controlled])
    (use support bad out-eq in simp-all)
qed
qed
have union-bound:
  wp-event
    (checked-staged-security-experiment-with-actual-alpha-prefix-data-state A)
    (λout. ?target out ∨ ?gap out)
    adversary-initial-state ≤
  wp-event
    (checked-staged-security-experiment-with-actual-alpha-prefix-data-state A)
    ?target adversary-initial-state +
  wp-event
    (checked-staged-security-experiment-with-actual-alpha-prefix-data-state A)
    ?gap adversary-initial-state
  by (rule wp-event-union-bound)

```

have *sum-bound*:
 wp-event
 (*checked-staged-security-experiment-with-actual-alpha-prefix-data-state* *A*)
 ?*target* *adversary-initial-state* +
 wp-event
 (*checked-staged-security-experiment-with-actual-alpha-prefix-data-state* *A*)
 ?*gap* *adversary-initial-state* $\leq T + \text{Gap}$
 by (rule *add-mono*[*OF target-bound gap-bound*])
show *?thesis*
 unfolding *projection*
 by (rule *order-trans*[*OF event-le order-trans*[*OF union-bound sum-bound*]])
qed

lemma *checked-staged-security-with-actual-alpha-prefix-candidate-pair-success-witness-gapE*:
assumes *i-bound*: $i < \text{rounds}$
and *support*:
 Some ((((*alpha-prefix*, *alpha-prefix-state*), *data*), *attacker-state*),
 result), *final-state*) $\in$
 set-dist
 (*execute*
 (*checked-staged-security-experiment-with-actual-alpha-prefix-data-state*
 A)
 adversary-initial-state)
and *success*:
 $\bigwedge \text{prefix } \text{prefix-state } \text{raw } \text{raw-state}.$
 Some ((((*prefix*, *prefix-state*), *raw*), *raw-state*),
 ((*data*, *attacker-state*), *result*)), *final-state*) $\in$
 set-dist
 (*execute*
 (*checked-staged-security-experiment-with-query-prefix-data-state*
 A i)
 adversary-initial-state) $\implies$
 sqp-alphas prefix = staged-alphas data $\implies$
 index (to-nat raw) \in
 query-sampling-success-space trace-table composition-table
 (*staged-alphas data*)
and *witness*:
 (*trace-openings*, *composition-openings*) $\in$
 query-header-supported-partial-opening-witnesses
 (*verifier-state-from-adversary attacker-state*
 (*staged-proof-transcript data*))
 (*staged-trace-root data*)
 (*staged-trace-fri-roots data*)
 (*staged-trace-final data*)
 (*staged-alphas data*)
 (*staged-degree data*)
 (*staged-composition-fri-roots data*)
 (*staged-composition-final data*)
and *trace-candidate*:

partial-trace-table-candidate trace-table
 ((*replicate rounds* []) [*i* := *trace-openings*])
and *comp-candidate*:
partial-composition-table-candidate composition-table
 ((*replicate rounds* []) [*i* := *composition-openings*])
and *notin*:
 $\bigwedge$ *prefix prefix-state raw raw-state*.
 Some ((((*prefix, prefix-state*), *raw*), *raw-state*),
 ((*data, attacker-state*), *result*)), *final-state*) $\in$
set-dist
 (*execute*
 (*checked-staged-security-experiment-with-query-prefix-data-state*
A i)
adversary-initial-state) $\implies$
sqp-alphas prefix = staged-alphas data $\implies$
 (*trace-table, composition-table*) $\notin$
query-header-supported-single-round-partial-table-candidates
prefix-state
 (*sqp-trace-root prefix*)
 (*sqp-trace-fri-roots prefix*)
 (*sqp-trace-final prefix*)
 (*sqp-alphas prefix*)
 (*sqp-degree prefix*)
 (*sqp-composition-fri-roots prefix*)
 (*sqp-composition-final prefix*)
i

obtains *prefix prefix-state raw raw-state* **where**
 Some ((((*prefix, prefix-state*), *raw*), *raw-state*),
 ((*data, attacker-state*), *result*)), *final-state*) $\in$
set-dist
 (*execute*
 (*checked-staged-security-experiment-with-query-prefix-data-state A i*)
adversary-initial-state)
checked-staged-security-with-query-prefix-single-round-candidate-pair-witness-gap-at
trace-openings composition-openings trace-table composition-table i
 (Some ((((*prefix, prefix-state*), *raw*), *raw-state*),
 ((*data, attacker-state*), *result*)), *final-state*))

proof –
from *checked-staged-security-with-actual-alpha-prefix-support-query-prefix-data-state-alphasE*
 [*OF i-bound support*]
obtain *prefix prefix-state raw raw-state* **where** *qsupport*:
 Some ((((*prefix, prefix-state*), *raw*), *raw-state*),
 ((*data, attacker-state*), *result*)), *final-state*) $\in$
set-dist
 (*execute*
 (*checked-staged-security-experiment-with-query-prefix-data-state A i*)
adversary-initial-state)
and *alphas-eq*: *sqp-alphas prefix = staged-alphas data*
by *blast*

```

have success-prefix:
  index (to-nat raw) ∈
    query-sampling-success-space trace-table composition-table
      (sqp-alphas prefix)
  using success[OF qsupport alphas-eq] alphas-eq by simp
have notin-prefix:
  (trace-table, composition-table) ∉
    query-header-supported-single-round-partial-table-candidates
      prefix-state
      (sqp-trace-root prefix)
      (sqp-trace-fri-roots prefix)
      (sqp-trace-final prefix)
      (sqp-alphas prefix)
      (sqp-degree prefix)
      (sqp-composition-fri-roots prefix)
      (sqp-composition-final prefix)
  i
  by (rule notin[OF qsupport alphas-eq])
have fixed-gap:
  checked-staged-security-with-query-prefix-single-round-candidate-pair-witness-gap-at
    trace-openings composition-openings trace-table composition-table i
    (Some ((((prefix, prefix-state), raw), raw-state),
      ((data, attacker-state), result)), final-state))
  by (rule checked-staged-security-with-query-prefix-single-round-candidate-pair-witness-gap-atI
    [OF success-prefix witness trace-candidate comp-candidate notin-prefix])
show ?thesis
  by (rule that[OF qsupport fixed-gap])
qed

lemma checked-staged-security-with-actual-alpha-prefix-candidate-pair-success-header-targetE:
assumes i-bound: i < rounds
and support:
  Some ((((alpha-prefix, alpha-prefix-state), data), attacker-state),
    result), final-state) ∈
  set-dist
    (execute
      (checked-staged-security-experiment-with-actual-alpha-prefix-data-state
        A)
      adversary-initial-state)
and success:
   $\bigwedge$ prefix prefix-state raw raw-state.
  Some ((((prefix, prefix-state), raw), raw-state),
    ((data, attacker-state), result)), final-state) ∈
  set-dist
    (execute
      (checked-staged-security-experiment-with-query-prefix-data-state
        A i)
      adversary-initial-state)  $\implies$ 
  sqp-alphas prefix = staged-alphas data  $\implies$ 

```

index (to-nat raw) ∈
query-sampling-success-space trace-table composition-table
(staged-alphas data)
and supported:
∧ prefix prefix-state raw raw-state.
Some (((((prefix, prefix-state), raw), raw-state),
((data, attacker-state), result)), final-state) ∈
set-dist
(execute
(checked-staged-security-experiment-with-query-prefix-data-state
A i)
adversary-initial-state) ⇒
sqp-alphas prefix = staged-alphas data ⇒
(trace-table, composition-table) ∈
query-header-supported-single-round-partial-table-candidates
prefix-state
(sqp-trace-root prefix)
(sqp-trace-fri-roots prefix)
(sqp-trace-final prefix)
(sqp-alphas prefix)
(sqp-degree prefix)
(sqp-composition-fri-roots prefix)
(sqp-composition-final prefix)
i
obtains *prefix prefix-state raw raw-state where*
Some (((((prefix, prefix-state), raw), raw-state),
((data, attacker-state), result)), final-state) ∈
set-dist
(execute
(checked-staged-security-experiment-with-query-prefix-data-state A i)
adversary-initial-state)
checked-staged-security-with-query-prefix-single-round-header-supported-query-target-hit-at
i
(Some (((((prefix, prefix-state), raw), raw-state),
((data, attacker-state), result)), final-state))
proof –
from *checked-staged-security-with-actual-alpha-prefix-support-query-prefix-data-state-alphasE*
[OF i-bound support]
obtain *prefix prefix-state raw raw-state where qsupport:*
Some (((((prefix, prefix-state), raw), raw-state),
((data, attacker-state), result)), final-state) ∈
set-dist
(execute
(checked-staged-security-experiment-with-query-prefix-data-state A i)
adversary-initial-state)
and *alphas-eq: sqp-alphas prefix = staged-alphas data*
by *blast*
have *success-prefix:*
index (to-nat raw) ∈


```

    query-sampling-success-space trace-table composition-table
    (sqp-alphas prefix)
using success[OF qsupport alphas-eq] alphas-eq by simp
have pair-target-hit:
  checked-staged-query-prefix-dynamic-index-hit
    (staged-query-prefix-candidate-pair-query-target-from-prefix
     trace-table composition-table)
    (Some (((prefix, prefix-state), raw), raw-state))
using success-prefix
unfolding
  checked-staged-query-prefix-dynamic-index-hit-def
  staged-query-prefix-candidate-pair-query-target-from-prefix-def
by simp
have supported-prefix:
  (trace-table, composition-table) ∈
    query-header-supported-single-round-partial-table-candidates
    prefix-state
    (sqp-trace-root prefix)
    (sqp-trace-fri-roots prefix)
    (sqp-trace-final prefix)
    (sqp-alphas prefix)
    (sqp-degree prefix)
    (sqp-composition-fri-roots prefix)
    (sqp-composition-final prefix)
    i
by (rule supported[OF qsupport alphas-eq])
have target-hit:
  checked-staged-query-prefix-dynamic-index-hit
    (staged-query-prefix-single-round-header-supported-query-target i)
    (Some (((prefix, prefix-state), raw), raw-state))
by (rule
    checked-staged-query-prefix-candidate-pair-hit-imp-single-round-header-target-hit
    [OF supported-prefix pair-target-hit])
have query-prefix-hit:
  checked-staged-security-with-query-prefix-single-round-header-supported-query-target-hit-at
    i
    (Some ((((((prefix, prefix-state), raw), raw-state),
      ((data, attacker-state), result)), final-state))
unfolding
  checked-staged-security-with-query-prefix-single-round-header-supported-query-target-hit-at-def
using target-hit by simp
show ?thesis
by (rule that[OF qsupport query-prefix-hit])
qed

end

end

```

theory *Soundness-Staged-Relevant-Drift*

imports

Soundness-Staged-Aligned-Current-Empty

Staged-Security-Experiment-Composition-Query-Aligned-Witness-Bridge

begin

Small downstream layer for the refined relevant-drift event. The purpose is to keep diagnostic and bridge lemmas out of the already large current-empty packaging theory.

context *soundness*

begin

definition *composition-randomization-bad-with-aligned-transcript-partial-candidates*

$:: (f, 'a) \text{ protocol-channel-scheme} \Rightarrow$
 $(\text{unit list} \times (f, 'a) \text{ protocol-channel-scheme}) \text{ option} \Rightarrow \text{bool}$

where

composition-randomization-bad-with-aligned-transcript-partial-candidates

$s \text{ out} \longleftrightarrow$

$(\exists \text{ fr } f\text{-fri-roots } f\text{-final as dg } \text{composition-fri-roots final}$
 $\text{query-idxs trace-openings composition-openings trace-table}$
 $\text{composition-table } f.$

$\text{accepted-with-partial-initial-openings-aligned-transcript-consistent } s$

$\text{out fr } f\text{-fri-roots } f\text{-final as dg } \text{composition-fri-roots final}$

$\text{query-idxs trace-openings composition-openings} \wedge$

$\text{partial-trace-table-candidate trace-table trace-openings} \wedge$

$\text{partial-composition-table-candidate composition-table}$

$\text{composition-openings} \wedge$

$\text{degree } f < \text{clength} \wedge$

$\text{trace-table} = \text{map } (\text{poly } f) \text{ eval-domain} \wedge$

$\text{violated-constraints } f \neq \{\}$ $\wedge$

$\text{composition-table-low-degree } \text{maxDegree } \text{composition-table} \wedge$

$\text{all-queries-consistent trace-table composition-table as} \wedge$

$\text{common-denominator-degree-bounds } f \text{ as} \wedge$

$\text{random-combination-common-denominator-hides-violations } f \text{ as})$

definition *soundness-bad-event-aligned-transcript-partial-candidate-with-aligned-randomization*

$:: (f, 'a) \text{ protocol-channel-scheme} \Rightarrow$
 $(\text{unit list} \times (f, 'a) \text{ protocol-channel-scheme}) \text{ option} \Rightarrow \text{bool}$

where

soundness-bad-event-aligned-transcript-partial-candidate-with-aligned-randomization

$s \text{ out} \longleftrightarrow$

$\text{composition-randomization-bad-with-aligned-transcript-partial-candidates}$

$s \text{ out} \vee$

$\text{trace-fri-bad-with-partial-candidates } s \text{ out} \vee$

$\text{composition-fri-bad-with-partial-candidates } s \text{ out} \vee$

$\text{query-bad-with-aligned-transcript-partial-candidates } s \text{ out}$

definition *soundness-bad-event-aligned-transcript-partial-candidate-with-aligned-randomization-and-merkle*

$:: (f, 'a) \text{ protocol-channel-scheme} \Rightarrow$

$(\text{unit list} \times ('f, 'a) \text{ protocol-channel-scheme}) \text{ option} \Rightarrow \text{bool}$
where
soundness-bad-event-aligned-transcript-partial-candidate-with-aligned-randomization-and-merkle
 $s \text{ out} \longleftrightarrow$
partial-merkle-inconsistency-bad $s \text{ out} \vee$
soundness-bad-event-aligned-transcript-partial-candidate-with-aligned-randomization
 $s \text{ out}$

lemma *accepted-aligned-transcript-partial-candidate-partition-aligned-randomization:*
assumes *false-statement:* $\neg \text{exists-valid-trace}$
and *partial:*
accepted-with-partial-initial-openings-aligned-transcript-consistent s
 $\text{out fr } f\text{-fri-roots } f\text{-final as dg composition-fri-roots final}$
 $\text{query-idxs trace-openings composition-openings}$
and *trace-candidate:*
partial-trace-table-candidate $\text{trace-table trace-openings}$
and *composition-candidate:*
partial-composition-table-candidate composition-table
 $\text{composition-openings}$
shows
trace-fri-bad-with-partial-candidates $s \text{ out} \vee$
composition-fri-bad-with-partial-candidates $s \text{ out} \vee$
query-bad-with-aligned-transcript-partial-candidates $s \text{ out} \vee$
composition-randomization-bad-with-aligned-transcript-partial-candidates
 $s \text{ out}$

proof (*cases trace-table-low-degree trace-table*)
case *False*
have *aligned:*
accepted-with-partial-initial-openings-aligned-consistent $s \text{ out fr}$
 $f\text{-fri-roots } f\text{-final as dg composition-fri-roots final query-idxs}$
 $\text{trace-openings composition-openings}$
by (*rule*
accepted-with-partial-initial-openings-aligned-transcript-consistent-imp-aligned
 $[\text{OF partial}]$)
have *trace-fri-bad-with-partial-candidates* $s \text{ out}$
unfolding *trace-fri-bad-with-partial-candidates-def*
using *accepted-with-partial-initial-openings-aligned-consistent-imp-unaligned*
 $[\text{OF aligned}]$
trace-candidate False **by** *blast*
then show *?thesis* **by** *blast*

next
case *True*
note *trace-low = True*
show *?thesis*
proof (*cases composition-table-low-degree maxDegree composition-table*)
case *False*
have *aligned:*
accepted-with-partial-initial-openings-aligned-consistent $s \text{ out fr}$
 $f\text{-fri-roots } f\text{-final as dg composition-fri-roots final query-idxs}$

```

    trace-openings composition-openings
  by (rule
    accepted-with-partial-initial-openings-aligned-transcript-consistent-imp-aligned
    [OF partial])
have composition-fri-bad-with-partial-candidates s out
  unfolding composition-fri-bad-with-partial-candidates-def
  using accepted-with-partial-initial-openings-aligned-consistent-imp-unaligned
    [OF aligned]
    trace-candidate composition-candidate trace-low False
  by blast
then show ?thesis by blast
next
case True
note composition-low = True
show ?thesis
proof (cases all-queries-consistent trace-table composition-table as)
  case False
  have query-bad-with-aligned-transcript-partial-candidates s out
    unfolding query-bad-with-aligned-transcript-partial-candidates-def
    using partial trace-candidate composition-candidate trace-low
      composition-low False
    by blast
  then show ?thesis by blast
next
case True
note all-queries = True
from trace-low obtain f where
  deg-f: degree f < clength
  and trace-table-eq: trace-table = map (poly f) eval-domain
  unfolding trace-table-low-degree-def by blast
have violated: violated-constraints f ≠ {}
  by (rule false-statement-violated-constraints
    [OF false-statement deg-f])
have aligned:
  accepted-with-partial-initial-openings-aligned-consistent s out fr
    f-fri-roots f-final as dg composition-fri-roots final query-idxs
    trace-openings composition-openings
  by (rule
    accepted-with-partial-initial-openings-aligned-transcript-consistent-imp-aligned
    [OF partial])
have unaligned:
  accepted-with-partial-initial-openings s out fr f-fri-roots f-final
    as dg composition-fri-roots final query-idxs trace-openings
    query-idxs composition-openings
  by (rule
    accepted-with-partial-initial-openings-aligned-consistent-imp-unaligned
    [OF aligned])
have len-as: length as = length spec
  by (rule accepted-with-partial-initial-openings-alpha-length

```

```

    [OF unaligned])
  have bounds: common-denominator-degree-bounds f as
    by (rule common-denominator-degree-bounds-from-spec-degree-wellformed
      [OF spec-degree-wellformed-from-spec-query-margin deg-f])
  have hides:
    random-combination-common-denominator-hides-violations f as
    by (rule random-combination-common-denominator-hides-from-queries
      [OF len-as false-statement deg-f trace-table-eq composition-low
        all-queries bounds])
  have
    composition-randomization-bad-with-aligned-transcript-partial-candidates
      s out
    unfolding
      composition-randomization-bad-with-aligned-transcript-partial-candidates-def
    using partial trace-candidate composition-candidate deg-f
      trace-table-eq violated composition-low all-queries bounds hides
    by blast
  then show ?thesis by blast
qed
qed
qed

lemma accepted-aligned-transcript-partial-candidate-partition-with-aligned-randomization-event:
  assumes false-statement:  $\neg$  exists-valid-trace
  and partial:
    accepted-with-partial-initial-openings-aligned-transcript-consistent s
      out fr f-fri-roots f-final as dg composition-fri-roots final
      query-idxs trace-openings composition-openings
  and trace-candidate:
    partial-trace-table-candidate trace-table trace-openings
  and composition-candidate:
    partial-composition-table-candidate composition-table
      composition-openings
  shows
    soundness-bad-event-aligned-transcript-partial-candidate-with-aligned-randomization
      s out
  using
    accepted-aligned-transcript-partial-candidate-partition-aligned-randomization
      [OF false-statement partial trace-candidate composition-candidate]
  unfolding
    soundness-bad-event-aligned-transcript-partial-candidate-with-aligned-randomization-def
  by blast

lemma checked-staged-security-with-data-state-aligned-transcript-partial-candidate-aligned-randomization-union-bound:
  fixes composition-error trace-fri-error' composition-fri-error'
  query-error :: prob
  assumes comp-bound:
    wp-event (checked-staged-security-experiment-with-data-state A)
      (staged-security-with-data-state-verifier-event

```

```

    composition-randomization-bad-with-aligned-transcript-partial-candidates)
  adversary-initial-state ≤ composition-error
and trace-fri-bound:
  wp-event (checked-staged-security-experiment-with-data-state A)
    (staged-security-with-data-state-verifier-event
     trace-fri-bad-with-partial-candidates)
  adversary-initial-state ≤ trace-fri-error'
and comp-fri-bound:
  wp-event (checked-staged-security-experiment-with-data-state A)
    (staged-security-with-data-state-verifier-event
     composition-fri-bad-with-partial-candidates)
  adversary-initial-state ≤ composition-fri-error'
and query-bound:
  wp-event (checked-staged-security-experiment-with-data-state A)
    (staged-security-with-data-state-verifier-event
     query-bad-with-aligned-transcript-partial-candidates)
  adversary-initial-state ≤ query-error
shows
  wp-event (checked-staged-security-experiment-with-data-state A)
    (staged-security-with-data-state-verifier-event
     soundness-bad-event-aligned-transcript-partial-candidate-with-aligned-randomization)
  adversary-initial-state ≤
    composition-error + trace-fri-error' + composition-fri-error' +
    query-error
proof –
  let ?M = checked-staged-security-experiment-with-data-state A
  let ?E = staged-security-with-data-state-verifier-event
  let ?comp =
    ?E composition-randomization-bad-with-aligned-transcript-partial-candidates
  let ?trace = ?E trace-fri-bad-with-partial-candidates
  let ?comp-fri = ?E composition-fri-bad-with-partial-candidates
  let ?query = ?E query-bad-with-aligned-transcript-partial-candidates
  let ?partial =
    ?E soundness-bad-event-aligned-transcript-partial-candidate-with-aligned-randomization
  have wp-event ?M ?partial adversary-initial-state ≤
    wp-event ?M
      (λout. ?comp out ∨ ?trace out ∨ ?comp-fri out ∨
        ?query out)
    adversary-initial-state
  by (rule wp-event-mono)
    (auto simp: staged-security-with-data-state-verifier-event-def
      soundness-bad-event-aligned-transcript-partial-candidate-with-aligned-randomization-def
      split: option.splits prod.splits)
  also have ... ≤
    wp-event ?M ?comp adversary-initial-state +
    wp-event ?M ?trace adversary-initial-state +
    wp-event ?M ?comp-fri adversary-initial-state +
    wp-event ?M ?query adversary-initial-state
  by (rule wp-event-union-bound4)

```

also have ... $\leq$
 $\text{composition-error} + \text{trace-fri-error}' + \text{composition-fri-error}' +$
 query-error
by (intro add-mono comp-bound trace-fri-bound comp-fri-bound
 query-bound)
finally show ?thesis .
qed

lemma *checked-staged-security-with-data-state-aligned-transcript-partial-candidate-aligned-randomization-with-merkle*

fixes merkle-error composition-error trace-fri-error'

$\text{composition-fri-error}' \text{ query-error} :: \text{prob}$

assumes merkle-bound:

$\text{wp-event } (\text{checked-staged-security-experiment-with-data-state } A)$
 $(\text{staged-security-with-data-state-verifier-event}$
 $\text{partial-merkle-inconsistency-bad})$
 $\text{adversary-initial-state} \leq \text{merkle-error}$

and comp-bound:

$\text{wp-event } (\text{checked-staged-security-experiment-with-data-state } A)$
 $(\text{staged-security-with-data-state-verifier-event}$
 $\text{composition-randomization-bad-with-aligned-transcript-partial-candidates})$
 $\text{adversary-initial-state} \leq \text{composition-error}$

and trace-fri-bound:

$\text{wp-event } (\text{checked-staged-security-experiment-with-data-state } A)$
 $(\text{staged-security-with-data-state-verifier-event}$
 $\text{trace-fri-bad-with-partial-candidates})$
 $\text{adversary-initial-state} \leq \text{trace-fri-error}'$

and comp-fri-bound:

$\text{wp-event } (\text{checked-staged-security-experiment-with-data-state } A)$
 $(\text{staged-security-with-data-state-verifier-event}$
 $\text{composition-fri-bad-with-partial-candidates})$
 $\text{adversary-initial-state} \leq \text{composition-fri-error}'$

and query-bound:

$\text{wp-event } (\text{checked-staged-security-experiment-with-data-state } A)$
 $(\text{staged-security-with-data-state-verifier-event}$
 $\text{query-bad-with-aligned-transcript-partial-candidates})$
 $\text{adversary-initial-state} \leq \text{query-error}$

shows

$\text{wp-event } (\text{checked-staged-security-experiment-with-data-state } A)$
 $(\text{staged-security-with-data-state-verifier-event}$
 $\text{soundness-bad-event-aligned-transcript-partial-candidate-with-aligned-randomization-and-merkle})$
 $\text{adversary-initial-state} \leq$
 $\text{merkle-error} + \text{composition-error} + \text{trace-fri-error}' +$
 $\text{composition-fri-error}' + \text{query-error}$

proof –

have partial-bound:

$\text{wp-event } (\text{checked-staged-security-experiment-with-data-state } A)$
 $(\text{staged-security-with-data-state-verifier-event}$
 $\text{soundness-bad-event-aligned-transcript-partial-candidate-with-aligned-randomization})$
 $\text{adversary-initial-state} \leq$

```

composition-error + trace-fri-error' + composition-fri-error' +
query-error
by (rule
  checked-staged-security-with-data-state-aligned-transcript-partial-candidate-aligned-randomization-union-bound
  [OF comp-bound trace-fri-bound comp-fri-bound query-bound])
have wp-event (checked-staged-security-experiment-with-data-state A)
  (staged-security-with-data-state-verifier-event
   soundness-bad-event-aligned-transcript-partial-candidate-with-aligned-randomization-and-merkle)
adversary-initial-state ≤
wp-event (checked-staged-security-experiment-with-data-state A)
  (staged-security-with-data-state-verifier-event
   partial-merkle-inconsistency-bad)
adversary-initial-state +
wp-event (checked-staged-security-experiment-with-data-state A)
  (staged-security-with-data-state-verifier-event
   soundness-bad-event-aligned-transcript-partial-candidate-with-aligned-randomization)
adversary-initial-state
proof -
have event-le:
wp-event (checked-staged-security-experiment-with-data-state A)
  (staged-security-with-data-state-verifier-event
   soundness-bad-event-aligned-transcript-partial-candidate-with-aligned-randomization-and-merkle)
adversary-initial-state ≤
wp-event (checked-staged-security-experiment-with-data-state A)
  (λout.
    staged-security-with-data-state-verifier-event
    partial-merkle-inconsistency-bad out ∨
    staged-security-with-data-state-verifier-event
    soundness-bad-event-aligned-transcript-partial-candidate-with-aligned-randomization
    out)
  adversary-initial-state
by (rule wp-event-mono)
  (auto simp: staged-security-with-data-state-verifier-event-def
   soundness-bad-event-aligned-transcript-partial-candidate-with-aligned-randomization-and-merkle-def
   split: option.splits prod.splits)
also have ... ≤
wp-event (checked-staged-security-experiment-with-data-state A)
  (staged-security-with-data-state-verifier-event
   partial-merkle-inconsistency-bad)
adversary-initial-state +
wp-event (checked-staged-security-experiment-with-data-state A)
  (staged-security-with-data-state-verifier-event
   soundness-bad-event-aligned-transcript-partial-candidate-with-aligned-randomization)
adversary-initial-state
by (rule wp-event-union-bound)
finally show ?thesis .
qed
also have ... ≤
merkle-error +

```



```

    (composition-error + trace-fri-error' + composition-fri-error' +
      query-error)
  by (intro add-mono merkle-bound partial-bound)
finally show ?thesis
  by (simp add: algebra-simps)
qed

```

lemma *accepted-partition-soundness-bad-event-aligned-transcript-partial-candidate-with-aligned-randomization-and-*

assumes *false-statement*: $\neg$ *exists-valid-trace*

and *supp*: $out \in \text{set-dist } (\text{execute verify-monad } s)$

and *acc*: *accepted out*

and *header*:

verifier-header-transcript s fr f-fri-roots f-final as dg

composition-fri-roots final rest

and *comp-nonempty*: *composition-fri-roots* $\neq []$

shows

soundness-bad-event-aligned-transcript-partial-candidate-with-aligned-randomization-and-merkle
s out

proof –

from *acc* **obtain** *result final-state* **where** *out-eq*:

out = Some (result, final-state)

unfolding *accepted-def* **by** (*cases out*) *auto*

from *verify-monad-supplied-header-transcript-shape*

[*OF supp[unfolded out-eq] header*]

obtain *query-idxs raw-idxs query-chunks* **where** *shape*:

accepted-transcript-shape s (Some (result, final-state)) as query-idxs

and *len-raw*: *length raw-idxs = rounds*

and *query-idxs-eq*:

query-idxs = map ($\lambda raw. \text{index } (to\text{-nat } raw)$) raw-idxs

and *len-query-chunks*: *length query-chunks = rounds*

and *transcript-rest*:

List.concat query-chunks @ PTranscript final-state = rest

and *query-chunk-header*:

$\bigwedge i. i < \text{rounds} \implies$

verifier-query-round-chunk (query-idxs ! i)

f-fri-roots composition-fri-roots (query-chunks ! i)

and *replay-lookup-header*:

$\bigwedge i. i < \text{rounds} \implies$

fmlookup (HashMap final-state)

(QueryIndexChallenge (PQueryCounter s + i)

(state-after-query-chunks

(verifier-header-state s fr f-fri-roots f-final as dg

composition-fri-roots final)

query-chunks i)) =

Some (raw-idxs ! i)

by *blast*

from *verify-monad-supplied-header-query-rounds*

[*OF supp[unfolded out-eq] header*]

obtain *f-fl fl query-state* **where** *f-roots*:

```

    map snd f-fl = f-fri-roots
and comp-roots: map snd fl = composition-fri-roots
and query-transcript-state: PTranscript query-state = rest
and query-hash-state:
    PState query-state =
        verifier-header-state s fr f-fri-roots f-final as dg
        composition-fri-roots final
and query-counter: PQueryCounter query-state = PQueryCounter s
and query-out:
    Some (result, final-state) ∈
        set-dist
        (execute
            (ntimes
                (verifier-query-round-program fr f-fl f-final as fl final)
                rounds)
            query-state)
by blast
have fl-nonempty: fl ≠ []
    using comp-nonempty comp-roots by (cases fl) simp-all
then obtain b composition-root fl-tail where fl-eq:
    fl = (b, composition-root) # fl-tail
    by (cases fl) auto
have transcript-query:
    PTranscript query-state =
        List.concat query-chunks @ PTranscript final-state
    using query-transcript-state transcript-rest by simp
have query-chunk:
    ∧i. i < rounds ⇒
        verifier-query-round-chunk (query-idxs ! i)
        (map snd f-fl) (map snd fl) (query-chunks ! i)
    using query-chunk-header f-roots comp-roots by simp
have replay-lookup:
    ∧i. i < rounds ⇒
        fmlookup (HashMap final-state)
        (QueryIndexChallenge (PQueryCounter query-state + i)
            (state-after-query-chunks (PState query-state) query-chunks i)) =
        Some (raw-idxs ! i)
    using replay-lookup-header query-hash-state query-counter by simp
from
    query-rounds-replay-accepted-with-partial-initial-openings-aligned-transcript-consistent
    [OF header comp-nonempty fl-eq f-roots comp-roots
        query-transcript-state query-hash-state query-counter query-out
        len-raw query-idxs-eq len-query-chunks transcript-query query-chunk
        replay-lookup]
obtain trace-openings composition-openings where partial:
    accepted-with-partial-initial-openings-aligned-transcript-consistent s
    (Some (result, final-state)) fr f-fri-roots f-final as dg
    composition-fri-roots final query-idxs trace-openings
    composition-openings

```

```

    by blast
show ?thesis
proof (cases
  partial-merkle-inconsistency-bad s (Some (result, final-state)))
case True
then show ?thesis
  unfolding out-eq
  soundness-bad-event-aligned-transcript-partial-candidate-with-aligned-randomization-and-merkle-def
  by simp
next
case False
have aligned0:
  accepted-with-partial-initial-openings-aligned s
  (Some (result, final-state)) fr f-fri-roots f-final as dg
  composition-fri-roots final query-idxs trace-openings
  composition-openings
using partial
by (blast dest:
  accepted-with-partial-initial-openings-aligned-transcript-consistent-imp-aligned
  accepted-with-partial-initial-openings-aligned-consistent-imp-aligned)
obtain trace-table composition-table where trace-candidate:
  partial-trace-table-candidate trace-table trace-openings
and composition-candidate:
  partial-composition-table-candidate composition-table
  composition-openings
using
  accepted-with-partial-initial-openings-aligned-candidates-if-no-partial-merkle-bad
  [OF aligned0 False]
by blast
have bad:
  soundness-bad-event-aligned-transcript-partial-candidate-with-aligned-randomization
  s (Some (result, final-state))
by (rule
  accepted-aligned-transcript-partial-candidate-partition-with-aligned-randomization-event
  [OF false-statement partial trace-candidate composition-candidate])
then show ?thesis
  unfolding out-eq
  by (simp add:
    soundness-bad-event-aligned-transcript-partial-candidate-with-aligned-randomization-and-merkle-def)
qed
qed

lemma accepted-partition-soundness-bad-event-aligned-transcript-partial-candidate-with-aligned-randomization-and-n
assumes false-statement:  $\neg$  exists-valid-trace
and supp:  $out \in \text{set-dist } (\text{execute verify-monad } s)$ 
and acc: accepted out
and header:
  verifier-header-transcript s fr f-fri-roots f-final as dg
  composition-fri-roots final rest

```

```

shows
  soundness-bad-event-aligned-transcript-partial-candidate-with-aligned-randomization-and-merkle
    s out  $\vee$ 
    soundness-bad-event-partial-candidate-empty-header s out
proof (cases composition-fri-roots = [])
case False
then have comp-nonempty: composition-fri-roots  $\neq$  []
  by simp
have
  soundness-bad-event-aligned-transcript-partial-candidate-with-aligned-randomization-and-merkle
    s out
  by (rule
    accepted-partition-soundness-bad-event-aligned-transcript-partial-candidate-with-aligned-randomization-and-merkle
      [OF false-statement supp acc header comp-nonempty])
then show ?thesis by simp
next
case True
note comp-empty = True
from acc obtain result final-state where out-eq:
  out = Some (result, final-state)
  unfolding accepted-def by (cases out) auto
show ?thesis
proof (cases
  partial-merkle-inconsistency-bad s (Some (result, final-state)))
case True
then have
  soundness-bad-event-aligned-transcript-partial-candidate-with-aligned-randomization-and-merkle
    s out
  unfolding out-eq
  soundness-bad-event-aligned-transcript-partial-candidate-with-aligned-randomization-and-merkle-def
  by simp
then show ?thesis by simp
next
case False
from verify-monad-accepted-with-partial-trace-openings-for-header
  [OF supp[unfolded out-eq] header]
obtain trace-query-idxs trace-openings where trace-partial:
  accepted-with-partial-trace-openings s
  (Some (result, final-state)) fr trace-query-idxs trace-openings
by blast
obtain trace-table where trace-candidate:
  partial-trace-table-candidate trace-table trace-openings
using accepted-with-partial-trace-openings-candidate-if-no-partial-merkle-bad
  [OF trace-partial False]
by blast
have accepted-out: accepted (Some (result, final-state))
  by (rule accepted-with-partial-trace-openings-imp-accepted
    [OF trace-partial])
let ?composition-table = replicate (scale * clength) final

```

```

have empty-partial:
  accepted-with-empty-composition-header-candidates s
    (Some (result, final-state)) fr f-fri-roots f-final as dg final
      trace-query-idxs trace-openings trace-table ?composition-table
  unfolding accepted-with-empty-composition-header-candidates-def
  by (intro conjI exI[of - rest])
    (use accepted-out header comp-empty trace-partial trace-candidate
      in simp-all)
have soundness-bad-event-partial-candidate-empty-header s
  (Some (result, final-state))
  by (rule accepted-empty-composition-header-candidate-partition
    [OF false-statement empty-partial])
then show ?thesis
  unfolding out-eq by simp
qed
qed

lemma checked-staged-soundness-from-aligned-transcript-partial-candidate-aligned-randomization-and-empty-header-
fixes partial-candidate-error empty-header-error :: prob
assumes false-statement: ¬ exists-valid-trace
and partial-candidate-bound:
  wp-event (checked-staged-security-experiment-with-data-state A)
    (staged-security-with-data-state-verifier-event
      soundness-bad-event-aligned-transcript-partial-candidate-with-aligned-randomization-and-merkle
      adversary-initial-state ≤ partial-candidate-error)
and empty-header-bound:
  wp-event (checked-staged-security-experiment-with-data-state A)
    (staged-security-with-data-state-verifier-event
      soundness-bad-event-partial-candidate-empty-header
      adversary-initial-state ≤ empty-header-error)
shows checked-staged-adversary-acceptance-probability A ≤
  partial-candidate-error + empty-header-error
proof -
let ?M = checked-staged-security-experiment-with-data-state A
let ?E = staged-security-with-data-state-verifier-event
let ?partial =
  ?E soundness-bad-event-aligned-transcript-partial-candidate-with-aligned-randomization-and-merkle
let ?empty = ?E soundness-bad-event-partial-candidate-empty-header
let ?bad = λout. ?partial out ∨ ?empty out
have accepted-bad:
  wp-event ?M accepted adversary-initial-state ≤
  wp-event ?M ?bad adversary-initial-state
proof (rule wp-event-mono-on-support)
fix out
assume support: out ∈ set-dist (execute ?M adversary-initial-state)
and acc: accepted out
show ?bad out
proof (cases out)
case None

```

```

then show ?thesis
  using acc unfolding accepted-def by simp
next
case (Some packed)
then obtain data attacker-state result final-state where out-eq:
  out = Some (((data, attacker-state), result), final-state)
  by (cases packed, auto split: prod.splits)
let ?s =
  verifier-state-from-adversary attacker-state
  (staged-proof-transcript data)
have verifier:
  Some (result, final-state) ∈ set-dist (execute verify-monad ?s)
  using checked-staged-security-experiment-with-data-state-outcomeE
  [OF support[unfolded out-eq]]
  by blast
have verifier-acc: accepted (Some (result, final-state))
  unfolding accepted-def by simp
from verify-monad-accepted-transcript-shape [OF verifier]
obtain alphas query-idxs where shape:
  accepted-transcript-shape ?s (Some (result, final-state))
  alphas query-idxs
  by blast
from shape obtain result' final-state' fr f-fri-roots f-final dg
  composition-fri-roots final rest where
  header:
  verifier-header-transcript ?s fr f-fri-roots f-final alphas dg
  composition-fri-roots final rest
  by (elim accepted-transcript-shape-header-query-extraction)
have local-bad:
  soundness-bad-event-aligned-transcript-partial-candidate-with-aligned-randomization-and-merkle
  ?s (Some (result, final-state)) ∨
  soundness-bad-event-partial-candidate-empty-header ?s
  (Some (result, final-state))
  by (rule
  accepted-partition-soundness-bad-event-aligned-transcript-partial-candidate-with-aligned-randomization-and-m
  [OF false-statement verifier verifier-acc header])
then show ?thesis
  unfolding out-eq staged-security-with-data-state-verifier-event-def
  by simp
qed
qed
have bad-bound:
  wp-event ?M ?bad adversary-initial-state ≤
  partial-candidate-error + empty-header-error
proof –
have wp-event ?M ?bad adversary-initial-state ≤
  wp-event ?M ?partial adversary-initial-state +
  wp-event ?M ?empty adversary-initial-state
  by (rule wp-event-union-bound)

```

```

also have ...  $\leq$  partial-candidate-error + empty-header-error
  by (intro add-mono partial-candidate-bound empty-header-bound)
finally show ?thesis .
qed
have staged-bound:
  wp-event (checked-staged-security-experiment A) accepted
    adversary-initial-state  $\leq$ 
      partial-candidate-error + empty-header-error
  unfolding checked-staged-security-experiment-acceptance-with-data-state
  by (rule order-trans[OF accepted-bad bad-bound])
show ?thesis
proof –
  have
    wp-event (checked-staged-security-experiment A)
      ( $\lambda$ out.  $\neg$  Option.is-none out) adversary-initial-state =
      wp-event (checked-staged-security-experiment A) accepted
        adversary-initial-state
    unfolding wp-event-def accepted-def
    by (rule arg-cong[where
      f= $\lambda Q.$  wp (checked-staged-security-experiment A) Q
      adversary-initial-state])
      (rule ext, simp)
  then show ?thesis
    unfolding checked-staged-adversary-acceptance-probability-def
    using staged-bound by simp
  qed
qed

lemma checked-staged-soundness-from-aligned-transcript-partial-candidate-aligned-randomization-components-and-en
fixes merkle-error composition-error trace-fri-error'
  composition-fri-error' query-error empty-header-error :: prob
assumes false-statement:  $\neg$  exists-valid-trace
and merkle-bound:
  wp-event (checked-staged-security-experiment-with-data-state A)
    (staged-security-with-data-state-verifier-event
      partial-merkle-inconsistency-bad)
    adversary-initial-state  $\leq$  merkle-error
and comp-bound:
  wp-event (checked-staged-security-experiment-with-data-state A)
    (staged-security-with-data-state-verifier-event
      composition-randomization-bad-with-aligned-transcript-partial-candidates)
    adversary-initial-state  $\leq$  composition-error
and trace-fri-bound:
  wp-event (checked-staged-security-experiment-with-data-state A)
    (staged-security-with-data-state-verifier-event
      trace-fri-bad-with-partial-candidates)
    adversary-initial-state  $\leq$  trace-fri-error'
and comp-fri-bound:
  wp-event (checked-staged-security-experiment-with-data-state A)

```

*(staged-security-with-data-state-verifier-event
composition-fri-bad-with-partial-candidates)*
adversary-initial-state $\leq$ *composition-fri-error'*
and *query-bound*:
wp-event (checked-staged-security-experiment-with-data-state A)
*(staged-security-with-data-state-verifier-event
query-bad-with-aligned-transcript-partial-candidates)*
adversary-initial-state $\leq$ *query-error*
and *empty-header-bound*:
wp-event (checked-staged-security-experiment-with-data-state A)
*(staged-security-with-data-state-verifier-event
soundness-bad-event-partial-candidate-empty-header)*
adversary-initial-state $\leq$ *empty-header-error*
shows *checked-staged-adversary-acceptance-probability A* $\leq$
merkle-error + composition-error + trace-fri-error' +
composition-fri-error' + query-error + empty-header-error
proof –
have *partial-candidate-bound*:
wp-event (checked-staged-security-experiment-with-data-state A)
*(staged-security-with-data-state-verifier-event
soundness-bad-event-aligned-transcript-partial-candidate-with-aligned-randomization-and-merkle)*
adversary-initial-state $\leq$
merkle-error + composition-error + trace-fri-error' +
composition-fri-error' + query-error
by (rule
checked-staged-security-with-data-state-aligned-transcript-partial-candidate-aligned-randomization-with-merkle-u
[OF *merkle-bound comp-bound trace-fri-bound comp-fri-bound*
query-bound])
have *bound*:
checked-staged-adversary-acceptance-probability A $\leq$
(merkle-error + composition-error + trace-fri-error' +
composition-fri-error' + query-error) + empty-header-error
by (rule
checked-staged-soundness-from-aligned-transcript-partial-candidate-aligned-randomization-and-empty-header-bou
[OF *false-statement partial-candidate-bound empty-header-bound*])
then show *?thesis*
by (*simp add: algebra-simps*)
qed

lemma *accepted-with-partial-initial-openings-aligned-transcript-consistent-imp-current-query-on-support*:

assumes *wf: staged-budget-wellformed budgets*

and *controlled: staged-adversary-controlled budgets A*

and *support*:

Some (((data, attacker-state), result), final-state) $\in$
set-dist

(execute

(checked-staged-security-experiment-with-data-state A)

adversary-initial-state)

and *partial*:


```

    accepted-with-partial-initial-openings-aligned-transcript-consistent
      (verifier-state-from-adversary attacker-state
        (staged-proof-transcript data))
      (Some (result, final-state)) fr f-fri-roots f-final as dg
      composition-fri-roots final query-idxs trace-openings
      composition-openings
  shows
    staged-security-with-data-state-current-query-partial-opening-hit
      (Some (((data, attacker-state), result), final-state))
proof -
  let ?s =
    verifier-state-from-adversary attacker-state
      (staged-proof-transcript data)
  from checked-staged-security-experiment-with-data-state-outcomeE
    [OF support]
  obtain builder verifier where builder:
    Some (data, attacker-state) ∈
      set-dist
        (execute (checked-staged-transcript-program A)
          adversary-initial-state)
  and verifier:
    Some (result, final-state) ∈ set-dist (execute verify-monad ?s)
  by blast
  have staged-header:
    verifier-header-transcript ?s
      (staged-trace-root data)
      (staged-trace-fri-roots data)
      (staged-trace-final data)
      (staged-alphas data)
      (staged-degree data)
      (staged-composition-fri-roots data)
      (staged-composition-final data)
      (List.concat (staged-query-chunks data))
  by (rule checked-staged-transcript-program-outcome-header-transcript
    [OF wf controlled builder])
  have shape:
    accepted-transcript-shape ?s (Some (result, final-state)) as
      query-idxs
  by (rule
    accepted-with-partial-initial-openings-aligned-transcript-consistent-imp-shape
    [OF partial])
  from checked-staged-security-with-data-state-accepted-shape-query-chunks
    [OF wf controlled support shape]
  obtain raw-idxs where as-eq: as = staged-alphas data
  and len-raw: length raw-idxs = rounds
  and query-idxs-eq:
    query-idxs = map (λraw. index (to-nat raw)) raw-idxs
  and lookup:
    ∀ i < rounds.

```

```

    fmlookup (HashMap final-state)
      (QueryIndexChallenge i
        (state-after-query-chunks
          (staged-query-start-hash data)
          (staged-query-chunks data) i)) =
        Some (raw-idxs ! i)
  by blast
have aligned:
  accepted-with-partial-initial-openings-aligned-consistent ?s
    (Some (result, final-state)) fr f-fri-roots f-final as dg
    composition-fri-roots final query-idxs trace-openings
    composition-openings
  by (rule
    accepted-with-partial-initial-openings-aligned-transcript-consistent-imp-aligned
      [OF partial])
have aligned0:
  accepted-with-partial-initial-openings-aligned ?s
    (Some (result, final-state)) fr f-fri-roots f-final as dg
    composition-fri-roots final query-idxs trace-openings
    composition-openings
  by (rule
    accepted-with-partial-initial-openings-aligned-consistent-imp-aligned
      [OF aligned])
from accepted-with-partial-initial-openings-aligned-shapes[OF aligned0]
obtain rest where header:
  verifier-header-transcript ?s fr f-fri-roots f-final as dg
  composition-fri-roots final rest
  by blast
have comp-nonempty: composition-fri-roots ≠ []
  using accepted-with-partial-initial-openings-aligned-shapes(1)
    [OF aligned0] .
have trace-partial:
  accepted-with-partial-trace-openings ?s
    (Some (result, final-state)) fr query-idxs trace-openings
  using accepted-with-partial-initial-openings-aligned-shapes(3)
    [OF aligned0] .
have comp-partial:
  accepted-with-partial-composition-openings ?s
    (Some (result, final-state)) (hd composition-fri-roots) query-idxs
    composition-openings
  using accepted-with-partial-initial-openings-aligned-shapes(4)
    [OF aligned0] .
have header-eq:
  fr = staged-trace-root data ∧
  f-fri-roots = staged-trace-fri-roots data ∧
  f-final = staged-trace-final data ∧
  as = staged-alphas data ∧
  dg = staged-degree data ∧
  composition-fri-roots = staged-composition-fri-roots data ∧

```

```

    final = staged-composition-final data ∧
    rest = List.concat (staged-query-chunks data)
using verifier-header-transcript-unique[OF header staged-header]
by simp
let ?i = 0
have i-bound: ?i < rounds
  using rounds-positive by simp
have round-consistent:
  partial-query-round-consistent trace-openings composition-openings as
    ?i (query-idxs ! ?i)
  using aligned i-bound
  unfolding accepted-with-partial-initial-openings-aligned-consistent-def
  by simp
have opening-consistent:
  partial-query-openings-consistent (trace-openings ! ?i)
    (composition-openings ! ?i) as (query-idxs ! ?i)
  using round-consistent
  unfolding partial-query-round-consistent-def
    partial-query-openings-consistent-def
  by simp
have trace-auth:
  partial-authenticated-table (staged-trace-root data)
    (scale * clength) (trace-openings ! ?i) final-state
  using trace-partial i-bound header-eq
  unfolding accepted-with-partial-trace-openings-def by blast
have comp-auth:
  partial-authenticated-table (hd (staged-composition-fri-roots data))
    (scale * clength) (composition-openings ! ?i) final-state
  using comp-partial i-bound header-eq
  unfolding accepted-with-partial-composition-openings-def by blast
have final-lookup:
  fmlookup (HashMap final-state)
    (QueryIndexChallenge ?i
      (state-after-query-chunks
        (staged-query-start-hash data)
        (staged-query-chunks data) ?i)) =
    Some (raw-idxs ! ?i)
  using lookup i-bound by simp
have idx-raw:
  query-idxs ! ?i = index (to-nat (raw-idxs ! ?i))
  using query-idxs-eq len-raw i-bound by simp
have hit-at:
  staged-security-with-data-state-current-query-partial-opening-hit-at ?i
    (Some (((data, attacker-state), result), final-state))
  unfolding staged-security-with-data-state-current-query-partial-opening-hit-at-def
    Let-def
  apply simp
by (intro exI[of - raw-idxs ! ?i]
    exI[of - trace-openings ! ?i])

```

```

    exI[of - composition-openings ! ?i] exI[of - rest] conjI)
  (use i-bound final-lookup comp-nonempty verifier trace-auth comp-auth
    staged-header opening-consistent idx-raw header-eq as-eq in simp-all)
show ?thesis
by (rule staged-security-with-data-state-current-query-partial-opening-hitI
    [OF i-bound hit-at])
qed

lemma checked-staged-security-with-data-state-aligned-partial-candidate-randomization-imp-current-query-on-support
assumes wf: staged-budget-wellformed budgets
and controlled: staged-adversary-controlled budgets A
and support:
  out ∈
  set-dist
  (execute
    (checked-staged-security-experiment-with-data-state A)
    adversary-initial-state)
and bad:
  staged-security-with-data-state-verifier-event
  composition-randomization-bad-with-aligned-transcript-partial-candidates
  out
shows staged-security-with-data-state-current-query-partial-opening-hit out
proof (cases out)
case None
then show ?thesis
  using bad
  unfolding staged-security-with-data-state-verifier-event-def
    composition-randomization-bad-with-aligned-transcript-partial-candidates-def
  by simp
next
case (Some packed)
then obtain data attacker-state result final-state where out-eq:
  out = Some (((data, attacker-state), result), final-state)
  by (cases packed, auto split: prod.splits)
let ?s =
  verifier-state-from-adversary attacker-state
  (staged-proof-transcript data)
have bad-some:
  composition-randomization-bad-with-aligned-transcript-partial-candidates
  ?s (Some (result, final-state))
using bad
unfolding out-eq staged-security-with-data-state-verifier-event-def
  Let-def
by simp
obtain fr f-fri-roots f-final as dg composition-fri-roots final
  query-idxs trace-openings composition-openings trace-table
  composition-table f
where partial:
  accepted-with-partial-initial-openings-aligned-transcript-consistent

```

```

    ?s (Some (result, final-state)) fr f-fri-roots f-final as dg
    composition-fri-roots final query-idxs trace-openings
    composition-openings
using bad-some
unfolding
    composition-randomization-bad-with-aligned-transcript-partial-candidates-def
by blast
show ?thesis
unfolding out-eq
by (rule
    accepted-with-partial-initial-openings-aligned-transcript-consistent-imp-current-query-on-support
    [OF wf controlled support[unfolded out-eq] partial])
qed

lemma checked-staged-security-with-data-state-aligned-partial-candidate-randomization-le-current-query:
assumes wf: staged-budget-wellformed budgets
and controlled: staged-adversary-controlled budgets A
shows
    wp-event
    (checked-staged-security-experiment-with-data-state A)
    (staged-security-with-data-state-verifier-event
    composition-randomization-bad-with-aligned-transcript-partial-candidates)
    adversary-initial-state  $\leq$ 
    wp-event
    (checked-staged-security-experiment-with-data-state A)
    staged-security-with-data-state-current-query-partial-opening-hit
    adversary-initial-state
by (rule wp-event-mono-on-support)
    (rule
    checked-staged-security-with-data-state-aligned-partial-candidate-randomization-imp-current-query-on-support
    [OF wf controlled])

lemma checked-staged-security-with-data-state-aligned-partial-candidate-randomization-bound-from-query-prefix-curr
assumes wf: staged-budget-wellformed budgets
and controlled: staged-adversary-controlled budgets A
and round-bound:
     $\bigwedge i. i < \text{rounds} \implies$ 
    wp-event
    (checked-staged-security-experiment-with-query-prefix-data-state A i)
    (checked-staged-security-with-query-prefix-current-query-partial-opening-hit-at i)
    adversary-initial-state  $\leq C i$ 
shows
    wp-event
    (checked-staged-security-experiment-with-data-state A)
    (staged-security-with-data-state-verifier-event
    composition-randomization-bad-with-aligned-transcript-partial-candidates)
    adversary-initial-state  $\leq (\sum i < \text{rounds}. C i)$ 
proof –
have current-bound:

```

wp-event (checked-staged-security-experiment-with-data-state A)
staged-security-with-data-state-current-query-partial-opening-hit
adversary-initial-state $\leq (\sum i < \text{rounds. } C \ i)$
by (rule
checked-staged-security-with-data-state-current-query-partial-opening-hit-bound-from-query-prefix-rounds
[OF round-bound wf controlled])
show ?thesis
by (rule order-trans[
OF
checked-staged-security-with-data-state-aligned-partial-candidate-randomization-le-current-query
current-bound])
(use wf controlled in simp-all)
qed

lemma *checked-staged-security-with-data-state-aligned-randomization-or-query-bad-le-current-query:*
assumes *wf: staged-budget-wellformed budgets*
and *controlled: staged-adversary-controlled budgets A*
shows
wp-event
(checked-staged-security-experiment-with-data-state A)
($\lambda out.$
staged-security-with-data-state-verifier-event
composition-randomization-bad-with-aligned-transcript-partial-candidates
out $\vee$
staged-security-with-data-state-verifier-event
query-bad-with-aligned-transcript-partial-candidates out)
adversary-initial-state $\leq$
wp-event
(checked-staged-security-experiment-with-data-state A)
staged-security-with-data-state-current-query-partial-opening-hit
adversary-initial-state

proof (rule *wp-event-mono-on-support*)
fix *out*
assume *support:*
out $\in$
set-dist
(execute
(checked-staged-security-experiment-with-data-state A)
adversary-initial-state)
and *bad:*
staged-security-with-data-state-verifier-event
composition-randomization-bad-with-aligned-transcript-partial-candidates
out $\vee$
staged-security-with-data-state-verifier-event
query-bad-with-aligned-transcript-partial-candidates out
show
staged-security-with-data-state-current-query-partial-opening-hit out
proof (cases
staged-security-with-data-state-verifier-event

```

      composition-randomization-bad-with-aligned-transcript-partial-candidates
      out)
    case True
    then show ?thesis
    by (rule
      checked-staged-security-with-data-state-aligned-partial-candidate-randomization-imp-current-query-on-support
      [OF wf controlled support])
  next
  case False
  then have query-bad:
    staged-security-with-data-state-verifier-event
    query-bad-with-aligned-transcript-partial-candidates out
  using bad by simp
  show ?thesis
  proof (cases out)
  case None
  then show ?thesis
  using query-bad
  unfolding staged-security-with-data-state-verifier-event-def
    query-bad-with-aligned-transcript-partial-candidates-def
  by simp
  next
  case (Some packed)
  then obtain data attacker-state result final-state where out-eq:
    out = Some (((data, attacker-state), result), final-state)
  by (cases packed, auto split: prod.splits)
  show ?thesis
  unfolding out-eq
  by (rule
    checked-staged-security-with-data-state-aligned-transcript-query-bad-imp-current-partial-opening-hit-on-support
    [OF wf controlled support[unfolded out-eq]
      query-bad[unfolded out-eq]])
qed
qed
qed

lemma checked-staged-security-with-data-state-aligned-transcript-partial-candidate-aligned-randomization-with-merkle
  fixes merkle-error current-query-error trace-fri-error'
  composition-fri-error' :: prob
  assumes wf: staged-budget-wellformed budgets
  and controlled: staged-adversary-controlled budgets A
  and merkle-bound:
    wp-event (checked-staged-security-experiment-with-data-state A)
      (staged-security-with-data-state-verifier-event
        partial-merkle-inconsistency-bad)
    adversary-initial-state ≤ merkle-error
  and current-query-bound:
    wp-event (checked-staged-security-experiment-with-data-state A)
      staged-security-with-data-state-current-query-partial-opening-hit

```

```

    adversary-initial-state ≤ current-query-error
and trace-fri-bound:
    wp-event (checked-staged-security-experiment-with-data-state A)
      (staged-security-with-data-state-verifier-event
        trace-fri-bad-with-partial-candidates)
    adversary-initial-state ≤ trace-fri-error'
and comp-fri-bound:
    wp-event (checked-staged-security-experiment-with-data-state A)
      (staged-security-with-data-state-verifier-event
        composition-fri-bad-with-partial-candidates)
    adversary-initial-state ≤ composition-fri-error'
shows
    wp-event (checked-staged-security-experiment-with-data-state A)
      (staged-security-with-data-state-verifier-event
        soundness-bad-event-aligned-transcript-partial-candidate-with-aligned-randomization-and-merkle)
    adversary-initial-state ≤
    merkle-error + current-query-error + trace-fri-error' +
    composition-fri-error'
proof –
  let ?M = checked-staged-security-experiment-with-data-state A
  let ?E = staged-security-with-data-state-verifier-event
  let ?merkle = ?E partial-merkle-inconsistency-bad
  let ?comp =
    ?E composition-randomization-bad-with-aligned-transcript-partial-candidates
  let ?trace = ?E trace-fri-bad-with-partial-candidates
  let ?comp-fri = ?E composition-fri-bad-with-partial-candidates
  let ?query = ?E query-bad-with-aligned-transcript-partial-candidates
  let ?current =
    staged-security-with-data-state-current-query-partial-opening-hit
  let ?combined = λout. ?comp out ∨ ?query out
  let ?partial =
    ?E soundness-bad-event-aligned-transcript-partial-candidate-with-aligned-randomization-and-merkle
  have event-le:
    wp-event ?M ?partial adversary-initial-state ≤
    wp-event ?M
      (λout. ?merkle out ∨ ?combined out ∨ ?trace out ∨
        ?comp-fri out)
    adversary-initial-state
  by (rule wp-event-mono)
    (auto simp: staged-security-with-data-state-verifier-event-def
      soundness-bad-event-aligned-transcript-partial-candidate-with-aligned-randomization-and-merkle-def
      soundness-bad-event-aligned-transcript-partial-candidate-with-aligned-randomization-def
      split: option.splits prod.splits)
  have union-bound:
    wp-event ?M
      (λout. ?merkle out ∨ ?combined out ∨ ?trace out ∨
        ?comp-fri out)
    adversary-initial-state ≤
    wp-event ?M ?merkle adversary-initial-state +

```



```

    wp-event ?M ?combined adversary-initial-state +
    wp-event ?M ?trace adversary-initial-state +
    wp-event ?M ?comp-fri adversary-initial-state
  by (rule wp-event-union-bound4)
have combined-bound:
  wp-event ?M ?combined adversary-initial-state ≤
  wp-event ?M ?current adversary-initial-state
  by (rule
    checked-staged-security-with-data-state-aligned-randomization-or-query-bad-le-current-query
    [OF wf controlled])
have
  wp-event ?M ?partial adversary-initial-state ≤
  wp-event ?M ?merkle adversary-initial-state +
  wp-event ?M ?current adversary-initial-state +
  wp-event ?M ?trace adversary-initial-state +
  wp-event ?M ?comp-fri adversary-initial-state
  by (rule order-trans[OF event-le])
  (rule order-trans[OF union-bound],
  intro add-mono order-refl combined-bound)
also have ... ≤
  merkle-error + current-query-error + trace-fri-error' +
  composition-fri-error'
  by (intro add-mono merkle-bound current-query-bound trace-fri-bound
    comp-fri-bound)
finally show ?thesis .
qed

```

lemma *checked-staged-security-with-actual-alpha-prefix-aligned-partial-candidate-randomization-imp-current-query-o*

assumes *wf: staged-budget-wellformed budgets*

and *controlled: staged-adversary-controlled budgets A*

and *support:*

```

  out ∈
  set-dist
  (execute
    (checked-staged-security-experiment-with-actual-alpha-prefix-data-state
      A)
    adversary-initial-state)

```

and *bad:*

```

  checked-staged-security-with-actual-alpha-prefix-verifier-event
  composition-randomization-bad-with-aligned-transcript-partial-candidates
  out

```

shows

```

  (case out of
    None ⇒ False
  | Some (((((alpha-prefix, alpha-prefix-state), data), attacker-state),
    result), final-state) ⇒
    staged-security-with-data-state-current-query-partial-opening-hit
    (Some (((data, attacker-state), result), final-state)))

```

proof (cases out)

```

case None
then show ?thesis
  using bad
  unfolding checked-staged-security-with-actual-alpha-prefix-verifier-event-def
    composition-randomization-bad-with-aligned-transcript-partial-candidates-def
  by simp
next
case (Some packed)
then obtain alpha-prefix alpha-prefix-state data attacker-state result
  final-state where out-eq:
    out =
      Some (((alpha-prefix, alpha-prefix-state), data), attacker-state),
        result), final-state)
    by (cases packed, auto split: prod.splits)
let ?s =
  verifier-state-from-adversary attacker-state
    (staged-proof-transcript data)
have bad-some:
  composition-randomization-bad-with-aligned-transcript-partial-candidates
    ?s (Some (result, final-state))
using bad
unfolding out-eq
  checked-staged-security-with-actual-alpha-prefix-verifier-event-def
  Let-def
by simp
obtain fr f-fri-roots f-final as dg composition-fri-roots final
  query-idxs trace-openings composition-openings trace-table
  composition-table f
where partial:
  accepted-with-partial-initial-openings-aligned-transcript-consistent
    ?s (Some (result, final-state)) fr f-fri-roots f-final as dg
    composition-fri-roots final query-idxs trace-openings
    composition-openings
using bad-some
unfolding
  composition-randomization-bad-with-aligned-transcript-partial-candidates-def
by blast
have data-support:
  Some (((data, attacker-state), result), final-state) ∈
    set-dist
    (execute (checked-staged-security-experiment-with-data-state A)
      adversary-initial-state)
by (rule
  checked-staged-security-with-actual-alpha-prefix-support-imp-data-state-support
    [OF support[unfolded out-eq]])
have data-hit:
  staged-security-with-data-state-current-query-partial-opening-hit
    (Some (((data, attacker-state), result), final-state))
by (rule

```

```

    accepted-with-partial-initial-openings-aligned-transcript-consistent-imp-current-query-on-support
    [OF wf controlled data-support partial])
  then show ?thesis
    unfolding out-eq by simp
qed

lemma checked-staged-security-with-actual-alpha-prefix-aligned-partial-candidate-randomization-le-current-query:
  assumes wf: staged-budget-wellformed budgets
    and controlled: staged-adversary-controlled budgets A
  shows
    wp-event
      (checked-staged-security-experiment-with-actual-alpha-prefix-data-state A)
      (checked-staged-security-with-actual-alpha-prefix-verifier-event
        composition-randomization-bad-with-aligned-transcript-partial-candidates)
      adversary-initial-state ≤
    wp-event
      (checked-staged-security-experiment-with-data-state A)
      staged-security-with-data-state-current-query-partial-opening-hit
      adversary-initial-state
  proof -
    let ?project =
      λx. ((snd (fst (fst x)), snd (fst x)), snd x)
    let ?projected =
      λout. case out of
        None ⇒ False
      | Some (packed, t) ⇒
        staged-security-with-data-state-current-query-partial-opening-hit
          (Some (?project packed, t))
    have event-le:
      wp-event
        (checked-staged-security-experiment-with-actual-alpha-prefix-data-state A)
        (checked-staged-security-with-actual-alpha-prefix-verifier-event
          composition-randomization-bad-with-aligned-transcript-partial-candidates)
        adversary-initial-state ≤
      wp-event
        (checked-staged-security-experiment-with-actual-alpha-prefix-data-state A)
        ?projected adversary-initial-state
    proof (rule wp-event-mono-on-support)
      fix out
      assume support:
        out ∈
          set-dist
            (execute
              (checked-staged-security-experiment-with-actual-alpha-prefix-data-state
                A)
              adversary-initial-state)
      and bad:
        checked-staged-security-with-actual-alpha-prefix-verifier-event
          composition-randomization-bad-with-aligned-transcript-partial-candidates
    
```

```

      out
show ?projected out
proof (cases out)
  case None
  then show ?thesis
    using bad
    unfolding checked-staged-security-with-actual-alpha-prefix-verifier-event-def
      composition-randomization-bad-with-aligned-transcript-partial-candidates-def
    by simp
next
  case (Some packed)
  then obtain alpha-prefix alpha-prefix-state data attacker-state result
    final-state where out-eq:
    out =
      Some (((((alpha-prefix, alpha-prefix-state), data),
        attacker-state), result), final-state)
    by (cases packed, auto split: prod.splits)
  have hit:
    staged-security-with-data-state-current-query-partial-opening-hit
      (Some (((data, attacker-state), result), final-state))
    using
      checked-staged-security-with-actual-alpha-prefix-aligned-partial-candidate-randomization-imp-current-query-on-s
        [OF wf controlled support bad]
    unfolding out-eq by simp
  show ?thesis
    unfolding out-eq using hit by simp
qed
qed
have projection:
  wp-event
    (checked-staged-security-experiment-with-actual-alpha-prefix-data-state A)
    ?projected adversary-initial-state =
  wp-event
    (checked-staged-security-experiment-with-data-state A)
    staged-security-with-data-state-current-query-partial-opening-hit
    adversary-initial-state
proof –
  have
    wp-event (checked-staged-security-experiment-with-data-state A)
      staged-security-with-data-state-current-query-partial-opening-hit
      adversary-initial-state =
    wp-event
      (checked-staged-security-experiment-with-actual-alpha-prefix-data-state A
         $\gg$  (λx. return (?project x)))
      staged-security-with-data-state-current-query-partial-opening-hit
      adversary-initial-state
    using
      checked-staged-security-experiment-with-actual-alpha-prefix-data-state-projection
      [of A]

```

```

    by simp
  also have ... =
    wp-event
      (checked-staged-security-experiment-with-actual-alpha-prefix-data-state A)
      ?projected adversary-initial-state
    by (subst wp-event-bind-return-map)
      (simp add:
        staged-security-with-data-state-current-query-partial-opening-hit-def
        staged-security-with-data-state-current-query-partial-opening-hit-at-def)
  finally show ?thesis
    by simp
qed
show ?thesis
  by (rule order-trans[OF event-le])
    (simp add: projection)
qed

lemma checked-staged-security-with-actual-alpha-prefix-aligned-partial-candidate-randomization-bound-from-query-pr
  assumes wf: staged-budget-wellformed budgets
  and controlled: staged-adversary-controlled budgets A
  and round-bound:
     $\bigwedge i. i < \text{rounds} \implies$ 
    wp-event
      (checked-staged-security-experiment-with-query-prefix-data-state A i)
      (checked-staged-security-with-query-prefix-current-query-partial-opening-hit-at i)
      adversary-initial-state  $\leq C i$ 
  shows
    wp-event
      (checked-staged-security-experiment-with-actual-alpha-prefix-data-state A)
      (checked-staged-security-with-actual-alpha-prefix-verifier-event
        composition-randomization-bad-with-aligned-transcript-partial-candidates)
      adversary-initial-state  $\leq (\sum i < \text{rounds}. C i)$ 
proof -
  have current-bound:
    wp-event (checked-staged-security-experiment-with-data-state A)
      staged-security-with-data-state-current-query-partial-opening-hit
      adversary-initial-state  $\leq (\sum i < \text{rounds}. C i)$ 
  by (rule
    checked-staged-security-with-data-state-current-query-partial-opening-hit-bound-from-query-prefix-rounds
    [OF round-bound wf controlled])
  show ?thesis
    by (rule order-trans[
      OF
        checked-staged-security-with-actual-alpha-prefix-aligned-partial-candidate-randomization-le-current-query
        current-bound])
    (use wf controlled in simp-all)
qed

```

lemma *checked-staged-security-with-actual-alpha-prefix-relevant-partial-header-candidate-driftE:*

assumes

checked-staged-security-with-actual-alpha-prefix-relevant-partial-header-candidate-drift
out

obtains

checked-staged-security-with-actual-alpha-prefix-partial-header-candidate-drift
composition-trace-bad-alpha-space out
checked-staged-security-with-actual-alpha-prefix-verifier-event
composition-randomization-bad-with-partial-candidates out
| *checked-staged-security-with-actual-alpha-prefix-partial-header-candidate-drift*
composition-trace-bad-alpha-space out
checked-staged-security-with-actual-alpha-prefix-verifier-event
composition-alpha-partial-opening-union-hit-with-empty-header out

using *assms*

unfolding

checked-staged-security-with-actual-alpha-prefix-relevant-partial-header-candidate-drift-def

by *blast*

lemma *checked-staged-security-with-actual-alpha-prefix-relevant-partial-header-candidate-drift-le-branches:*

wp-event

(*checked-staged-security-experiment-with-actual-alpha-prefix-data-state A*)
checked-staged-security-with-actual-alpha-prefix-relevant-partial-header-candidate-drift
adversary-initial-state $\leq$

wp-event

(*checked-staged-security-experiment-with-actual-alpha-prefix-data-state A*)
(*checked-staged-security-with-actual-alpha-prefix-verifier-event*
composition-randomization-bad-with-partial-candidates)
adversary-initial-state +

wp-event

(*checked-staged-security-experiment-with-actual-alpha-prefix-data-state A*)
(*checked-staged-security-with-actual-alpha-prefix-verifier-event*
composition-alpha-partial-opening-union-hit-with-empty-header)
adversary-initial-state

proof —

have *event-le:*

wp-event

(*checked-staged-security-experiment-with-actual-alpha-prefix-data-state A*)
checked-staged-security-with-actual-alpha-prefix-relevant-partial-header-candidate-drift
adversary-initial-state $\leq$

wp-event

(*checked-staged-security-experiment-with-actual-alpha-prefix-data-state A*)
($\lambda out.$

checked-staged-security-with-actual-alpha-prefix-verifier-event
composition-randomization-bad-with-partial-candidates out $\vee$
checked-staged-security-with-actual-alpha-prefix-verifier-event
composition-alpha-partial-opening-union-hit-with-empty-header out)

adversary-initial-state

by (*rule wp-event-mono*)

(*auto simp:*

checked-staged-security-with-actual-alpha-prefix-relevant-partial-header-candidate-drift-def)

have *union-le*:
wp-event
 (*checked-staged-security-experiment-with-actual-alpha-prefix-data-state A*)
 ($\lambda out.$
 checked-staged-security-with-actual-alpha-prefix-verifier-event
 composition-randomization-bad-with-partial-candidates out $\vee$
 checked-staged-security-with-actual-alpha-prefix-verifier-event
 composition-alpha-partial-opening-union-hit-with-empty-header out)
adversary-initial-state $\leq$
wp-event
 (*checked-staged-security-experiment-with-actual-alpha-prefix-data-state A*)
 (*checked-staged-security-with-actual-alpha-prefix-verifier-event*
 composition-randomization-bad-with-partial-candidates)
adversary-initial-state +
wp-event
 (*checked-staged-security-experiment-with-actual-alpha-prefix-data-state A*)
 (*checked-staged-security-with-actual-alpha-prefix-verifier-event*
 composition-alpha-partial-opening-union-hit-with-empty-header)
adversary-initial-state
by (rule *wp-event-union-bound*)
show *?thesis*
by (rule *order-trans*[*OF event-le union-le*])
qed

lemma *checked-staged-security-with-actual-alpha-prefix-relevant-partial-header-candidate-drift-bound-from-branch-bo*
assumes *random-bound*:
wp-event
 (*checked-staged-security-experiment-with-actual-alpha-prefix-data-state A*)
 (*checked-staged-security-with-actual-alpha-prefix-verifier-event*
 composition-randomization-bad-with-partial-candidates)
adversary-initial-state $\leq R$
and *empty-bound*:
wp-event
 (*checked-staged-security-experiment-with-actual-alpha-prefix-data-state A*)
 (*checked-staged-security-with-actual-alpha-prefix-verifier-event*
 composition-alpha-partial-opening-union-hit-with-empty-header)
adversary-initial-state $\leq E$
shows
wp-event
 (*checked-staged-security-experiment-with-actual-alpha-prefix-data-state A*)
checked-staged-security-with-actual-alpha-prefix-relevant-partial-header-candidate-drift
adversary-initial-state $\leq R + E$
by (rule *order-trans*[
OF
checked-staged-security-with-actual-alpha-prefix-relevant-partial-header-candidate-drift-le-branches])
 (intro *add-mono random-bound empty-bound*)

lemma *checked-staged-security-with-actual-alpha-prefix-relevant-partial-header-candidate-drift-le-not-prefix-bound*:
assumes *wf*: *staged-budget-wellformed budgets*

and *controlled: staged-adversary-controlled budgets A*
shows
wp-event
(checked-staged-security-experiment-with-actual-alpha-prefix-data-state A)
checked-staged-security-with-actual-alpha-prefix-relevant-partial-header-candidate-drift
adversary-initial-state $\leq$
wp-event
(checked-staged-security-experiment-with-actual-alpha-prefix-data-state A)
(checked-staged-security-with-actual-alpha-prefix-partial-header-candidate-not-prefix-bound
composition-trace-bad-alpha-space)
adversary-initial-state
proof –
have *event-le:*
wp-event
(checked-staged-security-experiment-with-actual-alpha-prefix-data-state A)
checked-staged-security-with-actual-alpha-prefix-relevant-partial-header-candidate-drift
adversary-initial-state $\leq$
wp-event
(checked-staged-security-experiment-with-actual-alpha-prefix-data-state A)
(checked-staged-security-with-actual-alpha-prefix-partial-header-candidate-drift
composition-trace-bad-alpha-space)
adversary-initial-state
by (rule *wp-event-mono*)
(auto simp:
checked-staged-security-with-actual-alpha-prefix-relevant-partial-header-candidate-drift-def)
have *drift-le:*
wp-event
(checked-staged-security-experiment-with-actual-alpha-prefix-data-state A)
(checked-staged-security-with-actual-alpha-prefix-partial-header-candidate-drift
composition-trace-bad-alpha-space)
adversary-initial-state $\leq$
wp-event
(checked-staged-security-experiment-with-actual-alpha-prefix-data-state A)
(checked-staged-security-with-actual-alpha-prefix-partial-header-candidate-not-prefix-bound
composition-trace-bad-alpha-space)
adversary-initial-state
by (rule
checked-staged-security-with-actual-alpha-prefix-partial-header-candidate-drift-le-not-prefix-bound
[OF *wf controlled*])
show *?thesis*
by (rule *order-trans*[OF *event-le drift-le*])
qed
lemma *checked-staged-security-with-actual-alpha-prefix-relevant-partial-header-candidate-drift-bound-from-not-prefix-*
assumes *wf: staged-budget-wellformed budgets*
and *controlled: staged-adversary-controlled budgets A*
and *collision-bound:*
wp-event
(checked-staged-security-experiment-with-actual-alpha-prefix-data-state

A)
checked-staged-security-with-actual-alpha-prefix-not-prefix-collision
adversary-initial-state $\leq C$
and *uncovered-bound*:
wp-event
 (*checked-staged-security-experiment-with-actual-alpha-prefix-data-state*
 A)
checked-staged-security-with-actual-alpha-prefix-not-prefix-uncovered-index
adversary-initial-state $\leq U$
and *no-prefix-bound*:
wp-event
 (*checked-staged-security-experiment-with-actual-alpha-prefix-data-state*
 A)
checked-staged-security-with-actual-alpha-prefix-not-prefix-no-prefix-candidate
adversary-initial-state $\leq N$
and *path-bound*:
wp-event
 (*checked-staged-security-experiment-with-actual-alpha-prefix-data-state*
 A)
checked-staged-security-with-actual-alpha-prefix-not-prefix-path-output
adversary-initial-state $\leq P$
shows
wp-event
 (*checked-staged-security-experiment-with-actual-alpha-prefix-data-state* A)
checked-staged-security-with-actual-alpha-prefix-relevant-partial-header-candidate-drift
adversary-initial-state $\leq C + U + N + P$
proof –
have *not-prefix-bound*:
wp-event
 (*checked-staged-security-experiment-with-actual-alpha-prefix-data-state* A)
 (*checked-staged-security-with-actual-alpha-prefix-partial-header-candidate-not-prefix-bound*
composition-trace-bad-alpha-space)
adversary-initial-state $\leq C + U + N + P$
by (rule
checked-staged-security-with-actual-alpha-prefix-not-prefix-bound-union-bound
 [OF *wf controlled collision-bound uncovered-bound no-prefix-bound*
path-bound])
have *relevant-le*:
wp-event
 (*checked-staged-security-experiment-with-actual-alpha-prefix-data-state* A)
checked-staged-security-with-actual-alpha-prefix-relevant-partial-header-candidate-drift
adversary-initial-state $\leq$
wp-event
 (*checked-staged-security-experiment-with-actual-alpha-prefix-data-state* A)
 (*checked-staged-security-with-actual-alpha-prefix-partial-header-candidate-not-prefix-bound*
composition-trace-bad-alpha-space)
adversary-initial-state
by (rule
checked-staged-security-with-actual-alpha-prefix-relevant-partial-header-candidate-drift-le-not-prefix-bound

[OF wf controlled])
show ?thesis
 by (rule order-trans[OF relevant-le not-prefix-bound])
qed

lemma checked-staged-security-with-actual-alpha-prefix-relevant-partial-header-candidate-drift-bound-from-cross-and-
assumes wf: staged-budget-wellformed budgets
and controlled: staged-adversary-controlled budgets A
and cross-bound:
 wp-event
 (checked-staged-security-experiment-with-actual-alpha-prefix-data-state
 A)
 checked-staged-security-with-actual-alpha-prefix-witnessed-cross-or-merkle-bad
 adversary-initial-state $\leq X$
and witness-bound:
 wp-event
 (checked-staged-security-experiment-with-actual-alpha-prefix-data-state
 A)
 checked-staged-security-with-actual-alpha-prefix-witness-transcript-hit
 adversary-initial-state $\leq W$
shows
 wp-event
 (checked-staged-security-experiment-with-actual-alpha-prefix-data-state A)
 checked-staged-security-with-actual-alpha-prefix-relevant-partial-header-candidate-drift
 adversary-initial-state $\leq$
 hash-collision-budget-value 0
 (staged-alpha-search-queries budgets 0) + X + W + W
proof –
have collision-bound:
 wp-event
 (checked-staged-security-experiment-with-actual-alpha-prefix-data-state
 A)
 checked-staged-security-with-actual-alpha-prefix-not-prefix-collision
 adversary-initial-state $\leq$
 hash-collision-budget-value 0 (staged-alpha-search-queries budgets 0)
by (rule
 checked-staged-security-with-actual-alpha-prefix-not-prefix-collision-bound
 [OF wf controlled])
have uncovered-bound:
 wp-event
 (checked-staged-security-experiment-with-actual-alpha-prefix-data-state
 A)
 checked-staged-security-with-actual-alpha-prefix-not-prefix-uncovered-index
 adversary-initial-state $\leq X$
by (rule order-trans
 [OF
 checked-staged-security-with-actual-alpha-prefix-not-prefix-uncovered-index-le-witnessed-cross-or-merkle-bad
 cross-bound])
have no-prefix-bound:

```

wp-event
  (checked-staged-security-experiment-with-actual-alpha-prefix-data-state
   A)
  checked-staged-security-with-actual-alpha-prefix-not-prefix-no-prefix-candidate
  adversary-initial-state  $\leq W$ 
by (rule order-trans
   [OF
    checked-staged-security-with-actual-alpha-prefix-not-prefix-no-prefix-candidate-le-witness-transcript-hit
    witness-bound])
have path-bound:
  wp-event
    (checked-staged-security-experiment-with-actual-alpha-prefix-data-state
     A)
    checked-staged-security-with-actual-alpha-prefix-not-prefix-path-output
    adversary-initial-state  $\leq W$ 
  by (rule order-trans
      [OF
       checked-staged-security-with-actual-alpha-prefix-not-prefix-path-output-le-witness-transcript-hit
       witness-bound])
show ?thesis
  by (rule
      checked-staged-security-with-actual-alpha-prefix-relevant-partial-header-candidate-drift-bound-from-not-prefix-com
      [OF wf controlled collision-bound uncovered-bound no-prefix-bound
       path-bound])
qed

lemma checked-staged-security-with-actual-alpha-prefix-relevant-partial-header-candidate-drift-bound-from-cross-data
assumes wf: staged-budget-wellformed budgets
and controlled: staged-adversary-controlled budgets A
and cross-bound:
  wp-event
    (checked-staged-security-experiment-with-actual-alpha-prefix-data-state
     A)
    checked-staged-security-with-actual-alpha-prefix-witnessed-cross-or-merkle-bad
    adversary-initial-state  $\leq X$ 
and data-pre-bound:
  wp-event (checked-staged-security-experiment-with-data-state A)
    staged-security-with-data-state-transcript-pre-hit
    adversary-initial-state  $\leq P$ 
and transcript-new-bound:
  wp-event
    (checked-staged-security-experiment-with-actual-alpha-prefix-data-state
     A)
    checked-staged-security-with-actual-alpha-prefix-witness-transcript-new-hit
    adversary-initial-state  $\leq R$ 
shows
  wp-event
    (checked-staged-security-experiment-with-actual-alpha-prefix-data-state A)
    checked-staged-security-with-actual-alpha-prefix-relevant-partial-header-candidate-drift

```

$adversary-initial-state \leq$
 $hash-collision-budget-value\ 0$
 $(staged-alpha-search-queries\ budgets\ 0) + X + (P + R) + (P + R)$

proof –

have *transcript-pre-bound*:

wp-event
 $(checked-staged-security-experiment-with-actual-alpha-prefix-data-state\ A)$
 $checked-staged-security-with-actual-alpha-prefix-witness-transcript-pre-hit$
 $adversary-initial-state \leq P$

by (rule
 $checked-staged-security-with-actual-alpha-prefix-witness-transcript-pre-hit-bound-from-data-state-pre-hit$
 $[OF\ data-pre-bound])$

have *witness-bound*:

wp-event
 $(checked-staged-security-experiment-with-actual-alpha-prefix-data-state\ A)$
 $checked-staged-security-with-actual-alpha-prefix-witness-transcript-hit$
 $adversary-initial-state \leq P + R$

by (rule
 $checked-staged-security-with-actual-alpha-prefix-witness-transcript-hit-bound-from-pre-and-new$
 $[OF\ transcript-pre-bound\ transcript-new-bound])$

show *?thesis*

by (rule
 $checked-staged-security-with-actual-alpha-prefix-relevant-partial-header-candidate-drift-bound-from-cross-and-wit$
 $[OF\ wf\ controlled\ cross-bound\ witness-bound])$

qed

lemma *checked-staged-soundness-from-aligned-partial-candidate-components-and-current-empty-query-from-cross-da*

fixes *trace-fri-error'* *composition-fri-error'* *query-error* *X* *P* *R*

empty-query-error :: *prob*

assumes *false-statement*: $\neg\ exists-valid-trace$

and *wf*: *staged-budget-wellformed* *budgets*

and *controlled*: *staged-adversary-controlled* *budgets* *A*

and *trace-fri-bound*:

wp-event $(checked-staged-security-experiment-with-data-state\ A)$
 $(staged-security-with-data-state-verifier-event$
 $trace-fri-bad-with-partial-candidates)$
 $adversary-initial-state \leq trace-fri-error'$

and *comp-fri-bound*:

wp-event $(checked-staged-security-experiment-with-data-state\ A)$
 $(staged-security-with-data-state-verifier-event$
 $composition-fri-bad-with-partial-candidates)$
 $adversary-initial-state \leq composition-fri-error'$

and *query-bound*:

wp-event $(checked-staged-security-experiment-with-data-state\ A)$
 $(staged-security-with-data-state-verifier-event$
 $query-bad-with-aligned-partial-candidates)$
 $adversary-initial-state \leq query-error$

and *cross-bound*:

wp-event

(checked-staged-security-experiment-with-actual-alpha-prefix-data-state A)
checked-staged-security-with-actual-alpha-prefix-witnessed-cross-or-merkle-bad
adversary-initial-state $\leq X$
and *data-pre-bound:*
wp-event (checked-staged-security-experiment-with-data-state A)
staged-security-with-data-state-transcript-pre-hit
adversary-initial-state $\leq P$
and *transcript-new-bound:*
wp-event
(checked-staged-security-experiment-with-actual-alpha-prefix-data-state A)
checked-staged-security-with-actual-alpha-prefix-witness-transcript-new-hit
adversary-initial-state $\leq R$
and *empty-trace-fri-bound:*
wp-event (checked-staged-security-experiment-with-data-state A)
(staged-security-with-data-state-verifier-event
trace-fri-bad-with-empty-composition-header-candidates)
adversary-initial-state $\leq$ trace-fri-error
and *current-empty-bound:*
wp-event (checked-staged-security-experiment-with-data-state A)
(staged-security-with-data-state-current-empty-query-opening-hit-at 0)
adversary-initial-state $\leq$ empty-query-error
shows *checked-staged-adversary-acceptance-probability $A \leq$*
hash-collision-budget-value 0
(staged-attacker-query-budget budgets +
staged-challenge-query-budget + verifier-hash-query-budget) +
(composition-error-bound +
staged-phase-relation-error size
(staged-alpha-search-queries budgets 0) +
hash-collision-budget-value 0
(staged-alpha-search-queries budgets 0) +
staged-phase-relation-error size
(staged-alpha-search-queries budgets 0) +
(hash-collision-budget-value 0
(staged-alpha-search-queries budgets 0) + $X + (P + R) + (P + R)$)) +
trace-fri-error' + composition-fri-error' +
query-error +
(composition-error-bound +
staged-phase-relation-error size
(staged-alpha-search-queries budgets 0) +
hash-collision-budget-value 0
(staged-alpha-search-queries budgets 0) +
staged-phase-relation-error size
(staged-alpha-search-queries budgets 0) +
(hash-collision-budget-value 0
(staged-alpha-search-queries budgets 0) + $X + (P + R) + (P + R)$)) +
trace-fri-error + empty-query-error)
proof –

```

let ?D =
  hash-collision-budget-value 0
  (staged-alpha-search-queries budgets 0) + X + (P + R) + (P + R)
have drift-bound:
  wp-event
    (checked-staged-security-experiment-with-actual-alpha-prefix-data-state A)
    checked-staged-security-with-actual-alpha-prefix-relevant-partial-header-candidate-drift
    adversary-initial-state ≤ ?D
  by (rule
    checked-staged-security-with-actual-alpha-prefix-relevant-partial-header-candidate-drift-bound-from-cross-data-pre
    [OF wf controlled cross-bound data-pre-bound transcript-new-bound])
show ?thesis
  by (rule
    checked-staged-soundness-from-aligned-partial-candidate-components-and-current-empty-query-from-relevant-dri
    [OF false-statement wf controlled trace-fri-bound comp-fri-bound
      query-bound drift-bound empty-trace-fri-bound current-empty-bound])
qed

lemma checked-staged-soundness-from-aligned-transcript-partial-candidate-components-and-current-empty-query-from
fixes trace-fri-error' composition-fri-error' query-error D
  empty-query-error :: prob
assumes false-statement: ¬ exists-valid-trace
and wf: staged-budget-wellformed budgets
and controlled: staged-adversary-controlled budgets A
and trace-fri-bound:
  wp-event (checked-staged-security-experiment-with-data-state A)
    (staged-security-with-data-state-verifier-event
      trace-fri-bad-with-partial-candidates)
    adversary-initial-state ≤ trace-fri-error'
and comp-fri-bound:
  wp-event (checked-staged-security-experiment-with-data-state A)
    (staged-security-with-data-state-verifier-event
      composition-fri-bad-with-partial-candidates)
    adversary-initial-state ≤ composition-fri-error'
and query-bound:
  wp-event (checked-staged-security-experiment-with-data-state A)
    (staged-security-with-data-state-verifier-event
      query-bad-with-aligned-transcript-partial-candidates)
    adversary-initial-state ≤ query-error
and drift-bound:
  wp-event
    (checked-staged-security-experiment-with-actual-alpha-prefix-data-state
      A)
    checked-staged-security-with-actual-alpha-prefix-relevant-partial-header-candidate-drift
    adversary-initial-state ≤ D
and empty-trace-fri-bound:
  wp-event (checked-staged-security-experiment-with-data-state A)
    (staged-security-with-data-state-verifier-event
      trace-fri-bad-with-empty-composition-header-candidates)

```

adversary-initial-state $\leq$ *trace-fri-error*
and *current-empty-bound*:
wp-event (*checked-staged-security-experiment-with-data-state A*)
(staged-security-with-data-state-current-empty-query-opening-hit-at 0)
adversary-initial-state $\leq$ *empty-query-error*
shows *checked-staged-adversary-acceptance-probability A* $\leq$
hash-collision-budget-value 0
(staged-attacker-query-budget budgets +
staged-challenge-query-budget + verifier-hash-query-budget) +
(composition-error-bound +
staged-phase-relation-error size
(staged-alpha-search-queries budgets 0) +
hash-collision-budget-value 0
(staged-alpha-search-queries budgets 0) +
staged-phase-relation-error size
(staged-alpha-search-queries budgets 0) +
D) +
trace-fri-error' + composition-fri-error' +
query-error +
(composition-error-bound +
staged-phase-relation-error size
(staged-alpha-search-queries budgets 0) +
hash-collision-budget-value 0
(staged-alpha-search-queries budgets 0) +
staged-phase-relation-error size
(staged-alpha-search-queries budgets 0) +
D + trace-fri-error + empty-query-error)
proof –
have *comp-bound*:
wp-event (*checked-staged-security-experiment-with-data-state A*)
(staged-security-with-data-state-verifier-event
composition-bad-with-partial-candidates)
adversary-initial-state $\leq$
composition-error-bound +
staged-phase-relation-error size
(staged-alpha-search-queries budgets 0) +
hash-collision-budget-value 0
(staged-alpha-search-queries budgets 0) +
staged-phase-relation-error size
(staged-alpha-search-queries budgets 0) +
D
by (*rule*
checked-staged-security-with-data-state-partial-candidate-composition-bad-bound-from-relevant-drift-and-budgets
[OF false-statement wf controlled drift-bound])
have *merkle-bound*:
wp-event (*checked-staged-security-experiment-with-data-state A*)
(staged-security-with-data-state-verifier-event
partial-merkle-inconsistency-bad)
adversary-initial-state $\leq$

hash-collision-budget-value 0
(staged-attacker-query-budget budgets +
staged-challenge-query-budget + verifier-hash-query-budget)
by (rule
checked-staged-security-with-data-state-partial-merkle-inconsistency-bad-bound-controlled
[OF wf controlled])
have *empty-query-bound*:
wp-event (checked-staged-security-experiment-with-data-state A)
(staged-security-with-data-state-verifier-event
query-bad-with-empty-composition-header-candidates)
adversary-initial-state ≤ empty-query-error
by (rule *order-trans*[
OF checked-staged-security-with-data-state-empty-query-bad-le-current-empty-opening-hit-at-zero
*OF wf controlled] *current-empty-bound*])
have *empty-header-bound*:
wp-event (checked-staged-security-experiment-with-data-state A)
(staged-security-with-data-state-verifier-event
soundness-bad-event-partial-candidate-empty-header)
adversary-initial-state ≤
composition-error-bound +
staged-phase-relation-error size
(staged-alpha-search-queries budgets 0) +
hash-collision-budget-value 0
(staged-alpha-search-queries budgets 0) +
staged-phase-relation-error size
(staged-alpha-search-queries budgets 0) +
D + trace-fri-error + empty-query-error
by (rule
checked-staged-security-with-data-state-empty-header-bad-event-bound-from-relevant-drift-trace-fri-and-query
[OF false-statement wf controlled drift-bound empty-trace-fri-bound
empty-query-bound])
show *?thesis*
by (rule
checked-staged-soundness-from-aligned-transcript-partial-candidate-components-and-empty-header-bound
[OF false-statement merkle-bound comp-bound trace-fri-bound
comp-fri-bound query-bound empty-header-bound])
qed*

lemma *checked-staged-soundness-from-aligned-transcript-components-and-current-query-prefix-rounds-from-relevant-*
fixes *trace-fri-error' composition-fri-error' D empty-query-error :: prob*
and *C :: nat ⇒ prob*
assumes *false-statement: ¬ exists-valid-trace*
and *wf: staged-budget-wellformed budgets*
and *controlled: staged-adversary-controlled budgets A*
and *trace-fri-bound*:
wp-event (checked-staged-security-experiment-with-data-state A)
(staged-security-with-data-state-verifier-event
trace-fri-bad-with-partial-candidates)
adversary-initial-state ≤ trace-fri-error'

and *comp-fri-bound*:
 $\text{wp-event } (\text{checked-staged-security-experiment-with-data-state } A)$
 $(\text{staged-security-with-data-state-verifier-event}$
 $\text{composition-fri-bad-with-partial-candidates})$
 $\text{adversary-initial-state} \leq \text{composition-fri-error'}$
and *round-bound*:
 $\bigwedge i. i < \text{rounds} \implies$
 wp-event
 $(\text{checked-staged-security-experiment-with-query-prefix-data-state } A \ i)$
 $(\text{checked-staged-security-with-query-prefix-current-query-partial-opening-hit-at } i)$
 $\text{adversary-initial-state} \leq C \ i$
and *drift-bound*:
 wp-event
 $(\text{checked-staged-security-experiment-with-actual-alpha-prefix-data-state}$
 $A)$
 $\text{checked-staged-security-with-actual-alpha-prefix-relevant-partial-header-candidate-drift}$
 $\text{adversary-initial-state} \leq D$
and *empty-trace-fri-bound*:
 $\text{wp-event } (\text{checked-staged-security-experiment-with-data-state } A)$
 $(\text{staged-security-with-data-state-verifier-event}$
 $\text{trace-fri-bad-with-empty-composition-header-candidates})$
 $\text{adversary-initial-state} \leq \text{trace-fri-error}$
and *current-empty-bound*:
 $\text{wp-event } (\text{checked-staged-security-experiment-with-data-state } A)$
 $(\text{staged-security-with-data-state-current-empty-query-opening-hit-at } 0)$
 $\text{adversary-initial-state} \leq \text{empty-query-error}$
shows $\text{checked-staged-adversary-acceptance-probability } A \leq$
 $\text{hash-collision-budget-value } 0$
 $(\text{staged-attacker-query-budget budgets} +$
 $\text{staged-challenge-query-budget} + \text{verifier-hash-query-budget}) +$
 $(\text{composition-error-bound} +$
 $\text{staged-phase-relation-error size}$
 $(\text{staged-alpha-search-queries budgets } 0) +$
 $\text{hash-collision-budget-value } 0$
 $(\text{staged-alpha-search-queries budgets } 0) +$
 $\text{staged-phase-relation-error size}$
 $(\text{staged-alpha-search-queries budgets } 0) +$
 $D) +$
 $\text{trace-fri-error'} + \text{composition-fri-error'} +$
 $(\sum i < \text{rounds}. C \ i) +$
 $(\text{composition-error-bound} +$
 $\text{staged-phase-relation-error size}$
 $(\text{staged-alpha-search-queries budgets } 0) +$
 $\text{hash-collision-budget-value } 0$
 $(\text{staged-alpha-search-queries budgets } 0) +$
 $\text{staged-phase-relation-error size}$
 $(\text{staged-alpha-search-queries budgets } 0) +$
 $D + \text{trace-fri-error} + \text{empty-query-error})$
proof –

have *current-query-bound*:
wp-event (*checked-staged-security-experiment-with-data-state A*)
staged-security-with-data-state-current-query-partial-opening-hit
adversary-initial-state $\leq (\sum i < \text{rounds}. C\ i)$
by (*rule*
checked-staged-security-with-data-state-current-query-partial-opening-hit-bound-from-query-prefix-rounds
[OF round-bound wf controlled])
have *query-bound*:
wp-event (*checked-staged-security-experiment-with-data-state A*)
(staged-security-with-data-state-verifier-event
query-bad-with-aligned-transcript-partial-candidates)
adversary-initial-state $\leq (\sum i < \text{rounds}. C\ i)$
by (*rule order-trans*[
OF checked-staged-security-with-data-state-aligned-transcript-query-bad-bound-from-current-partial-opening-hit
current-query-bound])
(use wf controlled in simp-all)
show *?thesis*
by (*rule*
checked-staged-soundness-from-aligned-transcript-partial-candidate-components-and-current-empty-query-from-re
[OF false-statement wf controlled trace-fri-bound comp-fri-bound
query-bound drift-bound empty-trace-fri-bound current-empty-bound])
qed

lemma *checked-staged-soundness-from-aligned-transcript-aligned-randomization-components-and-current-query-prefix*
fixes *trace-fri-error'* *composition-fri-error'* *D* *empty-query-error* :: *prob*
and *C* :: *nat* $\Rightarrow$ *prob*
assumes *false-statement*: $\neg$ *exists-valid-trace*
and *wf*: *staged-budget-wellformed budgets*
and *controlled*: *staged-adversary-controlled budgets A*
and *trace-fri-bound*:
wp-event (*checked-staged-security-experiment-with-data-state A*)
(staged-security-with-data-state-verifier-event
trace-fri-bad-with-partial-candidates)
adversary-initial-state $\leq$ *trace-fri-error'*
and *comp-fri-bound*:
wp-event (*checked-staged-security-experiment-with-data-state A*)
(staged-security-with-data-state-verifier-event
composition-fri-bad-with-partial-candidates)
adversary-initial-state $\leq$ *composition-fri-error'*
and *round-bound*:
 $\bigwedge i. i < \text{rounds} \implies$
wp-event
(checked-staged-security-experiment-with-query-prefix-data-state A i)
(checked-staged-security-with-query-prefix-current-query-partial-opening-hit-at i)
adversary-initial-state $\leq C\ i$
and *drift-bound*:
wp-event
(checked-staged-security-experiment-with-actual-alpha-prefix-data-state
A)

checked-staged-security-with-actual-alpha-prefix-relevant-partial-header-candidate-drift
adversary-initial-state $\leq D$
and *empty-trace-fri-bound*:
wp-event (*checked-staged-security-experiment-with-data-state* A)
(*staged-security-with-data-state-verifier-event*
trace-fri-bad-with-empty-composition-header-candidates)
adversary-initial-state $\leq$ *trace-fri-error*
and *current-empty-bound*:
wp-event (*checked-staged-security-experiment-with-data-state* A)
(*staged-security-with-data-state-current-empty-query-opening-hit-at* 0)
adversary-initial-state $\leq$ *empty-query-error*
shows *checked-staged-adversary-acceptance-probability* $A \leq$
hash-collision-budget-value 0
(*staged-attacker-query-budget* *budgets* +
staged-challenge-query-budget + *verifier-hash-query-budget*) +
 $(\sum i < \text{rounds}. C\ i)$ +
trace-fri-error' + *composition-fri-error'* +
 $(\sum i < \text{rounds}. C\ i)$ +
(*composition-error-bound* +
staged-phase-relation-error *size*
(*staged-alpha-search-queries* *budgets* 0) +
hash-collision-budget-value 0
(*staged-alpha-search-queries* *budgets* 0) +
staged-phase-relation-error *size*
(*staged-alpha-search-queries* *budgets* 0) +
 $D + \text{trace-fri-error} + \text{empty-query-error}$)
proof –
have *comp-bound*:
wp-event (*checked-staged-security-experiment-with-data-state* A)
(*staged-security-with-data-state-verifier-event*
composition-randomization-bad-with-aligned-transcript-partial-candidates)
adversary-initial-state $\leq (\sum i < \text{rounds}. C\ i)$
by (rule
checked-staged-security-with-data-state-aligned-partial-candidate-randomization-bound-from-query-prefix-current
[*OF* *wf* *controlled* *round-bound*])
have *current-query-bound*:
wp-event (*checked-staged-security-experiment-with-data-state* A)
staged-security-with-data-state-current-query-partial-opening-hit
adversary-initial-state $\leq (\sum i < \text{rounds}. C\ i)$
by (rule
checked-staged-security-with-data-state-current-query-partial-opening-hit-bound-from-query-prefix-rounds
[*OF* *round-bound* *wf* *controlled*])
have *query-bound*:
wp-event (*checked-staged-security-experiment-with-data-state* A)
(*staged-security-with-data-state-verifier-event*
query-bad-with-aligned-transcript-partial-candidates)
adversary-initial-state $\leq (\sum i < \text{rounds}. C\ i)$
by (rule *order-trans*[
OF *checked-staged-security-with-data-state-aligned-transcript-query-bad-bound-from-current-partial-opening-hit*

```

    current-query-bound])
  (use wf controlled in simp-all)
have merkle-bound:
  wp-event (checked-staged-security-experiment-with-data-state A)
    (staged-security-with-data-state-verifier-event
     partial-merkle-inconsistency-bad)
  adversary-initial-state ≤
  hash-collision-budget-value 0
    (staged-attacker-query-budget budgets +
     staged-challenge-query-budget + verifier-hash-query-budget)
  by (rule
    checked-staged-security-with-data-state-partial-merkle-inconsistency-bad-bound-controlled
    [OF wf controlled])
have empty-query-bound:
  wp-event (checked-staged-security-experiment-with-data-state A)
    (staged-security-with-data-state-verifier-event
     query-bad-with-empty-composition-header-candidates)
  adversary-initial-state ≤ empty-query-error
  by (rule order-trans[
    OF checked-staged-security-with-data-state-empty-query-bad-le-current-empty-opening-hit-at-zero[
      OF wf controlled] current-empty-bound])
have empty-header-bound:
  wp-event (checked-staged-security-experiment-with-data-state A)
    (staged-security-with-data-state-verifier-event
     soundness-bad-event-partial-candidate-empty-header)
  adversary-initial-state ≤
  composition-error-bound +
  staged-phase-relation-error size
    (staged-alpha-search-queries budgets 0) +
  hash-collision-budget-value 0
    (staged-alpha-search-queries budgets 0) +
  staged-phase-relation-error size
    (staged-alpha-search-queries budgets 0) +
  D + trace-fri-error + empty-query-error
  by (rule
    checked-staged-security-with-data-state-empty-header-bad-event-bound-from-relevant-drift-trace-fri-and-query
    [OF false-statement wf controlled drift-bound empty-trace-fri-bound
     empty-query-bound])
have bound:
  checked-staged-adversary-acceptance-probability A ≤
  hash-collision-budget-value 0
    (staged-attacker-query-budget budgets +
     staged-challenge-query-budget + verifier-hash-query-budget) +
  (∑ i < rounds. C i) +
  trace-fri-error' + composition-fri-error' +
  (∑ i < rounds. C i) +
  (composition-error-bound +
   staged-phase-relation-error size
    (staged-alpha-search-queries budgets 0) +

```

```

    hash-collision-budget-value 0
    (staged-alpha-search-queries budgets 0) +
    staged-phase-relation-error size
    (staged-alpha-search-queries budgets 0) +
    D + trace-fri-error + empty-query-error)
  by (rule
    checked-staged-soundness-from-aligned-transcript-partial-candidate-aligned-randomization-components-and-empty-header-query-error
    [OF false-statement merkle-bound comp-bound trace-fri-bound
      comp-fri-bound query-bound empty-header-bound])
  then show ?thesis .
qed

lemma checked-staged-soundness-from-aligned-transcript-aligned-randomization-components-and-structured-query-fri-error
  fixes trace-fri-error' composition-fri-error' D empty-query-error
    header-query-error :: prob
  and C :: nat  $\Rightarrow$  prob
  assumes false-statement:  $\neg$  exists-valid-trace
  and wf: staged-budget-wellformed budgets
  and controlled: staged-adversary-controlled budgets A
  and trace-fri-bound:
    wp-event (checked-staged-security-experiment-with-data-state A)
      (staged-security-with-data-state-verifier-event
        trace-fri-bad-with-partial-candidates)
      adversary-initial-state  $\leq$  trace-fri-error'
  and comp-fri-bound:
    wp-event (checked-staged-security-experiment-with-data-state A)
      (staged-security-with-data-state-verifier-event
        composition-fri-bad-with-partial-candidates)
      adversary-initial-state  $\leq$  composition-fri-error'
  and round-bound:
     $\bigwedge i. i < \text{rounds} \implies$ 
      wp-event
        (checked-staged-security-experiment-with-query-prefix-data-state A i)
        (checked-staged-security-with-query-prefix-current-query-partial-opening-hit-at i)
        adversary-initial-state  $\leq$  C i
  and header-query-bound:
    wp-event
      (checked-staged-security-experiment-with-actual-alpha-prefix-data-state A)
      (checked-staged-security-with-actual-alpha-prefix-header-authenticated-candidate-opening-query-hit-from-prefix A)
      adversary-initial-state  $\leq$  header-query-error
  and drift-bound:
    wp-event
      (checked-staged-security-experiment-with-actual-alpha-prefix-data-state A)
      (checked-staged-security-with-actual-alpha-prefix-relevant-partial-header-candidate-drift)
      adversary-initial-state  $\leq$  D
  and empty-trace-fri-bound:

```

$wp\text{-}event$ (*checked-staged-security-experiment-with-data-state* A)
 (*staged-security-with-data-state-verifier-event*
trace-fri-bad-with-empty-composition-header-candidates)
 $adversary\text{-}initial\text{-}state \leq trace\text{-}fri\text{-}error$
and *current-empty-bound*:
 $wp\text{-}event$ (*checked-staged-security-experiment-with-data-state* A)
 (*staged-security-with-data-state-current-empty-query-opening-hit-at* 0)
 $adversary\text{-}initial\text{-}state \leq empty\text{-}query\text{-}error$
shows *checked-staged-adversary-acceptance-probability* $A \leq$
hash-collision-budget-value 0
 (*staged-attacker-query-budget* *budgets* +
staged-challenge-query-budget + *verifier-hash-query-budget*) +
 $(\sum i < rounds. C\ i) +$
trace-fri-error' + *composition-fri-error'* +
header-query-error +
 (*composition-error-bound* +
staged-phase-relation-error *size*
 (*staged-alpha-search-queries* *budgets* 0) +
hash-collision-budget-value 0
 (*staged-alpha-search-queries* *budgets* 0) +
staged-phase-relation-error *size*
 (*staged-alpha-search-queries* *budgets* 0) +
 $D + trace\text{-}fri\text{-}error + empty\text{-}query\text{-}error$)
proof –
have *comp-bound*:
 $wp\text{-}event$ (*checked-staged-security-experiment-with-data-state* A)
 (*staged-security-with-data-state-verifier-event*
composition-randomization-bad-with-aligned-transcript-partial-candidates)
 $adversary\text{-}initial\text{-}state \leq (\sum i < rounds. C\ i)$
by (*rule*
checked-staged-security-with-data-state-aligned-partial-candidate-randomization-bound-from-query-prefix-current-
[OF wf controlled round-bound])
have *query-bound*:
 $wp\text{-}event$ (*checked-staged-security-experiment-with-data-state* A)
 (*staged-security-with-data-state-verifier-event*
query-bad-with-aligned-transcript-partial-candidates)
 $adversary\text{-}initial\text{-}state \leq header\text{-}query\text{-}error$
by (*rule*
checked-staged-security-with-data-state-aligned-transcript-query-bad-bound-from-header-authenticated-candidate-
[OF wf controlled header-query-bound])
have *merkle-bound*:
 $wp\text{-}event$ (*checked-staged-security-experiment-with-data-state* A)
 (*staged-security-with-data-state-verifier-event*
partial-merkle-inconsistency-bad)
 $adversary\text{-}initial\text{-}state \leq$
hash-collision-budget-value 0
 (*staged-attacker-query-budget* *budgets* +
staged-challenge-query-budget + *verifier-hash-query-budget*)
by (*rule*

checked-staged-security-with-data-state-partial-merkle-inconsistency-bad-bound-controlled
 [OF wf controlled])
have *empty-query-bound*:
wp-event (*checked-staged-security-experiment-with-data-state A*)
 (*staged-security-with-data-state-verifier-event*
query-bad-with-empty-composition-header-candidates)
adversary-initial-state $\leq$ *empty-query-error*
by (rule *order-trans*[
 OF *checked-staged-security-with-data-state-empty-query-bad-le-current-empty-opening-hit-at-zero*[
 OF wf controlled] *current-empty-bound*])
have *empty-header-bound*:
wp-event (*checked-staged-security-experiment-with-data-state A*)
 (*staged-security-with-data-state-verifier-event*
soundness-bad-event-partial-candidate-empty-header)
adversary-initial-state $\leq$
composition-error-bound +
staged-phase-relation-error size
 (*staged-alpha-search-queries budgets 0*) +
hash-collision-budget-value 0
 (*staged-alpha-search-queries budgets 0*) +
staged-phase-relation-error size
 (*staged-alpha-search-queries budgets 0*) +
D + trace-fri-error + empty-query-error
by (rule
checked-staged-security-with-data-state-empty-header-bad-event-bound-from-relevant-drift-trace-fri-and-query
 [OF *false-statement wf controlled drift-bound empty-trace-fri-bound*
empty-query-bound])
have *bound*:
checked-staged-adversary-acceptance-probability A $\leq$
 (*hash-collision-budget-value 0*
 (*staged-attacker-query-budget budgets +*
staged-challenge-query-budget + verifier-hash-query-budget) +
 ($\sum i < \text{rounds. } C \ i$) + *trace-fri-error'* +
composition-fri-error' + header-query-error) +
 (*composition-error-bound +*
staged-phase-relation-error size
 (*staged-alpha-search-queries budgets 0*) +
hash-collision-budget-value 0
 (*staged-alpha-search-queries budgets 0*) +
staged-phase-relation-error size
 (*staged-alpha-search-queries budgets 0*) +
D + trace-fri-error + empty-query-error)
by (rule
checked-staged-soundness-from-aligned-transcript-partial-candidate-aligned-randomization-components-and-empty
 [OF *false-statement merkle-bound comp-bound trace-fri-bound*
comp-fri-bound query-bound empty-header-bound])
then show *?thesis*
by (*simp add: algebra-simps*)
qed

lemma *checked-staged-soundness-from-aligned-transcript-aligned-randomization-components-and-current-query-prefix*
fixes *trace-fri-error'* *composition-fri-error'* *D* *empty-query-error* :: *prob*
and *C* :: *nat* $\Rightarrow$ *prob*
assumes *false-statement*: $\neg$ *exists-valid-trace*
and *wf*: *staged-budget-wellformed* *budgets*
and *controlled*: *staged-adversary-controlled* *budgets* *A*
and *trace-fri-bound*:
 wp-event (*checked-staged-security-experiment-with-data-state* *A*)
 (*staged-security-with-data-state-verifier-event*
 trace-fri-bad-with-partial-candidates)
 adversary-initial-state $\leq$ *trace-fri-error'*
and *comp-fri-bound*:
 wp-event (*checked-staged-security-experiment-with-data-state* *A*)
 (*staged-security-with-data-state-verifier-event*
 composition-fri-bad-with-partial-candidates)
 adversary-initial-state $\leq$ *composition-fri-error'*
and *round-bound*:
 $\bigwedge i. i < \text{rounds} \Rightarrow$
 wp-event
 (*checked-staged-security-experiment-with-query-prefix-data-state* *A* *i*)
 (*checked-staged-security-with-query-prefix-current-query-partial-opening-hit-at* *i*)
 adversary-initial-state $\leq$ *C* *i*
and *drift-bound*:
 wp-event
 (*checked-staged-security-experiment-with-actual-alpha-prefix-data-state*
 A)
 checked-staged-security-with-actual-alpha-prefix-relevant-partial-header-candidate-drift
 adversary-initial-state $\leq$ *D*
and *empty-trace-fri-bound*:
 wp-event (*checked-staged-security-experiment-with-data-state* *A*)
 (*staged-security-with-data-state-verifier-event*
 trace-fri-bad-with-empty-composition-header-candidates)
 adversary-initial-state $\leq$ *trace-fri-error*
and *current-empty-bound*:
 wp-event (*checked-staged-security-experiment-with-data-state* *A*)
 (*staged-security-with-data-state-current-empty-query-opening-hit-at* *0*)
 adversary-initial-state $\leq$ *empty-query-error*
shows *checked-staged-adversary-acceptance-probability* *A* $\leq$
 hash-collision-budget-value *0*
 (*staged-attacker-query-budget* *budgets* +
 staged-challenge-query-budget + *verifier-hash-query-budget*) +
 ($\sum i < \text{rounds}. C\ i$) +
 trace-fri-error' + *composition-fri-error'* +
 (*composition-error-bound* +
 staged-phase-relation-error *size*
 (*staged-alpha-search-queries* *budgets* *0*) +
 hash-collision-budget-value *0*
 (*staged-alpha-search-queries* *budgets* *0*) +

staged-phase-relation-error size
(staged-alpha-search-queries budgets 0) +
D + trace-fri-error + empty-query-error)
proof –
have *current-query-bound*:
wp-event (checked-staged-security-experiment-with-data-state A)
staged-security-with-data-state-current-query-partial-opening-hit
adversary-initial-state $\leq (\sum i < \text{rounds. } C i)$
by (rule
checked-staged-security-with-data-state-current-query-partial-opening-hit-bound-from-query-prefix-rounds
[OF round-bound wf controlled])
have *merkle-bound*:
wp-event (checked-staged-security-experiment-with-data-state A)
(staged-security-with-data-state-verifier-event
partial-merkle-inconsistency-bad)
adversary-initial-state $\leq$
hash-collision-budget-value 0
(staged-attacker-query-budget budgets +
staged-challenge-query-budget + verifier-hash-query-budget)
by (rule
checked-staged-security-with-data-state-partial-merkle-inconsistency-bad-bound-controlled
[OF wf controlled])
have *partial-candidate-bound*:
wp-event (checked-staged-security-experiment-with-data-state A)
(staged-security-with-data-state-verifier-event
soundness-bad-event-aligned-transcript-partial-candidate-with-aligned-randomization-and-merkle)
adversary-initial-state $\leq$
hash-collision-budget-value 0
(staged-attacker-query-budget budgets +
staged-challenge-query-budget + verifier-hash-query-budget) +
($\sum i < \text{rounds. } C i$) + trace-fri-error' + composition-fri-error'
by (rule
checked-staged-security-with-data-state-aligned-transcript-partial-candidate-aligned-randomization-with-merkle-c
[OF wf controlled merkle-bound current-query-bound trace-fri-bound
comp-fri-bound])
have *empty-query-bound*:
wp-event (checked-staged-security-experiment-with-data-state A)
(staged-security-with-data-state-verifier-event
query-bad-with-empty-composition-header-candidates)
adversary-initial-state $\leq$ empty-query-error
by (rule *order-trans*[
OF checked-staged-security-with-data-state-empty-query-bad-le-current-empty-opening-hit-at-zero
*OF wf controlled] *current-empty-bound*])
have *empty-header-bound*:
wp-event (checked-staged-security-experiment-with-data-state A)
(staged-security-with-data-state-verifier-event
soundness-bad-event-partial-candidate-empty-header)
adversary-initial-state $\leq$
*composition-error-bound +**

```

    staged-phase-relation-error size
      (staged-alpha-search-queries budgets 0) +
    hash-collision-budget-value 0
      (staged-alpha-search-queries budgets 0) +
    staged-phase-relation-error size
      (staged-alpha-search-queries budgets 0) +
    D + trace-fri-error + empty-query-error
  by (rule
    checked-staged-security-with-data-state-empty-header-bad-event-bound-from-relevant-drift-trace-fri-and-query
    [OF false-statement wf controlled drift-bound empty-trace-fri-bound
      empty-query-bound])
  have bound:
    checked-staged-adversary-acceptance-probability A ≤
      (hash-collision-budget-value 0
        (staged-attacker-query-budget budgets +
          staged-challenge-query-budget + verifier-hash-query-budget) +
        (∑ i < rounds. C i) + trace-fri-error' +
        composition-fri-error') +
      (composition-error-bound +
        staged-phase-relation-error size
          (staged-alpha-search-queries budgets 0) +
        hash-collision-budget-value 0
          (staged-alpha-search-queries budgets 0) +
        staged-phase-relation-error size
          (staged-alpha-search-queries budgets 0) +
        D + trace-fri-error + empty-query-error)
  by (rule
    checked-staged-soundness-from-aligned-transcript-partial-candidate-aligned-randomization-and-empty-header-bou
    [OF false-statement partial-candidate-bound empty-header-bound])
  then show ?thesis
  by (simp add: algebra-simps)
qed

lemma checked-staged-soundness-from-aligned-transcript-conceptual-default-alpha-prefix-union-clean-structured-paths
  fixes trace-fri-error' composition-fri-error' D empty-query-error :: prob
  and T C :: nat ⇒ prob
  and trace-default composition-default :: 'f list
  assumes false-statement: ¬ exists-valid-trace
  and wf: staged-budget-wellformed budgets
  and controlled: staged-adversary-controlled budgets A
  and trace-fri-bound:
    wp-event (checked-staged-security-experiment-with-data-state A)
      (staged-security-with-data-state-verifier-event
        trace-fri-bad-with-partial-candidates)
    adversary-initial-state ≤ trace-fri-error'
  and comp-fri-bound:
    wp-event (checked-staged-security-experiment-with-data-state A)
      (staged-security-with-data-state-verifier-event
        composition-fri-bad-with-partial-candidates)

```

$\text{adversary-initial-state} \leq \text{composition-fri-error}'$
and $\text{default-prefix-clean}$:
 $\bigwedge i \text{ prefix prefix-state.}$
 $i < \text{rounds} \implies$
 $\text{Some } ((\text{prefix}, \text{prefix-state}), \text{prefix-state}) \in$
 set-dist
 $(\text{execute } (\text{checked-staged-query-prefix-with-state } A \ i)$
 $\text{adversary-initial-state}) \implies$
 $\text{query-prefix-trace-conceptual-table-with-default prefix prefix-state}$
 $\text{trace-default} = \text{trace-default} \wedge$
 $\text{query-prefix-composition-conceptual-table-with-default prefix}$
 $\text{prefix-state composition-default} = \text{composition-default} \wedge$
 $\text{trace-table-low-degree trace-default} \wedge$
 $\text{composition-table-low-degree maxDegree composition-default} \wedge$
 $\text{trace-default} \in$
 $\text{alpha-prefix-trace-table-candidates prefix-state}$
 $(\text{spp-trace-root prefix}) \wedge$
 $\text{spp-alphas prefix} \notin$
 $\text{alpha-prefix-union-bad-sets prefix-state}$
 $\text{composition-trace-bad-alpha-space } (\text{spp-trace-root prefix})$
and trace-path-bound :
 $\bigwedge i. i < \text{rounds} \implies$
 wp-event
 $(\text{checked-staged-security-experiment-with-query-prefix-data-state } A \ i)$
 $(\text{checked-staged-security-with-query-prefix-current-trace-path-output-hit-at } i)$
 $\text{adversary-initial-state} \leq T \ i$
and $\text{composition-path-bound}$:
 $\bigwedge i. i < \text{rounds} \implies$
 wp-event
 $(\text{checked-staged-security-experiment-with-query-prefix-data-state } A \ i)$
 $(\text{checked-staged-security-with-query-prefix-current-composition-path-output-hit-at } i)$
 $\text{adversary-initial-state} \leq C \ i$
and drift-bound :
 wp-event
 $(\text{checked-staged-security-experiment-with-actual-alpha-prefix-data-state}$
 $A)$
 $\text{checked-staged-security-with-actual-alpha-prefix-relevant-partial-header-candidate-drift}$
 $\text{adversary-initial-state} \leq D$
and $\text{empty-trace-fri-bound}$:
 $\text{wp-event } (\text{checked-staged-security-experiment-with-data-state } A)$
 $(\text{staged-security-with-data-state-verifier-event}$
 $\text{trace-fri-bad-with-empty-composition-header-candidates})$
 $\text{adversary-initial-state} \leq \text{trace-fri-error}$
and $\text{current-empty-bound}$:
 $\text{wp-event } (\text{checked-staged-security-experiment-with-data-state } A)$
 $(\text{staged-security-with-data-state-current-empty-query-opening-hit-at } 0)$
 $\text{adversary-initial-state} \leq \text{empty-query-error}$
shows $\text{checked-staged-adversary-acceptance-probability } A \leq$
 $\text{hash-collision-budget-value } 0$

(staged-attacker-query-budget budgets +
 staged-challenge-query-budget + verifier-hash-query-budget) +
 ($\sum i < \text{rounds}$.
 staged-phase-relation-error size
 (staged-query-search-queries budgets $i + 1$) +
 query-error-bound +
 hash-collision-budget-value 0
 (staged-query-search-queries budgets $i + 1$) +
 $T\ i + C\ i$) +
 trace-fri-error' + composition-fri-error' +
 ($\sum i < \text{rounds}$.
 staged-phase-relation-error size
 (staged-query-search-queries budgets $i + 1$) +
 query-error-bound +
 hash-collision-budget-value 0
 (staged-query-search-queries budgets $i + 1$) +
 $T\ i + C\ i$) +
 (composition-error-bound +
 staged-phase-relation-error size
 (staged-alpha-search-queries budgets 0) +
 hash-collision-budget-value 0
 (staged-alpha-search-queries budgets 0) +
 staged-phase-relation-error size
 (staged-alpha-search-queries budgets 0) +
 $D + \text{trace-fri-error} + \text{empty-query-error}$)
proof (rule
 checked-staged-soundness-from-aligned-transcript-aligned-randomization-components-and-current-query-prefix-round
 [OF false-statement wf controlled trace-fri-bound comp-fri-bound -
 drift-bound empty-trace-fri-bound current-empty-bound])
fix i
assume $i\text{-bound}$: $i < \text{rounds}$
show
 wp-event
 (checked-staged-security-experiment-with-query-prefix-data-state $A\ i$)
 (checked-staged-security-with-query-prefix-current-query-partial-opening-hit-at i)
 adversary-initial-state $\leq$
 staged-phase-relation-error size
 (staged-query-search-queries budgets $i + 1$) +
 query-error-bound +
 hash-collision-budget-value 0
 (staged-query-search-queries budgets $i + 1$) +
 $T\ i + C\ i$
by (rule
 checked-staged-security-with-query-prefix-current-query-partial-opening-hit-at-bound-from-conceptual-default-alpha
 [OF false-statement wf controlled $i\text{-bound}$ - trace-path-bound[OF $i\text{-bound}$]
 composition-path-bound[OF $i\text{-bound}$]])
 (rule default-prefix-clean[OF $i\text{-bound}$])
qed

end

end

theory *Soundness-Staged-Empty-Relevant-Drift*
 imports *Soundness-Staged-Relevant-Drift*
begin

Empty-header-specific relevant-drift packaging.

This theory is kept downstream of the broader relevant-drift layer so that the latter remains focused on reusable drift decompositions.

context *soundness*
begin

definition *checked-staged-security-with-actual-alpha-prefix-witnessed-sparse-cross-bad*

$:: ((((((f \times 'f \text{ list} \times 'f \text{ list} \times 'f) \times$
 $'f \text{ protocol-channel}) \times 'f \text{ staged-proof-data}) \times$
 $'f \text{ protocol-channel}) \times \text{unit list}) \times$
 $'f \text{ protocol-channel}) \text{ option} \Rightarrow \text{bool}$

where

checked-staged-security-with-actual-alpha-prefix-witnessed-sparse-cross-bad
 $\text{out} \longleftrightarrow$
 (case out of
 None $\Rightarrow$ False
 | Some (x, -) $\Rightarrow$
 (let packed = fst (fst x);
 data = snd packed;
 attacker-state = snd (fst x);
 s =
 verifier-state-from-adversary attacker-state
 (staged-proof-transcript data)
 in $\exists$ fr f-fri-roots f-final trace-table final-state
 trace-openings trace-table' final-state' trace-openings'.
 alpha-header-supported-partial-trace-table-candidate-witness
 s fr f-fri-roots f-final trace-table final-state
 trace-openings $\wedge$
 alpha-header-supported-partial-trace-table-candidate-witness
 s fr f-fri-roots f-final trace-table' final-state'
 trace-openings' $\wedge$
 trace-table $\neq$ trace-table' $\wedge$
 $\neg$ partial-openings-cross-cover-all-indices trace-openings
 trace-openings'))

definition *checked-staged-security-with-actual-alpha-prefix-witnessed-merkle-side-bad*

$:: ((((((f \times 'f \text{ list} \times 'f \text{ list} \times 'f) \times$
 $'f \text{ protocol-channel}) \times 'f \text{ staged-proof-data}) \times$
 $'f \text{ protocol-channel}) \times \text{unit list}) \times$
 $'f \text{ protocol-channel}) \text{ option} \Rightarrow \text{bool}$

where

checked-staged-security-with-actual-alpha-prefix-witnessed-merkle-side-bad
out $\longleftrightarrow$
 (case *out* of
 None $\Rightarrow$ *False*
 | *Some* (*x*, -) $\Rightarrow$
 (let *packed* = *fst* (*fst* *x*);
 data = *snd* *packed*;
 attacker-state = *snd* (*fst* *x*);
 s =
 verifier-state-from-adversary attacker-state
 (*staged-proof-transcript data*)
 in $\exists$ *fr f-fri-roots f-final trace-table final-state*
 trace-openings trace-table' final-state' trace-openings'.
 alpha-header-supported-partial-trace-table-candidate-witness
 s fr f-fri-roots f-final trace-table final-state
 trace-openings $\wedge$
 alpha-header-supported-partial-trace-table-candidate-witness
 s fr f-fri-roots f-final trace-table' final-state'
 trace-openings' $\wedge$
 trace-table $\neq$ *trace-table'* $\wedge$
 (*merkle-hash-value-conflict final-state final-state'* $\vee$
 hash-map-output-collision
 (*merkle-hash-state-merge final-state final-state'*))))

definition *checked-staged-security-with-actual-alpha-prefix-witnessed-local-collision-bad*

$:: ((((((f \times 'f \text{ list} \times 'f \text{ list} \times 'f) \times$
 'f protocol-channel) $\times$ *'f staged-proof-data*) $\times$
 'f protocol-channel) $\times$ *unit list*) $\times$
 'f protocol-channel) *option* $\Rightarrow$ *bool*

where

checked-staged-security-with-actual-alpha-prefix-witnessed-local-collision-bad
out $\longleftrightarrow$
 (case *out* of
 None $\Rightarrow$ *False*
 | *Some* (*x*, -) $\Rightarrow$
 (let *packed* = *fst* (*fst* *x*);
 data = *snd* *packed*;
 attacker-state = *snd* (*fst* *x*);
 s =
 verifier-state-from-adversary attacker-state
 (*staged-proof-transcript data*)
 in $\exists$ *fr f-fri-roots f-final trace-table final-state*
 trace-openings.
 alpha-header-supported-partial-trace-table-candidate-witness
 s fr f-fri-roots f-final trace-table final-state
 trace-openings $\wedge$
 hash-map-output-collision final-state))

definition *checked-staged-security-with-actual-alpha-prefix-witnessed-coupling-bad*

$$\begin{aligned} &:: ((((((f \times 'f \text{ list} \times 'f \text{ list} \times 'f) \times \\ &\quad 'f \text{ protocol-channel}) \times 'f \text{ staged-proof-data}) \times \\ &\quad 'f \text{ protocol-channel}) \times \text{unit list}) \times \\ &\quad 'f \text{ protocol-channel}) \text{ option} \Rightarrow \text{bool} \end{aligned}$$

where

checked-staged-security-with-actual-alpha-prefix-witnessed-coupling-bad

$$\begin{aligned} &\text{out} \longleftrightarrow \\ &(\text{case out of} \\ &\quad \text{None} \Rightarrow \text{False} \\ &| \text{Some } (x, -) \Rightarrow \\ &\quad (\text{let packed} = \text{fst } (\text{fst } x); \\ &\quad \quad \text{data} = \text{snd packed}; \\ &\quad \quad \text{attacker-state} = \text{snd } (\text{fst } x); \\ &\quad \quad s = \\ &\quad \quad \quad \text{verifier-state-from-adversary attacker-state} \\ &\quad \quad \quad (\text{staged-proof-transcript data}) \\ &\quad \text{in } \exists \text{fr f-fri-roots f-final trace-table final-state} \\ &\quad \quad \text{trace-openings trace-table' final-state' trace-openings'.} \\ &\quad \quad \text{alpha-header-supported-partial-trace-table-candidate-witness} \\ &\quad \quad \quad s \text{ fr f-fri-roots f-final trace-table final-state} \\ &\quad \quad \quad \text{trace-openings} \wedge \\ &\quad \quad \text{alpha-header-supported-partial-trace-table-candidate-witness} \\ &\quad \quad \quad s \text{ fr f-fri-roots f-final trace-table' final-state'} \\ &\quad \quad \quad \text{trace-openings'} \wedge \\ &\quad \quad \text{trace-table} \neq \text{trace-table'} \wedge \\ &\quad \quad \text{merkle-hash-pairwise-coupling-bad final-state final-state'})) \end{aligned}$$

lemma *checked-staged-security-with-actual-alpha-prefix-witnessed-merkle-side-bad-split:*

assumes

checked-staged-security-with-actual-alpha-prefix-witnessed-merkle-side-bad

$$\text{out}$$

shows

checked-staged-security-with-actual-alpha-prefix-witnessed-local-collision-bad

$$\text{out} \vee$$
checked-staged-security-with-actual-alpha-prefix-witnessed-coupling-bad

$$\text{out}$$

proof (*cases out*)

case *None*

then show *?thesis*

using *assms*

unfolding

checked-staged-security-with-actual-alpha-prefix-witnessed-merkle-side-bad-def

by *simp*

next

case (*Some packed*)

then obtain *prefix prefix-state data attacker-state result final-state0*

where *out-eq:*

$$\text{out} =$$

```

    Some (((((prefix, prefix-state), data), attacker-state), result),
          final-state0)
  by (cases packed, auto split: prod.splits)
let ?s =
  verifier-state-from-adversary attacker-state
  (staged-proof-transcript data)
from assms obtain fr f-fri-roots f-final trace-table final-state
  trace-openings trace-table' final-state' trace-openings' where
witness:
  alpha-header-supported-partial-trace-table-candidate-witness ?s fr
  f-fri-roots f-final trace-table final-state trace-openings
and witness':
  alpha-header-supported-partial-trace-table-candidate-witness ?s fr
  f-fri-roots f-final trace-table' final-state' trace-openings'
and distinct: trace-table  $\neq$  trace-table'
and pair-bad:
  merkle-hash-value-conflict final-state final-state'  $\vee$ 
  hash-map-output-collision
  (merkle-hash-state-merge final-state final-state')
unfolding out-eq
  checked-staged-security-with-actual-alpha-prefix-witnessed-merkle-side-bad-def
  Let-def
by auto
have split:
  hash-map-output-collision final-state  $\vee$ 
  hash-map-output-collision final-state'  $\vee$ 
  merkle-hash-pairwise-coupling-bad final-state final-state'
by (rule merkle-hash-pairwise-bad-imp-local-collision-or-coupling-bad
    [OF pair-bad])
then show ?thesis
proof
  assume collision: hash-map-output-collision final-state
  have
    checked-staged-security-with-actual-alpha-prefix-witnessed-local-collision-bad
    out
  unfolding out-eq
    checked-staged-security-with-actual-alpha-prefix-witnessed-local-collision-bad-def
    Let-def
  apply simp
  apply (intro exI conjI)
  apply (rule witness)
  apply (rule collision)
  done
  then show ?thesis by simp
next
  assume rest:
    hash-map-output-collision final-state'  $\vee$ 
    merkle-hash-pairwise-coupling-bad final-state final-state'
  then show ?thesis

```



```

proof
  assume collision': hash-map-output-collision final-state'
  have
    checked-staged-security-with-actual-alpha-prefix-witnessed-local-collision-bad
    out
  unfolding out-eq
    checked-staged-security-with-actual-alpha-prefix-witnessed-local-collision-bad-def
    Let-def
    apply simp
    apply (intro exI conjI)
      apply (rule witness')
      apply (rule collision')
    done
  then show ?thesis by simp
next
  assume coupling:
    merkle-hash-pairwise-coupling-bad final-state final-state'
  have
    checked-staged-security-with-actual-alpha-prefix-witnessed-coupling-bad
    out
  unfolding out-eq
    checked-staged-security-with-actual-alpha-prefix-witnessed-coupling-bad-def
    Let-def
    apply simp
    apply (intro exI conjI)
      apply (rule witness)
      apply (rule witness')
      apply (rule distinct)
      apply (rule coupling)
    done
  then show ?thesis by simp
qed
qed
qed

lemma checked-staged-security-with-actual-alpha-prefix-witnessed-merkle-side-bad-le-local-or-coupling:
  wp-event
    (checked-staged-security-experiment-with-actual-alpha-prefix-data-state A)
    checked-staged-security-with-actual-alpha-prefix-witnessed-merkle-side-bad
    adversary-initial-state ≤
  wp-event
    (checked-staged-security-experiment-with-actual-alpha-prefix-data-state A)
    checked-staged-security-with-actual-alpha-prefix-witnessed-local-collision-bad
    adversary-initial-state +
  wp-event
    (checked-staged-security-experiment-with-actual-alpha-prefix-data-state A)
    checked-staged-security-with-actual-alpha-prefix-witnessed-coupling-bad
    adversary-initial-state
proof —

```

```

let ?M =
  checked-staged-security-experiment-with-actual-alpha-prefix-data-state A
let ?H =
  checked-staged-security-with-actual-alpha-prefix-witnessed-merkle-side-bad
let ?L =
  checked-staged-security-with-actual-alpha-prefix-witnessed-local-collision-bad
let ?C =
  checked-staged-security-with-actual-alpha-prefix-witnessed-coupling-bad
have wp-event ?M ?H adversary-initial-state ≤
  wp-event ?M (λout. ?L out ∨ ?C out) adversary-initial-state
by (rule wp-event-mono)
  (rule
    checked-staged-security-with-actual-alpha-prefix-witnessed-merkle-side-bad-split)
also have ... ≤
  wp-event ?M ?L adversary-initial-state +
  wp-event ?M ?C adversary-initial-state
by (rule wp-event-union-bound)
finally show ?thesis .
qed

```

```

lemma checked-staged-security-with-actual-alpha-prefix-witnessed-cross-or-merkle-bad-split:
assumes
  checked-staged-security-with-actual-alpha-prefix-witnessed-cross-or-merkle-bad
  out
shows
  checked-staged-security-with-actual-alpha-prefix-witnessed-sparse-cross-bad
  out ∨
  checked-staged-security-with-actual-alpha-prefix-witnessed-merkle-side-bad
  out
using assms
unfolding
  checked-staged-security-with-actual-alpha-prefix-witnessed-cross-or-merkle-bad-def
  checked-staged-security-with-actual-alpha-prefix-witnessed-sparse-cross-bad-def
  checked-staged-security-with-actual-alpha-prefix-witnessed-merkle-side-bad-def
  alpha-supported-witnessed-partial-trace-pairwise-cross-or-merkle-bad-def
  alpha-header-supported-witnessed-partial-trace-pairwise-cross-or-merkle-bad-def
  Let-def
by (auto split: option.splits prod.splits; blast)

```

```

lemma checked-staged-security-with-actual-alpha-prefix-witnessed-cross-or-merkle-bad-le-sparse-or-merkle-side:
  wp-event
  (checked-staged-security-experiment-with-actual-alpha-prefix-data-state A)
  checked-staged-security-with-actual-alpha-prefix-witnessed-cross-or-merkle-bad
  adversary-initial-state ≤
  wp-event
  (checked-staged-security-experiment-with-actual-alpha-prefix-data-state A)
  checked-staged-security-with-actual-alpha-prefix-witnessed-sparse-cross-bad
  adversary-initial-state +
  wp-event

```

(checked-staged-security-experiment-with-actual-alpha-prefix-data-state A)
checked-staged-security-with-actual-alpha-prefix-witnessed-merkle-side-bad
adversary-initial-state
proof –
let ?M =
checked-staged-security-experiment-with-actual-alpha-prefix-data-state A
let ?W =
checked-staged-security-with-actual-alpha-prefix-witnessed-cross-or-merkle-bad
let ?S =
checked-staged-security-with-actual-alpha-prefix-witnessed-sparse-cross-bad
let ?H =
checked-staged-security-with-actual-alpha-prefix-witnessed-merkle-side-bad
have *wp-event ?M ?W adversary-initial-state* ≤
wp-event ?M (λout. ?S out ∨ ?H out) adversary-initial-state
by (rule *wp-event-mono*)
(rule
checked-staged-security-with-actual-alpha-prefix-witnessed-cross-or-merkle-bad-split)
also have ... ≤
wp-event ?M ?S adversary-initial-state +
wp-event ?M ?H adversary-initial-state
by (rule *wp-event-union-bound*)
finally show ?thesis .
qed

definition *empty-header-partial-trace-candidate-nonunique*
:: (*f*, *'a*) *protocol-channel-scheme* ⇒
(*unit list* × (*f*, *'a*) *protocol-channel-scheme*) *option* ⇒ *bool*
where
empty-header-partial-trace-candidate-nonunique s out ⇔
(∃ *fr f-fri-roots f-final as dg final trace-query-idxs*
trace-openings trace-table composition-table.
accepted-with-empty-composition-header-candidates s out fr
f-fri-roots f-final as dg final trace-query-idxs trace-openings
trace-table composition-table ∧
¬ (∃ *trace-table0.*
partial-trace-table-candidates trace-openings ⊆
{*trace-table0*}))

lemma *composition-alpha-partial-opening-union-hit-with-empty-header-split*:
assumes *hit*:
composition-alpha-partial-opening-union-hit-with-empty-header s out
shows
composition-alpha-bad-set-hit-with-empty-composition-header-candidates
s out ∨
empty-header-partial-trace-candidate-nonunique s out
proof –
obtain *fr f-fri-roots f-final as dg final trace-query-idxs*
trace-openings trace-table composition-table **where** *partial*:
accepted-with-empty-composition-header-candidates s out fr

```

    f-fri-roots f-final as dg final trace-query-idxs trace-openings
    trace-table composition-table
and as-union:
    as  $\in$  partial-trace-opening-alpha-union-bad-sets trace-openings
using hit
unfolding composition-alpha-partial-opening-union-hit-with-empty-header-def
by blast
from as-union obtain trace-table' where candidate':
    trace-table'  $\in$  partial-trace-table-candidates trace-openings
and as-bad': as  $\in$  composition-trace-bad-alpha-space trace-table'
unfolding partial-trace-opening-alpha-union-bad-sets-def by blast
have candidate:
    trace-table  $\in$  partial-trace-table-candidates trace-openings
using partial
unfolding accepted-with-empty-composition-header-candidates-def
    partial-trace-table-candidates-def
by simp
show ?thesis
proof (cases
   $\exists$  trace-table0.
    partial-trace-table-candidates trace-openings  $\subseteq$ 
    {trace-table0})
case True
then obtain trace-table0 where subset:
    partial-trace-table-candidates trace-openings  $\subseteq$ 
    {trace-table0}
by blast
have trace-eq: trace-table = trace-table0
using subset candidate by blast
have trace'-eq: trace-table' = trace-table0
using subset candidate' by blast
have as  $\in$  composition-trace-bad-alpha-space trace-table
using as-bad' trace-eq trace'-eq by simp
then have
    composition-alpha-bad-set-hit-with-empty-composition-header-candidates
    s out
unfolding
    composition-alpha-bad-set-hit-with-empty-composition-header-candidates-def
using partial by blast
then show ?thesis by simp
next
case False
have empty-header-partial-trace-candidate-nonunique s out
unfolding empty-header-partial-trace-candidate-nonunique-def
using partial False by blast
then show ?thesis by simp
qed
qed

```

lemma *checked-staged-security-with-actual-alpha-prefix-empty-header-nonunique-imp-witnessed-cross-or-merkle-on-s*
assumes *support*:
 out $\in$
 set-dist
 (*execute*
 (*checked-staged-security-experiment-with-actual-alpha-prefix-data-state*
 A)
 adversary-initial-state)
and *nonunique*:
 checked-staged-security-with-actual-alpha-prefix-verifier-event
 empty-header-partial-trace-candidate-nonunique out
shows
 checked-staged-security-with-actual-alpha-prefix-witnessed-cross-or-merkle-bad
 out
proof (*cases out*)
case *None*
then show *?thesis*
 using *nonunique*
 unfolding *checked-staged-security-with-actual-alpha-prefix-verifier-event-def*
 by *simp*
next
case (*Some result-pack*)
then obtain *prefix prefix-state data attacker-state result final-state*
where *out-eq*:
 out =
 Some ((((*prefix*, *prefix-state*), *data*), *attacker-state*),
 result), *final-state*)
 by (*cases result-pack*, *auto split: prod.splits*)
let *?s* =
 verifier-state-from-adversary attacker-state
 (*staged-proof-transcript data*)
have *verifier*:
 Some (*result*, *final-state*) $\in$ *set-dist* (*execute verify-monad ?s*)
 using *support*
 unfolding *out-eq*
 checked-staged-security-experiment-with-actual-alpha-prefix-data-state-def
 by (*auto elim!: set-dist-bindE split: prod.splits*)
have *local-nonunique*:
 empty-header-partial-trace-candidate-nonunique ?s
 (*Some* (*result*, *final-state*))
 using *nonunique*
 unfolding *out-eq*
 checked-staged-security-with-actual-alpha-prefix-verifier-event-def
 Let-def
 by *simp*
then obtain *fr f-fri-roots f-final as dg final trace-query-idxs*
 trace-openings trace-table composition-table where empty:
 accepted-with-empty-composition-header-candidates ?s
 (*Some* (*result*, *final-state*)) *fr f-fri-roots f-final as dg final*

```

    trace-query-idxs trace-openings trace-table composition-table
and not-unique-partial:
  ¬ (∃ trace-table0.
    partial-trace-table-candidates trace-openings ⊆
      {trace-table0})
unfolding empty-header-partial-trace-candidate-nonunique-def
by blast
from empty obtain rest where header:
  verifier-header-transcript ?s fr f-fri-roots f-final as dg []
  final rest
and trace-partial:
  accepted-with-partial-trace-openings ?s
    (Some (result, final-state)) fr trace-query-idxs trace-openings
unfolding accepted-with-empty-composition-header-candidates-def
by blast
have partial-subset:
  partial-trace-table-candidates trace-openings ⊆
    alpha-header-supported-partial-trace-table-candidates ?s fr f-fri-roots
      f-final
proof
  fix trace-table'
  assume
    trace-table' ∈ partial-trace-table-candidates trace-openings
  then have candidate':
    partial-trace-table-candidate trace-table' trace-openings
  unfolding partial-trace-table-candidates-def by simp
  show
    trace-table' ∈
      alpha-header-supported-partial-trace-table-candidates ?s fr
        f-fri-roots f-final
  unfolding alpha-header-supported-partial-trace-table-candidates-def
  by (intro CollectI exI[of - Some (result, final-state)]
    exI[of - trace-query-idxs] exI[of - trace-openings]
    exI[of - as] exI[of - dg] exI[of - []]
    exI[of - final] exI[of - rest] conjI)
    (use verifier trace-partial candidate' header in simp-all)
qed
have not-unique-header:
  ¬ (∃ trace-table0.
    alpha-header-supported-partial-trace-table-candidates ?s fr
      f-fri-roots f-final ⊆ {trace-table0})
proof
  assume
    ∃ trace-table0.
      alpha-header-supported-partial-trace-table-candidates ?s fr
        f-fri-roots f-final ⊆ {trace-table0}
  then obtain trace-table0 where header-subset:
    alpha-header-supported-partial-trace-table-candidates ?s fr
      f-fri-roots f-final ⊆ {trace-table0}

```

```

    by blast
  have
    partial-trace-table-candidates trace-openings  $\subseteq$ 
      {trace-table0}
    using partial-subset header-subset by blast
  then show False
    using not-unique-partial by blast
qed
have not-unique-witnessed:
   $\neg (\exists \text{ trace-table0.}$ 
    alpha-header-supported-witnessed-partial-trace-table-candidates ?s fr
      f-fri-roots f-final  $\subseteq$  {trace-table0})
  using not-unique-header
  by (simp add: alpha-header-supported-witnessed-candidates-eq-candidates)
have
  alpha-header-supported-witnessed-partial-trace-pairwise-cross-or-merkle-bad
    ?s fr f-fri-roots f-final
  by (rule
    alpha-header-supported-witnessed-partial-trace-pairwise-cross-or-merkle-bad-if-not-singleton
      [OF not-unique-witnessed])
then have
  alpha-supported-witnessed-partial-trace-pairwise-cross-or-merkle-bad ?s
  unfolding
    alpha-supported-witnessed-partial-trace-pairwise-cross-or-merkle-bad-def
  by blast
then show ?thesis
  unfolding out-eq
    checked-staged-security-with-actual-alpha-prefix-witnessed-cross-or-merkle-bad-def
    Let-def
  by simp
qed

lemma checked-staged-security-with-actual-alpha-prefix-empty-header-nonunique-le-witnessed-cross-or-merkle:
  wp-event
    (checked-staged-security-experiment-with-actual-alpha-prefix-data-state A)
    (checked-staged-security-with-actual-alpha-prefix-verifier-event
      empty-header-partial-trace-candidate-nonunique)
  adversary-initial-state  $\leq$ 
  wp-event
    (checked-staged-security-experiment-with-actual-alpha-prefix-data-state A)
    checked-staged-security-with-actual-alpha-prefix-witnessed-cross-or-merkle-bad
    adversary-initial-state
  by (rule wp-event-mono-on-support)
  (rule
    checked-staged-security-with-actual-alpha-prefix-empty-header-nonunique-imp-witnessed-cross-or-merkle-on-support)

definition checked-staged-security-with-actual-alpha-prefix-empty-header-relevant-drift
  :: (((('f  $\times$  'f list  $\times$  'f list  $\times$  'f)  $\times$ 
    'f protocol-channel)  $\times$  'f staged-proof-data)  $\times$ 

```

$\text{'f protocol-channel) } \times \text{ unit list) } \times$
 $\text{'f protocol-channel) option } \Rightarrow \text{ bool}$

where

$\text{checked-staged-security-with-actual-alpha-prefix-empty-header-relevant-drift}$
 $\text{out} \longleftrightarrow$
 $\text{checked-staged-security-with-actual-alpha-prefix-partial-header-candidate-drift}$
 $\text{composition-trace-bad-alpha-space out} \wedge$
 $\text{checked-staged-security-with-actual-alpha-prefix-verifier-event}$
 $\text{composition-alpha-partial-opening-union-hit-with-empty-header out}$

lemma *checked-staged-security-with-actual-alpha-prefix-empty-header-relevant-drift-le-fixed-or-nonunique:*

wp-event
 $(\text{checked-staged-security-experiment-with-actual-alpha-prefix-data-state } A)$
 $\text{checked-staged-security-with-actual-alpha-prefix-empty-header-relevant-drift}$
 $\text{adversary-initial-state} \leq$
 wp-event
 $(\text{checked-staged-security-experiment-with-actual-alpha-prefix-data-state } A)$
 $(\text{checked-staged-security-with-actual-alpha-prefix-verifier-event}$
 $\text{composition-alpha-bad-set-hit-with-empty-composition-header-candidates})$
 $\text{adversary-initial-state} +$
 wp-event
 $(\text{checked-staged-security-experiment-with-actual-alpha-prefix-data-state } A)$
 $(\text{checked-staged-security-with-actual-alpha-prefix-verifier-event}$
 $\text{empty-header-partial-trace-candidate-nonunique})$
 $\text{adversary-initial-state}$

proof —

let $?M =$
 $\text{checked-staged-security-experiment-with-actual-alpha-prefix-data-state } A$
let $?fixed =$
 $\text{checked-staged-security-with-actual-alpha-prefix-verifier-event}$
 $\text{composition-alpha-bad-set-hit-with-empty-composition-header-candidates}$
let $?nonunique =$
 $\text{checked-staged-security-with-actual-alpha-prefix-verifier-event}$
 $\text{empty-header-partial-trace-candidate-nonunique}$
have *event-le:*
 $\text{wp-event } ?M$
 $\text{checked-staged-security-with-actual-alpha-prefix-empty-header-relevant-drift}$
 $\text{adversary-initial-state} \leq$
 $\text{wp-event } ?M (\lambda \text{out. } ?fixed \text{ out} \vee ?nonunique \text{ out})$
 $\text{adversary-initial-state}$
by (*rule wp-event-mono*)
 $(\text{auto simp:}$
 $\text{checked-staged-security-with-actual-alpha-prefix-empty-header-relevant-drift-def}$
 $\text{checked-staged-security-with-actual-alpha-prefix-verifier-event-def}$
 $\text{split: option.splits prod.splits}$
 dest:
 $\text{composition-alpha-partial-opening-union-hit-with-empty-header-split})$
also have $\dots \leq$
 $\text{wp-event } ?M ?fixed \text{ adversary-initial-state} +$


```

      wp-event ?M ?nonunique adversary-initial-state
    by (rule wp-event-union-bound)
  finally show ?thesis .
qed

lemma checked-staged-security-with-actual-alpha-prefix-empty-header-relevant-drift-le-fixed-or-witnessed-cross:
  wp-event
    (checked-staged-security-experiment-with-actual-alpha-prefix-data-state A)
    checked-staged-security-with-actual-alpha-prefix-empty-header-relevant-drift
    adversary-initial-state ≤
  wp-event
    (checked-staged-security-experiment-with-actual-alpha-prefix-data-state A)
    (checked-staged-security-with-actual-alpha-prefix-verifier-event
      composition-alpha-bad-set-hit-with-empty-composition-header-candidates)
    adversary-initial-state +
  wp-event
    (checked-staged-security-experiment-with-actual-alpha-prefix-data-state A)
    checked-staged-security-with-actual-alpha-prefix-witnessed-cross-or-merkle-bad
    adversary-initial-state
proof -
  let ?M =
    checked-staged-security-experiment-with-actual-alpha-prefix-data-state A
  let ?fixed =
    checked-staged-security-with-actual-alpha-prefix-verifier-event
    composition-alpha-bad-set-hit-with-empty-composition-header-candidates
  let ?nonunique =
    checked-staged-security-with-actual-alpha-prefix-verifier-event
    empty-header-partial-trace-candidate-nonunique
  have base:
    wp-event ?M
      checked-staged-security-with-actual-alpha-prefix-empty-header-relevant-drift
      adversary-initial-state ≤
    wp-event ?M ?fixed adversary-initial-state +
    wp-event ?M ?nonunique adversary-initial-state
  by (rule
    checked-staged-security-with-actual-alpha-prefix-empty-header-relevant-drift-le-fixed-or-nonunique)
  have nonunique-bound:
    wp-event ?M ?nonunique adversary-initial-state ≤
    wp-event ?M
      checked-staged-security-with-actual-alpha-prefix-witnessed-cross-or-merkle-bad
      adversary-initial-state
  by (rule
    checked-staged-security-with-actual-alpha-prefix-empty-header-nonunique-le-witnessed-cross-or-merkle)
  show ?thesis
  by (rule order-trans[OF base])
  (intro add-mono order-refl nonunique-bound)
qed

lemma checked-staged-security-with-actual-alpha-prefix-empty-header-relevant-drift-bound-from-fixed-and-witnessed-c
```

assumes *fixed-bound*:

wp-event

(*checked-staged-security-experiment-with-actual-alpha-prefix-data-state*
A)

(*checked-staged-security-with-actual-alpha-prefix-verifier-event*
composition-alpha-bad-set-hit-with-empty-composition-header-candidates)

adversary-initial-state $\leq F$

and *cross-bound*:

wp-event

(*checked-staged-security-experiment-with-actual-alpha-prefix-data-state*
A)

checked-staged-security-with-actual-alpha-prefix-witnessed-cross-or-merkle-bad
adversary-initial-state $\leq X$

shows

wp-event

(*checked-staged-security-experiment-with-actual-alpha-prefix-data-state A*)

checked-staged-security-with-actual-alpha-prefix-empty-header-relevant-drift

adversary-initial-state $\leq F + X$

by (rule *order-trans*[

OF

checked-staged-security-with-actual-alpha-prefix-empty-header-relevant-drift-le-fixed-or-witnessed-cross])

(intro *add-mono fixed-bound cross-bound*)

lemma *checked-staged-security-with-actual-alpha-prefix-empty-header-relevant-drift-bound-from-fixed-and-nonunique*:

assumes *fixed-bound*:

wp-event

(*checked-staged-security-experiment-with-actual-alpha-prefix-data-state*
A)

(*checked-staged-security-with-actual-alpha-prefix-verifier-event*
composition-alpha-bad-set-hit-with-empty-composition-header-candidates)

adversary-initial-state $\leq F$

and *nonunique-bound*:

wp-event

(*checked-staged-security-experiment-with-actual-alpha-prefix-data-state*
A)

(*checked-staged-security-with-actual-alpha-prefix-verifier-event*
empty-header-partial-trace-candidate-nonunique)

adversary-initial-state $\leq N$

shows

wp-event

(*checked-staged-security-experiment-with-actual-alpha-prefix-data-state A*)

checked-staged-security-with-actual-alpha-prefix-empty-header-relevant-drift

adversary-initial-state $\leq F + N$

by (rule *order-trans*[

OF

checked-staged-security-with-actual-alpha-prefix-empty-header-relevant-drift-le-fixed-or-nonunique])

(intro *add-mono fixed-bound nonunique-bound*)

lemma *checked-staged-security-with-actual-alpha-prefix-empty-header-relevant-drift-le-relevant-drift*:

wp-event
 (checked-staged-security-experiment-with-actual-alpha-prefix-data-state A)
 checked-staged-security-with-actual-alpha-prefix-empty-header-relevant-drift
 adversary-initial-state $\leq$
wp-event
 (checked-staged-security-experiment-with-actual-alpha-prefix-data-state A)
 checked-staged-security-with-actual-alpha-prefix-relevant-partial-header-candidate-drift
 adversary-initial-state
by (rule *wp-event-mono*)
 (auto simp:
 checked-staged-security-with-actual-alpha-prefix-empty-header-relevant-drift-def
 checked-staged-security-with-actual-alpha-prefix-relevant-partial-header-candidate-drift-def)

lemma *checked-staged-security-with-actual-alpha-prefix-empty-header-relevant-drift-bound-from-relevant-drift:*

assumes *drift-bound:*
wp-event
 (checked-staged-security-experiment-with-actual-alpha-prefix-data-state
 A)
 checked-staged-security-with-actual-alpha-prefix-relevant-partial-header-candidate-drift
 adversary-initial-state $\leq D$
shows
wp-event
 (checked-staged-security-experiment-with-actual-alpha-prefix-data-state A)
 checked-staged-security-with-actual-alpha-prefix-empty-header-relevant-drift
 adversary-initial-state $\leq D$
by (rule *order-trans*[
 OF
 checked-staged-security-with-actual-alpha-prefix-empty-header-relevant-drift-le-relevant-drift
 drift-bound])

lemma *checked-staged-security-with-actual-alpha-prefix-fixed-empty-alpha-hit-imp-prefix-or-not-prefix-on-support:*

assumes *wf: staged-budget-wellformed budgets*
and *controlled: staged-adversary-controlled budgets A*
and *support:*
 $out \in$
set-dist
 (execute
 (checked-staged-security-experiment-with-actual-alpha-prefix-data-state
 A)
 adversary-initial-state)
and *fixed-hit:*
 checked-staged-security-with-actual-alpha-prefix-verifier-event
 composition-alpha-bad-set-hit-with-empty-composition-header-candidates
 out
shows
 checked-staged-security-with-actual-alpha-prefix-bad-set-hit
 composition-trace-bad-alpha-space $out \vee$
 checked-staged-security-with-actual-alpha-prefix-partial-header-candidate-not-prefix-bound
 composition-trace-bad-alpha-space out

```

proof (cases out)
  case None
  then show ?thesis
    using fixed-hit
    unfolding checked-staged-security-with-actual-alpha-prefix-verifier-event-def
    by simp
next
  case (Some result-pack)
  then obtain prefix prefix-state data attacker-state result final-state
    where out-eq:
      out =
        Some (((((prefix, prefix-state), data), attacker-state),
          result), final-state)
    by (cases result-pack, auto split: prod.splits)
  let ?s =
    verifier-state-from-adversary attacker-state
    (staged-proof-transcript data)
  have verify-out:
    Some (result, final-state) ∈ set-dist (execute verify-monad ?s)
    using support
    unfolding out-eq
      checked-staged-security-experiment-with-actual-alpha-prefix-data-state-def
    by (auto elim!: set-dist-bindE split: prod.splits)
  have local-fixed:
    composition-alpha-bad-set-hit-with-empty-composition-header-candidates ?s
    (Some (result, final-state))
    using fixed-hit
    unfolding out-eq
      checked-staged-security-with-actual-alpha-prefix-verifier-event-def
      Let-def
    by simp
  then obtain fr f-fri-roots f-final as dg final trace-query-idxs
    trace-openings trace-table composition-table rest where empty:
      accepted-with-empty-composition-header-candidates ?s
      (Some (result, final-state)) fr f-fri-roots f-final as dg final
      trace-query-idxs trace-openings trace-table composition-table
  and header:
    verifier-header-transcript ?s fr f-fri-roots f-final as dg []
    final rest
  and trace-partial:
    accepted-with-partial-trace-openings ?s
    (Some (result, final-state)) fr trace-query-idxs trace-openings
  and trace-candidate:
    partial-trace-table-candidate trace-table trace-openings
  and as-bad:
    as ∈ composition-trace-bad-alpha-space trace-table
  unfolding
    composition-alpha-bad-set-hit-with-empty-composition-header-candidates-def
    accepted-with-empty-composition-header-candidates-def

```

```

    by blast
  from verify-monad-accepted-transcript-shape[OF verify-out]
  obtain as' query-idxs where shape':
    accepted-transcript-shape ?s (Some (result, final-state)) as'
    query-idxs
  by blast
  from shape' obtain result' final-state' fr' f-fri-roots' f-final' dg'
    composition-fri-roots' final' rest' where out-shape:
    Some (result, final-state) = Some (result', final-state')
  and header':
    verifier-header-transcript ?s fr' f-fri-roots' f-final' as' dg'
    composition-fri-roots' final' rest'
  by (elim accepted-transcript-shape-header-query-extraction)
  have as'-eq: as' = as
  using verifier-header-transcript-unique[OF header' header] by simp
  have shape:
    accepted-transcript-shape ?s (Some (result, final-state)) as
    query-idxs
  using shape' as'-eq by simp
  have header-union:
    as ∈
    alpha-header-supported-partial-union-bad-sets ?s
    composition-trace-bad-alpha-space fr f-fri-roots f-final
  proof —
  have candidate:
    trace-table ∈
    alpha-header-supported-partial-trace-table-candidates ?s fr
    f-fri-roots f-final
  unfolding alpha-header-supported-partial-trace-table-candidates-def
  by (intro CollectI exI[of - Some (result, final-state)]
    exI[of - trace-query-idxs] exI[of - trace-openings]
    exI[of - as] exI[of - dg] exI[of - []]
    exI[of - final] exI[of - rest] conjI)
    (use verify-out trace-partial trace-candidate header in simp-all)
  show ?thesis
  unfolding alpha-header-supported-partial-union-bad-sets-def
  using candidate as-bad
    composition-trace-bad-alpha-space-subset-alpha-space[of trace-table]
  by auto
qed
  have builder:
    Some (data, attacker-state) ∈
    set-dist
    (execute (checked-staged-transcript-program A)
    adversary-initial-state)
  proof —
  have data-support:
    Some (((data, attacker-state), result), final-state) ∈
    set-dist

```

```

      (execute (checked-staged-security-experiment-with-data-state A)
        adversary-initial-state)
    by (rule
      checked-staged-security-with-actual-alpha-prefix-support-imp-data-state-support
      [OF support[unfolded out-eq]])
  from checked-staged-security-experiment-with-data-state-outcomeE
    [OF data-support]
  show ?thesis
    by blast
qed
have staged-header:
  verifier-header-transcript ?s
    (staged-trace-root data)
    (staged-trace-fri-roots data)
    (staged-trace-final data)
    (staged-alphas data)
    (staged-degree data)
    (staged-composition-fri-roots data)
    (staged-composition-final data)
    (List.concat (staged-query-chunks data))
  by (rule checked-staged-transcript-program-outcome-header-transcript
    [OF wf controlled builder])
have prefix-root-eq: staged-trace-root data = fst prefix
  using support out-eq
  unfolding checked-staged-security-experiment-with-actual-alpha-prefix-data-state-def
    checked-staged-transcript-with-alpha-prefix-program-def
    checked-staged-after-alpha-prefix-program-def Let-def
  by (auto elim!: set-dist-bindE split: prod.splits)
have as-eq: as = staged-alphas data
  using verifier-header-transcript-unique[OF header staged-header]
  by simp
have fr-eq: fr = staged-trace-root data
  using verifier-header-transcript-unique[OF header staged-header]
  by simp
show ?thesis
proof (cases
  as ∈ alpha-prefix-union-bad-sets prefix-state
  composition-trace-bad-alpha-space fr)
case True
have
  checked-staged-security-with-actual-alpha-prefix-bad-set-hit
  composition-trace-bad-alpha-space out
  unfolding out-eq
    checked-staged-security-with-actual-alpha-prefix-bad-set-hit-def
    Let-def
  using True as-eq fr-eq prefix-root-eq by simp
then show ?thesis by simp
next
case not-prefix: False

```

```

have not-bound:
  alpha-header-partial-candidate-not-prefix-bound prefix-state ?s fr
  f-fri-roots f-final
by (rule alpha-header-partial-union-hit-not-prefix-hit-imp-not-prefix-bound
    [OF header-union not-prefix])
have
  checked-staged-security-with-actual-alpha-prefix-partial-header-candidate-not-prefix-bound
  composition-trace-bad-alpha-space out
unfolding out-eq
by (rule
  checked-staged-security-with-actual-alpha-prefix-partial-header-candidate-not-prefix-boundI
    [OF refl shape header header-union not-prefix not-bound])
then show ?thesis by simp
qed
qed

lemma checked-staged-security-with-actual-alpha-prefix-fixed-empty-alpha-hit-le-prefix-or-not-prefix:
assumes wf: staged-budget-wellformed budgets
and controlled: staged-adversary-controlled budgets A
shows
  wp-event
    (checked-staged-security-experiment-with-actual-alpha-prefix-data-state A)
    (checked-staged-security-with-actual-alpha-prefix-verifier-event
      composition-alpha-bad-set-hit-with-empty-composition-header-candidates)
  adversary-initial-state  $\leq$ 
  wp-event
    (checked-staged-security-experiment-with-actual-alpha-prefix-data-state A)
    (checked-staged-security-with-actual-alpha-prefix-bad-set-hit
      composition-trace-bad-alpha-space)
  adversary-initial-state +
  wp-event
    (checked-staged-security-experiment-with-actual-alpha-prefix-data-state A)
    (checked-staged-security-with-actual-alpha-prefix-partial-header-candidate-not-prefix-bound
      composition-trace-bad-alpha-space)
  adversary-initial-state
proof -
let ?M =
  checked-staged-security-experiment-with-actual-alpha-prefix-data-state A
let ?Fixed =
  checked-staged-security-with-actual-alpha-prefix-verifier-event
  composition-alpha-bad-set-hit-with-empty-composition-header-candidates
let ?Prefix =
  checked-staged-security-with-actual-alpha-prefix-bad-set-hit
  composition-trace-bad-alpha-space
let ?NotPrefix =
  checked-staged-security-with-actual-alpha-prefix-partial-header-candidate-not-prefix-bound
  composition-trace-bad-alpha-space
have event-le:
  wp-event ?M ?Fixed adversary-initial-state  $\leq$ 

```

$wp\text{-event } ?M (\lambda out. ?Prefix\ out \vee ?NotPrefix\ out)$
 $adversary\text{-initial-state}$
by (rule $wp\text{-event-mono-on-support}$)
(rule
 $checked\text{-staged-security-with-actual-alpha-prefix-fixed-empty-alpha-hit-imp-prefix-or-not-prefix-on-support}$
 $[OF\ wf\ controlled]$)
also have ... $\leq$
 $wp\text{-event } ?M ?Prefix\ adversary\text{-initial-state} +$
 $wp\text{-event } ?M ?NotPrefix\ adversary\text{-initial-state}$
by (rule $wp\text{-event-union-bound}$)
finally show $?thesis$.
qed

lemma $checked\text{-staged-security-with-actual-alpha-prefix-empty-header-alpha-hit-imp-prefix-prequery-or-empty-relevant}$
assumes $support$:
 $out \in$
 $set\text{-dist}$
(execute
($checked\text{-staged-security-experiment-with-actual-alpha-prefix-data-state}$
 A)
 $adversary\text{-initial-state}$)
and hit :
 $checked\text{-staged-security-with-actual-alpha-prefix-verifier-event}$
 $composition\text{-alpha-partial-opening-union-hit-with-empty-header}\ out$
shows
 $checked\text{-staged-security-with-actual-alpha-prefix-bad-set-hit}$
 $composition\text{-trace-bad-alpha-space}\ out \vee$
 $checked\text{-staged-security-with-actual-alpha-prefix-partial-header-prequery-hit}$
 $composition\text{-trace-bad-alpha-space}\ out \vee$
 $checked\text{-staged-security-with-actual-alpha-prefix-empty-header-relevant-drift}$
 out
proof –
have $partial$:
 $checked\text{-staged-security-with-actual-alpha-prefix-partial-header-bad-set-hit}$
 $composition\text{-trace-bad-alpha-space}\ out$
by (rule
 $checked\text{-staged-security-with-actual-alpha-prefix-empty-header-partial-alpha-hit-imp-partial-header-bad-set-hit-on-}$
 $[OF\ support\ hit]$)
have $fresh\text{-or-prequery}$:
 $checked\text{-staged-security-with-actual-alpha-prefix-partial-header-fresh-hit}$
 $composition\text{-trace-bad-alpha-space}\ out \vee$
 $checked\text{-staged-security-with-actual-alpha-prefix-partial-header-prequery-hit}$
 $composition\text{-trace-bad-alpha-space}\ out$
by (rule
 $checked\text{-staged-security-with-actual-alpha-prefix-partial-header-bad-set-hit-imp-fresh-or-prequery}$
 $[OF\ partial]$)
then show $?thesis$
proof
assume $fresh$:


```

    checked-staged-security-with-actual-alpha-prefix-partial-header-fresh-hit
    composition-trace-bad-alpha-space out
have prefix-or-drift:
    checked-staged-security-with-actual-alpha-prefix-bad-set-hit
    composition-trace-bad-alpha-space out  $\vee$ 
    checked-staged-security-with-actual-alpha-prefix-partial-header-candidate-drift
    composition-trace-bad-alpha-space out
by (rule
    checked-staged-security-with-actual-alpha-prefix-partial-header-fresh-hit-imp-prefix-or-drift
    [OF fresh])
then show ?thesis
proof
  assume
    checked-staged-security-with-actual-alpha-prefix-bad-set-hit
    composition-trace-bad-alpha-space out
  then show ?thesis by simp
next
  assume drift:
    checked-staged-security-with-actual-alpha-prefix-partial-header-candidate-drift
    composition-trace-bad-alpha-space out
  have
    checked-staged-security-with-actual-alpha-prefix-empty-header-relevant-drift
    out
  unfolding
    checked-staged-security-with-actual-alpha-prefix-empty-header-relevant-drift-def
  using drift hit by simp
  then show ?thesis by simp
qed
next
  assume
    checked-staged-security-with-actual-alpha-prefix-partial-header-prequery-hit
    composition-trace-bad-alpha-space out
  then show ?thesis by simp
qed
qed

lemma checked-staged-security-with-actual-alpha-prefix-empty-header-alpha-hit-bound-from-empty-relevant-drift-and-
assumes wf: staged-budget-wellformed budgets
and controlled: staged-adversary-controlled budgets A
and drift-bound:
  wp-event
  (checked-staged-security-experiment-with-actual-alpha-prefix-data-state
  A)
  checked-staged-security-with-actual-alpha-prefix-empty-header-relevant-drift
  adversary-initial-state  $\leq D$ 
shows
  wp-event
  (checked-staged-security-experiment-with-actual-alpha-prefix-data-state A)
  (checked-staged-security-with-actual-alpha-prefix-verifier-event

```

```

    composition-alpha-partial-opening-union-hit-with-empty-header)
  adversary-initial-state ≤
  composition-error-bound +
  staged-phase-relation-error size
  (staged-alpha-search-queries budgets 0) +
  hash-collision-budget-value 0
  (staged-alpha-search-queries budgets 0) +
  staged-phase-relation-error size
  (staged-alpha-search-queries budgets 0) +
  D
proof –
  let ?M =
    checked-staged-security-experiment-with-actual-alpha-prefix-data-state A
  let ?Prefix =
    checked-staged-security-with-actual-alpha-prefix-bad-set-hit
    composition-trace-bad-alpha-space
  let ?Prequery =
    checked-staged-security-with-actual-alpha-prefix-partial-header-prequery-hit
    composition-trace-bad-alpha-space
  let ?Drift =
    checked-staged-security-with-actual-alpha-prefix-empty-header-relevant-drift
  let ?Hit =
    checked-staged-security-with-actual-alpha-prefix-verifier-event
    composition-alpha-partial-opening-union-hit-with-empty-header
  let ?P =
    composition-error-bound +
    staged-phase-relation-error size
    (staged-alpha-search-queries budgets 0) +
    hash-collision-budget-value 0
    (staged-alpha-search-queries budgets 0)
  let ?Q =
    staged-phase-relation-error size
    (staged-alpha-search-queries budgets 0)
  have transcript-bound:
    wp-event (checked-staged-transcript-with-alpha-prefix-program A)
    (checked-staged-transcript-actual-alpha-prefix-bad-set-hit
     composition-trace-bad-alpha-space)
    adversary-initial-state ≤ ?P
  by (rule
    checked-staged-transcript-with-alpha-prefix-actual-bad-set-hit-bound-from-budgets
    [OF wf controlled])
  have prefix-bound:
    wp-event ?M ?Prefix adversary-initial-state ≤ ?P
  by (rule
    checked-staged-security-with-actual-alpha-prefix-bad-set-hit-bound-from-transcript
    [OF transcript-bound])
  have prequery-bound:
    wp-event ?M ?Prequery adversary-initial-state ≤ ?Q
  by (rule

```

```

    checked-staged-security-with-actual-alpha-prefix-vector-alpha-prequery-accounting
    [OF wf controlled])
have wp-event ?M ?Hit adversary-initial-state ≤
    wp-event ?M
    (λout. ?Prefix out ∨ ?Prequery out ∨ ?Drift out ∨ False)
    adversary-initial-state
by (rule wp-event-mono-on-support)
    (use
    checked-staged-security-with-actual-alpha-prefix-empty-header-alpha-hit-imp-prefix-prequery-or-empty-relevant-drift
    [of - A] in blast)
also have ... ≤
    wp-event ?M ?Prefix adversary-initial-state +
    wp-event ?M ?Prequery adversary-initial-state +
    wp-event ?M ?Drift adversary-initial-state +
    wp-event ?M (λ-. False) adversary-initial-state
by (rule wp-event-union-bound4)
also have ... ≤ ?P + ?Q + D + 0
by (intro add-mono prefix-bound prequery-bound drift-bound)
    (simp add: wp-event-def wp-def dist-expect-def)
finally show ?thesis
by (simp add: algebra-simps)
qed

lemma checked-staged-security-with-data-state-empty-header-composition-randomization-bound-from-empty-relevant-drift
assumes wf: staged-budget-wellformed budgets
and controlled: staged-adversary-controlled budgets A
and drift-bound:
    wp-event
    (checked-staged-security-experiment-with-actual-alpha-prefix-data-state
    A)
    checked-staged-security-with-actual-alpha-prefix-empty-header-relevant-drift
    adversary-initial-state ≤ D
shows
    wp-event (checked-staged-security-experiment-with-data-state A)
    (staged-security-with-data-state-verifier-event
    composition-randomization-bad-with-empty-composition-header-candidates)
    adversary-initial-state ≤
    composition-error-bound +
    staged-phase-relation-error size
    (staged-alpha-search-queries budgets 0) +
    hash-collision-budget-value 0
    (staged-alpha-search-queries budgets 0) +
    staged-phase-relation-error size
    (staged-alpha-search-queries budgets 0) +
    D
proof -
let ?M = checked-staged-security-experiment-with-data-state A
let ?random =
    staged-security-with-data-state-verifier-event

```

composition-randomization-bad-with-empty-composition-header-candidates
let ?alpha =
staged-security-with-data-state-verifier-event
composition-alpha-partial-opening-union-hit-with-empty-header
have alpha-projected:
wp-event ?M ?alpha *adversary-initial-state* =
wp-event
(checked-staged-security-experiment-with-actual-alpha-prefix-data-state A)
(checked-staged-security-with-actual-alpha-prefix-verifier-event
composition-alpha-partial-opening-union-hit-with-empty-header)
adversary-initial-state
by (rule
checked-staged-security-with-data-state-verifier-event-actual-alpha-prefix-projection)
have alpha-bound:
wp-event ?M ?alpha *adversary-initial-state* ≤
composition-error-bound +
staged-phase-relation-error size
(staged-alpha-search-queries budgets 0) +
hash-collision-budget-value 0
(staged-alpha-search-queries budgets 0) +
staged-phase-relation-error size
(staged-alpha-search-queries budgets 0) +
D
unfolding alpha-projected
by (rule
checked-staged-security-with-actual-alpha-prefix-empty-header-alpha-hit-bound-from-empty-relevant-drift-and-bud
[OF wf controlled drift-bound])
have *wp-event* ?M ?random *adversary-initial-state* ≤
wp-event ?M ?alpha *adversary-initial-state*
by (rule *wp-event-mono*)
(auto simp: staged-security-with-data-state-verifier-event-def
intro:
composition-alpha-bad-set-hit-with-empty-header-imp-partial-opening-union-hit
composition-randomization-bad-with-empty-composition-header-candidates-imp-alpha-hit
split: option.splits prod.splits)
also have ... ≤
composition-error-bound +
staged-phase-relation-error size
(staged-alpha-search-queries budgets 0) +
hash-collision-budget-value 0
(staged-alpha-search-queries budgets 0) +
staged-phase-relation-error size
(staged-alpha-search-queries budgets 0) +
D
by (rule alpha-bound)
finally show ?thesis .
qed

lemma *checked-staged-security-with-data-state-empty-header-bad-event-bound-from-empty-relevant-drift-trace-fri-and*

```

assumes false-statement:  $\neg$  exists-valid-trace
and wf: staged-budget-wellformed budgets
and controlled: staged-adversary-controlled budgets A
and drift-bound:
  wp-event
    (checked-staged-security-experiment-with-actual-alpha-prefix-data-state
      A)
    checked-staged-security-with-actual-alpha-prefix-empty-header-relevant-drift
    adversary-initial-state  $\leq$  D
and trace-fri-bound:
  wp-event (checked-staged-security-experiment-with-data-state A)
    (staged-security-with-data-state-verifier-event
      trace-fri-bad-with-empty-composition-header-candidates)
    adversary-initial-state  $\leq$  trace-fri-error
and query-bound:
  wp-event (checked-staged-security-experiment-with-data-state A)
    (staged-security-with-data-state-verifier-event
      query-bad-with-empty-composition-header-candidates)
    adversary-initial-state  $\leq$  empty-query-error
shows
  wp-event (checked-staged-security-experiment-with-data-state A)
    (staged-security-with-data-state-verifier-event
      soundness-bad-event-partial-candidate-empty-header)
    adversary-initial-state  $\leq$ 
      composition-error-bound +
      staged-phase-relation-error size
      (staged-alpha-search-queries budgets 0) +
      hash-collision-budget-value 0
      (staged-alpha-search-queries budgets 0) +
      staged-phase-relation-error size
      (staged-alpha-search-queries budgets 0) +
      D + trace-fri-error + empty-query-error
proof –
  let ?M = checked-staged-security-experiment-with-data-state A
  let ?E = staged-security-with-data-state-verifier-event
  let ?degree =
    ?E composition-degree-bad-with-empty-composition-header-candidates
  let ?random =
    ?E composition-randomization-bad-with-empty-composition-header-candidates
  let ?trace =
    ?E trace-fri-bad-with-empty-composition-header-candidates
  let ?query =
    ?E query-bad-with-empty-composition-header-candidates
  let ?bad = ?E soundness-bad-event-partial-candidate-empty-header
  have degree-false: ?degree = ( $\lambda$ out. False)
  by (rule ext)
    (auto simp: staged-security-with-data-state-verifier-event-def
      composition-degree-bad-with-empty-composition-header-candidates-false
      split: option.splits prod.splits)

```

```

have degree-bound:
  wp-event ?M ?degree adversary-initial-state ≤ 0
  unfolding degree-false wp-event-def wp-def dist-expect-def by simp
have random-bound:
  wp-event ?M ?random adversary-initial-state ≤
    composition-error-bound +
    staged-phase-relation-error size
    (staged-alpha-search-queries budgets 0) +
    hash-collision-budget-value 0
    (staged-alpha-search-queries budgets 0) +
    staged-phase-relation-error size
    (staged-alpha-search-queries budgets 0) +
    D
  by (rule
    checked-staged-security-with-data-state-empty-header-composition-randomization-bound-from-empty-relevant-dri
    [OF wf controlled drift-bound])
have wp-event ?M ?bad adversary-initial-state ≤
  wp-event ?M
    (λout. ?degree out ∨ ?random out ∨ ?trace out ∨
      ?query out)
    adversary-initial-state
proof (rule wp-event-mono)
  fix out
  assume bad: ?bad out
  show ?degree out ∨ ?random out ∨ ?trace out ∨ ?query out
  proof (cases out)
    case None
    then show ?thesis
      using bad
      unfolding staged-security-with-data-state-verifier-event-def
      by simp
  next
  case (Some packed)
  then obtain data attacker-state result final-state where out-eq:
    out = Some (((data, attacker-state), result), final-state)
    by (cases packed, auto split: prod.splits)
  let ?s =
    verifier-state-from-adversary attacker-state
    (staged-proof-transcript data)
  have local-bad:
    soundness-bad-event-partial-candidate-empty-header ?s
    (Some (result, final-state))
    using bad
    unfolding out-eq staged-security-with-data-state-verifier-event-def
    by simp
  then consider
    (comp)
    composition-bad-with-empty-composition-header-candidates ?s
    (Some (result, final-state))

```

```

| (trace)
  trace-fri-bad-with-empty-composition-header-candidates ?s
  (Some (result, final-state))
| (query)
  query-bad-with-empty-composition-header-candidates ?s
  (Some (result, final-state))
unfolding soundness-bad-event-partial-candidate-empty-header-def
by blast
then show ?thesis
proof cases
case comp
then have comp-split:
  composition-degree-bad-with-empty-composition-header-candidates ?s
  (Some (result, final-state))  $\vee$ 
  composition-randomization-bad-with-empty-composition-header-candidates
  ?s (Some (result, final-state))
by (rule
  composition-bad-with-empty-composition-header-candidates-split
  [OF false-statement])
then show ?thesis
  unfolding out-eq staged-security-with-data-state-verifier-event-def
  using comp-split by (auto split: option.splits prod.splits)
next
case trace
then show ?thesis
  unfolding out-eq staged-security-with-data-state-verifier-event-def
  by simp
next
case query
then show ?thesis
  unfolding out-eq staged-security-with-data-state-verifier-event-def
  by simp
qed
qed
qed
also have ...  $\leq$ 
  wp-event ?M ?degree adversary-initial-state +
  wp-event ?M ?random adversary-initial-state +
  wp-event ?M ?trace adversary-initial-state +
  wp-event ?M ?query adversary-initial-state
by (rule wp-event-union-bound4)
also have ...  $\leq$ 
  0 +
  (composition-error-bound +
  staged-phase-relation-error size
  (staged-alpha-search-queries budgets 0) +
  hash-collision-budget-value 0
  (staged-alpha-search-queries budgets 0) +
  staged-phase-relation-error size

```

(staged-alpha-search-queries budgets 0) +
 D) +
 trace-fri-error + empty-query-error
 by (intro add-mono degree-bound random-bound trace-fri-bound query-bound)
 finally show ?thesis
 by (simp add: algebra-simps)
 qed

lemma checked-staged-soundness-from-aligned-transcript-aligned-randomization-components-and-current-query-prefix
fixes trace-fri-error' composition-fri-error' D empty-query-error :: prob
and C :: nat $\Rightarrow$ prob
assumes false-statement: $\neg$ exists-valid-trace
and wf: staged-budget-wellformed budgets
and controlled: staged-adversary-controlled budgets A
and trace-fri-bound:
 wp-event (checked-staged-security-experiment-with-data-state A)
 (staged-security-with-data-state-verifier-event
 trace-fri-bad-with-partial-candidates)
 adversary-initial-state $\leq$ trace-fri-error'
and comp-fri-bound:
 wp-event (checked-staged-security-experiment-with-data-state A)
 (staged-security-with-data-state-verifier-event
 composition-fri-bad-with-partial-candidates)
 adversary-initial-state $\leq$ composition-fri-error'
and round-bound:
 $\bigwedge i. i < \text{rounds} \implies$
 wp-event
 (checked-staged-security-experiment-with-query-prefix-data-state A i)
 (checked-staged-security-with-query-prefix-current-query-partial-opening-hit-at i)
 adversary-initial-state $\leq$ C i
and drift-bound:
 wp-event
 (checked-staged-security-experiment-with-actual-alpha-prefix-data-state
 A)
 checked-staged-security-with-actual-alpha-prefix-empty-header-relevant-drift
 adversary-initial-state $\leq$ D
and empty-trace-fri-bound:
 wp-event (checked-staged-security-experiment-with-data-state A)
 (staged-security-with-data-state-verifier-event
 trace-fri-bad-with-empty-composition-header-candidates)
 adversary-initial-state $\leq$ trace-fri-error
and current-empty-bound:
 wp-event (checked-staged-security-experiment-with-data-state A)
 (staged-security-with-data-state-current-empty-query-opening-hit-at 0)
 adversary-initial-state $\leq$ empty-query-error
shows checked-staged-adversary-acceptance-probability A $\leq$
 hash-collision-budget-value 0
 (staged-attacker-query-budget budgets +
 staged-challenge-query-budget + verifier-hash-query-budget) +

$(\sum_{i < \text{rounds}} C\ i) +$
 $\text{trace-fri-error}' + \text{composition-fri-error}' +$
 $(\sum_{i < \text{rounds}} C\ i) +$
 $(\text{composition-error-bound} +$
 $\text{staged-phase-relation-error size}$
 $(\text{staged-alpha-search-queries budgets } 0) +$
 $\text{hash-collision-budget-value } 0$
 $(\text{staged-alpha-search-queries budgets } 0) +$
 $\text{staged-phase-relation-error size}$
 $(\text{staged-alpha-search-queries budgets } 0) +$
 $D + \text{trace-fri-error} + \text{empty-query-error})$
proof –
have *comp-bound*:
 $\text{wp-event } (\text{checked-staged-security-experiment-with-data-state } A)$
 $(\text{staged-security-with-data-state-verifier-event}$
 $\text{composition-randomization-bad-with-aligned-transcript-partial-candidates})$
 $\text{adversary-initial-state} \leq (\sum_{i < \text{rounds}} C\ i)$
by (rule
 $\text{checked-staged-security-with-data-state-aligned-partial-candidate-randomization-bound-from-query-prefix-current-}$
 $[\text{OF wf controlled round-bound}])$
have *current-query-bound*:
 $\text{wp-event } (\text{checked-staged-security-experiment-with-data-state } A)$
 $\text{staged-security-with-data-state-current-query-partial-opening-hit}$
 $\text{adversary-initial-state} \leq (\sum_{i < \text{rounds}} C\ i)$
by (rule
 $\text{checked-staged-security-with-data-state-current-query-partial-opening-hit-bound-from-query-prefix-rounds}$
 $[\text{OF round-bound wf controlled}])$
have *query-bound*:
 $\text{wp-event } (\text{checked-staged-security-experiment-with-data-state } A)$
 $(\text{staged-security-with-data-state-verifier-event}$
 $\text{query-bad-with-aligned-transcript-partial-candidates})$
 $\text{adversary-initial-state} \leq (\sum_{i < \text{rounds}} C\ i)$
by (rule *order-trans*[
 $\text{OF checked-staged-security-with-data-state-aligned-transcript-query-bad-bound-from-current-partial-opening-hit}$
 $\text{current-query-bound}])$
 $(\text{use wf controlled in simp-all})$
have *merkle-bound*:
 $\text{wp-event } (\text{checked-staged-security-experiment-with-data-state } A)$
 $(\text{staged-security-with-data-state-verifier-event}$
 $\text{partial-merkle-inconsistency-bad})$
 $\text{adversary-initial-state} \leq$
 $\text{hash-collision-budget-value } 0$
 $(\text{staged-attacker-query-budget budgets } +$
 $\text{staged-challenge-query-budget } + \text{verifier-hash-query-budget})$
by (rule
 $\text{checked-staged-security-with-data-state-partial-merkle-inconsistency-bad-bound-controlled}$
 $[\text{OF wf controlled}])$
have *empty-query-bound*:
 $\text{wp-event } (\text{checked-staged-security-experiment-with-data-state } A)$

(staged-security-with-data-state-verifier-event
 query-bad-with-empty-composition-header-candidates)
 adversary-initial-state $\leq$ empty-query-error
by (rule order-trans[
 OF checked-staged-security-with-data-state-empty-query-bad-le-current-empty-opening-hit-at-zero[
 OF wf controlled] current-empty-bound])
have empty-header-bound:
 wp-event (checked-staged-security-experiment-with-data-state A)
 (staged-security-with-data-state-verifier-event
 soundness-bad-event-partial-candidate-empty-header)
 adversary-initial-state $\leq$
 composition-error-bound +
 staged-phase-relation-error size
 (staged-alpha-search-queries budgets 0) +
 hash-collision-budget-value 0
 (staged-alpha-search-queries budgets 0) +
 staged-phase-relation-error size
 (staged-alpha-search-queries budgets 0) +
 D + trace-fri-error + empty-query-error
by (rule
 checked-staged-security-with-data-state-empty-header-bad-event-bound-from-empty-relevant-drift-trace-fri-and-qu
 [OF false-statement wf controlled drift-bound empty-trace-fri-bound
 empty-query-bound])
have bound:
 checked-staged-adversary-acceptance-probability A $\leq$
 hash-collision-budget-value 0
 (staged-attacker-query-budget budgets +
 staged-challenge-query-budget + verifier-hash-query-budget) +
 ($\sum i < \text{rounds}. C\ i$) +
 trace-fri-error' + composition-fri-error' +
 ($\sum i < \text{rounds}. C\ i$) +
 (composition-error-bound +
 staged-phase-relation-error size
 (staged-alpha-search-queries budgets 0) +
 hash-collision-budget-value 0
 (staged-alpha-search-queries budgets 0) +
 staged-phase-relation-error size
 (staged-alpha-search-queries budgets 0) +
 D + trace-fri-error + empty-query-error)
by (rule
 checked-staged-soundness-from-aligned-transcript-partial-candidate-aligned-randomization-components-and-empt
 [OF false-statement merkle-bound comp-bound trace-fri-bound
 comp-fri-bound query-bound empty-header-bound])
then show ?thesis .
qed

lemma checked-staged-soundness-from-aligned-transcript-aligned-randomization-components-and-structured-query-fr
fixes trace-fri-error' composition-fri-error' D empty-query-error
 header-query-error :: prob

and $C :: \text{nat} \Rightarrow \text{prob}$
assumes *false-statement*: $\neg \text{exists-valid-trace}$
and *wf*: *staged-budget-wellformed budgets*
and *controlled*: *staged-adversary-controlled budgets A*
and *trace-fri-bound*:
 wp-event (*checked-staged-security-experiment-with-data-state A*)
 (*staged-security-with-data-state-verifier-event*
 trace-fri-bad-with-partial-candidates)
 adversary-initial-state $\leq$ *trace-fri-error'*
and *comp-fri-bound*:
 wp-event (*checked-staged-security-experiment-with-data-state A*)
 (*staged-security-with-data-state-verifier-event*
 composition-fri-bad-with-partial-candidates)
 adversary-initial-state $\leq$ *composition-fri-error'*
and *round-bound*:
 $\bigwedge i. i < \text{rounds} \implies$
 wp-event
 (*checked-staged-security-experiment-with-query-prefix-data-state A i*)
 (*checked-staged-security-with-query-prefix-current-query-partial-opening-hit-at i*)
 adversary-initial-state $\leq C i$
and *header-query-bound*:
 wp-event
 (*checked-staged-security-experiment-with-actual-alpha-prefix-data-state*
 A)
 (*checked-staged-security-with-actual-alpha-prefix-header-authenticated-candidate-opening-query-hit-from-prefix*
 A)
 adversary-initial-state $\leq$ *header-query-error*
and *drift-bound*:
 wp-event
 (*checked-staged-security-experiment-with-actual-alpha-prefix-data-state*
 A)
 checked-staged-security-with-actual-alpha-prefix-empty-header-relevant-drift
 adversary-initial-state $\leq D$
and *empty-trace-fri-bound*:
 wp-event (*checked-staged-security-experiment-with-data-state A*)
 (*staged-security-with-data-state-verifier-event*
 trace-fri-bad-with-empty-composition-header-candidates)
 adversary-initial-state $\leq$ *trace-fri-error*
and *current-empty-bound*:
 wp-event (*checked-staged-security-experiment-with-data-state A*)
 (*staged-security-with-data-state-current-empty-query-opening-hit-at 0*)
 adversary-initial-state $\leq$ *empty-query-error*
shows *checked-staged-adversary-acceptance-probability A* $\leq$
 hash-collision-budget-value 0
 (*staged-attacker-query-budget budgets* +
 staged-challenge-query-budget + *verifier-hash-query-budget*) +
 $(\sum i < \text{rounds}. C i)$ +
 trace-fri-error' + composition-fri-error' +
 header-query-error +

(composition-error-bound +
 staged-phase-relation-error size
 (staged-alpha-search-queries budgets 0) +
 hash-collision-budget-value 0
 (staged-alpha-search-queries budgets 0) +
 staged-phase-relation-error size
 (staged-alpha-search-queries budgets 0) +
 D + trace-fri-error + empty-query-error)
proof –
have comp-bound:
 wp-event (checked-staged-security-experiment-with-data-state A)
 (staged-security-with-data-state-verifier-event
 composition-randomization-bad-with-aligned-transcript-partial-candidates)
 adversary-initial-state $\leq (\sum i < \text{rounds}. C\ i)$
by (rule
 checked-staged-security-with-data-state-aligned-partial-candidate-randomization-bound-from-query-prefix-current-
 [OF wf controlled round-bound])
have query-bound:
 wp-event (checked-staged-security-experiment-with-data-state A)
 (staged-security-with-data-state-verifier-event
 query-bad-with-aligned-transcript-partial-candidates)
 adversary-initial-state $\leq$ header-query-error
by (rule
 checked-staged-security-with-data-state-aligned-transcript-query-bad-bound-from-header-authenticated-candidate-
 [OF wf controlled header-query-bound])
have merkle-bound:
 wp-event (checked-staged-security-experiment-with-data-state A)
 (staged-security-with-data-state-verifier-event
 partial-merkle-inconsistency-bad)
 adversary-initial-state $\leq$
 hash-collision-budget-value 0
 (staged-attacker-query-budget budgets +
 staged-challenge-query-budget + verifier-hash-query-budget)
by (rule
 checked-staged-security-with-data-state-partial-merkle-inconsistency-bad-bound-controlled
 [OF wf controlled])
have empty-query-bound:
 wp-event (checked-staged-security-experiment-with-data-state A)
 (staged-security-with-data-state-verifier-event
 query-bad-with-empty-composition-header-candidates)
 adversary-initial-state $\leq$ empty-query-error
by (rule order-trans[
 OF checked-staged-security-with-data-state-empty-query-bad-le-current-empty-opening-hit-at-zero[
 OF wf controlled] current-empty-bound])
have empty-header-bound:
 wp-event (checked-staged-security-experiment-with-data-state A)
 (staged-security-with-data-state-verifier-event
 soundness-bad-event-partial-candidate-empty-header)
 adversary-initial-state $\leq$

composition-error-bound +
staged-phase-relation-error size
 (*staged-alpha-search-queries budgets 0*) +
hash-collision-budget-value 0
 (*staged-alpha-search-queries budgets 0*) +
staged-phase-relation-error size
 (*staged-alpha-search-queries budgets 0*) +
D + trace-fri-error + empty-query-error
by (rule
checked-staged-security-with-data-state-empty-header-bad-event-bound-from-empty-relevant-drift-trace-fri-and-qu
 [OF false-statement wf controlled drift-bound empty-trace-fri-bound
 empty-query-bound])
have bound:
checked-staged-adversary-acceptance-probability A ≤
 (*hash-collision-budget-value 0*
 (*staged-attacker-query-budget budgets +*
staged-challenge-query-budget + verifier-hash-query-budget) +
 ($\sum i < \text{rounds}. C\ i$) + *trace-fri-error' +*
composition-fri-error' + header-query-error) +
 (*composition-error-bound +*
staged-phase-relation-error size
 (*staged-alpha-search-queries budgets 0*) +
hash-collision-budget-value 0
 (*staged-alpha-search-queries budgets 0*) +
staged-phase-relation-error size
 (*staged-alpha-search-queries budgets 0*) +
D + trace-fri-error + empty-query-error)
by (rule
checked-staged-soundness-from-aligned-transcript-partial-candidate-aligned-randomization-components-and-empt
 [OF false-statement merkle-bound comp-bound trace-fri-bound
 comp-fri-bound query-bound empty-header-bound])
then show ?thesis
by (simp add: algebra-simps)
qed

lemma *checked-staged-soundness-from-aligned-transcript-aligned-randomization-components-and-current-query-prefix*
fixes *trace-fri-error' composition-fri-error' D empty-query-error :: prob*
and *C :: nat ⇒ prob*
assumes *false-statement: ¬ exists-valid-trace*
and *wf: staged-budget-wellformed budgets*
and *controlled: staged-adversary-controlled budgets A*
and *trace-fri-bound:*
wp-event (checked-staged-security-experiment-with-data-state A)
(staged-security-with-data-state-verifier-event
trace-fri-bad-with-partial-candidates)
adversary-initial-state ≤ trace-fri-error'
and *comp-fri-bound:*
wp-event (checked-staged-security-experiment-with-data-state A)
(staged-security-with-data-state-verifier-event

composition-fri-bad-with-partial-candidates
adversary-initial-state $\leq$ *composition-fri-error'*
and *round-bound*:
 $\bigwedge i. i < \text{rounds} \implies$
wp-event
(checked-staged-security-experiment-with-query-prefix-data-state A i)
(checked-staged-security-with-query-prefix-current-query-partial-opening-hit-at i)
adversary-initial-state $\leq C i$
and *drift-bound*:
wp-event
(checked-staged-security-experiment-with-actual-alpha-prefix-data-state A)
checked-staged-security-with-actual-alpha-prefix-empty-header-relevant-drift
adversary-initial-state $\leq D$
and *empty-trace-fri-bound*:
wp-event (checked-staged-security-experiment-with-data-state A)
(staged-security-with-data-state-verifier-event
trace-fri-bad-with-empty-composition-header-candidates)
adversary-initial-state $\leq$ *trace-fri-error*
and *current-empty-bound*:
wp-event (checked-staged-security-experiment-with-data-state A)
(staged-security-with-data-state-current-empty-query-opening-hit-at 0)
adversary-initial-state $\leq$ *empty-query-error*
shows *checked-staged-adversary-acceptance-probability A* $\leq$
hash-collision-budget-value 0
(staged-attacker-query-budget budgets +
staged-challenge-query-budget + verifier-hash-query-budget) +
 $(\sum i < \text{rounds}. C i) +$
trace-fri-error' + composition-fri-error' +
(composition-error-bound +
staged-phase-relation-error size
(staged-alpha-search-queries budgets 0) +
hash-collision-budget-value 0
(staged-alpha-search-queries budgets 0) +
staged-phase-relation-error size
(staged-alpha-search-queries budgets 0) +
D + trace-fri-error + empty-query-error)
proof –
have *current-query-bound*:
wp-event (checked-staged-security-experiment-with-data-state A)
staged-security-with-data-state-current-query-partial-opening-hit
adversary-initial-state $\leq (\sum i < \text{rounds}. C i)$
by (rule
checked-staged-security-with-data-state-current-query-partial-opening-hit-bound-from-query-prefix-rounds
[OF round-bound wf controlled])
have *merkle-bound*:
wp-event (checked-staged-security-experiment-with-data-state A)
(staged-security-with-data-state-verifier-event
partial-merkle-inconsistency-bad)

$adversary-initial-state \leq$
 $hash-collision-budget-value\ 0$
 $(staged-attacker-query-budget\ budgets +$
 $staged-challenge-query-budget + verifier-hash-query-budget)$
by (rule
 $checked-staged-security-with-data-state-partial-merkle-inconsistency-bad-bound-controlled$
 $[OF\ wf\ controlled])$
have *empty-query-bound*:
 $wp-event\ (checked-staged-security-experiment-with-data-state\ A)$
 $(staged-security-with-data-state-verifier-event$
 $query-bad-with-empty-composition-header-candidates)$
 $adversary-initial-state \leq empty-query-error$
by (rule *order-trans*[
 $OF\ checked-staged-security-with-data-state-empty-query-bad-le-current-empty-opening-hit-at-zero[$
 $OF\ wf\ controlled]\ current-empty-bound])$
have *empty-header-bound*:
 $wp-event\ (checked-staged-security-experiment-with-data-state\ A)$
 $(staged-security-with-data-state-verifier-event$
 $soundness-bad-event-partial-candidate-empty-header)$
 $adversary-initial-state \leq$
 $composition-error-bound +$
 $staged-phase-relation-error\ size$
 $(staged-alpha-search-queries\ budgets\ 0) +$
 $hash-collision-budget-value\ 0$
 $(staged-alpha-search-queries\ budgets\ 0) +$
 $staged-phase-relation-error\ size$
 $(staged-alpha-search-queries\ budgets\ 0) +$
 $D + trace-fri-error + empty-query-error$
by (rule
 $checked-staged-security-with-data-state-empty-header-bad-event-bound-from-empty-relevant-drift-trace-fri-and-qu$
 $[OF\ false-statement\ wf\ controlled\ drift-bound\ empty-trace-fri-bound$
 $empty-query-bound])$
have *partial-candidate-bound*:
 $wp-event\ (checked-staged-security-experiment-with-data-state\ A)$
 $(staged-security-with-data-state-verifier-event$
 $soundness-bad-event-aligned-transcript-partial-candidate-with-aligned-randomization-and-merkle)$
 $adversary-initial-state \leq$
 $hash-collision-budget-value\ 0$
 $(staged-attacker-query-budget\ budgets +$
 $staged-challenge-query-budget + verifier-hash-query-budget) +$
 $(\sum i < rounds.\ C\ i) + trace-fri-error' + composition-fri-error'$
by (rule
 $checked-staged-security-with-data-state-aligned-transcript-partial-candidate-aligned-randomization-with-merkle-c$
 $[OF\ wf\ controlled\ merkle-bound\ current-query-bound\ trace-fri-bound$
 $comp-fri-bound])$
have *bound*:
 $checked-staged-adversary-acceptance-probability\ A \leq$
 $(hash-collision-budget-value\ 0$
 $(staged-attacker-query-budget\ budgets +$

```

    staged-challenge-query-budget + verifier-hash-query-budget) +
    (∑ i<rounds. C i) + trace-fri-error' +
    composition-fri-error') +
    (composition-error-bound +
    staged-phase-relation-error size
    (staged-alpha-search-queries budgets 0) +
    hash-collision-budget-value 0
    (staged-alpha-search-queries budgets 0) +
    staged-phase-relation-error size
    (staged-alpha-search-queries budgets 0) +
    D + trace-fri-error + empty-query-error)
  by (rule
    checked-staged-soundness-from-aligned-transcript-partial-candidate-aligned-randomization-and-empty-header-bou
    [OF false-statement partial-candidate-bound empty-header-bound])
  then show ?thesis
  by (simp add: algebra-simps)
qed

end

end

```

```

theory Soundness-Relevant-Drift-Public
imports
  Soundness
  Soundness-Staged-Empty-Relevant-Drift
begin

```

Public migration wrappers for the refined relevant-drift route.

These theorems sit downstream of *Soundness* so the main public compatibility layer does not keep growing. The route below derives the aligned composition-randomization branch and aligned query branch from the same per-round current-query prefix bounds. It exposes only the empty-header-specific relevant-drift event and keeps the partial-candidate FRI premises internal to this route.

```

context soundness
begin

```

```

theorem stark-soundness-from-aligned-partial-components-current-query-prefix-and-current-empty-query:
fixes trace-fri-error' composition-fri-error' D empty-query-error :: prob
and C :: nat ⇒ prob
assumes false-statement: ¬ exists-valid-trace
and wf: staged-budget-wellformed budgets
and controlled: staged-adversary-controlled budgets A
and trace-fri-bound:
  wp-event (checked-staged-security-experiment-with-data-state A)
  (staged-security-with-data-state-verifier-event
   trace-fri-bad-with-partial-candidates)

```


$\text{adversary-initial-state} \leq \text{trace-fri-error}'$
and *comp-fri-bound*:
 $\text{wp-event } (\text{checked-staged-security-experiment-with-data-state } A)$
 $(\text{staged-security-with-data-state-verifier-event}$
 $\text{composition-fri-bad-with-partial-candidates})$
 $\text{adversary-initial-state} \leq \text{composition-fri-error}'$
and *round-bound*:
 $\bigwedge i. i < \text{rounds} \implies$
 wp-event
 $(\text{checked-staged-security-experiment-with-query-prefix-data-state } A \ i)$
 $(\text{checked-staged-security-with-query-prefix-current-query-partial-opening-hit-at } i)$
 $\text{adversary-initial-state} \leq C \ i$
and *drift-bound*:
 wp-event
 $(\text{checked-staged-security-experiment-with-actual-alpha-prefix-data-state}$
 $A)$
 $\text{checked-staged-security-with-actual-alpha-prefix-empty-header-relevant-drift}$
 $\text{adversary-initial-state} \leq D$
and *empty-trace-fri-bound*:
 $\text{wp-event } (\text{checked-staged-security-experiment-with-data-state } A)$
 $(\text{staged-security-with-data-state-verifier-event}$
 $\text{trace-fri-bad-with-empty-composition-header-candidates})$
 $\text{adversary-initial-state} \leq \text{trace-fri-error}$
and *current-empty-bound*:
 $\text{wp-event } (\text{checked-staged-security-experiment-with-data-state } A)$
 $(\text{staged-security-with-data-state-current-empty-query-opening-hit-at } 0)$
 $\text{adversary-initial-state} \leq \text{empty-query-error}$
shows $\text{checked-staged-adversary-acceptance-probability } A \leq$
 $\text{hash-collision-budget-value } 0$
 $(\text{staged-attacker-query-budget budgets} +$
 $\text{staged-challenge-query-budget} + \text{verifier-hash-query-budget}) +$
 $(\sum i < \text{rounds}. C \ i) +$
 $\text{trace-fri-error}' + \text{composition-fri-error}' +$
 $(\sum i < \text{rounds}. C \ i) +$
 $(\text{composition-error-bound} +$
 $\text{staged-phase-relation-error size}$
 $(\text{staged-alpha-search-queries budgets } 0) +$
 $\text{hash-collision-budget-value } 0$
 $(\text{staged-alpha-search-queries budgets } 0) +$
 $D +$
 $\text{staged-phase-relation-error size}$
 $(\text{staged-alpha-search-queries budgets } 0) +$
 $\text{trace-fri-error} + \text{empty-query-error})$
proof –
have *bound*:
 $\text{checked-staged-adversary-acceptance-probability } A \leq$
 $\text{hash-collision-budget-value } 0$
 $(\text{staged-attacker-query-budget budgets} +$
 $\text{staged-challenge-query-budget} + \text{verifier-hash-query-budget}) +$

$(\sum_{i < \text{rounds}} C \ i) +$
 $\text{trace-fri-error}' + \text{composition-fri-error}' +$
 $(\sum_{i < \text{rounds}} C \ i) +$
 $(\text{composition-error-bound} +$
 $\text{staged-phase-relation-error size}$
 $(\text{staged-alpha-search-queries budgets } 0) +$
 $\text{hash-collision-budget-value } 0$
 $(\text{staged-alpha-search-queries budgets } 0) +$
 $\text{staged-phase-relation-error size}$
 $(\text{staged-alpha-search-queries budgets } 0) +$
 $D + \text{trace-fri-error} + \text{empty-query-error})$
by (rule
 $\text{checked-staged-soundness-from-aligned-transcript-aligned-randomization-components-and-current-query-prefix-ro}$
 $[OF \text{ false-statement wf controlled trace-fri-bound comp-fri-bound}$
 $\text{round-bound drift-bound empty-trace-fri-bound current-empty-bound}]$)
then show ?thesis
by (simp add: algebra-simps)
qed

theorem stark-soundness-from-aligned-partial-components-current-query-prefix-and-current-empty-query-single-query
fixes trace-fri-error' composition-fri-error' D empty-query-error :: prob
and C :: nat $\Rightarrow$ prob
assumes false-statement: $\neg$ exists-valid-trace
and wf: staged-budget-wellformed budgets
and controlled: staged-adversary-controlled budgets A
and trace-fri-bound:
 $\text{wp-event } (\text{checked-staged-security-experiment-with-data-state } A)$
 $(\text{staged-security-with-data-state-verifier-event}$
 $\text{trace-fri-bad-with-partial-candidates})$
 $\text{adversary-initial-state} \leq \text{trace-fri-error}'$
and comp-fri-bound:
 $\text{wp-event } (\text{checked-staged-security-experiment-with-data-state } A)$
 $(\text{staged-security-with-data-state-verifier-event}$
 $\text{composition-fri-bad-with-partial-candidates})$
 $\text{adversary-initial-state} \leq \text{composition-fri-error}'$
and round-bound:
 $\bigwedge i. i < \text{rounds} \implies$
 wp-event
 $(\text{checked-staged-security-experiment-with-query-prefix-data-state } A \ i)$
 $(\text{checked-staged-security-with-query-prefix-current-query-partial-opening-hit-at } i)$
 $\text{adversary-initial-state} \leq C \ i$
and drift-bound:
 wp-event
 $(\text{checked-staged-security-experiment-with-actual-alpha-prefix-data-state}$
 $A)$
 $\text{checked-staged-security-with-actual-alpha-prefix-empty-header-relevant-drift}$
 $\text{adversary-initial-state} \leq D$
and empty-trace-fri-bound:
 $\text{wp-event } (\text{checked-staged-security-experiment-with-data-state } A)$

```

    (staged-security-with-data-state-verifier-event
     trace-fri-bad-with-empty-composition-header-candidates)
    adversary-initial-state ≤ trace-fri-error
and current-empty-bound:
    wp-event (checked-staged-security-experiment-with-data-state A)
    (staged-security-with-data-state-current-empty-query-opening-hit-at 0)
    adversary-initial-state ≤ empty-query-error
shows checked-staged-adversary-acceptance-probability A ≤
    hash-collision-budget-value 0
    (staged-attacker-query-budget budgets +
     staged-challenge-query-budget + verifier-hash-query-budget) +
    (∑ i<rounds. C i) +
    trace-fri-error' + composition-fri-error' +
    (composition-error-bound +
     staged-phase-relation-error size
     (staged-alpha-search-queries budgets 0) +
     hash-collision-budget-value 0
     (staged-alpha-search-queries budgets 0) +
     D +
     staged-phase-relation-error size
     (staged-alpha-search-queries budgets 0) +
     trace-fri-error + empty-query-error)
proof -
have bound:
    checked-staged-adversary-acceptance-probability A ≤
    hash-collision-budget-value 0
    (staged-attacker-query-budget budgets +
     staged-challenge-query-budget + verifier-hash-query-budget) +
    (∑ i<rounds. C i) +
    trace-fri-error' + composition-fri-error' +
    (composition-error-bound +
     staged-phase-relation-error size
     (staged-alpha-search-queries budgets 0) +
     hash-collision-budget-value 0
     (staged-alpha-search-queries budgets 0) +
     staged-phase-relation-error size
     (staged-alpha-search-queries budgets 0) +
     D + trace-fri-error + empty-query-error)
by (rule
    checked-staged-soundness-from-aligned-transcript-aligned-randomization-components-and-current-query-prefix-ro
    [OF false-statement wf controlled trace-fri-bound comp-fri-bound
     round-bound drift-bound empty-trace-fri-bound current-empty-bound])
then show ?thesis
by (simp add: algebra-simps)
qed

end

end

```

```

theory Soundness-Relevant-Drift-Reachable-FRI
  imports Soundness-Relevant-Drift-Public
begin

```

Reachable partial-candidate FRI interface for the refined relevant-drift migration path. This layer does not change the compatibility predicates *trace-fri-bad-with-partial-openings* and *composition-fri-bad-with-partial-openings*. Instead it records the partial-candidate FRI obligations used by the aligned partial-opening route: the verifier has authenticated sampled openings, and the FRI low-degree interface rules out a low-degree violation for any candidate table compatible with those openings.

```

context soundness
begin

```

```

definition trace-fri-bad-with-reachable-partial-candidate
  :: ('f, 'a) protocol-channel-scheme  $\Rightarrow$ 
    (unit list  $\times$  ('f, 'a) protocol-channel-scheme) option  $\Rightarrow$  bool

```

where

```

  trace-fri-bad-with-reachable-partial-candidate s out  $\longleftrightarrow$ 
    ( $\exists$  fr query-idxs trace-openings trace-table.
      accepted-with-partial-trace-openings s out fr query-idxs
        trace-openings  $\wedge$ 
        partial-trace-table-candidate trace-table trace-openings  $\wedge$ 
         $\neg$  trace-table-low-degree trace-table)

```

```

definition composition-fri-bad-with-reachable-partial-candidate
  :: ('f, 'a) protocol-channel-scheme  $\Rightarrow$ 
    (unit list  $\times$  ('f, 'a) protocol-channel-scheme) option  $\Rightarrow$  bool

```

where

```

  composition-fri-bad-with-reachable-partial-candidate s out  $\longleftrightarrow$ 
    ( $\exists$  fr f-fri-roots f-final as dg composition-fri-roots final
      trace-query-idxs trace-openings composition-query-idxs
        composition-openings trace-table composition-table.
      accepted-with-partial-initial-openings s out fr f-fri-roots f-final
        as dg composition-fri-roots final trace-query-idxs trace-openings
          composition-query-idxs composition-openings  $\wedge$ 
          partial-trace-table-candidate trace-table trace-openings  $\wedge$ 
          partial-composition-table-candidate composition-table
            composition-openings  $\wedge$ 
            trace-table-low-degree trace-table  $\wedge$ 
             $\neg$  composition-table-low-degree maxDegree composition-table)

```

```

definition trace-fri-reachable-partial-candidate-reduction-assumption
  :: ('f, 'a) protocol-channel-scheme  $\Rightarrow$  bool

```

where

```

  trace-fri-reachable-partial-candidate-reduction-assumption s  $\longleftrightarrow$ 
    wp-event verify-monad

```

$(\text{trace-fri-bad-with-reachable-partial-candidate } s) \leq \text{trace-fri-error}$

definition *composition-fri-reachable-partial-candidate-reduction-assumption*
 $:: ('f, 'a) \text{ protocol-channel-scheme} \Rightarrow \text{bool}$

where

composition-fri-reachable-partial-candidate-reduction-assumption s
 $\longleftrightarrow$
wp-event verify-monad
 $(\text{composition-fri-bad-with-reachable-partial-candidate } s) \leq \text{composition-fri-error}$

lemma *trace-fri-bad-with-partial-candidates-imp-reachable-partial-candidate:*

assumes *trace-fri-bad-with-partial-candidates* s *out*
shows *trace-fri-bad-with-reachable-partial-candidate* s *out*
using *assms*
unfolding *trace-fri-bad-with-partial-candidates-def*
trace-fri-bad-with-reachable-partial-candidate-def
accepted-with-partial-initial-openings-def
by *blast*

lemma *composition-fri-bad-with-partial-candidates-imp-reachable-partial-candidate:*

assumes *composition-fri-bad-with-partial-candidates* s *out*
shows *composition-fri-bad-with-reachable-partial-candidate* s *out*
using *assms*
unfolding *composition-fri-bad-with-partial-candidates-def*
composition-fri-bad-with-reachable-partial-candidate-def
by *blast*

lemma *trace-fri-bad-with-empty-composition-header-candidates-imp-reachable-partial-candidate:*

assumes *trace-fri-bad-with-empty-composition-header-candidates* s *out*
shows *trace-fri-bad-with-reachable-partial-candidate* s *out*
using *assms*
unfolding *trace-fri-bad-with-empty-composition-header-candidates-def*
accepted-with-empty-composition-header-candidates-def
trace-fri-bad-with-reachable-partial-candidate-def
by *blast*

lemma *trace-fri-bad-with-partial-candidates-not-None:*

$\neg \text{trace-fri-bad-with-partial-candidates } s \text{ None}$
unfolding *trace-fri-bad-with-partial-candidates-def*
accepted-with-partial-initial-openings-def *accepted-def*
by *simp*

lemma *composition-fri-bad-with-partial-candidates-not-None:*

$\neg \text{composition-fri-bad-with-partial-candidates } s \text{ None}$
unfolding *composition-fri-bad-with-partial-candidates-def*
accepted-with-partial-initial-openings-def *accepted-def*
by *simp*

lemma *trace-fri-bad-with-empty-composition-header-candidates-not-None*:
 $\neg \text{trace-fri-bad-with-empty-composition-header-candidates } s \text{ None}$
unfolding *trace-fri-bad-with-empty-composition-header-candidates-def*
accepted-with-empty-composition-header-candidates-def *accepted-def*
by *simp*

lemma *trace-fri-bad-with-partial-candidates-bound-from-reachable-reduction*:
assumes
 $\bigwedge \text{data attacker-state.}$
 $\text{Some (data, attacker-state)} \in$
 $\text{set-dist (execute (checked-staged-transcript-program } A)$
 $\text{adversary-initial-state})} \implies$
 $\text{trace-fri-reachable-partial-candidate-reduction-assumption}$
 $(\text{verifier-state-from-adversary attacker-state}$
 $(\text{staged-proof-transcript data}))$
shows
 $\text{wp-event (checked-staged-security-experiment-with-data-state } A)$
 $(\text{staged-security-with-data-state-verifier-event}$
 $\text{trace-fri-bad-with-partial-candidates})$
 $\text{adversary-initial-state} \leq \text{trace-fri-error}$
proof (rule *checked-staged-security-with-data-state-verifier-event-bound-from-cont*)
show $\bigwedge s. \neg \text{trace-fri-bad-with-partial-candidates } s \text{ None}$
by (rule *trace-fri-bad-with-partial-candidates-not-None*)
next
fix *data attacker-state*
assume *builder*:
 $\text{Some (data, attacker-state)} \in$
 $\text{set-dist (execute (checked-staged-transcript-program } A)$
 $\text{adversary-initial-state})}$
let $?s =$
 $\text{verifier-state-from-adversary attacker-state}$
 $(\text{staged-proof-transcript data})$
have *reduction*:
 $\text{wp-event verify-monad}$
 $(\text{trace-fri-bad-with-reachable-partial-candidate } ?s) \text{ } ?s \leq$
 trace-fri-error
using *assms*[*OF* *builder*]
unfolding *trace-fri-reachable-partial-candidate-reduction-assumption-def*
by *simp*
show $\text{wp-event verify-monad (trace-fri-bad-with-partial-candidates } ?s) \text{ } ?s$
 $\leq \text{trace-fri-error}$
by (rule *order-trans*
[*OF* - *reduction*])
(rule *wp-event-mono*,
rule *trace-fri-bad-with-partial-candidates-imp-reachable-partial-candidate*)
qed

lemma *composition-fri-bad-with-partial-candidates-bound-from-reachable-reduction*:

```

assumes
   $\bigwedge \text{data attacker-state.}$ 
   $\text{Some (data, attacker-state)} \in$ 
     $\text{set-dist (execute (checked-staged-transcript-program } A)$ 
       $\text{adversary-initial-state)} \implies$ 
     $\text{composition-fri-reachable-partial-candidate-reduction-assumption}$ 
       $(\text{verifier-state-from-adversary attacker-state}$ 
         $(\text{staged-proof-transcript data}))$ 
shows
   $\text{wp-event (checked-staged-security-experiment-with-data-state } A)$ 
     $(\text{staged-security-with-data-state-verifier-event}$ 
       $\text{composition-fri-bad-with-partial-candidates})$ 
     $\text{adversary-initial-state} \leq \text{composition-fri-error}$ 
proof (rule checked-staged-security-with-data-state-verifier-event-bound-from-cont)
  show  $\bigwedge s. \neg \text{composition-fri-bad-with-partial-candidates } s \text{ None}$ 
    by (rule composition-fri-bad-with-partial-candidates-not-None)
next
  fix data attacker-state
  assume builder:
     $\text{Some (data, attacker-state)} \in$ 
       $\text{set-dist (execute (checked-staged-transcript-program } A)$ 
         $\text{adversary-initial-state})$ 
  let  $?s =$ 
     $\text{verifier-state-from-adversary attacker-state}$ 
       $(\text{staged-proof-transcript data})$ 
  have reduction:
     $\text{wp-event verify-monad}$ 
       $(\text{composition-fri-bad-with-reachable-partial-candidate } ?s) ?s \leq$ 
       $\text{composition-fri-error}$ 
    using assms[OF builder]
    unfolding
      composition-fri-reachable-partial-candidate-reduction-assumption-def
    by simp
  show  $\text{wp-event verify-monad}$ 
     $(\text{composition-fri-bad-with-partial-candidates } ?s) ?s \leq$ 
     $\text{composition-fri-error}$ 
    by (rule order-trans
      [OF - reduction])
      (rule wp-event-mono,
        rule
           $\text{composition-fri-bad-with-partial-candidates-imp-reachable-partial-candidate})$ 
qed

lemma trace-fri-bad-with-empty-header-candidates-bound-from-reachable-reduction:
assumes
   $\bigwedge \text{data attacker-state.}$ 
   $\text{Some (data, attacker-state)} \in$ 
     $\text{set-dist (execute (checked-staged-transcript-program } A)$ 
       $\text{adversary-initial-state)} \implies$ 

```

```

    trace-fri-reachable-partial-candidate-reduction-assumption
      (verifier-state-from-adversary attacker-state
       (staged-proof-transcript data))
  shows
    wp-event (checked-staged-security-experiment-with-data-state A)
      (staged-security-with-data-state-verifier-event
       trace-fri-bad-with-empty-composition-header-candidates)
    adversary-initial-state ≤ trace-fri-error
  proof (rule checked-staged-security-with-data-state-verifier-event-bound-from-cont)
    show  $\bigwedge s. \neg \text{trace-fri-bad-with-empty-composition-header-candidates } s \text{ None}$ 
      by (rule trace-fri-bad-with-empty-composition-header-candidates-not-None)
  next
    fix data attacker-state
    assume builder:
      Some (data, attacker-state) ∈
        set-dist (execute (checked-staged-transcript-program A)
                       adversary-initial-state)
    let ?s =
      verifier-state-from-adversary attacker-state
      (staged-proof-transcript data)
    have reduction:
      wp-event verify-monad
        (trace-fri-bad-with-reachable-partial-candidate ?s) ?s ≤
          trace-fri-error
    using assms[OF builder]
    unfolding trace-fri-reachable-partial-candidate-reduction-assumption-def
    by simp
    show wp-event verify-monad
      (trace-fri-bad-with-empty-composition-header-candidates ?s) ?s ≤
        trace-fri-error
    by (rule order-trans
        [OF - reduction])
      (rule wp-event-mono,
       rule
         trace-fri-bad-with-empty-composition-header-candidates-imp-reachable-partial-candidate)
  qed

definition empty-header-low-degree-trace-candidate-nonunique
  :: ('f, 'a) protocol-channel-scheme ⇒
    (unit list × ('f, 'a) protocol-channel-scheme) option ⇒ bool
where
  empty-header-low-degree-trace-candidate-nonunique s out  $\longleftrightarrow$ 
    (∃ fr f-fri-roots f-final as dg final trace-query-idxs
     trace-openings trace-table composition-table trace-table'.
     accepted-with-empty-composition-header-candidates s out fr
     f-fri-roots f-final as dg final trace-query-idxs trace-openings
     trace-table composition-table ∧
     partial-trace-table-candidate trace-table' trace-openings ∧
     trace-table ≠ trace-table' ∧

```


trace-table-low-degree trace-table $\wedge$
trace-table-low-degree trace-table')

definition *empty-header-low-degree-trace-candidate-nonunique-query-agreement-hit*

$:: (f, 'a) \text{ protocol-channel-scheme} \Rightarrow$
 $(\text{unit list} \times (f, 'a) \text{ protocol-channel-scheme}) \text{ option} \Rightarrow \text{bool}$

where

empty-header-low-degree-trace-candidate-nonunique-query-agreement-hit s out
 $\longleftrightarrow$

$(\exists \text{ fr } f\text{-fri-roots } f\text{-final as dg final trace-query-idxs}$
 $\text{ trace-openings trace-table composition-table trace-table' } i.$
 $\text{ accepted-with-empty-composition-header-candidates s out fr}$
 $\text{ f-fri-roots f-final as dg final trace-query-idxs trace-openings}$
 $\text{ trace-table composition-table} \wedge$
 $\text{ partial-trace-table-candidate trace-table' trace-openings} \wedge$
 $\text{ trace-table} \neq \text{ trace-table'} \wedge$
 $\text{ trace-table-low-degree trace-table} \wedge$
 $\text{ trace-table-low-degree trace-table'} \wedge$
 $i < \text{ rounds} \wedge$
 $\text{ map } (!) \text{ trace-table) (powers-scaled (trace-query-idxs ! i)) =}$
 $\text{ map } (!) \text{ trace-table') (powers-scaled (trace-query-idxs ! i))})$

definition *empty-header-low-degree-trace-candidate-nonunique-transcript-query-agreement-hit*

$:: (f, 'a) \text{ protocol-channel-scheme} \Rightarrow$
 $(\text{unit list} \times (f, 'a) \text{ protocol-channel-scheme}) \text{ option} \Rightarrow \text{bool}$

where

empty-header-low-degree-trace-candidate-nonunique-transcript-query-agreement-hit
 $s \text{ out} \longleftrightarrow$

$(\exists \text{ fr } f\text{-fri-roots } f\text{-final as dg final trace-query-idxs}$
 $\text{ trace-openings trace-table composition-table trace-table' } i.$
 $\text{ accepted-with-empty-composition-header-candidates s out fr}$
 $\text{ f-fri-roots f-final as dg final trace-query-idxs trace-openings}$
 $\text{ trace-table composition-table} \wedge$
 $\text{ accepted-transcript-shape s out as trace-query-idxs} \wedge$
 $\text{ partial-trace-table-candidate trace-table' trace-openings} \wedge$
 $\text{ trace-table} \neq \text{ trace-table'} \wedge$
 $\text{ trace-table-low-degree trace-table} \wedge$
 $\text{ trace-table-low-degree trace-table'} \wedge$
 $i < \text{ rounds} \wedge$
 $\text{ map } (!) \text{ trace-table) (powers-scaled (trace-query-idxs ! i)) =}$
 $\text{ map } (!) \text{ trace-table') (powers-scaled (trace-query-idxs ! i))})$

lemma *empty-header-low-degree-trace-candidate-nonunique-transcript-query-agreement-imp-query-agreement:*

assumes

empty-header-low-degree-trace-candidate-nonunique-transcript-query-agreement-hit
 $s \text{ out}$

shows

empty-header-low-degree-trace-candidate-nonunique-query-agreement-hit
 $s \text{ out}$

using *assms*
unfolding
empty-header-low-degree-trace-candidate-nonunique-transcript-query-agreement-hit-def
empty-header-low-degree-trace-candidate-nonunique-query-agreement-hit-def
by *blast*

definition *trace-table-agreement-indices* :: 'f list $\Rightarrow$ 'f list $\Rightarrow$ nat set

where

trace-table-agreement-indices trace-table trace-table' =
 {*idx* $\in$ *query-sample-space*.
 map ((!) *trace-table*) (*powers-scaled idx*) =
 map ((!) *trace-table'*) (*powers-scaled idx*)}

lemma *empty-header-low-degree-trace-candidate-nonunique-transcript-query-agreementE*:

assumes

empty-header-low-degree-trace-candidate-nonunique-transcript-query-agreement-hit
s out

obtains *fr f-fri-roots f-final as dg final trace-query-idxs*

trace-openings trace-table composition-table trace-table' i

where

accepted-with-empty-composition-header-candidates s out fr
f-fri-roots f-final as dg final trace-query-idxs trace-openings
trace-table composition-table
accepted-transcript-shape s out as trace-query-idxs
partial-trace-table-candidate trace-table' trace-openings
trace-table $\neq$ trace-table'
trace-table-low-degree trace-table
trace-table-low-degree trace-table'
i < rounds
trace-query-idxs ! i $\in$
trace-table-agreement-indices trace-table trace-table'

proof –

obtain *fr f-fri-roots f-final as dg final trace-query-idxs*

trace-openings trace-table composition-table trace-table' i

where *empty*:

accepted-with-empty-composition-header-candidates s out fr
f-fri-roots f-final as dg final trace-query-idxs trace-openings
trace-table composition-table

and *shape*: *accepted-transcript-shape s out as trace-query-idxs*

and *cand'*: *partial-trace-table-candidate trace-table' trace-openings*

and *distinct*: *trace-table $\neq$ trace-table'*

and *trace-low*: *trace-table-low-degree trace-table*

and *trace'-low*: *trace-table-low-degree trace-table'*

and *i-bound*: *i < rounds*

and *agree*:

map ((!) *trace-table*) (*powers-scaled (trace-query-idxs ! i)*) =
 map ((!) *trace-table'*) (*powers-scaled (trace-query-idxs ! i)*)

using *assms*

unfolding

```

    empty-header-low-degree-trace-candidate-nonunique-transcript-query-agreement-hit-def
  by blast
have len-query: length trace-query-idxs = rounds
  using accepted-transcript-shape-query-shapes(1)[OF shape] .
have idx-mem: trace-query-idxs ! i ∈ set trace-query-idxs
  by (rule nth-mem) (use i-bound len-query in simp)
have idx-sample: trace-query-idxs ! i ∈ query-sample-space
  by (rule accepted-transcript-shape-query-sample-space[OF shape idx-mem])
have idx-agree:
  trace-query-idxs ! i ∈
    trace-table-agreement-indices trace-table trace-table'
  using idx-sample agree unfolding trace-table-agreement-indices-def
  by simp
show ?thesis
  by (rule that[OF empty shape cand' distinct trace-low trace'-low
    i-bound idx-agree])
qed

```

```

lemma trace-table-agreement-indices-card-bound:
  assumes trace-low: trace-table-low-degree trace-table
    and trace'-low: trace-table-low-degree trace-table'
    and distinct: trace-table ≠ trace-table'
  shows card (trace-table-agreement-indices trace-table trace-table') <
    clength
proof -
  from trace-low obtain f where deg-f: degree f < clength
    and trace-table: trace-table = map (poly f) eval-domain
    unfolding trace-table-low-degree-def by blast
  from trace'-low obtain f' where deg-f': degree f' < clength
    and trace-table': trace-table' = map (poly f') eval-domain
    unfolding trace-table-low-degree-def by blast
  let ?A = trace-table-agreement-indices trace-table trace-table'
  let ?R = f - f'
  have A-subset: ?A ⊆ query-sample-space
    unfolding trace-table-agreement-indices-def by auto
  have finite-A: finite ?A
    unfolding trace-table-agreement-indices-def query-sample-space-def by simp
  have inj: inj-on (λidx. h ^ idx * shift) ?A
  proof (intro inj-onI)
    fix i j
    assume i-in: i ∈ ?A
      and j-in: j ∈ ?A
      and eq: h ^ i * shift = h ^ j * shift
    have i-bound: i < clength * scale
      by (rule query-sample-space-less-domain[OF set-mp[OF A-subset i-in]])
    have j-bound: j < clength * scale
      by (rule query-sample-space-less-domain[OF set-mp[OF A-subset j-in]])
    have h ^ i = h ^ j
      using eq shift-nonzero by simp
  end

```

```

then show  $i = j$ 
  by (rule h-power-inj-on-eval-domain[OF i-bound j-bound])
qed
have residual-nonzero:  $?R \neq 0$ 
proof
  assume  $?R = 0$ 
  then have  $f = f'$ 
    by simp
  then show False
    using distinct trace-table trace-table' by simp
qed
have image-subset:
   $(\lambda idx. h \wedge idx * shift) \text{ ' } ?A \subseteq \{x. poly \text{ ?R } x = 0\}$ 
proof
  fix  $x$ 
  assume  $x\text{-in}: x \in (\lambda idx. h \wedge idx * shift) \text{ ' } ?A$ 
  then obtain  $idx$  where  $idx\text{-in}: idx \in ?A$ 
    and  $x\text{-eq}: x = h \wedge idx * shift$ 
    by blast
  have  $idx\text{-sample}: idx \in query\text{-sample-space}$ 
    by (rule set-mp[OF A-subset idx-in])
  have  $idx\text{-bound}: idx < clength * scale$ 
    by (rule query-sample-space-less-domain[OF idx-sample])
  have agree:
     $map \text{ (!) } trace\text{-table} \text{ (powers-scaled } idx) =$ 
     $map \text{ (!) } trace\text{-table}' \text{ (powers-scaled } idx)$ 
    using  $idx\text{-in}$  unfolding trace-table-agreement-indices-def by simp
  have  $ps0: powers\text{-scaled } idx \text{ ! } 0 = idx$ 
    using powers-pos unfolding powers-scaled-def by simp
  have  $len\text{-ps}: 0 < length \text{ (powers-scaled } idx)$ 
    using powers-pos unfolding powers-scaled-def by simp
  have  $value\text{-eq}: trace\text{-table} \text{ ! } idx = trace\text{-table}' \text{ ! } idx$ 
    using arg-cong[OF agree, of  $\lambda xs. xs \text{ ! } 0$ ]  $ps0 \text{ len-ps}$ 
    by simp
  have left:
     $trace\text{-table} \text{ ! } idx = poly \text{ f } (h \wedge idx * shift)$ 
    using trace-table eval-domain-nth[OF idx-bound]  $idx\text{-bound}$ 
    eval-domain-length by simp
  have right:
     $trace\text{-table}' \text{ ! } idx = poly \text{ f' } (h \wedge idx * shift)$ 
    using trace-table' eval-domain-nth[OF idx-bound]  $idx\text{-bound}$ 
    eval-domain-length by simp
  have  $poly \text{ f } (h \wedge idx * shift) = poly \text{ f' } (h \wedge idx * shift)$ 
    using value-eq left right by simp
  then show  $x \in \{x. poly \text{ ?R } x = 0\}$ 
    using  $x\text{-eq}$  by simp
qed
have  $card \text{ ?A} = card \text{ ((}\lambda idx. h \wedge idx * shift) \text{ ' } ?A)$ 
  by (simp add: card-image inj)

```

also have $\dots \leq \text{card } \{x. \text{poly } ?R \ x = 0\}$
by (rule card-mono[OF poly-roots-finite[OF residual-nonzero]
image-subset])
also have $\dots \leq \text{degree } ?R$
by (rule card-poly-roots-bound[OF residual-nonzero])
also have $\dots < \text{clength}$
using deg-f deg-f' **by** (simp add: degree-diff-less)
finally show ?thesis .
qed

lemma trace-table-agreement-indices-fraction-bound:
assumes trace-low: trace-table-low-degree trace-table
and trace'-low: trace-table-low-degree trace-table'
and distinct: trace-table $\neq$ trace-table'
shows nnreal
 $(\text{card } (\text{trace-table-agreement-indices trace-table trace-table'})) /$
 $\text{nnreal } (\text{card query-sample-space}) \leq$
 $\text{nnreal clength} / \text{nnreal } (\text{card query-sample-space})$
proof –
have card-le:
 $\text{card } (\text{trace-table-agreement-indices trace-table trace-table'}) \leq$
 clength
using trace-table-agreement-indices-card-bound
[OF trace-low trace'-low distinct]
by simp
show ?thesis
by (rule nnreal-nat-divide-right-mono[OF card-le])
qed

lemma checked-staged-query-prefix-trace-pair-agreement-hit-bound-by-prehit:
assumes trace-low: trace-table-low-degree trace-table
and trace'-low: trace-table-low-degree trace-table'
and distinct: trace-table $\neq$ trace-table'
shows
 $\text{wp-event } (\text{checked-staged-query-prefix-receive-with-state } A \ i)$
 $(\text{checked-staged-query-prefix-dynamic-index-hit}$
 $(\lambda -. \text{trace-table-agreement-indices trace-table trace-table'}))$
 $\text{adversary-initial-state} \leq$
 $\text{wp-event } (\text{checked-staged-query-prefix-receive-with-state } A \ i)$
 $(\text{checked-staged-query-prefix-dynamic-index-prehit}$
 $(\lambda -. \text{trace-table-agreement-indices trace-table trace-table'}))$
 $\text{adversary-initial-state} +$
 $\text{nnreal clength} / \text{nnreal } (\text{card query-sample-space})$
proof –
have raw-bound: query-index-raw-preimage-bound
by (rule query-index-raw-preimage-bound-from-sampler-wellformed)
show ?thesis
proof (rule
checked-staged-query-prefix-dynamic-index-hit-bound-by-prehit-and-query-error

```

    [OF raw-bound])
  fix prefix prefix-state
  assume
    Some ((prefix, prefix-state), prefix-state) ∈
      set-dist
      (execute (checked-staged-query-prefix-with-state A i)
        adversary-initial-state)
  show
    trace-table-agreement-indices trace-table trace-table'
      ⊆ query-sample-space ∧
      nnreal
      (card (trace-table-agreement-indices trace-table trace-table')) /
      nnreal (card query-sample-space) ≤
      nnreal clength / nnreal (card query-sample-space)
  using trace-table-agreement-indices-fraction-bound
    [OF trace-low trace'-low distinct]
  unfolding trace-table-agreement-indices-def by auto
qed
qed

definition checked-staged-security-with-actual-alpha-prefix-trace-pair-agreement-hit-from-prefix
  :: 'f staged-adversary ⇒ 'f list ⇒ 'f list ⇒ nat ⇒
    ((((((('f × 'f list × 'f list × 'f) ×
      'f protocol-channel) × 'f staged-proof-data) ×
      'f protocol-channel) × unit list) ×
      'f protocol-channel) option ⇒ bool
  where
    checked-staged-security-with-actual-alpha-prefix-trace-pair-agreement-hit-from-prefix
      A trace-table trace-table' i out ⟷
      (case out of
        None ⇒ False
      | Some (((((alpha-prefix, alpha-prefix-state), data), attacker-state),
        result), final-state) ⇒
        (∃ prefix prefix-state raw raw-state.
          Some (data, attacker-state) ∈
            set-dist
            (execute (checked-staged-transcript-program A)
              adversary-initial-state) ∧
          Some (((prefix, prefix-state), raw), raw-state) ∈
            set-dist
            (execute (checked-staged-query-prefix-receive-with-state A i)
              adversary-initial-state) ∧
          checked-staged-query-prefix-dynamic-index-hit
            (λ- -. trace-table-agreement-indices trace-table trace-table')
            (Some (((prefix, prefix-state), raw), raw-state))))))

lemma checked-staged-security-with-actual-alpha-prefix-trace-pair-agreement-hit-from-prefixE:
  assumes
    checked-staged-security-with-actual-alpha-prefix-trace-pair-agreement-hit-from-prefix

```

A trace-table trace-table' i
(Some (((((alpha-prefix, alpha-prefix-state), data), attacker-state),
result), final-state)))
obtains *prefix prefix-state raw raw-state where*
Some (data, attacker-state) ∈
set-dist
(execute (checked-staged-transcript-program A)
adversary-initial-state)
Some (((prefix, prefix-state), raw), raw-state) ∈
set-dist
(execute (checked-staged-query-prefix-receive-with-state A i)
adversary-initial-state)
checked-staged-query-prefix-dynamic-index-hit
(λ- -. trace-table-agreement-indices trace-table trace-table')
(Some (((prefix, prefix-state), raw), raw-state)))
using *assms*
proof –
obtain *prefix prefix-state raw raw-state where*
builder:
Some (data, attacker-state) ∈
set-dist
(execute (checked-staged-transcript-program A)
adversary-initial-state)
and *prefix-receive:*
Some (((prefix, prefix-state), raw), raw-state) ∈
set-dist
(execute (checked-staged-query-prefix-receive-with-state A i)
adversary-initial-state)
and *hit:*
checked-staged-query-prefix-dynamic-index-hit
(λ- -. trace-table-agreement-indices trace-table trace-table')
(Some (((prefix, prefix-state), raw), raw-state)))
using *assms*
unfolding
checked-staged-security-with-actual-alpha-prefix-trace-pair-agreement-hit-from-prefix-def
by *simp blast*
show *?thesis*
by *(rule that[OF builder prefix-receive hit])*
qed

definition *checked-staged-security-with-actual-alpha-prefix-empty-header-low-degree-prefix-agreement-hit*
:: 'f staged-adversary ⇒
(((((('f × 'f list × 'f list × 'f) ×
'f protocol-channel) × 'f staged-proof-data) ×
'f protocol-channel) × unit list) ×
'f protocol-channel) option ⇒ bool
where
checked-staged-security-with-actual-alpha-prefix-empty-header-low-degree-prefix-agreement-hit
A out ⇔

(case out of
 None $\Rightarrow$ False
 | Some (((alpha-prefix, alpha-prefix-state), data), attacker-state),
 result), final-state) $\Rightarrow$
 (let s =
 verifier-state-from-adversary attacker-state
 (staged-proof-transcript data)
 in
 ($\exists$ fr f-fri-roots f-final as dg final trace-query-idxs
 trace-openings trace-table composition-table trace-table' i
 prefix prefix-state raw raw-state.
 accepted-with-empty-composition-header-candidates s
 (Some (result, final-state)) fr f-fri-roots f-final as dg
 final trace-query-idxs trace-openings trace-table
 composition-table $\wedge$
 accepted-transcript-shape s (Some (result, final-state)) as
 trace-query-idxs $\wedge$
 partial-trace-table-candidate trace-table' trace-openings $\wedge$
 trace-table $\neq$ trace-table' $\wedge$
 trace-table-low-degree trace-table $\wedge$
 trace-table-low-degree trace-table' $\wedge$
 i < rounds $\wedge$
 Some (data, attacker-state) $\in$
 set-dist
 (execute (checked-staged-transcript-program A)
 adversary-initial-state) $\wedge$
 Some (((prefix, prefix-state), raw), raw-state) $\in$
 set-dist
 (execute (checked-staged-query-prefix-receive-with-state A i)
 adversary-initial-state) $\wedge$
 index (to-nat raw) $\in$
 trace-table-agreement-indices trace-table trace-table'))

lemma empty-header-low-degree-trace-candidate-nonunique-transcript-query-agreement-imp-prefix-agreement-hit-on-

assumes wf: staged-budget-wellformed budgets

and controlled: staged-adversary-controlled budgets A

and support:

out $\in$

set-dist

(execute

(checked-staged-security-experiment-with-actual-alpha-prefix-data-state

A)

adversary-initial-state)

and hit:

checked-staged-security-with-actual-alpha-prefix-verifier-event

empty-header-low-degree-trace-candidate-nonunique-transcript-query-agreement-hit

out

shows

checked-staged-security-with-actual-alpha-prefix-empty-header-low-degree-prefix-agreement-hit


```

    A out
proof (cases out)
  case None
  then show ?thesis
    using hit
    unfolding checked-staged-security-with-actual-alpha-prefix-verifier-event-def
    checked-staged-security-with-actual-alpha-prefix-empty-header-low-degree-prefix-agreement-hit-def
    by simp
next
  case (Some packed)
  then obtain alpha-prefix alpha-prefix-state data attacker-state result
    final-state where out-eq:
    out =
      Some (((alpha-prefix, alpha-prefix-state), data), attacker-state),
        result), final-state)
    by (cases packed, auto split: prod.splits)
  let ?s =
    verifier-state-from-adversary attacker-state
    (staged-proof-transcript data)
  have local-hit:
    empty-header-low-degree-trace-candidate-nonunique-transcript-query-agreement-hit
    ?s (Some (result, final-state))
  using hit
  unfolding out-eq
    checked-staged-security-with-actual-alpha-prefix-verifier-event-def
  by simp
from empty-header-low-degree-trace-candidate-nonunique-transcript-query-agreementE
  [OF local-hit]
obtain fr f-fri-roots f-final as dg final trace-query-idxs
  trace-openings trace-table composition-table trace-table' i where
  empty:
    accepted-with-empty-composition-header-candidates ?s
    (Some (result, final-state)) fr f-fri-roots f-final as dg final
    trace-query-idxs trace-openings trace-table composition-table
  and shape:
    accepted-transcript-shape ?s (Some (result, final-state)) as
    trace-query-idxs
  and cand': partial-trace-table-candidate trace-table' trace-openings
  and distinct: trace-table  $\neq$  trace-table'
  and trace-low: trace-table-low-degree trace-table
  and trace'-low: trace-table-low-degree trace-table'
  and i-bound: i < rounds
  and idx-agree:
    trace-query-idxs ! i  $\in$ 
    trace-table-agreement-indices trace-table trace-table'
  by blast
have data-support:
  Some (((data, attacker-state), result), final-state)  $\in$ 
  set-dist

```

(execute (checked-staged-security-experiment-with-data-state A)
 adversary-initial-state)
by (rule
 checked-staged-security-with-actual-alpha-prefix-support-imp-data-state-support
 [OF support[unfolded out-eq]])
from
 checked-staged-security-with-data-state-accepted-shape-query-prefix-receive-index
 [OF wf controlled data-support shape i-bound]
obtain prefix prefix-state raw raw-state **where**
 prefix-receive:
 Some (((prefix, prefix-state), raw), raw-state) ∈
 set-dist
 (execute (checked-staged-query-prefix-receive-with-state A i)
 adversary-initial-state)
and raw-idx: index (to-nat raw) = trace-query-idxs ! i
by blast
have raw-agree:
 index (to-nat raw) ∈
 trace-table-agreement-indices trace-table trace-table'
using raw-idx idx-agree **by** simp
have builder:
 Some (data, attacker-state) ∈
 set-dist
 (execute (checked-staged-transcript-program A)
 adversary-initial-state)
proof –
from checked-staged-security-experiment-with-data-state-outcomeE
 [OF data-support]
show ?thesis **by** blast
qed
show ?thesis
unfolding out-eq
 checked-staged-security-with-actual-alpha-prefix-empty-header-low-degree-prefix-agreement-hit-def
apply (simp add: Let-def)
by (intro exI[of - fr] exI[of - f-fri-roots]
 exI[of - f-final] exI[of - as] exI[of - dg]
 exI[of - final] exI[of - trace-query-idxs]
 exI[of - trace-openings] exI[of - trace-table]
 exI[of - composition-table] exI[of - trace-table']
 exI[of - i] exI[of - prefix] exI[of - prefix-state]
 exI[of - raw] exI[of - raw-state] conjI empty shape cand'
 distinct trace-low trace'-low i-bound builder prefix-receive
 raw-agree)
qed

lemma empty-header-low-degree-trace-candidate-nonunique-imp-query-agreement-hit:
assumes empty-header-low-degree-trace-candidate-nonunique s out
shows
 empty-header-low-degree-trace-candidate-nonunique-query-agreement-hit

```

    s out
proof –
  obtain fr f-fri-roots f-final as dg final trace-query-idxs
    trace-openings trace-table composition-table trace-table' where empty:
      accepted-with-empty-composition-header-candidates s out fr
        f-fri-roots f-final as dg final trace-query-idxs trace-openings
          trace-table composition-table
  and cand': partial-trace-table-candidate trace-table' trace-openings
  and distinct: trace-table  $\neq$  trace-table'
  and trace-low: trace-table-low-degree trace-table
  and trace'-low: trace-table-low-degree trace-table'
  using assms
  unfolding empty-header-low-degree-trace-candidate-nonunique-def by blast
  have partial:
    accepted-with-partial-trace-openings s out fr trace-query-idxs
      trace-openings
  using empty unfolding accepted-with-empty-composition-header-candidates-def
  by blast
  have cand:
    partial-trace-table-candidate trace-table trace-openings
  using empty unfolding accepted-with-empty-composition-header-candidates-def
  by blast
  have idxs:
    map opening-index (trace-openings ! 0) =
      powers-scaled (trace-query-idxs ! 0)
  using accepted-with-partial-trace-openings-shapes(3)
    [OF partial rounds-positive] .
  have vals:
    map (! trace-table) (powers-scaled (trace-query-idxs ! 0)) =
      map (! trace-table') (powers-scaled (trace-query-idxs ! 0))
  proof –
    have left:
      map (! trace-table) (powers-scaled (trace-query-idxs ! 0)) =
        map opening-value (trace-openings ! 0)
      by (rule partial-trace-table-candidate-query-values
        [OF cand rounds-positive idxs])
    have right:
      map (! trace-table') (powers-scaled (trace-query-idxs ! 0)) =
        map opening-value (trace-openings ! 0)
      by (rule partial-trace-table-candidate-query-values
        [OF cand' rounds-positive idxs])
    show ?thesis
      using left right by simp
  qed
  show ?thesis
    unfolding
      empty-header-low-degree-trace-candidate-nonunique-query-agreement-hit-def
    using empty cand' distinct trace-low trace'-low rounds-positive vals
    by blast

```

qed

lemma *empty-header-low-degree-trace-candidate-nonunique-le-query-agreement-hit*:

wp-event
 (*checked-staged-security-experiment-with-actual-alpha-prefix-data-state A*)
 (*checked-staged-security-with-actual-alpha-prefix-verifier-event*
 empty-header-low-degree-trace-candidate-nonunique)
adversary-initial-state $\leq$
wp-event
 (*checked-staged-security-experiment-with-actual-alpha-prefix-data-state A*)
 (*checked-staged-security-with-actual-alpha-prefix-verifier-event*
 empty-header-low-degree-trace-candidate-nonunique-query-agreement-hit)
adversary-initial-state
by (*rule wp-event-mono*)
 (*auto simp*:
 checked-staged-security-with-actual-alpha-prefix-verifier-event-def
 split: option.splits prod.splits
 dest:
 empty-header-low-degree-trace-candidate-nonunique-imp-query-agreement-hit)

lemma *empty-header-partial-trace-candidate-nonunique-imp-trace-fri-or-low-degree-nonunique*:

assumes *empty-header-partial-trace-candidate-nonunique s out*
shows
trace-fri-bad-with-empty-composition-header-candidates s out $\vee$
empty-header-low-degree-trace-candidate-nonunique s out

proof —

obtain *fr f-fri-roots f-final as dg final trace-query-idxs trace-openings*
trace-table composition-table **where** *empty*:
accepted-with-empty-composition-header-candidates s out fr
f-fri-roots f-final as dg final trace-query-idxs trace-openings
trace-table composition-table
and *not-singleton*:
 $\neg (\exists \text{ trace-table0.}$
 partial-trace-table-candidates trace-openings $\subseteq$
 $\{\text{trace-table0}\})$
using *assms*
unfolding *empty-header-partial-trace-candidate-nonunique-def* **by** *blast*
have *trace-candidate*:
trace-table $\in$ *partial-trace-table-candidates trace-openings*
using *empty*
unfolding *accepted-with-empty-composition-header-candidates-def*
partial-trace-table-candidates-def
by *simp*
obtain *trace-table'* **where** *trace-candidate'*:
trace-table' $\in$ *partial-trace-table-candidates trace-openings*
and *distinct*: *trace-table* $\neq$ *trace-table'*
proof —
have $\neg$ *partial-trace-table-candidates trace-openings* $\subseteq$
 $\{\text{trace-table}\}$

```

    using not-singleton by blast
  then obtain trace-table' where
    trace-table'  $\in$  partial-trace-table-candidates trace-openings
    trace-table'  $\notin$  {trace-table}
    by blast
  then show ?thesis
    using that by blast
qed
show ?thesis
proof (cases trace-table-low-degree trace-table)
  case False
  then have trace-fri-bad-with-empty-composition-header-candidates s out
    unfolding trace-fri-bad-with-empty-composition-header-candidates-def
    using empty by blast
  then show ?thesis by simp
next
case trace-low: True
show ?thesis
proof (cases trace-table-low-degree trace-table')
  case False
  have empty':
    accepted-with-empty-composition-header-candidates s out fr
      f-fri-roots f-final as dg final trace-query-idxs trace-openings
      trace-table' composition-table
    using empty trace-candidate'
    unfolding accepted-with-empty-composition-header-candidates-def
      partial-trace-table-candidates-def
    by blast
  then have
    trace-fri-bad-with-empty-composition-header-candidates s out
    unfolding trace-fri-bad-with-empty-composition-header-candidates-def
    using False by blast
  then show ?thesis by simp
next
case trace'-low: True
have empty-header-low-degree-trace-candidate-nonunique s out
  unfolding empty-header-low-degree-trace-candidate-nonunique-def
  using empty trace-candidate' distinct trace-low trace'-low
  unfolding partial-trace-table-candidates-def
  by blast
then show ?thesis by simp
qed
qed
qed

definition empty-header-partial-trace-candidate-nonunique-transcript
  :: ('f, 'a) protocol-channel-scheme  $\Rightarrow$ 
    (unit list  $\times$  ('f, 'a) protocol-channel-scheme) option  $\Rightarrow$  bool
where

```

$\text{empty-header-partial-trace-candidate-nonunique-transcript } s \text{ out} \longleftrightarrow$
 $(\exists \text{ fr } f\text{-fri-roots } f\text{-final as dg final trace-query-idxs}$
 $\text{trace-openings trace-table composition-table.}$
 $\text{accepted-with-empty-composition-header-candidates } s \text{ out fr}$
 $f\text{-fri-roots } f\text{-final as dg final trace-query-idxs trace-openings}$
 $\text{trace-table composition-table} \wedge$
 $\text{accepted-transcript-shape } s \text{ out as trace-query-idxs} \wedge$
 $\neg (\exists \text{ trace-table0.}$
 $\text{partial-trace-table-candidates trace-openings} \subseteq$
 $\{\text{trace-table0}\}))$

lemma *empty-header-partial-trace-candidate-nonunique-transcript-imp-trace-fri-or-low-degree-transcript-query-agree*

assumes *empty-header-partial-trace-candidate-nonunique-transcript } s out*

shows

$\text{trace-fri-bad-with-empty-composition-header-candidates } s \text{ out} \vee$
 $\text{empty-header-low-degree-trace-candidate-nonunique-transcript-query-agreement-hit}$
 $s \text{ out}$

proof –

obtain *fr f-fri-roots f-final as dg final trace-query-idxs trace-openings*

trace-table composition-table **where** *empty:*

accepted-with-empty-composition-header-candidates } s out fr

f-fri-roots f-final as dg final trace-query-idxs trace-openings

trace-table composition-table

and *shape: accepted-transcript-shape } s out as trace-query-idxs*

and *not-singleton:*

$\neg (\exists \text{ trace-table0.}$

$\text{partial-trace-table-candidates trace-openings} \subseteq$

$\{\text{trace-table0}\})$

using *assms*

unfolding *empty-header-partial-trace-candidate-nonunique-transcript-def*

by *blast*

have *trace-candidate:*

$\text{trace-table} \in \text{partial-trace-table-candidates trace-openings}$

using *empty*

unfolding *accepted-with-empty-composition-header-candidates-def*

partial-trace-table-candidates-def

by *simp*

obtain *trace-table' where trace-candidate':*

$\text{trace-table}' \in \text{partial-trace-table-candidates trace-openings}$

and *distinct: trace-table $\neq$ trace-table'*

proof –

have $\neg \text{partial-trace-table-candidates trace-openings} \subseteq$
 $\{\text{trace-table}\}$

using *not-singleton* **by** *blast*

then obtain *trace-table' where*

$\text{trace-table}' \in \text{partial-trace-table-candidates trace-openings}$

$\text{trace-table}' \notin \{\text{trace-table}\}$

by *blast*

then show *?thesis*

```

    using that by blast
qed
show ?thesis
proof (cases trace-table-low-degree trace-table)
  case False
  then have trace-fri-bad-with-empty-composition-header-candidates s out
    unfolding trace-fri-bad-with-empty-composition-header-candidates-def
    using empty by blast
  then show ?thesis by simp
next
case trace-low: True
show ?thesis
proof (cases trace-table-low-degree trace-table')
  case False
  have empty':
    accepted-with-empty-composition-header-candidates s out fr
      f-fri-roots f-final as dg final trace-query-idxs trace-openings
      trace-table' composition-table
  using empty trace-candidate'
  unfolding accepted-with-empty-composition-header-candidates-def
    partial-trace-table-candidates-def
  by blast
  then have
    trace-fri-bad-with-empty-composition-header-candidates s out
    unfolding trace-fri-bad-with-empty-composition-header-candidates-def
    using False by blast
  then show ?thesis by simp
next
case trace'-low: True
have idxs:
  map opening-index (trace-openings ! 0) =
    powers-scaled (trace-query-idxs ! 0)
proof -
  have partial:
    accepted-with-partial-trace-openings s out fr trace-query-idxs
      trace-openings
  using empty
  unfolding accepted-with-empty-composition-header-candidates-def
  by blast
  show ?thesis
    using accepted-with-partial-trace-openings-shapes(3)
      [OF partial rounds-positive] .
qed
have vals:
  map ((! trace-table) (powers-scaled (trace-query-idxs ! 0))) =
    map ((! trace-table') (powers-scaled (trace-query-idxs ! 0)))
proof -
  have cand:
    partial-trace-table-candidate trace-table trace-openings

```

```

    using empty
    unfolding accepted-with-empty-composition-header-candidates-def
    by blast
  have cand':
    partial-trace-table-candidate trace-table' trace-openings
    using trace-candidate'
    unfolding partial-trace-table-candidates-def by simp
  have left:
    map ((! trace-table) (powers-scaled (trace-query-idxs ! 0))) =
      map opening-value (trace-openings ! 0)
    by (rule partial-trace-table-candidate-query-values
        [OF cand rounds-positive idxs])
  have right:
    map ((! trace-table') (powers-scaled (trace-query-idxs ! 0))) =
      map opening-value (trace-openings ! 0)
    by (rule partial-trace-table-candidate-query-values
        [OF cand' rounds-positive idxs])
  show ?thesis
    using left right by simp
qed
have
  empty-header-low-degree-trace-candidate-nonunique-transcript-query-agreement-hit
    s out
  unfolding
    empty-header-low-degree-trace-candidate-nonunique-transcript-query-agreement-hit-def
  using empty shape trace-candidate' distinct trace-low trace'-low
    rounds-positive vals
  unfolding partial-trace-table-candidates-def
  by blast
then show ?thesis by simp
qed
qed
qed

```

lemma *empty-header-partial-trace-candidate-nonunique-transcript-le-trace-fri-or-low-degree-transcript-query-agreement-hit*

wp-event

(checked-staged-security-experiment-with-actual-alpha-prefix-data-state A)

(checked-staged-security-with-actual-alpha-prefix-verifier-event

 empty-header-partial-trace-candidate-nonunique-transcript)

adversary-initial-state $\leq$

wp-event

(checked-staged-security-experiment-with-actual-alpha-prefix-data-state A)

(checked-staged-security-with-actual-alpha-prefix-verifier-event

 trace-fri-bad-with-empty-composition-header-candidates)

adversary-initial-state +

wp-event

(checked-staged-security-experiment-with-actual-alpha-prefix-data-state A)

(checked-staged-security-with-actual-alpha-prefix-verifier-event

 empty-header-low-degree-trace-candidate-nonunique-transcript-query-agreement-hit)


```

    adversary-initial-state
proof –
  let ?M =
    checked-staged-security-experiment-with-actual-alpha-prefix-data-state A
  let ?N =
    checked-staged-security-with-actual-alpha-prefix-verifier-event
    empty-header-partial-trace-candidate-nonunique-transcript
  let ?T =
    checked-staged-security-with-actual-alpha-prefix-verifier-event
    trace-fri-bad-with-empty-composition-header-candidates
  let ?L =
    checked-staged-security-with-actual-alpha-prefix-verifier-event
    empty-header-low-degree-trace-candidate-nonunique-transcript-query-agreement-hit
  have wp-event ?M ?N adversary-initial-state ≤
    wp-event ?M (λout. ?T out ∨ ?L out) adversary-initial-state
  by (rule wp-event-mono)
    (auto simp: checked-staged-security-with-actual-alpha-prefix-verifier-event-def
      split: option.splits prod.splits
      dest:
        empty-header-partial-trace-candidate-nonunique-transcript-imp-trace-fri-or-low-degree-transcript-query-agreement)
  also have ... ≤
    wp-event ?M ?T adversary-initial-state +
    wp-event ?M ?L adversary-initial-state
  by (rule wp-event-union-bound)
  finally show ?thesis .
qed

```

definition *composition-alpha-partial-opening-union-hit-with-empty-header-transcript*

$:: (f, 'a) \text{ protocol-channel-scheme} \Rightarrow$
 $(\text{unit list} \times (f, 'a) \text{ protocol-channel-scheme}) \text{ option} \Rightarrow \text{bool}$

where

composition-alpha-partial-opening-union-hit-with-empty-header-transcript
 $s \text{ out} \longleftrightarrow$
 $(\exists fr \text{ f-fri-roots } f\text{-final as dg final trace-query-idxs}$
 $\text{trace-openings trace-table composition-table.}$
 $\text{accepted-with-empty-composition-header-candidates } s \text{ out fr}$
 $f\text{-fri-roots } f\text{-final as dg final trace-query-idxs trace-openings}$
 $\text{trace-table composition-table} \wedge$
 $\text{accepted-transcript-shape } s \text{ out as trace-query-idxs} \wedge$
 $as \in \text{partial-trace-opening-alpha-union-bad-sets trace-openings})$

lemma *composition-alpha-partial-opening-union-hit-with-empty-header-transcript-imp-union-hit:*

assumes

composition-alpha-partial-opening-union-hit-with-empty-header-transcript
 $s \text{ out}$

shows

composition-alpha-partial-opening-union-hit-with-empty-header $s \text{ out}$

using *assms*

unfolding

composition-alpha-partial-opening-union-hit-with-empty-header-transcript-def
composition-alpha-partial-opening-union-hit-with-empty-header-def
by *blast*

definition *composition-bad-with-empty-composition-header-candidates-transcript*

$:: (f, 'a) \text{ protocol-channel-scheme} \Rightarrow$
 $(\text{unit list} \times (f, 'a) \text{ protocol-channel-scheme}) \text{ option} \Rightarrow \text{bool}$

where

composition-bad-with-empty-composition-header-candidates-transcript
 $s \text{ out} \longleftrightarrow$
 $(\exists \text{ fr } f\text{-fri-roots } f\text{-final as dg final trace-query-idxs}$
 $\text{ trace-openings trace-table composition-table } f.$
 $\text{ accepted-with-empty-composition-header-candidates } s \text{ out fr}$
 $f\text{-fri-roots } f\text{-final as dg final trace-query-idxs trace-openings}$
 $\text{ trace-table composition-table} \wedge$
 $\text{ accepted-transcript-shape } s \text{ out as trace-query-idxs} \wedge$
 $\text{ degree } f < \text{ clength} \wedge$
 $\text{ trace-table} = \text{ map } (\text{poly } f) \text{ eval-domain} \wedge$
 $\text{ violated-constraints } f \neq \{\}$
 $\text{ composition-table-low-degree maxDegree composition-table} \wedge$
 $\text{ all-queries-consistent trace-table composition-table as})$

definition *composition-degree-bad-with-empty-composition-header-candidates-transcript*

$:: (f, 'a) \text{ protocol-channel-scheme} \Rightarrow$
 $(\text{unit list} \times (f, 'a) \text{ protocol-channel-scheme}) \text{ option} \Rightarrow \text{bool}$

where

composition-degree-bad-with-empty-composition-header-candidates-transcript
 $s \text{ out} \longleftrightarrow$
 $(\exists \text{ fr } f\text{-fri-roots } f\text{-final as dg final trace-query-idxs}$
 $\text{ trace-openings trace-table composition-table } f.$
 $\text{ accepted-with-empty-composition-header-candidates } s \text{ out fr}$
 $f\text{-fri-roots } f\text{-final as dg final trace-query-idxs trace-openings}$
 $\text{ trace-table composition-table} \wedge$
 $\text{ accepted-transcript-shape } s \text{ out as trace-query-idxs} \wedge$
 $\text{ degree } f < \text{ clength} \wedge$
 $\text{ trace-table} = \text{ map } (\text{poly } f) \text{ eval-domain} \wedge$
 $\text{ violated-constraints } f \neq \{\}$
 $\text{ composition-table-low-degree maxDegree composition-table} \wedge$
 $\text{ all-queries-consistent trace-table composition-table as} \wedge$
 $\neg \text{ common-denominator-degree-bounds } f \text{ as})$

definition *composition-randomization-bad-with-empty-composition-header-candidates-transcript*

$:: (f, 'a) \text{ protocol-channel-scheme} \Rightarrow$
 $(\text{unit list} \times (f, 'a) \text{ protocol-channel-scheme}) \text{ option} \Rightarrow \text{bool}$

where

composition-randomization-bad-with-empty-composition-header-candidates-transcript
 $s \text{ out} \longleftrightarrow$
 $(\exists \text{ fr } f\text{-fri-roots } f\text{-final as dg final trace-query-idxs}$
 $\text{ trace-openings trace-table composition-table } f.$

accepted-with-empty-composition-header-candidates s out fr
f-fri-roots f-final as dg final trace-query-idxs trace-openings
trace-table composition-table $\wedge$
accepted-transcript-shape s out as trace-query-idxs $\wedge$
degree f < clength $\wedge$
trace-table = map (poly f) eval-domain $\wedge$
violated-constraints f $\neq \{\}$ $\wedge$
composition-table-low-degree maxDegree composition-table $\wedge$
all-queries-consistent trace-table composition-table as $\wedge$
common-denominator-degree-bounds f as $\wedge$
random-combination-common-denominator-hides-violations f as)

lemma *composition-bad-with-empty-composition-header-candidates-transcript-split:*

assumes *false-statement: $\neg$ exists-valid-trace*

and *bad:*

composition-bad-with-empty-composition-header-candidates-transcript
s out

shows

composition-degree-bad-with-empty-composition-header-candidates-transcript
s out $\vee$

composition-randomization-bad-with-empty-composition-header-candidates-transcript
s out

proof —

obtain *fr f-fri-roots f-final as dg final trace-query-idxs*

trace-openings trace-table composition-table f **where** *partial:*
accepted-with-empty-composition-header-candidates s out fr
f-fri-roots f-final as dg final trace-query-idxs trace-openings
trace-table composition-table

and *shape: accepted-transcript-shape s out as trace-query-idxs*

and *deg-f: degree f < clength*

and *trace-table: trace-table = map (poly f) eval-domain*

and *violated: violated-constraints f $\neq \{\}$*

and *composition-low:*

composition-table-low-degree maxDegree composition-table

and *all-queries:*

all-queries-consistent trace-table composition-table as

using *bad*

unfolding

composition-bad-with-empty-composition-header-candidates-transcript-def

by *blast*

show *?thesis*

proof (*cases common-denominator-degree-bounds f as*)

case *False*

then have

composition-degree-bad-with-empty-composition-header-candidates-transcript
s out

unfolding

composition-degree-bad-with-empty-composition-header-candidates-transcript-def

using *partial shape deg-f trace-table violated composition-low*

```

    all-queries
  by blast
  then show ?thesis by simp
next
case True
note bounds = True
have len-as: length as = length spec
proof -
  obtain result final-state fr' f-fri-roots' f-final' dg'
    composition-fri-roots' final' rest' where
    verifier-header-transcript s fr' f-fri-roots' f-final' as dg'
    composition-fri-roots' final' rest'
  using shape
  by (elim accepted-transcript-shape-header-query-extraction)
  then show ?thesis
  by (rule verifier-header-transcript-shapes(3))
qed
have hides:
  random-combination-common-denominator-hides-violations f as
  by (rule random-combination-common-denominator-hides-from-queries
    [OF len-as false-statement deg-f trace-table composition-low
    all-queries bounds])
have
  composition-randomization-bad-with-empty-composition-header-candidates-transcript
  s out
  unfolding
    composition-randomization-bad-with-empty-composition-header-candidates-transcript-def
  using partial shape deg-f trace-table violated composition-low
  all-queries bounds hides
  by blast
  then show ?thesis by simp
qed
qed

lemma composition-degree-bad-with-empty-composition-header-candidates-transcript-imp-degree-bad:
  assumes
    composition-degree-bad-with-empty-composition-header-candidates-transcript
    s out
  shows
    composition-degree-bad-with-empty-composition-header-candidates s out
  using assms
  unfolding
    composition-degree-bad-with-empty-composition-header-candidates-transcript-def
    composition-degree-bad-with-empty-composition-header-candidates-def
  by blast

lemma composition-degree-bad-with-empty-composition-header-candidates-transcript-false:
  ¬ composition-degree-bad-with-empty-composition-header-candidates-transcript
  s out

```

```

using
  composition-degree-bad-with-empty-composition-header-candidates-transcript-imp-degree-bad
  composition-degree-bad-with-empty-composition-header-candidates-false
by blast

lemma composition-degree-bad-with-empty-composition-header-candidates-transcript-bound:
  wp-event verify-monad
    (composition-degree-bad-with-empty-composition-header-candidates-transcript
      s) s ≤
    0
proof –
  have composition-degree-bad-with-empty-composition-header-candidates-transcript
    s =
    (λout :: (unit list × ('f, 'a) protocol-channel-scheme) option.
      False)
  by (rule ext)
    (simp add:
      composition-degree-bad-with-empty-composition-header-candidates-transcript-false)
  then show ?thesis
    unfolding wp-event-def wp-def dist-expect-def by simp
qed

lemma composition-randomization-bad-with-empty-composition-header-candidates-transcript-imp-alpha-union-hit:
  assumes bad:
    composition-randomization-bad-with-empty-composition-header-candidates-transcript
      s out
  shows
    composition-alpha-partial-opening-union-hit-with-empty-header-transcript
      s out
proof –
  obtain fr f-fri-roots f-final as dg final trace-query-idxs
    trace-openings trace-table composition-table f where partial:
    accepted-with-empty-composition-header-candidates s out fr
    f-fri-roots f-final as dg final trace-query-idxs trace-openings
    trace-table composition-table
  and shape: accepted-transcript-shape s out as trace-query-idxs
  and deg-f: degree f < clength
  and trace-table: trace-table = map (poly f) eval-domain
  and violated: violated-constraints f ≠ {}
  and bounds: common-denominator-degree-bounds f as
  and hides:
    random-combination-common-denominator-hides-violations f as
  using bad
  unfolding
    composition-randomization-bad-with-empty-composition-header-candidates-transcript-def
  by blast
  have as-in:
    as ∈ common-denominator-hiding-alpha-space f
proof –

```

```

obtain result final-state fr' f-fri-roots' f-final' dg'
  composition-fri-roots' final' rest' where header:
  verifier-header-transcript s fr' f-fri-roots' f-final' as dg'
  composition-fri-roots' final' rest'
using shape
by (elim accepted-transcript-shape-header-query-extraction)
have len-as: length as = length spec
  by (rule verifier-header-transcript-shapes(3)[OF header])
show ?thesis
  using len-as hides
  unfolding common-denominator-hiding-alpha-space-def alpha-space-def
  by simp
qed
have candidate:
  trace-table ∈ partial-trace-table-candidates trace-openings
using partial
unfolding accepted-with-empty-composition-header-candidates-def
  partial-trace-table-candidates-def
by simp
have witness-eq: low-degree-trace-witness trace-table = f
  by (rule low-degree-trace-witness-eq[OF deg-f trace-table])
have as-bad: as ∈ composition-trace-bad-alpha-space trace-table
  using deg-f trace-table violated bounds as-in
  unfolding composition-trace-bad-alpha-space-def witness-eq
  by simp
have as ∈ partial-trace-opening-alpha-union-bad-sets trace-openings
  unfolding partial-trace-opening-alpha-union-bad-sets-def
  using candidate as-bad by blast
then show ?thesis
  unfolding
    composition-alpha-partial-opening-union-hit-with-empty-header-transcript-def
  using partial shape by blast
qed

lemma composition-alpha-partial-opening-union-hit-with-empty-header-transcript-split:
assumes hit:
  composition-alpha-partial-opening-union-hit-with-empty-header-transcript
  s out
shows
  composition-alpha-bad-set-hit-with-empty-composition-header-candidates
  s out ∨
  empty-header-partial-trace-candidate-nonunique-transcript s out
proof –
obtain fr f-fri-roots f-final as dg final trace-query-idxs
  trace-openings trace-table composition-table where partial:
  accepted-with-empty-composition-header-candidates s out fr
  f-fri-roots f-final as dg final trace-query-idxs trace-openings
  trace-table composition-table
and shape: accepted-transcript-shape s out as trace-query-idxs

```

```

and as-union:
  as  $\in$  partial-trace-opening-alpha-union-bad-sets trace-openings
using hit
unfolding
  composition-alpha-partial-opening-union-hit-with-empty-header-transcript-def
by blast
from as-union obtain trace-table' where candidate':
  trace-table'  $\in$  partial-trace-table-candidates trace-openings
and as-bad': as  $\in$  composition-trace-bad-alpha-space trace-table'
unfolding partial-trace-opening-alpha-union-bad-sets-def by blast
have candidate:
  trace-table  $\in$  partial-trace-table-candidates trace-openings
using partial
unfolding accepted-with-empty-composition-header-candidates-def
  partial-trace-table-candidates-def
by simp
show ?thesis
proof (cases
   $\exists$  trace-table0.
    partial-trace-table-candidates trace-openings  $\subseteq$ 
      {trace-table0})
case True
then obtain trace-table0 where subset:
  partial-trace-table-candidates trace-openings  $\subseteq$ 
    {trace-table0}
by blast
have trace-eq: trace-table = trace-table0
using subset candidate by blast
have trace'-eq: trace-table' = trace-table0
using subset candidate' by blast
have as  $\in$  composition-trace-bad-alpha-space trace-table
using as-bad' trace-eq trace'-eq by simp
then have
  composition-alpha-bad-set-hit-with-empty-composition-header-candidates
    s out
unfolding
  composition-alpha-bad-set-hit-with-empty-composition-header-candidates-def
using partial by blast
then show ?thesis by simp
next
case False
have empty-header-partial-trace-candidate-nonunique-transcript s out
unfolding empty-header-partial-trace-candidate-nonunique-transcript-def
using partial shape False by blast
then show ?thesis by simp
qed
qed

```

definition *checked-staged-security-with-actual-alpha-prefix-empty-header-relevant-drift-transcript*

```

:: ((((((f × 'f list × 'f list × 'f) ×
      'f protocol-channel) × 'f staged-proof-data) ×
      'f protocol-channel) × unit list) ×
      'f protocol-channel) option ⇒ bool

```

where

```

checked-staged-security-with-actual-alpha-prefix-empty-header-relevant-drift-transcript
  out ⇔
checked-staged-security-with-actual-alpha-prefix-partial-header-candidate-drift
  composition-trace-bad-alpha-space out ∧
checked-staged-security-with-actual-alpha-prefix-verifier-event
  composition-alpha-partial-opening-union-hit-with-empty-header-transcript
  out

```

lemma *checked-staged-security-with-actual-alpha-prefix-empty-header-relevant-drift-transcript-le-fixed-or-nonunique-*

```

wp-event
  (checked-staged-security-experiment-with-actual-alpha-prefix-data-state A)
  checked-staged-security-with-actual-alpha-prefix-empty-header-relevant-drift-transcript
  adversary-initial-state ≤
wp-event
  (checked-staged-security-experiment-with-actual-alpha-prefix-data-state A)
  (checked-staged-security-with-actual-alpha-prefix-verifier-event
    composition-alpha-bad-set-hit-with-empty-composition-header-candidates)
  adversary-initial-state +
wp-event
  (checked-staged-security-experiment-with-actual-alpha-prefix-data-state A)
  (checked-staged-security-with-actual-alpha-prefix-verifier-event
    empty-header-partial-trace-candidate-nonunique-transcript)
  adversary-initial-state

```

proof —

```

let ?M =
  checked-staged-security-experiment-with-actual-alpha-prefix-data-state A
let ?fixed =
  checked-staged-security-with-actual-alpha-prefix-verifier-event
  composition-alpha-bad-set-hit-with-empty-composition-header-candidates
let ?nonunique =
  checked-staged-security-with-actual-alpha-prefix-verifier-event
  empty-header-partial-trace-candidate-nonunique-transcript
have wp-event ?M
  checked-staged-security-with-actual-alpha-prefix-empty-header-relevant-drift-transcript
  adversary-initial-state ≤
  wp-event ?M (λout. ?fixed out ∨ ?nonunique out)
  adversary-initial-state
by (rule wp-event-mono)
  (auto simp:
    checked-staged-security-with-actual-alpha-prefix-empty-header-relevant-drift-transcript-def
    checked-staged-security-with-actual-alpha-prefix-verifier-event-def
    split: option.splits prod.splits
    dest:
      composition-alpha-partial-opening-union-hit-with-empty-header-transcript-split)

```


also have ... $\leq$
 $wp\text{-event } ?M \text{ ?fixed adversary-initial-state} +$
 $wp\text{-event } ?M \text{ ?nonunique adversary-initial-state}$
 by (rule *wp-event-union-bound*)
 finally show *?thesis* .
 qed

lemma *checked-staged-security-with-actual-alpha-prefix-empty-header-relevant-drift-transcript-bound-from-fixed-and-*
assumes *fixed-bound*:

wp-event
 (*checked-staged-security-experiment-with-actual-alpha-prefix-data-state*
 A)
 (*checked-staged-security-with-actual-alpha-prefix-verifier-event*
 composition-alpha-bad-set-hit-with-empty-composition-header-candidates)
adversary-initial-state $\leq F$

and *nonunique-bound*:

wp-event
 (*checked-staged-security-experiment-with-actual-alpha-prefix-data-state*
 A)
 (*checked-staged-security-with-actual-alpha-prefix-verifier-event*
 empty-header-partial-trace-candidate-nonunique-transcript)
adversary-initial-state $\leq N$

shows

wp-event
 (*checked-staged-security-experiment-with-actual-alpha-prefix-data-state* *A*)
checked-staged-security-with-actual-alpha-prefix-empty-header-relevant-drift-transcript
adversary-initial-state $\leq F + N$

by (rule *order-trans* [

OF
checked-staged-security-with-actual-alpha-prefix-empty-header-relevant-drift-transcript-le-fixed-or-nonunique-tran
(intro add-mono fixed-bound nonunique-bound)

lemma *checked-staged-security-with-actual-alpha-prefix-empty-header-relevant-drift-transcript-le-relevant-drift*:

wp-event
 (*checked-staged-security-experiment-with-actual-alpha-prefix-data-state* *A*)
checked-staged-security-with-actual-alpha-prefix-empty-header-relevant-drift-transcript
adversary-initial-state $\leq$
wp-event
 (*checked-staged-security-experiment-with-actual-alpha-prefix-data-state* *A*)
checked-staged-security-with-actual-alpha-prefix-relevant-partial-header-candidate-drift
adversary-initial-state
by (rule *wp-event-mono*)
 (auto simp:
 checked-staged-security-with-actual-alpha-prefix-empty-header-relevant-drift-transcript-def
 checked-staged-security-with-actual-alpha-prefix-relevant-partial-header-candidate-drift-def
 checked-staged-security-with-actual-alpha-prefix-verifier-event-def
 split: option.splits prod.splits
 intro:
 composition-alpha-partial-opening-union-hit-with-empty-header-transcript-imp-union-hit)

lemma *checked-staged-security-with-actual-alpha-prefix-empty-header-relevant-drift-transcript-bound-from-relevant-d*

assumes *drift-bound*:

wp-event

(*checked-staged-security-experiment-with-actual-alpha-prefix-data-state*

A)

checked-staged-security-with-actual-alpha-prefix-relevant-partial-header-candidate-drift

adversary-initial-state $\leq D$

shows

wp-event

(*checked-staged-security-experiment-with-actual-alpha-prefix-data-state A*)

checked-staged-security-with-actual-alpha-prefix-empty-header-relevant-drift-transcript

adversary-initial-state $\leq D$

by (*rule order-trans*[

OF

checked-staged-security-with-actual-alpha-prefix-empty-header-relevant-drift-transcript-le-relevant-drift

drift-bound])

lemma *checked-staged-security-with-actual-alpha-prefix-empty-header-relevant-drift-transcript-le-fixed-or-trace-fri-or-*

wp-event

(*checked-staged-security-experiment-with-actual-alpha-prefix-data-state A*)

checked-staged-security-with-actual-alpha-prefix-empty-header-relevant-drift-transcript

adversary-initial-state $\leq$

wp-event

(*checked-staged-security-experiment-with-actual-alpha-prefix-data-state A*)

(*checked-staged-security-with-actual-alpha-prefix-verifier-event*

composition-alpha-bad-set-hit-with-empty-composition-header-candidates)

adversary-initial-state +

(*wp-event*

(*checked-staged-security-experiment-with-actual-alpha-prefix-data-state A*)

(*checked-staged-security-with-actual-alpha-prefix-verifier-event*

trace-fri-bad-with-empty-composition-header-candidates)

adversary-initial-state +

wp-event

(*checked-staged-security-experiment-with-actual-alpha-prefix-data-state A*)

(*checked-staged-security-with-actual-alpha-prefix-verifier-event*

empty-header-low-degree-trace-candidate-nonunique-transcript-query-agreement-hit)

adversary-initial-state)

proof –

let *?M* =

checked-staged-security-experiment-with-actual-alpha-prefix-data-state A

let *?fixed* =

checked-staged-security-with-actual-alpha-prefix-verifier-event

composition-alpha-bad-set-hit-with-empty-composition-header-candidates

let *?nonunique* =

checked-staged-security-with-actual-alpha-prefix-verifier-event

empty-header-partial-trace-candidate-nonunique-transcript

let *?trace-fri* =

checked-staged-security-with-actual-alpha-prefix-verifier-event

```

    trace-fri-bad-with-empty-composition-header-candidates
let ?low-degree =
    checked-staged-security-with-actual-alpha-prefix-verifier-event
    empty-header-low-degree-trace-candidate-nonunique-transcript-query-agreement-hit
have split:
    wp-event ?M
    checked-staged-security-with-actual-alpha-prefix-empty-header-relevant-drift-transcript
    adversary-initial-state ≤
    wp-event ?M ?fixed adversary-initial-state +
    wp-event ?M ?nonunique adversary-initial-state
by (rule
    checked-staged-security-with-actual-alpha-prefix-empty-header-relevant-drift-transcript-le-fixed-or-nonunique-tran
have nonunique:
    wp-event ?M ?nonunique adversary-initial-state ≤
    wp-event ?M ?trace-fri adversary-initial-state +
    wp-event ?M ?low-degree adversary-initial-state
by (rule
    empty-header-partial-trace-candidate-nonunique-transcript-le-trace-fri-or-low-degree-transcript-query-agreement)
show ?thesis
by (rule order-trans[OF split])
    (intro add-mono order-refl nonunique)
qed

lemma checked-staged-security-with-actual-alpha-prefix-empty-header-relevant-drift-transcript-bound-from-fixed-trace
assumes fixed-bound:
    wp-event
    (checked-staged-security-experiment-with-actual-alpha-prefix-data-state
    A)
    (checked-staged-security-with-actual-alpha-prefix-verifier-event
    composition-alpha-bad-set-hit-with-empty-composition-header-candidates)
    adversary-initial-state ≤ F
and trace-fri-bound:
    wp-event
    (checked-staged-security-experiment-with-actual-alpha-prefix-data-state
    A)
    (checked-staged-security-with-actual-alpha-prefix-verifier-event
    trace-fri-bad-with-empty-composition-header-candidates)
    adversary-initial-state ≤ T
and low-degree-bound:
    wp-event
    (checked-staged-security-experiment-with-actual-alpha-prefix-data-state
    A)
    (checked-staged-security-with-actual-alpha-prefix-verifier-event
    empty-header-low-degree-trace-candidate-nonunique-transcript-query-agreement-hit)
    adversary-initial-state ≤ L
shows
    wp-event
    (checked-staged-security-experiment-with-actual-alpha-prefix-data-state A)
    checked-staged-security-with-actual-alpha-prefix-empty-header-relevant-drift-transcript

```

$\text{adversary-initial-state} \leq F + (T + L)$
by (rule order-trans[
OF
checked-staged-security-with-actual-alpha-prefix-empty-header-relevant-drift-transcript-le-fixed-or-trace-fri-or-low
(intro add-mono fixed-bound add-mono trace-fri-bound low-degree-bound)

lemma *empty-header-partial-trace-candidate-nonunique-le-trace-fri-or-low-degree-nonunique*:

wp-event
(checked-staged-security-experiment-with-actual-alpha-prefix-data-state A)
(checked-staged-security-with-actual-alpha-prefix-verifier-event
empty-header-partial-trace-candidate-nonunique)
 $\text{adversary-initial-state} \leq$
wp-event
(checked-staged-security-experiment-with-actual-alpha-prefix-data-state A)
(checked-staged-security-with-actual-alpha-prefix-verifier-event
trace-fri-bad-with-empty-composition-header-candidates)
 $\text{adversary-initial-state} +$
wp-event
(checked-staged-security-experiment-with-actual-alpha-prefix-data-state A)
(checked-staged-security-with-actual-alpha-prefix-verifier-event
empty-header-low-degree-trace-candidate-nonunique)
 $\text{adversary-initial-state}$

proof –

let $?M =$
checked-staged-security-experiment-with-actual-alpha-prefix-data-state A
let $?N =$
checked-staged-security-with-actual-alpha-prefix-verifier-event
empty-header-partial-trace-candidate-nonunique
let $?T =$
checked-staged-security-with-actual-alpha-prefix-verifier-event
trace-fri-bad-with-empty-composition-header-candidates
let $?L =$
checked-staged-security-with-actual-alpha-prefix-verifier-event
empty-header-low-degree-trace-candidate-nonunique
have $\text{wp-event } ?M \text{ } ?N \text{ adversary-initial-state} \leq$
 $\text{wp-event } ?M (\lambda \text{out. } ?T \text{ out} \vee ?L \text{ out}) \text{ adversary-initial-state}$
by (rule wp-event-mono)
(auto simp: checked-staged-security-with-actual-alpha-prefix-verifier-event-def
split: option.splits prod.splits
dest:
empty-header-partial-trace-candidate-nonunique-imp-trace-fri-or-low-degree-nonunique)
also have $\dots \leq$
 $\text{wp-event } ?M \text{ } ?T \text{ adversary-initial-state} +$
 $\text{wp-event } ?M \text{ } ?L \text{ adversary-initial-state}$
by (rule wp-event-union-bound)
finally show $?thesis$.

qed

theorem *stark-soundness-from-aligned-partial-components-current-query-prefix-and-current-empty-query-reachable-f*

fixes D *empty-query-error* :: *prob*
and C :: *nat* $\Rightarrow$ *prob*
assumes *false-statement*: $\neg$ *exists-valid-trace*
and *wf*: *staged-budget-wellformed budgets*
and *controlled*: *staged-adversary-controlled budgets A*
and *trace-fri-reduction*:
 $\bigwedge$ *data attacker-state*.
Some (data, attacker-state) $\in$
set-dist (execute (checked-staged-transcript-program A)
adversary-initial-state) $\Rightarrow$
trace-fri-reachable-partial-candidate-reduction-assumption
(verifier-state-from-adversary attacker-state
(staged-proof-transcript data))
and *composition-fri-reduction*:
 $\bigwedge$ *data attacker-state*.
Some (data, attacker-state) $\in$
set-dist (execute (checked-staged-transcript-program A)
adversary-initial-state) $\Rightarrow$
composition-fri-reachable-partial-candidate-reduction-assumption
(verifier-state-from-adversary attacker-state
(staged-proof-transcript data))
and *round-bound*:
 $\bigwedge i. i < \text{rounds} \Rightarrow$
wp-event
(checked-staged-security-experiment-with-query-prefix-data-state A i)
(checked-staged-security-with-query-prefix-current-query-partial-opening-hit-at i)
adversary-initial-state $\leq C i$
and *drift-bound*:
wp-event
(checked-staged-security-experiment-with-actual-alpha-prefix-data-state
A)
checked-staged-security-with-actual-alpha-prefix-empty-header-relevant-drift
adversary-initial-state $\leq D$
and *current-empty-bound*:
wp-event (checked-staged-security-experiment-with-data-state A)
(staged-security-with-data-state-current-empty-query-opening-hit-at 0)
adversary-initial-state $\leq$ empty-query-error
shows *checked-staged-adversary-acceptance-probability A $\leq$*
hash-collision-budget-value 0
(staged-attacker-query-budget budgets +
staged-challenge-query-budget + verifier-hash-query-budget) +
 $(\sum i < \text{rounds}. C i) +$
trace-fri-error + composition-fri-error +
 $(\sum i < \text{rounds}. C i) +$
(composition-error-bound +
staged-phase-relation-error size
(staged-alpha-search-queries budgets 0) +
hash-collision-budget-value 0
(staged-alpha-search-queries budgets 0) +

$D +$
staged-phase-relation-error size
(staged-alpha-search-queries budgets 0) +
trace-fri-error + empty-query-error)
proof –
have *trace-fri-bound*:
wp-event (checked-staged-security-experiment-with-data-state A)
(staged-security-with-data-state-verifier-event
trace-fri-bad-with-partial-candidates)
adversary-initial-state ≤ trace-fri-error
by (rule
trace-fri-bad-with-partial-candidates-bound-from-reachable-reduction
[OF trace-fri-reduction])
have *comp-fri-bound*:
wp-event (checked-staged-security-experiment-with-data-state A)
(staged-security-with-data-state-verifier-event
composition-fri-bad-with-partial-candidates)
adversary-initial-state ≤ composition-fri-error
by (rule
composition-fri-bad-with-partial-candidates-bound-from-reachable-reduction
[OF composition-fri-reduction])
have *empty-trace-fri-bound*:
wp-event (checked-staged-security-experiment-with-data-state A)
(staged-security-with-data-state-verifier-event
trace-fri-bad-with-empty-composition-header-candidates)
adversary-initial-state ≤ trace-fri-error
by (rule
trace-fri-bad-with-empty-header-candidates-bound-from-reachable-reduction
[OF trace-fri-reduction])
show *?thesis*
by (rule
stark-soundness-from-aligned-partial-components-current-query-prefix-and-current-empty-query
[OF false-statement wf controlled trace-fri-bound comp-fri-bound
round-bound drift-bound empty-trace-fri-bound current-empty-bound])
qed

theorem *stark-soundness-from-aligned-partial-components-current-query-prefix-and-current-empty-query-reachable-f*
fixes *F X empty-query-error :: prob*
and *C :: nat ⇒ prob*
assumes *false-statement: ¬ exists-valid-trace*
and *wf: staged-budget-wellformed budgets*
and *controlled: staged-adversary-controlled budgets A*
and *trace-fri-reduction:*
 $\bigwedge$ *data attacker-state.*
Some (data, attacker-state) ∈
set-dist (execute (checked-staged-transcript-program A)
adversary-initial-state) ⇒
trace-fri-reachable-partial-candidate-reduction-assumption
(verifier-state-from-adversary attacker-state

(staged-proof-transcript data))
and composition-fri-reduction:
 $\bigwedge \text{data attacker-state.}$
 Some (data, attacker-state) $\in$
 set-dist (execute (checked-staged-transcript-program A)
 adversary-initial-state) $\implies$
 composition-fri-reachable-partial-candidate-reduction-assumption
 (verifier-state-from-adversary attacker-state
 (staged-proof-transcript data))
and round-bound:
 $\bigwedge i. i < \text{rounds} \implies$
 wp-event
 (checked-staged-security-experiment-with-query-prefix-data-state A i)
 (checked-staged-security-with-query-prefix-current-query-partial-opening-hit-at i)
 adversary-initial-state $\leq C i$
and fixed-empty-alpha-bound:
 wp-event
 (checked-staged-security-experiment-with-actual-alpha-prefix-data-state
 A)
 (checked-staged-security-with-actual-alpha-prefix-verifier-event
 composition-alpha-bad-set-hit-with-empty-composition-header-candidates)
 adversary-initial-state $\leq F$
and cross-bound:
 wp-event
 (checked-staged-security-experiment-with-actual-alpha-prefix-data-state
 A)
 checked-staged-security-with-actual-alpha-prefix-witnessed-cross-or-merkle-bad
 adversary-initial-state $\leq X$
and current-empty-bound:
 wp-event (checked-staged-security-experiment-with-data-state A)
 (staged-security-with-data-state-current-empty-query-opening-hit-at 0)
 adversary-initial-state $\leq \text{empty-query-error}$
shows checked-staged-adversary-acceptance-probability A $\leq$
 hash-collision-budget-value 0
 (staged-attacker-query-budget budgets +
 staged-challenge-query-budget + verifier-hash-query-budget) +
 ($\sum i < \text{rounds. } C i$) +
 trace-fri-error + composition-fri-error +
 ($\sum i < \text{rounds. } C i$) +
 (composition-error-bound +
 staged-phase-relation-error size
 (staged-alpha-search-queries budgets 0) +
 hash-collision-budget-value 0
 (staged-alpha-search-queries budgets 0) +
 (F + X) +
 staged-phase-relation-error size
 (staged-alpha-search-queries budgets 0) +
 trace-fri-error + empty-query-error)
proof –

have *drift-bound*:
wp-event
(checked-staged-security-experiment-with-actual-alpha-prefix-data-state A)
checked-staged-security-with-actual-alpha-prefix-empty-header-relevant-drift
adversary-initial-state $\leq F + X$
by (rule
checked-staged-security-with-actual-alpha-prefix-empty-header-relevant-drift-bound-from-fixed-and-witnessed-cross-bound
[*OF fixed-empty-alpha-bound cross-bound*])
show *?thesis*
by (rule
stark-soundness-from-aligned-partial-components-current-query-prefix-and-current-empty-query-reachable-fri-reduction
[*OF false-statement wf controlled trace-fri-reduction*
composition-fri-reduction round-bound drift-bound
current-empty-bound])
qed

theorem *stark-soundness-from-aligned-partial-components-current-query-prefix-and-current-empty-query-reachable-fri-reduction*
fixes *F N empty-query-error :: prob*
and *C :: nat $\Rightarrow$ prob*
assumes *false-statement: $\neg$ exists-valid-trace*
and *wf: staged-budget-wellformed budgets*
and *controlled: staged-adversary-controlled budgets A*
and *trace-fri-reduction:*
 $\bigwedge$ *data attacker-state.*
Some (data, attacker-state) $\in$
set-dist (execute (checked-staged-transcript-program A)
adversary-initial-state) $\implies$
trace-fri-reachable-partial-candidate-reduction-assumption
(verifier-state-from-adversary attacker-state
(staged-proof-transcript data))
and *composition-fri-reduction:*
 $\bigwedge$ *data attacker-state.*
Some (data, attacker-state) $\in$
set-dist (execute (checked-staged-transcript-program A)
adversary-initial-state) $\implies$
composition-fri-reachable-partial-candidate-reduction-assumption
(verifier-state-from-adversary attacker-state
(staged-proof-transcript data))
and *round-bound:*
 $\bigwedge i. i < \text{rounds} \implies$
wp-event
(checked-staged-security-experiment-with-query-prefix-data-state A i)
(checked-staged-security-with-query-prefix-current-query-partial-opening-hit-at i)
adversary-initial-state $\leq C i$
and *fixed-empty-alpha-bound:*
wp-event
(checked-staged-security-experiment-with-actual-alpha-prefix-data-state
A)
(checked-staged-security-with-actual-alpha-prefix-verifier-event

composition-alpha-bad-set-hit-with-empty-composition-header-candidates)
adversary-initial-state $\leq F$
and *nonunique-bound*:
wp-event
(checked-staged-security-experiment-with-actual-alpha-prefix-data-state
A)
(checked-staged-security-with-actual-alpha-prefix-verifier-event
empty-header-partial-trace-candidate-nonunique)
adversary-initial-state $\leq N$
and *current-empty-bound*:
wp-event (checked-staged-security-experiment-with-data-state A)
(staged-security-with-data-state-current-empty-query-opening-hit-at 0)
adversary-initial-state $\leq$ *empty-query-error*
shows *checked-staged-adversary-acceptance-probability A* $\leq$
hash-collision-budget-value 0
(staged-attacker-query-budget budgets +
staged-challenge-query-budget + verifier-hash-query-budget) +
($\sum i < \text{rounds}. C i$ +
trace-fri-error + composition-fri-error +
($\sum i < \text{rounds}. C i$ +
(composition-error-bound +
staged-phase-relation-error size
(staged-alpha-search-queries budgets 0) +
hash-collision-budget-value 0
(staged-alpha-search-queries budgets 0) +
(F + N) +
staged-phase-relation-error size
(staged-alpha-search-queries budgets 0) +
trace-fri-error + empty-query-error)
proof –
have *drift-bound*:
wp-event
(checked-staged-security-experiment-with-actual-alpha-prefix-data-state A)
checked-staged-security-with-actual-alpha-prefix-empty-header-relevant-drift
adversary-initial-state $\leq F + N$
by (rule
checked-staged-security-with-actual-alpha-prefix-empty-header-relevant-drift-bound-from-fixed-and-nonunique
[OF fixed-empty-alpha-bound nonunique-bound])
show *?thesis*
by (rule
stark-soundness-from-aligned-partial-components-current-query-prefix-and-current-empty-query-reachable-fri
[OF false-statement wf controlled trace-fri-reduction
composition-fri-reduction round-bound drift-bound
current-empty-bound])
qed
theorem *stark-soundness-from-aligned-partial-components-current-query-prefix-and-current-empty-query-reachable-fri*
fixes *F L empty-query-error* :: *prob*
and *C* :: *nat* $\Rightarrow$ *prob*

assumes *false-statement*: $\neg \text{exists-valid-trace}$
and *wf*: *staged-budget-wellformed budgets*
and *controlled*: *staged-adversary-controlled budgets A*
and *trace-fri-reduction*:
 $\bigwedge \text{data attacker-state.}$
 $\text{Some (data, attacker-state)} \in$
 $\text{set-dist (execute (checked-staged-transcript-program A)}$
 $\text{adversary-initial-state)} \implies$
 $\text{trace-fri-reachable-partial-candidate-reduction-assumption}$
 $(\text{verifier-state-from-adversary attacker-state}$
 $(\text{staged-proof-transcript data}))$
and *composition-fri-reduction*:
 $\bigwedge \text{data attacker-state.}$
 $\text{Some (data, attacker-state)} \in$
 $\text{set-dist (execute (checked-staged-transcript-program A)}$
 $\text{adversary-initial-state)} \implies$
 $\text{composition-fri-reachable-partial-candidate-reduction-assumption}$
 $(\text{verifier-state-from-adversary attacker-state}$
 $(\text{staged-proof-transcript data}))$
and *round-bound*:
 $\bigwedge i. i < \text{rounds} \implies$
 wp-event
 $(\text{checked-staged-security-experiment-with-query-prefix-data-state A } i)$
 $(\text{checked-staged-security-with-query-prefix-current-query-partial-opening-hit-at } i)$
 $\text{adversary-initial-state} \leq C \ i$
and *fixed-empty-alpha-bound*:
 wp-event
 $(\text{checked-staged-security-experiment-with-actual-alpha-prefix-data-state}$
 $A)$
 $(\text{checked-staged-security-with-actual-alpha-prefix-verifier-event}$
 $\text{composition-alpha-bad-set-hit-with-empty-composition-header-candidates})$
 $\text{adversary-initial-state} \leq F$
and *low-degree-nonunique-bound*:
 wp-event
 $(\text{checked-staged-security-experiment-with-actual-alpha-prefix-data-state}$
 $A)$
 $(\text{checked-staged-security-with-actual-alpha-prefix-verifier-event}$
 $\text{empty-header-low-degree-trace-candidate-nonunique})$
 $\text{adversary-initial-state} \leq L$
and *current-empty-bound*:
 $\text{wp-event (checked-staged-security-experiment-with-data-state A)}$
 $(\text{staged-security-with-data-state-current-empty-query-opening-hit-at } 0)$
 $\text{adversary-initial-state} \leq \text{empty-query-error}$
shows *checked-staged-adversary-acceptance-probability A* $\leq$
 $\text{hash-collision-budget-value } 0$
 $(\text{staged-attacker-query-budget budgets} +$
 $\text{staged-challenge-query-budget} + \text{verifier-hash-query-budget}) +$
 $(\sum i < \text{rounds. } C \ i) +$
 $\text{trace-fri-error} + \text{composition-fri-error} +$

$(\sum_{i < \text{rounds}} C_i) +$
 $(\text{composition-error-bound} +$
 $\text{staged-phase-relation-error size}$
 $(\text{staged-alpha-search-queries budgets } 0) +$
 $\text{hash-collision-budget-value } 0$
 $(\text{staged-alpha-search-queries budgets } 0) +$
 $(F + (\text{trace-fri-error} + L)) +$
 $\text{staged-phase-relation-error size}$
 $(\text{staged-alpha-search-queries budgets } 0) +$
 $\text{trace-fri-error} + \text{empty-query-error})$
proof –
let $?M =$
 $\text{checked-staged-security-experiment-with-actual-alpha-prefix-data-state } A$
let $?T =$
 $\text{checked-staged-security-with-actual-alpha-prefix-verifier-event}$
 $\text{trace-fri-bad-with-empty-composition-header-candidates}$
let $?L =$
 $\text{checked-staged-security-with-actual-alpha-prefix-verifier-event}$
 $\text{empty-header-low-degree-trace-candidate-nonunique}$
have $\text{trace-fri-data-bound:}$
 $\text{wp-event } (\text{checked-staged-security-experiment-with-data-state } A)$
 $(\text{staged-security-with-data-state-verifier-event}$
 $\text{trace-fri-bad-with-empty-composition-header-candidates})$
 $\text{adversary-initial-state} \leq \text{trace-fri-error}$
by (rule
 $\text{trace-fri-bad-with-empty-header-candidates-bound-from-reachable-reduction}$
 $[OF \text{ trace-fri-reduction}])$
have $\text{trace-fri-prefix-bound:}$
 $\text{wp-event } ?M ?T \text{ adversary-initial-state} \leq \text{trace-fri-error}$
proof –
have projected:
 $\text{wp-event } (\text{checked-staged-security-experiment-with-data-state } A)$
 $(\text{staged-security-with-data-state-verifier-event}$
 $\text{trace-fri-bad-with-empty-composition-header-candidates})$
 $\text{adversary-initial-state} =$
 $\text{wp-event } ?M ?T \text{ adversary-initial-state}$
by (rule
 $\text{checked-staged-security-with-data-state-verifier-event-actual-alpha-prefix-projection})$
show $?thesis$
using $\text{projected trace-fri-data-bound by simp}$
qed
have nonunique-bound:
 $\text{wp-event } ?M$
 $(\text{checked-staged-security-with-actual-alpha-prefix-verifier-event}$
 $\text{empty-header-partial-trace-candidate-nonunique})$
 $\text{adversary-initial-state} \leq \text{trace-fri-error} + L$
proof (rule order-trans)
show $\text{wp-event } ?M$
 $(\text{checked-staged-security-with-actual-alpha-prefix-verifier-event}$

```

    empty-header-partial-trace-candidate-nonunique)
  adversary-initial-state ≤
wp-event ?M ?T adversary-initial-state +
wp-event ?M ?L adversary-initial-state
  by (rule
    empty-header-partial-trace-candidate-nonunique-le-trace-fri-or-low-degree-nonunique)
show ... ≤ trace-fri-error + L
  by (intro add-mono trace-fri-prefix-bound low-degree-nonunique-bound)
qed
show ?thesis
  by (rule
    stark-soundness-from-aligned-partial-components-current-query-prefix-and-current-empty-query-reachable-fri-spli
    [OF false-statement wf controlled trace-fri-reduction
      composition-fri-reduction round-bound fixed-empty-alpha-bound
      nonunique-bound current-empty-bound])
qed

theorem stark-soundness-from-aligned-partial-components-current-query-prefix-and-current-empty-query-reachable-fri-spli
fixes F L empty-query-error :: prob
and C :: nat ⇒ prob
assumes false-statement: ¬ exists-valid-trace
and wf: staged-budget-wellformed budgets
and controlled: staged-adversary-controlled budgets A
and trace-fri-reduction:
  ∧ data attacker-state.
    Some (data, attacker-state) ∈
      set-dist (execute (checked-staged-transcript-program A)
        adversary-initial-state) ⇒
      trace-fri-reachable-partial-candidate-reduction-assumption
        (verifier-state-from-adversary attacker-state
          (staged-proof-transcript data))
and composition-fri-reduction:
  ∧ data attacker-state.
    Some (data, attacker-state) ∈
      set-dist (execute (checked-staged-transcript-program A)
        adversary-initial-state) ⇒
      composition-fri-reachable-partial-candidate-reduction-assumption
        (verifier-state-from-adversary attacker-state
          (staged-proof-transcript data))
and round-bound:
  ∧ i. i < rounds ⇒
    wp-event
      (checked-staged-security-experiment-with-query-prefix-data-state A i)
      (checked-staged-security-with-query-prefix-current-query-partial-opening-hit-at i)
      adversary-initial-state ≤ C i
and fixed-empty-alpha-bound:
  wp-event
    (checked-staged-security-experiment-with-actual-alpha-prefix-data-state
      A)

```

(checked-staged-security-with-actual-alpha-prefix-verifier-event
 composition-alpha-bad-set-hit-with-empty-composition-header-candidates)
 adversary-initial-state $\leq F$
and low-degree-query-agreement-bound:
 wp-event
 (checked-staged-security-experiment-with-actual-alpha-prefix-data-state
 A)
 (checked-staged-security-with-actual-alpha-prefix-verifier-event
 empty-header-low-degree-trace-candidate-nonunique-query-agreement-hit)
 adversary-initial-state $\leq L$
and current-empty-bound:
 wp-event (checked-staged-security-experiment-with-data-state A)
 (staged-security-with-data-state-current-empty-query-opening-hit-at 0)
 adversary-initial-state $\leq$ empty-query-error
shows checked-staged-adversary-acceptance-probability A $\leq$
 hash-collision-budget-value 0
 (staged-attacker-query-budget budgets +
 staged-challenge-query-budget + verifier-hash-query-budget) +
 ($\sum i < \text{rounds}. C i$) +
 trace-fri-error + composition-fri-error +
 ($\sum i < \text{rounds}. C i$) +
 (composition-error-bound +
 staged-phase-relation-error size
 (staged-alpha-search-queries budgets 0) +
 hash-collision-budget-value 0
 (staged-alpha-search-queries budgets 0) +
 (F + (trace-fri-error + L)) +
 staged-phase-relation-error size
 (staged-alpha-search-queries budgets 0) +
 trace-fri-error + empty-query-error)
proof –
have low-degree-nonunique-bound:
 wp-event
 (checked-staged-security-experiment-with-actual-alpha-prefix-data-state A)
 (checked-staged-security-with-actual-alpha-prefix-verifier-event
 empty-header-low-degree-trace-candidate-nonunique)
 adversary-initial-state $\leq L$
by (rule order-trans
 [OF empty-header-low-degree-trace-candidate-nonunique-le-query-agreement-hit
 low-degree-query-agreement-bound])
show ?thesis
by (rule
 stark-soundness-from-aligned-partial-components-current-query-prefix-and-current-empty-query-reachable-fri-split
 [OF false-statement wf controlled trace-fri-reduction
 composition-fri-reduction round-bound fixed-empty-alpha-bound
 low-degree-nonunique-bound current-empty-bound])
qed

theorem stark-soundness-from-aligned-partial-components-current-query-prefix-and-current-empty-query-reachable-fri-split

fixes X N *empty-query-error* :: *prob*
and C :: *nat* $\Rightarrow$ *prob*
assumes *false-statement*: $\neg$ *exists-valid-trace*
and *wf*: *staged-budget-wellformed budgets*
and *controlled*: *staged-adversary-controlled budgets A*
and *trace-fri-reduction*:
 $\bigwedge$ *data attacker-state*.
 $\text{Some } (data, \text{attacker-state}) \in$
 $\text{set-dist } (\text{execute } (\text{checked-staged-transcript-program } A)$
 $\text{adversary-initial-state}) \Rightarrow$
 $\text{trace-fri-reachable-partial-candidate-reduction-assumption}$
 $(\text{verifier-state-from-adversary attacker-state}$
 $(\text{staged-proof-transcript data}))$
and *composition-fri-reduction*:
 $\bigwedge$ *data attacker-state*.
 $\text{Some } (data, \text{attacker-state}) \in$
 $\text{set-dist } (\text{execute } (\text{checked-staged-transcript-program } A)$
 $\text{adversary-initial-state}) \Rightarrow$
 $\text{composition-fri-reachable-partial-candidate-reduction-assumption}$
 $(\text{verifier-state-from-adversary attacker-state}$
 $(\text{staged-proof-transcript data}))$
and *round-bound*:
 $\bigwedge i. i < \text{rounds} \Rightarrow$
 wp-event
 $(\text{checked-staged-security-experiment-with-query-prefix-data-state } A \ i)$
 $(\text{checked-staged-security-with-query-prefix-current-query-partial-opening-hit-at } i)$
 $\text{adversary-initial-state} \leq C \ i$
and *not-prefix-bound*:
 wp-event
 $(\text{checked-staged-security-experiment-with-actual-alpha-prefix-data-state}$
 $A)$
 $(\text{checked-staged-security-with-actual-alpha-prefix-partial-header-candidate-not-prefix-bound}$
 $\text{composition-trace-bad-alpha-space})$
 $\text{adversary-initial-state} \leq N$
and *cross-bound*:
 wp-event
 $(\text{checked-staged-security-experiment-with-actual-alpha-prefix-data-state}$
 $A)$
 $\text{checked-staged-security-with-actual-alpha-prefix-witnessed-cross-or-merkle-bad}$
 $\text{adversary-initial-state} \leq X$
and *current-empty-bound*:
 $\text{wp-event } (\text{checked-staged-security-experiment-with-data-state } A)$
 $(\text{staged-security-with-data-state-current-empty-query-opening-hit-at } 0)$
 $\text{adversary-initial-state} \leq \text{empty-query-error}$
shows $\text{checked-staged-adversary-acceptance-probability } A \leq$
 $\text{hash-collision-budget-value } 0$
 $(\text{staged-attacker-query-budget budgets} +$
 $\text{staged-challenge-query-budget} + \text{verifier-hash-query-budget}) +$
 $(\sum i < \text{rounds}. C \ i) +$

```

    trace-fri-error + composition-fri-error +
    ( $\sum i < \text{rounds. } C \ i$ ) +
    (composition-error-bound +
      staged-phase-relation-error size
      (staged-alpha-search-queries budgets 0) +
      hash-collision-budget-value 0
      (staged-alpha-search-queries budgets 0) +
      ((composition-error-bound +
        staged-phase-relation-error size
        (staged-alpha-search-queries budgets 0) +
        hash-collision-budget-value 0
        (staged-alpha-search-queries budgets 0) + N) + X) +
      staged-phase-relation-error size
      (staged-alpha-search-queries budgets 0) +
      trace-fri-error + empty-query-error)
  proof -
  let ?M =
    checked-staged-security-experiment-with-actual-alpha-prefix-data-state A
  let ?P =
    composition-error-bound +
    staged-phase-relation-error size
    (staged-alpha-search-queries budgets 0) +
    hash-collision-budget-value 0
    (staged-alpha-search-queries budgets 0)
  have transcript-bound:
    wp-event (checked-staged-transcript-with-alpha-prefix-program A)
    (checked-staged-transcript-actual-alpha-prefix-bad-set-hit
      composition-trace-bad-alpha-space)
    adversary-initial-state  $\leq$  ?P
  by (rule
    checked-staged-transcript-with-alpha-prefix-actual-bad-set-hit-bound-from-budgets
    [OF wf controlled])
  have prefix-bound:
    wp-event ?M
    (checked-staged-security-with-actual-alpha-prefix-bad-set-hit
      composition-trace-bad-alpha-space)
    adversary-initial-state  $\leq$  ?P
  by (rule
    checked-staged-security-with-actual-alpha-prefix-bad-set-hit-bound-from-transcript
    [OF transcript-bound])
  have fixed-empty-alpha-bound:
    wp-event ?M
    (checked-staged-security-with-actual-alpha-prefix-verifier-event
      composition-alpha-bad-set-hit-with-empty-composition-header-candidates)
    adversary-initial-state  $\leq$  ?P + N
  proof (rule order-trans)
  show wp-event ?M
    (checked-staged-security-with-actual-alpha-prefix-verifier-event
      composition-alpha-bad-set-hit-with-empty-composition-header-candidates)

```

```

adversary-initial-state ≤
  wp-event ?M
    (checked-staged-security-with-actual-alpha-prefix-bad-set-hit
      composition-trace-bad-alpha-space)
    adversary-initial-state +
  wp-event ?M
    (checked-staged-security-with-actual-alpha-prefix-partial-header-candidate-not-prefix-bound
      composition-trace-bad-alpha-space)
    adversary-initial-state
  by (rule
    checked-staged-security-with-actual-alpha-prefix-fixed-empty-alpha-hit-le-prefix-or-not-prefix
      [OF wf controlled])
show ... ≤ ?P + N
  by (intro add-mono prefix-bound not-prefix-bound)
qed
show ?thesis
  by (rule
    stark-soundness-from-aligned-partial-components-current-query-prefix-and-current-empty-query-reachable-fri-spli
      [OF false-statement wf controlled trace-fri-reduction
        composition-fri-reduction round-bound fixed-empty-alpha-bound
        cross-bound current-empty-bound])
qed

theorem stark-soundness-from-aligned-partial-components-current-query-prefix-and-current-empty-query-reachable-fri-spli
fixes X P R empty-query-error :: prob
and C :: nat ⇒ prob
assumes false-statement: ¬ exists-valid-trace
and wf: staged-budget-wellformed budgets
and controlled: staged-adversary-controlled budgets A
and trace-fri-reduction:
  ∧ data attacker-state.
    Some (data, attacker-state) ∈
      set-dist (execute (checked-staged-transcript-program A)
        adversary-initial-state) ⇒
      trace-fri-reachable-partial-candidate-reduction-assumption
        (verifier-state-from-adversary attacker-state
          (staged-proof-transcript data))
and composition-fri-reduction:
  ∧ data attacker-state.
    Some (data, attacker-state) ∈
      set-dist (execute (checked-staged-transcript-program A)
        adversary-initial-state) ⇒
      composition-fri-reachable-partial-candidate-reduction-assumption
        (verifier-state-from-adversary attacker-state
          (staged-proof-transcript data))
and round-bound:
  ∧ i. i < rounds ⇒
    wp-event
      (checked-staged-security-experiment-with-query-prefix-data-state A i)

```


(checked-staged-security-with-query-prefix-current-query-partial-opening-hit-at i)
 adversary-initial-state $\leq C\ i$
and cross-bound:
 wp-event
 (checked-staged-security-experiment-with-actual-alpha-prefix-data-state
 A)
 checked-staged-security-with-actual-alpha-prefix-witnessed-cross-or-merkle-bad
 adversary-initial-state $\leq X$
and data-pre-bound:
 wp-event (checked-staged-security-experiment-with-data-state A)
 staged-security-with-data-state-transcript-pre-hit
 adversary-initial-state $\leq P$
and sampled-transcript-new-bound:
 wp-event (checked-staged-security-experiment-with-data-state A)
 staged-security-with-data-state-transcript-new-hit
 adversary-initial-state $\leq R$
and current-empty-bound:
 wp-event (checked-staged-security-experiment-with-data-state A)
 (staged-security-with-data-state-current-empty-query-opening-hit-at 0)
 adversary-initial-state $\leq$ empty-query-error
shows checked-staged-adversary-acceptance-probability $A \leq$
 hash-collision-budget-value 0
 (staged-attacker-query-budget budgets +
 staged-challenge-query-budget + verifier-hash-query-budget) +
 ($\sum i < \text{rounds. } C\ i$) +
 trace-fri-error + composition-fri-error +
 ($\sum i < \text{rounds. } C\ i$) +
 (composition-error-bound +
 staged-phase-relation-error size
 (staged-alpha-search-queries budgets 0) +
 hash-collision-budget-value 0
 (staged-alpha-search-queries budgets 0) +
 ((composition-error-bound +
 staged-phase-relation-error size
 (staged-alpha-search-queries budgets 0) +
 hash-collision-budget-value 0
 (staged-alpha-search-queries budgets 0) +
 (hash-collision-budget-value 0
 (staged-alpha-search-queries budgets 0) +
 $X + (P + R) + (P + R))) + X) +$
 staged-phase-relation-error size
 (staged-alpha-search-queries budgets 0) +
 trace-fri-error + empty-query-error)
proof –
let ?M =
 checked-staged-security-experiment-with-actual-alpha-prefix-data-state A
let ?N =
 hash-collision-budget-value 0
 (staged-alpha-search-queries budgets 0) +

$X + (P + R) + (P + R)$
have *no-prefix-bound*:
wp-event ?*M*
checked-staged-security-with-actual-alpha-prefix-not-prefix-no-prefix-candidate
adversary-initial-state $\leq P + R$
by (rule
checked-staged-security-with-actual-alpha-prefix-not-prefix-no-prefix-candidate-bound-from-sampled-transcript
[*OF data-pre-bound sampled-transcript-new-bound*])
have *path-bound*:
wp-event ?*M*
checked-staged-security-with-actual-alpha-prefix-not-prefix-path-output
adversary-initial-state $\leq P + R$
by (rule
checked-staged-security-with-actual-alpha-prefix-not-prefix-path-output-bound-from-sampled-transcript
[*OF data-pre-bound sampled-transcript-new-bound*])
have *collision-bound*:
wp-event ?*M*
checked-staged-security-with-actual-alpha-prefix-not-prefix-collision
adversary-initial-state $\leq$
hash-collision-budget-value 0 (*staged-alpha-search-queries* budgets 0)
by (rule
checked-staged-security-with-actual-alpha-prefix-not-prefix-collision-bound
[*OF wf controlled*])
have *uncovered-bound*:
wp-event ?*M*
checked-staged-security-with-actual-alpha-prefix-not-prefix-uncovered-index
adversary-initial-state $\leq X$
by (rule *order-trans*
[*OF*
checked-staged-security-with-actual-alpha-prefix-not-prefix-uncovered-index-le-witnessed-cross-or-merkle-bad
cross-bound])
have *not-prefix-bound*:
wp-event ?*M*
(*checked-staged-security-with-actual-alpha-prefix-partial-header-candidate-not-prefix-bound*
composition-trace-bad-alpha-space)
adversary-initial-state $\leq ?N$
by (rule
checked-staged-security-with-actual-alpha-prefix-not-prefix-bound-union-bound
[*OF wf controlled collision-bound uncovered-bound no-prefix-bound*
path-bound])
show ?*thesis*
by (rule
stark-soundness-from-aligned-partial-components-current-query-prefix-and-current-empty-query-reachable-fri-split
[*OF false-statement wf controlled trace-fri-reduction*
composition-fri-reduction round-bound not-prefix-bound cross-bound
current-empty-bound])
qed

theorem *stark-soundness-from-aligned-partial-components-current-query-prefix-and-current-empty-query-reachable-fri-split*

fixes X P *empty-query-error* :: *prob*
and C :: *nat* $\Rightarrow$ *prob*
assumes *false-statement*: $\neg$ *exists-valid-trace*
and *wf*: *staged-budget-wellformed budgets*
and *controlled*: *staged-adversary-controlled budgets A*
and *trace-fri-reduction*:
 $\bigwedge$ *data attacker-state*.
Some (data, attacker-state) $\in$
set-dist (execute (checked-staged-transcript-program A)
adversary-initial-state) $\Rightarrow$
trace-fri-reachable-partial-candidate-reduction-assumption
(verifier-state-from-adversary attacker-state
(staged-proof-transcript data))
and *composition-fri-reduction*:
 $\bigwedge$ *data attacker-state*.
Some (data, attacker-state) $\in$
set-dist (execute (checked-staged-transcript-program A)
adversary-initial-state) $\Rightarrow$
composition-fri-reachable-partial-candidate-reduction-assumption
(verifier-state-from-adversary attacker-state
(staged-proof-transcript data))
and *round-bound*:
 $\bigwedge i. i < \text{rounds} \Rightarrow$
wp-event
(checked-staged-security-experiment-with-query-prefix-data-state A i)
(checked-staged-security-with-query-prefix-current-query-partial-opening-hit-at i)
adversary-initial-state $\leq C$ i
and *cross-bound*:
wp-event
(checked-staged-security-experiment-with-actual-alpha-prefix-data-state
A)
checked-staged-security-with-actual-alpha-prefix-witnessed-cross-or-merkle-bad
adversary-initial-state $\leq X$
and *data-pre-bound*:
wp-event (checked-staged-security-experiment-with-data-state A)
staged-security-with-data-state-transcript-pre-hit
adversary-initial-state $\leq P$
and *current-empty-bound*:
wp-event (checked-staged-security-experiment-with-data-state A)
(staged-security-with-data-state-current-empty-query-opening-hit-at 0)
adversary-initial-state $\leq$ empty-query-error
shows *checked-staged-adversary-acceptance-probability A $\leq$*
hash-collision-budget-value 0
(staged-attacker-query-budget budgets +
staged-challenge-query-budget + verifier-hash-query-budget) +
($\sum i < \text{rounds}. C$ i) +
trace-fri-error + composition-fri-error +
($\sum i < \text{rounds}. C$ i) +
(composition-error-bound +

```

    staged-phase-relation-error size
      (staged-alpha-search-queries budgets 0) +
    hash-collision-budget-value 0
      (staged-alpha-search-queries budgets 0) +
    ((composition-error-bound +
      staged-phase-relation-error size
        (staged-alpha-search-queries budgets 0) +
      hash-collision-budget-value 0
        (staged-alpha-search-queries budgets 0) +
      (hash-collision-budget-value 0
        (staged-alpha-search-queries budgets 0) +
        X + (P + staged-concrete-transcript-target-error-bound) +
        (P + staged-concrete-transcript-target-error-bound))) + X) +
    staged-phase-relation-error size
      (staged-alpha-search-queries budgets 0) +
    trace-fri-error + empty-query-error)
  proof -
    have sampled-transcript-new-bound:
      wp-event (checked-staged-security-experiment-with-data-state A)
        staged-security-with-data-state-transcript-new-hit
        adversary-initial-state ≤ staged-concrete-transcript-target-error-bound
    by (rule checked-staged-security-with-data-state-transcript-new-hit-bound
      [OF wf controlled])
    show ?thesis
    by (rule
      stark-soundness-from-aligned-partial-components-current-query-prefix-and-current-empty-query-reachable-fri-spli
      [OF false-statement wf controlled trace-fri-reduction
        composition-fri-reduction round-bound cross-bound data-pre-bound
        sampled-transcript-new-bound current-empty-bound])
  qed

theorem stark-soundness-from-aligned-partial-components-current-query-prefix-and-current-empty-query-reachable-fri-spli
  fixes S H P empty-query-error :: prob
  and C :: nat ⇒ prob
  assumes false-statement: ¬ exists-valid-trace
  and wf: staged-budget-wellformed budgets
  and controlled: staged-adversary-controlled budgets A
  and trace-fri-reduction:
    ∧ data attacker-state.
      Some (data, attacker-state) ∈
        set-dist (execute (checked-staged-transcript-program A)
          adversary-initial-state) ⇒
        trace-fri-reachable-partial-candidate-reduction-assumption
          (verifier-state-from-adversary attacker-state
            (staged-proof-transcript data))
  and composition-fri-reduction:
    ∧ data attacker-state.
      Some (data, attacker-state) ∈
        set-dist (execute (checked-staged-transcript-program A)

```

$\text{adversary-initial-state}) \implies$
composition-fri-reachable-partial-candidate-reduction-assumption
 $(\text{verifier-state-from-adversary attacker-state}$
 $(\text{staged-proof-transcript data}))$
and *round-bound*:
 $\bigwedge i. i < \text{rounds} \implies$
 wp-event
 $(\text{checked-staged-security-experiment-with-query-prefix-data-state } A \ i)$
 $(\text{checked-staged-security-with-query-prefix-current-query-partial-opening-hit-at } i)$
 $\text{adversary-initial-state} \leq C \ i$
and *sparse-bound*:
 wp-event
 $(\text{checked-staged-security-experiment-with-actual-alpha-prefix-data-state}$
 $A)$
 $\text{checked-staged-security-with-actual-alpha-prefix-witnessed-sparse-cross-bad}$
 $\text{adversary-initial-state} \leq S$
and *merkle-side-bound*:
 wp-event
 $(\text{checked-staged-security-experiment-with-actual-alpha-prefix-data-state}$
 $A)$
 $\text{checked-staged-security-with-actual-alpha-prefix-witnessed-merkle-side-bad}$
 $\text{adversary-initial-state} \leq H$
and *data-pre-bound*:
 $\text{wp-event } (\text{checked-staged-security-experiment-with-data-state } A)$
 $\text{staged-security-with-data-state-transcript-pre-hit}$
 $\text{adversary-initial-state} \leq P$
and *current-empty-bound*:
 $\text{wp-event } (\text{checked-staged-security-experiment-with-data-state } A)$
 $(\text{staged-security-with-data-state-current-empty-query-opening-hit-at } 0)$
 $\text{adversary-initial-state} \leq \text{empty-query-error}$
shows $\text{checked-staged-adversary-acceptance-probability } A \leq$
 $\text{hash-collision-budget-value } 0$
 $(\text{staged-attacker-query-budget budgets} +$
 $\text{staged-challenge-query-budget} + \text{verifier-hash-query-budget}) +$
 $(\sum i < \text{rounds}. C \ i) +$
 $\text{trace-fri-error} + \text{composition-fri-error} +$
 $(\sum i < \text{rounds}. C \ i) +$
 $(\text{composition-error-bound} +$
 $\text{staged-phase-relation-error size}$
 $(\text{staged-alpha-search-queries budgets } 0) +$
 $\text{hash-collision-budget-value } 0$
 $(\text{staged-alpha-search-queries budgets } 0) +$
 $((\text{composition-error-bound} +$
 $\text{staged-phase-relation-error size}$
 $(\text{staged-alpha-search-queries budgets } 0) +$
 $\text{hash-collision-budget-value } 0$
 $(\text{staged-alpha-search-queries budgets } 0) +$
 $(\text{hash-collision-budget-value } 0$
 $(\text{staged-alpha-search-queries budgets } 0) +$

$(S + H) + (P + \text{staged-concrete-transcript-target-error-bound}) +$
 $(P + \text{staged-concrete-transcript-target-error-bound})) + (S + H) +$
 $\text{staged-phase-relation-error size}$
 $(\text{staged-alpha-search-queries budgets } 0) +$
 $\text{trace-fri-error} + \text{empty-query-error})$

proof –

have *cross-bound*:

wp-event

$(\text{checked-staged-security-experiment-with-actual-alpha-prefix-data-state}$
 $A)$
 $\text{checked-staged-security-with-actual-alpha-prefix-witnessed-cross-or-merkle-bad}$
 $\text{adversary-initial-state} \leq S + H$

proof (rule *order-trans*)

show *wp-event*

$(\text{checked-staged-security-experiment-with-actual-alpha-prefix-data-state}$
 $A)$
 $\text{checked-staged-security-with-actual-alpha-prefix-witnessed-cross-or-merkle-bad}$
 $\text{adversary-initial-state} \leq$
 wp-event
 $(\text{checked-staged-security-experiment-with-actual-alpha-prefix-data-state}$
 $A)$
 $\text{checked-staged-security-with-actual-alpha-prefix-witnessed-sparse-cross-bad}$
 $\text{adversary-initial-state} +$
 wp-event
 $(\text{checked-staged-security-experiment-with-actual-alpha-prefix-data-state}$
 $A)$
 $\text{checked-staged-security-with-actual-alpha-prefix-witnessed-merkle-side-bad}$
 $\text{adversary-initial-state}$
by (rule
 $\text{checked-staged-security-with-actual-alpha-prefix-witnessed-cross-or-merkle-bad-le-sparse-or-merkle-side})$

show $\dots \leq S + H$

by (intro *add-mono sparse-bound merkle-side-bound*)

qed

show *?thesis*

by (rule
 $\text{stark-soundness-from-aligned-partial-components-current-query-prefix-and-current-empty-query-reachable-fri-split}$
 $[\text{OF false-statement wf controlled trace-fri-reduction}$
 $\text{composition-fri-reduction round-bound cross-bound data-pre-bound}$
 $\text{current-empty-bound}]$)

qed

theorem *stark-soundness-from-aligned-partial-components-current-query-prefix-and-current-empty-query-reachable-fri-split*

fixes $S L K P \text{ empty-query-error} :: \text{prob}$

and $C :: \text{nat} \Rightarrow \text{prob}$

assumes *false-statement*: $\neg \text{exists-valid-trace}$

and *wf*: *staged-budget-wellformed* *budgets*

and *controlled*: *staged-adversary-controlled* *budgets* A

and *trace-fri-reduction*:

$\bigwedge \text{data attacker-state.}$

Some (data, attacker-state) ∈
set-dist (execute (checked-staged-transcript-program A)
adversary-initial-state) ⇒
trace-fri-reachable-partial-candidate-reduction-assumption
(verifier-state-from-adversary attacker-state
(staged-proof-transcript data))
and *composition-fri-reduction:*
 $\bigwedge$ *data attacker-state.*
Some (data, attacker-state) ∈
set-dist (execute (checked-staged-transcript-program A)
adversary-initial-state) ⇒
composition-fri-reachable-partial-candidate-reduction-assumption
(verifier-state-from-adversary attacker-state
(staged-proof-transcript data))
and *round-bound:*
 $\bigwedge i. i < \text{rounds} \Rightarrow$
wp-event
(checked-staged-security-experiment-with-query-prefix-data-state A i)
(checked-staged-security-with-query-prefix-current-query-partial-opening-hit-at i)
adversary-initial-state ≤ C i
and *sparse-bound:*
wp-event
(checked-staged-security-experiment-with-actual-alpha-prefix-data-state
A)
checked-staged-security-with-actual-alpha-prefix-witnessed-sparse-cross-bad
adversary-initial-state ≤ S
and *local-collision-bound:*
wp-event
(checked-staged-security-experiment-with-actual-alpha-prefix-data-state
A)
checked-staged-security-with-actual-alpha-prefix-witnessed-local-collision-bad
adversary-initial-state ≤ L
and *coupling-bound:*
wp-event
(checked-staged-security-experiment-with-actual-alpha-prefix-data-state
A)
checked-staged-security-with-actual-alpha-prefix-witnessed-coupling-bad
adversary-initial-state ≤ K
and *data-pre-bound:*
wp-event (checked-staged-security-experiment-with-data-state A)
staged-security-with-data-state-transcript-pre-hit
adversary-initial-state ≤ P
and *current-empty-bound:*
wp-event (checked-staged-security-experiment-with-data-state A)
(staged-security-with-data-state-current-empty-query-opening-hit-at 0)
adversary-initial-state ≤ empty-query-error
shows *checked-staged-adversary-acceptance-probability A ≤*
hash-collision-budget-value 0
(staged-attacker-query-budget budgets +

$$\begin{aligned}
& \text{staged-challenge-query-budget} + \text{verifier-hash-query-budget}) + \\
& (\sum_{i < \text{rounds}} C\ i) + \\
& \text{trace-fri-error} + \text{composition-fri-error} + \\
& (\sum_{i < \text{rounds}} C\ i) + \\
& (\text{composition-error-bound} + \\
& \quad \text{staged-phase-relation-error size} \\
& \quad (\text{staged-alpha-search-queries budgets } 0) + \\
& \quad \text{hash-collision-budget-value } 0 \\
& \quad (\text{staged-alpha-search-queries budgets } 0) + \\
& ((\text{composition-error-bound} + \\
& \quad \text{staged-phase-relation-error size} \\
& \quad (\text{staged-alpha-search-queries budgets } 0) + \\
& \quad \text{hash-collision-budget-value } 0 \\
& \quad (\text{staged-alpha-search-queries budgets } 0) + \\
& (\text{hash-collision-budget-value } 0 \\
& \quad (\text{staged-alpha-search-queries budgets } 0) + \\
& (S + (L + K)) + (P + \text{staged-concrete-transcript-target-error-bound}) + \\
& (P + \text{staged-concrete-transcript-target-error-bound}))) + \\
& (S + (L + K))) + \\
& \text{staged-phase-relation-error size} \\
& (\text{staged-alpha-search-queries budgets } 0) + \\
& \text{trace-fri-error} + \text{empty-query-error})
\end{aligned}$$

proof –

have *merkle-side-bound*:

wp-event

(*checked-staged-security-experiment-with-actual-alpha-prefix-data-state*
A)

checked-staged-security-with-actual-alpha-prefix-witnessed-merkle-side-bad
adversary-initial-state $\leq L + K$

proof (rule *order-trans*)

show *wp-event*

(*checked-staged-security-experiment-with-actual-alpha-prefix-data-state*
A)

checked-staged-security-with-actual-alpha-prefix-witnessed-merkle-side-bad
adversary-initial-state $\leq$

wp-event

(*checked-staged-security-experiment-with-actual-alpha-prefix-data-state*
A)

checked-staged-security-with-actual-alpha-prefix-witnessed-local-collision-bad
adversary-initial-state +

wp-event

(*checked-staged-security-experiment-with-actual-alpha-prefix-data-state*
A)

checked-staged-security-with-actual-alpha-prefix-witnessed-coupling-bad
adversary-initial-state

by (rule
checked-staged-security-with-actual-alpha-prefix-witnessed-merkle-side-bad-le-local-or-coupling)

show ... $\leq L + K$

by (*intro add-mono local-collision-bound coupling-bound*)


```

qed
show ?thesis
by (rule
  stark-soundness-from-aligned-partial-components-current-query-prefix-and-current-empty-query-reachable-fri-spli
  [OF false-statement wf controlled trace-fri-reduction
    composition-fri-reduction round-bound sparse-bound
    merkle-side-bound data-pre-bound current-empty-bound])
qed
end

```

These wrappers expose the broad witnessed cross/Merkle diagnostic split used during development of the reachable-FRI route. They remain available only as internal plumbing; the public route should use aligned transcript-indexed wrappers that do not expose these diagnostic events.

```

hide-fact
  soundness.stark-soundness-from-aligned-partial-components-current-query-prefix-and-current-empty-query-reachable
  soundness.stark-soundness-from-aligned-partial-components-current-query-prefix-and-current-empty-query-reachable
  soundness.stark-soundness-from-aligned-partial-components-current-query-prefix-and-current-empty-query-reachable
  soundness.stark-soundness-from-aligned-partial-components-current-query-prefix-and-current-empty-query-reachable
  soundness.stark-soundness-from-aligned-partial-components-current-query-prefix-and-current-empty-query-reachable
  soundness.stark-soundness-from-aligned-partial-components-current-query-prefix-and-current-empty-query-reachable
  soundness.stark-soundness-from-aligned-partial-components-current-query-prefix-and-current-empty-query-reachable
  soundness.stark-soundness-from-aligned-partial-components-current-query-prefix-and-current-empty-query-reachable
  soundness.stark-soundness-from-aligned-partial-components-current-query-prefix-and-current-empty-query-reachable
  soundness.stark-soundness-from-aligned-partial-components-current-query-prefix-and-current-empty-query-reachable
end

```

```

theory Soundness-FRI-Partial-Evidence
  imports Soundness-Relevant-Drift-Reachable-FRI
begin

```

Normalized partial-opening FRI evidence predicates for the reachable-state FRI obligations.

This theory is intentionally narrow. It does not change the public soundness theorem path; it only factors the remaining reachable FRI obligations into explicit evidence predicates that later proofs can target.

```

context soundness
begin

```

```

definition trace-fri-partial-candidate-evidence
  :: ('f, 'a) protocol-channel-scheme  $\Rightarrow$ 
    (unit list  $\times$  ('f, 'a) protocol-channel-scheme) option  $\Rightarrow$ 
    'f  $\Rightarrow$  nat list  $\Rightarrow$  'f authenticated-opening list list  $\Rightarrow$ 
    'f list  $\Rightarrow$  bool
where

```

trace-fri-partial-candidate-evidence s out fr query-idxs
trace-openings trace-table $\longleftrightarrow$
accepted-with-partial-trace-openings s out fr query-idxs
trace-openings $\wedge$
partial-trace-table-candidate trace-table trace-openings $\wedge$
 $\neg$ trace-table-low-degree trace-table

definition *composition-fri-partial-candidate-evidence*

:: (f, 'a) protocol-channel-scheme $\Rightarrow$
(unit list $\times$ (f, 'a) protocol-channel-scheme) option $\Rightarrow$
'f $\Rightarrow$ 'f list $\Rightarrow$ 'f $\Rightarrow$ 'f list $\Rightarrow$ 'f $\Rightarrow$
'f list $\Rightarrow$ 'f $\Rightarrow$ nat list $\Rightarrow$
'f authenticated-opening list list $\Rightarrow$ nat list $\Rightarrow$
'f authenticated-opening list list $\Rightarrow$ 'f list $\Rightarrow$
'f list $\Rightarrow$ bool

where

composition-fri-partial-candidate-evidence s out fr f-fri-roots
f-final as dg composition-fri-roots final trace-query-idxs
trace-openings composition-query-idxs composition-openings trace-table
composition-table $\longleftrightarrow$
accepted-with-partial-initial-openings s out fr f-fri-roots f-final
as dg composition-fri-roots final trace-query-idxs trace-openings
composition-query-idxs composition-openings $\wedge$
partial-trace-table-candidate trace-table trace-openings $\wedge$
partial-composition-table-candidate composition-table
composition-openings $\wedge$
trace-table-low-degree trace-table $\wedge$
 $\neg$ composition-table-low-degree maxDegree composition-table

lemma *trace-fri-bad-with-reachable-partial-candidate-iff-evidence:*

trace-fri-bad-with-reachable-partial-candidate s out $\longleftrightarrow$
($\exists$ fr query-idxs trace-openings trace-table.
trace-fri-partial-candidate-evidence s out fr query-idxs
trace-openings trace-table)

unfolding *trace-fri-bad-with-reachable-partial-candidate-def*
trace-fri-partial-candidate-evidence-def

by *blast*

lemma *composition-fri-bad-with-reachable-partial-candidate-iff-evidence:*

composition-fri-bad-with-reachable-partial-candidate s out $\longleftrightarrow$
($\exists$ fr f-fri-roots f-final as dg composition-fri-roots final
trace-query-idxs trace-openings composition-query-idxs
composition-openings trace-table composition-table.
composition-fri-partial-candidate-evidence s out fr f-fri-roots
f-final as dg composition-fri-roots final trace-query-idxs
trace-openings composition-query-idxs composition-openings
trace-table composition-table)

unfolding *composition-fri-bad-with-reachable-partial-candidate-def*
composition-fri-partial-candidate-evidence-def

by *blast*

lemma *trace-fri-partial-candidate-evidenceI*:

assumes

accepted-with-partial-trace-openings s out fr query-idxs
trace-openings
partial-trace-table-candidate trace-table trace-openings
 $\neg$ *trace-table-low-degree trace-table*

shows

trace-fri-partial-candidate-evidence s out fr query-idxs
trace-openings trace-table

using *assms* **unfolding** *trace-fri-partial-candidate-evidence-def* **by** *blast*

lemma *trace-fri-partial-candidate-evidenceD*:

assumes

trace-fri-partial-candidate-evidence s out fr query-idxs
trace-openings trace-table

shows

accepted-with-partial-trace-openings s out fr query-idxs
trace-openings
partial-trace-table-candidate trace-table trace-openings
 $\neg$ *trace-table-low-degree trace-table*

using *assms* **unfolding** *trace-fri-partial-candidate-evidence-def* **by** *blast+*

lemma *trace-fri-bad-with-reachable-partial-candidateE*:

assumes *trace-fri-bad-with-reachable-partial-candidate s out*

obtains *fr query-idxs trace-openings trace-table* **where**

trace-fri-partial-candidate-evidence s out fr query-idxs
trace-openings trace-table

using *assms*

unfolding *trace-fri-bad-with-reachable-partial-candidate-iff-evidence*

by *blast*

lemma *composition-fri-partial-candidate-evidenceI*:

assumes

accepted-with-partial-initial-openings s out fr f-fri-roots f-final
as dg composition-fri-roots final trace-query-idxs trace-openings
composition-query-idxs composition-openings
partial-trace-table-candidate trace-table trace-openings
partial-composition-table-candidate composition-table
composition-openings
trace-table-low-degree trace-table
 $\neg$ *composition-table-low-degree maxDegree composition-table*

shows

composition-fri-partial-candidate-evidence s out fr f-fri-roots
f-final as dg composition-fri-roots final trace-query-idxs
trace-openings composition-query-idxs composition-openings
trace-table composition-table

using *assms*

unfolding *composition-fri-partial-candidate-evidence-def* **by** *blast*

lemma *composition-fri-partial-candidate-evidenceD*:

assumes

composition-fri-partial-candidate-evidence s out fr f-fri-roots
f-final as dg composition-fri-roots final trace-query-idxs
trace-openings composition-query-idxs composition-openings trace-table
composition-table

shows

accepted-with-partial-initial-openings s out fr f-fri-roots f-final
as dg composition-fri-roots final trace-query-idxs trace-openings
composition-query-idxs composition-openings
partial-trace-table-candidate trace-table trace-openings
partial-composition-table-candidate composition-table
composition-openings
trace-table-low-degree trace-table
 $\neg$ *composition-table-low-degree maxDegree composition-table*

using *assms*

unfolding *composition-fri-partial-candidate-evidence-def* **by** *blast+*

lemma *composition-fri-bad-with-reachable-partial-candidateE*:

assumes *composition-fri-bad-with-reachable-partial-candidate s out*

obtains *fr f-fri-roots f-final as dg composition-fri-roots final*

trace-query-idxs trace-openings composition-query-idxs
composition-openings trace-table composition-table **where**
composition-fri-partial-candidate-evidence s out fr f-fri-roots
f-final as dg composition-fri-roots final trace-query-idxs
trace-openings composition-query-idxs composition-openings
trace-table composition-table

using *assms*

unfolding *composition-fri-bad-with-reachable-partial-candidate-iff-evidence*

by *blast*

lemma *trace-fri-reachable-partial-candidate-reductionD*:

assumes *trace-fri-reachable-partial-candidate-reduction-assumption s*

shows

wp-event verify-monad
 $(\text{trace-fri-bad-with-reachable-partial-candidate } s) \ s \leq$
trace-fri-error

using *assms*

unfolding *trace-fri-reachable-partial-candidate-reduction-assumption-def*

by *simp*

lemma *composition-fri-reachable-partial-candidate-reductionD*:

assumes *composition-fri-reachable-partial-candidate-reduction-assumption s*

shows

wp-event verify-monad
 $(\text{composition-fri-bad-with-reachable-partial-candidate } s) \ s \leq$
composition-fri-error

using *assms*

unfolding *composition-fri-reachable-partial-candidate-reduction-assumption-def*
by *simp*

definition *trace-fri-partial-candidate-opening-evidence*

$:: (f, 'a) \text{ protocol-channel-scheme} \Rightarrow$
 $(\text{unit list} \times (f, 'a) \text{ protocol-channel-scheme}) \text{ option} \Rightarrow$
 $'f \text{ list} \Rightarrow 'f \text{ list} \Rightarrow 'f \Rightarrow 'f \Rightarrow$
 $'f \text{ list} \Rightarrow 'f \text{ list} \Rightarrow 'f \Rightarrow \text{nat list} \Rightarrow$
 $'f \text{ list list list} \Rightarrow 'f \text{ list list list} \Rightarrow$
 $'f \Rightarrow \text{nat list} \Rightarrow 'f \text{ authenticated-opening list list} \Rightarrow$
 $'f \text{ list} \Rightarrow \text{bool}$

where

trace-fri-partial-candidate-opening-evidence s out trace-roots
trace-bs trace-final dg composition-roots composition-bs
composition-final fri-query-idxs trace-round-layers
composition-round-layers fr candidate-query-idxs trace-openings
trace-table $\longleftrightarrow$
accepted-fri-opening-transcript s out trace-roots trace-bs trace-final
dg composition-roots composition-bs composition-final fri-query-idxs
trace-round-layers composition-round-layers $\wedge$
trace-fri-partial-candidate-evidence s out fr candidate-query-idxs
trace-openings trace-table

definition *composition-fri-partial-candidate-opening-evidence*

$:: (f, 'a) \text{ protocol-channel-scheme} \Rightarrow$
 $(\text{unit list} \times (f, 'a) \text{ protocol-channel-scheme}) \text{ option} \Rightarrow$
 $'f \text{ list} \Rightarrow 'f \text{ list} \Rightarrow 'f \Rightarrow 'f \Rightarrow$
 $'f \text{ list} \Rightarrow 'f \text{ list} \Rightarrow 'f \Rightarrow \text{nat list} \Rightarrow$
 $'f \text{ list list list} \Rightarrow 'f \text{ list list list} \Rightarrow$
 $'f \Rightarrow 'f \text{ list} \Rightarrow 'f \Rightarrow 'f \text{ list} \Rightarrow 'f \Rightarrow$
 $'f \text{ list} \Rightarrow 'f \Rightarrow \text{nat list} \Rightarrow$
 $'f \text{ authenticated-opening list list} \Rightarrow \text{nat list} \Rightarrow$
 $'f \text{ authenticated-opening list list} \Rightarrow 'f \text{ list} \Rightarrow$
 $'f \text{ list} \Rightarrow \text{bool}$

where

composition-fri-partial-candidate-opening-evidence s out trace-roots
trace-bs trace-final fri-dg opening-composition-roots composition-bs
composition-final fri-query-idxs trace-round-layers
composition-round-layers fr f-fri-roots f-final as dg
composition-fri-roots final trace-query-idxs trace-openings
composition-query-idxs composition-openings trace-table
composition-table $\longleftrightarrow$
accepted-fri-opening-transcript s out trace-roots trace-bs trace-final
fri-dg opening-composition-roots composition-bs composition-final
fri-query-idxs trace-round-layers composition-round-layers $\wedge$
composition-fri-partial-candidate-evidence s out fr f-fri-roots
f-final as dg composition-fri-roots final trace-query-idxs
trace-openings composition-query-idxs composition-openings

trace-table composition-table

lemma *trace-fri-partial-candidate-opening-evidenceD:*

assumes

trace-fri-partial-candidate-opening-evidence s out trace-roots
trace-bs trace-final dg composition-roots composition-bs
composition-final fri-query-idxs trace-round-layers
composition-round-layers fr candidate-query-idxs trace-openings
trace-table

shows

accepted-fri-opening-transcript s out trace-roots trace-bs
trace-final dg composition-roots composition-bs composition-final
fri-query-idxs trace-round-layers composition-round-layers
trace-fri-partial-candidate-evidence s out fr candidate-query-idxs
trace-openings trace-table

using *assms*

unfolding *trace-fri-partial-candidate-opening-evidence-def* **by** *blast+*

lemma *composition-fri-partial-candidate-opening-evidenceD:*

assumes

composition-fri-partial-candidate-opening-evidence s out trace-roots
trace-bs trace-final fri-dg opening-composition-roots composition-bs
composition-final fri-query-idxs trace-round-layers
composition-round-layers fr f-fri-roots f-final as dg
composition-fri-roots final trace-query-idxs trace-openings
composition-query-idxs composition-openings trace-table
composition-table

shows

accepted-fri-opening-transcript s out trace-roots trace-bs
trace-final fri-dg opening-composition-roots composition-bs
composition-final fri-query-idxs trace-round-layers
composition-round-layers
composition-fri-partial-candidate-evidence s out fr f-fri-roots
f-final as dg composition-fri-roots final trace-query-idxs
trace-openings composition-query-idxs composition-openings
trace-table composition-table

using *assms*

unfolding *composition-fri-partial-candidate-opening-evidence-def* **by** *blast+*

lemma *verify-monad-trace-fri-bad-extracts-opening-evidence:*

assumes *outcome:*

Some (result, final-state) ∈ set-dist (execute verify-monad s)

and *bad:*

trace-fri-bad-with-reachable-partial-candidate s
(Some (result, final-state))

obtains *trace-roots trace-bs trace-final dg composition-roots*

composition-bs composition-final fri-query-idxs trace-round-layers
composition-round-layers fr candidate-query-idxs trace-openings
trace-table

where
trace-fri-partial-candidate-opening-evidence s
(Some (result, final-state)) trace-roots trace-bs trace-final dg
composition-roots composition-bs composition-final fri-query-idxs
trace-round-layers composition-round-layers fr candidate-query-idxs
trace-openings trace-table

proof –
from *verify-monad-accepted-fri-opening-transcript[OF outcome]*
obtain *trace-roots trace-bs trace-final dg composition-roots*
composition-bs composition-final fri-query-idxs trace-round-layers
composition-round-layers **where** *fri-opening:*
accepted-fri-opening-transcript s (Some (result, final-state))
trace-roots trace-bs trace-final dg composition-roots composition-bs
composition-final fri-query-idxs trace-round-layers
composition-round-layers
by *blast*
from *trace-fri-bad-with-reachable-partial-candidateE[OF bad]*
obtain *fr candidate-query-idxs trace-openings trace-table* **where** *evidence:*
trace-fri-partial-candidate-evidence s
(Some (result, final-state)) fr candidate-query-idxs trace-openings
trace-table
by *blast*
have *combined:*
trace-fri-partial-candidate-opening-evidence s
(Some (result, final-state)) trace-roots trace-bs trace-final dg
composition-roots composition-bs composition-final fri-query-idxs
trace-round-layers composition-round-layers fr candidate-query-idxs
trace-openings trace-table
unfolding *trace-fri-partial-candidate-opening-evidence-def*
using *fri-opening evidence* **by** *simp*
show *?thesis*
by *(rule that[OF combined])*
qed

lemma *verify-monad-composition-fri-bad-extracts-opening-evidence:*
assumes *outcome:*
Some (result, final-state) ∈ set-dist (execute verify-monad s)
and *bad:*
composition-fri-bad-with-reachable-partial-candidate s
(Some (result, final-state))
obtains *trace-roots trace-bs trace-final fri-dg opening-composition-roots*
composition-bs composition-final fri-query-idxs trace-round-layers
composition-round-layers fr f-fri-roots f-final as dg
composition-fri-roots final trace-query-idxs trace-openings
composition-query-idxs composition-openings trace-table
composition-table
where
composition-fri-partial-candidate-opening-evidence s
(Some (result, final-state)) trace-roots trace-bs trace-final fri-dg

```

    opening-composition-roots composition-bs composition-final
    fri-query-idxs trace-round-layers composition-round-layers fr
    f-fri-roots f-final as dg composition-fri-roots final
    trace-query-idxs trace-openings composition-query-idxs
    composition-openings trace-table composition-table
proof —
from verify-monad-accepted-fri-opening-transcript[OF outcome]
obtain trace-roots trace-bs trace-final fri-dg opening-composition-roots
    composition-bs composition-final fri-query-idxs trace-round-layers
    composition-round-layers where fri-opening:
    accepted-fri-opening-transcript s (Some (result, final-state))
    trace-roots trace-bs trace-final fri-dg opening-composition-roots
    composition-bs composition-final fri-query-idxs trace-round-layers
    composition-round-layers
by blast
from composition-fri-bad-with-reachable-partial-candidateE[OF bad]
obtain fr f-fri-roots f-final as dg composition-fri-roots final
    trace-query-idxs trace-openings composition-query-idxs
    composition-openings trace-table composition-table where evidence:
    composition-fri-partial-candidate-evidence s
    (Some (result, final-state)) fr f-fri-roots f-final as dg
    composition-fri-roots final trace-query-idxs trace-openings
    composition-query-idxs composition-openings trace-table
    composition-table
by blast
have combined:
    composition-fri-partial-candidate-opening-evidence s
    (Some (result, final-state)) trace-roots trace-bs trace-final fri-dg
    opening-composition-roots composition-bs composition-final
    fri-query-idxs trace-round-layers composition-round-layers fr
    f-fri-roots f-final as dg composition-fri-roots final
    trace-query-idxs trace-openings composition-query-idxs
    composition-openings trace-table composition-table
unfolding composition-fri-partial-candidate-opening-evidence-def
using fri-opening evidence by simp
show ?thesis
by (rule that[OF combined])
qed

lemma verify-monad-composition-fri-bad-extracts-opening-evidence-with-degree-bound:
assumes outcome:
    Some (result, final-state) ∈ set-dist (execute verify-monad s)
and bad:
    composition-fri-bad-with-reachable-partial-candidate s
    (Some (result, final-state))
obtains trace-roots trace-bs trace-final fri-dg opening-composition-roots
    composition-bs composition-final fri-query-idxs trace-round-layers
    composition-round-layers fr f-fri-roots f-final as dg
    composition-fri-roots final trace-query-idxs trace-openings

```



```

    composition-query-idxs composition-openings trace-table
    composition-table
where
    composition-fri-partial-candidate-opening-evidence s
    (Some (result, final-state)) trace-roots trace-bs trace-final fri-dg
    opening-composition-roots composition-bs composition-final
    fri-query-idxs trace-round-layers composition-round-layers fr
    f-fri-roots f-final as dg composition-fri-roots final
    trace-query-idxs trace-openings composition-query-idxs
    composition-openings trace-table composition-table
and to-nat fri-dg ≤ maxDegree
proof (rule verify-monad-composition-fri-bad-extracts-opening-evidence
    [OF outcome bad])
fix trace-roots trace-bs trace-final fri-dg opening-composition-roots
    composition-bs composition-final fri-query-idxs trace-round-layers
    composition-round-layers fr f-fri-roots f-final as dg
    composition-fri-roots final trace-query-idxs trace-openings
    composition-query-idxs composition-openings trace-table composition-table
assume evidence:
    composition-fri-partial-candidate-opening-evidence s
    (Some (result, final-state)) trace-roots trace-bs trace-final fri-dg
    opening-composition-roots composition-bs composition-final
    fri-query-idxs trace-round-layers composition-round-layers fr
    f-fri-roots f-final as dg composition-fri-roots final
    trace-query-idxs trace-openings composition-query-idxs
    composition-openings trace-table composition-table
have fri-opening:
    accepted-fri-opening-transcript s (Some (result, final-state))
    trace-roots trace-bs trace-final fri-dg opening-composition-roots
    composition-bs composition-final fri-query-idxs trace-round-layers
    composition-round-layers
using composition-fri-partial-candidate-opening-evidenceD(1)
    [OF evidence] .
have challenges:
    accepted-fri-challenges s (Some (result, final-state))
    trace-bs fri-dg composition-bs
by (rule accepted-fri-opening-transcript-challenges[OF fri-opening])
have degree-bound: to-nat fri-dg ≤ maxDegree
by (rule verify-monad-accepted-fri-challenges-degree-bound
    [OF outcome challenges])
show ?thesis
by (rule that[OF evidence degree-bound])
qed

```

lemma verifier-query-round-after-index-final-checks:

assumes outcome:

Some ((), t) ∈
 set-dist
 (execute

```

      (verifier-query-round-after-index-program
       fr f-fl f-final as fl final raw) s)
obtains idx fv s1 f-i f-len f-pow s2 c-i c-len c-pow s4 where
  idx = index (to-nat raw)
  Some (fv, s1) ∈
    set-dist (execute (mmap (check-decommit-on-query fr idx)) s)
  Some ((f-i, f-final, f-len, f-pow), s2) ∈
    set-dist
      (execute
       (mfold (idx, hd fv, clength * scale, 1)
        (receive-query-commits f-fl)) s1)
  Some ((c-i, final, c-len, c-pow), s4) ∈
    set-dist
      (execute
       (mfold (idx, cp-eval as fv (h ^ idx * shift),
        clength * scale, 1)
        (receive-query-commits fl)) s2)
  t = s4
proof -
let ?idx = index (to-nat raw)
from outcome obtain fv s1 f-out s2 s3 c-out s4 where
  query-decommit:
    Some (fv, s1) ∈
      set-dist (execute (mmap (check-decommit-on-query fr ?idx)) s)
and trace-fri:
  Some (f-out, s2) ∈
    set-dist (execute
      (mfold (?idx, hd fv, clength * scale, 1)
       (receive-query-commits f-fl)) s1)
and assert-trace:
  Some ((), s3) ∈
    set-dist
      (execute
       (assert (case f-out of (-, f-x, -, -) ⇒ f-x = f-final)) s2)
and comp-fri:
  Some (c-out, s4) ∈
    set-dist (execute
      (mfold (?idx, cp-eval as fv (h ^ ?idx * shift),
       clength * scale, 1)
       (receive-query-commits fl)) s3)
and assert-comp:
  Some ((), t) ∈
    set-dist
      (execute
       (assert (case c-out of (-, x, -, -) ⇒ x = final)) s4)
unfolding verifier-query-round-after-index-program-def
by (auto simp: Let-def split: prod.splits elim!: set-dist-bindE)
obtain f-i f-x f-len f-pow where f-out-eq:
  f-out = (f-i, f-x, f-len, f-pow)

```

```

    by (cases f-out) auto
  have f-x-eq: f-x = f-final
    using assert-trace unfolding assert-def f-out-eq
    by (cases f-x = f-final) (auto simp: throw-no-outcome)
  have s3-eq: s3 = s2
    using assert-trace unfolding assert-def f-out-eq f-x-eq by simp
  obtain c-i c-x c-len c-pow where c-out-eq:
    c-out = (c-i, c-x, c-len, c-pow)
    by (cases c-out) auto
  have c-x-eq: c-x = final
    using assert-comp unfolding assert-def c-out-eq
    by (cases c-x = final) (auto simp: throw-no-outcome)
  have t-eq: t = s4
    using assert-comp unfolding assert-def c-out-eq c-x-eq by simp
  have trace-fri':
    Some ((f-i, f-final, f-len, f-pow), s2) ∈
      set-dist
        (execute
          (mfold (?idx, hd fv, clength * scale, 1)
            (receive-query-commits f-fl)) s1)
    using trace-fri f-out-eq f-x-eq by simp
  have comp-fri':
    Some ((c-i, final, c-len, c-pow), s4) ∈
      set-dist
        (execute
          (mfold (?idx, cp-eval as fv (h ^ ?idx * shift),
            clength * scale, 1)
            (receive-query-commits fl)) s2)
    using comp-fri c-out-eq c-x-eq s3-eq by simp
  show ?thesis
    by (rule that[OF refl query-decommit trace-fri' comp-fri' t-eq])
qed

```

```

lemma verifier-query-round-program-final-checks:
  assumes outcome:
    Some (( ), t) ∈
      set-dist
        (execute
          (verifier-query-round-program fr f-fl f-final as fl final) s)
  obtains raw idx fv s0 s1 f-i f-len f-pow s2 c-i c-len c-pow s4
  where
    Some (raw, s0) ∈ set-dist (execute receive-query-index-challenge s)
    idx = index (to-nat raw)
    Some (fv, s1) ∈
      set-dist (execute (mmap (check-decommit-on-query fr idx)) s0)
    Some ((f-i, f-final, f-len, f-pow), s2) ∈
      set-dist
        (execute
          (mfold (idx, hd fv, clength * scale, 1)

```

```

      (receive-query-commits f-fl)) s1)
Some ((c-i, final, c-len, c-pow), s4) ∈
  set-dist
    (execute
      (mfold (idx, cp-eval as fv (h ^ idx * shift),
        clength * scale, 1)
        (receive-query-commits fl)) s2)
t = s4
proof -
from outcome[unfolded verifier-query-round-program-alt-def]
obtain raw s0 where
  raw:
    Some (raw, s0) ∈
      set-dist (execute receive-query-index-challenge s)
and after:
    Some (( ), t) ∈
      set-dist
        (execute
          (verifier-query-round-after-index-program
            fr f-fl f-final as fl final raw) s0)
by (auto elim!: set-dist-bindE)
from verifier-query-round-after-index-final-checks[OF after]
obtain idx fv s1 f-i f-len f-pow s2 c-i c-len c-pow s4 where
  idx-eq: idx = index (to-nat raw)
and query-decommit:
    Some (fv, s1) ∈
      set-dist (execute (mmap (check-decommit-on-query fr idx)) s0)
and trace-fri:
    Some ((f-i, f-final, f-len, f-pow), s2) ∈
      set-dist
        (execute
          (mfold (idx, hd fv, clength * scale, 1)
            (receive-query-commits f-fl)) s1)
and comp-fri:
    Some ((c-i, final, c-len, c-pow), s4) ∈
      set-dist
        (execute
          (mfold (idx, cp-eval as fv (h ^ idx * shift),
            clength * scale, 1)
            (receive-query-commits fl)) s2)
and t-eq: t = s4
by blast
show ?thesis
by (rule that[OF raw idx-eq query-decommit trace-fri comp-fri t-eq])
qed

end

end

```

```

theory Soundness-FRI-Layer-Evidence
  imports Soundness-FRI-Partial-Evidence
begin

```

Partial FRI layer evidence extracted from the verifier transcript.

This theory only packages facts already exposed by the trace and composition layer-opening lemmas for *accepted-fri-opening-transcript*. It does not reconstruct complete FRI tables from sampled openings.

```

context soundness
begin

```

```

definition fri-layer-step-evidence
  :: 'f  $\Rightarrow$  'f  $\Rightarrow$  nat  $\Rightarrow$  nat  $\Rightarrow$  nat  $\Rightarrow$ 
    nat  $\Rightarrow$  'f  $\Rightarrow$  'f list  $\Rightarrow$  'f  $\Rightarrow$  'f list  $\Rightarrow$ 
    nat  $\Rightarrow$  'f  $\Rightarrow$  'f list  $\Rightarrow$  bool

```

where

```

  fri-layer-step-evidence rt ch len idx pw sibling-idx
    xp xp-path xn xn-path next-idx next-value chunk  $\longleftrightarrow$ 
    sibling-idx = fri-sibling-index len idx  $\wedge$ 
    next-idx = idx mod (len div 2)  $\wedge$ 
    next-value =
      fri-fold-value ch xp xn
      (fri-fold-denominator ((h  $\wedge$  idx) * shift) pw)  $\wedge$ 
    fri-layer-opening-chunk len xp xp-path xn xn-path chunk

```

```

definition fri-round-layer-evidence
  :: 'f list  $\Rightarrow$  'f list  $\Rightarrow$  nat list  $\Rightarrow$ 
    nat  $\Rightarrow$  nat  $\Rightarrow$  'f list list list  $\Rightarrow$  bool

```

where

```

  fri-round-layer-evidence roots challenges query-idxs round-idx
    layer-idx round-layers  $\longleftrightarrow$ 
    round-idx < length query-idxs  $\wedge$ 
    layer-idx < length roots  $\wedge$ 
    length challenges = length roots  $\wedge$ 
    length round-layers = length query-idxs  $\wedge$ 
    ( $\exists$  xp xp-path xn xn-path.
      fri-layer-step-evidence
        (roots ! layer-idx)
        (challenges ! layer-idx)
        (fri-layer-lengths (length roots) (clength * scale) ! layer-idx)
        (fri-layer-indices (length roots) (query-idxs ! round-idx)
          (clength * scale) ! layer-idx)
        (2  $\wedge$  layer-idx)
        (fri-sibling-index
          (fri-layer-lengths (length roots) (clength * scale) ! layer-idx)
          (fri-layer-indices (length roots) (query-idxs ! round-idx)
            (clength * scale) ! layer-idx))

```

```

xp xp-path xn xn-path
((fri-layer-indices (length roots) (query-idxs ! round-idx)
  (clength * scale) ! layer-idx) mod
  ((fri-layer-lengths (length roots) (clength * scale) ! layer-idx)
    div 2))
(fri-fold-value (challenges ! layer-idx) xp xn
  (fri-fold-denominator
    ((h ^
      (fri-layer-indices (length roots) (query-idxs ! round-idx)
        (clength * scale) ! layer-idx)) * shift)
      (2 ^ layer-idx)))
(round-layers ! round-idx ! layer-idx))

```

definition *fri-all-round-layer-evidence*

```

:: 'f list ⇒ 'f list ⇒ nat list ⇒
  'f list list list ⇒ bool

```

where

```

fri-all-round-layer-evidence roots challenges query-idxs round-layers
↔
length challenges = length roots ∧
length round-layers = length query-idxs ∧
(∀ round-idx < length query-idxs.
  ∀ layer-idx < length roots.
    fri-round-layer-evidence roots challenges query-idxs round-idx
      layer-idx round-layers)

```

definition *fri-evidence-layer-len* :: 'f list ⇒ nat ⇒ nat

where

```

fri-evidence-layer-len roots layer-idx =
  fri-layer-lengths (length roots) (clength * scale) ! layer-idx

```

definition *fri-evidence-layer-idx*

```

:: 'f list ⇒ nat list ⇒ nat ⇒ nat ⇒ nat

```

where

```

fri-evidence-layer-idx roots query-idxs round-idx layer-idx =
  fri-layer-indices (length roots) (query-idxs ! round-idx)
    (clength * scale) ! layer-idx

```

definition *fri-evidence-next-idx*

```

:: 'f list ⇒ nat list ⇒ nat ⇒ nat ⇒ nat

```

where

```

fri-evidence-next-idx roots query-idxs round-idx layer-idx =
  fri-evidence-layer-idx roots query-idxs round-idx layer-idx mod
    (fri-evidence-layer-len roots layer-idx div 2)

```

definition *fri-evidence-next-value*

```

:: 'f list ⇒ 'f list ⇒ nat list ⇒ nat ⇒ nat ⇒
  'f ⇒ 'f ⇒ 'f

```

where

fri-evidence-next-value roots challenges query-idxs round-idx layer-idx
 $xp\ xn =$
fri-fold-value (challenges ! layer-idx) $xp\ xn$
fri-fold-denominator
 $((h \wedge \text{fri-evidence-layer-idx roots query-idxs round-idx layer-idx}) * \text{shift})$
 $(2 \wedge \text{layer-idx})$

definition *fri-sampled-table-fold*

$:: 'f \Rightarrow \text{nat} \Rightarrow \text{nat} \Rightarrow \text{nat} \Rightarrow 'f \text{ list} \Rightarrow$
 $'f \text{ list} \Rightarrow 'f \text{ list} \Rightarrow 'f \Rightarrow \text{bool}$

where

fri-sampled-table-fold ch len idx pw fri-dom layer chunk next-value
 $\longleftrightarrow$
 $(\exists xp\ xp\text{-path}\ xn\ xn\text{-path}.$
 $\text{fri-layer-opening-chunk len } xp\ xp\text{-path}\ xn\ xn\text{-path}\ \text{chunk} \wedge$
 $\text{fri-opening-matches-table len idx layer } xp\ xn \wedge$
 $\text{fri-dom ! idx} = (h \wedge \text{idx} * \text{shift}) \wedge pw \wedge$
 $\text{next-value} = \text{fri-table-fold-value ch layer fri-dom len pw idx})$

definition *generic-fri-round-forced-next-value*

$:: 'f \text{ list} \Rightarrow 'f \text{ list} \Rightarrow \text{nat list} \Rightarrow$
 $'f \text{ list list list} \Rightarrow \text{nat} \Rightarrow \text{nat} \Rightarrow 'f \Rightarrow \text{bool}$

where

generic-fri-round-forced-next-value roots challenges query-idxs
round-layers round-idx layer-idx $v \longleftrightarrow$
 $(\exists xp\ xp\text{-path}\ xn\ xn\text{-path}.$
fri-layer-step-evidence
 $(\text{roots ! layer-idx})$
 $(\text{challenges ! layer-idx})$
 $(\text{fri-evidence-layer-len roots layer-idx})$
 $(\text{fri-evidence-layer-idx roots query-idxs round-idx layer-idx})$
 $(2 \wedge \text{layer-idx})$
 $(\text{fri-sibling-index} (\text{fri-evidence-layer-len roots layer-idx})$
 $(\text{fri-evidence-layer-idx roots query-idxs round-idx layer-idx}))$
 $xp\ xp\text{-path}\ xn\ xn\text{-path}$
 $(\text{fri-evidence-next-idx roots query-idxs round-idx layer-idx})$
 v
 $(\text{round-layers ! round-idx ! layer-idx}))$

lemma *fri-layer-step-evidenceI*:

assumes $\text{sibling-idx} = \text{fri-sibling-index len idx}$
and $\text{next-idx} = \text{idx mod (len div 2)}$
and $\text{next-value} =$
 $\text{fri-fold-value ch } xp\ xn$
 $(\text{fri-fold-denominator} ((h \wedge \text{idx}) * \text{shift}) pw)$
and *fri-layer-opening-chunk* len $xp\ xp\text{-path}\ xn\ xn\text{-path}\ \text{chunk}$
shows *fri-layer-step-evidence* rt ch len idx pw sibling-idx
 $xp\ xp\text{-path}\ xn\ xn\text{-path}\ \text{next-idx}\ \text{next-value}\ \text{chunk}$

using *assms* **unfolding** *fri-layer-step-evidence-def* **by** *simp*

lemma *fri-layer-step-evidenceD*:

assumes *fri-layer-step-evidence* *rt ch len idx pw sibling-idx*
xp xp-path xn xn-path next-idx next-value chunk
shows *sibling-idx = fri-sibling-index len idx*
and *next-idx = idx mod (len div 2)*
and *next-value =*
fri-fold-value ch xp xn
*(fri-fold-denominator ((h ^ idx) * shift) pw)*
and *fri-layer-opening-chunk len xp xp-path xn xn-path chunk*
using *assms* **unfolding** *fri-layer-step-evidence-def* **by** *simp-all*

lemma *fri-layer-opening-chunk-values-unique*:

assumes *first*:
fri-layer-opening-chunk len xp xp-path xn xn-path chunk
and *second*:
fri-layer-opening-chunk len yp yp-path yn yn-path chunk
shows *xp = yp*
and *xn = yn*
proof –
have *chunk-eq*:
 $[xp] @ xp-path @ [xn] @ xn-path =$
 $[yp] @ yp-path @ [yn] @ yn-path$
using *first second* **unfolding** *fri-layer-opening-chunk-def* **by** *simp*
then show *xp = yp*
by *simp*
have *len-eq*: *length xp-path = length yp-path*
using *first second* **unfolding** *fri-layer-opening-chunk-def* **by** *simp*
have *left*:
 $([xp] @ xp-path @ [xn] @ xn-path) ! \text{Suc } (\text{length } xp-path) = xn$
by *simp*
have *right*:
 $([yp] @ yp-path @ [yn] @ yn-path) ! \text{Suc } (\text{length } xp-path) = yn$
using *len-eq* **by** *simp*
show *xn = yn*
using *chunk-eq left right* **by** *metis*
qed

lemma *fri-round-layer-evidenceE*:

assumes *evidence*:
fri-round-layer-evidence roots challenges query-idxs round-idx
layer-idx round-layers
obtains *xp xp-path xn xn-path*
where *round-idx < length query-idxs*
and *layer-idx < length roots*
and *length challenges = length roots*
and *length round-layers = length query-idxs*
and *fri-layer-step-evidence*


```

    (roots ! layer-idx)
    (challenges ! layer-idx)
    (fri-layer-lengths (length roots) (clength * scale) ! layer-idx)
    (fri-layer-indices (length roots) (query-idxs ! round-idx)
      (clength * scale) ! layer-idx)
    (2 ^ layer-idx)
    (fri-sibling-index
      (fri-layer-lengths (length roots) (clength * scale) ! layer-idx)
      (fri-layer-indices (length roots) (query-idxs ! round-idx)
        (clength * scale) ! layer-idx))
    xp xp-path xn xn-path
    ((fri-layer-indices (length roots) (query-idxs ! round-idx)
      (clength * scale) ! layer-idx) mod
      ((fri-layer-lengths (length roots) (clength * scale) ! layer-idx)
        div 2))
    (fri-fold-value (challenges ! layer-idx) xp xn
      (fri-fold-denominator
        ((h ^
          (fri-layer-indices (length roots) (query-idxs ! round-idx)
            (clength * scale) ! layer-idx)) * shift)
          (2 ^ layer-idx))))
    (round-layers ! round-idx ! layer-idx)
  using evidence unfolding fri-round-layer-evidence-def by blast

```

lemma *fri-round-layer-evidence-compactE*:

assumes *evidence*:

fri-round-layer-evidence roots challenges query-idxs round-idx
 layer-idx round-layers

obtains *xp xp-path xn xn-path*

where *round-idx < length query-idxs*

and *layer-idx < length roots*

and *length challenges = length roots*

and *length round-layers = length query-idxs*

and *fri-layer-step-evidence*

```

    (roots ! layer-idx)
    (challenges ! layer-idx)
    (fri-evidence-layer-len roots layer-idx)
    (fri-evidence-layer-idx roots query-idxs round-idx layer-idx)
    (2 ^ layer-idx)
    (fri-sibling-index (fri-evidence-layer-len roots layer-idx)
      (fri-evidence-layer-idx roots query-idxs round-idx layer-idx))
    xp xp-path xn xn-path
    (fri-evidence-next-idx roots query-idxs round-idx layer-idx)
    (fri-evidence-next-value roots challenges query-idxs round-idx
      layer-idx xp xn)
    (round-layers ! round-idx ! layer-idx)

```

proof –

from *fri-round-layer-evidenceE[OF evidence]*

obtain *xp xp-path xn xn-path* **where**

```

round-bound: round-idx < length query-idxs
and layer-bound: layer-idx < length roots
and challenges-len: length challenges = length roots
and layers-len: length round-layers = length query-idxs
and step:
  fri-layer-step-evidence
    (roots ! layer-idx)
    (challenges ! layer-idx)
    (fri-layer-lengths (length roots) (clength * scale) ! layer-idx)
    (fri-layer-indices (length roots) (query-idxs ! round-idx)
      (clength * scale) ! layer-idx)
    (2 ^ layer-idx)
    (fri-sibling-index
      (fri-layer-lengths (length roots) (clength * scale) ! layer-idx)
      (fri-layer-indices (length roots) (query-idxs ! round-idx)
        (clength * scale) ! layer-idx))
    xp xp-path xn xn-path
    ((fri-layer-indices (length roots) (query-idxs ! round-idx)
      (clength * scale) ! layer-idx) mod
      ((fri-layer-lengths (length roots) (clength * scale) ! layer-idx)
        div 2))
    (fri-fold-value (challenges ! layer-idx) xp xn
      (fri-fold-denominator
        ((h ^
          (fri-layer-indices (length roots) (query-idxs ! round-idx)
            (clength * scale) ! layer-idx)) * shift)
          (2 ^ layer-idx)))
      (round-layers ! round-idx ! layer-idx))
  by blast
have compact-step:
  fri-layer-step-evidence
    (roots ! layer-idx)
    (challenges ! layer-idx)
    (fri-evidence-layer-len roots layer-idx)
    (fri-evidence-layer-idx roots query-idxs round-idx layer-idx)
    (2 ^ layer-idx)
    (fri-sibling-index (fri-evidence-layer-len roots layer-idx)
      (fri-evidence-layer-idx roots query-idxs round-idx layer-idx))
    xp xp-path xn xn-path
    (fri-evidence-next-idx roots query-idxs round-idx layer-idx)
    (fri-evidence-next-value roots challenges query-idxs round-idx
      layer-idx xp xn)
    (round-layers ! round-idx ! layer-idx)
  using step
unfolding fri-evidence-layer-len-def fri-evidence-layer-idx-def
  fri-evidence-next-idx-def fri-evidence-next-value-def
by simp
show ?thesis
by (rule that[OF round-bound layer-bound challenges-len layers-len

```

```

compact-step]])
qed

lemma fri-all-round-layer-evidenceD:
  assumes evidence:
    fri-all-round-layer-evidence roots challenges query-idxs round-layers
    and round-bound: round-idx < length query-idxs
    and layer-bound: layer-idx < length roots
  shows fri-round-layer-evidence roots challenges query-idxs round-idx
    layer-idx round-layers
  using assms unfolding fri-all-round-layer-evidence-def by blast

lemma fri-layer-step-evidence-from-outcome:
  assumes outcome:
    Some ((next-idx, next-value, next-len, next-pw), t) ∈
      set-dist (execute (fri-layer-opening-step (ch, rt)
        (idx, x, len, pw)) s)
  obtains xp xp-path xn xn-path chunk
  where fri-layer-step-evidence rt ch len idx pw
    (fri-sibling-index len idx) xp xp-path xn xn-path
    next-idx next-value chunk
    and chunk = [xp] @ xp-path @ [xn] @ xn-path
    and x = xp
    and next-len = len div 2
    and next-pw = pw + pw
  proof -
    from fri-layer-opening-step-outcome[OF outcome]
    obtain xp xp-path xn xn-path x' where
      out-eq:
        (next-idx, next-value, next-len, next-pw) =
          (idx mod (len div 2), x', len div 2, pw + pw)
    and xp-eq: xp = x
    and x'-eq:
      x' = fri-fold-value ch xp xn
        (fri-fold-denominator ((h ^ idx) * shift) pw)
    and chunk:
      fri-layer-opening-chunk len xp xp-path xn xn-path
        ([xp] @ xp-path @ [xn] @ xn-path)
    by blast
    let ?chunk = [xp] @ xp-path @ [xn] @ xn-path
    have step:
      fri-layer-step-evidence rt ch len idx pw
        (fri-sibling-index len idx) xp xp-path xn xn-path next-idx
        next-value ?chunk
    using out-eq x'-eq chunk unfolding fri-layer-step-evidence-def by simp
    show ?thesis
      by (rule that[OF step refl])
        (use out-eq xp-eq in simp-all)
  qed

```

lemma *fri-layer-step-evidence-fold-table-if-opening-matches*:
assumes *evidence*:
fri-layer-step-evidence *rt ch len idx pw sibling-idx*
xp xp-path xn xn-path next-idx next-value chunk
and *dom-i*: *fri-dom ! idx = (h ^ idx * shift) ^ pw*
and *match*: *fri-opening-matches-table len idx layer xp xn*
shows *next-value = fri-table-fold-value ch layer fri-dom len pw idx*
proof –
have *next-value-eq*:
next-value =
fri-fold-value ch xp xn
*(fri-fold-denominator ((h ^ idx) * shift) pw)*
using *fri-layer-step-evidenceD(3)[OF evidence]* .
also have ... = *fri-table-fold-value ch layer fri-dom len pw idx*
by (*rule fri-fold-value-eq-table-fold-value[OF dom-i match]*)
finally show ?thesis .
qed

lemma *fri-layer-step-evidence-fold-table-if-matching-chunk*:
assumes *evidence*:
fri-layer-step-evidence *rt ch len idx pw sibling-idx*
xp xp-path xn xn-path next-idx next-value chunk
and *dom-i*: *fri-dom ! idx = (h ^ idx * shift) ^ pw*
and *chunk-match*:
fri-layer-opening-chunk len yp yp-path yn yn-path chunk
and *table-match*: *fri-opening-matches-table len idx layer yp yn*
shows *next-value = fri-table-fold-value ch layer fri-dom len pw idx*
proof –
have *step-chunk*: *fri-layer-opening-chunk len xp xp-path xn xn-path chunk*
by (*rule fri-layer-step-evidenceD(4)[OF evidence]*)
have *xp-eq*: *xp = yp*
by (*rule fri-layer-opening-chunk-values-unique(1)*
[OF step-chunk chunk-match])
have *xn-eq*: *xn = yn*
by (*rule fri-layer-opening-chunk-values-unique(2)*
[OF step-chunk chunk-match])
have *table-match'*:
fri-opening-matches-table len idx layer xp xn
using *table-match xp-eq xn-eq* **by** *simp*
show ?thesis
by (*rule fri-layer-step-evidence-fold-table-if-opening-matches*
[OF evidence dom-i table-match'])
qed

lemma *fri-sampled-table-foldI*:
assumes
fri-layer-opening-chunk len xp xp-path xn xn-path chunk
and *fri-opening-matches-table len idx layer xp xn*

and $\text{fri-dom} \ ! \ \text{idx} = (h \wedge \text{idx} * \text{shift}) \wedge \text{pw}$
and $\text{next-value} = \text{fri-table-fold-value } ch \ \text{layer} \ \text{fri-dom} \ \text{len} \ \text{pw} \ \text{idx}$
shows $\text{fri-sampled-table-fold } ch \ \text{len} \ \text{idx} \ \text{pw} \ \text{fri-dom} \ \text{layer} \ \text{chunk}$
 next-value
using *assms* **unfolding** *fri-sampled-table-fold-def* **by** *blast*

lemma *fri-sampled-table-foldE*:

assumes $\text{fri-sampled-table-fold } ch \ \text{len} \ \text{idx} \ \text{pw} \ \text{fri-dom} \ \text{layer} \ \text{chunk}$
 next-value
obtains $xp \ \text{xp-path} \ \text{xn} \ \text{xn-path}$
where $\text{fri-layer-opening-chunk } \text{len} \ \text{xp} \ \text{xp-path} \ \text{xn} \ \text{xn-path} \ \text{chunk}$
and $\text{fri-opening-matches-table } \text{len} \ \text{idx} \ \text{layer} \ \text{xp} \ \text{xn}$
and $\text{fri-dom} \ ! \ \text{idx} = (h \wedge \text{idx} * \text{shift}) \wedge \text{pw}$
and $\text{next-value} = \text{fri-table-fold-value } ch \ \text{layer} \ \text{fri-dom} \ \text{len} \ \text{pw} \ \text{idx}$
using *assms* **unfolding** *fri-sampled-table-fold-def* **by** *blast*

lemma *fri-layer-step-evidence-sampled-table-fold-if-matching-chunk*:

assumes *evidence*:
 $\text{fri-layer-step-evidence } rt \ ch \ \text{len} \ \text{idx} \ \text{pw} \ \text{sibling-idx}$
 $xp \ \text{xp-path} \ \text{xn} \ \text{xn-path} \ \text{next-idx} \ \text{next-value} \ \text{chunk}$
and $\text{dom-i: } \text{fri-dom} \ ! \ \text{idx} = (h \wedge \text{idx} * \text{shift}) \wedge \text{pw}$
and *chunk-match*:
 $\text{fri-layer-opening-chunk } \text{len} \ \text{yp} \ \text{yp-path} \ \text{yn} \ \text{yn-path} \ \text{chunk}$
and *table-match*: $\text{fri-opening-matches-table } \text{len} \ \text{idx} \ \text{layer} \ \text{yp} \ \text{yn}$
shows $\text{fri-sampled-table-fold } ch \ \text{len} \ \text{idx} \ \text{pw} \ \text{fri-dom} \ \text{layer} \ \text{chunk}$
 next-value

proof –

have *fold-eq*:
 $\text{next-value} = \text{fri-table-fold-value } ch \ \text{layer} \ \text{fri-dom} \ \text{len} \ \text{pw} \ \text{idx}$
by (rule *fri-layer-step-evidence-fold-table-if-matching-chunk*
 $[\text{OF } \text{evidence } \text{dom-i } \text{chunk-match } \text{table-match}]$)
show *?thesis*
by (rule *fri-sampled-table-foldI*
 $[\text{OF } \text{chunk-match } \text{table-match } \text{dom-i } \text{fold-eq}]$)

qed

lemma *fri-layer-opening-matches-bound-table-compactE*:

assumes *match*:
 $\text{fri-layer-opening-matches-bound-table } \text{query-idx} \ \text{layer-idx} \ \text{tables}$
 layer-chunks
and $\text{tables-len: } \text{length } \text{tables} = \text{length } \text{roots}$
obtains $xp \ \text{xp-path} \ \text{xn} \ \text{xn-path}$
where
 $\text{fri-layer-opening-chunk } (\text{fri-evidence-layer-len } \text{roots} \ \text{layer-idx})$
 $xp \ \text{xp-path} \ \text{xn} \ \text{xn-path} \ (\text{layer-chunks} \ ! \ \text{layer-idx})$
and *fri-opening-matches-table*
 $(\text{fri-evidence-layer-len } \text{roots} \ \text{layer-idx})$
 $(\text{fri-layer-indices } (\text{length } \text{roots}) \ \text{query-idx} \ (\text{clength} * \text{scale}) \ !$
 $\text{layer-idx})$

```

      (tables ! layer-idx) xp xn
proof –
  let ?len = fri-evidence-layer-len roots layer-idx
  let ?idx =
    fri-layer-indices (length roots) query-idx (clength * scale) !
    layer-idx
  from match obtain xp xp-path xn xn-path where
    chunk:
      fri-layer-opening-chunk ?len xp xp-path xn xn-path
      (layer-chunks ! layer-idx)
  and table-match:
    fri-opening-matches-table ?len ?idx (tables ! layer-idx) xp xn
  unfolding fri-layer-opening-matches-bound-table-def
  by (auto simp: Let-def tables-len fri-evidence-layer-len-def)
  show ?thesis
  by (rule that[OF chunk table-match])
qed

```

```

lemma fri-all-round-openings-match-tablesD:
  assumes all:
    fri-all-round-openings-match-tables tables query-idxs round-layers
    and round-bound: round-idx < length query-idxs
    and layer-bound: layer-idx < length tables
  shows fri-layer-opening-matches-bound-table
    (query-idxs ! round-idx) layer-idx tables
    (round-layers ! round-idx)
  using assms
  unfolding fri-all-round-openings-match-tables-def
    fri-round-openings-match-tables-def
  by blast

```

```

lemma accepted-fri-opening-transcript-trace-round-layer-evidence:
  assumes fri-openings:
    accepted-fri-opening-transcript s out trace-roots trace-bs trace-final
    dg composition-roots composition-bs composition-final query-idxs
    trace-round-layers composition-round-layers
    and round-bound: round-idx < rounds
    and layer-bound: layer-idx < length trace-roots
  shows fri-round-layer-evidence trace-roots trace-bs query-idxs
    round-idx layer-idx trace-round-layers
proof –
  have shapes:
    length trace-bs = length trace-roots
    length trace-round-layers = rounds
    length query-idxs = rounds
  using accepted-fri-opening-transcript-shapes[OF fri-openings] by blast+
from accepted-fri-opening-transcript-trace-layer-opening
  [OF fri-openings round-bound layer-bound]
obtain root challenge len idx sibling-idx xp xp-path xn xn-path where

```

```

root-eq: root = trace-roots ! layer-idx
and challenge-eq: challenge = trace-bs ! layer-idx
and len-eq:
  len =
    fri-layer-lengths (length trace-roots) (clength * scale) ! layer-idx
and idx-eq:
  idx =
    fri-layer-indices (length trace-roots) (query-idxs ! round-idx)
    (clength * scale) ! layer-idx
and sibling-eq: sibling-idx = fri-sibling-index len idx
and chunk:
  fri-layer-opening-chunk len xp xp-path xn xn-path
  (trace-round-layers ! round-idx ! layer-idx)
by blast
have step:
  fri-layer-step-evidence (trace-roots ! layer-idx)
  (trace-bs ! layer-idx)
  (fri-layer-lengths (length trace-roots) (clength * scale) ! layer-idx)
  (fri-layer-indices (length trace-roots) (query-idxs ! round-idx)
  (clength * scale) ! layer-idx)
  (2 ^ layer-idx)
  (fri-sibling-index
  (fri-layer-lengths (length trace-roots) (clength * scale) ! layer-idx)
  (fri-layer-indices (length trace-roots) (query-idxs ! round-idx)
  (clength * scale) ! layer-idx))
  xp xp-path xn xn-path
  ((fri-layer-indices (length trace-roots) (query-idxs ! round-idx)
  (clength * scale) ! layer-idx) mod
  ((fri-layer-lengths (length trace-roots) (clength * scale) ! layer-idx)
  div 2))
  (fri-fold-value (trace-bs ! layer-idx) xp xn
  (fri-fold-denominator
  ((h ^
  (fri-layer-indices (length trace-roots) (query-idxs ! round-idx)
  (clength * scale) ! layer-idx)) * shift)
  (2 ^ layer-idx)))
  (trace-round-layers ! round-idx ! layer-idx)
using sibling-eq chunk unfolding fri-layer-step-evidence-def
by (simp add: len-eq idx-eq)
show ?thesis
unfolding fri-round-layer-evidence-def
using round-bound layer-bound shapes step
by auto
qed

```

```

lemma accepted-fri-opening-transcript-composition-round-layer-evidence:
  assumes fri-openings:
    accepted-fri-opening-transcript s out trace-roots trace-bs trace-final
    dg composition-roots composition-bs composition-final query-idxs

```

```

    trace-round-layers composition-round-layers
  and round-bound: round-idx < rounds
  and layer-bound: layer-idx < length composition-roots
  shows fri-round-layer-evidence composition-roots composition-bs query-idxs
    round-idx layer-idx composition-round-layers
proof -
  have shapes:
    length composition-bs = length composition-roots
    length composition-round-layers = rounds
    length query-idxs = rounds
  using accepted-fri-opening-transcript-shapes[OF fri-openings] by blast+
  from accepted-fri-opening-transcript-composition-layer-opening
    [OF fri-openings round-bound layer-bound]
  obtain root challenge len idx sibling-idx xp xp-path xn xn-path where
    root-eq: root = composition-roots ! layer-idx
  and challenge-eq: challenge = composition-bs ! layer-idx
  and len-eq:
    len =
      fri-layer-lengths (length composition-roots) (clength * scale) !
        layer-idx
  and idx-eq:
    idx =
      fri-layer-indices (length composition-roots) (query-idxs ! round-idx)
        (clength * scale) ! layer-idx
  and sibling-eq: sibling-idx = fri-sibling-index len idx
  and chunk:
    fri-layer-opening-chunk len xp xp-path xn xn-path
      (composition-round-layers ! round-idx ! layer-idx)
  by blast
  have step:
    fri-layer-step-evidence (composition-roots ! layer-idx)
      (composition-bs ! layer-idx)
      (fri-layer-lengths (length composition-roots) (clength * scale) !
        layer-idx)
      (fri-layer-indices (length composition-roots) (query-idxs ! round-idx)
        (clength * scale) ! layer-idx)
      (2 ^ layer-idx)
      (fri-sibling-index
        (fri-layer-lengths (length composition-roots) (clength * scale) !
          layer-idx)
        (fri-layer-indices (length composition-roots) (query-idxs ! round-idx)
          (clength * scale) ! layer-idx))
    xp xp-path xn xn-path
    ((fri-layer-indices (length composition-roots) (query-idxs ! round-idx)
      (clength * scale) ! layer-idx) mod
      ((fri-layer-lengths (length composition-roots) (clength * scale) !
        layer-idx) div 2))
    (fri-fold-value (composition-bs ! layer-idx) xp xn
      (fri-fold-denominator

```



```

      ((h ^
        (fri-layer-indices (length composition-roots)
          (query-idxs ! round-idx) (clength * scale) ! layer-idx)) *
        shift)
      (2 ^ layer-idx)))
    (composition-round-layers ! round-idx ! layer-idx)
  using sibling-eq chunk unfolding fri-layer-step-evidence-def
  by (simp add: len-eq idx-eq)
show ?thesis
  unfolding fri-round-layer-evidence-def
  using round-bound layer-bound shapes step
  by auto
qed

lemma accepted-fri-opening-transcript-trace-all-round-layer-evidence:
  assumes fri-openings:
    accepted-fri-opening-transcript s out trace-roots trace-bs trace-final
    dg composition-roots composition-bs composition-final query-idxs
    trace-round-layers composition-round-layers
  shows fri-all-round-layer-evidence trace-roots trace-bs query-idxs
    trace-round-layers
proof -
  have shapes:
    length trace-bs = length trace-roots
    length trace-round-layers = rounds
    length query-idxs = rounds
  using accepted-fri-opening-transcript-shapes[OF fri-openings] by blast+
show ?thesis
  unfolding fri-all-round-layer-evidence-def
  using shapes
  by (intro conjI allI impI)
    (auto intro:
      accepted-fri-opening-transcript-trace-round-layer-evidence
      [OF fri-openings])
qed

lemma accepted-fri-opening-transcript-composition-all-round-layer-evidence:
  assumes fri-openings:
    accepted-fri-opening-transcript s out trace-roots trace-bs trace-final
    dg composition-roots composition-bs composition-final query-idxs
    trace-round-layers composition-round-layers
  shows fri-all-round-layer-evidence composition-roots composition-bs
    query-idxs composition-round-layers
proof -
  have shapes:
    length composition-bs = length composition-roots
    length composition-round-layers = rounds
    length query-idxs = rounds
  using accepted-fri-opening-transcript-shapes[OF fri-openings] by blast+

```

```

show ?thesis
unfolding fri-all-round-layer-evidence-def
using shapes
by (intro conjI allI impI)
  (auto intro:
    accepted-fri-opening-transcript-composition-round-layer-evidence
    [OF fri-openings])
qed

lemma ntimes-verifier-query-round-program-final-checks-at:
assumes outcome:
  Some (results, t) ∈
    set-dist
      (execute
        (ntimes
          (verifier-query-round-program fr f-fl f-final as fl final) n)
        s)
and i-bound: i < n
obtains prefix suffix s-i s-suc raw idx fv s0 s1
  f-i f-len f-pow s2 c-i c-len c-pow s4
where
  Some (prefix, s-i) ∈
    set-dist
      (execute
        (ntimes
          (verifier-query-round-program fr f-fl f-final as fl final) i)
        s)
  Some ((), s-suc) ∈
    set-dist
      (execute
        (verifier-query-round-program fr f-fl f-final as fl final) s-i)
  Some (suffix, t) ∈
    set-dist
      (execute
        (ntimes
          (verifier-query-round-program fr f-fl f-final as fl final)
            (n - Suc i))
        s-suc)
  results = prefix @ () # suffix
  Some (raw, s0) ∈ set-dist (execute receive-query-index-challenge s-i)
  idx = index (to-nat raw)
  Some (fv, s1) ∈
    set-dist (execute (mmap (check-decommit-on-query fr idx)) s0)
  Some ((f-i, f-final, f-len, f-pow), s2) ∈
    set-dist
      (execute
        (mfold (idx, hd fv, clength * scale, 1)
          (receive-query-commits f-fl)) s1)
  Some ((c-i, final, c-len, c-pow), s4) ∈

```

```

    set-dist
    (execute
      (mfold (idx, cp-eval as fv (h ^ idx * shift),
        clength * scale, 1)
        (receive-query-commits fl)) s2)
    s-suc = s4
proof –
let ?round = verifier-query-round-program fr f-fl f-final as fl final
from query-opening-ntimes-outcome-decomp-at[OF outcome i-bound]
obtain prefix x suffix s-i s-suc where
  prefix:
    Some (prefix, s-i) ∈ set-dist (execute (ntimes ?round i) s)
and round:
    Some (x, s-suc) ∈ set-dist (execute ?round s-i)
and suffix:
    Some (suffix, t) ∈
      set-dist (execute (ntimes ?round (n – Suc i)) s-suc)
and results-eq: results = prefix @ x # suffix
by blast
have x-eq: x = ()
by (cases x) simp
have round-unit:
    Some ((), s-suc) ∈ set-dist (execute ?round s-i)
using round x-eq by simp
show ?thesis
proof (rule verifier-query-round-program-final-checks[OF round-unit])
fix raw idx fv s0 s1 f-i f-len f-pow s2 c-i c-len c-pow s4
assume raw:
    Some (raw, s0) ∈
      set-dist (execute receive-query-index-challenge s-i)
and idx-eq: idx = index (to-nat raw)
and query-decommit:
    Some (fv, s1) ∈
      set-dist (execute (mmap (check-decommit-on-query fr idx)) s0)
and trace-final:
    Some ((f-i, f-final, f-len, f-pow), s2) ∈
      set-dist
        (execute
          (mfold (idx, hd fv, clength * scale, 1)
            (receive-query-commits f-fl)) s1)
and composition-final:
    Some ((c-i, final, c-len, c-pow), s4) ∈
      set-dist
        (execute
          (mfold (idx, cp-eval as fv (h ^ idx * shift),
            clength * scale, 1)
            (receive-query-commits fl)) s2)
and s-suc-eq: s-suc = s4
have results-unit: results = prefix @ () # suffix

```

```

    using results-eq x-eq by simp
  show ?thesis
    by (rule that[OF prefix round-unit suffix results-unit raw idx-eq
        query-decommit trace-final composition-final s-suc-eq])
qed
qed
end
end
end

```

```

theory Soundness-FRI-Generic-Low-Degree
imports Soundness-FRI-Layer-Evidence
begin

```

Generic FRI low-degree proof interface.

This theory factors the trace/composition-independent bad-challenge-list layer. It deliberately stays at the generic FRI level: no trace-specific or composition-specific candidate predicate is mentioned here.

```

context soundness
begin

```

definition *generic-fri-partial-evidence*

```

:: ('f list  $\Rightarrow$  bool)  $\Rightarrow$  'f list  $\Rightarrow$  nat  $\Rightarrow$ 
   'f list  $\Rightarrow$  'f list  $\Rightarrow$  'f  $\Rightarrow$  nat list  $\Rightarrow$ 
   'f list list list  $\Rightarrow$  bool

```

where

```

generic-fri-partial-evidence low-degree candidate-table degree-bound
  roots challenges final-value query-idxs round-layers  $\longleftrightarrow$ 
  length roots = fri-round-count-for-degree-bound degree-bound  $\wedge$ 
  length challenges = length roots  $\wedge$ 
  fri-all-round-layer-evidence roots challenges query-idxs round-layers

```

definition *generic-fri-bad-challenge-lists*

```

:: nat  $\Rightarrow$  (nat  $\Rightarrow$  'f list  $\Rightarrow$  'f set)  $\Rightarrow$  'f list set

```

where

```

generic-fri-bad-challenge-lists n bad =
  fri-multiround-bad-challenge-lists n bad

```

definition *generic-fri-round-bad-challenge-lists*

```

:: nat  $\Rightarrow$  (nat  $\Rightarrow$  'f list  $\Rightarrow$  'f set)  $\Rightarrow$  nat  $\Rightarrow$ 
   'f list set

```

where

```

generic-fri-round-bad-challenge-lists n bad i =
  {bs  $\in$  fri-challenge-space n. i < n  $\wedge$  bs ! i  $\in$  bad i (take i bs)}

```

definition *generic-fri-round-bad-witnesses*

```

:: nat  $\Rightarrow$  (nat  $\Rightarrow$  'f list  $\Rightarrow$  'f set)  $\Rightarrow$  nat  $\Rightarrow$ 

```

$(\text{'f list} \times \text{'f} \times \text{'f list}) \text{ set}$
where
 $\text{generic-fri-round-bad-witnesses } n \text{ bad } i =$
 $\{(prefix, b, suffix). \text{ } i < n \wedge$
 $\text{length } prefix = i \wedge$
 $b \in \text{bad } i \text{ prefix} \wedge$
 $\text{length } suffix = n - \text{Suc } i\}$

definition *generic-fri-bad-candidate*
 $:: (\text{'f list} \Rightarrow \text{bool}) \Rightarrow \text{'f list} \Rightarrow \text{nat} \Rightarrow$
 $\text{'f list} \Rightarrow \text{'f list} \Rightarrow \text{'f} \Rightarrow \text{nat list} \Rightarrow$
 $\text{'f list list list} \Rightarrow$
 $(\text{nat} \Rightarrow \text{'f list} \Rightarrow \text{'f set}) \Rightarrow \text{bool}$

where
 $\text{generic-fri-bad-candidate low-degree candidate-table degree-bound}$
 $\text{roots challenges final-value query-idxs round-layers bad} \longleftrightarrow$
 $\text{generic-fri-partial-evidence low-degree candidate-table degree-bound}$
 $\text{roots challenges final-value query-idxs round-layers} \wedge$
 $\neg \text{low-degree candidate-table} \wedge$
 $\text{challenges} \in \text{generic-fri-bad-challenge-lists (length challenges) bad}$

definition *generic-fri-proximity-evidence*
 $:: \text{nat} \Rightarrow \text{nat} \Rightarrow \text{nat list} \Rightarrow \text{nat list} \Rightarrow$
 $\text{'f list list} \Rightarrow \text{'f list list} \Rightarrow \text{'f list} \Rightarrow$
 $(\text{nat} \Rightarrow \text{'f list} \Rightarrow \text{'f set}) \Rightarrow \text{bool}$

where
 $\text{generic-fri-proximity-evidence } n \text{ d lens pws doms layers challenges bad}$
 $\longleftrightarrow$
 $\text{fri-multiround-proximity-steps } n \text{ d lens pws doms layers bad} \wedge$
 $\text{challenges} \in \text{fri-challenge-space } n \wedge$
 $(\forall i < n. \text{ } \text{fri-algebraic-layer-relation (lens ! } i) \text{ (pws ! } i)$
 $\text{(challenges ! } i) \text{ (doms ! } i) \text{ (layers ! } i)$
 $\text{(doms ! Suc } i) \text{ (layers ! Suc } i)) \wedge$
 $\neg \text{fri-table-low-degree-on } d \text{ (doms ! } 0) \text{ (layers ! } 0) \wedge$
 $\text{fri-table-low-degree-on (fri-degree-after } n \text{ d) (doms ! } n)$
 $\text{(layers ! } n)$

definition *generic-fri-canonical-layer-chain-evidence*
 $:: (\text{'f list} \Rightarrow \text{bool}) \Rightarrow \text{'f list} \Rightarrow \text{nat} \Rightarrow$
 $\text{'f list} \Rightarrow \text{'f list} \Rightarrow \text{'f} \Rightarrow \text{nat list} \Rightarrow$
 $\text{'f list list list} \Rightarrow \text{'f list list} \Rightarrow \text{'f list list} \Rightarrow$
 $(\text{nat} \Rightarrow \text{'f list} \Rightarrow \text{'f set}) \Rightarrow \text{bool}$

where
 $\text{generic-fri-canonical-layer-chain-evidence low-degree candidate-table}$
 $\text{degree-bound roots challenges final-value query-idxs round-layers}$
 $\text{doms layers bad} \longleftrightarrow$
 $\text{generic-fri-partial-evidence low-degree candidate-table degree-bound}$

$roots\ challenges\ final\text{-}value\ query\text{-}idxs\ round\text{-}layers \wedge$
 $doms ! 0 = eval\text{-}domain \wedge$
 $layers ! 0 = candidate\text{-}table \wedge$
 $fri\text{-}final\text{-}constant\text{-}consistent\ (layers ! length\ challenges)\ final\text{-}value \wedge$
 $generic\text{-}fri\text{-}proximity\text{-}evidence\ (length\ challenges)\ degree\text{-}bound$
 $(fri\text{-}layer\text{-}lengths\ (length\ challenges)\ (clength * scale))$
 $(map\ (\lambda i. 2 \wedge i)\ [0..<length\ challenges])$
 $doms\ layers\ challenges\ bad$

definition *generic-fri-sampled-layer-chain-evidence*

$:: ('f\ list \Rightarrow bool) \Rightarrow 'f\ list \Rightarrow nat \Rightarrow$
 $'f\ list \Rightarrow 'f\ list \Rightarrow 'f \Rightarrow nat\ list \Rightarrow$
 $'f\ list\ list\ list \Rightarrow 'f\ list\ list \Rightarrow 'f\ list\ list \Rightarrow bool$

where

$generic\text{-}fri\text{-}sampled\text{-}layer\text{-}chain\text{-}evidence\ low\text{-}degree\ candidate\text{-}table$
 $degree\text{-}bound\ roots\ challenges\ final\text{-}value\ query\text{-}idxs\ round\text{-}layers$
 $doms\ layers \longleftrightarrow$
 $generic\text{-}fri\text{-}partial\text{-}evidence\ low\text{-}degree\ candidate\text{-}table\ degree\text{-}bound$
 $roots\ challenges\ final\text{-}value\ query\text{-}idxs\ round\text{-}layers \wedge$
 $length\ doms = Suc\ (length\ challenges) \wedge$
 $length\ layers = Suc\ (length\ challenges) \wedge$
 $doms ! 0 = eval\text{-}domain \wedge$
 $layers ! 0 = candidate\text{-}table \wedge$
 $fri\text{-}final\text{-}constant\text{-}consistent\ (layers ! length\ challenges)\ final\text{-}value \wedge$
 $(\forall\ round\text{-}idx < length\ query\text{-}idxs.$
 $\forall\ layer\text{-}idx < length\ challenges.$
 $fri\text{-}evidence\text{-}next\text{-}idx\ roots\ query\text{-}idxs\ round\text{-}idx\ layer\text{-}idx <$
 $length\ (layers ! Suc\ layer\text{-}idx) \wedge$
 $fri\text{-}sampled\text{-}table\text{-}fold$
 $(challenges ! layer\text{-}idx)$
 $(fri\text{-}evidence\text{-}layer\text{-}len\ roots\ layer\text{-}idx)$
 $(fri\text{-}evidence\text{-}layer\text{-}idx\ roots\ query\text{-}idxs\ round\text{-}idx\ layer\text{-}idx)$
 $(2 \wedge layer\text{-}idx)$
 $(doms ! layer\text{-}idx)$
 $(layers ! layer\text{-}idx)$
 $(round\text{-}layers ! round\text{-}idx ! layer\text{-}idx)$
 $(layers ! Suc\ layer\text{-}idx !$
 $fri\text{-}evidence\text{-}next\text{-}idx\ roots\ query\text{-}idxs\ round\text{-}idx\ layer\text{-}idx))$

definition *fri-sampled-domains-align*

$:: 'f\ list \Rightarrow nat\ list \Rightarrow 'f\ list\ list \Rightarrow bool$

where

$fri\text{-}sampled\text{-}domains\text{-}align\ roots\ query\text{-}idxs\ doms \longleftrightarrow$
 $(\forall\ round\text{-}idx < length\ query\text{-}idxs.$
 $\forall\ layer\text{-}idx < length\ roots.$
 $doms ! layer\text{-}idx !$
 $fri\text{-}evidence\text{-}layer\text{-}idx\ roots\ query\text{-}idxs\ round\text{-}idx\ layer\text{-}idx =$
 $(h \wedge fri\text{-}evidence\text{-}layer\text{-}idx\ roots\ query\text{-}idxs\ round\text{-}idx\ layer\text{-}idx *$
 $shift) \wedge (2 \wedge layer\text{-}idx))$

definition *fri-sampled-next-domains-align*

$:: 'f \text{ list} \Rightarrow \text{nat list} \Rightarrow 'f \text{ list list} \Rightarrow \text{bool}$

where

fri-sampled-next-domains-align roots query-idxs doms $\longleftrightarrow$
 $(\forall \text{round-idx} < \text{length query-idxs}.$
 $\quad \forall \text{layer-idx} < \text{length roots}.$
 $\quad \text{doms ! layer-idx !}$
 $\quad \text{fri-evidence-next-idx roots query-idxs round-idx layer-idx} =$
 $\quad (h \wedge \text{fri-evidence-next-idx roots query-idxs round-idx layer-idx} *$
 $\quad \text{shift}) \wedge (2 \wedge \text{layer-idx}))$

definition *fri-sampled-indices-in-fold-range*

$:: 'f \text{ list} \Rightarrow \text{nat list} \Rightarrow \text{bool}$

where

fri-sampled-indices-in-fold-range roots query-idxs $\longleftrightarrow$
 $(\forall \text{round-idx} < \text{length query-idxs}.$
 $\quad \forall \text{layer-idx} < \text{length roots}.$
 $\quad \text{fri-evidence-next-idx roots query-idxs round-idx layer-idx} <$
 $\quad \text{fri-evidence-layer-len roots layer-idx div } 2)$

definition *fri-sampled-layer-covers-fold-indices*

$:: 'f \text{ list} \Rightarrow \text{nat list} \Rightarrow \text{nat} \Rightarrow \text{bool}$

where

fri-sampled-layer-covers-fold-indices roots query-idxs layer-idx
 $\longleftrightarrow$
 $(\forall \text{idx} < \text{fri-evidence-layer-len roots layer-idx div } 2.$
 $\quad \exists \text{round-idx} < \text{length query-idxs}.$
 $\quad \text{fri-evidence-next-idx roots query-idxs round-idx layer-idx} = \text{idx})$

definition *fri-sampled-layers-cover-fold-indices*

$:: 'f \text{ list} \Rightarrow \text{nat list} \Rightarrow \text{bool}$

where

fri-sampled-layers-cover-fold-indices roots query-idxs $\longleftrightarrow$
 $(\forall \text{layer-idx} < \text{length roots}.$
 $\quad \text{fri-sampled-layer-covers-fold-indices roots query-idxs layer-idx})$

definition *generic-fri-sampled-layer-chain-full-cover*

$:: ('f \text{ list} \Rightarrow \text{bool}) \Rightarrow 'f \text{ list} \Rightarrow \text{nat} \Rightarrow$
 $\quad 'f \text{ list} \Rightarrow 'f \text{ list} \Rightarrow 'f \Rightarrow \text{nat list} \Rightarrow$
 $\quad 'f \text{ list list list} \Rightarrow 'f \text{ list list} \Rightarrow 'f \text{ list list} \Rightarrow$
 $\quad (\text{nat} \Rightarrow 'f \text{ list} \Rightarrow 'f \text{ set}) \Rightarrow \text{bool}$

where

generic-fri-sampled-layer-chain-full-cover low-degree candidate-table
degree-bound roots challenges final-value query-idxs round-layers
doms layers bad $\longleftrightarrow$
generic-fri-sampled-layer-chain-evidence low-degree candidate-table
degree-bound roots challenges final-value query-idxs round-layers
doms layers $\wedge$

fri-multiround-proximity-steps (length challenges) degree-bound
 (fri-layer-lengths (length challenges) (clength * scale))
 (map ($\lambda i. 2 \wedge i$) [0..
 doms layers bad $\wedge$
fri-sampled-layers-cover-fold-indices roots query-idxs $\wedge$
 ($\forall$ layer-idx < length challenges.
 2 dvd fri-evidence-layer-len roots layer-idx) $\wedge$
 ($\forall$ layer-idx < length challenges.
 fri-evidence-layer-len roots layer-idx * $2 \wedge$ layer-idx =
 clength * scale) $\wedge$
 ($\forall$ layer-idx < length challenges.
 fri-evidence-layer-len roots layer-idx $\leq$
 length (doms ! layer-idx)) $\wedge$
 ($\forall$ layer-idx < length challenges.
 fri-evidence-layer-len roots layer-idx $\leq$
 length (layers ! layer-idx)) $\wedge$
 ($\forall$ layer-idx < length challenges.
 fri-evidence-layer-len roots layer-idx div 2 $\leq$
 length (doms ! Suc layer-idx)) $\wedge$
 ($\forall$ layer-idx < length challenges.
 fri-evidence-layer-len roots layer-idx div 2 $\leq$
 length (layers ! Suc layer-idx)) $\wedge$
 ($\forall$ layer-idx < length challenges.
 $\forall$ idx < fri-evidence-layer-len roots layer-idx div 2.
 doms ! Suc layer-idx ! idx =
 (doms ! layer-idx ! idx)²) $\wedge$
 ($\forall$ layer-idx < length challenges.
 $\forall$ idx < fri-evidence-layer-len roots layer-idx div 2.
 doms ! layer-idx ! idx = ($h \wedge$ idx * shift) $\wedge$ ($2 \wedge$ layer-idx)) $\wedge$
 $\neg$ fri-table-low-degree-on degree-bound eval-domain candidate-table $\wedge$
fri-table-low-degree-on
 (fri-degree-after (length challenges) degree-bound)
 (doms ! length challenges) (layers ! length challenges)

lemma generic-fri-partial-evidence-shapes:

assumes generic-fri-partial-evidence low-degree candidate-table degree-bound
 roots challenges final-value query-idxs round-layers
shows length roots = fri-round-count-for-degree-bound degree-bound
and length challenges = length roots
and fri-all-round-layer-evidence roots challenges query-idxs round-layers
using assms **unfolding** generic-fri-partial-evidence-def **by** simp-all

lemma generic-fri-sampled-layer-chain-evidenceD:

assumes
 generic-fri-sampled-layer-chain-evidence low-degree candidate-table
 degree-bound roots challenges final-value query-idxs round-layers
 doms layers
shows
 generic-fri-partial-evidence low-degree candidate-table degree-bound

roots challenges final-value query-idxs round-layers
length doms = Suc (length challenges)
length layers = Suc (length challenges)
doms ! 0 = eval-domain
layers ! 0 = candidate-table
fri-final-constant-consistent (layers ! length challenges) final-value
using *assms*
unfolding *generic-fri-sampled-layer-chain-evidence-def* **by** *simp-all*

lemma *generic-fri-sampled-layer-chain-evidence-sample:*

assumes *chain:*
generic-fri-sampled-layer-chain-evidence low-degree candidate-table
degree-bound roots challenges final-value query-idxs round-layers
doms layers
and *round-bound: round-idx < length query-idxs*
and *layer-bound: layer-idx < length challenges*
shows
fri-evidence-next-idx roots query-idxs round-idx layer-idx <
length (layers ! Suc layer-idx)
fri-sampled-table-fold
(challenges ! layer-idx)
(fri-evidence-layer-len roots layer-idx)
(fri-evidence-layer-idx roots query-idxs round-idx layer-idx)
(2 ^ layer-idx)
(doms ! layer-idx)
(layers ! layer-idx)
(round-layers ! round-idx ! layer-idx)
(layers ! Suc layer-idx !
fri-evidence-next-idx roots query-idxs round-idx layer-idx)
using *chain round-bound layer-bound*
unfolding *generic-fri-sampled-layer-chain-evidence-def* **by** *simp-all*

lemma *fri-sampled-domains-alignD:*

assumes *fri-sampled-domains-align roots query-idxs doms*
and *round-idx < length query-idxs*
and *layer-idx < length roots*
shows
doms ! layer-idx !
fri-evidence-layer-idx roots query-idxs round-idx layer-idx =
*(h ^ fri-evidence-layer-idx roots query-idxs round-idx layer-idx **
shift) ^ (2 ^ layer-idx)
using *assms* **unfolding** *fri-sampled-domains-align-def* **by** *blast*

lemma *fri-sampled-next-domains-alignD:*

assumes *fri-sampled-next-domains-align roots query-idxs doms*
and *round-idx < length query-idxs*
and *layer-idx < length roots*
shows
doms ! layer-idx !

$\text{fri-evidence-next-idx roots query-idxs round-idx layer-idx} =$
 $(h \wedge \text{fri-evidence-next-idx roots query-idxs round-idx layer-idx} * \text{shift}) \wedge (2 \wedge \text{layer-idx})$
using *assms* **unfolding** *fri-sampled-next-domains-align-def* **by** *blast*

lemma *fri-sampled-indices-in-fold-rangeD*:
assumes *fri-sampled-indices-in-fold-range roots query-idxs*
and *round-idx < length query-idxs*
and *layer-idx < length roots*
shows
 $\text{fri-evidence-next-idx roots query-idxs round-idx layer-idx} <$
 $\text{fri-evidence-layer-len roots layer-idx div 2}$
using *assms* **unfolding** *fri-sampled-indices-in-fold-range-def* **by** *blast*

lemma *fri-sampled-layers-cover-fold-indicesD*:
assumes *fri-sampled-layers-cover-fold-indices roots query-idxs*
and *layer-idx < length roots*
and *idx < fri-evidence-layer-len roots layer-idx div 2*
obtains *round-idx*
where *round-idx < length query-idxs*
and $\text{fri-evidence-next-idx roots query-idxs round-idx layer-idx} = \text{idx}$
using *assms*
unfolding *fri-sampled-layers-cover-fold-indices-def*
fri-sampled-layer-covers-fold-indices-def
by *blast*

lemma *fri-sampled-layer-covers-fold-indices-card-le-rounds*:
assumes *cover: fri-sampled-layer-covers-fold-indices roots query-idxs layer-idx*
shows $\text{fri-evidence-layer-len roots layer-idx div 2} \leq \text{length query-idxs}$

proof –
let $?N = \text{fri-evidence-layer-len roots layer-idx div 2}$
have *subset*:
 $\{..< ?N\} \subseteq$
 $(\lambda \text{round-idx}.$
 $\text{fri-evidence-next-idx roots query-idxs round-idx layer-idx})$ ‘
 $\{..< \text{length query-idxs}\}$

proof
fix *idx*
assume $\text{idx} \in \{..< ?N\}$
then have *idx-bound*: $\text{idx} < ?N$
by *simp*
then obtain *round-idx* **where** *round-bound*: $\text{round-idx} < \text{length query-idxs}$
and *idx-eq*:
 $\text{fri-evidence-next-idx roots query-idxs round-idx layer-idx} = \text{idx}$
using *cover*
unfolding *fri-sampled-layer-covers-fold-indices-def* **by** *blast*
then show $\text{idx} \in$
 $(\lambda \text{round-idx}.$
 $\text{fri-evidence-next-idx roots query-idxs round-idx layer-idx})$ ‘

```

      {.. $\text{length query-idxs}$ }
    by blast
  qed
  have ?N = card {.. $\text{length query-idxs}$ }
  by simp
  also have ...  $\leq$ 
    card (( $\lambda \text{round-idx}.$ 
      fri-evidence-next-idx roots query-idxs round-idx layer-idx) ‘
      {.. $\text{length query-idxs}$ })
  by (rule card-mono) (simp-all add: subset)
  also have ...  $\leq$  card {.. $\text{length query-idxs}$ }
  by (rule card-image-le) simp
  also have ... =  $\text{length query-idxs}$ 
  by simp
  finally show ?thesis .
qed

lemma fri-sampled-layers-cover-fold-indices-card-le-rounds:
  assumes cover: fri-sampled-layers-cover-fold-indices roots query-idxs
    and layer-bound: layer-idx <  $\text{length roots}$ 
  shows fri-evidence-layer-len roots layer-idx div 2  $\leq$   $\text{length query-idxs}$ 
proof -
  have layer-cover:
    fri-sampled-layer-covers-fold-indices roots query-idxs layer-idx
  using cover layer-bound
  unfolding fri-sampled-layers-cover-fold-indices-def by blast
  show ?thesis
  by (rule fri-sampled-layer-covers-fold-indices-card-le-rounds
    [OF layer-cover])
qed

lemma fri-sampled-layers-not-cover-if-layer-exceeds-rounds:
  assumes layer-bound: layer-idx <  $\text{length roots}$ 
    and too-few:
       $\text{length query-idxs}$  < fri-evidence-layer-len roots layer-idx div 2
  shows  $\neg$  fri-sampled-layers-cover-fold-indices roots query-idxs
proof
  assume cover: fri-sampled-layers-cover-fold-indices roots query-idxs
  have fri-evidence-layer-len roots layer-idx div 2  $\leq$   $\text{length query-idxs}$ 
  by (rule fri-sampled-layers-cover-fold-indices-card-le-rounds
    [OF cover layer-bound])
  then show False
  using too-few by simp
qed

lemma generic-fri-proximity-evidenceD:
  assumes generic-fri-proximity-evidence n d lens pws doms layers
    challenges bad
  shows

```

fri-multiround-proximity-steps n d lens pws doms layers bad
challenges $\in$ *fri-challenge-space* n
 $\neg$ *fri-table-low-degree-on* d (*doms* ! 0) (*layers* ! 0)
fri-table-low-degree-on (*fri-degree-after* n d) (*doms* ! n)
(*layers* ! n)
using *assms* **unfolding** *generic-fri-proximity-evidence-def* **by** *simp-all*

lemma *generic-fri-proximity-evidence-relation*:
assumes *proximity*:
generic-fri-proximity-evidence n d lens pws doms layers
challenges bad
and *i-bound*: $i < n$
shows
fri-algebraic-layer-relation (lens ! i) (pws ! i)
(*challenges* ! i) (*doms* ! i) (*layers* ! i)
(*doms* ! *Suc* i) (*layers* ! *Suc* i)
using *proximity* *i-bound*
unfolding *generic-fri-proximity-evidence-def* **by** *blast*

lemma *generic-fri-canonical-layer-chain-proximity*:
assumes *chain*:
generic-fri-canonical-layer-chain-evidence *low-degree* *candidate-table*
degree-bound *roots* *challenges* *final-value* *query-idxs* *round-layers*
doms layers bad
shows
generic-fri-proximity-evidence (*length* *challenges*) *degree-bound*
(*fri-layer-lengths* (*length* *challenges*) (*clength* * *scale*))
(*map* ($\lambda i. 2^i$) [$0..<\text{length } \text{challenges}$])
doms layers *challenges* bad
using *chain* **unfolding** *generic-fri-canonical-layer-chain-evidence-def*
by *simp*

lemma *generic-fri-canonical-layer-chain-relation*:
assumes *chain*:
generic-fri-canonical-layer-chain-evidence *low-degree* *candidate-table*
degree-bound *roots* *challenges* *final-value* *query-idxs* *round-layers*
doms layers bad
and *i-bound*: $i < \text{length } \text{challenges}$
shows
fri-algebraic-layer-relation
(*fri-layer-lengths* (*length* *challenges*) (*clength* * *scale*) ! i)
(2^i)
(*challenges* ! i) (*doms* ! i) (*layers* ! i)
(*doms* ! *Suc* i) (*layers* ! *Suc* i)

proof –
have *proximity*:
generic-fri-proximity-evidence (*length* *challenges*) *degree-bound*
(*fri-layer-lengths* (*length* *challenges*) (*clength* * *scale*))
(*map* ($\lambda i. 2^i$) [$0..<\text{length } \text{challenges}$])

```

    doms layers challenges bad
  by (rule generic-fri-canonical-layer-chain-proximity[OF chain])
have rel:
  fri-algebraic-layer-relation
    (fri-layer-lengths (length challenges) (clength * scale) ! i)
    ((map ( $\lambda i. 2 \wedge i$ ) [0.. $\text{length challenges}$ ]) ! i)
    (challenges ! i) (doms ! i) (layers ! i)
    (doms ! Suc i) (layers ! Suc i)
  by (rule generic-fri-proximity-evidence-relation
      [OF proximity i-bound])
then show ?thesis
  using i-bound by simp
qed

```

lemma *generic-fri-canonical-layer-chain-lengths*:

assumes *chain*:

generic-fri-canonical-layer-chain-evidence low-degree candidate-table
degree-bound roots challenges final-value query-idxs round-layers
doms layers bad

shows $\text{length doms} = \text{Suc } (\text{length challenges})$
and $\text{length layers} = \text{Suc } (\text{length challenges})$

proof –

have *proximity*:

generic-fri-proximity-evidence (length challenges) degree-bound
*(fri-layer-lengths (length challenges) (clength * scale))*
(map ($\lambda i. 2 \wedge i$) [0.. length challenges])
doms layers challenges bad

by (rule generic-fri-canonical-layer-chain-proximity[OF chain])

have *steps*:

fri-multiround-proximity-steps (length challenges) degree-bound
*(fri-layer-lengths (length challenges) (clength * scale))*
(map ($\lambda i. 2 \wedge i$) [0.. length challenges])
doms layers bad

by (rule generic-fri-proximity-evidenceD(1)[OF proximity])

then show $\text{length doms} = \text{Suc } (\text{length challenges})$
and $\text{length layers} = \text{Suc } (\text{length challenges})$
unfolding *fri-multiround-proximity-steps-def* **by** *simp-all*

qed

lemma *fri-algebraic-layer-relation-next-index-bound*:

assumes *relation*:

fri-algebraic-layer-relation len pw ch fri-dom layer next-dom
next-layer

and *idx-bound*: $\text{idx} < \text{len div } 2$

shows $\text{idx} < \text{length next-layer}$

proof –

have $\text{len div } 2 \leq \text{length next-layer}$

using *relation* **unfolding** *fri-algebraic-layer-relation-def* **by** *simp*

then show ?thesis

```

    using idx-bound by linarith
qed

lemma fri-algebraic-layer-relation-sampled-fold-eq:
  assumes relation:
    fri-algebraic-layer-relation len pw ch fri-dom layer next-dom
    next-layer
  and idx-bound: idx < len div 2
  shows next-layer ! idx =
    fri-table-fold-value ch layer fri-dom len pw idx
  by (rule fri-algebraic-layer-relationD(2)[OF relation idx-bound])

lemma fri-half-dvd-len-if-even:
  fixes len :: nat
  assumes even-len: 2 dvd len
  shows len div 2 dvd len
proof (cases len = 0)
  case True
  then show ?thesis by simp
next
  case False
  then have len = 2 * (len div 2)
    using even-len by simp
  then have len = (len div 2) * 2
    by simp
  then show ?thesis
    unfolding dvd-def by blast
qed

lemma fri-sibling-index-mod-half:
  fixes len idx :: nat
  assumes even-len: 2 dvd len
  shows fri-sibling-index len idx mod (len div 2) =
    idx mod (len div 2)
proof -
  have dvd-half: len div 2 dvd len
    by (rule fri-half-dvd-len-if-even[OF even-len])
  have fri-sibling-index len idx mod (len div 2) =
    ((idx + len div 2) mod len) mod (len div 2)
    unfolding fri-sibling-index-def by simp
  also have ... = (idx + len div 2) mod (len div 2)
    using mod-mod-cancel[OF dvd-half, of idx + len div 2] by simp
  also have ... = idx mod (len div 2)
    by simp
  finally show ?thesis .
qed

lemma fri-fold-value-swap-neg-denominator:
  fri-fold-value b xp xn (- denom) =

```

```

    fri-fold-value b xn xp denom
  proof (cases denom = 0)
  case True
  then show ?thesis
    unfolding fri-fold-value-def by (simp add: algebra-simps)
  next
  case False
  then show ?thesis
    unfolding fri-fold-value-def by (simp add: field-simps)
  qed

```

```

lemma soundness-h-power-mod-eval-domain:
   $h \wedge (k \bmod (\text{clength} * \text{scale})) = h \wedge k$ 
proof -
  let ?N = clength * scale
  have order:  $h \wedge ?N = 1$ 
    using h-order .
  have k-eq:  $k = (k \text{ div } ?N) * ?N + k \bmod ?N$ 
    using div-mult-mod-eq[of k ?N] by (simp add: mult.commute)
  have period:  $h \wedge ((k \text{ div } ?N) * ?N) = 1$ 
proof -
  have  $h \wedge ((k \text{ div } ?N) * ?N) = h \wedge (?N * (k \text{ div } ?N))$ 
    by (simp add: mult.commute)
  also have ... =  $(h \wedge ?N) \wedge (k \text{ div } ?N)$ 
    by (simp only: power-mult)
  also have ... = 1
    using order by simp
  finally show ?thesis .
qed
have  $h \wedge k = h \wedge ((k \text{ div } ?N) * ?N + k \bmod ?N)$ 
  using k-eq by simp
also have ... =  $h \wedge ((k \text{ div } ?N) * ?N) * h \wedge (k \bmod ?N)$ 
  by (simp add: power-add)
also have ... =  $h \wedge (k \bmod ?N)$ 
  using period by simp
finally show ?thesis
  by simp
qed

```

```

lemma soundness-h-half-shift:
   $h \wedge (k + (\text{clength} * \text{scale}) \text{ div } 2) = -(h \wedge k)$ 
  using h-half-order
  by (simp add: power-add algebra-simps)

```

```

lemma soundness-mod-mult-right-factor-nat:
  fixes a b c :: nat
  assumes b-pos:  $0 < b$ 
    and c-pos:  $0 < c$ 
  shows  $((a \bmod b) * c) \bmod (b * c) = (a * c) \bmod (b * c)$ 

```

```

proof -
  have decomp:  $a = a \text{ div } b * b + a \text{ mod } b$ 
    using div-mult-mod-eq[of a b] by simp
  have r-less:  $(a \text{ mod } b) * c < b * c$ 
    using b-pos c-pos mod-less-divisor[OF b-pos]
    by (simp add: mult-less-mono2)
  have r-le:  $(a \text{ mod } b) * c \leq a * c$ 
proof -
  have  $a \text{ mod } b \leq a$ 
    using decomp by linarith
  then show ?thesis
    by simp
qed
have dvd-diff:  $b * c \text{ dvd } a * c - (a \text{ mod } b) * c$ 
proof -
  have ac-decomp:  $a * c = (a \text{ div } b) * (b * c) + (a \text{ mod } b) * c$ 
proof -
  have  $a * c = (a \text{ div } b * b + a \text{ mod } b) * c$ 
    using decomp by simp
  also have  $\dots = (a \text{ div } b * b) * c + (a \text{ mod } b) * c$ 
    by (simp only: distrib-right)
  also have  $\dots = (a \text{ div } b) * (b * c) + (a \text{ mod } b) * c$ 
    by (simp add: algebra-simps)
  finally show ?thesis .
qed
have  $a * c - (a \text{ mod } b) * c = (a \text{ div } b) * (b * c)$ 
  using ac-decomp by simp
  then show ?thesis by simp
qed
have  $(a * c) \text{ mod } (b * c) = (a \text{ mod } b) * c$ 
  by (rule mod-nat-eqI[OF r-less r-le dvd-diff])
  then show ?thesis by simp
qed

lemma fri-layer-lengths-nth-div:
  assumes j-bound:  $j < n$ 
  shows fri-layer-lengths  $n \text{ len } ! j = \text{len div } 2 ^ j$ 
  using j-bound
proof (induction n arbitrary: len j)
  case 0
  then show ?case by simp
next
  case (Suc n)
  then show ?case
proof (cases j)
  case 0
  then show ?thesis by simp
next
  case (Suc k)

```



```

then have k-bound:  $k < n$ 
  using Suc.prem by simp
have fri-layer-lengths (Suc n) len ! j =
  fri-layer-lengths n (len div 2) ! k
  using Suc by simp
also have ... = (len div 2) div  $2^k$ 
  by (rule Suc.IH[OF k-bound])
also have ... = len div  $2^j$ 
  unfolding Suc by (simp add: div-mult2-eq mult.commute)
finally show ?thesis .
qed
qed

```

```

lemma fri-layer-lengths-round-product-if-dvd:
  assumes j-bound:  $j < n$ 
  and dvd:  $2^j \text{ dvd } len$ 
  shows fri-layer-lengths n len !  $j * 2^j = len$ 
proof -
  have fri-layer-lengths n len !  $j = len \text{ div } 2^j$ 
    by (rule fri-layer-lengths-nth-div[OF j-bound])
  then show ?thesis
    using dvd by (simp add: dvd-div-mult-self)
qed

```

```

lemma fri-layer-lengths-even-if-dvd:
  assumes j-bound:  $j < n$ 
  and dvd:  $2^{\text{Suc } j} \text{ dvd } len$ 
  shows  $2 \text{ dvd } fri\text{-layer-lengths } n \text{ len ! } j$ 
proof -
  obtain k where len-eq:  $len = 2^{\text{Suc } j} * k$ 
    using dvd by blast
  have fri-layer-lengths n len !  $j = len \text{ div } 2^j$ 
    by (rule fri-layer-lengths-nth-div[OF j-bound])
  also have ... =  $2 * k$ 
    using len-eq by simp
  finally show ?thesis by simp
qed

```

```

lemma fri-layer-lengths-Suc-nth:
  assumes Suc j < n
  shows fri-layer-lengths n len !  $\text{Suc } j =$ 
    fri-layer-lengths n len !  $j \text{ div } 2$ 
proof -
  have j-bound:  $j < n$ 
    using assms by simp
  have fri-layer-lengths n len !  $\text{Suc } j = len \text{ div } 2^{\text{Suc } j}$ 
    by (rule fri-layer-lengths-nth-div[OF assms])
  also have ... = (len div  $2^j$ ) div 2
    by (metis div-mult2-eq mult.commute power-Suc)

```

```

also have ... = fri-layer-lengths n len ! j div 2
  using fri-layer-lengths-nth-div[OF j-bound, of len] by simp
finally show ?thesis .
qed

lemma generic-fri-canonical-domain-power:
  assumes chain:
    generic-fri-canonical-layer-chain-evidence low-degree candidate-table
    degree-bound roots challenges final-value query-idxs round-layers
    doms layers bad
  and roots-len: length challenges = length roots
  and layer-bound: layer-idx < length challenges
  and idx-bound: idx < fri-evidence-layer-len roots layer-idx
  shows doms ! layer-idx ! idx = (h ^ idx * shift) ^ (2 ^ layer-idx)
  using layer-bound idx-bound
proof (induction layer-idx arbitrary: idx)
  case 0
  note zero = 0
  have dom0: doms ! 0 = eval-domain
    using chain unfolding generic-fri-canonical-layer-chain-evidence-def
    by simp
  have len0: fri-evidence-layer-len roots 0 = clength * scale
  proof (cases roots)
    case Nil
    then show ?thesis
      using zero roots-len unfolding fri-evidence-layer-len-def by simp
  next
    case (Cons r rs)
    then show ?thesis
      unfolding fri-evidence-layer-len-def by simp
  qed
  show ?case
    using eval-domain-nth[of idx] zero len0
    unfolding dom0 by simp
next
  case (Suc j)
  have j-bound: j < length challenges
    using Suc.premis by simp
  have len-eq-j:
    fri-layer-lengths (length challenges) (clength * scale) ! j =
    fri-evidence-layer-len roots j
  using roots-len unfolding fri-evidence-layer-len-def by simp
  have len-eq-suc:
    fri-layer-lengths (length challenges) (clength * scale) ! Suc j =
    fri-evidence-layer-len roots (Suc j)
  using roots-len unfolding fri-evidence-layer-len-def by simp
  have suc-len:
    fri-evidence-layer-len roots (Suc j) =
    fri-evidence-layer-len roots j div 2

```

```

    using fri-layer-lengths-Suc-nth[OF Suc.prem(1), of clength * scale]
    len-eq-j len-eq-suc by simp
have idx-half: idx < fri-evidence-layer-len roots j div 2
    using Suc.prem(2) suc-len by simp
have idx-prev: idx < fri-evidence-layer-len roots j
    using idx-half by simp
have rel:
  fri-algebraic-layer-relation
    (fri-evidence-layer-len roots j)
    (2 ^ j)
    (challenges ! j) (doms ! j) (layers ! j)
    (doms ! Suc j) (layers ! Suc j)
  using generic-fri-canonical-layer-chain-relation[OF chain j-bound]
  len-eq-j by simp
have next-dom:
  doms ! Suc j ! idx = (doms ! j ! idx)2
  by (rule fri-algebraic-layer-relationD(1)[OF rel idx-half])
have prev-dom:
  doms ! j ! idx = (h ^ idx * shift) ^ (2 ^ j)
  by (rule Suc.IH[OF j-bound idx-prev])
let ?x = h ^ idx * shift
have pow-step: (?x ^ (2 ^ j))2 = ?x ^ (2 ^ Suc j)
proof -
  have (?x ^ (2 ^ j))2 = ?x ^ ((2 ^ j) * 2)
    by (simp add: power-mult)
  also have ... = ?x ^ (2 ^ Suc j)
    by (simp add: mult.commute)
  finally show ?thesis .
qed
show ?case
  using next-dom prev-dom pow-step by simp
qed

lemma fri-sibling-domain-round:
  assumes len-pos: 0 < len
  and even-len: 2 dvd len
  and round: len * pw = clength * scale
  shows
    ((h ^ ((i + len div 2) mod len)) * shift) ^ pw =
      - (((h ^ i) * shift) ^ pw)
proof -
  let ?N = clength * scale
  have pw-pos: 0 < pw
    using len-pos round eval-domain-size-pos by (cases pw) simp-all
  have half-round: (len div 2) * pw = ?N div 2
  proof -
    from even-len obtain k where len-eq: len = 2 * k
      by blast
    then have ?N = 2 * (k * pw)

```

```

    using round by simp
  then show ?thesis
    using len-eq by simp
qed
have exp-mod:
  (((i + len div 2) mod len) * pw) mod ?N =
    (i * pw + ?N div 2) mod ?N
proof -
  have (((i + len div 2) mod len) * pw) mod ?N =
    ((i + len div 2) * pw) mod ?N
    using soundness-mod-mult-right-factor-nat
      [OF len-pos pw-pos, of i + len div 2] round by simp
  also have ... = (i * pw + ?N div 2) mod ?N
    using half-round by (simp add: algebra-simps)
  finally show ?thesis .
qed
have h-exp:
  h ^ (((i + len div 2) mod len) * pw) = -(h ^ (i * pw))
proof -
  have h ^ (((i + len div 2) mod len) * pw) =
    h ^ (((i + len div 2) mod len) * pw mod ?N)
    using soundness-h-power-mod-eval-domain
      [of ((i + len div 2) mod len) * pw]
    by simp
  also have ... = h ^ ((i * pw + ?N div 2) mod ?N)
    using exp-mod by simp
  also have ... = h ^ (i * pw + ?N div 2)
    using soundness-h-power-mod-eval-domain[of i * pw + ?N div 2]
    by simp
  also have ... = -(h ^ (i * pw))
    by (rule soundness-h-half-shift)
  finally show ?thesis .
qed
show ?thesis
  using h-exp by (simp add: power-mult power-mult-distrib)
qed

lemma fri-sibling-fold-denominator:
  assumes len-pos: 0 < len
    and even-len: 2 dvd len
    and round: len * pw = clength * scale
  shows
    fri-fold-denominator ((h ^ fri-sibling-index len i * shift) ^ pw) 1 =
      - fri-fold-denominator ((h ^ i * shift) ^ pw) 1
  using fri-sibling-domain-round[OF len-pos even-len round, of i]
  unfolding fri-sibling-index-def fri-fold-denominator-def
  by simp

lemma fri-sibling-index-first-half:

```

```

fixes len idx :: nat
assumes even-len: 2 dvd len
  and idx-bound: idx < len div 2
shows fri-sibling-index len idx = idx + len div 2
proof -
  have len-eq: len = 2 * (len div 2)
    using even-len by simp
  have idx + len div 2 < len
    using idx-bound len-eq by linarith
  then show ?thesis
    unfolding fri-sibling-index-def by simp
qed

```

```

lemma fri-sibling-index-involution-first-half:
  fixes len idx :: nat
  assumes even-len: 2 dvd len
    and idx-bound: idx < len div 2
  shows fri-sibling-index len (fri-sibling-index len idx) = idx
proof -
  have len-eq: len = 2 * (len div 2)
    using even-len by simp
  have sib-eq: fri-sibling-index len idx = idx + len div 2
    by (rule fri-sibling-index-first-half[OF even-len idx-bound])
  have idx-len: idx < len
    using idx-bound len-eq by linarith
  have sum-eq: idx + len div 2 + len div 2 = idx + len
    using len-eq by linarith
  have fri-sibling-index len (fri-sibling-index len idx) =
    ((idx + len div 2) mod len + len div 2) mod len
    unfolding sib-eq fri-sibling-index-def by simp
  also have ... = (idx + len div 2 + len div 2) mod len
    by (simp add: mod-add-left-eq)
  also have ... = (idx + len) mod len
    by (simp only: sum-eq)
  also have ... = idx
    using idx-len by simp
  finally show ?thesis .
qed

```

```

lemma fri-sibling-index-mod-half-second-half:
  fixes len raw :: nat
  assumes even-len: 2 dvd len
    and raw-bound: raw < len
    and second-half: ¬ raw < len div 2
  shows raw = fri-sibling-index len (raw mod (len div 2))
proof -
  let ?half = len div 2
  have len-eq: len = 2 * ?half
    using even-len by simp

```

```

have half-pos:  $0 < ?half$ 
  using raw-bound second-half len-eq by linarith
have half-le-raw:  $?half \leq raw$ 
  using second-half by simp
define k where  $k = raw - ?half$ 
have raw-eq:  $raw = ?half + k$ 
  using half-le-raw unfolding k-def by simp
have k-bound:  $k < ?half$ 
  using raw-bound len-eq raw-eq by linarith
have raw-mod:  $raw \bmod ?half = k$ 
  using raw-eq k-bound half-pos by simp
have fri-sibling-index len  $(raw \bmod ?half) =$ 
   $(k + ?half) \bmod len$ 
  unfolding raw-mod fri-sibling-index-def by simp
also have  $\dots = raw$ 
  using raw-eq raw-bound by simp
finally show ?thesis by simp
qed

lemma fri-table-fold-value-sibling-first-half:
  assumes even-len:  $2 \mid len$ 
  and idx-bound:  $idx < len \div 2$ 
  and round:  $len * pw = clength * scale$ 
  and dom-idx:  $fri\_dom \ ! \ idx = (h \wedge idx * shift) \wedge pw$ 
  and dom-sib:
     $fri\_dom \ ! \ fri\_sibling\_index \ len \ idx =$ 
     $(h \wedge fri\_sibling\_index \ len \ idx * shift) \wedge pw$ 
  shows
     $fri\_table\_fold\_value \ ch \ layer \ fri\_dom \ len \ pw$ 
     $(fri\_sibling\_index \ len \ idx) =$ 
     $fri\_table\_fold\_value \ ch \ layer \ fri\_dom \ len \ pw \ idx$ 
proof -
  have len-pos:  $0 < len$ 
  using idx-bound by simp
  have sib-sib:  $fri\_sibling\_index \ len \ (fri\_sibling\_index \ len \ idx) = idx$ 
  by (rule fri-sibling-index-involution-first-half
    [OF even-len idx-bound])
  have denom-sib:
     $fri\_fold\_denominator$ 
     $((h \wedge fri\_sibling\_index \ len \ idx * shift) \wedge pw) \ 1 =$ 
     $- \ fri\_fold\_denominator \ ((h \wedge idx * shift) \wedge pw) \ 1$ 
  by (rule fri-sibling-fold-denominator
    [OF len-pos even-len round])
  have fri-table-fold-value  $ch \ layer \ fri\_dom \ len \ pw$ 
     $(fri\_sibling\_index \ len \ idx) =$ 
     $fri\_fold\_value \ ch$ 
     $(layer \ ! \ fri\_sibling\_index \ len \ idx)$ 
     $(layer \ ! \ idx)$ 
     $(- \ fri\_fold\_denominator \ ((h \wedge idx * shift) \wedge pw) \ 1)$ 

```

```

unfolding fri-table-fold-value-def
using sib-sib dom-sib denom-sib by simp
also have ... =
  fri-fold-value ch
    (layer ! idx)
    (layer ! fri-sibling-index len idx)
    (fri-fold-denominator ((h ^ idx * shift) ^ pw) 1)
  by (rule fri-fold-value-swap-neg-denominator)
also have ... = fri-table-fold-value ch layer fri-dom len pw idx
  unfolding fri-table-fold-value-def using dom-idx by simp
finally show ?thesis .
qed

lemma fri-sampled-table-fold-from-algebraic-relation:
assumes relation:
  fri-algebraic-layer-relation len pw ch fri-dom layer next-dom
    next-layer
and idx-bound: idx < len div 2
and chunk:
  fri-layer-opening-chunk len xp xp-path xn xn-path chunk
and match: fri-opening-matches-table len idx layer xp xn
and dom-idx: fri-dom ! idx = (h ^ idx * shift) ^ pw
shows idx < length next-layer
and fri-sampled-table-fold ch len idx pw fri-dom layer chunk
  (next-layer ! idx)
proof –
show idx-len: idx < length next-layer
  by (rule fri-algebraic-layer-relation-next-index-bound
    [OF relation idx-bound])
have fold-eq:
  next-layer ! idx = fri-table-fold-value ch layer fri-dom len pw idx
  by (rule fri-algebraic-layer-relation-sampled-fold-eq
    [OF relation idx-bound])
show fri-sampled-table-fold ch len idx pw fri-dom layer chunk
  (next-layer ! idx)
  by (rule fri-sampled-table-foldI[OF chunk match dom-idx fold-eq])
qed

```

```

lemma fri-sampled-table-fold-from-algebraic-relation-sibling:
assumes relation:
  fri-algebraic-layer-relation len pw ch fri-dom layer next-dom
    next-layer
and even-len: 2 dvd len
and round: len * pw = clength * scale
and idx-bound: idx < len div 2
and raw-eq: raw = fri-sibling-index len idx
and chunk:
  fri-layer-opening-chunk len xp xp-path xn xn-path chunk
and match: fri-opening-matches-table len raw layer xp xn

```

and *dom-row*: $\text{fri-dom} \mid \text{raw} = (h \wedge \text{raw} * \text{shift}) \wedge \text{pw}$
and *dom-idx*: $\text{fri-dom} \mid \text{idx} = (h \wedge \text{idx} * \text{shift}) \wedge \text{pw}$
shows $\text{idx} < \text{length next-layer}$
and *fri-sampled-table-fold* *ch len raw pw fri-dom layer chunk*
 $(\text{next-layer} \mid \text{idx})$
proof –
show *idx-len*: $\text{idx} < \text{length next-layer}$
by (rule *fri-algebraic-layer-relation-next-index-bound*
 $[\text{OF relation idx-bound}]$)
have *fold-idx*:
 $\text{next-layer} \mid \text{idx} = \text{fri-table-fold-value } \text{ch layer fri-dom len pw idx}$
by (rule *fri-algebraic-layer-relation-sampled-fold-eq*
 $[\text{OF relation idx-bound}]$)
have *fold-row*:
 $\text{fri-table-fold-value } \text{ch layer fri-dom len pw raw} =$
 $\text{fri-table-fold-value } \text{ch layer fri-dom len pw idx}$
unfolding *raw-eq*
by (rule *fri-table-fold-value-sibling-first-half*
 $[\text{OF even-len idx-bound round dom-idx}]$)
 $(\text{use dom-row raw-eq in simp})$
have *fold-eq*:
 $\text{next-layer} \mid \text{idx} = \text{fri-table-fold-value } \text{ch layer fri-dom len pw raw}$
using *fold-idx fold-row* **by** *simp*
show *fri-sampled-table-fold* *ch len raw pw fri-dom layer chunk*
 $(\text{next-layer} \mid \text{idx})$
by (rule *fri-sampled-table-foldI* $[\text{OF chunk match dom-row fold-eq}]$)
qed

lemma *fri-sampled-table-fold-from-algebraic-relation-mod-index*:

assumes *relation*:

fri-algebraic-layer-relation len pw ch fri-dom layer next-dom
next-layer

and *even-len*: $2 \text{ dvd } \text{len}$

and *round*: $\text{len} * \text{pw} = \text{clength} * \text{scale}$

and *chunk*:

fri-layer-opening-chunk len xp xp-path xn xn-path chunk

and *match*: *fri-opening-matches-table len raw layer xp xn*

and *dom-row*: $\text{fri-dom} \mid \text{raw} = (h \wedge \text{raw} * \text{shift}) \wedge \text{pw}$

and *dom-norm*:

$\text{fri-dom} \mid (\text{raw mod } (\text{len div } 2)) =$
 $(h \wedge (\text{raw mod } (\text{len div } 2)) * \text{shift}) \wedge \text{pw}$

shows $\text{raw mod } (\text{len div } 2) < \text{length next-layer}$

and *fri-sampled-table-fold* *ch len raw pw fri-dom layer chunk*
 $(\text{next-layer} \mid (\text{raw mod } (\text{len div } 2)))$

proof –

have *raw-bound*: $\text{raw} < \text{len}$

by (rule *fri-opening-matches-tableD(1)* $[\text{OF match}]$)

show *idx-len*: $\text{raw mod } (\text{len div } 2) < \text{length next-layer}$

proof (cases $\text{raw} < \text{len div } 2$)


```

case True
then have raw-mod: raw mod (len div 2) = raw
  by simp
show ?thesis
  using fri-sampled-table-fold-from-algebraic-relation(1)
    [OF relation True chunk match dom-raw]
  unfolding raw-mod .
next
case False
let ?idx = raw mod (len div 2)
have half-pos: 0 < len div 2
  using raw-bound False even-len by (cases len) auto
have idx-bound: ?idx < len div 2
  using half-pos by simp
show ?thesis
  by (rule fri-sampled-table-fold-from-algebraic-relation-sibling(1)
    [OF relation even-len round idx-bound - chunk match dom-raw
      dom-norm])
    (rule fri-sibling-index-mod-half-second-half
      [OF even-len raw-bound False])
qed
show fri-sampled-table-fold ch len raw pw fri-dom layer chunk
  (next-layer ! (raw mod (len div 2)))
proof (cases raw < len div 2)
  case True
  then have raw-mod: raw mod (len div 2) = raw
    by simp
  show ?thesis
    using fri-sampled-table-fold-from-algebraic-relation(2)
      [OF relation True chunk match dom-raw]
    unfolding raw-mod .
  next
  case False
  let ?idx = raw mod (len div 2)
  have half-pos: 0 < len div 2
    using raw-bound False even-len by (cases len) auto
  have idx-bound: ?idx < len div 2
    using half-pos by simp
  show ?thesis
    by (rule fri-sampled-table-fold-from-algebraic-relation-sibling(2)
      [OF relation even-len round idx-bound - chunk match dom-raw
        dom-norm])
      (rule fri-sibling-index-mod-half-second-half
        [OF even-len raw-bound False])
qed
qed

lemma fri-sampled-table-fold-eq-at-mod-index:
  assumes fold:

```

```

fri-sampled-table-fold ch len raw pw fri-dom layer chunk next-value
and even-len: 2 dvd len
and round: len * pw = clength * scale
and dom-norm:
  fri-dom ! (raw mod (len div 2)) =
    (h ^ (raw mod (len div 2)) * shift) ^ pw
shows next-value =
  fri-table-fold-value ch layer fri-dom len pw (raw mod (len div 2))
proof -
from fold obtain xp xp-path xn xn-path where
  match: fri-opening-matches-table len raw layer xp xn
  and dom-row: fri-dom ! raw = (h ^ raw * shift) ^ pw
  and next-eq: next-value = fri-table-fold-value ch layer fri-dom len pw raw
  by (elim fri-sampled-table-foldE)
have raw-bound: raw < len
  by (rule fri-opening-matches-tableD(1)[OF match])
show ?thesis
proof (cases raw < len div 2)
case True
  then have raw mod (len div 2) = raw
    by simp
  then show ?thesis
    using next-eq by simp
next
case False
let ?idx = raw mod (len div 2)
have half-pos: 0 < len div 2
  using raw-bound False even-len by (cases len) auto
have idx-bound: ?idx < len div 2
  using half-pos by simp
have raw-eq: raw = fri-sibling-index len ?idx
  by (rule fri-sibling-index-mod-half-second-half
    [OF even-len raw-bound False])
have dom-sib:
  fri-dom ! fri-sibling-index len ?idx =
    (h ^ fri-sibling-index len ?idx * shift) ^ pw
  using dom-row raw-eq by simp
have fold-sib:
  fri-table-fold-value ch layer fri-dom len pw
    (fri-sibling-index len ?idx) =
    fri-table-fold-value ch layer fri-dom len pw ?idx
  by (rule fri-table-fold-value-sibling-first-half
    [OF even-len idx-bound round dom-norm dom-sib])
have fri-table-fold-value ch layer fri-dom len pw raw =
  fri-table-fold-value ch layer fri-dom len pw ?idx
  using raw-eq fold-sib by simp
then show ?thesis
  using next-eq by simp
qed

```

qed

lemma *fri-algebraic-layer-relation-from-sampled-coverage:*

assumes *even-len*: $2 \text{ dvd } \text{len}$
and *round*: $\text{len} * \text{pw} = \text{clength} * \text{scale}$
and *len-pos*: $0 < \text{len}$
and *dom-len*: $\text{len} \leq \text{length } \text{fri-dom}$
and *layer-len*: $\text{len} \leq \text{length } \text{layer}$
and *next-dom-len*: $\text{len} \text{ div } 2 \leq \text{length } \text{next-dom}$
and *next-layer-len*: $\text{len} \text{ div } 2 \leq \text{length } \text{next-layer}$
and *next-dom*:
 $\bigwedge \text{idx}. \text{idx} < \text{len} \text{ div } 2 \implies$
 $\text{next-dom} ! \text{idx} = (\text{fri-dom} ! \text{idx})^2$
and *dom-norm*:
 $\bigwedge \text{idx}. \text{idx} < \text{len} \text{ div } 2 \implies$
 $\text{fri-dom} ! \text{idx} = (h \wedge \text{idx} * \text{shift}) \wedge \text{pw}$
and *sampled-cover*:
 $\bigwedge \text{idx}. \text{idx} < \text{len} \text{ div } 2 \implies$
 $\exists \text{raw chunk}.$
 $\text{raw} \bmod (\text{len} \text{ div } 2) = \text{idx} \wedge$
 $\text{fri-sampled-table-fold } \text{ch } \text{len } \text{raw } \text{pw } \text{fri-dom } \text{layer } \text{chunk}$
 $(\text{next-layer} ! \text{idx})$

shows

fri-algebraic-layer-relation len pw ch fri-dom layer next-dom
next-layer

unfolding *fri-algebraic-layer-relation-def*

proof (*intro conjI allI impI*)

show $0 < \text{len}$
by (*rule len-pos*)
show $\text{len} \leq \text{length } \text{fri-dom}$
by (*rule dom-len*)
show $\text{len} \leq \text{length } \text{layer}$
by (*rule layer-len*)
show $\text{len} \text{ div } 2 \leq \text{length } \text{next-dom}$
by (*rule next-dom-len*)
show $\text{len} \text{ div } 2 \leq \text{length } \text{next-layer}$
by (*rule next-layer-len*)
fix *idx*
assume *idx-bound*: $\text{idx} < \text{len} \text{ div } 2$
show $\text{next-dom} ! \text{idx} = (\text{fri-dom} ! \text{idx})^2$
by (*rule next-dom*[*OF idx-bound*])
obtain *raw chunk* **where** *raw-mod*: $\text{raw} \bmod (\text{len} \text{ div } 2) = \text{idx}$
and *fold*:
 $\text{fri-sampled-table-fold } \text{ch } \text{len } \text{raw } \text{pw } \text{fri-dom } \text{layer } \text{chunk}$
 $(\text{next-layer} ! \text{idx})$
using *sampled-cover*[*OF idx-bound*] **by** *blast*
have *fold-eq*:
 $\text{next-layer} ! \text{idx} =$
 $\text{fri-table-fold-value } \text{ch } \text{layer } \text{fri-dom } \text{len } \text{pw } (\text{raw} \bmod (\text{len} \text{ div } 2))$

```

by (rule fri-sampled-table-fold-eq-at-mod-index
  [OF fold even-len round])
  (use dom-norm idx-bound raw-mod in simp)
then show next-layer ! idx =
  fri-table-fold-value ch layer fri-dom len pw idx
using raw-mod by simp
qed

lemma generic-fri-sampled-layer-chain-bad-challenge-list-from-coverage:
  assumes chain:
    generic-fri-sampled-layer-chain-evidence low-degree candidate-table
    degree-bound roots challenges final-value query-idxs round-layers
    doms layers
  and steps:
    fri-multiround-proximity-steps (length challenges) degree-bound
    (fri-layer-lengths (length challenges) (clength * scale))
    (map ( $\lambda i. 2^i$ ) [0.. $\text{length challenges}$ ])
    doms layers bad
  and covers: fri-sampled-layers-cover-fold-indices roots query-idxs
  and even-len:
     $\bigwedge \text{layer-idx}. \text{layer-idx} < \text{length challenges} \implies$ 
    2 dvd fri-evidence-layer-len roots layer-idx
  and round-product:
     $\bigwedge \text{layer-idx}. \text{layer-idx} < \text{length challenges} \implies$ 
    fri-evidence-layer-len roots layer-idx *  $2^{\text{layer-idx}}$  =
    clength * scale
  and dom-len:
     $\bigwedge \text{layer-idx}. \text{layer-idx} < \text{length challenges} \implies$ 
    fri-evidence-layer-len roots layer-idx  $\leq \text{length (doms ! layer-idx)}$ 
  and layer-len:
     $\bigwedge \text{layer-idx}. \text{layer-idx} < \text{length challenges} \implies$ 
    fri-evidence-layer-len roots layer-idx  $\leq$ 
    length (layers ! layer-idx)
  and next-dom-len:
     $\bigwedge \text{layer-idx}. \text{layer-idx} < \text{length challenges} \implies$ 
    fri-evidence-layer-len roots layer-idx div 2  $\leq$ 
    length (doms ! Suc layer-idx)
  and next-layer-len:
     $\bigwedge \text{layer-idx}. \text{layer-idx} < \text{length challenges} \implies$ 
    fri-evidence-layer-len roots layer-idx div 2  $\leq$ 
    length (layers ! Suc layer-idx)
  and next-dom:
     $\bigwedge \text{layer-idx idx}.$ 
    layer-idx < length challenges  $\implies$ 
    idx < fri-evidence-layer-len roots layer-idx div 2  $\implies$ 
    doms ! Suc layer-idx ! idx =
    (doms ! layer-idx ! idx)2
  and dom-norm:
     $\bigwedge \text{layer-idx idx}.$ 

```

```

    layer-idx < length challenges ==>
    idx < fri-evidence-layer-len roots layer-idx div 2 ==>
    doms ! layer-idx ! idx = (h ^ idx * shift) ^ (2 ^ layer-idx)
and start-not-low:
  ¬ fri-table-low-degree-on degree-bound eval-domain candidate-table
and final-low:
  fri-table-low-degree-on
    (fri-degree-after (length challenges) degree-bound)
    (doms ! length challenges) (layers ! length challenges)
shows challenges ∈
  generic-fri-bad-challenge-lists (length challenges) bad
proof –
  have partial:
    generic-fri-partial-evidence low-degree candidate-table degree-bound
    roots challenges final-value query-idxs round-layers
    by (rule generic-fri-sampled-layer-chain-evidenceD(1)[OF chain])
  have roots-len: length challenges = length roots
    by (rule generic-fri-partial-evidence-shapes(2)[OF partial])
  have challenges-space: challenges ∈ fri-challenge-space (length challenges)
    unfolding fri-challenge-space-def by simp
  have dom0: doms ! 0 = eval-domain
    by (rule generic-fri-sampled-layer-chain-evidenceD(4)[OF chain])
  have layer0: layers ! 0 = candidate-table
    by (rule generic-fri-sampled-layer-chain-evidenceD(5)[OF chain])
  have relations:
    ∧ i. i < length challenges ==>
    fri-algebraic-layer-relation
      (fri-layer-lengths (length challenges) (clength * scale) ! i)
      ((map (λ i. 2 ^ i) [0..<length challenges]) ! i)
      (challenges ! i) (doms ! i) (layers ! i)
      (doms ! Suc i) (layers ! Suc i)
  proof –
    fix i
    assume i-bound: i < length challenges
    have i-root-bound: i < length roots
      using i-bound roots-len by simp
    have len-eq:
      fri-layer-lengths (length challenges) (clength * scale) ! i =
      fri-evidence-layer-len roots i
      using roots-len unfolding fri-evidence-layer-len-def by simp
    have pw-eq:
      (map (λ i. 2 ^ i) [0..<length challenges]) ! i = 2 ^ i
      using i-bound by simp
    have sampled-cover:
      ∧ idx. idx < fri-evidence-layer-len roots i div 2 ==>
      ∃ raw chunk.
        raw mod (fri-evidence-layer-len roots i div 2) = idx ∧
        fri-sampled-table-fold (challenges ! i)
        (fri-evidence-layer-len roots i) raw (2 ^ i)

```

```

      (doms ! i) (layers ! i) chunk
      (layers ! Suc i ! idx)
proof -
  fix idx
  assume idx-bound: idx < fri-evidence-layer-len roots i div 2
  obtain round-idx where round-bound: round-idx < length query-idxs
  and idx-eq:
    fri-evidence-next-idx roots query-idxs round-idx i = idx
  by (rule fri-sampled-layers-cover-fold-indicesD
    [OF covers i-root-bound idx-bound])
  have sample:
    fri-sampled-table-fold (challenges ! i)
      (fri-evidence-layer-len roots i)
      (fri-evidence-layer-idx roots query-idxs round-idx i)
      (2 ^ i) (doms ! i) (layers ! i)
      (round-layers ! round-idx ! i)
      (layers ! Suc i !
        fri-evidence-next-idx roots query-idxs round-idx i)
  by (rule generic-fri-sampled-layer-chain-evidence-sample(2)
    [OF chain round-bound i-bound])
  show  $\exists$  raw chunk.
    raw mod (fri-evidence-layer-len roots i div 2) = idx  $\wedge$ 
    fri-sampled-table-fold (challenges ! i)
      (fri-evidence-layer-len roots i) raw (2 ^ i)
      (doms ! i) (layers ! i) chunk
      (layers ! Suc i ! idx)
  using sample idx-eq
  unfolding fri-evidence-next-idx-def by blast
qed
have len-pos: 0 < fri-evidence-layer-len roots i
  using round-product[OF i-bound] eval-domain-size-pos
  by (cases fri-evidence-layer-len roots i) simp-all
have rel:
  fri-algebraic-layer-relation
    (fri-evidence-layer-len roots i) (2 ^ i)
    (challenges ! i) (doms ! i) (layers ! i)
    (doms ! Suc i) (layers ! Suc i)
  by (rule fri-algebraic-layer-relation-from-sampled-coverage
    [OF even-len[OF i-bound] round-product[OF i-bound] len-pos
      dom-len[OF i-bound] layer-len[OF i-bound]
      next-dom-len[OF i-bound] next-layer-len[OF i-bound]])
    (use next-dom[OF i-bound] dom-norm[OF i-bound] sampled-cover
      in simp-all)
then show
  fri-algebraic-layer-relation
    (fri-layer-lengths (length challenges) (clength * scale) ! i)
    ((map ( $\lambda i$ . 2 ^ i) [0..<length challenges]) ! i)
    (challenges ! i) (doms ! i) (layers ! i)
    (doms ! Suc i) (layers ! Suc i)

```

```

    using rel by (simp add: len-eq pw-eq)
qed
show ?thesis
  unfolding generic-fri-bad-challenge-lists-def
  by (rule fri-multiround-proximity-skeleton
      [OF steps challenges-space relations])
    (use start-not-low final-low dom0 layer0 in simp-all)
qed

lemma generic-fri-sampled-layer-chain-full-cover-bad-challenge-list:
  assumes
    generic-fri-sampled-layer-chain-full-cover low-degree candidate-table
    degree-bound roots challenges final-value query-idxs round-layers
    doms layers bad
  shows challenges ∈
    generic-fri-bad-challenge-lists (length challenges) bad
proof -
  have chain:
    generic-fri-sampled-layer-chain-evidence low-degree candidate-table
    degree-bound roots challenges final-value query-idxs round-layers
    doms layers
  and steps:
    fri-multiround-proximity-steps (length challenges) degree-bound
    (fri-layer-lengths (length challenges) (clength * scale))
    (map (λi. 2 ^ i) [0..<length challenges])
    doms layers bad
  and covers: fri-sampled-layers-cover-fold-indices roots query-idxs
  and even-len:
    ∧layer-idx. layer-idx < length challenges ⇒
      2 dvd fri-evidence-layer-len roots layer-idx
  and round-product:
    ∧layer-idx. layer-idx < length challenges ⇒
      fri-evidence-layer-len roots layer-idx * 2 ^ layer-idx =
        clength * scale
  and dom-len:
    ∧layer-idx. layer-idx < length challenges ⇒
      fri-evidence-layer-len roots layer-idx ≤
        length (doms ! layer-idx)
  and layer-len:
    ∧layer-idx. layer-idx < length challenges ⇒
      fri-evidence-layer-len roots layer-idx ≤
        length (layers ! layer-idx)
  and next-dom-len:
    ∧layer-idx. layer-idx < length challenges ⇒
      fri-evidence-layer-len roots layer-idx div 2 ≤
        length (doms ! Suc layer-idx)
  and next-layer-len:
    ∧layer-idx. layer-idx < length challenges ⇒
      fri-evidence-layer-len roots layer-idx div 2 ≤

```

```

    length (layers ! Suc layer-idx)
and next-dom:
   $\bigwedge$  layer-idx idx.
    layer-idx < length challenges  $\implies$ 
    idx < fri-evidence-layer-len roots layer-idx div 2  $\implies$ 
    doms ! Suc layer-idx ! idx =
      (doms ! layer-idx ! idx)2
and dom-norm:
   $\bigwedge$  layer-idx idx.
    layer-idx < length challenges  $\implies$ 
    idx < fri-evidence-layer-len roots layer-idx div 2  $\implies$ 
    doms ! layer-idx ! idx = (h ^ idx * shift) ^ (2 ^ layer-idx)
and start-not-low:
   $\neg$  fri-table-low-degree-on degree-bound eval-domain candidate-table
and final-low:
  fri-table-low-degree-on
    (fri-degree-after (length challenges) degree-bound)
    (doms ! length challenges) (layers ! length challenges)
using assms
unfolding generic-fri-sampled-layer-chain-full-cover-def
by blast+
show ?thesis
by (rule generic-fri-sampled-layer-chain-bad-challenge-list-from-coverage
  [OF chain steps covers even-len round-product dom-len layer-len
    next-dom-len next-layer-len next-dom dom-norm start-not-low
    final-low])
qed

lemma generic-fri-bad-candidate-from-sampled-layer-chain-full-cover:
  assumes cover:
    generic-fri-sampled-layer-chain-full-cover low-degree candidate-table
    degree-bound roots challenges final-value query-idxs round-layers
    doms layers bad
  and not-low:  $\neg$  low-degree candidate-table
  shows
    generic-fri-bad-candidate low-degree candidate-table degree-bound
    roots challenges final-value query-idxs round-layers bad
proof -
  have sampled:
    generic-fri-sampled-layer-chain-evidence low-degree candidate-table
    degree-bound roots challenges final-value query-idxs round-layers
    doms layers
  using cover
  unfolding generic-fri-sampled-layer-chain-full-cover-def by blast
  have partial:
    generic-fri-partial-evidence low-degree candidate-table degree-bound
    roots challenges final-value query-idxs round-layers
  by (rule generic-fri-sampled-layer-chain-evidenceD(1)[OF sampled])
  have bad:

```


challenges $\in$ *generic-fri-bad-challenge-lists* (*length challenges*) *bad*
by (rule *generic-fri-sampled-layer-chain-full-cover-bad-challenge-list*
 $[OF\ cover]$)
show ?thesis
unfolding *generic-fri-bad-candidate-def*
using *partial not-low bad* **by** *simp*
qed

lemma *generic-fri-canonical-opening-match-sample*:

assumes *openings*:

fri-all-round-openings-match-tables

(*take* (*length challenges*) *layers*) *query-idxs* *round-layers*

and *layers-len*: *length layers* = *Suc* (*length challenges*)

and *roots-len*: *length challenges* = *length roots*

and *round-bound*: *round-idx* < *length query-idxs*

and *layer-bound*: *layer-idx* < *length challenges*

obtains *xp xp-path xn xn-path*

where

fri-layer-opening-chunk (*fri-evidence-layer-len* *roots* *layer-idx*)

xp xp-path xn xn-path (*round-layers* ! *round-idx* ! *layer-idx*)

and *fri-opening-matches-table*

(*fri-evidence-layer-len* *roots* *layer-idx*)

(*fri-evidence-layer-idx* *roots* *query-idxs* *round-idx* *layer-idx*)

(*layers* ! *layer-idx*) *xp xn*

proof —

have *layer-bound-take*:

layer-idx < *length* (*take* (*length challenges*) *layers*)

using *layer-bound layers-len* **by** *simp*

have *openings-match*:

fri-layer-opening-matches-bound-table (*query-idxs* ! *round-idx*)

layer-idx (*take* (*length challenges*) *layers*)

(*round-layers* ! *round-idx*)

by (rule *fri-all-round-openings-match-tablesD*

$[OF\ openings\ round-bound\ layer-bound-take]$)

have *tables-len*:

length (*take* (*length challenges*) *layers*) = *length roots*

using *layers-len roots-len* **by** *simp*

from *fri-layer-opening-matches-bound-table-compactE*

$[OF\ openings-match\ tables-len]$

obtain *xp xp-path xn xn-path* **where**

chunk:

fri-layer-opening-chunk (*fri-evidence-layer-len* *roots* *layer-idx*)

xp xp-path xn xn-path (*round-layers* ! *round-idx* ! *layer-idx*)

and *table-match-take*:

fri-opening-matches-table

(*fri-evidence-layer-len* *roots* *layer-idx*)

(*fri-layer-indices* (*length roots*) (*query-idxs* ! *round-idx*)

(*clength* * *scale*) ! *layer-idx*)

(*take* (*length challenges*) *layers* ! *layer-idx*) *xp xn*

```

  by blast
have idx-eq:
  fri-layer-indices (length roots) (query-idxs ! round-idx)
    (clength * scale) ! layer-idx =
    fri-evidence-layer-idx roots query-idxs round-idx layer-idx
  unfolding fri-evidence-layer-idx-def by simp
have table-eq:
  take (length challenges) layers ! layer-idx = layers ! layer-idx
  using layer-bound layers-len by simp
have table-match:
  fri-opening-matches-table
    (fri-evidence-layer-len roots layer-idx)
    (fri-evidence-layer-idx roots query-idxs round-idx layer-idx)
    (layers ! layer-idx) xp xn
  using table-match-take idx-eq table-eq by simp
show ?thesis
  by (rule that[OF chunk table-match])
qed

lemma generic-fri-canonical-sampled-fold-at-first-half:
  assumes chain:
    generic-fri-canonical-layer-chain-evidence low-degree candidate-table
      degree-bound roots challenges final-value query-idxs round-layers
      doms layers bad
  and openings:
    fri-all-round-openings-match-tables
      (take (length challenges) layers) query-idxs round-layers
  and domains: fri-sampled-domains-align roots query-idxs doms
  and idx-bound:
    fri-evidence-layer-idx roots query-idxs round-idx layer-idx <
    fri-evidence-layer-len roots layer-idx div 2
  and round-bound: round-idx < length query-idxs
  and layer-bound: layer-idx < length challenges
  shows
    fri-evidence-next-idx roots query-idxs round-idx layer-idx <
      length (layers ! Suc layer-idx)
  fri-sampled-table-fold
    (challenges ! layer-idx)
    (fri-evidence-layer-len roots layer-idx)
    (fri-evidence-layer-idx roots query-idxs round-idx layer-idx)
    (2 ^ layer-idx)
    (doms ! layer-idx)
    (layers ! layer-idx)
    (round-layers ! round-idx ! layer-idx)
    (layers ! Suc layer-idx !
      fri-evidence-next-idx roots query-idxs round-idx layer-idx)
proof -
  have partial:
    generic-fri-partial-evidence low-degree candidate-table degree-bound

```

```

    roots challenges final-value query-idxs round-layers
using chain unfolding generic-fri-canonical-layer-chain-evidence-def
by simp
have roots-len: length challenges = length roots
  by (rule generic-fri-partial-evidence-shapes(2)[OF partial])
have layers-len: length layers = Suc (length challenges)
  by (rule generic-fri-canonical-layer-chain-lengths(2)[OF chain])
have len-eq:
  fri-layer-lengths (length challenges) (clength * scale) ! layer-idx =
    fri-evidence-layer-len roots layer-idx
  using roots-len unfolding fri-evidence-layer-len-def by simp
have relation:
  fri-algebraic-layer-relation
    (fri-evidence-layer-len roots layer-idx)
    (2 ^ layer-idx)
    (challenges ! layer-idx) (doms ! layer-idx) (layers ! layer-idx)
    (doms ! Suc layer-idx) (layers ! Suc layer-idx)
  using generic-fri-canonical-layer-chain-relation
    [OF chain layer-bound] len-eq by simp
have dom-idx:
  doms ! layer-idx !
    fri-evidence-layer-idx roots query-idxs round-idx layer-idx =
    (h ^ fri-evidence-layer-idx roots query-idxs round-idx layer-idx *
      shift) ^ (2 ^ layer-idx)
  by (rule fri-sampled-domains-alignD[OF domains round-bound])
    (use layer-bound roots-len in simp)
obtain xp xp-path xn xn-path where
  chunk:
    fri-layer-opening-chunk (fri-evidence-layer-len roots layer-idx)
      xp xp-path xn xn-path (round-layers ! round-idx ! layer-idx)
and match:
  fri-opening-matches-table
    (fri-evidence-layer-len roots layer-idx)
    (fri-evidence-layer-idx roots query-idxs round-idx layer-idx)
    (layers ! layer-idx) xp xn
  by (rule generic-fri-canonical-opening-match-sample
    [OF openings layers-len roots-len round-bound layer-bound])
have next-idx-eq:
  fri-evidence-next-idx roots query-idxs round-idx layer-idx =
    fri-evidence-layer-idx roots query-idxs round-idx layer-idx
  using idx-bound unfolding fri-evidence-next-idx-def by simp
have idx-len-orig:
  fri-evidence-layer-idx roots query-idxs round-idx layer-idx <
    length (layers ! Suc layer-idx)
  by (rule fri-sampled-table-fold-from-algebraic-relation(1)
    [OF relation idx-bound chunk match dom-idx])
then show
  fri-evidence-next-idx roots query-idxs round-idx layer-idx <
    length (layers ! Suc layer-idx)

```

```

using next-idx-eq by simp
have fold-orig:
  fri-sampled-table-fold
    (challenges ! layer-idx)
    (fri-evidence-layer-len roots layer-idx)
    (fri-evidence-layer-idx roots query-idxs round-idx layer-idx)
    ( $2 \wedge \text{layer-idx}$ )
    (doms ! layer-idx)
    (layers ! layer-idx)
    (round-layers ! round-idx ! layer-idx)
    (layers ! Suc layer-idx !
      fri-evidence-layer-idx roots query-idxs round-idx layer-idx)
by (rule fri-sampled-table-fold-from-algebraic-relation
  [OF relation idx-bound chunk match dom-idx])
then show
  fri-sampled-table-fold
    (challenges ! layer-idx)
    (fri-evidence-layer-len roots layer-idx)
    (fri-evidence-layer-idx roots query-idxs round-idx layer-idx)
    ( $2 \wedge \text{layer-idx}$ )
    (doms ! layer-idx)
    (layers ! layer-idx)
    (round-layers ! round-idx ! layer-idx)
    (layers ! Suc layer-idx !
      fri-evidence-next-idx roots query-idxs round-idx layer-idx)
using next-idx-eq by simp
qed

lemma generic-fri-canonical-sampled-fold-at-any-index:
assumes chain:
  generic-fri-canonical-layer-chain-evidence low-degree candidate-table
  degree-bound roots challenges final-value query-idxs round-layers
  doms layers bad
and openings:
  fri-all-round-openings-match-tables
    (take (length challenges) layers) query-idxs round-layers
and domains: fri-sampled-domains-align roots query-idxs doms
and even-len:  $2 \text{ dvd } \text{fri-evidence-layer-len}$  roots layer-idx
and round:
  fri-evidence-layer-len roots layer-idx *  $2 \wedge \text{layer-idx} =$ 
    clength * scale
and dom-norm:
  doms ! layer-idx !
    fri-evidence-next-idx roots query-idxs round-idx layer-idx =
    ( $h \wedge \text{fri-evidence-next-idx}$  roots query-idxs round-idx layer-idx *
      shift)  $\wedge (2 \wedge \text{layer-idx})$ 
and round-bound: round-idx < length query-idxs
and layer-bound: layer-idx < length challenges
shows

```

```

fri-evidence-next-idx roots query-idxs round-idx layer-idx <
  length (layers ! Suc layer-idx)
fri-sampled-table-fold
  (challenges ! layer-idx)
  (fri-evidence-layer-len roots layer-idx)
  (fri-evidence-layer-idx roots query-idxs round-idx layer-idx)
  (2 ^ layer-idx)
  (doms ! layer-idx)
  (layers ! layer-idx)
  (round-layers ! round-idx ! layer-idx)
  (layers ! Suc layer-idx !
    fri-evidence-next-idx roots query-idxs round-idx layer-idx)
proof –
  have partial:
    generic-fri-partial-evidence low-degree candidate-table degree-bound
      roots challenges final-value query-idxs round-layers
    using chain unfolding generic-fri-canonical-layer-chain-evidence-def
    by simp
  have roots-len: length challenges = length roots
    by (rule generic-fri-partial-evidence-shapes(2)[OF partial])
  have layers-len: length layers = Suc (length challenges)
    by (rule generic-fri-canonical-layer-chain-lengths(2)[OF chain])
  have len-eq:
    fri-layer-lengths (length challenges) (clength * scale) ! layer-idx =
      fri-evidence-layer-len roots layer-idx
    using roots-len unfolding fri-evidence-layer-len-def by simp
  have relation:
    fri-algebraic-layer-relation
      (fri-evidence-layer-len roots layer-idx)
      (2 ^ layer-idx)
      (challenges ! layer-idx) (doms ! layer-idx) (layers ! layer-idx)
      (doms ! Suc layer-idx) (layers ! Suc layer-idx)
    using generic-fri-canonical-layer-chain-relation
      [OF chain layer-bound] len-eq by simp
  have dom-raw:
    doms ! layer-idx !
      fri-evidence-layer-idx roots query-idxs round-idx layer-idx =
      (h ^ fri-evidence-layer-idx roots query-idxs round-idx layer-idx *
        shift) ^ (2 ^ layer-idx)
    by (rule fri-sampled-domains-alignD[OF domains round-bound])
    (use layer-bound roots-len in simp)
  obtain xp xp-path xn xn-path where
    chunk:
      fri-layer-opening-chunk (fri-evidence-layer-len roots layer-idx)
        xp xp-path xn xn-path (round-layers ! round-idx ! layer-idx)
  and match:
    fri-opening-matches-table
      (fri-evidence-layer-len roots layer-idx)
      (fri-evidence-layer-idx roots query-idxs round-idx layer-idx)

```

```

    (layers ! layer-idx) xp xn
  by (rule generic-fri-canonical-opening-match-sample
    [OF openings layers-len roots-len round-bound layer-bound])
show
  fri-evidence-next-idx roots query-idxs round-idx layer-idx <
    length (layers ! Suc layer-idx)
  unfolding fri-evidence-next-idx-def
  by (rule fri-sampled-table-fold-from-algebraic-relation-mod-index(1)
    [OF relation even-len round chunk match dom-row])
    (use dom-norm fri-evidence-next-idx-def in simp)
show
  fri-sampled-table-fold
    (challenges ! layer-idx)
    (fri-evidence-layer-len roots layer-idx)
    (fri-evidence-layer-idx roots query-idxs round-idx layer-idx)
    (2 ^ layer-idx)
    (doms ! layer-idx)
    (layers ! layer-idx)
    (round-layers ! round-idx ! layer-idx)
    (layers ! Suc layer-idx !
      fri-evidence-next-idx roots query-idxs round-idx layer-idx)
  unfolding fri-evidence-next-idx-def
  by (rule fri-sampled-table-fold-from-algebraic-relation-mod-index(2)
    [OF relation even-len round chunk match dom-row])
    (use dom-norm fri-evidence-next-idx-def in simp)
qed

```

lemma *generic-fri-canonical-sampled-raw-domain-power:*

```

  assumes chain:
    generic-fri-canonical-layer-chain-evidence low-degree candidate-table
      degree-bound roots challenges final-value query-idxs round-layers
      doms layers bad
  and openings:
    fri-all-round-openings-match-tables
      (take (length challenges) layers) query-idxs round-layers
  and round-bound: round-idx < length query-idxs
  and layer-bound: layer-idx < length challenges
  shows
    doms ! layer-idx !
      fri-evidence-layer-idx roots query-idxs round-idx layer-idx =
      (h ^ fri-evidence-layer-idx roots query-idxs round-idx layer-idx *
        shift) ^ (2 ^ layer-idx)

```

proof –

```

  have partial:
    generic-fri-partial-evidence low-degree candidate-table degree-bound
      roots challenges final-value query-idxs round-layers
  using chain unfolding generic-fri-canonical-layer-chain-evidence-def
  by simp
  have roots-len: length challenges = length roots

```

```

  by (rule generic-fri-partial-evidence-shapes(2)[OF partial])
have layers-len: length layers = Suc (length challenges)
  by (rule generic-fri-canonical-layer-chain-lengths(2)[OF chain])
obtain xp xp-path xn xn-path where match:
  fri-opening-matches-table
    (fri-evidence-layer-len roots layer-idx)
    (fri-evidence-layer-idx roots query-idxs round-idx layer-idx)
    (layers ! layer-idx) xp xn
  by (rule generic-fri-canonical-opening-match-sample
      [OF openings layers-len roots-len round-bound layer-bound])
have idx-bound:
  fri-evidence-layer-idx roots query-idxs round-idx layer-idx <
    fri-evidence-layer-len roots layer-idx
  by (rule fri-opening-matches-tableD(1)[OF match])
show ?thesis
  by (rule generic-fri-canonical-domain-power
      [OF chain roots-len layer-bound idx-bound])
qed

lemma generic-fri-canonical-layer-chain-as-sampled:
  assumes chain:
    generic-fri-canonical-layer-chain-evidence low-degree candidate-table
      degree-bound roots challenges final-value query-idxs round-layers
      doms layers bad
  and openings:
    fri-all-round-openings-match-tables
      (take (length challenges) layers) query-idxs round-layers
  and domains: fri-sampled-domains-align roots query-idxs doms
  and next-domains: fri-sampled-next-domains-align roots query-idxs doms
  and even-len:
     $\bigwedge \text{layer-idx. layer-idx} < \text{length challenges} \implies$ 
    2 dvd fri-evidence-layer-len roots layer-idx
  and round:
     $\bigwedge \text{layer-idx. layer-idx} < \text{length challenges} \implies$ 
    fri-evidence-layer-len roots layer-idx * 2 ^ layer-idx =
    clength * scale
  shows
    generic-fri-sampled-layer-chain-evidence low-degree candidate-table
      degree-bound roots challenges final-value query-idxs round-layers
      doms layers
proof -
  have partial:
    generic-fri-partial-evidence low-degree candidate-table degree-bound
      roots challenges final-value query-idxs round-layers
  using chain unfolding generic-fri-canonical-layer-chain-evidence-def
  by simp
  have roots-len: length challenges = length roots
  by (rule generic-fri-partial-evidence-shapes(2)[OF partial])
  have doms-len: length doms = Suc (length challenges)

```

```

  by (rule generic-fri-canonical-layer-chain-lengths(1)[OF chain])
have layers-len: length layers = Suc (length challenges)
  by (rule generic-fri-canonical-layer-chain-lengths(2)[OF chain])
have dom0: doms ! 0 = eval-domain
  using chain unfolding generic-fri-canonical-layer-chain-evidence-def
  by simp
have layer0: layers ! 0 = candidate-table
  using chain unfolding generic-fri-canonical-layer-chain-evidence-def
  by simp
have final:
  fri-final-constant-consistent (layers ! length challenges) final-value
  using chain unfolding generic-fri-canonical-layer-chain-evidence-def
  by simp
have sampled:
   $\bigwedge \text{round-idx layer-idx.}$ 
    round-idx < length query-idxs  $\implies$ 
    layer-idx < length challenges  $\implies$ 
    fri-evidence-next-idx roots query-idxs round-idx layer-idx <
      length (layers ! Suc layer-idx)  $\wedge$ 
    fri-sampled-table-fold
      (challenges ! layer-idx)
      (fri-evidence-layer-len roots layer-idx)
      (fri-evidence-layer-idx roots query-idxs round-idx layer-idx)
      (2 ^ layer-idx)
      (doms ! layer-idx)
      (layers ! layer-idx)
      (round-layers ! round-idx ! layer-idx)
      (layers ! Suc layer-idx !
        fri-evidence-next-idx roots query-idxs round-idx layer-idx)
proof -
  fix round-idx layer-idx
  assume round-bound: round-idx < length query-idxs
  and layer-bound: layer-idx < length challenges
  have layer-bound-roots: layer-idx < length roots
  using layer-bound roots-len by simp
  have dom-norm:
    doms ! layer-idx !
      fri-evidence-next-idx roots query-idxs round-idx layer-idx =
      (h ^ fri-evidence-next-idx roots query-idxs round-idx layer-idx *
        shift) ^ (2 ^ layer-idx)
  by (rule fri-sampled-next-domains-alignD
    [OF next-domains round-bound layer-bound-roots])
show
  fri-evidence-next-idx roots query-idxs round-idx layer-idx <
    length (layers ! Suc layer-idx)  $\wedge$ 
  fri-sampled-table-fold
    (challenges ! layer-idx)
    (fri-evidence-layer-len roots layer-idx)
    (fri-evidence-layer-idx roots query-idxs round-idx layer-idx)

```



```

    (2 ^ layer-idx)
    (doms ! layer-idx)
    (layers ! layer-idx)
    (round-layers ! round-idx ! layer-idx)
    (layers ! Suc layer-idx !
      fri-evidence-next-idx roots query-idxs round-idx layer-idx)
  by (intro conjI)
    (rule generic-fri-canonical-sampled-fold-at-any-index
      [OF chain openings domains even-len[OF layer-bound]
        round[OF layer-bound] dom-norm round-bound layer-bound])+
qed
show ?thesis
  unfolding generic-fri-sampled-layer-chain-evidence-def
  using partial doms-len layers-len dom0 layer0 final sampled by blast
qed

```

lemma *generic-fri-bad-challenge-lists-subset:*
generic-fri-bad-challenge-lists n bad $\subseteq$ fri-challenge-space n
unfolding *generic-fri-bad-challenge-lists-def*
by (rule *fri-multiround-bad-challenge-lists-subset*)

lemma *generic-fri-bad-challenge-lists-round-union:*
generic-fri-bad-challenge-lists n bad =
($\bigcup i \in \{..<n\}. \text{generic-fri-round-bad-challenge-lists } n \text{ bad } i$)
unfolding *generic-fri-bad-challenge-lists-def*
fri-multiround-bad-challenge-lists-def
generic-fri-round-bad-challenge-lists-def
by *auto*

lemma *generic-fri-round-bad-challenge-lists-image-subset:*
generic-fri-round-bad-challenge-lists n bad i $\subseteq$
($\lambda(\text{prefix}, b, \text{suffix}). \text{prefix} @ b \# \text{suffix}$) ‘
generic-fri-round-bad-witnesses n bad i
proof
fix *bs*
assume *bs-in:* *bs $\in$ generic-fri-round-bad-challenge-lists n bad i*
then have *bs-space:* *bs $\in$ fri-challenge-space n*
and *i-bound:* *i < n*
and *hit:* *bs ! i $\in$ bad i (take i bs)*
unfolding *generic-fri-round-bad-challenge-lists-def* **by** *simp-all*
have *len-bs:* *length bs = n*
using *bs-space* **unfolding** *fri-challenge-space-def* **by** *simp*
let *?prefix = take i bs*
let *?b = bs ! i*
let *?suffix = drop (Suc i) bs*
have *witness:*
(?prefix, ?b, ?suffix) $\in$ generic-fri-round-bad-witnesses n bad i
unfolding *generic-fri-round-bad-witnesses-def*
using *i-bound hit len-bs* **by** *simp*

```

have bs-eq: bs = ?prefix @ ?b # ?suffix
  using i-bound len-bs by (simp add: id-take-nth-drop)
show bs ∈ (λ(prefix, b, suffix). prefix @ b # suffix) ‘
  generic-fri-round-bad-witnesses n bad i
  using witness bs-eq by force
qed

lemma generic-fri-round-bad-witnesses-subset:
  generic-fri-round-bad-witnesses n bad i ⊆
    {(prefix, b, suffix).
      length prefix = i ∧
      b ∈ bad i prefix ∧
      length suffix = n - Suc i}
  unfolding generic-fri-round-bad-witnesses-def by auto

lemma generic-fri-bad-challenge-lists-card-bound-by-rounds:
  card (generic-fri-bad-challenge-lists n bad) ≤
    (∑ i ∈ {.. $n$ }. card (generic-fri-round-bad-challenge-lists n bad i))
  unfolding generic-fri-bad-challenge-lists-round-union
  by (rule card-UN-le) simp

lemma generic-fri-bad-challenge-lists-bounded-byI:
  assumes card-bound: card (generic-fri-bad-challenge-lists n bad) ≤ bd
  shows fri-bad-challenge-lists-bounded-by n
    (generic-fri-bad-challenge-lists n bad) bd
  unfolding fri-bad-challenge-lists-bounded-by-def
  using generic-fri-bad-challenge-lists-subset card-bound by simp

lemma generic-fri-round-bad-witnesses-finite:
  finite (generic-fri-round-bad-witnesses n bad i)
proof -
  let ?P = {prefix :: 'f list. length prefix = i}
  let ?S = {suffix :: 'f list. length suffix = n - Suc i}
  have witness-eq:
    generic-fri-round-bad-witnesses n bad i =
      (if i < n then (SIGMA prefix: ?P. bad i prefix × ?S) else {})
  unfolding generic-fri-round-bad-witnesses-def by auto
  show ?thesis
proof (cases i < n)
  case True
  have finite-P: finite ?P
    by (rule finite-length-lists-UNIV)
  have finite-S: finite ?S
    by (rule finite-length-lists-UNIV)
  have finite-sigma: finite (SIGMA prefix: ?P. bad i prefix × ?S)
    by (rule finite-SigmaI[OF finite-P]) (use finite-S in simp)
  then show ?thesis
    using True witness-eq by simp
next

```

```

    case False
    then show ?thesis
      using witness-eq by simp
  qed
qed

lemma generic-fri-round-bad-witnesses-card-bound:
  assumes local-bound:
     $\bigwedge \text{prefix}. \text{length prefix} = i \implies \text{card} (\text{bad } i \text{ prefix}) \leq \text{bd}$ 
  shows  $\text{card} (\text{generic-fri-round-bad-witnesses } n \text{ bad } i) \leq$ 
     $\text{CARD}('f) \wedge i * \text{bd} * \text{CARD}('f) \wedge (n - \text{Suc } i)$ 
  proof -
    let ?P = {prefix :: 'f list. length prefix = i}
    let ?S = {suffix :: 'f list. length suffix = n - Suc i}
    have witness-eq:
      generic-fri-round-bad-witnesses n bad i =
        (if i < n then (SIGMA prefix: ?P. bad i prefix  $\times$  ?S) else {})
    unfolding generic-fri-round-bad-witnesses-def by auto
    show ?thesis
  proof (cases i < n)
    case False
    then show ?thesis
      using witness-eq by simp
  next
    case True
    have finite-P: finite ?P
      by (rule finite-length-lists-UNIV)
    have finite-S: finite ?S
      by (rule finite-length-lists-UNIV)
    have finite-products:  $\forall \text{prefix} \in ?P. \text{finite} (\text{bad } i \text{ prefix} \times ?S)$ 
      using finite-S by simp
    have card-witness:
       $\text{card} (\text{generic-fri-round-bad-witnesses } n \text{ bad } i) =$ 
       $(\sum \text{prefix} \in ?P. \text{card} (\text{bad } i \text{ prefix} \times ?S))$ 
    proof -
      have  $\text{card} (\text{SIGMA prefix: ?P. bad } i \text{ prefix} \times ?S) =$ 
         $(\sum \text{prefix} \in ?P. \text{card} (\text{bad } i \text{ prefix} \times ?S))$ 
        by (rule card-SigmaI[OF finite-P finite-products])
      then show ?thesis
        using True witness-eq by simp
    qed
    also have  $\dots \leq (\sum \text{prefix} \in ?P. \text{card} (\text{bad } i \text{ prefix}) * \text{CARD}('f) \wedge (n - \text{Suc } i))$ 
    proof (rule sum-mono)
      fix prefix
      assume prefix  $\in$  ?P
      have  $\text{card} (\text{bad } i \text{ prefix} \times ?S) =$ 
         $\text{card} (\text{bad } i \text{ prefix}) * \text{card } ?S$ 
        by (simp add: card-cartesian-product)
    qed
  qed

```

```

also have ... =
  card (bad i prefix) * CARD('f) ^ (n - Suc i)
  by (simp add: card-length-lists-UNIV)
finally show card (bad i prefix × ?S) ≤
  card (bad i prefix) * CARD('f) ^ (n - Suc i)
  by simp
qed
also have ... ≤ (∑ prefix ∈ ?P.
  bd * CARD('f) ^ (n - Suc i))
proof (rule sum-mono)
  fix prefix
  assume prefix-in: prefix ∈ ?P
  have card (bad i prefix) ≤ bd
    by (rule local-bound) (use prefix-in in simp)
  then show card (bad i prefix) * CARD('f) ^ (n - Suc i)
    ≤ bd * CARD('f) ^ (n - Suc i)
    by simp
  qed
also have ... = card ?P * (bd * CARD('f) ^ (n - Suc i))
  by simp
also have ... = CARD('f) ^ i * bd * CARD('f) ^ (n - Suc i)
  by (simp add: card-length-lists-UNIV mult.assoc)
finally show ?thesis
  using card-witness by simp
qed
qed

lemma generic-fri-round-bad-challenge-lists-card-bound:
  assumes local-bound:
    ∧ prefix. length prefix = i ⇒ card (bad i prefix) ≤ bd
  shows card (generic-fri-round-bad-challenge-lists n bad i) ≤
    CARD('f) ^ i * bd * CARD('f) ^ (n - Suc i)
  proof -
    have image-subset:
      generic-fri-round-bad-challenge-lists n bad i ⊆
        (λ(prefix, b, suffix). prefix @ b # suffix) ‘
          generic-fri-round-bad-witnesses n bad i
      by (rule generic-fri-round-bad-challenge-lists-image-subset)
    have finite-image:
      finite ((λ(prefix, b, suffix). prefix @ b # suffix) ‘
        generic-fri-round-bad-witnesses n bad i)
      by (rule finite-imageI)
        (rule generic-fri-round-bad-witnesses-finite)
    have card (generic-fri-round-bad-challenge-lists n bad i) ≤
      card ((λ(prefix, b, suffix). prefix @ b # suffix) ‘
        generic-fri-round-bad-witnesses n bad i)
      by (rule card-mono[OF finite-image image-subset])
    also have ... ≤ card (generic-fri-round-bad-witnesses n bad i)
      by (rule card-image-le)

```

(rule generic-fri-round-bad-witnesses-finite)
also have ... $\leq \text{CARD}(f) \wedge i * \text{bd} * \text{CARD}(f) \wedge (n - \text{Suc } i)$
by (rule generic-fri-round-bad-witnesses-card-bound)
 (rule local-bound)
finally show ?thesis .
qed

lemma generic-fri-bad-challenge-lists-card-bound-from-round-bounds:
assumes local-bound:
 $\bigwedge i \text{ prefix. } i < n \implies \text{length prefix} = i \implies$
 $\text{card (bad } i \text{ prefix)} \leq \text{bd } i$
shows $\text{card (generic-fri-bad-challenge-lists } n \text{ bad)} \leq$
 $(\sum i \in \{..<n\}. \text{CARD}(f) \wedge i * \text{bd } i * \text{CARD}(f) \wedge (n - \text{Suc } i))$
proof –
have $\text{card (generic-fri-bad-challenge-lists } n \text{ bad)} \leq$
 $(\sum i \in \{..<n\}. \text{card (generic-fri-round-bad-challenge-lists } n \text{ bad } i))$
by (rule generic-fri-bad-challenge-lists-card-bound-by-rounds)
also have ... $\leq$
 $(\sum i \in \{..<n\}. \text{CARD}(f) \wedge i * \text{bd } i * \text{CARD}(f) \wedge (n - \text{Suc } i))$
proof (rule sum-mono)
fix i
assume $i\text{-in: } i \in \{..<n\}$
show $\text{card (generic-fri-round-bad-challenge-lists } n \text{ bad } i) \leq$
 $\text{CARD}(f) \wedge i * \text{bd } i * \text{CARD}(f) \wedge (n - \text{Suc } i)$
by (rule generic-fri-round-bad-challenge-lists-card-bound)
 (rule local-bound[of i], use $i\text{-in}$ in simp-all)
qed
finally show ?thesis .
qed

lemma generic-fri-bad-challenge-lists-bounded-by-from-round-bounds:
assumes local-bound:
 $\bigwedge i \text{ prefix. } i < n \implies \text{length prefix} = i \implies$
 $\text{card (bad } i \text{ prefix)} \leq \text{bd } i$
shows fri-bad-challenge-lists-bounded-by n
 $(\text{generic-fri-bad-challenge-lists } n \text{ bad})$
 $(\sum i \in \{..<n\}. \text{CARD}(f) \wedge i * \text{bd } i * \text{CARD}(f) \wedge (n - \text{Suc } i))$
by (rule generic-fri-bad-challenge-lists-bounded-byI)
 (rule generic-fri-bad-challenge-lists-card-bound-from-round-bounds
 [OF local-bound])

lemma generic-fri-bad-challenge-lists-card-bound:
 $\text{card (generic-fri-bad-challenge-lists } n \text{ bad)} \leq \text{CARD}(f) \wedge n$
unfolding generic-fri-bad-challenge-lists-def
by (rule fri-multiround-bad-challenge-lists-card-bound)

lemma generic-fri-bad-challenge-lists-bounded-by:
 fri-bad-challenge-lists-bounded-by n
 $(\text{generic-fri-bad-challenge-lists } n \text{ bad}) (\text{CARD}(f) \wedge n)$

unfolding *generic-fri-bad-challenge-lists-def*
by (rule *fri-multiround-bad-challenge-lists-bounded-by*)

lemma *generic-fri-low-degree-bad-challenge-listI*:

assumes *steps*:

fri-multiround-proximity-steps *n d lens pws doms layers bad*

and *challenges*: *bs* $\in$ *fri-challenge-space* *n*

and *relations*:

$\bigwedge i. i < n \implies$

fri-algebraic-layer-relation (*lens* ! *i*) (*pws* ! *i*) (*bs* ! *i*)

(*doms* ! *i*) (*layers* ! *i*) (*doms* ! *Suc i*) (*layers* ! *Suc i*)

and *start-not-low*:

$\neg$ *fri-table-low-degree-on* *d* (*doms* ! 0) (*layers* ! 0)

and *final-low*:

fri-table-low-degree-on (*fri-degree-after* *n d*) (*doms* ! *n*)

(*layers* ! *n*)

shows *bs* $\in$ *generic-fri-bad-challenge-lists* *n bad*

unfolding *generic-fri-bad-challenge-lists-def*

by (rule *fri-multiround-proximity-skeleton*

[*OF steps challenges relations start-not-low final-low*])

lemma *generic-fri-proximity-evidence-bad-challenge-list*:

assumes *generic-fri-proximity-evidence* *n d lens pws doms layers bs bad*

shows *bs* $\in$ *generic-fri-bad-challenge-lists* *n bad*

proof –

have *steps*:

fri-multiround-proximity-steps *n d lens pws doms layers bad*

and *challenges*: *bs* $\in$ *fri-challenge-space* *n*

and *relations*:

$\bigwedge i. i < n \implies$

fri-algebraic-layer-relation (*lens* ! *i*) (*pws* ! *i*) (*bs* ! *i*)

(*doms* ! *i*) (*layers* ! *i*) (*doms* ! *Suc i*) (*layers* ! *Suc i*)

and *start-not-low*:

$\neg$ *fri-table-low-degree-on* *d* (*doms* ! 0) (*layers* ! 0)

and *final-low*:

fri-table-low-degree-on (*fri-degree-after* *n d*) (*doms* ! *n*)

(*layers* ! *n*)

using *assms* **unfolding** *generic-fri-proximity-evidence-def* **by** *blast+*

show *?thesis*

by (rule *generic-fri-low-degree-bad-challenge-listI*

[*OF steps challenges relations start-not-low final-low*])

qed

lemma *generic-fri-canonical-layer-chain-bad-challenge-list*:

assumes

generic-fri-canonical-layer-chain-evidence *low-degree* *candidate-table*

degree-bound *roots* *challenges* *final-value* *query-idxs* *round-layers*

doms *layers* *bad*

shows *challenges* $\in$

generic-fri-bad-challenge-lists (length challenges) bad
proof –
have *generic-fri-proximity-evidence (length challenges) degree-bound*
*(fri-layer-lengths (length challenges) (clength * scale))*
*(map ($\lambda i. 2^i$) [0..*length challenges*])*
doms layers challenges bad
using *assms*
unfolding *generic-fri-canonical-layer-chain-evidence-def* **by** *simp*
then show *?thesis*
by (rule *generic-fri-proximity-evidence-bad-challenge-list*)
qed

lemma *generic-fri-canonical-layer-chain-evidenceD*:
assumes
generic-fri-canonical-layer-chain-evidence low-degree candidate-table
degree-bound roots challenges final-value query-idxs round-layers
doms layers bad
shows
generic-fri-partial-evidence low-degree candidate-table degree-bound
roots challenges final-value query-idxs round-layers
doms ! 0 = eval-domain
layers ! 0 = candidate-table
fri-final-constant-consistent (layers ! length challenges) final-value
generic-fri-proximity-evidence (length challenges) degree-bound
*(fri-layer-lengths (length challenges) (clength * scale))*
*(map ($\lambda i. 2^i$) [0..*length challenges*])*
doms layers challenges bad
using *assms* **unfolding** *generic-fri-canonical-layer-chain-evidence-def*
by *simp-all*

lemma *generic-fri-bad-candidate-from-canonical-layer-chain*:
assumes *chain*:
generic-fri-canonical-layer-chain-evidence low-degree candidate-table
degree-bound roots challenges final-value query-idxs round-layers
doms layers bad
and *not-low*: $\neg$ *low-degree candidate-table*
shows
generic-fri-bad-candidate low-degree candidate-table degree-bound
roots challenges final-value query-idxs round-layers bad
proof –
have *partial*:
generic-fri-partial-evidence low-degree candidate-table degree-bound
roots challenges final-value query-idxs round-layers
by (rule *generic-fri-canonical-layer-chain-evidenceD(1)[OF chain]*)
have *bad*:
challenges $\in$
generic-fri-bad-challenge-lists (length challenges) bad
by (rule *generic-fri-canonical-layer-chain-bad-challenge-list[OF chain]*)
show *?thesis*

unfolding *generic-fri-bad-candidate-def*
using *partial not-low bad* **by** *simp*
qed

lemma *generic-fri-bad-challenge-lists-mono*:
assumes *mono*:
 $\bigwedge i \text{ prefix. } i < n \implies \text{length prefix} = i \implies$
 $\text{bad } i \text{ prefix} \subseteq \text{bad}' i \text{ prefix}$
shows *generic-fri-bad-challenge-lists* *n bad* $\subseteq$
generic-fri-bad-challenge-lists *n bad'*
proof
fix *bs*
assume *bs-in*: *bs* $\in$ *generic-fri-bad-challenge-lists* *n bad*
then have *space*: *bs* $\in$ *fri-challenge-space* *n*
unfolding *generic-fri-bad-challenge-lists-def*
using *fri-multiround-bad-challenge-lists-subset* **by** *blast*
from *bs-in* **obtain** *i* **where** *i-bound*: *i* $<$ *n*
and *hit*: *bs* ! *i* $\in$ *bad* *i* (*take* *i* *bs*)
unfolding *generic-fri-bad-challenge-lists-def*
fri-multiround-bad-challenge-lists-def
by *blast*
have *len-bs*: *length* *bs* = *n*
using *space* **unfolding** *fri-challenge-space-def* **by** *simp*
have *len-take*: *length* (*take* *i* *bs*) = *i*
using *i-bound len-bs* **by** *simp*
have *bs* ! *i* $\in$ *bad'* *i* (*take* *i* *bs*)
using *hit mono[OF i-bound len-take]* **by** *blast*
then show *bs* $\in$ *generic-fri-bad-challenge-lists* *n bad'*
using *space i-bound*
unfolding *generic-fri-bad-challenge-lists-def*
fri-multiround-bad-challenge-lists-def
by *blast*
qed

lemma *generic-fri-bad-candidateD*:
assumes *generic-fri-bad-candidate* *low-degree candidate-table degree-bound*
roots challenges final-value query-idxs round-layers bad
shows *generic-fri-partial-evidence* *low-degree candidate-table degree-bound*
roots challenges final-value query-idxs round-layers
and $\neg$ *low-degree candidate-table*
and *challenges* $\in$ *generic-fri-bad-challenge-lists* (*length challenges*) *bad*
using *assms* **unfolding** *generic-fri-bad-candidate-def* **by** *simp-all*

lemma *generic-fri-bad-candidate-challenges-in-space*:
assumes *bad-candidate*:
generic-fri-bad-candidate *low-degree candidate-table degree-bound*
roots challenges final-value query-idxs round-layers bad
shows *challenges* $\in$ *fri-challenge-space* (*length challenges*)
proof —


```

have challenges ∈ generic-fri-bad-challenge-lists (length challenges) bad
  using generic-fri-bad-candidateD(3)[OF bad-candidate] .
then show ?thesis
  using generic-fri-bad-challenge-lists-subset[of length challenges bad]
  by blast
qed

lemma generic-fri-bad-candidate-challenges-length:
  assumes bad-candidate:
    generic-fri-bad-candidate low-degree candidate-table degree-bound
    roots challenges final-value query-idxs round-layers bad
  shows length challenges = fri-round-count-for-degree-bound degree-bound
proof -
  have evidence:
    generic-fri-partial-evidence low-degree candidate-table degree-bound
    roots challenges final-value query-idxs round-layers
  using generic-fri-bad-candidateD(1)[OF bad-candidate] .
  then show ?thesis
    using generic-fri-partial-evidence-shapes(1,2)[OF evidence] by simp
qed

lemma trace-table-low-degree-iff-fri-table-low-degree-on-eval-domain:
  trace-table-low-degree trace-table  $\longleftrightarrow$ 
  fri-table-low-degree-on (clength - 1) eval-domain trace-table
proof
  assume trace-table-low-degree trace-table
  then obtain f where deg-f: degree f < clength
    and table: trace-table = map (poly f) eval-domain
    unfolding trace-table-low-degree-def by blast
  have degree f ≤ clength - 1
    using deg-f by simp
  then show fri-table-low-degree-on (clength - 1) eval-domain trace-table
    unfolding fri-table-low-degree-on-def
    using table by blast
next
  assume fri-table-low-degree-on (clength - 1) eval-domain trace-table
  then obtain p where deg-p: degree p ≤ clength - 1
    and table: trace-table = map (poly p) eval-domain
    unfolding fri-table-low-degree-on-def by blast
  have degree p < clength
    using deg-p clength-pos by simp
  then show trace-table-low-degree trace-table
    unfolding trace-table-low-degree-def
    using table by blast
qed

lemma composition-table-low-degree-iff-fri-table-low-degree-on-eval-domain:
  composition-table-low-degree d composition-table  $\longleftrightarrow$ 
  fri-table-low-degree-on d eval-domain composition-table

```

unfolding *composition-table-low-degree-def* *fri-table-low-degree-on-def*
by *simp*

lemma *composition-table-low-degree-mono*:

assumes $d \leq d'$

and *composition-table-low-degree* d *composition-table*

shows *composition-table-low-degree* d' *composition-table*

proof –

from *assms*(2) **obtain** q **where** *deg-q*: *degree* $q \leq d$

and *table*: *composition-table* = *map* (*poly* q) *eval-domain*

unfolding *composition-table-low-degree-def* **by** *blast*

have *degree* $q \leq d'$

using *deg-q* *assms*(1) **by** *simp*

then show *?thesis*

unfolding *composition-table-low-degree-def*

using *table* **by** *blast*

qed

lemma *composition-table-not-low-degree-mono*:

assumes $d \leq d'$

and $\neg$ *composition-table-low-degree* d' *composition-table*

shows $\neg$ *composition-table-low-degree* d *composition-table*

using *composition-table-low-degree-mono*[*OF* *assms*(1)] *assms*(2) **by** *blast*

lemma *trace-table-not-low-degree-iff-not-fri-table-low-degree-on-eval-domain*:

$\neg$ *trace-table-low-degree* *trace-table* $\longleftrightarrow$

$\neg$ *fri-table-low-degree-on* (*clength* – 1) *eval-domain* *trace-table*

by (*simp* *add*: *trace-table-low-degree-iff-fri-table-low-degree-on-eval-domain*)

lemma *composition-table-not-low-degree-iff-not-fri-table-low-degree-on-eval-domain*:

$\neg$ *composition-table-low-degree* d *composition-table* $\longleftrightarrow$

$\neg$ *fri-table-low-degree-on* d *eval-domain* *composition-table*

by (*simp* *add*:

composition-table-low-degree-iff-fri-table-low-degree-on-eval-domain)

lemma *accepted-fri-opening-transcript-trace-generic-partial-evidence*:

assumes *fri-openings*:

accepted-fri-opening-transcript s *out* *trace-roots* *trace-bs* *trace-final*

dg *composition-roots* *composition-bs* *composition-final* *query-idxs*

trace-round-layers *composition-round-layers*

shows

generic-fri-partial-evidence *trace-table-low-degree* *trace-table*

(*clength* – 1) *trace-roots* *trace-bs* *trace-final* *query-idxs*

trace-round-layers

proof –

have *count*:

length *trace-roots* = *fri-round-count-for-degree-bound* (*clength* – 1)

using *accepted-fri-opening-transcript-algebraic-round-counts*(1)

[*OF* *fri-openings*]

```

    by (simp add: trace-fri-algebraic-round-count-def)
  have len-bs: length trace-bs = length trace-roots
    using accepted-fri-opening-transcript-shapes[OF fri-openings] by blast
  have layers:
    fri-all-round-layer-evidence trace-roots trace-bs query-idxs
      trace-round-layers
  by (rule accepted-fri-opening-transcript-trace-all-round-layer-evidence
      [OF fri-openings])
  show ?thesis
    unfolding generic-fri-partial-evidence-def
    using count len-bs layers by simp
qed

```

lemma *accepted-fri-opening-transcript-composition-generic-partial-evidence:*

assumes *fri-openings:*

accepted-fri-opening-transcript s out trace-roots trace-bs trace-final
dg composition-roots composition-bs composition-final query-idxs
trace-round-layers composition-round-layers

shows

generic-fri-partial-evidence (composition-table-low-degree (to-nat dg))
composition-table (to-nat dg) composition-roots composition-bs
composition-final query-idxs composition-round-layers

proof —

have *count:*

length composition-roots = fri-round-count-for-degree-bound (to-nat dg)
using *accepted-fri-opening-transcript-algebraic-round-counts(3)*
[OF fri-openings]
by (simp add: composition-fri-algebraic-round-count-def)

have *len-bs:* *length composition-bs = length composition-roots*
using *accepted-fri-opening-transcript-shapes[OF fri-openings]* **by** blast

have *layers:*

fri-all-round-layer-evidence composition-roots composition-bs query-idxs
composition-round-layers

by (rule
accepted-fri-opening-transcript-composition-all-round-layer-evidence
[OF fri-openings])

show ?thesis
 unfolding generic-fri-partial-evidence-def
 using count len-bs layers by simp

qed

lemma *trace-fri-partial-candidate-opening-evidence-generic-partial:*

assumes *evidence:*

trace-fri-partial-candidate-opening-evidence s out trace-roots
trace-bs trace-final dg composition-roots composition-bs
composition-final fri-query-idxs trace-round-layers
composition-round-layers fr candidate-query-idxs trace-openings
trace-table

shows

generic-fri-partial-evidence trace-table-low-degree trace-table
(clength - 1) trace-roots trace-bs trace-final fri-query-idxs
trace-round-layers
proof –
have *fri-openings*:
accepted-fri-opening-transcript s out trace-roots trace-bs
trace-final dg composition-roots composition-bs composition-final
fri-query-idxs trace-round-layers composition-round-layers
using *trace-fri-partial-candidate-opening-evidenceD(1)[OF evidence]* .
show *?thesis*
by (rule *accepted-fri-opening-transcript-trace-generic-partial-evidence*
[OF fri-openings])
qed

lemma *composition-fri-partial-candidate-opening-evidence-generic-partial*:
assumes *evidence*:
composition-fri-partial-candidate-opening-evidence s out trace-roots
trace-bs trace-final fri-dg opening-composition-roots composition-bs
composition-final fri-query-idxs trace-round-layers
composition-round-layers fr f-fri-roots f-final as dg
composition-fri-roots final trace-query-idxs trace-openings
composition-query-idxs composition-openings trace-table
composition-table
shows
generic-fri-partial-evidence
(composition-table-low-degree (to-nat fri-dg))
composition-table (to-nat fri-dg) opening-composition-roots
composition-bs composition-final fri-query-idxs
composition-round-layers

proof –
have *fri-openings*:
accepted-fri-opening-transcript s out trace-roots trace-bs
trace-final fri-dg opening-composition-roots composition-bs
composition-final fri-query-idxs trace-round-layers
composition-round-layers
using *composition-fri-partial-candidate-opening-evidenceD(1)*
[OF evidence] .
show *?thesis*
by (rule
accepted-fri-opening-transcript-composition-generic-partial-evidence
[OF fri-openings])
qed

lemma *trace-fri-partial-candidate-opening-evidence-not-low-on-eval-domain*:
assumes *evidence*:
trace-fri-partial-candidate-opening-evidence s out trace-roots
trace-bs trace-final dg composition-roots composition-bs
composition-final fri-query-idxs trace-round-layers
composition-round-layers fr candidate-query-idxs trace-openings

```

      trace-table
    shows  $\neg$  fri-table-low-degree-on (clength - 1) eval-domain trace-table
  proof -
    have partial:
      trace-fri-partial-candidate-evidence s out fr candidate-query-idxs
      trace-openings trace-table
    using trace-fri-partial-candidate-opening-evidenceD(2)[OF evidence] .
    have  $\neg$  trace-table-low-degree trace-table
    using trace-fri-partial-candidate-evidenceD(3)[OF partial] .
    then show ?thesis
    using trace-table-not-low-degree-iff-not-fri-table-low-degree-on-eval-domain
    by simp
  qed

```

lemma *composition-fri-partial-candidate-opening-evidence-not-low-declared:*

```

  assumes evidence:
    composition-fri-partial-candidate-opening-evidence s out trace-roots
    trace-bs trace-final fri-dg opening-composition-roots composition-bs
    composition-final fri-query-idxs trace-round-layers
    composition-round-layers fr f-fri-roots f-final as dg
    composition-fri-roots final trace-query-idxs trace-openings
    composition-query-idxs composition-openings trace-table
    composition-table
  and degree-bound: to-nat fri-dg  $\leq$  maxDegree
  shows  $\neg$  composition-table-low-degree (to-nat fri-dg) composition-table
  proof -
    have partial:
      composition-fri-partial-candidate-evidence s out fr f-fri-roots
      f-final as dg composition-fri-roots final trace-query-idxs
      trace-openings composition-query-idxs composition-openings
      trace-table composition-table
    by (rule composition-fri-partial-candidate-opening-evidenceD(2)
        [OF evidence])
    have not-low-max:
       $\neg$  composition-table-low-degree maxDegree composition-table
    by (rule composition-fri-partial-candidate-evidenceD(5)[OF partial])
    show ?thesis
    by (rule composition-table-not-low-degree-mono
        [OF degree-bound not-low-max])
  qed

```

lemma *composition-fri-partial-candidate-opening-evidence-not-low-declared-on-eval-domain:*

```

  assumes evidence:
    composition-fri-partial-candidate-opening-evidence s out trace-roots
    trace-bs trace-final fri-dg opening-composition-roots composition-bs
    composition-final fri-query-idxs trace-round-layers
    composition-round-layers fr f-fri-roots f-final as dg
    composition-fri-roots final trace-query-idxs trace-openings
    composition-query-idxs composition-openings trace-table

```

composition-table
and *degree-bound: to-nat fri-dg ≤ maxDegree*
shows $\neg$ *fri-table-low-degree-on (to-nat fri-dg) eval-domain*
composition-table
proof –
have $\neg$ *composition-table-low-degree (to-nat fri-dg) composition-table*
by (rule *composition-fri-partial-candidate-opening-evidence-not-low-declared*
[*OF evidence degree-bound*])
then show ?thesis
using *composition-table-not-low-degree-iff-not-fri-table-low-degree-on-eval-domain*
by *simp*
qed

lemma *trace-fri-partial-candidate-opening-evidence-canonical-bad-candidate:*

assumes *evidence:*
trace-fri-partial-candidate-opening-evidence s out trace-roots
trace-bs trace-final dg composition-roots composition-bs
composition-final fri-query-idxs trace-round-layers
composition-round-layers fr candidate-query-idxs trace-openings
trace-table
and *chain:*
generic-fri-canonical-layer-chain-evidence trace-table-low-degree
trace-table (clength – 1) trace-roots trace-bs trace-final
fri-query-idxs trace-round-layers doms layers bad

shows
generic-fri-bad-candidate trace-table-low-degree trace-table
(clength – 1) trace-roots trace-bs trace-final fri-query-idxs
trace-round-layers bad

proof –
have $\neg$ *trace-table-low-degree trace-table*
using *trace-fri-partial-candidate-opening-evidenceD(2)[OF evidence]*
trace-fri-partial-candidate-evidenceD(3)
by *blast*
then show ?thesis
by (rule *generic-fri-bad-candidate-from-canonical-layer-chain*
[*OF chain*])
qed

lemma *composition-fri-partial-candidate-opening-evidence-canonical-bad-candidate:*

assumes *evidence:*
composition-fri-partial-candidate-opening-evidence s out trace-roots
trace-bs trace-final fri-dg opening-composition-roots composition-bs
composition-final fri-query-idxs trace-round-layers
composition-round-layers fr f-fri-roots f-final as dg
composition-fri-roots final trace-query-idxs trace-openings
composition-query-idxs composition-openings trace-table
composition-table
and *degree-bound: to-nat fri-dg ≤ maxDegree*
and *chain:*

```

generic-fri-canonical-layer-chain-evidence
  (composition-table-low-degree (to-nat fri-dg)) composition-table
  (to-nat fri-dg) opening-composition-roots composition-bs
  composition-final fri-query-idxs composition-round-layers doms layers bad
shows
  generic-fri-bad-candidate
    (composition-table-low-degree (to-nat fri-dg)) composition-table
    (to-nat fri-dg) opening-composition-roots composition-bs
    composition-final fri-query-idxs composition-round-layers bad
proof -
  have  $\neg$  composition-table-low-degree (to-nat fri-dg) composition-table
  by (rule composition-fri-partial-candidate-opening-evidence-not-low-declared
    [OF evidence degree-bound])
  then show ?thesis
  by (rule generic-fri-bad-candidate-from-canonical-layer-chain
    [OF chain])
qed

lemma trace-fri-multiround-bad-sets-as-generic:
  trace-fri-multiround-bad-sets bad trace-table =
    generic-fri-bad-challenge-lists trace-fri-algebraic-round-count
      (bad trace-table)
unfolding trace-fri-multiround-bad-sets-def
  generic-fri-bad-challenge-lists-def
by simp

lemma composition-fri-multiround-bad-sets-as-generic:
  composition-fri-multiround-bad-sets bad dg composition-table =
    generic-fri-bad-challenge-lists
      (composition-fri-algebraic-round-count dg)
      (bad dg composition-table)
unfolding composition-fri-multiround-bad-sets-def
  generic-fri-bad-challenge-lists-def
by simp

end

end

```

```

theory Soundness-FRI-Internal-Bounds
  imports Soundness-FRI-Generic-Low-Degree
begin

```

Internal concrete FRI challenge-list bounds.

These lemmas deliberately avoid the symbolic parameters *trace-fri-error* and *composition-fri-error*. They expose the concrete bad-set fractions obtained from the generic FRI bad challenge-list interface. Later public-route work can either use these concrete terms directly, or relate the symbolic error parameters to these terms via an explicitly chosen

FRI soundness interface.

context *soundness*

begin

definition *fri-bad-challenge-round-mass*

$:: \text{nat} \Rightarrow (\text{nat} \Rightarrow \text{nat}) \Rightarrow \text{nat}$

where

fri-bad-challenge-round-mass n $bd =$
 $(\sum i \in \{..<n\}. \text{CARD}('f) \wedge i * bd\ i * \text{CARD}('f) \wedge (n - \text{Suc}\ i))$

definition *fri-bad-challenge-round-fraction*

$:: \text{nat} \Rightarrow (\text{nat} \Rightarrow \text{nat}) \Rightarrow \text{prob}$

where

fri-bad-challenge-round-fraction n $bd =$
 $\text{nnreal} (\text{fri-bad-challenge-round-mass}\ n\ bd) / \text{nnreal} (\text{CARD}('f) \wedge n)$

definition *fri-one-step-disagreement-round-mass*

$:: \text{nat} \Rightarrow (\text{nat} \Rightarrow \text{nat}) \Rightarrow \text{nat}$

where

fri-one-step-disagreement-round-mass n $len =$
fri-bad-challenge-round-mass n $(\lambda i. \text{len}\ i \text{ div } 2)$

definition *fri-one-step-disagreement-round-fraction*

$:: \text{nat} \Rightarrow (\text{nat} \Rightarrow \text{nat}) \Rightarrow \text{prob}$

where

fri-one-step-disagreement-round-fraction n $len =$
fri-bad-challenge-round-fraction n $(\lambda i. \text{len}\ i \text{ div } 2)$

lemma *fri-one-step-disagreement-round-fraction-unfold:*

fri-one-step-disagreement-round-fraction n $len =$
 nnreal
 $(\sum i \in \{..<n\}. \text{CARD}('f) \wedge i * (\text{len}\ i \text{ div } 2) * \text{CARD}('f) \wedge (n - \text{Suc}\ i)) /$
 $\text{nnreal} (\text{CARD}('f) \wedge n)$

unfolding *fri-one-step-disagreement-round-fraction-def*

fri-bad-challenge-round-fraction-def

fri-bad-challenge-round-mass-def

by *simp*

lemma *fri-one-step-disagreement-round-fraction-refl-bound:*

nnreal
 $(\sum i \in \{..<n\}. \text{CARD}('f) \wedge i * (\text{len}\ i \text{ div } 2) * \text{CARD}('f) \wedge (n - \text{Suc}\ i)) /$
 $\text{nnreal} (\text{CARD}('f) \wedge n)$
 $\leq \text{fri-one-step-disagreement-round-fraction}\ n\ len$
by (*simp add: fri-one-step-disagreement-round-fraction-unfold*)

definition *trace-fri-canonical-layer-len* $:: \text{nat} \Rightarrow \text{nat}$

where

trace-fri-canonical-layer-len $i =$
fri-layer-lengths (*fri-round-count-for-degree-bound* (*clength* $- 1$))
(*clength* $* scale$) ! i

definition *composition-fri-canonical-layer-len* :: 'f $\Rightarrow$ nat $\Rightarrow$ nat
where

composition-fri-canonical-layer-len $dg\ i =$
fri-layer-lengths (*fri-round-count-for-degree-bound* (*to-nat* dg))
(*clength* $* scale$) ! i

definition *concrete-trace-fri-bad-challenge-fraction* :: prob
where

concrete-trace-fri-bad-challenge-fraction =
fri-one-step-disagreement-round-fraction
(*fri-round-count-for-degree-bound* (*clength* $- 1$))
trace-fri-canonical-layer-len

definition *concrete-composition-fri-bad-challenge-fraction* :: prob
where

concrete-composition-fri-bad-challenge-fraction =
Max
(($\lambda dg :: 'f.$
fri-one-step-disagreement-round-fraction
(*fri-round-count-for-degree-bound* (*to-nat* dg))
(*composition-fri-canonical-layer-len* dg)) ' UNIV)

lemma *concrete-trace-fri-bad-challenge-fraction-eq:*

fri-one-step-disagreement-round-fraction
(*fri-round-count-for-degree-bound* (*clength* $- 1$))
trace-fri-canonical-layer-len
= *concrete-trace-fri-bad-challenge-fraction*
by (*simp* *add: concrete-trace-fri-bad-challenge-fraction-def*)

lemma *concrete-composition-fri-bad-challenge-fraction-ge:*

fri-one-step-disagreement-round-fraction
(*fri-round-count-for-degree-bound* (*to-nat* dg))
(*composition-fri-canonical-layer-len* dg)
 $\leq$ *concrete-composition-fri-bad-challenge-fraction*
unfolding *concrete-composition-fri-bad-challenge-fraction-def*
by (*rule* *Max-ge*) *simp-all*

lemma *fri-bad-challenge-round-fraction-bound-from-round-bounds:*

assumes *local-bound:*
 $\bigwedge i\ prefix. i < n \implies length\ prefix = i \implies$
 $card\ (bad\ i\ prefix) \leq bd\ i$
shows $nnreal\ (card\ (generic-fri-bad-challenge-lists\ n\ bad)) /$
 $nnreal\ (CARD('f) ^ n)$
 $\leq fri-bad-challenge-round-fraction\ n\ bd$
unfolding *fri-bad-challenge-round-fraction-def*

fri-bad-challenge-round-mass-def
by (rule nnreal-nat-divide-right-mono)
(rule generic-fri-bad-challenge-lists-card-bound-from-round-bounds
[OF local-bound])

lemma *generic-fri-bad-challenge-lists-card-bound-from-one-step-disagreement-subset:*

assumes *local-subset:*

$\bigwedge i \text{ prefix}. i < n \implies \text{length prefix} = i \implies$
bad i prefix $\subseteq$

fri-one-step-disagreement-challenges

(*len i*) (*pw i*) (*layer i prefix*) (*claimed i prefix*) (*domains i prefix*)

shows *card (generic-fri-bad-challenge-lists n bad)* $\leq$

$(\sum i \in \{..<n\}).$

$CARD('f) \wedge i * (\text{len } i \text{ div } 2) * CARD('f) \wedge (n - \text{Suc } i))$

proof (rule generic-fri-bad-challenge-lists-card-bound-from-round-bounds)

fix *i :: nat and prefix :: 'f list*

assume *i-bound: i < n and prefix-len: length prefix = i*

have *card (bad i prefix)* $\leq$

card

(*fri-one-step-disagreement-challenges*

(*len i*) (*pw i*) (*layer i prefix*) (*claimed i prefix*) (*domains i prefix*))

by (rule card-mono) (simp-all add: local-subset[OF i-bound prefix-len])

also have ... $\leq \text{len } i \text{ div } 2$

by (rule fri-one-step-disagreement-challenges-card-bound-local)

finally show *card (bad i prefix)* $\leq \text{len } i \text{ div } 2$.

qed

lemma *generic-fri-bad-challenge-lists-bounded-by-from-one-step-disagreement-subset:*

assumes *local-subset:*

$\bigwedge i \text{ prefix}. i < n \implies \text{length prefix} = i \implies$
bad i prefix $\subseteq$

fri-one-step-disagreement-challenges

(*len i*) (*pw i*) (*layer i prefix*) (*claimed i prefix*) (*domains i prefix*)

shows *fri-bad-challenge-lists-bounded-by n*

(*generic-fri-bad-challenge-lists n bad*)

$(\sum i \in \{..<n\}).$

$CARD('f) \wedge i * (\text{len } i \text{ div } 2) * CARD('f) \wedge (n - \text{Suc } i))$

by (rule generic-fri-bad-challenge-lists-bounded-byI)

(rule generic-fri-bad-challenge-lists-card-bound-from-one-step-disagreement-subset
[OF local-subset])

lemma *fri-bad-challenge-round-fraction-bound-from-one-step-disagreement-subset:*

assumes *local-subset:*

$\bigwedge i \text{ prefix}. i < n \implies \text{length prefix} = i \implies$
bad i prefix $\subseteq$

fri-one-step-disagreement-challenges

(*len i*) (*pw i*) (*layer i prefix*) (*claimed i prefix*) (*domains i prefix*)

shows *nnreal (card (generic-fri-bad-challenge-lists n bad)) /*

nnreal (CARD('f) $\wedge$ n)

$\leq$ *fri-one-step-disagreement-round-fraction* *n* *len*
unfolding *fri-one-step-disagreement-round-fraction-def*
fri-bad-challenge-round-fraction-def
fri-one-step-disagreement-round-mass-def
fri-bad-challenge-round-mass-def
by (rule *nnreal-nat-divide-right-mono*)
(rule *generic-fri-bad-challenge-lists-card-bound-from-one-step-disagreement-subset*
[*OF local-subset*])

lemma *wp-verify-monad-trace-generic-fri-bad-challenge-list-bound:*

assumes *future: trace-fri-future-fresh s*

shows

wp-event verify-monad
(*trace-fri-challenge-list-set-hit s*
(*generic-fri-bad-challenge-lists trace-fri-algebraic-round-count bad*))
 s
 $\leq$
nnreal
(*card*
(*generic-fri-bad-challenge-lists trace-fri-algebraic-round-count*
bad)) /
nnreal (*CARD('f) ^ ceil-log clength*)

proof (rule *wp-verify-monad-trace-fri-challenge-list-set-bound[OF future]*)

show *generic-fri-bad-challenge-lists trace-fri-algebraic-round-count bad*

$\subseteq$ *fri-challenge-space (ceil-log clength)*

using *generic-fri-bad-challenge-lists-subset*

by (*simp add: trace-fri-algebraic-round-count-eq*)

qed

lemma *wp-verify-monad-trace-generic-fri-bad-challenge-list-bound-from-card:*

assumes *future: trace-fri-future-fresh s*

and *card-bound:*

card (generic-fri-bad-challenge-lists trace-fri-algebraic-round-count
bad) ≤ bd

shows

wp-event verify-monad
(*trace-fri-challenge-list-set-hit s*
(*generic-fri-bad-challenge-lists trace-fri-algebraic-round-count bad*))
 s
 $\leq$ *nnreal* *bd* / *nnreal* (*CARD('f) ^ ceil-log clength*)

proof –

have *wp-event verify-monad*

(*trace-fri-challenge-list-set-hit s*
(*generic-fri-bad-challenge-lists trace-fri-algebraic-round-count bad*))

s

$\leq$

nnreal
(*card*
(*generic-fri-bad-challenge-lists trace-fri-algebraic-round-count*

$bad)) /$
 $nnreal (CARD('f) \wedge \text{ceil-log } clength)$
by (rule *wp-verify-monad-trace-generic-fri-bad-challenge-list-bound*
 $[OF \text{ future}]$)
also have $\dots \leq nnreal \text{ bd} / nnreal (CARD('f) \wedge \text{ceil-log } clength)$
by (rule *nnreal-nat-divide-right-mono* $[OF \text{ card-bound}]$)
finally show *?thesis* .
qed

lemma *wp-verify-monad-trace-generic-fri-bad-challenge-list-bound-from-rounds*:
assumes *future: trace-fri-future-fresh s*
and *local-bound*:
 $\bigwedge i \text{ prefix. } i < \text{trace-fri-algebraic-round-count} \implies$
 $\text{length prefix} = i \implies \text{card } (bad \ i \ \text{prefix}) \leq \text{bd } i$
shows
 $\text{wp-event } \text{verify-monad}$
 $(\text{trace-fri-challenge-list-set-hit } s$
 $(\text{generic-fri-bad-challenge-lists } \text{trace-fri-algebraic-round-count } bad))$
 s
 $\leq$
 $nnreal$
 $(\sum i \in \{..<\text{trace-fri-algebraic-round-count}\}.$
 $CARD('f) \wedge i * \text{bd } i *$
 $CARD('f) \wedge (\text{trace-fri-algebraic-round-count} - \text{Suc } i)) /$
 $nnreal (CARD('f) \wedge \text{ceil-log } clength)$
by (rule *wp-verify-monad-trace-generic-fri-bad-challenge-list-bound-from-card*
 $[OF \text{ future}]$)
(rule *generic-fri-bad-challenge-lists-card-bound-from-round-bounds*
 $[OF \text{ local-bound}]$)

lemma *wp-verify-monad-composition-generic-fri-bad-challenge-list-bound*:
assumes *future: composition-fri-future-fresh s*
and *bound*:
 $\bigwedge dg.$
 $nnreal$
 $(\text{card}$
 $(\text{generic-fri-bad-challenge-lists}$
 $(\text{composition-fri-algebraic-round-count } dg) (bad \ dg))) /$
 $nnreal (CARD('f) \wedge \text{ceil-log } (\text{to-nat } dg + 1)) \leq C$
shows
 $\text{wp-event } \text{verify-monad}$
 $(\text{composition-fri-challenge-list-set-hit } s$
 $(\lambda dg. \text{generic-fri-bad-challenge-lists}$
 $(\text{composition-fri-algebraic-round-count } dg) (bad \ dg)))$
 $s \leq C$
proof (rule *wp-verify-monad-composition-fri-challenge-list-set-bound*
 $[OF \text{ future}]$)
fix *dg*
show *generic-fri-bad-challenge-lists*

```

    (composition-fri-algebraic-round-count dg) (bad dg)
    ⊆ fri-challenge-space (ceil-log (to-nat dg + 1))
using generic-fri-bad-challenge-lists-subset
by (simp add: composition-fri-algebraic-round-count-eq)
show nnreal
  (card
    (generic-fri-bad-challenge-lists
      (composition-fri-algebraic-round-count dg) (bad dg))) /
  nnreal (CARD('f) ^ ceil-log (to-nat dg + 1)) ≤ C
by (rule bound)
qed

lemma wp-verify-monad-composition-generic-fri-bad-challenge-list-bound-from-card:
assumes future: composition-fri-future-fresh s
and card-bound:
  ∧ dg. card
    (generic-fri-bad-challenge-lists
      (composition-fri-algebraic-round-count dg) (bad dg)) ≤ bd dg
and fraction-bound:
  ∧ dg. nnreal (bd dg) /
    nnreal (CARD('f) ^ ceil-log (to-nat dg + 1)) ≤ C
shows
  wp-event verify-monad
    (composition-fri-challenge-list-set-hit s
      (λ dg. generic-fri-bad-challenge-lists
        (composition-fri-algebraic-round-count dg) (bad dg)))
    s ≤ C
proof (rule wp-verify-monad-composition-generic-fri-bad-challenge-list-bound
  [OF future])
fix dg
have nnreal
  (card
    (generic-fri-bad-challenge-lists
      (composition-fri-algebraic-round-count dg) (bad dg))) /
  nnreal (CARD('f) ^ ceil-log (to-nat dg + 1))
  ≤ nnreal (bd dg) /
  nnreal (CARD('f) ^ ceil-log (to-nat dg + 1))
by (rule nnreal-nat-divide-right-mono[OF card-bound])
also have ... ≤ C
by (rule fraction-bound)
finally show nnreal
  (card
    (generic-fri-bad-challenge-lists
      (composition-fri-algebraic-round-count dg) (bad dg))) /
  nnreal (CARD('f) ^ ceil-log (to-nat dg + 1)) ≤ C .
qed

lemma wp-verify-monad-composition-generic-fri-bad-challenge-list-bound-from-rounds:
assumes future: composition-fri-future-fresh s

```

```

and local-bound:
   $\bigwedge dg\ i\ prefix.\ i < composition-fri-algebraic-round-count\ dg \implies$ 
     $length\ prefix = i \implies card\ (bad\ dg\ i\ prefix) \leq bd\ dg\ i$ 
and fraction-bound:
   $\bigwedge dg.\ nnreal$ 
     $(\sum i \in \{..<composition-fri-algebraic-round-count\ dg\}.$ 
       $CARD('f) \wedge i * bd\ dg\ i *$ 
       $CARD('f) \wedge (composition-fri-algebraic-round-count\ dg - Suc\ i)) /$ 
       $nnreal\ (CARD('f) \wedge ceil-log\ (to-nat\ dg + 1)) \leq C$ 
shows
  wp-event verify-monad
    (composition-fri-challenge-list-set-hit s
      ( $\lambda dg.\ generic-fri-bad-challenge-lists$ 
        (composition-fri-algebraic-round-count dg) (bad dg)))
     $s \leq C$ 
proof (rule
  wp-verify-monad-composition-generic-fri-bad-challenge-list-bound-from-card
  [OF future - fraction-bound])
fix dg
show card
  (generic-fri-bad-challenge-lists
    (composition-fri-algebraic-round-count dg) (bad dg))
   $\leq$ 
  ( $\sum i \in \{..<composition-fri-algebraic-round-count\ dg\}.$ 
     $CARD('f) \wedge i * bd\ dg\ i *$ 
     $CARD('f) \wedge (composition-fri-algebraic-round-count\ dg - Suc\ i))$ 
  by (rule generic-fri-bad-challenge-lists-card-bound-from-round-bounds)
    (rule local-bound[of - dg], simp-all)
qed

end

end

```

```

theory Soundness-FRI-Refined-Reachable
imports Soundness-FRI-Internal-Bounds
begin

```

Refined reachable FRI layer-chain targets.

Broad reachable partial-candidate events quantify full base tables compatible with sampled initial openings. Sampled base openings alone do not determine the committed FRI layer chain, so this theory introduces the narrower algebraic cover used by the next proof layer.

Its witnesses include the accepted FRI opening transcript and a canonical FRI layer chain. The cover is internal proof infrastructure; it does not change the protocol, verifier, Merkle interface, or public theorem assumptions.

```

context soundness

```

begin

definition *trace-fri-bad-with-refined-layer-chain*

$:: ('f, 'a) \text{ protocol-channel-scheme} \Rightarrow$
 $('f \text{ list} \Rightarrow \text{nat} \Rightarrow 'f \text{ list} \Rightarrow 'f \text{ set}) \Rightarrow$
 $(\text{unit list} \times ('f, 'a) \text{ protocol-channel-scheme}) \text{ option} \Rightarrow \text{bool}$

where

trace-fri-bad-with-refined-layer-chain $s \text{ bad out} \longleftrightarrow$
 $(\exists \text{ trace-roots trace-bs trace-final dg composition-roots}$
 $\text{composition-bs composition-final fri-query-idxs trace-round-layers}$
 $\text{composition-round-layers fr candidate-query-idxs trace-openings}$
 $\text{trace-table doms layers.}$
trace-fri-partial-candidate-opening-evidence $s \text{ out trace-roots}$
 $\text{trace-bs trace-final dg composition-roots composition-bs}$
 $\text{composition-final fri-query-idxs trace-round-layers}$
 $\text{composition-round-layers fr candidate-query-idxs trace-openings}$
 $\text{trace-table} \wedge$
generic-fri-canonical-layer-chain-evidence $\text{trace-table-low-degree}$
 $\text{trace-table (clength - 1) trace-roots trace-bs trace-final}$
 $\text{fri-query-idxs trace-round-layers doms layers (bad trace-table))}$

definition *composition-fri-bad-with-refined-layer-chain*

$:: ('f, 'a) \text{ protocol-channel-scheme} \Rightarrow$
 $('f \Rightarrow 'f \text{ list} \Rightarrow \text{nat} \Rightarrow 'f \text{ list} \Rightarrow 'f \text{ set}) \Rightarrow$
 $(\text{unit list} \times ('f, 'a) \text{ protocol-channel-scheme}) \text{ option} \Rightarrow \text{bool}$

where

composition-fri-bad-with-refined-layer-chain $s \text{ bad out} \longleftrightarrow$
 $(\exists \text{ trace-roots trace-bs trace-final fri-dg}$
 $\text{opening-composition-roots composition-bs composition-final}$
 $\text{fri-query-idxs trace-round-layers composition-round-layers fr}$
 $\text{f-fri-roots f-final as dg composition-fri-roots final}$
 $\text{trace-query-idxs trace-openings composition-query-idxs}$
 $\text{composition-openings trace-table composition-table doms layers.}$
composition-fri-partial-candidate-opening-evidence $s \text{ out trace-roots}$
 $\text{trace-bs trace-final fri-dg opening-composition-roots composition-bs}$
 $\text{composition-final fri-query-idxs trace-round-layers}$
 $\text{composition-round-layers fr f-fri-roots f-final as dg}$
 $\text{composition-fri-roots final trace-query-idxs trace-openings}$
 $\text{composition-query-idxs composition-openings trace-table}$
 $\text{composition-table} \wedge$
generic-fri-canonical-layer-chain-evidence
 $(\text{composition-table-low-degree (to-nat fri-dg)}) \text{ composition-table}$
 $(\text{to-nat fri-dg}) \text{ opening-composition-roots composition-bs}$
 $\text{composition-final fri-query-idxs composition-round-layers doms layers}$
 $(\text{bad fri-dg composition-table}))$

definition *trace-fri-reachable-missing-refined-layer-chain*

$:: ('f, 'a) \text{ protocol-channel-scheme} \Rightarrow$
 $('f \text{ list} \Rightarrow \text{nat} \Rightarrow 'f \text{ list} \Rightarrow 'f \text{ set}) \Rightarrow$

$(\text{unit list} \times ('f, 'a) \text{ protocol-channel-scheme}) \text{ option} \Rightarrow \text{bool}$
where
 $\text{trace-fri-reachable-missing-refined-layer-chain } s \text{ bad out} \longleftrightarrow$
 $\text{trace-fri-bad-with-reachable-partial-candidate } s \text{ out} \wedge$
 $\neg \text{trace-fri-bad-with-refined-layer-chain } s \text{ bad out}$

definition *composition-fri-reachable-missing-refined-layer-chain*
 $:: ('f, 'a) \text{ protocol-channel-scheme} \Rightarrow$
 $('f \Rightarrow 'f \text{ list} \Rightarrow \text{nat} \Rightarrow 'f \text{ list} \Rightarrow 'f \text{ set}) \Rightarrow$
 $(\text{unit list} \times ('f, 'a) \text{ protocol-channel-scheme}) \text{ option} \Rightarrow \text{bool}$
where
 $\text{composition-fri-reachable-missing-refined-layer-chain } s \text{ bad out}$
 $\longleftrightarrow$
 $\text{composition-fri-bad-with-reachable-partial-candidate } s \text{ out} \wedge$
 $\neg \text{composition-fri-bad-with-refined-layer-chain } s \text{ bad out}$

definition *trace-fri-bad-with-sampled-layer-chain*
 $:: ('f, 'a) \text{ protocol-channel-scheme} \Rightarrow$
 $(\text{unit list} \times ('f, 'a) \text{ protocol-channel-scheme}) \text{ option} \Rightarrow \text{bool}$
where
 $\text{trace-fri-bad-with-sampled-layer-chain } s \text{ out} \longleftrightarrow$
 $(\exists \text{ trace-roots trace-bs trace-final dg composition-roots}$
 $\text{composition-bs composition-final fri-query-idxs trace-round-layers}$
 $\text{composition-round-layers fr candidate-query-idxs trace-openings}$
 $\text{trace-table doms layers.}$
 $\text{trace-fri-partial-candidate-opening-evidence } s \text{ out trace-roots}$
 $\text{trace-bs trace-final dg composition-roots composition-bs}$
 $\text{composition-final fri-query-idxs trace-round-layers}$
 $\text{composition-round-layers fr candidate-query-idxs trace-openings}$
 $\text{trace-table} \wedge$
 $\text{generic-fri-sampled-layer-chain-evidence trace-table-low-degree}$
 $\text{trace-table (clength - 1) trace-roots trace-bs trace-final}$
 $\text{fri-query-idxs trace-round-layers doms layers})$

definition *composition-fri-bad-with-sampled-layer-chain*
 $:: ('f, 'a) \text{ protocol-channel-scheme} \Rightarrow$
 $(\text{unit list} \times ('f, 'a) \text{ protocol-channel-scheme}) \text{ option} \Rightarrow \text{bool}$
where

$\text{composition-fri-bad-with-sampled-layer-chain } s \text{ out} \longleftrightarrow$
 $(\exists \text{ trace-roots trace-bs trace-final fri-dg}$
 $\text{opening-composition-roots composition-bs composition-final}$
 $\text{fri-query-idxs trace-round-layers composition-round-layers fr}$
 $\text{f-fri-roots f-final as dg composition-fri-roots final}$
 $\text{trace-query-idxs trace-openings composition-query-idxs}$
 $\text{composition-openings trace-table composition-table doms layers.}$
 $\text{composition-fri-partial-candidate-opening-evidence } s \text{ out trace-roots}$
 $\text{trace-bs trace-final fri-dg opening-composition-roots composition-bs}$
 $\text{composition-final fri-query-idxs trace-round-layers}$
 $\text{composition-round-layers fr f-fri-roots f-final as dg}$

composition-fri-roots final trace-query-idxs trace-openings
composition-query-idxs composition-openings trace-table
composition-table $\wedge$
generic-fri-sampled-layer-chain-evidence
(composition-table-low-degree (to-nat fri-dg)) composition-table
(to-nat fri-dg) opening-composition-roots composition-bs
composition-final fri-query-idxs composition-round-layers doms layers)

definition *trace-fri-reachable-missing-sampled-layer-chain*

$:: ('f, 'a) \text{ protocol-channel-scheme} \Rightarrow$
 $(\text{unit list} \times ('f, 'a) \text{ protocol-channel-scheme}) \text{ option} \Rightarrow \text{bool}$

where

trace-fri-reachable-missing-sampled-layer-chain s out $\longleftrightarrow$
trace-fri-bad-with-reachable-partial-candidate s out $\wedge$
 $\neg \text{trace-fri-bad-with-sampled-layer-chain } s \text{ out}$

definition *composition-fri-reachable-missing-sampled-layer-chain*

$:: ('f, 'a) \text{ protocol-channel-scheme} \Rightarrow$
 $(\text{unit list} \times ('f, 'a) \text{ protocol-channel-scheme}) \text{ option} \Rightarrow \text{bool}$

where

composition-fri-reachable-missing-sampled-layer-chain s out $\longleftrightarrow$
composition-fri-bad-with-reachable-partial-candidate s out $\wedge$
 $\neg \text{composition-fri-bad-with-sampled-layer-chain } s \text{ out}$

definition *trace-fri-sampled-layer-chain-cover*

$:: ('f, 'a) \text{ protocol-channel-scheme} \Rightarrow$
 $('f \text{ list} \Rightarrow \text{nat} \Rightarrow 'f \text{ list} \Rightarrow 'f \text{ set}) \Rightarrow \text{bool}$

where

trace-fri-sampled-layer-chain-cover s bad $\longleftrightarrow$
 $(\forall \text{ out trace-roots trace-bs trace-final dg composition-roots}$
 $\text{composition-bs composition-final fri-query-idxs trace-round-layers}$
 $\text{composition-round-layers fr candidate-query-idxs trace-openings}$
 $\text{trace-table doms layers.}$
trace-fri-partial-candidate-opening-evidence s out trace-roots
 $\text{trace-bs trace-final dg composition-roots composition-bs}$
 $\text{composition-final fri-query-idxs trace-round-layers}$
 $\text{composition-round-layers fr candidate-query-idxs trace-openings}$
 $\text{trace-table} \longrightarrow$
generic-fri-sampled-layer-chain-evidence trace-table-low-degree
 $\text{trace-table (clength - 1) trace-roots trace-bs trace-final}$
 $\text{fri-query-idxs trace-round-layers doms layers} \longrightarrow$
 $\text{trace-bs} \in \text{trace-fri-multi-round-bad-sets bad trace-table})$

definition *composition-fri-sampled-layer-chain-cover*

$:: ('f, 'a) \text{ protocol-channel-scheme} \Rightarrow$
 $('f \Rightarrow 'f \text{ list} \Rightarrow \text{nat} \Rightarrow 'f \text{ list} \Rightarrow 'f \text{ set}) \Rightarrow \text{bool}$

where

composition-fri-sampled-layer-chain-cover s bad $\longleftrightarrow$
 $(\forall \text{ out trace-roots trace-bs trace-final fri-dg}$

*opening-composition-roots composition-bs composition-final
 fri-query-idxs trace-round-layers composition-round-layers fr
 f-fri-roots f-final as dg composition-fri-roots final
 trace-query-idxs trace-openings composition-query-idxs
 composition-openings trace-table composition-table doms layers.
 composition-fri-partial-candidate-opening-evidence s out trace-roots
 trace-bs trace-final fri-dg opening-composition-roots composition-bs
 composition-final fri-query-idxs trace-round-layers
 composition-round-layers fr f-fri-roots f-final as dg
 composition-fri-roots final trace-query-idxs trace-openings
 composition-query-idxs composition-openings trace-table
 composition-table $\longrightarrow$
 generic-fri-sampled-layer-chain-evidence
 (composition-table-low-degree (to-nat fri-dg)) composition-table
 (to-nat fri-dg) opening-composition-roots composition-bs
 composition-final fri-query-idxs composition-round-layers doms layers
 $\longrightarrow$
 composition-bs $\in$
 composition-fri-multi-round-bad-sets bad fri-dg composition-table)*

definition *trace-fri-bad-with-full-cover-layer-chain*

*:: (f, 'a) protocol-channel-scheme $\Rightarrow$
 (f list $\Rightarrow$ nat $\Rightarrow$ 'f list $\Rightarrow$ 'f set) $\Rightarrow$
 (unit list $\times$ (f, 'a) protocol-channel-scheme) option $\Rightarrow$ bool*

where

*trace-fri-bad-with-full-cover-layer-chain s bad out $\longleftrightarrow$
 ($\exists$ trace-roots trace-bs trace-final dg composition-roots
 composition-bs composition-final fri-query-idxs trace-round-layers
 composition-round-layers fr candidate-query-idxs trace-openings
 trace-table doms layers.
 trace-fri-partial-candidate-opening-evidence s out trace-roots
 trace-bs trace-final dg composition-roots composition-bs
 composition-final fri-query-idxs trace-round-layers
 composition-round-layers fr candidate-query-idxs trace-openings
 trace-table $\wedge$
 generic-fri-sampled-layer-chain-full-cover trace-table-low-degree
 trace-table (clength - 1) trace-roots trace-bs trace-final
 fri-query-idxs trace-round-layers doms layers (bad trace-table))*

definition *composition-fri-bad-with-full-cover-layer-chain*

*:: (f, 'a) protocol-channel-scheme $\Rightarrow$
 (f $\Rightarrow$ 'f list $\Rightarrow$ nat $\Rightarrow$ 'f list $\Rightarrow$ 'f set) $\Rightarrow$
 (unit list $\times$ (f, 'a) protocol-channel-scheme) option $\Rightarrow$ bool*

where

*composition-fri-bad-with-full-cover-layer-chain s bad out $\longleftrightarrow$
 ($\exists$ trace-roots trace-bs trace-final fri-dg
 opening-composition-roots composition-bs composition-final
 fri-query-idxs trace-round-layers composition-round-layers fr
 f-fri-roots f-final as dg composition-fri-roots final*

trace-query-idxs trace-openings composition-query-idxs
composition-openings trace-table composition-table doms layers.
composition-fri-partial-candidate-opening-evidence s out trace-roots
trace-bs trace-final fri-dg opening-composition-roots composition-bs
composition-final fri-query-idxs trace-round-layers
composition-round-layers fr f-fri-roots f-final as dg
composition-fri-roots final trace-query-idxs trace-openings
composition-query-idxs composition-openings trace-table
composition-table $\wedge$
generic-fri-sampled-layer-chain-full-cover
(composition-table-low-degree (to-nat fri-dg)) composition-table
(to-nat fri-dg) opening-composition-roots composition-bs
composition-final fri-query-idxs composition-round-layers doms layers
(bad fri-dg composition-table))

definition *trace-fri-sampled-missing-full-cover*

$:: ('f, 'a) \text{ protocol-channel-scheme} \Rightarrow$
 $('f \text{ list} \Rightarrow \text{nat} \Rightarrow 'f \text{ list} \Rightarrow 'f \text{ set}) \Rightarrow$
 $(\text{unit list} \times ('f, 'a) \text{ protocol-channel-scheme}) \text{ option} \Rightarrow \text{bool}$

where

trace-fri-sampled-missing-full-cover s bad out $\longleftrightarrow$
trace-fri-bad-with-sampled-layer-chain s out $\wedge$
 $\neg \text{ trace-fri-bad-with-full-cover-layer-chain s bad out}$

definition *composition-fri-sampled-missing-full-cover*

$:: ('f, 'a) \text{ protocol-channel-scheme} \Rightarrow$
 $('f \Rightarrow 'f \text{ list} \Rightarrow \text{nat} \Rightarrow 'f \text{ list} \Rightarrow 'f \text{ set}) \Rightarrow$
 $(\text{unit list} \times ('f, 'a) \text{ protocol-channel-scheme}) \text{ option} \Rightarrow \text{bool}$

where

composition-fri-sampled-missing-full-cover s bad out $\longleftrightarrow$
composition-fri-bad-with-sampled-layer-chain s out $\wedge$
 $\neg \text{ composition-fri-bad-with-full-cover-layer-chain s bad out}$

lemma *trace-fri-sampled-layer-chain-cover-from-full-cover:*

fixes $s :: ('f, 'a) \text{ protocol-channel-scheme}$

fixes $\text{bad} :: 'f \text{ list} \Rightarrow \text{nat} \Rightarrow 'f \text{ list} \Rightarrow 'f \text{ set}$

assumes *full-cover:*

$\wedge \text{out trace-roots trace-bs trace-final dg composition-roots}$
 $\text{composition-bs composition-final fri-query-idxs trace-round-layers}$
 $\text{composition-round-layers fr candidate-query-idxs trace-openings}$
 $\text{trace-table doms layers.}$
 $\text{trace-fri-partial-candidate-opening-evidence s out trace-roots}$
 $\text{trace-bs trace-final dg composition-roots composition-bs}$
 $\text{composition-final fri-query-idxs trace-round-layers}$
 $\text{composition-round-layers fr candidate-query-idxs trace-openings}$
 $\text{trace-table} \implies$
 $\text{generic-fri-sampled-layer-chain-evidence trace-table-low-degree}$
 $\text{trace-table (clength - 1) trace-roots trace-bs trace-final}$
 $\text{fri-query-idxs trace-round-layers doms layers} \implies$

```

    generic-fri-sampled-layer-chain-full-cover trace-table-low-degree
      trace-table (clength - 1) trace-roots trace-bs trace-final
      fri-query-idxs trace-round-layers doms layers (bad trace-table)
  shows trace-fri-sampled-layer-chain-cover s bad
proof (unfold trace-fri-sampled-layer-chain-cover-def, intro allI impI)
  fix out trace-roots trace-bs trace-final dg composition-roots
    composition-bs composition-final fri-query-idxs trace-round-layers
    composition-round-layers fr candidate-query-idxs trace-openings
    trace-table doms layers
  assume evidence:
    trace-fri-partial-candidate-opening-evidence s out trace-roots
    trace-bs trace-final dg composition-roots composition-bs
    composition-final fri-query-idxs trace-round-layers
    composition-round-layers fr candidate-query-idxs trace-openings
    trace-table
  assume chain:
    generic-fri-sampled-layer-chain-evidence trace-table-low-degree
      trace-table (clength - 1) trace-roots trace-bs trace-final
      fri-query-idxs trace-round-layers doms layers
  have full:
    generic-fri-sampled-layer-chain-full-cover trace-table-low-degree
      trace-table (clength - 1) trace-roots trace-bs trace-final
      fri-query-idxs trace-round-layers doms layers (bad trace-table)
  by (rule full-cover[OF evidence chain])
  have not-low:  $\neg$  trace-table-low-degree trace-table
  using trace-fri-partial-candidate-opening-evidenceD(2)[OF evidence]
    trace-fri-partial-candidate-evidenceD(3)
  by blast
  have bad-candidate:
    generic-fri-bad-candidate trace-table-low-degree trace-table
      (clength - 1) trace-roots trace-bs trace-final fri-query-idxs
      trace-round-layers (bad trace-table)
  by (rule generic-fri-bad-candidate-from-sampled-layer-chain-full-cover
    [OF full not-low])
  have bad-generic:
    trace-bs  $\in$  generic-fri-bad-challenge-lists (length trace-bs)
    (bad trace-table)
  by (rule generic-fri-bad-candidateD(3)[OF bad-candidate])
  have partial:
    generic-fri-partial-evidence trace-table-low-degree trace-table
      (clength - 1) trace-roots trace-bs trace-final fri-query-idxs
      trace-round-layers
  by (rule generic-fri-bad-candidateD(1)[OF bad-candidate])
  have len-trace:
    length trace-bs = trace-fri-algebraic-round-count
  using generic-fri-partial-evidence-shapes(1,2)[OF partial]
  by (simp add: trace-fri-algebraic-round-count-def)
  show trace-bs  $\in$  trace-fri-multiround-bad-sets bad trace-table
  using bad-generic len-trace

```

by (simp add: trace-fri-multiround-bad-sets-as-generic)
qed

lemma *composition-fri-sampled-layer-chain-cover-from-full-cover:*
fixes $s :: ('f, 'a) \text{ protocol-channel-scheme}$
fixes $\text{bad} :: 'f \Rightarrow 'f \text{ list} \Rightarrow \text{nat} \Rightarrow 'f \text{ list} \Rightarrow 'f \text{ set}$
assumes *full-cover:*
 $\bigwedge \text{out trace-roots trace-bs trace-final fri-dg}$
 $\text{opening-composition-roots composition-bs composition-final}$
 $\text{fri-query-idxs trace-round-layers composition-round-layers fr}$
 $\text{f-fri-roots f-final as dg composition-fri-roots final}$
 $\text{trace-query-idxs trace-openings composition-query-idxs}$
 $\text{composition-openings trace-table composition-table doms layers.}$
 $\text{composition-fri-partial-candidate-opening-evidence } s \text{ out trace-roots}$
 $\text{trace-bs trace-final fri-dg opening-composition-roots composition-bs}$
 $\text{composition-final fri-query-idxs trace-round-layers}$
 $\text{composition-round-layers fr f-fri-roots f-final as dg}$
 $\text{composition-fri-roots final trace-query-idxs trace-openings}$
 $\text{composition-query-idxs composition-openings trace-table}$
 $\text{composition-table} \implies$
generic-fri-sampled-layer-chain-evidence
 $(\text{composition-table-low-degree } (\text{to-nat fri-dg})) \text{ composition-table}$
 $(\text{to-nat fri-dg}) \text{ opening-composition-roots composition-bs}$
 $\text{composition-final fri-query-idxs composition-round-layers doms}$
 $\text{layers} \implies$
generic-fri-sampled-layer-chain-full-cover
 $(\text{composition-table-low-degree } (\text{to-nat fri-dg})) \text{ composition-table}$
 $(\text{to-nat fri-dg}) \text{ opening-composition-roots composition-bs}$
 $\text{composition-final fri-query-idxs composition-round-layers doms}$
 $\text{layers } (\text{bad fri-dg composition-table})$
shows *composition-fri-sampled-layer-chain-cover s bad*
proof (unfold *composition-fri-sampled-layer-chain-cover-def*,
intro allI impI)
fix $\text{out trace-roots trace-bs trace-final fri-dg}$
 $\text{opening-composition-roots composition-bs composition-final}$
 $\text{fri-query-idxs trace-round-layers composition-round-layers fr}$
 $\text{f-fri-roots f-final as dg composition-fri-roots final}$
 $\text{trace-query-idxs trace-openings composition-query-idxs}$
 $\text{composition-openings trace-table composition-table doms layers}$
assume *evidence:*
 $\text{composition-fri-partial-candidate-opening-evidence } s \text{ out trace-roots}$
 $\text{trace-bs trace-final fri-dg opening-composition-roots composition-bs}$
 $\text{composition-final fri-query-idxs trace-round-layers}$
 $\text{composition-round-layers fr f-fri-roots f-final as dg}$
 $\text{composition-fri-roots final trace-query-idxs trace-openings}$
 $\text{composition-query-idxs composition-openings trace-table}$
 composition-table
assume *chain:*
generic-fri-sampled-layer-chain-evidence

```

    (composition-table-low-degree (to-nat fri-dg)) composition-table
    (to-nat fri-dg) opening-composition-roots composition-bs
    composition-final fri-query-idxs composition-round-layers doms
    layers
  have full:
    generic-fri-sampled-layer-chain-full-cover
    (composition-table-low-degree (to-nat fri-dg)) composition-table
    (to-nat fri-dg) opening-composition-roots composition-bs
    composition-final fri-query-idxs composition-round-layers doms
    layers (bad fri-dg composition-table)
  by (rule full-cover[OF evidence chain])
  have not-low-on:
    ¬ fri-table-low-degree-on (to-nat fri-dg) eval-domain
    composition-table
  using full unfolding generic-fri-sampled-layer-chain-full-cover-def
  by blast
  have not-low:
    ¬ composition-table-low-degree (to-nat fri-dg) composition-table
  using not-low-on
    composition-table-not-low-degree-iff-not-fri-table-low-degree-on-eval-domain
  by simp
  have bad-candidate:
    generic-fri-bad-candidate
    (composition-table-low-degree (to-nat fri-dg)) composition-table
    (to-nat fri-dg) opening-composition-roots composition-bs
    composition-final fri-query-idxs composition-round-layers
    (bad fri-dg composition-table)
  by (rule generic-fri-bad-candidate-from-sampled-layer-chain-full-cover
    [OF full not-low])
  have bad-generic:
    composition-bs ∈ generic-fri-bad-challenge-lists
    (length composition-bs) (bad fri-dg composition-table)
  by (rule generic-fri-bad-candidateD(3)[OF bad-candidate])
  have partial:
    generic-fri-partial-evidence
    (composition-table-low-degree (to-nat fri-dg)) composition-table
    (to-nat fri-dg) opening-composition-roots composition-bs
    composition-final fri-query-idxs composition-round-layers
  by (rule generic-fri-bad-candidateD(1)[OF bad-candidate])
  have len-composition:
    length composition-bs = composition-fri-algebraic-round-count fri-dg
  using generic-fri-partial-evidence-shapes(1,2)[OF partial]
  by (simp add: composition-fri-algebraic-round-count-def)
  show composition-bs ∈
    composition-fri-multi-round-bad-sets bad fri-dg composition-table
  using bad-generic len-composition
  by (simp add: composition-fri-multi-round-bad-sets-as-generic)
qed

```

lemma *trace-fri-bad-with-full-cover-layer-chain-imp-list-hit*:
assumes *full*:
trace-fri-bad-with-full-cover-layer-chain s bad out
and *envelope*:
 $\bigwedge \text{trace-table. trace-fri-multiround-bad-sets bad trace-table} \subseteq B$
shows *trace-fri-challenge-list-set-hit s B out*
proof –
from *full* **obtain** *trace-roots trace-bs trace-final dg composition-roots*
composition-bs composition-final fri-query-idxs trace-round-layers
composition-round-layers fr candidate-query-idxs trace-openings
trace-table doms layers
where *evidence*:
trace-fri-partial-candidate-opening-evidence s out trace-roots
trace-bs trace-final dg composition-roots composition-bs
composition-final fri-query-idxs trace-round-layers
composition-round-layers fr candidate-query-idxs trace-openings
trace-table
and *full-cover*:
generic-fri-sampled-layer-chain-full-cover trace-table-low-degree
trace-table (clength – 1) trace-roots trace-bs trace-final
fri-query-idxs trace-round-layers doms layers (bad trace-table)
unfolding *trace-fri-bad-with-full-cover-layer-chain-def* **by** *blast*
have *bad-generic*:
trace-bs $\in$ *generic-fri-bad-challenge-lists (length trace-bs)*
(bad trace-table)
proof –
have *not-low*: $\neg$ *trace-table-low-degree trace-table*
using *trace-fri-partial-candidate-opening-evidenceD(2)[OF evidence]*
trace-fri-partial-candidate-evidenceD(3)
by *blast*
have *bad-candidate*:
generic-fri-bad-candidate trace-table-low-degree trace-table
(clength – 1) trace-roots trace-bs trace-final fri-query-idxs
trace-round-layers (bad trace-table)
by (rule *generic-fri-bad-candidate-from-sampled-layer-chain-full-cover*
 $[OF \text{ full-cover not-low}]$)
show *?thesis*
by (rule *generic-fri-bad-candidateD(3)[OF bad-candidate]*)
qed
have *partial*:
generic-fri-partial-evidence trace-table-low-degree trace-table
(clength – 1) trace-roots trace-bs trace-final fri-query-idxs
trace-round-layers
proof –
have *not-low*: $\neg$ *trace-table-low-degree trace-table*
using *trace-fri-partial-candidate-opening-evidenceD(2)[OF evidence]*
trace-fri-partial-candidate-evidenceD(3)
by *blast*
have *bad-candidate*:

```

generic-fri-bad-candidate trace-table-low-degree trace-table
  (clength - 1) trace-roots trace-bs trace-final fri-query-idxs
  trace-round-layers (bad trace-table)
by (rule generic-fri-bad-candidate-from-sampled-layer-chain-full-cover
  [OF full-cover not-low])
show ?thesis
  by (rule generic-fri-bad-candidateD(1)[OF bad-candidate])
qed
have len-trace:
  length trace-bs = trace-fri-algebraic-round-count
  using generic-fri-partial-evidence-shapes(1,2)[OF partial]
  by (simp add: trace-fri-algebraic-round-count-def)
have bad-hit: trace-bs ∈ trace-fri-multiround-bad-sets bad trace-table
  using bad-generic len-trace
  by (simp add: trace-fri-multiround-bad-sets-as-generic)
have challenges:
  accepted-fri-challenges s out trace-bs dg composition-bs
  using trace-fri-partial-candidate-opening-evidenceD(1)[OF evidence]
  accepted-fri-opening-transcript-challenges by blast
have trace-bs ∈ B
  using bad-hit envelope[of trace-table] by blast
then show ?thesis
  unfolding trace-fri-challenge-list-set-hit-def
  using challenges by blast
qed

lemma composition-fri-bad-with-full-cover-layer-chain-imp-list-hit:
assumes full:
  composition-fri-bad-with-full-cover-layer-chain s bad out
and envelope:
   $\bigwedge dg$  composition-table.
  composition-fri-multiround-bad-sets bad dg composition-table
   $\subseteq B dg$ 
shows composition-fri-challenge-list-set-hit s B out
proof -
from full obtain trace-roots trace-bs trace-final fri-dg
  opening-composition-roots composition-bs composition-final
  fri-query-idxs trace-round-layers composition-round-layers fr
  f-fri-roots f-final as dg composition-fri-roots final
  trace-query-idxs trace-openings composition-query-idxs
  composition-openings trace-table composition-table doms layers
where evidence:
  composition-fri-partial-candidate-opening-evidence s out trace-roots
  trace-bs trace-final fri-dg opening-composition-roots composition-bs
  composition-final fri-query-idxs trace-round-layers
  composition-round-layers fr f-fri-roots f-final as dg
  composition-fri-roots final trace-query-idxs trace-openings
  composition-query-idxs composition-openings trace-table
  composition-table

```



```

and full-cover:
  generic-fri-sampled-layer-chain-full-cover
    (composition-table-low-degree (to-nat fri-dg)) composition-table
    (to-nat fri-dg) opening-composition-roots composition-bs
    composition-final fri-query-idxs composition-round-layers doms layers
    (bad fri-dg composition-table)
unfolding composition-fri-bad-with-full-cover-layer-chain-def
by auto
have bad-generic:
  composition-bs ∈ generic-fri-bad-challenge-lists
    (length composition-bs) (bad fri-dg composition-table)
proof –
  have not-low-on:
    ¬ fri-table-low-degree-on (to-nat fri-dg) eval-domain
      composition-table
    using full-cover
    unfolding generic-fri-sampled-layer-chain-full-cover-def by blast
  have not-low:
    ¬ composition-table-low-degree (to-nat fri-dg) composition-table
    using not-low-on
      composition-table-not-low-degree-iff-not-fri-table-low-degree-on-eval-domain
    by simp
  have bad-candidate:
    generic-fri-bad-candidate
      (composition-table-low-degree (to-nat fri-dg)) composition-table
      (to-nat fri-dg) opening-composition-roots composition-bs
      composition-final fri-query-idxs composition-round-layers
      (bad fri-dg composition-table)
    by (rule generic-fri-bad-candidate-from-sampled-layer-chain-full-cover
      [OF full-cover not-low])
  show ?thesis
    by (rule generic-fri-bad-candidateD(?)[OF bad-candidate])
qed
have partial:
  generic-fri-partial-evidence
    (composition-table-low-degree (to-nat fri-dg)) composition-table
    (to-nat fri-dg) opening-composition-roots composition-bs
    composition-final fri-query-idxs composition-round-layers
proof –
  have not-low-on:
    ¬ fri-table-low-degree-on (to-nat fri-dg) eval-domain
      composition-table
    using full-cover
    unfolding generic-fri-sampled-layer-chain-full-cover-def by blast
  have not-low:
    ¬ composition-table-low-degree (to-nat fri-dg) composition-table
    using not-low-on
      composition-table-not-low-degree-iff-not-fri-table-low-degree-on-eval-domain
    by simp

```

```

have bad-candidate:
  generic-fri-bad-candidate
    (composition-table-low-degree (to-nat fri-dg)) composition-table
    (to-nat fri-dg) opening-composition-roots composition-bs
    composition-final fri-query-idxs composition-round-layers
    (bad fri-dg composition-table)
  by (rule generic-fri-bad-candidate-from-sampled-layer-chain-full-cover
    [OF full-cover not-low])
show ?thesis
  by (rule generic-fri-bad-candidateD(1)[OF bad-candidate])
qed
have len-composition:
  length composition-bs = composition-fri-algebraic-round-count fri-dg
  using generic-fri-partial-evidence-shapes(1,2)[OF partial]
  by (simp add: composition-fri-algebraic-round-count-def)
have bad-hit:
  composition-bs ∈
    composition-fri-multiround-bad-sets bad fri-dg composition-table
  using bad-generic len-composition
  by (simp add: composition-fri-multiround-bad-sets-as-generic)
have challenges:
  accepted-fri-challenges s out trace-bs fri-dg composition-bs
  using composition-fri-partial-candidate-opening-evidenceD(1)[OF evidence]
  accepted-fri-opening-transcript-challenges by blast
have composition-bs ∈ B fri-dg
  using bad-hit envelope[of fri-dg composition-table] by blast
then show ?thesis
  unfolding composition-fri-challenge-list-set-hit-def
  using challenges by blast
qed

lemma trace-fri-sampled-imp-full-cover-or-missing:
  assumes trace-fri-bad-with-sampled-layer-chain s out
  shows
    trace-fri-bad-with-full-cover-layer-chain s bad out ∨
    trace-fri-sampled-missing-full-cover s bad out
  using assms unfolding trace-fri-sampled-missing-full-cover-def by blast

lemma composition-fri-sampled-imp-full-cover-or-missing:
  assumes composition-fri-bad-with-sampled-layer-chain s out
  shows
    composition-fri-bad-with-full-cover-layer-chain s bad out ∨
    composition-fri-sampled-missing-full-cover s bad out
  using assms unfolding composition-fri-sampled-missing-full-cover-def
  by blast

lemma wp-trace-fri-bad-with-full-cover-layer-chain-bound-from-envelope:
  assumes future: trace-fri-future-fresh s
  and envelope:

```

$\bigwedge \text{trace-table. trace-fri-multiround-bad-sets bad trace-table} \subseteq B$
and *subset*: $B \subseteq \text{fri-challenge-space (ceil-log clength)}$
and *bound*:
 $\text{nnreal (card B) / nnreal (CARD('f) } \wedge \text{ ceil-log clength)} \leq C$
shows
 $\text{wp-event verify-monad (trace-fri-bad-with-full-cover-layer-chain s bad) s} \leq C$
proof –
have *wp-event verify-monad*
 $\text{(trace-fri-bad-with-full-cover-layer-chain s bad) s} \leq$
 $\text{wp-event verify-monad (trace-fri-challenge-list-set-hit s B) s}$
by (*rule wp-event-mono*)
 $\text{(rule trace-fri-bad-with-full-cover-layer-chain-imp-list-hit [OF - envelope])}$
also have $\dots \leq \text{nnreal (card B) / nnreal (CARD('f) } \wedge \text{ ceil-log clength)}$
by (*rule wp-verify-monad-trace-fri-challenge-list-set-bound [OF future subset]*)
also have $\dots \leq C$
by (*rule bound*)
finally show *?thesis* .
qed

lemma *wp-composition-fri-bad-with-full-cover-layer-chain-bound-from-envelope*:
assumes *future: composition-fri-future-fresh s*
and *envelope*:
 $\bigwedge \text{dg composition-table. composition-fri-multiround-bad-sets bad dg composition-table} \subseteq B \text{ dg}$
and *subset*:
 $\bigwedge \text{dg. } B \text{ dg} \subseteq \text{fri-challenge-space (ceil-log (to-nat dg + 1))}$
and *bound*:
 $\bigwedge \text{dg. nnreal (card (B dg)) / nnreal (CARD('f) } \wedge \text{ ceil-log (to-nat dg + 1))} \leq C$
shows
 $\text{wp-event verify-monad (composition-fri-bad-with-full-cover-layer-chain s bad) s} \leq C$
proof –
have *wp-event verify-monad*
 $\text{(composition-fri-bad-with-full-cover-layer-chain s bad) s} \leq$
 $\text{wp-event verify-monad (composition-fri-challenge-list-set-hit s B) s}$
by (*rule wp-event-mono*)
 $\text{(rule composition-fri-bad-with-full-cover-layer-chain-imp-list-hit [OF - envelope])}$
also have $\dots \leq C$
by (*rule wp-verify-monad-composition-fri-challenge-list-set-bound [OF future subset bound]*)
finally show *?thesis* .
qed

lemma *wp-trace-fri-sampled-bound-from-full-cover-and-missing:*

assumes *full-bound:*

wp-event verify-monad

(trace-fri-bad-with-full-cover-layer-chain s bad) s ≤ R

and *missing-bound:*

wp-event verify-monad

(trace-fri-sampled-missing-full-cover s bad) s ≤ M

shows

wp-event verify-monad

(trace-fri-bad-with-sampled-layer-chain s) s ≤ R + M

proof –

have *wp-event verify-monad*

(trace-fri-bad-with-sampled-layer-chain s) s ≤

wp-event verify-monad

*(λout. trace-fri-bad-with-full-cover-layer-chain s bad out ∨
trace-fri-sampled-missing-full-cover s bad out) s*

by *(rule wp-event-mono)*

(rule trace-fri-sampled-imp-full-cover-or-missing)

also have *... ≤*

wp-event verify-monad

(trace-fri-bad-with-full-cover-layer-chain s bad) s +

wp-event verify-monad

(trace-fri-sampled-missing-full-cover s bad) s

by *(rule wp-event-union-bound)*

also have *... ≤ R + M*

by *(intro add-mono full-bound missing-bound)*

finally show *?thesis .*

qed

lemma *wp-composition-fri-sampled-bound-from-full-cover-and-missing:*

assumes *full-bound:*

wp-event verify-monad

(composition-fri-bad-with-full-cover-layer-chain s bad) s ≤ R

and *missing-bound:*

wp-event verify-monad

(composition-fri-sampled-missing-full-cover s bad) s ≤ M

shows

wp-event verify-monad

(composition-fri-bad-with-sampled-layer-chain s) s ≤ R + M

proof –

have *wp-event verify-monad*

(composition-fri-bad-with-sampled-layer-chain s) s ≤

wp-event verify-monad

*(λout. composition-fri-bad-with-full-cover-layer-chain s bad out ∨
composition-fri-sampled-missing-full-cover s bad out) s*

by *(rule wp-event-mono)*

(rule composition-fri-sampled-imp-full-cover-or-missing)

also have *... ≤*

wp-event verify-monad
 (composition-fri-bad-with-full-cover-layer-chain *s bad*) *s* +
wp-event verify-monad
 (composition-fri-sampled-missing-full-cover *s bad*) *s*
 by (rule *wp-event-union-bound*)
 also have ... $\leq R + M$
 by (intro *add-mono full-bound missing-bound*)
 finally show ?thesis .
 qed

definition *trace-fri-canonical-layer-chain-cover*

:: (*f*, *a*) protocol-channel-scheme $\Rightarrow$
 (*f list* $\Rightarrow$ nat $\Rightarrow$ *f list* $\Rightarrow$ *f set*) $\Rightarrow$ bool

where

trace-fri-canonical-layer-chain-cover s bad $\longleftrightarrow$
 ($\forall$ out trace-roots trace-bs trace-final dg composition-roots
 composition-bs composition-final fri-query-idxs trace-round-layers
 composition-round-layers fr candidate-query-idxs trace-openings
 trace-table doms layers.
trace-fri-partial-candidate-opening-evidence s out trace-roots
 trace-bs trace-final dg composition-roots composition-bs
 composition-final fri-query-idxs trace-round-layers
 composition-round-layers fr candidate-query-idxs trace-openings
 trace-table $\longrightarrow$
generic-fri-canonical-layer-chain-evidence trace-table-low-degree
 trace-table (clength - 1) trace-roots trace-bs trace-final
 fri-query-idxs trace-round-layers doms layers (bad trace-table)
 $\longrightarrow$
 trace-bs $\in$ trace-fri-multiround-bad-sets bad trace-table)

definition *composition-fri-canonical-layer-chain-cover*

:: (*f*, *a*) protocol-channel-scheme $\Rightarrow$
 (*f* $\Rightarrow$ *f list* $\Rightarrow$ nat $\Rightarrow$ *f list* $\Rightarrow$ *f set*) $\Rightarrow$ bool

where

composition-fri-canonical-layer-chain-cover s bad $\longleftrightarrow$
 ($\forall$ out trace-roots trace-bs trace-final fri-dg
 opening-composition-roots composition-bs composition-final
 fri-query-idxs trace-round-layers composition-round-layers fr
 f-fri-roots f-final as dg composition-fri-roots final
 trace-query-idxs trace-openings composition-query-idxs
 composition-openings trace-table composition-table doms layers.
composition-fri-partial-candidate-opening-evidence s out trace-roots
 trace-bs trace-final fri-dg opening-composition-roots composition-bs
 composition-final fri-query-idxs trace-round-layers
 composition-round-layers fr f-fri-roots f-final as dg
 composition-fri-roots final trace-query-idxs trace-openings
 composition-query-idxs composition-openings trace-table
 composition-table $\longrightarrow$
generic-fri-canonical-layer-chain-evidence

$(\text{composition-table-low-degree } (\text{to-nat fri-dg})) \text{ composition-table}$
 $(\text{to-nat fri-dg}) \text{ opening-composition-roots composition-bs}$
 $\text{composition-final fri-query-idxs composition-round-layers doms layers}$
 $(\text{bad fri-dg composition-table}) \longrightarrow$
 $\text{composition-bs} \in$
 $\text{composition-fri-multiround-bad-sets bad fri-dg composition-table})$

lemma *trace-fri-canonical-layer-chain-cover-generic:*

fixes $s :: ('f, 'a) \text{ protocol-channel-scheme}$

fixes $\text{bad} :: 'f \text{ list} \Rightarrow \text{nat} \Rightarrow 'f \text{ list} \Rightarrow 'f \text{ set}$

shows *trace-fri-canonical-layer-chain-cover s bad*

proof (*unfold trace-fri-canonical-layer-chain-cover-def, intro allI impI*)

fix $\text{out trace-roots trace-bs trace-final dg composition-roots}$
 $\text{composition-bs composition-final fri-query-idxs trace-round-layers}$
 $\text{composition-round-layers fr candidate-query-idxs trace-openings}$
 $\text{trace-table doms layers}$

assume *evidence:*

$\text{trace-fri-partial-candidate-opening-evidence } s \text{ out trace-roots}$
 $\text{trace-bs trace-final dg composition-roots composition-bs}$
 $\text{composition-final fri-query-idxs trace-round-layers}$
 $\text{composition-round-layers fr candidate-query-idxs trace-openings}$
 trace-table

assume *chain:*

$\text{generic-fri-canonical-layer-chain-evidence trace-table-low-degree}$
 $\text{trace-table } (\text{clength} - 1) \text{ trace-roots trace-bs trace-final}$
 $\text{fri-query-idxs trace-round-layers doms layers } (\text{bad trace-table})$

have *bad-generic:*

$\text{trace-bs} \in \text{generic-fri-bad-challenge-lists } (\text{length trace-bs})$
 (bad trace-table)

by (*rule generic-fri-canonical-layer-chain-bad-challenge-list[OF chain]*)

have *partial:*

$\text{generic-fri-partial-evidence trace-table-low-degree trace-table}$
 $(\text{clength} - 1) \text{ trace-roots trace-bs trace-final fri-query-idxs}$
 $\text{trace-round-layers}$

by (*rule generic-fri-canonical-layer-chain-evidenceD(1)[OF chain]*)

have *len-trace:*

$\text{length trace-bs} = \text{trace-fri-algebraic-round-count}$
using *generic-fri-partial-evidence-shapes(1,2)[OF partial]*
by (*simp add: trace-fri-algebraic-round-count-def*)

show $\text{trace-bs} \in \text{trace-fri-multiround-bad-sets bad trace-table}$

using *bad-generic len-trace*

by (*simp add: trace-fri-multiround-bad-sets-as-generic*)

qed

lemma *composition-fri-canonical-layer-chain-cover-generic:*

fixes $s :: ('f, 'a) \text{ protocol-channel-scheme}$

fixes $\text{bad} :: 'f \Rightarrow 'f \text{ list} \Rightarrow \text{nat} \Rightarrow 'f \text{ list} \Rightarrow 'f \text{ set}$

shows *composition-fri-canonical-layer-chain-cover s bad*

proof (*unfold composition-fri-canonical-layer-chain-cover-def,*

intro allI impI)
fix *out trace-roots trace-bs trace-final fri-dg*
opening-composition-roots composition-bs composition-final
fri-query-idxs trace-round-layers composition-round-layers fr
f-fri-roots f-final as dg composition-fri-roots final
trace-query-idxs trace-openings composition-query-idxs
composition-openings trace-table composition-table doms layers
assume *evidence:*
composition-fri-partial-candidate-opening-evidence s out trace-roots
trace-bs trace-final fri-dg opening-composition-roots composition-bs
composition-final fri-query-idxs trace-round-layers
composition-round-layers fr f-fri-roots f-final as dg
composition-fri-roots final trace-query-idxs trace-openings
composition-query-idxs composition-openings trace-table
composition-table
assume *chain:*
generic-fri-canonical-layer-chain-evidence
(composition-table-low-degree (to-nat fri-dg)) composition-table
(to-nat fri-dg) opening-composition-roots composition-bs
composition-final fri-query-idxs composition-round-layers doms layers
(bad fri-dg composition-table)
have *bad-generic:*
composition-bs ∈ generic-fri-bad-challenge-lists
(length composition-bs) (bad fri-dg composition-table)
by *(rule generic-fri-canonical-layer-chain-bad-challenge-list[OF chain])*
have *partial:*
generic-fri-partial-evidence
(composition-table-low-degree (to-nat fri-dg)) composition-table
(to-nat fri-dg) opening-composition-roots composition-bs
composition-final fri-query-idxs composition-round-layers
by *(rule generic-fri-canonical-layer-chain-evidenceD(1)[OF chain])*
have *len-composition:*
length composition-bs = composition-fri-algebraic-round-count fri-dg
using *generic-fri-partial-evidence-shapes(1,2)[OF partial]*
by *(simp add: composition-fri-algebraic-round-count-def)*
show *composition-bs ∈*
composition-fri-multiround-bad-sets bad fri-dg composition-table
using *bad-generic len-composition*
by *(simp add: composition-fri-multiround-bad-sets-as-generic)*
qed

lemma *trace-fri-bad-with-refined-layer-chain-imp-list-hit:*
assumes *refined:*
trace-fri-bad-with-refined-layer-chain s bad out
and *envelope:*
 $\bigwedge \text{trace-table. trace-fri-multiround-bad-sets bad trace-table} \subseteq B$
shows *trace-fri-challenge-list-set-hit s B out*
proof —
from *refined obtain* *trace-roots trace-bs trace-final dg composition-roots*

composition-bs composition-final fri-query-idxs trace-round-layers
composition-round-layers fr candidate-query-idxs trace-openings
trace-table doms layers
where *evidence:*
trace-fri-partial-candidate-opening-evidence s out trace-roots
trace-bs trace-final dg composition-roots composition-bs
composition-final fri-query-idxs trace-round-layers
composition-round-layers fr candidate-query-idxs trace-openings
trace-table
and *chain:*
generic-fri-canonical-layer-chain-evidence trace-table-low-degree
trace-table (clength - 1) trace-roots trace-bs trace-final
fri-query-idxs trace-round-layers doms layers (bad trace-table)
unfolding *trace-fri-bad-with-refined-layer-chain-def* **by** *blast*
have *cover:* *trace-fri-canonical-layer-chain-cover s bad*
by *(rule trace-fri-canonical-layer-chain-cover-generic)*
have *bad-hit:* *trace-bs ∈ trace-fri-multiround-bad-sets bad trace-table*
using *cover evidence chain*
unfolding *trace-fri-canonical-layer-chain-cover-def* **by** *blast*
have *challenges:*
accepted-fri-challenges s out trace-bs dg composition-bs
using *trace-fri-partial-candidate-opening-evidenceD(1)[OF evidence]*
accepted-fri-opening-transcript-challenges **by** *blast*
have *trace-bs ∈ B*
using *bad-hit envelope[of trace-table]* **by** *blast*
then show *?thesis*
unfolding *trace-fri-challenge-list-set-hit-def*
using *challenges* **by** *blast*
qed

lemma *composition-fri-bad-with-refined-layer-chain-imp-list-hit:*
assumes *refined:*
composition-fri-bad-with-refined-layer-chain s bad out
and *envelope:*
 $\bigwedge dg$ *composition-table.*
composition-fri-multiround-bad-sets bad dg composition-table
 $\subseteq B$ *dg*
shows *composition-fri-challenge-list-set-hit s B out*
proof –
from *refined* **obtain** *trace-roots trace-bs trace-final fri-dg*
opening-composition-roots composition-bs composition-final
fri-query-idxs trace-round-layers composition-round-layers fr
f-fri-roots f-final as dg composition-fri-roots final
trace-query-idxs trace-openings composition-query-idxs
composition-openings trace-table composition-table doms layers
where *evidence:*
composition-fri-partial-candidate-opening-evidence s out trace-roots
trace-bs trace-final fri-dg opening-composition-roots composition-bs
composition-final fri-query-idxs trace-round-layers


```

    composition-round-layers fr f-fri-roots f-final as dg
    composition-fri-roots final trace-query-idxs trace-openings
    composition-query-idxs composition-openings trace-table
    composition-table
  and chain:
    generic-fri-canonical-layer-chain-evidence
      (composition-table-low-degree (to-nat fri-dg)) composition-table
      (to-nat fri-dg) opening-composition-roots composition-bs
      composition-final fri-query-idxs composition-round-layers doms layers
      (bad fri-dg composition-table)
  unfolding composition-fri-bad-with-refined-layer-chain-def
  by auto
  have bad-hit:
    composition-bs ∈
      composition-fri-multiround-bad-sets bad fri-dg composition-table
  proof -
    have bad-generic:
      composition-bs ∈ generic-fri-bad-challenge-lists
        (length composition-bs) (bad fri-dg composition-table)
    by (rule generic-fri-canonical-layer-chain-bad-challenge-list
        [OF chain])
    have partial:
      generic-fri-partial-evidence
        (composition-table-low-degree (to-nat fri-dg)) composition-table
        (to-nat fri-dg) opening-composition-roots composition-bs
        composition-final fri-query-idxs composition-round-layers
    by (rule generic-fri-canonical-layer-chain-evidenceD(1)[OF chain])
    have len-composition:
      length composition-bs = composition-fri-algebraic-round-count fri-dg
    using generic-fri-partial-evidence-shapes(1,2)[OF partial]
    by (simp add: composition-fri-algebraic-round-count-def)
    show ?thesis
      using bad-generic len-composition
    by (simp add: composition-fri-multiround-bad-sets-as-generic)
  qed
  have challenges:
    accepted-fri-challenges s out trace-bs fri-dg composition-bs
  using composition-fri-partial-candidate-opening-evidenceD(1)[OF evidence]
    accepted-fri-opening-transcript-challenges by blast
  have composition-bs ∈ B fri-dg
  using bad-hit envelope[of fri-dg composition-table] by blast
  then show ?thesis
    unfolding composition-fri-challenge-list-set-hit-def
    using challenges by blast
  qed

  lemma wp-trace-fri-bad-with-refined-layer-chain-bound-from-envelope:
    assumes future: trace-fri-future-fresh s
    and envelope:

```

$\bigwedge \text{trace-table. trace-fri-multiround-bad-sets bad trace-table} \subseteq B$
and *subset*: $B \subseteq \text{fri-challenge-space } (\text{ceil-log clength})$
and *bound*:
 $\text{nnreal } (\text{card } B) / \text{nnreal } (\text{CARD}('f) \wedge \text{ceil-log clength}) \leq C$
shows
 $\text{wp-event verify-monad}$
 $(\text{trace-fri-bad-with-refined-layer-chain } s \text{ bad}) \ s \leq C$
proof –
have $\text{wp-event verify-monad}$
 $(\text{trace-fri-bad-with-refined-layer-chain } s \text{ bad}) \ s \leq$
 $\text{wp-event verify-monad } (\text{trace-fri-challenge-list-set-hit } s \ B) \ s$
by $(\text{rule wp-event-mono})$
 $(\text{rule trace-fri-bad-with-refined-layer-chain-imp-list-hit}$
 $[\text{OF - envelope}])$
also have $\dots \leq \text{nnreal } (\text{card } B) /$
 $\text{nnreal } (\text{CARD}('f) \wedge \text{ceil-log clength})$
by $(\text{rule wp-verify-monad-trace-fri-challenge-list-set-bound}$
 $[\text{OF future subset}])$
also have $\dots \leq C$
by (rule bound)
finally show *?thesis* .
qed

lemma *wp-composition-fri-bad-with-refined-layer-chain-bound-from-envelope*:
assumes *future*: $\text{composition-fri-future-fresh } s$
and *envelope*:
 $\bigwedge dg \ \text{composition-table.}$
 $\text{composition-fri-multiround-bad-sets bad dg composition-table}$
 $\subseteq B \ dg$
and *subset*:
 $\bigwedge dg. \ B \ dg \subseteq \text{fri-challenge-space } (\text{ceil-log } (\text{to-nat } dg + 1))$
and *bound*:
 $\bigwedge dg. \ \text{nnreal } (\text{card } (B \ dg)) /$
 $\text{nnreal } (\text{CARD}('f) \wedge \text{ceil-log } (\text{to-nat } dg + 1)) \leq C$
shows
 $\text{wp-event verify-monad}$
 $(\text{composition-fri-bad-with-refined-layer-chain } s \text{ bad}) \ s \leq C$
proof –
have $\text{wp-event verify-monad}$
 $(\text{composition-fri-bad-with-refined-layer-chain } s \text{ bad}) \ s \leq$
 $\text{wp-event verify-monad } (\text{composition-fri-challenge-list-set-hit } s \ B) \ s$
by $(\text{rule wp-event-mono})$
 $(\text{rule composition-fri-bad-with-refined-layer-chain-imp-list-hit}$
 $[\text{OF - envelope}])$
also have $\dots \leq C$
by $(\text{rule wp-verify-monad-composition-fri-challenge-list-set-bound}$
 $[\text{OF future subset bound}])$
finally show *?thesis* .
qed

lemma *trace-fri-bad-with-sampled-layer-chain-imp-list-hit*:
assumes *sampled*:
trace-fri-bad-with-sampled-layer-chain s out
and *cover*: *trace-fri-sampled-layer-chain-cover s bad*
and *envelope*:
 $\bigwedge \text{trace-table. trace-fri-multiround-bad-sets bad trace-table} \subseteq B$
shows *trace-fri-challenge-list-set-hit s B out*
proof –
from *sampled* **obtain** *trace-roots trace-bs trace-final dg composition-roots*
composition-bs composition-final fri-query-idxs trace-round-layers
composition-round-layers fr candidate-query-idxs trace-openings
trace-table doms layers
where *evidence*:
trace-fri-partial-candidate-opening-evidence s out trace-roots
trace-bs trace-final dg composition-roots composition-bs
composition-final fri-query-idxs trace-round-layers
composition-round-layers fr candidate-query-idxs trace-openings
trace-table
and *chain*:
generic-fri-sampled-layer-chain-evidence trace-table-low-degree
trace-table (clength – 1) trace-roots trace-bs trace-final
fri-query-idxs trace-round-layers doms layers
unfolding *trace-fri-bad-with-sampled-layer-chain-def* **by** *blast*
have *bad-hit*: *trace-bs* $\in$ *trace-fri-multiround-bad-sets bad trace-table*
using *cover evidence chain*
unfolding *trace-fri-sampled-layer-chain-cover-def* **by** *blast*
have *challenges*:
accepted-fri-challenges s out trace-bs dg composition-bs
using *trace-fri-partial-candidate-opening-evidenceD(1)[OF evidence]*
accepted-fri-opening-transcript-challenges **by** *blast*
have *trace-bs* $\in B$
using *bad-hit envelope[of trace-table]* **by** *blast*
then show *?thesis*
unfolding *trace-fri-challenge-list-set-hit-def*
using *challenges* **by** *blast*
qed

lemma *composition-fri-bad-with-sampled-layer-chain-imp-list-hit*:
assumes *sampled*:
composition-fri-bad-with-sampled-layer-chain s out
and *cover*: *composition-fri-sampled-layer-chain-cover s bad*
and *envelope*:
 $\bigwedge dg \text{ composition-table. composition-fri-multiround-bad-sets bad dg composition-table}$
 $\subseteq B dg$
shows *composition-fri-challenge-list-set-hit s B out*
proof –
from *sampled* **obtain** *trace-roots trace-bs trace-final fri-dg*

opening-composition-roots composition-bs composition-final
fri-query-idxs trace-round-layers composition-round-layers fr
f-fri-roots f-final as dg composition-fri-roots final
trace-query-idxs trace-openings composition-query-idxs
composition-openings trace-table composition-table doms layers
where *evidence*:
composition-fri-partial-candidate-opening-evidence s out trace-roots
trace-bs trace-final fri-dg opening-composition-roots composition-bs
composition-final fri-query-idxs trace-round-layers
composition-round-layers fr f-fri-roots f-final as dg
composition-fri-roots final trace-query-idxs trace-openings
composition-query-idxs composition-openings trace-table
composition-table
and *chain*:
generic-fri-sampled-layer-chain-evidence
(composition-table-low-degree (to-nat fri-dg)) composition-table
(to-nat fri-dg) opening-composition-roots composition-bs
composition-final fri-query-idxs composition-round-layers doms layers
unfolding *composition-fri-bad-with-sampled-layer-chain-def* **by** *auto*
have *bad-hit*:
composition-bs $\in$
composition-fri-multiround-bad-sets bad fri-dg composition-table
using *cover evidence chain*
unfolding *composition-fri-sampled-layer-chain-cover-def* **by** *metis*
have *challenges*:
accepted-fri-challenges s out trace-bs fri-dg composition-bs
using *composition-fri-partial-candidate-opening-evidenceD(1)[OF evidence]*
accepted-fri-opening-transcript-challenges **by** *blast*
have *composition-bs* $\in B$ *fri-dg*
using *bad-hit envelope[of fri-dg composition-table]* **by** *blast*
then show *?thesis*
unfolding *composition-fri-challenge-list-set-hit-def*
using *challenges* **by** *blast*
qed

lemma *wp-trace-fri-bad-with-sampled-layer-chain-bound-from-envelope*:
assumes *future: trace-fri-future-fresh s*
and *cover: trace-fri-sampled-layer-chain-cover s bad*
and *envelope*:
 $\bigwedge \text{trace-table. trace-fri-multiround-bad-sets bad trace-table} \subseteq B$
and *subset*: $B \subseteq \text{fri-challenge-space (ceil-log clength)}$
and *bound*:
 $\text{nnreal (card } B) / \text{nnreal (CARD('f) } \wedge \text{ceil-log clength)} \leq C$
shows
wp-event verify-monad
 $(\text{trace-fri-bad-with-sampled-layer-chain } s) \ s \leq C$
proof –
have *wp-event verify-monad*
 $(\text{trace-fri-bad-with-sampled-layer-chain } s) \ s \leq$

```

    wp-event verify-monad (trace-fri-challenge-list-set-hit s B) s
  by (rule wp-event-mono)
    (rule trace-fri-bad-with-sampled-layer-chain-imp-list-hit
      [OF - cover envelope])
  also have ... ≤ nreal (card B) /
    nreal (CARD('f) ^ ceil-log clength)
  by (rule wp-verify-monad-trace-fri-challenge-list-set-bound
      [OF future subset])
  also have ... ≤ C
  by (rule bound)
  finally show ?thesis .
qed

lemma wp-composition-fri-bad-with-sampled-layer-chain-bound-from-envelope:
  assumes future: composition-fri-future-fresh s
  and cover: composition-fri-sampled-layer-chain-cover s bad
  and envelope:
    ∧ dg composition-table.
      composition-fri-multi-round-bad-sets bad dg composition-table
      ⊆ B dg
  and subset:
    ∧ dg. B dg ⊆ fri-challenge-space (ceil-log (to-nat dg + 1))
  and bound:
    ∧ dg. nreal (card (B dg)) /
      nreal (CARD('f) ^ ceil-log (to-nat dg + 1)) ≤ C
  shows
    wp-event verify-monad
      (composition-fri-bad-with-sampled-layer-chain s) s ≤ C
proof -
  have wp-event verify-monad
    (composition-fri-bad-with-sampled-layer-chain s) s ≤
    wp-event verify-monad (composition-fri-challenge-list-set-hit s B) s
  by (rule wp-event-mono)
    (rule composition-fri-bad-with-sampled-layer-chain-imp-list-hit
      [OF - cover envelope])
  also have ... ≤ C
  by (rule wp-verify-monad-composition-fri-challenge-list-set-bound
      [OF future subset bound])
  finally show ?thesis .
qed

lemma trace-fri-reachable-imp-refined-or-missing-layer-chain:
  assumes trace-fri-bad-with-reachable-partial-candidate s out
  shows
    trace-fri-bad-with-refined-layer-chain s bad out ∨
    trace-fri-reachable-missing-refined-layer-chain s bad out
  using assms
  unfolding trace-fri-reachable-missing-refined-layer-chain-def by blast

```

lemma *composition-fri-reachable-imp-refined-or-missing-layer-chain:*
assumes *composition-fri-bad-with-reachable-partial-candidate s out*
shows
 composition-fri-bad-with-refined-layer-chain s bad out $\vee$
 composition-fri-reachable-missing-refined-layer-chain s bad out
using *assms*
unfolding *composition-fri-reachable-missing-refined-layer-chain-def* **by** *blast*

lemma *trace-fri-reachable-imp-sampled-or-missing-layer-chain:*
assumes *trace-fri-bad-with-reachable-partial-candidate s out*
shows
 trace-fri-bad-with-sampled-layer-chain s out $\vee$
 trace-fri-reachable-missing-sampled-layer-chain s out
using *assms*
unfolding *trace-fri-reachable-missing-sampled-layer-chain-def* **by** *blast*

lemma *composition-fri-reachable-imp-sampled-or-missing-layer-chain:*
assumes *composition-fri-bad-with-reachable-partial-candidate s out*
shows
 composition-fri-bad-with-sampled-layer-chain s out $\vee$
 composition-fri-reachable-missing-sampled-layer-chain s out
using *assms*
unfolding *composition-fri-reachable-missing-sampled-layer-chain-def*
by *blast*

lemma *verify-monad-trace-fri-missing-extracts-opening-evidence:*
assumes *outcome:*
 Some (result, final-state) $\in$ set-dist (execute verify-monad s)
and *missing:*
 trace-fri-reachable-missing-refined-layer-chain s bad
 (Some (result, final-state))
obtains *trace-roots trace-bs trace-final dg composition-roots*
 composition-bs composition-final fri-query-idxs trace-round-layers
 composition-round-layers fr candidate-query-idxs trace-openings
 trace-table
where
 trace-fri-partial-candidate-opening-evidence s
 (Some (result, final-state)) trace-roots trace-bs trace-final dg
 composition-roots composition-bs composition-final fri-query-idxs
 trace-round-layers composition-round-layers fr candidate-query-idxs
 trace-openings trace-table
 $\wedge$ *doms layers. $\neg$*
 generic-fri-canonical-layer-chain-evidence trace-table-low-degree
 trace-table (clength - 1) trace-roots trace-bs trace-final
 fri-query-idxs trace-round-layers doms layers (bad trace-table)
proof –
have *reachable:*
 trace-fri-bad-with-reachable-partial-candidate s
 (Some (result, final-state))

```

and not-refined:
   $\neg$  trace-fri-bad-with-refined-layer-chain s bad
    (Some (result, final-state))
using missing
unfolding trace-fri-reachable-missing-refined-layer-chain-def
by blast+
from verify-monad-trace-fri-bad-extracts-opening-evidence
  [OF outcome reachable]
obtain trace-roots trace-bs trace-final dg composition-roots
  composition-bs composition-final fri-query-idxs trace-round-layers
  composition-round-layers fr candidate-query-idxs trace-openings
  trace-table
where evidence:
  trace-fri-partial-candidate-opening-evidence s
    (Some (result, final-state)) trace-roots trace-bs trace-final dg
    composition-roots composition-bs composition-final fri-query-idxs
    trace-round-layers composition-round-layers fr candidate-query-idxs
    trace-openings trace-table
by blast
have no-chain:
   $\bigwedge$  doms layers.  $\neg$ 
    generic-fri-canonical-layer-chain-evidence trace-table-low-degree
    trace-table (clength  $-$  1) trace-roots trace-bs trace-final
    fri-query-idxs trace-round-layers doms layers (bad trace-table)
proof
  fix doms layers
  assume chain:
    generic-fri-canonical-layer-chain-evidence trace-table-low-degree
    trace-table (clength  $-$  1) trace-roots trace-bs trace-final
    fri-query-idxs trace-round-layers doms layers (bad trace-table)
  have trace-fri-bad-with-refined-layer-chain s bad
    (Some (result, final-state))
    unfolding trace-fri-bad-with-refined-layer-chain-def
    using evidence chain by blast
  then show False
    using not-refined by contradiction
qed
show ?thesis
  by (rule that[OF evidence no-chain])
qed

```

```

lemma verify-monad-composition-fri-missing-extracts-opening-evidence:
assumes outcome:
  Some (result, final-state)  $\in$  set-dist (execute verify-monad s)
and missing:
  composition-fri-reachable-missing-refined-layer-chain s bad
    (Some (result, final-state))
obtains trace-roots trace-bs trace-final fri-dg
  opening-composition-roots composition-bs composition-final

```

```

    fri-query-idxs trace-round-layers composition-round-layers fr
    f-fri-roots f-final as dg composition-fri-roots final
    trace-query-idxs trace-openings composition-query-idxs
    composition-openings trace-table composition-table
  where
    composition-fri-partial-candidate-opening-evidence s
      (Some (result, final-state)) trace-roots trace-bs trace-final fri-dg
    opening-composition-roots composition-bs composition-final
    fri-query-idxs trace-round-layers composition-round-layers fr
    f-fri-roots f-final as dg composition-fri-roots final
    trace-query-idxs trace-openings composition-query-idxs
    composition-openings trace-table composition-table
    to-nat fri-dg ≤ maxDegree
     $\bigwedge$  doms layers.  $\neg$ 
      generic-fri-canonical-layer-chain-evidence
        (composition-table-low-degree (to-nat fri-dg)) composition-table
        (to-nat fri-dg) opening-composition-roots composition-bs
        composition-final fri-query-idxs composition-round-layers doms layers
        (bad fri-dg composition-table)
  proof -
  have reachable:
    composition-fri-bad-with-reachable-partial-candidate s
      (Some (result, final-state))
  and not-refined:
     $\neg$  composition-fri-bad-with-refined-layer-chain s bad
      (Some (result, final-state))
  using missing
  unfolding composition-fri-reachable-missing-refined-layer-chain-def
  by blast+
  show ?thesis
  proof (rule
    verify-monad-composition-fri-bad-extracts-opening-evidence-with-degree-bound
      [OF outcome reachable])
    fix trace-roots trace-bs trace-final fri-dg
    opening-composition-roots composition-bs composition-final
    fri-query-idxs trace-round-layers composition-round-layers fr
    f-fri-roots f-final as dg composition-fri-roots final
    trace-query-idxs trace-openings composition-query-idxs
    composition-openings trace-table composition-table
  assume evidence:
    composition-fri-partial-candidate-opening-evidence s
      (Some (result, final-state)) trace-roots trace-bs trace-final fri-dg
    opening-composition-roots composition-bs composition-final
    fri-query-idxs trace-round-layers composition-round-layers fr
    f-fri-roots f-final as dg composition-fri-roots final
    trace-query-idxs trace-openings composition-query-idxs
    composition-openings trace-table composition-table
  assume degree-bound: to-nat fri-dg ≤ maxDegree
  have no-chain:

```



```

 $\wedge$  doms layers.  $\neg$ 
  generic-fri-canonical-layer-chain-evidence
    (composition-table-low-degree (to-nat fri-dg)) composition-table
    (to-nat fri-dg) opening-composition-roots composition-bs
    composition-final fri-query-idxs composition-round-layers doms
    layers (bad fri-dg composition-table)
proof
  fix doms layers
  assume chain:
    generic-fri-canonical-layer-chain-evidence
      (composition-table-low-degree (to-nat fri-dg)) composition-table
      (to-nat fri-dg) opening-composition-roots composition-bs
      composition-final fri-query-idxs composition-round-layers doms
      layers (bad fri-dg composition-table)
  have composition-fri-bad-with-refined-layer-chain s bad
    (Some (result, final-state))
  using evidence chain
  unfolding composition-fri-bad-with-refined-layer-chain-def
  by metis
  then show False
    using not-refined by contradiction
qed
show thesis
  by (rule that[OF evidence degree-bound no-chain])
qed
qed

lemma verify-monad-trace-fri-missing-sampled-extracts-opening-evidence:
assumes outcome:
  Some (result, final-state)  $\in$  set-dist (execute verify-monad s)
and missing:
  trace-fri-reachable-missing-sampled-layer-chain s
  (Some (result, final-state))
obtains trace-roots trace-bs trace-final dg composition-roots
  composition-bs composition-final fri-query-idxs trace-round-layers
  composition-round-layers fr candidate-query-idxs trace-openings
  trace-table
where
  trace-fri-partial-candidate-opening-evidence s
    (Some (result, final-state)) trace-roots trace-bs trace-final dg
    composition-roots composition-bs composition-final fri-query-idxs
    trace-round-layers composition-round-layers fr candidate-query-idxs
    trace-openings trace-table
 $\wedge$  doms layers.  $\neg$ 
  generic-fri-sampled-layer-chain-evidence trace-table-low-degree
    trace-table (clength - 1) trace-roots trace-bs trace-final
    fri-query-idxs trace-round-layers doms layers
proof -
  have reachable:

```

```

    trace-fri-bad-with-reachable-partial-candidate s
      (Some (result, final-state))
  and not-sampled:
    ¬ trace-fri-bad-with-sampled-layer-chain s
      (Some (result, final-state))
  using missing
  unfolding trace-fri-reachable-missing-sampled-layer-chain-def
  by blast+
from verify-monad-trace-fri-bad-extracts-opening-evidence
  [OF outcome reachable]
obtain trace-roots trace-bs trace-final dg composition-roots
  composition-bs composition-final fri-query-idxs trace-round-layers
  composition-round-layers fr candidate-query-idxs trace-openings
  trace-table
where evidence:
  trace-fri-partial-candidate-opening-evidence s
    (Some (result, final-state)) trace-roots trace-bs trace-final dg
    composition-roots composition-bs composition-final fri-query-idxs
    trace-round-layers composition-round-layers fr candidate-query-idxs
    trace-openings trace-table
  by blast
have no-sampled:
  ∧ doms layers. ¬
    generic-fri-sampled-layer-chain-evidence trace-table-low-degree
      trace-table (clength - 1) trace-roots trace-bs trace-final
      fri-query-idxs trace-round-layers doms layers
proof
  fix doms layers
  assume chain:
    generic-fri-sampled-layer-chain-evidence trace-table-low-degree
      trace-table (clength - 1) trace-roots trace-bs trace-final
      fri-query-idxs trace-round-layers doms layers
  have trace-fri-bad-with-sampled-layer-chain s
    (Some (result, final-state))
  unfolding trace-fri-bad-with-sampled-layer-chain-def
  using evidence chain by blast
  then show False
  using not-sampled by contradiction
qed
show ?thesis
  by (rule that[OF evidence no-sampled])
qed

lemma verify-monad-composition-fri-missing-sampled-extracts-opening-evidence:
  assumes outcome:
    Some (result, final-state) ∈ set-dist (execute verify-monad s)
  and missing:
    composition-fri-reachable-missing-sampled-layer-chain s
      (Some (result, final-state))

```

obtains *trace-roots trace-bs trace-final fri-dg*
opening-composition-roots composition-bs composition-final
fri-query-idxs trace-round-layers composition-round-layers fr
f-fri-roots f-final as dg composition-fri-roots final
trace-query-idxs trace-openings composition-query-idxs
composition-openings trace-table composition-table
where
composition-fri-partial-candidate-opening-evidence s
(Some (result, final-state)) trace-roots trace-bs trace-final fri-dg
opening-composition-roots composition-bs composition-final
fri-query-idxs trace-round-layers composition-round-layers fr
f-fri-roots f-final as dg composition-fri-roots final
trace-query-idxs trace-openings composition-query-idxs
composition-openings trace-table composition-table
to-nat fri-dg ≤ maxDegree
 $\bigwedge \text{doms layers. } \neg$
generic-fri-sampled-layer-chain-evidence
(composition-table-low-degree (to-nat fri-dg)) composition-table
(to-nat fri-dg) opening-composition-roots composition-bs
composition-final fri-query-idxs composition-round-layers doms layers
proof –
have *reachable:*
composition-fri-bad-with-reachable-partial-candidate s
(Some (result, final-state))
and *not-sampled:*
 $\neg$ *composition-fri-bad-with-sampled-layer-chain s*
(Some (result, final-state))
using *missing*
unfolding *composition-fri-reachable-missing-sampled-layer-chain-def*
by *blast+*
show *?thesis*
proof (*rule*
verify-monad-composition-fri-bad-extracts-opening-evidence-with-degree-bound
[OF outcome reachable])
fix *trace-roots trace-bs trace-final fri-dg*
opening-composition-roots composition-bs composition-final
fri-query-idxs trace-round-layers composition-round-layers fr
f-fri-roots f-final as dg composition-fri-roots final
trace-query-idxs trace-openings composition-query-idxs
composition-openings trace-table composition-table
assume *evidence:*
composition-fri-partial-candidate-opening-evidence s
(Some (result, final-state)) trace-roots trace-bs trace-final fri-dg
opening-composition-roots composition-bs composition-final
fri-query-idxs trace-round-layers composition-round-layers fr
f-fri-roots f-final as dg composition-fri-roots final
trace-query-idxs trace-openings composition-query-idxs
composition-openings trace-table composition-table
assume *degree-bound: to-nat fri-dg ≤ maxDegree*

```

have no-sampled:
   $\bigwedge$  doms layers.  $\neg$ 
    generic-fri-sampled-layer-chain-evidence
      (composition-table-low-degree (to-nat fri-dg)) composition-table
      (to-nat fri-dg) opening-composition-roots composition-bs
      composition-final fri-query-idxs composition-round-layers doms
      layers
proof
  fix doms layers
  assume chain:
    generic-fri-sampled-layer-chain-evidence
      (composition-table-low-degree (to-nat fri-dg)) composition-table
      (to-nat fri-dg) opening-composition-roots composition-bs
      composition-final fri-query-idxs composition-round-layers doms
      layers
  have composition-fri-bad-with-sampled-layer-chain s
    (Some (result, final-state))
  using evidence chain
  unfolding composition-fri-bad-with-sampled-layer-chain-def
  by metis
  then show False
    using not-sampled by contradiction
qed
show thesis
  by (rule that[OF evidence degree-bound no-sampled])
qed
qed

lemma verify-monad-trace-fri-missing-full-cover-extracts-sampled-evidence:
assumes outcome:
  Some (result, final-state)  $\in$  set-dist (execute verify-monad s)
and missing:
  trace-fri-sampled-missing-full-cover s bad
  (Some (result, final-state))
obtains trace-roots trace-bs trace-final dg composition-roots
  composition-bs composition-final fri-query-idxs trace-round-layers
  composition-round-layers fr candidate-query-idxs trace-openings
  trace-table doms layers
where
  trace-fri-partial-candidate-opening-evidence s
    (Some (result, final-state)) trace-roots trace-bs trace-final dg
    composition-roots composition-bs composition-final fri-query-idxs
    trace-round-layers composition-round-layers fr candidate-query-idxs
    trace-openings trace-table
  generic-fri-sampled-layer-chain-evidence trace-table-low-degree
    trace-table (clength - 1) trace-roots trace-bs trace-final
    fri-query-idxs trace-round-layers doms layers
   $\neg$  generic-fri-sampled-layer-chain-full-cover trace-table-low-degree
    trace-table (clength - 1) trace-roots trace-bs trace-final

```

```

    fri-query-idxs trace-round-layers doms layers (bad trace-table)
proof —
  have sampld:
    trace-fri-bad-with-sampled-layer-chain s
      (Some (result, final-state))
  and not-full:
    ¬ trace-fri-bad-with-full-cover-layer-chain s bad
      (Some (result, final-state))
  using missing
  unfolding trace-fri-sampled-missing-full-cover-def by blast+
from sampld obtain trace-roots trace-bs trace-final dg composition-roots
  composition-bs composition-final fri-query-idxs trace-round-layers
  composition-round-layers fr candidate-query-idxs trace-openings
  trace-table doms layers
where evidence:
  trace-fri-partial-candidate-opening-evidence s
    (Some (result, final-state)) trace-roots trace-bs trace-final dg
    composition-roots composition-bs composition-final fri-query-idxs
    trace-round-layers composition-round-layers fr candidate-query-idxs
    trace-openings trace-table
and chain:
  generic-fri-sampled-layer-chain-evidence trace-table-low-degree
    trace-table (clength - 1) trace-roots trace-bs trace-final
    fri-query-idxs trace-round-layers doms layers
  unfolding trace-fri-bad-with-sampled-layer-chain-def by blast
have no-full:
  ¬ generic-fri-sampled-layer-chain-full-cover trace-table-low-degree
    trace-table (clength - 1) trace-roots trace-bs trace-final
    fri-query-idxs trace-round-layers doms layers (bad trace-table)
proof
  assume full:
    generic-fri-sampled-layer-chain-full-cover trace-table-low-degree
      trace-table (clength - 1) trace-roots trace-bs trace-final
      fri-query-idxs trace-round-layers doms layers (bad trace-table)
  have trace-fri-bad-with-full-cover-layer-chain s bad
    (Some (result, final-state))
  unfolding trace-fri-bad-with-full-cover-layer-chain-def
  using evidence full by blast
  then show False
    using not-full by contradiction
qed
show ?thesis
  by (rule that[OF evidence chain no-full])
qed

lemma verify-monad-composition-fri-missing-full-cover-extracts-sampled-evidence:
assumes outcome:
  Some (result, final-state) ∈ set-dist (execute verify-monad s)
and missing:

```

composition-fri-sampled-missing-full-cover s bad
(Some (result, final-state))
obtains *trace-roots trace-bs trace-final fri-dg*
opening-composition-roots composition-bs composition-final
fri-query-idxs trace-round-layers composition-round-layers fr
f-fri-roots f-final as dg composition-fri-roots final
trace-query-idxs trace-openings composition-query-idxs
composition-openings trace-table composition-table doms layers
where
composition-fri-partial-candidate-opening-evidence s
(Some (result, final-state)) trace-roots trace-bs trace-final fri-dg
opening-composition-roots composition-bs composition-final
fri-query-idxs trace-round-layers composition-round-layers fr
f-fri-roots f-final as dg composition-fri-roots final
trace-query-idxs trace-openings composition-query-idxs
composition-openings trace-table composition-table
generic-fri-sampled-layer-chain-evidence
(composition-table-low-degree (to-nat fri-dg)) composition-table
(to-nat fri-dg) opening-composition-roots composition-bs
composition-final fri-query-idxs composition-round-layers doms layers
 $\neg$ *generic-fri-sampled-layer-chain-full-cover*
(composition-table-low-degree (to-nat fri-dg)) composition-table
(to-nat fri-dg) opening-composition-roots composition-bs
composition-final fri-query-idxs composition-round-layers doms layers
(bad fri-dg composition-table)
proof –
have *sampled:*
composition-fri-bad-with-sampled-layer-chain s
(Some (result, final-state))
and *not-full:*
 $\neg$ *composition-fri-bad-with-full-cover-layer-chain s bad*
(Some (result, final-state))
using *missing*
unfolding *composition-fri-sampled-missing-full-cover-def* **by** *blast+*
from *sampled* **obtain** *trace-roots trace-bs trace-final fri-dg*
opening-composition-roots composition-bs composition-final
fri-query-idxs trace-round-layers composition-round-layers fr
f-fri-roots f-final as dg composition-fri-roots final
trace-query-idxs trace-openings composition-query-idxs
composition-openings trace-table composition-table doms layers
where *evidence:*
composition-fri-partial-candidate-opening-evidence s
(Some (result, final-state)) trace-roots trace-bs trace-final fri-dg
opening-composition-roots composition-bs composition-final
fri-query-idxs trace-round-layers composition-round-layers fr
f-fri-roots f-final as dg composition-fri-roots final
trace-query-idxs trace-openings composition-query-idxs
composition-openings trace-table composition-table
and *chain:*

```

    generic-fri-sampled-layer-chain-evidence
      (composition-table-low-degree (to-nat fri-dg)) composition-table
      (to-nat fri-dg) opening-composition-roots composition-bs
      composition-final fri-query-idxs composition-round-layers doms layers
  unfolding composition-fri-bad-with-sampled-layer-chain-def by auto
have no-full:
  ¬ generic-fri-sampled-layer-chain-full-cover
    (composition-table-low-degree (to-nat fri-dg)) composition-table
    (to-nat fri-dg) opening-composition-roots composition-bs
    composition-final fri-query-idxs composition-round-layers doms layers
    (bad fri-dg composition-table)
proof
  assume full:
    generic-fri-sampled-layer-chain-full-cover
      (composition-table-low-degree (to-nat fri-dg)) composition-table
      (to-nat fri-dg) opening-composition-roots composition-bs
      composition-final fri-query-idxs composition-round-layers doms layers
      (bad fri-dg composition-table)
  have composition-fri-bad-with-full-cover-layer-chain s bad
    (Some (result, final-state))
  unfolding composition-fri-bad-with-full-cover-layer-chain-def
  using evidence full by metis
  then show False
    using not-full by contradiction
qed
show ?thesis
  by (rule that[OF evidence chain no-full])
qed

lemma wp-trace-fri-reachable-bound-from-refined-and-missing:
  assumes refined-bound:
    wp-event verify-monad
      (trace-fri-bad-with-refined-layer-chain s bad) s ≤ R
  and missing-bound:
    wp-event verify-monad
      (trace-fri-reachable-missing-refined-layer-chain s bad) s ≤ M
  shows
    wp-event verify-monad
      (trace-fri-bad-with-reachable-partial-candidate s) s ≤ R + M
proof -
  have wp-event verify-monad
    (trace-fri-bad-with-reachable-partial-candidate s) s ≤
  wp-event verify-monad
    (λout. trace-fri-bad-with-refined-layer-chain s bad out ∨
      trace-fri-reachable-missing-refined-layer-chain s bad out) s
  by (rule wp-event-mono)
    (rule trace-fri-reachable-imp-refined-or-missing-layer-chain)
  also have ... ≤
    wp-event verify-monad

```

(trace-fri-bad-with-refined-layer-chain s bad) s +
 wp-event verify-monad
 (trace-fri-reachable-missing-refined-layer-chain s bad) s
 by (rule wp-event-union-bound)
 also have ... $\leq R + M$
 by (intro add-mono refined-bound missing-bound)
 finally show ?thesis .
 qed

lemma wp-composition-fri-reachable-bound-from-refined-and-missing:
 assumes refined-bound:
 wp-event verify-monad
 (composition-fri-bad-with-refined-layer-chain s bad) $s \leq R$
 and missing-bound:
 wp-event verify-monad
 (composition-fri-reachable-missing-refined-layer-chain s bad) $s \leq M$
 shows
 wp-event verify-monad
 (composition-fri-bad-with-reachable-partial-candidate s) $s \leq R + M$
proof –
 have wp-event verify-monad
 (composition-fri-bad-with-reachable-partial-candidate s) $s \leq$
 wp-event verify-monad
 ($\lambda out. \text{composition-fri-bad-with-refined-layer-chain } s \text{ bad } out \vee$
 composition-fri-reachable-missing-refined-layer-chain s bad out) s
 by (rule wp-event-mono)
 (rule composition-fri-reachable-imp-refined-or-missing-layer-chain)
 also have ... $\leq$
 wp-event verify-monad
 (composition-fri-bad-with-refined-layer-chain s bad) s +
 wp-event verify-monad
 (composition-fri-reachable-missing-refined-layer-chain s bad) s
 by (rule wp-event-union-bound)
 also have ... $\leq R + M$
 by (intro add-mono refined-bound missing-bound)
 finally show ?thesis .
 qed

lemma wp-trace-fri-reachable-bound-from-sampled-and-missing:
 assumes sampled-bound:
 wp-event verify-monad
 (trace-fri-bad-with-sampled-layer-chain s) $s \leq R$
 and missing-bound:
 wp-event verify-monad
 (trace-fri-reachable-missing-sampled-layer-chain s) $s \leq M$
 shows
 wp-event verify-monad
 (trace-fri-bad-with-reachable-partial-candidate s) $s \leq R + M$
proof –

have *wp-event verify-monad*
 (*trace-fri-bad-with-reachable-partial-candidate s*) $s \leq$
wp-event verify-monad
 ($\lambda out. trace-fri-bad-with-sampled-layer-chain\ s\ out \vee$
trace-fri-reachable-missing-sampled-layer-chain s out) s
by (*rule wp-event-mono*)
 (*rule trace-fri-reachable-imp-sampled-or-missing-layer-chain*)
also have ... $\leq$
wp-event verify-monad
 (*trace-fri-bad-with-sampled-layer-chain s*) $s +$
wp-event verify-monad
 (*trace-fri-reachable-missing-sampled-layer-chain s*) s
by (*rule wp-event-union-bound*)
also have ... $\leq R + M$
by (*intro add-mono sampled-bound missing-bound*)
finally show ?thesis .
qed

lemma *wp-composition-fri-reachable-bound-from-sampled-and-missing:*
assumes *sampled-bound:*
wp-event verify-monad
 (*composition-fri-bad-with-sampled-layer-chain s*) $s \leq R$
and *missing-bound:*
wp-event verify-monad
 (*composition-fri-reachable-missing-sampled-layer-chain s*) $s \leq M$
shows
wp-event verify-monad
 (*composition-fri-bad-with-reachable-partial-candidate s*) $s \leq R + M$
proof –
have *wp-event verify-monad*
 (*composition-fri-bad-with-reachable-partial-candidate s*) $s \leq$
wp-event verify-monad
 ($\lambda out. composition-fri-bad-with-sampled-layer-chain\ s\ out \vee$
composition-fri-reachable-missing-sampled-layer-chain s out) s
by (*rule wp-event-mono*)
 (*rule composition-fri-reachable-imp-sampled-or-missing-layer-chain*)
also have ... $\leq$
wp-event verify-monad
 (*composition-fri-bad-with-sampled-layer-chain s*) $s +$
wp-event verify-monad
 (*composition-fri-reachable-missing-sampled-layer-chain s*) s
by (*rule wp-event-union-bound*)
also have ... $\leq R + M$
by (*intro add-mono sampled-bound missing-bound*)
finally show ?thesis .
qed

lemma *trace-fri-reachable-partial-candidate-reduction-from-refined-and-missing:*
assumes *refined-bound:*

$wp\text{-event verify-monad}$
 $(\text{trace-fri-bad-with-refined-layer-chain } s \text{ bad}) \ s \leq R$
and missing-bound :
 $wp\text{-event verify-monad}$
 $(\text{trace-fri-reachable-missing-refined-layer-chain } s \text{ bad}) \ s \leq M$
and total-bound : $R + M \leq \text{trace-fri-error}$
shows $\text{trace-fri-reachable-partial-candidate-reduction-assumption } s$
proof –
have old-bound :
 $wp\text{-event verify-monad}$
 $(\text{trace-fri-bad-with-reachable-partial-candidate } s) \ s \leq R + M$
by (rule $wp\text{-trace-fri-reachable-bound-from-refined-and-missing}$
 $[OF \ \text{refined-bound} \ \text{missing-bound}]$)
show $?thesis$
unfolding $\text{trace-fri-reachable-partial-candidate-reduction-assumption-def}$
by (rule $\text{order-trans}[OF \ \text{old-bound} \ \text{total-bound}]$)
qed

lemma $\text{composition-fri-reachable-partial-candidate-reduction-from-refined-and-missing}$:
assumes refined-bound :
 $wp\text{-event verify-monad}$
 $(\text{composition-fri-bad-with-refined-layer-chain } s \text{ bad}) \ s \leq R$
and missing-bound :
 $wp\text{-event verify-monad}$
 $(\text{composition-fri-reachable-missing-refined-layer-chain } s \text{ bad}) \ s \leq M$
and total-bound : $R + M \leq \text{composition-fri-error}$
shows $\text{composition-fri-reachable-partial-candidate-reduction-assumption } s$
proof –
have old-bound :
 $wp\text{-event verify-monad}$
 $(\text{composition-fri-bad-with-reachable-partial-candidate } s) \ s \leq R + M$
by (rule $wp\text{-composition-fri-reachable-bound-from-refined-and-missing}$
 $[OF \ \text{refined-bound} \ \text{missing-bound}]$)
show $?thesis$
unfolding
 $\text{composition-fri-reachable-partial-candidate-reduction-assumption-def}$
by (rule $\text{order-trans}[OF \ \text{old-bound} \ \text{total-bound}]$)
qed

lemma $\text{trace-fri-reachable-partial-candidate-reduction-from-sampled-and-missing}$:
assumes sampled-bound :
 $wp\text{-event verify-monad}$
 $(\text{trace-fri-bad-with-sampled-layer-chain } s) \ s \leq R$
and missing-bound :
 $wp\text{-event verify-monad}$
 $(\text{trace-fri-reachable-missing-sampled-layer-chain } s) \ s \leq M$
and total-bound : $R + M \leq \text{trace-fri-error}$
shows $\text{trace-fri-reachable-partial-candidate-reduction-assumption } s$
proof –

have *old-bound*:
 wp-event verify-monad
 (*trace-fri-bad-with-reachable-partial-candidate s*) $s \leq R + M$
by (rule *wp-trace-fri-reachable-bound-from-sampled-and-missing*
 [*OF sampled-bound missing-bound*])
show *?thesis*
 unfolding *trace-fri-reachable-partial-candidate-reduction-assumption-def*
 by (rule *order-trans*[*OF old-bound total-bound*])
qed

lemma *composition-fri-reachable-partial-candidate-reduction-from-sampled-and-missing*:
assumes *sampled-bound*:
 wp-event verify-monad
 (*composition-fri-bad-with-sampled-layer-chain s*) $s \leq R$
and *missing-bound*:
 wp-event verify-monad
 (*composition-fri-reachable-missing-sampled-layer-chain s*) $s \leq M$
and *total-bound*: $R + M \leq \text{composition-fri-error}$
shows *composition-fri-reachable-partial-candidate-reduction-assumption s*
proof –
have *old-bound*:
 wp-event verify-monad
 (*composition-fri-bad-with-reachable-partial-candidate s*) $s \leq R + M$
by (rule *wp-composition-fri-reachable-bound-from-sampled-and-missing*
 [*OF sampled-bound missing-bound*])
show *?thesis*
 unfolding
 composition-fri-reachable-partial-candidate-reduction-assumption-def
by (rule *order-trans*[*OF old-bound total-bound*])
qed

lemma *trace-fri-bad-with-partial-candidates-bound-from-refined-and-missing*:
fixes $R\ M :: \text{prob}$
assumes *refined-bound*:
 $\bigwedge \text{data attacker-state.}$
 Some (*data, attacker-state*) $\in$
 set-dist (execute (checked-staged-transcript-program *A*)
 adversary-initial-state) $\implies$
 wp-event verify-monad
 (*trace-fri-bad-with-refined-layer-chain*
 (*verifier-state-from-adversary attacker-state*
 (*staged-proof-transcript data*)) *bad*)
 (*verifier-state-from-adversary attacker-state*
 (*staged-proof-transcript data*)) $\leq R$
and *missing-bound*:
 $\bigwedge \text{data attacker-state.}$
 Some (*data, attacker-state*) $\in$
 set-dist (execute (checked-staged-transcript-program *A*)
 adversary-initial-state) $\implies$

```

wp-event verify-monad
  (trace-fri-reachable-missing-refined-layer-chain
   (verifier-state-from-adversary attacker-state
    (staged-proof-transcript data)) bad)
  (verifier-state-from-adversary attacker-state
   (staged-proof-transcript data)) ≤ M
shows
  wp-event (checked-staged-security-experiment-with-data-state A)
    (staged-security-with-data-state-verifier-event
     trace-fri-bad-with-partial-candidates)
  adversary-initial-state ≤ R + M
proof (rule checked-staged-security-with-data-state-verifier-event-bound-from-cont)
  show  $\bigwedge s. \neg \text{trace-fri-bad-with-partial-candidates } s \text{ None}$ 
    by (rule trace-fri-bad-with-partial-candidates-not-None)
next
fix data attacker-state
assume builder:
  Some (data, attacker-state) ∈
    set-dist (execute (checked-staged-transcript-program A)
               adversary-initial-state)
let ?s =
  verifier-state-from-adversary attacker-state
    (staged-proof-transcript data)
have old-bound:
  wp-event verify-monad
    (trace-fri-bad-with-reachable-partial-candidate ?s) ?s ≤ R + M
  by (rule wp-trace-fri-reachable-bound-from-refined-and-missing
      [OF refined-bound[OF builder] missing-bound[OF builder]])
show wp-event verify-monad (trace-fri-bad-with-partial-candidates ?s) ?s
  ≤ R + M
  by (rule order-trans[OF - old-bound])
    (rule wp-event-mono,
     rule trace-fri-bad-with-partial-candidates-imp-reachable-partial-candidate)
qed

lemma composition-fri-bad-with-partial-candidates-bound-from-refined-and-missing:
  fixes R M :: prob
  assumes refined-bound:
     $\bigwedge \text{data attacker-state.}$ 
    Some (data, attacker-state) ∈
      set-dist (execute (checked-staged-transcript-program A)
                      adversary-initial-state)  $\implies$ 
    wp-event verify-monad
      (composition-fri-bad-with-refined-layer-chain
       (verifier-state-from-adversary attacker-state
        (staged-proof-transcript data)) bad)
      (verifier-state-from-adversary attacker-state
       (staged-proof-transcript data)) ≤ R
  and missing-bound:

```

$\wedge \text{data attacker-state.}$
Some (*data*, *attacker-state*) $\in$
set-dist (*execute* (*checked-staged-transcript-program A*)
adversary-initial-state) $\implies$
wp-event verify-monad
(*composition-fri-reachable-missing-refined-layer-chain*
(*verifier-state-from-adversary attacker-state*
(*staged-proof-transcript data*)) *bad*)
(*verifier-state-from-adversary attacker-state*
(*staged-proof-transcript data*)) $\leq M$
shows
wp-event (*checked-staged-security-experiment-with-data-state A*)
(*staged-security-with-data-state-verifier-event*
composition-fri-bad-with-partial-candidates)
adversary-initial-state $\leq R + M$
proof (*rule checked-staged-security-with-data-state-verifier-event-bound-from-cont*)
show $\wedge s. \neg \text{composition-fri-bad-with-partial-candidates } s \text{ None}$
by (*rule composition-fri-bad-with-partial-candidates-not-None*)
next
fix *data attacker-state*
assume *builder*:
Some (*data*, *attacker-state*) $\in$
set-dist (*execute* (*checked-staged-transcript-program A*)
adversary-initial-state)
let *?s* =
verifier-state-from-adversary attacker-state
(*staged-proof-transcript data*)
have *old-bound*:
wp-event verify-monad
(*composition-fri-bad-with-reachable-partial-candidate ?s*) *?s* $\leq$
R + M
by (*rule wp-composition-fri-reachable-bound-from-refined-and-missing*
[*OF refined-bound*[*OF builder*] *missing-bound*[*OF builder*]])
show *wp-event verify-monad*
(*composition-fri-bad-with-partial-candidates ?s*) *?s* $\leq R + M$
by (*rule order-trans*[*OF - old-bound*])
(*rule wp-event-mono*,
rule
composition-fri-bad-with-partial-candidates-imp-reachable-partial-candidate)
qed

lemma *trace-fri-bad-with-empty-header-candidates-bound-from-refined-and-missing*:
fixes *R M* :: *prob*
assumes *refined-bound*:
 $\wedge \text{data attacker-state.}$
Some (*data*, *attacker-state*) $\in$
set-dist (*execute* (*checked-staged-transcript-program A*)
adversary-initial-state) $\implies$
wp-event verify-monad

```

    (trace-fri-bad-with-refined-layer-chain
      (verifier-state-from-adversary attacker-state
        (staged-proof-transcript data)) bad)
    (verifier-state-from-adversary attacker-state
      (staged-proof-transcript data)) ≤ R
  and missing-bound:
  ∧ data attacker-state.
    Some (data, attacker-state) ∈
      set-dist (execute (checked-staged-transcript-program A)
        adversary-initial-state) ⇒
    wp-event verify-monad
      (trace-fri-reachable-missing-refined-layer-chain
        (verifier-state-from-adversary attacker-state
          (staged-proof-transcript data)) bad)
      (verifier-state-from-adversary attacker-state
        (staged-proof-transcript data)) ≤ M
  shows
    wp-event (checked-staged-security-experiment-with-data-state A)
      (staged-security-with-data-state-verifier-event
        trace-fri-bad-with-empty-composition-header-candidates)
      adversary-initial-state ≤ R + M
  proof (rule checked-staged-security-with-data-state-verifier-event-bound-from-cont)
    show ∧ s. ¬ trace-fri-bad-with-empty-composition-header-candidates s None
    by (rule trace-fri-bad-with-empty-composition-header-candidates-not-None)
  next
    fix data attacker-state
    assume builder:
      Some (data, attacker-state) ∈
        set-dist (execute (checked-staged-transcript-program A)
          adversary-initial-state)
    let ?s =
      verifier-state-from-adversary attacker-state
        (staged-proof-transcript data)
    have old-bound:
      wp-event verify-monad
        (trace-fri-bad-with-reachable-partial-candidate ?s) ?s ≤ R + M
    by (rule wp-trace-fri-reachable-bound-from-refined-and-missing
      [OF refined-bound[OF builder] missing-bound[OF builder]])
    show wp-event verify-monad
      (trace-fri-bad-with-empty-composition-header-candidates ?s) ?s ≤
        R + M
    by (rule order-trans[OF - old-bound])
      (rule wp-event-mono,
        rule
          trace-fri-bad-with-empty-composition-header-candidates-imp-reachable-partial-candidate)
  qed

  lemma trace-fri-bad-with-partial-candidates-bound-from-sampled-and-missing:
    fixes R M :: prob

```

assumes *sampled-bound*:
 $\bigwedge \text{data attacker-state.}$
 $\text{Some (data, attacker-state)} \in$
 $\text{set-dist (execute (checked-staged-transcript-program } A)$
 $\text{adversary-initial-state)} \implies$
 $\text{wp-event verify-monad}$
 $(\text{trace-fri-bad-with-sampled-layer-chain}$
 $(\text{verifier-state-from-adversary attacker-state}$
 $(\text{staged-proof-transcript data})))$
 $(\text{verifier-state-from-adversary attacker-state}$
 $(\text{staged-proof-transcript data})) \leq R$
and *missing-bound*:
 $\bigwedge \text{data attacker-state.}$
 $\text{Some (data, attacker-state)} \in$
 $\text{set-dist (execute (checked-staged-transcript-program } A)$
 $\text{adversary-initial-state)} \implies$
 $\text{wp-event verify-monad}$
 $(\text{trace-fri-reachable-missing-sampled-layer-chain}$
 $(\text{verifier-state-from-adversary attacker-state}$
 $(\text{staged-proof-transcript data})))$
 $(\text{verifier-state-from-adversary attacker-state}$
 $(\text{staged-proof-transcript data})) \leq M$
shows
 $\text{wp-event (checked-staged-security-experiment-with-data-state } A)$
 $(\text{staged-security-with-data-state-verifier-event}$
 $\text{trace-fri-bad-with-partial-candidates})$
 $\text{adversary-initial-state} \leq R + M$
proof (rule *checked-staged-security-with-data-state-verifier-event-bound-from-cont*)
show $\bigwedge s. \neg \text{trace-fri-bad-with-partial-candidates } s \text{ None}$
by (rule *trace-fri-bad-with-partial-candidates-not-None*)
next
fix *data attacker-state*
assume *builder*:
 $\text{Some (data, attacker-state)} \in$
 $\text{set-dist (execute (checked-staged-transcript-program } A)$
 $\text{adversary-initial-state})$
let $?s =$
 $\text{verifier-state-from-adversary attacker-state}$
 $(\text{staged-proof-transcript data})$
have *old-bound*:
 $\text{wp-event verify-monad}$
 $(\text{trace-fri-bad-with-reachable-partial-candidate } ?s) ?s \leq R + M$
by (rule *wp-trace-fri-reachable-bound-from-sampled-and-missing*
 $[\text{OF sampled-bound}[\text{OF builder}] \text{missing-bound}[\text{OF builder}]]$)
show $\text{wp-event verify-monad (trace-fri-bad-with-partial-candidates } ?s) ?s$
 $\leq R + M$
by (rule *order-trans*[*OF* - *old-bound*])
 $(\text{rule wp-event-mono,}$
 $\text{rule trace-fri-bad-with-partial-candidates-imp-reachable-partial-candidate})$

qed

lemma *composition-fri-bad-with-partial-candidates-bound-from-sampled-and-missing:*

fixes $R\ M :: \text{prob}$

assumes *sampled-bound:*

$\bigwedge \text{data attacker-state.}$

$\text{Some (data, attacker-state)} \in$
 $\text{set-dist (execute (checked-staged-transcript-program } A)$
 $\text{adversary-initial-state)} \implies$

$\text{wp-event verify-monad}$
 $(\text{composition-fri-bad-with-sampled-layer-chain}$
 $(\text{verifier-state-from-adversary attacker-state}$
 $(\text{staged-proof-transcript data})))$
 $(\text{verifier-state-from-adversary attacker-state}$
 $(\text{staged-proof-transcript data})) \leq R$

and *missing-bound:*

$\bigwedge \text{data attacker-state.}$

$\text{Some (data, attacker-state)} \in$
 $\text{set-dist (execute (checked-staged-transcript-program } A)$
 $\text{adversary-initial-state)} \implies$

$\text{wp-event verify-monad}$
 $(\text{composition-fri-reachable-missing-sampled-layer-chain}$
 $(\text{verifier-state-from-adversary attacker-state}$
 $(\text{staged-proof-transcript data})))$
 $(\text{verifier-state-from-adversary attacker-state}$
 $(\text{staged-proof-transcript data})) \leq M$

shows

$\text{wp-event (checked-staged-security-experiment-with-data-state } A)$
 $(\text{staged-security-with-data-state-verifier-event}$
 $\text{composition-fri-bad-with-partial-candidates})$
 $\text{adversary-initial-state} \leq R + M$

proof (rule *checked-staged-security-with-data-state-verifier-event-bound-from-cont*)

show $\bigwedge s. \neg \text{composition-fri-bad-with-partial-candidates } s \text{ None}$

by (rule *composition-fri-bad-with-partial-candidates-not-None*)

next

fix *data attacker-state*

assume *builder:*

$\text{Some (data, attacker-state)} \in$
 $\text{set-dist (execute (checked-staged-transcript-program } A)$
 $\text{adversary-initial-state})$

let $?s =$

$\text{verifier-state-from-adversary attacker-state}$
 $(\text{staged-proof-transcript data})$

have *old-bound:*

$\text{wp-event verify-monad}$
 $(\text{composition-fri-bad-with-reachable-partial-candidate } ?s) \text{ } ?s \leq$
 $R + M$

by (rule *wp-composition-fri-reachable-bound-from-sampled-and-missing*
 $[\text{OF sampled-bound}[\text{OF builder}] \text{ missing-bound}[\text{OF builder}]]$)


```

show wp-event verify-monad
  (composition-fri-bad-with-partial-candidates ?s) ?s ≤ R + M
by (rule order-trans[OF - old-bound])
  (rule wp-event-mono,
   rule
    composition-fri-bad-with-partial-candidates-imp-reachable-partial-candidate)
qed

lemma trace-fri-bad-with-empty-header-candidates-bound-from-sampled-and-missing:
fixes R M :: prob
assumes sampled-bound:
  ∧ data attacker-state.
    Some (data, attacker-state) ∈
      set-dist (execute (checked-staged-transcript-program A)
        adversary-initial-state) ⇒
    wp-event verify-monad
      (trace-fri-bad-with-sampled-layer-chain
        (verifier-state-from-adversary attacker-state
          (staged-proof-transcript data)))
      (verifier-state-from-adversary attacker-state
        (staged-proof-transcript data)) ≤ R
and missing-bound:
  ∧ data attacker-state.
    Some (data, attacker-state) ∈
      set-dist (execute (checked-staged-transcript-program A)
        adversary-initial-state) ⇒
    wp-event verify-monad
      (trace-fri-reachable-missing-sampled-layer-chain
        (verifier-state-from-adversary attacker-state
          (staged-proof-transcript data)))
      (verifier-state-from-adversary attacker-state
        (staged-proof-transcript data)) ≤ M
shows
  wp-event (checked-staged-security-experiment-with-data-state A)
    (staged-security-with-data-state-verifier-event
      trace-fri-bad-with-empty-composition-header-candidates)
    adversary-initial-state ≤ R + M
proof (rule checked-staged-security-with-data-state-verifier-event-bound-from-cont)
show ∧ s. ¬ trace-fri-bad-with-empty-composition-header-candidates s None
by (rule trace-fri-bad-with-empty-composition-header-candidates-not-None)
next
fix data attacker-state
assume builder:
  Some (data, attacker-state) ∈
    set-dist (execute (checked-staged-transcript-program A)
      adversary-initial-state)
let ?s =
  verifier-state-from-adversary attacker-state
    (staged-proof-transcript data)

```

have *old-bound*:
 wp-event verify-monad
 (trace-fri-bad-with-reachable-partial-candidate ?s) ?s $\leq R + M$
by (rule wp-trace-fri-reachable-bound-from-sampled-and-missing
 [OF sampled-bound[OF builder] missing-bound[OF builder]])
show wp-event verify-monad
 (trace-fri-bad-with-empty-composition-header-candidates ?s) ?s $\leq$
 R + M
by (rule order-trans[OF - old-bound])
 (rule wp-event-mono,
 rule
 trace-fri-bad-with-empty-composition-header-candidates-imp-reachable-partial-candidate)

qed

lemma trace-fri-bad-with-partial-candidates-bound-from-full-cover-and-missing:

fixes R MF MS :: prob

assumes full-bound:

$\wedge$ data attacker-state.

Some (data, attacker-state) $\in$
 set-dist (execute (checked-staged-transcript-program A)
 adversary-initial-state) $\implies$

wp-event verify-monad
 (trace-fri-bad-with-full-cover-layer-chain
 (verifier-state-from-adversary attacker-state
 (staged-proof-transcript data)) bad)
 (verifier-state-from-adversary attacker-state
 (staged-proof-transcript data)) $\leq R$

and missing-full-bound:

$\wedge$ data attacker-state.

Some (data, attacker-state) $\in$
 set-dist (execute (checked-staged-transcript-program A)
 adversary-initial-state) $\implies$

wp-event verify-monad
 (trace-fri-sampled-missing-full-cover
 (verifier-state-from-adversary attacker-state
 (staged-proof-transcript data)) bad)
 (verifier-state-from-adversary attacker-state
 (staged-proof-transcript data)) $\leq MF$

and missing-sampled-bound:

$\wedge$ data attacker-state.

Some (data, attacker-state) $\in$
 set-dist (execute (checked-staged-transcript-program A)
 adversary-initial-state) $\implies$

wp-event verify-monad
 (trace-fri-reachable-missing-sampled-layer-chain
 (verifier-state-from-adversary attacker-state
 (staged-proof-transcript data)))
 (verifier-state-from-adversary attacker-state
 (staged-proof-transcript data)) $\leq MS$

shows
wp-event (checked-staged-security-experiment-with-data-state A)
(staged-security-with-data-state-verifier-event
trace-fri-bad-with-partial-candidates)
adversary-initial-state $\leq (R + MF) + MS$

proof (rule *trace-fri-bad-with-partial-candidates-bound-from-sampled-and-missing*)
fix *data attacker-state*
assume *builder*:
Some (data, attacker-state) ∈
set-dist (execute (checked-staged-transcript-program A)
adversary-initial-state)
let *?s =*
verifier-state-from-adversary attacker-state
(staged-proof-transcript data)
show *wp-event verify-monad*
(trace-fri-bad-with-sampled-layer-chain ?s) ?s ≤ R + MF
by (rule *wp-trace-fri-sampled-bound-from-full-cover-and-missing*
 $[OF\ full-bound[OF\ builder]\ missing-full-bound[OF\ builder]]$)

next
fix *data attacker-state*
assume *builder*:
Some (data, attacker-state) ∈
set-dist (execute (checked-staged-transcript-program A)
adversary-initial-state)
show *wp-event verify-monad*
(trace-fri-reachable-missing-sampled-layer-chain
(verifier-state-from-adversary attacker-state
(staged-proof-transcript data)))
(verifier-state-from-adversary attacker-state
(staged-proof-transcript data)) ≤ MS
by (rule *missing-sampled-bound[OF builder]*)

qed

lemma *composition-fri-bad-with-partial-candidates-bound-from-full-cover-and-missing*:
fixes *R MF MS :: prob*
assumes *full-bound*:
 $\bigwedge data\ attacker-state.$
Some (data, attacker-state) ∈
set-dist (execute (checked-staged-transcript-program A)
adversary-initial-state) ⇒
wp-event verify-monad
(composition-fri-bad-with-full-cover-layer-chain
(verifier-state-from-adversary attacker-state
(staged-proof-transcript data)) bad)
(verifier-state-from-adversary attacker-state
(staged-proof-transcript data)) ≤ R
and *missing-full-bound*:
 $\bigwedge data\ attacker-state.$
Some (data, attacker-state) ∈

```

    set-dist (execute (checked-staged-transcript-program A)
      adversary-initial-state)  $\implies$ 
wp-event verify-monad
  (composition-fri-sampled-missing-full-cover
    (verifier-state-from-adversary attacker-state
      (staged-proof-transcript data)) bad)
  (verifier-state-from-adversary attacker-state
    (staged-proof-transcript data))  $\leq MF$ 
and missing-sampled-bound:
 $\wedge$ data attacker-state.
  Some (data, attacker-state)  $\in$ 
    set-dist (execute (checked-staged-transcript-program A)
      adversary-initial-state)  $\implies$ 
wp-event verify-monad
  (composition-fri-reachable-missing-sampled-layer-chain
    (verifier-state-from-adversary attacker-state
      (staged-proof-transcript data)))
  (verifier-state-from-adversary attacker-state
    (staged-proof-transcript data))  $\leq MS$ 
shows
wp-event (checked-staged-security-experiment-with-data-state A)
  (staged-security-with-data-state-verifier-event
    composition-fri-bad-with-partial-candidates)
  adversary-initial-state  $\leq (R + MF) + MS$ 
proof (rule
  composition-fri-bad-with-partial-candidates-bound-from-sampled-and-missing)
fix data attacker-state
assume builder:
  Some (data, attacker-state)  $\in$ 
    set-dist (execute (checked-staged-transcript-program A)
      adversary-initial-state)
let ?s =
  verifier-state-from-adversary attacker-state
    (staged-proof-transcript data)
show wp-event verify-monad
  (composition-fri-bad-with-sampled-layer-chain ?s) ?s  $\leq R + MF$ 
by (rule wp-composition-fri-sampled-bound-from-full-cover-and-missing
  [OF full-bound[OF builder] missing-full-bound[OF builder]])
next
fix data attacker-state
assume builder:
  Some (data, attacker-state)  $\in$ 
    set-dist (execute (checked-staged-transcript-program A)
      adversary-initial-state)
show wp-event verify-monad
  (composition-fri-reachable-missing-sampled-layer-chain
    (verifier-state-from-adversary attacker-state
      (staged-proof-transcript data)))
  (verifier-state-from-adversary attacker-state

```

$(\text{staged-proof-transcript data}) \leq MS$
 by (rule missing-sampled-bound[OF builder])
 qed

lemma *trace-fri-bad-with-empty-header-candidates-bound-from-full-cover-and-missing*:
fixes $R \ MF \ MS :: \text{prob}$
assumes *full-bound*:
 $\wedge \text{data attacker-state.}$
 $\text{Some (data, attacker-state)} \in$
 $\text{set-dist (execute (checked-staged-transcript-program } A)$
 $\text{adversary-initial-state)} \implies$
 $\text{wp-event verify-monad}$
 $(\text{trace-fri-bad-with-full-cover-layer-chain}$
 $(\text{verifier-state-from-adversary attacker-state}$
 $(\text{staged-proof-transcript data})) \text{ bad})$
 $(\text{verifier-state-from-adversary attacker-state}$
 $(\text{staged-proof-transcript data})) \leq R$
and *missing-full-bound*:
 $\wedge \text{data attacker-state.}$
 $\text{Some (data, attacker-state)} \in$
 $\text{set-dist (execute (checked-staged-transcript-program } A)$
 $\text{adversary-initial-state)} \implies$
 $\text{wp-event verify-monad}$
 $(\text{trace-fri-sampled-missing-full-cover}$
 $(\text{verifier-state-from-adversary attacker-state}$
 $(\text{staged-proof-transcript data})) \text{ bad})$
 $(\text{verifier-state-from-adversary attacker-state}$
 $(\text{staged-proof-transcript data})) \leq MF$
and *missing-sampled-bound*:
 $\wedge \text{data attacker-state.}$
 $\text{Some (data, attacker-state)} \in$
 $\text{set-dist (execute (checked-staged-transcript-program } A)$
 $\text{adversary-initial-state)} \implies$
 $\text{wp-event verify-monad}$
 $(\text{trace-fri-reachable-missing-sampled-layer-chain}$
 $(\text{verifier-state-from-adversary attacker-state}$
 $(\text{staged-proof-transcript data})))$
 $(\text{verifier-state-from-adversary attacker-state}$
 $(\text{staged-proof-transcript data})) \leq MS$
shows
 $\text{wp-event (checked-staged-security-experiment-with-data-state } A)$
 $(\text{staged-security-with-data-state-verifier-event}$
 $\text{trace-fri-bad-with-empty-composition-header-candidates})$
 $\text{adversary-initial-state} \leq (R + MF) + MS$
proof (rule
 $\text{trace-fri-bad-with-empty-header-candidates-bound-from-sampled-and-missing})$
fix *data attacker-state*
assume *builder*:
 $\text{Some (data, attacker-state)} \in$

```

    set-dist (execute (checked-staged-transcript-program A)
      adversary-initial-state)
  let ?s =
    verifier-state-from-adversary attacker-state
    (staged-proof-transcript data)
  show wp-event verify-monad
    (trace-fri-bad-with-sampled-layer-chain ?s) ?s ≤ R + MF
  by (rule wp-trace-fri-sampled-bound-from-full-cover-and-missing
    [OF full-bound[OF builder] missing-full-bound[OF builder]])
next
fix data attacker-state
assume builder:
  Some (data, attacker-state) ∈
    set-dist (execute (checked-staged-transcript-program A)
      adversary-initial-state)
show wp-event verify-monad
  (trace-fri-reachable-missing-sampled-layer-chain
    (verifier-state-from-adversary attacker-state
      (staged-proof-transcript data)))
  (verifier-state-from-adversary attacker-state
    (staged-proof-transcript data)) ≤ MS
by (rule missing-sampled-bound[OF builder])
qed

end

end

```

theory *Soundness-FRI-Full-Cover-Reasons*
imports *Soundness-FRI-Refined-Reachable*
begin

Diagnostic split for missing full sampled FRI cover.

The predicates below do not add assumptions. They expose which clause of *generic-fri-sampled-layer-chain-j* must fail when verifier sampled layer-chain evidence exists but the corresponding full-cover witness is absent. Later proof layers should bound these failures by existing Merkle/hash/proximity side events.

context *soundness*
begin

definition *generic-fri-sampled-full-cover-failure*

```

:: ('f list ⇒ bool) ⇒ 'f list ⇒ nat ⇒
  'f list ⇒ 'f list ⇒ 'f ⇒ nat list ⇒
  'f list list list ⇒ 'f list list ⇒ 'f list list ⇒
  (nat ⇒ 'f list ⇒ 'f set) ⇒ bool

```

where

```

generic-fri-sampled-full-cover-failure low-degree candidate-table
  degree-bound roots challenges final-value query-idxs round-layers

```

$\text{doms layers bad} \longleftrightarrow$
 $\neg \text{fri-multiround-proximity-steps (length challenges) degree-bound}$
 $\text{(fri-layer-lengths (length challenges) (clength * scale))}$
 $\text{(map } (\lambda i. 2 \wedge i) [0..<\text{length challenges}])$
 $\text{doms layers bad} \vee$
 $\neg \text{fri-sampled-layers-cover-fold-indices roots query-idxs} \vee$
 $\neg (\forall \text{layer-idx} < \text{length challenges.}$
 $2 \text{ dvd fri-evidence-layer-len roots layer-idx}) \vee$
 $\neg (\forall \text{layer-idx} < \text{length challenges.}$
 $\text{fri-evidence-layer-len roots layer-idx} * 2 \wedge \text{layer-idx} =$
 $\text{clength} * \text{scale}) \vee$
 $\neg (\forall \text{layer-idx} < \text{length challenges.}$
 $\text{fri-evidence-layer-len roots layer-idx} \leq$
 $\text{length (doms ! layer-idx)}) \vee$
 $\neg (\forall \text{layer-idx} < \text{length challenges.}$
 $\text{fri-evidence-layer-len roots layer-idx} \leq$
 $\text{length (layers ! layer-idx)}) \vee$
 $\neg (\forall \text{layer-idx} < \text{length challenges.}$
 $\text{fri-evidence-layer-len roots layer-idx div } 2 \leq$
 $\text{length (doms ! Suc layer-idx)}) \vee$
 $\neg (\forall \text{layer-idx} < \text{length challenges.}$
 $\text{fri-evidence-layer-len roots layer-idx div } 2 \leq$
 $\text{length (layers ! Suc layer-idx)}) \vee$
 $\neg (\forall \text{layer-idx} < \text{length challenges.}$
 $\forall \text{idx} < \text{fri-evidence-layer-len roots layer-idx div } 2.$
 $\text{doms ! Suc layer-idx ! idx} =$
 $(\text{doms ! layer-idx ! idx})^2) \vee$
 $\neg (\forall \text{layer-idx} < \text{length challenges.}$
 $\forall \text{idx} < \text{fri-evidence-layer-len roots layer-idx div } 2.$
 $\text{doms ! layer-idx ! idx} = (h \wedge \text{idx} * \text{shift}) \wedge (2 \wedge \text{layer-idx})) \vee$
 $\text{fri-table-low-degree-on degree-bound eval-domain candidate-table} \vee$
 $\neg \text{fri-table-low-degree-on}$
 $\text{(fri-degree-after (length challenges) degree-bound)}$
 $\text{(doms ! length challenges) (layers ! length challenges)}$

definition *generic-fri-full-cover-proximity-failure*

$:: \text{nat} \Rightarrow 'f \text{ list} \Rightarrow 'f \text{ list list} \Rightarrow 'f \text{ list list} \Rightarrow$
 $(\text{nat} \Rightarrow 'f \text{ list} \Rightarrow 'f \text{ set}) \Rightarrow \text{bool}$

where

$\text{generic-fri-full-cover-proximity-failure degree-bound challenges}$
 $\text{doms layers bad} \longleftrightarrow$
 $\neg \text{fri-multiround-proximity-steps (length challenges) degree-bound}$
 $\text{(fri-layer-lengths (length challenges) (clength * scale))}$
 $\text{(map } (\lambda i. 2 \wedge i) [0..<\text{length challenges}])$
 doms layers bad

definition *generic-fri-full-cover-index-failure*

$:: 'f \text{ list} \Rightarrow \text{nat list} \Rightarrow \text{bool}$

where

generic-fri-full-cover-index-failure roots query-idxs $\longleftrightarrow$
 $\neg$ *fri-sampled-layers-cover-fold-indices roots query-idxs*

definition *generic-fri-full-cover-shape-failure*
 $:: 'f \text{ list} \Rightarrow 'f \text{ list} \Rightarrow 'f \text{ list list} \Rightarrow 'f \text{ list list} \Rightarrow \text{bool}$

where

generic-fri-full-cover-shape-failure roots challenges doms layers $\longleftrightarrow$
 $\neg (\forall \text{ layer-idx} < \text{length challenges.}$
 $2 \text{ dvd fri-evidence-layer-len roots layer-idx}) \vee$
 $\neg (\forall \text{ layer-idx} < \text{length challenges.}$
 $\text{fri-evidence-layer-len roots layer-idx} * 2 \wedge \text{layer-idx} =$
 $\text{clength} * \text{scale}) \vee$
 $\neg (\forall \text{ layer-idx} < \text{length challenges.}$
 $\text{fri-evidence-layer-len roots layer-idx} \leq$
 $\text{length} (\text{doms} ! \text{layer-idx})) \vee$
 $\neg (\forall \text{ layer-idx} < \text{length challenges.}$
 $\text{fri-evidence-layer-len roots layer-idx} \leq$
 $\text{length} (\text{layers} ! \text{layer-idx})) \vee$
 $\neg (\forall \text{ layer-idx} < \text{length challenges.}$
 $\text{fri-evidence-layer-len roots layer-idx} \text{ div } 2 \leq$
 $\text{length} (\text{doms} ! \text{Suc layer-idx})) \vee$
 $\neg (\forall \text{ layer-idx} < \text{length challenges.}$
 $\text{fri-evidence-layer-len roots layer-idx} \text{ div } 2 \leq$
 $\text{length} (\text{layers} ! \text{Suc layer-idx})) \vee$
 $\text{length} (\text{doms} ! \text{length challenges}) \neq$
 $\text{length} (\text{layers} ! \text{length challenges})$

definition *generic-fri-full-cover-arithmetic-shape-failure*
 $:: 'f \text{ list} \Rightarrow 'f \text{ list} \Rightarrow \text{bool}$

where

generic-fri-full-cover-arithmetic-shape-failure roots challenges
 $\longleftrightarrow$
 $\neg (\forall \text{ layer-idx} < \text{length challenges.}$
 $2 \text{ dvd fri-evidence-layer-len roots layer-idx}) \vee$
 $\neg (\forall \text{ layer-idx} < \text{length challenges.}$
 $\text{fri-evidence-layer-len roots layer-idx} * 2 \wedge \text{layer-idx} =$
 $\text{clength} * \text{scale})$

definition *generic-fri-full-cover-witness-length-failure*
 $:: 'f \text{ list} \Rightarrow 'f \text{ list} \Rightarrow 'f \text{ list list} \Rightarrow 'f \text{ list list} \Rightarrow \text{bool}$

where

generic-fri-full-cover-witness-length-failure roots challenges doms layers
 $\longleftrightarrow$
 $\neg (\forall \text{ layer-idx} < \text{length challenges.}$
 $\text{fri-evidence-layer-len roots layer-idx} \leq$
 $\text{length} (\text{doms} ! \text{layer-idx})) \vee$
 $\neg (\forall \text{ layer-idx} < \text{length challenges.}$
 $\text{fri-evidence-layer-len roots layer-idx} \leq$
 $\text{length} (\text{layers} ! \text{layer-idx})) \vee$

$\neg (\forall \text{layer-idx} < \text{length challenges.}$
 $\text{fri-evidence-layer-len roots layer-idx div } 2 \leq$
 $\text{length (doms ! Suc layer-idx)}) \vee$
 $\neg (\forall \text{layer-idx} < \text{length challenges.}$
 $\text{fri-evidence-layer-len roots layer-idx div } 2 \leq$
 $\text{length (layers ! Suc layer-idx)}) \vee$
 $\text{length (doms ! length challenges)} \neq$
 $\text{length (layers ! length challenges)}$

lemma *generic-fri-full-cover-shape-failure-split:*
assumes *generic-fri-full-cover-shape-failure roots challenges doms layers*
shows
generic-fri-full-cover-arithmetic-shape-failure roots challenges $\vee$
generic-fri-full-cover-witness-length-failure roots challenges
doms layers
using *assms*
unfolding *generic-fri-full-cover-shape-failure-def*
generic-fri-full-cover-arithmetic-shape-failure-def
generic-fri-full-cover-witness-length-failure-def
by *blast*

lemma *ceil-log-Suc-le-power-local:*
assumes $d < 2 \wedge N$
shows $\text{ceil-log (Suc } d) \leq N$
proof (*cases* $d = 0$)
case *True*
then show *?thesis*
unfolding *ceil-log-def* **by** *simp*
next
case *False*
then have *d-pos: 0 < d*
by *simp*
have $2 \wedge \text{floor-log } d \leq d$
by (*rule floor-log-exp2-le[OF d-pos]*)
also have $\dots < 2 \wedge N$
using *assms* .
finally have $\text{floor-log } d < N$
by (*subst (asm) power-strict-increasing-iff*) *simp-all*
then show *?thesis*
using *False* **unfolding** *ceil-log-def* **by** *simp*
qed

lemma *ceil-log-le-power-bound:*
assumes $n \leq 2 \wedge N$
shows $\text{ceil-log } n \leq N$
proof (*cases* n)
case *0*
then show *?thesis*
unfolding *ceil-log-def* **by** *simp*

```

next
  case (Suc d)
  then have  $d < 2 \wedge N$ 
    using assms by simp
  then show ?thesis
    unfolding Suc by (rule ceil-log-Suc-le-power-local)
qed

lemma generic-fri-full-cover-arithmetic-shape-failure-false:
  assumes challenges-len:  $\text{length challenges} = \text{length roots}$ 
    and eval-power:  $\text{clength} * \text{scale} = 2 \wedge N$ 
    and rounds-bound:  $\text{length challenges} \leq N$ 
  shows  $\neg \text{generic-fri-full-cover-arithmetic-shape-failure roots challenges}$ 
proof -
  have even:
     $\bigwedge \text{layer-idx}. \text{layer-idx} < \text{length challenges} \implies$ 
       $2 \text{ dvd fri-evidence-layer-len roots layer-idx}$ 
  proof -
    fix layer-idx
    assume layer-bound:  $\text{layer-idx} < \text{length challenges}$ 
    have layer-root-bound:  $\text{layer-idx} < \text{length roots}$ 
      using layer-bound challenges-len by simp
    have suc-le:  $\text{Suc layer-idx} \leq N$ 
      using layer-bound rounds-bound by simp
    have  $(2::\text{nat}) \wedge \text{Suc layer-idx} \text{ dvd } (2::\text{nat}) \wedge N$ 
      by (rule le-imp-power-dvd[OF suc-le])
    then have dvd-eval:  $2 \wedge \text{Suc layer-idx} \text{ dvd } \text{clength} * \text{scale}$ 
      using eval-power by simp
    show  $2 \text{ dvd fri-evidence-layer-len roots layer-idx}$ 
      unfolding fri-evidence-layer-len-def
      by (rule fri-layer-lengths-even-if-dvd[OF layer-root-bound dvd-eval])
  qed
  have product:
     $\bigwedge \text{layer-idx}. \text{layer-idx} < \text{length challenges} \implies$ 
       $\text{fri-evidence-layer-len roots layer-idx} * 2 \wedge \text{layer-idx} =$ 
         $\text{clength} * \text{scale}$ 
  proof -
    fix layer-idx
    assume layer-bound:  $\text{layer-idx} < \text{length challenges}$ 
    have layer-root-bound:  $\text{layer-idx} < \text{length roots}$ 
      using layer-bound challenges-len by simp
    have le-N:  $\text{layer-idx} \leq N$ 
      using layer-bound rounds-bound by simp
    have  $(2::\text{nat}) \wedge \text{layer-idx} \text{ dvd } (2::\text{nat}) \wedge N$ 
      by (rule le-imp-power-dvd[OF le-N])
    then have dvd-eval:  $2 \wedge \text{layer-idx} \text{ dvd } \text{clength} * \text{scale}$ 
      using eval-power by simp
    show  $\text{fri-evidence-layer-len roots layer-idx} * 2 \wedge \text{layer-idx} =$ 
       $\text{clength} * \text{scale}$ 
  qed

```

```

unfolding fri-evidence-layer-len-def
by (rule fri-layer-lengths-round-product-if-dvd
      [OF layer-root-bound dvd-eval])
qed
show ?thesis
unfolding generic-fri-full-cover-arithmetic-shape-failure-def
using even product by simp
qed

definition generic-fri-full-cover-domain-failure
  :: 'f list  $\Rightarrow$  'f list  $\Rightarrow$  'f list list  $\Rightarrow$  bool
where
  generic-fri-full-cover-domain-failure roots challenges doms  $\longleftrightarrow$ 
     $\neg (\forall \text{layer-idx} < \text{length challenges}.$ 
       $\forall \text{idx} < \text{fri-evidence-layer-len roots layer-idx div } 2.$ 
         $\text{doms ! Suc layer-idx ! idx} =$ 
           $(\text{doms ! layer-idx ! idx})^2) \vee$ 
       $\neg (\forall \text{layer-idx} < \text{length challenges}.$ 
         $\forall \text{idx} < \text{fri-evidence-layer-len roots layer-idx div } 2.$ 
           $\text{doms ! layer-idx ! idx} = (h \wedge \text{idx} * \text{shift}) \wedge (2 \wedge \text{layer-idx}))$ 
       $)$ 

definition generic-fri-full-cover-start-low-failure
  :: nat  $\Rightarrow$  'f list  $\Rightarrow$  bool
where
  generic-fri-full-cover-start-low-failure degree-bound candidate-table
     $\longleftrightarrow$  fri-table-low-degree-on degree-bound eval-domain candidate-table

definition generic-fri-full-cover-final-low-failure
  :: nat  $\Rightarrow$  'f list  $\Rightarrow$  'f list list  $\Rightarrow$  'f list list  $\Rightarrow$  bool
where
  generic-fri-full-cover-final-low-failure degree-bound challenges
    doms layers  $\longleftrightarrow$ 
     $\neg \text{fri-table-low-degree-on}$ 
      (fri-degree-after (length challenges) degree-bound)
      (doms ! length challenges) (layers ! length challenges)

definition generic-fri-full-cover-final-length-failure
  :: 'f list  $\Rightarrow$  'f list list  $\Rightarrow$  'f list list  $\Rightarrow$  bool
where
  generic-fri-full-cover-final-length-failure challenges doms layers
     $\longleftrightarrow$ 
     $\text{length} (\text{doms ! length challenges}) \neq$ 
     $\text{length} (\text{layers ! length challenges})$ 

lemma fri-final-constant-consistent-low-degree-if-lengths:
  assumes final: fri-final-constant-consistent table final-value
  and len: length table = length fri-dom
  shows fri-table-low-degree-on d fri-dom table
proof –

```

```

have table: table = map (poly [:final-value:]) fri-dom
proof (rule nth-equalityI)
  show length table = length (map (poly [:final-value:]) fri-dom)
    using len by simp
next
fix i
assume i-bound: i < length table
have table ! i = final-value
  by (rule fri-final-constant-consistent-nth[OF final i-bound])
then show table ! i = map (poly [:final-value:]) fri-dom ! i
  using i-bound len by simp
qed
show ?thesis
  unfolding fri-table-low-degree-on-def
  using table by (intro exI[where x=:final-value:]) simp
qed

lemma generic-fri-final-low-failure-imp-length-failure:
  assumes chain:
    generic-fri-sampled-layer-chain-evidence low-degree candidate-table
    degree-bound roots challenges final-value query-idxs round-layers
    doms layers
  and final-low:
    generic-fri-full-cover-final-low-failure degree-bound challenges
    doms layers
  shows generic-fri-full-cover-final-length-failure challenges doms layers
proof (unfold generic-fri-full-cover-final-length-failure-def, rule notI)
  assume len:
    length (doms ! length challenges) =
    length (layers ! length challenges)
  have final:
    fri-final-constant-consistent (layers ! length challenges) final-value
  by (rule generic-fri-sampled-layer-chain-evidenceD(6)[OF chain])
  have low:
    fri-table-low-degree-on
    (fri-degree-after (length challenges) degree-bound)
    (doms ! length challenges) (layers ! length challenges)
  by (rule fri-final-constant-consistent-low-degree-if-lengths[OF final])
  (rule len[symmetric])
  show False
  using final-low low
  unfolding generic-fri-full-cover-final-low-failure-def by contradiction
qed

```

definition generic-fri-sampled-full-cover-failure-case

```

:: ('f list ⇒ bool) ⇒ 'f list ⇒ nat ⇒
  'f list ⇒ 'f list ⇒ 'f ⇒ nat list ⇒
  'f list list list ⇒ 'f list list ⇒ 'f list list ⇒
  (nat ⇒ 'f list ⇒ 'f set) ⇒ bool

```

where

generic-fri-sampled-full-cover-failure-case low-degree candidate-table
degree-bound roots challenges final-value query-idxs round-layers
doms layers bad $\longleftrightarrow$
generic-fri-full-cover-proximity-failure degree-bound challenges
doms layers bad $\vee$
generic-fri-full-cover-index-failure roots query-idxs $\vee$
generic-fri-full-cover-shape-failure roots challenges doms layers $\vee$
generic-fri-full-cover-domain-failure roots challenges doms $\vee$
generic-fri-full-cover-start-low-failure degree-bound candidate-table $\vee$
generic-fri-full-cover-final-low-failure degree-bound challenges
doms layers

lemma *generic-fri-sampled-full-cover-failure-iff-case:*

generic-fri-sampled-full-cover-failure low-degree candidate-table
degree-bound roots challenges final-value query-idxs round-layers
doms layers bad $\longleftrightarrow$
generic-fri-sampled-full-cover-failure-case low-degree candidate-table
degree-bound roots challenges final-value query-idxs round-layers
doms layers bad

unfolding *generic-fri-sampled-full-cover-failure-def*

generic-fri-sampled-full-cover-failure-case-def
generic-fri-full-cover-proximity-failure-def
generic-fri-full-cover-index-failure-def
generic-fri-full-cover-shape-failure-def
generic-fri-full-cover-domain-failure-def
generic-fri-full-cover-start-low-failure-def
generic-fri-full-cover-final-low-failure-def
fri-table-low-degree-on-def

by *auto*

lemma *generic-fri-sampled-full-cover-failure-cases:*

assumes

generic-fri-sampled-full-cover-failure low-degree candidate-table
degree-bound roots challenges final-value query-idxs round-layers
doms layers bad

shows

generic-fri-full-cover-proximity-failure degree-bound challenges
doms layers bad $\vee$
generic-fri-full-cover-index-failure roots query-idxs $\vee$
generic-fri-full-cover-shape-failure roots challenges doms layers $\vee$
generic-fri-full-cover-domain-failure roots challenges doms $\vee$
generic-fri-full-cover-start-low-failure degree-bound candidate-table $\vee$
generic-fri-full-cover-final-low-failure degree-bound challenges
doms layers

using *assms*

unfolding *generic-fri-sampled-full-cover-failure-iff-case*

generic-fri-sampled-full-cover-failure-case-def

by *blast*

definition *trace-fri-sampled-full-cover-failure*

$:: ('f, 'a) \text{ protocol-channel-scheme} \Rightarrow$
 $('f \text{ list} \Rightarrow \text{nat} \Rightarrow 'f \text{ list} \Rightarrow 'f \text{ set}) \Rightarrow$
 $(\text{unit list} \times ('f, 'a) \text{ protocol-channel-scheme}) \text{ option} \Rightarrow \text{bool}$

where

trace-fri-sampled-full-cover-failure $s \text{ bad out} \longleftrightarrow$
 $(\exists \text{ trace-roots trace-bs trace-final dg composition-roots}$
 $\text{composition-bs composition-final fri-query-idxs trace-round-layers}$
 $\text{composition-round-layers fr candidate-query-idxs trace-openings}$
 $\text{trace-table doms layers.}$
trace-fri-partial-candidate-opening-evidence $s \text{ out trace-roots}$
 $\text{trace-bs trace-final dg composition-roots composition-bs}$
 $\text{composition-final fri-query-idxs trace-round-layers}$
 $\text{composition-round-layers fr candidate-query-idxs trace-openings}$
 $\text{trace-table} \wedge$
generic-fri-sampled-layer-chain-evidence *trace-table-low-degree*
 $\text{trace-table (clength - 1) trace-roots trace-bs trace-final}$
 $\text{fri-query-idxs trace-round-layers doms layers} \wedge$
generic-fri-sampled-full-cover-failure *trace-table-low-degree*
 $\text{trace-table (clength - 1) trace-roots trace-bs trace-final}$
 $\text{fri-query-idxs trace-round-layers doms layers (bad trace-table)})$

definition *trace-fri-sampled-full-cover-failure-case*

$:: ('f, 'a) \text{ protocol-channel-scheme} \Rightarrow$
 $('f \text{ list} \Rightarrow \text{nat} \Rightarrow 'f \text{ list} \Rightarrow 'f \text{ set}) \Rightarrow$
 $(\text{unit list} \times ('f, 'a) \text{ protocol-channel-scheme}) \text{ option} \Rightarrow \text{bool}$

where

trace-fri-sampled-full-cover-failure-case $s \text{ bad out} \longleftrightarrow$
 $(\exists \text{ trace-roots trace-bs trace-final dg composition-roots}$
 $\text{composition-bs composition-final fri-query-idxs trace-round-layers}$
 $\text{composition-round-layers fr candidate-query-idxs trace-openings}$
 $\text{trace-table doms layers.}$
trace-fri-partial-candidate-opening-evidence $s \text{ out trace-roots}$
 $\text{trace-bs trace-final dg composition-roots composition-bs}$
 $\text{composition-final fri-query-idxs trace-round-layers}$
 $\text{composition-round-layers fr candidate-query-idxs trace-openings}$
 $\text{trace-table} \wedge$
generic-fri-sampled-layer-chain-evidence *trace-table-low-degree*
 $\text{trace-table (clength - 1) trace-roots trace-bs trace-final}$
 $\text{fri-query-idxs trace-round-layers doms layers} \wedge$
generic-fri-sampled-full-cover-failure-case *trace-table-low-degree*
 $\text{trace-table (clength - 1) trace-roots trace-bs trace-final}$
 $\text{fri-query-idxs trace-round-layers doms layers (bad trace-table)})$

definition *trace-fri-sampled-full-cover-proximity-failure*

$:: ('f, 'a) \text{ protocol-channel-scheme} \Rightarrow$
 $('f \text{ list} \Rightarrow \text{nat} \Rightarrow 'f \text{ list} \Rightarrow 'f \text{ set}) \Rightarrow$
 $(\text{unit list} \times ('f, 'a) \text{ protocol-channel-scheme}) \text{ option} \Rightarrow \text{bool}$

where

trace-fri-sampled-full-cover-proximity-failure s *bad out* $\longleftrightarrow$
 $(\exists$ *trace-roots* *trace-bs* *trace-final* *dg* *composition-roots*
composition-bs *composition-final* *fri-query-idxs* *trace-round-layers*
composition-round-layers *fr* *candidate-query-idxs* *trace-openings*
trace-table *doms* *layers*.
trace-fri-partial-candidate-opening-evidence s *out* *trace-roots*
trace-bs *trace-final* *dg* *composition-roots* *composition-bs*
composition-final *fri-query-idxs* *trace-round-layers*
composition-round-layers *fr* *candidate-query-idxs* *trace-openings*
trace-table $\wedge$
generic-fri-sampled-layer-chain-evidence *trace-table-low-degree*
trace-table $(\text{clength} - 1)$ *trace-roots* *trace-bs* *trace-final*
fri-query-idxs *trace-round-layers* *doms* *layers* $\wedge$
generic-fri-full-cover-proximity-failure $(\text{clength} - 1)$ *trace-bs*
doms *layers* (*bad* *trace-table*))

definition *trace-fri-sampled-full-cover-index-failure*

$:: (f, 'a)$ *protocol-channel-scheme* $\Rightarrow$
 $(\text{unit list} \times (f, 'a)$ *protocol-channel-scheme*) *option* $\Rightarrow$ *bool*

where

trace-fri-sampled-full-cover-index-failure s *out* $\longleftrightarrow$
 $(\exists$ *trace-roots* *trace-bs* *trace-final* *dg* *composition-roots*
composition-bs *composition-final* *fri-query-idxs* *trace-round-layers*
composition-round-layers *fr* *candidate-query-idxs* *trace-openings*
trace-table *doms* *layers*.
trace-fri-partial-candidate-opening-evidence s *out* *trace-roots*
trace-bs *trace-final* *dg* *composition-roots* *composition-bs*
composition-final *fri-query-idxs* *trace-round-layers*
composition-round-layers *fr* *candidate-query-idxs* *trace-openings*
trace-table $\wedge$
generic-fri-sampled-layer-chain-evidence *trace-table-low-degree*
trace-table $(\text{clength} - 1)$ *trace-roots* *trace-bs* *trace-final*
fri-query-idxs *trace-round-layers* *doms* *layers* $\wedge$
generic-fri-full-cover-index-failure *trace-roots* *fri-query-idxs*)

definition *trace-fri-sampled-full-cover-shape-failure*

$:: (f, 'a)$ *protocol-channel-scheme* $\Rightarrow$
 $(\text{unit list} \times (f, 'a)$ *protocol-channel-scheme*) *option* $\Rightarrow$ *bool*

where

trace-fri-sampled-full-cover-shape-failure s *out* $\longleftrightarrow$
 $(\exists$ *trace-roots* *trace-bs* *trace-final* *dg* *composition-roots*
composition-bs *composition-final* *fri-query-idxs* *trace-round-layers*
composition-round-layers *fr* *candidate-query-idxs* *trace-openings*
trace-table *doms* *layers*.
trace-fri-partial-candidate-opening-evidence s *out* *trace-roots*
trace-bs *trace-final* *dg* *composition-roots* *composition-bs*
composition-final *fri-query-idxs* *trace-round-layers*
composition-round-layers *fr* *candidate-query-idxs* *trace-openings*

$trace-table \wedge$
 $generic-fri-sampled-layer-chain-evidence \ trace-table-low-degree$
 $trace-table \ (clength - 1) \ trace-roots \ trace-bs \ trace-final$
 $fri-query-idxs \ trace-round-layers \ doms \ layers \wedge$
 $generic-fri-full-cover-shape-failure \ trace-roots \ trace-bs$
 $doms \ layers)$

definition $trace-fri-sampled-full-cover-arithmetic-shape-failure$
 $:: (f, 'a) \ protocol-channel-scheme \Rightarrow$
 $(unit \ list \times (f, 'a) \ protocol-channel-scheme) \ option \Rightarrow bool$

where

$trace-fri-sampled-full-cover-arithmetic-shape-failure \ s \ out \longleftrightarrow$
 $(\exists \ trace-roots \ trace-bs \ trace-final \ dg \ composition-roots$
 $\ \ composition-bs \ composition-final \ fri-query-idxs \ trace-round-layers$
 $\ \ composition-round-layers \ fr \ candidate-query-idxs \ trace-openings$
 $\ \ trace-table \ doms \ layers.$
 $trace-fri-partial-candidate-opening-evidence \ s \ out \ trace-roots$
 $\ trace-bs \ trace-final \ dg \ composition-roots \ composition-bs$
 $\ composition-final \ fri-query-idxs \ trace-round-layers$
 $\ composition-round-layers \ fr \ candidate-query-idxs \ trace-openings$
 $\ trace-table \wedge$
 $generic-fri-sampled-layer-chain-evidence \ trace-table-low-degree$
 $\ trace-table \ (clength - 1) \ trace-roots \ trace-bs \ trace-final$
 $\ fri-query-idxs \ trace-round-layers \ doms \ layers \wedge$
 $generic-fri-full-cover-arithmetic-shape-failure \ trace-roots \ trace-bs)$

definition $trace-fri-sampled-full-cover-witness-length-failure$
 $:: (f, 'a) \ protocol-channel-scheme \Rightarrow$
 $(unit \ list \times (f, 'a) \ protocol-channel-scheme) \ option \Rightarrow bool$

where

$trace-fri-sampled-full-cover-witness-length-failure \ s \ out \longleftrightarrow$
 $(\exists \ trace-roots \ trace-bs \ trace-final \ dg \ composition-roots$
 $\ \ composition-bs \ composition-final \ fri-query-idxs \ trace-round-layers$
 $\ \ composition-round-layers \ fr \ candidate-query-idxs \ trace-openings$
 $\ \ trace-table \ doms \ layers.$
 $trace-fri-partial-candidate-opening-evidence \ s \ out \ trace-roots$
 $\ trace-bs \ trace-final \ dg \ composition-roots \ composition-bs$
 $\ composition-final \ fri-query-idxs \ trace-round-layers$
 $\ composition-round-layers \ fr \ candidate-query-idxs \ trace-openings$
 $\ trace-table \wedge$
 $generic-fri-sampled-layer-chain-evidence \ trace-table-low-degree$
 $\ trace-table \ (clength - 1) \ trace-roots \ trace-bs \ trace-final$
 $\ fri-query-idxs \ trace-round-layers \ doms \ layers \wedge$
 $generic-fri-full-cover-witness-length-failure \ trace-roots \ trace-bs$
 $\ doms \ layers)$

definition $trace-fri-sampled-full-cover-domain-failure$
 $:: (f, 'a) \ protocol-channel-scheme \Rightarrow$
 $(unit \ list \times (f, 'a) \ protocol-channel-scheme) \ option \Rightarrow bool$

where

trace-fri-sampled-full-cover-domain-failure s out $\longleftrightarrow$
 $(\exists \text{ trace-roots trace-bs trace-final dg composition-roots}$
 $\text{composition-bs composition-final fri-query-idxs trace-round-layers}$
 $\text{composition-round-layers fr candidate-query-idxs trace-openings}$
 $\text{trace-table doms layers.}$
trace-fri-partial-candidate-opening-evidence s out trace-roots
 $\text{trace-bs trace-final dg composition-roots composition-bs}$
 $\text{composition-final fri-query-idxs trace-round-layers}$
 $\text{composition-round-layers fr candidate-query-idxs trace-openings}$
 $\text{trace-table} \wedge$
generic-fri-sampled-layer-chain-evidence trace-table-low-degree
 $\text{trace-table (clength - 1) trace-roots trace-bs trace-final}$
 $\text{fri-query-idxs trace-round-layers doms layers} \wedge$
generic-fri-full-cover-domain-failure trace-roots trace-bs doms)

definition *trace-fri-sampled-full-cover-start-low-failure*

$:: (f, 'a) \text{ protocol-channel-scheme} \Rightarrow$
 $(\text{unit list} \times (f, 'a) \text{ protocol-channel-scheme}) \text{ option} \Rightarrow \text{bool}$

where

trace-fri-sampled-full-cover-start-low-failure s out $\longleftrightarrow$
 $(\exists \text{ trace-roots trace-bs trace-final dg composition-roots}$
 $\text{composition-bs composition-final fri-query-idxs trace-round-layers}$
 $\text{composition-round-layers fr candidate-query-idxs trace-openings}$
 $\text{trace-table doms layers.}$
trace-fri-partial-candidate-opening-evidence s out trace-roots
 $\text{trace-bs trace-final dg composition-roots composition-bs}$
 $\text{composition-final fri-query-idxs trace-round-layers}$
 $\text{composition-round-layers fr candidate-query-idxs trace-openings}$
 $\text{trace-table} \wedge$
generic-fri-sampled-layer-chain-evidence trace-table-low-degree
 $\text{trace-table (clength - 1) trace-roots trace-bs trace-final}$
 $\text{fri-query-idxs trace-round-layers doms layers} \wedge$
generic-fri-full-cover-start-low-failure (clength - 1) trace-table)

definition *trace-fri-sampled-full-cover-final-low-failure*

$:: (f, 'a) \text{ protocol-channel-scheme} \Rightarrow$
 $(\text{unit list} \times (f, 'a) \text{ protocol-channel-scheme}) \text{ option} \Rightarrow \text{bool}$

where

trace-fri-sampled-full-cover-final-low-failure s out $\longleftrightarrow$
 $(\exists \text{ trace-roots trace-bs trace-final dg composition-roots}$
 $\text{composition-bs composition-final fri-query-idxs trace-round-layers}$
 $\text{composition-round-layers fr candidate-query-idxs trace-openings}$
 $\text{trace-table doms layers.}$
trace-fri-partial-candidate-opening-evidence s out trace-roots
 $\text{trace-bs trace-final dg composition-roots composition-bs}$
 $\text{composition-final fri-query-idxs trace-round-layers}$
 $\text{composition-round-layers fr candidate-query-idxs trace-openings}$
 $\text{trace-table} \wedge$

generic-fri-sampled-layer-chain-evidence trace-table-low-degree
trace-table (clength - 1) trace-roots trace-bs trace-final
fri-query-idxs trace-round-layers doms layers $\wedge$
generic-fri-full-cover-final-low-failure (clength - 1) trace-bs
doms layers)

definition *trace-fri-sampled-full-cover-final-length-failure*

$:: ('f, 'a) \text{ protocol-channel-scheme} \Rightarrow$
 $(\text{unit list} \times ('f, 'a) \text{ protocol-channel-scheme}) \text{ option} \Rightarrow \text{bool}$

where

trace-fri-sampled-full-cover-final-length-failure s out $\longleftrightarrow$
 $(\exists \text{ trace-roots trace-bs trace-final dg composition-roots}$
 $\text{ composition-bs composition-final fri-query-idxs trace-round-layers}$
 $\text{ composition-round-layers fr candidate-query-idxs trace-openings}$
 $\text{ trace-table doms layers.}$
trace-fri-partial-candidate-opening-evidence s out trace-roots
 $\text{ trace-bs trace-final dg composition-roots composition-bs}$
 $\text{ composition-final fri-query-idxs trace-round-layers}$
 $\text{ composition-round-layers fr candidate-query-idxs trace-openings}$
 $\text{ trace-table} \wedge$
generic-fri-sampled-layer-chain-evidence trace-table-low-degree
 $\text{ trace-table (clength - 1) trace-roots trace-bs trace-final}$
 $\text{ fri-query-idxs trace-round-layers doms layers} \wedge$
generic-fri-full-cover-final-length-failure trace-bs doms layers)

definition *composition-fri-sampled-full-cover-failure*

$:: ('f, 'a) \text{ protocol-channel-scheme} \Rightarrow$
 $(('f \Rightarrow 'f \text{ list} \Rightarrow \text{nat} \Rightarrow 'f \text{ list} \Rightarrow 'f \text{ set}) \Rightarrow$
 $(\text{unit list} \times ('f, 'a) \text{ protocol-channel-scheme}) \text{ option} \Rightarrow \text{bool}$

where

composition-fri-sampled-full-cover-failure s bad out $\longleftrightarrow$
 $(\exists \text{ trace-roots trace-bs trace-final fri-dg}$
 $\text{ opening-composition-roots composition-bs composition-final}$
 $\text{ fri-query-idxs trace-round-layers composition-round-layers fr}$
 $\text{ f-fri-roots f-final as dg composition-fri-roots final}$
 $\text{ trace-query-idxs trace-openings composition-query-idxs}$
 $\text{ composition-openings trace-table composition-table doms layers.}$
composition-fri-partial-candidate-opening-evidence s out trace-roots
 $\text{ trace-bs trace-final fri-dg opening-composition-roots composition-bs}$
 $\text{ composition-final fri-query-idxs trace-round-layers}$
 $\text{ composition-round-layers fr f-fri-roots f-final as dg}$
 $\text{ composition-fri-roots final trace-query-idxs trace-openings}$
 $\text{ composition-query-idxs composition-openings trace-table}$
 $\text{ composition-table} \wedge$
generic-fri-sampled-layer-chain-evidence
 $(\text{composition-table-low-degree (to-nat fri-dg)}) \text{ composition-table}$
 $(\text{to-nat fri-dg}) \text{ opening-composition-roots composition-bs}$
 $\text{ composition-final fri-query-idxs composition-round-layers doms layers}$
 $\wedge$

generic-fri-sampled-full-cover-failure
 (composition-table-low-degree (to-nat fri-dg)) composition-table
 (to-nat fri-dg) opening-composition-roots composition-bs
 composition-final fri-query-idxs composition-round-layers doms layers
 (bad fri-dg composition-table))

definition *composition-fri-sampled-full-cover-failure-case*

:: (f, 'a) protocol-channel-scheme $\Rightarrow$
 (f $\Rightarrow$ 'f list $\Rightarrow$ nat $\Rightarrow$ 'f list $\Rightarrow$ 'f set) $\Rightarrow$
 (unit list $\times$ (f, 'a) protocol-channel-scheme) option $\Rightarrow$ bool

where

composition-fri-sampled-full-cover-failure-case s bad out $\longleftrightarrow$
 ($\exists$ trace-roots trace-bs trace-final fri-dg
 opening-composition-roots composition-bs composition-final
 fri-query-idxs trace-round-layers composition-round-layers fr
 f-fri-roots f-final as dg composition-fri-roots final
 trace-query-idxs trace-openings composition-query-idxs
 composition-openings trace-table composition-table doms layers.
 composition-fri-partial-candidate-opening-evidence s out trace-roots
 trace-bs trace-final fri-dg opening-composition-roots composition-bs
 composition-final fri-query-idxs trace-round-layers
 composition-round-layers fr f-fri-roots f-final as dg
 composition-fri-roots final trace-query-idxs trace-openings
 composition-query-idxs composition-openings trace-table
 composition-table $\wedge$
 generic-fri-sampled-layer-chain-evidence
 (composition-table-low-degree (to-nat fri-dg)) composition-table
 (to-nat fri-dg) opening-composition-roots composition-bs
 composition-final fri-query-idxs composition-round-layers doms layers
 $\wedge$
 generic-fri-sampled-full-cover-failure-case
 (composition-table-low-degree (to-nat fri-dg)) composition-table
 (to-nat fri-dg) opening-composition-roots composition-bs
 composition-final fri-query-idxs composition-round-layers doms layers
 (bad fri-dg composition-table))

definition *composition-fri-sampled-full-cover-proximity-failure*

:: (f, 'a) protocol-channel-scheme $\Rightarrow$
 (f $\Rightarrow$ 'f list $\Rightarrow$ nat $\Rightarrow$ 'f list $\Rightarrow$ 'f set) $\Rightarrow$
 (unit list $\times$ (f, 'a) protocol-channel-scheme) option $\Rightarrow$ bool

where

composition-fri-sampled-full-cover-proximity-failure s bad out $\longleftrightarrow$
 ($\exists$ trace-roots trace-bs trace-final fri-dg
 opening-composition-roots composition-bs composition-final
 fri-query-idxs trace-round-layers composition-round-layers fr
 f-fri-roots f-final as dg composition-fri-roots final
 trace-query-idxs trace-openings composition-query-idxs
 composition-openings trace-table composition-table doms layers.
 composition-fri-partial-candidate-opening-evidence s out trace-roots

trace-bs trace-final fri-dg opening-composition-roots composition-bs
composition-final fri-query-idxs trace-round-layers
composition-round-layers fr f-fri-roots f-final as dg
composition-fri-roots final trace-query-idxs trace-openings
composition-query-idxs composition-openings trace-table
composition-table $\wedge$
generic-fri-sampled-layer-chain-evidence
(composition-table-low-degree (to-nat fri-dg)) composition-table
(to-nat fri-dg) opening-composition-roots composition-bs
composition-final fri-query-idxs composition-round-layers doms layers
 $\wedge$
generic-fri-full-cover-proximity-failure (to-nat fri-dg)
composition-bs doms layers (bad fri-dg composition-table))

definition *composition-fri-sampled-full-cover-index-failure*

$:: ('f, 'a) \text{ protocol-channel-scheme} \Rightarrow$
 $(\text{unit list} \times ('f, 'a) \text{ protocol-channel-scheme}) \text{ option} \Rightarrow \text{bool}$

where

composition-fri-sampled-full-cover-index-failure s out $\longleftrightarrow$
 $(\exists \text{ trace-roots trace-bs trace-final fri-dg}$
opening-composition-roots composition-bs composition-final
fri-query-idxs trace-round-layers composition-round-layers fr
f-fri-roots f-final as dg composition-fri-roots final
trace-query-idxs trace-openings composition-query-idxs
composition-openings trace-table composition-table doms layers.
composition-fri-partial-candidate-opening-evidence s out trace-roots
trace-bs trace-final fri-dg opening-composition-roots composition-bs
composition-final fri-query-idxs trace-round-layers
composition-round-layers fr f-fri-roots f-final as dg
composition-fri-roots final trace-query-idxs trace-openings
composition-query-idxs composition-openings trace-table
composition-table $\wedge$
generic-fri-sampled-layer-chain-evidence
(composition-table-low-degree (to-nat fri-dg)) composition-table
(to-nat fri-dg) opening-composition-roots composition-bs
composition-final fri-query-idxs composition-round-layers doms layers
 $\wedge$
generic-fri-full-cover-index-failure opening-composition-roots
fri-query-idxs)

definition *composition-fri-sampled-full-cover-shape-failure*

$:: ('f, 'a) \text{ protocol-channel-scheme} \Rightarrow$
 $(\text{unit list} \times ('f, 'a) \text{ protocol-channel-scheme}) \text{ option} \Rightarrow \text{bool}$

where

composition-fri-sampled-full-cover-shape-failure s out $\longleftrightarrow$
 $(\exists \text{ trace-roots trace-bs trace-final fri-dg}$
opening-composition-roots composition-bs composition-final
fri-query-idxs trace-round-layers composition-round-layers fr
f-fri-roots f-final as dg composition-fri-roots final

trace-query-idxs trace-openings composition-query-idxs
composition-openings trace-table composition-table doms layers.
composition-fri-partial-candidate-opening-evidence s out trace-roots
trace-bs trace-final fri-dg opening-composition-roots composition-bs
composition-final fri-query-idxs trace-round-layers
composition-round-layers fr f-fri-roots f-final as dg
composition-fri-roots final trace-query-idxs trace-openings
composition-query-idxs composition-openings trace-table
composition-table $\wedge$
generic-fri-sampled-layer-chain-evidence
(composition-table-low-degree (to-nat fri-dg)) composition-table
(to-nat fri-dg) opening-composition-roots composition-bs
composition-final fri-query-idxs composition-round-layers doms layers
 $\wedge$
generic-fri-full-cover-shape-failure opening-composition-roots
composition-bs doms layers)

definition *composition-fri-sampled-full-cover-arithmetic-shape-failure*

$:: ('f, 'a) \text{ protocol-channel-scheme} \Rightarrow$
 $(\text{unit list} \times ('f, 'a) \text{ protocol-channel-scheme}) \text{ option} \Rightarrow \text{bool}$

where

composition-fri-sampled-full-cover-arithmetic-shape-failure s out
 $\longleftrightarrow$
 $(\exists \text{ trace-roots trace-bs trace-final fri-dg}$
opening-composition-roots composition-bs composition-final
fri-query-idxs trace-round-layers composition-round-layers fr
f-fri-roots f-final as dg composition-fri-roots final
trace-query-idxs trace-openings composition-query-idxs
composition-openings trace-table composition-table doms layers.
composition-fri-partial-candidate-opening-evidence s out trace-roots
trace-bs trace-final fri-dg opening-composition-roots composition-bs
composition-final fri-query-idxs trace-round-layers
composition-round-layers fr f-fri-roots f-final as dg
composition-fri-roots final trace-query-idxs trace-openings
composition-query-idxs composition-openings trace-table
composition-table $\wedge$
generic-fri-sampled-layer-chain-evidence
(composition-table-low-degree (to-nat fri-dg)) composition-table
(to-nat fri-dg) opening-composition-roots composition-bs
composition-final fri-query-idxs composition-round-layers doms layers
 $\wedge$
generic-fri-full-cover-arithmetic-shape-failure
opening-composition-roots composition-bs)

definition *composition-fri-sampled-full-cover-witness-length-failure*

$:: ('f, 'a) \text{ protocol-channel-scheme} \Rightarrow$
 $(\text{unit list} \times ('f, 'a) \text{ protocol-channel-scheme}) \text{ option} \Rightarrow \text{bool}$

where

composition-fri-sampled-full-cover-witness-length-failure s out

$\longleftrightarrow$

($\exists$ trace-roots trace-bs trace-final fri-dg
 opening-composition-roots composition-bs composition-final
 fri-query-idxs trace-round-layers composition-round-layers fr
 f-fri-roots f-final as dg composition-fri-roots final
 trace-query-idxs trace-openings composition-query-idxs
 composition-openings trace-table composition-table doms layers.
 composition-fri-partial-candidate-opening-evidence s out trace-roots
 trace-bs trace-final fri-dg opening-composition-roots composition-bs
 composition-final fri-query-idxs trace-round-layers
 composition-round-layers fr f-fri-roots f-final as dg
 composition-fri-roots final trace-query-idxs trace-openings
 composition-query-idxs composition-openings trace-table
 composition-table $\wedge$
 generic-fri-sampled-layer-chain-evidence
 (composition-table-low-degree (to-nat fri-dg)) composition-table
 (to-nat fri-dg) opening-composition-roots composition-bs
 composition-final fri-query-idxs composition-round-layers doms layers
 $\wedge$
 generic-fri-full-cover-witness-length-failure
 opening-composition-roots composition-bs doms layers)

definition composition-fri-sampled-full-cover-domain-failure

$:: (f, 'a) \text{ protocol-channel-scheme} \Rightarrow$
 (unit list $\times$ ($f, 'a$) protocol-channel-scheme) option $\Rightarrow$ bool

where

composition-fri-sampled-full-cover-domain-failure s out $\longleftrightarrow$
 ($\exists$ trace-roots trace-bs trace-final fri-dg
 opening-composition-roots composition-bs composition-final
 fri-query-idxs trace-round-layers composition-round-layers fr
 f-fri-roots f-final as dg composition-fri-roots final
 trace-query-idxs trace-openings composition-query-idxs
 composition-openings trace-table composition-table doms layers.
 composition-fri-partial-candidate-opening-evidence s out trace-roots
 trace-bs trace-final fri-dg opening-composition-roots composition-bs
 composition-final fri-query-idxs trace-round-layers
 composition-round-layers fr f-fri-roots f-final as dg
 composition-fri-roots final trace-query-idxs trace-openings
 composition-query-idxs composition-openings trace-table
 composition-table $\wedge$
 generic-fri-sampled-layer-chain-evidence
 (composition-table-low-degree (to-nat fri-dg)) composition-table
 (to-nat fri-dg) opening-composition-roots composition-bs
 composition-final fri-query-idxs composition-round-layers doms layers
 $\wedge$
 generic-fri-full-cover-domain-failure opening-composition-roots
 composition-bs doms)

definition composition-fri-sampled-full-cover-start-low-failure

$:: ('f, 'a) \text{ protocol-channel-scheme} \Rightarrow$
 $(\text{unit list} \times ('f, 'a) \text{ protocol-channel-scheme}) \text{ option} \Rightarrow \text{bool}$
where
 $\text{composition-fri-sampled-full-cover-start-low-failure } s \text{ out} \longleftrightarrow$
 $(\exists \text{ trace-roots trace-bs trace-final fri-dg}$
 $\text{ opening-composition-roots composition-bs composition-final}$
 $\text{ fri-query-idxs trace-round-layers composition-round-layers fr}$
 $\text{ f-fri-roots f-final as dg composition-fri-roots final}$
 $\text{ trace-query-idxs trace-openings composition-query-idxs}$
 $\text{ composition-openings trace-table composition-table doms layers.}$
 $\text{composition-fri-partial-candidate-opening-evidence } s \text{ out trace-roots}$
 $\text{ trace-bs trace-final fri-dg opening-composition-roots composition-bs}$
 $\text{ composition-final fri-query-idxs trace-round-layers}$
 $\text{ composition-round-layers fr f-fri-roots f-final as dg}$
 $\text{ composition-fri-roots final trace-query-idxs trace-openings}$
 $\text{ composition-query-idxs composition-openings trace-table}$
 $\text{ composition-table} \wedge$
 $\text{generic-fri-sampled-layer-chain-evidence}$
 $(\text{composition-table-low-degree } (\text{to-nat fri-dg})) \text{ composition-table}$
 $(\text{to-nat fri-dg}) \text{ opening-composition-roots composition-bs}$
 $\text{ composition-final fri-query-idxs composition-round-layers doms layers}$
 $\wedge$
 $\text{generic-fri-full-cover-start-low-failure } (\text{to-nat fri-dg})$
 $\text{ composition-table})$

definition *composition-fri-sampled-full-cover-final-low-failure*

$:: ('f, 'a) \text{ protocol-channel-scheme} \Rightarrow$
 $(\text{unit list} \times ('f, 'a) \text{ protocol-channel-scheme}) \text{ option} \Rightarrow \text{bool}$

where

$\text{composition-fri-sampled-full-cover-final-low-failure } s \text{ out} \longleftrightarrow$
 $(\exists \text{ trace-roots trace-bs trace-final fri-dg}$
 $\text{ opening-composition-roots composition-bs composition-final}$
 $\text{ fri-query-idxs trace-round-layers composition-round-layers fr}$
 $\text{ f-fri-roots f-final as dg composition-fri-roots final}$
 $\text{ trace-query-idxs trace-openings composition-query-idxs}$
 $\text{ composition-openings trace-table composition-table doms layers.}$
 $\text{composition-fri-partial-candidate-opening-evidence } s \text{ out trace-roots}$
 $\text{ trace-bs trace-final fri-dg opening-composition-roots composition-bs}$
 $\text{ composition-final fri-query-idxs trace-round-layers}$
 $\text{ composition-round-layers fr f-fri-roots f-final as dg}$
 $\text{ composition-fri-roots final trace-query-idxs trace-openings}$
 $\text{ composition-query-idxs composition-openings trace-table}$
 $\text{ composition-table} \wedge$
 $\text{generic-fri-sampled-layer-chain-evidence}$
 $(\text{composition-table-low-degree } (\text{to-nat fri-dg})) \text{ composition-table}$
 $(\text{to-nat fri-dg}) \text{ opening-composition-roots composition-bs}$
 $\text{ composition-final fri-query-idxs composition-round-layers doms layers}$
 $\wedge$
 $\text{generic-fri-full-cover-final-low-failure } (\text{to-nat fri-dg})$

composition-bs doms layers)

definition *composition-fri-sampled-full-cover-final-length-failure*

$:: ('f, 'a) \text{ protocol-channel-scheme} \Rightarrow$
 $(\text{unit list} \times ('f, 'a) \text{ protocol-channel-scheme}) \text{ option} \Rightarrow \text{bool}$

where

composition-fri-sampled-full-cover-final-length-failure $s \text{ out} \longleftrightarrow$
 $(\exists \text{ trace-roots trace-bs trace-final fri-dg}$
 $\text{ opening-composition-roots composition-bs composition-final}$
 $\text{ fri-query-idxs trace-round-layers composition-round-layers fr}$
 $\text{ f-fri-roots f-final as dg composition-fri-roots final}$
 $\text{ trace-query-idxs trace-openings composition-query-idxs}$
 $\text{ composition-openings trace-table composition-table doms layers.}$
 $\text{ composition-fri-partial-candidate-opening-evidence } s \text{ out trace-roots}$
 $\text{ trace-bs trace-final fri-dg opening-composition-roots composition-bs}$
 $\text{ composition-final fri-query-idxs trace-round-layers}$
 $\text{ composition-round-layers fr f-fri-roots f-final as dg}$
 $\text{ composition-fri-roots final trace-query-idxs trace-openings}$
 $\text{ composition-query-idxs composition-openings trace-table}$
 $\text{ composition-table} \wedge$
 $\text{ generic-fri-sampled-layer-chain-evidence}$
 $(\text{ composition-table-low-degree (to-nat fri-dg) }) \text{ composition-table}$
 $(\text{ to-nat fri-dg }) \text{ opening-composition-roots composition-bs}$
 $\text{ composition-final fri-query-idxs composition-round-layers doms layers}$
 $\wedge$
 $\text{ generic-fri-full-cover-final-length-failure composition-bs doms}$
 $\text{ layers})$

lemma *trace-fri-sampled-full-cover-failure-iff-case:*

trace-fri-sampled-full-cover-failure $s \text{ bad out} \longleftrightarrow$
trace-fri-sampled-full-cover-failure-case $s \text{ bad out}$

unfolding *trace-fri-sampled-full-cover-failure-def*

trace-fri-sampled-full-cover-failure-case-def

by (*simp only: generic-fri-sampled-full-cover-failure-iff-case*)

lemma *composition-fri-sampled-full-cover-failure-iff-case:*

composition-fri-sampled-full-cover-failure $s \text{ bad out} \longleftrightarrow$
composition-fri-sampled-full-cover-failure-case $s \text{ bad out}$

unfolding *composition-fri-sampled-full-cover-failure-def*

composition-fri-sampled-full-cover-failure-case-def

by (*simp only: generic-fri-sampled-full-cover-failure-iff-case*)

lemma *trace-fri-sampled-full-cover-failure-case-split:*

assumes *trace-fri-sampled-full-cover-failure-case* $s \text{ bad out}$

shows

trace-fri-sampled-full-cover-proximity-failure $s \text{ bad out} \vee$

trace-fri-sampled-full-cover-index-failure $s \text{ out} \vee$

trace-fri-sampled-full-cover-shape-failure $s \text{ out} \vee$

trace-fri-sampled-full-cover-domain-failure $s \text{ out} \vee$

trace-fri-sampled-full-cover-start-low-failure s out $\vee$
trace-fri-sampled-full-cover-final-low-failure s out
using *assms*
unfolding *trace-fri-sampled-full-cover-failure-case-def*
trace-fri-sampled-full-cover-proximity-failure-def
trace-fri-sampled-full-cover-index-failure-def
trace-fri-sampled-full-cover-shape-failure-def
trace-fri-sampled-full-cover-domain-failure-def
trace-fri-sampled-full-cover-start-low-failure-def
trace-fri-sampled-full-cover-final-low-failure-def
generic-fri-sampled-full-cover-failure-case-def
by *meson*

lemma *composition-fri-sampled-full-cover-failure-case-split:*
assumes *composition-fri-sampled-full-cover-failure-case s bad out*
shows
composition-fri-sampled-full-cover-proximity-failure s bad out $\vee$
composition-fri-sampled-full-cover-index-failure s out $\vee$
composition-fri-sampled-full-cover-shape-failure s out $\vee$
composition-fri-sampled-full-cover-domain-failure s out $\vee$
composition-fri-sampled-full-cover-start-low-failure s out $\vee$
composition-fri-sampled-full-cover-final-low-failure s out
using *assms*
unfolding *composition-fri-sampled-full-cover-failure-case-def*
composition-fri-sampled-full-cover-proximity-failure-def
composition-fri-sampled-full-cover-index-failure-def
composition-fri-sampled-full-cover-shape-failure-def
composition-fri-sampled-full-cover-domain-failure-def
composition-fri-sampled-full-cover-start-low-failure-def
composition-fri-sampled-full-cover-final-low-failure-def
generic-fri-sampled-full-cover-failure-case-def
by *meson*

lemma *trace-fri-sampled-full-cover-shape-failure-split:*
assumes *trace-fri-sampled-full-cover-shape-failure s out*
shows
trace-fri-sampled-full-cover-arithmetic-shape-failure s out $\vee$
trace-fri-sampled-full-cover-witness-length-failure s out

proof —

from *assms* **obtain** *trace-roots trace-bs trace-final dg composition-roots*
composition-bs composition-final fri-query-idxs trace-round-layers
composition-round-layers fr candidate-query-idxs trace-openings
trace-table doms layers
where *evidence:*
trace-fri-partial-candidate-opening-evidence s out trace-roots
trace-bs trace-final dg composition-roots composition-bs
composition-final fri-query-idxs trace-round-layers
composition-round-layers fr candidate-query-idxs trace-openings
trace-table

and chain:
generic-fri-sampled-layer-chain-evidence trace-table-low-degree
trace-table (clength - 1) trace-roots trace-bs trace-final
fri-query-idxs trace-round-layers doms layers
and shape:
generic-fri-full-cover-shape-failure trace-roots trace-bs
doms layers
unfolding *trace-fri-sampled-full-cover-shape-failure-def* **by** *blast*
have *split:*
generic-fri-full-cover-arithmetic-shape-failure trace-roots trace-bs $\vee$
generic-fri-full-cover-witness-length-failure trace-roots trace-bs
doms layers
by (*rule generic-fri-full-cover-shape-failure-split[OF shape]*)
then show *?thesis*
unfolding *trace-fri-sampled-full-cover-arithmetic-shape-failure-def*
trace-fri-sampled-full-cover-witness-length-failure-def
using *evidence chain* **by** *meson*
qed

lemma *composition-fri-sampled-full-cover-shape-failure-split:*
assumes *composition-fri-sampled-full-cover-shape-failure s out*
shows
composition-fri-sampled-full-cover-arithmetic-shape-failure s out $\vee$
composition-fri-sampled-full-cover-witness-length-failure s out
proof —
from *assms* **obtain** *trace-roots trace-bs trace-final fri-dg*
opening-composition-roots composition-bs composition-final
fri-query-idxs trace-round-layers composition-round-layers fr
f-fri-roots f-final as dg composition-fri-roots final
trace-query-idxs trace-openings composition-query-idxs
composition-openings trace-table composition-table doms layers
where *evidence:*
composition-fri-partial-candidate-opening-evidence s out trace-roots
trace-bs trace-final fri-dg opening-composition-roots composition-bs
composition-final fri-query-idxs trace-round-layers
composition-round-layers fr f-fri-roots f-final as dg
composition-fri-roots final trace-query-idxs trace-openings
composition-query-idxs composition-openings trace-table
composition-table
and chain:
generic-fri-sampled-layer-chain-evidence
(composition-table-low-degree (to-nat fri-dg)) composition-table
(to-nat fri-dg) opening-composition-roots composition-bs
composition-final fri-query-idxs composition-round-layers doms layers
and shape:
generic-fri-full-cover-shape-failure opening-composition-roots
composition-bs doms layers
unfolding *composition-fri-sampled-full-cover-shape-failure-def*
by *meson*

have *split*:
generic-fri-full-cover-arithmetic-shape-failure
opening-composition-roots composition-bs $\vee$
generic-fri-full-cover-witness-length-failure
opening-composition-roots composition-bs doms layers
by (rule *generic-fri-full-cover-shape-failure-split*[*OF shape*])
then show *?thesis*
unfolding
composition-fri-sampled-full-cover-arithmetic-shape-failure-def
composition-fri-sampled-full-cover-witness-length-failure-def
using *evidence chain* **by** *meson*
qed

lemma *wp-trace-fri-sampled-full-cover-shape-bound-from-split*:
assumes *arithmetic*:
wp-event verify-monad
(trace-fri-sampled-full-cover-arithmetic-shape-failure s) $s \leq A$
and *witness*:
wp-event verify-monad
(trace-fri-sampled-full-cover-witness-length-failure s) $s \leq W$
shows
wp-event verify-monad
(trace-fri-sampled-full-cover-shape-failure s) $s \leq A + W$
proof –
have *wp-event verify-monad*
(trace-fri-sampled-full-cover-shape-failure s) s
 $\leq$ *wp-event verify-monad*
 $(\lambda out.$
trace-fri-sampled-full-cover-arithmetic-shape-failure s out $\vee$
trace-fri-sampled-full-cover-witness-length-failure s out) s
by (rule *wp-event-mono*)
(rule *trace-fri-sampled-full-cover-shape-failure-split*)
also have $\dots \leq A + W$
by (rule *order-trans*[*OF wp-event-union-bound*
add-mono[*OF arithmetic witness*]])
finally show *?thesis* .
qed

lemma *wp-composition-fri-sampled-full-cover-shape-bound-from-split*:
assumes *arithmetic*:
wp-event verify-monad
(composition-fri-sampled-full-cover-arithmetic-shape-failure s) s
 $\leq A$
and *witness*:
wp-event verify-monad
(composition-fri-sampled-full-cover-witness-length-failure s) s
 $\leq W$
shows
wp-event verify-monad

$(\text{composition-fri-sampled-full-cover-shape-failure } s) \ s \leq A + W$
proof –
have $\text{wp-event verify-monad}$
 $(\text{composition-fri-sampled-full-cover-shape-failure } s) \ s$
 $\leq \text{wp-event verify-monad}$
 $(\lambda \text{out.}$
 $\text{composition-fri-sampled-full-cover-arithmetic-shape-failure } s \ \text{out} \vee$
 $\text{composition-fri-sampled-full-cover-witness-length-failure } s \ \text{out}) \ s$
by $(\text{rule wp-event-mono})$
 $(\text{rule composition-fri-sampled-full-cover-shape-failure-split})$
also have $\dots \leq A + W$
by $(\text{rule order-trans}[\text{OF wp-event-union-bound}$
 $\text{add-mono}[\text{OF arithmetic witness}]])$
finally show $?thesis$.
qed

lemma $\text{wp-event-six-union-bound}$:
assumes $A: \text{wp-event } m \ A \ s \leq a$
and $B: \text{wp-event } m \ B \ s \leq b$
and $C: \text{wp-event } m \ C \ s \leq c$
and $D: \text{wp-event } m \ D \ s \leq d$
and $E: \text{wp-event } m \ E \ s \leq e$
and $F: \text{wp-event } m \ F \ s \leq f$
shows $\text{wp-event } m$
 $(\lambda x. A \ x \vee B \ x \vee C \ x \vee D \ x \vee E \ x \vee F \ x) \ s$
 $\leq a + b + c + d + e + f$
proof –
have EF :
 $\text{wp-event } m \ (\lambda x. E \ x \vee F \ x) \ s \leq e + f$
by $(\text{rule order-trans}[\text{OF wp-event-union-bound add-mono}[\text{OF } E \ F]])$
have DEF :
 $\text{wp-event } m \ (\lambda x. D \ x \vee E \ x \vee F \ x) \ s \leq d + e + f$
proof –
have $\text{wp-event } m \ (\lambda x. D \ x \vee E \ x \vee F \ x) \ s$
 $\leq \text{wp-event } m \ D \ s + \text{wp-event } m \ (\lambda x. E \ x \vee F \ x) \ s$
by $(\text{rule wp-event-union-bound})$
also have $\dots \leq d + (e + f)$
by $(\text{rule add-mono}[\text{OF } D \ EF])$
finally show $?thesis$
by $(\text{simp add: add.assoc})$
qed
have $CDEF$:
 $\text{wp-event } m \ (\lambda x. C \ x \vee D \ x \vee E \ x \vee F \ x) \ s$
 $\leq c + d + e + f$
proof –
have $\text{wp-event } m \ (\lambda x. C \ x \vee D \ x \vee E \ x \vee F \ x) \ s$
 $\leq \text{wp-event } m \ C \ s + \text{wp-event } m \ (\lambda x. D \ x \vee E \ x \vee F \ x) \ s$
by $(\text{rule wp-event-union-bound})$
also have $\dots \leq c + (d + e + f)$

```

    by (rule add-mono[OF C DEF])
  finally show ?thesis
    by (simp add: add.assoc)
qed
have BCDEF:
  wp-event m ( $\lambda x. B\ x \vee C\ x \vee D\ x \vee E\ x \vee F\ x$ ) s
     $\leq b + c + d + e + f$ 
proof -
  have wp-event m ( $\lambda x. B\ x \vee C\ x \vee D\ x \vee E\ x \vee F\ x$ ) s
     $\leq$  wp-event m B s +
      wp-event m ( $\lambda x. C\ x \vee D\ x \vee E\ x \vee F\ x$ ) s
    by (rule wp-event-union-bound)
  also have ...  $\leq b + (c + d + e + f)$ 
    by (rule add-mono[OF B CDEF])
  finally show ?thesis
    by (simp add: add.assoc)
qed
have wp-event m
  ( $\lambda x. A\ x \vee B\ x \vee C\ x \vee D\ x \vee E\ x \vee F\ x$ ) s
     $\leq$  wp-event m A s +
      wp-event m ( $\lambda x. B\ x \vee C\ x \vee D\ x \vee E\ x \vee F\ x$ ) s
    by (rule wp-event-union-bound)
  also have ...  $\leq a + (b + c + d + e + f)$ 
    by (rule add-mono[OF A BCDEF])
  finally show ?thesis
    by (simp add: add.assoc)
qed

lemma wp-trace-fri-sampled-full-cover-failure-case-union-bound:
  assumes proximity:
    wp-event verify-monad
      (trace-fri-sampled-full-cover-proximity-failure s bad) s  $\leq P$ 
  and index:
    wp-event verify-monad
      (trace-fri-sampled-full-cover-index-failure s) s  $\leq I$ 
  and shape:
    wp-event verify-monad
      (trace-fri-sampled-full-cover-shape-failure s) s  $\leq Sh$ 
  and domain:
    wp-event verify-monad
      (trace-fri-sampled-full-cover-domain-failure s) s  $\leq D$ 
  and start-low:
    wp-event verify-monad
      (trace-fri-sampled-full-cover-start-low-failure s) s  $\leq SL$ 
  and final-low:
    wp-event verify-monad
      (trace-fri-sampled-full-cover-final-low-failure s) s  $\leq FL$ 
  shows
    wp-event verify-monad

```

$(\text{trace-fri-sampled-full-cover-failure-case } s \text{ bad}) \ s$
 $\leq P + I + Sh + D + SL + FL$
proof –
have *step1*:
 $\text{wp-event verify-monad}$
 $(\text{trace-fri-sampled-full-cover-failure-case } s \text{ bad}) \ s$
 $\leq$
 $\text{wp-event verify-monad}$
 $(\lambda \text{out. trace-fri-sampled-full-cover-proximity-failure } s \text{ bad out } \vee$
 $\text{trace-fri-sampled-full-cover-index-failure } s \text{ out } \vee$
 $\text{trace-fri-sampled-full-cover-shape-failure } s \text{ out } \vee$
 $\text{trace-fri-sampled-full-cover-domain-failure } s \text{ out } \vee$
 $\text{trace-fri-sampled-full-cover-start-low-failure } s \text{ out } \vee$
 $\text{trace-fri-sampled-full-cover-final-low-failure } s \text{ out}) \ s$
by (rule *wp-event-mono*)
(rule *trace-fri-sampled-full-cover-failure-case-split*)
have *step2*:
 $\text{wp-event verify-monad}$
 $(\lambda \text{out. trace-fri-sampled-full-cover-proximity-failure } s \text{ bad out } \vee$
 $\text{trace-fri-sampled-full-cover-index-failure } s \text{ out } \vee$
 $\text{trace-fri-sampled-full-cover-shape-failure } s \text{ out } \vee$
 $\text{trace-fri-sampled-full-cover-domain-failure } s \text{ out } \vee$
 $\text{trace-fri-sampled-full-cover-start-low-failure } s \text{ out } \vee$
 $\text{trace-fri-sampled-full-cover-final-low-failure } s \text{ out}) \ s$
 $\leq P + I + Sh + D + SL + FL$
by (rule *wp-event-six-union-bound*)
[*OF proximity index shape domain start-low final-low*]
show *?thesis*
by (rule *order-trans*[*OF step1 step2*])
qed

lemma *wp-composition-fri-sampled-full-cover-failure-case-union-bound*:

assumes *proximity*:

$\text{wp-event verify-monad}$

$(\text{composition-fri-sampled-full-cover-proximity-failure } s \text{ bad}) \ s \leq P$

and *index*:

$\text{wp-event verify-monad}$

$(\text{composition-fri-sampled-full-cover-index-failure } s) \ s \leq I$

and *shape*:

$\text{wp-event verify-monad}$

$(\text{composition-fri-sampled-full-cover-shape-failure } s) \ s \leq Sh$

and *domain*:

$\text{wp-event verify-monad}$

$(\text{composition-fri-sampled-full-cover-domain-failure } s) \ s \leq D$

and *start-low*:

$\text{wp-event verify-monad}$

$(\text{composition-fri-sampled-full-cover-start-low-failure } s) \ s \leq SL$

and *final-low*:

$\text{wp-event verify-monad}$

(*composition-fri-sampled-full-cover-final-low-failure* *s*) *s* ≤ *FL*
shows
wp-event verify-monad
 (*composition-fri-sampled-full-cover-failure-case* *s bad*) *s*
 ≤ *P* + *I* + *Sh* + *D* + *SL* + *FL*
proof –
have *step1*:
wp-event verify-monad
 (*composition-fri-sampled-full-cover-failure-case* *s bad*) *s*
 ≤
wp-event verify-monad
 (λ*out*.
 composition-fri-sampled-full-cover-proximity-failure *s bad out* ∨
 composition-fri-sampled-full-cover-index-failure *s out* ∨
 composition-fri-sampled-full-cover-shape-failure *s out* ∨
 composition-fri-sampled-full-cover-domain-failure *s out* ∨
 composition-fri-sampled-full-cover-start-low-failure *s out* ∨
 composition-fri-sampled-full-cover-final-low-failure *s out*) *s*
by (*rule wp-event-mono*)
 (*rule composition-fri-sampled-full-cover-failure-case-split*)
have *step2*:
wp-event verify-monad
 (λ*out*.
 composition-fri-sampled-full-cover-proximity-failure *s bad out* ∨
 composition-fri-sampled-full-cover-index-failure *s out* ∨
 composition-fri-sampled-full-cover-shape-failure *s out* ∨
 composition-fri-sampled-full-cover-domain-failure *s out* ∨
 composition-fri-sampled-full-cover-start-low-failure *s out* ∨
 composition-fri-sampled-full-cover-final-low-failure *s out*) *s*
 ≤ *P* + *I* + *Sh* + *D* + *SL* + *FL*
by (*rule wp-event-six-union-bound*
 [*OF proximity index shape domain start-low final-low*])
show *?thesis*
by (*rule order-trans*[*OF step1 step2*])
qed

lemma *trace-fri-sampled-full-cover-start-low-failure-false*:

¬ *trace-fri-sampled-full-cover-start-low-failure* *s out*

proof

assume *failure*: *trace-fri-sampled-full-cover-start-low-failure* *s out*

then obtain *trace-roots* *trace-bs* *trace-final* *dg* *composition-roots*
composition-bs *composition-final* *fri-query-idxs* *trace-round-layers*
composition-round-layers *fr* *candidate-query-idxs* *trace-openings*
trace-table *doms* *layers*

where *evidence*:

trace-fri-partial-candidate-opening-evidence *s out* *trace-roots*
trace-bs *trace-final* *dg* *composition-roots* *composition-bs*
composition-final *fri-query-idxs* *trace-round-layers*
composition-round-layers *fr* *candidate-query-idxs* *trace-openings*

```

    trace-table
  and start-low:
    generic-fri-full-cover-start-low-failure (clength - 1)
    trace-table
  unfolding trace-fri-sampled-full-cover-start-low-failure-def by blast
have not-low:
  ¬ fri-table-low-degree-on (clength - 1) eval-domain trace-table
by (rule trace-fri-partial-candidate-opening-evidence-not-low-on-eval-domain
  [OF evidence])
have low:
  fri-table-low-degree-on (clength - 1) eval-domain trace-table
using start-low
unfolding generic-fri-full-cover-start-low-failure-def .
show False
using not-low low by contradiction
qed

```

lemma *wp-trace-fri-sampled-full-cover-start-low-failure-zero:*

```

  wp-event verify-monad
  (trace-fri-sampled-full-cover-start-low-failure s) s ≤ 0
proof -
  have wp-event verify-monad
    (trace-fri-sampled-full-cover-start-low-failure s) s
    ≤ wp-event verify-monad (λ-. False) s
  by (rule wp-event-mono)
  (simp add: trace-fri-sampled-full-cover-start-low-failure-false)
  also have ... = 0
  unfolding wp-event-def wp-def dist-expect-def by simp
  finally show ?thesis .
qed

```

lemma *composition-fri-sampled-full-cover-start-low-failure-false-on-support:*

```

  assumes outcome:
    out ∈ set-dist (execute verify-monad s)
  and failure:
    composition-fri-sampled-full-cover-start-low-failure s out
  shows False
proof -
  from failure obtain trace-roots trace-bs trace-final fri-dg
    opening-composition-roots composition-bs composition-final
    fri-query-idxs trace-round-layers composition-round-layers fr
    f-fri-roots f-final as dg composition-fri-roots final
    trace-query-idxs trace-openings composition-query-idxs
    composition-openings trace-table composition-table doms layers
  where evidence:
    composition-fri-partial-candidate-opening-evidence s out trace-roots
    trace-bs trace-final fri-dg opening-composition-roots composition-bs
    composition-final fri-query-idxs trace-round-layers
    composition-round-layers fr f-fri-roots f-final as dg

```



```

    composition-fri-roots final trace-query-idxs trace-openings
    composition-query-idxs composition-openings trace-table
    composition-table
and chain:
    generic-fri-sampled-layer-chain-evidence
    (composition-table-low-degree (to-nat fri-dg)) composition-table
    (to-nat fri-dg) opening-composition-roots composition-bs
    composition-final fri-query-idxs composition-round-layers doms layers
and start-low:
    generic-fri-full-cover-start-low-failure (to-nat fri-dg)
    composition-table
unfolding composition-fri-sampled-full-cover-start-low-failure-def
by meson
have fri-opening:
    accepted-fri-opening-transcript s out trace-roots trace-bs trace-final
    fri-dg opening-composition-roots composition-bs composition-final
    fri-query-idxs trace-round-layers composition-round-layers
by (rule composition-fri-partial-candidate-opening-evidenceD(1)
    [OF evidence])
from fri-opening obtain result final-state where
    out-eq: out = Some (result, final-state)
    unfolding accepted-fri-opening-transcript-def by blast
have outcome':
    Some (result, final-state) ∈ set-dist (execute verify-monad s)
    using outcome out-eq by simp
have challenges:
    accepted-fri-challenges s (Some (result, final-state)) trace-bs
    fri-dg composition-bs
    using accepted-fri-opening-transcript-challenges[OF fri-opening]
    out-eq by simp
have degree-bound: to-nat fri-dg ≤ maxDegree
    by (rule verify-monad-accepted-fri-challenges-degree-bound
    [OF outcome' challenges])
have not-low:
    ¬ fri-table-low-degree-on (to-nat fri-dg) eval-domain
    composition-table
    by (rule
    composition-fri-partial-candidate-opening-evidence-not-low-declared-on-eval-domain
    [OF evidence degree-bound])
have low:
    fri-table-low-degree-on (to-nat fri-dg) eval-domain composition-table
    using start-low
    unfolding generic-fri-full-cover-start-low-failure-def .
show False
    using not-low low by contradiction
qed

lemma wp-composition-fri-sampled-full-cover-start-low-failure-zero:
    wp-event verify-monad

```

(*composition-fri-sampled-full-cover-start-low-failure* *s*) $s \leq 0$
proof –
 have *wp-event verify-monad*
 (*composition-fri-sampled-full-cover-start-low-failure* *s*) *s*
 $\leq$ *wp-event verify-monad* ($\lambda\cdot$. *False*) *s*
 by (*rule wp-event-mono-on-support*)
 (*rule composition-fri-sampled-full-cover-start-low-failure-false-on-support*)
 also have ... = 0
 unfolding *wp-event-def wp-def dist-expect-def* by *simp*
 finally show ?thesis .
qed

lemma *trace-fri-sampled-full-cover-arithmetic-shape-failure-false*:
 assumes *trace-fri-sampled-full-cover-arithmetic-shape-failure* *s out*
 shows *False*
proof –
 from *assms* obtain *trace-roots trace-bs trace-final dg composition-roots*
composition-bs composition-final fri-query-idxs trace-round-layers
composition-round-layers fr candidate-query-idxs trace-openings
trace-table doms layers
 where *chain*:
generic-fri-sampled-layer-chain-evidence trace-table-low-degree
trace-table (clength - 1) trace-roots trace-bs trace-final
fri-query-idxs trace-round-layers doms layers
 and *arithmetic*:
generic-fri-full-cover-arithmetic-shape-failure trace-roots trace-bs
 unfolding *trace-fri-sampled-full-cover-arithmetic-shape-failure-def*
 by *meson*
 have *partial*:
generic-fri-partial-evidence trace-table-low-degree trace-table
(clength - 1) trace-roots trace-bs trace-final fri-query-idxs
trace-round-layers
 by (*rule generic-fri-sampled-layer-chain-evidenceD*(1)[*OF chain*])
 have *challenges-len*: *length trace-bs = length trace-roots*
 using *generic-fri-partial-evidence-shapes*(2)[*OF partial*] .
 have *rounds-eq*: *length trace-bs = ceil-log clength*
 using *generic-fri-partial-evidence-shapes*(1,2)[*OF partial*]
 unfolding *fri-round-count-for-degree-bound-def*
 using *clength-pos* by *simp*
 obtain *N* where *eval-power*: *clength * scale = 2 ^ N*
 using *eval-domain-length-power* by *blast*
 have *clength-le-eval*: *clength ≤ clength * scale*
 using *scale-pos* by *simp*
 have *rounds-bound*: *length trace-bs ≤ N*
 unfolding *rounds-eq*
 by (*rule ceil-log-le-power-bound*)
 (*use clength-le-eval eval-power in simp*)
 have $\neg$ *generic-fri-full-cover-arithmetic-shape-failure*
trace-roots trace-bs

```

    by (rule generic-fri-full-cover-arithmetic-shape-failure-false
        [OF challenges-len eval-power rounds-bound])
  then show False
    using arithmetic by contradiction
qed

lemma wp-trace-fri-sampled-full-cover-arithmetic-shape-failure-zero:
  wp-event verify-monad
    (trace-fri-sampled-full-cover-arithmetic-shape-failure s) s ≤ 0
proof -
  have wp-event verify-monad
    (trace-fri-sampled-full-cover-arithmetic-shape-failure s) s
    ≤ wp-event verify-monad (λ-. False) s
  by (rule wp-event-mono)
    (rule trace-fri-sampled-full-cover-arithmetic-shape-failure-false)
  also have ... = 0
    unfolding wp-event-def wp-def dist-expect-def by simp
  finally show ?thesis .
qed

lemma composition-fri-sampled-full-cover-arithmetic-shape-failure-false-on-support:
  assumes outcome:
    out ∈ set-dist (execute verify-monad s)
  and failure:
    composition-fri-sampled-full-cover-arithmetic-shape-failure s out
  shows False
proof -
  from failure obtain trace-roots trace-bs trace-final fri-dg
    opening-composition-roots composition-bs composition-final
    fri-query-idxs trace-round-layers composition-round-layers fr
    f-fri-roots f-final as dg composition-fri-roots final
    trace-query-idxs trace-openings composition-query-idxs
    composition-openings trace-table composition-table doms layers
  where evidence:
    composition-fri-partial-candidate-opening-evidence s out trace-roots
    trace-bs trace-final fri-dg opening-composition-roots composition-bs
    composition-final fri-query-idxs trace-round-layers
    composition-round-layers fr f-fri-roots f-final as dg
    composition-fri-roots final trace-query-idxs trace-openings
    composition-query-idxs composition-openings trace-table
    composition-table
  and chain:
    generic-fri-sampled-layer-chain-evidence
    (composition-table-low-degree (to-nat fri-dg)) composition-table
    (to-nat fri-dg) opening-composition-roots composition-bs
    composition-final fri-query-idxs composition-round-layers doms layers
  and arithmetic:
    generic-fri-full-cover-arithmetic-shape-failure
    opening-composition-roots composition-bs

```

```

unfolding composition-fri-sampled-full-cover-arithmetic-shape-failure-def
by meson
have partial:
  generic-fri-partial-evidence
    (composition-table-low-degree (to-nat fri-dg)) composition-table
    (to-nat fri-dg) opening-composition-roots composition-bs
    composition-final fri-query-idxs composition-round-layers
  by (rule generic-fri-sampled-layer-chain-evidenceD(1)[OF chain])
have challenges-len:
  length composition-bs = length opening-composition-roots
  using generic-fri-partial-evidence-shapes(2)[OF partial] .
have rounds-eq:
  length composition-bs = ceil-log (to-nat fri-dg + 1)
  using generic-fri-partial-evidence-shapes(1,2)[OF partial]
  unfolding fri-round-count-for-degree-bound-def by simp
have fri-opening:
  accepted-fri-opening-transcript s out trace-roots trace-bs trace-final
    fri-dg opening-composition-roots composition-bs composition-final
    fri-query-idxs trace-round-layers composition-round-layers
  by (rule composition-fri-partial-candidate-opening-evidenceD(1)
    [OF evidence])
from fri-opening obtain result final-state where
  out-eq: out = Some (result, final-state)
  unfolding accepted-fri-opening-transcript-def by blast
have outcome':
  Some (result, final-state) ∈ set-dist (execute verify-monad s)
  using outcome out-eq by simp
have challenges:
  accepted-fri-challenges s (Some (result, final-state)) trace-bs
    fri-dg composition-bs
  using accepted-fri-opening-transcript-challenges[OF fri-opening]
    out-eq by simp
have degree-bound: to-nat fri-dg ≤ maxDegree
  by (rule verify-monad-accepted-fri-challenges-degree-bound
    [OF outcome' challenges])
have degree-eval-bound: to-nat fri-dg + 1 ≤ clength * scale
  using degree-bound maxDegree-less-eval-domain by linarith
obtain N where eval-power: clength * scale =  $2^N$ 
  using eval-domain-length-power by blast
have rounds-bound: length composition-bs ≤ N
  unfolding rounds-eq
  by (rule ceil-log-le-power-bound)
    (use degree-eval-bound eval-power in simp)
have ¬ generic-fri-full-cover-arithmetic-shape-failure
  opening-composition-roots composition-bs
  by (rule generic-fri-full-cover-arithmetic-shape-failure-false
    [OF challenges-len eval-power rounds-bound])
then show False
  using arithmetic by contradiction

```

qed

lemma *wp-composition-fri-sampled-full-cover-arithmetic-shape-failure-zero:*

wp-event verify-monad

(composition-fri-sampled-full-cover-arithmetic-shape-failure s) s ≤ 0

proof –

have *wp-event verify-monad*

(composition-fri-sampled-full-cover-arithmetic-shape-failure s) s

≤ wp-event verify-monad (λ-. False) s

by *(rule wp-event-mono-on-support)*

(rule

composition-fri-sampled-full-cover-arithmetic-shape-failure-false-on-support)

also have *... = 0*

unfolding *wp-event-def wp-def dist-expect-def* **by** *simp*

finally show *?thesis* .

qed

lemma *wp-trace-fri-sampled-full-cover-shape-bound-from-witness-length:*

assumes *witness:*

wp-event verify-monad

(trace-fri-sampled-full-cover-witness-length-failure s) s ≤ W

shows

wp-event verify-monad

(trace-fri-sampled-full-cover-shape-failure s) s ≤ W

proof –

have *wp-event verify-monad*

(trace-fri-sampled-full-cover-shape-failure s) s ≤ 0 + W

by *(rule wp-trace-fri-sampled-full-cover-shape-bound-from-split*

[OF wp-trace-fri-sampled-full-cover-arithmetic-shape-failure-zero
witness])

then show *?thesis*

by *simp*

qed

lemma *wp-composition-fri-sampled-full-cover-shape-bound-from-witness-length:*

assumes *witness:*

wp-event verify-monad

(composition-fri-sampled-full-cover-witness-length-failure s) s

≤ W

shows

wp-event verify-monad

(composition-fri-sampled-full-cover-shape-failure s) s ≤ W

proof –

have *wp-event verify-monad*

(composition-fri-sampled-full-cover-shape-failure s) s ≤ 0 + W

by *(rule wp-composition-fri-sampled-full-cover-shape-bound-from-split*

[OF
wp-composition-fri-sampled-full-cover-arithmetic-shape-failure-zero
witness])

then show *?thesis*
by *simp*
qed

lemma *trace-fri-sampled-full-cover-final-low-imp-length:*
assumes *trace-fri-sampled-full-cover-final-low-failure s out*
shows *trace-fri-sampled-full-cover-final-length-failure s out*
proof –
from *assms* **obtain** *trace-roots trace-bs trace-final dg composition-roots*
composition-bs composition-final fri-query-idxs trace-round-layers
composition-round-layers fr candidate-query-idxs trace-openings
trace-table doms layers
where *evidence:*
trace-fri-partial-candidate-opening-evidence s out trace-roots
trace-bs trace-final dg composition-roots composition-bs
composition-final fri-query-idxs trace-round-layers
composition-round-layers fr candidate-query-idxs trace-openings
trace-table
and *chain:*
generic-fri-sampled-layer-chain-evidence trace-table-low-degree
trace-table (clength – 1) trace-roots trace-bs trace-final
fri-query-idxs trace-round-layers doms layers
and *final-low:*
generic-fri-full-cover-final-low-failure (clength – 1) trace-bs
doms layers
unfolding *trace-fri-sampled-full-cover-final-low-failure-def* **by** *blast*
have *length-failure:*
generic-fri-full-cover-final-length-failure trace-bs doms layers
by (*rule generic-fri-final-low-failure-imp-length-failure*
[OF chain final-low])
show *?thesis*
unfolding *trace-fri-sampled-full-cover-final-length-failure-def*
using *evidence chain length-failure* **by** *blast*
qed

lemma *composition-fri-sampled-full-cover-final-low-imp-length:*
assumes *composition-fri-sampled-full-cover-final-low-failure s out*
shows *composition-fri-sampled-full-cover-final-length-failure s out*
proof –
from *assms* **obtain** *trace-roots trace-bs trace-final fri-dg*
opening-composition-roots composition-bs composition-final
fri-query-idxs trace-round-layers composition-round-layers fr
f-fri-roots f-final as dg composition-fri-roots final
trace-query-idxs trace-openings composition-query-idxs
composition-openings trace-table composition-table doms layers
where *evidence:*
composition-fri-partial-candidate-opening-evidence s out trace-roots
trace-bs trace-final fri-dg opening-composition-roots composition-bs
composition-final fri-query-idxs trace-round-layers

```

    composition-round-layers fr f-fri-roots f-final as dg
    composition-fri-roots final trace-query-idxs trace-openings
    composition-query-idxs composition-openings trace-table
    composition-table
and chain:
    generic-fri-sampled-layer-chain-evidence
    (composition-table-low-degree (to-nat fri-dg)) composition-table
    (to-nat fri-dg) opening-composition-roots composition-bs
    composition-final fri-query-idxs composition-round-layers doms layers
and final-low:
    generic-fri-full-cover-final-low-failure (to-nat fri-dg)
    composition-bs doms layers
unfolding composition-fri-sampled-full-cover-final-low-failure-def
by meson
have length-failure:
    generic-fri-full-cover-final-length-failure composition-bs doms layers
by (rule generic-fri-final-low-failure-imp-length-failure
    [OF chain final-low])
show ?thesis
    unfolding composition-fri-sampled-full-cover-final-length-failure-def
    using evidence chain length-failure by meson
qed

lemma wp-trace-fri-sampled-full-cover-final-low-bound-from-length:
assumes length-bound:
    wp-event verify-monad
    (trace-fri-sampled-full-cover-final-length-failure s)  $s \leq L$ 
shows
    wp-event verify-monad
    (trace-fri-sampled-full-cover-final-low-failure s)  $s \leq L$ 
by (rule order-trans[OF - length-bound])
    (rule wp-event-mono,
    rule trace-fri-sampled-full-cover-final-low-imp-length)

lemma wp-composition-fri-sampled-full-cover-final-low-bound-from-length:
assumes length-bound:
    wp-event verify-monad
    (composition-fri-sampled-full-cover-final-length-failure s)  $s \leq L$ 
shows
    wp-event verify-monad
    (composition-fri-sampled-full-cover-final-low-failure s)  $s \leq L$ 
by (rule order-trans[OF - length-bound])
    (rule wp-event-mono,
    rule composition-fri-sampled-full-cover-final-low-imp-length)

lemma trace-fri-sampled-full-cover-final-length-imp-shape:
assumes trace-fri-sampled-full-cover-final-length-failure s out
shows trace-fri-sampled-full-cover-shape-failure s out
using assms

```

unfolding *trace-fri-sampled-full-cover-final-length-failure-def*
trace-fri-sampled-full-cover-shape-failure-def
generic-fri-full-cover-final-length-failure-def
generic-fri-full-cover-shape-failure-def
by *blast*

lemma *composition-fri-sampled-full-cover-final-length-imp-shape:*
assumes *composition-fri-sampled-full-cover-final-length-failure s out*
shows *composition-fri-sampled-full-cover-shape-failure s out*
using *assms*
unfolding *composition-fri-sampled-full-cover-final-length-failure-def*
composition-fri-sampled-full-cover-shape-failure-def
generic-fri-full-cover-final-length-failure-def
generic-fri-full-cover-shape-failure-def
by *meson*

lemma *trace-fri-sampled-full-cover-final-low-imp-shape:*
assumes *trace-fri-sampled-full-cover-final-low-failure s out*
shows *trace-fri-sampled-full-cover-shape-failure s out*
using *trace-fri-sampled-full-cover-final-low-imp-length[OF assms]*
trace-fri-sampled-full-cover-final-length-imp-shape
by *blast*

lemma *composition-fri-sampled-full-cover-final-low-imp-shape:*
assumes *composition-fri-sampled-full-cover-final-low-failure s out*
shows *composition-fri-sampled-full-cover-shape-failure s out*
using *composition-fri-sampled-full-cover-final-low-imp-length[OF assms]*
composition-fri-sampled-full-cover-final-length-imp-shape
by *blast*

lemma *trace-fri-sampled-full-cover-failure-case-split-no-final:*
assumes *trace-fri-sampled-full-cover-failure-case s bad out*
shows
trace-fri-sampled-full-cover-proximity-failure s bad out $\vee$
trace-fri-sampled-full-cover-index-failure s out $\vee$
trace-fri-sampled-full-cover-shape-failure s out $\vee$
trace-fri-sampled-full-cover-domain-failure s out $\vee$
trace-fri-sampled-full-cover-start-low-failure s out
using *trace-fri-sampled-full-cover-failure-case-split[OF assms]*
trace-fri-sampled-full-cover-final-low-imp-shape
by *blast*

lemma *composition-fri-sampled-full-cover-failure-case-split-no-final:*
assumes *composition-fri-sampled-full-cover-failure-case s bad out*
shows
composition-fri-sampled-full-cover-proximity-failure s bad out $\vee$
composition-fri-sampled-full-cover-index-failure s out $\vee$
composition-fri-sampled-full-cover-shape-failure s out $\vee$
composition-fri-sampled-full-cover-domain-failure s out $\vee$

composition-fri-sampled-full-cover-start-low-failure s out
using *composition-fri-sampled-full-cover-failure-case-split*[*OF assms*]
composition-fri-sampled-full-cover-final-low-imp-shape
by *blast*

lemma *wp-trace-fri-sampled-full-cover-failure-case-union-bound-no-final*:

assumes *proximity*:

wp-event verify-monad

$(\text{trace-fri-sampled-full-cover-proximity-failure } s \text{ bad}) \ s \leq P$

and *index*:

wp-event verify-monad

$(\text{trace-fri-sampled-full-cover-index-failure } s) \ s \leq I$

and *shape*:

wp-event verify-monad

$(\text{trace-fri-sampled-full-cover-shape-failure } s) \ s \leq Sh$

and *domain*:

wp-event verify-monad

$(\text{trace-fri-sampled-full-cover-domain-failure } s) \ s \leq D$

and *start-low*:

wp-event verify-monad

$(\text{trace-fri-sampled-full-cover-start-low-failure } s) \ s \leq SL$

shows

wp-event verify-monad

$(\text{trace-fri-sampled-full-cover-failure-case } s \text{ bad}) \ s$
 $\leq P + I + Sh + D + SL$

proof –

let $?A = \lambda out. \text{trace-fri-sampled-full-cover-proximity-failure } s \text{ bad } out$

let $?B = \lambda out. \text{trace-fri-sampled-full-cover-index-failure } s \text{ out}$

let $?C = \lambda out. \text{trace-fri-sampled-full-cover-shape-failure } s \text{ out}$

let $?D = \lambda out. \text{trace-fri-sampled-full-cover-domain-failure } s \text{ out}$

let $?E = \lambda out. \text{trace-fri-sampled-full-cover-start-low-failure } s \text{ out}$

have *step1*:

wp-event verify-monad

$(\text{trace-fri-sampled-full-cover-failure-case } s \text{ bad}) \ s$

$\leq \text{wp-event verify-monad}$

$(\lambda out. ?A \text{ out} \vee ?B \text{ out} \vee ?C \text{ out} \vee ?D \text{ out} \vee ?E \text{ out}) \ s$

by (*rule wp-event-mono*)

(*rule trace-fri-sampled-full-cover-failure-case-split-no-final*)

have *four*:

wp-event verify-monad

$(\lambda out. ?A \text{ out} \vee ?B \text{ out} \vee ?C \text{ out} \vee ?D \text{ out}) \ s$

$\leq P + I + Sh + D$

by (*rule order-trans*[*OF wp-event-union-bound4*

add-mono[*OF add-mono*[*OF add-mono*[*OF proximity index*] *shape*

domain]])

have *five*:

wp-event verify-monad

$(\lambda out. ?A \text{ out} \vee ?B \text{ out} \vee ?C \text{ out} \vee ?D \text{ out} \vee ?E \text{ out}) \ s$

$\leq P + I + Sh + D + SL$

```

proof –
  have wp-event verify-monad
    ( $\lambda out. ?A \text{ out} \vee ?B \text{ out} \vee ?C \text{ out} \vee ?D \text{ out} \vee ?E \text{ out}$ ) s
     $\leq$  wp-event verify-monad
      ( $\lambda out. ?A \text{ out} \vee ?B \text{ out} \vee ?C \text{ out} \vee ?D \text{ out}$ ) s +
      wp-event verify-monad ?E s
    by (rule order-trans[OF - wp-event-union-bound])
      (rule wp-event-mono, blast)
  also have ...  $\leq (P + I + Sh + D) + SL$ 
    by (rule add-mono[OF four start-low])
  finally show ?thesis
    by (simp add: add.assoc)
qed
show ?thesis
  by (rule order-trans[OF step1 five])
qed

lemma wp-composition-fri-sampled-full-cover-failure-case-union-bound-no-final:
assumes proximity:
  wp-event verify-monad
    (composition-fri-sampled-full-cover-proximity-failure s bad) s  $\leq P$ 
and index:
  wp-event verify-monad
    (composition-fri-sampled-full-cover-index-failure s) s  $\leq I$ 
and shape:
  wp-event verify-monad
    (composition-fri-sampled-full-cover-shape-failure s) s  $\leq Sh$ 
and domain:
  wp-event verify-monad
    (composition-fri-sampled-full-cover-domain-failure s) s  $\leq D$ 
and start-low:
  wp-event verify-monad
    (composition-fri-sampled-full-cover-start-low-failure s) s  $\leq SL$ 
shows
  wp-event verify-monad
    (composition-fri-sampled-full-cover-failure-case s bad) s
     $\leq P + I + Sh + D + SL$ 
proof –
  let ?A =
     $\lambda out. \text{composition-fri-sampled-full-cover-proximity-failure } s \text{ bad } out$ 
  let ?B =  $\lambda out. \text{composition-fri-sampled-full-cover-index-failure } s \text{ out}$ 
  let ?C =  $\lambda out. \text{composition-fri-sampled-full-cover-shape-failure } s \text{ out}$ 
  let ?D =  $\lambda out. \text{composition-fri-sampled-full-cover-domain-failure } s \text{ out}$ 
  let ?E =  $\lambda out. \text{composition-fri-sampled-full-cover-start-low-failure } s \text{ out}$ 
  have step1:
    wp-event verify-monad
      (composition-fri-sampled-full-cover-failure-case s bad) s
       $\leq$  wp-event verify-monad
        ( $\lambda out. ?A \text{ out} \vee ?B \text{ out} \vee ?C \text{ out} \vee ?D \text{ out} \vee ?E \text{ out}$ ) s

```

```

  by (rule wp-event-mono)
    (rule composition-fri-sampled-full-cover-failure-case-split-no-final)
have four:
  wp-event verify-mono
  (λout. ?A out ∨ ?B out ∨ ?C out ∨ ?D out) s
  ≤ P + I + Sh + D
  by (rule order-trans[OF wp-event-union-bound4
    add-mono[OF add-mono[OF add-mono[OF proximity index] shape]
      domain]])
have five:
  wp-event verify-mono
  (λout. ?A out ∨ ?B out ∨ ?C out ∨ ?D out ∨ ?E out) s
  ≤ P + I + Sh + D + SL
proof -
  have wp-event verify-mono
  (λout. ?A out ∨ ?B out ∨ ?C out ∨ ?D out ∨ ?E out) s
  ≤ wp-event verify-mono
  (λout. ?A out ∨ ?B out ∨ ?C out ∨ ?D out) s +
  wp-event verify-mono ?E s
  by (rule order-trans[OF - wp-event-union-bound])
  (rule wp-event-mono, blast)
  also have ... ≤ (P + I + Sh + D) + SL
  by (rule add-mono[OF four start-low])
  finally show ?thesis
  by (simp add: add.assoc)
qed
show ?thesis
  by (rule order-trans[OF step1 five])
qed

lemma generic-fri-sampled-layer-chain-full-cover-or-failure:
  assumes
    generic-fri-sampled-layer-chain-evidence low-degree candidate-table
    degree-bound roots challenges final-value query-idxs round-layers
    doms layers
  shows
    generic-fri-sampled-layer-chain-full-cover low-degree candidate-table
    degree-bound roots challenges final-value query-idxs round-layers
    doms layers bad ∨
    generic-fri-sampled-full-cover-failure low-degree candidate-table
    degree-bound roots challenges final-value query-idxs round-layers
    doms layers bad
  using assms
  unfolding generic-fri-sampled-layer-chain-full-cover-def
    generic-fri-sampled-full-cover-failure-def
  by blast

```

```

lemma generic-fri-sampled-layer-chain-missing-full-cover-failure:
  assumes chain:

```

*generic-fri-sampled-layer-chain-evidence low-degree candidate-table
 degree-bound roots challenges final-value query-idxs round-layers
 doms layers*
and *no-full:*
 $\neg$ *generic-fri-sampled-layer-chain-full-cover low-degree
 candidate-table degree-bound roots challenges final-value query-idxs
 round-layers doms layers bad*
shows
*generic-fri-sampled-full-cover-failure low-degree candidate-table
 degree-bound roots challenges final-value query-idxs round-layers
 doms layers bad*
using *generic-fri-sampled-layer-chain-full-cover-or-failure[OF chain]*
no-full by blast

lemma *trace-fri-sampled-missing-full-cover-imp-failure:*
assumes *trace-fri-sampled-missing-full-cover s bad out*
shows *trace-fri-sampled-full-cover-failure s bad out*
proof –
have *sampled: trace-fri-bad-with-sampled-layer-chain s out*
and *not-full: $\neg$ trace-fri-bad-with-full-cover-layer-chain s bad out*
using *assms unfolding trace-fri-sampled-missing-full-cover-def by blast+*
from *sampled obtain trace-roots trace-bs trace-final dg composition-roots
 composition-bs composition-final fri-query-idxs trace-round-layers
 composition-round-layers fr candidate-query-idxs trace-openings
 trace-table doms layers*
where *evidence:*
*trace-fri-partial-candidate-opening-evidence s out trace-roots
 trace-bs trace-final dg composition-roots composition-bs
 composition-final fri-query-idxs trace-round-layers
 composition-round-layers fr candidate-query-idxs trace-openings
 trace-table*
and *chain:*
*generic-fri-sampled-layer-chain-evidence trace-table-low-degree
 trace-table (clength – 1) trace-roots trace-bs trace-final
 fri-query-idxs trace-round-layers doms layers*
unfolding *trace-fri-bad-with-sampled-layer-chain-def by blast*
have *no-full:*
 $\neg$ *generic-fri-sampled-layer-chain-full-cover trace-table-low-degree
 trace-table (clength – 1) trace-roots trace-bs trace-final
 fri-query-idxs trace-round-layers doms layers (bad trace-table)*
proof
assume *full:*
*generic-fri-sampled-layer-chain-full-cover trace-table-low-degree
 trace-table (clength – 1) trace-roots trace-bs trace-final
 fri-query-idxs trace-round-layers doms layers (bad trace-table)*
have *trace-fri-bad-with-full-cover-layer-chain s bad out*
unfolding *trace-fri-bad-with-full-cover-layer-chain-def*
using *evidence full by blast*
then show *False*

```

    using not-full by contradiction
qed
have failure:
  generic-fri-sampled-full-cover-failure trace-table-low-degree
  trace-table (clength - 1) trace-roots trace-bs trace-final
  fri-query-idxs trace-round-layers doms layers (bad trace-table)
  by (rule generic-fri-sampled-layer-chain-missing-full-cover-failure
    [OF chain no-full])
show ?thesis
  unfolding trace-fri-sampled-full-cover-failure-def
  using evidence chain failure by blast
qed

lemma composition-fri-sampled-missing-full-cover-imp-failure:
  assumes composition-fri-sampled-missing-full-cover s bad out
  shows composition-fri-sampled-full-cover-failure s bad out
proof -
  have sampled: composition-fri-bad-with-sampled-layer-chain s out
  and not-full:
    ¬ composition-fri-bad-with-full-cover-layer-chain s bad out
  using assms unfolding composition-fri-sampled-missing-full-cover-def
  by blast+
  from sampled obtain trace-roots trace-bs trace-final fri-dg
  opening-composition-roots composition-bs composition-final
  fri-query-idxs trace-round-layers composition-round-layers fr
  f-fri-roots f-final as dg composition-fri-roots final
  trace-query-idxs trace-openings composition-query-idxs
  composition-openings trace-table composition-table doms layers
  where evidence:
    composition-fri-partial-candidate-opening-evidence s out trace-roots
    trace-bs trace-final fri-dg opening-composition-roots composition-bs
    composition-final fri-query-idxs trace-round-layers
    composition-round-layers fr f-fri-roots f-final as dg
    composition-fri-roots final trace-query-idxs trace-openings
    composition-query-idxs composition-openings trace-table
    composition-table
  and chain:
    generic-fri-sampled-layer-chain-evidence
    (composition-table-low-degree (to-nat fri-dg)) composition-table
    (to-nat fri-dg) opening-composition-roots composition-bs
    composition-final fri-query-idxs composition-round-layers doms layers
  unfolding composition-fri-bad-with-sampled-layer-chain-def by auto
  have no-full:
    ¬ generic-fri-sampled-layer-chain-full-cover
    (composition-table-low-degree (to-nat fri-dg)) composition-table
    (to-nat fri-dg) opening-composition-roots composition-bs
    composition-final fri-query-idxs composition-round-layers doms layers
    (bad fri-dg composition-table)
  proof

```

assume *full*:
generic-fri-sampled-layer-chain-full-cover
(composition-table-low-degree (to-nat fri-dg)) composition-table
(to-nat fri-dg) opening-composition-roots composition-bs
composition-final fri-query-idxs composition-round-layers doms
layers (bad fri-dg composition-table)
have *composition-fri-bad-with-full-cover-layer-chain s bad out*
unfolding *composition-fri-bad-with-full-cover-layer-chain-def*
using *evidence full* **by** *metis*
then show *False*
using *not-full* **by** *contradiction*
qed
have *failure*:
generic-fri-sampled-full-cover-failure
(composition-table-low-degree (to-nat fri-dg)) composition-table
(to-nat fri-dg) opening-composition-roots composition-bs
composition-final fri-query-idxs composition-round-layers doms layers
(bad fri-dg composition-table)
by (rule *generic-fri-sampled-layer-chain-missing-full-cover-failure*
[OF chain no-full])
show *?thesis*
unfolding *composition-fri-sampled-full-cover-failure-def*
using *evidence chain failure* **by** *metis*
qed

lemma *wp-trace-fri-sampled-missing-full-cover-bound-from-failure*:
assumes *failure-bound*:
wp-event verify-monad
(trace-fri-sampled-full-cover-failure s bad) s ≤ M
shows
wp-event verify-monad
(trace-fri-sampled-missing-full-cover s bad) s ≤ M
by (rule *order-trans[OF - failure-bound]*)
(rule wp-event-mono,
rule trace-fri-sampled-missing-full-cover-imp-failure)

lemma *wp-trace-fri-sampled-missing-full-cover-bound-from-failure-case*:
assumes *failure-bound*:
wp-event verify-monad
(trace-fri-sampled-full-cover-failure-case s bad) s ≤ M
shows
wp-event verify-monad
(trace-fri-sampled-missing-full-cover s bad) s ≤ M
by (rule *wp-trace-fri-sampled-missing-full-cover-bound-from-failure*)
(rule order-trans[OF - failure-bound],
rule wp-event-mono,
simp add: trace-fri-sampled-full-cover-failure-iff-case)

lemma *wp-trace-fri-sampled-missing-full-cover-bound-from-failure-subcases*:

assumes *proximity*:
 wp-event verify-monad
 (trace-fri-sampled-full-cover-proximity-failure s bad) $s \leq P$
and *index*:
 wp-event verify-monad
 (trace-fri-sampled-full-cover-index-failure s) $s \leq I$
and *shape*:
 wp-event verify-monad
 (trace-fri-sampled-full-cover-shape-failure s) $s \leq Sh$
and *domain*:
 wp-event verify-monad
 (trace-fri-sampled-full-cover-domain-failure s) $s \leq D$
and *start-low*:
 wp-event verify-monad
 (trace-fri-sampled-full-cover-start-low-failure s) $s \leq SL$
and *final-low*:
 wp-event verify-monad
 (trace-fri-sampled-full-cover-final-low-failure s) $s \leq FL$
shows
 wp-event verify-monad
 (trace-fri-sampled-missing-full-cover s bad) s
 $\leq P + I + Sh + D + SL + FL$
by (rule wp-trace-fri-sampled-missing-full-cover-bound-from-failure-case)
 (rule wp-trace-fri-sampled-full-cover-failure-case-union-bound
 [OF proximity index shape domain start-low final-low])

lemma wp-trace-fri-sampled-missing-full-cover-bound-from-five-subcases:

assumes *proximity*:
 wp-event verify-monad
 (trace-fri-sampled-full-cover-proximity-failure s bad) $s \leq P$
and *index*:
 wp-event verify-monad
 (trace-fri-sampled-full-cover-index-failure s) $s \leq I$
and *shape*:
 wp-event verify-monad
 (trace-fri-sampled-full-cover-shape-failure s) $s \leq Sh$
and *domain*:
 wp-event verify-monad
 (trace-fri-sampled-full-cover-domain-failure s) $s \leq D$
and *final-low*:
 wp-event verify-monad
 (trace-fri-sampled-full-cover-final-low-failure s) $s \leq FL$
shows
 wp-event verify-monad
 (trace-fri-sampled-missing-full-cover s bad) s
 $\leq P + I + Sh + D + FL$

proof –

have *six*:
 wp-event verify-monad

$(\text{trace-fri-sampled-missing-full-cover } s \text{ bad}) \ s$
 $\leq P + I + Sh + D + 0 + FL$
by (rule *wp-trace-fri-sampled-missing-full-cover-bound-from-failure-subcases*
 $[OF \text{ proximity index shape domain}$
 $\text{wp-trace-fri-sampled-full-cover-start-low-failure-zero final-low}]$)
then show *?thesis*
by *simp*
qed

lemma *wp-trace-fri-sampled-missing-full-cover-bound-from-length-subcases:*

assumes *proximity:*

wp-event verify-monad

$(\text{trace-fri-sampled-full-cover-proximity-failure } s \text{ bad}) \ s \leq P$

and *index:*

wp-event verify-monad

$(\text{trace-fri-sampled-full-cover-index-failure } s) \ s \leq I$

and *shape:*

wp-event verify-monad

$(\text{trace-fri-sampled-full-cover-shape-failure } s) \ s \leq Sh$

and *domain:*

wp-event verify-monad

$(\text{trace-fri-sampled-full-cover-domain-failure } s) \ s \leq D$

and *final-length:*

wp-event verify-monad

$(\text{trace-fri-sampled-full-cover-final-length-failure } s) \ s \leq FL$

shows

wp-event verify-monad

$(\text{trace-fri-sampled-missing-full-cover } s \text{ bad}) \ s$

$\leq P + I + Sh + D + FL$

by (rule *wp-trace-fri-sampled-missing-full-cover-bound-from-five-subcases*
 $[OF \text{ proximity index shape domain}]$)

(rule *wp-trace-fri-sampled-full-cover-final-low-bound-from-length*
 $[OF \text{ final-length}]$)

lemma *wp-trace-fri-sampled-missing-full-cover-bound-from-shape-subcases:*

assumes *proximity:*

wp-event verify-monad

$(\text{trace-fri-sampled-full-cover-proximity-failure } s \text{ bad}) \ s \leq P$

and *index:*

wp-event verify-monad

$(\text{trace-fri-sampled-full-cover-index-failure } s) \ s \leq I$

and *shape:*

wp-event verify-monad

$(\text{trace-fri-sampled-full-cover-shape-failure } s) \ s \leq Sh$

and *domain:*

wp-event verify-monad

$(\text{trace-fri-sampled-full-cover-domain-failure } s) \ s \leq D$

shows

wp-event verify-monad

$(\text{trace-fri-sampled-missing-full-cover } s \text{ bad}) \ s$
 $\leq P + I + Sh + D$
proof –
have *case-bound*:
 $\text{wp-event verify-monad}$
 $(\text{trace-fri-sampled-full-cover-failure-case } s \text{ bad}) \ s$
 $\leq P + I + Sh + D + 0$
by (rule *wp-trace-fri-sampled-full-cover-failure-case-union-bound-no-final*
 $[\text{OF proximity index shape domain}$
 $\text{wp-trace-fri-sampled-full-cover-start-low-failure-zero}]$)
have *wp-event verify-monad*
 $(\text{trace-fri-sampled-missing-full-cover } s \text{ bad}) \ s$
 $\leq P + I + Sh + D + 0$
by (rule *wp-trace-fri-sampled-missing-full-cover-bound-from-failure-case*
 $[\text{OF case-bound}]$)
then show *?thesis*
by *simp*
qed

lemma *wp-trace-fri-sampled-missing-full-cover-bound-from-split-shape-subcases*:
assumes *proximity*:
 $\text{wp-event verify-monad}$
 $(\text{trace-fri-sampled-full-cover-proximity-failure } s \text{ bad}) \ s \leq P$
and *index*:
 $\text{wp-event verify-monad}$
 $(\text{trace-fri-sampled-full-cover-index-failure } s) \ s \leq I$
and *arithmetic-shape*:
 $\text{wp-event verify-monad}$
 $(\text{trace-fri-sampled-full-cover-arithmetic-shape-failure } s) \ s \leq A$
and *witness-length*:
 $\text{wp-event verify-monad}$
 $(\text{trace-fri-sampled-full-cover-witness-length-failure } s) \ s \leq W$
and *domain*:
 $\text{wp-event verify-monad}$
 $(\text{trace-fri-sampled-full-cover-domain-failure } s) \ s \leq D$
shows
 $\text{wp-event verify-monad}$
 $(\text{trace-fri-sampled-missing-full-cover } s \text{ bad}) \ s$
 $\leq P + I + (A + W) + D$
by (rule *wp-trace-fri-sampled-missing-full-cover-bound-from-shape-subcases*
 $[\text{OF proximity index - domain}]$)
(rule *wp-trace-fri-sampled-full-cover-shape-bound-from-split*
 $[\text{OF arithmetic-shape witness-length}]$)

lemma *wp-trace-fri-sampled-missing-full-cover-bound-from-witness-length-subcases*:
assumes *proximity*:
 $\text{wp-event verify-monad}$
 $(\text{trace-fri-sampled-full-cover-proximity-failure } s \text{ bad}) \ s \leq P$
and *index*:

wp-event verify-monad
 (trace-fri-sampled-full-cover-index-failure s) $s \leq I$
and *witness-length:*
wp-event verify-monad
 (trace-fri-sampled-full-cover-witness-length-failure s) $s \leq W$
and *domain:*
wp-event verify-monad
 (trace-fri-sampled-full-cover-domain-failure s) $s \leq D$
shows
wp-event verify-monad
 (trace-fri-sampled-missing-full-cover s bad) s
 $\leq P + I + W + D$
proof –
have *wp-event verify-monad*
 (trace-fri-sampled-missing-full-cover s bad) s
 $\leq P + I + (0 + W) + D$
by (rule
wp-trace-fri-sampled-missing-full-cover-bound-from-split-shape-subcases
 [OF *proximity index*
wp-trace-fri-sampled-full-cover-arithmetic-shape-failure-zero
witness-length domain])
then show ?thesis
by *simp*
qed

lemma *wp-composition-fri-sampled-missing-full-cover-bound-from-failure:*
assumes *failure-bound:*
wp-event verify-monad
 (composition-fri-sampled-full-cover-failure s bad) $s \leq M$
shows
wp-event verify-monad
 (composition-fri-sampled-missing-full-cover s bad) $s \leq M$
by (rule *order-trans*[OF - *failure-bound*])
 (rule *wp-event-mono*,
 rule *composition-fri-sampled-missing-full-cover-imp-failure*)

lemma *wp-composition-fri-sampled-missing-full-cover-bound-from-failure-case:*
assumes *failure-bound:*
wp-event verify-monad
 (composition-fri-sampled-full-cover-failure-case s bad) $s \leq M$
shows
wp-event verify-monad
 (composition-fri-sampled-missing-full-cover s bad) $s \leq M$
by (rule *wp-composition-fri-sampled-missing-full-cover-bound-from-failure*)
 (rule *order-trans*[OF - *failure-bound*],
 rule *wp-event-mono*,
 simp add: *composition-fri-sampled-full-cover-failure-iff-case*)

lemma *wp-composition-fri-sampled-missing-full-cover-bound-from-failure-subcases:*

assumes *proximity*:

wp-event verify-monad

(*composition-fri-sampled-full-cover-proximity-failure s bad*) $s \leq P$

and *index*:

wp-event verify-monad

(*composition-fri-sampled-full-cover-index-failure s*) $s \leq I$

and *shape*:

wp-event verify-monad

(*composition-fri-sampled-full-cover-shape-failure s*) $s \leq Sh$

and *domain*:

wp-event verify-monad

(*composition-fri-sampled-full-cover-domain-failure s*) $s \leq D$

and *start-low*:

wp-event verify-monad

(*composition-fri-sampled-full-cover-start-low-failure s*) $s \leq SL$

and *final-low*:

wp-event verify-monad

(*composition-fri-sampled-full-cover-final-low-failure s*) $s \leq FL$

shows

wp-event verify-monad

(*composition-fri-sampled-missing-full-cover s bad*) s

$\leq P + I + Sh + D + SL + FL$

by (*rule wp-composition-fri-sampled-missing-full-cover-bound-from-failure-case*)

(*rule wp-composition-fri-sampled-full-cover-failure-case-union-bound*

[*OF proximity index shape domain start-low final-low*])

lemma *wp-composition-fri-sampled-missing-full-cover-bound-from-five-subcases*:

assumes *proximity*:

wp-event verify-monad

(*composition-fri-sampled-full-cover-proximity-failure s bad*) $s \leq P$

and *index*:

wp-event verify-monad

(*composition-fri-sampled-full-cover-index-failure s*) $s \leq I$

and *shape*:

wp-event verify-monad

(*composition-fri-sampled-full-cover-shape-failure s*) $s \leq Sh$

and *domain*:

wp-event verify-monad

(*composition-fri-sampled-full-cover-domain-failure s*) $s \leq D$

and *final-low*:

wp-event verify-monad

(*composition-fri-sampled-full-cover-final-low-failure s*) $s \leq FL$

shows

wp-event verify-monad

(*composition-fri-sampled-missing-full-cover s bad*) s

$\leq P + I + Sh + D + FL$

proof –

have *six*:

wp-event verify-monad

$(\text{composition-fri-sampled-missing-full-cover } s \text{ bad}) \ s$
 $\leq P + I + Sh + D + 0 + FL$
by (rule
 $\text{wp-composition-fri-sampled-missing-full-cover-bound-from-failure-subcases}$
 $[OF \text{ proximity index shape domain}$
 $\text{wp-composition-fri-sampled-full-cover-start-low-failure-zero}$
 $\text{final-low}]$)
then show ?thesis
by simp
qed

lemma *wp-composition-fri-sampled-missing-full-cover-bound-from-length-subcases:*

assumes *proximity:*

wp-event verify-monad

$(\text{composition-fri-sampled-full-cover-proximity-failure } s \text{ bad}) \ s \leq P$

and *index:*

wp-event verify-monad

$(\text{composition-fri-sampled-full-cover-index-failure } s) \ s \leq I$

and *shape:*

wp-event verify-monad

$(\text{composition-fri-sampled-full-cover-shape-failure } s) \ s \leq Sh$

and *domain:*

wp-event verify-monad

$(\text{composition-fri-sampled-full-cover-domain-failure } s) \ s \leq D$

and *final-length:*

wp-event verify-monad

$(\text{composition-fri-sampled-full-cover-final-length-failure } s) \ s \leq FL$

shows

wp-event verify-monad

$(\text{composition-fri-sampled-missing-full-cover } s \text{ bad}) \ s$

$\leq P + I + Sh + D + FL$

by (rule

$\text{wp-composition-fri-sampled-missing-full-cover-bound-from-five-subcases}$

$[OF \text{ proximity index shape domain}]$)

(rule $\text{wp-composition-fri-sampled-full-cover-final-low-bound-from-length}$

$[OF \text{ final-length}]$)

lemma *wp-composition-fri-sampled-missing-full-cover-bound-from-shape-subcases:*

assumes *proximity:*

wp-event verify-monad

$(\text{composition-fri-sampled-full-cover-proximity-failure } s \text{ bad}) \ s \leq P$

and *index:*

wp-event verify-monad

$(\text{composition-fri-sampled-full-cover-index-failure } s) \ s \leq I$

and *shape:*

wp-event verify-monad

$(\text{composition-fri-sampled-full-cover-shape-failure } s) \ s \leq Sh$

and *domain:*

wp-event verify-monad

(composition-fri-sampled-full-cover-domain-failure s) s ≤ D
shows
 wp-event verify-monad
 (composition-fri-sampled-missing-full-cover s bad) s
 ≤ P + I + Sh + D
proof –
have case-bound:
 wp-event verify-monad
 (composition-fri-sampled-full-cover-failure-case s bad) s
 ≤ P + I + Sh + D + 0
by (rule
 wp-composition-fri-sampled-full-cover-failure-case-union-bound-no-final
 [OF proximity index shape domain
 wp-composition-fri-sampled-full-cover-start-low-failure-zero])
have wp-event verify-monad
 (composition-fri-sampled-missing-full-cover s bad) s
 ≤ P + I + Sh + D + 0
by (rule
 wp-composition-fri-sampled-missing-full-cover-bound-from-failure-case
 [OF case-bound])
then show ?thesis
by simp
qed

lemma
 wp-composition-fri-sampled-missing-full-cover-bound-from-split-shape-subcases:
assumes proximity:
 wp-event verify-monad
 (composition-fri-sampled-full-cover-proximity-failure s bad) s ≤ P
and index:
 wp-event verify-monad
 (composition-fri-sampled-full-cover-index-failure s) s ≤ I
and arithmetic-shape:
 wp-event verify-monad
 (composition-fri-sampled-full-cover-arithmetic-shape-failure s) s
 ≤ A
and witness-length:
 wp-event verify-monad
 (composition-fri-sampled-full-cover-witness-length-failure s) s ≤ W
and domain:
 wp-event verify-monad
 (composition-fri-sampled-full-cover-domain-failure s) s ≤ D
shows
 wp-event verify-monad
 (composition-fri-sampled-missing-full-cover s bad) s
 ≤ P + I + (A + W) + D
by (rule
 wp-composition-fri-sampled-missing-full-cover-bound-from-shape-subcases
 [OF proximity index - domain])

(rule *wp-composition-fri-sampled-full-cover-shape-bound-from-split*
 [OF *arithmetic-shape witness-length*])

lemma

wp-composition-fri-sampled-missing-full-cover-bound-from-witness-length-subcases:

assumes *proximity:*

wp-event verify-monad

(*composition-fri-sampled-full-cover-proximity-failure s bad*) $s \leq P$

and *index:*

wp-event verify-monad

(*composition-fri-sampled-full-cover-index-failure s*) $s \leq I$

and *witness-length:*

wp-event verify-monad

(*composition-fri-sampled-full-cover-witness-length-failure s*) $s \leq W$

and *domain:*

wp-event verify-monad

(*composition-fri-sampled-full-cover-domain-failure s*) $s \leq D$

shows

wp-event verify-monad

(*composition-fri-sampled-missing-full-cover s bad*) s
 $\leq P + I + W + D$

proof –

have *wp-event verify-monad*

(*composition-fri-sampled-missing-full-cover s bad*) s
 $\leq P + I + (0 + W) + D$

by (rule

wp-composition-fri-sampled-missing-full-cover-bound-from-split-shape-subcases
 [OF *proximity index*
wp-composition-fri-sampled-full-cover-arithmetic-shape-failure-zero
witness-length domain])

then show *?thesis*

by *simp*

qed

end

end

theory *Soundness-FRI-Full-Cover-Lengths*

imports *Soundness-FRI-Full-Cover-Reasons*

begin

Length residuals for sampled full-cover FRI witnesses.

Arithmetic shape failures are impossible, and table-layer length failures follow from sampled fold coverage. This theory factors the remaining witness-length residual into domain-length/final-length obligations plus the already-classified sampled-index coverage failure.

context *soundness*

begin

definition *generic-fri-full-cover-domain-length-failure*

$:: 'f \text{ list} \Rightarrow 'f \text{ list} \Rightarrow 'f \text{ list list} \Rightarrow 'f \text{ list list} \Rightarrow \text{bool}$

where

generic-fri-full-cover-domain-length-failure roots challenges doms layers

$\longleftrightarrow$

$\neg (\forall \text{ layer-idx} < \text{length challenges.}$
 $\text{fri-evidence-layer-len roots layer-idx} \leq$
 $\text{length (doms ! layer-idx)}) \vee$
 $\neg (\forall \text{ layer-idx} < \text{length challenges.}$
 $\text{fri-evidence-layer-len roots layer-idx div } 2 \leq$
 $\text{length (doms ! Suc layer-idx)}) \vee$
 $\text{length (doms ! length challenges)} \neq$
 $\text{length (layers ! length challenges)}$

definition *trace-fri-sampled-full-cover-domain-length-failure*

$:: ('f, 'a) \text{ protocol-channel-scheme} \Rightarrow$

$(\text{unit list} \times ('f, 'a) \text{ protocol-channel-scheme}) \text{ option} \Rightarrow \text{bool}$

where

trace-fri-sampled-full-cover-domain-length-failure s out $\longleftrightarrow$

$(\exists \text{ trace-roots trace-bs trace-final dg composition-roots}$
 $\text{composition-bs composition-final fri-query-idxs trace-round-layers}$
 $\text{composition-round-layers fr candidate-query-idxs trace-openings}$
 $\text{trace-table doms layers.}$

trace-fri-partial-candidate-opening-evidence s out trace-roots
 $\text{trace-bs trace-final dg composition-roots composition-bs}$
 $\text{composition-final fri-query-idxs trace-round-layers}$
 $\text{composition-round-layers fr candidate-query-idxs trace-openings}$
 $\text{trace-table} \wedge$

generic-fri-sampled-layer-chain-evidence trace-table-low-degree
 $\text{trace-table (clength - 1) trace-roots trace-bs trace-final}$
 $\text{fri-query-idxs trace-round-layers doms layers} \wedge$

generic-fri-full-cover-domain-length-failure trace-roots trace-bs
 $\text{doms layers})$

definition *composition-fri-sampled-full-cover-domain-length-failure*

$:: ('f, 'a) \text{ protocol-channel-scheme} \Rightarrow$

$(\text{unit list} \times ('f, 'a) \text{ protocol-channel-scheme}) \text{ option} \Rightarrow \text{bool}$

where

composition-fri-sampled-full-cover-domain-length-failure s out

$\longleftrightarrow$

$(\exists \text{ trace-roots trace-bs trace-final fri-dg}$
 $\text{opening-composition-roots composition-bs composition-final}$
 $\text{fri-query-idxs trace-round-layers composition-round-layers fr}$
 $\text{f-fri-roots f-final as dg composition-fri-roots final}$
 $\text{trace-query-idxs trace-openings composition-query-idxs}$
 $\text{composition-openings trace-table composition-table doms layers.}$
 $\text{composition-fri-partial-candidate-opening-evidence s out trace-roots}$

trace-bs trace-final fri-dg opening-composition-roots composition-bs
composition-final fri-query-idxs trace-round-layers
composition-round-layers fr f-fri-roots f-final as dg
composition-fri-roots final trace-query-idxs trace-openings
composition-query-idxs composition-openings trace-table
composition-table $\wedge$
generic-fri-sampled-layer-chain-evidence
(composition-table-low-degree (to-nat fri-dg)) composition-table
(to-nat fri-dg) opening-composition-roots composition-bs
composition-final fri-query-idxs composition-round-layers doms layers
 $\wedge$
generic-fri-full-cover-domain-length-failure
opening-composition-roots composition-bs doms layers)

lemma *generic-fri-sampled-layer-chain-next-layer-lengths-if-cover:*

assumes *chain:*

generic-fri-sampled-layer-chain-evidence low-degree candidate-table
degree-bound roots challenges final-value query-idxs round-layers
doms layers

and *covers:* *fri-sampled-layers-cover-fold-indices roots query-idxs*

and *layer-bound:* *layer-idx < length challenges*

shows

fri-evidence-layer-len roots layer-idx div 2 $\leq$
length (layers ! Suc layer-idx)

proof (rule *ccontr*)

assume *not-len:*

$\neg$ *fri-evidence-layer-len roots layer-idx div 2 $\leq$*
length (layers ! Suc layer-idx)

let *?idx = length (layers ! Suc layer-idx)*

have *idx-bound:*

?idx < fri-evidence-layer-len roots layer-idx div 2

using *not-len* **by** *simp*

have *partial:*

generic-fri-partial-evidence low-degree candidate-table degree-bound
roots challenges final-value query-idxs round-layers

by (rule *generic-fri-sampled-layer-chain-evidenceD(1)*) [*OF chain*]

have *roots-len:* *length challenges = length roots*

by (rule *generic-fri-partial-evidence-shapes(2)*) [*OF partial*]

obtain *round-idx* **where** *round-bound:* *round-idx < length query-idxs*

and *next-eq:*

fri-evidence-next-idx roots query-idxs round-idx layer-idx = ?idx

by (rule *fri-sampled-layers-cover-fold-indicesD*) [*OF covers - idx-bound*]

(*use layer-bound roots-len* **in** *simp*)

have *?idx < length (layers ! Suc layer-idx)*

using *generic-fri-sampled-layer-chain-evidence-sample(1)*

[*OF chain round-bound layer-bound*]

next-eq **by** *simp*

then show *False* **by** *simp*

qed

lemma *generic-fri-sampled-layer-chain-layer-lengths-if-cover:*
assumes *chain:*
generic-fri-sampled-layer-chain-evidence low-degree candidate-table
degree-bound roots challenges final-value query-idxs round-layers
doms layers
and *covers:* *fri-sampled-layers-cover-fold-indices roots query-idxs*
and *base-len:* *length candidate-table = clength * scale*
and *layer-bound:* *layer-idx < length challenges*
shows *fri-evidence-layer-len roots layer-idx ≤*
length (layers ! layer-idx)
proof (*cases layer-idx*)
case 0
have *partial:*
generic-fri-partial-evidence low-degree candidate-table degree-bound
roots challenges final-value query-idxs round-layers
by (*rule generic-fri-sampled-layer-chain-evidenceD(1)[OF chain]*)
have *roots-len:* *length challenges = length roots*
by (*rule generic-fri-partial-evidence-shapes(2)[OF partial]*)
have *roots-nonempty:* *0 < length roots*
using *layer-bound roots-len* **unfolding** 0 **by** *simp*
then obtain *n* **where** *roots-len-suc:* *length roots = Suc n*
by (*cases length roots*) *auto*
have *len0:* *fri-evidence-layer-len roots 0 = clength * scale*
using *roots-len-suc* **unfolding** 0 *fri-evidence-layer-len-def* **by** *simp*
have *layer0:* *layers ! 0 = candidate-table*
by (*rule generic-fri-sampled-layer-chain-evidenceD(5)[OF chain]*)
show *?thesis*
using *len0 layer0 base-len* **unfolding** 0 **by** *simp*
next
case (*Suc j*)
have *partial:*
generic-fri-partial-evidence low-degree candidate-table degree-bound
roots challenges final-value query-idxs round-layers
by (*rule generic-fri-sampled-layer-chain-evidenceD(1)[OF chain]*)
have *roots-len:* *length challenges = length roots*
by (*rule generic-fri-partial-evidence-shapes(2)[OF partial]*)
have *j-bound:* *j < length challenges*
using *layer-bound* **unfolding** *Suc* **by** *simp*
have *len-suc:*
fri-evidence-layer-len roots (Suc j) =
fri-evidence-layer-len roots j div 2
using *fri-layer-lengths-Suc-nth[of j length roots clength * scale]*
layer-bound roots-len
unfolding *fri-evidence-layer-len-def Suc* **by** *simp*
have *next-len:*
fri-evidence-layer-len roots j div 2 ≤ length (layers ! Suc j)
by (*rule generic-fri-sampled-layer-chain-next-layer-lengths-if-cover*)

```

    [OF chain covers j-bound])
  show ?thesis
    using len-suc next-len unfolding Suc by simp
qed

lemma generic-fri-witness-length-failure-imp-domain-length-if-cover:
  assumes chain:
    generic-fri-sampled-layer-chain-evidence low-degree candidate-table
    degree-bound roots challenges final-value query-idxs round-layers
    doms layers
  and no-index:
    ¬ generic-fri-full-cover-index-failure roots query-idxs
  and base-len: length candidate-table = clength * scale
  and witness:
    generic-fri-full-cover-witness-length-failure roots challenges
    doms layers
  shows generic-fri-full-cover-domain-length-failure roots challenges
    doms layers
proof -
  have covers: fri-sampled-layers-cover-fold-indices roots query-idxs
    using no-index unfolding generic-fri-full-cover-index-failure-def by simp
  have layer-len:
     $\bigwedge \text{layer-idx. layer-idx} < \text{length challenges} \implies$ 
    fri-evidence-layer-len roots layer-idx  $\leq$ 
    length (layers ! layer-idx)
  by (rule generic-fri-sampled-layer-chain-layer-lengths-if-cover
    [OF chain covers base-len])
  have next-layer-len:
     $\bigwedge \text{layer-idx. layer-idx} < \text{length challenges} \implies$ 
    fri-evidence-layer-len roots layer-idx div 2  $\leq$ 
    length (layers ! Suc layer-idx)
  by (rule generic-fri-sampled-layer-chain-next-layer-lengths-if-cover
    [OF chain covers])
  show ?thesis
    using witness layer-len next-layer-len
    unfolding generic-fri-full-cover-witness-length-failure-def
    generic-fri-full-cover-domain-length-failure-def
    by blast
qed

lemma trace-fri-sampled-full-cover-witness-length-imp-index-or-domain-length:
  assumes witness:
    trace-fri-sampled-full-cover-witness-length-failure s out
  shows
    trace-fri-sampled-full-cover-index-failure s out  $\vee$ 
    trace-fri-sampled-full-cover-domain-length-failure s out
proof -
  from witness obtain trace-roots trace-bs trace-final dg composition-roots
    composition-bs composition-final fri-query-idxs trace-round-layers

```

```

    composition-round-layers fr candidate-query-idxs trace-openings
    trace-table doms layers
where evidence:
    trace-fri-partial-candidate-opening-evidence s out trace-roots
    trace-bs trace-final dg composition-roots composition-bs
    composition-final fri-query-idxs trace-round-layers
    composition-round-layers fr candidate-query-idxs trace-openings
    trace-table
and chain:
    generic-fri-sampled-layer-chain-evidence trace-table-low-degree
    trace-table (clength - 1) trace-roots trace-bs trace-final
    fri-query-idxs trace-round-layers doms layers
and generic-witness:
    generic-fri-full-cover-witness-length-failure trace-roots trace-bs
    doms layers
unfolding trace-fri-sampled-full-cover-witness-length-failure-def
by blast
show ?thesis
proof (cases
    generic-fri-full-cover-index-failure trace-roots fri-query-idxs)
case True
then show ?thesis
    unfolding trace-fri-sampled-full-cover-index-failure-def
    using evidence chain by blast
next
case False
have partial:
    trace-fri-partial-candidate-evidence s out fr candidate-query-idxs
    trace-openings trace-table
by (rule trace-fri-partial-candidate-opening-evidenceD(2)
    [OF evidence])
have cand:
    partial-trace-table-candidate trace-table trace-openings
by (rule trace-fri-partial-candidate-evidenceD(2)[OF partial])
have base-len: length trace-table = clength * scale
    using cand unfolding partial-trace-table-candidate-def by simp
have domain-length:
    generic-fri-full-cover-domain-length-failure trace-roots trace-bs
    doms layers
by (rule generic-fri-witness-length-failure-imp-domain-length-if-cover
    [OF chain False base-len generic-witness])
then show ?thesis
    unfolding trace-fri-sampled-full-cover-domain-length-failure-def
    using evidence chain by blast
qed
qed

lemma composition-fri-sampled-full-cover-witness-length-imp-index-or-domain-length:
assumes witness:

```

composition-fri-sampled-full-cover-witness-length-failure s out
shows
composition-fri-sampled-full-cover-index-failure s out $\vee$
composition-fri-sampled-full-cover-domain-length-failure s out
proof –
from *witness* **obtain** *trace-roots trace-bs trace-final fri-dg*
opening-composition-roots composition-bs composition-final
fri-query-idxs trace-round-layers composition-round-layers fr
f-fri-roots f-final as dg composition-fri-roots final
trace-query-idxs trace-openings composition-query-idxs
composition-openings trace-table composition-table doms layers
where *evidence*:
composition-fri-partial-candidate-opening-evidence s out trace-roots
trace-bs trace-final fri-dg opening-composition-roots composition-bs
composition-final fri-query-idxs trace-round-layers
composition-round-layers fr f-fri-roots f-final as dg
composition-fri-roots final trace-query-idxs trace-openings
composition-query-idxs composition-openings trace-table
composition-table
and *chain*:
generic-fri-sampled-layer-chain-evidence
(composition-table-low-degree (to-nat fri-dg)) composition-table
(to-nat fri-dg) opening-composition-roots composition-bs
composition-final fri-query-idxs composition-round-layers doms layers
and *generic-witness*:
generic-fri-full-cover-witness-length-failure
opening-composition-roots composition-bs doms layers
unfolding *composition-fri-sampled-full-cover-witness-length-failure-def*
by *meson*
show *?thesis*
proof (*cases*
generic-fri-full-cover-index-failure opening-composition-roots
fri-query-idxs)
case *True*
then show *?thesis*
unfolding *composition-fri-sampled-full-cover-index-failure-def*
using *evidence chain* **by** *meson*
next
case *False*
have *partial*:
composition-fri-partial-candidate-evidence s out fr f-fri-roots
f-final as dg composition-fri-roots final trace-query-idxs
trace-openings composition-query-idxs composition-openings
trace-table composition-table
by (*rule composition-fri-partial-candidate-opening-evidenceD(2)*
 $[OF\ evidence]$)
have *cand*:
partial-composition-table-candidate composition-table
composition-openings

by (rule composition-fri-partial-candidate-evidenceD(3)[OF partial])
 have base-len: length composition-table = clength * scale
 using cand unfolding partial-composition-table-candidate-def by simp
 have domain-length:
 generic-fri-full-cover-domain-length-failure
 opening-composition-roots composition-bs doms layers
 by (rule generic-fri-witness-length-failure-imp-domain-length-if-cover
 [OF chain False base-len generic-witness])
 then show ?thesis
 unfolding composition-fri-sampled-full-cover-domain-length-failure-def
 using evidence chain by meson
 qed
 qed

lemma wp-trace-fri-sampled-full-cover-witness-length-bound-from-index-or-domain-length:
 assumes index:
 wp-event verify-monad
 (trace-fri-sampled-full-cover-index-failure s) $s \leq I$
 and domain-length:
 wp-event verify-monad
 (trace-fri-sampled-full-cover-domain-length-failure s) $s \leq DL$
 shows
 wp-event verify-monad
 (trace-fri-sampled-full-cover-witness-length-failure s) $s \leq I + DL$
proof –
 have wp-event verify-monad
 (trace-fri-sampled-full-cover-witness-length-failure s) s
 $\leq$ wp-event verify-monad
 (λ out. trace-fri-sampled-full-cover-index-failure s out $\vee$
 trace-fri-sampled-full-cover-domain-length-failure s out) s
 by (rule wp-event-mono)
 (rule trace-fri-sampled-full-cover-witness-length-imp-index-or-domain-length)
 also have ... $\leq I + DL$
 by (rule order-trans[OF wp-event-union-bound
 add-mono[OF index domain-length]])
 finally show ?thesis .
 qed

lemma wp-composition-fri-sampled-full-cover-witness-length-bound-from-index-or-domain-length:
 assumes index:
 wp-event verify-monad
 (composition-fri-sampled-full-cover-index-failure s) $s \leq I$
 and domain-length:
 wp-event verify-monad
 (composition-fri-sampled-full-cover-domain-length-failure s) $s \leq DL$
 shows
 wp-event verify-monad
 (composition-fri-sampled-full-cover-witness-length-failure s) s

$\leq I + DL$
proof –
have *wp-event verify-monad*
 $(\text{composition-fri-sampled-full-cover-witness-length-failure } s) \ s$
 $\leq \text{wp-event verify-monad}$
 $(\lambda \text{out. composition-fri-sampled-full-cover-index-failure } s \ \text{out} \ \vee$
 $\text{composition-fri-sampled-full-cover-domain-length-failure } s \ \text{out}) \ s$
by (rule *wp-event-mono*)
(rule
 $\text{composition-fri-sampled-full-cover-witness-length-imp-index-or-domain-length}$)
also have $\dots \leq I + DL$
by (rule *order-trans*[*OF wp-event-union-bound*
 $\text{add-mono}[\text{OF index domain-length}]]$)
finally show *?thesis* .
qed

lemma *wp-trace-fri-sampled-missing-full-cover-bound-from-length-split:*

assumes *proximity:*
 $\text{wp-event verify-monad}$
 $(\text{trace-fri-sampled-full-cover-proximity-failure } s \ \text{bad}) \ s \leq P$
and *index:*
 $\text{wp-event verify-monad}$
 $(\text{trace-fri-sampled-full-cover-index-failure } s) \ s \leq I$
and *domain-length:*
 $\text{wp-event verify-monad}$
 $(\text{trace-fri-sampled-full-cover-domain-length-failure } s) \ s \leq DL$
and *domain:*
 $\text{wp-event verify-monad}$
 $(\text{trace-fri-sampled-full-cover-domain-failure } s) \ s \leq D$
shows
 $\text{wp-event verify-monad}$
 $(\text{trace-fri-sampled-missing-full-cover } s \ \text{bad}) \ s$
 $\leq P + I + (I + DL) + D$
by (rule *wp-trace-fri-sampled-missing-full-cover-bound-from-witness-length-subcases*
 $[\text{OF proximity index}$
 $\text{wp-trace-fri-sampled-full-cover-witness-length-bound-from-index-or-domain-length}$
 $\text{domain}]$)
(rule *index*, rule *domain-length*)

lemma *wp-composition-fri-sampled-missing-full-cover-bound-from-length-split:*

assumes *proximity:*
 $\text{wp-event verify-monad}$
 $(\text{composition-fri-sampled-full-cover-proximity-failure } s \ \text{bad}) \ s \leq P$
and *index:*
 $\text{wp-event verify-monad}$
 $(\text{composition-fri-sampled-full-cover-index-failure } s) \ s \leq I$
and *domain-length:*
 $\text{wp-event verify-monad}$
 $(\text{composition-fri-sampled-full-cover-domain-length-failure } s) \ s \leq DL$

```

and domain:
  wp-event verify-monad
    (composition-fri-sampled-full-cover-domain-failure s) s ≤ D
shows
  wp-event verify-monad
    (composition-fri-sampled-missing-full-cover s bad) s
    ≤ P + I + (I + DL) + D
by (rule
  wp-composition-fri-sampled-missing-full-cover-bound-from-witness-length-subcases
  [OF proximity index
  wp-composition-fri-sampled-full-cover-witness-length-bound-from-index-or-domain-length
  domain])
  (rule index, rule domain-length)

end

end

```

```

theory Soundness-FRI-Full-Cover-Domains
imports Soundness-FRI-Full-Cover-Lengths
begin

```

Domain residuals for sampled full-cover FRI witnesses.

This theory keeps the remaining domain issue explicitly verifier-sampled: a global full-cover domain failure is routed to missing sampled-index coverage or to sampled domain facts that still need to be bounded/extracted.

```

context soundness
begin

```

```

definition fri-canonical-domain-at :: nat ⇒ 'f list
where

```

```

  fri-canonical-domain-at layer-idx =
    map (λidx. (h ^ idx * shift) ^ (2 ^ layer-idx))
    [0.. $((clength * scale) \div 2 ^ layer-idx)$ ]

```

```

definition fri-canonical-domains :: nat ⇒ 'f list list
where

```

```

  fri-canonical-domains n =
    map fri-canonical-domain-at [0.. $Suc\ n$ ]

```

```

definition generic-fri-canonical-sampled-layer-chain-evidence

```

```

  :: ('f list ⇒ bool) ⇒ 'f list ⇒ nat ⇒
    'f list ⇒ 'f list ⇒ 'f ⇒ nat list ⇒
    'f list list list ⇒ 'f list list ⇒ bool

```

```

where

```

```

  generic-fri-canonical-sampled-layer-chain-evidence low-degree
  candidate-table degree-bound roots challenges final-value query-idxs
  round-layers layers ⟷

```

*generic-fri-sampled-layer-chain-evidence low-degree candidate-table
degree-bound roots challenges final-value query-idxs round-layers
(fri-canonical-domains (length challenges)) layers*

definition *trace-fri-bad-with-canonical-sampled-layer-chain*

$:: ('f, 'a) \text{ protocol-channel-scheme} \Rightarrow$
 $(\text{unit list} \times ('f, 'a) \text{ protocol-channel-scheme}) \text{ option} \Rightarrow \text{bool}$

where

trace-fri-bad-with-canonical-sampled-layer-chain s out $\longleftrightarrow$
 $(\exists \text{ trace-roots trace-bs trace-final dg composition-roots}$
 $\text{composition-bs composition-final fri-query-idxs trace-round-layers}$
 $\text{composition-round-layers fr candidate-query-idxs trace-openings}$
 $\text{trace-table layers.}$
trace-fri-partial-candidate-opening-evidence s out trace-roots
 $\text{trace-bs trace-final dg composition-roots composition-bs}$
 $\text{composition-final fri-query-idxs trace-round-layers}$
 $\text{composition-round-layers fr candidate-query-idxs trace-openings}$
 $\text{trace-table} \wedge$
generic-fri-canonical-sampled-layer-chain-evidence
 $\text{trace-table-low-degree trace-table (clength - 1) trace-roots}$
 $\text{trace-bs trace-final fri-query-idxs trace-round-layers layers})$

definition *composition-fri-bad-with-canonical-sampled-layer-chain*

$:: ('f, 'a) \text{ protocol-channel-scheme} \Rightarrow$
 $(\text{unit list} \times ('f, 'a) \text{ protocol-channel-scheme}) \text{ option} \Rightarrow \text{bool}$

where

composition-fri-bad-with-canonical-sampled-layer-chain s out
 $\longleftrightarrow$
 $(\exists \text{ trace-roots trace-bs trace-final fri-dg}$
 $\text{opening-composition-roots composition-bs composition-final}$
 $\text{fri-query-idxs trace-round-layers composition-round-layers fr}$
 $\text{f-fri-roots f-final as dg composition-fri-roots final}$
 $\text{trace-query-idxs trace-openings composition-query-idxs}$
 $\text{composition-openings trace-table composition-table layers.}$
composition-fri-partial-candidate-opening-evidence s out trace-roots
 $\text{trace-bs trace-final fri-dg opening-composition-roots composition-bs}$
 $\text{composition-final fri-query-idxs trace-round-layers}$
 $\text{composition-round-layers fr f-fri-roots f-final as dg}$
 $\text{composition-fri-roots final trace-query-idxs trace-openings}$
 $\text{composition-query-idxs composition-openings trace-table}$
 $\text{composition-table} \wedge$
generic-fri-canonical-sampled-layer-chain-evidence
 $(\text{composition-table-low-degree (to-nat fri-dg)}) \text{ composition-table}$
 $(\text{to-nat fri-dg}) \text{ opening-composition-roots composition-bs}$
 $\text{composition-final fri-query-idxs composition-round-layers layers})$

definition *trace-fri-sampled-missing-canonical-sampled-layer-chain*

$:: ('f, 'a) \text{ protocol-channel-scheme} \Rightarrow$
 $(\text{unit list} \times ('f, 'a) \text{ protocol-channel-scheme}) \text{ option} \Rightarrow \text{bool}$

where

trace-fri-sampled-missing-canonical-sampled-layer-chain s out
 $\longleftrightarrow$
trace-fri-bad-with-sampled-layer-chain s out $\wedge$
 $\neg$ *trace-fri-bad-with-canonical-sampled-layer-chain s out*

definition *composition-fri-sampled-missing-canonical-sampled-layer-chain*

$:: ('f, 'a) \text{ protocol-channel-scheme} \Rightarrow$
 $(\text{unit list} \times ('f, 'a) \text{ protocol-channel-scheme}) \text{ option} \Rightarrow \text{bool}$

where

composition-fri-sampled-missing-canonical-sampled-layer-chain s out
 $\longleftrightarrow$
composition-fri-bad-with-sampled-layer-chain s out $\wedge$
 $\neg$ *composition-fri-bad-with-canonical-sampled-layer-chain s out*

definition *generic-fri-canonical-raw-index-failure*

$:: 'f \text{ list} \Rightarrow 'f \text{ list} \Rightarrow \text{nat list} \Rightarrow \text{bool}$

where

generic-fri-canonical-raw-index-failure roots challenges query-idxs
 $\longleftrightarrow$
 $(\exists \text{ round-idx} < \text{length query-idxs}.$
 $\exists \text{ layer-idx} < \text{length challenges}.$
 $\text{fri-evidence-layer-idx roots query-idxs round-idx layer-idx} \geq$
 $\text{length (fri-canonical-domain-at layer-idx)})$

definition *trace-fri-canonical-raw-index-failure*

$:: ('f, 'a) \text{ protocol-channel-scheme} \Rightarrow$
 $(\text{unit list} \times ('f, 'a) \text{ protocol-channel-scheme}) \text{ option} \Rightarrow \text{bool}$

where

trace-fri-canonical-raw-index-failure s out $\longleftrightarrow$
 $(\exists \text{ trace-roots trace-bs trace-final dg composition-roots}$
 $\text{composition-bs composition-final fri-query-idxs trace-round-layers}$
 $\text{composition-round-layers fr candidate-query-idxs trace-openings}$
 $\text{trace-table doms layers}.$
 $\text{trace-fri-partial-candidate-opening-evidence s out trace-roots}$
 $\text{trace-bs trace-final dg composition-roots composition-bs}$
 $\text{composition-final fri-query-idxs trace-round-layers}$
 $\text{composition-round-layers fr candidate-query-idxs trace-openings}$
 $\text{trace-table} \wedge$
 $\text{generic-fri-sampled-layer-chain-evidence trace-table-low-degree}$
 $\text{trace-table (clength - 1) trace-roots trace-bs trace-final}$
 $\text{fri-query-idxs trace-round-layers doms layers} \wedge$
 $\text{generic-fri-canonical-raw-index-failure trace-roots trace-bs}$
 $\text{fri-query-idxs})$

definition *composition-fri-canonical-raw-index-failure*

$:: ('f, 'a) \text{ protocol-channel-scheme} \Rightarrow$
 $(\text{unit list} \times ('f, 'a) \text{ protocol-channel-scheme}) \text{ option} \Rightarrow \text{bool}$

where

composition-fri-canonical-raw-index-failure s out $\longleftrightarrow$
 $(\exists$ *trace-roots* *trace-bs* *trace-final* *fri-dg*
opening-composition-roots *composition-bs* *composition-final*
fri-query-idxs *trace-round-layers* *composition-round-layers* *fr*
f-fri-roots *f-final* as *dg* *composition-fri-roots* *final*
trace-query-idxs *trace-openings* *composition-query-idxs*
composition-openings *trace-table* *composition-table* *doms* *layers*.
composition-fri-partial-candidate-opening-evidence s out *trace-roots*
trace-bs *trace-final* *fri-dg* *opening-composition-roots* *composition-bs*
composition-final *fri-query-idxs* *trace-round-layers*
composition-round-layers *fr* *f-fri-roots* *f-final* as *dg*
composition-fri-roots *final* *trace-query-idxs* *trace-openings*
composition-query-idxs *composition-openings* *trace-table*
composition-table $\wedge$
generic-fri-sampled-layer-chain-evidence
(*composition-table-low-degree* (*to-nat* *fri-dg*)) *composition-table*
(*to-nat* *fri-dg*) *opening-composition-roots* *composition-bs*
composition-final *fri-query-idxs* *composition-round-layers* *doms* *layers*
 $\wedge$
generic-fri-canonical-raw-index-failure *opening-composition-roots*
composition-bs *fri-query-idxs*)

definition *fri-sampled-successor-domains-square*

$:: 'f$ list $\Rightarrow$ nat list $\Rightarrow 'f$ list list $\Rightarrow$ bool

where

fri-sampled-successor-domains-square *roots* *query-idxs* *doms* $\longleftrightarrow$
 $(\forall$ *round-idx* $<$ length *query-idxs*.
 $\forall$ *layer-idx* $<$ length *roots*.
doms ! *Suc* *layer-idx* !
fri-evidence-next-idx *roots* *query-idxs* *round-idx* *layer-idx* =
(*doms* ! *layer-idx* !
fri-evidence-next-idx *roots* *query-idxs* *round-idx* *layer-idx*)²)

definition *generic-fri-full-cover-sampled-domain-failure*

$:: 'f$ list $\Rightarrow$ nat list $\Rightarrow 'f$ list list $\Rightarrow$ bool

where

generic-fri-full-cover-sampled-domain-failure *roots* *query-idxs* *doms*
 $\longleftrightarrow$
 $\neg$ *fri-sampled-next-domains-align* *roots* *query-idxs* *doms* $\vee$
 $\neg$ *fri-sampled-successor-domains-square* *roots* *query-idxs* *doms*

definition *trace-fri-sampled-full-cover-sampled-domain-failure*

$:: ('f, 'a)$ protocol-channel-scheme $\Rightarrow$

(unit list $\times$ ($'f, 'a$) protocol-channel-scheme) option $\Rightarrow$ bool

where

trace-fri-sampled-full-cover-sampled-domain-failure s out $\longleftrightarrow$
 $(\exists$ *trace-roots* *trace-bs* *trace-final* *dg* *composition-roots*
composition-bs *composition-final* *fri-query-idxs* *trace-round-layers*
composition-round-layers *fr* *candidate-query-idxs* *trace-openings*

trace-table doms *layers*.
trace-fri-partial-candidate-opening-evidence *s* out *trace-roots*
trace-bs *trace-final* *dg* *composition-roots* *composition-bs*
composition-final *fri-query-idxs* *trace-round-layers*
composition-round-layers *fr* *candidate-query-idxs* *trace-openings*
trace-table $\wedge$
generic-fri-sampled-layer-chain-evidence *trace-table-low-degree*
trace-table (*clength* - 1) *trace-roots* *trace-bs* *trace-final*
fri-query-idxs *trace-round-layers* doms *layers* $\wedge$
generic-fri-full-cover-sampled-domain-failure *trace-roots*
fri-query-idxs doms)

definition *composition-fri-sampled-full-cover-sampled-domain-failure*
 $:: (f, 'a) \text{ protocol-channel-scheme} \Rightarrow$
 $(\text{unit list} \times (f, 'a) \text{ protocol-channel-scheme}) \text{ option} \Rightarrow \text{bool}$

where

composition-fri-sampled-full-cover-sampled-domain-failure *s* out
 $\longleftrightarrow$
 $(\exists \text{ trace-roots } \text{trace-bs } \text{trace-final } \text{fri-dg}$
opening-composition-roots *composition-bs* *composition-final*
fri-query-idxs *trace-round-layers* *composition-round-layers* *fr*
f-fri-roots *f-final* as *dg* *composition-fri-roots* *final*
trace-query-idxs *trace-openings* *composition-query-idxs*
composition-openings *trace-table* *composition-table* doms *layers*.
composition-fri-partial-candidate-opening-evidence *s* out *trace-roots*
trace-bs *trace-final* *fri-dg* *opening-composition-roots* *composition-bs*
composition-final *fri-query-idxs* *trace-round-layers*
composition-round-layers *fr* *f-fri-roots* *f-final* as *dg*
composition-fri-roots *final* *trace-query-idxs* *trace-openings*
composition-query-idxs *composition-openings* *trace-table*
composition-table $\wedge$
generic-fri-sampled-layer-chain-evidence
(composition-table-low-degree (to-nat fri-dg)) *composition-table*
(to-nat fri-dg) *opening-composition-roots* *composition-bs*
composition-final *fri-query-idxs* *composition-round-layers* doms *layers*
 $\wedge$
generic-fri-full-cover-sampled-domain-failure
opening-composition-roots *fri-query-idxs* doms)

lemma *fri-sampled-successor-domains-squareD*:

assumes *fri-sampled-successor-domains-square* *roots* *query-idxs* doms

and *round-idx* < *length* *query-idxs*

and *layer-idx* < *length* *roots*

shows

doms ! *Suc* *layer-idx* !

fri-evidence-next-idx *roots* *query-idxs* *round-idx* *layer-idx* =

(*doms* ! *layer-idx* !

fri-evidence-next-idx *roots* *query-idxs* *round-idx* *layer-idx*)²

using *assms* **unfolding** *fri-sampled-successor-domains-square-def* **by** *blast*

```

lemma fri-canonical-domain-at-length:
  length (fri-canonical-domain-at layer-idx) =
    (clength * scale) div 2 ^ layer-idx
unfolding fri-canonical-domain-at-def by simp

lemma fri-canonical-domain-at-nth:
  assumes idx < length (fri-canonical-domain-at layer-idx)
  shows fri-canonical-domain-at layer-idx ! idx =
    (h ^ idx * shift) ^ (2 ^ layer-idx)
  using assms unfolding fri-canonical-domain-at-def by simp

lemma fri-canonical-domains-length:
  length (fri-canonical-domains n) = Suc n
unfolding fri-canonical-domains-def by simp

lemma fri-canonical-domains-nth:
  assumes layer-idx ≤ n
  shows fri-canonical-domains n ! layer-idx =
    fri-canonical-domain-at layer-idx
proof -
  show ?thesis
proof (cases layer-idx < n)
  case True
  have in-left:
    layer-idx < length (map fri-canonical-domain-at [0..n])
    using True by simp
  have append-nth:
    (map fri-canonical-domain-at [0..n] @
      [fri-canonical-domain-at n] ! layer-idx =
      (if layer-idx < length (map fri-canonical-domain-at [0..n])
        then map fri-canonical-domain-at [0..n] ! layer-idx
        else [fri-canonical-domain-at n] !
          (layer-idx - length (map fri-canonical-domain-at [0..n]))))
    by (rule nth-append)
  have fri-canonical-domains n ! layer-idx =
    map fri-canonical-domain-at [0..n] ! layer-idx
    unfolding fri-canonical-domains-def
    using append-nth in-left by simp
  also have ... = fri-canonical-domain-at layer-idx
    using True by simp
  finally show ?thesis .
next
  case False
  then have layer-idx = n
    using assms by simp
  then have eq-n: layer-idx = n .
  have len-left: length (map fri-canonical-domain-at [0..n]) = n
    by simp

```

```

have append-nth:
  (map fri-canonical-domain-at [0..<n] @
    [fri-canonical-domain-at n] ! n =
    (if n < length (map fri-canonical-domain-at [0..<n])
      then map fri-canonical-domain-at [0..<n] ! n
      else [fri-canonical-domain-at n] !
        (n - length (map fri-canonical-domain-at [0..<n])))
  by (rule nth-append)
show ?thesis
  unfolding fri-canonical-domains-def
  using eq-n len-left append-nth by simp
qed
qed

lemma fri-canonical-domain-at-0:
  fri-canonical-domain-at 0 = eval-domain
proof (rule nth-equalityI)
  show length (fri-canonical-domain-at 0) = length eval-domain
  by (simp add: fri-canonical-domain-at-length eval-domain-length)
next
  fix i
  assume i-bound: i < length (fri-canonical-domain-at 0)
  then have i-eval: i < clength * scale
  by (simp add: fri-canonical-domain-at-length)
  show fri-canonical-domain-at 0 ! i = eval-domain ! i
  using fri-canonical-domain-at-nth[OF i-bound] eval-domain-nth[OF i-eval]
  by simp
qed

lemma fri-canonical-domains-0:
  0 < length (fri-canonical-domains n)
  by (simp add: fri-canonical-domains-length)

lemma fri-canonical-domains-first:
  fri-canonical-domains n ! 0 = eval-domain
  using fri-canonical-domains-nth[of 0 n] fri-canonical-domain-at-0
  by simp

lemma fri-sampled-table-fold-to-canonical-domain:
  assumes fold:
    fri-sampled-table-fold ch len raw (2 ^ layer-idx) fri-dom layer chunk
    next-value
  and raw-bound: raw < length (fri-canonical-domain-at layer-idx)
  shows
    fri-sampled-table-fold ch len raw (2 ^ layer-idx)
    (fri-canonical-domain-at layer-idx) layer chunk next-value
proof -
  from fold obtain xp xp-path xn xn-path where
    chunk: fri-layer-opening-chunk len xp xp-path xn xn-path chunk

```

```

and match: fri-opening-matches-table len raw layer xp xn
and dom-row:
  fri-dom ! raw = (h ^ raw * shift) ^ (2 ^ layer-idx)
and next-eq:
  next-value =
    fri-table-fold-value ch layer fri-dom len (2 ^ layer-idx) raw
by (elim fri-sampled-table-foldE)
have canonical-row:
  fri-canonical-domain-at layer-idx ! raw =
    (h ^ raw * shift) ^ (2 ^ layer-idx)
by (rule fri-canonical-domain-at-nth[OF raw-bound])
have fold-eq:
  next-value =
    fri-table-fold-value ch layer (fri-canonical-domain-at layer-idx)
      len (2 ^ layer-idx) raw
using next-eq dom-row canonical-row
unfolding fri-table-fold-value-def by simp
show ?thesis
by (rule fri-sampled-table-foldI
  [OF chunk match canonical-row fold-eq])
qed

lemma generic-fri-sampled-layer-chain-to-canonical-domains:
assumes chain:
  generic-fri-sampled-layer-chain-evidence low-degree candidate-table
    degree-bound roots challenges final-value query-idxs round-layers
    doms layers
and raw-bounds:
   $\bigwedge \text{round-idx } \text{layer-idx.}$ 
     $\text{round-idx} < \text{length } \text{query-idxs} \implies$ 
     $\text{layer-idx} < \text{length } \text{challenges} \implies$ 
     $\text{fri-evidence-layer-idx } \text{roots } \text{query-idxs } \text{round-idx } \text{layer-idx} <$ 
     $\text{length } (\text{fri-canonical-domain-at } \text{layer-idx})$ 
shows
  generic-fri-canonical-sampled-layer-chain-evidence low-degree
    candidate-table degree-bound roots challenges final-value query-idxs
    round-layers layers
proof –
have partial:
  generic-fri-partial-evidence low-degree candidate-table degree-bound
    roots challenges final-value query-idxs round-layers
by (rule generic-fri-sampled-layer-chain-evidenceD(1)[OF chain])
have len-layers: length layers = Suc (length challenges)
by (rule generic-fri-sampled-layer-chain-evidenceD(3)[OF chain])
have layer0: layers ! 0 = candidate-table
by (rule generic-fri-sampled-layer-chain-evidenceD(5)[OF chain])
have final:
  fri-final-constant-consistent (layers ! length challenges)
    final-value

```

```

  by (rule generic-fri-sampled-layer-chain-evidenceD(6)[OF chain])
have canonical-chain:
  generic-fri-sampled-layer-chain-evidence low-degree candidate-table
    degree-bound roots challenges final-value query-idxs round-layers
    (fri-canonical-domains (length challenges)) layers
  unfolding generic-fri-sampled-layer-chain-evidence-def
proof (intro conjI allI impI)
  show
    generic-fri-partial-evidence low-degree candidate-table degree-bound
      roots challenges final-value query-idxs round-layers
    by (rule partial)
next
  show length (fri-canonical-domains (length challenges)) =
    Suc (length challenges)
    by (rule fri-canonical-domains-length)
next
  show length layers = Suc (length challenges)
    by (rule len-layers)
next
  show fri-canonical-domains (length challenges) ! 0 = eval-domain
    by (rule fri-canonical-domains-first)
next
  show layers ! 0 = candidate-table
    by (rule layer0)
next
  show
    fri-final-constant-consistent (layers ! length challenges)
      final-value
    by (rule final)
next
  fix round-idx layer-idx
  assume round-bound: round-idx < length query-idxs
  assume layer-bound: layer-idx < length challenges
  show
    fri-evidence-next-idx roots query-idxs round-idx layer-idx <
      length (layers ! Suc layer-idx)
    by (rule generic-fri-sampled-layer-chain-evidence-sample(1)
      [OF chain round-bound layer-bound])
next
  fix round-idx layer-idx
  assume round-bound: round-idx < length query-idxs
  assume layer-bound: layer-idx < length challenges
  have old-fold:
    fri-sampled-table-fold (challenges ! layer-idx)
      (fri-evidence-layer-len roots layer-idx)
      (fri-evidence-layer-idx roots query-idxs round-idx layer-idx)
      (2 ^ layer-idx)
      (doms ! layer-idx)
      (layers ! layer-idx)

```

```

    (round-layers ! round-idx ! layer-idx)
    (layers ! Suc layer-idx !
      fri-evidence-next-idx roots query-idxs round-idx layer-idx)
  by (rule generic-fri-sampled-layer-chain-evidence-sample(2)
    [OF chain round-bound layer-bound])
have canonical-at:
  fri-sampled-table-fold (challenges ! layer-idx)
    (fri-evidence-layer-len roots layer-idx)
    (fri-evidence-layer-idx roots query-idxs round-idx layer-idx)
    (2 ^ layer-idx)
    (fri-canonical-domain-at layer-idx)
    (layers ! layer-idx)
    (round-layers ! round-idx ! layer-idx)
    (layers ! Suc layer-idx !
      fri-evidence-next-idx roots query-idxs round-idx layer-idx)
  by (rule fri-sampled-table-fold-to-canonical-domain
    [OF old-fold raw-bounds[OF round-bound layer-bound]])
have dom-nth:
  fri-canonical-domains (length challenges) ! layer-idx =
    fri-canonical-domain-at layer-idx
  by (rule fri-canonical-domains-nth) (use layer-bound in simp)
show
  fri-sampled-table-fold (challenges ! layer-idx)
    (fri-evidence-layer-len roots layer-idx)
    (fri-evidence-layer-idx roots query-idxs round-idx layer-idx)
    (2 ^ layer-idx)
    (fri-canonical-domains (length challenges) ! layer-idx)
    (layers ! layer-idx)
    (round-layers ! round-idx ! layer-idx)
    (layers ! Suc layer-idx !
      fri-evidence-next-idx roots query-idxs round-idx layer-idx)
  using canonical-at dom-nth by simp
qed
show ?thesis
  using canonical-chain
  unfolding generic-fri-canonical-sampled-layer-chain-evidence-def
  by simp
qed

lemma generic-fri-canonical-sampled-layer-chain-evidenceD:
  assumes
    generic-fri-canonical-sampled-layer-chain-evidence low-degree
    candidate-table degree-bound roots challenges final-value query-idxs
    round-layers layers
  shows
    generic-fri-sampled-layer-chain-evidence low-degree candidate-table
    degree-bound roots challenges final-value query-idxs round-layers
    (fri-canonical-domains (length challenges)) layers
  using assms

```


unfolding *generic-fri-canonical-sampled-layer-chain-evidence-def* **by** *simp*

lemma *generic-fri-canonical-sampled-layer-chain-evidenceI*:

assumes

generic-fri-sampled-layer-chain-evidence low-degree candidate-table
degree-bound roots challenges final-value query-idxs round-layers
(fri-canonical-domains (length challenges)) layers

shows

generic-fri-canonical-sampled-layer-chain-evidence low-degree
candidate-table degree-bound roots challenges final-value query-idxs
round-layers layers

using *assms*

unfolding *generic-fri-canonical-sampled-layer-chain-evidence-def* **by** *simp*

lemma *generic-fri-sampled-missing-canonical-imp-raw-index-failure*:

assumes *chain*:

generic-fri-sampled-layer-chain-evidence low-degree candidate-table
degree-bound roots challenges final-value query-idxs round-layers
doms layers

and *missing*:

$\neg$ *generic-fri-canonical-sampled-layer-chain-evidence low-degree*
candidate-table degree-bound roots challenges final-value query-idxs
round-layers layers

shows *generic-fri-canonical-raw-index-failure roots challenges query-idxs*

proof (rule *ccontr*)

assume *not-failure*:

$\neg$ *generic-fri-canonical-raw-index-failure roots challenges*
query-idxs

have *raw-bounds*:

$\bigwedge$ *round-idx layer-idx.*
round-idx < length query-idxs $\implies$
layer-idx < length challenges $\implies$
fri-evidence-layer-idx roots query-idxs round-idx layer-idx <
length (fri-canonical-domain-at layer-idx)

using *not-failure*

unfolding *generic-fri-canonical-raw-index-failure-def* **by** *force*

have *canonical*:

generic-fri-canonical-sampled-layer-chain-evidence low-degree
candidate-table degree-bound roots challenges final-value query-idxs
round-layers layers

by (rule *generic-fri-sampled-layer-chain-to-canonical-domains*
 $[OF\ chain\ raw-bounds]$)

show *False*

using *missing canonical* **by** *contradiction*

qed

lemma *trace-fri-missing-canonical-sampled-imp-raw-index-failure*:

assumes *missing*:

trace-fri-sampled-missing-canonical-sampled-layer-chain s out

shows *trace-fri-canonical-raw-index-failure s out*
proof –
from *missing* **have** *sampld:*
trace-fri-bad-with-sampled-layer-chain s out
and *not-canonical:*
 $\neg$ *trace-fri-bad-with-canonical-sampled-layer-chain s out*
unfolding *trace-fri-sampled-missing-canonical-sampled-layer-chain-def*
by *blast+*
from *sampld* **obtain** *trace-roots trace-bs trace-final dg*
composition-roots composition-bs composition-final fri-query-idxs
trace-round-layers composition-round-layers fr candidate-query-idxs
trace-openings trace-table doms layers
where *evidence:*
trace-fri-partial-candidate-opening-evidence s out trace-roots
trace-bs trace-final dg composition-roots composition-bs
composition-final fri-query-idxs trace-round-layers
composition-round-layers fr candidate-query-idxs trace-openings
trace-table
and *chain:*
generic-fri-sampled-layer-chain-evidence trace-table-low-degree
trace-table (clength – 1) trace-roots trace-bs trace-final
fri-query-idxs trace-round-layers doms layers
unfolding *trace-fri-bad-with-sampled-layer-chain-def* **by** *blast*
have *generic-missing:*
 $\neg$ *generic-fri-canonical-sampled-layer-chain-evidence*
trace-table-low-degree trace-table (clength – 1) trace-roots
trace-bs trace-final fri-query-idxs trace-round-layers layers
proof
assume *canonical:*
generic-fri-canonical-sampled-layer-chain-evidence
trace-table-low-degree trace-table (clength – 1) trace-roots
trace-bs trace-final fri-query-idxs trace-round-layers layers
then have *trace-fri-bad-with-canonical-sampled-layer-chain s out*
unfolding *trace-fri-bad-with-canonical-sampled-layer-chain-def*
using *evidence* **by** *blast*
then show *False*
using *not-canonical* **by** *contradiction*
qed
have *raw-failure:*
generic-fri-canonical-raw-index-failure trace-roots trace-bs
fri-query-idxs
by (*rule generic-fri-sampled-missing-canonical-imp-raw-index-failure*
 $[OF\ chain\ generic-missing]$)
show *?thesis*
unfolding *trace-fri-canonical-raw-index-failure-def*
using *evidence chain raw-failure* **by** *blast*
qed

lemma *composition-fri-missing-canonical-sampled-imp-raw-index-failure:*

assumes *missing*:
composition-fri-sampled-missing-canonical-sampled-layer-chain s out
shows *composition-fri-canonical-raw-index-failure s out*
proof –
from *missing* **have** *sampled*:
composition-fri-bad-with-sampled-layer-chain s out
and *not-canonical*:
 $\neg$ *composition-fri-bad-with-canonical-sampled-layer-chain s out*
unfolding *composition-fri-sampled-missing-canonical-sampled-layer-chain-def*
by *blast+*
from *sampled* **obtain** *trace-roots trace-bs trace-final fri-dg*
opening-composition-roots composition-bs composition-final
fri-query-idxs trace-round-layers composition-round-layers fr
f-fri-roots f-final as dg composition-fri-roots final
trace-query-idxs trace-openings composition-query-idxs
composition-openings trace-table composition-table doms layers
where *evidence*:
composition-fri-partial-candidate-opening-evidence s out trace-roots
trace-bs trace-final fri-dg opening-composition-roots composition-bs
composition-final fri-query-idxs trace-round-layers
composition-round-layers fr f-fri-roots f-final as dg
composition-fri-roots final trace-query-idxs trace-openings
composition-query-idxs composition-openings trace-table
composition-table
and *chain*:
generic-fri-sampled-layer-chain-evidence
(composition-table-low-degree (to-nat fri-dg)) composition-table
(to-nat fri-dg) opening-composition-roots composition-bs
composition-final fri-query-idxs composition-round-layers doms layers
unfolding *composition-fri-bad-with-sampled-layer-chain-def* **by** *auto*
have *generic-missing*:
 $\neg$ *generic-fri-canonical-sampled-layer-chain-evidence*
(composition-table-low-degree (to-nat fri-dg)) composition-table
(to-nat fri-dg) opening-composition-roots composition-bs
composition-final fri-query-idxs composition-round-layers layers
proof
assume *canonical*:
generic-fri-canonical-sampled-layer-chain-evidence
(composition-table-low-degree (to-nat fri-dg)) composition-table
(to-nat fri-dg) opening-composition-roots composition-bs
composition-final fri-query-idxs composition-round-layers layers
then have *composition-fri-bad-with-canonical-sampled-layer-chain s out*
unfolding *composition-fri-bad-with-canonical-sampled-layer-chain-def*
using *evidence* **by** *meson*
then show *False*
using *not-canonical* **by** *contradiction*
qed
have *raw-failure*:
generic-fri-canonical-raw-index-failure opening-composition-roots

```

    composition-bs fri-query-idxs
  by (rule generic-fri-sampled-missing-canonical-imp-raw-index-failure
      [OF chain generic-missing])
  show ?thesis
    unfolding composition-fri-canonical-raw-index-failure-def
    using evidence chain raw-failure by meson
qed

lemma wp-trace-fri-missing-canonical-sampled-bound-from-raw-index:
  assumes raw-index:
    wp-event verify-monad (trace-fri-canonical-raw-index-failure s) s
    ≤ R
  shows
    wp-event verify-monad
      (trace-fri-sampled-missing-canonical-sampled-layer-chain s) s ≤ R
  proof -
    have wp-event verify-monad
      (trace-fri-sampled-missing-canonical-sampled-layer-chain s) s
      ≤ wp-event verify-monad
        (trace-fri-canonical-raw-index-failure s) s
    by (rule wp-event-mono)
      (rule trace-fri-missing-canonical-sampled-imp-raw-index-failure)
    also have ... ≤ R
    by (rule raw-index)
    finally show ?thesis .
  qed

lemma wp-composition-fri-missing-canonical-sampled-bound-from-raw-index:
  assumes raw-index:
    wp-event verify-monad (composition-fri-canonical-raw-index-failure s) s
    ≤ R
  shows
    wp-event verify-monad
      (composition-fri-sampled-missing-canonical-sampled-layer-chain s) s
      ≤ R
  proof -
    have wp-event verify-monad
      (composition-fri-sampled-missing-canonical-sampled-layer-chain s) s
      ≤ wp-event verify-monad
        (composition-fri-canonical-raw-index-failure s) s
    by (rule wp-event-mono)
      (rule composition-fri-missing-canonical-sampled-imp-raw-index-failure)
    also have ... ≤ R
    by (rule raw-index)
    finally show ?thesis .
  qed

lemma generic-fri-sampled-layer-chain-no-canonical-raw-index-failure:
  assumes chain:

```

```

generic-fri-sampled-layer-chain-evidence low-degree candidate-table
degree-bound roots challenges final-value query-idxs round-layers
doms layers
shows  $\neg$  generic-fri-canonical-raw-index-failure roots challenges
query-idxs
proof
  assume failure:
    generic-fri-canonical-raw-index-failure roots challenges query-idxs
  then obtain round-idx layer-idx where round-bound:
    round-idx < length query-idxs
  and layer-bound: layer-idx < length challenges
  and raw-ge:
    fri-evidence-layer-idx roots query-idxs round-idx layer-idx  $\geq$ 
    length (fri-canonical-domain-at layer-idx)
  unfolding generic-fri-canonical-raw-index-failure-def by blast
  have partial:
    generic-fri-partial-evidence low-degree candidate-table degree-bound
    roots challenges final-value query-idxs round-layers
    by (rule generic-fri-sampled-layer-chain-evidenceD(1)[OF chain])
  have roots-challenges: length challenges = length roots
    by (rule generic-fri-partial-evidence-shapes(2)[OF partial])
  have roots-bound: layer-idx < length roots
    using layer-bound roots-challenges by simp
  have fold:
    fri-sampled-table-fold (challenges ! layer-idx)
    (fri-evidence-layer-len roots layer-idx)
    (fri-evidence-layer-idx roots query-idxs round-idx layer-idx)
    (2 ^ layer-idx)
    (doms ! layer-idx)
    (layers ! layer-idx)
    (round-layers ! round-idx ! layer-idx)
    (layers ! Suc layer-idx !
      fri-evidence-next-idx roots query-idxs round-idx layer-idx)
    by (rule generic-fri-sampled-layer-chain-evidence-sample(2)
      [OF chain round-bound layer-bound])
  from fold obtain xp xp-path xn xn-path where
    match:
      fri-opening-matches-table (fri-evidence-layer-len roots layer-idx)
      (fri-evidence-layer-idx roots query-idxs round-idx layer-idx)
      (layers ! layer-idx) xp xn
    by (elim fri-sampled-table-foldE)
  have raw-lt-len:
    fri-evidence-layer-idx roots query-idxs round-idx layer-idx <
    fri-evidence-layer-len roots layer-idx
    by (rule fri-opening-matches-tableD(1)[OF match])
  have len-eq:
    fri-evidence-layer-len roots layer-idx =
    length (fri-canonical-domain-at layer-idx)
  unfolding fri-evidence-layer-len-def fri-canonical-domain-at-length

```

```

    using fri-layer-lengths-nth-div[OF roots-bound, of clength * scale]
  by simp
show False
  using raw-lt-len raw-ge unfolding len-eq by simp
qed

lemma trace-fri-canonical-raw-index-failure-false:
   $\neg$  trace-fri-canonical-raw-index-failure s out
proof
  assume raw: trace-fri-canonical-raw-index-failure s out
  then obtain trace-roots trace-bs trace-final dg composition-roots
    composition-bs composition-final fri-query-idxs trace-round-layers
    composition-round-layers fr candidate-query-idxs trace-openings
    trace-table doms layers
  where chain:
    generic-fri-sampled-layer-chain-evidence trace-table-low-degree
      trace-table (clength - 1) trace-roots trace-bs trace-final
      fri-query-idxs trace-round-layers doms layers
  and failure:
    generic-fri-canonical-raw-index-failure trace-roots trace-bs
      fri-query-idxs
  unfolding trace-fri-canonical-raw-index-failure-def by blast
  have  $\neg$  generic-fri-canonical-raw-index-failure trace-roots trace-bs
    fri-query-idxs
  by (rule generic-fri-sampled-layer-chain-no-canonical-raw-index-failure
    [OF chain])
  then show False
  using failure by contradiction
qed

```

```

lemma composition-fri-canonical-raw-index-failure-false:
   $\neg$  composition-fri-canonical-raw-index-failure s out
proof
  assume raw: composition-fri-canonical-raw-index-failure s out
  then obtain trace-roots trace-bs trace-final fri-dg
    opening-composition-roots composition-bs composition-final
    fri-query-idxs trace-round-layers composition-round-layers fr
    f-fri-roots f-final as dg composition-fri-roots final
    trace-query-idxs trace-openings composition-query-idxs
    composition-openings trace-table composition-table doms layers
  where chain:
    generic-fri-sampled-layer-chain-evidence
      (composition-table-low-degree (to-nat fri-dg)) composition-table
      (to-nat fri-dg) opening-composition-roots composition-bs
      composition-final fri-query-idxs composition-round-layers doms layers
  and failure:
    generic-fri-canonical-raw-index-failure opening-composition-roots
      composition-bs fri-query-idxs
  unfolding composition-fri-canonical-raw-index-failure-def by meson

```

have $\neg$ *generic-fri-canonical-raw-index-failure*
opening-composition-roots composition-bs fri-query-idxs
by (rule *generic-fri-sampled-layer-chain-no-canonical-raw-index-failure*
[OF chain])
then show *False*
using *failure* **by** *contradiction*
qed

lemma *wp-trace-fri-canonical-raw-index-failure-zero*:
wp-event verify-monad (trace-fri-canonical-raw-index-failure s) s $\leq$ 0
proof –
have *wp-event verify-monad (trace-fri-canonical-raw-index-failure s) s*
 $\leq$ *wp-event verify-monad (λ -. *False*) s*
by (rule *wp-event-mono*)
(simp add: trace-fri-canonical-raw-index-failure-false)
also have ... = 0
unfolding *wp-event-def wp-def dist-expect-def* **by** *simp*
finally show *?thesis* .
qed

lemma *wp-composition-fri-canonical-raw-index-failure-zero*:
wp-event verify-monad
(composition-fri-canonical-raw-index-failure s) s $\leq$ 0
proof –
have *wp-event verify-monad*
(composition-fri-canonical-raw-index-failure s) s
 $\leq$ *wp-event verify-monad (λ -. *False*) s*
by (rule *wp-event-mono*)
(simp add: composition-fri-canonical-raw-index-failure-false)
also have ... = 0
unfolding *wp-event-def wp-def dist-expect-def* **by** *simp*
finally show *?thesis* .
qed

lemma *wp-trace-fri-missing-canonical-sampled-zero*:
wp-event verify-monad
(trace-fri-sampled-missing-canonical-sampled-layer-chain s) s $\leq$ 0
by (rule *wp-trace-fri-missing-canonical-sampled-bound-from-raw-index*)
(rule wp-trace-fri-canonical-raw-index-failure-zero)

lemma *wp-composition-fri-missing-canonical-sampled-zero*:
wp-event verify-monad
(composition-fri-sampled-missing-canonical-sampled-layer-chain s) s
 $\leq$ 0
by (rule *wp-composition-fri-missing-canonical-sampled-bound-from-raw-index*)
(rule wp-composition-fri-canonical-raw-index-failure-zero)

lemma *trace-fri-bad-with-canonical-sampled-layer-chain-imp-sampled*:
assumes *trace-fri-bad-with-canonical-sampled-layer-chain s out*

shows *trace-fri-bad-with-sampled-layer-chain s out*
proof –
from *assms* **obtain** *trace-roots trace-bs trace-final dg composition-roots*
composition-bs composition-final fri-query-idxs trace-round-layers
composition-round-layers fr candidate-query-idxs trace-openings
trace-table layers
where *evidence*:
trace-fri-partial-candidate-opening-evidence s out trace-roots
trace-bs trace-final dg composition-roots composition-bs
composition-final fri-query-idxs trace-round-layers
composition-round-layers fr candidate-query-idxs trace-openings
trace-table
and *chain*:
generic-fri-canonical-sampled-layer-chain-evidence
trace-table-low-degree trace-table (clength – 1) trace-roots
trace-bs trace-final fri-query-idxs trace-round-layers layers
unfolding *trace-fri-bad-with-canonical-sampled-layer-chain-def* **by** *blast*
have *sampled*:
generic-fri-sampled-layer-chain-evidence trace-table-low-degree
trace-table (clength – 1) trace-roots trace-bs trace-final
fri-query-idxs trace-round-layers
(fri-canonical-domains (length trace-bs)) layers
by *(rule generic-fri-canonical-sampled-layer-chain-evidenceD[OF chain])*
show *?thesis*
unfolding *trace-fri-bad-with-sampled-layer-chain-def*
using *evidence sampled by blast*
qed

lemma *composition-fri-bad-with-canonical-sampled-layer-chain-imp-sampled*:
assumes *composition-fri-bad-with-canonical-sampled-layer-chain s out*
shows *composition-fri-bad-with-sampled-layer-chain s out*
proof –
from *assms* **obtain** *trace-roots trace-bs trace-final fri-dg*
opening-composition-roots composition-bs composition-final
fri-query-idxs trace-round-layers composition-round-layers fr
f-fri-roots f-final as dg composition-fri-roots final
trace-query-idxs trace-openings composition-query-idxs
composition-openings trace-table composition-table layers
where *evidence*:
composition-fri-partial-candidate-opening-evidence s out trace-roots
trace-bs trace-final fri-dg opening-composition-roots composition-bs
composition-final fri-query-idxs trace-round-layers
composition-round-layers fr f-fri-roots f-final as dg
composition-fri-roots final trace-query-idxs trace-openings
composition-query-idxs composition-openings trace-table
composition-table
and *chain*:
generic-fri-canonical-sampled-layer-chain-evidence
(composition-table-low-degree (to-nat fri-dg)) composition-table

(to-nat fri-dg) opening-composition-roots composition-bs
 composition-final fri-query-idxs composition-round-layers layers
unfolding composition-fri-bad-with-canonical-sampled-layer-chain-def
by meson
have sampled:
 generic-fri-sampled-layer-chain-evidence
 (composition-table-low-degree (to-nat fri-dg)) composition-table
 (to-nat fri-dg) opening-composition-roots composition-bs
 composition-final fri-query-idxs composition-round-layers
 (fri-canonical-domains (length composition-bs)) layers
by (rule generic-fri-canonical-sampled-layer-chain-evidenceD[OF chain])
show ?thesis
unfolding composition-fri-bad-with-sampled-layer-chain-def
using evidence sampled **by** meson
qed

lemma trace-fri-sampled-layer-chain-canonical-or-missing:
assumes trace-fri-bad-with-sampled-layer-chain s out
shows
 trace-fri-bad-with-canonical-sampled-layer-chain s out $\vee$
 trace-fri-sampled-missing-canonical-sampled-layer-chain s out
using assms
unfolding trace-fri-sampled-missing-canonical-sampled-layer-chain-def
by blast

lemma composition-fri-sampled-layer-chain-canonical-or-missing:
assumes composition-fri-bad-with-sampled-layer-chain s out
shows
 composition-fri-bad-with-canonical-sampled-layer-chain s out $\vee$
 composition-fri-sampled-missing-canonical-sampled-layer-chain s out
using assms
unfolding
 composition-fri-sampled-missing-canonical-sampled-layer-chain-def
by blast

lemma wp-trace-fri-sampled-bound-from-canonical-and-missing:
assumes canonical:
 wp-event verify-monad
 (trace-fri-bad-with-canonical-sampled-layer-chain s) $s \leq C$
and missing:
 wp-event verify-monad
 (trace-fri-sampled-missing-canonical-sampled-layer-chain s) $s \leq M$
shows
 wp-event verify-monad (trace-fri-bad-with-sampled-layer-chain s) s
 $\leq C + M$
proof –
have wp-event verify-monad (trace-fri-bad-with-sampled-layer-chain s) s
 $\leq$ wp-event verify-monad
 (λ out. trace-fri-bad-with-canonical-sampled-layer-chain s out $\vee$

```

      trace-fri-sampled-missing-canonical-sampled-layer-chain s out) s
    by (rule wp-event-mono)
      (rule trace-fri-sampled-layer-chain-canonical-or-missing)
  also have ...  $\leq C + M$ 
    by (rule order-trans[OF wp-event-union-bound
      add-mono[OF canonical missing]])
  finally show ?thesis .
qed

lemma wp-composition-fri-sampled-bound-from-canonical-and-missing:
  assumes canonical:
    wp-event verify-monad
      (composition-fri-bad-with-canonical-sampled-layer-chain s) s  $\leq C$ 
  and missing:
    wp-event verify-monad
      (composition-fri-sampled-missing-canonical-sampled-layer-chain s) s
       $\leq M$ 
  shows
    wp-event verify-monad
      (composition-fri-bad-with-sampled-layer-chain s) s  $\leq C + M$ 
proof -
  have wp-event verify-monad
    (composition-fri-bad-with-sampled-layer-chain s) s
     $\leq$  wp-event verify-monad
      ( $\lambda$ out.
        composition-fri-bad-with-canonical-sampled-layer-chain s out  $\vee$ 
        composition-fri-sampled-missing-canonical-sampled-layer-chain s out)
      s
    by (rule wp-event-mono)
      (rule composition-fri-sampled-layer-chain-canonical-or-missing)
  also have ...  $\leq C + M$ 
    by (rule order-trans[OF wp-event-union-bound
      add-mono[OF canonical missing]])
  finally show ?thesis .
qed

lemma wp-trace-fri-sampled-bound-from-canonical:
  assumes canonical:
    wp-event verify-monad
      (trace-fri-bad-with-canonical-sampled-layer-chain s) s  $\leq C$ 
  shows
    wp-event verify-monad (trace-fri-bad-with-sampled-layer-chain s) s
     $\leq C$ 
proof -
  have wp-event verify-monad (trace-fri-bad-with-sampled-layer-chain s) s
     $\leq C + 0$ 
    by (rule wp-trace-fri-sampled-bound-from-canonical-and-missing
      [OF canonical wp-trace-fri-missing-canonical-sampled-zero])
  then show ?thesis by simp

```

qed

lemma *wp-composition-fri-sampled-bound-from-canonical:*

assumes *canonical:*

wp-event verify-monad

(composition-fri-bad-with-canonical-sampled-layer-chain s) s ≤ C

shows

wp-event verify-monad (composition-fri-bad-with-sampled-layer-chain s) s
≤ C

proof –

have *wp-event verify-monad*

(composition-fri-bad-with-sampled-layer-chain s) s ≤ C + 0

by (rule *wp-composition-fri-sampled-bound-from-canonical-and-missing*
[*OF canonical wp-composition-fri-missing-canonical-sampled-zero*])

then show *?thesis* **by** *simp*

qed

lemma *wp-trace-fri-canonical-sampled-bound-from-sampled:*

assumes *sampling:*

wp-event verify-monad (trace-fri-bad-with-sampled-layer-chain s) s
≤ R

shows

wp-event verify-monad
(trace-fri-bad-with-canonical-sampled-layer-chain s) s ≤ R

proof –

have *wp-event verify-monad*

(trace-fri-bad-with-canonical-sampled-layer-chain s) s ≤

wp-event verify-monad (trace-fri-bad-with-sampled-layer-chain s) s

by (rule *wp-event-mono*)

(rule *trace-fri-bad-with-canonical-sampled-layer-chain-imp-sampled*)

then show *?thesis*

by (rule *order-trans*[*OF - sampling*])

qed

lemma *wp-composition-fri-canonical-sampled-bound-from-sampled:*

assumes *sampling:*

wp-event verify-monad

(composition-fri-bad-with-sampled-layer-chain s) s ≤ R

shows

wp-event verify-monad
(composition-fri-bad-with-canonical-sampled-layer-chain s) s ≤ R

proof –

have *wp-event verify-monad*

(composition-fri-bad-with-canonical-sampled-layer-chain s) s ≤

wp-event verify-monad

(composition-fri-bad-with-sampled-layer-chain s) s

by (rule *wp-event-mono*)

(rule *composition-fri-bad-with-canonical-sampled-layer-chain-imp-sampled*)

then show *?thesis*

by (rule order-trans[OF - sampled])
qed

lemma *wp-trace-fri-canonical-sampled-bound-from-full-cover-and-missing:*

assumes *full-bound:*

wp-event verify-monad

(*trace-fri-bad-with-full-cover-layer-chain s bad*) $s \leq R$

and *missing-bound:*

wp-event verify-monad

(*trace-fri-sampled-missing-full-cover s bad*) $s \leq M$

shows

wp-event verify-monad

(*trace-fri-bad-with-canonical-sampled-layer-chain s*) $s \leq R + M$

by (rule *wp-trace-fri-canonical-sampled-bound-from-sampled*)

(rule *wp-trace-fri-sampled-bound-from-full-cover-and-missing*
[OF *full-bound missing-bound*])

lemma *wp-composition-fri-canonical-sampled-bound-from-full-cover-and-missing:*

assumes *full-bound:*

wp-event verify-monad

(*composition-fri-bad-with-full-cover-layer-chain s bad*) $s \leq R$

and *missing-bound:*

wp-event verify-monad

(*composition-fri-sampled-missing-full-cover s bad*) $s \leq M$

shows

wp-event verify-monad

(*composition-fri-bad-with-canonical-sampled-layer-chain s*) s
 $\leq R + M$

by (rule *wp-composition-fri-canonical-sampled-bound-from-sampled*)

(rule *wp-composition-fri-sampled-bound-from-full-cover-and-missing*
[OF *full-bound missing-bound*])

lemma *verify-monad-trace-fri-missing-canonical-sampled-extracts-evidence:*

assumes *outcome:*

Some (result, final-state) ∈ set-dist (execute verify-monad s)

and *missing:*

trace-fri-sampled-missing-canonical-sampled-layer-chain s
(*Some (result, final-state)*)

obtains *trace-roots trace-bs trace-final dg composition-roots*

composition-bs composition-final fri-query-idxs trace-round-layers

composition-round-layers fr candidate-query-idxs trace-openings

trace-table doms layers

where

trace-fri-partial-candidate-opening-evidence s

(*Some (result, final-state)*) *trace-roots trace-bs trace-final dg*

composition-roots composition-bs composition-final fri-query-idxs

trace-round-layers composition-round-layers fr candidate-query-idxs

trace-openings trace-table

generic-fri-sampled-layer-chain-evidence trace-table-low-degree

```

    trace-table (clength - 1) trace-roots trace-bs trace-final
    fri-query-idxs trace-round-layers doms layers
   $\wedge$  canonical-layers.  $\neg$ 
    generic-fri-canonical-sampled-layer-chain-evidence
    trace-table-low-degree trace-table (clength - 1) trace-roots
    trace-bs trace-final fri-query-idxs trace-round-layers
    canonical-layers
proof -
  have sampled:
    trace-fri-bad-with-sampled-layer-chain s
    (Some (result, final-state))
  and not-canonical:
     $\neg$  trace-fri-bad-with-canonical-sampled-layer-chain s
    (Some (result, final-state))
  using missing
  unfolding trace-fri-sampled-missing-canonical-sampled-layer-chain-def
  by blast+
from sampled obtain trace-roots trace-bs trace-final dg composition-roots
  composition-bs composition-final fri-query-idxs trace-round-layers
  composition-round-layers fr candidate-query-idxs trace-openings
  trace-table doms layers
where evidence:
  trace-fri-partial-candidate-opening-evidence s
  (Some (result, final-state)) trace-roots trace-bs trace-final dg
  composition-roots composition-bs composition-final fri-query-idxs
  trace-round-layers composition-round-layers fr candidate-query-idxs
  trace-openings trace-table
and chain:
  generic-fri-sampled-layer-chain-evidence trace-table-low-degree
  trace-table (clength - 1) trace-roots trace-bs trace-final
  fri-query-idxs trace-round-layers doms layers
  unfolding trace-fri-bad-with-sampled-layer-chain-def by blast
have no-canonical:
   $\wedge$  canonical-layers.  $\neg$ 
    generic-fri-canonical-sampled-layer-chain-evidence
    trace-table-low-degree trace-table (clength - 1) trace-roots
    trace-bs trace-final fri-query-idxs trace-round-layers
    canonical-layers
proof
  fix canonical-layers
  assume canonical:
    generic-fri-canonical-sampled-layer-chain-evidence
    trace-table-low-degree trace-table (clength - 1) trace-roots
    trace-bs trace-final fri-query-idxs trace-round-layers
    canonical-layers
  have trace-fri-bad-with-canonical-sampled-layer-chain s
    (Some (result, final-state))
  unfolding trace-fri-bad-with-canonical-sampled-layer-chain-def
  using evidence canonical by blast

```

```

then show False
  using not-canonical by contradiction
qed
show ?thesis
  by (rule that[OF evidence chain no-canonical])
qed

lemma verify-monad-composition-fri-missing-canonical-sampled-extracts-evidence:
  assumes outcome:
    Some (result, final-state) ∈ set-dist (execute verify-monad s)
  and missing:
    composition-fri-sampled-missing-canonical-sampled-layer-chain s
    (Some (result, final-state))
  obtains trace-roots trace-bs trace-final fri-dg
    opening-composition-roots composition-bs composition-final
    fri-query-idxs trace-round-layers composition-round-layers fr
    f-fri-roots f-final as dg composition-fri-roots final
    trace-query-idxs trace-openings composition-query-idxs
    composition-openings trace-table composition-table doms layers
  where
    composition-fri-partial-candidate-opening-evidence s
    (Some (result, final-state)) trace-roots trace-bs trace-final fri-dg
    opening-composition-roots composition-bs composition-final
    fri-query-idxs trace-round-layers composition-round-layers fr
    f-fri-roots f-final as dg composition-fri-roots final
    trace-query-idxs trace-openings composition-query-idxs
    composition-openings trace-table composition-table
    generic-fri-sampled-layer-chain-evidence
    (composition-table-low-degree (to-nat fri-dg)) composition-table
    (to-nat fri-dg) opening-composition-roots composition-bs
    composition-final fri-query-idxs composition-round-layers doms layers
     $\wedge$  canonical-layers.  $\neg$ 
    generic-fri-canonical-sampled-layer-chain-evidence
    (composition-table-low-degree (to-nat fri-dg)) composition-table
    (to-nat fri-dg) opening-composition-roots composition-bs
    composition-final fri-query-idxs composition-round-layers
    canonical-layers
  proof  $-$ 
    have sampled:
      composition-fri-bad-with-sampled-layer-chain s
      (Some (result, final-state))
    and not-canonical:
       $\neg$  composition-fri-bad-with-canonical-sampled-layer-chain s
      (Some (result, final-state))
    using missing
    unfolding
      composition-fri-sampled-missing-canonical-sampled-layer-chain-def
    by blast+
  from sampled obtain trace-roots trace-bs trace-final fri-dg

```

opening-composition-roots composition-bs composition-final
fri-query-idxs trace-round-layers composition-round-layers fr
f-fri-roots f-final as dg composition-fri-roots final
trace-query-idxs trace-openings composition-query-idxs
composition-openings trace-table composition-table doms layers
where *evidence*:
composition-fri-partial-candidate-opening-evidence s
(Some (result, final-state)) trace-roots trace-bs trace-final fri-dg
opening-composition-roots composition-bs composition-final
fri-query-idxs trace-round-layers composition-round-layers fr
f-fri-roots f-final as dg composition-fri-roots final
trace-query-idxs trace-openings composition-query-idxs
composition-openings trace-table composition-table
and *chain*:
generic-fri-sampled-layer-chain-evidence
(composition-table-low-degree (to-nat fri-dg)) composition-table
(to-nat fri-dg) opening-composition-roots composition-bs
composition-final fri-query-idxs composition-round-layers doms layers
unfolding *composition-fri-bad-with-sampled-layer-chain-def* **by** *meson*
have *no-canonical*:
 $\bigwedge \text{canonical-layers. } \neg$
generic-fri-canonical-sampled-layer-chain-evidence
(composition-table-low-degree (to-nat fri-dg)) composition-table
(to-nat fri-dg) opening-composition-roots composition-bs
composition-final fri-query-idxs composition-round-layers
canonical-layers
proof
fix *canonical-layers*
assume *canonical*:
generic-fri-canonical-sampled-layer-chain-evidence
(composition-table-low-degree (to-nat fri-dg)) composition-table
(to-nat fri-dg) opening-composition-roots composition-bs
composition-final fri-query-idxs composition-round-layers
canonical-layers
have *composition-fri-bad-with-canonical-sampled-layer-chain s*
(Some (result, final-state))
unfolding *composition-fri-bad-with-canonical-sampled-layer-chain-def*
using *evidence canonical* **by** *meson*
then show *False*
using *not-canonical* **by** *contradiction*
qed
show *?thesis*
by *(rule that[OF evidence chain no-canonical])*
qed

lemma *fri-canonical-domain-successor-square*:
assumes *next-bound*:
idx < length (fri-canonical-domain-at (Suc layer-idx))
and *cur-bound*:

```

    idx < length (fri-canonical-domain-at layer-idx)
  shows fri-canonical-domain-at (Suc layer-idx) ! idx =
    (fri-canonical-domain-at layer-idx ! idx)2
proof -
  have next-val:
    fri-canonical-domain-at (Suc layer-idx) ! idx =
      (h ^ idx * shift) ^ (2 ^ Suc layer-idx)
  by (rule fri-canonical-domain-at-nth[OF next-bound])
  have cur-val:
    fri-canonical-domain-at layer-idx ! idx =
      (h ^ idx * shift) ^ (2 ^ layer-idx)
  by (rule fri-canonical-domain-at-nth[OF cur-bound])
  have square:
    (h ^ idx * shift) ^ (2 ^ Suc layer-idx) =
      ((h ^ idx * shift) ^ (2 ^ layer-idx))2
  by (metis mult.commute power2-eq-square power-Suc power-mult)
  show ?thesis
    using next-val cur-val square by simp
qed

```

```

lemma fri-canonical-domains-successor-square:
  assumes layer-bound: Suc layer-idx ≤ n
  and idx-bound:
    idx < length (fri-canonical-domains n ! Suc layer-idx)
  and cur-idx-bound:
    idx < length (fri-canonical-domains n ! layer-idx)
  shows fri-canonical-domains n ! Suc layer-idx ! idx =
    (fri-canonical-domains n ! layer-idx ! idx)2
proof -
  have cur-bound: layer-idx ≤ n
  using layer-bound by simp
  have next-dom:
    fri-canonical-domains n ! Suc layer-idx =
      fri-canonical-domain-at (Suc layer-idx)
  by (rule fri-canonical-domains-nth[OF layer-bound])
  have cur-dom:
    fri-canonical-domains n ! layer-idx =
      fri-canonical-domain-at layer-idx
  by (rule fri-canonical-domains-nth[OF cur-bound])
  show ?thesis
    using fri-canonical-domain-successor-square
      [of idx layer-idx] idx-bound cur-idx-bound
    unfolding next-dom cur-dom by simp
qed

```

```

lemma fri-canonical-domains-sampled-next-align:
  assumes roots-bound: length roots ≤ n
  and idx-bound:
    ∧ round-idx layer-idx.

```



```

    round-idx < length query-idxs  $\implies$ 
    layer-idx < length roots  $\implies$ 
    fri-evidence-next-idx roots query-idxs round-idx layer-idx <
    length (fri-canonical-domain-at layer-idx)
  shows
    fri-sampled-next-domains-align roots query-idxs
    (fri-canonical-domains n)
  unfolding fri-sampled-next-domains-align-def
proof (intro allI impI)
  fix round-idx layer-idx
  assume round-bound: round-idx < length query-idxs
  and layer-bound: layer-idx < length roots
  have layer-le: layer-idx  $\leq$  n
  using layer-bound roots-bound by simp
  have dom-eq:
    fri-canonical-domains n ! layer-idx =
    fri-canonical-domain-at layer-idx
  by (rule fri-canonical-domains-nth[OF layer-le])
  have idx:
    fri-evidence-next-idx roots query-idxs round-idx layer-idx <
    length (fri-canonical-domain-at layer-idx)
  by (rule idx-bound[OF round-bound layer-bound])
  show
    fri-canonical-domains n ! layer-idx !
    fri-evidence-next-idx roots query-idxs round-idx layer-idx =
    (h  $\wedge$  fri-evidence-next-idx roots query-idxs round-idx layer-idx *
    shift)  $\wedge$  (2  $\wedge$  layer-idx)
  unfolding dom-eq
  by (rule fri-canonical-domain-at-nth[OF idx])
qed

```

```

lemma fri-canonical-domains-sampled-successor-square:
  assumes roots-bound: length roots  $\leq$  n
  and cur-idx-bound:
     $\bigwedge$  round-idx layer-idx.
    round-idx < length query-idxs  $\implies$ 
    layer-idx < length roots  $\implies$ 
    fri-evidence-next-idx roots query-idxs round-idx layer-idx <
    length (fri-canonical-domains n ! layer-idx)
  and next-idx-bound:
     $\bigwedge$  round-idx layer-idx.
    round-idx < length query-idxs  $\implies$ 
    layer-idx < length roots  $\implies$ 
    fri-evidence-next-idx roots query-idxs round-idx layer-idx <
    length (fri-canonical-domains n ! Suc layer-idx)
  shows
    fri-sampled-successor-domains-square roots query-idxs
    (fri-canonical-domains n)
  unfolding fri-sampled-successor-domains-square-def

```

```

proof (intro allI impI)
  fix round-idx layer-idx
  assume round-bound: round-idx < length query-idxs
    and layer-bound: layer-idx < length roots
  have layer-le: Suc layer-idx ≤ n
    using layer-bound roots-bound by simp
  show
    fri-canonical-domains n ! Suc layer-idx !
    fri-evidence-next-idx roots query-idxs round-idx layer-idx =
      (fri-canonical-domains n ! layer-idx !
        fri-evidence-next-idx roots query-idxs round-idx layer-idx)2
  by (rule fri-canonical-domains-successor-square[OF layer-le])
    (rule next-idx-bound[OF round-bound layer-bound],
      rule cur-idx-bound[OF round-bound layer-bound])
qed

lemma fri-canonical-domains-no-sampled-domain-failure:
  assumes roots-bound: length roots ≤ n
    and canonical-next-bound:
      ∧round-idx layer-idx.
        round-idx < length query-idxs ⇒
        layer-idx < length roots ⇒
        fri-evidence-next-idx roots query-idxs round-idx layer-idx <
        length (fri-canonical-domain-at layer-idx)
    and canonical-successor-bound:
      ∧round-idx layer-idx.
        round-idx < length query-idxs ⇒
        layer-idx < length roots ⇒
        fri-evidence-next-idx roots query-idxs round-idx layer-idx <
        length (fri-canonical-domains n ! Suc layer-idx)
  shows ¬ generic-fri-full-cover-sampled-domain-failure roots query-idxs
    (fri-canonical-domains n)
proof –
  have next-align:
    fri-sampled-next-domains-align roots query-idxs
    (fri-canonical-domains n)
  by (rule fri-canonical-domains-sampled-next-align
    [OF roots-bound canonical-next-bound])
  have cur-bound:
    ∧round-idx layer-idx.
      round-idx < length query-idxs ⇒
      layer-idx < length roots ⇒
      fri-evidence-next-idx roots query-idxs round-idx layer-idx <
      length (fri-canonical-domains n ! layer-idx)
  proof –
    fix round-idx layer-idx
    assume round-bound: round-idx < length query-idxs
      and layer-bound: layer-idx < length roots
    have layer-le: layer-idx ≤ n

```

```

    using layer-bound roots-bound by simp
  have dom-eq:
    fri-canonical-domains n ! layer-idx =
      fri-canonical-domain-at layer-idx
    by (rule fri-canonical-domains-nth[OF layer-le])
  show
    fri-evidence-next-idx roots query-idxs round-idx layer-idx <
      length (fri-canonical-domains n ! layer-idx)
    unfolding dom-eq
    by (rule canonical-next-bound[OF round-bound layer-bound])
qed
have succ-square:
  fri-sampled-successor-domains-square roots query-idxs
    (fri-canonical-domains n)
  by (rule fri-canonical-domains-sampled-successor-square
    [OF roots-bound cur-bound canonical-successor-bound])
show ?thesis
  unfolding generic-fri-full-cover-sampled-domain-failure-def
  using next-align succ-square by simp
qed

lemma generic-fri-full-cover-domain-failure-imp-index-or-sampled-domain:
  assumes roots-len: length challenges = length roots
  and domain: generic-fri-full-cover-domain-failure roots challenges doms
  shows
    generic-fri-full-cover-index-failure roots query-idxs ∨
    generic-fri-full-cover-sampled-domain-failure roots query-idxs doms
proof (cases generic-fri-full-cover-index-failure roots query-idxs)
  case True
  then show ?thesis by simp
next
  case False
  have covers: fri-sampled-layers-cover-fold-indices roots query-idxs
    using False unfolding generic-fri-full-cover-index-failure-def by simp
  show ?thesis
  proof (cases
    generic-fri-full-cover-sampled-domain-failure roots query-idxs doms)
    case True
    then show ?thesis by simp
  next
    case no-sampled: False
    have next-align: fri-sampled-next-domains-align roots query-idxs doms
      and succ-square:
        fri-sampled-successor-domains-square roots query-idxs doms
      using no-sampled
      unfolding generic-fri-full-cover-sampled-domain-failure-def
      by simp-all
    have no-square-fail:
      ∀ layer-idx < length challenges.

```

```

     $\forall \text{idx} < \text{fri-evidence-layer-len roots layer-idx div } 2.$ 
     $\text{doms} ! \text{Suc layer-idx} ! \text{idx} =$ 
       $(\text{doms} ! \text{layer-idx} ! \text{idx})^2$ 
proof (intro allI impI)
  fix layer-idx idx
  assume layer-bound: layer-idx < length challenges
    and idx-bound: idx < fri-evidence-layer-len roots layer-idx div 2
  have layer-root-bound: layer-idx < length roots
    using layer-bound roots-len by simp
  obtain round-idx where round-bound: round-idx < length query-idxs
    and idx-eq:
      fri-evidence-next-idx roots query-idxs round-idx layer-idx = idx
    by (rule fri-sampled-layers-cover-fold-indicesD
      [OF covers layer-root-bound idx-bound])
  show  $\text{doms} ! \text{Suc layer-idx} ! \text{idx} =$ 
     $(\text{doms} ! \text{layer-idx} ! \text{idx})^2$ 
    using fri-sampled-successor-domains-squareD
      [OF succ-square round-bound layer-root-bound] idx-eq
    by simp
qed
have no-norm-fail:
   $\forall \text{layer-idx} < \text{length challenges}.$ 
   $\forall \text{idx} < \text{fri-evidence-layer-len roots layer-idx div } 2.$ 
     $\text{doms} ! \text{layer-idx} ! \text{idx} =$ 
       $(h \wedge \text{idx} * \text{shift}) \wedge (2 \wedge \text{layer-idx})$ 
proof (intro allI impI)
  fix layer-idx idx
  assume layer-bound: layer-idx < length challenges
    and idx-bound: idx < fri-evidence-layer-len roots layer-idx div 2
  have layer-root-bound: layer-idx < length roots
    using layer-bound roots-len by simp
  obtain round-idx where round-bound: round-idx < length query-idxs
    and idx-eq:
      fri-evidence-next-idx roots query-idxs round-idx layer-idx = idx
    by (rule fri-sampled-layers-cover-fold-indicesD
      [OF covers layer-root-bound idx-bound])
  show  $\text{doms} ! \text{layer-idx} ! \text{idx} =$ 
     $(h \wedge \text{idx} * \text{shift}) \wedge (2 \wedge \text{layer-idx})$ 
    using fri-sampled-next-domains-alignD
      [OF next-align round-bound layer-root-bound] idx-eq
    by simp
qed
have False
  using domain no-square-fail no-norm-fail
  unfolding generic-fri-full-cover-domain-failure-def by simp
then show ?thesis by simp
qed
qed

```

lemma *trace-fri-sampled-full-cover-domain-imp-index-or-sampled-domain:*
assumes *domain: trace-fri-sampled-full-cover-domain-failure s out*
shows
trace-fri-sampled-full-cover-index-failure s out $\vee$
trace-fri-sampled-full-cover-sampled-domain-failure s out
proof –
from *domain* **obtain** *trace-roots trace-bs trace-final dg composition-roots*
composition-bs composition-final fri-query-idxs trace-round-layers
composition-round-layers fr candidate-query-idxs trace-openings
trace-table doms layers
where *evidence:*
trace-fri-partial-candidate-opening-evidence s out trace-roots
trace-bs trace-final dg composition-roots composition-bs
composition-final fri-query-idxs trace-round-layers
composition-round-layers fr candidate-query-idxs trace-openings
trace-table
and *chain:*
generic-fri-sampled-layer-chain-evidence trace-table-low-degree
trace-table (clength – 1) trace-roots trace-bs trace-final
fri-query-idxs trace-round-layers doms layers
and *generic-domain:*
generic-fri-full-cover-domain-failure trace-roots trace-bs doms
unfolding *trace-fri-sampled-full-cover-domain-failure-def* **by** *blast*
have *partial:*
generic-fri-partial-evidence trace-table-low-degree trace-table
(clength – 1) trace-roots trace-bs trace-final fri-query-idxs
trace-round-layers
by *(rule generic-fri-sampled-layer-chain-evidenceD(1)[OF chain])*
have *roots-len: length trace-bs = length trace-roots*
by *(rule generic-fri-partial-evidence-shapes(2)[OF partial])*
have *split:*
generic-fri-full-cover-index-failure trace-roots fri-query-idxs $\vee$
generic-fri-full-cover-sampled-domain-failure trace-roots
fri-query-idxs doms
by *(rule generic-fri-full-cover-domain-failure-imp-index-or-sampled-domain*
[OF roots-len generic-domain])
then show *?thesis*
proof
assume *generic-fri-full-cover-index-failure trace-roots fri-query-idxs*
then show *?thesis*
unfolding *trace-fri-sampled-full-cover-index-failure-def*
using *evidence chain* **by** *blast*
next
assume
generic-fri-full-cover-sampled-domain-failure trace-roots
fri-query-idxs doms
then show *?thesis*
unfolding *trace-fri-sampled-full-cover-sampled-domain-failure-def*
using *evidence chain* **by** *blast*

qed
qed

lemma *composition-fri-sampled-full-cover-domain-imp-index-or-sampled-domain:*

assumes *domain: composition-fri-sampled-full-cover-domain-failure s out*

shows

composition-fri-sampled-full-cover-index-failure s out $\vee$

composition-fri-sampled-full-cover-sampled-domain-failure s out

proof —

from *domain* **obtain** *trace-roots trace-bs trace-final fri-dg*

opening-composition-roots composition-bs composition-final

fri-query-idxs trace-round-layers composition-round-layers fr

f-fri-roots f-final as dg composition-fri-roots final

trace-query-idxs trace-openings composition-query-idxs

composition-openings trace-table composition-table doms layers

where *evidence:*

composition-fri-partial-candidate-opening-evidence s out trace-roots

trace-bs trace-final fri-dg opening-composition-roots composition-bs

composition-final fri-query-idxs trace-round-layers

composition-round-layers fr f-fri-roots f-final as dg

composition-fri-roots final trace-query-idxs trace-openings

composition-query-idxs composition-openings trace-table

composition-table

and *chain:*

generic-fri-sampled-layer-chain-evidence

(composition-table-low-degree (to-nat fri-dg)) composition-table

(to-nat fri-dg) opening-composition-roots composition-bs

composition-final fri-query-idxs composition-round-layers doms layers

and *generic-domain:*

generic-fri-full-cover-domain-failure opening-composition-roots

composition-bs doms

unfolding *composition-fri-sampled-full-cover-domain-failure-def*

by *meson*

have *partial:*

generic-fri-partial-evidence

(composition-table-low-degree (to-nat fri-dg)) composition-table

(to-nat fri-dg) opening-composition-roots composition-bs

composition-final fri-query-idxs composition-round-layers

by *(rule generic-fri-sampled-layer-chain-evidenceD(1)[OF chain])*

have *roots-len: length composition-bs = length opening-composition-roots*

by *(rule generic-fri-partial-evidence-shapes(2)[OF partial])*

have *split:*

generic-fri-full-cover-index-failure opening-composition-roots

fri-query-idxs $\vee$

generic-fri-full-cover-sampled-domain-failure

opening-composition-roots fri-query-idxs doms

by *(rule generic-fri-full-cover-domain-failure-imp-index-or-sampled-domain*

[OF roots-len generic-domain])

then show *?thesis*

```

proof
  assume
    generic-fri-full-cover-index-failure opening-composition-roots
    fri-query-idxs
  then show ?thesis
    unfolding composition-fri-sampled-full-cover-index-failure-def
    using evidence chain by meson
next
  assume
    generic-fri-full-cover-sampled-domain-failure
    opening-composition-roots fri-query-idxs doms
  then show ?thesis
    unfolding composition-fri-sampled-full-cover-sampled-domain-failure-def
    using evidence chain by meson
qed
qed

lemma wp-trace-fri-sampled-full-cover-domain-bound-from-index-or-sampled-domain:
assumes index:
  wp-event verify-monad
  (trace-fri-sampled-full-cover-index-failure s) s ≤ I
and sampled-domain:
  wp-event verify-monad
  (trace-fri-sampled-full-cover-sampled-domain-failure s) s ≤ SD
shows
  wp-event verify-monad
  (trace-fri-sampled-full-cover-domain-failure s) s ≤ I + SD
proof –
  have wp-event verify-monad
    (trace-fri-sampled-full-cover-domain-failure s) s
  ≤ wp-event verify-monad
    (λout. trace-fri-sampled-full-cover-index-failure s out ∨
      trace-fri-sampled-full-cover-sampled-domain-failure s out) s
  by (rule wp-event-mono)
    (rule trace-fri-sampled-full-cover-domain-imp-index-or-sampled-domain)
  also have ... ≤ I + SD
  by (rule order-trans[OF wp-event-union-bound
    add-mono[OF index sampled-domain]])
  finally show ?thesis .
qed

lemma
  wp-composition-fri-sampled-full-cover-domain-bound-from-index-or-sampled-domain:
assumes index:
  wp-event verify-monad
  (composition-fri-sampled-full-cover-index-failure s) s ≤ I
and sampled-domain:
  wp-event verify-monad
  (composition-fri-sampled-full-cover-sampled-domain-failure s) s

```

$\leq SD$
shows
 $wp\text{-event verify-monad}$
 $(composition\text{-fri-sampled-full-cover-domain-failure } s) s \leq I + SD$
proof –
have $wp\text{-event verify-monad}$
 $(composition\text{-fri-sampled-full-cover-domain-failure } s) s$
 $\leq wp\text{-event verify-monad}$
 $(\lambda out. composition\text{-fri-sampled-full-cover-index-failure } s \text{ out } \vee$
 $composition\text{-fri-sampled-full-cover-sampled-domain-failure } s \text{ out}) s$
by $(rule\ wp\text{-event-mono})$
 $(rule$
 $composition\text{-fri-sampled-full-cover-domain-imp-index-or-sampled-domain})$
also have $\dots \leq I + SD$
by $(rule\ order\text{-trans}[OF\ wp\text{-event-union-bound}$
 $add\text{-mono}[OF\ index\ sampled\text{-domain}]])$
finally show $?thesis$.
qed

lemma $wp\text{-trace-fri-sampled-missing-full-cover-bound-from-domain-split}$:

assumes $proximity$:
 $wp\text{-event verify-monad}$
 $(trace\text{-fri-sampled-full-cover-proximity-failure } s \text{ bad}) s \leq P$
and $index$:
 $wp\text{-event verify-monad}$
 $(trace\text{-fri-sampled-full-cover-index-failure } s) s \leq I$
and $domain\text{-length}$:
 $wp\text{-event verify-monad}$
 $(trace\text{-fri-sampled-full-cover-domain-length-failure } s) s \leq DL$
and $sampling\text{-domain}$:
 $wp\text{-event verify-monad}$
 $(trace\text{-fri-sampled-full-cover-sampled-domain-failure } s) s \leq SD$
shows
 $wp\text{-event verify-monad}$
 $(trace\text{-fri-sampled-missing-full-cover } s \text{ bad}) s$
 $\leq P + I + (I + DL) + (I + SD)$
by $(rule\ wp\text{-trace-fri-sampled-missing-full-cover-bound-from-length-split}$
 $[OF\ proximity\ index\ domain\text{-length}$
 $wp\text{-trace-fri-sampled-full-cover-domain-bound-from-index-or-sampled-domain}])$
 $(rule\ index, rule\ sampling\text{-domain})$

lemma $wp\text{-composition-fri-sampled-missing-full-cover-bound-from-domain-split}$:

assumes $proximity$:
 $wp\text{-event verify-monad}$
 $(composition\text{-fri-sampled-full-cover-proximity-failure } s \text{ bad}) s \leq P$
and $index$:
 $wp\text{-event verify-monad}$
 $(composition\text{-fri-sampled-full-cover-index-failure } s) s \leq I$
and $domain\text{-length}$:


```

wp-event verify-monad
  (composition-fri-sampled-full-cover-domain-length-failure s) s ≤ DL
and sampled-domain:
wp-event verify-monad
  (composition-fri-sampled-full-cover-sampled-domain-failure s) s
  ≤ SD
shows
wp-event verify-monad
  (composition-fri-sampled-missing-full-cover s bad) s
  ≤ P + I + (I + DL) + (I + SD)
by (rule
  wp-composition-fri-sampled-missing-full-cover-bound-from-length-split
  [OF proximity index domain-length
  wp-composition-fri-sampled-full-cover-domain-bound-from-index-or-sampled-domain])
  (rule index, rule sampled-domain)

end

end

```

```

theory Soundness-FRI-Sampled-Obstructions
imports Soundness-FRI-Full-Cover-Domains
begin

```

Explicit obstruction events for the branch where an accepting verifier execution reaches the broad reachable FRI bad event, but no sampled FRI layer-chain witness exists.

These predicates are proof-routing devices only. They do not reconstruct complete FRI tables from sampled openings; they record that the verifier's partial FRI evidence cannot be organized into the sampled layer-chain object required by the generic low-degree reduction.

```

context soundness
begin

```

```

definition generic-fri-sampled-next-value-conflict
  :: 'f list ⇒ 'f list ⇒ nat list ⇒
    'f list list list ⇒ bool

```

where

```

generic-fri-sampled-next-value-conflict roots challenges query-idxs
  round-layers ⟷
  (∃ round-idx round-idx' layer-idx v v'.
    round-idx < length query-idxs ∧
    round-idx' < length query-idxs ∧
    layer-idx < length challenges ∧
    fri-evidence-next-idx roots query-idxs round-idx layer-idx =
    fri-evidence-next-idx roots query-idxs round-idx' layer-idx ∧
    generic-fri-round-forced-next-value roots challenges query-idxs
    round-layers round-idx layer-idx v ∧

```

generic-fri-round-forced-next-value roots challenges query-idxs
round-layers round-idx' layer-idx v' $\wedge$
v $\neq$ v')

definition *generic-fri-sampled-final-value-conflict*

:: 'f list $\Rightarrow$ 'f list $\Rightarrow$ 'f $\Rightarrow$ nat list $\Rightarrow$
'f list list list $\Rightarrow$ bool

where

generic-fri-sampled-final-value-conflict roots challenges final-value
query-idxs round-layers $\longleftrightarrow$
($\exists$ round-idx v.
0 < length challenges $\wedge$
round-idx < length query-idxs $\wedge$
generic-fri-round-forced-next-value roots challenges query-idxs
round-layers round-idx (length challenges - 1) v $\wedge$
v $\neq$ final-value)

definition *generic-fri-sampled-base-opening-conflict*

:: 'f list $\Rightarrow$ 'f list $\Rightarrow$ 'f list $\Rightarrow$ nat list $\Rightarrow$
'f list list list $\Rightarrow$ bool

where

generic-fri-sampled-base-opening-conflict candidate-table roots challenges
query-idxs round-layers $\longleftrightarrow$
($\exists$ round-idx xp xp-path xn xn-path.
0 < length challenges $\wedge$
round-idx < length query-idxs $\wedge$
fri-layer-step-evidence
(roots ! 0)
(challenges ! 0)
(fri-evidence-layer-len roots 0)
(fri-evidence-layer-idx roots query-idxs round-idx 0)
1
(fri-sibling-index (fri-evidence-layer-len roots 0)
(fri-evidence-layer-idx roots query-idxs round-idx 0))
xp xp-path xn xn-path
(fri-evidence-next-idx roots query-idxs round-idx 0)
(fri-evidence-next-value roots challenges query-idxs round-idx 0
xp xn)
(round-layers ! round-idx ! 0) $\wedge$
 $\neg$ fri-opening-matches-table (fri-evidence-layer-len roots 0)
(fri-evidence-layer-idx roots query-idxs round-idx 0)
candidate-table xp xn)

definition *generic-fri-sampled-successor-opening-conflict*

:: 'f list $\Rightarrow$ 'f list $\Rightarrow$ nat list $\Rightarrow$
'f list list list $\Rightarrow$ bool

where

generic-fri-sampled-successor-opening-conflict roots challenges query-idxs
round-layers $\longleftrightarrow$

$(\exists \text{ source-round target-round layer-idx } v \text{ } xp \text{ } xp\text{-path } xn \text{ } xn\text{-path}.$
 $\text{source-round} < \text{length query-idxs} \wedge$
 $\text{target-round} < \text{length query-idxs} \wedge$
 $\text{Suc layer-idx} < \text{length challenges} \wedge$
 $\text{generic-fri-round-forced-next-value roots challenges query-idxs}$
 $\text{round-layers source-round layer-idx } v \wedge$
 $\text{fri-layer-step-evidence}$
 $(\text{roots ! Suc layer-idx})$
 $(\text{challenges ! Suc layer-idx})$
 $(\text{fri-evidence-layer-len roots (Suc layer-idx)})$
 $(\text{fri-evidence-layer-idx roots query-idxs target-round}$
 $(\text{Suc layer-idx}))$
 $(2 \wedge \text{Suc layer-idx})$
 $(\text{fri-sibling-index (fri-evidence-layer-len roots (Suc layer-idx)})$
 $(\text{fri-evidence-layer-idx roots query-idxs target-round}$
 $(\text{Suc layer-idx})))$
 $xp \text{ } xp\text{-path } xn \text{ } xn\text{-path}$
 $(\text{fri-evidence-next-idx roots query-idxs target-round}$
 $(\text{Suc layer-idx}))$
 $(\text{fri-evidence-next-value roots challenges query-idxs target-round}$
 $(\text{Suc layer-idx}) \text{ } xp \text{ } xn)$
 $(\text{round-layers ! target-round ! Suc layer-idx}) \wedge$
 $((\text{fri-evidence-next-idx roots query-idxs source-round layer-idx} =$
 $\text{fri-evidence-layer-idx roots query-idxs target-round}$
 $(\text{Suc layer-idx}) \wedge$
 $v \neq xp) \vee$
 $(\text{fri-evidence-next-idx roots query-idxs source-round layer-idx} =$
 $\text{fri-sibling-index (fri-evidence-layer-len roots (Suc layer-idx)})$
 $(\text{fri-evidence-layer-idx roots query-idxs target-round}$
 $(\text{Suc layer-idx})) \wedge$
 $v \neq xn)))$

definition *generic-fri-sampled-same-layer-opening-conflict*

$:: 'f \text{ list} \Rightarrow 'f \text{ list} \Rightarrow \text{nat list} \Rightarrow$
 $'f \text{ list list list} \Rightarrow \text{bool}$

where

$\text{generic-fri-sampled-same-layer-opening-conflict roots challenges}$
 $\text{query-idxs round-layers} \longleftrightarrow$
 $(\exists \text{ round-idx round-idx' layer-idx}$
 $xp \text{ } xp\text{-path } xn \text{ } xn\text{-path } yp \text{ } yp\text{-path } yn \text{ } yn\text{-path}.$
 $\text{round-idx} < \text{length query-idxs} \wedge$
 $\text{round-idx'} < \text{length query-idxs} \wedge$
 $\text{layer-idx} < \text{length challenges} \wedge$
 $\text{fri-layer-step-evidence}$
 $(\text{roots ! layer-idx})$
 $(\text{challenges ! layer-idx})$
 $(\text{fri-evidence-layer-len roots layer-idx})$
 $(\text{fri-evidence-layer-idx roots query-idxs round-idx layer-idx})$
 $(2 \wedge \text{layer-idx})$

(fri-sibling-index (fri-evidence-layer-len roots layer-idx)
 (fri-evidence-layer-idx roots query-idxs round-idx layer-idx))
 xp xp-path xn xn-path
 (fri-evidence-next-idx roots query-idxs round-idx layer-idx)
 (fri-evidence-next-value roots challenges query-idxs round-idx
 layer-idx xp xn)
 (round-layers ! round-idx ! layer-idx) ∧
 fri-layer-step-evidence
 (roots ! layer-idx)
 (challenges ! layer-idx)
 (fri-evidence-layer-len roots layer-idx)
 (fri-evidence-layer-idx roots query-idxs round-idx' layer-idx)
 (2 ^ layer-idx)
 (fri-sibling-index (fri-evidence-layer-len roots layer-idx)
 (fri-evidence-layer-idx roots query-idxs round-idx' layer-idx))
 yp yp-path yn yn-path
 (fri-evidence-next-idx roots query-idxs round-idx' layer-idx)
 (fri-evidence-next-value roots challenges query-idxs round-idx'
 layer-idx yp yn)
 (round-layers ! round-idx' ! layer-idx) ∧
 ((fri-evidence-layer-idx roots query-idxs round-idx layer-idx =
 fri-evidence-layer-idx roots query-idxs round-idx' layer-idx ∧
 xp ≠ yp) ∨
 (fri-evidence-layer-idx roots query-idxs round-idx layer-idx =
 fri-sibling-index (fri-evidence-layer-len roots layer-idx)
 (fri-evidence-layer-idx roots query-idxs round-idx' layer-idx) ∧
 xp ≠ yn) ∨
 (fri-sibling-index (fri-evidence-layer-len roots layer-idx)
 (fri-evidence-layer-idx roots query-idxs round-idx layer-idx) =
 fri-evidence-layer-idx roots query-idxs round-idx' layer-idx ∧
 xn ≠ yp) ∨
 (fri-sibling-index (fri-evidence-layer-len roots layer-idx)
 (fri-evidence-layer-idx roots query-idxs round-idx layer-idx) =
 fri-sibling-index (fri-evidence-layer-len roots layer-idx)
 (fri-evidence-layer-idx roots query-idxs round-idx' layer-idx) ∧
 xn ≠ yn)))

lemma generic-fri-sampled-next-value-conflictE:

assumes

generic-fri-sampled-next-value-conflict roots challenges query-idxs
 round-layers

obtains round-idx round-idx' layer-idx v v'

where

round-idx < length query-idxs

round-idx' < length query-idxs

layer-idx < length challenges

fri-evidence-next-idx roots query-idxs round-idx layer-idx =

fri-evidence-next-idx roots query-idxs round-idx' layer-idx

generic-fri-round-forced-next-value roots challenges query-idxs

round-layers round-idx layer-idx v
generic-fri-round-forced-next-value roots challenges query-idxs
round-layers round-idx' layer-idx v'
v ≠ v'
using *assms* **unfolding** *generic-fri-sampled-next-value-conflict-def*
by *blast*

lemma *generic-fri-sampled-final-value-conflictE*:

assumes
generic-fri-sampled-final-value-conflict roots challenges final-value
query-idxs round-layers
obtains *round-idx v*
where
0 < length challenges
round-idx < length query-idxs
generic-fri-round-forced-next-value roots challenges query-idxs
round-layers round-idx (length challenges - 1) v
v ≠ final-value
using *assms* **unfolding** *generic-fri-sampled-final-value-conflict-def*
by *blast*

lemma *generic-fri-sampled-successor-opening-conflictE*:

assumes
generic-fri-sampled-successor-opening-conflict roots challenges
query-idxs round-layers
obtains *source-round target-round layer-idx v xp xp-path xn xn-path*
where
source-round < length query-idxs
target-round < length query-idxs
Suc layer-idx < length challenges
generic-fri-round-forced-next-value roots challenges query-idxs
round-layers source-round layer-idx v
fri-layer-step-evidence
(roots ! Suc layer-idx)
(challenges ! Suc layer-idx)
(fri-evidence-layer-len roots (Suc layer-idx))
(fri-evidence-layer-idx roots query-idxs target-round
(Suc layer-idx))
(2 ^ Suc layer-idx)
(fri-sibling-index (fri-evidence-layer-len roots (Suc layer-idx))
(fri-evidence-layer-idx roots query-idxs target-round
(Suc layer-idx)))
xp xp-path xn xn-path
(fri-evidence-next-idx roots query-idxs target-round
(Suc layer-idx))
(fri-evidence-next-value roots challenges query-idxs target-round
(Suc layer-idx) xp xn)
(round-layers ! target-round ! Suc layer-idx)
((fri-evidence-next-idx roots query-idxs source-round layer-idx =

$\text{fri-evidence-layer-idx roots query-idxs target-round}$
 $(\text{Suc layer-idx}) \wedge$
 $v \neq xp) \vee$
 $(\text{fri-evidence-next-idx roots query-idxs source-round layer-idx} =$
 $\text{fri-sibling-index (fri-evidence-layer-len roots (Suc layer-idx))}$
 $(\text{fri-evidence-layer-idx roots query-idxs target-round}$
 $(\text{Suc layer-idx})) \wedge$
 $v \neq xn))$
using *assms unfolding generic-fri-sampled-successor-opening-conflict-def*
by *blast*

definition *generic-fri-sampled-layer-assignment-obstruction*

$:: ('f \text{ list} \Rightarrow \text{bool}) \Rightarrow 'f \text{ list} \Rightarrow \text{nat} \Rightarrow$
 $'f \text{ list} \Rightarrow 'f \text{ list} \Rightarrow 'f \Rightarrow \text{nat list} \Rightarrow$
 $'f \text{ list list list} \Rightarrow \text{bool}$

where

$\text{generic-fri-sampled-layer-assignment-obstruction low-degree}$
 $\text{candidate-table degree-bound roots challenges final-value query-idxs}$
 $\text{round-layers} \longleftrightarrow$
 $\text{generic-fri-partial-evidence low-degree candidate-table degree-bound}$
 $\text{roots challenges final-value query-idxs round-layers} \wedge$
 $(\forall \text{ doms layers. } \neg$
 $\text{generic-fri-sampled-layer-chain-evidence low-degree candidate-table}$
 $\text{degree-bound roots challenges final-value query-idxs round-layers}$
 $\text{doms layers})$

definition *generic-fri-sampled-assignment-conflict*

$:: 'f \text{ list} \Rightarrow 'f \text{ list} \Rightarrow 'f \text{ list} \Rightarrow 'f \Rightarrow$
 $\text{nat list} \Rightarrow 'f \text{ list list list} \Rightarrow \text{bool}$

where

$\text{generic-fri-sampled-assignment-conflict candidate-table roots challenges}$
 $\text{final-value query-idxs round-layers} \longleftrightarrow$
 $\text{generic-fri-sampled-base-opening-conflict candidate-table roots}$
 $\text{challenges query-idxs round-layers} \vee$
 $\text{generic-fri-sampled-next-value-conflict roots challenges query-idxs}$
 $\text{round-layers} \vee$
 $\text{generic-fri-sampled-successor-opening-conflict roots challenges}$
 $\text{query-idxs round-layers} \vee$
 $\text{generic-fri-sampled-same-layer-opening-conflict roots challenges}$
 $\text{query-idxs round-layers} \vee$
 $\text{generic-fri-sampled-final-value-conflict roots challenges final-value}$
 $\text{query-idxs round-layers}$

definition *generic-fri-sampled-layer-value-constraint*

$:: 'f \text{ list} \Rightarrow 'f \text{ list} \Rightarrow 'f \text{ list} \Rightarrow 'f \Rightarrow$
 $\text{nat list} \Rightarrow 'f \text{ list list list} \Rightarrow \text{nat} \Rightarrow \text{nat} \Rightarrow$
 $'f \Rightarrow \text{bool}$

where

$\text{generic-fri-sampled-layer-value-constraint candidate-table roots}$

$\longleftrightarrow$
challenges final-value query-idxs round-layers layer-idx idx v
 (if *layer-idx = 0* then
 idx < length candidate-table $\wedge$ *v = candidate-table ! idx*
 else if *layer-idx = length challenges* then
 v = final-value
 else
 ($\exists$ *round-idx*.
 round-idx < length query-idxs $\wedge$
 generic-fri-round-forced-next-value roots challenges query-idxs
 round-layers round-idx (layer-idx - 1) v $\wedge$
 fri-evidence-next-idx roots query-idxs round-idx
 (layer-idx - 1) = idx) $\vee$
 ($\exists$ *round-idx xp xp-path xn xn-path*.
 round-idx < length query-idxs $\wedge$
 layer-idx < length challenges $\wedge$
 fri-layer-step-evidence
 (*roots ! layer-idx*)
 (*challenges ! layer-idx*)
 (*fri-evidence-layer-len roots layer-idx*)
 (*fri-evidence-layer-idx roots query-idxs round-idx layer-idx*)
 ($2 \wedge \text{layer-idx}$)
 (*fri-sibling-index (fri-evidence-layer-len roots layer-idx)*
 (*fri-evidence-layer-idx roots query-idxs round-idx layer-idx*))
 xp xp-path xn xn-path
 (*fri-evidence-next-idx roots query-idxs round-idx layer-idx*)
 (*fri-evidence-next-value roots challenges query-idxs round-idx*
 layer-idx xp xn)
 (*round-layers ! round-idx ! layer-idx*) $\wedge$
 ((*idx = fri-evidence-layer-idx roots query-idxs round-idx*
 layer-idx $\wedge$ *v = xp*) $\vee$
 (*idx = fri-sibling-index (fri-evidence-layer-len roots layer-idx)*
 (*fri-evidence-layer-idx roots query-idxs round-idx layer-idx*) $\wedge$
 v = xn))))))

definition *generic-fri-sampled-layer-value*

$:: 'f \text{ list} \Rightarrow 'f \text{ list} \Rightarrow 'f \text{ list} \Rightarrow 'f \Rightarrow$
 $\text{nat list} \Rightarrow 'f \text{ list list list} \Rightarrow \text{nat} \Rightarrow \text{nat} \Rightarrow 'f$

where

generic-fri-sampled-layer-value candidate-table roots challenges
final-value query-idxs round-layers layer-idx idx =
 (if $\exists v$. *generic-fri-sampled-layer-value-constraint candidate-table*
 roots challenges final-value query-idxs round-layers layer-idx idx v
 then *SOME v. generic-fri-sampled-layer-value-constraint*
 candidate-table roots challenges final-value query-idxs round-layers
 layer-idx idx v
 else *final-value*)

definition *generic-fri-sampled-assignment-layer*

```

:: 'f list ⇒ 'f list ⇒ 'f list ⇒ 'f ⇒
   nat list ⇒ 'f list list list ⇒ nat ⇒ 'f list
where
  generic-fri-sampled-assignment-layer candidate-table roots challenges
    final-value query-idxs round-layers layer-idx =
    (if layer-idx = 0 then candidate-table
     else map (generic-fri-sampled-layer-value candidate-table roots
        challenges final-value query-idxs round-layers layer-idx)
      [0..definition generic-fri-sampled-assignment-layers
:: 'f list ⇒ 'f list ⇒ 'f list ⇒ 'f ⇒
   nat list ⇒ 'f list list list ⇒ 'f list list
where
  generic-fri-sampled-assignment-layers candidate-table roots challenges
    final-value query-idxs round-layers =
    map (generic-fri-sampled-assignment-layer candidate-table roots
      challenges final-value query-idxs round-layers)
      [0..lemma generic-fri-sampled-assignment-layer-length-nonzero[simp]:
  assumes layer-idx ≠ 0
  shows length (generic-fri-sampled-assignment-layer candidate-table roots
    challenges final-value query-idxs round-layers layer-idx) =
    clength * scale
  using assms unfolding generic-fri-sampled-assignment-layer-def by simp

lemma generic-fri-sampled-assignment-layer-zero[simp]:
  generic-fri-sampled-assignment-layer candidate-table roots challenges
    final-value query-idxs round-layers 0 = candidate-table
  unfolding generic-fri-sampled-assignment-layer-def by simp

lemma generic-fri-sampled-assignment-layers-length[simp]:
  length (generic-fri-sampled-assignment-layers candidate-table roots
    challenges final-value query-idxs round-layers) =
    Suc (length challenges)
  unfolding generic-fri-sampled-assignment-layers-def by simp

lemma generic-fri-sampled-assignment-layers-nth:
  assumes layer-idx ≤ length challenges
  shows
    generic-fri-sampled-assignment-layers candidate-table roots challenges
      final-value query-idxs round-layers ! layer-idx =
      generic-fri-sampled-assignment-layer candidate-table roots challenges
        final-value query-idxs round-layers layer-idx
proof (cases layer-idx < length challenges)
  case True
  then show ?thesis
    unfolding generic-fri-sampled-assignment-layers-def

```



```

    by (simp add: nth-append)
next
case False
then have layer-eq: layer-idx = length challenges
    using assms by simp
then show ?thesis
    unfolding generic-fri-sampled-assignment-layers-def
    by (simp add: nth-append)
qed

lemma generic-fri-sampled-assignment-layers-base[simp]:
  generic-fri-sampled-assignment-layers candidate-table roots challenges
    final-value query-idxs round-layers ! 0 = candidate-table
  using generic-fri-sampled-assignment-layers-nth[of 0 challenges
    candidate-table roots final-value query-idxs round-layers]
  by simp

lemma generic-fri-sampled-layer-value-unique:
  assumes constraint:
    generic-fri-sampled-layer-value-constraint candidate-table roots
      challenges final-value query-idxs round-layers layer-idx idx v
  and unique:
     $\bigwedge v'. \text{generic-fri-sampled-layer-value-constraint candidate-table}$ 
       $\text{roots challenges final-value query-idxs round-layers layer-idx idx } v'$ 
       $\implies v' = v$ 
  shows
    generic-fri-sampled-layer-value candidate-table roots challenges
      final-value query-idxs round-layers layer-idx idx = v
proof -
  have ex:  $\exists v. \text{generic-fri-sampled-layer-value-constraint}$ 
    candidate-table roots challenges final-value query-idxs round-layers
    layer-idx idx v
  using constraint by blast
  have some-eq:
    ( $\text{SOME } v. \text{generic-fri-sampled-layer-value-constraint candidate-table}$ 
       $\text{roots challenges final-value query-idxs round-layers layer-idx idx } v$ ) =
      v
  by (rule some-equality) (use constraint unique in blast)+
  show ?thesis
    unfolding generic-fri-sampled-layer-value-def
    using ex some-eq by simp
qed

lemma generic-fri-sampled-assignment-layer-nth-unique:
  assumes idx-bound:  $\text{idx} < \text{clength} * \text{scale}$ 
  and layer-nonzero:  $\text{layer-idx} \neq 0$ 
  and constraint:
    generic-fri-sampled-layer-value-constraint candidate-table roots
      challenges final-value query-idxs round-layers layer-idx idx v

```

and *unique*:
 $\bigwedge v'. \text{generic-fri-sampled-layer-value-constraint candidate-table}$
 $\text{roots challenges final-value query-idxs round-layers layer-idx idx } v'$
 $\implies v' = v$
shows
 $\text{generic-fri-sampled-assignment-layer candidate-table roots challenges}$
 $\text{final-value query-idxs round-layers layer-idx ! idx} = v$
unfolding *generic-fri-sampled-assignment-layer-def*
using *idx-bound layer-nonzero*
by (*simp add: generic-fri-sampled-layer-value-unique[OF constraint unique]*)

lemma *generic-fri-sampled-final-assignment-layer-consistent*:
assumes *challenges-nonempty*: $0 < \text{length challenges}$
shows
 $\text{fri-final-constant-consistent}$
 $(\text{generic-fri-sampled-assignment-layers candidate-table roots challenges}$
 $\text{final-value query-idxs round-layers ! length challenges})$
 final-value
proof –
let *?layer* =
 $\text{generic-fri-sampled-assignment-layers candidate-table roots challenges}$
 $\text{final-value query-idxs round-layers ! length challenges}$
have *layer-eq*:
 $?layer =$
 $\text{generic-fri-sampled-assignment-layer candidate-table roots challenges}$
 $\text{final-value query-idxs round-layers (length challenges)}$
by (*rule generic-fri-sampled-assignment-layers-nth*) *simp*
have *layer-len*: $\text{length } ?layer = \text{clength} * \text{scale}$
using *challenges-nonempty layer-eq* **by** *simp*
have *nth-final*:
 $\bigwedge \text{idx. idx} < \text{length } ?layer \implies ?layer ! \text{idx} = \text{final-value}$
proof –
fix *idx*
assume *idx-bound-layer*: $\text{idx} < \text{length } ?layer$
then have *idx-bound*: $\text{idx} < \text{clength} * \text{scale}$
using *layer-len* **by** *simp*
have *constraint*:
 $\text{generic-fri-sampled-layer-value-constraint candidate-table roots}$
 $\text{challenges final-value query-idxs round-layers}$
 $(\text{length challenges}) \text{ idx final-value}$
using *challenges-nonempty*
unfolding *generic-fri-sampled-layer-value-constraint-def* **by** *simp*
have *unique*:
 $\bigwedge v'. \text{generic-fri-sampled-layer-value-constraint candidate-table}$
 $\text{roots challenges final-value query-idxs round-layers}$
 $(\text{length challenges}) \text{ idx } v' \implies v' = \text{final-value}$
using *challenges-nonempty*
unfolding *generic-fri-sampled-layer-value-constraint-def* **by** *simp*
show $?layer ! \text{idx} = \text{final-value}$

```

unfolding layer-eq
by (rule generic-fri-sampled-assignment-layer-nth-unique
      [OF idx-bound - constraint unique])
      (use challenges-nonempty in simp)
qed
show ?thesis
unfolding fri-final-constant-consistent-def
using nth-final by (auto simp: in-set-conv-nth)
qed

definition trace-fri-sampled-layer-assignment-obstruction
  :: ('f, 'a) protocol-channel-scheme  $\Rightarrow$ 
    (unit list  $\times$  ('f, 'a) protocol-channel-scheme) option  $\Rightarrow$  bool
where
  trace-fri-sampled-layer-assignment-obstruction s out  $\longleftrightarrow$ 
    ( $\exists$  trace-roots trace-bs trace-final dg composition-roots
      composition-bs composition-final fri-query-idxs trace-round-layers
      composition-round-layers fr candidate-query-idxs trace-openings
      trace-table.
    trace-fri-partial-candidate-opening-evidence s out trace-roots
      trace-bs trace-final dg composition-roots composition-bs
      composition-final fri-query-idxs trace-round-layers
      composition-round-layers fr candidate-query-idxs trace-openings
      trace-table  $\wedge$ 
    generic-fri-sampled-layer-assignment-obstruction
      trace-table-low-degree trace-table (clength - 1) trace-roots
      trace-bs trace-final fri-query-idxs trace-round-layers)

definition composition-fri-sampled-layer-assignment-obstruction
  :: ('f, 'a) protocol-channel-scheme  $\Rightarrow$ 
    (unit list  $\times$  ('f, 'a) protocol-channel-scheme) option  $\Rightarrow$  bool
where
  composition-fri-sampled-layer-assignment-obstruction s out
     $\longleftrightarrow$ 
    ( $\exists$  trace-roots trace-bs trace-final fri-dg
      opening-composition-roots composition-bs composition-final
      fri-query-idxs trace-round-layers composition-round-layers fr
      f-fri-roots f-final as dg composition-fri-roots final
      trace-query-idxs trace-openings composition-query-idxs
      composition-openings trace-table composition-table.
    composition-fri-partial-candidate-opening-evidence s out trace-roots
      trace-bs trace-final fri-dg opening-composition-roots composition-bs
      composition-final fri-query-idxs trace-round-layers
      composition-round-layers fr f-fri-roots f-final as dg
      composition-fri-roots final trace-query-idxs trace-openings
      composition-query-idxs composition-openings trace-table
      composition-table  $\wedge$ 
    generic-fri-sampled-layer-assignment-obstruction
      (composition-table-low-degree (to-nat fri-dg)) composition-table

```

(to-nat fri-dg) opening-composition-roots composition-bs
composition-final fri-query-idxs composition-round-layers)

definition *trace-fri-sampled-assignment-conflict*

:: ('f, 'a) protocol-channel-scheme $\Rightarrow$
(unit list $\times$ ('f, 'a) protocol-channel-scheme) option $\Rightarrow$ bool

where

trace-fri-sampled-assignment-conflict s out $\longleftrightarrow$
($\exists$ trace-roots trace-bs trace-final dg composition-roots
composition-bs composition-final fri-query-idxs trace-round-layers
composition-round-layers fr candidate-query-idxs trace-openings
trace-table.
trace-fri-partial-candidate-opening-evidence s out trace-roots
trace-bs trace-final dg composition-roots composition-bs
composition-final fri-query-idxs trace-round-layers
composition-round-layers fr candidate-query-idxs trace-openings
trace-table $\wedge$
generic-fri-sampled-assignment-conflict trace-table trace-roots
trace-bs trace-final fri-query-idxs trace-round-layers)

definition *composition-fri-sampled-assignment-conflict*

:: ('f, 'a) protocol-channel-scheme $\Rightarrow$
(unit list $\times$ ('f, 'a) protocol-channel-scheme) option $\Rightarrow$ bool

where

composition-fri-sampled-assignment-conflict s out $\longleftrightarrow$
($\exists$ trace-roots trace-bs trace-final fri-dg
opening-composition-roots composition-bs composition-final
fri-query-idxs trace-round-layers composition-round-layers fr
f-fri-roots f-final as dg composition-fri-roots final
trace-query-idxs trace-openings composition-query-idxs
composition-openings trace-table composition-table.
composition-fri-partial-candidate-opening-evidence s out trace-roots
trace-bs trace-final fri-dg opening-composition-roots composition-bs
composition-final fri-query-idxs trace-round-layers
composition-round-layers fr f-fri-roots f-final as dg
composition-fri-roots final trace-query-idxs trace-openings
composition-query-idxs composition-openings trace-table
composition-table $\wedge$
generic-fri-sampled-assignment-conflict composition-table
opening-composition-roots composition-bs composition-final
fri-query-idxs composition-round-layers)

lemma *generic-fri-sampled-layer-assignment-obstructionD:*

assumes

generic-fri-sampled-layer-assignment-obstruction low-degree
candidate-table degree-bound roots challenges final-value query-idxs
round-layers

shows

generic-fri-partial-evidence low-degree candidate-table degree-bound

roots challenges final-value query-idxs round-layers
 $\wedge$ doms layers. $\neg$
generic-fri-sampled-layer-chain-evidence low-degree candidate-table
degree-bound roots challenges final-value query-idxs round-layers
doms layers
using *assms*
unfolding *generic-fri-sampled-layer-assignment-obstruction-def*
by *simp-all*

lemma *trace-fri-sampled-layer-assignment-obstructionE*:

assumes

trace-fri-sampled-layer-assignment-obstruction s out

obtains *trace-roots trace-bs trace-final dg composition-roots*

composition-bs composition-final fri-query-idxs trace-round-layers

composition-round-layers fr candidate-query-idxs trace-openings

trace-table

where

trace-fri-partial-candidate-opening-evidence s out trace-roots

trace-bs trace-final dg composition-roots composition-bs

composition-final fri-query-idxs trace-round-layers

composition-round-layers fr candidate-query-idxs trace-openings

trace-table

generic-fri-sampled-layer-assignment-obstruction

trace-table-low-degree trace-table (clength - 1) trace-roots

trace-bs trace-final fri-query-idxs trace-round-layers

using *assms*

unfolding *trace-fri-sampled-layer-assignment-obstruction-def*

apply (*elim exE conjE*)

apply (*rule that*)

apply *assumption*

apply *assumption*

done

lemma *composition-fri-sampled-layer-assignment-obstructionE*:

assumes

composition-fri-sampled-layer-assignment-obstruction s out

obtains *trace-roots trace-bs trace-final fri-dg*

opening-composition-roots composition-bs composition-final

fri-query-idxs trace-round-layers composition-round-layers fr

f-fri-roots f-final as dg composition-fri-roots final

trace-query-idxs trace-openings composition-query-idxs

composition-openings trace-table composition-table

where

composition-fri-partial-candidate-opening-evidence s out trace-roots

trace-bs trace-final fri-dg opening-composition-roots composition-bs

composition-final fri-query-idxs trace-round-layers

composition-round-layers fr f-fri-roots f-final as dg

composition-fri-roots final trace-query-idxs trace-openings

composition-query-idxs composition-openings trace-table

```

    composition-table
generic-fri-sampled-layer-assignment-obstruction
  (composition-table-low-degree (to-nat fri-dg)) composition-table
  (to-nat fri-dg) opening-composition-roots composition-bs
  composition-final fri-query-idxs composition-round-layers
using assms
unfolding composition-fri-sampled-layer-assignment-obstruction-def
apply (elim exE conjE)
apply (rule that)
  apply assumption
apply assumption
done

```

lemma *generic-fri-no-next-value-conflict-unique*:

assumes *no-conflict*:

- $\neg$ *generic-fri-sampled-next-value-conflict* *roots* *challenges*
query-idxs *round-layers*
- and** *round-bound*: *round-idx* < length *query-idxs*
- and** *round-bound'*: *round-idx'* < length *query-idxs*
- and** *layer-bound*: *layer-idx* < length *challenges*
- and** *same-idx*:
fri-evidence-next-idx *roots* *query-idxs* *round-idx* *layer-idx* =
fri-evidence-next-idx *roots* *query-idxs* *round-idx'* *layer-idx*
- and** *forced*:
generic-fri-round-forced-next-value *roots* *challenges* *query-idxs*
round-layers *round-idx* *layer-idx* *v*
- and** *forced'*:
generic-fri-round-forced-next-value *roots* *challenges* *query-idxs*
round-layers *round-idx'* *layer-idx* *v'*

shows *v* = *v'*

using *assms*

unfolding *generic-fri-sampled-next-value-conflict-def*

by *blast*

lemma *generic-fri-no-final-value-conflict-forced-eq*:

assumes *no-conflict*:

- $\neg$ *generic-fri-sampled-final-value-conflict* *roots* *challenges*
final-value *query-idxs* *round-layers*
- and** *challenges-nonempty*: 0 < length *challenges*
- and** *round-bound*: *round-idx* < length *query-idxs*
- and** *forced*:
generic-fri-round-forced-next-value *roots* *challenges* *query-idxs*
round-layers *round-idx* (length *challenges* - 1) *v*

shows *v* = *final-value*

using *assms*

unfolding *generic-fri-sampled-final-value-conflict-def*

by *blast*

lemma *generic-fri-no-assignment-conflict-no-final*:

assumes
 $\neg$ *generic-fri-sampled-assignment-conflict candidate-table roots*
challenges final-value query-idxs round-layers
shows
 $\neg$ *generic-fri-sampled-final-value-conflict roots challenges*
final-value query-idxs round-layers
using *assms unfolding generic-fri-sampled-assignment-conflict-def* **by** *simp*

lemma *generic-fri-no-assignment-conflict-no-next:*

assumes
 $\neg$ *generic-fri-sampled-assignment-conflict candidate-table roots*
challenges final-value query-idxs round-layers
shows
 $\neg$ *generic-fri-sampled-next-value-conflict roots challenges*
query-idxs round-layers
using *assms unfolding generic-fri-sampled-assignment-conflict-def* **by** *simp*

lemma *generic-fri-sampled-final-layer-forced-value:*

assumes *no-conflict:*
 $\neg$ *generic-fri-sampled-assignment-conflict candidate-table roots*
challenges final-value query-idxs round-layers
and *challenges-nonempty: 0 < length challenges*
and *round-bound: round-idx < length query-idxs*
and *idx-bound:*
fri-evidence-next-idx roots query-idxs round-idx
 $(\text{length challenges} - 1) < \text{clength} * \text{scale}$
and *forced:*
generic-fri-round-forced-next-value roots challenges query-idxs
round-layers round-idx (length challenges - 1) v
shows
generic-fri-sampled-assignment-layers candidate-table roots challenges
final-value query-idxs round-layers ! length challenges !
fri-evidence-next-idx roots query-idxs round-idx
 $(\text{length challenges} - 1) = v$

proof –

have *v-final: v = final-value*
by (rule *generic-fri-no-final-value-conflict-forced-eq*
 $[OF \text{ generic-fri-no-assignment-conflict-no-final } [OF \text{ no-conflict}]$
challenges-nonempty round-bound forced])
let *?idx =*
fri-evidence-next-idx roots query-idxs round-idx
 $(\text{length challenges} - 1)$
have *layer-eq:*
generic-fri-sampled-assignment-layers candidate-table roots challenges
final-value query-idxs round-layers ! length challenges =
generic-fri-sampled-assignment-layer candidate-table roots challenges
final-value query-idxs round-layers (length challenges)
by (rule *generic-fri-sampled-assignment-layers-nth*) *simp*
have *constraint:*

```

generic-fri-sampled-layer-value-constraint candidate-table roots
  challenges final-value query-idxs round-layers
  (length challenges) ?idx final-value
using challenges-nonempty
unfolding generic-fri-sampled-layer-value-constraint-def by simp
have unique:
 $\wedge v'. \text{generic-fri-sampled-layer-value-constraint candidate-table roots}$ 
  challenges final-value query-idxs round-layers
  (length challenges) ?idx  $v' \implies v' = \text{final-value}$ 
using challenges-nonempty
unfolding generic-fri-sampled-layer-value-constraint-def by simp
have nth-final:
generic-fri-sampled-assignment-layer candidate-table roots challenges
  final-value query-idxs round-layers (length challenges) ! ?idx =
  final-value
by (rule generic-fri-sampled-assignment-layer-nth-unique
  [OF idx-bound - constraint unique])
  (use challenges-nonempty in simp)
show ?thesis
using nth-final v-final unfolding layer-eq by simp
qed

```

```

lemma generic-fri-no-base-opening-conflict-matches-step:
assumes no-conflict:
 $\neg$  generic-fri-sampled-base-opening-conflict candidate-table roots
  challenges query-idxs round-layers
and challenges-nonempty:  $0 < \text{length challenges}$ 
and round-bound: round-idx < length query-idxs
and step:
  fri-layer-step-evidence
  (roots ! 0)
  (challenges ! 0)
  (fri-evidence-layer-len roots 0)
  (fri-evidence-layer-idx roots query-idxs round-idx 0)
  1
  (fri-sibling-index (fri-evidence-layer-len roots 0)
    (fri-evidence-layer-idx roots query-idxs round-idx 0))
  xp xp-path xn xn-path
  (fri-evidence-next-idx roots query-idxs round-idx 0)
  (fri-evidence-next-value roots challenges query-idxs round-idx 0
    xp xn)
  (round-layers ! round-idx ! 0)
shows
  fri-opening-matches-table (fri-evidence-layer-len roots 0)
  (fri-evidence-layer-idx roots query-idxs round-idx 0)
  candidate-table xp xn
using assms
unfolding generic-fri-sampled-base-opening-conflict-def
by blast

```


lemma *generic-fri-no-assignment-conflict-no-base*:
assumes
 $\neg$ *generic-fri-sampled-assignment-conflict candidate-table roots*
challenges final-value query-idxs round-layers
shows
 $\neg$ *generic-fri-sampled-base-opening-conflict candidate-table roots*
challenges query-idxs round-layers
using *assms unfolding generic-fri-sampled-assignment-conflict-def* **by** *simp*

lemma *generic-fri-sampled-assignment-base-matches-step*:
assumes *no-conflict*:
 $\neg$ *generic-fri-sampled-assignment-conflict candidate-table roots*
challenges final-value query-idxs round-layers
and *challenges-nonempty*: $0 < \text{length } \text{challenges}$
and *round-bound*: $\text{round-idx} < \text{length } \text{query-idxs}$
and *step*:
fri-layer-step-evidence
(roots ! 0)
(challenges ! 0)
(fri-evidence-layer-len roots 0)
(fri-evidence-layer-idx roots query-idxs round-idx 0)
 1
(fri-sibling-index (fri-evidence-layer-len roots 0)
(fri-evidence-layer-idx roots query-idxs round-idx 0))
xp xp-path xn xn-path
(fri-evidence-next-idx roots query-idxs round-idx 0)
(fri-evidence-next-value roots challenges query-idxs round-idx 0
xp xn)
(round-layers ! round-idx ! 0)
shows
fri-opening-matches-table (fri-evidence-layer-len roots 0)
(fri-evidence-layer-idx roots query-idxs round-idx 0)
(generic-fri-sampled-assignment-layers candidate-table roots
challenges final-value query-idxs round-layers ! 0) xp xn
proof –
have *base-match*:
fri-opening-matches-table (fri-evidence-layer-len roots 0)
(fri-evidence-layer-idx roots query-idxs round-idx 0)
candidate-table xp xn
by (*rule generic-fri-no-base-opening-conflict-matches-step*
 $[OF \text{ generic-fri-no-assignment-conflict-no-base } [OF \text{ no-conflict}]$
 $\text{challenges-nonempty round-bound step}]$)
show *?thesis*
using *base-match* **by** *simp*
qed

lemma *generic-fri-no-successor-opening-conflict-matches-left*:
assumes *no-conflict*:

$\neg$ generic-fri-sampled-successor-opening-conflict roots challenges
 query-idxs round-layers
and source-bound: source-round < length query-idxs
and target-bound: target-round < length query-idxs
and layer-bound: Suc layer-idx < length challenges
and forced:
 generic-fri-round-forced-next-value roots challenges query-idxs
 round-layers source-round layer-idx v
and step:
 fri-layer-step-evidence
 (roots ! Suc layer-idx)
 (challenges ! Suc layer-idx)
 (fri-evidence-layer-len roots (Suc layer-idx))
 (fri-evidence-layer-idx roots query-idxs target-round
 (Suc layer-idx))
 ($2 \wedge$ Suc layer-idx)
 (fri-sibling-index (fri-evidence-layer-len roots (Suc layer-idx))
 (fri-evidence-layer-idx roots query-idxs target-round
 (Suc layer-idx)))
 xp xp-path xn xn-path
 (fri-evidence-next-idx roots query-idxs target-round
 (Suc layer-idx))
 (fri-evidence-next-value roots challenges query-idxs target-round
 (Suc layer-idx) xp xn)
 (round-layers ! target-round ! Suc layer-idx)
and same-idx:
 fri-evidence-next-idx roots query-idxs source-round layer-idx =
 fri-evidence-layer-idx roots query-idxs target-round
 (Suc layer-idx)
shows v = xp
using assms
unfolding generic-fri-sampled-successor-opening-conflict-def
by blast

lemma generic-fri-no-successor-opening-conflict-matches-right:
assumes no-conflict:
 $\neg$ generic-fri-sampled-successor-opening-conflict roots challenges
 query-idxs round-layers
and source-bound: source-round < length query-idxs
and target-bound: target-round < length query-idxs
and layer-bound: Suc layer-idx < length challenges
and forced:
 generic-fri-round-forced-next-value roots challenges query-idxs
 round-layers source-round layer-idx v
and step:
 fri-layer-step-evidence
 (roots ! Suc layer-idx)
 (challenges ! Suc layer-idx)
 (fri-evidence-layer-len roots (Suc layer-idx))

```

(fri-evidence-layer-idx roots query-idxs target-round
 (Suc layer-idx))
(2 ^ Suc layer-idx)
(fri-sibling-index (fri-evidence-layer-len roots (Suc layer-idx))
 (fri-evidence-layer-idx roots query-idxs target-round
  (Suc layer-idx)))
xp xp-path xn xn-path
(fri-evidence-next-idx roots query-idxs target-round
 (Suc layer-idx))
(fri-evidence-next-value roots challenges query-idxs target-round
 (Suc layer-idx) xp xn)
(round-layers ! target-round ! Suc layer-idx)
and same-idx:
  fri-evidence-next-idx roots query-idxs source-round layer-idx =
  fri-sibling-index (fri-evidence-layer-len roots (Suc layer-idx))
  (fri-evidence-layer-idx roots query-idxs target-round
   (Suc layer-idx))
shows v = xn
using assms
unfolding generic-fri-sampled-successor-opening-conflict-def
by blast

```

lemma generic-fri-no-same-layer-opening-conflict-left-left:

```

assumes no-conflict:
  ¬ generic-fri-sampled-same-layer-opening-conflict roots challenges
  query-idxs round-layers
and round-bound: round-idx < length query-idxs
and round-bound': round-idx' < length query-idxs
and layer-bound: layer-idx < length challenges
and step:
  fri-layer-step-evidence
  (roots ! layer-idx)
  (challenges ! layer-idx)
  (fri-evidence-layer-len roots layer-idx)
  (fri-evidence-layer-idx roots query-idxs round-idx layer-idx)
  (2 ^ layer-idx)
  (fri-sibling-index (fri-evidence-layer-len roots layer-idx)
   (fri-evidence-layer-idx roots query-idxs round-idx layer-idx))
  xp xp-path xn xn-path
  (fri-evidence-next-idx roots query-idxs round-idx layer-idx)
  (fri-evidence-next-value roots challenges query-idxs round-idx
   layer-idx xp xn)
  (round-layers ! round-idx ! layer-idx)
and step':
  fri-layer-step-evidence
  (roots ! layer-idx)
  (challenges ! layer-idx)
  (fri-evidence-layer-len roots layer-idx)
  (fri-evidence-layer-idx roots query-idxs round-idx' layer-idx)

```

```

    (2 ^ layer-idx)
    (fri-sibling-index (fri-evidence-layer-len roots layer-idx)
      (fri-evidence-layer-idx roots query-idxs round-idx' layer-idx))
    yp yp-path yn yn-path
    (fri-evidence-next-idx roots query-idxs round-idx' layer-idx)
    (fri-evidence-next-value roots challenges query-idxs round-idx'
      layer-idx yp yn)
    (round-layers ! round-idx' ! layer-idx)
and same-idx:
    fri-evidence-layer-idx roots query-idxs round-idx layer-idx =
    fri-evidence-layer-idx roots query-idxs round-idx' layer-idx
shows xp = yp
using assms
unfolding generic-fri-sampled-same-layer-opening-conflict-def
by blast

```

lemma generic-fri-no-same-layer-opening-conflict-left-right:

```

assumes no-conflict:
  ¬ generic-fri-sampled-same-layer-opening-conflict roots challenges
    query-idxs round-layers
and round-bound: round-idx < length query-idxs
and round-bound': round-idx' < length query-idxs
and layer-bound: layer-idx < length challenges
and step:
  fri-layer-step-evidence
    (roots ! layer-idx)
    (challenges ! layer-idx)
    (fri-evidence-layer-len roots layer-idx)
    (fri-evidence-layer-idx roots query-idxs round-idx layer-idx)
    (2 ^ layer-idx)
    (fri-sibling-index (fri-evidence-layer-len roots layer-idx)
      (fri-evidence-layer-idx roots query-idxs round-idx layer-idx))
    xp xp-path xn xn-path
    (fri-evidence-next-idx roots query-idxs round-idx layer-idx)
    (fri-evidence-next-value roots challenges query-idxs round-idx
      layer-idx xp xn)
    (round-layers ! round-idx ! layer-idx)
and step':
  fri-layer-step-evidence
    (roots ! layer-idx)
    (challenges ! layer-idx)
    (fri-evidence-layer-len roots layer-idx)
    (fri-evidence-layer-idx roots query-idxs round-idx' layer-idx)
    (2 ^ layer-idx)
    (fri-sibling-index (fri-evidence-layer-len roots layer-idx)
      (fri-evidence-layer-idx roots query-idxs round-idx' layer-idx))
    yp yp-path yn yn-path
    (fri-evidence-next-idx roots query-idxs round-idx' layer-idx)
    (fri-evidence-next-value roots challenges query-idxs round-idx')

```

```

    layer-idx yp yn)
  (round-layers ! round-idx' ! layer-idx)
and same-idx:
  fri-evidence-layer-idx roots query-idxs round-idx layer-idx =
    fri-sibling-index (fri-evidence-layer-len roots layer-idx)
    (fri-evidence-layer-idx roots query-idxs round-idx' layer-idx)
shows xp = yn
using assms
unfolding generic-fri-sampled-same-layer-opening-conflict-def
by blast

```

lemma generic-fri-no-same-layer-opening-conflict-right-left:

```

assumes no-conflict:
  ¬ generic-fri-sampled-same-layer-opening-conflict roots challenges
    query-idxs round-layers
and round-bound: round-idx < length query-idxs
and round-bound': round-idx' < length query-idxs
and layer-bound: layer-idx < length challenges
and step:
  fri-layer-step-evidence
    (roots ! layer-idx)
    (challenges ! layer-idx)
    (fri-evidence-layer-len roots layer-idx)
    (fri-evidence-layer-idx roots query-idxs round-idx layer-idx)
    (2 ^ layer-idx)
    (fri-sibling-index (fri-evidence-layer-len roots layer-idx)
      (fri-evidence-layer-idx roots query-idxs round-idx layer-idx))
  xp xp-path xn xn-path
  (fri-evidence-next-idx roots query-idxs round-idx layer-idx)
  (fri-evidence-next-value roots challenges query-idxs round-idx
    layer-idx xp xn)
  (round-layers ! round-idx ! layer-idx)
and step':
  fri-layer-step-evidence
    (roots ! layer-idx)
    (challenges ! layer-idx)
    (fri-evidence-layer-len roots layer-idx)
    (fri-evidence-layer-idx roots query-idxs round-idx' layer-idx)
    (2 ^ layer-idx)
    (fri-sibling-index (fri-evidence-layer-len roots layer-idx)
      (fri-evidence-layer-idx roots query-idxs round-idx' layer-idx))
  yp yp-path yn yn-path
  (fri-evidence-next-idx roots query-idxs round-idx' layer-idx)
  (fri-evidence-next-value roots challenges query-idxs round-idx'
    layer-idx yp yn)
  (round-layers ! round-idx' ! layer-idx)
and same-idx:
  fri-sibling-index (fri-evidence-layer-len roots layer-idx)
    (fri-evidence-layer-idx roots query-idxs round-idx layer-idx) =

```

fri-evidence-layer-idx roots query-idxs round-idx' layer-idx
shows $xn = yp$
using *assms*
unfolding *generic-fri-sampled-same-layer-opening-conflict-def*
by *blast*

lemma *generic-fri-no-same-layer-opening-conflict-right-right:*
assumes *no-conflict:*
 $\neg$ *generic-fri-sampled-same-layer-opening-conflict roots challenges*
query-idxs round-layers
and *round-bound:* $\text{round-idx} < \text{length query-idxs}$
and *round-bound':* $\text{round-idx}' < \text{length query-idxs}$
and *layer-bound:* $\text{layer-idx} < \text{length challenges}$
and *step:*
fri-layer-step-evidence
(roots ! layer-idx)
(challenges ! layer-idx)
(fri-evidence-layer-len roots layer-idx)
(fri-evidence-layer-idx roots query-idxs round-idx layer-idx)
 $(2 \wedge \text{layer-idx})$
(fri-sibling-index (fri-evidence-layer-len roots layer-idx)
(fri-evidence-layer-idx roots query-idxs round-idx layer-idx))
xp xp-path xn xn-path
(fri-evidence-next-idx roots query-idxs round-idx layer-idx)
(fri-evidence-next-value roots challenges query-idxs round-idx
layer-idx xp xn)
(round-layers ! round-idx ! layer-idx)
and *step':*
fri-layer-step-evidence
(roots ! layer-idx)
(challenges ! layer-idx)
(fri-evidence-layer-len roots layer-idx)
(fri-evidence-layer-idx roots query-idxs round-idx' layer-idx)
 $(2 \wedge \text{layer-idx})$
(fri-sibling-index (fri-evidence-layer-len roots layer-idx)
(fri-evidence-layer-idx roots query-idxs round-idx' layer-idx))
yp yp-path yn yn-path
(fri-evidence-next-idx roots query-idxs round-idx' layer-idx)
(fri-evidence-next-value roots challenges query-idxs round-idx'
layer-idx yp yn)
(round-layers ! round-idx' ! layer-idx)
and *same-idx:*
fri-sibling-index (fri-evidence-layer-len roots layer-idx)
(fri-evidence-layer-idx roots query-idxs round-idx layer-idx) =
fri-sibling-index (fri-evidence-layer-len roots layer-idx)
(fri-evidence-layer-idx roots query-idxs round-idx' layer-idx)
shows $xn = yn$
using *assms*
unfolding *generic-fri-sampled-same-layer-opening-conflict-def*

by blast

lemma *generic-fri-no-assignment-conflict-no-successor:*

assumes

$\neg$ *generic-fri-sampled-assignment-conflict candidate-table roots*
challenges final-value query-idxs round-layers

shows

$\neg$ *generic-fri-sampled-successor-opening-conflict roots challenges*
query-idxs round-layers

using *assms unfolding generic-fri-sampled-assignment-conflict-def* **by** *simp*

lemma *generic-fri-no-assignment-conflict-no-same:*

assumes

$\neg$ *generic-fri-sampled-assignment-conflict candidate-table roots*
challenges final-value query-idxs round-layers

shows

$\neg$ *generic-fri-sampled-same-layer-opening-conflict roots challenges*
query-idxs round-layers

using *assms unfolding generic-fri-sampled-assignment-conflict-def* **by** *simp*

lemma *generic-fri-sampled-layer-value-constraint-current-left:*

assumes *layer-nonzero: 0 < layer-idx*

and *layer-bound: layer-idx < length challenges*

and *round-bound: round-idx < length query-idxs*

and *step:*

fri-layer-step-evidence

(roots ! layer-idx)

(challenges ! layer-idx)

(fri-evidence-layer-len roots layer-idx)

(fri-evidence-layer-idx roots query-idxs round-idx layer-idx)

(2 ^ layer-idx)

(fri-sibling-index (fri-evidence-layer-len roots layer-idx)

(fri-evidence-layer-idx roots query-idxs round-idx layer-idx))

xp xp-path xn xn-path

(fri-evidence-next-idx roots query-idxs round-idx layer-idx)

(fri-evidence-next-value roots challenges query-idxs round-idx
layer-idx xp xn)

(round-layers ! round-idx ! layer-idx)

shows

generic-fri-sampled-layer-value-constraint candidate-table roots

challenges final-value query-idxs round-layers layer-idx

(fri-evidence-layer-idx roots query-idxs round-idx layer-idx) xp

proof –

have *nz: layer-idx ≠ 0*

using *layer-nonzero* **by** *simp*

have *not-final: layer-idx ≠ length challenges*

using *layer-nonzero layer-bound* **by** *simp-all*

show *?thesis*

unfolding *generic-fri-sampled-layer-value-constraint-def*

```

apply (simp add: nz not-final)
apply (rule disjI2)
apply (rule exI[of - round-idx])
apply (intro conjI)
  apply (rule round-bound)
  apply (rule layer-bound)
apply (rule exI[of - xp])
apply (rule exI[of - xp-path])
apply (rule exI[of - xn])
apply (intro conjI)
  apply (rule exI[of - xn-path])
  apply (rule step)
  apply simp
done
qed

lemma generic-fri-sampled-layer-value-constraint-current-right:
  assumes layer-nonzero: 0 < layer-idx
  and layer-bound: layer-idx < length challenges
  and round-bound: round-idx < length query-idxs
  and step:
    fri-layer-step-evidence
    (roots ! layer-idx)
    (challenges ! layer-idx)
    (fri-evidence-layer-len roots layer-idx)
    (fri-evidence-layer-idx roots query-idxs round-idx layer-idx)
    ( $2 \wedge \text{layer-idx}$ )
    (fri-sibling-index (fri-evidence-layer-len roots layer-idx)
     (fri-evidence-layer-idx roots query-idxs round-idx layer-idx))
    xp xp-path xn xn-path
    (fri-evidence-next-idx roots query-idxs round-idx layer-idx)
    (fri-evidence-next-value roots challenges query-idxs round-idx
     layer-idx xp xn)
    (round-layers ! round-idx ! layer-idx)
  shows
    generic-fri-sampled-layer-value-constraint candidate-table roots
    challenges final-value query-idxs round-layers layer-idx
    (fri-sibling-index (fri-evidence-layer-len roots layer-idx)
     (fri-evidence-layer-idx roots query-idxs round-idx layer-idx)) xn
proof –
  have nz: layer-idx ≠ 0
  using layer-nonzero by simp
  have not-final: layer-idx ≠ length challenges
  using layer-nonzero layer-bound by simp-all
  show ?thesis
  unfolding generic-fri-sampled-layer-value-constraint-def
  apply (simp add: nz not-final)
  apply (rule disjI2)
  apply (rule exI[of - round-idx])

```



```

apply (intro conjI)
  apply (rule round-bound)
  apply (rule layer-bound)
apply (rule exI[of - xp])
apply (rule exI[of - xp-path])
apply (rule exI[of - xn])
apply (intro conjI)
  apply (rule exI[of - xn-path])
  apply (rule step)
  apply simp
done
qed

lemma generic-fri-sampled-assignment-current-left-matches-step:
assumes no-conflict:
   $\neg$  generic-fri-sampled-assignment-conflict candidate-table roots
  challenges final-value query-idxs round-layers
and layer-nonzero:  $0 < \text{layer-idx}$ 
and layer-bound:  $\text{layer-idx} < \text{length challenges}$ 
and round-bound:  $\text{round-idx} < \text{length query-idxs}$ 
and idx-bound:
  fri-evidence-layer-idx roots query-idxs round-idx layer-idx <
  length * scale
and step:
  fri-layer-step-evidence
  (roots ! layer-idx)
  (challenges ! layer-idx)
  (fri-evidence-layer-len roots layer-idx)
  (fri-evidence-layer-idx roots query-idxs round-idx layer-idx)
  ( $2^{\text{layer-idx}}$ )
  (fri-sibling-index (fri-evidence-layer-len roots layer-idx)
    (fri-evidence-layer-idx roots query-idxs round-idx layer-idx))
  xp xp-path xn xn-path
  (fri-evidence-next-idx roots query-idxs round-idx layer-idx)
  (fri-evidence-next-value roots challenges query-idxs round-idx
    layer-idx xp xn)
  (round-layers ! round-idx ! layer-idx)
shows
  generic-fri-sampled-assignment-layers candidate-table roots challenges
  final-value query-idxs round-layers ! layer-idx !
  fri-evidence-layer-idx roots query-idxs round-idx layer-idx = xp
proof –
let ?raw = fri-evidence-layer-idx roots query-idxs round-idx layer-idx
have layer-eq:
  generic-fri-sampled-assignment-layers candidate-table roots challenges
  final-value query-idxs round-layers ! layer-idx =
  generic-fri-sampled-assignment-layer candidate-table roots challenges
  final-value query-idxs round-layers layer-idx
by (rule generic-fri-sampled-assignment-layers-nth)

```

```

    (use layer-bound in simp)
  have constraint:
    generic-fri-sampled-layer-value-constraint candidate-table roots
      challenges final-value query-idxs round-layers layer-idx ?raw xp
  by (rule generic-fri-sampled-layer-value-constraint-current-left
      [OF layer-nonzero layer-bound round-bound step])
  have unique:
     $\wedge v'. \text{generic-fri-sampled-layer-value-constraint candidate-table roots}$ 
      challenges final-value query-idxs round-layers layer-idx ?raw v'
       $\implies v' = xp$ 
  proof -
    fix v'
    assume c:
      generic-fri-sampled-layer-value-constraint candidate-table roots
        challenges final-value query-idxs round-layers layer-idx ?raw v'
    have pred-or-current:
      ( $\exists$  source-round.
        source-round < length query-idxs  $\wedge$ 
        generic-fri-round-forced-next-value roots challenges query-idxs
          round-layers source-round (layer-idx - 1) v'  $\wedge$ 
        fri-evidence-next-idx roots query-idxs source-round
          (layer-idx - 1) = ?raw)  $\vee$ 
      ( $\exists$  round-idx' yp yp-path yn yn-path.
        round-idx' < length query-idxs  $\wedge$ 
        layer-idx < length challenges  $\wedge$ 
        fri-layer-step-evidence
          (roots ! layer-idx)
          (challenges ! layer-idx)
          (fri-evidence-layer-len roots layer-idx)
          (fri-evidence-layer-idx roots query-idxs round-idx' layer-idx)
          (2  $\wedge$  layer-idx)
          (fri-sibling-index (fri-evidence-layer-len roots layer-idx)
            (fri-evidence-layer-idx roots query-idxs round-idx' layer-idx))
          yp yp-path yn yn-path
          (fri-evidence-next-idx roots query-idxs round-idx' layer-idx)
          (fri-evidence-next-value roots challenges query-idxs round-idx'
            layer-idx yp yn)
          (round-layers ! round-idx' ! layer-idx)  $\wedge$ 
          ((?raw = fri-evidence-layer-idx roots query-idxs round-idx'
            layer-idx  $\wedge$  v' = yp)  $\vee$ 
          (?raw = fri-sibling-index (fri-evidence-layer-len roots layer-idx)
            (fri-evidence-layer-idx roots query-idxs round-idx' layer-idx)  $\wedge$ 
            v' = yn)))
    using c layer-nonzero layer-bound
  unfolding generic-fri-sampled-layer-value-constraint-def
  by auto
  then show v' = xp
  proof
    assume

```

```

 $\exists$  source-round.
  source-round < length query-idxs  $\wedge$ 
  generic-fri-round-forced-next-value roots challenges query-idxs
    round-layers source-round (layer-idx - 1) v'  $\wedge$ 
  fri-evidence-next-idx roots query-idxs source-round
    (layer-idx - 1) = ?raw
then obtain source-round where source-bound:
  source-round < length query-idxs
and forced:
  generic-fri-round-forced-next-value roots challenges query-idxs
    round-layers source-round (layer-idx - 1) v'
and same-idx:
  fri-evidence-next-idx roots query-idxs source-round
    (layer-idx - 1) = ?raw
by blast
have suc: Suc (layer-idx - 1) = layer-idx
using layer-nonzero by simp
have v' = xp
using generic-fri-no-successor-opening-conflict-matches-left
  [OF generic-fri-no-assignment-conflict-no-successor[OF no-conflict]
    source-bound round-bound, of layer-idx - 1 v'
    xp xp-path xn xn-path]
  layer-bound forced step same-idx suc by simp
then show ?thesis .
next
assume
 $\exists$  round-idx' yp yp-path yn yn-path.
  round-idx' < length query-idxs  $\wedge$ 
  layer-idx < length challenges  $\wedge$ 
  fri-layer-step-evidence
    (roots ! layer-idx)
    (challenges ! layer-idx)
    (fri-evidence-layer-len roots layer-idx)
    (fri-evidence-layer-idx roots query-idxs round-idx' layer-idx)
    (2  $\wedge$  layer-idx)
    (fri-sibling-index (fri-evidence-layer-len roots layer-idx)
      (fri-evidence-layer-idx roots query-idxs round-idx' layer-idx))
  yp yp-path yn yn-path
  (fri-evidence-next-idx roots query-idxs round-idx' layer-idx)
  (fri-evidence-next-value roots challenges query-idxs round-idx'
    layer-idx yp yn)
  (round-layers ! round-idx' ! layer-idx)  $\wedge$ 
  ((?raw = fri-evidence-layer-idx roots query-idxs round-idx'
    layer-idx  $\wedge$  v' = yp)  $\vee$ 
    (?raw = fri-sibling-index (fri-evidence-layer-len roots layer-idx)
      (fri-evidence-layer-idx roots query-idxs round-idx' layer-idx)  $\wedge$ 
      v' = yn))
then obtain round-idx' yp yp-path yn yn-path where round-bound':
  round-idx' < length query-idxs

```

```

and step':
  fri-layer-step-evidence
    (roots ! layer-idx)
    (challenges ! layer-idx)
    (fri-evidence-layer-len roots layer-idx)
    (fri-evidence-layer-idx roots query-idxs round-idx' layer-idx)
    ( $2 \wedge \text{layer-idx}$ )
    (fri-sibling-index (fri-evidence-layer-len roots layer-idx)
      (fri-evidence-layer-idx roots query-idxs round-idx' layer-idx))
    yp yp-path yn yn-path
    (fri-evidence-next-idx roots query-idxs round-idx' layer-idx)
    (fri-evidence-next-value roots challenges query-idxs round-idx'
      layer-idx yp yn)
    (round-layers ! round-idx' ! layer-idx)
and value-case:
  (?raw = fri-evidence-layer-idx roots query-idxs round-idx'
    layer-idx  $\wedge$  v' = yp)  $\vee$ 
  (?raw = fri-sibling-index (fri-evidence-layer-len roots layer-idx)
    (fri-evidence-layer-idx roots query-idxs round-idx' layer-idx)  $\wedge$ 
    v' = yn)
by blast
from value-case show ?thesis
proof
  assume raw-case:
    ?raw = fri-evidence-layer-idx roots query-idxs round-idx'
      layer-idx  $\wedge$  v' = yp
  have xp = yp
  by (rule generic-fri-no-same-layer-opening-conflict-left-left
    [OF generic-fri-no-assignment-conflict-no-same[OF no-conflict]
      round-bound round-bound' layer-bound step step'])
    (use raw-case in simp)
  then show ?thesis
    using raw-case by simp
next
  assume sibling-case:
    ?raw = fri-sibling-index (fri-evidence-layer-len roots layer-idx)
      (fri-evidence-layer-idx roots query-idxs round-idx' layer-idx)  $\wedge$ 
      v' = yn
  have xp = yn
  by (rule generic-fri-no-same-layer-opening-conflict-left-right
    [OF generic-fri-no-assignment-conflict-no-same[OF no-conflict]
      round-bound round-bound' layer-bound step step'])
    (use sibling-case in simp)
  then show ?thesis
    using sibling-case by simp
qed
qed
qed
have nth-raw:

```

```

generic-fri-sampled-assignment-layer candidate-table roots challenges
  final-value query-idxs round-layers layer-idx ! ?raw = xp
by (rule generic-fri-sampled-assignment-layer-nth-unique
  [OF idx-bound - constraint unique])
  (use layer-nonzero in simp)
show ?thesis
  using nth-row unfolding layer-eq by simp
qed

```

```

lemma generic-fri-sampled-assignment-current-right-matches-step:
  assumes no-conflict:
    ¬ generic-fri-sampled-assignment-conflict candidate-table roots
      challenges final-value query-idxs round-layers
  and layer-nonzero: 0 < layer-idx
  and layer-bound: layer-idx < length challenges
  and round-bound: round-idx < length query-idxs
  and idx-bound:
    fri-sibling-index (fri-evidence-layer-len roots layer-idx)
      (fri-evidence-layer-idx roots query-idxs round-idx layer-idx) <
        clength * scale
  and step:
    fri-layer-step-evidence
      (roots ! layer-idx)
      (challenges ! layer-idx)
      (fri-evidence-layer-len roots layer-idx)
      (fri-evidence-layer-idx roots query-idxs round-idx layer-idx)
      (2 ^ layer-idx)
      (fri-sibling-index (fri-evidence-layer-len roots layer-idx)
        (fri-evidence-layer-idx roots query-idxs round-idx layer-idx))
      xp xp-path xn xn-path
      (fri-evidence-next-idx roots query-idxs round-idx layer-idx)
      (fri-evidence-next-value roots challenges query-idxs round-idx
        layer-idx xp xn)
      (round-layers ! round-idx ! layer-idx)
  shows
    generic-fri-sampled-assignment-layers candidate-table roots challenges
      final-value query-idxs round-layers ! layer-idx !
      fri-sibling-index (fri-evidence-layer-len roots layer-idx)
        (fri-evidence-layer-idx roots query-idxs round-idx layer-idx) = xn
proof -
  let ?sib =
    fri-sibling-index (fri-evidence-layer-len roots layer-idx)
      (fri-evidence-layer-idx roots query-idxs round-idx layer-idx)
  have layer-eq:
    generic-fri-sampled-assignment-layers candidate-table roots challenges
      final-value query-idxs round-layers ! layer-idx =
    generic-fri-sampled-assignment-layer candidate-table roots challenges
      final-value query-idxs round-layers layer-idx
  by (rule generic-fri-sampled-assignment-layers-nth)

```

```

    (use layer-bound in simp)
  have constraint:
    generic-fri-sampled-layer-value-constraint candidate-table roots
    challenges final-value query-idxs round-layers layer-idx ?sib xn
  by (rule generic-fri-sampled-layer-value-constraint-current-right
    [OF layer-nonzero layer-bound round-bound step])
  have unique:
     $\wedge v'. \text{generic-fri-sampled-layer-value-constraint candidate-table roots}$ 
    challenges final-value query-idxs round-layers layer-idx ?sib  $v'$ 
     $\implies v' = xn$ 
  proof -
    fix  $v'$ 
    assume c:
      generic-fri-sampled-layer-value-constraint candidate-table roots
      challenges final-value query-idxs round-layers layer-idx ?sib  $v'$ 
    have pred-or-current:
      ( $\exists$  source-round.
        source-round < length query-idxs  $\wedge$ 
        generic-fri-round-forced-next-value roots challenges query-idxs
        round-layers source-round (layer-idx - 1)  $v' \wedge$ 
        fri-evidence-next-idx roots query-idxs source-round
        (layer-idx - 1) = ?sib)  $\vee$ 
      ( $\exists$  round-idx' yp yp-path yn yn-path.
        round-idx' < length query-idxs  $\wedge$ 
        layer-idx < length challenges  $\wedge$ 
        fri-layer-step-evidence
        (roots ! layer-idx)
        (challenges ! layer-idx)
        (fri-evidence-layer-len roots layer-idx)
        (fri-evidence-layer-idx roots query-idxs round-idx' layer-idx)
        ( $2 \wedge$  layer-idx)
        (fri-sibling-index (fri-evidence-layer-len roots layer-idx)
        (fri-evidence-layer-idx roots query-idxs round-idx' layer-idx))
        yp yp-path yn yn-path
        (fri-evidence-next-idx roots query-idxs round-idx' layer-idx)
        (fri-evidence-next-value roots challenges query-idxs round-idx'
        layer-idx yp yn)
        (round-layers ! round-idx' ! layer-idx)  $\wedge$ 
        ((?sib = fri-evidence-layer-idx roots query-idxs round-idx'
        layer-idx  $\wedge v' = yp$ )  $\vee$ 
        (?sib = fri-sibling-index (fri-evidence-layer-len roots layer-idx)
        (fri-evidence-layer-idx roots query-idxs round-idx' layer-idx)  $\wedge$ 
         $v' = yn$ )))
    using c layer-nonzero layer-bound
    unfolding generic-fri-sampled-layer-value-constraint-def
    by auto
  then show  $v' = xn$ 
  proof
    assume

```

$\exists$ *source-round*.
source-round < *length query-idxs* $\wedge$
generic-fri-round-forced-next-value roots challenges query-idxs
round-layers source-round (layer-idx - 1) v' $\wedge$
fri-evidence-next-idx roots query-idxs source-round
(layer-idx - 1) = ?sib
then obtain *source-round* **where** *source-bound*:
source-round < *length query-idxs*
and forced:
generic-fri-round-forced-next-value roots challenges query-idxs
round-layers source-round (layer-idx - 1) v'
and same-idx:
fri-evidence-next-idx roots query-idxs source-round
(layer-idx - 1) = ?sib
by *blast*
have *suc*: *Suc (layer-idx - 1) = layer-idx*
using *layer-nonzero* **by** *simp*
have *v' = xn*
using *generic-fri-no-successor-opening-conflict-matches-right*
 $[OF \text{ generic-fri-no-assignment-conflict-no-successor}[OF \text{ no-conflict}]$
source-bound round-bound, of layer-idx - 1 v'
xp xp-path xn xn-path]
layer-bound forced step same-idx suc **by** *simp*
then show *?thesis* .
next
assume
 $\exists$ *round-idx' yp yp-path yn yn-path*.
round-idx' < length query-idxs $\wedge$
layer-idx < length challenges $\wedge$
fri-layer-step-evidence
(roots ! layer-idx)
(challenges ! layer-idx)
(fri-evidence-layer-len roots layer-idx)
(fri-evidence-layer-idx roots query-idxs round-idx' layer-idx)
(2 $\wedge$ layer-idx)
(fri-sibling-index (fri-evidence-layer-len roots layer-idx)
(fri-evidence-layer-idx roots query-idxs round-idx' layer-idx))
yp yp-path yn yn-path
(fri-evidence-next-idx roots query-idxs round-idx' layer-idx)
(fri-evidence-next-value roots challenges query-idxs round-idx'
layer-idx yp yn)
(round-layers ! round-idx' ! layer-idx) $\wedge$
 $((?sib = \text{fri-evidence-layer-idx roots query-idxs round-idx'}$
layer-idx $\wedge$ v' = yp) $\vee$
 $(?sib = \text{fri-sibling-index (fri-evidence-layer-len roots layer-idx)}$
(fri-evidence-layer-idx roots query-idxs round-idx' layer-idx) $\wedge$
v' = yn))
then obtain *round-idx' yp yp-path yn yn-path* **where** *round-bound'*:
round-idx' < length query-idxs

```

and step':
  fri-layer-step-evidence
    (roots ! layer-idx)
    (challenges ! layer-idx)
    (fri-evidence-layer-len roots layer-idx)
    (fri-evidence-layer-idx roots query-idxs round-idx' layer-idx)
    ( $2 \wedge \text{layer-idx}$ )
    (fri-sibling-index (fri-evidence-layer-len roots layer-idx)
      (fri-evidence-layer-idx roots query-idxs round-idx' layer-idx))
    yp yp-path yn yn-path
    (fri-evidence-next-idx roots query-idxs round-idx' layer-idx)
    (fri-evidence-next-value roots challenges query-idxs round-idx'
      layer-idx yp yn)
    (round-layers ! round-idx' ! layer-idx)
and value-case:
  (?sib = fri-evidence-layer-idx roots query-idxs round-idx'
    layer-idx  $\wedge$  v' = yp)  $\vee$ 
  (?sib = fri-sibling-index (fri-evidence-layer-len roots layer-idx)
    (fri-evidence-layer-idx roots query-idxs round-idx' layer-idx)  $\wedge$ 
    v' = yn)
by blast
from value-case show ?thesis
proof
  assume raw-case:
    ?sib = fri-evidence-layer-idx roots query-idxs round-idx'
      layer-idx  $\wedge$  v' = yp
  have xn = yp
    by (rule generic-fri-no-same-layer-opening-conflict-right-left
      [OF generic-fri-no-assignment-conflict-no-same[OF no-conflict]
        round-bound round-bound' layer-bound step step'])
    (use raw-case in simp)
  then show ?thesis
    using raw-case by simp
next
  assume sibling-case:
    ?sib = fri-sibling-index (fri-evidence-layer-len roots layer-idx)
      (fri-evidence-layer-idx roots query-idxs round-idx' layer-idx)  $\wedge$ 
      v' = yn
  have xn = yn
    by (rule generic-fri-no-same-layer-opening-conflict-right-right
      [OF generic-fri-no-assignment-conflict-no-same[OF no-conflict]
        round-bound round-bound' layer-bound step step'])
    (use sibling-case in simp)
  then show ?thesis
    using sibling-case by simp
qed
qed
qed
have nth-sib:

```



```

generic-fri-sampled-assignment-layer candidate-table roots challenges
  final-value query-idxs round-layers layer-idx ! ?sib = xn
by (rule generic-fri-sampled-assignment-layer-nth-unique
  [OF idx-bound - constraint unique])
  (use layer-nonzero in simp)
show ?thesis
  using nth-sib unfolding layer-eq by simp
qed

```

```

lemma generic-fri-sampled-assignment-current-matches-step:
  assumes no-conflict:
    ¬ generic-fri-sampled-assignment-conflict candidate-table roots
      challenges final-value query-idxs round-layers
  and layer-nonzero: 0 < layer-idx
  and layer-bound: layer-idx < length challenges
  and round-bound: round-idx < length query-idxs
  and raw-bound:
    fri-evidence-layer-idx roots query-idxs round-idx layer-idx <
      fri-evidence-layer-len roots layer-idx
  and len-bound:
    fri-evidence-layer-len roots layer-idx ≤ clength * scale
  and sibling-bound:
    fri-sibling-index (fri-evidence-layer-len roots layer-idx)
      (fri-evidence-layer-idx roots query-idxs round-idx layer-idx) <
        clength * scale
  and step:
    fri-layer-step-evidence
      (roots ! layer-idx)
      (challenges ! layer-idx)
      (fri-evidence-layer-len roots layer-idx)
      (fri-evidence-layer-idx roots query-idxs round-idx layer-idx)
      (2 ^ layer-idx)
      (fri-sibling-index (fri-evidence-layer-len roots layer-idx)
        (fri-evidence-layer-idx roots query-idxs round-idx layer-idx))
      xp xp-path xn xn-path
      (fri-evidence-next-idx roots query-idxs round-idx layer-idx)
      (fri-evidence-next-value roots challenges query-idxs round-idx
        layer-idx xp xn)
      (round-layers ! round-idx ! layer-idx)
  shows
    fri-opening-matches-table (fri-evidence-layer-len roots layer-idx)
      (fri-evidence-layer-idx roots query-idxs round-idx layer-idx)
      (generic-fri-sampled-assignment-layers candidate-table roots
        challenges final-value query-idxs round-layers ! layer-idx) xp xn
proof -
  let ?len = fri-evidence-layer-len roots layer-idx
  let ?raw = fri-evidence-layer-idx roots query-idxs round-idx layer-idx
  let ?sib = fri-sibling-index ?len ?raw
  let ?layer =

```

```

    generic-fri-sampled-assignment-layers candidate-table roots challenges
    final-value query-idxs round-layers ! layer-idx
  have raw-bound-assignment: ?raw < clength * scale
    using raw-bound len-bound by simp
  have layer-eq:
    ?layer =
      generic-fri-sampled-assignment-layer candidate-table roots challenges
      final-value query-idxs round-layers layer-idx
    by (rule generic-fri-sampled-assignment-layers-nth)
      (use layer-bound in simp)
  have layer-len: length ?layer = clength * scale
    using layer-eq layer-nonzero by simp
  have left:
    ?layer ! ?raw = xp
    by (rule generic-fri-sampled-assignment-current-left-matches-step
      [OF no-conflict layer-nonzero layer-bound round-bound
        raw-bound-assignment step])
  have right:
    ?layer ! ?sib = xn
    by (rule generic-fri-sampled-assignment-current-right-matches-step
      [OF no-conflict layer-nonzero layer-bound round-bound
        sibling-bound step])
  show ?thesis
    unfolding fri-opening-matches-table-def
    using raw-bound len-bound layer-len left right by simp
qed

```

```

lemma generic-fri-sampled-assignment-successor-forced-value:
  assumes no-conflict:
    ¬ generic-fri-sampled-assignment-conflict candidate-table roots
    challenges final-value query-idxs round-layers
  and layer-bound: Suc layer-idx < length challenges
  and round-bound: round-idx < length query-idxs
  and next-bound:
    fri-evidence-next-idx roots query-idxs round-idx layer-idx <
    clength * scale
  and forced:
    generic-fri-round-forced-next-value roots challenges query-idxs
    round-layers round-idx layer-idx v
  shows
    generic-fri-sampled-assignment-layers candidate-table roots challenges
    final-value query-idxs round-layers ! Suc layer-idx !
    fri-evidence-next-idx roots query-idxs round-idx layer-idx = v
proof -
  let ?next = fri-evidence-next-idx roots query-idxs round-idx layer-idx
  have suc-nonzero: Suc layer-idx ≠ 0
    by simp
  have layer-eq:
    generic-fri-sampled-assignment-layers candidate-table roots challenges

```

```

    final-value query-idxs round-layers ! Suc layer-idx =
    generic-fri-sampled-assignment-layer candidate-table roots challenges
    final-value query-idxs round-layers (Suc layer-idx)
  by (rule generic-fri-sampled-assignment-layers-nth)
    (use layer-bound in simp)
have constraint:
  generic-fri-sampled-layer-value-constraint candidate-table roots
  challenges final-value query-idxs round-layers (Suc layer-idx)
  ?next v
using layer-bound round-bound forced
unfolding generic-fri-sampled-layer-value-constraint-def
by auto
have unique:
   $\bigwedge v'. \text{generic-fri-sampled-layer-value-constraint candidate-table roots}$ 
  challenges final-value query-idxs round-layers (Suc layer-idx)
  ?next v'  $\implies v' = v$ 
proof -
  fix v'
  assume c:
    generic-fri-sampled-layer-value-constraint candidate-table roots
    challenges final-value query-idxs round-layers (Suc layer-idx)
    ?next v'
  have pred-or-current:
    ( $\exists$  source-round.
      source-round < length query-idxs  $\wedge$ 
      generic-fri-round-forced-next-value roots challenges query-idxs
      round-layers source-round layer-idx v'  $\wedge$ 
      fri-evidence-next-idx roots query-idxs source-round layer-idx =
      ?next)  $\vee$ 
    ( $\exists$  target-round xp xp-path xn xn-path.
      target-round < length query-idxs  $\wedge$ 
      Suc layer-idx < length challenges  $\wedge$ 
      fri-layer-step-evidence
      (roots ! Suc layer-idx)
      (challenges ! Suc layer-idx)
      (fri-evidence-layer-len roots (Suc layer-idx))
      (fri-evidence-layer-idx roots query-idxs target-round
      (Suc layer-idx))
      ( $2 \wedge$  Suc layer-idx)
      (fri-sibling-index (fri-evidence-layer-len roots (Suc layer-idx))
      (fri-evidence-layer-idx roots query-idxs target-round
      (Suc layer-idx)))
      xp xp-path xn xn-path
      (fri-evidence-next-idx roots query-idxs target-round
      (Suc layer-idx))
      (fri-evidence-next-value roots challenges query-idxs target-round
      (Suc layer-idx) xp xn)
      (round-layers ! target-round ! Suc layer-idx)  $\wedge$ 
      ((?next = fri-evidence-layer-idx roots query-idxs target-round

```

```

      (Suc layer-idx) ∧ v' = xp) ∨
    (?next = fri-sibling-index
      (fri-evidence-layer-len roots (Suc layer-idx))
      (fri-evidence-layer-idx roots query-idxs target-round
        (Suc layer-idx)) ∧
      v' = xn)))
  using c layer-bound
  unfolding generic-fri-sampled-layer-value-constraint-def
  by auto
then show v' = v
proof
  assume
    ∃ source-round.
      source-round < length query-idxs ∧
      generic-fri-round-forced-next-value roots challenges query-idxs
        round-layers source-round layer-idx v' ∧
      fri-evidence-next-idx roots query-idxs source-round layer-idx =
        ?next
  then obtain source-round where source-bound:
    source-round < length query-idxs
  and forced':
    generic-fri-round-forced-next-value roots challenges query-idxs
      round-layers source-round layer-idx v'
  and same-idx:
    fri-evidence-next-idx roots query-idxs source-round layer-idx =
      ?next
  by blast
  have v = v'
  proof -
    have layer-bound0: layer-idx < length challenges
    using layer-bound by simp
    show ?thesis
    by (rule generic-fri-no-next-value-conflict-unique
      [OF generic-fri-no-assignment-conflict-no-next[OF no-conflict]
        round-bound source-bound layer-bound0 - forced forced'])
      (use same-idx in simp)
  qed
  then show ?thesis by simp
next
  assume
    ∃ target-round xp xp-path xn xn-path.
      target-round < length query-idxs ∧
      Suc layer-idx < length challenges ∧
      fri-layer-step-evidence
        (roots ! Suc layer-idx)
        (challenges ! Suc layer-idx)
        (fri-evidence-layer-len roots (Suc layer-idx))
        (fri-evidence-layer-idx roots query-idxs target-round
          (Suc layer-idx))

```

```

    (2  $\wedge$  Suc layer-idx)
    (fri-sibling-index (fri-evidence-layer-len roots (Suc layer-idx))
      (fri-evidence-layer-idx roots query-idxs target-round
        (Suc layer-idx)))
    xp xp-path xn xn-path
    (fri-evidence-next-idx roots query-idxs target-round
      (Suc layer-idx))
    (fri-evidence-next-value roots challenges query-idxs target-round
      (Suc layer-idx) xp xn)
    (round-layers ! target-round ! Suc layer-idx)  $\wedge$ 
    ((?next = fri-evidence-layer-idx roots query-idxs target-round
      (Suc layer-idx)  $\wedge$  v' = xp)  $\vee$ 
    (?next = fri-sibling-index
      (fri-evidence-layer-len roots (Suc layer-idx))
      (fri-evidence-layer-idx roots query-idxs target-round
        (Suc layer-idx))  $\wedge$ 
      v' = xn))
  then obtain target-round xp xp-path xn xn-path where target-bound:
    target-round < length query-idxs
  and step:
    fri-layer-step-evidence
    (roots ! Suc layer-idx)
    (challenges ! Suc layer-idx)
    (fri-evidence-layer-len roots (Suc layer-idx))
    (fri-evidence-layer-idx roots query-idxs target-round
      (Suc layer-idx))
    (2  $\wedge$  Suc layer-idx)
    (fri-sibling-index (fri-evidence-layer-len roots (Suc layer-idx))
      (fri-evidence-layer-idx roots query-idxs target-round
        (Suc layer-idx)))
    xp xp-path xn xn-path
    (fri-evidence-next-idx roots query-idxs target-round
      (Suc layer-idx))
    (fri-evidence-next-value roots challenges query-idxs target-round
      (Suc layer-idx) xp xn)
    (round-layers ! target-round ! Suc layer-idx)
  and value-case:
    (?next = fri-evidence-layer-idx roots query-idxs target-round
      (Suc layer-idx)  $\wedge$  v' = xp)  $\vee$ 
    (?next = fri-sibling-index
      (fri-evidence-layer-len roots (Suc layer-idx))
      (fri-evidence-layer-idx roots query-idxs target-round
        (Suc layer-idx))  $\wedge$ 
      v' = xn)
  by blast
from value-case show ?thesis
proof
  assume raw-case:
    ?next = fri-evidence-layer-idx roots query-idxs target-round

```

```

      (Suc layer-idx)  $\wedge$   $v' = xp$ 
have  $v = xp$ 
  by (rule generic-fri-no-successor-opening-conflict-matches-left
    [OF generic-fri-no-assignment-conflict-no-successor
      [OF no-conflict] round-bound target-bound layer-bound
      forced step])
    (use raw-case in simp)
then show ?thesis
  using raw-case by simp
next
assume sibling-case:
  ?next = fri-sibling-index
    (fri-evidence-layer-len roots (Suc layer-idx))
    (fri-evidence-layer-idx roots query-idxs target-round
      (Suc layer-idx))  $\wedge$ 
     $v' = xn$ 
have  $v = xn$ 
  by (rule generic-fri-no-successor-opening-conflict-matches-right
    [OF generic-fri-no-assignment-conflict-no-successor
      [OF no-conflict] round-bound target-bound layer-bound
      forced step])
    (use sibling-case in simp)
then show ?thesis
  using sibling-case by simp
qed
qed
qed
have nth-next:
  generic-fri-sampled-assignment-layer candidate-table roots challenges
    final-value query-idxs round-layers (Suc layer-idx) ! ?next = v
  by (rule generic-fri-sampled-assignment-layer-nth-unique
    [OF next-bound suc-nonzero constraint unique])
show ?thesis
  using nth-next unfolding layer-eq by simp
qed

lemma accepted-fri-opening-transcript-query-idx-bound:
assumes
  accepted-fri-opening-transcript s out trace-roots trace-bs
  trace-final dg composition-roots composition-bs composition-final
  query-idxs trace-round-layers composition-round-layers
and  $idx \in \text{set query-idxs}$ 
shows  $idx < \text{clength} * \text{scale}$ 
using assms
unfolding accepted-fri-opening-transcript-def by blast

lemma trace-fri-partial-candidate-opening-evidence-query-idx-bound:
assumes
  trace-fri-partial-candidate-opening-evidence s out trace-roots

```

```

    trace-bs trace-final dg composition-roots composition-bs
    composition-final fri-query-idxs trace-round-layers
    composition-round-layers fr candidate-query-idxs trace-openings
    trace-table
  and  $idx \in \text{set } \text{fri-query-idxs}$ 
  shows  $idx < \text{clength} * \text{scale}$ 
proof -
  have fri-openings:
    accepted-fri-opening-transcript s out trace-roots trace-bs
    trace-final dg composition-roots composition-bs composition-final
    fri-query-idxs trace-round-layers composition-round-layers
  by (rule trace-fri-partial-candidate-opening-evidenceD(1)[OF assms(1)])
  show ?thesis
  by (rule accepted-fri-opening-transcript-query-idx-bound
      [OF fri-openings assms(2)])
qed

```

lemma *composition-fri-partial-candidate-opening-evidence-query-idx-bound:*

```

  assumes
    composition-fri-partial-candidate-opening-evidence s out trace-roots
    trace-bs trace-final fri-dg opening-composition-roots composition-bs
    composition-final fri-query-idxs trace-round-layers
    composition-round-layers fr f-fri-roots f-final as dg
    composition-fri-roots final trace-query-idxs trace-openings
    composition-query-idxs composition-openings trace-table
    composition-table
  and  $idx \in \text{set } \text{fri-query-idxs}$ 
  shows  $idx < \text{clength} * \text{scale}$ 
proof -
  have fri-openings:
    accepted-fri-opening-transcript s out trace-roots trace-bs
    trace-final fri-dg opening-composition-roots composition-bs
    composition-final fri-query-idxs trace-round-layers
    composition-round-layers
  by (rule composition-fri-partial-candidate-opening-evidenceD(1)
      [OF assms(1)])
  show ?thesis
  by (rule accepted-fri-opening-transcript-query-idx-bound
      [OF fri-openings assms(2)])
qed

```

lemma *fri-layer-indices-nth-bound:*

```

  assumes query-bound:  $\text{query-idx} < \text{len}$ 
  and layer-bound:  $\text{layer-idx} < n$ 
  and eval-power:  $\text{len} = 2 \wedge N$ 
  and rounds-bound:  $n \leq N$ 
  shows
    fri-layer-indices n query-idx len ! layer-idx <
    fri-layer-lengths n len ! layer-idx

```

```

    using layer-bound query-bound rounds-bound eval-power
  proof (induction layer-idx arbitrary: n query-idx len N)
    case 0
    then show ?case
      by (cases n) simp-all
  next
    case (Suc layer-idx)
    then obtain n' where n-eq: n = Suc n'
      by (cases n) auto
    have layer-bound': layer-idx < n'
      using Suc.premis(1) unfolding n-eq by simp
    have len-even: 2 dvd len
      using Suc.premis(3,4) n-eq Suc.premis(1)
      by (cases N) simp-all
    have len-half-power:
       $\exists N'. \text{len div } 2 = 2 \wedge N' \wedge n' \leq N'$ 
    proof (cases N)
      case 0
      then show ?thesis
        using Suc.premis(3) n-eq by simp
    next
      case (Suc N')
      have len div 2 =  $2 \wedge N'$ 
        using Suc.premis(4) unfolding Suc by simp
      moreover have  $n' \leq N'$ 
        using Suc.premis(3) unfolding n-eq Suc by simp
      ultimately show ?thesis by blast
    qed
    then obtain N' where len-half-eq: len div 2 =  $2 \wedge N'$ 
      and rounds-bound':  $n' \leq N'$ 
      by blast
    have query-bound':
      query-idx mod (len div 2) < len div 2
      using len-half-eq by simp
    have ih:
      fri-layer-indices n' (query-idx mod (len div 2)) (len div 2) !
      layer-idx <
      fri-layer-lengths n' (len div 2) ! layer-idx
      by (rule Suc.IH[OF layer-bound' query-bound' rounds-bound'
        len-half-eq])
    show ?case
      using ih unfolding n-eq by simp
  qed

lemma fri-evidence-layer-idx-bound-from-query-bound:
  assumes partial:
    generic-fri-partial-evidence low-degree candidate-table degree-bound
    roots challenges final-value query-idxs round-layers
  and query-bound: query-idx < clength * scale

```


and *round-idx-eq*: $query_idx = query_idxs \mid round_idx$
and *layer-bound*: $layer_idx < length\ challenges$
and *eval-power*: $clength * scale = 2^N$
and *rounds-bound*: $length\ challenges \leq N$
shows
 $fri_evidence_layer_idx\ roots\ query_idxs\ round_idx\ layer_idx <$
 $fri_evidence_layer_len\ roots\ layer_idx$
proof –
have *roots-len*: $length\ challenges = length\ roots$
by (rule *generic-fri-partial-evidence-shapes*(2)[*OF partial*])
show ?thesis
unfolding *fri-evidence-layer-idx-def* *fri-evidence-layer-len-def*
using *fri-layer-indices-nth-bound*
[*OF query-bound*, *of layer-idx length roots N*]
 $layer_bound\ roots_len\ eval_power\ rounds_bound\ round_idx_eq$
by *simp*
qed

lemma *fri-evidence-layer-len-le-eval-domain*:
assumes *partial*:
 $generic_fri_partial_evidence\ low_degree\ candidate_table\ degree_bound$
 $roots\ challenges\ final_value\ query_idxs\ round_layers$
and *layer-bound*: $layer_idx < length\ challenges$
and *eval-power*: $clength * scale = 2^N$
shows $fri_evidence_layer_len\ roots\ layer_idx \leq clength * scale$
proof –
have *roots-len*: $length\ challenges = length\ roots$
by (rule *generic-fri-partial-evidence-shapes*(2)[*OF partial*])
have *roots-bound*: $layer_idx < length\ roots$
using *layer-bound roots-len* **by** *simp*
have *len-eq*:
 $fri_evidence_layer_len\ roots\ layer_idx =$
 $(clength * scale) \div 2^{layer_idx}$
unfolding *fri-evidence-layer-len-def*
by (rule *fri-layer-lengths-nth-div*[*OF roots-bound*])
show ?thesis
unfolding *len-eq* **by** *simp*
qed

lemma *fri-evidence-sibling-index-bound-from-raw*:
assumes *raw-bound*:
 $fri_evidence_layer_idx\ roots\ query_idxs\ round_idx\ layer_idx <$
 $fri_evidence_layer_len\ roots\ layer_idx$
and *len-bound*:
 $fri_evidence_layer_len\ roots\ layer_idx \leq clength * scale$
shows
 $fri_sibling_index\ (fri_evidence_layer_len\ roots\ layer_idx)$
 $(fri_evidence_layer_idx\ roots\ query_idxs\ round_idx\ layer_idx) <$
 $clength * scale$

```

proof –
  have len-pos:  $0 < \text{fri-evidence-layer-len roots layer-idx}$ 
    using raw-bound by simp
  have sibling-lt:
     $\text{fri-sibling-index } (\text{fri-evidence-layer-len roots layer-idx})$ 
       $(\text{fri-evidence-layer-idx roots query-idxs round-idx layer-idx}) <$ 
         $\text{fri-evidence-layer-len roots layer-idx}$ 
    unfolding fri-sibling-index-def
    using len-pos by simp
  then show ?thesis
    using len-bound by linarith
qed

lemma fri-evidence-next-idx-bound-from-raw:
  assumes raw-bound:
     $\text{fri-evidence-layer-idx roots query-idxs round-idx layer-idx} <$ 
       $\text{fri-evidence-layer-len roots layer-idx}$ 
  and len-bound:
     $\text{fri-evidence-layer-len roots layer-idx} \leq \text{clength} * \text{scale}$ 
  shows
     $\text{fri-evidence-next-idx roots query-idxs round-idx layer-idx} <$ 
       $\text{clength} * \text{scale}$ 
proof –
  have len-pos:  $0 < \text{fri-evidence-layer-len roots layer-idx}$ 
    using raw-bound by simp
  have half-le:  $\text{fri-evidence-layer-len roots layer-idx div } 2 \leq$ 
     $\text{clength} * \text{scale}$ 
    using len-bound by simp
  show ?thesis
proof (cases  $\text{fri-evidence-layer-len roots layer-idx div } 2 = 0$ )
  case True
    then show ?thesis
      unfolding fri-evidence-next-idx-def
      using raw-bound len-bound by simp
  next
  case False
    then have half-pos:
       $0 < \text{fri-evidence-layer-len roots layer-idx div } 2$ 
      by simp
    have mod-lt:
       $\text{fri-evidence-layer-idx roots query-idxs round-idx layer-idx mod}$ 
         $(\text{fri-evidence-layer-len roots layer-idx div } 2) <$ 
           $\text{fri-evidence-layer-len roots layer-idx div } 2$ 
      by (rule mod-less-divisor[OF half-pos])
    then show ?thesis
      unfolding fri-evidence-next-idx-def
      using half-le by linarith
qed
qed

```

lemma *trace-fri-reachable-missing-sampled-layer-chain-None*[simp]:
 $\neg$ *trace-fri-reachable-missing-sampled-layer-chain* *s* *None*
unfolding *trace-fri-reachable-missing-sampled-layer-chain-def*
trace-fri-bad-with-reachable-partial-candidate-def
accepted-with-partial-trace-openings-def
by *simp*

lemma *composition-fri-reachable-missing-sampled-layer-chain-None*[simp]:
 $\neg$ *composition-fri-reachable-missing-sampled-layer-chain* *s* *None*
unfolding *composition-fri-reachable-missing-sampled-layer-chain-def*
composition-fri-bad-with-reachable-partial-candidate-def
accepted-with-partial-initial-openings-def
accepted-with-partial-trace-openings-def
by *simp*

lemma *trace-fri-sampled-layer-assignment-obstruction-None*[simp]:
 $\neg$ *trace-fri-sampled-layer-assignment-obstruction* *s* *None*
unfolding *trace-fri-sampled-layer-assignment-obstruction-def*
trace-fri-partial-candidate-opening-evidence-def
accepted-fri-opening-transcript-def
by *simp*

lemma *composition-fri-sampled-layer-assignment-obstruction-None*[simp]:
 $\neg$ *composition-fri-sampled-layer-assignment-obstruction* *s* *None*
unfolding *composition-fri-sampled-layer-assignment-obstruction-def*
composition-fri-partial-candidate-opening-evidence-def
accepted-fri-opening-transcript-def
by *simp*

lemma *trace-fri-sampled-assignment-conflict-None*[simp]:
 $\neg$ *trace-fri-sampled-assignment-conflict* *s* *None*
unfolding *trace-fri-sampled-assignment-conflict-def*
trace-fri-partial-candidate-opening-evidence-def
accepted-fri-opening-transcript-def
by *simp*

lemma *composition-fri-sampled-assignment-conflict-None*[simp]:
 $\neg$ *composition-fri-sampled-assignment-conflict* *s* *None*
unfolding *composition-fri-sampled-assignment-conflict-def*
composition-fri-partial-candidate-opening-evidence-def
accepted-fri-opening-transcript-def
by *simp*

lemma *wp-trace-fri-assignment-obstruction-bound-from-conflict*:
assumes *support-imp*:
 $\bigwedge out. out \in \text{set-dist } (\text{execute verify-monad } s) \implies$
 $\text{trace-fri-sampled-layer-assignment-obstruction } s \text{ out} \implies$
 $\text{trace-fri-sampled-assignment-conflict } s \text{ out}$

and *conflict-bound*:

$$\text{wp-event verify-monad } (\text{trace-fri-sampled-assignment-conflict } s) \ s \leq C$$
shows

$$\text{wp-event verify-monad } (\text{trace-fri-sampled-layer-assignment-obstruction } s) \ s \leq C$$
proof –
have *wp-event verify-monad*

$$(\text{trace-fri-sampled-layer-assignment-obstruction } s) \ s \leq \text{wp-event verify-monad } (\text{trace-fri-sampled-assignment-conflict } s) \ s$$
by (rule *wp-event-mono-on-support*) (use *support-imp* **in** *blast*)
also have $\dots \leq C$
by (rule *conflict-bound*)
finally show *?thesis* .
qed

lemma *wp-composition-fri-assignment-obstruction-bound-from-conflict*:
assumes *support-imp*:

$$\bigwedge \text{out. out} \in \text{set-dist } (\text{execute verify-monad } s) \implies \text{composition-fri-sampled-layer-assignment-obstruction } s \ \text{out} \implies \text{composition-fri-sampled-assignment-conflict } s \ \text{out}$$
and *conflict-bound*:

$$\text{wp-event verify-monad } (\text{composition-fri-sampled-assignment-conflict } s) \ s \leq C$$
shows

$$\text{wp-event verify-monad } (\text{composition-fri-sampled-layer-assignment-obstruction } s) \ s \leq C$$
proof –
have *wp-event verify-monad*

$$(\text{composition-fri-sampled-layer-assignment-obstruction } s) \ s \leq \text{wp-event verify-monad } (\text{composition-fri-sampled-assignment-conflict } s) \ s$$
by (rule *wp-event-mono-on-support*) (use *support-imp* **in** *blast*)
also have $\dots \leq C$
by (rule *conflict-bound*)
finally show *?thesis* .
qed

lemma *trace-fri-reachable-missing-sampled-imp-assignment-obstruction-on-support*:
assumes *support*:

$$\text{out} \in \text{set-dist } (\text{execute verify-monad } s)$$
and *missing*:

$$\text{trace-fri-reachable-missing-sampled-layer-chain } s \ \text{out}$$
shows $\text{trace-fri-sampled-layer-assignment-obstruction } s \ \text{out}$
proof (cases *out*)
case *None*
then show *?thesis*
using *missing*
unfolding *trace-fri-reachable-missing-sampled-layer-chain-def*

```

    trace-fri-bad-with-reachable-partial-candidate-def
    accepted-with-partial-trace-openings-def
  by simp
next
case (Some pair)
then obtain result final-state where pair: pair = (result, final-state)
  by (cases pair) simp
have outcome:
  Some (result, final-state) ∈ set-dist (execute verify-monad s)
  using support Some pair by simp
show ?thesis
proof (rule verify-monad-trace-fri-missing-sampled-extracts-opening-evidence
  [OF outcome])
show
  trace-fri-reachable-missing-sampled-layer-chain s
  (Some (result, final-state))
  using missing Some pair by simp
next
fix trace-roots trace-bs trace-final dg composition-roots
  composition-bs composition-final fri-query-idxs trace-round-layers
  composition-round-layers fr candidate-query-idxs trace-openings
  trace-table
assume evidence:
  trace-fri-partial-candidate-opening-evidence s
  (Some (result, final-state)) trace-roots trace-bs trace-final dg
  composition-roots composition-bs composition-final fri-query-idxs
  trace-round-layers composition-round-layers fr candidate-query-idxs
  trace-openings trace-table
assume no-chain:
   $\bigwedge$  doms layers.  $\neg$ 
  generic-fri-sampled-layer-chain-evidence trace-table-low-degree
  trace-table (clength - 1) trace-roots trace-bs trace-final
  fri-query-idxs trace-round-layers doms layers
have partial:
  generic-fri-partial-evidence trace-table-low-degree trace-table
  (clength - 1) trace-roots trace-bs trace-final fri-query-idxs
  trace-round-layers
  by (rule trace-fri-partial-candidate-opening-evidence-generic-partial
    [OF evidence])
have obstruction:
  generic-fri-sampled-layer-assignment-obstruction
  trace-table-low-degree trace-table (clength - 1) trace-roots
  trace-bs trace-final fri-query-idxs trace-round-layers
  unfolding generic-fri-sampled-layer-assignment-obstruction-def
  using partial no-chain by blast
show trace-fri-sampled-layer-assignment-obstruction s out
  unfolding trace-fri-sampled-layer-assignment-obstruction-def
  using evidence obstruction Some pair by blast
qed

```

qed

lemma *composition-fri-reachable-missing-sampled-imp-assignment-obstruction-on-support:*

assumes *support:*

out $\in$ *set-dist* (*execute verify-monad s*)

and *missing:*

composition-fri-reachable-missing-sampled-layer-chain s out

shows *composition-fri-sampled-layer-assignment-obstruction s out*

proof (*cases out*)

case *None*

then show *?thesis*

unfolding *composition-fri-sampled-layer-assignment-obstruction-def*

using *missing*

unfolding *composition-fri-reachable-missing-sampled-layer-chain-def*

composition-fri-bad-with-reachable-partial-candidate-def

accepted-with-partial-initial-openings-def

accepted-with-partial-trace-openings-def

by *simp*

next

case (*Some pair*)

then obtain *result final-state* **where** *pair*: *pair* = (*result*, *final-state*)

by (*cases pair*) *simp*

have *outcome:*

Some (result, final-state) $\in$ *set-dist* (*execute verify-monad s*)

using *support Some pair* **by** *simp*

show *?thesis*

proof (*rule*

verify-monad-composition-fri-missing-sampled-extracts-opening-evidence

[*OF outcome*])

show

composition-fri-reachable-missing-sampled-layer-chain s

(*Some (result, final-state)*)

using *missing Some pair* **by** *simp*

next

fix *trace-roots trace-bs trace-final fri-dg opening-composition-roots*

composition-bs composition-final fri-query-idxs trace-round-layers

composition-round-layers fr f-fri-roots f-final as dg

composition-fri-roots final trace-query-idxs trace-openings

composition-query-idxs composition-openings trace-table

composition-table

assume *evidence:*

composition-fri-partial-candidate-opening-evidence s

(*Some (result, final-state)*) *trace-roots trace-bs trace-final*

fri-dg opening-composition-roots composition-bs composition-final

fri-query-idxs trace-round-layers composition-round-layers fr

f-fri-roots f-final as dg composition-fri-roots final

trace-query-idxs trace-openings composition-query-idxs

composition-openings trace-table composition-table

assume *degree-bound: to-nat fri-dg* $\leq$ *maxDegree*

```

assume no-chain:
   $\bigwedge \text{doms layers. } \neg$ 
    generic-fri-sampled-layer-chain-evidence
      (composition-table-low-degree (to-nat fri-dg))
      composition-table (to-nat fri-dg) opening-composition-roots
      composition-bs composition-final fri-query-idxs
      composition-round-layers doms layers
have partial:
  generic-fri-partial-evidence
    (composition-table-low-degree (to-nat fri-dg)) composition-table
    (to-nat fri-dg) opening-composition-roots composition-bs
    composition-final fri-query-idxs composition-round-layers
  by (rule composition-fri-partial-candidate-opening-evidence-generic-partial
    [OF evidence])
have obstruction:
  generic-fri-sampled-layer-assignment-obstruction
    (composition-table-low-degree (to-nat fri-dg)) composition-table
    (to-nat fri-dg) opening-composition-roots composition-bs
    composition-final fri-query-idxs composition-round-layers
  unfolding generic-fri-sampled-layer-assignment-obstruction-def
  using partial no-chain by blast
show composition-fri-sampled-layer-assignment-obstruction s out
  unfolding composition-fri-sampled-layer-assignment-obstruction-def
  using evidence obstruction Some pair degree-bound by meson
qed
qed

lemma wp-trace-fri-reachable-missing-sampled-bound-from-assignment-obstruction:
assumes obstruction:
  wp-event verify-monad
  (trace-fri-sampled-layer-assignment-obstruction s)  $s \leq M$ 
shows
  wp-event verify-monad
  (trace-fri-reachable-missing-sampled-layer-chain s)  $s \leq M$ 
proof –
have wp-event verify-monad
  (trace-fri-reachable-missing-sampled-layer-chain s) s
   $\leq$  wp-event verify-monad
  (trace-fri-sampled-layer-assignment-obstruction s) s
by (rule wp-event-mono-on-support)
  (rule trace-fri-reachable-missing-sampled-imp-assignment-obstruction-on-support)
also have  $\dots \leq M$ 
by (rule obstruction)
finally show ?thesis .
qed

lemma wp-composition-fri-reachable-missing-sampled-bound-from-assignment-obstruction:
assumes obstruction:
  wp-event verify-monad

```

(composition-fri-sampled-layer-assignment-obstruction s) $s \leq M$
shows
 wp-event verify-monad
 (composition-fri-reachable-missing-sampled-layer-chain s) $s \leq M$
proof –
 have wp-event verify-monad
 (composition-fri-reachable-missing-sampled-layer-chain s) s
 $\leq$ wp-event verify-monad
 (composition-fri-sampled-layer-assignment-obstruction s) s
 by (rule wp-event-mono-on-support)
 (rule composition-fri-reachable-missing-sampled-imp-assignment-obstruction-on-support)
 also have ... $\leq M$
 by (rule obstruction)
 finally show ?thesis .
qed

lemma trace-fri-reachable-partial-candidate-reduction-from-sampled-and-assignment-obstruction:

assumes sampled-bound:
 wp-event verify-monad
 (trace-fri-bad-with-sampled-layer-chain s) $s \leq R$
and obstruction-bound:
 wp-event verify-monad
 (trace-fri-sampled-layer-assignment-obstruction s) $s \leq M$
and total-bound: $R + M \leq \text{trace-fri-error}$
shows trace-fri-reachable-partial-candidate-reduction-assumption s
proof (rule trace-fri-reachable-partial-candidate-reduction-from-sampled-and-missing)
show wp-event verify-monad
 (trace-fri-bad-with-sampled-layer-chain s) $s \leq R$
 by (rule sampled-bound)
next
show wp-event verify-monad
 (trace-fri-reachable-missing-sampled-layer-chain s) $s \leq M$
 by (rule wp-trace-fri-reachable-missing-sampled-bound-from-assignment-obstruction
 [OF obstruction-bound])
next
show $R + M \leq \text{trace-fri-error}$
 by (rule total-bound)
qed

lemma composition-fri-reachable-partial-candidate-reduction-from-sampled-and-assignment-obstruction:

assumes sampled-bound:
 wp-event verify-monad
 (composition-fri-bad-with-sampled-layer-chain s) $s \leq R$
and obstruction-bound:
 wp-event verify-monad
 (composition-fri-sampled-layer-assignment-obstruction s) $s \leq M$
and total-bound: $R + M \leq \text{composition-fri-error}$
shows composition-fri-reachable-partial-candidate-reduction-assumption s
proof (rule

composition-fri-reachable-partial-candidate-reduction-from-sampled-and-missing)
show *wp-event verify-monad*
 (*composition-fri-bad-with-sampled-layer-chain s*) $s \leq R$
 by (rule *sampled-bound*)
next
show *wp-event verify-monad*
 (*composition-fri-reachable-missing-sampled-layer-chain s*) $s \leq M$
 by (rule
 wp-composition-fri-reachable-missing-sampled-bound-from-assignment-obstruction
 [*OF obstruction-bound*])
next
show $R + M \leq \text{composition-fri-error}$
 by (rule *total-bound*)
qed

lemma *trace-fri-reachable-partial-candidate-reduction-from-sampled-and-assignment-conflict:*

assumes *sampled-bound:*
 wp-event verify-monad
 (*trace-fri-bad-with-sampled-layer-chain s*) $s \leq R$
and *support-imp:*
 $\bigwedge \text{out. out} \in \text{set-dist (execute verify-monad s)} \implies$
 trace-fri-sampled-layer-assignment-obstruction s out $\implies$
 trace-fri-sampled-assignment-conflict s out
and *conflict-bound:*
 wp-event verify-monad
 (*trace-fri-sampled-assignment-conflict s*) $s \leq M$
and *total-bound:* $R + M \leq \text{trace-fri-error}$
shows *trace-fri-reachable-partial-candidate-reduction-assumption s*
proof (rule
 trace-fri-reachable-partial-candidate-reduction-from-sampled-and-assignment-obstruction)
show *wp-event verify-monad*
 (*trace-fri-bad-with-sampled-layer-chain s*) $s \leq R$
 by (rule *sampled-bound*)
next
show *wp-event verify-monad*
 (*trace-fri-sampled-layer-assignment-obstruction s*) $s \leq M$
 by (rule *wp-trace-fri-assignment-obstruction-bound-from-conflict*
 [*OF support-imp conflict-bound*])
next
show $R + M \leq \text{trace-fri-error}$
 by (rule *total-bound*)
qed

lemma *composition-fri-reachable-partial-candidate-reduction-from-sampled-and-assignment-conflict:*

assumes *sampled-bound:*
 wp-event verify-monad
 (*composition-fri-bad-with-sampled-layer-chain s*) $s \leq R$
and *support-imp:*
 $\bigwedge \text{out. out} \in \text{set-dist (execute verify-monad s)} \implies$

$\text{composition-fri-sampled-layer-assignment-obstruction } s \text{ out} \implies$
 $\text{composition-fri-sampled-assignment-conflict } s \text{ out}$
and *conflict-bound*:
 $\text{wp-event verify-monad}$
 $(\text{composition-fri-sampled-assignment-conflict } s) \ s \leq M$
and *total-bound*: $R + M \leq \text{composition-fri-error}$
shows *composition-fri-reachable-partial-candidate-reduction-assumption* s
proof (rule
 $\text{composition-fri-reachable-partial-candidate-reduction-from-sampled-and-assignment-obstruction}$)
show $\text{wp-event verify-monad}$
 $(\text{composition-fri-bad-with-sampled-layer-chain } s) \ s \leq R$
by (rule *sampled-bound*)
next
show $\text{wp-event verify-monad}$
 $(\text{composition-fri-sampled-layer-assignment-obstruction } s) \ s \leq M$
by (rule *wp-composition-fri-assignment-obstruction-bound-from-conflict*
 $[\text{OF support-imp conflict-bound}]$)
next
show $R + M \leq \text{composition-fri-error}$
by (rule *total-bound*)
qed

lemma *trace-fri-bad-with-partial-candidates-bound-from-sampled-and-assignment-obstruction*:

fixes $R \ M :: \text{prob}$

assumes *sampled-bound*:

$\bigwedge \text{data attacker-state.}$

$\text{Some } (\text{data}, \text{attacker-state}) \in$
 $\text{set-dist } (\text{execute } (\text{checked-staged-transcript-program } A))$
 $\text{adversary-initial-state} \implies$

$\text{wp-event verify-monad}$
 $(\text{trace-fri-bad-with-sampled-layer-chain}$
 $(\text{verifier-state-from-adversary attacker-state}$
 $(\text{staged-proof-transcript data})))$
 $(\text{verifier-state-from-adversary attacker-state}$
 $(\text{staged-proof-transcript data})) \leq R$

and *obstruction-bound*:

$\bigwedge \text{data attacker-state.}$

$\text{Some } (\text{data}, \text{attacker-state}) \in$
 $\text{set-dist } (\text{execute } (\text{checked-staged-transcript-program } A))$
 $\text{adversary-initial-state} \implies$

$\text{wp-event verify-monad}$
 $(\text{trace-fri-sampled-layer-assignment-obstruction}$
 $(\text{verifier-state-from-adversary attacker-state}$
 $(\text{staged-proof-transcript data})))$
 $(\text{verifier-state-from-adversary attacker-state}$
 $(\text{staged-proof-transcript data})) \leq M$

shows

$\text{wp-event } (\text{checked-staged-security-experiment-with-data-state } A)$
 $(\text{staged-security-with-data-state-verifier-event}$

```

    trace-fri-bad-with-partial-candidates)
  adversary-initial-state  $\leq R + M$ 
proof (rule trace-fri-bad-with-partial-candidates-bound-from-sampled-and-missing)
  fix data attacker-state
  assume builder:
    Some (data, attacker-state)  $\in$ 
      set-dist (execute (checked-staged-transcript-program A)
        adversary-initial-state)
  show wp-event verify-monad
    (trace-fri-bad-with-sampled-layer-chain
      (verifier-state-from-adversary attacker-state
        (staged-proof-transcript data)))
    (verifier-state-from-adversary attacker-state
      (staged-proof-transcript data))  $\leq R$ 
  by (rule sampled-bound[OF builder])
next
  fix data attacker-state
  assume builder:
    Some (data, attacker-state)  $\in$ 
      set-dist (execute (checked-staged-transcript-program A)
        adversary-initial-state)
  show wp-event verify-monad
    (trace-fri-reachable-missing-sampled-layer-chain
      (verifier-state-from-adversary attacker-state
        (staged-proof-transcript data)))
    (verifier-state-from-adversary attacker-state
      (staged-proof-transcript data))  $\leq M$ 
  by (rule
    wp-trace-fri-reachable-missing-sampled-bound-from-assignment-obstruction)
    (rule obstruction-bound[OF builder])
qed

lemma composition-fri-bad-with-partial-candidates-bound-from-sampled-and-assignment-obstruction:
  fixes R M :: prob
  assumes sampled-bound:
     $\bigwedge$  data attacker-state.
    Some (data, attacker-state)  $\in$ 
      set-dist (execute (checked-staged-transcript-program A)
        adversary-initial-state)  $\implies$ 
    wp-event verify-monad
      (composition-fri-bad-with-sampled-layer-chain
        (verifier-state-from-adversary attacker-state
          (staged-proof-transcript data)))
      (verifier-state-from-adversary attacker-state
        (staged-proof-transcript data))  $\leq R$ 
  and obstruction-bound:
     $\bigwedge$  data attacker-state.
    Some (data, attacker-state)  $\in$ 
      set-dist (execute (checked-staged-transcript-program A)

```

$\text{adversary-initial-state}) \implies$
 $\text{wp-event verify-monad}$
 $(\text{composition-fri-sampled-layer-assignment-obstruction}$
 $(\text{verifier-state-from-adversary attacker-state}$
 $(\text{staged-proof-transcript data})))$
 $(\text{verifier-state-from-adversary attacker-state}$
 $(\text{staged-proof-transcript data})) \leq M$
shows
 $\text{wp-event } (\text{checked-staged-security-experiment-with-data-state } A)$
 $(\text{staged-security-with-data-state-verifier-event}$
 $\text{composition-fri-bad-with-partial-candidates})$
 $\text{adversary-initial-state} \leq R + M$
proof (rule
 $\text{composition-fri-bad-with-partial-candidates-bound-from-sampled-and-missing})$
fix data attacker-state
assume builder:
 $\text{Some } (data, \text{attacker-state}) \in$
 $\text{set-dist } (\text{execute } (\text{checked-staged-transcript-program } A)$
 $\text{adversary-initial-state})$
show wp-event verify-monad
 $(\text{composition-fri-bad-with-sampled-layer-chain}$
 $(\text{verifier-state-from-adversary attacker-state}$
 $(\text{staged-proof-transcript data})))$
 $(\text{verifier-state-from-adversary attacker-state}$
 $(\text{staged-proof-transcript data})) \leq R$
by (rule sampled-bound[OF builder])
next
fix data attacker-state
assume builder:
 $\text{Some } (data, \text{attacker-state}) \in$
 $\text{set-dist } (\text{execute } (\text{checked-staged-transcript-program } A)$
 $\text{adversary-initial-state})$
show wp-event verify-monad
 $(\text{composition-fri-reachable-missing-sampled-layer-chain}$
 $(\text{verifier-state-from-adversary attacker-state}$
 $(\text{staged-proof-transcript data})))$
 $(\text{verifier-state-from-adversary attacker-state}$
 $(\text{staged-proof-transcript data})) \leq M$
by (rule
 $\text{wp-composition-fri-reachable-missing-sampled-bound-from-assignment-obstruction})$
 $(\text{rule obstruction-bound[OF builder]})$
qed

lemma trace-fri-bad-with-partial-candidates-bound-from-sampled-and-assignment-conflict:
fixes $R \ M :: \text{prob}$
assumes sampled-bound:
 $\bigwedge \text{data attacker-state.}$
 $\text{Some } (data, \text{attacker-state}) \in$
 $\text{set-dist } (\text{execute } (\text{checked-staged-transcript-program } A)$

$\text{adversary-initial-state}) \implies$
 $\text{wp-event verify-monad}$
 $(\text{trace-fri-bad-with-sampled-layer-chain}$
 $(\text{verifier-state-from-adversary attacker-state}$
 $(\text{staged-proof-transcript data})))$
 $(\text{verifier-state-from-adversary attacker-state}$
 $(\text{staged-proof-transcript data})) \leq R$
and support-imp:
 $\wedge \text{data attacker-state out.}$
 $\text{Some (data, attacker-state)} \in$
 $\text{set-dist (execute (checked-staged-transcript-program A)}$
 $\text{adversary-initial-state}) \implies}$
 $\text{out} \in \text{set-dist}$
 $(\text{execute verify-monad}$
 $(\text{verifier-state-from-adversary attacker-state}$
 $(\text{staged-proof-transcript data}))) \implies$
 $\text{trace-fri-sampled-layer-assignment-obstruction}$
 $(\text{verifier-state-from-adversary attacker-state}$
 $(\text{staged-proof-transcript data})) \text{ out} \implies$
 $\text{trace-fri-sampled-assignment-conflict}$
 $(\text{verifier-state-from-adversary attacker-state}$
 $(\text{staged-proof-transcript data})) \text{ out}$
and conflict-bound:
 $\wedge \text{data attacker-state.}$
 $\text{Some (data, attacker-state)} \in$
 $\text{set-dist (execute (checked-staged-transcript-program A)}$
 $\text{adversary-initial-state}) \implies}$
 $\text{wp-event verify-monad}$
 $(\text{trace-fri-sampled-assignment-conflict}$
 $(\text{verifier-state-from-adversary attacker-state}$
 $(\text{staged-proof-transcript data})))$
 $(\text{verifier-state-from-adversary attacker-state}$
 $(\text{staged-proof-transcript data})) \leq M$
shows
 $\text{wp-event (checked-staged-security-experiment-with-data-state A)}$
 $(\text{staged-security-with-data-state-verifier-event}$
 $\text{trace-fri-bad-with-partial-candidates})$
 $\text{adversary-initial-state} \leq R + M$
proof (rule
 $\text{trace-fri-bad-with-partial-candidates-bound-from-sampled-and-assignment-obstruction})$
fix $\text{data attacker-state}$
assume builder:
 $\text{Some (data, attacker-state)} \in$
 $\text{set-dist (execute (checked-staged-transcript-program A)}$
 $\text{adversary-initial-state})}$
show $\text{wp-event verify-monad}$
 $(\text{trace-fri-bad-with-sampled-layer-chain}$
 $(\text{verifier-state-from-adversary attacker-state}$
 $(\text{staged-proof-transcript data})))$

```

    (verifier-state-from-adversary attacker-state
     (staged-proof-transcript data)) ≤ R
  by (rule sampled-bound[OF builder])
next
fix data attacker-state
assume builder:
  Some (data, attacker-state) ∈
    set-dist (execute (checked-staged-transcript-program A)
                  adversary-initial-state)
show wp-event verify-monad
  (trace-fri-sampled-layer-assignment-obstruction
   (verifier-state-from-adversary attacker-state
    (staged-proof-transcript data)))
  (verifier-state-from-adversary attacker-state
   (staged-proof-transcript data)) ≤ M
by (rule wp-trace-fri-assignment-obstruction-bound-from-conflict)
  (use builder support-imp conflict-bound in blast)+
qed

lemma composition-fri-bad-with-partial-candidates-bound-from-sampled-and-assignment-conflict:
  fixes R M :: prob
  assumes sampled-bound:
    ∧data attacker-state.
      Some (data, attacker-state) ∈
        set-dist (execute (checked-staged-transcript-program A)
                          adversary-initial-state) ⇒
      wp-event verify-monad
        (composition-fri-bad-with-sampled-layer-chain
         (verifier-state-from-adversary attacker-state
          (staged-proof-transcript data)))
        (verifier-state-from-adversary attacker-state
         (staged-proof-transcript data)) ≤ R
  and support-imp:
    ∧data attacker-state out.
      Some (data, attacker-state) ∈
        set-dist (execute (checked-staged-transcript-program A)
                          adversary-initial-state) ⇒
      out ∈ set-dist
        (execute verify-monad
         (verifier-state-from-adversary attacker-state
          (staged-proof-transcript data))) ⇒
      composition-fri-sampled-layer-assignment-obstruction
        (verifier-state-from-adversary attacker-state
         (staged-proof-transcript data)) out ⇒
      composition-fri-sampled-assignment-conflict
        (verifier-state-from-adversary attacker-state
         (staged-proof-transcript data)) out
  and conflict-bound:
    ∧data attacker-state.

```

```

    Some (data, attacker-state) ∈
      set-dist (execute (checked-staged-transcript-program A)
        adversary-initial-state) ⇒
wp-event verify-monad
  (composition-fri-sampled-assignment-conflict
    (verifier-state-from-adversary attacker-state
      (staged-proof-transcript data)))
  (verifier-state-from-adversary attacker-state
    (staged-proof-transcript data)) ≤ M
shows
  wp-event (checked-staged-security-experiment-with-data-state A)
    (staged-security-with-data-state-verifier-event
      composition-fri-bad-with-partial-candidates)
    adversary-initial-state ≤ R + M
proof (rule
  composition-fri-bad-with-partial-candidates-bound-from-sampled-and-assignment-obstruction)
fix data attacker-state
assume builder:
  Some (data, attacker-state) ∈
    set-dist (execute (checked-staged-transcript-program A)
      adversary-initial-state)
show wp-event verify-monad
  (composition-fri-bad-with-sampled-layer-chain
    (verifier-state-from-adversary attacker-state
      (staged-proof-transcript data)))
  (verifier-state-from-adversary attacker-state
    (staged-proof-transcript data)) ≤ R
by (rule sampled-bound[OF builder])
next
fix data attacker-state
assume builder:
  Some (data, attacker-state) ∈
    set-dist (execute (checked-staged-transcript-program A)
      adversary-initial-state)
show wp-event verify-monad
  (composition-fri-sampled-layer-assignment-obstruction
    (verifier-state-from-adversary attacker-state
      (staged-proof-transcript data)))
  (verifier-state-from-adversary attacker-state
    (staged-proof-transcript data)) ≤ M
by (rule wp-composition-fri-assignment-obstruction-bound-from-conflict)
  (use builder support-imp conflict-bound in blast)+
qed

end

end

```

```

theory Soundness-FRI-Sampled-Assignment-Chain
  imports Soundness-FRI-Sampled-Obstructions
begin

```

Construction of sampled FRI layer-chain witnesses from verifier-local sampled assignment layers. This theory is kept separate from the sampled obstruction-routing layer to keep that layer below the project size threshold.

```

context soundness
begin

```

```

lemma ceil-log-le-power:
  assumes le-power:  $m \leq (2::nat) \wedge N$ 
  shows ceil-log  $m \leq N$ 
proof (cases  $m \leq 1$ )
  case True
  then show ?thesis
    unfolding ceil-log-def by simp
next
  case False
  then have m-gt:  $1 < m$ 
    by simp
  then have m-pred-pos:  $0 < m - 1$ 
    by simp
  have m-pred-lt:  $m - 1 < (2::nat) \wedge N$ 
    using le-power m-gt by linarith
  have floor-less: floor-log  $(m - 1) < N$ 
proof (rule ccontr)
  assume  $\neg$  floor-log  $(m - 1) < N$ 
  then have N-le:  $N \leq \text{floor-log } (m - 1)$ 
    by simp
  then have  $(2::nat) \wedge N \leq 2 \wedge \text{floor-log } (m - 1)$ 
    by simp
  also have ...  $\leq m - 1$ 
    by (rule floor-log-exp2-le[OF m-pred-pos])
  finally show False
    using m-pred-lt by simp
qed
then show ?thesis
  unfolding ceil-log-def using m-gt by simp
qed

```

```

lemma generic-fri-sampled-assignment-fold-at-step:
  assumes partial:
    generic-fri-partial-evidence low-degree candidate-table degree-bound
    roots challenges final-value query-idxs round-layers
  and no-conflict:
     $\neg$  generic-fri-sampled-assignment-conflict candidate-table roots
    challenges final-value query-idxs round-layers
  and round-bound: round-idx  $< \text{length query-idxs}$ 

```



```

and layer-bound: layer-idx < length challenges
and raw-bound:
  fri-evidence-layer-idx roots query-idxs round-idx layer-idx <
    fri-evidence-layer-len roots layer-idx
and len-bound:
  fri-evidence-layer-len roots layer-idx ≤ clength * scale
and step:
  fri-layer-step-evidence
    (roots ! layer-idx)
    (challenges ! layer-idx)
    (fri-evidence-layer-len roots layer-idx)
    (fri-evidence-layer-idx roots query-idxs round-idx layer-idx)
    (2 ^ layer-idx)
    (fri-sibling-index (fri-evidence-layer-len roots layer-idx)
      (fri-evidence-layer-idx roots query-idxs round-idx layer-idx))
    xp xp-path xn xn-path
    (fri-evidence-next-idx roots query-idxs round-idx layer-idx)
    (fri-evidence-next-value roots challenges query-idxs round-idx
      layer-idx xp xn)
    (round-layers ! round-idx ! layer-idx)
shows
  fri-evidence-next-idx roots query-idxs round-idx layer-idx <
    length (generic-fri-sampled-assignment-layers candidate-table roots
      challenges final-value query-idxs round-layers ! Suc layer-idx)
  fri-sampled-table-fold
    (challenges ! layer-idx)
    (fri-evidence-layer-len roots layer-idx)
    (fri-evidence-layer-idx roots query-idxs round-idx layer-idx)
    (2 ^ layer-idx)
    (fri-canonical-domains (length challenges) ! layer-idx)
    (generic-fri-sampled-assignment-layers candidate-table roots
      challenges final-value query-idxs round-layers ! layer-idx)
    (round-layers ! round-idx ! layer-idx)
    (generic-fri-sampled-assignment-layers candidate-table roots
      challenges final-value query-idxs round-layers ! Suc layer-idx !
      fri-evidence-next-idx roots query-idxs round-idx layer-idx)
proof —
  let ?layers =
    generic-fri-sampled-assignment-layers candidate-table roots challenges
    final-value query-idxs round-layers
  let ?len = fri-evidence-layer-len roots layer-idx
  let ?raw = fri-evidence-layer-idx roots query-idxs round-idx layer-idx
  let ?next = fri-evidence-next-idx roots query-idxs round-idx layer-idx
  let ?next-value =
    fri-evidence-next-value roots challenges query-idxs round-idx layer-idx
    xp xn
  have roots-len: length challenges = length roots
    by (rule generic-fri-partial-evidence-shapes(2)[OF partial])
  have sibling-bound:

```

```

fri-sibling-index ?len ?raw < clength * scale
by (rule fri-evidence-sibling-index-bound-from-row
  [OF raw-bound len-bound])
have next-bound: ?next < clength * scale
  by (rule fri-evidence-next-idx-bound-from-row[OF raw-bound len-bound])
have layer-next-len:
  length (?layers ! Suc layer-idx) = clength * scale
proof -
  have Suc layer-idx ≤ length challenges
    using layer-bound by simp
  then have ?layers ! Suc layer-idx =
    generic-fri-sampled-assignment-layer candidate-table roots challenges
      final-value query-idxs round-layers (Suc layer-idx)
    by (rule generic-fri-sampled-assignment-layers-nth)
  then show ?thesis by simp
qed
then show next-bound-layer:
  ?next < length (?layers ! Suc layer-idx)
  using next-bound by simp
have current-match:
  fri-opening-matches-table ?len ?raw (?layers ! layer-idx) xp xn
proof (cases layer-idx = 0)
  case True
  have challenges-nonempty: 0 < length challenges
    using layer-bound by (cases challenges) auto
  have base-match:
    fri-opening-matches-table (fri-evidence-layer-len roots 0)
      (fri-evidence-layer-idx roots query-idxs round-idx 0)
      (?layers ! 0) xp xn
    by (rule generic-fri-sampled-assignment-base-matches-step
      [OF no-conflict challenges-nonempty round-bound])
      (use step True in simp)
  show ?thesis
    using base-match True by simp
next
  case False
  then have layer-nonzero: 0 < layer-idx
    by simp
  show ?thesis
    by (rule generic-fri-sampled-assignment-current-matches-step
      [OF no-conflict layer-nonzero layer-bound round-bound raw-bound
        len-bound sibling-bound step])
qed
have forced:
  generic-fri-round-forced-next-value roots challenges query-idxs
    round-layers round-idx layer-idx ?next-value
  unfolding generic-fri-round-forced-next-value-def
  using step by blast
have next-value:

```

```

    ?layers ! Suc layer-idx ! ?next = ?next-value
proof (cases Suc layer-idx < length challenges)
  case True
  show ?thesis
    by (rule generic-fri-sampled-assignment-successor-forced-value
      [OF no-conflict True round-bound next-bound forced])
next
  case False
  have suc-eq: Suc layer-idx = length challenges
    using False layer-bound by simp
  have challenges-nonempty: 0 < length challenges
    using layer-bound by (cases challenges) auto
  have layer-eq: layer-idx = length challenges - 1
    using suc-eq by simp
  have forced-last:
    generic-fri-round-forced-next-value roots challenges query-idxs
      round-layers round-idx (length challenges - 1) ?next-value
    using forced layer-eq by simp
  have next-bound-last:
    fri-evidence-next-idx roots query-idxs round-idx
      (length challenges - 1) < clength * scale
    using next-bound layer-eq by simp
  have final-next:
    ?layers ! length challenges !
    fri-evidence-next-idx roots query-idxs round-idx
      (length challenges - 1) = ?next-value
    apply (rule generic-fri-sampled-final-layer-forced-value)
    using no-conflict challenges-nonempty round-bound next-bound-last forced-last
    by simp-all
  show ?thesis
    using final-next layer-eq suc-eq by simp
qed
have chunk:
  fri-layer-opening-chunk ?len xp xp-path xn xn-path
    (round-layers ! round-idx ! layer-idx)
  by (rule fri-layer-step-evidenceD(4)[OF step])
have canonical-layer:
  fri-canonical-domains (length challenges) ! layer-idx =
    fri-canonical-domain-at layer-idx
  by (rule fri-canonical-domains-nth) (use layer-bound in simp)
have canonical-len:
  length (fri-canonical-domain-at layer-idx) = ?len
  unfolding fri-canonical-domain-at-length fri-evidence-layer-len-def
  using roots-len fri-layer-lengths-nth-div
    [of layer-idx length roots clength * scale] layer-bound
  by simp
have dom-raw:
  fri-canonical-domains (length challenges) ! layer-idx ! ?raw =
    (h ^ ?raw * shift) ^ (2 ^ layer-idx)

```

```

unfolding canonical-layer
by (rule fri-canonical-domain-at-nth)
  (use raw-bound canonical-len in simp)
have fold:
  fri-sampled-table-fold
    (challenges ! layer-idx) ?len ?raw ( $2 \wedge$  layer-idx)
    (fri-canonical-domains (length challenges) ! layer-idx)
    (?layers ! layer-idx)
    (round-layers ! round-idx ! layer-idx)
    ?next-value
  by (rule fri-layer-step-evidence-sampled-table-fold-if-matching-chunk
    [OF step dom-row chunk current-match])
then show
  fri-sampled-table-fold
    (challenges ! layer-idx)
    (fri-evidence-layer-len roots layer-idx)
    (fri-evidence-layer-idx roots query-idxs round-idx layer-idx)
    ( $2 \wedge$  layer-idx)
    (fri-canonical-domains (length challenges) ! layer-idx)
    (?layers ! layer-idx)
    (round-layers ! round-idx ! layer-idx)
    (?layers ! Suc layer-idx ! ?next)
  using next-value by simp
qed

```

```

lemma generic-fri-sampled-assignment-layer-chain-evidence:
assumes partial:
  generic-fri-partial-evidence low-degree candidate-table degree-bound
    roots challenges final-value query-idxs round-layers
and no-conflict:
   $\neg$  generic-fri-sampled-assignment-conflict candidate-table roots
    challenges final-value query-idxs round-layers
and challenges-nonempty:  $0 < \text{length challenges}$ 
and query-bounds:
   $\bigwedge \text{round-idx. round-idx} < \text{length query-idxs} \implies$ 
    query-idxs ! round-idx < clength * scale
and eval-power: clength * scale =  $2 \wedge N$ 
and rounds-bound:  $\text{length challenges} \leq N$ 
shows
  generic-fri-sampled-layer-chain-evidence low-degree candidate-table
    degree-bound roots challenges final-value query-idxs round-layers
    (fri-canonical-domains (length challenges))
    (generic-fri-sampled-assignment-layers candidate-table roots
      challenges final-value query-idxs round-layers)
proof –
let ?doms = fri-canonical-domains (length challenges)
let ?layers =
  generic-fri-sampled-assignment-layers candidate-table roots challenges
    final-value query-idxs round-layers

```

```

have doms-len: length ?doms = Suc (length challenges)
  by (rule fri-canonical-domains-length)
have layers-len: length ?layers = Suc (length challenges)
  by (rule generic-fri-sampled-assignment-layers-length)
have doms0: ?doms ! 0 = eval-domain
  by (rule fri-canonical-domains-first)
have layers0: ?layers ! 0 = candidate-table
  by simp
have final:
  fri-final-constant-consistent (?layers ! length challenges)
    final-value
  by (rule generic-fri-sampled-final-assignment-layer-consistent
    [OF challenges-nonempty])
have samples:
   $\bigwedge$ round-idx layer-idx.
    round-idx < length query-idxs  $\implies$ 
    layer-idx < length challenges  $\implies$ 
    fri-evidence-next-idx roots query-idxs round-idx layer-idx <
      length (?layers ! Suc layer-idx)  $\wedge$ 
    fri-sampled-table-fold
      (challenges ! layer-idx)
      (fri-evidence-layer-len roots layer-idx)
      (fri-evidence-layer-idx roots query-idxs round-idx layer-idx)
      (2 ^ layer-idx)
      (?doms ! layer-idx)
      (?layers ! layer-idx)
      (round-layers ! round-idx ! layer-idx)
      (?layers ! Suc layer-idx !
        fri-evidence-next-idx roots query-idxs round-idx layer-idx)
proof –
  fix round-idx layer-idx
  assume round-bound: round-idx < length query-idxs
    and layer-bound: layer-idx < length challenges
  have roots-len: length challenges = length roots
    by (rule generic-fri-partial-evidence-shapes(2)[OF partial])
  have all-layers:
    fri-all-round-layer-evidence roots challenges query-idxs round-layers
    by (rule generic-fri-partial-evidence-shapes(3)[OF partial])
  have layer-root-bound: layer-idx < length roots
    using layer-bound roots-len by simp
  have round-layer:
    fri-round-layer-evidence roots challenges query-idxs round-idx
      layer-idx round-layers
    by (rule fri-all-round-layer-evidenceD
      [OF all-layers round-bound layer-root-bound])
  then obtain xp xp-path xn xn-path where step:
    fri-layer-step-evidence
      (roots ! layer-idx)
      (challenges ! layer-idx)

```

```

(fri-evidence-layer-len roots layer-idx)
(fri-evidence-layer-idx roots query-idxs round-idx layer-idx)
(2 ^ layer-idx)
(fri-sibling-index (fri-evidence-layer-len roots layer-idx)
  (fri-evidence-layer-idx roots query-idxs round-idx layer-idx))
xp xp-path xn xn-path
(fri-evidence-next-idx roots query-idxs round-idx layer-idx)
(fri-evidence-next-value roots challenges query-idxs round-idx
  layer-idx xp xn)
(round-layers ! round-idx ! layer-idx)
by (elim fri-round-layer-evidence-compactE)
have raw-bound:
  fri-evidence-layer-idx roots query-idxs round-idx layer-idx <
    fri-evidence-layer-len roots layer-idx
by (rule fri-evidence-layer-idx-bound-from-query-bound
  [OF partial query-bounds[OF round-bound] refl layer-bound
    eval-power rounds-bound])
have len-bound:
  fri-evidence-layer-len roots layer-idx ≤ clength * scale
by (rule fri-evidence-layer-len-le-eval-domain
  [OF partial layer-bound eval-power])
show
  fri-evidence-next-idx roots query-idxs round-idx layer-idx <
    length (?layers ! Suc layer-idx) ∧
  fri-sampled-table-fold
    (challenges ! layer-idx)
    (fri-evidence-layer-len roots layer-idx)
    (fri-evidence-layer-idx roots query-idxs round-idx layer-idx)
    (2 ^ layer-idx)
    (?doms ! layer-idx)
    (?layers ! layer-idx)
    (round-layers ! round-idx ! layer-idx)
    (?layers ! Suc layer-idx !
      fri-evidence-next-idx roots query-idxs round-idx layer-idx)
  by (rule conjI)
  (rule generic-fri-sampled-assignment-fold-at-step
    [OF partial no-conflict round-bound layer-bound raw-bound
      len-bound step])+)
qed
show ?thesis
  unfolding generic-fri-sampled-layer-chain-evidence-def
  using partial doms-len layers-len doms0 layers0 final samples
  by blast
qed

lemma generic-fri-sampled-assignment-layer-chain-evidence-zero-round:
  assumes partial:
    generic-fri-partial-evidence low-degree candidate-table degree-bound
      roots challenges final-value query-idxs round-layers

```

```

and challenges-empty: length challenges = 0
and final:
  fri-final-constant-consistent candidate-table final-value
shows
  generic-fri-sampled-layer-chain-evidence low-degree candidate-table
  degree-bound roots challenges final-value query-idxs round-layers
  (fri-canonical-domains (length challenges))
  (generic-fri-sampled-assignment-layers candidate-table roots
   challenges final-value query-idxs round-layers)
proof –
  let ?doms = fri-canonical-domains (length challenges)
  let ?layers =
    generic-fri-sampled-assignment-layers candidate-table roots challenges
    final-value query-idxs round-layers
  have doms-len: length ?doms = Suc (length challenges)
    by (rule fri-canonical-domains-length)
  have layers-len: length ?layers = Suc (length challenges)
    by (rule generic-fri-sampled-assignment-layers-length)
  have doms0: ?doms ! 0 = eval-domain
    by (rule fri-canonical-domains-first)
  have layers0: ?layers ! 0 = candidate-table
    by simp
  have final-layer:
    fri-final-constant-consistent (?layers ! length challenges)
    final-value
    using challenges-empty final by simp
  have samples:
     $\bigwedge \text{round-idx layer-idx.}$ 
    round-idx < length query-idxs  $\implies$ 
    layer-idx < length challenges  $\implies$ 
    fri-evidence-next-idx roots query-idxs round-idx layer-idx <
    length (?layers ! Suc layer-idx)  $\wedge$ 
    fri-sampled-table-fold
    (challenges ! layer-idx)
    (fri-evidence-layer-len roots layer-idx)
    (fri-evidence-layer-idx roots query-idxs round-idx layer-idx)
    ( $2 \wedge \text{layer-idx}$ )
    (?doms ! layer-idx)
    (?layers ! layer-idx)
    (round-layers ! round-idx ! layer-idx)
    (?layers ! Suc layer-idx !
     fri-evidence-next-idx roots query-idxs round-idx layer-idx)
    using challenges-empty by simp
  show ?thesis
  unfolding generic-fri-sampled-layer-chain-evidence-def
  using partial doms-len layers-len doms0 layers0 final-layer samples
  by blast
qed

```

lemma *generic-fri-sampled-layer-assignment-obstruction-imp-conflict:*
assumes *obstruction:*
generic-fri-sampled-layer-assignment-obstruction low-degree
candidate-table degree-bound roots challenges final-value query-idxs
round-layers
and *challenges-nonempty:* $0 < \text{length challenges}$
and *query-bounds:*
 $\bigwedge \text{round-idx. round-idx} < \text{length query-idxs} \implies$
 $\text{query-idxs} ! \text{round-idx} < \text{clength} * \text{scale}$
and *eval-power:* $\text{clength} * \text{scale} = 2^N$
and *rounds-bound:* $\text{length challenges} \leq N$
shows
generic-fri-sampled-assignment-conflict candidate-table roots
challenges final-value query-idxs round-layers
proof (rule *ccontr*)
assume *no-conflict:*
 $\neg \text{generic-fri-sampled-assignment-conflict candidate-table roots}$
 $\text{challenges final-value query-idxs round-layers}$
have *partial:*
generic-fri-partial-evidence low-degree candidate-table degree-bound
roots challenges final-value query-idxs round-layers
by (rule *generic-fri-sampled-layer-assignment-obstructionD*(1)
[OF *obstruction*])
have *no-chain:*
 $\bigwedge \text{doms layers. } \neg$
generic-fri-sampled-layer-chain-evidence low-degree candidate-table
degree-bound roots challenges final-value query-idxs round-layers
doms layers
by (rule *generic-fri-sampled-layer-assignment-obstructionD*(2)
[OF *obstruction*])
have *chain:*
generic-fri-sampled-layer-chain-evidence low-degree candidate-table
degree-bound roots challenges final-value query-idxs round-layers
(fri-canonical-domains (length challenges))
(generic-fri-sampled-assignment-layers candidate-table roots
challenges final-value query-idxs round-layers)
by (rule *generic-fri-sampled-assignment-layer-chain-evidence*
[OF *partial no-conflict challenges-nonempty query-bounds*
eval-power rounds-bound])
show *False*
using *no-chain*[of *fri-canonical-domains (length challenges)*
generic-fri-sampled-assignment-layers candidate-table roots
challenges final-value query-idxs round-layers] *chain*
by *blast*
qed

lemma *generic-fri-sampled-layer-assignment-obstruction-imp-conflict-or-zero-round-final:*
assumes *obstruction:*
generic-fri-sampled-layer-assignment-obstruction low-degree


```

    candidate-table degree-bound roots challenges final-value query-idxs
    round-layers
and query-bounds:
     $\bigwedge \text{round-idx. round-idx} < \text{length query-idxs} \implies$ 
     $\text{query-idxs} ! \text{round-idx} < \text{clength} * \text{scale}$ 
and eval-power:  $\text{clength} * \text{scale} = 2^N$ 
and rounds-bound:  $\text{length challenges} \leq N$ 
shows
    generic-fri-sampled-assignment-conflict candidate-table roots
    challenges final-value query-idxs round-layers  $\vee$ 
     $\text{length challenges} = 0 \wedge$ 
     $\neg \text{fri-final-constant-consistent candidate-table final-value}$ 
proof (cases  $0 < \text{length challenges}$ )
    case True
    then show ?thesis
    by (intro disjI1
    generic-fri-sampled-layer-assignment-obstruction-imp-conflict
    [OF obstruction - query-bounds eval-power rounds-bound])
next
    case False
    then have challenges-empty:  $\text{length challenges} = 0$ 
    by simp
    have partial:
    generic-fri-partial-evidence low-degree candidate-table degree-bound
    roots challenges final-value query-idxs round-layers
    by (rule generic-fri-sampled-layer-assignment-obstructionD(1)
    [OF obstruction])
    have no-chain:
     $\bigwedge \text{doms layers. } \neg$ 
    generic-fri-sampled-layer-chain-evidence low-degree candidate-table
    degree-bound roots challenges final-value query-idxs round-layers
    doms layers
    by (rule generic-fri-sampled-layer-assignment-obstructionD(2)
    [OF obstruction])
    show ?thesis
    proof (cases fri-final-constant-consistent candidate-table final-value)
    case True
    have chain:
    generic-fri-sampled-layer-chain-evidence low-degree candidate-table
    degree-bound roots challenges final-value query-idxs round-layers
    (fri-canonical-domains (length challenges))
    (generic-fri-sampled-assignment-layers candidate-table roots
    challenges final-value query-idxs round-layers)
    by (rule generic-fri-sampled-assignment-layer-chain-evidence-zero-round
    [OF partial challenges-empty True])
    then show ?thesis
    using no-chain by blast
next
    case False

```

then show *?thesis*
using *challenges-empty* **by** *blast*
qed
qed

definition *trace-fri-zero-round-final-obstruction*

$:: (f, 'a) \text{ protocol-channel-scheme} \Rightarrow$
 $(\text{unit list} \times (f, 'a) \text{ protocol-channel-scheme}) \text{ option} \Rightarrow \text{bool}$

where

$\text{trace-fri-zero-round-final-obstruction } s \text{ out} \longleftrightarrow$
 $(\exists \text{ trace-roots trace-bs trace-final dg composition-roots}$
 $\text{composition-bs composition-final fri-query-idxs trace-round-layers}$
 $\text{composition-round-layers fr candidate-query-idxs trace-openings}$
 trace-table.
 $\text{trace-fri-partial-candidate-opening-evidence } s \text{ out trace-roots}$
 $\text{trace-bs trace-final dg composition-roots composition-bs}$
 $\text{composition-final fri-query-idxs trace-round-layers}$
 $\text{composition-round-layers fr candidate-query-idxs trace-openings}$
 $\text{trace-table} \wedge$
 $\text{length trace-bs} = 0 \wedge$
 $\neg \text{fri-final-constant-consistent trace-table trace-final})$

definition *composition-fri-zero-round-final-obstruction*

$:: (f, 'a) \text{ protocol-channel-scheme} \Rightarrow$
 $(\text{unit list} \times (f, 'a) \text{ protocol-channel-scheme}) \text{ option} \Rightarrow \text{bool}$

where

$\text{composition-fri-zero-round-final-obstruction } s \text{ out} \longleftrightarrow$
 $(\exists \text{ trace-roots trace-bs trace-final fri-dg}$
 $\text{opening-composition-roots composition-bs composition-final}$
 $\text{fri-query-idxs trace-round-layers composition-round-layers fr}$
 $\text{f-fri-roots f-final as dg composition-fri-roots final}$
 $\text{trace-query-idxs trace-openings composition-query-idxs}$
 $\text{composition-openings trace-table composition-table.}$
 $\text{composition-fri-partial-candidate-opening-evidence } s \text{ out trace-roots}$
 $\text{trace-bs trace-final fri-dg opening-composition-roots}$
 $\text{composition-bs composition-final fri-query-idxs trace-round-layers}$
 $\text{composition-round-layers fr f-fri-roots f-final as dg}$
 $\text{composition-fri-roots final trace-query-idxs trace-openings}$
 $\text{composition-query-idxs composition-openings trace-table}$
 $\text{composition-table} \wedge$
 $\text{length composition-bs} = 0 \wedge$
 $\neg \text{fri-final-constant-consistent composition-table composition-final})$

lemma *trace-fri-sampled-layer-assignment-obstruction-imp-conflict-if-nonempty:*

assumes *obstruction: trace-fri-sampled-layer-assignment-obstruction s out*

and *challenges-nonempty:*

$\wedge \text{trace-roots trace-bs trace-final dg composition-roots}$
 $\text{composition-bs composition-final fri-query-idxs trace-round-layers}$
 $\text{composition-round-layers fr candidate-query-idxs trace-openings}$

```

    trace-table.
  trace-fri-partial-candidate-opening-evidence s out trace-roots
  trace-bs trace-final dg composition-roots composition-bs
  composition-final fri-query-idxs trace-round-layers
  composition-round-layers fr candidate-query-idxs trace-openings
  trace-table  $\implies$ 
  0 < length trace-bs
shows trace-fri-sampled-assignment-conflict s out
proof (rule trace-fri-sampled-layer-assignment-obstructionE[OF obstruction])
fix trace-roots trace-bs trace-final dg composition-roots
  composition-bs composition-final fri-query-idxs trace-round-layers
  composition-round-layers fr candidate-query-idxs trace-openings
  trace-table
assume evidence:
  trace-fri-partial-candidate-opening-evidence s out trace-roots
  trace-bs trace-final dg composition-roots composition-bs
  composition-final fri-query-idxs trace-round-layers
  composition-round-layers fr candidate-query-idxs trace-openings
  trace-table
assume obstruction-generic:
  generic-fri-sampled-layer-assignment-obstruction
  trace-table-low-degree trace-table (clength - 1) trace-roots
  trace-bs trace-final fri-query-idxs trace-round-layers
obtain N where eval-power: clength * scale =  $2^N$ 
using eval-domain-length-power by blast
have trace-rounds-le: length trace-bs  $\leq$  N
proof -
  have fri-openings:
    accepted-fri-opening-transcript s out trace-roots trace-bs
    trace-final dg composition-roots composition-bs composition-final
    fri-query-idxs trace-round-layers composition-round-layers
  by (rule trace-fri-partial-candidate-opening-evidenceD(1)
    [OF evidence])
  have len: length trace-bs = ceil-log clength
    using accepted-fri-opening-transcript-shapes(1,2)[OF fri-openings]
    by simp
  have clength  $\leq$  clength * scale
    using scale-pos by simp
  also have ... =  $2^N$ 
    by (rule eval-power)
  finally have ceil-log clength  $\leq$  N
    by (rule ceil-log-le-power)
  then show ?thesis
    using len by simp
qed
have query-bounds:
   $\bigwedge \text{round-idx. round-idx} < \text{length fri-query-idxs} \implies$ 
  fri-query-idxs ! round-idx < clength * scale
proof -

```

```

fix round-idx
assume round-bound: round-idx < length fri-query-idxs
have fri-query-idxs ! round-idx ∈ set fri-query-idxs
  using round-bound by simp
then show fri-query-idxs ! round-idx < clength * scale
  by (rule trace-fri-partial-candidate-opening-evidence-query-idx-bound
    [OF evidence])
qed
have conflict:
  generic-fri-sampled-assignment-conflict trace-table trace-roots
    trace-bs trace-final fri-query-idxs trace-round-layers
by (rule generic-fri-sampled-layer-assignment-obstruction-imp-conflict
  [OF obstruction-generic challenges-nonempty[OF evidence]
    query-bounds eval-power trace-rounds-le])
show ?thesis
  unfolding trace-fri-sampled-assignment-conflict-def
  using evidence conflict by blast
qed

lemma composition-fri-sampled-layer-assignment-obstruction-imp-conflict-if-nonempty:
assumes support:
  out ∈ set-dist (execute verify-monad s)
and obstruction:
  composition-fri-sampled-layer-assignment-obstruction s out
and challenges-nonempty:
   $\bigwedge$  trace-roots trace-bs trace-final fri-dg opening-composition-roots
    composition-bs composition-final fri-query-idxs trace-round-layers
    composition-round-layers fr f-fri-roots f-final as dg
    composition-fri-roots final trace-query-idxs trace-openings
    composition-query-idxs composition-openings trace-table
    composition-table.
  composition-fri-partial-candidate-opening-evidence s out trace-roots
    trace-bs trace-final fri-dg opening-composition-roots
    composition-bs composition-final fri-query-idxs trace-round-layers
    composition-round-layers fr f-fri-roots f-final as dg
    composition-fri-roots final trace-query-idxs trace-openings
    composition-query-idxs composition-openings trace-table
    composition-table  $\implies$ 
    0 < length composition-bs
shows composition-fri-sampled-assignment-conflict s out
proof (rule composition-fri-sampled-layer-assignment-obstructionE
  [OF obstruction])
fix trace-roots trace-bs trace-final fri-dg opening-composition-roots
  composition-bs composition-final fri-query-idxs trace-round-layers
  composition-round-layers fr f-fri-roots f-final as dg
  composition-fri-roots final trace-query-idxs trace-openings
  composition-query-idxs composition-openings trace-table composition-table
assume evidence:
  composition-fri-partial-candidate-opening-evidence s out trace-roots

```

```

    trace-bs trace-final fri-dg opening-composition-roots composition-bs
    composition-final fri-query-idxs trace-round-layers
    composition-round-layers fr f-fri-roots f-final as dg
    composition-fri-roots final trace-query-idxs trace-openings
    composition-query-idxs composition-openings trace-table
    composition-table
assume obstruction-generic:
    generic-fri-sampled-layer-assignment-obstruction
    (composition-table-low-degree (to-nat fri-dg)) composition-table
    (to-nat fri-dg) opening-composition-roots composition-bs
    composition-final fri-query-idxs composition-round-layers
obtain  $N$  where eval-power:  $\text{length} * \text{scale} = 2^N$ 
    using eval-domain-length-power by blast
have fri-openings:
    accepted-fri-opening-transcript s out trace-roots trace-bs
    trace-final fri-dg opening-composition-roots composition-bs
    composition-final fri-query-idxs trace-round-layers
    composition-round-layers
    by (rule composition-fri-partial-candidate-opening-evidenceD(1)
        [OF evidence])
obtain result final-state where out-eq: out = Some (result, final-state)
    using fri-openings unfolding accepted-fri-opening-transcript-def by blast
have outcome:
    Some (result, final-state)  $\in$  set-dist (execute verify-monad s)
    using support out-eq by simp
have challenges:
    accepted-fri-challenges s (Some (result, final-state))
    trace-bs fri-dg composition-bs
    using accepted-fri-opening-transcript-challenges[OF fri-openings]
    out-eq by simp
have degree-bound: to-nat fri-dg  $\leq$  maxDegree
    by (rule verify-monad-accepted-fri-challenges-degree-bound
        [OF outcome challenges])
have composition-rounds-le: length composition-bs  $\leq N$ 
proof –
    have len: length composition-bs = ceil-log (to-nat fri-dg + 1)
        using accepted-fri-opening-transcript-shapes(3,4)[OF fri-openings]
        by simp
    have to-nat fri-dg + 1  $\leq$  clength * scale
        using degree-bound maxDegree-less-eval-domain by linarith
    also have ... =  $2^N$ 
        by (rule eval-power)
    finally have ceil-log (to-nat fri-dg + 1)  $\leq N$ 
        by (rule ceil-log-le-power)
    then show ?thesis
        using len by simp
qed
have query-bounds:
     $\bigwedge \text{round-idx}. \text{round-idx} < \text{length fri-query-idxs} \implies$ 

```

```

    fri-query-idxs ! round-idx < clength * scale
proof -
  fix round-idx
  assume round-bound: round-idx < length fri-query-idxs
  have fri-query-idxs ! round-idx ∈ set fri-query-idxs
    using round-bound by simp
  then show fri-query-idxs ! round-idx < clength * scale
    by (rule composition-fri-partial-candidate-opening-evidence-query-idx-bound
      [OF evidence])
qed
have conflict:
  generic-fri-sampled-assignment-conflict composition-table
  opening-composition-roots composition-bs composition-final
  fri-query-idxs composition-round-layers
by (rule generic-fri-sampled-layer-assignment-obstruction-imp-conflict
  [OF obstruction-generic challenges-nonempty[OF evidence]
  query-bounds eval-power composition-rounds-le])
show ?thesis
  unfolding composition-fri-sampled-assignment-conflict-def
  apply (intro exI conjI)
  apply (rule evidence)
  apply (rule conflict)
  done
qed

lemma trace-fri-sampled-layer-assignment-obstruction-imp-conflict-or-zero-round-final:
  assumes obstruction: trace-fri-sampled-layer-assignment-obstruction s out
  shows
    trace-fri-sampled-assignment-conflict s out ∨
    trace-fri-zero-round-final-obstruction s out
proof (rule trace-fri-sampled-layer-assignment-obstructionE[OF obstruction])
  fix trace-roots trace-bs trace-final dg composition-roots
  composition-bs composition-final fri-query-idxs trace-round-layers
  composition-round-layers fr candidate-query-idxs trace-openings
  trace-table
  assume evidence:
    trace-fri-partial-candidate-opening-evidence s out trace-roots
    trace-bs trace-final dg composition-roots composition-bs
    composition-final fri-query-idxs trace-round-layers
    composition-round-layers fr candidate-query-idxs trace-openings
    trace-table
  assume obstruction-generic:
    generic-fri-sampled-layer-assignment-obstruction
    trace-table-low-degree trace-table (clength - 1) trace-roots
    trace-bs trace-final fri-query-idxs trace-round-layers
  obtain N where eval-power: clength * scale = 2 ^ N
  using eval-domain-length-power by blast
  have trace-rounds-le: length trace-bs ≤ N
proof -

```

```

have fri-openings:
  accepted-fri-opening-transcript s out trace-roots trace-bs
  trace-final dg composition-roots composition-bs composition-final
  fri-query-idxs trace-round-layers composition-round-layers
  by (rule trace-fri-partial-candidate-opening-evidenceD(1)
    [OF evidence])
have len: length trace-bs = ceil-log length
  using accepted-fri-opening-transcript-shapes(1,2)[OF fri-openings]
  by simp
have clength  $\leq$  clength * scale
  using scale-pos by simp
also have  $\dots = 2^N$ 
  by (rule eval-power)
finally have ceil-log length  $\leq N$ 
  by (rule ceil-log-le-power)
then show ?thesis
  using len by simp
qed
have query-bounds:
   $\bigwedge \text{round-idx. round-idx} < \text{length fri-query-idxs} \implies$ 
   $\text{fri-query-idxs} ! \text{round-idx} < \text{clength} * \text{scale}$ 
proof –
  fix round-idx
  assume round-bound: round-idx < length fri-query-idxs
  have fri-query-idxs ! round-idx  $\in$  set fri-query-idxs
  using round-bound by simp
  then show fri-query-idxs ! round-idx < clength * scale
  by (rule trace-fri-partial-candidate-opening-evidence-query-idx-bound
    [OF evidence])
qed
have generic:
  generic-fri-sampled-assignment-conflict trace-table trace-roots
  trace-bs trace-final fri-query-idxs trace-round-layers  $\vee$ 
  length trace-bs = 0  $\wedge$ 
   $\neg$  fri-final-constant-consistent trace-table trace-final
  by (rule
    generic-fri-sampled-layer-assignment-obstruction-imp-conflict-or-zero-round-final
    [OF obstruction-generic query-bounds eval-power trace-rounds-le])
then show ?thesis
proof
  assume conflict:
    generic-fri-sampled-assignment-conflict trace-table trace-roots
    trace-bs trace-final fri-query-idxs trace-round-layers
  then show ?thesis
    unfolding trace-fri-sampled-assignment-conflict-def
    using evidence by blast
next
  assume zero:
    length trace-bs = 0  $\wedge$ 

```

```

    ¬ fri-final-constant-consistent trace-table trace-final
  then show ?thesis
    unfolding trace-fri-zero-round-final-obstruction-def
    using evidence by blast
qed
qed

lemma composition-fri-sampled-layer-assignment-obstruction-imp-conflict-or-zero-round-final:
  assumes support:
    out ∈ set-dist (execute verify-monad s)
  and obstruction:
    composition-fri-sampled-layer-assignment-obstruction s out
  shows
    composition-fri-sampled-assignment-conflict s out ∨
    composition-fri-zero-round-final-obstruction s out
proof (rule composition-fri-sampled-layer-assignment-obstructionE
  [OF obstruction])
  fix trace-roots trace-bs trace-final fri-dg opening-composition-roots
    composition-bs composition-final fri-query-idxs trace-round-layers
    composition-round-layers fr f-fri-roots f-final as dg
    composition-fri-roots final trace-query-idxs trace-openings
    composition-query-idxs composition-openings trace-table composition-table
  assume evidence:
    composition-fri-partial-candidate-opening-evidence s out trace-roots
    trace-bs trace-final fri-dg opening-composition-roots composition-bs
    composition-final fri-query-idxs trace-round-layers
    composition-round-layers fr f-fri-roots f-final as dg
    composition-fri-roots final trace-query-idxs trace-openings
    composition-query-idxs composition-openings trace-table
    composition-table
  assume obstruction-generic:
    generic-fri-sampled-layer-assignment-obstruction
    (composition-table-low-degree (to-nat fri-dg)) composition-table
    (to-nat fri-dg) opening-composition-roots composition-bs
    composition-final fri-query-idxs composition-round-layers
  obtain N where eval-power: clength * scale = 2 ^ N
  using eval-domain-length-power by blast
  have fri-openings:
    accepted-fri-opening-transcript s out trace-roots trace-bs
    trace-final fri-dg opening-composition-roots composition-bs
    composition-final fri-query-idxs trace-round-layers
    composition-round-layers
  by (rule composition-fri-partial-candidate-opening-evidenceD(1)
    [OF evidence])
  obtain result final-state where out-eq: out = Some (result, final-state)
  using fri-openings unfolding accepted-fri-opening-transcript-def by blast
  have outcome:
    Some (result, final-state) ∈ set-dist (execute verify-monad s)
  using support out-eq by simp

```



```

have challenges:
  accepted-fri-challenges s (Some (result, final-state))
  trace-bs fri-dg composition-bs
using accepted-fri-opening-transcript-challenges[OF fri-openings]
  out-eq by simp
have degree-bound: to-nat fri-dg ≤ maxDegree
  by (rule verify-monad-accepted-fri-challenges-degree-bound
    [OF outcome challenges])
have composition-rounds-le: length composition-bs ≤ N
proof -
  have len: length composition-bs = ceil-log (to-nat fri-dg + 1)
    using accepted-fri-opening-transcript-shapes(3,4)[OF fri-openings]
    by simp
  have to-nat fri-dg + 1 ≤ clength * scale
    using degree-bound maxDegree-less-eval-domain by linarith
  also have ... = 2 ^ N
    by (rule eval-power)
  finally have ceil-log (to-nat fri-dg + 1) ≤ N
    by (rule ceil-log-le-power)
  then show ?thesis
    using len by simp
qed
have query-bounds:
  ∧ round-idx. round-idx < length fri-query-idxs ⇒
    fri-query-idxs ! round-idx < clength * scale
proof -
  fix round-idx
  assume round-bound: round-idx < length fri-query-idxs
  have fri-query-idxs ! round-idx ∈ set fri-query-idxs
    using round-bound by simp
  then show fri-query-idxs ! round-idx < clength * scale
    by (rule composition-fri-partial-candidate-opening-evidence-query-idx-bound
      [OF evidence])
qed
have generic:
  generic-fri-sampled-assignment-conflict composition-table
  opening-composition-roots composition-bs composition-final
  fri-query-idxs composition-round-layers ∨
  length composition-bs = 0 ∧
  ¬ fri-final-constant-consistent composition-table
  composition-final
  by (rule
    generic-fri-sampled-layer-assignment-obstruction-imp-conflict-or-zero-round-final
    [OF obstruction-generic query-bounds eval-power
      composition-rounds-le])
then show ?thesis
proof
  assume conflict:
    generic-fri-sampled-assignment-conflict composition-table

```

```

    opening-composition-roots composition-bs composition-final
    fri-query-idxs composition-round-layers
  then show ?thesis
    unfolding composition-fri-sampled-assignment-conflict-def
    apply (intro disjI1 exI conjI)
    apply (rule evidence)
    apply assumption
  done
next
assume zero:
  length composition-bs = 0 ∧
  ¬ fri-final-constant-consistent composition-table
  composition-final
then show ?thesis
  unfolding composition-fri-zero-round-final-obstruction-def
  apply (intro disjI2 exI conjI)
  apply (rule evidence)
  using zero
  apply simp-all
done
qed
qed

end

end

```

```

theory Soundness-FRI-Zero-Round-Target
  imports Soundness-FRI-Sampled-Assignment-Chain
begin

```

Query-style target events for the zero-round FRI branch.

If no FRI fold round is present, sampled fold equations cannot produce a successor layer. Acceptance can only test that sampled base-layer entries agree with the final value. This theory factors the corresponding target set without changing the protocol or the public soundness interface.

```

context soundness
begin

```

```

definition fri-final-value-agreement-set :: 'f list ⇒ 'f ⇒ nat set
  where
    fri-final-value-agreement-set table final =
      {idx ∈ query-sample-space. idx < length table ∧ table ! idx = final}

```

```

definition fri-zero-round-final-value-target-hit
  :: 'f list ⇒ 'f ⇒ nat list ⇒ bool
  where
    fri-zero-round-final-value-target-hit table final query-idxs

```

$\longleftrightarrow$
 $\neg \text{fri-final-constant-consistent table final} \wedge$
 $\text{set query-idxs} \subseteq \text{fri-final-value-agreement-set table final}$

definition *trace-zero-round-final-checked-candidate*

$:: (\text{'f}, \text{'a}) \text{protocol-channel-scheme} \Rightarrow$
 $(\text{unit list} \times (\text{'f}, \text{'a}) \text{protocol-channel-scheme}) \text{option} \Rightarrow$
 $\text{'f} \Rightarrow \text{nat list} \Rightarrow \text{'f authenticated-opening list list} \Rightarrow$
 $\text{'f list} \Rightarrow \text{'f} \Rightarrow \text{bool}$

where

$\text{trace-zero-round-final-checked-candidate } s \text{ out fr query-idxs}$
 $\text{trace-openings trace-table final} \longleftrightarrow$
 $\text{accepted-with-partial-trace-openings } s \text{ out fr query-idxs}$
 $\text{trace-openings} \wedge$
 $\text{partial-trace-table-candidate trace-table trace-openings} \wedge$
 $(\forall \text{idx} \in \text{set query-idxs}. \text{idx} \in \text{query-sample-space}) \wedge$
 $(\forall i < \text{rounds}. \text{opening-value} (\text{hd} (\text{trace-openings} ! i)) = \text{final})$

definition *trace-fri-zero-round-checked-final-obstruction*

$:: (\text{'f}, \text{'a}) \text{protocol-channel-scheme} \Rightarrow$
 $(\text{unit list} \times (\text{'f}, \text{'a}) \text{protocol-channel-scheme}) \text{option} \Rightarrow \text{bool}$

where

$\text{trace-fri-zero-round-checked-final-obstruction } s \text{ out} \longleftrightarrow$
 $(\exists \text{trace-roots trace-bs trace-final dg composition-roots}$
 $\text{composition-bs composition-final fri-query-idxs trace-round-layers}$
 $\text{composition-round-layers fr trace-openings trace-table.}$
 $\text{accepted-fri-opening-transcript } s \text{ out trace-roots trace-bs}$
 $\text{trace-final dg composition-roots composition-bs composition-final}$
 $\text{fri-query-idxs trace-round-layers composition-round-layers} \wedge$
 $\text{length trace-bs} = 0 \wedge$
 $\text{trace-zero-round-final-checked-candidate } s \text{ out fr fri-query-idxs}$
 $\text{trace-openings trace-table trace-final} \wedge$
 $\neg \text{fri-final-constant-consistent trace-table trace-final})$

lemma *fri-final-value-agreement-set-subset:*

$\text{fri-final-value-agreement-set table final} \subseteq \text{query-sample-space}$
unfolding *fri-final-value-agreement-set-def* **by** *blast*

lemma *finite-fri-final-value-agreement-set[simp]:*

$\text{finite} (\text{fri-final-value-agreement-set table final})$
by (*rule finite-subset[OF fri-final-value-agreement-set-subset]*)
simp

lemma *fri-final-value-agreement-setD:*

assumes $\text{idx} \in \text{fri-final-value-agreement-set table final}$
shows $\text{idx} \in \text{query-sample-space}$
and $\text{idx} < \text{length table}$
and $\text{table} ! \text{idx} = \text{final}$
using *assms* **unfolding** *fri-final-value-agreement-set-def* **by** *blast+*

lemma *fri-zero-round-final-value-target-hitD*:
assumes *fri-zero-round-final-value-target-hit table final query-idxs*
shows $\neg$ *fri-final-constant-consistent table final*
and $\bigwedge idx. idx \in \text{set } query-idxs \implies$
 $idx \in \text{fri-final-value-agreement-set table final}$
using *assms*
unfolding *fri-zero-round-final-value-target-hit-def* **by** *blast+*

lemma *powers-scaled-hd*:
 $hd \text{ (powers-scaled } idx) = idx$
unfolding *powers-scaled-def*
using *powers-pos* **by** (*simp add: upt-conv-Cons*)

lemma *trace-zero-round-final-checked-candidate-query-agrees*:
assumes *checked*:
 $trace-zero-round-final-checked-candidate \ s \ out \ fr \ query-idxs$
 $trace-openings \ trace-table \ final$
and *i-bound*: $i < rounds$
shows
 $query-idxs \ ! \ i \in \text{fri-final-value-agreement-set trace-table final}$
proof –
have *partial*:
 $accepted-with-partial-trace-openings \ s \ out \ fr \ query-idxs \ trace-openings$
using *checked* **unfolding** *trace-zero-round-final-checked-candidate-def*
by *simp*
have *cand*:
 $partial-trace-table-candidate \ trace-table \ trace-openings$
using *checked* **unfolding** *trace-zero-round-final-checked-candidate-def*
by *simp*
have *query-sample*:
 $query-idxs \ ! \ i \in \text{query-sample-space}$
using *checked i-bound accepted-with-partial-trace-openings-shapes(1)*
 $[OF \ partial]$
unfolding *trace-zero-round-final-checked-candidate-def*
by *simp*
have *idxs*:
 $map \ opening-index \ (trace-openings \ ! \ i) =$
 $powers-scaled \ (query-idxs \ ! \ i)$
by (*rule accepted-with-partial-trace-openings-shapes(3)*
 $[OF \ partial \ i-bound]$)
have *openings-nonempty*: $trace-openings \ ! \ i \neq []$
proof –
have $map \ opening-index \ (trace-openings \ ! \ i) \neq []$
using *idxs powers-pos* **unfolding** *powers-scaled-def* **by** *simp*
then show *?thesis* **by** *auto*
qed
have *hd-index*:
 $opening-index \ (hd \ (trace-openings \ ! \ i)) = query-idxs \ ! \ i$

proof –
have *opening-index* (*hd* (*trace-openings* ! *i*)) =
 hd (*map* *opening-index* (*trace-openings* ! *i*))
 using *openings-nonempty* **by** (*cases* *trace-openings* ! *i*) *auto*
also have ... = *hd* (*powers-scaled* (*query-idxs* ! *i*))
 using *idxs* **by** *simp*
also have ... = *query-idxs* ! *i*
 by (*rule* *powers-scaled-hd*)
finally show ?*thesis* .
qed
have *hd-in*: *hd* (*trace-openings* ! *i*) ∈ *set* (*trace-openings* ! *i*)
 using *openings-nonempty* **by** *simp*
have *table-value*:
 trace-table ! (*query-idxs* ! *i*) =
 opening-value (*hd* (*trace-openings* ! *i*))
 using *partial-trace-table-candidateD*(3)[*OF* *cand i-bound hd-in*]
 hd-index **by** *simp*
have *final*:
 opening-value (*hd* (*trace-openings* ! *i*)) = *final*
 using *checked i-bound*
 unfolding *trace-zero-round-final-checked-candidate-def* **by** *simp*
have *table-len*:
 length *trace-table* = *scale* * *clength*
 using *cand* **unfolding** *partial-trace-table-candidate-def* **by** *blast*
have *idx-bound*: *query-idxs* ! *i* < *length* *trace-table*
 using *partial-trace-table-candidateD*(2)[*OF* *cand i-bound hd-in*]
 hd-index table-len **by** *simp*
show ?*thesis*
 unfolding *fri-final-value-agreement-set-def*
 using *query-sample idx-bound table-value final* **by** *simp*
qed

lemma *trace-zero-round-final-checked-candidate-target-hit*:
assumes *checked*:
 trace-zero-round-final-checked-candidate *s* *out* *fr* *query-idxs*
 trace-openings *trace-table* *final*
and *not-final*:
 ¬ *fri-final-constant-consistent* *trace-table* *final*
shows
 fri-zero-round-final-value-target-hit *trace-table* *final* *query-idxs*

proof –
have *partial*:
 accepted-with-partial-trace-openings *s* *out* *fr* *query-idxs* *trace-openings*
 using *checked* **unfolding** *trace-zero-round-final-checked-candidate-def*
 by *simp*
have *len-query*: *length* *query-idxs* = *rounds*
 by (*rule* *accepted-with-partial-trace-openings-shapes*(1)[*OF* *partial*])
have *subset*:
 set *query-idxs* ⊆ *fri-final-value-agreement-set* *trace-table* *final*

```

proof
  fix idx
  assume idx-in: idx ∈ set query-idxs
  then obtain i where i-bound: i < length query-idxs
    and idx-eq: idx = query-idxs ! i
    by (auto simp: in-set-conv-nth)
  then have i < rounds
    using len-query by simp
  then show idx ∈ fri-final-value-agreement-set trace-table final
    using idx-eq
    by (simp add:
      trace-zero-round-final-checked-candidate-query-agrees[OF checked])
qed
show ?thesis
  unfolding fri-zero-round-final-value-target-hit-def
  using not-final subset by simp
qed

```

```

lemma trace-fri-zero-round-checked-final-obstructionE:
  assumes trace-fri-zero-round-checked-final-obstruction s out
  obtains trace-roots trace-bs trace-final dg composition-roots
    composition-bs composition-final fri-query-idxs trace-round-layers
    composition-round-layers fr trace-openings trace-table
  where
    accepted-fri-opening-transcript s out trace-roots trace-bs
    trace-final dg composition-roots composition-bs composition-final
    fri-query-idxs trace-round-layers composition-round-layers
    length trace-bs = 0
    trace-zero-round-final-checked-candidate s out fr fri-query-idxs
    trace-openings trace-table trace-final
     $\neg$  fri-final-constant-consistent trace-table trace-final
  using assms
  unfolding trace-fri-zero-round-checked-final-obstruction-def
  by blast

```

```

lemma trace-fri-zero-round-checked-final-obstruction-target-hit:
  assumes trace-fri-zero-round-checked-final-obstruction s out
  obtains trace-roots trace-bs trace-final dg composition-roots
    composition-bs composition-final fri-query-idxs trace-round-layers
    composition-round-layers fr trace-openings trace-table
  where
    accepted-fri-opening-transcript s out trace-roots trace-bs
    trace-final dg composition-roots composition-bs composition-final
    fri-query-idxs trace-round-layers composition-round-layers
    length trace-bs = 0
    trace-zero-round-final-checked-candidate s out fr fri-query-idxs
    trace-openings trace-table trace-final
     $\neg$  fri-final-constant-consistent trace-table trace-final
    fri-zero-round-final-value-target-hit trace-table trace-final

```

fri-query-idxs
proof (rule trace-fri-zero-round-checked-final-obstructionE[*OF assms*])
fix trace-roots trace-bs trace-final dg composition-roots composition-bs
composition-final fri-query-idxs trace-round-layers
composition-round-layers fr trace-openings trace-table
assume fri-openings:
accepted-fri-opening-transcript *s out* trace-roots trace-bs
trace-final dg composition-roots composition-bs composition-final
fri-query-idxs trace-round-layers composition-round-layers
and rounds-empty: length trace-bs = 0
and checked:
trace-zero-round-final-checked-candidate *s out* fr fri-query-idxs
trace-openings trace-table trace-final
and not-final:
 $\neg$ fri-final-constant-consistent trace-table trace-final
have target:
fri-zero-round-final-value-target-hit trace-table trace-final
fri-query-idxs
by (rule trace-zero-round-final-checked-candidate-target-hit
[*OF checked not-final*])
show ?thesis
by (rule that[*OF fri-openings rounds-empty checked not-final target*])
qed

lemma fri-final-value-agreement-set-card-le:
card (fri-final-value-agreement-set table final) $\leq$
query-sample-space-size
proof –
have card (fri-final-value-agreement-set table final) $\leq$
card query-sample-space
by (rule card-mono[*OF finite-query-sample-space*
fri-final-value-agreement-set-subset])
then show ?thesis **by** simp
qed

lemma fri-final-value-agreement-set-card-lt-if-query-sample-disagrees:
assumes idx-sample: $\text{idx} \in \text{query-sample-space}$
and idx-bound: $\text{idx} < \text{length table}$
and neg: $\text{table} ! \text{idx} \neq \text{final}$
shows card (fri-final-value-agreement-set table final) $<$
query-sample-space-size
proof –
have proper:
fri-final-value-agreement-set table final $\subset$ query-sample-space
proof
show fri-final-value-agreement-set table final $\subseteq$ query-sample-space
by (rule fri-final-value-agreement-set-subset)
next
show fri-final-value-agreement-set table final $\neq$ query-sample-space

```

proof
  assume eq: fri-final-value-agreement-set table final =
    query-sample-space
  then have idx ∈ fri-final-value-agreement-set table final
    using idx-sample by simp
  then show False
    using fri-final-value-agreement-setD(3) neq by blast
qed
qed
have card (fri-final-value-agreement-set table final) <
  card query-sample-space
  by (rule psubset-card-mono[OF finite-query-sample-space proper])
then show ?thesis by simp
qed

lemma fri-final-value-agreement-set-all-query-sample-iff:
  assumes table-covers-query-sample:
     $\bigwedge idx. idx \in \text{query-sample-space} \implies idx < \text{length table}$ 
  shows
     $\text{fri-final-value-agreement-set table final} = \text{query-sample-space}$ 
     $\longleftrightarrow$ 
     $(\forall idx \in \text{query-sample-space}. \text{table} ! idx = \text{final})$ 
  unfolding fri-final-value-agreement-set-def
  using table-covers-query-sample by blast

lemma fri-final-constant-consistent-if-query-sample-is-whole-table:
  assumes table-len: length table = query-sample-space-size
  and all-query:  $\forall idx \in \text{query-sample-space}. \text{table} ! idx = \text{final}$ 
  shows fri-final-constant-consistent table final
proof –
  have all-idx:  $\bigwedge idx. idx < \text{length table} \implies \text{table} ! idx = \text{final}$ 
proof –
  fix idx
  assume idx-bound: idx < length table
  then have idx ∈ query-sample-space
    using table-len unfolding query-sample-space-def by simp
  then show table ! idx = final
    using all-query by simp
qed
show ?thesis
  unfolding fri-final-constant-consistent-def
  using all-idx by (auto simp: in-set-conv-nth)
qed

lemma fri-final-value-agreement-set-card-lt-if-not-constant-whole-sample:
  assumes table-len: length table = query-sample-space-size
  and not-final:  $\neg \text{fri-final-constant-consistent table final}$ 
  shows card (fri-final-value-agreement-set table final) <
    query-sample-space-size

```



```

proof (rule ccontr)
  assume  $\neg$  card (fri-final-value-agreement-set table final) <
    query-sample-space-size
  then have card-eq:
    card (fri-final-value-agreement-set table final) =
      query-sample-space-size
  using fri-final-value-agreement-set-card-le[of table final] by linarith
  have set-eq:
    fri-final-value-agreement-set table final = query-sample-space
  by (rule card-subset-eq[OF finite-query-sample-space
    fri-final-value-agreement-set-subset])
    (use card-eq in simp)
  have all-query:
     $\forall$  idx  $\in$  query-sample-space. table ! idx = final
  using set-eq fri-final-value-agreement-setD(3) by metis
  have fri-final-constant-consistent table final
  by (rule fri-final-constant-consistent-if-query-sample-is-whole-table
    [OF table-len all-query])
  then show False
    using not-final by simp
qed

```

```

lemma trace-zero-round-table-len-equals-query-sample-space:
  assumes trace-rounds-empty: length trace-bs = 0
  and fri-openings:
    accepted-fri-opening-transcript s out trace-roots trace-bs
    trace-final dg composition-roots composition-bs composition-final
    fri-query-idxs trace-round-layers composition-round-layers
  and cand: partial-trace-table-candidate trace-table trace-openings
  shows length trace-table = query-sample-space-size
proof –
  have len-trace-bs: length trace-bs = ceil-log clength
    using accepted-fri-opening-transcript-shapes(1,2)[OF fri-openings]
    by simp
  have ceil-zero: ceil-log clength = 0
    using trace-rounds-empty len-trace-bs by simp
  have clength-le-one: clength  $\leq$  1
    using ceil-zero unfolding ceil-log-def
    by (cases clength  $\leq$  1) simp-all
  have clength-eq: clength = 1
    using clength-le-one clength-pos by simp
  have powers-eq: powers = 1
    using powers-pos powers-le-clength clength-eq by simp
  have table-len: length trace-table = scale * clength
    using cand unfolding partial-trace-table-candidate-def by blast
  show ?thesis
    unfolding query-sample-space-size-def
    using table-len clength-eq powers-eq by simp
qed

```

lemma *trace-fri-zero-round-checked-final-obstruction-agreement-set-card-lt*:
assumes *obstruction*:
trace-fri-zero-round-checked-final-obstruction s out
obtains *trace-roots trace-bs trace-final dg composition-roots*
composition-bs composition-final fri-query-idxs trace-round-layers
composition-round-layers fr trace-openings trace-table
where
accepted-fri-opening-transcript s out trace-roots trace-bs
trace-final dg composition-roots composition-bs composition-final
fri-query-idxs trace-round-layers composition-round-layers
length trace-bs = 0
trace-zero-round-final-checked-candidate s out fr fri-query-idxs
trace-openings trace-table trace-final
 $\neg$ *fri-final-constant-consistent trace-table trace-final*
fri-zero-round-final-value-target-hit trace-table trace-final
fri-query-idxs
card (fri-final-value-agreement-set trace-table trace-final) <
query-sample-space-size
proof (*rule trace-fri-zero-round-checked-final-obstruction-target-hit*
[OF obstruction])
fix *trace-roots trace-bs trace-final dg composition-roots composition-bs*
composition-final fri-query-idxs trace-round-layers
composition-round-layers fr trace-openings trace-table
assume *fri-openings*:
accepted-fri-opening-transcript s out trace-roots trace-bs
trace-final dg composition-roots composition-bs composition-final
fri-query-idxs trace-round-layers composition-round-layers
and *rounds-empty*: *length trace-bs = 0*
and *checked*:
trace-zero-round-final-checked-candidate s out fr fri-query-idxs
trace-openings trace-table trace-final
and *not-final*:
 $\neg$ *fri-final-constant-consistent trace-table trace-final*
and *target*:
fri-zero-round-final-value-target-hit trace-table trace-final
fri-query-idxs
have *cand*:
partial-trace-table-candidate trace-table trace-openings
using *checked* **unfolding** *trace-zero-round-final-checked-candidate-def*
by *simp*
have *table-len*:
length trace-table = query-sample-space-size
by (*rule trace-zero-round-table-len-equals-query-sample-space*
[OF rounds-empty fri-openings cand])
have *card-lt*:
card (fri-final-value-agreement-set trace-table trace-final) <
query-sample-space-size
by (*rule fri-final-value-agreement-set-card-lt-if-not-constant-whole-sample*

$[OF \text{ table-len not-final}]$)
show ?thesis
by (rule that[$OF \text{ fri-openings rounds-empty checked not-final target card-lt}$])
qed

lemma *trace-fri-zero-round-checked-final-obstruction-first-query-target*:
assumes *obstruction*:
trace-fri-zero-round-checked-final-obstruction s out
obtains *trace-roots trace-bs trace-final dg composition-roots*
composition-bs composition-final fri-query-idxs trace-round-layers
composition-round-layers fr trace-openings trace-table
where
accepted-fri-opening-transcript s out trace-roots trace-bs
trace-final dg composition-roots composition-bs composition-final
fri-query-idxs trace-round-layers composition-round-layers
length trace-bs = 0
trace-zero-round-final-checked-candidate s out fr fri-query-idxs
trace-openings trace-table trace-final
 $\neg \text{fri-final-constant-consistent trace-table trace-final}$
fri-query-idxs ! 0 ∈
fri-final-value-agreement-set trace-table trace-final
card (fri-final-value-agreement-set trace-table trace-final) <
query-sample-space-size
proof (rule *trace-fri-zero-round-checked-final-obstruction-agreement-set-card-lt*
 $[OF \text{ obstruction}]$)
fix *trace-roots trace-bs trace-final dg composition-roots composition-bs*
composition-final fri-query-idxs trace-round-layers
composition-round-layers fr trace-openings trace-table
assume *fri-openings*:
accepted-fri-opening-transcript s out trace-roots trace-bs
trace-final dg composition-roots composition-bs composition-final
fri-query-idxs trace-round-layers composition-round-layers
and *rounds-empty*: *length trace-bs = 0*
and *checked*:
trace-zero-round-final-checked-candidate s out fr fri-query-idxs
trace-openings trace-table trace-final
and *not-final*:
 $\neg \text{fri-final-constant-consistent trace-table trace-final}$
and *target*:
fri-zero-round-final-value-target-hit trace-table trace-final
fri-query-idxs
and *card-lt*:
card (fri-final-value-agreement-set trace-table trace-final) <
query-sample-space-size
have *first-in*:
fri-query-idxs ! 0 ∈
fri-final-value-agreement-set trace-table trace-final
proof –

```

have partial:
  accepted-with-partial-trace-openings s out fr fri-query-idxs
  trace-openings
  using checked unfolding trace-zero-round-final-checked-candidate-def
  by simp
have length fri-query-idxs = rounds
  by (rule accepted-with-partial-trace-openings-shapes(1)[OF partial])
then have fri-query-idxs ! 0 ∈ set fri-query-idxs
  using rounds-positive by (simp add: nth-mem)
then show ?thesis
  by (rule fri-zero-round-final-value-target-hitD(2)[OF target])
qed
show ?thesis
  by (rule that[OF fri-openings rounds-empty checked not-final first-in
    card-lt])
qed

lemma trace-zero-round-checked-candidate-not-trace-low-degree:
  assumes rounds-empty: length trace-bs = 0
  and fri-openings:
    accepted-fri-opening-transcript s out trace-roots trace-bs
    trace-final dg composition-roots composition-bs composition-final
    fri-query-idxs trace-round-layers composition-round-layers
  and checked:
    trace-zero-round-final-checked-candidate s out fr fri-query-idxs
    trace-openings trace-table trace-final
  and not-final:
    ¬ fri-final-constant-consistent trace-table trace-final
  shows ¬ trace-table-low-degree trace-table
proof
  assume low: trace-table-low-degree trace-table
  have len-trace-bs: length trace-bs = ceil-log clength
    using accepted-fri-opening-transcript-shapes(1,2)[OF fri-openings]
    by simp
  have ceil-zero: ceil-log clength = 0
    using rounds-empty len-trace-bs by simp
  have clength-le-one: clength ≤ 1
    using ceil-zero unfolding ceil-log-def
    by (cases clength ≤ 1) simp-all
  have clength-eq: clength = 1
    using clength-le-one clength-pos by simp
  obtain f where deg-f: degree f < clength
    and trace-table: trace-table = map (poly f) eval-domain
    using low unfolding trace-table-low-degree-def by blast
  have deg0: degree f = 0
    using deg-f clength-eq by simp
  obtain a where f-eq: f = [:a:]
    using degree0-coeffs[OF deg0] by blast
  have table-const:

```

```

 $\bigwedge y. y \in \text{set trace-table} \implies y = a$ 
using trace-table f-eq by auto
have sample-agrees:
  fri-query-idxs ! 0  $\in$ 
    fri-final-value-agreement-set trace-table trace-final
by (rule trace-zero-round-final-checked-candidate-query-agrees
    [OF checked rounds-positive])
have idx-bound: fri-query-idxs ! 0 < length trace-table
  using fri-final-value-agreement-setD(2)[OF sample-agrees] .
have table-at-final:
  trace-table ! (fri-query-idxs ! 0) = trace-final
  using fri-final-value-agreement-setD(3)[OF sample-agrees] .
have trace-table ! (fri-query-idxs ! 0)  $\in$  set trace-table
  using idx-bound by simp
then have a-eq: a = trace-final
  using table-const table-at-final by simp
have fri-final-constant-consistent trace-table trace-final
  unfolding fri-final-constant-consistent-def
  using table-const a-eq by auto
then show False
  using not-final by contradiction
qed

```

lemma trace-fri-zero-round-checked-final-obstruction-not-trace-low-degree:

assumes obstruction:

trace-fri-zero-round-checked-final-obstruction s out

obtains trace-roots trace-bs trace-final dg composition-roots

composition-bs composition-final fri-query-idxs trace-round-layers

composition-round-layers fr trace-openings trace-table

where

accepted-fri-opening-transcript s out trace-roots trace-bs

trace-final dg composition-roots composition-bs composition-final

fri-query-idxs trace-round-layers composition-round-layers

length trace-bs = 0

trace-zero-round-final-checked-candidate s out fr fri-query-idxs

trace-openings trace-table trace-final

$\neg$ fri-final-constant-consistent trace-table trace-final

$\neg$ trace-table-low-degree trace-table

proof (rule trace-fri-zero-round-checked-final-obstructionE[OF obstruction])

fix trace-roots trace-bs trace-final dg composition-roots composition-bs

composition-final fri-query-idxs trace-round-layers

composition-round-layers fr trace-openings trace-table

assume fri-openings:

accepted-fri-opening-transcript s out trace-roots trace-bs

trace-final dg composition-roots composition-bs composition-final

fri-query-idxs trace-round-layers composition-round-layers

and rounds-empty: length trace-bs = 0

and checked:

trace-zero-round-final-checked-candidate s out fr fri-query-idxs

trace-openings trace-table trace-final
and *not-final*:
 $\neg$ *fri-final-constant-consistent trace-table trace-final*
have *not-low*:
 $\neg$ *trace-table-low-degree trace-table*
by (rule *trace-zero-round-checked-candidate-not-trace-low-degree*
[*OF rounds-empty fri-openings checked not-final*])
show *?thesis*
by (rule *that[OF fri-openings rounds-empty checked not-final not-low]*)
qed

definition *trace-fri-zero-round-checked-first-query-target-hit*
 $:: (f, 'a) \text{ protocol-channel-scheme} \Rightarrow$
 $(\text{unit list} \times (f, 'a) \text{ protocol-channel-scheme}) \text{ option} \Rightarrow \text{bool}$
where
trace-fri-zero-round-checked-first-query-target-hit s out $\longleftrightarrow$
 $(\exists \text{ trace-roots trace-bs trace-final dg composition-roots}$
 $\text{composition-bs composition-final fri-query-idxs trace-round-layers}$
 $\text{composition-round-layers fr trace-openings trace-table.}$
 $\text{accepted-fri-opening-transcript s out trace-roots trace-bs}$
 $\text{trace-final dg composition-roots composition-bs composition-final}$
 $\text{fri-query-idxs trace-round-layers composition-round-layers} \wedge$
 $\text{length trace-bs} = 0 \wedge$
 $\text{trace-zero-round-final-checked-candidate s out fr fri-query-idxs}$
 $\text{trace-openings trace-table trace-final} \wedge$
 $\neg \text{fri-final-constant-consistent trace-table trace-final} \wedge$
 $\text{fri-query-idxs} ! 0 \in$
 $\text{fri-final-value-agreement-set trace-table trace-final} \wedge$
 $\text{card} (\text{fri-final-value-agreement-set trace-table trace-final}) <$
 $\text{query-sample-space-size})$

lemma *trace-fri-zero-round-checked-final-obstruction-imp-first-query-target-hit*:

assumes *trace-fri-zero-round-checked-final-obstruction s out*

shows *trace-fri-zero-round-checked-first-query-target-hit s out*

proof (rule *trace-fri-zero-round-checked-final-obstruction-first-query-target*
[*OF assms*])

fix *trace-roots trace-bs trace-final dg composition-roots composition-bs*
 $\text{composition-final fri-query-idxs trace-round-layers composition-round-layers}$
 $\text{fr trace-openings trace-table}$

assume *fri-openings*:

$\text{accepted-fri-opening-transcript s out trace-roots trace-bs}$
 $\text{trace-final dg composition-roots composition-bs composition-final}$
 $\text{fri-query-idxs trace-round-layers composition-round-layers}$

and *rounds-empty*: $\text{length trace-bs} = 0$

and *checked*:

$\text{trace-zero-round-final-checked-candidate s out fr fri-query-idxs}$
 $\text{trace-openings trace-table trace-final}$

and *not-final*:

$\neg \text{fri-final-constant-consistent trace-table trace-final}$

and *first-hit*:
fri-query-idxs ! 0 ∈
fri-final-value-agreement-set trace-table trace-final
and *card-lt*:
card (fri-final-value-agreement-set trace-table trace-final) <
query-sample-space-size
show *?thesis*
unfolding *trace-fri-zero-round-checked-first-query-target-hit-def*
by (*intro exI conjI*)
(rule fri-openings, rule rounds-empty, rule checked, rule not-final,
rule first-hit, rule card-lt)
qed

lemma *wp-trace-fri-zero-round-checked-final-obstruction-bound-from-first-query-target-hit*:
assumes *target-bound*:
wp-event verify-monad
(trace-fri-zero-round-checked-first-query-target-hit s) s ≤ T
shows
wp-event verify-monad
(trace-fri-zero-round-checked-final-obstruction s) s ≤ T
proof –
have *wp-event verify-monad*
(trace-fri-zero-round-checked-final-obstruction s) s ≤
wp-event verify-monad
(trace-fri-zero-round-checked-first-query-target-hit s) s
by (*rule wp-event-mono*)
(rule
trace-fri-zero-round-checked-final-obstruction-imp-first-query-target-hit)
then show *?thesis*
by (*rule order-trans[OF - target-bound]*)
qed

lemma *trace-fri-zero-round-final-obstruction-agreement-set-card-lt*:
assumes *obstruction: trace-fri-zero-round-final-obstruction s out*
obtains *trace-roots trace-bs trace-final dg composition-roots*
composition-bs composition-final fri-query-idxs trace-round-layers
composition-round-layers fr candidate-query-idxs trace-openings
trace-table
where
trace-fri-partial-candidate-opening-evidence s out trace-roots
trace-bs trace-final dg composition-roots composition-bs
composition-final fri-query-idxs trace-round-layers
composition-round-layers fr candidate-query-idxs trace-openings
trace-table
length trace-bs = 0
 $\neg$ *fri-final-constant-consistent trace-table trace-final*
card (fri-final-value-agreement-set trace-table trace-final) <
query-sample-space-size
proof –

from *obstruction* **obtain** *trace-roots trace-bs trace-final dg*
composition-roots composition-bs composition-final fri-query-idxs
trace-round-layers composition-round-layers fr candidate-query-idxs
trace-openings trace-table **where** *evidence:*
trace-fri-partial-candidate-opening-evidence s out trace-roots
trace-bs trace-final dg composition-roots composition-bs
composition-final fri-query-idxs trace-round-layers
composition-round-layers fr candidate-query-idxs trace-openings
trace-table
and *rounds-empty: length trace-bs = 0*
and *not-final:*
 $\neg$ *fri-final-constant-consistent trace-table trace-final*
unfolding *trace-fri-zero-round-final-obstruction-def* **by** *blast*
have *fri-openings:*
accepted-fri-opening-transcript s out trace-roots trace-bs
trace-final dg composition-roots composition-bs composition-final
fri-query-idxs trace-round-layers composition-round-layers
by (*rule trace-fri-partial-candidate-opening-evidenceD(1)*
 $[OF\ evidence]$)
have *partial:*
trace-fri-partial-candidate-evidence s out fr candidate-query-idxs
trace-openings trace-table
by (*rule trace-fri-partial-candidate-opening-evidenceD(2)*
 $[OF\ evidence]$)
have *cand:*
partial-trace-table-candidate trace-table trace-openings
by (*rule trace-fri-partial-candidate-evidenceD(2)[OF partial]*)
have *table-len:*
length trace-table = query-sample-space-size
by (*rule trace-zero-round-table-len-equals-query-sample-space*
 $[OF\ rounds-empty\ fri-openings\ cand]$)
have *card-lt:*
card (fri-final-value-agreement-set trace-table trace-final) <
query-sample-space-size
by (*rule fri-final-value-agreement-set-card-lt-if-not-constant-whole-sample*
 $[OF\ table-len\ not-final]$)
show *?thesis*
by (*rule that[OF evidence rounds-empty not-final card-lt]*)
qed

lemma *trace-fri-zero-round-final-obstructionE:*
assumes *trace-fri-zero-round-final-obstruction s out*
obtains *trace-roots trace-bs trace-final dg composition-roots*
composition-bs composition-final fri-query-idxs trace-round-layers
composition-round-layers fr candidate-query-idxs trace-openings
trace-table
where
trace-fri-partial-candidate-opening-evidence s out trace-roots
trace-bs trace-final dg composition-roots composition-bs

composition-final fri-query-idxs trace-round-layers
composition-round-layers fr candidate-query-idxs trace-openings
trace-table
length trace-bs = 0
 $\neg$ *fri-final-constant-consistent trace-table trace-final*
using *assms*
unfolding *trace-fri-zero-round-final-obstruction-def* **by** *blast*

definition *trace-fri-zero-round-candidate-small-agreement-set*

$:: (f, 'a) \text{ protocol-channel-scheme} \Rightarrow$
 $(\text{unit list} \times (f, 'a) \text{ protocol-channel-scheme}) \text{ option} \Rightarrow \text{bool}$

where

trace-fri-zero-round-candidate-small-agreement-set s out $\longleftrightarrow$
 $(\exists \text{ trace-roots trace-bs trace-final dg composition-roots}$
 $\text{composition-bs composition-final fri-query-idxs trace-round-layers}$
 $\text{composition-round-layers fr candidate-query-idxs trace-openings}$
 trace-table.
trace-fri-partial-candidate-opening-evidence s out trace-roots
 $\text{trace-bs trace-final dg composition-roots composition-bs}$
 $\text{composition-final fri-query-idxs trace-round-layers}$
 $\text{composition-round-layers fr candidate-query-idxs trace-openings}$
 $\text{trace-table} \wedge$
 $\text{length trace-bs} = 0 \wedge$
 $\neg \text{fri-final-constant-consistent trace-table trace-final} \wedge$
 $\text{card (fri-final-value-agreement-set trace-table trace-final)} <$
 $\text{query-sample-space-size})$

definition *trace-fri-zero-round-candidate-first-query-target-hit*

$:: (f, 'a) \text{ protocol-channel-scheme} \Rightarrow$
 $(\text{unit list} \times (f, 'a) \text{ protocol-channel-scheme}) \text{ option} \Rightarrow \text{bool}$

where

trace-fri-zero-round-candidate-first-query-target-hit s out $\longleftrightarrow$
 $(\exists \text{ trace-roots trace-bs trace-final dg composition-roots}$
 $\text{composition-bs composition-final fri-query-idxs trace-round-layers}$
 $\text{composition-round-layers fr candidate-query-idxs trace-openings}$
 trace-table.
trace-fri-partial-candidate-opening-evidence s out trace-roots
 $\text{trace-bs trace-final dg composition-roots composition-bs}$
 $\text{composition-final fri-query-idxs trace-round-layers}$
 $\text{composition-round-layers fr candidate-query-idxs trace-openings}$
 $\text{trace-table} \wedge$
 $\text{length trace-bs} = 0 \wedge$
 $\neg \text{fri-final-constant-consistent trace-table trace-final} \wedge$
 $\text{candidate-query-idxs} ! 0 \in$
 $\text{fri-final-value-agreement-set trace-table trace-final} \wedge$
 $\text{card (fri-final-value-agreement-set trace-table trace-final)} <$
 $\text{query-sample-space-size})$

definition *trace-fri-zero-round-candidate-first-query-alignment-gap*

$:: ('f, 'a) \text{ protocol-channel-scheme} \Rightarrow$
 $(\text{unit list} \times ('f, 'a) \text{ protocol-channel-scheme}) \text{ option} \Rightarrow \text{bool}$

where

$\text{trace-fri-zero-round-candidate-first-query-alignment-gap } s \text{ out} \longleftrightarrow$
 $(\exists \text{ trace-roots trace-bs trace-final dg composition-roots}$
 $\text{composition-bs composition-final fri-query-idxs trace-round-layers}$
 $\text{composition-round-layers fr candidate-query-idxs trace-openings}$
 trace-table.
 $\text{trace-fri-partial-candidate-opening-evidence } s \text{ out trace-roots}$
 $\text{trace-bs trace-final dg composition-roots composition-bs}$
 $\text{composition-final fri-query-idxs trace-round-layers}$
 $\text{composition-round-layers fr candidate-query-idxs trace-openings}$
 $\text{trace-table} \wedge$
 $\text{length trace-bs} = 0 \wedge$
 $\neg \text{fri-final-constant-consistent trace-table trace-final} \wedge$
 $\text{candidate-query-idxs} ! 0 \in$
 $\text{fri-final-value-agreement-set trace-table trace-final} \wedge$
 $\text{fri-query-idxs} ! 0 \notin$
 $\text{fri-final-value-agreement-set trace-table trace-final} \wedge$
 $\text{card} (\text{fri-final-value-agreement-set trace-table trace-final}) <$
 $\text{query-sample-space-size})$

definition $\text{trace-fri-zero-round-candidate-first-query-value-gap}$

$:: ('f, 'a) \text{ protocol-channel-scheme} \Rightarrow$
 $(\text{unit list} \times ('f, 'a) \text{ protocol-channel-scheme}) \text{ option} \Rightarrow \text{bool}$

where

$\text{trace-fri-zero-round-candidate-first-query-value-gap } s \text{ out} \longleftrightarrow$
 $(\exists \text{ trace-roots trace-bs trace-final dg composition-roots}$
 $\text{composition-bs composition-final fri-query-idxs trace-round-layers}$
 $\text{composition-round-layers fr candidate-query-idxs trace-openings}$
 trace-table.
 $\text{trace-fri-partial-candidate-opening-evidence } s \text{ out trace-roots}$
 $\text{trace-bs trace-final dg composition-roots composition-bs}$
 $\text{composition-final fri-query-idxs trace-round-layers}$
 $\text{composition-round-layers fr candidate-query-idxs trace-openings}$
 $\text{trace-table} \wedge$
 $\text{length trace-bs} = 0 \wedge$
 $\neg \text{fri-final-constant-consistent trace-table trace-final} \wedge$
 $\text{candidate-query-idxs} ! 0 \notin$
 $\text{fri-final-value-agreement-set trace-table trace-final} \wedge$
 $\text{card} (\text{fri-final-value-agreement-set trace-table trace-final}) <$
 $\text{query-sample-space-size})$

lemma $\text{trace-fri-zero-round-candidate-small-agreement-setE:}$

assumes $\text{trace-fri-zero-round-candidate-small-agreement-set } s \text{ out}$

obtains $\text{trace-roots trace-bs trace-final dg composition-roots}$
 $\text{composition-bs composition-final fri-query-idxs trace-round-layers}$
 $\text{composition-round-layers fr candidate-query-idxs trace-openings}$
 trace-table

where

trace-fri-partial-candidate-opening-evidence s out trace-roots
trace-bs trace-final dg composition-roots composition-bs
composition-final fri-query-idxs trace-round-layers
composition-round-layers fr candidate-query-idxs trace-openings
trace-table
length trace-bs = 0
 $\neg$ *fri-final-constant-consistent trace-table trace-final*
card (fri-final-value-agreement-set trace-table trace-final) <
query-sample-space-size

using *assms*

unfolding *trace-fri-zero-round-candidate-small-agreement-set-def*
by *blast*

lemma *trace-fri-zero-round-candidate-first-query-target-hitE:*

assumes *trace-fri-zero-round-candidate-first-query-target-hit s out*

obtains *trace-roots trace-bs trace-final dg composition-roots*
composition-bs composition-final fri-query-idxs trace-round-layers
composition-round-layers fr candidate-query-idxs trace-openings
trace-table

where

trace-fri-partial-candidate-opening-evidence s out trace-roots
trace-bs trace-final dg composition-roots composition-bs
composition-final fri-query-idxs trace-round-layers
composition-round-layers fr candidate-query-idxs trace-openings
trace-table
length trace-bs = 0
 $\neg$ *fri-final-constant-consistent trace-table trace-final*
candidate-query-idxs ! 0 ∈
fri-final-value-agreement-set trace-table trace-final
card (fri-final-value-agreement-set trace-table trace-final) <
query-sample-space-size

using *assms*

unfolding *trace-fri-zero-round-candidate-first-query-target-hit-def*
by *blast*

lemma *trace-fri-zero-round-candidate-first-query-alignment-gapE:*

assumes *trace-fri-zero-round-candidate-first-query-alignment-gap s out*

obtains *trace-roots trace-bs trace-final dg composition-roots*
composition-bs composition-final fri-query-idxs trace-round-layers
composition-round-layers fr candidate-query-idxs trace-openings
trace-table

where

trace-fri-partial-candidate-opening-evidence s out trace-roots
trace-bs trace-final dg composition-roots composition-bs
composition-final fri-query-idxs trace-round-layers
composition-round-layers fr candidate-query-idxs trace-openings
trace-table
length trace-bs = 0

$\neg$ *fri-final-constant-consistent* *trace-table* *trace-final*
candidate-query-idxs ! 0 $\in$
fri-final-value-agreement-set *trace-table* *trace-final*
fri-query-idxs ! 0 $\notin$
fri-final-value-agreement-set *trace-table* *trace-final*
card (*fri-final-value-agreement-set* *trace-table* *trace-final*) <
query-sample-space-size
using *assms*
unfolding *trace-fri-zero-round-candidate-first-query-alignment-gap-def*
by *blast*

lemma *trace-fri-zero-round-candidate-first-query-value-gapE*:
assumes *trace-fri-zero-round-candidate-first-query-value-gap* *s* *out*
obtains *trace-roots* *trace-bs* *trace-final* *dg* *composition-roots*
composition-bs *composition-final* *fri-query-idxs* *trace-round-layers*
composition-round-layers *fr* *candidate-query-idxs* *trace-openings*
trace-table
where
trace-fri-partial-candidate-opening-evidence *s* *out* *trace-roots*
trace-bs *trace-final* *dg* *composition-roots* *composition-bs*
composition-final *fri-query-idxs* *trace-round-layers*
composition-round-layers *fr* *candidate-query-idxs* *trace-openings*
trace-table
length *trace-bs* = 0
 $\neg$ *fri-final-constant-consistent* *trace-table* *trace-final*
candidate-query-idxs ! 0 $\notin$
fri-final-value-agreement-set *trace-table* *trace-final*
card (*fri-final-value-agreement-set* *trace-table* *trace-final*) <
query-sample-space-size
using *assms*
unfolding *trace-fri-zero-round-candidate-first-query-value-gap-def*
by *blast*

lemma *trace-fri-zero-round-candidate-first-query-target-hit-imp-small*:
assumes *trace-fri-zero-round-candidate-first-query-target-hit* *s* *out*
shows *trace-fri-zero-round-candidate-small-agreement-set* *s* *out*
proof (*rule* *trace-fri-zero-round-candidate-first-query-target-hitE* [*OF* *assms*])
fix *trace-roots* *trace-bs* *trace-final* *dg* *composition-roots*
composition-bs *composition-final* *fri-query-idxs* *trace-round-layers*
composition-round-layers *fr* *candidate-query-idxs* *trace-openings*
trace-table
assume *evidence*:
trace-fri-partial-candidate-opening-evidence *s* *out* *trace-roots*
trace-bs *trace-final* *dg* *composition-roots* *composition-bs*
composition-final *fri-query-idxs* *trace-round-layers*
composition-round-layers *fr* *candidate-query-idxs* *trace-openings*
trace-table
and *rounds-empty*: *length* *trace-bs* = 0
and *not-final*:

```

    ¬ fri-final-constant-consistent trace-table trace-final
  and card-lt:
    card (fri-final-value-agreement-set trace-table trace-final) <
      query-sample-space-size
  show ?thesis
    unfolding trace-fri-zero-round-candidate-small-agreement-set-def
    by (intro exI conjI)
      (rule evidence, rule rounds-empty, rule not-final, rule card-lt)
qed

lemma trace-fri-zero-round-checked-first-query-target-hit-imp-candidate:
  assumes trace-fri-zero-round-checked-first-query-target-hit s out
  shows trace-fri-zero-round-candidate-first-query-target-hit s out
proof -
  from assms obtain trace-roots trace-bs trace-final dg composition-roots
    composition-bs composition-final fri-query-idxs trace-round-layers
    composition-round-layers fr trace-openings trace-table
  where fri-openings:
    accepted-fri-opening-transcript s out trace-roots trace-bs
    trace-final dg composition-roots composition-bs composition-final
    fri-query-idxs trace-round-layers composition-round-layers
  and rounds-empty: length trace-bs = 0
  and checked:
    trace-zero-round-final-checked-candidate s out fr fri-query-idxs
    trace-openings trace-table trace-final
  and not-final:
    ¬ fri-final-constant-consistent trace-table trace-final
  and first-hit:
    fri-query-idxs ! 0 ∈
    fri-final-value-agreement-set trace-table trace-final
  and card-lt:
    card (fri-final-value-agreement-set trace-table trace-final) <
      query-sample-space-size
  unfolding trace-fri-zero-round-checked-first-query-target-hit-def
  by blast
  have partial:
    accepted-with-partial-trace-openings s out fr fri-query-idxs
    trace-openings
  using checked unfolding trace-zero-round-final-checked-candidate-def
  by simp
  have candidate:
    partial-trace-table-candidate trace-table trace-openings
  using checked unfolding trace-zero-round-final-checked-candidate-def
  by simp
  have not-low:
    ¬ trace-table-low-degree trace-table
  by (rule trace-zero-round-checked-candidate-not-trace-low-degree
    [OF rounds-empty fri-openings checked not-final])
  have evidence:

```

```

    trace-fri-partial-candidate-opening-evidence s out trace-roots
    trace-bs trace-final dg composition-roots composition-bs
    composition-final fri-query-idxs trace-round-layers
    composition-round-layers fr fri-query-idxs trace-openings trace-table
unfolding trace-fri-partial-candidate-opening-evidence-def
    trace-fri-partial-candidate-evidence-def
by (intro conjI fri-openings partial candidate not-low)
show ?thesis
unfolding trace-fri-zero-round-candidate-first-query-target-hit-def
by (intro exI conjI)
    (rule evidence, rule rounds-empty, rule not-final, rule first-hit,
     rule card-lt)
qed

lemma trace-fri-zero-round-final-obstruction-imp-candidate-small-agreement-set:
  assumes trace-fri-zero-round-final-obstruction s out
  shows trace-fri-zero-round-candidate-small-agreement-set s out
proof (rule trace-fri-zero-round-final-obstruction-agreement-set-card-lt
  [OF assms])
  fix trace-roots trace-bs trace-final dg composition-roots
  composition-bs composition-final fri-query-idxs trace-round-layers
  composition-round-layers fr candidate-query-idxs trace-openings
  trace-table
  assume evidence:
    trace-fri-partial-candidate-opening-evidence s out trace-roots
    trace-bs trace-final dg composition-roots composition-bs
    composition-final fri-query-idxs trace-round-layers
    composition-round-layers fr candidate-query-idxs trace-openings
    trace-table
  and rounds-empty: length trace-bs = 0
  and not-final:
     $\neg$  fri-final-constant-consistent trace-table trace-final
  and card-lt:
    card (fri-final-value-agreement-set trace-table trace-final) <
    query-sample-space-size
  show ?thesis
  unfolding trace-fri-zero-round-candidate-small-agreement-set-def
  by (intro exI conjI)
    (rule evidence, rule rounds-empty, rule not-final, rule card-lt)
qed

lemma wp-trace-fri-zero-round-final-obstruction-bound-from-candidate-small-agreement-set:
  assumes small-bound:
    wp-event verify-monad
    (trace-fri-zero-round-candidate-small-agreement-set s)  $s \leq T$ 
  shows
    wp-event verify-monad
    (trace-fri-zero-round-final-obstruction s)  $s \leq T$ 
proof –

```

```

have wp-event verify-monad
  (trace-fri-zero-round-final-obstruction s) s ≤
  wp-event verify-monad
    (trace-fri-zero-round-candidate-small-agreement-set s) s
by (rule wp-event-mono)
  (rule
    trace-fri-zero-round-final-obstruction-imp-candidate-small-agreement-set)
then show ?thesis
  by (rule order-trans[OF - small-bound])
qed

end

end

```

```

theory Soundness-Query-Transcript-Openings
imports Soundness-Query-Opening-Consistency
begin

```

Combined query-round extraction lemmas.

The existing execution layer extracts transcript chunks and authenticated Merkle openings separately. This layer keeps the witnesses coupled to the same verifier query-round execution, so downstream transcript-indexed reductions can use verifier-derived query indices.

```

context soundness
begin

```

```

lemma verifier-query-round-program-outcome-with-authenticated-trace-openings:
fixes s t :: ('f, 'a) protocol-channel-scheme
assumes outcome:
  Some ((), t) ∈
    set-dist
      (execute
        (verifier-query-round-program fr f-fl f-final as fl final) s)
obtains raw idx chunk openings where
  idx = index (to-nat raw)
  verifier-query-round-chunk idx (map snd f-fl) (map snd fl) chunk
  PTranscript s = chunk @ PTranscript t
  PState t = foldl concat (PState s) chunk
  s ≤ t
  PQueryCounter t = Suc (PQueryCounter s)
  fmlookup (HashMap t) (QueryIndexChallenge (PQueryCounter s) (PState s)) =
    Some raw
  length openings = length (powers-scaled idx)
  map opening-index openings = powers-scaled idx
  partial-authenticated-table fr (scale * clength) openings t
proof –
from outcome[unfolded verifier-query-round-program-alt-def]

```

```

obtain raw s0 where
  rand:
    Some (raw, s0) ∈
      set-dist (execute receive-query-index-challenge s)
  and tail:
    Some ((), t) ∈
      set-dist
        (execute
          (verifier-query-round-after-index-program
            fr f-fl f-final as fl final raw) s0)
  by (auto elim!: set-dist-bindE)
let ?idx = index (to-nat raw)
from tail obtain fv s1 f-out s2 s3 c-out s4 where
  query-decommit:
    Some (fv, s1) ∈
      set-dist (execute (mmap (check-decommit-on-query fr ?idx)) s0)
  and trace-fri:
    Some (f-out, s2) ∈
      set-dist (execute
        (mfold (?idx, hd fv, clength * scale, 1)
          (receive-query-commits f-fl)) s1)
  and assert-trace:
    Some ((), s3) ∈
      set-dist
        (execute
          (assert (case f-out of (-, f-x, -, -) ⇒ f-x = f-final)) s2)
  and comp-fri:
    Some (c-out, s4) ∈
      set-dist (execute
        (mfold
          (?idx, cp-eval as fv (h ^ ?idx * shift), clength * scale, 1)
          (receive-query-commits fl)) s3)
  and assert-comp:
    Some ((), t) ∈
      set-dist
        (execute (assert (case c-out of (-, x, -, -) ⇒ x = final)) s4)
  unfolding verifier-query-round-after-index-program-def
  by (auto simp: Let-def split: prod.splits elim!: set-dist-bindE)
have rand-res:
  s ≤ s0 ∧
    PState s0 = PState s ∧
    PTranscript s0 = PTranscript s ∧
    fmlookup (HashMap s0)
      (QueryIndexChallenge (PQueryCounter s) (PState s)) = Some raw
  using receive-query-index-challenge-outcome[OF rand] .
have rand-counter:
  PQueryCounter s0 = Suc (PQueryCounter s)
  using receive-query-index-challenge-counter-outcome[OF rand] by simp
from check-decommit-on-query-outcome[OF query-decommit]

```



```

obtain query-paths query-chunk where
  query-chunk-shape:
    query-decommitment-transcript ?idx fv query-paths query-chunk
  and tr-s0: PTranscript s0 = query-chunk @ PTranscript s1
  and st-s1: PState s1 = foldl concat (PState s0) query-chunk
  and ext-s0-s1: s0 ≤ s1
  and query-count-s1: PQueryCounter s1 = PQueryCounter s0
  by blast
from receive-query-commits-layers-outcome[OF trace-fri]
obtain trace-layer-chunks trace-fri-chunk where
  trace-layers:
    fri-layers-transcript (length f-fl) (clength * scale)
    trace-layer-chunks trace-fri-chunk
  and tr-s1: PTranscript s1 = trace-fri-chunk @ PTranscript s2
  and st-s2: PState s2 = foldl concat (PState s1) trace-fri-chunk
  and ext-s1-s2: s1 ≤ s2
  and query-count-s2: PQueryCounter s2 = PQueryCounter s1
  by blast
have s3-eq: s3 = s2
  using assert-trace unfolding assert-def
  by (cases case f-out of (-, f-x, -, -) ⇒ f-x = f-final)
    (auto simp: throw-no-outcome)
from receive-query-commits-layers-outcome[OF comp-fri]
obtain comp-layer-chunks comp-fri-chunk where
  comp-layers:
    fri-layers-transcript (length fl) (clength * scale)
    comp-layer-chunks comp-fri-chunk
  and tr-s3: PTranscript s3 = comp-fri-chunk @ PTranscript s4
  and st-s4: PState s4 = foldl concat (PState s3) comp-fri-chunk
  and ext-s3-s4: s3 ≤ s4
  and query-count-s4: PQueryCounter s4 = PQueryCounter s3
  by blast
have t-eq: t = s4
  using assert-comp unfolding assert-def
  by (cases case c-out of (-, x, -, -) ⇒ x = final)
    (auto simp: throw-no-outcome)
let ?chunk = query-chunk @ trace-fri-chunk @ comp-fri-chunk
have round-chunk:
  verifier-query-round-chunk ?idx (map snd f-fl) (map snd fl) ?chunk
  unfolding verifier-query-round-chunk-def
  apply (rule exI[where x=query-chunk])
  apply (rule exI[where x=trace-fri-chunk])
  apply (rule exI[where x=comp-fri-chunk])
  apply (rule exI[where x=fv])
  apply (rule exI[where x=query-paths])
  apply (rule exI[where x=trace-layer-chunks])
  apply (rule exI[where x=comp-layer-chunks])
  using query-chunk-shape trace-layers comp-layers
  by simp

```

```

have tr-t: PTranscript s = ?chunk @ PTranscript t
  using rand-res tr-s0 tr-s1 tr-s3 s3-eq t-eq by simp
have st-t: PState t = foldl concat (PState s) ?chunk
  using rand-res st-s1 st-s2 st-s4 s3-eq t-eq by simp
have ext-s0-t: s0 ≤ t
  using ext-s0-s1 ext-s1-s2 ext-s3-s4 t-eq unfolding s3-eq
  by (meson hash-ext-trans)
have ext-s-t: s ≤ t
  using rand-res ext-s0-t by (meson hash-ext-trans)
have query-count-t: PQueryCounter t = Suc (PQueryCounter s)
  using rand-counter query-count-s1 query-count-s2 query-count-s4
  unfolding s3-eq t-eq by simp
have lookup-t:
  fmlookup (HashMap t)
  (QueryIndexChallenge (PQueryCounter s) (PState s)) = Some raw
proof -
  let ?key = QueryIndexChallenge (PQueryCounter s) (PState s)
  have fmlookup (HashMap s0) ?key = fmlookup (HashMap t) ?key
    using ext-s0-t rand-res
    unfolding less-eq-hash-ext-def less-eq-fmap-def
    by (metis option.distinct(1))
  then show ?thesis
    using rand-res by simp
qed
from verifier-query-round-after-index-authenticated-trace-openings[OF tail]
obtain idx' leaves openings where
  idx'-eq: idx' = ?idx
  and len-openings: length openings = length (powers-scaled idx')
  and idx-openings: map opening-index openings = powers-scaled idx'
  and table-t:
    partial-authenticated-table fr (scale * clength) openings t
  by blast
show ?thesis
proof (rule that)
  show ?idx = index (to-nat raw)
    by simp
  show verifier-query-round-chunk ?idx (map snd f-fl) (map snd fl)
    ?chunk
    by (rule round-chunk)
  show PTranscript s = ?chunk @ PTranscript t
    by (rule tr-t)
  show PState t = foldl concat (PState s) ?chunk
    by (rule st-t)
  show s ≤ t
    by (rule ext-s-t)
  show PQueryCounter t = Suc (PQueryCounter s)
    by (rule query-count-t)
  show fmlookup (HashMap t)
    (QueryIndexChallenge (PQueryCounter s) (PState s)) = Some raw

```

```

    by (rule lookup-t)
  show length openings = length (powers-scaled ?idx)
    using idx'-eq len-openings by simp
  show map opening-index openings = powers-scaled ?idx
    using idx'-eq idx-openings by simp
  show partial-authenticated-table fr (scale * clength) openings t
    by (rule table-t)
qed
qed

lemma ntimes-verifier-query-rounds-selected-trace-openings-aligned-with-replay-at:
  fixes query-state final-state :: ('f, 'a) protocol-channel-scheme
  assumes outcome:
    Some (results, final-state) ∈
      set-dist
        (execute
          (ntimes
            (verifier-query-round-program fr f-fl f-final as fl final)
            rounds)
          query-state)
  and i-bound: i < rounds
  and len-raw: length raw-idxs = rounds
  and query-idxs-eq:
    query-idxs = map (λraw. index (to-nat raw)) raw-idxs
  and len-query-chunks: length query-chunks = rounds
  and transcript-query:
    PTranscript query-state =
      List.concat query-chunks @ PTranscript final-state
  and query-chunk:
    ∧ i. i < rounds ⇒
      verifier-query-round-chunk (query-idxs ! i)
        (map snd f-fl) (map snd fl) (query-chunks ! i)
  and replay-lookup:
    ∧ i. i < rounds ⇒
      fmlookup (HashMap final-state)
        (QueryIndexChallenge (PQueryCounter query-state + i)
          (state-after-query-chunks
            (PState query-state) query-chunks i)) =
        Some (raw-idxs ! i)
  obtains openings where
    map opening-index openings = powers-scaled (query-idxs ! i)
    partial-authenticated-table fr (scale * clength) openings final-state
proof -
  let ?round = verifier-query-round-program fr f-fl f-final as fl final
  from query-opening-ntimes-outcome-decomp-at[OF outcome i-bound]
  obtain prefix x suffix s-i s-suc where
    prefix:
      Some (prefix, s-i) ∈
        set-dist (execute (ntimes ?round i) query-state)

```

```

and round:
  Some (x, s-suc) ∈ set-dist (execute ?round s-i)
and suffix:
  Some (suffix, final-state) ∈
    set-dist (execute (ntimes ?round (rounds - Suc i)) s-suc)
by blast
have x-eq: x = ()
  by (cases x) simp
show ?thesis
proof (rule query-opening-ntimes-prefix-transcript-state-alignment
  [OF prefix])
fix prefix-query-idxs prefix-chunks
assume len-prefix-chunks: length prefix-chunks = i
and transcript-prefix:
  PTranscript query-state =
    List.concat prefix-chunks @ PTranscript s-i
and state-i:
  PState s-i =
    state-after-query-chunks (PState query-state) prefix-chunks i
and counter-i: PQueryCounter s-i = PQueryCounter query-state + i
and prefix-rounds:
   $\bigwedge k. k < i \implies$ 
    verifier-query-round-chunk (prefix-query-idxs ! k)
      (map snd f-fl) (map snd fl) (prefix-chunks ! k)
show ?thesis
proof (rule
  verifier-query-round-program-outcome-with-authenticated-trace-openings
  [OF round[unfolded x-eq]])
fix raw idx chunk openings
assume idx-eq: idx = index (to-nat raw)
and chunk-shape:
  verifier-query-round-chunk idx (map snd f-fl) (map snd fl)
    chunk
and transcript-round: PTranscript s-i = chunk @ PTranscript s-suc
and lookup-suc:
  fmlookup (HashMap s-suc)
    (QueryIndexChallenge (PQueryCounter s-i) (PState s-i)) =
    Some raw
and opening-indices: map opening-index openings = powers-scaled idx
and auth-table-suc:
  partial-authenticated-table fr (scale * clength) openings s-suc
show ?thesis
proof (rule query-opening-ntimes-suffix-transcript-state-alignment
  [OF suffix])
fix suffix-query-idxs suffix-chunks
assume len-suffix-chunks:
  length suffix-chunks = rounds - Suc i
and transcript-suffix:
  PTranscript s-suc =

```

```

    List.concat suffix-chunks @ PTranscript final-state
  and suffix-ext: s-suc ≤ final-state
  and suffix-rounds:
     $\bigwedge k. k < \text{rounds} - \text{Suc } i \implies$ 
      verifier-query-round-chunk (suffix-query-idxs ! k)
      (map snd f-fl) (map snd fl) (suffix-chunks ! k)
  let ?selected-chunks = prefix-chunks @ chunk # suffix-chunks
  have len-selected-chunks: length ?selected-chunks = rounds
    using len-prefix-chunks len-suffix-chunks i-bound by simp
  have selected-chunk-lens:
     $\bigwedge k. k < \text{rounds} \implies$ 
      length (?selected-chunks ! k) =
        verifier-query-round-transcript-length (query-idxs ! k)
        (map snd f-fl) (map snd fl)
  proof -
    fix k
    assume k-bound: k < rounds
    show
      length (?selected-chunks ! k) =
        verifier-query-round-transcript-length (query-idxs ! k)
        (map snd f-fl) (map snd fl)
  proof (cases k < i)
    case True
      have nth-eq: ?selected-chunks ! k = prefix-chunks ! k
        using True len-prefix-chunks by (simp add: nth-append)
      have
        length (prefix-chunks ! k) =
          verifier-query-round-transcript-length
            (prefix-query-idxs ! k) (map snd f-fl) (map snd fl)
        by (rule verifier-query-round-chunk-length
            [OF prefix-rounds[OF True]])
      then show ?thesis
        using nth-eq
        by (simp add:
            verifier-query-round-transcript-length-index-irrelevant)
    next
      case False
      show ?thesis
      proof (cases k = i)
        case True
          have nth-eq: ?selected-chunks ! k = chunk
            using True len-prefix-chunks by (simp add: nth-append)
          have
            length chunk =
              verifier-query-round-transcript-length idx
                (map snd f-fl) (map snd fl)
            by (rule verifier-query-round-chunk-length[OF chunk-shape])
          then show ?thesis
            using nth-eq

```

```

    by (simp add:
        verifier-query-round-transcript-length-index-irrelevant)
  next
  case False
  let ?k = k - Suc i
  have ge: Suc i ≤ k
    using ⟨¬ k < i⟩ False by linarith
  have k-bound': ?k < rounds - Suc i
    using ge k-bound by linarith
  have nth-eq: ?selected-chunks ! k = suffix-chunks ! ?k
    using ge len-prefix-chunks by (simp add: nth-append)
  have
    length (suffix-chunks ! ?k) =
      verifier-query-round-transcript-length
        (suffix-query-idxs ! ?k) (map snd f-fl) (map snd fl)
    by (rule verifier-query-round-chunk-length
        [OF suffix-rounds[OF k-bound']])
  then show ?thesis
    using nth-eq
    by (simp add:
        verifier-query-round-transcript-length-index-irrelevant)
  qed
qed
qed
have query-chunk-lens:
  ∧k. k < rounds ⇒
    length (query-chunks ! k) =
      verifier-query-round-transcript-length (query-idxs ! k)
        (map snd f-fl) (map snd fl)
  using query-chunk verifier-query-round-chunk-length by blast
have transcript-all:
  PTranscript query-state =
    List.concat ?selected-chunks @ PTranscript final-state
  using transcript-prefix transcript-round transcript-suffix by simp
have concat-eq:
  List.concat ?selected-chunks = List.concat query-chunks
  using transcript-all transcript-query by simp
have chunks-eq: ?selected-chunks = query-chunks
proof -
  have same-lens:
    ∧k. k < length ?selected-chunks ⇒
      length (?selected-chunks ! k) = length (query-chunks ! k)
    using selected-chunk-lens query-chunk-lens len-selected-chunks
    by simp
  have concat-eq-empty:
    List.concat ?selected-chunks @ [] = List.concat query-chunks
    using concat-eq by simp
  have len-eq: length ?selected-chunks = length query-chunks
    using len-selected-chunks len-query-chunks by simp

```

```

have ?selected-chunks = query-chunks  $\wedge$  ( $[] :: 'f \text{ list}$ ) = []
  by (rule concat-append-eq-concat-same-chunk-lengths
    [OF len-eq same-lens concat-eq-empty])
then show ?thesis
  by simp
qed
have prefix-state-eq:
  state-after-query-chunks (PState query-state) prefix-chunks i =
    state-after-query-chunks (PState query-state) query-chunks i
proof -
  have
    state-after-query-chunks (PState query-state)
      ?selected-chunks i =
        state-after-query-chunks (PState query-state)
          prefix-chunks i
  by (rule state-after-query-chunks-append-prefix
    [OF len-prefix-chunks])
  then show ?thesis
    using chunks-eq by simp
qed
have lookup-final:
  fmlookup (HashMap final-state)
    (QueryIndexChallenge (PQueryCounter query-state + i)
      (state-after-query-chunks
        (PState query-state) query-chunks i)) =
    Some raw
  using hash-extension-lookup[OF lookup-suc suffix-ext]
  unfolding counter-i state-i prefix-state-eq .
have raw-eq: raw = raw-idxs ! i
  using lookup-final replay-lookup[OF i-bound] by simp
have idx-eq-query: idx = query-idxs ! i
  using idx-eq raw-eq query-idxs-eq len-raw i-bound by simp
have auth-table-final:
  partial-authenticated-table fr (scale * clength) openings
    final-state
  by (rule partial-authenticated-table-mono
    [OF auth-table-suc suffix-ext])
show ?thesis
  by (rule that[OF - auth-table-final])
    (use opening-indices idx-eq-query in simp)
qed
qed
qed
qed

lemma verifier-query-round-program-outcome-with-zero-trace-final-opening:
  fixes s t :: ('f, 'a) protocol-channel-scheme
  assumes f-fl-empty: f-fl = []
  and outcome:

```

```

    Some (( ), t) ∈
      set-dist
        (execute
          (verifier-query-round-program fr f-fl f-final as fl final) s)
obtains raw idx chunk openings where
  idx = index (to-nat raw)
  verifier-query-round-chunk idx [] (map snd fl) chunk
  PTranscript s = chunk @ PTranscript t
  PState t = foldl concat (PState s) chunk
  s ≤ t
  PQueryCounter t = Suc (PQueryCounter s)
  fmlookup (HashMap t) (QueryIndexChallenge (PQueryCounter s) (PState s)) =
    Some raw
  map opening-index openings = powers-scaled idx
  partial-authenticated-table fr (scale * clength) openings t
  opening-value (hd openings) = f-final
proof –
from outcome[unfolded verifier-query-round-program-alt-def]
obtain raw s0 where
  rand:
    Some (raw, s0) ∈
      set-dist (execute receive-query-index-challenge s)
and tail:
  Some (( ), t) ∈
    set-dist
      (execute
        (verifier-query-round-after-index-program
          fr f-fl f-final as fl final raw) s0)
by (auto elim!: set-dist-bindE)
let ?idx = index (to-nat raw)
from tail obtain fv s1 f-out s2 s3 c-out s4 where
  query-decommit:
    Some (fv, s1) ∈
      set-dist (execute (mmap (check-decommit-on-query fr ?idx)) s0)
and trace-fri:
  Some (f-out, s2) ∈
    set-dist (execute
      (mfold (?idx, hd fv, clength * scale, 1)
        (receive-query-commits f-fl)) s1)
and assert-trace:
  Some (( ), s3) ∈
    set-dist
      (execute
        (assert (case f-out of (_, f-x, -, -) ⇒ f-x = f-final)) s2)
and comp-fri:
  Some (c-out, s4) ∈
    set-dist (execute
      (mfold
        (?idx, cp-eval as fv (h ^ ?idx * shift), clength * scale, 1)

```



```

      (receive-query-commits fl)) s3)
and assert-comp:
  Some ((), t) ∈
    set-dist
      (execute (assert (case c-out of (-, x, -, -) ⇒ x = final)) s4)
unfolding verifier-query-round-after-index-program-def
by (auto simp: Let-def split: prod.splits elim!: set-dist-bindE)
have rand-res:
  s ≤ s0 ∧
  PState s0 = PState s ∧
  PTranscript s0 = PTranscript s ∧
  fmlookup (HashMap s0)
    (QueryIndexChallenge (PQueryCounter s) (PState s)) = Some raw
using receive-query-index-challenge-outcome[OF rand] .
have rand-counter:
  PQueryCounter s0 = Suc (PQueryCounter s)
using receive-query-index-challenge-counter-outcome[OF rand] by simp
from check-decommit-on-query-outcome[OF query-decommit]
obtain query-paths query-chunk where
  query-chunk-shape:
    query-decommitment-transcript ?idx fv query-paths query-chunk
and tr-s0: PTranscript s0 = query-chunk @ PTranscript s1
and st-s1: PState s1 = foldl concat (PState s0) query-chunk
and ext-s0-s1: s0 ≤ s1
and query-count-s1: PQueryCounter s1 = PQueryCounter s0
by blast
have trace-out:
  f-out = (?idx, hd fv, clength * scale, 1)
and s2-eq: s2 = s1
using trace-fri f-fl-empty
unfolding receive-query-commits-def by simp-all
have hd-fv: hd fv = f-final
using assert-trace unfolding assert-def trace-out
by (cases hd fv = f-final) (auto simp: throw-no-outcome)
have s3-eq: s3 = s2
using assert-trace unfolding assert-def trace-out hd-fv by simp
from receive-query-commits-layers-outcome[OF comp-fri]
obtain comp-layer-chunks comp-fri-chunk where
  comp-layers:
    fri-layers-transcript (length fl) (clength * scale)
    comp-layer-chunks comp-fri-chunk
and tr-s3: PTranscript s3 = comp-fri-chunk @ PTranscript s4
and st-s4: PState s4 = foldl concat (PState s3) comp-fri-chunk
and ext-s3-s4: s3 ≤ s4
and query-count-s4: PQueryCounter s4 = PQueryCounter s3
by blast
have t-eq: t = s4
using assert-comp unfolding assert-def
by (cases case c-out of (-, x, -, -) ⇒ x = final)

```

```

(auto simp: throw-no-outcome)
have trace-layers-empty:
  fri-layers-transcript 0 (clength * scale) [] []
  unfolding fri-layers-transcript-def by simp
let ?chunk = query-chunk @ [] @ comp-fri-chunk
have round-chunk:
  verifier-query-round-chunk ?idx [] (map snd fl) ?chunk
  unfolding verifier-query-round-chunk-def
  apply (rule exI[where x=query-chunk])
  apply (rule exI[where x=[]])
  apply (rule exI[where x=comp-fri-chunk])
  apply (rule exI[where x=fv])
  apply (rule exI[where x=query-paths])
  apply (rule exI[where x=[]])
  apply (rule exI[where x=comp-layer-chunks])
  using query-chunk-shape trace-layers-empty comp-layers
  by simp
have tr-t: PTranscript s = ?chunk @ PTranscript t
  using rand-res tr-s0 tr-s3 s3-eq s2-eq t-eq by simp
have st-t: PState t = foldl concat (PState s) ?chunk
  using rand-res st-s1 st-s4 s3-eq s2-eq t-eq by simp
have ext-s0-t: s0 ≤ t
  using ext-s0-s1 ext-s3-s4 t-eq unfolding s3-eq s2-eq
  by (meson hash-ext-trans)
have ext-s-t: s ≤ t
  using rand-res ext-s0-t by (meson hash-ext-trans)
have query-count-t: PQueryCounter t = Suc (PQueryCounter s)
  using rand-counter query-count-s1 query-count-s4
  unfolding s3-eq s2-eq t-eq by simp
have lookup-t:
  fmlookup (HashMap t)
  (QueryIndexChallenge (PQueryCounter s) (PState s)) = Some raw
  using hash-extension-lookup[OF conjunct2[OF conjunct2[OF conjunct2[OF rand-res]]] ext-s0-t]
  rand-res
  by simp
have idx-in: ?idx ∈ query-sample-space
  using index-less-query-sample-space
  unfolding query-sample-space-def by simp
from check-decommit-on-query-authenticated-openings
  [OF idx-in query-decommit]
obtain openings where values-openings:
  map opening-value openings = fv
  and idx-openings: map opening-index openings = powers-scaled ?idx
  and table-s1:
    partial-authenticated-table fr (scale * clength) openings s1
  by blast
have s1-t: s1 ≤ t
  using ext-s3-s4 unfolding s3-eq s2-eq t-eq .
have table-t:

```

```

    partial-authenticated-table fr (scale * clength) openings t
  by (rule partial-authenticated-table-mono[OF table-s1 s1-t])
have openings-nonempty: openings ≠ []
proof -
  have map opening-index openings ≠ []
    using idx-openings powers-pos unfolding powers-scaled-def by simp
  then show ?thesis by auto
qed
have hd-value: opening-value (hd openings) = hd fv
  using values-openings openings-nonempty
  by (cases openings; cases fv) simp-all
show ?thesis
proof (rule that)
  show ?idx = index (to-nat raw)
    by simp
  show verifier-query-round-chunk ?idx [] (map snd fl) ?chunk
    by (rule round-chunk)
  show PTranscript s = ?chunk @ PTranscript t
    by (rule tr-t)
  show PState t = foldl concat (PState s) ?chunk
    by (rule st-t)
  show s ≤ t
    by (rule ext-s-t)
  show PQueryCounter t = Suc (PQueryCounter s)
    by (rule query-count-t)
  show fmllookup (HashMap t)
    (QueryIndexChallenge (PQueryCounter s) (PState s)) = Some raw
    by (rule lookup-t)
  show map opening-index openings = powers-scaled ?idx
    by (rule idx-openings)
  show partial-authenticated-table fr (scale * clength) openings t
    by (rule table-t)
  show opening-value (hd openings) = f-final
    using hd-value hd-fv by simp
qed
qed

lemma ntimes-verifier-query-rounds-selected-empty-trace-chunks-at:
  fixes s t :: ('f, 'a) protocol-channel-scheme
  assumes fl-empty: fl = []
  and outcome:
    Some (results, t) ∈
      set-dist
        (execute
          (ntimes
            (verifier-query-round-program fr f-fl f-final as fl final) n)
          s)
  and i-bound: i < n
  obtains raw idx prefix-query-idxs prefix-chunks chunk suffix-query-idxs

```

```

    suffix-chunks trace-openings where
    idx = index (to-nat raw)
    length prefix-query-idxs = i
    length prefix-chunks = i
    length suffix-query-idxs = n - Suc i
    length suffix-chunks = n - Suc i
    verifier-query-round-chunk idx (map snd f-fl) [] chunk
     $\bigwedge j. j < i \implies$ 
      verifier-query-round-chunk (prefix-query-idxs ! j)
        (map snd f-fl) [] (prefix-chunks ! j)
     $\bigwedge j. j < n - \text{Suc } i \implies$ 
      verifier-query-round-chunk (suffix-query-idxs ! j)
        (map snd f-fl) [] (suffix-chunks ! j)
    PTranscript s =
      List.concat (prefix-chunks @ chunk # suffix-chunks) @ PTranscript t
    map opening-index trace-openings = powers-scaled idx
    partial-authenticated-table fr (scale * clength) trace-openings t
    fmlookup (HashMap t)
      (QueryIndexChallenge (PQueryCounter s + i)
        (state-after-query-chunks (PState s) prefix-chunks i)) = Some raw
proof -
from query-opening-ntimes-outcome-decomp-at[OF outcome i-bound]
obtain prefix round suffix s-i s-suc where
  prefix:
    Some (prefix, s-i)  $\in$ 
      set-dist
        (execute
          (ntimes
            (verifier-query-round-program fr f-fl f-final as fl final) i)
          s)
  and round:
    Some (round, s-suc)  $\in$ 
      set-dist
        (execute
          (verifier-query-round-program fr f-fl f-final as fl final) s-i)
  and suffix:
    Some (suffix, t)  $\in$ 
      set-dist
        (execute
          (ntimes
            (verifier-query-round-program fr f-fl f-final as fl final)
            (n - Suc i))
          s-suc)
  by blast
have round-unit: round = ()
by (cases round) simp
show ?thesis
proof (rule query-opening-ntimes-prefix-transcript-state-alignment
  [OF prefix])

```

```

fix prefix-query-idxs prefix-chunks
assume len-prefix: length prefix-query-idxs = i
and len-prefix-chunks: length prefix-chunks = i
and transcript-prefix:
  PTranscript s = List.concat prefix-chunks @ PTranscript s-i
and state-i:
  PState s-i =
    state-after-query-chunks (PState s) prefix-chunks i
and counter-i: PQueryCounter s-i = PQueryCounter s + i
and prefix-ext: s ≤ s-i
and prefix-rounds:
   $\bigwedge j. j < i \implies$ 
    verifier-query-round-chunk (prefix-query-idxs ! j)
      (map snd f-fl) (map snd fl) (prefix-chunks ! j)
show ?thesis
proof (rule
  verifier-query-round-program-outcome-with-authenticated-trace-openings
    [OF round[unfolded round-unit]])
fix raw idx chunk trace-openings
assume idx-eq: idx = index (to-nat raw)
and chunk-shape:
  verifier-query-round-chunk idx (map snd f-fl) (map snd fl)
    chunk
and transcript-round:
  PTranscript s-i = chunk @ PTranscript s-suc
and state-suc:
  PState s-suc = foldl concat (PState s-i) chunk
and round-ext: s-i ≤ s-suc
and counter-suc: PQueryCounter s-suc = Suc (PQueryCounter s-i)
and lookup-suc:
  fmlookup (HashMap s-suc)
    (QueryIndexChallenge (PQueryCounter s-i) (PState s-i)) =
    Some raw
and len-trace:
  length trace-openings = length (powers-scaled idx)
and trace-indices:
  map opening-index trace-openings = powers-scaled idx
and trace-table-suc:
  partial-authenticated-table fr (scale * clength)
    trace-openings s-suc
show ?thesis
proof (rule query-opening-ntimes-suffix-transcript-state-alignment
  [OF suffix])
fix suffix-query-idxs suffix-chunks
assume len-suffix-idxs:
  length suffix-query-idxs = n - Suc i
and len-suffix-chunks:
  length suffix-chunks = n - Suc i
and transcript-suffix:

```

```

    PTranscript s-suc =
      List.concat suffix-chunks @ PTranscript t
  and suffix-state:
    PState t =
      state-after-query-chunks (PState s-suc) suffix-chunks
      (n - Suc i)
  and suffix-counter:
    PQueryCounter t = PQueryCounter s-suc + (n - Suc i)
  and suffix-ext: s-suc ≤ t
  and suffix-rounds:
    ∧j. j < n - Suc i ⇒
      verifier-query-round-chunk (suffix-query-idxs ! j)
      (map snd f-fl) (map snd fl) (suffix-chunks ! j)
  have transcript-all:
    PTranscript s =
      List.concat (prefix-chunks @ chunk # suffix-chunks) @
      PTranscript t
    using transcript-prefix transcript-round transcript-suffix
    by simp
  have trace-table-t:
    partial-authenticated-table fr (scale * clength)
    trace-openings t
    by (rule partial-authenticated-table-mono
      [OF trace-table-suc suffix-ext])
  have lookup-t:
    fmlookup (HashMap t)
    (QueryIndexChallenge (PQueryCounter s + i)
      (state-after-query-chunks (PState s) prefix-chunks i)) =
      Some raw
    using hash-extension-lookup[OF lookup-suc suffix-ext]
    unfolding counter-i state-i .
  show ?thesis
    by (rule that[OF idx-eq len-prefix len-prefix-chunks
      len-suffix-idxs len-suffix-chunks - - transcript-all
      trace-indices trace-table-t lookup-t])
      (use chunk-shape prefix-rounds suffix-rounds fl-empty in simp-all)
qed
qed
qed
qed

```

lemma *ntimes-verifier-query-rounds-selected-empty-trace-chunks-aligned-with-replay-at:*
assumes *outcome:*
Some (results, final-state) ∈
set-dist
(execute
(ntimes
(verifier-query-round-program fr f-fl f-final as [] final)
rounds)

```

    query-state)
  and i-bound:  $i < \text{rounds}$ 
  and len-raw:  $\text{length raw-idxs} = \text{rounds}$ 
  and query-idxs-eq:
    query-idxs = map ( $\lambda \text{raw}. \text{index } (\text{to-nat } \text{raw})$ ) raw-idxs
  and len-query-chunks:  $\text{length query-chunks} = \text{rounds}$ 
  and transcript-query:
    PTranscript query-state =
      List.concat query-chunks @ PTranscript final-state
  and query-chunk:
     $\bigwedge i. i < \text{rounds} \implies$ 
      verifier-query-round-chunk (query-idxs ! i)
      (map snd f-fl) [] (query-chunks ! i)
  and replay-lookup:
     $\bigwedge i. i < \text{rounds} \implies$ 
      fmllookup (HashMap final-state)
      (QueryIndexChallenge (PQueryCounter query-state + i)
      (state-after-query-chunks
      (PState query-state) query-chunks i)) =
      Some (raw-idxs ! i)
  obtains trace-openings where
    map opening-index trace-openings = powers-scaled (query-idxs ! i)
    partial-authenticated-table fr (scale * clength) trace-openings
    final-state
proof (rule ntimes-verifier-query-rounds-selected-empty-trace-chunks-at
  [OF refl outcome i-bound])
fix raw idx prefix-query-idxs prefix-chunks chunk suffix-query-idxs
  suffix-chunks trace-openings
assume selected-idx:  $\text{idx} = \text{index } (\text{to-nat } \text{raw})$ 
and len-prefix-chunks:  $\text{length prefix-chunks} = i$ 
and len-suffix-chunks:  $\text{length suffix-chunks} = \text{rounds} - \text{Suc } i$ 
and chunk-shape:
  verifier-query-round-chunk idx (map snd f-fl) [] chunk
and prefix-rounds:
   $\bigwedge j. j < i \implies$ 
    verifier-query-round-chunk (prefix-query-idxs ! j)
    (map snd f-fl) [] (prefix-chunks ! j)
and suffix-rounds:
   $\bigwedge j. j < \text{rounds} - \text{Suc } i \implies$ 
    verifier-query-round-chunk (suffix-query-idxs ! j)
    (map snd f-fl) [] (suffix-chunks ! j)
and selected-transcript:
  PTranscript query-state =
    List.concat (prefix-chunks @ chunk # suffix-chunks) @
    PTranscript final-state
and trace-indices:
  map opening-index trace-openings = powers-scaled idx
and trace-table:
  partial-authenticated-table fr (scale * clength) trace-openings

```

```

    final-state
  and selected-lookup:
    fmlookup (HashMap final-state)
      (QueryIndexChallenge (PQueryCounter query-state + i)
        (state-after-query-chunks (PState query-state) prefix-chunks i)) =
    Some raw
  let ?selected-chunks = prefix-chunks @ chunk # suffix-chunks
  have len-selected-chunks: length ?selected-chunks = rounds
    using len-prefix-chunks len-suffix-chunks i-bound by simp
  have selected-chunk-lens:
     $\bigwedge j. j < rounds \implies$ 
    length (?selected-chunks ! j) =
      verifier-query-round-transcript-length (query-idxs ! j)
      (map snd f-fl) []
  proof -
    fix j
    assume j-bound: j < rounds
    show
      length (?selected-chunks ! j) =
        verifier-query-round-transcript-length (query-idxs ! j)
        (map snd f-fl) []
  proof (cases j < i)
    case True
    have nth-eq: ?selected-chunks ! j = prefix-chunks ! j
      using True len-prefix-chunks by (simp add: nth-append)
    have
      length (prefix-chunks ! j) =
        verifier-query-round-transcript-length
          (prefix-query-idxs ! j) (map snd f-fl) []
      by (rule verifier-query-round-chunk-length[OF prefix-rounds[OF True]])
    then show ?thesis
      using nth-eq
      by (simp add: verifier-query-round-transcript-length-index-irrelevant)
  next
    case False
    show ?thesis
  proof (cases j = i)
    case True
    have nth-eq: ?selected-chunks ! j = chunk
      using True len-prefix-chunks by (simp add: nth-append)
    have
      length chunk =
        verifier-query-round-transcript-length idx (map snd f-fl) []
      by (rule verifier-query-round-chunk-length[OF chunk-shape])
    then show ?thesis
      using nth-eq
      by (simp add:
        verifier-query-round-transcript-length-index-irrelevant)
  next

```



```

case False
let ?k = j - Suc i
have ge: Suc i ≤ j
  using  $\langle \neg j < i \rangle$  False by linarith
have k-bound: ?k < rounds - Suc i
  using ge j-bound by linarith
have nth-eq: ?selected-chunks ! j = suffix-chunks ! ?k
  using ge len-prefix-chunks by (simp add: nth-append)
have
  length (suffix-chunks ! ?k) =
    verifier-query-round-transcript-length
      (suffix-query-idxs ! ?k) (map snd f-fl) []
  by (rule verifier-query-round-chunk-length
    [OF suffix-rounds [OF k-bound]])
then show ?thesis
  using nth-eq
  by (simp add:
    verifier-query-round-transcript-length-index-irrelevant)
qed
qed
qed
have query-chunk-lens:
   $\bigwedge j. j < \text{rounds} \implies$ 
    length (query-chunks ! j) =
      verifier-query-round-transcript-length (query-idxs ! j)
        (map snd f-fl) []
  using query-chunk verifier-query-round-chunk-length by blast
have concat-eq: List.concat ?selected-chunks = List.concat query-chunks
  using selected-transcript transcript-query by simp
have chunks-eq: ?selected-chunks = query-chunks
proof -
  have same-lens:
     $\bigwedge j. j < \text{length } ?\text{selected-chunks} \implies$ 
      length (?selected-chunks ! j) = length (query-chunks ! j)
    using selected-chunk-lens query-chunk-lens len-selected-chunks by simp
have concat-eq-empty:
    List.concat ?selected-chunks @ [] = List.concat query-chunks
    using concat-eq by simp
have len-eq: length ?selected-chunks = length query-chunks
    using len-selected-chunks len-query-chunks by simp
have ?selected-chunks = query-chunks  $\wedge$  ( [] :: 'f list ) = []
    by (rule concat-append-eq-concat-same-chunk-lengths
      [OF len-eq same-lens concat-eq-empty])
then show ?thesis
  by simp
qed
have prefix-state-eq:
  state-after-query-chunks (PState query-state) prefix-chunks i =
    state-after-query-chunks (PState query-state) query-chunks i

```

```

proof –
  have
    state-after-query-chunks (PState query-state) ?selected-chunks i =
      state-after-query-chunks (PState query-state) prefix-chunks i
  by (rule state-after-query-chunks-append-prefix[OF len-prefix-chunks])
  then show ?thesis
    using chunks-eq by simp
qed
have selected-lookup-global:
  fmlookup (HashMap final-state)
    (QueryIndexChallenge (PQueryCounter query-state + i)
      (state-after-query-chunks (PState query-state) query-chunks i)) =
    Some raw
  using selected-lookup prefix-state-eq by simp
have raw-eq: raw = raw-idxs ! i
  using selected-lookup-global replay-lookup[OF i-bound] by simp
have idx-eq: idx = query-idxs ! i
  using selected-idx raw-eq query-idxs-eq len-raw i-bound by simp
show ?thesis
  by (rule that)
    (use idx-eq trace-indices trace-table in simp-all)
qed

lemma ntimes-verifier-query-rounds-empty-trace-openings-aligned-with-replay:
assumes outcome:
  Some (results, final-state) ∈
    set-dist
      (execute
        (ntimes
          (verifier-query-round-program fr f-fl f-final as [] final)
            rounds)
          query-state)
and len-raw: length raw-idxs = rounds
and query-idxs-eq:
  query-idxs = map (λraw. index (to-nat raw)) raw-idxs
and len-query-chunks: length query-chunks = rounds
and transcript-query:
  PTranscript query-state =
    List.concat query-chunks @ PTranscript final-state
and query-chunk:
  ∧ i. i < rounds ⇒
    verifier-query-round-chunk (query-idxs ! i)
      (map snd f-fl) [] (query-chunks ! i)
and replay-lookup:
  ∧ i. i < rounds ⇒
    fmlookup (HashMap final-state)
      (QueryIndexChallenge (PQueryCounter query-state + i)
        (state-after-query-chunks
          (PState query-state) query-chunks i)) =

```

```

    Some (raw-idxs ! i)
obtains trace-openings where
  length trace-openings = rounds
   $\bigwedge i. i < \text{rounds} \implies$ 
    map opening-index (trace-openings ! i) =
      powers-scaled (query-idxs ! i)
   $\bigwedge i. i < \text{rounds} \implies$ 
    partial-authenticated-table fr (scale * clength)
      (trace-openings ! i) final-state
proof –
  let ?P =
     $\lambda i$  openings.
      map opening-index openings = powers-scaled (query-idxs ! i)  $\wedge$ 
      partial-authenticated-table fr (scale * clength) openings final-state
  have ex-round:  $\bigwedge i. i < \text{rounds} \implies \exists \text{ openings. } ?P\ i\ \text{openings}$ 
proof –
  fix i
  assume i-bound:  $i < \text{rounds}$ 
  show  $\exists \text{ openings. } ?P\ i\ \text{openings}$ 
  proof (rule
    ntimes-verifier-query-rounds-selected-empty-trace-chunks-aligned-with-replay-at
    [OF outcome i-bound len-raw query-idxs-eq len-query-chunks
      transcript-query query-chunk replay-lookup])
  fix trace-openings
  assume
    map opening-index trace-openings =
      powers-scaled (query-idxs ! i)
      partial-authenticated-table fr (scale * clength)
      trace-openings final-state
  then show ?thesis
    by blast
  qed
qed
let ?w =  $\lambda i. \text{SOME openings. } ?P\ i\ \text{openings}$ 
let ?trace-openings = map ?w [0.. $\text{rounds}$ ]
have len-trace: length ?trace-openings = rounds
  by simp
have props:
   $\bigwedge i. i < \text{rounds} \implies ?P\ i\ (?\text{trace-openings}\ !\ i)$ 
proof –
  fix i
  assume i-bound:  $i < \text{rounds}$ 
  have ?P i (?w i)
    by (rule someI-ex[OF ex-round[OF i-bound]])
  then show ?P i (?trace-openings ! i)
    using i-bound by simp
  qed
show ?thesis
  by (rule that[OF len-trace])

```

```

      (use props in simp-all)
qed

lemma ntimes-verifier-query-rounds-selected-zero-trace-final-opening-at:
  assumes outcome:
    Some (results, final-state) ∈
      set-dist
        (execute
          (ntimes
            (verifier-query-round-program fr [] f-final as fl final)
            rounds)
          query-state)
  and i-bound: i < rounds
  and len-raw: length raw-idxs = rounds
  and query-idxs-eq:
    query-idxs = map (λraw. index (to-nat raw)) raw-idxs
  and len-query-chunks: length query-chunks = rounds
  and transcript-query:
    PTranscript query-state =
      List.concat query-chunks @ PTranscript final-state
  and query-chunk:
    ∧ i. i < rounds ⇒
      verifier-query-round-chunk (query-idxs ! i)
        [] (map snd fl) (query-chunks ! i)
  and replay-lookup:
    ∧ i. i < rounds ⇒
      fmlookup (HashMap final-state)
        (QueryIndexChallenge (PQueryCounter query-state + i)
          (state-after-query-chunks
            (PState query-state) query-chunks i)) =
        Some (raw-idxs ! i)
  obtains trace-openings where
    map opening-index trace-openings = powers-scaled (query-idxs ! i)
    partial-authenticated-table fr (scale * clength) trace-openings
    final-state
    opening-value (hd trace-openings) = f-final
proof –
  from query-opening-ntimes-outcome-decomp-at[OF outcome i-bound]
  obtain prefix round suffix s-i s-suc where
    prefix:
      Some (prefix, s-i) ∈
        set-dist
          (execute
            (ntimes
              (verifier-query-round-program fr [] f-final as fl final) i)
            query-state)
  and round:
    Some (round, s-suc) ∈
      set-dist

```

```

      (execute
        (verifier-query-round-program fr [] f-final as fl final) s-i)
and suffix:
  Some (suffix, final-state) ∈
    set-dist
      (execute
        (ntimes
          (verifier-query-round-program fr [] f-final as fl final)
          (rounds - Suc i))
        s-suc)
by blast
have round-unit: round = ()
by (cases round) simp
show ?thesis
proof (rule query-opening-ntimes-prefix-transcript-state-alignment
  [OF prefix])
fix prefix-query-idxs prefix-chunks
assume len-prefix: length prefix-query-idxs = i
and len-prefix-chunks: length prefix-chunks = i
and transcript-prefix:
  PTranscript query-state = List.concat prefix-chunks @ PTranscript s-i
and state-i:
  PState s-i =
    state-after-query-chunks (PState query-state) prefix-chunks i
and counter-i: PQueryCounter s-i = PQueryCounter query-state + i
and prefix-ext: query-state ≤ s-i
and prefix-rounds:
  ∧j. j < i ⇒
    verifier-query-round-chunk (prefix-query-idxs ! j)
      (map snd []) (map snd fl) (prefix-chunks ! j)
show ?thesis
proof (rule
  verifier-query-round-program-outcome-with-zero-trace-final-opening
    [OF refl round[unfolded round-unit]])
fix raw idx chunk trace-openings
assume selected-idx: idx = index (to-nat raw)
and chunk-shape:
  verifier-query-round-chunk idx [] (map snd fl) chunk
and transcript-round:
  PTranscript s-i = chunk @ PTranscript s-suc
and state-suc:
  PState s-suc = foldl concat (PState s-i) chunk
and round-ext: s-i ≤ s-suc
and counter-suc: PQueryCounter s-suc = Suc (PQueryCounter s-i)
and lookup-suc:
  fmlookup (HashMap s-suc)
    (QueryIndexChallenge (PQueryCounter s-i) (PState s-i)) =
    Some raw
and trace-indices:

```

```

    map opening-index trace-openings = powers-scaled idx
and trace-table-suc:
    partial-authenticated-table fr (scale * clength)
    trace-openings s-suc
and trace-value:
    opening-value (hd trace-openings) = f-final
show ?thesis
proof (rule query-opening-ntimes-suffix-transcript-state-alignment
  [OF suffix])
fix suffix-query-idxs suffix-chunks
assume len-suffix-idxs:
    length suffix-query-idxs = rounds - Suc i
and len-suffix-chunks:
    length suffix-chunks = rounds - Suc i
and transcript-suffix:
    PTranscript s-suc =
      List.concat suffix-chunks @ PTranscript final-state
and suffix-state:
    PState final-state =
      state-after-query-chunks (PState s-suc) suffix-chunks
      (rounds - Suc i)
and suffix-counter:
    PQueryCounter final-state =
      PQueryCounter s-suc + (rounds - Suc i)
and suffix-ext: s-suc ≤ final-state
and suffix-rounds:
     $\bigwedge j. j < rounds - Suc i \implies$ 
      verifier-query-round-chunk (suffix-query-idxs ! j)
      (map snd []) (map snd fl) (suffix-chunks ! j)
let ?selected-chunks = prefix-chunks @ chunk # suffix-chunks
have selected-transcript:
    PTranscript query-state =
      List.concat ?selected-chunks @ PTranscript final-state
    using transcript-prefix transcript-round transcript-suffix
    by simp
have len-selected-chunks: length ?selected-chunks = rounds
    using len-prefix-chunks len-suffix-chunks i-bound by simp
have selected-chunk-lens:
     $\bigwedge j. j < rounds \implies$ 
      length (?selected-chunks ! j) =
        verifier-query-round-transcript-length (query-idxs ! j)
        [] (map snd fl)
proof -
fix j
assume j-bound: j < rounds
show
    length (?selected-chunks ! j) =
      verifier-query-round-transcript-length (query-idxs ! j)
      [] (map snd fl)

```

```

proof (cases j < i)
  case True
  have nth-eq: ?selected-chunks ! j = prefix-chunks ! j
    using True len-prefix-chunks by (simp add: nth-append)
  have
    length (prefix-chunks ! j) =
      verifier-query-round-transcript-length
        (prefix-query-idxs ! j) [] (map snd fl)
    using verifier-query-round-chunk-length
      [OF prefix-rounds[OF True]]
    by simp
  then show ?thesis
    using nth-eq
    by (simp add:
      verifier-query-round-transcript-length-index-irrelevant)
next
  case False
  show ?thesis
  proof (cases j = i)
    case True
    have nth-eq: ?selected-chunks ! j = chunk
      using True len-prefix-chunks by (simp add: nth-append)
    have
      length chunk =
        verifier-query-round-transcript-length idx [] (map snd fl)
      by (rule verifier-query-round-chunk-length[OF chunk-shape])
    then show ?thesis
      using nth-eq
      by (simp add:
        verifier-query-round-transcript-length-index-irrelevant)
    next
    case False
    let ?k = j - Suc i
    have ge: Suc i ≤ j
      using ⟨¬ j < i⟩ False by linarith
    have k-bound: ?k < rounds - Suc i
      using ge j-bound by linarith
    have nth-eq: ?selected-chunks ! j = suffix-chunks ! ?k
      using ge len-prefix-chunks by (simp add: nth-append)
    have
      length (suffix-chunks ! ?k) =
        verifier-query-round-transcript-length
          (suffix-query-idxs ! ?k) [] (map snd fl)
      using verifier-query-round-chunk-length
        [OF suffix-rounds[OF k-bound]]
      by simp
    then show ?thesis
      using nth-eq
      by (simp add:

```

```

verifier-query-round-transcript-length-index-irrelevant)
qed
qed
qed
have query-chunk-lens:
   $\bigwedge j. j < \text{rounds} \implies$ 
    length (query-chunks ! j) =
      verifier-query-round-transcript-length (query-idxs ! j)
       $\square$  (map snd fl)
  using query-chunk verifier-query-round-chunk-length by blast
have concat-eq:
  List.concat ?selected-chunks = List.concat query-chunks
  using selected-transcript transcript-query by simp
have chunks-eq: ?selected-chunks = query-chunks
proof -
  have same-lens:
     $\bigwedge j. j < \text{length } ?\text{selected-chunks} \implies$ 
      length (?selected-chunks ! j) = length (query-chunks ! j)
    using selected-chunk-lens query-chunk-lens len-selected-chunks
    by simp
  have concat-eq-empty:
    List.concat ?selected-chunks @  $\square$  = List.concat query-chunks
    using concat-eq by simp
  have len-eq: length ?selected-chunks = length query-chunks
    using len-selected-chunks len-query-chunks by simp
  have ?selected-chunks = query-chunks  $\wedge$  ( $\square :: 'f \text{ list} = \square$ )
    by (rule concat-append-eq-concat-same-chunk-lengths
      [OF len-eq same-lens concat-eq-empty])
  then show ?thesis
    by simp
qed
have prefix-state-eq:
  state-after-query-chunks (PState query-state) prefix-chunks i =
    state-after-query-chunks (PState query-state) query-chunks i
proof -
  have
    state-after-query-chunks (PState query-state) ?selected-chunks i =
      state-after-query-chunks (PState query-state) prefix-chunks i
    by (rule state-after-query-chunks-append-prefix
      [OF len-prefix-chunks])
  then show ?thesis
    using chunks-eq by simp
qed
have selected-lookup-global:
  fmllookup (HashMap final-state)
    (QueryIndexChallenge (PQueryCounter query-state + i)
      (state-after-query-chunks (PState query-state)
        query-chunks i)) =
    Some raw

```



```

    using hash-extension-lookup[OF lookup-suc suffix-ext]
    unfolding counter-i state-i prefix-state-eq .
  have raw-eq: raw = raw-idxs ! i
    using selected-lookup-global replay-lookup[OF i-bound] by simp
  have idx-eq: idx = query-idxs ! i
    using selected-idx raw-eq query-idxs-eq len-raw i-bound by simp
  have trace-table:
    partial-authenticated-table fr (scale * clength) trace-openings
    final-state
    by (rule partial-authenticated-table-mono
        [OF trace-table-suc suffix-ext])
  show ?thesis
    by (rule that)
      (use idx-eq trace-indices trace-table trace-value in simp-all)
qed
qed
qed
qed

lemma ntimes-verifier-query-rounds-zero-trace-final-openings-aligned-with-replay:
  assumes outcome:
    Some (results, final-state) ∈
      set-dist
        (execute
          (ntimes
            (verifier-query-round-program fr [] f-final as fl final)
            rounds)
          query-state)
  and len-raw: length raw-idxs = rounds
  and query-idxs-eq:
    query-idxs = map (λraw. index (to-nat raw)) raw-idxs
  and len-query-chunks: length query-chunks = rounds
  and transcript-query:
    PTranscript query-state =
      List.concat query-chunks @ PTranscript final-state
  and query-chunk:
    ∧ i. i < rounds ⇒
      verifier-query-round-chunk (query-idxs ! i)
      [] (map snd fl) (query-chunks ! i)
  and replay-lookup:
    ∧ i. i < rounds ⇒
      fmlookup (HashMap final-state)
      (QueryIndexChallenge (PQueryCounter query-state + i)
        (state-after-query-chunks
          (PState query-state) query-chunks i)) =
      Some (raw-idxs ! i)
  obtains trace-openings where
    length trace-openings = rounds
    ∧ i. i < rounds ⇒

```

```

    map opening-index (trace-openings ! i) =
      powers-scaled (query-idxs ! i)
   $\wedge i. i < \text{rounds} \implies$ 
    partial-authenticated-table fr (scale * clength)
      (trace-openings ! i) final-state
   $\wedge i. i < \text{rounds} \implies$ 
    opening-value (hd (trace-openings ! i)) = f-final
proof –
  let ?P =
     $\lambda i$  openings.
      map opening-index openings = powers-scaled (query-idxs ! i)  $\wedge$ 
      partial-authenticated-table fr (scale * clength) openings final-state  $\wedge$ 
      opening-value (hd openings) = f-final
  have ex-round:  $\wedge i. i < \text{rounds} \implies \exists \text{openings}. ?P i \text{ openings}$ 
proof –
  fix i
  assume i-bound:  $i < \text{rounds}$ 
  show  $\exists \text{openings}. ?P i \text{ openings}$ 
proof (rule
    ntimes-verifier-query-rounds-selected-zero-trace-final-opening-at
    [OF outcome i-bound len-raw query-idxs-eq len-query-chunks
      transcript-query query-chunk replay-lookup])
  fix trace-openings
  assume
    map opening-index trace-openings =
      powers-scaled (query-idxs ! i)
      partial-authenticated-table fr (scale * clength)
      trace-openings final-state
    opening-value (hd trace-openings) = f-final
  then show ?thesis
    by blast
  qed
qed
let ?w =  $\lambda i. \text{SOME openings}. ?P i \text{ openings}$ 
let ?trace-openings = map ?w [0.. $\text{rounds}$ ]
have len-trace: length ?trace-openings = rounds
  by simp
have props:
   $\wedge i. i < \text{rounds} \implies ?P i (?trace-openings ! i)$ 
proof –
  fix i
  assume i-bound:  $i < \text{rounds}$ 
  have ?P i (?w i)
    by (rule someI-ex[OF ex-round[OF i-bound]])
  then show ?P i (?trace-openings ! i)
    using i-bound by simp
  qed
show ?thesis
  by (rule that[OF len-trace])

```

```

      (use props in simp-all)
qed

lemma verify-monad-supplied-empty-header-partial-trace-openings:
  fixes s :: ('f, 'a) protocol-channel-scheme
  assumes outcome:
    Some (result, final-state) ∈ set-dist (execute verify-monad s)
  and header0:
    verifier-header-transcript s fr f-fri-roots f-final as dg []
    final rest
  obtains query-idxs trace-openings where
    accepted-transcript-shape s (Some (result, final-state)) as
    query-idxs
    accepted-with-partial-trace-openings s
    (Some (result, final-state)) fr query-idxs trace-openings
proof -
  obtain f-fl query-state where f-roots:
    map snd f-fl = f-fri-roots
  and query-rest: PTranscript query-state = rest
  and query-state-start:
    PState query-state =
      verifier-header-state s fr f-fri-roots f-final as dg [] final
  and query-count:
    PQueryCounter query-state = PQueryCounter s
  and query-out:
    Some (result, final-state) ∈
      set-dist
        (execute
          (ntimes
            (verifier-query-round-program fr f-fl f-final as [] final)
            rounds)
          query-state)
  by (rule verify-monad-supplied-empty-header-query-rounds
      [OF outcome header0])
show ?thesis
proof (rule verify-monad-supplied-header-transcript-shape
  [OF outcome header0])
  fix query-idxs raw-idxs query-chunks
  assume shape:
    accepted-transcript-shape s (Some (result, final-state)) as
    query-idxs
  and len-raw: length raw-idxs = rounds
  and query-idxs-eq:
    query-idxs = map (λraw. index (to-nat raw)) raw-idxs
  and len-query-chunks: length query-chunks = rounds
  and transcript-query:
    List.concat query-chunks @ PTranscript final-state = rest
  and query-chunk:
    ∧ i. i < rounds ⇒

```

```

    verifier-query-round-chunk (query-idxs ! i)
    f-fri-roots [] (query-chunks ! i)
and replay-lookup:
   $\bigwedge i. i < \text{rounds} \implies$ 
    fmlookup (HashMap final-state)
      (QueryIndexChallenge (PQueryCounter s + i)
        (state-after-query-chunks
          (verifier-header-state s fr f-fri-roots f-final as dg []
            final)
          query-chunks i)) =
      Some (raw-idxs ! i)
have transcript-query-state:
  PTranscript query-state =
    List.concat query-chunks @ PTranscript final-state
  using transcript-query query-rest by simp
have query-chunk-empty:
   $\bigwedge i. i < \text{rounds} \implies$ 
    verifier-query-round-chunk (query-idxs ! i)
      (map snd f-fl) [] (query-chunks ! i)
  using query-chunk f-roots by simp
have replay-lookup-query-state:
   $\bigwedge i. i < \text{rounds} \implies$ 
    fmlookup (HashMap final-state)
      (QueryIndexChallenge (PQueryCounter query-state + i)
        (state-after-query-chunks
          (PState query-state) query-chunks i)) =
      Some (raw-idxs ! i)
  using replay-lookup query-state-start query-count by simp
show ?thesis
proof (rule
  ntimes-verifier-query-rounds-empty-trace-openings-aligned-with-replay
  [OF query-out len-raw query-idxs-eq len-query-chunks
    transcript-query-state query-chunk-empty
    replay-lookup-query-state])
fix trace-openings
assume len-trace: length trace-openings = rounds
and trace-indices:
   $\bigwedge i. i < \text{rounds} \implies$ 
    map opening-index (trace-openings ! i) =
    powers-scaled (query-idxs ! i)
and trace-tables:
   $\bigwedge i. i < \text{rounds} \implies$ 
    partial-authenticated-table fr (scale * clength)
      (trace-openings ! i) final-state
have len-query-idxs: length query-idxs = rounds
  using len-raw query-idxs-eq by simp
have partial:
  accepted-with-partial-trace-openings s
    (Some (result, final-state)) fr query-idxs trace-openings

```

```

    unfolding accepted-with-partial-trace-openings-def accepted-def
  by (intro conjI exI[of - result] exI[of - final-state])
    (use len-query-idxs len-trace trace-indices trace-tables in auto)
show ?thesis
  by (rule that[OF shape partial])
qed
qed
qed
end
end
end

```

```

theory Soundness-FRI-Verifier-Tied
imports
  Soundness-FRI-Zero-Round-Target
  Soundness-Query-Transcript-Openings
begin

```

Verifier-tied FRI partial-candidate events.

Broad reachable partial-candidate events allow the candidate table to be compatible with sampled authenticated openings whose indices are not tied to the FRI opening transcript. This theory introduces narrower internal events where the candidate openings use the same query indices as the verifier's FRI transcript. The protocol and public soundness statement are unchanged.

```

context soundness
begin

```

```

lemma partial-authenticated-tables-values-unique-if-clean:
  assumes table:
    partial-authenticated-table rt len openings s
  and table':
    partial-authenticated-table rt len openings' s
  and clean:  $\neg$  hash-map-output-collision s
  and first:  $\text{opening} \in \text{set openings}$ 
  and second:  $\text{opening}' \in \text{set openings}'$ 
  and same-index:  $\text{opening-index opening} = \text{opening-index opening}'$ 
shows  $\text{opening-value opening} = \text{opening-value opening}'$ 
proof –
  have combined:
    partial-authenticated-table rt len (openings @ openings') s
  using table table'
  unfolding partial-authenticated-table-def by auto
show ?thesis
  by (rule partial-authenticated-table-values-unique-if-clean[
    OF combined clean])
    (use first second same-index in auto)

```

qed

lemma *verifier-query-round-after-index-zero-trace-final-opening:*

fixes $s\ t :: ('f, 'a)\ \text{protocol-channel-scheme}$

assumes *empty-trace*: $f\text{-}fl = []$

and *outcome*:

$\text{Some } ((), t) \in$

set-dist

(execute

(verifier-query-round-after-index-program

fr f-fl f-final as fl final raw) s)

obtains *idx openings where*

idx = index (to-nat raw)

and *map opening-index openings = powers-scaled idx*

and *partial-authenticated-table fr (scale * clength) openings t*

and *opening-value (hd openings) = f-final*

proof –

let $?idx = \text{index } (to\text{-}nat\ raw)$

from *outcome* **obtain** $fv\ s1\ f\text{-}out\ s2\ s3\ c\text{-}out\ s4$ **where**

query-decommit:

$\text{Some } (fv, s1) \in$

set-dist (execute (mmap (check-decommit-on-query fr ?idx)) s)

and *trace-fri:*

$\text{Some } (f\text{-}out, s2) \in$

set-dist

(execute

*(mfold (?idx, hd fv, clength * scale, 1)*

(receive-query-commits f-fl)) s1)

and *assert-trace:*

$\text{Some } ((), s3) \in$

set-dist

(execute

(assert (case f-out of (-, f-x, -, -) $\Rightarrow$ $f\text{-}x = f\text{-}final$)) s2)

and *comp-fri:*

$\text{Some } (c\text{-}out, s4) \in$

set-dist (execute

*(mfold (?idx, cp-eval as fv (h ^ ?idx * shift),*

*clength * scale, 1)*

(receive-query-commits fl)) s3)

and *assert-comp:*

$\text{Some } ((), t) \in$

set-dist

(execute

(assert (case c-out of (-, x, -, -) $\Rightarrow$ $x = final$)) s4)

unfolding *verifier-query-round-after-index-program-def*

by *(auto simp: Let-def split: prod.splits elim!: set-dist-bindE)*

have *trace-out:*

$f\text{-}out = (?idx, hd fv, clength * scale, 1)$

and *s2-eq*: $s2 = s1$

```

using trace-fri empty-trace
unfolding receive-query-commits-def by simp-all
have hd-fv: hd fv = f-final
using assert-trace unfolding assert-def trace-out
by (cases hd fv = f-final) (auto simp: throw-no-outcome)
have s3-eq: s3 = s2
using assert-trace unfolding assert-def trace-out hd-fv by simp
have idx-in: ?idx ∈ query-sample-space
using index-less-query-sample-space
unfolding query-sample-space-def by simp
from check-decommit-on-query-authenticated-openings
  [OF idx-in query-decommit]
obtain openings where len-openings:
  length openings = length (powers-scaled ?idx)
and values-openings: map opening-value openings = fv
and idx-openings: map opening-index openings = powers-scaled ?idx
and table-s1:
  partial-authenticated-table fr (scale * clength) openings s1
by blast
from receive-query-commits-layers-outcome[OF comp-fri]
obtain comp-layer-chunks comp-fri-chunk where s3-s4: s3 ≤ s4
by blast
have t-eq: t = s4
using assert-comp unfolding assert-def
by (cases case c-out of  $(-, x, -, -) \Rightarrow x = \text{final}$ )
  (auto simp: throw-no-outcome)
have s1-t: s1 ≤ t
using s3-s4 unfolding s3-eq s2-eq t-eq .
have table-t:
  partial-authenticated-table fr (scale * clength) openings t
by (rule partial-authenticated-table-mono[OF table-s1 s1-t])
have openings-nonempty: openings ≠ []
using len-openings powers-pos unfolding powers-scaled-def by auto
have hd-value: opening-value (hd openings) = hd fv
using values-openings openings-nonempty
by (cases openings; cases fv) simp-all
show ?thesis
by (rule that[OF refl idx-openings table-t])
  (simp add: hd-value hd-fv)
qed

```

```

lemma verifier-query-round-program-zero-trace-final-opening:
fixes s t :: ('f, 'a) protocol-channel-scheme
assumes empty-trace: f-fl = []
and outcome:
  Some (( ), t) ∈
    set-dist
    (execute
      (verifier-query-round-program fr f-fl f-final as fl final) s)

```

shows $\exists \text{ raw idx openings.}$
 $\text{idx} = \text{index } (\text{to-nat raw}) \wedge$
 $\text{map opening-index openings} = \text{powers-scaled idx} \wedge$
 $\text{partial-authenticated-table fr } (\text{scale} * \text{clength}) \text{ openings } t \wedge$
 $\text{opening-value } (\text{hd openings}) = \text{f-final}$
proof –
from *outcome[unfolded verifier-query-round-program-alt-def]*
obtain *raw s0* **where**
 $\text{Some } (\text{raw}, s0) \in \text{set-dist } (\text{execute receive-query-index-challenge } s)$
and *after:*
 $\text{Some } ((), t) \in$
 set-dist
 $(\text{execute}$
 $\quad (\text{verifier-query-round-after-index-program}$
 $\quad \text{fr f-fl f-final as fl final raw}) s0)$
by *(auto elim!: set-dist-bindE)*
from *verifier-query-round-after-index-zero-trace-final-opening*
 $[OF \text{ empty-trace after}]$
show *?thesis*
by *blast*
qed

lemma *trace-zero-round-checked-candidate-from-verified-openings-if-no-merkle:*
fixes *final-state :: ('f, 'a) protocol-channel-scheme*
assumes *partial:*
 $\text{accepted-with-partial-trace-openings } s$
 $(\text{Some } (\text{result}, \text{final-state})) \text{ fr query-idxs trace-openings}$
and *candidate:*
 $\text{partial-trace-table-candidate trace-table trace-openings}$
and *no-merkle:*
 $\neg \text{partial-merkle-inconsistency-bad } s (\text{Some } (\text{result}, \text{final-state}))$
and *query-sample:*
 $\bigwedge \text{idx. idx} \in \text{set query-idxs} \implies \text{idx} \in \text{query-sample-space}$
and *verifier-openings:*
 $\bigwedge i. i < \text{rounds} \implies$
 $\exists \text{ openings.}$
 $\text{map opening-index openings} = \text{powers-scaled } (\text{query-idxs ! } i) \wedge$
 $\text{partial-authenticated-table fr } (\text{scale} * \text{clength}) \text{ openings}$
 $\text{final-state} \wedge$
 $\text{opening-value } (\text{hd openings}) = \text{final}$
shows
 $\text{trace-zero-round-final-checked-candidate } s$
 $(\text{Some } (\text{result}, \text{final-state})) \text{ fr query-idxs trace-openings}$
 trace-table final
proof –
have *final-values:*
 $\bigwedge i. i < \text{rounds} \implies$
 $\text{opening-value } (\text{hd } (\text{trace-openings ! } i)) = \text{final}$
proof –


```

fix i
assume i-bound: i < rounds
have candidate-indices:
  map opening-index (trace-openings ! i) =
    powers-scaled (query-idxs ! i)
  by (rule accepted-with-partial-trace-openings-shapes(3)
      [OF partial i-bound])
have candidate-table:
  partial-authenticated-table fr (scale * clength)
    (trace-openings ! i) final-state
  using partial i-bound
  unfolding accepted-with-partial-trace-openings-def by blast
from verifier-openings[OF i-bound]
obtain openings where verified-indices:
  map opening-index openings = powers-scaled (query-idxs ! i)
and verified-table:
  partial-authenticated-table fr (scale * clength) openings
    final-state
and verified-value: opening-value (hd openings) = final
  by blast
have candidate-nonempty: trace-openings ! i ≠ []
proof -
  have map opening-index (trace-openings ! i) ≠ []
    using candidate-indices powers-pos unfolding powers-scaled-def by simp
  then show ?thesis by auto
qed
have verified-nonempty: openings ≠ []
proof -
  have map opening-index openings ≠ []
    using verified-indices powers-pos unfolding powers-scaled-def by simp
  then show ?thesis by auto
qed
let ?candidate = hd (trace-openings ! i)
let ?verified = hd openings
have candidate-in: ?candidate ∈ set (trace-openings ! i)
  using candidate-nonempty by simp
have verified-in: ?verified ∈ set openings
  using verified-nonempty by simp
have same-index:
  opening-index ?candidate = opening-index ?verified
proof -
  have opening-index ?candidate =
    hd (map opening-index (trace-openings ! i))
    using candidate-nonempty by (cases trace-openings ! i) auto
  also have ... = hd (powers-scaled (query-idxs ! i))
    using candidate-indices by simp
  also have ... = hd (map opening-index openings)
    using verified-indices by simp
  also have ... = opening-index ?verified

```

```

    using verified-nonempty by (cases openings) auto
    finally show ?thesis .
qed
have candidate-auth:
  authenticated-opening-in final-state ?candidate
  using candidate-table candidate-in
  unfolding partial-authenticated-table-def by blast
have verified-auth:
  authenticated-opening-in final-state ?verified
  using verified-table verified-in
  unfolding partial-authenticated-table-def by blast
have same-root:
  opening-root ?candidate = opening-root ?verified
  using candidate-table verified-table candidate-in verified-in
  unfolding partial-authenticated-table-def by auto
have same-length:
  opening-length ?candidate = opening-length ?verified
  using candidate-table verified-table candidate-in verified-in
  unfolding partial-authenticated-table-def by auto
show opening-value ?candidate = final
proof (rule ccontr)
  assume neq-final: opening-value ?candidate  $\neq$  final
  have diff:
    opening-value ?candidate  $\neq$  opening-value ?verified
    using neq-final verified-value by simp
  have partial-merkle-inconsistency-bad s
    (Some (result, final-state))
    unfolding partial-merkle-inconsistency-bad-def
    by (intro conjI exI)
    (use partial candidate-auth verified-auth same-root same-length
      same-index diff in
      ⟨auto simp: accepted-with-partial-trace-openings-def⟩)
  then show False
    using no-merkle by contradiction
qed
qed
show ?thesis
  unfolding trace-zero-round-final-checked-candidate-def
  using partial candidate query-sample final-values by blast
qed

```

definition *trace-fri-bad-with-verifier-tied-partial-candidate*
 $:: ('f, 'a) \text{ protocol-channel-scheme} \Rightarrow$
 $(\text{unit list} \times ('f, 'a) \text{ protocol-channel-scheme}) \text{ option} \Rightarrow \text{bool}$
where
 $\text{trace-fri-bad-with-verifier-tied-partial-candidate } s \text{ out} \iff$
 $(\exists \text{ trace-roots trace-bs trace-final dg composition-roots}$
 $\text{composition-bs composition-final fri-query-idxs trace-round-layers}$
 $\text{composition-round-layers fr trace-openings trace-table.}$

accepted-fri-opening-transcript s out trace-roots trace-bs
trace-final dg composition-roots composition-bs composition-final
fri-query-idxs trace-round-layers composition-round-layers $\wedge$
accepted-with-partial-trace-openings s out fr fri-query-idxs
trace-openings $\wedge$
partial-trace-table-candidate trace-table trace-openings $\wedge$
 $\neg$ *trace-table-low-degree trace-table*)

definition *trace-fri-bad-with-header-tied-partial-candidate*

$:: (f, 'a) \text{ protocol-channel-scheme} \Rightarrow$
 $(\text{unit list} \times (f, 'a) \text{ protocol-channel-scheme}) \text{ option} \Rightarrow \text{bool}$

where

trace-fri-bad-with-header-tied-partial-candidate s out $\longleftrightarrow$
 $(\exists \text{ trace-roots trace-bs trace-final dg composition-roots}$
 $\text{composition-bs composition-final fri-query-idxs trace-round-layers}$
 $\text{composition-round-layers fr as rest trace-openings trace-table.}$
accepted-fri-opening-transcript s out trace-roots trace-bs
trace-final dg composition-roots composition-bs composition-final
fri-query-idxs trace-round-layers composition-round-layers $\wedge$
verifier-header-transcript s fr trace-roots trace-final as dg
 $\text{composition-roots composition-final rest} \wedge$
accepted-with-partial-trace-openings s out fr fri-query-idxs
trace-openings $\wedge$
partial-trace-table-candidate trace-table trace-openings $\wedge$
 $\neg$ *trace-table-low-degree trace-table*)

lemma *trace-fri-bad-with-header-tied-imp-verifier-tied:*

assumes *trace-fri-bad-with-header-tied-partial-candidate s out*
shows *trace-fri-bad-with-verifier-tied-partial-candidate s out*
using *assms*
unfolding *trace-fri-bad-with-header-tied-partial-candidate-def*
trace-fri-bad-with-verifier-tied-partial-candidate-def
by *blast*

lemma *accepted-fri-opening-transcript-query-idx-in-sample-space:*

assumes *fri-openings:*
accepted-fri-opening-transcript s out trace-roots trace-bs
trace-final dg composition-roots composition-bs composition-final
query-idxs trace-round-layers composition-round-layers
and *idx-in: idx $\in$ set query-idxs*
shows *idx $\in$ query-sample-space*
using *fri-openings idx-in index-less-query-sample-space*
unfolding *accepted-fri-opening-transcript-def query-sample-space-def* **by** *auto*

definition *trace-fri-header-tied-zero-round-final-obstruction*

$:: (f, 'a) \text{ protocol-channel-scheme} \Rightarrow$
 $(\text{unit list} \times (f, 'a) \text{ protocol-channel-scheme}) \text{ option} \Rightarrow \text{bool}$

where

trace-fri-header-tied-zero-round-final-obstruction s out

$\longleftrightarrow$

($\exists$ *trace-roots trace-bs trace-final dg composition-roots*
composition-bs composition-final fri-query-idxs trace-round-layers
composition-round-layers fr as rest trace-openings trace-table.
accepted-fri-opening-transcript s out trace-roots trace-bs
trace-final dg composition-roots composition-bs composition-final
fri-query-idxs trace-round-layers composition-round-layers $\wedge$
verifier-header-transcript s fr trace-roots trace-final as dg
composition-roots composition-final rest $\wedge$
accepted-with-partial-trace-openings s out fr fri-query-idxs
trace-openings $\wedge$
partial-trace-table-candidate trace-table trace-openings $\wedge$
 $\neg$ *trace-table-low-degree trace-table* $\wedge$
length trace-bs = 0 $\wedge$
 $\neg$ *fri-final-constant-consistent trace-table trace-final*)

lemma *trace-fri-header-tied-zero-round-final-obstruction-imp-checked-or-merkle:*

assumes *obstruction:*

trace-fri-header-tied-zero-round-final-obstruction s out

shows

trace-fri-zero-round-checked-final-obstruction s out $\vee$

partial-merkle-inconsistency-bad s out

proof —

from *obstruction* **obtain** *trace-roots trace-bs trace-final dg*
composition-roots composition-bs composition-final fri-query-idxs
trace-round-layers composition-round-layers fr as rest trace-openings
trace-table **where** *fri-openings:*

accepted-fri-opening-transcript s out trace-roots trace-bs
trace-final dg composition-roots composition-bs composition-final
fri-query-idxs trace-round-layers composition-round-layers

and *header:*

verifier-header-transcript s fr trace-roots trace-final as dg
composition-roots composition-final rest

and *partial:*

accepted-with-partial-trace-openings s out fr fri-query-idxs
trace-openings

and *candidate:*

partial-trace-table-candidate trace-table trace-openings

and *not-low:* $\neg$ *trace-table-low-degree trace-table*

and *zero-rounds:* *length trace-bs = 0*

and *not-final:*

$\neg$ *fri-final-constant-consistent trace-table trace-final*

unfolding *trace-fri-header-tied-zero-round-final-obstruction-def*

by *blast*

from *fri-openings* **obtain** *result final-state fr0 f-fl as0 fl query-state*
raw-idxs query-chunks **where** *out-eq:*

out = Some (result, final-state)

and *trace-roots-eq:* *trace-roots = map snd f-fl*

and *trace-bs-eq:* *trace-bs = map fst f-fl*

```

and composition-roots-eq: composition-roots = map snd fl
and composition-bs-eq: composition-bs = map fst fl
and header0:
  verifier-header-transcript s fr0 trace-roots trace-final as0 dg
  composition-roots composition-final (PTranscript query-state)
and query-state-eq:
  PState query-state =
    verifier-header-state s fr0 trace-roots trace-final as0 dg
    composition-roots composition-final
and query-counter-eq:
  PQueryCounter query-state = PQueryCounter s
and query-out:
  Some (result, final-state) ∈
    set-dist
    (execute
      (ntimes
        (verifier-query-round-program fr0 f-fl trace-final as0 fl
          composition-final)
        rounds)
      query-state)
and len-raw: length raw-idxs = rounds
and query-idxs-eq:
  fri-query-idxs = map (λraw. index (to-nat raw)) raw-idxs
and len-query-chunks: length query-chunks = rounds
and transcript-query:
  PTranscript query-state =
    List.concat query-chunks @ PTranscript final-state
and query-chunk:
  ∧i. i < rounds ⇒
    verifier-query-round-chunk (fri-query-idxs ! i)
    trace-roots composition-roots (query-chunks ! i)
and replay-lookup:
  ∧i. i < rounds ⇒
    fmlookup (HashMap final-state)
      (QueryIndexChallenge (PQueryCounter query-state + i)
        (state-after-query-chunks (PState query-state) query-chunks i)) =
      Some (raw-idxs ! i)
unfolding accepted-fri-opening-transcript-def by blast
have f-fl-empty: f-fl = []
using zero-rounds trace-bs-eq by simp
have trace-roots-empty: trace-roots = []
using trace-roots-eq f-fl-empty by simp
have header-eqs:
  fr0 = fr ∧
  as0 = as ∧
  PTranscript query-state = rest
using verifier-header-transcript-unique[OF header0 header] by simp
have query-out-empty:
  Some (result, final-state) ∈

```

```

    set-dist
    (execute
      (ntimes
        (verifier-query-round-program fr [] trace-final as fl
          composition-final)
        rounds)
      query-state)
  using query-out f-fl-empty header-egs by simp
have query-chunk-empty:
   $\bigwedge i. i < \text{rounds} \implies$ 
    verifier-query-round-chunk (fri-query-idxs ! i)
    [] (map snd fl) (query-chunks ! i)
  using query-chunk trace-roots-empty composition-roots-eq by simp
show ?thesis
proof (cases partial-merkle-inconsistency-bad s out)
  case True
  then show ?thesis by simp
next
  case False
  have no-merkle:
     $\neg$  partial-merkle-inconsistency-bad s
    (Some (result, final-state))
  using False out-eq by simp
have verifier-openings:
   $\bigwedge i. i < \text{rounds} \implies$ 
     $\exists$  openings.
    map opening-index openings =
      powers-scaled (fri-query-idxs ! i)  $\wedge$ 
      partial-authenticated-table fr (scale * clength) openings
      final-state  $\wedge$ 
      opening-value (hd openings) = trace-final
proof -
  fix i
  assume i-bound:  $i < \text{rounds}$ 
  show
     $\exists$  openings.
    map opening-index openings =
      powers-scaled (fri-query-idxs ! i)  $\wedge$ 
      partial-authenticated-table fr (scale * clength) openings
      final-state  $\wedge$ 
      opening-value (hd openings) = trace-final
proof (rule
  ntimes-verifier-query-rounds-selected-zero-trace-final-opening-at
  [OF query-out-empty i-bound len-raw query-idxs-eq len-query-chunks
    transcript-query query-chunk-empty replay-lookup])
  fix verified-openings
  assume
    map opening-index verified-openings =
      powers-scaled (fri-query-idxs ! i)

```

```

    partial-authenticated-table fr (scale * clength)
    verified-openings final-state
    opening-value (hd verified-openings) = trace-final
then show
   $\exists$  openings.
    map opening-index openings =
      powers-scaled (fri-query-idxs ! i)  $\wedge$ 
      partial-authenticated-table fr (scale * clength) openings
      final-state  $\wedge$ 
      opening-value (hd openings) = trace-final
by blast
qed
qed
have query-sample:
   $\bigwedge idx. idx \in \text{set } \text{fri-query-idxs} \implies idx \in \text{query-sample-space}$ 
by (rule accepted-fri-opening-transcript-query-idx-in-sample-space
    [OF fri-openings])
have partial-some:
  accepted-with-partial-trace-openings s
  (Some (result, final-state)) fr fri-query-idxs trace-openings
using partial out-eq by simp
have checked:
  trace-zero-round-final-checked-candidate s
  (Some (result, final-state)) fr fri-query-idxs trace-openings
  trace-table trace-final
by (rule trace-zero-round-checked-candidate-from-verified-openings-if-no-merkle
    [OF partial-some candidate no-merkle query-sample verifier-openings])
have trace-fri-zero-round-checked-final-obstruction s out
unfolding trace-fri-zero-round-checked-final-obstruction-def out-eq
by (intro exI conjI)
  (use fri-openings[unfolded out-eq] zero-rounds checked not-final
    in simp-all)
then show ?thesis by simp
qed
qed

definition composition-fri-bad-with-verifier-tied-partial-candidate
  :: ('f, 'a) protocol-channel-scheme  $\Rightarrow$ 
    (unit list  $\times$  ('f, 'a) protocol-channel-scheme) option  $\Rightarrow$  bool
where
  composition-fri-bad-with-verifier-tied-partial-candidate s out
   $\longleftrightarrow$ 
  ( $\exists$  trace-roots trace-bs trace-final fri-dg composition-roots
    composition-bs composition-final fri-query-idxs trace-round-layers
    composition-round-layers fr as trace-openings composition-openings
    trace-table composition-table.
    accepted-fri-opening-transcript s out trace-roots trace-bs
    trace-final fri-dg composition-roots composition-bs
    composition-final fri-query-idxs trace-round-layers

```

composition-round-layers $\wedge$
accepted-with-partial-initial-openings-aligned *s out fr trace-roots*
trace-final as fri-dg composition-roots composition-final fri-query-idxs
trace-openings composition-openings $\wedge$
partial-trace-table-candidate trace-table trace-openings $\wedge$
partial-composition-table-candidate composition-table
composition-openings $\wedge$
trace-table-low-degree trace-table $\wedge$
 $\neg$ *composition-table-low-degree maxDegree composition-table*)

definition *trace-fri-bad-with-verifier-tied-sampled-layer-chain*

$:: ('f, 'a) \text{ protocol-channel-scheme} \Rightarrow$
 $(\text{unit list} \times ('f, 'a) \text{ protocol-channel-scheme}) \text{ option} \Rightarrow \text{bool}$

where

trace-fri-bad-with-verifier-tied-sampled-layer-chain *s out* $\longleftrightarrow$
 $(\exists \text{ trace-roots trace-bs trace-final dg composition-roots}$
 $\text{ composition-bs composition-final fri-query-idxs trace-round-layers}$
 $\text{ composition-round-layers fr trace-openings trace-table doms layers.}$
accepted-fri-opening-transcript *s out trace-roots trace-bs*
trace-final dg composition-roots composition-bs composition-final
fri-query-idxs trace-round-layers composition-round-layers $\wedge$
accepted-with-partial-trace-openings *s out fr fri-query-idxs*
trace-openings $\wedge$
partial-trace-table-candidate trace-table trace-openings $\wedge$
 $\neg$ *trace-table-low-degree trace-table* $\wedge$
generic-fri-sampled-layer-chain-evidence trace-table-low-degree
trace-table (clength - 1) trace-roots trace-bs trace-final
fri-query-idxs trace-round-layers doms layers)

definition *composition-fri-bad-with-verifier-tied-sampled-layer-chain*

$:: ('f, 'a) \text{ protocol-channel-scheme} \Rightarrow$
 $(\text{unit list} \times ('f, 'a) \text{ protocol-channel-scheme}) \text{ option} \Rightarrow \text{bool}$

where

composition-fri-bad-with-verifier-tied-sampled-layer-chain *s out*
 $\longleftrightarrow$
 $(\exists \text{ trace-roots trace-bs trace-final fri-dg composition-roots}$
 $\text{ composition-bs composition-final fri-query-idxs trace-round-layers}$
 $\text{ composition-round-layers fr as trace-openings composition-openings}$
 $\text{ trace-table composition-table doms layers.}$
accepted-fri-opening-transcript *s out trace-roots trace-bs*
trace-final fri-dg composition-roots composition-bs
composition-final fri-query-idxs trace-round-layers
composition-round-layers $\wedge$
accepted-with-partial-initial-openings-aligned *s out fr trace-roots*
trace-final as fri-dg composition-roots composition-final fri-query-idxs
trace-openings composition-openings $\wedge$
partial-trace-table-candidate trace-table trace-openings $\wedge$
partial-composition-table-candidate composition-table
composition-openings $\wedge$

$\text{trace-table-low-degree } \text{trace-table} \wedge$
 $\neg \text{composition-table-low-degree } \text{maxDegree } \text{composition-table} \wedge$
 $\text{generic-fri-sampled-layer-chain-evidence}$
 $(\text{composition-table-low-degree } (\text{to-nat fri-dg})) \text{composition-table}$
 $(\text{to-nat fri-dg}) \text{composition-roots } \text{composition-bs}$
 $\text{composition-final } \text{fri-query-idxs } \text{composition-round-layers } \text{doms } \text{layers})$

definition $\text{trace-fri-verifier-tied-missing-sampled-layer-chain}$
 $:: (\text{'f}, \text{'a}) \text{protocol-channel-scheme} \Rightarrow$
 $(\text{unit list} \times (\text{'f}, \text{'a}) \text{protocol-channel-scheme}) \text{option} \Rightarrow \text{bool}$

where

$\text{trace-fri-verifier-tied-missing-sampled-layer-chain } s \text{ out} \longleftrightarrow$
 $\text{trace-fri-bad-with-verifier-tied-partial-candidate } s \text{ out} \wedge$
 $\neg \text{trace-fri-bad-with-verifier-tied-sampled-layer-chain } s \text{ out}$

definition $\text{composition-fri-verifier-tied-missing-sampled-layer-chain}$
 $:: (\text{'f}, \text{'a}) \text{protocol-channel-scheme} \Rightarrow$
 $(\text{unit list} \times (\text{'f}, \text{'a}) \text{protocol-channel-scheme}) \text{option} \Rightarrow \text{bool}$

where

$\text{composition-fri-verifier-tied-missing-sampled-layer-chain } s \text{ out}$
 $\longleftrightarrow$
 $\text{composition-fri-bad-with-verifier-tied-partial-candidate } s \text{ out} \wedge$
 $\neg \text{composition-fri-bad-with-verifier-tied-sampled-layer-chain } s \text{ out}$

definition $\text{trace-fri-verifier-tied-reduction}$
 $:: (\text{'f}, \text{'a}) \text{protocol-channel-scheme} \Rightarrow \text{bool}$

where

$\text{trace-fri-verifier-tied-reduction } s \longleftrightarrow$
 $\text{wp-event } \text{verify-monad}$
 $(\text{trace-fri-bad-with-verifier-tied-partial-candidate } s) s \leq$
 trace-fri-error

definition $\text{composition-fri-verifier-tied-reduction}$
 $:: (\text{'f}, \text{'a}) \text{protocol-channel-scheme} \Rightarrow \text{bool}$

where

$\text{composition-fri-verifier-tied-reduction } s \longleftrightarrow$
 $\text{wp-event } \text{verify-monad}$
 $(\text{composition-fri-bad-with-verifier-tied-partial-candidate } s) s \leq$
 $\text{composition-fri-error}$

definition $\text{soundness-bad-event-verifier-tied-aligned-transcript-partial-candidate}$
 $:: (\text{'f}, \text{'a}) \text{protocol-channel-scheme} \Rightarrow$
 $(\text{unit list} \times (\text{'f}, \text{'a}) \text{protocol-channel-scheme}) \text{option} \Rightarrow \text{bool}$

where

$\text{soundness-bad-event-verifier-tied-aligned-transcript-partial-candidate}$
 $s \text{ out} \longleftrightarrow$
 $\text{composition-bad-with-partial-candidates } s \text{ out} \vee$
 $\text{trace-fri-bad-with-header-tied-partial-candidate } s \text{ out} \vee$
 $\text{composition-fri-bad-with-verifier-tied-partial-candidate } s \text{ out} \vee$

query-bad-with-aligned-transcript-partial-candidates s out

definition *soundness-bad-event-verifier-tied-aligned-transcript-partial-candidate-with-partial-merkle*

$:: (f, 'a) \text{ protocol-channel-scheme} \Rightarrow$
 $(\text{unit list} \times (f, 'a) \text{ protocol-channel-scheme}) \text{ option} \Rightarrow \text{bool}$

where

soundness-bad-event-verifier-tied-aligned-transcript-partial-candidate-with-partial-merkle
 $s \text{ out} \longleftrightarrow$
partial-merkle-inconsistency-bad s out $\vee$
soundness-bad-event-verifier-tied-aligned-transcript-partial-candidate
 $s \text{ out}$

lemma *accepted-fri-opening-transcript-obtains-aligned-partial-openings:*

assumes *fri-openings:*

accepted-fri-opening-transcript s (Some (result, final-state))
trace-roots trace-bs trace-final dg composition-roots composition-bs
composition-final query-idxs trace-round-layers
composition-round-layers

and *comp-nonempty:* $\text{composition-roots} \neq []$

obtains *fr as rest trace-openings composition-openings* **where**

verifier-header-transcript s fr trace-roots trace-final as dg
composition-roots composition-final rest
accepted-with-partial-initial-openings-aligned-transcript-consistent s
(Some (result, final-state)) fr trace-roots trace-final as dg
composition-roots composition-final query-idxs trace-openings
composition-openings

proof –

from *fri-openings* **obtain** *result' final-state'* *fr f-fl as fl query-state*

raw-idxs query-chunks **where** *out-eq:*

$\text{Some (result, final-state)} = \text{Some (result', final-state')}$

and *trace-roots-eq:* $\text{trace-roots} = \text{map snd f-fl}$

and *comp-roots-eq:* $\text{composition-roots} = \text{map snd fl}$

and *header:*

verifier-header-transcript s fr trace-roots trace-final as dg
composition-roots composition-final (PTranscript query-state)

and *query-state-state:*

$\text{PState query-state} =$
verifier-header-state s fr trace-roots trace-final as dg
composition-roots composition-final

and *query-state-counter:*

$\text{PQueryCounter query-state} = \text{PQueryCounter s}$

and *query-out:*

$\text{Some (result', final-state')} \in$
set-dist
 $(\text{execute}$
 $(\text{ntimes}$
 $(\text{verifier-query-round-program fr f-fl trace-final as fl}$
 $\text{composition-final})$
 $\text{rounds})$

```

    query-state)
  and len-raw: length raw-idxs = rounds
  and query-idxs-eq:
    query-idxs = map (λraw. index (to-nat raw)) raw-idxs
  and len-query-chunks: length query-chunks = rounds
  and transcript-query:
    PTranscript query-state =
      List.concat query-chunks @ PTranscript final-state'
  and query-chunk:
    ∧i. i < rounds ⇒
      verifier-query-round-chunk (query-idxs ! i)
      trace-roots composition-roots (query-chunks ! i)
  and replay-lookup:
    ∧i. i < rounds ⇒
      fmllookup (HashMap final-state')
      (QueryIndexChallenge
        (PQueryCounter query-state + i)
        (state-after-query-chunks (PState query-state) query-chunks i)) =
        Some (raw-idxs ! i)
  unfolding accepted-fri-opening-transcript-def by blast
  obtain b composition-root fl-tail where fl-eq:
    fl = (b, composition-root) # fl-tail
  using comp-nonempty comp-roots-eq by (cases fl) auto
  have f-roots: map snd f-fl = trace-roots
  using trace-roots-eq by simp
  have comp-roots: map snd fl = composition-roots
  using comp-roots-eq by simp
  have result-eq: result' = result
  and final-state-eq: final-state' = final-state
  using out-eq by simp-all
  have query-out':
    Some (result, final-state) ∈
      set-dist
      (execute
        (ntimes
          (verifier-query-round-program fr f-fl trace-final as fl
            composition-final)
            rounds)
        query-state)
  using query-out result-eq final-state-eq by simp
  have transcript-query':
    PTranscript query-state =
      List.concat query-chunks @ PTranscript final-state
  using transcript-query final-state-eq by simp
  have query-chunk':
    ∧i. i < rounds ⇒
      verifier-query-round-chunk (query-idxs ! i)
      (map snd f-fl) (map snd fl) (query-chunks ! i)
  using query-chunk trace-roots-eq comp-roots-eq by simp

```

```

have replay-lookup':
   $\wedge i. i < \text{rounds} \implies$ 
    fmlookup (HashMap final-state)
      (QueryIndexChallenge
        (PQueryCounter query-state + i)
        (state-after-query-chunks (PState query-state) query-chunks i)) =
        Some (raw-idxs ! i)
  using replay-lookup final-state-eq by simp
from
  query-rounds-replay-accepted-with-partial-initial-openings-aligned-transcript-consistent
    [OF header comp-nonempty fl-eq f-roots comp-roots refl
     query-state-state query-state-counter query-out' len-raw query-idxs-eq
     len-query-chunks transcript-query' query-chunk' replay-lookup']
obtain trace-openings composition-openings where partial:
  accepted-with-partial-initial-openings-aligned-transcript-consistent s
    (Some (result, final-state)) fr trace-roots trace-final as dg
  composition-roots composition-final query-idxs trace-openings
  composition-openings
by blast
show ?thesis
by (rule that[OF header partial])
qed

lemma accepted-fri-opening-transcript-obtains-aligned-partial-candidates-or-partial-merkle:
assumes fri-openings:
  accepted-fri-opening-transcript s (Some (result, final-state))
  trace-roots trace-bs trace-final dg composition-roots composition-bs
  composition-final query-idxs trace-round-layers
  composition-round-layers
and comp-nonempty: composition-roots  $\neq []$ 
shows
  partial-merkle-inconsistency-bad s (Some (result, final-state))  $\vee$ 
  ( $\exists$  fr as rest trace-openings composition-openings trace-table
   composition-table.
    verifier-header-transcript s fr trace-roots trace-final as dg
    composition-roots composition-final rest  $\wedge$ 
    accepted-with-partial-initial-openings-aligned s
    (Some (result, final-state)) fr trace-roots trace-final as dg
    composition-roots composition-final query-idxs trace-openings
    composition-openings  $\wedge$ 
    partial-trace-table-candidate trace-table trace-openings  $\wedge$ 
    partial-composition-table-candidate composition-table
    composition-openings)
proof (cases
  partial-merkle-inconsistency-bad s (Some (result, final-state)))
case True
then show ?thesis by simp
next
case no-merkle: False

```

from *accepted-fri-opening-transcript-obtains-aligned-partial-openings*
 [OF *fri-openings comp-nonempty*]
obtain *fr as rest trace-openings composition-openings* **where** *header:*
verifier-header-transcript s fr trace-roots trace-final as dg
composition-roots composition-final rest
and *partial:*
accepted-with-partial-initial-openings-aligned-transcript-consistent s
(Some (result, final-state)) fr trace-roots trace-final as dg
composition-roots composition-final query-idxs trace-openings
composition-openings
by *blast*
have *aligned:*
accepted-with-partial-initial-openings-aligned s
(Some (result, final-state)) fr trace-roots trace-final as dg
composition-roots composition-final query-idxs trace-openings
composition-openings
using *partial*
by (*blast dest:*
accepted-with-partial-initial-openings-aligned-transcript-consistent-imp-aligned
accepted-with-partial-initial-openings-aligned-consistent-imp-aligned)
obtain *trace-table composition-table* **where** *trace-candidate:*
partial-trace-table-candidate trace-table trace-openings
and *composition-candidate:*
partial-composition-table-candidate composition-table
composition-openings
using
accepted-with-partial-initial-openings-aligned-candidates-if-no-partial-merkle-bad
 [OF *aligned no-merkle*]
by *blast*
show *?thesis*
by (*intro disjI2 exI conjI*)
(rule header, rule aligned, rule trace-candidate,
rule composition-candidate)
qed

lemma *accepted-verifier-tied-aligned-transcript-partial-candidate-partition:*
assumes *false-statement: $\neg$ exists-valid-trace*
and *fri-openings:*
accepted-fri-opening-transcript s out trace-roots trace-bs
trace-final fri-dg composition-roots composition-bs
composition-final query-idxs trace-round-layers
composition-round-layers
and *partial:*
accepted-with-partial-initial-openings-aligned-transcript-consistent s
out fr trace-roots trace-final as fri-dg composition-roots
composition-final
query-idxs trace-openings composition-openings
and *header:*
verifier-header-transcript s fr trace-roots trace-final as fri-dg

```

    composition-roots composition-final rest
  and trace-candidate:
    partial-trace-table-candidate trace-table trace-openings
  and composition-candidate:
    partial-composition-table-candidate composition-table
    composition-openings
shows
  soundness-bad-event-verifier-tied-aligned-transcript-partial-candidate
  s out
proof (cases trace-table-low-degree trace-table)
case False
have aligned:
  accepted-with-partial-initial-openings-aligned-consistent s out fr
  trace-roots trace-final as fri-dg composition-roots composition-final query-idxs
  trace-openings composition-openings
by (rule
  accepted-with-partial-initial-openings-aligned-transcript-consistent-imp-aligned
  [OF partial])
have aligned0:
  accepted-with-partial-initial-openings-aligned s out fr trace-roots
  trace-final as fri-dg composition-roots composition-final query-idxs
  trace-openings composition-openings
by (rule accepted-with-partial-initial-openings-aligned-consistent-imp-aligned
  [OF aligned])
have trace-partial:
  accepted-with-partial-trace-openings s out fr query-idxs
  trace-openings
by (rule accepted-with-partial-initial-openings-aligned-shapes(3)
  [OF aligned0])
have trace-fri-bad-with-header-tied-partial-candidate s out
unfolding trace-fri-bad-with-header-tied-partial-candidate-def
by (intro exI conjI)
  (rule fri-openings, rule header, rule trace-partial,
  rule trace-candidate, rule False)
then show ?thesis
unfolding
  soundness-bad-event-verifier-tied-aligned-transcript-partial-candidate-def
by blast
next
case True
note trace-low = True
show ?thesis
proof (cases composition-table-low-degree maxDegree composition-table)
case False
have aligned:
  accepted-with-partial-initial-openings-aligned-consistent s out fr
  trace-roots trace-final as fri-dg composition-roots composition-final query-idxs
  trace-openings composition-openings
by (rule

```

```

    accepted-with-partial-initial-openings-aligned-transcript-consistent-imp-aligned
    [OF partial])
have aligned0:
  accepted-with-partial-initial-openings-aligned s out fr trace-roots
  trace-final as fri-dg composition-roots composition-final query-idxs
  trace-openings composition-openings
by (rule
  accepted-with-partial-initial-openings-aligned-consistent-imp-aligned
  [OF aligned])
have composition-fri-bad-with-verifier-tied-partial-candidate s out
  unfolding composition-fri-bad-with-verifier-tied-partial-candidate-def
by (intro exI conjI)
  (rule fri-openings, rule aligned0, rule trace-candidate,
  rule composition-candidate, rule trace-low, rule False)
then show ?thesis
  unfolding
    soundness-bad-event-verifier-tied-aligned-transcript-partial-candidate-def
  by blast
next
case True
note composition-low = True
show ?thesis
proof (cases all-queries-consistent trace-table composition-table as)
  case False
  have query-bad-with-aligned-transcript-partial-candidates s out
  unfolding query-bad-with-aligned-transcript-partial-candidates-def
  using partial trace-candidate composition-candidate trace-low
  composition-low False
  by blast
then show ?thesis
  unfolding
    soundness-bad-event-verifier-tied-aligned-transcript-partial-candidate-def
  by blast
next
case True
note all-queries = True
from trace-low obtain f where
  deg-f: degree f < clength
  and trace-table-eq: trace-table = map (poly f) eval-domain
  unfolding trace-table-low-degree-def by blast
have violated: violated-constraints f ≠ {}
  by (rule false-statement-violated-constraints
  [OF false-statement deg-f])
have aligned:
  accepted-with-partial-initial-openings-aligned-consistent s out fr
  trace-roots trace-final as fri-dg composition-roots composition-final query-idxs
  trace-openings composition-openings
by (rule
  accepted-with-partial-initial-openings-aligned-transcript-consistent-imp-aligned

```

```

    [OF partial])
  have composition-bad-with-partial-candidates s out
    unfolding composition-bad-with-partial-candidates-def
    using accepted-with-partial-initial-openings-aligned-consistent-imp-unaligned
      [OF aligned]
      trace-candidate composition-candidate deg-f trace-table-eq
      violated composition-low all-queries
    by blast
  then show ?thesis
    unfolding
      soundness-bad-event-verifier-tied-aligned-transcript-partial-candidate-def
    by blast
qed
qed
qed

lemma accepted-fri-opening-transcript-verifier-tied-partition-with-partial-merkle:
  assumes false-statement:  $\neg$  exists-valid-trace
  and fri-openings:
    accepted-fri-opening-transcript s (Some (result, final-state))
    trace-roots trace-bs trace-final dg composition-roots composition-bs
    composition-final query-idxs trace-round-layers
    composition-round-layers
  and comp-nonempty: composition-roots  $\neq$  []
  shows
    soundness-bad-event-verifier-tied-aligned-transcript-partial-candidate-with-partial-merkle
      s (Some (result, final-state))
  proof -
    from accepted-fri-opening-transcript-obtains-aligned-partial-openings
      [OF fri-openings comp-nonempty]
    obtain fr as rest trace-openings composition-openings where header:
      verifier-header-transcript s fr trace-roots trace-final as dg
      composition-roots composition-final rest
    and partial:
      accepted-with-partial-initial-openings-aligned-transcript-consistent s
        (Some (result, final-state)) fr trace-roots trace-final as dg
        composition-roots composition-final query-idxs trace-openings
        composition-openings
    by blast
  show ?thesis
  proof (cases
    partial-merkle-inconsistency-bad s (Some (result, final-state)))
  case True
  then show ?thesis
    unfolding
      soundness-bad-event-verifier-tied-aligned-transcript-partial-candidate-with-partial-merkle-def
    by simp
  next
  case False

```



```

have aligned:
  accepted-with-partial-initial-openings-aligned s
    (Some (result, final-state)) fr trace-roots trace-final as dg
      composition-roots composition-final query-idxs trace-openings
        composition-openings
  using partial
  by (blast dest:
    accepted-with-partial-initial-openings-aligned-transcript-consistent-imp-aligned
    accepted-with-partial-initial-openings-aligned-consistent-imp-aligned)
from accepted-with-partial-initial-openings-aligned-candidates-if-no-partial-merkle-bad
  [OF aligned False]
obtain trace-table composition-table where trace-candidate:
  partial-trace-table-candidate trace-table trace-openings
and composition-candidate:
  partial-composition-table-candidate composition-table
    composition-openings
by blast
have bad:
  soundness-bad-event-verifier-tied-aligned-transcript-partial-candidate
    s (Some (result, final-state))
by (rule accepted-verifier-tied-aligned-transcript-partial-candidate-partition
  [OF false-statement fri-openings partial header trace-candidate
    composition-candidate])
then show ?thesis
  unfolding
    soundness-bad-event-verifier-tied-aligned-transcript-partial-candidate-with-partial-merkle-def
  by simp
qed
qed

lemma accepted-fri-opening-transcript-headerE:
assumes fri-openings:
  accepted-fri-opening-transcript s out trace-roots trace-bs trace-final
  dg composition-roots composition-bs composition-final query-idxs
  trace-round-layers composition-round-layers
obtains fr as rest where
  verifier-header-transcript s fr trace-roots trace-final as dg
    composition-roots composition-final rest
using fri-openings
unfolding accepted-fri-opening-transcript-def
by blast

lemma accepted-partition-soundness-bad-event-verifier-tied-aligned-transcript-partial-candidate-with-partial-merkle-o
assumes false-statement:  $\neg$  exists-valid-trace
  and supp: out  $\in$  set-dist (execute verify-monad s)
  and acc: accepted out
shows
  soundness-bad-event-verifier-tied-aligned-transcript-partial-candidate-with-partial-merkle
    s out  $\vee$ 

```

```

    soundness-bad-event-partial-candidate-empty-header s out
proof –
  from acc obtain result final-state where out-eq:
    out = Some (result, final-state)
    unfolding accepted-def by (cases out) auto
  from verify-monad-accepted-fri-opening-transcript[OF supp[unfolded out-eq]]
obtain trace-roots trace-bs trace-final dg composition-roots
  composition-bs composition-final query-idxs trace-round-layers
  composition-round-layers
where fri-openings:
  accepted-fri-opening-transcript s (Some (result, final-state))
  trace-roots trace-bs trace-final dg composition-roots composition-bs
  composition-final query-idxs trace-round-layers
  composition-round-layers
by blast
show ?thesis
proof (cases composition-roots = [])
  case False
  have
    soundness-bad-event-verifier-tied-aligned-transcript-partial-candidate-with-partial-merkle
      s (Some (result, final-state))
    by (rule
      accepted-fri-opening-transcript-verifier-tied-partition-with-partial-merkle
      [OF false-statement fri-openings False])
  then show ?thesis
    unfolding out-eq by simp
next
  case True
  note comp-empty = True
  from accepted-fri-opening-transcript-headerE[OF fri-openings]
obtain fr as rest where header:
  verifier-header-transcript s fr trace-roots trace-final as dg
  composition-roots composition-final rest
  by blast
show ?thesis
proof (cases
  partial-merkle-inconsistency-bad s (Some (result, final-state)))
  case True
  then have
    soundness-bad-event-verifier-tied-aligned-transcript-partial-candidate-with-partial-merkle
      s (Some (result, final-state))
    unfolding
    soundness-bad-event-verifier-tied-aligned-transcript-partial-candidate-with-partial-merkle-def
    by simp
  then show ?thesis
    unfolding out-eq by simp
next
  case False
  from verify-monad-accepted-with-partial-trace-openings-for-header

```

```

    [OF supp[unfolded out-eq] header]
obtain trace-query-idxs trace-openings where trace-partial:
  accepted-with-partial-trace-openings s
  (Some (result, final-state)) fr trace-query-idxs trace-openings
by blast
obtain trace-table where trace-candidate:
  partial-trace-table-candidate trace-table trace-openings
using
  accepted-with-partial-trace-openings-candidate-if-no-partial-merkle-bad
  [OF trace-partial False]
by blast
have accepted-out: accepted (Some (result, final-state))
by (rule accepted-with-partial-trace-openings-imp-accepted
  [OF trace-partial])
let ?composition-table = replicate (scale * clength) composition-final
have empty-partial:
  accepted-with-empty-composition-header-candidates s
  (Some (result, final-state)) fr trace-roots trace-final as dg
  composition-final trace-query-idxs trace-openings trace-table
  ?composition-table
unfolding accepted-with-empty-composition-header-candidates-def
by (intro conjI exI[of - rest])
  (use accepted-out header comp-empty trace-partial trace-candidate
  in simp-all)
have soundness-bad-event-partial-candidate-empty-header s
  (Some (result, final-state))
by (rule accepted-empty-composition-header-candidate-partition
  [OF false-statement empty-partial])
then show ?thesis
  unfolding out-eq by simp
qed
qed
qed

lemma soundness-bad-event-verifier-tied-aligned-transcript-partial-candidate-union-bound:
assumes comp-bound:
  wp-event verify-monad
  (composition-bad-with-partial-candidates s) s ≤
  composition-error
and trace-fri-bound:
  wp-event verify-monad
  (trace-fri-bad-with-header-tied-partial-candidate s) s ≤
  trace-fri-error'
and comp-fri-bound:
  wp-event verify-monad
  (composition-fri-bad-with-verifier-tied-partial-candidate s) s ≤
  composition-fri-error'
and query-bound:
  wp-event verify-monad

```

$(\text{query-bad-with-aligned-transcript-partial-candidates } s) \leq \text{query-error}$

shows

$\text{wp-event verify-monad } (\text{soundness-bad-event-verifier-tied-aligned-transcript-partial-candidate } s) \leq \text{composition-error} + \text{trace-fri-error}' + \text{composition-fri-error}' + \text{query-error}$

proof –

have $\text{wp-event verify-monad } (\text{soundness-bad-event-verifier-tied-aligned-transcript-partial-candidate } s) \leq$
 $\text{wp-event verify-monad } (\text{composition-bad-with-partial-candidates } s) +$
 $\text{wp-event verify-monad } (\text{trace-fri-bad-with-header-tied-partial-candidate } s) +$
 $\text{wp-event verify-monad } (\text{composition-fri-bad-with-verifier-tied-partial-candidate } s) +$
 $\text{wp-event verify-monad } (\text{query-bad-with-aligned-transcript-partial-candidates } s)$

unfolding

$\text{soundness-bad-event-verifier-tied-aligned-transcript-partial-candidate-def}$
by $(\text{rule wp-event-union-bound}_4)$

also have $\dots \leq \text{composition-error} + \text{trace-fri-error}' + \text{composition-fri-error}' + \text{query-error}$

by $(\text{intro add-mono comp-bound trace-fri-bound comp-fri-bound query-bound})$

finally show $?thesis$.

qed

lemma *soundness-bad-event-verifier-tied-aligned-transcript-partial-candidate-with-partial-merkle-union-bound:*

assumes *merkle-bound:*

$\text{wp-event verify-monad } (\text{partial-merkle-inconsistency-bad } s) \leq \text{merkle-error}$

and *comp-bound:*

$\text{wp-event verify-monad } (\text{composition-bad-with-partial-candidates } s) \leq \text{composition-error}$

and *trace-fri-bound:*

$\text{wp-event verify-monad } (\text{trace-fri-bad-with-header-tied-partial-candidate } s) \leq \text{trace-fri-error}'$

and *comp-fri-bound:*

$\text{wp-event verify-monad } (\text{composition-fri-bad-with-verifier-tied-partial-candidate } s) \leq \text{composition-fri-error}'$

and *query-bound:*

$\text{wp-event verify-monad } (\text{query-bad-with-aligned-transcript-partial-candidates } s) \leq \text{query-error}$

$(\text{query-bad-with-aligned-transcript-partial-candidates } s) \ s \leq \text{query-error}$

shows

$\text{wp-event verify-monad}$
 $(\text{soundness-bad-event-verifier-tied-aligned-transcript-partial-candidate-with-partial-merkle } s) \ s \leq$
 $\text{merkle-error} + \text{composition-error} + \text{trace-fri-error}' +$
 $\text{composition-fri-error}' + \text{query-error}$

proof –

have *partial-bound*:

$\text{wp-event verify-monad}$
 $(\text{soundness-bad-event-verifier-tied-aligned-transcript-partial-candidate } s) \ s \leq$
 $\text{composition-error} + \text{trace-fri-error}' + \text{composition-fri-error}' +$
 query-error

by (rule
 $\text{soundness-bad-event-verifier-tied-aligned-transcript-partial-candidate-union-bound}$
 $[\text{OF comp-bound trace-fri-bound comp-fri-bound query-bound}]$)

have *wp-event verify-monad*

$(\text{soundness-bad-event-verifier-tied-aligned-transcript-partial-candidate-with-partial-merkle } s) \ s \leq$
 $\text{wp-event verify-monad } (\text{partial-merkle-inconsistency-bad } s) \ s +$
 $\text{wp-event verify-monad}$
 $(\text{soundness-bad-event-verifier-tied-aligned-transcript-partial-candidate } s) \ s$

unfolding

$\text{soundness-bad-event-verifier-tied-aligned-transcript-partial-candidate-with-partial-merkle-def}$

by (rule *wp-event-union-bound*)

also have $\dots \leq$

$\text{merkle-error} +$
 $(\text{composition-error} + \text{trace-fri-error}' + \text{composition-fri-error}' +$
 $\text{query-error})$

by (intro *add-mono merkle-bound partial-bound*)

finally show *?thesis*

by (*simp add: algebra-simps*)

qed

lemma *checked-staged-security-with-data-state-verifier-tied-aligned-transcript-partial-candidate-union-bound*:

assumes *comp-bound*:

$\text{wp-event } (\text{checked-staged-security-experiment-with-data-state } A)$
 $(\text{staged-security-with-data-state-verifier-event}$
 $\text{composition-bad-with-partial-candidates})$
 $\text{adversary-initial-state} \leq \text{composition-error}$

and *trace-fri-bound*:

$\text{wp-event } (\text{checked-staged-security-experiment-with-data-state } A)$
 $(\text{staged-security-with-data-state-verifier-event}$
 $\text{trace-fri-bad-with-header-tied-partial-candidate})$
 $\text{adversary-initial-state} \leq \text{trace-fri-error}'$

and *comp-fri-bound*:

```

    wp-event (checked-staged-security-experiment-with-data-state A)
      (staged-security-with-data-state-verifier-event
        composition-fri-bad-with-verifier-tied-partial-candidate)
      adversary-initial-state ≤ composition-fri-error'
  and query-bound:
    wp-event (checked-staged-security-experiment-with-data-state A)
      (staged-security-with-data-state-verifier-event
        query-bad-with-aligned-transcript-partial-candidates)
      adversary-initial-state ≤ query-error
  shows
    wp-event (checked-staged-security-experiment-with-data-state A)
      (staged-security-with-data-state-verifier-event
        soundness-bad-event-verifier-tied-aligned-transcript-partial-candidate)
      adversary-initial-state ≤
        composition-error + trace-fri-error' + composition-fri-error' +
        query-error
  proof -
    let ?M = checked-staged-security-experiment-with-data-state A
    let ?E = staged-security-with-data-state-verifier-event
    let ?comp = ?E composition-bad-with-partial-candidates
    let ?trace = ?E trace-fri-bad-with-header-tied-partial-candidate
    let ?comp-fri =
      ?E composition-fri-bad-with-verifier-tied-partial-candidate
    let ?query = ?E query-bad-with-aligned-transcript-partial-candidates
    let ?partial =
      ?E soundness-bad-event-verifier-tied-aligned-transcript-partial-candidate
    have wp-event ?M ?partial adversary-initial-state ≤
      wp-event ?M
        (λout. ?comp out ∨ ?trace out ∨ ?comp-fri out ∨
          ?query out)
        adversary-initial-state
    by (rule wp-event-mono)
      (auto simp: staged-security-with-data-state-verifier-event-def
        soundness-bad-event-verifier-tied-aligned-transcript-partial-candidate-def
        split: option.splits prod.splits)
    also have ... ≤
      wp-event ?M ?comp adversary-initial-state +
      wp-event ?M ?trace adversary-initial-state +
      wp-event ?M ?comp-fri adversary-initial-state +
      wp-event ?M ?query adversary-initial-state
    by (rule wp-event-union-bound4)
    also have ... ≤
      composition-error + trace-fri-error' + composition-fri-error' +
      query-error
    by (intro add-mono comp-bound trace-fri-bound comp-fri-bound
      query-bound)
    finally show ?thesis .
  qed

```

lemma *checked-staged-security-with-data-state-verifier-tied-aligned-transcript-partial-candidate-with-partial-merkle-u*

assumes *merkle-bound*:

wp-event (*checked-staged-security-experiment-with-data-state A*)
 (*staged-security-with-data-state-verifier-event*
partial-merkle-inconsistency-bad)
adversary-initial-state $\leq$ *merkle-error*

and *comp-bound*:

wp-event (*checked-staged-security-experiment-with-data-state A*)
 (*staged-security-with-data-state-verifier-event*
composition-bad-with-partial-candidates)
adversary-initial-state $\leq$ *composition-error*

and *trace-fri-bound*:

wp-event (*checked-staged-security-experiment-with-data-state A*)
 (*staged-security-with-data-state-verifier-event*
trace-fri-bad-with-header-tied-partial-candidate)
adversary-initial-state $\leq$ *trace-fri-error'*

and *comp-fri-bound*:

wp-event (*checked-staged-security-experiment-with-data-state A*)
 (*staged-security-with-data-state-verifier-event*
composition-fri-bad-with-verifier-tied-partial-candidate)
adversary-initial-state $\leq$ *composition-fri-error'*

and *query-bound*:

wp-event (*checked-staged-security-experiment-with-data-state A*)
 (*staged-security-with-data-state-verifier-event*
query-bad-with-aligned-transcript-partial-candidates)
adversary-initial-state $\leq$ *query-error*

shows

wp-event (*checked-staged-security-experiment-with-data-state A*)
 (*staged-security-with-data-state-verifier-event*
soundness-bad-event-verifier-tied-aligned-transcript-partial-candidate-with-partial-merkle)
adversary-initial-state $\leq$
merkle-error + *composition-error* + *trace-fri-error'* +
composition-fri-error' + *query-error*

proof –

have *partial-bound*:

wp-event (*checked-staged-security-experiment-with-data-state A*)
 (*staged-security-with-data-state-verifier-event*
soundness-bad-event-verifier-tied-aligned-transcript-partial-candidate)
adversary-initial-state $\leq$
composition-error + *trace-fri-error'* + *composition-fri-error'* +
query-error

by (rule

checked-staged-security-with-data-state-verifier-tied-aligned-transcript-partial-candidate-union-bound
 [OF *comp-bound trace-fri-bound comp-fri-bound query-bound*])

let *?M* = *checked-staged-security-experiment-with-data-state A*

let *?E* = *staged-security-with-data-state-verifier-event*

let *?merkle* = *?E partial-merkle-inconsistency-bad*

let *?partial* =

?E soundness-bad-event-verifier-tied-aligned-transcript-partial-candidate

```

let ?with-merkle =
  ?E soundness-bad-event-verifier-tied-aligned-transcript-partial-candidate-with-partial-merkle
have wp-event ?M ?with-merkle adversary-initial-state ≤
  wp-event ?M (λout. ?merkle out ∨ ?partial out)
  adversary-initial-state
by (rule wp-event-mono)
  (auto simp: staged-security-with-data-state-verifier-event-def
    soundness-bad-event-verifier-tied-aligned-transcript-partial-candidate-with-partial-merkle-def
    split: option.splits prod.splits)
also have ... ≤
  wp-event ?M ?merkle adversary-initial-state +
  wp-event ?M ?partial adversary-initial-state
by (rule wp-event-union-bound)
also have ... ≤
  merkle-error +
  (composition-error + trace-fri-error' + composition-fri-error' +
    query-error)
by (intro add-mono merkle-bound partial-bound)
finally show ?thesis
by (simp add: algebra-simps)
qed

```

```

lemma checked-staged-soundness-from-verifier-tied-aligned-transcript-partial-candidate-and-empty-header-bounds:
fixes partial-candidate-error empty-header-error :: prob
assumes false-statement: ¬ exists-valid-trace
and partial-candidate-bound:
  wp-event (checked-staged-security-experiment-with-data-state A)
  (staged-security-with-data-state-verifier-event
    soundness-bad-event-verifier-tied-aligned-transcript-partial-candidate-with-partial-merkle)
  adversary-initial-state ≤ partial-candidate-error
and empty-header-bound:
  wp-event (checked-staged-security-experiment-with-data-state A)
  (staged-security-with-data-state-verifier-event
    soundness-bad-event-partial-candidate-empty-header)
  adversary-initial-state ≤ empty-header-error
shows checked-staged-adversary-acceptance-probability A ≤
  partial-candidate-error + empty-header-error
proof -
let ?M = checked-staged-security-experiment-with-data-state A
let ?E = staged-security-with-data-state-verifier-event
let ?partial =
  ?E soundness-bad-event-verifier-tied-aligned-transcript-partial-candidate-with-partial-merkle
let ?empty = ?E soundness-bad-event-partial-candidate-empty-header
let ?bad = λout. ?partial out ∨ ?empty out
have accepted-bad:
  wp-event ?M accepted adversary-initial-state ≤
  wp-event ?M ?bad adversary-initial-state
proof (rule wp-event-mono-on-support)
fix out

```



```

assume support: out  $\in$  set-dist (execute ?M adversary-initial-state)
  and acc: accepted out
show ?bad out
proof (cases out)
  case None
  then show ?thesis
    using acc unfolding accepted-def by simp
next
  case (Some packed)
  then obtain data attacker-state result final-state where out-eq:
    out = Some (((data, attacker-state), result), final-state)
    by (cases packed, auto split: prod.splits)
  let ?s =
    verifier-state-from-adversary attacker-state
    (staged-proof-transcript data)
  have verifier:
    Some (result, final-state)  $\in$  set-dist (execute verify-monad ?s)
    using checked-staged-security-experiment-with-data-state-outcomeE
      [OF support[unfolded out-eq]]
    by blast
  have verifier-acc: accepted (Some (result, final-state))
    unfolding accepted-def by simp
  have local-bad:
    soundness-bad-event-verifier-tied-aligned-transcript-partial-candidate-with-partial-merkle
      ?s (Some (result, final-state))  $\vee$ 
    soundness-bad-event-partial-candidate-empty-header ?s
      (Some (result, final-state))
    by (rule
      accepted-partition-soundness-bad-event-verifier-tied-aligned-transcript-partial-candidate-with-partial-merkle-or
      [OF false-statement verifier verifier-acc])
  then show ?thesis
    unfolding out-eq staged-security-with-data-state-verifier-event-def
    by simp
  qed
qed
have bad-bound:
  wp-event ?M ?bad adversary-initial-state  $\leq$ 
    partial-candidate-error + empty-header-error
proof –
  have wp-event ?M ?bad adversary-initial-state  $\leq$ 
    wp-event ?M ?partial adversary-initial-state +
    wp-event ?M ?empty adversary-initial-state
    by (rule wp-event-union-bound)
  also have ...  $\leq$  partial-candidate-error + empty-header-error
    by (intro add-mono partial-candidate-bound empty-header-bound)
  finally show ?thesis .
qed
have staged-bound:
  wp-event (checked-staged-security-experiment A) accepted

```

```

    adversary-initial-state ≤
    partial-candidate-error + empty-header-error
unfolding checked-staged-security-experiment-acceptance-with-data-state
by (rule order-trans[OF accepted-bad bad-bound])
show ?thesis
proof –
  have
    wp-event (checked-staged-security-experiment A)
      (λout. ¬ Option.is-none out) adversary-initial-state =
    wp-event (checked-staged-security-experiment A) accepted
      adversary-initial-state
  unfolding wp-event-def accepted-def
  by (rule arg-cong[where
    f=λQ. wp (checked-staged-security-experiment A) Q
    adversary-initial-state])
    (rule ext, simp)
  then show ?thesis
    unfolding checked-staged-adversary-acceptance-probability-def
    using staged-bound by simp
qed
qed

```

lemma checked-staged-soundness-from-verifier-tied-aligned-transcript-partial-candidate-components-and-empty-head

fixes merkle-error composition-error trace-fri-error'
 composition-fri-error' query-error empty-header-error :: prob

assumes false-statement: ¬ exists-valid-trace

and merkle-bound:

wp-event (checked-staged-security-experiment-with-data-state A)
 (staged-security-with-data-state-verifier-event
 partial-merkle-inconsistency-bad)
 adversary-initial-state ≤ merkle-error

and comp-bound:

wp-event (checked-staged-security-experiment-with-data-state A)
 (staged-security-with-data-state-verifier-event
 composition-bad-with-partial-candidates)
 adversary-initial-state ≤ composition-error

and trace-fri-bound:

wp-event (checked-staged-security-experiment-with-data-state A)
 (staged-security-with-data-state-verifier-event
 trace-fri-bad-with-header-tied-partial-candidate)
 adversary-initial-state ≤ trace-fri-error'

and comp-fri-bound:

wp-event (checked-staged-security-experiment-with-data-state A)
 (staged-security-with-data-state-verifier-event
 composition-fri-bad-with-verifier-tied-partial-candidate)
 adversary-initial-state ≤ composition-fri-error'

and query-bound:

wp-event (checked-staged-security-experiment-with-data-state A)
 (staged-security-with-data-state-verifier-event

query-bad-with-aligned-transcript-partial-candidates)
adversary-initial-state $\leq$ *query-error*
and *empty-header-bound*:
wp-event (*checked-staged-security-experiment-with-data-state* *A*)
(*staged-security-with-data-state-verifier-event*
soundness-bad-event-partial-candidate-empty-header)
adversary-initial-state $\leq$ *empty-header-error*
shows *checked-staged-adversary-acceptance-probability* *A* $\leq$
merkle-error + *composition-error* + *trace-fri-error'* +
composition-fri-error' + *query-error* + *empty-header-error*
proof –
have *partial-candidate-bound*:
wp-event (*checked-staged-security-experiment-with-data-state* *A*)
(*staged-security-with-data-state-verifier-event*
soundness-bad-event-verifier-tied-aligned-transcript-partial-candidate-with-partial-merkle)
adversary-initial-state $\leq$
merkle-error + *composition-error* + *trace-fri-error'* +
composition-fri-error' + *query-error*
by (rule
checked-staged-security-with-data-state-verifier-tied-aligned-transcript-partial-candidate-with-partial-merkle-union
[*OF merkle-bound comp-bound trace-fri-bound comp-fri-bound*
query-bound])
have *bound*:
checked-staged-adversary-acceptance-probability *A* $\leq$
(*merkle-error* + *composition-error* + *trace-fri-error'* +
composition-fri-error' + *query-error*) + *empty-header-error*
by (rule
checked-staged-soundness-from-verifier-tied-aligned-transcript-partial-candidate-and-empty-header-bounds
[*OF false-statement partial-candidate-bound empty-header-bound*])
then show *?thesis*
by (*simp add: algebra-simps*)
qed

lemma *trace-fri-bad-with-verifier-tied-partial-candidateE*:
assumes
trace-fri-bad-with-verifier-tied-partial-candidate *s out*
obtains *trace-roots trace-bs trace-final dg composition-roots*
composition-bs composition-final fri-query-idxs trace-round-layers
composition-round-layers fr trace-openings trace-table
where
accepted-fri-opening-transcript *s out trace-roots trace-bs*
trace-final dg composition-roots composition-bs composition-final
fri-query-idxs trace-round-layers composition-round-layers
accepted-with-partial-trace-openings *s out fr fri-query-idxs*
trace-openings
partial-trace-table-candidate trace-table trace-openings
 $\neg$ *trace-table-low-degree trace-table*
using *assms*
unfolding *trace-fri-bad-with-verifier-tied-partial-candidate-def*

by blast

lemma *composition-fri-bad-with-verifier-tied-partial-candidateE:*

assumes

composition-fri-bad-with-verifier-tied-partial-candidate s out

obtains *trace-roots trace-bs trace-final fri-dg composition-roots*

composition-bs composition-final fri-query-idxs trace-round-layers

composition-round-layers fr as trace-openings composition-openings

trace-table composition-table

where

accepted-fri-opening-transcript s out trace-roots trace-bs

trace-final fri-dg composition-roots composition-bs

composition-final fri-query-idxs trace-round-layers

composition-round-layers

accepted-with-partial-initial-openings-aligned s out fr trace-roots

trace-final as fri-dg composition-roots composition-final fri-query-idxs

trace-openings composition-openings

partial-trace-table-candidate trace-table trace-openings

partial-composition-table-candidate composition-table

composition-openings

trace-table-low-degree trace-table

$\neg$ *composition-table-low-degree maxDegree composition-table*

proof —

obtain *trace-roots trace-bs trace-final fri-dg composition-roots*

composition-bs composition-final fri-query-idxs trace-round-layers

composition-round-layers fr as trace-openings composition-openings

trace-table composition-table

where *fri-openings:*

accepted-fri-opening-transcript s out trace-roots trace-bs

trace-final fri-dg composition-roots composition-bs

composition-final fri-query-idxs trace-round-layers

composition-round-layers

and *aligned:*

accepted-with-partial-initial-openings-aligned s out fr trace-roots

trace-final as fri-dg composition-roots composition-final fri-query-idxs

trace-openings composition-openings

and *trace-cand:*

partial-trace-table-candidate trace-table trace-openings

and *comp-cand:*

partial-composition-table-candidate composition-table

composition-openings

and *trace-low:*

trace-table-low-degree trace-table

and *comp-not-low:*

$\neg$ *composition-table-low-degree maxDegree composition-table*

using *assms*

unfolding *composition-fri-bad-with-verifier-tied-partial-candidate-def*

by *metis*

show *?thesis*

by (rule that[*OF fri-openings aligned trace-cand comp-cand trace-low comp-not-low*])

qed

lemma *trace-fri-bad-with-verifier-tied-imp-reachable-partial-candidate*:

assumes

trace-fri-bad-with-verifier-tied-partial-candidate s out

shows *trace-fri-bad-with-reachable-partial-candidate s out*

proof (rule *trace-fri-bad-with-verifier-tied-partial-candidateE*[*OF assms*])

fix *trace-roots trace-bs trace-final dg composition-roots*
composition-bs composition-final fri-query-idxs trace-round-layers
composition-round-layers fr trace-openings trace-table

assume *partial*:

accepted-with-partial-trace-openings s out fr fri-query-idxs
 trace-openings

and *cand*:

partial-trace-table-candidate trace-table trace-openings

and *not-low*: $\neg$ *trace-table-low-degree trace-table*

show ?thesis

 unfolding *trace-fri-bad-with-reachable-partial-candidate-def*

 by (intro *exI conjI*; (rule *partial* | rule *cand* | rule *not-low*))

qed

lemma *composition-fri-bad-with-verifier-tied-imp-reachable-partial-candidate*:

assumes

composition-fri-bad-with-verifier-tied-partial-candidate s out

shows *composition-fri-bad-with-reachable-partial-candidate s out*

proof (rule

composition-fri-bad-with-verifier-tied-partial-candidateE[*OF assms*])

fix *trace-roots trace-bs trace-final fri-dg composition-roots*
composition-bs composition-final fri-query-idxs trace-round-layers
composition-round-layers fr as trace-openings composition-openings
trace-table composition-table

assume *aligned*:

accepted-with-partial-initial-openings-aligned s out fr trace-roots
 trace-final as fri-dg composition-roots composition-final fri-query-idxs
 trace-openings composition-openings

and *trace-cand*:

partial-trace-table-candidate trace-table trace-openings

and *comp-cand*:

partial-composition-table-candidate composition-table
 composition-openings

and *trace-low*: *trace-table-low-degree trace-table*

and *comp-not-low*:

$\neg$ *composition-table-low-degree maxDegree composition-table*

have *unaligned*:

accepted-with-partial-initial-openings s out fr trace-roots
 trace-final as fri-dg composition-roots composition-final
 fri-query-idxs trace-openings fri-query-idxs composition-openings

```

    by (rule accepted-with-partial-initial-openings-aligned-imp-unaligned
        [OF aligned])
  show ?thesis
    unfolding composition-fri-bad-with-reachable-partial-candidate-def
    by (intro exI conjI; (rule unaligned | rule trace-cand |
        rule comp-cand | rule trace-low | rule comp-not-low))
qed

lemma trace-fri-bad-with-verifier-tied-partial-candidate-None[simp]:
  ¬ trace-fri-bad-with-verifier-tied-partial-candidate s None
  unfolding trace-fri-bad-with-verifier-tied-partial-candidate-def
    accepted-fri-opening-transcript-def
  by simp

lemma composition-fri-bad-with-verifier-tied-partial-candidate-None[simp]:
  ¬ composition-fri-bad-with-verifier-tied-partial-candidate s None
  unfolding composition-fri-bad-with-verifier-tied-partial-candidate-def
    accepted-fri-opening-transcript-def
  by simp

lemma trace-fri-verifier-tied-imp-sampled-or-missing-layer-chain:
  assumes trace-fri-bad-with-verifier-tied-partial-candidate s out
  shows
    trace-fri-bad-with-verifier-tied-sampled-layer-chain s out ∨
    trace-fri-verifier-tied-missing-sampled-layer-chain s out
  using assms
  unfolding trace-fri-verifier-tied-missing-sampled-layer-chain-def
  by blast

lemma composition-fri-verifier-tied-imp-sampled-or-missing-layer-chain:
  assumes composition-fri-bad-with-verifier-tied-partial-candidate s out
  shows
    composition-fri-bad-with-verifier-tied-sampled-layer-chain s out ∨
    composition-fri-verifier-tied-missing-sampled-layer-chain s out
  using assms
  unfolding composition-fri-verifier-tied-missing-sampled-layer-chain-def
  by blast

lemma wp-trace-fri-verifier-tied-bound-from-sampled-and-missing:
  assumes sampled-bound:
    wp-event verify-monad
      (trace-fri-bad-with-verifier-tied-sampled-layer-chain s) s ≤ R
  and missing-bound:
    wp-event verify-monad
      (trace-fri-verifier-tied-missing-sampled-layer-chain s) s ≤ M
  shows
    wp-event verify-monad
      (trace-fri-bad-with-verifier-tied-partial-candidate s) s ≤ R + M
proof -

```

have *wp-event verify-monad*
 (*trace-fri-bad-with-verifier-tied-partial-candidate s*) $s \leq$
wp-event verify-monad
 ($\lambda out. \text{trace-fri-bad-with-verifier-tied-sampled-layer-chain } s \text{ out} \vee$
 $\text{trace-fri-verifier-tied-missing-sampled-layer-chain } s \text{ out}$) s
by (*rule wp-event-mono*)
 (*rule trace-fri-verifier-tied-imp-sampled-or-missing-layer-chain*)
also have ... $\leq$
wp-event verify-monad
 (*trace-fri-bad-with-verifier-tied-sampled-layer-chain s*) $s +$
wp-event verify-monad
 (*trace-fri-verifier-tied-missing-sampled-layer-chain s*) s
by (*rule wp-event-union-bound*)
also have ... $\leq R + M$
by (*rule add-mono[OF sampled-bound missing-bound]*)
finally show ?thesis .
qed

lemma *wp-composition-fri-verifier-tied-bound-from-sampled-and-missing:*
assumes *sampled-bound:*
 wp-event verify-monad
 (*composition-fri-bad-with-verifier-tied-sampled-layer-chain s*) $s \leq R$
and *missing-bound:*
 wp-event verify-monad
 (*composition-fri-verifier-tied-missing-sampled-layer-chain s*) $s \leq M$
shows
 wp-event verify-monad
 (*composition-fri-bad-with-verifier-tied-partial-candidate s*) $s \leq$
 $R + M$
proof –
have *wp-event verify-monad*
 (*composition-fri-bad-with-verifier-tied-partial-candidate s*) $s \leq$
wp-event verify-monad
 ($\lambda out. \text{composition-fri-bad-with-verifier-tied-sampled-layer-chain } s \text{ out} \vee$
 $\text{composition-fri-verifier-tied-missing-sampled-layer-chain } s \text{ out}$) s
by (*rule wp-event-mono*)
 (*rule composition-fri-verifier-tied-imp-sampled-or-missing-layer-chain*)
also have ... $\leq$
wp-event verify-monad
 (*composition-fri-bad-with-verifier-tied-sampled-layer-chain s*) $s +$
wp-event verify-monad
 (*composition-fri-verifier-tied-missing-sampled-layer-chain s*) s
by (*rule wp-event-union-bound*)
also have ... $\leq R + M$
by (*rule add-mono[OF sampled-bound missing-bound]*)
finally show ?thesis .
qed

definition *trace-fri-verifier-tied-sampled-layer-assignment-obstruction*

$:: ('f, 'a) \text{ protocol-channel-scheme} \Rightarrow$
 $(\text{unit list} \times ('f, 'a) \text{ protocol-channel-scheme}) \text{ option} \Rightarrow \text{bool}$

where

trace-fri-verifier-tied-sampled-layer-assignment-obstruction s out
 $\longleftrightarrow$
 $(\exists \text{ trace-roots trace-bs trace-final dg composition-roots}$
 $\text{ composition-bs composition-final fri-query-idxs trace-round-layers}$
 $\text{ composition-round-layers fr trace-openings trace-table.}$
 $\text{ accepted-fri-opening-transcript s out trace-roots trace-bs}$
 $\text{ trace-final dg composition-roots composition-bs composition-final}$
 $\text{ fri-query-idxs trace-round-layers composition-round-layers} \wedge$
 $\text{ accepted-with-partial-trace-openings s out fr fri-query-idxs}$
 $\text{ trace-openings} \wedge$
 $\text{ partial-trace-table-candidate trace-table trace-openings} \wedge$
 $\neg \text{ trace-table-low-degree trace-table} \wedge$
 $\text{ generic-fri-sampled-layer-assignment-obstruction}$
 $\text{ trace-table-low-degree trace-table (clength - 1) trace-roots}$
 $\text{ trace-bs trace-final fri-query-idxs trace-round-layers})$

definition *trace-fri-verifier-tied-sampled-assignment-conflict*

$:: ('f, 'a) \text{ protocol-channel-scheme} \Rightarrow$
 $(\text{unit list} \times ('f, 'a) \text{ protocol-channel-scheme}) \text{ option} \Rightarrow \text{bool}$

where

trace-fri-verifier-tied-sampled-assignment-conflict s out
 $\longleftrightarrow$
 $(\exists \text{ trace-roots trace-bs trace-final dg composition-roots}$
 $\text{ composition-bs composition-final fri-query-idxs trace-round-layers}$
 $\text{ composition-round-layers fr trace-openings trace-table.}$
 $\text{ accepted-fri-opening-transcript s out trace-roots trace-bs}$
 $\text{ trace-final dg composition-roots composition-bs composition-final}$
 $\text{ fri-query-idxs trace-round-layers composition-round-layers} \wedge$
 $\text{ accepted-with-partial-trace-openings s out fr fri-query-idxs}$
 $\text{ trace-openings} \wedge$
 $\text{ partial-trace-table-candidate trace-table trace-openings} \wedge$
 $\neg \text{ trace-table-low-degree trace-table} \wedge$
 $\text{ generic-fri-sampled-assignment-conflict trace-table trace-roots}$
 $\text{ trace-bs trace-final fri-query-idxs trace-round-layers})$

definition *trace-fri-verifier-tied-zero-round-final-obstruction*

$:: ('f, 'a) \text{ protocol-channel-scheme} \Rightarrow$
 $(\text{unit list} \times ('f, 'a) \text{ protocol-channel-scheme}) \text{ option} \Rightarrow \text{bool}$

where

trace-fri-verifier-tied-zero-round-final-obstruction s out
 $\longleftrightarrow$
 $(\exists \text{ trace-roots trace-bs trace-final dg composition-roots}$
 $\text{ composition-bs composition-final fri-query-idxs trace-round-layers}$
 $\text{ composition-round-layers fr trace-openings trace-table.}$
 $\text{ accepted-fri-opening-transcript s out trace-roots trace-bs}$

$\text{trace-final dg composition-roots composition-bs composition-final}$
 $\text{fri-query-idxs trace-round-layers composition-round-layers} \wedge$
 $\text{accepted-with-partial-trace-openings } s \text{ out fr fri-query-idxs}$
 $\text{trace-openings} \wedge$
 $\text{partial-trace-table-candidate trace-table trace-openings} \wedge$
 $\neg \text{trace-table-low-degree trace-table} \wedge$
 $\text{length trace-bs} = 0 \wedge$
 $\neg \text{fri-final-constant-consistent trace-table trace-final})$

definition *composition-fri-verifier-tied-sampled-layer-assignment-obstruction*

$:: ('f, 'a) \text{ protocol-channel-scheme} \Rightarrow$
 $(\text{unit list} \times ('f, 'a) \text{ protocol-channel-scheme}) \text{ option} \Rightarrow \text{bool}$

where

$\text{composition-fri-verifier-tied-sampled-layer-assignment-obstruction } s \text{ out}$
 $\longleftrightarrow$
 $(\exists \text{ trace-roots trace-bs trace-final fri-dg composition-roots}$
 $\text{composition-bs composition-final fri-query-idxs trace-round-layers}$
 $\text{composition-round-layers fr as trace-openings composition-openings}$
 $\text{trace-table composition-table.}$
 $\text{accepted-fri-opening-transcript } s \text{ out trace-roots trace-bs}$
 $\text{trace-final fri-dg composition-roots composition-bs}$
 $\text{composition-final fri-query-idxs trace-round-layers}$
 $\text{composition-round-layers} \wedge$
 $\text{accepted-with-partial-initial-openings-aligned } s \text{ out fr trace-roots}$
 $\text{trace-final as fri-dg composition-roots composition-final fri-query-idxs}$
 $\text{trace-openings composition-openings} \wedge$
 $\text{partial-trace-table-candidate trace-table trace-openings} \wedge$
 $\text{partial-composition-table-candidate composition-table}$
 $\text{composition-openings} \wedge$
 $\text{trace-table-low-degree trace-table} \wedge$
 $\neg \text{composition-table-low-degree maxDegree composition-table} \wedge$
 $\text{generic-fri-sampled-layer-assignment-obstruction}$
 $(\text{composition-table-low-degree (to-nat fri-dg)}) \text{ composition-table}$
 $(\text{to-nat fri-dg}) \text{ composition-roots composition-bs}$
 $\text{composition-final fri-query-idxs composition-round-layers})$

definition *composition-fri-verifier-tied-sampled-assignment-conflict*

$:: ('f, 'a) \text{ protocol-channel-scheme} \Rightarrow$
 $(\text{unit list} \times ('f, 'a) \text{ protocol-channel-scheme}) \text{ option} \Rightarrow \text{bool}$

where

$\text{composition-fri-verifier-tied-sampled-assignment-conflict } s \text{ out}$
 $\longleftrightarrow$
 $(\exists \text{ trace-roots trace-bs trace-final fri-dg composition-roots}$
 $\text{composition-bs composition-final fri-query-idxs trace-round-layers}$
 $\text{composition-round-layers fr as trace-openings composition-openings}$
 $\text{trace-table composition-table.}$
 $\text{accepted-fri-opening-transcript } s \text{ out trace-roots trace-bs}$
 $\text{trace-final fri-dg composition-roots composition-bs}$
 $\text{composition-final fri-query-idxs trace-round-layers}$

composition-round-layers $\wedge$
accepted-with-partial-initial-openings-aligned *s out fr trace-roots*
trace-final as fri-dg composition-roots composition-final fri-query-idxs
trace-openings composition-openings $\wedge$
partial-trace-table-candidate trace-table trace-openings $\wedge$
partial-composition-table-candidate composition-table
composition-openings $\wedge$
trace-table-low-degree trace-table $\wedge$
 $\neg$ *composition-table-low-degree maxDegree composition-table* $\wedge$
generic-fri-sampled-assignment-conflict composition-table
composition-roots composition-bs composition-final fri-query-idxs
composition-round-layers)

definition *composition-fri-verifier-tied-zero-round-final-obstruction*

$:: ('f, 'a) \text{ protocol-channel-scheme} \Rightarrow$
 $(\text{unit list} \times ('f, 'a) \text{ protocol-channel-scheme}) \text{ option} \Rightarrow \text{bool}$

where

composition-fri-verifier-tied-zero-round-final-obstruction s out
 $\longleftrightarrow$
 $(\exists \text{ trace-roots trace-bs trace-final fri-dg composition-roots}$
 $\text{ composition-bs composition-final fri-query-idxs trace-round-layers}$
 $\text{ composition-round-layers fr as trace-openings composition-openings}$
 $\text{ trace-table composition-table.}$
 $\text{ accepted-fri-opening-transcript s out trace-roots trace-bs}$
 $\text{ trace-final fri-dg composition-roots composition-bs}$
 $\text{ composition-final fri-query-idxs trace-round-layers}$
 $\text{ composition-round-layers} \wedge$
 $\text{ accepted-with-partial-initial-openings-aligned s out fr trace-roots}$
 $\text{ trace-final as fri-dg composition-roots composition-final fri-query-idxs}$
 $\text{ trace-openings composition-openings} \wedge$
 $\text{ partial-trace-table-candidate trace-table trace-openings} \wedge$
 $\text{ partial-composition-table-candidate composition-table}$
 $\text{ composition-openings} \wedge$
 $\text{ trace-table-low-degree trace-table} \wedge$
 $\neg \text{ composition-table-low-degree maxDegree composition-table} \wedge$
 $\text{ length composition-bs} = 0 \wedge$
 $\neg \text{ fri-final-constant-consistent composition-table}$
 $\text{ composition-final})$

lemma *trace-fri-header-tied-zero-round-final-obstruction-imp-verifier-tied:*

assumes *trace-fri-header-tied-zero-round-final-obstruction s out*

shows *trace-fri-verifier-tied-zero-round-final-obstruction s out*

using *assms*

unfolding *trace-fri-header-tied-zero-round-final-obstruction-def*

trace-fri-verifier-tied-zero-round-final-obstruction-def

by *blast*

lemma *composition-fri-verifier-tied-zero-round-final-obstruction-false:*

$\neg \text{ composition-fri-verifier-tied-zero-round-final-obstruction s out}$

proof

assume *obstruction*:

composition-fri-verifier-tied-zero-round-final-obstruction s out

then obtain *trace-roots trace-bs trace-final fri-dg composition-roots*
composition-bs composition-final fri-query-idxs trace-round-layers
composition-round-layers fr as trace-openings composition-openings
trace-table composition-table

where *fri-openings*:

accepted-fri-opening-transcript s out trace-roots trace-bs
trace-final fri-dg composition-roots composition-bs
composition-final fri-query-idxs trace-round-layers
composition-round-layers

and *aligned*:

accepted-with-partial-initial-openings-aligned s out fr trace-roots
trace-final as fri-dg composition-roots composition-final fri-query-idxs
trace-openings composition-openings

and *zero*: *length composition-bs = 0*

unfolding *composition-fri-verifier-tied-zero-round-final-obstruction-def*
by *blast*

have *nonempty*: *composition-roots* $\neq []$

by (*rule accepted-with-partial-initial-openings-aligned-shapes(1)*
[OF aligned])

have *composition-roots* = []

using *accepted-fri-opening-transcript-shapes(4)[OF fri-openings]* *zero*
by *simp*

then show *False*

using *nonempty* **by** *simp*

qed

lemma *wp-composition-fri-verifier-tied-zero-round-final-obstruction-zero*:

wp-event verify-monad

(*composition-fri-verifier-tied-zero-round-final-obstruction s*) *s = 0*

unfolding *wp-event-def wp-def dist-expect-def*

by (*simp add*:

composition-fri-verifier-tied-zero-round-final-obstruction-false)

lemma *trace-fri-verifier-tied-assignment-obstruction-imp-conflict-or-zero*:

assumes *obstruction*:

trace-fri-verifier-tied-sampled-layer-assignment-obstruction s out

shows

trace-fri-verifier-tied-sampled-assignment-conflict s out $\vee$

trace-fri-verifier-tied-zero-round-final-obstruction s out

proof –

obtain *trace-roots trace-bs' trace-final dg composition-roots*

composition-bs composition-final fri-query-idxs trace-round-layers

composition-round-layers fr trace-openings trace-table

where *fri-openings*:

accepted-fri-opening-transcript s out trace-roots trace-bs'

trace-final dg composition-roots composition-bs composition-final

```

    fri-query-idxs trace-round-layers composition-round-layers
and partial:
    accepted-with-partial-trace-openings s out fr fri-query-idxs
    trace-openings
and cand: partial-trace-table-candidate trace-table trace-openings
and not-low:  $\neg$  trace-table-low-degree trace-table
and generic:
    generic-fri-sampled-layer-assignment-obstruction
    trace-table-low-degree trace-table (clength - 1) trace-roots
    trace-bs' trace-final fri-query-idxs trace-round-layers
using obstruction
unfolding trace-fri-verifier-tied-sampled-layer-assignment-obstruction-def
by blast
obtain N where eval-power: clength * scale =  $2^N$ 
    using eval-domain-length-power by blast
have trace-rounds-le: length trace-bs'  $\leq$  N
proof -
    have len: length trace-bs' = ceil-log clength
        using accepted-fri-opening-transcript-shapes(1,2)[OF fri-openings]
        by simp
    have clength  $\leq$  clength * scale
        using scale-pos by simp
    also have ... =  $2^N$ 
        by (rule eval-power)
    finally have ceil-log clength  $\leq$  N
        by (rule ceil-log-le-power)
    then show ?thesis
        using len by simp
qed
have query-bounds':
     $\bigwedge \text{round-idx. round-idx} < \text{length fri-query-idxs} \implies$ 
    fri-query-idxs ! round-idx < clength * scale
proof -
    fix round-idx
    assume round-bound: round-idx < length fri-query-idxs
    have fri-query-idxs ! round-idx  $\in$  set fri-query-idxs
        using round-bound by simp
    then show fri-query-idxs ! round-idx < clength * scale
        by (rule accepted-fri-opening-transcript-query-idx-bound
            [OF fri-openings])
qed
have split:
    generic-fri-sampled-assignment-conflict trace-table trace-roots
    trace-bs' trace-final fri-query-idxs trace-round-layers  $\vee$ 
    length trace-bs' = 0  $\wedge$ 
     $\neg$  fri-final-constant-consistent trace-table trace-final
by (rule
    generic-fri-sampled-layer-assignment-obstruction-imp-conflict-or-zero-round-final
    [OF generic query-bounds' eval-power trace-rounds-le])

```

```

then show ?thesis
proof
  assume conflict:
    generic-fri-sampled-assignment-conflict trace-table trace-roots
    trace-bs' trace-final fri-query-idxs trace-round-layers
  then show ?thesis
    unfolding trace-fri-verifier-tied-sampled-assignment-conflict-def
    by (intro disjI1 exI conjI)
      (rule fri-openings, rule partial, rule cand, rule not-low,
       rule conflict)
  next
    assume zero:
      length trace-bs' = 0  $\wedge$ 
       $\neg$  fri-final-constant-consistent trace-table trace-final
    then show ?thesis
      unfolding trace-fri-verifier-tied-zero-round-final-obstruction-def
      by (intro disjI2 exI conjI)
        (rule fri-openings, rule partial, rule cand, rule not-low,
         simp-all)
  qed
qed

lemma composition-fri-verifier-tied-assignment-obstruction-imp-conflict-or-zero-on-support:
  assumes support: out  $\in$  set-dist (execute verify-monad s)
  and obstruction:
    composition-fri-verifier-tied-sampled-layer-assignment-obstruction
    s out
  shows
    composition-fri-verifier-tied-sampled-assignment-conflict s out  $\vee$ 
    composition-fri-verifier-tied-zero-round-final-obstruction s out
proof —
  obtain trace-roots trace-bs trace-final fri-dg composition-roots
    composition-bs composition-final fri-query-idxs trace-round-layers
    composition-round-layers fr as trace-openings composition-openings
    trace-table composition-table
  where fri-openings:
    accepted-fri-opening-transcript s out trace-roots trace-bs
    trace-final fri-dg composition-roots composition-bs
    composition-final fri-query-idxs trace-round-layers
    composition-round-layers
  and aligned:
    accepted-with-partial-initial-openings-aligned s out fr trace-roots
    trace-final as fri-dg composition-roots composition-final fri-query-idxs
    trace-openings composition-openings
  and trace-cand:
    partial-trace-table-candidate trace-table trace-openings
  and comp-cand:
    partial-composition-table-candidate composition-table
    composition-openings

```

```

and trace-low: trace-table-low-degree trace-table
and comp-not-low:
   $\neg$  composition-table-low-degree maxDegree composition-table
and generic:
  generic-fri-sampled-layer-assignment-obstruction
    (composition-table-low-degree (to-nat fri-dg)) composition-table
    (to-nat fri-dg) composition-roots composition-bs
    composition-final fri-query-idxs composition-round-layers
using obstruction
unfolding
  composition-fri-verifier-tied-sampled-layer-assignment-obstruction-def
by metis
obtain  $N$  where eval-power: clength * scale =  $2^N$ 
  using eval-domain-length-power by blast
obtain result final-state where out-eq: out = Some (result, final-state)
  using fri-openings unfolding accepted-fri-opening-transcript-def by blast
have outcome:
  Some (result, final-state)  $\in$  set-dist (execute verify-monad s)
  using support out-eq by simp
have challenges:
  accepted-fri-challenges s (Some (result, final-state))
  trace-bs fri-dg composition-bs
  using accepted-fri-opening-transcript-challenges[OF fri-openings]
  out-eq by simp
have degree-bound: to-nat fri-dg  $\leq$  maxDegree
  by (rule verify-monad-accepted-fri-challenges-degree-bound
    [OF outcome challenges])
have composition-rounds-le: length composition-bs  $\leq N$ 
proof –
  have len: length composition-bs = ceil-log (to-nat fri-dg + 1)
    using accepted-fri-opening-transcript-shapes(3,4)[OF fri-openings]
    by simp
  have to-nat fri-dg + 1  $\leq$  clength * scale
    using degree-bound maxDegree-less-eval-domain by linarith
  also have ... =  $2^N$ 
    by (rule eval-power)
  finally have ceil-log (to-nat fri-dg + 1)  $\leq N$ 
    by (rule ceil-log-le-power)
  then show ?thesis
    using len by simp
qed
have query-bounds:
   $\bigwedge \text{round-idx. round-idx} < \text{length fri-query-idxs} \implies$ 
  fri-query-idxs ! round-idx < clength * scale
proof –
  fix round-idx
  assume round-bound: round-idx < length fri-query-idxs
  have fri-query-idxs ! round-idx  $\in$  set fri-query-idxs
    using round-bound by simp

```

```

then show fri-query-idxs ! round-idx < clength * scale
  by (rule accepted-fri-opening-transcript-query-idx-bound
    [OF fri-openings])
qed
have split:
  generic-fri-sampled-assignment-conflict composition-table
    composition-roots composition-bs composition-final fri-query-idxs
    composition-round-layers  $\vee$ 
    length composition-bs = 0  $\wedge$ 
     $\neg$  fri-final-constant-consistent composition-table
      composition-final
  by (rule
    generic-fri-sampled-layer-assignment-obstruction-imp-conflict-or-zero-round-final
    [OF generic query-bounds eval-power composition-rounds-le])
then show ?thesis
proof
  assume conflict:
    generic-fri-sampled-assignment-conflict composition-table
      composition-roots composition-bs composition-final fri-query-idxs
      composition-round-layers
  then show ?thesis
    unfolding
      composition-fri-verifier-tied-sampled-assignment-conflict-def
    by (intro disjI1 exI conjI)
      (rule fri-openings, rule aligned, rule trace-cand, rule comp-cand,
        rule trace-low, rule comp-not-low, rule conflict)
  next
    assume zero:
      length composition-bs = 0  $\wedge$ 
       $\neg$  fri-final-constant-consistent composition-table
        composition-final
    then show ?thesis
      unfolding
        composition-fri-verifier-tied-zero-round-final-obstruction-def
      by (intro disjI2 exI conjI)
        (rule fri-openings, rule aligned, rule trace-cand, rule comp-cand,
          rule trace-low, rule comp-not-low, simp-all)
    qed
  qed

lemma trace-fri-verifier-tied-missing-sampled-imp-assignment-obstruction:
  assumes missing:
    trace-fri-verifier-tied-missing-sampled-layer-chain s out
  shows
    trace-fri-verifier-tied-sampled-layer-assignment-obstruction s out
proof –
  have bad:
    trace-fri-bad-with-verifier-tied-partial-candidate s out
  and not-sampled:

```

```

     $\neg$  trace-fri-bad-with-verifier-tied-sampled-layer-chain s out
  using missing
  unfolding trace-fri-verifier-tied-missing-sampled-layer-chain-def
  by blast+
  from trace-fri-bad-with-verifier-tied-partial-candidateE[OF bad]
  obtain trace-roots trace-bs trace-final dg composition-roots
    composition-bs composition-final fri-query-idxs trace-round-layers
    composition-round-layers fr trace-openings trace-table
  where fri-openings:
    accepted-fri-opening-transcript s out trace-roots trace-bs
      trace-final dg composition-roots composition-bs composition-final
      fri-query-idxs trace-round-layers composition-round-layers
  and partial:
    accepted-with-partial-trace-openings s out fr fri-query-idxs
      trace-openings
  and cand: partial-trace-table-candidate trace-table trace-openings
  and not-low:  $\neg$  trace-table-low-degree trace-table
  by blast
  have partial-generic:
    generic-fri-partial-evidence trace-table-low-degree trace-table
      (clength - 1) trace-roots trace-bs trace-final fri-query-idxs
      trace-round-layers
  by (rule accepted-fri-opening-transcript-trace-generic-partial-evidence
    [OF fri-openings])
  have no-chain:
     $\bigwedge$  doms layers.  $\neg$ 
      generic-fri-sampled-layer-chain-evidence trace-table-low-degree
        trace-table (clength - 1) trace-roots trace-bs trace-final
        fri-query-idxs trace-round-layers doms layers
  proof
    fix doms layers
    assume chain:
      generic-fri-sampled-layer-chain-evidence trace-table-low-degree
        trace-table (clength - 1) trace-roots trace-bs trace-final
        fri-query-idxs trace-round-layers doms layers
    have trace-fri-bad-with-verifier-tied-sampled-layer-chain s out
      unfolding trace-fri-bad-with-verifier-tied-sampled-layer-chain-def
      by (intro exI conjI)
        (rule fri-openings, rule partial, rule cand, rule not-low,
          rule chain)
    then show False
      using not-sampled by contradiction
  qed
  have obstruction:
    generic-fri-sampled-layer-assignment-obstruction
      trace-table-low-degree trace-table (clength - 1) trace-roots
      trace-bs trace-final fri-query-idxs trace-round-layers
  unfolding generic-fri-sampled-layer-assignment-obstruction-def
  using partial-generic no-chain by blast

```


show *?thesis*
unfolding *trace-fri-verifier-tied-sampled-layer-assignment-obstruction-def*
by (*intro exI conjI*)
 (*rule fri-openings, rule partial, rule cand, rule not-low,*
 rule obstruction)

qed

lemma *composition-fri-verifier-tied-missing-sampled-imp-assignment-obstruction:*

assumes *missing:*

composition-fri-verifier-tied-missing-sampled-layer-chain s out

shows

composition-fri-verifier-tied-sampled-layer-assignment-obstruction s out

proof –

have *bad:*

composition-fri-bad-with-verifier-tied-partial-candidate s out

and *not-sampled:*

$\neg$ *composition-fri-bad-with-verifier-tied-sampled-layer-chain s out*

using *missing*

unfolding *composition-fri-verifier-tied-missing-sampled-layer-chain-def*

by *blast+*

from *composition-fri-bad-with-verifier-tied-partial-candidateE[OF bad]*

obtain *trace-roots trace-bs trace-final fri-dg composition-roots*

composition-bs composition-final fri-query-idxs trace-round-layers

composition-round-layers fr as trace-openings composition-openings

trace-table composition-table

where *fri-openings:*

accepted-fri-opening-transcript s out trace-roots trace-bs

trace-final fri-dg composition-roots composition-bs

composition-final fri-query-idxs trace-round-layers

composition-round-layers

and *aligned:*

accepted-with-partial-initial-openings-aligned s out fr trace-roots

trace-final as fri-dg composition-roots composition-final fri-query-idxs

trace-openings composition-openings

and *trace-cand:*

partial-trace-table-candidate trace-table trace-openings

and *comp-cand:*

partial-composition-table-candidate composition-table

composition-openings

and *trace-low: trace-table-low-degree trace-table*

and *comp-not-low:*

$\neg$ *composition-table-low-degree maxDegree composition-table*

by *metis*

have *partial-generic:*

generic-fri-partial-evidence

 (*composition-table-low-degree (to-nat fri-dg)*) *composition-table*

 (*to-nat fri-dg*) *composition-roots composition-bs composition-final*

fri-query-idxs composition-round-layers

by (*rule*

```

    accepted-fri-opening-transcript-composition-generic-partial-evidence
    [OF fri-openings])
have no-chain:
   $\bigwedge \text{doms layers. } \neg$ 
    generic-fri-sampled-layer-chain-evidence
    (composition-table-low-degree (to-nat fri-dg)) composition-table
    (to-nat fri-dg) composition-roots composition-bs
    composition-final fri-query-idxs composition-round-layers doms
    layers
proof
  fix doms layers
  assume chain:
    generic-fri-sampled-layer-chain-evidence
    (composition-table-low-degree (to-nat fri-dg)) composition-table
    (to-nat fri-dg) composition-roots composition-bs
    composition-final fri-query-idxs composition-round-layers doms
    layers
  have composition-fri-bad-with-verifier-tied-sampled-layer-chain s out
  unfolding composition-fri-bad-with-verifier-tied-sampled-layer-chain-def
  by (intro exI conjI)
    (rule fri-openings, rule aligned, rule trace-cand, rule comp-cand,
     rule trace-low, rule comp-not-low, rule chain)
  then show False
  using not-sampled by contradiction
qed
have obstruction:
  generic-fri-sampled-layer-assignment-obstruction
  (composition-table-low-degree (to-nat fri-dg)) composition-table
  (to-nat fri-dg) composition-roots composition-bs composition-final
  fri-query-idxs composition-round-layers
  unfolding generic-fri-sampled-layer-assignment-obstruction-def
  using partial-generic no-chain by blast
show ?thesis
  unfolding
    composition-fri-verifier-tied-sampled-layer-assignment-obstruction-def
  by (intro exI conjI)
    (rule fri-openings, rule aligned, rule trace-cand, rule comp-cand,
     rule trace-low, rule comp-not-low, rule obstruction)
qed

lemma wp-trace-fri-verifier-tied-missing-sampled-bound-from-assignment-obstruction:
  assumes obstruction-bound:
    wp-event verify-monad
    (trace-fri-verifier-tied-sampled-layer-assignment-obstruction s) s
     $\leq M$ 
  shows
    wp-event verify-monad
    (trace-fri-verifier-tied-missing-sampled-layer-chain s) s  $\leq M$ 
proof —

```

have *wp-event verify-monad*
 (*trace-fri-verifier-tied-missing-sampled-layer-chain s*) $s \leq$
wp-event verify-monad
 (*trace-fri-verifier-tied-sampled-layer-assignment-obstruction s*) s
by (*rule wp-event-mono*)
 (*rule trace-fri-verifier-tied-missing-sampled-imp-assignment-obstruction*)
also have $\dots \leq M$
by (*rule obstruction-bound*)
finally show ?thesis .
qed

lemma *wp-composition-fri-verifier-tied-missing-sampled-bound-from-assignment-obstruction:*
assumes *obstruction-bound:*
wp-event verify-monad
 (*composition-fri-verifier-tied-sampled-layer-assignment-obstruction s*) s
 $\leq M$
shows
wp-event verify-monad
 (*composition-fri-verifier-tied-missing-sampled-layer-chain s*) $s \leq M$
proof –
have *wp-event verify-monad*
 (*composition-fri-verifier-tied-missing-sampled-layer-chain s*) $s \leq$
wp-event verify-monad
 (*composition-fri-verifier-tied-sampled-layer-assignment-obstruction s*) s
by (*rule wp-event-mono*)
 (*rule*
composition-fri-verifier-tied-missing-sampled-imp-assignment-obstruction)
also have $\dots \leq M$
by (*rule obstruction-bound*)
finally show ?thesis .
qed

lemma *wp-trace-fri-verifier-tied-bound-from-sampled-and-assignment-obstruction:*
assumes *sampled-bound:*
wp-event verify-monad
 (*trace-fri-bad-with-verifier-tied-sampled-layer-chain s*) $s \leq R$
and *obstruction-bound:*
wp-event verify-monad
 (*trace-fri-verifier-tied-sampled-layer-assignment-obstruction s*) s
 $\leq M$
shows
wp-event verify-monad
 (*trace-fri-bad-with-verifier-tied-partial-candidate s*) $s \leq R + M$
by (*rule wp-trace-fri-verifier-tied-bound-from-sampled-and-missing*
 [*OF sampled-bound*
wp-trace-fri-verifier-tied-missing-sampled-bound-from-assignment-obstruction
 [*OF obstruction-bound*]])

lemma *wp-composition-fri-verifier-tied-bound-from-sampled-and-assignment-obstruction:*

assumes *sampld-bound*:
wp-event verify-monad
(composition-fri-bad-with-verifier-tied-sampled-layer-chain s) s ≤ R
and *obstruction-bound*:
wp-event verify-monad
(composition-fri-verifier-tied-sampled-layer-assignment-obstruction s) s ≤ M
shows
wp-event verify-monad
(composition-fri-bad-with-verifier-tied-partial-candidate s) s ≤ R + M
by (*rule wp-composition-fri-verifier-tied-bound-from-sampled-and-missing*
[OF sampld-bound
wp-composition-fri-verifier-tied-missing-sampled-bound-from-assignment-obstruction
[OF obstruction-bound]])

lemma *trace-fri-bad-with-verifier-tied-partial-candidates-staged-bound*:

assumes *verifier-bound*:
 $\bigwedge \text{data attacker-state.}$
Some (data, attacker-state) ∈
set-dist (execute (checked-staged-transcript-program A)
adversary-initial-state) ⇒
wp-event verify-monad
(trace-fri-bad-with-verifier-tied-partial-candidate
(verifier-state-from-adversary attacker-state
(staged-proof-transcript data)))
(verifier-state-from-adversary attacker-state
(staged-proof-transcript data)) ≤ C
shows
wp-event (checked-staged-security-experiment-with-data-state A)
(staged-security-with-data-state-verifier-event
trace-fri-bad-with-verifier-tied-partial-candidate)
adversary-initial-state ≤ C
by (*rule checked-staged-security-with-data-state-verifier-event-bound-from-cont*)
(simp-all add: verifier-bound)

lemma *composition-fri-bad-with-verifier-tied-partial-candidates-staged-bound*:

assumes *verifier-bound*:
 $\bigwedge \text{data attacker-state.}$
Some (data, attacker-state) ∈
set-dist (execute (checked-staged-transcript-program A)
adversary-initial-state) ⇒
wp-event verify-monad
(composition-fri-bad-with-verifier-tied-partial-candidate
(verifier-state-from-adversary attacker-state
(staged-proof-transcript data)))
(verifier-state-from-adversary attacker-state
(staged-proof-transcript data)) ≤ C
shows

$wp\text{-}event$ (*checked-staged-security-experiment-with-data-state* A)
 (*staged-security-with-data-state-verifier-event*
composition-fri-bad-with-verifier-tied-partial-candidate)
adversary-initial-state $\leq C$
by (*rule checked-staged-security-with-data-state-verifier-event-bound-from-cont*)
 (*simp-all add: verifier-bound*)

lemma *wp-trace-fri-verifier-tied-assignment-obstruction-bound-from-conflict-and-zero:*

assumes *conflict-bound:*

$wp\text{-}event$ *verify-monad*
 (*trace-fri-verifier-tied-sampled-assignment-conflict* s) $s \leq C$

and *zero-bound:*

$wp\text{-}event$ *verify-monad*
 (*trace-fri-verifier-tied-zero-round-final-obstruction* s) $s \leq Z$

shows

$wp\text{-}event$ *verify-monad*
 (*trace-fri-verifier-tied-sampled-layer-assignment-obstruction* s) s
 $\leq C + Z$

proof –

have $wp\text{-}event$ *verify-monad*

(*trace-fri-verifier-tied-sampled-layer-assignment-obstruction* s) s
 $\leq wp\text{-}event$ *verify-monad*
 ($\lambda out. trace\text{-}fri\text{-}verifier\text{-}tied\text{-}sampled\text{-}assignment\text{-}conflict\ s\ out \vee$
trace-fri-verifier-tied-zero-round-final-obstruction $s\ out$) s

by (*rule wp-event-mono*)

(*rule trace-fri-verifier-tied-assignment-obstruction-imp-conflict-or-zero*)

also have $\dots \leq$

$wp\text{-}event$ *verify-monad*
 (*trace-fri-verifier-tied-sampled-assignment-conflict* s) s +
 $wp\text{-}event$ *verify-monad*
 (*trace-fri-verifier-tied-zero-round-final-obstruction* s) s

by (*rule wp-event-union-bound*)

also have $\dots \leq C + Z$

by (*rule add-mono[OF conflict-bound zero-bound]*)

finally show *?thesis* .

qed

lemma *wp-composition-fri-verifier-tied-assignment-obstruction-bound-from-conflict-and-zero:*

assumes *conflict-bound:*

$wp\text{-}event$ *verify-monad*
 (*composition-fri-verifier-tied-sampled-assignment-conflict* s) $s \leq C$

and *zero-bound:*

$wp\text{-}event$ *verify-monad*
 (*composition-fri-verifier-tied-zero-round-final-obstruction* s) $s \leq Z$

shows

$wp\text{-}event$ *verify-monad*
 (*composition-fri-verifier-tied-sampled-layer-assignment-obstruction* s) s
 $\leq C + Z$

proof –

have *wp-event verify-monad*
 (*composition-fri-verifier-tied-sampled-layer-assignment-obstruction s*) *s*
 $\leq$ *wp-event verify-monad*
 ($\lambda out. \text{composition-fri-verifier-tied-sampled-assignment-conflict } s \text{ out} \vee$
 $\text{composition-fri-verifier-tied-zero-round-final-obstruction } s \text{ out}$) *s*
by (rule *wp-event-mono-on-support*)
 (rule
 $\text{composition-fri-verifier-tied-assignment-obstruction-imp-conflict-or-zero-on-support}$)
also have ... $\leq$
wp-event verify-monad
 (*composition-fri-verifier-tied-sampled-assignment-conflict s*) *s* +
wp-event verify-monad
 (*composition-fri-verifier-tied-zero-round-final-obstruction s*) *s*
by (rule *wp-event-union-bound*)
also have ... $\leq C + Z$
by (rule *add-mono[OF conflict-bound zero-bound]*)
finally show ?thesis .
qed

lemma *wp-composition-fri-verifier-tied-assignment-obstruction-bound-from-conflict:*

assumes *conflict-bound:*

wp-event verify-monad

(*composition-fri-verifier-tied-sampled-assignment-conflict s*) *s* $\leq C$

shows

wp-event verify-monad

(*composition-fri-verifier-tied-sampled-layer-assignment-obstruction s*) *s*
 $\leq C$

proof –

have *zero-bound:*

wp-event verify-monad

(*composition-fri-verifier-tied-zero-round-final-obstruction s*) *s* ≤ 0

by (subst *wp-composition-fri-verifier-tied-zero-round-final-obstruction-zero*)
simp

have *wp-event verify-monad*

(*composition-fri-verifier-tied-sampled-layer-assignment-obstruction s*) *s*
 $\leq C + 0$

by (rule
 $\text{wp-composition-fri-verifier-tied-assignment-obstruction-bound-from-conflict-and-zero}$
 $[\text{OF conflict-bound zero-bound}]$)

then show ?thesis

by *simp*

qed

lemma *wp-trace-fri-verifier-tied-bound-from-sampled-conflict-and-zero:*

assumes *sampled-bound:*

wp-event verify-monad

(*trace-fri-bad-with-verifier-tied-sampled-layer-chain s*) *s* $\leq R$

and *conflict-bound:*

wp-event verify-monad

(trace-fri-verifier-tied-sampled-assignment-conflict s) $s \leq C$
and zero-bound:
 wp-event verify-monad
 (trace-fri-verifier-tied-zero-round-final-obstruction s) $s \leq Z$
shows
 wp-event verify-monad
 (trace-fri-bad-with-verifier-tied-partial-candidate s) s
 $\leq R + (C + Z)$
by (rule
 wp-trace-fri-verifier-tied-bound-from-sampled-and-assignment-obstruction
 [OF sampled-bound
 wp-trace-fri-verifier-tied-assignment-obstruction-bound-from-conflict-and-zero
 [OF conflict-bound zero-bound]])

lemma wp-composition-fri-verifier-tied-bound-from-sampled-conflict-and-zero:

assumes sampled-bound:
 wp-event verify-monad
 (composition-fri-bad-with-verifier-tied-sampled-layer-chain s) $s \leq R$
and conflict-bound:
 wp-event verify-monad
 (composition-fri-verifier-tied-sampled-assignment-conflict s) $s \leq C$
and zero-bound:
 wp-event verify-monad
 (composition-fri-verifier-tied-zero-round-final-obstruction s) $s \leq Z$
shows
 wp-event verify-monad
 (composition-fri-bad-with-verifier-tied-partial-candidate s) s
 $\leq R + (C + Z)$
by (rule
 wp-composition-fri-verifier-tied-bound-from-sampled-and-assignment-obstruction
 [OF sampled-bound
 wp-composition-fri-verifier-tied-assignment-obstruction-bound-from-conflict-and-zero
 [OF conflict-bound zero-bound]])

lemma wp-composition-fri-verifier-tied-bound-from-sampled-conflict:

assumes sampled-bound:
 wp-event verify-monad
 (composition-fri-bad-with-verifier-tied-sampled-layer-chain s) $s \leq R$
and conflict-bound:
 wp-event verify-monad
 (composition-fri-verifier-tied-sampled-assignment-conflict s) $s \leq C$
shows
 wp-event verify-monad
 (composition-fri-bad-with-verifier-tied-partial-candidate s) s
 $\leq R + C$
by (rule
 wp-composition-fri-verifier-tied-bound-from-sampled-and-assignment-obstruction
 [OF sampled-bound
 wp-composition-fri-verifier-tied-assignment-obstruction-bound-from-conflict

[*OF conflict-bound*]])

lemma *trace-fri-verifier-tied-reductionD*:

assumes *trace-fri-verifier-tied-reduction s*

shows

wp-event verify-monad

(*trace-fri-bad-with-verifier-tied-partial-candidate s*) $s \leq$

trace-fri-error

using *assms* **unfolding** *trace-fri-verifier-tied-reduction-def* **by** *simp*

lemma *composition-fri-verifier-tied-reductionD*:

assumes *composition-fri-verifier-tied-reduction s*

shows

wp-event verify-monad

(*composition-fri-bad-with-verifier-tied-partial-candidate s*) $s \leq$

composition-fri-error

using *assms* **unfolding** *composition-fri-verifier-tied-reduction-def* **by** *simp*

lemma *trace-fri-verifier-tied-reduction-from-sampled-conflict-and-zero*:

assumes *sampled-bound*:

wp-event verify-monad

(*trace-fri-bad-with-verifier-tied-sampled-layer-chain s*) $s \leq R$

and *conflict-bound*:

wp-event verify-monad

(*trace-fri-verifier-tied-sampled-assignment-conflict s*) $s \leq C$

and *zero-bound*:

wp-event verify-monad

(*trace-fri-verifier-tied-zero-round-final-obstruction s*) $s \leq Z$

and *total-bound*: $R + (C + Z) \leq \text{trace-fri-error}$

shows *trace-fri-verifier-tied-reduction s*

unfolding *trace-fri-verifier-tied-reduction-def*

by (rule *order-trans*[

OF wp-trace-fri-verifier-tied-bound-from-sampled-conflict-and-zero

[*OF sampled-bound conflict-bound zero-bound*] *total-bound*])

lemma *composition-fri-verifier-tied-reduction-from-sampled-conflict-and-zero*:

assumes *sampled-bound*:

wp-event verify-monad

(*composition-fri-bad-with-verifier-tied-sampled-layer-chain s*) $s \leq R$

and *conflict-bound*:

wp-event verify-monad

(*composition-fri-verifier-tied-sampled-assignment-conflict s*) $s \leq C$

and *zero-bound*:

wp-event verify-monad

(*composition-fri-verifier-tied-zero-round-final-obstruction s*) $s \leq Z$

and *total-bound*: $R + (C + Z) \leq \text{composition-fri-error}$

shows *composition-fri-verifier-tied-reduction s*

unfolding *composition-fri-verifier-tied-reduction-def*

by (rule *order-trans*[

OF wp-composition-fri-verifier-tied-bound-from-sampled-conflict-and-zero
[OF sampled-bound conflict-bound zero-bound] total-bound])

lemma *composition-fri-verifier-tied-reduction-from-sampled-conflict:*

assumes *sampled-bound:*

wp-event verify-monad

(composition-fri-bad-with-verifier-tied-sampled-layer-chain s) s ≤ R

and *conflict-bound:*

wp-event verify-monad

(composition-fri-verifier-tied-sampled-assignment-conflict s) s ≤ C

and *total-bound: R + C ≤ composition-fri-error*

shows *composition-fri-verifier-tied-reduction s*

unfolding *composition-fri-verifier-tied-reduction-def*

by *(rule order-trans[*

OF wp-composition-fri-verifier-tied-bound-from-sampled-conflict

[OF sampled-bound conflict-bound] total-bound])

lemma *trace-fri-zero-round-checked-final-obstruction-imp-verifier-tied:*

assumes

trace-fri-zero-round-checked-final-obstruction s out

shows *trace-fri-bad-with-verifier-tied-partial-candidate s out*

proof *(rule trace-fri-zero-round-checked-final-obstruction-not-trace-low-degree*
[OF assms])

fix *trace-roots trace-bs trace-final dg composition-roots composition-bs*

composition-final fri-query-idxs trace-round-layers

composition-round-layers fr trace-openings trace-table

assume *fri-openings:*

accepted-fri-opening-transcript s out trace-roots trace-bs

trace-final dg composition-roots composition-bs composition-final

fri-query-idxs trace-round-layers composition-round-layers

and *checked:*

trace-zero-round-final-checked-candidate s out fr fri-query-idxs

trace-openings trace-table trace-final

and *not-low: ¬ trace-table-low-degree trace-table*

have *partial:*

accepted-with-partial-trace-openings s out fr fri-query-idxs

trace-openings

using *checked* **unfolding** *trace-zero-round-final-checked-candidate-def*

by *simp*

have *cand:*

partial-trace-table-candidate trace-table trace-openings

using *checked* **unfolding** *trace-zero-round-final-checked-candidate-def*

by *simp*

show *?thesis*

unfolding *trace-fri-bad-with-verifier-tied-partial-candidate-def*

by *(intro exI conjI)*

(rule fri-openings, rule partial, rule cand, rule not-low)

qed

lemma *trace-fri-zero-round-checked-final-obstruction-mono-verifier-tied*:
 $wp\text{-}event\ verify\text{-}monad\ (trace\text{-}fri\text{-}zero\text{-}round\text{-}checked\text{-}final\text{-}obstruction\ s)\ s \leq$
 $wp\text{-}event\ verify\text{-}monad$
 $(trace\text{-}fri\text{-}bad\text{-}with\text{-}verifier\text{-}tied\text{-}partial\text{-}candidate\ s)\ s$
by $(rule\ wp\text{-}event\text{-}mono)$
 $(rule\ trace\text{-}fri\text{-}zero\text{-}round\text{-}checked\text{-}final\text{-}obstruction\text{-}imp\text{-}verifier\text{-}tied)$

end

end

theory *Soundness-Relevant-Drift-Query-Agreement*
imports *Soundness-Relevant-Drift-Reachable-FRI*
begin

Fixed trace-pair query-agreement bounds for the empty-header relevant-drift residual. This layer keeps the candidate pair fixed before the query challenge is sampled. It therefore avoids the invalid broad fixed-cover route where partial openings selected after the query determine the target.

context *soundness*
begin

definition *staged-security-with-data-state-trace-pair-agreement-hit-at*
 $:: 'f\ list \Rightarrow 'f\ list \Rightarrow nat \Rightarrow$
 $((('f\ staged\text{-}proof\text{-}data \times 'f\ protocol\text{-}channel) \times unit\ list) \times$
 $'f\ protocol\text{-}channel)\ option \Rightarrow bool$

where

staged-security-with-data-state-trace-pair-agreement-hit-at
 $trace\text{-}table\ trace\text{-}table'\ i\ out \longleftrightarrow$
 $(case\ out\ of$
 $None \Rightarrow False$
 $| Some\ (((data, attacker\text{-}state), result), final\text{-}state) \Rightarrow$
 $(\exists\ as\ query\text{-}idxs.$
 $accepted\text{-}transcript\text{-}shape$
 $(verifier\text{-}state\text{-}from\text{-}adversary\ attacker\text{-}state$
 $(staged\text{-}proof\text{-}transcript\ data))$
 $(Some\ (result, final\text{-}state))\ as\ query\text{-}idxs \wedge$
 $i < rounds \wedge$
 $query\text{-}idxs\ !\ i \in$
 $trace\text{-}table\text{-}agreement\text{-}indices\ trace\text{-}table\ trace\text{-}table'))$

definition *checked-staged-security-with-actual-alpha-prefix-trace-pair-agreement-hit-at*
 $:: 'f\ list \Rightarrow 'f\ list \Rightarrow nat \Rightarrow$
 $(((((('f \times 'f\ list \times 'f\ list \times 'f) \times$
 $'f\ protocol\text{-}channel) \times 'f\ staged\text{-}proof\text{-}data) \times$
 $'f\ protocol\text{-}channel) \times unit\ list) \times$
 $'f\ protocol\text{-}channel)\ option \Rightarrow bool$

where

checked-staged-security-with-actual-alpha-prefix-trace-pair-agreement-hit-at
trace-table trace-table' i out $\longleftrightarrow$
 (case out of
 None $\Rightarrow$ False
 | Some (((alpha-prefix, alpha-prefix-state), data), attacker-state),
 result), final-state) $\Rightarrow$
 staged-security-with-data-state-trace-pair-agreement-hit-at
 trace-table trace-table' i
 (Some (((data, attacker-state), result), final-state)))

definition *checked-staged-security-with-query-prefix-trace-pair-agreement-hit-at*
 :: 'f list $\Rightarrow$ 'f list $\Rightarrow$ nat $\Rightarrow$
 ((((('f staged-query-prefix-data $\times$ 'f protocol-channel) $\times$
 'f) $\times$ 'f protocol-channel) $\times$
 (('f staged-proof-data $\times$ 'f protocol-channel) $\times$ unit list)) $\times$
 'f protocol-channel) option) $\Rightarrow$ bool

where

checked-staged-security-with-query-prefix-trace-pair-agreement-hit-at
trace-table trace-table' i out $\longleftrightarrow$
checked-staged-security-with-query-prefix-dynamic-index-hit
 (λ - -. *trace-table-agreement-indices trace-table trace-table'*) out

lemma *checked-staged-query-prefix-trace-pair-agreement-prehit-bound:*

assumes wf: *staged-budget-wellformed budgets*
and controlled: *staged-adversary-controlled budgets A*
and i-bound: $i < \text{rounds}$

shows

wp-event (*checked-staged-query-prefix-receive-with-state A i*)
 (*checked-staged-query-prefix-dynamic-index-prehit*
 (λ - -. *trace-table-agreement-indices trace-table trace-table'*))
adversary-initial-state $\leq$
staged-phase-relation-error size
 (*staged-query-search-queries budgets i + 1*)

proof (rule *checked-staged-query-prefix-dynamic-index-prehit-bound-from-relation-program*)

show *HashMap adversary-initial-state = fmempty*
by (*simp add: adversary-initial-state-def*)

show *hash-relation-program*

(*checked-staged-query-prefix-dynamic-index-prehit-relation A i*
 adversary-initial-state
 (λ - -. *trace-table-agreement-indices trace-table trace-table'*))
size (*staged-query-search-queries budgets i + 1*)
 (*checked-staged-query-prefix-receive-with-state A i*)
by (rule *hash-relation-program-checked-staged-query-prefix-receive-with-state*
 [OF wf controlled less-imp-le[OF i-bound]])
 (rule *checked-staged-query-prefix-dynamic-index-prehit-relation-fiber-bound-size*)

qed

lemma *checked-staged-query-prefix-trace-pair-agreement-hit-bound:*

assumes wf: *staged-budget-wellformed budgets*

and *controlled*: staged-adversary-controlled budgets A
and *i-bound*: $i < \text{rounds}$
and *trace-low*: trace-table-low-degree trace-table
and *trace'-low*: trace-table-low-degree trace-table'
and *distinct*: trace-table $\neq$ trace-table'
shows
wp-event (checked-staged-query-prefix-receive-with-state A i)
(checked-staged-query-prefix-dynamic-index-hit
(λ - . trace-table-agreement-indices trace-table trace-table'))
adversary-initial-state $\leq$
staged-phase-relation-error size
(staged-query-search-queries budgets $i + 1$) +
nnreal clength / nnreal (card query-sample-space)
proof –
have *hit*:
wp-event (checked-staged-query-prefix-receive-with-state A i)
(checked-staged-query-prefix-dynamic-index-hit
(λ - . trace-table-agreement-indices trace-table trace-table'))
adversary-initial-state $\leq$
wp-event (checked-staged-query-prefix-receive-with-state A i)
(checked-staged-query-prefix-dynamic-index-prehit
(λ - . trace-table-agreement-indices trace-table trace-table'))
adversary-initial-state +
nnreal clength / nnreal (card query-sample-space)
by (rule checked-staged-query-prefix-trace-pair-agreement-hit-bound-by-prehit
[OF trace-low trace'-low distinct])
have *pre*:
wp-event (checked-staged-query-prefix-receive-with-state A i)
(checked-staged-query-prefix-dynamic-index-prehit
(λ - . trace-table-agreement-indices trace-table trace-table'))
adversary-initial-state $\leq$
staged-phase-relation-error size
(staged-query-search-queries budgets $i + 1$)
by (rule checked-staged-query-prefix-trace-pair-agreement-prehit-bound
[OF wf controlled i-bound])
show ?thesis
by (rule order-trans[OF hit]) (intro add-mono pre order-refl)
qed

lemma checked-staged-security-with-query-prefix-trace-pair-agreement-hit-bound:
assumes *wf*: staged-budget-wellformed budgets
and *controlled*: staged-adversary-controlled budgets A
and *i-bound*: $i < \text{rounds}$
and *trace-low*: trace-table-low-degree trace-table
and *trace'-low*: trace-table-low-degree trace-table'
and *distinct*: trace-table $\neq$ trace-table'
shows
wp-event
(checked-staged-security-experiment-with-query-prefix-data-state A i)

(checked-staged-security-with-query-prefix-trace-pair-agreement-hit-at
 trace-table trace-table' i)
 adversary-initial-state $\leq$
 staged-phase-relation-error size
 (staged-query-search-queries budgets i + 1) +
 nnreal clength / nnreal (card query-sample-space)
unfolding checked-staged-security-with-query-prefix-trace-pair-agreement-hit-at-def
by (rule checked-staged-security-with-query-prefix-dynamic-index-hit-bound)
 (rule checked-staged-query-prefix-trace-pair-agreement-hit-bound
 [OF wf controlled i-bound trace-low trace'-low distinct])

lemma staged-security-with-data-state-trace-pair-agreement-hit-at-le-query-prefix:
assumes wf: staged-budget-wellformed budgets
and controlled: staged-adversary-controlled budgets A
and i-bound: i < rounds
shows
 wp-event (checked-staged-security-experiment-with-data-state A)
 (staged-security-with-data-state-trace-pair-agreement-hit-at
 trace-table trace-table' i)
 adversary-initial-state $\leq$
 wp-event
 (checked-staged-security-experiment-with-query-prefix-data-state A i)
 (checked-staged-security-with-query-prefix-trace-pair-agreement-hit-at
 trace-table trace-table' i)
 adversary-initial-state

proof –
let ?E =
 staged-security-with-data-state-trace-pair-agreement-hit-at
 trace-table trace-table' i
let ?Q =
 checked-staged-security-with-query-prefix-trace-pair-agreement-hit-at
 trace-table trace-table' i
have proj:
 wp-event (checked-staged-security-experiment-with-data-state A) ?E
 adversary-initial-state =
 wp-event
 (checked-staged-security-experiment-with-query-prefix-data-state A i)
 (λ out. case out of
 None $\Rightarrow$?E None
 | Some (packed, t) $\Rightarrow$?E (Some (snd packed, t)))
 adversary-initial-state
by (rule checked-staged-security-experiment-with-query-prefix-data-state-projection-event
 [OF i-bound])

show ?thesis
unfolding proj
proof (rule wp-event-mono-on-support)
fix out
assume support:
 out $\in$

```

    set-dist
      (execute
        (checked-staged-security-experiment-with-query-prefix-data-state
          A i)
        adversary-initial-state)
  and hit:
    (case out of None  $\Rightarrow$  ?E None
      | Some (packed, t)  $\Rightarrow$  ?E (Some (snd packed, t)))
show ?Q out
proof (cases out)
  case None
  then show ?thesis
    using hit
    unfolding staged-security-with-data-state-trace-pair-agreement-hit-at-def
      checked-staged-security-with-query-prefix-trace-pair-agreement-hit-at-def
      checked-staged-security-with-query-prefix-dynamic-index-hit-def
    by simp
next
  case (Some packed)
  then obtain prefix prefix-state raw raw-state data attacker-state
    result final-state where out-eq:
    out =
      Some (((((prefix, prefix-state), raw), raw-state),
        ((data, attacker-state), result)), final-state)
    by (cases packed, auto split: prod.splits)
  have support':
    Some (((((prefix, prefix-state), raw), raw-state),
      ((data, attacker-state), result)), final-state)
     $\in$ 
    set-dist
      (execute
        (checked-staged-security-experiment-with-query-prefix-data-state
          A i)
        adversary-initial-state)
    using support unfolding out-eq .
  have data-support:
    Some (((data, attacker-state), result), final-state)  $\in$ 
    set-dist
      (execute (checked-staged-security-experiment-with-data-state A)
        adversary-initial-state)
    by (rule
      checked-staged-security-experiment-with-query-prefix-data-state-imp-data-state-support
      [OF i-bound support'])
  have data-hit:
    staged-security-with-data-state-trace-pair-agreement-hit-at
      trace-table trace-table' i
      (Some (((data, attacker-state), result), final-state))
    using hit unfolding out-eq by simp
  obtain as query-idxs where shape:

```

```

    accepted-transcript-shape
      (verifier-state-from-adversary attacker-state
        (staged-proof-transcript data))
      (Some (result, final-state)) as query-idxs
  and idx-agree:
    query-idxs ! i ∈
      trace-table-agreement-indices trace-table trace-table'
  using data-hit unfolding
    staged-security-with-data-state-trace-pair-agreement-hit-at-def
  by auto
from checked-staged-security-with-data-state-accepted-shape-query-keys
  [OF wf controlled data-support shape]
obtain raw-idxs where len-raw: length raw-idxs = rounds
and query-idxs-eq:
  query-idxs = map (λraw. index (to-nat raw)) raw-idxs
and lookup-final:
  ∧j. j < rounds ⇒
    fmllookup (HashMap final-state)
      (QueryIndexChallenge j
        (state-after-query-chunks
          (staged-query-start-hash data)
          (staged-query-chunks data) j)) =
      Some (raw-idxs ! j)
  by blast
have final-lookup:
  fmllookup (HashMap final-state)
    (QueryIndexChallenge i
      (state-after-query-chunks
        (staged-query-start-hash data) (staged-query-chunks data) i)) =
    Some raw
  by (rule
    checked-staged-security-experiment-with-query-prefix-data-state-raw-final-lookup
    [OF wf controlled i-bound support'])
have raw-eq: raw-idxs ! i = raw
  using lookup-final[OF i-bound] final-lookup by simp
have raw-agree:
  index (to-nat raw) ∈
    trace-table-agreement-indices trace-table trace-table'
  using idx-agree query-idxs-eq len-raw i-bound raw-eq by simp
show ?thesis
  unfolding out-eq
    checked-staged-security-with-query-prefix-trace-pair-agreement-hit-at-def
    checked-staged-security-with-query-prefix-dynamic-index-hit-def
    checked-staged-query-prefix-dynamic-index-hit-def
  using raw-agree by simp
qed
qed
qed

```

lemma *staged-security-with-data-state-trace-pair-agreement-hit-at-bound:*
assumes *wf: staged-budget-wellformed budgets*
and *controlled: staged-adversary-controlled budgets A*
and *i-bound: i < rounds*
and *trace-low: trace-table-low-degree trace-table*
and *trace'-low: trace-table-low-degree trace-table'*
and *distinct: trace-table $\neq$ trace-table'*
shows
wp-event (checked-staged-security-experiment-with-data-state A)
(staged-security-with-data-state-trace-pair-agreement-hit-at
trace-table trace-table' i)
adversary-initial-state $\leq$
staged-phase-relation-error size
(staged-query-search-queries budgets i + 1) +
nnreal clength / nnreal (card query-sample-space)
by (*rule order-trans[*
OF staged-security-with-data-state-trace-pair-agreement-hit-at-le-query-prefix
checked-staged-security-with-query-prefix-trace-pair-agreement-hit-bound])
(use assms in simp-all)

lemma *checked-staged-security-with-actual-alpha-prefix-trace-pair-agreement-hit-at-projection:*
wp-event
(checked-staged-security-experiment-with-actual-alpha-prefix-data-state A)
(checked-staged-security-with-actual-alpha-prefix-trace-pair-agreement-hit-at
trace-table trace-table' i)
adversary-initial-state =
wp-event (checked-staged-security-experiment-with-data-state A)
(staged-security-with-data-state-trace-pair-agreement-hit-at
trace-table trace-table' i)
adversary-initial-state
proof –
let *?project =*
 $\lambda x. ((\text{snd } (\text{fst } (\text{fst } x)), \text{snd } (\text{fst } x)), \text{snd } x)$
have
wp-event (checked-staged-security-experiment-with-data-state A)
(staged-security-with-data-state-trace-pair-agreement-hit-at
trace-table trace-table' i)
adversary-initial-state =
wp-event
(checked-staged-security-experiment-with-actual-alpha-prefix-data-state A
 $\gg (\lambda x. \text{return } (?project\ x)))$
(staged-security-with-data-state-trace-pair-agreement-hit-at
trace-table trace-table' i)
adversary-initial-state
using
checked-staged-security-experiment-with-actual-alpha-prefix-data-state-projection
[of A]
by *simp*
also have ... =


```

wp-event
  (checked-staged-security-experiment-with-actual-alpha-prefix-data-state A)
  (checked-staged-security-with-actual-alpha-prefix-trace-pair-agreement-hit-at
    trace-table trace-table' i)
  adversary-initial-state
proof —
  have pred-eq:
    (λout. case out of
      None ⇒
        staged-security-with-data-state-trace-pair-agreement-hit-at
          trace-table trace-table' i None
      | Some (x, t) ⇒
        staged-security-with-data-state-trace-pair-agreement-hit-at
          trace-table trace-table' i (Some (?project x, t))) =
        checked-staged-security-with-actual-alpha-prefix-trace-pair-agreement-hit-at
          trace-table trace-table' i
    by (rule ext)
    (auto simp:
      staged-security-with-data-state-trace-pair-agreement-hit-at-def
      checked-staged-security-with-actual-alpha-prefix-trace-pair-agreement-hit-at-def
      split: option.splits prod.splits)
  show ?thesis
    by (subst wp-event-bind-return-map) (simp add: pred-eq)
qed
finally show ?thesis
  by simp
qed

lemma checked-staged-security-with-actual-alpha-prefix-trace-pair-agreement-hit-at-bound:
  assumes wf: staged-budget-wellformed budgets
  and controlled: staged-adversary-controlled budgets A
  and i-bound: i < rounds
  and trace-low: trace-table-low-degree trace-table
  and trace'-low: trace-table-low-degree trace-table'
  and distinct: trace-table ≠ trace-table'
  shows
    wp-event
      (checked-staged-security-experiment-with-actual-alpha-prefix-data-state A)
      (checked-staged-security-with-actual-alpha-prefix-trace-pair-agreement-hit-at
        trace-table trace-table' i)
      adversary-initial-state ≤
      staged-phase-relation-error size
        (staged-query-search-queries budgets i + 1) +
        nnreal clength / nnreal (card query-sample-space)
  unfolding
    checked-staged-security-with-actual-alpha-prefix-trace-pair-agreement-hit-at-projection
  by (rule staged-security-with-data-state-trace-pair-agreement-hit-at-bound
    [OF wf controlled i-bound trace-low trace'-low distinct])

```

definition *low-degree-trace-pair-space* :: ('f list × 'f list) set

where

low-degree-trace-pair-space =
 {(trace-table, trace-table').
 trace-table-low-degree trace-table ∧
 trace-table-low-degree trace-table' ∧
 trace-table ≠ trace-table'}

lemma *trace-table-low-degree-length*:

assumes *trace-table-low-degree trace-table*

shows *length trace-table = clength * scale*

using *assms unfolding trace-table-low-degree-def*

by (*auto simp: eval-domain-length*)

lemma *finite-low-degree-trace-pair-space[simp]*:

finite low-degree-trace-pair-space

proof –

have *subset*:

low-degree-trace-pair-space ⊆
 {xs. *length xs = clength * scale*} ×
 {xs. *length xs = clength * scale*}

unfolding *low-degree-trace-pair-space-def*

by (*auto dest: trace-table-low-degree-length*)

show *?thesis*

by (*rule finite-subset[OF subset]*)

(*simp add: finite-length-lists-UNIV*)

qed

definition *low-degree-trace-pair-agreement-round-error*

where

low-degree-trace-pair-agreement-round-error budgets i =
 (∑ (trace-table, trace-table') ∈ *low-degree-trace-pair-space*.
 staged-phase-relation-error size
 (*staged-query-search-queries budgets i + 1*) +
 nnreal *clength* / nnreal (*card query-sample-space*))

definition *low-degree-trace-pair-agreement-error*

where

low-degree-trace-pair-agreement-error budgets =
 (∑ *x* ∈ *low-degree-trace-pair-space* × {..*rounds*}.
 case *x* of (*pair*, *i*) ⇒
 staged-phase-relation-error size
 (*staged-query-search-queries budgets i + 1*) +
 nnreal *clength* / nnreal (*card query-sample-space*))

lemma *empty-header-low-degree-transcript-query-agreement-imp-pair-hit*:

assumes

checked-staged-security-with-actual-alpha-prefix-verifier-event

empty-header-low-degree-trace-candidate-nonunique-transcript-query-agreement-hit

```

    out
  shows
     $\exists \text{pair} \in \text{low-degree-trace-pair-space}.$ 
     $\exists i \in \{..<\text{rounds}\}.$ 
     $\text{checked-staged-security-with-actual-alpha-prefix-trace-pair-agreement-hit-at}$ 
       $(\text{fst pair}) (\text{snd pair}) i \text{ out}$ 
  proof (cases out)
  case None
  then show ?thesis
    using assms
    unfolding checked-staged-security-with-actual-alpha-prefix-verifier-event-def
    by simp
next
case (Some packed)
then obtain alpha-prefix alpha-prefix-state data attacker-state result
  final-state where out-eq:
    out =
      Some (((alpha-prefix, alpha-prefix-state), data), attacker-state),
        result), final-state)
  by (cases packed, auto split: prod.splits)
let ?s =
  verifier-state-from-adversary attacker-state
  (staged-proof-transcript data)
have hit:
  empty-header-low-degree-trace-candidate-nonunique-transcript-query-agreement-hit
    ?s (Some (result, final-state))
  using assms
  unfolding out-eq
  checked-staged-security-with-actual-alpha-prefix-verifier-event-def
  by simp
then obtain fr f-fri-roots f-final as dg final trace-query-idxs
  trace-openings trace-table composition-table trace-table' i where
  shape:
    accepted-transcript-shape ?s (Some (result, final-state)) as
    trace-query-idxs
  and distinct: trace-table  $\neq$  trace-table'
  and trace-low: trace-table-low-degree trace-table
  and trace'-low: trace-table-low-degree trace-table'
  and i-bound:  $i < \text{rounds}$ 
  and idx-agree:
    trace-query-idxs !  $i \in$ 
    trace-table-agreement-indices trace-table trace-table'
  by (rule
    empty-header-low-degree-trace-candidate-nonunique-transcript-query-agreementE)
have pair-in:
  (trace-table, trace-table')  $\in$  low-degree-trace-pair-space
  unfolding low-degree-trace-pair-space-def
  using trace-low trace'-low distinct by simp
have pair-hit:

```

```

checked-staged-security-with-actual-alpha-prefix-trace-pair-agreement-hit-at
  trace-table trace-table' i out
unfolding out-eq
  checked-staged-security-with-actual-alpha-prefix-trace-pair-agreement-hit-at-def
  staged-security-with-data-state-trace-pair-agreement-hit-at-def
using shape i-bound idx-agree by auto
show ?thesis
by (intro bexI[of - (trace-table, trace-table')])
    bexI[of - i])
    (use pair-in i-bound pair-hit in simp-all)
qed

lemma checked-staged-security-with-actual-alpha-prefix-empty-header-low-degree-transcript-query-agreement-bound:
assumes wf: staged-budget-wellformed budgets
and controlled: staged-adversary-controlled budgets A
shows
  wp-event
    (checked-staged-security-experiment-with-actual-alpha-prefix-data-state A)
    (checked-staged-security-with-actual-alpha-prefix-verifier-event
      empty-header-low-degree-trace-candidate-nonunique-transcript-query-agreement-hit)
    adversary-initial-state  $\leq$ 
    low-degree-trace-pair-agreement-error budgets
proof -
let ?M =
  checked-staged-security-experiment-with-actual-alpha-prefix-data-state A
let ?pair-hit =
   $\lambda$ (pair, i) out.
    checked-staged-security-with-actual-alpha-prefix-trace-pair-agreement-hit-at
      (fst pair) (snd pair) i out
let ?A = low-degree-trace-pair-space  $\times$  {.. $\text{rounds}$ }
have event-le:
  wp-event ?M
    (checked-staged-security-with-actual-alpha-prefix-verifier-event
      empty-header-low-degree-trace-candidate-nonunique-transcript-query-agreement-hit)
    adversary-initial-state  $\leq$ 
  wp-event ?M ( $\lambda$ out.  $\exists x \in ?A. ?\text{pair-hit } x \text{ out}$ )
  adversary-initial-state
by (rule wp-event-mono)
    (auto dest:
      empty-header-low-degree-transcript-query-agreement-imp-pair-hit)
have finite-A: finite ?A
by simp
have union-bound:
  wp-event ?M ( $\lambda$ out.  $\exists x \in ?A. ?\text{pair-hit } x \text{ out}$ )
  adversary-initial-state  $\leq$ 
  ( $\sum x \in ?A.$ 
    case x of (pair, i)  $\Rightarrow$ 
      staged-phase-relation-error size
      (staged-query-search-queries budgets i + 1) +

```

```

      nnreal clength / nnreal (card query-sample-space))
proof (rule wp-event-finite-UN-bound[OF finite-A])
  fix x
  assume x-in:  $x \in ?A$ 
  then obtain trace-table trace-table' i where x-eq:
     $x = ((\text{trace-table}, \text{trace-table}'), i)$ 
  and pair-in:
     $(\text{trace-table}, \text{trace-table}') \in \text{low-degree-trace-pair-space}$ 
  and i-bound:  $i < \text{rounds}$ 
  by (cases x, auto)
have trace-low: trace-table-low-degree trace-table
  using pair-in unfolding low-degree-trace-pair-space-def by auto
have trace'-low: trace-table-low-degree trace-table'
  using pair-in unfolding low-degree-trace-pair-space-def by auto
have distinct: trace-table  $\neq$  trace-table'
  using pair-in unfolding low-degree-trace-pair-space-def by auto
have bound:
  wp-event ?M
    (checked-staged-security-with-actual-alpha-prefix-trace-pair-agreement-hit-at
      trace-table trace-table' i)
    adversary-initial-state  $\leq$ 
    staged-phase-relation-error size
      (staged-query-search-queries budgets  $i + 1$ ) +
      nnreal clength / nnreal (card query-sample-space)
  by (rule
    checked-staged-security-with-actual-alpha-prefix-trace-pair-agreement-hit-at-bound
      [OF wf controlled i-bound trace-low trace'-low distinct])
show wp-event ?M (?pair-hit x) adversary-initial-state
   $\leq$  (case x of (pair, i)  $\Rightarrow$ 
    staged-phase-relation-error size
      (staged-query-search-queries budgets  $i + 1$ ) +
      nnreal clength / nnreal (card query-sample-space))
  unfolding x-eq
  using bound by simp
qed
have sum-eq:
   $(\sum x \in ?A.$ 
    case x of (pair, i)  $\Rightarrow$ 
      staged-phase-relation-error size
        (staged-query-search-queries budgets  $i + 1$ ) +
        nnreal clength / nnreal (card query-sample-space)) =
    low-degree-trace-pair-agreement-error budgets
  unfolding low-degree-trace-pair-agreement-error-def
    low-degree-trace-pair-agreement-round-error-def
  by (simp add: sum.cartesian-product case-prod-unfold)
show ?thesis
proof (rule order-trans[OF event-le])
  show wp-event ?M ( $\lambda \text{out}. \exists x \in ?A. ?\text{pair-hit } x \text{ out}$ )
    adversary-initial-state  $\leq$ 

```

```

    low-degree-trace-pair-agreement-error budgets
    using union-bound sum-eq by simp
qed
qed
end
end

```

```

theory Soundness-Relevant-Drift-Transcript-Empty
imports
  Soundness-Relevant-Drift-Query-Agreement
  Soundness-Query-Transcript-Openings
begin

```

Transcript-indexed empty-header packaging.

This layer avoids the broad empty-header composition residual whose authenticated-opening witnesses carry existential query indices. The composition branch instead carries the verifier transcript shape, so later query-agreement bounds can use verifier-derived query indices.

```

context soundness
begin

```

```

definition soundness-bad-event-partial-candidate-empty-header-transcript
  :: ('f, 'a) protocol-channel-scheme  $\Rightarrow$ 
    (unit list  $\times$  ('f, 'a) protocol-channel-scheme) option  $\Rightarrow$  bool

```

```

where
  soundness-bad-event-partial-candidate-empty-header-transcript s out
     $\iff$ 
    composition-bad-with-empty-composition-header-candidates-transcript
      s out  $\vee$ 
    trace-fri-bad-with-empty-composition-header-candidates s out  $\vee$ 
    query-bad-with-empty-composition-header-candidates s out

```

```

lemma accepted-empty-composition-header-candidate-partition-transcript:
  assumes false-statement:  $\neg$  exists-valid-trace
  and partial:
    accepted-with-empty-composition-header-candidates s out fr
    f-fri-roots f-final as dg final trace-query-idxs trace-openings
    trace-table composition-table
  and shape: accepted-transcript-shape s out as trace-query-idxs
  shows soundness-bad-event-partial-candidate-empty-header-transcript s out
proof (cases trace-table-low-degree trace-table)
  case False
  have trace-fri-bad-with-empty-composition-header-candidates s out
    unfolding trace-fri-bad-with-empty-composition-header-candidates-def
    using partial False by blast
  then show ?thesis

```

```

    unfolding soundness-bad-event-partial-candidate-empty-header-transcript-def
  by blast
next
case True
note trace-low = True
have composition-low:
  composition-table-low-degree maxDegree composition-table
proof -
  have composition-table = replicate (scale * clength) final
    using partial
    unfolding accepted-with-empty-composition-header-candidates-def by simp
  then show ?thesis
    using constant-composition-table-low-degree by simp
qed
show ?thesis
proof (cases all-queries-consistent trace-table composition-table as)
case False
have query-bad-with-empty-composition-header-candidates s out
  unfolding query-bad-with-empty-composition-header-candidates-def
  using partial trace-low composition-low False by blast
then show ?thesis
  unfolding soundness-bad-event-partial-candidate-empty-header-transcript-def
  by blast
next
case True
note all-queries = True
from trace-low obtain f where
  deg-f: degree f < clength
  and trace-table-eq: trace-table = map (poly f) eval-domain
  unfolding trace-table-low-degree-def by blast
have violated: violated-constraints f ≠ {}
  by (rule false-statement-violated-constraints
    [OF false-statement deg-f])
have composition-bad-with-empty-composition-header-candidates-transcript
  s out
  unfolding composition-bad-with-empty-composition-header-candidates-transcript-def
  using partial shape deg-f trace-table-eq violated composition-low
  all-queries
  by blast
then show ?thesis
  unfolding soundness-bad-event-partial-candidate-empty-header-transcript-def
  by blast
qed
qed

lemma accepted-partition-soundness-bad-event-aligned-transcript-partial-candidate-with-partial-merkle-or-empty-head
assumes false-statement: ¬ exists-valid-trace
and supp: out ∈ set-dist (execute verify-monad s)
and acc: accepted out

```

```

and shape: accepted-transcript-shape s out as query-idxs
and header:
  verifier-header-transcript s fr f-fri-roots f-final as dg
    composition-fri-roots final rest
and trace-partial-if-empty:
  composition-fri-roots = []  $\implies$ 
    accepted-with-partial-trace-openings s out fr query-idxs
      trace-openings
shows
  soundness-bad-event-aligned-transcript-partial-candidate-with-partial-merkle
    s out  $\vee$ 
    soundness-bad-event-partial-candidate-empty-header-transcript s out
proof (cases composition-fri-roots = [])
case False
then have comp-nonempty: composition-fri-roots  $\neq$  []
  by simp
have
  soundness-bad-event-aligned-transcript-partial-candidate-with-partial-merkle
    s out
  by (rule
    accepted-partition-soundness-bad-event-aligned-transcript-partial-candidate-with-partial-merkle-if-header-nonempty
      [OF false-statement supp acc header comp-nonempty])
then show ?thesis by simp
next
case True
note comp-empty = True
from acc obtain result final-state where out-eq:
  out = Some (result, final-state)
  unfolding accepted-def by (cases out) auto
have trace-partial:
  accepted-with-partial-trace-openings s (Some (result, final-state))
    fr query-idxs trace-openings
  using trace-partial-if-empty[OF comp-empty] unfolding out-eq .
show ?thesis
proof (cases
  partial-merkle-inconsistency-bad s (Some (result, final-state)))
case True
then have
  soundness-bad-event-aligned-transcript-partial-candidate-with-partial-merkle
    s out
  unfolding out-eq
    soundness-bad-event-aligned-transcript-partial-candidate-with-partial-merkle-def
  by simp
then show ?thesis by simp
next
case False
obtain trace-table where trace-candidate:
  partial-trace-table-candidate trace-table trace-openings
  using accepted-with-partial-trace-openings-candidate-if-no-partial-merkle-bad

```



```

    [OF trace-partial False]
  by blast
have accepted-out: accepted (Some (result, final-state))
  by (rule accepted-with-partial-trace-openings-imp-accepted
    [OF trace-partial])
let ?composition-table = replicate (scale * clength) final
have empty-partial:
  accepted-with-empty-composition-header-candidates s
    (Some (result, final-state)) fr f-fri-roots f-final as dg final
    query-idxs trace-openings trace-table ?composition-table
unfolding accepted-with-empty-composition-header-candidates-def
  by (intro conjI exI[of - rest])
    (use accepted-out header comp-empty trace-partial trace-candidate
      in simp-all)
have soundness-bad-event-partial-candidate-empty-header-transcript s
  (Some (result, final-state))
  by (rule accepted-empty-composition-header-candidate-partition-transcript
    [OF false-statement empty-partial shape[unfolded out-eq]])
then show ?thesis
  unfolding out-eq by simp
qed
qed

lemma accepted-partition-soundness-bad-event-aligned-transcript-partial-candidate-with-partial-merkle-or-empty-header
assumes false-statement:  $\neg$  exists-valid-trace
and supp:  $out \in \text{set-dist } (\text{execute verify-monad } s)$ 
and acc: accepted out
and header:
  verifier-header-transcript s fr f-fri-roots f-final as dg
  composition-fri-roots final rest
shows
  soundness-bad-event-aligned-transcript-partial-candidate-with-partial-merkle
    s out  $\vee$ 
  soundness-bad-event-partial-candidate-empty-header-transcript s out
proof (cases composition-fri-roots = [])
case False
then have comp-nonempty: composition-fri-roots  $\neq []$ 
  by simp
have
  soundness-bad-event-aligned-transcript-partial-candidate-with-partial-merkle
    s out
  by (rule
    accepted-partition-soundness-bad-event-aligned-transcript-partial-candidate-with-partial-merkle-if-header-nonempty
    [OF false-statement supp acc header comp-nonempty])
then show ?thesis by simp
next
case True
note comp-empty = True
from acc obtain result final-state where out-eq:

```

```

    out = Some (result, final-state)
  unfolding accepted-def by (cases out) auto
have header-empty:
  verifier-header-transcript s fr f-fri-roots f-final as dg []
  final rest
  using header comp-empty by simp
from verify-monad-supplied-empty-header-partial-trace-openings
  [OF supp[unfolded out-eq] header-empty]
obtain query-idxs trace-openings where shape:
  accepted-transcript-shape s (Some (result, final-state)) as
    query-idxs
and trace-partial:
  accepted-with-partial-trace-openings s
  (Some (result, final-state)) fr query-idxs trace-openings
by blast
show ?thesis
proof (cases
  partial-merkle-inconsistency-bad s (Some (result, final-state)))
case True
then have
  soundness-bad-event-aligned-transcript-partial-candidate-with-partial-merkle
    s out
  unfolding out-eq
    soundness-bad-event-aligned-transcript-partial-candidate-with-partial-merkle-def
  by simp
then show ?thesis by simp
next
case False
obtain trace-table where trace-candidate:
  partial-trace-table-candidate trace-table trace-openings
  using accepted-with-partial-trace-openings-candidate-if-no-partial-merkle-bad
    [OF trace-partial False]
  by blast
have accepted-out: accepted (Some (result, final-state))
  by (rule accepted-with-partial-trace-openings-imp-accepted
    [OF trace-partial])
let ?composition-table = replicate (scale * clength) final
have empty-partial:
  accepted-with-empty-composition-header-candidates s
    (Some (result, final-state)) fr f-fri-roots f-final as dg final
    query-idxs trace-openings trace-table ?composition-table
  unfolding accepted-with-empty-composition-header-candidates-def
  by (intro conjI exI [of - rest])
    (use accepted-out header-empty trace-partial trace-candidate
      in simp-all)
have soundness-bad-event-partial-candidate-empty-header-transcript s
  (Some (result, final-state))
  by (rule accepted-empty-composition-header-candidate-partition-transcript
    [OF false-statement empty-partial shape])

```



```

proof
  assume fresh:
    checked-staged-security-with-actual-alpha-prefix-partial-header-fresh-hit
    composition-trace-bad-alpha-space out
  have prefix-or-drift:
    checked-staged-security-with-actual-alpha-prefix-bad-set-hit
    composition-trace-bad-alpha-space out  $\vee$ 
    checked-staged-security-with-actual-alpha-prefix-partial-header-candidate-drift
    composition-trace-bad-alpha-space out
  by (rule
    checked-staged-security-with-actual-alpha-prefix-partial-header-fresh-hit-imp-prefix-or-drift
    [OF fresh])
  then show ?thesis
proof
  assume
    checked-staged-security-with-actual-alpha-prefix-bad-set-hit
    composition-trace-bad-alpha-space out
  then show ?thesis by simp
next
  assume drift:
    checked-staged-security-with-actual-alpha-prefix-partial-header-candidate-drift
    composition-trace-bad-alpha-space out
  have
    checked-staged-security-with-actual-alpha-prefix-empty-header-relevant-drift-transcript
    out
  unfolding
    checked-staged-security-with-actual-alpha-prefix-empty-header-relevant-drift-transcript-def
  using drift hit by simp
  then show ?thesis by simp
qed
next
  assume
    checked-staged-security-with-actual-alpha-prefix-partial-header-prequery-hit
    composition-trace-bad-alpha-space out
  then show ?thesis by simp
qed
qed

lemma checked-staged-security-with-actual-alpha-prefix-empty-header-alpha-hit-transcript-bound-from-empty-relevant
assumes wf: staged-budget-wellformed budgets
and controlled: staged-adversary-controlled budgets A
and drift-bound:
  wp-event
  (checked-staged-security-experiment-with-actual-alpha-prefix-data-state
    A)
  checked-staged-security-with-actual-alpha-prefix-empty-header-relevant-drift-transcript
  adversary-initial-state  $\leq D$ 
shows
  wp-event

```

(checked-staged-security-experiment-with-actual-alpha-prefix-data-state A)
 (checked-staged-security-with-actual-alpha-prefix-verifier-event
 composition-alpha-partial-opening-union-hit-with-empty-header-transcript)
 adversary-initial-state $\leq$
 composition-error-bound +
 staged-phase-relation-error size
 (staged-alpha-search-queries budgets 0) +
 hash-collision-budget-value 0
 (staged-alpha-search-queries budgets 0) +
 staged-phase-relation-error size
 (staged-alpha-search-queries budgets 0) +
 D

proof –

let $?M =$
 checked-staged-security-experiment-with-actual-alpha-prefix-data-state A

let $?Prefix =$
 checked-staged-security-with-actual-alpha-prefix-bad-set-hit
 composition-trace-bad-alpha-space

let $?Prequery =$
 checked-staged-security-with-actual-alpha-prefix-partial-header-prequery-hit
 composition-trace-bad-alpha-space

let $?Drift =$
 checked-staged-security-with-actual-alpha-prefix-empty-header-relevant-drift-transcript

let $?Hit =$
 checked-staged-security-with-actual-alpha-prefix-verifier-event
 composition-alpha-partial-opening-union-hit-with-empty-header-transcript

let $?P =$
 composition-error-bound +
 staged-phase-relation-error size
 (staged-alpha-search-queries budgets 0) +
 hash-collision-budget-value 0
 (staged-alpha-search-queries budgets 0)

let $?Q =$
 staged-phase-relation-error size
 (staged-alpha-search-queries budgets 0)

have transcript-bound:
 wp-event (checked-staged-transcript-with-alpha-prefix-program A)
 (checked-staged-transcript-actual-alpha-prefix-bad-set-hit
 composition-trace-bad-alpha-space)
 adversary-initial-state $\leq ?P$

by (rule
 checked-staged-transcript-with-alpha-prefix-actual-bad-set-hit-bound-from-budgets
 [OF wf controlled])

have prefix-bound:
 wp-event $?M ?Prefix$ adversary-initial-state $\leq ?P$

by (rule
 checked-staged-security-with-actual-alpha-prefix-bad-set-hit-bound-from-transcript
 [OF transcript-bound])

have prequery-bound:

```

wp-event ?M ?Prequery adversary-initial-state ≤ ?Q
by (rule
  checked-staged-security-with-actual-alpha-prefix-vector-alpha-prequery-accounting
  [OF wf controlled])
have event-le:
wp-event ?M ?Hit adversary-initial-state ≤
wp-event ?M
  (λout. ?Prefix out ∨ ?Prequery out ∨ ?Drift out ∨ False)
  adversary-initial-state
by (rule wp-event-mono-on-support)
  (use
    checked-staged-security-with-actual-alpha-prefix-empty-header-alpha-hit-transcript-imp-prefix-prequery-or-empty-
    [of - A] in blast)
also have ... ≤
  wp-event ?M ?Prefix adversary-initial-state +
  wp-event ?M ?Prequery adversary-initial-state +
  wp-event ?M ?Drift adversary-initial-state +
  wp-event ?M (λ-. False) adversary-initial-state
by (rule wp-event-union-bound4)
also have ... ≤ ?P + ?Q + D + 0
by (intro add-mono prefix-bound prequery-bound drift-bound)
  (simp add: wp-event-def wp-def dist-expect-def)
finally show ?thesis
by (simp add: algebra-simps)
qed

lemma checked-staged-security-with-data-state-empty-header-composition-randomization-transcript-bound-from-empty
assumes wf: staged-budget-wellformed budgets
and controlled: staged-adversary-controlled budgets A
and drift-bound:
  wp-event
    (checked-staged-security-experiment-with-actual-alpha-prefix-data-state
      A)
  checked-staged-security-with-actual-alpha-prefix-empty-header-relevant-drift-transcript
  adversary-initial-state ≤ D
shows
  wp-event (checked-staged-security-experiment-with-data-state A)
    (staged-security-with-data-state-verifier-event
      composition-randomization-bad-with-empty-composition-header-candidates-transcript)
  adversary-initial-state ≤
    composition-error-bound +
    staged-phase-relation-error size
    (staged-alpha-search-queries budgets 0) +
    hash-collision-budget-value 0
    (staged-alpha-search-queries budgets 0) +
    staged-phase-relation-error size
    (staged-alpha-search-queries budgets 0) +
    D
proof -

```

```

let ?M = checked-staged-security-experiment-with-data-state A
let ?random =
  staged-security-with-data-state-verifier-event
  composition-randomization-bad-with-empty-composition-header-candidates-transcript
let ?alpha =
  staged-security-with-data-state-verifier-event
  composition-alpha-partial-opening-union-hit-with-empty-header-transcript
have alpha-projected:
  wp-event ?M ?alpha adversary-initial-state =
    wp-event
    (checked-staged-security-experiment-with-actual-alpha-prefix-data-state A)
    (checked-staged-security-with-actual-alpha-prefix-verifier-event
     composition-alpha-partial-opening-union-hit-with-empty-header-transcript)
    adversary-initial-state
  by (rule
    checked-staged-security-with-data-state-verifier-event-actual-alpha-prefix-projection)
have alpha-bound:
  wp-event ?M ?alpha adversary-initial-state ≤
    composition-error-bound +
    staged-phase-relation-error size
    (staged-alpha-search-queries budgets 0) +
    hash-collision-budget-value 0
    (staged-alpha-search-queries budgets 0) +
    staged-phase-relation-error size
    (staged-alpha-search-queries budgets 0) +
    D
unfolding alpha-projected
by (rule
  checked-staged-security-with-actual-alpha-prefix-empty-header-alpha-hit-transcript-bound-from-empty-relevant-drift
  [OF wf controlled drift-bound])
have wp-event ?M ?random adversary-initial-state ≤
  wp-event ?M ?alpha adversary-initial-state
by (rule wp-event-mono)
  (auto simp: staged-security-with-data-state-verifier-event-def
   intro:
   composition-randomization-bad-with-empty-composition-header-candidates-transcript-imp-alpha-union-hit
   split: option.splits prod.splits)
also have ... ≤
  composition-error-bound +
  staged-phase-relation-error size
  (staged-alpha-search-queries budgets 0) +
  hash-collision-budget-value 0
  (staged-alpha-search-queries budgets 0) +
  staged-phase-relation-error size
  (staged-alpha-search-queries budgets 0) +
  D
by (rule alpha-bound)
finally show ?thesis .
qed

```

lemma *checked-staged-security-with-data-state-empty-header-bad-event-transcript-bound-from-empty-relevant-drift-tr*
fixes *empty-trace-fri-error* :: *prob*
assumes *false-statement*: $\neg$ *exists-valid-trace*
and *wf*: *staged-budget-wellformed budgets*
and *controlled*: *staged-adversary-controlled budgets A*
and *drift-bound*:
 wp-event
 (*checked-staged-security-experiment-with-actual-alpha-prefix-data-state*
 A)
 checked-staged-security-with-actual-alpha-prefix-empty-header-relevant-drift-transcript
 adversary-initial-state $\leq D$
and *trace-fri-bound*:
 wp-event (*checked-staged-security-experiment-with-data-state A*)
 (*staged-security-with-data-state-verifier-event*
 trace-fri-bad-with-empty-composition-header-candidates)
 adversary-initial-state $\leq$ *empty-trace-fri-error*
and *query-bound*:
 wp-event (*checked-staged-security-experiment-with-data-state A*)
 (*staged-security-with-data-state-verifier-event*
 query-bad-with-empty-composition-header-candidates)
 adversary-initial-state $\leq$ *empty-query-error*
shows
 wp-event (*checked-staged-security-experiment-with-data-state A*)
 (*staged-security-with-data-state-verifier-event*
 soundness-bad-event-partial-candidate-empty-header-transcript)
 adversary-initial-state $\leq$
 composition-error-bound +
 staged-phase-relation-error size
 (*staged-alpha-search-queries budgets 0*) +
 hash-collision-budget-value 0
 (*staged-alpha-search-queries budgets 0*) +
 staged-phase-relation-error size
 (*staged-alpha-search-queries budgets 0*) +
 D + *empty-trace-fri-error* + *empty-query-error*
proof –
 let *?M* = *checked-staged-security-experiment-with-data-state A*
 let *?E* = *staged-security-with-data-state-verifier-event*
 let *?degree* =
 ?E composition-degree-bad-with-empty-composition-header-candidates-transcript
 let *?random* =
 ?E composition-randomization-bad-with-empty-composition-header-candidates-transcript
 let *?trace* =
 ?E trace-fri-bad-with-empty-composition-header-candidates
 let *?query* =
 ?E query-bad-with-empty-composition-header-candidates
 let *?bad* =
 ?E soundness-bad-event-partial-candidate-empty-header-transcript
 have *degree-false*: *?degree* = (λ out. *False*)


```

by (rule ext)
  (auto simp: staged-security-with-data-state-verifier-event-def
    composition-degree-bad-with-empty-composition-header-candidates-transcript-false
    split: option.splits prod.splits)
have degree-bound:
  wp-event ?M ?degree adversary-initial-state ≤ 0
  unfolding degree-false wp-event-def wp-def dist-expect-def by simp
have random-bound:
  wp-event ?M ?random adversary-initial-state ≤
    composition-error-bound +
    staged-phase-relation-error size
    (staged-alpha-search-queries budgets 0) +
    hash-collision-budget-value 0
    (staged-alpha-search-queries budgets 0) +
    staged-phase-relation-error size
    (staged-alpha-search-queries budgets 0) +
    D
by (rule
  checked-staged-security-with-data-state-empty-header-composition-randomization-transcript-bound-from-empty-re
  [OF wf controlled drift-bound])
have wp-event ?M ?bad adversary-initial-state ≤
  wp-event ?M
  (λout. ?degree out ∨ ?random out ∨ ?trace out ∨
    ?query out)
  adversary-initial-state
proof (rule wp-event-mono)
  fix out
  assume bad: ?bad out
  show ?degree out ∨ ?random out ∨ ?trace out ∨ ?query out
  proof (cases out)
    case None
    then show ?thesis
      using bad
      unfolding staged-security-with-data-state-verifier-event-def
      by simp
  next
  case (Some packed)
  then obtain data attacker-state result final-state where out-eq:
    out = Some (((data, attacker-state), result), final-state)
    by (cases packed, auto split: prod.splits)
  let ?s =
    verifier-state-from-adversary attacker-state
    (staged-proof-transcript data)
  have local-bad:
    soundness-bad-event-partial-candidate-empty-header-transcript ?s
    (Some (result, final-state))
    using bad
    unfolding out-eq staged-security-with-data-state-verifier-event-def
    by simp

```

```

then consider
  (comp)
    composition-bad-with-empty-composition-header-candidates-transcript ?s
    (Some (result, final-state))
  | (trace)
    trace-fri-bad-with-empty-composition-header-candidates ?s
    (Some (result, final-state))
  | (query)
    query-bad-with-empty-composition-header-candidates ?s
    (Some (result, final-state))
unfolding
  soundness-bad-event-partial-candidate-empty-header-transcript-def
by blast
then show ?thesis
proof cases
case comp
then have comp-split:
  composition-degree-bad-with-empty-composition-header-candidates-transcript
    ?s (Some (result, final-state)) ∨
  composition-randomization-bad-with-empty-composition-header-candidates-transcript
    ?s (Some (result, final-state))
by (rule
  composition-bad-with-empty-composition-header-candidates-transcript-split
  [OF false-statement])
then show ?thesis
  unfolding out-eq staged-security-with-data-state-verifier-event-def
  using comp-split by (auto split: option.splits prod.splits)
next
case trace
then show ?thesis
  unfolding out-eq staged-security-with-data-state-verifier-event-def
  by simp
next
case query
then show ?thesis
  unfolding out-eq staged-security-with-data-state-verifier-event-def
  by simp
qed
qed
qed
also have ... ≤
  wp-event ?M ?degree adversary-initial-state +
  wp-event ?M ?random adversary-initial-state +
  wp-event ?M ?trace adversary-initial-state +
  wp-event ?M ?query adversary-initial-state
by (rule wp-event-union-bound4)
also have ... ≤
  0 +
  (composition-error-bound +

```

```

    staged-phase-relation-error size
      (staged-alpha-search-queries budgets 0) +
    hash-collision-budget-value 0
      (staged-alpha-search-queries budgets 0) +
    staged-phase-relation-error size
      (staged-alpha-search-queries budgets 0) +
    D) +
    empty-trace-fri-error + empty-query-error
  by (intro add-mono degree-bound random-bound trace-fri-bound query-bound)
finally show ?thesis
  by (simp add: algebra-simps)
qed

lemma checked-staged-soundness-from-aligned-transcript-partial-candidate-and-empty-header-transcript-bounds:
  fixes partial-candidate-error empty-header-error :: prob
  assumes false-statement:  $\neg$  exists-valid-trace
  and partial-candidate-bound:
    wp-event (checked-staged-security-experiment-with-data-state A)
      (staged-security-with-data-state-verifier-event
        soundness-bad-event-aligned-transcript-partial-candidate-with-partial-merkle)
    adversary-initial-state  $\leq$  partial-candidate-error
  and empty-header-bound:
    wp-event (checked-staged-security-experiment-with-data-state A)
      (staged-security-with-data-state-verifier-event
        soundness-bad-event-partial-candidate-empty-header-transcript)
    adversary-initial-state  $\leq$  empty-header-error
  shows checked-staged-adversary-acceptance-probability A  $\leq$ 
    partial-candidate-error + empty-header-error
proof -
  let ?M = checked-staged-security-experiment-with-data-state A
  let ?E = staged-security-with-data-state-verifier-event
  let ?partial =
    ?E soundness-bad-event-aligned-transcript-partial-candidate-with-partial-merkle
  let ?empty = ?E soundness-bad-event-partial-candidate-empty-header-transcript
  let ?bad =  $\lambda$ out. ?partial out  $\vee$  ?empty out
  have accepted-bad:
    wp-event ?M accepted adversary-initial-state  $\leq$ 
    wp-event ?M ?bad adversary-initial-state
proof (rule wp-event-mono-on-support)
  fix out
  assume support: out  $\in$  set-dist (execute ?M adversary-initial-state)
  and acc: accepted out
  show ?bad out
proof (cases out)
  case None
  then show ?thesis
    using acc unfolding accepted-def by simp
next
  case (Some packed)

```

```

then obtain data attacker-state result final-state where out-eq:
  out = Some (((data, attacker-state), result), final-state)
  by (cases packed, auto split: prod.splits)
let ?s =
  verifier-state-from-adversary attacker-state
  (staged-proof-transcript data)
have verifier:
  Some (result, final-state) ∈ set-dist (execute verify-monad ?s)
  using checked-staged-security-experiment-with-data-state-outcomeE
  [OF support[unfolded out-eq]]
  by blast
have verifier-acc: accepted (Some (result, final-state))
  unfolding accepted-def by simp
from verify-monad-accepted-transcript-shape [OF verifier]
obtain alphas query-idxs where shape:
  accepted-transcript-shape ?s (Some (result, final-state))
  alphas query-idxs
  by blast
from shape obtain result' final-state' fr f-fri-roots f-final dg
  composition-fri-roots final rest where
  header:
    verifier-header-transcript ?s fr f-fri-roots f-final alphas dg
    composition-fri-roots final rest
  by (elim accepted-transcript-shape-header-query-extraction)
have local-bad:
  soundness-bad-event-aligned-transcript-partial-candidate-with-partial-merkle
  ?s (Some (result, final-state)) ∨
  soundness-bad-event-partial-candidate-empty-header-transcript ?s
  (Some (result, final-state))
  by (rule
    accepted-partition-soundness-bad-event-aligned-transcript-partial-candidate-with-partial-merkle-or-empty-head
    [OF false-statement verifier verifier-acc header])
then show ?thesis
  unfolding out-eq staged-security-with-data-state-verifier-event-def
  by simp
qed
qed
have bad-bound:
  wp-event ?M ?bad adversary-initial-state ≤
  partial-candidate-error + empty-header-error
proof –
  have wp-event ?M ?bad adversary-initial-state ≤
  wp-event ?M ?partial adversary-initial-state +
  wp-event ?M ?empty adversary-initial-state
  by (rule wp-event-union-bound)
  also have ... ≤ partial-candidate-error + empty-header-error
  by (intro add-mono partial-candidate-bound empty-header-bound)
  finally show ?thesis .
qed

```

```

have staged-bound:
  wp-event (checked-staged-security-experiment A) accepted
    adversary-initial-state  $\leq$ 
      partial-candidate-error + empty-header-error
  unfolding checked-staged-security-experiment-acceptance-with-data-state
  by (rule order-trans[OF accepted-bad bad-bound])
show ?thesis
proof –
  have
    wp-event (checked-staged-security-experiment A)
      ( $\lambda$ out.  $\neg$  Option.is-none out) adversary-initial-state =
      wp-event (checked-staged-security-experiment A) accepted
        adversary-initial-state
  unfolding wp-event-def accepted-def
  by (rule arg-cong[where
    f= $\lambda$ Q. wp (checked-staged-security-experiment A) Q
    adversary-initial-state])
    (rule ext, simp)
  then show ?thesis
    unfolding checked-staged-adversary-acceptance-probability-def
    using staged-bound by simp
qed
qed

```

```

lemma checked-staged-soundness-from-aligned-transcript-partial-candidate-components-and-current-empty-query-from
fixes trace-fri-error' composition-fri-error' query-error D E
  empty-trace-fri-error
  empty-query-error :: prob
assumes false-statement:  $\neg$  exists-valid-trace
and wf: staged-budget-wellformed budgets
and controlled: staged-adversary-controlled budgets A
and trace-fri-bound:
  wp-event (checked-staged-security-experiment-with-data-state A)
    (staged-security-with-data-state-verifier-event
      trace-fri-bad-with-partial-candidates)
    adversary-initial-state  $\leq$  trace-fri-error'
and comp-fri-bound:
  wp-event (checked-staged-security-experiment-with-data-state A)
    (staged-security-with-data-state-verifier-event
      composition-fri-bad-with-partial-candidates)
    adversary-initial-state  $\leq$  composition-fri-error'
and query-bound:
  wp-event (checked-staged-security-experiment-with-data-state A)
    (staged-security-with-data-state-verifier-event
      query-bad-with-aligned-transcript-partial-candidates)
    adversary-initial-state  $\leq$  query-error
and drift-bound:
  wp-event
    (checked-staged-security-experiment-with-actual-alpha-prefix-data-state

```

A)
checked-staged-security-with-actual-alpha-prefix-relevant-partial-header-candidate-drift
adversary-initial-state $\leq D$
and *empty-drift-bound*:
wp-event
 (*checked-staged-security-experiment-with-actual-alpha-prefix-data-state*
 A)
checked-staged-security-with-actual-alpha-prefix-empty-header-relevant-drift-transcript
adversary-initial-state $\leq E$
and *empty-trace-fri-bound*:
wp-event (*checked-staged-security-experiment-with-data-state* A)
 (*staged-security-with-data-state-verifier-event*
trace-fri-bad-with-empty-composition-header-candidates)
adversary-initial-state $\leq$ *empty-trace-fri-error*
and *current-empty-bound*:
wp-event (*checked-staged-security-experiment-with-data-state* A)
 (*staged-security-with-data-state-current-empty-query-opening-hit-at* 0)
adversary-initial-state $\leq$ *empty-query-error*
shows *checked-staged-adversary-acceptance-probability* A $\leq$
hash-collision-budget-value 0
 (*staged-attacker-query-budget* budgets +
staged-challenge-query-budget + *verifier-hash-query-budget*) +
 (*composition-error-bound* +
staged-phase-relation-error size
 (*staged-alpha-search-queries* budgets 0) +
hash-collision-budget-value 0
 (*staged-alpha-search-queries* budgets 0) +
staged-phase-relation-error size
 (*staged-alpha-search-queries* budgets 0) +
 D) +
trace-fri-error' + *composition-fri-error'* +
query-error +
 (*composition-error-bound* +
staged-phase-relation-error size
 (*staged-alpha-search-queries* budgets 0) +
hash-collision-budget-value 0
 (*staged-alpha-search-queries* budgets 0) +
staged-phase-relation-error size
 (*staged-alpha-search-queries* budgets 0) +
 E + *empty-trace-fri-error* + *empty-query-error*)
proof –
have *comp-bound*:
wp-event (*checked-staged-security-experiment-with-data-state* A)
 (*staged-security-with-data-state-verifier-event*
composition-bad-with-partial-candidates)
adversary-initial-state $\leq$
composition-error-bound +
staged-phase-relation-error size
 (*staged-alpha-search-queries* budgets 0) +

hash-collision-budget-value 0
(staged-alpha-search-queries budgets 0) +
staged-phase-relation-error size
(staged-alpha-search-queries budgets 0) +
D
by (rule
checked-staged-security-with-data-state-partial-candidate-composition-bad-bound-from-relevant-drift-and-budgets
[OF false-statement wf controlled drift-bound])
have *merkle-bound*:
wp-event (checked-staged-security-experiment-with-data-state A)
(staged-security-with-data-state-verifier-event
partial-merkle-inconsistency-bad)
adversary-initial-state ≤
hash-collision-budget-value 0
(staged-attacker-query-budget budgets +
staged-challenge-query-budget + verifier-hash-query-budget)
by (rule
checked-staged-security-with-data-state-partial-merkle-inconsistency-bad-bound-controlled
[OF wf controlled])
have *empty-query-bound*:
wp-event (checked-staged-security-experiment-with-data-state A)
(staged-security-with-data-state-verifier-event
query-bad-with-empty-composition-header-candidates)
adversary-initial-state ≤ empty-query-error
by (rule *order-trans*[
OF checked-staged-security-with-data-state-empty-query-bad-le-current-empty-opening-hit-at-zero
OF wf controlled] current-empty-bound])
have *empty-header-bound*:
wp-event (checked-staged-security-experiment-with-data-state A)
(staged-security-with-data-state-verifier-event
soundness-bad-event-partial-candidate-empty-header-transcript)
adversary-initial-state ≤
composition-error-bound +
staged-phase-relation-error size
(staged-alpha-search-queries budgets 0) +
hash-collision-budget-value 0
(staged-alpha-search-queries budgets 0) +
staged-phase-relation-error size
(staged-alpha-search-queries budgets 0) +
E + empty-trace-fri-error + empty-query-error
by (rule
checked-staged-security-with-data-state-empty-header-bad-event-transcript-bound-from-empty-relevant-drift-trans
[OF false-statement wf controlled empty-drift-bound
empty-trace-fri-bound empty-query-bound])
have *partial-candidate-bound*:
wp-event (checked-staged-security-experiment-with-data-state A)
(staged-security-with-data-state-verifier-event
soundness-bad-event-aligned-transcript-partial-candidate-with-partial-merkle)
adversary-initial-state ≤

```

    hash-collision-budget-value 0
    (staged-attacker-query-budget budgets +
     staged-challenge-query-budget + verifier-hash-query-budget) +
  (composition-error-bound +
   staged-phase-relation-error size
   (staged-alpha-search-queries budgets 0) +
   hash-collision-budget-value 0
   (staged-alpha-search-queries budgets 0) +
   staged-phase-relation-error size
   (staged-alpha-search-queries budgets 0) +
   D) +
  trace-fri-error' + composition-fri-error' + query-error
by (rule
  checked-staged-security-with-data-state-aligned-transcript-partial-candidate-with-partial-merkle-component-union
  [OF merkle-bound comp-bound trace-fri-bound comp-fri-bound
   query-bound])
have bound:
  checked-staged-adversary-acceptance-probability  $A \leq$ 
  (hash-collision-budget-value 0
   (staged-attacker-query-budget budgets +
    staged-challenge-query-budget + verifier-hash-query-budget) +
  (composition-error-bound +
   staged-phase-relation-error size
   (staged-alpha-search-queries budgets 0) +
   hash-collision-budget-value 0
   (staged-alpha-search-queries budgets 0) +
   staged-phase-relation-error size
   (staged-alpha-search-queries budgets 0) +
   D) +
  trace-fri-error' + composition-fri-error' + query-error) +
  (composition-error-bound +
   staged-phase-relation-error size
   (staged-alpha-search-queries budgets 0) +
  hash-collision-budget-value 0
  (staged-alpha-search-queries budgets 0) +
  staged-phase-relation-error size
  (staged-alpha-search-queries budgets 0) +
  E + empty-trace-fri-error + empty-query-error)
by (rule
  checked-staged-soundness-from-aligned-transcript-partial-candidate-and-empty-header-transcript-bounds
  [OF false-statement partial-candidate-bound empty-header-bound])
then show ?thesis
by (simp add: algebra-simps)
qed

lemma checked-staged-soundness-from-aligned-transcript-partial-candidate-components-and-current-empty-query-from
fixes trace-fri-error' composition-fri-error' query-error D
  empty-trace-fri-error empty-query-error :: prob
assumes false-statement:  $\neg$  exists-valid-trace

```


and *wf: staged-budget-wellformed budgets*
and *controlled: staged-adversary-controlled budgets A*
and *trace-fri-bound:*
 wp-event (checked-staged-security-experiment-with-data-state A)
 (staged-security-with-data-state-verifier-event
 trace-fri-bad-with-partial-candidates)
 adversary-initial-state $\leq$ trace-fri-error'
and *comp-fri-bound:*
 wp-event (checked-staged-security-experiment-with-data-state A)
 (staged-security-with-data-state-verifier-event
 composition-fri-bad-with-partial-candidates)
 adversary-initial-state $\leq$ composition-fri-error'
and *query-bound:*
 wp-event (checked-staged-security-experiment-with-data-state A)
 (staged-security-with-data-state-verifier-event
 query-bad-with-aligned-transcript-partial-candidates)
 adversary-initial-state $\leq$ query-error
and *drift-bound:*
 wp-event
 (checked-staged-security-experiment-with-actual-alpha-prefix-data-state
 A)
 checked-staged-security-with-actual-alpha-prefix-relevant-partial-header-candidate-drift
 adversary-initial-state $\leq$ D
and *empty-trace-fri-bound:*
 wp-event (checked-staged-security-experiment-with-data-state A)
 (staged-security-with-data-state-verifier-event
 trace-fri-bad-with-empty-composition-header-candidates)
 adversary-initial-state $\leq$ empty-trace-fri-error
and *current-empty-bound:*
 wp-event (checked-staged-security-experiment-with-data-state A)
 (staged-security-with-data-state-current-empty-query-opening-hit-at 0)
 adversary-initial-state $\leq$ empty-query-error
shows *checked-staged-adversary-acceptance-probability A $\leq$*
 hash-collision-budget-value 0
 (staged-attacker-query-budget budgets +
 staged-challenge-query-budget + verifier-hash-query-budget) +
 (composition-error-bound +
 staged-phase-relation-error size
 (staged-alpha-search-queries budgets 0) +
 hash-collision-budget-value 0
 (staged-alpha-search-queries budgets 0) +
 staged-phase-relation-error size
 (staged-alpha-search-queries budgets 0) +
 D) +
 trace-fri-error' + composition-fri-error' +
 query-error +
 (composition-error-bound +
 staged-phase-relation-error size
 (staged-alpha-search-queries budgets 0) +

hash-collision-budget-value 0
(staged-alpha-search-queries budgets 0) +
staged-phase-relation-error size
(staged-alpha-search-queries budgets 0) +
D + empty-trace-fri-error + empty-query-error)
proof –
have *empty-drift-bound*:
wp-event
(checked-staged-security-experiment-with-actual-alpha-prefix-data-state A)
checked-staged-security-with-actual-alpha-prefix-empty-header-relevant-drift-transcript
adversary-initial-state ≤ D
by (rule
checked-staged-security-with-actual-alpha-prefix-empty-header-relevant-drift-transcript-bound-from-relevant-drift
[OF drift-bound])
show *?thesis*
by (rule
checked-staged-soundness-from-aligned-transcript-partial-candidate-components-and-current-empty-query-from-re
[OF false-statement wf controlled trace-fri-bound comp-fri-bound
query-bound drift-bound empty-drift-bound empty-trace-fri-bound
current-empty-bound])
qed
lemma *checked-staged-security-with-actual-alpha-prefix-empty-header-relevant-drift-transcript-bound-from-reachable-*
fixes *F L :: prob*
assumes *trace-fri-reduction*:
 $\bigwedge \text{data attacker-state.}$
Some (data, attacker-state) ∈
set-dist (execute (checked-staged-transcript-program A)
adversary-initial-state) ⇒
trace-fri-reachable-partial-candidate-reduction-assumption
(verifier-state-from-adversary attacker-state
(staged-proof-transcript data))
and *fixed-empty-alpha-bound*:
wp-event
(checked-staged-security-experiment-with-actual-alpha-prefix-data-state
A)
(checked-staged-security-with-actual-alpha-prefix-verifier-event
composition-alpha-bad-set-hit-with-empty-composition-header-candidates)
adversary-initial-state ≤ F
and *low-degree-query-agreement-bound*:
wp-event
(checked-staged-security-experiment-with-actual-alpha-prefix-data-state
A)
(checked-staged-security-with-actual-alpha-prefix-verifier-event
empty-header-low-degree-trace-candidate-nonunique-query-agreement-hit)
adversary-initial-state ≤ L
shows
wp-event
(checked-staged-security-experiment-with-actual-alpha-prefix-data-state A)

checked-staged-security-with-actual-alpha-prefix-empty-header-relevant-drift-transcript
adversary-initial-state $\leq F + (\text{trace-fri-error} + L)$

proof –

let *?M* =
checked-staged-security-experiment-with-actual-alpha-prefix-data-state A
let *?trace* =
checked-staged-security-with-actual-alpha-prefix-verifier-event
trace-fri-bad-with-empty-composition-header-candidates
let *?low-degree-transcript* =
checked-staged-security-with-actual-alpha-prefix-verifier-event
empty-header-low-degree-trace-candidate-nonunique-transcript-query-agreement-hit
let *?low-degree* =
checked-staged-security-with-actual-alpha-prefix-verifier-event
empty-header-low-degree-trace-candidate-nonunique-query-agreement-hit
have *trace-data-bound*:
wp-event (checked-staged-security-experiment-with-data-state A)
(staged-security-with-data-state-verifier-event
trace-fri-bad-with-empty-composition-header-candidates)
adversary-initial-state $\leq$ *trace-fri-error*
by (rule
trace-fri-bad-with-empty-header-candidates-bound-from-reachable-reduction
[OF trace-fri-reduction])
have *trace-bound*:
wp-event ?M ?trace adversary-initial-state $\leq$ *trace-fri-error*
using *trace-data-bound*
checked-staged-security-with-data-state-verifier-event-actual-alpha-prefix-projection
[of A trace-fri-bad-with-empty-composition-header-candidates]
by *simp*
have *low-degree-transcript-bound*:
wp-event ?M ?low-degree-transcript adversary-initial-state $\leq$ *L*
proof (rule *order-trans*)
show *wp-event ?M ?low-degree-transcript adversary-initial-state* $\leq$
wp-event ?M ?low-degree adversary-initial-state
by (rule *wp-event-mono*)
(auto simp: checked-staged-security-with-actual-alpha-prefix-verifier-event-def
intro:
empty-header-low-degree-trace-candidate-nonunique-transcript-query-agreement-imp-query-agreement
split: option.splits prod.splits)
show *wp-event ?M ?low-degree adversary-initial-state* $\leq$ *L*
by (rule *low-degree-query-agreement-bound*)
qed
show *?thesis*
by (rule
checked-staged-security-with-actual-alpha-prefix-empty-header-relevant-drift-transcript-bound-from-fixed-trace-fri-
[OF fixed-empty-alpha-bound trace-bound low-degree-transcript-bound])
qed

lemma *checked-staged-security-with-actual-alpha-prefix-empty-header-relevant-drift-transcript-bound-from-reachable-*
fixes *F* :: *prob*

assumes *wf: staged-budget-wellformed budgets*
and *controlled: staged-adversary-controlled budgets A*
and *trace-fri-reduction:*
 $\bigwedge \text{data attacker-state.}$
 $\text{Some (data, attacker-state)} \in$
 $\text{set-dist (execute (checked-staged-transcript-program A)}$
 $\text{adversary-initial-state)} \implies$
 $\text{trace-fri-reachable-partial-candidate-reduction-assumption}$
 $(\text{verifier-state-from-adversary attacker-state}$
 $(\text{staged-proof-transcript data}))$
and *fixed-empty-alpha-bound:*
 wp-event
 $(\text{checked-staged-security-experiment-with-actual-alpha-prefix-data-state}$
 $A)$
 $(\text{checked-staged-security-with-actual-alpha-prefix-verifier-event}$
 $\text{composition-alpha-bad-set-hit-with-empty-composition-header-candidates})$
 $\text{adversary-initial-state} \leq F$
shows
 wp-event
 $(\text{checked-staged-security-experiment-with-actual-alpha-prefix-data-state } A)$
 $\text{checked-staged-security-with-actual-alpha-prefix-empty-header-relevant-drift-transcript}$
 $\text{adversary-initial-state} \leq$
 $F + (\text{trace-fri-error} + \text{low-degree-trace-pair-agreement-error budgets})$
proof –
let $?M =$
 $\text{checked-staged-security-experiment-with-actual-alpha-prefix-data-state } A$
have *trace-data-bound:*
 $\text{wp-event (checked-staged-security-experiment-with-data-state } A)$
 $(\text{staged-security-with-data-state-verifier-event}$
 $\text{trace-fri-bad-with-empty-composition-header-candidates})$
 $\text{adversary-initial-state} \leq \text{trace-fri-error}$
by (rule
 $\text{trace-fri-bad-with-empty-header-candidates-bound-from-reachable-reduction}$
 $[\text{OF trace-fri-reduction}])$
have *trace-bound:*
 $\text{wp-event } ?M$
 $(\text{checked-staged-security-with-actual-alpha-prefix-verifier-event}$
 $\text{trace-fri-bad-with-empty-composition-header-candidates})$
 $\text{adversary-initial-state} \leq \text{trace-fri-error}$
using *trace-data-bound*
 $\text{checked-staged-security-with-data-state-verifier-event-actual-alpha-prefix-projection}$
 $[\text{of } A \text{ trace-fri-bad-with-empty-composition-header-candidates}]$
by *simp*
have *low-degree-bound:*
 $\text{wp-event } ?M$
 $(\text{checked-staged-security-with-actual-alpha-prefix-verifier-event}$
 $\text{empty-header-low-degree-trace-candidate-nonunique-transcript-query-agreement-hit})$
 $\text{adversary-initial-state} \leq$
 $\text{low-degree-trace-pair-agreement-error budgets}$

```

    by (rule
      checked-staged-security-with-actual-alpha-prefix-empty-header-low-degree-transcript-query-agreement-bound
      [OF wf controlled])
  show ?thesis
    by (rule
      checked-staged-security-with-actual-alpha-prefix-empty-header-relevant-drift-transcript-bound-from-fixed-trace-fri-
      [OF fixed-empty-alpha-bound trace-bound low-degree-bound])
qed

lemma checked-staged-soundness-from-aligned-transcript-partial-candidate-components-and-current-empty-query-from
  fixes trace-fri-error' composition-fri-error' query-error D F L
  empty-query-error :: prob
  assumes false-statement:  $\neg$  exists-valid-trace
  and wf: staged-budget-wellformed budgets
  and controlled: staged-adversary-controlled budgets A
  and trace-fri-bound:
    wp-event (checked-staged-security-experiment-with-data-state A)
      (staged-security-with-data-state-verifier-event
        trace-fri-bad-with-partial-candidates)
    adversary-initial-state  $\leq$  trace-fri-error'
  and comp-fri-bound:
    wp-event (checked-staged-security-experiment-with-data-state A)
      (staged-security-with-data-state-verifier-event
        composition-fri-bad-with-partial-candidates)
    adversary-initial-state  $\leq$  composition-fri-error'
  and query-bound:
    wp-event (checked-staged-security-experiment-with-data-state A)
      (staged-security-with-data-state-verifier-event
        query-bad-with-aligned-transcript-partial-candidates)
    adversary-initial-state  $\leq$  query-error
  and drift-bound:
    wp-event
      (checked-staged-security-experiment-with-actual-alpha-prefix-data-state
        A)
    checked-staged-security-with-actual-alpha-prefix-relevant-partial-header-candidate-drift
    adversary-initial-state  $\leq$  D
  and trace-fri-reduction:
     $\bigwedge$  data attacker-state.
    Some (data, attacker-state)  $\in$ 
      set-dist (execute (checked-staged-transcript-program A)
        adversary-initial-state)  $\implies$ 
    trace-fri-reachable-partial-candidate-reduction-assumption
      (verifier-state-from-adversary attacker-state
        (staged-proof-transcript data))
  and fixed-empty-alpha-bound:
    wp-event
      (checked-staged-security-experiment-with-actual-alpha-prefix-data-state
        A)
    (checked-staged-security-with-actual-alpha-prefix-verifier-event

```

composition-alpha-bad-set-hit-with-empty-composition-header-candidates)
adversary-initial-state $\leq F$
and *low-degree-query-agreement-bound*:
wp-event
(checked-staged-security-experiment-with-actual-alpha-prefix-data-state
A)
(checked-staged-security-with-actual-alpha-prefix-verifier-event
empty-header-low-degree-trace-candidate-nonunique-query-agreement-hit)
adversary-initial-state $\leq L$
and *current-empty-bound*:
wp-event (checked-staged-security-experiment-with-data-state A)
(staged-security-with-data-state-current-empty-query-opening-hit-at 0)
adversary-initial-state $\leq$ *empty-query-error*
shows *checked-staged-adversary-acceptance-probability A* $\leq$
hash-collision-budget-value 0
(staged-attacker-query-budget budgets +
staged-challenge-query-budget + verifier-hash-query-budget) +
(composition-error-bound +
staged-phase-relation-error size
(staged-alpha-search-queries budgets 0) +
hash-collision-budget-value 0
(staged-alpha-search-queries budgets 0) +
staged-phase-relation-error size
(staged-alpha-search-queries budgets 0) +
D) +
trace-fri-error' + composition-fri-error' +
query-error +
(composition-error-bound +
staged-phase-relation-error size
(staged-alpha-search-queries budgets 0) +
hash-collision-budget-value 0
(staged-alpha-search-queries budgets 0) +
staged-phase-relation-error size
(staged-alpha-search-queries budgets 0) +
(F + (trace-fri-error + L)) + trace-fri-error +
empty-query-error)
proof –
have *empty-drift-bound*:
wp-event
(checked-staged-security-experiment-with-actual-alpha-prefix-data-state A)
checked-staged-security-with-actual-alpha-prefix-empty-header-relevant-drift-transcript
adversary-initial-state $\leq F + (\text{trace-fri-error} + L)$
by (rule
checked-staged-security-with-actual-alpha-prefix-empty-header-relevant-drift-transcript-bound-from-reachable-trac
[OF trace-fri-reduction fixed-empty-alpha-bound
low-degree-query-agreement-bound])
have *empty-trace-fri-bound*:
wp-event (checked-staged-security-experiment-with-data-state A)
(staged-security-with-data-state-verifier-event

```

    trace-fri-bad-with-empty-composition-header-candidates)
  adversary-initial-state  $\leq$  trace-fri-error
by (rule
  trace-fri-bad-with-empty-header-candidates-bound-from-reachable-reduction
  [OF trace-fri-reduction])
show ?thesis
by (rule
  checked-staged-soundness-from-aligned-transcript-partial-candidate-components-and-current-empty-query-from-re
  [OF false-statement wf controlled trace-fri-bound comp-fri-bound
    query-bound drift-bound empty-drift-bound empty-trace-fri-bound
    current-empty-bound])
qed

lemma checked-staged-soundness-from-aligned-transcript-partial-candidate-components-and-current-empty-query-from-
fixes trace-fri-error' composition-fri-error' query-error D F
  empty-query-error :: prob
assumes false-statement:  $\neg$  exists-valid-trace
and wf: staged-budget-wellformed budgets
and controlled: staged-adversary-controlled budgets A
and trace-fri-bound:
  wp-event (checked-staged-security-experiment-with-data-state A)
  (staged-security-with-data-state-verifier-event
    trace-fri-bad-with-partial-candidates)
  adversary-initial-state  $\leq$  trace-fri-error'
and comp-fri-bound:
  wp-event (checked-staged-security-experiment-with-data-state A)
  (staged-security-with-data-state-verifier-event
    composition-fri-bad-with-partial-candidates)
  adversary-initial-state  $\leq$  composition-fri-error'
and query-bound:
  wp-event (checked-staged-security-experiment-with-data-state A)
  (staged-security-with-data-state-verifier-event
    query-bad-with-aligned-transcript-partial-candidates)
  adversary-initial-state  $\leq$  query-error
and drift-bound:
  wp-event
  (checked-staged-security-experiment-with-actual-alpha-prefix-data-state
    A)
  checked-staged-security-with-actual-alpha-prefix-relevant-partial-header-candidate-drift
  adversary-initial-state  $\leq$  D
and trace-fri-reduction:
   $\bigwedge$  data attacker-state.
  Some (data, attacker-state)  $\in$ 
    set-dist (execute (checked-staged-transcript-program A)
      adversary-initial-state)  $\implies$ 
    trace-fri-reachable-partial-candidate-reduction-assumption
      (verifier-state-from-adversary attacker-state
        (staged-proof-transcript data))
and fixed-empty-alpha-bound:

```

wp-event
 (checked-staged-security-experiment-with-actual-alpha-prefix-data-state
 A)
 (checked-staged-security-with-actual-alpha-prefix-verifier-event
 composition-alpha-bad-set-hit-with-empty-composition-header-candidates)
 adversary-initial-state $\leq F$
and current-empty-bound:
 wp-event (checked-staged-security-experiment-with-data-state *A*)
 (staged-security-with-data-state-current-empty-query-opening-hit-at 0)
 adversary-initial-state $\leq$ empty-query-error
shows checked-staged-adversary-acceptance-probability *A* $\leq$
 hash-collision-budget-value 0
 (staged-attacker-query-budget budgets +
 staged-challenge-query-budget + verifier-hash-query-budget) +
 (composition-error-bound +
 staged-phase-relation-error size
 (staged-alpha-search-queries budgets 0) +
 hash-collision-budget-value 0
 (staged-alpha-search-queries budgets 0) +
 staged-phase-relation-error size
 (staged-alpha-search-queries budgets 0) +
 D) +
 trace-fri-error' + composition-fri-error' +
 query-error +
 (composition-error-bound +
 staged-phase-relation-error size
 (staged-alpha-search-queries budgets 0) +
 hash-collision-budget-value 0
 (staged-alpha-search-queries budgets 0) +
 staged-phase-relation-error size
 (staged-alpha-search-queries budgets 0) +
 (F + (trace-fri-error +
 low-degree-trace-pair-agreement-error budgets)) +
 trace-fri-error + empty-query-error)
proof –
have empty-drift-bound:
 wp-event
 (checked-staged-security-experiment-with-actual-alpha-prefix-data-state *A*)
 checked-staged-security-with-actual-alpha-prefix-empty-header-relevant-drift-transcript
 adversary-initial-state $\leq$
 F + (trace-fri-error + low-degree-trace-pair-agreement-error budgets)
by (rule
 checked-staged-security-with-actual-alpha-prefix-empty-header-relevant-drift-transcript-bound-from-reachable-trac
 [OF wf controlled trace-fri-reduction fixed-empty-alpha-bound])
have empty-trace-fri-bound:
 wp-event (checked-staged-security-experiment-with-data-state *A*)
 (staged-security-with-data-state-verifier-event
 trace-fri-bad-with-empty-composition-header-candidates)
 adversary-initial-state $\leq$ trace-fri-error


```

    by (rule
      trace-fri-bad-with-empty-header-candidates-bound-from-reachable-reduction
      [OF trace-fri-reduction])
  show ?thesis
  by (rule
    checked-staged-soundness-from-aligned-transcript-partial-candidate-components-and-current-empty-query-from-re
    [OF false-statement wf controlled trace-fri-bound comp-fri-bound
      query-bound drift-bound empty-drift-bound empty-trace-fri-bound
      current-empty-bound])
qed

theorem stark-soundness-from-aligned-transcript-partial-components-current-query-prefix-and-current-empty-query-n
  fixes  $D F L$  empty-query-error :: prob
  and  $C :: \text{nat} \Rightarrow \text{prob}$ 
  assumes false-statement:  $\neg \text{exists-valid-trace}$ 
  and wf: staged-budget-wellformed budgets
  and controlled: staged-adversary-controlled budgets  $A$ 
  and trace-fri-reduction:
     $\bigwedge \text{data attacker-state.}$ 
    Some (data, attacker-state)  $\in$ 
      set-dist (execute (checked-staged-transcript-program  $A$ )
        adversary-initial-state)  $\implies$ 
      trace-fri-reachable-partial-candidate-reduction-assumption
        (verifier-state-from-adversary attacker-state
          (staged-proof-transcript data))
  and composition-fri-reduction:
     $\bigwedge \text{data attacker-state.}$ 
    Some (data, attacker-state)  $\in$ 
      set-dist (execute (checked-staged-transcript-program  $A$ )
        adversary-initial-state)  $\implies$ 
      composition-fri-reachable-partial-candidate-reduction-assumption
        (verifier-state-from-adversary attacker-state
          (staged-proof-transcript data))
  and round-bound:
     $\bigwedge i. i < \text{rounds} \implies$ 
    wp-event
      (checked-staged-security-experiment-with-query-prefix-data-state  $A$   $i$ )
      (checked-staged-security-with-query-prefix-current-query-partial-opening-hit-at  $i$ )
      adversary-initial-state  $\leq C$   $i$ 
  and drift-bound:
    wp-event
      (checked-staged-security-experiment-with-actual-alpha-prefix-data-state
         $A$ )
      checked-staged-security-with-actual-alpha-prefix-relevant-partial-header-candidate-drift
      adversary-initial-state  $\leq D$ 
  and fixed-empty-alpha-bound:
    wp-event
      (checked-staged-security-experiment-with-actual-alpha-prefix-data-state
         $A$ )

```

*(checked-staged-security-with-actual-alpha-prefix-verifier-event
composition-alpha-bad-set-hit-with-empty-composition-header-candidates)*
adversary-initial-state $\leq F$
and *low-degree-query-agreement-bound:*
wp-event
(checked-staged-security-experiment-with-actual-alpha-prefix-data-state
A)
(checked-staged-security-with-actual-alpha-prefix-verifier-event
empty-header-low-degree-trace-candidate-nonunique-query-agreement-hit)
adversary-initial-state $\leq L$
and *current-empty-bound:*
wp-event (checked-staged-security-experiment-with-data-state A)
(staged-security-with-data-state-current-empty-query-opening-hit-at 0)
adversary-initial-state $\leq$ *empty-query-error*
shows *checked-staged-adversary-acceptance-probability A* $\leq$
hash-collision-budget-value 0
(staged-attacker-query-budget budgets +
staged-challenge-query-budget + verifier-hash-query-budget) +
(composition-error-bound +
staged-phase-relation-error size
(staged-alpha-search-queries budgets 0) +
hash-collision-budget-value 0
(staged-alpha-search-queries budgets 0) +
staged-phase-relation-error size
(staged-alpha-search-queries budgets 0) +
D) +
trace-fri-error + composition-fri-error +
($\sum_{i < \text{rounds}} C i$) +
(composition-error-bound +
staged-phase-relation-error size
(staged-alpha-search-queries budgets 0) +
hash-collision-budget-value 0
(staged-alpha-search-queries budgets 0) +
staged-phase-relation-error size
(staged-alpha-search-queries budgets 0) +
(F + (trace-fri-error + L)) + trace-fri-error +
empty-query-error)
proof –
have *trace-fri-bound:*
wp-event (checked-staged-security-experiment-with-data-state A)
(staged-security-with-data-state-verifier-event
trace-fri-bad-with-partial-candidates)
adversary-initial-state $\leq$ *trace-fri-error*
by (*rule*
trace-fri-bad-with-partial-candidates-bound-from-reachable-reduction
[*OF trace-fri-reduction*])
have *comp-fri-bound:*
wp-event (checked-staged-security-experiment-with-data-state A)
(staged-security-with-data-state-verifier-event

```

    composition-fri-bad-with-partial-candidates)
  adversary-initial-state ≤ composition-fri-error
by (rule
  composition-fri-bad-with-partial-candidates-bound-from-reachable-reduction
  [OF composition-fri-reduction])
have current-query-bound:
  wp-event (checked-staged-security-experiment-with-data-state A)
  staged-security-with-data-state-current-query-partial-opening-hit
  adversary-initial-state ≤ (∑ i<rounds. C i)
by (rule
  checked-staged-security-with-data-state-current-query-partial-opening-hit-bound-from-query-prefix-rounds
  [OF round-bound wf controlled])
have query-bound:
  wp-event (checked-staged-security-experiment-with-data-state A)
  (staged-security-with-data-state-verifier-event
    query-bad-with-aligned-transcript-partial-candidates)
  adversary-initial-state ≤ (∑ i<rounds. C i)
by (rule order-trans[
  OF checked-staged-security-with-data-state-aligned-transcript-query-bad-bound-from-current-partial-opening-hit
  current-query-bound])
  (use wf controlled in simp-all)
show ?thesis
by (rule
  checked-staged-soundness-from-aligned-transcript-partial-candidate-components-and-current-empty-query-from-re
  [OF false-statement wf controlled trace-fri-bound comp-fri-bound
    query-bound drift-bound trace-fri-reduction fixed-empty-alpha-bound
    low-degree-query-agreement-bound current-empty-bound])
qed

theorem stark-soundness-from-aligned-transcript-partial-components-current-query-prefix-and-current-empty-query-r
fixes D F empty-query-error :: prob
and C :: nat ⇒ prob
assumes false-statement: ¬ exists-valid-trace
and wf: staged-budget-wellformed budgets
and controlled: staged-adversary-controlled budgets A
and trace-fri-reduction:
  ∧ data attacker-state.
  Some (data, attacker-state) ∈
    set-dist (execute (checked-staged-transcript-program A)
      adversary-initial-state) ⇒
    trace-fri-reachable-partial-candidate-reduction-assumption
    (verifier-state-from-adversary attacker-state
      (staged-proof-transcript data))
and composition-fri-reduction:
  ∧ data attacker-state.
  Some (data, attacker-state) ∈
    set-dist (execute (checked-staged-transcript-program A)
      adversary-initial-state) ⇒
    composition-fri-reachable-partial-candidate-reduction-assumption

```

(verifier-state-from-adversary attacker-state
 (staged-proof-transcript data))
and round-bound:
 $\bigwedge i. i < \text{rounds} \implies$
 wp-event
 (checked-staged-security-experiment-with-query-prefix-data-state A i)
 (checked-staged-security-with-query-prefix-current-query-partial-opening-hit-at i)
 adversary-initial-state $\leq C$ i
and drift-bound:
 wp-event
 (checked-staged-security-experiment-with-actual-alpha-prefix-data-state
 A)
 checked-staged-security-with-actual-alpha-prefix-relevant-partial-header-candidate-drift
 adversary-initial-state $\leq D$
and fixed-empty-alpha-bound:
 wp-event
 (checked-staged-security-experiment-with-actual-alpha-prefix-data-state
 A)
 (checked-staged-security-with-actual-alpha-prefix-verifier-event
 composition-alpha-bad-set-hit-with-empty-composition-header-candidates)
 adversary-initial-state $\leq F$
and current-empty-bound:
 wp-event (checked-staged-security-experiment-with-data-state A)
 (staged-security-with-data-state-current-empty-query-opening-hit-at 0)
 adversary-initial-state $\leq$ empty-query-error
shows checked-staged-adversary-acceptance-probability $A \leq$
 hash-collision-budget-value 0
 (staged-attacker-query-budget budgets +
 staged-challenge-query-budget + verifier-hash-query-budget) +
 (composition-error-bound +
 staged-phase-relation-error size
 (staged-alpha-search-queries budgets 0) +
 hash-collision-budget-value 0
 (staged-alpha-search-queries budgets 0) +
 staged-phase-relation-error size
 (staged-alpha-search-queries budgets 0) +
 D) +
 trace-fri-error + composition-fri-error +
 ($\sum i < \text{rounds}. C$ i) +
 (composition-error-bound +
 staged-phase-relation-error size
 (staged-alpha-search-queries budgets 0) +
 hash-collision-budget-value 0
 (staged-alpha-search-queries budgets 0) +
 staged-phase-relation-error size
 (staged-alpha-search-queries budgets 0) +
 (F + (trace-fri-error +
 low-degree-trace-pair-agreement-error budgets)) +
 trace-fri-error + empty-query-error)

proof –

have *trace-fri-bound*:

wp-event (*checked-staged-security-experiment-with-data-state A*)
 (*staged-security-with-data-state-verifier-event*
trace-fri-bad-with-partial-candidates)
adversary-initial-state $\leq$ *trace-fri-error*

by (*rule*

trace-fri-bad-with-partial-candidates-bound-from-reachable-reduction
 [*OF trace-fri-reduction*])

have *comp-fri-bound*:

wp-event (*checked-staged-security-experiment-with-data-state A*)
 (*staged-security-with-data-state-verifier-event*
composition-fri-bad-with-partial-candidates)
adversary-initial-state $\leq$ *composition-fri-error*

by (*rule*

composition-fri-bad-with-partial-candidates-bound-from-reachable-reduction
 [*OF composition-fri-reduction*])

have *current-query-bound*:

wp-event (*checked-staged-security-experiment-with-data-state A*)
staged-security-with-data-state-current-query-partial-opening-hit
adversary-initial-state $\leq$ ($\sum i < \text{rounds}. C\ i$)

by (*rule*

checked-staged-security-with-data-state-current-query-partial-opening-hit-bound-from-query-prefix-rounds
 [*OF round-bound wf controlled*])

have *query-bound*:

wp-event (*checked-staged-security-experiment-with-data-state A*)
 (*staged-security-with-data-state-verifier-event*
query-bad-with-aligned-transcript-partial-candidates)
adversary-initial-state $\leq$ ($\sum i < \text{rounds}. C\ i$)

by (*rule* *order-trans*[

OF checked-staged-security-with-data-state-aligned-transcript-query-bad-bound-from-current-partial-opening-hit
current-query-bound])

(*use wf controlled in simp-all*)

show *?thesis*

by (*rule*

checked-staged-soundness-from-aligned-transcript-partial-candidate-components-and-current-empty-query-from-reachable-reduction
 [*OF false-statement wf controlled trace-fri-bound comp-fri-bound*
query-bound drift-bound trace-fri-reduction fixed-empty-alpha-bound
current-empty-bound])

qed

theorem *stark-soundness-from-aligned-transcript-partial-components-current-query-prefix-and-current-empty-query-from-reachable-reduction*

fixes *D empty-query-error* :: *prob*

and *C* :: *nat* $\Rightarrow$ *prob*

assumes *false-statement*: $\neg$ *exists-valid-trace*

and *wf*: *staged-budget-wellformed budgets*

and *controlled*: *staged-adversary-controlled budgets A*

and *trace-fri-reduction*:

$\bigwedge$ *data attacker-state*.

Some (data, attacker-state) ∈
set-dist (execute (checked-staged-transcript-program A)
adversary-initial-state) ⇒
trace-fri-reachable-partial-candidate-reduction-assumption
(verifier-state-from-adversary attacker-state
(staged-proof-transcript data))
and *composition-fri-reduction:*
 $\bigwedge$ *data attacker-state.*
Some (data, attacker-state) ∈
set-dist (execute (checked-staged-transcript-program A)
adversary-initial-state) ⇒
composition-fri-reachable-partial-candidate-reduction-assumption
(verifier-state-from-adversary attacker-state
(staged-proof-transcript data))
and *round-bound:*
 $\bigwedge i. i < \text{rounds} \Rightarrow$
wp-event
(checked-staged-security-experiment-with-query-prefix-data-state A i)
(checked-staged-security-with-query-prefix-current-query-partial-opening-hit-at i)
adversary-initial-state ≤ C i
and *drift-bound:*
wp-event
(checked-staged-security-experiment-with-actual-alpha-prefix-data-state
A)
checked-staged-security-with-actual-alpha-prefix-relevant-partial-header-candidate-drift
adversary-initial-state ≤ D
and *current-empty-bound:*
wp-event (checked-staged-security-experiment-with-data-state A)
(staged-security-with-data-state-current-empty-query-opening-hit-at 0)
adversary-initial-state ≤ empty-query-error
shows *checked-staged-adversary-acceptance-probability A ≤*
hash-collision-budget-value 0
(staged-attacker-query-budget budgets +
staged-challenge-query-budget + verifier-hash-query-budget) +
(composition-error-bound +
staged-phase-relation-error size
(staged-alpha-search-queries budgets 0) +
hash-collision-budget-value 0
(staged-alpha-search-queries budgets 0) +
staged-phase-relation-error size
(staged-alpha-search-queries budgets 0) +
D) +
trace-fri-error + composition-fri-error +
 $(\sum i < \text{rounds}. C i) +$
(composition-error-bound +
staged-phase-relation-error size
(staged-alpha-search-queries budgets 0) +
hash-collision-budget-value 0
(staged-alpha-search-queries budgets 0) +

staged-phase-relation-error size
(staged-alpha-search-queries budgets 0) +
D + trace-fri-error + empty-query-error)
proof –
have *trace-fri-bound*:
wp-event (checked-staged-security-experiment-with-data-state A)
(staged-security-with-data-state-verifier-event
trace-fri-bad-with-partial-candidates)
adversary-initial-state ≤ trace-fri-error
by (rule
trace-fri-bad-with-partial-candidates-bound-from-reachable-reduction
[OF trace-fri-reduction])
have *comp-fri-bound*:
wp-event (checked-staged-security-experiment-with-data-state A)
(staged-security-with-data-state-verifier-event
composition-fri-bad-with-partial-candidates)
adversary-initial-state ≤ composition-fri-error
by (rule
composition-fri-bad-with-partial-candidates-bound-from-reachable-reduction
[OF composition-fri-reduction])
have *current-query-bound*:
wp-event (checked-staged-security-experiment-with-data-state A)
staged-security-with-data-state-current-query-partial-opening-hit
adversary-initial-state ≤ (∑ i<rounds. C i)
by (rule
checked-staged-security-with-data-state-current-query-partial-opening-hit-bound-from-query-prefix-rounds
[OF round-bound wf controlled])
have *query-bound*:
wp-event (checked-staged-security-experiment-with-data-state A)
(staged-security-with-data-state-verifier-event
query-bad-with-aligned-transcript-partial-candidates)
adversary-initial-state ≤ (∑ i<rounds. C i)
by (rule *order-trans*[
OF checked-staged-security-with-data-state-aligned-transcript-query-bad-bound-from-current-partial-opening-hit
current-query-bound])
(use wf controlled in simp-all)
have *empty-trace-fri-bound*:
wp-event (checked-staged-security-experiment-with-data-state A)
(staged-security-with-data-state-verifier-event
trace-fri-bad-with-empty-composition-header-candidates)
adversary-initial-state ≤ trace-fri-error
by (rule
trace-fri-bad-with-empty-header-candidates-bound-from-reachable-reduction
[OF trace-fri-reduction])
show *?thesis*
by (rule
checked-staged-soundness-from-aligned-transcript-partial-candidate-components-and-current-empty-query-from-re
[OF false-statement wf controlled trace-fri-bound comp-fri-bound
query-bound drift-bound empty-trace-fri-bound current-empty-bound])

qed

theorem *stark-soundness-from-aligned-transcript-partial-components-current-query-prefix-and-current-empty-query-n*

fixes $D\ N\ L$ *empty-query-error* :: prob

and $C :: \text{nat} \Rightarrow \text{prob}$

assumes *false-statement*: $\neg \text{exists-valid-trace}$

and *wf*: *staged-budget-wellformed budgets*

and *controlled*: *staged-adversary-controlled budgets A*

and *trace-fri-reduction*:

$\bigwedge \text{data attacker-state.}$

Some (*data, attacker-state*) $\in$

set-dist (*execute* (*checked-staged-transcript-program A*)

adversary-initial-state) $\implies$

trace-fri-reachable-partial-candidate-reduction-assumption

(*verifier-state-from-adversary attacker-state*

(*staged-proof-transcript data*))

and *composition-fri-reduction*:

$\bigwedge \text{data attacker-state.}$

Some (*data, attacker-state*) $\in$

set-dist (*execute* (*checked-staged-transcript-program A*)

adversary-initial-state) $\implies$

composition-fri-reachable-partial-candidate-reduction-assumption

(*verifier-state-from-adversary attacker-state*

(*staged-proof-transcript data*))

and *round-bound*:

$\bigwedge i. i < \text{rounds} \implies$

wp-event

(*checked-staged-security-experiment-with-query-prefix-data-state A i*)

(*checked-staged-security-with-query-prefix-current-query-partial-opening-hit-at i*)

adversary-initial-state $\leq C\ i$

and *drift-bound*:

wp-event

(*checked-staged-security-experiment-with-actual-alpha-prefix-data-state*

A)

checked-staged-security-with-actual-alpha-prefix-relevant-partial-header-candidate-drift

adversary-initial-state $\leq D$

and *not-prefix-bound*:

wp-event

(*checked-staged-security-experiment-with-actual-alpha-prefix-data-state*

A)

(*checked-staged-security-with-actual-alpha-prefix-partial-header-candidate-not-prefix-bound*

composition-trace-bad-alpha-space)

adversary-initial-state $\leq N$

and *low-degree-query-agreement-bound*:

wp-event

(*checked-staged-security-experiment-with-actual-alpha-prefix-data-state*

A)

(*checked-staged-security-with-actual-alpha-prefix-verifier-event*

empty-header-low-degree-trace-candidate-nonunique-query-agreement-hit)

adversary-initial-state $\leq L$
and *current-empty-bound*:
wp-event (*checked-staged-security-experiment-with-data-state* A)
(*staged-security-with-data-state-current-empty-query-opening-hit-at* 0)
adversary-initial-state $\leq$ *empty-query-error*
shows *checked-staged-adversary-acceptance-probability* $A \leq$
hash-collision-budget-value 0
(*staged-attacker-query-budget* *budgets* +
staged-challenge-query-budget + *verifier-hash-query-budget*) +
(*composition-error-bound* +
staged-phase-relation-error *size*
(*staged-alpha-search-queries* *budgets* 0) +
hash-collision-budget-value 0
(*staged-alpha-search-queries* *budgets* 0) +
staged-phase-relation-error *size*
(*staged-alpha-search-queries* *budgets* 0) +
 D) +
trace-fri-error + *composition-fri-error* +
 $(\sum_{i < \text{rounds}} C_i)$ +
(*composition-error-bound* +
staged-phase-relation-error *size*
(*staged-alpha-search-queries* *budgets* 0) +
hash-collision-budget-value 0
(*staged-alpha-search-queries* *budgets* 0) +
staged-phase-relation-error *size*
(*staged-alpha-search-queries* *budgets* 0) +
((*composition-error-bound* +
staged-phase-relation-error *size*
(*staged-alpha-search-queries* *budgets* 0) +
hash-collision-budget-value 0
(*staged-alpha-search-queries* *budgets* 0) + N) +
(*trace-fri-error* + L)) +
trace-fri-error + *empty-query-error*)
proof –
let $?M =$
checked-staged-security-experiment-with-actual-alpha-prefix-data-state A
let $?P =$
composition-error-bound +
staged-phase-relation-error *size*
(*staged-alpha-search-queries* *budgets* 0) +
hash-collision-budget-value 0
(*staged-alpha-search-queries* *budgets* 0)
have *transcript-bound*:
wp-event (*checked-staged-transcript-with-alpha-prefix-program* A)
(*checked-staged-transcript-actual-alpha-prefix-bad-set-hit*
composition-trace-bad-alpha-space)
adversary-initial-state $\leq ?P$
by (*rule*
checked-staged-transcript-with-alpha-prefix-actual-bad-set-hit-bound-from-budgets

```

    [OF wf controlled])
have prefix-bound:
  wp-event ?M
  (checked-staged-security-with-actual-alpha-prefix-bad-set-hit
   composition-trace-bad-alpha-space)
  adversary-initial-state ≤ ?P
by (rule
  checked-staged-security-with-actual-alpha-prefix-bad-set-hit-bound-from-transcript
  [OF transcript-bound])
have fixed-empty-alpha-bound:
  wp-event ?M
  (checked-staged-security-with-actual-alpha-prefix-verifier-event
   composition-alpha-bad-set-hit-with-empty-composition-header-candidates)
  adversary-initial-state ≤ ?P + N
proof (rule order-trans)
show wp-event ?M
  (checked-staged-security-with-actual-alpha-prefix-verifier-event
   composition-alpha-bad-set-hit-with-empty-composition-header-candidates)
  adversary-initial-state ≤
  wp-event ?M
  (checked-staged-security-with-actual-alpha-prefix-bad-set-hit
   composition-trace-bad-alpha-space)
  adversary-initial-state +
  wp-event ?M
  (checked-staged-security-with-actual-alpha-prefix-partial-header-candidate-not-prefix-bound
   composition-trace-bad-alpha-space)
  adversary-initial-state
by (rule
  checked-staged-security-with-actual-alpha-prefix-fixed-empty-alpha-hit-le-prefix-or-not-prefix
  [OF wf controlled])
show ... ≤ ?P + N
by (intro add-mono prefix-bound not-prefix-bound)
qed
show ?thesis
by (rule
  stark-soundness-from-aligned-transcript-partial-components-current-query-prefix-and-current-empty-query-reach
  [OF false-statement wf controlled trace-fri-reduction
   composition-fri-reduction round-bound drift-bound
   fixed-empty-alpha-bound low-degree-query-agreement-bound
   current-empty-bound])
qed

theorem stark-soundness-from-aligned-transcript-partial-components-current-query-prefix-and-current-empty-query-r
fixes D N empty-query-error :: prob
and C :: nat ⇒ prob
assumes false-statement: ¬ exists-valid-trace
and wf: staged-budget-wellformed budgets
and controlled: staged-adversary-controlled budgets A
and trace-fri-reduction:

```

$\wedge$ *data attacker-state.*
Some (data, attacker-state) $\in$
set-dist (execute (checked-staged-transcript-program A)
adversary-initial-state) $\implies$
trace-fri-reachable-partial-candidate-reduction-assumption
(verifier-state-from-adversary attacker-state
(staged-proof-transcript data))
and *composition-fri-reduction:*
 $\wedge$ *data attacker-state.*
Some (data, attacker-state) $\in$
set-dist (execute (checked-staged-transcript-program A)
adversary-initial-state) $\implies$
composition-fri-reachable-partial-candidate-reduction-assumption
(verifier-state-from-adversary attacker-state
(staged-proof-transcript data))
and *round-bound:*
 $\wedge$ *i. i < rounds $\implies$*
wp-event
(checked-staged-security-experiment-with-query-prefix-data-state A i)
(checked-staged-security-with-query-prefix-current-query-partial-opening-hit-at i)
adversary-initial-state $\leq C$ i
and *drift-bound:*
wp-event
(checked-staged-security-experiment-with-actual-alpha-prefix-data-state
A)
checked-staged-security-with-actual-alpha-prefix-relevant-partial-header-candidate-drift
adversary-initial-state $\leq D$
and *not-prefix-bound:*
wp-event
(checked-staged-security-experiment-with-actual-alpha-prefix-data-state
A)
(checked-staged-security-with-actual-alpha-prefix-partial-header-candidate-not-prefix-bound
composition-trace-bad-alpha-space)
adversary-initial-state $\leq N$
and *current-empty-bound:*
wp-event (checked-staged-security-experiment-with-data-state A)
(staged-security-with-data-state-current-empty-query-opening-hit-at 0)
adversary-initial-state $\leq$ empty-query-error
shows *checked-staged-adversary-acceptance-probability A $\leq$*
hash-collision-budget-value 0
(staged-attacker-query-budget budgets +
staged-challenge-query-budget + verifier-hash-query-budget) +
(composition-error-bound +
staged-phase-relation-error size
(staged-alpha-search-queries budgets 0) +
hash-collision-budget-value 0
(staged-alpha-search-queries budgets 0) +
staged-phase-relation-error size
(staged-alpha-search-queries budgets 0) +

$D) +$
 $\text{trace-fri-error} + \text{composition-fri-error} +$
 $(\sum_{i < \text{rounds}} C_i) +$
 $(\text{composition-error-bound} +$
 $\text{staged-phase-relation-error size}$
 $(\text{staged-alpha-search-queries budgets } 0) +$
 $\text{hash-collision-budget-value } 0$
 $(\text{staged-alpha-search-queries budgets } 0) +$
 $\text{staged-phase-relation-error size}$
 $(\text{staged-alpha-search-queries budgets } 0) +$
 $((\text{composition-error-bound} +$
 $\text{staged-phase-relation-error size}$
 $(\text{staged-alpha-search-queries budgets } 0) +$
 $\text{hash-collision-budget-value } 0$
 $(\text{staged-alpha-search-queries budgets } 0) + N) +$
 $(\text{trace-fri-error} + \text{low-degree-trace-pair-agreement-error budgets})) +$
 $\text{trace-fri-error} + \text{empty-query-error})$
proof $-$
let $?M =$
 $\text{checked-staged-security-experiment-with-actual-alpha-prefix-data-state } A$
let $?P =$
 $\text{composition-error-bound} +$
 $\text{staged-phase-relation-error size}$
 $(\text{staged-alpha-search-queries budgets } 0) +$
 $\text{hash-collision-budget-value } 0$
 $(\text{staged-alpha-search-queries budgets } 0)$
have transcript-bound:
 $\text{wp-event } (\text{checked-staged-transcript-with-alpha-prefix-program } A)$
 $(\text{checked-staged-transcript-actual-alpha-prefix-bad-set-hit}$
 $\text{composition-trace-bad-alpha-space})$
 $\text{adversary-initial-state} \leq ?P$
by (rule
 $\text{checked-staged-transcript-with-alpha-prefix-actual-bad-set-hit-bound-from-budgets}$
 $[\text{OF wf controlled}])$
have prefix-bound:
 $\text{wp-event } ?M$
 $(\text{checked-staged-security-with-actual-alpha-prefix-bad-set-hit}$
 $\text{composition-trace-bad-alpha-space})$
 $\text{adversary-initial-state} \leq ?P$
by (rule
 $\text{checked-staged-security-with-actual-alpha-prefix-bad-set-hit-bound-from-transcript}$
 $[\text{OF transcript-bound}])$
have $\text{fixed-empty-alpha-bound:}$
 $\text{wp-event } ?M$
 $(\text{checked-staged-security-with-actual-alpha-prefix-verifier-event}$
 $\text{composition-alpha-bad-set-hit-with-empty-composition-header-candidates})$
 $\text{adversary-initial-state} \leq ?P + N$
proof (rule order-trans)
show $\text{wp-event } ?M$

```

    (checked-staged-security-with-actual-alpha-prefix-verifier-event
      composition-alpha-bad-set-hit-with-empty-composition-header-candidates)
  adversary-initial-state ≤
  wp-event ?M
    (checked-staged-security-with-actual-alpha-prefix-bad-set-hit
      composition-trace-bad-alpha-space)
  adversary-initial-state +
  wp-event ?M
    (checked-staged-security-with-actual-alpha-prefix-partial-header-candidate-not-prefix-bound
      composition-trace-bad-alpha-space)
  adversary-initial-state
  by (rule
    checked-staged-security-with-actual-alpha-prefix-fixed-empty-alpha-hit-le-prefix-or-not-prefix
    [OF wf controlled])
  show ... ≤ ?P + N
  by (intro add-mono prefix-bound not-prefix-bound)
qed
show ?thesis
  by (rule
    stark-soundness-from-aligned-transcript-partial-components-current-query-prefix-and-current-empty-query-reach
    [OF false-statement wf controlled trace-fri-reduction
      composition-fri-reduction round-bound drift-bound
      fixed-empty-alpha-bound current-empty-bound])
qed

theorem stark-soundness-from-aligned-transcript-partial-components-current-query-prefix-and-current-empty-query-j
fixes D N empty-query-error :: prob
and C :: nat ⇒ prob
assumes false-statement: ¬ exists-valid-trace
and wf: staged-budget-wellformed budgets
and controlled: staged-adversary-controlled budgets A
and trace-fri-reduction:
  ∧ data attacker-state.
    Some (data, attacker-state) ∈
      set-dist (execute (checked-staged-transcript-program A)
        adversary-initial-state) ⇒
      trace-fri-reachable-partial-candidate-reduction-assumption
        (verifier-state-from-adversary attacker-state
          (staged-proof-transcript data))
and composition-fri-reduction:
  ∧ data attacker-state.
    Some (data, attacker-state) ∈
      set-dist (execute (checked-staged-transcript-program A)
        adversary-initial-state) ⇒
      composition-fri-reachable-partial-candidate-reduction-assumption
        (verifier-state-from-adversary attacker-state
          (staged-proof-transcript data))
and round-bound:
  ∧ i. i < rounds ⇒

```

wp-event
 (checked-staged-security-experiment-with-query-prefix-data-state A i)
 (checked-staged-security-with-query-prefix-current-query-partial-opening-hit-at i)
 adversary-initial-state $\leq C$ i
and drift-bound:
wp-event
 (checked-staged-security-experiment-with-actual-alpha-prefix-data-state
 A)
 checked-staged-security-with-actual-alpha-prefix-relevant-partial-header-candidate-drift
 adversary-initial-state $\leq D$
and not-prefix-bound:
wp-event
 (checked-staged-security-experiment-with-actual-alpha-prefix-data-state
 A)
 (checked-staged-security-with-actual-alpha-prefix-partial-header-candidate-not-prefix-bound
 composition-trace-bad-alpha-space)
 adversary-initial-state $\leq N$
and current-empty-bound:
wp-event (checked-staged-security-experiment-with-data-state A)
 (staged-security-with-data-state-current-empty-query-opening-hit-at 0)
 adversary-initial-state $\leq$ empty-query-error
shows checked-staged-adversary-acceptance-probability $A \leq$
 hash-collision-budget-value 0
 (staged-attacker-query-budget budgets +
 staged-challenge-query-budget + verifier-hash-query-budget) +
 (composition-error-bound +
 staged-phase-relation-error size
 (staged-alpha-search-queries budgets 0) +
 hash-collision-budget-value 0
 (staged-alpha-search-queries budgets 0) +
 staged-phase-relation-error size
 (staged-alpha-search-queries budgets 0) +
 D) +
 trace-fri-error + composition-fri-error +
 ($\sum i < \text{rounds. } C$ i) +
 (composition-error-bound +
 staged-phase-relation-error size
 (staged-alpha-search-queries budgets 0) +
 hash-collision-budget-value 0
 (staged-alpha-search-queries budgets 0) +
 staged-phase-relation-error size
 (staged-alpha-search-queries budgets 0) +
 D + trace-fri-error + empty-query-error)
by (rule
 stark-soundness-from-aligned-transcript-partial-components-current-query-prefix-and-current-empty-query-relevant
 [OF false-statement wf controlled trace-fri-reduction
 composition-fri-reduction round-bound drift-bound
 current-empty-bound])

theorem *stark-soundness-from-aligned-transcript-partial-components-current-query-prefix-and-current-empty-query-n*
fixes $X P R L$ *empty-query-error* $:: \text{prob}$
and $C :: \text{nat} \Rightarrow \text{prob}$
assumes *false-statement*: $\neg \text{exists-valid-trace}$
and *wf*: *staged-budget-wellformed budgets*
and *controlled*: *staged-adversary-controlled budgets A*
and *trace-fri-reduction*:
 $\bigwedge \text{data attacker-state.}$
 $\text{Some (data, attacker-state)} \in$
 $\text{set-dist (execute (checked-staged-transcript-program A)}$
 $\text{adversary-initial-state)} \implies$
 $\text{trace-fri-reachable-partial-candidate-reduction-assumption}$
 $(\text{verifier-state-from-adversary attacker-state}$
 $(\text{staged-proof-transcript data}))$
and *composition-fri-reduction*:
 $\bigwedge \text{data attacker-state.}$
 $\text{Some (data, attacker-state)} \in$
 $\text{set-dist (execute (checked-staged-transcript-program A)}$
 $\text{adversary-initial-state)} \implies$
 $\text{composition-fri-reachable-partial-candidate-reduction-assumption}$
 $(\text{verifier-state-from-adversary attacker-state}$
 $(\text{staged-proof-transcript data}))$
and *round-bound*:
 $\bigwedge i. i < \text{rounds} \implies$
 wp-event
 $(\text{checked-staged-security-experiment-with-query-prefix-data-state A } i)$
 $(\text{checked-staged-security-with-query-prefix-current-query-partial-opening-hit-at } i)$
 $\text{adversary-initial-state} \leq C \ i$
and *cross-bound*:
 wp-event
 $(\text{checked-staged-security-experiment-with-actual-alpha-prefix-data-state}$
 $A)$
 $\text{checked-staged-security-with-actual-alpha-prefix-witnessed-cross-or-merkle-bad}$
 $\text{adversary-initial-state} \leq X$
and *data-pre-bound*:
 $\text{wp-event (checked-staged-security-experiment-with-data-state A)}$
 $\text{staged-security-with-data-state-transcript-pre-hit}$
 $\text{adversary-initial-state} \leq P$
and *witness-new-bound*:
 wp-event
 $(\text{checked-staged-security-experiment-with-actual-alpha-prefix-data-state}$
 $A)$
 $\text{checked-staged-security-with-actual-alpha-prefix-witness-transcript-new-hit}$
 $\text{adversary-initial-state} \leq R$
and *low-degree-query-agreement-bound*:
 wp-event
 $(\text{checked-staged-security-experiment-with-actual-alpha-prefix-data-state}$
 $A)$
 $(\text{checked-staged-security-with-actual-alpha-prefix-verifier-event}$

empty-header-low-degree-trace-candidate-nonunique-query-agreement-hit)
adversary-initial-state $\leq L$
and *current-empty-bound*:
wp-event (*checked-staged-security-experiment-with-data-state* *A*)
(*staged-security-with-data-state-current-empty-query-opening-hit-at* *0*)
adversary-initial-state $\leq$ *empty-query-error*
shows *checked-staged-adversary-acceptance-probability* *A* $\leq$
hash-collision-budget-value *0*
(*staged-attacker-query-budget* *budgets* +
staged-challenge-query-budget + *verifier-hash-query-budget*) +
(*composition-error-bound* +
staged-phase-relation-error *size*
(*staged-alpha-search-queries* *budgets* *0*) +
hash-collision-budget-value *0*
(*staged-alpha-search-queries* *budgets* *0*) +
staged-phase-relation-error *size*
(*staged-alpha-search-queries* *budgets* *0*) +
(*hash-collision-budget-value* *0*
(*staged-alpha-search-queries* *budgets* *0*) + *X* + (*P* + *R*) + (*P* + *R*))) +
trace-fri-error + *composition-fri-error* +
($\sum i < \text{rounds. } C \ i$) +
(*composition-error-bound* +
staged-phase-relation-error *size*
(*staged-alpha-search-queries* *budgets* *0*) +
hash-collision-budget-value *0*
(*staged-alpha-search-queries* *budgets* *0*) +
staged-phase-relation-error *size*
(*staged-alpha-search-queries* *budgets* *0*) +
((*composition-error-bound* +
staged-phase-relation-error *size*
(*staged-alpha-search-queries* *budgets* *0*) +
hash-collision-budget-value *0*
(*staged-alpha-search-queries* *budgets* *0*) +
(*hash-collision-budget-value* *0*
(*staged-alpha-search-queries* *budgets* *0*) + *X* + (*P* + *R*) + (*P* + *R*))) +
(*trace-fri-error* + *L*)) +
trace-fri-error + *empty-query-error*)
proof –
let *?D* =
hash-collision-budget-value *0*
(*staged-alpha-search-queries* *budgets* *0*) + *X* + (*P* + *R*) + (*P* + *R*)
have *drift-bound*:
wp-event
(*checked-staged-security-experiment-with-actual-alpha-prefix-data-state* *A*)
checked-staged-security-with-actual-alpha-prefix-relevant-partial-header-candidate-drift
adversary-initial-state $\leq$ *?D*
by (*rule*
checked-staged-security-with-actual-alpha-prefix-relevant-partial-header-candidate-drift-bound-from-cross-data-pre
[*OF* *wf* *controlled cross-bound data-pre-bound witness-new-bound*])

have *transcript-pre-bound*:
wp-event
 (*checked-staged-security-experiment-with-actual-alpha-prefix-data-state A*)
checked-staged-security-with-actual-alpha-prefix-witness-transcript-pre-hit
adversary-initial-state $\leq P$
by (rule
checked-staged-security-with-actual-alpha-prefix-witness-transcript-pre-hit-bound-from-data-state-pre-hit
 [OF *data-pre-bound*])
have *witness-bound*:
wp-event
 (*checked-staged-security-experiment-with-actual-alpha-prefix-data-state A*)
checked-staged-security-with-actual-alpha-prefix-witness-transcript-hit
adversary-initial-state $\leq P + R$
by (rule
checked-staged-security-with-actual-alpha-prefix-witness-transcript-hit-bound-from-pre-and-new
 [OF *transcript-pre-bound witness-new-bound*])
have *no-prefix-bound*:
wp-event
 (*checked-staged-security-experiment-with-actual-alpha-prefix-data-state A*)
checked-staged-security-with-actual-alpha-prefix-not-prefix-no-prefix-candidate
adversary-initial-state $\leq P + R$
by (rule *order-trans*
 [OF
checked-staged-security-with-actual-alpha-prefix-not-prefix-no-prefix-candidate-le-witness-transcript-hit
witness-bound])
have *path-bound*:
wp-event
 (*checked-staged-security-experiment-with-actual-alpha-prefix-data-state A*)
checked-staged-security-with-actual-alpha-prefix-not-prefix-path-output
adversary-initial-state $\leq P + R$
by (rule *order-trans*
 [OF
checked-staged-security-with-actual-alpha-prefix-not-prefix-path-output-le-witness-transcript-hit
witness-bound])
have *collision-bound*:
wp-event
 (*checked-staged-security-experiment-with-actual-alpha-prefix-data-state A*)
checked-staged-security-with-actual-alpha-prefix-not-prefix-collision
adversary-initial-state $\leq$
hash-collision-budget-value 0 (*staged-alpha-search-queries budgets 0*)
by (rule
checked-staged-security-with-actual-alpha-prefix-not-prefix-collision-bound
 [OF *wf controlled*])
have *uncovered-bound*:
wp-event
 (*checked-staged-security-experiment-with-actual-alpha-prefix-data-state A*)
checked-staged-security-with-actual-alpha-prefix-not-prefix-uncovered-index

$adversary-initial-state \leq X$
by (rule order-trans
[OF
checked-staged-security-with-actual-alpha-prefix-not-prefix-uncovered-index-le-witnessed-cross-or-merkle-bad
cross-bound])
have not-prefix-bound:
wp-event
(checked-staged-security-experiment-with-actual-alpha-prefix-data-state A)
(checked-staged-security-with-actual-alpha-prefix-partial-header-candidate-not-prefix-bound
composition-trace-bad-alpha-space)
 $adversary-initial-state \leq ?D$
by (rule
checked-staged-security-with-actual-alpha-prefix-not-prefix-bound-union-bound
[OF wf controlled collision-bound uncovered-bound no-prefix-bound
path-bound])
show ?thesis
by (rule
stark-soundness-from-aligned-transcript-partial-components-current-query-prefix-and-current-empty-query-reach
[OF false-statement wf controlled trace-fri-reduction
composition-fri-reduction round-bound drift-bound not-prefix-bound
low-degree-query-agreement-bound current-empty-bound])
qed

theorem stark-soundness-from-aligned-transcript-partial-components-current-query-prefix-and-current-empty-query-reach
fixes X P R empty-query-error :: prob
and C :: nat $\Rightarrow$ prob
assumes false-statement: $\neg$ exists-valid-trace
and wf: staged-budget-wellformed budgets
and controlled: staged-adversary-controlled budgets A
and trace-fri-reduction:
 $\bigwedge$ data attacker-state.
Some (data, attacker-state) $\in$
set-dist (execute (checked-staged-transcript-program A)
adversary-initial-state) $\implies$
trace-fri-reachable-partial-candidate-reduction-assumption
(verifier-state-from-adversary attacker-state
(staged-proof-transcript data))
and composition-fri-reduction:
 $\bigwedge$ data attacker-state.
Some (data, attacker-state) $\in$
set-dist (execute (checked-staged-transcript-program A)
adversary-initial-state) $\implies$
composition-fri-reachable-partial-candidate-reduction-assumption
(verifier-state-from-adversary attacker-state
(staged-proof-transcript data))
and round-bound:
 $\bigwedge i. i < rounds \implies$
wp-event
(checked-staged-security-experiment-with-query-prefix-data-state A i)

(checked-staged-security-with-query-prefix-current-query-partial-opening-hit-at i)
adversary-initial-state $\leq C i$
and *cross-bound*:
wp-event
(checked-staged-security-experiment-with-actual-alpha-prefix-data-state
A)
checked-staged-security-with-actual-alpha-prefix-witnessed-cross-or-merkle-bad
adversary-initial-state $\leq X$
and *data-pre-bound*:
wp-event (checked-staged-security-experiment-with-data-state A)
staged-security-with-data-state-transcript-pre-hit
adversary-initial-state $\leq P$
and *witness-new-bound*:
wp-event
(checked-staged-security-experiment-with-actual-alpha-prefix-data-state
A)
checked-staged-security-with-actual-alpha-prefix-witness-transcript-new-hit
adversary-initial-state $\leq R$
and *current-empty-bound*:
wp-event (checked-staged-security-experiment-with-data-state A)
(staged-security-with-data-state-current-empty-query-opening-hit-at 0)
adversary-initial-state $\leq \text{empty-query-error}$
shows *checked-staged-adversary-acceptance-probability A* $\leq$
hash-collision-budget-value 0
(staged-attacker-query-budget budgets +
staged-challenge-query-budget + verifier-hash-query-budget) +
(composition-error-bound +
staged-phase-relation-error size
(staged-alpha-search-queries budgets 0) +
hash-collision-budget-value 0
(staged-alpha-search-queries budgets 0) +
staged-phase-relation-error size
(staged-alpha-search-queries budgets 0) +
(hash-collision-budget-value 0
(staged-alpha-search-queries budgets 0) + X + (P + R) + (P + R))) +
trace-fri-error + composition-fri-error +
($\sum i < \text{rounds. } C i$) +
(composition-error-bound +
staged-phase-relation-error size
(staged-alpha-search-queries budgets 0) +
hash-collision-budget-value 0
(staged-alpha-search-queries budgets 0) +
staged-phase-relation-error size
(staged-alpha-search-queries budgets 0) +
((composition-error-bound +
staged-phase-relation-error size
(staged-alpha-search-queries budgets 0) +
hash-collision-budget-value 0
(staged-alpha-search-queries budgets 0) +

$(\text{hash-collision-budget-value } 0$
 $(\text{staged-alpha-search-queries budgets } 0) + X + (P + R) + (P + R))) +$
 $(\text{trace-fri-error} + \text{low-degree-trace-pair-agreement-error budgets})) +$
 $\text{trace-fri-error} + \text{empty-query-error})$

proof –

let $?D =$

$\text{hash-collision-budget-value } 0$
 $(\text{staged-alpha-search-queries budgets } 0) + X + (P + R) + (P + R)$

have *drift-bound*:

wp-event
 $(\text{checked-staged-security-experiment-with-actual-alpha-prefix-data-state } A)$
 $\text{checked-staged-security-with-actual-alpha-prefix-relevant-partial-header-candidate-drift}$
 $\text{adversary-initial-state} \leq ?D$

by (rule
 $\text{checked-staged-security-with-actual-alpha-prefix-relevant-partial-header-candidate-drift-bound-from-cross-data-pre}$
 $[OF \text{ wf controlled cross-bound data-pre-bound witness-new-bound}]$)

have *transcript-pre-bound*:

wp-event
 $(\text{checked-staged-security-experiment-with-actual-alpha-prefix-data-state } A)$
 $\text{checked-staged-security-with-actual-alpha-prefix-witness-transcript-pre-hit}$
 $\text{adversary-initial-state} \leq P$

by (rule
 $\text{checked-staged-security-with-actual-alpha-prefix-witness-transcript-pre-hit-bound-from-data-state-pre-hit}$
 $[OF \text{ data-pre-bound}]$)

have *witness-bound*:

wp-event
 $(\text{checked-staged-security-experiment-with-actual-alpha-prefix-data-state } A)$
 $\text{checked-staged-security-with-actual-alpha-prefix-witness-transcript-hit}$
 $\text{adversary-initial-state} \leq P + R$

by (rule
 $\text{checked-staged-security-with-actual-alpha-prefix-witness-transcript-hit-bound-from-pre-and-new}$
 $[OF \text{ transcript-pre-bound witness-new-bound}]$)

have *no-prefix-bound*:

wp-event
 $(\text{checked-staged-security-experiment-with-actual-alpha-prefix-data-state } A)$
 $\text{checked-staged-security-with-actual-alpha-prefix-not-prefix-no-prefix-candidate}$
 $\text{adversary-initial-state} \leq P + R$

by (rule *order-trans*
 $[OF$
 $\text{checked-staged-security-with-actual-alpha-prefix-not-prefix-no-prefix-candidate-le-witness-transcript-hit}$
 $\text{witness-bound}]$)

have *path-bound*:

wp-event
 $(\text{checked-staged-security-experiment-with-actual-alpha-prefix-data-state } A)$
 $\text{checked-staged-security-with-actual-alpha-prefix-not-prefix-path-output}$
 $\text{adversary-initial-state} \leq P + R$

by (rule *order-trans*
 $[OF$
 $\text{checked-staged-security-with-actual-alpha-prefix-not-prefix-path-output-le-witness-transcript-hit}$

```

        witness-bound]))
have collision-bound:
  wp-event
    (checked-staged-security-experiment-with-actual-alpha-prefix-data-state
     A)
    checked-staged-security-with-actual-alpha-prefix-not-prefix-collision
    adversary-initial-state ≤
    hash-collision-budget-value 0 (staged-alpha-search-queries budgets 0)
  by (rule
    checked-staged-security-with-actual-alpha-prefix-not-prefix-collision-bound
    [OF wf controlled])
have uncovered-bound:
  wp-event
    (checked-staged-security-experiment-with-actual-alpha-prefix-data-state
     A)
    checked-staged-security-with-actual-alpha-prefix-not-prefix-uncovered-index
    adversary-initial-state ≤ X
  by (rule order-trans
    [OF
     checked-staged-security-with-actual-alpha-prefix-not-prefix-uncovered-index-le-witnessed-cross-or-merkle-bad
     cross-bound])
have not-prefix-bound:
  wp-event
    (checked-staged-security-experiment-with-actual-alpha-prefix-data-state A)
    (checked-staged-security-with-actual-alpha-prefix-partial-header-candidate-not-prefix-bound
     composition-trace-bad-alpha-space)
    adversary-initial-state ≤ ?D
  by (rule
    checked-staged-security-with-actual-alpha-prefix-not-prefix-bound-union-bound
    [OF wf controlled collision-bound uncovered-bound no-prefix-bound
     path-bound])
show ?thesis
  by (rule
    stark-soundness-from-aligned-transcript-partial-components-current-query-prefix-and-current-empty-query-reach
    [OF false-statement wf controlled trace-fri-reduction
     composition-fri-reduction round-bound drift-bound not-prefix-bound
     current-empty-bound])
qed

end

```

These transcript-empty wrappers expose the broad witnessed cross/Merkle diagnostic event directly. They are retained only for internal comparison with the sampled-transcript route and are not part of the exported public path.

```

hide-fact
  soundness.stark-soundness-from-aligned-transcript-partial-components-current-query-prefix-and-current-empty-query
  soundness.stark-soundness-from-aligned-transcript-partial-components-current-query-prefix-and-current-empty-query

```

soundness.stark-soundness-from-aligned-transcript-partial-components-current-query-prefix-and-current-empty-query
soundness.stark-soundness-from-aligned-transcript-partial-components-current-query-prefix-and-current-empty-query
soundness.stark-soundness-from-aligned-transcript-partial-components-current-query-prefix-and-current-empty-query
soundness.stark-soundness-from-aligned-transcript-partial-components-current-query-prefix-and-current-empty-query
soundness.stark-soundness-from-aligned-transcript-partial-components-current-query-prefix-and-current-empty-query
soundness.stark-soundness-from-aligned-transcript-partial-components-current-query-prefix-and-current-empty-query

end

theory *Soundness-Relevant-Drift-Transcript-Public*
imports *Soundness-Relevant-Drift-Transcript-Empty*
begin

Public-facing transcript-indexed packaging for the current relevant-drift proof route. This layer keeps new wrappers out of the larger transcript-empty theory and avoids the diagnostic broad witnessed cross/Merkle route for the nonempty partial-candidate branch.

context *soundness*
begin

lemma *accepted-partition-soundness-bad-event-aligned-transcript-partial-candidate-with-aligned-randomization-and-*

assumes *false-statement: $\neg$ exists-valid-trace*
and *supp: out $\in$ set-dist (execute verify-monad s)*
and *acc: accepted out*
and *header:*
verifier-header-transcript s fr f-fri-roots f-final as dg
composition-fri-roots final rest

shows

soundness-bad-event-aligned-transcript-partial-candidate-with-aligned-randomization-and-merkle
s out $\vee$
soundness-bad-event-partial-candidate-empty-header-transcript s out

proof (*cases composition-fri-roots = []*)

case *False*

then have *comp-nonempty: composition-fri-roots $\neq$ []*
by *simp*

have

soundness-bad-event-aligned-transcript-partial-candidate-with-aligned-randomization-and-merkle
s out

by (*rule*

accepted-partition-soundness-bad-event-aligned-transcript-partial-candidate-with-aligned-randomization-and-merkle
[OF false-statement supp acc header comp-nonempty])

then show *?thesis* **by** *simp*

next

case *True*

note *comp-empty = True*

from *acc* **obtain** *result final-state* **where** *out-eq:*

out = Some (result, final-state)

unfolding *accepted-def* **by** (*cases out*) *auto*

```

have header-empty:
  verifier-header-transcript s fr f-fri-roots f-final as dg []
  final rest
using header comp-empty by simp
from verify-monad-supplied-empty-header-partial-trace-openings
  [OF supp[unfolded out-eq] header-empty]
obtain query-idxs trace-openings where shape:
  accepted-transcript-shape s (Some (result, final-state)) as
  query-idxs
and trace-partial:
  accepted-with-partial-trace-openings s
  (Some (result, final-state)) fr query-idxs trace-openings
by blast
show ?thesis
proof (cases
  partial-merkle-inconsistency-bad s (Some (result, final-state)))
case True
then have
  soundness-bad-event-aligned-transcript-partial-candidate-with-aligned-randomization-and-merkle
  s out
  unfolding out-eq
  soundness-bad-event-aligned-transcript-partial-candidate-with-aligned-randomization-and-merkle-def
  by simp
then show ?thesis by simp
next
case False
obtain trace-table where trace-candidate:
  partial-trace-table-candidate trace-table trace-openings
using accepted-with-partial-trace-openings-candidate-if-no-partial-merkle-bad
  [OF trace-partial False]
by blast
have accepted-out: accepted (Some (result, final-state))
  by (rule accepted-with-partial-trace-openings-imp-accepted
  [OF trace-partial])
let ?composition-table = replicate (scale * clength) final
have empty-partial:
  accepted-with-empty-composition-header-candidates s
  (Some (result, final-state)) fr f-fri-roots f-final as dg final
  query-idxs trace-openings trace-table ?composition-table
unfolding accepted-with-empty-composition-header-candidates-def
by (intro conjI exI[of - rest])
  (use accepted-out header-empty trace-partial trace-candidate
  in simp-all)
have soundness-bad-event-partial-candidate-empty-header-transcript s
  (Some (result, final-state))
  by (rule accepted-empty-composition-header-candidate-partition-transcript
  [OF false-statement empty-partial shape])
then show ?thesis
  unfolding out-eq by simp

```

qed
qed

lemma *checked-staged-soundness-from-aligned-transcript-partial-candidate-aligned-randomization-and-empty-header-*
fixes *partial-candidate-error empty-header-error :: prob*

assumes *false-statement: $\neg$ exists-valid-trace*

and *partial-candidate-bound:*

wp-event (checked-staged-security-experiment-with-data-state A)

(staged-security-with-data-state-verifier-event

soundness-bad-event-aligned-transcript-partial-candidate-with-aligned-randomization-and-merkle)

adversary-initial-state $\leq$ partial-candidate-error

and *empty-header-bound:*

wp-event (checked-staged-security-experiment-with-data-state A)

(staged-security-with-data-state-verifier-event

soundness-bad-event-partial-candidate-empty-header-transcript)

adversary-initial-state $\leq$ empty-header-error

shows *checked-staged-adversary-acceptance-probability A $\leq$*

partial-candidate-error + empty-header-error

proof *–*

let *?M = checked-staged-security-experiment-with-data-state A*

let *?E = staged-security-with-data-state-verifier-event*

let *?partial =*

?E soundness-bad-event-aligned-transcript-partial-candidate-with-aligned-randomization-and-merkle

let *?empty = ?E soundness-bad-event-partial-candidate-empty-header-transcript*

let *?bad = λ out. ?partial out $\vee$?empty out*

have *accepted-bad:*

wp-event ?M accepted adversary-initial-state $\leq$

wp-event ?M ?bad adversary-initial-state

proof *(rule wp-event-mono-on-support)*

fix *out*

assume *support: out $\in$ set-dist (execute ?M adversary-initial-state)*

and *acc: accepted out*

show *?bad out*

proof *(cases out)*

case *None*

then show *?thesis*

using *acc unfolding accepted-def by simp*

next

case *(Some packed)*

then obtain *data attacker-state result final-state where out-eq:*

out = Some (((data, attacker-state), result), final-state)

by *(cases packed, auto split: prod.splits)*

let *?s =*

verifier-state-from-adversary attacker-state

(staged-proof-transcript data)

have *verifier:*

Some (result, final-state) $\in$ set-dist (execute verify-monad ?s)

using *checked-staged-security-experiment-with-data-state-outcomeE*

[OF support[unfolded out-eq]]


```

    by blast
  have verifier-acc: accepted (Some (result, final-state))
    unfolding accepted-def by simp
  from verify-monad-accepted-transcript-shape[OF verifier]
  obtain alphas query-idxs where shape:
    accepted-transcript-shape ?s (Some (result, final-state))
    alphas query-idxs
  by blast
  from shape obtain result' final-state' fr f-fri-roots f-final dg
    composition-fri-roots final rest where
    header:
      verifier-header-transcript ?s fr f-fri-roots f-final alphas dg
      composition-fri-roots final rest
  by (elim accepted-transcript-shape-header-query-extraction)
  have local-bad:
    soundness-bad-event-aligned-transcript-partial-candidate-with-aligned-randomization-and-merkle
      ?s (Some (result, final-state))  $\vee$ 
    soundness-bad-event-partial-candidate-empty-header-transcript ?s
      (Some (result, final-state))
  by (rule
    accepted-partition-soundness-bad-event-aligned-transcript-partial-candidate-with-aligned-randomization-and-merkle
      [OF false-statement verifier verifier-acc header])
  then show ?thesis
    unfolding out-eq staged-security-with-data-state-verifier-event-def
    by simp
qed
qed
have bad-bound:
  wp-event ?M ?bad adversary-initial-state  $\leq$ 
  partial-candidate-error + empty-header-error
proof -
  have wp-event ?M ?bad adversary-initial-state  $\leq$ 
    wp-event ?M ?partial adversary-initial-state +
    wp-event ?M ?empty adversary-initial-state
  by (rule wp-event-union-bound)
  also have ...  $\leq$  partial-candidate-error + empty-header-error
  by (intro add-mono partial-candidate-bound empty-header-bound)
  finally show ?thesis .
qed
have staged-bound:
  wp-event (checked-staged-security-experiment A) accepted
    adversary-initial-state  $\leq$ 
    partial-candidate-error + empty-header-error
  unfolding checked-staged-security-experiment-acceptance-with-data-state
  by (rule order-trans[OF accepted-bad bad-bound])
show ?thesis
proof -
  have
    wp-event (checked-staged-security-experiment A)

```

```

    (λout. ¬ Option.is-none out) adversary-initial-state =
wp-event (checked-staged-security-experiment A) accepted
adversary-initial-state
unfolding wp-event-def accepted-def
by (rule arg-cong[where
  f=λQ. wp (checked-staged-security-experiment A) Q
  adversary-initial-state])
  (rule ext, simp)
then show ?thesis
  unfolding checked-staged-adversary-acceptance-probability-def
  using staged-bound by simp
qed
qed

theorem stark-soundness-from-transcript-indexed-partial-opening-route:
fixes F empty-query-error :: prob
and C :: nat ⇒ prob
assumes false-statement: ¬ exists-valid-trace
and wf: staged-budget-wellformed budgets
and controlled: staged-adversary-controlled budgets A
and trace-fri-reduction:
  ∧data attacker-state.
    Some (data, attacker-state) ∈
      set-dist (execute (checked-staged-transcript-program A)
        adversary-initial-state) ⇒
      trace-fri-reachable-partial-candidate-reduction-assumption
        (verifier-state-from-adversary attacker-state
          (staged-proof-transcript data))
and composition-fri-reduction:
  ∧data attacker-state.
    Some (data, attacker-state) ∈
      set-dist (execute (checked-staged-transcript-program A)
        adversary-initial-state) ⇒
      composition-fri-reachable-partial-candidate-reduction-assumption
        (verifier-state-from-adversary attacker-state
          (staged-proof-transcript data))
and round-bound:
  ∧i. i < rounds ⇒
    wp-event
      (checked-staged-security-experiment-with-query-prefix-data-state A i)
      (checked-staged-security-with-query-prefix-current-query-partial-opening-hit-at i)
      adversary-initial-state ≤ C i
and fixed-empty-alpha-bound:
  wp-event
    (checked-staged-security-experiment-with-actual-alpha-prefix-data-state
      A)
    (checked-staged-security-with-actual-alpha-prefix-verifier-event
      composition-alpha-bad-set-hit-with-empty-composition-header-candidates)
    adversary-initial-state ≤ F

```

and *current-empty-bound*:

$$\begin{aligned} & wp\text{-event } (checked\text{-staged}\text{-security}\text{-experiment}\text{-with}\text{-data}\text{-state } A) \\ & \quad (staged\text{-security}\text{-with}\text{-data}\text{-state}\text{-current}\text{-empty}\text{-query}\text{-opening}\text{-hit}\text{-at } 0) \\ & \quad adversary\text{-initial}\text{-state} \leq empty\text{-query}\text{-error} \end{aligned}$$
shows *checked-staged-adversary-acceptance-probability* $A \leq$

$$\begin{aligned} & hash\text{-collision}\text{-budget}\text{-value } 0 \\ & \quad (staged\text{-attacker}\text{-query}\text{-budget } budgets + \\ & \quad \quad staged\text{-challenge}\text{-query}\text{-budget} + verifier\text{-hash}\text{-query}\text{-budget}) + \\ & \quad (\sum i < rounds. C \ i) + \\ & \quad trace\text{-fri}\text{-error} + composition\text{-fri}\text{-error} + \\ & \quad (\sum i < rounds. C \ i) + \\ & \quad (composition\text{-error}\text{-bound} + \\ & \quad \quad staged\text{-phase}\text{-relation}\text{-error } size \\ & \quad \quad (staged\text{-alpha}\text{-search}\text{-queries } budgets \ 0) + \\ & \quad hash\text{-collision}\text{-budget}\text{-value } 0 \\ & \quad \quad (staged\text{-alpha}\text{-search}\text{-queries } budgets \ 0) + \\ & \quad \quad staged\text{-phase}\text{-relation}\text{-error } size \\ & \quad \quad (staged\text{-alpha}\text{-search}\text{-queries } budgets \ 0) + \\ & \quad (F + (trace\text{-fri}\text{-error} + \\ & \quad \quad low\text{-degree}\text{-trace}\text{-pair}\text{-agreement}\text{-error } budgets)) + \\ & \quad trace\text{-fri}\text{-error} + empty\text{-query}\text{-error}) \end{aligned}$$
proof –
have *merkle-bound*:

$$\begin{aligned} & wp\text{-event } (checked\text{-staged}\text{-security}\text{-experiment}\text{-with}\text{-data}\text{-state } A) \\ & \quad (staged\text{-security}\text{-with}\text{-data}\text{-state}\text{-verifier}\text{-event} \\ & \quad \quad partial\text{-merkle}\text{-inconsistency}\text{-bad}) \\ & \quad adversary\text{-initial}\text{-state} \leq \\ & \quad hash\text{-collision}\text{-budget}\text{-value } 0 \\ & \quad \quad (staged\text{-attacker}\text{-query}\text{-budget } budgets + \\ & \quad \quad \quad staged\text{-challenge}\text{-query}\text{-budget} + verifier\text{-hash}\text{-query}\text{-budget}) \end{aligned}$$
by (rule

$$\begin{aligned} & checked\text{-staged}\text{-security}\text{-with}\text{-data}\text{-state}\text{-partial}\text{-merkle}\text{-inconsistency}\text{-bad}\text{-bound}\text{-controlled} \\ & [OF \ wf \ controlled]) \end{aligned}$$
have *comp-bound*:

$$\begin{aligned} & wp\text{-event } (checked\text{-staged}\text{-security}\text{-experiment}\text{-with}\text{-data}\text{-state } A) \\ & \quad (staged\text{-security}\text{-with}\text{-data}\text{-state}\text{-verifier}\text{-event} \\ & \quad \quad composition\text{-randomization}\text{-bad}\text{-with}\text{-aligned}\text{-transcript}\text{-partial}\text{-candidates}) \\ & \quad adversary\text{-initial}\text{-state} \leq (\sum i < rounds. C \ i) \end{aligned}$$
by (rule

$$\begin{aligned} & checked\text{-staged}\text{-security}\text{-with}\text{-data}\text{-state}\text{-aligned}\text{-partial}\text{-candidate}\text{-randomization}\text{-bound}\text{-from}\text{-query}\text{-prefix}\text{-current}\text{-} \\ & [OF \ wf \ controlled \ round\text{-bound}]) \end{aligned}$$
have *trace-fri-bound*:

$$\begin{aligned} & wp\text{-event } (checked\text{-staged}\text{-security}\text{-experiment}\text{-with}\text{-data}\text{-state } A) \\ & \quad (staged\text{-security}\text{-with}\text{-data}\text{-state}\text{-verifier}\text{-event} \\ & \quad \quad trace\text{-fri}\text{-bad}\text{-with}\text{-partial}\text{-candidates}) \\ & \quad adversary\text{-initial}\text{-state} \leq trace\text{-fri}\text{-error} \end{aligned}$$
by (rule

$$\begin{aligned} & trace\text{-fri}\text{-bad}\text{-with}\text{-partial}\text{-candidates}\text{-bound}\text{-from}\text{-reachable}\text{-reduction} \\ & [OF \ trace\text{-fri}\text{-reduction}]) \end{aligned}$$

have *comp-fri-bound*:
wp-event (*checked-staged-security-experiment-with-data-state A*)
(*staged-security-with-data-state-verifier-event*
composition-fri-bad-with-partial-candidates)
adversary-initial-state $\leq$ *composition-fri-error*
by (*rule*
composition-fri-bad-with-partial-candidates-bound-from-reachable-reduction
[*OF composition-fri-reduction*])

have *current-query-bound*:
wp-event (*checked-staged-security-experiment-with-data-state A*)
staged-security-with-data-state-current-query-partial-opening-hit
adversary-initial-state $\leq$ ($\sum i < \text{rounds. } C \ i$)
by (*rule*
checked-staged-security-with-data-state-current-query-partial-opening-hit-bound-from-query-prefix-rounds
[*OF round-bound wf controlled*])

have *query-bound*:
wp-event (*checked-staged-security-experiment-with-data-state A*)
(*staged-security-with-data-state-verifier-event*
query-bad-with-aligned-transcript-partial-candidates)
adversary-initial-state $\leq$ ($\sum i < \text{rounds. } C \ i$)
by (*rule order-trans*[
OF checked-staged-security-with-data-state-aligned-transcript-query-bad-bound-from-current-partial-opening-hit
current-query-bound])
(*use wf controlled in simp-all*)

have *partial-candidate-bound*:
wp-event (*checked-staged-security-experiment-with-data-state A*)
(*staged-security-with-data-state-verifier-event*
soundness-bad-event-aligned-transcript-partial-candidate-with-aligned-randomization-and-merkle)
adversary-initial-state $\leq$
hash-collision-budget-value 0
(*staged-attacker-query-budget budgets* +
staged-challenge-query-budget + *verifier-hash-query-budget*) +
($\sum i < \text{rounds. } C \ i$) + *trace-fri-error* + *composition-fri-error* +
($\sum i < \text{rounds. } C \ i$)
by (*rule*
checked-staged-security-with-data-state-aligned-transcript-partial-candidate-aligned-randomization-with-merkle-un
[*OF merkle-bound comp-bound trace-fri-bound comp-fri-bound*
query-bound])

have *empty-drift-bound*:
wp-event
(*checked-staged-security-experiment-with-actual-alpha-prefix-data-state A*)
checked-staged-security-with-actual-alpha-prefix-empty-header-relevant-drift-transcript
adversary-initial-state $\leq$
F + (*trace-fri-error* + *low-degree-trace-pair-agreement-error budgets*)
by (*rule*
checked-staged-security-with-actual-alpha-prefix-empty-header-relevant-drift-transcript-bound-from-reachable-trac
[*OF wf controlled trace-fri-reduction fixed-empty-alpha-bound*])

have *empty-trace-fri-bound*:
wp-event (*checked-staged-security-experiment-with-data-state A*)

(staged-security-with-data-state-verifier-event
trace-fri-bad-with-empty-composition-header-candidates)
adversary-initial-state $\leq$ *trace-fri-error*
by (rule
trace-fri-bad-with-empty-header-candidates-bound-from-reachable-reduction
[*OF trace-fri-reduction*])
have *empty-query-bound*:
wp-event (checked-staged-security-experiment-with-data-state A)
(staged-security-with-data-state-verifier-event
query-bad-with-empty-composition-header-candidates)
adversary-initial-state $\leq$ *empty-query-error*
by (rule *order-trans*[
OF checked-staged-security-with-data-state-empty-query-bad-le-current-empty-opening-hit-at-zero
OF wf controlled] *current-empty-bound*])
have *empty-header-bound*:
wp-event (checked-staged-security-experiment-with-data-state A)
(staged-security-with-data-state-verifier-event
soundness-bad-event-partial-candidate-empty-header-transcript)
adversary-initial-state $\leq$
composition-error-bound +
staged-phase-relation-error size
(staged-alpha-search-queries budgets 0) +
hash-collision-budget-value 0
(staged-alpha-search-queries budgets 0) +
staged-phase-relation-error size
(staged-alpha-search-queries budgets 0) +
(F + (trace-fri-error +
low-degree-trace-pair-agreement-error budgets)) +
trace-fri-error + empty-query-error
by (rule
checked-staged-security-with-data-state-empty-header-bad-event-transcript-bound-from-empty-relevant-drift-trans
[*OF false-statement wf controlled empty-drift-bound*
empty-trace-fri-bound empty-query-bound])
have *bound*:
checked-staged-adversary-acceptance-probability A $\leq$
(hash-collision-budget-value 0
(staged-attacker-query-budget budgets +
staged-challenge-query-budget + verifier-hash-query-budget) +
($\sum_{i < \text{rounds. } C} i$) + trace-fri-error + composition-fri-error +
($\sum_{i < \text{rounds. } C} i$)) +
(composition-error-bound +
staged-phase-relation-error size
(staged-alpha-search-queries budgets 0) +
hash-collision-budget-value 0
(staged-alpha-search-queries budgets 0) +
staged-phase-relation-error size
(staged-alpha-search-queries budgets 0) +
(F + (trace-fri-error +
low-degree-trace-pair-agreement-error budgets)) +

$trace\text{-}fri\text{-}error + empty\text{-}query\text{-}error)$
by (rule
checked-staged-soundness-from-aligned-transcript-partial-candidate-aligned-randomization-and-empty-header-tran
 $[OF\ false\text{-}statement\ partial\text{-}candidate\text{-}bound\ empty\text{-}header\text{-}bound])$
then show *?thesis*
by (simp add: *algebra-simps*)
qed

theorem *stark-soundness-from-transcript-indexed-partial-opening-route-single-query-charge:*

fixes *F empty-query-error :: prob*

and *C :: nat $\Rightarrow$ prob*

assumes *false-statement: $\neg$ exists-valid-trace*

and *wf: staged-budget-wellformed budgets*

and *controlled: staged-adversary-controlled budgets A*

and *trace-fri-reduction:*

$\bigwedge data\ attacker\text{-}state.$

Some (data, attacker-state) $\in$

set-dist (execute (checked-staged-transcript-program A)

adversary-initial-state) $\implies$

trace-fri-reachable-partial-candidate-reduction-assumption

(verifier-state-from-adversary attacker-state

(staged-proof-transcript data))

and *composition-fri-reduction:*

$\bigwedge data\ attacker\text{-}state.$

Some (data, attacker-state) $\in$

set-dist (execute (checked-staged-transcript-program A)

adversary-initial-state) $\implies$

composition-fri-reachable-partial-candidate-reduction-assumption

(verifier-state-from-adversary attacker-state

(staged-proof-transcript data))

and *round-bound:*

$\bigwedge i. i < rounds \implies$

wp-event

(checked-staged-security-experiment-with-query-prefix-data-state A i)

(checked-staged-security-with-query-prefix-current-query-partial-opening-hit-at i)

adversary-initial-state $\leq C\ i$

and *fixed-empty-alpha-bound:*

wp-event

(checked-staged-security-experiment-with-actual-alpha-prefix-data-state

A)

(checked-staged-security-with-actual-alpha-prefix-verifier-event

composition-alpha-bad-set-hit-with-empty-composition-header-candidates)

adversary-initial-state $\leq F$

and *current-empty-bound:*

wp-event (checked-staged-security-experiment-with-data-state A)

(staged-security-with-data-state-current-empty-query-opening-hit-at 0)

adversary-initial-state $\leq empty\text{-}query\text{-}error$

shows *checked-staged-adversary-acceptance-probability A $\leq$*

hash-collision-budget-value 0

$(\text{staged-attacker-query-budget budgets} +$
 $\text{staged-challenge-query-budget} + \text{verifier-hash-query-budget}) +$
 $(\sum_{i < \text{rounds. } C} i) +$
 $\text{trace-fri-error} + \text{composition-fri-error} +$
 $(\text{composition-error-bound} +$
 $\text{staged-phase-relation-error size}$
 $(\text{staged-alpha-search-queries budgets } 0) +$
 $\text{hash-collision-budget-value } 0$
 $(\text{staged-alpha-search-queries budgets } 0) +$
 $\text{staged-phase-relation-error size}$
 $(\text{staged-alpha-search-queries budgets } 0) +$
 $(F + (\text{trace-fri-error} +$
 $\text{low-degree-trace-pair-agreement-error budgets})) +$
 $\text{trace-fri-error} + \text{empty-query-error})$

proof –

have merkle-bound:

$\text{wp-event } (\text{checked-staged-security-experiment-with-data-state } A)$
 $(\text{staged-security-with-data-state-verifier-event}$
 $\text{partial-merkle-inconsistency-bad})$
 $\text{adversary-initial-state} \leq$
 $\text{hash-collision-budget-value } 0$
 $(\text{staged-attacker-query-budget budgets} +$
 $\text{staged-challenge-query-budget} + \text{verifier-hash-query-budget})$

by (rule
 $\text{checked-staged-security-with-data-state-partial-merkle-inconsistency-bad-bound-controlled}$
 $[\text{OF wf controlled}])$

have trace-fri-bound:

$\text{wp-event } (\text{checked-staged-security-experiment-with-data-state } A)$
 $(\text{staged-security-with-data-state-verifier-event}$
 $\text{trace-fri-bad-with-partial-candidates})$
 $\text{adversary-initial-state} \leq \text{trace-fri-error}$

by (rule
 $\text{trace-fri-bad-with-partial-candidates-bound-from-reachable-reduction}$
 $[\text{OF trace-fri-reduction}])$

have comp-fri-bound:

$\text{wp-event } (\text{checked-staged-security-experiment-with-data-state } A)$
 $(\text{staged-security-with-data-state-verifier-event}$
 $\text{composition-fri-bad-with-partial-candidates})$
 $\text{adversary-initial-state} \leq \text{composition-fri-error}$

by (rule
 $\text{composition-fri-bad-with-partial-candidates-bound-from-reachable-reduction}$
 $[\text{OF composition-fri-reduction}])$

have current-query-bound:

$\text{wp-event } (\text{checked-staged-security-experiment-with-data-state } A)$
 $\text{staged-security-with-data-state-current-query-partial-opening-hit}$
 $\text{adversary-initial-state} \leq (\sum_{i < \text{rounds. } C} i)$

by (rule
 $\text{checked-staged-security-with-data-state-current-query-partial-opening-hit-bound-from-query-prefix-rounds}$
 $[\text{OF round-bound wf controlled}])$

have *partial-candidate-bound*:
wp-event (*checked-staged-security-experiment-with-data-state A*)
(*staged-security-with-data-state-verifier-event*
soundness-bad-event-aligned-transcript-partial-candidate-with-aligned-randomization-and-merkle)
adversary-initial-state $\leq$
hash-collision-budget-value 0
(*staged-attacker-query-budget budgets* +
staged-challenge-query-budget + *verifier-hash-query-budget*) +
($\sum i < \text{rounds}. C\ i$) + *trace-fri-error* + *composition-fri-error*
by (*rule*
checked-staged-security-with-data-state-aligned-transcript-partial-candidate-aligned-randomization-with-merkle-c
[*OF wf controlled merkle-bound current-query-bound trace-fri-bound*
comp-fri-bound])

have *empty-drift-bound*:
wp-event
(*checked-staged-security-experiment-with-actual-alpha-prefix-data-state A*)
checked-staged-security-with-actual-alpha-prefix-empty-header-relevant-drift-transcript
adversary-initial-state $\leq$
F + (*trace-fri-error* + *low-degree-trace-pair-agreement-error budgets*)
by (*rule*
checked-staged-security-with-actual-alpha-prefix-empty-header-relevant-drift-transcript-bound-from-reachable-trac
[*OF wf controlled trace-fri-reduction fixed-empty-alpha-bound*])

have *empty-trace-fri-bound*:
wp-event (*checked-staged-security-experiment-with-data-state A*)
(*staged-security-with-data-state-verifier-event*
trace-fri-bad-with-empty-composition-header-candidates)
adversary-initial-state $\leq$ *trace-fri-error*
by (*rule*
trace-fri-bad-with-empty-header-candidates-bound-from-reachable-reduction
[*OF trace-fri-reduction*])

have *empty-query-bound*:
wp-event (*checked-staged-security-experiment-with-data-state A*)
(*staged-security-with-data-state-verifier-event*
query-bad-with-empty-composition-header-candidates)
adversary-initial-state $\leq$ *empty-query-error*
by (*rule order-trans*[
OF checked-staged-security-with-data-state-empty-query-bad-le-current-empty-opening-hit-at-zero[
OF wf controlled] *current-empty-bound*])

have *empty-header-bound*:
wp-event (*checked-staged-security-experiment-with-data-state A*)
(*staged-security-with-data-state-verifier-event*
soundness-bad-event-partial-candidate-empty-header-transcript)
adversary-initial-state $\leq$
composition-error-bound +
staged-phase-relation-error size
(*staged-alpha-search-queries budgets 0*) +
hash-collision-budget-value 0
(*staged-alpha-search-queries budgets 0*) +
staged-phase-relation-error size

(staged-alpha-search-queries budgets 0) +
 (F + (trace-fri-error +
 low-degree-trace-pair-agreement-error budgets)) +
 trace-fri-error + empty-query-error
by (rule
 checked-staged-security-with-data-state-empty-header-bad-event-transcript-bound-from-empty-relevant-drift-trans
 [OF false-statement wf controlled empty-drift-bound
 empty-trace-fri-bound empty-query-bound])
have bound:
 checked-staged-adversary-acceptance-probability A ≤
 (hash-collision-budget-value 0
 (staged-attacker-query-budget budgets +
 staged-challenge-query-budget + verifier-hash-query-budget) +
 (∑ i<rounds. C i) + trace-fri-error + composition-fri-error) +
 (composition-error-bound +
 staged-phase-relation-error size
 (staged-alpha-search-queries budgets 0) +
 hash-collision-budget-value 0
 (staged-alpha-search-queries budgets 0) +
 staged-phase-relation-error size
 (staged-alpha-search-queries budgets 0) +
 (F + (trace-fri-error +
 low-degree-trace-pair-agreement-error budgets)) +
 trace-fri-error + empty-query-error)
by (rule
 checked-staged-soundness-from-aligned-transcript-partial-candidate-aligned-randomization-and-empty-header-tran
 [OF false-statement partial-candidate-bound empty-header-bound])
then show ?thesis
by (simp add: algebra-simps)
qed

theorem stark-soundness-from-transcript-indexed-partial-opening-route-from-not-prefix:

fixes N empty-query-error :: prob
and C :: nat ⇒ prob
assumes false-statement: ¬ exists-valid-trace
and wf: staged-budget-wellformed budgets
and controlled: staged-adversary-controlled budgets A
and trace-fri-reduction:
 ∧ data attacker-state.
 Some (data, attacker-state) ∈
 set-dist (execute (checked-staged-transcript-program A)
 adversary-initial-state) ⇒
 trace-fri-reachable-partial-candidate-reduction-assumption
 (verifier-state-from-adversary attacker-state
 (staged-proof-transcript data))
and composition-fri-reduction:
 ∧ data attacker-state.
 Some (data, attacker-state) ∈
 set-dist (execute (checked-staged-transcript-program A)

adversary-initial-state) $\implies$
composition-fri-reachable-partial-candidate-reduction-assumption
(verifier-state-from-adversary attacker-state
(staged-proof-transcript data))
and *round-bound*:
 $\bigwedge i. i < \text{rounds} \implies$
wp-event
(checked-staged-security-experiment-with-query-prefix-data-state A i)
(checked-staged-security-with-query-prefix-current-query-partial-opening-hit-at i)
adversary-initial-state $\leq C i$
and *not-prefix-bound*:
wp-event
(checked-staged-security-experiment-with-actual-alpha-prefix-data-state
A)
(checked-staged-security-with-actual-alpha-prefix-partial-header-candidate-not-prefix-bound
composition-trace-bad-alpha-space)
adversary-initial-state $\leq N$
and *current-empty-bound*:
wp-event (checked-staged-security-experiment-with-data-state A)
(staged-security-with-data-state-current-empty-query-opening-hit-at 0)
adversary-initial-state $\leq \text{empty-query-error}$
shows *checked-staged-adversary-acceptance-probability A* $\leq$
hash-collision-budget-value 0
(staged-attacker-query-budget budgets +
staged-challenge-query-budget + verifier-hash-query-budget) +
 $(\sum i < \text{rounds}. C i) +$
trace-fri-error + composition-fri-error +
 $(\sum i < \text{rounds}. C i) +$
(composition-error-bound +
staged-phase-relation-error size
(staged-alpha-search-queries budgets 0) +
hash-collision-budget-value 0
(staged-alpha-search-queries budgets 0) +
staged-phase-relation-error size
(staged-alpha-search-queries budgets 0) +
 $((\text{composition-error-bound} +$
staged-phase-relation-error size
(staged-alpha-search-queries budgets 0) +
hash-collision-budget-value 0
(staged-alpha-search-queries budgets 0) + N) +
 $(\text{trace-fri-error} +$
low-degree-trace-pair-agreement-error budgets)) +
trace-fri-error + empty-query-error)
proof –
let *?M* =
checked-staged-security-experiment-with-actual-alpha-prefix-data-state A
let *?P* =
composition-error-bound +
staged-phase-relation-error size

(staged-alpha-search-queries budgets 0) +
 hash-collision-budget-value 0
 (staged-alpha-search-queries budgets 0)
have transcript-bound:
 wp-event (checked-staged-transcript-with-alpha-prefix-program A)
 (checked-staged-transcript-actual-alpha-prefix-bad-set-hit
 composition-trace-bad-alpha-space)
 adversary-initial-state $\leq$?P
by (rule
 checked-staged-transcript-with-alpha-prefix-actual-bad-set-hit-bound-from-budgets
 [OF wf controlled])
have prefix-bound:
 wp-event ?M
 (checked-staged-security-with-actual-alpha-prefix-bad-set-hit
 composition-trace-bad-alpha-space)
 adversary-initial-state $\leq$?P
by (rule
 checked-staged-security-with-actual-alpha-prefix-bad-set-hit-bound-from-transcript
 [OF transcript-bound])
have fixed-empty-alpha-bound:
 wp-event ?M
 (checked-staged-security-with-actual-alpha-prefix-verifier-event
 composition-alpha-bad-set-hit-with-empty-composition-header-candidates)
 adversary-initial-state $\leq$?P + N
proof (rule order-trans)
show wp-event ?M
 (checked-staged-security-with-actual-alpha-prefix-verifier-event
 composition-alpha-bad-set-hit-with-empty-composition-header-candidates)
 adversary-initial-state $\leq$
 wp-event ?M
 (checked-staged-security-with-actual-alpha-prefix-bad-set-hit
 composition-trace-bad-alpha-space)
 adversary-initial-state +
 wp-event ?M
 (checked-staged-security-with-actual-alpha-prefix-partial-header-candidate-not-prefix-bound
 composition-trace-bad-alpha-space)
 adversary-initial-state
by (rule
 checked-staged-security-with-actual-alpha-prefix-fixed-empty-alpha-hit-le-prefix-or-not-prefix
 [OF wf controlled])
show ... $\leq$?P + N
by (intro add-mono prefix-bound not-prefix-bound)
qed
show ?thesis
by (rule
 stark-soundness-from-transcript-indexed-partial-opening-route
 [OF false-statement wf controlled trace-fri-reduction
 composition-fri-reduction round-bound fixed-empty-alpha-bound
 current-empty-bound])

qed

theorem *stark-soundness-from-transcript-indexed-partial-opening-route-from-not-prefix-single-query-charge:*

fixes N *empty-query-error* :: prob

and C :: nat $\Rightarrow$ prob

assumes *false-statement*: $\neg$ *exists-valid-trace*

and *wf*: *staged-budget-wellformed budgets*

and *controlled*: *staged-adversary-controlled budgets A*

and *trace-fri-reduction*:

$\bigwedge$ *data attacker-state*.

Some (data, attacker-state) $\in$

set-dist (execute (checked-staged-transcript-program A)

adversary-initial-state) $\Rightarrow$

trace-fri-reachable-partial-candidate-reduction-assumption

(verifier-state-from-adversary attacker-state

(staged-proof-transcript data))

and *composition-fri-reduction*:

$\bigwedge$ *data attacker-state*.

Some (data, attacker-state) $\in$

set-dist (execute (checked-staged-transcript-program A)

adversary-initial-state) $\Rightarrow$

composition-fri-reachable-partial-candidate-reduction-assumption

(verifier-state-from-adversary attacker-state

(staged-proof-transcript data))

and *round-bound*:

$\bigwedge i. i < \text{rounds} \Rightarrow$

wp-event

(checked-staged-security-experiment-with-query-prefix-data-state A i)

(checked-staged-security-with-query-prefix-current-query-partial-opening-hit-at i)

adversary-initial-state $\leq C i$

and *not-prefix-bound*:

wp-event

(checked-staged-security-experiment-with-actual-alpha-prefix-data-state

A)

(checked-staged-security-with-actual-alpha-prefix-partial-header-candidate-not-prefix-bound

composition-trace-bad-alpha-space)

adversary-initial-state $\leq N$

and *current-empty-bound*:

wp-event (checked-staged-security-experiment-with-data-state A)

(staged-security-with-data-state-current-empty-query-opening-hit-at 0)

adversary-initial-state $\leq$ empty-query-error

shows *checked-staged-adversary-acceptance-probability A $\leq$*

hash-collision-budget-value 0

(staged-attacker-query-budget budgets +

staged-challenge-query-budget + verifier-hash-query-budget) +

($\sum i < \text{rounds}. C i$) +

trace-fri-error + composition-fri-error +

(composition-error-bound +

staged-phase-relation-error size

$(\text{staged-alpha-search-queries budgets } 0) +$
 $\text{hash-collision-budget-value } 0$
 $(\text{staged-alpha-search-queries budgets } 0) +$
 $\text{staged-phase-relation-error size}$
 $(\text{staged-alpha-search-queries budgets } 0) +$
 $((\text{composition-error-bound} +$
 $\text{staged-phase-relation-error size}$
 $(\text{staged-alpha-search-queries budgets } 0) +$
 $\text{hash-collision-budget-value } 0$
 $(\text{staged-alpha-search-queries budgets } 0) + N) +$
 $(\text{trace-fri-error} +$
 $\text{low-degree-trace-pair-agreement-error budgets})) +$
 $\text{trace-fri-error} + \text{empty-query-error})$

proof –

let $?M =$
 $\text{checked-staged-security-experiment-with-actual-alpha-prefix-data-state } A$

let $?P =$
 $\text{composition-error-bound} +$
 $\text{staged-phase-relation-error size}$
 $(\text{staged-alpha-search-queries budgets } 0) +$
 $\text{hash-collision-budget-value } 0$
 $(\text{staged-alpha-search-queries budgets } 0)$

have transcript-bound:
 $\text{wp-event } (\text{checked-staged-transcript-with-alpha-prefix-program } A)$
 $(\text{checked-staged-transcript-actual-alpha-prefix-bad-set-hit}$
 $\text{composition-trace-bad-alpha-space})$
 $\text{adversary-initial-state} \leq ?P$

by (rule
 $\text{checked-staged-transcript-with-alpha-prefix-actual-bad-set-hit-bound-from-budgets}$
 $[\text{OF wf controlled}])$

have prefix-bound:
 $\text{wp-event } ?M$
 $(\text{checked-staged-security-with-actual-alpha-prefix-bad-set-hit}$
 $\text{composition-trace-bad-alpha-space})$
 $\text{adversary-initial-state} \leq ?P$

by (rule
 $\text{checked-staged-security-with-actual-alpha-prefix-bad-set-hit-bound-from-transcript}$
 $[\text{OF transcript-bound}])$

have $\text{fixed-empty-alpha-bound:}$
 $\text{wp-event } ?M$
 $(\text{checked-staged-security-with-actual-alpha-prefix-verifier-event}$
 $\text{composition-alpha-bad-set-hit-with-empty-composition-header-candidates})$
 $\text{adversary-initial-state} \leq ?P + N$

proof (rule order-trans)

show $\text{wp-event } ?M$
 $(\text{checked-staged-security-with-actual-alpha-prefix-verifier-event}$
 $\text{composition-alpha-bad-set-hit-with-empty-composition-header-candidates})$
 $\text{adversary-initial-state} \leq$
 $\text{wp-event } ?M$

```

    (checked-staged-security-with-actual-alpha-prefix-bad-set-hit
      composition-trace-bad-alpha-space)
    adversary-initial-state +
wp-event ?M
    (checked-staged-security-with-actual-alpha-prefix-partial-header-candidate-not-prefix-bound
      composition-trace-bad-alpha-space)
    adversary-initial-state
  by (rule
    checked-staged-security-with-actual-alpha-prefix-fixed-empty-alpha-hit-le-prefix-or-not-prefix
      [OF wf controlled])
show ... ≤ ?P + N
  by (intro add-mono prefix-bound not-prefix-bound)
qed
show ?thesis
  by (rule
    stark-soundness-from-transcript-indexed-partial-opening-route-single-query-charge
      [OF false-statement wf controlled trace-fri-reduction
        composition-fri-reduction round-bound fixed-empty-alpha-bound
        current-empty-bound])
qed

theorem stark-soundness-from-transcript-indexed-partial-opening-route-from-uncovered-and-transcript:
  fixes U P empty-query-error :: prob
  and C :: nat ⇒ prob
  assumes false-statement: ¬ exists-valid-trace
  and wf: staged-budget-wellformed budgets
  and controlled: staged-adversary-controlled budgets A
  and trace-fri-reduction:
    ∧ data attacker-state.
      Some (data, attacker-state) ∈
        set-dist (execute (checked-staged-transcript-program A)
          adversary-initial-state) ⇒
        trace-fri-reachable-partial-candidate-reduction-assumption
          (verifier-state-from-adversary attacker-state
            (staged-proof-transcript data))
  and composition-fri-reduction:
    ∧ data attacker-state.
      Some (data, attacker-state) ∈
        set-dist (execute (checked-staged-transcript-program A)
          adversary-initial-state) ⇒
        composition-fri-reachable-partial-candidate-reduction-assumption
          (verifier-state-from-adversary attacker-state
            (staged-proof-transcript data))
  and round-bound:
    ∧ i. i < rounds ⇒
      wp-event
        (checked-staged-security-experiment-with-query-prefix-data-state A i)
        (checked-staged-security-with-query-prefix-current-query-partial-opening-hit-at i)
        adversary-initial-state ≤ C i

```

and *uncovered-bound*:

wp-event
 (*checked-staged-security-experiment-with-actual-alpha-prefix-data-state*
A)
checked-staged-security-with-actual-alpha-prefix-not-prefix-uncovered-index
adversary-initial-state $\leq U$

and *data-pre-bound*:

wp-event (*checked-staged-security-experiment-with-data-state A*)
staged-security-with-data-state-transcript-pre-hit
adversary-initial-state $\leq P$

and *current-empty-bound*:

wp-event (*checked-staged-security-experiment-with-data-state A*)
 (*staged-security-with-data-state-current-empty-query-opening-hit-at 0*)
adversary-initial-state $\leq$ *empty-query-error*

shows *checked-staged-adversary-acceptance-probability A* $\leq$

hash-collision-budget-value 0
 (*staged-attacker-query-budget budgets* +
staged-challenge-query-budget + *verifier-hash-query-budget*) +
 ($\sum i < \text{rounds}. C\ i$) +
trace-fri-error + *composition-fri-error* +
 ($\sum i < \text{rounds}. C\ i$) +
 (*composition-error-bound* +
staged-phase-relation-error size
 (*staged-alpha-search-queries budgets 0*) +
hash-collision-budget-value 0
 (*staged-alpha-search-queries budgets 0*) +
staged-phase-relation-error size
 (*staged-alpha-search-queries budgets 0*) +
 ((*composition-error-bound* +
staged-phase-relation-error size
 (*staged-alpha-search-queries budgets 0*) +
hash-collision-budget-value 0
 (*staged-alpha-search-queries budgets 0*) +
 (*hash-collision-budget-value 0*
 (*staged-alpha-search-queries budgets 0*) + *U* +
 (*P* + *staged-concrete-transcript-target-error-bound*) +
 (*P* + *staged-concrete-transcript-target-error-bound*))) +
 (*trace-fri-error* +
low-degree-trace-pair-agreement-error budgets)) +
trace-fri-error + *empty-query-error*)

proof –

have *transcript-new-bound*:

wp-event (*checked-staged-security-experiment-with-data-state A*)
staged-security-with-data-state-transcript-new-hit
adversary-initial-state $\leq$ *staged-concrete-transcript-target-error-bound*

by (*rule*
checked-staged-security-with-data-state-transcript-new-hit-bound
 [*OF wf controlled*])

have *no-prefix-bound*:

wp-event
 (*checked-staged-security-experiment-with-actual-alpha-prefix-data-state A*)
checked-staged-security-with-actual-alpha-prefix-not-prefix-no-prefix-candidate
adversary-initial-state $\leq$
P + staged-concrete-transcript-target-error-bound
by (rule
checked-staged-security-with-actual-alpha-prefix-not-prefix-no-prefix-candidate-bound-from-sampled-transcript
 [OF *data-pre-bound transcript-new-bound*])
have *path-bound*:
wp-event
 (*checked-staged-security-experiment-with-actual-alpha-prefix-data-state A*)
checked-staged-security-with-actual-alpha-prefix-not-prefix-path-output
adversary-initial-state $\leq$
P + staged-concrete-transcript-target-error-bound
by (rule
checked-staged-security-with-actual-alpha-prefix-not-prefix-path-output-bound-from-sampled-transcript
 [OF *data-pre-bound transcript-new-bound*])
have *collision-bound*:
wp-event
 (*checked-staged-security-experiment-with-actual-alpha-prefix-data-state A*)
checked-staged-security-with-actual-alpha-prefix-not-prefix-collision
adversary-initial-state $\leq$
hash-collision-budget-value 0 (staged-alpha-search-queries budgets 0)
by (rule
checked-staged-security-with-actual-alpha-prefix-not-prefix-collision-bound
 [OF *wf controlled*])
have *not-prefix-bound*:
wp-event
 (*checked-staged-security-experiment-with-actual-alpha-prefix-data-state A*)
 (*checked-staged-security-with-actual-alpha-prefix-partial-header-candidate-not-prefix-bound*
composition-trace-bad-alpha-space)
adversary-initial-state $\leq$
hash-collision-budget-value 0 (staged-alpha-search-queries budgets 0) +
U + (P + staged-concrete-transcript-target-error-bound) +
(P + staged-concrete-transcript-target-error-bound)
by (rule
checked-staged-security-with-actual-alpha-prefix-not-prefix-bound-union-bound
 [OF *wf controlled collision-bound uncovered-bound no-prefix-bound*
path-bound])
show *?thesis*
by (rule
stark-soundness-from-transcript-indexed-partial-opening-route-from-not-prefix
 [OF *false-statement wf controlled trace-fri-reduction*
composition-fri-reduction round-bound not-prefix-bound
current-empty-bound])
qed

theorem *stark-soundness-from-transcript-indexed-partial-opening-route-from-uncovered-and-transcript-single-query-c*

fixes U P *empty-query-error* :: *prob*
and C :: *nat* $\Rightarrow$ *prob*
assumes *false-statement*: $\neg$ *exists-valid-trace*
and *wf*: *staged-budget-wellformed budgets*
and *controlled*: *staged-adversary-controlled budgets A*
and *trace-fri-reduction*:
 $\bigwedge$ *data attacker-state*.
Some (data, attacker-state) $\in$
set-dist (execute (checked-staged-transcript-program A)
adversary-initial-state) $\Rightarrow$
trace-fri-reachable-partial-candidate-reduction-assumption
(verifier-state-from-adversary attacker-state
(staged-proof-transcript data))
and *composition-fri-reduction*:
 $\bigwedge$ *data attacker-state*.
Some (data, attacker-state) $\in$
set-dist (execute (checked-staged-transcript-program A)
adversary-initial-state) $\Rightarrow$
composition-fri-reachable-partial-candidate-reduction-assumption
(verifier-state-from-adversary attacker-state
(staged-proof-transcript data))
and *round-bound*:
 $\bigwedge i. i < \text{rounds} \Rightarrow$
wp-event
(checked-staged-security-experiment-with-query-prefix-data-state A i)
(checked-staged-security-with-query-prefix-current-query-partial-opening-hit-at i)
adversary-initial-state $\leq C$ i
and *uncovered-bound*:
wp-event
(checked-staged-security-experiment-with-actual-alpha-prefix-data-state
A)
checked-staged-security-with-actual-alpha-prefix-not-prefix-uncovered-index
adversary-initial-state $\leq U$
and *data-pre-bound*:
wp-event (checked-staged-security-experiment-with-data-state A)
staged-security-with-data-state-transcript-pre-hit
adversary-initial-state $\leq P$
and *current-empty-bound*:
wp-event (checked-staged-security-experiment-with-data-state A)
(staged-security-with-data-state-current-empty-query-opening-hit-at 0)
adversary-initial-state $\leq$ empty-query-error
shows *checked-staged-adversary-acceptance-probability A $\leq$*
hash-collision-budget-value 0
(staged-attacker-query-budget budgets +
staged-challenge-query-budget + verifier-hash-query-budget) +
 $(\sum i < \text{rounds}. C i) +$
trace-fri-error + composition-fri-error +
(composition-error-bound +
staged-phase-relation-error size

$(\text{staged-alpha-search-queries budgets } 0) +$
 $\text{hash-collision-budget-value } 0$
 $(\text{staged-alpha-search-queries budgets } 0) +$
 $\text{staged-phase-relation-error size}$
 $(\text{staged-alpha-search-queries budgets } 0) +$
 $((\text{composition-error-bound} +$
 $\text{staged-phase-relation-error size}$
 $(\text{staged-alpha-search-queries budgets } 0) +$
 $\text{hash-collision-budget-value } 0$
 $(\text{staged-alpha-search-queries budgets } 0) +$
 $(\text{hash-collision-budget-value } 0$
 $(\text{staged-alpha-search-queries budgets } 0) + U +$
 $(P + \text{staged-concrete-transcript-target-error-bound}) +$
 $(P + \text{staged-concrete-transcript-target-error-bound}))) +$
 $(\text{trace-fri-error} +$
 $\text{low-degree-trace-pair-agreement-error budgets})) +$
 $\text{trace-fri-error} + \text{empty-query-error})$

proof –

have *transcript-new-bound*:

$\text{wp-event } (\text{checked-staged-security-experiment-with-data-state } A)$
 $\text{staged-security-with-data-state-transcript-new-hit}$
 $\text{adversary-initial-state} \leq \text{staged-concrete-transcript-target-error-bound}$

by (rule
 $\text{checked-staged-security-with-data-state-transcript-new-hit-bound}$
 $[\text{OF wf controlled}]$)

have *no-prefix-bound*:

wp-event
 $(\text{checked-staged-security-experiment-with-actual-alpha-prefix-data-state } A)$
 $\text{checked-staged-security-with-actual-alpha-prefix-not-prefix-no-prefix-candidate}$
 $\text{adversary-initial-state} \leq$
 $P + \text{staged-concrete-transcript-target-error-bound}$

by (rule
 $\text{checked-staged-security-with-actual-alpha-prefix-not-prefix-no-prefix-candidate-bound-from-sampled-transcript}$
 $[\text{OF data-pre-bound transcript-new-bound}]$)

have *path-bound*:

wp-event
 $(\text{checked-staged-security-experiment-with-actual-alpha-prefix-data-state } A)$
 $\text{checked-staged-security-with-actual-alpha-prefix-not-prefix-path-output}$
 $\text{adversary-initial-state} \leq$
 $P + \text{staged-concrete-transcript-target-error-bound}$

by (rule
 $\text{checked-staged-security-with-actual-alpha-prefix-not-prefix-path-output-bound-from-sampled-transcript}$
 $[\text{OF data-pre-bound transcript-new-bound}]$)

have *collision-bound*:

wp-event
 $(\text{checked-staged-security-experiment-with-actual-alpha-prefix-data-state } A)$
 $\text{checked-staged-security-with-actual-alpha-prefix-not-prefix-collision}$
 $\text{adversary-initial-state} \leq$

```

    hash-collision-budget-value 0 (staged-alpha-search-queries budgets 0)
  by (rule
    checked-staged-security-with-actual-alpha-prefix-not-prefix-collision-bound
    [OF wf controlled])
  have not-prefix-bound:
    wp-event
    (checked-staged-security-experiment-with-actual-alpha-prefix-data-state A)
    (checked-staged-security-with-actual-alpha-prefix-partial-header-candidate-not-prefix-bound
      composition-trace-bad-alpha-space)
    adversary-initial-state ≤
    hash-collision-budget-value 0 (staged-alpha-search-queries budgets 0) +
    U + (P + staged-concrete-transcript-target-error-bound) +
    (P + staged-concrete-transcript-target-error-bound)
  by (rule
    checked-staged-security-with-actual-alpha-prefix-not-prefix-bound-union-bound
    [OF wf controlled collision-bound uncovered-bound no-prefix-bound
      path-bound])
  show ?thesis
  by (rule
    stark-soundness-from-transcript-indexed-partial-opening-route-from-not-prefix-single-query-charge
    [OF false-statement wf controlled trace-fri-reduction
      composition-fri-reduction round-bound not-prefix-bound
      current-empty-bound])
qed
end
end

```

```

theory Soundness-Not-Prefix-Partial-Opening
imports Soundness-Relevant-Drift-Transcript-Public
begin

```

Same-run partial-opening packaging for the remaining not-prefix residual.

The residual is not a probability-small sparse-cover event by itself. Instead, it records the precise verifier evidence still carried by the current route: an accepting verifier run, authenticated partial trace openings from that same run, and a trace-domain index left unopened by those sampled openings. This layer gives that evidence a non-diagnostic name and keeps the broad witnessed cross/Merkle classifier out of the checked wrapper.

```

context soundness
begin

```

```

definition partial-trace-openings-leave-domain-gap
  :: 'f authenticated-opening list list ⇒ bool
where
  partial-trace-openings-leave-domain-gap trace-openings ⟷
    (∃ j < scale * clength.

```

$\forall i \text{ opn. } i < \text{rounds} \longrightarrow$
 $\text{opn} \in \text{set } (\text{trace-openings} ! i) \longrightarrow$
 $\text{opening-index } \text{opn} \neq j)$

definition *checked-staged-security-with-actual-alpha-prefix-not-prefix-same-run-partial-opening-gap*

$:: ((((((f \times 'f \text{ list} \times 'f \text{ list} \times 'f) \times$
 $\quad 'f \text{ protocol-channel}) \times 'f \text{ staged-proof-data}) \times$
 $\quad 'f \text{ protocol-channel}) \times \text{unit list}) \times$
 $\quad 'f \text{ protocol-channel}) \text{ option} \Rightarrow \text{bool}$

where

checked-staged-security-with-actual-alpha-prefix-not-prefix-same-run-partial-opening-gap
 $\text{out} \longleftrightarrow$
 $(\text{case } \text{out} \text{ of}$
 $\quad \text{None} \Rightarrow \text{False}$
 $\mid \text{Some } (x, -) \Rightarrow$
 $\quad (\text{let } \text{packed} = \text{fst } (\text{fst } x);$
 $\quad \quad \text{prefix-state} = \text{snd } (\text{fst } \text{packed});$
 $\quad \quad \text{data} = \text{snd } \text{packed};$
 $\quad \quad \text{attacker-state} = \text{snd } (\text{fst } x);$
 $\quad \quad s =$
 $\quad \quad \quad \text{verifier-state-from-adversary } \text{attacker-state}$
 $\quad \quad \quad (\text{staged-proof-transcript } \text{data})$
 $\text{in } \exists \text{fr } f\text{-fri-roots } f\text{-final } \text{trace-table } \text{witness-final-state}$
 $\quad \text{trace-openings.}$
 $\quad \text{alpha-header-supported-partial-trace-table-candidate-witness}$
 $\quad \quad s \text{ fr } f\text{-fri-roots } f\text{-final } \text{trace-table } \text{witness-final-state}$
 $\quad \quad \text{trace-openings} \wedge$
 $\quad \text{trace-table} \notin$
 $\quad \quad \text{alpha-prefix-trace-table-candidates } \text{prefix-state } \text{fr} \wedge$
 $\quad \text{prefix-state} \leq \text{witness-final-state} \wedge$
 $\quad \text{partial-trace-openings-leave-domain-gap } \text{trace-openings}))$

lemma *checked-staged-security-with-actual-alpha-prefix-not-prefix-same-run-partial-opening-gap-iff-uncovered-index:*

checked-staged-security-with-actual-alpha-prefix-not-prefix-same-run-partial-opening-gap
 $\text{out} \longleftrightarrow$
checked-staged-security-with-actual-alpha-prefix-not-prefix-uncovered-index
 out

unfolding

checked-staged-security-with-actual-alpha-prefix-not-prefix-same-run-partial-opening-gap-def
checked-staged-security-with-actual-alpha-prefix-not-prefix-uncovered-index-def
partial-trace-openings-leave-domain-gap-def
by (*auto split: option.splits prod.splits*)

definition *checked-staged-security-with-actual-alpha-prefix-not-prefix-sampled-agreement*

$:: ((((((f \times 'f \text{ list} \times 'f \text{ list} \times 'f) \times$
 $\quad 'f \text{ protocol-channel}) \times 'f \text{ staged-proof-data}) \times$
 $\quad 'f \text{ protocol-channel}) \times \text{unit list}) \times$
 $\quad 'f \text{ protocol-channel}) \text{ option} \Rightarrow \text{bool}$

where

checked-staged-security-with-actual-alpha-prefix-not-prefix-sampled-agreement

```

  out  $\longleftrightarrow$ 
  (case out of
    None  $\Rightarrow$  False
  | Some (x, -)  $\Rightarrow$ 
    (let packed = fst (fst x);
      prefix-state = snd (fst packed);
      data = snd packed;
      attacker-state = snd (fst x);
      s =
        verifier-state-from-adversary attacker-state
          (staged-proof-transcript data)
    in  $\exists$  fr f-fri-roots f-final trace-table prefix-trace-table
      witness-final-state trace-openings result query-idxs i.
      alpha-header-supported-partial-trace-table-candidate-witness
        s fr f-fri-roots f-final trace-table witness-final-state
          trace-openings  $\wedge$ 
      trace-table  $\notin$ 
        alpha-prefix-trace-table-candidates prefix-state fr  $\wedge$ 
      prefix-trace-table  $\in$ 
        alpha-prefix-trace-table-candidates prefix-state fr  $\wedge$ 
      prefix-state  $\leq$  witness-final-state  $\wedge$ 
      accepted-with-partial-trace-openings s
        (Some (result, witness-final-state)) fr query-idxs
          trace-openings  $\wedge$ 
      i < rounds  $\wedge$ 
      query-idxs ! i  $\in$ 
        trace-table-agreement-indices trace-table
          prefix-trace-table))

```

definition *checked-staged-security-with-actual-alpha-prefix-not-prefix-low-degree-sampled-agreement*

```

:: 'f staged-adversary  $\Rightarrow$ 
  ((((((('f  $\times$  'f list  $\times$  'f list  $\times$  'f)  $\times$ 
    'f protocol-channel)  $\times$  'f staged-proof-data)  $\times$ 
    'f protocol-channel)  $\times$  unit list)  $\times$ 
    'f protocol-channel) option  $\Rightarrow$  bool

```

where

checked-staged-security-with-actual-alpha-prefix-not-prefix-low-degree-sampled-agreement

```

  A out  $\longleftrightarrow$ 
  (case out of
    None  $\Rightarrow$  False
  | Some (x, -)  $\Rightarrow$ 
    (let packed = fst (fst x);
      prefix-state = snd (fst packed);
      data = snd packed;
      attacker-state = snd (fst x);
      s =
        verifier-state-from-adversary attacker-state
          (staged-proof-transcript data)
    in

```

in $\exists$ *fr* *f-fri-roots* *f-final* *trace-table* *prefix-trace-table*
witness-final-state *trace-openings* *result* *query-idxs* *as* *i*
prefix *prefix-state'* *raw* *raw-state*.
alpha-header-supported-partial-trace-table-candidate-witness
s *fr* *f-fri-roots* *f-final* *trace-table* *witness-final-state*
trace-openings $\wedge$
trace-table $\notin$
alpha-prefix-trace-table-candidates *prefix-state* *fr* $\wedge$
prefix-trace-table $\in$
alpha-prefix-trace-table-candidates *prefix-state* *fr* $\wedge$
trace-table-low-degree *trace-table* $\wedge$
trace-table-low-degree *prefix-trace-table* $\wedge$
trace-table $\neq$ *prefix-trace-table* $\wedge$
accepted-with-partial-trace-openings *s*
(Some (result, witness-final-state)) *fr* *query-idxs*
trace-openings $\wedge$
accepted-transcript-shape *s* *(Some (result, witness-final-state))*
as *query-idxs* $\wedge$
i $<$ *rounds* $\wedge$
Some (data, attacker-state) $\in$
set-dist
(execute (checked-staged-transcript-program *A*)
adversary-initial-state) $\wedge$
Some (((prefix, prefix-state'), raw), raw-state) $\in$
set-dist
(execute (checked-staged-query-prefix-receive-with-state *A* *i*)
adversary-initial-state) $\wedge$
index *(to-nat raw)* $\in$
trace-table-agreement-indices *trace-table*
prefix-trace-table)

definition *checked-staged-security-with-actual-alpha-prefix-not-prefix-low-degree-actual-sampled-agreement*

$:: ((((((f \times f \text{ list} \times f \text{ list} \times f) \times$
 $f \text{ protocol-channel}) \times f \text{ staged-proof-data}) \times$
 $f \text{ protocol-channel}) \times \text{unit list}) \times$
 $f \text{ protocol-channel}) \text{ option} \Rightarrow \text{bool}$

where

checked-staged-security-with-actual-alpha-prefix-not-prefix-low-degree-actual-sampled-agreement
out $\longleftrightarrow$
(case out of
None $\Rightarrow$ *False*
 $|$ *Some (((((alpha-prefix, prefix-state), data), attacker-state),*
result), final-state) $\Rightarrow$
(let *s* $=$
verifier-state-from-adversary *attacker-state*
(staged-proof-transcript *data)*
in $\exists$ *fr* *f-fri-roots* *f-final* *trace-table* *prefix-trace-table*
witness-final-state *trace-openings* *as* *query-idxs* *i*.
alpha-header-supported-partial-trace-table-candidate-witness

```

    s fr f-fri-roots f-final trace-table witness-final-state
    trace-openings  $\wedge$ 
    trace-table  $\notin$ 
    alpha-prefix-trace-table-candidates prefix-state fr  $\wedge$ 
    prefix-trace-table  $\in$ 
    alpha-prefix-trace-table-candidates prefix-state fr  $\wedge$ 
    trace-table-low-degree trace-table  $\wedge$ 
    trace-table-low-degree prefix-trace-table  $\wedge$ 
    trace-table  $\neq$  prefix-trace-table  $\wedge$ 
    accepted-transcript-shape s (Some (result, final-state))
    as query-idxs  $\wedge$ 
    i < rounds  $\wedge$ 
    query-idxs ! i  $\in$ 
    trace-table-agreement-indices trace-table
    prefix-trace-table))

```

lemma *checked-staged-security-with-actual-alpha-prefix-not-prefix-uncovered-index-le-same-run-partial-opening-gap:*

```

wp-event
  (checked-staged-security-experiment-with-actual-alpha-prefix-data-state A)
  checked-staged-security-with-actual-alpha-prefix-not-prefix-uncovered-index
  adversary-initial-state  $\leq$ 
wp-event
  (checked-staged-security-experiment-with-actual-alpha-prefix-data-state A)
  checked-staged-security-with-actual-alpha-prefix-not-prefix-same-run-partial-opening-gap
  adversary-initial-state
by (rule wp-event-mono)
  (simp add:
    checked-staged-security-with-actual-alpha-prefix-not-prefix-same-run-partial-opening-gap-iff-uncovered-index)

```

lemma *checked-staged-security-with-actual-alpha-prefix-same-run-gap-imp-witness-transcript-hit:*

```

assumes
  checked-staged-security-with-actual-alpha-prefix-not-prefix-same-run-partial-opening-gap
  out
shows
  checked-staged-security-with-actual-alpha-prefix-witness-transcript-hit
  out
proof (cases out)
case None
then show ?thesis
  using assms
  unfolding
    checked-staged-security-with-actual-alpha-prefix-not-prefix-same-run-partial-opening-gap-def
  by simp
next
case (Some packed)
then obtain prefix prefix-state data attacker-state result final-state
where out-eq:
  out =
    Some (((((prefix, prefix-state), data), attacker-state), result),

```

```

    final-state)
  by (cases packed, auto split: prod.splits)
let ?s =
  verifier-state-from-adversary attacker-state
  (staged-proof-transcript data)
from assms obtain fr f-fri-roots f-final trace-table
  witness-final-state trace-openings where witness:
  alpha-header-supported-partial-trace-table-candidate-witness ?s fr
  f-fri-roots f-final trace-table witness-final-state trace-openings
unfolding out-eq
  checked-staged-security-with-actual-alpha-prefix-not-prefix-same-run-partial-opening-gap-def
  Let-def
  by (auto split: prod.splits)
from witness obtain result' query-idxs as dg composition-fri-roots final
  rest where outcome:
  Some (result', witness-final-state) ∈
  set-dist (execute verify-monad ?s)
unfolding alpha-header-supported-partial-trace-table-candidate-witness-def
  by blast
have tr-eq: PTranscript ?s = staged-proof-transcript data
  by simp
have transcript-hit:
  hash-map-output-values ?s ∩ set (staged-proof-transcript data)
  ≠ {} ∨
  hash-map-new-output-hit (set (staged-proof-transcript data))
  ?s witness-final-state
  by (rule verify-monad-trace-root-target-preexisting-or-new
    [OF outcome tr-eq])
show ?thesis
  unfolding out-eq
  checked-staged-security-with-actual-alpha-prefix-witness-transcript-hit-def
  Let-def
  using outcome transcript-hit
  by (auto split: prod.splits)
qed

lemma checked-staged-security-with-actual-alpha-prefix-same-run-gap-le-witness-transcript-hit:
  wp-event
  (checked-staged-security-experiment-with-actual-alpha-prefix-data-state A)
  checked-staged-security-with-actual-alpha-prefix-not-prefix-same-run-partial-opening-gap
  adversary-initial-state ≤
  wp-event
  (checked-staged-security-experiment-with-actual-alpha-prefix-data-state A)
  checked-staged-security-with-actual-alpha-prefix-witness-transcript-hit
  adversary-initial-state
  by (rule wp-event-mono)
  (rule
    checked-staged-security-with-actual-alpha-prefix-same-run-gap-imp-witness-transcript-hit)

```


lemma *checked-staged-security-with-actual-alpha-prefix-same-run-gap-bound-from-pre-and-new:*

assumes *pre-bound:*

wp-event

(*checked-staged-security-experiment-with-actual-alpha-prefix-data-state*
A)

checked-staged-security-with-actual-alpha-prefix-witness-transcript-pre-hit

adversary-initial-state $\leq P$

and *new-bound:*

wp-event

(*checked-staged-security-experiment-with-actual-alpha-prefix-data-state*
A)

checked-staged-security-with-actual-alpha-prefix-witness-transcript-new-hit

adversary-initial-state $\leq Q$

shows

wp-event

(*checked-staged-security-experiment-with-actual-alpha-prefix-data-state*
A)

checked-staged-security-with-actual-alpha-prefix-not-prefix-same-run-partial-opening-gap

adversary-initial-state $\leq P + Q$

by (*rule order-trans*

[*OF*

checked-staged-security-with-actual-alpha-prefix-same-run-gap-le-witness-transcript-hit

checked-staged-security-with-actual-alpha-prefix-witness-transcript-hit-bound-from-pre-and-new

[*OF pre-bound new-bound*]])

lemma *checked-staged-security-with-actual-alpha-prefix-same-run-gap-bound-from-data-pre-and-new:*

assumes *data-pre-bound:*

wp-event (*checked-staged-security-experiment-with-data-state A*)

staged-security-with-data-state-transcript-pre-hit

adversary-initial-state $\leq P$

and *new-bound:*

wp-event

(*checked-staged-security-experiment-with-actual-alpha-prefix-data-state*
A)

checked-staged-security-with-actual-alpha-prefix-witness-transcript-new-hit

adversary-initial-state $\leq Q$

shows

wp-event

(*checked-staged-security-experiment-with-actual-alpha-prefix-data-state*
A)

checked-staged-security-with-actual-alpha-prefix-not-prefix-same-run-partial-opening-gap

adversary-initial-state $\leq P + Q$

proof –

have *pre-bound:*

wp-event

(*checked-staged-security-experiment-with-actual-alpha-prefix-data-state A*)

checked-staged-security-with-actual-alpha-prefix-witness-transcript-pre-hit

adversary-initial-state $\leq P$

by (*rule*

```

    checked-staged-security-with-actual-alpha-prefix-witness-transcript-pre-hit-bound-from-data-state-pre-hit
    [OF data-pre-bound])
  show ?thesis
  by (rule
    checked-staged-security-with-actual-alpha-prefix-same-run-gap-bound-from-pre-and-new
    [OF pre-bound new-bound])
qed

lemma checked-staged-security-with-actual-alpha-prefix-witness-transcript-new-hit-bound-from-sampled-transcript:
  assumes pre-bound:
    wp-event (checked-staged-security-experiment-with-data-state A)
    staged-security-with-data-state-transcript-pre-hit
    adversary-initial-state  $\leq P$ 
  and new-bound:
    wp-event (checked-staged-security-experiment-with-data-state A)
    staged-security-with-data-state-transcript-new-hit
    adversary-initial-state  $\leq R$ 
  shows
    wp-event
      (checked-staged-security-experiment-with-actual-alpha-prefix-data-state A)
      checked-staged-security-with-actual-alpha-prefix-witness-transcript-new-hit
      adversary-initial-state  $\leq P + R$ 
proof -
  have event-le:
    wp-event
      (checked-staged-security-experiment-with-actual-alpha-prefix-data-state A)
      checked-staged-security-with-actual-alpha-prefix-witness-transcript-new-hit
      adversary-initial-state  $\leq$ 
    wp-event
      (checked-staged-security-experiment-with-actual-alpha-prefix-data-state A)
      checked-staged-security-with-actual-alpha-prefix-sampled-transcript-hit
      adversary-initial-state
  proof (rule wp-event-mono-on-support)
    fix out
    assume support:
      out  $\in$ 
      set-dist
      (execute
        (checked-staged-security-experiment-with-actual-alpha-prefix-data-state A)
        adversary-initial-state)
    and hit:
      checked-staged-security-with-actual-alpha-prefix-witness-transcript-new-hit
      out
    show
      checked-staged-security-with-actual-alpha-prefix-sampled-transcript-hit
      out
  proof (cases out)
    case None
    then show ?thesis

```

```

    using hit
    unfolding
      checked-staged-security-with-actual-alpha-prefix-witness-transcript-new-hit-def
    by simp
next
case (Some packed)
then obtain prefix prefix-state data attacker-state result final-state
  where out-eq:
    out =
      Some (((((prefix, prefix-state), data), attacker-state), result),
        final-state)
    by (cases packed) (auto split: prod.splits)
show ?thesis
  unfolding out-eq
  by (rule
    checked-staged-security-with-actual-alpha-prefix-some-support-imp-sampled-transcript-hit)
    (rule support[unfolded out-eq])
qed
qed
have sampled-bound:
  wp-event
    (checked-staged-security-experiment-with-actual-alpha-prefix-data-state A)
    checked-staged-security-with-actual-alpha-prefix-sampled-transcript-hit
    adversary-initial-state  $\leq P + R$ 
  by (rule
    checked-staged-security-with-actual-alpha-prefix-sampled-transcript-hit-bound-from-data-state
    [OF pre-bound new-bound])
show ?thesis
  by (rule order-trans[OF event-le sampled-bound])
qed

lemma checked-staged-security-with-actual-alpha-prefix-relevant-partial-header-candidate-drift-bound-from-same-run-
  fixes G P R :: prob
  assumes wf: staged-budget-wellformed budgets
  and controlled: staged-adversary-controlled budgets A
  and gap-bound:
    wp-event
      (checked-staged-security-experiment-with-actual-alpha-prefix-data-state
        A)
      checked-staged-security-with-actual-alpha-prefix-not-prefix-same-run-partial-opening-gap
      adversary-initial-state  $\leq G$ 
  and data-pre-bound:
    wp-event (checked-staged-security-experiment-with-data-state A)
      staged-security-with-data-state-transcript-pre-hit
      adversary-initial-state  $\leq P$ 
  and transcript-new-bound:
    wp-event (checked-staged-security-experiment-with-data-state A)
      staged-security-with-data-state-transcript-new-hit
      adversary-initial-state  $\leq R$ 

```

shows

wp-event
(*checked-staged-security-experiment-with-actual-alpha-prefix-data-state A*)
checked-staged-security-with-actual-alpha-prefix-relevant-partial-header-candidate-drift
adversary-initial-state $\leq$
hash-collision-budget-value 0
(*staged-alpha-search-queries budgets 0*) + G + ($P + R$) + ($P + R$)

proof –

have *collision-bound*:

wp-event
(*checked-staged-security-experiment-with-actual-alpha-prefix-data-state A*)
checked-staged-security-with-actual-alpha-prefix-not-prefix-collision
adversary-initial-state $\leq$
hash-collision-budget-value 0 (*staged-alpha-search-queries budgets 0*)
by (rule
 checked-staged-security-with-actual-alpha-prefix-not-prefix-collision-bound
 [*OF wf controlled*])

have *uncovered-bound*:

wp-event
(*checked-staged-security-experiment-with-actual-alpha-prefix-data-state A*)
checked-staged-security-with-actual-alpha-prefix-not-prefix-uncovered-index
adversary-initial-state $\leq G$
by (rule *order-trans*
 [*OF*
 checked-staged-security-with-actual-alpha-prefix-not-prefix-uncovered-index-le-same-run-partial-opening-gap
 gap-bound])

have *no-prefix-bound*:

wp-event
(*checked-staged-security-experiment-with-actual-alpha-prefix-data-state A*)
checked-staged-security-with-actual-alpha-prefix-not-prefix-no-prefix-candidate
adversary-initial-state $\leq P + R$
by (rule
 checked-staged-security-with-actual-alpha-prefix-not-prefix-no-prefix-candidate-bound-from-sampled-transcript
 [*OF data-pre-bound transcript-new-bound*])

have *path-bound*:

wp-event
(*checked-staged-security-experiment-with-actual-alpha-prefix-data-state A*)
checked-staged-security-with-actual-alpha-prefix-not-prefix-path-output
adversary-initial-state $\leq P + R$
by (rule
 checked-staged-security-with-actual-alpha-prefix-not-prefix-path-output-bound-from-sampled-transcript
 [*OF data-pre-bound transcript-new-bound*])

show *?thesis*

by (rule
 checked-staged-security-with-actual-alpha-prefix-relevant-partial-header-candidate-drift-bound-from-not-prefix-com

[OF wf controlled collision-bound uncovered-bound no-prefix-bound
path-bound])

qed

lemma *checked-staged-security-with-actual-alpha-prefix-relevant-partial-header-candidate-drift-bound-from-same-run*
fixes $P\ Q\ R :: \text{prob}$
assumes wf: *staged-budget-wellformed budgets*
and controlled: *staged-adversary-controlled budgets A*
and data-pre-bound:
 wp-event (*checked-staged-security-experiment-with-data-state A*)
 staged-security-with-data-state-transcript-pre-hit
 adversary-initial-state $\leq P$
and witness-new-bound:
 wp-event
 (*checked-staged-security-experiment-with-actual-alpha-prefix-data-state*
 A)
 checked-staged-security-with-actual-alpha-prefix-witness-transcript-new-hit
 adversary-initial-state $\leq Q$
and transcript-new-bound:
 wp-event (*checked-staged-security-experiment-with-data-state A*)
 staged-security-with-data-state-transcript-new-hit
 adversary-initial-state $\leq R$
shows
 wp-event
 (*checked-staged-security-experiment-with-actual-alpha-prefix-data-state A*)
 checked-staged-security-with-actual-alpha-prefix-relevant-partial-header-candidate-drift
 adversary-initial-state $\leq$
 hash-collision-budget-value 0
 (*staged-alpha-search-queries budgets 0*) + ($P + Q$) + ($P + R$) +
 ($P + R$)
proof –
 have gap-bound:
 wp-event
 (*checked-staged-security-experiment-with-actual-alpha-prefix-data-state*
 A)
 checked-staged-security-with-actual-alpha-prefix-not-prefix-same-run-partial-opening-gap
 adversary-initial-state $\leq P + Q$
 by (rule
 checked-staged-security-with-actual-alpha-prefix-same-run-gap-bound-from-data-pre-and-new
 [OF data-pre-bound witness-new-bound])
 show ?thesis
 by (rule
 checked-staged-security-with-actual-alpha-prefix-relevant-partial-header-candidate-drift-bound-from-same-run-gap
 [OF wf controlled gap-bound data-pre-bound transcript-new-bound])
qed

theorem *stark-soundness-from-aligned-partial-components-current-query-prefix-and-current-empty-query-from-same-run*
fixes $G\ P\ \text{empty-query-error} :: \text{prob}$
and $C :: \text{nat} \Rightarrow \text{prob}$

assumes *false-statement*: $\neg \text{exists-valid-trace}$
and *wf*: *staged-budget-wellformed budgets*
and *controlled*: *staged-adversary-controlled budgets A*
and *trace-fri-reduction*:
 $\bigwedge \text{data attacker-state.}$
 $\text{Some (data, attacker-state)} \in$
 $\text{set-dist (execute (checked-staged-transcript-program A)}$
 $\text{adversary-initial-state)} \implies$
 $\text{trace-fri-reachable-partial-candidate-reduction-assumption}$
 $(\text{verifier-state-from-adversary attacker-state}$
 $(\text{staged-proof-transcript data}))$
and *composition-fri-reduction*:
 $\bigwedge \text{data attacker-state.}$
 $\text{Some (data, attacker-state)} \in$
 $\text{set-dist (execute (checked-staged-transcript-program A)}$
 $\text{adversary-initial-state)} \implies$
 $\text{composition-fri-reachable-partial-candidate-reduction-assumption}$
 $(\text{verifier-state-from-adversary attacker-state}$
 $(\text{staged-proof-transcript data}))$
and *round-bound*:
 $\bigwedge i. i < \text{rounds} \implies$
 wp-event
 $(\text{checked-staged-security-experiment-with-query-prefix-data-state A } i)$
 $(\text{checked-staged-security-with-query-prefix-current-query-partial-opening-hit-at } i)$
 $\text{adversary-initial-state} \leq C \ i$
and *gap-bound*:
 wp-event
 $(\text{checked-staged-security-experiment-with-actual-alpha-prefix-data-state}$
 $A)$
 $\text{checked-staged-security-with-actual-alpha-prefix-not-prefix-same-run-partial-opening-gap}$
 $\text{adversary-initial-state} \leq G$
and *data-pre-bound*:
 $\text{wp-event (checked-staged-security-experiment-with-data-state A)}$
 $\text{staged-security-with-data-state-transcript-pre-hit}$
 $\text{adversary-initial-state} \leq P$
and *current-empty-bound*:
 $\text{wp-event (checked-staged-security-experiment-with-data-state A)}$
 $(\text{staged-security-with-data-state-current-empty-query-opening-hit-at } 0)$
 $\text{adversary-initial-state} \leq \text{empty-query-error}$
shows *checked-staged-adversary-acceptance-probability A* $\leq$
 $\text{hash-collision-budget-value } 0$
 $(\text{staged-attacker-query-budget budgets} +$
 $\text{staged-challenge-query-budget} + \text{verifier-hash-query-budget}) +$
 $(\sum i < \text{rounds. } C \ i) +$
 $\text{trace-fri-error} + \text{composition-fri-error} +$
 $(\sum i < \text{rounds. } C \ i) +$
 $(\text{composition-error-bound} +$
 $\text{staged-phase-relation-error size}$
 $(\text{staged-alpha-search-queries budgets } 0) +$

hash-collision-budget-value 0
(staged-alpha-search-queries budgets 0) +
(hash-collision-budget-value 0
(staged-alpha-search-queries budgets 0) + G +
(P + staged-concrete-transcript-target-error-bound) +
(P + staged-concrete-transcript-target-error-bound)) +
staged-phase-relation-error size
(staged-alpha-search-queries budgets 0) +
trace-fri-error + empty-query-error)

proof –

have *trace-fri-bound*:

wp-event (checked-staged-security-experiment-with-data-state A)
(staged-security-with-data-state-verifier-event
trace-fri-bad-with-partial-candidates)
adversary-initial-state ≤ trace-fri-error

by (rule
trace-fri-bad-with-partial-candidates-bound-from-reachable-reduction
[OF trace-fri-reduction])

have *comp-fri-bound*:

wp-event (checked-staged-security-experiment-with-data-state A)
(staged-security-with-data-state-verifier-event
composition-fri-bad-with-partial-candidates)
adversary-initial-state ≤ composition-fri-error

by (rule
composition-fri-bad-with-partial-candidates-bound-from-reachable-reduction
[OF composition-fri-reduction])

have *empty-trace-fri-bound*:

wp-event (checked-staged-security-experiment-with-data-state A)
(staged-security-with-data-state-verifier-event
trace-fri-bad-with-empty-composition-header-candidates)
adversary-initial-state ≤ trace-fri-error

by (rule
trace-fri-bad-with-empty-header-candidates-bound-from-reachable-reduction
[OF trace-fri-reduction])

have *transcript-new-bound*:

wp-event (checked-staged-security-experiment-with-data-state A)
staged-security-with-data-state-transcript-new-hit
adversary-initial-state ≤ staged-concrete-transcript-target-error-bound

by (rule *checked-staged-security-with-data-state-transcript-new-hit-bound*
[OF wf controlled])

have *drift-bound*:

wp-event
(checked-staged-security-experiment-with-actual-alpha-prefix-data-state A)
checked-staged-security-with-actual-alpha-prefix-relevant-partial-header-candidate-drift
adversary-initial-state ≤
hash-collision-budget-value 0
(staged-alpha-search-queries budgets 0) + G +
(P + staged-concrete-transcript-target-error-bound) +
(P + staged-concrete-transcript-target-error-bound)

```

by (rule
  checked-staged-security-with-actual-alpha-prefix-relevant-partial-header-candidate-drift-bound-from-same-run-gap
  [OF wf controlled gap-bound data-pre-bound transcript-new-bound])
have base:
  checked-staged-adversary-acceptance-probability  $A \leq$ 
    hash-collision-budget-value 0
    (staged-attacker-query-budget budgets +
     staged-challenge-query-budget + verifier-hash-query-budget) +
    ( $\sum i < \text{rounds}. C\ i$ ) +
    trace-fri-error + composition-fri-error +
    ( $\sum i < \text{rounds}. C\ i$ ) +
    (composition-error-bound +
     staged-phase-relation-error size
     (staged-alpha-search-queries budgets 0) +
     hash-collision-budget-value 0
     (staged-alpha-search-queries budgets 0) +
     staged-phase-relation-error size
     (staged-alpha-search-queries budgets 0) +
     (hash-collision-budget-value 0
      (staged-alpha-search-queries budgets 0) +  $G$  +
      ( $P$  + staged-concrete-transcript-target-error-bound) +
      ( $P$  + staged-concrete-transcript-target-error-bound))) +
     trace-fri-error + empty-query-error)
by (rule
  checked-staged-soundness-from-aligned-transcript-aligned-randomization-components-and-current-query-prefix-ro
  [OF false-statement wf controlled trace-fri-bound comp-fri-bound
   round-bound drift-bound empty-trace-fri-bound current-empty-bound])
then show ?thesis
  by (simp add: algebra-simps)
qed

lemma powers-scaled-domain-bound-imp-query-sample-space:
  assumes bounded:
     $\bigwedge j. j \in \text{set } (\text{powers-scaled } \text{idx}) \implies j < \text{clength} * \text{scale}$ 
  shows  $\text{idx} \in \text{query-sample-space}$ 
proof -
  let ?m = Max (set [0.. $\text{powers}$ ])
  have m-in: ?m  $\in$  set [0.. $\text{powers}$ ]
  using powers-pos by (intro Max-in) simp-all
  have m-eq: ?m = powers - 1
  using powers-pos by (intro Max-eqI) auto
  have idx-m-bound:  $\text{idx} + ?m * \text{scale} < \text{clength} * \text{scale}$ 
  by (rule bounded) (use m-in in  $\langle$ auto simp: powers-scaled-def $\rangle$ )
  have m-le: ?m * scale  $\leq$  clength * scale
proof -
  have ?m < clength
  using m-eq powers-le-clength powers-pos by simp
then show ?thesis
  by (simp add: mult-le-mono)

```



```

qed
have  $idx < clength * scale - ?m * scale$ 
  using  $idx\text{-}m\text{-}bound\ m\text{-}le$  by linarith
then show  $?thesis$ 
  unfolding query-sample-space-def query-sample-space-size-def by simp
qed

lemma partial-authenticated-trace-openings-query-index-in-sample-space:
  assumes indices:
     $map\ opening\text{-}index\ openings = powers\text{-}scaled\ idx$ 
  and auth:
     $partial\text{-}authenticated\text{-}table\ fr\ (scale * clength)\ openings\ s$ 
  shows  $idx \in query\text{-}sample\text{-}space$ 
proof (rule powers-scaled-domain-bound-imp-query-sample-space)
  fix  $j$ 
  assume  $j\text{-}in$ :  $j \in set\ (powers\text{-}scaled\ idx)$ 
  have  $j \in opening\text{-}index\ \text{'}\ set\ openings$ 
    by (metis indices j-in set-map)
  then obtain opn where  $opn\text{-}in$ :  $opn \in set\ openings$ 
    and  $j\text{-}eq$ :  $j = opening\text{-}index\ opn$ 
    by blast
  have  $opening\text{-}index\ opn < opening\text{-}length\ opn$ 
    using auth opn-in unfolding partial-authenticated-table-def
      authenticated-opening-in-def by blast
  moreover have  $opening\text{-}length\ opn = scale * clength$ 
    using auth opn-in unfolding partial-authenticated-table-def by blast
  ultimately show  $j < clength * scale$ 
    using  $j\text{-}eq$  by (simp add: mult.commute)
qed

lemma accepted-partial-trace-openings-leave-domain-gap-if-sparse-queries:
  assumes partial:
     $accepted\text{-}with\text{-}partial\text{-}trace\text{-}openings\ s\ out\ fr\ query\text{-}idxs\ trace\text{-}openings$ 
  and sparse:  $rounds * powers < scale * clength$ 
  shows partial-trace-openings-leave-domain-gap trace-openings
proof -
  have  $rounds\text{-}len$ :  $length\ trace\text{-}openings = rounds$ 
    by (rule accepted-with-partial-trace-openings-shapes(2)[OF partial])
  have concat-sparse:
     $length\ (List.concat\ trace\text{-}openings) < scale * clength$ 
    using accepted-with-partial-trace-openings-concat-length[OF partial]
      sparse by simp
  obtain  $j$  where  $j\text{-}bound$ :  $j < scale * clength$ 
    and unopened:
       $\bigwedge i\ opn. i < rounds \implies$ 
         $opn \in set\ (trace\text{-}openings\ !\ i) \implies$ 
         $opening\text{-}index\ opn \neq j$ 
    using sparse-openings-obtain-unopened-index[OF rounds-len concat-sparse]
      by blast

```

```

show ?thesis
  unfolding partial-trace-openings-leave-domain-gap-def
  using j-bound unopened by blast
qed

lemma checked-staged-security-with-actual-alpha-prefix-not-prefix-same-run-partial-opening-gap-if-sparse-queries:
  assumes wf: staged-budget-wellformed budgets
  and controlled: staged-adversary-controlled budgets A
  and sparse: rounds * powers < scale * clength
  and support:
    out ∈
      set-dist
      (execute
        (checked-staged-security-experiment-with-actual-alpha-prefix-data-state
          A)
        adversary-initial-state)
  and hit:
    checked-staged-security-with-actual-alpha-prefix-partial-header-candidate-not-prefix-bound
    bad-sets out
  shows
    checked-staged-security-with-actual-alpha-prefix-not-prefix-collision out ∨
    checked-staged-security-with-actual-alpha-prefix-not-prefix-no-prefix-candidate out ∨
    checked-staged-security-with-actual-alpha-prefix-not-prefix-path-output out ∨
    checked-staged-security-with-actual-alpha-prefix-not-prefix-same-run-partial-opening-gap out
  proof (cases out)
  case None
  then show ?thesis
    using hit
    unfolding
      checked-staged-security-with-actual-alpha-prefix-partial-header-candidate-not-prefix-bound-def
    by simp
  next
  case (Some packed)
  then obtain prefix prefix-state data attacker-state result final-state
  where out-eq:
    out =
      Some (((((prefix, prefix-state), data), attacker-state), result),
        final-state)
  by (cases packed, auto split: prod.splits)
  let ?s =
    verifier-state-from-adversary attacker-state
    (staged-proof-transcript data)
  from hit obtain fr f-fri-roots f-final as where not-prefix:
    alpha-header-partial-candidate-not-prefix-bound prefix-state ?s fr
    f-fri-roots f-final
  unfolding out-eq
    checked-staged-security-with-actual-alpha-prefix-partial-header-candidate-not-prefix-bound-def
    Let-def
  by (auto split: option.splits prod.splits)

```

```

from checked-staged-security-with-actual-alpha-prefix-not-prefix-bound-obtain-witness-final-ext
  [OF wf controlled support[unfolded out-eq] not-prefix]
obtain trace-table witness-final-state trace-openings where witness:
  alpha-header-supported-partial-trace-table-candidate-witness ?s fr
  f-fri-roots f-final trace-table witness-final-state trace-openings
and table-not-prefix:
  trace-table ∉ alpha-prefix-trace-table-candidates prefix-state fr
and prefix-ext: prefix-state ≤ witness-final-state
by blast
from witness obtain result' query-idxs as' dg composition-fri-roots final rest
where partial:
  accepted-with-partial-trace-openings ?s
  (Some (result', witness-final-state)) fr query-idxs trace-openings
unfolding alpha-header-supported-partial-trace-table-candidate-witness-def
by blast
have gap:
  partial-trace-openings-leave-domain-gap trace-openings
by (rule accepted-partial-trace-openings-leave-domain-gap-if-sparse-queries
  [OF partial sparse])
have
  checked-staged-security-with-actual-alpha-prefix-not-prefix-same-run-partial-opening-gap
  out
unfolding out-eq
  checked-staged-security-with-actual-alpha-prefix-not-prefix-same-run-partial-opening-gap-def
  Let-def
using witness table-not-prefix prefix-ext gap
by (simp split: prod.splits, blast)
then show ?thesis by simp
qed

lemma alpha-header-witness-prefix-candidate-values-agree-on-openings:
assumes prefix-candidate:
  prefix-trace-table ∈
  alpha-prefix-trace-table-candidates prefix-state fr
and witness:
  alpha-header-supported-partial-trace-table-candidate-witness s fr
  f-fri-roots f-final trace-table final-state trace-openings
and pullback:
  partial-authenticated-table fr (scale * clength)
  (trace-openings ! i) prefix-state
and clean-prefix: ¬ hash-map-output-collision prefix-state
and i-bound: i < rounds
shows
  map (! trace-table) (map opening-index (trace-openings ! i)) =
  map (! prefix-trace-table) (map opening-index (trace-openings ! i))
proof –
have prefix-len:
  length prefix-trace-table = clength * scale
using prefix-candidate

```

```

  unfolding alpha-prefix-trace-table-candidates-def by simp
have prefix-bind:
  merkle-root-binds-table fr prefix-trace-table prefix-state
  using prefix-candidate
  unfolding alpha-prefix-trace-table-candidates-def by simp
from witness obtain result query-idxs as dg composition-fri-roots final rest
  where trace-cand:
    partial-trace-table-candidate trace-table trace-openings
  unfolding alpha-header-supported-partial-trace-table-candidate-witness-def
  by blast
have trace-agrees:
  table-agrees-with-authenticated-openings trace-table
    (scale * clength) (trace-openings ! i)
  using trace-cand i-bound
  unfolding partial-trace-table-candidate-def by blast
have point-eq:
   $\bigwedge \text{opn. } \text{opn} \in \text{set } (\text{trace-openings} ! i) \implies$ 
    trace-table ! opening-index opn =
    prefix-trace-table ! opening-index opn
proof -
  fix opn
  assume opn-in: opn  $\in$  set (trace-openings ! i)
  have trace-value:
    trace-table ! opening-index opn = opening-value opn
  using trace-agrees opn-in
  unfolding table-agrees-with-authenticated-openings-def by blast
have root: opening-root opn = fr
  using pullback opn-in unfolding partial-authenticated-table-def
  by blast
have len: opening-length opn = length prefix-trace-table
  using pullback opn-in prefix-len
  unfolding partial-authenticated-table-def by simp
have auth: authenticated-opening-in prefix-state opn
  using pullback opn-in unfolding partial-authenticated-table-def
  by blast
obtain n where len-pow: clength * scale =  $2^n$ 
  using eval-domain-length-power by blast
have table-pow: length prefix-trace-table =  $2^n$ 
  using prefix-len len-pow by (simp add: mult.commute)
have prefix-value:
  prefix-trace-table ! opening-index opn = opening-value opn
  using
    merkle-bound-table-agrees-with-authenticated-opening-if-clean
    [OF prefix-bind table-pow auth root len clean-prefix]
  by simp
show
  trace-table ! opening-index opn =
  prefix-trace-table ! opening-index opn
  using trace-value prefix-value by simp

```

```

qed
show ?thesis
  by (rule nth-equalityI)
    (use point-eq in auto)
qed

```

lemma *alpha-header-witness-prefix-candidate-trace-table-agreement-index:*

assumes *prefix-candidate:*

prefix-trace-table $\in$
alpha-prefix-trace-table-candidates *prefix-state* *fr*

and *witness:*

alpha-header-supported-partial-trace-table-candidate-witness *s* *fr*
f-fri-roots *f-final* *trace-table* *final-state* *trace-openings*

and *pullback:*

partial-authenticated-table *fr* (*scale* * *clength*)
(*trace-openings* ! *i*) *prefix-state*

and *clean-prefix:* $\neg$ *hash-map-output-collision* *prefix-state*

and *i-bound:* *i* < *rounds*

and *idx-sample:* *idx* $\in$ *query-sample-space*

and *idx-openings:*

map *opening-index* (*trace-openings* ! *i*) = *powers-scaled* *idx*

shows *idx* $\in$ *trace-table-agreement-indices* *trace-table* *prefix-trace-table*

proof –

have *agree:*

map ((!) *trace-table*) (*map* *opening-index* (*trace-openings* ! *i*)) =
map ((!) *prefix-trace-table*) (*map* *opening-index* (*trace-openings* ! *i*))

by (rule *alpha-header-witness-prefix-candidate-values-agree-on-openings*
[*OF* *prefix-candidate* *witness* *pullback* *clean-prefix* *i-bound*])

show ?thesis

unfolding *trace-table-agreement-indices-def*

using *idx-sample* *agree* *idx-openings* **by** *simp*

qed

lemma *checked-staged-security-with-actual-alpha-prefix-not-prefix-bound-imp-sampled-agreement-on-support:*

assumes *wf:* *staged-budget-wellformed* *budgets*

and *controlled:* *staged-adversary-controlled* *budgets* *A*

and *support:*

out $\in$
set-dist
(*execute*
(*checked-staged-security-experiment-with-actual-alpha-prefix-data-state*
A)
adversary-initial-state)

and *hit:*

checked-staged-security-with-actual-alpha-prefix-partial-header-candidate-not-prefix-bound
bad-sets *out*

shows

checked-staged-security-with-actual-alpha-prefix-not-prefix-collision *out* $\vee$
checked-staged-security-with-actual-alpha-prefix-not-prefix-no-prefix-candidate *out* $\vee$

```

    checked-staged-security-with-actual-alpha-prefix-not-prefix-path-output out ∨
    checked-staged-security-with-actual-alpha-prefix-not-prefix-sampled-agreement out
proof (cases out)
  case None
  then show ?thesis
    using hit
    unfolding
      checked-staged-security-with-actual-alpha-prefix-partial-header-candidate-not-prefix-bound-def
    by simp
next
  case (Some packed)
  then obtain prefix prefix-state data attacker-state result final-state
    where out-eq:
      out =
        Some (((((prefix, prefix-state), data), attacker-state), result),
              final-state)
    by (cases packed, auto split: prod.splits)
  let ?s =
    verifier-state-from-adversary attacker-state
      (staged-proof-transcript data)
  from hit obtain fr f-fri-roots f-final as where not-prefix-bound:
    alpha-header-partial-candidate-not-prefix-bound prefix-state ?s fr
    f-fri-roots f-final
  unfolding out-eq
    checked-staged-security-with-actual-alpha-prefix-partial-header-candidate-not-prefix-bound-def
    Let-def
  by (auto split: option.splits prod.splits)
from checked-staged-security-with-actual-alpha-prefix-not-prefix-bound-obtain-witness-final-ext
  [OF wf controlled support[unfolded out-eq] not-prefix-bound]
obtain trace-table witness-final-state trace-openings where witness:
  alpha-header-supported-partial-trace-table-candidate-witness ?s fr
  f-fri-roots f-final trace-table witness-final-state trace-openings
and table-not-prefix:
  trace-table ∉ alpha-prefix-trace-table-candidates prefix-state fr
and prefix-ext: prefix-state ≤ witness-final-state
by blast
show ?thesis
proof (cases hash-map-output-collision prefix-state)
  case True
  then show ?thesis
    unfolding out-eq
      checked-staged-security-with-actual-alpha-prefix-not-prefix-collision-def
    by simp
next
  case clean-prefix: False
  show ?thesis
  proof (cases alpha-prefix-trace-table-candidates prefix-state fr = {})
    case True
    then have

```

```

checked-staged-security-with-actual-alpha-prefix-not-prefix-no-prefix-candidate
  out
unfolding out-eq
  checked-staged-security-with-actual-alpha-prefix-not-prefix-no-prefix-candidate-def
  Let-def
  using witness table-not-prefix prefix-ext by (simp, blast)
then show ?thesis by simp
next
case nonempty: False
then obtain prefix-trace-table where prefix-candidate:
  prefix-trace-table ∈
  alpha-prefix-trace-table-candidates prefix-state fr
by blast
have pullback-or-new:
  (∀ i < rounds.
    partial-authenticated-table fr (scale * clength)
    (trace-openings ! i) prefix-state) ∨
  (∃ i < rounds. ∃ opn ∈ set (trace-openings ! i).
    hash-map-new-output-hit
    (merkle-path-target-roots witness-final-state fr
      (scale * clength) (opening-index opn)
      (opening-value opn) (opening-path opn))
    prefix-state witness-final-state)
by (rule alpha-header-witness-openings-pullback-or-new-output-hit
  [OF prefix-ext witness])
from pullback-or-new show ?thesis
proof
assume pullback-all:
  ∀ i < rounds.
    partial-authenticated-table fr (scale * clength)
    (trace-openings ! i) prefix-state
from witness obtain witness-result query-idxs as dg
  composition-fri-roots final rest where partial:
  accepted-with-partial-trace-openings ?s
  (Some (witness-result, witness-final-state)) fr query-idxs
  trace-openings
unfolding alpha-header-supported-partial-trace-table-candidate-witness-def
by blast
have i-bound: 0 < rounds
by (rule rounds-positive)
have idx-openings:
  map opening-index (trace-openings ! 0) =
  powers-scaled (query-idxs ! 0)
by (rule accepted-with-partial-trace-openings-shapes(3)
  [OF partial i-bound])
have pullback0:
  partial-authenticated-table fr (scale * clength)
  (trace-openings ! 0) prefix-state
using pullback-all i-bound by blast

```

```

have idx-sample: query-idxs !  $0 \in \text{query-sample-space}$ 
  by (rule partial-authenticated-trace-openings-query-index-in-sample-space
    [OF idx-openings pullback0])
have agreement:
  query-idxs !  $0 \in$ 
    trace-table-agreement-indices trace-table prefix-trace-table
  by (rule
    alpha-header-witness-prefix-candidate-trace-table-agreement-index
    [OF prefix-candidate witness pullback0 clean-prefix i-bound
      idx-sample idx-openings])
have
  checked-staged-security-with-actual-alpha-prefix-not-prefix-sampled-agreement
    out
  unfolding out-eq
    checked-staged-security-with-actual-alpha-prefix-not-prefix-sampled-agreement-def
    Let-def
  using witness table-not-prefix prefix-candidate prefix-ext partial
    i-bound agreement
  by (simp split: prod.splits, blast)
  then show ?thesis by simp
next
  assume new:
     $\exists i < \text{rounds}. \exists \text{opn} \in \text{set} (\text{trace-openings} ! i).$ 
    hash-map-new-output-hit
    (merkle-path-target-roots witness-final-state fr
      (scale * clength) (opening-index opn)
      (opening-value opn) (opening-path opn))
    prefix-state witness-final-state
  have
    checked-staged-security-with-actual-alpha-prefix-not-prefix-path-output
      out
    unfolding out-eq
      checked-staged-security-with-actual-alpha-prefix-not-prefix-path-output-def
      Let-def
    using witness table-not-prefix prefix-ext new
    by (simp split: prod.splits, blast)
    then show ?thesis by simp
  qed
qed
qed
qed

lemma checked-staged-security-with-actual-alpha-prefix-not-prefix-bound-sampled-agreement-union-bound:
  fixes C S N P :: prob
  assumes wf: staged-budget-wellformed budgets
  and controlled: staged-adversary-controlled budgets A
  and collision-bound:
    wp-event
    (checked-staged-security-experiment-with-actual-alpha-prefix-data-state

```



```

    A)
    checked-staged-security-with-actual-alpha-prefix-not-prefix-collision
    adversary-initial-state  $\leq C$ 
and sampled-agreement-bound:
    wp-event
    (checked-staged-security-experiment-with-actual-alpha-prefix-data-state
    A)
    checked-staged-security-with-actual-alpha-prefix-not-prefix-sampled-agreement
    adversary-initial-state  $\leq S$ 
and no-prefix-bound:
    wp-event
    (checked-staged-security-experiment-with-actual-alpha-prefix-data-state
    A)
    checked-staged-security-with-actual-alpha-prefix-not-prefix-no-prefix-candidate
    adversary-initial-state  $\leq N$ 
and path-bound:
    wp-event
    (checked-staged-security-experiment-with-actual-alpha-prefix-data-state
    A)
    checked-staged-security-with-actual-alpha-prefix-not-prefix-path-output
    adversary-initial-state  $\leq P$ 
shows
    wp-event
    (checked-staged-security-experiment-with-actual-alpha-prefix-data-state
    A)
    (checked-staged-security-with-actual-alpha-prefix-partial-header-candidate-not-prefix-bound
    bad-sets)
    adversary-initial-state  $\leq C + S + N + P$ 
proof –
    let ?M =
    checked-staged-security-experiment-with-actual-alpha-prefix-data-state A
    let ?B =
    checked-staged-security-with-actual-alpha-prefix-partial-header-candidate-not-prefix-bound
    bad-sets
    let ?C = checked-staged-security-with-actual-alpha-prefix-not-prefix-collision
    let ?S =
    checked-staged-security-with-actual-alpha-prefix-not-prefix-sampled-agreement
    let ?N =
    checked-staged-security-with-actual-alpha-prefix-not-prefix-no-prefix-candidate
    let ?P = checked-staged-security-with-actual-alpha-prefix-not-prefix-path-output
    have wp-event ?M ?B adversary-initial-state  $\leq$ 
    wp-event ?M ( $\lambda out. ?C out \vee ?S out \vee ?N out \vee ?P out$ )
    adversary-initial-state
    proof (rule wp-event-mono-on-support)
    fix out
    assume support:
    out  $\in$  set-dist (execute ?M adversary-initial-state)
    and hit: ?B out
    from checked-staged-security-with-actual-alpha-prefix-not-prefix-bound-imp-sampled-agreement-on-support

```

```

    [OF wf controlled support hit]
  show ?C out  $\vee$  ?S out  $\vee$  ?N out  $\vee$  ?P out
  by blast
qed
also have ...  $\leq$ 
  wp-event ?M ?C adversary-initial-state +
  wp-event ?M ?S adversary-initial-state +
  wp-event ?M ?N adversary-initial-state +
  wp-event ?M ?P adversary-initial-state
  by (rule wp-event-union-bound4)
also have ...  $\leq C + S + N + P$ 
  by (intro add-mono collision-bound sampled-agreement-bound
    no-prefix-bound path-bound)
finally show ?thesis .
qed

lemma checked-staged-security-with-actual-alpha-prefix-not-prefix-sampled-agreement-imp-witness-transcript-hit:
  assumes
    checked-staged-security-with-actual-alpha-prefix-not-prefix-sampled-agreement
    out
  shows
    checked-staged-security-with-actual-alpha-prefix-witness-transcript-hit
    out
proof (cases out)
case None
  then show ?thesis
  using assms
  unfolding
    checked-staged-security-with-actual-alpha-prefix-not-prefix-sampled-agreement-def
  by simp
next
case (Some packed)
  then obtain prefix prefix-state data attacker-state result final-state
  where out-eq:
    out =
      Some (((((prefix, prefix-state), data), attacker-state), result),
        final-state)
  by (cases packed, auto split: prod.splits)
  let ?s =
    verifier-state-from-adversary attacker-state
    (staged-proof-transcript data)
  from assms obtain fr f-fri-roots f-final trace-table prefix-trace-table
    witness-final-state trace-openings witness-result query-idxs i where
    witness:
      alpha-header-supported-partial-trace-table-candidate-witness ?s fr
      f-fri-roots f-final trace-table witness-final-state trace-openings
  unfolding out-eq
    checked-staged-security-with-actual-alpha-prefix-not-prefix-sampled-agreement-def
    Let-def

```

```

    by (auto split: prod.splits)
  from witness obtain result' query-idxs' as dg composition-fri-roots final
    rest where outcome:
      Some (result', witness-final-state) ∈
        set-dist (execute verify-monad ?s)
  unfolding alpha-header-supported-partial-trace-table-candidate-witness-def
  by blast
  have tr-eq: PTranscript ?s = staged-proof-transcript data
  by simp
  have transcript-hit:
    hash-map-output-values ?s ∩ set (staged-proof-transcript data)
      ≠ {} ∨
    hash-map-new-output-hit (set (staged-proof-transcript data))
      ?s witness-final-state
  by (rule verify-monad-trace-root-target-preexisting-or-new
    [OF outcome tr-eq])
  show ?thesis
  unfolding out-eq
    checked-staged-security-with-actual-alpha-prefix-witness-transcript-hit-def
    Let-def
  using outcome transcript-hit
  by (auto split: prod.splits)
qed

lemma checked-staged-security-with-actual-alpha-prefix-not-prefix-sampled-agreement-le-witness-transcript-hit:
  wp-event
    (checked-staged-security-experiment-with-actual-alpha-prefix-data-state A)
    checked-staged-security-with-actual-alpha-prefix-not-prefix-sampled-agreement
    adversary-initial-state ≤
  wp-event
    (checked-staged-security-experiment-with-actual-alpha-prefix-data-state A)
    checked-staged-security-with-actual-alpha-prefix-witness-transcript-hit
    adversary-initial-state
  by (rule wp-event-mono)
    (rule
      checked-staged-security-with-actual-alpha-prefix-not-prefix-sampled-agreement-imp-witness-transcript-hit)

lemma checked-staged-security-with-actual-alpha-prefix-not-prefix-sampled-agreement-bound-from-pre-and-new:
  assumes pre-bound:
    wp-event
      (checked-staged-security-experiment-with-actual-alpha-prefix-data-state
        A)
      checked-staged-security-with-actual-alpha-prefix-witness-transcript-pre-hit
      adversary-initial-state ≤ P
  and new-bound:
    wp-event
      (checked-staged-security-experiment-with-actual-alpha-prefix-data-state
        A)
      checked-staged-security-with-actual-alpha-prefix-witness-transcript-new-hit

```

$adversary-initial-state \leq Q$
shows
wp-event
 (*checked-staged-security-experiment-with-actual-alpha-prefix-data-state*
 A)
checked-staged-security-with-actual-alpha-prefix-not-prefix-sampled-agreement
 $adversary-initial-state \leq P + Q$
by (*rule order-trans*
 [*OF*
 checked-staged-security-with-actual-alpha-prefix-not-prefix-sampled-agreement-le-witness-transcript-hit
 checked-staged-security-with-actual-alpha-prefix-witness-transcript-hit-bound-from-pre-and-new
 [*OF pre-bound new-bound*]])

lemma *checked-staged-security-with-actual-alpha-prefix-not-prefix-bound-imp-sampled-transcript-hit-on-support*:
assumes *support*:
 out $\in$
 set-dist
 (*execute*
 (*checked-staged-security-experiment-with-actual-alpha-prefix-data-state*
 A)
 adversary-initial-state)
and *hit*:
 checked-staged-security-with-actual-alpha-prefix-partial-header-candidate-not-prefix-bound
 bad-sets out
shows
 checked-staged-security-with-actual-alpha-prefix-sampled-transcript-hit
 out
proof (*cases out*)
case *None*
then show *?thesis*
 using *hit*
 unfolding
 checked-staged-security-with-actual-alpha-prefix-partial-header-candidate-not-prefix-bound-def
 by *simp*
next
case (*Some packed*)
then obtain *prefix prefix-state data attacker-state result final-state*
where *out-eq*:
 out =
 Some (((((*prefix, prefix-state*), *data*), *attacker-state*), *result*),
 final-state)
 by (*cases packed, auto split: prod.splits*)
let *?s* =
 verifier-state-from-adversary attacker-state
 (*staged-proof-transcript data*)
have *data-support*:
 Some (((*data, attacker-state*), *result*), *final-state*) $\in$
 set-dist
 (*execute* (*checked-staged-security-experiment-with-data-state A*)

```

    adversary-initial-state)
  by (rule
    checked-staged-security-with-actual-alpha-prefix-support-imp-data-state-support
    [OF support[unfolded out-eq]])
  have verifier:
    Some (result, final-state) ∈ set-dist (execute verify-monad ?s)
  using checked-staged-security-experiment-with-data-state-outcomeE
    [OF data-support]
  by blast
  have tr-eq: PTranscript ?s = staged-proof-transcript data
  by simp
  have split:
    hash-map-output-values ?s ∩ set (staged-proof-transcript data)
      ≠ {} ∨
    hash-map-new-output-hit (set (staged-proof-transcript data))
      ?s final-state
  by (rule verify-monad-trace-root-target-preexisting-or-new
    [OF verifier tr-eq])
  show ?thesis
  unfolding out-eq
    checked-staged-security-with-actual-alpha-prefix-sampled-transcript-hit-def
    staged-security-with-data-state-transcript-pre-hit-def
    staged-security-with-data-state-transcript-new-hit-def
  using split by simp
qed

lemma checked-staged-security-with-actual-alpha-prefix-not-prefix-bound-le-sampled-transcript-hit:
  wp-event
    (checked-staged-security-experiment-with-actual-alpha-prefix-data-state A)
    (checked-staged-security-with-actual-alpha-prefix-partial-header-candidate-not-prefix-bound
      bad-sets)
  adversary-initial-state ≤
  wp-event
    (checked-staged-security-experiment-with-actual-alpha-prefix-data-state A)
    checked-staged-security-with-actual-alpha-prefix-sampled-transcript-hit
    adversary-initial-state
  by (rule wp-event-mono-on-support)
  (rule
    checked-staged-security-with-actual-alpha-prefix-not-prefix-bound-imp-sampled-transcript-hit-on-support)

lemma checked-staged-security-with-actual-alpha-prefix-not-prefix-bound-from-sampled-transcript:
  assumes pre-bound:
    wp-event (checked-staged-security-experiment-with-data-state A)
      staged-security-with-data-state-transcript-pre-hit
      adversary-initial-state ≤ P
  and new-bound:
    wp-event (checked-staged-security-experiment-with-data-state A)
      staged-security-with-data-state-transcript-new-hit
      adversary-initial-state ≤ R

```

shows

wp-event

(*checked-staged-security-experiment-with-actual-alpha-prefix-data-state A*)

(*checked-staged-security-with-actual-alpha-prefix-partial-header-candidate-not-prefix-bound*
bad-sets)

adversary-initial-state $\leq P + R$

by (*rule order-trans*

[*OF*

checked-staged-security-with-actual-alpha-prefix-not-prefix-bound-le-sampled-transcript-hit

checked-staged-security-with-actual-alpha-prefix-sampled-transcript-hit-bound-from-data-state

[*OF pre-bound new-bound*]])

lemma *checked-staged-security-with-actual-alpha-prefix-relevant-partial-header-candidate-drift-bound-from-sampled-t*

fixes *P R :: prob*

assumes *wf: staged-budget-wellformed budgets*

and *controlled: staged-adversary-controlled budgets A*

and *data-pre-bound:*

wp-event (*checked-staged-security-experiment-with-data-state A*)

staged-security-with-data-state-transcript-pre-hit

adversary-initial-state $\leq P$

and *transcript-new-bound:*

wp-event (*checked-staged-security-experiment-with-data-state A*)

staged-security-with-data-state-transcript-new-hit

adversary-initial-state $\leq R$

shows

wp-event

(*checked-staged-security-experiment-with-actual-alpha-prefix-data-state A*)

checked-staged-security-with-actual-alpha-prefix-relevant-partial-header-candidate-drift

adversary-initial-state $\leq P + R$

proof –

have *not-prefix-bound:*

wp-event

(*checked-staged-security-experiment-with-actual-alpha-prefix-data-state A*)

(*checked-staged-security-with-actual-alpha-prefix-partial-header-candidate-not-prefix-bound*
composition-trace-bad-alpha-space)

adversary-initial-state $\leq P + R$

by (*rule*

checked-staged-security-with-actual-alpha-prefix-not-prefix-bound-from-sampled-transcript

[*OF data-pre-bound transcript-new-bound*])

show *?thesis*

by (*rule order-trans*

[*OF*

checked-staged-security-with-actual-alpha-prefix-relevant-partial-header-candidate-drift-le-not-prefix-bound
not-prefix-bound])

(*use wf controlled in simp-all*)

qed

theorem *stark-soundness-from-aligned-partial-components-current-query-prefix-and-current-empty-query-from-sampled-t*

fixes *P empty-query-error :: prob*

and $C :: \text{nat} \Rightarrow \text{prob}$
assumes *false-statement*: $\neg \text{exists-valid-trace}$
and *wf*: *staged-budget-wellformed budgets*
and *controlled*: *staged-adversary-controlled budgets A*
and *trace-fri-reduction*:
 $\bigwedge \text{data attacker-state.}$
 $\text{Some (data, attacker-state)} \in$
 $\text{set-dist (execute (checked-staged-transcript-program A)}$
 $\text{adversary-initial-state)} \implies$
 $\text{trace-fri-reachable-partial-candidate-reduction-assumption}$
 $(\text{verifier-state-from-adversary attacker-state}$
 $(\text{staged-proof-transcript data}))$
and *composition-fri-reduction*:
 $\bigwedge \text{data attacker-state.}$
 $\text{Some (data, attacker-state)} \in$
 $\text{set-dist (execute (checked-staged-transcript-program A)}$
 $\text{adversary-initial-state)} \implies$
 $\text{composition-fri-reachable-partial-candidate-reduction-assumption}$
 $(\text{verifier-state-from-adversary attacker-state}$
 $(\text{staged-proof-transcript data}))$
and *round-bound*:
 $\bigwedge i. i < \text{rounds} \implies$
 wp-event
 $(\text{checked-staged-security-experiment-with-query-prefix-data-state A } i)$
 $(\text{checked-staged-security-with-query-prefix-current-query-partial-opening-hit-at } i)$
 $\text{adversary-initial-state} \leq C \ i$
and *data-pre-bound*:
 $\text{wp-event (checked-staged-security-experiment-with-data-state A)}$
 $\text{staged-security-with-data-state-transcript-pre-hit}$
 $\text{adversary-initial-state} \leq P$
and *current-empty-bound*:
 $\text{wp-event (checked-staged-security-experiment-with-data-state A)}$
 $(\text{staged-security-with-data-state-current-empty-query-opening-hit-at } 0)$
 $\text{adversary-initial-state} \leq \text{empty-query-error}$
shows *checked-staged-adversary-acceptance-probability A* $\leq$
 $\text{hash-collision-budget-value } 0$
 $(\text{staged-attacker-query-budget budgets} +$
 $\text{staged-challenge-query-budget} + \text{verifier-hash-query-budget}) +$
 $(\sum i < \text{rounds. } C \ i) +$
 $\text{trace-fri-error} + \text{composition-fri-error} +$
 $(\sum i < \text{rounds. } C \ i) +$
 $(\text{composition-error-bound} +$
 $\text{staged-phase-relation-error size}$
 $(\text{staged-alpha-search-queries budgets } 0) +$
 $\text{hash-collision-budget-value } 0$
 $(\text{staged-alpha-search-queries budgets } 0) +$
 $\text{staged-phase-relation-error size}$
 $(\text{staged-alpha-search-queries budgets } 0) +$
 $(P + \text{staged-concrete-transcript-target-error-bound}) +$

$\text{trace-fri-error} + \text{empty-query-error}$)

proof –

have *trace-fri-bound*:

wp-event (*checked-staged-security-experiment-with-data-state A*)
 (*staged-security-with-data-state-verifier-event*
trace-fri-bad-with-partial-candidates)
adversary-initial-state $\leq$ *trace-fri-error*

by (*rule*
trace-fri-bad-with-partial-candidates-bound-from-reachable-reduction
 [OF *trace-fri-reduction*])

have *comp-fri-bound*:

wp-event (*checked-staged-security-experiment-with-data-state A*)
 (*staged-security-with-data-state-verifier-event*
composition-fri-bad-with-partial-candidates)
adversary-initial-state $\leq$ *composition-fri-error*

by (*rule*
composition-fri-bad-with-partial-candidates-bound-from-reachable-reduction
 [OF *composition-fri-reduction*])

have *empty-trace-fri-bound*:

wp-event (*checked-staged-security-experiment-with-data-state A*)
 (*staged-security-with-data-state-verifier-event*
trace-fri-bad-with-empty-composition-header-candidates)
adversary-initial-state $\leq$ *trace-fri-error*

by (*rule*
trace-fri-bad-with-empty-header-candidates-bound-from-reachable-reduction
 [OF *trace-fri-reduction*])

have *transcript-new-bound*:

wp-event (*checked-staged-security-experiment-with-data-state A*)
staged-security-with-data-state-transcript-new-hit
adversary-initial-state $\leq$ *staged-concrete-transcript-target-error-bound*

by (*rule* *checked-staged-security-with-data-state-transcript-new-hit-bound*
 [OF *wf controlled*])

have *drift-bound*:

wp-event
 (*checked-staged-security-experiment-with-actual-alpha-prefix-data-state A*)
checked-staged-security-with-actual-alpha-prefix-relevant-partial-header-candidate-drift
adversary-initial-state $\leq$
P + *staged-concrete-transcript-target-error-bound*

by (*rule*
checked-staged-security-with-actual-alpha-prefix-relevant-partial-header-candidate-drift-bound-from-sampled-trans
 [OF *wf controlled data-pre-bound transcript-new-bound*])

show *?thesis*

by (*rule*
checked-staged-soundness-from-aligned-transcript-aligned-randomization-components-and-current-query-prefix-ro
 [OF *false-statement wf controlled trace-fri-bound comp-fri-bound*
round-bound drift-bound empty-trace-fri-bound current-empty-bound])

qed

theorem *stark-soundness-from-aligned-partial-components-current-query-prefix-and-current-empty-query-from-sampled*

fixes P *empty-query-error* :: *prob*
and C :: *nat* $\Rightarrow$ *prob*
assumes *false-statement*: $\neg$ *exists-valid-trace*
and *wf*: *staged-budget-wellformed budgets*
and *controlled*: *staged-adversary-controlled budgets A*
and *trace-fri-reduction*:
 $\bigwedge$ *data attacker-state*.
 $\text{Some } (data, \text{attacker-state}) \in$
 $\text{set-dist } (\text{execute } (\text{checked-staged-transcript-program } A)$
 $\text{adversary-initial-state}) \Rightarrow$
 $\text{trace-fri-reachable-partial-candidate-reduction-assumption}$
 $(\text{verifier-state-from-adversary attacker-state}$
 $(\text{staged-proof-transcript data}))$
and *composition-fri-reduction*:
 $\bigwedge$ *data attacker-state*.
 $\text{Some } (data, \text{attacker-state}) \in$
 $\text{set-dist } (\text{execute } (\text{checked-staged-transcript-program } A)$
 $\text{adversary-initial-state}) \Rightarrow$
 $\text{composition-fri-reachable-partial-candidate-reduction-assumption}$
 $(\text{verifier-state-from-adversary attacker-state}$
 $(\text{staged-proof-transcript data}))$
and *round-bound*:
 $\bigwedge i. i < \text{rounds} \Rightarrow$
 wp-event
 $(\text{checked-staged-security-experiment-with-query-prefix-data-state } A \ i)$
 $(\text{checked-staged-security-with-query-prefix-current-query-partial-opening-hit-at } i)$
 $\text{adversary-initial-state} \leq C \ i$
and *data-pre-bound*:
 $\text{wp-event } (\text{checked-staged-security-experiment-with-data-state } A)$
 $\text{staged-security-with-data-state-transcript-pre-hit}$
 $\text{adversary-initial-state} \leq P$
and *current-empty-bound*:
 $\text{wp-event } (\text{checked-staged-security-experiment-with-data-state } A)$
 $(\text{staged-security-with-data-state-current-empty-query-opening-hit-at } 0)$
 $\text{adversary-initial-state} \leq \text{empty-query-error}$
shows $\text{checked-staged-adversary-acceptance-probability } A \leq$
 $\text{hash-collision-budget-value } 0$
 $(\text{staged-attacker-query-budget budgets} +$
 $\text{staged-challenge-query-budget} + \text{verifier-hash-query-budget}) +$
 $(\sum i < \text{rounds}. C \ i) +$
 $\text{trace-fri-error} + \text{composition-fri-error} +$
 $(\text{composition-error-bound} +$
 $\text{staged-phase-relation-error size}$
 $(\text{staged-alpha-search-queries budgets } 0) +$
 $\text{hash-collision-budget-value } 0$
 $(\text{staged-alpha-search-queries budgets } 0) +$
 $\text{staged-phase-relation-error size}$
 $(\text{staged-alpha-search-queries budgets } 0) +$
 $(P + \text{staged-concrete-transcript-target-error-bound}) +$

$\text{trace-fri-error} + \text{empty-query-error}$)
proof –
have *trace-fri-bound*:
 $\text{wp-event } (\text{checked-staged-security-experiment-with-data-state } A)$
 $(\text{staged-security-with-data-state-verifier-event}$
 $\text{trace-fri-bad-with-partial-candidates})$
 $\text{adversary-initial-state} \leq \text{trace-fri-error}$
by (rule
 $\text{trace-fri-bad-with-partial-candidates-bound-from-reachable-reduction}$
 $[\text{OF trace-fri-reduction}]$)
have *comp-fri-bound*:
 $\text{wp-event } (\text{checked-staged-security-experiment-with-data-state } A)$
 $(\text{staged-security-with-data-state-verifier-event}$
 $\text{composition-fri-bad-with-partial-candidates})$
 $\text{adversary-initial-state} \leq \text{composition-fri-error}$
by (rule
 $\text{composition-fri-bad-with-partial-candidates-bound-from-reachable-reduction}$
 $[\text{OF composition-fri-reduction}]$)
have *empty-trace-fri-bound*:
 $\text{wp-event } (\text{checked-staged-security-experiment-with-data-state } A)$
 $(\text{staged-security-with-data-state-verifier-event}$
 $\text{trace-fri-bad-with-empty-composition-header-candidates})$
 $\text{adversary-initial-state} \leq \text{trace-fri-error}$
by (rule
 $\text{trace-fri-bad-with-empty-header-candidates-bound-from-reachable-reduction}$
 $[\text{OF trace-fri-reduction}]$)
have *transcript-new-bound*:
 $\text{wp-event } (\text{checked-staged-security-experiment-with-data-state } A)$
 $\text{staged-security-with-data-state-transcript-new-hit}$
 $\text{adversary-initial-state} \leq \text{staged-concrete-transcript-target-error-bound}$
by (rule $\text{checked-staged-security-with-data-state-transcript-new-hit-bound}$
 $[\text{OF wf controlled}]$)
have *drift-bound*:
 wp-event
 $(\text{checked-staged-security-experiment-with-actual-alpha-prefix-data-state } A)$
 $\text{checked-staged-security-with-actual-alpha-prefix-relevant-partial-header-candidate-drift}$
 $\text{adversary-initial-state} \leq$
 $P + \text{staged-concrete-transcript-target-error-bound}$
by (rule
 $\text{checked-staged-security-with-actual-alpha-prefix-relevant-partial-header-candidate-drift-bound-from-sampled-trans}$
 $[\text{OF wf controlled data-pre-bound transcript-new-bound}]$)
show *?thesis*
by (rule
 $\text{checked-staged-soundness-from-aligned-transcript-aligned-randomization-components-and-current-query-prefix-ro}$
 $[\text{OF false-statement wf controlled trace-fri-bound comp-fri-bound}$
 $\text{round-bound drift-bound empty-trace-fri-bound current-empty-bound}]$)
qed

theorem *stark-soundness-from-aligned-partial-components-structured-query-and-current-empty-query-from-sampled-t*

fixes P *empty-query-error* :: *prob*
and C :: *nat* $\Rightarrow$ *prob*
assumes *false-statement*: $\neg$ *exists-valid-trace*
and *wf*: *staged-budget-wellformed budgets*
and *controlled*: *staged-adversary-controlled budgets A*
and *trace-fri-reduction*:
 $\bigwedge$ *data attacker-state*.
 $\text{Some } (data, \text{attacker-state}) \in$
 $\text{set-dist } (\text{execute } (\text{checked-staged-transcript-program } A)$
 $\text{adversary-initial-state}) \Rightarrow$
 $\text{trace-fri-reachable-partial-candidate-reduction-assumption}$
 $(\text{verifier-state-from-adversary attacker-state}$
 $(\text{staged-proof-transcript data}))$
and *composition-fri-reduction*:
 $\bigwedge$ *data attacker-state*.
 $\text{Some } (data, \text{attacker-state}) \in$
 $\text{set-dist } (\text{execute } (\text{checked-staged-transcript-program } A)$
 $\text{adversary-initial-state}) \Rightarrow$
 $\text{composition-fri-reachable-partial-candidate-reduction-assumption}$
 $(\text{verifier-state-from-adversary attacker-state}$
 $(\text{staged-proof-transcript data}))$
and *round-bound*:
 $\bigwedge i. i < \text{rounds} \Rightarrow$
 wp-event
 $(\text{checked-staged-security-experiment-with-query-prefix-data-state } A \ i)$
 $(\text{checked-staged-security-with-query-prefix-current-query-partial-opening-hit-at } i)$
 $\text{adversary-initial-state} \leq C \ i$
and *data-pre-bound*:
 $\text{wp-event } (\text{checked-staged-security-experiment-with-data-state } A)$
 $\text{staged-security-with-data-state-transcript-pre-hit}$
 $\text{adversary-initial-state} \leq P$
and *current-empty-bound*:
 $\text{wp-event } (\text{checked-staged-security-experiment-with-data-state } A)$
 $(\text{staged-security-with-data-state-current-empty-query-opening-hit-at } 0)$
 $\text{adversary-initial-state} \leq \text{empty-query-error}$
shows $\text{checked-staged-adversary-acceptance-probability } A \leq$
 $\text{hash-collision-budget-value } 0$
 $(\text{staged-attacker-query-budget budgets } +$
 $\text{staged-challenge-query-budget } + \text{verifier-hash-query-budget}) +$
 $(\sum i < \text{rounds}. C \ i) +$
 $\text{trace-fri-error } + \text{composition-fri-error } + 1 +$
 $(\text{composition-error-bound } +$
 $\text{staged-phase-relation-error size}$
 $(\text{staged-alpha-search-queries budgets } 0) +$
 $\text{hash-collision-budget-value } 0$
 $(\text{staged-alpha-search-queries budgets } 0) +$
 $\text{staged-phase-relation-error size}$
 $(\text{staged-alpha-search-queries budgets } 0) +$
 $(P + \text{staged-concrete-transcript-target-error-bound}) +$

$\text{trace-fri-error} + \text{empty-query-error}$)
proof –
have *trace-fri-bound*:
 $\text{wp-event } (\text{checked-staged-security-experiment-with-data-state } A)$
 $(\text{staged-security-with-data-state-verifier-event}$
 $\text{trace-fri-bad-with-partial-candidates})$
 $\text{adversary-initial-state} \leq \text{trace-fri-error}$
by (rule
 $\text{trace-fri-bad-with-partial-candidates-bound-from-reachable-reduction}$
 $[\text{OF trace-fri-reduction}])$
have *comp-fri-bound*:
 $\text{wp-event } (\text{checked-staged-security-experiment-with-data-state } A)$
 $(\text{staged-security-with-data-state-verifier-event}$
 $\text{composition-fri-bad-with-partial-candidates})$
 $\text{adversary-initial-state} \leq \text{composition-fri-error}$
by (rule
 $\text{composition-fri-bad-with-partial-candidates-bound-from-reachable-reduction}$
 $[\text{OF composition-fri-reduction}])$
have *empty-trace-fri-bound*:
 $\text{wp-event } (\text{checked-staged-security-experiment-with-data-state } A)$
 $(\text{staged-security-with-data-state-verifier-event}$
 $\text{trace-fri-bad-with-empty-composition-header-candidates})$
 $\text{adversary-initial-state} \leq \text{trace-fri-error}$
by (rule
 $\text{trace-fri-bad-with-empty-header-candidates-bound-from-reachable-reduction}$
 $[\text{OF trace-fri-reduction}])$
have *transcript-new-bound*:
 $\text{wp-event } (\text{checked-staged-security-experiment-with-data-state } A)$
 $\text{staged-security-with-data-state-transcript-new-hit}$
 $\text{adversary-initial-state} \leq \text{staged-concrete-transcript-target-error-bound}$
by (rule $\text{checked-staged-security-with-data-state-transcript-new-hit-bound}$
 $[\text{OF wf controlled}])$
have *drift-bound*:
 wp-event
 $(\text{checked-staged-security-experiment-with-actual-alpha-prefix-data-state } A)$
 $\text{checked-staged-security-with-actual-alpha-prefix-relevant-partial-header-candidate-drift}$
 $\text{adversary-initial-state} \leq$
 $P + \text{staged-concrete-transcript-target-error-bound}$
by (rule
 $\text{checked-staged-security-with-actual-alpha-prefix-relevant-partial-header-candidate-drift-bound-from-sampled-trans}$
 $[\text{OF wf controlled data-pre-bound transcript-new-bound}])$
have *header-query-bound*:
 wp-event
 $(\text{checked-staged-security-experiment-with-actual-alpha-prefix-data-state}$
 $A)$
 $(\text{checked-staged-security-with-actual-alpha-prefix-header-authenticated-candidate-opening-query-hit-from-prefix}$
 $A)$
 $\text{adversary-initial-state} \leq (1::\text{prob})$
by (rule wp-event-le-1)

show ?thesis

by (rule

checked-staged-soundness-from-aligned-transcript-aligned-randomization-components-and-structured-query-from-
[*OF false-statement wf controlled trace-fri-bound comp-fri-bound*
round-bound header-query-bound drift-bound empty-trace-fri-bound
current-empty-bound])

qed

theorem *stark-soundness-from-transcript-indexed-partial-opening-route-from-sampled-transcript:*

fixes *P empty-query-error :: prob*

and *C :: nat $\Rightarrow$ prob*

assumes *false-statement: $\neg$ exists-valid-trace*

and *wf: staged-budget-wellformed budgets*

and *controlled: staged-adversary-controlled budgets A*

and *trace-fri-reduction:*

$\wedge$ *data attacker-state.*

Some (data, attacker-state) $\in$

set-dist (execute (checked-staged-transcript-program A)

adversary-initial-state) $\Rightarrow$

trace-fri-reachable-partial-candidate-reduction-assumption

(verifier-state-from-adversary attacker-state

(staged-proof-transcript data))

and *composition-fri-reduction:*

$\wedge$ *data attacker-state.*

Some (data, attacker-state) $\in$

set-dist (execute (checked-staged-transcript-program A)

adversary-initial-state) $\Rightarrow$

composition-fri-reachable-partial-candidate-reduction-assumption

(verifier-state-from-adversary attacker-state

(staged-proof-transcript data))

and *round-bound:*

$\wedge$ *i. i < rounds $\Rightarrow$*

wp-event

(checked-staged-security-experiment-with-query-prefix-data-state A i)

(checked-staged-security-with-query-prefix-current-query-partial-opening-hit-at i)

adversary-initial-state $\leq$ C i

and *data-pre-bound:*

wp-event (checked-staged-security-experiment-with-data-state A)

staged-security-with-data-state-transcript-pre-hit

adversary-initial-state $\leq$ P

and *current-empty-bound:*

wp-event (checked-staged-security-experiment-with-data-state A)

(staged-security-with-data-state-current-empty-query-opening-hit-at 0)

adversary-initial-state $\leq$ empty-query-error

shows *checked-staged-adversary-acceptance-probability A $\leq$*

hash-collision-budget-value 0

(staged-attacker-query-budget budgets +

staged-challenge-query-budget + verifier-hash-query-budget) +

($\sum$ i < rounds. C i) +

$\text{trace-fri-error} + \text{composition-fri-error} +$
 $(\sum_{i < \text{rounds}} C_i) +$
 $(\text{composition-error-bound} +$
 $\text{staged-phase-relation-error size}$
 $(\text{staged-alpha-search-queries budgets } 0) +$
 $\text{hash-collision-budget-value } 0$
 $(\text{staged-alpha-search-queries budgets } 0) +$
 $\text{staged-phase-relation-error size}$
 $(\text{staged-alpha-search-queries budgets } 0) +$
 $((\text{composition-error-bound} +$
 $\text{staged-phase-relation-error size}$
 $(\text{staged-alpha-search-queries budgets } 0) +$
 $\text{hash-collision-budget-value } 0$
 $(\text{staged-alpha-search-queries budgets } 0) +$
 $(P + \text{staged-concrete-transcript-target-error-bound})) +$
 $(\text{trace-fri-error} +$
 $\text{low-degree-trace-pair-agreement-error budgets})) +$
 $\text{trace-fri-error} + \text{empty-query-error})$

proof –

have *sampled-transcript-new-bound*:

$\text{wp-event } (\text{checked-staged-security-experiment-with-data-state } A)$
 $\text{staged-security-with-data-state-transcript-new-hit}$
 $\text{adversary-initial-state} \leq \text{staged-concrete-transcript-target-error-bound}$

by (rule

$\text{checked-staged-security-with-data-state-transcript-new-hit-bound}$
 $[\text{OF wf controlled}]$)

have *not-prefix-bound*:

wp-event
 $(\text{checked-staged-security-experiment-with-actual-alpha-prefix-data-state } A)$
 $(\text{checked-staged-security-with-actual-alpha-prefix-partial-header-candidate-not-prefix-bound}$
 $\text{composition-trace-bad-alpha-space})$
 $\text{adversary-initial-state} \leq$
 $P + \text{staged-concrete-transcript-target-error-bound}$

by (rule

$\text{checked-staged-security-with-actual-alpha-prefix-not-prefix-bound-from-sampled-transcript}$
 $[\text{OF data-pre-bound sampled-transcript-new-bound}]$)

show *?thesis*

by (rule

$\text{stark-soundness-from-transcript-indexed-partial-opening-route-from-not-prefix}$
 $[\text{OF false-statement wf controlled trace-fri-reduction}$
 $\text{composition-fri-reduction round-bound not-prefix-bound}$
 $\text{current-empty-bound}]$)

qed

lemma *checked-staged-security-with-actual-alpha-prefix-not-prefix-low-degree-sampled-agreement-imp-prefix-pair-hit*:

assumes

$\text{checked-staged-security-with-actual-alpha-prefix-not-prefix-low-degree-sampled-agreement}$
 $A \text{ out}$

shows

```

     $\exists \text{pair} \in \text{low-degree-trace-pair-space}.$ 
     $\exists i \in \{..<\text{rounds}\}.$ 
    checked-staged-security-with-actual-alpha-prefix-trace-pair-agreement-hit-from-prefix
     $A \text{ (fst pair) (snd pair) } i \text{ out}$ 
proof (cases out)
  case None
  then show ?thesis
    using assms
    unfolding
      checked-staged-security-with-actual-alpha-prefix-not-prefix-low-degree-sampled-agreement-def
    by simp
next
  case (Some packed)
  then obtain alpha-prefix alpha-prefix-state data attacker-state result
    final-state where out-eq:
    out =
      Some (((((alpha-prefix, alpha-prefix-state), data), attacker-state),
        result), final-state)
    by (cases packed, auto split: prod.splits)
  from assms obtain fr f-fri-roots f-final trace-table prefix-trace-table
    witness-final-state trace-openings witness-result query-idxs as i
    prefix prefix-state' raw raw-state where
    trace-low: trace-table-low-degree trace-table
    and prefix-low: trace-table-low-degree prefix-trace-table
    and distinct: trace-table  $\neq$  prefix-trace-table
    and i-bound: i < rounds
    and builder:
      Some (data, attacker-state)  $\in$ 
        set-dist
        (execute (checked-staged-transcript-program A)
          adversary-initial-state)
    and prefix-receive:
      Some (((prefix, prefix-state'), raw), raw-state)  $\in$ 
        set-dist
        (execute (checked-staged-query-prefix-receive-with-state A i)
          adversary-initial-state)
    and raw-agree:
      index (to-nat raw)  $\in$ 
        trace-table-agreement-indices trace-table prefix-trace-table
    unfolding out-eq
      checked-staged-security-with-actual-alpha-prefix-not-prefix-low-degree-sampled-agreement-def
      Let-def
    by (auto split: prod.splits)
  have pair-in:
    (trace-table, prefix-trace-table)  $\in$  low-degree-trace-pair-space
    unfolding low-degree-trace-pair-space-def
    using trace-low prefix-low distinct by simp
  have prefix-hit:
    checked-staged-security-with-actual-alpha-prefix-trace-pair-agreement-hit-from-prefix

```

```

    A trace-table prefix-trace-table i out
unfolding out-eq
    checked-staged-security-with-actual-alpha-prefix-trace-pair-agreement-hit-from-prefix-def
    checked-staged-query-prefix-dynamic-index-hit-def
apply (simp add: builder)
using prefix-receive raw-agree by blast
show ?thesis
by (intro bexI[of - (trace-table, prefix-trace-table)]
    bexI[of - i])
    (use pair-in i-bound prefix-hit in simp-all)
qed

lemma checked-staged-security-with-actual-alpha-prefix-not-prefix-low-degree-actual-sampled-agreement-imp-pair-hit:
assumes
    checked-staged-security-with-actual-alpha-prefix-not-prefix-low-degree-actual-sampled-agreement
    out
shows
     $\exists \text{pair} \in \text{low-degree-trace-pair-space}.$ 
     $\exists i \in \{..<\text{rounds}\}.$ 
    checked-staged-security-with-actual-alpha-prefix-trace-pair-agreement-hit-at
    (fst pair) (snd pair) i out
proof (cases out)
case None
then show ?thesis
    using assms
    unfolding
    checked-staged-security-with-actual-alpha-prefix-not-prefix-low-degree-actual-sampled-agreement-def
    by simp
next
case (Some packed)
then obtain alpha-prefix prefix-state data attacker-state result
    final-state where out-eq:
    out =
        Some (((((alpha-prefix, prefix-state), data), attacker-state),
            result), final-state)
    by (cases packed, auto split: prod.splits)
from assms obtain fr f-fri-roots f-final trace-table prefix-trace-table
    witness-final-state trace-openings as query-idxs i where
    trace-low: trace-table-low-degree trace-table
    and prefix-low: trace-table-low-degree prefix-trace-table
    and distinct: trace-table  $\neq$  prefix-trace-table
    and shape:
        accepted-transcript-shape
        (verifier-state-from-adversary attacker-state
            (staged-proof-transcript data))
        (Some (result, final-state)) as query-idxs
    and i-bound:  $i < \text{rounds}$ 
    and idx-agree:
        query-idxs ! i  $\in$ 

```



```

    trace-table-agreement-indices trace-table prefix-trace-table
unfolding out-eq
checked-staged-security-with-actual-alpha-prefix-not-prefix-low-degree-actual-sampled-agreement-def
  Let-def
  by auto
have pair-in:
  (trace-table, prefix-trace-table) ∈ low-degree-trace-pair-space
unfolding low-degree-trace-pair-space-def
using trace-low prefix-low distinct by simp
have pair-hit:
checked-staged-security-with-actual-alpha-prefix-trace-pair-agreement-hit-at
  trace-table prefix-trace-table i out
unfolding out-eq
  checked-staged-security-with-actual-alpha-prefix-trace-pair-agreement-hit-at-def
  staged-security-with-data-state-trace-pair-agreement-hit-at-def
using shape i-bound idx-agree by auto
show ?thesis
  by (intro bexI[of - (trace-table, prefix-trace-table)]
    bexI[of - i])
    (use pair-in i-bound pair-hit in simp-all)
qed

lemma checked-staged-security-with-actual-alpha-prefix-not-prefix-low-degree-actual-sampled-agreement-bound:
assumes wf: staged-budget-wellformed budgets
and controlled: staged-adversary-controlled budgets A
shows
  wp-event
    (checked-staged-security-experiment-with-actual-alpha-prefix-data-state A)
checked-staged-security-with-actual-alpha-prefix-not-prefix-low-degree-actual-sampled-agreement
  adversary-initial-state ≤
  low-degree-trace-pair-agreement-error budgets
proof -
let ?M =
  checked-staged-security-experiment-with-actual-alpha-prefix-data-state A
let ?pair-hit =
  λ(pair, i) out.
    checked-staged-security-with-actual-alpha-prefix-trace-pair-agreement-hit-at
      (fst pair) (snd pair) i out
let ?A = low-degree-trace-pair-space × {.. $\text{rounds}$ }
have event-le:
  wp-event ?M
checked-staged-security-with-actual-alpha-prefix-not-prefix-low-degree-actual-sampled-agreement
  adversary-initial-state ≤
  wp-event ?M (λout. ∃ x ∈ ?A. ?pair-hit x out)
  adversary-initial-state
by (rule wp-event-mono)
  (auto dest:
    checked-staged-security-with-actual-alpha-prefix-not-prefix-low-degree-actual-sampled-agreement-imp-pair-hit)
have finite-A: finite ?A

```

```

by simp
have union-bound:
  wp-event ?M ( $\lambda out. \exists x \in ?A. ?pair\text{-}hit\ x\ out$ )
    adversary-initial-state  $\leq$ 
    ( $\sum x \in ?A.$ 
      case x of (pair, i)  $\Rightarrow$ 
        staged-phase-relation-error size
          (staged-query-search-queries budgets i + 1) +
          nnreal clength / nnreal (card query-sample-space))
proof (rule wp-event-finite-UN-bound[OF finite-A])
  fix x
  assume x-in:  $x \in ?A$ 
  then obtain trace-table prefix-trace-table i where x-eq:
     $x = ((trace\text{-}table, prefix\text{-}trace\text{-}table), i)$ 
  and pair-in:
     $(trace\text{-}table, prefix\text{-}trace\text{-}table) \in low\text{-}degree\text{-}trace\text{-}pair\text{-}space$ 
  and i-bound:  $i < rounds$ 
  by (cases x, auto)
  have trace-low: trace-table-low-degree trace-table
    using pair-in unfolding low-degree-trace-pair-space-def by auto
  have prefix-low: trace-table-low-degree prefix-trace-table
    using pair-in unfolding low-degree-trace-pair-space-def by auto
  have distinct: trace-table  $\neq$  prefix-trace-table
    using pair-in unfolding low-degree-trace-pair-space-def by auto
  have bound:
    wp-event ?M
      (checked-staged-security-with-actual-alpha-prefix-trace-pair-agreement-hit-at
        trace-table prefix-trace-table i)
      adversary-initial-state  $\leq$ 
      staged-phase-relation-error size
        (staged-query-search-queries budgets i + 1) +
        nnreal clength / nnreal (card query-sample-space)
    by (rule
      checked-staged-security-with-actual-alpha-prefix-trace-pair-agreement-hit-at-bound
      [OF wf controlled i-bound trace-low prefix-low distinct])
  show wp-event ?M (?pair-hit x) adversary-initial-state
     $\leq$  (case x of (pair, i)  $\Rightarrow$ 
      staged-phase-relation-error size
        (staged-query-search-queries budgets i + 1) +
        nnreal clength / nnreal (card query-sample-space))
    unfolding x-eq
    using bound by simp
qed
have sum-eq:
  ( $\sum x \in ?A.$ 
    case x of (pair, i)  $\Rightarrow$ 
      staged-phase-relation-error size
        (staged-query-search-queries budgets i + 1) +
        nnreal clength / nnreal (card query-sample-space)) =

```

```

    low-degree-trace-pair-agreement-error budgets
unfolding low-degree-trace-pair-agreement-error-def
    low-degree-trace-pair-agreement-round-error-def
by (simp add: sum.cartesian-product case-prod-unfold)
show ?thesis
proof (rule order-trans[OF event-le])
    show wp-event ?M ( $\lambda$ out.  $\exists x \in ?A. ?pair\text{-}hit\ x\ out$ )
        adversary-initial-state  $\leq$ 
        low-degree-trace-pair-agreement-error budgets
    using union-bound sum-eq by simp
qed
qed

theorem stark-soundness-from-transcript-indexed-partial-opening-route-from-sampled-transcript-not-prefix:
fixes P empty-query-error :: prob
and C :: nat  $\Rightarrow$  prob
assumes false-statement:  $\neg$  exists-valid-trace
and wf: staged-budget-wellformed budgets
and controlled: staged-adversary-controlled budgets A
and trace-fri-reduction:
     $\bigwedge$ data attacker-state.
    Some (data, attacker-state)  $\in$ 
    set-dist (execute (checked-staged-transcript-program A)
    adversary-initial-state)  $\implies$ 
    trace-fri-reachable-partial-candidate-reduction-assumption
    (verifier-state-from-adversary attacker-state
    (staged-proof-transcript data))
and composition-fri-reduction:
     $\bigwedge$ data attacker-state.
    Some (data, attacker-state)  $\in$ 
    set-dist (execute (checked-staged-transcript-program A)
    adversary-initial-state)  $\implies$ 
    composition-fri-reachable-partial-candidate-reduction-assumption
    (verifier-state-from-adversary attacker-state
    (staged-proof-transcript data))
and round-bound:
     $\bigwedge i. i < rounds \implies$ 
    wp-event
    (checked-staged-security-experiment-with-query-prefix-data-state A i)
    (checked-staged-security-with-query-prefix-current-query-partial-opening-hit-at i)
    adversary-initial-state  $\leq C\ i$ 
and data-pre-bound:
    wp-event (checked-staged-security-experiment-with-data-state A)
    staged-security-with-data-state-transcript-pre-hit
    adversary-initial-state  $\leq P$ 
and current-empty-bound:
    wp-event (checked-staged-security-experiment-with-data-state A)
    (staged-security-with-data-state-current-empty-query-opening-hit-at 0)
    adversary-initial-state  $\leq$  empty-query-error

```

shows *checked-staged-adversary-acceptance-probability* $A \leq$
hash-collision-budget-value 0
 (*staged-attacker-query-budget* budgets +
staged-challenge-query-budget + *verifier-hash-query-budget*) +
 ($\sum i < \text{rounds}. C\ i$) +
trace-fri-error + *composition-fri-error* +
 ($\sum i < \text{rounds}. C\ i$) +
 (*composition-error-bound* +
staged-phase-relation-error size
 (*staged-alpha-search-queries* budgets 0) +
hash-collision-budget-value 0
 (*staged-alpha-search-queries* budgets 0) +
staged-phase-relation-error size
 (*staged-alpha-search-queries* budgets 0) +
 ((*composition-error-bound* +
staged-phase-relation-error size
 (*staged-alpha-search-queries* budgets 0) +
hash-collision-budget-value 0
 (*staged-alpha-search-queries* budgets 0) +
 ($P + \text{staged-concrete-transcript-target-error-bound}$)) +
 (*trace-fri-error* +
low-degree-trace-pair-agreement-error budgets)) +
trace-fri-error + *empty-query-error*)
proof –
have *transcript-new-bound*:
wp-event (*checked-staged-security-experiment-with-data-state* A)
staged-security-with-data-state-transcript-new-hit
adversary-initial-state $\leq$
staged-concrete-transcript-target-error-bound
by (rule
checked-staged-security-with-data-state-transcript-new-hit-bound
 [*OF* *wf* controlled])
have *not-prefix-bound*:
wp-event
 (*checked-staged-security-experiment-with-actual-alpha-prefix-data-state* A)
 (*checked-staged-security-with-actual-alpha-prefix-partial-header-candidate-not-prefix-bound*
composition-trace-bad-alpha-space)
adversary-initial-state $\leq$
 $P + \text{staged-concrete-transcript-target-error-bound}$
by (rule
checked-staged-security-with-actual-alpha-prefix-not-prefix-bound-from-sampled-transcript
 [*OF* *data-pre-bound* *transcript-new-bound*])
show ?thesis
by (rule
stark-soundness-from-transcript-indexed-partial-opening-route-from-not-prefix
 [*OF* *false-statement* *wf* controlled *trace-fri-reduction*
composition-fri-reduction *round-bound* *not-prefix-bound*
current-empty-bound])
qed

theorem *stark-soundness-from-transcript-indexed-partial-opening-route-from-residual-sampled-transcript-not-prefix:*
fixes P *empty-query-error* :: *prob*
and R :: *nat* $\Rightarrow$ *prob*
assumes *false-statement*: $\neg$ *exists-valid-trace*
and *wf*: *staged-budget-wellformed budgets*
and *controlled*: *staged-adversary-controlled budgets A*
and *trace-fri-reduction*:
 $\bigwedge$ *data attacker-state*.
Some (data, attacker-state) $\in$
set-dist (execute (checked-staged-transcript-program A)
adversary-initial-state) $\Rightarrow$
trace-fri-reachable-partial-candidate-reduction-assumption
(verifier-state-from-adversary attacker-state
(staged-proof-transcript data))
and *composition-fri-reduction*:
 $\bigwedge$ *data attacker-state*.
Some (data, attacker-state) $\in$
set-dist (execute (checked-staged-transcript-program A)
adversary-initial-state) $\Rightarrow$
composition-fri-reachable-partial-candidate-reduction-assumption
(verifier-state-from-adversary attacker-state
(staged-proof-transcript data))
and *residual-bound*:
 $\bigwedge i. i < \text{rounds} \Rightarrow$
wp-event
(checked-staged-security-experiment-with-query-prefix-data-state A i)
(checked-staged-security-with-query-prefix-conceptual-context-residual-hit-at
i)
adversary-initial-state $\leq R i$
and *data-pre-bound*:
wp-event (checked-staged-security-experiment-with-data-state A)
staged-security-with-data-state-transcript-pre-hit
adversary-initial-state $\leq P$
and *current-empty-bound*:
wp-event (checked-staged-security-experiment-with-data-state A)
(staged-security-with-data-state-current-empty-query-opening-hit-at 0)
adversary-initial-state $\leq$ empty-query-error
shows *checked-staged-adversary-acceptance-probability A $\leq$*
hash-collision-budget-value 0
(staged-attacker-query-budget budgets +
staged-challenge-query-budget + verifier-hash-query-budget) +
 $(\sum i < \text{rounds}.$
staged-phase-relation-error size
(staged-query-search-queries budgets i + 1) +
query-error-bound + R i +
(P + staged-concrete-transcript-target-error-bound) +
(P + staged-concrete-transcript-target-error-bound)) +
trace-fri-error + composition-fri-error +

$(\sum i < \text{rounds}.$
staged-phase-relation-error size
 $(\text{staged-query-search-queries budgets } i + 1) +$
 $\text{query-error-bound} + R \ i +$
 $(P + \text{staged-concrete-transcript-target-error-bound}) +$
 $(P + \text{staged-concrete-transcript-target-error-bound})) +$
 $(\text{composition-error-bound} +$
staged-phase-relation-error size
 $(\text{staged-alpha-search-queries budgets } 0) +$
hash-collision-budget-value 0
 $(\text{staged-alpha-search-queries budgets } 0) +$
staged-phase-relation-error size
 $(\text{staged-alpha-search-queries budgets } 0) +$
 $((\text{composition-error-bound} +$
staged-phase-relation-error size
 $(\text{staged-alpha-search-queries budgets } 0) +$
hash-collision-budget-value 0
 $(\text{staged-alpha-search-queries budgets } 0) +$
 $(P + \text{staged-concrete-transcript-target-error-bound})) +$
 $(\text{trace-fri-error} +$
 $\text{low-degree-trace-pair-agreement-error budgets})) +$
 $\text{trace-fri-error} + \text{empty-query-error})$
proof –
have *transcript-new-bound*:
 $\text{wp-event } (\text{checked-staged-security-experiment-with-data-state } A)$
 $\text{staged-security-with-data-state-transcript-new-hit}$
 $\text{adversary-initial-state} \leq$
 $\text{staged-concrete-transcript-target-error-bound}$
by (rule
 $\text{checked-staged-security-with-data-state-transcript-new-hit-bound}$
 $[\text{OF wf controlled}])$
have *trace-path-bound*:
 $\bigwedge i. i < \text{rounds} \implies$
 wp-event
 $(\text{checked-staged-security-experiment-with-query-prefix-data-state } A \ i)$
 $(\text{checked-staged-security-with-query-prefix-current-trace-path-output-hit-at}$
 $i)$
 $\text{adversary-initial-state} \leq$
 $P + \text{staged-concrete-transcript-target-error-bound}$
by (rule
 $\text{checked-staged-security-with-query-prefix-current-trace-path-output-hit-at-bound-from-transcript}$
 $[\text{OF - data-pre-bound transcript-new-bound}])$
have *composition-path-bound*:
 $\bigwedge i. i < \text{rounds} \implies$
 wp-event
 $(\text{checked-staged-security-experiment-with-query-prefix-data-state } A \ i)$
 $(\text{checked-staged-security-with-query-prefix-current-composition-path-output-hit-at}$
 $i)$
 $\text{adversary-initial-state} \leq$

$P + \text{staged-concrete-transcript-target-error-bound}$
by (rule
checked-staged-security-with-query-prefix-current-composition-path-output-hit-at-bound-from-transcript
 $[OF - \text{data-pre-bound transcript-new-bound}]$)
have *round-bound*:
 $\bigwedge i. i < \text{rounds} \implies$
wp-event
 $(\text{checked-staged-security-experiment-with-query-prefix-data-state } A \ i)$
 $(\text{checked-staged-security-with-query-prefix-current-query-partial-opening-hit-at } i)$
 $\text{adversary-initial-state} \leq$
 $\text{staged-phase-relation-error size}$
 $(\text{staged-query-search-queries budgets } i + 1) +$
 $\text{query-error-bound} + R \ i +$
 $(P + \text{staged-concrete-transcript-target-error-bound}) +$
 $(P + \text{staged-concrete-transcript-target-error-bound})$
by (rule
checked-staged-security-with-query-prefix-current-query-partial-opening-hit-at-bound-from-conceptual-residual-and
 $[OF \text{ wf controlled - residual-bound trace-path-bound}$
 $\text{composition-path-bound}]$)
show *?thesis*
by (rule
stark-soundness-from-transcript-indexed-partial-opening-route-from-sampled-transcript-not-prefix
 $[OF \text{ false-statement wf controlled trace-fri-reduction}$
 $\text{composition-fri-reduction round-bound data-pre-bound}$
 $\text{current-empty-bound}]$)
qed

theorem *stark-soundness-from-transcript-indexed-partial-opening-route-from-residual-split-sampled-transcript-not-pr*
fixes $P \text{ empty-query-error} :: \text{prob}$
and $LT \ LC \ Alpha :: \text{nat} \Rightarrow \text{prob}$
assumes *false-statement*: $\neg \text{exists-valid-trace}$
and *wf*: *staged-budget-wellformed* *budgets*
and *controlled*: *staged-adversary-controlled* *budgets* A
and *trace-fri-reduction*:
 $\bigwedge \text{data attacker-state.}$
 $\text{Some } (data, \text{attacker-state}) \in$
 $\text{set-dist } (\text{execute } (\text{checked-staged-transcript-program } A)$
 $\text{adversary-initial-state}) \implies$
 $\text{trace-fri-reachable-partial-candidate-reduction-assumption}$
 $(\text{verifier-state-from-adversary attacker-state}$
 $(\text{staged-proof-transcript data}))$
and *composition-fri-reduction*:
 $\bigwedge \text{data attacker-state.}$
 $\text{Some } (data, \text{attacker-state}) \in$
 $\text{set-dist } (\text{execute } (\text{checked-staged-transcript-program } A)$
 $\text{adversary-initial-state}) \implies$
 $\text{composition-fri-reachable-partial-candidate-reduction-assumption}$
 $(\text{verifier-state-from-adversary attacker-state}$
 $(\text{staged-proof-transcript data}))$

and *trace-low-degree-bound*:

$\bigwedge i. i < \text{rounds} \implies$

wp-event

(*checked-staged-security-experiment-with-query-prefix-data-state A i*)

(*checked-staged-security-with-query-prefix-conceptual-trace-low-degree-bad-at i*)

adversary-initial-state $\leq$ *LT i*

and *composition-low-degree-bound*:

$\bigwedge i. i < \text{rounds} \implies$

wp-event

(*checked-staged-security-experiment-with-query-prefix-data-state A i*)

(*checked-staged-security-with-query-prefix-conceptual-composition-low-degree-bad-at i*)

adversary-initial-state $\leq$ *LC i*

and *alpha-bound*:

$\bigwedge i. i < \text{rounds} \implies$

wp-event

(*checked-staged-security-experiment-with-query-prefix-data-state A i*)

(*checked-staged-security-with-query-prefix-conceptual-alpha-bad-set-hit-at i*)

adversary-initial-state $\leq$ *Alpha i*

and *data-pre-bound*:

wp-event (*checked-staged-security-experiment-with-data-state A*)

staged-security-with-data-state-transcript-pre-hit

adversary-initial-state $\leq$ *P*

and *current-empty-bound*:

wp-event (*checked-staged-security-experiment-with-data-state A*)

(*staged-security-with-data-state-current-empty-query-opening-hit-at 0*)

adversary-initial-state $\leq$ *empty-query-error*

shows *checked-staged-adversary-acceptance-probability A* $\leq$

hash-collision-budget-value 0

(*staged-attacker-query-budget budgets* +

staged-challenge-query-budget + *verifier-hash-query-budget*) +

($\sum i < \text{rounds}.$

staged-phase-relation-error size

(*staged-query-search-queries budgets i + 1*) +

query-error-bound +

hash-collision-budget-value 0

(*staged-query-search-queries budgets i + 1*) +

(*LT i* + *LC i* + *Alpha i*) +

(*P* + *staged-concrete-transcript-target-error-bound*) +

(*P* + *staged-concrete-transcript-target-error-bound*)) +

trace-fri-error + *composition-fri-error* +

($\sum i < \text{rounds}.$

staged-phase-relation-error size

(*staged-query-search-queries budgets i + 1*) +

query-error-bound +

hash-collision-budget-value 0

(*staged-query-search-queries budgets i + 1*) +

$(LT\ i + LC\ i + Alpha\ i) +$
 $(P + staged-concrete-transcript-target-error-bound) +$
 $(P + staged-concrete-transcript-target-error-bound)) +$
 $(composition-error-bound +$
 $staged-phase-relation-error\ size$
 $(staged-alpha-search-queries\ budgets\ 0) +$
 $hash-collision-budget-value\ 0$
 $(staged-alpha-search-queries\ budgets\ 0) +$
 $staged-phase-relation-error\ size$
 $(staged-alpha-search-queries\ budgets\ 0) +$
 $((composition-error-bound +$
 $staged-phase-relation-error\ size$
 $(staged-alpha-search-queries\ budgets\ 0) +$
 $hash-collision-budget-value\ 0$
 $(staged-alpha-search-queries\ budgets\ 0) +$
 $(P + staged-concrete-transcript-target-error-bound)) +$
 $(trace-fri-error +$
 $low-degree-trace-pair-agreement-error\ budgets)) +$
 $trace-fri-error + empty-query-error)$

proof –

have *residual-bound*:

$\bigwedge i. i < rounds \implies$

wp-event

$(checked-staged-security-experiment-with-query-prefix-data-state\ A\ i)$

$(checked-staged-security-with-query-prefix-conceptual-context-residual-hit-at$

$i)$

$adversary-initial-state \leq$

$hash-collision-budget-value\ 0$

$(staged-query-search-queries\ budgets\ i + 1) +$

$(LT\ i + LC\ i + Alpha\ i)$

proof –

fix *i*

assume *i-bound*: $i < rounds$

have *algebraic-bound*:

wp-event

$(checked-staged-security-experiment-with-query-prefix-data-state\ A\ i)$

$(checked-staged-security-with-query-prefix-conceptual-context-algebraic-residual-hit-at$

$i)$

$adversary-initial-state \leq LT\ i + LC\ i + Alpha\ i$

by (rule

checked-staged-security-with-query-prefix-conceptual-context-algebraic-residual-hit-at-bound-from-split

$[OF\ false-statement\ trace-low-degree-bound[OF\ i-bound]$

$composition-low-degree-bound[OF\ i-bound]$

$alpha-bound[OF\ i-bound]])$

show

wp-event

$(checked-staged-security-experiment-with-query-prefix-data-state\ A\ i)$

$(checked-staged-security-with-query-prefix-conceptual-context-residual-hit-at$

$i)$

adversary-initial-state $\leq$
hash-collision-budget-value 0
 (*staged-query-search-queries* budgets $i + 1$) +
 (*LT* i + *LC* i + *Alpha* i)
by (rule
 checked-staged-security-with-query-prefix-conceptual-context-residual-hit-at-bound-from-split
 [*OF* wf controlled *i-bound algebraic-bound*])
qed
have bound:
 checked-staged-adversary-acceptance-probability $A \leq$
 hash-collision-budget-value 0
 (*staged-attacker-query-budget* budgets +
staged-challenge-query-budget + *verifier-hash-query-budget*) +
 ($\sum i < \text{rounds}$.
 staged-phase-relation-error size
 (*staged-query-search-queries* budgets $i + 1$) +
 query-error-bound +
 (*hash-collision-budget-value* 0
 (*staged-query-search-queries* budgets $i + 1$) +
 (*LT* i + *LC* i + *Alpha* i)) +
 (P + *staged-concrete-transcript-target-error-bound*) +
 (P + *staged-concrete-transcript-target-error-bound*)) +
 trace-fri-error + composition-fri-error +
 ($\sum i < \text{rounds}$.
 staged-phase-relation-error size
 (*staged-query-search-queries* budgets $i + 1$) +
 query-error-bound +
 (*hash-collision-budget-value* 0
 (*staged-query-search-queries* budgets $i + 1$) +
 (*LT* i + *LC* i + *Alpha* i)) +
 (P + *staged-concrete-transcript-target-error-bound*) +
 (P + *staged-concrete-transcript-target-error-bound*)) +
 (*composition-error-bound* +
 staged-phase-relation-error size
 (*staged-alpha-search-queries* budgets 0) +
 hash-collision-budget-value 0
 (*staged-alpha-search-queries* budgets 0) +
 staged-phase-relation-error size
 (*staged-alpha-search-queries* budgets 0) +
 ((*composition-error-bound* +
 staged-phase-relation-error size
 (*staged-alpha-search-queries* budgets 0) +
 hash-collision-budget-value 0
 (*staged-alpha-search-queries* budgets 0) +
 (P + *staged-concrete-transcript-target-error-bound*)) +
 (*trace-fri-error* +
 low-degree-trace-pair-agreement-error budgets)) +
 trace-fri-error + empty-query-error)
by (rule

```

    stark-soundness-from-transcript-indexed-partial-opening-route-from-residual-sampled-transcript-not-prefix
    [OF false-statement wf controlled trace-fri-reduction
     composition-fri-reduction residual-bound data-pre-bound
     current-empty-bound])
  show ?thesis
  using bound by (simp add: algebra-simps)
qed

theorem stark-soundness-from-transcript-indexed-partial-opening-route-from-same-run-partial-opening-gap-and-tran
fixes G P empty-query-error :: prob
and C :: nat  $\Rightarrow$  prob
assumes false-statement:  $\neg$  exists-valid-trace
and wf: staged-budget-wellformed budgets
and controlled: staged-adversary-controlled budgets A
and trace-fri-reduction:
   $\bigwedge$  data attacker-state.
  Some (data, attacker-state)  $\in$ 
    set-dist (execute (checked-staged-transcript-program A)
      adversary-initial-state)  $\implies$ 
    trace-fri-reachable-partial-candidate-reduction-assumption
      (verifier-state-from-adversary attacker-state
        (staged-proof-transcript data))
and composition-fri-reduction:
   $\bigwedge$  data attacker-state.
  Some (data, attacker-state)  $\in$ 
    set-dist (execute (checked-staged-transcript-program A)
      adversary-initial-state)  $\implies$ 
    composition-fri-reachable-partial-candidate-reduction-assumption
      (verifier-state-from-adversary attacker-state
        (staged-proof-transcript data))
and round-bound:
   $\bigwedge i. i < \text{rounds} \implies$ 
    wp-event
      (checked-staged-security-experiment-with-query-prefix-data-state A i)
      (checked-staged-security-with-query-prefix-current-query-partial-opening-hit-at i)
      adversary-initial-state  $\leq$  C i
and gap-bound:
  wp-event
    (checked-staged-security-experiment-with-actual-alpha-prefix-data-state
      A)
    checked-staged-security-with-actual-alpha-prefix-not-prefix-same-run-partial-opening-gap
    adversary-initial-state  $\leq$  G
and data-pre-bound:
  wp-event (checked-staged-security-experiment-with-data-state A)
    staged-security-with-data-state-transcript-pre-hit
    adversary-initial-state  $\leq$  P
and current-empty-bound:
  wp-event (checked-staged-security-experiment-with-data-state A)
    (staged-security-with-data-state-current-empty-query-opening-hit-at 0)

```

$adversary-initial-state \leq empty-query-error$
shows $checked-staged-adversary-acceptance-probability\ A \leq$
 $hash-collision-budget-value\ 0$
 $(staged-attacker-query-budget\ budgets +$
 $staged-challenge-query-budget + verifier-hash-query-budget) +$
 $(\sum i < rounds. C\ i) +$
 $trace-fri-error + composition-fri-error +$
 $(\sum i < rounds. C\ i) +$
 $(composition-error-bound +$
 $staged-phase-relation-error\ size$
 $(staged-alpha-search-queries\ budgets\ 0) +$
 $hash-collision-budget-value\ 0$
 $(staged-alpha-search-queries\ budgets\ 0) +$
 $staged-phase-relation-error\ size$
 $(staged-alpha-search-queries\ budgets\ 0) +$
 $((composition-error-bound +$
 $staged-phase-relation-error\ size$
 $(staged-alpha-search-queries\ budgets\ 0) +$
 $hash-collision-budget-value\ 0$
 $(staged-alpha-search-queries\ budgets\ 0) +$
 $(hash-collision-budget-value\ 0$
 $(staged-alpha-search-queries\ budgets\ 0) + G +$
 $(P + staged-concrete-transcript-target-error-bound) +$
 $(P + staged-concrete-transcript-target-error-bound)))) +$
 $(trace-fri-error +$
 $low-degree-trace-pair-agreement-error\ budgets)) +$
 $trace-fri-error + empty-query-error)$
proof –
have $uncovered-bound:$
 $wp-event$
 $(checked-staged-security-experiment-with-actual-alpha-prefix-data-state$
 $A)$
 $checked-staged-security-with-actual-alpha-prefix-not-prefix-uncovered-index$
 $adversary-initial-state \leq G$
by $(rule\ order-trans$
 $[OF\ checked-staged-security-with-actual-alpha-prefix-not-prefix-uncovered-index-le-same-run-partial-opening-gap$
 $gap-bound])$
show $?thesis$
by $(rule$
 $stark-soundness-from-transcript-indexed-partial-opening-route-from-uncovered-and-transcript$
 $[OF\ false-statement\ wf\ controlled\ trace-fri-reduction$
 $composition-fri-reduction\ round-bound\ uncovered-bound\ data-pre-bound$
 $current-empty-bound])$
qed
end
end

theory *Soundness-Public-Route-Single-Query*

imports

Soundness-FRI-Verifier-Tied

Soundness-Not-Prefix-Partial-Opening

begin

Narrow downstream wrappers for the public route that use the transcript-indexed single-current-query charge. This theory depends directly on the partial-opening route needed by the final endpoint.

context *soundness*

begin

theorem *stark-soundness-from-transcript-indexed-partial-opening-route-from-sampled-transcript-relevant-drift-only-s*

fixes *P trace-fri-error' composition-fri-error'*

empty-trace-fri-error' empty-query-error :: prob

and *C :: nat $\Rightarrow$ prob*

assumes *false-statement: $\neg$ exists-valid-trace*

and *wf: staged-budget-wellformed budgets*

and *controlled: staged-adversary-controlled budgets A*

and *round-bound:*

$\bigwedge i. i < \text{rounds} \implies$

wp-event

(checked-staged-security-experiment-with-query-prefix-data-state A i)

(checked-staged-security-with-query-prefix-current-query-partial-opening-hit-at i)

adversary-initial-state $\leq C i$

and *data-pre-bound:*

wp-event (checked-staged-security-experiment-with-data-state A)

staged-security-with-data-state-transcript-pre-hit

adversary-initial-state $\leq P$

and *current-empty-bound:*

wp-event (checked-staged-security-experiment-with-data-state A)

(staged-security-with-data-state-current-empty-query-opening-hit-at 0)

adversary-initial-state $\leq$ empty-query-error

and *trace-fri-bound:*

wp-event (checked-staged-security-experiment-with-data-state A)

(staged-security-with-data-state-verifier-event

trace-fri-bad-with-header-tied-partial-candidate)

adversary-initial-state $\leq$ trace-fri-error'

and *comp-fri-bound:*

wp-event (checked-staged-security-experiment-with-data-state A)

(staged-security-with-data-state-verifier-event

composition-fri-bad-with-verifier-tied-partial-candidate)

adversary-initial-state $\leq$ composition-fri-error'

and *empty-trace-fri-bound:*

wp-event (checked-staged-security-experiment-with-data-state A)

(staged-security-with-data-state-verifier-event

trace-fri-bad-with-empty-composition-header-candidates)

adversary-initial-state $\leq$ empty-trace-fri-error'

shows *checked-staged-adversary-acceptance-probability* $A \leq$
hash-collision-budget-value 0
 (*staged-attacker-query-budget* *budgets* +
staged-challenge-query-budget + *verifier-hash-query-budget*) +
 (*composition-error-bound* +
staged-phase-relation-error *size*
 (*staged-alpha-search-queries* *budgets* 0) +
hash-collision-budget-value 0
 (*staged-alpha-search-queries* *budgets* 0) +
staged-phase-relation-error *size*
 (*staged-alpha-search-queries* *budgets* 0) +
 (P + *staged-concrete-transcript-target-error-bound*)) +
trace-fri-error' + *composition-fri-error'* +
 ($\sum i < \text{rounds}. C\ i$) +
 (*composition-error-bound* +
staged-phase-relation-error *size*
 (*staged-alpha-search-queries* *budgets* 0) +
hash-collision-budget-value 0
 (*staged-alpha-search-queries* *budgets* 0) +
staged-phase-relation-error *size*
 (*staged-alpha-search-queries* *budgets* 0) +
 (P + *staged-concrete-transcript-target-error-bound*) +
empty-trace-fri-error' + *empty-query-error*)

proof –

have *current-query-bound*:

wp-event (*checked-staged-security-experiment-with-data-state* A)
staged-security-with-data-state-current-query-partial-opening-hit
adversary-initial-state $\leq$ ($\sum i < \text{rounds}. C\ i$)

by (rule

checked-staged-security-with-data-state-current-query-partial-opening-hit-bound-from-query-prefix-rounds
 [*OF* *round-bound* *wf* *controlled*])

have *query-bound*:

wp-event (*checked-staged-security-experiment-with-data-state* A)
 (*staged-security-with-data-state-verifier-event*
query-bad-with-aligned-transcript-partial-candidates)
adversary-initial-state $\leq$ ($\sum i < \text{rounds}. C\ i$)

by (rule *order-trans*[

OF *checked-staged-security-with-data-state-aligned-transcript-query-bad-bound-from-current-partial-opening-hit*
current-query-bound])

(*use* *wf* *controlled* **in** *simp-all*)

have *transcript-new-bound*:

wp-event (*checked-staged-security-experiment-with-data-state* A)
staged-security-with-data-state-transcript-new-hit
adversary-initial-state $\leq$ *staged-concrete-transcript-target-error-bound*

by (rule

checked-staged-security-with-data-state-transcript-new-hit-bound
 [*OF* *wf* *controlled*])

have *drift-bound*:

wp-event

(checked-staged-security-experiment-with-actual-alpha-prefix-data-state A)
checked-staged-security-with-actual-alpha-prefix-relevant-partial-header-candidate-drift
adversary-initial-state $\leq$
P + staged-concrete-transcript-target-error-bound
by (rule
checked-staged-security-with-actual-alpha-prefix-relevant-partial-header-candidate-drift-bound-from-sampled-trans
[OF wf controlled data-pre-bound transcript-new-bound])
have *comp-bound*:
wp-event (checked-staged-security-experiment-with-data-state A)
(staged-security-with-data-state-verifier-event
composition-bad-with-partial-candidates)
adversary-initial-state $\leq$
composition-error-bound +
staged-phase-relation-error size
(staged-alpha-search-queries budgets 0) +
hash-collision-budget-value 0
(staged-alpha-search-queries budgets 0) +
staged-phase-relation-error size
(staged-alpha-search-queries budgets 0) +
(P + staged-concrete-transcript-target-error-bound)
by (rule
checked-staged-security-with-data-state-partial-candidate-composition-bad-bound-from-relevant-drift-and-budgets
[OF false-statement wf controlled drift-bound])
have *merkle-bound*:
wp-event (checked-staged-security-experiment-with-data-state A)
(staged-security-with-data-state-verifier-event
partial-merkle-inconsistency-bad)
adversary-initial-state $\leq$
hash-collision-budget-value 0
(staged-attacker-query-budget budgets +
staged-challenge-query-budget + verifier-hash-query-budget)
by (rule
checked-staged-security-with-data-state-partial-merkle-inconsistency-bad-bound-controlled
[OF wf controlled])
have *empty-query-bound*:
wp-event (checked-staged-security-experiment-with-data-state A)
(staged-security-with-data-state-verifier-event
query-bad-with-empty-composition-header-candidates)
adversary-initial-state $\leq$ *empty-query-error*
by (rule *order-trans*[
OF checked-staged-security-with-data-state-empty-query-bad-le-current-empty-opening-hit-at-zero
OF wf controlled] current-empty-bound])
have *empty-header-bound*:
wp-event (checked-staged-security-experiment-with-data-state A)
(staged-security-with-data-state-verifier-event
soundness-bad-event-partial-candidate-empty-header)
adversary-initial-state $\leq$
composition-error-bound +
staged-phase-relation-error size

```

    (staged-alpha-search-queries budgets 0) +
    hash-collision-budget-value 0
    (staged-alpha-search-queries budgets 0) +
    staged-phase-relation-error size
    (staged-alpha-search-queries budgets 0) +
    (P + staged-concrete-transcript-target-error-bound) +
    empty-trace-fri-error' + empty-query-error
  by (rule
    checked-staged-security-with-data-state-empty-header-bad-event-bound-from-relevant-drift-trace-fri-and-query
    [OF false-statement wf controlled drift-bound empty-trace-fri-bound
      empty-query-bound])
  show ?thesis
  by (rule
    checked-staged-soundness-from-verifier-tied-aligned-transcript-partial-candidate-components-and-empty-header-b
    [OF false-statement merkle-bound comp-bound trace-fri-bound
      comp-fri-bound query-bound empty-header-bound])
qed
end

```

These single-query-charge wrappers are downstream plumbing for the public same-run route. The visible theorem remains in *Soundness-Public-Route-Same-Run*.

```

hide-fact
  soundness.stark-soundness-from-transcript-indexed-partial-opening-route-from-sampled-transcript-relevant-drift-only
end

```

```

theory Soundness-FRI-Header-Tied-Reduction
  imports Soundness-FRI-Verifier-Tied
begin

```

Header-tied trace FRI reductions.

The active public route uses trace candidates tied to the verifier header root. This layer mirrors the verifier-tied sampled/missing decomposition without widening back to arbitrary authenticated roots.

```

context soundness
begin

```

```

lemma trace-fri-bad-with-header-tied-partial-candidate-None[simp]:
  ¬ trace-fri-bad-with-header-tied-partial-candidate s None
  unfolding trace-fri-bad-with-header-tied-partial-candidate-def
    accepted-fri-opening-transcript-def
  by simp

```

```

definition trace-fri-bad-with-header-tied-sampled-layer-chain
  :: ('f, 'a) protocol-channel-scheme ⇒
    (unit list × ('f, 'a) protocol-channel-scheme) option ⇒ bool

```


where

$\text{trace-fri-bad-with-header-tied-sampled-layer-chain } s \text{ out} \longleftrightarrow$
 $(\exists \text{ trace-roots trace-bs trace-final dg composition-roots}$
 $\text{composition-bs composition-final fri-query-idxs trace-round-layers}$
 $\text{composition-round-layers fr as rest trace-openings trace-table}$
 doms layers.
 $\text{accepted-fri-opening-transcript } s \text{ out trace-roots trace-bs}$
 $\text{trace-final dg composition-roots composition-bs composition-final}$
 $\text{fri-query-idxs trace-round-layers composition-round-layers} \wedge$
 $\text{verifier-header-transcript } s \text{ fr trace-roots trace-final as dg}$
 $\text{composition-roots composition-final rest} \wedge$
 $\text{accepted-with-partial-trace-openings } s \text{ out fr fri-query-idxs}$
 $\text{trace-openings} \wedge$
 $\text{partial-trace-table-candidate trace-table trace-openings} \wedge$
 $\neg \text{trace-table-low-degree trace-table} \wedge$
 $\text{generic-fri-sampled-layer-chain-evidence trace-table-low-degree}$
 $\text{trace-table (clength - 1) trace-roots trace-bs trace-final}$
 $\text{fri-query-idxs trace-round-layers doms layers})$

definition *trace-fri-header-tied-missing-sampled-layer-chain*

$:: (f, 'a) \text{ protocol-channel-scheme} \Rightarrow$
 $(\text{unit list} \times (f, 'a) \text{ protocol-channel-scheme}) \text{ option} \Rightarrow \text{bool}$

where

$\text{trace-fri-header-tied-missing-sampled-layer-chain } s \text{ out} \longleftrightarrow$
 $\text{trace-fri-bad-with-header-tied-partial-candidate } s \text{ out} \wedge$
 $\neg \text{trace-fri-bad-with-header-tied-sampled-layer-chain } s \text{ out}$

definition *trace-fri-header-tied-sampled-layer-assignment-obstruction*

$:: (f, 'a) \text{ protocol-channel-scheme} \Rightarrow$
 $(\text{unit list} \times (f, 'a) \text{ protocol-channel-scheme}) \text{ option} \Rightarrow \text{bool}$

where

$\text{trace-fri-header-tied-sampled-layer-assignment-obstruction } s \text{ out}$
 $\longleftrightarrow$
 $(\exists \text{ trace-roots trace-bs trace-final dg composition-roots}$
 $\text{composition-bs composition-final fri-query-idxs trace-round-layers}$
 $\text{composition-round-layers fr as rest trace-openings trace-table.}$
 $\text{accepted-fri-opening-transcript } s \text{ out trace-roots trace-bs}$
 $\text{trace-final dg composition-roots composition-bs composition-final}$
 $\text{fri-query-idxs trace-round-layers composition-round-layers} \wedge$
 $\text{verifier-header-transcript } s \text{ fr trace-roots trace-final as dg}$
 $\text{composition-roots composition-final rest} \wedge$
 $\text{accepted-with-partial-trace-openings } s \text{ out fr fri-query-idxs}$
 $\text{trace-openings} \wedge$
 $\text{partial-trace-table-candidate trace-table trace-openings} \wedge$
 $\neg \text{trace-table-low-degree trace-table} \wedge$
 $\text{generic-fri-sampled-layer-assignment-obstruction}$
 $\text{trace-table-low-degree trace-table (clength - 1) trace-roots}$
 $\text{trace-bs trace-final fri-query-idxs trace-round-layers})$

definition *trace-fri-header-tied-sampled-assignment-conflict*
 $:: ('f, 'a) \text{ protocol-channel-scheme} \Rightarrow$
 $(\text{unit list} \times ('f, 'a) \text{ protocol-channel-scheme}) \text{ option} \Rightarrow \text{bool}$

where
 $\text{trace-fri-header-tied-sampled-assignment-conflict } s \text{ out}$
 $\longleftrightarrow$
 $(\exists \text{ trace-roots trace-bs trace-final dg composition-roots}$
 $\text{ composition-bs composition-final fri-query-idxs trace-round-layers}$
 $\text{ composition-round-layers fr as rest trace-openings trace-table.}$
 $\text{accepted-fri-opening-transcript } s \text{ out trace-roots trace-bs}$
 $\text{ trace-final dg composition-roots composition-bs composition-final}$
 $\text{ fri-query-idxs trace-round-layers composition-round-layers} \wedge$
 $\text{verifier-header-transcript } s \text{ fr trace-roots trace-final as dg}$
 $\text{ composition-roots composition-final rest} \wedge$
 $\text{accepted-with-partial-trace-openings } s \text{ out fr fri-query-idxs}$
 $\text{ trace-openings} \wedge$
 $\text{partial-trace-table-candidate trace-table trace-openings} \wedge$
 $\neg \text{ trace-table-low-degree trace-table} \wedge$
 $\text{generic-fri-sampled-assignment-conflict trace-table trace-roots}$
 $\text{ trace-bs trace-final fri-query-idxs trace-round-layers})$

lemma *trace-fri-header-tied-sampled-assignment-conflict-imp-sampled:*
assumes
 $\text{trace-fri-header-tied-sampled-assignment-conflict } s \text{ out}$
shows $\text{trace-fri-sampled-assignment-conflict } s \text{ out}$

proof —
from *assms* **obtain** $\text{trace-roots trace-bs trace-final dg composition-roots}$
 $\text{ composition-bs composition-final fri-query-idxs trace-round-layers}$
 $\text{ composition-round-layers fr as rest trace-openings trace-table}$
where *fri-openings:*
 $\text{accepted-fri-opening-transcript } s \text{ out trace-roots trace-bs}$
 $\text{ trace-final dg composition-roots composition-bs composition-final}$
 $\text{ fri-query-idxs trace-round-layers composition-round-layers}$
and *partial:*
 $\text{accepted-with-partial-trace-openings } s \text{ out fr fri-query-idxs}$
 trace-openings
and *cand:* $\text{partial-trace-table-candidate trace-table trace-openings}$
and *not-low:* $\neg \text{ trace-table-low-degree trace-table}$
and *conflict:*
 $\text{generic-fri-sampled-assignment-conflict trace-table trace-roots}$
 $\text{ trace-bs trace-final fri-query-idxs trace-round-layers}$
unfolding *trace-fri-header-tied-sampled-assignment-conflict-def*
by *blast*
have *evidence:*
 $\text{trace-fri-partial-candidate-opening-evidence } s \text{ out trace-roots}$
 $\text{ trace-bs trace-final dg composition-roots composition-bs}$
 $\text{ composition-final fri-query-idxs trace-round-layers}$
 $\text{ composition-round-layers fr fri-query-idxs trace-openings}$
 trace-table

```

unfolding trace-fri-partial-candidate-opening-evidence-def
  trace-fri-partial-candidate-evidence-def
by (intro conjI fri-openings partial cand not-low)
show ?thesis
unfolding trace-fri-sampled-assignment-conflict-def
by (intro exI conjI)
  (rule evidence, rule conflict)
qed

lemma wp-trace-fri-header-tied-sampled-assignment-conflict-bound-from-sampled:
assumes sampled-bound:
  wp-event verify-monad (trace-fri-sampled-assignment-conflict s)  $s \leq C$ 
shows
  wp-event verify-monad
    (trace-fri-header-tied-sampled-assignment-conflict s)  $s \leq C$ 
proof -
have wp-event verify-monad
  (trace-fri-header-tied-sampled-assignment-conflict s)  $s \leq$ 
  wp-event verify-monad (trace-fri-sampled-assignment-conflict s) s
by (rule wp-event-mono)
  (rule trace-fri-header-tied-sampled-assignment-conflict-imp-sampled)
then show ?thesis
by (rule order-trans[OF - sampled-bound])
qed

lemma trace-fri-header-tied-missing-sampled-imp-assignment-obstruction:
assumes missing:
  trace-fri-header-tied-missing-sampled-layer-chain s out
shows
  trace-fri-header-tied-sampled-layer-assignment-obstruction s out
proof -
have bad:
  trace-fri-bad-with-header-tied-partial-candidate s out
and not-sampled:
   $\neg$  trace-fri-bad-with-header-tied-sampled-layer-chain s out
using missing
unfolding trace-fri-header-tied-missing-sampled-layer-chain-def
by blast+
from bad obtain trace-roots trace-bs trace-final dg composition-roots
  composition-bs composition-final fri-query-idxs trace-round-layers
  composition-round-layers fr as rest trace-openings trace-table
where fri-openings:
  accepted-fri-opening-transcript s out trace-roots trace-bs
  trace-final dg composition-roots composition-bs composition-final
  fri-query-idxs trace-round-layers composition-round-layers
and header:
  verifier-header-transcript s fr trace-roots trace-final as dg
  composition-roots composition-final rest
and partial:

```

accepted-with-partial-trace-openings s out fr fri-query-idxs
trace-openings
and *cand: partial-trace-table-candidate trace-table trace-openings*
and *not-low: $\neg$ trace-table-low-degree trace-table*
unfolding *trace-fri-bad-with-header-tied-partial-candidate-def*
by *blast*
have *partial-generic:*
generic-fri-partial-evidence trace-table-low-degree trace-table
(clength - 1) trace-roots trace-bs trace-final fri-query-idxs
trace-round-layers
by *(rule accepted-fri-opening-transcript-trace-generic-partial-evidence*
[OF fri-openings])
have *no-chain:*
 $\bigwedge$ *doms layers. $\neg$*
generic-fri-sampled-layer-chain-evidence trace-table-low-degree
trace-table (clength - 1) trace-roots trace-bs trace-final
fri-query-idxs trace-round-layers doms layers
proof
fix *doms layers*
assume *chain:*
generic-fri-sampled-layer-chain-evidence trace-table-low-degree
trace-table (clength - 1) trace-roots trace-bs trace-final
fri-query-idxs trace-round-layers doms layers
have *trace-fri-bad-with-header-tied-sampled-layer-chain s out*
unfolding *trace-fri-bad-with-header-tied-sampled-layer-chain-def*
by *(intro exI conjI)*
(rule fri-openings, rule header, rule partial, rule cand,
rule not-low, rule chain)
then show *False*
using *not-sampled by contradiction*
qed
have *obstruction:*
generic-fri-sampled-layer-assignment-obstruction
trace-table-low-degree trace-table (clength - 1) trace-roots
trace-bs trace-final fri-query-idxs trace-round-layers
unfolding *generic-fri-sampled-layer-assignment-obstruction-def*
using *partial-generic no-chain by blast*
show *?thesis*
unfolding *trace-fri-header-tied-sampled-layer-assignment-obstruction-def*
by *(intro exI conjI)*
(rule fri-openings, rule header, rule partial, rule cand,
rule not-low, rule obstruction)
qed
lemma *trace-fri-header-tied-assignment-obstruction-imp-conflict-or-zero:*
assumes *obstruction:*
trace-fri-header-tied-sampled-layer-assignment-obstruction s out
shows
trace-fri-header-tied-sampled-assignment-conflict s out $\vee$

trace-fri-header-tied-zero-round-final-obstruction s out
proof –
obtain *trace-roots trace-bs' trace-final dg composition-roots*
composition-bs composition-final fri-query-idxs trace-round-layers
composition-round-layers fr as rest trace-openings trace-table
where *fri-openings*:
accepted-fri-opening-transcript s out trace-roots trace-bs'
trace-final dg composition-roots composition-bs composition-final
fri-query-idxs trace-round-layers composition-round-layers
and *header*:
verifier-header-transcript s fr trace-roots trace-final as dg
composition-roots composition-final rest
and *partial*:
accepted-with-partial-trace-openings s out fr fri-query-idxs
trace-openings
and *cand*: *partial-trace-table-candidate trace-table trace-openings*
and *not-low*: $\neg$ *trace-table-low-degree trace-table*
and *generic*:
generic-fri-sampled-layer-assignment-obstruction
trace-table-low-degree trace-table (clength – 1) trace-roots
trace-bs' trace-final fri-query-idxs trace-round-layers
using *obstruction*
unfolding *trace-fri-header-tied-sampled-layer-assignment-obstruction-def*
by *blast*
obtain *N* **where** *eval-power*: $\text{clength} * \text{scale} = 2^N$
using *eval-domain-length-power* **by** *blast*
have *trace-rounds-le*: $\text{length trace-bs}' \leq N$
proof –
have *len*: $\text{length trace-bs}' = \text{ceil-log clength}$
using *accepted-fri-opening-transcript-shapes(1,2)[OF fri-openings]*
by *simp*
have $\text{clength} \leq \text{length} * \text{scale}$
using *scale-pos* **by** *simp*
also have $\dots = 2^N$
by *(rule eval-power)*
finally have $\text{ceil-log clength} \leq N$
by *(rule ceil-log-le-power)*
then show *?thesis*
using *len* **by** *simp*
qed
have *query-bounds'*:
 $\bigwedge \text{round-idx. round-idx} < \text{length fri-query-idxs} \implies$
 $\text{fri-query-idxs} ! \text{round-idx} < \text{clength} * \text{scale}$
proof –
fix *round-idx*
assume *round-bound*: $\text{round-idx} < \text{length fri-query-idxs}$
have *fri-query-idxs* ! $\text{round-idx} \in \text{set fri-query-idxs}$
using *round-bound* **by** *simp*
then show *fri-query-idxs* ! $\text{round-idx} < \text{clength} * \text{scale}$

```

    by (rule accepted-fri-opening-transcript-query-idx-bound
        [OF fri-openings])
qed
have split:
  generic-fri-sampled-assignment-conflict trace-table trace-roots
    trace-bs' trace-final fri-query-idxs trace-round-layers  $\vee$ 
    length trace-bs' = 0  $\wedge$ 
     $\neg$  fri-final-constant-consistent trace-table trace-final
  by (rule
      generic-fri-sampled-layer-assignment-obstruction-imp-conflict-or-zero-round-final
      [OF generic query-bounds' eval-power trace-rounds-le])
then show ?thesis
proof
  assume conflict:
    generic-fri-sampled-assignment-conflict trace-table trace-roots
      trace-bs' trace-final fri-query-idxs trace-round-layers
  then show ?thesis
    unfolding trace-fri-header-tied-sampled-assignment-conflict-def
    by (intro disjI1 exI conjI)
      (rule fri-openings, rule header, rule partial, rule cand,
        rule not-low, rule conflict)
next
  assume zero:
    length trace-bs' = 0  $\wedge$ 
     $\neg$  fri-final-constant-consistent trace-table trace-final
  then show ?thesis
    unfolding trace-fri-header-tied-zero-round-final-obstruction-def
    by (intro disjI2 exI conjI)
      (rule fri-openings, rule header, rule partial, rule cand,
        rule not-low,
        simp-all)
qed
qed

lemma wp-trace-fri-header-tied-missing-sampled-bound-from-assignment-obstruction:
  assumes assignment-bound:
    wp-event verify-monad
      (trace-fri-header-tied-sampled-layer-assignment-obstruction s) s
       $\leq M$ 
  shows
    wp-event verify-monad
      (trace-fri-header-tied-missing-sampled-layer-chain s) s  $\leq M$ 
proof -
  have wp-event verify-monad
    (trace-fri-header-tied-missing-sampled-layer-chain s) s  $\leq$ 
    wp-event verify-monad
      (trace-fri-header-tied-sampled-layer-assignment-obstruction s) s
  by (rule wp-event-mono)
    (rule trace-fri-header-tied-missing-sampled-imp-assignment-obstruction)

```

then show *?thesis*
by (rule order-trans[*OF - assignment-bound*])
qed

lemma *wp-trace-fri-header-tied-bound-from-sampled-and-missing*:
assumes *sampld-bound*:
wp-event verify-monad
(trace-fri-bad-with-header-tied-sampled-layer-chain s) s ≤ R
and *missing-bound*:
wp-event verify-monad
(trace-fri-header-tied-missing-sampled-layer-chain s) s ≤ M
shows
wp-event verify-monad
(trace-fri-bad-with-header-tied-partial-candidate s) s ≤ R + M
proof –
have *wp-event verify-monad*
(trace-fri-bad-with-header-tied-partial-candidate s) s ≤
wp-event verify-monad
(λout. trace-fri-bad-with-header-tied-sampled-layer-chain s out ∨
trace-fri-header-tied-missing-sampled-layer-chain s out) s
unfolding *trace-fri-header-tied-missing-sampled-layer-chain-def*
by (rule *wp-event-mono*) *blast*
also have ... ≤
wp-event verify-monad
(trace-fri-bad-with-header-tied-sampled-layer-chain s) s +
wp-event verify-monad
(trace-fri-header-tied-missing-sampled-layer-chain s) s
by (rule *wp-event-union-bound*)
also have ... ≤ *R + M*
by (rule *add-mono*[*OF sampld-bound missing-bound*])
finally show *?thesis* .
qed

lemma *wp-trace-fri-header-tied-assignment-obstruction-bound-from-conflict-and-zero*:
assumes *conflict-bound*:
wp-event verify-monad
(trace-fri-header-tied-sampled-assignment-conflict s) s ≤ C
and *zero-bound*:
wp-event verify-monad
(trace-fri-header-tied-zero-round-final-obstruction s) s ≤ Z
shows
wp-event verify-monad
(trace-fri-header-tied-sampled-layer-assignment-obstruction s) s
≤ C + Z
proof –
have *wp-event verify-monad*
(trace-fri-header-tied-sampled-layer-assignment-obstruction s) s
≤ wp-event verify-monad
(λout. trace-fri-header-tied-sampled-assignment-conflict s out ∨

$\text{trace-fri-header-tied-zero-round-final-obstruction } s \text{ out}) \ s$
by (rule *wp-event-mono*)
 $(\text{rule } \text{trace-fri-header-tied-assignment-obstruction-imp-conflict-or-zero})$
also have $\dots \leq$
 $\text{wp-event verify-monad}$
 $(\text{trace-fri-header-tied-sampled-assignment-conflict } s) \ s +$
 $\text{wp-event verify-monad}$
 $(\text{trace-fri-header-tied-zero-round-final-obstruction } s) \ s$
by (rule *wp-event-union-bound*)
also have $\dots \leq C + Z$
by (rule *add-mono*[*OF conflict-bound zero-bound*])
finally show *?thesis* .
qed

lemma *wp-trace-fri-header-tied-bound-from-sampled-conflict-and-zero*:
assumes *sampler-bound*:
 $\text{wp-event verify-monad}$
 $(\text{trace-fri-bad-with-header-tied-sampled-layer-chain } s) \ s \leq R$
and *conflict-bound*:
 $\text{wp-event verify-monad}$
 $(\text{trace-fri-header-tied-sampled-assignment-conflict } s) \ s \leq C$
and *zero-bound*:
 $\text{wp-event verify-monad}$
 $(\text{trace-fri-header-tied-zero-round-final-obstruction } s) \ s \leq Z$
shows
 $\text{wp-event verify-monad}$
 $(\text{trace-fri-bad-with-header-tied-partial-candidate } s) \ s$
 $\leq R + (C + Z)$
by (rule *wp-trace-fri-header-tied-bound-from-sampled-and-missing*
 $[\text{OF } \text{sampler-bound}$
 $\text{wp-trace-fri-header-tied-missing-sampled-bound-from-assignment-obstruction}$
 $[\text{OF } \text{wp-trace-fri-header-tied-assignment-obstruction-bound-from-conflict-and-zero}$
 $[\text{OF } \text{conflict-bound zero-bound}]]])$

lemma *trace-fri-bad-with-header-tied-partial-candidates-staged-bound*:
assumes *verifier-bound*:
 $\bigwedge \text{data attacker-state.}$
 $\text{Some } (\text{data}, \text{attacker-state}) \in$
 $\text{set-dist } (\text{execute } (\text{checked-staged-transcript-program } A)$
 $\text{adversary-initial-state}) \implies$
 $\text{wp-event verify-monad}$
 $(\text{trace-fri-bad-with-header-tied-partial-candidate}$
 $(\text{verifier-state-from-adversary attacker-state}$
 $(\text{staged-proof-transcript data})))$
 $(\text{verifier-state-from-adversary attacker-state}$
 $(\text{staged-proof-transcript data})) \leq C$
shows
 $\text{wp-event } (\text{checked-staged-security-experiment-with-data-state } A)$
 $(\text{staged-security-with-data-state-verifier-event}$


```

    trace-fri-bad-with-header-tied-partial-candidate)
  adversary-initial-state  $\leq C$ 
by (rule checked-staged-security-with-data-state-verifier-event-bound-from-cont)
  (simp-all add: verifier-bound)

lemma wp-trace-fri-header-tied-zero-round-bound-from-checked-and-merkle:
assumes checked-bound:
  wp-event verify-monad
  (trace-fri-zero-round-checked-final-obstruction s)  $s \leq Z$ 
and merkle-bound:
  wp-event verify-monad (partial-merkle-inconsistency-bad s)  $s \leq M$ 
shows
  wp-event verify-monad
  (trace-fri-header-tied-zero-round-final-obstruction s)  $s \leq Z + M$ 
proof -
have wp-event verify-monad
  (trace-fri-header-tied-zero-round-final-obstruction s)  $s \leq$ 
  wp-event verify-monad
  ( $\lambda$ out. trace-fri-zero-round-checked-final-obstruction s out  $\vee$ 
    partial-merkle-inconsistency-bad s out) s
by (rule wp-event-mono)
  (rule trace-fri-header-tied-zero-round-final-obstruction-imp-checked-or-merkle)
also have ...  $\leq$ 
  wp-event verify-monad
  (trace-fri-zero-round-checked-final-obstruction s)  $s +$ 
  wp-event verify-monad (partial-merkle-inconsistency-bad s) s
by (rule wp-event-union-bound)
also have ...  $\leq Z + M$ 
by (rule add-mono[OF checked-bound merkle-bound])
finally show ?thesis .
qed

end

end

```

```

theory Soundness-FRI-Layer-Merkle
imports Soundness-FRI-Header-Tied-Reduction
begin

```

Authenticated-opening facts for sampled FRI layer chunks.

The sampled-assignment conflict predicates are stated over transcript chunks. This layer connects those chunks back to the Merkle checks performed by the verifier. It is intentionally local: it extracts only sampled openings from the verifier execution and does not reconstruct complete FRI tables.

```

context soundness
begin

```

definition *fri-layer-chunk-authenticated*

$:: 'f \Rightarrow \text{nat} \Rightarrow \text{nat} \Rightarrow 'f \text{ list} \Rightarrow$
 $('f, 'a) \text{ protocol-channel-scheme} \Rightarrow \text{bool}$

where

fri-layer-chunk-authenticated *rt len idx chunk s* $\longleftrightarrow$
 $(\exists xp \text{ xp-path } xn \text{ xn-path}.$
 $\text{fri-layer-opening-chunk } \text{len } xp \text{ xp-path } xn \text{ xn-path } \text{chunk} \wedge$
 $\text{authenticated-opening-in } s$
 $(\parallel \text{opening-root} = \text{rt},$
 $\text{opening-length} = \text{len},$
 $\text{opening-index} = \text{idx},$
 $\text{opening-value} = xp,$
 $\text{opening-path} = \text{xp-path} \parallel) \wedge$
 $\text{authenticated-opening-in } s$
 $(\parallel \text{opening-root} = \text{rt},$
 $\text{opening-length} = \text{len},$
 $\text{opening-index} = \text{fri-sibling-index } \text{len } \text{idx},$
 $\text{opening-value} = xn,$
 $\text{opening-path} = \text{xn-path} \parallel))$

definition *fri-mfold-chunks-authenticated*

$:: ('f \times 'f) \text{ list} \Rightarrow \text{nat} \Rightarrow \text{nat} \Rightarrow 'f \text{ list list} \Rightarrow$
 $('f, 'a) \text{ protocol-channel-scheme} \Rightarrow \text{bool}$

where

fri-mfold-chunks-authenticated *bfs len idx chunks s* $\longleftrightarrow$
 $\text{length } \text{chunks} = \text{length } \text{bfs} \wedge$
 $(\forall j < \text{length } \text{bfs}.$
 $\text{fri-layer-chunk-authenticated } (\text{snd } (\text{bfs } ! j))$
 $(\text{fri-layer-lengths } (\text{length } \text{bfs}) \text{ len } ! j)$
 $(\text{fri-layer-indices } (\text{length } \text{bfs}) \text{ idx } \text{len } ! j)$
 $(\text{chunks } ! j) \text{ s})$

lemma *fri-layers-transcript-nth-length:*

assumes *layers:* *fri-layers-transcript* *n len layer-chunks chunk*

and *j-bound:* *j < n*

shows

$\text{length } (\text{layer-chunks } ! j) =$
 $2 + 2 * \text{floor-log } (\text{fri-layer-lengths } n \text{ len } ! j)$

proof –

from *fri-layers-transcript-nth-opening*[*OF layers j-bound*]

obtain *xp xp-path xn xn-path* **where**

layer-open:

$\text{fri-layer-opening-chunk } (\text{fri-layer-lengths } n \text{ len } ! j)$

$xp \text{ xp-path } xn \text{ xn-path } (\text{layer-chunks } ! j)$

by *blast*

show *?thesis*

using *fri-layer-opening-chunk-length*[*OF layer-open*] .

qed

lemma *fri-layers-transcript-unique*:
assumes *left*: *fri-layers-transcript* *n* *len* *layer-chunks* *chunk*
and *right*: *fri-layers-transcript* *n* *len* *layer-chunks'* *chunk*
shows *layer-chunks* = *layer-chunks'*
proof –
have *len-left*: *length layer-chunks* = *n*
using *left* **unfolding** *fri-layers-transcript-def* **by** *simp*
have *len-right*: *length layer-chunks'* = *n*
using *right* **unfolding** *fri-layers-transcript-def* **by** *simp*
have *concat-eq*:
List.concat layer-chunks @ [] = List.concat layer-chunks'
using *left right* **unfolding** *fri-layers-transcript-def* **by** *simp*
have *same-lens*:
 $\bigwedge j. j < \text{length } \text{layer-chunks} \implies$
 $\text{length } (\text{layer-chunks} ! j) = \text{length } (\text{layer-chunks}' ! j)$
using *fri-layers-transcript-nth-length* [*OF left*]
fri-layers-transcript-nth-length [*OF right*]
len-left len-right
by *simp*
have *layer-chunks* = *layer-chunks'* $\wedge$ ($[] :: 'f \text{ list} = []$)
by (*rule concat-append-eq-concat-same-chunk-lengths*
[*OF - same-lens concat-eq*])
(*use len-left len-right in simp*)
then show *?thesis*
by *simp*
qed

lemma *query-round-fri-layer-transcripts-unique*:
assumes *left*:
query-round-fri-layer-transcripts idx trace-roots composition-roots
chunk trace-layers composition-layers
and *right*:
query-round-fri-layer-transcripts idx trace-roots composition-roots
chunk trace-layers' composition-layers'
shows *trace-layers* = *trace-layers'*
and *composition-layers* = *composition-layers'*
proof –
from *left* **obtain** *query-chunk trace-fri-chunk composition-fri-chunk*
leaves paths **where**
chunk-left:
chunk = *query-chunk @ trace-fri-chunk @ composition-fri-chunk*
and *query-left*:
query-decommitment-transcript idx leaves paths query-chunk
and *trace-left*:
*fri-layers-transcript (length trace-roots) (clength * scale)*
trace-layers trace-fri-chunk
and *composition-left*:
*fri-layers-transcript (length composition-roots) (clength * scale)*
composition-layers composition-fri-chunk

```

unfolding query-round-fri-layer-transcripts-def by blast
from right obtain query-chunk' trace-fri-chunk' composition-fri-chunk'
  leaves' paths' where
    chunk-right:
      chunk = query-chunk' @ trace-fri-chunk' @ composition-fri-chunk'
and query-right:
      query-decommitment-transcript idx leaves' paths' query-chunk'
and trace-right:
      fri-layers-transcript (length trace-roots) (clength * scale)
        trace-layers' trace-fri-chunk'
and composition-right:
      fri-layers-transcript (length composition-roots) (clength * scale)
        composition-layers' composition-fri-chunk'
unfolding query-round-fri-layer-transcripts-def by blast
have len-query:
  length query-chunk = length query-chunk'
using query-decommitment-transcript-length[OF query-left]
  query-decommitment-transcript-length[OF query-right]
by simp
have len-trace:
  length trace-fri-chunk = length trace-fri-chunk'
using fri-layers-transcript-length[OF trace-left]
  fri-layers-transcript-length[OF trace-right]
by simp
have trace-chunk-eq: trace-fri-chunk = trace-fri-chunk'
proof -
  have left-drop:
    drop (length query-chunk) chunk =
      trace-fri-chunk @ composition-fri-chunk
    using chunk-left by simp
  have right-drop:
    drop (length query-chunk) chunk =
      trace-fri-chunk' @ composition-fri-chunk'
    using chunk-right len-query by simp
  have left-take:
    trace-fri-chunk =
      take (length trace-fri-chunk) (drop (length query-chunk) chunk)
    using left-drop by simp
  have right-take:
    trace-fri-chunk' =
      take (length trace-fri-chunk') (drop (length query-chunk) chunk)
    using right-drop by simp
  show ?thesis
    using left-take right-take len-trace by simp
qed
have composition-chunk-eq:
  composition-fri-chunk = composition-fri-chunk'
proof -
  have left-drop:

```

```

    drop (length query-chunk + length trace-fri-chunk) chunk =
      composition-fri-chunk
  using chunk-left by simp
have right-drop:
  drop (length query-chunk + length trace-fri-chunk) chunk =
    composition-fri-chunk'
  using chunk-right len-query len-trace by simp
show ?thesis
  using left-drop right-drop by simp
qed
have trace-right':
  fri-layers-transcript (length trace-roots) (clength * scale)
    trace-layers' trace-fri-chunk
  using trace-right trace-chunk-eq by simp
have composition-right':
  fri-layers-transcript (length composition-roots) (clength * scale)
    composition-layers' composition-fri-chunk
  using composition-right composition-chunk-eq by simp
show trace-layers = trace-layers'
  by (rule fri-layers-transcript-unique[OF trace-left trace-right'])
show composition-layers = composition-layers'
  by (rule fri-layers-transcript-unique
      [OF composition-left composition-right'])
qed

lemma fri-layer-chunk-authenticatedI:
  assumes fri-layer-opening-chunk len xp xp-path xn xn-path chunk
  and authenticated-opening-in s
    (opening-root = rt,
     opening-length = len,
     opening-index = idx,
     opening-value = xp,
     opening-path = xp-path)
  and authenticated-opening-in s
    (opening-root = rt,
     opening-length = len,
     opening-index = fri-sibling-index len idx,
     opening-value = xn,
     opening-path = xn-path)
  shows fri-layer-chunk-authenticated rt len idx chunk s
  using assms unfolding fri-layer-chunk-authenticated-def by blast

lemma fri-layer-chunk-authenticatedE:
  assumes fri-layer-chunk-authenticated rt len idx chunk s
  obtains xp xp-path xn xn-path where
    fri-layer-opening-chunk len xp xp-path xn xn-path chunk
    authenticated-opening-in s
      (opening-root = rt,
       opening-length = len,

```

```

    opening-index = idx,
    opening-value = xp,
    opening-path = xp-path)
authenticated-opening-in s
  (|opening-root = rt,
    opening-length = len,
    opening-index = fri-sibling-index len idx,
    opening-value = xn,
    opening-path = xn-path|)
using assms unfolding fri-layer-chunk-authenticated-def by blast

```

lemma *fri-layer-chunk-authenticated-matching-values*:

assumes *auth*: *fri-layer-chunk-authenticated* *rt len idx chunk s*
and *chunk*:

fri-layer-opening-chunk len yp yp-path yn yn-path chunk

obtains *xp-path xn-path* **where**

fri-layer-opening-chunk len yp xp-path yn xn-path chunk

authenticated-opening-in s

(|opening-root = rt,
 opening-length = len,
 opening-index = idx,
 opening-value = yp,
 opening-path = xp-path|)

authenticated-opening-in s

(|opening-root = rt,
 opening-length = len,
 opening-index = fri-sibling-index len idx,
 opening-value = yn,
 opening-path = xn-path|)

proof –

from *auth* **obtain** *xp xp-path xn xn-path* **where**

chunk': *fri-layer-opening-chunk len xp xp-path xn xn-path chunk*

and *xp-auth*:

authenticated-opening-in s

(|opening-root = rt,
 opening-length = len,
 opening-index = idx,
 opening-value = xp,
 opening-path = xp-path|)

and *xn-auth*:

authenticated-opening-in s

(|opening-root = rt,
 opening-length = len,
 opening-index = fri-sibling-index len idx,
 opening-value = xn,
 opening-path = xn-path|)

by (*rule fri-layer-chunk-authenticatedE*)

have *xp-eq*: *xp = yp*

by (*rule fri-layer-opening-chunk-values-unique(1)*[*OF chunk' chunk*])

have $xn\text{-eq}$: $xn = yn$
by (rule *fri-layer-opening-chunk-values-unique*(2)[*OF chunk' chunk*])
have $chunk\text{-y}$:
fri-layer-opening-chunk len yp xp-path yn xn-path chunk
using $chunk'$ $xp\text{-eq}$ $xn\text{-eq}$ **by** *simp*
show *?thesis*
by (rule *that*[*OF chunk-y*])
 (use $xp\text{-auth}$ $xn\text{-auth}$ $xp\text{-eq}$ $xn\text{-eq}$ **in** *simp-all*)
qed

lemma *authenticated-openings-values-eq-if-no-partial-merkle*:
assumes *no-bad*:
 $\neg$ *partial-merkle-inconsistency-bad* s
 (Some (result, final-state))
and *first*: *authenticated-opening-in final-state opn*
and *second*: *authenticated-opening-in final-state opn'*
and *same-root*: *opening-root opn = opening-root opn'*
and *same-length*: *opening-length opn = opening-length opn'*
and *same-index*: *opening-index opn = opening-index opn'*
shows *opening-value opn = opening-value opn'*
proof (rule *ccontr*)
assume *neg*: *opening-value opn $\neq$ opening-value opn'*
have *partial-merkle-inconsistency-bad* s
 (Some (result, final-state))
unfolding *partial-merkle-inconsistency-bad-def* *accepted-def*
by (intro *conjI* *exI*[*of - result*] *exI*[*of - final-state*]
exI[*of - opn*] *exI*[*of - opn'*])
 (use *first second same-root same-length same-index neg* **in** *simp-all*)
then show *False* **using** *no-bad* **by** *contradiction*
qed

lemma *fri-layer-authenticated-base-values-eq-if-no-partial-merkle*:
assumes *no-bad*:
 $\neg$ *partial-merkle-inconsistency-bad* s
 (Some (result, final-state))
and *auth*:
fri-layer-chunk-authenticated rt len idx chunk final-state
and *auth'*:
fri-layer-chunk-authenticated rt len idx chunk' final-state
and *chunk*:
fri-layer-opening-chunk len xp xp-path xn xn-path chunk
and *chunk'*:
fri-layer-opening-chunk len yp yp-path yn yn-path chunk'
shows $xp = yp$
proof (rule *ccontr*)
assume *neg*: $xp \neq yp$
from *fri-layer-chunk-authenticated-matching-values*[*OF auth chunk*]
obtain $xp\text{-path}'$ $xn\text{-path}'$ **where**
xp-auth:

```

    authenticated-opening-in final-state
    (opening-root = rt,
     opening-length = len,
     opening-index = idx,
     opening-value = xp,
     opening-path = xp-path')
  by blast
from fri-layer-chunk-authenticated-matching-values[OF auth' chunk']
obtain yp-path' yn-path' where
  yp-auth:
    authenticated-opening-in final-state
    (opening-root = rt,
     opening-length = len,
     opening-index = idx,
     opening-value = yp,
     opening-path = yp-path')
  by blast
have partial-merkle-inconsistency-bad s
  (Some (result, final-state))
  unfolding partial-merkle-inconsistency-bad-def accepted-def
  by (intro conjI exI[of - result] exI[of - final-state]
      exI[of - (opening-root = rt,
                opening-length = len,
                opening-index = idx,
                opening-value = xp,
                opening-path = xp-path')])
      exI[of - (opening-root = rt,
                opening-length = len,
                opening-index = idx,
                opening-value = yp,
                opening-path = yp-path')])
    (use xp-auth yp-auth neq in simp-all)
  then show False using no-bad by contradiction
qed

lemma fri-layer-authenticated-sibling-values-eq-if-no-partial-merkle:
  assumes no-bad:
    ¬ partial-merkle-inconsistency-bad s
    (Some (result, final-state))
  and auth:
    fri-layer-chunk-authenticated rt len idx chunk final-state
  and auth':
    fri-layer-chunk-authenticated rt len idx chunk' final-state
  and chunk:
    fri-layer-opening-chunk len xp xp-path xn xn-path chunk
  and chunk':
    fri-layer-opening-chunk len yp yp-path yn yn-path chunk'
  shows xn = yn
proof (rule ccontr)

```



```

assume neq:  $xn \neq yn$ 
from fri-layer-chunk-authenticated-matching-values[OF auth chunk]
obtain xp-path' xn-path' where
  xn-auth:
    authenticated-opening-in final-state
    ( $\langle$ opening-root = rt,
      opening-length = len,
      opening-index = fri-sibling-index len idx,
      opening-value = xn,
      opening-path = xn-path' $\rangle$ )
  by blast
from fri-layer-chunk-authenticated-matching-values[OF auth' chunk']
obtain yp-path' yn-path' where
  yn-auth:
    authenticated-opening-in final-state
    ( $\langle$ opening-root = rt,
      opening-length = len,
      opening-index = fri-sibling-index len idx,
      opening-value = yn,
      opening-path = yn-path' $\rangle$ )
  by blast
have partial-merkle-inconsistency-bad s
  (Some (result, final-state))
unfolding partial-merkle-inconsistency-bad-def accepted-def
by (intro conjI exI[of - result] exI[of - final-state]
  exI[of - ( $\langle$ opening-root = rt,
    opening-length = len,
    opening-index = fri-sibling-index len idx,
    opening-value = xn,
    opening-path = xn-path' $\rangle$ )]
  exI[of - ( $\langle$ opening-root = rt,
    opening-length = len,
    opening-index = fri-sibling-index len idx,
    opening-value = yn,
    opening-path = yn-path' $\rangle$ )]])
  (use xn-auth yn-auth neq in simp-all)
then show False using no-bad by contradiction
qed

lemma fri-layer-authenticated-base-sibling-values-eq-if-no-partial-merkle:
assumes no-bad:
   $\neg$  partial-merkle-inconsistency-bad s
  (Some (result, final-state))
and auth:
  fri-layer-chunk-authenticated rt len idx chunk final-state
and auth':
  fri-layer-chunk-authenticated rt len idx' chunk' final-state
and chunk:
  fri-layer-opening-chunk len xp xp-path xn xn-path chunk

```

```

and chunk':
  fri-layer-opening-chunk len yp yp-path yn yn-path chunk'
and same-index: idx = fri-sibling-index len idx'
shows xp = yn
proof –
from fri-layer-chunk-authenticated-matching-values[OF auth chunk]
obtain xp-path' xn-path' where
  xp-auth:
    authenticated-opening-in final-state
    ( $\parallel$  opening-root = rt,
      opening-length = len,
      opening-index = idx,
      opening-value = xp,
      opening-path = xp-path')
  by blast
from fri-layer-chunk-authenticated-matching-values[OF auth' chunk']
obtain yp-path' yn-path' where
  yn-auth:
    authenticated-opening-in final-state
    ( $\parallel$  opening-root = rt,
      opening-length = len,
      opening-index = fri-sibling-index len idx',
      opening-value = yn,
      opening-path = yn-path')
  by blast
show ?thesis
using authenticated-openings-values-eq-if-no-partial-merkle
  [OF no-bad xp-auth yn-auth] same-index
by simp
qed

lemma fri-layer-authenticated-sibling-base-values-eq-if-no-partial-merkle:
assumes no-bad:
   $\neg$  partial-merkle-inconsistency-bad s
  (Some (result, final-state))
and auth:
  fri-layer-chunk-authenticated rt len idx chunk final-state
and auth':
  fri-layer-chunk-authenticated rt len idx' chunk' final-state
and chunk:
  fri-layer-opening-chunk len xp xp-path xn xn-path chunk
and chunk':
  fri-layer-opening-chunk len yp yp-path yn yn-path chunk'
and same-index: fri-sibling-index len idx = idx'
shows xn = yp
proof –
from fri-layer-chunk-authenticated-matching-values[OF auth chunk]
obtain xp-path' xn-path' where
  xn-auth:

```

```

    authenticated-opening-in final-state
    (opening-root = rt,
     opening-length = len,
     opening-index = fri-sibling-index len idx,
     opening-value = xn,
     opening-path = xn-path')
  by blast
from fri-layer-chunk-authenticated-matching-values[OF auth' chunk']
obtain yp-path' yn-path' where
  yp-auth:
    authenticated-opening-in final-state
    (opening-root = rt,
     opening-length = len,
     opening-index = idx',
     opening-value = yp,
     opening-path = yp-path')
  by blast
show ?thesis
  using authenticated-openings-values-eq-if-no-partial-merkle
    [OF no-bad xn-auth yp-auth] same-index
  by simp
qed

lemma fri-layer-authenticated-sibling-sibling-values-eq-if-no-partial-merkle:
  assumes no-bad:
    ¬ partial-merkle-inconsistency-bad s
    (Some (result, final-state))
  and auth:
    fri-layer-chunk-authenticated rt len idx chunk final-state
  and auth':
    fri-layer-chunk-authenticated rt len idx' chunk' final-state
  and chunk:
    fri-layer-opening-chunk len xp xp-path xn xn-path chunk
  and chunk':
    fri-layer-opening-chunk len yp yp-path yn yn-path chunk'
  and same-index:
    fri-sibling-index len idx = fri-sibling-index len idx'
  shows xn = yn
proof -
  from fri-layer-chunk-authenticated-matching-values[OF auth chunk]
  obtain xp-path' xn-path' where
    xn-auth:
      authenticated-opening-in final-state
      (opening-root = rt,
       opening-length = len,
       opening-index = fri-sibling-index len idx,
       opening-value = xn,
       opening-path = xn-path')
  by blast

```

```

from fri-layer-chunk-authenticated-matching-values[OF auth' chunk']
obtain yp-path' yn-path' where
  yn-auth:
    authenticated-opening-in final-state
    (|opening-root = rt,
      opening-length = len,
      opening-index = fri-sibling-index len idx',
      opening-value = yn,
      opening-path = yn-path'|)
  by blast
show ?thesis
  using authenticated-openings-values-eq-if-no-partial-merkle
    [OF no-bad xn-auth yn-auth] same-index
  by simp
qed

```

```

lemma fri-layer-chunk-authenticated-mono:
  assumes auth: fri-layer-chunk-authenticated rt len idx chunk s
    and ext:  $s \leq t$ 
  shows fri-layer-chunk-authenticated rt len idx chunk t
  using auth
  unfolding fri-layer-chunk-authenticated-def
  by (meson authenticated-opening-in-mono ext)

```

```

lemma fri-mfold-chunks-authenticated-mono:
  assumes auth: fri-mfold-chunks-authenticated bfs len idx chunks s
    and ext:  $s \leq t$ 
  shows fri-mfold-chunks-authenticated bfs len idx chunks t
  using auth
  unfolding fri-mfold-chunks-authenticated-def
  by (meson fri-layer-chunk-authenticated-mono ext)

```

```

lemma fri-layer-opening-step-authenticated-chunk:
  fixes  $s\ t :: ('f, 'a)\text{protocol-channel-scheme}$ 
  assumes len-pos:  $0 < \text{len}$ 
    and idx-bound:  $\text{idx} < \text{len}$ 
    and outcome:
      Some (out, t) ∈
        set-dist
          (execute (fri-layer-opening-step (b, rt) (idx, x, len, pw)) s)
  obtains xp xp-path xn xn-path where
    fri-layer-opening-chunk len xp xp-path xn xn-path
      ([xp] @ xp-path @ [xn] @ xn-path)
    authenticated-opening-in t
      (|opening-root = rt,
        opening-length = len,
        opening-index = idx,
        opening-value = xp,
        opening-path = xp-path|)

```

```

    authenticated-opening-in t
    (opening-root = rt,
     opening-length = len,
     opening-index = fri-sibling-index len idx,
     opening-value = xn,
     opening-path = xn-path)
proof –
from outcome[unfolded fri-layer-opening-step-unfold-finish]
obtain xp s1 xp-path s2 xn s3 xn-path s4 where
  read-xp: Some (xp, s1) ∈ set-dist (execute read s)
and read-xp-path:
  Some (xp-path, s2) ∈
    set-dist (execute (ntimes read (floor-log len)) s1)
and read-xn: Some (xn, s3) ∈ set-dist (execute read s2)
and read-xn-path:
  Some (xn-path, s4) ∈
    set-dist (execute (ntimes read (floor-log len)) s3)
and finish:
  Some (out, t) ∈
    set-dist
    (execute
     (fri-layer-opening-finish b rt idx x len pw
      xp xp-path xn xn-path) s4)
by (auto elim!: set-dist-bindE)
have xp-path-length: length xp-path = floor-log len
  using ntimes-read-any-outcome[OF read-xp-path] by simp
have xn-path-length: length xn-path = floor-log len
  using ntimes-read-any-outcome[OF read-xn-path] by simp
have sibling-bound: (idx + len div 2) mod len < len
  using len-pos by simp
from fri-layer-opening-finish-authenticated-openings
  [OF idx-bound sibling-bound xp-path-length xn-path-length finish]
have xp-auth:
  authenticated-opening-in t
  (opening-root = rt,
   opening-length = len,
   opening-index = idx,
   opening-value = xp,
   opening-path = xp-path)
and xn-auth:
  authenticated-opening-in t
  (opening-root = rt,
   opening-length = len,
   opening-index = fri-sibling-index len idx,
   opening-value = xn,
   opening-path = xn-path)
unfolding fri-sibling-index-def
by blast+
have chunk:

```

```

    fri-layer-opening-chunk len xp xp-path xn xn-path
      ([xp] @ xp-path @ [xn] @ xn-path)
    using xp-path-length xn-path-length
    unfolding fri-layer-opening-chunk-def by simp
  show ?thesis
    by (rule that[OF chunk xp-auth xn-auth])
qed

lemma fri-layer-opening-step-authenticated-chunk-pred:
  fixes s t :: ('f, 'a) protocol-channel-scheme
  assumes len-pos: 0 < len
  and idx-bound: idx < len
  and outcome:
    Some (out, t) ∈
      set-dist
        (execute (fri-layer-opening-step (b, rt) (idx, x, len, pw)) s)
  obtains chunk where fri-layer-chunk-authenticated rt len idx chunk t
proof -
  from fri-layer-opening-step-authenticated-chunk
    [OF len-pos idx-bound outcome]
  obtain xp xp-path xn xn-path where
    chunk:
      fri-layer-opening-chunk len xp xp-path xn xn-path
        ([xp] @ xp-path @ [xn] @ xn-path)
  and xp-auth:
    authenticated-opening-in t
      (|opening-root = rt,
        opening-length = len,
        opening-index = idx,
        opening-value = xp,
        opening-path = xp-path)
  and xn-auth:
    authenticated-opening-in t
      (|opening-root = rt,
        opening-length = len,
        opening-index = fri-sibling-index len idx,
        opening-value = xn,
        opening-path = xn-path)
  by blast
  have fri-layer-chunk-authenticated rt len idx
    ([xp] @ xp-path @ [xn] @ xn-path) t
    by (rule fri-layer-chunk-authenticatedI[OF chunk xp-auth xn-auth])
  then show ?thesis
    by (rule that)
qed

lemma fri-layer-opening-step-outcome-authenticated:
  fixes s t :: ('f, 'a) protocol-channel-scheme
  assumes len-pos: 0 < len

```

```

and idx-bound: idx < len
and outcome:
  Some (out, t) ∈
    set-dist
      (execute (fri-layer-opening-step (b, rt) (idx, x, len, pw)) s)
obtains xp xp-path xn xn-path x' chunk where
  out = (idx mod (len div 2), x', len div 2, pw + pw)
  xp = x
  x' = fri-fold-value b xp xn
    (fri-fold-denominator ((h ^ idx) * shift) pw)
  chunk = [xp] @ xp-path @ [xn] @ xn-path
  fri-layer-opening-chunk len xp xp-path xn xn-path chunk
  PTranscript s = chunk @ PTranscript t
  PState t = foldl concat (PState s) chunk
  s ≤ t
  PQueryCounter t = PQueryCounter s
  authenticated-opening-in t
    (opening-root = rt,
     opening-length = len,
     opening-index = idx,
     opening-value = xp,
     opening-path = xp-path)
  authenticated-opening-in t
    (opening-root = rt,
     opening-length = len,
     opening-index = fri-sibling-index len idx,
     opening-value = xn,
     opening-path = xn-path)
proof –
from outcome obtain xp s1 xp-path s2 xn s3 xn-path s4 s5 ap1 s6 s7
  ap2 s8 s9 where
    read-xp: Some (xp, s1) ∈ set-dist (execute read s)
    and read-xp-path:
      Some (xp-path, s2) ∈
        set-dist (execute (ntimes read (floor-log len)) s1)
    and read-xn: Some (xn, s3) ∈ set-dist (execute read s2)
    and read-xn-path:
      Some (xn-path, s4) ∈
        set-dist (execute (ntimes read (floor-log len)) s3)
    and assert-xp: Some ((), s5) ∈ set-dist (execute (assert (xp = x)) s4)
    and check1:
      Some (ap1, s6) ∈
        set-dist (execute (check-authentication-path len idx xp xp-path) s5)
    and assert-ap1:
      Some ((), s7) ∈ set-dist (execute (assert (ap1 = rt)) s6)
    and check2:
      Some (ap2, s8) ∈
        set-dist (execute
          (check-authentication-path len ((idx + len div 2) mod len) xn

```

```

       $xn\text{-}path) \ s7)$ 
and assert-ap2:
  Some  $(((), \ s9) \in \text{set-dist} \ (\text{execute} \ (\text{assert} \ (ap2 = rt)) \ s8)$ 
and ret:
  Some  $(out, \ t) \in$ 
    set-dist  $(\text{execute}$ 
       $(\text{return}$ 
         $(idx \bmod (len \div 2),$ 
           $(xp + xn) \div 2 +$ 
             $b * ((xp - xn) \div (2 * ((h \wedge idx) * shift) \wedge pw)),$ 
             $len \div 2, \ pw + pw)) \ s9)$ 
      unfolding fri-layer-opening-step-def
      by  $(\text{auto simp: Let-def elim!: set-dist-bindE})$ 
from read-outcome $[OF \ \text{read-xp}]$  obtain rest1 where
  tr-s:  $PTranscript \ s = xp \ \# \ rest1$ 
  and st-s1:  $PState \ s1 = \text{concat} \ (PState \ s) \ xp$ 
  and tr-s1:  $PTranscript \ s1 = rest1$ 
  and ext-s-s1:  $s \leq s1$ 
  and query-count-s1:  $PQueryCounter \ s1 = PQueryCounter \ s$ 
  by blast
have xp-path-res:
   $length \ xp\text{-}path = \text{floor-log} \ len \wedge$ 
   $PTranscript \ s1 = xp\text{-}path \ @ \ PTranscript \ s2 \wedge$ 
   $PState \ s2 = \text{foldl} \ \text{concat} \ (PState \ s1) \ xp\text{-}path \wedge$ 
   $s1 \leq s2 \wedge$ 
   $PQueryCounter \ s2 = PQueryCounter \ s1$ 
  using ntimes-read-any-outcome $[OF \ \text{read-xp-path}]$  by simp
from read-outcome $[OF \ \text{read-xn}]$  obtain rest3 where
  tr-s2:  $PTranscript \ s2 = xn \ \# \ rest3$ 
  and st-s3:  $PState \ s3 = \text{concat} \ (PState \ s2) \ xn$ 
  and tr-s3:  $PTranscript \ s3 = rest3$ 
  and ext-s2-s3:  $s2 \leq s3$ 
  and query-count-s3:  $PQueryCounter \ s3 = PQueryCounter \ s2$ 
  by blast
have xn-path-res:
   $length \ xn\text{-}path = \text{floor-log} \ len \wedge$ 
   $PTranscript \ s3 = xn\text{-}path \ @ \ PTranscript \ s4 \wedge$ 
   $PState \ s4 = \text{foldl} \ \text{concat} \ (PState \ s3) \ xn\text{-}path \wedge$ 
   $s3 \leq s4 \wedge$ 
   $PQueryCounter \ s4 = PQueryCounter \ s3$ 
  using ntimes-read-any-outcome $[OF \ \text{read-xn-path}]$  by simp
have xp-eq:  $xp = x$ 
  using assert-xp unfolding assert-def
  by  $(\text{cases} \ xp = x) \ (\text{auto simp: throw-no-outcome})$ 
have s5-eq:  $s5 = s4$ 
  using assert-xp xp-eq unfolding assert-def by simp
have s5-s6:  $s5 \leq s6$ 
  by  $(\text{rule} \ \text{check-authentication-path-hash-extends}[OF \ \text{check1}])$ 
have st-s6:  $PState \ s6 = PState \ s5$ 

```



```

    using check-authentication-path-preserves-channel(1)[OF check1] .
have tr-s6: PTranscript s6 = PTranscript s5
    using check-authentication-path-preserves-channel(2)[OF check1] .
have query-count-s6: PQueryCounter s6 = PQueryCounter s5
    using check-authentication-path-preserves-channel(6)[OF check1] .
have ap1-eq: ap1 = rt
    using assert-ap1 unfolding assert-def
    by (cases ap1 = rt) (auto simp: throw-no-outcome)
have s7-eq: s7 = s6
    using assert-ap1 ap1-eq unfolding assert-def by simp
have s7-s8: s7 ≤ s8
    by (rule check-authentication-path-hash-extends[OF check2])
have st-s8: PState s8 = PState s7
    using check-authentication-path-preserves-channel(1)[OF check2] .
have tr-s8: PTranscript s8 = PTranscript s7
    using check-authentication-path-preserves-channel(2)[OF check2] .
have query-count-s8: PQueryCounter s8 = PQueryCounter s7
    using check-authentication-path-preserves-channel(6)[OF check2] .
have ap2-eq: ap2 = rt
    using assert-ap2 unfolding assert-def
    by (cases ap2 = rt) (auto simp: throw-no-outcome)
have s9-eq: s9 = s8
    using assert-ap2 ap2-eq unfolding assert-def by simp
let ?x' =
  (xp + xn) div 2 +
  b * ((xp - xn) div (2 * ((h ^ idx) * shift) ^ pw))
let ?chunk = [xp] @ xp-path @ [xn] @ xn-path
have out-t:
  out = (idx mod (len div 2), ?x', len div 2, pw + pw) ∧ t = s9
    using ret by simp
have x'-fold:
  ?x' = fri-fold-value b xp xn
    (fri-fold-denominator ((h ^ idx) * shift) pw)
    unfolding fri-fold-value-def fri-fold-denominator-def by simp
have chunk-shape:
  fri-layer-opening-chunk len xp xp-path xn xn-path ?chunk
    using xp-path-res xn-path-res unfolding fri-layer-opening-chunk-def by simp
have tr-t: PTranscript s = ?chunk @ PTranscript t
    using tr-s tr-s1 xp-path-res tr-s2 tr-s3 xn-path-res s5-eq tr-s6
    s7-eq tr-s8 s9-eq out-t
    by simp
have st-t: PState t = foldl concat (PState s) ?chunk
    using st-s1 xp-path-res st-s3 xn-path-res s5-eq st-s6 s7-eq st-s8
    s9-eq out-t
    by simp
have s-t: s ≤ t
    using ext-s-s1 xp-path-res ext-s2-s3 xn-path-res s5-s6 s7-s8 out-t
    unfolding s5-eq s7-eq s9-eq
    by (meson hash-ext-trans)

```

```

have query-count-t: PQueryCounter t = PQueryCounter s
  using out-t query-count-s1
  xp-path-res query-count-s3 xn-path-res query-count-s6 query-count-s8
  s5-eq s7-eq s9-eq
  by simp
have sibling-bound: (idx + len div 2) mod len < len
  using len-pos by simp
have xp-path-length: length xp-path = floor-log len
  using xp-path-res by simp
have xn-path-length: length xn-path = floor-log len
  using xn-path-res by simp
have s6-t: s6 ≤ t
  using s7-s8 out-t unfolding s7-eq s9-eq by simp
have xp-bound:
  merkle-path-bound rt len idx xp xp-path t
proof -
  have merkle-path-bound ap1 len idx xp xp-path s6
    by (rule check-authentication-path-outcome-bound[OF check1])
  then have merkle-path-bound rt len idx xp xp-path s6
    unfolding ap1-eq .
  then show ?thesis
    by (rule merkle-path-bound-mono[OF - s6-t])
qed
have xn-bound:
  merkle-path-bound rt len (fri-sibling-index len idx) xn xn-path t
proof -
  have merkle-path-bound ap2 len ((idx + len div 2) mod len)
    xn xn-path s8
    by (rule check-authentication-path-outcome-bound[OF check2])
  then show ?thesis
    using ap2-eq out-t unfolding fri-sibling-index-def s9-eq by simp
qed
have xp-auth:
  authenticated-opening-in t
  (|opening-root = rt, opening-length = len,
   opening-index = idx, opening-value = xp,
   opening-path = xp-path|)
  unfolding authenticated-opening-in-def
  using idx-bound xp-path-length xp-bound by simp
have xn-auth:
  authenticated-opening-in t
  (|opening-root = rt, opening-length = len,
   opening-index = fri-sibling-index len idx,
   opening-value = xn, opening-path = xn-path|)
  unfolding authenticated-opening-in-def
  using sibling-bound xn-path-length xn-bound
  unfolding fri-sibling-index-def by simp
show ?thesis
proof (rule that)

```

```

show out = (idx mod (len div 2), ?x', len div 2, pw + pw)
  using out-t by simp
show xp = x
  by (rule xp-eq)
show ?x' =
  fri-fold-value b xp xn
  (fri-fold-denominator ((h ^ idx) * shift) pw)
  by (rule x'-fold)
show ?chunk = [xp] @ xp-path @ [xn] @ xn-path
  by simp
show fri-layer-opening-chunk len xp xp-path xn xn-path ?chunk
  by (rule chunk-shape)
show PTranscript s = ?chunk @ PTranscript t
  by (rule tr-t)
show PState t = foldl concat (PState s) ?chunk
  by (rule st-t)
show s ≤ t
  by (rule s-t)
show PQueryCounter t = PQueryCounter s
  by (rule query-count-t)
show authenticated-opening-in t
  (opening-root = rt, opening-length = len,
   opening-index = idx, opening-value = xp,
   opening-path = xp-path)
  by (rule xp-auth)
show authenticated-opening-in t
  (opening-root = rt, opening-length = len,
   opening-index = fri-sibling-index len idx,
   opening-value = xn, opening-path = xn-path)
  by (rule xn-auth)
qed
qed

lemma mfold-fri-layer-opening-steps-hash-extends:
  fixes s t :: ('f, 'a) protocol-channel-scheme
  assumes outcome:
    Some (out, t) ∈
      set-dist
        (execute (mfold (idx, x, len, pw)
          (map fri-layer-opening-step bfs)) s)
  shows s ≤ t
proof –
  from mfold-fri-layer-openings-outcome[OF outcome]
  obtain layer-chunks chunk where
    s ≤ t
    by blast
  then show ?thesis .
qed

```

lemma *mfold-fri-layer-opening-steps-authenticated-opening-mono*:
fixes $s\ t :: ('f, 'a)\ \text{protocol-channel-scheme}$
assumes *auth*: *authenticated-opening-in s opening*
and *outcome*:
 $\text{Some } (out, t) \in$
 set-dist
 $(\text{execute } (\text{mfold } (idx, x, len, pw)$
 $(\text{map } \text{fri-layer-opening-step } bfs))\ s)$
shows *authenticated-opening-in t opening*
by (*rule authenticated-opening-in-mono*
 $[OF\ \text{auth}\ \text{mfold-fri-layer-opening-steps-hash-extends}[OF\ \text{outcome}]])$

lemma *mfold-fri-layer-opening-head-chunk-authenticated*:
fixes $s\ t :: ('f, 'a)\ \text{protocol-channel-scheme}$
assumes *len-pos*: $0 < len$
and *idx-bound*: $idx < len$
and *outcome*:
 $\text{Some } (out, t) \in$
 set-dist
 $(\text{execute}$
 $(\text{mfold } (idx, x, len, pw)$
 $(\text{map } \text{fri-layer-opening-step } ((b, rt) \# bfs)))\ s)$
obtains *chunk* **where** *fri-layer-chunk-authenticated rt len idx chunk t*
proof –
from *outcome* **obtain** *out1 s1* **where**
head:
 $\text{Some } (out1, s1) \in$
 set-dist
 $(\text{execute } (\text{fri-layer-opening-step } (b, rt) (idx, x, len, pw))\ s)$
and *tail*:
 $\text{Some } (out, t) \in$
 $\text{set-dist } (\text{execute } (\text{mfold } out1\ (\text{map } \text{fri-layer-opening-step } bfs))\ s1)$
by (*auto elim!*: *set-dist-bindE*)
from *fri-layer-opening-step-authenticated-chunk-pred*
 $[OF\ \text{len-pos}\ \text{idx-bound}\ \text{head}]$
obtain *chunk* **where** *auth-s1*:
fri-layer-chunk-authenticated rt len idx chunk s1
by *blast*
obtain *idx1 x1 len1 pw1* **where** *out1-eq*:
 $out1 = (idx1, x1, len1, pw1)$
by (*cases out1*)
have *s1-t*: $s1 \leq t$
by (*rule mfold-fri-layer-opening-steps-hash-extends*
 $[OF\ \text{tail}[\text{unfolded } out1\text{-eq}]])$
have *fri-layer-chunk-authenticated rt len idx chunk t*
by (*rule fri-layer-chunk-authenticated-mono* $[OF\ \text{auth-s1}\ s1\text{-}t])$
then show *?thesis*
by (*rule that*)
qed

```

lemma mfold-fri-layer-opening-selected-chunk-authenticated-at-exists:
  fixes  $s\ t :: ('f, 'a)\ \text{protocol-channel-scheme}$ 
  assumes outcome:
     $\text{Some } (out, t) \in$ 
       $\text{set-dist}$ 
       $(\text{execute}$ 
         $(\text{mfold } (idx, x, len, pw)$ 
           $(\text{map } \text{fri-layer-opening-step } bfs))\ s)$ 
      and j-bound:  $j < \text{length } bfs$ 
      and layer-len-pos:
         $0 < \text{fri-layer-lengths } (\text{length } bfs)\ len\ !\ j$ 
      and layer-idx-bound:
         $\text{fri-layer-indices } (\text{length } bfs)\ idx\ len\ !\ j <$ 
         $\text{fri-layer-lengths } (\text{length } bfs)\ len\ !\ j$ 
      shows  $\exists\ chunk.$ 
         $\text{fri-layer-chunk-authenticated } (\text{snd } (bfs\ !\ j))$ 
         $(\text{fri-layer-lengths } (\text{length } bfs)\ len\ !\ j)$ 
         $(\text{fri-layer-indices } (\text{length } bfs)\ idx\ len\ !\ j)$ 
         $chunk\ t$ 
      using outcome j-bound layer-len-pos layer-idx-bound
proof  $(\text{induction } bfs\ \text{arbitrary: } idx\ x\ len\ pw\ s\ out\ t\ j)$ 
  case Nil
    then show ?case by simp
  next
    case  $(Cons\ bf\ bfs)$ 
    obtain  $b\ rt$  where  $bf\text{-eq: } bf = (b, rt)$ 
    by  $(\text{cases } bf)$ 
    from  $Cons.\text{prems}(1)[\text{unfolded } bf\text{-eq}]$ 
    obtain  $idx1\ x1\ len1\ pw1\ s1$  where
      head:
         $\text{Some } ((idx1, x1, len1, pw1), s1) \in$ 
           $\text{set-dist}$ 
           $(\text{execute } (\text{fri-layer-opening-step } (b, rt)\ (idx, x, len, pw))\ s)$ 
      and tail:
         $\text{Some } (out, t) \in$ 
           $\text{set-dist}$ 
           $(\text{execute}$ 
             $(\text{mfold } (idx1, x1, len1, pw1)$ 
               $(\text{map } \text{fri-layer-opening-step } bfs))\ s1)$ 
          by  $(\text{auto elim!: set-dist-bindE})$ 
    show ?case
  proof  $(\text{cases } j)$ 
  case 0
    have len-pos:  $0 < len$ 
    using  $Cons.\text{prems}(3)\ 0$  by simp
    have idx-bound:  $idx < len$ 
    using  $Cons.\text{prems}(4)\ 0$  by simp
    from fri-layer-opening-step-authenticated-chunk-pred

```

```

    [OF len-pos idx-bound head]
  obtain chunk where auth-s1:
    fri-layer-chunk-authenticated rt len idx chunk s1
  by blast
  have s1-t: s1 ≤ t
  by (rule mfold-fri-layer-opening-steps-hash-extends
    [OF tail])
  have auth-t:
    fri-layer-chunk-authenticated rt len idx chunk t
  by (rule fri-layer-chunk-authenticated-mono[OF auth-s1 s1-t])
  show ?thesis
  using auth-t 0 bf-eq by auto
next
case (Suc k)
from fri-layer-opening-step-outcome[OF head]
obtain xp xp-path xn xn-path x' where out1-eq:
  (idx1, x1, len1, pw1) =
  (idx mod (len div 2), x', len div 2, pw + pw)
  by blast
have k-bound: k < length bfs
  using Cons.premis(2) Suc by simp
have tail-len-pos:
  0 < fri-layer-lengths (length bfs) (len div 2) ! k
  using Cons.premis(3) Suc by simp
have tail-idx-bound:
  fri-layer-indices (length bfs) (idx mod (len div 2))
  (len div 2) ! k <
  fri-layer-lengths (length bfs) (len div 2) ! k
  using Cons.premis(4) Suc by simp
show ?thesis
  using Cons.IH[OF tail[unfolded out1-eq] k-bound
    tail-len-pos tail-idx-bound] Suc
  by auto
qed
qed

lemma mfold-fri-layer-opening-selected-chunk-authenticated-at:
  fixes s t :: ('f, 'a) protocol-channel-scheme
  assumes outcome:
    Some (out, t) ∈
      set-dist
      (execute
        (mfold (idx, x, len, pw)
          (map fri-layer-opening-step bfs)) s)
  and j-bound: j < length bfs
  and layer-len-pos:
    0 < fri-layer-lengths (length bfs) len ! j
  and layer-idx-bound:
    fri-layer-indices (length bfs) idx len ! j <

```

```

      fri-layer-lengths (length bfs) len ! j
obtains chunk where
  fri-layer-chunk-authenticated (snd (bfs ! j))
    (fri-layer-lengths (length bfs) len ! j)
    (fri-layer-indices (length bfs) idx len ! j)
    chunk t
using mfold-fri-layer-opening-selected-chunk-authenticated-at-exists
  [OF outcome j-bound layer-len-pos layer-idx-bound]
by blast

```

lemma *receive-query-commits-selected-chunk-authenticated-at:*

```

fixes s t :: ('f, 'a) protocol-channel-scheme
assumes outcome:
  Some (out, t) ∈
    set-dist
    (execute
      (mfold (idx, x, len, pw) (receive-query-commits bfs)) s)
and j-bound: j < length bfs
and layer-len-pos:
  0 < fri-layer-lengths (length bfs) len ! j
and layer-idx-bound:
  fri-layer-indices (length bfs) idx len ! j <
    fri-layer-lengths (length bfs) len ! j
obtains chunk where
  fri-layer-chunk-authenticated (snd (bfs ! j))
    (fri-layer-lengths (length bfs) len ! j)
    (fri-layer-indices (length bfs) idx len ! j)
    chunk t
proof –
  have map-eq: receive-query-commits bfs = map fri-layer-opening-step bfs
  unfolding receive-query-commits-def fri-layer-opening-step-def
  by (simp add: fun-eq-iff split: prod.splits)
  show ?thesis
  by (rule mfold-fri-layer-opening-selected-chunk-authenticated-at
    [OF outcome[unfolded map-eq] j-bound layer-len-pos
      layer-idx-bound that])

```

qed

lemma *receive-query-commits-mfold-hash-extends:*

```

fixes s t :: ('f, 'a) protocol-channel-scheme
assumes outcome:
  Some (out, t) ∈
    set-dist
    (execute (mfold (idx, x, len, pw)
      (receive-query-commits bfs)) s)
shows s ≤ t

```

proof –

```

have map-eq: receive-query-commits bfs = map fri-layer-opening-step bfs
unfolding receive-query-commits-def fri-layer-opening-step-def

```

```

    by (simp add: fun-eq-iff split: prod.splits)
  show ?thesis
    by (rule mfold-fri-layer-opening-steps-hash-extends
        [OF outcome[unfolded map-eq]])
qed

lemma receive-query-commits-mfold-authenticated-opening-mono:
  fixes s t :: ('f, 'a) protocol-channel-scheme
  assumes auth: authenticated-opening-in s opening
  and outcome:
    Some (out, t) ∈
      set-dist
        (execute (mfold (idx, x, len, pw)
            (receive-query-commits bfs)) s)
  shows authenticated-opening-in t opening
  by (rule authenticated-opening-in-mono
      [OF auth receive-query-commits-mfold-hash-extends[OF outcome]])

lemma receive-query-commits-mfold-head-chunk-authenticated:
  fixes s t :: ('f, 'a) protocol-channel-scheme
  assumes len-pos: 0 < len
  and idx-bound: idx < len
  and outcome:
    Some (out, t) ∈
      set-dist
        (execute
            (mfold (idx, x, len, pw)
                (receive-query-commits ((b, rt) # bfs))) s)
  obtains chunk where fri-layer-chunk-authenticated rt len idx chunk t
proof –
  have map-eq:
    receive-query-commits ((b, rt) # bfs) =
      map fri-layer-opening-step ((b, rt) # bfs)
  unfolding receive-query-commits-def fri-layer-opening-step-def
  by (simp add: fun-eq-iff split: prod.splits)
from mfold-fri-layer-opening-head-chunk-authenticated
  [OF len-pos idx-bound outcome[unfolded map-eq]]
obtain chunk where fri-layer-chunk-authenticated rt len idx chunk t
  by blast
then show ?thesis
  by (rule that)
qed

lemma verifier-query-round-after-index-trace-fri-head-chunk-authenticated:
  fixes s t :: ('f, 'a) protocol-channel-scheme
  assumes f-fl-eq: f-fl = (b, rt) # f-fl-tail
  and outcome:
    Some ((), t) ∈
      set-dist

```



```

      (execute
        (verifier-query-round-after-index-program
          fr f-fl f-final as fl final raw) s)
obtains idx chunk where
  idx = index (to-nat raw)
  fri-layer-chunk-authenticated rt (clength * scale) idx chunk t
proof –
let ?idx = index (to-nat raw)
from outcome obtain fv s1 f-out s2 s3 c-out s4 where
  query-decommit:
    Some (fv, s1) ∈
      set-dist (execute (mmap (check-decommit-on-query fr ?idx)) s)
and trace-fri:
    Some (f-out, s2) ∈
      set-dist (execute
        (mfold (?idx, hd fv, clength * scale, 1)
          (receive-query-commits f-fl)) s1)
and assert-trace:
    Some ((), s3) ∈
      set-dist
        (execute
          (assert (case f-out of (-, f-x, -, -) ⇒ f-x = f-final)) s2)
and comp-fri:
    Some (c-out, s4) ∈
      set-dist (execute
        (mfold (?idx, cp-eval as fv (h ^ ?idx * shift),
          clength * scale, 1)
          (receive-query-commits fl)) s3)
and assert-comp:
    Some ((), t) ∈
      set-dist
        (execute
          (assert (case c-out of (-, x, -, -) ⇒ x = final)) s4)
unfolding verifier-query-round-after-index-program-def
by (auto simp: Let-def split: prod.splits elim!: set-dist-bindE)
have len-pos: 0 < clength * scale
using eval-domain-nontrivial by linarith
have idx-bound: ?idx < clength * scale
by (rule index-less-domain)
from receive-query-commits-mfold-head-chunk-authenticated
  [OF len-pos idx-bound trace-fri[unfolded f-fl-eq]]
obtain chunk where auth-s2:
  fri-layer-chunk-authenticated rt (clength * scale) ?idx chunk s2
by blast
have s3-eq: s3 = s2
using assert-trace unfolding assert-def
by (cases case f-out of (-, f-x, -, -) ⇒ f-x = f-final)
  (auto simp: throw-no-outcome)
have s3-s4: s3 ≤ s4

```

```

    by (rule receive-query-commits-mfold-hash-extends[OF comp-fri])
  have t-eq:  $t = s_4$ 
    using assert-comp unfolding assert-def
    by (cases case c-out of  $(-, x, -, -) \Rightarrow x = \text{final}$ )
      (auto simp: throw-no-outcome)
  have s2-t:  $s_2 \leq t$ 
    using s3-s4 unfolding s3-eq t-eq by simp
  have auth-t:
    fri-layer-chunk-authenticated rt (clength * scale) ?idx chunk t
    by (rule fri-layer-chunk-authenticated-mono[OF auth-s2 s2-t])
  show ?thesis
    by (rule that[OF refl auth-t])
qed

```

lemma *verifier-query-round-after-index-trace-fri-selected-chunk-authenticated:*

```

fixes  $s\ t :: ('f, 'a)\ \text{protocol-channel-scheme}$ 
assumes outcome:
  Some  $((\ ), t) \in$ 
    set-dist
      (execute
        (verifier-query-round-after-index-program
          fr f-fl f-final as fl final raw) s)
and j-bound:  $j < \text{length } f\text{-fl}$ 
and layer-len-pos:
   $0 < \text{fri-layer-lengths } (\text{length } f\text{-fl})\ (\text{clength} * \text{scale})\ !\ j$ 
and layer-idx-bound:
  fri-layer-indices  $(\text{length } f\text{-fl})\ (\text{index } (\text{to-nat raw}))$ 
     $(\text{clength} * \text{scale})\ !\ j <$ 
    fri-layer-lengths  $(\text{length } f\text{-fl})\ (\text{clength} * \text{scale})\ !\ j$ 
obtains chunk where
  fri-layer-chunk-authenticated  $(\text{snd } (f\text{-fl}\ !\ j))$ 
    (fri-layer-lengths  $(\text{length } f\text{-fl})\ (\text{clength} * \text{scale})\ !\ j)$ 
    (fri-layer-indices  $(\text{length } f\text{-fl})\ (\text{index } (\text{to-nat raw}))$ 
       $(\text{clength} * \text{scale})\ !\ j)$ 
    chunk t
proof -
  let ?idx =  $\text{index } (\text{to-nat raw})$ 
from outcome obtain fv s1 f-out s2 s3 c-out s4 where
  trace-fri:
    Some  $(f\text{-out}, s_2) \in$ 
      set-dist (execute
        (mfold (?idx, hd fv, clength * scale, 1)
          (receive-query-commits f-fl)) s1)
and assert-trace:
    Some  $((\ ), s_3) \in$ 
      set-dist
        (execute
          (assert (case f-out of  $(-, f\text{-x}, -, -) \Rightarrow f\text{-x} = f\text{-final}$ )) s2)
and comp-fri:

```

```

    Some (c-out, s4) ∈
      set-dist (execute
        (mfold (?idx, cp-eval as fv (h ^ ?idx * shift),
          clength * scale, 1)
          (receive-query-commits fl)) s3)
  and assert-comp:
    Some (( ), t) ∈
      set-dist
        (execute
          (assert (case c-out of (-, x, -, -) ⇒ x = final)) s4)
  unfolding verifier-query-round-after-index-program-def
  by (auto simp: Let-def split: prod.splits elim!: set-dist-bindE)
from receive-query-commits-selected-chunk-authenticated-at
  [OF trace-fri j-bound layer-len-pos layer-idx-bound]
obtain chunk where auth-s2:
  fri-layer-chunk-authenticated (snd (f-fl ! j))
  (fri-layer-lengths (length f-fl) (clength * scale) ! j)
  (fri-layer-indices (length f-fl) ?idx (clength * scale) ! j)
  chunk s2
by blast
have s3-eq: s3 = s2
  using assert-trace unfolding assert-def
  by (cases case f-out of (-, f-x, -, -) ⇒ f-x = f-final)
  (auto simp: throw-no-outcome)
have s3-s4: s3 ≤ s4
  by (rule receive-query-commits-mfold-hash-extends[OF comp-fri])
have t-eq: t = s4
  using assert-comp unfolding assert-def
  by (cases case c-out of (-, x, -, -) ⇒ x = final)
  (auto simp: throw-no-outcome)
have s2-t: s2 ≤ t
  using s3-s4 unfolding s3-eq t-eq by simp
have auth-t:
  fri-layer-chunk-authenticated (snd (f-fl ! j))
  (fri-layer-lengths (length f-fl) (clength * scale) ! j)
  (fri-layer-indices (length f-fl) ?idx (clength * scale) ! j)
  chunk t
  by (rule fri-layer-chunk-authenticated-mono[OF auth-s2 s2-t])
then show ?thesis
  by (rule that)
qed

```

```

lemma verifier-query-round-after-index-composition-fri-head-chunk-authenticated:
  fixes s t :: ('f, 'a) protocol-channel-scheme
  assumes fl-eq: fl = (b, rt) # fl-tail
  and outcome:
    Some (( ), t) ∈
      set-dist
        (execute

```

```

      (verifier-query-round-after-index-program
       fr f-fl f-final as fl final raw) s)
obtains idx chunk where
  idx = index (to-nat raw)
  fri-layer-chunk-authenticated rt (clength * scale) idx chunk t
proof –
  let ?idx = index (to-nat raw)
  from outcome obtain fv s1 f-out s2 s3 c-out s4 where
    query-decommit:
      Some (fv, s1) ∈
        set-dist (execute (mmap (check-decommit-on-query fr ?idx)) s)
    and trace-fri:
      Some (f-out, s2) ∈
        set-dist (execute
          (mfold (?idx, hd fv, clength * scale, 1)
            (receive-query-commits f-fl)) s1)
    and assert-trace:
      Some ((), s3) ∈
        set-dist
          (execute
            (assert (case f-out of (-, f-x, -, -) ⇒ f-x = f-final)) s2)
    and comp-fri:
      Some (c-out, s4) ∈
        set-dist (execute
          (mfold (?idx, cp-eval as fv (h ^ ?idx * shift),
            clength * scale, 1)
            (receive-query-commits fl)) s3)
    and assert-comp:
      Some ((), t) ∈
        set-dist
          (execute
            (assert (case c-out of (-, x, -, -) ⇒ x = final)) s4)
    unfolding verifier-query-round-after-index-program-def
    by (auto simp: Let-def split: prod.splits elim!: set-dist-bindE)
  have len-pos: 0 < clength * scale
    using eval-domain-nontrivial by linarith
  have idx-bound: ?idx < clength * scale
    by (rule index-less-domain)
  from receive-query-commits-mfold-head-chunk-authenticated
    [OF len-pos idx-bound comp-fri[unfolded fl-eq]]
  obtain chunk where auth-s4:
    fri-layer-chunk-authenticated rt (clength * scale) ?idx chunk s4
    by blast
  have t-eq: t = s4
    using assert-comp unfolding assert-def
    by (cases case c-out of (-, x, -, -) ⇒ x = final)
    (auto simp: throw-no-outcome)
  have auth-t:
    fri-layer-chunk-authenticated rt (clength * scale) ?idx chunk t

```

```

    using auth-s4 unfolding t-eq .
  show ?thesis
    by (rule that[OF refl auth-t])
qed

lemma verifier-query-round-after-index-composition-fri-selected-chunk-authenticated:
  fixes s t :: ('f, 'a) protocol-channel-scheme
  assumes outcome:
    Some (( ), t) ∈
      set-dist
        (execute
          (verifier-query-round-after-index-program
            fr f-fl f-final as fl final raw) s)
  and j-bound: j < length fl
  and layer-len-pos:
    0 < fri-layer-lengths (length fl) (clength * scale) ! j
  and layer-idx-bound:
    fri-layer-indices (length fl) (index (to-nat raw))
      (clength * scale) ! j <
      fri-layer-lengths (length fl) (clength * scale) ! j
  obtains chunk where
    fri-layer-chunk-authenticated (snd (fl ! j))
      (fri-layer-lengths (length fl) (clength * scale) ! j)
      (fri-layer-indices (length fl) (index (to-nat raw))
        (clength * scale) ! j)
    chunk t
proof -
  let ?idx = index (to-nat raw)
  from outcome obtain fv s1 f-out s2 s3 c-out s4 where
    comp-fri:
      Some (c-out, s4) ∈
        set-dist (execute
          (mfold (?idx, cp-eval as fv (h ^ ?idx * shift),
            clength * scale, 1)
            (receive-query-commits fl)) s3)
  and assert-comp:
    Some (( ), t) ∈
      set-dist
        (execute
          (assert (case c-out of (-, x, -, -) ⇒ x = final)) s4)
  unfolding verifier-query-round-after-index-program-def
  by (auto simp: Let-def split: prod.splits elim!: set-dist-bindE)
  from receive-query-commits-selected-chunk-authenticated-at
    [OF comp-fri j-bound layer-len-pos layer-idx-bound]
  obtain chunk where auth-s4:
    fri-layer-chunk-authenticated (snd (fl ! j))
      (fri-layer-lengths (length fl) (clength * scale) ! j)
      (fri-layer-indices (length fl) ?idx (clength * scale) ! j)
    chunk s4

```

```

    by blast
  have t-eq:  $t = s_4$ 
    using assert-comp unfolding assert-def
    by (cases case c-out of  $(-, x, -, -) \Rightarrow x = final$ )
      (auto simp: throw-no-outcome)
  have auth-t:
    fri-layer-chunk-authenticated (snd (fl ! j))
      (fri-layer-lengths (length fl) (clength * scale) ! j)
      (fri-layer-indices (length fl) ?idx (clength * scale) ! j)
      chunk t
    using auth-s4 unfolding t-eq .
  then show ?thesis
    by (rule that)
qed

lemma verifier-query-round-program-trace-fri-head-chunk-authenticated:
  fixes  $s\ t :: ('f, 'a) \text{protocol-channel-scheme}$ 
  assumes f-fl-eq:  $f\text{-fl} = (b, rt) \# f\text{-fl-tail}$ 
  and outcome:
    Some  $(((), t) \in \text{set-dist} (\text{execute} (\text{verifier-query-round-program fr } f\text{-fl } f\text{-final as fl final) s))$ 
  obtains raw idx chunk where
    idx = index (to-nat raw)
    fri-layer-chunk-authenticated rt (clength * scale) idx chunk t
proof -
  from outcome[unfolded verifier-query-round-program-alt-def]
  obtain raw s0 where
    raw:
      Some  $(\text{raw}, s0) \in \text{set-dist} (\text{execute receive-query-index-challenge } s)$ 
  and tail:
    Some  $(((), t) \in \text{set-dist} (\text{execute} (\text{verifier-query-round-after-index-program fr } f\text{-fl } f\text{-final as fl final raw) s0))$ 
  by (auto elim!: set-dist-bindE)
  from verifier-query-round-after-index-trace-fri-head-chunk-authenticated
    [OF f-fl-eq tail]
  obtain idx chunk where
    idx-eq:  $idx = \text{index} (\text{to-nat raw})$ 
    and auth: fri-layer-chunk-authenticated rt (clength * scale) idx chunk t
  by blast
  show ?thesis
    by (rule that[OF idx-eq auth])
qed

```

lemma *verifier-query-round-program-trace-fri-selected-chunk-authenticated:*
fixes $s\ t :: ('f, 'a)\ \text{protocol-channel-scheme}$
assumes *outcome:*
 Some $((\),\ t) \in$
 set-dist
 (*execute*
 (*verifier-query-round-program* *fr f-fl f-final as fl final*) s)
and *j-bound:* $j < \text{length } f\text{-fl}$
and *raw-layer:*
 $\wedge$ *raw.*
 $0 < \text{fri-layer-lengths } (\text{length } f\text{-fl})\ (\text{clength} * \text{scale})\ !\ j \wedge$
 fri-layer-indices $(\text{length } f\text{-fl})\ (\text{index } (\text{to-nat } \text{raw}))$
 $(\text{clength} * \text{scale})\ !\ j <$
 fri-layer-lengths $(\text{length } f\text{-fl})\ (\text{clength} * \text{scale})\ !\ j$
obtains *raw chunk where*
 fri-layer-chunk-authenticated $(\text{snd } (f\text{-fl}\ !\ j))$
 (*fri-layer-lengths* $(\text{length } f\text{-fl})\ (\text{clength} * \text{scale})\ !\ j$)
 (*fri-layer-indices* $(\text{length } f\text{-fl})\ (\text{index } (\text{to-nat } \text{raw}))$)
 $(\text{clength} * \text{scale})\ !\ j$)
 chunk t
proof –
from *outcome*[*unfolded verifier-query-round-program-alt-def*]
obtain *raw s0 where*
 raw:
 Some $(\text{raw},\ s0) \in$
 set-dist (*execute* *receive-query-index-challenge* s)
and *tail:*
 Some $((\),\ t) \in$
 set-dist
 (*execute*
 (*verifier-query-round-after-index-program*
 fr f-fl f-final as fl final raw) $s0$)
by (*auto elim!:* *set-dist-bindE*)
have *layer-len-pos:*
 $0 < \text{fri-layer-lengths } (\text{length } f\text{-fl})\ (\text{clength} * \text{scale})\ !\ j$
using *raw-layer*[*of raw*] **by** *simp*
have *layer-idx-bound:*
 fri-layer-indices $(\text{length } f\text{-fl})\ (\text{index } (\text{to-nat } \text{raw}))$
 $(\text{clength} * \text{scale})\ !\ j <$
 fri-layer-lengths $(\text{length } f\text{-fl})\ (\text{clength} * \text{scale})\ !\ j$
using *raw-layer*[*of raw*] **by** *simp*
from *verifier-query-round-after-index-trace-fri-selected-chunk-authenticated*
 [*OF tail j-bound layer-len-pos layer-idx-bound*]
obtain *chunk where auth:*
 fri-layer-chunk-authenticated $(\text{snd } (f\text{-fl}\ !\ j))$
 (*fri-layer-lengths* $(\text{length } f\text{-fl})\ (\text{clength} * \text{scale})\ !\ j$)
 (*fri-layer-indices* $(\text{length } f\text{-fl})\ (\text{index } (\text{to-nat } \text{raw}))$)
 $(\text{clength} * \text{scale})\ !\ j$)
 chunk t

```

    by blast
  show ?thesis
    by (rule that[OF auth])
qed

lemma verifier-query-round-program-trace-fri-selected-chunk-authenticated-with-lookup:
  fixes s t :: ('f, 'a) protocol-channel-scheme
  assumes outcome:
    Some (( ), t) ∈
      set-dist
        (execute
          (verifier-query-round-program fr f-fl f-final as fl final) s)
  and j-bound: j < length f-fl
  and raw-layer:
    ∧ raw.
      0 < fri-layer-lengths (length f-fl) (clength * scale) ! j ∧
      fri-layer-indices (length f-fl) (index (to-nat raw))
        (clength * scale) ! j <
      fri-layer-lengths (length f-fl) (clength * scale) ! j
  obtains raw chunk where
    fri-layer-chunk-authenticated (snd (f-fl ! j))
      (fri-layer-lengths (length f-fl) (clength * scale) ! j)
      (fri-layer-indices (length f-fl) (index (to-nat raw))
        (clength * scale) ! j)
    chunk t
    fmlookup (HashMap t)
      (QueryIndexChallenge (PQueryCounter s) (PState s)) = Some raw
proof -
  from outcome[unfolded verifier-query-round-program-alt-def]
  obtain raw s0 where
    raw:
      Some (raw, s0) ∈
        set-dist (execute receive-query-index-challenge s)
  and tail:
    Some (( ), t) ∈
      set-dist
        (execute
          (verifier-query-round-after-index-program
            fr f-fl f-final as fl final raw) s0)
  by (auto elim!: set-dist-bindE)
  have layer-len-pos:
    0 < fri-layer-lengths (length f-fl) (clength * scale) ! j
  using raw-layer[of raw] by simp
  have layer-idx-bound:
    fri-layer-indices (length f-fl) (index (to-nat raw))
      (clength * scale) ! j <
    fri-layer-lengths (length f-fl) (clength * scale) ! j
  using raw-layer[of raw] by simp
  from verifier-query-round-after-index-trace-fri-selected-chunk-authenticated

```



```

    [OF tail j-bound layer-len-pos layer-idx-bound]
obtain chunk where auth:
  fri-layer-chunk-authenticated (snd (f-fl ! j))
    (fri-layer-lengths (length f-fl) (clength * scale) ! j)
    (fri-layer-indices (length f-fl) (index (to-nat raw))
      (clength * scale) ! j)
    chunk t
by blast
have lookup-s0:
  fmlookup (HashMap s0)
    (QueryIndexChallenge (PQueryCounter s) (PState s)) = Some raw
using receive-query-index-challenge-outcome[OF raw] by simp
have s0-t: s0 ≤ t
using verifier-query-round-after-index-program-outcome[OF tail] by simp
have lookup-t:
  fmlookup (HashMap t)
    (QueryIndexChallenge (PQueryCounter s) (PState s)) = Some raw
by (rule hash-extension-lookup[OF lookup-s0 s0-t])
show ?thesis
by (rule that[OF auth lookup-t])
qed

lemma verifier-query-round-program-composition-fri-head-chunk-authenticated:
fixes s t :: ('f, 'a) protocol-channel-scheme
assumes fl-eq: fl = (b, rt) # fl-tail
and outcome:
  Some ((), t) ∈
    set-dist
      (execute
        (verifier-query-round-program fr f-fl f-final as fl final) s)
obtains raw idx chunk where
  idx = index (to-nat raw)
  fri-layer-chunk-authenticated rt (clength * scale) idx chunk t
proof –
from outcome[unfolded verifier-query-round-program-alt-def]
obtain raw s0 where
  raw:
    Some (raw, s0) ∈
      set-dist (execute receive-query-index-challenge s)
and tail:
  Some ((), t) ∈
    set-dist
      (execute
        (verifier-query-round-after-index-program
          fr f-fl f-final as fl final raw) s0)
by (auto elim!: set-dist-bindE)
from verifier-query-round-after-index-composition-fri-head-chunk-authenticated
  [OF fl-eq tail]
obtain idx chunk where

```

```

    idx-eq: idx = index (to-nat raw)
    and auth: fri-layer-chunk-authenticated rt (clength * scale) idx chunk t
    by blast
show ?thesis
  by (rule that[OF idx-eq auth])
qed

lemma verifier-query-round-program-composition-fri-selected-chunk-authenticated:
  fixes s t :: ('f, 'a) protocol-channel-scheme
  assumes outcome:
    Some (( ), t) ∈
      set-dist
        (execute
          (verifier-query-round-program fr f-fl f-final as fl final) s)
  and j-bound: j < length fl
  and raw-layer:
    ∧ raw.
      0 < fri-layer-lengths (length fl) (clength * scale) ! j ∧
      fri-layer-indices (length fl) (index (to-nat raw))
        (clength * scale) ! j <
      fri-layer-lengths (length fl) (clength * scale) ! j
  obtains raw chunk where
    fri-layer-chunk-authenticated (snd (fl ! j))
    (fri-layer-lengths (length fl) (clength * scale) ! j)
    (fri-layer-indices (length fl) (index (to-nat raw))
      (clength * scale) ! j)
    chunk t
proof -
  from outcome[unfolded verifier-query-round-program-alt-def]
  obtain raw s0 where
    raw:
      Some (raw, s0) ∈
        set-dist (execute receive-query-index-challenge s)
  and tail:
    Some (( ), t) ∈
      set-dist
        (execute
          (verifier-query-round-after-index-program
            fr f-fl f-final as fl final raw) s0)
  by (auto elim!: set-dist-bindE)
  have layer-len-pos:
    0 < fri-layer-lengths (length fl) (clength * scale) ! j
  using raw-layer[of raw] by simp
  have layer-idx-bound:
    fri-layer-indices (length fl) (index (to-nat raw))
      (clength * scale) ! j <
    fri-layer-lengths (length fl) (clength * scale) ! j
  using raw-layer[of raw] by simp
  from

```

```

    verifier-query-round-after-index-composition-fri-selected-chunk-authenticated
      [OF tail j-bound layer-len-pos layer-idx-bound]
obtain chunk where auth:
  fri-layer-chunk-authenticated (snd (fl ! j))
    (fri-layer-lengths (length fl) (clength * scale) ! j)
    (fri-layer-indices (length fl) (index (to-nat raw))
      (clength * scale) ! j)
    chunk t
by blast
show ?thesis
by (rule that[OF auth])
qed

lemma verifier-query-round-program-composition-fri-selected-chunk-authenticated-with-lookup:
fixes s t :: ('f, 'a) protocol-channel-scheme
assumes outcome:
  Some ((), t) ∈
    set-dist
      (execute
        (verifier-query-round-program fr f-fl f-final as fl final) s)
and j-bound: j < length fl
and raw-layer:
   $\bigwedge_{raw.}$ 
    0 < fri-layer-lengths (length fl) (clength * scale) ! j ∧
    fri-layer-indices (length fl) (index (to-nat raw))
      (clength * scale) ! j <
    fri-layer-lengths (length fl) (clength * scale) ! j
obtains raw chunk where
  fri-layer-chunk-authenticated (snd (fl ! j))
    (fri-layer-lengths (length fl) (clength * scale) ! j)
    (fri-layer-indices (length fl) (index (to-nat raw))
      (clength * scale) ! j)
    chunk t
  fmlookup (HashMap t)
    (QueryIndexChallenge (PQueryCounter s) (PState s)) = Some raw
proof –
from outcome[unfolded verifier-query-round-program-alt-def]
obtain raw s0 where
  raw:
    Some (raw, s0) ∈
      set-dist (execute receive-query-index-challenge s)
and tail:
    Some ((), t) ∈
      set-dist
        (execute
          (verifier-query-round-after-index-program
            fr f-fl f-final as fl final raw) s0)
by (auto elim!: set-dist-bindE)
have layer-len-pos:

```

```

    0 < fri-layer-lengths (length fl) (clength * scale) ! j
    using raw-layer[of raw] by simp
  have layer-idx-bound:
    fri-layer-indices (length fl) (index (to-nat raw))
      (clength * scale) ! j <
      fri-layer-lengths (length fl) (clength * scale) ! j
    using raw-layer[of raw] by simp
  from
    verifier-query-round-after-index-composition-fri-selected-chunk-authenticated
    [OF tail j-bound layer-len-pos layer-idx-bound]
  obtain chunk where auth:
    fri-layer-chunk-authenticated (snd (fl ! j))
      (fri-layer-lengths (length fl) (clength * scale) ! j)
      (fri-layer-indices (length fl) (index (to-nat raw))
        (clength * scale) ! j)
    chunk t
  by blast
  have lookup-s0:
    fmlookup (HashMap s0)
      (QueryIndexChallenge (PQueryCounter s) (PState s)) = Some raw
    using receive-query-index-challenge-outcome[OF raw] by simp
  have s0-t: s0 ≤ t
    using verifier-query-round-after-index-program-outcome[OF tail] by simp
  have lookup-t:
    fmlookup (HashMap t)
      (QueryIndexChallenge (PQueryCounter s) (PState s)) = Some raw
    by (rule hash-extension-lookup[OF lookup-s0 s0-t])
  show ?thesis
    by (rule that[OF auth lookup-t])
qed

lemma ntimes-verifier-query-rounds-selected-trace-fri-head-chunk-authenticated-at:
  fixes s t :: ('f, 'a) protocol-channel-scheme
  assumes f-fl-eq: f-fl = (b, rt) # f-fl-tail
  and outcome:
    Some (results, t) ∈
      set-dist
        (execute
          (ntimes
            (verifier-query-round-program fr f-fl f-final as fl final) n)
          s)
  and i-bound: i < n
  obtains raw idx chunk where
    idx = index (to-nat raw)
    fri-layer-chunk-authenticated rt (clength * scale) idx chunk t
proof -
  let ?round = verifier-query-round-program fr f-fl f-final as fl final
  from query-opening-ntimes-outcome-decomp-at[OF outcome i-bound]
  obtain prefix x suffix s-i s-suc where

```

```

round:
  Some (x, s-suc) ∈ set-dist (execute ?round s-i)
and suffix:
  Some (suffix, t) ∈
    set-dist (execute (ntimes ?round (n - Suc i)) s-suc)
by blast
have x-eq: x = ()
  by (cases x) simp
from verifier-query-round-program-trace-fri-head-chunk-authenticated
  [OF f-fl-eq round[unfolded x-eq]]
obtain raw idx chunk where
  idx-eq: idx = index (to-nat raw)
  and auth-suc:
    fri-layer-chunk-authenticated rt (clength * scale) idx chunk s-suc
  by blast
have s-suc-t: s-suc ≤ t
  using ntimes-verifier-query-rounds-outcome[OF suffix] by blast
have auth-t:
  fri-layer-chunk-authenticated rt (clength * scale) idx chunk t
  by (rule fri-layer-chunk-authenticated-mono[OF auth-suc s-suc-t])
show ?thesis
  by (rule that[OF idx-eq auth-t])
qed

lemma ntimes-verifier-query-rounds-selected-trace-fri-chunk-authenticated-at:
  fixes s t :: ('f, 'a) protocol-channel-scheme
  assumes outcome:
    Some (results, t) ∈
      set-dist
        (execute
          (ntimes
            (verifier-query-round-program fr f-fl f-final as fl final) n)
          s)
  and i-bound: i < n
  and j-bound: j < length f-fl
  and raw-layer:
     $\bigwedge^{\text{raw.}}$ 
    0 < fri-layer-lengths (length f-fl) (clength * scale) ! j ∧
    fri-layer-indices (length f-fl) (index (to-nat raw))
    (clength * scale) ! j <
    fri-layer-lengths (length f-fl) (clength * scale) ! j
  obtains raw chunk where
    fri-layer-chunk-authenticated (snd (f-fl ! j))
    (fri-layer-lengths (length f-fl) (clength * scale) ! j)
    (fri-layer-indices (length f-fl) (index (to-nat raw))
    (clength * scale) ! j)
    chunk t
proof -
  let ?round = verifier-query-round-program fr f-fl f-final as fl final

```

```

from query-opening-ntimes-outcome-decomp-at[OF outcome i-bound]
obtain prefix x suffix s-i s-suc where
  round:
    Some (x, s-suc) ∈ set-dist (execute ?round s-i)
  and suffix:
    Some (suffix, t) ∈
      set-dist (execute (ntimes ?round (n - Suc i)) s-suc)
  by blast
have x-eq: x = ()
  by (cases x) simp
from verifier-query-round-program-trace-fri-selected-chunk-authenticated
  [OF round[unfolded x-eq] j-bound raw-layer]
obtain raw chunk where auth-suc:
  fri-layer-chunk-authenticated (snd (f-fl ! j))
  (fri-layer-lengths (length f-fl) (clength * scale) ! j)
  (fri-layer-indices (length f-fl) (index (to-nat raw))
    (clength * scale) ! j)
  chunk s-suc
  by blast
have s-suc-t: s-suc ≤ t
  using ntimes-verifier-query-rounds-outcome[OF suffix] by blast
have auth-t:
  fri-layer-chunk-authenticated (snd (f-fl ! j))
  (fri-layer-lengths (length f-fl) (clength * scale) ! j)
  (fri-layer-indices (length f-fl) (index (to-nat raw))
    (clength * scale) ! j)
  chunk t
  by (rule fri-layer-chunk-authenticated-mono[OF auth-suc s-suc-t])
show ?thesis
  by (rule that[OF auth-t])
qed

lemma ntimes-verifier-query-rounds-selected-trace-fri-chunk-authenticated-with-prefix-at:
fixes s t :: ('f, 'a) protocol-channel-scheme
assumes outcome:
  Some (results, t) ∈
    set-dist
      (execute
        (ntimes
          (verifier-query-round-program fr f-fl f-final as fl final) n)
        s)
and i-bound: i < n
and j-bound: j < length f-fl
and raw-layer:
   $\bigwedge$  raw.
  0 < fri-layer-lengths (length f-fl) (clength * scale) ! j  $\wedge$ 
  fri-layer-indices (length f-fl) (index (to-nat raw))
    (clength * scale) ! j <
  fri-layer-lengths (length f-fl) (clength * scale) ! j

```

obtains *raw idx prefix-query-idxs prefix-chunks chunk suffix-query-idxs*
suffix-chunks layer-chunk **where**
idx = index (to-nat raw)
length prefix-query-idxs = i
length prefix-chunks = i
length suffix-query-idxs = n - Suc i
length suffix-chunks = n - Suc i
verifier-query-round-chunk idx (map snd f-fl) (map snd fl) chunk
 $\wedge k. k < i \implies$
verifier-query-round-chunk (prefix-query-idxs ! k)
(map snd f-fl) (map snd fl) (prefix-chunks ! k)
 $\wedge k. k < n - Suc i \implies$
verifier-query-round-chunk (suffix-query-idxs ! k)
(map snd f-fl) (map snd fl) (suffix-chunks ! k)
PTranscript s =
List.concat (prefix-chunks @ chunk # suffix-chunks) @ PTranscript t
fri-layer-chunk-authenticated (snd (f-fl ! j))
*(fri-layer-lengths (length f-fl) (clength * scale) ! j)*
*(fri-layer-indices (length f-fl) idx (clength * scale) ! j)*
layer-chunk t
fmlookup (HashMap t)
(QueryIndexChallenge (PQueryCounter s + i)
(state-after-query-chunks (PState s) prefix-chunks i)) =
Some raw

proof —

let *?round = verifier-query-round-program fr f-fl f-final as fl final*
from *query-opening-ntimes-outcome-decomp-at[OF outcome i-bound]*
obtain *prefix x suffix s-i s-suc* **where**
prefix:
Some (prefix, s-i) ∈
set-dist (execute (ntimes ?round i) s)
and *round:*
Some (x, s-suc) ∈ set-dist (execute ?round s-i)
and *suffix:*
Some (suffix, t) ∈
set-dist (execute (ntimes ?round (n - Suc i)) s-suc)
by *blast*
have *x-eq: x = ()*
by *(cases x) simp*
show *?thesis*
proof (*rule query-opening-ntimes-prefix-transcript-state-alignment*
[OF prefix])
fix *prefix-query-idxs prefix-chunks*
assume *len-prefix-idxs: length prefix-query-idxs = i*
and *len-prefix-chunks: length prefix-chunks = i*
and *transcript-prefix:*
PTranscript s = List.concat prefix-chunks @ PTranscript s-i
and *state-i:*
PState s-i =

```

    state-after-query-chunks (PState s) prefix-chunks i
  and counter-i: PQueryCounter s-i = PQueryCounter s + i
  and prefix-ext: s ≤ s-i
  and prefix-rounds:
    ∧k. k < i ⇒
      verifier-query-round-chunk (prefix-query-idxs ! k)
      (map snd f-fl) (map snd fl) (prefix-chunks ! k)
show ?thesis
proof (rule verifier-query-round-program-outcome-with-authenticated-trace-openings
  [OF round[unfolded x-eq]])
  fix raw idx chunk openings
  assume idx-eq: idx = index (to-nat raw)
  and chunk-shape:
    verifier-query-round-chunk idx (map snd f-fl) (map snd fl)
    chunk
  and transcript-round:
    PTranscript s-i = chunk @ PTranscript s-suc
  and state-suc:
    PState s-suc = foldl concat (PState s-i) chunk
  and round-ext: s-i ≤ s-suc
  and counter-suc: PQueryCounter s-suc = Suc (PQueryCounter s-i)
  and lookup-suc:
    fmlookup (HashMap s-suc)
    (QueryIndexChallenge (PQueryCounter s-i) (PState s-i)) =
    Some raw
  and len-openings:
    length openings = length (powers-scaled idx)
  and trace-indices:
    map opening-index openings = powers-scaled idx
  and trace-table-suc:
    partial-authenticated-table fr (scale * clength) openings
    s-suc
show ?thesis
proof (rule
  verifier-query-round-program-trace-fri-selected-chunk-authenticated-with-lookup
  [OF round[unfolded x-eq] j-bound raw-layer])
  fix raw' layer-chunk
  assume auth-suc:
    fri-layer-chunk-authenticated (snd (f-fl ! j))
    (fri-layer-lengths (length f-fl) (clength * scale) ! j)
    (fri-layer-indices (length f-fl) (index (to-nat raw')))
    (clength * scale) ! j
    layer-chunk s-suc
  and lookup-suc':
    fmlookup (HashMap s-suc)
    (QueryIndexChallenge (PQueryCounter s-i) (PState s-i)) =
    Some raw'
  have raw'-eq: raw' = raw
  using lookup-suc lookup-suc' by simp

```



```

show ?thesis
proof (rule query-opening-ntimes-suffix-transcript-state-alignment
  [OF suffix])
fix suffix-query-idxs suffix-chunks
assume len-suffix-idxs: length suffix-query-idxs = n - Suc i
and len-suffix-chunks:
  length suffix-chunks = n - Suc i
and transcript-suffix:
  PTranscript s-suc =
    List.concat suffix-chunks @ PTranscript t
and suffix-state:
  PState t =
    state-after-query-chunks (PState s-suc) suffix-chunks
    (n - Suc i)
and suffix-counter:
  PQueryCounter t = PQueryCounter s-suc + (n - Suc i)
and suffix-ext: s-suc ≤ t
and suffix-rounds:
   $\bigwedge k. k < n - \text{Suc } i \implies$ 
    verifier-query-round-chunk (suffix-query-idxs ! k)
    (map snd f-fl) (map snd fl) (suffix-chunks ! k)
have transcript-all:
  PTranscript s =
    List.concat (prefix-chunks @ chunk # suffix-chunks) @
    PTranscript t
using transcript-prefix transcript-round transcript-suffix
by simp
have auth-t-row:
  fri-layer-chunk-authenticated (snd (f-fl ! j))
  (fri-layer-lengths (length f-fl) (clength * scale) ! j)
  (fri-layer-indices (length f-fl) (index (to-nat raw))
    (clength * scale) ! j)
  layer-chunk t
by (rule fri-layer-chunk-authenticated-mono
  [OF auth-suc[unfolded raw'-eq] suffix-ext])
have auth-t:
  fri-layer-chunk-authenticated (snd (f-fl ! j))
  (fri-layer-lengths (length f-fl) (clength * scale) ! j)
  (fri-layer-indices (length f-fl) idx (clength * scale) ! j)
  layer-chunk t
using auth-t-row idx-eq by simp
have lookup-t:
  fmlookup (HashMap t)
  (QueryIndexChallenge (PQueryCounter s + i)
    (state-after-query-chunks (PState s) prefix-chunks i)) =
    Some raw
using hash-extension-lookup[OF lookup-suc suffix-ext]
unfolding counter-i state-i .
show ?thesis

```

```

    by (rule that[OF idx-eq len-prefix-idxs len-prefix-chunks
        len-suffix-idxs len-suffix-chunks chunk-shape
        prefix-rounds suffix-rounds transcript-all auth-t
        lookup-t])
  qed
qed
qed
qed
qed

```

lemma *ntimes-verifier-query-rounds-selected-trace-fri-chunk-aligned-with-replay-at*:

fixes *query-state final-state* :: ('f, 'a) *protocol-channel-scheme*

assumes *outcome*:

Some (results, final-state) ∈
set-dist
(execute
(ntimes
(verifier-query-round-program fr f-fl f-final as fl final)
rounds)
query-state)

and *i-bound*: $i < \text{rounds}$

and *j-bound*: $j < \text{length } f\text{-fl}$

and *raw-layer*:

$\bigwedge \text{raw.}$
 $0 < \text{fri-layer-lengths } (\text{length } f\text{-fl}) \ (\text{clength} * \text{scale}) \ ! \ j \wedge$
 $\text{fri-layer-indices } (\text{length } f\text{-fl}) \ (\text{index } (\text{to-nat } \text{raw}))$
 $(\text{clength} * \text{scale}) \ ! \ j <$
 $\text{fri-layer-lengths } (\text{length } f\text{-fl}) \ (\text{clength} * \text{scale}) \ ! \ j$

and *len-raw*: $\text{length } \text{raw-idxs} = \text{rounds}$

and *query-idxs-eq*:

$\text{query-idxs} = \text{map } (\lambda \text{raw. index } (\text{to-nat } \text{raw})) \ \text{raw-idxs}$

and *len-query-chunks*: $\text{length } \text{query-chunks} = \text{rounds}$

and *transcript-query*:

$P\text{Transcript } \text{query-state} =$
 $\text{List.concat } \text{query-chunks} \ @ \ P\text{Transcript } \text{final-state}$

and *query-chunk*:

$\bigwedge i. i < \text{rounds} \implies$
 $\text{verifier-query-round-chunk } (\text{query-idxs} \ ! \ i)$
 $(\text{map } \text{snd } f\text{-fl}) \ (\text{map } \text{snd } fl) \ (\text{query-chunks} \ ! \ i)$

and *replay-lookup*:

$\bigwedge i. i < \text{rounds} \implies$
 $\text{fmlookup } (\text{HashMap } \text{final-state})$
 $(\text{QueryIndexChallenge } (P\text{QueryCounter } \text{query-state} + i)$
 $(\text{state-after-query-chunks}$
 $(P\text{State } \text{query-state}) \ \text{query-chunks } i)) =$
 $\text{Some } (\text{raw-idxs} \ ! \ i)$

obtains *layer-chunk* **where**

$\text{fri-layer-chunk-authenticated } (\text{snd } (f\text{-fl} \ ! \ j))$
 $(\text{fri-layer-lengths } (\text{length } f\text{-fl}) \ (\text{clength} * \text{scale}) \ ! \ j)$

```

    (fri-layer-indices (length f-fl) (query-idxs ! i)
      (clength * scale) ! j)
    layer-chunk final-state
proof (rule
  ntimes-verifier-query-rounds-selected-trace-fri-chunk-authenticated-with-prefix-at
  [OF outcome i-bound j-bound raw-layer])
fix raw idx prefix-query-idxs prefix-chunks chunk suffix-query-idxs
  suffix-chunks layer-chunk
assume selected-idx: idx = index (to-nat raw)
and len-prefix-idxs: length prefix-query-idxs = i
and len-prefix-chunks: length prefix-chunks = i
and len-suffix-idxs: length suffix-query-idxs = rounds - Suc i
and len-suffix-chunks: length suffix-chunks = rounds - Suc i
and chunk-shape:
  verifier-query-round-chunk idx (map snd f-fl) (map snd fl) chunk
and prefix-rounds:
   $\bigwedge k. k < i \implies$ 
    verifier-query-round-chunk (prefix-query-idxs ! k)
      (map snd f-fl) (map snd fl) (prefix-chunks ! k)
and suffix-rounds:
   $\bigwedge k. k < rounds - Suc i \implies$ 
    verifier-query-round-chunk (suffix-query-idxs ! k)
      (map snd f-fl) (map snd fl) (suffix-chunks ! k)
and selected-transcript:
  PTranscript query-state =
    List.concat (prefix-chunks @ chunk # suffix-chunks) @
      PTranscript final-state
and auth:
  fri-layer-chunk-authenticated (snd (f-fl ! j))
    (fri-layer-lengths (length f-fl) (clength * scale) ! j)
    (fri-layer-indices (length f-fl) idx (clength * scale) ! j)
    layer-chunk final-state
and selected-lookup:
  fmlookup (HashMap final-state)
    (QueryIndexChallenge (PQueryCounter query-state + i)
      (state-after-query-chunks (PState query-state) prefix-chunks i)) =
    Some raw
let ?selected-chunks = prefix-chunks @ chunk # suffix-chunks
have len-selected-chunks: length ?selected-chunks = rounds
  using len-prefix-chunks len-suffix-chunks i-bound by simp
have selected-chunk-lens:
   $\bigwedge k. k < rounds \implies$ 
    length (?selected-chunks ! k) =
      verifier-query-round-transcript-length (query-idxs ! k)
        (map snd f-fl) (map snd fl)
proof -
  fix k
  assume k-bound: k < rounds
  show

```

```

length (?selected-chunks ! k) =
  verifier-query-round-transcript-length (query-idxs ! k)
  (map snd f-fl) (map snd fl)
proof (cases k < i)
  case True
  have nth-eq: ?selected-chunks ! k = prefix-chunks ! k
    using True len-prefix-chunks by (simp add: nth-append)
  have
    length (prefix-chunks ! k) =
      verifier-query-round-transcript-length
        (prefix-query-idxs ! k) (map snd f-fl) (map snd fl)
    by (rule verifier-query-round-chunk-length[OF prefix-rounds[OF True]])
  then show ?thesis
    using nth-eq
    by (simp add: verifier-query-round-transcript-length-index-irrelevant)
next
  case False
  show ?thesis
  proof (cases k = i)
    case True
    have nth-eq: ?selected-chunks ! k = chunk
      using True len-prefix-chunks by (simp add: nth-append)
    have
      length chunk =
        verifier-query-round-transcript-length idx
        (map snd f-fl) (map snd fl)
      by (rule verifier-query-round-chunk-length[OF chunk-shape])
    then show ?thesis
      using nth-eq
      by (simp add:
        verifier-query-round-transcript-length-index-irrelevant)
    next
    case False
    let ?k = k - Suc i
    have ge: Suc i ≤ k
      using <¬ k < i> False by linarith
    have k-bound': ?k < rounds - Suc i
      using ge k-bound by linarith
    have nth-eq: ?selected-chunks ! k = suffix-chunks ! ?k
      using ge len-prefix-chunks by (simp add: nth-append)
    have
      length (suffix-chunks ! ?k) =
        verifier-query-round-transcript-length
          (suffix-query-idxs ! ?k) (map snd f-fl) (map snd fl)
      by (rule verifier-query-round-chunk-length
        [OF suffix-rounds[OF k-bound']])
    then show ?thesis
      using nth-eq
      by (simp add:

```

```

    verifier-query-round-transcript-length-index-irrelevant)
  qed
  qed
  qed
  have query-chunk-lens:
     $\bigwedge k. k < \text{rounds} \implies$ 
    length (query-chunks ! k) =
      verifier-query-round-transcript-length (query-idxs ! k)
      (map snd f-fl) (map snd fl)
    using query-chunk verifier-query-round-chunk-length by blast
  have concat-eq: List.concat ?selected-chunks = List.concat query-chunks
    using selected-transcript transcript-query by simp
  have chunks-eq: ?selected-chunks = query-chunks
  proof -
    have same-lens:
       $\bigwedge k. k < \text{length } ?\text{selected-chunks} \implies$ 
      length (?selected-chunks ! k) = length (query-chunks ! k)
      using selected-chunk-lens query-chunk-lens len-selected-chunks by simp
    have concat-eq-empty:
      List.concat ?selected-chunks @ [] = List.concat query-chunks
      using concat-eq by simp
    have len-eq: length ?selected-chunks = length query-chunks
      using len-selected-chunks len-query-chunks by simp
    have ?selected-chunks = query-chunks  $\wedge$  ([] :: 'f list) = []
      by (rule concat-append-eq-concat-same-chunk-lengths
        [OF len-eq same-lens concat-eq-empty])
    then show ?thesis
      by simp
  qed
  have prefix-state-eq:
    state-after-query-chunks (PState query-state) prefix-chunks i =
    state-after-query-chunks (PState query-state) query-chunks i
  proof -
    have
      state-after-query-chunks (PState query-state) ?selected-chunks i =
      state-after-query-chunks (PState query-state) prefix-chunks i
      by (rule state-after-query-chunks-append-prefix[OF len-prefix-chunks])
    then show ?thesis
      using chunks-eq by simp
  qed
  have selected-lookup-global:
    fmlookup (HashMap final-state)
      (QueryIndexChallenge (PQueryCounter query-state + i)
        (state-after-query-chunks (PState query-state) query-chunks i)) =
    Some raw
    using selected-lookup prefix-state-eq by simp
  have raw-eq: raw = raw-idxs ! i
    using selected-lookup-global replay-lookup[OF i-bound] by simp
  have idx-eq: idx = query-idxs ! i

```

using *selected-idx raw-eq query-idxs-eq len-raw i-bound* **by** *simp*
show *?thesis*
by (*rule that[of layer-chunk]*) (*use auth idx-eq in simp*)
qed

lemma *ntimes-verifier-query-rounds-selected-composition-fri-head-chunk-authenticated-at:*
fixes *s t :: ('f, 'a) protocol-channel-scheme*
assumes *fl-eq: fl = (b, rt) # fl-tail*
and *outcome:*
Some (results, t) ∈
set-dist
(execute
(ntimes
(verifier-query-round-program fr f-fl f-final as fl final) n)
s)
and *i-bound: i < n*
obtains *raw idx chunk* **where**
idx = index (to-nat raw)
*fri-layer-chunk-authenticated rt (clength * scale) idx chunk t*
proof –
let *?round = verifier-query-round-program fr f-fl f-final as fl final*
from *query-opening-ntimes-outcome-decomp-at[OF outcome i-bound]*
obtain *prefix x suffix s-i s-suc* **where**
round:
Some (x, s-suc) ∈ set-dist (execute ?round s-i)
and *suffix:*
Some (suffix, t) ∈
set-dist (execute (ntimes ?round (n – Suc i)) s-suc)
by *blast*
have *x-eq: x = ()*
by (*cases x*) *simp*
from *verifier-query-round-program-composition-fri-head-chunk-authenticated*
[OF fl-eq round[unfolded x-eq]]
obtain *raw idx chunk* **where**
idx-eq: idx = index (to-nat raw)
and *auth-suc:*
*fri-layer-chunk-authenticated rt (clength * scale) idx chunk s-suc*
by *blast*
have *s-suc-t: s-suc ≤ t*
using *ntimes-verifier-query-rounds-outcome[OF suffix]* **by** *blast*
have *auth-t:*
*fri-layer-chunk-authenticated rt (clength * scale) idx chunk t*
by (*rule fri-layer-chunk-authenticated-mono[OF auth-suc s-suc-t]*)
show *?thesis*
by (*rule that[OF idx-eq auth-t]*)
qed

lemma *ntimes-verifier-query-rounds-selected-composition-fri-chunk-authenticated-at:*
fixes *s t :: ('f, 'a) protocol-channel-scheme*

```

assumes outcome:
  Some (results, t) ∈
    set-dist
      (execute
        (ntimes
          (verifier-query-round-program fr f-fl f-final as fl final) n)
        s)
and i-bound: i < n
and j-bound: j < length fl
and raw-layer:
   $\bigwedge$ raw.
    0 < fri-layer-lengths (length fl) (clength * scale) ! j ∧
    fri-layer-indices (length fl) (index (to-nat raw))
      (clength * scale) ! j <
    fri-layer-lengths (length fl) (clength * scale) ! j
obtains raw chunk where
  fri-layer-chunk-authenticated (snd (fl ! j))
    (fri-layer-lengths (length fl) (clength * scale) ! j)
    (fri-layer-indices (length fl) (index (to-nat raw))
      (clength * scale) ! j)
  chunk t
proof –
let ?round = verifier-query-round-program fr f-fl f-final as fl final
from query-opening-ntimes-outcome-decomp-at[OF outcome i-bound]
obtain prefix x suffix s-i s-suc where
  round:
    Some (x, s-suc) ∈ set-dist (execute ?round s-i)
and suffix:
  Some (suffix, t) ∈
    set-dist (execute (ntimes ?round (n - Suc i)) s-suc)
by blast
have x-eq: x = ()
by (cases x) simp
from
  verifier-query-round-program-composition-fri-selected-chunk-authenticated
    [OF round[unfolded x-eq] j-bound raw-layer]
obtain raw chunk where auth-suc:
  fri-layer-chunk-authenticated (snd (fl ! j))
    (fri-layer-lengths (length fl) (clength * scale) ! j)
    (fri-layer-indices (length fl) (index (to-nat raw))
      (clength * scale) ! j)
  chunk s-suc
by blast
have s-suc-t: s-suc ≤ t
using ntimes-verifier-query-rounds-outcome[OF suffix] by blast
have auth-t:
  fri-layer-chunk-authenticated (snd (fl ! j))
    (fri-layer-lengths (length fl) (clength * scale) ! j)
    (fri-layer-indices (length fl) (index (to-nat raw))
      (clength * scale) ! j)

```

```

      (clength * scale) ! j)
    chunk t
  by (rule fri-layer-chunk-authenticated-mono[OF auth-suc s-suc-t])
show ?thesis
  by (rule that[OF auth-t])
qed

```

lemma *ntimes-verifier-query-rounds-selected-composition-fri-chunk-authenticated-with-prefix-at:*

fixes $s\ t :: ('f, 'a)\ \text{protocol-channel-scheme}$

assumes *outcome:*

Some (results, t) ∈

set-dist

(execute

(ntimes

(verifier-query-round-program fr f-fl f-final as fl final) n)

s)

and *i-bound:* $i < n$

and *j-bound:* $j < \text{length } fl$

and *raw-layer:*

$\bigwedge \text{raw.}$

$0 < \text{fri-layer-lengths } (\text{length } fl) (\text{clength} * \text{scale}) ! j \wedge$

$\text{fri-layer-indices } (\text{length } fl) (\text{index } (\text{to-nat } \text{raw}))$

$(\text{clength} * \text{scale}) ! j <$

$\text{fri-layer-lengths } (\text{length } fl) (\text{clength} * \text{scale}) ! j$

obtains *raw idx prefix-query-idxs prefix-chunks chunk suffix-query-idxs*

suffix-chunks layer-chunk **where**

idx = index (to-nat raw)

length prefix-query-idxs = i

length prefix-chunks = i

length suffix-query-idxs = n - Suc i

length suffix-chunks = n - Suc i

verifier-query-round-chunk idx (map snd f-fl) (map snd fl) chunk

$\bigwedge k. k < i \implies$

verifier-query-round-chunk (prefix-query-idxs ! k)

(map snd f-fl) (map snd fl) (prefix-chunks ! k)

$\bigwedge k. k < n - \text{Suc } i \implies$

verifier-query-round-chunk (suffix-query-idxs ! k)

(map snd f-fl) (map snd fl) (suffix-chunks ! k)

PTranscript s =

List.concat (prefix-chunks @ chunk # suffix-chunks) @ PTranscript t

fri-layer-chunk-authenticated (snd (fl ! j))

*(fri-layer-lengths (length fl) (clength * scale) ! j)*

*(fri-layer-indices (length fl) idx (clength * scale) ! j)*

layer-chunk t

fmlookup (HashMap t)

(QueryIndexChallenge (PQueryCounter s + i)

(state-after-query-chunks (PState s) prefix-chunks i)) =

Some raw

proof –


```

let ?round = verifier-query-round-program fr f-fl f-final as fl final
from query-opening-ntimes-outcome-decomp-at[OF outcome i-bound]
obtain prefix x suffix s-i s-suc where
  prefix:
    Some (prefix, s-i) ∈
      set-dist (execute (ntimes ?round i) s)
and round:
    Some (x, s-suc) ∈ set-dist (execute ?round s-i)
and suffix:
    Some (suffix, t) ∈
      set-dist (execute (ntimes ?round (n - Suc i)) s-suc)
by blast
have x-eq: x = ()
  by (cases x) simp
show ?thesis
proof (rule query-opening-ntimes-prefix-transcript-state-alignment
  [OF prefix])
  fix prefix-query-idxs prefix-chunks
  assume len-prefix-idxs: length prefix-query-idxs = i
    and len-prefix-chunks: length prefix-chunks = i
    and transcript-prefix:
      PTranscript s = List.concat prefix-chunks @ PTranscript s-i
  and state-i:
    PState s-i =
      state-after-query-chunks (PState s) prefix-chunks i
  and counter-i: PQueryCounter s-i = PQueryCounter s + i
  and prefix-ext: s ≤ s-i
  and prefix-rounds:
     $\bigwedge k. k < i \implies$ 
      verifier-query-round-chunk (prefix-query-idxs ! k)
      (map snd f-fl) (map snd fl) (prefix-chunks ! k)
  show ?thesis
proof (rule verifier-query-round-program-outcome-with-authenticated-trace-openings
  [OF round[unfolded x-eq]])
  fix raw idx chunk openings
  assume idx-eq: idx = index (to-nat raw)
    and chunk-shape:
      verifier-query-round-chunk idx (map snd f-fl) (map snd fl)
      chunk
  and transcript-round:
    PTranscript s-i = chunk @ PTranscript s-suc
  and state-suc:
    PState s-suc = foldl concat (PState s-i) chunk
  and round-ext: s-i ≤ s-suc
  and counter-suc: PQueryCounter s-suc = Suc (PQueryCounter s-i)
  and lookup-suc:
    fmlookup (HashMap s-suc)
    (QueryIndexChallenge (PQueryCounter s-i) (PState s-i)) =
      Some raw

```

```

and len-openings:
  length openings = length (powers-scaled idx)
and trace-indices:
  map opening-index openings = powers-scaled idx
and trace-table-suc:
  partial-authenticated-table fr (scale * clength) openings
    s-suc
show ?thesis
proof (rule
  verifier-query-round-program-composition-fri-selected-chunk-authenticated-with-lookup
    [OF round[unfolded x-eq] j-bound raw-layer])
fix raw' layer-chunk
assume auth-suc:
  fri-layer-chunk-authenticated (snd (fl ! j))
    (fri-layer-lengths (length fl) (clength * scale) ! j)
    (fri-layer-indices (length fl) (index (to-nat raw'))
      (clength * scale) ! j)
  layer-chunk s-suc
and lookup-suc':
  fmlookup (HashMap s-suc)
    (QueryIndexChallenge (PQueryCounter s-i) (PState s-i)) =
    Some raw'
have raw'-eq: raw' = raw
  using lookup-suc lookup-suc' by simp
show ?thesis
proof (rule query-opening-ntimes-suffix-transcript-state-alignment
  [OF suffix])
fix suffix-query-idxs suffix-chunks
assume len-suffix-idxs: length suffix-query-idxs = n - Suc i
  and len-suffix-chunks:
    length suffix-chunks = n - Suc i
  and transcript-suffix:
    PTranscript s-suc =
      List.concat suffix-chunks @ PTranscript t
  and suffix-state:
    PState t =
      state-after-query-chunks (PState s-suc) suffix-chunks
        (n - Suc i)
  and suffix-counter:
    PQueryCounter t = PQueryCounter s-suc + (n - Suc i)
  and suffix-ext: s-suc ≤ t
  and suffix-rounds:
     $\bigwedge k. k < n - \text{Suc } i \implies$ 
      verifier-query-round-chunk (suffix-query-idxs ! k)
        (map snd f-fl) (map snd fl) (suffix-chunks ! k)
have transcript-all:
  PTranscript s =
    List.concat (prefix-chunks @ chunk # suffix-chunks) @
      PTranscript t

```

```

    using transcript-prefix transcript-round transcript-suffix
    by simp
  have auth-t-raw:
    fri-layer-chunk-authenticated (snd (fl ! j))
      (fri-layer-lengths (length fl) (clength * scale) ! j)
      (fri-layer-indices (length fl) (index (to-nat raw))
        (clength * scale) ! j)
    layer-chunk t
  by (rule fri-layer-chunk-authenticated-mono
    [OF auth-suc[unfolded raw'-eq] suffix-ext])
  have auth-t:
    fri-layer-chunk-authenticated (snd (fl ! j))
      (fri-layer-lengths (length fl) (clength * scale) ! j)
      (fri-layer-indices (length fl) idx (clength * scale) ! j)
    layer-chunk t
  using auth-t-raw idx-eq by simp
  have lookup-t:
    fmllookup (HashMap t)
      (QueryIndexChallenge (PQueryCounter s + i)
        (state-after-query-chunks (PState s) prefix-chunks i)) =
    Some raw
  using hash-extension-lookup[OF lookup-suc suffix-ext]
  unfolding counter-i state-i .
  show ?thesis
  by (rule that[OF idx-eq len-prefix-idxs len-prefix-chunks
    len-suffix-idxs len-suffix-chunks chunk-shape
    prefix-rounds suffix-rounds transcript-all auth-t
    lookup-t])
qed
qed
qed
qed
qed

lemma ntimes-verifier-query-rounds-selected-composition-fri-chunk-aligned-with-replay-at:
  fixes query-state final-state :: ('f, 'a) protocol-channel-scheme
  assumes outcome:
    Some (results, final-state) ∈
      set-dist
        (execute
          (ntimes
            (verifier-query-round-program fr f-fl f-final as fl final)
            rounds)
          query-state)
  and i-bound: i < rounds
  and j-bound: j < length fl
  and raw-layer:
     $\bigwedge_{\text{raw.}}$ 
    0 < fri-layer-lengths (length fl) (clength * scale) ! j  $\wedge$ 

```

```

    fri-layer-indices (length fl) (index (to-nat raw))
      (clength * scale) ! j <
    fri-layer-lengths (length fl) (clength * scale) ! j
  and len-raw: length raw-idxs = rounds
  and query-idxs-eq:
    query-idxs = map (λraw. index (to-nat raw)) raw-idxs
  and len-query-chunks: length query-chunks = rounds
  and transcript-query:
    PTranscript query-state =
      List.concat query-chunks @ PTranscript final-state
  and query-chunk:
    ∧i. i < rounds ⇒
      verifier-query-round-chunk (query-idxs ! i)
        (map snd f-fl) (map snd fl) (query-chunks ! i)
  and replay-lookup:
    ∧i. i < rounds ⇒
      fmllookup (HashMap final-state)
        (QueryIndexChallenge (PQueryCounter query-state + i)
          (state-after-query-chunks
            (PState query-state) query-chunks i)) =
        Some (raw-idxs ! i)
  obtains layer-chunk where
    fri-layer-chunk-authenticated (snd (fl ! j))
      (fri-layer-lengths (length fl) (clength * scale) ! j)
      (fri-layer-indices (length fl) (query-idxs ! i)
        (clength * scale) ! j)
    layer-chunk final-state
  proof (rule
    ntimes-verifier-query-rounds-selected-composition-fri-chunk-authenticated-with-prefix-at
      [OF outcome i-bound j-bound raw-layer])
  fix raw idx prefix-query-idxs prefix-chunks chunk suffix-query-idxs
    suffix-chunks layer-chunk
  assume selected-idx: idx = index (to-nat raw)
  and len-prefix-idxs: length prefix-query-idxs = i
  and len-prefix-chunks: length prefix-chunks = i
  and len-suffix-idxs: length suffix-query-idxs = rounds - Suc i
  and len-suffix-chunks: length suffix-chunks = rounds - Suc i
  and chunk-shape:
    verifier-query-round-chunk idx (map snd f-fl) (map snd fl) chunk
  and prefix-rounds:
    ∧k. k < i ⇒
      verifier-query-round-chunk (prefix-query-idxs ! k)
        (map snd f-fl) (map snd fl) (prefix-chunks ! k)
  and suffix-rounds:
    ∧k. k < rounds - Suc i ⇒
      verifier-query-round-chunk (suffix-query-idxs ! k)
        (map snd f-fl) (map snd fl) (suffix-chunks ! k)
  and selected-transcript:
    PTranscript query-state =

```

```

    List.concat (prefix-chunks @ chunk # suffix-chunks) @
      PTranscript final-state
and auth:
  fri-layer-chunk-authenticated (snd (fl ! j))
  (fri-layer-lengths (length fl) (clength * scale) ! j)
  (fri-layer-indices (length fl) idx (clength * scale) ! j)
  layer-chunk final-state
and selected-lookup:
  fmlookup (HashMap final-state)
    (QueryIndexChallenge (PQueryCounter query-state + i)
      (state-after-query-chunks (PState query-state) prefix-chunks i)) =
    Some raw
let ?selected-chunks = prefix-chunks @ chunk # suffix-chunks
have len-selected-chunks: length ?selected-chunks = rounds
  using len-prefix-chunks len-suffix-chunks i-bound by simp
have selected-chunk-lens:
   $\wedge k. k < rounds \implies$ 
    length (?selected-chunks ! k) =
      verifier-query-round-transcript-length (query-idxs ! k)
        (map snd f-fl) (map snd fl)
proof -
  fix k
  assume k-bound: k < rounds
  show
    length (?selected-chunks ! k) =
      verifier-query-round-transcript-length (query-idxs ! k)
        (map snd f-fl) (map snd fl)
proof (cases k < i)
  case True
  have nth-eq: ?selected-chunks ! k = prefix-chunks ! k
    using True len-prefix-chunks by (simp add: nth-append)
  have
    length (prefix-chunks ! k) =
      verifier-query-round-transcript-length
        (prefix-query-idxs ! k) (map snd f-fl) (map snd fl)
    by (rule verifier-query-round-chunk-length[OF prefix-rounds[OF True]])
  then show ?thesis
    using nth-eq
    by (simp add: verifier-query-round-transcript-length-index-irrelevant)
next
  case False
  show ?thesis
  proof (cases k = i)
  case True
  have nth-eq: ?selected-chunks ! k = chunk
    using True len-prefix-chunks by (simp add: nth-append)
  have
    length chunk =
      verifier-query-round-transcript-length idx

```

```

      (map snd f-fl) (map snd fl)
    by (rule verifier-query-round-chunk-length[OF chunk-shape])
  then show ?thesis
    using nth-eq
    by (simp add:
        verifier-query-round-transcript-length-index-irrelevant)
next
case False
let ?k = k - Suc i
have ge: Suc i ≤ k
  using <¬ k < i> False by linarith
have k-bound': ?k < rounds - Suc i
  using ge k-bound by linarith
have nth-eq: ?selected-chunks ! k = suffix-chunks ! ?k
  using ge len-prefix-chunks by (simp add: nth-append)
have
  length (suffix-chunks ! ?k) =
    verifier-query-round-transcript-length
      (suffix-query-idxs ! ?k) (map snd f-fl) (map snd fl)
  by (rule verifier-query-round-chunk-length
      [OF suffix-rounds[OF k-bound']])
then show ?thesis
  using nth-eq
  by (simp add:
      verifier-query-round-transcript-length-index-irrelevant)
qed
qed
qed
have query-chunk-lens:
  ∧k. k < rounds ⇒
    length (query-chunks ! k) =
      verifier-query-round-transcript-length (query-idxs ! k)
        (map snd f-fl) (map snd fl)
  using query-chunk verifier-query-round-chunk-length by blast
have concat-eq: List.concat ?selected-chunks = List.concat query-chunks
  using selected-transcript transcript-query by simp
have chunks-eq: ?selected-chunks = query-chunks
proof -
  have same-lens:
    ∧k. k < length ?selected-chunks ⇒
      length (?selected-chunks ! k) = length (query-chunks ! k)
    using selected-chunk-lens query-chunk-lens len-selected-chunks by simp
  have concat-eq-empty:
    List.concat ?selected-chunks @ [] = List.concat query-chunks
    using concat-eq by simp
  have len-eq: length ?selected-chunks = length query-chunks
    using len-selected-chunks len-query-chunks by simp
  have ?selected-chunks = query-chunks ∧ ([] :: 'f list) = []
    by (rule concat-append-eq-concat-same-chunk-lengths)

```

```

    [OF len-eq same-lens concat-eq-empty])
  then show ?thesis
    by simp
qed
have prefix-state-eq:
  state-after-query-chunks (PState query-state) prefix-chunks i =
    state-after-query-chunks (PState query-state) query-chunks i
proof -
  have
    state-after-query-chunks (PState query-state) ?selected-chunks i =
      state-after-query-chunks (PState query-state) prefix-chunks i
    by (rule state-after-query-chunks-append-prefix[OF len-prefix-chunks])
  then show ?thesis
    using chunks-eq by simp
qed
have selected-lookup-global:
  fmlookup (HashMap final-state)
    (QueryIndexChallenge (PQueryCounter query-state + i)
      (state-after-query-chunks (PState query-state) query-chunks i)) =
    Some raw
  using selected-lookup prefix-state-eq by simp
have raw-eq: raw = raw-idxs ! i
  using selected-lookup-global replay-lookup[OF i-bound] by simp
have idx-eq: idx = query-idxs ! i
  using selected-idx raw-eq query-idxs-eq len-raw i-bound by simp
show ?thesis
  by (rule that[of layer-chunk]) (use auth idx-eq in simp)
qed

lemma accepted-fri-opening-transcript-trace-selected-chunk-authenticated-at:
  assumes fri-openings:
    accepted-fri-opening-transcript s out trace-roots trace-bs trace-final
    dg composition-roots composition-bs composition-final query-idxs
    trace-round-layers composition-round-layers
  and out-eq: out = Some (result, final-state)
  and round-bound: i < rounds
  and layer-bound: j < length trace-roots
  and raw-layer:
     $\bigwedge_{\text{raw.}}$ 
    0 < fri-layer-lengths (length trace-roots) (clength * scale) ! j  $\wedge$ 
    fri-layer-indices (length trace-roots) (index (to-nat raw))
    (clength * scale) ! j <
    fri-layer-lengths (length trace-roots) (clength * scale) ! j
  obtains layer-chunk where
    fri-layer-chunk-authenticated (trace-roots ! j)
    (fri-layer-lengths (length trace-roots) (clength * scale) ! j)
    (fri-layer-indices (length trace-roots) (query-idxs ! i)
      (clength * scale) ! j)
    layer-chunk final-state

```

proof –

```

from fri-openings obtain result' final-state' fr f-fl as fl query-state
  raw-idxs query-chunks where
    out-def: out = Some (result', final-state')
    and trace-roots-eq: trace-roots = map snd f-fl
    and composition-roots-eq: composition-roots = map snd fl
    and outcome:
      Some (result', final-state') ∈
        set-dist (execute
          (ntimes
            (verifier-query-round-program fr f-fl trace-final as fl
              composition-final)
            rounds)
          query-state)
    and len-raw: length raw-idxs = rounds
    and query-idxs-eq:
      query-idxs = map (λraw. index (to-nat raw)) raw-idxs
    and len-query-chunks: length query-chunks = rounds
    and transcript-query:
      PTranscript query-state =
        List.concat query-chunks @ PTranscript final-state'
    and query-chunk:
       $\bigwedge i. i < rounds \implies$ 
        verifier-query-round-chunk (query-idxs ! i)
        trace-roots composition-roots (query-chunks ! i)
    and replay-lookup:
       $\bigwedge i. i < rounds \implies$ 
        fmllookup (HashMap final-state')
        (QueryIndexChallenge (PQueryCounter query-state + i)
          (state-after-query-chunks (PState query-state) query-chunks i)) =
          Some (raw-idxs ! i)
    unfolding accepted-fri-opening-transcript-def by blast
have result-eq: result' = result
  using out-def out-eq by simp
have final-state-eq: final-state' = final-state
  using out-def out-eq by simp
have outcome':
  Some (result, final-state) ∈
    set-dist (execute
      (ntimes
        (verifier-query-round-program fr f-fl trace-final as fl
          composition-final)
        rounds)
      query-state)
  using outcome result-eq final-state-eq by simp
have j-bound-f: j < length f-fl
  using layer-bound trace-roots-eq by simp
have raw-layer-f:
   $\bigwedge raw.$ 

```


$0 < \text{fri-layer-lengths } (\text{length } f\text{-fl}) (\text{clength} * \text{scale}) ! j \wedge$
 $\text{fri-layer-indices } (\text{length } f\text{-fl}) (\text{index } (\text{to-nat } \text{raw}))$
 $(\text{clength} * \text{scale}) ! j <$
 $\text{fri-layer-lengths } (\text{length } f\text{-fl}) (\text{clength} * \text{scale}) ! j$
using $\text{raw-layer trace-roots-eq}$ **by** simp
have query-chunk-f :
 $\wedge i. i < \text{rounds} \implies$
 $\text{verifier-query-round-chunk } (\text{query-idxs} ! i)$
 $(\text{map } \text{snd } f\text{-fl}) (\text{map } \text{snd } \text{fl}) (\text{query-chunks} ! i)$
using $\text{query-chunk trace-roots-eq composition-roots-eq}$ **by** simp
from $\text{ntimes-verifier-query-rounds-selected-trace-fri-chunk-aligned-with-replay-at}$
 $[\text{OF } \text{outcome}' \text{ round-bound } j\text{-bound-}f \text{ raw-layer-}f \text{ len-raw } \text{query-idxs-eq}$
 $\text{len-query-chunks } \text{transcript-query}[\text{unfolded } \text{final-state-eq}]$
 $\text{query-chunk-}f \text{ replay-lookup}[\text{unfolded } \text{final-state-eq}]]$
obtain layer-chunk **where** auth :
 $\text{fri-layer-chunk-authenticated } (\text{snd } (f\text{-fl} ! j))$
 $(\text{fri-layer-lengths } (\text{length } f\text{-fl}) (\text{clength} * \text{scale}) ! j)$
 $(\text{fri-layer-indices } (\text{length } f\text{-fl}) (\text{query-idxs} ! i))$
 $(\text{clength} * \text{scale}) ! j)$
 $\text{layer-chunk } \text{final-state}$
by blast
have root-eq : $\text{trace-roots} ! j = \text{snd } (f\text{-fl} ! j)$
using $\text{trace-roots-eq layer-bound}$ **by** simp
show $?thesis$
by $(\text{rule that}[\text{of } \text{layer-chunk}])$
 $(\text{use } \text{auth } \text{trace-roots-eq } \text{root-eq in } \text{simp})$
qed

lemma $\text{accepted-fri-opening-transcript-composition-selected-chunk-authenticated-at}$:

assumes fri-openings :

$\text{accepted-fri-opening-transcript } s \text{ out } \text{trace-roots } \text{trace-bs } \text{trace-final}$
 $\text{dg } \text{composition-roots } \text{composition-bs } \text{composition-final } \text{query-idxs}$
 $\text{trace-round-layers } \text{composition-round-layers}$

and out-eq : $\text{out} = \text{Some } (\text{result}, \text{final-state})$

and round-bound : $i < \text{rounds}$

and layer-bound : $j < \text{length } \text{composition-roots}$

and raw-layer :

$\wedge \text{raw.}$

$0 < \text{fri-layer-lengths } (\text{length } \text{composition-roots}) (\text{clength} * \text{scale}) ! j \wedge$
 $\text{fri-layer-indices } (\text{length } \text{composition-roots}) (\text{index } (\text{to-nat } \text{raw}))$
 $(\text{clength} * \text{scale}) ! j <$
 $\text{fri-layer-lengths } (\text{length } \text{composition-roots}) (\text{clength} * \text{scale}) ! j$

obtains layer-chunk **where**

$\text{fri-layer-chunk-authenticated } (\text{composition-roots} ! j)$
 $(\text{fri-layer-lengths } (\text{length } \text{composition-roots}) (\text{clength} * \text{scale}) ! j)$
 $(\text{fri-layer-indices } (\text{length } \text{composition-roots}) (\text{query-idxs} ! i))$
 $(\text{clength} * \text{scale}) ! j)$
 $\text{layer-chunk } \text{final-state}$

proof —

```

from fri-openings obtain result' final-state' fr f-fl as fl query-state
  raw-idxs query-chunks where
    out-def: out = Some (result', final-state')
    and trace-roots-eq: trace-roots = map snd f-fl
    and composition-roots-eq: composition-roots = map snd fl
    and outcome:
      Some (result', final-state') ∈
        set-dist (execute
          (ntimes
            (verifier-query-round-program fr f-fl trace-final as fl
              composition-final)
            rounds)
          query-state)
    and len-raw: length raw-idxs = rounds
    and query-idxs-eq:
      query-idxs = map (λraw. index (to-nat raw)) raw-idxs
    and len-query-chunks: length query-chunks = rounds
    and transcript-query:
      PTranscript query-state =
        List.concat query-chunks @ PTranscript final-state'
    and query-chunk:
       $\bigwedge i. i < rounds \implies$ 
        verifier-query-round-chunk (query-idxs ! i)
        trace-roots composition-roots (query-chunks ! i)
    and replay-lookup:
       $\bigwedge i. i < rounds \implies$ 
        fmlookup (HashMap final-state')
        (QueryIndexChallenge (PQueryCounter query-state + i)
          (state-after-query-chunks (PState query-state) query-chunks i)) =
          Some (raw-idxs ! i)
    unfolding accepted-fri-opening-transcript-def by blast
have result-eq: result' = result
  using out-def out-eq by simp
have final-state-eq: final-state' = final-state
  using out-def out-eq by simp
have outcome':
  Some (result, final-state) ∈
    set-dist (execute
      (ntimes
        (verifier-query-round-program fr f-fl trace-final as fl
          composition-final)
        rounds)
      query-state)
  using outcome result-eq final-state-eq by simp
have j-bound-f: j < length fl
  using layer-bound composition-roots-eq by simp
have raw-layer-f:
   $\bigwedge raw.$ 
     $0 < fri-layer-lengths (length fl) (clength * scale) ! j \wedge$ 

```

```

    fri-layer-indices (length fl) (index (to-nat raw))
    (clength * scale) ! j <
    fri-layer-lengths (length fl) (clength * scale) ! j
    using raw-layer composition-roots-eq by simp
  have query-chunk-f:
     $\wedge i. i < rounds \implies$ 
    verifier-query-round-chunk (query-idxs ! i)
    (map snd f-fl) (map snd fl) (query-chunks ! i)
    using query-chunk trace-roots-eq composition-roots-eq by simp
  from ntimes-verifier-query-rounds-selected-composition-fri-chunk-aligned-with-replay-at
    [OF outcome' round-bound j-bound-f raw-layer-f len-raw query-idxs-eq
    len-query-chunks transcript-query[unfolded final-state-eq]
    query-chunk-f replay-lookup[unfolded final-state-eq]]
  obtain layer-chunk where auth:
    fri-layer-chunk-authenticated (snd (fl ! j))
    (fri-layer-lengths (length fl) (clength * scale) ! j)
    (fri-layer-indices (length fl) (query-idxs ! i)
    (clength * scale) ! j)
    layer-chunk final-state
  by blast
  have root-eq: composition-roots ! j = snd (fl ! j)
    using composition-roots-eq layer-bound by simp
  show ?thesis
    by (rule that[of layer-chunk])
    (use auth composition-roots-eq root-eq in simp)
qed

end

end

```

```

theory Soundness-FRI-Layer-Authenticated
  imports Soundness-FRI-Layer-Merkle
begin

```

Authenticated FRI layer evidence paired with transcript layer evidence.

The exact equality between the layer chunk recorded in *accepted-fri-opening-transcript* and the layer chunk authenticated by the verifier is still intentionally not asserted here. Instead this theory packages the checked facts that are currently available without a broad replay induction: the transcript gives a local FRI step, and the verifier execution gives an authenticated FRI chunk at the same root, layer length, and replayed query index.

```

context soundness
begin

```

```

definition generic-fri-transcript-step-with-authenticated-chunk
  :: 'f list  $\Rightarrow$  'f list  $\Rightarrow$  nat list  $\Rightarrow$ 

```

*'f list list list $\Rightarrow$ ('f, 'a) protocol-channel-scheme $\Rightarrow$
nat $\Rightarrow$ nat $\Rightarrow$ bool*

where

*generic-fri-transcript-step-with-authenticated-chunk roots challenges
query-idxs round-layers final-state round-idx layer-idx $\longleftrightarrow$
fri-round-layer-evidence roots challenges query-idxs round-idx
layer-idx round-layers $\wedge$
($\exists$ auth-chunk.
fri-layer-chunk-authenticated (roots ! layer-idx)
(fri-evidence-layer-len roots layer-idx)
(fri-evidence-layer-idx roots query-idxs round-idx layer-idx)
auth-chunk final-state)*

lemma *generic-fri-transcript-step-with-authenticated-chunkI:*

assumes *step:*

*fri-round-layer-evidence roots challenges query-idxs round-idx
layer-idx round-layers*

and *auth:*

*fri-layer-chunk-authenticated (roots ! layer-idx)
(fri-evidence-layer-len roots layer-idx)
(fri-evidence-layer-idx roots query-idxs round-idx layer-idx)
auth-chunk final-state*

shows

*generic-fri-transcript-step-with-authenticated-chunk roots challenges
query-idxs round-layers final-state round-idx layer-idx*

using *assms*

unfolding *generic-fri-transcript-step-with-authenticated-chunk-def*

by *blast*

lemma *generic-fri-transcript-step-with-authenticated-chunkE:*

assumes

*generic-fri-transcript-step-with-authenticated-chunk roots challenges
query-idxs round-layers final-state round-idx layer-idx*

obtains *auth-chunk where*

*fri-round-layer-evidence roots challenges query-idxs round-idx
layer-idx round-layers
fri-layer-chunk-authenticated (roots ! layer-idx)
(fri-evidence-layer-len roots layer-idx)
(fri-evidence-layer-idx roots query-idxs round-idx layer-idx)
auth-chunk final-state*

using *assms*

unfolding *generic-fri-transcript-step-with-authenticated-chunk-def*

by *blast*

definition *generic-fri-authenticated-same-layer-opening-conflict*

:: 'f list $\Rightarrow$ nat list $\Rightarrow$

('f, 'a) protocol-channel-scheme $\Rightarrow$ bool

where

generic-fri-authenticated-same-layer-opening-conflict roots query-idxs

$final-state \longleftrightarrow$
 $(\exists round-idx\ round-idx'\ layer-idx\ opn\ opn'. \\
round-idx < length\ query-idxs \wedge \\
round-idx' < length\ query-idxs \wedge \\
layer-idx < length\ roots \wedge \\
authenticated-opening-in\ final-state\ opn \wedge \\
authenticated-opening-in\ final-state\ opn' \wedge \\
opening-root\ opn = roots\ !\ layer-idx \wedge \\
opening-root\ opn' = roots\ !\ layer-idx \wedge \\
opening-length\ opn = fri-evidence-layer-len\ roots\ layer-idx \wedge \\
opening-length\ opn' = fri-evidence-layer-len\ roots\ layer-idx \wedge \\
(opening-index\ opn = \\
\quad fri-evidence-layer-idx\ roots\ query-idxs\ round-idx\ layer-idx \vee \\
opening-index\ opn = \\
\quad fri-sibling-index\ (fri-evidence-layer-len\ roots\ layer-idx) \\
\quad (fri-evidence-layer-idx\ roots\ query-idxs\ round-idx\ layer-idx)) \wedge \\
(opening-index\ opn' = \\
\quad fri-evidence-layer-idx\ roots\ query-idxs\ round-idx'\ layer-idx \vee \\
opening-index\ opn' = \\
\quad fri-sibling-index\ (fri-evidence-layer-len\ roots\ layer-idx) \\
\quad (fri-evidence-layer-idx\ roots\ query-idxs\ round-idx'\ layer-idx)) \wedge \\
opening-index\ opn = opening-index\ opn' \wedge \\
opening-value\ opn \neq opening-value\ opn')$

lemma *generic-fri-authenticated-same-layer-opening-conflictI:*

assumes *round-bound:* $round-idx < length\ query-idxs$
and *round-bound':* $round-idx' < length\ query-idxs$
and *layer-bound:* $layer-idx < length\ roots$
and *auth:* $authenticated-opening-in\ final-state\ opn$
 $authenticated-opening-in\ final-state\ opn'$
and *root:* $opening-root\ opn = roots\ !\ layer-idx$
 $opening-root\ opn' = roots\ !\ layer-idx$
and *len:*
 $opening-length\ opn = fri-evidence-layer-len\ roots\ layer-idx$
 $opening-length\ opn' = fri-evidence-layer-len\ roots\ layer-idx$
and *idx:*
 $opening-index\ opn =$
 $\quad fri-evidence-layer-idx\ roots\ query-idxs\ round-idx\ layer-idx \vee$
 $opening-index\ opn =$
 $\quad fri-sibling-index\ (fri-evidence-layer-len\ roots\ layer-idx)$
 $\quad (fri-evidence-layer-idx\ roots\ query-idxs\ round-idx\ layer-idx)$
 $opening-index\ opn' =$
 $\quad fri-evidence-layer-idx\ roots\ query-idxs\ round-idx'\ layer-idx \vee$
 $opening-index\ opn' =$
 $\quad fri-sibling-index\ (fri-evidence-layer-len\ roots\ layer-idx)$
 $\quad (fri-evidence-layer-idx\ roots\ query-idxs\ round-idx'\ layer-idx)$
and *same-index:* $opening-index\ opn = opening-index\ opn'$
and *neg:* $opening-value\ opn \neq opening-value\ opn'$
shows

```

    generic-fri-authenticated-same-layer-opening-conflict roots query-idxs
    final-state
unfolding generic-fri-authenticated-same-layer-opening-conflict-def
by (intro exI[of - round-idx] exI[of - round-idx']
    exI[of - layer-idx] exI[of - opn] exI[of - opn'] conjI)
    (use assms in auto)

lemma generic-fri-authenticated-same-layer-opening-conflict-imp-partial-merkle:
  assumes
    generic-fri-authenticated-same-layer-opening-conflict roots query-idxs
    final-state
  shows partial-merkle-inconsistency-bad s (Some (result, final-state))
proof -
from assms obtain round-idx round-idx' layer-idx opn opn' where
  auth: authenticated-opening-in final-state opn
    authenticated-opening-in final-state opn'
and roots:
  opening-root opn = roots ! layer-idx
  opening-root opn' = roots ! layer-idx
and lengths:
  opening-length opn = fri-evidence-layer-len roots layer-idx
  opening-length opn' = fri-evidence-layer-len roots layer-idx
and same-index: opening-index opn = opening-index opn'
and neq: opening-value opn  $\neq$  opening-value opn'
unfolding generic-fri-authenticated-same-layer-opening-conflict-def
by blast
have same-root: opening-root opn = opening-root opn'
  using roots by simp
have same-length: opening-length opn = opening-length opn'
  using lengths by simp
show ?thesis
  unfolding partial-merkle-inconsistency-bad-def accepted-def
  by (intro conjI exI[of - result] exI[of - final-state]
    exI[of - opn] exI[of - opn'])
    (use auth same-root same-length same-index neq in simp-all)
qed

lemma generic-fri-authenticated-same-layer-opening-conflictI-base-base:
  assumes round-bound: round-idx < length query-idxs
  and round-bound': round-idx' < length query-idxs
  and layer-bound: layer-idx < length roots
  and auth:
    fri-layer-chunk-authenticated (roots ! layer-idx)
    (fri-evidence-layer-len roots layer-idx)
    (fri-evidence-layer-idx roots query-idxs round-idx layer-idx)
    chunk final-state
  and auth':
    fri-layer-chunk-authenticated (roots ! layer-idx)
    (fri-evidence-layer-len roots layer-idx)

```

```

    (fri-evidence-layer-idx roots query-idxs round-idx' layer-idx)
    chunk' final-state
and chunk:
    fri-layer-opening-chunk (fri-evidence-layer-len roots layer-idx)
    xp xp-path xn xn-path chunk
and chunk':
    fri-layer-opening-chunk (fri-evidence-layer-len roots layer-idx)
    yp yp-path yn yn-path chunk'
and same-index:
    fri-evidence-layer-idx roots query-idxs round-idx layer-idx =
    fri-evidence-layer-idx roots query-idxs round-idx' layer-idx
and neg: xp ≠ yp
shows
    generic-fri-authenticated-same-layer-opening-conflict roots query-idxs
    final-state
proof –
from fri-layer-chunk-authenticated-matching-values[OF auth chunk]
obtain xp-path' xn-path' where auth-xp:
    authenticated-opening-in final-state
    (opening-root = roots ! layer-idx,
     opening-length = fri-evidence-layer-len roots layer-idx,
     opening-index = fri-evidence-layer-idx roots query-idxs round-idx
     layer-idx,
     opening-value = xp,
     opening-path = xp-path')
by blast
from fri-layer-chunk-authenticated-matching-values[OF auth' chunk']
obtain yp-path' yn-path' where auth-yp:
    authenticated-opening-in final-state
    (opening-root = roots ! layer-idx,
     opening-length = fri-evidence-layer-len roots layer-idx,
     opening-index = fri-evidence-layer-idx roots query-idxs round-idx'
     layer-idx,
     opening-value = yp,
     opening-path = yp-path')
by blast
let ?opn =
    (opening-root = roots ! layer-idx,
     opening-length = fri-evidence-layer-len roots layer-idx,
     opening-index = fri-evidence-layer-idx roots query-idxs round-idx
     layer-idx,
     opening-value = xp,
     opening-path = xp-path')
let ?opn' =
    (opening-root = roots ! layer-idx,
     opening-length = fri-evidence-layer-len roots layer-idx,
     opening-index = fri-evidence-layer-idx roots query-idxs round-idx'
     layer-idx,
     opening-value = yp,

```

```

    opening-path = yp-path')
show ?thesis
by (rule generic-fri-authenticated-same-layer-opening-conflictI
    [OF round-bound round-bound' layer-bound auth-xp auth-yp])
    (use same-index neq in simp-all)
qed

lemma generic-fri-authenticated-same-layer-opening-conflictI-base-sibling:
assumes round-bound: round-idx < length query-idxs
and round-bound': round-idx' < length query-idxs
and layer-bound: layer-idx < length roots
and auth:
    fri-layer-chunk-authenticated (roots ! layer-idx)
    (fri-evidence-layer-len roots layer-idx)
    (fri-evidence-layer-idx roots query-idxs round-idx layer-idx)
    chunk final-state
and auth':
    fri-layer-chunk-authenticated (roots ! layer-idx)
    (fri-evidence-layer-len roots layer-idx)
    (fri-evidence-layer-idx roots query-idxs round-idx' layer-idx)
    chunk' final-state
and chunk:
    fri-layer-opening-chunk (fri-evidence-layer-len roots layer-idx)
    xp xp-path xn xn-path chunk
and chunk':
    fri-layer-opening-chunk (fri-evidence-layer-len roots layer-idx)
    yp yp-path yn yn-path chunk'
and same-index:
    fri-evidence-layer-idx roots query-idxs round-idx layer-idx =
    fri-sibling-index (fri-evidence-layer-len roots layer-idx)
    (fri-evidence-layer-idx roots query-idxs round-idx' layer-idx)
and neq: xp ≠ yn
shows
    generic-fri-authenticated-same-layer-opening-conflict roots query-idxs
    final-state
proof -
from fri-layer-chunk-authenticated-matching-values[OF auth chunk]
obtain xp-path' xn-path' where auth-xp:
    authenticated-opening-in final-state
    (|opening-root = roots ! layer-idx,
    opening-length = fri-evidence-layer-len roots layer-idx,
    opening-index = fri-evidence-layer-idx roots query-idxs round-idx
    layer-idx,
    opening-value = xp,
    opening-path = xp-path'|)
by blast
from fri-layer-chunk-authenticated-matching-values[OF auth' chunk']
obtain yp-path' yn-path' where auth-yn:
    authenticated-opening-in final-state

```



```

    (opening-root = roots ! layer-idx,
     opening-length = fri-evidence-layer-len roots layer-idx,
     opening-index =
       fri-sibling-index (fri-evidence-layer-len roots layer-idx)
       (fri-evidence-layer-idx roots query-idxs round-idx' layer-idx),
     opening-value = yn,
     opening-path = yn-path')
  by blast
show ?thesis
  by (rule generic-fri-authenticated-same-layer-opening-conflictI
      [OF round-bound round-bound' layer-bound auth-xp auth-yn])
      (use same-index neq in simp-all)
qed

lemma generic-fri-authenticated-same-layer-opening-conflictI-sibling-base:
  assumes round-bound: round-idx < length query-idxs
  and round-bound': round-idx' < length query-idxs
  and layer-bound: layer-idx < length roots
  and auth:
    fri-layer-chunk-authenticated (roots ! layer-idx)
    (fri-evidence-layer-len roots layer-idx)
    (fri-evidence-layer-idx roots query-idxs round-idx layer-idx)
    chunk final-state
  and auth':
    fri-layer-chunk-authenticated (roots ! layer-idx)
    (fri-evidence-layer-len roots layer-idx)
    (fri-evidence-layer-idx roots query-idxs round-idx' layer-idx)
    chunk' final-state
  and chunk:
    fri-layer-opening-chunk (fri-evidence-layer-len roots layer-idx)
    xp xp-path xn xn-path chunk
  and chunk':
    fri-layer-opening-chunk (fri-evidence-layer-len roots layer-idx)
    yp yp-path yn yn-path chunk'
  and same-index:
    fri-sibling-index (fri-evidence-layer-len roots layer-idx)
    (fri-evidence-layer-idx roots query-idxs round-idx layer-idx) =
    fri-evidence-layer-idx roots query-idxs round-idx' layer-idx
  and neq: xn ≠ yp
  shows
    generic-fri-authenticated-same-layer-opening-conflict roots query-idxs
    final-state
proof -
  from fri-layer-chunk-authenticated-matching-values[OF auth chunk]
  obtain xp-path' xn-path' where auth-xn:
    authenticated-opening-in final-state
    (opening-root = roots ! layer-idx,
     opening-length = fri-evidence-layer-len roots layer-idx,
     opening-index =

```

```

    fri-sibling-index (fri-evidence-layer-len roots layer-idx)
    (fri-evidence-layer-idx roots query-idxs round-idx layer-idx),
    opening-value = xn,
    opening-path = xn-path')
  by blast
from fri-layer-chunk-authenticated-matching-values[OF auth' chunk']
obtain yp-path' yn-path' where auth-yp:
  authenticated-opening-in final-state
  (|opening-root = roots ! layer-idx,
    opening-length = fri-evidence-layer-len roots layer-idx,
    opening-index = fri-evidence-layer-idx roots query-idxs round-idx'
    layer-idx,
    opening-value = yp,
    opening-path = yp-path'|)
  by blast
show ?thesis
  by (rule generic-fri-authenticated-same-layer-opening-conflictI
    [OF round-bound round-bound' layer-bound auth-xn auth-yp])
    (use same-index neq in simp-all)
qed

lemma generic-fri-authenticated-same-layer-opening-conflictI-sibling-sibling:
  assumes round-bound: round-idx < length query-idxs
  and round-bound': round-idx' < length query-idxs
  and layer-bound: layer-idx < length roots
  and auth:
    fri-layer-chunk-authenticated (roots ! layer-idx)
    (fri-evidence-layer-len roots layer-idx)
    (fri-evidence-layer-idx roots query-idxs round-idx layer-idx)
    chunk final-state
  and auth':
    fri-layer-chunk-authenticated (roots ! layer-idx)
    (fri-evidence-layer-len roots layer-idx)
    (fri-evidence-layer-idx roots query-idxs round-idx' layer-idx)
    chunk' final-state
  and chunk:
    fri-layer-opening-chunk (fri-evidence-layer-len roots layer-idx)
    xp xp-path xn xn-path chunk
  and chunk':
    fri-layer-opening-chunk (fri-evidence-layer-len roots layer-idx)
    yp yp-path yn yn-path chunk'
  and same-index:
    fri-sibling-index (fri-evidence-layer-len roots layer-idx)
    (fri-evidence-layer-idx roots query-idxs round-idx layer-idx) =
    fri-sibling-index (fri-evidence-layer-len roots layer-idx)
    (fri-evidence-layer-idx roots query-idxs round-idx' layer-idx)
  and neq: xn ≠ yn
shows
  generic-fri-authenticated-same-layer-opening-conflict roots query-idxs

```

final-state
proof –
from *fri-layer-chunk-authenticated-matching-values*[*OF auth chunk*]
obtain *xp-path' xn-path'* **where** *auth-xn*:
authenticated-opening-in final-state
(*opening-root* = *roots ! layer-idx*,
opening-length = *fri-evidence-layer-len roots layer-idx*,
opening-index =
fri-sibling-index (fri-evidence-layer-len roots layer-idx)
(*fri-evidence-layer-idx roots query-idxs round-idx layer-idx*),
opening-value = *xn*,
opening-path = *xn-path'*)
by *blast*
from *fri-layer-chunk-authenticated-matching-values*[*OF auth' chunk'*]
obtain *yp-path' yn-path'* **where** *auth-yn*:
authenticated-opening-in final-state
(*opening-root* = *roots ! layer-idx*,
opening-length = *fri-evidence-layer-len roots layer-idx*,
opening-index =
fri-sibling-index (fri-evidence-layer-len roots layer-idx)
(*fri-evidence-layer-idx roots query-idxs round-idx' layer-idx*),
opening-value = *yn*,
opening-path = *yn-path'*)
by *blast*
show *?thesis*
by (*rule generic-fri-authenticated-same-layer-opening-conflictI*
[*OF round-bound round-bound' layer-bound auth-xn auth-yn*])
(*use same-index neq in simp-all*)
qed

definition *generic-fri-recorded-layer-chunk-authenticated*
:: '*f list* ⇒ *nat list* ⇒ '*f list list list* ⇒
(*f*, '*a*) *protocol-channel-scheme* ⇒ *nat* ⇒ *nat* ⇒ *bool*
where
generic-fri-recorded-layer-chunk-authenticated roots query-idxs
round-layers final-state round-idx layer-idx $\longleftrightarrow$
fri-layer-chunk-authenticated (roots ! layer-idx)
(*fri-evidence-layer-len roots layer-idx*)
(*fri-evidence-layer-idx roots query-idxs round-idx layer-idx*)
(*round-layers ! round-idx ! layer-idx*) *final-state*

definition *generic-fri-authenticated-slot-recorded-chunk-gap*
:: '*f list* ⇒ *nat list* ⇒ '*f list list list* ⇒
(*f*, '*a*) *protocol-channel-scheme* ⇒ *bool*
where
generic-fri-authenticated-slot-recorded-chunk-gap roots query-idxs
round-layers final-state $\longleftrightarrow$
($\exists$ *round-idx layer-idx auth-chunk*.
round-idx < *length query-idxs* ∧

$layer_idx < length\ roots \wedge$
 $fri_layer_chunk_authenticated\ (roots\ !\ layer_idx)$
 $(fri_evidence_layer_len\ roots\ layer_idx)$
 $(fri_evidence_layer_idx\ roots\ query_idxs\ round_idx\ layer_idx)$
 $auth_chunk\ final_state \wedge$
 $\neg\ generic_fri_recorded_layer_chunk_authenticated\ roots\ query_idxs$
 $round_layers\ final_state\ round_idx\ layer_idx)$

lemma *generic-fri-transcript-step-auth-missing-imp-authenticated-slot-recorded-chunk-gap*:

assumes *step*:

generic-fri-transcript-step-with-authenticated-chunk roots challenges
query_idxs round_layers final_state round_idx layer_idx

and *missing*:

$\neg\ generic_fri_recorded_layer_chunk_authenticated\ roots\ query_idxs$
 $round_layers\ final_state\ round_idx\ layer_idx$

shows

generic-fri-authenticated-slot-recorded-chunk-gap roots query_idxs
round_layers final_state

proof –

from *step* **obtain** *auth-chunk* **where** *evidence*:

fri-round-layer-evidence roots challenges query_idxs round_idx layer_idx
round_layers

and *auth*:

$fri_layer_chunk_authenticated\ (roots\ !\ layer_idx)$
 $(fri_evidence_layer_len\ roots\ layer_idx)$
 $(fri_evidence_layer_idx\ roots\ query_idxs\ round_idx\ layer_idx)$
 $auth_chunk\ final_state$

by (rule *generic-fri-transcript-step-with-authenticated-chunkE*)

from *evidence* **have** *round-bound*: $round_idx < length\ query_idxs$

and *layer-bound*: $layer_idx < length\ roots$

by (auto elim: *fri-round-layer-evidenceE*)

show *?thesis*

unfolding *generic-fri-authenticated-slot-recorded-chunk-gap-def*

by (intro *exI conjI*)

(rule *round-bound*, rule *layer-bound*, rule *auth*, rule *missing*)

qed

definition *generic-fri-sampled-same-layer-recorded-auth-gap*

$:: 'f\ list \Rightarrow 'f\ list \Rightarrow nat\ list \Rightarrow$

$'f\ list\ list\ list \Rightarrow ('f, 'a)\ protocol_channel_scheme \Rightarrow bool$

where

generic-fri-sampled-same-layer-recorded-auth-gap roots challenges
query_idxs round_layers final_state $\longleftrightarrow$

$(\exists\ round_idx\ round_idx'\ layer_idx.$

$round_idx < length\ query_idxs \wedge$

$round_idx' < length\ query_idxs \wedge$

$layer_idx < length\ challenges \wedge$

generic-fri-sampled-same-layer-opening-conflict roots challenges

$query_idxs\ round_layers \wedge$

$(\neg \text{generic-fri-recorded-layer-chunk-authenticated roots query-idxs}$
 $\text{round-layers final-state round-idx layer-idx} \vee$
 $\neg \text{generic-fri-recorded-layer-chunk-authenticated roots query-idxs}$
 $\text{round-layers final-state round-idx' layer-idx}))$

definition *generic-fri-sampled-same-layer-authenticated-recorded-auth-gap*

$:: 'f \text{ list} \Rightarrow 'f \text{ list} \Rightarrow \text{nat list} \Rightarrow$
 $'f \text{ list list list} \Rightarrow ('f, 'a) \text{ protocol-channel-scheme} \Rightarrow \text{bool}$

where

generic-fri-sampled-same-layer-authenticated-recorded-auth-gap roots
 $\text{challenges query-idxs round-layers final-state} \longleftrightarrow$
 $(\exists \text{round-idx round-idx' layer-idx.}$
 $\text{round-idx} < \text{length query-idxs} \wedge$
 $\text{round-idx'} < \text{length query-idxs} \wedge$
 $\text{layer-idx} < \text{length challenges} \wedge$
 $\text{generic-fri-sampled-same-layer-opening-conflict roots challenges}$
 $\text{query-idxs round-layers} \wedge$
 $((\text{generic-fri-transcript-step-with-authenticated-chunk roots challenges}$
 $\text{query-idxs round-layers final-state round-idx layer-idx} \wedge$
 $\neg \text{generic-fri-recorded-layer-chunk-authenticated roots query-idxs}$
 $\text{round-layers final-state round-idx layer-idx}) \vee$
 $(\text{generic-fri-transcript-step-with-authenticated-chunk roots challenges}$
 $\text{query-idxs round-layers final-state round-idx' layer-idx} \wedge$
 $\neg \text{generic-fri-recorded-layer-chunk-authenticated roots query-idxs}$
 $\text{round-layers final-state round-idx' layer-idx})))$

lemma *generic-fri-authenticated-recorded-auth-gap-imp-slot-recorded-chunk-gap:*

assumes

generic-fri-sampled-same-layer-authenticated-recorded-auth-gap roots
 $\text{challenges query-idxs round-layers final-state}$

shows

generic-fri-authenticated-slot-recorded-chunk-gap roots query-idxs
 $\text{round-layers final-state}$

proof —

from *assms* **obtain** $\text{round-idx round-idx' layer-idx}$ **where**

gap:

$(\text{generic-fri-transcript-step-with-authenticated-chunk roots challenges}$
 $\text{query-idxs round-layers final-state round-idx layer-idx} \wedge$
 $\neg \text{generic-fri-recorded-layer-chunk-authenticated roots query-idxs}$
 $\text{round-layers final-state round-idx layer-idx}) \vee$
 $(\text{generic-fri-transcript-step-with-authenticated-chunk roots challenges}$
 $\text{query-idxs round-layers final-state round-idx' layer-idx} \wedge$
 $\neg \text{generic-fri-recorded-layer-chunk-authenticated roots query-idxs}$
 $\text{round-layers final-state round-idx' layer-idx})$

unfolding *generic-fri-sampled-same-layer-authenticated-recorded-auth-gap-def*

by *blast*

then show *?thesis*

proof

assume *left:*

```

generic-fri-transcript-step-with-authenticated-chunk roots challenges
  query-idxs round-layers final-state round-idx layer-idx ∧
  ¬ generic-fri-recorded-layer-chunk-authenticated roots query-idxs
    round-layers final-state round-idx layer-idx
show ?thesis
by (rule
  generic-fri-transcript-step-auth-missing-imp-authenticated-slot-recorded-chunk-gap
  [of roots challenges query-idxs round-layers final-state round-idx
    layer-idx])
  (use left in simp-all)
next
assume right:
  generic-fri-transcript-step-with-authenticated-chunk roots challenges
    query-idxs round-layers final-state round-idx' layer-idx ∧
  ¬ generic-fri-recorded-layer-chunk-authenticated roots query-idxs
    round-layers final-state round-idx' layer-idx
show ?thesis
by (rule
  generic-fri-transcript-step-auth-missing-imp-authenticated-slot-recorded-chunk-gap
  [of roots challenges query-idxs round-layers final-state round-idx'
    layer-idx])
  (use right in simp-all)
qed
qed

lemma generic-fri-recorded-auth-gap-imp-authenticated-recorded-auth-gap:
assumes gap:
  generic-fri-sampled-same-layer-recorded-auth-gap roots challenges
    query-idxs round-layers final-state
and auth-step:
  ∧ round-idx layer-idx.
  round-idx < length query-idxs ⇒
  layer-idx < length challenges ⇒
  generic-fri-transcript-step-with-authenticated-chunk roots
    challenges query-idxs round-layers final-state round-idx layer-idx
shows
  generic-fri-sampled-same-layer-authenticated-recorded-auth-gap roots
    challenges query-idxs round-layers final-state
proof –
from gap obtain round-idx round-idx' layer-idx where
  round-bound: round-idx < length query-idxs
and round-bound': round-idx' < length query-idxs
and layer-bound: layer-idx < length challenges
and conflict:
  generic-fri-sampled-same-layer-opening-conflict roots challenges
    query-idxs round-layers
and missing:
  ¬ generic-fri-recorded-layer-chunk-authenticated roots query-idxs
    round-layers final-state round-idx layer-idx ∨

```

```

    ¬ generic-fri-recorded-layer-chunk-authenticated roots query-idxs
      round-layers final-state round-idx' layer-idx
  unfolding generic-fri-sampled-same-layer-recorded-auth-gap-def
  by blast
show ?thesis
proof (cases
  ¬ generic-fri-recorded-layer-chunk-authenticated roots query-idxs
    round-layers final-state round-idx layer-idx)
case True
have step:
  generic-fri-transcript-step-with-authenticated-chunk roots challenges
    query-idxs round-layers final-state round-idx layer-idx
  by (rule auth-step[OF round-bound layer-bound])
show ?thesis
  unfolding
    generic-fri-sampled-same-layer-authenticated-recorded-auth-gap-def
  by (intro exI conjI disjI1)
    (rule round-bound, rule round-bound', rule layer-bound,
      rule conflict, rule step, rule True)
next
case False
then have missing':
  ¬ generic-fri-recorded-layer-chunk-authenticated roots query-idxs
    round-layers final-state round-idx' layer-idx
  using missing by simp
have step:
  generic-fri-transcript-step-with-authenticated-chunk roots challenges
    query-idxs round-layers final-state round-idx' layer-idx
  by (rule auth-step[OF round-bound' layer-bound])
show ?thesis
  unfolding
    generic-fri-sampled-same-layer-authenticated-recorded-auth-gap-def
  by (intro exI conjI disjI2)
    (rule round-bound, rule round-bound', rule layer-bound,
      rule conflict, rule step, rule missing')
qed
qed

definition generic-fri-sampled-same-layer-opening-conflict-with-authenticated-chunks
  :: 'f list ⇒ 'f list ⇒ nat list ⇒
    'f list list list ⇒ ('f, 'a) protocol-channel-scheme ⇒ bool
where
  generic-fri-sampled-same-layer-opening-conflict-with-authenticated-chunks
    roots challenges query-idxs round-layers final-state ⟷
  (∃ round-idx round-idx' layer-idx
    xp xp-path xn xn-path yp yp-path yn yn-path.
    round-idx < length query-idxs ∧
    round-idx' < length query-idxs ∧
    layer-idx < length challenges ∧

```

$fri\text{-}layer\text{-}step\text{-}evidence$
 $(roots \ ! \ layer\text{-}idx)$
 $(challenges \ ! \ layer\text{-}idx)$
 $(fri\text{-}evidence\text{-}layer\text{-}len \ roots \ layer\text{-}idx)$
 $(fri\text{-}evidence\text{-}layer\text{-}idx \ roots \ query\text{-}idxs \ round\text{-}idx \ layer\text{-}idx)$
 $(2 \wedge layer\text{-}idx)$
 $(fri\text{-}sibling\text{-}index \ (fri\text{-}evidence\text{-}layer\text{-}len \ roots \ layer\text{-}idx)$
 $\quad (fri\text{-}evidence\text{-}layer\text{-}idx \ roots \ query\text{-}idxs \ round\text{-}idx \ layer\text{-}idx))$
 $xp \ xp\text{-}path \ xn \ xn\text{-}path$
 $(fri\text{-}evidence\text{-}next\text{-}idx \ roots \ query\text{-}idxs \ round\text{-}idx \ layer\text{-}idx)$
 $(fri\text{-}evidence\text{-}next\text{-}value \ roots \ challenges \ query\text{-}idxs \ round\text{-}idx$
 $\quad layer\text{-}idx \ xp \ xn)$
 $(round\text{-}layers \ ! \ round\text{-}idx \ ! \ layer\text{-}idx) \wedge$
 $fri\text{-}layer\text{-}step\text{-}evidence$
 $(roots \ ! \ layer\text{-}idx)$
 $(challenges \ ! \ layer\text{-}idx)$
 $(fri\text{-}evidence\text{-}layer\text{-}len \ roots \ layer\text{-}idx)$
 $(fri\text{-}evidence\text{-}layer\text{-}idx \ roots \ query\text{-}idxs \ round\text{-}idx' \ layer\text{-}idx)$
 $(2 \wedge layer\text{-}idx)$
 $(fri\text{-}sibling\text{-}index \ (fri\text{-}evidence\text{-}layer\text{-}len \ roots \ layer\text{-}idx)$
 $\quad (fri\text{-}evidence\text{-}layer\text{-}idx \ roots \ query\text{-}idxs \ round\text{-}idx' \ layer\text{-}idx))$
 $yp \ yp\text{-}path \ yn \ yn\text{-}path$
 $(fri\text{-}evidence\text{-}next\text{-}idx \ roots \ query\text{-}idxs \ round\text{-}idx' \ layer\text{-}idx)$
 $(fri\text{-}evidence\text{-}next\text{-}value \ roots \ challenges \ query\text{-}idxs \ round\text{-}idx'$
 $\quad layer\text{-}idx \ yp \ yn)$
 $(round\text{-}layers \ ! \ round\text{-}idx' \ ! \ layer\text{-}idx) \wedge$
 $fri\text{-}layer\text{-}chunk\text{-}authenticated \ (roots \ ! \ layer\text{-}idx)$
 $(fri\text{-}evidence\text{-}layer\text{-}len \ roots \ layer\text{-}idx)$
 $(fri\text{-}evidence\text{-}layer\text{-}idx \ roots \ query\text{-}idxs \ round\text{-}idx \ layer\text{-}idx)$
 $(round\text{-}layers \ ! \ round\text{-}idx \ ! \ layer\text{-}idx) \ final\text{-}state \wedge$
 $fri\text{-}layer\text{-}chunk\text{-}authenticated \ (roots \ ! \ layer\text{-}idx)$
 $(fri\text{-}evidence\text{-}layer\text{-}len \ roots \ layer\text{-}idx)$
 $(fri\text{-}evidence\text{-}layer\text{-}idx \ roots \ query\text{-}idxs \ round\text{-}idx' \ layer\text{-}idx)$
 $(round\text{-}layers \ ! \ round\text{-}idx' \ ! \ layer\text{-}idx) \ final\text{-}state \wedge$
 $((fri\text{-}evidence\text{-}layer\text{-}idx \ roots \ query\text{-}idxs \ round\text{-}idx \ layer\text{-}idx =$
 $\quad fri\text{-}evidence\text{-}layer\text{-}idx \ roots \ query\text{-}idxs \ round\text{-}idx' \ layer\text{-}idx \wedge$
 $\quad xp \neq yp) \vee$
 $(fri\text{-}evidence\text{-}layer\text{-}idx \ roots \ query\text{-}idxs \ round\text{-}idx \ layer\text{-}idx =$
 $\quad fri\text{-}sibling\text{-}index \ (fri\text{-}evidence\text{-}layer\text{-}len \ roots \ layer\text{-}idx)$
 $\quad (fri\text{-}evidence\text{-}layer\text{-}idx \ roots \ query\text{-}idxs \ round\text{-}idx' \ layer\text{-}idx) \wedge$
 $\quad xp \neq yn) \vee$
 $(fri\text{-}sibling\text{-}index \ (fri\text{-}evidence\text{-}layer\text{-}len \ roots \ layer\text{-}idx)$
 $\quad (fri\text{-}evidence\text{-}layer\text{-}idx \ roots \ query\text{-}idxs \ round\text{-}idx \ layer\text{-}idx) =$
 $\quad fri\text{-}evidence\text{-}layer\text{-}idx \ roots \ query\text{-}idxs \ round\text{-}idx' \ layer\text{-}idx \wedge$
 $\quad xn \neq yp) \vee$
 $(fri\text{-}sibling\text{-}index \ (fri\text{-}evidence\text{-}layer\text{-}len \ roots \ layer\text{-}idx)$
 $\quad (fri\text{-}evidence\text{-}layer\text{-}idx \ roots \ query\text{-}idxs \ round\text{-}idx \ layer\text{-}idx) =$
 $\quad fri\text{-}sibling\text{-}index \ (fri\text{-}evidence\text{-}layer\text{-}len \ roots \ layer\text{-}idx)$
 $\quad (fri\text{-}evidence\text{-}layer\text{-}idx \ roots \ query\text{-}idxs \ round\text{-}idx' \ layer\text{-}idx) \wedge$

$xn \neq yn)))$

lemma *generic-fri-sampled-same-layer-opening-conflict-with-authenticated-chunks-imp-authenticated:*

assumes *conflict:*

generic-fri-sampled-same-layer-opening-conflict-with-authenticated-chunks

roots challenges query-idxs round-layers final-state

and *len-challenges:* $\text{length } \text{challenges} = \text{length } \text{roots}$

shows

generic-fri-authenticated-same-layer-opening-conflict roots query-idxs

final-state

proof –

from *conflict* **obtain** *round-idx round-idx' layer-idx*

xp xp-path xn xn-path yp yp-path yn yn-path **where**

round-bound: $\text{round-idx} < \text{length } \text{query-idxs}$

and *round-bound':* $\text{round-idx}' < \text{length } \text{query-idxs}$

and *layer-bound-ch:* $\text{layer-idx} < \text{length } \text{challenges}$

and *step:*

fri-layer-step-evidence

$(\text{roots} \ ! \ \text{layer-idx})$

$(\text{challenges} \ ! \ \text{layer-idx})$

$(\text{fri-evidence-layer-len } \text{roots } \text{layer-idx})$

$(\text{fri-evidence-layer-idx } \text{roots } \text{query-idxs } \text{round-idx } \text{layer-idx})$

$(2 \wedge \text{layer-idx})$

$(\text{fri-sibling-index } (\text{fri-evidence-layer-len } \text{roots } \text{layer-idx})$

$(\text{fri-evidence-layer-idx } \text{roots } \text{query-idxs } \text{round-idx } \text{layer-idx}))$

xp xp-path xn xn-path

$(\text{fri-evidence-next-idx } \text{roots } \text{query-idxs } \text{round-idx } \text{layer-idx})$

$(\text{fri-evidence-next-value } \text{roots } \text{challenges } \text{query-idxs } \text{round-idx}$

$\text{layer-idx } \text{xp } \text{xn})$

$(\text{round-layers} \ ! \ \text{round-idx} \ ! \ \text{layer-idx})$

and *step':*

fri-layer-step-evidence

$(\text{roots} \ ! \ \text{layer-idx})$

$(\text{challenges} \ ! \ \text{layer-idx})$

$(\text{fri-evidence-layer-len } \text{roots } \text{layer-idx})$

$(\text{fri-evidence-layer-idx } \text{roots } \text{query-idxs } \text{round-idx}' \ \text{layer-idx})$

$(2 \wedge \text{layer-idx})$

$(\text{fri-sibling-index } (\text{fri-evidence-layer-len } \text{roots } \text{layer-idx})$

$(\text{fri-evidence-layer-idx } \text{roots } \text{query-idxs } \text{round-idx}' \ \text{layer-idx}))$

yp yp-path yn yn-path

$(\text{fri-evidence-next-idx } \text{roots } \text{query-idxs } \text{round-idx}' \ \text{layer-idx})$

$(\text{fri-evidence-next-value } \text{roots } \text{challenges } \text{query-idxs } \text{round-idx}'$

$\text{layer-idx } \text{yp } \text{yn})$

$(\text{round-layers} \ ! \ \text{round-idx}' \ ! \ \text{layer-idx})$

and *auth:*

fri-layer-chunk-authenticated $(\text{roots} \ ! \ \text{layer-idx})$

$(\text{fri-evidence-layer-len } \text{roots } \text{layer-idx})$

$(\text{fri-evidence-layer-idx } \text{roots } \text{query-idxs } \text{round-idx } \text{layer-idx})$

$(\text{round-layers} \ ! \ \text{round-idx} \ ! \ \text{layer-idx})$ *final-state*

```

and auth':
  fri-layer-chunk-authenticated (roots ! layer-idx)
    (fri-evidence-layer-len roots layer-idx)
    (fri-evidence-layer-idx roots query-idxs round-idx' layer-idx)
    (round-layers ! round-idx' ! layer-idx) final-state
and cases:
  (fri-evidence-layer-idx roots query-idxs round-idx layer-idx =
    fri-evidence-layer-idx roots query-idxs round-idx' layer-idx  $\wedge$ 
    xp  $\neq$  yp)  $\vee$ 
  (fri-evidence-layer-idx roots query-idxs round-idx layer-idx =
    fri-sibling-index (fri-evidence-layer-len roots layer-idx)
    (fri-evidence-layer-idx roots query-idxs round-idx' layer-idx)  $\wedge$ 
    xp  $\neq$  yn)  $\vee$ 
  (fri-sibling-index (fri-evidence-layer-len roots layer-idx)
    (fri-evidence-layer-idx roots query-idxs round-idx layer-idx) =
    fri-evidence-layer-idx roots query-idxs round-idx' layer-idx  $\wedge$ 
    xn  $\neq$  yp)  $\vee$ 
  (fri-sibling-index (fri-evidence-layer-len roots layer-idx)
    (fri-evidence-layer-idx roots query-idxs round-idx layer-idx) =
    fri-sibling-index (fri-evidence-layer-len roots layer-idx)
    (fri-evidence-layer-idx roots query-idxs round-idx' layer-idx)  $\wedge$ 
    xn  $\neq$  yn)
unfolding
  generic-fri-sampled-same-layer-opening-conflict-with-authenticated-chunks-def
by blast
have layer-bound: layer-idx < length roots
  using layer-bound-ch len-challenges by simp
have chunk:
  fri-layer-opening-chunk (fri-evidence-layer-len roots layer-idx)
    xp xp-path xn xn-path (round-layers ! round-idx ! layer-idx)
  using fri-layer-step-evidenceD(4)[OF step] .
have chunk':
  fri-layer-opening-chunk (fri-evidence-layer-len roots layer-idx)
    yp yp-path yn yn-path (round-layers ! round-idx' ! layer-idx)
  using fri-layer-step-evidenceD(4)[OF step'] .
from cases show ?thesis
proof
  assume c:
    fri-evidence-layer-idx roots query-idxs round-idx layer-idx =
      fri-evidence-layer-idx roots query-idxs round-idx' layer-idx  $\wedge$ 
      xp  $\neq$  yp
  show ?thesis
    by (rule generic-fri-authenticated-same-layer-opening-conflictI-base-base
      [OF round-bound round-bound' layer-bound auth auth' chunk chunk'])
      (use c in simp-all)
next
  assume rest:
    (fri-evidence-layer-idx roots query-idxs round-idx layer-idx =
      fri-sibling-index (fri-evidence-layer-len roots layer-idx)

```

$(\text{fri-evidence-layer-idx roots query-idxs round-idx' layer-idx}) \wedge$
 $xp \neq yn) \vee$
 $(\text{fri-sibling-index (fri-evidence-layer-len roots layer-idx)}$
 $(\text{fri-evidence-layer-idx roots query-idxs round-idx layer-idx}) =$
 $\text{fri-evidence-layer-idx roots query-idxs round-idx' layer-idx} \wedge$
 $xn \neq yp) \vee$
 $(\text{fri-sibling-index (fri-evidence-layer-len roots layer-idx)}$
 $(\text{fri-evidence-layer-idx roots query-idxs round-idx layer-idx}) =$
 $\text{fri-sibling-index (fri-evidence-layer-len roots layer-idx)}$
 $(\text{fri-evidence-layer-idx roots query-idxs round-idx' layer-idx}) \wedge$
 $xn \neq yn)$
then show ?thesis
proof
assume c:
 $\text{fri-evidence-layer-idx roots query-idxs round-idx layer-idx} =$
 $\text{fri-sibling-index (fri-evidence-layer-len roots layer-idx)}$
 $(\text{fri-evidence-layer-idx roots query-idxs round-idx' layer-idx}) \wedge$
 $xp \neq yn$
show ?thesis
by (rule generic-fri-authenticated-same-layer-opening-conflictI-base-sibling
 $[\text{OF round-bound round-bound' layer-bound auth auth' chunk chunk'}])$
 (use c **in** simp-all)
next
assume rest':
 $(\text{fri-sibling-index (fri-evidence-layer-len roots layer-idx)}$
 $(\text{fri-evidence-layer-idx roots query-idxs round-idx layer-idx}) =$
 $\text{fri-evidence-layer-idx roots query-idxs round-idx' layer-idx} \wedge$
 $xn \neq yp) \vee$
 $(\text{fri-sibling-index (fri-evidence-layer-len roots layer-idx)}$
 $(\text{fri-evidence-layer-idx roots query-idxs round-idx layer-idx}) =$
 $\text{fri-sibling-index (fri-evidence-layer-len roots layer-idx)}$
 $(\text{fri-evidence-layer-idx roots query-idxs round-idx' layer-idx}) \wedge$
 $xn \neq yn)$
then show ?thesis
proof
assume c:
 $\text{fri-sibling-index (fri-evidence-layer-len roots layer-idx)}$
 $(\text{fri-evidence-layer-idx roots query-idxs round-idx layer-idx}) =$
 $\text{fri-evidence-layer-idx roots query-idxs round-idx' layer-idx} \wedge$
 $xn \neq yp$
show ?thesis
by (rule generic-fri-authenticated-same-layer-opening-conflictI-sibling-base
 $[\text{OF round-bound round-bound' layer-bound auth auth' chunk chunk'}])$
 (use c **in** simp-all)
next
assume c:
 $\text{fri-sibling-index (fri-evidence-layer-len roots layer-idx)}$
 $(\text{fri-evidence-layer-idx roots query-idxs round-idx layer-idx}) =$
 $\text{fri-sibling-index (fri-evidence-layer-len roots layer-idx)}$

```

      (fri-evidence-layer-idx roots query-idxs round-idx' layer-idx) ∧
    xn ≠ yn
  show ?thesis
    by (rule generic-fri-authenticated-same-layer-opening-conflictI-sibling-sibling
        [OF round-bound round-bound' layer-bound auth auth' chunk chunk'])
        (use c in simp-all)
  qed
qed
qed
qed

```

lemma *generic-fri-sampled-same-layer-opening-conflict-with-authenticated-chunks-imp-sampled:*

assumes

generic-fri-sampled-same-layer-opening-conflict-with-authenticated-chunks
roots challenges query-idxs round-layers final-state

shows

generic-fri-sampled-same-layer-opening-conflict roots challenges
query-idxs round-layers

using *assms*

unfolding

generic-fri-sampled-same-layer-opening-conflict-with-authenticated-chunks-def
generic-fri-sampled-same-layer-opening-conflict-def

by *blast*

lemma *generic-fri-sampled-same-layer-imp-authenticated-or-recorded-auth-gap:*

assumes *conflict:*

generic-fri-sampled-same-layer-opening-conflict roots challenges
query-idxs round-layers

shows

generic-fri-sampled-same-layer-opening-conflict-with-authenticated-chunks
roots challenges query-idxs round-layers final-state ∨
generic-fri-sampled-same-layer-recorded-auth-gap roots challenges
query-idxs round-layers final-state

proof —

from *conflict* **obtain** *round-idx round-idx' layer-idx*
xp xp-path xn xn-path yp yp-path yn yn-path **where**
round-bound: round-idx < length query-idxs
and *round-bound': round-idx' < length query-idxs*
and *layer-bound: layer-idx < length challenges*
and *step:*
fri-layer-step-evidence
(roots ! layer-idx)
(challenges ! layer-idx)
(fri-evidence-layer-len roots layer-idx)
(fri-evidence-layer-idx roots query-idxs round-idx layer-idx)
(2 ^ layer-idx)
(fri-sibling-index (fri-evidence-layer-len roots layer-idx)
(fri-evidence-layer-idx roots query-idxs round-idx layer-idx))
xp xp-path xn xn-path

```

(fri-evidence-next-idx roots query-idxs round-idx layer-idx)
(fri-evidence-next-value roots challenges query-idxs round-idx
 layer-idx xp xn)
(round-layers ! round-idx ! layer-idx)
and step':
  fri-layer-step-evidence
    (roots ! layer-idx)
    (challenges ! layer-idx)
    (fri-evidence-layer-len roots layer-idx)
    (fri-evidence-layer-idx roots query-idxs round-idx' layer-idx)
    (2 ^ layer-idx)
    (fri-sibling-index (fri-evidence-layer-len roots layer-idx)
      (fri-evidence-layer-idx roots query-idxs round-idx' layer-idx))
    yp yp-path yn yn-path
    (fri-evidence-next-idx roots query-idxs round-idx' layer-idx)
    (fri-evidence-next-value roots challenges query-idxs round-idx'
      layer-idx yp yn)
    (round-layers ! round-idx' ! layer-idx)
and cases:
  (fri-evidence-layer-idx roots query-idxs round-idx layer-idx =
    fri-evidence-layer-idx roots query-idxs round-idx' layer-idx ∧
    xp ≠ yp) ∨
  (fri-evidence-layer-idx roots query-idxs round-idx layer-idx =
    fri-sibling-index (fri-evidence-layer-len roots layer-idx)
      (fri-evidence-layer-idx roots query-idxs round-idx' layer-idx) ∧
    xp ≠ yn) ∨
  (fri-sibling-index (fri-evidence-layer-len roots layer-idx)
    (fri-evidence-layer-idx roots query-idxs round-idx layer-idx) =
    fri-evidence-layer-idx roots query-idxs round-idx' layer-idx ∧
    xn ≠ yp) ∨
  (fri-sibling-index (fri-evidence-layer-len roots layer-idx)
    (fri-evidence-layer-idx roots query-idxs round-idx layer-idx) =
    fri-sibling-index (fri-evidence-layer-len roots layer-idx)
      (fri-evidence-layer-idx roots query-idxs round-idx' layer-idx) ∧
    xn ≠ yn)
unfolding generic-fri-sampled-same-layer-opening-conflict-def
by blast
show ?thesis
proof (cases
  generic-fri-recorded-layer-chunk-authenticated roots query-idxs
    round-layers final-state round-idx layer-idx ∧
  generic-fri-recorded-layer-chunk-authenticated roots query-idxs
    round-layers final-state round-idx' layer-idx)
case True
have auth:
  fri-layer-chunk-authenticated (roots ! layer-idx)
    (fri-evidence-layer-len roots layer-idx)
    (fri-evidence-layer-idx roots query-idxs round-idx layer-idx)
    (round-layers ! round-idx ! layer-idx) final-state

```

```

using True
unfolding generic-fri-recorded-layer-chunk-authenticated-def
by simp
have auth':
  fri-layer-chunk-authenticated (roots ! layer-idx)
    (fri-evidence-layer-len roots layer-idx)
    (fri-evidence-layer-idx roots query-idxs round-idx' layer-idx)
    (round-layers ! round-idx' ! layer-idx) final-state
using True
unfolding generic-fri-recorded-layer-chunk-authenticated-def
by simp
have
  generic-fri-sampled-same-layer-opening-conflict-with-authenticated-chunks
    roots challenges query-idxs round-layers final-state
unfolding
  generic-fri-sampled-same-layer-opening-conflict-with-authenticated-chunks-def
by (intro exI conjI)
  (rule round-bound, rule round-bound', rule layer-bound, rule step,
    rule step', rule auth, rule auth', rule cases)
then show ?thesis by simp
next
case False
have
  generic-fri-sampled-same-layer-recorded-auth-gap roots challenges
    query-idxs round-layers final-state
unfolding generic-fri-sampled-same-layer-recorded-auth-gap-def
by (intro exI conjI)
  (rule round-bound, rule round-bound', rule layer-bound,
    rule conflict, use False in simp)
then show ?thesis by simp
qed
qed

lemma generic-fri-sampled-same-layer-auth-gap-imp-recorded-auth-gap:
assumes conflict:
  generic-fri-sampled-same-layer-opening-conflict roots challenges
    query-idxs round-layers
and no-auth:
  ¬ generic-fri-sampled-same-layer-opening-conflict-with-authenticated-chunks
    roots challenges query-idxs round-layers final-state
shows
  generic-fri-sampled-same-layer-recorded-auth-gap roots challenges
    query-idxs round-layers final-state
using generic-fri-sampled-same-layer-imp-authenticated-or-recorded-auth-gap
  [OF conflict, of final-state] no-auth
by blast

lemma generic-fri-sampled-same-layer-opening-conflict-with-authenticated-chunks-imp-partial-merkle:
assumes conflict:

```

generic-fri-sampled-same-layer-opening-conflict-with-authenticated-chunks
roots challenges query-idxs round-layers final-state
and *len-challenges*: *length challenges = length roots*
shows *partial-merkle-inconsistency-bad s* (*Some (result, final-state)*)
proof –
have *auth-conflict*:
generic-fri-authenticated-same-layer-opening-conflict roots query-idxs
final-state
by (*rule*
generic-fri-sampled-same-layer-opening-conflict-with-authenticated-chunks-imp-authenticated
[*OF conflict len-challenges*])
show *?thesis*
by (*rule generic-fri-authenticated-same-layer-opening-conflict-imp-partial-merkle*
[*OF auth-conflict*])
qed

definition *generic-fri-sampled-assignment-conflict-with-authenticated-same-layer*

:: *'f list ⇒ 'f list ⇒ 'f list ⇒ 'f ⇒*
nat list ⇒ 'f list list list ⇒
(*'f, 'a*) *protocol-channel-scheme ⇒ bool*

where

generic-fri-sampled-assignment-conflict-with-authenticated-same-layer
candidate-table roots challenges final-value query-idxs round-layers
final-state $\longleftrightarrow$
generic-fri-sampled-base-opening-conflict candidate-table roots
challenges query-idxs round-layers $\vee$
generic-fri-sampled-next-value-conflict roots challenges query-idxs
round-layers $\vee$
generic-fri-sampled-successor-opening-conflict roots challenges
query-idxs round-layers $\vee$
generic-fri-sampled-same-layer-opening-conflict-with-authenticated-chunks
roots challenges query-idxs round-layers final-state $\vee$
generic-fri-sampled-final-value-conflict roots challenges final-value
query-idxs round-layers

definition *generic-fri-sampled-assignment-conflict-without-same-layer*

:: *'f list ⇒ 'f list ⇒ 'f list ⇒ 'f ⇒*
nat list ⇒ 'f list list list ⇒ bool

where

generic-fri-sampled-assignment-conflict-without-same-layer
candidate-table roots challenges final-value query-idxs round-layers
 $\longleftrightarrow$
generic-fri-sampled-base-opening-conflict candidate-table roots
challenges query-idxs round-layers $\vee$
generic-fri-sampled-next-value-conflict roots challenges query-idxs
round-layers $\vee$
generic-fri-sampled-successor-opening-conflict roots challenges
query-idxs round-layers $\vee$
generic-fri-sampled-final-value-conflict roots challenges final-value

query-idxs round-layers

lemma *generic-fri-sampled-assignment-conflict-without-same-layer-split:*

assumes

generic-fri-sampled-assignment-conflict-without-same-layer
candidate-table roots challenges final-value query-idxs round-layers

shows

generic-fri-sampled-base-opening-conflict candidate-table roots
challenges query-idxs round-layers $\vee$
generic-fri-sampled-next-value-conflict roots challenges query-idxs
round-layers $\vee$
generic-fri-sampled-successor-opening-conflict roots challenges
query-idxs round-layers $\vee$
generic-fri-sampled-final-value-conflict roots challenges final-value
query-idxs round-layers

using *assms*

unfolding *generic-fri-sampled-assignment-conflict-without-same-layer-def*
by *blast*

lemma *generic-fri-sampled-assignment-conflict-without-same-layerI-base:*

assumes

generic-fri-sampled-base-opening-conflict candidate-table roots
challenges query-idxs round-layers

shows

generic-fri-sampled-assignment-conflict-without-same-layer
candidate-table roots challenges final-value query-idxs round-layers

using *assms*

unfolding *generic-fri-sampled-assignment-conflict-without-same-layer-def*
by *blast*

lemma *generic-fri-sampled-assignment-conflict-without-same-layerI-next:*

assumes

generic-fri-sampled-next-value-conflict roots challenges query-idxs
round-layers

shows

generic-fri-sampled-assignment-conflict-without-same-layer
candidate-table roots challenges final-value query-idxs round-layers

using *assms*

unfolding *generic-fri-sampled-assignment-conflict-without-same-layer-def*
by *blast*

lemma *generic-fri-sampled-assignment-conflict-without-same-layerI-successor:*

assumes

generic-fri-sampled-successor-opening-conflict roots challenges
query-idxs round-layers

shows

generic-fri-sampled-assignment-conflict-without-same-layer
candidate-table roots challenges final-value query-idxs round-layers

using *assms*

unfolding *generic-fri-sampled-assignment-conflict-without-same-layer-def*
by *blast*

lemma *generic-fri-sampled-assignment-conflict-without-same-layerI-final:*

assumes

generic-fri-sampled-final-value-conflict roots challenges final-value
query-idxs round-layers

shows

generic-fri-sampled-assignment-conflict-without-same-layer
candidate-table roots challenges final-value query-idxs round-layers

using *assms*

unfolding *generic-fri-sampled-assignment-conflict-without-same-layer-def*
by *blast*

lemma *generic-fri-sampled-assignment-conflict-without-same-layer-iff:*

generic-fri-sampled-assignment-conflict-without-same-layer
candidate-table roots challenges final-value query-idxs round-layers

$\longleftrightarrow$

generic-fri-sampled-base-opening-conflict candidate-table roots
challenges query-idxs round-layers $\vee$

generic-fri-sampled-next-value-conflict roots challenges query-idxs
round-layers $\vee$

generic-fri-sampled-successor-opening-conflict roots challenges
query-idxs round-layers $\vee$

generic-fri-sampled-final-value-conflict roots challenges final-value
query-idxs round-layers

unfolding *generic-fri-sampled-assignment-conflict-without-same-layer-def*
by *simp*

lemma *generic-fri-sampled-assignment-conflict-with-authenticated-same-layer-split:*

assumes

generic-fri-sampled-assignment-conflict-with-authenticated-same-layer
candidate-table roots challenges final-value query-idxs round-layers
final-state

shows

generic-fri-sampled-assignment-conflict-without-same-layer
candidate-table roots challenges final-value query-idxs round-layers $\vee$
generic-fri-sampled-same-layer-opening-conflict-with-authenticated-chunks
roots challenges query-idxs round-layers final-state

using *assms*

unfolding

generic-fri-sampled-assignment-conflict-with-authenticated-same-layer-def
generic-fri-sampled-assignment-conflict-without-same-layer-def

by *blast*

lemma *generic-fri-sampled-assignment-conflict-with-authenticated-same-layerI-without:*

assumes

generic-fri-sampled-assignment-conflict-without-same-layer
candidate-table roots challenges final-value query-idxs round-layers

shows

generic-fri-sampled-assignment-conflict-with-authenticated-same-layer
candidate-table roots challenges final-value query-idxs round-layers
final-state

using *assms*

unfolding

generic-fri-sampled-assignment-conflict-with-authenticated-same-layer-def
generic-fri-sampled-assignment-conflict-without-same-layer-def

by *blast*

lemma *generic-fri-sampled-assignment-conflict-with-authenticated-same-layerI-same:*

assumes

generic-fri-sampled-same-layer-opening-conflict-with-authenticated-chunks
roots challenges query-idxs round-layers final-state

shows

generic-fri-sampled-assignment-conflict-with-authenticated-same-layer
candidate-table roots challenges final-value query-idxs round-layers
final-state

using *assms*

unfolding *generic-fri-sampled-assignment-conflict-with-authenticated-same-layer-def*

by *blast*

lemma *generic-fri-sampled-assignment-conflict-with-authenticated-same-layer-iff:*

generic-fri-sampled-assignment-conflict-with-authenticated-same-layer
candidate-table roots challenges final-value query-idxs round-layers
final-state $\longleftrightarrow$

generic-fri-sampled-assignment-conflict-without-same-layer
candidate-table roots challenges final-value query-idxs round-layers $\vee$

generic-fri-sampled-same-layer-opening-conflict-with-authenticated-chunks
roots challenges query-idxs round-layers final-state

proof

assume

generic-fri-sampled-assignment-conflict-with-authenticated-same-layer
candidate-table roots challenges final-value query-idxs round-layers
final-state

then show

generic-fri-sampled-assignment-conflict-without-same-layer
candidate-table roots challenges final-value query-idxs round-layers $\vee$
generic-fri-sampled-same-layer-opening-conflict-with-authenticated-chunks
roots challenges query-idxs round-layers final-state

by (rule *generic-fri-sampled-assignment-conflict-with-authenticated-same-layer-split*)

next

assume

generic-fri-sampled-assignment-conflict-without-same-layer
candidate-table roots challenges final-value query-idxs round-layers $\vee$
generic-fri-sampled-same-layer-opening-conflict-with-authenticated-chunks
roots challenges query-idxs round-layers final-state

then show

generic-fri-sampled-assignment-conflict-with-authenticated-same-layer

candidate-table roots challenges final-value query-idxs round-layers
final-state
by (*elim disjE*)
 (rule *generic-fri-sampled-assignment-conflict-with-authenticated-same-layerI-without*,
assumption,
 rule *generic-fri-sampled-assignment-conflict-with-authenticated-same-layerI-same*,
assumption)
qed

lemma *generic-fri-sampled-assignment-conflict-with-authenticated-same-layer-imp-sampled*:
assumes
generic-fri-sampled-assignment-conflict-with-authenticated-same-layer
candidate-table roots challenges final-value query-idxs round-layers
final-state
shows
generic-fri-sampled-assignment-conflict candidate-table roots challenges
final-value query-idxs round-layers
using *assms*
generic-fri-sampled-same-layer-opening-conflict-with-authenticated-chunks-imp-sampled
unfolding
generic-fri-sampled-assignment-conflict-with-authenticated-same-layer-def
generic-fri-sampled-assignment-conflict-def
by *blast*

lemma *generic-fri-sampled-assignment-conflict-imp-authenticated-or-same-layer-auth-gap*:
assumes
generic-fri-sampled-assignment-conflict candidate-table roots
challenges final-value query-idxs round-layers
shows
generic-fri-sampled-assignment-conflict-with-authenticated-same-layer
candidate-table roots challenges final-value query-idxs round-layers
final-state $\vee$
 (generic-fri-sampled-same-layer-opening-conflict roots challenges
query-idxs round-layers $\wedge$
 $\neg$ generic-fri-sampled-same-layer-opening-conflict-with-authenticated-chunks
roots challenges query-idxs round-layers final-state)
using *assms*
unfolding *generic-fri-sampled-assignment-conflict-def*
generic-fri-sampled-assignment-conflict-with-authenticated-same-layer-def
generic-fri-sampled-assignment-conflict-without-same-layer-def
by *blast*

lemma *generic-fri-authenticated-same-layer-branch-imp-partial-merkle*:
assumes
generic-fri-sampled-same-layer-opening-conflict-with-authenticated-chunks
roots challenges query-idxs round-layers final-state
and *length challenges = length roots*
shows *partial-merkle-inconsistency-bad s (Some (result, final-state))*
by (rule

generic-fri-sampled-same-layer-opening-conflict-with-authenticated-chunks-imp-partial-merkle
 [OF assms])

lemma *generic-fri-sampled-same-layer-opening-conflict-imp-authenticated-if-recorded:*

assumes *conflict:*

generic-fri-sampled-same-layer-opening-conflict roots challenges

query-idxs round-layers

and *len-challenges:* *length challenges = length roots*

and *recorded:*

$\bigwedge$ *round-idx layer-idx.*

round-idx < length query-idxs $\implies$

layer-idx < length roots $\implies$

generic-fri-recorded-layer-chunk-authenticated roots query-idxs

round-layers final-state round-idx layer-idx

shows

generic-fri-authenticated-same-layer-opening-conflict roots query-idxs

final-state

proof –

from *conflict* **obtain** *round-idx round-idx' layer-idx*

xp xp-path xn xn-path yp yp-path yn yn-path **where**

round-bound: *round-idx < length query-idxs*

and *round-bound':* *round-idx' < length query-idxs*

and *layer-bound-ch:* *layer-idx < length challenges*

and *step:*

fri-layer-step-evidence

(roots ! layer-idx)

(challenges ! layer-idx)

(fri-evidence-layer-len roots layer-idx)

(fri-evidence-layer-idx roots query-idxs round-idx layer-idx)

(2 ^ layer-idx)

(fri-sibling-index (fri-evidence-layer-len roots layer-idx)

(fri-evidence-layer-idx roots query-idxs round-idx layer-idx))

xp xp-path xn xn-path

(fri-evidence-next-idx roots query-idxs round-idx layer-idx)

(fri-evidence-next-value roots challenges query-idxs round-idx

layer-idx xp xn)

(round-layers ! round-idx ! layer-idx)

and *step':*

fri-layer-step-evidence

(roots ! layer-idx)

(challenges ! layer-idx)

(fri-evidence-layer-len roots layer-idx)

(fri-evidence-layer-idx roots query-idxs round-idx' layer-idx)

(2 ^ layer-idx)

(fri-sibling-index (fri-evidence-layer-len roots layer-idx)

(fri-evidence-layer-idx roots query-idxs round-idx' layer-idx))

yp yp-path yn yn-path

(fri-evidence-next-idx roots query-idxs round-idx' layer-idx)

(fri-evidence-next-value roots challenges query-idxs round-idx')

```

    layer-idx yp yn)
  (round-layers ! round-idx' ! layer-idx)
and cases:
  (fri-evidence-layer-idx roots query-idxs round-idx layer-idx =
    fri-evidence-layer-idx roots query-idxs round-idx' layer-idx ∧
    xp ≠ yp) ∨
  (fri-evidence-layer-idx roots query-idxs round-idx layer-idx =
    fri-sibling-index (fri-evidence-layer-len roots layer-idx)
      (fri-evidence-layer-idx roots query-idxs round-idx' layer-idx) ∧
    xp ≠ yn) ∨
  (fri-sibling-index (fri-evidence-layer-len roots layer-idx)
    (fri-evidence-layer-idx roots query-idxs round-idx layer-idx) =
    fri-evidence-layer-idx roots query-idxs round-idx' layer-idx ∧
    xn ≠ yp) ∨
  (fri-sibling-index (fri-evidence-layer-len roots layer-idx)
    (fri-evidence-layer-idx roots query-idxs round-idx layer-idx) =
    fri-sibling-index (fri-evidence-layer-len roots layer-idx)
      (fri-evidence-layer-idx roots query-idxs round-idx' layer-idx) ∧
    xn ≠ yn)
unfolding generic-fri-sampled-same-layer-opening-conflict-def
  fri-evidence-layer-len-def fri-evidence-layer-idx-def
  fri-evidence-next-idx-def fri-evidence-next-value-def
by blast
have layer-bound: layer-idx < length roots
  using layer-bound-ch len-challenges by simp
have chunk:
  fri-layer-opening-chunk (fri-evidence-layer-len roots layer-idx)
    xp xp-path xn xn-path (round-layers ! round-idx ! layer-idx)
  using fri-layer-step-evidenceD(4)[OF step] .
have chunk':
  fri-layer-opening-chunk (fri-evidence-layer-len roots layer-idx)
    yp yp-path yn yn-path (round-layers ! round-idx' ! layer-idx)
  using fri-layer-step-evidenceD(4)[OF step'] .
have auth:
  fri-layer-chunk-authenticated (roots ! layer-idx)
    (fri-evidence-layer-len roots layer-idx)
    (fri-evidence-layer-idx roots query-idxs round-idx layer-idx)
    (round-layers ! round-idx ! layer-idx) final-state
  using recorded[OF round-bound layer-bound]
  unfolding generic-fri-recorded-layer-chunk-authenticated-def .
have auth':
  fri-layer-chunk-authenticated (roots ! layer-idx)
    (fri-evidence-layer-len roots layer-idx)
    (fri-evidence-layer-idx roots query-idxs round-idx' layer-idx)
    (round-layers ! round-idx' ! layer-idx) final-state
  using recorded[OF round-bound' layer-bound]
  unfolding generic-fri-recorded-layer-chunk-authenticated-def .
from cases show ?thesis
proof

```

```

assume c:
  fri-evidence-layer-idx roots query-idxs round-idx layer-idx =
    fri-evidence-layer-idx roots query-idxs round-idx' layer-idx ∧
    xp ≠ yp
show ?thesis
  by (rule generic-fri-authenticated-same-layer-opening-conflictI-base-base
    [OF round-bound round-bound' layer-bound auth auth' chunk chunk'])
    (use c in simp-all)
next
assume rest:
  (fri-evidence-layer-idx roots query-idxs round-idx layer-idx =
    fri-sibling-index (fri-evidence-layer-len roots layer-idx)
    (fri-evidence-layer-idx roots query-idxs round-idx' layer-idx) ∧
    xp ≠ yn) ∨
  (fri-sibling-index (fri-evidence-layer-len roots layer-idx)
    (fri-evidence-layer-idx roots query-idxs round-idx layer-idx) =
    fri-evidence-layer-idx roots query-idxs round-idx' layer-idx ∧
    xn ≠ yp) ∨
  (fri-sibling-index (fri-evidence-layer-len roots layer-idx)
    (fri-evidence-layer-idx roots query-idxs round-idx layer-idx) =
    fri-sibling-index (fri-evidence-layer-len roots layer-idx)
    (fri-evidence-layer-idx roots query-idxs round-idx' layer-idx) ∧
    xn ≠ yn)
then show ?thesis
proof
  assume c:
    fri-evidence-layer-idx roots query-idxs round-idx layer-idx =
      fri-sibling-index (fri-evidence-layer-len roots layer-idx)
      (fri-evidence-layer-idx roots query-idxs round-idx' layer-idx) ∧
      xp ≠ yn
  show ?thesis
  by (rule generic-fri-authenticated-same-layer-opening-conflictI-base-sibling
    [OF round-bound round-bound' layer-bound auth auth' chunk chunk'])
    (use c in simp-all)
next
assume rest':
  (fri-sibling-index (fri-evidence-layer-len roots layer-idx)
    (fri-evidence-layer-idx roots query-idxs round-idx layer-idx) =
    fri-evidence-layer-idx roots query-idxs round-idx' layer-idx ∧
    xn ≠ yp) ∨
  (fri-sibling-index (fri-evidence-layer-len roots layer-idx)
    (fri-evidence-layer-idx roots query-idxs round-idx layer-idx) =
    fri-sibling-index (fri-evidence-layer-len roots layer-idx)
    (fri-evidence-layer-idx roots query-idxs round-idx' layer-idx) ∧
    xn ≠ yn)
then show ?thesis
proof
  assume c:
    fri-sibling-index (fri-evidence-layer-len roots layer-idx)

```

```

      (fri-evidence-layer-idx roots query-idxs round-idx layer-idx) =
      fri-evidence-layer-idx roots query-idxs round-idx' layer-idx ∧
      xn ≠ yp
show ?thesis
  by (rule generic-fri-authenticated-same-layer-opening-conflictI-sibling-base
      [OF round-bound round-bound' layer-bound auth auth' chunk chunk'])
      (use c in simp-all)
next
  assume c:
    fri-sibling-index (fri-evidence-layer-len roots layer-idx)
    (fri-evidence-layer-idx roots query-idxs round-idx layer-idx) =
    fri-sibling-index (fri-evidence-layer-len roots layer-idx)
    (fri-evidence-layer-idx roots query-idxs round-idx' layer-idx) ∧
    xn ≠ yn
  show ?thesis
    by (rule generic-fri-authenticated-same-layer-opening-conflictI-sibling-sibling
        [OF round-bound round-bound' layer-bound auth auth' chunk chunk'])
        (use c in simp-all)
  qed
qed
qed
qed

lemma accepted-fri-opening-transcript-trace-step-with-authenticated-chunk-at:
assumes fri-openings:
  accepted-fri-opening-transcript s out trace-roots trace-bs trace-final
  dg composition-roots composition-bs composition-final query-idxs
  trace-round-layers composition-round-layers
and out-eq: out = Some (result, final-state)
and round-bound: i < rounds
and layer-bound: j < length trace-roots
and raw-layer:
  ∧ raw.
  0 < fri-layer-lengths (length trace-roots) (clength * scale) ! j ∧
  fri-layer-indices (length trace-roots) (index (to-nat raw))
  (clength * scale) ! j <
  fri-layer-lengths (length trace-roots) (clength * scale) ! j
shows
  generic-fri-transcript-step-with-authenticated-chunk trace-roots
  trace-bs query-idxs trace-round-layers final-state i j
proof –
have step:
  fri-round-layer-evidence trace-roots trace-bs query-idxs i j
  trace-round-layers
by (rule accepted-fri-opening-transcript-trace-round-layer-evidence
      [OF fri-openings round-bound layer-bound])
from accepted-fri-opening-transcript-trace-selected-chunk-authenticated-at
      [OF fri-openings out-eq round-bound layer-bound raw-layer]
obtain auth-chunk where auth:

```

```

  fri-layer-chunk-authenticated (trace-roots ! j)
    (fri-layer-lengths (length trace-roots) (clength * scale) ! j)
    (fri-layer-indices (length trace-roots) (query-idxs ! i)
      (clength * scale) ! j)
    auth-chunk final-state
  by blast
have auth-compact:
  fri-layer-chunk-authenticated (trace-roots ! j)
    (fri-evidence-layer-len trace-roots j)
    (fri-evidence-layer-idx trace-roots query-idxs i j)
    auth-chunk final-state
  using auth
  unfolding fri-evidence-layer-len-def fri-evidence-layer-idx-def
  by simp
show ?thesis
  by (rule generic-fri-transcript-step-with-authenticated-chunkI
    [OF step auth-compact])
qed

lemma accepted-fri-opening-transcript-composition-step-with-authenticated-chunk-at:
  assumes fri-openings:
    accepted-fri-opening-transcript s out trace-roots trace-bs trace-final
      dg composition-roots composition-bs composition-final query-idxs
      trace-round-layers composition-round-layers
  and out-eq: out = Some (result, final-state)
  and round-bound: i < rounds
  and layer-bound: j < length composition-roots
  and raw-layer:
     $\bigwedge \text{raw.}$ 
    0 < fri-layer-lengths (length composition-roots) (clength * scale) ! j  $\wedge$ 
    fri-layer-indices (length composition-roots) (index (to-nat raw))
      (clength * scale) ! j <
    fri-layer-lengths (length composition-roots) (clength * scale) ! j
  shows
    generic-fri-transcript-step-with-authenticated-chunk composition-roots
      composition-bs query-idxs composition-round-layers final-state i j
proof -
  have step:
    fri-round-layer-evidence composition-roots composition-bs query-idxs i j
      composition-round-layers
  by (rule accepted-fri-opening-transcript-composition-round-layer-evidence
    [OF fri-openings round-bound layer-bound])
  from
    accepted-fri-opening-transcript-composition-selected-chunk-authenticated-at
      [OF fri-openings out-eq round-bound layer-bound raw-layer]
  obtain auth-chunk where auth:
    fri-layer-chunk-authenticated (composition-roots ! j)
      (fri-layer-lengths (length composition-roots) (clength * scale) ! j)
      (fri-layer-indices (length composition-roots) (query-idxs ! i)

```



```

      (clength * scale) ! j)
    auth-chunk final-state
  by blast
have auth-compact:
  fri-layer-chunk-authenticated (composition-roots ! j)
  (fri-evidence-layer-len composition-roots j)
  (fri-evidence-layer-idx composition-roots query-idxs i j)
  auth-chunk final-state
using auth
unfolding fri-evidence-layer-len-def fri-evidence-layer-idx-def
by simp
show ?thesis
by (rule generic-fri-transcript-step-with-authenticated-chunkI
  [OF step auth-compact])
qed

end

end

```

```

theory Soundness-FRI-Layer-Replay
imports Soundness-FRI-Layer-Authenticated
begin

```

Narrow replay facts for selected FRI layer openings.

The first lemma is a generic selected-position decomposition for *mfold*. It avoids adding another proof by induction over the full FRI verifier layer in the already-large Merkle theory.

```

context soundness
begin

```

```

lemma fri-layer-lengths-prefix-nth:
  assumes j < n
  shows fri-layer-lengths n len ! j = fri-layer-lengths (Suc j) len ! j
  using assms
proof (induction j arbitrary: n len)
  case 0
  then show ?case
    by (cases n) simp-all
next
  case (Suc j)
  then obtain n' where n-eq: n = Suc n'
    by (cases n) auto
  have j-bound: j < n'
    using Suc.prem1 unfolding n-eq by simp
  show ?case
    using Suc.IH[OF j-bound, of len div 2]
    unfolding n-eq by simp

```

qed

lemma *fri-layer-indices-prefix-nth*:

assumes $j < n$

shows

$fri-layer-indices\ n\ idx\ len\ !\ j =$
 $fri-layer-indices\ (Suc\ j)\ idx\ len\ !\ j$

using *assms*

proof (*induction j arbitrary: n idx len*)

case 0

then show *?case*

by (*cases n*) *simp-all*

next

case (*Suc j*)

then obtain n' **where** $n\text{-eq}: n = Suc\ n'$

by (*cases n*) *auto*

have $j\text{-bound}: j < n'$

using *Suc.prem*s **unfolding** $n\text{-eq}$ **by** *simp*

show *?case*

using *Suc.IH*[*OF j-bound, of idx mod (len div 2) len div 2*]

unfolding $n\text{-eq}$ **by** *simp*

qed

lemma *fri-layer-lengths-prefix-same*:

assumes $k\text{-bound}: k < j$

shows $fri-layer-lengths\ (Suc\ j)\ len\ !\ k = fri-layer-lengths\ j\ len\ !\ k$

using *fri-layer-lengths-prefix-nth*[*OF less-trans*[*OF k-bound lessI*], *of len*]

fri-layer-lengths-prefix-nth[*OF k-bound, of len*]

by *simp*

lemma *fri-layers-transcript-snoc*:

assumes *prefix*:

fri-layers-transcript j len prefix-chunks prefix-chunk

and *last*:

fri-layer-opening-chunk (fri-layer-lengths (Suc j) len ! j)

xp xp-path xn xn-path last-chunk

shows

fri-layers-transcript (Suc j) len (prefix-chunks @ [last-chunk])
(prefix-chunk @ last-chunk)

using *prefix last*

proof (*induction j arbitrary: len prefix-chunks prefix-chunk*)

case 0

then have $chunks\text{-eq}: prefix-chunks = []$

and $prefix-chunk\text{-eq}: prefix-chunk = []$

unfolding *fri-layers-transcript-def* **by** *simp-all*

have *last-fact*:

fri-layer-opening-chunk len xp xp-path xn xn-path last-chunk

using *0.prem*s(2) **by** *simp*

have *last-open*:

```

     $\exists$  yp yp-path yn yn-path.
    fri-layer-opening-chunk len yp yp-path yn yn-path last-chunk
    apply (rule exI[where x=xp])
    apply (rule exI[where x=xp-path])
    apply (rule exI[where x=xn])
    apply (rule exI[where x=xn-path])
    apply (rule last-fact)
    done
  show ?case
    unfolding chunks-eq prefix-chunk-eq fri-layers-transcript-def
    using last-open by simp
next
case (Suc j)
obtain first rest where chunks-eq: prefix-chunks = first # rest
  using Suc.premis(1) unfolding fri-layers-transcript-def
  by (cases prefix-chunks) auto
have first-open:
   $\exists$  yp yp-path yn yn-path.
  fri-layer-opening-chunk len yp yp-path yn yn-path first
  using Suc.premis(1) unfolding fri-layers-transcript-def chunks-eq
  by auto
have rest-layers:
  fri-layers-transcript j (len div 2) rest (List.concat rest)
  using Suc.premis(1) unfolding fri-layers-transcript-def chunks-eq
  by auto
have last-tail:
  fri-layer-opening-chunk
    (fri-layer-lengths (Suc j) (len div 2) ! j)
    xp xp-path xn xn-path last-chunk
  using Suc.premis(2) by simp
have tail:
  fri-layers-transcript (Suc j) (len div 2) (rest @ [last-chunk])
    (List.concat rest @ last-chunk)
  by (rule Suc.IH[OF rest-layers last-tail])
have prefix-chunk-eq: prefix-chunk = first @ List.concat rest
  using Suc.premis(1) unfolding fri-layers-transcript-def chunks-eq by simp
show ?case
  unfolding chunks-eq prefix-chunk-eq fri-layers-transcript-def
  using first-open tail unfolding fri-layers-transcript-def by auto
qed

```

```

lemma fri-layers-transcript-last-chunk-eq:
  assumes layers:
    fri-layers-transcript (Suc j) len layer-chunks chunk
  and prefix:
    fri-layers-transcript j len prefix-chunks prefix-chunk
  and last:
    fri-layer-opening-chunk (fri-layer-lengths (Suc j) len ! j)
    xp xp-path xn xn-path last-chunk

```

and *chunk-eq*: *chunk* = *prefix-chunk* @ *last-chunk*
shows *layer-chunks* ! *j* = *last-chunk*
proof –
have *combined*:
fri-layers-transcript (*Suc j*) *len* (*prefix-chunks* @ [*last-chunk*])
(*prefix-chunk* @ *last-chunk*)
by (*rule fri-layers-transcript-snoc* [*OF prefix last*])
have *layers-eq*: *layer-chunks* = *prefix-chunks* @ [*last-chunk*]
by (*rule fri-layers-transcript-unique*
[*OF layers combined*[*unfolded chunk-eq*[*symmetric*]]])
have *len-prefix*: *length prefix-chunks* = *j*
using *prefix unfolding fri-layers-transcript-def* **by** *simp*
show ?*thesis*
using *layers-eq len-prefix* **by** (*simp add: nth-append*)
qed

lemma *generic-fri-round-forced-next-value-from-step-evidence*:
assumes *step*:
fri-layer-step-evidence
(*roots* ! *layer-idx*)
(*challenges* ! *layer-idx*)
(*fri-evidence-layer-len roots layer-idx*)
(*fri-evidence-layer-idx roots query-idxs round-idx layer-idx*)
(*2 ^ layer-idx*)
(*fri-sibling-index* (*fri-evidence-layer-len roots layer-idx*)
(*fri-evidence-layer-idx roots query-idxs round-idx layer-idx*))
xp xp-path xn xn-path
(*fri-evidence-next-idx roots query-idxs round-idx layer-idx*)
v
(*round-layers* ! *round-idx* ! *layer-idx*)
shows
generic-fri-round-forced-next-value roots challenges query-idxs
round-layers round-idx layer-idx v
using *step*
unfolding *generic-fri-round-forced-next-value-def* **by** *blast*

lemma *mfold-outcome-decomp-at*:
assumes *outcome*:
Some (out, t) ∈ set-dist (execute (mfold a ms) s)
and *j-bound*: *j* < *length ms*
shows
 $\exists a-j\ s-j\ out-j\ s-suc.$
Some (a-j, s-j) ∈ set-dist (execute (mfold a (take j ms)) s) ∧
Some (out-j, s-suc) ∈ set-dist (execute ((ms ! j) a-j) s-j) ∧
Some (out, t) ∈
set-dist (execute (mfold out-j (drop (Suc j) ms)) s-suc)
using *outcome j-bound*
proof (*induction j arbitrary: a ms s*)
case 0

```

then obtain  $m \ ms'$  where  $ms\text{-eq}$ :  $ms = m \# ms'$ 
  by (cases  $ms$ ) auto
from  $0.\text{prems}(1)[\text{unfolded } ms\text{-eq}]$ 
obtain  $out\text{-}j \ s\text{-}suc$  where
   $head$ :  $\text{Some } (out\text{-}j, s\text{-}suc) \in \text{set-dist } (\text{execute } (m \ a) \ s)$ 
  and  $tail$ :
     $\text{Some } (out, t) \in \text{set-dist } (\text{execute } (mfold \ out\text{-}j \ ms') \ s\text{-}suc)$ 
  by (auto elim!: set-dist-bindE)
have  $prefix$ :  $\text{Some } (a, s) \in \text{set-dist } (\text{execute } (mfold \ a \ (\text{take } 0 \ ms)) \ s)$ 
  by simp
have  $selected\text{-orig}$ :
   $\text{Some } (out\text{-}j, s\text{-}suc) \in \text{set-dist } (\text{execute } ((ms \ ! \ 0) \ a) \ s)$ 
  using  $head$  unfolding  $ms\text{-eq}$  by simp
have  $suffix\text{-orig}$ :
   $\text{Some } (out, t) \in$ 
     $\text{set-dist } (\text{execute } (mfold \ out\text{-}j \ (\text{drop } (Suc \ 0) \ ms)) \ s\text{-}suc)$ 
  using  $tail$  unfolding  $ms\text{-eq}$  by simp
show  $?case$ 
  by (intro  $exI[of \ - \ a] \ exI[of \ - \ s] \ exI[of \ - \ out\text{-}j]$ 
     $exI[of \ - \ s\text{-}suc] \ conjI$ )
    (use  $prefix \ selected\text{-orig} \ suffix\text{-orig}$  in simp-all)
next
case ( $Suc \ j$ )
then obtain  $m \ ms'$  where  $ms\text{-eq}$ :  $ms = m \# ms'$ 
  by (cases  $ms$ ) auto
from  $Suc.\text{prems}(1)[\text{unfolded } ms\text{-eq}]$ 
obtain  $out1 \ s1$  where
   $head$ :  $\text{Some } (out1, s1) \in \text{set-dist } (\text{execute } (m \ a) \ s)$ 
  and  $tail$ :
     $\text{Some } (out, t) \in \text{set-dist } (\text{execute } (mfold \ out1 \ ms') \ s1)$ 
  by (auto elim!: set-dist-bindE)
have  $j\text{-bound}'$ :  $j < \text{length } ms'$ 
  using  $Suc.\text{prems}(2)$  unfolding  $ms\text{-eq}$  by simp
from  $Suc.IH[OF \ tail \ j\text{-bound}']$ 
obtain  $a\text{-}j \ s\text{-}j \ out\text{-}j \ s\text{-}suc$  where
   $prefix\text{-tail}$ :
     $\text{Some } (a\text{-}j, s\text{-}j) \in$ 
       $\text{set-dist } (\text{execute } (mfold \ out1 \ (\text{take } j \ ms')) \ s1)$ 
  and  $selected$ :
     $\text{Some } (out\text{-}j, s\text{-}suc) \in$ 
       $\text{set-dist } (\text{execute } ((ms' \ ! \ j) \ a\text{-}j) \ s\text{-}j)$ 
  and  $suffix$ :
     $\text{Some } (out, t) \in$ 
       $\text{set-dist } (\text{execute } (mfold \ out\text{-}j \ (\text{drop } (Suc \ j) \ ms')) \ s\text{-}suc)$ 
  by blast
have  $prefix\text{-unfolded}$ :
   $\text{Some } (a\text{-}j, s\text{-}j) \in$ 
     $\text{set-dist } (\text{execute } ((m \ a) \gg (\lambda x. \ mfold \ x \ (\text{take } j \ ms')))) \ s)$ 
  by (rule-tac  $x=out1$  and  $t=s1$  in set-dist-bindI)

```

```

    (use head prefix-tail in simp-all)
  have prefix-mfold:
    Some (a-j, s-j) ∈
      set-dist (execute (mfold a (m # take j ms^) s)
    using prefix-unfolded by simp
  have prefix:
    Some (a-j, s-j) ∈
      set-dist (execute (mfold a (take (Suc j) ms)) s)
    using prefix-mfold unfolding ms-eq by simp
  have selected-orig:
    Some (out-j, s-suc) ∈
      set-dist (execute ((ms ! Suc j) a-j) s-j)
    using selected unfolding ms-eq by simp
  have suffix-orig:
    Some (out, t) ∈
      set-dist (execute (mfold out-j (drop (Suc (Suc j)) ms)) s-suc)
    using suffix unfolding ms-eq by simp
  show ?case
  by (intro exI[of - a-j] exI[of - s-j] exI[of - out-j]
    exI[of - s-suc] conjI)
    (use prefix selected-orig suffix-orig in simp-all)
qed

```

```

lemma mfold-fri-layer-openings-decomp-at:
  fixes s t :: ('f, 'a) protocol-channel-scheme
  assumes outcome:
    Some (out, t) ∈
      set-dist
        (execute
          (mfold (idx, x, len, pw) (map fri-layer-opening-step bfs)) s)
  and j-bound: j < length bfs
  obtains idx-j x-j len-j pw-j out-j s-j s-suc where
    Some ((idx-j, x-j, len-j, pw-j), s-j) ∈
      set-dist
        (execute
          (mfold (idx, x, len, pw)
            (take j (map fri-layer-opening-step bfs)))) s)
    Some (out-j, s-suc) ∈
      set-dist
        (execute
          (fri-layer-opening-step (bfs ! j)
            (idx-j, x-j, len-j, pw-j)) s-j)
    Some (out, t) ∈
      set-dist
        (execute
          (mfold out-j (drop (Suc j) (map fri-layer-opening-step bfs)))
            s-suc)
  proof -
    have mapped-bound: j < length (map fri-layer-opening-step bfs)

```

```

using j-bound by simp
from mfold-outcome-decomp-at[OF outcome mapped-bound]
obtain a-j s-j out-j s-suc where
  prefix:
    Some (a-j, s-j)  $\in$ 
      set-dist
        (execute
          (mfold (idx, x, len, pw)
            (take j (map fri-layer-opening-step bfs)))) s)
and selected:
  Some (out-j, s-suc)  $\in$ 
    set-dist
      (execute
        ((map fri-layer-opening-step bfs ! j) a-j) s-j)
and suffix:
  Some (out, t)  $\in$ 
    set-dist
      (execute
        (mfold out-j (drop (Suc j) (map fri-layer-opening-step bfs)))
          s-suc)
by blast
obtain idx-j x-j len-j pw-j where a-j-eq:
  a-j = (idx-j, x-j, len-j, pw-j)
by (cases a-j)
have selected':
  Some (out-j, s-suc)  $\in$ 
    set-dist
      (execute
        (fri-layer-opening-step (bfs ! j)
          (idx-j, x-j, len-j, pw-j)) s-j)
using selected j-bound unfolding a-j-eq by simp
show ?thesis
by (rule that[OF prefix[unfolded a-j-eq] selected' suffix])
qed

```

```

lemma mfold-fri-layer-openings-prefix-idx-len:
fixes s s-j :: ('f, 'a) protocol-channel-scheme
assumes prefix:
  Some ((idx-j, x-j, len-j, pw-j), s-j)  $\in$ 
    set-dist
      (execute
        (mfold (idx, x, len, pw)
          (take j (map fri-layer-opening-step bfs)))) s)
and j-bound: j  $\leq$  length bfs
shows
  idx-j = fri-layer-indices (Suc j) idx len ! j  $\wedge$ 
  len-j = fri-layer-lengths (Suc j) len ! j
using prefix j-bound
proof (induction j arbitrary: idx x len pw bfs s idx-j x-j len-j pw-j s-j)

```

```

case 0
then show ?case
  by simp
next
case (Suc j)
then obtain bf bfs' where bfs-eq: bfs = bf # bfs'
  by (cases bfs) auto
obtain b rt where bf-eq: bf = (b, rt)
  by (cases bf)
from Suc.premis(1)[unfolded bfs-eq]
obtain out1 s1 where
  head:
    Some (out1, s1) ∈
      set-dist
        (execute (fri-layer-opening-step bf (idx, x, len, pw)) s)
  and tail:
    Some ((idx-j, x-j, len-j, pw-j), s-j) ∈
      set-dist
        (execute
          (mfold out1 (take j (map fri-layer-opening-step bfs')) s1)
        )
  by (auto elim!: set-dist-bindE)
from fri-layer-opening-step-outcome[OF head[unfolded bf-eq]]
obtain x' xp xp-path xn xn-path where out1-eq:
  out1 = (idx mod (len div 2), x', len div 2, pw + pw)
  by blast
have j-bound': j ≤ length bfs'
  using Suc.premis(2) unfolding bfs-eq by simp
have tail-shape:
  idx-j =
    fri-layer-indices (Suc j) (idx mod (len div 2)) (len div 2) ! j ∧
    len-j = fri-layer-lengths (Suc j) (len div 2) ! j
  by (rule Suc.IH[OF tail[unfolded out1-eq] j-bound'])
show ?case
  using tail-shape by simp
qed

lemma mfold-fri-layer-openings-prefix-pow:
fixes s s-j :: ('f, 'a) protocol-channel-scheme
assumes prefix:
  Some ((idx-j, x-j, len-j, pw-j), s-j) ∈
    set-dist
      (execute
        (mfold (idx, x, len, pw)
          (take j (map fri-layer-opening-step bfs))) s)
  and j-bound: j ≤ length bfs
shows pw-j = pw * 2 ^ j
using prefix j-bound
proof (induction j arbitrary: idx x len pw bfs s idx-j x-j len-j pw-j s-j)
case 0

```



```

then show ?case
  by simp
next
case (Suc j)
then obtain bf bfs' where bfs-eq: bfs = bf # bfs'
  by (cases bfs) auto
obtain b rt where bf-eq: bf = (b, rt)
  by (cases bf)
from Suc.premis(1)[unfolded bfs-eq]
obtain out1 s1 where
  head:
    Some (out1, s1) ∈
      set-dist
        (execute (fri-layer-opening-step bf (idx, x, len, pw)) s)
  and tail:
    Some ((idx-j, x-j, len-j, pw-j), s-j) ∈
      set-dist
        (execute
          (mfold out1 (take j (map fri-layer-opening-step bfs'))) s1)
  by (auto elim!: set-dist-bindE)
from fri-layer-opening-step-outcome[OF head[unfolded bf-eq]]
obtain x' xp xp-path xn xn-path where out1-eq:
  out1 = (idx mod (len div 2), x', len div 2, pw + pw)
  by blast
have j-bound': j ≤ length bfs'
  using Suc.premis(2) unfolding bfs-eq by simp
have tail-pow: pw-j = (pw + pw) * 2 ^ j
  by (rule Suc.IH[OF tail[unfolded out1-eq] j-bound'])
then show ?case
  by simp
qed

```

```

lemma mfold-fri-layer-openings-prefix-pow-full:
fixes s s-j :: ('f, 'a) protocol-channel-scheme
assumes prefix:
  Some ((idx-j, x-j, len-j, pw-j), s-j) ∈
    set-dist
      (execute
        (mfold (idx, x, len, pw)
          (take j (map fri-layer-opening-step bfs)))) s)
and j-bound: j < length bfs
shows pw-j = pw * 2 ^ j
by (rule mfold-fri-layer-openings-prefix-pow[OF prefix])
  (use j-bound in simp)

```

```

lemma mfold-fri-layer-openings-prefix-idx-len-full:
fixes s s-j :: ('f, 'a) protocol-channel-scheme
assumes prefix:
  Some ((idx-j, x-j, len-j, pw-j), s-j) ∈

```

```

    set-dist
    (execute
      (mfold (idx, x, len, pw)
        (take j (map fri-layer-opening-step bfs)))) s)
  and j-bound: j < length bfs
shows
  idx-j = fri-layer-indices (length bfs) idx len ! j ∧
  len-j = fri-layer-lengths (length bfs) len ! j
proof -
  have shape:
    idx-j = fri-layer-indices (Suc j) idx len ! j ∧
    len-j = fri-layer-lengths (Suc j) len ! j
  by (rule mfold-fri-layer-openings-prefix-idx-len[OF prefix])
    (use j-bound in simp)
  show ?thesis
  using shape fri-layer-indices-prefix-nth[OF j-bound, of idx len]
    fri-layer-lengths-prefix-nth[OF j-bound, of len]
  by simp
qed

```

lemma *fri-layer-opening-selected-step-authenticated-full:*

fixes *s s-j s-suc* :: ('f, 'a) protocol-channel-scheme

assumes *prefix:*

Some ((idx-j, x-j, len-j, pw-j), s-j) ∈

set-dist

(execute

(mfold (idx, x, len, pw)

(take j (map fri-layer-opening-step bfs)))) s)

and *selected:*

Some (out-j, s-suc) ∈

set-dist

(execute

(fri-layer-opening-step (bfs ! j)

(idx-j, x-j, len-j, pw-j)) s-j)

and *j-bound: j < length bfs*

and *layer-len-pos:*

0 < fri-layer-lengths (length bfs) len ! j

and *layer-idx-bound:*

fri-layer-indices (length bfs) idx len ! j <

fri-layer-lengths (length bfs) len ! j

obtains *chunk where*

fri-layer-chunk-authenticated (snd (bfs ! j))

(fri-layer-lengths (length bfs) len ! j)

(fri-layer-indices (length bfs) idx len ! j)

chunk s-suc

proof -

have *shape:*

idx-j = fri-layer-indices (length bfs) idx len ! j ∧

len-j = fri-layer-lengths (length bfs) len ! j

```

    by (rule mfold-fri-layer-openings-prefix-idx-len-full
        [OF prefix j-bound])
  obtain b rt where bf-eq: bfs ! j = (b, rt)
    by (cases bfs ! j)
  from fri-layer-opening-step-authenticated-chunk-pred
    [OF - - selected[unfolded bf-eq]]
  obtain chunk where auth:
    fri-layer-chunk-authenticated rt len-j idx-j chunk s-suc
    using layer-len-pos layer-idx-bound shape
    by blast
  show ?thesis
    by (rule that[of chunk])
      (use auth shape bf-eq in simp)
qed

```

```

lemma receive-query-commits-decomp-at:
  fixes s t :: ('f, 'a) protocol-channel-scheme
  assumes outcome:
    Some (out, t) ∈
      set-dist
        (execute
          (mfold (idx, x, len, pw) (receive-query-commits bfs)) s)
  and j-bound: j < length bfs
  obtains idx-j x-j len-j pw-j out-j s-j s-suc where
    Some ((idx-j, x-j, len-j, pw-j), s-j) ∈
      set-dist
        (execute
          (mfold (idx, x, len, pw)
            (take j (map fri-layer-opening-step bfs)))) s)
    Some (out-j, s-suc) ∈
      set-dist
        (execute
          (fri-layer-opening-step (bfs ! j)
            (idx-j, x-j, len-j, pw-j)) s-j)
    Some (out, t) ∈
      set-dist
        (execute
          (mfold out-j (drop (Suc j) (map fri-layer-opening-step bfs)))
            s-suc)

```

```

proof -
  have map-eq: receive-query-commits bfs = map fri-layer-opening-step bfs
    unfolding receive-query-commits-def fri-layer-opening-step-def
    by (simp add: fun-eq-iff split: prod.splits)
  show ?thesis
    by (rule mfold-fri-layer-openings-decomp-at
        [OF outcome[unfolded map-eq] j-bound that])
qed

```

```

lemma receive-query-commits-prefix-idx-len:

```

fixes $s \ s\text{-}j :: ('f, 'a) \text{ protocol-channel-scheme}$
assumes prefix :
Some $((\text{id}x\text{-}j, x\text{-}j, \text{len}\text{-}j, \text{pw}\text{-}j), s\text{-}j) \in$
 set-dist
 $(\text{execute}$
 $(\text{mfold } (\text{id}x, x, \text{len}, \text{pw})$
 $(\text{take } j \ (\text{map } \text{fri-layer-opening-step } \text{bfs}))) \ s)$
and $j\text{-bound}$: $j \leq \text{length } \text{bfs}$
shows
 $\text{id}x\text{-}j = \text{fri-layer-indices } (\text{Suc } j) \ \text{id}x \ \text{len} \ ! \ j \wedge$
 $\text{len}\text{-}j = \text{fri-layer-lengths } (\text{Suc } j) \ \text{len} \ ! \ j$
by $(\text{rule } \text{mfold-fri-layer-openings-prefix-id}x\text{-len}[OF \ \text{prefix } j\text{-bound}])$

lemma $\text{receive-query-commits-prefix-pow}$:
fixes $s \ s\text{-}j :: ('f, 'a) \text{ protocol-channel-scheme}$
assumes prefix :
Some $((\text{id}x\text{-}j, x\text{-}j, \text{len}\text{-}j, \text{pw}\text{-}j), s\text{-}j) \in$
 set-dist
 $(\text{execute}$
 $(\text{mfold } (\text{id}x, x, \text{len}, \text{pw})$
 $(\text{take } j \ (\text{map } \text{fri-layer-opening-step } \text{bfs}))) \ s)$
and $j\text{-bound}$: $j \leq \text{length } \text{bfs}$
shows $\text{pw}\text{-}j = \text{pw} * 2^j$
by $(\text{rule } \text{mfold-fri-layer-openings-prefix-pow}[OF \ \text{prefix } j\text{-bound}])$

lemma $\text{receive-query-commits-prefix-pow-full}$:
fixes $s \ s\text{-}j :: ('f, 'a) \text{ protocol-channel-scheme}$
assumes prefix :
Some $((\text{id}x\text{-}j, x\text{-}j, \text{len}\text{-}j, \text{pw}\text{-}j), s\text{-}j) \in$
 set-dist
 $(\text{execute}$
 $(\text{mfold } (\text{id}x, x, \text{len}, \text{pw})$
 $(\text{take } j \ (\text{map } \text{fri-layer-opening-step } \text{bfs}))) \ s)$
and $j\text{-bound}$: $j < \text{length } \text{bfs}$
shows $\text{pw}\text{-}j = \text{pw} * 2^j$
by $(\text{rule } \text{mfold-fri-layer-openings-prefix-pow-full}[OF \ \text{prefix } j\text{-bound}])$

lemma $\text{receive-query-commits-prefix-id}x\text{-len-full}$:
fixes $s \ s\text{-}j :: ('f, 'a) \text{ protocol-channel-scheme}$
assumes prefix :
Some $((\text{id}x\text{-}j, x\text{-}j, \text{len}\text{-}j, \text{pw}\text{-}j), s\text{-}j) \in$
 set-dist
 $(\text{execute}$
 $(\text{mfold } (\text{id}x, x, \text{len}, \text{pw})$
 $(\text{take } j \ (\text{map } \text{fri-layer-opening-step } \text{bfs}))) \ s)$
and $j\text{-bound}$: $j < \text{length } \text{bfs}$
shows
 $\text{id}x\text{-}j = \text{fri-layer-indices } (\text{length } \text{bfs}) \ \text{id}x \ \text{len} \ ! \ j \wedge$
 $\text{len}\text{-}j = \text{fri-layer-lengths } (\text{length } \text{bfs}) \ \text{len} \ ! \ j$

by (rule mfold-fri-layer-openings-prefix-idx-len-full[OF prefix j-bound])

lemma *fri-layer-opening-step-outcome-evidence*:

fixes $s\ t :: ('f, 'a)\ \text{protocol-channel-scheme}$

assumes *outcome*:

$\text{Some } ((\text{next-idx}, \text{next-value}, \text{next-len}, \text{next-pow}), t) \in$
 $\text{set-dist } (\text{execute } (\text{fri-layer-opening-step } (b, rt) (\text{idx}, x, \text{len}, pw))\ s)$

obtains $xp\ xp\text{-path}\ xn\ xn\text{-path}$ **where**

$xp = x$
 $\text{next-idx} = \text{idx mod } (\text{len div } 2)$
 $\text{next-len} = \text{len div } 2$
 $\text{next-pow} = pw + pw$
 $\text{fri-layer-step-evidence } rt\ b\ \text{len}\ \text{idx}\ pw$
 $(\text{fri-sibling-index } \text{len}\ \text{idx})\ xp\ xp\text{-path}\ xn\ xn\text{-path}$
 $\text{next-idx}\ \text{next-value } ([xp] @ xp\text{-path} @ [xn] @ xn\text{-path})$

proof –

from *fri-layer-opening-step-outcome*[OF *outcome*]

obtain $xp\ xp\text{-path}\ xn\ xn\text{-path}\ x'$ **where** *out-eq*:

$(\text{next-idx}, \text{next-value}, \text{next-len}, \text{next-pow}) =$
 $(\text{idx mod } (\text{len div } 2), x', \text{len div } 2, pw + pw)$

and *xp-eq*: $xp = x$

and *x'-eq*:

$x' = \text{fri-fold-value } b\ xp\ xn$
 $(\text{fri-fold-denominator } ((h \wedge \text{idx}) * \text{shift})\ pw)$

and *chunk*:

$\text{fri-layer-opening-chunk } \text{len}\ xp\ xp\text{-path}\ xn\ xn\text{-path}$
 $([xp] @ xp\text{-path} @ [xn] @ xn\text{-path})$

by *blast*

have *next-idx-eq*: $\text{next-idx} = \text{idx mod } (\text{len div } 2)$

using *out-eq* **by** *simp*

have *next-value-eq*: $\text{next-value} = x'$

using *out-eq* **by** *simp*

have *next-len-eq*: $\text{next-len} = \text{len div } 2$

using *out-eq* **by** *simp*

have *next-pow-eq*: $\text{next-pow} = pw + pw$

using *out-eq* **by** *simp*

have *evidence*:

$\text{fri-layer-step-evidence } rt\ b\ \text{len}\ \text{idx}\ pw$
 $(\text{fri-sibling-index } \text{len}\ \text{idx})\ xp\ xp\text{-path}\ xn\ xn\text{-path}$
 $\text{next-idx}\ \text{next-value } ([xp] @ xp\text{-path} @ [xn] @ xn\text{-path})$

by (rule *fri-layer-step-evidenceI*)

(use *next-idx-eq next-value-eq x'-eq chunk* **in** *simp-all*)

show *?thesis*

by (rule *that*[OF *xp-eq next-idx-eq next-len-eq next-pow-eq evidence*])

qed

lemma *receive-query-commits-last-step-evidence*:

fixes $s\ t :: ('f, 'a)\ \text{protocol-channel-scheme}$

assumes *outcome*:

```

Some ((out-idx, out-value, out-len, out-pow), t) ∈
  set-dist
  (execute
    (mfold (idx, x, len, pw) (receive-query-commits bfs)) s)
and nonempty: 0 < length bfs
obtains j idx-j x-j len-j pw-j b rt xp xp-path xn xn-path s-j
  s-suc where
  j = length bfs - 1
  j < length bfs
  bfs ! j = (b, rt)
  Some ((idx-j, x-j, len-j, pw-j), s-j) ∈
    set-dist
    (execute
      (mfold (idx, x, len, pw)
        (take j (map fri-layer-opening-step bfs))) s)
  Some ((out-idx, out-value, out-len, out-pow), s-suc) ∈
    set-dist
    (execute
      (fri-layer-opening-step (b, rt)
        (idx-j, x-j, len-j, pw-j)) s-j)
  t = s-suc
  idx-j = fri-layer-indices (length bfs) idx len ! j
  len-j = fri-layer-lengths (length bfs) len ! j
  pw-j = pw * 2 ^ j
  fri-layer-step-evidence rt b len-j idx-j pw-j
  (fri-sibling-index len-j idx-j) xp xp-path xn xn-path
  out-idx out-value ([xp] @ xp-path @ [xn] @ xn-path)
proof -
let ?j = length bfs - 1
have j-bound: ?j < length bfs
using nonempty by simp
show ?thesis
proof (rule receive-query-commits-decomp-at[OF outcome j-bound])
fix idx-j x-j len-j pw-j out-j s-j s-suc
assume prefix:
  Some ((idx-j, x-j, len-j, pw-j), s-j) ∈
    set-dist
    (execute
      (mfold (idx, x, len, pw)
        (take ?j (map fri-layer-opening-step bfs))) s)
and selected:
  Some (out-j, s-suc) ∈
    set-dist
    (execute
      (fri-layer-opening-step (bfs ! ?j)
        (idx-j, x-j, len-j, pw-j)) s-j)
and suffix:
  Some ((out-idx, out-value, out-len, out-pow), t) ∈
    set-dist

```

```

      (execute
        (mfold out-j
          (drop (Suc ?j) (map fri-layer-opening-step bfs))) s-suc)
have drop-empty:
  drop (Suc ?j) (map fri-layer-opening-step bfs) = []
  using nonempty by simp
have out-j-eq: out-j = (out-idx, out-value, out-len, out-pow)
  and t-eq: t = s-suc
  using suffix unfolding drop-empty by simp-all
obtain b rt where bf-eq: bfs ! ?j = (b, rt)
  by (cases bfs ! ?j)
have shape:
  idx-j = fri-layer-indices (length bfs) idx len ! ?j ∧
  len-j = fri-layer-lengths (length bfs) len ! ?j
  by (rule receive-query-commits-prefix-idx-len-full
    [OF prefix j-bound])
have pow: pw-j = pw * 2 ^ ?j
  by (rule receive-query-commits-prefix-pow-full[OF prefix j-bound])
show ?thesis
proof (rule fri-layer-opening-step-outcome-evidence
  [OF selected[unfolded bf-eq out-j-eq]])
  fix xp xp-path xn xn-path
  assume
    xp = x-j
    out-idx = idx-j mod (len-j div 2)
    out-len = len-j div 2
    out-pow = pw-j + pw-j
  and step:
    fri-layer-step-evidence rt b len-j idx-j pw-j
    (fri-sibling-index len-j idx-j) xp xp-path xn xn-path
    out-idx out-value ([xp] @ xp-path @ [xn] @ xn-path)
  show ?thesis
  by (rule that[OF refl j-bound bf-eq prefix
    selected[unfolded bf-eq out-j-eq] t-eq conjunct1[OF shape]
    conjunct2[OF shape] pow step])
  qed
qed
qed

lemma verifier-query-round-after-index-trace-fri-last-step-evidence:
fixes s t :: ('f, 'a) protocol-channel-scheme
assumes outcome:
  Some (( ), t) ∈
  set-dist
  (execute
    (verifier-query-round-after-index-program
      fr f-fl f-final as fl final raw) s)
and nonempty: 0 < length f-fl
and raw-layer:

```

$\wedge \text{raw}'.$
 $0 < \text{fri-layer-lengths } (\text{length } f\text{-fl}) (\text{clength} * \text{scale}) !$
 $(\text{length } f\text{-fl} - 1) \wedge$
 $\text{fri-layer-indices } (\text{length } f\text{-fl}) (\text{index } (\text{to-nat } \text{raw}'))$
 $(\text{clength} * \text{scale}) ! (\text{length } f\text{-fl} - 1) <$
 $\text{fri-layer-lengths } (\text{length } f\text{-fl}) (\text{clength} * \text{scale}) !$
 $(\text{length } f\text{-fl} - 1)$
obtains *actual-chunk* *axp* *axp-path* *axn* *axn-path* **where**
fri-layer-chunk-authenticated
 $(\text{snd } (f\text{-fl} ! (\text{length } f\text{-fl} - 1)))$
 $(\text{fri-layer-lengths } (\text{length } f\text{-fl}) (\text{clength} * \text{scale}) !$
 $(\text{length } f\text{-fl} - 1))$
 $(\text{fri-layer-indices } (\text{length } f\text{-fl}) (\text{index } (\text{to-nat } \text{raw})))$
 $(\text{clength} * \text{scale}) ! (\text{length } f\text{-fl} - 1))$
actual-chunk *t*
fri-layer-step-evidence
 $(\text{snd } (f\text{-fl} ! (\text{length } f\text{-fl} - 1)))$
 $(\text{fst } (f\text{-fl} ! (\text{length } f\text{-fl} - 1)))$
 $(\text{fri-layer-lengths } (\text{length } f\text{-fl}) (\text{clength} * \text{scale}) !$
 $(\text{length } f\text{-fl} - 1))$
 $(\text{fri-layer-indices } (\text{length } f\text{-fl}) (\text{index } (\text{to-nat } \text{raw})))$
 $(\text{clength} * \text{scale}) ! (\text{length } f\text{-fl} - 1))$
 $(2 \wedge (\text{length } f\text{-fl} - 1))$
fri-sibling-index
 $(\text{fri-layer-lengths } (\text{length } f\text{-fl}) (\text{clength} * \text{scale}) !$
 $(\text{length } f\text{-fl} - 1))$
 $(\text{fri-layer-indices } (\text{length } f\text{-fl}) (\text{index } (\text{to-nat } \text{raw})))$
 $(\text{clength} * \text{scale}) ! (\text{length } f\text{-fl} - 1)))$
axp *axp-path* *axn* *axn-path*
 $((\text{fri-layer-indices } (\text{length } f\text{-fl}) (\text{index } (\text{to-nat } \text{raw})))$
 $(\text{clength} * \text{scale}) ! (\text{length } f\text{-fl} - 1)) \bmod$
 $((\text{fri-layer-lengths } (\text{length } f\text{-fl}) (\text{clength} * \text{scale}) !$
 $(\text{length } f\text{-fl} - 1)) \text{ div } 2))$
f-final *actual-chunk*
proof –
show *?thesis*
proof (*rule verifier-query-round-after-index-final-checks*[*OF* *outcome*])
fix *idx* *fv* *s1* *f-i* *f-len* *f-pow* *s2* *c-i* *c-len* *c-pow* *s4*
assume *idx-eq*: *idx* = *index* (*to-nat* *raw*)
and *trace-fri*:
Some ((*f-i*, *f-final*, *f-len*, *f-pow*), *s2*) ∈
set-dist
(execute
(mfold (*idx*, *hd* *fv*, *clength* * *scale*, 1)
(receive-query-commits *f-fl*)) *s1*)
and *comp-fri*:
Some ((*c-i*, *final*, *c-len*, *c-pow*), *s4*) ∈
set-dist
(execute


```

      (mfold (idx, cp-eval as fv (h ^ idx * shift),
        clength * scale, 1)
        (receive-query-commits fl)) s2)
  and t-eq: t = s4
let ?j = length f-fl - 1
let ?len = fri-layer-lengths (length f-fl) (clength * scale) ! ?j
let ?idx =
  fri-layer-indices (length f-fl) (index (to-nat raw))
  (clength * scale) ! ?j
have len-pos: 0 < ?len
  using raw-layer[of raw] by simp
have idx-bound: ?idx < ?len
  using raw-layer[of raw] by simp
show ?thesis
proof (rule receive-query-commits-last-step-evidence
  [OF trace-fri[unfolded idx-eq] nonempty])
fix j idx-j x-j len-j pw-j b rt xp xp-path xn xn-path s-j s-suc
assume j-eq: j = ?j
  and bf-eq: f-fl ! j = (b, rt)
  and selected:
    Some ((f-i, f-final, f-len, f-pow), s-suc) ∈
      set-dist
      (execute
        (fri-layer-opening-step (b, rt)
          (idx-j, x-j, len-j, pw-j)) s-j)
  and s2-eq: s2 = s-suc
  and idx-j-eq:
    idx-j =
      fri-layer-indices (length f-fl) (index (to-nat raw))
      (clength * scale) ! j
  and len-j-eq:
    len-j =
      fri-layer-lengths (length f-fl) (clength * scale) ! j
  and pw-j-eq: pw-j = 1 * 2 ^ j
have len-pos-j: 0 < len-j
  using len-pos len-j-eq j-eq by simp
have idx-bound-j: idx-j < len-j
  using idx-bound idx-j-eq len-j-eq j-eq by simp
show ?thesis
proof (rule fri-layer-opening-step-outcome-authenticated
  [OF len-pos-j idx-bound-j selected])
fix axp axp-path axn axn-path next-value actual-chunk
assume selected-out:
  (f-i, f-final, f-len, f-pow) =
    (idx-j mod (len-j div 2), next-value, len-j div 2,
      pw-j + pw-j)
  and next-value-eq:
    next-value =
      fri-fold-value b axp axn

```

```

      (fri-fold-denominator ((h ^ idx-j) * shift) pw-j)
    and chunk:
      fri-layer-opening-chunk len-j axp axp-path axn axn-path
      actual-chunk
    and axp-auth:
      authenticated-opening-in s-suc
      (opening-root = rt, opening-length = len-j,
       opening-index = idx-j, opening-value = axp,
       opening-path = axp-path)
    and axn-auth:
      authenticated-opening-in s-suc
      (opening-root = rt, opening-length = len-j,
       opening-index = fri-sibling-index len-j idx-j,
       opening-value = axn, opening-path = axn-path)
  have f-final-eq: f-final = next-value
    using selected-out by simp
  have auth-s2:
    fri-layer-chunk-authenticated rt len-j idx-j actual-chunk s2
    unfolding s2-eq
    by (rule fri-layer-chunk-authenticatedI[OF chunk axp-auth axn-auth])
  have s2-t: s2 ≤ t
    using receive-query-commits-mfold-hash-extends[OF comp-fri] t-eq
    by simp
  have auth-t:
    fri-layer-chunk-authenticated rt len-j idx-j actual-chunk t
    by (rule fri-layer-chunk-authenticated-mono[OF auth-s2 s2-t])
  have step:
    fri-layer-step-evidence rt b len-j idx-j pw-j
    (fri-sibling-index len-j idx-j) axp axp-path axn axn-path
    (idx-j mod (len-j div 2)) f-final actual-chunk
    by (rule fri-layer-step-evidenceI)
    (use next-value-eq f-final-eq chunk in simp-all)
  have step':
    fri-layer-step-evidence
    (snd (f-fl ! ?j)) (fst (f-fl ! ?j)) ?len ?idx (2 ^ ?j)
    (fri-sibling-index ?len ?idx) axp axp-path axn axn-path
    (?idx mod (?len div 2)) f-final actual-chunk
    using step bf-eq idx-j-eq len-j-eq pw-j-eq j-eq by simp
  show ?thesis
    by (rule that[OF - step'])
    (use auth-t bf-eq idx-j-eq len-j-eq j-eq in simp)
qed
qed
qed
qed

```

lemma *verifier-query-round-after-index-composition-fri-last-step-evidence:*
fixes $s\ t :: ('f, 'a)\ \text{protocol-channel-scheme}$
assumes *outcome:*

```

Some (((), t) ∈
  set-dist
  (execute
    (verifier-query-round-after-index-program
      fr f-fl f-final as fl final raw) s)
and nonempty: 0 < length fl
and raw-layer:
  ∧ raw'.
    0 < fri-layer-lengths (length fl) (clength * scale) !
    (length fl - 1) ∧
    fri-layer-indices (length fl) (index (to-nat raw'))
    (clength * scale) ! (length fl - 1) <
    fri-layer-lengths (length fl) (clength * scale) !
    (length fl - 1)
obtains actual-chunk axp axp-path axn axn-path where
  fri-layer-chunk-authenticated
    (snd (fl ! (length fl - 1)))
    (fri-layer-lengths (length fl) (clength * scale) !
      (length fl - 1))
    (fri-layer-indices (length fl) (index (to-nat raw)))
    (clength * scale) ! (length fl - 1))
  actual-chunk t
  fri-layer-step-evidence
    (snd (fl ! (length fl - 1)))
    (fst (fl ! (length fl - 1)))
    (fri-layer-lengths (length fl) (clength * scale) !
      (length fl - 1))
    (fri-layer-indices (length fl) (index (to-nat raw)))
    (clength * scale) ! (length fl - 1))
    (2 ^ (length fl - 1))
    (fri-sibling-index
      (fri-layer-lengths (length fl) (clength * scale) !
        (length fl - 1))
      (fri-layer-indices (length fl) (index (to-nat raw)))
      (clength * scale) ! (length fl - 1)))
  axp axp-path axn axn-path
    ((fri-layer-indices (length fl) (index (to-nat raw)))
      (clength * scale) ! (length fl - 1)) mod
      ((fri-layer-lengths (length fl) (clength * scale) !
        (length fl - 1)) div 2))
  final actual-chunk
proof -
  show ?thesis
proof (rule verifier-query-round-after-index-final-checks[OF outcome])
  fix idx fv s1 f-i f-len f-pow s2 c-i c-len c-pow s4
  assume idx-eq: idx = index (to-nat raw)
  and comp-fri:
    Some ((c-i, final, c-len, c-pow), s4) ∈
      set-dist

```

```

      (execute
        (mfold (idx, cp-eval as fv (h ^ idx * shift),
          clength * scale, 1)
          (receive-query-commits fl)) s2)
    and t-eq: t = s4
  let ?j = length fl - 1
  let ?len = fri-layer-lengths (length fl) (clength * scale) ! ?j
  let ?idx =
    fri-layer-indices (length fl) (index (to-nat raw))
    (clength * scale) ! ?j
  have len-pos: 0 < ?len
    using raw-layer[of raw] by simp
  have idx-bound: ?idx < ?len
    using raw-layer[of raw] by simp
  show ?thesis
  proof (rule receive-query-commits-last-step-evidence
    [OF comp-fri[unfolded idx-eq] nonempty])
    fix j idx-j x-j len-j pw-j b rt xp xp-path xn xn-path s-j s-suc
    assume j-eq: j = ?j
    and bf-eq: fl ! j = (b, rt)
    and selected:
      Some ((c-i, final, c-len, c-pow), s-suc) ∈
        set-dist
        (execute
          (fri-layer-opening-step (b, rt)
            (idx-j, x-j, len-j, pw-j)) s-j)
    and s4-eq: s4 = s-suc
    and idx-j-eq:
      idx-j =
        fri-layer-indices (length fl) (index (to-nat raw))
        (clength * scale) ! j
    and len-j-eq:
      len-j =
        fri-layer-lengths (length fl) (clength * scale) ! j
    and pw-j-eq: pw-j = 1 * 2 ^ j
  have len-pos-j: 0 < len-j
    using len-pos len-j-eq j-eq by simp
  have idx-bound-j: idx-j < len-j
    using idx-bound idx-j-eq len-j-eq j-eq by simp
  show ?thesis
  proof (rule fri-layer-opening-step-outcome-authenticated
    [OF len-pos-j idx-bound-j selected])
    fix axp axp-path axn axn-path next-value actual-chunk
    assume selected-out:
      (c-i, final, c-len, c-pow) =
        (idx-j mod (len-j div 2), next-value, len-j div 2,
          pw-j + pw-j)
    and next-value-eq:
      next-value =

```

```

    fri-fold-value b axp axn
    (fri-fold-denominator ((h ^ idx-j) * shift) pw-j)
  and chunk:
    fri-layer-opening-chunk len-j axp axp-path axn axn-path
    actual-chunk
  and axp-auth:
    authenticated-opening-in s-suc
    (opening-root = rt, opening-length = len-j,
     opening-index = idx-j, opening-value = axp,
     opening-path = axp-path)
  and axn-auth:
    authenticated-opening-in s-suc
    (opening-root = rt, opening-length = len-j,
     opening-index = fri-sibling-index len-j idx-j,
     opening-value = axn, opening-path = axn-path)
  have final-eq: final = next-value
  using selected-out by simp
  have auth-t:
    fri-layer-chunk-authenticated rt len-j idx-j actual-chunk t
  unfolding t-eq s4-eq
  by (rule fri-layer-chunk-authenticatedI[OF chunk axp-auth axn-auth])
  have step:
    fri-layer-step-evidence rt b len-j idx-j pw-j
    (fri-sibling-index len-j idx-j) axp axp-path axn axn-path
    (idx-j mod (len-j div 2)) final actual-chunk
  by (rule fri-layer-step-evidenceI)
    (use next-value-eq final-eq chunk in simp-all)
  have step':
    fri-layer-step-evidence
    (snd (fl ! ?j)) (fst (fl ! ?j)) ?len ?idx (2 ^ ?j)
    (fri-sibling-index ?len ?idx) axp axp-path axn axn-path
    (?idx mod (?len div 2)) final actual-chunk
  using step bf-eq idx-j-eq len-j-eq pw-j-eq j-eq by simp
  show ?thesis
  by (rule that[OF - step'])
    (use auth-t bf-eq idx-j-eq len-j-eq j-eq in simp)
qed
qed
qed
qed

```

lemma *ntimes-verifier-query-rounds-selected-trace-fri-last-step-with-prefix-at:*
fixes query-state final-state :: ('f, 'a) protocol-channel-scheme
assumes outcome:
 Some (results, final-state) ∈
 set-dist
 (execute
 (ntimes
 (verifier-query-round-program fr f-fl f-final as fl final))

```

    rounds)
    query-state)
and i-bound:  $i < \text{rounds}$ 
and nonempty:  $0 < \text{length } f\text{-fl}$ 
and raw-layer:
   $\bigwedge_{\text{raw.}}$ 
     $0 < \text{fri-layer-lengths } (\text{length } f\text{-fl}) (\text{clength} * \text{scale}) !$ 
     $(\text{length } f\text{-fl} - 1) \wedge$ 
     $\text{fri-layer-indices } (\text{length } f\text{-fl}) (\text{index } (\text{to-nat } \text{raw}))$ 
     $(\text{clength} * \text{scale}) ! (\text{length } f\text{-fl} - 1) <$ 
     $\text{fri-layer-lengths } (\text{length } f\text{-fl}) (\text{clength} * \text{scale}) !$ 
     $(\text{length } f\text{-fl} - 1)$ 
obtains raw idx prefix-query-idxs prefix-chunks chunk suffix-query-idxs
  suffix-chunks actual-chunk axp axp-path axn axn-path where
    idx = index (to-nat raw)
    length prefix-query-idxs = i
    length prefix-chunks = i
    length suffix-query-idxs = rounds - Suc i
    length suffix-chunks = rounds - Suc i
    verifier-query-round-chunk idx (map snd f-fl) (map snd fl) chunk
     $\bigwedge k. k < i \implies$ 
      verifier-query-round-chunk (prefix-query-idxs ! k)
      (map snd f-fl) (map snd fl) (prefix-chunks ! k)
     $\bigwedge k. k < \text{rounds} - \text{Suc } i \implies$ 
      verifier-query-round-chunk (suffix-query-idxs ! k)
      (map snd f-fl) (map snd fl) (suffix-chunks ! k)
    PTranscript query-state =
      List.concat (prefix-chunks @ chunk # suffix-chunks) @
      PTranscript final-state
    fri-layer-chunk-authenticated
      (snd (f-fl ! (length f-fl - 1)))
      (fri-layer-lengths (length f-fl) (clength * scale) !
      (length f-fl - 1))
      (fri-layer-indices (length f-fl) idx (clength * scale) !
      (length f-fl - 1))
      actual-chunk final-state
    fri-layer-step-evidence
      (snd (f-fl ! (length f-fl - 1)))
      (fst (f-fl ! (length f-fl - 1)))
      (fri-layer-lengths (length f-fl) (clength * scale) !
      (length f-fl - 1))
      (fri-layer-indices (length f-fl) idx (clength * scale) !
      (length f-fl - 1))
      ( $2 \wedge (\text{length } f\text{-fl} - 1)$ )
      (fri-sibling-index
        (fri-layer-lengths (length f-fl) (clength * scale) !
        (length f-fl - 1))
        (fri-layer-indices (length f-fl) idx (clength * scale) !
        (length f-fl - 1)))

```

```

    axp axp-path axn axn-path
    ((fri-layer-indices (length f-fl) idx (clength * scale) !
      (length f-fl - 1)) mod
      ((fri-layer-lengths (length f-fl) (clength * scale) !
        (length f-fl - 1)) div 2))
    f-final actual-chunk
    fmlookup (HashMap final-state)
      (QueryIndexChallenge (PQueryCounter query-state + i)
        (state-after-query-chunks (PState query-state) prefix-chunks i)) =
    Some raw
proof -
  let ?round = verifier-query-round-program fr f-fl f-final as fl final
  from query-opening-ntimes-outcome-decomp-at[OF outcome i-bound]
  obtain prefix x suffix s-i s-suc where
    prefix:
      Some (prefix, s-i) ∈
        set-dist (execute (ntimes ?round i) query-state)
    and round:
      Some (x, s-suc) ∈ set-dist (execute ?round s-i)
    and suffix:
      Some (suffix, final-state) ∈
        set-dist (execute (ntimes ?round (rounds - Suc i)) s-suc)
  by blast
  have x-eq: x = ()
  by (cases x) simp
  show ?thesis
  proof (rule query-opening-ntimes-prefix-transcript-state-alignment
    [OF prefix])
    fix prefix-query-idxs prefix-chunks
    assume len-prefix-idxs: length prefix-query-idxs = i
    and len-prefix-chunks: length prefix-chunks = i
    and transcript-prefix:
      PTranscript query-state =
        List.concat prefix-chunks @ PTranscript s-i
    and state-i:
      PState s-i =
        state-after-query-chunks (PState query-state) prefix-chunks i
    and counter-i: PQueryCounter s-i = PQueryCounter query-state + i
    and prefix-rounds:
       $\bigwedge k. k < i \implies$ 
        verifier-query-round-chunk (prefix-query-idxs ! k)
          (map snd f-fl) (map snd fl) (prefix-chunks ! k)
    from round[unfolded x-eq verifier-query-round-program-alt-def]
    obtain raw s0 where
      raw-out:
        Some (raw, s0) ∈
          set-dist (execute receive-query-index-challenge s-i)
    and after:
      Some ((), s-suc) ∈

```

```

    set-dist
      (execute
        (verifier-query-round-after-index-program fr f-fl f-final as
          fl final raw) s0)
  by (auto elim!: set-dist-bindE)
have lookup-s0:
  fmlookup (HashMap s0)
    (QueryIndexChallenge (PQueryCounter s-i) (PState s-i)) =
  Some raw
  using receive-query-index-challenge-outcome[OF raw-out] by simp
show ?thesis
proof (rule verifier-query-round-program-outcome[OF round[unfolded x-eq]])
  fix raw' idx chunk
  assume idx-eq: idx = index (to-nat raw')
  and chunk-shape:
    verifier-query-round-chunk idx (map snd f-fl) (map snd fl)
    chunk
  and transcript-round:
    PTranscript s-i = chunk @ PTranscript s-suc
  and lookup-suc:
    fmlookup (HashMap s-suc)
      (QueryIndexChallenge (PQueryCounter s-i) (PState s-i)) =
    Some raw'
  have s0-suc: s0 ≤ s-suc
    using verifier-query-round-after-index-program-outcome[OF after]
    by simp
  have lookup-suc-raw:
    fmlookup (HashMap s-suc)
      (QueryIndexChallenge (PQueryCounter s-i) (PState s-i)) =
    Some raw
    by (rule hash-extension-lookup[OF lookup-s0 s0-suc])
  have raw-eq': raw = raw'
    using lookup-suc lookup-suc-raw by simp
  from verifier-query-round-after-index-trace-fri-last-step-evidence
    [OF after nonempty raw-layer]
  obtain actual-chunk axp axp-path axn axn-path where
    auth-suc:
      fri-layer-chunk-authenticated
        (snd (f-fl ! (length f-fl - 1)))
        (fri-layer-lengths (length f-fl) (clength * scale) !
          (length f-fl - 1))
        (fri-layer-indices (length f-fl) (index (to-nat raw))
          (clength * scale) ! (length f-fl - 1))
        actual-chunk s-suc
  and step-raw:
    fri-layer-step-evidence
      (snd (f-fl ! (length f-fl - 1)))
      (fst (f-fl ! (length f-fl - 1)))
      (fri-layer-lengths (length f-fl) (clength * scale) !

```



```

    (length f-fl - 1))
  (fri-layer-indices (length f-fl) (index (to-nat raw))
    (clength * scale) ! (length f-fl - 1))
  (2 ^ (length f-fl - 1))
  (fri-sibling-index
    (fri-layer-lengths (length f-fl) (clength * scale) !
      (length f-fl - 1))
    (fri-layer-indices (length f-fl) (index (to-nat raw))
      (clength * scale) ! (length f-fl - 1)))
  axp axp-path axn axn-path
  ((fri-layer-indices (length f-fl) (index (to-nat raw))
    (clength * scale) ! (length f-fl - 1)) mod
    ((fri-layer-lengths (length f-fl) (clength * scale) !
      (length f-fl - 1)) div 2))
  f-final actual-chunk
by blast
show ?thesis
proof (rule query-opening-ntimes-suffix-transcript-state-alignment
  [OF suffix])
fix suffix-query-idxs suffix-chunks
assume len-suffix-idxs:
  length suffix-query-idxs = rounds - Suc i
and len-suffix-chunks:
  length suffix-chunks = rounds - Suc i
and transcript-suffix:
  PTranscript s-suc =
    List.concat suffix-chunks @ PTranscript final-state
and suffix-ext: s-suc ≤ final-state
and suffix-rounds:
  ∧k. k < rounds - Suc i ⇒
    verifier-query-round-chunk (suffix-query-idxs ! k)
      (map snd f-fl) (map snd fl) (suffix-chunks ! k)
have transcript-all:
  PTranscript query-state =
    List.concat (prefix-chunks @ chunk # suffix-chunks) @
      PTranscript final-state
using transcript-prefix transcript-round transcript-suffix by simp
have auth-final-raw:
  fri-layer-chunk-authenticated
    (snd (f-fl ! (length f-fl - 1)))
    (fri-layer-lengths (length f-fl) (clength * scale) !
      (length f-fl - 1))
    (fri-layer-indices (length f-fl) (index (to-nat raw))
      (clength * scale) !
      (length f-fl - 1))
    actual-chunk final-state
by (rule fri-layer-chunk-authenticated-mono[OF auth-suc suffix-ext])
have auth-final:
  fri-layer-chunk-authenticated

```

```

    (snd (f-fl ! (length f-fl - 1)))
    (fri-layer-lengths (length f-fl) (clength * scale) !
      (length f-fl - 1))
    (fri-layer-indices (length f-fl) idx (clength * scale) !
      (length f-fl - 1))
    actual-chunk final-state
  using auth-final-raw raw-eq' idx-eq by simp
have step-idx:
  fri-layer-step-evidence
  (snd (f-fl ! (length f-fl - 1)))
  (fst (f-fl ! (length f-fl - 1)))
  (fri-layer-lengths (length f-fl) (clength * scale) !
    (length f-fl - 1))
  (fri-layer-indices (length f-fl) idx (clength * scale) !
    (length f-fl - 1))
  (2 ^ (length f-fl - 1))
  (fri-sibling-index
    (fri-layer-lengths (length f-fl) (clength * scale) !
      (length f-fl - 1))
    (fri-layer-indices (length f-fl) idx (clength * scale) !
      (length f-fl - 1)))
  axp axp-path axn axn-path
  ((fri-layer-indices (length f-fl) idx (clength * scale) !
    (length f-fl - 1)) mod
    ((fri-layer-lengths (length f-fl) (clength * scale) !
      (length f-fl - 1)) div 2))
  f-final actual-chunk
  using step-raw raw-eq' idx-eq by simp
have lookup-final:
  fmlookup (HashMap final-state)
  (QueryIndexChallenge (PQueryCounter query-state + i)
    (state-after-query-chunks (PState query-state)
      prefix-chunks i)) =
  Some raw
proof -
  have lookup-suc':
    fmlookup (HashMap s-suc)
    (QueryIndexChallenge (PQueryCounter query-state + i)
      (state-after-query-chunks (PState query-state)
        prefix-chunks i)) =
    Some raw
  using lookup-suc-raw state-i counter-i by simp
show ?thesis
  by (rule hash-extension-lookup[OF lookup-suc' suffix-ext])
qed
show ?thesis
  by (rule that[OF - len-prefix-idxs len-prefix-chunks
    len-suffix-idxs len-suffix-chunks chunk-shape prefix-rounds
    suffix-rounds transcript-all auth-final step-idx])

```

```

        lookup-final]])
      (use idx-eq raw-eq' in simp)
    qed
  qed
  qed
  qed

lemma ntimes-verifier-query-rounds-selected-composition-fri-last-step-with-prefix-at:
fixes query-state final-state :: ('f, 'a) protocol-channel-scheme
assumes outcome:
  Some (results, final-state) ∈
    set-dist
      (execute
        (ntimes
          (verifier-query-round-program fr f-fl f-final as fl final)
          rounds)
        query-state)
and i-bound:  $i < \text{rounds}$ 
and nonempty:  $0 < \text{length } fl$ 
and raw-layer:
   $\bigwedge \text{raw.}$ 
     $0 < \text{fri-layer-lengths } (\text{length } fl) (\text{clength} * \text{scale}) !$ 
     $(\text{length } fl - 1) \wedge$ 
     $\text{fri-layer-indices } (\text{length } fl) (\text{index } (\text{to-nat } \text{raw}))$ 
     $(\text{clength} * \text{scale}) ! (\text{length } fl - 1) <$ 
     $\text{fri-layer-lengths } (\text{length } fl) (\text{clength} * \text{scale}) !$ 
     $(\text{length } fl - 1)$ 
obtains raw idx prefix-query-idxs prefix-chunks chunk suffix-query-idxs
  suffix-chunks actual-chunk axp axp-path axn axn-path where
  idx =  $\text{index } (\text{to-nat } \text{raw})$ 
   $\text{length } \text{prefix-query-idxs} = i$ 
   $\text{length } \text{prefix-chunks} = i$ 
   $\text{length } \text{suffix-query-idxs} = \text{rounds} - \text{Suc } i$ 
   $\text{length } \text{suffix-chunks} = \text{rounds} - \text{Suc } i$ 
   $\text{verifier-query-round-chunk } \text{idx } (\text{map } \text{snd } f\text{-fl}) (\text{map } \text{snd } fl) \text{ chunk}$ 
   $\bigwedge k. k < i \implies$ 
     $\text{verifier-query-round-chunk } (\text{prefix-query-idxs } ! k)$ 
     $(\text{map } \text{snd } f\text{-fl}) (\text{map } \text{snd } fl) (\text{prefix-chunks } ! k)$ 
   $\bigwedge k. k < \text{rounds} - \text{Suc } i \implies$ 
     $\text{verifier-query-round-chunk } (\text{suffix-query-idxs } ! k)$ 
     $(\text{map } \text{snd } f\text{-fl}) (\text{map } \text{snd } fl) (\text{suffix-chunks } ! k)$ 
   $P\text{Transcript } \text{query-state} =$ 
     $\text{List.concat } (\text{prefix-chunks } @ \text{chunk } \# \text{suffix-chunks}) @$ 
     $P\text{Transcript } \text{final-state}$ 
  fri-layer-chunk-authenticated
     $(\text{snd } (fl ! (\text{length } fl - 1)))$ 
     $(\text{fri-layer-lengths } (\text{length } fl) (\text{clength} * \text{scale}) !$ 
     $(\text{length } fl - 1))$ 
     $(\text{fri-layer-indices } (\text{length } fl) \text{ idx } (\text{clength} * \text{scale}) !$ 

```

```

    (length fl - 1))
  actual-chunk final-state
fri-layer-step-evidence
  (snd (fl ! (length fl - 1)))
  (fst (fl ! (length fl - 1)))
  (fri-layer-lengths (length fl) (clength * scale) !
    (length fl - 1))
  (fri-layer-indices (length fl) idx (clength * scale) !
    (length fl - 1))
  (2 ^ (length fl - 1))
  (fri-sibling-index
    (fri-layer-lengths (length fl) (clength * scale) !
      (length fl - 1))
    (fri-layer-indices (length fl) idx (clength * scale) !
      (length fl - 1)))
  arp arp-path arn arn-path
  ((fri-layer-indices (length fl) idx (clength * scale) !
    (length fl - 1)) mod
    ((fri-layer-lengths (length fl) (clength * scale) !
      (length fl - 1)) div 2))
  final actual-chunk
fmlookup (HashMap final-state)
  (QueryIndexChallenge (PQueryCounter query-state + i)
    (state-after-query-chunks (PState query-state) prefix-chunks i)) =
  Some raw
proof -
let ?round = verifier-query-round-program fr f-fl f-final as fl final
from query-opening-ntimes-outcome-decomp-at[OF outcome i-bound]
obtain prefix x suffix s-i s-suc where
  prefix:
    Some (prefix, s-i) ∈
      set-dist (execute (ntimes ?round i) query-state)
and round:
    Some (x, s-suc) ∈ set-dist (execute ?round s-i)
and suffix:
    Some (suffix, final-state) ∈
      set-dist (execute (ntimes ?round (rounds - Suc i)) s-suc)
by blast
have x-eq: x = ()
by (cases x) simp
show ?thesis
proof (rule query-opening-ntimes-prefix-transcript-state-alignment
  [OF prefix])
  fix prefix-query-idxs prefix-chunks
  assume len-prefix-idxs: length prefix-query-idxs = i
  and len-prefix-chunks: length prefix-chunks = i
  and transcript-prefix:
    PTranscript query-state =
      List.concat prefix-chunks @ PTranscript s-i

```

```

and state-i:
  PState s-i =
    state-after-query-chunks (PState query-state) prefix-chunks i
and counter-i: PQueryCounter s-i = PQueryCounter query-state + i
and prefix-rounds:
   $\bigwedge k. k < i \implies$ 
    verifier-query-round-chunk (prefix-query-idxs ! k)
      (map snd f-fl) (map snd fl) (prefix-chunks ! k)
from round[unfolded x-eq verifier-query-round-program-alt-def]
obtain raw s0 where
  raw-out:
    Some (raw, s0)  $\in$ 
      set-dist (execute receive-query-index-challenge s-i)
and after:
  Some ((), s-suc)  $\in$ 
    set-dist
      (execute
        (verifier-query-round-after-index-program fr f-fl f-final as
          fl final raw) s0)
by (auto elim! set-dist-bindE)
have lookup-s0:
  fmlookup (HashMap s0)
    (QueryIndexChallenge (PQueryCounter s-i) (PState s-i)) =
    Some raw
using receive-query-index-challenge-outcome[OF raw-out] by simp
show ?thesis
proof (rule verifier-query-round-program-outcome[OF round[unfolded x-eq]])
fix raw' idx chunk
assume idx-eq: idx = index (to-nat raw')
and chunk-shape:
  verifier-query-round-chunk idx (map snd f-fl) (map snd fl)
    chunk
and transcript-round:
  PTranscript s-i = chunk @ PTranscript s-suc
and lookup-suc:
  fmlookup (HashMap s-suc)
    (QueryIndexChallenge (PQueryCounter s-i) (PState s-i)) =
    Some raw'
have s0-suc: s0  $\leq$  s-suc
using verifier-query-round-after-index-program-outcome[OF after]
by simp
have lookup-suc-raw:
  fmlookup (HashMap s-suc)
    (QueryIndexChallenge (PQueryCounter s-i) (PState s-i)) =
    Some raw
by (rule hash-extension-lookup[OF lookup-s0 s0-suc])
have raw-eq': raw = raw'
using lookup-suc lookup-suc-raw by simp
from verifier-query-round-after-index-composition-fri-last-step-evidence

```

```

[OF after nonempty raw-layer]
obtain actual-chunk axp axp-path axn axn-path where
  auth-suc:
    fri-layer-chunk-authenticated
      (snd (fl ! (length fl - 1)))
      (fri-layer-lengths (length fl) (clength * scale) !
        (length fl - 1))
      (fri-layer-indices (length fl) (index (to-nat raw))
        (clength * scale) ! (length fl - 1))
    actual-chunk s-suc
and step-raw:
  fri-layer-step-evidence
    (snd (fl ! (length fl - 1)))
    (fst (fl ! (length fl - 1)))
    (fri-layer-lengths (length fl) (clength * scale) !
      (length fl - 1))
    (fri-layer-indices (length fl) (index (to-nat raw))
      (clength * scale) ! (length fl - 1))
    (2 ^ (length fl - 1))
    (fri-sibling-index
      (fri-layer-lengths (length fl) (clength * scale) !
        (length fl - 1))
      (fri-layer-indices (length fl) (index (to-nat raw))
        (clength * scale) ! (length fl - 1)))
    axp axp-path axn axn-path
    ((fri-layer-indices (length fl) (index (to-nat raw))
      (clength * scale) ! (length fl - 1)) mod
      ((fri-layer-lengths (length fl) (clength * scale) !
        (length fl - 1)) div 2))
    final actual-chunk
by blast
show ?thesis
proof (rule query-opening-ntimes-suffix-transcript-state-alignment
  [OF suffix])
fix suffix-query-idxs suffix-chunks
assume len-suffix-idxs:
  length suffix-query-idxs = rounds - Suc i
and len-suffix-chunks:
  length suffix-chunks = rounds - Suc i
and transcript-suffix:
  PTranscript s-suc =
    List.concat suffix-chunks @ PTranscript final-state
and suffix-ext: s-suc ≤ final-state
and suffix-rounds:
  ∧k. k < rounds - Suc i ⇒
    verifier-query-round-chunk (suffix-query-idxs ! k)
      (map snd f-fl) (map snd fl) (suffix-chunks ! k)
have transcript-all:
  PTranscript query-state =

```

```

    List.concat (prefix-chunks @ chunk # suffix-chunks) @
    PTranscript final-state
using transcript-prefix transcript-round transcript-suffix by simp
have auth-final-raw:
  fri-layer-chunk-authenticated
    (snd (fl ! (length fl - 1)))
    (fri-layer-lengths (length fl) (clength * scale) !
      (length fl - 1))
    (fri-layer-indices (length fl) (index (to-nat raw))
      (clength * scale) !
      (length fl - 1))
    actual-chunk final-state
  by (rule fri-layer-chunk-authenticated-mono[OF auth-suc suffix-ext])
have auth-final:
  fri-layer-chunk-authenticated
    (snd (fl ! (length fl - 1)))
    (fri-layer-lengths (length fl) (clength * scale) !
      (length fl - 1))
    (fri-layer-indices (length fl) idx (clength * scale) !
      (length fl - 1))
    actual-chunk final-state
  using auth-final-raw raw-eq' idx-eq by simp
have step-idx:
  fri-layer-step-evidence
    (snd (fl ! (length fl - 1)))
    (fst (fl ! (length fl - 1)))
    (fri-layer-lengths (length fl) (clength * scale) !
      (length fl - 1))
    (fri-layer-indices (length fl) idx (clength * scale) !
      (length fl - 1))
    (2 ^ (length fl - 1))
    (fri-sibling-index
      (fri-layer-lengths (length fl) (clength * scale) !
        (length fl - 1))
      (fri-layer-indices (length fl) idx (clength * scale) !
        (length fl - 1)))
    arp arp-path arn arn-path
    ((fri-layer-indices (length fl) idx (clength * scale) !
      (length fl - 1)) mod
      ((fri-layer-lengths (length fl) (clength * scale) !
        (length fl - 1)) div 2))
    final actual-chunk
  using step-raw raw-eq' idx-eq by simp
have lookup-final:
  fmlookup (HashMap final-state)
    (QueryIndexChallenge (PQueryCounter query-state + i)
      (state-after-query-chunks (PState query-state)
        prefix-chunks i)) =
  Some raw

```

```

proof –
  have lookup-suc':
    fmlookup (HashMap s-suc)
      (QueryIndexChallenge (PQueryCounter query-state + i)
        (state-after-query-chunks (PState query-state)
          prefix-chunks i)) =
      Some raw
    using lookup-suc-raw state-i counter-i by simp
  show ?thesis
    by (rule hash-extension-lookup[OF lookup-suc' suffix-ext])
qed
show ?thesis
  by (rule that[OF - len-prefix-idxs len-prefix-chunks
    len-suffix-idxs len-suffix-chunks chunk-shape prefix-rounds
    suffix-rounds transcript-all auth-final step-idx
    lookup-final])
    (use idx-eq raw-eq' in simp)
qed
qed
qed
qed

lemma generic-fri-forced-value-eq-from-authenticated-step:
  assumes forced:
    generic-fri-round-forced-next-value roots challenges query-idxs
    round-layers round-idx layer-idx v
  and recorded-auth:
    generic-fri-recorded-layer-chunk-authenticated roots query-idxs
    round-layers final-state round-idx layer-idx
  and actual-auth:
    fri-layer-chunk-authenticated (roots ! layer-idx)
    (fri-evidence-layer-len roots layer-idx)
    (fri-evidence-layer-idx roots query-idxs round-idx layer-idx)
    actual-chunk final-state
  and actual-step:
    fri-layer-step-evidence
    (roots ! layer-idx)
    (challenges ! layer-idx)
    (fri-evidence-layer-len roots layer-idx)
    (fri-evidence-layer-idx roots query-idxs round-idx layer-idx)
    (2 ^ layer-idx)
    (fri-sibling-index (fri-evidence-layer-len roots layer-idx)
      (fri-evidence-layer-idx roots query-idxs round-idx layer-idx))
    axp axp-path axn axn-path
    (fri-evidence-next-idx roots query-idxs round-idx layer-idx)
    actual-value actual-chunk
  and no-merkle:
     $\neg$  partial-merkle-inconsistency-bad s (Some (result, final-state))
  shows v = actual-value

```


proof –

from forced obtain $rxp\ rxp\text{-}path\ rxn\ rxn\text{-}path$ **where** $recorded\text{-}step$:

$fri\text{-}layer\text{-}step\text{-}evidence$
 $(roots\ !\ layer\text{-}idx)$
 $(challenges\ !\ layer\text{-}idx)$
 $(fri\text{-}evidence\text{-}layer\text{-}len\ roots\ layer\text{-}idx)$
 $(fri\text{-}evidence\text{-}layer\text{-}idx\ roots\ query\text{-}idxs\ round\text{-}idx\ layer\text{-}idx)$
 $(2\ ^\wedge\ layer\text{-}idx)$
 $(fri\text{-}sibling\text{-}index\ (fri\text{-}evidence\text{-}layer\text{-}len\ roots\ layer\text{-}idx)$
 $\quad (fri\text{-}evidence\text{-}layer\text{-}idx\ roots\ query\text{-}idxs\ round\text{-}idx\ layer\text{-}idx))$
 $rxp\ rxp\text{-}path\ rxn\ rxn\text{-}path$
 $(fri\text{-}evidence\text{-}next\text{-}idx\ roots\ query\text{-}idxs\ round\text{-}idx\ layer\text{-}idx)$
 v
 $(round\text{-}layers\ !\ round\text{-}idx\ !\ layer\text{-}idx)$

unfolding $generic\text{-}fri\text{-}round\text{-}forced\text{-}next\text{-}value\text{-}def$ **by** $blast$

have $recorded\text{-}chunk$:

$fri\text{-}layer\text{-}chunk\text{-}authenticated\ (roots\ !\ layer\text{-}idx)$
 $(fri\text{-}evidence\text{-}layer\text{-}len\ roots\ layer\text{-}idx)$
 $(fri\text{-}evidence\text{-}layer\text{-}idx\ roots\ query\text{-}idxs\ round\text{-}idx\ layer\text{-}idx)$
 $(round\text{-}layers\ !\ round\text{-}idx\ !\ layer\text{-}idx)\ final\text{-}state$

using $recorded\text{-}auth$

unfolding $generic\text{-}fri\text{-}recorded\text{-}layer\text{-}chunk\text{-}authenticated\text{-}def$.

have $rxp\text{-}eq$: $rxp = axp$

by $(rule\ fri\text{-}layer\text{-}authenticated\text{-}base\text{-}values\text{-}eq\text{-}if\text{-}no\text{-}partial\text{-}merkle$
 $\quad [OF\ no\text{-}merkle\ recorded\text{-}chunk\ actual\text{-}auth])$
 $(use\ fri\text{-}layer\text{-}step\text{-}evidenceD(4)[OF\ recorded\text{-}step]$
 $\quad fri\text{-}layer\text{-}step\text{-}evidenceD(4)[OF\ actual\text{-}step]\ in\ simp\text{-}all)$

have $rxn\text{-}eq$: $rxn = axn$

by $(rule\ fri\text{-}layer\text{-}authenticated\text{-}sibling\text{-}values\text{-}eq\text{-}if\text{-}no\text{-}partial\text{-}merkle$
 $\quad [OF\ no\text{-}merkle\ recorded\text{-}chunk\ actual\text{-}auth])$
 $(use\ fri\text{-}layer\text{-}step\text{-}evidenceD(4)[OF\ recorded\text{-}step]$
 $\quad fri\text{-}layer\text{-}step\text{-}evidenceD(4)[OF\ actual\text{-}step]\ in\ simp\text{-}all)$

show $?thesis$

using $fri\text{-}layer\text{-}step\text{-}evidenceD(3)[OF\ recorded\text{-}step]$
 $fri\text{-}layer\text{-}step\text{-}evidenceD(3)[OF\ actual\text{-}step]$
 $rxp\text{-}eq\ rxn\text{-}eq$

by $simp$

qed

lemma $generic\text{-}fri\text{-}final\text{-}conflict\text{-}false\text{-}from\text{-}authenticated\text{-}step$:

assumes $forced$:

$generic\text{-}fri\text{-}round\text{-}forced\text{-}next\text{-}value\ roots\ challenges\ query\text{-}idxs$
 $round\text{-}layers\ round\text{-}idx\ layer\text{-}idx\ v$

and $recorded\text{-}auth$:

$generic\text{-}fri\text{-}recorded\text{-}layer\text{-}chunk\text{-}authenticated\ roots\ query\text{-}idxs$
 $round\text{-}layers\ final\text{-}state\ round\text{-}idx\ layer\text{-}idx$

and $actual\text{-}auth$:

$fri\text{-}layer\text{-}chunk\text{-}authenticated\ (roots\ !\ layer\text{-}idx)$
 $(fri\text{-}evidence\text{-}layer\text{-}len\ roots\ layer\text{-}idx)$

```

    (fri-evidence-layer-idx roots query-idxs round-idx layer-idx)
    actual-chunk final-state
and actual-step:
  fri-layer-step-evidence
    (roots ! layer-idx)
    (challenges ! layer-idx)
    (fri-evidence-layer-len roots layer-idx)
    (fri-evidence-layer-idx roots query-idxs round-idx layer-idx)
    (2 ^ layer-idx)
    (fri-sibling-index (fri-evidence-layer-len roots layer-idx)
      (fri-evidence-layer-idx roots query-idxs round-idx layer-idx))
    axp axp-path axn axn-path
    (fri-evidence-next-idx roots query-idxs round-idx layer-idx)
    final-value actual-chunk
and no-merkle:
  ¬ partial-merkle-inconsistency-bad s (Some (result, final-state))
and conflict: v ≠ final-value
shows False
using generic-fri-forced-value-eq-from-authenticated-step
  [OF forced recorded-auth actual-auth actual-step no-merkle]
  conflict
by simp

lemma generic-fri-selected-final-step-missing-false-from-authenticated-step:
  fixes root challenge final-value :: 'f
    and final-state :: ('f, 'a) protocol-channel-scheme
  assumes actual-auth:
    fri-layer-chunk-authenticated root len idx actual-chunk final-state
  and actual-step:
    fri-layer-step-evidence root challenge len idx pw sibling-idx
    axp axp-path axn axn-path next-idx final-value actual-chunk
  and missing:
    ∧ actual-chunk axp axp-path axn axn-path.
    ¬ (fri-layer-chunk-authenticated root len idx actual-chunk final-state ∧
      fri-layer-step-evidence root challenge len idx pw sibling-idx
      axp axp-path axn axn-path next-idx final-value actual-chunk)
  shows False
  using actual-auth actual-step missing by blast

end

end

theory Soundness-FRI-Authenticated-Route
  imports Soundness-FRI-Layer-Replay
begin

```

Route-level FRI events that keep the existing public reduction shape but refine the

sampld same-layer branch to carry authenticated Merkle evidence.

context *soundness*

begin

lemma *fri-first-layer-row-bound*:

assumes *roots-nonempty*: $0 < \text{length roots}$

shows

$0 < \text{fri-layer-lengths } (\text{length roots}) (\text{clength} * \text{scale}) ! 0 \wedge$
 $\text{fri-layer-indices } (\text{length roots}) (\text{index } (\text{to-nat raw}))$
 $(\text{clength} * \text{scale}) ! 0 <$
 $\text{fri-layer-lengths } (\text{length roots}) (\text{clength} * \text{scale}) ! 0$

proof –

have *idx-bound*: $\text{index } (\text{to-nat raw}) < \text{clength} * \text{scale}$

proof –

have $\text{index } (\text{to-nat raw}) < \text{query-sample-space-size}$

by (rule *index-less-query-sample-space*)

also have $\dots \leq \text{clength} * \text{scale}$

by (rule *query-sample-space-size-le-domain*)

finally show *?thesis* .

qed

show *?thesis*

using *roots-nonempty idx-bound clength-pos scale-pos*

by (cases *roots*) *simp-all*

qed

lemma *fri-layer-row-bound*:

assumes *layer-bound*: $j < \text{length roots}$

and *eval-power*: $\text{clength} * \text{scale} = 2^N$

and *roots-rounds*: $\text{length roots} \leq N$

shows

$0 < \text{fri-layer-lengths } (\text{length roots}) (\text{clength} * \text{scale}) ! j \wedge$
 $\text{fri-layer-indices } (\text{length roots}) (\text{index } (\text{to-nat raw}))$
 $(\text{clength} * \text{scale}) ! j <$
 $\text{fri-layer-lengths } (\text{length roots}) (\text{clength} * \text{scale}) ! j$

proof –

have *idx-bound*: $\text{index } (\text{to-nat raw}) < \text{clength} * \text{scale}$

proof –

have $\text{index } (\text{to-nat raw}) < \text{query-sample-space-size}$

by (rule *index-less-query-sample-space*)

also have $\dots \leq \text{clength} * \text{scale}$

by (rule *query-sample-space-size-le-domain*)

finally show *?thesis* .

qed

have *idx-layer*:

$\text{fri-layer-indices } (\text{length roots}) (\text{index } (\text{to-nat raw}))$

$(\text{clength} * \text{scale}) ! j <$

$\text{fri-layer-lengths } (\text{length roots}) (\text{clength} * \text{scale}) ! j$

by (rule *fri-layer-indices-nth-bound*)

[OF *idx-bound layer-bound eval-power roots-rounds*]

```

have len-pos:
  0 < fri-layer-lengths (length roots) (clength * scale) ! j
  using idx-layer by linarith
show ?thesis
  using len-pos idx-layer by simp
qed

definition trace-fri-header-tied-sampled-assignment-conflict-with-authenticated-same-layer
  :: ('f, 'a) protocol-channel-scheme  $\Rightarrow$ 
    (unit list  $\times$  ('f, 'a) protocol-channel-scheme) option  $\Rightarrow$  bool
where
  trace-fri-header-tied-sampled-assignment-conflict-with-authenticated-same-layer
    s out  $\longleftrightarrow$ 
    ( $\exists$  trace-roots trace-bs trace-final dg composition-roots
      composition-bs composition-final fri-query-idxs trace-round-layers
      composition-round-layers fr as rest trace-openings trace-table
      result final-state.
      accepted-fri-opening-transcript s out trace-roots trace-bs
      trace-final dg composition-roots composition-bs composition-final
      fri-query-idxs trace-round-layers composition-round-layers  $\wedge$ 
      out = Some (result, final-state)  $\wedge$ 
      verifier-header-transcript s fr trace-roots trace-final as dg
      composition-roots composition-final rest  $\wedge$ 
      accepted-with-partial-trace-openings s out fr fri-query-idxs
      trace-openings  $\wedge$ 
      partial-trace-table-candidate trace-table trace-openings  $\wedge$ 
       $\neg$  trace-table-low-degree trace-table  $\wedge$ 
      generic-fri-sampled-assignment-conflict-with-authenticated-same-layer
      trace-table trace-roots trace-bs trace-final fri-query-idxs
      trace-round-layers final-state)

definition trace-fri-header-tied-sampled-assignment-conflict-without-same-layer
  :: ('f, 'a) protocol-channel-scheme  $\Rightarrow$ 
    (unit list  $\times$  ('f, 'a) protocol-channel-scheme) option  $\Rightarrow$  bool
where
  trace-fri-header-tied-sampled-assignment-conflict-without-same-layer
    s out  $\longleftrightarrow$ 
    ( $\exists$  trace-roots trace-bs trace-final dg composition-roots
      composition-bs composition-final fri-query-idxs trace-round-layers
      composition-round-layers fr as rest trace-openings trace-table.
      accepted-fri-opening-transcript s out trace-roots trace-bs
      trace-final dg composition-roots composition-bs composition-final
      fri-query-idxs trace-round-layers composition-round-layers  $\wedge$ 
      verifier-header-transcript s fr trace-roots trace-final as dg
      composition-roots composition-final rest  $\wedge$ 
      accepted-with-partial-trace-openings s out fr fri-query-idxs
      trace-openings  $\wedge$ 
      partial-trace-table-candidate trace-table trace-openings  $\wedge$ 
       $\neg$  trace-table-low-degree trace-table  $\wedge$ 

```

generic-fri-sampled-assignment-conflict-without-same-layer
trace-table trace-roots trace-bs trace-final fri-query-idxs
trace-round-layers)

definition *trace-fri-header-tied-sampled-base-opening-conflict*

$:: ('f, 'a) \text{ protocol-channel-scheme} \Rightarrow$
 $(\text{unit list} \times ('f, 'a) \text{ protocol-channel-scheme}) \text{ option} \Rightarrow \text{bool}$

where

trace-fri-header-tied-sampled-base-opening-conflict s out $\longleftrightarrow$
 $(\exists \text{ trace-roots trace-bs trace-final dg composition-roots}$
 $\text{ composition-bs composition-final fri-query-idxs trace-round-layers}$
 $\text{ composition-round-layers fr as rest trace-openings trace-table.}$
 $\text{ accepted-fri-opening-transcript s out trace-roots trace-bs}$
 $\text{ trace-final dg composition-roots composition-bs composition-final}$
 $\text{ fri-query-idxs trace-round-layers composition-round-layers} \wedge$
 $\text{ verifier-header-transcript s fr trace-roots trace-final as dg}$
 $\text{ composition-roots composition-final rest} \wedge$
 $\text{ accepted-with-partial-trace-openings s out fr fri-query-idxs}$
 $\text{ trace-openings} \wedge$
 $\text{ partial-trace-table-candidate trace-table trace-openings} \wedge$
 $\neg \text{ trace-table-low-degree trace-table} \wedge$
 $\text{ generic-fri-sampled-base-opening-conflict trace-table trace-roots}$
 $\text{ trace-bs fri-query-idxs trace-round-layers})$

definition *trace-fri-header-tied-sampled-next-value-conflict*

$:: ('f, 'a) \text{ protocol-channel-scheme} \Rightarrow$
 $(\text{unit list} \times ('f, 'a) \text{ protocol-channel-scheme}) \text{ option} \Rightarrow \text{bool}$

where

trace-fri-header-tied-sampled-next-value-conflict s out $\longleftrightarrow$
 $(\exists \text{ trace-roots trace-bs trace-final dg composition-roots}$
 $\text{ composition-bs composition-final fri-query-idxs trace-round-layers}$
 $\text{ composition-round-layers fr as rest trace-openings trace-table.}$
 $\text{ accepted-fri-opening-transcript s out trace-roots trace-bs}$
 $\text{ trace-final dg composition-roots composition-bs composition-final}$
 $\text{ fri-query-idxs trace-round-layers composition-round-layers} \wedge$
 $\text{ verifier-header-transcript s fr trace-roots trace-final as dg}$
 $\text{ composition-roots composition-final rest} \wedge$
 $\text{ accepted-with-partial-trace-openings s out fr fri-query-idxs}$
 $\text{ trace-openings} \wedge$
 $\text{ partial-trace-table-candidate trace-table trace-openings} \wedge$
 $\neg \text{ trace-table-low-degree trace-table} \wedge$
 $\text{ generic-fri-sampled-next-value-conflict trace-roots trace-bs}$
 $\text{ fri-query-idxs trace-round-layers})$

definition *trace-fri-header-tied-sampled-successor-opening-conflict*

$:: ('f, 'a) \text{ protocol-channel-scheme} \Rightarrow$
 $(\text{unit list} \times ('f, 'a) \text{ protocol-channel-scheme}) \text{ option} \Rightarrow \text{bool}$

where

trace-fri-header-tied-sampled-successor-opening-conflict s out $\longleftrightarrow$

$(\exists \text{ trace-roots trace-bs trace-final dg composition-roots}$
 $\text{composition-bs composition-final fri-query-idxs trace-round-layers}$
 $\text{composition-round-layers fr as rest trace-openings trace-table.}$
 $\text{accepted-fri-opening-transcript s out trace-roots trace-bs}$
 $\text{trace-final dg composition-roots composition-bs composition-final}$
 $\text{fri-query-idxs trace-round-layers composition-round-layers} \wedge$
 $\text{verifier-header-transcript s fr trace-roots trace-final as dg}$
 $\text{composition-roots composition-final rest} \wedge$
 $\text{accepted-with-partial-trace-openings s out fr fri-query-idxs}$
 $\text{trace-openings} \wedge$
 $\text{partial-trace-table-candidate trace-table trace-openings} \wedge$
 $\neg \text{trace-table-low-degree trace-table} \wedge$
 $\text{generic-fri-sampled-successor-opening-conflict trace-roots trace-bs}$
 $\text{fri-query-idxs trace-round-layers})$

definition *trace-fri-header-tied-sampled-final-value-conflict*

$:: ('f, 'a) \text{ protocol-channel-scheme} \Rightarrow$
 $(\text{unit list} \times ('f, 'a) \text{ protocol-channel-scheme}) \text{ option} \Rightarrow \text{bool}$

where

$\text{trace-fri-header-tied-sampled-final-value-conflict s out} \longleftrightarrow$
 $(\exists \text{ trace-roots trace-bs trace-final dg composition-roots}$
 $\text{composition-bs composition-final fri-query-idxs trace-round-layers}$
 $\text{composition-round-layers fr as rest trace-openings trace-table.}$
 $\text{accepted-fri-opening-transcript s out trace-roots trace-bs}$
 $\text{trace-final dg composition-roots composition-bs composition-final}$
 $\text{fri-query-idxs trace-round-layers composition-round-layers} \wedge$
 $\text{verifier-header-transcript s fr trace-roots trace-final as dg}$
 $\text{composition-roots composition-final rest} \wedge$
 $\text{accepted-with-partial-trace-openings s out fr fri-query-idxs}$
 $\text{trace-openings} \wedge$
 $\text{partial-trace-table-candidate trace-table trace-openings} \wedge$
 $\neg \text{trace-table-low-degree trace-table} \wedge$
 $\text{generic-fri-sampled-final-value-conflict trace-roots trace-bs}$
 $\text{trace-final fri-query-idxs trace-round-layers})$

lemma *trace-fri-header-tied-without-same-imp-branch:*

assumes

$\text{trace-fri-header-tied-sampled-assignment-conflict-without-same-layer}$
 $s \text{ out}$

shows

$\text{trace-fri-header-tied-sampled-base-opening-conflict s out} \vee$
 $\text{trace-fri-header-tied-sampled-next-value-conflict s out} \vee$
 $\text{trace-fri-header-tied-sampled-successor-opening-conflict s out} \vee$
 $\text{trace-fri-header-tied-sampled-final-value-conflict s out}$

proof —

from *assms* **obtain** $\text{trace-roots trace-bs trace-final dg composition-roots}$
 $\text{composition-bs composition-final fri-query-idxs trace-round-layers}$
 $\text{composition-round-layers fr as rest trace-openings trace-table}$

where *fri-openings:*

accepted-fri-opening-transcript s out trace-roots trace-bs
trace-final dg composition-roots composition-bs composition-final
fri-query-idxs trace-round-layers composition-round-layers
and *header:*
verifier-header-transcript s fr trace-roots trace-final as dg
composition-roots composition-final rest
and *partial:*
accepted-with-partial-trace-openings s out fr fri-query-idxs
trace-openings
and *cand: partial-trace-table-candidate trace-table trace-openings*
and *not-low: $\neg$ trace-table-low-degree trace-table*
and *without:*
generic-fri-sampled-assignment-conflict-without-same-layer
trace-table trace-roots trace-bs trace-final fri-query-idxs
trace-round-layers
unfolding
trace-fri-header-tied-sampled-assignment-conflict-without-same-layer-def
by *blast*
have *split:*
generic-fri-sampled-base-opening-conflict trace-table trace-roots
trace-bs fri-query-idxs trace-round-layers $\vee$
generic-fri-sampled-next-value-conflict trace-roots trace-bs
fri-query-idxs trace-round-layers $\vee$
generic-fri-sampled-successor-opening-conflict trace-roots trace-bs
fri-query-idxs trace-round-layers $\vee$
generic-fri-sampled-final-value-conflict trace-roots trace-bs
trace-final fri-query-idxs trace-round-layers
by (*rule generic-fri-sampled-assignment-conflict-without-same-layer-split*
[OF without])
then show *?thesis*
proof
assume *base:*
generic-fri-sampled-base-opening-conflict trace-table trace-roots
trace-bs fri-query-idxs trace-round-layers
then show *?thesis*
unfolding *trace-fri-header-tied-sampled-base-opening-conflict-def*
by (*intro disjI1 exI conjI*)
(rule fri-openings, rule header, rule partial, rule cand,
rule not-low, rule base)
next
assume *rest-split:*
generic-fri-sampled-next-value-conflict trace-roots trace-bs
fri-query-idxs trace-round-layers $\vee$
generic-fri-sampled-successor-opening-conflict trace-roots trace-bs
fri-query-idxs trace-round-layers $\vee$
generic-fri-sampled-final-value-conflict trace-roots trace-bs
trace-final fri-query-idxs trace-round-layers
then show *?thesis*
proof

```

assume next-conflict:
  generic-fri-sampled-next-value-conflict trace-roots trace-bs
  fri-query-idxs trace-round-layers
then show ?thesis
  unfolding trace-fri-header-tied-sampled-next-value-conflict-def
  by (intro disjI2 disjI1 exI conjI)
    (rule fri-openings, rule header, rule partial, rule cand,
      rule not-low, rule next-conflict)
next
assume rest-split':
  generic-fri-sampled-successor-opening-conflict trace-roots trace-bs
  fri-query-idxs trace-round-layers  $\vee$ 
  generic-fri-sampled-final-value-conflict trace-roots trace-bs
  trace-final fri-query-idxs trace-round-layers
then show ?thesis
proof
  assume successor:
    generic-fri-sampled-successor-opening-conflict trace-roots
    trace-bs fri-query-idxs trace-round-layers
  then show ?thesis
  unfolding
    trace-fri-header-tied-sampled-successor-opening-conflict-def
  by (intro disjI2 disjI2 disjI1 exI conjI)
    (rule fri-openings, rule header, rule partial, rule cand,
      rule not-low, rule successor)
next
assume final-conflict:
  generic-fri-sampled-final-value-conflict trace-roots trace-bs
  trace-final fri-query-idxs trace-round-layers
then show ?thesis
  unfolding trace-fri-header-tied-sampled-final-value-conflict-def
  by (intro disjI2 disjI2 disjI2 exI conjI)
    (rule fri-openings, rule header, rule partial, rule cand,
      rule not-low, rule final-conflict)
qed
qed
qed
qed

lemma wp-trace-fri-header-tied-without-same-bound-from-branches:
assumes base-bound:
  wp-event verify-monad
  (trace-fri-header-tied-sampled-base-opening-conflict s)  $s \leq B$ 
and next-bound:
  wp-event verify-monad
  (trace-fri-header-tied-sampled-next-value-conflict s)  $s \leq N$ 
and successor-bound:
  wp-event verify-monad
  (trace-fri-header-tied-sampled-successor-opening-conflict s)  $s \leq S$ 

```


and *final-bound*:
wp-event verify-monad
 (trace-fri-header-tied-sampled-final-value-conflict *s*) $s \leq F$
shows
wp-event verify-monad
 (trace-fri-header-tied-sampled-assignment-conflict-without-same-layer *s*)
 $s \leq B + N + S + F$
proof –
have *wp-event verify-monad*
 (trace-fri-header-tied-sampled-assignment-conflict-without-same-layer *s*)
 $s \leq$
wp-event verify-monad
 ($\lambda out.$
 trace-fri-header-tied-sampled-base-opening-conflict *s out* $\vee$
 trace-fri-header-tied-sampled-next-value-conflict *s out* $\vee$
 trace-fri-header-tied-sampled-successor-opening-conflict *s out* $\vee$
 trace-fri-header-tied-sampled-final-value-conflict *s out*) *s*
by (rule *wp-event-mono*)
 (rule *trace-fri-header-tied-without-same-imp-branch*)
also have ... $\leq$
wp-event verify-monad
 (trace-fri-header-tied-sampled-base-opening-conflict *s*) *s* +
wp-event verify-monad
 (trace-fri-header-tied-sampled-next-value-conflict *s*) *s* +
wp-event verify-monad
 (trace-fri-header-tied-sampled-successor-opening-conflict *s*) *s* +
wp-event verify-monad
 (trace-fri-header-tied-sampled-final-value-conflict *s*) *s*
by (rule *wp-event-union-bound4*)
also have ... $\leq B + N + S + F$
by (intro *add-mono base-bound next-bound successor-bound final-bound*)
finally show ?thesis .
qed

lemma *trace-fri-header-tied-without-sameI-base*:
assumes *trace-fri-header-tied-sampled-base-opening-conflict s out*
shows
trace-fri-header-tied-sampled-assignment-conflict-without-same-layer
s out
using *assms*
unfolding *trace-fri-header-tied-sampled-base-opening-conflict-def*
trace-fri-header-tied-sampled-assignment-conflict-without-same-layer-def
by (blast intro:
 generic-fri-sampled-assignment-conflict-without-same-layerI-base)

lemma *trace-fri-header-tied-without-sameI-next*:
assumes *trace-fri-header-tied-sampled-next-value-conflict s out*
shows
trace-fri-header-tied-sampled-assignment-conflict-without-same-layer

s out
using *assms*
unfolding *trace-fri-header-tied-sampled-next-value-conflict-def*
trace-fri-header-tied-sampled-assignment-conflict-without-same-layer-def
by (*blast intro*:
generic-fri-sampled-assignment-conflict-without-same-layerI-next)

lemma *trace-fri-header-tied-without-sameI-successor*:
assumes *trace-fri-header-tied-sampled-successor-opening-conflict s out*
shows
trace-fri-header-tied-sampled-assignment-conflict-without-same-layer
s out
using *assms*
unfolding *trace-fri-header-tied-sampled-successor-opening-conflict-def*
trace-fri-header-tied-sampled-assignment-conflict-without-same-layer-def
by (*blast intro*:
generic-fri-sampled-assignment-conflict-without-same-layerI-successor)

lemma *trace-fri-header-tied-without-sameI-final*:
assumes *trace-fri-header-tied-sampled-final-value-conflict s out*
shows
trace-fri-header-tied-sampled-assignment-conflict-without-same-layer
s out
using *assms*
unfolding *trace-fri-header-tied-sampled-final-value-conflict-def*
trace-fri-header-tied-sampled-assignment-conflict-without-same-layer-def
by (*blast intro*:
generic-fri-sampled-assignment-conflict-without-same-layerI-final)

lemma *trace-fri-header-tied-without-same-iff-branches*:
trace-fri-header-tied-sampled-assignment-conflict-without-same-layer s out
 $\longleftrightarrow$
trace-fri-header-tied-sampled-base-opening-conflict s out $\vee$
trace-fri-header-tied-sampled-next-value-conflict s out $\vee$
trace-fri-header-tied-sampled-successor-opening-conflict s out $\vee$
trace-fri-header-tied-sampled-final-value-conflict s out
using *trace-fri-header-tied-without-same-imp-branch*
trace-fri-header-tied-without-sameI-base
trace-fri-header-tied-without-sameI-next
trace-fri-header-tied-without-sameI-successor
trace-fri-header-tied-without-sameI-final
by *blast*

lemma *trace-fri-header-tied-base-conflict-obtains-authenticated-first-chunk*:
assumes *base*:
trace-fri-header-tied-sampled-base-opening-conflict
s (Some (result, final-state))
obtains *trace-roots trace-bs trace-final dg composition-roots*
composition-bs composition-final fri-query-idxs trace-round-layers

composition-round-layers fr as rest trace-openings trace-table
round-idx xp xp-path xn xn-path auth-chunk
where
accepted-fri-opening-transcript s (Some (result, final-state))
trace-roots trace-bs trace-final dg composition-roots composition-bs
composition-final fri-query-idxs trace-round-layers
composition-round-layers
verifier-header-transcript s fr trace-roots trace-final as dg
composition-roots composition-final rest
accepted-with-partial-trace-openings s
(Some (result, final-state)) fr fri-query-idxs trace-openings
partial-trace-table-candidate trace-table trace-openings
 $\neg$ *trace-table-low-degree trace-table*
round-idx < length fri-query-idxs
0 < length trace-bs
fri-layer-step-evidence
(trace-roots ! 0)
(trace-bs ! 0)
(fri-evidence-layer-len trace-roots 0)
(fri-evidence-layer-idx trace-roots fri-query-idxs round-idx 0)
1
(fri-sibling-index (fri-evidence-layer-len trace-roots 0)
(fri-evidence-layer-idx trace-roots fri-query-idxs round-idx 0))
xp xp-path xn xn-path
(fri-evidence-next-idx trace-roots fri-query-idxs round-idx 0)
(fri-evidence-next-value trace-roots trace-bs fri-query-idxs
round-idx 0 xp xn)
(trace-round-layers ! round-idx ! 0)
 $\neg$ *fri-opening-matches-table*
(fri-evidence-layer-len trace-roots 0)
(fri-evidence-layer-idx trace-roots fri-query-idxs round-idx 0)
trace-table xp xn
fri-layer-chunk-authenticated (trace-roots ! 0)
(fri-evidence-layer-len trace-roots 0)
(fri-evidence-layer-idx trace-roots fri-query-idxs round-idx 0)
auth-chunk final-state

proof —

from base obtain *trace-roots trace-bs trace-final dg composition-roots*
composition-bs composition-final fri-query-idxs trace-round-layers
composition-round-layers fr as rest trace-openings trace-table

where *fri-openings:*

accepted-fri-opening-transcript s (Some (result, final-state))
trace-roots trace-bs trace-final dg composition-roots composition-bs
composition-final fri-query-idxs trace-round-layers
composition-round-layers

and header:

verifier-header-transcript s fr trace-roots trace-final as dg
composition-roots composition-final rest

and partial:

```

    accepted-with-partial-trace-openings s
      (Some (result, final-state)) fr fri-query-idxs trace-openings
  and cand: partial-trace-table-candidate trace-table trace-openings
  and not-low:  $\neg$  trace-table-low-degree trace-table
  and generic:
    generic-fri-sampled-base-opening-conflict trace-table trace-roots
    trace-bs fri-query-idxs trace-round-layers
  unfolding trace-fri-header-tied-sampled-base-opening-conflict-def
  by blast
from generic obtain round-idx xp xp-path xn xn-path where
  trace-bs-nonempty:  $0 < \text{length } \text{trace-bs}$ 
  and round-bound: round-idx < length fri-query-idxs
  and step:
    fri-layer-step-evidence
      (trace-roots ! 0)
      (trace-bs ! 0)
      (fri-evidence-layer-len trace-roots 0)
      (fri-evidence-layer-idx trace-roots fri-query-idxs round-idx 0)
      1
      (fri-sibling-index (fri-evidence-layer-len trace-roots 0)
        (fri-evidence-layer-idx trace-roots fri-query-idxs round-idx 0))
      xp xp-path xn xn-path
      (fri-evidence-next-idx trace-roots fri-query-idxs round-idx 0)
      (fri-evidence-next-value trace-roots trace-bs fri-query-idxs
        round-idx 0 xp xn)
      (trace-round-layers ! round-idx ! 0)
  and no-match:
     $\neg$  fri-opening-matches-table
      (fri-evidence-layer-len trace-roots 0)
      (fri-evidence-layer-idx trace-roots fri-query-idxs round-idx 0)
      trace-table xp xn
  unfolding generic-fri-sampled-base-opening-conflict-def
  by blast
have roots-nonempty:  $0 < \text{length } \text{trace-roots}$ 
  using accepted-fri-opening-transcript-shapes(2)[OF fri-openings]
  trace-bs-nonempty by simp
have round-bound-rounds: round-idx < rounds
  using round-bound accepted-fri-opening-transcript-shapes(7)[OF fri-openings]
  by simp
obtain auth-chunk where auth:
  fri-layer-chunk-authenticated (trace-roots ! 0)
  (fri-evidence-layer-len trace-roots 0)
  (fri-evidence-layer-idx trace-roots fri-query-idxs round-idx 0)
  auth-chunk final-state
proof -
  have layer-bound:  $0 < \text{length } \text{trace-roots}$ 
    by (rule roots-nonempty)
  have raw-layer:
     $\bigwedge \text{raw.}$ 

```

```

    0 < fri-layer-lengths (length trace-roots) (clength * scale) ! 0 ∧
    fri-layer-indices (length trace-roots) (index (to-nat raw))
    (clength * scale) ! 0 <
    fri-layer-lengths (length trace-roots) (clength * scale) ! 0
  by (rule fri-first-layer-raw-bound[OF roots-nonempty])
from accepted-fri-opening-transcript-trace-step-with-authenticated-chunk-at
  [OF fri-openings refl round-bound-rounds layer-bound raw-layer]
obtain auth-chunk where
  generic-fri-transcript-step-with-authenticated-chunk trace-roots
  trace-bs fri-query-idxs trace-round-layers final-state round-idx 0
by blast
then obtain auth-chunk where
  fri-layer-chunk-authenticated (trace-roots ! 0)
  (fri-evidence-layer-len trace-roots 0)
  (fri-evidence-layer-idx trace-roots fri-query-idxs round-idx 0)
  auth-chunk final-state
by (rule generic-fri-transcript-step-with-authenticated-chunkE)
then show ?thesis
  by (rule that)
qed
show ?thesis
  by (rule that[OF fri-openings header partial cand not-low round-bound
    trace-bs-nonempty step no-match auth])
qed

lemma trace-fri-header-tied-sampled-assignment-conflict-with-authenticated-same-layer-imp-header-tied:
  assumes
    trace-fri-header-tied-sampled-assignment-conflict-with-authenticated-same-layer
    s out
  shows trace-fri-header-tied-sampled-assignment-conflict s out
proof -
  from assms obtain trace-roots trace-bs trace-final dg composition-roots
    composition-bs composition-final fri-query-idxs trace-round-layers
    composition-round-layers fr as rest trace-openings trace-table
    result final-state
  where fri-openings:
    accepted-fri-opening-transcript s out trace-roots trace-bs
    trace-final dg composition-roots composition-bs composition-final
    fri-query-idxs trace-round-layers composition-round-layers
  and out-eq: out = Some (result, final-state)
  and header:
    verifier-header-transcript s fr trace-roots trace-final as dg
    composition-roots composition-final rest
  and partial:
    accepted-with-partial-trace-openings s out fr fri-query-idxs
    trace-openings
  and cand: partial-trace-table-candidate trace-table trace-openings
  and not-low: ¬ trace-table-low-degree trace-table
  and conflict:

```

```

    generic-fri-sampled-assignment-conflict-with-authenticated-same-layer
      trace-table trace-roots trace-bs trace-final fri-query-idxs
      trace-round-layers final-state
  unfolding
    trace-fri-header-tied-sampled-assignment-conflict-with-authenticated-same-layer-def
  by blast
have sampled:
  generic-fri-sampled-assignment-conflict trace-table trace-roots
    trace-bs trace-final fri-query-idxs trace-round-layers
  by (rule
    generic-fri-sampled-assignment-conflict-with-authenticated-same-layer-imp-sampled
    [OF conflict])
show ?thesis
  unfolding trace-fri-header-tied-sampled-assignment-conflict-def
  by (intro exI conjI)
    (rule fri-openings, rule header, rule partial, rule cand,
     rule not-low, rule sampled)
qed

lemma wp-trace-fri-header-tied-sampled-assignment-conflict-with-authenticated-same-layer-bound-from-header-tied:
  assumes header-bound:
    wp-event verify-monad
      (trace-fri-header-tied-sampled-assignment-conflict s) s ≤ C
  shows
    wp-event verify-monad
      (trace-fri-header-tied-sampled-assignment-conflict-with-authenticated-same-layer s)
      s ≤ C
proof -
  have wp-event verify-monad
    (trace-fri-header-tied-sampled-assignment-conflict-with-authenticated-same-layer s)
    s ≤
    wp-event verify-monad
      (trace-fri-header-tied-sampled-assignment-conflict s) s
  by (rule wp-event-mono)
    (rule
      trace-fri-header-tied-sampled-assignment-conflict-with-authenticated-same-layer-imp-header-tied)
  then show ?thesis
    by (rule order-trans[OF - header-bound])
qed

definition trace-fri-header-tied-same-layer-conflict-with-authenticated-chunks
  :: ('f, 'a) protocol-channel-scheme ⇒
    (unit list × ('f, 'a) protocol-channel-scheme) option ⇒ bool
where
  trace-fri-header-tied-same-layer-conflict-with-authenticated-chunks
    s out ⟷
    (∃ trace-roots trace-bs trace-final dg composition-roots
      composition-bs composition-final fri-query-idxs trace-round-layers
      composition-round-layers fr as rest trace-openings trace-table

```

result final-state.
accepted-fri-opening-transcript s out trace-roots trace-bs
trace-final dg composition-roots composition-bs composition-final
fri-query-idxs trace-round-layers composition-round-layers $\wedge$
out = Some (result, final-state) $\wedge$
verifier-header-transcript s fr trace-roots trace-final as dg
composition-roots composition-final rest $\wedge$
accepted-with-partial-trace-openings s out fr fri-query-idxs
trace-openings $\wedge$
partial-trace-table-candidate trace-table trace-openings $\wedge$
 $\neg$ *trace-table-low-degree trace-table $\wedge$*
generic-fri-sampled-same-layer-opening-conflict-with-authenticated-chunks
trace-roots trace-bs fri-query-idxs trace-round-layers final-state)

lemma *trace-fri-header-tied-same-layer-conflict-with-authenticated-chunks-imp-partial-merkle:*

assumes

trace-fri-header-tied-same-layer-conflict-with-authenticated-chunks
s out

shows *partial-merkle-inconsistency-bad s out*

proof —

from *assms* **obtain** *trace-roots trace-bs trace-final dg composition-roots*
composition-bs composition-final fri-query-idxs trace-round-layers
composition-round-layers fr as rest trace-openings trace-table result
final-state

where *fri-openings:*

accepted-fri-opening-transcript s out trace-roots trace-bs
trace-final dg composition-roots composition-bs composition-final
fri-query-idxs trace-round-layers composition-round-layers

and *out-eq:* *out = Some (result, final-state)*

and *conflict:*

generic-fri-sampled-same-layer-opening-conflict-with-authenticated-chunks
trace-roots trace-bs fri-query-idxs trace-round-layers final-state

unfolding

trace-fri-header-tied-same-layer-conflict-with-authenticated-chunks-def

by *blast*

have *len:* *length trace-bs = length trace-roots*

using *accepted-fri-opening-transcript-shapes(2)[OF fri-openings]* .

have *partial-merkle-inconsistency-bad s (Some (result, final-state))*

by *(rule*

generic-fri-sampled-same-layer-opening-conflict-with-authenticated-chunks-imp-partial-merkle
 $[OF\ conflict\ len])$

then show *?thesis*

by *(simp add: out-eq)*

qed

lemma *wp-trace-fri-header-tied-same-layer-conflict-with-authenticated-chunks-bound-from-partial-merkle:*

assumes *merkle-bound:*

wp-event verify-monad (partial-merkle-inconsistency-bad s) s $\leq M$

shows

wp-event verify-monad
 (trace-fri-header-tied-same-layer-conflict-with-authenticated-chunks *s*)
s ≤ *M*
proof –
have *wp-event verify-monad*
 (trace-fri-header-tied-same-layer-conflict-with-authenticated-chunks *s*)
s ≤ *wp-event verify-monad* (*partial-merkle-inconsistency-bad s*) *s*
by (rule *wp-event-mono*)
 (rule
 trace-fri-header-tied-same-layer-conflict-with-authenticated-chunks-imp-partial-merkle)
then show ?thesis
by (rule *order-trans*[*OF* - *merkle-bound*])
qed

lemma *trace-fri-header-tied-authenticated-conflict-imp-without-same-or-same-layer*:
assumes
trace-fri-header-tied-sampled-assignment-conflict-with-authenticated-same-layer
s out
shows
trace-fri-header-tied-sampled-assignment-conflict-without-same-layer
s out ∨
trace-fri-header-tied-same-layer-conflict-with-authenticated-chunks s out
proof –
from *assms* **obtain** *trace-roots trace-bs trace-final dg composition-roots*
composition-bs composition-final fri-query-idxs trace-round-layers
composition-round-layers fr as rest trace-openings trace-table result
final-state
where *fri-openings*:
accepted-fri-opening-transcript s out trace-roots trace-bs
trace-final dg composition-roots composition-bs composition-final
fri-query-idxs trace-round-layers composition-round-layers
and *out-eq*: *out* = *Some (result, final-state)*
and *header*:
verifier-header-transcript s fr trace-roots trace-final as dg
composition-roots composition-final rest
and *partial*:
accepted-with-partial-trace-openings s out fr fri-query-idxs
trace-openings
and *cand*: *partial-trace-table-candidate trace-table trace-openings*
and *not-low*: $\neg$ *trace-table-low-degree trace-table*
and *conflict*:
generic-fri-sampled-assignment-conflict-with-authenticated-same-layer
trace-table trace-roots trace-bs trace-final fri-query-idxs
trace-round-layers final-state
unfolding
trace-fri-header-tied-sampled-assignment-conflict-with-authenticated-same-layer-def
by *blast*
have *split*:
generic-fri-sampled-assignment-conflict-without-same-layer


```

    trace-table trace-roots trace-bs trace-final fri-query-idxs
    trace-round-layers  $\vee$ 
    generic-fri-sampled-same-layer-opening-conflict-with-authenticated-chunks
    trace-roots trace-bs fri-query-idxs trace-round-layers final-state
  by (rule
    generic-fri-sampled-assignment-conflict-with-authenticated-same-layer-split
    [OF conflict])
  then show ?thesis
proof
  assume without:
    generic-fri-sampled-assignment-conflict-without-same-layer
    trace-table trace-roots trace-bs trace-final fri-query-idxs
    trace-round-layers
  then show ?thesis
  unfolding
    trace-fri-header-tied-sampled-assignment-conflict-without-same-layer-def
  by (intro disjI1 exI conjI)
    (rule fri-openings, rule header, rule partial, rule cand,
    rule not-low, rule without)
next
  assume same:
    generic-fri-sampled-same-layer-opening-conflict-with-authenticated-chunks
    trace-roots trace-bs fri-query-idxs trace-round-layers final-state
  then show ?thesis
  unfolding
    trace-fri-header-tied-same-layer-conflict-with-authenticated-chunks-def
  by (intro disjI2 exI conjI)
    (rule fri-openings, rule out-eq, rule header, rule partial,
    rule cand, rule not-low, rule same)
qed
qed

lemma trace-fri-header-tied-authenticated-conflictI-without-same:
  assumes
    trace-fri-header-tied-sampled-assignment-conflict-without-same-layer
    s out
  shows
    trace-fri-header-tied-sampled-assignment-conflict-with-authenticated-same-layer
    s out
proof -
  from assms obtain trace-roots trace-bs trace-final dg composition-roots
    composition-bs composition-final fri-query-idxs trace-round-layers
    composition-round-layers fr as rest trace-openings trace-table
  where fri-openings:
    accepted-fri-opening-transcript s out trace-roots trace-bs
    trace-final dg composition-roots composition-bs composition-final
    fri-query-idxs trace-round-layers composition-round-layers
  and header:
    verifier-header-transcript s fr trace-roots trace-final as dg

```

```

    composition-roots composition-final rest
and partial:
    accepted-with-partial-trace-openings s out fr fri-query-idxs
    trace-openings
and cand: partial-trace-table-candidate trace-table trace-openings
and not-low:  $\neg$  trace-table-low-degree trace-table
and without:
    generic-fri-sampled-assignment-conflict-without-same-layer
    trace-table trace-roots trace-bs trace-final fri-query-idxs
    trace-round-layers
unfolding
    trace-fri-header-tied-sampled-assignment-conflict-without-same-layer-def
by blast
from fri-openings obtain result final-state where out-eq:
    out = Some (result, final-state)
    unfolding accepted-fri-opening-transcript-def by blast
have conflict:
    generic-fri-sampled-assignment-conflict-with-authenticated-same-layer
    trace-table trace-roots trace-bs trace-final fri-query-idxs
    trace-round-layers final-state
by (rule
    generic-fri-sampled-assignment-conflict-with-authenticated-same-layerI-without
    [OF without])
show ?thesis
    unfolding
    trace-fri-header-tied-sampled-assignment-conflict-with-authenticated-same-layer-def
by (intro exI conjI)
    (rule fri-openings, rule out-eq, rule header, rule partial, rule cand,
    rule not-low, rule conflict)
qed

lemma trace-fri-header-tied-authenticated-conflictI-same-layer:
assumes
    trace-fri-header-tied-same-layer-conflict-with-authenticated-chunks s out
shows
    trace-fri-header-tied-sampled-assignment-conflict-with-authenticated-same-layer
    s out
proof –
from assms obtain trace-roots trace-bs trace-final dg composition-roots
    composition-bs composition-final fri-query-idxs trace-round-layers
    composition-round-layers fr as rest trace-openings trace-table result
    final-state
where fri-openings:
    accepted-fri-opening-transcript s out trace-roots trace-bs
    trace-final dg composition-roots composition-bs composition-final
    fri-query-idxs trace-round-layers composition-round-layers
and out-eq: out = Some (result, final-state)
and header:
    verifier-header-transcript s fr trace-roots trace-final as dg

```

composition-roots composition-final rest
and *partial:*
accepted-with-partial-trace-openings s out fr fri-query-idxs
trace-openings
and *cand: partial-trace-table-candidate trace-table trace-openings*
and *not-low: $\neg$ trace-table-low-degree trace-table*
and *same:*
generic-fri-sampled-same-layer-opening-conflict-with-authenticated-chunks
trace-roots trace-bs fri-query-idxs trace-round-layers final-state
unfolding
trace-fri-header-tied-same-layer-conflict-with-authenticated-chunks-def
by *blast*
have *conflict:*
generic-fri-sampled-assignment-conflict-with-authenticated-same-layer
trace-table trace-roots trace-bs trace-final fri-query-idxs
trace-round-layers final-state
by (rule
generic-fri-sampled-assignment-conflict-with-authenticated-same-layerI-same
[OF same])
show *?thesis*
unfolding
trace-fri-header-tied-sampled-assignment-conflict-with-authenticated-same-layer-def
by (intro exI conjI)
(rule fri-openings, rule out-eq, rule header, rule partial, rule cand,
rule not-low, rule conflict)
qed

lemma *trace-fri-header-tied-authenticated-conflict-iff:*
trace-fri-header-tied-sampled-assignment-conflict-with-authenticated-same-layer
s out $\longleftrightarrow$
trace-fri-header-tied-sampled-assignment-conflict-without-same-layer
s out $\vee$
trace-fri-header-tied-same-layer-conflict-with-authenticated-chunks s out
using *trace-fri-header-tied-authenticated-conflict-imp-without-same-or-same-layer*
trace-fri-header-tied-authenticated-conflictI-without-same
trace-fri-header-tied-authenticated-conflictI-same-layer
by *blast*

lemma *wp-trace-fri-header-tied-authenticated-conflict-bound-from-without-same-and-merkle:*
assumes *without-bound:*
wp-event verify-monad
(trace-fri-header-tied-sampled-assignment-conflict-without-same-layer s)
s $\leq$ W
and *merkle-bound:*
wp-event verify-monad (partial-merkle-inconsistency-bad s) s $\leq$ M
shows
wp-event verify-monad
(trace-fri-header-tied-sampled-assignment-conflict-with-authenticated-same-layer s)
s $\leq$ W + M

proof –
have *wp-event verify-monad*
 (*trace-fri-header-tied-sampled-assignment-conflict-with-authenticated-same-layer s*)
 $s \leq$
wp-event verify-monad
 ($\lambda out.$
 trace-fri-header-tied-sampled-assignment-conflict-without-same-layer
 s out $\vee$
 trace-fri-header-tied-same-layer-conflict-with-authenticated-chunks
 s out) *s*
by (*rule wp-event-mono*)
 (*rule*
 trace-fri-header-tied-authenticated-conflict-imp-without-same-or-same-layer)
also have ... $\leq$
wp-event verify-monad
 (*trace-fri-header-tied-sampled-assignment-conflict-without-same-layer s*)
 $s +$
wp-event verify-monad
 (*trace-fri-header-tied-same-layer-conflict-with-authenticated-chunks s*)
 s
by (*rule wp-event-union-bound*)
also have ... $\leq W + M$
by (*rule add-mono*[*OF without-bound*
 wp-trace-fri-header-tied-same-layer-conflict-with-authenticated-chunks-bound-from-partial-merkle
 [*OF merkle-bound*]])
finally show *?thesis* .
qed

definition *trace-fri-verifier-tied-sampled-assignment-conflict-with-authenticated-same-layer*
 $:: ('f, 'a) \text{ protocol-channel-scheme} \Rightarrow$
 ($\text{unit list} \times ('f, 'a) \text{ protocol-channel-scheme}$) $\text{option} \Rightarrow \text{bool}$

where

trace-fri-verifier-tied-sampled-assignment-conflict-with-authenticated-same-layer
 $s \text{ out} \longleftrightarrow$
 ($\exists$ *trace-roots trace-bs trace-final dg composition-roots*
 composition-bs composition-final fri-query-idxs trace-round-layers
 composition-round-layers fr trace-openings trace-table result
 final-state.
accepted-fri-opening-transcript s out trace-roots trace-bs
 trace-final dg composition-roots composition-bs composition-final
 fri-query-idxs trace-round-layers composition-round-layers $\wedge$
out = Some (result, final-state) $\wedge$
accepted-with-partial-trace-openings s out fr fri-query-idxs
 trace-openings $\wedge$
partial-trace-table-candidate trace-table trace-openings $\wedge$
 $\neg$ *trace-table-low-degree trace-table* $\wedge$
generic-fri-sampled-assignment-conflict-with-authenticated-same-layer
 trace-table trace-roots trace-bs trace-final fri-query-idxs
 trace-round-layers final-state)

lemma *trace-fri-verifier-tied-sampled-assignment-conflict-with-authenticated-same-layer-imp-verifier-tied:*

assumes

trace-fri-verifier-tied-sampled-assignment-conflict-with-authenticated-same-layer
s out

shows *trace-fri-verifier-tied-sampled-assignment-conflict s out*

proof –

from *assms* **obtain** *trace-roots trace-bs trace-final dg composition-roots*
composition-bs composition-final fri-query-idxs trace-round-layers
composition-round-layers fr trace-openings trace-table result
final-state

where *fri-openings:*

accepted-fri-opening-transcript s out trace-roots trace-bs
trace-final dg composition-roots composition-bs composition-final
fri-query-idxs trace-round-layers composition-round-layers

and *out-eq:* *out = Some (result, final-state)*

and *partial:*

accepted-with-partial-trace-openings s out fr fri-query-idxs
trace-openings

and *cand:* *partial-trace-table-candidate trace-table trace-openings*

and *not-low:* $\neg$ *trace-table-low-degree trace-table*

and *conflict:*

generic-fri-sampled-assignment-conflict-with-authenticated-same-layer
trace-table trace-roots trace-bs trace-final fri-query-idxs
trace-round-layers final-state

unfolding

trace-fri-verifier-tied-sampled-assignment-conflict-with-authenticated-same-layer-def

by *blast*

have *sampled:*

generic-fri-sampled-assignment-conflict trace-table trace-roots
trace-bs trace-final fri-query-idxs trace-round-layers

by (*rule*

generic-fri-sampled-assignment-conflict-with-authenticated-same-layer-imp-sampled
 $[OF\ conflict])$

show *?thesis*

unfolding *trace-fri-verifier-tied-sampled-assignment-conflict-def*

by (*intro exI conjI*)

(*rule fri-openings*, *rule partial*, *rule cand*, *rule not-low*,
rule sampled)

qed

lemma *wp-trace-fri-verifier-tied-sampled-assignment-conflict-with-authenticated-same-layer-bound-from-verifier-tied:*

assumes *verifier-bound:*

wp-event verify-monad
 $(trace-fri-verifier-tied-sampled-assignment-conflict\ s)\ s \leq C$

shows

wp-event verify-monad
 $(trace-fri-verifier-tied-sampled-assignment-conflict-with-authenticated-same-layer\ s)$
 $s \leq C$

proof –
have *wp-event verify-monad*
 (*trace-fri-verifier-tied-sampled-assignment-conflict-with-authenticated-same-layer s*)
 $s \leq$
wp-event verify-monad
 (*trace-fri-verifier-tied-sampled-assignment-conflict s*) *s*
by (*rule wp-event-mono*)
 (*rule*
trace-fri-verifier-tied-sampled-assignment-conflict-with-authenticated-same-layer-imp-verifier-tied)
then show *?thesis*
by (*rule order-trans[OF - verifier-bound]*)
qed

definition *trace-fri-verifier-tied-same-layer-conflict-with-authenticated-chunks*

$:: (f, 'a) \text{ protocol-channel-scheme} \Rightarrow$
 (*unit list* $\times$ (*f, 'a*) *protocol-channel-scheme*) *option* $\Rightarrow$ *bool*

where

trace-fri-verifier-tied-same-layer-conflict-with-authenticated-chunks
 $s \text{ out} \longleftrightarrow$
 $(\exists \text{ trace-roots trace-bs trace-final dg composition-roots}$
composition-bs composition-final fri-query-idxs trace-round-layers
composition-round-layers fr trace-openings trace-table result
final-state.
accepted-fri-opening-transcript s out trace-roots trace-bs
trace-final dg composition-roots composition-bs composition-final
fri-query-idxs trace-round-layers composition-round-layers $\wedge$
 $\text{out} = \text{Some}(\text{result}, \text{final-state}) \wedge$
accepted-with-partial-trace-openings s out fr fri-query-idxs
trace-openings $\wedge$
partial-trace-table-candidate trace-table trace-openings $\wedge$
 $\neg \text{trace-table-low-degree trace-table} \wedge$
generic-fri-sampled-same-layer-opening-conflict-with-authenticated-chunks
trace-roots trace-bs fri-query-idxs trace-round-layers final-state)

lemma *trace-fri-verifier-tied-same-layer-conflict-with-authenticated-chunks-imp-partial-merkle:*

assumes

trace-fri-verifier-tied-same-layer-conflict-with-authenticated-chunks
 $s \text{ out}$

shows *partial-merkle-inconsistency-bad s out*

proof –

from *assms* **obtain** *trace-roots trace-bs trace-final dg composition-roots*
composition-bs composition-final fri-query-idxs trace-round-layers
composition-round-layers fr trace-openings trace-table result
final-state

where *fri-openings:*

accepted-fri-opening-transcript s out trace-roots trace-bs
trace-final dg composition-roots composition-bs composition-final
fri-query-idxs trace-round-layers composition-round-layers

and *out-eq:* $\text{out} = \text{Some}(\text{result}, \text{final-state})$

and *conflict*:
generic-fri-sampled-same-layer-opening-conflict-with-authenticated-chunks
trace-roots trace-bs fri-query-idxs trace-round-layers final-state
unfolding
trace-fri-verifier-tied-same-layer-conflict-with-authenticated-chunks-def
by *blast*
have *len: length trace-bs = length trace-roots*
using *accepted-fri-opening-transcript-shapes(2)[OF fri-openings]* .
have *partial-merkle-inconsistency-bad s (Some (result, final-state))*
by (rule
generic-fri-sampled-same-layer-opening-conflict-with-authenticated-chunks-imp-partial-merkle
[OF conflict len])
then show *?thesis*
by (*simp add: out-eq*)
qed

lemma *wp-trace-fri-verifier-tied-same-layer-conflict-with-authenticated-chunks-bound-from-partial-merkle*:
assumes *merkle-bound*:
wp-event verify-monad (partial-merkle-inconsistency-bad s) s ≤ M
shows
wp-event verify-monad
(trace-fri-verifier-tied-same-layer-conflict-with-authenticated-chunks s)
s ≤ M
proof –
have *wp-event verify-monad*
(trace-fri-verifier-tied-same-layer-conflict-with-authenticated-chunks s)
s ≤ wp-event verify-monad (partial-merkle-inconsistency-bad s) s
by (rule *wp-event-mono*)
(rule
trace-fri-verifier-tied-same-layer-conflict-with-authenticated-chunks-imp-partial-merkle)
then show *?thesis*
by (rule *order-trans[OF - merkle-bound]*)
qed

definition *composition-fri-verifier-tied-sampled-assignment-conflict-with-authenticated-same-layer*
:: (*'f, 'a*) *protocol-channel-scheme* ⇒
(*unit list* × (*'f, 'a*) *protocol-channel-scheme*) *option* ⇒ *bool*
where
composition-fri-verifier-tied-sampled-assignment-conflict-with-authenticated-same-layer
s out ⇔
(∃ *trace-roots trace-bs trace-final fri-dg composition-roots*
composition-bs composition-final fri-query-idxs trace-round-layers
composition-round-layers fr as trace-openings composition-openings
trace-table composition-table result final-state.
accepted-fri-opening-transcript s out trace-roots trace-bs
trace-final fri-dg composition-roots composition-bs
composition-final fri-query-idxs trace-round-layers
composition-round-layers ∧
out = Some (result, final-state) ∧

accepted-with-partial-initial-openings-aligned *s out fr trace-roots*
trace-final as fri-dg composition-roots composition-final fri-query-idxs
trace-openings composition-openings $\wedge$
partial-trace-table-candidate trace-table trace-openings $\wedge$
partial-composition-table-candidate composition-table
composition-openings $\wedge$
trace-table-low-degree trace-table $\wedge$
 $\neg$ *composition-table-low-degree maxDegree composition-table* $\wedge$
generic-fri-sampled-assignment-conflict-with-authenticated-same-layer
composition-table composition-roots composition-bs
composition-final fri-query-idxs composition-round-layers
final-state)

definition *composition-fri-verifier-tied-sampled-assignment-conflict-without-same-layer*

$:: ('f, 'a) \text{ protocol-channel-scheme} \Rightarrow$
 $(\text{unit list} \times ('f, 'a) \text{ protocol-channel-scheme}) \text{ option} \Rightarrow \text{bool}$

where

composition-fri-verifier-tied-sampled-assignment-conflict-without-same-layer
s out $\longleftrightarrow$
 $(\exists \text{ trace-roots trace-bs trace-final fri-dg composition-roots}$
 $\text{composition-bs composition-final fri-query-idxs trace-round-layers}$
 $\text{composition-round-layers fr as trace-openings composition-openings}$
 $\text{trace-table composition-table.}$
accepted-fri-opening-transcript *s out trace-roots trace-bs*
trace-final fri-dg composition-roots composition-bs
composition-final fri-query-idxs trace-round-layers
composition-round-layers $\wedge$
accepted-with-partial-initial-openings-aligned *s out fr trace-roots*
trace-final as fri-dg composition-roots composition-final fri-query-idxs
trace-openings composition-openings $\wedge$
partial-trace-table-candidate trace-table trace-openings $\wedge$
partial-composition-table-candidate composition-table
composition-openings $\wedge$
trace-table-low-degree trace-table $\wedge$
 $\neg$ *composition-table-low-degree maxDegree composition-table* $\wedge$
generic-fri-sampled-assignment-conflict-without-same-layer
composition-table composition-roots composition-bs
composition-final fri-query-idxs composition-round-layers)

definition *composition-fri-verifier-tied-sampled-base-opening-conflict*

$:: ('f, 'a) \text{ protocol-channel-scheme} \Rightarrow$
 $(\text{unit list} \times ('f, 'a) \text{ protocol-channel-scheme}) \text{ option} \Rightarrow \text{bool}$

where

composition-fri-verifier-tied-sampled-base-opening-conflict *s out*
 $\longleftrightarrow$
 $(\exists \text{ trace-roots trace-bs trace-final fri-dg composition-roots}$
 $\text{composition-bs composition-final fri-query-idxs trace-round-layers}$
 $\text{composition-round-layers fr as trace-openings composition-openings}$
 $\text{trace-table composition-table.}$

accepted-fri-opening-transcript s out trace-roots trace-bs
trace-final fri-dg composition-roots composition-bs
composition-final fri-query-idxs trace-round-layers
composition-round-layers $\wedge$
accepted-with-partial-initial-openings-aligned s out fr trace-roots
trace-final as fri-dg composition-roots composition-final fri-query-idxs
trace-openings composition-openings $\wedge$
partial-trace-table-candidate trace-table trace-openings $\wedge$
partial-composition-table-candidate composition-table
composition-openings $\wedge$
trace-table-low-degree trace-table $\wedge$
 $\neg$ *composition-table-low-degree maxDegree composition-table* $\wedge$
generic-fri-sampled-base-opening-conflict composition-table
composition-roots composition-bs fri-query-idxs
composition-round-layers)

definition *composition-fri-verifier-tied-sampled-next-value-conflict*

$:: ('f, 'a) \text{ protocol-channel-scheme} \Rightarrow$
 $(\text{unit list} \times ('f, 'a) \text{ protocol-channel-scheme}) \text{ option} \Rightarrow \text{bool}$

where

composition-fri-verifier-tied-sampled-next-value-conflict s out
 $\longleftrightarrow$
 $(\exists \text{ trace-roots trace-bs trace-final fri-dg composition-roots}$
 $\text{ composition-bs composition-final fri-query-idxs trace-round-layers}$
 $\text{ composition-round-layers fr as trace-openings composition-openings}$
 $\text{ trace-table composition-table.}$
accepted-fri-opening-transcript s out trace-roots trace-bs
trace-final fri-dg composition-roots composition-bs
composition-final fri-query-idxs trace-round-layers
composition-round-layers $\wedge$
accepted-with-partial-initial-openings-aligned s out fr trace-roots
trace-final as fri-dg composition-roots composition-final fri-query-idxs
trace-openings composition-openings $\wedge$
partial-trace-table-candidate trace-table trace-openings $\wedge$
partial-composition-table-candidate composition-table
composition-openings $\wedge$
trace-table-low-degree trace-table $\wedge$
 $\neg$ *composition-table-low-degree maxDegree composition-table* $\wedge$
generic-fri-sampled-next-value-conflict composition-roots
composition-bs fri-query-idxs composition-round-layers)

definition *composition-fri-verifier-tied-sampled-successor-opening-conflict*

$:: ('f, 'a) \text{ protocol-channel-scheme} \Rightarrow$
 $(\text{unit list} \times ('f, 'a) \text{ protocol-channel-scheme}) \text{ option} \Rightarrow \text{bool}$

where

composition-fri-verifier-tied-sampled-successor-opening-conflict s out
 $\longleftrightarrow$
 $(\exists \text{ trace-roots trace-bs trace-final fri-dg composition-roots}$
 $\text{ composition-bs composition-final fri-query-idxs trace-round-layers}$

composition-round-layers fr as trace-openings composition-openings
trace-table composition-table.
accepted-fri-opening-transcript s out trace-roots trace-bs
trace-final fri-dg composition-roots composition-bs
composition-final fri-query-idxs trace-round-layers
composition-round-layers $\wedge$
accepted-with-partial-initial-openings-aligned s out fr trace-roots
trace-final as fri-dg composition-roots composition-final fri-query-idxs
trace-openings composition-openings $\wedge$
partial-trace-table-candidate trace-table trace-openings $\wedge$
partial-composition-table-candidate composition-table
composition-openings $\wedge$
trace-table-low-degree trace-table $\wedge$
 $\neg$ *composition-table-low-degree maxDegree composition-table* $\wedge$
generic-fri-sampled-successor-opening-conflict composition-roots
composition-bs fri-query-idxs composition-round-layers)

definition *composition-fri-verifier-tied-sampled-final-value-conflict*

$:: (f, 'a) \text{ protocol-channel-scheme} \Rightarrow$
 $(\text{unit list} \times (f, 'a) \text{ protocol-channel-scheme}) \text{ option} \Rightarrow \text{bool}$

where

composition-fri-verifier-tied-sampled-final-value-conflict s out
 $\iff$
 $(\exists \text{ trace-roots trace-bs trace-final fri-dg composition-roots}$
 $\text{ composition-bs composition-final fri-query-idxs trace-round-layers}$
 $\text{ composition-round-layers fr as trace-openings composition-openings}$
 $\text{ trace-table composition-table.}$
accepted-fri-opening-transcript s out trace-roots trace-bs
trace-final fri-dg composition-roots composition-bs
composition-final fri-query-idxs trace-round-layers
composition-round-layers $\wedge$
accepted-with-partial-initial-openings-aligned s out fr trace-roots
trace-final as fri-dg composition-roots composition-final fri-query-idxs
trace-openings composition-openings $\wedge$
partial-trace-table-candidate trace-table trace-openings $\wedge$
partial-composition-table-candidate composition-table
composition-openings $\wedge$
trace-table-low-degree trace-table $\wedge$
 $\neg$ *composition-table-low-degree maxDegree composition-table* $\wedge$
generic-fri-sampled-final-value-conflict composition-roots
composition-bs composition-final fri-query-idxs
composition-round-layers)

lemma *composition-fri-verifier-tied-without-same-imp-branch:*

assumes

composition-fri-verifier-tied-sampled-assignment-conflict-without-same-layer
s out

shows

composition-fri-verifier-tied-sampled-base-opening-conflict s out $\vee$

composition-fri-verifier-tied-sampled-next-value-conflict s out $\vee$
composition-fri-verifier-tied-sampled-successor-opening-conflict
s out $\vee$
composition-fri-verifier-tied-sampled-final-value-conflict s out
proof –
from *assms* **obtain** *trace-roots trace-bs trace-final fri-dg*
composition-roots composition-bs composition-final fri-query-idxs
trace-round-layers composition-round-layers fr as trace-openings
composition-openings trace-table composition-table
where *fri-openings*:
accepted-fri-opening-transcript s out trace-roots trace-bs
trace-final fri-dg composition-roots composition-bs
composition-final fri-query-idxs trace-round-layers
composition-round-layers
and *aligned*:
accepted-with-partial-initial-openings-aligned s out fr trace-roots
trace-final as fri-dg composition-roots composition-final fri-query-idxs
trace-openings composition-openings
and *trace-cand*:
partial-trace-table-candidate trace-table trace-openings
and *comp-cand*:
partial-composition-table-candidate composition-table
composition-openings
and *trace-low*: *trace-table-low-degree trace-table*
and *comp-not-low*:
 $\neg$ *composition-table-low-degree maxDegree composition-table*
and *without*:
generic-fri-sampled-assignment-conflict-without-same-layer
composition-table composition-roots composition-bs composition-final
fri-query-idxs composition-round-layers
unfolding
composition-fri-verifier-tied-sampled-assignment-conflict-without-same-layer-def
by *blast*
have *split*:
generic-fri-sampled-base-opening-conflict composition-table
composition-roots composition-bs fri-query-idxs
composition-round-layers $\vee$
generic-fri-sampled-next-value-conflict composition-roots
composition-bs fri-query-idxs composition-round-layers $\vee$
generic-fri-sampled-successor-opening-conflict composition-roots
composition-bs fri-query-idxs composition-round-layers $\vee$
generic-fri-sampled-final-value-conflict composition-roots
composition-bs composition-final fri-query-idxs
composition-round-layers
by (*rule generic-fri-sampled-assignment-conflict-without-same-layer-split*
 $[OF\ without]$)
then show *?thesis*
proof
assume *base*:

```

generic-fri-sampled-base-opening-conflict composition-table
composition-roots composition-bs fri-query-idxs
composition-round-layers
then show ?thesis
unfolding composition-fri-verifier-tied-sampled-base-opening-conflict-def
by (intro disjI1 exI conjI)
(rule fri-openings, rule aligned, rule trace-cand, rule comp-cand,
rule trace-low, rule comp-not-low, rule base)
next
assume rest-split:
generic-fri-sampled-next-value-conflict composition-roots
composition-bs fri-query-idxs composition-round-layers ∨
generic-fri-sampled-successor-opening-conflict composition-roots
composition-bs fri-query-idxs composition-round-layers ∨
generic-fri-sampled-final-value-conflict composition-roots
composition-bs composition-final fri-query-idxs
composition-round-layers
then show ?thesis
proof
assume next-conflict:
generic-fri-sampled-next-value-conflict composition-roots
composition-bs fri-query-idxs composition-round-layers
then show ?thesis
unfolding
composition-fri-verifier-tied-sampled-next-value-conflict-def
by (intro disjI2 disjI1 exI conjI)
(rule fri-openings, rule aligned, rule trace-cand, rule comp-cand,
rule trace-low, rule comp-not-low, rule next-conflict)
next
assume rest-split':
generic-fri-sampled-successor-opening-conflict composition-roots
composition-bs fri-query-idxs composition-round-layers ∨
generic-fri-sampled-final-value-conflict composition-roots
composition-bs composition-final fri-query-idxs
composition-round-layers
then show ?thesis
proof
assume successor:
generic-fri-sampled-successor-opening-conflict composition-roots
composition-bs fri-query-idxs composition-round-layers
then show ?thesis
unfolding
composition-fri-verifier-tied-sampled-successor-opening-conflict-def
by (intro disjI2 disjI2 disjI1 exI conjI)
(rule fri-openings, rule aligned, rule trace-cand,
rule comp-cand, rule trace-low, rule comp-not-low,
rule successor)
next
assume final-conflict:

```

```

    generic-fri-sampled-final-value-conflict composition-roots
    composition-bs composition-final fri-query-idxs
    composition-round-layers
then show ?thesis
  unfolding
    composition-fri-verifier-tied-sampled-final-value-conflict-def
  by (intro disjI2 disjI2 disjI2 exI conjI)
    (rule fri-openings, rule aligned, rule trace-cand,
     rule comp-cand, rule trace-low, rule comp-not-low,
     rule final-conflict)
qed
qed
qed
qed

lemma wp-composition-fri-verifier-tied-without-same-bound-from-branches:
assumes base-bound:
  wp-event verify-monad
  (composition-fri-verifier-tied-sampled-base-opening-conflict s)  $s \leq B$ 
and next-bound:
  wp-event verify-monad
  (composition-fri-verifier-tied-sampled-next-value-conflict s)  $s \leq N$ 
and successor-bound:
  wp-event verify-monad
  (composition-fri-verifier-tied-sampled-successor-opening-conflict s)
   $s \leq S$ 
and final-bound:
  wp-event verify-monad
  (composition-fri-verifier-tied-sampled-final-value-conflict s)  $s \leq F$ 
shows
  wp-event verify-monad
  (composition-fri-verifier-tied-sampled-assignment-conflict-without-same-layer s)
   $s \leq B + N + S + F$ 
proof –
have wp-event verify-monad
  (composition-fri-verifier-tied-sampled-assignment-conflict-without-same-layer s)
   $s \leq$ 
  wp-event verify-monad
  ( $\lambda$ out.
    composition-fri-verifier-tied-sampled-base-opening-conflict
    s out  $\vee$ 
    composition-fri-verifier-tied-sampled-next-value-conflict
    s out  $\vee$ 
    composition-fri-verifier-tied-sampled-successor-opening-conflict
    s out  $\vee$ 
    composition-fri-verifier-tied-sampled-final-value-conflict
    s out) s
by (rule wp-event-mono)
  (rule composition-fri-verifier-tied-without-same-imp-branch)

```

also have ... $\leq$
 $\text{wp-event verify-monad}$
 $(\text{composition-fri-verifier-tied-sampled-base-opening-conflict } s) \ s +$
 $\text{wp-event verify-monad}$
 $(\text{composition-fri-verifier-tied-sampled-next-value-conflict } s) \ s +$
 $\text{wp-event verify-monad}$
 $(\text{composition-fri-verifier-tied-sampled-successor-opening-conflict } s)$
 $s +$
 $\text{wp-event verify-monad}$
 $(\text{composition-fri-verifier-tied-sampled-final-value-conflict } s) \ s$
by (rule *wp-event-union-bound4*)
also have ... $\leq B + N + S + F$
by (intro *add-mono base-bound next-bound successor-bound final-bound*)
finally show ?thesis .
qed

lemma *composition-fri-verifier-tied-without-sameI-base:*
assumes *composition-fri-verifier-tied-sampled-base-opening-conflict s out*
shows
composition-fri-verifier-tied-sampled-assignment-conflict-without-same-layer
s out
using *assms*
unfolding
composition-fri-verifier-tied-sampled-base-opening-conflict-def
composition-fri-verifier-tied-sampled-assignment-conflict-without-same-layer-def
by (blast intro:
generic-fri-sampled-assignment-conflict-without-same-layerI-base)

lemma *composition-fri-verifier-tied-without-sameI-next:*
assumes *composition-fri-verifier-tied-sampled-next-value-conflict s out*
shows
composition-fri-verifier-tied-sampled-assignment-conflict-without-same-layer
s out
using *assms*
unfolding
composition-fri-verifier-tied-sampled-next-value-conflict-def
composition-fri-verifier-tied-sampled-assignment-conflict-without-same-layer-def
by (blast intro:
generic-fri-sampled-assignment-conflict-without-same-layerI-next)

lemma *composition-fri-verifier-tied-without-sameI-successor:*
assumes
composition-fri-verifier-tied-sampled-successor-opening-conflict s out
shows
composition-fri-verifier-tied-sampled-assignment-conflict-without-same-layer
s out
using *assms*
unfolding
composition-fri-verifier-tied-sampled-successor-opening-conflict-def

composition-fri-verifier-tied-sampled-assignment-conflict-without-same-layer-def
by (blast intro:
generic-fri-sampled-assignment-conflict-without-same-layerI-successor)

lemma *composition-fri-verifier-tied-without-sameI-final*:
assumes *composition-fri-verifier-tied-sampled-final-value-conflict s out*
shows
composition-fri-verifier-tied-sampled-assignment-conflict-without-same-layer
s out
using *assms*
unfolding
composition-fri-verifier-tied-sampled-final-value-conflict-def
composition-fri-verifier-tied-sampled-assignment-conflict-without-same-layer-def
by (blast intro:
generic-fri-sampled-assignment-conflict-without-same-layerI-final)

lemma *composition-fri-verifier-tied-without-same-iff-branches*:
composition-fri-verifier-tied-sampled-assignment-conflict-without-same-layer
s out
 $\longleftrightarrow$
composition-fri-verifier-tied-sampled-base-opening-conflict s out $\vee$
composition-fri-verifier-tied-sampled-next-value-conflict s out $\vee$
composition-fri-verifier-tied-sampled-successor-opening-conflict s out $\vee$
composition-fri-verifier-tied-sampled-final-value-conflict s out
using *composition-fri-verifier-tied-without-same-imp-branch*
composition-fri-verifier-tied-without-sameI-base
composition-fri-verifier-tied-without-sameI-next
composition-fri-verifier-tied-without-sameI-successor
composition-fri-verifier-tied-without-sameI-final
by *blast*

lemma *composition-fri-verifier-tied-base-conflict-obtains-authenticated-first-chunk*:
assumes *base*:
composition-fri-verifier-tied-sampled-base-opening-conflict
s (Some (result, final-state))
obtains *trace-roots trace-bs trace-final fri-dg composition-roots*
composition-bs composition-final fri-query-idxs trace-round-layers
composition-round-layers fr as trace-openings composition-openings
trace-table composition-table round-idx xp xp-path xn xn-path
auth-chunk
where
accepted-fri-opening-transcript s (Some (result, final-state))
trace-roots trace-bs trace-final fri-dg composition-roots
composition-bs composition-final fri-query-idxs trace-round-layers
composition-round-layers
accepted-with-partial-initial-openings-aligned s
(Some (result, final-state)) fr trace-roots trace-final as fri-dg
composition-roots composition-final fri-query-idxs trace-openings
composition-openings

```

partial-trace-table-candidate trace-table trace-openings
partial-composition-table-candidate composition-table
  composition-openings
trace-table-low-degree trace-table
¬ composition-table-low-degree maxDegree composition-table
round-idx < length fri-query-idxs
0 < length composition-bs
fri-layer-step-evidence
  (composition-roots ! 0)
  (composition-bs ! 0)
  (fri-evidence-layer-len composition-roots 0)
  (fri-evidence-layer-idx composition-roots fri-query-idxs round-idx 0)
  1
  (fri-sibling-index (fri-evidence-layer-len composition-roots 0)
    (fri-evidence-layer-idx composition-roots fri-query-idxs round-idx 0))
  xp xp-path xn xn-path
  (fri-evidence-next-idx composition-roots fri-query-idxs round-idx 0)
  (fri-evidence-next-value composition-roots composition-bs
    fri-query-idxs round-idx 0 xp xn)
  (composition-round-layers ! round-idx ! 0)
¬ fri-opening-matches-table
  (fri-evidence-layer-len composition-roots 0)
  (fri-evidence-layer-idx composition-roots fri-query-idxs round-idx 0)
  composition-table xp xn
fri-layer-chunk-authenticated (composition-roots ! 0)
  (fri-evidence-layer-len composition-roots 0)
  (fri-evidence-layer-idx composition-roots fri-query-idxs round-idx 0)
  auth-chunk final-state
proof —
from base obtain trace-roots trace-bs trace-final fri-dg
  composition-roots composition-bs composition-final fri-query-idxs
  trace-round-layers composition-round-layers fr as trace-openings
  composition-openings trace-table composition-table
where fri-openings:
  accepted-fri-opening-transcript s (Some (result, final-state))
  trace-roots trace-bs trace-final fri-dg composition-roots
  composition-bs composition-final fri-query-idxs trace-round-layers
  composition-round-layers
and aligned:
  accepted-with-partial-initial-openings-aligned s
  (Some (result, final-state)) fr trace-roots trace-final as fri-dg
  composition-roots composition-final fri-query-idxs trace-openings
  composition-openings
and trace-cand:
  partial-trace-table-candidate trace-table trace-openings
and comp-cand:
  partial-composition-table-candidate composition-table
  composition-openings
and trace-low: trace-table-low-degree trace-table

```



```

and comp-not-low:
   $\neg$  composition-table-low-degree maxDegree composition-table
and generic:
  generic-fri-sampled-base-opening-conflict composition-table
    composition-roots composition-bs fri-query-idxs
    composition-round-layers
unfolding composition-fri-verifier-tied-sampled-base-opening-conflict-def
by blast
from generic obtain round-idx xp xp-path xn xn-path where
  composition-bs-nonempty:  $0 < \text{length } \text{composition-bs}$ 
and round-bound: round-idx  $< \text{length } \text{fri-query-idxs}$ 
and step:
  fri-layer-step-evidence
    (composition-roots ! 0)
    (composition-bs ! 0)
    (fri-evidence-layer-len composition-roots 0)
    (fri-evidence-layer-idx composition-roots fri-query-idxs round-idx 0)
    1
    (fri-sibling-index (fri-evidence-layer-len composition-roots 0)
      (fri-evidence-layer-idx composition-roots fri-query-idxs round-idx 0))
    xp xp-path xn xn-path
    (fri-evidence-next-idx composition-roots fri-query-idxs round-idx 0)
    (fri-evidence-next-value composition-roots composition-bs
      fri-query-idxs round-idx 0 xp xn)
    (composition-round-layers ! round-idx ! 0)
and no-match:
   $\neg$  fri-opening-matches-table
    (fri-evidence-layer-len composition-roots 0)
    (fri-evidence-layer-idx composition-roots fri-query-idxs round-idx 0)
    composition-table xp xn
unfolding generic-fri-sampled-base-opening-conflict-def
by blast
have roots-nonempty:  $0 < \text{length } \text{composition-roots}$ 
using accepted-fri-opening-transcript-shapes(4)[OF fri-openings]
  composition-bs-nonempty by simp
have round-bound-rounds: round-idx  $< \text{rounds}$ 
using round-bound accepted-fri-opening-transcript-shapes(7)[OF fri-openings]
by simp
obtain auth-chunk where auth:
  fri-layer-chunk-authenticated (composition-roots ! 0)
    (fri-evidence-layer-len composition-roots 0)
    (fri-evidence-layer-idx composition-roots fri-query-idxs round-idx 0)
    auth-chunk final-state
proof –
have layer-bound:  $0 < \text{length } \text{composition-roots}$ 
by (rule roots-nonempty)
have raw-layer:
   $\bigwedge^{\text{raw.}}$ 
     $0 < \text{fri-layer-lengths } (\text{length } \text{composition-roots}) (\text{clength} * \text{scale}) ! 0 \wedge$ 

```

```

    fri-layer-indices (length composition-roots) (index (to-nat raw))
      (clength * scale) ! 0 <
    fri-layer-lengths (length composition-roots) (clength * scale) ! 0
  by (rule fri-first-layer-raw-bound[OF roots-nonempty])
from accepted-fri-opening-transcript-composition-step-with-authenticated-chunk-at
  [OF fri-openings refl round-bound-rounds layer-bound raw-layer]
obtain auth-chunk where
  generic-fri-transcript-step-with-authenticated-chunk composition-roots
    composition-bs fri-query-idxs composition-round-layers final-state
    round-idx 0
  by blast
then obtain auth-chunk where
  fri-layer-chunk-authenticated (composition-roots ! 0)
    (fri-evidence-layer-len composition-roots 0)
    (fri-evidence-layer-idx composition-roots fri-query-idxs round-idx 0)
    auth-chunk final-state
  by (rule generic-fri-transcript-step-with-authenticated-chunkE)
then show ?thesis
  by (rule that)
qed
show ?thesis
  by (rule that[OF fri-openings aligned trace-cand comp-cand trace-low
    comp-not-low round-bound composition-bs-nonempty step no-match auth])
qed

```

definition *composition-fri-verifier-tied-authenticated-base-opening-conflict*

$:: (f, 'a) \text{ protocol-channel-scheme} \Rightarrow$
 $(\text{unit list} \times (f, 'a) \text{ protocol-channel-scheme}) \text{ option} \Rightarrow \text{bool}$

where

composition-fri-verifier-tied-authenticated-base-opening-conflict s out
 $\iff$
 $(\exists \text{ trace-roots trace-bs trace-final fri-dg composition-roots}$
 $\text{ composition-bs composition-final fri-query-idxs trace-round-layers}$
 $\text{ composition-round-layers fr as trace-openings composition-openings}$
 $\text{ trace-table composition-table round-idx xp xp-path xn xn-path}$
 $\text{ result final-state.}$
 $\text{out} = \text{Some (result, final-state)} \wedge$
 $\text{accepted-fri-opening-transcript } s \text{ out trace-roots trace-bs}$
 $\text{ trace-final fri-dg composition-roots composition-bs}$
 $\text{ composition-final fri-query-idxs trace-round-layers}$
 $\text{ composition-round-layers} \wedge$
 $\text{accepted-with-partial-initial-openings-aligned } s \text{ out fr trace-roots}$
 $\text{ trace-final as fri-dg composition-roots composition-final fri-query-idxs}$
 $\text{ trace-openings composition-openings} \wedge$
 $\text{partial-trace-table-candidate trace-table trace-openings} \wedge$
 $\text{partial-composition-table-candidate composition-table}$
 $\text{ composition-openings} \wedge$
 $\text{trace-table-low-degree trace-table} \wedge$
 $\neg \text{ composition-table-low-degree maxDegree composition-table} \wedge$

$round_idx < length\ fri_query_idxs \wedge$
 $0 < length\ composition_bs \wedge$
 $fri_layer_step_evidence$
 $(composition_roots ! 0)$
 $(composition_bs ! 0)$
 $(fri_evidence_layer_len\ composition_roots\ 0)$
 $(fri_evidence_layer_idx\ composition_roots\ fri_query_idxs\ round_idx\ 0)$
 1
 $(fri_sibling_index\ (fri_evidence_layer_len\ composition_roots\ 0)$
 $(fri_evidence_layer_idx\ composition_roots\ fri_query_idxs\ round_idx\ 0))$
 $xp\ xp_path\ xn\ xn_path$
 $(fri_evidence_next_idx\ composition_roots\ fri_query_idxs\ round_idx\ 0)$
 $(fri_evidence_next_value\ composition_roots\ composition_bs$
 $fri_query_idxs\ round_idx\ 0\ xp\ xn)$
 $(composition_round_layers ! round_idx ! 0) \wedge$
 $\neg fri_opening_matches_table$
 $(fri_evidence_layer_len\ composition_roots\ 0)$
 $(fri_evidence_layer_idx\ composition_roots\ fri_query_idxs\ round_idx\ 0)$
 $composition_table\ xp\ xn \wedge$
 $fri_layer_chunk_authenticated\ (composition_roots ! 0)$
 $(fri_evidence_layer_len\ composition_roots\ 0)$
 $(fri_evidence_layer_idx\ composition_roots\ fri_query_idxs\ round_idx\ 0)$
 $(composition_round_layers ! round_idx ! 0)\ final_state)$

definition *composition-fri-verifier-tied-base-chunk-auth-gap*

$:: (f, 'a)\ protocol_channel_scheme \Rightarrow$
 $(unit\ list \times (f, 'a)\ protocol_channel_scheme)\ option \Rightarrow bool$

where

$composition_fri_verifier_tied_base_chunk_auth_gap\ s\ out \longleftrightarrow$
 $composition_fri_verifier_tied_sampled_base_opening_conflict\ s\ out \wedge$
 $\neg composition_fri_verifier_tied_authenticated_base_opening_conflict$
 $s\ out$

lemma *composition-fri-verifier-tied-sampled-base-imp-authenticated-or-auth-gap:*

assumes

$composition_fri_verifier_tied_sampled_base_opening_conflict\ s\ out$

shows

$composition_fri_verifier_tied_authenticated_base_opening_conflict\ s\ out \vee$
 $composition_fri_verifier_tied_base_chunk_auth_gap\ s\ out$

using *assms*

unfolding *composition-fri-verifier-tied-base-chunk-auth-gap-def*

by *blast*

lemma *wp-composition-fri-verifier-tied-sampled-base-bound-from-authenticated-and-auth-gap:*

fixes $A\ G :: prob$

assumes *authenticated-bound:*

$wp_event\ verify_monad$

$(composition_fri_verifier_tied_authenticated_base_opening_conflict\ s)$

$s \leq A$

and *gap-bound*:
wp-event verify-monad
(composition-fri-verifier-tied-base-chunk-auth-gap s) s ≤ G
shows
wp-event verify-monad
(composition-fri-verifier-tied-sampled-base-opening-conflict s) s
 $\leq A + G$
proof –
have *wp-event verify-monad*
(composition-fri-verifier-tied-sampled-base-opening-conflict s) s ≤
wp-event verify-monad
(λout.
composition-fri-verifier-tied-authenticated-base-opening-conflict
s out ∨
composition-fri-verifier-tied-base-chunk-auth-gap s out) s
by (rule *wp-event-mono*)
(rule composition-fri-verifier-tied-sampled-base-imp-authenticated-or-auth-gap)
also have ... $\leq$
wp-event verify-monad
(composition-fri-verifier-tied-authenticated-base-opening-conflict s) s +
wp-event verify-monad
(composition-fri-verifier-tied-base-chunk-auth-gap s) s
by (rule *wp-event-union-bound*)
also have ... $\leq A + G$
by (intro *add-mono authenticated-bound gap-bound*)
finally show ?thesis .
qed

lemma *composition-fri-verifier-tied-authenticated-base-conflict-imp-sampled-base*:
assumes
composition-fri-verifier-tied-authenticated-base-opening-conflict s out
shows *composition-fri-verifier-tied-sampled-base-opening-conflict s out*
proof –
from *assms* **obtain** *trace-roots trace-bs trace-final fri-dg composition-roots*
composition-bs composition-final fri-query-idxs trace-round-layers
composition-round-layers fr as trace-openings composition-openings
trace-table composition-table round-idx xp xp-path xn xn-path
result final-state
where *out-eq*: *out = Some (result, final-state)*
and *fri-openings*:
accepted-fri-opening-transcript s out trace-roots trace-bs
trace-final fri-dg composition-roots composition-bs
composition-final fri-query-idxs trace-round-layers
composition-round-layers
and *aligned*:
accepted-with-partial-initial-openings-aligned s out fr trace-roots
trace-final as fri-dg composition-roots composition-final fri-query-idxs
trace-openings composition-openings
and *trace-cand*:

```

    partial-trace-table-candidate trace-table trace-openings
and comp-cand:
    partial-composition-table-candidate composition-table
    composition-openings
and trace-low: trace-table-low-degree trace-table
and comp-not-low:
    ¬ composition-table-low-degree maxDegree composition-table
and round-bound: round-idx < length fri-query-idxs
and composition-bs-nonempty: 0 < length composition-bs
and step:
    fri-layer-step-evidence
    (composition-roots ! 0)
    (composition-bs ! 0)
    (fri-evidence-layer-len composition-roots 0)
    (fri-evidence-layer-idx composition-roots fri-query-idxs round-idx 0)
    1
    (fri-sibling-index (fri-evidence-layer-len composition-roots 0)
      (fri-evidence-layer-idx composition-roots fri-query-idxs round-idx 0))
    xp xp-path xn xn-path
    (fri-evidence-next-idx composition-roots fri-query-idxs round-idx 0)
    (fri-evidence-next-value composition-roots composition-bs
      fri-query-idxs round-idx 0 xp xn)
    (composition-round-layers ! round-idx ! 0)
and no-match:
    ¬ fri-opening-matches-table
    (fri-evidence-layer-len composition-roots 0)
    (fri-evidence-layer-idx composition-roots fri-query-idxs round-idx 0)
    composition-table xp xn
and auth:
    fri-layer-chunk-authenticated (composition-roots ! 0)
    (fri-evidence-layer-len composition-roots 0)
    (fri-evidence-layer-idx composition-roots fri-query-idxs round-idx 0)
    (composition-round-layers ! round-idx ! 0) final-state
unfolding
    composition-fri-verifier-tied-authenticated-base-opening-conflict-def
by auto
have base:
    generic-fri-sampled-base-opening-conflict composition-table
    composition-roots composition-bs fri-query-idxs composition-round-layers
unfolding generic-fri-sampled-base-opening-conflict-def
by (intro exI[of - round-idx] exI[of - xp] exI[of - xp-path]
    exI[of - xn] exI[of - xn-path] conjI)
    (use round-bound composition-bs-nonempty step no-match in auto)
show ?thesis
unfolding composition-fri-verifier-tied-sampled-base-opening-conflict-def
by (intro exI conjI)
    (use out-eq fri-openings aligned trace-cand comp-cand trace-low
    comp-not-low base in auto)
qed

```

lemma *composition-fri-verifier-tied-authenticated-base-conflict-imp-partial-merkle:*

assumes *base:*

composition-fri-verifier-tied-authenticated-base-opening-conflict
s (Some (result, final-state))

shows

partial-merkle-inconsistency-bad s (Some (result, final-state))

proof –

show *?thesis*

proof (*rule ccontr*)

assume *no-bad:*

¬ *partial-merkle-inconsistency-bad s*
(Some (result, final-state))

obtain *trace-roots trace-bs trace-final fri-dg composition-roots*
composition-bs composition-final fri-query-idxs trace-round-layers
composition-round-layers fr as trace-openings composition-openings
trace-table composition-table round-idx xp xp-path xn xn-path

where *aligned:*

accepted-with-partial-initial-openings-aligned s
(Some (result, final-state)) fr trace-roots trace-final as fri-dg
composition-roots composition-final fri-query-idxs trace-openings
composition-openings

and *comp-cand:*

partial-composition-table-candidate composition-table
composition-openings

and *round-bound: round-idx < length fri-query-idxs*

and *step:*

fri-layer-step-evidence
(composition-roots ! 0)
(composition-bs ! 0)
(fri-evidence-layer-len composition-roots 0)
(fri-evidence-layer-idx composition-roots fri-query-idxs round-idx 0)
1
(fri-sibling-index (fri-evidence-layer-len composition-roots 0)
(fri-evidence-layer-idx composition-roots fri-query-idxs round-idx 0))
xp xp-path xn xn-path
(fri-evidence-next-idx composition-roots fri-query-idxs round-idx 0)
(fri-evidence-next-value composition-roots composition-bs
fri-query-idxs round-idx 0 xp xn)
(composition-round-layers ! round-idx ! 0)

and *no-match:*

¬ *fri-opening-matches-table*
(fri-evidence-layer-len composition-roots 0)
(fri-evidence-layer-idx composition-roots fri-query-idxs round-idx 0)
composition-table xp xn

and *auth:*

fri-layer-chunk-authenticated (composition-roots ! 0)
(fri-evidence-layer-len composition-roots 0)
(fri-evidence-layer-idx composition-roots fri-query-idxs round-idx 0)

```

    (composition-round-layers ! round-idx ! 0) final-state
using base
unfolding
    composition-fri-verifier-tied-authenticated-base-opening-conflict-def
by blast
have initial:
    accepted-with-partial-initial-openings s
    (Some (result, final-state)) fr trace-roots trace-final as fri-dg
    composition-roots composition-final fri-query-idxs trace-openings
    fri-query-idxs composition-openings
by (rule accepted-with-partial-initial-openings-aligned-imp-unaligned
    [OF aligned])
have comp-partial:
    accepted-with-partial-composition-openings s
    (Some (result, final-state)) (hd composition-roots) fri-query-idxs
    composition-openings
by (rule accepted-with-partial-initial-openings-shapes(4)
    [OF initial])
have roots-nonempty: composition-roots  $\neq$  []
by (rule accepted-with-partial-initial-openings-shapes(1)[OF initial])
have round-bound-rounds: round-idx < rounds
using round-bound
    accepted-with-partial-composition-openings-shapes(1)[OF comp-partial]
by simp
let ?idx =
    fri-evidence-layer-idx composition-roots fri-query-idxs round-idx 0
have idx-eq:
    ?idx = fri-query-idxs ! round-idx
unfolding fri-evidence-layer-idx-def
using roots-nonempty by (cases composition-roots) simp-all
have len-eq:
    fri-evidence-layer-len composition-roots 0 = scale * clength
unfolding fri-evidence-layer-len-def
using roots-nonempty by (cases composition-roots) simp-all
have root-eq: composition-roots ! 0 = hd composition-roots
using roots-nonempty by (cases composition-roots) simp-all
have idxs:
    map opening-index (composition-openings ! round-idx) =
    [?idx, fri-sibling-index (scale * clength) ?idx]
using accepted-with-partial-composition-openings-shapes(3)
    [OF comp-partial round-bound-rounds]
unfolding idx-eq by simp
have comp-value:
    composition-table ! ?idx =
    opening-value ((composition-openings ! round-idx) ! 0)
by (rule partial-composition-table-candidate-query-value
    [OF comp-cand round-bound-rounds idxs])
have len-openings: length (composition-openings ! round-idx) = 2
proof -

```

```

have length (map opening-index (composition-openings ! round-idx)) =
  length [?idx, fri-sibling-index (scale * clength) ?idx]
  using arg-cong[OF idxs, of length] by simp
then show ?thesis by simp
qed
have opn0-in:
  (composition-openings ! round-idx) ! 0
  ∈ set (composition-openings ! round-idx)
  using len-openings by (cases composition-openings ! round-idx) auto
have opn1-in:
  (composition-openings ! round-idx) ! 1
  ∈ set (composition-openings ! round-idx)
  using len-openings
  by (cases composition-openings ! round-idx; cases tl (composition-openings ! round-idx))
    auto
have comp-table:
  partial-authenticated-table (hd composition-roots) (scale * clength)
    (composition-openings ! round-idx) final-state
  using comp-partial round-bound-rounds
  unfolding accepted-with-partial-composition-openings-def by blast
have opn0-auth:
  authenticated-opening-in final-state
    ((composition-openings ! round-idx) ! 0)
  using comp-table opn0-in
  unfolding partial-authenticated-table-def by blast
have opn0-root:
  opening-root ((composition-openings ! round-idx) ! 0) =
    composition-roots ! 0
  using comp-table opn0-in root-eq
  unfolding partial-authenticated-table-def by simp
have opn0-len:
  opening-length ((composition-openings ! round-idx) ! 0) =
    fri-evidence-layer-len composition-roots 0
  using comp-table opn0-in len-eq
  unfolding partial-authenticated-table-def by simp
have opn0-idx:
  opening-index ((composition-openings ! round-idx) ! 0) = ?idx
  using idxs len-openings by (cases composition-openings ! round-idx) auto
have opn0-value: opening-value ((composition-openings ! round-idx) ! 0) = xp
proof –
  from step have chunk:
    fri-layer-opening-chunk
      (fri-evidence-layer-len composition-roots 0)
      xp xp-path xn xn-path
      (composition-round-layers ! round-idx ! 0)
    unfolding fri-layer-step-evidence-def by blast
  from fri-layer-chunk-authenticated-matching-values[OF auth chunk]
  obtain xp-path' xn-path' where xp-auth:
    authenticated-opening-in final-state

```



```

    (opening-root = composition-roots ! 0,
    opening-length = fri-evidence-layer-len composition-roots 0,
    opening-index =
      fri-evidence-layer-idx composition-roots fri-query-idxs round-idx 0,
    opening-value = xp,
    opening-path = xp-path')
  by (metis)
have opening-value ((composition-openings ! round-idx) ! 0) =
  opening-value
  (opening-root = composition-roots ! 0,
  opening-length = fri-evidence-layer-len composition-roots 0,
  opening-index =
    fri-evidence-layer-idx composition-roots fri-query-idxs round-idx 0,
  opening-value = xp,
  opening-path = xp-path')
by (rule authenticated-openings-values-eq-if-no-partial-merkle
  [OF no-bad opn0-auth xp-auth])
  (use opn0-root opn0-len opn0-idx in simp-all)
then show ?thesis by simp
qed
have comp-match-base: composition-table ! ?idx = xp
  using comp-value opn0-value by simp
have comp-sibling:
  composition-table ! fri-sibling-index (scale * clength) ?idx =
    opening-value ((composition-openings ! round-idx) ! 1)
proof -
  have vals:
    map (!) composition-table)
    [?idx, fri-sibling-index (scale * clength) ?idx] =
    map opening-value (composition-openings ! round-idx)
  by (rule partial-composition-table-candidate-query-values
    [OF comp-cand round-bound-rounds idxs])
  have lhs:
    map (!) composition-table)
    [?idx, fri-sibling-index (scale * clength) ?idx] ! 1 =
    composition-table ! fri-sibling-index (scale * clength) ?idx
  by simp
  have rhs:
    map opening-value (composition-openings ! round-idx) ! 1 =
    opening-value ((composition-openings ! round-idx) ! 1)
  using len-openings by simp
show ?thesis
  using arg-cong[OF vals, of  $\lambda x s. xs ! 1$ ] lhs rhs by simp
qed
have opn1-auth:
  authenticated-opening-in final-state
  ((composition-openings ! round-idx) ! 1)
  using comp-table opn1-in
  unfolding partial-authenticated-table-def by blast

```

```

have opn1-root:
  opening-root ((composition-openings ! round-idx) ! 1) =
    composition-roots ! 0
  using comp-table opn1-in root-eq
  unfolding partial-authenticated-table-def by simp
have opn1-len:
  opening-length ((composition-openings ! round-idx) ! 1) =
    fri-evidence-layer-len composition-roots 0
  using comp-table opn1-in len-eq
  unfolding partial-authenticated-table-def by simp
have opn1-idx:
  opening-index ((composition-openings ! round-idx) ! 1) =
    fri-sibling-index (fri-evidence-layer-len composition-roots 0) ?idx
  using idxs len-openings len-eq
  by (cases composition-openings ! round-idx; cases tl (composition-openings ! round-idx))
    auto
have opn1-value: opening-value ((composition-openings ! round-idx) ! 1) = xn
proof –
  from step have chunk:
    fri-layer-opening-chunk
      (fri-evidence-layer-len composition-roots 0)
      xp xp-path xn xn-path
      (composition-round-layers ! round-idx ! 0)
    unfolding fri-layer-step-evidence-def by blast
  from fri-layer-chunk-authenticated-matching-values[OF auth chunk]
  obtain xp-path' xn-path' where xn-auth:
    authenticated-opening-in final-state
      (|opening-root = composition-roots ! 0,
        opening-length = fri-evidence-layer-len composition-roots 0,
        opening-index =
          fri-sibling-index (fri-evidence-layer-len composition-roots 0)
            (fri-evidence-layer-idx composition-roots fri-query-idxs
              round-idx 0),
        opening-value = xn,
        opening-path = xn-path')
    by (metis)
  have opening-value ((composition-openings ! round-idx) ! 1) =
    opening-value
      (|opening-root = composition-roots ! 0,
        opening-length = fri-evidence-layer-len composition-roots 0,
        opening-index =
          fri-sibling-index (fri-evidence-layer-len composition-roots 0)
            (fri-evidence-layer-idx composition-roots fri-query-idxs
              round-idx 0),
        opening-value = xn,
        opening-path = xn-path')
    by (rule authenticated-openings-values-eq-if-no-partial-merkle
      [OF no-bad opn1-auth xn-auth])
    (use opn1-root opn1-len opn1-idx in simp-all)

```

```

    then show ?thesis by simp
qed
have comp-match-sibling:
  composition-table !
  fri-sibling-index (fri-evidence-layer-len composition-roots 0) ?idx =
  xn
  using comp-sibling opn1-value len-eq by simp
have idx-bound: ?idx < fri-evidence-layer-len composition-roots 0
proof -
  from auth obtain xp0 xp0-path xn0 xn0-path where
    authenticated-opening-in final-state
    (|opening-root = composition-roots ! 0,
     opening-length = fri-evidence-layer-len composition-roots 0,
     opening-index = ?idx,
     opening-value = xp0,
     opening-path = xp0-path|)
  by (rule fri-layer-chunk-authenticatedE)
  then show ?thesis
    unfolding authenticated-opening-in-def by simp
qed
have table-len:
  fri-evidence-layer-len composition-roots 0 ≤
  length composition-table
  using partial-composition-table-candidateD(1)
  [OF comp-cand round-bound-rounds opn0-in]
  unfolding len-eq by simp
have match:
  fri-opening-matches-table
  (fri-evidence-layer-len composition-roots 0)
  ?idx composition-table xp xn
  unfolding fri-opening-matches-table-def
  using idx-bound table-len comp-match-base comp-match-sibling
  by simp
then show False
  using no-match by contradiction
qed
qed

lemma wp-composition-fri-verifier-tied-authenticated-base-conflict-bound-by-partial-merkle:
  wp-event verify-monad
    (composition-fri-verifier-tied-authenticated-base-opening-conflict s) s
  ≤ wp-event verify-monad (partial-merkle-inconsistency-bad s) s
proof (rule wp-event-mono)
  fix out
  assume conflict:
    composition-fri-verifier-tied-authenticated-base-opening-conflict s out
  from conflict obtain result final-state where out-eq:
    out = Some (result, final-state)
  unfolding composition-fri-verifier-tied-authenticated-base-opening-conflict-def

```

```

    by blast
  show partial-merkle-inconsistency-bad s out
    unfolding out-eq
    by (rule composition-fri-verifier-tied-authenticated-base-conflict-imp-partial-merkle)
      (use conflict out-eq in simp)
qed

lemma wp-composition-fri-verifier-tied-sampled-base-bound-from-partial-merkle-and-auth-gap:
  fixes G :: prob
  assumes gap-bound:
    wp-event verify-monad
      (composition-fri-verifier-tied-base-chunk-auth-gap s) s ≤ G
  shows
    wp-event verify-monad
      (composition-fri-verifier-tied-sampled-base-opening-conflict s) s
      ≤ wp-event verify-monad (partial-merkle-inconsistency-bad s) s + G
proof (rule wp-composition-fri-verifier-tied-sampled-base-bound-from-authenticated-and-auth-gap)
  show wp-event verify-monad
    (composition-fri-verifier-tied-authenticated-base-opening-conflict s) s
    ≤ wp-event verify-monad (partial-merkle-inconsistency-bad s) s
  by (rule wp-composition-fri-verifier-tied-authenticated-base-conflict-bound-by-partial-merkle)
  show wp-event verify-monad
    (composition-fri-verifier-tied-base-chunk-auth-gap s) s ≤ G
  by (rule gap-bound)
qed

lemma wp-composition-fri-verifier-tied-without-same-bound-from-partial-merkle-auth-gap-and-branches:
  fixes G N S F :: prob
  assumes gap-bound:
    wp-event verify-monad
      (composition-fri-verifier-tied-base-chunk-auth-gap s) s ≤ G
  and next-bound:
    wp-event verify-monad
      (composition-fri-verifier-tied-sampled-next-value-conflict s) s ≤ N
  and successor-bound:
    wp-event verify-monad
      (composition-fri-verifier-tied-sampled-successor-opening-conflict s)
      s ≤ S
  and final-bound:
    wp-event verify-monad
      (composition-fri-verifier-tied-sampled-final-value-conflict s) s ≤ F
  shows
    wp-event verify-monad
      (composition-fri-verifier-tied-sampled-assignment-conflict-without-same-layer s)
      s ≤ (wp-event verify-monad (partial-merkle-inconsistency-bad s) s + G) + N + S + F
  by (rule wp-composition-fri-verifier-tied-without-same-bound-from-branches)
    (rule wp-composition-fri-verifier-tied-sampled-base-bound-from-partial-merkle-and-auth-gap
      [OF gap-bound],
      rule next-bound,

```

rule successor-bound,
rule final-bound)

lemma *composition-fri-verifier-tied-sampled-assignment-conflict-with-authenticated-same-layer-imp-verifier-tied:*
assumes

composition-fri-verifier-tied-sampled-assignment-conflict-with-authenticated-same-layer
s out

shows *composition-fri-verifier-tied-sampled-assignment-conflict s out*

proof —

from *assms* **obtain** *trace-roots trace-bs trace-final fri-dg*

composition-roots composition-bs composition-final fri-query-idxs
trace-round-layers composition-round-layers fr as trace-openings
composition-openings trace-table composition-table result final-state

where *fri-openings:*

accepted-fri-opening-transcript s out trace-roots trace-bs
trace-final fri-dg composition-roots composition-bs
composition-final fri-query-idxs trace-round-layers
composition-round-layers

and *out-eq:* *out = Some (result, final-state)*

and *aligned:*

accepted-with-partial-initial-openings-aligned s out fr trace-roots
trace-final as fri-dg composition-roots composition-final fri-query-idxs
trace-openings composition-openings

and *trace-cand:*

partial-trace-table-candidate trace-table trace-openings

and *comp-cand:*

partial-composition-table-candidate composition-table
composition-openings

and *trace-low:* *trace-table-low-degree trace-table*

and *comp-not-low:*

$\neg$ *composition-table-low-degree maxDegree composition-table*

and *conflict:*

generic-fri-sampled-assignment-conflict-with-authenticated-same-layer
composition-table composition-roots composition-bs
composition-final fri-query-idxs composition-round-layers
final-state

unfolding

composition-fri-verifier-tied-sampled-assignment-conflict-with-authenticated-same-layer-def

by *blast*

have *sampled:*

generic-fri-sampled-assignment-conflict composition-table
composition-roots composition-bs composition-final fri-query-idxs
composition-round-layers

by *(rule*

generic-fri-sampled-assignment-conflict-with-authenticated-same-layer-imp-sampled
[OF conflict])

show *?thesis*

unfolding *composition-fri-verifier-tied-sampled-assignment-conflict-def*

by *(intro exI conjI)*

(rule fri-openings, rule aligned, rule trace-cand, rule comp-cand,
rule trace-low, rule comp-not-low, rule sampled)

qed

lemma *wp-composition-fri-verifier-tied-sampled-assignment-conflict-with-authenticated-same-layer-bound-from-verifi*

assumes *verifier-bound:*

wp-event verify-monad

(*composition-fri-verifier-tied-sampled-assignment-conflict s*) $s \leq C$

shows

wp-event verify-monad

(*composition-fri-verifier-tied-sampled-assignment-conflict-with-authenticated-same-layer s*)

$s \leq C$

proof –

have *wp-event verify-monad*

(*composition-fri-verifier-tied-sampled-assignment-conflict-with-authenticated-same-layer s*)

$s \leq$

wp-event verify-monad

(*composition-fri-verifier-tied-sampled-assignment-conflict s*) s

by (rule *wp-event-mono*)

(rule

composition-fri-verifier-tied-sampled-assignment-conflict-with-authenticated-same-layer-imp-verifier-tied)

then show *?thesis*

by (rule *order-trans[OF - verifier-bound]*)

qed

definition *composition-fri-verifier-tied-same-layer-conflict-with-authenticated-chunks*

:: (*f*, *a*) *protocol-channel-scheme* $\Rightarrow$

(*unit list* $\times$ (*f*, *a*) *protocol-channel-scheme*) *option* $\Rightarrow$ *bool*

where

composition-fri-verifier-tied-same-layer-conflict-with-authenticated-chunks

s out $\longleftrightarrow$

($\exists$ *trace-roots trace-bs trace-final fri-dg composition-roots*

composition-bs composition-final fri-query-idxs trace-round-layers

composition-round-layers fr as trace-openings composition-openings

trace-table composition-table result final-state.

accepted-fri-opening-transcript s out trace-roots trace-bs

trace-final fri-dg composition-roots composition-bs

composition-final fri-query-idxs trace-round-layers

composition-round-layers $\wedge$

out = *Some (result, final-state)* $\wedge$

accepted-with-partial-initial-openings-aligned s out fr trace-roots

trace-final as fri-dg composition-roots composition-final fri-query-idxs

trace-openings composition-openings $\wedge$

partial-trace-table-candidate trace-table trace-openings $\wedge$

partial-composition-table-candidate composition-table

composition-openings $\wedge$

trace-table-low-degree trace-table $\wedge$

$\neg$ *composition-table-low-degree maxDegree composition-table* $\wedge$

generic-fri-sampled-same-layer-opening-conflict-with-authenticated-chunks

composition-roots composition-bs fri-query-idxs
composition-round-layers final-state)

lemma *composition-fri-verifier-tied-same-layer-conflict-with-authenticated-chunks-imp-partial-merkle:*
assumes

composition-fri-verifier-tied-same-layer-conflict-with-authenticated-chunks
s out

shows *partial-merkle-inconsistency-bad s out*

proof —

from *assms* **obtain** *trace-roots trace-bs trace-final fri-dg*

composition-roots composition-bs composition-final fri-query-idxs
trace-round-layers composition-round-layers fr as trace-openings
composition-openings trace-table composition-table result final-state

where *fri-openings:*

accepted-fri-opening-transcript s out trace-roots trace-bs
trace-final fri-dg composition-roots composition-bs
composition-final fri-query-idxs trace-round-layers
composition-round-layers

and *out-eq: out = Some (result, final-state)*

and *conflict:*

generic-fri-sampled-same-layer-opening-conflict-with-authenticated-chunks
composition-roots composition-bs fri-query-idxs
composition-round-layers final-state

unfolding

composition-fri-verifier-tied-same-layer-conflict-with-authenticated-chunks-def

by *blast*

have *len: length composition-bs = length composition-roots*

using *accepted-fri-opening-transcript-shapes(4)[OF fri-openings]* .

have *partial-merkle-inconsistency-bad s (Some (result, final-state))*

by *(rule*

generic-fri-sampled-same-layer-opening-conflict-with-authenticated-chunks-imp-partial-merkle
[OF conflict len])

then show *?thesis*

by *(simp add: out-eq)*

qed

lemma *wp-composition-fri-verifier-tied-same-layer-conflict-with-authenticated-chunks-bound-from-partial-merkle:*

assumes *merkle-bound:*

wp-event verify-monad (partial-merkle-inconsistency-bad s) s ≤ M

shows

wp-event verify-monad
(composition-fri-verifier-tied-same-layer-conflict-with-authenticated-chunks s)
s ≤ M

proof —

have *wp-event verify-monad*

(composition-fri-verifier-tied-same-layer-conflict-with-authenticated-chunks s)
s ≤ wp-event verify-monad (partial-merkle-inconsistency-bad s) s

by *(rule wp-event-mono)*

(rule

composition-fri-verifier-tied-same-layer-conflict-with-authenticated-chunks-imp-partial-merkle)
then show *?thesis*
by (rule order-trans[*OF - merkle-bound*])
qed

lemma *composition-fri-verifier-tied-authenticated-conflict-imp-without-same-or-same-layer:*

assumes

composition-fri-verifier-tied-sampled-assignment-conflict-with-authenticated-same-layer
s out

shows

composition-fri-verifier-tied-sampled-assignment-conflict-without-same-layer
s out $\vee$
composition-fri-verifier-tied-same-layer-conflict-with-authenticated-chunks
s out

proof —

from *assms* **obtain** *trace-roots trace-bs trace-final fri-dg*

composition-roots composition-bs composition-final fri-query-idxs
trace-round-layers composition-round-layers fr as trace-openings
composition-openings trace-table composition-table result final-state

where *fri-openings:*

accepted-fri-opening-transcript s out trace-roots trace-bs
trace-final fri-dg composition-roots composition-bs
composition-final fri-query-idxs trace-round-layers
composition-round-layers

and *out-eq:* *out = Some (result, final-state)*

and *aligned:*

accepted-with-partial-initial-openings-aligned s out fr trace-roots
trace-final as fri-dg composition-roots composition-final fri-query-idxs
trace-openings composition-openings

and *trace-cand:*

partial-trace-table-candidate trace-table trace-openings

and *comp-cand:*

partial-composition-table-candidate composition-table
composition-openings

and *trace-low:* *trace-table-low-degree trace-table*

and *comp-not-low:*

$\neg$ *composition-table-low-degree maxDegree composition-table*

and *conflict:*

generic-fri-sampled-assignment-conflict-with-authenticated-same-layer
composition-table composition-roots composition-bs
composition-final fri-query-idxs composition-round-layers
final-state

unfolding

composition-fri-verifier-tied-sampled-assignment-conflict-with-authenticated-same-layer-def

by *blast*

have *split:*

generic-fri-sampled-assignment-conflict-without-same-layer
composition-table composition-roots composition-bs composition-final
fri-query-idxs composition-round-layers $\vee$


```

generic-fri-sampled-same-layer-opening-conflict-with-authenticated-chunks
composition-roots composition-bs fri-query-idxs composition-round-layers
final-state
by (rule
  generic-fri-sampled-assignment-conflict-with-authenticated-same-layer-split
  [OF conflict])
then show ?thesis
proof
  assume without:
    generic-fri-sampled-assignment-conflict-without-same-layer
    composition-table composition-roots composition-bs composition-final
    fri-query-idxs composition-round-layers
  then show ?thesis
  unfolding
    composition-fri-verifier-tied-sampled-assignment-conflict-without-same-layer-def
  by (intro disjI1 exI conjI)
    (rule fri-openings, rule aligned, rule trace-cand, rule comp-cand,
     rule trace-low, rule comp-not-low, rule without)
next
  assume same:
    generic-fri-sampled-same-layer-opening-conflict-with-authenticated-chunks
    composition-roots composition-bs fri-query-idxs
    composition-round-layers final-state
  then show ?thesis
  unfolding
    composition-fri-verifier-tied-same-layer-conflict-with-authenticated-chunks-def
  by (intro disjI2 exI conjI)
    (rule fri-openings, rule out-eq, rule aligned, rule trace-cand,
     rule comp-cand, rule trace-low, rule comp-not-low, rule same)
qed
qed

lemma composition-fri-verifier-tied-authenticated-conflictI-without-same:
  assumes
    composition-fri-verifier-tied-sampled-assignment-conflict-without-same-layer
    s out
  shows
    composition-fri-verifier-tied-sampled-assignment-conflict-with-authenticated-same-layer
    s out
proof -
  from assms obtain trace-roots trace-bs trace-final fri-dg
    composition-roots composition-bs composition-final fri-query-idxs
    trace-round-layers composition-round-layers fr as trace-openings
    composition-openings trace-table composition-table
  where fri-openings:
    accepted-fri-opening-transcript s out trace-roots trace-bs
    trace-final fri-dg composition-roots composition-bs
    composition-final fri-query-idxs trace-round-layers
    composition-round-layers

```

and *aligned*:
accepted-with-partial-initial-openings-aligned s out fr trace-roots
trace-final as fri-dg composition-roots composition-final fri-query-idxs
trace-openings composition-openings
and *trace-cand*:
partial-trace-table-candidate trace-table trace-openings
and *comp-cand*:
partial-composition-table-candidate composition-table
composition-openings
and *trace-low*: *trace-table-low-degree trace-table*
and *comp-not-low*:
 $\neg$ *composition-table-low-degree maxDegree composition-table*
and *without*:
generic-fri-sampled-assignment-conflict-without-same-layer
composition-table composition-roots composition-bs
composition-final fri-query-idxs composition-round-layers
unfolding
composition-fri-verifier-tied-sampled-assignment-conflict-without-same-layer-def
by *blast*
from *fri-openings* **obtain** *result final-state* **where** *out-eq*:
out = Some (result, final-state)
unfolding *accepted-fri-opening-transcript-def* **by** *blast*
have *conflict*:
generic-fri-sampled-assignment-conflict-with-authenticated-same-layer
composition-table composition-roots composition-bs composition-final
fri-query-idxs composition-round-layers final-state
by (rule
generic-fri-sampled-assignment-conflict-with-authenticated-same-layerI-without
[OF without])
show *?thesis*
unfolding
composition-fri-verifier-tied-sampled-assignment-conflict-with-authenticated-same-layer-def
by (intro *exI conjI*)
(rule *fri-openings*, rule *out-eq*, rule *aligned*, rule *trace-cand*,
rule *comp-cand*, rule *trace-low*, rule *comp-not-low*, rule *conflict*)
qed

lemma *composition-fri-verifier-tied-authenticated-conflictI-same-layer*:
assumes
composition-fri-verifier-tied-same-layer-conflict-with-authenticated-chunks
s out
shows
composition-fri-verifier-tied-sampled-assignment-conflict-with-authenticated-same-layer
s out
proof –
from *assms* **obtain** *trace-roots trace-bs trace-final fri-dg*
composition-roots composition-bs composition-final fri-query-idxs
trace-round-layers composition-round-layers fr as trace-openings
composition-openings trace-table composition-table result final-state

where *fri-openings*:
accepted-fri-opening-transcript s out trace-roots trace-bs
trace-final fri-dg composition-roots composition-bs
composition-final fri-query-idxs trace-round-layers
composition-round-layers
and *out-eq*: *out = Some (result, final-state)*
and *aligned*:
accepted-with-partial-initial-openings-aligned s out fr trace-roots
trace-final as fri-dg composition-roots composition-final fri-query-idxs
trace-openings composition-openings
and *trace-cand*:
partial-trace-table-candidate trace-table trace-openings
and *comp-cand*:
partial-composition-table-candidate composition-table
composition-openings
and *trace-low*: *trace-table-low-degree trace-table*
and *comp-not-low*:
 $\neg$ *composition-table-low-degree maxDegree composition-table*
and *same*:
generic-fri-sampled-same-layer-opening-conflict-with-authenticated-chunks
composition-roots composition-bs fri-query-idxs
composition-round-layers final-state
unfolding
composition-fri-verifier-tied-same-layer-conflict-with-authenticated-chunks-def
by *blast*
have *conflict*:
generic-fri-sampled-assignment-conflict-with-authenticated-same-layer
composition-table composition-roots composition-bs composition-final
fri-query-idxs composition-round-layers final-state
by (rule
generic-fri-sampled-assignment-conflict-with-authenticated-same-layerI-same
[OF *same*])
show *?thesis*
unfolding
composition-fri-verifier-tied-sampled-assignment-conflict-with-authenticated-same-layer-def
by (intro *exI conjI*)
(rule *fri-openings*, rule *out-eq*, rule *aligned*, rule *trace-cand*,
rule *comp-cand*, rule *trace-low*, rule *comp-not-low*, rule *conflict*)
qed

lemma *composition-fri-verifier-tied-authenticated-conflict-iff*:
composition-fri-verifier-tied-sampled-assignment-conflict-with-authenticated-same-layer
s out $\longleftrightarrow$
composition-fri-verifier-tied-sampled-assignment-conflict-without-same-layer
s out $\vee$
composition-fri-verifier-tied-same-layer-conflict-with-authenticated-chunks
s out
using *composition-fri-verifier-tied-authenticated-conflict-imp-without-same-or-same-layer*
composition-fri-verifier-tied-authenticated-conflictI-without-same

composition-fri-verifier-tied-authenticated-conflictI-same-layer
by *blast*

lemma *wp-composition-fri-verifier-tied-authenticated-conflict-bound-from-without-same-and-merkle:*

assumes *without-bound:*

wp-event verify-monad

(*composition-fri-verifier-tied-sampled-assignment-conflict-without-same-layer s*)

$s \leq W$

and *merkle-bound:*

wp-event verify-monad (partial-merkle-inconsistency-bad s) s ≤ M

shows

wp-event verify-monad

(*composition-fri-verifier-tied-sampled-assignment-conflict-with-authenticated-same-layer s*)

$s \leq W + M$

proof –

have *wp-event verify-monad*

(*composition-fri-verifier-tied-sampled-assignment-conflict-with-authenticated-same-layer s*)

$s \leq$

wp-event verify-monad

($\lambda out.$

composition-fri-verifier-tied-sampled-assignment-conflict-without-same-layer

$s out \vee$

composition-fri-verifier-tied-same-layer-conflict-with-authenticated-chunks

$s out) s$

by (*rule wp-event-mono*)

(*rule*

composition-fri-verifier-tied-authenticated-conflict-imp-without-same-or-same-layer)

also have $\dots \leq$

wp-event verify-monad

(*composition-fri-verifier-tied-sampled-assignment-conflict-without-same-layer s*)

$s +$

wp-event verify-monad

(*composition-fri-verifier-tied-same-layer-conflict-with-authenticated-chunks s*)

s

by (*rule wp-event-union-bound*)

also have $\dots \leq W + M$

by (*rule add-mono[OF without-bound*

wp-composition-fri-verifier-tied-same-layer-conflict-with-authenticated-chunks-bound-from-partial-merkle

[*OF merkle-bound*]])

finally show *?thesis* .

qed

end

end

theory *Soundness-FRI-Base-Recorded-Chunk*

imports *Soundness-FRI-Layer-Merkle*

begin

Base-layer recorded chunk replay. This is the small case needed for the existing first-layer composition residual: the head chunk read by *receive-query-commits* is the first chunk in the extracted layer transcript, hence the recorded first layer is authenticated.

context *soundness*

begin

lemma *mfold-fri-layer-opening-head-recorded-chunk-authenticated:*

assumes *outcome:*

Some (out, t) ∈
set-dist
(execute
(mfold (idx, x, len, pw)
(map fri-layer-opening-step ((b, rt) # bfs))) s)

and *layers:*

fri-layers-transcript (length ((b, rt) # bfs)) len layer-chunks
total-chunk

and *transcript:* *PTranscript s = total-chunk @ PTranscript t*

and *len-pos:* *0 < len*

and *idx-bound:* *idx < len*

shows

fri-layer-chunk-authenticated rt len idx (layer-chunks ! 0) t

proof –

from *outcome* **obtain** *out1 s1* **where**

head:

Some (out1, s1) ∈
set-dist
(execute (fri-layer-opening-step (b, rt) (idx, x, len, pw)) s)

and *tail:*

Some (out, t) ∈
set-dist (execute (mfold out1 (map fri-layer-opening-step bfs)) s1)
by *(auto elim!: set-dist-bindE)*

obtain *xp xp-path xn xn-path x' head-chunk* **where**

out1-eq:

out1 = (idx mod (len div 2), x', len div 2, pw + pw)

and *head-open:*

fri-layer-opening-chunk len xp xp-path xn xn-path head-chunk

and *tr-head:* *PTranscript s = head-chunk @ PTranscript s1*

and *auth-xp:*

authenticated-opening-in s1
(|opening-root = rt,
opening-length = len,
opening-index = idx,
opening-value = xp,
opening-path = xp-path)

and *auth-xn:*

authenticated-opening-in s1
(|opening-root = rt,

```

    opening-length = len,
    opening-index = fri-sibling-index len idx,
    opening-value = xn,
    opening-path = xn-path)
  by (rule fri-layer-opening-step-outcome-authenticated
    [OF len-pos idx-bound head])
from mfold-fri-layer-openings-outcome[OF tail[unfolded out1-eq]]
obtain tail-layers tail-chunk where
  tail-layers:
    fri-layers-transcript (length bfs) (len div 2) tail-layers
    tail-chunk
  and tr-tail: PTranscript s1 = tail-chunk @ PTranscript t
  and ext-tail: s1 ≤ t
  by auto
let ?layers = head-chunk # tail-layers
let ?chunk = head-chunk @ tail-chunk
have exec-layers:
  fri-layers-transcript (length ((b, rt) # bfs)) len ?layers ?chunk
  using head-open tail-layers
  unfolding fri-layers-transcript-def by auto
have exec-transcript: PTranscript s = ?chunk @ PTranscript t
  using tr-head tr-tail by simp
have chunk-eq: ?chunk = total-chunk
  using exec-transcript transcript by simp
have layers-eq: ?layers = layer-chunks
  by (rule fri-layers-transcript-unique[OF exec-layers])
  (use layers chunk-eq in simp)
have head-auth-s1:
  fri-layer-chunk-authenticated rt len idx head-chunk s1
  by (rule fri-layer-chunk-authenticatedI[OF head-open auth-xp auth-xn])
have head-auth-t:
  fri-layer-chunk-authenticated rt len idx head-chunk t
  by (rule fri-layer-chunk-authenticated-mono[OF head-auth-s1 ext-tail])
show ?thesis
  using head-auth-t layers-eq[symmetric] by simp
qed

```

lemma *receive-query-commits-head-recorded-chunk-authenticated:*

assumes *outcome:*

Some (out, t) ∈
set-dist
(execute
(mfold (idx, x, len, pw)
(receive-query-commits ((b, rt) # bfs))) s)

and *layers:*

fri-layers-transcript (length ((b, rt) # bfs)) len layer-chunks
total-chunk

and *transcript:* *PTranscript s = total-chunk @ PTranscript t*

and *len-pos:* *0 < len*

```

    and idx-bound: idx < len
  shows
    fri-layer-chunk-authenticated rt len idx (layer-chunks ! 0) t
proof -
  have map-eq:
    receive-query-commits ((b, rt) # bfs) =
      map fri-layer-opening-step ((b, rt) # bfs)
  unfolding receive-query-commits-def fri-layer-opening-step-def
  by (simp add: fun-eq-iff split: prod.splits)
  show ?thesis
  by (rule mfold-fri-layer-opening-head-recorded-chunk-authenticated
      [OF outcome[unfolded map-eq] layers transcript len-pos idx-bound])
qed

lemma verifier-query-round-after-index-trace-base-recorded-chunk-authenticated:
  fixes s t :: ('f, 'a) protocol-channel-scheme
  assumes f-fl-eq: f-fl = (b, rt) # f-fl-tail
  and outcome:
    Some ((l), t) ∈
      set-dist
        (execute
          (verifier-query-round-after-index-program
            fr f-fl f-final as fl final raw) s)
  and recorded:
    query-round-fri-layer-transcripts (index (to-nat raw))
      (map snd f-fl) (map snd fl) round-chunk trace-layers
      composition-layers
  and transcript: PTranscript s = round-chunk @ PTranscript t
  shows
    fri-layer-chunk-authenticated rt (clength * scale)
      (index (to-nat raw)) (trace-layers ! 0) t
proof -
  let ?idx = index (to-nat raw)
  from outcome obtain fv s1 f-out s2 s3 c-out s4 where
    query-decommit:
      Some (fv, s1) ∈
        set-dist (execute (mmap (check-decommit-on-query fr ?idx)) s)
  and trace-fri:
    Some (f-out, s2) ∈
      set-dist (execute
        (mfold (?idx, hd fv, clength * scale, 1)
          (receive-query-commits f-fl)) s1)
  and assert-trace:
    Some ((l), s3) ∈
      set-dist
        (execute
          (assert (case f-out of (-, f-x, -, -) ⇒ f-x = f-final)) s2)
  and comp-fri:
    Some (c-out, s4) ∈

```

```

    set-dist (execute
      (mfold
        (?idx, cp-eval as fv (h ^ ?idx * shift), clength * scale, 1)
        (receive-query-commits fl)) s3)
  and assert-comp:
    Some (((), t) ∈
      set-dist
        (execute
          (assert (case c-out of (-, x, -, -) ⇒ x = final)) s4)
    unfolding verifier-query-round-after-index-program-def
    by (auto simp: Let-def split: prod.splits elim!: set-dist-bindE)
  from check-decommit-on-query-outcome[OF query-decommit]
  obtain query-paths query-chunk where
    query-chunk-shape:
      query-decommitment-transcript ?idx fv query-paths query-chunk
  and tr-s: PTranscript s = query-chunk @ PTranscript s1
  by blast
  from receive-query-commits-layers-outcome[OF trace-fri]
  obtain trace-layer-chunks trace-fri-chunk where
    trace-layers-actual:
      fri-layers-transcript (length f-fl) (clength * scale)
      trace-layer-chunks trace-fri-chunk
  and tr-s1: PTranscript s1 = trace-fri-chunk @ PTranscript s2
  and ext-s1-s2: s1 ≤ s2
  by blast
  have s3-eq: s3 = s2
  using assert-trace unfolding assert-def
  by (cases case f-out of (-, f-x, -, -) ⇒ f-x = f-final)
    (auto simp: throw-no-outcome)
  from receive-query-commits-layers-outcome[OF comp-fri]
  obtain comp-layer-chunks comp-fri-chunk where
    comp-layers-actual:
      fri-layers-transcript (length fl) (clength * scale)
      comp-layer-chunks comp-fri-chunk
  and tr-s3: PTranscript s3 = comp-fri-chunk @ PTranscript s4
  and ext-s3-s4: s3 ≤ s4
  by blast
  have t-eq: t = s4
  using assert-comp unfolding assert-def
  by (cases case c-out of (-, x, -, -) ⇒ x = final)
    (auto simp: throw-no-outcome)
  let ?actual-chunk = query-chunk @ trace-fri-chunk @ comp-fri-chunk
  have actual-recorded:
    query-round-fri-layer-transcripts ?idx (map snd f-fl) (map snd fl)
    ?actual-chunk trace-layer-chunks comp-layer-chunks
  unfolding query-round-fri-layer-transcripts-def
  by (intro exI[of - query-chunk] exI[of - trace-fri-chunk]
    exI[of - comp-fri-chunk] exI[of - fv] exI[of - query-paths] conjI)
    (use query-chunk-shape trace-layers-actual comp-layers-actual in

```



```

    ⟨simp-all add: mult.commute⟩)
  have actual-transcript:
    PTranscript s = ?actual-chunk @ PTranscript t
    using tr-s tr-s1 tr-s3 s3-eq t-eq by simp
  have actual-chunk-eq: ?actual-chunk = round-chunk
    using actual-transcript transcript by simp
  have trace-layer-chunks-eq:
    trace-layer-chunks = trace-layers
    by (rule query-round-fri-layer-transcripts-unique(1)
        [OF actual-recorded[unfolded actual-chunk-eq] recorded])
  have len-pos: 0 < clength * scale
    by (rule eval-domain-size-pos)
  have idx-bound: ?idx < clength * scale
    by (rule index-less-domain)
  have auth-s2:
    fri-layer-chunk-authenticated rt (clength * scale) ?idx
      (trace-layer-chunks ! 0) s2
    by (rule receive-query-commits-head-recorded-chunk-authenticated
        [OF trace-fri[unfolded f-fl-eq]
            trace-layers-actual[unfolded f-fl-eq] tr-s1 len-pos idx-bound])
  have s2-t: s2 ≤ t
    using ext-s3-s4 s3-eq t-eq by simp
  have auth-t:
    fri-layer-chunk-authenticated rt (clength * scale) ?idx
      (trace-layer-chunks ! 0) t
    by (rule fri-layer-chunk-authenticated-mono[OF auth-s2 s2-t])
  show ?thesis
    using auth-t trace-layer-chunks-eq by simp
qed

lemma verifier-query-round-after-index-composition-base-recorded-chunk-authenticated:
  fixes s t :: ('f, 'a) protocol-channel-scheme
  assumes fl-eq: fl = (b, rt) # fl-tail
  and outcome:
    Some (((), t) ∈
      set-dist
        (execute
          (verifier-query-round-after-index-program
            fr f-fl f-final as fl final raw) s)
    and recorded:
      query-round-fri-layer-transcripts (index (to-nat raw))
      (map snd f-fl) (map snd fl) round-chunk trace-layers
      composition-layers
    and transcript: PTranscript s = round-chunk @ PTranscript t
  shows
    fri-layer-chunk-authenticated rt (clength * scale)
      (index (to-nat raw)) (composition-layers ! 0) t
  proof -
    let ?idx = index (to-nat raw)

```

```

from outcome obtain fv s1 f-out s2 s3 c-out s4 where
  query-decommit:
    Some (fv, s1) ∈
      set-dist (execute (mmap (check-decommit-on-query fr ?idx)) s)
  and trace-fri:
    Some (f-out, s2) ∈
      set-dist (execute
        (mfold (?idx, hd fv, clength * scale, 1)
          (receive-query-commits f-fl)) s1)
  and assert-trace:
    Some (( ), s3) ∈
      set-dist
        (execute
          (assert (case f-out of (-, f-x, -, -) ⇒ f-x = f-final)) s2)
  and comp-fri:
    Some (c-out, s4) ∈
      set-dist (execute
        (mfold
          (?idx, cp-eval as fv (h ^ ?idx * shift), clength * scale, 1)
          (receive-query-commits fl)) s3)
  and assert-comp:
    Some (( ), t) ∈
      set-dist
        (execute
          (assert (case c-out of (-, x, -, -) ⇒ x = final)) s4)
  unfolding verifier-query-round-after-index-program-def
  by (auto simp: Let-def split: prod.splits elim!: set-dist-bindE)
from check-decommit-on-query-outcome[OF query-decommit]
obtain query-paths query-chunk where
  query-chunk-shape:
    query-decommitment-transcript ?idx fv query-paths query-chunk
  and tr-s: PTranscript s = query-chunk @ PTranscript s1
  by blast
from receive-query-commits-layers-outcome[OF trace-fri]
obtain trace-layer-chunks trace-fri-chunk where
  trace-layers-actual:
    fri-layers-transcript (length f-fl) (clength * scale)
    trace-layer-chunks trace-fri-chunk
  and tr-s1: PTranscript s1 = trace-fri-chunk @ PTranscript s2
  by blast
have s3-eq: s3 = s2
  using assert-trace unfolding assert-def
  by (cases case f-out of (-, f-x, -, -) ⇒ f-x = f-final)
    (auto simp: throw-no-outcome)
from receive-query-commits-layers-outcome[OF comp-fri]
obtain comp-layer-chunks comp-fri-chunk where
  comp-layers-actual:
    fri-layers-transcript (length fl) (clength * scale)
    comp-layer-chunks comp-fri-chunk

```

```

and tr-s3: PTranscript s3 = comp-fri-chunk @ PTranscript s4
by blast
have t-eq: t = s4
using assert-comp unfolding assert-def
by (cases case c-out of (-, x, -, -)  $\Rightarrow$  x = final)
    (auto simp: throw-no-outcome)
let ?actual-chunk = query-chunk @ trace-fri-chunk @ comp-fri-chunk
have actual-recorded:
    query-round-fri-layer-transcripts ?idx (map snd f-fl) (map snd fl)
    ?actual-chunk trace-layer-chunks comp-layer-chunks
unfolding query-round-fri-layer-transcripts-def
by (intro exI[of - query-chunk] exI[of - trace-fri-chunk]
    exI[of - comp-fri-chunk] exI[of - fv] exI[of - query-paths] conjI)
    (use query-chunk-shape trace-layers-actual comp-layers-actual in
     $\langle \text{simp-all add: mult.commute} \rangle$ )
have actual-transcript:
    PTranscript s = ?actual-chunk @ PTranscript t
using tr-s tr-s1 tr-s3 s3-eq t-eq by simp
have actual-chunk-eq: ?actual-chunk = round-chunk
using actual-transcript transcript by simp
have comp-layer-chunks-eq:
    comp-layer-chunks = composition-layers
by (rule query-round-fri-layer-transcripts-unique(2)
    [OF actual-recorded[unfolded actual-chunk-eq] recorded])
have len-pos: 0 < clength * scale
by (rule eval-domain-size-pos)
have idx-bound: ?idx < clength * scale
by (rule index-less-domain)
have auth-s4:
    fri-layer-chunk-authenticated rt (clength * scale) ?idx
    (comp-layer-chunks ! 0) s4
by (rule receive-query-commits-head-recorded-chunk-authenticated
    [OF comp-fri[unfolded fl-eq] comp-layers-actual[unfolded fl-eq]
    tr-s3 len-pos idx-bound])
show ?thesis
using auth-s4 comp-layer-chunks-eq t-eq by simp
qed

```

lemma *accepted-fri-opening-transcript-trace-base-recorded-chunk-authenticated-at*:

assumes *fri-openings*:

accepted-fri-opening-transcript s out trace-roots trace-bs trace-final
dg composition-roots composition-bs composition-final query-idxs
trace-round-layers composition-round-layers

and *out-eq*: *out* = *Some* (*result*, *final-state*)

and *round-bound*: *i* < *rounds*

and *trace-nonempty*: *0* < *length trace-roots*

shows

fri-layer-chunk-authenticated (*trace-roots ! 0*) (*clength * scale*)
 (*query-idxs ! i*) (*trace-round-layers ! i ! 0*) *final-state*

proof –

```

from fri-openings obtain result' final-state' fr f-fl as fl query-state
  raw-idxs query-chunks where
    out-def: out = Some (result', final-state')
    and trace-roots-eq: trace-roots = map snd f-fl
    and composition-roots-eq: composition-roots = map snd fl
    and outcome:
      Some (result', final-state') ∈
        set-dist (execute
          (ntimes
            (verifier-query-round-program fr f-fl trace-final as fl
              composition-final)
            rounds)
          query-state)
    and len-raw: length raw-idxs = rounds
    and query-idxs-eq:
      query-idxs = map (λraw. index (to-nat raw)) raw-idxs
    and len-query-chunks: length query-chunks = rounds
    and transcript-query:
      PTranscript query-state =
        List.concat query-chunks @ PTranscript final-state'
    and query-chunk:
       $\bigwedge i. i < rounds \implies$ 
        verifier-query-round-chunk (query-idxs ! i)
          (map snd f-fl) (map snd fl) (query-chunks ! i)
    and replay-lookup:
       $\bigwedge i. i < rounds \implies$ 
        fmllookup (HashMap final-state')
          (QueryIndexChallenge (PQueryCounter query-state + i)
            (state-after-query-chunks (PState query-state) query-chunks i)) =
          Some (raw-idxs ! i)
    and recorded:
       $\bigwedge i. i < rounds \implies$ 
        query-round-fri-layer-transcripts (query-idxs ! i)
          trace-roots composition-roots (query-chunks ! i)
            (trace-round-layers ! i) (composition-round-layers ! i)
    unfolding accepted-fri-opening-transcript-def by blast
have result-eq: result' = result
using out-def out-eq by simp
have final-state-eq: final-state' = final-state
using out-def out-eq by simp
have outcome':
  Some (result, final-state) ∈
    set-dist (execute
      (ntimes
        (verifier-query-round-program fr f-fl trace-final as fl
          composition-final)
        rounds)
      query-state)

```

```

using outcome result-eq final-state-eq by simp
have f-fl-nonempty:  $f\text{-fl} \neq []$ 
using trace-nonempty trace-roots-eq by auto
then obtain bf f-fl-tail where f-fl-cons:  $f\text{-fl} = bf \# f\text{-fl-tail}$ 
by (cases f-fl) auto
obtain b rt where bf-eq:  $bf = (b, rt)$ 
by (cases bf)
have f-fl-eq:  $f\text{-fl} = (b, rt) \# f\text{-fl-tail}$ 
using f-fl-cons bf-eq by simp
have rt-eq:  $rt = \text{trace-roots} ! 0$ 
using f-fl-cons bf-eq trace-roots-eq by simp
let ?round =
  verifier-query-round-program fr f-fl trace-final as fl composition-final
from query-opening-ntimes-outcome-decomp-at[OF outcome' round-bound]
obtain prefix x suffix s-i s-suc where
  prefix:
    Some (prefix, s-i) ∈
      set-dist (execute (ntimes ?round i) query-state)
and round:
    Some (x, s-suc) ∈ set-dist (execute ?round s-i)
and suffix:
    Some (suffix, final-state) ∈
      set-dist (execute (ntimes ?round (rounds - Suc i)) s-suc)
by blast
have x-eq:  $x = ()$ 
by (cases x) simp
show ?thesis
proof (rule query-opening-ntimes-prefix-transcript-state-alignment
  [OF prefix])
fix prefix-query-idxs prefix-chunks
assume len-prefix-idxs:  $\text{length prefix-query-idxs} = i$ 
and len-prefix-chunks:  $\text{length prefix-chunks} = i$ 
and transcript-prefix:
  PTranscript query-state =
    List.concat prefix-chunks @ PTranscript s-i
and state-i:
  PState s-i =
    state-after-query-chunks (PState query-state) prefix-chunks i
and counter-i: PQueryCounter s-i = PQueryCounter query-state + i
and prefix-ext:  $\text{query-state} \leq s\text{-i}$ 
and prefix-rounds:
   $\bigwedge k. k < i \implies$ 
    verifier-query-round-chunk (prefix-query-idxs ! k)
    (map snd f-fl) (map snd fl) (prefix-chunks ! k)
from round[unfolded x-eq verifier-query-round-program-alt-def]
obtain raw s0 where
  raw-out:
    Some (raw, s0) ∈
      set-dist (execute receive-query-index-challenge s-i)

```

```

and after:
  Some ((), s-suc) ∈
    set-dist (execute
      (verifier-query-round-after-index-program fr f-fl trace-final as
        fl composition-final raw) s0)
by (auto elim!: set-dist-bindE)
have tr-s0: PTranscript s0 = PTranscript s-i
  using receive-query-index-challenge-outcome[OF raw-out] by simp
show ?thesis
proof (rule verifier-query-round-program-outcome[OF round[unfolded x-eq]])
  fix raw' idx chunk
  assume idx-eq: idx = index (to-nat raw')
  and chunk-shape:
    verifier-query-round-chunk idx (map snd f-fl) (map snd fl)
    chunk
  and transcript-round:
    PTranscript s-i = chunk @ PTranscript s-suc
  and state-suc:
    PState s-suc = foldl concat (PState s-i) chunk
  and round-ext: s-i ≤ s-suc
  and counter-suc:
    PQueryCounter s-suc = Suc (PQueryCounter s-i)
  and lookup-suc:
    fmlookup (HashMap s-suc)
      (QueryIndexChallenge (PQueryCounter s-i) (PState s-i)) =
      Some raw'
  have raw-eq': raw = raw'
  proof -
    have lookup-s0:
      fmlookup (HashMap s0)
        (QueryIndexChallenge (PQueryCounter s-i) (PState s-i)) =
        Some raw
      using receive-query-index-challenge-outcome[OF raw-out] by simp
    have s0-suc: s0 ≤ s-suc
      using verifier-query-round-after-index-program-outcome[OF after]
      by simp
    have lookup-suc-raw:
      fmlookup (HashMap s-suc)
        (QueryIndexChallenge (PQueryCounter s-i) (PState s-i)) =
        Some raw
      using hash-extension-lookup[OF lookup-s0 s0-suc] .
    show ?thesis
      using lookup-suc lookup-suc-raw by simp
  qed
show ?thesis
proof (rule query-opening-ntimes-suffix-transcript-state-alignment
  [OF suffix])
  fix suffix-query-idxs suffix-chunks
  assume len-suffix-idxs:

```

```

    length suffix-query-idxs = rounds - Suc i
  and len-suffix-chunks:
    length suffix-chunks = rounds - Suc i
  and transcript-suffix:
    PTranscript s-suc =
      List.concat suffix-chunks @ PTranscript final-state
  and suffix-state:
    PState final-state =
      state-after-query-chunks (PState s-suc) suffix-chunks
      (rounds - Suc i)
  and suffix-counter:
    PQueryCounter final-state =
      PQueryCounter s-suc + (rounds - Suc i)
  and suffix-ext: s-suc ≤ final-state
  and suffix-rounds:
     $\bigwedge k. k < rounds - Suc i \implies$ 
      verifier-query-round-chunk (suffix-query-idxs ! k)
      (map snd f-fl) (map snd fl) (suffix-chunks ! k)
  let ?selected-chunks = prefix-chunks @ chunk # suffix-chunks
  have len-selected-chunks: length ?selected-chunks = rounds
    using len-prefix-chunks len-suffix-chunks round-bound by simp
  have selected-chunk-lens:
     $\bigwedge k. k < rounds \implies$ 
      length (?selected-chunks ! k) =
        verifier-query-round-transcript-length (query-idxs ! k)
        (map snd f-fl) (map snd fl)
  proof -
    fix k
    assume k-bound: k < rounds
    show
      length (?selected-chunks ! k) =
        verifier-query-round-transcript-length (query-idxs ! k)
        (map snd f-fl) (map snd fl)
    proof (cases k < i)
      case True
      have nth-eq: ?selected-chunks ! k = prefix-chunks ! k
        using True len-prefix-chunks by (simp add: nth-append)
      have
        length (prefix-chunks ! k) =
          verifier-query-round-transcript-length
            (prefix-query-idxs ! k) (map snd f-fl) (map snd fl)
        by (rule verifier-query-round-chunk-length
            [OF prefix-rounds[OF True]])
      then show ?thesis
        using nth-eq
        by (simp add:
            verifier-query-round-transcript-length-index-irrelevant)
    next
      case False

```

```

show ?thesis
proof (cases k = i)
  case True
  have nth-eq: ?selected-chunks ! k = chunk
    using True len-prefix-chunks by (simp add: nth-append)
  have
    length chunk =
      verifier-query-round-transcript-length idx
        (map snd f-fl) (map snd fl)
    by (rule verifier-query-round-chunk-length[OF chunk-shape])
  then show ?thesis
    using nth-eq
    by (simp add:
      verifier-query-round-transcript-length-index-irrelevant)
  next
  case False
  let ?k = k - Suc i
  have ge: Suc i ≤ k
    using <¬ k < i> False by linarith
  have k-bound': ?k < rounds - Suc i
    using ge k-bound by linarith
  have nth-eq: ?selected-chunks ! k = suffix-chunks ! ?k
    using ge len-prefix-chunks by (simp add: nth-append)
  have
    length (suffix-chunks ! ?k) =
      verifier-query-round-transcript-length
        (suffix-query-idxs ! ?k) (map snd f-fl) (map snd fl)
    by (rule verifier-query-round-chunk-length
      [OF suffix-rounds[OF k-bound']])
  then show ?thesis
    using nth-eq
    by (simp add:
      verifier-query-round-transcript-length-index-irrelevant)
qed
qed
qed
have query-chunk-lens:
   $\bigwedge k. k < rounds \implies$ 
  length (query-chunks ! k) =
    verifier-query-round-transcript-length (query-idxs ! k)
      (map snd f-fl) (map snd fl)
  using query-chunk verifier-query-round-chunk-length by blast
have selected-transcript:
  PTranscript query-state =
    List.concat ?selected-chunks @ PTranscript final-state
  using transcript-prefix transcript-round transcript-suffix by simp
have concat-eq: List.concat ?selected-chunks = List.concat query-chunks
  using selected-transcript transcript-query final-state-eq by simp
have chunks-eq: ?selected-chunks = query-chunks

```



```

proof —
  have same-lens:
     $\bigwedge k. k < \text{length } ?\text{selected-chunks} \implies$ 
       $\text{length } (? \text{selected-chunks } ! k) = \text{length } (\text{query-chunks } ! k)$ 
    using selected-chunk-lens query-chunk-lens len-selected-chunks
    by simp
  have concat-eq-empty:
     $\text{List.concat } ?\text{selected-chunks } @ [] = \text{List.concat } \text{query-chunks}$ 
    using concat-eq by simp
  have len-eq:  $\text{length } ?\text{selected-chunks} = \text{length } \text{query-chunks}$ 
    using len-selected-chunks len-query-chunks by simp
  have  $? \text{selected-chunks} = \text{query-chunks} \wedge ([] :: 'f \text{ list}) = []$ 
    by (rule concat-append-eq-concat-same-chunk-lengths
      [OF len-eq same-lens concat-eq-empty])
  then show ?thesis
    by simp
qed
have prefix-state-eq:
   $\text{state-after-query-chunks } (P\text{State } \text{query-state}) \text{ prefix-chunks } i =$ 
   $\text{state-after-query-chunks } (P\text{State } \text{query-state}) \text{ query-chunks } i$ 
proof —
  have
     $\text{state-after-query-chunks } (P\text{State } \text{query-state}) ?\text{selected-chunks } i =$ 
     $\text{state-after-query-chunks } (P\text{State } \text{query-state}) \text{ prefix-chunks } i$ 
    by (rule state-after-query-chunks-append-prefix
      [OF len-prefix-chunks])
  then show ?thesis
    using chunks-eq by simp
qed
have lookup-final:
   $\text{fmlookup } (\text{HashMap } \text{final-state})$ 
   $(\text{QueryIndexChallenge } (P\text{QueryCounter } \text{query-state} + i)$ 
   $(\text{state-after-query-chunks } (P\text{State } \text{query-state}) \text{ query-chunks } i)) =$ 
   $\text{Some } \text{raw}'$ 
proof —
  have lookup-suc':
     $\text{fmlookup } (\text{HashMap } s\text{-suc})$ 
     $(\text{QueryIndexChallenge } (P\text{QueryCounter } \text{query-state} + i)$ 
     $(\text{state-after-query-chunks } (P\text{State } \text{query-state})$ 
     $\text{prefix-chunks } i)) =$ 
     $\text{Some } \text{raw}'$ 
    using lookup-suc state-i counter-i by simp
  have lookup-final-prefix:
     $\text{fmlookup } (\text{HashMap } \text{final-state})$ 
     $(\text{QueryIndexChallenge } (P\text{QueryCounter } \text{query-state} + i)$ 
     $(\text{state-after-query-chunks } (P\text{State } \text{query-state})$ 
     $\text{prefix-chunks } i)) =$ 
     $\text{Some } \text{raw}'$ 
    using hash-extension-lookup [OF lookup-suc' suffix-ext] .

```

```

    show ?thesis
      using lookup-final-prefix prefix-state-eq by simp
qed
have raw-idx-eq: raw' = raw-idxs ! i
  using lookup-final replay-lookup[OF round-bound] final-state-eq
  by simp
have idx-eq-query: idx = query-idxs ! i
  using idx-eq raw-idx-eq query-idxs-eq len-raw round-bound by simp
have selected-i: ?selected-chunks ! i = chunk
  using len-prefix-chunks by (simp add: nth-append)
have chunk-i-eq: chunk = query-chunks ! i
  using chunks-eq selected-i by simp
have recorded-i:
  query-round-fri-layer-transcripts (index (to-nat raw))
    (map snd f-fl) (map snd fl) chunk (trace-round-layers ! i)
    (composition-round-layers ! i)
  using recorded[OF round-bound] idx-eq idx-eq-query raw-eq'
    chunk-i-eq trace-roots-eq composition-roots-eq
  by simp
have tr-s0-suc:
  PTranscript s0 = chunk @ PTranscript s-suc
  using tr-s0 transcript-round by simp
have auth-suc:
  fri-layer-chunk-authenticated rt (clength * scale)
    (index (to-nat raw)) (trace-round-layers ! i ! 0) s-suc
  by (rule
    verifier-query-round-after-index-trace-base-recorded-chunk-authenticated
    [OF f-fl-eq after recorded-i tr-s0-suc])
have auth-final:
  fri-layer-chunk-authenticated rt (clength * scale)
    (index (to-nat raw)) (trace-round-layers ! i ! 0)
    final-state
  by (rule fri-layer-chunk-authenticated-mono[OF auth-suc suffix-ext])
have raw-idx-query: index (to-nat raw) = query-idxs ! i
  using idx-eq idx-eq-query raw-eq' by simp
show ?thesis
  using auth-final rt-eq raw-idx-query by simp
qed
qed
qed
qed

```

lemma *accepted-fri-opening-transcript-composition-base-recorded-chunk-authenticated-at:*
assumes *fri-openings:*
accepted-fri-opening-transcript s out trace-roots trace-bs trace-final
dg composition-roots composition-bs composition-final query-idxs
trace-round-layers composition-round-layers
and *out-eq:* *out = Some (result, final-state)*
and *round-bound:* *i < rounds*

and *composition-nonempty*: $0 < \text{length } \text{composition-roots}$
shows
fri-layer-chunk-authenticated (*composition-roots* ! 0) (*clength* * *scale*)
(*query-idxs* ! *i*) (*composition-round-layers* ! *i* ! 0) *final-state*
proof –
from *fri-openings* **obtain** *result'* *final-state'* *fr* *f-fl* *as* *fl* *query-state*
raw-idxs *query-chunks* **where**
out-def: *out* = *Some* (*result'*, *final-state'*)
and *trace-roots-eq*: *trace-roots* = *map snd f-fl*
and *composition-roots-eq*: *composition-roots* = *map snd fl*
and *outcome*:
Some (*result'*, *final-state'*) ∈
set-dist (*execute*
(*ntimes*
(*verifier-query-round-program* *fr* *f-fl* *trace-final* *as* *fl*
composition-final)
rounds)
query-state)
and *len-raw*: *length raw-idxs* = *rounds*
and *query-idxs-eq*:
query-idxs = *map* ($\lambda \text{raw. index (to-nat raw)}$) *raw-idxs*
and *len-query-chunks*: *length query-chunks* = *rounds*
and *transcript-query*:
PTranscript query-state =
List.concat query-chunks @ PTranscript final-state'
and *query-chunk*:
 $\bigwedge i. i < \text{rounds} \implies$
verifier-query-round-chunk (*query-idxs* ! *i*)
(*map snd f-fl*) (*map snd fl*) (*query-chunks* ! *i*)
and *replay-lookup*:
 $\bigwedge i. i < \text{rounds} \implies$
fmlookup (*HashMap final-state'*)
(*QueryIndexChallenge* (*PQueryCounter query-state* + *i*)
(*state-after-query-chunks* (*PState query-state*) *query-chunks* *i*)) =
Some (*raw-idxs* ! *i*)
and *recorded*:
 $\bigwedge i. i < \text{rounds} \implies$
query-round-fri-layer-transcripts (*query-idxs* ! *i*)
trace-roots *composition-roots* (*query-chunks* ! *i*)
(*trace-round-layers* ! *i*) (*composition-round-layers* ! *i*)
unfolding *accepted-fri-opening-transcript-def* **by** *blast*
have *result-eq*: *result'* = *result*
using *out-def out-eq* **by** *simp*
have *final-state-eq*: *final-state'* = *final-state*
using *out-def out-eq* **by** *simp*
have *outcome'*:
Some (*result*, *final-state*) ∈
set-dist (*execute*
(*ntimes*

```

    (verifier-query-round-program fr f-fl trace-final as fl
      composition-final)
    rounds)
  query-state)
  using outcome result-eq final-state-eq by simp
have fl-nonempty: fl ≠ []
  using composition-nonempty composition-roots-eq by auto
then obtain bf fl-tail where fl-cons: fl = bf # fl-tail
  by (cases fl) auto
obtain b rt where bf-eq: bf = (b, rt)
  by (cases bf)
have fl-eq: fl = (b, rt) # fl-tail
  using fl-cons bf-eq by simp
have rt-eq: rt = composition-roots ! 0
  using fl-cons bf-eq composition-roots-eq by simp
let ?round =
  verifier-query-round-program fr f-fl trace-final as fl composition-final
from query-opening-ntimes-outcome-decomp-at[OF outcome' round-bound]
obtain prefix x suffix s-i s-suc where
  prefix:
    Some (prefix, s-i) ∈
      set-dist (execute (ntimes ?round i) query-state)
  and round:
    Some (x, s-suc) ∈ set-dist (execute ?round s-i)
  and suffix:
    Some (suffix, final-state) ∈
      set-dist (execute (ntimes ?round (rounds - Suc i)) s-suc)
  by blast
have x-eq: x = ()
  by (cases x) simp
show ?thesis
proof (rule query-opening-ntimes-prefix-transcript-state-alignment
  [OF prefix])
  fix prefix-query-idxs prefix-chunks
  assume len-prefix-idxs: length prefix-query-idxs = i
  and len-prefix-chunks: length prefix-chunks = i
  and transcript-prefix:
    PTranscript query-state =
      List.concat prefix-chunks @ PTranscript s-i
  and state-i:
    PState s-i =
      state-after-query-chunks (PState query-state) prefix-chunks i
  and counter-i: PQueryCounter s-i = PQueryCounter query-state + i
  and prefix-ext: query-state ≤ s-i
  and prefix-rounds:
    ∧ k. k < i ⇒
      verifier-query-round-chunk (prefix-query-idxs ! k)
        (map snd f-fl) (map snd fl) (prefix-chunks ! k)
  from round[unfolded x-eq verifier-query-round-program-alt-def]

```

```

obtain raw s0 where
  raw-out:
    Some (raw, s0) ∈
      set-dist (execute receive-query-index-challenge s-i)
  and after:
    Some ((), s-suc) ∈
      set-dist (execute
        (verifier-query-round-after-index-program fr f-fl trace-final as
          fl composition-final raw) s0)
  by (auto elim!: set-dist-bindE)
have tr-s0: PTranscript s0 = PTranscript s-i
  using receive-query-index-challenge-outcome[OF raw-out] by simp
show ?thesis
proof (rule verifier-query-round-program-outcome[OF round[unfolded x-eq]])
  fix raw' idx chunk
  assume idx-eq: idx = index (to-nat raw')
  and chunk-shape:
    verifier-query-round-chunk idx (map snd f-fl) (map snd fl)
    chunk
  and transcript-round:
    PTranscript s-i = chunk @ PTranscript s-suc
  and state-suc:
    PState s-suc = foldl concat (PState s-i) chunk
  and round-ext: s-i ≤ s-suc
  and counter-suc:
    PQueryCounter s-suc = Suc (PQueryCounter s-i)
  and lookup-suc:
    fmlookup (HashMap s-suc)
      (QueryIndexChallenge (PQueryCounter s-i) (PState s-i)) =
      Some raw'
  have raw-eq': raw = raw'
  proof -
    have lookup-s0:
      fmlookup (HashMap s0)
        (QueryIndexChallenge (PQueryCounter s-i) (PState s-i)) =
        Some raw
    using receive-query-index-challenge-outcome[OF raw-out] by simp
  have s0-suc: s0 ≤ s-suc
  using verifier-query-round-after-index-program-outcome[OF after]
  by simp
  have lookup-suc-raw:
    fmlookup (HashMap s-suc)
      (QueryIndexChallenge (PQueryCounter s-i) (PState s-i)) =
      Some raw
  using hash-extension-lookup[OF lookup-s0 s0-suc] .
  show ?thesis
  using lookup-suc lookup-suc-raw by simp
qed
show ?thesis

```

```

proof (rule query-opening-ntimes-suffix-transcript-state-alignment
  [OF suffix])
fix suffix-query-idxs suffix-chunks
assume len-suffix-idxs:
  length suffix-query-idxs = rounds - Suc i
and len-suffix-chunks:
  length suffix-chunks = rounds - Suc i
and transcript-suffix:
  PTranscript s-suc =
    List.concat suffix-chunks @ PTranscript final-state
and suffix-state:
  PState final-state =
    state-after-query-chunks (PState s-suc) suffix-chunks
    (rounds - Suc i)
and suffix-counter:
  PQueryCounter final-state =
    PQueryCounter s-suc + (rounds - Suc i)
and suffix-ext: s-suc ≤ final-state
and suffix-rounds:
   $\bigwedge k. k < rounds - Suc i \implies$ 
    verifier-query-round-chunk (suffix-query-idxs ! k)
    (map snd f-fl) (map snd fl) (suffix-chunks ! k)
let ?selected-chunks = prefix-chunks @ chunk # suffix-chunks
have len-selected-chunks: length ?selected-chunks = rounds
  using len-prefix-chunks len-suffix-chunks round-bound by simp
have selected-chunk-lens:
   $\bigwedge k. k < rounds \implies$ 
    length (?selected-chunks ! k) =
      verifier-query-round-transcript-length (query-idxs ! k)
      (map snd f-fl) (map snd fl)
proof -
  fix k
  assume k-bound: k < rounds
  show
    length (?selected-chunks ! k) =
      verifier-query-round-transcript-length (query-idxs ! k)
      (map snd f-fl) (map snd fl)
proof (cases k < i)
  case True
  have nth-eq: ?selected-chunks ! k = prefix-chunks ! k
    using True len-prefix-chunks by (simp add: nth-append)
  have
    length (prefix-chunks ! k) =
      verifier-query-round-transcript-length
        (prefix-query-idxs ! k) (map snd f-fl) (map snd fl)
    by (rule verifier-query-round-chunk-length
      [OF prefix-rounds[OF True]])
  then show ?thesis
    using nth-eq

```

```

    by (simp add:
        verifier-query-round-transcript-length-index-irrelevant)
next
case False
show ?thesis
proof (cases k = i)
case True
have nth-eq: ?selected-chunks ! k = chunk
    using True len-prefix-chunks by (simp add: nth-append)
have
    length chunk =
        verifier-query-round-transcript-length idx
        (map snd f-fl) (map snd fl)
    by (rule verifier-query-round-chunk-length[OF chunk-shape])
then show ?thesis
    using nth-eq
    by (simp add:
        verifier-query-round-transcript-length-index-irrelevant)
next
case False
let ?k = k - Suc i
have ge: Suc i ≤ k
    using <¬ k < i> False by linarith
have k-bound!: ?k < rounds - Suc i
    using ge k-bound by linarith
have nth-eq: ?selected-chunks ! k = suffix-chunks ! ?k
    using ge len-prefix-chunks by (simp add: nth-append)
have
    length (suffix-chunks ! ?k) =
        verifier-query-round-transcript-length
        (suffix-query-idxs ! ?k) (map snd f-fl) (map snd fl)
    by (rule verifier-query-round-chunk-length
        [OF suffix-rounds[OF k-bound!]])
then show ?thesis
    using nth-eq
    by (simp add:
        verifier-query-round-transcript-length-index-irrelevant)
qed
qed
qed
have query-chunk-lens:
     $\bigwedge k. k < rounds \implies$ 
    length (query-chunks ! k) =
        verifier-query-round-transcript-length (query-idxs ! k)
        (map snd f-fl) (map snd fl)
    using query-chunk verifier-query-round-chunk-length by blast
have selected-transcript:
    PTranscript query-state =
        List.concat ?selected-chunks @ PTranscript final-state

```

```

using transcript-prefix transcript-round transcript-suffix by simp
have concat-eq: List.concat ?selected-chunks = List.concat query-chunks
using selected-transcript transcript-query final-state-eq by simp
have chunks-eq: ?selected-chunks = query-chunks
proof -
  have same-lens:
     $\bigwedge k. k < \text{length } ?\text{selected-chunks} \implies$ 
       $\text{length } (?selected-chunks ! k) = \text{length } (query-chunks ! k)$ 
    using selected-chunk-lens query-chunk-lens len-selected-chunks
    by simp
  have concat-eq-empty:
    List.concat ?selected-chunks @ [] = List.concat query-chunks
    using concat-eq by simp
  have len-eq: length ?selected-chunks = length query-chunks
    using len-selected-chunks len-query-chunks by simp
  have ?selected-chunks = query-chunks  $\wedge$  ( [] :: 'f list ) = []
    by (rule concat-append-eq-concat-same-chunk-lengths
      [OF len-eq same-lens concat-eq-empty])
  then show ?thesis
    by simp
qed
have prefix-state-eq:
  state-after-query-chunks (PState query-state) prefix-chunks i =
    state-after-query-chunks (PState query-state) query-chunks i
proof -
  have
    state-after-query-chunks (PState query-state) ?selected-chunks i =
      state-after-query-chunks (PState query-state) prefix-chunks i
    by (rule state-after-query-chunks-append-prefix
      [OF len-prefix-chunks])
  then show ?thesis
    using chunks-eq by simp
qed
have lookup-final:
  fmlookup (HashMap final-state)
    (QueryIndexChallenge (PQueryCounter query-state + i)
      (state-after-query-chunks (PState query-state) query-chunks i)) =
    Some raw'
proof -
  have lookup-suc':
    fmlookup (HashMap s-suc)
      (QueryIndexChallenge (PQueryCounter query-state + i)
        (state-after-query-chunks (PState query-state)
          prefix-chunks i)) =
      Some raw'
    using lookup-suc state-i counter-i by simp
  have lookup-final-prefix:
    fmlookup (HashMap final-state)
      (QueryIndexChallenge (PQueryCounter query-state + i)

```



```

      (state-after-query-chunks (PState query-state)
        prefix-chunks i)) =
    Some raw'
    using hash-extension-lookup[OF lookup-suc' suffix-ext] .
  show ?thesis
    using lookup-final-prefix prefix-state-eq by simp
qed
have raw-idx-eq: raw' = raw-idxs ! i
  using lookup-final replay-lookup[OF round-bound] final-state-eq
  by simp
have idx-eq-query: idx = query-idxs ! i
  using idx-eq raw-idx-eq query-idxs-eq len-raw round-bound by simp
have selected-i: ?selected-chunks ! i = chunk
  using len-prefix-chunks by (simp add: nth-append)
have chunk-i-eq: chunk = query-chunks ! i
  using chunks-eq selected-i by simp
have recorded-i:
  query-round-fri-layer-transcripts (index (to-nat raw))
    (map snd f-fl) (map snd fl) chunk (trace-round-layers ! i)
    (composition-round-layers ! i)
  using recorded[OF round-bound] idx-eq idx-eq-query raw-eq'
    chunk-i-eq trace-roots-eq composition-roots-eq
  by simp
have tr-s0-suc:
  PTranscript s0 = chunk @ PTranscript s-suc
  using tr-s0 transcript-round by simp
have auth-suc:
  fri-layer-chunk-authenticated rt (clength * scale)
    (index (to-nat raw)) (composition-round-layers ! i ! 0) s-suc
  by (rule
    verifier-query-round-after-index-composition-base-recorded-chunk-authenticated
    [OF fl-eq after recorded-i tr-s0-suc])
have auth-final:
  fri-layer-chunk-authenticated rt (clength * scale)
    (index (to-nat raw)) (composition-round-layers ! i ! 0)
    final-state
  by (rule fri-layer-chunk-authenticated-mono[OF auth-suc suffix-ext])
have raw-idx-query: index (to-nat raw) = query-idxs ! i
  using idx-eq idx-eq-query raw-eq' by simp
show ?thesis
  using auth-final rt-eq raw-idx-query by simp
qed
qed
qed
qed

end

end

```

```

theory Soundness-FRI-Raw-Layer-Bounds
  imports Soundness-FRI-Authenticated-Route
begin

```

Small reusable bounds for FRI layer indices derived from accepted verifier transcripts. These keep later replay proofs from repeatedly rebuilding the same *fri-layer-raw-bound* arithmetic.

```

context soundness
begin

```

lemma *accepted-fri-opening-transcript-trace-raw-layer-bound*:

assumes *fri-openings*:

accepted-fri-opening-transcript s out trace-roots trace-bs trace-final
dg composition-roots composition-bs composition-final query-idxs
trace-round-layers composition-round-layers

and *layer-bound*: $j < \text{length } \text{trace-roots}$

shows

$\bigwedge \text{raw.}$
 $0 < \text{fri-layer-lengths } (\text{length } \text{trace-roots}) (\text{clength} * \text{scale}) ! j \wedge$
 $\text{fri-layer-indices } (\text{length } \text{trace-roots}) (\text{index } (\text{to-nat } \text{raw}))$
 $(\text{clength} * \text{scale}) ! j <$
 $\text{fri-layer-lengths } (\text{length } \text{trace-roots}) (\text{clength} * \text{scale}) ! j$

proof –

obtain N **where** *eval-power*: $\text{clength} * \text{scale} = 2^N$

using *eval-domain-length-power* **by** *blast*

have *trace-roots-le*: $\text{length } \text{trace-roots} \leq N$

proof –

have *len*: $\text{length } \text{trace-roots} = \text{ceil-log } \text{clength}$

using *accepted-fri-opening-transcript-shapes(1)[OF fri-openings]* .

have $\text{clength} \leq \text{clength} * \text{scale}$

using *scale-pos* **by** *simp*

also have $\dots = 2^N$

by (*rule eval-power*)

finally have $\text{ceil-log } \text{clength} \leq N$

by (*rule ceil-log-le-power*)

then show *?thesis*

using *len* **by** *simp*

qed

show $\bigwedge \text{raw.}$

$0 < \text{fri-layer-lengths } (\text{length } \text{trace-roots}) (\text{clength} * \text{scale}) ! j \wedge$

$\text{fri-layer-indices } (\text{length } \text{trace-roots}) (\text{index } (\text{to-nat } \text{raw}))$

$(\text{clength} * \text{scale}) ! j <$

$\text{fri-layer-lengths } (\text{length } \text{trace-roots}) (\text{clength} * \text{scale}) ! j$

by (*rule fri-layer-raw-bound[OF layer-bound eval-power trace-roots-le]*)

qed

lemma *accepted-fri-opening-transcript-composition-raw-layer-bound*:

```

assumes fri-openings:
  accepted-fri-opening-transcript s out trace-roots trace-bs trace-final
  dg composition-roots composition-bs composition-final query-idxs
  trace-round-layers composition-round-layers
and degree-bound:  $\text{to-nat } dg \leq \text{maxDegree}$ 
and layer-bound:  $j < \text{length } \text{composition-roots}$ 
shows
   $\bigwedge \text{raw.}$ 
     $0 < \text{fri-layer-lengths } (\text{length } \text{composition-roots}) (\text{clength} * \text{scale}) ! j \wedge$ 
     $\text{fri-layer-indices } (\text{length } \text{composition-roots}) (\text{index } (\text{to-nat } \text{raw}))$ 
     $(\text{clength} * \text{scale}) ! j <$ 
     $\text{fri-layer-lengths } (\text{length } \text{composition-roots}) (\text{clength} * \text{scale}) ! j$ 
proof –
  obtain  $N$  where eval-power:  $\text{clength} * \text{scale} = 2^N$ 
  using eval-domain-length-power by blast
have composition-roots-le:  $\text{length } \text{composition-roots} \leq N$ 
proof –
  have len:  $\text{length } \text{composition-roots} = \text{ceil-log } (\text{to-nat } dg + 1)$ 
  using accepted-fri-opening-transcript-shapes(3)[OF fri-openings] .
  have  $\text{to-nat } dg + 1 \leq \text{clength} * \text{scale}$ 
  using degree-bound maxDegree-less-eval-domain by linarith
also have  $\dots = 2^N$ 
  by (rule eval-power)
finally have  $\text{ceil-log } (\text{to-nat } dg + 1) \leq N$ 
  by (rule ceil-log-le-power)
then show ?thesis
  using len by simp
qed
show  $\bigwedge \text{raw.}$ 
   $0 < \text{fri-layer-lengths } (\text{length } \text{composition-roots})$ 
   $(\text{clength} * \text{scale}) ! j \wedge$ 
   $\text{fri-layer-indices } (\text{length } \text{composition-roots}) (\text{index } (\text{to-nat } \text{raw}))$ 
   $(\text{clength} * \text{scale}) ! j <$ 
   $\text{fri-layer-lengths } (\text{length } \text{composition-roots}) (\text{clength} * \text{scale}) ! j$ 
by (rule fri-layer-raw-bound
  [OF layer-bound eval-power composition-roots-le])
qed

lemma accepted-fri-opening-transcript-trace-step-with-authenticated-chunk-at-raw:
assumes fri-openings:
  accepted-fri-opening-transcript s out trace-roots trace-bs trace-final
  dg composition-roots composition-bs composition-final query-idxs
  trace-round-layers composition-round-layers
and out-eq:  $\text{out} = \text{Some } (\text{result}, \text{final-state})$ 
and round-bound:  $i < \text{rounds}$ 
and layer-bound:  $j < \text{length } \text{trace-roots}$ 
shows
  generic-fri-transcript-step-with-authenticated-chunk trace-roots
  trace-bs query-idxs trace-round-layers final-state i j

```

by (rule accepted-fri-opening-transcript-trace-step-with-authenticated-chunk-at
 [OF fri-openings out-eq round-bound layer-bound])
 (rule accepted-fri-opening-transcript-trace-raw-layer-bound
 [OF fri-openings layer-bound])

lemma *accepted-fri-opening-transcript-composition-step-with-authenticated-chunk-at-raw:*

assumes *fri-openings:*

accepted-fri-opening-transcript s out trace-roots trace-bs trace-final
dg composition-roots composition-bs composition-final query-idxs
trace-round-layers composition-round-layers

and *out-eq:* *out = Some (result, final-state)*
and *degree-bound:* *to-nat dg ≤ maxDegree*
and *round-bound:* *i < rounds*
and *layer-bound:* *j < length composition-roots*

shows

generic-fri-transcript-step-with-authenticated-chunk composition-roots
composition-bs query-idxs composition-round-layers final-state i j

by (rule

accepted-fri-opening-transcript-composition-step-with-authenticated-chunk-at
 [OF fri-openings out-eq round-bound layer-bound])

(rule *accepted-fri-opening-transcript-composition-raw-layer-bound*
 [OF fri-openings degree-bound layer-bound])

end

end

theory *Soundness-FRI-Final-Step-Replay*

imports *Soundness-FRI-Raw-Layer-Bounds*

begin

Selected final-step replay for trace FRI openings.

This layer lifts the monadic replay fact for the last trace FRI fold step through the accepted FRI transcript. It keeps the result local to sampled verifier evidence and does not reconstruct complete FRI tables.

context *soundness*

begin

lemma *accepted-fri-opening-transcript-trace-final-step-evidence-at:*

assumes *fri-openings:*

accepted-fri-opening-transcript s out trace-roots trace-bs trace-final
dg composition-roots composition-bs composition-final query-idxs
trace-round-layers composition-round-layers

and *out-eq:* *out = Some (result, final-state)*

and *round-bound:* *i < rounds*

and *nonempty:* *0 < length trace-roots*

and *raw-layer:*

$\bigwedge$ *raw.*

```

0 < fri-layer-lengths (length trace-roots) (clength * scale) !
  (length trace-roots - 1) ∧
fri-layer-indices (length trace-roots) (index (to-nat raw))
  (clength * scale) ! (length trace-roots - 1) <
fri-layer-lengths (length trace-roots) (clength * scale) !
  (length trace-roots - 1)
obtains actual-chunk axp axp-path axn axn-path where
  fri-layer-chunk-authenticated
    (trace-roots ! (length trace-roots - 1))
    (fri-evidence-layer-len trace-roots (length trace-roots - 1))
    (fri-evidence-layer-idx trace-roots query-idxs i
      (length trace-roots - 1))
    actual-chunk final-state
  fri-layer-step-evidence
    (trace-roots ! (length trace-roots - 1))
    (trace-bs ! (length trace-roots - 1))
    (fri-evidence-layer-len trace-roots (length trace-roots - 1))
    (fri-evidence-layer-idx trace-roots query-idxs i
      (length trace-roots - 1))
    (2 ^ (length trace-roots - 1))
    (fri-sibling-index
      (fri-evidence-layer-len trace-roots (length trace-roots - 1))
      (fri-evidence-layer-idx trace-roots query-idxs i
        (length trace-roots - 1)))
    axp axp-path axn axn-path
    (fri-evidence-next-idx trace-roots query-idxs i
      (length trace-roots - 1))
    trace-final actual-chunk
proof -
from fri-openings obtain result' final-state' fr f-fl as fl query-state
  raw-idxs query-chunks where
  out-def: out = Some (result', final-state')
and trace-roots-eq: trace-roots = map snd f-fl
and trace-bs-eq: trace-bs = map fst f-fl
and composition-roots-eq: composition-roots = map snd fl
and query-out:
  Some (result', final-state') ∈
    set-dist
      (execute
        (ntimes
          (verifier-query-round-program fr f-fl trace-final as fl
            composition-final)
          rounds)
        query-state)
and len-raw: length raw-idxs = rounds
and query-idxs-eq:
  query-idxs = map (λraw. index (to-nat raw)) raw-idxs
and len-query-chunks: length query-chunks = rounds
and transcript-query:

```

```

PTranscript query-state =
  List.concat query-chunks @ PTranscript final-state'
and query-chunk:
   $\bigwedge i. i < \text{rounds} \implies$ 
    verifier-query-round-chunk (query-idxs ! i)
    trace-roots composition-roots (query-chunks ! i)
and replay-lookup:
   $\bigwedge i. i < \text{rounds} \implies$ 
    fmllookup (HashMap final-state')
    (QueryIndexChallenge (PQueryCounter query-state + i)
      (state-after-query-chunks (PState query-state) query-chunks i)) =
    Some (raw-idxs ! i)
unfolding accepted-fri-opening-transcript-def by blast
have result-eq: result' = result
using out-def out-eq by simp
have final-state-eq: final-state' = final-state
using out-def out-eq by simp
have query-out':
  Some (result, final-state)  $\in$ 
    set-dist
    (execute
      (ntimes
        (verifier-query-round-program fr f-fl trace-final as fl
          composition-final)
        rounds)
      query-state)
using query-out result-eq final-state-eq by simp
have f-fl-nonempty: 0 < length f-fl
using nonempty trace-roots-eq by simp
have raw-layer-f:
   $\bigwedge \text{raw.}$ 
    0 < fri-layer-lengths (length f-fl) (clength * scale) !
    (length f-fl - 1)  $\wedge$ 
    fri-layer-indices (length f-fl) (index (to-nat raw))
    (clength * scale) ! (length f-fl - 1) <
    fri-layer-lengths (length f-fl) (clength * scale) !
    (length f-fl - 1)
using raw-layer trace-roots-eq by simp
from ntimes-verifier-query-rounds-selected-trace-fri-last-step-with-prefix-at
  [OF query-out' round-bound f-fl-nonempty raw-layer-f]
obtain raw idx prefix-query-idxs prefix-chunks chunk suffix-query-idxs
  suffix-chunks actual-chunk axp axp-path axn axn-path where
idx-eq: idx = index (to-nat raw)
and len-prefix-idxs: length prefix-query-idxs = i
and len-prefix-chunks: length prefix-chunks = i
and len-suffix-idxs: length suffix-query-idxs = rounds - Suc i
and len-suffix-chunks: length suffix-chunks = rounds - Suc i
and chunk-shape:
  verifier-query-round-chunk idx (map snd f-fl) (map snd fl) chunk

```

```

and prefix-rounds:
   $\bigwedge k. k < i \implies$ 
    verifier-query-round-chunk (prefix-query-idxs ! k)
      (map snd f-fl) (map snd fl) (prefix-chunks ! k)
and suffix-rounds:
   $\bigwedge k. k < \text{rounds} - \text{Suc } i \implies$ 
    verifier-query-round-chunk (suffix-query-idxs ! k)
      (map snd f-fl) (map snd fl) (suffix-chunks ! k)
and selected-transcript:
  PTranscript query-state =
    List.concat (prefix-chunks @ chunk # suffix-chunks) @
      PTranscript final-state
and auth:
  fri-layer-chunk-authenticated
    (snd (f-fl ! (length f-fl - 1)))
    (fri-layer-lengths (length f-fl) (clength * scale) !
      (length f-fl - 1))
    (fri-layer-indices (length f-fl) idx (clength * scale) !
      (length f-fl - 1))
    actual-chunk final-state
and step:
  fri-layer-step-evidence
    (snd (f-fl ! (length f-fl - 1)))
    (fst (f-fl ! (length f-fl - 1)))
    (fri-layer-lengths (length f-fl) (clength * scale) !
      (length f-fl - 1))
    (fri-layer-indices (length f-fl) idx (clength * scale) !
      (length f-fl - 1))
    ( $2 \wedge (\text{length } f\text{-fl} - 1)$ )
    (fri-sibling-index
      (fri-layer-lengths (length f-fl) (clength * scale) !
        (length f-fl - 1))
      (fri-layer-indices (length f-fl) idx (clength * scale) !
        (length f-fl - 1)))
    axp axp-path axn axn-path
    ((fri-layer-indices (length f-fl) idx (clength * scale) !
      (length f-fl - 1)) mod
      ((fri-layer-lengths (length f-fl) (clength * scale) !
        (length f-fl - 1)) div 2))
    trace-final actual-chunk
and selected-lookup:
  fmlookup (HashMap final-state)
    (QueryIndexChallenge (PQueryCounter query-state + i)
      (state-after-query-chunks (PState query-state) prefix-chunks i)) =
    Some raw
by (rule
  ntimes-verifier-query-rounds-selected-trace-fri-last-step-with-prefix-at
    [OF query-out' round-bound f-fl-nonempty raw-layer-f])
  (rule that, assumption+)

```

```

let ?selected-chunks = prefix-chunks @ chunk # suffix-chunks
have len-selected-chunks: length ?selected-chunks = rounds
  using len-prefix-chunks len-suffix-chunks round-bound by simp
have selected-chunk-lens:
   $\wedge k. k < rounds \implies$ 
    length (?selected-chunks ! k) =
      verifier-query-round-transcript-length (query-idxs ! k)
        (map snd f-fl) (map snd fl)
proof -
  fix k
  assume k-bound: k < rounds
  show
    length (?selected-chunks ! k) =
      verifier-query-round-transcript-length (query-idxs ! k)
        (map snd f-fl) (map snd fl)
  proof (cases k < i)
    case True
    have nth-eq: ?selected-chunks ! k = prefix-chunks ! k
      using True len-prefix-chunks by (simp add: nth-append)
    have
      length (prefix-chunks ! k) =
        verifier-query-round-transcript-length
          (prefix-query-idxs ! k) (map snd f-fl) (map snd fl)
      by (rule verifier-query-round-chunk-length[OF prefix-rounds[OF True]])
    then show ?thesis
      using nth-eq
      by (simp add: verifier-query-round-transcript-length-index-irrelevant)
  next
    case False
    show ?thesis
    proof (cases k = i)
      case True
      have nth-eq: ?selected-chunks ! k = chunk
        using True len-prefix-chunks by (simp add: nth-append)
      have
        length chunk =
          verifier-query-round-transcript-length idx
            (map snd f-fl) (map snd fl)
        by (rule verifier-query-round-chunk-length[OF chunk-shape])
      then show ?thesis
        using nth-eq
        by (simp add: verifier-query-round-transcript-length-index-irrelevant)
    next
      case False
      let ?k = k - Suc i
      have ge: Suc i  $\leq$  k
        using  $\langle \neg k < i \rangle$  False by linarith
      have k-bound': ?k < rounds - Suc i
        using ge k-bound by linarith

```



```

have nth-eq: ?selected-chunks ! k = suffix-chunks ! ?k
  using ge len-prefix-chunks by (simp add: nth-append)
have
  length (suffix-chunks ! ?k) =
    verifier-query-round-transcript-length
      (suffix-query-idxs ! ?k) (map snd f-fl) (map snd fl)
  by (rule verifier-query-round-chunk-length
    [OF suffix-rounds[OF k-bound]])
then show ?thesis
  using nth-eq
  by (simp add: verifier-query-round-transcript-length-index-irrelevant)
qed
qed
qed
have query-chunk-lens:
   $\bigwedge k. k < \text{rounds} \implies$ 
    length (query-chunks ! k) =
      verifier-query-round-transcript-length (query-idxs ! k)
        (map snd f-fl) (map snd fl)
proof –
  fix k
  assume k-bound: k < rounds
  have
    verifier-query-round-chunk (query-idxs ! k)
      (map snd f-fl) (map snd fl) (query-chunks ! k)
  using query-chunk[OF k-bound] trace-roots-eq composition-roots-eq
  by simp
  then show
    length (query-chunks ! k) =
      verifier-query-round-transcript-length (query-idxs ! k)
        (map snd f-fl) (map snd fl)
  by (rule verifier-query-round-chunk-length)
qed
have concat-eq: List.concat ?selected-chunks = List.concat query-chunks
  using selected-transcript transcript-query final-state-eq by simp
have chunks-eq: ?selected-chunks = query-chunks
proof –
  have same-lens:
     $\bigwedge k. k < \text{length } ?\text{selected-chunks} \implies$ 
      length (?selected-chunks ! k) = length (query-chunks ! k)
  using selected-chunk-lens query-chunk-lens len-selected-chunks by simp
have concat-eq-empty:
    List.concat ?selected-chunks @ [] = List.concat query-chunks
  using concat-eq by simp
have len-eq: length ?selected-chunks = length query-chunks
  using len-selected-chunks len-query-chunks by simp
have ?selected-chunks = query-chunks  $\wedge$  ( [] :: 'f list ) = []
  by (rule concat-append-eq-concat-same-chunk-lengths
    [OF len-eq same-lens concat-eq-empty])

```

```

then show ?thesis by simp
qed
have prefix-state-eq:
  state-after-query-chunks (PState query-state) prefix-chunks i =
    state-after-query-chunks (PState query-state) query-chunks i
proof -
  have
    state-after-query-chunks (PState query-state) ?selected-chunks i =
      state-after-query-chunks (PState query-state) prefix-chunks i
    by (rule state-after-query-chunks-append-prefix[OF len-prefix-chunks])
  then show ?thesis
    using chunks-eq by simp
qed
have selected-lookup-global:
  fmlookup (HashMap final-state)
    (QueryIndexChallenge (PQueryCounter query-state + i)
      (state-after-query-chunks (PState query-state) query-chunks i)) =
    Some raw
  using selected-lookup prefix-state-eq by simp
have raw-eq: raw = raw-idxs ! i
  using selected-lookup-global replay-lookup[OF round-bound]
    final-state-eq by simp
have idx-eq-query: idx = query-idxs ! i
  using idx-eq raw-eq query-idxs-eq len-raw round-bound by simp
have auth':
  fri-layer-chunk-authenticated
    (trace-roots ! (length trace-roots - 1))
    (fri-evidence-layer-len trace-roots (length trace-roots - 1))
    (fri-evidence-layer-idx trace-roots query-idxs i
      (length trace-roots - 1))
    actual-chunk final-state
  using auth idx-eq-query trace-roots-eq f-fl-nonempty
  unfolding fri-evidence-layer-len-def fri-evidence-layer-idx-def
  by simp
have step':
  fri-layer-step-evidence
    (trace-roots ! (length trace-roots - 1))
    (trace-bs ! (length trace-roots - 1))
    (fri-evidence-layer-len trace-roots (length trace-roots - 1))
    (fri-evidence-layer-idx trace-roots query-idxs i
      (length trace-roots - 1))
    (2 ^ (length trace-roots - 1))
    (fri-sibling-index
      (fri-evidence-layer-len trace-roots (length trace-roots - 1))
      (fri-evidence-layer-idx trace-roots query-idxs i
        (length trace-roots - 1)))
    axp axp-path axn axn-path
    (fri-evidence-next-idx trace-roots query-idxs i
      (length trace-roots - 1))

```

```

    trace-final actual-chunk
using step idx-eq-query trace-roots-eq trace-bs-eq f-fl-nonempty
unfolding fri-evidence-layer-len-def fri-evidence-layer-idx-def
    fri-evidence-next-idx-def
by simp
show ?thesis
    by (rule that[OF auth' step'])
qed

```

lemma *accepted-fri-opening-transcript-composition-final-step-evidence-at:*
assumes *fri-openings:*

```

    accepted-fri-opening-transcript s out trace-roots trace-bs trace-final
    dg composition-roots composition-bs composition-final query-idxs
    trace-round-layers composition-round-layers

```

```

and out-eq: out = Some (result, final-state)
and round-bound: i < rounds
and nonempty: 0 < length composition-roots
and raw-layer:

```

```

     $\bigwedge \text{raw.}$ 
    0 < fri-layer-lengths (length composition-roots) (clength * scale) !
    (length composition-roots - 1)  $\wedge$ 
    fri-layer-indices (length composition-roots) (index (to-nat raw))
    (clength * scale) ! (length composition-roots - 1) <
    fri-layer-lengths (length composition-roots) (clength * scale) !
    (length composition-roots - 1)

```

obtains *actual-chunk axp axp-path axn axn-path* **where**

```

    fri-layer-chunk-authenticated
    (composition-roots ! (length composition-roots - 1))
    (fri-evidence-layer-len composition-roots (length composition-roots - 1))
    (fri-evidence-layer-idx composition-roots query-idxs i
    (length composition-roots - 1))
    actual-chunk final-state
    fri-layer-step-evidence
    (composition-roots ! (length composition-roots - 1))
    (composition-bs ! (length composition-roots - 1))
    (fri-evidence-layer-len composition-roots (length composition-roots - 1))
    (fri-evidence-layer-idx composition-roots query-idxs i
    (length composition-roots - 1))
    (2 ^ (length composition-roots - 1))
    (fri-sibling-index
    (fri-evidence-layer-len composition-roots (length composition-roots - 1))
    (fri-evidence-layer-idx composition-roots query-idxs i
    (length composition-roots - 1)))
    axp axp-path axn axn-path
    (fri-evidence-next-idx composition-roots query-idxs i
    (length composition-roots - 1))
    composition-final actual-chunk

```

proof —

from *fri-openings* **obtain** *result' final-state' fr f-fl as fl query-state*

```

    raw-idxs query-chunks where
    out-def: out = Some (result', final-state')
    and trace-roots-eq: trace-roots = map snd f-fl
    and composition-bs-eq: composition-bs = map fst fl
    and composition-roots-eq: composition-roots = map snd fl
    and query-out:
      Some (result', final-state') ∈
        set-dist
          (execute
            (ntimes
              (verifier-query-round-program fr f-fl trace-final as fl
                composition-final)
              rounds)
            query-state)
    and len-raw: length raw-idxs = rounds
    and query-idxs-eq:
      query-idxs = map (λraw. index (to-nat raw)) raw-idxs
    and len-query-chunks: length query-chunks = rounds
    and transcript-query:
      PTranscript query-state =
        List.concat query-chunks @ PTranscript final-state'
    and query-chunk:
      ∧ i. i < rounds ⇒
        verifier-query-round-chunk (query-idxs ! i)
        trace-roots composition-roots (query-chunks ! i)
    and replay-lookup:
      ∧ i. i < rounds ⇒
        fmlookup (HashMap final-state')
          (QueryIndexChallenge (PQueryCounter query-state + i)
            (state-after-query-chunks (PState query-state) query-chunks i)) =
          Some (raw-idxs ! i)
    unfolding accepted-fri-opening-transcript-def by blast
    have result-eq: result' = result
      using out-def out-eq by simp
    have final-state-eq: final-state' = final-state
      using out-def out-eq by simp
    have query-out':
      Some (result, final-state) ∈
        set-dist
          (execute
            (ntimes
              (verifier-query-round-program fr f-fl trace-final as fl
                composition-final)
              rounds)
            query-state)
      using query-out result-eq final-state-eq by simp
    have fl-nonempty: 0 < length fl
      using nonempty composition-roots-eq by simp
    have raw-layer-fl:

```

```

 $\wedge$ raw.
  0 < fri-layer-lengths (length fl) (clength * scale) !
  (length fl - 1)  $\wedge$ 
  fri-layer-indices (length fl) (index (to-nat raw))
  (clength * scale) ! (length fl - 1) <
  fri-layer-lengths (length fl) (clength * scale) !
  (length fl - 1)
using raw-layer composition-roots-eq by simp
from ntimes-verifier-query-rounds-selected-composition-fri-last-step-with-prefix-at
  [OF query-out' round-bound fl-nonempty raw-layer-fl]
obtain raw idx prefix-query-idxs prefix-chunks chunk suffix-query-idxs
  suffix-chunks actual-chunk axp axp-path axn axn-path where
  idx-eq: idx = index (to-nat raw)
and len-prefix-idxs: length prefix-query-idxs = i
and len-prefix-chunks: length prefix-chunks = i
and len-suffix-idxs: length suffix-query-idxs = rounds - Suc i
and len-suffix-chunks: length suffix-chunks = rounds - Suc i
and chunk-shape:
  verifier-query-round-chunk idx (map snd f-fl) (map snd fl) chunk
and prefix-rounds:
   $\wedge$ k. k < i  $\implies$ 
    verifier-query-round-chunk (prefix-query-idxs ! k)
    (map snd f-fl) (map snd fl) (prefix-chunks ! k)
and suffix-rounds:
   $\wedge$ k. k < rounds - Suc i  $\implies$ 
    verifier-query-round-chunk (suffix-query-idxs ! k)
    (map snd f-fl) (map snd fl) (suffix-chunks ! k)
and selected-transcript:
  PTranscript query-state =
    List.concat (prefix-chunks @ chunk # suffix-chunks) @
    PTranscript final-state
and auth:
  fri-layer-chunk-authenticated
  (snd (fl ! (length fl - 1)))
  (fri-layer-lengths (length fl) (clength * scale) !
  (length fl - 1))
  (fri-layer-indices (length fl) idx (clength * scale) !
  (length fl - 1))
  actual-chunk final-state
and step:
  fri-layer-step-evidence
  (snd (fl ! (length fl - 1)))
  (fst (fl ! (length fl - 1)))
  (fri-layer-lengths (length fl) (clength * scale) !
  (length fl - 1))
  (fri-layer-indices (length fl) idx (clength * scale) !
  (length fl - 1))
  (2  $\wedge$  (length fl - 1))
  (fri-sibling-index

```

```

    (fri-layer-lengths (length fl) (clength * scale) !
      (length fl - 1))
    (fri-layer-indices (length fl) idx (clength * scale) !
      (length fl - 1)))
  axp axp-path axn axn-path
  ((fri-layer-indices (length fl) idx (clength * scale) !
    (length fl - 1)) mod
    ((fri-layer-lengths (length fl) (clength * scale) !
      (length fl - 1)) div 2))
  composition-final actual-chunk
and selected-lookup:
  fmlookup (HashMap final-state)
    (QueryIndexChallenge (PQueryCounter query-state + i)
      (state-after-query-chunks (PState query-state) prefix-chunks i)) =
    Some raw
by (rule
  ntimes-verifier-query-rounds-selected-composition-fri-last-step-with-prefix-at
    [OF query-out' round-bound fl-nonempty raw-layer-fl])
  (rule that, assumption+)
let ?selected-chunks = prefix-chunks @ chunk # suffix-chunks
have len-selected-chunks: length ?selected-chunks = rounds
  using len-prefix-chunks len-suffix-chunks round-bound by simp
have selected-chunk-lens:
   $\wedge k. k < rounds \implies$ 
    length (?selected-chunks ! k) =
      verifier-query-round-transcript-length (query-idxs ! k)
      (map snd f-fl) (map snd fl)
proof -
  fix k
  assume k-bound: k < rounds
  show
    length (?selected-chunks ! k) =
      verifier-query-round-transcript-length (query-idxs ! k)
      (map snd f-fl) (map snd fl)
proof (cases k < i)
  case True
  have nth-eq: ?selected-chunks ! k = prefix-chunks ! k
    using True len-prefix-chunks by (simp add: nth-append)
  have
    length (prefix-chunks ! k) =
      verifier-query-round-transcript-length
        (prefix-query-idxs ! k) (map snd f-fl) (map snd fl)
    by (rule verifier-query-round-chunk-length[OF prefix-rounds[OF True]])
  then show ?thesis
    using nth-eq
    by (simp add: verifier-query-round-transcript-length-index-irrelevant)
next
  case False
  show ?thesis

```

```

proof (cases k = i)
  case True
    have nth-eq: ?selected-chunks ! k = chunk
      using True len-prefix-chunks by (simp add: nth-append)
    have
      length chunk =
        verifier-query-round-transcript-length idx
          (map snd f-fl) (map snd fl)
      by (rule verifier-query-round-chunk-length[OF chunk-shape])
    then show ?thesis
      using nth-eq
      by (simp add: verifier-query-round-transcript-length-index-irrelevant)
  next
    case False
    let ?k = k - Suc i
    have ge: Suc i ≤ k
      using <¬ k < i> False by linarith
    have k-bound': ?k < rounds - Suc i
      using ge k-bound by linarith
    have nth-eq: ?selected-chunks ! k = suffix-chunks ! ?k
      using ge len-prefix-chunks by (simp add: nth-append)
    have
      length (suffix-chunks ! ?k) =
        verifier-query-round-transcript-length
          (suffix-query-idxs ! ?k) (map snd f-fl) (map snd fl)
      by (rule verifier-query-round-chunk-length
        [OF suffix-rounds[OF k-bound']])
    then show ?thesis
      using nth-eq
      by (simp add: verifier-query-round-transcript-length-index-irrelevant)
  qed
qed
qed
have query-chunk-lens:
  ∧k. k < rounds ⇒
    length (query-chunks ! k) =
      verifier-query-round-transcript-length (query-idxs ! k)
        (map snd f-fl) (map snd fl)
proof -
  fix k
  assume k-bound: k < rounds
  have
    verifier-query-round-chunk (query-idxs ! k)
      (map snd f-fl) (map snd fl) (query-chunks ! k)
    using query-chunk[OF k-bound] trace-roots-eq composition-roots-eq
    by simp
  then show
    length (query-chunks ! k) =
      verifier-query-round-transcript-length (query-idxs ! k)

```

```

      (map snd f-fl) (map snd fl)
    by (rule verifier-query-round-chunk-length)
qed
have concat-eq: List.concat ?selected-chunks = List.concat query-chunks
  using selected-transcript transcript-query final-state-eq by simp
have chunks-eq: ?selected-chunks = query-chunks
proof -
  have same-lens:
     $\bigwedge k. k < \text{length } ?\text{selected-chunks} \implies$ 
     $\text{length } (?selected-chunks ! k) = \text{length } (query-chunks ! k)$ 
    using selected-chunk-lens query-chunk-lens len-selected-chunks by simp
  have concat-eq-empty:
    List.concat ?selected-chunks @ [] = List.concat query-chunks
    using concat-eq by simp
  have len-eq: length ?selected-chunks = length query-chunks
    using len-selected-chunks len-query-chunks by simp
  have ?selected-chunks = query-chunks  $\wedge$  ([] :: 'f list) = []
    by (rule concat-append-eq-concat-same-chunk-lengths
      [OF len-eq same-lens concat-eq-empty])
  then show ?thesis by simp
qed
have prefix-state-eq:
  state-after-query-chunks (PState query-state) prefix-chunks i =
  state-after-query-chunks (PState query-state) query-chunks i
proof -
  have
    state-after-query-chunks (PState query-state) ?selected-chunks i =
    state-after-query-chunks (PState query-state) prefix-chunks i
    by (rule state-after-query-chunks-append-prefix[OF len-prefix-chunks])
  then show ?thesis
    using chunks-eq by simp
qed
have selected-lookup-global:
  fmllookup (HashMap final-state)
    (QueryIndexChallenge (PQueryCounter query-state + i)
      (state-after-query-chunks (PState query-state) query-chunks i)) =
  Some raw
  using selected-lookup prefix-state-eq by simp
have raw-eq: raw = raw-idxs ! i
  using selected-lookup-global replay-lookup[OF round-bound]
  final-state-eq by simp
have idx-eq-query: idx = query-idxs ! i
  using idx-eq raw-eq query-idxs-eq len-raw round-bound by simp
have auth':
  fri-layer-chunk-authenticated
    (composition-roots ! (length composition-roots - 1))
    (fri-evidence-layer-len composition-roots (length composition-roots - 1))
    (fri-evidence-layer-idx composition-roots query-idxs i
      (length composition-roots - 1))

```



```

    actual-chunk final-state
using auth idx-eq-query composition-roots-eq fl-nonempty
unfolding fri-evidence-layer-len-def fri-evidence-layer-idx-def
by simp
have step':
  fri-layer-step-evidence
    (composition-roots ! (length composition-roots - 1))
    (composition-bs ! (length composition-roots - 1))
    (fri-evidence-layer-len composition-roots (length composition-roots - 1))
    (fri-evidence-layer-idx composition-roots query-idxs i
      (length composition-roots - 1))
    (2 ^ (length composition-roots - 1))
    (fri-sibling-index
      (fri-evidence-layer-len composition-roots (length composition-roots - 1))
      (fri-evidence-layer-idx composition-roots query-idxs i
        (length composition-roots - 1)))
    axp axp-path axn axn-path
    (fri-evidence-next-idx composition-roots query-idxs i
      (length composition-roots - 1))
    composition-final actual-chunk
using step idx-eq-query composition-roots-eq composition-bs-eq fl-nonempty
unfolding fri-evidence-layer-len-def fri-evidence-layer-idx-def
  fri-evidence-next-idx-def
by simp
show ?thesis
by (rule that[OF auth' step'])
qed

end

end

```

```

theory Soundness-FRI-Recorded-Chunk-Replay
imports Soundness-FRI-Raw-Layer-Bounds
begin

```

Selected recorded FRI layer replay.

The authenticated-route layer already proves that the verifier authenticates some chunk at the replayed root/index of a selected FRI layer. The residual proofs need the stronger fact that the specific chunk recorded in the extracted verifier transcript is authenticated. This theory proves that narrow replay fact for one selected layer, avoiding the earlier broad all-layer replay induction.

```

context soundness
begin

```

```

lemma fri-layer-lengths-nth-pos-imp-initial-pos:
  assumes j-bound:  $j < n$ 

```

```

    and pos: 0 < fri-layer-lengths n len ! j
  shows 0 < len
  using assms
proof (induction j arbitrary: n len)
  case 0
  then show ?case
    by (cases n) simp-all
next
  case (Suc j)
  then obtain n' where n-eq: n = Suc n'
    by (cases n) auto
  have j-bound': j < n'
    using Suc.prem1(1) unfolding n-eq by simp
  have pos': 0 < fri-layer-lengths n' (len div 2) ! j
    using Suc.prem1(2) unfolding n-eq by simp
  have 0 < len div 2
    by (rule Suc.IH[OF j-bound' pos'])
  then show ?case
    by simp
qed

lemma mfold-fri-layer-opening-selected-recorded-chunk-authenticated-at:
  fixes s t :: ('f, 'a) protocol-channel-scheme
  assumes outcome:
    Some (out, t) ∈
      set-dist
        (execute
          (mfold (idx, x, len, pw)
            (map fri-layer-opening-step bfs)) s)
  and layers:
    fri-layers-transcript (length bfs) len layer-chunks total-chunk
  and transcript:
    PTranscript s = total-chunk @ PTranscript t
  and initial-idx-bound: idx < len
  and all-layer-bounds:
     $\bigwedge k. k < \text{length } \text{bfs} \implies$ 
    0 < fri-layer-lengths (length bfs) len ! k  $\wedge$ 
    fri-layer-indices (length bfs) idx len ! k <
    fri-layer-lengths (length bfs) len ! k
  and j-bound: j < length bfs
  shows
    fri-layer-chunk-authenticated (snd (bfs ! j))
    (fri-layer-lengths (length bfs) len ! j)
    (fri-layer-indices (length bfs) idx len ! j)
    (layer-chunks ! j) t
  using outcome layers transcript initial-idx-bound all-layer-bounds j-bound
proof (induction bfs arbitrary: idx x len pw s out t j layer-chunks total-chunk)
  case Nil
  then show ?case by simp

```

```

next
case (Cons bf bfs)
obtain b rt where bf-eq: bf = (b, rt)
  by (cases bf)
from Cons.premis(1)[unfolded bf-eq]
obtain out1 s1 where
  head:
    Some (out1, s1) ∈
      set-dist
        (execute (fri-layer-opening-step (b, rt) (idx, x, len, pw)) s)
  and tail:
    Some (out, t) ∈
      set-dist
        (execute (mfold out1 (map fri-layer-opening-step bfs)) s1)
  by (auto elim!: set-dist-bindE)
obtain first rest where chunks-eq: layer-chunks = first # rest
  using Cons.premis(6) Cons.premis(2)
  unfolding fri-layers-transcript-def by (cases layer-chunks) auto
have total-eq: total-chunk = first @ List.concat rest
  using Cons.premis(2) unfolding fri-layers-transcript-def chunks-eq
  by simp
have first-open:
  ∃ xp xp-path xn xn-path.
    fri-layer-opening-chunk len xp xp-path xn xn-path first
  using Cons.premis(2) unfolding fri-layers-transcript-def chunks-eq
  by auto
have rest-layers:
  fri-layers-transcript (length bfs) (len div 2) rest
  (List.concat rest)
  using Cons.premis(2) unfolding fri-layers-transcript-def chunks-eq
  by auto
obtain x' xp xp-path xn xn-path head-chunk where
  out1-eq:
    out1 = (idx mod (len div 2), x', len div 2, pw + pw)
  and head-chunk-def: head-chunk = [xp] @ xp-path @ [xn] @ xn-path
  and head-open:
    fri-layer-opening-chunk len xp xp-path xn xn-path head-chunk
  and tr-head: PTranscript s = head-chunk @ PTranscript s1
  and xp-auth:
    authenticated-opening-in s1
    (|opening-root = rt,
      opening-length = len,
      opening-index = idx,
      opening-value = xp,
      opening-path = xp-path|)
  and xn-auth:
    authenticated-opening-in s1
    (|opening-root = rt,
      opening-length = len,

```

```

    opening-index = fri-sibling-index len idx,
    opening-value = xn,
    opening-path = xn-path)
proof –
  have len-pos:  $0 < \text{len}$ 
    using Cons.prem5(5)[of 0] by simp
  have idx-bound:  $\text{idx} < \text{len}$ 
    by (rule Cons.prem4(4))
  show ?thesis
proof (rule fri-layer-opening-step-outcome-authenticated
  [OF len-pos idx-bound head])
  fix xp' xp-path' xn' xn-path' x'' chunk
  assume out-eq:
    out1 = (idx mod (len div 2), x'', len div 2, pw + pw)
  and chunk-eq: chunk = [xp'] @ xp-path' @ [xn'] @ xn-path'
  and chunk-open:
    fri-layer-opening-chunk len xp' xp-path' xn' xn-path' chunk
  and tr: PTranscript s = chunk @ PTranscript s1
  and ext:  $s \leq s1$ 
  and auth-xp:
    authenticated-opening-in s1
    (opening-root = rt, opening-length = len,
     opening-index = idx, opening-value = xp',
     opening-path = xp-path')
  and auth-xn:
    authenticated-opening-in s1
    (opening-root = rt, opening-length = len,
     opening-index = fri-sibling-index len idx,
     opening-value = xn', opening-path = xn-path')
  show ?thesis
    by (rule that[OF out-eq chunk-eq chunk-open tr auth-xp auth-xn])
qed
qed
from mfold-fri-layer-openings-outcome[OF tail[unfolded out1-eq]]
obtain tail-layers tail-chunk where
  tail-transcript:
    PTranscript s1 = tail-chunk @ PTranscript t
  and tail-ext:  $s1 \leq t$ 
  by blast
have head-len: length head-chunk = length first
proof –
  obtain yp yp-path yn yn-path where first-open':
    fri-layer-opening-chunk len yp yp-path yn yn-path first
  using first-open by blast
  show ?thesis
    using fri-layer-opening-chunk-length[OF head-open]
    fri-layer-opening-chunk-length[OF first-open']
    by simp
qed

```

```

have chunks-total-eq:
  head-chunk @ tail-chunk = first @ List.concat rest
  using tr-head tail-transcript Cons.premis(3) total-eq by simp
have head-eq: head-chunk = first
  using arg-cong[OF chunks-total-eq, of take (length head-chunk)]
  head-len
  by simp
have tail-transcript-rest:
  PTranscript s1 = List.concat rest @ PTranscript t
  using chunks-total-eq head-eq tail-transcript by simp
show ?case
proof (cases j)
  case 0
  have auth-s1:
    fri-layer-chunk-authenticated rt len idx head-chunk s1
    by (rule fri-layer-chunk-authenticatedI[OF head-open xp-auth xn-auth])
  have auth-t:
    fri-layer-chunk-authenticated rt len idx head-chunk t
    by (rule fri-layer-chunk-authenticated-mono[OF auth-s1 tail-ext])
  show ?thesis
    using auth-t head-eq 0 chunks-eq bf-eq by simp
next
  case (Suc k)
  have k-bound: k < length bfs
    using Cons.premis(6) Suc by simp
  have tail-all-layer-bounds:
     $\bigwedge m. m < \text{length bfs} \implies$ 
    0 < fri-layer-lengths (length bfs) (len div 2) ! m  $\wedge$ 
    fri-layer-indices (length bfs) (idx mod (len div 2))
    (len div 2) ! m <
    fri-layer-lengths (length bfs) (len div 2) ! m
    using Cons.premis(5)[of Suc m for m] by simp
  have tail-initial-idx-bound: idx mod (len div 2) < len div 2
    using tail-all-layer-bounds[of 0] k-bound by (cases bfs) simp-all
  have tail-auth:
    fri-layer-chunk-authenticated (snd (bfs ! k))
    (fri-layer-lengths (length bfs) (len div 2) ! k)
    (fri-layer-indices (length bfs) (idx mod (len div 2))
    (len div 2) ! k)
    (rest ! k) t
    by (rule Cons.IH[OF tail[unfolded out1-eq] rest-layers
    tail-transcript-rest tail-initial-idx-bound tail-all-layer-bounds
    k-bound])
  show ?thesis
    using tail-auth Suc chunks-eq by simp
qed
qed

```

lemma receive-query-commits-selected-recorded-chunk-authenticated-at:

fixes $s\ t :: ('f, 'a)\ \text{protocol-channel-scheme}$
assumes *outcome*:
 $\text{Some } (out, t) \in$
 set-dist
 $(\text{execute}$
 $\quad (\text{mfold } (idx, x, len, pw) (\text{receive-query-commits } bfs))\ s)$
and *layers*:
 $\text{fri-layers-transcript } (length\ bfs)\ len\ \text{layer-chunks}\ \text{total-chunk}$
and *transcript*:
 $P\text{Transcript } s = \text{total-chunk} @ P\text{Transcript } t$
and *initial-idx-bound*: $idx < len$
and *all-layer-bounds*:
 $\bigwedge k. k < length\ bfs \implies$
 $0 < \text{fri-layer-lengths } (length\ bfs)\ len ! k \wedge$
 $\text{fri-layer-indices } (length\ bfs)\ idx\ len ! k <$
 $\text{fri-layer-lengths } (length\ bfs)\ len ! k$
and *j-bound*: $j < length\ bfs$
shows
 $\text{fri-layer-chunk-authenticated } (snd\ (bfs ! j))$
 $(\text{fri-layer-lengths } (length\ bfs)\ len ! j)$
 $(\text{fri-layer-indices } (length\ bfs)\ idx\ len ! j)$
 $(\text{layer-chunks } ! j)\ t$
proof –
have *map-eq*: $\text{receive-query-commits } bfs = \text{map } \text{fri-layer-opening-step } bfs$
unfolding $\text{receive-query-commits-def } \text{fri-layer-opening-step-def}$
by $(\text{simp add: fun-eq-iff split: prod.splits})$
show *?thesis*
by $(\text{rule } \text{mfold-fri-layer-opening-selected-recorded-chunk-authenticated-at}$
 $\quad [OF\ \text{outcome}[\text{unfolded } \text{map-eq}]\ \text{layers}\ \text{transcript}\ \text{initial-idx-bound}$
 $\quad \text{all-layer-bounds } j\text{-bound}])$
qed

lemma *verifier-query-round-after-index-trace-recorded-layer-chunk-authenticated*:

fixes $s\ t :: ('f, 'a)\ \text{protocol-channel-scheme}$
assumes *outcome*:
 $\text{Some } ((), t) \in$
 set-dist
 $(\text{execute}$
 $\quad (\text{verifier-query-round-after-index-program}$
 $\quad \text{fr } f\text{-fl } f\text{-final as fl final raw})\ s)$
and *recorded*:
 $\text{query-round-fri-layer-transcripts } (index\ (to\text{-nat } raw))$
 $(\text{map } snd\ f\text{-fl})\ (\text{map } snd\ fl)\ \text{round-chunk}\ \text{trace-layers}$
 $\text{composition-layers}$
and *transcript*: $P\text{Transcript } s = \text{round-chunk} @ P\text{Transcript } t$
and *j-bound*: $j < length\ f\text{-fl}$
and *all-layer-bounds*:
 $\bigwedge k. k < length\ f\text{-fl} \implies$
 $0 < \text{fri-layer-lengths } (length\ f\text{-fl})\ (clength * scale) ! k \wedge$

```

    fri-layer-indices (length f-fl) (index (to-nat raw))
      (clength * scale) ! k <
    fri-layer-lengths (length f-fl) (clength * scale) ! k
  shows
    fri-layer-chunk-authenticated (snd (f-fl ! j))
      (fri-layer-lengths (length f-fl) (clength * scale) ! j)
      (fri-layer-indices (length f-fl) (index (to-nat raw))
        (clength * scale) ! j)
      (trace-layers ! j) t
  proof -
  let ?idx = index (to-nat raw)
  from outcome obtain fv s1 f-out s2 s3 c-out s4 where
    query-decommit:
      Some (fv, s1) ∈
        set-dist (execute (mmap (check-decommit-on-query fr ?idx)) s)
  and trace-fri:
    Some (f-out, s2) ∈
      set-dist (execute
        (mfold (?idx, hd fv, clength * scale, 1)
          (receive-query-commits f-fl)) s1)
  and assert-trace:
    Some ((), s3) ∈
      set-dist
        (execute
          (assert (case f-out of (-, f-x, -, -) ⇒ f-x = f-final)) s2)
  and comp-fri:
    Some (c-out, s4) ∈
      set-dist (execute
        (mfold
          (?idx, cp-eval as fv (h ^ ?idx * shift), clength * scale, 1)
          (receive-query-commits fl)) s3)
  and assert-comp:
    Some ((), t) ∈
      set-dist
        (execute
          (assert (case c-out of (-, x, -, -) ⇒ x = final)) s4)
  unfolding verifier-query-round-after-index-program-def
  by (auto simp: Let-def split: prod.splits elim!: set-dist-bindE)
  from check-decommit-on-query-outcome[OF query-decommit]
  obtain query-paths query-chunk where
    query-chunk-shape:
      query-decommitment-transcript ?idx fv query-paths query-chunk
  and tr-s: PTranscript s = query-chunk @ PTranscript s1
  by blast
  from receive-query-commits-layers-outcome[OF trace-fri]
  obtain trace-layer-chunks trace-fri-chunk where
    trace-layers-actual:
      fri-layers-transcript (length f-fl) (clength * scale)
      trace-layer-chunks trace-fri-chunk

```

```

and tr-s1: PTranscript s1 = trace-fri-chunk @ PTranscript s2
and ext-s1-s2: s1 ≤ s2
by blast
have s3-eq: s3 = s2
  using assert-trace unfolding assert-def
  by (cases case f-out of (-, f-x, -, -) ⇒ f-x = f-final)
    (auto simp: throw-no-outcome)
from receive-query-commits-layers-outcome[OF comp-fri]
obtain comp-layer-chunks comp-fri-chunk where
  comp-layers-actual:
    fri-layers-transcript (length fl) (clength * scale)
    comp-layer-chunks comp-fri-chunk
and tr-s3: PTranscript s3 = comp-fri-chunk @ PTranscript s4
and ext-s3-s4: s3 ≤ s4
by blast
have t-eq: t = s4
  using assert-comp unfolding assert-def
  by (cases case c-out of (-, x, -, -) ⇒ x = final)
    (auto simp: throw-no-outcome)
let ?actual-chunk = query-chunk @ trace-fri-chunk @ comp-fri-chunk
have actual-recorded:
  query-round-fri-layer-transcripts ?idx (map snd f-fl) (map snd fl)
  ?actual-chunk trace-layer-chunks comp-layer-chunks
unfolding query-round-fri-layer-transcripts-def
by (intro exI[of - query-chunk] exI[of - trace-fri-chunk]
  exI[of - comp-fri-chunk] exI[of - fv] exI[of - query-paths] conjI)
  (use query-chunk-shape trace-layers-actual comp-layers-actual in
  ⟨simp-all add: mult commute⟩)
have actual-transcript:
  PTranscript s = ?actual-chunk @ PTranscript t
  using tr-s tr-s1 tr-s3 s3-eq t-eq by simp
have actual-chunk-eq: ?actual-chunk = round-chunk
  using actual-transcript transcript by simp
have trace-layer-chunks-eq:
  trace-layer-chunks = trace-layers
  by (rule query-round-fri-layer-transcripts-unique(1)
  [OF actual-recorded[unfolded actual-chunk-eq] recorded])
have idx-bound: ?idx < clength * scale
  by (rule index-less-domain)
have auth-s2:
  fri-layer-chunk-authenticated (snd (f-fl ! j))
  (fri-layer-lengths (length f-fl) (clength * scale) ! j)
  (fri-layer-indices (length f-fl) ?idx (clength * scale) ! j)
  (trace-layer-chunks ! j) s2
  by (rule receive-query-commits-selected-recorded-chunk-authenticated-at
  [OF trace-fri trace-layers-actual tr-s1 idx-bound
  all-layer-bounds j-bound])
have s2-t: s2 ≤ t
  using ext-s3-s4 s3-eq t-eq by simp

```



```

have auth-t:
  fri-layer-chunk-authenticated (snd (f-fl ! j))
    (fri-layer-lengths (length f-fl) (clength * scale) ! j)
    (fri-layer-indices (length f-fl) ?idx (clength * scale) ! j)
    (trace-layer-chunks ! j) t
  by (rule fri-layer-chunk-authenticated-mono[OF auth-s2 s2-t])
show ?thesis
  using auth-t trace-layer-chunks-eq by simp
qed

lemma verifier-query-round-after-index-composition-recorded-layer-chunk-authenticated:
fixes s t :: ('f, 'a) protocol-channel-scheme
assumes outcome:
  Some ((), t) ∈
    set-dist
      (execute
        (verifier-query-round-after-index-program
          fr f-fl f-final as fl final raw) s)
and recorded:
  query-round-fri-layer-transcripts (index (to-nat raw))
    (map snd f-fl) (map snd fl) round-chunk trace-layers
    composition-layers
and transcript: PTranscript s = round-chunk @ PTranscript t
and j-bound: j < length fl
and all-layer-bounds:
   $\bigwedge k. k < \text{length } fl \implies$ 
     $0 < \text{fri-layer-lengths } (\text{length } fl) (\text{clength} * \text{scale}) ! k \wedge$ 
     $\text{fri-layer-indices } (\text{length } fl) (\text{index } (\text{to-nat } \text{raw}))$ 
     $(\text{clength} * \text{scale}) ! k <$ 
     $\text{fri-layer-lengths } (\text{length } fl) (\text{clength} * \text{scale}) ! k$ 
shows
  fri-layer-chunk-authenticated (snd (fl ! j))
    (fri-layer-lengths (length fl) (clength * scale) ! j)
    (fri-layer-indices (length fl) (index (to-nat raw)))
    (clength * scale) ! j)
    (composition-layers ! j) t
proof –
  let ?idx = index (to-nat raw)
from outcome obtain fv s1 f-out s2 s3 c-out s4 where
    query-decommit:
      Some (fv, s1) ∈
        set-dist (execute (mmap (check-decommit-on-query fr ?idx)) s)
and trace-fri:
      Some (f-out, s2) ∈
        set-dist (execute
          (mfold (?idx, hd fv, clength * scale, 1)
            (receive-query-commits f-fl)) s1)
and assert-trace:
      Some ((), s3) ∈

```

```

    set-dist
      (execute
        (assert (case f-out of (-, f-x, -, -)  $\Rightarrow$  f-x = f-final)) s2)
  and comp-fri:
    Some (c-out, s4)  $\in$ 
      set-dist (execute
        (mfold
          (?idx, cp-eval as fv (h  $\wedge$  ?idx * shift), clength * scale, 1)
          (receive-query-commits fl)) s3)
  and assert-comp:
    Some (( ), t)  $\in$ 
      set-dist
        (execute
          (assert (case c-out of (-, x, -, -)  $\Rightarrow$  x = final)) s4)
  unfolding verifier-query-round-after-index-program-def
  by (auto simp: Let-def split: prod.splits elim!: set-dist-bindE)
from check-decommit-on-query-outcome[OF query-decommit]
obtain query-paths query-chunk where
  query-chunk-shape:
    query-decommitment-transcript ?idx fv query-paths query-chunk
  and tr-s: PTranscript s = query-chunk @ PTranscript s1
  by blast
from receive-query-commits-layers-outcome[OF trace-fri]
obtain trace-layer-chunks trace-fri-chunk where
  trace-layers-actual:
    fri-layers-transcript (length f-fl) (clength * scale)
    trace-layer-chunks trace-fri-chunk
  and tr-s1: PTranscript s1 = trace-fri-chunk @ PTranscript s2
  by blast
have s3-eq: s3 = s2
  using assert-trace unfolding assert-def
  by (cases case f-out of (-, f-x, -, -)  $\Rightarrow$  f-x = f-final)
    (auto simp: throw-no-outcome)
from receive-query-commits-layers-outcome[OF comp-fri]
obtain comp-layer-chunks comp-fri-chunk where
  comp-layers-actual:
    fri-layers-transcript (length fl) (clength * scale)
    comp-layer-chunks comp-fri-chunk
  and tr-s3: PTranscript s3 = comp-fri-chunk @ PTranscript s4
  by blast
have t-eq: t = s4
  using assert-comp unfolding assert-def
  by (cases case c-out of (-, x, -, -)  $\Rightarrow$  x = final)
    (auto simp: throw-no-outcome)
let ?actual-chunk = query-chunk @ trace-fri-chunk @ comp-fri-chunk
have actual-recorded:
  query-round-fri-layer-transcripts ?idx (map snd f-fl) (map snd fl)
  ?actual-chunk trace-layer-chunks comp-layer-chunks
  unfolding query-round-fri-layer-transcripts-def

```

```

  by (intro exI[of - query-chunk] exI[of - trace-fri-chunk]
      exI[of - comp-fri-chunk] exI[of - fv] exI[of - query-paths] conjI)
    (use query-chunk-shape trace-layers-actual comp-layers-actual in
      ⟨simp-all add: mult.commute⟩)
have actual-transcript:
  PTranscript s = ?actual-chunk @ PTranscript t
  using tr-s tr-s1 tr-s3 s3-eq t-eq by simp
have actual-chunk-eq: ?actual-chunk = round-chunk
  using actual-transcript transcript by simp
have comp-layer-chunks-eq:
  comp-layer-chunks = composition-layers
  by (rule query-round-fri-layer-transcripts-unique(2)
      [OF actual-recorded[unfolded actual-chunk-eq] recorded])
have idx-bound: ?idx < clength * scale
  by (rule index-less-domain)
have auth-s4:
  fri-layer-chunk-authenticated (snd (fl ! j))
    (fri-layer-lengths (length fl) (clength * scale) ! j)
    (fri-layer-indices (length fl) ?idx (clength * scale) ! j)
    (comp-layer-chunks ! j) s4
  by (rule receive-query-commits-selected-recorded-chunk-authenticated-at
      [OF comp-fri comp-layers-actual tr-s3 idx-bound
        all-layer-bounds j-bound])
show ?thesis
  using auth-s4 comp-layer-chunks-eq t-eq by simp
qed

lemma accepted-fri-opening-transcript-trace-recorded-layer-chunk-authenticated-at:
  assumes fri-openings:
    accepted-fri-opening-transcript s out trace-roots trace-bs trace-final
    dg composition-roots composition-bs composition-final query-idxs
    trace-round-layers composition-round-layers
  and out-eq: out = Some (result, final-state)
  and round-bound: i < rounds
  and layer-bound: j < length trace-roots
  and all-layer-bounds:
     $\bigwedge \text{raw } k. k < \text{length trace-roots} \implies$ 
     $0 < \text{fri-layer-lengths } (\text{length trace-roots}) (\text{clength} * \text{scale}) ! k \wedge$ 
     $\text{fri-layer-indices } (\text{length trace-roots}) (\text{index } (\text{to-nat raw}))$ 
     $(\text{clength} * \text{scale}) ! k <$ 
     $\text{fri-layer-lengths } (\text{length trace-roots}) (\text{clength} * \text{scale}) ! k$ 
  shows
    generic-fri-recorded-layer-chunk-authenticated trace-roots query-idxs
    trace-round-layers final-state i j
proof -
  from fri-openings obtain result' final-state' fr f-fl as fl query-state
    raw-idxs query-chunks where
    out-def: out = Some (result', final-state')
  and trace-roots-eq: trace-roots = map snd f-fl

```

```

and trace-bs-eq: trace-bs = map fst f-fl
and composition-roots-eq: composition-roots = map snd fl
and outcome:
  Some (result', final-state') ∈
    set-dist (execute
      (ntimes
        (verifier-query-round-program fr f-fl trace-final as fl
          composition-final)
        rounds)
      query-state)
and len-raw: length raw-idxs = rounds
and query-idxs-eq:
  query-idxs = map (λraw. index (to-nat raw)) raw-idxs
and len-query-chunks: length query-chunks = rounds
and transcript-query:
  PTranscript query-state =
    List.concat query-chunks @ PTranscript final-state'
and query-chunk:
  ∧i. i < rounds ⇒
    verifier-query-round-chunk (query-idxs ! i)
    trace-roots composition-roots (query-chunks ! i)
and replay-lookup:
  ∧i. i < rounds ⇒
    fmlookup (HashMap final-state')
      (QueryIndexChallenge (PQueryCounter query-state + i)
        (state-after-query-chunks (PState query-state) query-chunks i)) =
      Some (raw-idxs ! i)
and recorded:
  ∧i. i < rounds ⇒
    query-round-fri-layer-transcripts (query-idxs ! i)
    trace-roots composition-roots (query-chunks ! i)
    (trace-round-layers ! i) (composition-round-layers ! i)
unfolding accepted-fri-opening-transcript-def by blast
have result-eq: result' = result
using out-def out-eq by simp
have final-state-eq: final-state' = final-state
using out-def out-eq by simp
have outcome':
  Some (result, final-state) ∈
    set-dist (execute
      (ntimes
        (verifier-query-round-program fr f-fl trace-final as fl
          composition-final)
        rounds)
      query-state)
using outcome result-eq final-state-eq by simp
let ?round =
  verifier-query-round-program fr f-fl trace-final as fl composition-final
from query-opening-ntimes-outcome-decomp-at[OF outcome' round-bound]

```

obtain *prefix* *x* *suffix* *s-i* *s-suc* **where**
prefix:
Some (*prefix*, *s-i*) $\in$
set-dist (*execute* (*ntimes* ?*round* *i*) *query-state*)
and *round*:
Some (*x*, *s-suc*) $\in$ *set-dist* (*execute* ?*round* *s-i*)
and *suffix*:
Some (*suffix*, *final-state*) $\in$
set-dist (*execute* (*ntimes* ?*round* (*rounds* - *Suc* *i*)) *s-suc*)
by *blast*
have *x-eq*: *x* = ()
by (*cases* *x*) *simp*
show ?*thesis*
proof (*rule* *query-opening-ntimes-prefix-transcript-state-alignment*
[*OF* *prefix*])
fix *prefix-query-idxs* *prefix-chunks*
assume *len-prefix-idxs*: *length* *prefix-query-idxs* = *i*
and *len-prefix-chunks*: *length* *prefix-chunks* = *i*
and *transcript-prefix*:
PTranscript *query-state* =
List.concat *prefix-chunks* @ *PTranscript* *s-i*
and *state-i*:
PState *s-i* =
state-after-query-chunks (*PState* *query-state*) *prefix-chunks* *i*
and *counter-i*: *PQueryCounter* *s-i* = *PQueryCounter* *query-state* + *i*
and *prefix-ext*: *query-state* $\leq$ *s-i*
and *prefix-rounds*:
 $\bigwedge k. k < i \implies$
verifier-query-round-chunk (*prefix-query-idxs* ! *k*)
(*map* *snd* *f-fl*) (*map* *snd* *fl*) (*prefix-chunks* ! *k*)
from *round*[*unfolded* *x-eq* *verifier-query-round-program-alt-def*]
obtain *raw* *s0* **where**
raw-out:
Some (*raw*, *s0*) $\in$
set-dist (*execute* *receive-query-index-challenge* *s-i*)
and *after*:
Some ((), *s-suc*) $\in$
set-dist (*execute*
(*verifier-query-round-after-index-program* *fr* *f-fl* *trace-final* *as*
fl *composition-final* *raw*) *s0*)
by (*auto* *elim!*: *set-dist-bindE*)
have *tr-s0*: *PTranscript* *s0* = *PTranscript* *s-i*
using *receive-query-index-challenge-outcome*[*OF* *raw-out*] **by** *simp*
show ?*thesis*
proof (*rule* *verifier-query-round-program-outcome*[*OF* *round*[*unfolded* *x-eq*]])
fix *raw'* *idx* *chunk*
assume *idx-eq*: *idx* = *index* (*to-nat* *raw'*)
and *chunk-shape*:
verifier-query-round-chunk *idx* (*map* *snd* *f-fl*) (*map* *snd* *fl*)

```

    chunk
and transcript-round:
    PTranscript s-i = chunk @ PTranscript s-suc
and state-suc:
    PState s-suc = foldl concat (PState s-i) chunk
and round-ext: s-i ≤ s-suc
and counter-suc:
    PQueryCounter s-suc = Suc (PQueryCounter s-i)
and lookup-suc:
    fmlookup (HashMap s-suc)
    (QueryIndexChallenge (PQueryCounter s-i) (PState s-i)) =
    Some raw'
have raw-eq': raw = raw'
proof -
    have lookup-s0:
    fmlookup (HashMap s0)
    (QueryIndexChallenge (PQueryCounter s-i) (PState s-i)) =
    Some raw
    using receive-query-index-challenge-outcome[OF raw-out] by simp
have s0-suc: s0 ≤ s-suc
    using verifier-query-round-after-index-program-outcome[OF after]
    by simp
have lookup-suc-raw:
    fmlookup (HashMap s-suc)
    (QueryIndexChallenge (PQueryCounter s-i) (PState s-i)) =
    Some raw
    using hash-extension-lookup[OF lookup-s0 s0-suc] .
show ?thesis
    using lookup-suc lookup-suc-raw by simp
qed
show ?thesis
proof (rule query-opening-ntimes-suffix-transcript-state-alignment
    [OF suffix])
    fix suffix-query-idxs suffix-chunks
    assume len-suffix-idxs:
    length suffix-query-idxs = rounds - Suc i
    and len-suffix-chunks:
    length suffix-chunks = rounds - Suc i
    and transcript-suffix:
    PTranscript s-suc =
    List.concat suffix-chunks @ PTranscript final-state
    and suffix-state:
    PState final-state =
    state-after-query-chunks (PState s-suc) suffix-chunks
    (rounds - Suc i)
    and suffix-counter:
    PQueryCounter final-state =
    PQueryCounter s-suc + (rounds - Suc i)
    and suffix-ext: s-suc ≤ final-state

```

```

and suffix-rounds:
   $\bigwedge k. k < \text{rounds} - \text{Suc } i \implies$ 
    verifier-query-round-chunk (suffix-query-idxs ! k)
      (map snd f-fl) (map snd fl) (suffix-chunks ! k)
let ?selected-chunks = prefix-chunks @ chunk # suffix-chunks
have len-selected-chunks: length ?selected-chunks = rounds
  using len-prefix-chunks len-suffix-chunks round-bound by simp
have selected-chunk-lens:
   $\bigwedge k. k < \text{rounds} \implies$ 
    length (?selected-chunks ! k) =
      verifier-query-round-transcript-length (query-idxs ! k)
        (map snd f-fl) (map snd fl)
proof -
  fix k
  assume k-bound: k < rounds
  show
    length (?selected-chunks ! k) =
      verifier-query-round-transcript-length (query-idxs ! k)
        (map snd f-fl) (map snd fl)
  proof (cases k < i)
    case True
    have nth-eq: ?selected-chunks ! k = prefix-chunks ! k
      using True len-prefix-chunks by (simp add: nth-append)
    have
      length (prefix-chunks ! k) =
        verifier-query-round-transcript-length
          (prefix-query-idxs ! k) (map snd f-fl) (map snd fl)
      by (rule verifier-query-round-chunk-length
        [OF prefix-rounds[OF True]])
    then show ?thesis
      using nth-eq
      by (simp add:
        verifier-query-round-transcript-length-index-irrelevant)
  next
  case False
  show ?thesis
  proof (cases k = i)
    case True
    have nth-eq: ?selected-chunks ! k = chunk
      using True len-prefix-chunks by (simp add: nth-append)
    have
      length chunk =
        verifier-query-round-transcript-length idx
          (map snd f-fl) (map snd fl)
      by (rule verifier-query-round-chunk-length[OF chunk-shape])
    then show ?thesis
      using nth-eq
      by (simp add:
        verifier-query-round-transcript-length-index-irrelevant)

```

```

next
  case False
  let ?k = k - Suc i
  have ge: Suc i ≤ k
    using ⟨¬ k < i⟩ False by linarith
  have k-bound': ?k < rounds - Suc i
    using ge k-bound by linarith
  have nth-eq: ?selected-chunks ! k = suffix-chunks ! ?k
    using ge len-prefix-chunks by (simp add: nth-append)
  have
    length (suffix-chunks ! ?k) =
      verifier-query-round-transcript-length
        (suffix-query-idxs ! ?k) (map snd f-fl) (map snd fl)
    by (rule verifier-query-round-chunk-length
        [OF suffix-rounds[OF k-bound']])
  then show ?thesis
    using nth-eq
    by (simp add:
        verifier-query-round-transcript-length-index-irrelevant)
qed
qed
qed
have query-chunk-f:
  ∧k. k < rounds ⇒
    verifier-query-round-chunk (query-idxs ! k)
      (map snd f-fl) (map snd fl) (query-chunks ! k)
  using query-chunk trace-roots-eq composition-roots-eq by simp
have query-chunk-lens:
  ∧k. k < rounds ⇒
    length (query-chunks ! k) =
      verifier-query-round-transcript-length (query-idxs ! k)
        (map snd f-fl) (map snd fl)
  using query-chunk-f verifier-query-round-chunk-length by blast
have selected-transcript:
  PTranscript query-state =
    List.concat ?selected-chunks @ PTranscript final-state
  using transcript-prefix transcript-round transcript-suffix by simp
have concat-eq: List.concat ?selected-chunks = List.concat query-chunks
  using selected-transcript transcript-query final-state-eq by simp
have chunks-eq: ?selected-chunks = query-chunks
proof -
  have same-lens:
    ∧k. k < length ?selected-chunks ⇒
      length (?selected-chunks ! k) = length (query-chunks ! k)
    using selected-chunk-lens query-chunk-lens len-selected-chunks
    by simp
  have concat-eq-empty:
    List.concat ?selected-chunks @ [] = List.concat query-chunks
  using concat-eq by simp

```



```

have len-eq: length ?selected-chunks = length query-chunks
  using len-selected-chunks len-query-chunks by simp
have ?selected-chunks = query-chunks  $\wedge$  ( $[] :: 'f \text{ list}$ ) =  $[]$ 
  by (rule concat-append-eq-concat-same-chunk-lengths
    [OF len-eq same-lens concat-eq-empty])
then show ?thesis
  by simp
qed
have prefix-state-eq:
  state-after-query-chunks (PState query-state) prefix-chunks i =
  state-after-query-chunks (PState query-state) query-chunks i
proof -
  have
    state-after-query-chunks (PState query-state) ?selected-chunks i =
    state-after-query-chunks (PState query-state) prefix-chunks i
  by (rule state-after-query-chunks-append-prefix
    [OF len-prefix-chunks])
  then show ?thesis
    using chunks-eq by simp
qed
have lookup-final:
  fmlookup (HashMap final-state)
    (QueryIndexChallenge (PQueryCounter query-state + i)
      (state-after-query-chunks (PState query-state) query-chunks i)) =
  Some raw'
proof -
  have lookup-suc':
    fmlookup (HashMap s-suc)
      (QueryIndexChallenge (PQueryCounter query-state + i)
        (state-after-query-chunks (PState query-state)
          prefix-chunks i)) =
    Some raw'
  using lookup-suc state-i counter-i by simp
have lookup-final-prefix:
  fmlookup (HashMap final-state)
    (QueryIndexChallenge (PQueryCounter query-state + i)
      (state-after-query-chunks (PState query-state)
        prefix-chunks i)) =
  Some raw'
  using hash-extension-lookup[OF lookup-suc' suffix-ext] .
show ?thesis
  using lookup-final-prefix prefix-state-eq by simp
qed
have raw-idx-eq: raw' = raw-idxs ! i
  using lookup-final replay-lookup[OF round-bound] final-state-eq
  by simp
have idx-eq-query: idx = query-idxs ! i
  using idx-eq raw-idx-eq query-idxs-eq len-raw round-bound by simp
have selected-i: ?selected-chunks ! i = chunk

```

```

using len-prefix-chunks by (simp add: nth-append)
have chunk-i-eq: chunk = query-chunks ! i
using chunks-eq selected-i by simp
have recorded-i:
  query-round-fri-layer-transcripts (index (to-nat raw))
    (map snd f-fl) (map snd fl) chunk (trace-round-layers ! i)
    (composition-round-layers ! i)
using recorded[OF round-bound] idx-eq idx-eq-query raw-eq'
  chunk-i-eq trace-roots-eq composition-roots-eq
by simp
have tr-s0-suc:
  PTranscript s0 = chunk @ PTranscript s-suc
using tr-s0 transcript-round by simp
have j-bound-f: j < length f-fl
using layer-bound trace-roots-eq by simp
have all-layer-bounds-f:
   $\bigwedge k. k < \text{length } f\text{-fl} \implies$ 
     $0 < \text{fri-layer-lengths } (\text{length } f\text{-fl}) (\text{clength} * \text{scale}) ! k \wedge$ 
     $\text{fri-layer-indices } (\text{length } f\text{-fl}) (\text{index } (\text{to-nat raw}))$ 
     $(\text{clength} * \text{scale}) ! k <$ 
     $\text{fri-layer-lengths } (\text{length } f\text{-fl}) (\text{clength} * \text{scale}) ! k$ 
using all-layer-bounds trace-roots-eq by simp
have auth-suc:
  fri-layer-chunk-authenticated (snd (f-fl ! j))
    (fri-layer-lengths (length f-fl) (clength * scale) ! j)
    (fri-layer-indices (length f-fl) (index (to-nat raw)))
    (clength * scale) ! j)
    (trace-round-layers ! i ! j) s-suc
by (rule
  verifier-query-round-after-index-trace-recorded-layer-chunk-authenticated
    [OF after recorded-i tr-s0-suc j-bound-f
      all-layer-bounds-f])
have auth-final:
  fri-layer-chunk-authenticated (snd (f-fl ! j))
    (fri-layer-lengths (length f-fl) (clength * scale) ! j)
    (fri-layer-indices (length f-fl) (index (to-nat raw)))
    (clength * scale) ! j)
    (trace-round-layers ! i ! j) final-state
by (rule fri-layer-chunk-authenticated-mono[OF auth-suc suffix-ext])
have raw-idx-query: index (to-nat raw) = query-idxs ! i
using idx-eq idx-eq-query raw-eq' by simp
have auth:
  fri-layer-chunk-authenticated (trace-roots ! j)
    (fri-evidence-layer-len trace-roots j)
    (fri-evidence-layer-idx trace-roots query-idxs i j)
    (trace-round-layers ! i ! j) final-state
using auth-final trace-roots-eq raw-idx-query layer-bound
unfolding fri-evidence-layer-len-def fri-evidence-layer-idx-def
by simp

```

```

show ?thesis
  unfolding generic-fri-recorded-layer-chunk-authenticated-def
  by (rule auth)
qed
qed
qed
qed

lemma accepted-fri-opening-transcript-composition-recorded-layer-chunk-authenticated-at:
assumes fri-openings:
  accepted-fri-opening-transcript s out trace-roots trace-bs trace-final
  dg composition-roots composition-bs composition-final query-idxs
  trace-round-layers composition-round-layers
and out-eq: out = Some (result, final-state)
and round-bound: i < rounds
and layer-bound: j < length composition-roots
and all-layer-bounds:
   $\bigwedge \text{raw } k. k < \text{length } \text{composition-roots} \implies$ 
   $0 < \text{fri-layer-lengths } (\text{length } \text{composition-roots}) (\text{clength} * \text{scale}) ! k \wedge$ 
   $\text{fri-layer-indices } (\text{length } \text{composition-roots}) (\text{index } (\text{to-nat } \text{raw}))$ 
   $(\text{clength} * \text{scale}) ! k <$ 
   $\text{fri-layer-lengths } (\text{length } \text{composition-roots}) (\text{clength} * \text{scale}) ! k$ 
shows
  generic-fri-recorded-layer-chunk-authenticated composition-roots
  query-idxs composition-round-layers final-state i j
proof –
from fri-openings obtain result' final-state' fr f-fl as fl query-state
  raw-idxs query-chunks where
  out-def: out = Some (result', final-state')
and trace-roots-eq: trace-roots = map snd f-fl
and composition-roots-eq: composition-roots = map snd fl
and composition-bs-eq: composition-bs = map fst fl
and outcome:
  Some (result', final-state') ∈
  set-dist (execute
    (ntimes
      (verifier-query-round-program fr f-fl trace-final as fl
        composition-final)
      rounds)
    query-state)
and len-raw: length raw-idxs = rounds
and query-idxs-eq:
  query-idxs = map (λraw. index (to-nat raw)) raw-idxs
and len-query-chunks: length query-chunks = rounds
and transcript-query:
  PTranscript query-state =
  List.concat query-chunks @ PTranscript final-state'
and query-chunk:
   $\bigwedge i. i < \text{rounds} \implies$ 

```

```

    verifier-query-round-chunk (query-idxs ! i)
    trace-roots composition-roots (query-chunks ! i)
and replay-lookup:
   $\bigwedge i. i < \text{rounds} \implies$ 
    fmllookup (HashMap final-state')
      (QueryIndexChallenge (PQueryCounter query-state + i)
        (state-after-query-chunks (PState query-state) query-chunks i)) =
      Some (raw-idxs ! i)
and recorded:
   $\bigwedge i. i < \text{rounds} \implies$ 
    query-round-fri-layer-transcripts (query-idxs ! i)
    trace-roots composition-roots (query-chunks ! i)
    (trace-round-layers ! i) (composition-round-layers ! i)
unfolding accepted-fri-opening-transcript-def by blast
have result-eq: result' = result
using out-def out-eq by simp
have final-state-eq: final-state' = final-state
using out-def out-eq by simp
have outcome':
  Some (result, final-state)  $\in$ 
    set-dist (execute
      (ntimes
        (verifier-query-round-program fr f-fl trace-final as fl
          composition-final)
        rounds)
      query-state)
using outcome result-eq final-state-eq by simp
let ?round =
  verifier-query-round-program fr f-fl trace-final as fl composition-final
from query-opening-ntimes-outcome-decomp-at[OF outcome' round-bound]
obtain prefix x suffix s-i s-suc where
  prefix:
    Some (prefix, s-i)  $\in$ 
      set-dist (execute (ntimes ?round i) query-state)
and round:
    Some (x, s-suc)  $\in$  set-dist (execute ?round s-i)
and suffix:
    Some (suffix, final-state)  $\in$ 
      set-dist (execute (ntimes ?round (rounds - Suc i)) s-suc)
by blast
have x-eq: x = ()
by (cases x) simp
show ?thesis
proof (rule query-opening-ntimes-prefix-transcript-state-alignment
  [OF prefix])
  fix prefix-query-idxs prefix-chunks
  assume len-prefix-idxs: length prefix-query-idxs = i
    and len-prefix-chunks: length prefix-chunks = i
    and transcript-prefix:

```

```

PTranscript query-state =
  List.concat prefix-chunks @ PTranscript s-i
and state-i:
  PState s-i =
    state-after-query-chunks (PState query-state) prefix-chunks i
and counter-i: PQueryCounter s-i = PQueryCounter query-state + i
and prefix-ext: query-state ≤ s-i
and prefix-rounds:
   $\bigwedge k. k < i \implies$ 
    verifier-query-round-chunk (prefix-query-idxs ! k)
      (map snd f-fl) (map snd fl) (prefix-chunks ! k)
from round[unfolded x-eq verifier-query-round-program-alt-def]
obtain raw s0 where
  raw-out:
    Some (raw, s0) ∈
      set-dist (execute receive-query-index-challenge s-i)
and after:
    Some ((), s-suc) ∈
      set-dist (execute
        (verifier-query-round-after-index-program fr f-fl trace-final as
          fl composition-final raw) s0)
      by (auto elim! set-dist-bindE)
have tr-s0: PTranscript s0 = PTranscript s-i
  using receive-query-index-challenge-outcome[OF raw-out] by simp
show ?thesis
proof (rule verifier-query-round-program-outcome[OF round[unfolded x-eq]])
  fix raw' idx chunk
  assume idx-eq: idx = index (to-nat raw')
  and chunk-shape:
    verifier-query-round-chunk idx (map snd f-fl) (map snd fl)
      chunk
  and transcript-round:
    PTranscript s-i = chunk @ PTranscript s-suc
  and state-suc:
    PState s-suc = foldl concat (PState s-i) chunk
  and round-ext: s-i ≤ s-suc
  and counter-suc:
    PQueryCounter s-suc = Suc (PQueryCounter s-i)
  and lookup-suc:
    fmlookup (HashMap s-suc)
      (QueryIndexChallenge (PQueryCounter s-i) (PState s-i)) =
        Some raw'
  have raw-eq': raw = raw'
proof –
  have lookup-s0:
    fmlookup (HashMap s0)
      (QueryIndexChallenge (PQueryCounter s-i) (PState s-i)) =
        Some raw
  using receive-query-index-challenge-outcome[OF raw-out] by simp

```

```

have s0-suc:  $s0 \leq s\text{-suc}$ 
  using verifier-query-round-after-index-program-outcome[OF after]
  by simp
have lookup-suc-raw:
  fmlookup (HashMap s-suc)
    (QueryIndexChallenge (PQueryCounter s-i) (PState s-i)) =
    Some raw
  using hash-extension-lookup[OF lookup-s0 s0-suc] .
show ?thesis
  using lookup-suc lookup-suc-raw by simp
qed
show ?thesis
proof (rule query-opening-ntimes-suffix-transcript-state-alignment
  [OF suffix])
fix suffix-query-idxs suffix-chunks
assume len-suffix-idxs:
  length suffix-query-idxs = rounds - Suc i
and len-suffix-chunks:
  length suffix-chunks = rounds - Suc i
and transcript-suffix:
  PTranscript s-suc =
    List.concat suffix-chunks @ PTranscript final-state
and suffix-state:
  PState final-state =
    state-after-query-chunks (PState s-suc) suffix-chunks
    (rounds - Suc i)
and suffix-counter:
  PQueryCounter final-state =
    PQueryCounter s-suc + (rounds - Suc i)
and suffix-ext: s-suc  $\leq$  final-state
and suffix-rounds:
   $\bigwedge k. k < \text{rounds} - \text{Suc } i \implies$ 
    verifier-query-round-chunk (suffix-query-idxs ! k)
    (map snd f-fl) (map snd fl) (suffix-chunks ! k)
let ?selected-chunks = prefix-chunks @ chunk # suffix-chunks
have len-selected-chunks: length ?selected-chunks = rounds
  using len-prefix-chunks len-suffix-chunks round-bound by simp
have selected-chunk-lens:
   $\bigwedge k. k < \text{rounds} \implies$ 
    length (?selected-chunks ! k) =
      verifier-query-round-transcript-length (query-idxs ! k)
      (map snd f-fl) (map snd fl)
proof -
  fix k
  assume k-bound:  $k < \text{rounds}$ 
  show
    length (?selected-chunks ! k) =
      verifier-query-round-transcript-length (query-idxs ! k)
      (map snd f-fl) (map snd fl)

```

```

proof (cases k < i)
  case True
  have nth-eq: ?selected-chunks ! k = prefix-chunks ! k
    using True len-prefix-chunks by (simp add: nth-append)
  have
    length (prefix-chunks ! k) =
      verifier-query-round-transcript-length
        (prefix-query-idxs ! k) (map snd f-fl) (map snd fl)
    by (rule verifier-query-round-chunk-length
      [OF prefix-rounds[OF True]])
  then show ?thesis
    using nth-eq
    by (simp add:
      verifier-query-round-transcript-length-index-irrelevant)
next
  case False
  show ?thesis
  proof (cases k = i)
    case True
    have nth-eq: ?selected-chunks ! k = chunk
      using True len-prefix-chunks by (simp add: nth-append)
    have
      length chunk =
        verifier-query-round-transcript-length idx
          (map snd f-fl) (map snd fl)
      by (rule verifier-query-round-chunk-length[OF chunk-shape])
    then show ?thesis
      using nth-eq
      by (simp add:
        verifier-query-round-transcript-length-index-irrelevant)
    next
    case False
    let ?k = k - Suc i
    have ge: Suc i ≤ k
      using ⟨¬ k < i⟩ False by linarith
    have k-bound': ?k < rounds - Suc i
      using ge k-bound by linarith
    have nth-eq: ?selected-chunks ! k = suffix-chunks ! ?k
      using ge len-prefix-chunks by (simp add: nth-append)
    have
      length (suffix-chunks ! ?k) =
        verifier-query-round-transcript-length
          (suffix-query-idxs ! ?k) (map snd f-fl) (map snd fl)
      by (rule verifier-query-round-chunk-length
        [OF suffix-rounds[OF k-bound']])
    then show ?thesis
      using nth-eq
      by (simp add:
        verifier-query-round-transcript-length-index-irrelevant)

```

```

    qed
  qed
  qed
  have query-chunk-f:
     $\bigwedge k. k < \text{rounds} \implies$ 
      verifier-query-round-chunk (query-idxs ! k)
      (map snd f-fl) (map snd fl) (query-chunks ! k)
    using query-chunk trace-roots-eq composition-roots-eq by simp
  have query-chunk-lens:
     $\bigwedge k. k < \text{rounds} \implies$ 
      length (query-chunks ! k) =
      verifier-query-round-transcript-length (query-idxs ! k)
      (map snd f-fl) (map snd fl)
    using query-chunk-f verifier-query-round-chunk-length by blast
  have selected-transcript:
    PTranscript query-state =
      List.concat ?selected-chunks @ PTranscript final-state
    using transcript-prefix transcript-round transcript-suffix by simp
  have concat-eq: List.concat ?selected-chunks = List.concat query-chunks
    using selected-transcript transcript-query final-state-eq by simp
  have chunks-eq: ?selected-chunks = query-chunks
  proof -
    have same-lens:
       $\bigwedge k. k < \text{length } ?\text{selected-chunks} \implies$ 
        length (?selected-chunks ! k) = length (query-chunks ! k)
      using selected-chunk-lens query-chunk-lens len-selected-chunks
      by simp
    have concat-eq-empty:
      List.concat ?selected-chunks @ [] = List.concat query-chunks
      using concat-eq by simp
    have len-eq: length ?selected-chunks = length query-chunks
      using len-selected-chunks len-query-chunks by simp
    have ?selected-chunks = query-chunks  $\wedge$  ([] :: 'f list) = []
      by (rule concat-append-eq-concat-same-chunk-lengths
        [OF len-eq same-lens concat-eq-empty])
    then show ?thesis
      by simp
  qed
  have prefix-state-eq:
    state-after-query-chunks (PState query-state) prefix-chunks i =
    state-after-query-chunks (PState query-state) query-chunks i
  proof -
    have
      state-after-query-chunks (PState query-state) ?selected-chunks i =
      state-after-query-chunks (PState query-state) prefix-chunks i
      by (rule state-after-query-chunks-append-prefix
        [OF len-prefix-chunks])
    then show ?thesis
      using chunks-eq by simp
  
```



```

qed
have lookup-final:
  fmlookup (HashMap final-state)
    (QueryIndexChallenge (PQueryCounter query-state + i)
      (state-after-query-chunks (PState query-state) query-chunks i)) =
    Some raw'
proof -
  have lookup-suc':
    fmlookup (HashMap s-suc)
      (QueryIndexChallenge (PQueryCounter query-state + i)
        (state-after-query-chunks (PState query-state)
          prefix-chunks i)) =
    Some raw'
  using lookup-suc state-i counter-i by simp
  have lookup-final-prefix:
    fmlookup (HashMap final-state)
      (QueryIndexChallenge (PQueryCounter query-state + i)
        (state-after-query-chunks (PState query-state)
          prefix-chunks i)) =
    Some raw'
  using hash-extension-lookup[OF lookup-suc' suffix-ext] .
  show ?thesis
    using lookup-final-prefix prefix-state-eq by simp
qed
have raw-idx-eq: raw' = raw-idxs ! i
  using lookup-final replay-lookup[OF round-bound] final-state-eq
  by simp
have idx-eq-query: idx = query-idxs ! i
  using idx-eq raw-idx-eq query-idxs-eq len-raw round-bound by simp
have selected-i: ?selected-chunks ! i = chunk
  using len-prefix-chunks by (simp add: nth-append)
have chunk-i-eq: chunk = query-chunks ! i
  using chunks-eq selected-i by simp
have recorded-i:
  query-round-fri-layer-transcripts (index (to-nat raw))
    (map snd f-fl) (map snd fl) chunk (trace-round-layers ! i)
    (composition-round-layers ! i)
  using recorded[OF round-bound] idx-eq idx-eq-query raw-eq'
    chunk-i-eq trace-roots-eq composition-roots-eq
  by simp
have tr-s0-suc:
  PTranscript s0 = chunk @ PTranscript s-suc
  using tr-s0 transcript-round by simp
have j-bound-fl: j < length fl
  using layer-bound composition-roots-eq by simp
have all-layer-bounds-fl:
   $\bigwedge k. k < \text{length } fl \implies$ 
   $0 < \text{fri-layer-lengths } (\text{length } fl) (\text{clength} * \text{scale}) ! k \wedge$ 
   $\text{fri-layer-indices } (\text{length } fl) (\text{index } (\text{to-nat } \text{raw}))$ 

```

```

      (clength * scale) ! k <
      fri-layer-lengths (length fl) (clength * scale) ! k
using all-layer-bounds composition-roots-eq by simp
have auth-suc:
  fri-layer-chunk-authenticated (snd (fl ! j))
  (fri-layer-lengths (length fl) (clength * scale) ! j)
  (fri-layer-indices (length fl) (index (to-nat raw))
    (clength * scale) ! j)
  (composition-round-layers ! i ! j) s-suc
by (rule
  verifier-query-round-after-index-composition-recorded-layer-chunk-authenticated
  [OF after recorded-i tr-s0-suc j-bound-fl
    all-layer-bounds-fl])
have auth-final:
  fri-layer-chunk-authenticated (snd (fl ! j))
  (fri-layer-lengths (length fl) (clength * scale) ! j)
  (fri-layer-indices (length fl) (index (to-nat raw))
    (clength * scale) ! j)
  (composition-round-layers ! i ! j) final-state
by (rule fri-layer-chunk-authenticated-mono[OF auth-suc suffix-ext])
have raw-idx-query: index (to-nat raw) = query-idxs ! i
using idx-eq idx-eq-query raw-eq' by simp
have auth:
  fri-layer-chunk-authenticated (composition-roots ! j)
  (fri-evidence-layer-len composition-roots j)
  (fri-evidence-layer-idx composition-roots query-idxs i j)
  (composition-round-layers ! i ! j) final-state
using auth-final composition-roots-eq raw-idx-query layer-bound
unfolding fri-evidence-layer-len-def fri-evidence-layer-idx-def
by simp
show ?thesis
unfolding generic-fri-recorded-layer-chunk-authenticated-def
by (rule auth)
qed
qed
qed
qed
end

end

```

```

theory Soundness-FRI-Composition-Residuals
imports
  Soundness-FRI-Authenticated-Route
  Soundness-FRI-Base-Recorded-Chunk
  Soundness-FRI-Final-Step-Replay
  Soundness-FRI-Raw-Layer-Bounds

```

begin

Narrow residual refinements for the verifier-tied composition FRI route. This layer keeps the larger authenticated route theory stable while naming the exact remaining base-opening gap: the recorded first-layer composition FRI chunk is not known to be authenticated.

context *soundness*

begin

definition *composition-fri-reachable-verifier-tie-gap*

$:: ('f, 'a) \text{ protocol-channel-scheme} \Rightarrow$
 $(\text{unit list} \times ('f, 'a) \text{ protocol-channel-scheme}) \text{ option} \Rightarrow \text{bool}$

where

$\text{composition-fri-reachable-verifier-tie-gap } s \text{ out} \longleftrightarrow$
 $\text{composition-fri-bad-with-reachable-partial-candidate } s \text{ out} \wedge$
 $\neg \text{composition-fri-bad-with-verifier-tied-partial-candidate } s \text{ out}$

definition *composition-fri-verifier-tied-candidate-transfer-gap*

$:: ('f, 'a) \text{ protocol-channel-scheme} \Rightarrow$
 $(\text{unit list} \times ('f, 'a) \text{ protocol-channel-scheme}) \text{ option} \Rightarrow \text{bool}$

where

$\text{composition-fri-verifier-tied-candidate-transfer-gap } s \text{ out} \longleftrightarrow$
 $(\exists \text{ trace-roots trace-bs trace-final fri-dg composition-roots}$
 $\text{composition-bs composition-final fri-query-idxs trace-round-layers}$
 $\text{composition-round-layers fr as trace-openings composition-openings}$
 $\text{trace-table composition-table.}$
 $\text{composition-fri-bad-with-reachable-partial-candidate } s \text{ out} \wedge$
 $\text{accepted-fri-opening-transcript } s \text{ out trace-roots trace-bs}$
 $\text{trace-final fri-dg composition-roots composition-bs}$
 $\text{composition-final fri-query-idxs trace-round-layers}$
 $\text{composition-round-layers} \wedge$
 $\text{accepted-with-partial-initial-openings-aligned } s \text{ out fr trace-roots}$
 $\text{trace-final as fri-dg composition-roots composition-final}$
 $\text{fri-query-idxs trace-openings composition-openings} \wedge$
 $\text{partial-trace-table-candidate trace-table trace-openings} \wedge$
 $\text{partial-composition-table-candidate composition-table}$
 $\text{composition-openings} \wedge$
 $\neg (\text{trace-table-low-degree trace-table} \wedge$
 $\neg \text{composition-table-low-degree maxDegree composition-table}))$

definition *composition-fri-verifier-tied-candidate-available-gap*

$:: ('f, 'a) \text{ protocol-channel-scheme} \Rightarrow$
 $(\text{unit list} \times ('f, 'a) \text{ protocol-channel-scheme}) \text{ option} \Rightarrow \text{bool}$

where

$\text{composition-fri-verifier-tied-candidate-available-gap } s \text{ out} \longleftrightarrow$
 $(\exists \text{ trace-roots trace-bs trace-final fri-dg composition-roots}$
 $\text{composition-bs composition-final fri-query-idxs trace-round-layers}$
 $\text{composition-round-layers fr as trace-openings composition-openings}$

trace-table composition-table.
composition-fri-reachable-verifier-tie-gap s out $\wedge$
accepted-fri-opening-transcript s out trace-roots trace-bs
trace-final fri-dg composition-roots composition-bs
composition-final fri-query-idxs trace-round-layers
composition-round-layers $\wedge$
accepted-with-partial-initial-openings-aligned s out fr trace-roots
trace-final as fri-dg composition-roots composition-final
fri-query-idxs trace-openings composition-openings $\wedge$
partial-trace-table-candidate trace-table trace-openings $\wedge$
partial-composition-table-candidate composition-table
composition-openings)

lemma *composition-fri-aligned-candidate-bad-contradicts-verifier-tie-gap:*

assumes *gap: composition-fri-reachable-verifier-tie-gap s out*

and *fri-openings:*

accepted-fri-opening-transcript s out trace-roots trace-bs
trace-final fri-dg composition-roots composition-bs
composition-final fri-query-idxs trace-round-layers
composition-round-layers

and *aligned:*

accepted-with-partial-initial-openings-aligned s out fr trace-roots
trace-final as fri-dg composition-roots composition-final
fri-query-idxs trace-openings composition-openings

and *trace-candidate:*

partial-trace-table-candidate trace-table trace-openings

and *composition-candidate:*

partial-composition-table-candidate composition-table
composition-openings

and *trace-low: trace-table-low-degree trace-table*

and *composition-not-low:*

$\neg$ *composition-table-low-degree maxDegree composition-table*

shows *False*

proof —

have *composition-fri-bad-with-verifier-tied-partial-candidate s out*

unfolding *composition-fri-bad-with-verifier-tied-partial-candidate-def*

by (*intro exI conjI*)

(rule fri-openings, rule aligned, rule trace-candidate,
rule composition-candidate, rule trace-low, rule composition-not-low)

then show *False*

using *gap unfolding composition-fri-reachable-verifier-tie-gap-def* **by** *simp*

qed

lemma *composition-fri-aligned-candidate-transfer-imp-transfer-gap:*

assumes *gap: composition-fri-reachable-verifier-tie-gap s out*

and *fri-openings:*

accepted-fri-opening-transcript s out trace-roots trace-bs
trace-final fri-dg composition-roots composition-bs
composition-final fri-query-idxs trace-round-layers

composition-round-layers
and aligned:
accepted-with-partial-initial-openings-aligned s out fr trace-roots
trace-final as fri-dg composition-roots composition-final
fri-query-idxs trace-openings composition-openings
and trace-candidate:
partial-trace-table-candidate trace-table trace-openings
and composition-candidate:
partial-composition-table-candidate composition-table
composition-openings
and transfer:
 $\neg$ (*trace-table-low-degree trace-table* $\wedge$
 $\neg$ *composition-table-low-degree maxDegree composition-table*)
shows *composition-fri-verifier-tied-candidate-transfer-gap s out*
proof –
have *raw: composition-fri-bad-with-reachable-partial-candidate s out*
using *gap unfolding composition-fri-reachable-verifier-tie-gap-def*
by *simp*
show *?thesis*
unfolding *composition-fri-verifier-tied-candidate-transfer-gap-def*
by (*intro exI conjI*)
(rule raw, rule fri-openings, rule aligned, rule trace-candidate,
rule composition-candidate, rule transfer)
qed

lemma *composition-fri-verifier-tied-candidate-available-gap-imp-transfer-gap:*
assumes *composition-fri-verifier-tied-candidate-available-gap s out*
shows *composition-fri-verifier-tied-candidate-transfer-gap s out*
proof –
obtain *trace-roots trace-bs trace-final fri-dg composition-roots*
composition-bs composition-final fri-query-idxs trace-round-layers
composition-round-layers fr as trace-openings composition-openings
trace-table composition-table
where *gap: composition-fri-reachable-verifier-tie-gap s out*
and *fri-openings:*
accepted-fri-opening-transcript s out trace-roots trace-bs
trace-final fri-dg composition-roots composition-bs
composition-final fri-query-idxs trace-round-layers
composition-round-layers
and aligned:
accepted-with-partial-initial-openings-aligned s out fr trace-roots
trace-final as fri-dg composition-roots composition-final
fri-query-idxs trace-openings composition-openings
and trace-candidate:
partial-trace-table-candidate trace-table trace-openings
and composition-candidate:
partial-composition-table-candidate composition-table
composition-openings
using *assms*

```

unfolding composition-fri-verifier-tied-candidate-available-gap-def
by blast
show ?thesis
proof (cases
  trace-table-low-degree trace-table  $\wedge$ 
   $\neg$  composition-table-low-degree maxDegree composition-table)
case True
have False
  by (rule composition-fri-aligned-candidate-bad-contradicts-verifier-tie-gap
    [OF gap fri-openings aligned trace-candidate composition-candidate
      True[THEN conjunct1] True[THEN conjunct2]])
then show ?thesis by simp
next
case False
show ?thesis
  by (rule composition-fri-aligned-candidate-transfer-imp-transfer-gap
    [OF gap fri-openings aligned trace-candidate composition-candidate
      False])
qed
qed

lemma wp-composition-fri-verifier-tied-candidate-available-gap-bound-from-transfer-gap:
assumes transfer-bound:
  wp-event verify-monad
  (composition-fri-verifier-tied-candidate-transfer-gap s)  $s \leq T$ 
shows
  wp-event verify-monad
  (composition-fri-verifier-tied-candidate-available-gap s)  $s \leq T$ 
by (rule order-trans[OF - transfer-bound])
  (rule wp-event-mono,
    rule composition-fri-verifier-tied-candidate-available-gap-imp-transfer-gap)

lemma composition-fri-verifier-tied-reduction-staged-bound:
assumes reductions:
   $\wedge$  data attacker-state.
  Some (data, attacker-state) \in
    set-dist (execute (checked-staged-transcript-program A)
      adversary-initial-state) \implies
    composition-fri-verifier-tied-reduction
    (verifier-state-from-adversary attacker-state
      (staged-proof-transcript data))
shows
  wp-event (checked-staged-security-experiment-with-data-state A)
  (staged-security-with-data-state-verifier-event
    composition-fri-bad-with-verifier-tied-partial-candidate)
  adversary-initial-state \leq composition-fri-error
by (rule composition-fri-bad-with-verifier-tied-partial-candidates-staged-bound)
  (rule composition-fri-verifier-tied-reductionD[OF reductions])

```

lemma *composition-fri-verifier-tied-reduction-from-reachable-reduction:*
assumes *composition-fri-reachable-partial-candidate-reduction-assumption s*
shows *composition-fri-verifier-tied-reduction s*
proof –
have *reachable:*
 wp-event verify-monad
 (*composition-fri-bad-with-reachable-partial-candidate s*) $s \leq$
 composition-fri-error
using *assms*
unfolding
 composition-fri-reachable-partial-candidate-reduction-assumption-def
by *simp*
show *?thesis*
unfolding *composition-fri-verifier-tied-reduction-def*
by (*rule order-trans[OF - reachable]*)
 (*rule wp-event-mono,*
 rule composition-fri-bad-with-verifier-tied-imp-reachable-partial-candidate)
qed

lemma *composition-fri-reachable-imp-verifier-tied-or-gap:*
assumes *composition-fri-bad-with-reachable-partial-candidate s out*
shows
 composition-fri-bad-with-verifier-tied-partial-candidate s out $\vee$
 composition-fri-reachable-verifier-tie-gap s out
using *assms*
unfolding *composition-fri-reachable-verifier-tie-gap-def*
by *blast*

lemma *wp-composition-fri-reachable-bound-from-verifier-tied-and-gap:*
fixes $V\ G :: \text{prob}$
assumes *verifier-bound:*
 wp-event verify-monad
 (*composition-fri-bad-with-verifier-tied-partial-candidate s*) $s \leq V$
and *gap-bound:*
 wp-event verify-monad
 (*composition-fri-reachable-verifier-tie-gap s*) $s \leq G$
shows
 wp-event verify-monad
 (*composition-fri-bad-with-reachable-partial-candidate s*) $s \leq V + G$
proof –
have *wp-event verify-monad*
 (*composition-fri-bad-with-reachable-partial-candidate s*) $s \leq$
 wp-event verify-monad
 ($\lambda out.$
 composition-fri-bad-with-verifier-tied-partial-candidate s out $\vee$
 composition-fri-reachable-verifier-tie-gap s out) s
by (*rule wp-event-mono*)
 (*rule composition-fri-reachable-imp-verifier-tied-or-gap*)
also have $\dots \leq$

```

    wp-event verify-monad
      (composition-fri-bad-with-verifier-tied-partial-candidate s) s +
    wp-event verify-monad
      (composition-fri-reachable-verifier-tie-gap s) s
  by (rule wp-event-union-bound)
also have ... ≤ V + G
  by (rule add-mono[OF verifier-bound gap-bound])
finally show ?thesis .
qed

lemma composition-fri-reachable-partial-candidate-reduction-from-verifier-tied-and-gap:
  fixes V G :: prob
  assumes verifier-bound:
    wp-event verify-monad
      (composition-fri-bad-with-verifier-tied-partial-candidate s) s ≤ V
  and gap-bound:
    wp-event verify-monad
      (composition-fri-reachable-verifier-tie-gap s) s ≤ G
  and total-bound: V + G ≤ composition-fri-error
  shows composition-fri-reachable-partial-candidate-reduction-assumption s
proof -
  have old-bound:
    wp-event verify-monad
      (composition-fri-bad-with-reachable-partial-candidate s) s ≤ V + G
  by (rule wp-composition-fri-reachable-bound-from-verifier-tied-and-gap
    [OF verifier-bound gap-bound])
  show ?thesis
  unfolding
    composition-fri-reachable-partial-candidate-reduction-assumption-def
  by (rule order-trans[OF old-bound total-bound])
qed

definition composition-fri-verifier-tied-recorded-base-chunk-auth-missing
  :: ('f, 'a) protocol-channel-scheme ⇒
    (unit list × ('f, 'a) protocol-channel-scheme) option ⇒ bool
where
  composition-fri-verifier-tied-recorded-base-chunk-auth-missing s out
  ⇔
  (∃ trace-roots trace-bs trace-final fri-dg composition-roots
    composition-bs composition-final fri-query-idxs trace-round-layers
    composition-round-layers fr as trace-openings composition-openings
    trace-table composition-table round-idx xp xp-path xn xn-path
    result final-state.
    out = Some (result, final-state) ∧
    accepted-fri-opening-transcript s out trace-roots trace-bs
    trace-final fri-dg composition-roots composition-bs
    composition-final fri-query-idxs trace-round-layers
    composition-round-layers ∧
    accepted-with-partial-initial-openings-aligned s out fr trace-roots

```


trace-final as fri-dg composition-roots composition-final fri-query-idxs
trace-openings composition-openings $\wedge$
partial-trace-table-candidate trace-table trace-openings $\wedge$
partial-composition-table-candidate composition-table
composition-openings $\wedge$
trace-table-low-degree trace-table $\wedge$
 $\neg$ *composition-table-low-degree maxDegree composition-table* $\wedge$
round-idx < length fri-query-idxs $\wedge$
0 < length composition-bs $\wedge$
fri-layer-step-evidence
(composition-roots ! 0)
(composition-bs ! 0)
(fri-evidence-layer-len composition-roots 0)
(fri-evidence-layer-idx composition-roots fri-query-idxs round-idx 0)
1
(fri-sibling-index (fri-evidence-layer-len composition-roots 0)
(fri-evidence-layer-idx composition-roots fri-query-idxs round-idx 0))
xp xp-path xn xn-path
(fri-evidence-next-idx composition-roots fri-query-idxs round-idx 0)
(fri-evidence-next-value composition-roots composition-bs
fri-query-idxs round-idx 0 xp xn)
(composition-round-layers ! round-idx ! 0) $\wedge$
 $\neg$ *fri-opening-matches-table*
(fri-evidence-layer-len composition-roots 0)
(fri-evidence-layer-idx composition-roots fri-query-idxs round-idx 0)
composition-table xp xn $\wedge$
 $\neg$ *fri-layer-chunk-authenticated (composition-roots ! 0)*
(fri-evidence-layer-len composition-roots 0)
(fri-evidence-layer-idx composition-roots fri-query-idxs round-idx 0)
(composition-round-layers ! round-idx ! 0) final-state)

lemma *composition-fri-verifier-tied-recorded-base-chunk-auth-missing-false:*

assumes

composition-fri-verifier-tied-recorded-base-chunk-auth-missing s out

shows *False*

proof —

from *assms obtain trace-roots trace-bs trace-final fri-dg*

composition-roots composition-bs composition-final fri-query-idxs

trace-round-layers composition-round-layers fr as trace-openings

composition-openings trace-table composition-table round-idx xp xp-path

xn xn-path result final-state

where *out-eq: out = Some (result, final-state)*

and *fri-openings:*

accepted-fri-opening-transcript s out trace-roots trace-bs

trace-final fri-dg composition-roots composition-bs

composition-final fri-query-idxs trace-round-layers

composition-round-layers

and *round-bound: round-idx < length fri-query-idxs*

and *composition-bs-nonempty: 0 < length composition-bs*

```

and missing:
  ¬ fri-layer-chunk-authenticated (composition-roots ! 0)
    (fri-evidence-layer-len composition-roots 0)
    (fri-evidence-layer-idx composition-roots fri-query-idxs round-idx 0)
    (composition-round-layers ! round-idx ! 0) final-state
unfolding
  composition-fri-verifier-tied-recorded-base-chunk-auth-missing-def
by blast
have len-composition-bs:
  length composition-bs = length composition-roots
  using accepted-fri-opening-transcript-shapes(4)[OF fri-openings] .
have composition-nonempty: 0 < length composition-roots
  using composition-bs-nonempty len-composition-bs by simp
have len-query: length fri-query-idxs = rounds
  using accepted-fri-opening-transcript-shapes(7)[OF fri-openings] .
have round-bound-rounds: round-idx < rounds
  using round-bound len-query by simp
have auth:
  fri-layer-chunk-authenticated (composition-roots ! 0) (clength * scale)
    (fri-query-idxs ! round-idx)
    (composition-round-layers ! round-idx ! 0) final-state
by (rule
  accepted-fri-opening-transcript-composition-base-recorded-chunk-authenticated-at
    [OF fri-openings out-eq round-bound-rounds composition-nonempty])
have len0:
  fri-evidence-layer-len composition-roots 0 = clength * scale
  using composition-nonempty
  unfolding fri-evidence-layer-len-def
  by (cases composition-roots) simp-all
have idx0:
  fri-evidence-layer-idx composition-roots fri-query-idxs round-idx 0 =
    fri-query-idxs ! round-idx
  using composition-nonempty
  unfolding fri-evidence-layer-idx-def
  by (cases composition-roots) simp-all
show False
  using missing auth unfolding len0 idx0 by contradiction
qed

lemma wp-composition-fri-verifier-tied-recorded-base-chunk-auth-missing-zero:
shows
  wp-event verify-monad
    (composition-fri-verifier-tied-recorded-base-chunk-auth-missing s)
    s ≤ 0
proof –
have wp-event verify-monad
  (composition-fri-verifier-tied-recorded-base-chunk-auth-missing s) s
  ≤ wp-event verify-monad (λ-. False) s
by (rule wp-event-mono)

```

(rule composition-fri-verifier-tied-recorded-base-chunk-auth-missing-false)
also have ... = 0
unfolding wp-event-def wp-def dist-expect-def **by** simp
finally show ?thesis .
qed

lemma composition-fri-verifier-tied-base-chunk-auth-gap-imp-recorded-missing:
assumes composition-fri-verifier-tied-base-chunk-auth-gap s out
shows
 composition-fri-verifier-tied-recorded-base-chunk-auth-missing s out
proof –
from assms **have** base:
 composition-fri-verifier-tied-sampled-base-opening-conflict s out
and no-auth:
 ¬ composition-fri-verifier-tied-authenticated-base-opening-conflict
 s out
unfolding composition-fri-verifier-tied-base-chunk-auth-gap-def
by simp-all
from base **obtain** trace-roots trace-bs trace-final fri-dg composition-roots
 composition-bs composition-final fri-query-idxs trace-round-layers
 composition-round-layers fr as trace-openings composition-openings
 trace-table composition-table
where fri-openings:
 accepted-fri-opening-transcript s out trace-roots trace-bs
 trace-final fri-dg composition-roots composition-bs
 composition-final fri-query-idxs trace-round-layers
 composition-round-layers
and aligned:
 accepted-with-partial-initial-openings-aligned s out fr trace-roots
 trace-final as fri-dg composition-roots composition-final fri-query-idxs
 trace-openings composition-openings
and trace-cand:
 partial-trace-table-candidate trace-table trace-openings
and comp-cand:
 partial-composition-table-candidate composition-table
 composition-openings
and trace-low: trace-table-low-degree trace-table
and comp-not-low:
 ¬ composition-table-low-degree maxDegree composition-table
and generic:
 generic-fri-sampled-base-opening-conflict composition-table
 composition-roots composition-bs fri-query-idxs
 composition-round-layers
unfolding composition-fri-verifier-tied-sampled-base-opening-conflict-def
by blast
from fri-openings **obtain** result final-state **where** out-eq:
 out = Some (result, final-state)
unfolding accepted-fri-opening-transcript-def **by** blast
from generic **obtain** round-idx xp xp-path xn xn-path **where**

```

round-bound: round-idx < length fri-query-idxs
and composition-bs-nonempty: 0 < length composition-bs
and step:
  fri-layer-step-evidence
    (composition-roots ! 0)
    (composition-bs ! 0)
    (fri-evidence-layer-len composition-roots 0)
    (fri-evidence-layer-idx composition-roots fri-query-idxs round-idx 0)
    1
    (fri-sibling-index (fri-evidence-layer-len composition-roots 0)
      (fri-evidence-layer-idx composition-roots fri-query-idxs round-idx 0))
    xp xp-path xn xn-path
    (fri-evidence-next-idx composition-roots fri-query-idxs round-idx 0)
    (fri-evidence-next-value composition-roots composition-bs
      fri-query-idxs round-idx 0 xp xn)
    (composition-round-layers ! round-idx ! 0)
and no-match:
  ¬ fri-opening-matches-table
    (fri-evidence-layer-len composition-roots 0)
    (fri-evidence-layer-idx composition-roots fri-query-idxs round-idx 0)
    composition-table xp xn
unfolding generic-fri-sampled-base-opening-conflict-def
by blast
have missing:
  ¬ fri-layer-chunk-authenticated (composition-roots ! 0)
    (fri-evidence-layer-len composition-roots 0)
    (fri-evidence-layer-idx composition-roots fri-query-idxs round-idx 0)
    (composition-round-layers ! round-idx ! 0) final-state
proof
  assume auth:
    fri-layer-chunk-authenticated (composition-roots ! 0)
      (fri-evidence-layer-len composition-roots 0)
      (fri-evidence-layer-idx composition-roots fri-query-idxs round-idx 0)
      (composition-round-layers ! round-idx ! 0) final-state
  have
    composition-fri-verifier-tied-authenticated-base-opening-conflict
      s out
  unfolding composition-fri-verifier-tied-authenticated-base-opening-conflict-def
  by (intro exI conjI)
    (rule out-eq, rule fri-openings, rule aligned, rule trace-cand,
      rule comp-cand, rule trace-low, rule comp-not-low, rule round-bound,
      rule composition-bs-nonempty, rule step, rule no-match, rule auth)
  then show False
    using no-auth by contradiction
qed
show ?thesis
  unfolding composition-fri-verifier-tied-recorded-base-chunk-auth-missing-def
  by (intro exI conjI)
    (rule out-eq, rule fri-openings, rule aligned, rule trace-cand,

```

*rule comp-cand, rule trace-low, rule comp-not-low, rule round-bound,
rule composition-bs-nonempty, rule step, rule no-match, rule missing)*

qed

lemma *wp-composition-fri-verifier-tied-base-chunk-auth-gap-bound-from-recorded-missing:*

fixes $M :: \text{prob}$

assumes *missing-bound:*

wp-event verify-monad

(composition-fri-verifier-tied-recorded-base-chunk-auth-missing s)

$s \leq M$

shows

wp-event verify-monad

(composition-fri-verifier-tied-base-chunk-auth-gap s) s ≤ M

proof –

have *wp-event verify-monad*

(composition-fri-verifier-tied-base-chunk-auth-gap s) s ≤

wp-event verify-monad

(composition-fri-verifier-tied-recorded-base-chunk-auth-missing s) s

by *(rule wp-event-mono)*

(rule composition-fri-verifier-tied-base-chunk-auth-gap-imp-recorded-missing)

then show *?thesis*

by *(rule order-trans[OF - missing-bound])*

qed

lemma *wp-composition-fri-verifier-tied-sampled-base-bound-from-partial-merkle-and-recorded-missing:*

fixes $M :: \text{prob}$

assumes *missing-bound:*

wp-event verify-monad

(composition-fri-verifier-tied-recorded-base-chunk-auth-missing s)

$s \leq M$

shows

wp-event verify-monad

(composition-fri-verifier-tied-sampled-base-opening-conflict s) s

$\leq \text{wp-event verify-monad } (\text{partial-merkle-inconsistency-bad } s) s + M$

by *(rule wp-composition-fri-verifier-tied-sampled-base-bound-from-partial-merkle-and-auth-gap)*

(rule

wp-composition-fri-verifier-tied-base-chunk-auth-gap-bound-from-recorded-missing

[OF missing-bound])

lemma *wp-composition-fri-verifier-tied-without-same-bound-from-partial-merkle-recorded-missing-and-branches:*

fixes $M N S F :: \text{prob}$

assumes *missing-bound:*

wp-event verify-monad

(composition-fri-verifier-tied-recorded-base-chunk-auth-missing s)

$s \leq M$

and *next-bound:*

wp-event verify-monad

(composition-fri-verifier-tied-sampled-next-value-conflict s) s ≤ N

and *successor-bound:*

wp-event verify-monad
 (composition-fri-verifier-tied-sampled-successor-opening-conflict s)
 $s \leq S$
and *final-bound*:
wp-event verify-monad
 (composition-fri-verifier-tied-sampled-final-value-conflict s) $s \leq F$
shows
wp-event verify-monad
 (composition-fri-verifier-tied-sampled-assignment-conflict-without-same-layer s)
 $s \leq (\text{wp-event verify-monad } (\text{partial-merkle-inconsistency-bad } s) s + M) + N + S + F$
by (rule wp-composition-fri-verifier-tied-without-same-bound-from-branches)
 (rule wp-composition-fri-verifier-tied-sampled-base-bound-from-partial-merkle-and-recorded-missing
 [OF missing-bound],
 rule next-bound,
 rule successor-bound,
 rule final-bound)

lemma wp-composition-fri-verifier-tied-authenticated-conflict-bound-from-recorded-missing-and-branches:

fixes $P M N S F :: \text{prob}$

assumes *partial-merkle-bound*:

wp-event verify-monad (partial-merkle-inconsistency-bad s) $s \leq P$

and *missing-bound*:

wp-event verify-monad
 (composition-fri-verifier-tied-recorded-base-chunk-auth-missing s)
 $s \leq M$

and *next-bound*:

wp-event verify-monad
 (composition-fri-verifier-tied-sampled-next-value-conflict s) $s \leq N$

and *successor-bound*:

wp-event verify-monad
 (composition-fri-verifier-tied-sampled-successor-opening-conflict s)
 $s \leq S$

and *final-bound*:

wp-event verify-monad
 (composition-fri-verifier-tied-sampled-final-value-conflict s) $s \leq F$

shows

wp-event verify-monad
 (composition-fri-verifier-tied-sampled-assignment-conflict-with-authenticated-same-layer s)
 $s \leq (P + M) + N + S + F + P$

proof –

have *without-bound*:

wp-event verify-monad
 (composition-fri-verifier-tied-sampled-assignment-conflict-without-same-layer s)
 $s \leq (P + M) + N + S + F$

proof –

have *wp-event verify-monad*

(composition-fri-verifier-tied-sampled-assignment-conflict-without-same-layer s)
 $s \leq$
 (wp-event verify-monad (partial-merkle-inconsistency-bad s) $s + M$)

```

    + N + S + F
  by (rule
    wp-composition-fri-verifier-tied-without-same-bound-from-partial-merkle-recorded-missing-and-branches
    [OF missing-bound next-bound successor-bound final-bound])
  also have ... ≤ (P + M) + N + S + F
    by (intro add-mono partial-merkle-bound order-refl)
  finally show ?thesis .
qed
show ?thesis
  by (rule
    wp-composition-fri-verifier-tied-authenticated-conflict-bound-from-without-same-and-merkle
    [OF without-bound partial-merkle-bound])
qed

```

definition *composition-fri-verifier-tied-next-value-replay-checked-conflict*

$:: (f, 'a) \text{ protocol-channel-scheme} \Rightarrow$
 $(\text{unit list} \times (f, 'a) \text{ protocol-channel-scheme}) \text{ option} \Rightarrow \text{bool}$

where

composition-fri-verifier-tied-next-value-replay-checked-conflict s out
 $\longleftrightarrow$
 $(\exists \text{ trace-roots trace-bs trace-final fri-dg composition-roots}$
 $\text{ composition-bs composition-final fri-query-idxs trace-round-layers}$
 $\text{ composition-round-layers fr as trace-openings composition-openings}$
 $\text{ trace-table composition-table round-idx round-idx' layer-idx } v \ v'.$
 $\text{accepted-fri-opening-transcript } s \text{ out trace-roots trace-bs}$
 $\text{ trace-final fri-dg composition-roots composition-bs}$
 $\text{ composition-final fri-query-idxs trace-round-layers}$
 $\text{ composition-round-layers} \wedge$
 $\text{accepted-with-partial-initial-openings-aligned } s \text{ out fr trace-roots}$
 $\text{ trace-final as fri-dg composition-roots composition-final fri-query-idxs}$
 $\text{ trace-openings composition-openings} \wedge$
 $\text{partial-trace-table-candidate trace-table trace-openings} \wedge$
 $\text{partial-composition-table-candidate composition-table}$
 $\text{ composition-openings} \wedge$
 $\text{trace-table-low-degree trace-table} \wedge$
 $\neg \text{ composition-table-low-degree maxDegree composition-table} \wedge$
 $\text{round-idx} < \text{length fri-query-idxs} \wedge$
 $\text{round-idx}' < \text{length fri-query-idxs} \wedge$
 $\text{layer-idx} < \text{length composition-bs} \wedge$
 $\text{fri-evidence-next-idx composition-roots fri-query-idxs round-idx}$
 $\text{ layer-idx} =$
 $\text{fri-evidence-next-idx composition-roots fri-query-idxs round-idx}'$
 $\text{ layer-idx} \wedge$
 $\text{generic-fri-round-forced-next-value composition-roots composition-bs}$
 $\text{ fri-query-idxs composition-round-layers round-idx layer-idx } v \wedge$
 $\text{generic-fri-round-forced-next-value composition-roots composition-bs}$
 $\text{ fri-query-idxs composition-round-layers round-idx' layer-idx } v' \wedge$
 $v \neq v' \wedge$
 $v = v')$

definition *composition-fri-verifier-tied-next-value-replay-gap*
 $:: ('f, 'a) \text{ protocol-channel-scheme} \Rightarrow$
 $(\text{unit list} \times ('f, 'a) \text{ protocol-channel-scheme}) \text{ option} \Rightarrow \text{bool}$

where
 $\text{composition-fri-verifier-tied-next-value-replay-gap } s \text{ out} \longleftrightarrow$
 $\text{composition-fri-verifier-tied-sampled-next-value-conflict } s \text{ out} \wedge$
 $\neg \text{composition-fri-verifier-tied-next-value-replay-checked-conflict}$
 $s \text{ out}$

lemma *composition-fri-verifier-tied-next-value-replay-checked-conflict-false*:
 $\neg \text{composition-fri-verifier-tied-next-value-replay-checked-conflict } s \text{ out}$

unfolding *composition-fri-verifier-tied-next-value-replay-checked-conflict-def*
by *blast*

lemma *composition-fri-verifier-tied-sampled-next-value-imp-replay-gap*:
assumes *composition-fri-verifier-tied-sampled-next-value-conflict* $s \text{ out}$
shows *composition-fri-verifier-tied-next-value-replay-gap* $s \text{ out}$
using *assms composition-fri-verifier-tied-next-value-replay-checked-conflict-false*
unfolding *composition-fri-verifier-tied-next-value-replay-gap-def*
by *simp*

lemma *composition-fri-verifier-tied-sampled-next-value-conflictE*:
assumes *composition-fri-verifier-tied-sampled-next-value-conflict* $s \text{ out}$
obtains *trace-roots trace-bs trace-final fri-dg composition-roots*
composition-bs composition-final fri-query-idxs trace-round-layers
composition-round-layers fr as trace-openings composition-openings
trace-table composition-table round-idx round-idx' layer-idx v v'

where
 $\text{accepted-fri-opening-transcript } s \text{ out trace-roots trace-bs}$
 $\text{trace-final fri-dg composition-roots composition-bs composition-final}$
 $\text{fri-query-idxs trace-round-layers composition-round-layers}$
 $\text{accepted-with-partial-initial-openings-aligned } s \text{ out fr trace-roots}$
 $\text{trace-final as fri-dg composition-roots composition-final fri-query-idxs}$
 $\text{trace-openings composition-openings}$
 $\text{partial-trace-table-candidate trace-table trace-openings}$
 $\text{partial-composition-table-candidate composition-table}$
 $\text{composition-openings}$
 $\text{trace-table-low-degree trace-table}$
 $\neg \text{composition-table-low-degree maxDegree composition-table}$
 $\text{round-idx} < \text{length fri-query-idxs}$
 $\text{round-idx}' < \text{length fri-query-idxs}$
 $\text{layer-idx} < \text{length composition-bs}$
 $\text{fri-evidence-next-idx composition-roots fri-query-idxs round-idx}$
 $\text{layer-idx} =$
 $\text{fri-evidence-next-idx composition-roots fri-query-idxs round-idx}'$
 layer-idx
 $\text{generic-fri-round-forced-next-value composition-roots composition-bs}$
 $\text{fri-query-idxs composition-round-layers round-idx layer-idx v}$

*generic-fri-round-forced-next-value composition-roots composition-bs
 fri-query-idxs composition-round-layers round-idx' layer-idx v'
 v ≠ v'*
proof –
from *assms* **obtain** *trace-roots trace-bs trace-final fri-dg
 composition-roots composition-bs composition-final fri-query-idxs
 trace-round-layers composition-round-layers fr as trace-openings
 composition-openings trace-table composition-table*
where *fri-openings:*
*accepted-fri-opening-transcript s out trace-roots trace-bs
 trace-final fri-dg composition-roots composition-bs
 composition-final fri-query-idxs trace-round-layers
 composition-round-layers*
and *aligned:*
*accepted-with-partial-initial-openings-aligned s out fr trace-roots
 trace-final as fri-dg composition-roots composition-final
 fri-query-idxs trace-openings composition-openings*
and *trace-cand:*
partial-trace-table-candidate trace-table trace-openings
and *comp-cand:*
*partial-composition-table-candidate composition-table
 composition-openings*
and *trace-low: trace-table-low-degree trace-table*
and *comp-not-low:*
 $\neg$ *composition-table-low-degree maxDegree composition-table*
and *conflict:*
*generic-fri-sampled-next-value-conflict composition-roots
 composition-bs fri-query-idxs composition-round-layers*
unfolding *composition-fri-verifier-tied-sampled-next-value-conflict-def*
by *blast*
from *generic-fri-sampled-next-value-conflictE[OF conflict]*
obtain *round-idx round-idx' layer-idx v v'*
where *round-bound: round-idx < length fri-query-idxs*
and *round-bound': round-idx' < length fri-query-idxs*
and *layer-bound: layer-idx < length composition-bs*
and *same-idx:*
*fri-evidence-next-idx composition-roots fri-query-idxs round-idx
 layer-idx =
 fri-evidence-next-idx composition-roots fri-query-idxs round-idx'
 layer-idx*
and *forced:*
*generic-fri-round-forced-next-value composition-roots composition-bs
 fri-query-idxs composition-round-layers round-idx layer-idx v*
and *forced':*
*generic-fri-round-forced-next-value composition-roots composition-bs
 fri-query-idxs composition-round-layers round-idx' layer-idx v'*
and *neg: v ≠ v'*
by *blast*
show *?thesis*

by (rule that[*OF fri-openings aligned trace-cand comp-cand trace-low
 comp-not-low round-bound round-bound' layer-bound same-idx forced
 forced' neq*])
 qed

lemma *wp-composition-fri-verifier-tied-sampled-next-value-bound-from-replay-gap*:
 fixes *G* :: *prob*
 assumes *gap-bound*:
 wp-event verify-monad
 (*composition-fri-verifier-tied-next-value-replay-gap s*) *s* ≤ *G*
 shows
 wp-event verify-monad
 (*composition-fri-verifier-tied-sampled-next-value-conflict s*) *s* ≤ *G*
proof –
 have *wp-event verify-monad*
 (*composition-fri-verifier-tied-sampled-next-value-conflict s*) *s* ≤
 wp-event verify-monad
 (*composition-fri-verifier-tied-next-value-replay-gap s*) *s*
 by (rule *wp-event-mono*)
 (rule *composition-fri-verifier-tied-sampled-next-value-imp-replay-gap*)
 then show ?thesis
 by (rule *order-trans*[*OF - gap-bound*])
 qed

definition *composition-fri-verifier-tied-successor-replay-checked-conflict*
 :: (*f*, *a*) *protocol-channel-scheme* ⇒
 (*unit list* × (*f*, *a*) *protocol-channel-scheme*) *option* ⇒ *bool*
where
composition-fri-verifier-tied-successor-replay-checked-conflict s out
 ⇔
 (∃ *trace-roots trace-bs trace-final fri-dg composition-roots*
 composition-bs composition-final fri-query-idxs trace-round-layers
 composition-round-layers fr as trace-openings composition-openings
 trace-table composition-table source-round target-round layer-idx
 v xp xp-path xn xn-path.
accepted-fri-opening-transcript s out trace-roots trace-bs
trace-final fri-dg composition-roots composition-bs
composition-final fri-query-idxs trace-round-layers
composition-round-layers ∧
accepted-with-partial-initial-openings-aligned s out fr trace-roots
trace-final as fri-dg composition-roots composition-final fri-query-idxs
trace-openings composition-openings ∧
partial-trace-table-candidate trace-table trace-openings ∧
partial-composition-table-candidate composition-table
composition-openings ∧
trace-table-low-degree trace-table ∧
 ¬ *composition-table-low-degree maxDegree composition-table* ∧
source-round < length fri-query-idxs ∧
target-round < length fri-query-idxs ∧

$\text{Suc layer-idx} < \text{length composition-bs} \wedge$
 $\text{generic-fri-round-forced-next-value composition-roots composition-bs}$
 $\text{fri-query-idxs composition-round-layers source-round layer-idx } v \wedge$
 $\text{fri-layer-step-evidence}$
 $(\text{composition-roots} ! \text{Suc layer-idx})$
 $(\text{composition-bs} ! \text{Suc layer-idx})$
 $(\text{fri-evidence-layer-len composition-roots} (\text{Suc layer-idx}))$
 $(\text{fri-evidence-layer-idx composition-roots fri-query-idxs target-round}$
 $(\text{Suc layer-idx}))$
 $(2 \wedge \text{Suc layer-idx})$
 $(\text{fri-sibling-index}$
 $(\text{fri-evidence-layer-len composition-roots} (\text{Suc layer-idx}))$
 $(\text{fri-evidence-layer-idx composition-roots fri-query-idxs}$
 $\text{target-round} (\text{Suc layer-idx})))$
 $xp xp\text{-path } xn xn\text{-path}$
 $(\text{fri-evidence-next-idx composition-roots fri-query-idxs target-round}$
 $(\text{Suc layer-idx}))$
 $(\text{fri-evidence-next-value composition-roots composition-bs}$
 $\text{fri-query-idxs target-round} (\text{Suc layer-idx}) xp xn)$
 $(\text{composition-round-layers} ! \text{target-round} ! \text{Suc layer-idx}) \wedge$
 $((\text{fri-evidence-next-idx composition-roots fri-query-idxs source-round}$
 $\text{layer-idx} =$
 $\text{fri-evidence-layer-idx composition-roots fri-query-idxs target-round}$
 $(\text{Suc layer-idx}) \wedge$
 $v \neq xp \wedge v = xp) \vee$
 $(\text{fri-evidence-next-idx composition-roots fri-query-idxs source-round}$
 $\text{layer-idx} =$
 fri-sibling-index
 $(\text{fri-evidence-layer-len composition-roots} (\text{Suc layer-idx}))$
 $(\text{fri-evidence-layer-idx composition-roots fri-query-idxs}$
 $\text{target-round} (\text{Suc layer-idx})) \wedge$
 $v \neq xn \wedge v = xn)))$

definition *composition-fri-verifier-tied-successor-replay-gap*

$:: ('f, 'a) \text{ protocol-channel-scheme} \Rightarrow$
 $(\text{unit list} \times ('f, 'a) \text{ protocol-channel-scheme}) \text{ option} \Rightarrow \text{bool}$

where

$\text{composition-fri-verifier-tied-successor-replay-gap } s \text{ out} \longleftrightarrow$
 $\text{composition-fri-verifier-tied-sampled-successor-opening-conflict } s \text{ out} \wedge$
 $\neg \text{composition-fri-verifier-tied-successor-replay-checked-conflict}$
 $s \text{ out}$

lemma *composition-fri-verifier-tied-successor-replay-checked-conflict-false:*

$\neg \text{composition-fri-verifier-tied-successor-replay-checked-conflict } s \text{ out}$

unfolding

$\text{composition-fri-verifier-tied-successor-replay-checked-conflict-def}$

by *blast*

lemma *composition-fri-verifier-tied-sampled-successor-imp-replay-gap:*

assumes *composition-fri-verifier-tied-sampled-successor-opening-conflict s out*
shows *composition-fri-verifier-tied-successor-replay-gap s out*
using *assms*
composition-fri-verifier-tied-successor-replay-checked-conflict-false
unfolding *composition-fri-verifier-tied-successor-replay-gap-def*
by *simp*

lemma *composition-fri-verifier-tied-sampled-successor-opening-conflictE:*

assumes *composition-fri-verifier-tied-sampled-successor-opening-conflict s out*
obtains *trace-roots trace-bs trace-final fri-dg composition-roots*
composition-bs composition-final fri-query-idxs trace-round-layers
composition-round-layers fr as trace-openings composition-openings
trace-table composition-table source-round target-round layer-idx v
xp xp-path xn xn-path
where
accepted-fri-opening-transcript s out trace-roots trace-bs
trace-final fri-dg composition-roots composition-bs composition-final
fri-query-idxs trace-round-layers composition-round-layers
accepted-with-partial-initial-openings-aligned s out fr trace-roots
trace-final as fri-dg composition-roots composition-final fri-query-idxs
trace-openings composition-openings
partial-trace-table-candidate trace-table trace-openings
partial-composition-table-candidate composition-table
composition-openings
trace-table-low-degree trace-table
 $\neg$ *composition-table-low-degree maxDegree composition-table*
source-round < length fri-query-idxs
target-round < length fri-query-idxs
Suc layer-idx < length composition-bs
generic-fri-round-forced-next-value composition-roots composition-bs
fri-query-idxs composition-round-layers source-round layer-idx v
fri-layer-step-evidence
(composition-roots ! Suc layer-idx)
(composition-bs ! Suc layer-idx)
(fri-evidence-layer-len composition-roots (Suc layer-idx))
(fri-evidence-layer-idx composition-roots fri-query-idxs target-round
(Suc layer-idx))
(2 ^ Suc layer-idx)
(fri-sibling-index
(fri-evidence-layer-len composition-roots (Suc layer-idx))
(fri-evidence-layer-idx composition-roots fri-query-idxs
target-round (Suc layer-idx)))
xp xp-path xn xn-path
(fri-evidence-next-idx composition-roots fri-query-idxs target-round
(Suc layer-idx))
(fri-evidence-next-value composition-roots composition-bs
fri-query-idxs target-round (Suc layer-idx) xp xn)

(composition-round-layers ! target-round ! Suc layer-idx)
 (fri-evidence-next-idx composition-roots fri-query-idxs source-round
 layer-idx =
 fri-evidence-layer-idx composition-roots fri-query-idxs target-round
 (Suc layer-idx) $\wedge$
 $v \neq xp$) $\vee$
 (fri-evidence-next-idx composition-roots fri-query-idxs source-round
 layer-idx =
 fri-sibling-index
 (fri-evidence-layer-len composition-roots (Suc layer-idx))
 (fri-evidence-layer-idx composition-roots fri-query-idxs
 target-round (Suc layer-idx)) $\wedge$
 $v \neq xn$)
proof –
from *assms* **obtain** *trace-roots trace-bs trace-final fri-dg*
composition-roots composition-bs composition-final fri-query-idxs
trace-round-layers composition-round-layers fr as trace-openings
composition-openings trace-table composition-table
where *fri-openings*:
accepted-fri-opening-transcript s out trace-roots trace-bs
trace-final fri-dg composition-roots composition-bs
composition-final fri-query-idxs trace-round-layers
composition-round-layers
and *aligned*:
accepted-with-partial-initial-openings-aligned s out fr trace-roots
trace-final as fri-dg composition-roots composition-final
fri-query-idxs trace-openings composition-openings
and *trace-cand*:
partial-trace-table-candidate trace-table trace-openings
and *comp-cand*:
partial-composition-table-candidate composition-table
composition-openings
and *trace-low*: *trace-table-low-degree trace-table*
and *comp-not-low*:
 $\neg$ *composition-table-low-degree maxDegree composition-table*
and *conflict*:
generic-fri-sampled-successor-opening-conflict composition-roots
composition-bs fri-query-idxs composition-round-layers
unfolding
composition-fri-verifier-tied-sampled-successor-opening-conflict-def
by *blast*
from *generic-fri-sampled-successor-opening-conflictE[OF conflict]*
obtain *source-round target-round layer-idx v xp xp-path xn xn-path*
where *source-bound*: *source-round < length fri-query-idxs*
and *target-bound*: *target-round < length fri-query-idxs*
and *layer-bound*: *Suc layer-idx < length composition-bs*
and *forced*:
generic-fri-round-forced-next-value composition-roots composition-bs
fri-query-idxs composition-round-layers source-round layer-idx v

```

and step:
  fri-layer-step-evidence
    (composition-roots ! Suc layer-idx)
    (composition-bs ! Suc layer-idx)
    (fri-evidence-layer-len composition-roots (Suc layer-idx))
    (fri-evidence-layer-idx composition-roots fri-query-idxs
      target-round (Suc layer-idx))
    ( $2 \wedge \text{Suc layer-idx}$ )
    (fri-sibling-index
      (fri-evidence-layer-len composition-roots (Suc layer-idx))
      (fri-evidence-layer-idx composition-roots fri-query-idxs
        target-round (Suc layer-idx)))
  xp xp-path xn xn-path
    (fri-evidence-next-idx composition-roots fri-query-idxs
      target-round (Suc layer-idx))
    (fri-evidence-next-value composition-roots composition-bs
      fri-query-idxs target-round (Suc layer-idx) xp xn)
    (composition-round-layers ! target-round ! Suc layer-idx)
and cases:
  (fri-evidence-next-idx composition-roots fri-query-idxs source-round
    layer-idx =
    fri-evidence-layer-idx composition-roots fri-query-idxs target-round
      (Suc layer-idx)  $\wedge$ 
     $v \neq xp$ )  $\vee$ 
  (fri-evidence-next-idx composition-roots fri-query-idxs source-round
    layer-idx =
    fri-sibling-index
      (fri-evidence-layer-len composition-roots (Suc layer-idx))
      (fri-evidence-layer-idx composition-roots fri-query-idxs
        target-round (Suc layer-idx))  $\wedge$ 
     $v \neq xn$ )
by blast
show ?thesis
by (rule that[OF fri-openings aligned trace-cand comp-cand trace-low
  comp-not-low source-bound target-bound layer-bound forced step
  cases])
qed

lemma wp-composition-fri-verifier-tied-sampled-successor-bound-from-replay-gap:
  fixes G :: prob
  assumes gap-bound:
    wp-event verify-monad
    (composition-fri-verifier-tied-successor-replay-gap s)  $s \leq G$ 
  shows
    wp-event verify-monad
    (composition-fri-verifier-tied-sampled-successor-opening-conflict s)
     $s \leq G$ 
proof –
  have wp-event verify-monad

```

```

    (composition-fri-verifier-tied-sampled-successor-opening-conflict s)
  s ≤
  wp-event verify-monad
    (composition-fri-verifier-tied-successor-replay-gap s) s
  by (rule wp-event-mono)
    (rule composition-fri-verifier-tied-sampled-successor-imp-replay-gap)
  then show ?thesis
  by (rule order-trans[OF - gap-bound])
qed

```

definition *composition-fri-verifier-tied-authenticated-slot-recorded-chunk-gap*

```

:: ('f, 'a) protocol-channel-scheme ⇒
  (unit list × ('f, 'a) protocol-channel-scheme) option ⇒ bool

```

where

```

composition-fri-verifier-tied-authenticated-slot-recorded-chunk-gap
  s out ⟷
  (∃ trace-roots trace-bs trace-final fri-dg composition-roots
    composition-bs composition-final fri-query-idxs trace-round-layers
    composition-round-layers fr as trace-openings composition-openings
    trace-table composition-table result final-state.
    out = Some (result, final-state) ∧
    accepted-fri-opening-transcript s out trace-roots trace-bs
      trace-final fri-dg composition-roots composition-bs
      composition-final fri-query-idxs trace-round-layers
      composition-round-layers ∧
    accepted-with-partial-initial-openings-aligned s out fr trace-roots
      trace-final as fri-dg composition-roots composition-final
      fri-query-idxs trace-openings composition-openings ∧
    partial-trace-table-candidate trace-table trace-openings ∧
    partial-composition-table-candidate composition-table
      composition-openings ∧
    trace-table-low-degree trace-table ∧
    ¬ composition-table-low-degree maxDegree composition-table ∧
    generic-fri-authenticated-slot-recorded-chunk-gap composition-roots
      fri-query-idxs composition-round-layers final-state)

```

definition *composition-fri-verifier-tied-final-value-replay-checked-conflict*

```

:: ('f, 'a) protocol-channel-scheme ⇒
  (unit list × ('f, 'a) protocol-channel-scheme) option ⇒ bool

```

where

```

composition-fri-verifier-tied-final-value-replay-checked-conflict s out
  ⟷
  (∃ trace-roots trace-bs trace-final fri-dg composition-roots
    composition-bs composition-final fri-query-idxs trace-round-layers
    composition-round-layers fr as trace-openings composition-openings
    trace-table composition-table round-idx v.
    accepted-fri-opening-transcript s out trace-roots trace-bs
      trace-final fri-dg composition-roots composition-bs
      composition-final fri-query-idxs trace-round-layers

```

$\text{composition-round-layers} \wedge$
 $\text{accepted-with-partial-initial-openings-aligned } s \text{ out fr trace-roots}$
 $\text{trace-final as fri-dg composition-roots composition-final fri-query-idxs}$
 $\text{trace-openings composition-openings} \wedge$
 $\text{partial-trace-table-candidate trace-table trace-openings} \wedge$
 $\text{partial-composition-table-candidate composition-table}$
 $\text{composition-openings} \wedge$
 $\text{trace-table-low-degree trace-table} \wedge$
 $\neg \text{composition-table-low-degree maxDegree composition-table} \wedge$
 $0 < \text{length composition-bs} \wedge$
 $\text{round-idx} < \text{length fri-query-idxs} \wedge$
 $\text{generic-fri-round-forced-next-value composition-roots composition-bs}$
 $\text{fri-query-idxs composition-round-layers round-idx}$
 $(\text{length composition-bs} - 1) v \wedge$
 $v \neq \text{composition-final} \wedge$
 $v = \text{composition-final})$

definition $\text{composition-fri-verifier-tied-final-value-replay-gap}$

$:: ('f, 'a) \text{protocol-channel-scheme} \Rightarrow$
 $(\text{unit list} \times ('f, 'a) \text{protocol-channel-scheme}) \text{option} \Rightarrow \text{bool}$

where

$\text{composition-fri-verifier-tied-final-value-replay-gap } s \text{ out} \longleftrightarrow$
 $\text{composition-fri-verifier-tied-sampled-final-value-conflict } s \text{ out} \wedge$
 $\neg \text{composition-fri-verifier-tied-final-value-replay-checked-conflict}$
 $s \text{ out}$

lemma $\text{composition-fri-verifier-tied-sampled-final-value-conflictE}$:

assumes

$\text{composition-fri-verifier-tied-sampled-final-value-conflict } s \text{ out}$

obtains $\text{trace-roots trace-bs trace-final fri-dg composition-roots}$

$\text{composition-bs composition-final fri-query-idxs trace-round-layers}$

$\text{composition-round-layers fr as trace-openings composition-openings}$

$\text{trace-table composition-table round-idx } v$

where

$\text{accepted-fri-opening-transcript } s \text{ out trace-roots trace-bs}$

$\text{trace-final fri-dg composition-roots composition-bs composition-final}$

$\text{fri-query-idxs trace-round-layers composition-round-layers}$

$\text{accepted-with-partial-initial-openings-aligned } s \text{ out fr trace-roots}$

$\text{trace-final as fri-dg composition-roots composition-final fri-query-idxs}$

$\text{trace-openings composition-openings}$

$\text{partial-trace-table-candidate trace-table trace-openings}$

$\text{partial-composition-table-candidate composition-table}$

$\text{composition-openings}$

$\text{trace-table-low-degree trace-table}$

$\neg \text{composition-table-low-degree maxDegree composition-table}$

$0 < \text{length composition-bs}$

$\text{round-idx} < \text{length fri-query-idxs}$

$\text{generic-fri-round-forced-next-value composition-roots composition-bs}$

$\text{fri-query-idxs composition-round-layers round-idx}$

$(\text{length } \text{composition-bs} - 1) \ v$
 $v \neq \text{composition-final}$
proof –
from *assms* **obtain** *trace-roots trace-bs trace-final fri-dg*
composition-roots composition-bs composition-final fri-query-idxs
trace-round-layers composition-round-layers fr as trace-openings
composition-openings trace-table composition-table **where**
fri-openings:
accepted-fri-opening-transcript s out trace-roots trace-bs
trace-final fri-dg composition-roots composition-bs
composition-final fri-query-idxs trace-round-layers
composition-round-layers
and *aligned:*
accepted-with-partial-initial-openings-aligned s out fr trace-roots
trace-final as fri-dg composition-roots composition-final
fri-query-idxs trace-openings composition-openings
and *trace-candidate:*
partial-trace-table-candidate trace-table trace-openings
and *composition-candidate:*
partial-composition-table-candidate composition-table
composition-openings
and *trace-low:* *trace-table-low-degree trace-table*
and *composition-nonlow:*
 $\neg$ *composition-table-low-degree maxDegree composition-table*
and *final-conflict:*
generic-fri-sampled-final-value-conflict composition-roots
composition-bs composition-final fri-query-idxs
composition-round-layers
unfolding *composition-fri-verifier-tied-sampled-final-value-conflict-def*
by *blast*
from *generic-fri-sampled-final-value-conflictE[OF final-conflict]*
obtain *round-idx v* **where**
nonempty: $0 < \text{length } \text{composition-bs}$
and *round-bound:* $\text{round-idx} < \text{length } \text{fri-query-idxs}$
and *forced:*
generic-fri-round-forced-next-value composition-roots composition-bs
fri-query-idxs composition-round-layers round-idx
 $(\text{length } \text{composition-bs} - 1) \ v$
and *neq:* $v \neq \text{composition-final}$
by *blast*
show *?thesis*
by (*rule that*[*OF fri-openings aligned trace-candidate*
composition-candidate trace-low composition-nonlow nonempty
round-bound forced neq])
qed

lemma *composition-fri-verifier-tied-final-value-replay-checked-conflict-false:*
 $\neg$ *composition-fri-verifier-tied-final-value-replay-checked-conflict s out*
unfolding *composition-fri-verifier-tied-final-value-replay-checked-conflict-def*

by blast

lemma *composition-fri-verifier-tied-sampled-final-value-imp-replay-gap*:
assumes *composition-fri-verifier-tied-sampled-final-value-conflict s out*
shows *composition-fri-verifier-tied-final-value-replay-gap s out*
using *assms composition-fri-verifier-tied-final-value-replay-checked-conflict-false*
unfolding *composition-fri-verifier-tied-final-value-replay-gap-def*
by *simp*

lemma *wp-composition-fri-verifier-tied-sampled-final-value-bound-from-replay-gap*:
fixes $G :: \text{prob}$
assumes *gap-bound*:
 wp-event verify-monad
 (composition-fri-verifier-tied-final-value-replay-gap s) s ≤ G
shows
 wp-event verify-monad
 (composition-fri-verifier-tied-sampled-final-value-conflict s) s ≤ G
proof –
have *wp-event verify-monad*
 (composition-fri-verifier-tied-sampled-final-value-conflict s) s ≤
 wp-event verify-monad
 (composition-fri-verifier-tied-final-value-replay-gap s) s
by (*rule wp-event-mono*)
 (*rule composition-fri-verifier-tied-sampled-final-value-imp-replay-gap*)
then show *?thesis*
by (*rule order-trans[OF gap-bound]*)
qed

definition *composition-fri-verifier-tied-final-value-authenticated-replay-gap*
 $:: ('f, 'a) \text{protocol-channel-scheme} \Rightarrow$
 $(\text{unit list} \times ('f, 'a) \text{protocol-channel-scheme}) \text{option} \Rightarrow \text{bool}$

where
composition-fri-verifier-tied-final-value-authenticated-replay-gap s out
 $\iff$
 $(\exists \text{trace-roots trace-bs trace-final fri-dg composition-roots}$
 composition-bs composition-final fri-query-idxs trace-round-layers
 composition-round-layers fr as trace-openings composition-openings
 trace-table composition-table result final-state round-idx v.
 out = Some (result, final-state) ∧
 accepted-fri-opening-transcript s out trace-roots trace-bs
 trace-final fri-dg composition-roots composition-bs
 composition-final fri-query-idxs trace-round-layers
 composition-round-layers ∧
 accepted-with-partial-initial-openings-aligned s out fr trace-roots
 trace-final as fri-dg composition-roots composition-final fri-query-idxs
 trace-openings composition-openings ∧
 partial-trace-table-candidate trace-table trace-openings ∧
 partial-composition-table-candidate composition-table
 composition-openings ∧

$\text{trace-table-low-degree trace-table} \wedge$
 $\neg \text{composition-table-low-degree maxDegree composition-table} \wedge$
 $0 < \text{length composition-bs} \wedge$
 $\text{round-idx} < \text{length fri-query-idxs} \wedge$
 $\text{generic-fri-round-forced-next-value composition-roots composition-bs}$
 $\text{fri-query-idxs composition-round-layers round-idx}$
 $(\text{length composition-bs} - 1) v \wedge$
 $v \neq \text{composition-final} \wedge$
 $\neg \text{partial-merkle-inconsistency-bad s out} \wedge$
 $\text{generic-fri-recorded-layer-chunk-authenticated composition-roots}$
 $\text{fri-query-idxs composition-round-layers final-state round-idx}$
 $(\text{length composition-bs} - 1))$

definition $\text{composition-fri-verifier-tied-final-value-selected-step-missing-gap}$

$:: ('f, 'a) \text{protocol-channel-scheme} \Rightarrow$
 $(\text{unit list} \times ('f, 'a) \text{protocol-channel-scheme}) \text{option} \Rightarrow \text{bool}$

where

$\text{composition-fri-verifier-tied-final-value-selected-step-missing-gap s out}$
 $\longleftrightarrow$
 $\text{composition-fri-verifier-tied-final-value-authenticated-replay-gap s out} \wedge$
 $(\exists \text{trace-roots trace-bs trace-final fri-dg composition-roots}$
 $\text{composition-bs composition-final fri-query-idxs trace-round-layers}$
 $\text{composition-round-layers result final-state round-idx } v.$
 $\text{out} = \text{Some (result, final-state)} \wedge$
 $\text{accepted-fri-opening-transcript s out trace-roots trace-bs}$
 $\text{trace-final fri-dg composition-roots composition-bs}$
 $\text{composition-final fri-query-idxs trace-round-layers}$
 $\text{composition-round-layers} \wedge$
 $0 < \text{length composition-bs} \wedge$
 $\text{round-idx} < \text{length fri-query-idxs} \wedge$
 $\text{generic-fri-round-forced-next-value composition-roots composition-bs}$
 $\text{fri-query-idxs composition-round-layers round-idx}$
 $(\text{length composition-bs} - 1) v \wedge$
 $v \neq \text{composition-final} \wedge$
 $\neg \text{partial-merkle-inconsistency-bad s out} \wedge$
 $\text{generic-fri-recorded-layer-chunk-authenticated composition-roots}$
 $\text{fri-query-idxs composition-round-layers final-state round-idx}$
 $(\text{length composition-bs} - 1) \wedge$
 $(\forall \text{actual-chunk axp axp-path axn axn-path.}$
 $\neg (\text{fri-layer-chunk-authenticated}$
 $(\text{composition-roots ! (length composition-bs - 1)})$
 $(\text{fri-evidence-layer-len composition-roots (length composition-bs - 1)})$
 $(\text{fri-evidence-layer-idx composition-roots fri-query-idxs round-idx}$
 $(\text{length composition-bs} - 1)))$
 $\text{actual-chunk final-state} \wedge$
 $\text{fri-layer-step-evidence}$
 $(\text{composition-roots ! (length composition-bs - 1)})$
 $(\text{composition-bs ! (length composition-bs - 1)})$
 $(\text{fri-evidence-layer-len composition-roots (length composition-bs - 1)}))$

```

(fri-evidence-layer-idx composition-roots fri-query-idxs round-idx
 (length composition-bs - 1))
(2 ^ (length composition-bs - 1))
(fri-sibling-index
 (fri-evidence-layer-len composition-roots (length composition-bs - 1))
 (fri-evidence-layer-idx composition-roots fri-query-idxs round-idx
 (length composition-bs - 1)))
axp axp-path axn axn-path
(fri-evidence-next-idx composition-roots fri-query-idxs round-idx
 (length composition-bs - 1))
composition-final actual-chunk)))

```

lemma *composition-fri-verifier-tied-final-value-selected-step-missing-gap-false-on-support:*

assumes *support:* $out \in \text{set-dist } (\text{execute verify-monad } s)$

and *gap:*

composition-fri-verifier-tied-final-value-selected-step-missing-gap
s out

shows *False*

proof —

from *gap* **obtain** *trace-roots trace-bs trace-final fri-dg composition-roots*
composition-bs composition-final fri-query-idxs trace-round-layers
composition-round-layers result final-state round-idx v

where *out-eq:* $out = \text{Some } (result, final-state)$

and *fri-openings:*

accepted-fri-opening-transcript s out trace-roots trace-bs
trace-final fri-dg composition-roots composition-bs
composition-final fri-query-idxs trace-round-layers
composition-round-layers

and *nonempty-bs:* $0 < \text{length } composition-bs$

and *round-bound:* $\text{round-idx} < \text{length } fri\text{-query-idxs}$

and *missing:*

$\bigwedge \text{actual-chunk } axp \ axp\text{-path } axn \ axn\text{-path}.$
 $\neg (fri\text{-layer-chunk-authenticated}$
 $(composition\text{-roots } ! (\text{length } composition\text{-bs} - 1))$
 $(fri\text{-evidence-layer-len } composition\text{-roots } (\text{length } composition\text{-bs} - 1))$
 $(fri\text{-evidence-layer-idx } composition\text{-roots } fri\text{-query-idxs } round\text{-idx}$
 $(\text{length } composition\text{-bs} - 1))$
 $actual\text{-chunk } final\text{-state} \wedge$
fri-layer-step-evidence
 $(composition\text{-roots } ! (\text{length } composition\text{-bs} - 1))$
 $(composition\text{-bs } ! (\text{length } composition\text{-bs} - 1))$
 $(fri\text{-evidence-layer-len } composition\text{-roots } (\text{length } composition\text{-bs} - 1))$
 $(fri\text{-evidence-layer-idx } composition\text{-roots } fri\text{-query-idxs } round\text{-idx}$
 $(\text{length } composition\text{-bs} - 1))$
 $(2 \wedge (\text{length } composition\text{-bs} - 1))$
 $(fri\text{-sibling-index}$
 $(fri\text{-evidence-layer-len } composition\text{-roots } (\text{length } composition\text{-bs} - 1))$
 $(fri\text{-evidence-layer-idx } composition\text{-roots } fri\text{-query-idxs } round\text{-idx}$
 $(\text{length } composition\text{-bs} - 1)))$

```

    arp arp-path arn arn-path
    (fri-evidence-next-idx composition-roots fri-query-idxs round-idx
      (length composition-bs - 1))
    composition-final actual-chunk)
unfolding
  composition-fri-verifier-tied-final-value-selected-step-missing-gap-def
by blast
have outcome:
  Some (result, final-state) ∈ set-dist (execute verify-monad s)
using support out-eq by simp
have challenges:
  accepted-fri-challenges s (Some (result, final-state))
  trace-bs fri-dg composition-bs
using accepted-fri-opening-transcript-challenges[OF fri-openings]
  out-eq by simp
have degree-bound: to-nat fri-dg ≤ maxDegree
by (rule verify-monad-accepted-fri-challenges-degree-bound
  [OF outcome challenges])
have len-bs-roots: length composition-bs = length composition-roots
using accepted-fri-opening-transcript-shapes(4)[OF fri-openings] .
have nonempty-roots: 0 < length composition-roots
using nonempty-bs len-bs-roots by simp
have round-bound-rounds: round-idx < rounds
using round-bound
  accepted-fri-opening-transcript-shapes(7)[OF fri-openings]
by simp
have raw-layer:
  ∧raw.
  0 < fri-layer-lengths (length composition-roots) (clength * scale) !
  (length composition-roots - 1) ∧
  fri-layer-indices (length composition-roots) (index (to-nat raw))
  (clength * scale) ! (length composition-roots - 1) <
  fri-layer-lengths (length composition-roots) (clength * scale) !
  (length composition-roots - 1)
by (rule accepted-fri-opening-transcript-composition-raw-layer-bound
  [OF fri-openings degree-bound])
  (use nonempty-roots in simp)
from accepted-fri-opening-transcript-composition-final-step-evidence-at
  [OF fri-openings out-eq round-bound-rounds nonempty-roots raw-layer]
obtain actual-chunk arp arp-path arn arn-path where
  auth:
  fri-layer-chunk-authenticated
  (composition-roots ! (length composition-roots - 1))
  (fri-evidence-layer-len composition-roots (length composition-roots - 1))
  (fri-evidence-layer-idx composition-roots fri-query-idxs round-idx
    (length composition-roots - 1))
  actual-chunk final-state
and step:
  fri-layer-step-evidence

```

```

    (composition-roots ! (length composition-roots - 1))
    (composition-bs ! (length composition-roots - 1))
    (fri-evidence-layer-len composition-roots (length composition-roots - 1))
    (fri-evidence-layer-idx composition-roots fri-query-idxs round-idx
      (length composition-roots - 1))
    (2 ^ (length composition-roots - 1))
    (fri-sibling-index
      (fri-evidence-layer-len composition-roots (length composition-roots - 1))
      (fri-evidence-layer-idx composition-roots fri-query-idxs round-idx
        (length composition-roots - 1)))
    arp arp-path arn arn-path
    (fri-evidence-next-idx composition-roots fri-query-idxs round-idx
      (length composition-roots - 1))
    composition-final actual-chunk
  by blast
show False
  using missing[of actual-chunk arp arp-path arn arn-path]
    auth step len-bs-roots
  by simp
qed

lemma wp-composition-fri-verifier-tied-final-value-selected-step-missing-gap-zero:
  shows
    wp-event verify-monad
      (composition-fri-verifier-tied-final-value-selected-step-missing-gap s)
      s ≤ (0::prob)
proof -
  have
    wp-event verify-monad
      (composition-fri-verifier-tied-final-value-selected-step-missing-gap s)
      s ≤ wp-event verify-monad (λ-. False) s
  by (rule wp-event-mono-on-support)
  (rule
    composition-fri-verifier-tied-final-value-selected-step-missing-gap-false-on-support)
  also have ... = 0
  unfolding wp-event-def wp-def dist-expect-def by simp
  finally show ?thesis by simp
qed

lemma composition-fri-verifier-tied-final-value-authenticated-replay-gap-imp-selected-step-missing:
  assumes
    composition-fri-verifier-tied-final-value-authenticated-replay-gap s out
  shows
    composition-fri-verifier-tied-final-value-selected-step-missing-gap s out
proof -
  from assms obtain trace-roots trace-bs trace-final fri-dg composition-roots
    composition-bs composition-final fri-query-idxs trace-round-layers
    composition-round-layers fr as trace-openings composition-openings
    trace-table composition-table result final-state round-idx v

```

```

where out-eq: out = Some (result, final-state)
and fri-openings:
  accepted-fri-opening-transcript s out trace-roots trace-bs
  trace-final fri-dg composition-roots composition-bs
  composition-final fri-query-idxs trace-round-layers
  composition-round-layers
and nonempty: 0 < length composition-bs
and round-bound: round-idx < length fri-query-idxs
and forced:
  generic-fri-round-forced-next-value composition-roots composition-bs
  fri-query-idxs composition-round-layers round-idx
  (length composition-bs - 1) v
and neg: v ≠ composition-final
and no-merkle: ¬ partial-merkle-inconsistency-bad s out
and recorded-auth:
  generic-fri-recorded-layer-chunk-authenticated composition-roots
  fri-query-idxs composition-round-layers final-state round-idx
  (length composition-bs - 1)
unfolding
  composition-fri-verifier-tied-final-value-authenticated-replay-gap-def
by blast
have no-step:
  ∧ actual-chunk arp arp-path arn arn-path.
  ¬ (fri-layer-chunk-authenticated
    (composition-roots ! (length composition-bs - 1))
    (fri-evidence-layer-len composition-roots (length composition-bs - 1))
    (fri-evidence-layer-idx composition-roots fri-query-idxs round-idx
      (length composition-bs - 1))
    actual-chunk final-state ∧
    fri-layer-step-evidence
    (composition-roots ! (length composition-bs - 1))
    (composition-bs ! (length composition-bs - 1))
    (fri-evidence-layer-len composition-roots (length composition-bs - 1))
    (fri-evidence-layer-idx composition-roots fri-query-idxs round-idx
      (length composition-bs - 1))
    (2 ^ (length composition-bs - 1))
    (fri-sibling-index
      (fri-evidence-layer-len composition-roots (length composition-bs - 1))
      (fri-evidence-layer-idx composition-roots fri-query-idxs round-idx
        (length composition-bs - 1)))
    arp arp-path arn arn-path
    (fri-evidence-next-idx composition-roots fri-query-idxs round-idx
      (length composition-bs - 1))
    composition-final actual-chunk)
proof
fix actual-chunk arp arp-path arn arn-path
assume step-auth:
  fri-layer-chunk-authenticated
    (composition-roots ! (length composition-bs - 1))

```

```

(fri-evidence-layer-len composition-roots (length composition-bs - 1))
(fri-evidence-layer-idx composition-roots fri-query-idxs round-idx
 (length composition-bs - 1))
actual-chunk final-state ∧
fri-layer-step-evidence
 (composition-roots ! (length composition-bs - 1))
 (composition-bs ! (length composition-bs - 1))
 (fri-evidence-layer-len composition-roots (length composition-bs - 1))
 (fri-evidence-layer-idx composition-roots fri-query-idxs round-idx
 (length composition-bs - 1))
 (2 ^ (length composition-bs - 1))
 (fri-sibling-index
 (fri-evidence-layer-len composition-roots (length composition-bs - 1))
 (fri-evidence-layer-idx composition-roots fri-query-idxs round-idx
 (length composition-bs - 1)))
arp arp-path arn arn-path
(fri-evidence-next-idx composition-roots fri-query-idxs round-idx
 (length composition-bs - 1))
composition-final actual-chunk
have False
  by (rule generic-fri-final-conflict-false-from-authenticated-step
    [OF forced recorded-auth conjunct1[OF step-auth]
      conjunct2[OF step-auth] no-merkle[unfolded out-eq] neq])
  then show False .
qed
have witness:
  ∃ trace-roots trace-bs trace-final fri-dg composition-roots
    composition-bs composition-final fri-query-idxs trace-round-layers
    composition-round-layers result final-state round-idx v.
  out = Some (result, final-state) ∧
  accepted-fri-opening-transcript s out trace-roots trace-bs
    trace-final fri-dg composition-roots composition-bs
    composition-final fri-query-idxs trace-round-layers
    composition-round-layers ∧
  0 < length composition-bs ∧
  round-idx < length fri-query-idxs ∧
  generic-fri-round-forced-next-value composition-roots composition-bs
    fri-query-idxs composition-round-layers round-idx
    (length composition-bs - 1) v ∧
  v ≠ composition-final ∧
  ¬ partial-merkle-inconsistency-bad s out ∧
  generic-fri-recorded-layer-chunk-authenticated composition-roots
    fri-query-idxs composition-round-layers final-state round-idx
    (length composition-bs - 1) ∧
  (∀ actual-chunk arp arp-path arn arn-path.
    ¬ (fri-layer-chunk-authenticated
      (composition-roots ! (length composition-bs - 1))
      (fri-evidence-layer-len composition-roots (length composition-bs - 1))
      (fri-evidence-layer-idx composition-roots fri-query-idxs round-idx

```



```

    (length composition-bs - 1))
  actual-chunk final-state ∧
  fri-layer-step-evidence
    (composition-roots ! (length composition-bs - 1))
    (composition-bs ! (length composition-bs - 1))
    (fri-evidence-layer-len composition-roots (length composition-bs - 1))
    (fri-evidence-layer-idx composition-roots fri-query-idxs round-idx
      (length composition-bs - 1))
    (2 ^ (length composition-bs - 1))
    (fri-sibling-index
      (fri-evidence-layer-len composition-roots (length composition-bs - 1))
      (fri-evidence-layer-idx composition-roots fri-query-idxs round-idx
        (length composition-bs - 1)))
    arp arp-path arn arn-path
    (fri-evidence-next-idx composition-roots fri-query-idxs round-idx
      (length composition-bs - 1))
    composition-final actual-chunk))
  using out-eq fri-openings nonempty round-bound forced neq no-merkle
    recorded-auth no-step
  by blast
show ?thesis
  unfolding
    composition-fri-verifier-tied-final-value-selected-step-missing-gap-def
  using assms witness by blast
qed

lemma wp-composition-fri-verifier-tied-final-value-authenticated-replay-gap-bound-from-selected-step-missing:
  fixes G :: prob
  assumes gap-bound:
    wp-event verify-monad
      (composition-fri-verifier-tied-final-value-selected-step-missing-gap s)
    s ≤ G
  shows
    wp-event verify-monad
      (composition-fri-verifier-tied-final-value-authenticated-replay-gap s)
    s ≤ G
proof -
  have wp-event verify-monad
    (composition-fri-verifier-tied-final-value-authenticated-replay-gap s)
    s ≤
    wp-event verify-monad
      (composition-fri-verifier-tied-final-value-selected-step-missing-gap s)
    s
  by (rule wp-event-mono)
  (rule
    composition-fri-verifier-tied-final-value-authenticated-replay-gap-imp-selected-step-missing)
  then show ?thesis
  by (rule order-trans[OF - gap-bound])
qed

```

lemma *composition-fri-verifier-tied-final-value-replay-gap-imp-merkle-or-slot-or-authenticated-on-support*:
assumes *support*: $out \in \text{set-dist } (\text{execute verify-monad } s)$
and *gap*: *composition-fri-verifier-tied-final-value-replay-gap* *s out*
shows
partial-merkle-inconsistency-bad *s out* $\vee$
composition-fri-verifier-tied-authenticated-slot-recorded-chunk-gap
s out $\vee$
composition-fri-verifier-tied-final-value-authenticated-replay-gap *s out*
proof –
from *gap* **have** *sampled*:
composition-fri-verifier-tied-sampled-final-value-conflict *s out*
unfolding *composition-fri-verifier-tied-final-value-replay-gap-def*
by *simp*
from *composition-fri-verifier-tied-sampled-final-value-conflictE* [*OF* *sampled*]
obtain *trace-roots trace-bs trace-final fri-dg composition-roots*
composition-bs composition-final fri-query-idxs trace-round-layers
composition-round-layers fr as trace-openings composition-openings
trace-table composition-table round-idx v
where *fri-openings*:
accepted-fri-opening-transcript *s out trace-roots trace-bs*
trace-final fri-dg composition-roots composition-bs
composition-final fri-query-idxs trace-round-layers
composition-round-layers
and *aligned*:
accepted-with-partial-initial-openings-aligned *s out fr trace-roots*
trace-final as fri-dg composition-roots composition-final
fri-query-idxs trace-openings composition-openings
and *trace-cand*: *partial-trace-table-candidate trace-table trace-openings*
and *comp-cand*:
partial-composition-table-candidate composition-table
composition-openings
and *trace-low*: *trace-table-low-degree trace-table*
and *comp-not-low*:
 $\neg$ *composition-table-low-degree maxDegree composition-table*
and *nonempty*: $0 < \text{length } \text{composition-bs}$
and *round-bound*: $\text{round-idx} < \text{length } \text{fri-query-idxs}$
and *forced*:
generic-fri-round-forced-next-value composition-roots composition-bs
fri-query-idxs composition-round-layers round-idx
 $(\text{length } \text{composition-bs} - 1) v$
and *neq*: $v \neq \text{composition-final}$
by *blast*
from *fri-openings* **obtain** *result final-state* **where** *out-eq*:
 $out = \text{Some } (\text{result}, \text{final-state})$
unfolding *accepted-fri-opening-transcript-def* **by** *blast*
show *?thesis*
proof (*cases partial-merkle-inconsistency-bad s out*)
case *True*

```

then show ?thesis by simp
next
case no-merkle: False
let ?j = length composition-bs - 1
have len-composition-bs: length composition-bs = length composition-roots
  using accepted-fri-opening-transcript-shapes(4)[OF fri-openings] .
have layer-bound: ?j < length composition-roots
  using nonempty len-composition-bs by simp
show ?thesis
proof (cases
  generic-fri-recorded-layer-chunk-authenticated composition-roots
  fri-query-idxs composition-round-layers final-state round-idx ?j)
case True
have authenticated-gap:
  composition-fri-verifier-tied-final-value-authenticated-replay-gap
  s out
unfolding
  composition-fri-verifier-tied-final-value-authenticated-replay-gap-def
by (intro exI conjI)
  (rule out-eq, rule fri-openings, rule aligned, rule trace-cand,
   rule comp-cand, rule trace-low, rule comp-not-low, rule nonempty,
   rule round-bound, rule forced, rule neq,
   use no-merkle True in simp-all)
then show ?thesis by simp
next
case missing-recorded: False
have outcome:
  Some (result, final-state) ∈ set-dist (execute verify-monad s)
  using support out-eq by simp
have challenges:
  accepted-fri-challenges s (Some (result, final-state))
  trace-bs fri-dg composition-bs
  using accepted-fri-opening-transcript-challenges[OF fri-openings]
  out-eq by simp
have degree-bound: to-nat fri-dg ≤ maxDegree
  by (rule verify-monad-accepted-fri-challenges-degree-bound
  [OF outcome challenges])
have round-bound-rounds: round-idx < rounds
  using round-bound
  accepted-fri-opening-transcript-shapes(7)[OF fri-openings]
  by simp
have raw-layer:
   $\bigwedge$  raw.
  0 < fri-layer-lengths (length composition-roots)
  (clength * scale) ! ?j ∧
  fri-layer-indices (length composition-roots) (index (to-nat raw))
  (clength * scale) ! ?j <
  fri-layer-lengths (length composition-roots)
  (clength * scale) ! ?j

```

```

    by (rule accepted-fri-opening-transcript-composition-raw-layer-bound
        [OF fri-openings degree-bound layer-bound])
  from accepted-fri-opening-transcript-composition-selected-chunk-authenticated-at
    [OF fri-openings out-eq round-bound-rounds layer-bound raw-layer]
  obtain auth-chunk where auth:
    fri-layer-chunk-authenticated (composition-roots ! ?j)
    (fri-layer-lengths (length composition-roots) (clength * scale) ! ?j)
    (fri-layer-indices (length composition-roots)
      (fri-query-idxs ! round-idx) (clength * scale) ! ?j)
    auth-chunk final-state
  by blast
  have step:
    fri-round-layer-evidence composition-roots composition-bs
    fri-query-idxs round-idx ?j composition-round-layers
  by (rule accepted-fri-opening-transcript-composition-round-layer-evidence
      [OF fri-openings round-bound-rounds layer-bound])
  have generic-step:
    generic-fri-transcript-step-with-authenticated-chunk
    composition-roots composition-bs fri-query-idxs
    composition-round-layers final-state round-idx ?j
  by (rule generic-fri-transcript-step-with-authenticated-chunkI
      [OF step])
    (use auth in
      ⟨simp add: fri-evidence-layer-len-def
        fri-evidence-layer-idx-def⟩)
  have slot-gap:
    generic-fri-authenticated-slot-recorded-chunk-gap
    composition-roots fri-query-idxs composition-round-layers
    final-state
  by (rule
    generic-fri-transcript-step-auth-missing-imp-authenticated-slot-recorded-chunk-gap
    [OF generic-step missing-recorded])
  have route-slot-gap:
    composition-fri-verifier-tied-authenticated-slot-recorded-chunk-gap
    s out
  unfolding
    composition-fri-verifier-tied-authenticated-slot-recorded-chunk-gap-def
  by (intro exI conjI)
    (rule out-eq, rule fri-openings, rule aligned, rule trace-cand,
      rule comp-cand, rule trace-low, rule comp-not-low, rule slot-gap)
  then show ?thesis by simp
qed
qed
qed

lemma wp-composition-fri-verifier-tied-final-value-replay-gap-bound-from-merkle-slot-and-authenticated:
  fixes M A R :: prob
  assumes merkle-bound:
    wp-event verify-monad (partial-merkle-inconsistency-bad s) s ≤ M

```

```

and slot-bound:
  wp-event verify-monad
    (composition-fri-verifier-tied-authenticated-slot-recorded-chunk-gap s)
     $s \leq A$ 
and replay-bound:
  wp-event verify-monad
    (composition-fri-verifier-tied-final-value-authenticated-replay-gap s)
     $s \leq R$ 
shows
  wp-event verify-monad
    (composition-fri-verifier-tied-final-value-replay-gap s) s
     $\leq M + A + R$ 
proof –
  let ?P = partial-merkle-inconsistency-bad s
  let ?Q = composition-fri-verifier-tied-authenticated-slot-recorded-chunk-gap s
  let ?R =
    composition-fri-verifier-tied-final-value-authenticated-replay-gap s
  have event-le:
    wp-event verify-monad
      (composition-fri-verifier-tied-final-value-replay-gap s)  $s \leq$ 
      wp-event verify-monad ( $\lambda out. ?P\ out \vee ?Q\ out \vee ?R\ out$ ) s
    by (rule wp-event-mono-on-support)
      (rule
        composition-fri-verifier-tied-final-value-replay-gap-imp-merkle-or-slot-or-authenticated-on-support)
  have union-le:
    wp-event verify-monad ( $\lambda out. ?P\ out \vee ?Q\ out \vee ?R\ out$ )  $s \leq$ 
      wp-event verify-monad ?P s +
      wp-event verify-monad ?Q s +
      wp-event verify-monad ?R s
  proof –
    have wp-event verify-monad ( $\lambda out. ?P\ out \vee ?Q\ out \vee ?R\ out$ )  $s \leq$ 
      wp-event verify-monad ?P s +
      wp-event verify-monad ( $\lambda out. ?Q\ out \vee ?R\ out$ ) s
    by (rule wp-event-union-bound)
    also have ...  $\leq$ 
      wp-event verify-monad ?P s +
      (wp-event verify-monad ?Q s + wp-event verify-monad ?R s)
    by (intro add-mono order.refl wp-event-union-bound)
    finally show ?thesis
      by (simp add: algebra-simps)
  qed
have bounds:
  wp-event verify-monad ?P s +
  wp-event verify-monad ?Q s +
  wp-event verify-monad ?R s  $\leq M + A + R$ 
using merkle-bound slot-bound replay-bound
by (intro add-mono)
show ?thesis
  by (rule order-trans[OF event-le order-trans[OF union-le bounds]])

```

qed

lemma *wp-composition-fri-verifier-tied-final-value-replay-gap-bound-from-merkle-slot-and-selected-step*:
fixes $M A R :: \text{prob}$
assumes *merkle-bound*:
 $\text{wp-event verify-monad } (\text{partial-merkle-inconsistency-bad } s) \ s \leq M$
and *slot-bound*:
 $\text{wp-event verify-monad}$
 $(\text{composition-fri-verifier-tied-authenticated-slot-recorded-chunk-gap } s)$
 $s \leq A$
and *selected-step-bound*:
 $\text{wp-event verify-monad}$
 $(\text{composition-fri-verifier-tied-final-value-selected-step-missing-gap } s)$
 $s \leq R$
shows
 $\text{wp-event verify-monad}$
 $(\text{composition-fri-verifier-tied-final-value-replay-gap } s) \ s$
 $\leq M + A + R$
proof –
have *authenticated-bound*:
 $\text{wp-event verify-monad}$
 $(\text{composition-fri-verifier-tied-final-value-authenticated-replay-gap } s)$
 $s \leq R$
by (rule
 $\text{wp-composition-fri-verifier-tied-final-value-authenticated-replay-gap-bound-from-selected-step-missing}$
 $[\text{OF selected-step-bound}]$)
show ?thesis
by (rule
 $\text{wp-composition-fri-verifier-tied-final-value-replay-gap-bound-from-merkle-slot-and-authenticated}$
 $(\text{rule merkle-bound, rule slot-bound, rule authenticated-bound})$)
qed

lemma *wp-composition-fri-verifier-tied-authenticated-conflict-bound-from-recorded-missing-next-successor-and-final-g*:
fixes $P M N S G :: \text{prob}$
assumes *partial-merkle-bound*:
 $\text{wp-event verify-monad } (\text{partial-merkle-inconsistency-bad } s) \ s \leq P$
and *missing-bound*:
 $\text{wp-event verify-monad}$
 $(\text{composition-fri-verifier-tied-recorded-base-chunk-auth-missing } s)$
 $s \leq M$
and *next-bound*:
 $\text{wp-event verify-monad}$
 $(\text{composition-fri-verifier-tied-sampled-next-value-conflict } s) \ s \leq N$
and *successor-bound*:
 $\text{wp-event verify-monad}$
 $(\text{composition-fri-verifier-tied-sampled-successor-opening-conflict } s)$
 $s \leq S$
and *final-gap-bound*:
 $\text{wp-event verify-monad}$

$(\text{composition-fri-verifier-tied-final-value-replay-gap } s) \ s \leq G$
shows
 $\text{wp-event verify-monad}$
 $(\text{composition-fri-verifier-tied-sampled-assignment-conflict-with-authenticated-same-layer } s)$
 $s \leq (P + M) + N + S + G + P$
by (rule
 $\text{wp-composition-fri-verifier-tied-authenticated-conflict-bound-from-recorded-missing-and-branches})$
(rule $\text{partial-merkle-bound}$,
rule missing-bound ,
rule next-bound ,
rule successor-bound ,
rule $\text{wp-composition-fri-verifier-tied-sampled-final-value-bound-from-replay-gap}$
 $[\text{OF final-gap-bound}]$)

lemma $\text{wp-composition-fri-verifier-tied-authenticated-conflict-bound-from-all-replay-gaps}$:

fixes $P \ M \ N \ S \ G :: \text{prob}$

assumes $\text{partial-merkle-bound}$:

$\text{wp-event verify-monad } (\text{partial-merkle-inconsistency-bad } s) \ s \leq P$

and missing-bound :

$\text{wp-event verify-monad}$
 $(\text{composition-fri-verifier-tied-recorded-base-chunk-auth-missing } s)$
 $s \leq M$

and next-gap-bound :

$\text{wp-event verify-monad}$
 $(\text{composition-fri-verifier-tied-next-value-replay-gap } s) \ s \leq N$

and $\text{successor-gap-bound}$:

$\text{wp-event verify-monad}$
 $(\text{composition-fri-verifier-tied-successor-replay-gap } s) \ s \leq S$

and final-gap-bound :

$\text{wp-event verify-monad}$
 $(\text{composition-fri-verifier-tied-final-value-replay-gap } s) \ s \leq G$

shows

$\text{wp-event verify-monad}$
 $(\text{composition-fri-verifier-tied-sampled-assignment-conflict-with-authenticated-same-layer } s)$
 $s \leq (P + M) + N + S + G + P$

by (rule

$\text{wp-composition-fri-verifier-tied-authenticated-conflict-bound-from-recorded-missing-next-successor-and-final-gap})$
(rule $\text{partial-merkle-bound}$,
rule missing-bound ,
rule $\text{wp-composition-fri-verifier-tied-sampled-next-value-bound-from-replay-gap}$
 $[\text{OF next-gap-bound}]$,
rule $\text{wp-composition-fri-verifier-tied-sampled-successor-bound-from-replay-gap}$
 $[\text{OF successor-gap-bound}]$,
rule final-gap-bound)

definition $\text{composition-fri-verifier-tied-assignment-auth-gap}$

$:: ('f, 'a) \text{protocol-channel-scheme} \Rightarrow$

$(\text{unit list} \times ('f, 'a) \text{protocol-channel-scheme}) \text{option} \Rightarrow \text{bool}$

where

$\text{composition-fri-verifier-tied-assignment-auth-gap } s \text{ out} \longleftrightarrow$
 $\text{composition-fri-verifier-tied-sampled-assignment-conflict } s \text{ out} \wedge$
 $\neg \text{composition-fri-verifier-tied-sampled-assignment-conflict-with-authenticated-same-layer}$
 $s \text{ out}$

definition *composition-fri-verifier-tied-same-layer-auth-gap*

$:: ('f, 'a) \text{ protocol-channel-scheme} \Rightarrow$
 $(\text{unit list} \times ('f, 'a) \text{ protocol-channel-scheme}) \text{ option} \Rightarrow \text{bool}$

where

$\text{composition-fri-verifier-tied-same-layer-auth-gap } s \text{ out} \longleftrightarrow$
 $(\exists \text{ trace-roots trace-bs trace-final fri-dg composition-roots}$
 $\text{composition-bs composition-final fri-query-idxs trace-round-layers}$
 $\text{composition-round-layers fr as trace-openings composition-openings}$
 $\text{trace-table composition-table result final-state.}$
 $\text{out} = \text{Some (result, final-state)} \wedge$
 $\text{accepted-fri-opening-transcript } s \text{ out trace-roots trace-bs}$
 $\text{trace-final fri-dg composition-roots composition-bs}$
 $\text{composition-final fri-query-idxs trace-round-layers}$
 $\text{composition-round-layers} \wedge$
 $\text{accepted-with-partial-initial-openings-aligned } s \text{ out fr trace-roots}$
 $\text{trace-final as fri-dg composition-roots composition-final}$
 $\text{fri-query-idxs trace-openings composition-openings} \wedge$
 $\text{partial-trace-table-candidate trace-table trace-openings} \wedge$
 $\text{partial-composition-table-candidate composition-table}$
 $\text{composition-openings} \wedge$
 $\text{trace-table-low-degree trace-table} \wedge$
 $\neg \text{composition-table-low-degree maxDegree composition-table} \wedge$
 $\text{generic-fri-sampled-same-layer-opening-conflict composition-roots}$
 $\text{composition-bs fri-query-idxs composition-round-layers} \wedge$
 $\neg \text{generic-fri-sampled-same-layer-opening-conflict-with-authenticated-chunks}$
 $\text{composition-roots composition-bs fri-query-idxs composition-round-layers}$
 $\text{final-state})$

definition *composition-fri-verifier-tied-same-layer-recorded-auth-gap*

$:: ('f, 'a) \text{ protocol-channel-scheme} \Rightarrow$
 $(\text{unit list} \times ('f, 'a) \text{ protocol-channel-scheme}) \text{ option} \Rightarrow \text{bool}$

where

$\text{composition-fri-verifier-tied-same-layer-recorded-auth-gap } s \text{ out} \longleftrightarrow$
 $(\exists \text{ trace-roots trace-bs trace-final fri-dg composition-roots}$
 $\text{composition-bs composition-final fri-query-idxs trace-round-layers}$
 $\text{composition-round-layers fr as trace-openings composition-openings}$
 $\text{trace-table composition-table result final-state.}$
 $\text{out} = \text{Some (result, final-state)} \wedge$
 $\text{accepted-fri-opening-transcript } s \text{ out trace-roots trace-bs}$
 $\text{trace-final fri-dg composition-roots composition-bs}$
 $\text{composition-final fri-query-idxs trace-round-layers}$
 $\text{composition-round-layers} \wedge$
 $\text{accepted-with-partial-initial-openings-aligned } s \text{ out fr trace-roots}$
 $\text{trace-final as fri-dg composition-roots composition-final}$

fri-query-idxs trace-openings composition-openings $\wedge$
partial-trace-table-candidate trace-table trace-openings $\wedge$
partial-composition-table-candidate composition-table
composition-openings $\wedge$
trace-table-low-degree trace-table $\wedge$
 $\neg$ *composition-table-low-degree maxDegree composition-table $\wedge$*
generic-fri-sampled-same-layer-recorded-auth-gap composition-roots
composition-bs fri-query-idxs composition-round-layers final-state)

definition *composition-fri-verifier-tied-same-layer-authenticated-recorded-auth-gap*

$:: (f, 'a) \text{ protocol-channel-scheme} \Rightarrow$
 $(\text{unit list} \times (f, 'a) \text{ protocol-channel-scheme}) \text{ option} \Rightarrow \text{bool}$

where

composition-fri-verifier-tied-same-layer-authenticated-recorded-auth-gap
 $s \text{ out} \longleftrightarrow$
 $(\exists \text{ trace-roots trace-bs trace-final fri-dg composition-roots}$
 $\text{composition-bs composition-final fri-query-idxs trace-round-layers}$
 $\text{composition-round-layers fr as trace-openings composition-openings}$
 $\text{trace-table composition-table result final-state.}$
 $\text{out} = \text{Some (result, final-state)} \wedge$
 $\text{accepted-fri-opening-transcript } s \text{ out trace-roots trace-bs}$
 $\text{trace-final fri-dg composition-roots composition-bs}$
 $\text{composition-final fri-query-idxs trace-round-layers}$
 $\text{composition-round-layers} \wedge$
 $\text{accepted-with-partial-initial-openings-aligned } s \text{ out fr trace-roots}$
 $\text{trace-final as fri-dg composition-roots composition-final}$
 $\text{fri-query-idxs trace-openings composition-openings} \wedge$
 $\text{partial-trace-table-candidate trace-table trace-openings} \wedge$
 $\text{partial-composition-table-candidate composition-table}$
 $\text{composition-openings} \wedge$
 $\text{trace-table-low-degree trace-table} \wedge$
 $\neg \text{composition-table-low-degree maxDegree composition-table} \wedge$
 $\text{generic-fri-sampled-same-layer-authenticated-recorded-auth-gap}$
 $\text{composition-roots composition-bs fri-query-idxs composition-round-layers}$
 $\text{final-state})$

lemma *composition-fri-verifier-tied-authenticated-slot-recorded-chunk-gap-false-on-support:*

assumes *support: out $\in$ set-dist (execute verify-monad s)*

and event:

composition-fri-verifier-tied-authenticated-slot-recorded-chunk-gap
 $s \text{ out}$

shows *False*

proof —

from event obtain *trace-roots trace-bs trace-final fri-dg*
 $\text{composition-roots composition-bs composition-final fri-query-idxs}$
 $\text{trace-round-layers composition-round-layers fr as trace-openings}$
 $\text{composition-openings trace-table composition-table result final-state}$
where *out-eq: out = Some (result, final-state)*
and *fri-openings:*

```

    accepted-fri-opening-transcript s out trace-roots trace-bs
    trace-final fri-dg composition-roots composition-bs
    composition-final fri-query-idxs trace-round-layers
    composition-round-layers
  and gap:
    generic-fri-authenticated-slot-recorded-chunk-gap composition-roots
    fri-query-idxs composition-round-layers final-state
  unfolding
    composition-fri-verifier-tied-authenticated-slot-recorded-chunk-gap-def
  by blast
  from gap obtain round-idx layer-idx auth-chunk where
    round-bound: round-idx < length fri-query-idxs
  and layer-bound: layer-idx < length composition-roots
  and missing:
    ¬ generic-fri-recorded-layer-chunk-authenticated composition-roots
    fri-query-idxs composition-round-layers final-state round-idx
    layer-idx
  unfolding generic-fri-authenticated-slot-recorded-chunk-gap-def
  by blast
  have support-some:
    Some (result, final-state) ∈ set-dist (execute verify-monad s)
  using support out-eq by simp
  have challenges:
    accepted-fri-challenges s (Some (result, final-state))
    trace-bs fri-dg composition-bs
  using accepted-fri-opening-transcript-challenges[OF fri-openings]
    out-eq by simp
  have degree-bound: to-nat fri-dg ≤ maxDegree
  by (rule verify-monad-accepted-fri-challenges-degree-bound
    [OF support-some challenges])
  have round-bound-rounds: round-idx < rounds
  using round-bound accepted-fri-opening-transcript-shapes(7)[OF fri-openings]
  by simp
  have all-layer-bounds:
    ∧ raw k. k < length composition-roots ⇒
    0 < fri-layer-lengths (length composition-roots) (clength * scale) ! k ∧
    fri-layer-indices (length composition-roots) (index (to-nat raw))
    (clength * scale) ! k <
    fri-layer-lengths (length composition-roots) (clength * scale) ! k
  by (rule accepted-fri-opening-transcript-composition-raw-layer-bound
    [OF fri-openings degree-bound])
  have recorded:
    generic-fri-recorded-layer-chunk-authenticated composition-roots
    fri-query-idxs composition-round-layers final-state round-idx layer-idx
  by (rule
    accepted-fri-opening-transcript-composition-recorded-layer-chunk-authenticated-at
    [OF fri-openings out-eq round-bound-rounds layer-bound
    all-layer-bounds])
  show False

```

using *missing recorded* by *simp*
 qed

lemma *wp-composition-fri-verifier-tied-authenticated-slot-recorded-chunk-gap-zero*:
 shows
 wp-event verify-monad
 (*composition-fri-verifier-tied-authenticated-slot-recorded-chunk-gap* *s*)
 s ≤ (*0::prob*)
proof –
 have
 wp-event verify-monad
 (*composition-fri-verifier-tied-authenticated-slot-recorded-chunk-gap* *s*)
 s ≤ *wp-event verify-monad* (λ -. *False*) *s*
 by (rule *wp-event-mono-on-support*)
 (rule
 composition-fri-verifier-tied-authenticated-slot-recorded-chunk-gap-false-on-support)
 also have ... = 0
 unfolding *wp-event-def wp-def dist-expect-def* by *simp*
 finally show ?thesis by *simp*
 qed

lemma *composition-fri-verifier-tied-recorded-base-chunk-auth-missing-imp-slot-recorded-chunk-gap*:
 assumes
 composition-fri-verifier-tied-recorded-base-chunk-auth-missing *s out*
 shows
 composition-fri-verifier-tied-authenticated-slot-recorded-chunk-gap *s out*
proof –
 from *assms* obtain *trace-roots trace-bs trace-final fri-dg*
 composition-roots composition-bs composition-final fri-query-idxs
 trace-round-layers composition-round-layers fr as trace-openings
 composition-openings trace-table composition-table round-idx xp xp-path
 xn xn-path result final-state
 where *out-eq*: *out* = *Some* (*result*, *final-state*)
 and *fri-openings*:
 accepted-fri-opening-transcript *s out trace-roots trace-bs*
 trace-final fri-dg composition-roots composition-bs
 composition-final fri-query-idxs trace-round-layers
 composition-round-layers
 and *aligned*:
 accepted-with-partial-initial-openings-aligned *s out fr trace-roots*
 trace-final as fri-dg composition-roots composition-final
 fri-query-idxs trace-openings composition-openings
 and *trace-cand*:
 partial-trace-table-candidate trace-table trace-openings
 and *comp-cand*:
 partial-composition-table-candidate composition-table
 composition-openings
 and *trace-low*: *trace-table-low-degree trace-table*
 and *comp-not-low*:

```

  ¬ composition-table-low-degree maxDegree composition-table
and round-bound: round-idx < length fri-query-idxs
and composition-bs-nonempty: 0 < length composition-bs
and missing:
  ¬ fri-layer-chunk-authenticated (composition-roots ! 0)
    (fri-evidence-layer-len composition-roots 0)
    (fri-evidence-layer-idx composition-roots fri-query-idxs round-idx 0)
    (composition-round-layers ! round-idx ! 0) final-state
unfolding
  composition-fri-verifier-tied-recorded-base-chunk-auth-missing-def
by blast
have len-composition-bs:
  length composition-bs = length composition-roots
  using accepted-fri-opening-transcript-shapes(4)[OF fri-openings] .
have layer-bound: 0 < length composition-roots
  using composition-bs-nonempty len-composition-bs by simp
have len-query: length fri-query-idxs = rounds
  using accepted-fri-opening-transcript-shapes(7)[OF fri-openings] .
have round-bound-rounds: round-idx < rounds
  using round-bound len-query by simp
have raw-layer:
  ∧ raw.
  0 < fri-layer-lengths (length composition-roots) (clength * scale) ! 0 ∧
  fri-layer-indices (length composition-roots) (index (to-nat raw))
    (clength * scale) ! 0 <
  fri-layer-lengths (length composition-roots) (clength * scale) ! 0
  by (rule fri-first-layer-raw-bound[OF layer-bound])
have auth-step:
  generic-fri-transcript-step-with-authenticated-chunk composition-roots
    composition-bs fri-query-idxs composition-round-layers final-state
    round-idx 0
  by (rule
    accepted-fri-opening-transcript-composition-step-with-authenticated-chunk-at
    [OF fri-openings out-eq round-bound-rounds layer-bound raw-layer])
have recorded-missing:
  ¬ generic-fri-recorded-layer-chunk-authenticated composition-roots
    fri-query-idxs composition-round-layers final-state round-idx 0
  using missing
  unfolding generic-fri-recorded-layer-chunk-authenticated-def
  by simp
have slot-gap:
  generic-fri-authenticated-slot-recorded-chunk-gap composition-roots
    fri-query-idxs composition-round-layers final-state
  by (rule
    generic-fri-transcript-step-auth-missing-imp-authenticated-slot-recorded-chunk-gap
    [OF auth-step recorded-missing])
show ?thesis
unfolding
  composition-fri-verifier-tied-authenticated-slot-recorded-chunk-gap-def

```

```

    by (intro exI conjI)
      (rule out-eq, rule fri-openings, rule aligned, rule trace-cand,
       rule comp-cand, rule trace-low, rule comp-not-low, rule slot-gap)
qed

lemma wp-composition-fri-verifier-tied-recorded-base-chunk-auth-missing-bound-from-slot-recorded-chunk-gap:
  fixes A :: prob
  assumes slot-gap-bound:
    wp-event verify-monad
      (composition-fri-verifier-tied-authenticated-slot-recorded-chunk-gap s)
    s ≤ A
  shows
    wp-event verify-monad
      (composition-fri-verifier-tied-recorded-base-chunk-auth-missing s)
    s ≤ A
proof -
  have wp-event verify-monad
    (composition-fri-verifier-tied-recorded-base-chunk-auth-missing s)
    s ≤
    wp-event verify-monad
      (composition-fri-verifier-tied-authenticated-slot-recorded-chunk-gap s)
    s
  by (rule wp-event-mono)
    (rule
      composition-fri-verifier-tied-recorded-base-chunk-auth-missing-imp-slot-recorded-chunk-gap)
  then show ?thesis
    by (rule order-trans[OF slot-gap-bound])
qed

lemma composition-fri-verifier-tied-assignment-imp-authenticated-or-auth-gap:
  assumes composition-fri-verifier-tied-sampled-assignment-conflict s out
  shows
    composition-fri-verifier-tied-sampled-assignment-conflict-with-authenticated-same-layer
      s out ∨
    composition-fri-verifier-tied-assignment-auth-gap s out
  using assms
  unfolding composition-fri-verifier-tied-assignment-auth-gap-def
  by blast

lemma composition-fri-verifier-tied-assignment-auth-gap-imp-same-layer-auth-gap:
  assumes composition-fri-verifier-tied-assignment-auth-gap s out
  shows composition-fri-verifier-tied-same-layer-auth-gap s out
proof -
  from assms have conflict:
    composition-fri-verifier-tied-sampled-assignment-conflict s out
  and no-auth:
    ¬ composition-fri-verifier-tied-sampled-assignment-conflict-with-authenticated-same-layer
      s out
  unfolding composition-fri-verifier-tied-assignment-auth-gap-def

```

by *simp-all*
from *conflict* **obtain** *trace-roots trace-bs trace-final fri-dg*
composition-roots composition-bs composition-final fri-query-idxs
trace-round-layers composition-round-layers fr as trace-openings
composition-openings trace-table composition-table
where *fri-openings*:
accepted-fri-opening-transcript s out trace-roots trace-bs
trace-final fri-dg composition-roots composition-bs
composition-final fri-query-idxs trace-round-layers
composition-round-layers
and *aligned*:
accepted-with-partial-initial-openings-aligned s out fr trace-roots
trace-final as fri-dg composition-roots composition-final
fri-query-idxs trace-openings composition-openings
and *trace-cand*:
partial-trace-table-candidate trace-table trace-openings
and *comp-cand*:
partial-composition-table-candidate composition-table
composition-openings
and *trace-low*: *trace-table-low-degree trace-table*
and *comp-not-low*:
 $\neg$ *composition-table-low-degree maxDegree composition-table*
and *sampled*:
generic-fri-sampled-assignment-conflict composition-table
composition-roots composition-bs composition-final fri-query-idxs
composition-round-layers
unfolding
composition-fri-verifier-tied-sampled-assignment-conflict-def
by *blast*
from *fri-openings* **obtain** *result final-state* **where** *out-eq*:
out = Some (result, final-state)
unfolding *accepted-fri-opening-transcript-def* **by** *blast*
have *split*:
generic-fri-sampled-assignment-conflict-with-authenticated-same-layer
composition-table composition-roots composition-bs composition-final
fri-query-idxs composition-round-layers final-state $\vee$
(generic-fri-sampled-same-layer-opening-conflict composition-roots
composition-bs fri-query-idxs composition-round-layers $\wedge$
 $\neg$ *generic-fri-sampled-same-layer-opening-conflict-with-authenticated-chunks*
composition-roots composition-bs fri-query-idxs composition-round-layers
final-state)
by (*rule*
generic-fri-sampled-assignment-conflict-imp-authenticated-or-same-layer-auth-gap
[OF sampled])
then show *?thesis*
proof
assume *auth*:
generic-fri-sampled-assignment-conflict-with-authenticated-same-layer
composition-table composition-roots composition-bs composition-final

```

    fri-query-idxs composition-round-layers final-state
have
  composition-fri-verifier-tied-sampled-assignment-conflict-with-authenticated-same-layer
    s out
unfolding
  composition-fri-verifier-tied-sampled-assignment-conflict-with-authenticated-same-layer-def
by (intro exI conjI)
  (rule fri-openings, rule out-eq, rule aligned, rule trace-cand,
    rule comp-cand, rule trace-low, rule comp-not-low, rule auth)
then show ?thesis
  using no-auth by contradiction
next
assume gap:
  generic-fri-sampled-same-layer-opening-conflict composition-roots
    composition-bs fri-query-idxs composition-round-layers  $\wedge$ 
   $\neg$  generic-fri-sampled-same-layer-opening-conflict-with-authenticated-chunks
    composition-roots composition-bs fri-query-idxs composition-round-layers
    final-state
show ?thesis
unfolding composition-fri-verifier-tied-same-layer-auth-gap-def
by (intro exI conjI)
  (rule out-eq, rule fri-openings, rule aligned, rule trace-cand,
    rule comp-cand, rule trace-low, rule comp-not-low, use gap in simp-all)
qed
qed

lemma composition-fri-verifier-tied-same-layer-auth-gap-imp-recorded-auth-gap:
  assumes composition-fri-verifier-tied-same-layer-auth-gap s out
  shows composition-fri-verifier-tied-same-layer-recorded-auth-gap s out
proof –
from assms obtain trace-roots trace-bs trace-final fri-dg
  composition-roots composition-bs composition-final fri-query-idxs
  trace-round-layers composition-round-layers fr as trace-openings
  composition-openings trace-table composition-table result final-state
where out-eq: out = Some (result, final-state)
and fri-openings:
  accepted-fri-opening-transcript s out trace-roots trace-bs
  trace-final fri-dg composition-roots composition-bs
  composition-final fri-query-idxs trace-round-layers
  composition-round-layers
and aligned:
  accepted-with-partial-initial-openings-aligned s out fr trace-roots
  trace-final as fri-dg composition-roots composition-final
  fri-query-idxs trace-openings composition-openings
and trace-cand:
  partial-trace-table-candidate trace-table trace-openings
and comp-cand:
  partial-composition-table-candidate composition-table
  composition-openings

```

and *trace-low*: *trace-table-low-degree trace-table*
and *comp-not-low*:
 $\neg$ *composition-table-low-degree maxDegree composition-table*
and *same*:
 generic-fri-sampled-same-layer-opening-conflict composition-roots
 composition-bs fri-query-idxs composition-round-layers
and *no-auth*:
 $\neg$ *generic-fri-sampled-same-layer-opening-conflict-with-authenticated-chunks*
 composition-roots composition-bs fri-query-idxs composition-round-layers
 final-state
unfolding *composition-fri-verifier-tied-same-layer-auth-gap-def*
by *blast*
have *recorded-gap*:
 generic-fri-sampled-same-layer-recorded-auth-gap composition-roots
 composition-bs fri-query-idxs composition-round-layers final-state
by (rule *generic-fri-sampled-same-layer-auth-gap-imp-recorded-auth-gap*
 [*OF same no-auth*])
show *?thesis*
unfolding *composition-fri-verifier-tied-same-layer-recorded-auth-gap-def*
by (*intro exI conjI*)
 (rule *out-eq*, rule *fri-openings*, rule *aligned*, rule *trace-cand*,
 rule *comp-cand*, rule *trace-low*, rule *comp-not-low*, rule *recorded-gap*)
qed

lemma *composition-fri-verifier-tied-same-layer-recorded-auth-gap-imp-authenticated-recorded-auth-gap-on-support*:
assumes *support*: *out* $\in$ *set-dist* (*execute verify-monad s*)
and *gap*: *composition-fri-verifier-tied-same-layer-recorded-auth-gap s out*
shows
 composition-fri-verifier-tied-same-layer-authenticated-recorded-auth-gap
 s out

proof —
from *gap* **obtain** *trace-roots trace-bs trace-final fri-dg composition-roots*
 composition-bs composition-final fri-query-idxs trace-round-layers
 composition-round-layers fr as trace-openings composition-openings
 trace-table composition-table result final-state
where *out-eq*: *out* = *Some* (*result*, *final-state*)
and *fri-openings*:
 accepted-fri-opening-transcript s out trace-roots trace-bs
 trace-final fri-dg composition-roots composition-bs
 composition-final fri-query-idxs trace-round-layers
 composition-round-layers
and *aligned*:
 accepted-with-partial-initial-openings-aligned s out fr trace-roots
 trace-final as fri-dg composition-roots composition-final
 fri-query-idxs trace-openings composition-openings
and *trace-cand*:
 partial-trace-table-candidate trace-table trace-openings
and *comp-cand*:
 partial-composition-table-candidate composition-table


```

    composition-openings
  and trace-low: trace-table-low-degree trace-table
  and comp-not-low:
    ¬ composition-table-low-degree maxDegree composition-table
  and recorded-gap:
    generic-fri-sampled-same-layer-recorded-auth-gap composition-roots
    composition-bs fri-query-idxs composition-round-layers final-state
  unfolding composition-fri-verifier-tied-same-layer-recorded-auth-gap-def
  by blast
have outcome:
  Some (result, final-state) ∈ set-dist (execute verify-monad s)
  using support out-eq by simp
have challenges:
  accepted-fri-challenges s (Some (result, final-state))
  trace-bs fri-dg composition-bs
  using accepted-fri-opening-transcript-challenges[OF fri-openings]
  out-eq by simp
have degree-bound: to-nat fri-dg ≤ maxDegree
  by (rule verify-monad-accepted-fri-challenges-degree-bound
    [OF outcome challenges])
have len-composition-bs:
  length composition-bs = length composition-roots
  using accepted-fri-opening-transcript-shapes(4)[OF fri-openings] .
have len-query: length fri-query-idxs = rounds
  using accepted-fri-opening-transcript-shapes(7)[OF fri-openings] .
have auth-step:
  ∧ round-idx layer-idx.
  round-idx < length fri-query-idxs ⇒
  layer-idx < length composition-bs ⇒
  generic-fri-transcript-step-with-authenticated-chunk composition-roots
  composition-bs fri-query-idxs composition-round-layers final-state
  round-idx layer-idx
proof -
  fix round-idx layer-idx
  assume round-bound: round-idx < length fri-query-idxs
  and layer-bound-bs: layer-idx < length composition-bs
  have round-bound-rounds: round-idx < rounds
  using round-bound len-query by simp
  have layer-bound-roots: layer-idx < length composition-roots
  using layer-bound-bs len-composition-bs by simp
  show
    generic-fri-transcript-step-with-authenticated-chunk
    composition-roots composition-bs fri-query-idxs
    composition-round-layers final-state round-idx layer-idx
  by (rule
    accepted-fri-opening-transcript-composition-step-with-authenticated-chunk-at-raw
    [OF fri-openings out-eq degree-bound round-bound-rounds
      layer-bound-roots])
qed

```

```

have authenticated-recorded-gap:
  generic-fri-sampled-same-layer-authenticated-recorded-auth-gap
    composition-roots composition-bs fri-query-idxs composition-round-layers
    final-state
by (rule
  generic-fri-recorded-auth-gap-imp-authenticated-recorded-auth-gap
  [OF recorded-gap auth-step])
show ?thesis
unfolding
  composition-fri-verifier-tied-same-layer-authenticated-recorded-auth-gap-def
by (intro exI conjI)
  (rule out-eq, rule fri-openings, rule aligned, rule trace-cand,
   rule comp-cand, rule trace-low, rule comp-not-low,
   rule authenticated-recorded-gap)
qed

lemma composition-fri-verifier-tied-same-layer-authenticated-recorded-auth-gap-imp-slot-recorded-chunk-gap:
assumes
  composition-fri-verifier-tied-same-layer-authenticated-recorded-auth-gap
  s out
shows
  composition-fri-verifier-tied-authenticated-slot-recorded-chunk-gap
  s out
proof —
from assms obtain trace-roots trace-bs trace-final fri-dg composition-roots
  composition-bs composition-final fri-query-idxs trace-round-layers
  composition-round-layers fr as trace-openings composition-openings
  trace-table composition-table result final-state
where out-eq: out = Some (result, final-state)
and fri-openings:
  accepted-fri-opening-transcript s out trace-roots trace-bs
  trace-final fri-dg composition-roots composition-bs
  composition-final fri-query-idxs trace-round-layers
  composition-round-layers
and aligned:
  accepted-with-partial-initial-openings-aligned s out fr trace-roots
  trace-final as fri-dg composition-roots composition-final
  fri-query-idxs trace-openings composition-openings
and trace-cand:
  partial-trace-table-candidate trace-table trace-openings
and comp-cand:
  partial-composition-table-candidate composition-table
  composition-openings
and trace-low: trace-table-low-degree trace-table
and comp-not-low:
   $\neg$  composition-table-low-degree maxDegree composition-table
and gap:
  generic-fri-sampled-same-layer-authenticated-recorded-auth-gap
  composition-roots composition-bs fri-query-idxs composition-round-layers

```

```

    final-state
  unfolding
    composition-fri-verifier-tied-same-layer-authenticated-recorded-auth-gap-def
  by blast
have slot-gap:
  generic-fri-authenticated-slot-recorded-chunk-gap composition-roots
  fri-query-idxs composition-round-layers final-state
by (rule
  generic-fri-authenticated-recorded-auth-gap-imp-slot-recorded-chunk-gap
  [OF gap])
show ?thesis
  unfolding
    composition-fri-verifier-tied-authenticated-slot-recorded-chunk-gap-def
  by (intro exI conjI)
    (rule out-eq, rule fri-openings, rule aligned, rule trace-cand,
     rule comp-cand, rule trace-low, rule comp-not-low, rule slot-gap)
qed

lemma wp-composition-fri-verifier-tied-sampled-assignment-conflict-bound-from-authenticated-and-auth-gap:
  fixes C G :: prob
  assumes authenticated-bound:
    wp-event verify-monad
      (composition-fri-verifier-tied-sampled-assignment-conflict-with-authenticated-same-layer s)
    s ≤ C
  and auth-gap-bound:
    wp-event verify-monad
      (composition-fri-verifier-tied-assignment-auth-gap s) s ≤ G
  shows
    wp-event verify-monad
      (composition-fri-verifier-tied-sampled-assignment-conflict s) s
      ≤ C + G
proof -
  have wp-event verify-monad
    (composition-fri-verifier-tied-sampled-assignment-conflict s) s ≤
    wp-event verify-monad
      (λout.
        composition-fri-verifier-tied-sampled-assignment-conflict-with-authenticated-same-layer
          s out ∨
          composition-fri-verifier-tied-assignment-auth-gap s out) s
  by (rule wp-event-mono)
    (rule
      composition-fri-verifier-tied-assignment-imp-authenticated-or-auth-gap)
  also have ... ≤
    wp-event verify-monad
      (composition-fri-verifier-tied-sampled-assignment-conflict-with-authenticated-same-layer s)
    s +
    wp-event verify-monad
      (composition-fri-verifier-tied-assignment-auth-gap s) s
  by (rule wp-event-union-bound)

```

also have $\dots \leq C + G$
 by (intro add-mono authenticated-bound auth-gap-bound)
 finally show ?thesis .
 qed

lemma wp-composition-fri-verifier-tied-assignment-auth-gap-bound-from-same-layer-auth-gap:
 fixes $G :: \text{prob}$
 assumes gap-bound:
 wp-event verify-monad
 (composition-fri-verifier-tied-same-layer-auth-gap s) $s \leq G$
 shows
 wp-event verify-monad
 (composition-fri-verifier-tied-assignment-auth-gap s) $s \leq G$
proof –
 have wp-event verify-monad
 (composition-fri-verifier-tied-assignment-auth-gap s) $s \leq$
 wp-event verify-monad
 (composition-fri-verifier-tied-same-layer-auth-gap s) s
 by (rule wp-event-mono)
 (rule
 composition-fri-verifier-tied-assignment-auth-gap-imp-same-layer-auth-gap)
 then show ?thesis
 by (rule order-trans[OF - gap-bound])
 qed

lemma wp-composition-fri-verifier-tied-same-layer-auth-gap-bound-from-recorded-auth-gap:
 fixes $G :: \text{prob}$
 assumes gap-bound:
 wp-event verify-monad
 (composition-fri-verifier-tied-same-layer-recorded-auth-gap s) s
 $\leq G$
 shows
 wp-event verify-monad
 (composition-fri-verifier-tied-same-layer-auth-gap s) $s \leq G$
proof –
 have wp-event verify-monad
 (composition-fri-verifier-tied-same-layer-auth-gap s) $s \leq$
 wp-event verify-monad
 (composition-fri-verifier-tied-same-layer-recorded-auth-gap s) s
 by (rule wp-event-mono)
 (rule
 composition-fri-verifier-tied-same-layer-auth-gap-imp-recorded-auth-gap)
 then show ?thesis
 by (rule order-trans[OF - gap-bound])
 qed

lemma wp-composition-fri-verifier-tied-same-layer-recorded-auth-gap-bound-from-authenticated-recorded-auth-gap:
 fixes $G :: \text{prob}$
 assumes gap-bound:

```

    wp-event verify-monad
      (composition-fri-verifier-tied-same-layer-authenticated-recorded-auth-gap
        s) s ≤ G
  shows
    wp-event verify-monad
      (composition-fri-verifier-tied-same-layer-recorded-auth-gap s) s
      ≤ G
  proof -
    have wp-event verify-monad
      (composition-fri-verifier-tied-same-layer-recorded-auth-gap s) s ≤
      wp-event verify-monad
        (composition-fri-verifier-tied-same-layer-authenticated-recorded-auth-gap
          s) s
    by (rule wp-event-mono-on-support)
      (rule
        composition-fri-verifier-tied-same-layer-recorded-auth-gap-imp-authenticated-recorded-auth-gap-on-support)
    then show ?thesis
      by (rule order-trans[OF - gap-bound])
  qed

lemma wp-composition-fri-verifier-tied-same-layer-authenticated-recorded-auth-gap-bound-from-slot-recorded-chunk-gap
  fixes G :: prob
  assumes gap-bound:
    wp-event verify-monad
      (composition-fri-verifier-tied-authenticated-slot-recorded-chunk-gap s)
      s ≤ G
  shows
    wp-event verify-monad
      (composition-fri-verifier-tied-same-layer-authenticated-recorded-auth-gap
        s) s ≤ G
  proof -
    have wp-event verify-monad
      (composition-fri-verifier-tied-same-layer-authenticated-recorded-auth-gap
        s) s ≤
      wp-event verify-monad
        (composition-fri-verifier-tied-authenticated-slot-recorded-chunk-gap s)
        s
    by (rule wp-event-mono)
      (rule
        composition-fri-verifier-tied-same-layer-authenticated-recorded-auth-gap-imp-slot-recorded-chunk-gap)
    then show ?thesis
      by (rule order-trans[OF - gap-bound])
  qed

lemma wp-composition-fri-verifier-tied-sampled-assignment-conflict-bound-from-auth-gap-and-residuals:
  fixes A P M N S G :: prob
  assumes auth-gap-bound:
    wp-event verify-monad
      (composition-fri-verifier-tied-assignment-auth-gap s) s ≤ A

```

and *partial-merkle-bound*:
 $\text{wp-event verify-monad } (\text{partial-merkle-inconsistency-bad } s) \ s \leq P$
and *missing-bound*:
 $\text{wp-event verify-monad}$
 $(\text{composition-fri-verifier-tied-recorded-base-chunk-auth-missing } s)$
 $s \leq M$
and *next-gap-bound*:
 $\text{wp-event verify-monad}$
 $(\text{composition-fri-verifier-tied-next-value-replay-gap } s) \ s \leq N$
and *successor-gap-bound*:
 $\text{wp-event verify-monad}$
 $(\text{composition-fri-verifier-tied-successor-replay-gap } s) \ s \leq S$
and *final-gap-bound*:
 $\text{wp-event verify-monad}$
 $(\text{composition-fri-verifier-tied-final-value-replay-gap } s) \ s \leq G$
shows
 $\text{wp-event verify-monad}$
 $(\text{composition-fri-verifier-tied-sampled-assignment-conflict } s) \ s$
 $\leq ((P + M) + N + S + G + P) + A$
by (rule
 $\text{wp-composition-fri-verifier-tied-sampled-assignment-conflict-bound-from-authenticated-and-auth-gap})$
 $(\text{rule wp-composition-fri-verifier-tied-authenticated-conflict-bound-from-all-replay-gaps}$
 $[\text{OF partial-merkle-bound missing-bound next-gap-bound}$
 $\text{successor-gap-bound final-gap-bound}],$
 $\text{rule auth-gap-bound})$

lemma *wp-composition-fri-verifier-tied-sampled-assignment-conflict-bound-from-same-layer-auth-gap-and-residuals*:

fixes $A \ P \ M \ N \ S \ G :: \text{prob}$

assumes *same-layer-gap-bound*:

$\text{wp-event verify-monad}$
 $(\text{composition-fri-verifier-tied-same-layer-auth-gap } s) \ s \leq A$
and *partial-merkle-bound*:
 $\text{wp-event verify-monad } (\text{partial-merkle-inconsistency-bad } s) \ s \leq P$
and *missing-bound*:
 $\text{wp-event verify-monad}$
 $(\text{composition-fri-verifier-tied-recorded-base-chunk-auth-missing } s)$
 $s \leq M$

and *next-gap-bound*:
 $\text{wp-event verify-monad}$
 $(\text{composition-fri-verifier-tied-next-value-replay-gap } s) \ s \leq N$

and *successor-gap-bound*:
 $\text{wp-event verify-monad}$
 $(\text{composition-fri-verifier-tied-successor-replay-gap } s) \ s \leq S$

and *final-gap-bound*:
 $\text{wp-event verify-monad}$
 $(\text{composition-fri-verifier-tied-final-value-replay-gap } s) \ s \leq G$

shows

$\text{wp-event verify-monad}$
 $(\text{composition-fri-verifier-tied-sampled-assignment-conflict } s) \ s$

$\leq ((P + M) + N + S + G + P) + A$
by (rule
wp-composition-fri-verifier-tied-sampled-assignment-conflict-bound-from-auth-gap-and-residuals)
(rule
wp-composition-fri-verifier-tied-assignment-auth-gap-bound-from-same-layer-auth-gap
[OF *same-layer-gap-bound*],
rule *partial-merkle-bound*,
rule *missing-bound*,
rule *next-gap-bound*,
rule *successor-gap-bound*,
rule *final-gap-bound*)

lemma *wp-composition-fri-verifier-tied-sampled-assignment-conflict-bound-from-recorded-auth-gap-and-residuals*:

fixes $A P M N S G :: \text{prob}$

assumes *recorded-gap-bound*:

wp-event verify-monad
(*composition-fri-verifier-tied-same-layer-recorded-auth-gap s*) s
 $\leq A$

and *partial-merkle-bound*:

wp-event verify-monad (*partial-merkle-inconsistency-bad s*) $s \leq P$

and *missing-bound*:

wp-event verify-monad
(*composition-fri-verifier-tied-recorded-base-chunk-auth-missing s*)
 $s \leq M$

and *next-gap-bound*:

wp-event verify-monad
(*composition-fri-verifier-tied-next-value-replay-gap s*) $s \leq N$

and *successor-gap-bound*:

wp-event verify-monad
(*composition-fri-verifier-tied-successor-replay-gap s*) $s \leq S$

and *final-gap-bound*:

wp-event verify-monad
(*composition-fri-verifier-tied-final-value-replay-gap s*) $s \leq G$

shows

wp-event verify-monad
(*composition-fri-verifier-tied-sampled-assignment-conflict s*) s
 $\leq ((P + M) + N + S + G + P) + A$

by (rule

wp-composition-fri-verifier-tied-sampled-assignment-conflict-bound-from-same-layer-auth-gap-and-residuals)
(rule
wp-composition-fri-verifier-tied-same-layer-auth-gap-bound-from-recorded-auth-gap
[OF *recorded-gap-bound*],
rule *partial-merkle-bound*,
rule *missing-bound*,
rule *next-gap-bound*,
rule *successor-gap-bound*,
rule *final-gap-bound*)

lemma *wp-composition-fri-verifier-tied-sampled-assignment-conflict-bound-from-authenticated-recorded-auth-gap-and*

fixes $A P M N S G :: \text{prob}$
assumes *gap-bound*:
 wp-event verify-monad
 (*composition-fri-verifier-tied-same-layer-authenticated-recorded-auth-gap*
 s) $s \leq A$
and *partial-merkle-bound*:
 wp-event verify-monad (*partial-merkle-inconsistency-bad s*) $s \leq P$
and *missing-bound*:
 wp-event verify-monad
 (*composition-fri-verifier-tied-recorded-base-chunk-auth-missing s*)
 $s \leq M$
and *next-gap-bound*:
 wp-event verify-monad
 (*composition-fri-verifier-tied-next-value-replay-gap s*) $s \leq N$
and *successor-gap-bound*:
 wp-event verify-monad
 (*composition-fri-verifier-tied-successor-replay-gap s*) $s \leq S$
and *final-gap-bound*:
 wp-event verify-monad
 (*composition-fri-verifier-tied-final-value-replay-gap s*) $s \leq G$
shows
 wp-event verify-monad
 (*composition-fri-verifier-tied-sampled-assignment-conflict s*) s
 $\leq ((P + M) + N + S + G + P) + A$
by (*rule*
 wp-composition-fri-verifier-tied-sampled-assignment-conflict-bound-from-recorded-auth-gap-and-residuals)
 (*rule*
 wp-composition-fri-verifier-tied-same-layer-recorded-auth-gap-bound-from-authenticated-recorded-auth-gap
 [*OF gap-bound*],
 rule partial-merkle-bound,
 rule missing-bound,
 rule next-gap-bound,
 rule successor-gap-bound,
 rule final-gap-bound)

lemma *wp-composition-fri-verifier-tied-sampled-assignment-conflict-bound-from-slot-recorded-chunk-gap-and-residuals*

fixes $A P M N S G :: \text{prob}$
assumes *gap-bound*:
 wp-event verify-monad
 (*composition-fri-verifier-tied-authenticated-slot-recorded-chunk-gap s*)
 $s \leq A$
and *partial-merkle-bound*:
 wp-event verify-monad (*partial-merkle-inconsistency-bad s*) $s \leq P$
and *missing-bound*:
 wp-event verify-monad
 (*composition-fri-verifier-tied-recorded-base-chunk-auth-missing s*)
 $s \leq M$
and *next-gap-bound*:
 wp-event verify-monad

(*composition-fri-verifier-tied-next-value-replay-gap* s) $s \leq N$
and *successor-gap-bound*:
wp-event verify-monad
 (*composition-fri-verifier-tied-successor-replay-gap* s) $s \leq S$
and *final-gap-bound*:
wp-event verify-monad
 (*composition-fri-verifier-tied-final-value-replay-gap* s) $s \leq G$
shows
wp-event verify-monad
 (*composition-fri-verifier-tied-sampled-assignment-conflict* s) s
 $\leq ((P + M) + N + S + G + P) + A$
by (*rule*
wp-composition-fri-verifier-tied-sampled-assignment-conflict-bound-from-authenticated-recorded-auth-gap-and-residuals
 (*rule*
wp-composition-fri-verifier-tied-same-layer-authenticated-recorded-auth-gap-bound-from-slot-recorded-chunk-gap
 [*OF gap-bound*],
rule partial-merkle-bound,
rule missing-bound,
rule next-gap-bound,
rule successor-gap-bound,
rule final-gap-bound)

lemma *wp-composition-fri-verifier-tied-bound-from-sampled-auth-gap-and-residuals*:

fixes $R A P M N S G :: \text{prob}$

assumes *sampled-bound*:

wp-event verify-monad
 (*composition-fri-bad-with-verifier-tied-sampled-layer-chain* s) $s \leq R$

and *auth-gap-bound*:

wp-event verify-monad
 (*composition-fri-verifier-tied-assignment-auth-gap* s) $s \leq A$

and *partial-merkle-bound*:

wp-event verify-monad (*partial-merkle-inconsistency-bad* s) $s \leq P$

and *missing-bound*:

wp-event verify-monad
 (*composition-fri-verifier-tied-recorded-base-chunk-auth-missing* s)
 $s \leq M$

and *next-gap-bound*:

wp-event verify-monad
 (*composition-fri-verifier-tied-next-value-replay-gap* s) $s \leq N$

and *successor-gap-bound*:

wp-event verify-monad
 (*composition-fri-verifier-tied-successor-replay-gap* s) $s \leq S$

and *final-gap-bound*:

wp-event verify-monad
 (*composition-fri-verifier-tied-final-value-replay-gap* s) $s \leq G$

shows

wp-event verify-monad
 (*composition-fri-bad-with-verifier-tied-partial-candidate* s) s
 $\leq R + (((P + M) + N + S + G + P) + A)$

proof –

have *conflict-bound*:

wp-event verify-monad

(*composition-fri-verifier-tied-sampled-assignment-conflict s*) s
 $\leq ((P + M) + N + S + G + P) + A$

by (rule

wp-composition-fri-verifier-tied-sampled-assignment-conflict-bound-from-auth-gap-and-residuals)

(rule *auth-gap-bound*,

rule *partial-merkle-bound*,

rule *missing-bound*,

rule *next-gap-bound*,

rule *successor-gap-bound*,

rule *final-gap-bound*)

show *?thesis*

by (rule

wp-composition-fri-verifier-tied-bound-from-sampled-conflict
[*OF sampled-bound conflict-bound*])

qed

lemma *wp-composition-fri-verifier-tied-bound-from-sampled-same-layer-auth-gap-and-residuals*:

fixes $R\ A\ P\ M\ N\ S\ G :: \text{prob}$

assumes *sampled-bound*:

wp-event verify-monad

(*composition-fri-bad-with-verifier-tied-sampled-layer-chain s*) $s \leq R$

and *same-layer-gap-bound*:

wp-event verify-monad

(*composition-fri-verifier-tied-same-layer-auth-gap s*) $s \leq A$

and *partial-merkle-bound*:

wp-event verify-monad (*partial-merkle-inconsistency-bad s*) $s \leq P$

and *missing-bound*:

wp-event verify-monad

(*composition-fri-verifier-tied-recorded-base-chunk-auth-missing s*)
 $s \leq M$

and *next-gap-bound*:

wp-event verify-monad

(*composition-fri-verifier-tied-next-value-replay-gap s*) $s \leq N$

and *successor-gap-bound*:

wp-event verify-monad

(*composition-fri-verifier-tied-successor-replay-gap s*) $s \leq S$

and *final-gap-bound*:

wp-event verify-monad

(*composition-fri-verifier-tied-final-value-replay-gap s*) $s \leq G$

shows

wp-event verify-monad

(*composition-fri-bad-with-verifier-tied-partial-candidate s*) s
 $\leq R + (((P + M) + N + S + G + P) + A)$

by (rule

wp-composition-fri-verifier-tied-bound-from-sampled-auth-gap-and-residuals)

(rule *sampled-bound*,

rule
wp-composition-fri-verifier-tied-assignment-auth-gap-bound-from-same-layer-auth-gap
 [OF *same-layer-gap-bound*],
rule partial-merkle-bound,
rule missing-bound,
rule next-gap-bound,
rule successor-gap-bound,
rule final-gap-bound)

lemma *wp-composition-fri-verifier-tied-bound-from-sampled-recorded-auth-gap-and-residuals*:

fixes $R\ A\ P\ M\ N\ S\ G :: \text{prob}$

assumes *sampled-bound*:

wp-event verify-monad
 (*composition-fri-bad-with-verifier-tied-sampled-layer-chain* s) $s \leq R$

and *recorded-gap-bound*:

wp-event verify-monad
 (*composition-fri-verifier-tied-same-layer-recorded-auth-gap* s) s
 $\leq A$

and *partial-merkle-bound*:

wp-event verify-monad (*partial-merkle-inconsistency-bad* s) $s \leq P$

and *missing-bound*:

wp-event verify-monad
 (*composition-fri-verifier-tied-recorded-base-chunk-auth-missing* s)
 $s \leq M$

and *next-gap-bound*:

wp-event verify-monad
 (*composition-fri-verifier-tied-next-value-replay-gap* s) $s \leq N$

and *successor-gap-bound*:

wp-event verify-monad
 (*composition-fri-verifier-tied-successor-replay-gap* s) $s \leq S$

and *final-gap-bound*:

wp-event verify-monad
 (*composition-fri-verifier-tied-final-value-replay-gap* s) $s \leq G$

shows

wp-event verify-monad
 (*composition-fri-bad-with-verifier-tied-partial-candidate* s) s
 $\leq R + (((P + M) + N + S + G + P) + A)$

by (*rule*

wp-composition-fri-verifier-tied-bound-from-sampled-same-layer-auth-gap-and-residuals)

(*rule sampled-bound*,

rule

wp-composition-fri-verifier-tied-same-layer-auth-gap-bound-from-recorded-auth-gap
 [OF *recorded-gap-bound*],

rule partial-merkle-bound,

rule missing-bound,

rule next-gap-bound,

rule successor-gap-bound,

rule final-gap-bound)

lemma *wp-composition-fri-verifier-tied-bound-from-sampled-authenticated-recorded-auth-gap-and-residuals:*
fixes $R\ A\ P\ M\ N\ S\ G :: \text{prob}$
assumes *sampled-bound:*
 wp-event verify-monad
 (composition-fri-bad-with-verifier-tied-sampled-layer-chain s) s ≤ R
and *gap-bound:*
 wp-event verify-monad
 (composition-fri-verifier-tied-same-layer-authenticated-recorded-auth-gap s) s ≤ A
and *partial-merkle-bound:*
 wp-event verify-monad (partial-merkle-inconsistency-bad s) s ≤ P
and *missing-bound:*
 wp-event verify-monad
 (composition-fri-verifier-tied-recorded-base-chunk-auth-missing s)
 s ≤ M
and *next-gap-bound:*
 wp-event verify-monad
 (composition-fri-verifier-tied-next-value-replay-gap s) s ≤ N
and *successor-gap-bound:*
 wp-event verify-monad
 (composition-fri-verifier-tied-successor-replay-gap s) s ≤ S
and *final-gap-bound:*
 wp-event verify-monad
 (composition-fri-verifier-tied-final-value-replay-gap s) s ≤ G
shows
 wp-event verify-monad
 (composition-fri-bad-with-verifier-tied-partial-candidate s) s
 $\leq R + (((P + M) + N + S + G + P) + A)$
by (*rule*
 wp-composition-fri-verifier-tied-bound-from-sampled-recorded-auth-gap-and-residuals)
 (*rule sampled-bound,*
 rule
 wp-composition-fri-verifier-tied-same-layer-recorded-auth-gap-bound-from-authenticated-recorded-auth-gap
 [OF gap-bound],
 rule partial-merkle-bound,
 rule missing-bound,
 rule next-gap-bound,
 rule successor-gap-bound,
 rule final-gap-bound)

lemma *wp-composition-fri-verifier-tied-bound-from-sampled-slot-recorded-chunk-gap-and-residuals:*
fixes $R\ A\ P\ M\ N\ S\ G :: \text{prob}$
assumes *sampled-bound:*
 wp-event verify-monad
 (composition-fri-bad-with-verifier-tied-sampled-layer-chain s) s ≤ R
and *gap-bound:*
 wp-event verify-monad
 (composition-fri-verifier-tied-authenticated-slot-recorded-chunk-gap s) s ≤ A

and *partial-merkle-bound*:
 $wp\text{-}event\ verify\text{-}monad\ (partial\text{-}merkle\text{-}inconsistency\text{-}bad\ s)\ s \leq P$
and *missing-bound*:
 $wp\text{-}event\ verify\text{-}monad\ (composition\text{-}fri\text{-}verifier\text{-}tied\text{-}recorded\text{-}base\text{-}chunk\text{-}auth\text{-}missing\ s)\ s \leq M$
and *next-gap-bound*:
 $wp\text{-}event\ verify\text{-}monad\ (composition\text{-}fri\text{-}verifier\text{-}tied\text{-}next\text{-}value\text{-}replay\text{-}gap\ s)\ s \leq N$
and *successor-gap-bound*:
 $wp\text{-}event\ verify\text{-}monad\ (composition\text{-}fri\text{-}verifier\text{-}tied\text{-}successor\text{-}replay\text{-}gap\ s)\ s \leq S$
and *final-gap-bound*:
 $wp\text{-}event\ verify\text{-}monad\ (composition\text{-}fri\text{-}verifier\text{-}tied\text{-}final\text{-}value\text{-}replay\text{-}gap\ s)\ s \leq G$
shows
 $wp\text{-}event\ verify\text{-}monad\ (composition\text{-}fri\text{-}bad\text{-}with\text{-}verifier\text{-}tied\text{-}partial\text{-}candidate\ s)\ s \leq R + (((P + M) + N + S + G + P) + A)$
by (rule
 $wp\text{-}composition\text{-}fri\text{-}verifier\text{-}tied\text{-}bound\text{-}from\text{-}sampled\text{-}authenticated\text{-}recorded\text{-}auth\text{-}gap\text{-}and\text{-}residuals)$
(rule *sampled-bound*,
rule
 $wp\text{-}composition\text{-}fri\text{-}verifier\text{-}tied\text{-}same\text{-}layer\text{-}authenticated\text{-}recorded\text{-}auth\text{-}gap\text{-}bound\text{-}from\text{-}slot\text{-}recorded\text{-}chunk\text{-}gap$
[*OF gap-bound*],
rule *partial-merkle-bound*,
rule *missing-bound*,
rule *next-gap-bound*,
rule *successor-gap-bound*,
rule *final-gap-bound*)

lemma *composition-fri-reachable-partial-candidate-reduction-from-verifier-tied-residual-bounds*:

fixes $R\ A\ P\ M\ N\ S\ G\ T :: prob$

assumes *sampled-bound*:

$wp\text{-}event\ verify\text{-}monad\ (composition\text{-}fri\text{-}bad\text{-}with\text{-}verifier\text{-}tied\text{-}sampled\text{-}layer\text{-}chain\ s)\ s \leq R$

and *slot-gap-bound*:

$wp\text{-}event\ verify\text{-}monad\ (composition\text{-}fri\text{-}verifier\text{-}tied\text{-}authenticated\text{-}slot\text{-}recorded\text{-}chunk\text{-}gap\ s)\ s \leq A$

and *partial-merkle-bound*:

$wp\text{-}event\ verify\text{-}monad\ (partial\text{-}merkle\text{-}inconsistency\text{-}bad\ s)\ s \leq P$

and *missing-bound*:

$wp\text{-}event\ verify\text{-}monad\ (composition\text{-}fri\text{-}verifier\text{-}tied\text{-}recorded\text{-}base\text{-}chunk\text{-}auth\text{-}missing\ s)\ s \leq M$

and *next-gap-bound*:

$wp\text{-}event\ verify\text{-}monad\ (composition\text{-}fri\text{-}verifier\text{-}tied\text{-}next\text{-}value\text{-}replay\text{-}gap\ s)\ s \leq N$

and *successor-gap-bound*:
 $wp\text{-event } verify\text{-monad}$
 $(composition\text{-fri-verifier-tied-successor-replay-gap } s) \ s \leq S$
and *final-gap-bound*:
 $wp\text{-event } verify\text{-monad}$
 $(composition\text{-fri-verifier-tied-final-value-replay-gap } s) \ s \leq G$
and *verifier-gap-bound*:
 $wp\text{-event } verify\text{-monad}$
 $(composition\text{-fri-reachable-verifier-tie-gap } s) \ s \leq T$
and *total-bound*:
 $R + (((P + M) + N + S + G + P) + A) + T$
 $\leq composition\text{-fri-error}$
shows *composition-fri-reachable-partial-candidate-reduction-assumption* s
proof –
have *verifier-bound*:
 $wp\text{-event } verify\text{-monad}$
 $(composition\text{-fri-bad-with-verifier-tied-partial-candidate } s) \ s$
 $\leq R + (((P + M) + N + S + G + P) + A)$
by (rule
 $wp\text{-composition-fri-verifier-tied-bound-from-sampled-slot-recorded-chunk-gap-and-residuals}$
 $[OF \text{ sampled-bound slot-gap-bound partial-merkle-bound missing-bound}$
 $next-gap-bound successor-gap-bound final-gap-bound])$
show *?thesis*
by (rule
 $composition\text{-fri-reachable-partial-candidate-reduction-from-verifier-tied-and-gap}$
 $[OF \text{ verifier-bound verifier-gap-bound total-bound}])$
qed

lemma *composition-fri-reachable-partial-candidate-reduction-from-verifier-tied-residual-bounds-with-recorded-missing*
fixes $R \ A \ P \ N \ S \ G \ T :: prob$
assumes *sampled-bound*:
 $wp\text{-event } verify\text{-monad}$
 $(composition\text{-fri-bad-with-verifier-tied-sampled-layer-chain } s) \ s \leq R$
and *slot-gap-bound*:
 $wp\text{-event } verify\text{-monad}$
 $(composition\text{-fri-verifier-tied-authenticated-slot-recorded-chunk-gap}$
 $s) \ s \leq A$
and *partial-merkle-bound*:
 $wp\text{-event } verify\text{-monad } (partial\text{-merkle-inconsistency-bad } s) \ s \leq P$
and *next-gap-bound*:
 $wp\text{-event } verify\text{-monad}$
 $(composition\text{-fri-verifier-tied-next-value-replay-gap } s) \ s \leq N$
and *successor-gap-bound*:
 $wp\text{-event } verify\text{-monad}$
 $(composition\text{-fri-verifier-tied-successor-replay-gap } s) \ s \leq S$
and *final-gap-bound*:
 $wp\text{-event } verify\text{-monad}$
 $(composition\text{-fri-verifier-tied-final-value-replay-gap } s) \ s \leq G$
and *verifier-gap-bound*:

wp-event verify-monad
 (composition-fri-reachable-verifier-tie-gap s) $s \leq T$
and total-bound:
 $R + (((P + 0) + N + S + G + P) + A) + T$
 $\leq$ composition-fri-error
shows composition-fri-reachable-partial-candidate-reduction-assumption s
by (rule composition-fri-reachable-partial-candidate-reduction-from-verifier-tied-residual-bounds
 [where $M = 0$])
 (rule sampled-bound,
 rule slot-gap-bound,
 rule partial-merkle-bound,
 rule wp-composition-fri-verifier-tied-recorded-base-chunk-auth-missing-zero,
 rule next-gap-bound,
 rule successor-gap-bound,
 rule final-gap-bound,
 rule verifier-gap-bound,
 rule total-bound)

lemma composition-fri-verifier-tied-reduction-from-residual-bounds:
fixes $R A P M N S G :: \text{prob}$
assumes sampled-bound:
wp-event verify-monad
 (composition-fri-bad-with-verifier-tied-sampled-layer-chain s) $s \leq R$
and slot-gap-bound:
wp-event verify-monad
 (composition-fri-verifier-tied-authenticated-slot-recorded-chunk-gap
 s) $s \leq A$
and partial-merkle-bound:
wp-event verify-monad (partial-merkle-inconsistency-bad s) $s \leq P$
and missing-bound:
wp-event verify-monad
 (composition-fri-verifier-tied-recorded-base-chunk-auth-missing s)
 $s \leq M$
and next-gap-bound:
wp-event verify-monad
 (composition-fri-verifier-tied-next-value-replay-gap s) $s \leq N$
and successor-gap-bound:
wp-event verify-monad
 (composition-fri-verifier-tied-successor-replay-gap s) $s \leq S$
and final-gap-bound:
wp-event verify-monad
 (composition-fri-verifier-tied-final-value-replay-gap s) $s \leq G$
and total-bound:
 $R + (((P + M) + N + S + G + P) + A)$
 $\leq$ composition-fri-error
shows composition-fri-verifier-tied-reduction s
unfolding composition-fri-verifier-tied-reduction-def
by (rule order-trans[
 OF wp-composition-fri-verifier-tied-bound-from-sampled-slot-recorded-chunk-gap-and-residuals

[OF sampled-bound slot-gap-bound partial-merkle-bound missing-bound
next-gap-bound successor-gap-bound final-gap-bound] total-bound])

lemma *composition-fri-verifier-tied-reduction-from-residual-bounds-without-recorded-missing:*

fixes $R\ A\ P\ N\ S\ G :: \text{prob}$

assumes *sampling-bound:*

wp-event verify-monad

(*composition-fri-bad-with-verifier-tied-sampled-layer-chain* s) $s \leq R$

and *slot-gap-bound:*

wp-event verify-monad

(*composition-fri-verifier-tied-authenticated-slot-recorded-chunk-gap* s) $s \leq A$

and *partial-merkle-bound:*

wp-event verify-monad (*partial-merkle-inconsistency-bad* s) $s \leq P$

and *next-gap-bound:*

wp-event verify-monad

(*composition-fri-verifier-tied-next-value-replay-gap* s) $s \leq N$

and *successor-gap-bound:*

wp-event verify-monad

(*composition-fri-verifier-tied-successor-replay-gap* s) $s \leq S$

and *final-gap-bound:*

wp-event verify-monad

(*composition-fri-verifier-tied-final-value-replay-gap* s) $s \leq G$

and *total-bound:*

$R + (((P + A) + N + S + G + P) + A)$
 $\leq \text{composition-fri-error}$

shows *composition-fri-verifier-tied-reduction* s

proof –

have *missing-bound:*

wp-event verify-monad

(*composition-fri-verifier-tied-recorded-base-chunk-auth-missing* s)

$s \leq A$

by (rule

wp-composition-fri-verifier-tied-recorded-base-chunk-auth-missing-bound-from-slot-recorded-chunk-gap
[OF slot-gap-bound])

show *?thesis*

by (rule *composition-fri-verifier-tied-reduction-from-residual-bounds*

[OF sampled-bound slot-gap-bound partial-merkle-bound missing-bound
next-gap-bound successor-gap-bound final-gap-bound total-bound])

qed

lemma *composition-fri-verifier-tied-reduction-from-residual-bounds-with-recorded-missing-zero:*

fixes $R\ A\ P\ N\ S\ G :: \text{prob}$

assumes *sampling-bound:*

wp-event verify-monad

(*composition-fri-bad-with-verifier-tied-sampled-layer-chain* s) $s \leq R$

and *slot-gap-bound:*

wp-event verify-monad

(*composition-fri-verifier-tied-authenticated-slot-recorded-chunk-gap*


```

    s)  $s \leq A$ 
and partial-merkle-bound:
    wp-event verify-monad (partial-merkle-inconsistency-bad s)  $s \leq P$ 
and next-gap-bound:
    wp-event verify-monad
      (composition-fri-verifier-tied-next-value-replay-gap s)  $s \leq N$ 
and successor-gap-bound:
    wp-event verify-monad
      (composition-fri-verifier-tied-successor-replay-gap s)  $s \leq S$ 
and final-gap-bound:
    wp-event verify-monad
      (composition-fri-verifier-tied-final-value-replay-gap s)  $s \leq G$ 
and total-bound:
     $R + (((P + 0) + N + S + G + P) + A)$ 
     $\leq$  composition-fri-error
shows composition-fri-verifier-tied-reduction s
by (rule composition-fri-verifier-tied-reduction-from-residual-bounds
  [where  $M = 0$ , OF sampled-bound slot-gap-bound partial-merkle-bound
    wp-composition-fri-verifier-tied-recorded-base-chunk-auth-missing-zero
    next-gap-bound successor-gap-bound final-gap-bound total-bound])

end

end

```

```

theory Soundness-FRI-Value-Aligned-Events
  imports Soundness-FRI-Composition-Residuals
begin

```

Value-aligned FRI branch events.

These events are narrower than the raw sampled base-opening conflicts: they keep the trace query opening that supplies the first trace FRI input value. The intended use is to avoid exporting broad verifier-query-round decomposition lemmas, which are expensive for Isabelle to finalize.

```

context soundness
begin

```

```

lemma receive-query-commits-head-layer-outcome:
  fixes  $s\ t :: ('f, 'a)\ \text{protocol-channel-scheme}$ 
  assumes bfs-eq:  $bfs = (b, rt) \# bfs\text{-tail}$ 
  and outcome:
     $\text{Some } (out, t) \in$ 
      set-dist
      (execute
        (mfold (idx, x, len, pw) (receive-query-commits bfs)) s)
  obtains layer-chunks chunk xp xp-path xn xn-path where
    fri-layers-transcript (length bfs) len layer-chunks chunk
     $P\text{Transcript } s = \text{chunk} @ P\text{Transcript } t$ 

```

```

PState t = foldl concat (PState s) chunk
s ≤ t
PQueryCounter t = PQueryCounter s
layer-chunks ≠ []
xp = x
fri-layer-opening-chunk len xp xp-path xn xn-path (hd layer-chunks)
proof –
have steps:
  receive-query-commits bfs =
    fri-layer-opening-step (b, rt) #
      receive-query-commits bfs-tail
  unfolding bfs-eq receive-query-commits-def fri-layer-opening-step-def
  by simp
from outcome[unfolded steps]
obtain out1 s1 where head:
  Some (out1, s1) ∈
    set-dist
      (execute
        (fri-layer-opening-step (b, rt) (idx, x, len, pw)) s)
and tail:
  Some (out, t) ∈
    set-dist (execute
      (mfold out1 (receive-query-commits bfs-tail)) s1)
  by (auto elim!: set-dist-bindE)
from fri-layer-opening-step-outcome[OF head]
obtain xp xp-path xn xn-path x' where out1-eq:
  out1 = (idx mod (len div 2), x', len div 2, pw + pw)
and xp-eq: xp = x
and head-chunk:
  fri-layer-opening-chunk len xp xp-path xn xn-path
    ([xp] @ xp-path @ [xn] @ xn-path)
and transcript-head:
  PTranscript s =
    ([xp] @ xp-path @ [xn] @ xn-path) @ PTranscript s1
and state-head:
  PState s1 =
    foldl concat (PState s) ([xp] @ xp-path @ [xn] @ xn-path)
and ext-head: s ≤ s1
and counter-head: PQueryCounter s1 = PQueryCounter s
using fri-layer-opening-step-outcome[OF head]
by blast
from receive-query-commits-layers-outcome[OF tail[unfolded out1-eq]]
obtain tail-layers tail-chunk where tail-layers:
  fri-layers-transcript (length bfs-tail) (len div 2)
    tail-layers tail-chunk
and transcript-tail:
  PTranscript s1 = tail-chunk @ PTranscript t
and state-tail:
  PState t = foldl concat (PState s1) tail-chunk

```

```

and ext-tail:  $s1 \leq t$ 
and counter-tail:  $PQueryCounter\ t = PQueryCounter\ s1$ 
by blast
let ?head-chunk =  $[xp] @ xp\text{-}path @ [xn] @ xn\text{-}path$ 
let ?layers = ?head-chunk # tail-layers
let ?chunk = ?head-chunk @ tail-chunk
have layers:
  fri-layers-transcript (length bfs) len ?layers ?chunk
  using tail-layers head-chunk unfolding bfs-eq fri-layers-transcript-def
  by auto
have transcript:
   $PTranscript\ s = ?chunk @ PTranscript\ t$ 
  using transcript-head transcript-tail by simp
have state:
   $PState\ t = foldl\ concat\ (PState\ s)\ ?chunk$ 
  using state-head state-tail by simp
have ext:  $s \leq t$ 
  by (rule hash-ext-trans[OF ext-head ext-tail])
have counter:  $PQueryCounter\ t = PQueryCounter\ s$ 
  using counter-head counter-tail by simp
show ?thesis
  by (rule that[OF layers transcript state ext counter - xp-eq])
    (use head-chunk in simp-all)
qed

```

lemma verifier-query-round-after-index-trace-fri-head-value-aligned:

```

fixes s t :: ('f, 'a) protocol-channel-scheme
assumes f-fl-eq:  $f\text{-}fl = (b, rt) \# f\text{-}fl\text{-}tail$ 
and outcome:
  Some  $((\ ), t) \in$ 
    set-dist
      (execute
        (verifier-query-round-after-index-program
          fr f-fl f-final as fl final raw) s)
obtains idx openings xp xp-path xn xn-path where
  idx = index (to-nat raw)
  map opening-index openings = powers-scaled idx
  partial-authenticated-table fr (scale * clength) openings t
  openings  $\neq []$ 
  opening-value (hd openings) = xp
  fri-layer-opening-chunk (scale * clength) xp xp-path xn xn-path
     $([xp] @ xp\text{-}path @ [xn] @ xn\text{-}path)$ 

```

proof –

```

let ?idx = index (to-nat raw)
from outcome obtain fv s1 f-out s2 s3 c-out s4 where
  query-decommit:
    Some  $(fv, s1) \in$ 
      set-dist (execute (mmap (check-decommit-on-query fr ?idx)) s)
and trace-fri:

```

```

Some (f-out, s2) ∈
  set-dist (execute
    (mfold (?idx, hd fv, clength * scale, 1)
      (receive-query-commits f-fl)) s1)
and assert-trace:
Some (( ), s3) ∈
  set-dist
    (execute
      (assert (case f-out of (-, f-x, -, -) ⇒ f-x = f-final)) s2)
and comp-fri:
Some (c-out, s4) ∈
  set-dist (execute
    (mfold (?idx, cp-eval as fv (h ^ ?idx * shift),
      clength * scale, 1)
      (receive-query-commits fl)) s3)
and assert-comp:
Some (( ), t) ∈
  set-dist
    (execute
      (assert (case c-out of (-, x, -, -) ⇒ x = final)) s4)
unfolding verifier-query-round-after-index-program-def
by (auto simp: Let-def split: prod.splits elim!: set-dist-bindE)
have idx-in: ?idx ∈ query-sample-space
using index-less-query-sample-space
unfolding query-sample-space-def by simp
from check-decommit-on-query-authenticated-openings
  [OF idx-in query-decommit]
obtain openings where len-openings:
  length openings = length (powers-scaled ?idx)
and values-openings: map opening-value openings = fv
and idx-openings: map opening-index openings = powers-scaled ?idx
and table-s1:
  partial-authenticated-table fr (scale * clength) openings s1
by blast
from receive-query-commits-decomp-at
  [OF trace-fri[unfolded f-fl-eq], of 0]
obtain idx0 x0 len0 pw0 out1 s1' s-suc where
  head:
  Some (out1, s1') ∈
    set-dist
      (execute (fri-layer-opening-step (b, rt)
        (?idx, hd fv, clength * scale, 1)) s1)
by simp blast
from fri-layer-opening-step-outcome[OF head]
obtain xp xp-path xn xn-path x' where
  xp-eq: xp = hd fv
and chunk:
  fri-layer-opening-chunk (clength * scale) xp xp-path xn xn-path
    ([xp] @ xp-path @ [xn] @ xn-path)

```

```

  by blast
have s1-t:  $s1 \leq t$ 
proof -
  obtain idx1 x1 len1 pw1 where f-out-eq:
    f-out = (idx1, x1, len1, pw1)
  by (cases f-out)
have s1-s2:
   $s1 \leq s2$ 
  by (rule receive-query-commits-mfold-hash-extends
    [OF trace-fri[unfolded f-out-eq]])
have s3-eq:  $s3 = s2$ 
  using assert-trace unfolding assert-def f-out-eq
  by (cases x1 = f-final) (auto simp: throw-no-outcome)
have s3-s4:
   $s3 \leq s4$ 
  by (rule receive-query-commits-mfold-hash-extends[OF comp-fri])
have t-eq:  $t = s4$ 
  using assert-comp unfolding assert-def
  by (cases case c-out of (-, x, -, -)  $\Rightarrow$   $x = \text{final}$ )
  (auto simp: throw-no-outcome)
have s2-s4:  $s2 \leq s4$ 
  using s3-s4 unfolding s3-eq .
show ?thesis
  using hash-ext-trans[OF s1-s2 s2-s4] t-eq by simp
qed
have table-t:
  partial-authenticated-table fr (scale * clength) openings t
  by (rule partial-authenticated-table-mono[OF table-s1 s1-t])
have openings-nonempty: openings  $\neq$  []
  using len-openings powers-pos unfolding powers-scaled-def by auto
have hd-value: opening-value (hd openings) = hd fv
  using values-openings openings-nonempty
  by (cases openings; cases fv) simp-all
show ?thesis
  by (rule that[OF refl idx-openings table-t openings-nonempty])
  (use hd-value xp-eq chunk in <simp-all add: mult commute>)
qed

lemma verifier-query-round-after-index-trace-fri-head-layer-value-aligned:
  fixes s t :: ('f, 'a) protocol-channel-scheme
  assumes f-fl-eq: f-fl = (b, rt) # f-fl-tail
  and outcome:
    Some (( ), t)  $\in$ 
      set-dist
        (execute
          (verifier-query-round-after-index-program
            fr f-fl f-final as fl final raw) s)
  obtains idx openings query-chunk trace-layer-chunks composition-layer-chunks
    xp xp-path xn xn-path where

```

```

idx = index (to-nat raw)
query-round-fri-layer-transcripts idx (map snd f-fl) (map snd fl)
  query-chunk trace-layer-chunks composition-layer-chunks
PTranscript s = query-chunk @ PTranscript t
PState t = foldl concat (PState s) query-chunk
s ≤ t
PQueryCounter t = PQueryCounter s
map opening-index openings = powers-scaled idx
partial-authenticated-table fr (scale * clength) openings t
openings ≠ []
opening-value (hd openings) = xp
fri-layer-opening-chunk (scale * clength) xp xp-path xn xn-path
  (trace-layer-chunks ! 0)
proof –
  let ?idx = index (to-nat raw)
  from outcome obtain fv s1 f-out s2 s3 c-out s4 where
    query-decommit:
      Some (fv, s1) ∈
        set-dist (execute (mmap (check-decommit-on-query fr ?idx)) s)
    and trace-fri:
      Some (f-out, s2) ∈
        set-dist (execute
          (mfold (?idx, hd fv, clength * scale, 1)
            (receive-query-commits f-fl)) s1)
    and assert-trace:
      Some ((), s3) ∈
        set-dist
          (execute
            (assert (case f-out of (-, f-x, -, -) ⇒ f-x = f-final)) s2)
    and comp-fri:
      Some (c-out, s4) ∈
        set-dist (execute
          (mfold (?idx, cp-eval as fv (h ^ ?idx * shift),
            clength * scale, 1)
            (receive-query-commits fl)) s3)
    and assert-comp:
      Some ((), t) ∈
        set-dist
          (execute
            (assert (case c-out of (-, x, -, -) ⇒ x = final)) s4)
    unfolding verifier-query-round-after-index-program-def
    by (auto simp: Let-def split: prod.splits elim!: set-dist-bindE)
  have idx-in: ?idx ∈ query-sample-space
    using index-less-query-sample-space
    unfolding query-sample-space-def by simp
  from check-decommit-on-query-authenticated-openings
    [OF idx-in query-decommit]
  obtain openings where values-openings:
    map opening-value openings = fv

```

```

and idx-openings: map opening-index openings = powers-scaled ?idx
and table-s1:
  partial-authenticated-table fr (scale * clength) openings s1
by blast
from check-decommit-on-query-outcome[OF query-decommit]
obtain query-paths decommit-chunk where query-chunk-shape:
  query-decommitment-transcript ?idx fv query-paths decommit-chunk
and tr-s-s1:
  PTranscript s = decommit-chunk @ PTranscript s1
and st-s1:
  PState s1 = foldl concat (PState s) decommit-chunk
and ext-s-s1: s ≤ s1
and query-counter-s1: PQueryCounter s1 = PQueryCounter s
by blast
from receive-query-commits-head-layer-outcome[OF f-fl-eq trace-fri]
obtain trace-layer-chunks trace-fri-chunk xp xp-path xn xn-path where
  trace-layers:
    fri-layers-transcript (length f-fl) (clength * scale)
    trace-layer-chunks trace-fri-chunk
and tr-s1-s2:
  PTranscript s1 = trace-fri-chunk @ PTranscript s2
and st-s2:
  PState s2 = foldl concat (PState s1) trace-fri-chunk
and ext-s1-s2: s1 ≤ s2
and query-counter-s2: PQueryCounter s2 = PQueryCounter s1
and trace-layers-nonempty: trace-layer-chunks ≠ []
and xp-eq: xp = hd fv
and head-chunk:
  fri-layer-opening-chunk (clength * scale) xp xp-path xn xn-path
  (hd trace-layer-chunks)
by blast
have s3-eq: s3 = s2
  using assert-trace unfolding assert-def
  by (cases case f-out of (-, f-x, -, -) ⇒ f-x = f-final)
    (auto simp: throw-no-outcome)
from receive-query-commits-layers-outcome[OF comp-fri]
obtain composition-layer-chunks composition-fri-chunk where
  composition-layers:
    fri-layers-transcript (length fl) (clength * scale)
    composition-layer-chunks composition-fri-chunk
and tr-s3-s4:
  PTranscript s3 = composition-fri-chunk @ PTranscript s4
and st-s4:
  PState s4 = foldl concat (PState s3) composition-fri-chunk
and ext-s3-s4: s3 ≤ s4
and query-counter-s4: PQueryCounter s4 = PQueryCounter s3
by blast
have t-eq: t = s4
  using assert-comp unfolding assert-def

```

```

  by (cases case c-out of (-, x, -, -) ⇒ x = final)
    (auto simp: throw-no-outcome)
let ?query-chunk =
  decommit-chunk @ trace-fri-chunk @ composition-fri-chunk
have round-layers:
  query-round-fri-layer-transcripts ?idx (map snd f-fl) (map snd fl)
  ?query-chunk trace-layer-chunks composition-layer-chunks
unfolding query-round-fri-layer-transcripts-def
by (intro exI[of - decommit-chunk] exI[of - trace-fri-chunk]
    exI[of - composition-fri-chunk] exI[of - fv]
    exI[of - query-paths] conjI)
  (use query-chunk-shape trace-layers composition-layers in
    ⟨simp-all add: mult commute⟩)
have tr-t: PTranscript s = ?query-chunk @ PTranscript t
  using tr-s-s1 tr-s1-s2 tr-s3-s4 s3-eq t-eq by simp
have st-t: PState t = foldl concat (PState s) ?query-chunk
  using st-s1 st-s2 st-s4 s3-eq t-eq by simp
have ext-s-t: s ≤ t
  using ext-s-s1 ext-s1-s2 ext-s3-s4 t-eq unfolding s3-eq
  by (meson hash-ext-trans)
have query-counter-t: PQueryCounter t = PQueryCounter s
  using query-counter-s1 query-counter-s2 query-counter-s4
  unfolding s3-eq t-eq by simp
have table-t:
  partial-authenticated-table fr (scale * clength) openings t
  by (rule partial-authenticated-table-mono[OF table-s1])
  (use ext-s1-s2 ext-s3-s4 t-eq s3-eq in
    ⟨meson hash-ext-trans⟩)
have openings-nonempty: openings ≠ []
  using idx-openings powers-pos unfolding powers-scaled-def by auto
have hd-value: opening-value (hd openings) = hd fv
  using values-openings openings-nonempty
  by (cases openings; cases fv) simp-all
have head-chunk-nth:
  fri-layer-opening-chunk (scale * clength) xp xp-path xn xn-path
  (trace-layer-chunks ! 0)
  using head-chunk trace-layers-nonempty
  by (cases trace-layer-chunks) (simp-all add: mult commute)
show ?thesis
  by (rule that[OF refl round-layers tr-t st-t ext-s-t query-counter-t
    idx-openings table-t openings-nonempty - head-chunk-nth])
  (use hd-value xp-eq in simp)
qed

```

```

lemma verifier-query-round-program-trace-fri-head-value-aligned-with-lookup:
  fixes s t :: ('f, 'a) protocol-channel-scheme
  assumes f-fl-eq: f-fl = (b, rt) # f-fl-tail
  and outcome:
    Some ((), t) ∈

```



```

    set-dist
      (execute
        (verifier-query-round-program fr f-fl f-final as fl final) s)
obtains raw idx openings xp xp-path xn xn-path where
  idx = index (to-nat raw)
  map opening-index openings = powers-scaled idx
  partial-authenticated-table fr (scale * clength) openings t
  openings ≠ []
  opening-value (hd openings) = xp
  fri-layer-opening-chunk (scale * clength) xp xp-path xn xn-path
    ([xp] @ xp-path @ [xn] @ xn-path)
  fmlookup (HashMap t)
    (QueryIndexChallenge (PQueryCounter s) (PState s)) = Some raw
proof –
from outcome[unfolded verifier-query-round-program-alt-def]
obtain raw s0 where raw:
  Some (raw, s0) ∈
    set-dist (execute receive-query-index-challenge s)
and tail:
  Some ((), t) ∈
    set-dist
      (execute
        (verifier-query-round-after-index-program
          fr f-fl f-final as fl final raw) s0)
by (auto elim!: set-dist-bindE)
obtain idx openings xp xp-path xn xn-path where
  idx-eq: idx = index (to-nat raw)
and idxs: map opening-index openings = powers-scaled idx
and auth-table:
  partial-authenticated-table fr (scale * clength) openings t
and nonempty: openings ≠ []
and opening-value-eq: opening-value (hd openings) = xp
and chunk:
  fri-layer-opening-chunk (scale * clength) xp xp-path xn xn-path
    ([xp] @ xp-path @ [xn] @ xn-path)
using verifier-query-round-after-index-trace-fri-head-value-aligned
  [OF f-fl-eq tail]
by metis
have lookup-s0:
  fmlookup (HashMap s0)
    (QueryIndexChallenge (PQueryCounter s) (PState s)) = Some raw
using receive-query-index-challenge-outcome[OF raw] by simp
have s0-t: s0 ≤ t
using verifier-query-round-after-index-program-outcome[OF tail]
by simp
have lookup-t:
  fmlookup (HashMap t)
    (QueryIndexChallenge (PQueryCounter s) (PState s)) = Some raw
by (rule hash-extension-lookup[OF lookup-s0 s0-t])

```

```

show ?thesis
  by (rule that[OF idx-eq idxs auth-table nonempty opening-value-eq chunk lookup-t])
qed

lemma ntimes-verifier-query-rounds-selected-trace-fri-head-value-aligned-with-replay-at:
fixes query-state final-state :: ('f, 'a) protocol-channel-scheme
assumes f-fl-eq: f-fl = (b, rt) # f-fl-tail
and outcome:
  Some (results, final-state) ∈
    set-dist
      (execute
        (ntimes
          (verifier-query-round-program fr f-fl f-final as fl final)
          rounds)
        query-state)
and i-bound: i < rounds
and len-raw: length raw-idxs = rounds
and query-idxs-eq:
  query-idxs = map (λraw. index (to-nat raw)) raw-idxs
and len-query-chunks: length query-chunks = rounds
and transcript-query:
  PTranscript query-state =
    List.concat query-chunks @ PTranscript final-state
and query-chunk:
  ∧ i. i < rounds ⇒
    verifier-query-round-chunk (query-idxs ! i)
      (map snd f-fl) (map snd fl) (query-chunks ! i)
and replay-lookup:
  ∧ i. i < rounds ⇒
    fmlookup (HashMap final-state)
      (QueryIndexChallenge (PQueryCounter query-state + i)
        (state-after-query-chunks
          (PState query-state) query-chunks i)) =
      Some (raw-idxs ! i)
obtains openings xp xp-path xn xn-path where
  map opening-index openings = powers-scaled (query-idxs ! i)
  partial-authenticated-table fr (scale * clength) openings final-state
  openings ≠ []
  opening-value (hd openings) = xp
  fri-layer-opening-chunk (scale * clength) xp xp-path xn xn-path
    ([xp] @ xp-path @ [xn] @ xn-path)
proof –
let ?round = verifier-query-round-program fr f-fl f-final as fl final
from query-opening-ntimes-outcome-decomp-at[OF outcome i-bound]
obtain prefix x suffix s-i s-suc where
  prefix:
    Some (prefix, s-i) ∈
      set-dist (execute (ntimes ?round i) query-state)
and round:

```

Some $(x, s\text{-suc}) \in \text{set-dist } (\text{execute } ?\text{round } s\text{-}i)$
and *suffix*:
 Some $(\text{suffix}, \text{final-state}) \in$
 $\text{set-dist } (\text{execute } (\text{ntimes } ?\text{round } (\text{rounds} - \text{Suc } i)) \text{ } s\text{-suc})$
by *blast*
have *x-eq*: $x = ()$
by $(\text{cases } x) \text{ simp}$
show *?thesis*
proof (rule *query-opening-ntimes-prefix-transcript-state-alignment*
 $[\text{OF } \text{prefix}]$)
fix *prefix-query-idxs prefix-chunks*
assume *len-prefix-idxs*: $\text{length } \text{prefix-query-idxs} = i$
and *len-prefix-chunks*: $\text{length } \text{prefix-chunks} = i$
and *transcript-prefix*:
 $P\text{Transcript } \text{query-state} =$
 $\text{List.concat } \text{prefix-chunks } @ \text{ } P\text{Transcript } s\text{-}i$
and *state-i*:
 $P\text{State } s\text{-}i =$
 $\text{state-after-query-chunks } (P\text{State } \text{query-state}) \text{ prefix-chunks } i$
and *counter-i*:
 $P\text{QueryCounter } s\text{-}i = P\text{QueryCounter } \text{query-state} + i$
and *prefix-ext*: $\text{query-state} \leq s\text{-}i$
and *prefix-rounds*:
 $\bigwedge k. k < i \implies$
 $\text{verifier-query-round-chunk } (\text{prefix-query-idxs } ! k)$
 $(\text{map } \text{snd } f\text{-}fl) (\text{map } \text{snd } fl) (\text{prefix-chunks } ! k)$
show *?thesis*
proof (rule *verifier-query-round-program-outcome-with-authenticated-trace-openings*
 $[\text{OF } \text{round}[\text{unfolded } x\text{-}eq]]$)
fix *raw idx chunk round-openings*
assume *idx-eq*: $\text{idx} = \text{index } (\text{to-nat } \text{raw})$
and *chunk-shape*:
 $\text{verifier-query-round-chunk } \text{idx } (\text{map } \text{snd } f\text{-}fl) (\text{map } \text{snd } fl)$
 chunk
and *transcript-round*:
 $P\text{Transcript } s\text{-}i = \text{chunk } @ \text{ } P\text{Transcript } s\text{-suc}$
and *state-suc*:
 $P\text{State } s\text{-suc} = \text{foldl } \text{concat } (P\text{State } s\text{-}i) \text{ chunk}$
and *round-ext*: $s\text{-}i \leq s\text{-suc}$
and *counter-suc*:
 $P\text{QueryCounter } s\text{-suc} = \text{Suc } (P\text{QueryCounter } s\text{-}i)$
and *lookup-suc*:
 $\text{fmlookup } (\text{HashMap } s\text{-suc})$
 $(\text{QueryIndexChallenge } (P\text{QueryCounter } s\text{-}i) (P\text{State } s\text{-}i)) =$
 $\text{Some } \text{raw}$
and *len-round-openings*:
 $\text{length } \text{round-openings} = \text{length } (\text{powers-scaled } \text{idx})$
and *trace-indices*:
 $\text{map } \text{opening-index } \text{round-openings} = \text{powers-scaled } \text{idx}$

```

and trace-table-suc:
  partial-authenticated-table fr (scale * clength)
  round-openings s-suc
show ?thesis
proof (rule verifier-query-round-program-trace-fri-head-value-aligned-with-lookup
  [OF f-fl-eq round[unfolded x-eq]])
fix raw' idx' openings xp xp-path xn xn-path
assume idx'-eq: idx' = index (to-nat raw')
and opening-indices:
  map opening-index openings = powers-scaled idx'
and auth-table-suc:
  partial-authenticated-table fr (scale * clength) openings
  s-suc
and openings-nonempty: openings ≠ []
and opening-value-eq: opening-value (hd openings) = xp
and head-chunk:
  fri-layer-opening-chunk (scale * clength) xp xp-path xn
  xn-path ([xp] @ xp-path @ [xn] @ xn-path)
and lookup-suc':
  fmlookup (HashMap s-suc)
  (QueryIndexChallenge (PQueryCounter s-i) (PState s-i)) =
  Some raw'
have raw'-eq: raw' = raw
using lookup-suc lookup-suc' by simp
show ?thesis
proof (rule query-opening-ntimes-suffix-transcript-state-alignment
  [OF suffix])
fix suffix-query-idxs suffix-chunks
assume len-suffix-idxs:
  length suffix-query-idxs = rounds - Suc i
and len-suffix-chunks:
  length suffix-chunks = rounds - Suc i
and transcript-suffix:
  PTranscript s-suc =
  List.concat suffix-chunks @ PTranscript final-state
and suffix-state:
  PState final-state =
  state-after-query-chunks (PState s-suc) suffix-chunks
  (rounds - Suc i)
and suffix-counter:
  PQueryCounter final-state =
  PQueryCounter s-suc + (rounds - Suc i)
and suffix-ext: s-suc ≤ final-state
and suffix-rounds:
   $\bigwedge k. k < rounds - Suc i \implies$ 
  verifier-query-round-chunk (suffix-query-idxs ! k)
  (map snd f-fl) (map snd fl) (suffix-chunks ! k)
let ?selected-chunks = prefix-chunks @ chunk # suffix-chunks
have len-selected-chunks: length ?selected-chunks = rounds

```

```

using len-prefix-chunks len-suffix-chunks i-bound by simp
have selected-chunk-lens:
   $\bigwedge k. k < \text{rounds} \implies$ 
    length (?selected-chunks ! k) =
      verifier-query-round-transcript-length (query-idxs ! k)
        (map snd f-fl) (map snd fl)
proof -
  fix k
  assume k-bound: k < rounds
  show
    length (?selected-chunks ! k) =
      verifier-query-round-transcript-length (query-idxs ! k)
        (map snd f-fl) (map snd fl)
proof (cases k < i)
  case True
  have nth-eq: ?selected-chunks ! k = prefix-chunks ! k
    using True len-prefix-chunks by (simp add: nth-append)
  have
    length (prefix-chunks ! k) =
      verifier-query-round-transcript-length
        (prefix-query-idxs ! k) (map snd f-fl) (map snd fl)
    by (rule verifier-query-round-chunk-length
      [OF prefix-rounds[OF True]])
  then show ?thesis
    using nth-eq
    by (simp add:
      verifier-query-round-transcript-length-index-irrelevant)
next
  case False
  show ?thesis
  proof (cases k = i)
  case True
  have nth-eq: ?selected-chunks ! k = chunk
    using True len-prefix-chunks by (simp add: nth-append)
  have
    length chunk =
      verifier-query-round-transcript-length idx
        (map snd f-fl) (map snd fl)
    by (rule verifier-query-round-chunk-length[OF chunk-shape])
  then show ?thesis
    using nth-eq
    by (simp add:
      verifier-query-round-transcript-length-index-irrelevant)
next
  case False
  let ?k = k - Suc i
  have ge: Suc i ≤ k
    using <¬ k < i> False by linarith
  have k-bound': ?k < rounds - Suc i

```

```

    using ge k-bound by linarith
  have nth-eq: ?selected-chunks ! k = suffix-chunks ! ?k
    using ge len-prefix-chunks by (simp add: nth-append)
  have
    length (suffix-chunks ! ?k) =
      verifier-query-round-transcript-length
        (suffix-query-idxs ! ?k) (map snd f-fl) (map snd fl)
    by (rule verifier-query-round-chunk-length
        [OF suffix-rounds[OF k-bound]])
  then show ?thesis
    using nth-eq
    by (simp add:
        verifier-query-round-transcript-length-index-irrelevant)
qed
qed
qed
have query-chunk-lens:
   $\bigwedge k. k < rounds \implies$ 
    length (query-chunks ! k) =
      verifier-query-round-transcript-length (query-idxs ! k)
        (map snd f-fl) (map snd fl)
    using query-chunk verifier-query-round-chunk-length by blast
have transcript-all:
  PTranscript query-state =
    List.concat ?selected-chunks @ PTranscript final-state
    using transcript-prefix transcript-round transcript-suffix by simp
have concat-eq:
  List.concat ?selected-chunks = List.concat query-chunks
    using transcript-all transcript-query by simp
have chunks-eq: ?selected-chunks = query-chunks
proof -
  have same-lens:
     $\bigwedge k. k < length \text{ ?selected-chunks } \implies$ 
      length (?selected-chunks ! k) = length (query-chunks ! k)
    using selected-chunk-lens query-chunk-lens len-selected-chunks
    by simp
  have concat-eq-empty:
    List.concat ?selected-chunks @ [] = List.concat query-chunks
    using concat-eq by simp
  have len-eq: length ?selected-chunks = length query-chunks
    using len-selected-chunks len-query-chunks by simp
  have ?selected-chunks = query-chunks  $\wedge$  ([] :: 'f list) = []
    by (rule concat-append-eq-concat-same-chunk-lengths
        [OF len-eq same-lens concat-eq-empty])
  then show ?thesis
    by simp
qed
have prefix-state-eq:
  state-after-query-chunks (PState query-state) prefix-chunks i =

```

```

      state-after-query-chunks (PState query-state) query-chunks i
proof -
  have
    state-after-query-chunks (PState query-state)
      ?selected-chunks i =
      state-after-query-chunks (PState query-state)
        prefix-chunks i
    by (rule state-after-query-chunks-append-prefix
      [OF len-prefix-chunks])
  then show ?thesis
    using chunks-eq by simp
qed
have lookup-final:
  fmlookup (HashMap final-state)
    (QueryIndexChallenge (PQueryCounter query-state + i)
      (state-after-query-chunks
        (PState query-state) query-chunks i)) =
    Some raw
  using hash-extension-lookup[OF lookup-suc suffix-ext]
  unfolding counter-i state-i prefix-state-eq .
have raw-eq: raw = raw-idxs ! i
  using lookup-final replay-lookup[OF i-bound] by simp
have idx'-eq-query: idx' = query-idxs ! i
  using idx'-eq raw'-eq raw-eq query-idxs-eq len-raw i-bound
  by simp
have auth-table-final:
  partial-authenticated-table fr (scale * clength) openings
    final-state
  by (rule partial-authenticated-table-mono
    [OF auth-table-suc suffix-ext])
show ?thesis
  by (rule that[OF - auth-table-final openings-nonempty
    opening-value-eq head-chunk])
    (use opening-indices idx'-eq-query in simp)
qed
qed
qed
qed
qed

```

definition *trace-fri-header-tied-value-aligned-base-opening-conflict*
 $\text{:: } ('f, 'a) \text{ protocol-channel-scheme} \Rightarrow$
 $(\text{unit list} \times ('f, 'a) \text{ protocol-channel-scheme}) \text{ option} \Rightarrow \text{bool}$
where
trace-fri-header-tied-value-aligned-base-opening-conflict s out
 $\iff$
 $(\exists \text{ trace-roots trace-bs trace-final dg composition-roots}$
 $\text{composition-bs composition-final fri-query-idxs trace-round-layers}$
 $\text{composition-round-layers fr as rest trace-openings trace-table}$

$round_idx\ xp\ xp_path\ xn\ xn_path\ result\ final_state.$
 $out = Some\ (result,\ final_state) \wedge$
 $accepted_fri_opening_transcript\ s\ out\ trace_roots\ trace_bs\ trace_final$
 $\quad dg\ composition_roots\ composition_bs\ composition_final\ fri_query_idxs$
 $\quad trace_round_layers\ composition_round_layers \wedge$
 $verifier_header_transcript\ s\ fr\ trace_roots\ trace_final\ as\ dg$
 $\quad composition_roots\ composition_final\ rest \wedge$
 $accepted_with_partial_trace_openings\ s\ out\ fr\ fri_query_idxs$
 $\quad trace_openings \wedge$
 $partial_trace_table_candidate\ trace_table\ trace_openings \wedge$
 $\neg\ trace_table_low_degree\ trace_table \wedge$
 $round_idx < length\ fri_query_idxs \wedge$
 $0 < length\ trace_bs \wedge$
 $trace_openings ! round_idx \neq [] \wedge$
 $opening_index\ (hd\ (trace_openings ! round_idx)) =$
 $\quad fri_evidence_layer_idx\ trace_roots\ fri_query_idxs\ round_idx\ 0 \wedge$
 $opening_value\ (hd\ (trace_openings ! round_idx)) = xp \wedge$
 $fri_layer_step_evidence$
 $\quad (trace_roots ! 0)$
 $\quad (trace_bs ! 0)$
 $\quad (fri_evidence_layer_len\ trace_roots\ 0)$
 $\quad (fri_evidence_layer_idx\ trace_roots\ fri_query_idxs\ round_idx\ 0)$
 $\quad 1$
 $\quad (fri_sibling_index\ (fri_evidence_layer_len\ trace_roots\ 0)$
 $\quad \quad (fri_evidence_layer_idx\ trace_roots\ fri_query_idxs\ round_idx\ 0))$
 $\quad xp\ xp_path\ xn\ xn_path$
 $\quad (fri_evidence_next_idx\ trace_roots\ fri_query_idxs\ round_idx\ 0)$
 $\quad (fri_evidence_next_value\ trace_roots\ trace_bs\ fri_query_idxs$
 $\quad \quad round_idx\ 0\ xp\ xn)$
 $\quad (trace_round_layers ! round_idx ! 0) \wedge$
 $\neg\ fri_opening_matches_table$
 $\quad (fri_evidence_layer_len\ trace_roots\ 0)$
 $\quad (fri_evidence_layer_idx\ trace_roots\ fri_query_idxs\ round_idx\ 0)$
 $\quad trace_table\ xp\ xn)$

definition *trace-fri-header-tied-value-aligned-sibling-mismatch*

$:: (f,\ 'a)\ protocol_channel_scheme \Rightarrow$
 $\quad (unit\ list \times (f,\ 'a)\ protocol_channel_scheme)\ option \Rightarrow bool$

where

$trace_fri_header_tied_value_aligned_sibling_mismatch\ s\ out$
 $\longleftrightarrow$
 $(\exists\ trace_roots\ trace_bs\ trace_final\ dg\ composition_roots$
 $\quad composition_bs\ composition_final\ fri_query_idxs\ trace_round_layers$
 $\quad composition_round_layers\ fr\ as\ rest\ trace_openings\ trace_table$
 $\quad round_idx\ xp\ xp_path\ xn\ xn_path\ result\ final_state.$
 $out = Some\ (result,\ final_state) \wedge$
 $accepted_fri_opening_transcript\ s\ out\ trace_roots\ trace_bs\ trace_final$
 $\quad dg\ composition_roots\ composition_bs\ composition_final\ fri_query_idxs$
 $\quad trace_round_layers\ composition_round_layers \wedge$

verifier-header-transcript s fr trace-roots trace-final as dg
composition-roots composition-final rest $\wedge$
accepted-with-partial-trace-openings s out fr fri-query-idxs
trace-openings $\wedge$
partial-trace-table-candidate trace-table trace-openings $\wedge$
 $\neg$ *trace-table-low-degree trace-table* $\wedge$
round-idx < length fri-query-idxs $\wedge$
0 < length trace-bs $\wedge$
trace-openings ! round-idx $\neq []$ $\wedge$
opening-index (hd (trace-openings ! round-idx)) =
fri-evidence-layer-idx trace-roots fri-query-idxs round-idx 0 $\wedge$
opening-value (hd (trace-openings ! round-idx)) = xp $\wedge$
fri-layer-step-evidence
(trace-roots ! 0)
(trace-bs ! 0)
(fri-evidence-layer-len trace-roots 0)
(fri-evidence-layer-idx trace-roots fri-query-idxs round-idx 0)
1
(fri-sibling-index (fri-evidence-layer-len trace-roots 0)
(fri-evidence-layer-idx trace-roots fri-query-idxs round-idx 0))
xp xp-path xn xn-path
(fri-evidence-next-idx trace-roots fri-query-idxs round-idx 0)
(fri-evidence-next-value trace-roots trace-bs fri-query-idxs
round-idx 0 xp xn)
(trace-round-layers ! round-idx ! 0) $\wedge$
trace-table !
fri-sibling-index (fri-evidence-layer-len trace-roots 0)
(fri-evidence-layer-idx trace-roots fri-query-idxs round-idx 0)
 $\neq xn$)

lemma *trace-fri-header-tied-value-aligned-base-imp-sampled-base:*

assumes

trace-fri-header-tied-value-aligned-base-opening-conflict s out

shows *trace-fri-header-tied-sampled-base-opening-conflict s out*

proof —

from *assms* **obtain** *trace-roots trace-bs trace-final dg composition-roots*

composition-bs composition-final fri-query-idxs trace-round-layers

composition-round-layers fr as rest trace-openings trace-table

round-idx xp xp-path xn xn-path result final-state

where *out-eq: out = Some (result, final-state)*

and *fri-openings:*

accepted-fri-opening-transcript s out trace-roots trace-bs trace-final

dg composition-roots composition-bs composition-final fri-query-idxs

trace-round-layers composition-round-layers

and *header:*

verifier-header-transcript s fr trace-roots trace-final as dg

composition-roots composition-final rest

and *partial:*

accepted-with-partial-trace-openings s out fr fri-query-idxs

```

    trace-openings
  and cand: partial-trace-table-candidate trace-table trace-openings
  and not-low:  $\neg$  trace-table-low-degree trace-table
  and round-bound: round-idx < length fri-query-idxs
  and trace-bs-nonempty: 0 < length trace-bs
  and openings-nonempty: trace-openings ! round-idx  $\neq$  []
  and hd-idx:
    opening-index (hd (trace-openings ! round-idx)) =
      fri-evidence-layer-idx trace-roots fri-query-idxs round-idx 0
  and hd-value: opening-value (hd (trace-openings ! round-idx)) = xp
  and step:
    fri-layer-step-evidence
      (trace-roots ! 0)
      (trace-bs ! 0)
      (fri-evidence-layer-len trace-roots 0)
      (fri-evidence-layer-idx trace-roots fri-query-idxs round-idx 0)
      1
      (fri-sibling-index (fri-evidence-layer-len trace-roots 0)
        (fri-evidence-layer-idx trace-roots fri-query-idxs round-idx 0))
      xp xp-path xn xn-path
      (fri-evidence-next-idx trace-roots fri-query-idxs round-idx 0)
      (fri-evidence-next-value trace-roots trace-bs fri-query-idxs
        round-idx 0 xp xn)
      (trace-round-layers ! round-idx ! 0)
  and no-match:
     $\neg$  fri-opening-matches-table
      (fri-evidence-layer-len trace-roots 0)
      (fri-evidence-layer-idx trace-roots fri-query-idxs round-idx 0)
      trace-table xp xn
  unfolding
    trace-fri-header-tied-value-aligned-base-opening-conflict-def
  by auto
  have base:
    generic-fri-sampled-base-opening-conflict trace-table trace-roots
      trace-bs fri-query-idxs trace-round-layers
  unfolding generic-fri-sampled-base-opening-conflict-def
  by (intro exI[of - round-idx] exI[of - xp] exI[of - xp-path]
    exI[of - xn] exI[of - xn-path] conjI)
    (use round-bound trace-bs-nonempty step no-match in auto)
  show ?thesis
  unfolding trace-fri-header-tied-sampled-base-opening-conflict-def
  by (intro exI conjI)
    (use out-eq fri-openings header partial cand not-low base in auto)
qed

```

lemma trace-fri-header-tied-value-aligned-base-imp-sibling-mismatch:
assumes
 trace-fri-header-tied-value-aligned-base-opening-conflict s out
shows trace-fri-header-tied-value-aligned-sibling-mismatch s out

proof –

from *assms* **obtain** *trace-roots trace-bs trace-final dg composition-roots composition-bs composition-final fri-query-idxs trace-round-layers composition-round-layers fr as rest trace-openings trace-table round-idx xp xp-path xn xn-path result final-state*

where *out-eq*: *out* = *Some (result, final-state)*

and *fri-openings*:
accepted-fri-opening-transcript s out trace-roots trace-bs trace-final dg composition-roots composition-bs composition-final fri-query-idxs trace-round-layers composition-round-layers

and *header*:
verifier-header-transcript s fr trace-roots trace-final as dg composition-roots composition-final rest

and *partial*:
accepted-with-partial-trace-openings s out fr fri-query-idxs trace-openings

and *cand*: *partial-trace-table-candidate trace-table trace-openings*

and *not-low*: $\neg$ *trace-table-low-degree trace-table*

and *round-bound*: *round-idx* < *length fri-query-idxs*

and *trace-bs-nonempty*: *0* < *length trace-bs*

and *openings-nonempty*: *trace-openings* ! *round-idx* $\neq$ []

and *hd-idx*:
opening-index (hd (trace-openings ! round-idx)) = fri-evidence-layer-idx trace-roots fri-query-idxs round-idx 0

and *hd-value*: *opening-value (hd (trace-openings ! round-idx)) = xp*

and *step*:
fri-layer-step-evidence
(trace-roots ! 0)
(trace-bs ! 0)
(fri-evidence-layer-len trace-roots 0)
(fri-evidence-layer-idx trace-roots fri-query-idxs round-idx 0)
1
(fri-sibling-index (fri-evidence-layer-len trace-roots 0) (fri-evidence-layer-idx trace-roots fri-query-idxs round-idx 0))
xp xp-path xn xn-path
(fri-evidence-next-idx trace-roots fri-query-idxs round-idx 0)
(fri-evidence-next-value trace-roots trace-bs fri-query-idxs round-idx 0 xp xn)
(trace-round-layers ! round-idx ! 0)

and *no-match*:
 $\neg$ *fri-opening-matches-table*
(fri-evidence-layer-len trace-roots 0)
(fri-evidence-layer-idx trace-roots fri-query-idxs round-idx 0)
trace-table xp xn

unfolding
trace-fri-header-tied-value-aligned-base-opening-conflict-def

by *auto*

let *?idx* =
fri-evidence-layer-idx trace-roots fri-query-idxs round-idx 0

```

let ?len = fri-evidence-layer-len trace-roots 0
let ?opn = hd (trace-openings ! round-idx)
have round-bound-rounds: round-idx < rounds
  using round-bound-accepted-fri-opening-transcript-shapes(7)[OF fri-openings]
  by simp
have opn-in: ?opn ∈ set (trace-openings ! round-idx)
  using openings-nonempty by simp
have table-len: length trace-table = scale * clength
  by (rule partial-trace-table-candidateD(1)
    [OF cand round-bound-rounds opn-in])
have idx-bound-eval: ?idx < scale * clength
  using partial-trace-table-candidateD(2)
    [OF cand round-bound-rounds opn-in] hd-idx
  by simp
have table-left: trace-table ! ?idx = xp
  using partial-trace-table-candidateD(3)
    [OF cand round-bound-rounds opn-in] hd-idx hd-value
  by simp
have roots-nonempty: 0 < length trace-roots
  using accepted-fri-opening-transcript-shapes(2)[OF fri-openings]
    trace-bs-nonempty
  by simp
have len-eq: ?len = scale * clength
  unfolding fri-evidence-layer-len-def
  using fri-layer-lengths-nth-div[of 0 length trace-roots
    clength * scale] roots-nonempty
  by (simp add: mult.commute)
have sibling-mismatch:
  trace-table ! fri-sibling-index ?len ?idx ≠ xn
proof
  assume sibling-eq: trace-table ! fri-sibling-index ?len ?idx = xn
  have fri-opening-matches-table ?len ?idx trace-table xp xn
    unfolding fri-opening-matches-table-def
    using idx-bound-eval table-len table-left sibling-eq len-eq
    by (simp add: mult.commute)
  then show False
    using no-match by contradiction
qed
show ?thesis
  unfolding trace-fri-header-tied-value-aligned-sibling-mismatch-def
  by (intro exI conjI)
    (use out-eq fri-openings header partial cand not-low round-bound
      trace-bs-nonempty openings-nonempty hd-idx hd-value step
      sibling-mismatch in auto)
qed

lemma wp-trace-fri-header-tied-value-aligned-base-bound-from-sibling-mismatch:
  assumes mismatch-bound:
    wp-event verify-monad

```

$(\text{trace-fri-header-tied-value-aligned-sibling-mismatch } s) \ s \leq M$
shows
 $\text{wp-event verify-monad}$
 $(\text{trace-fri-header-tied-value-aligned-base-opening-conflict } s) \ s \leq M$
proof –
have $\text{wp-event verify-monad}$
 $(\text{trace-fri-header-tied-value-aligned-base-opening-conflict } s) \ s \leq$
 $\text{wp-event verify-monad}$
 $(\text{trace-fri-header-tied-value-aligned-sibling-mismatch } s) \ s$
by $(\text{rule wp-event-mono})$
 $(\text{rule trace-fri-header-tied-value-aligned-base-imp-sibling-mismatch})$
then show $?thesis$
by $(\text{rule order-trans}[OF - \text{mismatch-bound}])$
qed

definition $\text{trace-fri-header-tied-sibling-mismatch-structural-gap}$
 $:: (\text{'f}, \text{'a}) \text{ protocol-channel-scheme} \Rightarrow$
 $(\text{unit list} \times (\text{'f}, \text{'a}) \text{ protocol-channel-scheme}) \text{ option} \Rightarrow \text{bool}$
where
 $\text{trace-fri-header-tied-sibling-mismatch-structural-gap } s \text{ out}$
 $\longleftrightarrow$
 $\text{trace-fri-header-tied-value-aligned-sibling-mismatch } s \text{ out} \wedge$
 $\neg \text{partial-merkle-inconsistency-bad } s \text{ out}$

lemma $\text{trace-fri-header-tied-value-aligned-sibling-mismatch-imp-partial-merkle-or-structural-gap}$:
assumes mismatch :
 $\text{trace-fri-header-tied-value-aligned-sibling-mismatch } s \text{ out}$
shows
 $\text{partial-merkle-inconsistency-bad } s \text{ out} \vee$
 $\text{trace-fri-header-tied-sibling-mismatch-structural-gap } s \text{ out}$
proof –
show $?thesis$
using mismatch
unfolding $\text{trace-fri-header-tied-sibling-mismatch-structural-gap-def}$
by blast
qed

lemma $\text{wp-trace-fri-header-tied-value-aligned-sibling-mismatch-bound-from-partial-merkle-and-structural-gap}$:
fixes $P \ G :: \text{prob}$
assumes $\text{partial-merkle-bound}$:
 $\text{wp-event verify-monad } (\text{partial-merkle-inconsistency-bad } s) \ s \leq P$
and gap-bound :
 $\text{wp-event verify-monad}$
 $(\text{trace-fri-header-tied-sibling-mismatch-structural-gap } s) \ s \leq G$
shows
 $\text{wp-event verify-monad}$
 $(\text{trace-fri-header-tied-value-aligned-sibling-mismatch } s) \ s \leq P + G$
proof –
have $\text{wp-event verify-monad}$

$(\text{trace-fri-header-tied-value-aligned-sibling-mismatch } s) \ s \leq$
 $\text{wp-event verify-monad}$
 $(\lambda \text{out. partial-merkle-inconsistency-bad } s \ \text{out} \ \vee$
 $\text{trace-fri-header-tied-sibling-mismatch-structural-gap } s \ \text{out}) \ s$
by (rule wp-event-mono)
(rule
 $\text{trace-fri-header-tied-value-aligned-sibling-mismatch-imp-partial-merkle-or-structural-gap}$)
also have ... $\leq$
 $\text{wp-event verify-monad (partial-merkle-inconsistency-bad } s) \ s +$
 $\text{wp-event verify-monad}$
 $(\text{trace-fri-header-tied-sibling-mismatch-structural-gap } s) \ s$
by (rule wp-event-union-bound)
also have ... $\leq P + G$
by (intro add-mono partial-merkle-bound gap-bound)
finally show ?thesis .
qed

definition $\text{trace-fri-header-tied-base-value-alignment-gap}$
 $:: (\text{'f, 'a}) \text{protocol-channel-scheme} \Rightarrow$
 $(\text{unit list} \times (\text{'f, 'a}) \text{protocol-channel-scheme}) \text{option} \Rightarrow \text{bool}$
where
 $\text{trace-fri-header-tied-base-value-alignment-gap } s \ \text{out} \longleftrightarrow$
 $\text{trace-fri-header-tied-sampled-base-opening-conflict } s \ \text{out} \wedge$
 $\neg \text{trace-fri-header-tied-value-aligned-base-opening-conflict } s \ \text{out}$

lemma $\text{trace-fri-header-tied-sampled-base-imp-value-aligned-or-gap}$:
assumes $\text{trace-fri-header-tied-sampled-base-opening-conflict } s \ \text{out}$
shows
 $\text{trace-fri-header-tied-value-aligned-base-opening-conflict } s \ \text{out} \vee$
 $\text{trace-fri-header-tied-base-value-alignment-gap } s \ \text{out}$
using *assms*
unfolding $\text{trace-fri-header-tied-base-value-alignment-gap-def}$
by *blast*

lemma $\text{wp-trace-fri-header-tied-sampled-base-bound-from-value-aligned-and-gap}$:
fixes $V \ G :: \text{prob}$
assumes $\text{value-aligned-bound}$:
 $\text{wp-event verify-monad}$
 $(\text{trace-fri-header-tied-value-aligned-base-opening-conflict } s) \ s \leq V$
and gap-bound :
 $\text{wp-event verify-monad}$
 $(\text{trace-fri-header-tied-base-value-alignment-gap } s) \ s \leq G$
shows
 $\text{wp-event verify-monad}$
 $(\text{trace-fri-header-tied-sampled-base-opening-conflict } s) \ s \leq V + G$
proof –
have $\text{wp-event verify-monad}$
 $(\text{trace-fri-header-tied-sampled-base-opening-conflict } s) \ s \leq$
 $\text{wp-event verify-monad}$

```

    (λout.
      trace-fri-header-tied-value-aligned-base-opening-conflict s out ∨
      trace-fri-header-tied-base-value-alignment-gap s out) s
  by (rule wp-event-mono)
    (rule trace-fri-header-tied-sampled-base-imp-value-aligned-or-gap)
  also have ... ≤
    wp-event verify-monad
      (trace-fri-header-tied-value-aligned-base-opening-conflict s) s +
    wp-event verify-monad
      (trace-fri-header-tied-base-value-alignment-gap s) s
  by (rule wp-event-union-bound)
  also have ... ≤ V + G
    by (intro add-mono value-aligned-bound gap-bound)
  finally show ?thesis .
qed

lemma wp-trace-fri-header-tied-sampled-base-bound-from-sibling-mismatch-and-gap:
  fixes M G :: prob
  assumes mismatch-bound:
    wp-event verify-monad
      (trace-fri-header-tied-value-aligned-sibling-mismatch s) s ≤ M
  and gap-bound:
    wp-event verify-monad
      (trace-fri-header-tied-base-value-alignment-gap s) s ≤ G
  shows
    wp-event verify-monad
      (trace-fri-header-tied-sampled-base-opening-conflict s) s ≤ M + G
  by (rule
    wp-trace-fri-header-tied-sampled-base-bound-from-value-aligned-and-gap)
  (rule wp-trace-fri-header-tied-value-aligned-base-bound-from-sibling-mismatch
    [OF mismatch-bound], rule gap-bound)

end

end

```

```

theory Soundness-FRI-Trace-Residuals
  imports
    Soundness-FRI-Value-Aligned-Events
    Soundness-FRI-Final-Step-Replay
    Soundness-FRI-Recorded-Chunk-Replay
begin

```

Narrow residual refinements for the header-tied trace FRI route. These predicates name the remaining replay gaps for non-base FRI branches without changing the protocol or exporting broad selected-step replay lemmas.

```

context soundness
begin

```

definition *trace-fri-reachable-header-tie-gap*

$:: ('f, 'a) \text{ protocol-channel-scheme} \Rightarrow$
 $(\text{unit list} \times ('f, 'a) \text{ protocol-channel-scheme}) \text{ option} \Rightarrow \text{bool}$

where

$\text{trace-fri-reachable-header-tie-gap } s \text{ out} \longleftrightarrow$
 $\text{trace-fri-bad-with-reachable-partial-candidate } s \text{ out} \wedge$
 $\neg \text{trace-fri-bad-with-header-tied-partial-candidate } s \text{ out}$

definition *trace-fri-header-tied-candidate-transfer-gap*

$:: ('f, 'a) \text{ protocol-channel-scheme} \Rightarrow$
 $(\text{unit list} \times ('f, 'a) \text{ protocol-channel-scheme}) \text{ option} \Rightarrow \text{bool}$

where

$\text{trace-fri-header-tied-candidate-transfer-gap } s \text{ out} \longleftrightarrow$
 $(\exists \text{ trace-roots trace-bs trace-final fri-dg composition-roots}$
 $\text{composition-bs composition-final fri-query-idxs trace-round-layers}$
 $\text{composition-round-layers fr as rest trace-openings trace-table.}$
 $\text{trace-fri-bad-with-reachable-partial-candidate } s \text{ out} \wedge$
 $\text{accepted-fri-opening-transcript } s \text{ out trace-roots trace-bs}$
 $\text{trace-final fri-dg composition-roots composition-bs}$
 $\text{composition-final fri-query-idxs trace-round-layers}$
 $\text{composition-round-layers} \wedge$
 $\text{verifier-header-transcript } s \text{ fr trace-roots trace-final as fri-dg}$
 $\text{composition-roots composition-final rest} \wedge$
 $\text{accepted-with-partial-trace-openings } s \text{ out fr fri-query-idxs}$
 $\text{trace-openings} \wedge$
 $\text{partial-trace-table-candidate trace-table trace-openings} \wedge$
 $\text{trace-table-low-degree trace-table})$

definition *trace-fri-header-tied-candidate-available-gap*

$:: ('f, 'a) \text{ protocol-channel-scheme} \Rightarrow$
 $(\text{unit list} \times ('f, 'a) \text{ protocol-channel-scheme}) \text{ option} \Rightarrow \text{bool}$

where

$\text{trace-fri-header-tied-candidate-available-gap } s \text{ out} \longleftrightarrow$
 $(\exists \text{ trace-roots trace-bs trace-final fri-dg composition-roots}$
 $\text{composition-bs composition-final fri-query-idxs trace-round-layers}$
 $\text{composition-round-layers fr as rest trace-openings trace-table.}$
 $\text{trace-fri-reachable-header-tie-gap } s \text{ out} \wedge$
 $\text{accepted-fri-opening-transcript } s \text{ out trace-roots trace-bs}$
 $\text{trace-final fri-dg composition-roots composition-bs}$
 $\text{composition-final fri-query-idxs trace-round-layers}$
 $\text{composition-round-layers} \wedge$
 $\text{verifier-header-transcript } s \text{ fr trace-roots trace-final as fri-dg}$
 $\text{composition-roots composition-final rest} \wedge$
 $\text{accepted-with-partial-trace-openings } s \text{ out fr fri-query-idxs}$
 $\text{trace-openings} \wedge$
 $\text{partial-trace-table-candidate trace-table trace-openings})$

lemma *trace-fri-zero-round-final-obstruction-imp-header-tied-or-gap:*


```

assumes trace-fri-zero-round-final-obstruction s out
shows
  trace-fri-bad-with-header-tied-partial-candidate s out  $\vee$ 
  trace-fri-reachable-header-tie-gap s out
proof –
  have reachable:
    trace-fri-bad-with-reachable-partial-candidate s out
  proof –
    from assms obtain trace-roots trace-bs trace-final dg
      composition-roots composition-bs composition-final fri-query-idxs
      trace-round-layers composition-round-layers fr candidate-query-idxs
      trace-openings trace-table
    where evidence:
      trace-fri-partial-candidate-opening-evidence s out trace-roots
      trace-bs trace-final dg composition-roots composition-bs
      composition-final fri-query-idxs trace-round-layers
      composition-round-layers fr candidate-query-idxs trace-openings
      trace-table
    unfolding trace-fri-zero-round-final-obstruction-def by blast
    have partial:
      trace-fri-partial-candidate-evidence s out fr candidate-query-idxs
      trace-openings trace-table
    using trace-fri-partial-candidate-opening-evidenceD(2)[OF evidence] .
    show ?thesis
    unfolding trace-fri-bad-with-reachable-partial-candidate-def
    by (intro exI conjI)
      (rule trace-fri-partial-candidate-evidenceD(1)[OF partial],
       rule trace-fri-partial-candidate-evidenceD(2)[OF partial],
       rule trace-fri-partial-candidate-evidenceD(3)[OF partial])
  qed
show ?thesis
proof (cases trace-fri-bad-with-header-tied-partial-candidate s out)
  case True
    then show ?thesis by simp
  next
    case False
    then have trace-fri-reachable-header-tie-gap s out
      unfolding trace-fri-reachable-header-tie-gap-def
      using reachable by simp
    then show ?thesis by simp
  qed
qed

lemma wp-trace-fri-zero-round-final-obstruction-bound-from-header-tied-and-gap:
  fixes H G :: prob
  assumes header-bound:
    wp-event verify-monad
    (trace-fri-bad-with-header-tied-partial-candidate s)  $s \leq H$ 
  and gap-bound:

```

$wp\text{-event verify-monad}$
 $(\text{trace-fri-reachable-header-tie-gap } s) \ s \leq G$
shows
 $wp\text{-event verify-monad}$
 $(\text{trace-fri-zero-round-final-obstruction } s) \ s \leq H + G$
proof –
have $wp\text{-event verify-monad}$
 $(\text{trace-fri-zero-round-final-obstruction } s) \ s \leq$
 $wp\text{-event verify-monad}$
 $(\lambda out.$
 $\text{trace-fri-bad-with-header-tied-partial-candidate } s \ out \ \vee$
 $\text{trace-fri-reachable-header-tie-gap } s \ out) \ s$
by $(\text{rule } wp\text{-event-mono})$
 $(\text{rule } \text{trace-fri-zero-round-final-obstruction-imp-header-tied-or-gap})$
also have $\dots \leq$
 $wp\text{-event verify-monad}$
 $(\text{trace-fri-bad-with-header-tied-partial-candidate } s) \ s +$
 $wp\text{-event verify-monad } (\text{trace-fri-reachable-header-tie-gap } s) \ s$
by $(\text{rule } wp\text{-event-union-bound})$
also have $\dots \leq H + G$
by $(\text{rule } \text{add-mono}[OF \ \text{header-bound } \text{gap-bound}])$
finally show $?thesis$.
qed

definition $\text{trace-fri-header-tied-reduction}$
 $:: (\text{'f}, \text{'a}) \ \text{protocol-channel-scheme} \Rightarrow \text{bool}$
where

$\text{trace-fri-header-tied-reduction } s \longleftrightarrow$
 $wp\text{-event verify-monad}$
 $(\text{trace-fri-bad-with-header-tied-partial-candidate } s) \ s \leq$
 trace-fri-error

lemma $\text{trace-fri-header-tied-reductionD}$:
assumes $\text{trace-fri-header-tied-reduction } s$
shows
 $wp\text{-event verify-monad}$
 $(\text{trace-fri-bad-with-header-tied-partial-candidate } s) \ s \leq$
 trace-fri-error
using assms **unfolding** $\text{trace-fri-header-tied-reduction-def}$ **by** simp

lemma $\text{trace-fri-header-tied-reduction-staged-bound}$:
assumes reductions :
 $\bigwedge \text{data } \text{attacker-state}.$
 $\text{Some } (\text{data}, \text{attacker-state}) \in$
 $\text{set-dist } (\text{execute } (\text{checked-staged-transcript-program } A)$
 $\text{adversary-initial-state}) \implies$
 $\text{trace-fri-header-tied-reduction}$
 $(\text{verifier-state-from-adversary } \text{attacker-state}$
 $(\text{staged-proof-transcript } \text{data}))$

shows

wp-event (checked-staged-security-experiment-with-data-state A)
(staged-security-with-data-state-verifier-event
trace-fri-bad-with-header-tied-partial-candidate)
adversary-initial-state ≤ trace-fri-error
by (*rule trace-fri-bad-with-header-tied-partial-candidates-staged-bound*)
(rule trace-fri-header-tied-reductionD[OF reductions])

lemma *trace-fri-header-tied-reduction-from-reachable-reduction:*

assumes *trace-fri-reachable-partial-candidate-reduction-assumption s*
shows *trace-fri-header-tied-reduction s*

proof –

have *reachable:*

wp-event verify-monad
(trace-fri-bad-with-reachable-partial-candidate s) s ≤
trace-fri-error

using *assms*

unfolding *trace-fri-reachable-partial-candidate-reduction-assumption-def*

by *simp*

have *verifier-tied:*

wp-event verify-monad
(trace-fri-bad-with-verifier-tied-partial-candidate s) s ≤
trace-fri-error

by (*rule order-trans[OF - reachable]*)

(rule wp-event-mono,
rule trace-fri-bad-with-verifier-tied-imp-reachable-partial-candidate)

show *?thesis*

unfolding *trace-fri-header-tied-reduction-def*

by (*rule order-trans[OF - verifier-tied]*)

(rule wp-event-mono,
rule trace-fri-bad-with-header-tied-imp-verifier-tied)

qed

lemma *trace-fri-header-candidate-non-low-contradicts-header-tie-gap:*

assumes *gap: trace-fri-reachable-header-tie-gap s out*

and *fri-openings:*

accepted-fri-opening-transcript s out trace-roots trace-bs
trace-final fri-dg composition-roots composition-bs
composition-final fri-query-idxs trace-round-layers
composition-round-layers

and *header:*

verifier-header-transcript s fr trace-roots trace-final as fri-dg
composition-roots composition-final rest

and *partial:*

accepted-with-partial-trace-openings s out fr fri-query-idxs
trace-openings

and *candidate:*

partial-trace-table-candidate trace-table trace-openings

and *not-low: ¬ trace-table-low-degree trace-table*

shows *False*
proof –
 have *trace-fri-bad-with-header-tied-partial-candidate s out*
 unfolding *trace-fri-bad-with-header-tied-partial-candidate-def*
 by (*intro exI conjI*)
 (*rule fri-openings, rule header, rule partial, rule candidate,*
 rule not-low)
 then show *False*
 using *gap unfolding trace-fri-reachable-header-tie-gap-def* **by** *simp*
qed

lemma *trace-fri-header-candidate-low-imp-transfer-gap:*
assumes *gap: trace-fri-reachable-header-tie-gap s out*
and *fri-openings:*
 accepted-fri-opening-transcript s out trace-roots trace-bs
 trace-final fri-dg composition-roots composition-bs
 composition-final fri-query-idxs trace-round-layers
 composition-round-layers
and *header:*
 verifier-header-transcript s fr trace-roots trace-final as fri-dg
 composition-roots composition-final rest
and *partial:*
 accepted-with-partial-trace-openings s out fr fri-query-idxs
 trace-openings
and *candidate:*
 partial-trace-table-candidate trace-table trace-openings
and *low: trace-table-low-degree trace-table*
shows *trace-fri-header-tied-candidate-transfer-gap s out*
proof –
 have *raw: trace-fri-bad-with-reachable-partial-candidate s out*
 using *gap unfolding trace-fri-reachable-header-tie-gap-def* **by** *simp*
show *?thesis*
 unfolding *trace-fri-header-tied-candidate-transfer-gap-def*
 by (*intro exI conjI*)
 (*rule raw, rule fri-openings, rule header, rule partial,*
 rule candidate, rule low)
qed

lemma *trace-fri-header-tied-candidate-available-gap-imp-transfer-gap:*
assumes *trace-fri-header-tied-candidate-available-gap s out*
shows *trace-fri-header-tied-candidate-transfer-gap s out*
proof –
obtain *trace-roots trace-bs trace-final fri-dg composition-roots*
 composition-bs composition-final fri-query-idxs trace-round-layers
 composition-round-layers fr as rest trace-openings trace-table
where *gap: trace-fri-reachable-header-tie-gap s out*
and *fri-openings:*
 accepted-fri-opening-transcript s out trace-roots trace-bs
 trace-final fri-dg composition-roots composition-bs

```

    composition-final fri-query-idxs trace-round-layers
    composition-round-layers
and header:
    verifier-header-transcript s fr trace-roots trace-final as fri-dg
    composition-roots composition-final rest
and partial:
    accepted-with-partial-trace-openings s out fr fri-query-idxs
    trace-openings
and candidate:
    partial-trace-table-candidate trace-table trace-openings
using assms
unfolding trace-fri-header-tied-candidate-available-gap-def by blast
show ?thesis
proof (cases trace-table-low-degree trace-table)
  case True
  show ?thesis
    by (rule trace-fri-header-candidate-low-imp-transfer-gap
      [OF gap fri-openings header partial candidate True])
  next
  case False
  have False
    by (rule trace-fri-header-candidate-non-low-contradicts-header-tie-gap
      [OF gap fri-openings header partial candidate False])
  then show ?thesis by simp
qed
qed

lemma wp-trace-fri-header-tied-candidate-available-gap-bound-from-transfer-gap:
assumes transfer-bound:
  wp-event verify-monad
  (trace-fri-header-tied-candidate-transfer-gap s)  $s \leq T$ 
shows
  wp-event verify-monad
  (trace-fri-header-tied-candidate-available-gap s)  $s \leq T$ 
by (rule order-trans[OF - transfer-bound])
  (rule wp-event-mono,
    rule trace-fri-header-tied-candidate-available-gap-imp-transfer-gap)

lemma trace-fri-reachable-imp-header-tied-or-gap:
assumes trace-fri-bad-with-reachable-partial-candidate s out
shows
  trace-fri-bad-with-header-tied-partial-candidate s out  $\vee$ 
  trace-fri-reachable-header-tie-gap s out
using assms
unfolding trace-fri-reachable-header-tie-gap-def
by blast

lemma wp-trace-fri-reachable-bound-from-header-tied-and-gap:
fixes H G :: prob

```

assumes *header-bound*:
wp-event verify-monad
 $(\text{trace-fri-bad-with-header-tied-partial-candidate } s) \ s \leq H$
and *gap-bound*:
wp-event verify-monad
 $(\text{trace-fri-reachable-header-tie-gap } s) \ s \leq G$
shows
wp-event verify-monad
 $(\text{trace-fri-bad-with-reachable-partial-candidate } s) \ s \leq H + G$
proof –
have *wp-event verify-monad*
 $(\text{trace-fri-bad-with-reachable-partial-candidate } s) \ s \leq$
wp-event verify-monad
 $(\lambda out.$
 $\text{trace-fri-bad-with-header-tied-partial-candidate } s \ out \ \vee$
 $\text{trace-fri-reachable-header-tie-gap } s \ out) \ s$
by (rule *wp-event-mono*)
(rule *trace-fri-reachable-imp-header-tied-or-gap*)
also have $\dots \leq$
wp-event verify-monad
 $(\text{trace-fri-bad-with-header-tied-partial-candidate } s) \ s +$
wp-event verify-monad
 $(\text{trace-fri-reachable-header-tie-gap } s) \ s$
by (rule *wp-event-union-bound*)
also have $\dots \leq H + G$
by (rule *add-mono*[*OF header-bound gap-bound*])
finally show *?thesis* .
qed

lemma *trace-fri-reachable-partial-candidate-reduction-from-header-tied-and-gap*:
fixes $H \ G :: \text{prob}$
assumes *header-bound*:
wp-event verify-monad
 $(\text{trace-fri-bad-with-header-tied-partial-candidate } s) \ s \leq H$
and *gap-bound*:
wp-event verify-monad
 $(\text{trace-fri-reachable-header-tie-gap } s) \ s \leq G$
and *total-bound*: $H + G \leq \text{trace-fri-error}$
shows *trace-fri-reachable-partial-candidate-reduction-assumption s*
proof –
have *old-bound*:
wp-event verify-monad
 $(\text{trace-fri-bad-with-reachable-partial-candidate } s) \ s \leq H + G$
by (rule *wp-trace-fri-reachable-bound-from-header-tied-and-gap*
[*OF header-bound gap-bound*])
show *?thesis*
unfolding *trace-fri-reachable-partial-candidate-reduction-assumption-def*
by (rule *order-trans*[*OF old-bound total-bound*])
qed

definition *trace-fri-header-tied-recorded-base-chunk-auth-missing*
 $:: ('f, 'a) \text{ protocol-channel-scheme} \Rightarrow$
 $(\text{unit list} \times ('f, 'a) \text{ protocol-channel-scheme}) \text{ option} \Rightarrow \text{bool}$
where
 $\text{trace-fri-header-tied-recorded-base-chunk-auth-missing } s \text{ out} \longleftrightarrow$
 $(\exists \text{ trace-roots trace-bs trace-final dg composition-roots}$
 $\text{composition-bs composition-final fri-query-idxs trace-round-layers}$
 $\text{composition-round-layers fr as rest trace-openings trace-table}$
 $\text{round-idx xp xp-path xn xn-path result final-state.}$
 $\text{out} = \text{Some (result, final-state)} \wedge$
 $\text{accepted-fri-opening-transcript } s \text{ out trace-roots trace-bs}$
 $\text{trace-final dg composition-roots composition-bs composition-final}$
 $\text{fri-query-idxs trace-round-layers composition-round-layers} \wedge$
 $\text{verifier-header-transcript } s \text{ fr trace-roots trace-final as dg}$
 $\text{composition-roots composition-final rest} \wedge$
 $\text{accepted-with-partial-trace-openings } s \text{ out fr fri-query-idxs}$
 $\text{trace-openings} \wedge$
 $\text{partial-trace-table-candidate trace-table trace-openings} \wedge$
 $\neg \text{trace-table-low-degree trace-table} \wedge$
 $\text{round-idx} < \text{length fri-query-idxs} \wedge$
 $0 < \text{length trace-bs} \wedge$
 $\text{fri-layer-step-evidence}$
 $(\text{trace-roots} ! 0)$
 $(\text{trace-bs} ! 0)$
 $(\text{fri-evidence-layer-len trace-roots } 0)$
 $(\text{fri-evidence-layer-idx trace-roots fri-query-idxs round-idx } 0)$
 1
 $(\text{fri-sibling-index (fri-evidence-layer-len trace-roots } 0)$
 $(\text{fri-evidence-layer-idx trace-roots fri-query-idxs round-idx } 0)))$
 $\text{xp xp-path xn xn-path}$
 $(\text{fri-evidence-next-idx trace-roots fri-query-idxs round-idx } 0)$
 $(\text{fri-evidence-next-value trace-roots trace-bs fri-query-idxs}$
 $\text{round-idx } 0 \text{ xp xn})$
 $(\text{trace-round-layers} ! \text{round-idx} ! 0) \wedge$
 $\neg \text{fri-opening-matches-table}$
 $(\text{fri-evidence-layer-len trace-roots } 0)$
 $(\text{fri-evidence-layer-idx trace-roots fri-query-idxs round-idx } 0)$
 $\text{trace-table xp xn} \wedge$
 $\neg \text{fri-layer-chunk-authenticated (trace-roots} ! 0)$
 $(\text{fri-evidence-layer-len trace-roots } 0)$
 $(\text{fri-evidence-layer-idx trace-roots fri-query-idxs round-idx } 0)$
 $(\text{trace-round-layers} ! \text{round-idx} ! 0) \text{ final-state})$

lemma *trace-fri-header-tied-recorded-base-chunk-auth-missing-false:*

assumes

trace-fri-header-tied-recorded-base-chunk-auth-missing s out

shows *False*

proof —

```

from assms obtain trace-roots trace-bs trace-final dg composition-roots
composition-bs composition-final fri-query-idxs trace-round-layers
composition-round-layers fr as rest trace-openings trace-table
round-idx xp xp-path xn xn-path result final-state
where out-eq: out = Some (result, final-state)
and fri-openings:
  accepted-fri-opening-transcript s out trace-roots trace-bs
  trace-final dg composition-roots composition-bs composition-final
  fri-query-idxs trace-round-layers composition-round-layers
and round-bound: round-idx < length fri-query-idxs
and trace-bs-nonempty: 0 < length trace-bs
and missing:
  ¬ fri-layer-chunk-authenticated (trace-roots ! 0)
  (fri-evidence-layer-len trace-roots 0)
  (fri-evidence-layer-idx trace-roots fri-query-idxs round-idx 0)
  (trace-round-layers ! round-idx ! 0) final-state
unfolding
  trace-fri-header-tied-recorded-base-chunk-auth-missing-def
by blast
have len-trace-bs:
  length trace-bs = length trace-roots
  using accepted-fri-opening-transcript-shapes(2)[OF fri-openings] .
have trace-nonempty: 0 < length trace-roots
  using trace-bs-nonempty len-trace-bs by simp
have len-query: length fri-query-idxs = rounds
  using accepted-fri-opening-transcript-shapes(7)[OF fri-openings] .
have round-bound-rounds: round-idx < rounds
  using round-bound len-query by simp
have auth:
  fri-layer-chunk-authenticated (trace-roots ! 0) (clength * scale)
  (fri-query-idxs ! round-idx)
  (trace-round-layers ! round-idx ! 0) final-state
by (rule
  accepted-fri-opening-transcript-trace-base-recorded-chunk-authenticated-at
  [OF fri-openings out-eq round-bound-rounds trace-nonempty])
have len0:
  fri-evidence-layer-len trace-roots 0 = clength * scale
  using trace-nonempty
  unfolding fri-evidence-layer-len-def
  by (cases trace-roots) simp-all
have idx0:
  fri-evidence-layer-idx trace-roots fri-query-idxs round-idx 0 =
  fri-query-idxs ! round-idx
  using trace-nonempty
  unfolding fri-evidence-layer-idx-def
  by (cases trace-roots) simp-all
show False
  using missing auth unfolding len0 idx0 by contradiction
qed

```


lemma *wp-trace-fri-header-tied-recorded-base-chunk-auth-missing-zero:*
shows

wp-event verify-monad
 $(\text{trace-fri-header-tied-recorded-base-chunk-auth-missing } s) \ s \leq 0$

proof –

have *wp-event verify-monad*
 $(\text{trace-fri-header-tied-recorded-base-chunk-auth-missing } s) \ s$
 $\leq \text{wp-event verify-monad } (\lambda-. \text{ False}) \ s$
by (*rule wp-event-mono*)
(rule trace-fri-header-tied-recorded-base-chunk-auth-missing-false)

also have $\dots = 0$

unfolding *wp-event-def wp-def dist-expect-def* **by** *simp*

finally show *?thesis* .

qed

definition *trace-fri-header-tied-next-value-replay-checked-conflict*

$:: (f, 'a) \text{ protocol-channel-scheme} \Rightarrow$
 $(\text{unit list} \times (f, 'a) \text{ protocol-channel-scheme}) \text{ option} \Rightarrow \text{bool}$

where

trace-fri-header-tied-next-value-replay-checked-conflict s out
 $\longleftrightarrow$
 $(\exists \text{ trace-roots trace-bs trace-final fri-dg composition-roots}$
 $\text{ composition-bs composition-final fri-query-idxs trace-round-layers}$
 $\text{ composition-round-layers fr trace-openings trace-table}$
 $\text{ round-idx round-idx' layer-idx } v \ v'.$
 $\text{accepted-fri-opening-transcript } s \ \text{out trace-roots trace-bs}$
 $\text{ trace-final fri-dg composition-roots composition-bs}$
 $\text{ composition-final fri-query-idxs trace-round-layers}$
 $\text{ composition-round-layers} \wedge$
 $\text{fr} = \text{hd trace-roots} \wedge$
 $\text{accepted-with-partial-trace-openings } s \ \text{out fr fri-query-idxs}$
 $\text{ trace-openings} \wedge$
 $\text{partial-trace-table-candidate trace-table trace-openings} \wedge$
 $\neg \text{trace-table-low-degree trace-table} \wedge$
 $\text{round-idx} < \text{length fri-query-idxs} \wedge$
 $\text{round-idx'} < \text{length fri-query-idxs} \wedge$
 $\text{layer-idx} < \text{length trace-bs} \wedge$
 $\text{fri-evidence-next-idx trace-roots fri-query-idxs round-idx layer-idx} =$
 $\text{fri-evidence-next-idx trace-roots fri-query-idxs round-idx'}$
 $\text{ layer-idx} \wedge$
 $\text{generic-fri-round-forced-next-value trace-roots trace-bs}$
 $\text{ fri-query-idxs trace-round-layers round-idx layer-idx } v \wedge$
 $\text{generic-fri-round-forced-next-value trace-roots trace-bs}$
 $\text{ fri-query-idxs trace-round-layers round-idx' layer-idx } v' \wedge$
 $v \neq v' \wedge$
 $v = v')$

definition *trace-fri-header-tied-next-value-replay-gap*

```

:: ('f, 'a) protocol-channel-scheme  $\Rightarrow$ 
  (unit list  $\times$  ('f, 'a) protocol-channel-scheme) option  $\Rightarrow$  bool
where
  trace-fri-header-tied-next-value-replay-gap s out  $\longleftrightarrow$ 
    trace-fri-header-tied-sampled-next-value-conflict s out  $\wedge$ 
     $\neg$  trace-fri-header-tied-next-value-replay-checked-conflict s out

lemma trace-fri-header-tied-next-value-replay-checked-conflict-false:
   $\neg$  trace-fri-header-tied-next-value-replay-checked-conflict s out
unfolding trace-fri-header-tied-next-value-replay-checked-conflict-def
by blast

lemma trace-fri-header-tied-sampled-next-value-imp-replay-gap:
assumes trace-fri-header-tied-sampled-next-value-conflict s out
shows trace-fri-header-tied-next-value-replay-gap s out
using assms trace-fri-header-tied-next-value-replay-checked-conflict-false
unfolding trace-fri-header-tied-next-value-replay-gap-def
by simp

lemma wp-trace-fri-header-tied-sampled-next-value-bound-from-replay-gap:
fixes G :: prob
assumes gap-bound:
  wp-event verify-monad
  (trace-fri-header-tied-next-value-replay-gap s) s  $\leq$  G
shows
  wp-event verify-monad
  (trace-fri-header-tied-sampled-next-value-conflict s) s  $\leq$  G
proof -
have wp-event verify-monad
  (trace-fri-header-tied-sampled-next-value-conflict s) s  $\leq$ 
  wp-event verify-monad
  (trace-fri-header-tied-next-value-replay-gap s) s
by (rule wp-event-mono)
  (rule trace-fri-header-tied-sampled-next-value-imp-replay-gap)
then show ?thesis
by (rule order-trans[OF - gap-bound])
qed

definition trace-fri-header-tied-successor-replay-checked-conflict
:: ('f, 'a) protocol-channel-scheme  $\Rightarrow$ 
  (unit list  $\times$  ('f, 'a) protocol-channel-scheme) option  $\Rightarrow$  bool
where
  trace-fri-header-tied-successor-replay-checked-conflict s out
   $\longleftrightarrow$ 
  ( $\exists$  trace-roots trace-bs trace-final fri-dg composition-roots
    composition-bs composition-final fri-query-idxs trace-round-layers
    composition-round-layers fr trace-openings trace-table
    source-round target-round layer-idx v xp xp-path xn xn-path.
    accepted-fri-opening-transcript s out trace-roots trace-bs

```

$\text{trace-final fri-dg composition-roots composition-bs}$
 $\text{composition-final fri-query-idxs trace-round-layers}$
 $\text{composition-round-layers} \wedge$
 $\text{fr} = \text{hd trace-roots} \wedge$
 $\text{accepted-with-partial-trace-openings } s \text{ out fr fri-query-idxs}$
 $\text{trace-openings} \wedge$
 $\text{partial-trace-table-candidate trace-table trace-openings} \wedge$
 $\neg \text{trace-table-low-degree trace-table} \wedge$
 $\text{source-round} < \text{length fri-query-idxs} \wedge$
 $\text{target-round} < \text{length fri-query-idxs} \wedge$
 $\text{Suc layer-idx} < \text{length trace-bs} \wedge$
 $\text{generic-fri-round-forced-next-value trace-roots trace-bs}$
 $\text{fri-query-idxs trace-round-layers source-round layer-idx } v \wedge$
 $\text{fri-layer-step-evidence}$
 $(\text{trace-roots} ! \text{Suc layer-idx})$
 $(\text{trace-bs} ! \text{Suc layer-idx})$
 $(\text{fri-evidence-layer-len trace-roots} (\text{Suc layer-idx}))$
 $(\text{fri-evidence-layer-idx trace-roots fri-query-idxs target-round}$
 $(\text{Suc layer-idx}))$
 $(2 \wedge \text{Suc layer-idx})$
 $(\text{fri-sibling-index}$
 $(\text{fri-evidence-layer-len trace-roots} (\text{Suc layer-idx}))$
 $(\text{fri-evidence-layer-idx trace-roots fri-query-idxs}$
 $\text{target-round} (\text{Suc layer-idx})))$
 $\text{xp xp-path xn xn-path}$
 $(\text{fri-evidence-next-idx trace-roots fri-query-idxs target-round}$
 $(\text{Suc layer-idx}))$
 $(\text{fri-evidence-next-value trace-roots trace-bs fri-query-idxs}$
 $\text{target-round} (\text{Suc layer-idx}) \text{ xp xn})$
 $(\text{trace-round-layers} ! \text{target-round} ! \text{Suc layer-idx}) \wedge$
 $((\text{fri-evidence-next-idx trace-roots fri-query-idxs source-round}$
 $\text{layer-idx} =$
 $\text{fri-evidence-layer-idx trace-roots fri-query-idxs target-round}$
 $(\text{Suc layer-idx}) \wedge$
 $v \neq \text{xp} \wedge v = \text{xp}) \vee$
 $(\text{fri-evidence-next-idx trace-roots fri-query-idxs source-round}$
 $\text{layer-idx} =$
 fri-sibling-index
 $(\text{fri-evidence-layer-len trace-roots} (\text{Suc layer-idx}))$
 $(\text{fri-evidence-layer-idx trace-roots fri-query-idxs target-round}$
 $(\text{Suc layer-idx})) \wedge$
 $v \neq \text{xn} \wedge v = \text{xn})))$

definition $\text{trace-fri-header-tied-successor-replay-gap}$

$:: ('f, 'a) \text{protocol-channel-scheme} \Rightarrow$

$(\text{unit list} \times ('f, 'a) \text{protocol-channel-scheme}) \text{option} \Rightarrow \text{bool}$

where

$\text{trace-fri-header-tied-successor-replay-gap } s \text{ out} \longleftrightarrow$

$\text{trace-fri-header-tied-sampled-successor-opening-conflict } s \text{ out} \wedge$

$\neg \text{trace-fri-header-tied-successor-replay-checked-conflict } s \text{ out}$

lemma *trace-fri-header-tied-successor-replay-checked-conflict-false:*

$\neg \text{trace-fri-header-tied-successor-replay-checked-conflict } s \text{ out}$

unfolding *trace-fri-header-tied-successor-replay-checked-conflict-def*
by *blast*

lemma *trace-fri-header-tied-sampled-successor-imp-replay-gap:*

assumes *trace-fri-header-tied-sampled-successor-opening-conflict s out*

shows *trace-fri-header-tied-successor-replay-gap s out*

using *assms trace-fri-header-tied-successor-replay-checked-conflict-false*

unfolding *trace-fri-header-tied-successor-replay-gap-def*

by *simp*

lemma *wp-trace-fri-header-tied-sampled-successor-bound-from-replay-gap:*

fixes *G :: prob*

assumes *gap-bound:*

wp-event verify-monad

$(\text{trace-fri-header-tied-successor-replay-gap } s) \ s \leq G$

shows

wp-event verify-monad

$(\text{trace-fri-header-tied-sampled-successor-opening-conflict } s) \ s \leq G$

proof –

have *wp-event verify-monad*

$(\text{trace-fri-header-tied-sampled-successor-opening-conflict } s) \ s \leq$

wp-event verify-monad

$(\text{trace-fri-header-tied-successor-replay-gap } s) \ s$

by *(rule wp-event-mono)*

(rule trace-fri-header-tied-sampled-successor-imp-replay-gap)

then show *?thesis*

by *(rule order-trans[OF - gap-bound])*

qed

definition *trace-fri-header-tied-final-value-replay-checked-conflict*

$:: (f, 'a) \text{ protocol-channel-scheme} \Rightarrow$

$(\text{unit list} \times (f, 'a) \text{ protocol-channel-scheme}) \text{ option} \Rightarrow \text{bool}$

where

trace-fri-header-tied-final-value-replay-checked-conflict s out

$\longleftrightarrow$

$(\exists \text{ trace-roots trace-bs trace-final fri-dg composition-roots}$

composition-bs composition-final fri-query-idxs trace-round-layers

composition-round-layers fr trace-openings trace-table round-idx v.

accepted-fri-opening-transcript s out trace-roots trace-bs

trace-final fri-dg composition-roots composition-bs

composition-final fri-query-idxs trace-round-layers

composition-round-layers $\wedge$

fr = hd trace-roots $\wedge$

accepted-with-partial-trace-openings s out fr fri-query-idxs

trace-openings $\wedge$

partial-trace-table-candidate *trace-table* *trace-openings* $\wedge$
 $\neg$ *trace-table-low-degree* *trace-table* $\wedge$
 $0 < \text{length } \text{trace-bs} \wedge$
 $\text{round-idx} < \text{length } \text{fri-query-idxs} \wedge$
generic-fri-round-forced-next-value *trace-roots* *trace-bs*
fri-query-idxs *trace-round-layers* *round-idx* ($\text{length } \text{trace-bs} - 1$)
 $v \wedge$
 $v \neq \text{trace-final} \wedge$
 $v = \text{trace-final}$)

definition *trace-fri-header-tied-final-value-replay-gap*

$:: ('f, 'a) \text{ protocol-channel-scheme} \Rightarrow$
 $(\text{unit list} \times ('f, 'a) \text{ protocol-channel-scheme}) \text{ option} \Rightarrow \text{bool}$

where

trace-fri-header-tied-final-value-replay-gap *s out* $\longleftrightarrow$
trace-fri-header-tied-sampled-final-value-conflict *s out* $\wedge$
 $\neg$ *trace-fri-header-tied-final-value-replay-checked-conflict* *s out*

lemma *trace-fri-header-tied-sampled-final-value-conflictE*:

assumes *trace-fri-header-tied-sampled-final-value-conflict* *s out*

obtains *trace-roots* *trace-bs* *trace-final* *fri-dg* *composition-roots*
composition-bs *composition-final* *fri-query-idxs* *trace-round-layers*
composition-round-layers *fr* *as* *rest* *trace-openings* *trace-table*
round-idx *v*

where

accepted-fri-opening-transcript *s out* *trace-roots* *trace-bs*
trace-final *fri-dg* *composition-roots* *composition-bs* *composition-final*
fri-query-idxs *trace-round-layers* *composition-round-layers*
verifier-header-transcript *s fr* *trace-roots* *trace-final* *as* *fri-dg*
composition-roots *composition-final* *rest*
accepted-with-partial-trace-openings *s out* *fr* *fri-query-idxs*
trace-openings
partial-trace-table-candidate *trace-table* *trace-openings*
 $\neg$ *trace-table-low-degree* *trace-table*
 $0 < \text{length } \text{trace-bs}$
 $\text{round-idx} < \text{length } \text{fri-query-idxs}$
generic-fri-round-forced-next-value *trace-roots* *trace-bs*
fri-query-idxs *trace-round-layers* *round-idx* ($\text{length } \text{trace-bs} - 1$) *v*
 $v \neq \text{trace-final}$

proof –

from *assms* **obtain** *trace-roots* *trace-bs* *trace-final* *fri-dg*
composition-roots *composition-bs* *composition-final* *fri-query-idxs*
trace-round-layers *composition-round-layers* *fr* *as* *rest* *trace-openings*
trace-table **where**

fri-openings:

accepted-fri-opening-transcript *s out* *trace-roots* *trace-bs*
trace-final *fri-dg* *composition-roots* *composition-bs* *composition-final*
fri-query-idxs *trace-round-layers* *composition-round-layers*

and *header*:

verifier-header-transcript s fr trace-roots trace-final as fri-dg
composition-roots composition-final rest
and *partial:*
accepted-with-partial-trace-openings s out fr fri-query-idxs
trace-openings
and *candidate:*
partial-trace-table-candidate trace-table trace-openings
and *nonlow:* $\neg$ *trace-table-low-degree trace-table*
and *final-conflict:*
generic-fri-sampled-final-value-conflict trace-roots trace-bs
trace-final fri-query-idxs trace-round-layers
unfolding *trace-fri-header-tied-sampled-final-value-conflict-def*
by *blast*
from *generic-fri-sampled-final-value-conflictE[OF final-conflict]*
obtain *round-idx v where*
nonempty: 0 < length trace-bs
and *round-bound: round-idx < length fri-query-idxs*
and *forced:*
generic-fri-round-forced-next-value trace-roots trace-bs
fri-query-idxs trace-round-layers round-idx (length trace-bs - 1) v
and *neq: v $\neq$ trace-final*
by *blast*
show *?thesis*
by (*rule that[OF fri-openings header partial candidate nonlow nonempty*
round-bound forced neq])
qed

lemma *trace-fri-header-tied-final-value-replay-checked-conflict-false:*
 $\neg$ *trace-fri-header-tied-final-value-replay-checked-conflict s out*
unfolding *trace-fri-header-tied-final-value-replay-checked-conflict-def*
by *blast*

lemma *trace-fri-header-tied-sampled-final-value-imp-replay-gap:*
assumes *trace-fri-header-tied-sampled-final-value-conflict s out*
shows *trace-fri-header-tied-final-value-replay-gap s out*
using *assms trace-fri-header-tied-final-value-replay-checked-conflict-false*
unfolding *trace-fri-header-tied-final-value-replay-gap-def*
by *simp*

lemma *wp-trace-fri-header-tied-sampled-final-value-bound-from-replay-gap:*
fixes *G :: prob*
assumes *gap-bound:*
wp-event verify-monad
(trace-fri-header-tied-final-value-replay-gap s) s $\leq$ G
shows
wp-event verify-monad
(trace-fri-header-tied-sampled-final-value-conflict s) s $\leq$ G
proof –
have *wp-event verify-monad*

```

    (trace-fri-header-tied-sampled-final-value-conflict s) s ≤
    wp-event verify-monad
    (trace-fri-header-tied-final-value-replay-gap s) s
  by (rule wp-event-mono)
    (rule trace-fri-header-tied-sampled-final-value-imp-replay-gap)
  then show ?thesis
    by (rule order-trans[OF - gap-bound])
qed

```

lemma *wp-trace-fri-header-tied-without-same-bound-from-replay-gaps:*

fixes $B\ N\ S\ F :: \text{prob}$

assumes *base-bound:*

wp-event verify-monad

(trace-fri-header-tied-sampled-base-opening-conflict s) s ≤ B

and *next-gap-bound:*

wp-event verify-monad

(trace-fri-header-tied-next-value-replay-gap s) s ≤ N

and *successor-gap-bound:*

wp-event verify-monad

(trace-fri-header-tied-successor-replay-gap s) s ≤ S

and *final-gap-bound:*

wp-event verify-monad

(trace-fri-header-tied-final-value-replay-gap s) s ≤ F

shows

wp-event verify-monad

(trace-fri-header-tied-sampled-assignment-conflict-without-same-layer s)

s ≤ B + N + S + F

by (rule *wp-trace-fri-header-tied-without-same-bound-from-branches*)

(rule *base-bound*,

rule *wp-trace-fri-header-tied-sampled-next-value-bound-from-replay-gap*
[*OF next-gap-bound*],

rule *wp-trace-fri-header-tied-sampled-successor-bound-from-replay-gap*
[*OF successor-gap-bound*],

rule *wp-trace-fri-header-tied-sampled-final-value-bound-from-replay-gap*
[*OF final-gap-bound*])

lemma *wp-trace-fri-header-tied-without-same-bound-from-value-alignment-and-replay-gaps:*

fixes $M\ G\ N\ S\ F :: \text{prob}$

assumes *mismatch-bound:*

wp-event verify-monad

(trace-fri-header-tied-value-aligned-sibling-mismatch s) s ≤ M

and *alignment-gap-bound:*

wp-event verify-monad

(trace-fri-header-tied-base-value-alignment-gap s) s ≤ G

and *next-gap-bound:*

wp-event verify-monad

(trace-fri-header-tied-next-value-replay-gap s) s ≤ N

and *successor-gap-bound:*

wp-event verify-monad

(trace-fri-header-tied-successor-replay-gap s) $s \leq S$
and final-gap-bound:
 wp-event verify-monad
 (trace-fri-header-tied-final-value-replay-gap s) $s \leq F$
shows
 wp-event verify-monad
 (trace-fri-header-tied-sampled-assignment-conflict-without-same-layer s)
 $s \leq (M + G) + N + S + F$
by (rule wp-trace-fri-header-tied-without-same-bound-from-replay-gaps)
 (rule
 wp-trace-fri-header-tied-sampled-base-bound-from-sibling-mismatch-and-gap
 [OF mismatch-bound alignment-gap-bound],
 rule next-gap-bound,
 rule successor-gap-bound,
 rule final-gap-bound)

lemma wp-trace-fri-header-tied-without-same-bound-from-partial-merkle-structural-and-replay-gaps:

fixes $P H G N S F :: \text{prob}$

assumes partial-merkle-bound:

wp-event verify-monad (partial-merkle-inconsistency-bad s) $s \leq P$

and structural-gap-bound:

wp-event verify-monad

(trace-fri-header-tied-sibling-mismatch-structural-gap s) $s \leq H$

and alignment-gap-bound:

wp-event verify-monad

(trace-fri-header-tied-base-value-alignment-gap s) $s \leq G$

and next-gap-bound:

wp-event verify-monad

(trace-fri-header-tied-next-value-replay-gap s) $s \leq N$

and successor-gap-bound:

wp-event verify-monad

(trace-fri-header-tied-successor-replay-gap s) $s \leq S$

and final-gap-bound:

wp-event verify-monad

(trace-fri-header-tied-final-value-replay-gap s) $s \leq F$

shows

wp-event verify-monad

(trace-fri-header-tied-sampled-assignment-conflict-without-same-layer s)

$s \leq ((P + H) + G) + N + S + F$

by (rule

wp-trace-fri-header-tied-without-same-bound-from-value-alignment-and-replay-gaps)

(rule

wp-trace-fri-header-tied-value-aligned-sibling-mismatch-bound-from-partial-merkle-and-structural-gap

[OF partial-merkle-bound structural-gap-bound],

rule alignment-gap-bound,

rule next-gap-bound,

rule successor-gap-bound,

rule final-gap-bound)

lemma *wp-trace-fri-header-tied-authenticated-conflict-bound-from-partial-merkle-structural-and-replay-gaps:*
fixes $P\ H\ G\ N\ S\ F :: \text{prob}$

assumes *partial-merkle-bound:*

wp-event verify-monad (partial-merkle-inconsistency-bad s) s $\leq P$

and *structural-gap-bound:*

wp-event verify-monad

(trace-fri-header-tied-sibling-mismatch-structural-gap s) s $\leq H$

and *alignment-gap-bound:*

wp-event verify-monad

(trace-fri-header-tied-base-value-alignment-gap s) s $\leq G$

and *next-gap-bound:*

wp-event verify-monad

(trace-fri-header-tied-next-value-replay-gap s) s $\leq N$

and *successor-gap-bound:*

wp-event verify-monad

(trace-fri-header-tied-successor-replay-gap s) s $\leq S$

and *final-gap-bound:*

wp-event verify-monad

(trace-fri-header-tied-final-value-replay-gap s) s $\leq F$

shows

wp-event verify-monad

(trace-fri-header-tied-sampled-assignment-conflict-with-authenticated-same-layer s)

$s \leq (((P + H) + G) + N + S + F) + P$

proof –

have *without-bound:*

wp-event verify-monad

(trace-fri-header-tied-sampled-assignment-conflict-without-same-layer s)

$s \leq ((P + H) + G) + N + S + F$

by (rule

wp-trace-fri-header-tied-without-same-bound-from-partial-merkle-structural-and-replay-gaps)

(rule *partial-merkle-bound*,

rule *structural-gap-bound*,

rule *alignment-gap-bound*,

rule *next-gap-bound*,

rule *successor-gap-bound*,

rule *final-gap-bound*)

show *?thesis*

by (rule

wp-trace-fri-header-tied-authenticated-conflict-bound-from-without-same-and-merkle)

(rule *without-bound*, rule *partial-merkle-bound*)

qed

definition *trace-fri-header-tied-assignment-auth-gap*

$:: ('f, 'a) \text{ protocol-channel-scheme} \Rightarrow$

$(\text{unit list} \times ('f, 'a) \text{ protocol-channel-scheme}) \text{ option} \Rightarrow \text{bool}$

where

trace-fri-header-tied-assignment-auth-gap s out $\longleftrightarrow$

trace-fri-header-tied-sampled-assignment-conflict s out $\wedge$

$\neg \text{trace-fri-header-tied-sampled-assignment-conflict-with-authenticated-same-layer}$

s out

definition *trace-fri-header-tied-same-layer-auth-gap*

$:: ('f, 'a) \text{ protocol-channel-scheme} \Rightarrow$
 $(\text{unit list} \times ('f, 'a) \text{ protocol-channel-scheme}) \text{ option} \Rightarrow \text{bool}$

where

trace-fri-header-tied-same-layer-auth-gap s out $\longleftrightarrow$
 $(\exists \text{ trace-roots trace-bs trace-final dg composition-roots}$
 $\text{ composition-bs composition-final fri-query-idxs trace-round-layers}$
 $\text{ composition-round-layers fr as rest trace-openings trace-table}$
 $\text{ result final-state.}$
 $\text{out} = \text{Some} (\text{result}, \text{final-state}) \wedge$
 $\text{accepted-fri-opening-transcript } s \text{ out trace-roots trace-bs}$
 $\text{ trace-final dg composition-roots composition-bs composition-final}$
 $\text{ fri-query-idxs trace-round-layers composition-round-layers} \wedge$
 $\text{verifier-header-transcript } s \text{ fr trace-roots trace-final as dg}$
 $\text{ composition-roots composition-final rest} \wedge$
 $\text{accepted-with-partial-trace-openings } s \text{ out fr fri-query-idxs}$
 $\text{ trace-openings} \wedge$
 $\text{partial-trace-table-candidate trace-table trace-openings} \wedge$
 $\neg \text{ trace-table-low-degree trace-table} \wedge$
 $\text{generic-fri-sampled-same-layer-opening-conflict trace-roots trace-bs}$
 $\text{ fri-query-idxs trace-round-layers} \wedge$
 $\neg \text{ generic-fri-sampled-same-layer-opening-conflict-with-authenticated-chunks}$
 $\text{ trace-roots trace-bs fri-query-idxs trace-round-layers final-state})$

definition *trace-fri-header-tied-same-layer-recorded-auth-gap*

$:: ('f, 'a) \text{ protocol-channel-scheme} \Rightarrow$
 $(\text{unit list} \times ('f, 'a) \text{ protocol-channel-scheme}) \text{ option} \Rightarrow \text{bool}$

where

trace-fri-header-tied-same-layer-recorded-auth-gap s out $\longleftrightarrow$
 $(\exists \text{ trace-roots trace-bs trace-final dg composition-roots}$
 $\text{ composition-bs composition-final fri-query-idxs trace-round-layers}$
 $\text{ composition-round-layers fr as rest trace-openings trace-table}$
 $\text{ result final-state.}$
 $\text{out} = \text{Some} (\text{result}, \text{final-state}) \wedge$
 $\text{accepted-fri-opening-transcript } s \text{ out trace-roots trace-bs}$
 $\text{ trace-final dg composition-roots composition-bs composition-final}$
 $\text{ fri-query-idxs trace-round-layers composition-round-layers} \wedge$
 $\text{verifier-header-transcript } s \text{ fr trace-roots trace-final as dg}$
 $\text{ composition-roots composition-final rest} \wedge$
 $\text{accepted-with-partial-trace-openings } s \text{ out fr fri-query-idxs}$
 $\text{ trace-openings} \wedge$
 $\text{partial-trace-table-candidate trace-table trace-openings} \wedge$
 $\neg \text{ trace-table-low-degree trace-table} \wedge$
 $\text{generic-fri-sampled-same-layer-recorded-auth-gap trace-roots trace-bs}$
 $\text{ fri-query-idxs trace-round-layers final-state})$

definition *trace-fri-header-tied-same-layer-authenticated-recorded-auth-gap*

$:: ('f, 'a) \text{ protocol-channel-scheme} \Rightarrow$
 $(\text{unit list} \times ('f, 'a) \text{ protocol-channel-scheme}) \text{ option} \Rightarrow \text{bool}$
where
 $\text{trace-fri-header-tied-same-layer-authenticated-recorded-auth-gap } s \text{ out}$
 $\longleftrightarrow$
 $(\exists \text{ trace-roots trace-bs trace-final dg composition-roots}$
 $\text{ composition-bs composition-final fri-query-idxs trace-round-layers}$
 $\text{ composition-round-layers fr as rest trace-openings trace-table}$
 $\text{ result final-state.}$
 $\text{out} = \text{Some} (\text{result}, \text{final-state}) \wedge$
 $\text{accepted-fri-opening-transcript } s \text{ out trace-roots trace-bs}$
 $\text{ trace-final dg composition-roots composition-bs composition-final}$
 $\text{ fri-query-idxs trace-round-layers composition-round-layers} \wedge$
 $\text{verifier-header-transcript } s \text{ fr trace-roots trace-final as dg}$
 $\text{ composition-roots composition-final rest} \wedge$
 $\text{accepted-with-partial-trace-openings } s \text{ out fr fri-query-idxs}$
 $\text{ trace-openings} \wedge$
 $\text{partial-trace-table-candidate trace-table trace-openings} \wedge$
 $\neg \text{ trace-table-low-degree trace-table} \wedge$
 $\text{generic-fri-sampled-same-layer-authenticated-recorded-auth-gap}$
 $\text{ trace-roots trace-bs fri-query-idxs trace-round-layers final-state})$

definition *trace-fri-header-tied-authenticated-slot-recorded-chunk-gap*

$:: ('f, 'a) \text{ protocol-channel-scheme} \Rightarrow$
 $(\text{unit list} \times ('f, 'a) \text{ protocol-channel-scheme}) \text{ option} \Rightarrow \text{bool}$

where

$\text{trace-fri-header-tied-authenticated-slot-recorded-chunk-gap } s \text{ out}$
 $\longleftrightarrow$
 $(\exists \text{ trace-roots trace-bs trace-final dg composition-roots}$
 $\text{ composition-bs composition-final fri-query-idxs trace-round-layers}$
 $\text{ composition-round-layers fr as rest trace-openings trace-table}$
 $\text{ result final-state.}$
 $\text{out} = \text{Some} (\text{result}, \text{final-state}) \wedge$
 $\text{accepted-fri-opening-transcript } s \text{ out trace-roots trace-bs}$
 $\text{ trace-final dg composition-roots composition-bs composition-final}$
 $\text{ fri-query-idxs trace-round-layers composition-round-layers} \wedge$
 $\text{verifier-header-transcript } s \text{ fr trace-roots trace-final as dg}$
 $\text{ composition-roots composition-final rest} \wedge$
 $\text{accepted-with-partial-trace-openings } s \text{ out fr fri-query-idxs}$
 $\text{ trace-openings} \wedge$
 $\text{partial-trace-table-candidate trace-table trace-openings} \wedge$
 $\neg \text{ trace-table-low-degree trace-table} \wedge$
 $\text{generic-fri-authenticated-slot-recorded-chunk-gap trace-roots}$
 $\text{ fri-query-idxs trace-round-layers final-state})$

definition *trace-fri-header-tied-final-value-authenticated-replay-gap*

$:: ('f, 'a) \text{ protocol-channel-scheme} \Rightarrow$
 $(\text{unit list} \times ('f, 'a) \text{ protocol-channel-scheme}) \text{ option} \Rightarrow \text{bool}$

where

$\text{trace-fri-header-tied-final-value-authenticated-replay-gap } s \text{ out}$
 $\longleftrightarrow$
 $(\exists \text{ trace-roots trace-bs trace-final dg composition-roots}$
 $\quad \text{composition-bs composition-final fri-query-idxs trace-round-layers}$
 $\quad \text{composition-round-layers fr as rest trace-openings trace-table}$
 $\quad \text{result final-state round-idx } v.$
 $\text{out} = \text{Some (result, final-state)} \wedge$
 $\text{accepted-fri-opening-transcript } s \text{ out trace-roots trace-bs}$
 $\quad \text{trace-final dg composition-roots composition-bs composition-final}$
 $\quad \text{fri-query-idxs trace-round-layers composition-round-layers} \wedge$
 $\text{verifier-header-transcript } s \text{ fr trace-roots trace-final as dg}$
 $\quad \text{composition-roots composition-final rest} \wedge$
 $\text{accepted-with-partial-trace-openings } s \text{ out fr fri-query-idxs}$
 $\quad \text{trace-openings} \wedge$
 $\text{partial-trace-table-candidate trace-table trace-openings} \wedge$
 $\neg \text{trace-table-low-degree trace-table} \wedge$
 $0 < \text{length trace-bs} \wedge$
 $\text{round-idx} < \text{length fri-query-idxs} \wedge$
 $\text{generic-fri-round-forced-next-value trace-roots trace-bs}$
 $\quad \text{fri-query-idxs trace-round-layers round-idx (length trace-bs - 1)}$
 $\quad v \wedge$
 $v \neq \text{trace-final} \wedge$
 $\neg \text{partial-merkle-inconsistency-bad } s \text{ out} \wedge$
 $\text{generic-fri-recorded-layer-chunk-authenticated trace-roots}$
 $\quad \text{fri-query-idxs trace-round-layers final-state round-idx}$
 $\quad (\text{length trace-bs} - 1))$

lemma $\text{trace-fri-header-tied-authenticated-slot-recorded-chunk-gap-false:}$

assumes

$\text{trace-fri-header-tied-authenticated-slot-recorded-chunk-gap } s \text{ out}$

shows False

proof —

from assms **obtain** $\text{trace-roots trace-bs trace-final dg composition-roots}$
 $\text{composition-bs composition-final fri-query-idxs trace-round-layers}$
 $\text{composition-round-layers fr as rest trace-openings trace-table}$
 $\text{result final-state}$

where $\text{out-eq: out} = \text{Some (result, final-state)}$

and fri-openings:

$\text{accepted-fri-opening-transcript } s \text{ out trace-roots trace-bs}$
 $\text{trace-final dg composition-roots composition-bs composition-final}$
 $\text{fri-query-idxs trace-round-layers composition-round-layers}$

and gap:

$\text{generic-fri-authenticated-slot-recorded-chunk-gap trace-roots}$
 $\text{fri-query-idxs trace-round-layers final-state}$

unfolding $\text{trace-fri-header-tied-authenticated-slot-recorded-chunk-gap-def}$

by blast

from gap **obtain** $\text{round-idx layer-idx auth-chunk}$ **where**

$\text{round-bound: round-idx} < \text{length fri-query-idxs}$

and $\text{layer-bound: layer-idx} < \text{length trace-roots}$

```

and missing:
  ¬ generic-fri-recorded-layer-chunk-authenticated trace-roots
    fri-query-idxs trace-round-layers final-state round-idx layer-idx
unfolding generic-fri-authenticated-slot-recorded-chunk-gap-def
by blast
have round-bound-rounds: round-idx < rounds
  using round-bound accepted-fri-opening-transcript-shapes(7)[OF fri-openings]
by simp
have all-layer-bounds:
  ∧ raw k. k < length trace-roots ⇒
    0 < fri-layer-lengths (length trace-roots) (clength * scale) ! k ∧
    fri-layer-indices (length trace-roots) (index (to-nat raw))
      (clength * scale) ! k <
    fri-layer-lengths (length trace-roots) (clength * scale) ! k
by (rule accepted-fri-opening-transcript-trace-raw-layer-bound
  [OF fri-openings])
have recorded:
  generic-fri-recorded-layer-chunk-authenticated trace-roots
    fri-query-idxs trace-round-layers final-state round-idx layer-idx
by (rule
  accepted-fri-opening-transcript-trace-recorded-layer-chunk-authenticated-at
  [OF fri-openings out-eq round-bound-rounds layer-bound
    all-layer-bounds])
show False
  using missing recorded by simp
qed

lemma wp-trace-fri-header-tied-authenticated-slot-recorded-chunk-gap-zero:
shows
  wp-event verify-monad
    (trace-fri-header-tied-authenticated-slot-recorded-chunk-gap s) s
    ≤ (0::prob)
proof -
have
  wp-event verify-monad
    (trace-fri-header-tied-authenticated-slot-recorded-chunk-gap s) s
    ≤ wp-event verify-monad (λ-. False) s
by (rule wp-event-mono)
  (rule trace-fri-header-tied-authenticated-slot-recorded-chunk-gap-false)
also have ... = 0
  unfolding wp-event-def wp-def dist-expect-def by simp
finally show ?thesis by simp
qed

definition trace-fri-header-tied-final-value-selected-step-missing-gap
  :: ('f, 'a) protocol-channel-scheme ⇒
    (unit list × ('f, 'a) protocol-channel-scheme) option ⇒ bool
where
  trace-fri-header-tied-final-value-selected-step-missing-gap s out

```

$\longleftrightarrow$
trace-fri-header-tied-final-value-authenticated-replay-gap *s out* $\wedge$
 $(\exists$ *trace-roots trace-bs trace-final dg composition-roots*
composition-bs composition-final fri-query-idxs trace-round-layers
composition-round-layers result final-state round-idx v.
out = Some (result, final-state) $\wedge$
accepted-fri-opening-transcript s out trace-roots trace-bs
trace-final dg composition-roots composition-bs composition-final
fri-query-idxs trace-round-layers composition-round-layers $\wedge$
 $0 < \text{length } \text{trace-bs} \wedge$
 $\text{round-idx} < \text{length } \text{fri-query-idxs} \wedge$
generic-fri-round-forced-next-value trace-roots trace-bs
fri-query-idxs trace-round-layers round-idx (length trace-bs - 1)
v $\wedge$
v $\neq \text{trace-final} \wedge$
 $\neg$ *partial-merkle-inconsistency-bad s out* $\wedge$
generic-fri-recorded-layer-chunk-authenticated trace-roots
fri-query-idxs trace-round-layers final-state round-idx
 $(\text{length } \text{trace-bs} - 1) \wedge$
 $(\forall$ *actual-chunk axp axp-path axn axn-path.*
 $\neg$ (*fri-layer-chunk-authenticated*
 $(\text{trace-roots} ! (\text{length } \text{trace-bs} - 1))$
 $(\text{fri-evidence-layer-len } \text{trace-roots} (\text{length } \text{trace-bs} - 1))$
 $(\text{fri-evidence-layer-idx } \text{trace-roots } \text{fri-query-idxs } \text{round-idx}$
 $(\text{length } \text{trace-bs} - 1))$
actual-chunk final-state $\wedge$
fri-layer-step-evidence
 $(\text{trace-roots} ! (\text{length } \text{trace-bs} - 1))$
 $(\text{trace-bs} ! (\text{length } \text{trace-bs} - 1))$
 $(\text{fri-evidence-layer-len } \text{trace-roots} (\text{length } \text{trace-bs} - 1))$
 $(\text{fri-evidence-layer-idx } \text{trace-roots } \text{fri-query-idxs } \text{round-idx}$
 $(\text{length } \text{trace-bs} - 1))$
 $(2 \wedge (\text{length } \text{trace-bs} - 1))$
fri-sibling-index
 $(\text{fri-evidence-layer-len } \text{trace-roots} (\text{length } \text{trace-bs} - 1))$
 $(\text{fri-evidence-layer-idx } \text{trace-roots } \text{fri-query-idxs } \text{round-idx}$
 $(\text{length } \text{trace-bs} - 1)))$
axp axp-path axn axn-path
 $(\text{fri-evidence-next-idx } \text{trace-roots } \text{fri-query-idxs } \text{round-idx}$
 $(\text{length } \text{trace-bs} - 1))$
trace-final actual-chunk)))

lemma *trace-fri-header-tied-final-value-selected-step-missing-gap-false:*

assumes *gap:*

trace-fri-header-tied-final-value-selected-step-missing-gap s out

shows *False*

proof —

from *gap obtain* *trace-roots trace-bs trace-final dg composition-roots*

composition-bs composition-final fri-query-idxs trace-round-layers

```

    composition-round-layers result final-state round-idx v
where out-eq: out = Some (result, final-state)
and fri-openings:
    accepted-fri-opening-transcript s out trace-roots trace-bs
    trace-final dg composition-roots composition-bs composition-final
    fri-query-idxs trace-round-layers composition-round-layers
and nonempty-bs: 0 < length trace-bs
and round-bound: round-idx < length fri-query-idxs
and missing:
     $\wedge$  actual-chunk axp axp-path axn axn-path.
     $\neg$  (fri-layer-chunk-authenticated
      (trace-roots ! (length trace-bs - 1))
      (fri-evidence-layer-len trace-roots (length trace-bs - 1))
      (fri-evidence-layer-idx trace-roots fri-query-idxs round-idx
        (length trace-bs - 1))
      actual-chunk final-state  $\wedge$ 
      fri-layer-step-evidence
      (trace-roots ! (length trace-bs - 1))
      (trace-bs ! (length trace-bs - 1))
      (fri-evidence-layer-len trace-roots (length trace-bs - 1))
      (fri-evidence-layer-idx trace-roots fri-query-idxs round-idx
        (length trace-bs - 1))
      (2  $\wedge$  (length trace-bs - 1))
      (fri-sibling-index
        (fri-evidence-layer-len trace-roots (length trace-bs - 1))
        (fri-evidence-layer-idx trace-roots fri-query-idxs round-idx
          (length trace-bs - 1))))
    axp axp-path axn axn-path
    (fri-evidence-next-idx trace-roots fri-query-idxs round-idx
      (length trace-bs - 1))
    trace-final actual-chunk)
unfolding trace-fri-header-tied-final-value-selected-step-missing-gap-def
by blast
have len-bs-roots: length trace-bs = length trace-roots
using accepted-fri-opening-transcript-shapes(2)[OF fri-openings] .
have nonempty-roots: 0 < length trace-roots
using nonempty-bs len-bs-roots by simp
have round-bound-rounds: round-idx < rounds
using round-bound
    accepted-fri-opening-transcript-shapes(7)[OF fri-openings]
by simp
have raw-layer:
     $\wedge$  raw.
    0 < fri-layer-lengths (length trace-roots) (clength * scale) !
    (length trace-roots - 1)  $\wedge$ 
    fri-layer-indices (length trace-roots) (index (to-nat raw))
    (clength * scale) ! (length trace-roots - 1) <
    fri-layer-lengths (length trace-roots) (clength * scale) !
    (length trace-roots - 1)

```

```

by (rule accepted-fri-opening-transcript-trace-raw-layer-bound
      [OF fri-openings])
      (use nonempty-roots in simp)
from accepted-fri-opening-transcript-trace-final-step-evidence-at
      [OF fri-openings out-eq round-bound-rounds nonempty-roots raw-layer]
obtain actual-chunk axp axp-path axn axn-path where
  auth:
    fri-layer-chunk-authenticated
      (trace-roots ! (length trace-roots - 1))
      (fri-evidence-layer-len trace-roots (length trace-roots - 1))
      (fri-evidence-layer-idx trace-roots fri-query-idxs round-idx
        (length trace-roots - 1))
      actual-chunk final-state
and step:
  fri-layer-step-evidence
    (trace-roots ! (length trace-roots - 1))
    (trace-bs ! (length trace-roots - 1))
    (fri-evidence-layer-len trace-roots (length trace-roots - 1))
    (fri-evidence-layer-idx trace-roots fri-query-idxs round-idx
      (length trace-roots - 1))
    (2 ^ (length trace-roots - 1))
    (fri-sibling-index
      (fri-evidence-layer-len trace-roots (length trace-roots - 1))
      (fri-evidence-layer-idx trace-roots fri-query-idxs round-idx
        (length trace-roots - 1)))
    axp axp-path axn axn-path
    (fri-evidence-next-idx trace-roots fri-query-idxs round-idx
      (length trace-roots - 1))
    trace-final actual-chunk
by blast
show False
using missing[of actual-chunk axp axp-path axn axn-path]
      auth step len-bs-roots
by simp
qed

lemma wp-trace-fri-header-tied-final-value-selected-step-missing-gap-zero:
shows
  wp-event verify-monad
    (trace-fri-header-tied-final-value-selected-step-missing-gap s) s
    ≤ (0::prob)
proof -
have
  wp-event verify-monad
    (trace-fri-header-tied-final-value-selected-step-missing-gap s) s ≤
    wp-event verify-monad (λ-. False) s
by (rule wp-event-mono)
      (rule trace-fri-header-tied-final-value-selected-step-missing-gap-false)
also have ... = 0

```


unfolding *wp-event-def wp-def dist-expect-def* **by** *simp*
finally show *?thesis* **by** *simp*
qed

lemma *trace-fri-header-tied-final-value-authenticated-replay-gap-imp-selected-step-missing:*
assumes
trace-fri-header-tied-final-value-authenticated-replay-gap s out
shows
trace-fri-header-tied-final-value-selected-step-missing-gap s out
proof –
from *assms* **obtain** *trace-roots trace-bs trace-final dg composition-roots*
composition-bs composition-final fri-query-idxs trace-round-layers
composition-round-layers fr as rest trace-openings trace-table
result final-state round-idx v
where *out-eq: out = Some (result, final-state)*
and *fri-openings:*
accepted-fri-opening-transcript s out trace-roots trace-bs
trace-final dg composition-roots composition-bs composition-final
fri-query-idxs trace-round-layers composition-round-layers
and *nonempty: 0 < length trace-bs*
and *round-bound: round-idx < length fri-query-idxs*
and *forced:*
generic-fri-round-forced-next-value trace-roots trace-bs
fri-query-idxs trace-round-layers round-idx (length trace-bs - 1) v
and *neq: v ≠ trace-final*
and *no-merkle: ¬ partial-merkle-inconsistency-bad s out*
and *recorded-auth:*
generic-fri-recorded-layer-chunk-authenticated trace-roots
fri-query-idxs trace-round-layers final-state round-idx
(length trace-bs - 1)
unfolding *trace-fri-header-tied-final-value-authenticated-replay-gap-def*
by *blast*
have *no-step:*
 $\bigwedge \text{actual-chunk } \text{axp } \text{axp-path } \text{axn } \text{axn-path}.$
 $\neg (\text{fri-layer-chunk-authenticated}$
 $(\text{trace-roots } ! (\text{length trace-bs} - 1))$
 $(\text{fri-evidence-layer-len trace-roots } (\text{length trace-bs} - 1))$
 $(\text{fri-evidence-layer-idx trace-roots fri-query-idxs round-idx}$
 $(\text{length trace-bs} - 1))$
 $\text{actual-chunk final-state} \wedge$
 $\text{fri-layer-step-evidence}$
 $(\text{trace-roots } ! (\text{length trace-bs} - 1))$
 $(\text{trace-bs } ! (\text{length trace-bs} - 1))$
 $(\text{fri-evidence-layer-len trace-roots } (\text{length trace-bs} - 1))$
 $(\text{fri-evidence-layer-idx trace-roots fri-query-idxs round-idx}$
 $(\text{length trace-bs} - 1))$
 $(2 \wedge (\text{length trace-bs} - 1))$
 $(\text{fri-sibling-index}$
 $(\text{fri-evidence-layer-len trace-roots } (\text{length trace-bs} - 1)))$

```

      (fri-evidence-layer-idx trace-roots fri-query-idxs round-idx
       (length trace-bs - 1)))
    arp arp-path arn arn-path
    (fri-evidence-next-idx trace-roots fri-query-idxs round-idx
     (length trace-bs - 1))
    trace-final actual-chunk)
proof
  fix actual-chunk arp arp-path arn arn-path
  assume step-auth:
    fri-layer-chunk-authenticated
      (trace-roots ! (length trace-bs - 1))
      (fri-evidence-layer-len trace-roots (length trace-bs - 1))
      (fri-evidence-layer-idx trace-roots fri-query-idxs round-idx
       (length trace-bs - 1))
      actual-chunk final-state ∧
    fri-layer-step-evidence
      (trace-roots ! (length trace-bs - 1))
      (trace-bs ! (length trace-bs - 1))
      (fri-evidence-layer-len trace-roots (length trace-bs - 1))
      (fri-evidence-layer-idx trace-roots fri-query-idxs round-idx
       (length trace-bs - 1))
      (2 ^ (length trace-bs - 1))
      (fri-sibling-index
       (fri-evidence-layer-len trace-roots (length trace-bs - 1))
       (fri-evidence-layer-idx trace-roots fri-query-idxs round-idx
        (length trace-bs - 1)))
      arp arp-path arn arn-path
      (fri-evidence-next-idx trace-roots fri-query-idxs round-idx
       (length trace-bs - 1))
      trace-final actual-chunk
  have False
    by (rule generic-fri-final-conflict-false-from-authenticated-step
      [OF forced recorded-auth conjunct1[OF step-auth]
       conjunct2[OF step-auth] no-merkle[unfolded out-eq] neq])
  then show False .
qed
have witness:
  ∃ trace-roots trace-bs trace-final dg composition-roots
    composition-bs composition-final fri-query-idxs trace-round-layers
    composition-round-layers result final-state round-idx v.
  out = Some (result, final-state) ∧
  accepted-fri-opening-transcript s out trace-roots trace-bs
    trace-final dg composition-roots composition-bs composition-final
    fri-query-idxs trace-round-layers composition-round-layers ∧
  0 < length trace-bs ∧
  round-idx < length fri-query-idxs ∧
  generic-fri-round-forced-next-value trace-roots trace-bs
    fri-query-idxs trace-round-layers round-idx (length trace-bs - 1)
  v ∧

```

```

v ≠ trace-final ∧
¬ partial-merkle-inconsistency-bad s out ∧
generic-fri-recorded-layer-chunk-authenticated trace-roots
  fri-query-idxs trace-round-layers final-state round-idx
  (length trace-bs - 1) ∧
(∀ actual-chunk axp axp-path axn axn-path.
  ¬ (fri-layer-chunk-authenticated
    (trace-roots ! (length trace-bs - 1))
    (fri-evidence-layer-len trace-roots (length trace-bs - 1))
    (fri-evidence-layer-idx trace-roots fri-query-idxs round-idx
      (length trace-bs - 1))
    actual-chunk final-state ∧
    fri-layer-step-evidence
      (trace-roots ! (length trace-bs - 1))
      (trace-bs ! (length trace-bs - 1))
      (fri-evidence-layer-len trace-roots (length trace-bs - 1))
      (fri-evidence-layer-idx trace-roots fri-query-idxs round-idx
        (length trace-bs - 1))
      (2 ^ (length trace-bs - 1))
      (fri-sibling-index
        (fri-evidence-layer-len trace-roots (length trace-bs - 1))
        (fri-evidence-layer-idx trace-roots fri-query-idxs round-idx
          (length trace-bs - 1)))
      axp axp-path axn axn-path
      (fri-evidence-next-idx trace-roots fri-query-idxs round-idx
        (length trace-bs - 1))
      trace-final actual-chunk))
using out-eq fri-openings nonempty round-bound forced neq no-merkle
  recorded-auth no-step
by blast
show ?thesis
  unfolding trace-fri-header-tied-final-value-selected-step-missing-gap-def
  using assms witness by blast
qed

```

```

lemma wp-trace-fri-header-tied-final-value-authenticated-replay-gap-bound-from-selected-step-missing:
  fixes G :: prob
  assumes gap-bound:
    wp-event verify-monad
      (trace-fri-header-tied-final-value-selected-step-missing-gap s) s
    ≤ G
  shows
    wp-event verify-monad
      (trace-fri-header-tied-final-value-authenticated-replay-gap s) s
    ≤ G
proof -
  have wp-event verify-monad
    (trace-fri-header-tied-final-value-authenticated-replay-gap s) s ≤
    wp-event verify-monad

```

```

    (trace-fri-header-tied-final-value-selected-step-missing-gap s) s
  by (rule wp-event-mono)
    (rule
      trace-fri-header-tied-final-value-authenticated-replay-gap-imp-selected-step-missing)
  then show ?thesis
    by (rule order-trans[OF - gap-bound])
qed

lemma trace-fri-header-tied-final-value-replay-gap-imp-merkle-or-slot-or-authenticated:
  assumes trace-fri-header-tied-final-value-replay-gap s out
  shows
    partial-merkle-inconsistency-bad s out ∨
    trace-fri-header-tied-authenticated-slot-recorded-chunk-gap s out ∨
    trace-fri-header-tied-final-value-authenticated-replay-gap s out
proof -
  from assms have sampled:
    trace-fri-header-tied-sampled-final-value-conflict s out
  unfolding trace-fri-header-tied-final-value-replay-gap-def by simp
  from trace-fri-header-tied-sampled-final-value-conflictE[OF sampled]
  obtain trace-roots trace-bs trace-final dg composition-roots
    composition-bs composition-final fri-query-idxs trace-round-layers
    composition-round-layers fr as rest trace-openings trace-table
    round-idx v
  where fri-openings:
    accepted-fri-opening-transcript s out trace-roots trace-bs
    trace-final dg composition-roots composition-bs composition-final
    fri-query-idxs trace-round-layers composition-round-layers
  and header:
    verifier-header-transcript s fr trace-roots trace-final as dg
    composition-roots composition-final rest
  and partial:
    accepted-with-partial-trace-openings s out fr fri-query-idxs
    trace-openings
  and cand: partial-trace-table-candidate trace-table trace-openings
  and not-low: ¬ trace-table-low-degree trace-table
  and nonempty: 0 < length trace-bs
  and round-bound: round-idx < length fri-query-idxs
  and forced:
    generic-fri-round-forced-next-value trace-roots trace-bs
    fri-query-idxs trace-round-layers round-idx (length trace-bs - 1) v
  and neg: v ≠ trace-final
  by blast
  from fri-openings obtain result final-state where out-eq:
    out = Some (result, final-state)
  unfolding accepted-fri-opening-transcript-def by blast
  show ?thesis
proof (cases partial-merkle-inconsistency-bad s out)
  case True
  then show ?thesis by simp

```

```

next
case no-merkle: False
let ?j = length trace-bs - 1
have len-trace-bs: length trace-bs = length trace-roots
  using accepted-fri-opening-transcript-shapes(2)[OF fri-openings] .
have layer-bound: ?j < length trace-roots
  using nonempty len-trace-bs by simp
show ?thesis
proof (cases generic-fri-recorded-layer-chunk-authenticated trace-roots
  fri-query-idxs trace-round-layers final-state round-idx ?j)
case True
have authenticated-gap:
  trace-fri-header-tied-final-value-authenticated-replay-gap s out
unfolding
  trace-fri-header-tied-final-value-authenticated-replay-gap-def
by (intro exI conjI)
  (rule out-eq, rule fri-openings, rule header, rule partial,
   rule cand, rule not-low, rule nonempty, rule round-bound,
   rule forced, rule neg, use no-merkle True in simp-all)
then show ?thesis by simp
next
case missing-recorded: False
have round-bound-rounds: round-idx < rounds
  using round-bound
  accepted-fri-opening-transcript-shapes(7)[OF fri-openings]
  by simp
have raw-layer:
   $\bigwedge_{\text{raw.}}$ 
  0 < fri-layer-lengths (length trace-roots) (clength * scale) !
  ?j  $\wedge$ 
  fri-layer-indices (length trace-roots) (index (to-nat raw))
  (clength * scale) ! ?j <
  fri-layer-lengths (length trace-roots) (clength * scale) ! ?j
  by (rule accepted-fri-opening-transcript-trace-raw-layer-bound
    [OF fri-openings layer-bound])
from accepted-fri-opening-transcript-trace-selected-chunk-authenticated-at
  [OF fri-openings out-eq round-bound-rounds layer-bound raw-layer]
obtain auth-chunk where auth:
  fri-layer-chunk-authenticated (trace-roots ! ?j)
  (fri-layer-lengths (length trace-roots) (clength * scale) ! ?j)
  (fri-layer-indices (length trace-roots) (fri-query-idxs ! round-idx)
    (clength * scale) ! ?j)
  auth-chunk final-state
  by blast
have step:
  fri-round-layer-evidence trace-roots trace-bs fri-query-idxs
  round-idx ?j trace-round-layers
  by (rule accepted-fri-opening-transcript-trace-round-layer-evidence
    [OF fri-openings round-bound-rounds layer-bound])

```

```

have generic-step:
  generic-fri-transcript-step-with-authenticated-chunk trace-roots
    trace-bs fri-query-idxs trace-round-layers final-state
    round-idx ?j
by (rule generic-fri-transcript-step-with-authenticated-chunkI
      [OF step])
  (use auth in
    ⟨simp add: fri-evidence-layer-len-def
      fri-evidence-layer-idx-def⟩)
have slot-gap:
  generic-fri-authenticated-slot-recorded-chunk-gap trace-roots
    fri-query-idxs trace-round-layers final-state
by (rule
  generic-fri-transcript-step-auth-missing-imp-authenticated-slot-recorded-chunk-gap
    [OF generic-step missing-recorded])
have route-slot-gap:
  trace-fri-header-tied-authenticated-slot-recorded-chunk-gap s out
unfolding trace-fri-header-tied-authenticated-slot-recorded-chunk-gap-def
by (intro exI conjI)
  (rule out-eq, rule fri-openings, rule header, rule partial,
    rule cand, rule not-low, rule slot-gap)
then show ?thesis by simp
qed
qed
qed

lemma wp-trace-fri-header-tied-final-value-replay-gap-bound-from-merkle-slot-and-authenticated:
fixes M A R :: prob
assumes merkle-bound:
  wp-event verify-monad (partial-merkle-inconsistency-bad s) s ≤ M
and slot-bound:
  wp-event verify-monad
    (trace-fri-header-tied-authenticated-slot-recorded-chunk-gap s) s ≤ A
and replay-bound:
  wp-event verify-monad
    (trace-fri-header-tied-final-value-authenticated-replay-gap s) s ≤ R
shows
  wp-event verify-monad
    (trace-fri-header-tied-final-value-replay-gap s) s ≤ M + A + R
proof –
let ?P = partial-merkle-inconsistency-bad s
let ?Q = trace-fri-header-tied-authenticated-slot-recorded-chunk-gap s
let ?R =
  trace-fri-header-tied-final-value-authenticated-replay-gap s
have event-le:
  wp-event verify-monad
    (trace-fri-header-tied-final-value-replay-gap s) s ≤
  wp-event verify-monad (λout. ?P out ∨ ?Q out ∨ ?R out) s
by (rule wp-event-mono)

```

```

(rule
  trace-fri-header-tied-final-value-replay-gap-imp-merkle-or-slot-or-authenticated)
have union-le:
  wp-event verify-monad ( $\lambda out. ?P \text{ out} \vee ?Q \text{ out} \vee ?R \text{ out}$ )  $s \leq$ 
    wp-event verify-monad  $?P \ s +$ 
    wp-event verify-monad  $?Q \ s +$ 
    wp-event verify-monad  $?R \ s$ 
proof -
  have wp-event verify-monad ( $\lambda out. ?P \text{ out} \vee ?Q \text{ out} \vee ?R \text{ out}$ )  $s \leq$ 
    wp-event verify-monad  $?P \ s +$ 
    wp-event verify-monad ( $\lambda out. ?Q \text{ out} \vee ?R \text{ out}$ )  $s$ 
  by (rule wp-event-union-bound)
  also have ...  $\leq$ 
    wp-event verify-monad  $?P \ s +$ 
    (wp-event verify-monad  $?Q \ s +$  wp-event verify-monad  $?R \ s$ )
  by (intro add-mono order.refl wp-event-union-bound)
  finally show ?thesis
  by (simp add: algebra-simps)
qed
have bounds:
  wp-event verify-monad  $?P \ s +$ 
  wp-event verify-monad  $?Q \ s +$ 
  wp-event verify-monad  $?R \ s \leq M + A + R$ 
using merkle-bound slot-bound replay-bound
by (intro add-mono)
show ?thesis
by (rule order-trans[OF event-le order-trans[OF union-le bounds]])
qed

lemma wp-trace-fri-header-tied-final-value-replay-gap-bound-from-merkle-slot-and-selected-step:
  fixes  $M \ A \ R :: \text{prob}$ 
  assumes merkle-bound:
    wp-event verify-monad (partial-merkle-inconsistency-bad  $s$ )  $s \leq M$ 
  and slot-bound:
    wp-event verify-monad
      (trace-fri-header-tied-authenticated-slot-recorded-chunk-gap  $s$ )  $s \leq A$ 
  and selected-step-bound:
    wp-event verify-monad
      (trace-fri-header-tied-final-value-selected-step-missing-gap  $s$ )  $s$ 
       $\leq R$ 
  shows
    wp-event verify-monad
      (trace-fri-header-tied-final-value-replay-gap  $s$ )  $s \leq M + A + R$ 
proof -
  have authenticated-bound:
    wp-event verify-monad
      (trace-fri-header-tied-final-value-authenticated-replay-gap  $s$ )  $s$ 
       $\leq R$ 
  by (rule

```

$wp\text{-}trace\text{-}fri\text{-}header\text{-}tied\text{-}final\text{-}value\text{-}authenticated\text{-}replay\text{-}gap\text{-}bound\text{-}from\text{-}selected\text{-}step\text{-}missing$
 $[OF\ selected\text{-}step\text{-}bound])$
show $?thesis$
by (rule
 $wp\text{-}trace\text{-}fri\text{-}header\text{-}tied\text{-}final\text{-}value\text{-}replay\text{-}gap\text{-}bound\text{-}from\text{-}merkle\text{-}slot\text{-}and\text{-}authenticated$)
(rule merkle-bound, rule slot-bound, rule authenticated-bound)
qed

definition $trace\text{-}fri\text{-}header\text{-}tied\text{-}base\text{-}head\text{-}value\text{-}mismatch\text{-}gap$
 $:: (f, 'a)\ protocol\text{-}channel\text{-}scheme \Rightarrow$
 $(unit\ list \times (f, 'a)\ protocol\text{-}channel\text{-}scheme)\ option \Rightarrow bool$
where

$trace\text{-}fri\text{-}header\text{-}tied\text{-}base\text{-}head\text{-}value\text{-}mismatch\text{-}gap\ s\ out \longleftrightarrow$
 $(\exists\ trace\text{-}roots\ trace\text{-}bs\ trace\text{-}final\ dg\ composition\text{-}roots$
 $\ composition\text{-}bs\ composition\text{-}final\ fri\text{-}query\text{-}idxs\ trace\text{-}round\text{-}layers$
 $\ composition\text{-}round\text{-}layers\ fr\ as\ rest\ trace\text{-}openings\ trace\text{-}table$
 $\ round\text{-}idx\ xp\ xp\text{-}path\ xn\ xn\text{-}path\ result\ final\text{-}state.$
 $out = Some\ (result, final\text{-}state) \wedge$
 $accepted\text{-}fri\text{-}opening\text{-}transcript\ s\ out\ trace\text{-}roots\ trace\text{-}bs$
 $\ trace\text{-}final\ dg\ composition\text{-}roots\ composition\text{-}bs\ composition\text{-}final$
 $\ fri\text{-}query\text{-}idxs\ trace\text{-}round\text{-}layers\ composition\text{-}round\text{-}layers \wedge$
 $verifier\text{-}header\text{-}transcript\ s\ fr\ trace\text{-}roots\ trace\text{-}final\ as\ dg$
 $\ composition\text{-}roots\ composition\text{-}final\ rest \wedge$
 $accepted\text{-}with\text{-}partial\text{-}trace\text{-}openings\ s\ out\ fr\ fri\text{-}query\text{-}idxs$
 $\ trace\text{-}openings \wedge$
 $partial\text{-}trace\text{-}table\text{-}candidate\ trace\text{-}table\ trace\text{-}openings \wedge$
 $\neg\ trace\text{-}table\text{-}low\text{-}degree\ trace\text{-}table \wedge$
 $round\text{-}idx < length\ fri\text{-}query\text{-}idxs \wedge$
 $0 < length\ trace\text{-}bs \wedge$
 $fri\text{-}layer\text{-}step\text{-}evidence$
 $\ (trace\text{-}roots\ !\ 0)$
 $\ (trace\text{-}bs\ !\ 0)$
 $\ (fri\text{-}evidence\text{-}layer\text{-}len\ trace\text{-}roots\ 0)$
 $\ (fri\text{-}evidence\text{-}layer\text{-}idx\ trace\text{-}roots\ fri\text{-}query\text{-}idxs\ round\text{-}idx\ 0)$
 $\ 1$
 $\ (fri\text{-}sibling\text{-}index\ (fri\text{-}evidence\text{-}layer\text{-}len\ trace\text{-}roots\ 0)$
 $\ \ (fri\text{-}evidence\text{-}layer\text{-}idx\ trace\text{-}roots\ fri\text{-}query\text{-}idxs\ round\text{-}idx\ 0))$
 $\ xp\ xp\text{-}path\ xn\ xn\text{-}path$
 $\ (fri\text{-}evidence\text{-}next\text{-}idx\ trace\text{-}roots\ fri\text{-}query\text{-}idxs\ round\text{-}idx\ 0)$
 $\ (fri\text{-}evidence\text{-}next\text{-}value\ trace\text{-}roots\ trace\text{-}bs\ fri\text{-}query\text{-}idxs$
 $\ \ round\text{-}idx\ 0\ xp\ xn)$
 $\ (trace\text{-}round\text{-}layers\ !\ round\text{-}idx\ !\ 0) \wedge$
 $\neg\ fri\text{-}opening\text{-}matches\text{-}table$
 $\ (fri\text{-}evidence\text{-}layer\text{-}len\ trace\text{-}roots\ 0)$
 $\ (fri\text{-}evidence\text{-}layer\text{-}idx\ trace\text{-}roots\ fri\text{-}query\text{-}idxs\ round\text{-}idx\ 0)$
 $\ trace\text{-}table\ xp\ xn \wedge$
 $\neg\ (trace\text{-}openings\ !\ round\text{-}idx \neq [] \wedge$
 $\ opening\text{-}index\ (hd\ (trace\text{-}openings\ !\ round\text{-}idx)) =$
 $\ \ fri\text{-}evidence\text{-}layer\text{-}idx\ trace\text{-}roots\ fri\text{-}query\text{-}idxs\ round\text{-}idx\ 0 \wedge$

opening-value (*hd* (*trace-openings* ! *round-idx*)) = *xp*))

lemma *trace-fri-header-tied-base-value-alignment-gap-imp-head-value-mismatch-gap*:

assumes *trace-fri-header-tied-base-value-alignment-gap s out*

shows *trace-fri-header-tied-base-head-value-mismatch-gap s out*

proof –

from *assms* **have** *sampled*:

trace-fri-header-tied-sampled-base-opening-conflict s out

and *no-aligned*:

¬ *trace-fri-header-tied-value-aligned-base-opening-conflict s out*

unfolding *trace-fri-header-tied-base-value-alignment-gap-def* **by** *simp-all*

from *sampled* **obtain** *trace-roots trace-bs trace-final dg composition-roots*

composition-bs composition-final fri-query-idxs trace-round-layers

composition-round-layers fr as rest trace-openings trace-table

where *fri-openings*:

accepted-fri-opening-transcript s out trace-roots trace-bs

trace-final dg composition-roots composition-bs composition-final

fri-query-idxs trace-round-layers composition-round-layers

and *header*:

verifier-header-transcript s fr trace-roots trace-final as dg

composition-roots composition-final rest

and *partial*:

accepted-with-partial-trace-openings s out fr fri-query-idxs

trace-openings

and *cand*: *partial-trace-table-candidate trace-table trace-openings*

and *not-low*: ¬ *trace-table-low-degree trace-table*

and *generic*:

generic-fri-sampled-base-opening-conflict trace-table trace-roots

trace-bs fri-query-idxs trace-round-layers

unfolding *trace-fri-header-tied-sampled-base-opening-conflict-def*

by *blast*

from *fri-openings* **obtain** *result final-state* **where** *out-eq*:

out = *Some (result, final-state)*

unfolding *accepted-fri-opening-transcript-def* **by** *blast*

from *generic* **obtain** *round-idx xp xp-path xn xn-path* **where**

trace-bs-nonempty: *0 < length trace-bs*

and *round-bound*: *round-idx < length fri-query-idxs*

and *step*:

fri-layer-step-evidence

(*trace-roots* ! *0*)

(*trace-bs* ! *0*)

(*fri-evidence-layer-len trace-roots* *0*)

(*fri-evidence-layer-idx trace-roots fri-query-idxs round-idx* *0*)

1

(*fri-sibling-index (fri-evidence-layer-len trace-roots* *0)*

(*fri-evidence-layer-idx trace-roots fri-query-idxs round-idx* *0*))

xp xp-path xn xn-path

(*fri-evidence-next-idx trace-roots fri-query-idxs round-idx* *0*)

(*fri-evidence-next-value trace-roots trace-bs fri-query-idxs*

```

    round-idx 0 xp xn)
  (trace-round-layers ! round-idx ! 0)
and no-match:
  ¬ fri-opening-matches-table
  (fri-evidence-layer-len trace-roots 0)
  (fri-evidence-layer-idx trace-roots fri-query-idxs round-idx 0)
  trace-table xp xn
unfolding generic-fri-sampled-base-opening-conflict-def
by blast
have no-head:
  ¬ (trace-openings ! round-idx ≠ [] ∧
    opening-index (hd (trace-openings ! round-idx)) =
      fri-evidence-layer-idx trace-roots fri-query-idxs round-idx 0 ∧
    opening-value (hd (trace-openings ! round-idx)) = xp)
proof
assume head:
  trace-openings ! round-idx ≠ [] ∧
  opening-index (hd (trace-openings ! round-idx)) =
    fri-evidence-layer-idx trace-roots fri-query-idxs round-idx 0 ∧
  opening-value (hd (trace-openings ! round-idx)) = xp
have aligned:
  trace-fri-header-tied-value-aligned-base-opening-conflict s out
unfolding trace-fri-header-tied-value-aligned-base-opening-conflict-def
by (intro exI conjI)
  (rule out-eq, rule fri-openings, rule header, rule partial, rule cand,
    rule not-low, rule round-bound, rule trace-bs-nonempty,
    use head step no-match in simp-all)
then show False
using no-aligned by contradiction
qed
show ?thesis
unfolding trace-fri-header-tied-base-head-value-mismatch-gap-def
by (intro exI conjI)
  (rule out-eq, rule fri-openings, rule header, rule partial, rule cand,
    rule not-low, rule round-bound, rule trace-bs-nonempty, rule step,
    rule no-match, rule no-head)
qed

lemma wp-trace-fri-header-tied-base-value-alignment-gap-bound-from-head-value-mismatch-gap:
fixes G :: prob
assumes mismatch-bound:
  wp-event verify-monad
  (trace-fri-header-tied-base-head-value-mismatch-gap s) s ≤ G
shows
  wp-event verify-monad
  (trace-fri-header-tied-base-value-alignment-gap s) s ≤ G
proof –
have wp-event verify-monad
  (trace-fri-header-tied-base-value-alignment-gap s) s ≤

```

$wp\text{-}event\ verify\text{-}monad$
 $(trace\text{-}fri\text{-}header\text{-}tied\text{-}base\text{-}head\text{-}value\text{-}mismatch\text{-}gap\ s)\ s$
by (rule $wp\text{-}event\text{-}mono$)
(rule
 $trace\text{-}fri\text{-}header\text{-}tied\text{-}base\text{-}value\text{-}alignment\text{-}gap\text{-}imp\text{-}head\text{-}value\text{-}mismatch\text{-}gap$)
then show $?thesis$
by (rule $order\text{-}trans[OF - mismatch\text{-}bound]$)
qed

definition $trace\text{-}fri\text{-}header\text{-}tied\text{-}base\text{-}head\text{-}value\text{-}neg\text{-}gap$
 $:: (f, 'a)\ protocol\text{-}channel\text{-}scheme \Rightarrow$
 $(unit\ list \times (f, 'a)\ protocol\text{-}channel\text{-}scheme)\ option \Rightarrow bool$
where
 $trace\text{-}fri\text{-}header\text{-}tied\text{-}base\text{-}head\text{-}value\text{-}neg\text{-}gap\ s\ out \longleftrightarrow$
 $(\exists\ trace\text{-}roots\ trace\text{-}bs\ trace\text{-}final\ dg\ composition\text{-}roots$
 $\quad composition\text{-}bs\ composition\text{-}final\ fri\text{-}query\text{-}idxs\ trace\text{-}round\text{-}layers$
 $\quad composition\text{-}round\text{-}layers\ fr\ as\ rest\ trace\text{-}openings\ trace\text{-}table$
 $\quad round\text{-}idx\ xp\ xp\text{-}path\ xn\ xn\text{-}path\ result\ final\text{-}state.$
 $out = Some\ (result, final\text{-}state) \wedge$
 $accepted\text{-}fri\text{-}opening\text{-}transcript\ s\ out\ trace\text{-}roots\ trace\text{-}bs$
 $\quad trace\text{-}final\ dg\ composition\text{-}roots\ composition\text{-}bs\ composition\text{-}final$
 $\quad fri\text{-}query\text{-}idxs\ trace\text{-}round\text{-}layers\ composition\text{-}round\text{-}layers \wedge$
 $verifier\text{-}header\text{-}transcript\ s\ fr\ trace\text{-}roots\ trace\text{-}final\ as\ dg$
 $\quad composition\text{-}roots\ composition\text{-}final\ rest \wedge$
 $accepted\text{-}with\text{-}partial\text{-}trace\text{-}openings\ s\ out\ fr\ fri\text{-}query\text{-}idxs$
 $\quad trace\text{-}openings \wedge$
 $partial\text{-}trace\text{-}table\text{-}candidate\ trace\text{-}table\ trace\text{-}openings \wedge$
 $\neg\ trace\text{-}table\text{-}low\text{-}degree\ trace\text{-}table \wedge$
 $round\text{-}idx < length\ fri\text{-}query\text{-}idxs \wedge$
 $0 < length\ trace\text{-}bs \wedge$
 $trace\text{-}openings ! round\text{-}idx \neq [] \wedge$
 $opening\text{-}index\ (hd\ (trace\text{-}openings ! round\text{-}idx)) =$
 $\quad fri\text{-}evidence\text{-}layer\text{-}idx\ trace\text{-}roots\ fri\text{-}query\text{-}idxs\ round\text{-}idx\ 0 \wedge$
 $opening\text{-}value\ (hd\ (trace\text{-}openings ! round\text{-}idx)) \neq xp \wedge$
 $fri\text{-}layer\text{-}step\text{-}evidence$
 $\quad (trace\text{-}roots ! 0)$
 $\quad (trace\text{-}bs ! 0)$
 $\quad (fri\text{-}evidence\text{-}layer\text{-}len\ trace\text{-}roots\ 0)$
 $\quad (fri\text{-}evidence\text{-}layer\text{-}idx\ trace\text{-}roots\ fri\text{-}query\text{-}idxs\ round\text{-}idx\ 0)$
 $\quad 1$
 $\quad (fri\text{-}sibling\text{-}index\ (fri\text{-}evidence\text{-}layer\text{-}len\ trace\text{-}roots\ 0)$
 $\quad \quad (fri\text{-}evidence\text{-}layer\text{-}idx\ trace\text{-}roots\ fri\text{-}query\text{-}idxs\ round\text{-}idx\ 0))$
 $\quad xp\ xp\text{-}path\ xn\ xn\text{-}path$
 $\quad (fri\text{-}evidence\text{-}next\text{-}idx\ trace\text{-}roots\ fri\text{-}query\text{-}idxs\ round\text{-}idx\ 0)$
 $\quad (fri\text{-}evidence\text{-}next\text{-}value\ trace\text{-}roots\ trace\text{-}bs\ fri\text{-}query\text{-}idxs$
 $\quad \quad round\text{-}idx\ 0\ xp\ xn)$
 $\quad (trace\text{-}round\text{-}layers ! round\text{-}idx ! 0) \wedge$
 $\neg\ fri\text{-}opening\text{-}matches\text{-}table$
 $\quad (fri\text{-}evidence\text{-}layer\text{-}len\ trace\text{-}roots\ 0)$

(*fri-evidence-layer-idx trace-roots fri-query-idxs round-idx 0*)
trace-table xp xn)

lemma *trace-fri-header-tied-base-head-value-mismatch-gap-imp-neq-gap:*

assumes *trace-fri-header-tied-base-head-value-mismatch-gap s out*

shows *trace-fri-header-tied-base-head-value-neq-gap s out*

proof –

from *assms show ?thesis*

unfolding *trace-fri-header-tied-base-head-value-mismatch-gap-def*

proof (*elim exE conjE*)

fix *trace-roots trace-bs trace-final dg composition-roots composition-bs*

composition-final fri-query-idxs trace-round-layers

composition-round-layers fr as rest trace-openings trace-table

round-idx xp xp-path xn xn-path result final-state

assume *out-eq: out = Some (result, final-state)*

assume *fri-openings:*

accepted-fri-opening-transcript s out trace-roots trace-bs

trace-final dg composition-roots composition-bs composition-final

fri-query-idxs trace-round-layers composition-round-layers

assume *header:*

verifier-header-transcript s fr trace-roots trace-final as dg

composition-roots composition-final rest

assume *partial:*

accepted-with-partial-trace-openings s out fr fri-query-idxs

trace-openings

assume *cand: partial-trace-table-candidate trace-table trace-openings*

assume *not-low: $\neg$ trace-table-low-degree trace-table*

assume *round-bound: round-idx < length fri-query-idxs*

assume *trace-bs-nonempty: 0 < length trace-bs*

assume *step:*

fri-layer-step-evidence

(trace-roots ! 0)

(trace-bs ! 0)

(fri-evidence-layer-len trace-roots 0)

(fri-evidence-layer-idx trace-roots fri-query-idxs round-idx 0)

1

(fri-sibling-index (fri-evidence-layer-len trace-roots 0)

(fri-evidence-layer-idx trace-roots fri-query-idxs round-idx 0))

xp xp-path xn xn-path

(fri-evidence-next-idx trace-roots fri-query-idxs round-idx 0)

(fri-evidence-next-value trace-roots trace-bs fri-query-idxs

round-idx 0 xp xn)

(trace-round-layers ! round-idx ! 0)

assume *no-match:*

$\neg$ *fri-opening-matches-table*

(fri-evidence-layer-len trace-roots 0)

(fri-evidence-layer-idx trace-roots fri-query-idxs round-idx 0)

trace-table xp xn

assume *no-head:*

```

  ¬ (trace-openings ! round-idx ≠ [] ∧
    opening-index (hd (trace-openings ! round-idx)) =
      fri-evidence-layer-idx trace-roots fri-query-idxs round-idx 0 ∧
    opening-value (hd (trace-openings ! round-idx)) = xp)
have round-bound-rounds: round-idx < rounds
  using round-bound
  accepted-fri-opening-transcript-shapes(7)[OF fri-openings]
  by simp
have roots-nonempty: 0 < length trace-roots
  using accepted-fri-opening-transcript-shapes(2)[OF fri-openings]
  trace-bs-nonempty by simp
have idxs:
  map opening-index (trace-openings ! round-idx) =
    powers-scaled (fri-query-idxs ! round-idx)
  by (rule accepted-with-partial-trace-openings-shapes(3)
    [OF partial round-bound-rounds])
have openings-nonempty: trace-openings ! round-idx ≠ []
  using idxs powers-pos unfolding powers-scaled-def by auto
have hd-idx:
  opening-index (hd (trace-openings ! round-idx)) =
    fri-evidence-layer-idx trace-roots fri-query-idxs round-idx 0
proof -
  have opening-index (hd (trace-openings ! round-idx)) =
    fri-query-idxs ! round-idx
  proof -
    have opening-index (hd (trace-openings ! round-idx)) =
      hd (map opening-index (trace-openings ! round-idx))
    using openings-nonempty by (cases trace-openings ! round-idx) auto
    also have ... = hd (powers-scaled (fri-query-idxs ! round-idx))
    using idxs by simp
    also have ... = fri-query-idxs ! round-idx
    by (rule powers-scaled-hd)
    finally show ?thesis .
  qed
moreover have
  fri-evidence-layer-idx trace-roots fri-query-idxs round-idx 0 =
    fri-query-idxs ! round-idx
  using roots-nonempty
  unfolding fri-evidence-layer-idx-def
  by (cases trace-roots) simp-all
ultimately show ?thesis by simp
qed
have hd-neq:
  opening-value (hd (trace-openings ! round-idx)) ≠ xp
  using no-head openings-nonempty hd-idx by blast
show trace-fri-header-tied-base-head-value-neq-gap s out
  unfolding trace-fri-header-tied-base-head-value-neq-gap-def
  by (intro exI conjI)
  (rule out-eq, rule fri-openings, rule header, rule partial, rule cand,

```

rule not-low, rule round-bound, rule trace-bs-nonempty,
rule openings-nonempty, rule hd-idx, rule hd-neq, rule step,
rule no-match)

qed
qed

lemma *wp-trace-fri-header-tied-base-head-value-mismatch-gap-bound-from-neq-gap:*
fixes $G :: \text{prob}$
assumes *neq-bound:*
 wp-event verify-monad
 $(\text{trace-fri-header-tied-base-head-value-neq-gap } s) \ s \leq G$
shows
 wp-event verify-monad
 $(\text{trace-fri-header-tied-base-head-value-mismatch-gap } s) \ s \leq G$
proof –
 have *wp-event verify-monad*
 $(\text{trace-fri-header-tied-base-head-value-mismatch-gap } s) \ s \leq$
 wp-event verify-monad
 $(\text{trace-fri-header-tied-base-head-value-neq-gap } s) \ s$
 by (*rule wp-event-mono*)
 (*rule trace-fri-header-tied-base-head-value-mismatch-gap-imp-neq-gap*)
 then show *?thesis*
 by (*rule order-trans[OF - neq-bound]*)
qed

lemma *trace-fri-header-tied-assignment-imp-authenticated-or-auth-gap:*
assumes *trace-fri-header-tied-sampled-assignment-conflict s out*
shows
 trace-fri-header-tied-sampled-assignment-conflict-with-authenticated-same-layer
 $s \text{ out} \vee$
 trace-fri-header-tied-assignment-auth-gap s out
using *assms*
unfolding *trace-fri-header-tied-assignment-auth-gap-def*
by *blast*

lemma *trace-fri-header-tied-assignment-auth-gap-imp-same-layer-auth-gap:*
assumes *trace-fri-header-tied-assignment-auth-gap s out*
shows *trace-fri-header-tied-same-layer-auth-gap s out*
proof –
 from *assms* **have** *conflict:*
 trace-fri-header-tied-sampled-assignment-conflict s out
 and *no-auth:*
 $\neg \text{trace-fri-header-tied-sampled-assignment-conflict-with-authenticated-same-layer}$
 $s \text{ out}$
 unfolding *trace-fri-header-tied-assignment-auth-gap-def*
 by *simp-all*
from *conflict* **obtain** *trace-roots trace-bs trace-final dg composition-roots*
 composition-bs composition-final fri-query-idxs trace-round-layers
 composition-round-layers fr as rest trace-openings trace-table

where *fri-openings*:
accepted-fri-opening-transcript s out trace-roots trace-bs
trace-final dg composition-roots composition-bs composition-final
fri-query-idxs trace-round-layers composition-round-layers
and *header*:
verifier-header-transcript s fr trace-roots trace-final as dg
composition-roots composition-final rest
and *partial*:
accepted-with-partial-trace-openings s out fr fri-query-idxs
trace-openings
and *cand*: *partial-trace-table-candidate trace-table trace-openings*
and *not-low*: $\neg$ *trace-table-low-degree trace-table*
and *sampled*:
generic-fri-sampled-assignment-conflict trace-table trace-roots
trace-bs trace-final fri-query-idxs trace-round-layers
unfolding *trace-fri-header-tied-sampled-assignment-conflict-def*
by *blast*
from *fri-openings* **obtain** *result final-state* **where** *out-eq*:
out = Some (result, final-state)
unfolding *accepted-fri-opening-transcript-def* **by** *blast*
have *split*:
generic-fri-sampled-assignment-conflict-with-authenticated-same-layer
trace-table trace-roots trace-bs trace-final fri-query-idxs
trace-round-layers final-state $\vee$
(generic-fri-sampled-same-layer-opening-conflict trace-roots trace-bs
fri-query-idxs trace-round-layers $\wedge$
 $\neg$ *generic-fri-sampled-same-layer-opening-conflict-with-authenticated-chunks*
trace-roots trace-bs fri-query-idxs trace-round-layers final-state)
by (rule
generic-fri-sampled-assignment-conflict-imp-authenticated-or-same-layer-auth-gap
[*OF sampled*])
then show *?thesis*
proof
assume *auth*:
generic-fri-sampled-assignment-conflict-with-authenticated-same-layer
trace-table trace-roots trace-bs trace-final fri-query-idxs
trace-round-layers final-state
have
trace-fri-header-tied-sampled-assignment-conflict-with-authenticated-same-layer
s out
unfolding
trace-fri-header-tied-sampled-assignment-conflict-with-authenticated-same-layer-def
by (*intro exI conjI*)
(*rule fri-openings, rule out-eq, rule header, rule partial, rule cand,*
rule not-low, rule auth)
then show *?thesis*
using *no-auth* **by** *contradiction*
next
assume *gap*:

```

generic-fri-sampled-same-layer-opening-conflict trace-roots trace-bs
  fri-query-idxs trace-round-layers  $\wedge$ 
 $\neg$  generic-fri-sampled-same-layer-opening-conflict-with-authenticated-chunks
  trace-roots trace-bs fri-query-idxs trace-round-layers final-state
show ?thesis
  unfolding trace-fri-header-tied-same-layer-auth-gap-def
  by (intro exI conjI)
    (rule out-eq, rule fri-openings, rule header, rule partial, rule cand,
     rule not-low, use gap in simp-all)
qed
qed

lemma trace-fri-header-tied-same-layer-auth-gap-imp-recorded-auth-gap:
  assumes trace-fri-header-tied-same-layer-auth-gap s out
  shows trace-fri-header-tied-same-layer-recorded-auth-gap s out
proof -
  from assms obtain trace-roots trace-bs trace-final dg composition-roots
    composition-bs composition-final fri-query-idxs trace-round-layers
    composition-round-layers fr as rest trace-openings trace-table
    result final-state
  where out-eq: out = Some (result, final-state)
  and fri-openings:
    accepted-fri-opening-transcript s out trace-roots trace-bs
    trace-final dg composition-roots composition-bs composition-final
    fri-query-idxs trace-round-layers composition-round-layers
  and header:
    verifier-header-transcript s fr trace-roots trace-final as dg
    composition-roots composition-final rest
  and partial:
    accepted-with-partial-trace-openings s out fr fri-query-idxs
    trace-openings
  and cand: partial-trace-table-candidate trace-table trace-openings
  and not-low:  $\neg$  trace-table-low-degree trace-table
  and same:
    generic-fri-sampled-same-layer-opening-conflict trace-roots trace-bs
    fri-query-idxs trace-round-layers
  and no-auth:
     $\neg$  generic-fri-sampled-same-layer-opening-conflict-with-authenticated-chunks
    trace-roots trace-bs fri-query-idxs trace-round-layers final-state
  unfolding trace-fri-header-tied-same-layer-auth-gap-def
  by blast
have recorded-gap:
  generic-fri-sampled-same-layer-recorded-auth-gap trace-roots trace-bs
  fri-query-idxs trace-round-layers final-state
  by (rule generic-fri-sampled-same-layer-auth-gap-imp-recorded-auth-gap
    [OF same no-auth])
show ?thesis
  unfolding trace-fri-header-tied-same-layer-recorded-auth-gap-def
  by (intro exI conjI)

```


(rule out-eq, rule fri-openings, rule header, rule partial, rule cand,
rule not-low, rule recorded-gap)

qed

lemma *trace-fri-header-tied-same-layer-recorded-auth-gap-imp-authenticated-recorded-auth-gap*:

assumes *trace-fri-header-tied-same-layer-recorded-auth-gap s out*

shows

trace-fri-header-tied-same-layer-authenticated-recorded-auth-gap s out

proof –

from *assms* **obtain** *trace-roots trace-bs trace-final dg composition-roots*

composition-bs composition-final fri-query-idxs trace-round-layers

composition-round-layers fr as rest trace-openings trace-table

result final-state

where *out-eq*: *out* = *Some (result, final-state)*

and *fri-openings*:

accepted-fri-opening-transcript s out trace-roots trace-bs

trace-final dg composition-roots composition-bs composition-final

fri-query-idxs trace-round-layers composition-round-layers

and *header*:

verifier-header-transcript s fr trace-roots trace-final as dg

composition-roots composition-final rest

and *partial*:

accepted-with-partial-trace-openings s out fr fri-query-idxs

trace-openings

and *cand*: *partial-trace-table-candidate trace-table trace-openings*

and *not-low*: $\neg$ *trace-table-low-degree trace-table*

and *recorded-gap*:

generic-fri-sampled-same-layer-recorded-auth-gap trace-roots

trace-bs fri-query-idxs trace-round-layers final-state

unfolding *trace-fri-header-tied-same-layer-recorded-auth-gap-def*

by *blast*

have *len-trace-bs*: *length trace-bs* = *length trace-roots*

using *accepted-fri-opening-transcript-shapes(2)[OF fri-openings]* .

have *len-query*: *length fri-query-idxs* = *rounds*

using *accepted-fri-opening-transcript-shapes(7)[OF fri-openings]* .

have *auth-step*:

$\bigwedge$ *round-idx layer-idx*.

round-idx < *length fri-query-idxs* $\implies$

layer-idx < *length trace-bs* $\implies$

generic-fri-transcript-step-with-authenticated-chunk trace-roots

trace-bs fri-query-idxs trace-round-layers final-state round-idx

layer-idx

proof –

fix *round-idx layer-idx*

assume *round-bound*: *round-idx* < *length fri-query-idxs*

and *layer-bound-bs*: *layer-idx* < *length trace-bs*

have *round-bound-rounds*: *round-idx* < *rounds*

using *round-bound len-query* **by** *simp*

have *layer-bound-roots*: *layer-idx* < *length trace-roots*

```

    using layer-bound-bs len-trace-bs by simp
  show
    generic-fri-transcript-step-with-authenticated-chunk trace-roots
      trace-bs fri-query-idxs trace-round-layers final-state round-idx
      layer-idx
  by (rule
    accepted-fri-opening-transcript-trace-step-with-authenticated-chunk-at-raw
    [OF fri-openings out-eq round-bound-rounds layer-bound-roots])
qed
have authenticated-recorded-gap:
  generic-fri-sampled-same-layer-authenticated-recorded-auth-gap
    trace-roots trace-bs fri-query-idxs trace-round-layers final-state
  by (rule
    generic-fri-recorded-auth-gap-imp-authenticated-recorded-auth-gap
    [OF recorded-gap auth-step])
show ?thesis
unfolding
  trace-fri-header-tied-same-layer-authenticated-recorded-auth-gap-def
  by (intro exI conjI)
    (rule out-eq, rule fri-openings, rule header, rule partial, rule cand,
     rule not-low, rule authenticated-recorded-gap)
qed

lemma trace-fri-header-tied-same-layer-authenticated-recorded-auth-gap-imp-slot-recorded-chunk-gap:
  assumes
    trace-fri-header-tied-same-layer-authenticated-recorded-auth-gap s out
  shows trace-fri-header-tied-authenticated-slot-recorded-chunk-gap s out
proof -
  from assms obtain trace-roots trace-bs trace-final dg composition-roots
    composition-bs composition-final fri-query-idxs trace-round-layers
    composition-round-layers fr as rest trace-openings trace-table
    result final-state
  where out-eq: out = Some (result, final-state)
  and fri-openings:
    accepted-fri-opening-transcript s out trace-roots trace-bs
      trace-final dg composition-roots composition-bs composition-final
      fri-query-idxs trace-round-layers composition-round-layers
  and header:
    verifier-header-transcript s fr trace-roots trace-final as dg
      composition-roots composition-final rest
  and partial:
    accepted-with-partial-trace-openings s out fr fri-query-idxs
      trace-openings
  and cand: partial-trace-table-candidate trace-table trace-openings
  and not-low:  $\neg$  trace-table-low-degree trace-table
  and gap:
    generic-fri-sampled-same-layer-authenticated-recorded-auth-gap
      trace-roots trace-bs fri-query-idxs trace-round-layers final-state
  unfolding

```

```

    trace-fri-header-tied-same-layer-authenticated-recorded-auth-gap-def
  by blast
have slot-gap:
  generic-fri-authenticated-slot-recorded-chunk-gap trace-roots
  fri-query-idxs trace-round-layers final-state
by (rule
  generic-fri-authenticated-recorded-auth-gap-imp-slot-recorded-chunk-gap
  [OF gap])
show ?thesis
  unfolding trace-fri-header-tied-authenticated-slot-recorded-chunk-gap-def
  by (intro exI conjI)
    (rule out-eq, rule fri-openings, rule header, rule partial, rule cand,
     rule not-low, rule slot-gap)
qed

lemma wp-trace-fri-header-tied-sampled-assignment-conflict-bound-from-authenticated-and-auth-gap:
  fixes C G :: prob
  assumes authenticated-bound:
    wp-event verify-monad
      (trace-fri-header-tied-sampled-assignment-conflict-with-authenticated-same-layer s)
    s ≤ C
  and auth-gap-bound:
    wp-event verify-monad
      (trace-fri-header-tied-assignment-auth-gap s) s ≤ G
  shows
    wp-event verify-monad
      (trace-fri-header-tied-sampled-assignment-conflict s) s ≤ C + G
proof -
  have wp-event verify-monad
    (trace-fri-header-tied-sampled-assignment-conflict s) s ≤
    wp-event verify-monad
      (λout.
        trace-fri-header-tied-sampled-assignment-conflict-with-authenticated-same-layer
          s out ∨
          trace-fri-header-tied-assignment-auth-gap s out) s
  by (rule wp-event-mono)
    (rule trace-fri-header-tied-assignment-imp-authenticated-or-auth-gap)
  also have ... ≤
    wp-event verify-monad
      (trace-fri-header-tied-sampled-assignment-conflict-with-authenticated-same-layer s)
    s +
    wp-event verify-monad
      (trace-fri-header-tied-assignment-auth-gap s) s
  by (rule wp-event-union-bound)
  also have ... ≤ C + G
  by (intro add-mono authenticated-bound auth-gap-bound)
  finally show ?thesis .
qed

```

lemma *wp-trace-fri-header-tied-assignment-auth-gap-bound-from-same-layer-auth-gap:*
fixes $G :: \text{prob}$
assumes *gap-bound:*
 wp-event verify-monad
 $(\text{trace-fri-header-tied-same-layer-auth-gap } s) \leq G$
shows
 wp-event verify-monad
 $(\text{trace-fri-header-tied-assignment-auth-gap } s) \leq G$
proof –
 have *wp-event verify-monad*
 $(\text{trace-fri-header-tied-assignment-auth-gap } s) \leq$
 wp-event verify-monad
 $(\text{trace-fri-header-tied-same-layer-auth-gap } s) \leq$
 by (rule *wp-event-mono*)
 (rule *trace-fri-header-tied-assignment-auth-gap-imp-same-layer-auth-gap*)
then show *?thesis*
 by (rule *order-trans[OF - gap-bound]*)
qed

lemma *wp-trace-fri-header-tied-same-layer-auth-gap-bound-from-recorded-auth-gap:*
fixes $G :: \text{prob}$
assumes *gap-bound:*
 wp-event verify-monad
 $(\text{trace-fri-header-tied-same-layer-recorded-auth-gap } s) \leq G$
shows
 wp-event verify-monad
 $(\text{trace-fri-header-tied-same-layer-auth-gap } s) \leq G$
proof –
 have *wp-event verify-monad*
 $(\text{trace-fri-header-tied-same-layer-auth-gap } s) \leq$
 wp-event verify-monad
 $(\text{trace-fri-header-tied-same-layer-recorded-auth-gap } s) \leq$
 by (rule *wp-event-mono*)
 (rule *trace-fri-header-tied-same-layer-auth-gap-imp-recorded-auth-gap*)
then show *?thesis*
 by (rule *order-trans[OF - gap-bound]*)
qed

lemma *wp-trace-fri-header-tied-same-layer-recorded-auth-gap-bound-from-authenticated-recorded-auth-gap:*
fixes $G :: \text{prob}$
assumes *gap-bound:*
 wp-event verify-monad
 $(\text{trace-fri-header-tied-same-layer-authenticated-recorded-auth-gap } s)$
 $s \leq G$
shows
 wp-event verify-monad
 $(\text{trace-fri-header-tied-same-layer-recorded-auth-gap } s) \leq G$
proof –
 have *wp-event verify-monad*

```

    (trace-fri-header-tied-same-layer-recorded-auth-gap s) s ≤
    wp-event verify-monad
    (trace-fri-header-tied-same-layer-authenticated-recorded-auth-gap s)
    s
  by (rule wp-event-mono)
    (rule
      trace-fri-header-tied-same-layer-recorded-auth-gap-imp-authenticated-recorded-auth-gap)
  then show ?thesis
    by (rule order-trans[OF - gap-bound])
qed

lemma wp-trace-fri-header-tied-same-layer-authenticated-recorded-auth-gap-bound-from-slot-recorded-chunk-gap:
  fixes G :: prob
  assumes gap-bound:
    wp-event verify-monad
    (trace-fri-header-tied-authenticated-slot-recorded-chunk-gap s) s
    ≤ G
  shows
    wp-event verify-monad
    (trace-fri-header-tied-same-layer-authenticated-recorded-auth-gap s)
    s ≤ G
proof -
  have wp-event verify-monad
    (trace-fri-header-tied-same-layer-authenticated-recorded-auth-gap s)
    s ≤
    wp-event verify-monad
    (trace-fri-header-tied-authenticated-slot-recorded-chunk-gap s) s
  by (rule wp-event-mono)
    (rule
      trace-fri-header-tied-same-layer-authenticated-recorded-auth-gap-imp-slot-recorded-chunk-gap)
  then show ?thesis
    by (rule order-trans[OF - gap-bound])
qed

lemma wp-trace-fri-header-tied-sampled-assignment-conflict-bound-from-auth-gap-and-residuals:
  fixes A P H G N S F :: prob
  assumes auth-gap-bound:
    wp-event verify-monad
    (trace-fri-header-tied-assignment-auth-gap s) s ≤ A
  and partial-merkle-bound:
    wp-event verify-monad (partial-merkle-inconsistency-bad s) s ≤ P
  and structural-gap-bound:
    wp-event verify-monad
    (trace-fri-header-tied-sibling-mismatch-structural-gap s) s ≤ H
  and alignment-gap-bound:
    wp-event verify-monad
    (trace-fri-header-tied-base-value-alignment-gap s) s ≤ G
  and next-gap-bound:
    wp-event verify-monad

```

(trace-fri-header-tied-next-value-replay-gap s) $s \leq N$
and successor-gap-bound:
 wp-event verify-monad
 (trace-fri-header-tied-successor-replay-gap s) $s \leq S$
and final-gap-bound:
 wp-event verify-monad
 (trace-fri-header-tied-final-value-replay-gap s) $s \leq F$
shows
 wp-event verify-monad
 (trace-fri-header-tied-sampled-assignment-conflict s) s
 $\leq (((((P + H) + G) + N + S + F) + P) + A$
by (rule
 wp-trace-fri-header-tied-sampled-assignment-conflict-bound-from-authenticated-and-auth-gap)
 (rule
 wp-trace-fri-header-tied-authenticated-conflict-bound-from-partial-merkle-structural-and-replay-gaps
 [OF partial-merkle-bound structural-gap-bound alignment-gap-bound
 next-gap-bound successor-gap-bound final-gap-bound],
 rule auth-gap-bound)

lemma wp-trace-fri-header-tied-sampled-assignment-conflict-bound-from-same-layer-auth-gap-and-residuals:

fixes $A P H G N S F :: \text{prob}$

assumes same-layer-gap-bound:

wp-event verify-monad

(trace-fri-header-tied-same-layer-auth-gap s) $s \leq A$

and partial-merkle-bound:

wp-event verify-monad (partial-merkle-inconsistency-bad s) $s \leq P$

and structural-gap-bound:

wp-event verify-monad

(trace-fri-header-tied-sibling-mismatch-structural-gap s) $s \leq H$

and alignment-gap-bound:

wp-event verify-monad

(trace-fri-header-tied-base-value-alignment-gap s) $s \leq G$

and next-gap-bound:

wp-event verify-monad

(trace-fri-header-tied-next-value-replay-gap s) $s \leq N$

and successor-gap-bound:

wp-event verify-monad

(trace-fri-header-tied-successor-replay-gap s) $s \leq S$

and final-gap-bound:

wp-event verify-monad

(trace-fri-header-tied-final-value-replay-gap s) $s \leq F$

shows

wp-event verify-monad

(trace-fri-header-tied-sampled-assignment-conflict s) s

$\leq (((((P + H) + G) + N + S + F) + P) + A$

by (rule

wp-trace-fri-header-tied-sampled-assignment-conflict-bound-from-auth-gap-and-residuals)

(rule

wp-trace-fri-header-tied-assignment-auth-gap-bound-from-same-layer-auth-gap

[OF same-layer-gap-bound],
 rule partial-merkle-bound,
 rule structural-gap-bound,
 rule alignment-gap-bound,
 rule next-gap-bound,
 rule successor-gap-bound,
 rule final-gap-bound)

lemma *wp-trace-fri-header-tied-sampled-assignment-conflict-bound-from-recorded-auth-gap-and-residuals:*

fixes $A\ P\ H\ G\ N\ S\ F :: \text{prob}$

assumes *recorded-gap-bound:*

wp-event verify-monad

(*trace-fri-header-tied-same-layer-recorded-auth-gap* s) $s \leq A$

and *partial-merkle-bound:*

wp-event verify-monad (*partial-merkle-inconsistency-bad* s) $s \leq P$

and *structural-gap-bound:*

wp-event verify-monad

(*trace-fri-header-tied-sibling-mismatch-structural-gap* s) $s \leq H$

and *alignment-gap-bound:*

wp-event verify-monad

(*trace-fri-header-tied-base-value-alignment-gap* s) $s \leq G$

and *next-gap-bound:*

wp-event verify-monad

(*trace-fri-header-tied-next-value-replay-gap* s) $s \leq N$

and *successor-gap-bound:*

wp-event verify-monad

(*trace-fri-header-tied-successor-replay-gap* s) $s \leq S$

and *final-gap-bound:*

wp-event verify-monad

(*trace-fri-header-tied-final-value-replay-gap* s) $s \leq F$

shows

wp-event verify-monad

(*trace-fri-header-tied-sampled-assignment-conflict* s) s

$\leq (((((P + H) + G) + N + S + F) + P) + A$

by (rule

wp-trace-fri-header-tied-sampled-assignment-conflict-bound-from-same-layer-auth-gap-and-residuals)

(rule

wp-trace-fri-header-tied-same-layer-auth-gap-bound-from-recorded-auth-gap

[OF *recorded-gap-bound*],

rule *partial-merkle-bound*,

rule *structural-gap-bound*,

rule *alignment-gap-bound*,

rule *next-gap-bound*,

rule *successor-gap-bound*,

rule *final-gap-bound*)

lemma *wp-trace-fri-header-tied-sampled-assignment-conflict-bound-from-authenticated-recorded-auth-gap-and-residuals:*

fixes $A\ P\ H\ G\ N\ S\ F :: \text{prob}$

assumes *gap-bound:*

wp-event verify-monad
 (trace-fri-header-tied-same-layer-authenticated-recorded-auth-gap s)
 $s \leq A$
and *partial-merkle-bound*:
wp-event verify-monad (partial-merkle-inconsistency-bad s) $s \leq P$
and *structural-gap-bound*:
wp-event verify-monad
 (trace-fri-header-tied-sibling-mismatch-structural-gap s) $s \leq H$
and *alignment-gap-bound*:
wp-event verify-monad
 (trace-fri-header-tied-base-value-alignment-gap s) $s \leq G$
and *next-gap-bound*:
wp-event verify-monad
 (trace-fri-header-tied-next-value-replay-gap s) $s \leq N$
and *successor-gap-bound*:
wp-event verify-monad
 (trace-fri-header-tied-successor-replay-gap s) $s \leq S$
and *final-gap-bound*:
wp-event verify-monad
 (trace-fri-header-tied-final-value-replay-gap s) $s \leq F$
shows
wp-event verify-monad
 (trace-fri-header-tied-sampled-assignment-conflict s) s
 $\leq (((((P + H) + G) + N + S + F) + P) + A$
by (rule
wp-trace-fri-header-tied-sampled-assignment-conflict-bound-from-recorded-auth-gap-and-residuals)
 (rule
wp-trace-fri-header-tied-same-layer-recorded-auth-gap-bound-from-authenticated-recorded-auth-gap
 [OF gap-bound],
 rule *partial-merkle-bound*,
 rule *structural-gap-bound*,
 rule *alignment-gap-bound*,
 rule *next-gap-bound*,
 rule *successor-gap-bound*,
 rule *final-gap-bound*)

lemma *wp-trace-fri-header-tied-sampled-assignment-conflict-bound-from-slot-recorded-chunk-gap-and-residuals*:

fixes $A P H G N S F :: \text{prob}$

assumes *gap-bound*:

wp-event verify-monad
 (trace-fri-header-tied-authenticated-slot-recorded-chunk-gap s) s
 $\leq A$
and *partial-merkle-bound*:
wp-event verify-monad (partial-merkle-inconsistency-bad s) $s \leq P$
and *structural-gap-bound*:
wp-event verify-monad
 (trace-fri-header-tied-sibling-mismatch-structural-gap s) $s \leq H$
and *alignment-gap-bound*:
wp-event verify-monad

(trace-fri-header-tied-base-value-alignment-gap s) $s \leq G$
and next-gap-bound:
 wp-event verify-monad
 (trace-fri-header-tied-next-value-replay-gap s) $s \leq N$
and successor-gap-bound:
 wp-event verify-monad
 (trace-fri-header-tied-successor-replay-gap s) $s \leq S$
and final-gap-bound:
 wp-event verify-monad
 (trace-fri-header-tied-final-value-replay-gap s) $s \leq F$
shows
 wp-event verify-monad
 (trace-fri-header-tied-sampled-assignment-conflict s) s
 $\leq (((((P + H) + G) + N + S + F) + P) + A$
by (rule
 wp-trace-fri-header-tied-sampled-assignment-conflict-bound-from-authenticated-recorded-auth-gap-and-residuals)
 (rule
 wp-trace-fri-header-tied-same-layer-authenticated-recorded-auth-gap-bound-from-slot-recorded-chunk-gap
 [OF gap-bound],
 rule partial-merkle-bound,
 rule structural-gap-bound,
 rule alignment-gap-bound,
 rule next-gap-bound,
 rule successor-gap-bound,
 rule final-gap-bound)

lemma wp-trace-fri-header-tied-bound-from-sampled-zero-auth-gap-and-residuals:

fixes $R\ Z\ A\ P\ H\ G\ N\ S\ F :: \text{prob}$

assumes sampled-bound:

wp-event verify-monad
 (trace-fri-bad-with-header-tied-sampled-layer-chain s) $s \leq R$

and zero-bound:

wp-event verify-monad
 (trace-fri-header-tied-zero-round-final-obstruction s) $s \leq Z$

and auth-gap-bound:

wp-event verify-monad
 (trace-fri-header-tied-assignment-auth-gap s) $s \leq A$

and partial-merkle-bound:

wp-event verify-monad (partial-merkle-inconsistency-bad s) $s \leq P$

and structural-gap-bound:

wp-event verify-monad
 (trace-fri-header-tied-sibling-mismatch-structural-gap s) $s \leq H$

and alignment-gap-bound:

wp-event verify-monad
 (trace-fri-header-tied-base-value-alignment-gap s) $s \leq G$

and next-gap-bound:

wp-event verify-monad
 (trace-fri-header-tied-next-value-replay-gap s) $s \leq N$

and successor-gap-bound:

$wp\text{-event verify-monad}$
 $(\text{trace-fri-header-tied-successor-replay-gap } s) \ s \leq S$
and final-gap-bound :
 $wp\text{-event verify-monad}$
 $(\text{trace-fri-header-tied-final-value-replay-gap } s) \ s \leq F$
shows
 $wp\text{-event verify-monad}$
 $(\text{trace-fri-bad-with-header-tied-partial-candidate } s) \ s$
 $\leq R + (((((P + H) + G) + N + S + F) + P) + A + Z)$
proof –
have conflict-bound :
 $wp\text{-event verify-monad}$
 $(\text{trace-fri-header-tied-sampled-assignment-conflict } s) \ s$
 $\leq (((((P + H) + G) + N + S + F) + P) + A$
by (rule
 $wp\text{-trace-fri-header-tied-sampled-assignment-conflict-bound-from-auth-gap-and-residuals})$
(rule auth-gap-bound ,
rule $\text{partial-merkle-bound}$,
rule $\text{structural-gap-bound}$,
rule $\text{alignment-gap-bound}$,
rule next-gap-bound ,
rule $\text{successor-gap-bound}$,
rule final-gap-bound)
have $wp\text{-event verify-monad}$
 $(\text{trace-fri-bad-with-header-tied-partial-candidate } s) \ s$
 $\leq R + (((((P + H) + G) + N + S + F) + P) + A + Z)$
by (rule order-trans[
 $OF \ wp\text{-trace-fri-header-tied-bound-from-sampled-conflict-and-zero}$
 $[OF \ \text{sampled-bound} \ \text{conflict-bound} \ \text{zero-bound}]])$
 $(\text{simp add: algebra-simps})$
then show $?thesis$.
qed

lemma $wp\text{-trace-fri-header-tied-bound-from-sampled-zero-same-layer-auth-gap-and-residuals}$:
fixes $R \ Z \ A \ P \ H \ G \ N \ S \ F :: \text{prob}$
assumes sampled-bound :
 $wp\text{-event verify-monad}$
 $(\text{trace-fri-bad-with-header-tied-sampled-layer-chain } s) \ s \leq R$
and zero-bound :
 $wp\text{-event verify-monad}$
 $(\text{trace-fri-header-tied-zero-round-final-obstruction } s) \ s \leq Z$
and $\text{same-layer-gap-bound}$:
 $wp\text{-event verify-monad}$
 $(\text{trace-fri-header-tied-same-layer-auth-gap } s) \ s \leq A$
and $\text{partial-merkle-bound}$:
 $wp\text{-event verify-monad} \ (\text{partial-merkle-inconsistency-bad } s) \ s \leq P$
and $\text{structural-gap-bound}$:
 $wp\text{-event verify-monad}$
 $(\text{trace-fri-header-tied-sibling-mismatch-structural-gap } s) \ s \leq H$

and *alignment-gap-bound*:
 wp-event verify-monad
 (*trace-fri-header-tied-base-value-alignment-gap s*) $s \leq G$
and *next-gap-bound*:
 wp-event verify-monad
 (*trace-fri-header-tied-next-value-replay-gap s*) $s \leq N$
and *successor-gap-bound*:
 wp-event verify-monad
 (*trace-fri-header-tied-successor-replay-gap s*) $s \leq S$
and *final-gap-bound*:
 wp-event verify-monad
 (*trace-fri-header-tied-final-value-replay-gap s*) $s \leq F$
shows
 wp-event verify-monad
 (*trace-fri-bad-with-header-tied-partial-candidate s*) s
 $\leq R + (((((P + H) + G) + N + S + F) + P) + A + Z)$
by (*rule*
 wp-trace-fri-header-tied-bound-from-sampled-zero-auth-gap-and-residuals)
 (*rule sampled-bound*,
 rule zero-bound,
 rule
 wp-trace-fri-header-tied-assignment-auth-gap-bound-from-same-layer-auth-gap
 [*OF same-layer-gap-bound*],
 rule partial-merkle-bound,
 rule structural-gap-bound,
 rule alignment-gap-bound,
 rule next-gap-bound,
 rule successor-gap-bound,
 rule final-gap-bound)

lemma *wp-trace-fri-header-tied-bound-from-sampled-zero-recorded-auth-gap-and-residuals*:

fixes $R\ Z\ A\ P\ H\ G\ N\ S\ F :: \text{prob}$

assumes *sampled-bound*:

wp-event verify-monad
 (*trace-fri-bad-with-header-tied-sampled-layer-chain s*) $s \leq R$

and *zero-bound*:

wp-event verify-monad
 (*trace-fri-header-tied-zero-round-final-obstruction s*) $s \leq Z$

and *recorded-gap-bound*:

wp-event verify-monad
 (*trace-fri-header-tied-same-layer-recorded-auth-gap s*) $s \leq A$

and *partial-merkle-bound*:

wp-event verify-monad (*partial-merkle-inconsistency-bad s*) $s \leq P$

and *structural-gap-bound*:

wp-event verify-monad
 (*trace-fri-header-tied-sibling-mismatch-structural-gap s*) $s \leq H$

and *alignment-gap-bound*:

wp-event verify-monad
 (*trace-fri-header-tied-base-value-alignment-gap s*) $s \leq G$

and *next-gap-bound*:
 wp-event verify-monad
 (*trace-fri-header-tied-next-value-replay-gap s*) $s \leq N$
and *successor-gap-bound*:
 wp-event verify-monad
 (*trace-fri-header-tied-successor-replay-gap s*) $s \leq S$
and *final-gap-bound*:
 wp-event verify-monad
 (*trace-fri-header-tied-final-value-replay-gap s*) $s \leq F$
shows
 wp-event verify-monad
 (*trace-fri-bad-with-header-tied-partial-candidate s*) s
 $\leq R + (((((P + H) + G) + N + S + F) + P) + A + Z)$
by (*rule*
 wp-trace-fri-header-tied-bound-from-sampled-zero-same-layer-auth-gap-and-residuals)
 (*rule sampled-bound*,
 rule zero-bound,
 rule
 wp-trace-fri-header-tied-same-layer-auth-gap-bound-from-recorded-auth-gap
 [*OF recorded-gap-bound*],
 rule partial-merkle-bound,
 rule structural-gap-bound,
 rule alignment-gap-bound,
 rule next-gap-bound,
 rule successor-gap-bound,
 rule final-gap-bound)

lemma *wp-trace-fri-header-tied-bound-from-sampled-zero-authenticated-recorded-auth-gap-and-residuals*:

fixes $R\ Z\ A\ P\ H\ G\ N\ S\ F :: \text{prob}$
assumes *sampled-bound*:
 wp-event verify-monad
 (*trace-fri-bad-with-header-tied-sampled-layer-chain s*) $s \leq R$
and *zero-bound*:
 wp-event verify-monad
 (*trace-fri-header-tied-zero-round-final-obstruction s*) $s \leq Z$
and *gap-bound*:
 wp-event verify-monad
 (*trace-fri-header-tied-same-layer-authenticated-recorded-auth-gap s*)
 $s \leq A$
and *partial-merkle-bound*:
 wp-event verify-monad (*partial-merkle-inconsistency-bad s*) $s \leq P$
and *structural-gap-bound*:
 wp-event verify-monad
 (*trace-fri-header-tied-sibling-mismatch-structural-gap s*) $s \leq H$
and *alignment-gap-bound*:
 wp-event verify-monad
 (*trace-fri-header-tied-base-value-alignment-gap s*) $s \leq G$
and *next-gap-bound*:
 wp-event verify-monad

(trace-fri-header-tied-next-value-replay-gap s) $s \leq N$
and successor-gap-bound:
 wp-event verify-monad
 (trace-fri-header-tied-successor-replay-gap s) $s \leq S$
and final-gap-bound:
 wp-event verify-monad
 (trace-fri-header-tied-final-value-replay-gap s) $s \leq F$
shows
 wp-event verify-monad
 (trace-fri-bad-with-header-tied-partial-candidate s) s
 $\leq R + (((((P + H) + G) + N + S + F) + P) + A + Z)$
by (rule
 wp-trace-fri-header-tied-bound-from-sampled-zero-recorded-auth-gap-and-residuals)
 (rule sampled-bound,
 rule zero-bound,
 rule
 wp-trace-fri-header-tied-same-layer-recorded-auth-gap-bound-from-authenticated-recorded-auth-gap
 [OF gap-bound],
 rule partial-merkle-bound,
 rule structural-gap-bound,
 rule alignment-gap-bound,
 rule next-gap-bound,
 rule successor-gap-bound,
 rule final-gap-bound)

lemma wp-trace-fri-header-tied-bound-from-sampled-zero-slot-recorded-chunk-gap-and-residuals:

fixes $R\ Z\ A\ P\ H\ G\ N\ S\ F :: \text{prob}$

assumes sampled-bound:

wp-event verify-monad

(trace-fri-bad-with-header-tied-sampled-layer-chain s) $s \leq R$

and zero-bound:

wp-event verify-monad

(trace-fri-header-tied-zero-round-final-obstruction s) $s \leq Z$

and gap-bound:

wp-event verify-monad

(trace-fri-header-tied-authenticated-slot-recorded-chunk-gap s)

$s \leq A$

and partial-merkle-bound:

wp-event verify-monad (partial-merkle-inconsistency-bad s) $s \leq P$

and structural-gap-bound:

wp-event verify-monad

(trace-fri-header-tied-sibling-mismatch-structural-gap s) $s \leq H$

and alignment-gap-bound:

wp-event verify-monad

(trace-fri-header-tied-base-value-alignment-gap s) $s \leq G$

and next-gap-bound:

wp-event verify-monad

(trace-fri-header-tied-next-value-replay-gap s) $s \leq N$

and successor-gap-bound:

wp-event verify-monad
 (trace-fri-header-tied-successor-replay-gap s) $s \leq S$
and final-gap-bound:
wp-event verify-monad
 (trace-fri-header-tied-final-value-replay-gap s) $s \leq F$
shows
wp-event verify-monad
 (trace-fri-bad-with-header-tied-partial-candidate s) s
 $\leq R + (((((P + H) + G) + N + S + F) + P) + A + Z)$
by (rule
wp-trace-fri-header-tied-bound-from-sampled-zero-authenticated-recorded-auth-gap-and-residuals)
 (rule sampled-bound,
 rule zero-bound,
 rule
wp-trace-fri-header-tied-same-layer-authenticated-recorded-auth-gap-bound-from-slot-recorded-chunk-gap
 [OF gap-bound],
 rule partial-merkle-bound,
 rule structural-gap-bound,
 rule alignment-gap-bound,
 rule next-gap-bound,
 rule successor-gap-bound,
 rule final-gap-bound)

lemma *wp-trace-fri-header-tied-bound-from-sampled-zero-slot-and-authenticated-final-residuals:*

fixes $R\ Z\ A\ P\ H\ G\ N\ S\ F :: \text{prob}$

assumes sampled-bound:

wp-event verify-monad
 (trace-fri-bad-with-header-tied-sampled-layer-chain s) $s \leq R$

and zero-bound:

wp-event verify-monad
 (trace-fri-header-tied-zero-round-final-obstruction s) $s \leq Z$

and slot-gap-bound:

wp-event verify-monad
 (trace-fri-header-tied-authenticated-slot-recorded-chunk-gap s)
 $s \leq A$

and partial-merkle-bound:

wp-event verify-monad (partial-merkle-inconsistency-bad s) $s \leq P$

and structural-gap-bound:

wp-event verify-monad
 (trace-fri-header-tied-sibling-mismatch-structural-gap s) $s \leq H$

and alignment-gap-bound:

wp-event verify-monad
 (trace-fri-header-tied-base-value-alignment-gap s) $s \leq G$

and next-gap-bound:

wp-event verify-monad
 (trace-fri-header-tied-next-value-replay-gap s) $s \leq N$

and successor-gap-bound:

wp-event verify-monad
 (trace-fri-header-tied-successor-replay-gap s) $s \leq S$

and *authenticated-final-bound*:

$$\text{wp-event verify-monad } (\text{trace-fri-header-tied-final-value-authenticated-replay-gap } s) \ s \leq F$$
shows

$$\text{wp-event verify-monad } (\text{trace-fri-bad-with-header-tied-partial-candidate } s) \ s \leq R + (((((P + H) + G) + N + S + (P + A + F)) + P) + A + Z)$$
proof –
have *final-gap-bound*:

$$\text{wp-event verify-monad } (\text{trace-fri-header-tied-final-value-replay-gap } s) \ s \leq P + A + F$$
by (rule

$$\text{wp-trace-fri-header-tied-final-value-replay-gap-bound-from-merkle-slot-and-authenticated}$$
(rule *partial-merkle-bound*, rule *slot-gap-bound*,
rule *authenticated-final-bound*)
have *old-bound*:

$$\text{wp-event verify-monad } (\text{trace-fri-bad-with-header-tied-partial-candidate } s) \ s \leq R + (((((P + H) + G) + N + S + (P + A + F)) + P) + A + Z)$$
by (rule

$$\text{wp-trace-fri-header-tied-bound-from-sampled-zero-slot-recorded-chunk-gap-and-residuals}$$
(rule *sampling-bound*,
rule *zero-bound*,
rule *slot-gap-bound*,
rule *partial-merkle-bound*,
rule *structural-gap-bound*,
rule *alignment-gap-bound*,
rule *next-gap-bound*,
rule *successor-gap-bound*,
rule *final-gap-bound*)
then show *?thesis* .
qed

lemma *wp-trace-fri-header-tied-bound-from-sampled-zero-slot-and-selected-step-final-residuals*:
fixes $R \ Z \ A \ P \ H \ G \ N \ S \ F :: \text{prob}$
assumes *sampling-bound*:

$$\text{wp-event verify-monad } (\text{trace-fri-bad-with-header-tied-sampled-layer-chain } s) \ s \leq R$$
and *zero-bound*:

$$\text{wp-event verify-monad } (\text{trace-fri-header-tied-zero-round-final-obstruction } s) \ s \leq Z$$
and *slot-gap-bound*:

$$\text{wp-event verify-monad } (\text{trace-fri-header-tied-authenticated-slot-recorded-chunk-gap } s) \ s \leq A$$
and *partial-merkle-bound*:

$$\text{wp-event verify-monad } (\text{partial-merkle-inconsistency-bad } s) \ s \leq P$$
and *structural-gap-bound*:

wp-event verify-monad
 (trace-fri-header-tied-sibling-mismatch-structural-gap s) $s \leq H$
and alignment-gap-bound:
wp-event verify-monad
 (trace-fri-header-tied-base-value-alignment-gap s) $s \leq G$
and next-gap-bound:
wp-event verify-monad
 (trace-fri-header-tied-next-value-replay-gap s) $s \leq N$
and successor-gap-bound:
wp-event verify-monad
 (trace-fri-header-tied-successor-replay-gap s) $s \leq S$
and selected-step-bound:
wp-event verify-monad
 (trace-fri-header-tied-final-value-selected-step-missing-gap s) $s \leq F$
shows
wp-event verify-monad
 (trace-fri-bad-with-header-tied-partial-candidate s) s
 $\leq R + (((((P + H) + G) + N + S + (P + A + F)) + P) + A + Z)$
proof –
have final-gap-bound:
wp-event verify-monad
 (trace-fri-header-tied-final-value-replay-gap s) $s \leq P + A + F$
by (rule
 wp-trace-fri-header-tied-final-value-replay-gap-bound-from-merkle-slot-and-selected-step)
 (rule partial-merkle-bound, rule slot-gap-bound,
 rule selected-step-bound)
show ?thesis
by (rule
 wp-trace-fri-header-tied-bound-from-sampled-zero-slot-recorded-chunk-gap-and-residuals)
 (rule sampled-bound,
 rule zero-bound,
 rule slot-gap-bound,
 rule partial-merkle-bound,
 rule structural-gap-bound,
 rule alignment-gap-bound,
 rule next-gap-bound,
 rule successor-gap-bound,
 rule final-gap-bound)
qed

lemma trace-fri-reachable-partial-candidate-reduction-from-header-tied-residual-bounds:

fixes $R\ Z\ A\ P\ H\ G\ N\ S\ F\ T :: \text{prob}$

assumes sampled-bound:

wp-event verify-monad

(trace-fri-bad-with-header-tied-sampled-layer-chain s) $s \leq R$

and zero-bound:

wp-event verify-monad

(trace-fri-header-tied-zero-round-final-obstruction s) $s \leq Z$

and *slot-gap-bound*:
 $wp\text{-event verify-monad}$
 $(\text{trace-fri-header-tied-authenticated-slot-recorded-chunk-gap } s)$
 $s \leq A$
and *partial-merkle-bound*:
 $wp\text{-event verify-monad } (\text{partial-merkle-inconsistency-bad } s) \ s \leq P$
and *structural-gap-bound*:
 $wp\text{-event verify-monad}$
 $(\text{trace-fri-header-tied-sibling-mismatch-structural-gap } s) \ s \leq H$
and *alignment-gap-bound*:
 $wp\text{-event verify-monad}$
 $(\text{trace-fri-header-tied-base-value-alignment-gap } s) \ s \leq G$
and *next-gap-bound*:
 $wp\text{-event verify-monad}$
 $(\text{trace-fri-header-tied-next-value-replay-gap } s) \ s \leq N$
and *successor-gap-bound*:
 $wp\text{-event verify-monad}$
 $(\text{trace-fri-header-tied-successor-replay-gap } s) \ s \leq S$
and *final-gap-bound*:
 $wp\text{-event verify-monad}$
 $(\text{trace-fri-header-tied-final-value-replay-gap } s) \ s \leq F$
and *header-gap-bound*:
 $wp\text{-event verify-monad}$
 $(\text{trace-fri-reachable-header-tie-gap } s) \ s \leq T$
and *total-bound*:
 $R + (((((P + H) + G) + N + S + F) + P) + A + Z) + T$
 $\leq \text{trace-fri-error}$
shows *trace-fri-reachable-partial-candidate-reduction-assumption* s
proof –
have *header-bound*:
 $wp\text{-event verify-monad}$
 $(\text{trace-fri-bad-with-header-tied-partial-candidate } s) \ s$
 $\leq R + (((((P + H) + G) + N + S + F) + P) + A + Z)$
by (rule
 $wp\text{-trace-fri-header-tied-bound-from-sampled-zero-slot-recorded-chunk-gap-and-residuals}$
 $[OF \ \text{sampled-bound} \ \text{zero-bound} \ \text{slot-gap-bound} \ \text{partial-merkle-bound}$
 $\ \text{structural-gap-bound} \ \text{alignment-gap-bound} \ \text{next-gap-bound}$
 $\ \text{successor-gap-bound} \ \text{final-gap-bound}]$)
show *?thesis*
by (rule
 $\text{trace-fri-reachable-partial-candidate-reduction-from-header-tied-and-gap}$
 $[OF \ \text{header-bound} \ \text{header-gap-bound} \ \text{total-bound}]$)
qed

lemma *trace-fri-header-tied-reduction-from-residual-bounds*:
fixes $R \ Z \ A \ P \ H \ G \ N \ S \ F :: \text{prob}$
assumes *sampled-bound*:
 $wp\text{-event verify-monad}$
 $(\text{trace-fri-bad-with-header-tied-sampled-layer-chain } s) \ s \leq R$

```

and zero-bound:
  wp-event verify-monad
    (trace-fri-header-tied-zero-round-final-obstruction s)  $s \leq Z$ 
and slot-gap-bound:
  wp-event verify-monad
    (trace-fri-header-tied-authenticated-slot-recorded-chunk-gap s)
       $s \leq A$ 
and partial-merkle-bound:
  wp-event verify-monad (partial-merkle-inconsistency-bad s)  $s \leq P$ 
and structural-gap-bound:
  wp-event verify-monad
    (trace-fri-header-tied-sibling-mismatch-structural-gap s)  $s \leq H$ 
and alignment-gap-bound:
  wp-event verify-monad
    (trace-fri-header-tied-base-value-alignment-gap s)  $s \leq G$ 
and next-gap-bound:
  wp-event verify-monad
    (trace-fri-header-tied-next-value-replay-gap s)  $s \leq N$ 
and successor-gap-bound:
  wp-event verify-monad
    (trace-fri-header-tied-successor-replay-gap s)  $s \leq S$ 
and final-gap-bound:
  wp-event verify-monad
    (trace-fri-header-tied-final-value-replay-gap s)  $s \leq F$ 
and total-bound:
   $R + (((((P + H) + G) + N + S + F) + P) + A + Z)$ 
     $\leq \text{trace-fri-error}$ 
shows trace-fri-header-tied-reduction s
unfolding trace-fri-header-tied-reduction-def
by (rule order-trans[
  OF wp-trace-fri-header-tied-bound-from-sampled-zero-slot-recorded-chunk-gap-and-residuals
  [OF sampled-bound zero-bound slot-gap-bound partial-merkle-bound
    structural-gap-bound alignment-gap-bound next-gap-bound
    successor-gap-bound final-gap-bound] total-bound])

end

end

```

```

theory Soundness-FRI-Query-Openings
imports
  Soundness-FRI
  Soundness-Query-Transcript-Openings
begin

```

Bridge from the FRI opening transcript evidence to the authenticated trace openings for the exact verifier-derived query-index list.

```

context soundness

```

begin

lemma *accepted-fri-opening-transcript-trace-openings-for-query-indices:*

assumes *fri-openings:*

accepted-fri-opening-transcript s out trace-roots trace-bs
trace-final dg composition-roots composition-bs composition-final
query-idxs trace-round-layers composition-round-layers

and *header:*

verifier-header-transcript s fr trace-roots trace-final as dg
composition-roots composition-final rest

and *out-eq:* *out = Some (result, final-state)*

obtains *trace-openings* **where**

accepted-with-partial-trace-openings s out fr query-idxs trace-openings

proof –

let *?P =*

λi openings. map opening-index openings =
powers-scaled (query-idxs ! i) ∧
*partial-authenticated-table fr (scale * clength) openings final-state*

have *len-query-idxs:* *length query-idxs = rounds*

by (*rule accepted-fri-opening-transcript-shapes(7)[OF fri-openings]*)

from *fri-openings* **obtain** *result' final-state' fr' f-fl header-as fl*

query-state raw-idxs query-chunks

where *out-def:* *out = Some (result', final-state')*

and *trace-roots-eq:* *trace-roots = map snd f-fl*

and *composition-roots-eq:* *composition-roots = map snd fl*

and *header':*

verifier-header-transcript s fr' trace-roots trace-final header-as dg
composition-roots composition-final (PTranscript query-state)

and *query-out:*

Some (result', final-state') ∈

set-dist

(execute

(ntimes

(verifier-query-round-program fr' f-fl trace-final header-as fl
composition-final)

rounds)

query-state)

and *len-raw:* *length raw-idxs = rounds*

and *query-idxs-eq:*

query-idxs = map (λraw. index (to-nat raw)) raw-idxs

and *len-query-chunks:* *length query-chunks = rounds*

and *transcript-query:*

PTranscript query-state =

List.concat query-chunks @ PTranscript final-state'

and *query-chunk:*

∧i. i < rounds ⇒

verifier-query-round-chunk (query-idxs ! i)

trace-roots composition-roots (query-chunks ! i)

and *replay-lookup:*

```

 $\bigwedge i. i < \text{rounds} \implies$ 
  fmlookup (HashMap final-state')
    (QueryIndexChallenge (PQueryCounter query-state + i)
      (state-after-query-chunks
        (PState query-state) query-chunks i)) =
    Some (raw-idxs ! i)
unfolding accepted-fri-opening-transcript-def
by blast
have final-state-eq: final-state' = final-state
  using out-def out-eq by simp
have fr-eq: fr' = fr
  using verifier-header-transcript-unique[OF header' header] by simp
have ex-round:  $\bigwedge i. i < \text{rounds} \implies \exists \text{openings}. ?P\ i\ \text{openings}$ 
proof -
  fix i
  assume i-bound:  $i < \text{rounds}$ 
  show  $\exists \text{openings}. ?P\ i\ \text{openings}$ 
  proof (rule
    ntimes-verifier-query-rounds-selected-trace-openings-aligned-with-replay-at
    [OF query-out i-bound len-raw query-idxs-eq len-query-chunks])
  show PTranscript query-state =
    List.concat query-chunks @ PTranscript final-state'
    by (rule transcript-query)
  show  $\bigwedge j. j < \text{rounds} \implies$ 
    verifier-query-round-chunk (query-idxs ! j)
      (map snd f-fl) (map snd fl) (query-chunks ! j)
    using query-chunk trace-roots-eq composition-roots-eq by simp
  show  $\bigwedge j. j < \text{rounds} \implies$ 
    fmlookup (HashMap final-state')
      (QueryIndexChallenge (PQueryCounter query-state + j)
        (state-after-query-chunks
          (PState query-state) query-chunks j)) =
      Some (raw-idxs ! j)
    using replay-lookup by simp
  next
  fix openings
  assume
    map opening-index openings = powers-scaled (query-idxs ! i)
    partial-authenticated-table fr' (scale * clength) openings
    final-state'
  then show ?thesis
    using fr-eq final-state-eq by blast
  qed
qed
let ?w =  $\lambda i. \text{SOME openings}. ?P\ i\ \text{openings}$ 
let ?trace-openings = map ?w [0.. $\text{rounds}$ ]
have len-trace: length ?trace-openings = rounds
  by simp
have props:  $\bigwedge i. i < \text{rounds} \implies ?P\ i\ (?trace-openings\ !\ i)$ 

```

```

proof –
  fix i
  assume i-bound: i < rounds
  have ?P i (?w i)
    by (rule someI-ex[OF ex-round[OF i-bound]])
  then show ?P i (?trace-openings ! i)
    using i-bound by simp
qed
have partial:
  accepted-with-partial-trace-openings s out fr query-idxs
    ?trace-openings
  unfolding accepted-with-partial-trace-openings-def accepted-def
  by (intro conjI exI[of - result] exI[of - final-state])
    (use out-eq len-query-idxs len-trace props in auto)
show ?thesis
  by (rule that[OF partial])
qed

end

end

```

theory *Soundness-FRI-Trace-Head-Value*

imports

Soundness-FRI-Trace-Residuals

Soundness-FRI-Query-Openings

begin

Local replay facts for the trace base head-value residual. This theory keeps the exact selected trace-layer replay needed by the residual proof out of the already large trace residual layer.

context *soundness*

begin

lemma *verifier-query-round-program-trace-fri-head-layer-value-aligned-with-lookup*:

fixes *s t* :: (*f*, *a*) *protocol-channel-scheme*

assumes *f-fl-eq*: *f-fl* = (*b*, *rt*) # *f-fl-tail*

and *outcome*:

Some (*()*, *t*) ∈

set-dist

(*execute*

(*verifier-query-round-program* *fr* *f-fl* *f-final* *as* *fl* *final*) *s*)

obtains *raw idx openings query-chunk trace-layer-chunks*

composition-layer-chunks xp xp-path xn xn-path **where**

idx = *index* (*to-nat* *raw*)

query-round-fri-layer-transcripts *idx* (*map snd* *f-fl*) (*map snd* *fl*)

query-chunk trace-layer-chunks composition-layer-chunks

PTranscript *s* = *query-chunk* @ *PTranscript* *t*

```

PState t = foldl concat (PState s) query-chunk
s ≤ t
map opening-index openings = powers-scaled idx
partial-authenticated-table fr (scale * clength) openings t
openings ≠ []
opening-value (hd openings) = xp
fri-layer-opening-chunk (scale * clength) xp xp-path xn xn-path
(trace-layer-chunks ! 0)
fmlookup (HashMap t)
(QueryIndexChallenge (PQueryCounter s) (PState s)) = Some raw
proof –
from outcome[unfolded verifier-query-round-program-alt-def]
obtain raw s0 where raw:
  Some (raw, s0) ∈
    set-dist (execute receive-query-index-challenge s)
and tail:
  Some ((), t) ∈
    set-dist
      (execute
        (verifier-query-round-after-index-program
          fr f-fl f-final as fl final raw) s0)
by (auto elim!: set-dist-bindE)
obtain idx openings query-chunk trace-layer-chunks composition-layer-chunks
  xp xp-path xn xn-path
where idx-eq: idx = index (to-nat raw)
and layers:
  query-round-fri-layer-transcripts idx (map snd f-fl) (map snd fl)
  query-chunk trace-layer-chunks composition-layer-chunks
and tr-t: PTranscript s0 = query-chunk @ PTranscript t
and st-t: PState t = foldl concat (PState s0) query-chunk
and ext-s0-t: s0 ≤ t
and idxs: map opening-index openings = powers-scaled idx
and auth-table:
  partial-authenticated-table fr (scale * clength) openings t
and nonempty: openings ≠ []
and opening-value-eq: opening-value (hd openings) = xp
and head-chunk:
  fri-layer-opening-chunk (scale * clength) xp xp-path xn xn-path
  (trace-layer-chunks ! 0)
using verifier-query-round-after-index-trace-fri-head-layer-value-aligned
  [OF f-fl-eq tail]
by metis
have lookup-s0:
  fmlookup (HashMap s0)
  (QueryIndexChallenge (PQueryCounter s) (PState s)) = Some raw
using receive-query-index-challenge-outcome[OF raw] by simp
have lookup-t:
  fmlookup (HashMap t)
  (QueryIndexChallenge (PQueryCounter s) (PState s)) = Some raw

```

```

    by (rule hash-extension-lookup[OF lookup-s0 ext-s0-t])
  have tr-s0: PTranscript s = PTranscript s0
    using receive-query-index-challenge-outcome[OF raw] by simp
  have st-s0: PState s0 = PState s
    using receive-query-index-challenge-outcome[OF raw] by simp
  have ext-s-s0: s ≤ s0
    using receive-query-index-challenge-outcome[OF raw] by simp
  have ext-s-t: s ≤ t
    by (rule hash-ext-trans[OF ext-s-s0 ext-s0-t])
  have tr-goal: PTranscript s = query-chunk @ PTranscript t
    using tr-t tr-s0 by simp
  have st-goal: PState t = foldl concat (PState s) query-chunk
    using st-t st-s0 by simp
  show ?thesis
    by (rule that[OF idx-eq layers tr-goal st-goal ext-s-t idxs auth-table
        nonempty opening-value-eq head-chunk lookup-t])
qed

```

lemma *ntimes-verifier-query-rounds-selected-trace-fri-head-layer-value-aligned-with-replay-at:*

fixes *query-state final-state* :: ('f, 'a) *protocol-channel-scheme*

assumes *f-fl-eq*: *f-fl* = (*b*, *rt*) # *f-fl-tail*

and *outcome*:

Some (results, final-state) ∈

set-dist

(execute

(ntimes

(verifier-query-round-program fr f-fl f-final as fl final)

rounds)

query-state)

and *i-bound*: *i* < *rounds*

and *len-raw*: *length raw-idxs* = *rounds*

and *query-idxs-eq*:

query-idxs = *map (λraw. index (to-nat raw)) raw-idxs*

and *len-query-chunks*: *length query-chunks* = *rounds*

and *transcript-query*:

PTranscript query-state =

List.concat query-chunks @ PTranscript final-state

and *query-chunk*:

$\bigwedge i. i < rounds \implies$

verifier-query-round-chunk (query-idxs ! i)

(map snd f-fl) (map snd fl) (query-chunks ! i)

and *query-layers*:

query-round-fri-layer-transcripts (query-idxs ! i)

(map snd f-fl) (map snd fl) (query-chunks ! i)

trace-layer-chunks composition-layer-chunks

and *replay-lookup*:

$\bigwedge i. i < rounds \implies$

fmlookup (HashMap final-state)

(QueryIndexChallenge (PQueryCounter query-state + i))

```

      (state-after-query-chunks
       (PState query-state) query-chunks i)) =
    Some (raw-idxs ! i)
obtains openings xp xp-path xn xn-path where
  map opening-index openings = powers-scaled (query-idxs ! i)
  partial-authenticated-table fr (scale * clength) openings final-state
  openings ≠ []
  opening-value (hd openings) = xp
  fri-layer-opening-chunk (scale * clength) xp xp-path xn xn-path
    (trace-layer-chunks ! 0)
proof –
let ?round = verifier-query-round-program fr f-fl f-final as fl final
from query-opening-ntimes-outcome-decomp-at[OF outcome i-bound]
obtain prefix x suffix s-i s-suc where
  prefix:
    Some (prefix, s-i) ∈
      set-dist (execute (ntimes ?round i) query-state)
and round:
    Some (x, s-suc) ∈ set-dist (execute ?round s-i)
and suffix:
    Some (suffix, final-state) ∈
      set-dist (execute (ntimes ?round (rounds – Suc i)) s-suc)
by blast
have x-eq: x = ()
by (cases x) simp
show ?thesis
proof (rule query-opening-ntimes-prefix-transcript-state-alignment
  [OF prefix])
  fix prefix-query-idxs prefix-chunks
  assume len-prefix-idxs: length prefix-query-idxs = i
  and len-prefix-chunks: length prefix-chunks = i
  and transcript-prefix:
    PTranscript query-state =
      List.concat prefix-chunks @ PTranscript s-i
  and state-i:
    PState s-i =
      state-after-query-chunks (PState query-state) prefix-chunks i
  and counter-i:
    PQueryCounter s-i = PQueryCounter query-state + i
  and prefix-ext: query-state ≤ s-i
  and prefix-rounds:
     $\bigwedge k. k < i \implies$ 
      verifier-query-round-chunk (prefix-query-idxs ! k)
        (map snd f-fl) (map snd fl) (prefix-chunks ! k)
  show ?thesis
proof (rule
  verifier-query-round-program-trace-fri-head-layer-value-aligned-with-lookup
  [OF f-fl-eq round[unfolded x-eq]])
  fix raw idx chunk actual-trace-layers actual-composition-layers

```



```

    openings xp xp-path xn xn-path
assume idx-eq: idx = index (to-nat raw)
and actual-layers:
    query-round-fri-layer-transcripts idx (map snd f-fl)
    (map snd fl) chunk actual-trace-layers actual-composition-layers
and transcript-round: PTranscript s-i = chunk @ PTranscript s-suc
and state-suc: PState s-suc = foldl concat (PState s-i) chunk
and round-ext: s-i ≤ s-suc
and opening-indices: map opening-index openings = powers-scaled idx
and auth-table-suc:
    partial-authenticated-table fr (scale * clength) openings s-suc
and openings-nonempty: openings ≠ []
and opening-value-eq: opening-value (hd openings) = xp
and head-chunk:
    fri-layer-opening-chunk (scale * clength) xp xp-path xn
    xn-path (actual-trace-layers ! 0)
and lookup-suc:
    fmlookup (HashMap s-suc)
    (QueryIndexChallenge (PQueryCounter s-i) (PState s-i)) =
    Some raw
show ?thesis
proof (rule query-opening-ntimes-suffix-transcript-state-alignment
    [OF suffix])
fix suffix-query-idxs suffix-chunks
assume len-suffix-idxs:
    length suffix-query-idxs = rounds - Suc i
and len-suffix-chunks:
    length suffix-chunks = rounds - Suc i
and transcript-suffix:
    PTranscript s-suc =
    List.concat suffix-chunks @ PTranscript final-state
and suffix-state:
    PState final-state =
    state-after-query-chunks (PState s-suc) suffix-chunks
    (rounds - Suc i)
and suffix-counter:
    PQueryCounter final-state =
    PQueryCounter s-suc + (rounds - Suc i)
and suffix-ext: s-suc ≤ final-state
and suffix-rounds:
     $\bigwedge k. k < rounds - Suc i \implies$ 
    verifier-query-round-chunk (suffix-query-idxs ! k)
    (map snd f-fl) (map snd fl) (suffix-chunks ! k)
let ?selected-chunks = prefix-chunks @ chunk # suffix-chunks
have len-selected-chunks: length ?selected-chunks = rounds
    using len-prefix-chunks len-suffix-chunks i-bound by simp
have selected-chunk-lens:
     $\bigwedge k. k < rounds \implies$ 
    length (?selected-chunks ! k) =

```

```

      verifier-query-round-transcript-length (query-idxs ! k)
      (map snd f-fl) (map snd fl)
proof -
  fix k
  assume k-bound:  $k < \text{rounds}$ 
  show
    length (?selected-chunks ! k) =
      verifier-query-round-transcript-length (query-idxs ! k)
      (map snd f-fl) (map snd fl)
  proof (cases k < i)
  case True
  have nth-eq: ?selected-chunks ! k = prefix-chunks ! k
    using True len-prefix-chunks by (simp add: nth-append)
  have
    length (prefix-chunks ! k) =
      verifier-query-round-transcript-length
      (prefix-query-idxs ! k) (map snd f-fl) (map snd fl)
    by (rule verifier-query-round-chunk-length
      [OF prefix-rounds[OF True]])
  then show ?thesis
    using nth-eq
    by (simp add:
      verifier-query-round-transcript-length-index-irrelevant)
  next
  case False
  show ?thesis
  proof (cases k = i)
  case True
  have nth-eq: ?selected-chunks ! k = chunk
    using True len-prefix-chunks by (simp add: nth-append)
  have chunk-shape:
    verifier-query-round-chunk idx (map snd f-fl) (map snd fl)
    chunk
  unfolding verifier-query-round-chunk-def
  using actual-layers
  unfolding query-round-fri-layer-transcripts-def by blast
  have
    length chunk =
      verifier-query-round-transcript-length idx
      (map snd f-fl) (map snd fl)
    by (rule verifier-query-round-chunk-length[OF chunk-shape])
  then show ?thesis
    using nth-eq
    by (simp add:
      verifier-query-round-transcript-length-index-irrelevant)
  next
  case False
  let ?k = k - Suc i
  have ge: Suc i ≤ k

```

```

    using  $\langle \neg k < i \rangle$  False by linarith
  have k-bound':  $?k < \text{rounds} - \text{Suc } i$ 
    using ge k-bound by linarith
  have nth-eq:  $?selected\text{-chunks} ! k = \text{suffix-chunks} ! ?k$ 
    using ge len-prefix-chunks by (simp add: nth-append)
  have
    length (suffix-chunks ! ?k) =
      verifier-query-round-transcript-length
        (suffix-query-idxs ! ?k) (map snd f-fl) (map snd fl)
    by (rule verifier-query-round-chunk-length
        [OF suffix-rounds[OF k-bound']])
  then show ?thesis
    using nth-eq
    by (simp add:
        verifier-query-round-transcript-length-index-irrelevant)
qed
qed
qed
have query-chunk-lens:
   $\bigwedge k. k < \text{rounds} \implies$ 
    length (query-chunks ! k) =
      verifier-query-round-transcript-length (query-idxs ! k)
        (map snd f-fl) (map snd fl)
  using query-chunk verifier-query-round-chunk-length by blast
have transcript-all:
  PTranscript query-state =
    List.concat ?selected-chunks @ PTranscript final-state
  using transcript-prefix transcript-round transcript-suffix by simp
have concat-eq:
  List.concat ?selected-chunks = List.concat query-chunks
  using transcript-all transcript-query by simp
have chunks-eq:  $?selected\text{-chunks} = \text{query-chunks}$ 
proof -
  have same-lens:
     $\bigwedge k. k < \text{length } ?selected\text{-chunks} \implies$ 
      length (?selected-chunks ! k) = length (query-chunks ! k)
    using selected-chunk-lens query-chunk-lens len-selected-chunks
    by simp
  have concat-eq-empty:
    List.concat ?selected-chunks @ [] = List.concat query-chunks
    using concat-eq by simp
  have len-eq:  $\text{length } ?selected\text{-chunks} = \text{length query-chunks}$ 
    using len-selected-chunks len-query-chunks by simp
  have  $?selected\text{-chunks} = \text{query-chunks} \wedge ([\ ] :: 'f \text{ list}) = [\ ]$ 
    by (rule concat-append-eq-concat-same-chunk-lengths
        [OF len-eq same-lens concat-eq-empty])
  then show ?thesis
    by simp
qed

```

```

have prefix-state-eq:
  state-after-query-chunks (PState query-state) prefix-chunks i =
    state-after-query-chunks (PState query-state) query-chunks i
proof —
  have
    state-after-query-chunks (PState query-state)
      ?selected-chunks i =
        state-after-query-chunks (PState query-state)
          prefix-chunks i
    by (rule state-after-query-chunks-append-prefix
      [OF len-prefix-chunks])
    then show ?thesis
      using chunks-eq by simp
qed
have lookup-final:
  fmlookup (HashMap final-state)
    (QueryIndexChallenge (PQueryCounter query-state + i)
      (state-after-query-chunks
        (PState query-state) query-chunks i)) =
    Some raw
  using hash-extension-lookup [OF lookup-suc suffix-ext]
  unfolding counter-i state-i prefix-state-eq .
have raw-eq: raw = raw-idxs ! i
  using lookup-final replay-lookup [OF i-bound] by simp
have idx-eq-query: idx = query-idxs ! i
  using idx-eq raw-eq query-idxs-eq len-raw i-bound by simp
have auth-table-final:
  partial-authenticated-table fr (scale * clength) openings
    final-state
  by (rule partial-authenticated-table-mono
    [OF auth-table-suc suffix-ext])
have selected-nth: ?selected-chunks ! i = chunk
  using len-prefix-chunks by (simp add: nth-append)
have actual-layers-query:
  query-round-fri-layer-transcripts (query-idxs ! i)
    (map snd f-fl) (map snd fl) (query-chunks ! i)
    actual-trace-layers actual-composition-layers
  using actual-layers idx-eq-query chunks-eq selected-nth by simp
have actual-trace-layers-eq:
  actual-trace-layers = trace-layer-chunks
  by (rule query-round-fri-layer-transcripts-unique(1)
    [OF actual-layers-query query-layers])
show ?thesis
  by (rule that [OF - auth-table-final openings-nonempty
    opening-value-eq])
    (use opening-indices idx-eq-query head-chunk
      actual-trace-layers-eq in simp-all)
qed
qed

```

qed
qed

lemma *accepted-fri-opening-transcript-trace-head-layer-value-aligned-at:*
assumes *fri-openings:*
accepted-fri-opening-transcript s out trace-roots trace-bs
trace-final dg composition-roots composition-bs composition-final
query-idxs trace-round-layers composition-round-layers
and *header:*
verifier-header-transcript s fr trace-roots trace-final header-as dg
composition-roots composition-final header-rest
and *out-eq:* *out = Some (result, final-state)*
and *round-bound:* *round-idx < length query-idxs*
and *trace-bs-nonempty:* *0 < length trace-bs*
obtains *openings xp xp-path xn xn-path* **where**
map opening-index openings = powers-scaled (query-idxs ! round-idx)
*partial-authenticated-table fr (scale * clength) openings final-state*
openings ≠ []
opening-value (hd openings) = xp
*fri-layer-opening-chunk (scale * clength) xp xp-path xn xn-path*
(trace-round-layers ! round-idx ! 0)
proof –
from *fri-openings* **obtain** *result' final-state' fr' f-fl as fl query-state*
raw-idxs query-chunks
where *out-def:* *out = Some (result', final-state')*
and *trace-roots-eq:* *trace-roots = map snd f-fl*
and *trace-bs-eq:* *trace-bs = map fst f-fl*
and *composition-roots-eq:* *composition-roots = map snd fl*
and *header':*
verifier-header-transcript s fr' trace-roots trace-final as dg
composition-roots composition-final (PTranscript query-state)
and *query-out:*
Some (result', final-state') ∈
set-dist (execute
(ntimes
(verifier-query-round-program fr' f-fl trace-final as fl
composition-final)
rounds)
query-state)
and *len-raw:* *length raw-idxs = rounds*
and *query-idxs-eq:* *query-idxs = map (λraw. index (to-nat raw)) raw-idxs*
and *len-query-chunks:* *length query-chunks = rounds*
and *transcript-query:*
PTranscript query-state =
List.concat query-chunks @ PTranscript final-state'
and *query-chunk:*
 $\bigwedge i. i < rounds \implies$
verifier-query-round-chunk (query-idxs ! i)
trace-roots composition-roots (query-chunks ! i)

```

and query-layers:
   $\wedge i. i < \text{rounds} \implies$ 
    query-round-fri-layer-transcripts (query-idxs ! i)
      trace-roots composition-roots (query-chunks ! i)
        (trace-round-layers ! i) (composition-round-layers ! i)
and replay-lookup:
   $\wedge i. i < \text{rounds} \implies$ 
    fmlookup (HashMap final-state)
      (QueryIndexChallenge (PQueryCounter query-state + i)
        (state-after-query-chunks
          (PState query-state) query-chunks i)) =
      Some (raw-idxs ! i)
unfolding accepted-fri-opening-transcript-def by blast
have fr'-eq: fr' = fr
  using verifier-header-transcript-unique[OF header header'] by simp
have result'-eq: result' = result
  using out-def out-eq by simp
have final-state'-eq: final-state' = final-state
  using out-def out-eq by simp
have query-out':
  Some (result, final-state)  $\in$ 
    set-dist (execute
      (ntimes
        (verifier-query-round-program fr' f-fl trace-final as fl
          composition-final)
        rounds)
      query-state)
  using query-out result'-eq final-state'-eq by simp
have round-bound-rounds: round-idx < rounds
  using round-bound
    accepted-fri-opening-transcript-shapes( $\gamma$ )[OF fri-openings]
  by simp
obtain b rt f-fl-tail where f-fl-eq: f-fl = (b, rt) # f-fl-tail
proof (cases f-fl)
  case Nil
  then show ?thesis
    using trace-bs-nonempty trace-bs-eq by simp
next
  case (Cons head tail)
  then obtain b rt where head-eq: head = (b, rt)
    by (cases head)
  then show ?thesis
    using Cons that by simp
qed
have query-layer:
  query-round-fri-layer-transcripts (query-idxs ! round-idx)
    (map snd f-fl) (map snd fl) (query-chunks ! round-idx)
    (trace-round-layers ! round-idx)
    (composition-round-layers ! round-idx)

```

```

using query-layers[OF round-bound-rounds]
  trace-roots-eq composition-roots-eq by simp
have ex:
  ∃ openings xp xp-path xn xn-path.
    map opening-index openings = powers-scaled (query-idxs ! round-idx) ∧
    partial-authenticated-table fr' (scale * clength) openings final-state ∧
    openings ≠ [] ∧
    opening-value (hd openings) = xp ∧
    fri-layer-opening-chunk (scale * clength) xp xp-path xn xn-path
      (trace-round-layers ! round-idx ! 0)
proof (rule
  ntimes-verifier-query-rounds-selected-trace-fri-head-layer-value-aligned-with-replay-at
    [OF f-fl-eq query-out' round-bound-rounds len-raw query-idxs-eq
      len-query-chunks])
show PTranscript query-state =
  List.concat query-chunks @ PTranscript final-state
using transcript-query final-state'-eq by simp
show ∧i. i < rounds ⇒
  verifier-query-round-chunk (query-idxs ! i)
    (map snd f-fl) (map snd fl) (query-chunks ! i)
using query-chunk trace-roots-eq composition-roots-eq by simp
show query-round-fri-layer-transcripts (query-idxs ! round-idx)
  (map snd f-fl) (map snd fl) (query-chunks ! round-idx)
  (trace-round-layers ! round-idx)
  (composition-round-layers ! round-idx)
by (rule query-layer)
show ∧i. i < rounds ⇒
  fmlookup (HashMap final-state)
    (QueryIndexChallenge (PQueryCounter query-state + i)
      (state-after-query-chunks
        (PState query-state) query-chunks i)) =
    Some (raw-idxs ! i)
using replay-lookup final-state'-eq by simp
next
fix openings xp xp-path xn xn-path
assume map opening-index openings =
  powers-scaled (query-idxs ! round-idx)
and partial-authenticated-table fr' (scale * clength) openings
  final-state
and openings ≠ []
and opening-value (hd openings) = xp
and fri-layer-opening-chunk (scale * clength) xp xp-path xn xn-path
  (trace-round-layers ! round-idx ! 0)
then show ?thesis
by blast
qed
from ex show ?thesis
by (metis fr'-eq that)
qed

```

lemma *trace-fri-header-tied-base-head-value-neq-gap-imp-partial-merkle:*
assumes *trace-fri-header-tied-base-head-value-neq-gap s out*
shows *partial-merkle-inconsistency-bad s out*
proof –
from *assms* **obtain** *trace-roots trace-bs trace-final dg composition-roots*
composition-bs composition-final fri-query-idxs trace-round-layers
composition-round-layers fr as rest trace-openings trace-table
round-idx xp xp-path xn xn-path result final-state
where *out-eq: out = Some (result, final-state)*
and *fri-openings:*
accepted-fri-opening-transcript s out trace-roots trace-bs
trace-final dg composition-roots composition-bs composition-final
fri-query-idxs trace-round-layers composition-round-layers
and *header:*
verifier-header-transcript s fr trace-roots trace-final as dg
composition-roots composition-final rest
and *partial:*
accepted-with-partial-trace-openings s out fr fri-query-idxs
trace-openings
and *candidate: partial-trace-table-candidate trace-table trace-openings*
and *nonlow: $\neg$ trace-table-low-degree trace-table*
and *round-bound: round-idx < length fri-query-idxs*
and *trace-bs-nonempty: 0 < length trace-bs*
and *openings-nonempty: trace-openings ! round-idx $\neq$ []*
and *hd-idx:*
opening-index (hd (trace-openings ! round-idx)) =
fri-evidence-layer-idx trace-roots fri-query-idxs round-idx 0
and *hd-neq: opening-value (hd (trace-openings ! round-idx)) $\neq$ xp*
and *step:*
fri-layer-step-evidence
(trace-roots ! 0)
(trace-bs ! 0)
(fri-evidence-layer-len trace-roots 0)
(fri-evidence-layer-idx trace-roots fri-query-idxs round-idx 0)
1
(fri-sibling-index (fri-evidence-layer-len trace-roots 0)
(fri-evidence-layer-idx trace-roots fri-query-idxs round-idx 0))
xp xp-path xn xn-path
(fri-evidence-next-idx trace-roots fri-query-idxs round-idx 0)
(fri-evidence-next-value trace-roots trace-bs fri-query-idxs
round-idx 0 xp xn)
(trace-round-layers ! round-idx ! 0)
and *no-match:*
 $\neg$ fri-opening-matches-table (fri-evidence-layer-len trace-roots 0)
(fri-evidence-layer-idx trace-roots fri-query-idxs round-idx 0)
trace-table xp xn
unfolding *trace-fri-header-tied-base-head-value-neq-gap-def*
by *metis*


```

have round-bound-rounds: round-idx < rounds
  using round-bound
    accepted-fri-opening-transcript-shapes( $\gamma$ )[OF fri-openings]
  by simp
from accepted-fri-opening-transcript-trace-head-layer-value-aligned-at
  [OF fri-openings header out-eq round-bound trace-bs-nonempty]
obtain openings yp yp-path yn yn-path where actual-idxs:
  map opening-index openings = powers-scaled (fri-query-idxs ! round-idx)
and actual-auth:
  partial-authenticated-table fr (scale * clength) openings final-state
and actual-nonempty: openings  $\neq$  []
and actual-value: opening-value (hd openings) = yp
and actual-chunk:
  fri-layer-opening-chunk (scale * clength) yp yp-path yn yn-path
  (trace-round-layers ! round-idx ! 0)
by blast
have roots-nonempty: 0 < length trace-roots
  using accepted-fri-opening-transcript-shapes(2)[OF fri-openings]
  trace-bs-nonempty by simp
have len-eq:
  fri-evidence-layer-len trace-roots 0 = scale * clength
  unfolding fri-evidence-layer-len-def
  using roots-nonempty
  by (cases trace-roots) (simp-all add: mult.commute)
have step-chunk:
  fri-layer-opening-chunk (scale * clength) xp xp-path xn xn-path
  (trace-round-layers ! round-idx ! 0)
  using fri-layer-step-evidenceD(4)[OF step]
  unfolding len-eq .
have yp-eq: yp = xp
  by (rule fri-layer-opening-chunk-values-unique(1)
    [OF actual-chunk step-chunk])
have actual-head-idx:
  opening-index (hd openings) =
    fri-evidence-layer-idx trace-roots fri-query-idxs round-idx 0
proof -
  have opening-index (hd openings) = fri-query-idxs ! round-idx
proof -
  have opening-index (hd openings) =
    hd (map opening-index openings)
    using actual-nonempty by (cases openings) auto
  also have ... = hd (powers-scaled (fri-query-idxs ! round-idx))
    using actual-idxs by simp
  also have ... = fri-query-idxs ! round-idx
    by (rule powers-scaled-hd)
  finally show ?thesis .
qed
moreover have
  fri-evidence-layer-idx trace-roots fri-query-idxs round-idx 0 =

```

```

    fri-query-idxs ! round-idx
  using roots-nonempty
  unfolding fri-evidence-layer-idx-def
  by (cases trace-roots) simp-all
  ultimately show ?thesis by simp
qed
let ?opn = hd (trace-openings ! round-idx)
let ?opn' = hd openings
have trace-table:
  partial-authenticated-table fr (scale * clength)
    (trace-openings ! round-idx) final-state
  using partial round-bound-rounds out-eq
  unfolding accepted-with-partial-trace-openings-def by blast
have opn-auth: authenticated-opening-in final-state ?opn
  using trace-table openings-nonempty
  unfolding partial-authenticated-table-def by simp
have opn'-auth: authenticated-opening-in final-state ?opn'
  using actual-auth actual-nonempty
  unfolding partial-authenticated-table-def by simp
have same-root: opening-root ?opn = opening-root ?opn'
  using trace-table actual-auth openings-nonempty actual-nonempty
  unfolding partial-authenticated-table-def by auto
have same-length: opening-length ?opn = opening-length ?opn'
  using trace-table actual-auth openings-nonempty actual-nonempty
  unfolding partial-authenticated-table-def by auto
have same-index: opening-index ?opn = opening-index ?opn'
  using hd-idx actual-head-idx by simp
have diff-value: opening-value ?opn  $\neq$  opening-value ?opn'
  using hd-neq actual-value yp-eq by simp
have acc: accepted out
  unfolding accepted-def out-eq by simp
show ?thesis
  unfolding partial-merkle-inconsistency-bad-def
  by (intro conjI exI)
    (rule acc, rule out-eq, rule opn-auth, rule opn'-auth,
     rule same-root, rule same-length, rule same-index, rule diff-value)
qed

lemma wp-trace-fri-header-tied-base-head-value-neq-gap-bound-from-partial-merkle:
  assumes merkle-bound:
    wp-event verify-monad (partial-merkle-inconsistency-bad s)  $s \leq M$ 
  shows
    wp-event verify-monad
      (trace-fri-header-tied-base-head-value-neq-gap s)  $s \leq M$ 
proof -
  have wp-event verify-monad
    (trace-fri-header-tied-base-head-value-neq-gap s)  $s \leq$ 
    wp-event verify-monad (partial-merkle-inconsistency-bad s) s
  by (rule wp-event-mono)

```

```

      (rule trace-fri-header-tied-base-head-value-neq-gap-imp-partial-merkle)
    then show ?thesis
      by (rule order-trans[OF - merkle-bound])
  qed

end

end

```

```

theory Soundness-FRI-Trace-Candidate-Gap
  imports Soundness-FRI-Trace-Head-Value
begin

```

Candidate-availability split for the reachable/header-tied trace FRI gap. This layer depends on the exact FRI-query opening bridge but keeps the exported split separate from the base head-value residual theory.

```

context soundness
begin

```

```

lemma trace-fri-reachable-header-tie-gap-imp-merkle-or-candidate-available-on-support:

```

```

  assumes support: out ∈ set-dist (execute verify-monad s)
  and gap: trace-fri-reachable-header-tie-gap s out
  shows

```

```

    partial-merkle-inconsistency-bad s out ∨
    trace-fri-header-tied-candidate-available-gap s out

```

```

proof (cases partial-merkle-inconsistency-bad s out)

```

```

  case True
  then show ?thesis by simp

```

```

next

```

```

  case no-merkle: False

```

```

  from gap have raw: trace-fri-bad-with-reachable-partial-candidate s out

```

```

    unfolding trace-fri-reachable-header-tie-gap-def by simp

```

```

  from trace-fri-bad-with-reachable-partial-candidateE[OF raw]

```

```

  obtain fr0 query-idxs0 trace-openings0 trace-table0 where partial0:

```

```

    accepted-with-partial-trace-openings s out fr0 query-idxs0
    trace-openings0

```

```

  using raw unfolding trace-fri-bad-with-reachable-partial-candidate-def
  by blast

```

```

  have acc: accepted out

```

```

    by (rule accepted-with-partial-trace-openings-imp-accepted[OF partial0])

```

```

  from acc obtain result final-state where out-eq:

```

```

    out = Some (result, final-state)

```

```

    unfolding accepted-def by (cases out) auto

```

```

  from verify-monad-trace-fri-bad-extracts-opening-evidence

```

```

    [OF support[unfolded out-eq] raw[unfolded out-eq]]

```

```

  obtain trace-roots trace-bs trace-final dg composition-roots

```

```

    composition-bs composition-final fri-query-idxs trace-round-layers

```

```

    composition-round-layers fr1 candidate-query-idxs trace-openings1

```

trace-table1
where *evidence*:
trace-fri-partial-candidate-opening-evidence s
(Some (result, final-state)) trace-roots trace-bs trace-final dg
composition-roots composition-bs composition-final fri-query-idxs
trace-round-layers composition-round-layers fr1 candidate-query-idxs
trace-openings1 trace-table1
by (rule *verify-monad-trace-fri-bad-extracts-opening-evidence*
 $[OF\ support[unfolded\ out-eq]\ raw[unfolded\ out-eq]]$)
have *fri-openings*:
accepted-fri-opening-transcript s out trace-roots trace-bs
trace-final dg composition-roots composition-bs composition-final
fri-query-idxs trace-round-layers composition-round-layers
using *trace-fri-partial-candidate-opening-evidenceD(1)[OF evidence]*
unfolding *out-eq* .
from *accepted-fri-opening-transcript-headerE[OF fri-openings]*
obtain *fr as rest* **where** *header*:
verifier-header-transcript s fr trace-roots trace-final as dg
composition-roots composition-final rest
by (rule *accepted-fri-opening-transcript-headerE[OF fri-openings]*)
from *accepted-fri-opening-transcript-trace-openings-for-query-indices*
 $[OF\ fri-openings\ header\ out-eq]$
obtain *trace-openings* **where** *partial*:
accepted-with-partial-trace-openings s out fr fri-query-idxs
trace-openings
by (rule *accepted-fri-opening-transcript-trace-openings-for-query-indices*
 $[OF\ fri-openings\ header\ out-eq]$)
obtain *trace-table* **where** *candidate*:
partial-trace-table-candidate trace-table trace-openings
using *accepted-with-partial-trace-openings-candidate-if-no-partial-merkle-bad*
 $[OF\ partial[unfolded\ out-eq]\ no-merkle[unfolded\ out-eq]]$
by *blast*
show *?thesis*
unfolding *trace-fri-header-tied-candidate-available-gap-def*
by (intro *disjI2 exI conjI*)
(rule gap, rule fri-openings, rule header, rule partial,
rule candidate)
qed

lemma *wp-trace-fri-reachable-header-tie-gap-bound-from-merkle-and-candidate-transfer*:
assumes *merkle-bound*:
wp-event verify-monad (partial-merkle-inconsistency-bad s) s ≤ M
and *transfer-bound*:
wp-event verify-monad
(trace-fri-header-tied-candidate-transfer-gap s) s ≤ T
shows
wp-event verify-monad
(trace-fri-reachable-header-tie-gap s) s ≤ M + T
proof –

have *split-bound*:

$$\text{wp-event verify-monad } (\text{trace-fri-reachable-header-tie-gap } s) \ s \leq$$

$$\text{wp-event verify-monad } (\lambda \text{out. partial-merkle-inconsistency-bad } s \ \text{out} \ \vee$$

$$\text{trace-fri-header-tied-candidate-available-gap } s \ \text{out}) \ s$$
by (rule *wp-event-mono-on-support*)
(rule *trace-fri-reachable-header-tie-gap-imp-merkle-or-candidate-available-on-support*)
also have $\dots \leq$

$$\text{wp-event verify-monad } (\text{partial-merkle-inconsistency-bad } s) \ s +$$

$$\text{wp-event verify-monad } (\text{trace-fri-header-tied-candidate-available-gap } s) \ s$$
by (rule *wp-event-union-bound*)
also have $\dots \leq M + T$
by (rule *add-mono*[*OF merkle-bound*])
(rule *wp-trace-fri-header-tied-candidate-available-gap-bound-from-transfer-gap*
[*OF transfer-bound*])
finally show *?thesis* .
qed

lemma *trace-fri-reachable-partial-candidate-reduction-from-header-merkle-and-candidate-transfer*:
assumes *header-bound*:

$$\text{wp-event verify-monad } (\text{trace-fri-bad-with-header-tied-partial-candidate } s) \ s \leq Ht$$
and *merkle-bound*:

$$\text{wp-event verify-monad } (\text{partial-merkle-inconsistency-bad } s) \ s \leq Mt$$
and *transfer-bound*:

$$\text{wp-event verify-monad } (\text{trace-fri-header-tied-candidate-transfer-gap } s) \ s \leq Tt$$
and *total-bound*: $Ht + (Mt + Tt) \leq \text{trace-fri-error}$
shows *trace-fri-reachable-partial-candidate-reduction-assumption* *s*
proof –
have *gap-bound*:

$$\text{wp-event verify-monad } (\text{trace-fri-reachable-header-tie-gap } s) \ s \leq Mt + Tt$$
by (rule *wp-trace-fri-reachable-header-tie-gap-bound-from-merkle-and-candidate-transfer*
[*OF merkle-bound transfer-bound*])
have *reachable-bound*:

$$\text{wp-event verify-monad } (\text{trace-fri-bad-with-reachable-partial-candidate } s) \ s \leq$$

$$Ht + (Mt + Tt)$$
by (rule *wp-trace-fri-reachable-bound-from-header-tied-and-gap*
[*OF header-bound gap-bound*])
show *?thesis*
unfolding *trace-fri-reachable-partial-candidate-reduction-assumption-def*
by (rule *order-trans*[*OF reachable-bound total-bound*])

qed

end

end

theory *Soundness-FRI-Composition-Support*
 imports *Soundness-FRI-Composition-Residuals*
begin

Small support/extraction facts for the composition FRI reachable gap. These are factored out to avoid large direct proofs that stall theory finalization.

context *soundness*
begin

definition *composition-fri-reachable-verifier-tie-empty-header-gap*
 $:: (f, 'a) \text{ protocol-channel-scheme} \Rightarrow$
 $(\text{unit list} \times (f, 'a) \text{ protocol-channel-scheme}) \text{ option} \Rightarrow \text{bool}$

where
 $\text{composition-fri-reachable-verifier-tie-empty-header-gap } s \text{ out}$
 $\iff$
 $(\exists \text{ trace-roots trace-bs trace-final fri-dg composition-roots}$
 $\text{composition-bs composition-final fri-query-idxs trace-round-layers}$
 $\text{composition-round-layers.}$
 $\text{composition-fri-reachable-verifier-tie-gap } s \text{ out} \wedge$
 $\text{accepted-fri-opening-transcript } s \text{ out trace-roots trace-bs}$
 $\text{trace-final fri-dg composition-roots composition-bs}$
 $\text{composition-final fri-query-idxs trace-round-layers}$
 $\text{composition-round-layers} \wedge$
 $\text{composition-roots} = [])$

lemma *composition-fri-bad-with-reachable-partial-candidate-imp-accepted:*

assumes *composition-fri-bad-with-reachable-partial-candidate* $s \text{ out}$

shows *accepted* out

proof (rule *composition-fri-bad-with-reachable-partial-candidateE*[*OF assms*])

fix $fr f\text{-roots } f\text{-final } as \text{ dg composition-roots final}$
 $\text{trace-query-idxs trace-openings composition-query-idxs}$
 $\text{composition-openings trace-table composition-table}$

assume *evidence:*

$\text{composition-fri-partial-candidate-evidence } s \text{ out } fr f\text{-roots}$
 $f\text{-final } as \text{ dg composition-roots final trace-query-idxs}$
 $\text{trace-openings composition-query-idxs composition-openings}$
 $\text{trace-table composition-table}$

have *partial:*

$\text{accepted-with-partial-initial-openings } s \text{ out } fr f\text{-roots } f\text{-final}$
 $as \text{ dg composition-roots final trace-query-idxs trace-openings}$
 $\text{composition-query-idxs composition-openings}$

by (rule *composition-fri-partial-candidate-evidenceD*(1)[*OF evidence*])

then show *accepted out*
by (rule *accepted-with-partial-initial-openings-imp-accepted*)
qed

lemma *composition-fri-reachable-verifier-tie-gap-imp-reachable-bad*:
assumes *composition-fri-reachable-verifier-tie-gap s out*
shows *composition-fri-bad-with-reachable-partial-candidate s out*
using *assms unfolding composition-fri-reachable-verifier-tie-gap-def* **by** *simp*

lemma *composition-fri-reachable-verifier-tie-gap-imp-accepted*:
assumes *composition-fri-reachable-verifier-tie-gap s out*
shows *accepted out*
by (rule *composition-fri-bad-with-reachable-partial-candidate-imp-accepted*)
(rule *composition-fri-reachable-verifier-tie-gap-imp-reachable-bad* [OF *assms*])

lemma *composition-fri-reachable-verifier-tie-empty-header-gapI*:
assumes *gap: composition-fri-reachable-verifier-tie-gap s out*
and *fri-openings*:
accepted-fri-opening-transcript s out trace-roots trace-bs
trace-final fri-dg composition-roots composition-bs
composition-final fri-query-idxs trace-round-layers
composition-round-layers
and *empty: composition-roots = []*
shows *composition-fri-reachable-verifier-tie-empty-header-gap s out*
unfolding *composition-fri-reachable-verifier-tie-empty-header-gap-def*
by (*intro exI conjI*)
(rule *gap, rule fri-openings, rule empty*)

lemma *composition-fri-reachable-verifier-tie-empty-header-gap-false*:
 $\neg$ *composition-fri-reachable-verifier-tie-empty-header-gap s out*
proof
assume *empty-gap*:
composition-fri-reachable-verifier-tie-empty-header-gap s out
then obtain *trace-roots trace-bs trace-final fri-dg composition-roots*
composition-bs composition-final fri-query-idxs trace-round-layers
composition-round-layers
where *gap: composition-fri-reachable-verifier-tie-gap s out*
and *fri-openings*:
accepted-fri-opening-transcript s out trace-roots trace-bs
trace-final fri-dg composition-roots composition-bs
composition-final fri-query-idxs trace-round-layers
composition-round-layers
and *empty: composition-roots = []*
unfolding *composition-fri-reachable-verifier-tie-empty-header-gap-def*
by *blast*
obtain *fr as rest* **where** *verifier-header*:
verifier-header-transcript s fr trace-roots trace-final as fri-dg
composition-roots composition-final rest
by (rule *accepted-fri-opening-transcript-headerE* [OF *fri-openings*])

have *raw*: *composition-fri-bad-with-reachable-partial-candidate s out*
using *gap unfolding composition-fri-reachable-verifier-tie-gap-def*
by *simp*
from *composition-fri-bad-with-reachable-partial-candidateE[OF raw]*
obtain *fr' f-fri-roots f-final as' dg' composition-fri-roots final*
trace-query-idxs trace-openings composition-query-idxs
composition-openings trace-table composition-table
where *evidence*:
composition-fri-partial-candidate-evidence s out fr' f-fri-roots
f-final as' dg' composition-fri-roots final trace-query-idxs
trace-openings composition-query-idxs composition-openings
trace-table composition-table
by *blast*
have *partial*:
accepted-with-partial-initial-openings s out fr' f-fri-roots
f-final as' dg' composition-fri-roots final trace-query-idxs
trace-openings composition-query-idxs composition-openings
by (*rule composition-fri-partial-candidate-evidenceD(1)[OF evidence]*)
have *nonempty*: *composition-fri-roots* $\neq []$
by (*rule accepted-with-partial-initial-openings-shapes(1)[OF partial]*)
obtain *rest' where candidate-header*:
verifier-header-transcript s fr' f-fri-roots f-final as' dg'
composition-fri-roots final rest'
using *accepted-with-partial-initial-openings-shapes(2)[OF partial]*
by *blast*
have *composition-fri-roots = composition-roots*
using *verifier-header-transcript-unique*
[OF verifier-header candidate-header]
by *simp*
then show *False*
using *empty nonempty by simp*
qed

lemma *wp-composition-fri-reachable-verifier-tie-empty-header-gap-zero*:
wp-event verify-monad
(composition-fri-reachable-verifier-tie-empty-header-gap s) s = 0
unfolding *wp-event-def wp-def dist-expect-def*
by (*simp add*:
composition-fri-reachable-verifier-tie-empty-header-gap-false)

lemma *composition-fri-reachable-verifier-tie-gap-with-fri-openings-empty-or-nonempty*:
assumes *gap*: *composition-fri-reachable-verifier-tie-gap s out*
and *fri-openings*:
accepted-fri-opening-transcript s out trace-roots trace-bs
trace-final fri-dg composition-roots composition-bs
composition-final fri-query-idxs trace-round-layers
composition-round-layers
shows
composition-fri-reachable-verifier-tie-empty-header-gap s out $\vee$


```

    composition-roots  $\neq$  []
proof (cases composition-roots = [])
  case True
    have composition-fri-reachable-verifier-tie-empty-header-gap s out
      by (rule composition-fri-reachable-verifier-tie-empty-header-gapI
        [OF gap fri-openings True])
    then show ?thesis by simp
  next
    case False
    then show ?thesis by simp
qed

```

lemma composition-fri-verifier-tied-candidate-available-gapI:

assumes gap: composition-fri-reachable-verifier-tie-gap s out
and fri-openings:
 accepted-fri-opening-transcript s out trace-roots trace-bs
 trace-final fri-dg composition-roots composition-bs
 composition-final fri-query-idxs trace-round-layers
 composition-round-layers
and aligned:
 accepted-with-partial-initial-openings-aligned s out fr trace-roots
 trace-final as fri-dg composition-roots composition-final
 fri-query-idxs trace-openings composition-openings
and trace-candidate:
 partial-trace-table-candidate trace-table trace-openings
and composition-candidate:
 partial-composition-table-candidate composition-table
 composition-openings
shows composition-fri-verifier-tied-candidate-available-gap s out
unfolding composition-fri-verifier-tied-candidate-available-gap-def
by (intro exI conjI)
 (rule gap, rule fri-openings, rule aligned, rule trace-candidate,
 rule composition-candidate)

lemma composition-fri-reachable-verifier-tie-gap-with-aligned-openings-imp-merkle-or-candidate-available:

assumes out-eq: out = Some (result, final-state)
and gap: composition-fri-reachable-verifier-tie-gap s out
and fri-openings:
 accepted-fri-opening-transcript s out trace-roots trace-bs
 trace-final fri-dg composition-roots composition-bs
 composition-final fri-query-idxs trace-round-layers
 composition-round-layers
and aligned:
 accepted-with-partial-initial-openings-aligned s out fr trace-roots
 trace-final as fri-dg composition-roots composition-final
 fri-query-idxs trace-openings composition-openings
shows
 partial-merkle-inconsistency-bad s out $\vee$
 composition-fri-verifier-tied-candidate-available-gap s out

proof (*cases partial-merkle-inconsistency-bad s out*)
case *True*
then show *?thesis by simp*
next
case *no-merkle: False*
obtain *trace-table composition-table where trace-candidate:*
partial-trace-table-candidate trace-table trace-openings
and *composition-candidate:*
partial-composition-table-candidate composition-table
composition-openings
using
accepted-with-partial-initial-openings-aligned-candidates-if-no-partial-merkle-bad
[OF aligned[unfolded out-eq] no-merkle[unfolded out-eq]]
by *blast*
have *available:*
composition-fri-verifier-tied-candidate-available-gap s out
by (*rule composition-fri-verifier-tied-candidate-available-gapI*
[OF gap fri-openings aligned trace-candidate composition-candidate])
then show *?thesis by simp*
qed

lemma *composition-fri-reachable-verifier-tie-gap-with-fri-openings-headerE:*
assumes
accepted-fri-opening-transcript s out trace-roots trace-bs
trace-final fri-dg composition-roots composition-bs
composition-final fri-query-idxs trace-round-layers
composition-round-layers
obtains *fr as rest where*
verifier-header-transcript s fr trace-roots trace-final as fri-dg
composition-roots composition-final rest
by (*rule accepted-fri-opening-transcript-headerE[OF assms]*)

lemma *composition-fri-reachable-verifier-tie-gap-with-fri-openings-header-and-branchE:*
assumes *gap: composition-fri-reachable-verifier-tie-gap s out*
and *fri-openings:*
accepted-fri-opening-transcript s out trace-roots trace-bs
trace-final fri-dg composition-roots composition-bs
composition-final fri-query-idxs trace-round-layers
composition-round-layers
obtains *fr as rest where*
verifier-header-transcript s fr trace-roots trace-final as fri-dg
composition-roots composition-final rest
composition-fri-reachable-verifier-tie-empty-header-gap s out $\vee$
composition-roots $\neq []$

proof –
obtain *fr as rest where header:*
verifier-header-transcript s fr trace-roots trace-final as fri-dg
composition-roots composition-final rest
by (*rule composition-fri-reachable-verifier-tie-gap-with-fri-openings-headerE*)

```

    [OF fri-openings])
  have branch:
    composition-fri-reachable-verifier-tie-empty-header-gap s out ∨
    composition-roots ≠ []
  by (rule
    composition-fri-reachable-verifier-tie-gap-with-fri-openings-empty-or-nonempty
    [OF gap fri-openings])
  show ?thesis
    by (rule that[OF header branch])
qed

lemma composition-fri-reachable-verifier-tie-gap-nonempty-imp-merkle-or-candidate-available:
  assumes out-eq: out = Some (result, final-state)
  and gap: composition-fri-reachable-verifier-tie-gap s out
  and fri-openings:
    accepted-fri-opening-transcript s out trace-roots trace-bs
    trace-final fri-dg composition-roots composition-bs
    composition-final fri-query-idxs trace-round-layers
    composition-round-layers
  and nonempty: composition-roots ≠ []
  shows
    partial-merkle-inconsistency-bad s out ∨
    composition-fri-verifier-tied-candidate-available-gap s out
proof -
  have split:
    partial-merkle-inconsistency-bad s (Some (result, final-state)) ∨
    (∃ fr as rest trace-openings composition-openings trace-table
    composition-table.
    verifier-header-transcript s fr trace-roots trace-final as fri-dg
    composition-roots composition-final rest ∧
    accepted-with-partial-initial-openings-aligned s
    (Some (result, final-state)) fr trace-roots trace-final as fri-dg
    composition-roots composition-final fri-query-idxs trace-openings
    composition-openings ∧
    partial-trace-table-candidate trace-table trace-openings ∧
    partial-composition-table-candidate composition-table
    composition-openings)
  by (rule
    accepted-fri-opening-transcript-obtains-aligned-partial-candidates-or-partial-merkle
    [OF fri-openings[unfolded out-eq] nonempty])
  show ?thesis
proof (rule disjE[OF split])
  assume partial-merkle-inconsistency-bad s (Some (result, final-state))
  then show ?thesis
    by (simp add: out-eq)
next
  assume
    ∃ fr as rest trace-openings composition-openings trace-table
    composition-table.

```

```

    verifier-header-transcript s fr trace-roots trace-final as fri-dg
    composition-roots composition-final rest  $\wedge$ 
    accepted-with-partial-initial-openings-aligned s
    (Some (result, final-state)) fr trace-roots trace-final as fri-dg
    composition-roots composition-final fri-query-idxs trace-openings
    composition-openings  $\wedge$ 
    partial-trace-table-candidate trace-table trace-openings  $\wedge$ 
    partial-composition-table-candidate composition-table
    composition-openings
  then obtain fr as rest trace-openings composition-openings trace-table
    composition-table where aligned:
    accepted-with-partial-initial-openings-aligned s out fr trace-roots
    trace-final as fri-dg composition-roots composition-final
    fri-query-idxs trace-openings composition-openings
  and trace-candidate:
    partial-trace-table-candidate trace-table trace-openings
  and composition-candidate:
    partial-composition-table-candidate composition-table
    composition-openings
  by (auto simp: out-eq)
  have available:
    composition-fri-verifier-tied-candidate-available-gap s out
  by (rule composition-fri-verifier-tied-candidate-available-gapI
    [OF gap fri-openings aligned trace-candidate composition-candidate])
  then show ?thesis by simp
qed
qed

lemma composition-fri-reachable-verifier-tie-gap-imp-empty-or-merkle-or-candidate-available-on-support:
  assumes support: out  $\in$  set-dist (execute verify-monad s)
  and gap: composition-fri-reachable-verifier-tie-gap s out
  shows
    composition-fri-reachable-verifier-tie-empty-header-gap s out  $\vee$ 
    partial-merkle-inconsistency-bad s out  $\vee$ 
    composition-fri-verifier-tied-candidate-available-gap s out
proof –
  have raw: composition-fri-bad-with-reachable-partial-candidate s out
  using gap
  unfolding composition-fri-reachable-verifier-tie-gap-def by simp
  have acc: accepted out
  by (rule composition-fri-bad-with-reachable-partial-candidate-imp-accepted
    [OF raw])
  obtain result final-state where out-eq: out = Some (result, final-state)
  using acc unfolding accepted-def by (cases out) auto
  from verify-monad-accepted-fri-opening-transcript[OF support[unfolded out-eq]]
  obtain trace-roots trace-bs trace-final fri-dg composition-roots
    composition-bs composition-final fri-query-idxs trace-round-layers
    composition-round-layers
  where fri-openings:

```

```

    accepted-fri-opening-transcript s out trace-roots trace-bs
    trace-final fri-dg composition-roots composition-bs
    composition-final fri-query-idxs trace-round-layers
    composition-round-layers
  unfolding out-eq by blast
show ?thesis
proof (cases composition-roots = [])
  case True
  have composition-fri-reachable-verifier-tie-empty-header-gap s out
    by (rule composition-fri-reachable-verifier-tie-empty-header-gapI
        [OF gap fri-openings True])
  then show ?thesis by simp
next
  case False
  have
    partial-merkle-inconsistency-bad s out  $\vee$ 
    composition-fri-verifier-tied-candidate-available-gap s out
  by (rule
      composition-fri-reachable-verifier-tie-gap-nonempty-imp-merkle-or-candidate-available
      [OF out-eq gap fri-openings False])
  then show ?thesis by simp
qed
qed

lemma wp-composition-fri-reachable-verifier-tie-gap-bound-from-empty-merkle-and-candidate-transfer:
  assumes empty-bound:
    wp-event verify-monad
      (composition-fri-reachable-verifier-tie-empty-header-gap s)  $s \leq E$ 
  and merkle-bound:
    wp-event verify-monad (partial-merkle-inconsistency-bad s)  $s \leq M$ 
  and transfer-bound:
    wp-event verify-monad
      (composition-fri-verifier-tied-candidate-transfer-gap s)  $s \leq T$ 
  shows
    wp-event verify-monad
      (composition-fri-reachable-verifier-tie-gap s)  $s \leq E + (M + T)$ 
proof -
  have split-bound:
    wp-event verify-monad
      (composition-fri-reachable-verifier-tie-gap s)  $s \leq$ 
      wp-event verify-monad
        ( $\lambda out.$ 
          composition-fri-reachable-verifier-tie-empty-header-gap s out  $\vee$ 
          partial-merkle-inconsistency-bad s out  $\vee$ 
          composition-fri-verifier-tied-candidate-available-gap s out) s
  by (rule wp-event-mono-on-support)
  (rule
    composition-fri-reachable-verifier-tie-gap-imp-empty-or-merkle-or-candidate-available-on-support)
  also have ...  $\leq$ 

```

$wp\text{-}event\ verify\text{-}monad$
 $(composition\text{-}fri\text{-}reachable\text{-}verifier\text{-}tie\text{-}empty\text{-}header\text{-}gap\ s)\ s +$
 $wp\text{-}event\ verify\text{-}monad$
 $(\lambda out.$
 $\quad partial\text{-}merkle\text{-}inconsistency\text{-}bad\ s\ out \vee$
 $\quad composition\text{-}fri\text{-}verifier\text{-}tied\text{-}candidate\text{-}available\text{-}gap\ s\ out)\ s$
by (rule $wp\text{-}event\text{-}union\text{-}bound$)
also have $\dots \leq$
 $wp\text{-}event\ verify\text{-}monad$
 $(composition\text{-}fri\text{-}reachable\text{-}verifier\text{-}tie\text{-}empty\text{-}header\text{-}gap\ s)\ s +$
 $(wp\text{-}event\ verify\text{-}monad\ (partial\text{-}merkle\text{-}inconsistency\text{-}bad\ s)\ s) +$
 $wp\text{-}event\ verify\text{-}monad$
 $(composition\text{-}fri\text{-}verifier\text{-}tied\text{-}candidate\text{-}available\text{-}gap\ s)\ s)$
by (rule $add\text{-}mono[OF\ order\text{-}reft\ wp\text{-}event\text{-}union\text{-}bound]$)
also have $\dots \leq E + (M + T)$
by (intro $add\text{-}mono\ empty\text{-}bound\ merkle\text{-}bound$
 $wp\text{-}composition\text{-}fri\text{-}verifier\text{-}tied\text{-}candidate\text{-}available\text{-}gap\text{-}bound\text{-}from\text{-}transfer\text{-}gap$
 $transfer\text{-}bound$)
finally show $?thesis$.
qed

lemma $composition\text{-}fri\text{-}reachable\text{-}partial\text{-}candidate\text{-}reduction\text{-}from\text{-}split\text{-}residual\text{-}bounds$:
fixes $R\ A\ P\ M\ N\ S\ G\ E\ Q\ T :: prob$
assumes $sampld\text{-}bound$:
 $wp\text{-}event\ verify\text{-}monad$
 $(composition\text{-}fri\text{-}bad\text{-}with\text{-}verifier\text{-}tied\text{-}sampled\text{-}layer\text{-}chain\ s)\ s \leq R$
and $slot\text{-}gap\text{-}bound$:
 $wp\text{-}event\ verify\text{-}monad$
 $(composition\text{-}fri\text{-}verifier\text{-}tied\text{-}authenticated\text{-}slot\text{-}recorded\text{-}chunk\text{-}gap\ s)\ s \leq A$
and $verifier\text{-}partial\text{-}merkle\text{-}bound$:
 $wp\text{-}event\ verify\text{-}monad\ (partial\text{-}merkle\text{-}inconsistency\text{-}bad\ s)\ s \leq P$
and $missing\text{-}bound$:
 $wp\text{-}event\ verify\text{-}monad$
 $(composition\text{-}fri\text{-}verifier\text{-}tied\text{-}recorded\text{-}base\text{-}chunk\text{-}auth\text{-}missing\ s)$
 $s \leq M$
and $next\text{-}gap\text{-}bound$:
 $wp\text{-}event\ verify\text{-}monad$
 $(composition\text{-}fri\text{-}verifier\text{-}tied\text{-}next\text{-}value\text{-}replay\text{-}gap\ s)\ s \leq N$
and $successor\text{-}gap\text{-}bound$:
 $wp\text{-}event\ verify\text{-}monad$
 $(composition\text{-}fri\text{-}verifier\text{-}tied\text{-}successor\text{-}replay\text{-}gap\ s)\ s \leq S$
and $final\text{-}gap\text{-}bound$:
 $wp\text{-}event\ verify\text{-}monad$
 $(composition\text{-}fri\text{-}verifier\text{-}tied\text{-}final\text{-}value\text{-}replay\text{-}gap\ s)\ s \leq G$
and $empty\text{-}bound$:
 $wp\text{-}event\ verify\text{-}monad$
 $(composition\text{-}fri\text{-}reachable\text{-}verifier\text{-}tie\text{-}empty\text{-}header\text{-}gap\ s)\ s \leq E$
and $gap\text{-}partial\text{-}merkle\text{-}bound$:

$wp\text{-event verify-monad (partial-merkle-inconsistency-bad } s) s \leq Q$
and *candidate-transfer-bound*:
 $wp\text{-event verify-monad}$
 $(composition\text{-fri-verifier-tied-candidate-transfer-gap } s) s \leq T$
and *total-bound*:
 $R + (((P + M) + N + S + G + P) + A) + (E + (Q + T))$
 $\leq composition\text{-fri-error}$
shows *composition-fri-reachable-partial-candidate-reduction-assumption* s
proof –
have *verifier-bound*:
 $wp\text{-event verify-monad}$
 $(composition\text{-fri-bad-with-verifier-tied-partial-candidate } s) s$
 $\leq R + (((P + M) + N + S + G + P) + A)$
by (rule
 $wp\text{-composition-fri-verifier-tied-bound-from-sampled-slot-recorded-chunk-gap-and-residuals}$
 $[OF\ sampled\text{-bound slot-gap-bound verifier-partial-merkle-bound}$
 $missing\text{-bound next-gap-bound successor-gap-bound final-gap-bound}]$)
have *gap-bound*:
 $wp\text{-event verify-monad}$
 $(composition\text{-fri-reachable-verifier-tie-gap } s) s \leq$
 $E + (Q + T)$
by (rule
 $wp\text{-composition-fri-reachable-verifier-tie-gap-bound-from-empty-merkle-and-candidate-transfer}$
 $[OF\ empty\text{-bound gap-partial-merkle-bound candidate-transfer-bound}]$)
show *?thesis*
by (rule
 $composition\text{-fri-reachable-partial-candidate-reduction-from-verifier-tied-and-gap}$
 $[OF\ verifier\text{-bound gap-bound total-bound}]$)
qed

lemma *composition-fri-reachable-partial-candidate-reduction-from-split-residual-bounds-with-recorded-missing-zero*:

fixes $R A P N S G E Q T :: prob$

assumes *sampled-bound*:

$wp\text{-event verify-monad}$

$(composition\text{-fri-bad-with-verifier-tied-sampled-layer-chain } s) s \leq R$

and *slot-gap-bound*:

$wp\text{-event verify-monad}$

$(composition\text{-fri-verifier-tied-authenticated-slot-recorded-chunk-gap}$
 $s) s \leq A$

and *verifier-partial-merkle-bound*:

$wp\text{-event verify-monad (partial-merkle-inconsistency-bad } s) s \leq P$

and *next-gap-bound*:

$wp\text{-event verify-monad}$

$(composition\text{-fri-verifier-tied-next-value-replay-gap } s) s \leq N$

and *successor-gap-bound*:

$wp\text{-event verify-monad}$

$(composition\text{-fri-verifier-tied-successor-replay-gap } s) s \leq S$

and *final-gap-bound*:

$wp\text{-event verify-monad}$

```

    (composition-fri-verifier-tied-final-value-replay-gap s) s ≤ G
and empty-bound:
wp-event verify-monad
    (composition-fri-reachable-verifier-tie-empty-header-gap s) s ≤ E
and gap-partial-merkle-bound:
wp-event verify-monad (partial-merkle-inconsistency-bad s) s ≤ Q
and candidate-transfer-bound:
wp-event verify-monad
    (composition-fri-verifier-tied-candidate-transfer-gap s) s ≤ T
and total-bound:
R + (((P + 0) + N + S + G + P) + A) + (E + (Q + T))
    ≤ composition-fri-error
shows composition-fri-reachable-partial-candidate-reduction-assumption s
by (rule
    composition-fri-reachable-partial-candidate-reduction-from-split-residual-bounds
    [where M = 0])
(rule sampled-bound,
rule slot-gap-bound,
rule verifier-partial-merkle-bound,
rule wp-composition-fri-verifier-tied-recorded-base-chunk-auth-missing-zero,
rule next-gap-bound,
rule successor-gap-bound,
rule final-gap-bound,
rule empty-bound,
rule gap-partial-merkle-bound,
rule candidate-transfer-bound,
rule total-bound)

end

end

```

```

theory Soundness-FRI-Candidate-Transfer-Support
imports
    Soundness-FRI-Trace-Candidate-Gap
    Soundness-FRI-Composition-Support
begin

```

Small destructors for the remaining FRI candidate-transfer branches. These branches are intentionally not closed by complete-table uniqueness: sampled partial openings can admit multiple completions. The lemmas here expose the exact low/non-low candidate shape that later proofs must charge to a partial-opening inconsistency or a narrower candidate-indexed event.

```

context soundness
begin

```

```

definition trace-fri-header-tied-low-candidate-available
:: ('f, 'a) protocol-channel-scheme ⇒

```


$(\text{unit list} \times ('f, 'a) \text{ protocol-channel-scheme}) \text{ option} \Rightarrow \text{bool}$
where
 $\text{trace-fri-header-tied-low-candidate-available } s \text{ out} \longleftrightarrow$
 $(\exists \text{ trace-roots trace-bs trace-final fri-dg composition-roots}$
 $\text{composition-bs composition-final fri-query-idxs trace-round-layers}$
 $\text{composition-round-layers fr as rest trace-openings trace-table.}$
 $\text{accepted-fri-opening-transcript } s \text{ out trace-roots trace-bs}$
 $\text{trace-final fri-dg composition-roots composition-bs}$
 $\text{composition-final fri-query-idxs trace-round-layers}$
 $\text{composition-round-layers} \wedge$
 $\text{verifier-header-transcript } s \text{ fr trace-roots trace-final as fri-dg}$
 $\text{composition-roots composition-final rest} \wedge$
 $\text{accepted-with-partial-trace-openings } s \text{ out fr fri-query-idxs}$
 $\text{trace-openings} \wedge$
 $\text{partial-trace-table-candidate trace-table trace-openings} \wedge$
 $\text{trace-table-low-degree trace-table})$

definition $\text{trace-fri-header-tied-low-candidate-gap}$
 $:: ('f, 'a) \text{ protocol-channel-scheme} \Rightarrow$
 $(\text{unit list} \times ('f, 'a) \text{ protocol-channel-scheme}) \text{ option} \Rightarrow \text{bool}$
where

$\text{trace-fri-header-tied-low-candidate-gap } s \text{ out} \longleftrightarrow$
 $(\exists \text{ trace-roots trace-bs trace-final fri-dg composition-roots}$
 $\text{composition-bs composition-final fri-query-idxs trace-round-layers}$
 $\text{composition-round-layers fr as rest trace-openings trace-table.}$
 $\text{trace-fri-reachable-header-tie-gap } s \text{ out} \wedge$
 $\text{accepted-fri-opening-transcript } s \text{ out trace-roots trace-bs}$
 $\text{trace-final fri-dg composition-roots composition-bs}$
 $\text{composition-final fri-query-idxs trace-round-layers}$
 $\text{composition-round-layers} \wedge$
 $\text{verifier-header-transcript } s \text{ fr trace-roots trace-final as fri-dg}$
 $\text{composition-roots composition-final rest} \wedge$
 $\text{accepted-with-partial-trace-openings } s \text{ out fr fri-query-idxs}$
 $\text{trace-openings} \wedge$
 $\text{partial-trace-table-candidate trace-table trace-openings} \wedge$
 $\text{trace-table-low-degree trace-table})$

definition $\text{composition-fri-verifier-tied-misaligned-candidate-classification}$
 $:: ('f, 'a) \text{ protocol-channel-scheme} \Rightarrow$
 $(\text{unit list} \times ('f, 'a) \text{ protocol-channel-scheme}) \text{ option} \Rightarrow \text{bool}$
where

$\text{composition-fri-verifier-tied-misaligned-candidate-classification } s \text{ out}$
 $\longleftrightarrow$
 $(\exists \text{ trace-roots trace-bs trace-final fri-dg composition-roots}$
 $\text{composition-bs composition-final fri-query-idxs trace-round-layers}$
 $\text{composition-round-layers fr as trace-openings composition-openings}$
 $\text{trace-table composition-table.}$
 $\text{accepted-fri-opening-transcript } s \text{ out trace-roots trace-bs}$
 $\text{trace-final fri-dg composition-roots composition-bs}$

composition-final fri-query-idxs trace-round-layers
composition-round-layers $\wedge$
accepted-with-partial-initial-openings-aligned s out fr trace-roots
trace-final as fri-dg composition-roots composition-final
fri-query-idxs trace-openings composition-openings $\wedge$
partial-trace-table-candidate trace-table trace-openings $\wedge$
partial-composition-table-candidate composition-table
composition-openings $\wedge$
 $\neg$ (*trace-table-low-degree trace-table* $\wedge$
 $\neg$ *composition-table-low-degree maxDegree composition-table*)

definition *composition-fri-verifier-tied-transcript-consistent-misaligned-classification*

$:: ('f, 'a) \text{ protocol-channel-scheme} \Rightarrow$
 $(\text{unit list} \times ('f, 'a) \text{ protocol-channel-scheme}) \text{ option} \Rightarrow \text{bool}$

where

composition-fri-verifier-tied-transcript-consistent-misaligned-classification
s out $\longleftrightarrow$
 $(\exists \text{ trace-roots trace-bs trace-final fri-dg composition-roots}$
 $\text{composition-bs composition-final fri-query-idxs trace-round-layers}$
 $\text{composition-round-layers fr as rest trace-openings}$
 $\text{composition-openings trace-table composition-table.}$
accepted-fri-opening-transcript s out trace-roots trace-bs
trace-final fri-dg composition-roots composition-bs
composition-final fri-query-idxs trace-round-layers
composition-round-layers $\wedge$
verifier-header-transcript s fr trace-roots trace-final as fri-dg
composition-roots composition-final rest $\wedge$
accepted-with-partial-initial-openings-aligned-transcript-consistent s
out fr trace-roots trace-final as fri-dg composition-roots
composition-final fri-query-idxs trace-openings composition-openings $\wedge$
partial-trace-table-candidate trace-table trace-openings $\wedge$
partial-composition-table-candidate composition-table
composition-openings $\wedge$
 $\neg$ (*trace-table-low-degree trace-table* $\wedge$
 $\neg$ *composition-table-low-degree maxDegree composition-table*)

definition *composition-fri-verifier-tied-transcript-consistent-classification-cover*

$:: ('f, 'a) \text{ protocol-channel-scheme} \Rightarrow$
 $(\text{unit list} \times ('f, 'a) \text{ protocol-channel-scheme}) \text{ option} \Rightarrow \text{bool}$

where

composition-fri-verifier-tied-transcript-consistent-classification-cover
s out $\longleftrightarrow$
composition-bad-with-partial-candidates s out $\vee$
trace-fri-bad-with-header-tied-partial-candidate s out $\vee$
query-bad-with-aligned-transcript-partial-candidates s out

lemma *trace-fri-header-tied-candidate-transfer-gapD:*

assumes *trace-fri-header-tied-candidate-transfer-gap s out*

shows

trace-fri-bad-with-reachable-partial-candidate s out
trace-fri-header-tied-low-candidate-available s out
proof –
from *assms* **obtain** *trace-roots trace-bs trace-final fri-dg*
composition-roots composition-bs composition-final fri-query-idxs
trace-round-layers composition-round-layers fr as rest trace-openings
trace-table **where**
bad: trace-fri-bad-with-reachable-partial-candidate s out
and *fri-openings:*
accepted-fri-opening-transcript s out trace-roots trace-bs
trace-final fri-dg composition-roots composition-bs
composition-final fri-query-idxs trace-round-layers
composition-round-layers
and *header:*
verifier-header-transcript s fr trace-roots trace-final as fri-dg
composition-roots composition-final rest
and *partial:*
accepted-with-partial-trace-openings s out fr fri-query-idxs
trace-openings
and *candidate:*
partial-trace-table-candidate trace-table trace-openings
and *low: trace-table-low-degree trace-table*
unfolding *trace-fri-header-tied-candidate-transfer-gap-def*
by *blast*
show *trace-fri-bad-with-reachable-partial-candidate s out*
by (*rule bad*)
show *trace-fri-header-tied-low-candidate-available s out*
unfolding *trace-fri-header-tied-low-candidate-available-def*
by (*intro exI conjI*)
(*rule fri-openings, rule header, rule partial, rule candidate,*
rule low)
qed

lemma *trace-fri-header-tied-candidate-transfer-gap-imp-accepted:*
assumes *trace-fri-header-tied-candidate-transfer-gap s out*
shows *accepted out*
proof –
have *bad: trace-fri-bad-with-reachable-partial-candidate s out*
by (*rule trace-fri-header-tied-candidate-transfer-gapD(1)[OF assms]*)
from *trace-fri-bad-with-reachable-partial-candidateE[OF bad]*
obtain *fr query-idxs trace-openings trace-table* **where** *evidence:*
trace-fri-partial-candidate-evidence s out fr query-idxs
trace-openings trace-table
by *blast*
have *partial:*
accepted-with-partial-trace-openings s out fr query-idxs
trace-openings
by (*rule trace-fri-partial-candidate-evidenceD(1)[OF evidence]*)
show *?thesis*

by (rule *accepted-with-partial-trace-openings-imp-accepted*[*OF partial*])
 qed

lemma *wp-trace-fri-header-tied-candidate-transfer-gap-bound-from-low-candidate:*
 assumes *low-bound:*
 wp-event verify-monad
 (*trace-fri-header-tied-low-candidate-available s*) $s \leq T$
 shows
 wp-event verify-monad
 (*trace-fri-header-tied-candidate-transfer-gap s*) $s \leq T$
proof –
 have *wp-event verify-monad*
 (*trace-fri-header-tied-candidate-transfer-gap s*) $s \leq$
 wp-event verify-monad
 (*trace-fri-header-tied-low-candidate-available s*) s
 by (rule *wp-event-mono*)
 (rule *trace-fri-header-tied-candidate-transfer-gapD*(2))
 then show ?thesis
 by (rule *order-trans*[*OF - low-bound*])
 qed

lemma *trace-fri-header-tied-low-candidate-gap-imp-low-candidate-available:*
 assumes *trace-fri-header-tied-low-candidate-gap s out*
 shows *trace-fri-header-tied-low-candidate-available s out*
 using *assms*
 unfolding *trace-fri-header-tied-low-candidate-gap-def*
 trace-fri-header-tied-low-candidate-available-def
 by *blast*

lemma *trace-fri-header-tied-low-candidate-gap-imp-transfer-gap:*
 assumes *trace-fri-header-tied-low-candidate-gap s out*
 shows *trace-fri-header-tied-candidate-transfer-gap s out*
proof –
 from *assms* obtain *trace-roots trace-bs trace-final fri-dg*
 composition-roots composition-bs composition-final fri-query-idxs
 trace-round-layers composition-round-layers fr as rest trace-openings
 trace-table **where**
gap: trace-fri-reachable-header-tie-gap s out
and *fri-openings:*
 accepted-fri-opening-transcript s out trace-roots trace-bs
 trace-final fri-dg composition-roots composition-bs
 composition-final fri-query-idxs trace-round-layers
 composition-round-layers
and *header:*
 verifier-header-transcript s fr trace-roots trace-final as fri-dg
 composition-roots composition-final rest
and *partial:*
 accepted-with-partial-trace-openings s out fr fri-query-idxs
 trace-openings

```

and candidate:
  partial-trace-table-candidate trace-table trace-openings
and low: trace-table-low-degree trace-table
unfolding trace-fri-header-tied-low-candidate-gap-def by blast
show ?thesis
by (rule trace-fri-header-candidate-low-imp-transfer-gap
  [OF gap fri-openings header partial candidate low])
qed

lemma trace-fri-header-tied-candidate-available-gap-imp-low-candidate-gap:
  assumes trace-fri-header-tied-candidate-available-gap s out
  shows trace-fri-header-tied-low-candidate-gap s out
proof –
  from assms obtain trace-roots trace-bs trace-final fri-dg
    composition-roots composition-bs composition-final fri-query-idxs
    trace-round-layers composition-round-layers fr as rest trace-openings
    trace-table where
  gap: trace-fri-reachable-header-tie-gap s out
  and fri-openings:
    accepted-fri-opening-transcript s out trace-roots trace-bs
    trace-final fri-dg composition-roots composition-bs
    composition-final fri-query-idxs trace-round-layers
    composition-round-layers
  and header:
    verifier-header-transcript s fr trace-roots trace-final as fri-dg
    composition-roots composition-final rest
  and partial:
    accepted-with-partial-trace-openings s out fr fri-query-idxs
    trace-openings
  and candidate:
    partial-trace-table-candidate trace-table trace-openings
  unfolding trace-fri-header-tied-candidate-available-gap-def by blast
show ?thesis
proof (cases trace-table-low-degree trace-table)
  case True
  show ?thesis
    unfolding trace-fri-header-tied-low-candidate-gap-def
    by (intro exI conjI)
      (rule gap, rule fri-openings, rule header, rule partial,
       rule candidate, rule True)
  next
  case False
  have False
    by (rule trace-fri-header-candidate-non-low-contradicts-header-tie-gap
      [OF gap fri-openings header partial candidate False])
  then show ?thesis by simp
qed
qed

```

lemma *wp-trace-fri-header-tied-candidate-available-gap-bound-from-low-candidate-gap:*

assumes *low-gap-bound:*

wp-event verify-monad

$(\text{trace-fri-header-tied-low-candidate-gap } s) \ s \leq T$

shows

wp-event verify-monad

$(\text{trace-fri-header-tied-candidate-available-gap } s) \ s \leq T$

proof –

have *wp-event verify-monad*

$(\text{trace-fri-header-tied-candidate-available-gap } s) \ s \leq$

wp-event verify-monad

$(\text{trace-fri-header-tied-low-candidate-gap } s) \ s$

by (rule *wp-event-mono*)

(rule *trace-fri-header-tied-candidate-available-gap-imp-low-candidate-gap*)

then show *?thesis*

by (rule *order-trans[OF - low-gap-bound]*)

qed

lemma *wp-trace-fri-reachable-header-tie-gap-bound-from-merkle-and-low-candidate-gap:*

assumes *merkle-bound:*

wp-event verify-monad $(\text{partial-merkle-inconsistency-bad } s) \ s \leq M$

and *low-gap-bound:*

wp-event verify-monad

$(\text{trace-fri-header-tied-low-candidate-gap } s) \ s \leq T$

shows

wp-event verify-monad

$(\text{trace-fri-reachable-header-tie-gap } s) \ s \leq M + T$

proof –

have *split-bound:*

wp-event verify-monad

$(\text{trace-fri-reachable-header-tie-gap } s) \ s \leq$

wp-event verify-monad

$(\lambda \text{out. } \text{partial-merkle-inconsistency-bad } s \ \text{out} \ \vee$

$\text{trace-fri-header-tied-candidate-available-gap } s \ \text{out}) \ s$

by (rule *wp-event-mono-on-support*)

(rule

trace-fri-reachable-header-tie-gap-imp-merkle-or-candidate-available-on-support)

also have $\dots \leq$

wp-event verify-monad $(\text{partial-merkle-inconsistency-bad } s) \ s +$

wp-event verify-monad

$(\text{trace-fri-header-tied-candidate-available-gap } s) \ s$

by (rule *wp-event-union-bound*)

also have $\dots \leq M + T$

by (rule *add-mono[OF merkle-bound]*)

(rule

wp-trace-fri-header-tied-candidate-available-gap-bound-from-low-candidate-gap

[OF low-gap-bound])

finally show *?thesis* .

qed

lemma *trace-fri-reachable-partial-candidate-reduction-from-header-merkle-and-low-candidate-gap:*

assumes *header-bound:*

wp-event verify-monad

(trace-fri-bad-with-header-tied-partial-candidate s) s ≤ Ht

and *merkle-bound:*

wp-event verify-monad (partial-merkle-inconsistency-bad s) s ≤ Mt

and *low-gap-bound:*

wp-event verify-monad

(trace-fri-header-tied-low-candidate-gap s) s ≤ Tt

and *total-bound: Ht + (Mt + Tt) ≤ trace-fri-error*

shows *trace-fri-reachable-partial-candidate-reduction-assumption s*

proof –

have *gap-bound:*

wp-event verify-monad

(trace-fri-reachable-header-tie-gap s) s ≤ Mt + Tt

by (rule

wp-trace-fri-reachable-header-tie-gap-bound-from-merkle-and-low-candidate-gap

[OF merkle-bound low-gap-bound])

have *reachable-bound:*

wp-event verify-monad

(trace-fri-bad-with-reachable-partial-candidate s) s ≤

Ht + (Mt + Tt)

by (rule *wp-trace-fri-reachable-bound-from-header-tied-and-gap*

[OF header-bound gap-bound])

show *?thesis*

unfolding *trace-fri-reachable-partial-candidate-reduction-assumption-def*

by (rule *order-trans*[OF reachable-bound total-bound])

qed

lemma *composition-fri-verifier-tied-candidate-transfer-gapD:*

assumes *composition-fri-verifier-tied-candidate-transfer-gap s out*

shows

composition-fri-bad-with-reachable-partial-candidate s out

composition-fri-verifier-tied-misaligned-candidate-classification s out

proof –

from *assms* **obtain** *trace-roots trace-bs trace-final fri-dg*

composition-roots composition-bs composition-final fri-query-idxs

trace-round-layers composition-round-layers fr as trace-openings

composition-openings trace-table composition-table **where**

bad: composition-fri-bad-with-reachable-partial-candidate s out

and *fri-openings:*

accepted-fri-opening-transcript s out trace-roots trace-bs

trace-final fri-dg composition-roots composition-bs

composition-final fri-query-idxs trace-round-layers

composition-round-layers

and *aligned:*

accepted-with-partial-initial-openings-aligned s out fr trace-roots

trace-final as fri-dg composition-roots composition-final

```

    fri-query-idxs trace-openings composition-openings
  and trace-candidate:
    partial-trace-table-candidate trace-table trace-openings
  and composition-candidate:
    partial-composition-table-candidate composition-table
    composition-openings
  and classified:
    ¬ (trace-table-low-degree trace-table ∧
      ¬ composition-table-low-degree maxDegree composition-table)
  unfolding composition-fri-verifier-tied-candidate-transfer-gap-def
  by blast
show composition-fri-bad-with-reachable-partial-candidate s out
  by (rule bad)
show
  composition-fri-verifier-tied-misaligned-candidate-classification s out
  unfolding
    composition-fri-verifier-tied-misaligned-candidate-classification-def
  by (intro exI conjI)
    (rule fri-openings, rule aligned, rule trace-candidate,
     rule composition-candidate, rule classified)
qed

lemma composition-fri-verifier-tied-candidate-transfer-gap-imp-accepted:
  assumes composition-fri-verifier-tied-candidate-transfer-gap s out
  shows accepted out
  by (rule composition-fri-bad-with-reachable-partial-candidate-imp-accepted)
    (rule composition-fri-verifier-tied-candidate-transfer-gapD(1)[OF assms])

lemma wp-composition-fri-verifier-tied-candidate-transfer-gap-bound-from-misaligned-classification:
  assumes classification-bound:
    wp-event verify-monad
      (composition-fri-verifier-tied-misaligned-candidate-classification s)
    s ≤ T
  shows
    wp-event verify-monad
      (composition-fri-verifier-tied-candidate-transfer-gap s) s ≤ T
  proof -
    have wp-event verify-monad
      (composition-fri-verifier-tied-candidate-transfer-gap s) s ≤
      wp-event verify-monad
        (composition-fri-verifier-tied-misaligned-candidate-classification s)
      s
    by (rule wp-event-mono)
      (rule composition-fri-verifier-tied-candidate-transfer-gapD(2))
    then show ?thesis
      by (rule order-trans[OF - classification-bound])
  qed

lemma composition-fri-verifier-tied-transcript-consistent-misaligned-classification-imp-misaligned-classification:

```



```

assumes
  composition-fri-verifier-tied-transcript-consistent-misaligned-classification
  s out
shows composition-fri-verifier-tied-misaligned-candidate-classification
  s out
proof —
from assms obtain trace-roots trace-bs trace-final fri-dg
  composition-roots composition-bs composition-final fri-query-idxs
  trace-round-layers composition-round-layers fr as rest trace-openings
  composition-openings trace-table composition-table where
  fri-openings:
    accepted-fri-opening-transcript s out trace-roots trace-bs
    trace-final fri-dg composition-roots composition-bs
    composition-final fri-query-idxs trace-round-layers
    composition-round-layers
and partial:
  accepted-with-partial-initial-openings-aligned-transcript-consistent s
  out fr trace-roots trace-final as fri-dg composition-roots
  composition-final fri-query-idxs trace-openings composition-openings
and trace-candidate:
  partial-trace-table-candidate trace-table trace-openings
and composition-candidate:
  partial-composition-table-candidate composition-table
  composition-openings
and classified:
   $\neg$  (trace-table-low-degree trace-table  $\wedge$ 
     $\neg$  composition-table-low-degree maxDegree composition-table)
unfolding
  composition-fri-verifier-tied-transcript-consistent-misaligned-classification-def
by blast
have aligned:
  accepted-with-partial-initial-openings-aligned s out fr trace-roots
  trace-final as fri-dg composition-roots composition-final fri-query-idxs
  trace-openings composition-openings
using partial
by (blast dest:
  accepted-with-partial-initial-openings-aligned-transcript-consistent-imp-aligned
  accepted-with-partial-initial-openings-aligned-consistent-imp-aligned)
show ?thesis
unfolding
  composition-fri-verifier-tied-misaligned-candidate-classification-def
by (intro exI conjI)
  (rule fri-openings, rule aligned, rule trace-candidate,
  rule composition-candidate, rule classified)
qed

lemma composition-fri-verifier-tied-transcript-consistent-misaligned-classification-imp-aligned-bad-event:
assumes false-statement:  $\neg$  exists-valid-trace
and classification:

```

```

    composition-fri-verifier-tied-transcript-consistent-misaligned-classification
    s out
shows
    soundness-bad-event-verifier-tied-aligned-transcript-partial-candidate
    s out
proof –
from classification obtain trace-roots trace-bs trace-final fri-dg
    composition-roots composition-bs composition-final fri-query-idxs
    trace-round-layers composition-round-layers fr as rest trace-openings
    composition-openings trace-table composition-table where
    fri-openings:
    accepted-fri-opening-transcript s out trace-roots trace-bs
    trace-final fri-dg composition-roots composition-bs
    composition-final fri-query-idxs trace-round-layers
    composition-round-layers
and header:
    verifier-header-transcript s fr trace-roots trace-final as fri-dg
    composition-roots composition-final rest
and partial:
    accepted-with-partial-initial-openings-aligned-transcript-consistent s
    out fr trace-roots trace-final as fri-dg composition-roots
    composition-final fri-query-idxs trace-openings composition-openings
and trace-candidate:
    partial-trace-table-candidate trace-table trace-openings
and composition-candidate:
    partial-composition-table-candidate composition-table
    composition-openings
unfolding
    composition-fri-verifier-tied-transcript-consistent-misaligned-classification-def
by blast
show ?thesis
by (rule
    accepted-verifier-tied-aligned-transcript-partial-candidate-partition
    [OF false-statement fri-openings partial header trace-candidate
    composition-candidate])
qed

lemma wp-composition-fri-verifier-tied-transcript-consistent-misaligned-classification-bound-from-aligned-bad-event:
assumes false-statement:  $\neg$  exists-valid-trace
and aligned-bound:
    wp-event verify-monad
    (soundness-bad-event-verifier-tied-aligned-transcript-partial-candidate
    s) s  $\leq$  T
shows
    wp-event verify-monad
    (composition-fri-verifier-tied-transcript-consistent-misaligned-classification
    s) s  $\leq$  T
proof –
have wp-event verify-monad

```

```

    (composition-fri-verifier-tied-transcript-consistent-misaligned-classification
      s) s ≤
wp-event verify-monad
  (soundness-bad-event-verifier-tied-aligned-transcript-partial-candidate
    s) s
by (rule wp-event-mono)
  (rule
    composition-fri-verifier-tied-transcript-consistent-misaligned-classification-imp-aligned-bad-event
    [OF false-statement])
then show ?thesis
  by (rule order-trans[OF - aligned-bound])
qed

```

lemma *composition-fri-verifier-tied-transcript-consistent-misaligned-classification-imp-cover*:

assumes *false-statement*: $\neg \text{exists-valid-trace}$

and *classification*:

composition-fri-verifier-tied-transcript-consistent-misaligned-classification
s out

shows

composition-fri-verifier-tied-transcript-consistent-classification-cover
s out

proof —

from *classification* **obtain** *trace-roots trace-bs trace-final fri-dg*

composition-roots composition-bs composition-final fri-query-idxs

trace-round-layers composition-round-layers fr as rest trace-openings

composition-openings trace-table composition-table **where**

fri-openings:

accepted-fri-opening-transcript s out trace-roots trace-bs

trace-final fri-dg composition-roots composition-bs

composition-final fri-query-idxs trace-round-layers

composition-round-layers

and *header*:

verifier-header-transcript s fr trace-roots trace-final as fri-dg

composition-roots composition-final rest

and *partial*:

accepted-with-partial-initial-openings-aligned-transcript-consistent s

out fr trace-roots trace-final as fri-dg composition-roots

composition-final fri-query-idxs trace-openings composition-openings

and *trace-candidate*:

partial-trace-table-candidate trace-table trace-openings

and *composition-candidate*:

partial-composition-table-candidate composition-table

composition-openings

and *classified*:

$\neg (\text{trace-table-low-degree trace-table} \wedge$

$\neg \text{composition-table-low-degree maxDegree composition-table})$

unfolding

composition-fri-verifier-tied-transcript-consistent-misaligned-classification-def

by *blast*

```

show ?thesis
proof (cases trace-table-low-degree trace-table)
  case False
  have aligned:
    accepted-with-partial-initial-openings-aligned-consistent s out fr
      trace-roots trace-final as fri-dg composition-roots
      composition-final fri-query-idxs trace-openings composition-openings
  by (rule
    accepted-with-partial-initial-openings-aligned-transcript-consistent-imp-aligned
    [OF partial])
  have aligned0:
    accepted-with-partial-initial-openings-aligned s out fr trace-roots
      trace-final as fri-dg composition-roots composition-final
      fri-query-idxs trace-openings composition-openings
  by (rule
    accepted-with-partial-initial-openings-aligned-consistent-imp-aligned
    [OF aligned])
  have trace-partial:
    accepted-with-partial-trace-openings s out fr fri-query-idxs
      trace-openings
  by (rule accepted-with-partial-initial-openings-aligned-shapes(3)
    [OF aligned0])
  have trace-fri-bad-with-header-tied-partial-candidate s out
    unfolding trace-fri-bad-with-header-tied-partial-candidate-def
  by (intro exI conjI)
    (rule fri-openings, rule header, rule trace-partial,
      rule trace-candidate, rule False)
  then show ?thesis
    unfolding
      composition-fri-verifier-tied-transcript-consistent-classification-cover-def
    by blast
next
  case True
  note trace-low = True
  have composition-low:
    composition-table-low-degree maxDegree composition-table
    using classified trace-low by blast
  show ?thesis
  proof (cases all-queries-consistent trace-table composition-table as)
    case False
    have query-bad-with-aligned-transcript-partial-candidates s out
      unfolding query-bad-with-aligned-transcript-partial-candidates-def
      using partial trace-candidate composition-candidate trace-low
      composition-low False
    by blast
    then show ?thesis
      unfolding
        composition-fri-verifier-tied-transcript-consistent-classification-cover-def
      by blast

```

```

next
  case True
  note all-queries = True
  from trace-low obtain f where
    deg-f: degree f < clength
    and trace-table-eq: trace-table = map (poly f) eval-domain
    unfolding trace-table-low-degree-def by blast
  have violated: violated-constraints f ≠ {}
    by (rule false-statement-violated-constraints
      [OF false-statement deg-f])
  have aligned:
    accepted-with-partial-initial-openings-aligned-consistent s out fr
    trace-roots trace-final as fri-dg composition-roots
    composition-final fri-query-idxs trace-openings
    composition-openings
  by (rule
    accepted-with-partial-initial-openings-aligned-transcript-consistent-imp-aligned
    [OF partial])
  have composition-bad-with-partial-candidates s out
    unfolding composition-bad-with-partial-candidates-def
    using accepted-with-partial-initial-openings-aligned-consistent-imp-unaligned
    [OF aligned]
    trace-candidate composition-candidate deg-f trace-table-eq
    violated composition-low all-queries
  by blast
  then show ?thesis
    unfolding
      composition-fri-verifier-tied-transcript-consistent-classification-cover-def
    by blast
qed
qed
qed

lemma wp-composition-fri-verifier-tied-transcript-consistent-classification-cover-union-bound:
  assumes comp-bound:
    wp-event verify-monad
    (composition-bad-with-partial-candidates s) s ≤ C
  and trace-bound:
    wp-event verify-monad
    (trace-fri-bad-with-header-tied-partial-candidate s) s ≤ Ft
  and query-bound:
    wp-event verify-monad
    (query-bad-with-aligned-transcript-partial-candidates s) s ≤ Q
  shows
    wp-event verify-monad
    (composition-fri-verifier-tied-transcript-consistent-classification-cover
    s) s ≤ C + Ft + Q
proof -
  have wp-event verify-monad

```

```

    (composition-fri-verifier-tied-transcript-consistent-classification-cover
      s) s ≤
  wp-event verify-monad
    (composition-bad-with-partial-candidates s) s +
  wp-event verify-monad
    (trace-fri-bad-with-header-tied-partial-candidate s) s +
  wp-event verify-monad
    (query-bad-with-aligned-transcript-partial-candidates s) s
proof -
  have wp-event verify-monad
    (composition-fri-verifier-tied-transcript-consistent-classification-cover
      s) s ≤
  wp-event verify-monad
    (composition-bad-with-partial-candidates s) s +
  wp-event verify-monad
    (λout.
      trace-fri-bad-with-header-tied-partial-candidate s out ∨
      query-bad-with-aligned-transcript-partial-candidates s out) s
  unfolding
    composition-fri-verifier-tied-transcript-consistent-classification-cover-def
  by (rule wp-event-union-bound)
also have ... ≤
  wp-event verify-monad
    (composition-bad-with-partial-candidates s) s +
  (wp-event verify-monad
    (trace-fri-bad-with-header-tied-partial-candidate s) s +
  wp-event verify-monad
    (query-bad-with-aligned-transcript-partial-candidates s) s)
  by (rule add-mono[OF order-refl wp-event-union-bound])
  finally show ?thesis by (simp add: algebra-simps)
qed
also have ... ≤ C + Ft + Q
  by (intro add-mono comp-bound trace-bound query-bound)
  finally show ?thesis .
qed

lemma wp-composition-fri-verifier-tied-transcript-consistent-misaligned-classification-bound-from-cover-components:
assumes false-statement: ¬ exists-valid-trace
and comp-bound:
  wp-event verify-monad
    (composition-bad-with-partial-candidates s) s ≤ C
and trace-bound:
  wp-event verify-monad
    (trace-fri-bad-with-header-tied-partial-candidate s) s ≤ Ft
and query-bound:
  wp-event verify-monad
    (query-bad-with-aligned-transcript-partial-candidates s) s ≤ Q
shows
  wp-event verify-monad

```

(*composition-fri-verifier-tied-transcript-consistent-misaligned-classification*
s) $s \leq C + Ft + Q$
proof –
have *wp-event verify-monad*
 (*composition-fri-verifier-tied-transcript-consistent-misaligned-classification*
s) $s \leq$
wp-event verify-monad
 (*composition-fri-verifier-tied-transcript-consistent-classification-cover*
s) s
by (*rule wp-event-mono*)
 (*rule*
composition-fri-verifier-tied-transcript-consistent-misaligned-classification-imp-cover
 [*OF false-statement*])
also have $\dots \leq C + Ft + Q$
by (*rule*
wp-composition-fri-verifier-tied-transcript-consistent-classification-cover-union-bound
 [*OF comp-bound trace-bound query-bound*])
finally show *?thesis* .
qed

lemma *composition-fri-verifier-tied-candidate-available-gap-imp-empty-or-merkle-or-transcript-consistent-classification*
assumes *available*:

composition-fri-verifier-tied-candidate-available-gap s out

shows

composition-fri-reachable-verifier-tie-empty-header-gap s out $\vee$

partial-merkle-inconsistency-bad s out $\vee$

composition-fri-verifier-tied-transcript-consistent-misaligned-classification

s out

proof –

from *available* **obtain** *trace-roots trace-bs trace-final fri-dg*
composition-roots composition-bs composition-final fri-query-idxs
trace-round-layers composition-round-layers fr as trace-openings
composition-openings trace-table composition-table **where**

gap: composition-fri-reachable-verifier-tie-gap s out

and *fri-openings*:

accepted-fri-opening-transcript s out trace-roots trace-bs

trace-final fri-dg composition-roots composition-bs

composition-final fri-query-idxs trace-round-layers

composition-round-layers

and *aligned*:

accepted-with-partial-initial-openings-aligned s out fr trace-roots

trace-final as fri-dg composition-roots composition-final

fri-query-idxs trace-openings composition-openings

and *trace-candidate*:

partial-trace-table-candidate trace-table trace-openings

and *composition-candidate*:

partial-composition-table-candidate composition-table

composition-openings

unfolding *composition-fri-verifier-tied-candidate-available-gap-def*

```

    by blast
  from fri-openings obtain result final-state where out-eq:
    out = Some (result, final-state)
    unfolding accepted-fri-opening-transcript-def by blast
  show ?thesis
  proof (cases composition-roots = [])
    case True
    have composition-fri-reachable-verifier-tie-empty-header-gap s out
      by (rule composition-fri-reachable-verifier-tie-empty-header-gapI
        [OF gap fri-openings True])
    then show ?thesis by simp
  next
    case comp-nonempty: False
    show ?thesis
    proof (cases partial-merkle-inconsistency-bad s out)
      case True
      then show ?thesis by simp
    next
      case no-merkle: False
      from accepted-fri-opening-transcript-obtains-aligned-partial-openings
        [OF fri-openings[unfolded out-eq] comp-nonempty]
      obtain fr' as' rest' trace-openings' composition-openings'
      where header':
        verifier-header-transcript s fr' trace-roots trace-final as' fri-dg
        composition-roots composition-final rest'
      and partial':
        accepted-with-partial-initial-openings-aligned-transcript-consistent s
        (Some (result, final-state)) fr' trace-roots trace-final as'
        fri-dg composition-roots composition-final fri-query-idxs
        trace-openings' composition-openings'
      by blast
      have aligned':
        accepted-with-partial-initial-openings-aligned s out fr'
        trace-roots trace-final as' fri-dg composition-roots
        composition-final fri-query-idxs trace-openings'
        composition-openings'
      using partial'[folded out-eq]
      by (blast dest:
        accepted-with-partial-initial-openings-aligned-transcript-consistent-imp-aligned
        accepted-with-partial-initial-openings-aligned-consistent-imp-aligned)
      obtain trace-table' composition-table'
      where trace-candidate':
        partial-trace-table-candidate trace-table' trace-openings'
      and composition-candidate':
        partial-composition-table-candidate composition-table'
        composition-openings'
      using
        accepted-with-partial-initial-openings-aligned-candidates-if-no-partial-merkle-bad
        [OF aligned[unfolded out-eq] no-merkle[unfolded out-eq]]

```



```

    by blast
  have classified:
     $\neg$  (trace-table-low-degree trace-table'  $\wedge$ 
       $\neg$  composition-table-low-degree maxDegree composition-table')
  proof
    assume bad-class:
      trace-table-low-degree trace-table'  $\wedge$ 
       $\neg$  composition-table-low-degree maxDegree composition-table'
    have composition-fri-bad-with-verifier-tied-partial-candidate s out
    unfolding composition-fri-bad-with-verifier-tied-partial-candidate-def
    by (intro exI conjI)
      (rule fri-openings, rule aligned', rule trace-candidate',
       rule composition-candidate', rule bad-class[THEN conjunct1],
       rule bad-class[THEN conjunct2])
    then show False
      using gap
      unfolding composition-fri-reachable-verifier-tie-gap-def by simp
  qed
  have
    composition-fri-verifier-tied-transcript-consistent-misaligned-classification
      s out
    unfolding
      composition-fri-verifier-tied-transcript-consistent-misaligned-classification-def
    by (intro exI conjI)
      (rule fri-openings, rule header'[folded out-eq],
       rule partial'[folded out-eq], rule trace-candidate',
       rule composition-candidate', rule classified)
    then show ?thesis by simp
  qed
  qed
  qed

```

lemma wp-composition-fri-verifier-tied-candidate-available-gap-bound-from-empty-merkle-and-transcript-consistent-c

assumes empty-bound:

wp-event verify-monad

(composition-fri-reachable-verifier-tie-empty-header-gap s) $s \leq E$

and merkle-bound:

wp-event verify-monad (partial-merkle-inconsistency-bad s) $s \leq M$

and classification-bound:

wp-event verify-monad

(composition-fri-verifier-tied-transcript-consistent-misaligned-classification
s) $s \leq T$

shows

wp-event verify-monad

(composition-fri-verifier-tied-candidate-available-gap s) $s \leq$
 $E + (M + T)$

proof –

have wp-event verify-monad

(composition-fri-verifier-tied-candidate-available-gap s) $s \leq$

```

wp-event verify-monad
  (λout.
    composition-fri-reachable-verifier-tie-empty-header-gap s out ∨
    partial-merkle-inconsistency-bad s out ∨
    composition-fri-verifier-tied-transcript-consistent-misaligned-classification
      s out) s
by (rule wp-event-mono)
  (rule
    composition-fri-verifier-tied-candidate-available-gap-imp-empty-or-merkle-or-transcript-consistent-classification)
also have ... ≤
  wp-event verify-monad
    (composition-fri-reachable-verifier-tie-empty-header-gap s) s +
  wp-event verify-monad
    (λout.
      partial-merkle-inconsistency-bad s out ∨
      composition-fri-verifier-tied-transcript-consistent-misaligned-classification
        s out) s
  by (rule wp-event-union-bound)
also have ... ≤
  wp-event verify-monad
    (composition-fri-reachable-verifier-tie-empty-header-gap s) s +
  (wp-event verify-monad (partial-merkle-inconsistency-bad s) s) +
  wp-event verify-monad
    (composition-fri-verifier-tied-transcript-consistent-misaligned-classification
      s) s
  by (rule add-mono[OF order-refl wp-event-union-bound])
also have ... ≤ E + (M + T)
  by (intro add-mono empty-bound merkle-bound classification-bound)
finally show ?thesis .
qed

lemma wp-composition-fri-reachable-verifier-tie-gap-bound-from-empty-merkle-and-candidate-available:
  assumes empty-bound:
    wp-event verify-monad
      (composition-fri-reachable-verifier-tie-empty-header-gap s) s ≤ E
  and merkle-bound:
    wp-event verify-monad (partial-merkle-inconsistency-bad s) s ≤ M
  and available-bound:
    wp-event verify-monad
      (composition-fri-verifier-tied-candidate-available-gap s) s ≤ T
  shows
    wp-event verify-monad
      (composition-fri-reachable-verifier-tie-gap s) s ≤ E + (M + T)
proof -
  have split-bound:
    wp-event verify-monad
      (composition-fri-reachable-verifier-tie-gap s) s ≤
    wp-event verify-monad
      (λout.

```

```

      composition-fri-reachable-verifier-tie-empty-header-gap s out ∨
      partial-merkle-inconsistency-bad s out ∨
      composition-fri-verifier-tied-candidate-available-gap s out) s
  by (rule wp-event-mono-on-support)
    (rule
      composition-fri-reachable-verifier-tie-gap-imp-empty-or-merkle-or-candidate-available-on-support)
  also have ... ≤
    wp-event verify-monad
      (composition-fri-reachable-verifier-tie-empty-header-gap s) s +
    wp-event verify-monad
      (λout.
        partial-merkle-inconsistency-bad s out ∨
        composition-fri-verifier-tied-candidate-available-gap s out) s
  by (rule wp-event-union-bound)
  also have ... ≤
    wp-event verify-monad
      (composition-fri-reachable-verifier-tie-empty-header-gap s) s +
    (wp-event verify-monad (partial-merkle-inconsistency-bad s) s) +
    wp-event verify-monad
      (composition-fri-verifier-tied-candidate-available-gap s) s
  by (rule add-mono[OF order_refl wp-event-union-bound])
  also have ... ≤ E + (M + T)
  by (intro add-mono empty-bound merkle-bound available-bound)
  finally show ?thesis .
qed

lemma composition-fri-reachable-partial-candidate-reduction-from-split-residual-bounds-with-candidate-available:
  fixes R A P M N S G E Q T :: prob
  assumes sampled-bound:
    wp-event verify-monad
      (composition-fri-bad-with-verifier-tied-sampled-layer-chain s) s ≤ R
  and slot-gap-bound:
    wp-event verify-monad
      (composition-fri-verifier-tied-authenticated-slot-recorded-chunk-gap
        s) s ≤ A
  and verifier-partial-merkle-bound:
    wp-event verify-monad (partial-merkle-inconsistency-bad s) s ≤ P
  and missing-bound:
    wp-event verify-monad
      (composition-fri-verifier-tied-recorded-base-chunk-auth-missing s)
      s ≤ M
  and next-gap-bound:
    wp-event verify-monad
      (composition-fri-verifier-tied-next-value-replay-gap s) s ≤ N
  and successor-gap-bound:
    wp-event verify-monad
      (composition-fri-verifier-tied-successor-replay-gap s) s ≤ S
  and final-gap-bound:
    wp-event verify-monad

```

(*composition-fri-verifier-tied-final-value-replay-gap* s) $s \leq G$
and *empty-bound*:
wp-event verify-monad
 (*composition-fri-reachable-verifier-tie-empty-header-gap* s) $s \leq E$
and *gap-partial-merkle-bound*:
wp-event verify-monad (*partial-merkle-inconsistency-bad* s) $s \leq Q$
and *candidate-available-bound*:
wp-event verify-monad
 (*composition-fri-verifier-tied-candidate-available-gap* s) $s \leq T$
and *total-bound*:
 $R + (((P + M) + N + S + G + P) + A) + (E + (Q + T))$
 $\leq \text{composition-fri-error}$
shows *composition-fri-reachable-partial-candidate-reduction-assumption* s
proof –
have *verifier-bound*:
wp-event verify-monad
 (*composition-fri-bad-with-verifier-tied-partial-candidate* s) s
 $\leq R + (((P + M) + N + S + G + P) + A)$
by (*rule*
wp-composition-fri-verifier-tied-bound-from-sampled-slot-recorded-chunk-gap-and-residuals
 [*OF sampled-bound slot-gap-bound verifier-partial-merkle-bound*
missing-bound next-gap-bound successor-gap-bound final-gap-bound])
have *gap-bound*:
wp-event verify-monad
 (*composition-fri-reachable-verifier-tie-gap* s) $s \leq$
 $E + (Q + T)$
by (*rule*
wp-composition-fri-reachable-verifier-tie-gap-bound-from-empty-merkle-and-candidate-available
 [*OF empty-bound gap-partial-merkle-bound candidate-available-bound*])
show *?thesis*
by (*rule*
composition-fri-reachable-partial-candidate-reduction-from-verifier-tied-and-gap
 [*OF verifier-bound gap-bound total-bound*])
qed

lemma *composition-fri-reachable-partial-candidate-reduction-from-split-residual-bounds-with-candidate-available-and-*

fixes $R A P N S G E Q T :: \text{prob}$

assumes *sampled-bound*:

wp-event verify-monad

(*composition-fri-bad-with-verifier-tied-sampled-layer-chain* s) $s \leq R$

and *slot-gap-bound*:

wp-event verify-monad

(*composition-fri-verifier-tied-authenticated-slot-recorded-chunk-gap*
 s) $s \leq A$

and *verifier-partial-merkle-bound*:

wp-event verify-monad (*partial-merkle-inconsistency-bad* s) $s \leq P$

and *next-gap-bound*:

wp-event verify-monad

(*composition-fri-verifier-tied-next-value-replay-gap* s) $s \leq N$

```

and successor-gap-bound:
  wp-event verify-monad
    (composition-fri-verifier-tied-successor-replay-gap s)  $s \leq S$ 
and final-gap-bound:
  wp-event verify-monad
    (composition-fri-verifier-tied-final-value-replay-gap s)  $s \leq G$ 
and empty-bound:
  wp-event verify-monad
    (composition-fri-reachable-verifier-tie-empty-header-gap s)  $s \leq E$ 
and gap-partial-merkle-bound:
  wp-event verify-monad (partial-merkle-inconsistency-bad s)  $s \leq Q$ 
and candidate-available-bound:
  wp-event verify-monad
    (composition-fri-verifier-tied-candidate-available-gap s)  $s \leq T$ 
and total-bound:
   $R + (((P + 0) + N + S + G + P) + A) + (E + (Q + T))$ 
     $\leq \text{composition-fri-error}$ 
shows composition-fri-reachable-partial-candidate-reduction-assumption s
by (rule
  composition-fri-reachable-partial-candidate-reduction-from-split-residual-bounds-with-candidate-available
    [where  $M = 0$ ])
  (rule sampled-bound,
   rule slot-gap-bound,
   rule verifier-partial-merkle-bound,
   rule wp-composition-fri-verifier-tied-recorded-base-chunk-auth-missing-zero,
   rule next-gap-bound,
   rule successor-gap-bound,
   rule final-gap-bound,
   rule empty-bound,
   rule gap-partial-merkle-bound,
   rule candidate-available-bound,
   rule total-bound)

end

end

theory Soundness-FRI-Empty-Header-Reduction
  imports Soundness-FRI-Trace-Residuals
begin

  Named reduction interface for the empty-composition-header trace FRI branch. This
  keeps the large reachable-FRI theory stable while giving the final route the same active-
  bound shape for all three FRI components.

  context soundness
begin

  definition trace-fri-empty-header-reduction

```

```

:: ('f, 'a) protocol-channel-scheme  $\Rightarrow$  bool
where
  trace-fri-empty-header-reduction s  $\longleftrightarrow$ 
    wp-event verify-monad
      (trace-fri-bad-with-empty-composition-header-candidates s) s  $\leq$ 
      trace-fri-error

lemma trace-fri-empty-header-reductionD:
  assumes trace-fri-empty-header-reduction s
  shows
    wp-event verify-monad
      (trace-fri-bad-with-empty-composition-header-candidates s) s  $\leq$ 
      trace-fri-error
  using assms unfolding trace-fri-empty-header-reduction-def by simp

lemma trace-fri-empty-header-reduction-staged-bound:
  assumes reductions:
     $\bigwedge$ data attacker-state.
      Some (data, attacker-state)  $\in$ 
        set-dist (execute (checked-staged-transcript-program A)
          adversary-initial-state)  $\implies$ 
        trace-fri-empty-header-reduction
          (verifier-state-from-adversary attacker-state
            (staged-proof-transcript data))
  shows
    wp-event (checked-staged-security-experiment-with-data-state A)
      (staged-security-with-data-state-verifier-event
        trace-fri-bad-with-empty-composition-header-candidates)
      adversary-initial-state  $\leq$  trace-fri-error
  by (rule checked-staged-security-with-data-state-verifier-event-bound-from-cont)
    (simp-all add: trace-fri-bad-with-empty-composition-header-candidates-not-None
      trace-fri-empty-header-reductionD reductions)

lemma trace-fri-empty-header-reduction-from-reachable-reduction:
  assumes trace-fri-reachable-partial-candidate-reduction-assumption s
  shows trace-fri-empty-header-reduction s
proof -
  have reachable:
    wp-event verify-monad
      (trace-fri-bad-with-reachable-partial-candidate s) s  $\leq$ 
      trace-fri-error
  using assms
  unfolding trace-fri-reachable-partial-candidate-reduction-assumption-def
  by simp
show ?thesis
  unfolding trace-fri-empty-header-reduction-def
  by (rule order-trans[OF - reachable])
    (rule wp-event-mono,
      rule

```

```

    trace-fri-bad-with-empty-composition-header-candidates-imp-reachable-partial-candidate)
qed
end
end

```

```

theory Soundness-FRI-Active-Reductions
imports
  Soundness-FRI-Candidate-Transfer-Support
  Soundness-FRI-Empty-Header-Reduction
begin

```

Staged packaging for the active FRI reductions used by the public route. This layer does not add assumptions; it names the exact verifier-local reductions consumed by the public-route bounds.

Internal symbolic FRI soundness interface for the public staged theorem.

The trace side is intentionally stated in terms of the verifier-consumed header-tied branch and the empty-composition-header branch. The composition side is stated in terms of the verifier-tied branch consumed by the active public route.

```

locale soundness-fri = soundness +
assumes trace-fri-active-reductions:
   $\bigwedge A$  data attacker-state.
  Some (data, attacker-state)  $\in$ 
    set-dist (execute (checked-staged-transcript-program A)
      adversary-initial-state)  $\implies$ 
    trace-fri-header-tied-reduction
      (verifier-state-from-adversary attacker-state
        (staged-proof-transcript data))  $\wedge$ 
    trace-fri-empty-header-reduction
      (verifier-state-from-adversary attacker-state
        (staged-proof-transcript data))
assumes composition-fri-active-reduction:
   $\bigwedge A$  data attacker-state.
  Some (data, attacker-state)  $\in$ 
    set-dist (execute (checked-staged-transcript-program A)
      adversary-initial-state)  $\implies$ 
    composition-fri-verifier-tied-reduction
      (verifier-state-from-adversary attacker-state
        (staged-proof-transcript data))

context soundness
begin

```

```

lemma active-composition-candidate-transfer-bound-from-classification-bound:
assumes classification-bound:
   $\bigwedge$  data attacker-state.

```

Some (data, attacker-state) ∈
set-dist (execute (checked-staged-transcript-program A)
adversary-initial-state) ⇒
wp-event verify-monad
(composition-fri-verifier-tied-misaligned-candidate-classification
(verifier-state-from-adversary attacker-state
(staged-proof-transcript data)))
(verifier-state-from-adversary attacker-state
(staged-proof-transcript data)) ≤
T data attacker-state
and support:
Some (data, attacker-state) ∈
set-dist (execute (checked-staged-transcript-program A)
adversary-initial-state)
shows
wp-event verify-monad
(composition-fri-verifier-tied-candidate-transfer-gap
(verifier-state-from-adversary attacker-state
(staged-proof-transcript data)))
(verifier-state-from-adversary attacker-state
(staged-proof-transcript data)) ≤
T data attacker-state
by (rule
wp-composition-fri-verifier-tied-candidate-transfer-gap-bound-from-misaligned-classification)
(rule classification-bound[OF support])

lemma *active-composition-candidate-available-bound-from-transcript-consistent-classification-bound:*

assumes *empty-bound:*

∧ data attacker-state.

Some (data, attacker-state) ∈
set-dist (execute (checked-staged-transcript-program A)
adversary-initial-state) ⇒

wp-event verify-monad
(composition-fri-reachable-verifier-tie-empty-header-gap
(verifier-state-from-adversary attacker-state
(staged-proof-transcript data)))
(verifier-state-from-adversary attacker-state
(staged-proof-transcript data)) ≤
E data attacker-state

and merkle-bound:

∧ data attacker-state.

Some (data, attacker-state) ∈
set-dist (execute (checked-staged-transcript-program A)
adversary-initial-state) ⇒

wp-event verify-monad
(partial-merkle-inconsistency-bad
(verifier-state-from-adversary attacker-state
(staged-proof-transcript data)))
(verifier-state-from-adversary attacker-state

$(\text{staged-proof-transcript data}) \leq$
 $M \text{ data attacker-state}$
and *classification-bound*:
 $\wedge \text{data attacker-state.}$
 $\text{Some (data, attacker-state)} \in$
 $\text{set-dist (execute (checked-staged-transcript-program } A)$
 $\text{adversary-initial-state)} \implies$
 $\text{wp-event verify-monad}$
 $(\text{composition-fri-verifier-tied-transcript-consistent-misaligned-classification}$
 $(\text{verifier-state-from-adversary attacker-state}$
 $(\text{staged-proof-transcript data})))$
 $(\text{verifier-state-from-adversary attacker-state}$
 $(\text{staged-proof-transcript data})) \leq$
 $T \text{ data attacker-state}$
and *support*:
 $\text{Some (data, attacker-state)} \in$
 $\text{set-dist (execute (checked-staged-transcript-program } A)$
 $\text{adversary-initial-state)}$
shows
 $\text{wp-event verify-monad}$
 $(\text{composition-fri-verifier-tied-candidate-available-gap}$
 $(\text{verifier-state-from-adversary attacker-state}$
 $(\text{staged-proof-transcript data})))$
 $(\text{verifier-state-from-adversary attacker-state}$
 $(\text{staged-proof-transcript data})) \leq$
 $E \text{ data attacker-state} + (M \text{ data attacker-state} +$
 $T \text{ data attacker-state})$
by (*rule*
 $\text{wp-composition-fri-verifier-tied-candidate-available-gap-bound-from-empty-merkle-and-transcript-consistent-classification}$
 $(\text{rule empty-bound[OF support], rule merkle-bound[OF support],}$
 $\text{rule classification-bound[OF support]})$

lemma *active-composition-transcript-consistent-classification-bound-from-components*:

assumes *false-statement*: $\neg \text{exists-valid-trace}$

and *comp-bound*:

$\wedge \text{data attacker-state.}$

$\text{Some (data, attacker-state)} \in$

$\text{set-dist (execute (checked-staged-transcript-program } A)$

$\text{adversary-initial-state)} \implies$

$\text{wp-event verify-monad}$

$(\text{composition-bad-with-partial-candidates}$

$(\text{verifier-state-from-adversary attacker-state}$

$(\text{staged-proof-transcript data})))$

$(\text{verifier-state-from-adversary attacker-state}$

$(\text{staged-proof-transcript data})) \leq$

$C \text{ data attacker-state}$

and *trace-fri-bound*:

$\wedge \text{data attacker-state.}$

$\text{Some (data, attacker-state)} \in$

$set-dist \ (execute \ (checked-staged-transcript-program \ A) \ adversary-initial-state) \implies$
 $wp-event \ verify-monad$
 $(trace-fri-bad-with-header-tied-partial-candidate$
 $(verifier-state-from-adversary \ attacker-state$
 $(staged-proof-transcript \ data)))$
 $(verifier-state-from-adversary \ attacker-state$
 $(staged-proof-transcript \ data)) \leq$
 $Ft \ data \ attacker-state$
and $query-bound:$
 $\wedge data \ attacker-state.$
 $Some \ (data, \ attacker-state) \in$
 $set-dist \ (execute \ (checked-staged-transcript-program \ A) \ adversary-initial-state) \implies$
 $wp-event \ verify-monad$
 $(query-bad-with-aligned-transcript-partial-candidates$
 $(verifier-state-from-adversary \ attacker-state$
 $(staged-proof-transcript \ data)))$
 $(verifier-state-from-adversary \ attacker-state$
 $(staged-proof-transcript \ data)) \leq$
 $Q \ data \ attacker-state$
and $support:$
 $Some \ (data, \ attacker-state) \in$
 $set-dist \ (execute \ (checked-staged-transcript-program \ A) \ adversary-initial-state)$
shows
 $wp-event \ verify-monad$
 $(composition-fri-verifier-tied-transcript-consistent-misaligned-classification$
 $(verifier-state-from-adversary \ attacker-state$
 $(staged-proof-transcript \ data)))$
 $(verifier-state-from-adversary \ attacker-state$
 $(staged-proof-transcript \ data)) \leq$
 $C \ data \ attacker-state + Ft \ data \ attacker-state +$
 $Q \ data \ attacker-state$
by $(rule$
 $wp-composition-fri-verifier-tied-transcript-consistent-misaligned-classification-bound-from-cover-components$
 $[OF \ false-statement])$
 $(rule \ comp-bound[OF \ support], \ rule \ trace-fri-bound[OF \ support],$
 $rule \ query-bound[OF \ support])$

lemma *active-trace-fri-reachable-reduction-from-header-merkle-and-low-candidate-gap:*

fixes $Ht \ Mt \ Tt ::$

$(\text{'f staged-proof-data} \Rightarrow \text{'f protocol-channel} \Rightarrow prob)$

assumes $trace-header-bound:$

$\wedge data \ attacker-state.$

$Some \ (data, \ attacker-state) \in$

$set-dist \ (execute \ (checked-staged-transcript-program \ A)$

$adversary-initial-state) \implies$

$wp-event \ verify-monad$

$$\begin{aligned}
& (\text{trace-fri-bad-with-header-tied-partial-candidate} \\
& \quad (\text{verifier-state-from-adversary attacker-state} \\
& \quad \quad (\text{staged-proof-transcript data}))) \\
& (\text{verifier-state-from-adversary attacker-state} \\
& \quad (\text{staged-proof-transcript data})) \leq \\
& \text{Ht data attacker-state} \\
\text{and trace-merkle-bound:} \\
& \wedge \text{data attacker-state.} \\
& \text{Some (data, attacker-state)} \in \\
& \quad \text{set-dist (execute (checked-staged-transcript-program A) \\
& \quad \quad \text{adversary-initial-state})} \implies \\
& \text{wp-event verify-monad} \\
& \quad (\text{partial-merkle-inconsistency-bad} \\
& \quad \quad (\text{verifier-state-from-adversary attacker-state} \\
& \quad \quad \quad (\text{staged-proof-transcript data}))) \\
& \quad (\text{verifier-state-from-adversary attacker-state} \\
& \quad \quad (\text{staged-proof-transcript data})) \leq \\
& \text{Mt data attacker-state} \\
\text{and trace-low-gap-bound:} \\
& \wedge \text{data attacker-state.} \\
& \text{Some (data, attacker-state)} \in \\
& \quad \text{set-dist (execute (checked-staged-transcript-program A) \\
& \quad \quad \text{adversary-initial-state})} \implies \\
& \text{wp-event verify-monad} \\
& \quad (\text{trace-fri-header-tied-low-candidate-gap} \\
& \quad \quad (\text{verifier-state-from-adversary attacker-state} \\
& \quad \quad \quad (\text{staged-proof-transcript data}))) \\
& \quad (\text{verifier-state-from-adversary attacker-state} \\
& \quad \quad (\text{staged-proof-transcript data})) \leq \\
& \text{Tt data attacker-state} \\
\text{and total-bound:} \\
& \wedge \text{data attacker-state.} \\
& \text{Some (data, attacker-state)} \in \\
& \quad \text{set-dist (execute (checked-staged-transcript-program A) \\
& \quad \quad \text{adversary-initial-state})} \implies \\
& \text{Ht data attacker-state} + \\
& \quad (\text{Mt data attacker-state} + \text{Tt data attacker-state}) \leq \\
& \text{trace-fri-error} \\
\text{and support:} \\
& \text{Some (data, attacker-state)} \in \\
& \quad \text{set-dist (execute (checked-staged-transcript-program A) \\
& \quad \quad \text{adversary-initial-state})} \\
\text{shows} \\
& \text{trace-fri-reachable-partial-candidate-reduction-assumption} \\
& \quad (\text{verifier-state-from-adversary attacker-state} \\
& \quad \quad (\text{staged-proof-transcript data})) \\
\text{by (rule} \\
& \quad \text{trace-fri-reachable-partial-candidate-reduction-from-header-merkle-and-low-candidate-gap)} \\
& \quad (\text{rule trace-header-bound[OF support]},
\end{aligned}$$

$\text{rule trace-merkle-bound}[OF \text{ support}],$
 $\text{rule trace-low-gap-bound}[OF \text{ support}],$
 $\text{rule total-bound}[OF \text{ support}])$

definition *active-fri-reductions-for* **where**

$\text{active-fri-reductions-for } A \longleftrightarrow$
 $(\forall \text{ data attacker-state.}$
 $\text{Some (data, attacker-state)} \in$
 $\text{set-dist (execute (checked-staged-transcript-program } A)$
 $\text{adversary-initial-state)} \longrightarrow$
 $\text{trace-fri-header-tied-reduction}$
 $(\text{verifier-state-from-adversary attacker-state}$
 $(\text{staged-proof-transcript data})) \wedge$
 $\text{composition-fri-verifier-tied-reduction}$
 $(\text{verifier-state-from-adversary attacker-state}$
 $(\text{staged-proof-transcript data})) \wedge$
 $\text{trace-fri-empty-header-reduction}$
 $(\text{verifier-state-from-adversary attacker-state}$
 $(\text{staged-proof-transcript data})))$

definition *trace-fri-header-tied-residual-obligations*

$:: (\text{'f, 'a}) \text{ protocol-channel-scheme} \Rightarrow$
 $\text{prob} \Rightarrow \text{prob} \Rightarrow \text{prob} \Rightarrow \text{prob} \Rightarrow \text{prob} \Rightarrow \text{prob} \Rightarrow$
 $\text{prob} \Rightarrow \text{prob} \Rightarrow \text{prob} \Rightarrow \text{bool}$

where

$\text{trace-fri-header-tied-residual-obligations } s \text{ r-b z-b a-b p-b h-b g-b n-b s-b f-b}$
 $\longleftrightarrow$
 $\text{wp-event verify-monad}$
 $(\text{trace-fri-bad-with-header-tied-sampled-layer-chain } s) \text{ } s \leq \text{r-b} \wedge$
 $\text{wp-event verify-monad}$
 $(\text{trace-fri-header-tied-zero-round-final-obstruction } s) \text{ } s \leq \text{z-b} \wedge$
 $\text{wp-event verify-monad}$
 $(\text{trace-fri-header-tied-authenticated-slot-recorded-chunk-gap } s) \text{ } s$
 $\leq \text{a-b} \wedge$
 $\text{wp-event verify-monad (partial-merkle-inconsistency-bad } s) \text{ } s \leq \text{p-b} \wedge$
 $\text{wp-event verify-monad}$
 $(\text{trace-fri-header-tied-sibling-mismatch-structural-gap } s) \text{ } s \leq \text{h-b} \wedge$
 $\text{wp-event verify-monad}$
 $(\text{trace-fri-header-tied-base-head-value-neq-gap } s) \text{ } s \leq \text{g-b} \wedge$
 $\text{wp-event verify-monad}$
 $(\text{trace-fri-header-tied-next-value-replay-gap } s) \text{ } s \leq \text{n-b} \wedge$
 $\text{wp-event verify-monad}$
 $(\text{trace-fri-header-tied-successor-replay-gap } s) \text{ } s \leq \text{s-b} \wedge$
 $\text{wp-event verify-monad}$
 $(\text{trace-fri-header-tied-final-value-selected-step-missing-gap } s) \text{ } s$
 $\leq \text{f-b} \wedge$
 $\text{r-b} + (((((\text{p-b} + \text{h-b}) + \text{g-b}) + \text{n-b} + \text{s-b} +$
 $(\text{p-b} + \text{a-b} + \text{f-b})) + \text{p-b}) + \text{a-b} + \text{z-b})$
 $\leq \text{trace-fri-error}$

definition *composition-fri-verifier-tied-residual-obligations*

$:: ('f, 'a) \text{ protocol-channel-scheme} \Rightarrow$
 $\text{prob} \Rightarrow \text{prob} \Rightarrow \text{prob} \Rightarrow \text{prob} \Rightarrow \text{prob} \Rightarrow$
 $\text{prob} \Rightarrow \text{bool}$

where

composition-fri-verifier-tied-residual-obligations s r - b a - b p - b n - b s - b g - b
 $\longleftrightarrow$
 $\text{wp-event verify-monad}$
 $(\text{composition-fri-bad-with-verifier-tied-sampled-layer-chain } s) \ s$
 $\leq r\text{-}b \wedge$
 $\text{wp-event verify-monad}$
 $(\text{composition-fri-verifier-tied-authenticated-slot-recorded-chunk-gap } s)$
 $s \leq a\text{-}b \wedge$
 $\text{wp-event verify-monad } (\text{partial-merkle-inconsistency-bad } s) \ s \leq p\text{-}b \wedge$
 $\text{wp-event verify-monad}$
 $(\text{composition-fri-verifier-tied-next-value-replay-gap } s) \ s \leq n\text{-}b \wedge$
 $\text{wp-event verify-monad}$
 $(\text{composition-fri-verifier-tied-successor-replay-gap } s) \ s \leq s\text{-}b \wedge$
 $\text{wp-event verify-monad}$
 $(\text{composition-fri-verifier-tied-final-value-replay-gap } s) \ s \leq g\text{-}b \wedge$
 $r\text{-}b + (((p\text{-}b + a\text{-}b) + n\text{-}b + s\text{-}b + g\text{-}b + p\text{-}b) + a\text{-}b)$
 $\leq \text{composition-fri-error}$

lemma *trace-fri-header-tied-residual-obligations-from-bounds-without-head:*

assumes *sampled:*

$\text{wp-event verify-monad}$
 $(\text{trace-fri-bad-with-header-tied-sampled-layer-chain } s) \ s \leq r\text{-}b$

and *zero:*

$\text{wp-event verify-monad}$
 $(\text{trace-fri-header-tied-zero-round-final-obstruction } s) \ s \leq z\text{-}b$

and *slot:*

$\text{wp-event verify-monad}$
 $(\text{trace-fri-header-tied-authenticated-slot-recorded-chunk-gap } s) \ s$
 $\leq a\text{-}b$

and *merkle:*

$\text{wp-event verify-monad } (\text{partial-merkle-inconsistency-bad } s) \ s \leq p\text{-}b$

and *structural:*

$\text{wp-event verify-monad}$
 $(\text{trace-fri-header-tied-sibling-mismatch-structural-gap } s) \ s \leq h\text{-}b$

and *next-bound:*

$\text{wp-event verify-monad}$
 $(\text{trace-fri-header-tied-next-value-replay-gap } s) \ s \leq n\text{-}b$

and *successor:*

$\text{wp-event verify-monad}$
 $(\text{trace-fri-header-tied-successor-replay-gap } s) \ s \leq s\text{-}b$

and *final-bound:*

$\text{wp-event verify-monad}$
 $(\text{trace-fri-header-tied-final-value-selected-step-missing-gap } s) \ s$

$\leq f-b$
and total:
 $r-b + (((((p-b + h-b) + p-b) + n-b + s-b +$
 $(p-b + a-b + f-b)) + p-b) + a-b + z-b)$
 $\leq \text{trace-fri-error}$
shows
 $\text{trace-fri-header-tied-residual-obligations } s$
 $r-b \ z-b \ a-b \ p-b \ h-b \ p-b \ n-b \ s-b \ f-b$
proof –
have head:
 $\text{wp-event verify-monad}$
 $(\text{trace-fri-header-tied-base-head-value-neq-gap } s) \ s \leq p-b$
by (rule
 $\text{wp-trace-fri-header-tied-base-head-value-neq-gap-bound-from-partial-merkle}$
 $[\text{OF merkle}]$)
show ?thesis
using sampled zero slot merkle structural head next-bound successor
 final-bound total
unfolding trace-fri-header-tied-residual-obligations-def **by** blast
qed

lemma trace-fri-header-tied-residual-obligations-from-bounds-without-head-and-selected-zero:
assumes sampled:
 $\text{wp-event verify-monad}$
 $(\text{trace-fri-bad-with-header-tied-sampled-layer-chain } s) \ s \leq r-b$
and zero:
 $\text{wp-event verify-monad}$
 $(\text{trace-fri-header-tied-zero-round-final-obstruction } s) \ s \leq z-b$
and slot:
 $\text{wp-event verify-monad}$
 $(\text{trace-fri-header-tied-authenticated-slot-recorded-chunk-gap } s) \ s$
 $\leq a-b$
and merkle:
 $\text{wp-event verify-monad } (\text{partial-merkle-inconsistency-bad } s) \ s \leq p-b$
and structural:
 $\text{wp-event verify-monad}$
 $(\text{trace-fri-header-tied-sibling-mismatch-structural-gap } s) \ s \leq h-b$
and next-bound:
 $\text{wp-event verify-monad}$
 $(\text{trace-fri-header-tied-next-value-replay-gap } s) \ s \leq n-b$
and successor:
 $\text{wp-event verify-monad}$
 $(\text{trace-fri-header-tied-successor-replay-gap } s) \ s \leq s-b$
and total:
 $r-b + (((((p-b + h-b) + p-b) + n-b + s-b +$
 $(p-b + a-b + 0)) + p-b) + a-b + z-b)$
 $\leq \text{trace-fri-error}$
shows
 $\text{trace-fri-header-tied-residual-obligations } s$

$r-b \ z-b \ a-b \ p-b \ h-b \ p-b \ n-b \ s-b \ 0$
by (rule *trace-fri-header-tied-residual-obligations-from-bounds-without-head*)
(rule *sampled*, rule *zero*, rule *slot*, rule *merkle*, rule *structural*,
rule *next-bound*, rule *successor*,
rule *wp-trace-fri-header-tied-final-value-selected-step-missing-gap-zero*,
rule *total*)

lemma *trace-fri-header-tied-residual-obligations-from-bounds-without-head-slot-and-selected-zero*:

assumes *sampled*:

wp-event verify-monad

(*trace-fri-bad-with-header-tied-sampled-layer-chain s*) $s \leq r-b$

and *zero*:

wp-event verify-monad

(*trace-fri-header-tied-zero-round-final-obstruction s*) $s \leq z-b$

and *merkle*:

wp-event verify-monad (*partial-merkle-inconsistency-bad s*) $s \leq p-b$

and *structural*:

wp-event verify-monad

(*trace-fri-header-tied-sibling-mismatch-structural-gap s*) $s \leq h-b$

and *next-bound*:

wp-event verify-monad

(*trace-fri-header-tied-next-value-replay-gap s*) $s \leq n-b$

and *successor*:

wp-event verify-monad

(*trace-fri-header-tied-successor-replay-gap s*) $s \leq s-b$

and *total*:

$r-b + (((((p-b + h-b) + p-b) + n-b + s-b +$
 $(p-b + 0 + 0)) + p-b) + 0 + z-b)$
 $\leq \text{trace-fri-error}$

shows

trace-fri-header-tied-residual-obligations s

$r-b \ z-b \ 0 \ p-b \ h-b \ p-b \ n-b \ s-b \ 0$

by (rule

trace-fri-header-tied-residual-obligations-from-bounds-without-head-and-selected-zero)

(rule *sampled*, rule *zero*,

rule *wp-trace-fri-header-tied-authenticated-slot-recorded-chunk-gap-zero*,

rule *merkle*, rule *structural*, rule *next-bound*, rule *successor*, rule *total*)

lemma *composition-fri-verifier-tied-residual-obligations-from-bounds*:

assumes *sampled*:

wp-event verify-monad

(*composition-fri-bad-with-verifier-tied-sampled-layer-chain s*) s
 $\leq r-b$

and *slot*:

wp-event verify-monad

(*composition-fri-verifier-tied-authenticated-slot-recorded-chunk-gap s*)
 $s \leq a-b$

and *merkle*:

wp-event verify-monad (*partial-merkle-inconsistency-bad s*) $s \leq p-b$

and *next-bound*:
wp-event verify-monad
 $(\text{composition-fri-verifier-tied-next-value-replay-gap } s) \ s \leq n\text{-}b$
and *successor*:
wp-event verify-monad
 $(\text{composition-fri-verifier-tied-successor-replay-gap } s) \ s \leq s\text{-}b$
and *final-bound*:
wp-event verify-monad
 $(\text{composition-fri-verifier-tied-final-value-replay-gap } s) \ s \leq g\text{-}b$
and *total*:
 $r\text{-}b + (((p\text{-}b + a\text{-}b) + n\text{-}b + s\text{-}b + g\text{-}b + p\text{-}b) + a\text{-}b)$
 $\leq \text{composition-fri-error}$
shows
 $\text{composition-fri-verifier-tied-residual-obligations } s$
 $r\text{-}b \ a\text{-}b \ p\text{-}b \ n\text{-}b \ s\text{-}b \ g\text{-}b$
using *sampled slot merkle next-bound successor final-bound total*
unfolding *composition-fri-verifier-tied-residual-obligations-def* **by** *blast*

lemma *composition-fri-verifier-tied-residual-obligations-from-bounds-slot-zero*:

assumes *sampled*:
wp-event verify-monad
 $(\text{composition-fri-bad-with-verifier-tied-sampled-layer-chain } s) \ s$
 $\leq r\text{-}b$
and *merkle*:
wp-event verify-monad $(\text{partial-merkle-inconsistency-bad } s) \ s \leq p\text{-}b$
and *next-bound*:
wp-event verify-monad
 $(\text{composition-fri-verifier-tied-next-value-replay-gap } s) \ s \leq n\text{-}b$
and *successor*:
wp-event verify-monad
 $(\text{composition-fri-verifier-tied-successor-replay-gap } s) \ s \leq s\text{-}b$
and *final-bound*:
wp-event verify-monad
 $(\text{composition-fri-verifier-tied-final-value-replay-gap } s) \ s \leq g\text{-}b$
and *total*:
 $r\text{-}b + (((p\text{-}b + 0) + n\text{-}b + s\text{-}b + g\text{-}b + p\text{-}b) + 0)$
 $\leq \text{composition-fri-error}$
shows
 $\text{composition-fri-verifier-tied-residual-obligations } s$
 $r\text{-}b \ 0 \ p\text{-}b \ n\text{-}b \ s\text{-}b \ g\text{-}b$
by (*rule composition-fri-verifier-tied-residual-obligations-from-bounds*)
(*rule sampled*,
rule wp-composition-fri-verifier-tied-authenticated-slot-recorded-chunk-gap-zero,
rule merkle, *rule next-bound*, *rule successor*, *rule final-bound*,
rule total)

lemma *active-composition-fri-reachable-reduction-from-split-bounds-without-empty-with-candidate-available*:

fixes
 $Rc \ Ac \ Pc \ Nc \ Sc \ Gc \ Qc \ Tc ::$

('f staged-proof-data $\Rightarrow$
 'f protocol-channel $\Rightarrow$ prob)
assumes comp-sampled-bound:
 $\wedge$ data attacker-state.
 Some (data, attacker-state) $\in$
 set-dist (execute (checked-staged-transcript-program A)
 adversary-initial-state) $\Rightarrow$
 wp-event verify-monad
 (composition-fri-bad-with-verifier-tied-sampled-layer-chain
 (verifier-state-from-adversary attacker-state
 (staged-proof-transcript data)))
 (verifier-state-from-adversary attacker-state
 (staged-proof-transcript data)) $\leq$
 Rc data attacker-state
and comp-slot-bound:
 $\wedge$ data attacker-state.
 Some (data, attacker-state) $\in$
 set-dist (execute (checked-staged-transcript-program A)
 adversary-initial-state) $\Rightarrow$
 wp-event verify-monad
 (composition-fri-verifier-tied-authenticated-slot-recorded-chunk-gap
 (verifier-state-from-adversary attacker-state
 (staged-proof-transcript data)))
 (verifier-state-from-adversary attacker-state
 (staged-proof-transcript data)) $\leq$
 Ac data attacker-state
and comp-merkle-bound:
 $\wedge$ data attacker-state.
 Some (data, attacker-state) $\in$
 set-dist (execute (checked-staged-transcript-program A)
 adversary-initial-state) $\Rightarrow$
 wp-event verify-monad
 (partial-merkle-inconsistency-bad
 (verifier-state-from-adversary attacker-state
 (staged-proof-transcript data)))
 (verifier-state-from-adversary attacker-state
 (staged-proof-transcript data)) $\leq$
 Pc data attacker-state
and comp-next-bound:
 $\wedge$ data attacker-state.
 Some (data, attacker-state) $\in$
 set-dist (execute (checked-staged-transcript-program A)
 adversary-initial-state) $\Rightarrow$
 wp-event verify-monad
 (composition-fri-verifier-tied-next-value-replay-gap
 (verifier-state-from-adversary attacker-state
 (staged-proof-transcript data)))
 (verifier-state-from-adversary attacker-state
 (staged-proof-transcript data)) $\leq$

Nc data attacker-state
and *comp-successor-bound:*
 $\wedge$ *data attacker-state.*
Some (data, attacker-state) $\in$
set-dist (execute (checked-staged-transcript-program A)
adversary-initial-state) $\implies$
wp-event verify-monad
(composition-fri-verifier-tied-successor-replay-gap
(verifier-state-from-adversary attacker-state
(staged-proof-transcript data)))
(verifier-state-from-adversary attacker-state
(staged-proof-transcript data)) $\leq$
Sc data attacker-state
and *comp-final-bound:*
 $\wedge$ *data attacker-state.*
Some (data, attacker-state) $\in$
set-dist (execute (checked-staged-transcript-program A)
adversary-initial-state) $\implies$
wp-event verify-monad
(composition-fri-verifier-tied-final-value-replay-gap
(verifier-state-from-adversary attacker-state
(staged-proof-transcript data)))
(verifier-state-from-adversary attacker-state
(staged-proof-transcript data)) $\leq$
Gc data attacker-state
and *comp-gap-merkle-bound:*
 $\wedge$ *data attacker-state.*
Some (data, attacker-state) $\in$
set-dist (execute (checked-staged-transcript-program A)
adversary-initial-state) $\implies$
wp-event verify-monad
(partial-merkle-inconsistency-bad
(verifier-state-from-adversary attacker-state
(staged-proof-transcript data)))
(verifier-state-from-adversary attacker-state
(staged-proof-transcript data)) $\leq$
Qc data attacker-state
and *comp-candidate-available-bound:*
 $\wedge$ *data attacker-state.*
Some (data, attacker-state) $\in$
set-dist (execute (checked-staged-transcript-program A)
adversary-initial-state) $\implies$
wp-event verify-monad
(composition-fri-verifier-tied-candidate-available-gap
(verifier-state-from-adversary attacker-state
(staged-proof-transcript data)))
(verifier-state-from-adversary attacker-state
(staged-proof-transcript data)) $\leq$
Tc data attacker-state

and *comp-total-bound*:
 $\wedge$ *data attacker-state*.
Some (*data*, *attacker-state*) $\in$
set-dist (*execute* (*checked-staged-transcript-program* *A*)
adversary-initial-state) $\implies$
Rc data attacker-state +
 $((Pc \text{ data attacker-state} + 0) +$
 $Nc \text{ data attacker-state} + Sc \text{ data attacker-state} +$
 $Gc \text{ data attacker-state} + Pc \text{ data attacker-state}) +$
 $Ac \text{ data attacker-state}) +$
 $(Qc \text{ data attacker-state} + Tc \text{ data attacker-state})$
 $\leq$ *composition-fri-error*
assumes *support*:
Some (*data*, *attacker-state*) $\in$
set-dist (*execute* (*checked-staged-transcript-program* *A*)
adversary-initial-state)
shows
composition-fri-reachable-partial-candidate-reduction-assumption
(*verifier-state-from-adversary attacker-state*
(*staged-proof-transcript data*))
proof –
let *?s* =
verifier-state-from-adversary attacker-state
(*staged-proof-transcript data*)
show *?thesis*
by (*rule*
composition-fri-reachable-partial-candidate-reduction-from-split-residual-bounds-with-candidate-available-and-rec
[**where** *E* = 0])
(*rule comp-sampled-bound*[*OF support*],
rule comp-slot-bound[*OF support*],
rule comp-merkle-bound[*OF support*],
rule comp-next-bound[*OF support*],
rule comp-successor-bound[*OF support*],
rule comp-final-bound[*OF support*],
simp add: wp-composition-fri-reachable-verifier-tie-empty-header-gap-zero,
rule comp-gap-merkle-bound[*OF support*],
rule comp-candidate-available-bound[*OF support*],
insert comp-total-bound[*OF support*], *simp*)
qed

definition *active-fri-residual-obligations-for where*
active-fri-residual-obligations-for *A Rt Zt At Pt Ht Gt Nt St Ft*
Rc Ac Pc Nc Sc Gc $\longleftrightarrow$
 $(\forall$ *data attacker-state*.
Some (*data*, *attacker-state*) $\in$
set-dist (*execute* (*checked-staged-transcript-program* *A*)
adversary-initial-state) $\longrightarrow$
 $(let$ *s* =
verifier-state-from-adversary attacker-state

(staged-proof-transcript data)
 in
 trace-fri-header-tied-residual-obligations s
 (Rt data attacker-state) (Zt data attacker-state)
 (At data attacker-state) (Pt data attacker-state)
 (Ht data attacker-state) (Gt data attacker-state)
 (Nt data attacker-state) (St data attacker-state)
 (Ft data attacker-state) $\wedge$
 composition-fri-verifier-tied-residual-obligations s
 (Rc data attacker-state) (Ac data attacker-state)
 (Pc data attacker-state)
 (Nc data attacker-state) (Sc data attacker-state)
 (Gc data attacker-state) $\wedge$
 trace-fri-empty-header-reduction s))

definition active-trace-fri-residual-obligations-for **where**

active-trace-fri-residual-obligations-for A Rt Zt At Pt Ht Gt Nt St Ft

$\longleftrightarrow$

($\forall$ data attacker-state.

Some (data, attacker-state) $\in$

set-dist (execute (checked-staged-transcript-program A)

adversary-initial-state) $\longrightarrow$

trace-fri-header-tied-residual-obligations

(verifier-state-from-adversary attacker-state

(staged-proof-transcript data))

(Rt data attacker-state) (Zt data attacker-state)

(At data attacker-state) (Pt data attacker-state)

(Ht data attacker-state) (Gt data attacker-state)

(Nt data attacker-state) (St data attacker-state)

(Ft data attacker-state))

definition active-composition-fri-residual-obligations-for **where**

active-composition-fri-residual-obligations-for A Rc Ac Pc Nc Sc Gc

$\longleftrightarrow$

($\forall$ data attacker-state.

Some (data, attacker-state) $\in$

set-dist (execute (checked-staged-transcript-program A)

adversary-initial-state) $\longrightarrow$

composition-fri-verifier-tied-residual-obligations

(verifier-state-from-adversary attacker-state

(staged-proof-transcript data))

(Rc data attacker-state) (Ac data attacker-state)

(Pc data attacker-state)

(Nc data attacker-state) (Sc data attacker-state)

(Gc data attacker-state))

definition active-empty-trace-fri-reduction-for **where**

active-empty-trace-fri-reduction-for A $\longleftrightarrow$

($\forall$ data attacker-state.

Some (data, attacker-state) ∈
set-dist (execute (checked-staged-transcript-program A)
adversary-initial-state) →
trace-fri-empty-header-reduction
(verifier-state-from-adversary attacker-state
(staged-proof-transcript data)))

lemma *active-fri-residual-obligations-from-components:*

assumes

active-trace-fri-residual-obligations-for A Rt Zt At Pt Ht Gt Nt St Ft
active-composition-fri-residual-obligations-for A Rc Ac Pc Nc Sc Gc
active-empty-trace-fri-reduction-for A

shows *active-fri-residual-obligations-for A Rt Zt At Pt Ht Gt Nt St Ft*
Rc Ac Pc Nc Sc Gc

using *assms*

unfolding *active-fri-residual-obligations-for-def*

active-trace-fri-residual-obligations-for-def
active-composition-fri-residual-obligations-for-def
active-empty-trace-fri-reduction-for-def

by (*simp add: Let-def*)

lemma *active-fri-residual-obligations-trace-component:*

assumes *active-fri-residual-obligations-for A Rt Zt At Pt Ht Gt Nt St Ft*
Rc Ac Pc Nc Sc Gc

shows *active-trace-fri-residual-obligations-for A Rt Zt At Pt Ht Gt Nt St Ft*

using *assms*

unfolding *active-fri-residual-obligations-for-def*

active-trace-fri-residual-obligations-for-def

by (*simp add: Let-def*)

lemma *active-fri-residual-obligations-composition-component:*

assumes *active-fri-residual-obligations-for A Rt Zt At Pt Ht Gt Nt St Ft*
Rc Ac Pc Nc Sc Gc

shows *active-composition-fri-residual-obligations-for A Rc Ac Pc Nc Sc Gc*

using *assms*

unfolding *active-fri-residual-obligations-for-def*

active-composition-fri-residual-obligations-for-def

by (*simp add: Let-def*)

lemma *active-fri-residual-obligations-empty-trace-component:*

assumes *active-fri-residual-obligations-for A Rt Zt At Pt Ht Gt Nt St Ft*
Rc Ac Pc Nc Sc Gc

shows *active-empty-trace-fri-reduction-for A*

using *assms*

unfolding *active-fri-residual-obligations-for-def*

active-empty-trace-fri-reduction-for-def

by (*simp add: Let-def*)

lemma *active-trace-fri-residual-obligations-from-bounds-without-head:*

fixes
Rt Zt At Pt Ht Nt St Ft ::
('f staged-proof-data $\Rightarrow$
'f protocol-channel $\Rightarrow$ prob)
assumes *trace-sampled-bound:*
 $\wedge$ *data attacker-state.*
Some (data, attacker-state) $\in$
set-dist (execute (checked-staged-transcript-program A)
adversary-initial-state) $\Rightarrow$
wp-event verify-monad
(trace-fri-bad-with-header-tied-sampled-layer-chain
(verifier-state-from-adversary attacker-state
(staged-proof-transcript data)))
(verifier-state-from-adversary attacker-state
(staged-proof-transcript data)) $\leq$
Rt data attacker-state
and *trace-zero-bound:*
 $\wedge$ *data attacker-state.*
Some (data, attacker-state) $\in$
set-dist (execute (checked-staged-transcript-program A)
adversary-initial-state) $\Rightarrow$
wp-event verify-monad
(trace-fri-header-tied-zero-round-final-obstruction
(verifier-state-from-adversary attacker-state
(staged-proof-transcript data)))
(verifier-state-from-adversary attacker-state
(staged-proof-transcript data)) $\leq$
Zt data attacker-state
and *trace-slot-bound:*
 $\wedge$ *data attacker-state.*
Some (data, attacker-state) $\in$
set-dist (execute (checked-staged-transcript-program A)
adversary-initial-state) $\Rightarrow$
wp-event verify-monad
(trace-fri-header-tied-authenticated-slot-recorded-chunk-gap
(verifier-state-from-adversary attacker-state
(staged-proof-transcript data)))
(verifier-state-from-adversary attacker-state
(staged-proof-transcript data)) $\leq$
At data attacker-state
and *trace-merkle-bound:*
 $\wedge$ *data attacker-state.*
Some (data, attacker-state) $\in$
set-dist (execute (checked-staged-transcript-program A)
adversary-initial-state) $\Rightarrow$
wp-event verify-monad
(partial-merkle-inconsistency-bad
(verifier-state-from-adversary attacker-state
(staged-proof-transcript data)))

$(\text{verifier-state-from-adversary attacker-state}$
 $(\text{staged-proof-transcript data})) \leq$
 $Pt \text{ data attacker-state}$
and *trace-structural-bound*:
 $\wedge \text{data attacker-state.}$
 $\text{Some (data, attacker-state)} \in$
 $\text{set-dist (execute (checked-staged-transcript-program A)}$
 $\text{adversary-initial-state)} \implies$
 $wp\text{-event verify-monad}$
 $(\text{trace-fri-header-tied-sibling-mismatch-structural-gap}$
 $(\text{verifier-state-from-adversary attacker-state}$
 $(\text{staged-proof-transcript data})))$
 $(\text{verifier-state-from-adversary attacker-state}$
 $(\text{staged-proof-transcript data})) \leq$
 $Ht \text{ data attacker-state}$
and *trace-next-bound*:
 $\wedge \text{data attacker-state.}$
 $\text{Some (data, attacker-state)} \in$
 $\text{set-dist (execute (checked-staged-transcript-program A)}$
 $\text{adversary-initial-state)} \implies$
 $wp\text{-event verify-monad}$
 $(\text{trace-fri-header-tied-next-value-replay-gap}$
 $(\text{verifier-state-from-adversary attacker-state}$
 $(\text{staged-proof-transcript data})))$
 $(\text{verifier-state-from-adversary attacker-state}$
 $(\text{staged-proof-transcript data})) \leq$
 $Nt \text{ data attacker-state}$
and *trace-successor-bound*:
 $\wedge \text{data attacker-state.}$
 $\text{Some (data, attacker-state)} \in$
 $\text{set-dist (execute (checked-staged-transcript-program A)}$
 $\text{adversary-initial-state)} \implies$
 $wp\text{-event verify-monad}$
 $(\text{trace-fri-header-tied-successor-replay-gap}$
 $(\text{verifier-state-from-adversary attacker-state}$
 $(\text{staged-proof-transcript data})))$
 $(\text{verifier-state-from-adversary attacker-state}$
 $(\text{staged-proof-transcript data})) \leq$
 $St \text{ data attacker-state}$
and *trace-final-bound*:
 $\wedge \text{data attacker-state.}$
 $\text{Some (data, attacker-state)} \in$
 $\text{set-dist (execute (checked-staged-transcript-program A)}$
 $\text{adversary-initial-state)} \implies$
 $wp\text{-event verify-monad}$
 $(\text{trace-fri-header-tied-final-value-selected-step-missing-gap}$
 $(\text{verifier-state-from-adversary attacker-state}$
 $(\text{staged-proof-transcript data})))$
 $(\text{verifier-state-from-adversary attacker-state}$

```

    (staged-proof-transcript data)) ≤
    Ft data attacker-state
and trace-total-bound:
 $\wedge$  data attacker-state.
    Some (data, attacker-state) ∈
    set-dist (execute (checked-staged-transcript-program A)
    adversary-initial-state)  $\implies$ 
    Rt data attacker-state +
    (((((Pt data attacker-state + Ht data attacker-state) +
    Pt data attacker-state) +
    Nt data attacker-state + St data attacker-state +
    (Pt data attacker-state + At data attacker-state +
    Ft data attacker-state)) +
    Pt data attacker-state) +
    At data attacker-state + Zt data attacker-state)
    ≤ trace-fri-error
shows active-trace-fri-residual-obligations-for A Rt Zt At Pt Ht Pt
    Nt St Ft
unfolding active-trace-fri-residual-obligations-for-def
proof (intro allI impI)
fix data attacker-state
assume support:
    Some (data, attacker-state) ∈
    set-dist (execute (checked-staged-transcript-program A)
    adversary-initial-state)
let ?s =
    verifier-state-from-adversary attacker-state
    (staged-proof-transcript data)
show trace-fri-header-tied-residual-obligations ?s
    (Rt data attacker-state) (Zt data attacker-state)
    (At data attacker-state) (Pt data attacker-state)
    (Ht data attacker-state) (Pt data attacker-state)
    (Nt data attacker-state) (St data attacker-state)
    (Ft data attacker-state)
by (rule trace-fri-header-tied-residual-obligations-from-bounds-without-head)
    (rule trace-sampled-bound[OF support],
    rule trace-zero-bound[OF support],
    rule trace-slot-bound[OF support],
    rule trace-merkle-bound[OF support],
    rule trace-structural-bound[OF support],
    rule trace-next-bound[OF support],
    rule trace-successor-bound[OF support],
    rule trace-final-bound[OF support],
    rule trace-total-bound[OF support])
qed

lemma active-trace-fri-residual-obligations-from-bounds-without-head-and-selected-zero:
fixes
    Rt Zt At Pt Ht Nt St ::

```


('f staged-proof-data $\Rightarrow$
 'f protocol-channel $\Rightarrow$ prob)
assumes trace-sampled-bound:
 $\wedge$ data attacker-state.
 Some (data, attacker-state) $\in$
 set-dist (execute (checked-staged-transcript-program A)
 adversary-initial-state) $\Rightarrow$
 wp-event verify-monad
 (trace-fri-bad-with-header-tied-sampled-layer-chain
 (verifier-state-from-adversary attacker-state
 (staged-proof-transcript data)))
 (verifier-state-from-adversary attacker-state
 (staged-proof-transcript data)) $\leq$
 Rt data attacker-state
and trace-zero-bound:
 $\wedge$ data attacker-state.
 Some (data, attacker-state) $\in$
 set-dist (execute (checked-staged-transcript-program A)
 adversary-initial-state) $\Rightarrow$
 wp-event verify-monad
 (trace-fri-header-tied-zero-round-final-obstruction
 (verifier-state-from-adversary attacker-state
 (staged-proof-transcript data)))
 (verifier-state-from-adversary attacker-state
 (staged-proof-transcript data)) $\leq$
 Zt data attacker-state
and trace-slot-bound:
 $\wedge$ data attacker-state.
 Some (data, attacker-state) $\in$
 set-dist (execute (checked-staged-transcript-program A)
 adversary-initial-state) $\Rightarrow$
 wp-event verify-monad
 (trace-fri-header-tied-authenticated-slot-recorded-chunk-gap
 (verifier-state-from-adversary attacker-state
 (staged-proof-transcript data)))
 (verifier-state-from-adversary attacker-state
 (staged-proof-transcript data)) $\leq$
 At data attacker-state
and trace-merkle-bound:
 $\wedge$ data attacker-state.
 Some (data, attacker-state) $\in$
 set-dist (execute (checked-staged-transcript-program A)
 adversary-initial-state) $\Rightarrow$
 wp-event verify-monad
 (partial-merkle-inconsistency-bad
 (verifier-state-from-adversary attacker-state
 (staged-proof-transcript data)))
 (verifier-state-from-adversary attacker-state
 (staged-proof-transcript data)) $\leq$

Pt data attacker-state
and *trace-structural-bound:*
 $\wedge$ *data attacker-state.*
Some (data, attacker-state) $\in$
set-dist (execute (checked-staged-transcript-program A)
adversary-initial-state) $\implies$
wp-event verify-monad
(trace-fri-header-tied-sibling-mismatch-structural-gap
(verifier-state-from-adversary attacker-state
(staged-proof-transcript data)))
(verifier-state-from-adversary attacker-state
(staged-proof-transcript data)) $\leq$
Ht data attacker-state
and *trace-next-bound:*
 $\wedge$ *data attacker-state.*
Some (data, attacker-state) $\in$
set-dist (execute (checked-staged-transcript-program A)
adversary-initial-state) $\implies$
wp-event verify-monad
(trace-fri-header-tied-next-value-replay-gap
(verifier-state-from-adversary attacker-state
(staged-proof-transcript data)))
(verifier-state-from-adversary attacker-state
(staged-proof-transcript data)) $\leq$
Nt data attacker-state
and *trace-successor-bound:*
 $\wedge$ *data attacker-state.*
Some (data, attacker-state) $\in$
set-dist (execute (checked-staged-transcript-program A)
adversary-initial-state) $\implies$
wp-event verify-monad
(trace-fri-header-tied-successor-replay-gap
(verifier-state-from-adversary attacker-state
(staged-proof-transcript data)))
(verifier-state-from-adversary attacker-state
(staged-proof-transcript data)) $\leq$
St data attacker-state
and *trace-total-bound:*
 $\wedge$ *data attacker-state.*
Some (data, attacker-state) $\in$
set-dist (execute (checked-staged-transcript-program A)
adversary-initial-state) $\implies$
Rt data attacker-state +
(((((Pt data attacker-state + Ht data attacker-state) +
Pt data attacker-state) +
Nt data attacker-state + St data attacker-state +
(Pt data attacker-state + At data attacker-state + 0)) +
Pt data attacker-state) +
At data attacker-state + Zt data attacker-state)

```

    ≤ trace-fri-error
shows active-trace-fri-residual-obligations-for A Rt Zt At Pt Ht Pt
  Nt St (λ- -. (0::prob))
unfolding active-trace-fri-residual-obligations-for-def
proof (intro allI impI)
fix data attacker-state
assume support:
  Some (data, attacker-state) ∈
    set-dist (execute (checked-staged-transcript-program A)
      adversary-initial-state)
let ?s =
  verifier-state-from-adversary attacker-state
    (staged-proof-transcript data)
show trace-fri-header-tied-residual-obligations ?s
  (Rt data attacker-state) (Zt data attacker-state)
  (At data attacker-state) (Pt data attacker-state)
  (Ht data attacker-state) (Pt data attacker-state)
  (Nt data attacker-state) (St data attacker-state) 0
by (rule
  trace-fri-header-tied-residual-obligations-from-bounds-without-head-and-selected-zero)
  (rule trace-sampled-bound[OF support],
  rule trace-zero-bound[OF support],
  rule trace-slot-bound[OF support],
  rule trace-merkle-bound[OF support],
  rule trace-structural-bound[OF support],
  rule trace-next-bound[OF support],
  rule trace-successor-bound[OF support],
  rule trace-total-bound[OF support])
qed

lemma active-trace-fri-residual-obligations-from-bounds-without-head-slot-and-selected-zero:
fixes
  Rt Zt Pt Ht Nt St ::
    ('f staged-proof-data ⇒
    'f protocol-channel ⇒ prob)
assumes trace-sampled-bound:
  ∧data attacker-state.
  Some (data, attacker-state) ∈
    set-dist (execute (checked-staged-transcript-program A)
      adversary-initial-state) ⇒
  wp-event verify-monad
    (trace-fri-bad-with-header-tied-sampled-layer-chain
      (verifier-state-from-adversary attacker-state
        (staged-proof-transcript data)))
    (verifier-state-from-adversary attacker-state
      (staged-proof-transcript data)) ≤
  Rt data attacker-state
and trace-zero-bound:
  ∧data attacker-state.

```

Some (data, attacker-state) ∈
set-dist (execute (checked-staged-transcript-program A)
adversary-initial-state) ⇒
wp-event verify-monad
(trace-fri-header-tied-zero-round-final-obstruction
(verifier-state-from-adversary attacker-state
(staged-proof-transcript data)))
(verifier-state-from-adversary attacker-state
(staged-proof-transcript data)) ≤
Zt data attacker-state
and *trace-merkle-bound:*
 $\wedge$ *data attacker-state.*
Some (data, attacker-state) ∈
set-dist (execute (checked-staged-transcript-program A)
adversary-initial-state) ⇒
wp-event verify-monad
(partial-merkle-inconsistency-bad
(verifier-state-from-adversary attacker-state
(staged-proof-transcript data)))
(verifier-state-from-adversary attacker-state
(staged-proof-transcript data)) ≤
Pt data attacker-state
and *trace-structural-bound:*
 $\wedge$ *data attacker-state.*
Some (data, attacker-state) ∈
set-dist (execute (checked-staged-transcript-program A)
adversary-initial-state) ⇒
wp-event verify-monad
(trace-fri-header-tied-sibling-mismatch-structural-gap
(verifier-state-from-adversary attacker-state
(staged-proof-transcript data)))
(verifier-state-from-adversary attacker-state
(staged-proof-transcript data)) ≤
Ht data attacker-state
and *trace-next-bound:*
 $\wedge$ *data attacker-state.*
Some (data, attacker-state) ∈
set-dist (execute (checked-staged-transcript-program A)
adversary-initial-state) ⇒
wp-event verify-monad
(trace-fri-header-tied-next-value-replay-gap
(verifier-state-from-adversary attacker-state
(staged-proof-transcript data)))
(verifier-state-from-adversary attacker-state
(staged-proof-transcript data)) ≤
Nt data attacker-state
and *trace-successor-bound:*
 $\wedge$ *data attacker-state.*
Some (data, attacker-state) ∈

```

    set-dist (execute (checked-staged-transcript-program A)
      adversary-initial-state)  $\implies$ 
wp-event verify-monad
  (trace-fri-header-tied-successor-replay-gap
    (verifier-state-from-adversary attacker-state
      (staged-proof-transcript data)))
  (verifier-state-from-adversary attacker-state
    (staged-proof-transcript data))  $\leq$ 
  St data attacker-state
and trace-total-bound:
 $\wedge$ data attacker-state.
  Some (data, attacker-state)  $\in$ 
    set-dist (execute (checked-staged-transcript-program A)
      adversary-initial-state)  $\implies$ 
  Rt data attacker-state +
    (((((Pt data attacker-state + Ht data attacker-state) +
      Pt data attacker-state) +
      Nt data attacker-state + St data attacker-state +
      (Pt data attacker-state + 0 + 0)) +
      Pt data attacker-state) +
      0 + Zt data attacker-state)
     $\leq$  trace-fri-error
shows
  active-trace-fri-residual-obligations-for A Rt Zt
    ( $\lambda$ - -. (0::prob)) Pt Ht Pt Nt St ( $\lambda$ - -. (0::prob))
unfolding active-trace-fri-residual-obligations-for-def
proof (intro allI impI)
fix data attacker-state
assume support:
  Some (data, attacker-state)  $\in$ 
    set-dist (execute (checked-staged-transcript-program A)
      adversary-initial-state)
let ?s =
  verifier-state-from-adversary attacker-state
    (staged-proof-transcript data)
show trace-fri-header-tied-residual-obligations ?s
  (Rt data attacker-state) (Zt data attacker-state) 0
  (Pt data attacker-state) (Ht data attacker-state)
  (Pt data attacker-state) (Nt data attacker-state)
  (St data attacker-state) 0
by (rule
  trace-fri-header-tied-residual-obligations-from-bounds-without-head-slot-and-selected-zero)
  (rule trace-sampled-bound[OF support],
  rule trace-zero-bound[OF support],
  rule trace-merkle-bound[OF support],
  rule trace-structural-bound[OF support],
  rule trace-next-bound[OF support],
  rule trace-successor-bound[OF support],
  rule trace-total-bound[OF support])

```

qed

lemma *active-composition-fri-residual-obligations-from-bounds:*

fixes

Rc Ac Pc Nc Sc Gc ::

('f staged-proof-data $\Rightarrow$

'f protocol-channel $\Rightarrow$ prob)

assumes *comp-sampled-bound:*

$\wedge$ *data attacker-state.*

Some (data, attacker-state) $\in$

set-dist (execute (checked-staged-transcript-program A)

adversary-initial-state) $\Rightarrow$

wp-event verify-monad

(composition-fri-bad-with-verifier-tied-sampled-layer-chain

(verifier-state-from-adversary attacker-state

(staged-proof-transcript data)))

(verifier-state-from-adversary attacker-state

(staged-proof-transcript data)) $\leq$

Rc data attacker-state

and *comp-slot-bound:*

$\wedge$ *data attacker-state.*

Some (data, attacker-state) $\in$

set-dist (execute (checked-staged-transcript-program A)

adversary-initial-state) $\Rightarrow$

wp-event verify-monad

(composition-fri-verifier-tied-authenticated-slot-recorded-chunk-gap

(verifier-state-from-adversary attacker-state

(staged-proof-transcript data)))

(verifier-state-from-adversary attacker-state

(staged-proof-transcript data)) $\leq$

Ac data attacker-state

and *comp-merkle-bound:*

$\wedge$ *data attacker-state.*

Some (data, attacker-state) $\in$

set-dist (execute (checked-staged-transcript-program A)

adversary-initial-state) $\Rightarrow$

wp-event verify-monad

(partial-merkle-inconsistency-bad

(verifier-state-from-adversary attacker-state

(staged-proof-transcript data)))

(verifier-state-from-adversary attacker-state

(staged-proof-transcript data)) $\leq$

Pc data attacker-state

and *comp-next-bound:*

$\wedge$ *data attacker-state.*

Some (data, attacker-state) $\in$

set-dist (execute (checked-staged-transcript-program A)

adversary-initial-state) $\Rightarrow$

wp-event verify-monad

```

    (composition-fri-verifier-tied-next-value-replay-gap
      (verifier-state-from-adversary attacker-state
        (staged-proof-transcript data))))
    (verifier-state-from-adversary attacker-state
      (staged-proof-transcript data)) ≤
  Nc data attacker-state
and comp-successor-bound:
  ∧ data attacker-state.
    Some (data, attacker-state) ∈
      set-dist (execute (checked-staged-transcript-program A)
        adversary-initial-state) ⇒
    wp-event verify-monad
      (composition-fri-verifier-tied-successor-replay-gap
        (verifier-state-from-adversary attacker-state
          (staged-proof-transcript data))))
      (verifier-state-from-adversary attacker-state
        (staged-proof-transcript data)) ≤
    Sc data attacker-state
and comp-final-bound:
  ∧ data attacker-state.
    Some (data, attacker-state) ∈
      set-dist (execute (checked-staged-transcript-program A)
        adversary-initial-state) ⇒
    wp-event verify-monad
      (composition-fri-verifier-tied-final-value-replay-gap
        (verifier-state-from-adversary attacker-state
          (staged-proof-transcript data))))
      (verifier-state-from-adversary attacker-state
        (staged-proof-transcript data)) ≤
    Gc data attacker-state
and comp-total-bound:
  ∧ data attacker-state.
    Some (data, attacker-state) ∈
      set-dist (execute (checked-staged-transcript-program A)
        adversary-initial-state) ⇒
    Rc data attacker-state +
      (((Pc data attacker-state + Ac data attacker-state) +
        Nc data attacker-state + Sc data attacker-state +
        Gc data attacker-state + Pc data attacker-state) +
        Ac data attacker-state)
    ≤ composition-fri-error
shows active-composition-fri-residual-obligations-for A Rc Ac Pc Nc Sc Gc
unfolding active-composition-fri-residual-obligations-for-def
proof (intro allI impI)
fix data attacker-state
assume support:
  Some (data, attacker-state) ∈
    set-dist (execute (checked-staged-transcript-program A)
      adversary-initial-state)

```

```

let ?s =
  verifier-state-from-adversary attacker-state
  (staged-proof-transcript data)
show composition-fri-verifier-tied-residual-obligations ?s
  (Rc data attacker-state) (Ac data attacker-state)
  (Pc data attacker-state)
  (Nc data attacker-state) (Sc data attacker-state)
  (Gc data attacker-state)
by (rule composition-fri-verifier-tied-residual-obligations-from-bounds)
  (rule comp-sampled-bound[OF support],
   rule comp-slot-bound[OF support],
   rule comp-merkle-bound[OF support],
   rule comp-next-bound[OF support],
   rule comp-successor-bound[OF support],
   rule comp-final-bound[OF support],
   rule comp-total-bound[OF support])
qed

```

lemma active-composition-fri-residual-obligations-from-bounds-slot-zero:

```

fixes
  Rc Pc Nc Sc Gc ::
    ('f staged-proof-data  $\Rightarrow$ 
     'f protocol-channel  $\Rightarrow$  prob)
assumes comp-sampled-bound:
   $\wedge$ data attacker-state.
  Some (data, attacker-state)  $\in$ 
    set-dist (execute (checked-staged-transcript-program A)
      adversary-initial-state)  $\Longrightarrow$ 
  wp-event verify-monad
    (composition-fri-bad-with-verifier-tied-sampled-layer-chain
      (verifier-state-from-adversary attacker-state
        (staged-proof-transcript data)))
    (verifier-state-from-adversary attacker-state
      (staged-proof-transcript data))  $\leq$ 
  Rc data attacker-state
and comp-merkle-bound:
   $\wedge$ data attacker-state.
  Some (data, attacker-state)  $\in$ 
    set-dist (execute (checked-staged-transcript-program A)
      adversary-initial-state)  $\Longrightarrow$ 
  wp-event verify-monad
    (partial-merkle-inconsistency-bad
      (verifier-state-from-adversary attacker-state
        (staged-proof-transcript data)))
    (verifier-state-from-adversary attacker-state
      (staged-proof-transcript data))  $\leq$ 
  Pc data attacker-state
and comp-next-bound:
   $\wedge$ data attacker-state.

```


Some (data, attacker-state) ∈
set-dist (execute (checked-staged-transcript-program A)
adversary-initial-state) ⇒
wp-event verify-monad
(composition-fri-verifier-tied-next-value-replay-gap
(verifier-state-from-adversary attacker-state
(staged-proof-transcript data)))
(verifier-state-from-adversary attacker-state
(staged-proof-transcript data)) ≤
Nc data attacker-state
and *comp-successor-bound:*
 $\wedge$ *data attacker-state.*
Some (data, attacker-state) ∈
set-dist (execute (checked-staged-transcript-program A)
adversary-initial-state) ⇒
wp-event verify-monad
(composition-fri-verifier-tied-successor-replay-gap
(verifier-state-from-adversary attacker-state
(staged-proof-transcript data)))
(verifier-state-from-adversary attacker-state
(staged-proof-transcript data)) ≤
Sc data attacker-state
and *comp-final-bound:*
 $\wedge$ *data attacker-state.*
Some (data, attacker-state) ∈
set-dist (execute (checked-staged-transcript-program A)
adversary-initial-state) ⇒
wp-event verify-monad
(composition-fri-verifier-tied-final-value-replay-gap
(verifier-state-from-adversary attacker-state
(staged-proof-transcript data)))
(verifier-state-from-adversary attacker-state
(staged-proof-transcript data)) ≤
Gc data attacker-state
and *comp-total-bound:*
 $\wedge$ *data attacker-state.*
Some (data, attacker-state) ∈
set-dist (execute (checked-staged-transcript-program A)
adversary-initial-state) ⇒
Rc data attacker-state +
((Pc data attacker-state + 0) +
Nc data attacker-state + Sc data attacker-state +
Gc data attacker-state + Pc data attacker-state) +
0)
 $\leq$ *composition-fri-error*
shows
active-composition-fri-residual-obligations-for A Rc
(λ- -. (0::prob)) Pc Nc Sc Gc
unfolding *active-composition-fri-residual-obligations-for-def*

```

proof (intro allI impI)
  fix data attacker-state
  assume support:
    Some (data, attacker-state) ∈
      set-dist (execute (checked-staged-transcript-program A)
        adversary-initial-state)
  let ?s =
    verifier-state-from-adversary attacker-state
    (staged-proof-transcript data)
  show composition-fri-verifier-tied-residual-obligations ?s
    (Rc data attacker-state) 0 (Pc data attacker-state)
    (Nc data attacker-state) (Sc data attacker-state)
    (Gc data attacker-state)
  by (rule composition-fri-verifier-tied-residual-obligations-from-bounds-slot-zero)
    (rule comp-sampled-bound[OF support],
      rule comp-merkle-bound[OF support],
      rule comp-next-bound[OF support],
      rule comp-successor-bound[OF support],
      rule comp-final-bound[OF support],
      rule comp-total-bound[OF support])
qed

lemma active-fri-reductions-forD:
  assumes active-fri-reductions-for A
  and Some (data, attacker-state) ∈
    set-dist (execute (checked-staged-transcript-program A)
      adversary-initial-state)
  shows
    trace-fri-header-tied-reduction
      (verifier-state-from-adversary attacker-state
        (staged-proof-transcript data))
    composition-fri-verifier-tied-reduction
      (verifier-state-from-adversary attacker-state
        (staged-proof-transcript data))
    trace-fri-empty-header-reduction
      (verifier-state-from-adversary attacker-state
        (staged-proof-transcript data))
  using assms unfolding active-fri-reductions-for-def by blast+

lemma trace-fri-header-tied-reduction-from-residual-obligations:
  assumes
    trace-fri-header-tied-residual-obligations s r-b z-b a-b p-b h-b g-b n-b s-b f-b
  shows trace-fri-header-tied-reduction s
proof –
  have sampled:
    wp-event verify-monad
      (trace-fri-bad-with-header-tied-sampled-layer-chain s) s ≤ r-b
  using assms unfolding trace-fri-header-tied-residual-obligations-def
  by blast

```

have *zero*:
 wp-event verify-monad
 (trace-fri-header-tied-zero-round-final-obstruction *s*) $s \leq z-b$
using *assms* **unfolding** *trace-fri-header-tied-residual-obligations-def*
by *blast*

have *slot*:
 wp-event verify-monad
 (trace-fri-header-tied-authenticated-slot-recorded-chunk-gap *s*) $s \leq a-b$
using *assms* **unfolding** *trace-fri-header-tied-residual-obligations-def*
by *blast*

have *merkle*:
 wp-event verify-monad (partial-merkle-inconsistency-bad *s*) $s \leq p-b$
using *assms* **unfolding** *trace-fri-header-tied-residual-obligations-def*
by *blast*

have *structural*:
 wp-event verify-monad
 (trace-fri-header-tied-sibling-mismatch-structural-gap *s*) $s \leq h-b$
using *assms* **unfolding** *trace-fri-header-tied-residual-obligations-def*
by *blast*

have *head-neq*:
 wp-event verify-monad
 (trace-fri-header-tied-base-head-value-neq-gap *s*) $s \leq g-b$
using *assms* **unfolding** *trace-fri-header-tied-residual-obligations-def*
by *blast*

have *head-mismatch*:
 wp-event verify-monad
 (trace-fri-header-tied-base-head-value-mismatch-gap *s*) $s \leq g-b$
by (rule
 wp-trace-fri-header-tied-base-head-value-mismatch-gap-bound-from-neq-gap
 [OF *head-neq*])

have *alignment*:
 wp-event verify-monad
 (trace-fri-header-tied-base-value-alignment-gap *s*) $s \leq g-b$
by (rule
 wp-trace-fri-header-tied-base-value-alignment-gap-bound-from-head-value-mismatch-gap
 [OF *head-mismatch*])

have *next-bound*:
 wp-event verify-monad
 (trace-fri-header-tied-next-value-replay-gap *s*) $s \leq n-b$
using *assms* **unfolding** *trace-fri-header-tied-residual-obligations-def*
by *blast*

have *successor*:
 wp-event verify-monad
 (trace-fri-header-tied-successor-replay-gap *s*) $s \leq s-b$
using *assms* **unfolding** *trace-fri-header-tied-residual-obligations-def*
by *blast*

have *selected-final*:
 wp-event verify-monad

```

    (trace-fri-header-tied-final-value-selected-step-missing-gap s) s
    ≤ f-b
  using assms unfolding trace-fri-header-tied-residual-obligations-def
  by blast
have final-bound:
  wp-event verify-monad
    (trace-fri-header-tied-final-value-replay-gap s) s ≤
    p-b + a-b + f-b
  by (rule
    wp-trace-fri-header-tied-final-value-replay-gap-bound-from-merkle-slot-and-selected-step)
    (rule merkle, rule slot, rule selected-final)
have total:
  r-b + (((((p-b + h-b) + g-b) + n-b + s-b +
    (p-b + a-b + f-b)) + p-b) + a-b + z-b)
    ≤ trace-fri-error
  using assms unfolding trace-fri-header-tied-residual-obligations-def
  by blast
show ?thesis
  by (rule trace-fri-header-tied-reduction-from-residual-bounds
    [OF sampled zero slot merkle structural alignment next-bound successor final-bound
    total])
qed

```

lemma *composition-fri-verifier-tied-reduction-from-residual-obligations:*

assumes

composition-fri-verifier-tied-residual-obligations s r-b a-b p-b n-b s-b g-b

shows *composition-fri-verifier-tied-reduction s*

proof —

have *sampled:*

wp-event verify-monad
(composition-fri-bad-with-verifier-tied-sampled-layer-chain s) s
 ≤ r-b

using *assms*

unfolding *composition-fri-verifier-tied-residual-obligations-def*

by *blast*

have *slot:*

wp-event verify-monad
(composition-fri-verifier-tied-authenticated-slot-recorded-chunk-gap s)
s ≤ a-b

using *assms*

unfolding *composition-fri-verifier-tied-residual-obligations-def*

by *blast*

have *merkle:*

wp-event verify-monad (partial-merkle-inconsistency-bad s) s ≤ p-b

using *assms*

unfolding *composition-fri-verifier-tied-residual-obligations-def*

by *blast*

have *next-bound:*

wp-event verify-monad

```

    (composition-fri-verifier-tied-next-value-replay-gap s) s ≤ n-b
using assms
unfolding composition-fri-verifier-tied-residual-obligations-def
by blast
have successor:
  wp-event verify-monad
  (composition-fri-verifier-tied-successor-replay-gap s) s ≤ s-b
using assms
unfolding composition-fri-verifier-tied-residual-obligations-def
by blast
have final-bound:
  wp-event verify-monad
  (composition-fri-verifier-tied-final-value-replay-gap s) s ≤ g-b
using assms
unfolding composition-fri-verifier-tied-residual-obligations-def
by blast
have total:
  r-b + (((p-b + a-b) + n-b + s-b + g-b + p-b) + a-b)
    ≤ composition-fri-error
using assms
unfolding composition-fri-verifier-tied-residual-obligations-def
by blast
have total-zero:
  r-b + (((p-b + 0) + n-b + s-b + g-b + p-b) + a-b)
    ≤ composition-fri-error
by (rule order-trans[OF - total]) simp
show ?thesis
by (rule
  composition-fri-verifier-tied-reduction-from-residual-bounds-with-recorded-missing-zero
  [OF sampled slot merkle next-bound successor final-bound total-zero])
qed

lemma active-fri-reductions-from-residual-obligations:
assumes active-fri-residual-obligations-for A Rt Zt At Pt Ht Gt Nt St Ft
  Rc Ac Pc Nc Sc Gc
shows active-fri-reductions-for A
unfolding active-fri-reductions-for-def
proof (intro allI impI conjI)
fix data attacker-state
assume support:
  Some (data, attacker-state) ∈
    set-dist (execute (checked-staged-transcript-program A)
      adversary-initial-state)
let ?s =
  verifier-state-from-adversary attacker-state
  (staged-proof-transcript data)
have trace-obs:
  trace-fri-header-tied-residual-obligations ?s
  (Rt data attacker-state) (Zt data attacker-state)

```

```

    (At data attacker-state) (Pt data attacker-state)
    (Ht data attacker-state) (Gt data attacker-state)
    (Nt data attacker-state) (St data attacker-state)
    (Ft data attacker-state)
  using assms support
  unfolding active-fri-residual-obligations-for-def by (simp add: Let-def)
have composition-obs:
  composition-fri-verifier-tied-residual-obligations ?s
  (Rc data attacker-state) (Ac data attacker-state)
  (Pc data attacker-state)
  (Nc data attacker-state) (Sc data attacker-state)
  (Gc data attacker-state)
  using assms support
  unfolding active-fri-residual-obligations-for-def by (simp add: Let-def)
have empty-obs:
  trace-fri-empty-header-reduction ?s
  using assms support
  unfolding active-fri-residual-obligations-for-def by (simp add: Let-def)
show trace-fri-header-tied-reduction ?s
  by (rule trace-fri-header-tied-reduction-from-residual-obligations
      [OF trace-obs])
show composition-fri-verifier-tied-reduction ?s
  by (rule
      composition-fri-verifier-tied-reduction-from-residual-obligations
      [OF composition-obs])
show trace-fri-empty-header-reduction ?s
  by (rule empty-obs)
qed

```

lemma *active-fri-reductions-from-reachable:*

assumes *trace-reachable:*

$\wedge$ *data attacker-state.*

Some (data, attacker-state) $\in$
 set-dist (execute (checked-staged-transcript-program A)
 adversary-initial-state) $\implies$
 trace-fri-reachable-partial-candidate-reduction-assumption
 (verifier-state-from-adversary attacker-state
 (staged-proof-transcript data))

and *composition-reachable:*

$\wedge$ *data attacker-state.*

Some (data, attacker-state) $\in$
 set-dist (execute (checked-staged-transcript-program A)
 adversary-initial-state) $\implies$
 composition-fri-reachable-partial-candidate-reduction-assumption
 (verifier-state-from-adversary attacker-state
 (staged-proof-transcript data))

shows *active-fri-reductions-for A*

unfolding *active-fri-reductions-for-def*

proof (intro allI impI conjI)

```

fix data attacker-state
assume support:
  Some (data, attacker-state) ∈
    set-dist (execute (checked-staged-transcript-program A)
      adversary-initial-state)
show trace-fri-header-tied-reduction
  (verifier-state-from-adversary attacker-state
    (staged-proof-transcript data))
by (rule trace-fri-header-tied-reduction-from-reachable-reduction)
  (rule trace-reachable[OF support])
show composition-fri-verifier-tied-reduction
  (verifier-state-from-adversary attacker-state
    (staged-proof-transcript data))
by (rule composition-fri-verifier-tied-reduction-from-reachable-reduction)
  (rule composition-reachable[OF support])
show trace-fri-empty-header-reduction
  (verifier-state-from-adversary attacker-state
    (staged-proof-transcript data))
by (rule trace-fri-empty-header-reduction-from-reachable-reduction)
  (rule trace-reachable[OF support])
qed

lemma active-fri-residual-obligations-from-bounds:
fixes
  Rt Zt At Pt Ht Gt Nt St Ft ::
    ('f staged-proof-data ⇒
      'f protocol-channel ⇒ prob)
and Rc Ac Pc Nc Sc Gc ::
    ('f staged-proof-data ⇒
      'f protocol-channel ⇒ prob)
assumes trace-sampled-bound:
  ∧ data attacker-state.
  Some (data, attacker-state) ∈
    set-dist (execute (checked-staged-transcript-program A)
      adversary-initial-state) ⇒
  wp-event verify-monad
    (trace-fri-bad-with-header-tied-sampled-layer-chain
      (verifier-state-from-adversary attacker-state
        (staged-proof-transcript data)))
    (verifier-state-from-adversary attacker-state
      (staged-proof-transcript data)) ≤
  Rt data attacker-state
and trace-zero-bound:
  ∧ data attacker-state.
  Some (data, attacker-state) ∈
    set-dist (execute (checked-staged-transcript-program A)
      adversary-initial-state) ⇒
  wp-event verify-monad
    (trace-fri-header-tied-zero-round-final-obstruction

```

$(\text{verifier-state-from-adversary attacker-state}$
 $(\text{staged-proof-transcript data}))$
 $(\text{verifier-state-from-adversary attacker-state}$
 $(\text{staged-proof-transcript data})) \leq$
 $Zt \text{ data attacker-state}$
and *trace-slot-bound*:
 $\wedge \text{data attacker-state.}$
 $\text{Some (data, attacker-state)} \in$
 $\text{set-dist (execute (checked-staged-transcript-program A)}$
 $\text{adversary-initial-state)} \implies$
 $\text{wp-event verify-monad}$
 $(\text{trace-fri-header-tied-authenticated-slot-recorded-chunk-gap}$
 $(\text{verifier-state-from-adversary attacker-state}$
 $(\text{staged-proof-transcript data}))$
 $(\text{verifier-state-from-adversary attacker-state}$
 $(\text{staged-proof-transcript data})) \leq$
 $At \text{ data attacker-state}$
and *trace-merkle-bound*:
 $\wedge \text{data attacker-state.}$
 $\text{Some (data, attacker-state)} \in$
 $\text{set-dist (execute (checked-staged-transcript-program A)}$
 $\text{adversary-initial-state)} \implies$
 $\text{wp-event verify-monad}$
 $(\text{partial-merkle-inconsistency-bad}$
 $(\text{verifier-state-from-adversary attacker-state}$
 $(\text{staged-proof-transcript data}))$
 $(\text{verifier-state-from-adversary attacker-state}$
 $(\text{staged-proof-transcript data})) \leq$
 $Pt \text{ data attacker-state}$
and *trace-structural-bound*:
 $\wedge \text{data attacker-state.}$
 $\text{Some (data, attacker-state)} \in$
 $\text{set-dist (execute (checked-staged-transcript-program A)}$
 $\text{adversary-initial-state)} \implies$
 $\text{wp-event verify-monad}$
 $(\text{trace-fri-header-tied-sibling-mismatch-structural-gap}$
 $(\text{verifier-state-from-adversary attacker-state}$
 $(\text{staged-proof-transcript data}))$
 $(\text{verifier-state-from-adversary attacker-state}$
 $(\text{staged-proof-transcript data})) \leq$
 $Ht \text{ data attacker-state}$
and *trace-head-neq-bound*:
 $\wedge \text{data attacker-state.}$
 $\text{Some (data, attacker-state)} \in$
 $\text{set-dist (execute (checked-staged-transcript-program A)}$
 $\text{adversary-initial-state)} \implies$
 $\text{wp-event verify-monad}$
 $(\text{trace-fri-header-tied-base-head-value-neq-gap}$
 $(\text{verifier-state-from-adversary attacker-state}$

$(\text{staged-proof-transcript data}))$
 $(\text{verifier-state-from-adversary attacker-state}$
 $(\text{staged-proof-transcript data})) \leq$
 $Gt \text{ data attacker-state}$
and trace-next-bound:
 $\wedge \text{data attacker-state.}$
 $\text{Some (data, attacker-state)} \in$
 $\text{set-dist (execute (checked-staged-transcript-program A)}$
 $\text{adversary-initial-state)} \implies$
 $\text{wp-event verify-monad}$
 $(\text{trace-fri-header-tied-next-value-replay-gap}$
 $(\text{verifier-state-from-adversary attacker-state}$
 $(\text{staged-proof-transcript data}))$
 $(\text{verifier-state-from-adversary attacker-state}$
 $(\text{staged-proof-transcript data})) \leq$
 $Nt \text{ data attacker-state}$
and $\text{trace-successor-bound:}$
 $\wedge \text{data attacker-state.}$
 $\text{Some (data, attacker-state)} \in$
 $\text{set-dist (execute (checked-staged-transcript-program A)}$
 $\text{adversary-initial-state)} \implies$
 $\text{wp-event verify-monad}$
 $(\text{trace-fri-header-tied-successor-replay-gap}$
 $(\text{verifier-state-from-adversary attacker-state}$
 $(\text{staged-proof-transcript data}))$
 $(\text{verifier-state-from-adversary attacker-state}$
 $(\text{staged-proof-transcript data})) \leq$
 $St \text{ data attacker-state}$
and $\text{trace-final-bound:}$
 $\wedge \text{data attacker-state.}$
 $\text{Some (data, attacker-state)} \in$
 $\text{set-dist (execute (checked-staged-transcript-program A)}$
 $\text{adversary-initial-state)} \implies$
 $\text{wp-event verify-monad}$
 $(\text{trace-fri-header-tied-final-value-selected-step-missing-gap}$
 $(\text{verifier-state-from-adversary attacker-state}$
 $(\text{staged-proof-transcript data}))$
 $(\text{verifier-state-from-adversary attacker-state}$
 $(\text{staged-proof-transcript data})) \leq$
 $Ft \text{ data attacker-state}$
and $\text{trace-total-bound:}$
 $\wedge \text{data attacker-state.}$
 $\text{Some (data, attacker-state)} \in$
 $\text{set-dist (execute (checked-staged-transcript-program A)}$
 $\text{adversary-initial-state)} \implies$
 $Rt \text{ data attacker-state} +$
 $(((((Pt \text{ data attacker-state} + Ht \text{ data attacker-state}) +$
 $Gt \text{ data attacker-state}) +$
 $Nt \text{ data attacker-state} + St \text{ data attacker-state} +$

$(Pt \text{ data attacker-state} + At \text{ data attacker-state} +$
 $Ft \text{ data attacker-state})) +$
 $Pt \text{ data attacker-state} +$
 $At \text{ data attacker-state} + Zt \text{ data attacker-state})$
 $\leq \text{trace-fri-error}$
and *comp-sampled-bound*:
 $\wedge \text{data attacker-state.}$
 $\text{Some } (data, \text{attacker-state}) \in$
 $\text{set-dist } (\text{execute } (\text{checked-staged-transcript-program } A)$
 $\text{adversary-initial-state}) \implies$
 $\text{wp-event verify-monad}$
 $(\text{composition-fri-bad-with-verifier-tied-sampled-layer-chain}$
 $(\text{verifier-state-from-adversary attacker-state}$
 $(\text{staged-proof-transcript data})))$
 $(\text{verifier-state-from-adversary attacker-state}$
 $(\text{staged-proof-transcript data})) \leq$
 $Rc \text{ data attacker-state}$
and *comp-slot-bound*:
 $\wedge \text{data attacker-state.}$
 $\text{Some } (data, \text{attacker-state}) \in$
 $\text{set-dist } (\text{execute } (\text{checked-staged-transcript-program } A)$
 $\text{adversary-initial-state}) \implies$
 $\text{wp-event verify-monad}$
 $(\text{composition-fri-verifier-tied-authenticated-slot-recorded-chunk-gap}$
 $(\text{verifier-state-from-adversary attacker-state}$
 $(\text{staged-proof-transcript data})))$
 $(\text{verifier-state-from-adversary attacker-state}$
 $(\text{staged-proof-transcript data})) \leq$
 $Ac \text{ data attacker-state}$
and *comp-merkle-bound*:
 $\wedge \text{data attacker-state.}$
 $\text{Some } (data, \text{attacker-state}) \in$
 $\text{set-dist } (\text{execute } (\text{checked-staged-transcript-program } A)$
 $\text{adversary-initial-state}) \implies$
 $\text{wp-event verify-monad}$
 $(\text{partial-merkle-inconsistency-bad}$
 $(\text{verifier-state-from-adversary attacker-state}$
 $(\text{staged-proof-transcript data})))$
 $(\text{verifier-state-from-adversary attacker-state}$
 $(\text{staged-proof-transcript data})) \leq$
 $Pc \text{ data attacker-state}$
and *comp-next-bound*:
 $\wedge \text{data attacker-state.}$
 $\text{Some } (data, \text{attacker-state}) \in$
 $\text{set-dist } (\text{execute } (\text{checked-staged-transcript-program } A)$
 $\text{adversary-initial-state}) \implies$
 $\text{wp-event verify-monad}$
 $(\text{composition-fri-verifier-tied-next-value-replay-gap}$
 $(\text{verifier-state-from-adversary attacker-state}$

$(\text{staged-proof-transcript } data)))$
 $(\text{verifier-state-from-adversary attacker-state}$
 $(\text{staged-proof-transcript } data)) \leq$
 $Nc \text{ data attacker-state}$
and *comp-successor-bound*:
 $\wedge \text{data attacker-state.}$
 $\text{Some } (data, \text{attacker-state}) \in$
 $\text{set-dist } (\text{execute } (\text{checked-staged-transcript-program } A)$
 $\text{adversary-initial-state}) \implies$
 $\text{wp-event verify-monad}$
 $(\text{composition-fri-verifier-tied-successor-replay-gap}$
 $(\text{verifier-state-from-adversary attacker-state}$
 $(\text{staged-proof-transcript } data)))$
 $(\text{verifier-state-from-adversary attacker-state}$
 $(\text{staged-proof-transcript } data)) \leq$
 $Sc \text{ data attacker-state}$
and *comp-final-bound*:
 $\wedge \text{data attacker-state.}$
 $\text{Some } (data, \text{attacker-state}) \in$
 $\text{set-dist } (\text{execute } (\text{checked-staged-transcript-program } A)$
 $\text{adversary-initial-state}) \implies$
 $\text{wp-event verify-monad}$
 $(\text{composition-fri-verifier-tied-final-value-replay-gap}$
 $(\text{verifier-state-from-adversary attacker-state}$
 $(\text{staged-proof-transcript } data)))$
 $(\text{verifier-state-from-adversary attacker-state}$
 $(\text{staged-proof-transcript } data)) \leq$
 $Gc \text{ data attacker-state}$
and *comp-total-bound*:
 $\wedge \text{data attacker-state.}$
 $\text{Some } (data, \text{attacker-state}) \in$
 $\text{set-dist } (\text{execute } (\text{checked-staged-transcript-program } A)$
 $\text{adversary-initial-state}) \implies$
 $Rc \text{ data attacker-state} +$
 $((Pc \text{ data attacker-state} + Ac \text{ data attacker-state}) +$
 $Nc \text{ data attacker-state} + Sc \text{ data attacker-state} +$
 $Gc \text{ data attacker-state} + Pc \text{ data attacker-state}) +$
 $Ac \text{ data attacker-state})$
 $\leq \text{composition-fri-error}$
and *empty-header-reduction*:
 $\wedge \text{data attacker-state.}$
 $\text{Some } (data, \text{attacker-state}) \in$
 $\text{set-dist } (\text{execute } (\text{checked-staged-transcript-program } A)$
 $\text{adversary-initial-state}) \implies$
 $\text{trace-fri-empty-header-reduction}$
 $(\text{verifier-state-from-adversary attacker-state}$
 $(\text{staged-proof-transcript } data))$
shows *active-fri-residual-obligations-for* $A \text{ Rt Zt At Pt Ht Gt Nt St Ft}$
 $Rc \text{ Ac Pc Nc Sc Gc}$

```

unfolding active-fri-residual-obligations-for-def
proof (intro allI impI)
fix data attacker-state
assume support:
  Some (data, attacker-state) ∈
    set-dist (execute (checked-staged-transcript-program A)
      adversary-initial-state)
show (let s =
  verifier-state-from-adversary attacker-state
    (staged-proof-transcript data)
  in
    trace-fri-header-tied-residual-obligations s
      (Rt data attacker-state) (Zt data attacker-state)
      (At data attacker-state) (Pt data attacker-state)
      (Ht data attacker-state) (Gt data attacker-state)
      (Nt data attacker-state) (St data attacker-state)
      (Ft data attacker-state)  $\wedge$ 
    composition-fri-verifier-tied-residual-obligations s
      (Rc data attacker-state) (Ac data attacker-state)
      (Pc data attacker-state)
      (Nc data attacker-state) (Sc data attacker-state)
      (Gc data attacker-state)  $\wedge$ 
    trace-fri-empty-header-reduction s)
using trace-sampled-bound[OF support] trace-zero-bound[OF support]
  trace-slot-bound[OF support] trace-merkle-bound[OF support]
  trace-structural-bound[OF support] trace-head-neq-bound[OF support]
  trace-next-bound[OF support] trace-successor-bound[OF support]
  trace-final-bound[OF support] trace-total-bound[OF support]
  comp-sampled-bound[OF support] comp-slot-bound[OF support]
  comp-merkle-bound[OF support] comp-next-bound[OF support]
  comp-successor-bound[OF support]
  comp-final-bound[OF support] comp-total-bound[OF support]
  empty-header-reduction[OF support]
unfolding trace-fri-header-tied-residual-obligations-def
  composition-fri-verifier-tied-residual-obligations-def
by (simp add: Let-def)
qed

```

lemma *active-fri-residual-obligations-from-bounds-without-head:*

```

fixes
  Rt Zt At Pt Ht Nt St Ft ::
    ('f staged-proof-data ⇒
      'f protocol-channel ⇒ prob)
and Rc Ac Pc Nc Sc Gc ::
    ('f staged-proof-data ⇒
      'f protocol-channel ⇒ prob)
assumes trace-sampled-bound:
   $\bigwedge$  data attacker-state.
  Some (data, attacker-state) ∈

```

$\text{set-dist } (\text{execute } (\text{checked-staged-transcript-program } A) \text{ adversary-initial-state}) \implies$
 $\text{wp-event verify-monad}$
 $(\text{trace-fri-bad-with-header-tied-sampled-layer-chain}$
 $(\text{verifier-state-from-adversary attacker-state}$
 $(\text{staged-proof-transcript data})))$
 $(\text{verifier-state-from-adversary attacker-state}$
 $(\text{staged-proof-transcript data})) \leq$
 $\text{Rt data attacker-state}$
and trace-zero-bound:
 $\wedge \text{data attacker-state.}$
 $\text{Some } (data, \text{attacker-state}) \in$
 $\text{set-dist } (\text{execute } (\text{checked-staged-transcript-program } A) \text{ adversary-initial-state}) \implies$
 $\text{wp-event verify-monad}$
 $(\text{trace-fri-header-tied-zero-round-final-obstruction}$
 $(\text{verifier-state-from-adversary attacker-state}$
 $(\text{staged-proof-transcript data})))$
 $(\text{verifier-state-from-adversary attacker-state}$
 $(\text{staged-proof-transcript data})) \leq$
 $\text{Zt data attacker-state}$
and trace-slot-bound:
 $\wedge \text{data attacker-state.}$
 $\text{Some } (data, \text{attacker-state}) \in$
 $\text{set-dist } (\text{execute } (\text{checked-staged-transcript-program } A) \text{ adversary-initial-state}) \implies$
 $\text{wp-event verify-monad}$
 $(\text{trace-fri-header-tied-authenticated-slot-recorded-chunk-gap}$
 $(\text{verifier-state-from-adversary attacker-state}$
 $(\text{staged-proof-transcript data})))$
 $(\text{verifier-state-from-adversary attacker-state}$
 $(\text{staged-proof-transcript data})) \leq$
 $\text{At data attacker-state}$
and $\text{trace-merkle-bound:}$
 $\wedge \text{data attacker-state.}$
 $\text{Some } (data, \text{attacker-state}) \in$
 $\text{set-dist } (\text{execute } (\text{checked-staged-transcript-program } A) \text{ adversary-initial-state}) \implies$
 $\text{wp-event verify-monad}$
 $(\text{partial-merkle-inconsistency-bad}$
 $(\text{verifier-state-from-adversary attacker-state}$
 $(\text{staged-proof-transcript data})))$
 $(\text{verifier-state-from-adversary attacker-state}$
 $(\text{staged-proof-transcript data})) \leq$
 $\text{Pt data attacker-state}$
and $\text{trace-structural-bound:}$
 $\wedge \text{data attacker-state.}$
 $\text{Some } (data, \text{attacker-state}) \in$
 $\text{set-dist } (\text{execute } (\text{checked-staged-transcript-program } A)$

$\text{adversary-initial-state}) \implies$
 $\text{wp-event verify-monad}$
 $(\text{trace-fri-header-tied-sibling-mismatch-structural-gap}$
 $(\text{verifier-state-from-adversary attacker-state}$
 $(\text{staged-proof-transcript data})))$
 $(\text{verifier-state-from-adversary attacker-state}$
 $(\text{staged-proof-transcript data})) \leq$
 $\text{Ht data attacker-state}$
and trace-next-bound:
 $\wedge \text{data attacker-state.}$
 $\text{Some (data, attacker-state)} \in$
 $\text{set-dist (execute (checked-staged-transcript-program A)}$
 $\text{adversary-initial-state}) \implies}$
 $\text{wp-event verify-monad}$
 $(\text{trace-fri-header-tied-next-value-replay-gap}$
 $(\text{verifier-state-from-adversary attacker-state}$
 $(\text{staged-proof-transcript data})))$
 $(\text{verifier-state-from-adversary attacker-state}$
 $(\text{staged-proof-transcript data})) \leq$
 $\text{Nt data attacker-state}$
and $\text{trace-successor-bound:}$
 $\wedge \text{data attacker-state.}$
 $\text{Some (data, attacker-state)} \in$
 $\text{set-dist (execute (checked-staged-transcript-program A)}$
 $\text{adversary-initial-state}) \implies}$
 $\text{wp-event verify-monad}$
 $(\text{trace-fri-header-tied-successor-replay-gap}$
 $(\text{verifier-state-from-adversary attacker-state}$
 $(\text{staged-proof-transcript data})))$
 $(\text{verifier-state-from-adversary attacker-state}$
 $(\text{staged-proof-transcript data})) \leq$
 $\text{St data attacker-state}$
and $\text{trace-final-bound:}$
 $\wedge \text{data attacker-state.}$
 $\text{Some (data, attacker-state)} \in$
 $\text{set-dist (execute (checked-staged-transcript-program A)}$
 $\text{adversary-initial-state}) \implies}$
 $\text{wp-event verify-monad}$
 $(\text{trace-fri-header-tied-final-value-selected-step-missing-gap}$
 $(\text{verifier-state-from-adversary attacker-state}$
 $(\text{staged-proof-transcript data})))$
 $(\text{verifier-state-from-adversary attacker-state}$
 $(\text{staged-proof-transcript data})) \leq$
 $\text{Ft data attacker-state}$
and $\text{trace-total-bound:}$
 $\wedge \text{data attacker-state.}$
 $\text{Some (data, attacker-state)} \in$
 $\text{set-dist (execute (checked-staged-transcript-program A)}$
 $\text{adversary-initial-state}) \implies}$

$Rt \text{ data attacker-state} +$
 $(((((Pt \text{ data attacker-state} + Ht \text{ data attacker-state}) +$
 $Pt \text{ data attacker-state}) +$
 $Nt \text{ data attacker-state} + St \text{ data attacker-state} +$
 $(Pt \text{ data attacker-state} + At \text{ data attacker-state} +$
 $Ft \text{ data attacker-state})) +$
 $Pt \text{ data attacker-state}) +$
 $At \text{ data attacker-state} + Zt \text{ data attacker-state})$
 $\leq \text{trace-fri-error}$
and comp-sampled-bound:
 $\wedge \text{data attacker-state.}$
 $\text{Some (data, attacker-state)} \in$
 $\text{set-dist (execute (checked-staged-transcript-program } A)$
 $\text{adversary-initial-state})} \implies$
 $\text{wp-event verify-monad}$
 $(\text{composition-fri-bad-with-verifier-tied-sampled-layer-chain}$
 $(\text{verifier-state-from-adversary attacker-state}$
 $(\text{staged-proof-transcript data})))$
 $(\text{verifier-state-from-adversary attacker-state}$
 $(\text{staged-proof-transcript data})) \leq$
 $Rc \text{ data attacker-state}$
and comp-slot-bound:
 $\wedge \text{data attacker-state.}$
 $\text{Some (data, attacker-state)} \in$
 $\text{set-dist (execute (checked-staged-transcript-program } A)$
 $\text{adversary-initial-state})} \implies$
 $\text{wp-event verify-monad}$
 $(\text{composition-fri-verifier-tied-authenticated-slot-recorded-chunk-gap}$
 $(\text{verifier-state-from-adversary attacker-state}$
 $(\text{staged-proof-transcript data})))$
 $(\text{verifier-state-from-adversary attacker-state}$
 $(\text{staged-proof-transcript data})) \leq$
 $Ac \text{ data attacker-state}$
and comp-merkle-bound:
 $\wedge \text{data attacker-state.}$
 $\text{Some (data, attacker-state)} \in$
 $\text{set-dist (execute (checked-staged-transcript-program } A)$
 $\text{adversary-initial-state})} \implies$
 $\text{wp-event verify-monad}$
 $(\text{partial-merkle-inconsistency-bad}$
 $(\text{verifier-state-from-adversary attacker-state}$
 $(\text{staged-proof-transcript data})))$
 $(\text{verifier-state-from-adversary attacker-state}$
 $(\text{staged-proof-transcript data})) \leq$
 $Pc \text{ data attacker-state}$
and comp-next-bound:
 $\wedge \text{data attacker-state.}$
 $\text{Some (data, attacker-state)} \in$
 $\text{set-dist (execute (checked-staged-transcript-program } A)$

$\text{adversary-initial-state}) \implies$
 $\text{wp-event verify-monad}$
 $(\text{composition-fri-verifier-tied-next-value-replay-gap}$
 $(\text{verifier-state-from-adversary attacker-state}$
 $(\text{staged-proof-transcript data})))$
 $(\text{verifier-state-from-adversary attacker-state}$
 $(\text{staged-proof-transcript data})) \leq$
 $\text{Nc data attacker-state}$
and comp-successor-bound:
 $\wedge \text{data attacker-state.}$
 $\text{Some (data, attacker-state)} \in$
 $\text{set-dist (execute (checked-staged-transcript-program A)}$
 $\text{adversary-initial-state}) \implies}$
 $\text{wp-event verify-monad}$
 $(\text{composition-fri-verifier-tied-successor-replay-gap}$
 $(\text{verifier-state-from-adversary attacker-state}$
 $(\text{staged-proof-transcript data})))$
 $(\text{verifier-state-from-adversary attacker-state}$
 $(\text{staged-proof-transcript data})) \leq$
 $\text{Sc data attacker-state}$
and comp-final-bound:
 $\wedge \text{data attacker-state.}$
 $\text{Some (data, attacker-state)} \in$
 $\text{set-dist (execute (checked-staged-transcript-program A)}$
 $\text{adversary-initial-state}) \implies}$
 $\text{wp-event verify-monad}$
 $(\text{composition-fri-verifier-tied-final-value-replay-gap}$
 $(\text{verifier-state-from-adversary attacker-state}$
 $(\text{staged-proof-transcript data})))$
 $(\text{verifier-state-from-adversary attacker-state}$
 $(\text{staged-proof-transcript data})) \leq$
 $\text{Gc data attacker-state}$
and comp-total-bound:
 $\wedge \text{data attacker-state.}$
 $\text{Some (data, attacker-state)} \in$
 $\text{set-dist (execute (checked-staged-transcript-program A)}$
 $\text{adversary-initial-state}) \implies}$
 $\text{Rc data attacker-state} +$
 $((\text{Pc data attacker-state} + \text{Ac data attacker-state}) +$
 $\text{Nc data attacker-state} + \text{Sc data attacker-state} +$
 $\text{Gc data attacker-state} + \text{Pc data attacker-state}) +$
 $\text{Ac data attacker-state})$
 $\leq \text{composition-fri-error}$
and empty-header-reduction:
 $\wedge \text{data attacker-state.}$
 $\text{Some (data, attacker-state)} \in$
 $\text{set-dist (execute (checked-staged-transcript-program A)}$
 $\text{adversary-initial-state}) \implies}$
 $\text{trace-fri-empty-header-reduction}$


```

    (verifier-state-from-adversary attacker-state
     (staged-proof-transcript data))
shows active-fri-residual-obligations-for A Rt Zt At Pt Ht Pt Nt St Ft
    Rc Ac Pc Nc Sc Gc
proof (rule active-fri-residual-obligations-from-bounds)
fix data attacker-state
assume support:
    Some (data, attacker-state) ∈
      set-dist (execute (checked-staged-transcript-program A)
                    adversary-initial-state)
show wp-event verify-monad
    (trace-fri-bad-with-header-tied-sampled-layer-chain
     (verifier-state-from-adversary attacker-state
      (staged-proof-transcript data)))
    (verifier-state-from-adversary attacker-state
     (staged-proof-transcript data)) ≤
    Rt data attacker-state
by (rule trace-sampled-bound[OF support])
show wp-event verify-monad
    (trace-fri-header-tied-zero-round-final-obstruction
     (verifier-state-from-adversary attacker-state
      (staged-proof-transcript data)))
    (verifier-state-from-adversary attacker-state
     (staged-proof-transcript data)) ≤
    Zt data attacker-state
by (rule trace-zero-bound[OF support])
show wp-event verify-monad
    (trace-fri-header-tied-authenticated-slot-recorded-chunk-gap
     (verifier-state-from-adversary attacker-state
      (staged-proof-transcript data)))
    (verifier-state-from-adversary attacker-state
     (staged-proof-transcript data)) ≤
    At data attacker-state
by (rule trace-slot-bound[OF support])
show wp-event verify-monad
    (partial-merkle-inconsistency-bad
     (verifier-state-from-adversary attacker-state
      (staged-proof-transcript data)))
    (verifier-state-from-adversary attacker-state
     (staged-proof-transcript data)) ≤
    Pt data attacker-state
by (rule trace-merkle-bound[OF support])
show wp-event verify-monad
    (trace-fri-header-tied-sibling-mismatch-structural-gap
     (verifier-state-from-adversary attacker-state
      (staged-proof-transcript data)))
    (verifier-state-from-adversary attacker-state
     (staged-proof-transcript data)) ≤
    Ht data attacker-state

```

```

    by (rule trace-structural-bound[OF support])
  show wp-event verify-monad
    (trace-fri-header-tied-base-head-value-neq-gap
      (verifier-state-from-adversary attacker-state
        (staged-proof-transcript data)))
    (verifier-state-from-adversary attacker-state
      (staged-proof-transcript data)) ≤
  Pt data attacker-state
  by (rule
    wp-trace-fri-header-tied-base-head-value-neq-gap-bound-from-partial-merkle)
    (rule trace-merkle-bound[OF support])
  show wp-event verify-monad
    (trace-fri-header-tied-next-value-replay-gap
      (verifier-state-from-adversary attacker-state
        (staged-proof-transcript data)))
    (verifier-state-from-adversary attacker-state
      (staged-proof-transcript data)) ≤
  Nt data attacker-state
  by (rule trace-next-bound[OF support])
  show wp-event verify-monad
    (trace-fri-header-tied-successor-replay-gap
      (verifier-state-from-adversary attacker-state
        (staged-proof-transcript data)))
    (verifier-state-from-adversary attacker-state
      (staged-proof-transcript data)) ≤
  St data attacker-state
  by (rule trace-successor-bound[OF support])
  show wp-event verify-monad
    (trace-fri-header-tied-final-value-selected-step-missing-gap
      (verifier-state-from-adversary attacker-state
        (staged-proof-transcript data)))
    (verifier-state-from-adversary attacker-state
      (staged-proof-transcript data)) ≤
  Ft data attacker-state
  by (rule trace-final-bound[OF support])
  show Rt data attacker-state +
    (((((Pt data attacker-state + Ht data attacker-state) +
      Pt data attacker-state) +
      Nt data attacker-state + St data attacker-state +
      (Pt data attacker-state + At data attacker-state +
        Ft data attacker-state)) +
      Pt data attacker-state) +
      At data attacker-state + Zt data attacker-state)
    ≤ trace-fri-error
  by (rule trace-total-bound[OF support])
  show wp-event verify-monad
    (composition-fri-bad-with-verifier-tied-sampled-layer-chain
      (verifier-state-from-adversary attacker-state
        (staged-proof-transcript data)))

```

```

    (verifier-state-from-adversary attacker-state
      (staged-proof-transcript data)) ≤
  Rc data attacker-state
  by (rule comp-sampled-bound[OF support])
show wp-event verify-monad
  (composition-fri-verifier-tied-authenticated-slot-recorded-chunk-gap
    (verifier-state-from-adversary attacker-state
      (staged-proof-transcript data)))
  (verifier-state-from-adversary attacker-state
    (staged-proof-transcript data)) ≤
  Ac data attacker-state
  by (rule comp-slot-bound[OF support])
show wp-event verify-monad
  (partial-merkle-inconsistency-bad
    (verifier-state-from-adversary attacker-state
      (staged-proof-transcript data)))
  (verifier-state-from-adversary attacker-state
    (staged-proof-transcript data)) ≤
  Pc data attacker-state
  by (rule comp-merkle-bound[OF support])
show wp-event verify-monad
  (composition-fri-verifier-tied-next-value-replay-gap
    (verifier-state-from-adversary attacker-state
      (staged-proof-transcript data)))
  (verifier-state-from-adversary attacker-state
    (staged-proof-transcript data)) ≤
  Nc data attacker-state
  by (rule comp-next-bound[OF support])
show wp-event verify-monad
  (composition-fri-verifier-tied-successor-replay-gap
    (verifier-state-from-adversary attacker-state
      (staged-proof-transcript data)))
  (verifier-state-from-adversary attacker-state
    (staged-proof-transcript data)) ≤
  Sc data attacker-state
  by (rule comp-successor-bound[OF support])
show wp-event verify-monad
  (composition-fri-verifier-tied-final-value-replay-gap
    (verifier-state-from-adversary attacker-state
      (staged-proof-transcript data)))
  (verifier-state-from-adversary attacker-state
    (staged-proof-transcript data)) ≤
  Gc data attacker-state
  by (rule comp-final-bound[OF support])
show Rc data attacker-state +
  (((Pc data attacker-state + Ac data attacker-state) +
    Nc data attacker-state + Sc data attacker-state +
    Gc data attacker-state + Pc data attacker-state) +
    Ac data attacker-state)

```

```

    ≤ composition-fri-error
  by (rule comp-total-bound[OF support])
show trace-fri-empty-header-reduction
  (verifier-state-from-adversary attacker-state
   (staged-proof-transcript data))
  by (rule empty-header-reduction[OF support])
qed

lemma active-fri-residual-obligations-from-bounds-without-head-and-selected-zero:
fixes
  Rt Zt At Pt Ht Nt St ::
    ('f staged-proof-data ⇒
     'f protocol-channel ⇒ prob)
and Rc Ac Pc Nc Sc Gc ::
    ('f staged-proof-data ⇒
     'f protocol-channel ⇒ prob)
assumes trace-sampled-bound:
  ∧ data attacker-state.
    Some (data, attacker-state) ∈
      set-dist (execute (checked-staged-transcript-program A)
                     adversary-initial-state) ⇒
      wp-event verify-monad
        (trace-fri-bad-with-header-tied-sampled-layer-chain
         (verifier-state-from-adversary attacker-state
          (staged-proof-transcript data)))
        (verifier-state-from-adversary attacker-state
         (staged-proof-transcript data)) ≤
      Rt data attacker-state
and trace-zero-bound:
  ∧ data attacker-state.
    Some (data, attacker-state) ∈
      set-dist (execute (checked-staged-transcript-program A)
                     adversary-initial-state) ⇒
      wp-event verify-monad
        (trace-fri-header-tied-zero-round-final-obstruction
         (verifier-state-from-adversary attacker-state
          (staged-proof-transcript data)))
        (verifier-state-from-adversary attacker-state
         (staged-proof-transcript data)) ≤
      Zt data attacker-state
and trace-slot-bound:
  ∧ data attacker-state.
    Some (data, attacker-state) ∈
      set-dist (execute (checked-staged-transcript-program A)
                     adversary-initial-state) ⇒
      wp-event verify-monad
        (trace-fri-header-tied-authenticated-slot-recorded-chunk-gap
         (verifier-state-from-adversary attacker-state
          (staged-proof-transcript data)))

```

$(\text{verifier-state-from-adversary attacker-state}$
 $(\text{staged-proof-transcript data})) \leq$
 $\text{At data attacker-state}$
and $\text{trace-merkle-bound:}$
 $\wedge \text{data attacker-state.}$
 $\text{Some (data, attacker-state)} \in$
 $\text{set-dist (execute (checked-staged-transcript-program A)}$
 $\text{adversary-initial-state)} \implies$
 $\text{wp-event verify-monad}$
 $(\text{partial-merkle-inconsistency-bad}$
 $(\text{verifier-state-from-adversary attacker-state}$
 $(\text{staged-proof-transcript data})))$
 $(\text{verifier-state-from-adversary attacker-state}$
 $(\text{staged-proof-transcript data})) \leq$
 $\text{Pt data attacker-state}$
and $\text{trace-structural-bound:}$
 $\wedge \text{data attacker-state.}$
 $\text{Some (data, attacker-state)} \in$
 $\text{set-dist (execute (checked-staged-transcript-program A)}$
 $\text{adversary-initial-state)} \implies$
 $\text{wp-event verify-monad}$
 $(\text{trace-fri-header-tied-sibling-mismatch-structural-gap}$
 $(\text{verifier-state-from-adversary attacker-state}$
 $(\text{staged-proof-transcript data})))$
 $(\text{verifier-state-from-adversary attacker-state}$
 $(\text{staged-proof-transcript data})) \leq$
 $\text{Ht data attacker-state}$
and trace-next-bound:
 $\wedge \text{data attacker-state.}$
 $\text{Some (data, attacker-state)} \in$
 $\text{set-dist (execute (checked-staged-transcript-program A)}$
 $\text{adversary-initial-state)} \implies$
 $\text{wp-event verify-monad}$
 $(\text{trace-fri-header-tied-next-value-replay-gap}$
 $(\text{verifier-state-from-adversary attacker-state}$
 $(\text{staged-proof-transcript data})))$
 $(\text{verifier-state-from-adversary attacker-state}$
 $(\text{staged-proof-transcript data})) \leq$
 $\text{Nt data attacker-state}$
and $\text{trace-successor-bound:}$
 $\wedge \text{data attacker-state.}$
 $\text{Some (data, attacker-state)} \in$
 $\text{set-dist (execute (checked-staged-transcript-program A)}$
 $\text{adversary-initial-state)} \implies$
 $\text{wp-event verify-monad}$
 $(\text{trace-fri-header-tied-successor-replay-gap}$
 $(\text{verifier-state-from-adversary attacker-state}$
 $(\text{staged-proof-transcript data})))$
 $(\text{verifier-state-from-adversary attacker-state}$

$(\text{staged-proof-transcript data}) \leq$
 $\text{St data attacker-state}$
and *trace-total-bound*:
 $\wedge \text{data attacker-state.}$
 $\text{Some (data, attacker-state)} \in$
 $\text{set-dist (execute (checked-staged-transcript-program A)}$
 $\text{adversary-initial-state)} \implies$
 $\text{Rt data attacker-state} +$
 $(((((\text{Pt data attacker-state} + \text{Ht data attacker-state}) +$
 $\text{Pt data attacker-state}) +$
 $\text{Nt data attacker-state} + \text{St data attacker-state} +$
 $(\text{Pt data attacker-state} + \text{At data attacker-state} + 0)) +$
 $\text{Pt data attacker-state}) +$
 $\text{At data attacker-state} + \text{Zt data attacker-state})$
 $\leq \text{trace-fri-error}$
and *comp-sampled-bound*:
 $\wedge \text{data attacker-state.}$
 $\text{Some (data, attacker-state)} \in$
 $\text{set-dist (execute (checked-staged-transcript-program A)}$
 $\text{adversary-initial-state)} \implies$
 $\text{wp-event verify-monad}$
 $(\text{composition-fri-bad-with-verifier-tied-sampled-layer-chain}$
 $(\text{verifier-state-from-adversary attacker-state}$
 $(\text{staged-proof-transcript data})))$
 $(\text{verifier-state-from-adversary attacker-state}$
 $(\text{staged-proof-transcript data})) \leq$
 $\text{Rc data attacker-state}$
and *comp-slot-bound*:
 $\wedge \text{data attacker-state.}$
 $\text{Some (data, attacker-state)} \in$
 $\text{set-dist (execute (checked-staged-transcript-program A)}$
 $\text{adversary-initial-state)} \implies$
 $\text{wp-event verify-monad}$
 $(\text{composition-fri-verifier-tied-authenticated-slot-recorded-chunk-gap}$
 $(\text{verifier-state-from-adversary attacker-state}$
 $(\text{staged-proof-transcript data})))$
 $(\text{verifier-state-from-adversary attacker-state}$
 $(\text{staged-proof-transcript data})) \leq$
 $\text{Ac data attacker-state}$
and *comp-merkle-bound*:
 $\wedge \text{data attacker-state.}$
 $\text{Some (data, attacker-state)} \in$
 $\text{set-dist (execute (checked-staged-transcript-program A)}$
 $\text{adversary-initial-state)} \implies$
 $\text{wp-event verify-monad}$
 $(\text{partial-merkle-inconsistency-bad}$
 $(\text{verifier-state-from-adversary attacker-state}$
 $(\text{staged-proof-transcript data})))$
 $(\text{verifier-state-from-adversary attacker-state}$

$(\text{staged-proof-transcript data}) \leq$
 $Pc \text{ data attacker-state}$
and *comp-next-bound*:
 $\wedge \text{data attacker-state.}$
 $\text{Some (data, attacker-state)} \in$
 $\text{set-dist (execute (checked-staged-transcript-program A)}$
 $\text{adversary-initial-state)} \implies$
 $\text{wp-event verify-monad}$
 $(\text{composition-fri-verifier-tied-next-value-replay-gap}$
 $(\text{verifier-state-from-adversary attacker-state}$
 $(\text{staged-proof-transcript data})))$
 $(\text{verifier-state-from-adversary attacker-state}$
 $(\text{staged-proof-transcript data})) \leq$
 $Nc \text{ data attacker-state}$
and *comp-successor-bound*:
 $\wedge \text{data attacker-state.}$
 $\text{Some (data, attacker-state)} \in$
 $\text{set-dist (execute (checked-staged-transcript-program A)}$
 $\text{adversary-initial-state)} \implies$
 $\text{wp-event verify-monad}$
 $(\text{composition-fri-verifier-tied-successor-replay-gap}$
 $(\text{verifier-state-from-adversary attacker-state}$
 $(\text{staged-proof-transcript data})))$
 $(\text{verifier-state-from-adversary attacker-state}$
 $(\text{staged-proof-transcript data})) \leq$
 $Sc \text{ data attacker-state}$
and *comp-final-bound*:
 $\wedge \text{data attacker-state.}$
 $\text{Some (data, attacker-state)} \in$
 $\text{set-dist (execute (checked-staged-transcript-program A)}$
 $\text{adversary-initial-state)} \implies$
 $\text{wp-event verify-monad}$
 $(\text{composition-fri-verifier-tied-final-value-replay-gap}$
 $(\text{verifier-state-from-adversary attacker-state}$
 $(\text{staged-proof-transcript data})))$
 $(\text{verifier-state-from-adversary attacker-state}$
 $(\text{staged-proof-transcript data})) \leq$
 $Gc \text{ data attacker-state}$
and *comp-total-bound*:
 $\wedge \text{data attacker-state.}$
 $\text{Some (data, attacker-state)} \in$
 $\text{set-dist (execute (checked-staged-transcript-program A)}$
 $\text{adversary-initial-state)} \implies$
 $Rc \text{ data attacker-state} +$
 $((Pc \text{ data attacker-state} + Ac \text{ data attacker-state}) +$
 $Nc \text{ data attacker-state} + Sc \text{ data attacker-state} +$
 $Gc \text{ data attacker-state} + Pc \text{ data attacker-state}) +$
 $Ac \text{ data attacker-state})$
 $\leq \text{composition-fri-error}$

and *empty-header-reduction*:
 $\wedge$ *data attacker-state*.
Some (*data*, *attacker-state*) $\in$
set-dist (*execute* (*checked-staged-transcript-program* *A*)
adversary-initial-state) $\implies$
trace-fri-empty-header-reduction
(*verifier-state-from-adversary attacker-state*
(*staged-proof-transcript data*))
shows *active-fri-residual-obligations-for* *A Rt Zt At Pt Ht Pt Nt St*
(λ - $\cdot$. ($0::prob$)) *Rc Ac Pc Nc Sc Gc*
proof (*rule active-fri-residual-obligations-from-components*)
show *active-trace-fri-residual-obligations-for* *A Rt Zt At Pt Ht Pt*
Nt St (λ - $\cdot$. ($0::prob$))
by (*rule*
active-trace-fri-residual-obligations-from-bounds-without-head-and-selected-zero)
(*erule trace-sampled-bound*, *erule trace-zero-bound*,
erule trace-slot-bound, *erule trace-merkle-bound*,
erule trace-structural-bound, *erule trace-next-bound*,
erule trace-successor-bound, *erule trace-total-bound*)
show *active-composition-fri-residual-obligations-for* *A Rc Ac Pc Nc Sc Gc*
by (*rule active-composition-fri-residual-obligations-from-bounds*)
(*erule comp-sampled-bound*, *erule comp-slot-bound*,
erule comp-merkle-bound, *erule comp-next-bound*,
erule comp-successor-bound, *erule comp-final-bound*,
erule comp-total-bound)
show *active-empty-trace-fri-reduction-for* *A*
unfolding *active-empty-trace-fri-reduction-for-def*
by (*intro allI impI*, *erule empty-header-reduction*)
qed

lemma *active-fri-residual-obligations-from-bounds-without-head-slot-and-selected-zero*:

fixes

Rt Zt Pt Ht Nt St ::

(*'f staged-proof-data* $\Rightarrow$

'f protocol-channel $\Rightarrow$ *prob*)

and *Rc Pc Nc Sc Gc* ::

(*'f staged-proof-data* $\Rightarrow$

'f protocol-channel $\Rightarrow$ *prob*)

assumes *trace-sampled-bound*:

$\wedge$ *data attacker-state*.

Some (*data*, *attacker-state*) $\in$

set-dist (*execute* (*checked-staged-transcript-program* *A*)

adversary-initial-state) $\implies$

wp-event verify-monad

(*trace-fri-bad-with-header-tied-sampled-layer-chain*

(*verifier-state-from-adversary attacker-state*

(*staged-proof-transcript data*)))

(*verifier-state-from-adversary attacker-state*

(*staged-proof-transcript data*)) $\leq$

Rt data attacker-state
and *trace-zero-bound:*
 $\wedge$ *data attacker-state.*
Some (data, attacker-state) $\in$
set-dist (execute (checked-staged-transcript-program A)
adversary-initial-state) $\implies$
wp-event verify-monad
(trace-fri-header-tied-zero-round-final-obstruction
(verifier-state-from-adversary attacker-state
(staged-proof-transcript data)))
(verifier-state-from-adversary attacker-state
(staged-proof-transcript data)) $\leq$
Zt data attacker-state
and *trace-merkle-bound:*
 $\wedge$ *data attacker-state.*
Some (data, attacker-state) $\in$
set-dist (execute (checked-staged-transcript-program A)
adversary-initial-state) $\implies$
wp-event verify-monad
(partial-merkle-inconsistency-bad
(verifier-state-from-adversary attacker-state
(staged-proof-transcript data)))
(verifier-state-from-adversary attacker-state
(staged-proof-transcript data)) $\leq$
Pt data attacker-state
and *trace-structural-bound:*
 $\wedge$ *data attacker-state.*
Some (data, attacker-state) $\in$
set-dist (execute (checked-staged-transcript-program A)
adversary-initial-state) $\implies$
wp-event verify-monad
(trace-fri-header-tied-sibling-mismatch-structural-gap
(verifier-state-from-adversary attacker-state
(staged-proof-transcript data)))
(verifier-state-from-adversary attacker-state
(staged-proof-transcript data)) $\leq$
Ht data attacker-state
and *trace-next-bound:*
 $\wedge$ *data attacker-state.*
Some (data, attacker-state) $\in$
set-dist (execute (checked-staged-transcript-program A)
adversary-initial-state) $\implies$
wp-event verify-monad
(trace-fri-header-tied-next-value-replay-gap
(verifier-state-from-adversary attacker-state
(staged-proof-transcript data)))
(verifier-state-from-adversary attacker-state
(staged-proof-transcript data)) $\leq$
Nt data attacker-state

and *trace-successor-bound*:
 $\wedge$ *data attacker-state*.
Some (*data*, *attacker-state*) $\in$
set-dist (*execute* (*checked-staged-transcript-program* *A*)
adversary-initial-state) $\implies$
wp-event verify-monad
(*trace-fri-header-tied-successor-replay-gap*
(*verifier-state-from-adversary attacker-state*
(*staged-proof-transcript data*)))
(*verifier-state-from-adversary attacker-state*
(*staged-proof-transcript data*)) $\leq$
St data attacker-state
and *trace-total-bound*:
 $\wedge$ *data attacker-state*.
Some (*data*, *attacker-state*) $\in$
set-dist (*execute* (*checked-staged-transcript-program* *A*)
adversary-initial-state) $\implies$
Rt data attacker-state +
((((*Pt data attacker-state* + *Ht data attacker-state*) +
Pt data attacker-state) +
Nt data attacker-state + *St data attacker-state* +
(*Pt data attacker-state* + 0 + 0)) +
Pt data attacker-state) +
0 + *Zt data attacker-state*)
 $\leq$ *trace-fri-error*
and *comp-sampled-bound*:
 $\wedge$ *data attacker-state*.
Some (*data*, *attacker-state*) $\in$
set-dist (*execute* (*checked-staged-transcript-program* *A*)
adversary-initial-state) $\implies$
wp-event verify-monad
(*composition-fri-bad-with-verifier-tied-sampled-layer-chain*
(*verifier-state-from-adversary attacker-state*
(*staged-proof-transcript data*)))
(*verifier-state-from-adversary attacker-state*
(*staged-proof-transcript data*)) $\leq$
Rc data attacker-state
and *comp-merkle-bound*:
 $\wedge$ *data attacker-state*.
Some (*data*, *attacker-state*) $\in$
set-dist (*execute* (*checked-staged-transcript-program* *A*)
adversary-initial-state) $\implies$
wp-event verify-monad
(*partial-merkle-inconsistency-bad*
(*verifier-state-from-adversary attacker-state*
(*staged-proof-transcript data*)))
(*verifier-state-from-adversary attacker-state*
(*staged-proof-transcript data*)) $\leq$
Pc data attacker-state

and *comp-next-bound*:

$\wedge$ *data attacker-state*.

Some (*data*, *attacker-state*) $\in$
set-dist (*execute* (*checked-staged-transcript-program* *A*)
adversary-initial-state) $\implies$

wp-event verify-monad
(*composition-fri-verifier-tied-next-value-replay-gap*
(*verifier-state-from-adversary attacker-state*
(*staged-proof-transcript data*)))
(*verifier-state-from-adversary attacker-state*
(*staged-proof-transcript data*)) $\leq$

Nc data attacker-state

and *comp-successor-bound*:

$\wedge$ *data attacker-state*.

Some (*data*, *attacker-state*) $\in$
set-dist (*execute* (*checked-staged-transcript-program* *A*)
adversary-initial-state) $\implies$

wp-event verify-monad
(*composition-fri-verifier-tied-successor-replay-gap*
(*verifier-state-from-adversary attacker-state*
(*staged-proof-transcript data*)))
(*verifier-state-from-adversary attacker-state*
(*staged-proof-transcript data*)) $\leq$

Sc data attacker-state

and *comp-final-bound*:

$\wedge$ *data attacker-state*.

Some (*data*, *attacker-state*) $\in$
set-dist (*execute* (*checked-staged-transcript-program* *A*)
adversary-initial-state) $\implies$

wp-event verify-monad
(*composition-fri-verifier-tied-final-value-replay-gap*
(*verifier-state-from-adversary attacker-state*
(*staged-proof-transcript data*)))
(*verifier-state-from-adversary attacker-state*
(*staged-proof-transcript data*)) $\leq$

Gc data attacker-state

and *comp-total-bound*:

$\wedge$ *data attacker-state*.

Some (*data*, *attacker-state*) $\in$
set-dist (*execute* (*checked-staged-transcript-program* *A*)
adversary-initial-state) $\implies$

Rc data attacker-state +
(((*Pc data attacker-state* + 0) +
Nc data attacker-state + *Sc data attacker-state* +
Gc data attacker-state + *Pc data attacker-state*) +
0)

$\leq$ *composition-fri-error*

and *empty-header-reduction*:

$\wedge$ *data attacker-state*.

Some (data, attacker-state) ∈
set-dist (execute (checked-staged-transcript-program A)
adversary-initial-state) ⇒
trace-fri-empty-header-reduction
(verifier-state-from-adversary attacker-state
(staged-proof-transcript data))
shows
active-fri-residual-obligations-for A Rt Zt (λ- -. (0::prob))
Pt Ht Pt Nt St (λ- -. (0::prob)) Rc (λ- -. (0::prob))
Pc Nc Sc Gc
proof (rule *active-fri-residual-obligations-from-components*)
show *active-trace-fri-residual-obligations-for A Rt Zt*
(λ- -. (0::prob)) Pt Ht Pt Nt St (λ- -. (0::prob))
by (rule
active-trace-fri-residual-obligations-from-bounds-without-head-slot-and-selected-zero)
(erule trace-sampled-bound, erule trace-zero-bound,
erule trace-merkle-bound, erule trace-structural-bound,
erule trace-next-bound, erule trace-successor-bound,
erule trace-total-bound)
show *active-composition-fri-residual-obligations-for A Rc*
(λ- -. (0::prob)) Pc Nc Sc Gc
by (rule *active-composition-fri-residual-obligations-from-bounds-slot-zero*)
(erule comp-sampled-bound, erule comp-merkle-bound,
erule comp-next-bound, erule comp-successor-bound,
erule comp-final-bound, erule comp-total-bound)
show *active-empty-trace-fri-reduction-for A*
unfolding *active-empty-trace-fri-reduction-for-def*
by (intro *allI impI*, erule *empty-header-reduction*)
qed

lemmas *active-fri-reductions-from-residual-bounds =*
active-fri-reductions-from-residual-obligations[
OF active-fri-residual-obligations-from-bounds]

lemmas *active-fri-reductions-from-residual-bounds-without-head =*
active-fri-reductions-from-residual-obligations[
OF active-fri-residual-obligations-from-bounds-without-head]

lemmas *active-fri-reductions-from-residual-bounds-without-head-and-selected-zero =*
active-fri-reductions-from-residual-obligations[
OF active-fri-residual-obligations-from-bounds-without-head-and-selected-zero]

lemmas *active-fri-reductions-from-residual-bounds-without-head-slot-and-selected-zero =*
active-fri-reductions-from-residual-obligations[
OF active-fri-residual-obligations-from-bounds-without-head-slot-and-selected-zero]

lemma *active-fri-reductions-staged-trace-bound:*
assumes *active: active-fri-reductions-for A*
shows

wp-event (checked-staged-security-experiment-with-data-state A)
(staged-security-with-data-state-verifier-event
trace-fri-bad-with-header-tied-partial-candidate)
adversary-initial-state $\leq$ trace-fri-error
by (rule trace-fri-header-tied-reduction-staged-bound)
(rule active-fri-reductions-forD(1)[OF active])

lemma active-fri-reductions-staged-composition-bound:

assumes active: active-fri-reductions-for A

shows

wp-event (checked-staged-security-experiment-with-data-state A)
(staged-security-with-data-state-verifier-event
composition-fri-bad-with-verifier-tied-partial-candidate)
adversary-initial-state $\leq$ composition-fri-error
by (rule composition-fri-verifier-tied-reduction-staged-bound)
(rule active-fri-reductions-forD(2)[OF active])

lemma active-fri-reductions-staged-empty-header-bound:

assumes active: active-fri-reductions-for A

shows

wp-event (checked-staged-security-experiment-with-data-state A)
(staged-security-with-data-state-verifier-event
trace-fri-bad-with-empty-composition-header-candidates)
adversary-initial-state $\leq$ trace-fri-error
by (rule trace-fri-empty-header-reduction-staged-bound)
(rule active-fri-reductions-forD(3)[OF active])

lemma active-trace-fri-residual-obligations-staged-trace-bound:

assumes

active-trace-fri-residual-obligations-for A Rt Zt At Pt Ht Gt Nt St Ft

shows

wp-event (checked-staged-security-experiment-with-data-state A)
(staged-security-with-data-state-verifier-event
trace-fri-bad-with-header-tied-partial-candidate)
adversary-initial-state $\leq$ trace-fri-error

proof (rule trace-fri-header-tied-reduction-staged-bound)

fix data attacker-state

assume support:

Some (data, attacker-state) $\in$
set-dist (execute (checked-staged-transcript-program A)
adversary-initial-state)

let ?s =

verifier-state-from-adversary attacker-state
(staged-proof-transcript data)

have residual:

trace-fri-header-tied-residual-obligations ?s
(Rt data attacker-state) (Zt data attacker-state)
(At data attacker-state) (Pt data attacker-state)
(Ht data attacker-state) (Gt data attacker-state)

```

    (Nt data attacker-state) (St data attacker-state)
    (Ft data attacker-state)
  using assms support
  unfolding active-trace-fri-residual-obligations-for-def by simp
show trace-fri-header-tied-reduction ?s
  by (rule trace-fri-header-tied-reduction-from-residual-obligations
      [OF residual])
qed

lemma active-composition-fri-residual-obligations-staged-composition-bound:
  assumes
    active-composition-fri-residual-obligations-for A Rc Ac Pc Nc Sc Gc
  shows
    wp-event (checked-staged-security-experiment-with-data-state A)
      (staged-security-with-data-state-verifier-event
        composition-fri-bad-with-verifier-tied-partial-candidate)
    adversary-initial-state ≤ composition-fri-error
proof (rule composition-fri-verifier-tied-reduction-staged-bound)
  fix data attacker-state
  assume support:
    Some (data, attacker-state) ∈
      set-dist (execute (checked-staged-transcript-program A)
        adversary-initial-state)
  let ?s =
    verifier-state-from-adversary attacker-state
    (staged-proof-transcript data)
  have residual:
    composition-fri-verifier-tied-residual-obligations ?s
      (Rc data attacker-state) (Ac data attacker-state)
      (Pc data attacker-state)
      (Nc data attacker-state) (Sc data attacker-state)
      (Gc data attacker-state)
  using assms support
  unfolding active-composition-fri-residual-obligations-for-def by simp
show composition-fri-verifier-tied-reduction ?s
  by (rule composition-fri-verifier-tied-reduction-from-residual-obligations
      [OF residual])
qed

lemma active-empty-trace-fri-reduction-staged-empty-header-bound:
  assumes active-empty-trace-fri-reduction-for A
  shows
    wp-event (checked-staged-security-experiment-with-data-state A)
      (staged-security-with-data-state-verifier-event
        trace-fri-bad-with-empty-composition-header-candidates)
    adversary-initial-state ≤ trace-fri-error
  by (rule trace-fri-empty-header-reduction-staged-bound)
    (use assms in
      ⟨unfold active-empty-trace-fri-reduction-for-def, blast⟩)

```

lemma *active-fri-residual-obligations-staged-trace-bound*:
assumes *active-fri-residual-obligations-for* A Rt Zt At Pt Ht Gt Nt St Ft
 Rc Ac Pc Nc Sc Gc
shows
 $wp\text{-}event$ (*checked-staged-security-experiment-with-data-state* A)
(*staged-security-with-data-state-verifier-event*
trace-fri-bad-with-header-tied-partial-candidate)
 $adversary\text{-}initial\text{-}state \leq trace\text{-}fri\text{-}error$
by (*rule active-fri-reductions-staged-trace-bound*)
(*rule active-fri-reductions-from-residual-obligations*[*OF* *assms*])

lemma *active-fri-residual-obligations-staged-composition-bound*:
assumes *active-fri-residual-obligations-for* A Rt Zt At Pt Ht Gt Nt St Ft
 Rc Ac Pc Nc Sc Gc
shows
 $wp\text{-}event$ (*checked-staged-security-experiment-with-data-state* A)
(*staged-security-with-data-state-verifier-event*
composition-fri-bad-with-verifier-tied-partial-candidate)
 $adversary\text{-}initial\text{-}state \leq composition\text{-}fri\text{-}error$
by (*rule active-fri-reductions-staged-composition-bound*)
(*rule active-fri-reductions-from-residual-obligations*[*OF* *assms*])

lemma *active-fri-residual-obligations-staged-empty-header-bound*:
assumes *active-fri-residual-obligations-for* A Rt Zt At Pt Ht Gt Nt St Ft
 Rc Ac Pc Nc Sc Gc
shows
 $wp\text{-}event$ (*checked-staged-security-experiment-with-data-state* A)
(*staged-security-with-data-state-verifier-event*
trace-fri-bad-with-empty-composition-header-candidates)
 $adversary\text{-}initial\text{-}state \leq trace\text{-}fri\text{-}error$
by (*rule active-fri-reductions-staged-empty-header-bound*)
(*rule active-fri-reductions-from-residual-obligations*[*OF* *assms*])

end

context *soundness-fri*
begin

lemma *active-fri-reductions-for-from-locale*:
active-fri-reductions-for A
unfolding *active-fri-reductions-for-def*
proof (*intro allI impI conjI*)
fix *data attacker-state*
assume *support*:
 $Some$ (*data*, *attacker-state*) $\in$
 $set\text{-}dist$ (*execute* (*checked-staged-transcript-program* A)
adversary-initial-state)
show *trace-fri-header-tied-reduction*

```

      (verifier-state-from-adversary attacker-state
       (staged-proof-transcript data))
    using trace-fri-active-reductions[OF support] by blast
next
fix data attacker-state
assume support:
  Some (data, attacker-state) ∈
    set-dist (execute (checked-staged-transcript-program A)
                 adversary-initial-state)
show composition-fri-verifier-tied-reduction
  (verifier-state-from-adversary attacker-state
   (staged-proof-transcript data))
  by (rule composition-fri-active-reduction[OF support])
next
fix data attacker-state
assume support:
  Some (data, attacker-state) ∈
    set-dist (execute (checked-staged-transcript-program A)
                 adversary-initial-state)
show trace-fri-empty-header-reduction
  (verifier-state-from-adversary attacker-state
   (staged-proof-transcript data))
  using trace-fri-active-reductions[OF support] by blast
qed

```

end

end

theory *Soundness-FRI-Obligation-Normalization*
imports *Soundness-FRI-Active-Reductions*
begin

Normal form for the three FRI obligations that remain internal to locale *soundness-fri*. This theory does not change the protocol or the public soundness route. It only records the exact verifier-local events behind the internal FRI assumptions and exposes the generic partial FRI evidence already available from accepted verifier transcripts.

context *soundness*
begin

lemma *trace-fri-header-tied-reduction-event*:
 $\text{trace-fri-header-tied-reduction } s \longleftrightarrow$
 $\text{wp-event verify-monad}$
 $(\lambda \text{out. } \exists \text{ trace-roots trace-bs trace-final dg composition-roots}$
 $\text{composition-bs composition-final fri-query-idxs trace-round-layers}$
 $\text{composition-round-layers fr as rest trace-openings trace-table.}$
 $\text{accepted-fri-opening-transcript } s \text{ out trace-roots trace-bs}$
 $\text{trace-final dg composition-roots composition-bs composition-final})$

fri-query-idxs trace-round-layers composition-round-layers $\wedge$
verifier-header-transcript s fr trace-roots trace-final as dg
composition-roots composition-final rest $\wedge$
accepted-with-partial-trace-openings s out fr fri-query-idxs
trace-openings $\wedge$
partial-trace-table-candidate trace-table trace-openings $\wedge$
 $\neg$ *trace-table-low-degree trace-table*) $s \leq$
trace-fri-error
unfolding *trace-fri-header-tied-reduction-def*
trace-fri-bad-with-header-tied-partial-candidate-def
by *simp*

lemma *trace-fri-empty-header-reduction-event:*
trace-fri-empty-header-reduction s $\longleftrightarrow$
wp-event verify-monad
 $(\lambda out. \exists fr f-fri-roots f-final as dg final trace-query-idxs$
trace-openings trace-table composition-table.
accepted-with-empty-composition-header-candidates s out fr
f-fri-roots f-final as dg final trace-query-idxs trace-openings
trace-table composition-table $\wedge$
 $\neg$ *trace-table-low-degree trace-table*) $s \leq$
trace-fri-error
unfolding *trace-fri-empty-header-reduction-def*
trace-fri-bad-with-empty-composition-header-candidates-def
by *simp*

lemma *composition-fri-verifier-tied-reduction-event:*
composition-fri-verifier-tied-reduction s $\longleftrightarrow$
wp-event verify-monad
 $(\lambda out. \exists trace-roots trace-bs trace-final fri-dg composition-roots$
composition-bs composition-final fri-query-idxs trace-round-layers
composition-round-layers fr as trace-openings composition-openings
trace-table composition-table.
accepted-fri-opening-transcript s out trace-roots trace-bs
trace-final fri-dg composition-roots composition-bs
composition-final fri-query-idxs trace-round-layers
composition-round-layers $\wedge$
accepted-with-partial-initial-openings-aligned s out fr trace-roots
trace-final as fri-dg composition-roots composition-final
fri-query-idxs trace-openings composition-openings $\wedge$
partial-trace-table-candidate trace-table trace-openings $\wedge$
partial-composition-table-candidate composition-table
composition-openings $\wedge$
trace-table-low-degree trace-table $\wedge$
 $\neg$ *composition-table-low-degree maxDegree composition-table*) $s \leq$
composition-fri-error
unfolding *composition-fri-verifier-tied-reduction-def*
composition-fri-bad-with-verifier-tied-partial-candidate-def
by *simp*

lemma *trace-fri-header-tied-badE*:
assumes *trace-fri-bad-with-header-tied-partial-candidate s out*
obtains *trace-roots trace-bs trace-final dg composition-roots*
composition-bs composition-final fri-query-idxs trace-round-layers
composition-round-layers fr as rest trace-openings trace-table **where**
accepted-fri-opening-transcript s out trace-roots trace-bs
trace-final dg composition-roots composition-bs composition-final
fri-query-idxs trace-round-layers composition-round-layers
verifier-header-transcript s fr trace-roots trace-final as dg
composition-roots composition-final rest
accepted-with-partial-trace-openings s out fr fri-query-idxs
trace-openings
partial-trace-table-candidate trace-table trace-openings
 $\neg$ *trace-table-low-degree trace-table*
using *assms*
unfolding *trace-fri-bad-with-header-tied-partial-candidate-def*
by *blast*

lemma *trace-fri-empty-header-badE*:
assumes *trace-fri-bad-with-empty-composition-header-candidates s out*
obtains *fr f-fri-roots f-final as dg final trace-query-idxs*
trace-openings trace-table composition-table **where**
accepted-with-empty-composition-header-candidates s out fr
f-fri-roots f-final as dg final trace-query-idxs trace-openings
trace-table composition-table
 $\neg$ *trace-table-low-degree trace-table*
using *assms*
unfolding *trace-fri-bad-with-empty-composition-header-candidates-def*
by *blast*

lemma *trace-fri-empty-header-bad-extracts-generic-partial-evidence*:
assumes *outcome*:
Some (result, final-state) $\in$ set-dist (execute verify-monad s)
and *bad*:
trace-fri-bad-with-empty-composition-header-candidates s
(Some (result, final-state))
obtains *trace-roots trace-bs trace-final dg composition-roots*
composition-bs composition-final fri-query-idxs trace-round-layers
composition-round-layers fr candidate-query-idxs trace-openings
trace-table **where**
trace-fri-partial-candidate-opening-evidence s
(Some (result, final-state)) trace-roots trace-bs trace-final dg
composition-roots composition-bs composition-final fri-query-idxs
trace-round-layers composition-round-layers fr candidate-query-idxs
trace-openings trace-table
generic-fri-partial-evidence trace-table-low-degree trace-table
(clength - 1) trace-roots trace-bs trace-final fri-query-idxs
trace-round-layers

proof –
have *reachable*:
trace-fri-bad-with-reachable-partial-candidate s
(Some (result, final-state))
by (rule
trace-fri-bad-with-empty-composition-header-candidates-imp-reachable-partial-candidate
[OF bad])
from *verify-monad-trace-fri-bad-extracts-opening-evidence*
[OF outcome reachable]
obtain *trace-roots trace-bs trace-final dg composition-roots*
composition-bs composition-final fri-query-idxs trace-round-layers
composition-round-layers fr candidate-query-idxs trace-openings
trace-table
where *evidence*:
trace-fri-partial-candidate-opening-evidence s
(Some (result, final-state)) trace-roots trace-bs trace-final dg
composition-roots composition-bs composition-final fri-query-idxs
trace-round-layers composition-round-layers fr candidate-query-idxs
trace-openings trace-table
by *blast*
have *generic*:
generic-fri-partial-evidence trace-table-low-degree trace-table
(clength – 1) trace-roots trace-bs trace-final fri-query-idxs
trace-round-layers
by (rule *trace-fri-partial-candidate-opening-evidence-generic-partial*
[OF evidence])
show *?thesis*
by (rule *that[OF evidence generic]*)
qed

lemma *trace-fri-header-tied-sampled-layer-chain-imp-sampled-layer-chain*:
assumes *trace-fri-bad-with-header-tied-sampled-layer-chain s out*
shows *trace-fri-bad-with-sampled-layer-chain s out*

proof –
from *assms* **obtain** *trace-roots trace-bs trace-final dg composition-roots*
composition-bs composition-final fri-query-idxs trace-round-layers
composition-round-layers fr as rest trace-openings trace-table doms
layers
where *fri*:
accepted-fri-opening-transcript s out trace-roots trace-bs
trace-final dg composition-roots composition-bs composition-final
fri-query-idxs trace-round-layers composition-round-layers
and *partial*:
accepted-with-partial-trace-openings s out fr fri-query-idxs
trace-openings
and *candidate*:
partial-trace-table-candidate trace-table trace-openings
and *not-low*: $\neg$ *trace-table-low-degree trace-table*
and *chain*:

generic-fri-sampled-layer-chain-evidence trace-table-low-degree
trace-table (clength - 1) trace-roots trace-bs trace-final
fri-query-idxs trace-round-layers doms layers
unfolding *trace-fri-bad-with-header-tied-sampled-layer-chain-def*
by *blast*
have *evidence:*
trace-fri-partial-candidate-opening-evidence s out trace-roots
trace-bs trace-final dg composition-roots composition-bs
composition-final fri-query-idxs trace-round-layers
composition-round-layers fr fri-query-idxs trace-openings
trace-table
unfolding *trace-fri-partial-candidate-opening-evidence-def*
trace-fri-partial-candidate-evidence-def
by *(intro conjI fri partial candidate not-low)*
show *?thesis*
unfolding *trace-fri-bad-with-sampled-layer-chain-def*
by *(intro exI conjI)*
(rule evidence, rule chain)
qed

lemma *wp-trace-fri-header-tied-sampled-layer-chain-bound-from-sampled:*
assumes *sampled-bound:*
wp-event verify-monad (trace-fri-bad-with-sampled-layer-chain s) s ≤ R
shows
wp-event verify-monad
(trace-fri-bad-with-header-tied-sampled-layer-chain s) s ≤ R
proof –
have *wp-event verify-monad*
(trace-fri-bad-with-header-tied-sampled-layer-chain s) s ≤
wp-event verify-monad (trace-fri-bad-with-sampled-layer-chain s) s
by *(rule wp-event-mono)*
(rule trace-fri-header-tied-sampled-layer-chain-imp-sampled-layer-chain)
then show *?thesis*
by *(rule order-trans[OF - sampled-bound])*
qed

lemma *wp-trace-fri-header-tied-sampled-layer-chain-bound-from-full-cover-and-missing:*
assumes *full-bound:*
wp-event verify-monad
(trace-fri-bad-with-full-cover-layer-chain s bad) s ≤ R
and *missing-bound:*
wp-event verify-monad
(trace-fri-sampled-missing-full-cover s bad) s ≤ M
shows
wp-event verify-monad
(trace-fri-bad-with-header-tied-sampled-layer-chain s) s ≤ R + M
by *(rule wp-trace-fri-header-tied-sampled-layer-chain-bound-from-sampled)*
(rule wp-trace-fri-sampled-bound-from-full-cover-and-missing
[OF full-bound missing-bound])

lemma *composition-fri-verifier-tied-sampled-layer-chain-imp-sampled-layer-chain:*

assumes *composition-fri-bad-with-verifier-tied-sampled-layer-chain s out*

shows *composition-fri-bad-with-sampled-layer-chain s out*

proof –

from *assms* **obtain** *trace-roots trace-bs trace-final fri-dg*

composition-roots composition-bs composition-final fri-query-idxs

trace-round-layers composition-round-layers fr as trace-openings

composition-openings trace-table composition-table doms layers

where *fri:*

accepted-fri-opening-transcript s out trace-roots trace-bs

trace-final fri-dg composition-roots composition-bs

composition-final fri-query-idxs trace-round-layers

composition-round-layers

and *aligned:*

accepted-with-partial-initial-openings-aligned s out fr trace-roots

trace-final as fri-dg composition-roots composition-final

fri-query-idxs trace-openings composition-openings

and *trace-candidate:*

partial-trace-table-candidate trace-table trace-openings

and *composition-candidate:*

partial-composition-table-candidate composition-table

composition-openings

and *trace-low: trace-table-low-degree trace-table*

and *composition-not-low:*

$\neg$ *composition-table-low-degree maxDegree composition-table*

and *chain:*

generic-fri-sampled-layer-chain-evidence

(composition-table-low-degree (to-nat fri-dg)) composition-table

(to-nat fri-dg) composition-roots composition-bs composition-final

fri-query-idxs composition-round-layers doms layers

unfolding *composition-fri-bad-with-verifier-tied-sampled-layer-chain-def*

by *blast*

have *unaligned:*

accepted-with-partial-initial-openings s out fr trace-roots

trace-final as fri-dg composition-roots composition-final

fri-query-idxs trace-openings fri-query-idxs composition-openings

by *(rule accepted-with-partial-initial-openings-aligned-imp-unaligned*

[OF aligned])

have *evidence:*

composition-fri-partial-candidate-opening-evidence s out trace-roots

trace-bs trace-final fri-dg composition-roots composition-bs

composition-final fri-query-idxs trace-round-layers

composition-round-layers fr trace-roots trace-final as fri-dg

composition-roots composition-final fri-query-idxs trace-openings

fri-query-idxs composition-openings trace-table composition-table

unfolding *composition-fri-partial-candidate-opening-evidence-def*

composition-fri-partial-candidate-evidence-def

by *(intro conjI fri unaligned trace-candidate composition-candidate*

$\text{trace-low composition-not-low})$
show $?thesis$
unfolding $\text{composition-fri-bad-with-sampled-layer-chain-def}$
by (intro exI conjI)
 $(\text{rule evidence, rule chain})$
qed

lemma $\text{wp-composition-fri-verifier-tied-sampled-layer-chain-bound-from-sampled:}$
assumes sampled-bound:
 $\text{wp-event verify-monad (composition-fri-bad-with-sampled-layer-chain s) s}$
 $\leq R$
shows
 $\text{wp-event verify-monad}$
 $(\text{composition-fri-bad-with-verifier-tied-sampled-layer-chain s) s}$
 $\leq R$
proof –
have $\text{wp-event verify-monad}$
 $(\text{composition-fri-bad-with-verifier-tied-sampled-layer-chain s) s} \leq$
 $\text{wp-event verify-monad (composition-fri-bad-with-sampled-layer-chain s) s}$
by $(\text{rule wp-event-mono})$
 $(\text{rule}$
 $\text{composition-fri-verifier-tied-sampled-layer-chain-imp-sampled-layer-chain})$
then show $?thesis$
by $(\text{rule order-trans[OF - sampled-bound]})$
qed

lemma $\text{wp-composition-fri-verifier-tied-sampled-layer-chain-bound-from-full-cover-and-missing:}$
assumes full-bound:
 $\text{wp-event verify-monad}$
 $(\text{composition-fri-bad-with-full-cover-layer-chain s bad) s} \leq R$
and missing-bound:
 $\text{wp-event verify-monad}$
 $(\text{composition-fri-sampled-missing-full-cover s bad) s} \leq M$
shows
 $\text{wp-event verify-monad}$
 $(\text{composition-fri-bad-with-verifier-tied-sampled-layer-chain s) s}$
 $\leq R + M$
by $(\text{rule}$
 $\text{wp-composition-fri-verifier-tied-sampled-layer-chain-bound-from-sampled})$
 $(\text{rule wp-composition-fri-sampled-bound-from-full-cover-and-missing}$
 $[\text{OF full-bound missing-bound}])$

lemma $\text{trace-fri-header-tied-residual-obligations-from-full-cover-missing-without-head-slot-and-selected-zero:}$
assumes full-bound:
 $\text{wp-event verify-monad}$
 $(\text{trace-fri-bad-with-full-cover-layer-chain s bad) s} \leq R$
and missing-bound:
 $\text{wp-event verify-monad}$
 $(\text{trace-fri-sampled-missing-full-cover s bad) s} \leq M$

and zero:
 $wp\text{-event verify-monad}$
 $(\text{trace-fri-header-tied-zero-round-final-obstruction } s) \ s \leq z\text{-b}$
and merkle:
 $wp\text{-event verify-monad } (\text{partial-merkle-inconsistency-bad } s) \ s \leq p\text{-b}$
and structural:
 $wp\text{-event verify-monad}$
 $(\text{trace-fri-header-tied-sibling-mismatch-structural-gap } s) \ s \leq h\text{-b}$
and next-bound:
 $wp\text{-event verify-monad}$
 $(\text{trace-fri-header-tied-next-value-replay-gap } s) \ s \leq n\text{-b}$
and successor:
 $wp\text{-event verify-monad}$
 $(\text{trace-fri-header-tied-successor-replay-gap } s) \ s \leq s\text{-b}$
and total:
 $(R + M) + (((((p\text{-b} + h\text{-b}) + p\text{-b}) + n\text{-b} + s\text{-b} +$
 $(p\text{-b} + 0 + 0)) + p\text{-b}) + 0 + z\text{-b})$
 $\leq \text{trace-fri-error}$
shows
 $\text{trace-fri-header-tied-residual-obligations } s$
 $(R + M) \ z\text{-b } 0 \ p\text{-b } h\text{-b } p\text{-b } n\text{-b } s\text{-b } 0$
by (rule
 $\text{trace-fri-header-tied-residual-obligations-from-bounds-without-head-slot-and-selected-zero})$
(rule
 $wp\text{-trace-fri-header-tied-sampled-layer-chain-bound-from-full-cover-and-missing}$
 $[\text{OF full-bound missing-bound}],$
rule zero, rule merkle, rule structural, rule next-bound,
rule successor, rule total)

lemma *composition-fri-verifier-tied-residual-obligations-from-full-cover-missing-slot-zero:*

assumes full-bound:
 $wp\text{-event verify-monad}$
 $(\text{composition-fri-bad-with-full-cover-layer-chain } s \text{ bad}) \ s \leq R$
and missing-bound:
 $wp\text{-event verify-monad}$
 $(\text{composition-fri-sampled-missing-full-cover } s \text{ bad}) \ s \leq M$
and merkle:
 $wp\text{-event verify-monad } (\text{partial-merkle-inconsistency-bad } s) \ s \leq p\text{-b}$
and next-bound:
 $wp\text{-event verify-monad}$
 $(\text{composition-fri-verifier-tied-next-value-replay-gap } s) \ s \leq n\text{-b}$
and successor:
 $wp\text{-event verify-monad}$
 $(\text{composition-fri-verifier-tied-successor-replay-gap } s) \ s \leq s\text{-b}$
and final-bound:
 $wp\text{-event verify-monad}$
 $(\text{composition-fri-verifier-tied-final-value-replay-gap } s) \ s \leq g\text{-b}$
and total:
 $(R + M) + (((p\text{-b} + 0) + n\text{-b} + s\text{-b} + g\text{-b} + p\text{-b}) + 0)$

$\leq \text{composition-fri-error}$
shows
composition-fri-verifier-tied-residual-obligations s
(R + M) 0 p-b n-b s-b g-b
by (*rule composition-fri-verifier-tied-residual-obligations-from-bounds-slot-zero*)
 (rule
 wp-composition-fri-verifier-tied-sampled-layer-chain-bound-from-full-cover-and-missing
 [*OF full-bound missing-bound*],
 rule merkle, rule next-bound, rule successor, rule final-bound,
 rule total)

lemma *composition-fri-verifier-tied-badE:*
assumes *composition-fri-bad-with-verifier-tied-partial-candidate s out*
obtains *trace-roots trace-bs trace-final fri-dg composition-roots*
 composition-bs composition-final fri-query-idxs trace-round-layers
 composition-round-layers fr as trace-openings composition-openings
 trace-table composition-table **where**
accepted-fri-opening-transcript s out trace-roots trace-bs
trace-final fri-dg composition-roots composition-bs
composition-final fri-query-idxs trace-round-layers
composition-round-layers
accepted-with-partial-initial-openings-aligned s out fr trace-roots
trace-final as fri-dg composition-roots composition-final
fri-query-idxs trace-openings composition-openings
partial-trace-table-candidate trace-table trace-openings
partial-composition-table-candidate composition-table
 composition-openings
trace-table-low-degree trace-table
 $\neg$ *composition-table-low-degree maxDegree composition-table*
using *assms*
unfolding *composition-fri-bad-with-verifier-tied-partial-candidate-def*
by *blast*

lemma *trace-fri-header-tied-bad-generic-partial-evidence:*
assumes *trace-fri-bad-with-header-tied-partial-candidate s out*
obtains *trace-roots trace-bs trace-final dg composition-roots*
 composition-bs composition-final fri-query-idxs trace-round-layers
 composition-round-layers fr as rest trace-openings trace-table **where**
accepted-fri-opening-transcript s out trace-roots trace-bs
trace-final dg composition-roots composition-bs composition-final
fri-query-idxs trace-round-layers composition-round-layers
verifier-header-transcript s fr trace-roots trace-final as dg
 composition-roots composition-final rest
accepted-with-partial-trace-openings s out fr fri-query-idxs
 trace-openings
partial-trace-table-candidate trace-table trace-openings
 $\neg$ *trace-table-low-degree trace-table*
generic-fri-partial-evidence trace-table-low-degree trace-table
 (*clength - 1*) *trace-roots trace-bs trace-final fri-query-idxs*

trace-round-layers
proof –
from *trace-fri-header-tied-badE[OF assms]* **obtain** *trace-roots trace-bs*
trace-final dg composition-roots composition-bs composition-final
fri-query-idxs trace-round-layers composition-round-layers fr as rest
trace-openings trace-table
where *fri*:
accepted-fri-opening-transcript s out trace-roots trace-bs
trace-final dg composition-roots composition-bs composition-final
fri-query-idxs trace-round-layers composition-round-layers
and *header*:
verifier-header-transcript s fr trace-roots trace-final as dg
composition-roots composition-final rest
and *partial*:
accepted-with-partial-trace-openings s out fr fri-query-idxs
trace-openings
and *candidate*:
partial-trace-table-candidate trace-table trace-openings
and *not-low*: $\neg$ *trace-table-low-degree trace-table*
by *blast*
have *generic*:
generic-fri-partial-evidence trace-table-low-degree trace-table
(clength – 1) trace-roots trace-bs trace-final fri-query-idxs
trace-round-layers
by (rule *accepted-fri-opening-transcript-trace-generic-partial-evidence*
 $[OF\ fri]$)
show *?thesis*
by (rule *that[OF fri header partial candidate not-low generic]*)
qed

lemma *composition-fri-verifier-tied-bad-generic-partial-evidence*:
assumes *composition-fri-bad-with-verifier-tied-partial-candidate s out*
obtains *trace-roots trace-bs trace-final fri-dg composition-roots*
composition-bs composition-final fri-query-idxs trace-round-layers
composition-round-layers fr as trace-openings composition-openings
trace-table composition-table **where**
accepted-fri-opening-transcript s out trace-roots trace-bs
trace-final fri-dg composition-roots composition-bs
composition-final fri-query-idxs trace-round-layers
composition-round-layers
accepted-with-partial-initial-openings-aligned s out fr trace-roots
trace-final as fri-dg composition-roots composition-final
fri-query-idxs trace-openings composition-openings
partial-trace-table-candidate trace-table trace-openings
partial-composition-table-candidate composition-table
composition-openings
trace-table-low-degree trace-table
 $\neg$ *composition-table-low-degree maxDegree composition-table*
generic-fri-partial-evidence

```

    (composition-table-low-degree (to-nat fri-dg)) composition-table
    (to-nat fri-dg) composition-roots composition-bs composition-final
    fri-query-idxs composition-round-layers
proof –
  from composition-fri-verifier-tied-badE[OF assms] obtain trace-roots
    trace-bs trace-final fri-dg composition-roots composition-bs
    composition-final fri-query-idxs trace-round-layers
    composition-round-layers fr as trace-openings composition-openings
    trace-table composition-table
  where fri:
    accepted-fri-opening-transcript s out trace-roots trace-bs
    trace-final fri-dg composition-roots composition-bs
    composition-final fri-query-idxs trace-round-layers
    composition-round-layers
  and aligned:
    accepted-with-partial-initial-openings-aligned s out fr trace-roots
    trace-final as fri-dg composition-roots composition-final
    fri-query-idxs trace-openings composition-openings
  and trace-candidate:
    partial-trace-table-candidate trace-table trace-openings
  and composition-candidate:
    partial-composition-table-candidate composition-table
    composition-openings
  and trace-low: trace-table-low-degree trace-table
  and composition-not-low:
    ¬ composition-table-low-degree maxDegree composition-table
  by blast
have generic:
  generic-fri-partial-evidence
    (composition-table-low-degree (to-nat fri-dg)) composition-table
    (to-nat fri-dg) composition-roots composition-bs composition-final
    fri-query-idxs composition-round-layers
  by (rule
    accepted-fri-opening-transcript-composition-generic-partial-evidence
    [OF fri])
show ?thesis
  by (rule that[OF fri aligned trace-candidate composition-candidate
    trace-low composition-not-low generic])
qed

end

end

```

```

theory Soundness-FRI-Prequery-Core
imports
  Soundness-FRI-Obligation-Normalization
  Staged-Security-Experiment-Composition-Branch

```

begin

Verifier-local FRI challenge prequery accounting.

The concrete FRI challenge-list bounds require freshness of future Trace/Composition FRI challenge keys. In the staged adversary experiment the verifier receives the adversary's final oracle map, so freshness is not automatic. This layer defines the exact verifier-header key paths and proves deterministic fresh/prequeried splits. It does not add assumptions or change the oracle model.

context *soundness*

begin

definition *trace-fri-challenge-key-at*

$:: ('f, 'a) \text{ protocol-channel-scheme} \Rightarrow 'f \Rightarrow 'f \text{ list} \Rightarrow$
 $\text{nat} \Rightarrow 'f \text{ protocol-hash-input}$

where

trace-fri-challenge-key-at s fr roots i =
TraceFriChallenge (*PTraceFriCounter s + i*)
(*foldl concat (concat (PState s) fr) (take (Suc i) roots)*)

definition *composition-fri-challenge-key-at*

$:: ('f, 'a) \text{ protocol-channel-scheme} \Rightarrow 'f \Rightarrow 'f \text{ list} \Rightarrow$
 $'f \Rightarrow 'f \text{ list} \Rightarrow 'f \Rightarrow 'f \text{ list} \Rightarrow \text{nat} \Rightarrow$
 $'f \text{ protocol-hash-input}$

where

composition-fri-challenge-key-at s fr trace-roots trace-final as dg
composition-roots i =
CompositionFriChallenge (*PCompositionFriCounter s + i*)
(*foldl concat*
(*concat*
(*foldl concat*
(*concat*
(*foldl concat (concat (PState s) fr) trace-roots*)
trace-final)
as)
dg)
(*take (Suc i) composition-roots*))

definition *trace-fri-challenge-path-fresh*

$:: ('f, 'a) \text{ protocol-channel-scheme} \Rightarrow 'f \Rightarrow 'f \text{ list} \Rightarrow$
 $\text{nat} \Rightarrow \text{bool}$

where

trace-fri-challenge-path-fresh s fr roots n $\longleftrightarrow$
($\forall i < n.$
 $i < \text{length roots} \longrightarrow$
fmlookup (*HashMap s*) (*trace-fri-challenge-key-at s fr roots i*) =
None)

definition *trace-fri-challenge-path-prequeried*

$:: ('f, 'a) \text{ protocol-channel-scheme} \Rightarrow 'f \Rightarrow 'f \text{ list} \Rightarrow$
 $\text{nat} \Rightarrow \text{bool}$

where

$\text{trace-fri-challenge-path-prequeried } s \text{ fr roots } n \longleftrightarrow$
 $(\exists i < n.$
 $i < \text{length roots} \wedge$
 $\text{fmlookup } (\text{HashMap } s) (\text{trace-fri-challenge-key-at } s \text{ fr roots } i)$
 $\neq \text{None})$

definition *composition-fri-challenge-path-fresh*

$:: ('f, 'a) \text{ protocol-channel-scheme} \Rightarrow 'f \Rightarrow 'f \text{ list} \Rightarrow$
 $'f \Rightarrow 'f \text{ list} \Rightarrow 'f \Rightarrow 'f \text{ list} \Rightarrow \text{nat} \Rightarrow \text{bool}$

where

$\text{composition-fri-challenge-path-fresh } s \text{ fr trace-roots trace-final as dg}$
 $\text{composition-roots } n \longleftrightarrow$
 $(\forall i < n.$
 $i < \text{length composition-roots} \longrightarrow$
 $\text{fmlookup } (\text{HashMap } s)$
 $(\text{composition-fri-challenge-key-at } s \text{ fr trace-roots trace-final}$
 $\text{as dg composition-roots } i) = \text{None})$

definition *composition-fri-challenge-path-prequeried*

$:: ('f, 'a) \text{ protocol-channel-scheme} \Rightarrow 'f \Rightarrow 'f \text{ list} \Rightarrow$
 $'f \Rightarrow 'f \text{ list} \Rightarrow 'f \Rightarrow 'f \text{ list} \Rightarrow \text{nat} \Rightarrow \text{bool}$

where

$\text{composition-fri-challenge-path-prequeried } s \text{ fr trace-roots trace-final as dg}$
 $\text{composition-roots } n \longleftrightarrow$
 $(\exists i < n.$
 $i < \text{length composition-roots} \wedge$
 $\text{fmlookup } (\text{HashMap } s)$
 $(\text{composition-fri-challenge-key-at } s \text{ fr trace-roots trace-final}$
 $\text{as dg composition-roots } i) \neq \text{None})$

definition *trace-fri-challenge-list-fresh-hit*

$:: ('f, 'a) \text{ protocol-channel-scheme} \Rightarrow 'f \text{ list set} \Rightarrow$
 $(\text{unit list} \times ('f, 'a) \text{ protocol-channel-scheme}) \text{ option} \Rightarrow \text{bool}$

where

$\text{trace-fri-challenge-list-fresh-hit } s B \text{ out} \longleftrightarrow$
 $(\exists \text{ trace-bs dg comp-bs fr trace-roots trace-final as}$
 $\text{composition-roots final rest.}$
 $\text{accepted-fri-challenges } s \text{ out trace-bs dg comp-bs} \wedge$
 $\text{verifier-header-transcript } s \text{ fr trace-roots trace-final as dg}$
 $\text{composition-roots final rest} \wedge$
 $\text{trace-bs} \in B \wedge$
 $\text{trace-fri-challenge-path-fresh } s \text{ fr trace-roots}$
 $(\text{length trace-roots}))$

definition *trace-fri-challenge-list-prequery-hit*

$:: ('f, 'a) \text{ protocol-channel-scheme} \Rightarrow 'f \text{ list set} \Rightarrow$

$(\text{unit list} \times ('f, 'a) \text{ protocol-channel-scheme}) \text{ option} \Rightarrow \text{bool}$
where
 $\text{trace-fri-challenge-list-prequery-hit } s \ B \ \text{out} \longleftrightarrow$
 $(\exists \text{ trace-bs } dg \ \text{comp-bs } fr \ \text{trace-roots } \text{trace-final} \ \text{as}$
 $\quad \text{composition-roots } \text{final } \text{rest}.$
 $\text{accepted-fri-challenges } s \ \text{out } \text{trace-bs } dg \ \text{comp-bs} \wedge$
 $\text{verifier-header-transcript } s \ \text{fr } \text{trace-roots } \text{trace-final} \ \text{as } dg$
 $\quad \text{composition-roots } \text{final } \text{rest} \wedge$
 $\text{trace-bs} \in B \wedge$
 $\text{trace-fri-challenge-path-prequeried } s \ \text{fr } \text{trace-roots}$
 $\quad (\text{length } \text{trace-roots}))$

definition *composition-fri-challenge-list-fresh-hit*
 $:: ('f, 'a) \text{ protocol-channel-scheme} \Rightarrow ('f \Rightarrow 'f \text{ list set}) \Rightarrow$
 $(\text{unit list} \times ('f, 'a) \text{ protocol-channel-scheme}) \text{ option} \Rightarrow \text{bool}$
where
 $\text{composition-fri-challenge-list-fresh-hit } s \ B \ \text{out} \longleftrightarrow$
 $(\exists \text{ trace-bs } dg \ \text{comp-bs } fr \ \text{trace-roots } \text{trace-final} \ \text{as}$
 $\quad \text{composition-roots } \text{final } \text{rest}.$
 $\text{accepted-fri-challenges } s \ \text{out } \text{trace-bs } dg \ \text{comp-bs} \wedge$
 $\text{verifier-header-transcript } s \ \text{fr } \text{trace-roots } \text{trace-final} \ \text{as } dg$
 $\quad \text{composition-roots } \text{final } \text{rest} \wedge$
 $\text{comp-bs} \in B \ dg \wedge$
 $\text{composition-fri-challenge-path-fresh } s \ \text{fr } \text{trace-roots } \text{trace-final}$
 $\quad \text{as } dg \ \text{composition-roots } (\text{length } \text{composition-roots}))$

definition *composition-fri-challenge-list-prequery-hit*
 $:: ('f, 'a) \text{ protocol-channel-scheme} \Rightarrow ('f \Rightarrow 'f \text{ list set}) \Rightarrow$
 $(\text{unit list} \times ('f, 'a) \text{ protocol-channel-scheme}) \text{ option} \Rightarrow \text{bool}$
where
 $\text{composition-fri-challenge-list-prequery-hit } s \ B \ \text{out} \longleftrightarrow$
 $(\exists \text{ trace-bs } dg \ \text{comp-bs } fr \ \text{trace-roots } \text{trace-final} \ \text{as}$
 $\quad \text{composition-roots } \text{final } \text{rest}.$
 $\text{accepted-fri-challenges } s \ \text{out } \text{trace-bs } dg \ \text{comp-bs} \wedge$
 $\text{verifier-header-transcript } s \ \text{fr } \text{trace-roots } \text{trace-final} \ \text{as } dg$
 $\quad \text{composition-roots } \text{final } \text{rest} \wedge$
 $\text{comp-bs} \in B \ dg \wedge$
 $\text{composition-fri-challenge-path-prequeried } s \ \text{fr } \text{trace-roots}$
 $\quad \text{trace-final } \text{as } dg \ \text{composition-roots } (\text{length } \text{composition-roots}))$

definition *trace-fri-challenge-key-relation*
 $:: ('f, 'a) \text{ protocol-channel-scheme} \Rightarrow 'f \text{ list set} \Rightarrow \text{nat} \Rightarrow$
 $'f \text{ protocol-hash-input} \Rightarrow 'f \Rightarrow \text{bool}$
where
 $\text{trace-fri-challenge-key-relation } s \ B \ n \ x \ y \longleftrightarrow$
 $(\exists bs \in B. \exists \text{fr roots } i.$
 $\quad i < n \wedge$
 $\quad i < \text{length } bs \wedge$
 $\quad i < \text{length } \text{roots} \wedge$

$x = \text{trace-fri-challenge-key-at } s \text{ fr roots } i \wedge$
 $y = bs ! i)$

definition *composition-fri-challenge-key-relation*

$:: ('f, 'a) \text{ protocol-channel-scheme} \Rightarrow ('f \Rightarrow 'f \text{ list set}) \Rightarrow$
 $\text{nat} \Rightarrow 'f \text{ protocol-hash-input} \Rightarrow 'f \Rightarrow \text{bool}$

where

$\text{composition-fri-challenge-key-relation } s \ B \ n \ x \ y \longleftrightarrow$
 $(\exists dg. \exists bs \in B \ dg. \exists fr \ \text{trace-roots} \ \text{trace-final} \ \text{as}$
 $\quad \text{composition-roots } i.$
 $i < n \wedge$
 $i < \text{length } bs \wedge$
 $i < \text{length } \text{composition-roots} \wedge$
 $x = \text{composition-fri-challenge-key-at } s \ \text{fr } \text{trace-roots} \ \text{trace-final}$
 $\quad \text{as } dg \ \text{composition-roots } i \wedge$
 $y = bs ! i)$

definition *checked-staged-trace-fri-prequery-hit*

$:: 'f \text{ list set} \Rightarrow$
 $('f \text{ staged-proof-data} \times 'f \text{ protocol-channel}) \text{ option} \Rightarrow \text{bool}$

where

$\text{checked-staged-trace-fri-prequery-hit } B \ \text{out} \longleftrightarrow$
 $(\text{case out of}$
 $\quad \text{None} \Rightarrow \text{False}$
 $\mid \text{Some } (data, \text{attacker-state}) \Rightarrow$
 $\quad \text{let } s = \text{verifier-state-from-adversary } \text{attacker-state}$
 $\quad (\text{staged-proof-transcript } data)$
 $\quad \text{in } \text{staged-trace-fri-challenges } data \in B \wedge$
 $\quad \text{trace-fri-challenge-path-prequeried } s$
 $\quad (\text{staged-trace-root } data)$
 $\quad (\text{staged-trace-fri-roots } data)$
 $\quad (\text{length } (\text{staged-trace-fri-roots } data)))$

definition *checked-staged-composition-fri-prequery-hit*

$:: ('f \Rightarrow 'f \text{ list set}) \Rightarrow$
 $('f \text{ staged-proof-data} \times 'f \text{ protocol-channel}) \text{ option} \Rightarrow \text{bool}$

where

$\text{checked-staged-composition-fri-prequery-hit } B \ \text{out} \longleftrightarrow$
 $(\text{case out of}$
 $\quad \text{None} \Rightarrow \text{False}$
 $\mid \text{Some } (data, \text{attacker-state}) \Rightarrow$
 $\quad \text{let } s = \text{verifier-state-from-adversary } \text{attacker-state}$
 $\quad (\text{staged-proof-transcript } data)$
 $\quad \text{in } \text{staged-composition-fri-challenges } data \in$
 $\quad B \ (\text{staged-degree } data) \wedge$
 $\quad \text{composition-fri-challenge-path-prequeried } s$
 $\quad (\text{staged-trace-root } data)$
 $\quad (\text{staged-trace-fri-roots } data)$
 $\quad (\text{staged-trace-final } data)$

(staged-alphas data)
 (staged-degree data)
 (staged-composition-fri-roots data)
 (length (staged-composition-fri-roots data)))

definition *checked-staged-trace-fri-prequery-relation*

:: 'f staged-adversary $\Rightarrow$ 'f list set $\Rightarrow$
 'f protocol-hash-input $\Rightarrow$ 'f $\Rightarrow$ bool

where

checked-staged-trace-fri-prequery-relation A B x y $\longleftrightarrow$
 ($\exists$ data attacker-state i.
 Some (data, attacker-state) $\in$
 set-dist
 (execute (checked-staged-transcript-program A)
 adversary-initial-state) $\wedge$
 staged-trace-fri-challenges data $\in$ B $\wedge$
 i < ceil-log clength $\wedge$
 i < length (staged-trace-fri-challenges data) $\wedge$
 i < length (staged-trace-fri-roots data) $\wedge$
 x = trace-fri-challenge-key-at
 (verifier-state-from-adversary attacker-state
 (staged-proof-transcript data))
 (staged-trace-root data) (staged-trace-fri-roots data) i $\wedge$
 y = staged-trace-fri-challenges data ! i)

definition *checked-staged-composition-fri-prequery-relation*

:: 'f staged-adversary $\Rightarrow$ ('f $\Rightarrow$ 'f list set) $\Rightarrow$
 'f protocol-hash-input $\Rightarrow$ 'f $\Rightarrow$ bool

where

checked-staged-composition-fri-prequery-relation A B x y $\longleftrightarrow$
 ($\exists$ data attacker-state i.
 Some (data, attacker-state) $\in$
 set-dist
 (execute (checked-staged-transcript-program A)
 adversary-initial-state) $\wedge$
 staged-composition-fri-challenges data $\in$ B (staged-degree data) $\wedge$
 i < ceil-log (maxDegree + 1) $\wedge$
 i < length (staged-composition-fri-challenges data) $\wedge$
 i < length (staged-composition-fri-roots data) $\wedge$
 x = composition-fri-challenge-key-at
 (verifier-state-from-adversary attacker-state
 (staged-proof-transcript data))
 (staged-trace-root data) (staged-trace-fri-roots data)
 (staged-trace-final data) (staged-alphas data)
 (staged-degree data) (staged-composition-fri-roots data) i $\wedge$
 y = staged-composition-fri-challenges data ! i)

definition *checked-staged-trace-fri-verifier-prequery-relation*

:: 'f staged-adversary $\Rightarrow$ 'f list set $\Rightarrow$

$'f \text{ protocol-hash-input} \Rightarrow 'f \Rightarrow \text{bool}$
where
checked-staged-trace-fri-verifier-prequery-relation $A \ B \ x \ y \longleftrightarrow$
 $(\exists \text{ data attacker-state } bs \text{ fr roots } i.$
 $\text{Some } (data, \text{attacker-state}) \in$
 set-dist
 $(\text{execute } (\text{checked-staged-transcript-program } A)$
 $\text{adversary-initial-state}) \wedge$
 $bs \in B \wedge$
 $i < \text{ceil-log } \text{clength} \wedge$
 $i < \text{length } bs \wedge$
 $i < \text{length } roots \wedge$
 $x = \text{trace-fri-challenge-key-at}$
 $(\text{verifier-state-from-adversary } \text{attacker-state}$
 $(\text{staged-proof-transcript } data))$
 $\text{fr roots } i \wedge$
 $y = bs ! i)$

definition *checked-staged-composition-fri-verifier-prequery-relation*
 $:: 'f \text{ staged-adversary} \Rightarrow ('f \Rightarrow 'f \text{ list set}) \Rightarrow$
 $'f \text{ protocol-hash-input} \Rightarrow 'f \Rightarrow \text{bool}$
where
checked-staged-composition-fri-verifier-prequery-relation $A \ B \ x \ y$
 $\longleftrightarrow$
 $(\exists \text{ data attacker-state } dg \ bs \text{ fr trace-roots trace-final as}$
 $\text{composition-roots } i.$
 $\text{Some } (data, \text{attacker-state}) \in$
 set-dist
 $(\text{execute } (\text{checked-staged-transcript-program } A)$
 $\text{adversary-initial-state}) \wedge$
 $bs \in B \ dg \wedge$
 $i < \text{ceil-log } (\text{maxDegree} + 1) \wedge$
 $i < \text{length } bs \wedge$
 $i < \text{length } \text{composition-roots} \wedge$
 $x = \text{composition-fri-challenge-key-at}$
 $(\text{verifier-state-from-adversary } \text{attacker-state}$
 $(\text{staged-proof-transcript } data))$
 $\text{fr trace-roots trace-final as } dg \text{ composition-roots } i \wedge$
 $y = bs ! i)$

lemma *trace-fri-challenge-key-relation-fiber-card-bound:*
assumes *finite-B:* *finite B*
shows
 $\text{card } \{y. \text{trace-fri-challenge-key-relation } s \ B \ n \ x \ y\} \leq \text{card } B * n$
proof –
let $?pairs = B \times \{..<n\}$
let $?values = (\lambda(bs, i). bs ! i) \text{ ` } ?pairs$
have *subset:*
 $\{y. \text{trace-fri-challenge-key-relation } s \ B \ n \ x \ y\} \subseteq ?values$


```

    unfolding trace-fri-challenge-key-relation-def
    by force
  have finite-values: finite ?values
    using finite-B by simp
  have card {y. trace-fri-challenge-key-relation s B n x y} ≤
    card ?values
    by (rule card-mono[OF finite-values subset])
  also have ... ≤ card ?pairs
    by (rule card-image-le) (use finite-B in simp)
  also have ... = card B * n
    using finite-B by simp
  finally show ?thesis .
qed

lemma composition-fri-challenge-key-relation-fiber-card-bound:
  assumes finite-B:  $\bigwedge dg. \text{finite } (B \ dg)$ 
  shows
    card {y. composition-fri-challenge-key-relation s B n x y} ≤
      ( $\sum dg \in (UNIV :: 'f \text{ set}). \text{card } (B \ dg) * n$ )
proof -
  let ?pairs = ( $\text{SIGMA } dg:(UNIV :: 'f \text{ set}). B \ dg \times \{..<n\}$ )
  let ?values = ( $\lambda(dg, bs, i). bs \ ! \ i$ ) ' ?pairs
  have subset:
    {y. composition-fri-challenge-key-relation s B n x y}  $\subseteq$  ?values
    unfolding composition-fri-challenge-key-relation-def
    by force
  have finite-values: finite ?values
    using finite-B by simp
  have card {y. composition-fri-challenge-key-relation s B n x y} ≤
    card ?values
    by (rule card-mono[OF finite-values subset])
  also have ... ≤ card ?pairs
    by (rule card-image-le) (use finite-B in simp)
  also have ... =
    ( $\sum dg \in (UNIV :: 'f \text{ set}). \text{card } (B \ dg \times \{..<n\})$ )
    using finite-B by (simp add: card-SigmaI)
  also have ... = ( $\sum dg \in (UNIV :: 'f \text{ set}). \text{card } (B \ dg) * n$ )
    using finite-B by simp
  finally show ?thesis .
qed

lemma hash-relation-program-checked-staged-transcript-trace-fri-prequery:
  assumes wf: staged-budget-wellformed budgets
    and controlled: staged-adversary-controlled budgets A
    and finite-B: finite B
  shows
    hash-relation-program (trace-fri-challenge-key-relation s B n)
      (card B * n)
      (staged-attacker-query-budget budgets + staged-challenge-query-budget)

```

(checked-staged-transcript-program A)
by (rule hash-relation-program-checked-staged-transcript-program
 [OF wf controlled])
 (rule trace-fri-challenge-key-relation-fiber-card-bound[OF finite-B])

lemma checked-staged-transcript-trace-fri-prequery-relation-bound:
assumes wf: staged-budget-wellformed budgets
and controlled: staged-adversary-controlled budgets A
and finite-B: finite B
shows
 wp-event (checked-staged-transcript-program A)
 (hash-relation-hit-event (trace-fri-challenge-key-relation s B n)
 adversary-initial-state)
 adversary-initial-state $\leq$
 hash-relation-budget-value (card B * n)
 (staged-attacker-query-budget budgets + staged-challenge-query-budget)

proof –
have program:
 hash-relation-program (trace-fri-challenge-key-relation s B n)
 (card B * n)
 (staged-attacker-query-budget budgets + staged-challenge-query-budget)
 (checked-staged-transcript-program A)
by (rule hash-relation-program-checked-staged-transcript-trace-fri-prequery
 [OF wf controlled finite-B])
show ?thesis
using program
unfolding hash-relation-program-def hash-relation-budget-def **by** blast
qed

lemma hash-relation-program-checked-staged-transcript-composition-fri-prequery:
assumes wf: staged-budget-wellformed budgets
and controlled: staged-adversary-controlled budgets A
and finite-B: $\bigwedge dg. \text{finite } (B \ dg)$
shows
 hash-relation-program (composition-fri-challenge-key-relation s B n)
 ($\sum dg \in (UNIV :: 'f \text{ set}). \text{card } (B \ dg) * n$)
 (staged-attacker-query-budget budgets + staged-challenge-query-budget)
 (checked-staged-transcript-program A)
by (rule hash-relation-program-checked-staged-transcript-program
 [OF wf controlled])
 (rule composition-fri-challenge-key-relation-fiber-card-bound
 [OF finite-B])

lemma checked-staged-transcript-composition-fri-prequery-relation-bound:
assumes wf: staged-budget-wellformed budgets
and controlled: staged-adversary-controlled budgets A
and finite-B: $\bigwedge dg. \text{finite } (B \ dg)$
shows
 wp-event (checked-staged-transcript-program A)

```

    (hash-relation-hit-event
      (composition-fri-challenge-key-relation s B n)
      adversary-initial-state)
    adversary-initial-state ≤
    hash-relation-budget-value
    (∑ dg ∈ (UNIV :: 'f set). card (B dg) * n)
    (staged-attacker-query-budget budgets + staged-challenge-query-budget)
  proof -
    have program:
      hash-relation-program (composition-fri-challenge-key-relation s B n)
      (∑ dg ∈ (UNIV :: 'f set). card (B dg) * n)
      (staged-attacker-query-budget budgets + staged-challenge-query-budget)
      (checked-staged-transcript-program A)
    by (rule
      hash-relation-program-checked-staged-transcript-composition-fri-prequery
      [OF wf controlled finite-B])
    show ?thesis
      using program
      unfolding hash-relation-program-def hash-relation-budget-def by blast
  qed

end

context soundness
begin

lemma checked-staged-trace-fri-prequery-relation-fiber-card-bound:
  assumes finite-B: finite B
  shows
    card {y. checked-staged-trace-fri-prequery-relation A B x y} ≤
    card B * ceil-log clength
  proof -
    let ?pairs = B × {..

```

qed

lemma *checked-staged-composition-fri-prequery-relation-fiber-card-bound:*

assumes *finite-B*: $\bigwedge dg. \text{finite } (B \ dg)$

shows

$$\begin{aligned} & \text{card } \{y. \text{checked-staged-composition-fri-prequery-relation } A \ B \ x \ y\} \\ & \leq (\sum dg \in (UNIV :: 'f \text{ set}). \\ & \quad \text{card } (B \ dg) * \text{ceil-log } (\text{maxDegree} + 1)) \end{aligned}$$

proof –

let *?pairs* =

$$(\text{SIGMA } dg:(UNIV :: 'f \text{ set}). B \ dg \times \{..<\text{ceil-log } (\text{maxDegree} + 1)\})$$

let *?values* = $(\lambda(dg, bs, i). bs \ ! \ i) \text{ ' } ?pairs$

have *subset*:

$$\begin{aligned} & \{y. \text{checked-staged-composition-fri-prequery-relation } A \ B \ x \ y\} \\ & \subseteq ?values \end{aligned}$$

unfolding *checked-staged-composition-fri-prequery-relation-def*

by *force*

have *finite-values*: *finite* *?values*

using *finite-B* **by** *simp*

have $\text{card } \{y. \text{checked-staged-composition-fri-prequery-relation } A \ B \ x \ y\}$

$$\leq \text{card } ?values$$

by (rule *card-mono[OF finite-values subset]*)

also have $\dots \leq \text{card } ?pairs$

by (rule *card-image-le*) (use *finite-B* **in** *simp*)

also have $\dots =$

$$\begin{aligned} & (\sum dg \in (UNIV :: 'f \text{ set}). \\ & \quad \text{card } (B \ dg \times \{..<\text{ceil-log } (\text{maxDegree} + 1)\})) \end{aligned}$$

using *finite-B* **by** (*simp add: card-SigmaI*)

also have $\dots =$

$$\begin{aligned} & (\sum dg \in (UNIV :: 'f \text{ set}). \\ & \quad \text{card } (B \ dg) * \text{ceil-log } (\text{maxDegree} + 1)) \end{aligned}$$

using *finite-B* **by** *simp*

finally show *?thesis* .

qed

lemma *checked-staged-trace-fri-verifier-prequery-relation-fiber-card-bound:*

assumes *finite-B*: *finite* *B*

shows

$$\begin{aligned} & \text{card } \{y. \text{checked-staged-trace-fri-verifier-prequery-relation } A \ B \ x \ y\} \\ & \leq \text{card } B * \text{ceil-log } \text{clength} \end{aligned}$$

proof –

let *?pairs* = $B \times \{..<\text{ceil-log } \text{clength}\}$

let *?values* = $(\lambda(bs, i). bs \ ! \ i) \text{ ' } ?pairs$

have *subset*:

$$\begin{aligned} & \{y. \text{checked-staged-trace-fri-verifier-prequery-relation } A \ B \ x \ y\} \\ & \subseteq ?values \end{aligned}$$

unfolding *checked-staged-trace-fri-verifier-prequery-relation-def*

by *auto*

have $\text{card } \{y. \text{checked-staged-trace-fri-verifier-prequery-relation } A \ B \ x \ y\}$

$\leq \text{card } ?\text{values}$
by (rule card-mono[*OF finite-imageI*[*OF finite-cartesian-product*
 $[\text{OF finite-}B \text{ finite-lessThan}] \text{ subset}$]])
also have $\dots \leq \text{card } ?\text{pairs}$
by (rule card-image-le) (use finite-*B* in simp)
also have $\dots = \text{card } B * \text{ceil-log } \text{clength}$
using finite-*B* **by** simp
finally show ?thesis .
qed

lemma

checked-staged-composition-fri-verifier-prequery-relation-fiber-card-bound:

assumes finite-*B*: $\bigwedge dg. \text{finite } (B \ dg)$

shows

card

$\{y. \text{checked-staged-composition-fri-verifier-prequery-relation } A \ B \ x \ y\}$
 $\leq (\sum dg \in (UNIV :: 'f \text{ set}).$
 $\text{card } (B \ dg) * \text{ceil-log } (\text{maxDegree} + 1))$

proof –

let ?pairs =

$SIGMA \ dg:(UNIV :: 'f \text{ set}). B \ dg \times \{..<\text{ceil-log } (\text{maxDegree} + 1)\}$

let ?values = $(\lambda(dg, bs, i). bs \ ! \ i) \text{ ' } ?\text{pairs}$

have subset:

$\{y. \text{checked-staged-composition-fri-verifier-prequery-relation } A \ B \ x \ y\}$
 $\subseteq ?\text{values}$

unfolding checked-staged-composition-fri-verifier-prequery-relation-def

by force

have finite-pairs: finite ?pairs

using finite-*B* **by** simp

have *card*

$\{y. \text{checked-staged-composition-fri-verifier-prequery-relation } A \ B \ x \ y\}$
 $\leq \text{card } ?\text{values}$

by (rule card-mono[*OF finite-imageI*[*OF finite-pairs*] subset])

also have $\dots \leq \text{card } ?\text{pairs}$

by (rule card-image-le) (rule finite-pairs)

also have $\dots =$

$(\sum dg \in (UNIV :: 'f \text{ set}).$
 $\text{card } (B \ dg) * \text{ceil-log } (\text{maxDegree} + 1))$

using finite-*B* **by** (simp add: card-SigmaI)

finally show ?thesis .

qed

lemma *checked-staged-transcript-trace-fri-verifier-prequery-relation-bound:*

assumes wf: staged-budget-wellformed budgets

and controlled: staged-adversary-controlled budgets *A*

and finite-*B*: finite *B*

shows

wp-event (checked-staged-transcript-program *A*)

(hash-relation-hit-event

(*checked-staged-trace-fri-verifier-prequery-relation* A B)
adversary-initial-state
adversary-initial-state $\leq$
hash-relation-budget-value (*card* $B * \text{ceil-log } \text{clength}$)
 (*staged-attacker-query-budget* *budgets* + *staged-challenge-query-budget*)
proof –
have *program*:
hash-relation-program
 (*checked-staged-trace-fri-verifier-prequery-relation* A B)
 (*card* $B * \text{ceil-log } \text{clength}$)
 (*staged-attacker-query-budget* *budgets* + *staged-challenge-query-budget*)
 (*checked-staged-transcript-program* A)
by (*rule hash-relation-program-checked-staged-transcript-program*
 [*OF wf controlled*])
 (*rule checked-staged-trace-fri-verifier-prequery-relation-fiber-card-bound*
 [*OF finite-B*])
then show *?thesis*
unfolding *hash-relation-program-def hash-relation-budget-def* **by** *blast*
qed

lemma

checked-staged-transcript-composition-fri-verifier-prequery-relation-bound:

assumes *wf*: *staged-budget-wellformed* *budgets*

and *controlled*: *staged-adversary-controlled* *budgets* A

and *finite-B*: $\bigwedge dg. \text{finite } (B \ dg)$

shows

wp-event (*checked-staged-transcript-program* A)
 (*hash-relation-hit-event*
 (*checked-staged-composition-fri-verifier-prequery-relation* A B)
adversary-initial-state)
adversary-initial-state $\leq$
hash-relation-budget-value
 ($\sum dg \in (UNIV :: 'f \text{ set}).$
card ($B \ dg$) * *ceil-log* (*maxDegree* + 1))
 (*staged-attacker-query-budget* *budgets* + *staged-challenge-query-budget*)

proof –

have *program*:

hash-relation-program
 (*checked-staged-composition-fri-verifier-prequery-relation* A B)
 ($\sum dg \in (UNIV :: 'f \text{ set}).$
card ($B \ dg$) * *ceil-log* (*maxDegree* + 1))
 (*staged-attacker-query-budget* *budgets* + *staged-challenge-query-budget*)
 (*checked-staged-transcript-program* A)
by (*rule hash-relation-program-checked-staged-transcript-program*
 [*OF wf controlled*])
 (*rule*
checked-staged-composition-fri-verifier-prequery-relation-fiber-card-bound
 [*OF finite-B*])

then show *?thesis*

```

    unfolding hash-relation-program-def hash-relation-budget-def by blast
qed

end

context soundness
begin

lemma checked-staged-trace-fri-verifier-prequery-hit-imp-relation-hit:
  assumes builder:
    Some (data, attacker-state) ∈
      set-dist
        (execute (checked-staged-transcript-program A)
          adversary-initial-state)
  and verifier:
    Some (result, final-state) ∈
      set-dist
        (execute verify-monad
          (verifier-state-from-adversary attacker-state
            (staged-proof-transcript data)))
  and hit:
    trace-fri-challenge-list-prequery-hit
      (verifier-state-from-adversary attacker-state
        (staged-proof-transcript data))
    B (Some (result, final-state))
  shows
    hash-relation-hit
      (checked-staged-trace-fri-verifier-prequery-relation A B)
      adversary-initial-state attacker-state
proof -
  let ?s =
    verifier-state-from-adversary attacker-state
      (staged-proof-transcript data)
  from hit obtain trace-bs dg comp-bs fr roots trace-final as
    composition-roots final rest where
    challenges:
      accepted-fri-challenges ?s (Some (result, final-state))
      trace-bs dg comp-bs
  and header:
    verifier-header-transcript ?s fr roots trace-final as dg
    composition-roots final rest
  and trace-bs-B: trace-bs ∈ B
  and prequeried:
    trace-fri-challenge-path-prequeried ?s fr roots (length roots)
  unfolding trace-fri-challenge-list-prequery-hit-def by blast
  from prequeried obtain i y where
    i-len: i < length roots
  and lookup-s:
    fmllookup (HashMap ?s) (trace-fri-challenge-key-at ?s fr roots i) =

```

```

    Some y
  unfolding trace-fri-challenge-path-prequeried-def
  by (auto split: option.splits)
have lookup-info:
  i < length trace-bs  $\wedge$  y = trace-bs ! i
  by (rule verify-monad-trace-fri-prequery-lookup
      [OF verifier challenges header i-len])
      (use lookup-s in ⟨simp add: trace-fri-challenge-key-at-def⟩)
have i-round: i < ceil-log clength
proof -
  have trace-bs ∈ fri-challenge-space (ceil-log clength)
    by (rule accepted-fri-challenges-trace-space[OF challenges])
  then have length trace-bs = ceil-log clength
    unfolding fri-challenge-space-def by simp
  then show ?thesis
    using lookup-info by simp
qed
have relation:
  checked-staged-trace-fri-verifier-prequery-relation A B
    (trace-fri-challenge-key-at ?s fr roots i) y
  unfolding checked-staged-trace-fri-verifier-prequery-relation-def
  apply (rule exI[where x = data])
  apply (rule exI[where x = attacker-state])
  apply (rule exI[where x = trace-bs])
  apply (rule exI[where x = fr])
  apply (rule exI[where x = roots])
  apply (rule exI[where x = i])
  using builder trace-bs-B i-round i-len lookup-info by simp
have initial-absent:
  fmlookup (HashMap adversary-initial-state)
    (trace-fri-challenge-key-at ?s fr roots i) = None
  unfolding adversary-initial-state-def by simp
have attacker-lookup:
  fmlookup (HashMap attacker-state)
    (trace-fri-challenge-key-at ?s fr roots i) = Some y
  using lookup-s unfolding verifier-state-from-adversary-def by simp
show ?thesis
  unfolding hash-relation-hit-def
  using initial-absent attacker-lookup relation by blast
qed

lemma checked-staged-composition-fri-verifier-prequery-hit-imp-relation-hit:
  assumes builder:
    Some (data, attacker-state) ∈
      set-dist
        (execute (checked-staged-transcript-program A)
          adversary-initial-state)
  and verifier:
    Some (result, final-state) ∈

```



```

    set-dist
    (execute verify-monad
      (verifier-state-from-adversary attacker-state
        (staged-proof-transcript data)))
  and hit:
    composition-fri-challenge-list-prequery-hit
    (verifier-state-from-adversary attacker-state
      (staged-proof-transcript data))
    B (Some (result, final-state))
  shows
    hash-relation-hit
    (checked-staged-composition-fri-verifier-prequery-relation A B)
    adversary-initial-state attacker-state
  proof -
    let ?s =
      verifier-state-from-adversary attacker-state
      (staged-proof-transcript data)
  from hit obtain trace-bs dg comp-bs fr trace-roots trace-final as
    composition-roots final rest where
    challenges:
      accepted-fri-challenges ?s (Some (result, final-state))
      trace-bs dg comp-bs
  and header:
    verifier-header-transcript ?s fr trace-roots trace-final as dg
    composition-roots final rest
  and comp-bs-B: comp-bs ∈ B dg
  and prequeried:
    composition-fri-challenge-path-prequeried ?s fr trace-roots
    trace-final as dg composition-roots (length composition-roots)
  unfolding composition-fri-challenge-list-prequery-hit-def by blast
  from prequeried obtain i y where
    i-len: i < length composition-roots
  and lookup-s:
    fmlookup (HashMap ?s)
    (composition-fri-challenge-key-at ?s fr trace-roots trace-final as
      dg composition-roots i) =
    Some y
  unfolding composition-fri-challenge-path-prequeried-def
  by (auto split: option.splits)
  have lookup-info:
    i < length comp-bs ∧ y = comp-bs ! i
  by (rule verify-monad-composition-fri-prequery-lookup
    [OF verifier challenges header i-len])
    (use lookup-s in
      ⟨simp add: composition-fri-challenge-key-at-def⟩)
  have i-round: i < ceil-log (maxDegree + 1)
  proof -
    have comp-bs ∈ fri-challenge-space (ceil-log (to-nat dg + 1))
    by (rule accepted-fri-challenges-composition-space[OF challenges])

```

```

then have len-comp: length comp-bs = ceil-log (to-nat dg + 1)
  unfolding fri-challenge-space-def by simp
have dg-bound: to-nat dg ≤ maxDegree
  by (rule verify-monad-accepted-fri-challenges-degree-bound
    [OF verifier challenges])
have ceil-log (to-nat dg + 1) ≤ ceil-log (maxDegree + 1)
  by (rule ceil-log-to-nat-degree-bound [OF dg-bound])
then show ?thesis
  using lookup-info len-comp by simp
qed
have relation:
  checked-staged-composition-fri-verifier-prequery-relation A B
    (composition-fri-challenge-key-at ?s fr trace-roots trace-final as dg
      composition-roots i) y
  unfolding checked-staged-composition-fri-verifier-prequery-relation-def
  apply (rule exI [where x = data])
  apply (rule exI [where x = attacker-state])
  apply (rule exI [where x = dg])
  apply (rule exI [where x = comp-bs])
  apply (rule exI [where x = fr])
  apply (rule exI [where x = trace-roots])
  apply (rule exI [where x = trace-final])
  apply (rule exI [where x = as])
  apply (rule exI [where x = composition-roots])
  apply (rule exI [where x = i])
  using builder comp-bs-B i-round i-len lookup-info by simp
have initial-absent:
  fmlookup (HashMap adversary-initial-state)
    (composition-fri-challenge-key-at ?s fr trace-roots trace-final as dg
      composition-roots i) = None
  unfolding adversary-initial-state-def by simp
have attacker-lookup:
  fmlookup (HashMap attacker-state)
    (composition-fri-challenge-key-at ?s fr trace-roots trace-final as dg
      composition-roots i) = Some y
  using lookup-s unfolding verifier-state-from-adversary-def by simp
show ?thesis
  unfolding hash-relation-hit-def
  using initial-absent attacker-lookup relation by blast
qed

lemma checked-staged-security-trace-fri-verifier-prequery-hit-bound:
assumes wf: staged-budget-wellformed budgets
and controlled: staged-adversary-controlled budgets A
and finite-B: finite B
shows
  wp-event (checked-staged-security-experiment-with-data-state A)
    (staged-security-with-data-state-verifier-event
      ( $\lambda s. \text{trace-fri-challenge-list-prequery-hit } s \ B$ ))

```

```

    adversary-initial-state ≤
    hash-relation-budget-value (card B * ceil-log clength)
    (staged-attacker-query-budget budgets + staged-challenge-query-budget)
unfolding checked-staged-security-experiment-with-data-state-def
proof (rule wp-event-bind-bound-by-head-event)
show wp-event (checked-staged-transcript-program A)
    (hash-relation-hit-event
    (checked-staged-trace-fri-verifier-prequery-relation A B)
    adversary-initial-state)
    adversary-initial-state
    ≤ hash-relation-budget-value (card B * ceil-log clength)
    (staged-attacker-query-budget budgets + staged-challenge-query-budget)
by (rule checked-staged-transcript-trace-fri-verifier-prequery-relation-bound
    [OF wf controlled finite-B])
next
show staged-security-with-data-state-verifier-event
    (λs. trace-fri-challenge-list-prequery-hit s B) None ⇒
    hash-relation-hit-event
    (checked-staged-trace-fri-verifier-prequery-relation A B)
    adversary-initial-state None
unfolding staged-security-with-data-state-verifier-event-def by simp
next
fix data attacker-state out
assume builder:
    Some (data, attacker-state) ∈
    set-dist
    (execute (checked-staged-transcript-program A)
    adversary-initial-state)
and cont:
    out ∈
    set-dist
    (execute
    (get >>=
    (λs. put
    (verifier-state-from-adversary s
    (staged-proof-transcript data)) >>=
    (λ-. verify-monad >>=
    (λresult. return ((data, s), result))))))
    attacker-state)
and hit:
    staged-security-with-data-state-verifier-event
    (λs. trace-fri-challenge-list-prequery-hit s B) out
let ?s =
    verifier-state-from-adversary attacker-state
    (staged-proof-transcript data)
from cont hit obtain result final-state where
    verifier:
    Some (result, final-state) ∈ set-dist (execute verify-monad ?s)
and prequery:

```

```

    trace-fri-challenge-list-prequery-hit ?s B
    (Some (result, final-state))
  unfolding staged-security-with-data-state-verifier-event-def
  by (auto elim!: set-dist-bindE split: option.splits prod.splits)
have relation-hit:
  hash-relation-hit
    (checked-staged-trace-fri-verifier-prequery-relation A B)
    adversary-initial-state attacker-state
  by (rule checked-staged-trace-fri-verifier-prequery-hit-imp-relation-hit
    [OF builder verifier prequery])
then show
  hash-relation-hit-event
    (checked-staged-trace-fri-verifier-prequery-relation A B)
    adversary-initial-state (Some (data, attacker-state))
  unfolding hash-relation-hit-event-def by simp
qed

lemma checked-staged-security-composition-fri-verifier-prequery-hit-bound:
  assumes wf: staged-budget-wellformed budgets
  and controlled: staged-adversary-controlled budgets A
  and finite-B:  $\bigwedge dg. \text{finite } (B \ dg)$ 
  shows
    wp-event (checked-staged-security-experiment-with-data-state A)
      (staged-security-with-data-state-verifier-event
        ( $\lambda s. \text{composition-fri-challenge-list-prequery-hit } s \ B$ ))
      adversary-initial-state  $\leq$ 
      hash-relation-budget-value
        ( $\sum dg \in (UNIV :: 'f \text{ set}).$ 
           $\text{card } (B \ dg) * \text{ceil-log } (\text{maxDegree} + 1)$ )
        (staged-attacker-query-budget budgets + staged-challenge-query-budget)
  unfolding checked-staged-security-experiment-with-data-state-def
proof (rule wp-event-bind-bound-by-head-event)
  show wp-event (checked-staged-transcript-program A)
    (hash-relation-hit-event
      (checked-staged-composition-fri-verifier-prequery-relation A B)
      adversary-initial-state)
    adversary-initial-state
     $\leq$  hash-relation-budget-value
      ( $\sum dg \in (UNIV :: 'f \text{ set}).$ 
         $\text{card } (B \ dg) * \text{ceil-log } (\text{maxDegree} + 1)$ )
      (staged-attacker-query-budget budgets + staged-challenge-query-budget)
  by (rule
    checked-staged-transcript-composition-fri-verifier-prequery-relation-bound
    [OF wf controlled finite-B])
next
  show staged-security-with-data-state-verifier-event
    ( $\lambda s. \text{composition-fri-challenge-list-prequery-hit } s \ B$ ) None  $\implies$ 
    hash-relation-hit-event
      (checked-staged-composition-fri-verifier-prequery-relation A B)

```

```

    adversary-initial-state None
  unfolding staged-security-with-data-state-verifier-event-def by simp
next
fix data attacker-state out
assume builder:
  Some (data, attacker-state) ∈
    set-dist
      (execute (checked-staged-transcript-program A)
        adversary-initial-state)
and cont:
  out ∈
    set-dist
      (execute
        (get >>=
          (λs. put
            (verifier-state-from-adversary s
              (staged-proof-transcript data)) >>=
            (λ-. verify-monad >>=
              (λresult. return ((data, s), result))))))
          attacker-state)
and hit:
  staged-security-with-data-state-verifier-event
    (λs. composition-fri-challenge-list-prequery-hit s B) out
let ?s =
  verifier-state-from-adversary attacker-state
    (staged-proof-transcript data)
from cont hit obtain result final-state where
  verifier:
    Some (result, final-state) ∈ set-dist (execute verify-monad ?s)
  and prequery:
    composition-fri-challenge-list-prequery-hit ?s B
    (Some (result, final-state))
  unfolding staged-security-with-data-state-verifier-event-def
  by (auto elim!: set-dist-bindE split: option.splits prod.splits)
have relation-hit:
  hash-relation-hit
    (checked-staged-composition-fri-verifier-prequery-relation A B)
    adversary-initial-state attacker-state
  by (rule
    checked-staged-composition-fri-verifier-prequery-hit-imp-relation-hit
      [OF builder verifier prequery])
then show
  hash-relation-hit-event
    (checked-staged-composition-fri-verifier-prequery-relation A B)
    adversary-initial-state (Some (data, attacker-state))
  unfolding hash-relation-hit-event-def by simp
qed

```

lemma *checked-staged-trace-fri-prequery-hit-imp-fixed-relation-hit:*

assumes *wf: staged-budget-wellformed budgets*
and *controlled: staged-adversary-controlled budgets A*
and *outcome:*
Some (data, attacker-state) ∈
set-dist
(execute (checked-staged-transcript-program A)
adversary-initial-state)
and *hit:*
checked-staged-trace-fri-prequery-hit B
(Some (data, attacker-state))
shows
hash-relation-hit
(checked-staged-trace-fri-prequery-relation A B)
adversary-initial-state attacker-state
proof –
let *?s =*
verifier-state-from-adversary attacker-state
(staged-proof-transcript data)
have *member: staged-trace-fri-challenges data ∈ B*
using *hit unfolding checked-staged-trace-fri-prequery-hit-def* **by** *simp*
have *prequeried:*
trace-fri-challenge-path-prequeried ?s
(staged-trace-root data) (staged-trace-fri-roots data)
(length (staged-trace-fri-roots data))
using *hit unfolding checked-staged-trace-fri-prequery-hit-def* **by** *simp*
from *prequeried* **obtain** *i* **where**
i-len: i < length (staged-trace-fri-roots data)
and *lookup-s:*
fmlookup (HashMap ?s)
(trace-fri-challenge-key-at ?s (staged-trace-root data)
(staged-trace-fri-roots data) i) ≠ None
unfolding *trace-fri-challenge-path-prequeried-def* **by** *blast*
have *shape:*
length (staged-trace-fri-roots data) = ceil-log clength ∧
length (staged-trace-fri-challenges data) = ceil-log clength ∧
length (staged-alphas data) = length spec ∧
length (staged-composition-fri-roots data) =
ceil-log (to-nat (staged-degree data) + 1) ∧
length (staged-composition-fri-challenges data) =
ceil-log (to-nat (staged-degree data) + 1) ∧
ceil-log (to-nat (staged-degree data) + 1) ≤
ceil-log (maxDegree + 1) ∧
length (staged-query-chunks data) = rounds
by *(rule checked-staged-transcript-program-outcome-shape*
[OF wf controlled outcome])
have *unchecked-out:*
Some (data, attacker-state) ∈
set-dist (execute (staged-transcript-program A)
adversary-initial-state)

```

  by (rule checked-staged-transcript-program-outcome-imp-staged-transcript-program
      [OF outcome])
have staged-lookup:
  fmllookup (HashMap attacker-state)
    (TraceFriChallenge i
      (foldl concat (staged-trace-fri-start-hash data)
        (take (Suc i) (staged-trace-fri-roots data)))) =
    Some (staged-trace-fri-challenges data ! i)
  by (rule staged-transcript-program-outcome-trace-challenge-lookup
      [OF wf controlled unchecked-out])
    (use i-len shape in simp)
have key-eq:
  trace-fri-challenge-key-at ?s (staged-trace-root data)
    (staged-trace-fri-roots data) i =
    TraceFriChallenge i
      (foldl concat (staged-trace-fri-start-hash data)
        (take (Suc i) (staged-trace-fri-roots data)))
  unfolding trace-fri-challenge-key-at-def staged-trace-fri-start-hash-def
  by simp
have relation:
  checked-staged-trace-fri-prequery-relation A B
    (trace-fri-challenge-key-at ?s (staged-trace-root data)
      (staged-trace-fri-roots data) i)
    (staged-trace-fri-challenges data ! i)
  unfolding checked-staged-trace-fri-prequery-relation-def
  by (intro exI[of - data] exI[of - attacker-state] exI[of - i])
    (use outcome member i-len shape in simp-all)
have initial-absent:
  fmllookup (HashMap adversary-initial-state)
    (trace-fri-challenge-key-at ?s (staged-trace-root data)
      (staged-trace-fri-roots data) i) = None
  unfolding adversary-initial-state-def by simp
have staged-lookup-key:
  fmllookup (HashMap attacker-state)
    (trace-fri-challenge-key-at ?s (staged-trace-root data)
      (staged-trace-fri-roots data) i) =
    Some (staged-trace-fri-challenges data ! i)
  using staged-lookup key-eq by simp
show ?thesis
  unfolding hash-relation-hit-def
  using initial-absent staged-lookup-key relation by blast
qed

lemma checked-staged-composition-fri-prequery-hit-imp-fixed-relation-hit:
  assumes wf: staged-budget-wellformed budgets
  and controlled: staged-adversary-controlled budgets A
  and outcome:
    Some (data, attacker-state) ∈
      set-dist

```

```

      (execute (checked-staged-transcript-program A)
        adversary-initial-state)
    and hit:
      checked-staged-composition-fri-prequery-hit B
      (Some (data, attacker-state))
    shows
      hash-relation-hit
      (checked-staged-composition-fri-prequery-relation A B)
      adversary-initial-state attacker-state
  proof -
    let ?s =
      verifier-state-from-adversary attacker-state
      (staged-proof-transcript data)
    have member:
      staged-composition-fri-challenges data ∈ B (staged-degree data)
    using hit unfolding checked-staged-composition-fri-prequery-hit-def
    by simp
    have prequeried:
      composition-fri-challenge-path-prequeried ?s
      (staged-trace-root data) (staged-trace-fri-roots data)
      (staged-trace-final data) (staged-alphas data) (staged-degree data)
      (staged-composition-fri-roots data)
      (length (staged-composition-fri-roots data))
    using hit unfolding checked-staged-composition-fri-prequery-hit-def
    by simp
    from prequeried obtain i where
      i-len: i < length (staged-composition-fri-roots data)
    and lookup-s:
      fmlookup (HashMap ?s)
      (composition-fri-challenge-key-at ?s (staged-trace-root data)
        (staged-trace-fri-roots data) (staged-trace-final data)
        (staged-alphas data) (staged-degree data)
        (staged-composition-fri-roots data) i) ≠ None
    unfolding composition-fri-challenge-path-prequeried-def by blast
    have shape:
      length (staged-trace-fri-roots data) = ceil-log clength ∧
      length (staged-trace-fri-challenges data) = ceil-log clength ∧
      length (staged-alphas data) = length spec ∧
      length (staged-composition-fri-roots data) =
        ceil-log (to-nat (staged-degree data) + 1) ∧
      length (staged-composition-fri-challenges data) =
        ceil-log (to-nat (staged-degree data) + 1) ∧
      ceil-log (to-nat (staged-degree data) + 1) ≤
        ceil-log (maxDegree + 1) ∧
      length (staged-query-chunks data) = rounds
    by (rule checked-staged-transcript-program-outcome-shape
      [OF wf controlled outcome])
    have unchecked-out:
      Some (data, attacker-state) ∈

```



```

    set-dist (execute (staged-transcript-program A)
      adversary-initial-state)
  by (rule checked-staged-transcript-program-outcome-imp-staged-transcript-program
    [OF outcome])
have staged-lookup:
  fmlookup (HashMap attacker-state)
    (CompositionFriChallenge i
      (foldl concat (staged-composition-fri-start-hash data)
        (take (Suc i) (staged-composition-fri-roots data)))) =
    Some (staged-composition-fri-challenges data ! i)
  by (rule staged-transcript-program-outcome-composition-challenge-lookup
    [OF wf controlled unchecked-out])
    (use i-len shape in simp)
have key-eq:
  composition-fri-challenge-key-at ?s (staged-trace-root data)
    (staged-trace-fri-roots data) (staged-trace-final data)
    (staged-alphas data) (staged-degree data)
    (staged-composition-fri-roots data) i =
    CompositionFriChallenge i
      (foldl concat (staged-composition-fri-start-hash data)
        (take (Suc i) (staged-composition-fri-roots data)))
  unfolding composition-fri-challenge-key-at-def
    staged-composition-fri-start-hash-def staged-trace-fri-start-hash-def
  by simp
have relation:
  checked-staged-composition-fri-prequery-relation A B
    (composition-fri-challenge-key-at ?s (staged-trace-root data)
      (staged-trace-fri-roots data) (staged-trace-final data)
      (staged-alphas data) (staged-degree data)
      (staged-composition-fri-roots data) i)
    (staged-composition-fri-challenges data ! i)
  unfolding checked-staged-composition-fri-prequery-relation-def
  by (intro exI[of - data] exI[of - attacker-state] exI[of - i])
    (use outcome member i-len shape in simp-all)
have initial-absent:
  fmlookup (HashMap adversary-initial-state)
    (composition-fri-challenge-key-at ?s (staged-trace-root data)
      (staged-trace-fri-roots data) (staged-trace-final data)
      (staged-alphas data) (staged-degree data)
      (staged-composition-fri-roots data) i) = None
  unfolding adversary-initial-state-def by simp
have staged-lookup-key:
  fmlookup (HashMap attacker-state)
    (composition-fri-challenge-key-at ?s (staged-trace-root data)
      (staged-trace-fri-roots data) (staged-trace-final data)
      (staged-alphas data) (staged-degree data)
      (staged-composition-fri-roots data) i) =
    Some (staged-composition-fri-challenges data ! i)
  using staged-lookup key-eq by simp

```

```

show ?thesis
  unfolding hash-relation-hit-def
  using initial-absent staged-lookup-key relation by blast
qed

lemma checked-staged-transcript-trace-fri-prequery-hit-bound:
  assumes wf: staged-budget-wellformed budgets
  and controlled: staged-adversary-controlled budgets A
  and finite-B: finite B
  shows
    wp-event (checked-staged-transcript-program A)
      (checked-staged-trace-fri-prequery-hit B)
      adversary-initial-state  $\leq$ 
      hash-relation-budget-value (card B * ceil-log clength)
      (staged-attacker-query-budget budgets + staged-challenge-query-budget)
proof -
  have prehit-le-relation:
    wp-event (checked-staged-transcript-program A)
      (checked-staged-trace-fri-prequery-hit B)
      adversary-initial-state  $\leq$ 
    wp-event (checked-staged-transcript-program A)
      (hash-relation-hit-event
        (checked-staged-trace-fri-prequery-relation A B)
        adversary-initial-state)
      adversary-initial-state
  proof (rule wp-event-mono-on-support)
  fix out
  assume support:
    out  $\in$  set-dist
    (execute (checked-staged-transcript-program A)
      adversary-initial-state)
  and hit: checked-staged-trace-fri-prequery-hit B out
  show
    hash-relation-hit-event
      (checked-staged-trace-fri-prequery-relation A B)
      adversary-initial-state out
  proof (cases out)
  case None
  then show ?thesis
    using hit unfolding checked-staged-trace-fri-prequery-hit-def by simp
  next
  case (Some result)
  then obtain data attacker-state where out-eq:
    out = Some (data, attacker-state)
  by (cases result) simp
  have relation-hit:
    hash-relation-hit
      (checked-staged-trace-fri-prequery-relation A B)
      adversary-initial-state attacker-state

```

```

    by (rule checked-staged-trace-fri-prequery-hit-imp-fixed-relation-hit
        [OF wf controlled - -])
        (use support hit out-eq in simp-all)
  then show ?thesis
    unfolding out-eq hash-relation-hit-event-def by simp
qed
qed
also have ... ≤
  hash-relation-budget-value (card B * ceil-log clength)
  (staged-attacker-query-budget budgets + staged-challenge-query-budget)
proof -
  have program:
    hash-relation-program
    (checked-staged-trace-fri-prequery-relation A B)
    (card B * ceil-log clength)
    (staged-attacker-query-budget budgets + staged-challenge-query-budget)
    (checked-staged-transcript-program A)
  by (rule hash-relation-program-checked-staged-transcript-program
      [OF wf controlled])
      (rule checked-staged-trace-fri-prequery-relation-fiber-card-bound
          [OF finite-B])
  show ?thesis
    using program
    unfolding hash-relation-program-def hash-relation-budget-def by blast
qed
finally show ?thesis .
qed

```

lemma *checked-staged-transcript-composition-fri-prequery-hit-bound:*

assumes *wf: staged-budget-wellformed budgets*
and *controlled: staged-adversary-controlled budgets A*
and *finite-B: $\bigwedge dg. \text{finite } (B \ dg)$*

shows

wp-event (checked-staged-transcript-program A)
(checked-staged-composition-fri-prequery-hit B)
adversary-initial-state ≤
hash-relation-budget-value
 $(\sum dg \in (UNIV :: 'f \text{ set}).$
 $\text{card } (B \ dg) * \text{ceil-log } (\text{maxDegree} + 1))$
(staged-attacker-query-budget budgets + staged-challenge-query-budget)

proof -

have *prehit-le-relation:*

wp-event (checked-staged-transcript-program A)
(checked-staged-composition-fri-prequery-hit B)
adversary-initial-state ≤
wp-event (checked-staged-transcript-program A)
(hash-relation-hit-event
(checked-staged-composition-fri-prequery-relation A B)
adversary-initial-state)

```

    adversary-initial-state
proof (rule wp-event-mono-on-support)
  fix out
  assume support:
    out ∈ set-dist
    (execute (checked-staged-transcript-program A)
      adversary-initial-state)
  and hit: checked-staged-composition-fri-prequery-hit B out
show
  hash-relation-hit-event
  (checked-staged-composition-fri-prequery-relation A B)
  adversary-initial-state out
proof (cases out)
  case None
  then show ?thesis
    using hit unfolding checked-staged-composition-fri-prequery-hit-def
    by simp
next
  case (Some result)
  then obtain data attacker-state where out-eq:
    out = Some (data, attacker-state)
  by (cases result) simp
have relation-hit:
  hash-relation-hit
  (checked-staged-composition-fri-prequery-relation A B)
  adversary-initial-state attacker-state
  by (rule
    checked-staged-composition-fri-prequery-hit-imp-fixed-relation-hit
    [OF wf controlled - -])
  (use support hit out-eq in simp-all)
  then show ?thesis
    unfolding out-eq hash-relation-hit-event-def by simp
qed
qed
also have ... ≤
  hash-relation-budget-value
  (∑ dg ∈ (UNIV :: 'f set).
    card (B dg) * ceil-log (maxDegree + 1))
  (staged-attacker-query-budget budgets + staged-challenge-query-budget)
proof -
  have program:
  hash-relation-program
  (checked-staged-composition-fri-prequery-relation A B)
  (∑ dg ∈ (UNIV :: 'f set).
    card (B dg) * ceil-log (maxDegree + 1))
  (staged-attacker-query-budget budgets + staged-challenge-query-budget)
  (checked-staged-transcript-program A)
  by (rule hash-relation-program-checked-staged-transcript-program
    [OF wf controlled])

```

```

      (rule
        checked-staged-composition-fri-prequery-relation-fiber-card-bound
        [OF finite-B])
    show ?thesis
      using program
      unfolding hash-relation-program-def hash-relation-budget-def by blast
  qed
  finally show ?thesis .
qed

end

context soundness
begin

lemma checked-staged-trace-fri-prequery-hit-imp-relation-hit:
  assumes wf: staged-budget-wellformed budgets
  and controlled: staged-adversary-controlled budgets A
  and outcome:
    Some (data, attacker-state) ∈
      set-dist
        (execute (checked-staged-transcript-program A)
          adversary-initial-state)
  and hit:
    checked-staged-trace-fri-prequery-hit B
    (Some (data, attacker-state))
  shows
    hash-relation-hit
      (trace-fri-challenge-key-relation
        (verifier-state-from-adversary attacker-state
          (staged-proof-transcript data))
        B (length (staged-trace-fri-roots data)))
      adversary-initial-state attacker-state
proof -
  let ?s =
    verifier-state-from-adversary attacker-state
    (staged-proof-transcript data)
  have member: staged-trace-fri-challenges data ∈ B
  using hit unfolding checked-staged-trace-fri-prequery-hit-def by simp
  have prequeried:
    trace-fri-challenge-path-prequeried ?s
      (staged-trace-root data) (staged-trace-fri-roots data)
      (length (staged-trace-fri-roots data))
  using hit unfolding checked-staged-trace-fri-prequery-hit-def by simp
  from prequeried obtain i where
    i-len: i < length (staged-trace-fri-roots data)
  and lookup-s:
    fmlookup (HashMap ?s)
      (trace-fri-challenge-key-at ?s (staged-trace-root data))

```

```

    (staged-trace-fri-roots data) i) ≠ None
unfolding trace-fri-challenge-path-prequeried-def by blast
have shape:
  length (staged-trace-fri-roots data) = ceil-log clength ∧
  length (staged-trace-fri-challenges data) = ceil-log clength ∧
  length (staged-alphas data) = length spec ∧
  length (staged-composition-fri-roots data) =
    ceil-log (to-nat (staged-degree data) + 1) ∧
  length (staged-composition-fri-challenges data) =
    ceil-log (to-nat (staged-degree data) + 1) ∧
  ceil-log (to-nat (staged-degree data) + 1) ≤
    ceil-log (maxDegree + 1) ∧
  length (staged-query-chunks data) = rounds
by (rule checked-staged-transcript-program-outcome-shape
  [OF wf controlled outcome])
have unchecked-out:
  Some (data, attacker-state) ∈
    set-dist (execute (staged-transcript-program A)
      adversary-initial-state)
by (rule checked-staged-transcript-program-outcome-imp-staged-transcript-program
  [OF outcome])
have staged-lookup:
  fmlookup (HashMap attacker-state)
    (TraceFriChallenge i
      (foldl concat (staged-trace-fri-start-hash data)
        (take (Suc i) (staged-trace-fri-roots data)))) =
    Some (staged-trace-fri-challenges data ! i)
by (rule staged-transcript-program-outcome-trace-challenge-lookup
  [OF wf controlled unchecked-out])
  (use i-len shape in simp)
have key-eq:
  trace-fri-challenge-key-at ?s (staged-trace-root data)
    (staged-trace-fri-roots data) i =
    TraceFriChallenge i
      (foldl concat (staged-trace-fri-start-hash data)
        (take (Suc i) (staged-trace-fri-roots data)))
unfolding trace-fri-challenge-key-at-def staged-trace-fri-start-hash-def
by simp
have relation:
  trace-fri-challenge-key-relation ?s B
    (length (staged-trace-fri-roots data))
    (trace-fri-challenge-key-at ?s (staged-trace-root data)
      (staged-trace-fri-roots data) i)
    (staged-trace-fri-challenges data ! i)
proof —
  have i-challenges: i < length (staged-trace-fri-challenges data)
    using i-len shape by simp
  show ?thesis
    unfolding trace-fri-challenge-key-relation-def

```

```

    by (intro bexI[of - staged-trace-fri-challenges data]
        exI[of - staged-trace-root data]
        exI[of - staged-trace-fri-roots data]
        exI[of - i])
    (use member i-len i-challenges in simp-all)
qed
have initial-absent:
  fmllookup (HashMap adversary-initial-state)
    (trace-fri-challenge-key-at ?s (staged-trace-root data)
      (staged-trace-fri-roots data) i) = None
  unfolding adversary-initial-state-def by simp
have staged-lookup-key:
  fmllookup (HashMap attacker-state)
    (trace-fri-challenge-key-at ?s (staged-trace-root data)
      (staged-trace-fri-roots data) i) =
    Some (staged-trace-fri-challenges data ! i)
  using staged-lookup key-eq by simp
show ?thesis
  unfolding hash-relation-hit-def
  using initial-absent staged-lookup-key relation by blast
qed

lemma checked-staged-composition-fri-prequery-hit-imp-relation-hit:
  assumes wf: staged-budget-wellformed budgets
  and controlled: staged-adversary-controlled budgets A
  and outcome:
    Some (data, attacker-state) ∈
      set-dist
        (execute (checked-staged-transcript-program A)
          adversary-initial-state)
  and hit:
    checked-staged-composition-fri-prequery-hit B
      (Some (data, attacker-state))
  shows
    hash-relation-hit
      (composition-fri-challenge-key-relation
        (verifier-state-from-adversary attacker-state
          (staged-proof-transcript data)))
      B (length (staged-composition-fri-roots data)))
      adversary-initial-state attacker-state
proof -
  let ?s =
    verifier-state-from-adversary attacker-state
      (staged-proof-transcript data)
  have member:
    staged-composition-fri-challenges data ∈ B (staged-degree data)
  using hit unfolding checked-staged-composition-fri-prequery-hit-def
  by simp
have prequeried:

```

```

composition-fri-challenge-path-prequeried ?s
  (staged-trace-root data) (staged-trace-fri-roots data)
  (staged-trace-final data) (staged-alphas data) (staged-degree data)
  (staged-composition-fri-roots data)
  (length (staged-composition-fri-roots data))
using hit unfolding checked-staged-composition-fri-prequery-hit-def
by simp
from prequeried obtain i where
  i-len: i < length (staged-composition-fri-roots data)
and lookup-s:
  fmlookup (HashMap ?s)
    (composition-fri-challenge-key-at ?s (staged-trace-root data)
      (staged-trace-fri-roots data) (staged-trace-final data)
      (staged-alphas data) (staged-degree data)
      (staged-composition-fri-roots data) i) ≠ None
unfolding composition-fri-challenge-path-prequeried-def by blast
have shape:
  length (staged-trace-fri-roots data) = ceil-log clength ∧
  length (staged-trace-fri-challenges data) = ceil-log clength ∧
  length (staged-alphas data) = length spec ∧
  length (staged-composition-fri-roots data) =
    ceil-log (to-nat (staged-degree data) + 1) ∧
  length (staged-composition-fri-challenges data) =
    ceil-log (to-nat (staged-degree data) + 1) ∧
  ceil-log (to-nat (staged-degree data) + 1) ≤
    ceil-log (maxDegree + 1) ∧
  length (staged-query-chunks data) = rounds
by (rule checked-staged-transcript-program-outcome-shape
  [OF wf controlled outcome])
have unchecked-out:
  Some (data, attacker-state) ∈
    set-dist (execute (staged-transcript-program A)
      adversary-initial-state)
by (rule checked-staged-transcript-program-outcome-imp-staged-transcript-program
  [OF outcome])
have staged-lookup:
  fmlookup (HashMap attacker-state)
    (CompositionFriChallenge i
      (foldl concat (staged-composition-fri-start-hash data)
        (take (Suc i) (staged-composition-fri-roots data)))) =
    Some (staged-composition-fri-challenges data ! i)
by (rule staged-transcript-program-outcome-composition-challenge-lookup
  [OF wf controlled unchecked-out])
  (use i-len shape in simp)
have key-eq:
  composition-fri-challenge-key-at ?s (staged-trace-root data)
    (staged-trace-fri-roots data) (staged-trace-final data)
    (staged-alphas data) (staged-degree data)
    (staged-composition-fri-roots data) i =

```



```

    CompositionFriChallenge i
      (foldl concat (staged-composition-fri-start-hash data)
        (take (Suc i) (staged-composition-fri-roots data)))
  unfolding composition-fri-challenge-key-at-def
    staged-composition-fri-start-hash-def staged-trace-fri-start-hash-def
  by simp
have relation:
  composition-fri-challenge-key-relation ?s B
    (length (staged-composition-fri-roots data))
    (composition-fri-challenge-key-at ?s (staged-trace-root data)
      (staged-trace-fri-roots data) (staged-trace-final data)
      (staged-alphas data) (staged-degree data)
      (staged-composition-fri-roots data) i)
    (staged-composition-fri-challenges data ! i)
proof -
  have i-challenges: i < length (staged-composition-fri-challenges data)
    using i-len shape by simp
  show ?thesis
    unfolding composition-fri-challenge-key-relation-def
    by (intro exI[of - staged-degree data]
      bexI[of - staged-composition-fri-challenges data]
      exI[of - staged-trace-root data]
      exI[of - staged-trace-fri-roots data]
      exI[of - staged-trace-final data]
      exI[of - staged-alphas data]
      exI[of - staged-composition-fri-roots data]
      exI[of - i])
      (use member i-len i-challenges in simp-all)
qed
have initial-absent:
  fmlookup (HashMap adversary-initial-state)
    (composition-fri-challenge-key-at ?s (staged-trace-root data)
      (staged-trace-fri-roots data) (staged-trace-final data)
      (staged-alphas data) (staged-degree data)
      (staged-composition-fri-roots data) i) = None
  unfolding adversary-initial-state-def by simp
have staged-lookup-key:
  fmlookup (HashMap attacker-state)
    (composition-fri-challenge-key-at ?s (staged-trace-root data)
      (staged-trace-fri-roots data) (staged-trace-final data)
      (staged-alphas data) (staged-degree data)
      (staged-composition-fri-roots data) i) =
    Some (staged-composition-fri-challenges data ! i)
  using staged-lookup key-eq by simp
show ?thesis
  unfolding hash-relation-hit-def
  using initial-absent staged-lookup-key relation by blast
qed

```

end

end

theory *Soundness-FRI-Prequery-Challenge-Bounds*
 imports *Soundness-FRI-Prequery-Core*
begin

Staged and verifier-local FRI challenge-list accounting built on the prequery core definitions and relation bounds.

context *soundness*
begin

definition *trace-fri-root-path-fresh-from*
 $:: (f, 'a) \text{ protocol-channel-scheme} \Rightarrow 'f \text{ list} \Rightarrow \text{nat} \Rightarrow \text{bool}$
where
 $\text{trace-fri-root-path-fresh-from } s \text{ roots } n \longleftrightarrow$
 $(\forall i < n.$
 $i < \text{length roots} \longrightarrow$
 $\text{fmlookup } (\text{HashMap } s)$
 $(\text{TraceFriChallenge } (P\text{TraceFriCounter } s + i)$
 $(\text{foldl concat } (P\text{State } s) (\text{take } (\text{Suc } i) \text{ roots}))) = \text{None})$

definition *composition-fri-root-path-fresh-from*
 $:: (f, 'a) \text{ protocol-channel-scheme} \Rightarrow 'f \text{ list} \Rightarrow \text{nat} \Rightarrow \text{bool}$
where
 $\text{composition-fri-root-path-fresh-from } s \text{ roots } n \longleftrightarrow$
 $(\forall i < n.$
 $i < \text{length roots} \longrightarrow$
 $\text{fmlookup } (\text{HashMap } s)$
 $(\text{CompositionFriChallenge } (P\text{CompositionFriCounter } s + i)$
 $(\text{foldl concat } (P\text{State } s) (\text{take } (\text{Suc } i) \text{ roots}))) = \text{None})$

lemma *trace-fri-challenge-path-fresh-after-read-iff:*
 assumes *read-fr:* $\text{Some } (fr, s1) \in \text{set-dist } (\text{execute read } s)$
 shows
 $\text{trace-fri-challenge-path-fresh } s \text{ fr roots } n \longleftrightarrow$
 $\text{trace-fri-root-path-fresh-from } s1 \text{ roots } n$

proof –

have *state:* $P\text{State } s1 = \text{concat } (P\text{State } s) \text{ fr}$
 using *read-outcome*[*OF read-fr*] **by** *blast*
 have *counter:* $P\text{TraceFriCounter } s1 = P\text{TraceFriCounter } s$
 using *read-outcome*[*OF read-fr*] **by** *blast*
 have *hash:* $\text{HashMap } s1 = \text{HashMap } s$
 by (*rule read-preserves-hash-map*[*OF read-fr*])
 show *?thesis*
 unfolding *trace-fri-challenge-path-fresh-def*
 trace-fri-root-path-fresh-from-def

by (simp add: trace-fri-challenge-key-at-def state counter hash)
qed

lemma *receive-trace-fri-commits-preserves-trace-root-path-fresh-tail:*
assumes *path:*
trace-fri-root-path-fresh-from s (r # roots) (Suc n)
and *outcome:*
Some ((b, r), t) ∈ set-dist (execute receive-trace-fri-commits s)
shows *trace-fri-root-path-fresh-from t roots n*

proof –
from *outcome* **obtain** *s-read* **where**
read-r: Some (r, s-read) ∈ set-dist (execute read s)
and *rand-b:*
Some (b, t) ∈ set-dist (execute receive-trace-fri-challenge s-read)
unfolding *receive-trace-fri-commits-def receive-fri-commits-with-def*
by (auto elim!: set-dist-bindE)
from *read-outcome[OF read-r]* **have** *read-state:*
PState s-read = concat (PState s) r
by blast
from *read-outcome[OF read-r]* **have** *read-counter:*
PTraceFriCounter s-read = PTraceFriCounter s
by blast
have *read-hash: HashMap s-read = HashMap s*
by (rule read-preserves-hash-map[OF read-r])
have *counter-t:*
PTraceFriCounter t = Suc (PTraceFriCounter s)
using *receive-trace-fri-challenge-counter-outcome[OF rand-b]*
read-counter **by** simp
have *state-t: PState t = concat (PState s) r*
using *receive-trace-fri-challenge-outcome[OF rand-b]* *read-state* **by** simp
show *?thesis*
unfolding *trace-fri-root-path-fresh-from-def*
proof (intro allI impI)
fix *i*
assume *i-lt: i < n*
and *i-len: i < length roots*
let *?key =*
TraceFriChallenge (PTraceFriCounter t + i)
(foldl concat (PState t) (take (Suc i) roots))
have *key-eq:*
?key =
TraceFriChallenge (PTraceFriCounter s + Suc i)
(foldl concat (PState s) (take (Suc (Suc i)) (r # roots)))
using *counter-t state-t* **by** simp
have *neg:*
?key ≠ TraceFriChallenge (PTraceFriCounter s-read) (PState s-read)
using *counter-t read-counter* **by** simp
have *fmlookup (HashMap t) ?key =*
fmlookup (HashMap s-read) ?key

```

    by (rule receive-trace-fri-challenge-preserves-other-lookup
        [OF rand-b neq])
  also have ... = fmlookup (HashMap s) ?key
    using read-hash by simp
  also have ... = None
  proof -
    have Suc i < Suc n
      using i-lt by simp
    moreover have Suc i < length (r # roots)
      using i-len by simp
    ultimately show ?thesis
      using path unfolding trace-fri-root-path-fresh-from-def key-eq
      by blast
  qed
  finally show fmlookup (HashMap t) ?key = None .
qed
qed

lemma wp-receive-trace-fri-commits-actual-fresh-set:
  shows
    wp-event receive-trace-fri-commits
      (λout. case out of
        None ⇒ False
        | Some ((b, r), -) ⇒
          b ∈ B ∧
          fmlookup (HashMap s)
            (TraceFriChallenge (PTraceFriCounter s)
              (concat (PState s) r)) = None) s
      ≤ nnreal (card B) / nnreal size
  unfolding receive-trace-fri-commits-def receive-fri-commits-with-def
  proof (rule wp-event-bind-bound-by-cont)
    show ¬ (case None of None ⇒ False
      | Some ((b, r), -) ⇒
        b ∈ B ∧
        fmlookup (HashMap s)
          (TraceFriChallenge (PTraceFriCounter s)
            (concat (PState s) r)) = None)
      by simp
  next
    fix r s-read
    assume read-r: Some (r, s-read) ∈ set-dist (execute read s)
    have read-state: PState s-read = concat (PState s) r
      using read-outcome[OF read-r] by blast
    have read-counter: PTraceFriCounter s-read = PTraceFriCounter s
      using read-outcome[OF read-r] by blast
    have read-hash: HashMap s-read = HashMap s
      by (rule read-preserves-hash-map[OF read-r])
    let ?fresh =
      fmlookup (HashMap s-read)

```

```

    (TraceFriChallenge (PTraceFriCounter s-read) (PState s-read)) = None
  let ?Q =
    λout :: (('f × 'f) × ('f, 'a) protocol-channel-scheme) option.
      case out of
      None ⇒ False
      | Some ((b, r'), -) ⇒
        b ∈ B ∧
        fmlookup (HashMap s)
          (TraceFriChallenge (PTraceFriCounter s)
            (concat (PState s) r')) = None
  show wp-event
    (receive-trace-fri-challenge ≫= (λb. return (b, r))) ?Q
    s-read
    ≤ nnreal (card B) / nnreal size
  proof (cases ?fresh)
  case True
  have fresh-key:
    fmlookup (HashMap s)
      (TraceFriChallenge (PTraceFriCounter s) (concat (PState s) r)) =
      None
  using True by (simp add: read-state read-counter read-hash)
  have cont-eq:
    (λout. case out of
      None ⇒ (0 :: prob)
      | Some (b, u) ⇒
        wp (return (b, r)) (λout. if ?Q out then 1 else 0) u) =
    (λout :: ('f × ('f, 'a) protocol-channel-scheme) option.
      if (case out of None ⇒ False | Some (b, -) ⇒ b ∈ B)
      then 1 else 0)
  by (rule ext)
  (auto simp: wpsimps fresh-key split: option.splits prod.splits)
  have wp-event
    (receive-trace-fri-challenge ≫= (λb. return (b, r))) ?Q
    s-read =
    wp-event receive-trace-fri-challenge
      (λout. case out of None ⇒ False | Some (b, -) ⇒ b ∈ B)
      s-read
  unfolding wp-event-def by (simp add: wpsimps cont-eq)
  also have ... = nnreal (card B) / nnreal size
  by (rule wp-receive-trace-fri-challenge-fresh-set[OF True])
  finally show ?thesis by simp
next
case False
have not-fresh-s:
  fmlookup (HashMap s)
    (TraceFriChallenge (PTraceFriCounter s) (concat (PState s) r))
    ≠ None
  using False by (simp add: read-state read-counter read-hash)
have wp-event

```

```

    (receive-trace-fri-challenge  $\gg$  ( $\lambda b$ . return (b, r))) ?Q
  s-read
 $\leq$  wp-event
    (receive-trace-fri-challenge  $\gg$  ( $\lambda b$ . return (b, r)))
    ( $\lambda$ - :: (('f  $\times$  'f)  $\times$ 
      ('f, 'a) protocol-channel-scheme) option. False) s-read
  by (rule wp-event-mono-on-support)
    (use not-fresh-s in
       $\langle$ auto simp: wpsimps elim!: set-dist-bindE
        split: option.splits prod.splits $\rangle$ )
  also have ... = 0
    unfolding wp-event-def wp-def dist-expect-def by simp
  finally have zero-bound:
    wp-event
      (receive-trace-fri-challenge  $\gg$  ( $\lambda b$ . return (b, r))) ?Q
      s-read  $\leq$  0 .
  also have ...  $\leq$  nnreal (card B) / nnreal size
    by simp
  finally show ?thesis .
qed
qed

lemma wp-ntimes-receive-trace-fri-commits-exact-challenges-actual-fresh-bound:
  assumes len: length bs = n
  shows
    wp-event (ntimes receive-trace-fri-commits n)
      ( $\lambda$ out. case out of
        None  $\Rightarrow$  False
        | Some (brs, -)  $\Rightarrow$ 
          map fst brs = bs  $\wedge$ 
          trace-fri-root-path-fresh-from s (map snd brs) n) s
       $\leq$  (1 / nnreal size)  $^n$ 
  using len
proof (induction n arbitrary: s bs)
  case 0
  then have bs-empty: bs = []
    by simp
  show ?case
    unfolding bs-empty by (simp add: wp-event-def wpsimps
      trace-fri-root-path-fresh-from-def)
next
  case (Suc n)
  obtain b bs' where bs-eq: bs = b # bs'
    using Suc.prem by (cases bs) auto
  have len-bs': length bs' = n
    using Suc.prem bs-eq by simp
  let ?tail =
    (ntimes receive-trace-fri-commits n ::
      (('f  $\times$  'f) list, ('f, 'a) protocol-channel-scheme) state-monad)

```

```

let ?Q =
  λout :: (((f × f) list) ×
    (f, a) protocol-channel-scheme) option.
  case out of
    None ⇒ False
  | Some (brs, -) ⇒
    map fst brs = bs ∧
    trace-fri-root-path-fresh-from s (map snd brs) (Suc n)
let ?Head =
  λout :: (((f × f) × (f, a) protocol-channel-scheme) option.
    case out of
      None ⇒ False
    | Some ((b0, r0), -) ⇒
      b0 ∈ {b} ∧
      fmlookup (HashMap s)
        (TraceFriChallenge (PTraceFriCounter s)
          (concat (PState s) r0)) = None
have head-bound:
  wp-event receive-trace-fri-commits ?Head s ≤ 1 / nnreal size
proof -
  have wp-event receive-trace-fri-commits ?Head s
    ≤ nnreal (card ({b} :: f set)) / nnreal size
    by (rule wp-receive-trace-fri-commits-actual-fresh-set)
  then show ?thesis by simp
qed
have bind-bound:
  wp-event
    (receive-trace-fri-commits ≫=
      (λbr :: f × f.
        ?tail ≫= (λbrs :: (f × f) list. return (br # brs))))
    ?Q s
  ≤ wp-event receive-trace-fri-commits ?Head s *
    (1 / nnreal size) ^ n
proof (rule wp-event-bind-bound-by-head-and-cont[where P = ?Head])
  show ¬ ?Q None
  by simp
next
  fix br t
  assume not-head: ¬ ?Head (Some (br, t))
  obtain b0 r0 where br-eq: br = (b0, r0)
  by (cases br)
  have cont-false:
    wp-event (?tail ≫= (λbrs :: (f × f) list. return (br # brs)))
      ?Q t
    ≤ wp-event (?tail ≫= (λbrs :: (f × f) list. return (br # brs)))
      (λ- :: (((f × f) list) ×
        (f, a) protocol-channel-scheme) option. False) t
  by (rule wp-event-mono-on-support)
    (use not-head bs-eq br-eq in

```

```

    ‹auto simp: wpsimps trace-fri-root-path-fresh-from-def
      elim!: set-dist-bindE split: option.splits prod.splits›)
  also have ... = 0
    unfolding wp-event-def wp-def dist-expect-def by simp
  finally have le-zero:
    wp-event (?tail  $\gg$  ( $\lambda$ brs :: ('f  $\times$  'f) list. return (br # brs)))
    ?Q t  $\leq$  0 .
  show
    wp-event (?tail  $\gg$  ( $\lambda$ brs :: ('f  $\times$  'f) list. return (br # brs)))
    ?Q t = 0
    by (rule antisym[OF le-zero]) simp
next
  fix br t
  assume head:
    Some (br, t)  $\in$  set-dist (execute receive-trace-fri-commits s)
    and head-hit: ?Head (Some (br, t))
  obtain b0 r0 where br-eq: br = (b0, r0)
    by (cases br)
  have b0-eq: b0 = b
    using head-hit unfolding br-eq by simp
  let ?TailQ =
     $\lambda$ out :: (((('f  $\times$  'f) list)  $\times$ 
      ('f, 'a) protocol-channel-scheme) option.
      case out of
        None  $\Rightarrow$  False
      | Some (brs, -)  $\Rightarrow$ 
        map fst brs = bs'  $\wedge$ 
        trace-fri-root-path-fresh-from t (map snd brs) n
  have cont-le:
    wp-event (?tail  $\gg$  ( $\lambda$ brs :: ('f  $\times$  'f) list. return (br # brs)))
    ?Q t  $\leq$  wp-event ?tail ?TailQ t
  proof (rule wp-event-bind-bound-by-head-event)
    show wp-event ?tail ?TailQ t  $\leq$  wp-event ?tail ?TailQ t
      by simp
  next
    show ?Q None  $\implies$  ?TailQ None
      by simp
next
  fix brs u out
  assume tail-out:
    Some (brs, u)  $\in$  set-dist (execute ?tail t)
    and ret-out:
      out  $\in$  set-dist (execute (return (br # brs)) u)
    and hit: ?Q out
  have out-eq: out = Some (br # brs, u)
    using ret-out
    unfolding set-dist-def return.rep-eq dist-return-def
      dist-delta-dist delta-map-def by simp
  have map-eq: map fst brs = bs'

```



```

    using hit out-eq b0-eq bs-eq br-eq by simp
  have path-full:
    trace-fri-root-path-fresh-from s (map snd (br # brs)) (Suc n)
    using hit out-eq by simp
  have path-full':
    trace-fri-root-path-fresh-from s (r0 # map snd brs) (Suc n)
    using path-full unfolding br-eq by simp
  have path-tail:
    trace-fri-root-path-fresh-from t (map snd brs) n
    by (rule receive-trace-fri-commits-preserves-trace-root-path-fresh-tail
        [OF path-full' head[unfolded br-eq]])
  show ?TailQ (Some (brs, u))
    using map-eq path-tail by simp
qed
also have ... ≤ (1 / nnreal size) ^ n
  by (rule Suc.IH[OF len-bs])
finally show
  wp-event (?tail ≫ (λbrs :: ('f × 'f) list. return (br # brs)))
    ?Q t ≤ (1 / nnreal size) ^ n .
qed
have wp-event (ntimes receive-trace-fri-commits (Suc n)) ?Q s
  ≤ wp-event receive-trace-fri-commits ?Head s *
    (1 / nnreal size) ^ n
  using bind-bound by simp
also have ... ≤ (1 / nnreal size) * (1 / nnreal size) ^ n
  by (rule mult-right-mono[OF head-bound]) simp
also have ... = (1 / nnreal size) ^ Suc n
  by simp
finally show ?case .
qed

lemma wp-ntimes-receive-trace-fri-commits-finite-set-actual-fresh-bound:
  assumes finite-B: finite B
  and lengths: ⋀bs. bs ∈ B ⇒ length bs = n
  shows
    wp-event (ntimes receive-trace-fri-commits n)
      (λout. case out of
        None ⇒ False
        | Some (brs, -) ⇒
          map fst brs ∈ B ∧
          trace-fri-root-path-fresh-from s (map snd brs) n) s
      ≤ nnreal (card B) / (nnreal size) ^ n
proof -
  let ?P =
    λbs out. case out of
      None ⇒ False
      | Some (brs, -) ⇒
        map fst brs = bs ∧
        trace-fri-root-path-fresh-from s (map snd brs) n

```

have *event-mono*:
 $wp\text{-}event\ (ntimes\ receive\text{-}trace\text{-}fri\text{-}commits\ n)$
 $(\lambda out. case\ out\ of$
 $\quad None \Rightarrow False$
 $\quad | Some\ (brs, -) \Rightarrow$
 $\quad\quad map\ fst\ brs \in B \wedge$
 $\quad\quad trace\text{-}fri\text{-}root\text{-}path\text{-}fresh\text{-}from\ s\ (map\ snd\ brs)\ n)\ s$
 $\leq wp\text{-}event\ (ntimes\ receive\text{-}trace\text{-}fri\text{-}commits\ n)$
 $(\lambda out. \exists bs \in B. ?P\ bs\ out)\ s$
by (*rule wp-event-mono*) (*auto split: option.splits prod.splits*)
also have $\dots \leq (\sum bs \in B. (1 / nnreal\ size) ^ n)$
by (*rule wp-event-finite-UN-bound[OF finite-B]*)
 $(rule\ wp\text{-}ntimes\ receive\text{-}trace\text{-}fri\text{-}commits\text{-}exact\text{-}challenges\text{-}actual\text{-}fresh\text{-}bound$
 $\quad [OF\ lengths])$
also have $\dots = nnreal\ (card\ B) * ((1 / nnreal\ size) ^ n)$
using *finite-B* **by** *simp*
also have $\dots = nnreal\ (card\ B) / (nnreal\ size) ^ n$
using *size-card* **by** *transfer (simp add: power-divide)*
finally show *?thesis* .
qed

lemma *wp-ntimes-receive-trace-fri-commits-challenge-space-actual-fresh-bound*:
assumes *subset: B ⊆ fri-challenge-space n*
shows

$wp\text{-}event\ (ntimes\ receive\text{-}trace\text{-}fri\text{-}commits\ n)$
 $(\lambda out. case\ out\ of$
 $\quad None \Rightarrow False$
 $\quad | Some\ (brs, -) \Rightarrow$
 $\quad\quad map\ fst\ brs \in B \wedge$
 $\quad\quad trace\text{-}fri\text{-}root\text{-}path\text{-}fresh\text{-}from\ s\ (map\ snd\ brs)\ n)\ s$
 $\leq nnreal\ (card\ B) / nnreal\ (CARD('f) ^ n)$
proof –
have *finite-B: finite B*
by (*rule finite-subset[OF subset finite-fri-challenge-space]*)
have *lengths: $\bigwedge bs. bs \in B \implies length\ bs = n$*
using *subset unfolding fri-challenge-space-def* **by** *auto*
have $wp\text{-}event\ (ntimes\ receive\text{-}trace\text{-}fri\text{-}commits\ n)$
 $(\lambda out. case\ out\ of$
 $\quad None \Rightarrow False$
 $\quad | Some\ (brs, -) \Rightarrow$
 $\quad\quad map\ fst\ brs \in B \wedge$
 $\quad\quad trace\text{-}fri\text{-}root\text{-}path\text{-}fresh\text{-}from\ s\ (map\ snd\ brs)\ n)\ s$
 $\leq nnreal\ (card\ B) / (nnreal\ size) ^ n$
by (*rule wp-ntimes-receive-trace-fri-commits-finite-set-actual-fresh-bound*
 $\quad [OF\ finite\text{-}B\ lengths])$
also have $\dots = nnreal\ (card\ B) / nnreal\ (CARD('f) ^ n)$
using *size-card* **by** *simp*
finally show *?thesis* .
qed

lemma *wp-verifier-trace-fri-prefix-challenge-list-actual-fresh-bound:*
assumes *subset: $B \subseteq \text{fri-challenge-space } (\text{ceil-log } \text{clength})$*
shows
 $\text{wp-event verifier-trace-fri-prefix}$
 $(\lambda \text{out. case out of}$
 $\quad \text{None} \Rightarrow \text{False}$
 $\quad | \text{Some } ((\text{fr}, \text{f-fl}), -) \Rightarrow$
 $\quad \quad \text{map fst f-fl} \in B \wedge$
 $\quad \quad \text{trace-fri-challenge-path-fresh } s \text{ fr } (\text{map snd f-fl})$
 $\quad \quad (\text{length } (\text{map snd f-fl})) \text{ s}$
 $\quad \leq \text{nnreal } (\text{card } B) / \text{nnreal } (\text{CARD('f)} \wedge \text{ceil-log } \text{clength})$
proof –
let $?Q =$
 $\lambda \text{out. case out of}$
 $\quad \text{None} \Rightarrow \text{False}$
 $\quad | \text{Some } ((\text{fr}, \text{f-fl}), -) \Rightarrow$
 $\quad \quad \text{map fst f-fl} \in B \wedge$
 $\quad \quad \text{trace-fri-challenge-path-fresh } s \text{ fr } (\text{map snd f-fl})$
 $\quad \quad (\text{length } (\text{map snd f-fl}))$
let $?C = \text{nnreal } (\text{card } B) / \text{nnreal } (\text{CARD('f)} \wedge \text{ceil-log } \text{clength})$
show *?thesis*
unfolding *verifier-trace-fri-prefix-def*
proof (rule *wp-event-bind-bound-by-cont*)
show $\neg ?Q \text{ None}$
by *simp*
next
fix $\text{fr } s1$
assume *read-fr: $\text{Some } (\text{fr}, s1) \in \text{set-dist } (\text{execute read } s)$*
let $?Head =$
 $\lambda \text{out. case out of}$
 $\quad \text{None} \Rightarrow \text{False}$
 $\quad | \text{Some } (\text{f-fl}, -) \Rightarrow$
 $\quad \quad \text{map fst f-fl} \in B \wedge$
 $\quad \quad \text{trace-fri-root-path-fresh-from } s1 \text{ (map snd f-fl)}$
 $\quad \quad (\text{ceil-log } \text{clength})$
have *trace-bound:*
 $\text{wp-event } (\text{ntimes receive-trace-fri-commits } (\text{ceil-log } \text{clength}))$
 $?Head \text{ s1} \leq ?C$
by (rule *wp-ntimes-receive-trace-fri-commits-challenge-space-actual-fresh-bound*
 $[\text{OF subset}]$)
show *wp-event*
 $(\text{ntimes receive-trace-fri-commits } (\text{ceil-log } \text{clength}) \gg=$
 $\quad (\lambda \text{f-fl. return } (\text{fr}, \text{f-fl})))$
 $?Q \text{ s1} \leq ?C$
proof (rule *wp-event-bind-bound-by-head-event[OF trace-bound]*)
show $?Q \text{ None} \Longrightarrow ?Head \text{ None}$
by *simp*
next

```

fix f-fl t out
assume trace-out:
  Some (f-fl, t) ∈
    set-dist
      (execute (ntimes receive-trace-fri-commits (ceil-log clength))
        s1)
and cont:
  out ∈ set-dist (execute (return (fr, f-fl)) t)
and hit: ?Q out
have out-eq: out = Some ((fr, f-fl), t)
using cont
unfolding set-dist-def return.rep-eq dist-return-def
  dist-delta-dist delta-map-def by simp
have len: length f-fl = ceil-log clength
using ntimes-receive-trace-fri-commits-outcome[OF trace-out] by simp
have path:
  trace-fri-root-path-fresh-from s1 (map snd f-fl)
    (ceil-log clength)
using hit out-eq len
  trace-fri-challenge-path-fresh-after-read-iff[OF read-fr]
by simp
show ?Head (Some (f-fl, t))
using hit out-eq path by simp
qed
qed
qed

lemma wp-verify-monad-trace-fri-challenge-list-fresh-hit-actual-bound:
assumes subset:  $B \subseteq \text{fri-challenge-space } (\text{ceil-log clength})$ 
shows
  wp-event verify-monad (trace-fri-challenge-list-fresh-hit s B)  $s \leq$ 
    nnreal (card B) / nnreal ( $\text{CARD}(f) \wedge \text{ceil-log clength}$ )
proof –
let ?Head =
  λout. case out of
    None ⇒ False
  | Some ((fr, f-fl), -) ⇒
    map fst f-fl ∈ B ∧
    trace-fri-challenge-path-fresh s fr (map snd f-fl)
      (length (map snd f-fl))
have prefix-bound:
  wp-event verifier-trace-fri-prefix ?Head  $s \leq$ 
    nnreal (card B) / nnreal ( $\text{CARD}(f) \wedge \text{ceil-log clength}$ )
by (rule wp-verifier-trace-fri-prefix-challenge-list-actual-fresh-bound
  [OF subset])
show ?thesis
unfolding verify-monad-trace-fri-decomposition
proof (rule wp-event-bind-bound-by-head-event[OF prefix-bound])
show trace-fri-challenge-list-fresh-hit s B None ⇒

```

?Head None
unfolding *trace-fri-challenge-list-fresh-hit-def*
accepted-fri-challenges-def **by** *blast*
next
fix *header prefix-state out*
assume *prefix*:
Some (header, prefix-state) ∈
set-dist (execute verifier-trace-fri-prefix s)
and *suffix*:
out ∈
set-dist (execute (verifier-after-trace-fri header) prefix-state)
and *hit: trace-fri-challenge-list-fresh-hit s B out*
obtain *fr f-fl* **where** *header-eq: header = (fr, f-fl)*
by *(cases header) auto*
from *hit* **obtain** *trace-bs dg comp-bs fr' trace-roots' trace-final' as'*
composition-roots' final' rest' where
challenges-hit: accepted-fri-challenges s out trace-bs dg comp-bs
and *header-hit:*
verifier-header-transcript s fr' trace-roots' trace-final' as' dg
composition-roots' final' rest'
and *trace-bs-B: trace-bs ∈ B*
and *fresh-hit:*
trace-fri-challenge-path-fresh s fr' trace-roots'
(length trace-roots')
unfolding *trace-fri-challenge-list-fresh-hit-def* **by** *blast*
from *challenges-hit* **obtain** *result final-state* **where** *out-eq:*
out = Some (result, final-state)
unfolding *accepted-fri-challenges-def* **by** *blast*
have *prefix'*:
Some ((fr, f-fl), prefix-state) ∈
set-dist (execute verifier-trace-fri-prefix s)
using *prefix header-eq* **by** *simp*
have *suffix'*:
Some (result, final-state) ∈
set-dist
(execute (verifier-after-trace-fri (fr, f-fl)) prefix-state)
using *suffix header-eq out-eq* **by** *simp*
from *verifier-after-trace-fri-accepted-fri-challenges[OF prefix' suffix']*
obtain *dg' comp-bs'* **where**
challenges-prefix:
accepted-fri-challenges s (Some (result, final-state))
(map fst f-fl) dg' comp-bs'
by *blast*
have *trace-bs-eq: trace-bs = map fst f-fl*
using *accepted-fri-challenges-unique*
[OF challenges-prefix challenges-hit[unfolded out-eq]]
by *simp*
from *challenges-prefix* **obtain** *result0 final-state0 fr0 f-fl0*
trace-final0 as0 fl0 final0 query-state0 **where** *body0:*

$\text{Some } (\text{result}, \text{final-state}) = \text{Some } (\text{result0}, \text{final-state0}) \wedge$
 $\text{verifier-header-transcript } s \text{ fr0 } (\text{map snd f-fl0}) \text{ trace-final0 as0}$
 $\text{dg}' (\text{map snd fl0}) \text{ final0 } (P\text{Transcript query-state0}) \wedge$
 $P\text{State query-state0} =$
 $\text{verifier-header-state } s \text{ fr0 } (\text{map snd f-fl0}) \text{ trace-final0 as0}$
 $\text{dg}' (\text{map snd fl0}) \text{ final0 } \wedge$
 $\text{Some } (\text{result0}, \text{final-state0}) \in$
 $\text{set-dist } (\text{execute}$
 $\quad (\text{ntimes}$
 $\quad \quad (\text{verifier-query-round-program fr0 f-fl0 trace-final0 as0}$
 $\quad \quad \quad \text{fl0 final0})$
 $\quad \quad \text{rounds})$
 $\quad \text{query-state0}) \wedge$
 $P\text{QueryCounter query-state0} = P\text{QueryCounter } s \wedge$
 $\text{map fst f-fl} = \text{map fst f-fl0} \wedge$
 $\text{comp-bs}' = \text{map fst fl0} \wedge$
 $(\forall i < \text{length f-fl0}.$
 $\quad \text{fmlookup } (\text{HashMap query-state0})$
 $\quad \quad (\text{TraceFriChallenge } (P\text{TraceFriCounter } s + i)$
 $\quad \quad \quad (\text{foldl concat } (\text{concat } (P\text{State } s) \text{ fr0})$
 $\quad \quad \quad \quad (\text{take } (\text{Suc } i) (\text{map snd f-fl0})))) =$
 $\quad \quad \text{Some } (\text{fst } (f-fl0 ! i))) \wedge$
 $(\forall i < \text{length fl0}.$
 $\quad \text{fmlookup } (\text{HashMap query-state0})$
 $\quad \quad (\text{CompositionFriChallenge } (P\text{CompositionFriCounter } s + i)$
 $\quad \quad \quad (\text{foldl concat}$
 $\quad \quad \quad \quad (\text{concat}$
 $\quad \quad \quad \quad \quad (\text{foldl concat}$
 $\quad \quad \quad \quad \quad \quad (\text{concat}$
 $\quad \quad \quad \quad \quad \quad \quad (\text{foldl concat } (\text{concat } (P\text{State } s) \text{ fr0})$
 $\quad \quad \quad \quad \quad \quad \quad \quad (\text{map snd f-fl0}))$
 $\quad \quad \quad \quad \quad \quad \quad \quad \text{trace-final0})$
 $\quad \quad \quad \quad \quad \quad \quad \quad \text{as0})$
 $\quad \quad \quad \quad \quad \quad \quad \quad \text{dg}')$
 $\quad \quad \quad (\text{take } (\text{Suc } i) (\text{map snd fl0})))) =$
 $\quad \quad \text{Some } (\text{fst } (fl0 ! i)))$
unfolding *accepted-fri-challenges-def* **by** *blast*
have *header-prefix0*:
 $\text{verifier-header-transcript } s \text{ fr0 } (\text{map snd f-fl0}) \text{ trace-final0 as0}$
 $\text{dg}' (\text{map snd fl0}) \text{ final0 } (P\text{Transcript query-state0})$
using *body0* **by** *simp*
have *trace-bs0*: $\text{map fst f-fl} = \text{map fst f-fl0}$
using *body0* **by** *simp*
have *prefix-res*:
 $P\text{Transcript } s = [\text{fr}] @ \text{map snd f-fl} @ P\text{Transcript prefix-state}$
using *verifier-trace-fri-prefix-outcome[OF prefix]* **by** *simp*
have *len-f-fl*: $\text{length f-fl} = \text{ceil-log clength}$
using *verifier-trace-fri-prefix-outcome[OF prefix]* **by** *simp*
have *fr0-eq*: $\text{fr0} = \text{fr}$

```

using prefix-res header-prefix0
unfolding verifier-header-transcript-def verifier-header-messages-def
by simp
have len-f-fl0: length f-fl0 = ceil-log clength
using header-prefix0
unfolding verifier-header-transcript-def by simp
have roots0-eq: map snd f-fl0 = map snd f-fl
proof –
  let ?rest0 =
    trace-final0 # as0 @ dg' # map snd fl0 @ final0 #
    PTranscript query-state0
  have prefix0:
    PTranscript s = [fr] @ map snd f-fl0 @ ?rest0
  using header-prefix0 fr0-eq
  unfolding verifier-header-transcript-def verifier-header-messages-def
  by simp
  have eq:
    [fr] @ map snd f-fl @ PTranscript prefix-state =
    [fr] @ map snd f-fl0 @ ?rest0
  using prefix-res prefix0 by simp
  have take-eq:
    take (Suc (length (map snd f-fl)))
      ([fr] @ map snd f-fl @ PTranscript prefix-state) =
    take (Suc (length (map snd f-fl)))
      ([fr] @ map snd f-fl0 @ ?rest0)
  using eq by simp
  then show ?thesis
  using len-f-fl len-f-fl0 by simp
qed
have header-prefix:
  verifier-header-transcript s fr (map snd f-fl) trace-final0 as0
  dg' (map snd fl0) final0 (PTranscript query-state0)
  using header-prefix0 fr0-eq roots0-eq by simp
have roots-eq:
  fr' = fr  $\wedge$  trace-roots' = map snd f-fl
  using verifier-header-transcript-unique[OF header-prefix header-hit]
  by simp
then have fresh-prefix:
  trace-fri-challenge-path-fresh s fr (map snd f-fl)
  (length (map snd f-fl))
  using fresh-hit by simp
show ?Head (Some (header, prefix-state))
  using trace-bs-B trace-bs-eq fresh-prefix header-eq by simp
qed
qed

lemma receive-composition-fri-commits-preserves-composition-root-path-fresh-tail:
assumes path:
  composition-fri-root-path-fresh-from s (r # roots) (Suc n)

```

and *outcome*:
Some ((b, r), t) ∈ set-dist (execute receive-composition-fri-commits s)
shows *composition-fri-root-path-fresh-from t roots n*
proof –
from *outcome* **obtain** *s-read* **where**
read-r: Some (r, s-read) ∈ set-dist (execute read s)
and *rand-b*:
Some (b, t) ∈
set-dist (execute receive-composition-fri-challenge s-read)
unfolding *receive-composition-fri-commits-def* *receive-fri-commits-with-def*
by (*auto elim! set-dist-bindE*)
from *read-outcome*[*OF read-r*] **have** *read-state*:
PState s-read = concat (PState s) r
by *blast*
from *read-outcome*[*OF read-r*] **have** *read-counter*:
PCompositionFriCounter s-read = PCompositionFriCounter s
by *blast*
have *read-hash: HashMap s-read = HashMap s*
by (*rule read-preserves-hash-map*[*OF read-r*])
have *counter-t*:
PCompositionFriCounter t = Suc (PCompositionFriCounter s)
using *receive-composition-fri-challenge-counter-outcome*[*OF rand-b*]
read-counter **by** *simp*
have *state-t: PState t = concat (PState s) r*
using *receive-composition-fri-challenge-outcome*[*OF rand-b*] *read-state*
by *simp*
show *?thesis*
unfolding *composition-fri-root-path-fresh-from-def*
proof (*intro allI impI*)
fix *i*
assume *i-lt: i < n*
and *i-len: i < length roots*
let *?key =*
CompositionFriChallenge (PCompositionFriCounter t + i)
(foldl concat (PState t) (take (Suc i) roots))
have *key-eq*:
?key =
CompositionFriChallenge (PCompositionFriCounter s + Suc i)
(foldl concat (PState s) (take (Suc (Suc i)) (r # roots)))
using *counter-t state-t* **by** *simp*
have *neg*:
?key ≠
CompositionFriChallenge (PCompositionFriCounter s-read)
(PState s-read)
using *counter-t read-counter* **by** *simp*
have *fmlookup (HashMap t) ?key =*
fmlookup (HashMap s-read) ?key
by (*rule receive-composition-fri-challenge-preserves-other-lookup*
[OF rand-b neg])


```

also have ... = fmlookup (HashMap s) ?key
  using read-hash by simp
also have ... = None
proof –
  have Suc i < Suc n
    using i-lt by simp
  moreover have Suc i < length (r # roots)
    using i-len by simp
  ultimately show ?thesis
    using path unfolding composition-fri-root-path-fresh-from-def key-eq
    by blast
qed
finally show fmlookup (HashMap t) ?key = None .
qed
qed

lemma wp-receive-composition-fri-commits-actual-fresh-set:
shows
  wp-event receive-composition-fri-commits
    ( $\lambda out. case\ out\ of$ 
      None  $\Rightarrow$  False
      | Some ((b, r), -)  $\Rightarrow$ 
        b  $\in$  B  $\wedge$ 
        fmlookup (HashMap s)
          (CompositionFriChallenge (PCompositionFriCounter s)
            (concat (PState s) r)) = None) s
         $\leq$  nnreal (card B) / nnreal size
    unfolding receive-composition-fri-commits-def receive-fri-commits-with-def
proof (rule wp-event-bind-bound-by-cont)
show  $\neg$  (case None of None  $\Rightarrow$  False
  | Some ((b, r), -)  $\Rightarrow$ 
    b  $\in$  B  $\wedge$ 
    fmlookup (HashMap s)
      (CompositionFriChallenge (PCompositionFriCounter s)
        (concat (PState s) r)) = None)
    by simp
next
fix r s-read
assume read-r: Some (r, s-read)  $\in$  set-dist (execute read s)
have read-state: PState s-read = concat (PState s) r
  using read-outcome[OF read-r] by blast
have read-counter:
  PCompositionFriCounter s-read = PCompositionFriCounter s
  using read-outcome[OF read-r] by blast
have read-hash: HashMap s-read = HashMap s
  by (rule read-preserves-hash-map[OF read-r])
let ?fresh =
  fmlookup (HashMap s-read)
    (CompositionFriChallenge (PCompositionFriCounter s-read)

```

```

    (PState s-read)) = None
let ?Q =
  λout :: (('f × 'f) × ('f, 'a) protocol-channel-scheme) option.
    case out of
      None ⇒ False
    | Some ((b, r'), -) ⇒
      b ∈ B ∧
      fmlookup (HashMap s)
        (CompositionFriChallenge (PCompositionFriCounter s)
          (concat (PState s) r')) = None
show wp-event
  (receive-composition-fri-challenge ≫= (λb. return (b, r))) ?Q
  s-read
  ≤ nnreal (card B) / nnreal size
proof (cases ?fresh)
case True
have fresh-key:
  fmlookup (HashMap s)
    (CompositionFriChallenge (PCompositionFriCounter s)
      (concat (PState s) r)) = None
using True by (simp add: read-state read-counter read-hash)
have cont-eq:
  (λout. case out of
    None ⇒ (0 :: prob)
  | Some (b, u) ⇒
    wp (return (b, r)) (λout. if ?Q out then 1 else 0) u) =
  (λout :: ('f × ('f, 'a) protocol-channel-scheme) option.
    if (case out of None ⇒ False | Some (b, -) ⇒ b ∈ B)
    then 1 else 0)
by (rule ext)
(auto simp: wpsimps fresh-key split: option.splits prod.splits)
have wp-event
  (receive-composition-fri-challenge ≫= (λb. return (b, r))) ?Q
  s-read =
  wp-event receive-composition-fri-challenge
    (λout. case out of None ⇒ False | Some (b, -) ⇒ b ∈ B)
  s-read
  unfolding wp-event-def by (simp add: wpsimps cont-eq)
also have ... = nnreal (card B) / nnreal size
  by (rule wp-receive-composition-fri-challenge-fresh-set[OF True])
finally show ?thesis by simp
next
case False
have not-fresh-s:
  fmlookup (HashMap s)
    (CompositionFriChallenge (PCompositionFriCounter s)
      (concat (PState s) r)) ≠ None
using False by (simp add: read-state read-counter read-hash)
have wp-event

```

```

    (receive-composition-fri-challenge  $\gg$  ( $\lambda b. \text{return } (b, r)$ )) ?Q
  s-read
 $\leq$  wp-event
    (receive-composition-fri-challenge  $\gg$  ( $\lambda b. \text{return } (b, r)$ ))
    ( $\lambda - :: (('f \times 'f) \times$ 
      ( $'f, 'a$ ) protocol-channel-scheme) option. False) s-read
  by (rule wp-event-mono-on-support)
    (use not-fresh-s in
       $\langle$ auto simp: wpsimps elim!: set-dist-bindE
        split: option.splits prod.splits $\rangle$ )
  also have ... = 0
    unfolding wp-event-def wp-def dist-expect-def by simp
  finally have zero-bound:
    wp-event
      (receive-composition-fri-challenge  $\gg$  ( $\lambda b. \text{return } (b, r)$ )) ?Q
      s-read  $\leq 0$  .
  also have ...  $\leq$  nnreal (card B) / nnreal size
    by simp
  finally show ?thesis .
qed
qed

lemma wp-ntimes-receive-composition-fri-commits-exact-challenges-actual-fresh-bound:
  assumes len: length bs = n
  shows
    wp-event (ntimes receive-composition-fri-commits n)
      ( $\lambda$ out. case out of
        None  $\Rightarrow$  False
        | Some (brs, -)  $\Rightarrow$ 
          map fst brs = bs  $\wedge$ 
          composition-fri-root-path-fresh-from s (map snd brs) n) s
       $\leq (1 / \text{nnreal size}) ^ n$ 
  using len
proof (induction n arbitrary: s bs)
  case 0
  then have bs-empty: bs = []
    by simp
  show ?case
    unfolding bs-empty by (simp add: wp-event-def wpsimps
      composition-fri-root-path-fresh-from-def)
next
  case (Suc n)
  obtain b bs' where bs-eq: bs = b # bs'
    using Suc.prem by (cases bs) auto
  have len-bs': length bs' = n
    using Suc.prem bs-eq by simp
  let ?tail =
    (ntimes receive-composition-fri-commits n ::
      (('f  $\times$  'f) list, ('f, 'a) protocol-channel-scheme) state-monad)

```

```

let ?Q =
  λout :: (((f × f) list) ×
    (f, a) protocol-channel-scheme) option.
  case out of
    None ⇒ False
  | Some (brs, -) ⇒
    map fst brs = bs ∧
    composition-fri-root-path-fresh-from s (map snd brs) (Suc n)
let ?Head =
  λout :: (((f × f) × (f, a) protocol-channel-scheme) option.
    case out of
      None ⇒ False
    | Some ((b0, r0), -) ⇒
      b0 ∈ {b} ∧
      fmlookup (HashMap s)
        (CompositionFriChallenge (PCompositionFriCounter s)
          (concat (PState s) r0)) = None
have head-bound:
  wp-event receive-composition-fri-commits ?Head s ≤ 1 / nnreal size
proof -
  have wp-event receive-composition-fri-commits ?Head s
    ≤ nnreal (card ({b} :: f set)) / nnreal size
  by (rule wp-receive-composition-fri-commits-actual-fresh-set)
  then show ?thesis by simp
qed
have bind-bound:
  wp-event
    (receive-composition-fri-commits ≫
      (λbr :: f × f.
        ?tail ≫ (λbrs :: (f × f) list. return (br # brs))))
  ?Q s
  ≤ wp-event receive-composition-fri-commits ?Head s *
    (1 / nnreal size) ^ n
proof (rule wp-event-bind-bound-by-head-and-cont[where P = ?Head])
  show ¬ ?Q None
  by simp
next
  fix br t
  assume not-head: ¬ ?Head (Some (br, t))
  obtain b0 r0 where br-eq: br = (b0, r0)
  by (cases br)
  have cont-false:
    wp-event (?tail ≫ (λbrs :: (f × f) list. return (br # brs)))
    ?Q t
    ≤ wp-event (?tail ≫ (λbrs :: (f × f) list. return (br # brs)))
      (λ- :: (((f × f) list) ×
        (f, a) protocol-channel-scheme) option. False) t
  by (rule wp-event-mono-on-support)
  (use not-head bs-eq br-eq in

```

```

    ⟨auto simp: wpsimps composition-fri-root-path-fresh-from-def
      elim!: set-dist-bindE split: option.splits prod.splits⟩
  also have ... = 0
    unfolding wp-event-def wp-def dist-expect-def by simp
  finally have le-zero:
    wp-event (?tail ≫= (λbrs :: ('f × 'f) list. return (br # brs)))
      ?Q t ≤ 0 .
  show
    wp-event (?tail ≫= (λbrs :: ('f × 'f) list. return (br # brs)))
      ?Q t = 0
    by (rule antisym[OF le-zero]) simp
next
fix br t
assume head:
  Some (br, t) ∈
    set-dist (execute receive-composition-fri-commits s)
  and head-hit: ?Head (Some (br, t))
obtain b0 r0 where br-eq: br = (b0, r0)
  by (cases br)
have b0-eq: b0 = b
  using head-hit unfolding br-eq by simp
let ?TailQ =
  λout :: (((('f × 'f) list) ×
    ('f, 'a) protocol-channel-scheme) option.
    case out of
      None ⇒ False
    | Some (brs, -) ⇒
      map fst brs = bs' ∧
      composition-fri-root-path-fresh-from t (map snd brs) n
have cont-le:
  wp-event (?tail ≫= (λbrs :: ('f × 'f) list. return (br # brs)))
    ?Q t ≤ wp-event ?tail ?TailQ t
proof (rule wp-event-bind-bound-by-head-event)
  show wp-event ?tail ?TailQ t ≤ wp-event ?tail ?TailQ t
    by simp
next
  show ?Q None ⇒ ?TailQ None
    by simp
next
fix brs u out
assume tail-out:
  Some (brs, u) ∈ set-dist (execute ?tail t)
  and ret-out:
    out ∈ set-dist (execute (return (br # brs)) u)
  and hit: ?Q out
have out-eq: out = Some (br # brs, u)
  using ret-out
  unfolding set-dist-def return.rep-eq dist-return-def
    dist-delta-dist delta-map-def by simp

```

```

have map-eq: map fst brs = bs'
  using hit out-eq b0-eq bs-eq br-eq by simp
have path-full:
  composition-fri-root-path-fresh-from s (map snd (br # brs))
    (Suc n)
  using hit out-eq by simp
have path-full':
  composition-fri-root-path-fresh-from s (r0 # map snd brs)
    (Suc n)
  using path-full unfolding br-eq by simp
have path-tail:
  composition-fri-root-path-fresh-from t (map snd brs) n
  by (rule
    receive-composition-fri-commits-preserves-composition-root-path-fresh-tail
    [OF path-full' head[unfolded br-eq]])
show ?TailQ (Some (brs, u))
  using map-eq path-tail by simp
qed
also have ...  $\leq (1 / \text{nnreal size}) ^ n$ 
  by (rule Suc.IH[OF len-bs])
finally show
  wp-event (?tail  $\gg$  ( $\lambda \text{brs} :: ('f \times 'f) \text{ list. return (br \# brs)$ ))
    ?Q t  $\leq (1 / \text{nnreal size}) ^ n$  .
qed
have wp-event (ntimes receive-composition-fri-commits (Suc n)) ?Q s
   $\leq$  wp-event receive-composition-fri-commits ?Head s *
    (1 / nnreal size) ^ n
  using bind-bound by simp
also have ...  $\leq (1 / \text{nnreal size}) * (1 / \text{nnreal size}) ^ n$ 
  by (rule mult-right-mono[OF head-bound]) simp
also have ...  $= (1 / \text{nnreal size}) ^ \text{Suc } n$ 
  by simp
finally show ?case .
qed

lemma wp-ntimes-receive-composition-fri-commits-finite-set-actual-fresh-bound:
assumes finite-B: finite B
and lengths:  $\bigwedge bs. bs \in B \implies \text{length } bs = n$ 
shows
  wp-event (ntimes receive-composition-fri-commits n)
    ( $\lambda \text{out. case out of}$ 
      None  $\Rightarrow$  False
      | Some (brs, -)  $\Rightarrow$ 
        map fst brs  $\in B \wedge$ 
        composition-fri-root-path-fresh-from s (map snd brs) n) s
     $\leq \text{nnreal (card B) / (nnreal size)} ^ n$ 
proof -
let ?P =
   $\lambda bs \text{ out. case out of}$ 

```

```

    None  $\Rightarrow$  False
  | Some (brs, -)  $\Rightarrow$ 
    map fst brs = bs  $\wedge$ 
    composition-fri-root-path-fresh-from s (map snd brs) n
have event-mono:
  wp-event (ntimes receive-composition-fri-commits n)
    ( $\lambda$ out. case out of
      None  $\Rightarrow$  False
    | Some (brs, -)  $\Rightarrow$ 
      map fst brs  $\in B \wedge$ 
      composition-fri-root-path-fresh-from s (map snd brs) n) s
     $\leq$  wp-event (ntimes receive-composition-fri-commits n)
      ( $\lambda$ out.  $\exists$  bs  $\in B$ . ?P bs out) s
  by (rule wp-event-mono) (auto split: option.splits prod.splits)
also have ...  $\leq$  ( $\sum$  bs  $\in B$ . (1 / nnreal size)  $^n$ )
  by (rule wp-event-finite-UN-bound[OF finite-B])
    (rule wp-ntimes-receive-composition-fri-commits-exact-challenges-actual-fresh-bound
      [OF lengths])
also have ... = nnreal (card B) * ((1 / nnreal size)  $^n$ )
  using finite-B by simp
also have ... = nnreal (card B) / (nnreal size)  $^n$ 
  using size-card by transfer (simp add: power-divide)
finally show ?thesis .
qed

```

lemma wp-ntimes-receive-composition-fri-commits-challenge-space-actual-fresh-bound:

assumes subset: $B \subseteq \text{fri-challenge-space } n$

shows

```

  wp-event (ntimes receive-composition-fri-commits n)
    ( $\lambda$ out. case out of
      None  $\Rightarrow$  False
    | Some (brs, -)  $\Rightarrow$ 
      map fst brs  $\in B \wedge$ 
      composition-fri-root-path-fresh-from s (map snd brs) n) s
     $\leq$  nnreal (card B) / nnreal (CARD('f)  $^n$ )

```

proof –

have finite-B: finite B

by (rule finite-subset[OF subset finite-fri-challenge-space])

have lengths: $\bigwedge$ bs. bs $\in B \implies \text{length } bs = n$

using subset **unfolding** fri-challenge-space-def **by** auto

have wp-event (ntimes receive-composition-fri-commits n)

```

  ( $\lambda$ out. case out of
    None  $\Rightarrow$  False
  | Some (brs, -)  $\Rightarrow$ 
    map fst brs  $\in B \wedge$ 
    composition-fri-root-path-fresh-from s (map snd brs) n) s
   $\leq$  nnreal (card B) / (nnreal size)  $^n$ 

```

by (rule

wp-ntimes-receive-composition-fri-commits-finite-set-actual-fresh-bound

```

    [OF finite-B lengths])
also have ... = nnreal (card B) / nnreal (CARD('f) ^ n)
    using size-card by simp
    finally show ?thesis .
qed

lemma composition-fri-challenge-path-fresh-before-composition-iff:
assumes state:
  PState s-comp =
    concat
      (foldl concat
        (concat
          (foldl concat (concat (PState s) fr) trace-roots)
            trace-final)
        as)
      dg
and counter: PCompositionFriCounter s-comp = PCompositionFriCounter s
and hash: HashMap s-comp = HashMap s
shows
  composition-fri-challenge-path-fresh s fr trace-roots trace-final as dg
  composition-roots n  $\longleftrightarrow$ 
  composition-fri-root-path-fresh-from s-comp composition-roots n
unfolding composition-fri-challenge-path-fresh-def
  composition-fri-root-path-fresh-from-def
by (simp add: composition-fri-challenge-key-at-def state counter hash)

lemma composition-fri-challenge-path-fresh-from-preservedI:
assumes state:
  PState s-comp =
    concat
      (foldl concat
        (concat
          (foldl concat (concat (PState s) fr) trace-roots)
            trace-final)
        as)
      dg
and counter: PCompositionFriCounter s-comp = PCompositionFriCounter s
and lookup:
   $\bigwedge i x. \text{fmlookup} (\text{HashMap } s\text{-comp}) (\text{CompositionFriChallenge } i \ x) =$ 
    fmlookup (HashMap s) (CompositionFriChallenge i x)
and fresh:
  composition-fri-root-path-fresh-from s-comp composition-roots n
shows
  composition-fri-challenge-path-fresh s fr trace-roots trace-final as dg
  composition-roots n
unfolding composition-fri-challenge-path-fresh-def
  composition-fri-challenge-key-at-def
proof (intro allI impI)
fix i

```



```

assume i-lt:  $i < n$ 
and i-len:  $i < \text{length } \text{composition-roots}$ 
let ?key =
  CompositionFriChallenge (PCompositionFriCounter  $s + i$ )
    (foldl concat
      (concat
        (foldl concat
          (concat (foldl concat (concat (PState  $s$ ) fr) trace-roots)
            trace-final)
          as)
        dg)
      (take (Suc  $i$ ) composition-roots))
have ?key =
  CompositionFriChallenge (PCompositionFriCounter  $s\text{-comp} + i$ )
    (foldl concat (PState  $s\text{-comp}$ ) (take (Suc  $i$ ) composition-roots))
using state counter by simp
moreover have
  fmlookup (HashMap  $s\text{-comp}$ )
    (CompositionFriChallenge (PCompositionFriCounter  $s\text{-comp} + i$ )
      (foldl concat (PState  $s\text{-comp}$ ) (take (Suc  $i$ ) composition-roots))) =
    None
using fresh i-lt i-len
unfolding composition-fri-root-path-fresh-from-def by blast
ultimately show fmlookup (HashMap  $s$ ) ?key = None
using lookup[of PCompositionFriCounter s-comp + i
  foldl concat (PState s-comp) (take (Suc i) composition-roots)]
by simp
qed

lemma receive-trace-fri-commits-preserves-composition-fri-lookup:
assumes outcome:
  Some (br, t)  $\in \text{set-dist } (\text{execute } \text{receive-trace-fri-commits } s)$ 
shows
  fmlookup (HashMap  $t$ ) (CompositionFriChallenge  $i$   $x$ ) =
    fmlookup (HashMap  $s$ ) (CompositionFriChallenge  $i$   $x$ )
proof –
obtain  $b$   $r$  where br-eq:  $br = (b, r)$ 
by (cases br)
from outcome[unfolded br-eq] obtain s-read where
  read-r: Some ( $r$ , s-read)  $\in \text{set-dist } (\text{execute } \text{read } s)$ 
and rand-b:
  Some ( $b$ ,  $t$ )  $\in \text{set-dist } (\text{execute } \text{receive-trace-fri-challenge } s\text{-read})$ 
unfolding receive-trace-fri-commits-def receive-fri-commits-with-def
by (auto elim!: set-dist-bindE)
have read-hash: HashMap  $s\text{-read} = \text{HashMap } s$ 
by (rule read-preserves-hash-map[OF read-r])
have neg:
  CompositionFriChallenge  $i$   $x \neq$ 
    TraceFriChallenge (PTraceFriCounter  $s\text{-read}$ ) (PState  $s\text{-read}$ )

```

by *simp*
 have *fmlookup* (*HashMap* *t*) (*CompositionFriChallenge* *i* *x*) =
 fmlookup (*HashMap* *s-read*) (*CompositionFriChallenge* *i* *x*)
 by (rule *receive-trace-fri-challenge-preserves-other-lookup*
 [*OF rand-b neq*])
 also have ... = *fmlookup* (*HashMap* *s*) (*CompositionFriChallenge* *i* *x*)
 using *read-hash* by *simp*
 finally show ?thesis .
 qed

lemma *ntimes-receive-trace-fri-commits-preserves-composition-fri-lookup*:

assumes *outcome*:

Some (*brs*, *t*) ∈
 set-dist (*execute* (*ntimes receive-trace-fri-commits* *n*) *s*)

shows

fmlookup (*HashMap* *t*) (*CompositionFriChallenge* *i* *x*) =
 fmlookup (*HashMap* *s*) (*CompositionFriChallenge* *i* *x*)

using *outcome*

proof (*induction* *n* *arbitrary*: *s* *brs* *t*)

case 0

then show ?*case*

 by *simp*

next

case (*Suc* *n*)

from *Suc.prem*s **obtain** *br* *brs'* *s1* **where**

head: *Some* (*br*, *s1*) ∈ *set-dist* (*execute receive-trace-fri-commits* *s*)

and *tail*:

Some (*brs'*, *t*) ∈
 set-dist (*execute* (*ntimes receive-trace-fri-commits* *n*) *s1*)

 by (*auto elim!*: *set-dist-bindE*)

have *fmlookup* (*HashMap* *t*) (*CompositionFriChallenge* *i* *x*) =
 fmlookup (*HashMap* *s1*) (*CompositionFriChallenge* *i* *x*)

 by (rule *Suc.IH*[*OF tail*])

also have ... = *fmlookup* (*HashMap* *s*) (*CompositionFriChallenge* *i* *x*)

 by (rule *receive-trace-fri-commits-preserves-composition-fri-lookup*
 [*OF head*])

finally show ?*case* .

qed

lemma *alpha-round-preserves-composition-fri-lookup*:

assumes *outcome*: *Some* (*a*, *t*) ∈ *set-dist* (*execute alpha-round* *s*)

shows

fmlookup (*HashMap* *t*) (*CompositionFriChallenge* *i* *x*) =
 fmlookup (*HashMap* *s*) (*CompositionFriChallenge* *i* *x*)

proof –

from *outcome* **obtain** *a0* *s0* *a1* *s1* *s2* **where**

rand: *Some* (*a0*, *s0*) ∈ *set-dist* (*execute receive-alpha-challenge* *s*)

and *read-a*: *Some* (*a1*, *s1*) ∈ *set-dist* (*execute read* *s0*)

and *assert-a*: *Some* (*()*, *s2*) ∈ *set-dist* (*execute (assert (a0 = a1))* *s1*)

and *ret-a*: *Some* (*a*, *t*) ∈ *set-dist* (*execute* (*return a1*) *s2*)
unfolding *alpha-round-def*
by (*auto simp*: *Let-def elim*!: *set-dist-bindE*)
have *neq*:
 CompositionFriChallenge i x ≠
 AlphaChallenge (*PAlphaCounter s*) (*PState s*)
by *simp*
have *fmlookup* (*HashMap s0*) (*CompositionFriChallenge i x*) =
 fmlookup (*HashMap s*) (*CompositionFriChallenge i x*)
by (*rule receive-alpha-challenge-preserves-other-lookup*[*OF rand neq*])
moreover have *HashMap s1* = *HashMap s0*
by (*rule read-preserves-hash-map*[*OF read-a*])
moreover have *s2* = *s1*
using *assert-a* **unfolding** *assert-def*
by (*cases a0 = a1*) (*auto simp*: *throw-no-outcome*)
moreover have *t* = *s2*
using *ret-a* **by** *simp*
ultimately show *?thesis*
by *simp*
qed

lemma *mmap-alpha-round-preserves-composition-fri-lookup*:
assumes *outcome*:
 Some (*as*, *t*) ∈
 set-dist (*execute* (*mmap* (*replicate n alpha-round*)) *s*)
shows
 fmlookup (*HashMap t*) (*CompositionFriChallenge i x*) =
 fmlookup (*HashMap s*) (*CompositionFriChallenge i x*)
using *outcome*
proof (*induction n arbitrary: s as t*)
case 0
then show *?case*
by *simp*
next
case (*Suc n*)
from *Suc.prem*s **obtain** *a as' s1* **where**
 head: *Some* (*a*, *s1*) ∈ *set-dist* (*execute alpha-round s*)
and *tail*:
 Some (*as'*, *t*) ∈
 set-dist (*execute* (*mmap* (*replicate n alpha-round*)) *s1*)
by (*auto elim*!: *set-dist-bindE*)
have *fmlookup* (*HashMap t*) (*CompositionFriChallenge i x*) =
 fmlookup (*HashMap s1*) (*CompositionFriChallenge i x*)
by (*rule Suc.IH*[*OF tail*])
also have ... = *fmlookup* (*HashMap s*) (*CompositionFriChallenge i x*)
by (*rule alpha-round-preserves-composition-fri-lookup*[*OF head*])
finally show *?case* .
qed

lemma *wp-verifier-composition-fri-prefix-challenge-list-actual-fresh-bound:*

fixes $C :: \text{prob}$

assumes *subset:*

$\bigwedge dg. B \ dg \subseteq \text{fri-challenge-space } (\text{ceil-log } (\text{to-nat } dg + 1))$

and *bound:*

$\bigwedge dg. \text{nnreal } (\text{card } (B \ dg)) /$
 $\text{nnreal } (\text{CARD}('f) \wedge \text{ceil-log } (\text{to-nat } dg + 1)) \leq C$

shows

wp-event verifier-composition-fri-prefix

$(\lambda \text{out. case out of}$

$\text{None} \Rightarrow \text{False}$

$| \text{Some } ((fr, f-fl, f-final, as, dg, fl), -) \Rightarrow$

$\text{map fst fl} \in B \ dg \wedge$

$\text{composition-fri-challenge-path-fresh } s \ fr \ (\text{map snd f-fl})$

$f-final \ as \ dg \ (\text{map snd fl}) \ (\text{length } (\text{map snd fl})) \ s$

$\leq C$

proof $-$

let $?Q =$

$\lambda \text{out. case out of}$

$\text{None} \Rightarrow \text{False}$

$| \text{Some } ((fr, f-fl, f-final, as, dg, fl), -) \Rightarrow$

$\text{map fst fl} \in B \ dg \wedge$

$\text{composition-fri-challenge-path-fresh } s \ fr \ (\text{map snd f-fl})$

$f-final \ as \ dg \ (\text{map snd fl}) \ (\text{length } (\text{map snd fl}))$

show *?thesis*

unfolding *verifier-composition-fri-prefix-def*

proof *(rule wp-event-bind-bound-by-cont)*

show $\neg ?Q \ \text{None}$

by *simp*

next

fix $fr \ s1$

assume *read-fr: Some (fr, s1) ∈ set-dist (execute read s)*

from *read-outcome[OF read-fr]* **have** *st-s1:*

$PState \ s1 = \text{concat } (PState \ s) \ fr$

by *blast*

from *read-outcome[OF read-fr]* **have** *comp-count-s1:*

$PCompositionFriCounter \ s1 = PCompositionFriCounter \ s$

by *blast*

have *hash-s1: HashMap s1 = HashMap s*

by *(rule read-preserves-hash-map[OF read-fr])*

show *wp-event*

$(\text{ntimes receive-trace-fri-commits } (\text{ceil-log } \text{clength}) \gg=$

$(\lambda f-fl. \text{read } \gg=$

$(\lambda f-final.$

$\text{mmap } (\text{replicate } (\text{length spec}) \ \text{alpha-round}) \gg=$

$(\lambda as. \text{read } \gg=$

$(\lambda dg.$

$\text{assert } (\text{to-nat } dg \leq \text{maxDegree}) \gg=$

$(\lambda -. \text{ntimes receive-composition-fri-commits}$

```

      (ceil-log (to-nat dg + 1)) >>=
      (λfl.
        return (fr, f-fl, f-final, as, dg, fl))))))
    ?Q s1 ≤ C
  proof (rule wp-event-bind-bound-by-cont)
    show ¬ ?Q None
      by simp
  next
    fix f-fl s2
    assume trace-fri:
      Some (f-fl, s2) ∈
        set-dist
        (execute (ntimes receive-trace-fri-commits (ceil-log clength)) s1)
    have trace-res:
      PState s2 = foldl concat (PState s1) (map snd f-fl) ∧
      PCompositionFriCounter s2 = PCompositionFriCounter s1
    using ntimes-receive-trace-fri-commits-outcome[OF trace-fri]
    by simp
    have lookup-s2:
      ∧ i x. fmlookup (HashMap s2) (CompositionFriChallenge i x) =
        fmlookup (HashMap s) (CompositionFriChallenge i x)
    proof -
      fix i x
      have fmlookup (HashMap s2) (CompositionFriChallenge i x) =
        fmlookup (HashMap s1) (CompositionFriChallenge i x)
      by (rule ntimes-receive-trace-fri-commits-preserves-composition-fri-lookup
        [OF trace-fri])
      also have ... = fmlookup (HashMap s) (CompositionFriChallenge i x)
      using hash-s1 by simp
      finally show
        fmlookup (HashMap s2) (CompositionFriChallenge i x) =
          fmlookup (HashMap s) (CompositionFriChallenge i x) .
    qed
    show wp-event
      (read >>=
        (λf-final.
          mmap (replicate (length spec) alpha-round) >>=
            (λas. read >>=
              (λdg.
                assert (to-nat dg ≤ maxDegree) >>=
                (λ-. ntimes receive-composition-fri-commits
                  (ceil-log (to-nat dg + 1)) >>=
                  (λfl.
                    return (fr, f-fl, f-final, as, dg, fl))))))
            ?Q s2 ≤ C
        )
    proof (rule wp-event-bind-bound-by-cont)
      show ¬ ?Q None
        by simp
    next

```

```

fix f-final s3
assume read-final: Some (f-final, s3) ∈ set-dist (execute read s2)
from read-outcome[OF read-final] have st-s3:
  PState s3 = concat (PState s2) f-final
  by blast
from read-outcome[OF read-final] have comp-count-s3:
  PCompositionFriCounter s3 = PCompositionFriCounter s2
  by blast
have hash-s3: HashMap s3 = HashMap s2
  by (rule read-preserves-hash-map[OF read-final])
have lookup-s3:
   $\bigwedge i x. \text{fmlookup } (\text{HashMap } s3) (\text{CompositionFriChallenge } i x) =$ 
     $\text{fmlookup } (\text{HashMap } s) (\text{CompositionFriChallenge } i x)$ 
  using hash-s3 lookup-s2 by simp
show wp-event
  (mmap (replicate (length spec) alpha-round) >>=
    (λ as. read >>=
      (λ dg.
        assert (to-nat dg ≤ maxDegree) >>=
          (λ-. ntimes receive-composition-fri-commits
            (ceil-log (to-nat dg + 1)) >>=
              (λ fl.
                return (fr, f-fl, f-final, as, dg, fl)))))))
  ?Q s3 ≤ C
proof (rule wp-event-bind-bound-by-cont)
  show  $\neg ?Q \text{ None}$ 
  by simp
next
  fix as s4
  assume alpha:
    Some (as, s4) ∈
      set-dist
        (execute (mmap (replicate (length spec) alpha-round)) s3)
  have alpha-res:
    PState s4 = foldl concat (PState s3) as ∧
    PCompositionFriCounter s4 = PCompositionFriCounter s3
    using mmap-alpha-round-outcome[OF alpha] by simp
  have lookup-s4:
     $\bigwedge i x. \text{fmlookup } (\text{HashMap } s4) (\text{CompositionFriChallenge } i x) =$ 
       $\text{fmlookup } (\text{HashMap } s) (\text{CompositionFriChallenge } i x)$ 
  proof –
    fix i x
    have  $\text{fmlookup } (\text{HashMap } s4) (\text{CompositionFriChallenge } i x) =$ 
       $\text{fmlookup } (\text{HashMap } s3) (\text{CompositionFriChallenge } i x)$ 
    by (rule mmap-alpha-round-preserves-composition-fri-lookup
      [OF alpha])
    also have ... =
       $\text{fmlookup } (\text{HashMap } s) (\text{CompositionFriChallenge } i x)$ 
    by (rule lookup-s3)

```

```

finally show
  fmllookup (HashMap s4) (CompositionFriChallenge i x) =
    fmllookup (HashMap s) (CompositionFriChallenge i x) .
qed
show wp-event
  (read  $\gg$ 
    ( $\lambda dg.$ 
      assert (to-nat dg  $\leq$  maxDegree)  $\gg$ 
      ( $\lambda-. ntimes receive-composition-fri-commits$ 
        (ceil-log (to-nat dg + 1))  $\gg$ 
        ( $\lambda fl.$ 
          return (fr, f-fl, f-final, as, dg, fl))))))
  ?Q s4  $\leq$  C
proof (rule wp-event-bind-bound-by-cont)
  show  $\neg$  ?Q None
  by simp
next
  fix dg s5
  assume read-dg: Some (dg, s5)  $\in$  set-dist (execute read s4)
  from read-outcome[OF read-dg] have st-s5:
    PState s5 = concat (PState s4) dg
  by blast
  from read-outcome[OF read-dg] have comp-count-s5:
    PCompositionFriCounter s5 = PCompositionFriCounter s4
  by blast
  have hash-s5: HashMap s5 = HashMap s4
  by (rule read-preserves-hash-map[OF read-dg])
  have lookup-s5:
     $\bigwedge i x. fmllookup (HashMap s5) (CompositionFriChallenge i x) =$ 
      fmllookup (HashMap s) (CompositionFriChallenge i x)
  using hash-s5 lookup-s4 by simp
  show wp-event
    (assert (to-nat dg  $\leq$  maxDegree)  $\gg$ 
      ( $\lambda-. ntimes receive-composition-fri-commits$ 
        (ceil-log (to-nat dg + 1))  $\gg$ 
        ( $\lambda fl.$ 
          return (fr, f-fl, f-final, as, dg, fl))))))
    ?Q s5  $\leq$  C
  proof (rule wp-event-bind-bound-by-cont)
    show  $\neg$  ?Q None
    by simp
  next
    fix u s6
    assume assert-out:
      Some (u, s6)  $\in$ 
        set-dist (execute (assert (to-nat dg  $\leq$  maxDegree))) s5)
    have s6-eq: s6 = s5
    using assert-out unfolding assert-def
    by (cases to-nat dg  $\leq$  maxDegree)

```

```

(auto simp: throw-no-outcome)
have state-s6:
  PState s6 =
    concat
      (foldl concat
        (concat
          (foldl concat (concat (PState s) fr)
            (map snd f-fl))
          f-final)
        as)
      dg
  using st-s1 trace-res st-s3 alpha-res st-s5 s6-eq
  by simp
have counter-s6:
  PCompositionFriCounter s6 = PCompositionFriCounter s
  using comp-count-s1 trace-res comp-count-s3 alpha-res
    comp-count-s5 s6-eq
  by simp
have lookup-s6:
   $\bigwedge i x. \text{fmlookup } (\text{HashMap } s6) \text{ (CompositionFriChallenge } i \text{ } x) =$ 
    fmlookup (HashMap s) (CompositionFriChallenge i x)
  using lookup-s5 s6-eq by simp
let ?Head =
   $\lambda out. \text{case out of}$ 
    None  $\Rightarrow$  False
  | Some (fl, -)  $\Rightarrow$ 
    map fst fl  $\in B$  dg  $\wedge$ 
    composition-fri-root-path-fresh-from s6 (map snd fl)
    (ceil-log (to-nat dg + 1))
have comp-bound:
  wp-event
    (ntimes receive-composition-fri-commits
      (ceil-log (to-nat dg + 1)))
  ?Head s6  $\leq$ 
  nnreal (card (B dg)) /
  nnreal (CARD('f)  $\wedge$  ceil-log (to-nat dg + 1))
  by (rule
    wp-ntimes-receive-composition-fri-commits-challenge-space-actual-fresh-bound
    [OF subset])
have comp-bound-C:
  wp-event
    (ntimes receive-composition-fri-commits
      (ceil-log (to-nat dg + 1)))
  ?Head s6  $\leq C$ 
  by (rule order-trans[OF comp-bound bound])
show wp-event
  (ntimes receive-composition-fri-commits
    (ceil-log (to-nat dg + 1)))  $\gg=$ 

```



```

      (λfl.
        return (fr, f-fl, f-final, as, dg, fl)))
    ?Q s6 ≤ C
proof (rule wp-event-bind-bound-by-head-event[OF comp-bound-C])
  show ?Q None ⇒ ?Head None
  by simp
next
  fix fl t out
  assume comp-out:
    Some (fl, t) ∈
      set-dist
        (execute
          (ntimes receive-composition-fri-commits
            (ceil-log (to-nat dg + 1))) s6)
  and cont:
    out ∈
      set-dist
        (execute
          (return (fr, f-fl, f-final, as, dg, fl)) t)
  and hit: ?Q out
have out-eq:
  out = Some ((fr, f-fl, f-final, as, dg, fl), t)
  using cont
unfolding set-dist-def return.rep-eq dist-return-def
  dist-delta-dist delta-map-def
  by simp
have len-fl: length fl = ceil-log (to-nat dg + 1)
  using ntimes-receive-composition-fri-commits-outcome
  [OF comp-out] by simp
have path-fresh-len:
  composition-fri-challenge-path-fresh s fr
    (map snd f-fl) f-final as dg (map snd fl)
    (length fl)
  using hit out-eq by simp
have root-fresh-head:
  composition-fri-root-path-fresh-from s6 (map snd fl)
    (ceil-log (to-nat dg + 1))
  using path-fresh-len len-fl state-s6 counter-s6 lookup-s6
unfolding composition-fri-challenge-path-fresh-def
  composition-fri-root-path-fresh-from-def
  composition-fri-challenge-key-at-def
  by simp
have root-fresh:
  composition-fri-root-path-fresh-from s6 (map snd fl)
    (ceil-log (Suc (to-nat dg)))
  using root-fresh-head by simp
have path-fresh:
  composition-fri-challenge-path-fresh s fr (map snd f-fl)
    f-final as dg (map snd fl) (length (map snd fl))

```

```

      using composition-fri-challenge-path-fresh-from-preservedI
      [OF state-s6 counter-s6 lookup-s6 root-fresh]
      len-fl
    by simp
  show ?Head (Some (fl, t))
    using hit out-eq root-fresh-head by simp
qed
qed
qed
qed
qed
qed
qed
qed
qed

lemma wp-verify-monad-composition-fri-challenge-list-fresh-hit-actual-bound:
  fixes C :: prob
  assumes subset:
     $\bigwedge dg. B \ dg \subseteq \text{fri-challenge-space } (\text{ceil-log } (to\text{-nat } dg + 1))$ 
  and bound:
     $\bigwedge dg. \text{nnreal } (\text{card } (B \ dg)) / \text{nnreal } (\text{CARD}(f) \wedge \text{ceil-log } (to\text{-nat } dg + 1)) \leq C$ 
  shows
    wp-event verify-monad (composition-fri-challenge-list-fresh-hit s B) s
       $\leq C$ 
proof -
  let ?Head =
     $\lambda out. \text{case out of}$ 
     $\text{None} \Rightarrow \text{False}$ 
     $| \text{Some } ((fr, f\text{-fl}, f\text{-final}, as, dg, fl), -) \Rightarrow$ 
     $\text{map fst } fl \in B \ dg \wedge$ 
     $\text{composition-fri-challenge-path-fresh } s \ fr \ (\text{map snd } f\text{-fl})$ 
     $f\text{-final } as \ dg \ (\text{map snd } fl) \ (\text{length } (\text{map snd } fl))$ 
  have prefix-bound:
    wp-event verifier-composition-fri-prefix ?Head s  $\leq C$ 
  by (rule
    wp-verifier-composition-fri-prefix-challenge-list-actual-fresh-bound
    [OF subset bound])
  show ?thesis
    unfolding verify-monad-composition-fri-decomposition
  proof (rule wp-event-bind-bound-by-head-event[OF prefix-bound])
    show composition-fri-challenge-list-fresh-hit s B None  $\implies$ 
      ?Head None
    unfolding composition-fri-challenge-list-fresh-hit-def
      accepted-fri-challenges-def by blast
  next
    fix header prefix-state out
    assume prefix:
      Some (header, prefix-state)  $\in$ 

```

```

    set-dist (execute verifier-composition-fri-prefix s)
and suffix:
    out ∈
      set-dist (execute
        (verifier-after-composition-fri header) prefix-state)
    and hit: composition-fri-challenge-list-fresh-hit s B out
obtain fr f-fl f-final as dg fl where header-eq:
    header = (fr, f-fl, f-final, as, dg, fl)
    by (cases header) auto
from hit obtain trace-bs dg' comp-bs fr' trace-roots' trace-final'
    as' composition-roots' final' rest' where
    challenges-hit: accepted-fri-challenges s out trace-bs dg' comp-bs
    and header-hit:
      verifier-header-transcript s fr' trace-roots' trace-final' as' dg'
      composition-roots' final' rest'
    and comp-bs-B: comp-bs ∈ B dg'
    and fresh-hit:
      composition-fri-challenge-path-fresh s fr' trace-roots'
      trace-final' as' dg' composition-roots'
      (length composition-roots')
    unfolding composition-fri-challenge-list-fresh-hit-def by blast
from challenges-hit obtain result final-state where out-eq:
    out = Some (result, final-state)
    unfolding accepted-fri-challenges-def by blast
have prefix':
    Some ((fr, f-fl, f-final, as, dg, fl), prefix-state) ∈
      set-dist (execute verifier-composition-fri-prefix s)
    using prefix header-eq by simp
have suffix':
    Some (result, final-state) ∈
      set-dist
        (execute
          (verifier-after-composition-fri
            (fr, f-fl, f-final, as, dg, fl))
          prefix-state)
    using suffix header-eq out-eq by simp
from suffix' obtain final query-state where
    read-final:
      Some (final, query-state) ∈ set-dist (execute read prefix-state)
    and query-out:
      Some (result, final-state) ∈
        set-dist
          (execute
            (ntimes
              (verifier-query-round-program fr f-fl f-final as fl final)
              rounds)
            query-state)
    unfolding verifier-after-composition-fri-def
    by (auto elim!: set-dist-bindE)

```

```

have prefix-res:
  length f-fl = ceil-log clength ∧
  length as = length spec ∧
  length fl = ceil-log (to-nat dg + 1) ∧
  PTranscript s =
    [fr] @ map snd f-fl @ [f-final] @ as @ [dg] @
    map snd fl @ PTranscript prefix-state
  using verifier-composition-fri-prefix-outcome[OF prefix'] by simp
from read-outcome[OF read-final] obtain rest-query where
  tr-prefix: PTranscript prefix-state = final # rest-query
  and tr-query: PTranscript query-state = rest-query
  by blast
have header-prefix:
  verifier-header-transcript s fr (map snd f-fl) f-final as dg
    (map snd fl) final (PTranscript query-state)
  using prefix-res tr-prefix tr-query
  unfolding verifier-header-transcript-def verifier-header-messages-def
  by simp
have challenges-prefix:
  accepted-fri-challenges s (Some (result, final-state))
    (map fst f-fl) dg (map fst fl)
  by (rule verifier-after-composition-fri-accepted-fri-challenges
    [OF prefix' suffix'])
have eqs:
  trace-bs = map fst f-fl ∧ dg' = dg ∧ comp-bs = map fst fl
  using accepted-fri-challenges-unique
    [OF challenges-prefix challenges-hit[unfolded out-eq]]
  by simp
have header-eqs:
  fr' = fr ∧
  trace-roots' = map snd f-fl ∧
  trace-final' = f-final ∧
  as' = as ∧
  composition-roots' = map snd fl
  using verifier-header-transcript-unique[OF header-prefix header-hit]
  eqs
  by simp
then have fresh-prefix:
  composition-fri-challenge-path-fresh s fr (map snd f-fl)
    f-final as dg (map snd fl) (length (map snd fl))
  using fresh-hit eqs by simp
show ?Head (Some (header, prefix-state))
  using comp-bs-B eqs fresh-prefix header-eq by simp
qed
qed

lemma trace-fri-challenge-list-fresh-hit-imp-set-hit:
  assumes trace-fri-challenge-list-fresh-hit s B out
  shows trace-fri-challenge-list-set-hit s B out

```

using *assms*
unfolding *trace-fri-challenge-list-fresh-hit-def*
trace-fri-challenge-list-set-hit-def
by *blast*

lemma *composition-fri-challenge-list-fresh-hit-imp-set-hit:*
assumes *composition-fri-challenge-list-fresh-hit s B out*
shows *composition-fri-challenge-list-set-hit s B out*
using *assms*
unfolding *composition-fri-challenge-list-fresh-hit-def*
composition-fri-challenge-list-set-hit-def
by *blast*

lemma *wp-trace-fri-challenge-list-fresh-hit-bound-from-future-fresh:*
assumes *future: trace-fri-future-fresh s*
and *subset: B ⊆ fri-challenge-space (ceil-log clength)*
shows
wp-event verify-monad (trace-fri-challenge-list-fresh-hit s B) s ≤
nnreal (card B) / nnreal (CARD('f) ^ ceil-log clength)

proof –
have *wp-event verify-monad (trace-fri-challenge-list-fresh-hit s B) s*
≤ wp-event verify-monad (trace-fri-challenge-list-set-hit s B) s
by (*rule wp-event-mono*)
(*rule trace-fri-challenge-list-fresh-hit-imp-set-hit*)
also have ... *≤*
nnreal (card B) / nnreal (CARD('f) ^ ceil-log clength)
by (*rule wp-verify-monad-trace-fri-challenge-list-set-bound*
[*OF future subset*])
finally show *?thesis* .

qed

lemma *wp-composition-fri-challenge-list-fresh-hit-bound-from-future-fresh:*
assumes *future: composition-fri-future-fresh s*
and *subset:*
∧ dg. B dg ⊆ fri-challenge-space (ceil-log (to-nat dg + 1))
and *bound:*
∧ dg. nnreal (card (B dg)) /
nnreal (CARD('f) ^ ceil-log (to-nat dg + 1)) ≤ C

shows
wp-event verify-monad (composition-fri-challenge-list-fresh-hit s B) s
≤ C

proof –
have *wp-event verify-monad (composition-fri-challenge-list-fresh-hit s B) s*
≤ wp-event verify-monad (composition-fri-challenge-list-set-hit s B) s
by (*rule wp-event-mono*)
(*rule composition-fri-challenge-list-fresh-hit-imp-set-hit*)
also have ... *≤ C*
by (*rule wp-verify-monad-composition-fri-challenge-list-set-bound*
[*OF future subset bound*])

finally show ?thesis .
qed

lemma *accepted-fri-challenges-headerE*:

assumes *accepted-fri-challenges s out trace-bs dg comp-bs*

obtains *fr f-fl trace-final alphas fl final rest* **where**

verifier-header-transcript s fr (map snd f-fl) trace-final alphas dg

(map snd fl) final rest

trace-bs = map fst f-fl

comp-bs = map fst fl

proof –

from *assms* **obtain** *result final-state fr f-fl trace-final alphas fl final*

query-state **where** *body*:

out = Some (result, final-state) ∧

verifier-header-transcript s fr (map snd f-fl) trace-final alphas dg

(map snd fl) final (PTranscript query-state) ∧

PState query-state =

verifier-header-state s fr (map snd f-fl) trace-final alphas dg

(map snd fl) final ∧

Some (result, final-state) ∈

set-dist (execute

(ntimes (verifier-query-round-program fr f-fl trace-final alphas fl final)

rounds)

query-state) ∧

PQueryCounter query-state = PQueryCounter s ∧

trace-bs = map fst f-fl ∧

comp-bs = map fst fl ∧

(∀ i < length f-fl.

fmlookup (HashMap query-state)

(TraceFriChallenge (PTraceFriCounter s + i)

(foldl concat (concat (PState s) fr)

(take (Suc i) (map snd f-fl)))) =

Some (fst (f-fl ! i)) ∧

(∀ i < length fl.

fmlookup (HashMap query-state)

(CompositionFriChallenge (PCompositionFriCounter s + i)

(foldl concat

(concat

(foldl concat

(concat

(foldl concat (concat (PState s) fr) (map snd f-fl))

trace-final)

alphas)

dg)

(take (Suc i) (map snd fl)))) =

Some (fst (fl ! i))

unfolding *accepted-fri-challenges-def* **by** *blast*

have *header*:

verifier-header-transcript s fr (map snd f-fl) trace-final alphas dg

```

    (map snd fl) final (PTranscript query-state)
  using body by simp
have trace-eq: trace-bs = map fst f-fl
  using body by simp
have comp-eq: comp-bs = map fst fl
  using body by simp
show thesis
  by (rule that[of fr f-fl trace-final alphas fl final
    PTranscript query-state,
    OF header trace-eq comp-eq])
qed

lemma trace-fri-challenge-path-fresh-or-prequeried:
  trace-fri-challenge-path-fresh s fr roots n  $\vee$ 
  trace-fri-challenge-path-prequeried s fr roots n
proof (cases  $\exists i < n. i < \text{length roots} \wedge$ 
  fmlookup (HashMap s) (trace-fri-challenge-key-at s fr roots i)  $\neq$ 
  None)
case True
then have trace-fri-challenge-path-prequeried s fr roots n
  unfolding trace-fri-challenge-path-prequeried-def .
then show ?thesis by simp
next
case False
then have trace-fri-challenge-path-fresh s fr roots n
  unfolding trace-fri-challenge-path-fresh-def
  by auto
then show ?thesis by simp
qed

lemma composition-fri-challenge-path-fresh-or-prequeried:
  composition-fri-challenge-path-fresh s fr trace-roots trace-final as dg
  composition-roots n  $\vee$ 
  composition-fri-challenge-path-prequeried s fr trace-roots trace-final as
  dg composition-roots n
proof (cases  $\exists i < n. i < \text{length composition-roots} \wedge$ 
  fmlookup (HashMap s)
  (composition-fri-challenge-key-at s fr trace-roots trace-final as dg
  composition-roots i)  $\neq$  None)
case True
then have composition-fri-challenge-path-prequeried s fr trace-roots
  trace-final as dg composition-roots n
  unfolding composition-fri-challenge-path-prequeried-def .
then show ?thesis by simp
next
case False
then have composition-fri-challenge-path-fresh s fr trace-roots
  trace-final as dg composition-roots n
  unfolding composition-fri-challenge-path-fresh-def

```

```

    by auto
  then show ?thesis by simp
qed

lemma trace-fri-challenge-list-set-hit-imp-fresh-or-prequery:
  assumes trace-fri-challenge-list-set-hit s B out
  shows
    trace-fri-challenge-list-fresh-hit s B out  $\vee$ 
    trace-fri-challenge-list-prequery-hit s B out
proof -
  from assms obtain trace-bs dg comp-bs where hit:
    accepted-fri-challenges s out trace-bs dg comp-bs
    trace-bs  $\in$  B
  unfolding trace-fri-challenge-list-set-hit-def by auto
  from hit(1) obtain fr f-fl trace-final alphas fl final rest where
    header:
      verifier-header-transcript s fr (map snd f-fl) trace-final alphas dg
      (map snd fl) final rest
    trace-bs = map fst f-fl
    comp-bs = map fst fl
  by (rule accepted-fri-challenges-headerE)
  have split: trace-fri-challenge-path-fresh s fr (map snd f-fl)
    (length (map snd f-fl))  $\vee$ 
    trace-fri-challenge-path-prequeried s fr (map snd f-fl)
    (length (map snd f-fl))
  by (rule trace-fri-challenge-path-fresh-or-prequeried)
  then show ?thesis
proof
  assume fresh: trace-fri-challenge-path-fresh s fr (map snd f-fl)
    (length (map snd f-fl))
  have trace-fri-challenge-list-fresh-hit s B out
  unfolding trace-fri-challenge-list-fresh-hit-def
  by (intro exI conjI; (rule hit(1) | rule header | rule hit(2) |
    rule fresh))
  then show ?thesis by simp
next
  assume prequeried: trace-fri-challenge-path-prequeried s fr
    (map snd f-fl) (length (map snd f-fl))
  have trace-fri-challenge-list-prequery-hit s B out
  unfolding trace-fri-challenge-list-prequery-hit-def
  by (intro exI conjI; (rule hit(1) | rule header | rule hit(2) |
    rule prequeried))
  then show ?thesis by simp
qed
qed

lemma composition-fri-challenge-list-set-hit-imp-fresh-or-prequery:
  assumes composition-fri-challenge-list-set-hit s B out
  shows

```


composition-fri-challenge-list-fresh-hit s B out $\vee$
composition-fri-challenge-list-prequery-hit s B out
proof –
from *assms* **obtain** *trace-bs dg comp-bs* **where** *hit*:
accepted-fri-challenges s out trace-bs dg comp-bs
comp-bs $\in B$ *dg*
unfolding *composition-fri-challenge-list-set-hit-def* **by** *auto*
from *hit(1)* **obtain** *fr f-fl trace-final alphas fl final rest* **where**
header:
verifier-header-transcript s fr (map snd f-fl) trace-final alphas dg
(map snd fl) final rest
trace-bs = map fst f-fl
comp-bs = map fst fl
by (*rule accepted-fri-challenges-headerE*)
have *split: composition-fri-challenge-path-fresh s fr (map snd f-fl)*
trace-final alphas dg (map snd fl) (length (map snd fl)) $\vee$
composition-fri-challenge-path-prequeried s fr (map snd f-fl)
trace-final alphas dg (map snd fl) (length (map snd fl))
by (*rule composition-fri-challenge-path-fresh-or-prequeried*)
then show *?thesis*
proof
assume *fresh: composition-fri-challenge-path-fresh s fr (map snd f-fl)*
trace-final alphas dg (map snd fl) (length (map snd fl))
have *composition-fri-challenge-list-fresh-hit s B out*
unfolding *composition-fri-challenge-list-fresh-hit-def*
by (*intro exI conjI; (rule hit(1) | rule header | rule hit(2) |*
rule fresh))
then show *?thesis* **by** *simp*
next
assume *prequeried: composition-fri-challenge-path-prequeried s fr*
(map snd f-fl) trace-final alphas dg (map snd fl) (length (map snd fl))
have *composition-fri-challenge-list-prequery-hit s B out*
unfolding *composition-fri-challenge-list-prequery-hit-def*
by (*intro exI conjI; (rule hit(1) | rule header | rule hit(2) |*
rule prequeried))
then show *?thesis* **by** *simp*
qed
qed
lemma *wp-trace-fri-challenge-list-set-hit-bound-from-fresh-and-prequery*:
assumes *fresh-bound*:
wp-event verify-monad (trace-fri-challenge-list-fresh-hit s B) s $\leq F$
and *prequery-bound*:
wp-event verify-monad (trace-fri-challenge-list-prequery-hit s B) s
 $\leq P$
shows *wp-event verify-monad (trace-fri-challenge-list-set-hit s B) s*
 $\leq F + P$
proof –
have *wp-event verify-monad (trace-fri-challenge-list-set-hit s B) s* $\leq$

$wp\text{-event } verify\text{-monad}$
 $(\lambda out. trace\text{-fri-challenge-list-fresh-hit } s \ B \ out \vee$
 $trace\text{-fri-challenge-list-prequery-hit } s \ B \ out) \ s$
by (rule $wp\text{-event-mono}$)
(rule $trace\text{-fri-challenge-list-set-hit-imp-fresh-or-prequery}$)
also have $\dots \leq$
 $wp\text{-event } verify\text{-monad } (trace\text{-fri-challenge-list-fresh-hit } s \ B) \ s +$
 $wp\text{-event } verify\text{-monad } (trace\text{-fri-challenge-list-prequery-hit } s \ B) \ s$
by (rule $wp\text{-event-union-bound}$)
also have $\dots \leq F + P$
by (rule $add\text{-mono}[OF \ fresh\text{-bound } prequery\text{-bound}]$)
finally show $?thesis$.
qed

lemma $wp\text{-composition-fri-challenge-list-set-hit-bound-from-fresh-and-prequery}$:
assumes $fresh\text{-bound}$:
 $wp\text{-event } verify\text{-monad } (composition\text{-fri-challenge-list-fresh-hit } s \ B) \ s$
 $\leq F$
and $prequery\text{-bound}$:
 $wp\text{-event } verify\text{-monad } (composition\text{-fri-challenge-list-prequery-hit } s \ B) \ s$
 $\leq P$
shows $wp\text{-event } verify\text{-monad } (composition\text{-fri-challenge-list-set-hit } s \ B) \ s$
 $\leq F + P$
proof –
have $wp\text{-event } verify\text{-monad } (composition\text{-fri-challenge-list-set-hit } s \ B) \ s$
 $\leq wp\text{-event } verify\text{-monad}$
 $(\lambda out. composition\text{-fri-challenge-list-fresh-hit } s \ B \ out \vee$
 $composition\text{-fri-challenge-list-prequery-hit } s \ B \ out) \ s$
by (rule $wp\text{-event-mono}$)
(rule $composition\text{-fri-challenge-list-set-hit-imp-fresh-or-prequery}$)
also have $\dots \leq$
 $wp\text{-event } verify\text{-monad } (composition\text{-fri-challenge-list-fresh-hit } s \ B) \ s +$
 $wp\text{-event } verify\text{-monad } (composition\text{-fri-challenge-list-prequery-hit } s \ B) \ s$
by (rule $wp\text{-event-union-bound}$)
also have $\dots \leq F + P$
by (rule $add\text{-mono}[OF \ fresh\text{-bound } prequery\text{-bound}]$)
finally show $?thesis$.
qed

end

context $soundness$
begin

lemma $checked\text{-staged-security-trace-fri-challenge-list-set-hit-bound-from-fresh-and-prequery}$:
assumes $fresh\text{-bound}$:
 $wp\text{-event } (checked\text{-staged-security-experiment-with-data-state } A)$
 $(staged\text{-security-with-data-state-verifier-event}$
 $(\lambda s. trace\text{-fri-challenge-list-fresh-hit } s \ B))$

$adversary-initial-state \leq F$
and *prequery-bound*:
 $wp-event \ (checked-staged-security-experiment-with-data-state \ A)$
 $(staged-security-with-data-state-verifier-event$
 $(\lambda s. trace-fri-challenge-list-prequery-hit \ s \ B))$
 $adversary-initial-state \leq P$
shows
 $wp-event \ (checked-staged-security-experiment-with-data-state \ A)$
 $(staged-security-with-data-state-verifier-event$
 $(\lambda s. trace-fri-challenge-list-set-hit \ s \ B))$
 $adversary-initial-state \leq F + P$
proof –
have $wp-event \ (checked-staged-security-experiment-with-data-state \ A)$
 $(staged-security-with-data-state-verifier-event$
 $(\lambda s. trace-fri-challenge-list-set-hit \ s \ B))$
 $adversary-initial-state \leq$
 $wp-event \ (checked-staged-security-experiment-with-data-state \ A)$
 $(staged-security-with-data-state-verifier-event$
 $(\lambda s \ out.$
 $trace-fri-challenge-list-fresh-hit \ s \ B \ out \ \vee$
 $trace-fri-challenge-list-prequery-hit \ s \ B \ out))$
 $adversary-initial-state$
proof (rule *wp-event-mono-on-support*)
fix *out*
assume
 $out \in$
 $set-dist$
 $(execute \ (checked-staged-security-experiment-with-data-state \ A)$
 $adversary-initial-state)$
and *hit*:
 $staged-security-with-data-state-verifier-event$
 $(\lambda s. trace-fri-challenge-list-set-hit \ s \ B) \ out$
then show
 $staged-security-with-data-state-verifier-event$
 $(\lambda s \ out.$
 $trace-fri-challenge-list-fresh-hit \ s \ B \ out \ \vee$
 $trace-fri-challenge-list-prequery-hit \ s \ B \ out) \ out$
unfolding *staged-security-with-data-state-verifier-event-def*
by (auto dest: *trace-fri-challenge-list-set-hit-imp-fresh-or-prequery*
split: option.splits prod.splits)
qed
also have $\dots \leq$
 $wp-event \ (checked-staged-security-experiment-with-data-state \ A)$
 $(\lambda out.$
 $staged-security-with-data-state-verifier-event$
 $(\lambda s. trace-fri-challenge-list-fresh-hit \ s \ B) \ out \ \vee$
 $staged-security-with-data-state-verifier-event$
 $(\lambda s. trace-fri-challenge-list-prequery-hit \ s \ B) \ out)$
 $adversary-initial-state$

by (rule wp-event-mono-on-support)
 (auto simp: staged-security-with-data-state-verifier-event-def
 split: option.splits prod.splits)
 also have ... $\leq$
 wp-event (checked-staged-security-experiment-with-data-state A)
 (staged-security-with-data-state-verifier-event
 (λs . trace-fri-challenge-list-fresh-hit s B))
 adversary-initial-state +
 wp-event (checked-staged-security-experiment-with-data-state A)
 (staged-security-with-data-state-verifier-event
 (λs . trace-fri-challenge-list-prequery-hit s B))
 adversary-initial-state
 by (rule wp-event-union-bound)
 also have ... $\leq F + P$
 by (rule add-mono[OF fresh-bound prequery-bound])
 finally show ?thesis .
 qed

lemma checked-staged-security-trace-fri-challenge-list-set-hit-bound:
 assumes wf: staged-budget-wellformed budgets
 and controlled: staged-adversary-controlled budgets A
 and finite-B: finite B
 and fresh-bound:
 wp-event (checked-staged-security-experiment-with-data-state A)
 (staged-security-with-data-state-verifier-event
 (λs . trace-fri-challenge-list-fresh-hit s B))
 adversary-initial-state $\leq F$
 shows
 wp-event (checked-staged-security-experiment-with-data-state A)
 (staged-security-with-data-state-verifier-event
 (λs . trace-fri-challenge-list-set-hit s B))
 adversary-initial-state $\leq$
 F +
 hash-relation-budget-value (card B * ceil-log clength)
 (staged-attacker-query-budget budgets + staged-challenge-query-budget)
 by (rule
 checked-staged-security-trace-fri-challenge-list-set-hit-bound-from-fresh-and-prequery)
 (rule fresh-bound,
 rule checked-staged-security-trace-fri-verifier-prequery-hit-bound
 [OF wf controlled finite-B])

lemma checked-staged-security-trace-fri-challenge-list-fresh-hit-actual-bound:
 assumes subset: $B \subseteq \text{fri-challenge-space } (\text{ceil-log clength})$
 shows
 wp-event (checked-staged-security-experiment-with-data-state A)
 (staged-security-with-data-state-verifier-event
 (λs . trace-fri-challenge-list-fresh-hit s B))
 adversary-initial-state $\leq$
 nnreal (card B) / nnreal (CARD('f) $\wedge$ ceil-log clength)

```

proof (rule checked-staged-security-with-data-state-verifier-event-bound-from-cont)
  show  $\bigwedge s. \neg \text{trace-fri-challenge-list-fresh-hit } s \ B \ \text{None}$ 
    unfolding trace-fri-challenge-list-fresh-hit-def
      accepted-fri-challenges-def by blast
next
  fix data attacker-state
  assume Some (data, attacker-state)  $\in$ 
    set-dist
    (execute (checked-staged-transcript-program A)
      adversary-initial-state)
  show wp-event verify-monad
    (trace-fri-challenge-list-fresh-hit
      (verifier-state-from-adversary attacker-state
        (staged-proof-transcript data)) B)
    (verifier-state-from-adversary attacker-state
      (staged-proof-transcript data))
     $\leq \text{nnreal } (\text{card } B) / \text{nnreal } (\text{CARD('f')} \wedge \text{ceil-log } \text{clength})$ 
  by (rule wp-verify-monad-trace-fri-challenge-list-fresh-hit-actual-bound
    [OF subset])
qed

lemma checked-staged-security-trace-fri-bad-challenge-list-fresh-hit-bound-from-round-bounds:
  assumes local-bound:
     $\bigwedge i \text{ prefix.}$ 
     $i < \text{fri-round-count-for-degree-bound } (\text{clength} - 1) \implies$ 
     $\text{length prefix} = i \implies$ 
     $\text{card } (\text{bad } i \text{ prefix}) \leq \text{bd } i$ 
  shows
    wp-event (checked-staged-security-experiment-with-data-state A)
      (staged-security-with-data-state-verifier-event
        ( $\lambda s. \text{trace-fri-challenge-list-fresh-hit } s$ 
          (generic-fri-bad-challenge-lists
            (fri-round-count-for-degree-bound (clength - 1)) bad)))
      adversary-initial-state  $\leq$ 
      nnreal
        ( $\sum i \in \{..<\text{fri-round-count-for-degree-bound } (\text{clength} - 1)\}.$ 
           $\text{CARD('f')} \wedge i * \text{bd } i * \text{CARD('f')} \wedge$ 
            (fri-round-count-for-degree-bound (clength - 1) - Suc i)) /
          nnreal (CARD('f')  $\wedge$  fri-round-count-for-degree-bound (clength - 1)))
  proof -
  let ?n = fri-round-count-for-degree-bound (clength - 1)
  let ?B = generic-fri-bad-challenge-lists ?n bad
  let ?K =
    ( $\sum i \in \{..<?n\}.$ 
       $\text{CARD('f')} \wedge i * \text{bd } i * \text{CARD('f')} \wedge (?n - \text{Suc } i))$ )
  have subset: ?B  $\subseteq$  fri-challenge-space (ceil-log clength)
  using generic-fri-bad-challenge-lists-subset[of ?n bad]
  by (simp add: fri-round-count-for-degree-bound-def clength-pos)

```

```

have base:
  wp-event (checked-staged-security-experiment-with-data-state A)
    (staged-security-with-data-state-verifier-event
      ( $\lambda s.$  trace-fri-challenge-list-fresh-hit s ?B))
  adversary-initial-state  $\leq$ 
    nnreal (card ?B) / nnreal (CARD('f)  $\wedge$  ceil-log clength)
by (rule checked-staged-security-trace-fri-challenge-list-fresh-hit-actual-bound
    [OF subset])
have card-bound: card ?B  $\leq$  ?K
by (rule generic-fri-bad-challenge-lists-card-bound-from-round-bounds)
    (rule local-bound)
have ratio-bound:
  nnreal (card ?B) / nnreal (CARD('f)  $\wedge$  ceil-log clength)
     $\leq$  nnreal ?K / nnreal (CARD('f)  $\wedge$  ?n)
proof -
  have nnreal (card ?B) / nnreal (CARD('f)  $\wedge$  ceil-log clength)
     $\leq$  nnreal ?K / nnreal (CARD('f)  $\wedge$  ceil-log clength)
    using card-bound by (rule nnreal-nat-divide-right-mono)
    also have ... = nnreal ?K / nnreal (CARD('f)  $\wedge$  ?n)
    by (simp add: fri-round-count-for-degree-bound-def clength-pos)
    finally show ?thesis .
qed
show ?thesis
by (rule order-trans[OF base ratio-bound])
qed

lemma checked-staged-security-composition-fri-challenge-list-fresh-hit-actual-bound:
  fixes C :: prob
  assumes subset:
     $\bigwedge dg. B \ dg \subseteq \text{fri-challenge-space } (\text{ceil-log } (\text{to-nat } dg + 1))$ 
  and bound:
     $\bigwedge dg. \text{nnreal } (\text{card } (B \ dg)) / \text{nnreal } (\text{CARD('f) } \wedge \text{ceil-log } (\text{to-nat } dg + 1)) \leq C$ 
  shows
    wp-event (checked-staged-security-experiment-with-data-state A)
      (staged-security-with-data-state-verifier-event
        ( $\lambda s.$  composition-fri-challenge-list-fresh-hit s B))
      adversary-initial-state  $\leq C$ 
proof (rule checked-staged-security-with-data-state-verifier-event-bound-from-cont)
  show  $\bigwedge s. \neg \text{composition-fri-challenge-list-fresh-hit } s \ B \ \text{None}$ 
    unfolding composition-fri-challenge-list-fresh-hit-def
      accepted-fri-challenges-def by blast
next
  fix data attacker-state
  assume Some (data, attacker-state)  $\in$ 
    set-dist
    (execute (checked-staged-transcript-program A)
      adversary-initial-state)
  show wp-event verify-monad

```

```

    (composition-fri-challenge-list-fresh-hit
      (verifier-state-from-adversary attacker-state
        (staged-proof-transcript data)) B)
    (verifier-state-from-adversary attacker-state
      (staged-proof-transcript data))
    ≤ C
  by (rule wp-verify-monad-composition-fri-challenge-list-fresh-hit-actual-bound
      [OF subset bound])
qed

lemma checked-staged-security-composition-fri-bad-challenge-list-fresh-hit-bound-from-round-bounds:
  fixes C :: prob
  assumes local-bound:
     $\bigwedge dg\ i\ prefix.$ 
     $i < \text{fri-round-count-for-degree-bound } (to\_nat\ dg) \implies$ 
     $length\ prefix = i \implies$ 
     $card\ (bad\ dg\ i\ prefix) \leq bd\ dg\ i$ 
  and fraction-bound:
     $\bigwedge dg.$ 
     $nnreal$ 
     $(\sum i \in \{..<\text{fri-round-count-for-degree-bound } (to\_nat\ dg)\}.$ 
     $CARD('f) \wedge^i * bd\ dg\ i *$ 
     $CARD('f) \wedge$ 
     $(\text{fri-round-count-for-degree-bound } (to\_nat\ dg) - Suc\ i)) /$ 
     $nnreal\ (CARD('f) \wedge \text{fri-round-count-for-degree-bound } (to\_nat\ dg)) \leq C$ 
  shows
    wp-event (checked-staged-security-experiment-with-data-state A)
    (staged-security-with-data-state-verifier-event
      ( $\lambda s.$  composition-fri-challenge-list-fresh-hit s
        ( $\lambda dg.$  generic-fri-bad-challenge-lists
          ( $\text{fri-round-count-for-degree-bound } (to\_nat\ dg)$ ) (bad dg))))
    adversary-initial-state ≤ C
proof -
  let ?B =
     $\lambda dg.$  generic-fri-bad-challenge-lists
    ( $\text{fri-round-count-for-degree-bound } (to\_nat\ dg)$ ) (bad dg)
  have subset:
     $\bigwedge dg.$  ?B dg  $\subseteq$  fri-challenge-space (ceil-log (to-nat dg + 1))
  using generic-fri-bad-challenge-lists-subset
  by (simp add: fri-round-count-for-degree-bound-def)
  have ratio-bound:
     $\bigwedge dg.$  nnreal (card (?B dg)) /
    nnreal (CARD('f)  $\wedge$  ceil-log (to-nat dg + 1)) ≤ C
proof -
  fix dg
  let ?n = fri-round-count-for-degree-bound (to-nat dg)
  let ?K =
    ( $\sum i \in \{..<?n\}.$ 
    CARD('f)  $\wedge^i * bd\ dg\ i * CARD('f) \wedge$  (?n - Suc i))

```

```

have card-bound: card (?B dg) ≤ ?K
  by (rule generic-fri-bad-challenge-lists-card-bound-from-round-bounds)
    (rule local-bound)
have nnreal (card (?B dg)) /
  nnreal (CARD('f) ^ ceil-log (to-nat dg + 1))
  ≤ nnreal ?K / nnreal (CARD('f) ^ ceil-log (to-nat dg + 1))
  using card-bound by (rule nnreal-nat-divide-right-mono)
also have ... = nnreal ?K / nnreal (CARD('f) ^ ?n)
  by (simp add: fri-round-count-for-degree-bound-def)
also have ... ≤ C
  by (rule fraction-bound)
finally show
  nnreal (card (?B dg)) /
  nnreal (CARD('f) ^ ceil-log (to-nat dg + 1)) ≤ C .
qed
show ?thesis
  by (rule
    checked-staged-security-composition-fri-challenge-list-fresh-hit-actual-bound
    [OF subset ratio-bound])
qed

lemma checked-staged-security-composition-fri-challenge-list-set-hit-bound-from-fresh-and-prequery:
assumes fresh-bound:
  wp-event (checked-staged-security-experiment-with-data-state A)
  (staged-security-with-data-state-verifier-event
    (λs. composition-fri-challenge-list-fresh-hit s B))
  adversary-initial-state ≤ F
and prequery-bound:
  wp-event (checked-staged-security-experiment-with-data-state A)
  (staged-security-with-data-state-verifier-event
    (λs. composition-fri-challenge-list-prequery-hit s B))
  adversary-initial-state ≤ P
shows
  wp-event (checked-staged-security-experiment-with-data-state A)
  (staged-security-with-data-state-verifier-event
    (λs. composition-fri-challenge-list-set-hit s B))
  adversary-initial-state ≤ F + P
proof –
have wp-event (checked-staged-security-experiment-with-data-state A)
  (staged-security-with-data-state-verifier-event
    (λs. composition-fri-challenge-list-set-hit s B))
  adversary-initial-state ≤
  wp-event (checked-staged-security-experiment-with-data-state A)
  (staged-security-with-data-state-verifier-event
    (λs out.
      composition-fri-challenge-list-fresh-hit s B out ∨
      composition-fri-challenge-list-prequery-hit s B out))
  adversary-initial-state
proof (rule wp-event-mono-on-support)

```



```

fix out
assume
  out ∈
    set-dist
      (execute (checked-staged-security-experiment-with-data-state A)
        adversary-initial-state)
and hit:
  staged-security-with-data-state-verifier-event
    (λs. composition-fri-challenge-list-set-hit s B) out
then show
  staged-security-with-data-state-verifier-event
    (λs out.
      composition-fri-challenge-list-fresh-hit s B out ∨
      composition-fri-challenge-list-prequery-hit s B out) out
unfolding staged-security-with-data-state-verifier-event-def
by (auto dest:
  composition-fri-challenge-list-set-hit-imp-fresh-or-prequery
  split: option.splits prod.splits)
qed
also have ... ≤
  wp-event (checked-staged-security-experiment-with-data-state A)
    (λout.
      staged-security-with-data-state-verifier-event
        (λs. composition-fri-challenge-list-fresh-hit s B) out ∨
      staged-security-with-data-state-verifier-event
        (λs. composition-fri-challenge-list-prequery-hit s B) out)
    adversary-initial-state
by (rule wp-event-mono-on-support)
  (auto simp: staged-security-with-data-state-verifier-event-def
    split: option.splits prod.splits)
also have ... ≤
  wp-event (checked-staged-security-experiment-with-data-state A)
    (staged-security-with-data-state-verifier-event
      (λs. composition-fri-challenge-list-fresh-hit s B))
    adversary-initial-state +
  wp-event (checked-staged-security-experiment-with-data-state A)
    (staged-security-with-data-state-verifier-event
      (λs. composition-fri-challenge-list-prequery-hit s B))
    adversary-initial-state
by (rule wp-event-union-bound)
also have ... ≤ F + P
by (rule add-mono[OF fresh-bound prequery-bound])
finally show ?thesis .
qed

lemma checked-staged-security-composition-fri-challenge-list-set-hit-bound:
assumes wf: staged-budget-wellformed budgets
and controlled: staged-adversary-controlled budgets A
and finite-B:  $\bigwedge dg. \text{finite } (B \ dg)$ 

```

```

and fresh-bound:
  wp-event (checked-staged-security-experiment-with-data-state A)
    (staged-security-with-data-state-verifier-event
      ( $\lambda s. \text{composition-fri-challenge-list-fresh-hit } s \ B$ ))
    adversary-initial-state  $\leq F$ 
shows
  wp-event (checked-staged-security-experiment-with-data-state A)
    (staged-security-with-data-state-verifier-event
      ( $\lambda s. \text{composition-fri-challenge-list-set-hit } s \ B$ ))
    adversary-initial-state  $\leq$ 
       $F +$ 
      hash-relation-budget-value
        ( $\sum dg \in (UNIV :: 'f \text{ set}).$ 
           $\text{card } (B \ dg) * \text{ceil-log } (\text{maxDegree} + 1))$ 
        (staged-attacker-query-budget budgets + staged-challenge-query-budget)
by (rule
  checked-staged-security-composition-fri-challenge-list-set-hit-bound-from-fresh-and-prequery)
(rule fresh-bound,
  rule checked-staged-security-composition-fri-verifier-prequery-hit-bound
    [OF wf controlled finite-B])

end

end

theory Soundness-FRI-Prequery-Trace-Sampled-Bounds
imports Soundness-FRI-Prequery-Challenge-Bounds
begin

  Staged bounds for trace FRI sampled-layer events. The lemmas combine the fresh-
  challenge case with prequery accounting from the shared random oracle.

context soundness
begin

lemma checked-staged-security-trace-fri-sampled-layer-chain-bound-from-fresh-and-prequery:
assumes wf: staged-budget-wellformed budgets
and controlled: staged-adversary-controlled budgets A
and finite-B: finite B
and cover:
   $\bigwedge \text{data attacker-state.}$ 
     $\text{Some } (data, \text{attacker-state}) \in$ 
       $\text{set-dist } (\text{execute } (\text{checked-staged-transcript-program } A)$ 
         $\text{adversary-initial-state}) \implies$ 
      trace-fri-sampled-layer-chain-cover
        (verifier-state-from-adversary attacker-state
          (staged-proof-transcript data)) bad
and envelope:
   $\bigwedge \text{data attacker-state trace-table.}$ 

```

Some (data, attacker-state) ∈
set-dist (execute (checked-staged-transcript-program A)
adversary-initial-state) ⇒
trace-fri-multiround-bad-sets bad trace-table ⊆ B
and *fresh-bound:*
wp-event (checked-staged-security-experiment-with-data-state A)
(staged-security-with-data-state-verifier-event
(λs. trace-fri-challenge-list-fresh-hit s B))
adversary-initial-state ≤ F
shows
wp-event (checked-staged-security-experiment-with-data-state A)
(staged-security-with-data-state-verifier-event
trace-fri-bad-with-sampled-layer-chain)
adversary-initial-state ≤
F +
*hash-relation-budget-value (card B * ceil-log clength)*
(staged-attacker-query-budget budgets + staged-challenge-query-budget)
proof –
have *wp-event (checked-staged-security-experiment-with-data-state A)*
(staged-security-with-data-state-verifier-event
trace-fri-bad-with-sampled-layer-chain)
adversary-initial-state ≤
wp-event (checked-staged-security-experiment-with-data-state A)
(staged-security-with-data-state-verifier-event
(λs. trace-fri-challenge-list-set-hit s B))
adversary-initial-state
proof (rule *wp-event-mono-on-support*)
fix *out*
assume *support:*
out ∈
set-dist
(execute (checked-staged-security-experiment-with-data-state A)
adversary-initial-state)
and *hit:*
staged-security-with-data-state-verifier-event
trace-fri-bad-with-sampled-layer-chain out
from *hit* **obtain** *data attacker-state result final-state* **where**
out-eq: out = Some (((data, attacker-state), result), final-state)
unfolding *staged-security-with-data-state-verifier-event-def*
by (cases *out*) (auto split: *option.splits prod.splits*)
have *builder:*
Some (data, attacker-state) ∈
set-dist (execute (checked-staged-transcript-program A)
adversary-initial-state)
using *checked-staged-security-experiment-with-data-state-outcomeE*
[OF support[unfolded out-eq]]
by *blast*
let *?s =*
verifier-state-from-adversary attacker-state

```

    (staged-proof-transcript data)
have sampled:
  trace-fri-bad-with-sampled-layer-chain ?s
  (Some (result, final-state))
using hit unfolding staged-security-with-data-state-verifier-event-def
  out-eq by simp
have list-hit:
  trace-fri-challenge-list-set-hit ?s B (Some (result, final-state))
by (rule trace-fri-bad-with-sampled-layer-chain-imp-list-hit
    [OF sampled cover[OF builder]])
  (rule envelope[OF builder])
show
  staged-security-with-data-state-verifier-event
  ( $\lambda s.$  trace-fri-challenge-list-set-hit s B) out
unfolding staged-security-with-data-state-verifier-event-def out-eq
using list-hit by simp
qed
also have ...  $\leq$ 
  F +
  hash-relation-budget-value (card B * ceil-log clength)
  (staged-attacker-query-budget budgets + staged-challenge-query-budget)
by (rule checked-staged-security-trace-fri-challenge-list-set-hit-bound
  [OF wf controlled finite-B fresh-bound])
finally show ?thesis .
qed

lemma checked-staged-security-trace-fri-header-tied-sampled-layer-chain-bound-from-fresh-and-prequery:
assumes wf: staged-budget-wellformed budgets
and controlled: staged-adversary-controlled budgets A
and finite-B: finite B
and cover:
 $\wedge$ data attacker-state.
  Some (data, attacker-state)  $\in$ 
  set-dist (execute (checked-staged-transcript-program A)
    adversary-initial-state)  $\implies$ 
  trace-fri-sampled-layer-chain-cover
  (verifier-state-from-adversary attacker-state
    (staged-proof-transcript data)) bad
and envelope:
 $\wedge$ data attacker-state trace-table.
  Some (data, attacker-state)  $\in$ 
  set-dist (execute (checked-staged-transcript-program A)
    adversary-initial-state)  $\implies$ 
  trace-fri-multi-round-bad-sets bad trace-table  $\subseteq B$ 
and fresh-bound:
  wp-event (checked-staged-security-experiment-with-data-state A)
  (staged-security-with-data-state-verifier-event
    ( $\lambda s.$  trace-fri-challenge-list-fresh-hit s B))
  adversary-initial-state  $\leq F$ 

```

```

shows
  wp-event (checked-staged-security-experiment-with-data-state A)
    (staged-security-with-data-state-verifier-event
      trace-fri-bad-with-header-tied-sampled-layer-chain)
  adversary-initial-state ≤
    F +
    hash-relation-budget-value (card B * ceil-log clength)
    (staged-attacker-query-budget budgets + staged-challenge-query-budget)
proof –
  have wp-event (checked-staged-security-experiment-with-data-state A)
    (staged-security-with-data-state-verifier-event
      trace-fri-bad-with-header-tied-sampled-layer-chain)
  adversary-initial-state ≤
    wp-event (checked-staged-security-experiment-with-data-state A)
      (staged-security-with-data-state-verifier-event
        trace-fri-bad-with-sampled-layer-chain)
    adversary-initial-state
  by (rule wp-event-mono-on-support)
    (auto simp: staged-security-with-data-state-verifier-event-def
      intro: trace-fri-header-tied-sampled-layer-chain-imp-sampled-layer-chain
      split: option.splits prod.splits)
  also have ... ≤
    F +
    hash-relation-budget-value (card B * ceil-log clength)
    (staged-attacker-query-budget budgets + staged-challenge-query-budget)
  by (rule
    checked-staged-security-trace-fri-sampled-layer-chain-bound-from-fresh-and-prequery
    [OF wf controlled finite-B cover envelope fresh-bound])
  finally show ?thesis .
qed

end

end

```

```

theory Soundness-FRI-Prequery-Composition-Sampled-Bounds
imports Soundness-FRI-Prequery-Trace-Sampled-Bounds
begin

```

Staged bounds for composition FRI sampled-layer events. This theory mirrors the trace-side prequery accounting with the composition degree parameter.

```

context soundness
begin

```

```

lemma checked-staged-security-composition-fri-sampled-layer-chain-bound-from-fresh-and-prequery:
assumes wf: staged-budget-wellformed budgets
and controlled: staged-adversary-controlled budgets A
and finite-B:  $\bigwedge dg. \text{finite } (B \ dg)$ 

```

and *cover*:
 $\wedge$ *data attacker-state*.
Some (*data*, *attacker-state*) $\in$
set-dist (*execute* (*checked-staged-transcript-program* *A*)
adversary-initial-state) $\implies$
composition-fri-sampled-layer-chain-cover
(*verifier-state-from-adversary attacker-state*
(*staged-proof-transcript data*)) *bad*
and *envelope*:
 $\wedge$ *data attacker-state dg composition-table*.
Some (*data*, *attacker-state*) $\in$
set-dist (*execute* (*checked-staged-transcript-program* *A*)
adversary-initial-state) $\implies$
composition-fri-multiround-bad-sets bad dg composition-table
 $\subseteq B \text{ dg}$
and *fresh-bound*:
wp-event (*checked-staged-security-experiment-with-data-state A*)
(*staged-security-with-data-state-verifier-event*
($\lambda s. \text{composition-fri-challenge-list-fresh-hit } s \text{ } B$))
adversary-initial-state $\leq F$
shows
wp-event (*checked-staged-security-experiment-with-data-state A*)
(*staged-security-with-data-state-verifier-event*
composition-fri-bad-with-sampled-layer-chain)
adversary-initial-state $\leq$
 $F +$
hash-relation-budget-value
 $(\sum dg \in (UNIV :: 'f \text{ set}).$
 $\text{card } (B \text{ dg}) * \text{ceil-log } (\text{maxDegree} + 1))$
 $(\text{staged-attacker-query-budget budgets} + \text{staged-challenge-query-budget})$
proof –
have *wp-event* (*checked-staged-security-experiment-with-data-state A*)
(*staged-security-with-data-state-verifier-event*
composition-fri-bad-with-sampled-layer-chain)
adversary-initial-state $\leq$
wp-event (*checked-staged-security-experiment-with-data-state A*)
(*staged-security-with-data-state-verifier-event*
($\lambda s. \text{composition-fri-challenge-list-set-hit } s \text{ } B$))
adversary-initial-state
proof (*rule wp-event-mono-on-support*)
fix *out*
assume *support*:
out $\in$
set-dist
(*execute* (*checked-staged-security-experiment-with-data-state A*)
adversary-initial-state)
and *hit*:
staged-security-with-data-state-verifier-event
composition-fri-bad-with-sampled-layer-chain out

```

from hit obtain data attacker-state result final-state where
  out-eq: out = Some (((data, attacker-state), result), final-state)
  unfolding staged-security-with-data-state-verifier-event-def
  by (cases out) (auto split: option.splits prod.splits)
have builder:
  Some (data, attacker-state) ∈
    set-dist (execute (checked-staged-transcript-program A)
      adversary-initial-state)
  using checked-staged-security-experiment-with-data-state-outcomeE
    [OF support[unfolded out-eq]]
  by blast
let ?s =
  verifier-state-from-adversary attacker-state
    (staged-proof-transcript data)
have sampled:
  composition-fri-bad-with-sampled-layer-chain ?s
    (Some (result, final-state))
  using hit unfolding staged-security-with-data-state-verifier-event-def
    out-eq by simp
have list-hit:
  composition-fri-challenge-list-set-hit ?s B
    (Some (result, final-state))
  by (rule composition-fri-bad-with-sampled-layer-chain-imp-list-hit
    [OF sampled cover[OF builder]])
    (rule envelope[OF builder])
show
  staged-security-with-data-state-verifier-event
    ( $\lambda s. \text{composition-fri-challenge-list-set-hit } s \ B$ ) out
  unfolding staged-security-with-data-state-verifier-event-def out-eq
  using list-hit by simp
qed
also have ...  $\leq$ 
  F +
  hash-relation-budget-value
    ( $\sum dg \in (UNIV :: 'f \text{ set}).$ 
       $\text{card } (B \ dg) * \text{ceil-log } (\text{maxDegree} + 1))$ 
    (staged-attacker-query-budget budgets + staged-challenge-query-budget)
  by (rule
    checked-staged-security-composition-fri-challenge-list-set-hit-bound
    [OF wf controlled finite-B fresh-bound])
finally show ?thesis .
qed

lemma checked-staged-security-composition-fri-verifier-tied-sampled-layer-chain-bound-from-fresh-and-prequery:
assumes wf: staged-budget-wellformed budgets
and controlled: staged-adversary-controlled budgets A
and finite-B:  $\bigwedge dg. \text{finite } (B \ dg)$ 
and cover:
 $\bigwedge \text{data attacker-state.}$ 

```

Some (data, attacker-state) $\in$
 set-dist (execute (checked-staged-transcript-program A)
 adversary-initial-state) $\implies$
 composition-fri-sampled-layer-chain-cover
 (verifier-state-from-adversary attacker-state
 (staged-proof-transcript data)) bad
and envelope:
 $\wedge$ data attacker-state dg composition-table.
 Some (data, attacker-state) $\in$
 set-dist (execute (checked-staged-transcript-program A)
 adversary-initial-state) $\implies$
 composition-fri-multiround-bad-sets bad dg composition-table
 $\subseteq B$ dg
and fresh-bound:
 wp-event (checked-staged-security-experiment-with-data-state A)
 (staged-security-with-data-state-verifier-event
 (λs . composition-fri-challenge-list-fresh-hit s B))
 adversary-initial-state $\leq F$
shows
 wp-event (checked-staged-security-experiment-with-data-state A)
 (staged-security-with-data-state-verifier-event
 composition-fri-bad-with-verifier-tied-sampled-layer-chain)
 adversary-initial-state $\leq$
 F +
 hash-relation-budget-value
 ($\sum dg \in (UNIV :: 'f \text{ set})$.
 card (B dg) * ceil-log (maxDegree + 1))
 (staged-attacker-query-budget budgets + staged-challenge-query-budget)
proof –
have wp-event (checked-staged-security-experiment-with-data-state A)
 (staged-security-with-data-state-verifier-event
 composition-fri-bad-with-verifier-tied-sampled-layer-chain)
 adversary-initial-state $\leq$
 wp-event (checked-staged-security-experiment-with-data-state A)
 (staged-security-with-data-state-verifier-event
 composition-fri-bad-with-sampled-layer-chain)
 adversary-initial-state
by (rule wp-event-mono-on-support)
 (auto simp: staged-security-with-data-state-verifier-event-def
 intro:
 composition-fri-verifier-tied-sampled-layer-chain-imp-sampled-layer-chain
 split: option.splits prod.splits)
also have ... $\leq$
 F +
 hash-relation-budget-value
 ($\sum dg \in (UNIV :: 'f \text{ set})$.
 card (B dg) * ceil-log (maxDegree + 1))
 (staged-attacker-query-budget budgets + staged-challenge-query-budget)
by (rule


```

      checked-staged-security-composition-fri-sampled-layer-chain-bound-from-fresh-and-prequery
      [OF wf controlled finite-B cover envelope fresh-bound])
    finally show ?thesis .
qed

end

end

```

```

theory Soundness-FRI-Staged-Active-Bounds
  imports Soundness-FRI-Prequery-Composition-Sampled-Bounds
begin

```

Staged integration lemmas for the active FRI route.

The prequery accounting layer proves whole checked-security-experiment bounds for sampled FRI layer-chain events. The active route still reasons about partial-candidate FRI events. This layer bridges those levels by splitting a partial-candidate FRI bad event into the sampled layer-chain case and the remaining missing-sampled-chain residual.

```

context soundness
begin

```

```

lemma trace-fri-header-tied-partial-candidate-imp-sampled-conflict-or-zero:
  assumes bad: trace-fri-bad-with-header-tied-partial-candidate s out
  shows
    trace-fri-bad-with-header-tied-sampled-layer-chain s out ∨
    trace-fri-header-tied-sampled-assignment-conflict s out ∨
    trace-fri-header-tied-zero-round-final-obstruction s out
proof (cases trace-fri-bad-with-header-tied-sampled-layer-chain s out)
  case True
    then show ?thesis by blast
  next
    case False
    have missing: trace-fri-header-tied-missing-sampled-layer-chain s out
    unfolding trace-fri-header-tied-missing-sampled-layer-chain-def
    using bad False by blast
    have obstruction:
      trace-fri-header-tied-sampled-layer-assignment-obstruction s out
    by (rule trace-fri-header-tied-missing-sampled-imp-assignment-obstruction
      [OF missing])
    then show ?thesis
      using trace-fri-header-tied-assignment-obstruction-imp-conflict-or-zero
      by blast
  qed

```

```

lemma checked-staged-security-trace-fri-header-tied-partial-candidate-bound-from-sampled-and-missing:
  assumes sampled-bound:
    wp-event (checked-staged-security-experiment-with-data-state A)
    (staged-security-with-data-state-verifier-event

```

$\text{trace-fri-bad-with-header-tied-sampled-layer-chain})$
 $\text{adversary-initial-state} \leq R$
and *missing-bound*:
 $\wedge \text{data attacker-state.}$
 $\text{Some (data, attacker-state)} \in$
 $\text{set-dist (execute (checked-staged-transcript-program } A)$
 $\text{adversary-initial-state)} \implies$
 $\text{wp-event verify-monad}$
 $(\text{trace-fri-header-tied-missing-sampled-layer-chain}$
 $(\text{verifier-state-from-adversary attacker-state}$
 $(\text{staged-proof-transcript data})))$
 $(\text{verifier-state-from-adversary attacker-state}$
 $(\text{staged-proof-transcript data})) \leq M$
shows
 $\text{wp-event (checked-staged-security-experiment-with-data-state } A)$
 $(\text{staged-security-with-data-state-verifier-event}$
 $\text{trace-fri-bad-with-header-tied-partial-candidate})$
 $\text{adversary-initial-state} \leq R + M$
proof –
have *missing-staged*:
 $\text{wp-event (checked-staged-security-experiment-with-data-state } A)$
 $(\text{staged-security-with-data-state-verifier-event}$
 $\text{trace-fri-header-tied-missing-sampled-layer-chain})$
 $\text{adversary-initial-state} \leq M$
by (rule *checked-staged-security-with-data-state-verifier-event-bound-from-cont*)
 $(\text{auto simp: trace-fri-header-tied-missing-sampled-layer-chain-def}$
 $\text{missing-bound})$
have $\text{wp-event (checked-staged-security-experiment-with-data-state } A)$
 $(\text{staged-security-with-data-state-verifier-event}$
 $\text{trace-fri-bad-with-header-tied-partial-candidate})$
 $\text{adversary-initial-state} \leq$
 $\text{wp-event (checked-staged-security-experiment-with-data-state } A)$
 $(\lambda \text{out.}$
 $\text{staged-security-with-data-state-verifier-event}$
 $\text{trace-fri-bad-with-header-tied-sampled-layer-chain out} \vee$
 $\text{staged-security-with-data-state-verifier-event}$
 $\text{trace-fri-header-tied-missing-sampled-layer-chain out})$
 $\text{adversary-initial-state}$
unfolding *trace-fri-header-tied-missing-sampled-layer-chain-def*
by (rule *wp-event-mono-on-support*)
 $(\text{auto simp: staged-security-with-data-state-verifier-event-def}$
 $\text{split: option.splits prod.splits})$
also have $\dots \leq$
 $\text{wp-event (checked-staged-security-experiment-with-data-state } A)$
 $(\text{staged-security-with-data-state-verifier-event}$
 $\text{trace-fri-bad-with-header-tied-sampled-layer-chain})$
 $\text{adversary-initial-state} +$
 $\text{wp-event (checked-staged-security-experiment-with-data-state } A)$
 $(\text{staged-security-with-data-state-verifier-event}$

```

      trace-fri-header-tied-missing-sampled-layer-chain)
    adversary-initial-state
  by (rule wp-event-union-bound)
  also have ...  $\leq R + M$ 
    by (rule add-mono[OF sampled-bound missing-staged])
  finally show ?thesis .
qed

```

lemma *checked-staged-security-trace-fri-header-tied-partial-candidate-bound-from-sampled-conflict-and-staged-zero:*

assumes *sampled-bound:*

```

  wp-event (checked-staged-security-experiment-with-data-state A)
    (staged-security-with-data-state-verifier-event
      trace-fri-bad-with-header-tied-sampled-layer-chain)
  adversary-initial-state  $\leq R$ 

```

and *conflict-bound:*

```

  wp-event (checked-staged-security-experiment-with-data-state A)
    (staged-security-with-data-state-verifier-event
      trace-fri-header-tied-sampled-assignment-conflict)
  adversary-initial-state  $\leq C$ 

```

and *zero-bound:*

```

  wp-event (checked-staged-security-experiment-with-data-state A)
    (staged-security-with-data-state-verifier-event
      trace-fri-header-tied-zero-round-final-obstruction)
  adversary-initial-state  $\leq Z$ 

```

shows

```

  wp-event (checked-staged-security-experiment-with-data-state A)
    (staged-security-with-data-state-verifier-event
      trace-fri-bad-with-header-tied-partial-candidate)
  adversary-initial-state  $\leq R + (C + Z)$ 

```

proof –

let *?M* = *checked-staged-security-experiment-with-data-state A*

let *?sampled* =

```

  staged-security-with-data-state-verifier-event
  trace-fri-bad-with-header-tied-sampled-layer-chain

```

let *?conflict* =

```

  staged-security-with-data-state-verifier-event
  trace-fri-header-tied-sampled-assignment-conflict

```

let *?zero* =

```

  staged-security-with-data-state-verifier-event
  trace-fri-header-tied-zero-round-final-obstruction

```

have *split:*

```

  wp-event ?M
    (staged-security-with-data-state-verifier-event
      trace-fri-bad-with-header-tied-partial-candidate)
  adversary-initial-state  $\leq$ 
  wp-event ?M ( $\lambda out. ?sampled\ out \vee ?conflict\ out \vee ?zero\ out$ )
  adversary-initial-state

```

unfolding *staged-security-with-data-state-verifier-event-def*

by (rule wp-event-mono-on-support)

```

(auto dest:
  trace-fri-header-tied-partial-candidate-imp-sampled-conflict-or-zero
  split: option.splits prod.splits)
also have ... ≤
  wp-event ?M ?sampled adversary-initial-state +
  wp-event ?M (λout. ?conflict out ∨ ?zero out)
  adversary-initial-state
by (rule wp-event-union-bound)
also have ... ≤
  wp-event ?M ?sampled adversary-initial-state +
  (wp-event ?M ?conflict adversary-initial-state +
   wp-event ?M ?zero adversary-initial-state)
by (intro add-mono order-refl wp-event-union-bound)
also have ... ≤ R + (C + Z)
by (intro add-mono sampled-bound conflict-bound zero-bound)
finally show ?thesis .
qed

lemma checked-staged-security-trace-fri-header-tied-partial-candidate-bound-from-fresh-prequery-and-missing:
  assumes wf: staged-budget-wellformed budgets
  and controlled: staged-adversary-controlled budgets A
  and finite-B: finite B
  and cover:
    ∧data attacker-state.
      Some (data, attacker-state) ∈
        set-dist (execute (checked-staged-transcript-program A)
          adversary-initial-state) ⇒
        trace-fri-sampled-layer-chain-cover
          (verifier-state-from-adversary attacker-state
            (staged-proof-transcript data)) bad
  and envelope:
    ∧data attacker-state trace-table.
      Some (data, attacker-state) ∈
        set-dist (execute (checked-staged-transcript-program A)
          adversary-initial-state) ⇒
        trace-fri-multi-round-bad-sets bad trace-table ⊆ B
  and fresh-bound:
    wp-event (checked-staged-security-experiment-with-data-state A)
      (staged-security-with-data-state-verifier-event
        (λs. trace-fri-challenge-list-fresh-hit s B))
      adversary-initial-state ≤ F
  and missing-bound:
    ∧data attacker-state.
      Some (data, attacker-state) ∈
        set-dist (execute (checked-staged-transcript-program A)
          adversary-initial-state) ⇒
      wp-event verify-mono
        (trace-fri-header-tied-missing-sampled-layer-chain
          (verifier-state-from-adversary attacker-state

```

$$\begin{aligned} & (\text{staged-proof-transcript data})) \\ & (\text{verifier-state-from-adversary attacker-state} \\ & (\text{staged-proof-transcript data})) \leq M \end{aligned}$$

shows

$$\begin{aligned} & \text{wp-event } (\text{checked-staged-security-experiment-with-data-state } A) \\ & (\text{staged-security-with-data-state-verifier-event} \\ & \quad \text{trace-fri-bad-with-header-tied-partial-candidate}) \\ & \text{adversary-initial-state} \leq \\ & (F + \\ & \quad \text{hash-relation-budget-value } (\text{card } B * \text{ceil-log clength}) \\ & \quad (\text{staged-attacker-query-budget budgets} + \\ & \quad \text{staged-challenge-query-budget})) + M \end{aligned}$$

proof –

have *sampled*:

$$\begin{aligned} & \text{wp-event } (\text{checked-staged-security-experiment-with-data-state } A) \\ & (\text{staged-security-with-data-state-verifier-event} \\ & \quad \text{trace-fri-bad-with-header-tied-sampled-layer-chain}) \\ & \text{adversary-initial-state} \leq \\ & F + \\ & \quad \text{hash-relation-budget-value } (\text{card } B * \text{ceil-log clength}) \\ & \quad (\text{staged-attacker-query-budget budgets} + \\ & \quad \text{staged-challenge-query-budget}) \end{aligned}$$

by (rule

$$\begin{aligned} & \text{checked-staged-security-trace-fri-header-tied-sampled-layer-chain-bound-from-fresh-and-prequery} \\ & [\text{OF wf controlled finite-B cover envelope fresh-bound}]) \end{aligned}$$

show *?thesis*

by (rule

$$\begin{aligned} & \text{checked-staged-security-trace-fri-header-tied-partial-candidate-bound-from-sampled-and-missing} \\ & [\text{OF sampled missing-bound}]) \end{aligned}$$

qed

lemma *checked-staged-security-trace-fri-header-tied-partial-candidate-bound-from-fresh-prequery-and-assignment-obs*

assumes *wf: staged-budget-wellformed budgets*

and *controlled: staged-adversary-controlled budgets A*

and *finite-B: finite B*

and *cover:*

$$\begin{aligned} & \bigwedge \text{data attacker-state.} \\ & \text{Some } (\text{data, attacker-state}) \in \\ & \quad \text{set-dist } (\text{execute } (\text{checked-staged-transcript-program } A) \\ & \quad \text{adversary-initial-state}) \implies \\ & \text{trace-fri-sampled-layer-chain-cover} \\ & \quad (\text{verifier-state-from-adversary attacker-state} \\ & \quad (\text{staged-proof-transcript data})) \text{ bad} \end{aligned}$$

and *envelope:*

$$\begin{aligned} & \bigwedge \text{data attacker-state trace-table.} \\ & \text{Some } (\text{data, attacker-state}) \in \\ & \quad \text{set-dist } (\text{execute } (\text{checked-staged-transcript-program } A) \\ & \quad \text{adversary-initial-state}) \implies \\ & \text{trace-fri-multi-round-bad-sets bad trace-table} \subseteq B \end{aligned}$$

and *fresh-bound*:
wp-event (*checked-staged-security-experiment-with-data-state A*)
 (*staged-security-with-data-state-verifier-event*
 ($\lambda s. \text{trace-fri-challenge-list-fresh-hit } s \ B$))
 adversary-initial-state $\leq F$
and *assignment-bound*:
 $\wedge \text{data attacker-state.}$
 Some (*data*, *attacker-state*) $\in$
 set-dist (*execute* (*checked-staged-transcript-program A*)
 adversary-initial-state) $\implies$
wp-event verify-monad
 (*trace-fri-header-tied-sampled-layer-assignment-obstruction*
 (*verifier-state-from-adversary attacker-state*
 (*staged-proof-transcript data*)))
 (*verifier-state-from-adversary attacker-state*
 (*staged-proof-transcript data*)) $\leq M$
shows
wp-event (*checked-staged-security-experiment-with-data-state A*)
 (*staged-security-with-data-state-verifier-event*
 trace-fri-bad-with-header-tied-partial-candidate)
 adversary-initial-state $\leq$
 ($F +$
 hash-relation-budget-value (*card B* * *ceil-log clength*)
 (*staged-attacker-query-budget budgets* +
 staged-challenge-query-budget)) + M
proof (*rule*
 checked-staged-security-trace-fri-header-tied-partial-candidate-bound-from-fresh-prequery-and-missing
 [*OF wf controlled finite-B cover envelope fresh-bound*])
fix *data attacker-state*
assume *support*:
 Some (*data*, *attacker-state*) $\in$
 set-dist (*execute* (*checked-staged-transcript-program A*)
 adversary-initial-state)
show *wp-event verify-monad*
 (*trace-fri-header-tied-missing-sampled-layer-chain*
 (*verifier-state-from-adversary attacker-state*
 (*staged-proof-transcript data*)))
 (*verifier-state-from-adversary attacker-state*
 (*staged-proof-transcript data*)) $\leq M$
by (*rule wp-trace-fri-header-tied-missing-sampled-bound-from-assignment-obstruction*)
 (*rule assignment-bound*[*OF support*])
qed

lemma *checked-staged-security-trace-fri-header-tied-partial-candidate-bound-from-fresh-prequery-conflict-and-staged-2*
assumes *wf*: *staged-budget-wellformed budgets*
and *controlled*: *staged-adversary-controlled budgets A*
and *finite-B*: *finite B*
and *cover*:
 $\wedge \text{data attacker-state.}$

Some (*data*, *attacker-state*) $\in$
set-dist (*execute* (*checked-staged-transcript-program* *A*)
adversary-initial-state) $\implies$
trace-fri-sampled-layer-chain-cover
(*verifier-state-from-adversary* *attacker-state*
(*staged-proof-transcript* *data*)) *bad*
and *envelope*:
 $\wedge$ *data* *attacker-state* *trace-table*.
Some (*data*, *attacker-state*) $\in$
set-dist (*execute* (*checked-staged-transcript-program* *A*)
adversary-initial-state) $\implies$
trace-fri-multiround-bad-sets *bad* *trace-table* $\subseteq B$
and *fresh-bound*:
wp-event (*checked-staged-security-experiment-with-data-state* *A*)
(*staged-security-with-data-state-verifier-event*
($\lambda s.$ *trace-fri-challenge-list-fresh-hit* *s* *B*))
adversary-initial-state $\leq F$
and *conflict-bound*:
 $\wedge$ *data* *attacker-state*.
Some (*data*, *attacker-state*) $\in$
set-dist (*execute* (*checked-staged-transcript-program* *A*)
adversary-initial-state) $\implies$
wp-event *verify-monad*
(*trace-fri-header-tied-sampled-assignment-conflict*
(*verifier-state-from-adversary* *attacker-state*
(*staged-proof-transcript* *data*)))
(*verifier-state-from-adversary* *attacker-state*
(*staged-proof-transcript* *data*)) $\leq C$
and *zero-bound*:
wp-event (*checked-staged-security-experiment-with-data-state* *A*)
(*staged-security-with-data-state-verifier-event*
trace-fri-header-tied-zero-round-final-obstruction)
adversary-initial-state $\leq Z$
shows
wp-event (*checked-staged-security-experiment-with-data-state* *A*)
(*staged-security-with-data-state-verifier-event*
trace-fri-bad-with-header-tied-partial-candidate)
adversary-initial-state $\leq$
(*F* +
hash-relation-budget-value (*card* *B* * *ceil-log* *clength*)
(*staged-attacker-query-budget* *budgets* +
staged-challenge-query-budget)) + (*C* + *Z*)
proof (*rule*
checked-staged-security-trace-fri-header-tied-partial-candidate-bound-from-sampled-conflict-and-staged-zero)
show *wp-event* (*checked-staged-security-experiment-with-data-state* *A*)
(*staged-security-with-data-state-verifier-event*
trace-fri-bad-with-header-tied-sampled-layer-chain)
adversary-initial-state $\leq$
F +

```

    hash-relation-budget-value (card  $B$  * ceil-log clength)
      (staged-attacker-query-budget budgets +
       staged-challenge-query-budget)
  by (rule
    checked-staged-security-trace-fri-header-tied-sampled-layer-chain-bound-from-fresh-and-prequery
    [OF wf controlled finite-B cover envelope fresh-bound])
  show wp-event (checked-staged-security-experiment-with-data-state  $A$ )
    (staged-security-with-data-state-verifier-event
     trace-fri-header-tied-sampled-assignment-conflict)
    adversary-initial-state  $\leq C$ 
  by (rule checked-staged-security-with-data-state-verifier-event-bound-from-cont)
    (simp-all add: conflict-bound
     trace-fri-header-tied-sampled-assignment-conflict-def
     accepted-fri-opening-transcript-def)
  show wp-event (checked-staged-security-experiment-with-data-state  $A$ )
    (staged-security-with-data-state-verifier-event
     trace-fri-header-tied-zero-round-final-obstruction)
    adversary-initial-state  $\leq Z$ 
  by (rule zero-bound)
qed

lemma checked-staged-security-trace-fri-header-tied-partial-candidate-bound-from-fresh-prequery-conflict-and-zero:
  assumes wf: staged-budget-wellformed budgets
  and controlled: staged-adversary-controlled budgets  $A$ 
  and finite-B: finite  $B$ 
  and cover:
     $\bigwedge$ data attacker-state.
      Some (data, attacker-state)  $\in$ 
        set-dist (execute (checked-staged-transcript-program  $A$ )
          adversary-initial-state)  $\implies$ 
        trace-fri-sampled-layer-chain-cover
          (verifier-state-from-adversary attacker-state
            (staged-proof-transcript data)) bad
  and envelope:
     $\bigwedge$ data attacker-state trace-table.
      Some (data, attacker-state)  $\in$ 
        set-dist (execute (checked-staged-transcript-program  $A$ )
          adversary-initial-state)  $\implies$ 
        trace-fri-multi-round-bad-sets bad trace-table  $\subseteq B$ 
  and fresh-bound:
    wp-event (checked-staged-security-experiment-with-data-state  $A$ )
      (staged-security-with-data-state-verifier-event
       ( $\lambda s$ . trace-fri-challenge-list-fresh-hit  $s$   $B$ ))
      adversary-initial-state  $\leq F$ 
  and conflict-bound:
     $\bigwedge$ data attacker-state.
      Some (data, attacker-state)  $\in$ 
        set-dist (execute (checked-staged-transcript-program  $A$ )
          adversary-initial-state)  $\implies$ 

```


wp-event verify-monad
 (trace-fri-header-tied-sampled-assignment-conflict
 (verifier-state-from-adversary attacker-state
 (staged-proof-transcript data)))
 (verifier-state-from-adversary attacker-state
 (staged-proof-transcript data)) $\leq C$
and zero-bound:
 $\wedge$ data attacker-state.
 Some (data, attacker-state) $\in$
 set-dist (execute (checked-staged-transcript-program A)
 adversary-initial-state) $\implies$
wp-event verify-monad
 (trace-fri-header-tied-zero-round-final-obstruction
 (verifier-state-from-adversary attacker-state
 (staged-proof-transcript data)))
 (verifier-state-from-adversary attacker-state
 (staged-proof-transcript data)) $\leq Z$
shows
wp-event (checked-staged-security-experiment-with-data-state A)
 (staged-security-with-data-state-verifier-event
 trace-fri-bad-with-header-tied-partial-candidate)
 adversary-initial-state $\leq$
 (F +
 hash-relation-budget-value (card B * ceil-log clength)
 (staged-attacker-query-budget budgets +
 staged-challenge-query-budget)) + (C + Z)
proof (rule
 checked-staged-security-trace-fri-header-tied-partial-candidate-bound-from-fresh-prequery-and-assignment-obstruction
 [OF wf controlled finite-B cover envelope fresh-bound])
fix data attacker-state
assume support:
 Some (data, attacker-state) $\in$
 set-dist (execute (checked-staged-transcript-program A)
 adversary-initial-state)
show *wp-event verify-monad*
 (trace-fri-header-tied-sampled-layer-assignment-obstruction
 (verifier-state-from-adversary attacker-state
 (staged-proof-transcript data)))
 (verifier-state-from-adversary attacker-state
 (staged-proof-transcript data)) $\leq C + Z$
by (rule
 wp-trace-fri-header-tied-assignment-obstruction-bound-from-conflict-and-zero)
 (rule conflict-bound[OF support], rule zero-bound[OF support])
qed

lemma checked-staged-security-trace-fri-header-tied-partial-candidate-bound-from-fresh-prequery-and-residual-gaps:
assumes wf: staged-budget-wellformed budgets
and controlled: staged-adversary-controlled budgets A
and finite-B: finite B

and *cover*:
 $\wedge$ *data attacker-state*.
Some (*data*, *attacker-state*) $\in$
set-dist (*execute* (*checked-staged-transcript-program* *A*)
adversary-initial-state) $\implies$
trace-fri-sampled-layer-chain-cover
(*verifier-state-from-adversary attacker-state*
(*staged-proof-transcript data*)) *bad*
and *envelope*:
 $\wedge$ *data attacker-state trace-table*.
Some (*data*, *attacker-state*) $\in$
set-dist (*execute* (*checked-staged-transcript-program* *A*)
adversary-initial-state) $\implies$
trace-fri-multiround-bad-sets bad trace-table $\subseteq B$
and *fresh-bound*:
wp-event (*checked-staged-security-experiment-with-data-state A*)
(*staged-security-with-data-state-verifier-event*
($\lambda s. \text{trace-fri-challenge-list-fresh-hit } s \ B$))
adversary-initial-state $\leq$ *Fresh*
and *slot-bound*:
 $\wedge$ *data attacker-state*.
Some (*data*, *attacker-state*) $\in$
set-dist (*execute* (*checked-staged-transcript-program* *A*)
adversary-initial-state) $\implies$
wp-event verify-monad
(*trace-fri-header-tied-authenticated-slot-recorded-chunk-gap*
(*verifier-state-from-adversary attacker-state*
(*staged-proof-transcript data*)))
(*verifier-state-from-adversary attacker-state*
(*staged-proof-transcript data*)) $\leq$ *Slot*
and *merkle-bound*:
 $\wedge$ *data attacker-state*.
Some (*data*, *attacker-state*) $\in$
set-dist (*execute* (*checked-staged-transcript-program* *A*)
adversary-initial-state) $\implies$
wp-event verify-monad
(*partial-merkle-inconsistency-bad*
(*verifier-state-from-adversary attacker-state*
(*staged-proof-transcript data*)))
(*verifier-state-from-adversary attacker-state*
(*staged-proof-transcript data*)) $\leq$ *Merkle*
and *structural-bound*:
 $\wedge$ *data attacker-state*.
Some (*data*, *attacker-state*) $\in$
set-dist (*execute* (*checked-staged-transcript-program* *A*)
adversary-initial-state) $\implies$
wp-event verify-monad
(*trace-fri-header-tied-sibling-mismatch-structural-gap*
(*verifier-state-from-adversary attacker-state*

$(\text{staged-proof-transcript data}))$
 $(\text{verifier-state-from-adversary attacker-state}$
 $(\text{staged-proof-transcript data})) \leq \text{Structural}$
and *alignment-bound*:
 $\wedge \text{data attacker-state.}$
 $\text{Some (data, attacker-state)} \in$
 $\text{set-dist (execute (checked-staged-transcript-program A)}$
 $\text{adversary-initial-state)} \implies$
 $\text{wp-event verify-monad}$
 $(\text{trace-fri-header-tied-base-value-alignment-gap}$
 $(\text{verifier-state-from-adversary attacker-state}$
 $(\text{staged-proof-transcript data}))$
 $(\text{verifier-state-from-adversary attacker-state}$
 $(\text{staged-proof-transcript data})) \leq \text{Align}$
and *next-bound*:
 $\wedge \text{data attacker-state.}$
 $\text{Some (data, attacker-state)} \in$
 $\text{set-dist (execute (checked-staged-transcript-program A)}$
 $\text{adversary-initial-state)} \implies$
 $\text{wp-event verify-monad}$
 $(\text{trace-fri-header-tied-next-value-replay-gap}$
 $(\text{verifier-state-from-adversary attacker-state}$
 $(\text{staged-proof-transcript data}))$
 $(\text{verifier-state-from-adversary attacker-state}$
 $(\text{staged-proof-transcript data})) \leq \text{Next}$
and *successor-bound*:
 $\wedge \text{data attacker-state.}$
 $\text{Some (data, attacker-state)} \in$
 $\text{set-dist (execute (checked-staged-transcript-program A)}$
 $\text{adversary-initial-state)} \implies$
 $\text{wp-event verify-monad}$
 $(\text{trace-fri-header-tied-successor-replay-gap}$
 $(\text{verifier-state-from-adversary attacker-state}$
 $(\text{staged-proof-transcript data}))$
 $(\text{verifier-state-from-adversary attacker-state}$
 $(\text{staged-proof-transcript data})) \leq \text{Successor}$
and *final-bound*:
 $\wedge \text{data attacker-state.}$
 $\text{Some (data, attacker-state)} \in$
 $\text{set-dist (execute (checked-staged-transcript-program A)}$
 $\text{adversary-initial-state)} \implies$
 $\text{wp-event verify-monad}$
 $(\text{trace-fri-header-tied-final-value-replay-gap}$
 $(\text{verifier-state-from-adversary attacker-state}$
 $(\text{staged-proof-transcript data}))$
 $(\text{verifier-state-from-adversary attacker-state}$
 $(\text{staged-proof-transcript data})) \leq \text{Final}$
and *zero-bound*:
 $\wedge \text{data attacker-state.}$

Some (data, attacker-state) ∈
set-dist (execute (checked-staged-transcript-program A)
adversary-initial-state) ⇒
wp-event verify-monad
(trace-fri-header-tied-zero-round-final-obstruction
(verifier-state-from-adversary attacker-state
(staged-proof-transcript data)))
(verifier-state-from-adversary attacker-state
(staged-proof-transcript data)) ≤ Zero
shows
wp-event (checked-staged-security-experiment-with-data-state A)
(staged-security-with-data-state-verifier-event
trace-fri-bad-with-header-tied-partial-candidate)
adversary-initial-state ≤
(Fresh +
*hash-relation-budget-value (card B * ceil-log clength)*
(staged-attacker-query-budget budgets +
staged-challenge-query-budget)) +
(((((Merkle + Structural) + Align) + Next + Successor + Final) +
Merkle) + Slot + Zero)
proof (rule
checked-staged-security-trace-fri-header-tied-partial-candidate-bound-from-fresh-prequery-conflict-and-zero
[OF wf controlled finite-B cover envelope fresh-bound])
fix data attacker-state
assume support:
Some (data, attacker-state) ∈
set-dist (execute (checked-staged-transcript-program A)
adversary-initial-state)
let ?s =
verifier-state-from-adversary attacker-state
(staged-proof-transcript data)
show wp-event verify-monad
(trace-fri-header-tied-sampled-assignment-conflict ?s) ?s
≤ ((((((Merkle + Structural) + Align) + Next + Successor + Final) +
Merkle) + Slot)
by (rule
wp-trace-fri-header-tied-sampled-assignment-conflict-bound-from-slot-recorded-chunk-gap-and-residuals)
(rule slot-bound[OF support],
rule merkle-bound[OF support],
rule structural-bound[OF support],
rule alignment-bound[OF support],
rule next-bound[OF support],
rule successor-bound[OF support],
rule final-bound[OF support])
show wp-event verify-monad
(trace-fri-header-tied-zero-round-final-obstruction ?s) ?s ≤ Zero
by (rule zero-bound[OF support])
qed

lemma *checked-staged-security-trace-fri-empty-header-bound-from-sampled-and-missing:*

assumes *sampled-bound:*

wp-event (checked-staged-security-experiment-with-data-state A)
(staged-security-with-data-state-verifier-event
trace-fri-bad-with-sampled-layer-chain)
adversary-initial-state ≤ R

and *missing-bound:*

$\wedge$ *data attacker-state.*
Some (data, attacker-state) ∈
set-dist (execute (checked-staged-transcript-program A)
adversary-initial-state) $\implies$
wp-event verify-monad
(trace-fri-reachable-missing-sampled-layer-chain
(verifier-state-from-adversary attacker-state
(staged-proof-transcript data)))
(verifier-state-from-adversary attacker-state
(staged-proof-transcript data)) ≤ M

shows

wp-event (checked-staged-security-experiment-with-data-state A)
(staged-security-with-data-state-verifier-event
trace-fri-bad-with-empty-composition-header-candidates)
adversary-initial-state ≤ R + M

proof —

have *missing-staged:*

wp-event (checked-staged-security-experiment-with-data-state A)
(staged-security-with-data-state-verifier-event
trace-fri-reachable-missing-sampled-layer-chain)
adversary-initial-state ≤ M

by (rule *checked-staged-security-with-data-state-verifier-event-bound-from-cont*)
(simp add: *trace-fri-reachable-missing-sampled-layer-chain-def*
trace-fri-bad-with-reachable-partial-candidate-def
accepted-with-partial-trace-openings-def,
rule *missing-bound*)

have *wp-event (checked-staged-security-experiment-with-data-state A)*
(staged-security-with-data-state-verifier-event
trace-fri-bad-with-empty-composition-header-candidates)
adversary-initial-state ≤
wp-event (checked-staged-security-experiment-with-data-state A)
(λout.
staged-security-with-data-state-verifier-event
trace-fri-bad-with-sampled-layer-chain out ∨
staged-security-with-data-state-verifier-event
trace-fri-reachable-missing-sampled-layer-chain out)
adversary-initial-state

by (rule *wp-event-mono-on-support*)
(auto simp: *staged-security-with-data-state-verifier-event-def*
dest:
trace-fri-bad-with-empty-composition-header-candidates-imp-reachable-partial-candidate
trace-fri-reachable-imp-sampled-or-missing-layer-chain

```

    split: option.splits prod.splits)
also have ...  $\leq$ 
  wp-event (checked-staged-security-experiment-with-data-state A)
    (staged-security-with-data-state-verifier-event
      trace-fri-bad-with-sampled-layer-chain)
  adversary-initial-state +
  wp-event (checked-staged-security-experiment-with-data-state A)
    (staged-security-with-data-state-verifier-event
      trace-fri-reachable-missing-sampled-layer-chain)
  adversary-initial-state
by (rule wp-event-union-bound)
also have ...  $\leq R + M$ 
  by (rule add-mono[OF sampled-bound missing-staged])
finally show ?thesis .
qed

lemma checked-staged-security-trace-fri-empty-header-bound-from-fresh-prequery-and-missing:
assumes wf: staged-budget-wellformed budgets
and controlled: staged-adversary-controlled budgets A
and finite-B: finite B
and cover:
 $\wedge$ data attacker-state.
  Some (data, attacker-state)  $\in$ 
    set-dist (execute (checked-staged-transcript-program A)
      adversary-initial-state)  $\implies$ 
    trace-fri-sampled-layer-chain-cover
      (verifier-state-from-adversary attacker-state
        (staged-proof-transcript data)) bad
and envelope:
 $\wedge$ data attacker-state trace-table.
  Some (data, attacker-state)  $\in$ 
    set-dist (execute (checked-staged-transcript-program A)
      adversary-initial-state)  $\implies$ 
    trace-fri-multi-round-bad-sets bad trace-table  $\subseteq B$ 
and fresh-bound:
  wp-event (checked-staged-security-experiment-with-data-state A)
    (staged-security-with-data-state-verifier-event
      ( $\lambda s$ . trace-fri-challenge-list-fresh-hit s B))
  adversary-initial-state  $\leq F$ 
and missing-bound:
 $\wedge$ data attacker-state.
  Some (data, attacker-state)  $\in$ 
    set-dist (execute (checked-staged-transcript-program A)
      adversary-initial-state)  $\implies$ 
  wp-event verify-monad
    (trace-fri-reachable-missing-sampled-layer-chain
      (verifier-state-from-adversary attacker-state
        (staged-proof-transcript data)))
    (verifier-state-from-adversary attacker-state

```

$(\text{staged-proof-transcript data}) \leq M$
shows
 $\text{wp-event } (\text{checked-staged-security-experiment-with-data-state } A)$
 $(\text{staged-security-with-data-state-verifier-event}$
 $\text{trace-fri-bad-with-empty-composition-header-candidates})$
 $\text{adversary-initial-state} \leq$
 $(F +$
 $\text{hash-relation-budget-value } (\text{card } B * \text{ceil-log } \text{clength})$
 $(\text{staged-attacker-query-budget budgets} +$
 $\text{staged-challenge-query-budget})) + M$
proof –
have *sampled*:
 $\text{wp-event } (\text{checked-staged-security-experiment-with-data-state } A)$
 $(\text{staged-security-with-data-state-verifier-event}$
 $\text{trace-fri-bad-with-sampled-layer-chain})$
 $\text{adversary-initial-state} \leq$
 $F +$
 $\text{hash-relation-budget-value } (\text{card } B * \text{ceil-log } \text{clength})$
 $(\text{staged-attacker-query-budget budgets} +$
 $\text{staged-challenge-query-budget})$
by (rule
 $\text{checked-staged-security-trace-fri-sampled-layer-chain-bound-from-fresh-and-prequery}$
 $[\text{OF wf controlled finite-B cover envelope fresh-bound}])$
show *?thesis*
by (rule
 $\text{checked-staged-security-trace-fri-empty-header-bound-from-sampled-and-missing}$
 $[\text{OF sampled missing-bound}])$
qed

lemma *checked-staged-security-trace-fri-empty-header-bound-from-fresh-prequery-and-assignment-obstruction:*

assumes *wf: staged-budget-wellformed budgets*
and *controlled: staged-adversary-controlled budgets A*
and *finite-B: finite B*
and *cover:*
 $\bigwedge \text{data attacker-state.}$
 $\text{Some } (\text{data, attacker-state}) \in$
 $\text{set-dist } (\text{execute } (\text{checked-staged-transcript-program } A)$
 $\text{adversary-initial-state}) \implies$
 $\text{trace-fri-sampled-layer-chain-cover}$
 $(\text{verifier-state-from-adversary attacker-state}$
 $(\text{staged-proof-transcript data})) \text{ bad}$
and *envelope:*
 $\bigwedge \text{data attacker-state trace-table.}$
 $\text{Some } (\text{data, attacker-state}) \in$
 $\text{set-dist } (\text{execute } (\text{checked-staged-transcript-program } A)$
 $\text{adversary-initial-state}) \implies$
 $\text{trace-fri-multi-round-bad-sets bad trace-table} \subseteq B$
and *fresh-bound:*
 $\text{wp-event } (\text{checked-staged-security-experiment-with-data-state } A)$

(staged-security-with-data-state-verifier-event
($\lambda s. \text{trace-fri-challenge-list-fresh-hit } s \ B$))
adversary-initial-state $\leq F$
and *assignment-bound:*
 $\wedge \text{data attacker-state.}$
Some (data, attacker-state) $\in$
set-dist (execute (checked-staged-transcript-program A)
adversary-initial-state) $\implies$
wp-event verify-monad
(trace-fri-sampled-layer-assignment-obstruction
(verifier-state-from-adversary attacker-state
(staged-proof-transcript data)))
(verifier-state-from-adversary attacker-state
(staged-proof-transcript data)) $\leq M$
shows
wp-event (checked-staged-security-experiment-with-data-state A)
(staged-security-with-data-state-verifier-event
trace-fri-bad-with-empty-composition-header-candidates)
adversary-initial-state $\leq$
(F +
*hash-relation-budget-value (card B * ceil-log clength)*
(staged-attacker-query-budget budgets +
staged-challenge-query-budget)) + M
proof (rule
checked-staged-security-trace-fri-empty-header-bound-from-fresh-prequery-and-missing
[OF wf controlled finite-B cover envelope fresh-bound])
fix *data attacker-state*
assume *support:*
Some (data, attacker-state) $\in$
set-dist (execute (checked-staged-transcript-program A)
adversary-initial-state)
show *wp-event verify-monad*
(trace-fri-reachable-missing-sampled-layer-chain
(verifier-state-from-adversary attacker-state
(staged-proof-transcript data)))
(verifier-state-from-adversary attacker-state
(staged-proof-transcript data)) $\leq M$
by (rule *wp-trace-fri-reachable-missing-sampled-bound-from-assignment-obstruction*)
(rule *assignment-bound[OF support]*)
qed

lemma *checked-staged-security-composition-fri-verifier-tied-partial-candidate-bound-from-sampled-and-missing:*
assumes *sampled-bound:*
wp-event (checked-staged-security-experiment-with-data-state A)
(staged-security-with-data-state-verifier-event
composition-fri-bad-with-verifier-tied-sampled-layer-chain)
adversary-initial-state $\leq R$
and *missing-bound:*
 $\wedge \text{data attacker-state.}$

Some $(data, attacker-state) \in$
 set-dist (execute (checked-staged-transcript-program A)
 adversary-initial-state) $\implies$
 wp-event verify-monad
 (composition-fri-verifier-tied-missing-sampled-layer-chain
 (verifier-state-from-adversary attacker-state
 (staged-proof-transcript data)))
 (verifier-state-from-adversary attacker-state
 (staged-proof-transcript data)) $\leq M$

shows

wp-event (checked-staged-security-experiment-with-data-state A)
 (staged-security-with-data-state-verifier-event
 composition-fri-bad-with-verifier-tied-partial-candidate)
 adversary-initial-state $\leq R + M$

proof –

have missing-staged:

wp-event (checked-staged-security-experiment-with-data-state A)
 (staged-security-with-data-state-verifier-event
 composition-fri-verifier-tied-missing-sampled-layer-chain)
 adversary-initial-state $\leq M$

by (rule checked-staged-security-with-data-state-verifier-event-bound-from-cont)
 (auto simp: composition-fri-verifier-tied-missing-sampled-layer-chain-def
 missing-bound)

have wp-event (checked-staged-security-experiment-with-data-state A)
 (staged-security-with-data-state-verifier-event
 composition-fri-bad-with-verifier-tied-partial-candidate)
 adversary-initial-state $\leq$
 wp-event (checked-staged-security-experiment-with-data-state A)
 ($\lambda out.$
 staged-security-with-data-state-verifier-event
 composition-fri-bad-with-verifier-tied-sampled-layer-chain out $\vee$
 staged-security-with-data-state-verifier-event
 composition-fri-verifier-tied-missing-sampled-layer-chain out)
 adversary-initial-state

unfolding composition-fri-verifier-tied-missing-sampled-layer-chain-def

by (rule wp-event-mono-on-support)
 (auto simp: staged-security-with-data-state-verifier-event-def
 split: option.splits prod.splits)

also have ... $\leq$
 wp-event (checked-staged-security-experiment-with-data-state A)
 (staged-security-with-data-state-verifier-event
 composition-fri-bad-with-verifier-tied-sampled-layer-chain)
 adversary-initial-state +
 wp-event (checked-staged-security-experiment-with-data-state A)
 (staged-security-with-data-state-verifier-event
 composition-fri-verifier-tied-missing-sampled-layer-chain)
 adversary-initial-state

by (rule wp-event-union-bound)

also have ... $\leq R + M$

by (rule add-mono[OF sampled-bound missing-staged])
 finally show ?thesis .
 qed

lemma *checked-staged-security-composition-fri-verifier-tied-partial-candidate-bound-from-fresh-prequery-and-missing*

assumes *wf*: staged-budget-wellformed budgets
and *controlled*: staged-adversary-controlled budgets *A*
and *finite-B*: $\bigwedge dg. \text{finite } (B \ dg)$
and *cover*:
 $\bigwedge \text{data attacker-state.}$
 Some (data, attacker-state) $\in$
 set-dist (execute (checked-staged-transcript-program *A*)
 adversary-initial-state) $\implies$
 composition-fri-sampled-layer-chain-cover
 (verifier-state-from-adversary attacker-state
 (staged-proof-transcript data)) bad
and *envelope*:
 $\bigwedge \text{data attacker-state dg composition-table.}$
 Some (data, attacker-state) $\in$
 set-dist (execute (checked-staged-transcript-program *A*)
 adversary-initial-state) $\implies$
 composition-fri-multiround-bad-sets bad dg composition-table
 $\subseteq B \ dg$
and *fresh-bound*:
 wp-event (checked-staged-security-experiment-with-data-state *A*)
 (staged-security-with-data-state-verifier-event
 ($\lambda s. \text{composition-fri-challenge-list-fresh-hit } s \ B$))
 adversary-initial-state $\leq F$
and *missing-bound*:
 $\bigwedge \text{data attacker-state.}$
 Some (data, attacker-state) $\in$
 set-dist (execute (checked-staged-transcript-program *A*)
 adversary-initial-state) $\implies$
 wp-event verify-monad
 (composition-fri-verifier-tied-missing-sampled-layer-chain
 (verifier-state-from-adversary attacker-state
 (staged-proof-transcript data)))
 (verifier-state-from-adversary attacker-state
 (staged-proof-transcript data)) $\leq M$
shows
 wp-event (checked-staged-security-experiment-with-data-state *A*)
 (staged-security-with-data-state-verifier-event
 composition-fri-bad-with-verifier-tied-partial-candidate)
 adversary-initial-state $\leq$
 ($F +$
 hash-relation-budget-value
 ($\sum dg \in (UNIV :: 'f \ \text{set}).$
 card ($B \ dg$) * ceil-log (maxDegree + 1))
 (staged-attacker-query-budget budgets +

$\text{staged-challenge-query-budget})) + M$

proof –

have *sampled*:

wp-event (*checked-staged-security-experiment-with-data-state* *A*)
 (*staged-security-with-data-state-verifier-event*
composition-fri-bad-with-verifier-tied-sampled-layer-chain)
adversary-initial-state $\leq$
 $F +$
hash-relation-budget-value
 $(\sum dg \in (UNIV :: 'f \text{ set}).$
 $\text{card } (B \ dg) * \text{ceil-log } (\text{maxDegree} + 1))$
 $(\text{staged-attacker-query-budget } \text{budgets} +$
 $\text{staged-challenge-query-budget})$

by (*rule*
checked-staged-security-composition-fri-verifier-tied-sampled-layer-chain-bound-from-fresh-and-prequery
 $[OF \ wf \ \text{controlled } \text{finite-}B \ \text{cover } \text{envelope } \text{fresh-bound}]$)

show *?thesis*

by (*rule*
checked-staged-security-composition-fri-verifier-tied-partial-candidate-bound-from-sampled-and-missing
 $[OF \ \text{sampled } \text{missing-bound}]$)

qed

lemma *checked-staged-security-composition-fri-verifier-tied-partial-candidate-bound-from-fresh-prequery-and-assignment*

assumes *wf*: *staged-budget-wellformed* *budgets*
and *controlled*: *staged-adversary-controlled* *budgets* *A*
and *finite-B*: $\bigwedge dg. \text{finite } (B \ dg)$
and *cover*:
 $\bigwedge \text{data } \text{attacker-state}.$
Some (*data*, *attacker-state*) $\in$
 $\text{set-dist } (\text{execute } (\text{checked-staged-transcript-program } A)$
 $\text{adversary-initial-state}) \implies$
composition-fri-sampled-layer-chain-cover
 $(\text{verifier-state-from-adversary } \text{attacker-state}$
 $(\text{staged-proof-transcript } \text{data})) \text{ bad}$

and *envelope*:
 $\bigwedge \text{data } \text{attacker-state } dg \ \text{composition-table}.$
Some (*data*, *attacker-state*) $\in$
 $\text{set-dist } (\text{execute } (\text{checked-staged-transcript-program } A)$
 $\text{adversary-initial-state}) \implies$
 $\text{composition-fri-multiround-bad-sets bad } dg \ \text{composition-table}$
 $\subseteq B \ dg$

and *fresh-bound*:
wp-event (*checked-staged-security-experiment-with-data-state* *A*)
 (*staged-security-with-data-state-verifier-event*
 $(\lambda s. \text{composition-fri-challenge-list-fresh-hit } s \ B))$
adversary-initial-state $\leq F$

and *assignment-bound*:
 $\bigwedge \text{data } \text{attacker-state}.$
Some (*data*, *attacker-state*) $\in$

```

    set-dist (execute (checked-staged-transcript-program A)
      adversary-initial-state)  $\implies$ 
wp-event verify-monad
  (composition-fri-verifier-tied-sampled-layer-assignment-obstruction
    (verifier-state-from-adversary attacker-state
      (staged-proof-transcript data)))
  (verifier-state-from-adversary attacker-state
    (staged-proof-transcript data))  $\leq M$ 
shows
  wp-event (checked-staged-security-experiment-with-data-state A)
    (staged-security-with-data-state-verifier-event
      composition-fri-bad-with-verifier-tied-partial-candidate)
  adversary-initial-state  $\leq$ 
  (F +
    hash-relation-budget-value
      ( $\sum dg \in (UNIV :: 'f \text{ set}).$ 
        card (B dg) * ceil-log (maxDegree + 1))
    (staged-attacker-query-budget budgets +
      staged-challenge-query-budget)) + M
proof (rule
  checked-staged-security-composition-fri-verifier-tied-partial-candidate-bound-from-fresh-prequery-and-missing
    [OF wf controlled finite-B cover envelope fresh-bound])
fix data attacker-state
assume support:
  Some (data, attacker-state)  $\in$ 
    set-dist (execute (checked-staged-transcript-program A)
      adversary-initial-state)
show wp-event verify-monad
  (composition-fri-verifier-tied-missing-sampled-layer-chain
    (verifier-state-from-adversary attacker-state
      (staged-proof-transcript data)))
  (verifier-state-from-adversary attacker-state
    (staged-proof-transcript data))  $\leq M$ 
by (rule
  wp-composition-fri-verifier-tied-missing-sampled-bound-from-assignment-obstruction)
  (rule assignment-bound[OF support])
qed

lemma checked-staged-security-composition-fri-verifier-tied-partial-candidate-bound-from-fresh-prequery-conflict-and-
assumes wf: staged-budget-wellformed budgets
and controlled: staged-adversary-controlled budgets A
and finite-B:  $\bigwedge dg. \text{finite } (B \ dg)$ 
and cover:
 $\bigwedge \text{data attacker-state.}$ 
  Some (data, attacker-state)  $\in$ 
    set-dist (execute (checked-staged-transcript-program A)
      adversary-initial-state)  $\implies$ 
composition-fri-sampled-layer-chain-cover
  (verifier-state-from-adversary attacker-state

```

(staged-proof-transcript data)) bad
and envelope:
 $\wedge$ data attacker-state dg composition-table.
 Some (data, attacker-state) $\in$
 set-dist (execute (checked-staged-transcript-program A)
 adversary-initial-state) $\implies$
 composition-fri-multiround-bad-sets bad dg composition-table
 $\subseteq B$ dg
and fresh-bound:
 wp-event (checked-staged-security-experiment-with-data-state A)
 (staged-security-with-data-state-verifier-event
 (λs . composition-fri-challenge-list-fresh-hit s B))
 adversary-initial-state $\leq F$
and conflict-bound:
 $\wedge$ data attacker-state.
 Some (data, attacker-state) $\in$
 set-dist (execute (checked-staged-transcript-program A)
 adversary-initial-state) $\implies$
 wp-event verify-monad
 (composition-fri-verifier-tied-sampled-assignment-conflict
 (verifier-state-from-adversary attacker-state
 (staged-proof-transcript data)))
 (verifier-state-from-adversary attacker-state
 (staged-proof-transcript data)) $\leq C$
and zero-bound:
 $\wedge$ data attacker-state.
 Some (data, attacker-state) $\in$
 set-dist (execute (checked-staged-transcript-program A)
 adversary-initial-state) $\implies$
 wp-event verify-monad
 (composition-fri-verifier-tied-zero-round-final-obstruction
 (verifier-state-from-adversary attacker-state
 (staged-proof-transcript data)))
 (verifier-state-from-adversary attacker-state
 (staged-proof-transcript data)) $\leq Z$
shows
 wp-event (checked-staged-security-experiment-with-data-state A)
 (staged-security-with-data-state-verifier-event
 composition-fri-bad-with-verifier-tied-partial-candidate)
 adversary-initial-state $\leq$
 (F +
 hash-relation-budget-value
 ($\sum dg \in (UNIV :: 'f \text{ set}).$
 card (B dg) * ceil-log (maxDegree + 1))
 (staged-attacker-query-budget budgets +
 staged-challenge-query-budget)) + (C + Z)
proof (rule
 checked-staged-security-composition-fri-verifier-tied-partial-candidate-bound-from-fresh-prequery-and-assignment-ob
 [OF wf controlled finite-B cover envelope fresh-bound])

fix *data attacker-state*
assume *support*:
Some (data, attacker-state) ∈
set-dist (execute (checked-staged-transcript-program A)
adversary-initial-state)
show *wp-event verify-monad*
(composition-fri-verifier-tied-sampled-layer-assignment-obstruction
(verifier-state-from-adversary attacker-state
(staged-proof-transcript data)))
(verifier-state-from-adversary attacker-state
(staged-proof-transcript data)) ≤ C + Z
by (rule
wp-composition-fri-verifier-tied-assignment-obstruction-bound-from-conflict-and-zero)
(rule conflict-bound[OF support], rule zero-bound[OF support])
qed

lemma *checked-staged-security-composition-fri-verifier-tied-partial-candidate-bound-from-fresh-prequery-and-residual*

assumes *wf: staged-budget-wellformed budgets*
and *controlled: staged-adversary-controlled budgets A*
and *finite-B: $\bigwedge dg. \text{finite } (B \ dg)$*
and *cover*:
 $\bigwedge \text{data attacker-state.}$
Some (data, attacker-state) ∈
set-dist (execute (checked-staged-transcript-program A)
adversary-initial-state) $\implies$
composition-fri-sampled-layer-chain-cover
(verifier-state-from-adversary attacker-state
(staged-proof-transcript data)) bad
and *envelope*:
 $\bigwedge \text{data attacker-state dg composition-table.}$
Some (data, attacker-state) ∈
set-dist (execute (checked-staged-transcript-program A)
adversary-initial-state) $\implies$
composition-fri-multiround-bad-sets bad dg composition-table
 $\subseteq B \ dg$
and *fresh-bound*:
wp-event (checked-staged-security-experiment-with-data-state A)
(staged-security-with-data-state-verifier-event
($\lambda s. \text{composition-fri-challenge-list-fresh-hit } s \ B$))
adversary-initial-state ≤ Fresh
and *slot-bound*:
 $\bigwedge \text{data attacker-state.}$
Some (data, attacker-state) ∈
set-dist (execute (checked-staged-transcript-program A)
adversary-initial-state) $\implies$
wp-event verify-monad
(composition-fri-verifier-tied-authenticated-slot-recorded-chunk-gap
(verifier-state-from-adversary attacker-state
(staged-proof-transcript data)))

$(\text{verifier-state-from-adversary attacker-state} \\ (\text{staged-proof-transcript data})) \leq \text{Slot}$
and merkle-bound:
 $\wedge \text{data attacker-state.}$
 $\text{Some (data, attacker-state)} \in$
 $\text{set-dist (execute (checked-staged-transcript-program A) \\ adversary-initial-state)} \implies$
 $\text{wp-event verify-monad}$
 $(\text{partial-merkle-inconsistency-bad} \\ (\text{verifier-state-from-adversary attacker-state} \\ (\text{staged-proof-transcript data})))$
 $(\text{verifier-state-from-adversary attacker-state} \\ (\text{staged-proof-transcript data})) \leq \text{Merkle}$
and missing-bound:
 $\wedge \text{data attacker-state.}$
 $\text{Some (data, attacker-state)} \in$
 $\text{set-dist (execute (checked-staged-transcript-program A) \\ adversary-initial-state)} \implies$
 $\text{wp-event verify-monad}$
 $(\text{composition-fri-verifier-tied-recorded-base-chunk-auth-missing} \\ (\text{verifier-state-from-adversary attacker-state} \\ (\text{staged-proof-transcript data})))$
 $(\text{verifier-state-from-adversary attacker-state} \\ (\text{staged-proof-transcript data})) \leq \text{Missing}$
and next-bound:
 $\wedge \text{data attacker-state.}$
 $\text{Some (data, attacker-state)} \in$
 $\text{set-dist (execute (checked-staged-transcript-program A) \\ adversary-initial-state)} \implies$
 $\text{wp-event verify-monad}$
 $(\text{composition-fri-verifier-tied-next-value-replay-gap} \\ (\text{verifier-state-from-adversary attacker-state} \\ (\text{staged-proof-transcript data})))$
 $(\text{verifier-state-from-adversary attacker-state} \\ (\text{staged-proof-transcript data})) \leq \text{Next}$
and successor-bound:
 $\wedge \text{data attacker-state.}$
 $\text{Some (data, attacker-state)} \in$
 $\text{set-dist (execute (checked-staged-transcript-program A) \\ adversary-initial-state)} \implies$
 $\text{wp-event verify-monad}$
 $(\text{composition-fri-verifier-tied-successor-replay-gap} \\ (\text{verifier-state-from-adversary attacker-state} \\ (\text{staged-proof-transcript data})))$
 $(\text{verifier-state-from-adversary attacker-state} \\ (\text{staged-proof-transcript data})) \leq \text{Successor}$
and final-bound:
 $\wedge \text{data attacker-state.}$
 $\text{Some (data, attacker-state)} \in$

```

    set-dist (execute (checked-staged-transcript-program A)
      adversary-initial-state)  $\implies$ 
wp-event verify-monad
  (composition-fri-verifier-tied-final-value-replay-gap
    (verifier-state-from-adversary attacker-state
      (staged-proof-transcript data)))
  (verifier-state-from-adversary attacker-state
    (staged-proof-transcript data))  $\leq$  Final
and zero-bound:
 $\wedge$ data attacker-state.
  Some (data, attacker-state)  $\in$ 
    set-dist (execute (checked-staged-transcript-program A)
      adversary-initial-state)  $\implies$ 
wp-event verify-monad
  (composition-fri-verifier-tied-zero-round-final-obstruction
    (verifier-state-from-adversary attacker-state
      (staged-proof-transcript data)))
  (verifier-state-from-adversary attacker-state
    (staged-proof-transcript data))  $\leq$  Zero
shows
wp-event (checked-staged-security-experiment-with-data-state A)
  (staged-security-with-data-state-verifier-event
    composition-fri-bad-with-verifier-tied-partial-candidate)
  adversary-initial-state  $\leq$ 
  (Fresh +
    hash-relation-budget-value
      ( $\sum dg \in (UNIV :: 'f \text{ set}).$ 
        card (B dg) * ceil-log (maxDegree + 1))
      (staged-attacker-query-budget budgets +
        staged-challenge-query-budget)) +
    (((Merkle + Missing) + Next + Successor + Final + Merkle) +
      Slot + Zero)
proof (rule
  checked-staged-security-composition-fri-verifier-tied-partial-candidate-bound-from-fresh-prequery-conflict-and-zero
    [OF wf controlled finite-B cover envelope fresh-bound])
fix data attacker-state
assume support:
  Some (data, attacker-state)  $\in$ 
    set-dist (execute (checked-staged-transcript-program A)
      adversary-initial-state)
let ?s =
  verifier-state-from-adversary attacker-state
    (staged-proof-transcript data)
show wp-event verify-monad
  (composition-fri-verifier-tied-sampled-assignment-conflict ?s) ?s
 $\leq$  ((Merkle + Missing) + Next + Successor + Final + Merkle) + Slot
by (rule
  wp-composition-fri-verifier-tied-sampled-assignment-conflict-bound-from-slot-recorded-chunk-gap-and-residuals)
  (rule slot-bound[OF support],

```



```

    rule merkle-bound[OF support],
    rule missing-bound[OF support],
    rule next-bound[OF support],
    rule successor-bound[OF support],
    rule final-bound[OF support])
  show wp-event verify-monad
    (composition-fri-verifier-tied-zero-round-final-obstruction ?s) ?s ≤ Zero
  by (rule zero-bound[OF support])
qed

end

end

```

```

theory Soundness-FRI-Trace-Candidate-Blocker
  imports Soundness-FRI-Candidate-Transfer-Support
begin

```

Diagnostic normalization facts for the remaining trace candidate-transfer branch. These lemmas do not close the branch; they identify the exact obstacle: the broad reachable witness may use a different authenticated root, query-index list, or opening set than the verifier-consumed/header-tied FRI witness.

```

context soundness
begin

```

```

definition trace-fri-header-tied-low-candidate-untied-reachable-witness-gap

```

```

  :: ('f, 'a) protocol-channel-scheme ⇒
    (unit list × ('f, 'a) protocol-channel-scheme) option ⇒ bool

```

```

where

```

```

  trace-fri-header-tied-low-candidate-untied-reachable-witness-gap s out
  ⇔
  (∃ trace-roots trace-bs trace-final fri-dg composition-roots
    composition-bs composition-final fri-query-idxs trace-round-layers
    composition-round-layers fr as rest trace-openings trace-table
    fr0 query-idxs0 trace-openings0 trace-table0.
    trace-fri-reachable-header-tie-gap s out ∧
    accepted-fri-opening-transcript s out trace-roots trace-bs
    trace-final fri-dg composition-roots composition-bs
    composition-final fri-query-idxs trace-round-layers
    composition-round-layers ∧
    verifier-header-transcript s fr trace-roots trace-final as fri-dg
    composition-roots composition-final rest ∧
    accepted-with-partial-trace-openings s out fr fri-query-idxs
    trace-openings ∧
    partial-trace-table-candidate trace-table trace-openings ∧
    trace-table-low-degree trace-table ∧
    accepted-with-partial-trace-openings s out fr0 query-idxs0
    trace-openings0 ∧

```

$\text{partial-trace-table-candidate } \text{trace-table0 } \text{trace-openings0} \wedge$
 $\neg \text{trace-table-low-degree } \text{trace-table0} \wedge$
 $(\text{fr0} \neq \text{fr} \vee \text{query-idxs0} \neq \text{fri-query-idxs} \vee$
 $\text{trace-openings0} \neq \text{trace-openings}))$

lemma *trace-fri-header-tied-low-candidate-gap-imp-untied-reachable-witness:*

assumes *trace-fri-header-tied-low-candidate-gap s out*

shows

trace-fri-header-tied-low-candidate-untied-reachable-witness-gap s out

proof –

from *assms* **obtain** *trace-roots trace-bs trace-final fri-dg*

composition-roots composition-bs composition-final fri-query-idxs

trace-round-layers composition-round-layers fr as rest trace-openings

trace-table

where *gap: trace-fri-reachable-header-tie-gap s out*

and *fri-openings:*

accepted-fri-opening-transcript s out trace-roots trace-bs

trace-final fri-dg composition-roots composition-bs

composition-final fri-query-idxs trace-round-layers

composition-round-layers

and *header:*

verifier-header-transcript s fr trace-roots trace-final as fri-dg

composition-roots composition-final rest

and *partial:*

accepted-with-partial-trace-openings s out fr fri-query-idxs

trace-openings

and *candidate:*

partial-trace-table-candidate trace-table trace-openings

and *low: trace-table-low-degree trace-table*

unfolding *trace-fri-header-tied-low-candidate-gap-def* **by** *blast*

have *raw: trace-fri-bad-with-reachable-partial-candidate s out*

using *gap* **unfolding** *trace-fri-reachable-header-tie-gap-def* **by** *simp*

from *trace-fri-bad-with-reachable-partial-candidateE[OF raw]*

obtain *fr0 query-idxs0 trace-openings0 trace-table0* **where** *evidence0:*

trace-fri-partial-candidate-evidence s out fr0 query-idxs0

trace-openings0 trace-table0

by *blast*

have *partial0:*

accepted-with-partial-trace-openings s out fr0 query-idxs0

trace-openings0

by *(rule trace-fri-partial-candidate-evidenceD(1)[OF evidence0])*

have *candidate0:*

partial-trace-table-candidate trace-table0 trace-openings0

by *(rule trace-fri-partial-candidate-evidenceD(2)[OF evidence0])*

have *not-low0: $\neg \text{trace-table-low-degree } \text{trace-table0}$*

by *(rule trace-fri-partial-candidate-evidenceD(3)[OF evidence0])*

have *untied:*

fr0 $\neq$ fr $\vee$ query-idxs0 $\neq$ fri-query-idxs $\vee$

trace-openings0 $\neq$ trace-openings

```

proof (rule ccontr)
  assume  $\neg (fr0 \neq fr \vee query\_idxs0 \neq fri\_query\_idxs \vee$ 
     $trace\_openings0 \neq trace\_openings)$ 
  then have eqs:
     $fr0 = fr$ 
     $query\_idxs0 = fri\_query\_idxs$ 
     $trace\_openings0 = trace\_openings$ 
  by auto
  have trace-fri-bad-with-header-tied-partial-candidate s out
  unfolding trace-fri-bad-with-header-tied-partial-candidate-def
  by (intro exI conjI)
    (rule fri-openings, rule header,
      use partial0 candidate0 not-low0 eqs in auto)
  then show False
    using gap unfolding trace-fri-reachable-header-tie-gap-def by simp
qed
show ?thesis
  unfolding
    trace-fri-header-tied-low-candidate-untied-reachable-witness-gap-def
  by (intro exI conjI)
    (rule gap, rule fri-openings, rule header, rule partial,
      rule candidate, rule low, rule partial0, rule candidate0,
      rule not-low0, rule untied)
qed

lemma wp-trace-fri-header-tied-low-candidate-gap-bound-from-untied-witness:
  assumes
    wp-event verify-monad
    (trace-fri-header-tied-low-candidate-untied-reachable-witness-gap s)
     $s \leq T$ 
  shows
    wp-event verify-monad
    (trace-fri-header-tied-low-candidate-gap s)  $s \leq T$ 
  by (rule order-trans[OF - assms])
    (rule wp-event-mono,
      rule trace-fri-header-tied-low-candidate-gap-imp-untied-reachable-witness)

lemma trace-fri-reachable-header-tie-gap-imp-merkle-or-untied-on-support:
  assumes support: out  $\in$  set-dist (execute verify-monad s)
  and gap: trace-fri-reachable-header-tie-gap s out
  shows
    partial-merkle-inconsistency-bad s out  $\vee$ 
    trace-fri-header-tied-low-candidate-untied-reachable-witness-gap s out
proof –
  have split:
    partial-merkle-inconsistency-bad s out  $\vee$ 
    trace-fri-header-tied-candidate-available-gap s out
  by (rule
    trace-fri-reachable-header-tie-gap-imp-merkle-or-candidate-available-on-support

```

$[OF \text{ support gap}]$
then show $?thesis$
proof
 assume $partial\text{-merkle-inconsistency-bad } s \text{ out}$
 then show $?thesis$ **by** $simp$
next
 assume $available: trace\text{-fri-header-tied-candidate-available-gap } s \text{ out}$
 have $low\text{-gap}: trace\text{-fri-header-tied-low-candidate-gap } s \text{ out}$
 by (rule $trace\text{-fri-header-tied-candidate-available-gap-imp-low-candidate-gap}$
 $[OF \text{ available}]$)
 have
 $trace\text{-fri-header-tied-low-candidate-untied-reachable-witness-gap } s \text{ out}$
 by (rule
 $trace\text{-fri-header-tied-low-candidate-gap-imp-untied-reachable-witness}$
 $[OF \text{ low-gap}]$)
 then show $?thesis$ **by** $simp$
qed
qed

lemma $wp\text{-trace-fri-reachable-header-tie-gap-bound-from-merkle-and-untied}$:
assumes $merkle\text{-bound}$:
 $wp\text{-event verify-monad } (partial\text{-merkle-inconsistency-bad } s) \ s \leq M$
and $untied\text{-bound}$:
 $wp\text{-event verify-monad}$
 $(trace\text{-fri-header-tied-low-candidate-untied-reachable-witness-gap } s)$
 $s \leq U$
shows
 $wp\text{-event verify-monad}$
 $(trace\text{-fri-reachable-header-tie-gap } s) \ s \leq M + U$
proof –
 have $split\text{-bound}$:
 $wp\text{-event verify-monad}$
 $(trace\text{-fri-reachable-header-tie-gap } s) \ s \leq$
 $wp\text{-event verify-monad}$
 $(\lambda out. partial\text{-merkle-inconsistency-bad } s \text{ out} \vee$
 $trace\text{-fri-header-tied-low-candidate-untied-reachable-witness-gap}$
 $s \text{ out}) \ s$
 by (rule $wp\text{-event-mono-on-support}$)
 (rule $trace\text{-fri-reachable-header-tie-gap-imp-merkle-or-untied-on-support}$)
 also have $\dots \leq$
 $wp\text{-event verify-monad } (partial\text{-merkle-inconsistency-bad } s) \ s +$
 $wp\text{-event verify-monad}$
 $(trace\text{-fri-header-tied-low-candidate-untied-reachable-witness-gap } s)$
 s
 by (rule $wp\text{-event-union-bound}$)
 also have $\dots \leq M + U$
 by (rule $add\text{-mono}[OF \text{ merkle-bound untied-bound}]$)
 finally show $?thesis$.
qed

```

lemma trace-fri-reachable-partial-candidate-reduction-from-header-merkle-and-untied:
  fixes  $H\ M\ U :: \text{prob}$ 
  assumes header-bound:
     $\text{wp-event verify-monad}$ 
      ( $\text{trace-fri-bad-with-header-tied-partial-candidate } s$ )  $s \leq H$ 
  and merkle-bound:
     $\text{wp-event verify-monad}$  ( $\text{partial-merkle-inconsistency-bad } s$ )  $s \leq M$ 
  and untied-bound:
     $\text{wp-event verify-monad}$ 
      ( $\text{trace-fri-header-tied-low-candidate-untied-reachable-witness-gap } s$ )
       $s \leq U$ 
  and total-bound:  $H + (M + U) \leq \text{trace-fri-error}$ 
  shows trace-fri-reachable-partial-candidate-reduction-assumption  $s$ 
proof –
  have gap-bound:
     $\text{wp-event verify-monad}$ 
      ( $\text{trace-fri-reachable-header-tie-gap } s$ )  $s \leq M + U$ 
  by (rule
     $\text{wp-trace-fri-reachable-header-tie-gap-bound-from-merkle-and-untied}$ 
    [ $\text{OF merkle-bound untied-bound}$ ])
  have reachable-bound:
     $\text{wp-event verify-monad}$ 
      ( $\text{trace-fri-bad-with-reachable-partial-candidate } s$ )  $s \leq H + (M + U)$ 
  by (rule  $\text{wp-trace-fri-reachable-bound-from-header-tied-and-gap}$ 
    [ $\text{OF header-bound gap-bound}$ ])
  show ?thesis
  unfolding trace-fri-reachable-partial-candidate-reduction-assumption-def
  by (rule  $\text{order-trans}$  [ $\text{OF reachable-bound total-bound}$ ])
qed

end

end

```

```

theory Soundness-FRI-Trace-Untied-Cases
  imports Soundness-FRI-Trace-Candidate-Blocker
begin

```

Case split for the remaining trace FRI untied-witness residual.

The broad residual says that the reachable low-degree witness consumed by the verifier and the separate non-low-degree reachable witness disagree in at least one of: authenticated root, query-index list, or sampled openings. This theory exposes these three causes as separate events so subsequent proof layers can bound them independently.

```

context soundness
begin

```

```

definition trace-fri-header-tied-low-candidate-untied-reachable-witness-case

```

$$\begin{aligned}
&:: ('f \Rightarrow 'f \Rightarrow \text{nat list} \Rightarrow \text{nat list} \Rightarrow \\
&\quad 'f \text{ authenticated-opening list list} \Rightarrow \\
&\quad 'f \text{ authenticated-opening list list} \Rightarrow \text{bool}) \Rightarrow \\
&\quad ('f, 'a) \text{ protocol-channel-scheme} \Rightarrow \\
&\quad (\text{unit list} \times ('f, 'a) \text{ protocol-channel-scheme}) \text{ option} \Rightarrow \text{bool}
\end{aligned}$$

where

$$\begin{aligned}
&\text{trace-fri-header-tied-low-candidate-untied-reachable-witness-case } P \text{ s out} \\
&\longleftrightarrow \\
&(\exists \text{ trace-roots trace-bs trace-final fri-dg composition-roots} \\
&\quad \text{composition-bs composition-final fri-query-idxs trace-round-layers} \\
&\quad \text{composition-round-layers fr as rest trace-openings trace-table} \\
&\quad \text{fr0 query-idxs0 trace-openings0 trace-table0}. \\
&\text{trace-fri-reachable-header-tie-gap s out} \wedge \\
&\text{accepted-fri-opening-transcript s out trace-roots trace-bs} \\
&\quad \text{trace-final fri-dg composition-roots composition-bs} \\
&\quad \text{composition-final fri-query-idxs trace-round-layers} \\
&\quad \text{composition-round-layers} \wedge \\
&\text{verifier-header-transcript s fr trace-roots trace-final as fri-dg} \\
&\quad \text{composition-roots composition-final rest} \wedge \\
&\text{accepted-with-partial-trace-openings s out fr fri-query-idxs} \\
&\quad \text{trace-openings} \wedge \\
&\text{partial-trace-table-candidate trace-table trace-openings} \wedge \\
&\text{trace-table-low-degree trace-table} \wedge \\
&\text{accepted-with-partial-trace-openings s out fr0 query-idxs0} \\
&\quad \text{trace-openings0} \wedge \\
&\text{partial-trace-table-candidate trace-table0 trace-openings0} \wedge \\
&\neg \text{trace-table-low-degree trace-table0} \wedge \\
&P \text{ fr0 fr query-idxs0 fri-query-idxs trace-openings0 trace-openings}
\end{aligned}$$

lemma *trace-fri-header-tied-low-candidate-untied-reachable-witness-caseI*:

assumes

$$\begin{aligned}
&\text{trace-fri-reachable-header-tie-gap s out} \\
&\text{accepted-fri-opening-transcript s out trace-roots trace-bs} \\
&\quad \text{trace-final fri-dg composition-roots composition-bs} \\
&\quad \text{composition-final fri-query-idxs trace-round-layers} \\
&\quad \text{composition-round-layers} \\
&\text{verifier-header-transcript s fr trace-roots trace-final as fri-dg} \\
&\quad \text{composition-roots composition-final rest} \\
&\text{accepted-with-partial-trace-openings s out fr fri-query-idxs} \\
&\quad \text{trace-openings} \\
&\text{partial-trace-table-candidate trace-table trace-openings} \\
&\text{trace-table-low-degree trace-table} \\
&\text{accepted-with-partial-trace-openings s out fr0 query-idxs0} \\
&\quad \text{trace-openings0} \\
&\text{partial-trace-table-candidate trace-table0 trace-openings0} \\
&\neg \text{trace-table-low-degree trace-table0} \\
&P \text{ fr0 fr query-idxs0 fri-query-idxs trace-openings0 trace-openings}
\end{aligned}$$

shows

trace-fri-header-tied-low-candidate-untied-reachable-witness-case

$P \ s \ out$
unfolding
 $trace-fri-header-tied-low-candidate-untied-reachable-witness-case-def$
by (intro exI conjI)
(rule assms)+

definition $trace-fri-header-tied-low-candidate-untied-root-gap$
 $:: (f, 'a) \text{ protocol-channel-scheme} \Rightarrow$
 $(unit \ list \times (f, 'a) \text{ protocol-channel-scheme}) \text{ option} \Rightarrow bool$
where
 $trace-fri-header-tied-low-candidate-untied-root-gap \ s \ out$
 $\longleftrightarrow$
 $trace-fri-header-tied-low-candidate-untied-reachable-witness-case$
 $(\lambda fr0 \ fr \ - \ - \ -. \ fr0 \neq fr) \ s \ out$

definition $trace-fri-header-tied-low-candidate-untied-query-gap$
 $:: (f, 'a) \text{ protocol-channel-scheme} \Rightarrow$
 $(unit \ list \times (f, 'a) \text{ protocol-channel-scheme}) \text{ option} \Rightarrow bool$
where
 $trace-fri-header-tied-low-candidate-untied-query-gap \ s \ out$
 $\longleftrightarrow$
 $trace-fri-header-tied-low-candidate-untied-reachable-witness-case$
 $(\lambda - \ query-idxs0 \ fri-query-idxs \ - \ -. \ query-idxs0 \neq fri-query-idxs) \ s \ out$

definition $trace-fri-header-tied-low-candidate-untied-openings-gap$
 $:: (f, 'a) \text{ protocol-channel-scheme} \Rightarrow$
 $(unit \ list \times (f, 'a) \text{ protocol-channel-scheme}) \text{ option} \Rightarrow bool$
where
 $trace-fri-header-tied-low-candidate-untied-openings-gap \ s \ out$
 $\longleftrightarrow$
 $trace-fri-header-tied-low-candidate-untied-reachable-witness-case$
 $(\lambda - \ - \ - \ trace-openings0 \ trace-openings. \ trace-openings0 \neq trace-openings) \ s \ out$

definition $trace-fri-header-tied-low-candidate-untied-normalized-openings-gap$
 $:: (f, 'a) \text{ protocol-channel-scheme} \Rightarrow$
 $(unit \ list \times (f, 'a) \text{ protocol-channel-scheme}) \text{ option} \Rightarrow bool$
where
 $trace-fri-header-tied-low-candidate-untied-normalized-openings-gap \ s \ out$
 $\longleftrightarrow$
 $trace-fri-header-tied-low-candidate-untied-reachable-witness-case$
 $(\lambda fr0 \ fr \ query-idxs0 \ fri-query-idxs \ trace-openings0 \ trace-openings. \ fr0 = fr \wedge query-idxs0 = fri-query-idxs \wedge \ trace-openings0 \neq trace-openings) \ s \ out$

definition $trace-fri-header-tied-low-candidate-untied-nonnormalized-openings-gap$
 $:: (f, 'a) \text{ protocol-channel-scheme} \Rightarrow$
 $(unit \ list \times (f, 'a) \text{ protocol-channel-scheme}) \text{ option} \Rightarrow bool$

where
trace-fri-header-tied-low-candidate-untied-nonnormalized-openings-gap s out
 $\longleftrightarrow$
trace-fri-header-tied-low-candidate-untied-reachable-witness-case
 $(\lambda fr0\ fr\ query\ idxs0\ fri\ query\ idxs\ trace\ openings0\ trace\ openings.$
 $(fr0 \neq fr \vee query\ idxs0 \neq fri\ query\ idxs) \wedge$
 $trace\ openings0 \neq trace\ openings) s\ out$

lemma *trace-fri-header-tied-low-candidate-untied-reachable-witness-gap-imp-cases:*
assumes
trace-fri-header-tied-low-candidate-untied-reachable-witness-gap s out
shows
trace-fri-header-tied-low-candidate-untied-root-gap s out $\vee$
trace-fri-header-tied-low-candidate-untied-query-gap s out $\vee$
trace-fri-header-tied-low-candidate-untied-openings-gap s out

proof –
from *assms* **obtain** *trace-roots trace-bs trace-final fri-dg*
composition-roots composition-bs composition-final fri-query-idxs
trace-round-layers composition-round-layers fr as rest trace-openings
trace-table fr0 query-idxs0 trace-openings0 trace-table0 **where**
gap: trace-fri-reachable-header-tie-gap s out
and *fri-openings:*
accepted-fri-opening-transcript s out trace-roots trace-bs
trace-final fri-dg composition-roots composition-bs
composition-final fri-query-idxs trace-round-layers
composition-round-layers
and *header:*
verifier-header-transcript s fr trace-roots trace-final as fri-dg
composition-roots composition-final rest
and *partial:*
accepted-with-partial-trace-openings s out fr fri-query-idxs
trace-openings
and *candidate:*
partial-trace-table-candidate trace-table trace-openings
and *low: trace-table-low-degree trace-table*
and *partial0:*
accepted-with-partial-trace-openings s out fr0 query-idxs0
trace-openings0
and *candidate0:*
partial-trace-table-candidate trace-table0 trace-openings0
and *not-low0: $\neg$ trace-table-low-degree trace-table0*
and *mismatch:*
 $fr0 \neq fr \vee query\ idxs0 \neq fri\ query\ idxs \vee$
 $trace\ openings0 \neq trace\ openings$
unfolding
trace-fri-header-tied-low-candidate-untied-reachable-witness-gap-def
by *blast*
from *mismatch* **show** *?thesis*
proof


```

assume root-mismatch:  $fr0 \neq fr$ 
have trace-fri-header-tied-low-candidate-untied-root-gap s out
  unfolding trace-fri-header-tied-low-candidate-untied-root-gap-def
  by (rule
    trace-fri-header-tied-low-candidate-untied-reachable-witness-caseI)
    (rule gap, rule fri-openings, rule header, rule partial,
      rule candidate, rule low, rule partial0, rule candidate0,
      rule not-low0, rule root-mismatch)
  then show ?thesis by simp
next
assume tail:
  query-idxs0  $\neq$  fri-query-idxs  $\vee$ 
  trace-openings0  $\neq$  trace-openings
then show ?thesis
proof
  assume query-mismatch: query-idxs0  $\neq$  fri-query-idxs
  have trace-fri-header-tied-low-candidate-untied-query-gap s out
    unfolding trace-fri-header-tied-low-candidate-untied-query-gap-def
    by (rule
      trace-fri-header-tied-low-candidate-untied-reachable-witness-caseI)
      (rule gap, rule fri-openings, rule header, rule partial,
        rule candidate, rule low, rule partial0, rule candidate0,
        rule not-low0, rule query-mismatch)
    then show ?thesis by simp
  next
  assume openings-mismatch:
    trace-openings0  $\neq$  trace-openings
  have trace-fri-header-tied-low-candidate-untied-openings-gap s out
    unfolding trace-fri-header-tied-low-candidate-untied-openings-gap-def
    by (rule
      trace-fri-header-tied-low-candidate-untied-reachable-witness-caseI)
      (rule gap, rule fri-openings, rule header, rule partial,
        rule candidate, rule low, rule partial0, rule candidate0,
        rule not-low0, rule openings-mismatch)
    then show ?thesis by simp
  qed
qed
qed

lemma trace-fri-header-tied-low-candidate-untied-root-gap-imp-witness-gap:
assumes trace-fri-header-tied-low-candidate-untied-root-gap s out
shows
  trace-fri-header-tied-low-candidate-untied-reachable-witness-gap s out
using assms
unfolding
  trace-fri-header-tied-low-candidate-untied-reachable-witness-gap-def
  trace-fri-header-tied-low-candidate-untied-root-gap-def
  trace-fri-header-tied-low-candidate-untied-reachable-witness-case-def
by blast

```

lemma *trace-fri-header-tied-low-candidate-untied-query-gap-imp-witness-gap:*
assumes *trace-fri-header-tied-low-candidate-untied-query-gap s out*
shows
trace-fri-header-tied-low-candidate-untied-reachable-witness-gap s out
using *assms*
unfolding
trace-fri-header-tied-low-candidate-untied-reachable-witness-gap-def
trace-fri-header-tied-low-candidate-untied-query-gap-def
trace-fri-header-tied-low-candidate-untied-reachable-witness-case-def
by *blast*

lemma *trace-fri-header-tied-low-candidate-untied-openings-gap-imp-witness-gap:*
assumes *trace-fri-header-tied-low-candidate-untied-openings-gap s out*
shows
trace-fri-header-tied-low-candidate-untied-reachable-witness-gap s out
using *assms*
unfolding
trace-fri-header-tied-low-candidate-untied-reachable-witness-gap-def
trace-fri-header-tied-low-candidate-untied-openings-gap-def
trace-fri-header-tied-low-candidate-untied-reachable-witness-case-def
by *blast*

lemma *merkle-path-bound-unique-path-if-clean:*
assumes *clean: $\neg$ hash-map-output-collision s*
and *first: merkle-path-bound rt len idx v path s*
and *second: merkle-path-bound rt len idx v path' s*
and *same-length: length path = length path'*
shows *path = path'*
using *first second same-length*
proof (*induction path arbitrary: path' rt len idx*)
case *Nil*
then show *?case by simp*
next
case (*Cons sibling path*)
from *Cons.premis(3)* **obtain** *sibling' path'' where*
path': path' = sibling' # path''
and *lengths: length path = length path''*
by (*cases path'*) *auto*
show *?case*
proof (*cases idx < len div 2*)
case *True*
obtain *child where*
child:
merkle-path-bound child (len div 2) idx v path s
and *first-lookup:*
fmllookup (HashMap s) (MerkleNode child sibling) = Some rt
using *Cons.premis(1) True by auto*
obtain *child' where*

```

  child':
    merkle-path-bound child' (len div 2) idx v path'' s
  and second-lookup:
    fmlookup (HashMap s) (MerkleNode child' sibling') = Some rt
  using Cons.premis(2) True path' by auto
have node-eq:
  MerkleNode child sibling = MerkleNode child' sibling'
proof (rule ccontr)
  assume neg:
    MerkleNode child sibling ≠ MerkleNode child' sibling'
  have hash-map-output-collision s
  by (rule hash-map-output-collisionI
    [OF neg first-lookup second-lookup])
  then show False using clean by contradiction
qed
have child-eq: child = child' using node-eq by simp
have sibling-eq: sibling = sibling' using node-eq by simp
have path-eq: path = path''
  by (rule Cons.IH[OF child - lengths])
  (use child' child-eq in simp)
show ?thesis
  using path' sibling-eq path-eq by simp
next
case False
obtain child where
  child:
    merkle-path-bound child (len div 2) (idx - len div 2)
    v path s
  and first-lookup:
    fmlookup (HashMap s) (MerkleNode sibling child) = Some rt
  using Cons.premis(1) False by auto
obtain child' where
  child':
    merkle-path-bound child' (len div 2) (idx - len div 2)
    v path'' s
  and second-lookup:
    fmlookup (HashMap s) (MerkleNode sibling' child') = Some rt
  using Cons.premis(2) False path' by auto
have node-eq:
  MerkleNode sibling child = MerkleNode sibling' child'
proof (rule ccontr)
  assume neg:
    MerkleNode sibling child ≠ MerkleNode sibling' child'
  have hash-map-output-collision s
  by (rule hash-map-output-collisionI
    [OF neg first-lookup second-lookup])
  then show False using clean by contradiction
qed
have child-eq: child = child' using node-eq by simp

```

```

have sibling-eq: sibling = sibling' using node-eq by simp
have path-eq: path = path''
  by (rule Cons.IH[OF child - lengths])
    (use child' child-eq in simp)
show ?thesis
  using path' sibling-eq path-eq by simp
qed
qed

lemma authenticated-opening-eq-if-clean:
  assumes clean:  $\neg$  hash-map-output-collision s
  and first: authenticated-opening-in s opening
  and second: authenticated-opening-in s opening'
  and same-root: opening-root opening = opening-root opening'
  and same-length: opening-length opening = opening-length opening'
  and same-index: opening-index opening = opening-index opening'
  and same-value: opening-value opening = opening-value opening'
  shows opening = opening'
proof -
  have path-lengths:
    length (opening-path opening) = length (opening-path opening')
  using first second same-length
  unfolding authenticated-opening-in-def by simp
  have first-bound:
    merkle-path-bound
      (opening-root opening)
      (opening-length opening)
      (opening-index opening)
      (opening-value opening)
      (opening-path opening)
    s
  using first unfolding authenticated-opening-in-def by blast
  have second-bound:
    merkle-path-bound
      (opening-root opening)
      (opening-length opening)
      (opening-index opening)
      (opening-value opening)
      (opening-path opening')
    s
  using second same-root same-length same-index same-value
  unfolding authenticated-opening-in-def by simp
  have paths:
    opening-path opening = opening-path opening'
  by (rule merkle-path-bound-unique-path-if-clean
    [OF clean first-bound second-bound path-lengths])
show ?thesis
  using same-root same-length same-index same-value paths
  by (cases opening; cases opening'; simp)

```

qed

lemma *accepted-with-partial-trace-openings-same-root-query-openings-eq-if-clean:*

assumes *partial:*

accepted-with-partial-trace-openings s (Some (result, final-state)) fr
query-idxs trace-openings

and *partial0:*

accepted-with-partial-trace-openings s (Some (result, final-state)) fr
query-idxs trace-openings0

and *clean:* $\neg$ *hash-map-output-collision final-state*

shows *trace-openings0 = trace-openings*

proof –

have *len-openings:* *length trace-openings = rounds*

using *partial* **unfolding** *accepted-with-partial-trace-openings-def* **by** *blast*

have *len-openings0:* *length trace-openings0 = rounds*

using *partial0* **unfolding** *accepted-with-partial-trace-openings-def* **by** *blast*

have *outer-len:* *length trace-openings0 = length trace-openings*

using *len-openings len-openings0* **by** *simp*

have *outer-nth:*

$\bigwedge i. i < \text{length } \text{trace-openings0} \implies$
 $\text{trace-openings0} ! i = \text{trace-openings} ! i$

proof –

fix *i*

assume *i-bound0:* *i < length trace-openings0*

then have *i-bound:* *i < rounds*

using *len-openings0* **by** *simp*

have *idxs:*

map opening-index (trace-openings ! i) =
powers-scaled (query-idxs ! i)

using *partial i-bound*

unfolding *accepted-with-partial-trace-openings-def* **by** *blast*

have *idxs0:*

map opening-index (trace-openings0 ! i) =
powers-scaled (query-idxs ! i)

using *partial0 i-bound*

unfolding *accepted-with-partial-trace-openings-def* **by** *blast*

have *table:*

*partial-authenticated-table fr (scale * clength)*
(trace-openings ! i) final-state

using *partial i-bound*

unfolding *accepted-with-partial-trace-openings-def* **by** *blast*

have *table0:*

*partial-authenticated-table fr (scale * clength)*
(trace-openings0 ! i) final-state

using *partial0 i-bound*

unfolding *accepted-with-partial-trace-openings-def* **by** *blast*

have *len-inner:*

length (trace-openings0 ! i) = length (trace-openings ! i)

using *idxs idxs0* **by** *(metis length-map)*

```

have inner-nth:
   $\bigwedge j. j < \text{length} (\text{trace-openings0} ! i) \implies$ 
     $\text{nth} (\text{nth trace-openings0 } i) j = \text{nth} (\text{nth trace-openings } i) j$ 
proof -
  fix j
  assume j-bound0:  $j < \text{length} (\text{trace-openings0} ! i)$ 
  have j-bound:  $j < \text{length} (\text{trace-openings} ! i)$ 
    using j-bound0 len-inner by simp
  define opn0 where opn0 =  $\text{nth} (\text{nth trace-openings0 } i) j$ 
  define opn where opn =  $\text{nth} (\text{nth trace-openings } i) j$ 
  have opn0-in:  $\text{opn0} \in \text{set} (\text{trace-openings0} ! i)$ 
    unfolding opn0-def using j-bound0 by simp
  have opn-in:  $\text{opn} \in \text{set} (\text{trace-openings} ! i)$ 
    unfolding opn-def using j-bound by simp
  have auth0: authenticated-opening-in final-state opn0
    using table0 opn0-in unfolding partial-authenticated-table-def
    by blast
  have auth: authenticated-opening-in final-state opn
    using table opn-in unfolding partial-authenticated-table-def
    by blast
  have root0: opening-root opn0 = fr
    using table0 opn0-in unfolding partial-authenticated-table-def
    by blast
  have root: opening-root opn = fr
    using table opn-in unfolding partial-authenticated-table-def
    by blast
  have length0: opening-length opn0 = scale * clength
    using table0 opn0-in unfolding partial-authenticated-table-def
    by blast
  have length: opening-length opn = scale * clength
    using table opn-in unfolding partial-authenticated-table-def
    by blast
  have same-index: opening-index opn0 = opening-index opn
proof -
  have map opening-index ( $\text{trace-openings0} ! i$ ) ! j =
    map opening-index ( $\text{trace-openings} ! i$ ) ! j
    using idxs idxs0 by simp
  then show ?thesis
    unfolding opn0-def opn-def using j-bound0 j-bound by simp
qed
have same-value: opening-value opn0 = opening-value opn
proof (rule ccontr)
  assume neq: opening-value opn0  $\neq$  opening-value opn
  have hash-map-output-collision final-state
    by (rule inconsistent-authenticated-openings-imp-hash-collision
      [OF auth0 auth])
    (use root0 root length0 length same-index neq in simp-all)
  then show False using clean by contradiction
qed

```

```

    have opn-eq: opn0 = opn
      by (rule authenticated-opening-eq-if-clean[OF clean auth0 auth])
        (use root0 root length0 length same-index same-value in simp-all)
    show nth (nth trace-openings0 i) j = nth (nth trace-openings i) j
      using opn-eq unfolding opn0-def opn-def by simp
  qed
  show trace-openings0 ! i = trace-openings ! i
    using len-inner inner-nth by (metis nth-equalityI)
  qed
  show ?thesis
    using outer-len outer-nth by (metis nth-equalityI)
  qed

lemma trace-fri-header-tied-low-candidate-untied-normalized-openings-gap-imp-hash-collision:
  assumes
    trace-fri-header-tied-low-candidate-untied-normalized-openings-gap s out
  shows hash-map-output-collision-bad s out
proof -
  from assms obtain trace-roots trace-bs trace-final fri-dg
    composition-roots composition-bs composition-final fri-query-idxs
    trace-round-layers composition-round-layers fr as rest trace-openings
    trace-table fr0 query-idxs0 trace-openings0 trace-table0 where
  partial:
    accepted-with-partial-trace-openings s out fr fri-query-idxs
    trace-openings
  and partial0:
    accepted-with-partial-trace-openings s out fr0 query-idxs0
    trace-openings0
  and root-eq: fr0 = fr
  and query-eq: query-idxs0 = fri-query-idxs
  and openings-neq: trace-openings0 ≠ trace-openings
  unfolding
    trace-fri-header-tied-low-candidate-untied-normalized-openings-gap-def
    trace-fri-header-tied-low-candidate-untied-reachable-witness-case-def
  by blast
  from partial obtain result final-state where out:
    out = Some (result, final-state)
  unfolding accepted-with-partial-trace-openings-def by blast
  show ?thesis
proof (cases hash-map-output-collision final-state)
  case True
  then show ?thesis
    unfolding hash-map-output-collision-bad-def accepted-def
    using out by simp
next
  case clean: False
  have partial-some:
    accepted-with-partial-trace-openings s (Some (result, final-state))
    fr fri-query-idxs trace-openings

```

```

    using partial out by simp
  have partial0-some:
    accepted-with-partial-trace-openings s (Some (result, final-state))
    fr fri-query-idxs trace-openings0
  using partial0 out root-eq query-eq by simp
  have trace-openings0 = trace-openings
  by (rule
    accepted-with-partial-trace-openings-same-root-query-openings-eq-if-clean
    [OF partial-some partial0-some clean])
  then show ?thesis
  using openings-neq by contradiction
qed
qed

lemma trace-fri-header-tied-low-candidate-untied-normalized-openings-gap-mono-hash-collision:
  wp-event verify-monad
  (trace-fri-header-tied-low-candidate-untied-normalized-openings-gap s) s ≤
  wp-event verify-monad (hash-map-output-collision-bad s) s
by (rule wp-event-mono)
  (rule
    trace-fri-header-tied-low-candidate-untied-normalized-openings-gap-imp-hash-collision)

lemma trace-fri-header-tied-low-candidate-untied-openings-gap-imp-normalized-or-nonnormalized:
  assumes trace-fri-header-tied-low-candidate-untied-openings-gap s out
  shows
    trace-fri-header-tied-low-candidate-untied-normalized-openings-gap s out ∨
    trace-fri-header-tied-low-candidate-untied-nonnormalized-openings-gap s out
proof —
  from assms obtain trace-roots trace-bs trace-final fri-dg
    composition-roots composition-bs composition-final fri-query-idxs
    trace-round-layers composition-round-layers fr as rest trace-openings
    trace-table fr0 query-idxs0 trace-openings0 trace-table0 where
    gap: trace-fri-reachable-header-tie-gap s out
  and fri-openings:
    accepted-fri-opening-transcript s out trace-roots trace-bs
    trace-final fri-dg composition-roots composition-bs
    composition-final fri-query-idxs trace-round-layers
    composition-round-layers
  and header:
    verifier-header-transcript s fr trace-roots trace-final as fri-dg
    composition-roots composition-final rest
  and partial:
    accepted-with-partial-trace-openings s out fr fri-query-idxs
    trace-openings
  and candidate:
    partial-trace-table-candidate trace-table trace-openings
  and low: trace-table-low-degree trace-table
  and partial0:
    accepted-with-partial-trace-openings s out fr0 query-idxs0

```



```

    trace-openings0
  and candidate0:
    partial-trace-table-candidate trace-table0 trace-openings0
  and not-low0:  $\neg$  trace-table-low-degree trace-table0
  and openings-neg: trace-openings0  $\neq$  trace-openings
  unfolding trace-fri-header-tied-low-candidate-untied-openings-gap-def
    trace-fri-header-tied-low-candidate-untied-reachable-witness-case-def
  by blast
show ?thesis
proof (cases fr0 = fr  $\wedge$  query-idxs0 = fri-query-idxs)
  case True
  have trace-fri-header-tied-low-candidate-untied-normalized-openings-gap
    s out
  unfolding
    trace-fri-header-tied-low-candidate-untied-normalized-openings-gap-def
  by (rule
    trace-fri-header-tied-low-candidate-untied-reachable-witness-caseI
    [OF gap fri-openings header partial candidate low partial0 candidate0
      not-low0])
    (use True openings-neg in simp)
  then show ?thesis by simp
next
  case False
  then have diff: fr0  $\neq$  fr  $\vee$  query-idxs0  $\neq$  fri-query-idxs
    by blast
  have trace-fri-header-tied-low-candidate-untied-nonnormalized-openings-gap
    s out
  unfolding
    trace-fri-header-tied-low-candidate-untied-nonnormalized-openings-gap-def
  by (rule
    trace-fri-header-tied-low-candidate-untied-reachable-witness-caseI
    [OF gap fri-openings header partial candidate low partial0 candidate0
      not-low0])
    (use diff openings-neg in simp)
  then show ?thesis by simp
qed
qed

lemma wp-trace-fri-header-tied-low-candidate-untied-openings-gap-bound-from-hash-and-nonnormalized:
  fixes hash-err nonnormalized-err :: prob
  assumes hash-bound:
    wp-event verify-monad (hash-map-output-collision-bad s) s  $\leq$ 
      hash-err
  and nonnormalized-bound:
    wp-event verify-monad
      (trace-fri-header-tied-low-candidate-untied-nonnormalized-openings-gap s)
      s  $\leq$  nonnormalized-err
  shows
    wp-event verify-monad

```

$(\text{trace-fri-header-tied-low-candidate-untied-openings-gap } s) \leq$
 $\text{hash-err} + \text{nonnormalized-err}$
proof –
have *wp-event verify-monad*
 $(\text{trace-fri-header-tied-low-candidate-untied-openings-gap } s) \leq$
 $\text{wp-event verify-monad}$
 $(\lambda \text{out.}$
 $\text{trace-fri-header-tied-low-candidate-untied-normalized-openings-gap}$
 $s \text{ out} \vee$
 $\text{trace-fri-header-tied-low-candidate-untied-nonnormalized-openings-gap}$
 $s \text{ out}) s$
by (*rule wp-event-mono*)
 $(\text{rule}$
 $\text{trace-fri-header-tied-low-candidate-untied-openings-gap-imp-normalized-or-nonnormalized})$
also have $\dots \leq$
 $\text{wp-event verify-monad}$
 $(\text{trace-fri-header-tied-low-candidate-untied-normalized-openings-gap } s)$
 $s +$
 $\text{wp-event verify-monad}$
 $(\text{trace-fri-header-tied-low-candidate-untied-nonnormalized-openings-gap}$
 $s) s$
by (*rule wp-event-union-bound*)
also have $\dots \leq \text{hash-err} + \text{nonnormalized-err}$
by (*rule add-mono*)
 $(\text{rule order-trans[}$
 $\text{OF trace-fri-header-tied-low-candidate-untied-normalized-openings-gap-mono-hash-collision}$
 $\text{hash-bound}],$
 $\text{rule nonnormalized-bound})$
finally show *?thesis* .
qed

lemma *trace-fri-header-tied-low-candidate-untied-nonnormalized-openings-gap-imp-root-or-query:*
assumes

$\text{trace-fri-header-tied-low-candidate-untied-nonnormalized-openings-gap}$
 $s \text{ out}$

shows

$\text{trace-fri-header-tied-low-candidate-untied-root-gap } s \text{ out} \vee$
 $\text{trace-fri-header-tied-low-candidate-untied-query-gap } s \text{ out}$

proof –

from *assms obtain* $\text{trace-roots trace-bs trace-final fri-dg}$
 $\text{composition-roots composition-bs composition-final fri-query-idxs}$
 $\text{trace-round-layers composition-round-layers fr as rest trace-openings}$
 $\text{trace-table fr0 query-idxs0 trace-openings0 trace-table0}$ **where**
 $\text{gap: trace-fri-reachable-header-tie-gap } s \text{ out}$
and *fri-openings:*
 $\text{accepted-fri-opening-transcript } s \text{ out trace-roots trace-bs}$
 $\text{trace-final fri-dg composition-roots composition-bs}$
 $\text{composition-final fri-query-idxs trace-round-layers}$
 $\text{composition-round-layers}$

```

and header:
  verifier-header-transcript s fr trace-roots trace-final as fri-dg
    composition-roots composition-final rest
and partial:
  accepted-with-partial-trace-openings s out fr fri-query-idxs
    trace-openings
and candidate:
  partial-trace-table-candidate trace-table trace-openings
and low: trace-table-low-degree trace-table
and partial0:
  accepted-with-partial-trace-openings s out fr0 query-idxs0
    trace-openings0
and candidate0:
  partial-trace-table-candidate trace-table0 trace-openings0
and not-low0:  $\neg$  trace-table-low-degree trace-table0
and mismatch:  $fr0 \neq fr \vee query-idxs0 \neq fri-query-idxs$ 
unfolding
  trace-fri-header-tied-low-candidate-untied-nonnormalized-openings-gap-def
  trace-fri-header-tied-low-candidate-untied-reachable-witness-case-def
by blast
from mismatch show ?thesis
proof
  assume root-mismatch:  $fr0 \neq fr$ 
  have trace-fri-header-tied-low-candidate-untied-root-gap s out
    unfolding trace-fri-header-tied-low-candidate-untied-root-gap-def
    by (rule
      trace-fri-header-tied-low-candidate-untied-reachable-witness-caseI)
      (rule gap, rule fri-openings, rule header, rule partial,
        rule candidate, rule low, rule partial0, rule candidate0,
        rule not-low0, rule root-mismatch)
  then show ?thesis by simp
next
  assume query-mismatch:  $query-idxs0 \neq fri-query-idxs$ 
  have trace-fri-header-tied-low-candidate-untied-query-gap s out
    unfolding trace-fri-header-tied-low-candidate-untied-query-gap-def
    by (rule
      trace-fri-header-tied-low-candidate-untied-reachable-witness-caseI)
      (rule gap, rule fri-openings, rule header, rule partial,
        rule candidate, rule low, rule partial0, rule candidate0,
        rule not-low0, rule query-mismatch)
  then show ?thesis by simp
qed
qed

lemma wp-trace-fri-header-tied-low-candidate-untied-nonnormalized-openings-gap-bound-from-root-query:
  fixes root-err query-err :: prob
  assumes root-bound:
    wp-event verify-monad
    (trace-fri-header-tied-low-candidate-untied-root-gap s)  $s \leq$ 

```

```

    root-err
  and query-bound:
    wp-event verify-monad
      (trace-fri-header-tied-low-candidate-untied-query-gap s) s ≤
    query-err
  shows
    wp-event verify-monad
      (trace-fri-header-tied-low-candidate-untied-nonnormalized-openings-gap s)
      s ≤ root-err + query-err
  proof -
  have wp-event verify-monad
    (trace-fri-header-tied-low-candidate-untied-nonnormalized-openings-gap s)
    s ≤
  wp-event verify-monad
    (λout.
      trace-fri-header-tied-low-candidate-untied-root-gap s out ∨
      trace-fri-header-tied-low-candidate-untied-query-gap s out) s
  by (rule wp-event-mono)
  (rule
    trace-fri-header-tied-low-candidate-untied-nonnormalized-openings-gap-imp-root-or-query)
  also have ... ≤
    wp-event verify-monad
      (trace-fri-header-tied-low-candidate-untied-root-gap s) s +
    wp-event verify-monad
      (trace-fri-header-tied-low-candidate-untied-query-gap s) s
  by (rule wp-event-union-bound)
  also have ... ≤ root-err + query-err
  by (rule add-mono[OF root-bound query-bound])
  finally show ?thesis .
qed

```

lemma *trace-fri-header-tied-low-candidate-untied-root-gap-imp-query-or-openings:*

assumes *trace-fri-header-tied-low-candidate-untied-root-gap s out*

shows

trace-fri-header-tied-low-candidate-untied-query-gap s out ∨
trace-fri-header-tied-low-candidate-untied-openings-gap s out

proof -

from *assms* **obtain** *trace-roots trace-bs trace-final fri-dg*

composition-roots composition-bs composition-final fri-query-idxs

trace-round-layers composition-round-layers fr as rest trace-openings

trace-table fr0 query-idxs0 trace-openings0 trace-table0 **where**

gap: trace-fri-reachable-header-tie-gap s out

and *fri-openings:*

accepted-fri-opening-transcript s out trace-roots trace-bs

trace-final fri-dg composition-roots composition-bs

composition-final fri-query-idxs trace-round-layers

composition-round-layers

and *header:*

verifier-header-transcript s fr trace-roots trace-final as fri-dg

```

    composition-roots composition-final rest
and partial:
    accepted-with-partial-trace-openings s out fr fri-query-idxs
    trace-openings
and candidate:
    partial-trace-table-candidate trace-table trace-openings
and low: trace-table-low-degree trace-table
and partial0:
    accepted-with-partial-trace-openings s out fr0 query-idxs0
    trace-openings0
and candidate0:
    partial-trace-table-candidate trace-table0 trace-openings0
and not-low0:  $\neg$  trace-table-low-degree trace-table0
and root-mismatch: fr0  $\neq$  fr
unfolding trace-fri-header-tied-low-candidate-untied-root-gap-def
    trace-fri-header-tied-low-candidate-untied-reachable-witness-case-def
by blast
show ?thesis
proof (cases query-idxs0  $\neq$  fri-query-idxs)
  case True
  have trace-fri-header-tied-low-candidate-untied-query-gap s out
    unfolding trace-fri-header-tied-low-candidate-untied-query-gap-def
    by (rule
      trace-fri-header-tied-low-candidate-untied-reachable-witness-caseI)
    (rule gap, rule fri-openings, rule header, rule partial,
      rule candidate, rule low, rule partial0, rule candidate0,
      rule not-low0, rule True)
  then show ?thesis by simp
next
  case query-same: False
  show ?thesis
  proof (cases trace-openings0  $\neq$  trace-openings)
    case True
    have trace-fri-header-tied-low-candidate-untied-openings-gap s out
      unfolding trace-fri-header-tied-low-candidate-untied-openings-gap-def
      by (rule
        trace-fri-header-tied-low-candidate-untied-reachable-witness-caseI)
        (rule gap, rule fri-openings, rule header, rule partial,
          rule candidate, rule low, rule partial0, rule candidate0,
          rule not-low0, rule True)
    then show ?thesis by simp
  next
  case openings-same: False
  have same-query: query-idxs0 = fri-query-idxs
    using query-same by simp
  have same-openings: trace-openings0 = trace-openings
    using openings-same by simp
  have len-openings: length trace-openings = rounds
    using partial unfolding accepted-with-partial-trace-openings-def

```

```

    by blast
have idxs0:
  map opening-index (trace-openings ! 0) =
    powers-scaled (fri-query-idxs ! 0)
  using partial rounds-positive
  unfolding accepted-with-partial-trace-openings-def by blast
have nonempty: trace-openings ! 0 ≠ []
proof -
  have powers-scaled (fri-query-idxs ! 0) ≠ []
    unfolding powers-scaled-def using powers-pos by simp
  then show ?thesis
    using idxs0 by auto
qed
then obtain opn where opn-in: opn ∈ set (trace-openings ! 0)
  by (cases trace-openings ! 0) auto
have table:
  partial-authenticated-table fr (scale * clength)
    (trace-openings ! 0) (snd (the out))
proof -
  from partial obtain result final-state where
    out: out = Some (result, final-state)
  and table':
    partial-authenticated-table fr (scale * clength)
      (trace-openings ! 0) final-state
  using rounds-positive
  unfolding accepted-with-partial-trace-openings-def by blast
  then show ?thesis by simp
qed
have table0:
  partial-authenticated-table fr0 (scale * clength)
    (trace-openings ! 0) (snd (the out))
proof -
  from partial0 obtain result final-state where
    out: out = Some (result, final-state)
  and table':
    partial-authenticated-table fr0 (scale * clength)
      (trace-openings0 ! 0) final-state
  using rounds-positive
  unfolding accepted-with-partial-trace-openings-def by blast
  then show ?thesis
    using same-openings by simp
qed
have opening-root opn = fr
  using table opn-in unfolding partial-authenticated-table-def by blast
moreover have opening-root opn = fr0
  using table0 opn-in unfolding partial-authenticated-table-def by blast
ultimately have fr0 = fr by simp
then show ?thesis
  using root-mismatch by contradiction

```

```

    qed
  qed
qed

lemma trace-fri-header-tied-low-candidate-untied-gap-imp-query-or-openings:
  assumes
    trace-fri-header-tied-low-candidate-untied-reachable-witness-gap s out
  shows
    trace-fri-header-tied-low-candidate-untied-query-gap s out ∨
    trace-fri-header-tied-low-candidate-untied-openings-gap s out
proof -
  have cases:
    trace-fri-header-tied-low-candidate-untied-root-gap s out ∨
    trace-fri-header-tied-low-candidate-untied-query-gap s out ∨
    trace-fri-header-tied-low-candidate-untied-openings-gap s out
  by (rule
    trace-fri-header-tied-low-candidate-untied-reachable-witness-gap-imp-cases
    [OF assms])
  then show ?thesis
proof
  assume trace-fri-header-tied-low-candidate-untied-root-gap s out
  then show ?thesis
  by (rule
    trace-fri-header-tied-low-candidate-untied-root-gap-imp-query-or-openings)
next
  assume
    trace-fri-header-tied-low-candidate-untied-query-gap s out ∨
    trace-fri-header-tied-low-candidate-untied-openings-gap s out
  then show ?thesis .
qed
qed

lemma trace-untied-powers-scaled-eq-imp-eq:
  assumes powers-scaled idx = powers-scaled idx'
  shows idx = idx'
proof -
  have left: powers-scaled idx ! 0 = idx
    unfolding powers-scaled-def using powers-pos by simp
  have right: powers-scaled idx' ! 0 = idx'
    unfolding powers-scaled-def using powers-pos by simp
  have powers-scaled idx ! 0 = powers-scaled idx' ! 0
    using assms by simp
  then show ?thesis
    using left right by simp
qed

lemma trace-fri-header-tied-low-candidate-untied-query-gap-imp-openings:
  assumes trace-fri-header-tied-low-candidate-untied-query-gap s out
  shows trace-fri-header-tied-low-candidate-untied-openings-gap s out

```

proof –

from *assms* **obtain** *trace-roots trace-bs trace-final fri-dg*
composition-roots composition-bs composition-final fri-query-idxs
trace-round-layers composition-round-layers fr as rest trace-openings
trace-table fr0 query-idxs0 trace-openings0 trace-table0 **where**
gap: trace-fri-reachable-header-tie-gap s out
and *fri-openings:*
accepted-fri-opening-transcript s out trace-roots trace-bs
trace-final fri-dg composition-roots composition-bs
composition-final fri-query-idxs trace-round-layers
composition-round-layers
and *header:*
verifier-header-transcript s fr trace-roots trace-final as fri-dg
composition-roots composition-final rest
and *partial:*
accepted-with-partial-trace-openings s out fr fri-query-idxs
trace-openings
and *candidate:*
partial-trace-table-candidate trace-table trace-openings
and *low: trace-table-low-degree trace-table*
and *partial0:*
accepted-with-partial-trace-openings s out fr0 query-idxs0
trace-openings0
and *candidate0:*
partial-trace-table-candidate trace-table0 trace-openings0
and *not-low0: $\neg$ trace-table-low-degree trace-table0*
and *query-mismatch: query-idxs0 $\neq$ fri-query-idxs*
unfolding *trace-fri-header-tied-low-candidate-untied-query-gap-def*
trace-fri-header-tied-low-candidate-untied-reachable-witness-case-def
by *blast*
show *?thesis*
proof (*cases trace-openings0 $\neq$ trace-openings*)
case *True*
show *?thesis*
unfolding *trace-fri-header-tied-low-candidate-untied-openings-gap-def*
by (*rule*
trace-fri-header-tied-low-candidate-untied-reachable-witness-caseI)
(rule gap, rule fri-openings, rule header, rule partial,
rule candidate, rule low, rule partial0, rule candidate0,
rule not-low0, rule True)
next
case *openings-same: False*
have *same-openings: trace-openings0 = trace-openings*
using *openings-same* **by** *simp*
have *len-query: length fri-query-idxs = rounds*
using *partial unfolding accepted-with-partial-trace-openings-def*
by *blast*
have *len-query0: length query-idxs0 = rounds*
using *partial0 unfolding accepted-with-partial-trace-openings-def*


```

    by blast
have nth-round-eq:
   $\bigwedge i. i < \text{rounds} \implies \text{query-idxs0} ! i = \text{fri-query-idxs} ! i$ 
proof -
  fix i
  assume i-bound:  $i < \text{rounds}$ 
  have idxs:
    map opening-index (trace-openings ! i) =
      powers-scaled (fri-query-idxs ! i)
  using partial i-bound
  unfolding accepted-with-partial-trace-openings-def by blast
  have idxs0:
    map opening-index (trace-openings0 ! i) =
      powers-scaled (query-idxs0 ! i)
  using partial0 i-bound
  unfolding accepted-with-partial-trace-openings-def by blast
  have powers-scaled (query-idxs0 ! i) =
    powers-scaled (fri-query-idxs ! i)
  using idxs idxs0 same-openings by simp
  then show query-idxs0 ! i = fri-query-idxs ! i
    by (rule trace-untied-powers-scaled-eq-imp-eq)
qed
have len-eq: length query-idxs0 = length fri-query-idxs
  using len-query len-query0 by simp
have query-idxs0 = fri-query-idxs
proof (rule nth-equalityI)
  show length query-idxs0 = length fri-query-idxs
    by (rule len-eq)
next
  fix i
  assume  $i < \text{length query-idxs0}$ 
  then have  $i < \text{rounds}$ 
    using len-query0 by simp
  then show query-idxs0 ! i = fri-query-idxs ! i
    by (rule nth-round-eq)
qed
then show ?thesis
  using query-mismatch by contradiction
qed
qed

lemma trace-fri-header-tied-low-candidate-untied-gap-imp-openings:
  assumes
    trace-fri-header-tied-low-candidate-untied-reachable-witness-gap s out
  shows
    trace-fri-header-tied-low-candidate-untied-openings-gap s out
proof -
  have split:
    trace-fri-header-tied-low-candidate-untied-query-gap s out  $\vee$ 

```

```

    trace-fri-header-tied-low-candidate-untied-openings-gap s out
  by (rule
    trace-fri-header-tied-low-candidate-untied-gap-imp-query-or-openings
    [OF assms])
  then show ?thesis
  proof
    assume trace-fri-header-tied-low-candidate-untied-query-gap s out
    then show ?thesis
      by (rule
        trace-fri-header-tied-low-candidate-untied-query-gap-imp-openings)
  next
    assume trace-fri-header-tied-low-candidate-untied-openings-gap s out
    then show ?thesis .
  qed
qed

lemma trace-fri-header-tied-low-candidate-untied-gap-imp-hash-root-or-query:
  assumes
    trace-fri-header-tied-low-candidate-untied-reachable-witness-gap s out
  shows
    hash-map-output-collision-bad s out  $\vee$ 
    trace-fri-header-tied-low-candidate-untied-root-gap s out  $\vee$ 
    trace-fri-header-tied-low-candidate-untied-query-gap s out
  proof -
    have openings:
      trace-fri-header-tied-low-candidate-untied-openings-gap s out
    by (rule trace-fri-header-tied-low-candidate-untied-gap-imp-openings
      [OF assms])
    have normalized-or:
      trace-fri-header-tied-low-candidate-untied-normalized-openings-gap
        s out  $\vee$ 
      trace-fri-header-tied-low-candidate-untied-nonnormalized-openings-gap
        s out
    by (rule
      trace-fri-header-tied-low-candidate-untied-openings-gap-imp-normalized-or-nonnormalized
      [OF openings])
    then show ?thesis
    proof
      assume
        trace-fri-header-tied-low-candidate-untied-normalized-openings-gap
          s out
      then have hash-map-output-collision-bad s out
        by (rule
          trace-fri-header-tied-low-candidate-untied-normalized-openings-gap-imp-hash-collision)
      then show ?thesis by simp
    next
      assume
        trace-fri-header-tied-low-candidate-untied-nonnormalized-openings-gap
          s out
  
```

```

then show ?thesis
using
  trace-fri-header-tied-low-candidate-untied-nonnormalized-openings-gap-imp-root-or-query
by blast
qed
qed

lemma wp-trace-fri-header-tied-low-candidate-untied-gap-bound-from-cases:
fixes root-err query-err opening-err :: prob
assumes root-bound:
  wp-event verify-monad
  (trace-fri-header-tied-low-candidate-untied-root-gap s) s ≤
  root-err
and query-bound:
  wp-event verify-monad
  (trace-fri-header-tied-low-candidate-untied-query-gap s) s ≤
  query-err
and openings-bound:
  wp-event verify-monad
  (trace-fri-header-tied-low-candidate-untied-openings-gap s) s ≤
  opening-err
shows
  wp-event verify-monad
  (trace-fri-header-tied-low-candidate-untied-reachable-witness-gap s)
  s ≤ root-err + (query-err + opening-err)
proof –
have split-bound:
  wp-event verify-monad
  (trace-fri-header-tied-low-candidate-untied-reachable-witness-gap s)
  s ≤
  wp-event verify-monad
  (λout.
    trace-fri-header-tied-low-candidate-untied-root-gap s out ∨
    trace-fri-header-tied-low-candidate-untied-query-gap s out ∨
    trace-fri-header-tied-low-candidate-untied-openings-gap s out)
  s
by (rule wp-event-mono)
  (rule
    trace-fri-header-tied-low-candidate-untied-reachable-witness-gap-imp-cases)
also have ... ≤
  wp-event verify-monad
  (trace-fri-header-tied-low-candidate-untied-root-gap s) s +
  wp-event verify-monad
  (λout.
    trace-fri-header-tied-low-candidate-untied-query-gap s out ∨
    trace-fri-header-tied-low-candidate-untied-openings-gap s out)
  s
by (rule wp-event-union-bound)
also have ... ≤

```

```

    wp-event verify-monad
      (trace-fri-header-tied-low-candidate-untied-root-gap s) s +
    (wp-event verify-monad
      (trace-fri-header-tied-low-candidate-untied-query-gap s) s +
    wp-event verify-monad
      (trace-fri-header-tied-low-candidate-untied-openings-gap s) s)
  by (intro add-mono order-reft wp-event-union-bound)
also have ... ≤ root-err + (query-err + opening-err)
  by (intro add-mono root-bound query-bound openings-bound)
finally show ?thesis .
qed

lemma wp-trace-fri-header-tied-low-candidate-untied-gap-bound-from-query-and-openings:
  fixes query-err opening-err :: prob
  assumes query-bound:
    wp-event verify-monad
      (trace-fri-header-tied-low-candidate-untied-query-gap s) s ≤
      query-err
  and openings-bound:
    wp-event verify-monad
      (trace-fri-header-tied-low-candidate-untied-openings-gap s) s ≤
      opening-err
  shows
    wp-event verify-monad
      (trace-fri-header-tied-low-candidate-untied-reachable-witness-gap s)
      s ≤ query-err + opening-err
proof -
  have split-bound:
    wp-event verify-monad
      (trace-fri-header-tied-low-candidate-untied-reachable-witness-gap s)
      s ≤
    wp-event verify-monad
      (λout.
        trace-fri-header-tied-low-candidate-untied-query-gap s out ∨
        trace-fri-header-tied-low-candidate-untied-openings-gap s out)
      s
  by (rule wp-event-mono)
  (rule trace-fri-header-tied-low-candidate-untied-gap-imp-query-or-openings)
also have ... ≤
  wp-event verify-monad
    (trace-fri-header-tied-low-candidate-untied-query-gap s) s +
  wp-event verify-monad
    (trace-fri-header-tied-low-candidate-untied-openings-gap s) s
  by (rule wp-event-union-bound)
also have ... ≤ query-err + opening-err
  by (rule add-mono[OF query-bound openings-bound])
finally show ?thesis .
qed

```

lemma *wp-trace-fri-header-tied-low-candidate-untied-gap-bound-from-openings:*
fixes *opening-err :: prob*
assumes *openings-bound:*
 wp-event verify-monad
 (trace-fri-header-tied-low-candidate-untied-openings-gap s) s ≤
 opening-err
shows
 wp-event verify-monad
 (trace-fri-header-tied-low-candidate-untied-reachable-witness-gap s)
 s ≤ opening-err
by (*rule order-trans[OF - openings-bound]*)
 (*rule wp-event-mono,*
 rule trace-fri-header-tied-low-candidate-untied-gap-imp-openings)

lemma *trace-fri-reachable-partial-candidate-reduction-from-header-merkle-and-untied-cases:*
fixes *header-err merkle-err root-err query-err opening-err :: prob*
assumes *header-bound:*
 wp-event verify-monad
 (trace-fri-bad-with-header-tied-partial-candidate s) s ≤
 header-err
and *merkle-bound:*
 wp-event verify-monad (partial-merkle-inconsistency-bad s) s ≤
 merkle-err
and *root-bound:*
 wp-event verify-monad
 (trace-fri-header-tied-low-candidate-untied-root-gap s) s ≤
 root-err
and *query-bound:*
 wp-event verify-monad
 (trace-fri-header-tied-low-candidate-untied-query-gap s) s ≤
 query-err
and *openings-bound:*
 wp-event verify-monad
 (trace-fri-header-tied-low-candidate-untied-openings-gap s) s ≤
 opening-err
and *total-bound:*
 header-err + (merkle-err + (root-err + (query-err + opening-err)))
 ≤ trace-fri-error
shows *trace-fri-reachable-partial-candidate-reduction-assumption s*
proof –
have *untied-bound:*
 wp-event verify-monad
 (trace-fri-header-tied-low-candidate-untied-reachable-witness-gap s)
 s ≤ root-err + (query-err + opening-err)
by (*rule*
 wp-trace-fri-header-tied-low-candidate-untied-gap-bound-from-cases
 [OF root-bound query-bound openings-bound])
have *reduction:*
 trace-fri-reachable-partial-candidate-reduction-assumption s

```

    by (rule
      trace-fri-reachable-partial-candidate-reduction-from-header-merkle-and-untied
      [OF header-bound merkle-bound untied-bound])
    (rule total-bound)
  show ?thesis
    by (rule reduction)
qed

```

lemma *trace-fri-reachable-partial-candidate-reduction-from-header-merkle-and-untied-openings:*

fixes *header-err merkle-err opening-err :: prob*

assumes *header-bound:*

wp-event verify-monad

$(\text{trace-fri-bad-with-header-tied-partial-candidate } s) \leq$
header-err

and *merkle-bound:*

wp-event verify-monad (partial-merkle-inconsistency-bad s) s ≤
merkle-err

and *openings-bound:*

wp-event verify-monad

$(\text{trace-fri-header-tied-low-candidate-untied-openings-gap } s) \leq$
opening-err

and *total-bound:*

header-err + (merkle-err + opening-err) ≤ trace-fri-error

shows *trace-fri-reachable-partial-candidate-reduction-assumption s*

proof –

have *untied-bound:*

wp-event verify-monad

$(\text{trace-fri-header-tied-low-candidate-untied-reachable-witness-gap } s)$
 $s \leq \text{opening-err}$

by (rule

wp-trace-fri-header-tied-low-candidate-untied-gap-bound-from-openings
 [OF openings-bound])

have *reduction:*

trace-fri-reachable-partial-candidate-reduction-assumption s

by (rule

trace-fri-reachable-partial-candidate-reduction-from-header-merkle-and-untied
 [OF header-bound merkle-bound untied-bound])
 (rule total-bound)

show ?thesis

by (rule reduction)

qed

lemma *trace-fri-reachable-partial-candidate-reduction-from-header-merkle-hash-and-nonnormalized-openings:*

fixes *header-err merkle-err hash-err nonnormalized-err :: prob*

assumes *header-bound:*

wp-event verify-monad

$(\text{trace-fri-bad-with-header-tied-partial-candidate } s) \leq$
header-err

and *merkle-bound:*

$wp\text{-event } verify\text{-monad } (partial\text{-merkle-inconsistency-bad } s) \ s \leq$
 $merkle\text{-err}$
and $hash\text{-bound}$:
 $wp\text{-event } verify\text{-monad } (hash\text{-map-output-collision-bad } s) \ s \leq$
 $hash\text{-err}$
and $nonnormalized\text{-bound}$:
 $wp\text{-event } verify\text{-monad}$
 $(trace\text{-fri-header-tied-low-candidate-untied-nonnormalized-openings-gap } s)$
 $s \leq nonnormalized\text{-err}$
and $total\text{-bound}$:
 $header\text{-err} + (merkle\text{-err} + (hash\text{-err} + nonnormalized\text{-err})) \leq$
 $trace\text{-fri-error}$
shows $trace\text{-fri-reachable-partial-candidate-reduction-assumption } s$
proof –
have $openings\text{-bound}$:
 $wp\text{-event } verify\text{-monad}$
 $(trace\text{-fri-header-tied-low-candidate-untied-openings-gap } s) \ s \leq$
 $hash\text{-err} + nonnormalized\text{-err}$
by (rule
 $wp\text{-trace-fri-header-tied-low-candidate-untied-openings-gap-bound-from-hash-and-nonnormalized}$
 $[OF \ hash\text{-bound} \ nonnormalized\text{-bound}])$
show $?thesis$
by (rule
 $trace\text{-fri-reachable-partial-candidate-reduction-from-header-merkle-and-untied-openings}$
 $[OF \ header\text{-bound} \ merkle\text{-bound} \ openings\text{-bound}])$
 $(rule \ total\text{-bound})$
qed

lemma $trace\text{-fri-reachable-partial-candidate-reduction-from-header-merkle-hash-root-query}$:
fixes $header\text{-err} \ merkle\text{-err} \ hash\text{-err} \ root\text{-err} \ query\text{-err} :: prob$
assumes $header\text{-bound}$:
 $wp\text{-event } verify\text{-monad}$
 $(trace\text{-fri-bad-with-header-tied-partial-candidate } s) \ s \leq$
 $header\text{-err}$
and $merkle\text{-bound}$:
 $wp\text{-event } verify\text{-monad } (partial\text{-merkle-inconsistency-bad } s) \ s \leq$
 $merkle\text{-err}$
and $hash\text{-bound}$:
 $wp\text{-event } verify\text{-monad } (hash\text{-map-output-collision-bad } s) \ s \leq$
 $hash\text{-err}$
and $root\text{-bound}$:
 $wp\text{-event } verify\text{-monad}$
 $(trace\text{-fri-header-tied-low-candidate-untied-root-gap } s) \ s \leq$
 $root\text{-err}$
and $query\text{-bound}$:
 $wp\text{-event } verify\text{-monad}$
 $(trace\text{-fri-header-tied-low-candidate-untied-query-gap } s) \ s \leq$
 $query\text{-err}$
and $total\text{-bound}$:

$$\text{header-err} + (\text{merkle-err} + (\text{hash-err} + (\text{root-err} + \text{query-err}))) \leq \text{trace-fri-error}$$

shows *trace-fri-reachable-partial-candidate-reduction-assumption s*

proof –

have *nonnormalized-bound*:

wp-event verify-monad
(trace-fri-header-tied-low-candidate-untied-nonnormalized-openings-gap s)
 $s \leq \text{root-err} + \text{query-err}$

by (rule
wp-trace-fri-header-tied-low-candidate-untied-nonnormalized-openings-gap-bound-from-root-query
 [OF *root-bound query-bound*])

show *?thesis*

by (rule
trace-fri-reachable-partial-candidate-reduction-from-header-merkle-hash-and-nonnormalized-openings
 [OF *header-bound merkle-bound hash-bound nonnormalized-bound*])
 (rule *total-bound*)

qed

lemma *wp-trace-fri-header-tied-low-candidate-untied-gap-bound-from-hash-root-query*:

fixes *hash-err root-err query-err :: prob*

assumes *hash-bound*:

wp-event verify-monad (hash-map-output-collision-bad s) s $\leq$
hash-err

and *root-bound*:

wp-event verify-monad
(trace-fri-header-tied-low-candidate-untied-root-gap s) s $\leq$
root-err

and *query-bound*:

wp-event verify-monad
(trace-fri-header-tied-low-candidate-untied-query-gap s) s $\leq$
query-err

shows

wp-event verify-monad
(trace-fri-header-tied-low-candidate-untied-reachable-witness-gap s)
 $s \leq \text{hash-err} + (\text{root-err} + \text{query-err})$

proof –

have *nonnormalized-bound*:

wp-event verify-monad
(trace-fri-header-tied-low-candidate-untied-nonnormalized-openings-gap s)
 $s \leq \text{root-err} + \text{query-err}$

by (rule
wp-trace-fri-header-tied-low-candidate-untied-nonnormalized-openings-gap-bound-from-root-query
 [OF *root-bound query-bound*])

have *openings-bound*:

wp-event verify-monad
(trace-fri-header-tied-low-candidate-untied-openings-gap s) s $\leq$
 $\text{hash-err} + (\text{root-err} + \text{query-err})$

by (rule
wp-trace-fri-header-tied-low-candidate-untied-openings-gap-bound-from-hash-and-nonnormalized


```

    [OF hash-bound nonnormalized-bound])
  show ?thesis
  by (rule
    wp-trace-fri-header-tied-low-candidate-untied-gap-bound-from-openings
    [OF openings-bound])
qed

lemma trace-fri-reachable-partial-candidate-reduction-from-header-merkle-and-untied-query-openings:
  fixes header-err merkle-err query-err opening-err :: prob
  assumes header-bound:
    wp-event verify-monad
      (trace-fri-bad-with-header-tied-partial-candidate s) s ≤
      header-err
  and merkle-bound:
    wp-event verify-monad (partial-merkle-inconsistency-bad s) s ≤
    merkle-err
  and query-bound:
    wp-event verify-monad
      (trace-fri-header-tied-low-candidate-untied-query-gap s) s ≤
      query-err
  and openings-bound:
    wp-event verify-monad
      (trace-fri-header-tied-low-candidate-untied-openings-gap s) s ≤
      opening-err
  and total-bound:
    header-err + (merkle-err + (query-err + opening-err))
    ≤ trace-fri-error
  shows trace-fri-reachable-partial-candidate-reduction-assumption s
proof -
  have untied-bound:
    wp-event verify-monad
      (trace-fri-header-tied-low-candidate-untied-reachable-witness-gap s)
      s ≤ query-err + opening-err
  by (rule
    wp-trace-fri-header-tied-low-candidate-untied-gap-bound-from-query-and-openings
    [OF query-bound openings-bound])
  have reduction:
    trace-fri-reachable-partial-candidate-reduction-assumption s
  by (rule
    trace-fri-reachable-partial-candidate-reduction-from-header-merkle-and-untied
    [OF header-bound merkle-bound untied-bound])
    (rule total-bound)
  show ?thesis
  by (rule reduction)
qed

end

end

```

theory *Soundness-FRI-Trace-Untied-Active*

imports

Soundness-FRI-Active-Reductions

Soundness-FRI-Trace-Untied-Cases

begin

Active-level packaging for the trace FRI untied-witness blocker. This theory is intentionally downstream of the large active-reduction layer so that the new diagnostic event can be used without growing that file further.

context *soundness*

begin

lemma *active-trace-fri-reachable-reduction-from-header-merkle-and-untied:*

fixes *Ht Mt Ut ::*

('f staged-proof-data $\Rightarrow$ 'f protocol-channel $\Rightarrow$ prob)

assumes *trace-header-bound:*

$\wedge$ *data attacker-state.*

Some (data, attacker-state) $\in$

set-dist (execute (checked-staged-transcript-program A)

adversary-initial-state) $\Rightarrow$

wp-event verify-monad

(trace-fri-bad-with-header-tied-partial-candidate

(verifier-state-from-adversary attacker-state

(staged-proof-transcript data)))

(verifier-state-from-adversary attacker-state

(staged-proof-transcript data)) $\leq$

Ht data attacker-state

and *trace-merkle-bound:*

$\wedge$ *data attacker-state.*

Some (data, attacker-state) $\in$

set-dist (execute (checked-staged-transcript-program A)

adversary-initial-state) $\Rightarrow$

wp-event verify-monad

(partial-merkle-inconsistency-bad

(verifier-state-from-adversary attacker-state

(staged-proof-transcript data)))

(verifier-state-from-adversary attacker-state

(staged-proof-transcript data)) $\leq$

Mt data attacker-state

and *trace-untied-bound:*

$\wedge$ *data attacker-state.*

Some (data, attacker-state) $\in$

set-dist (execute (checked-staged-transcript-program A)

adversary-initial-state) $\Rightarrow$

wp-event verify-monad

(trace-fri-header-tied-low-candidate-untied-reachable-witness-gap

(verifier-state-from-adversary attacker-state

$(\text{staged-proof-transcript data}))$
 $(\text{verifier-state-from-adversary attacker-state}$
 $(\text{staged-proof-transcript data})) \leq$
 $Ut \text{ data attacker-state}$
and *total-bound*:
 $\wedge \text{data attacker-state.}$
 $\text{Some (data, attacker-state)} \in$
 $\text{set-dist (execute (checked-staged-transcript-program A)}$
 $\text{adversary-initial-state)} \implies$
 $Ht \text{ data attacker-state} +$
 $(Mt \text{ data attacker-state} + Ut \text{ data attacker-state}) \leq$
 trace-fri-error
and *support*:
 $\text{Some (data, attacker-state)} \in$
 $\text{set-dist (execute (checked-staged-transcript-program A)}$
 $\text{adversary-initial-state})$
shows
 $\text{trace-fri-reachable-partial-candidate-reduction-assumption}$
 $(\text{verifier-state-from-adversary attacker-state}$
 $(\text{staged-proof-transcript data}))$
by (*rule*
 $\text{trace-fri-reachable-partial-candidate-reduction-from-header-merkle-and-untied})$
 $(\text{rule trace-header-bound[OF support],}$
 $\text{rule trace-merkle-bound[OF support],}$
 $\text{rule trace-untied-bound[OF support],}$
 $\text{rule total-bound[OF support]})$

lemma *active-trace-fri-reachable-reduction-from-header-merkle-and-untied-cases*:

fixes $Ht Mt Rt Qt Ot ::$

$(\text{'f staged-proof-data} \Rightarrow \text{'f protocol-channel} \Rightarrow \text{prob})$

assumes *trace-header-bound*:

$\wedge \text{data attacker-state.}$

$\text{Some (data, attacker-state)} \in$
 $\text{set-dist (execute (checked-staged-transcript-program A)}$
 $\text{adversary-initial-state)} \implies$

$\text{wp-event verify-monad}$
 $(\text{trace-fri-bad-with-header-tied-partial-candidate}$
 $(\text{verifier-state-from-adversary attacker-state}$
 $(\text{staged-proof-transcript data}))$
 $(\text{verifier-state-from-adversary attacker-state}$
 $(\text{staged-proof-transcript data})) \leq$

$Ht \text{ data attacker-state}$

and *trace-merkle-bound*:

$\wedge \text{data attacker-state.}$

$\text{Some (data, attacker-state)} \in$
 $\text{set-dist (execute (checked-staged-transcript-program A)}$
 $\text{adversary-initial-state)} \implies$

$\text{wp-event verify-monad}$
 $(\text{partial-merkle-inconsistency-bad}$

$(\text{verifier-state-from-adversary attacker-state}$
 $(\text{staged-proof-transcript data}))$
 $(\text{verifier-state-from-adversary attacker-state}$
 $(\text{staged-proof-transcript data})) \leq$
 $Mt \text{ data attacker-state}$
and *trace-root-bound*:
 $\wedge \text{data attacker-state.}$
 $\text{Some (data, attacker-state)} \in$
 $\text{set-dist (execute (checked-staged-transcript-program A)}$
 $\text{adversary-initial-state)} \implies$
 $\text{wp-event verify-monad}$
 $(\text{trace-fri-header-tied-low-candidate-untied-root-gap}$
 $(\text{verifier-state-from-adversary attacker-state}$
 $(\text{staged-proof-transcript data}))$
 $(\text{verifier-state-from-adversary attacker-state}$
 $(\text{staged-proof-transcript data})) \leq$
 $Rt \text{ data attacker-state}$
and *trace-query-bound*:
 $\wedge \text{data attacker-state.}$
 $\text{Some (data, attacker-state)} \in$
 $\text{set-dist (execute (checked-staged-transcript-program A)}$
 $\text{adversary-initial-state)} \implies$
 $\text{wp-event verify-monad}$
 $(\text{trace-fri-header-tied-low-candidate-untied-query-gap}$
 $(\text{verifier-state-from-adversary attacker-state}$
 $(\text{staged-proof-transcript data}))$
 $(\text{verifier-state-from-adversary attacker-state}$
 $(\text{staged-proof-transcript data})) \leq$
 $Qt \text{ data attacker-state}$
and *trace-openings-bound*:
 $\wedge \text{data attacker-state.}$
 $\text{Some (data, attacker-state)} \in$
 $\text{set-dist (execute (checked-staged-transcript-program A)}$
 $\text{adversary-initial-state)} \implies$
 $\text{wp-event verify-monad}$
 $(\text{trace-fri-header-tied-low-candidate-untied-openings-gap}$
 $(\text{verifier-state-from-adversary attacker-state}$
 $(\text{staged-proof-transcript data}))$
 $(\text{verifier-state-from-adversary attacker-state}$
 $(\text{staged-proof-transcript data})) \leq$
 $Ot \text{ data attacker-state}$
and *total-bound*:
 $\wedge \text{data attacker-state.}$
 $\text{Some (data, attacker-state)} \in$
 $\text{set-dist (execute (checked-staged-transcript-program A)}$
 $\text{adversary-initial-state)} \implies$
 $Ht \text{ data attacker-state} +$
 $(Mt \text{ data attacker-state} +$
 $(Rt \text{ data attacker-state} +$

$(Qt \text{ data attacker-state} + Ot \text{ data attacker-state})) \leq$
 trace-fri-error
and *support*:
Some (*data*, *attacker-state*) $\in$
 $\text{set-dist } (\text{execute } (\text{checked-staged-transcript-program } A)$
 $\text{adversary-initial-state})$
shows
 $\text{trace-fri-reachable-partial-candidate-reduction-assumption}$
 $(\text{verifier-state-from-adversary attacker-state}$
 $(\text{staged-proof-transcript data}))$
by (*rule*
 $\text{trace-fri-reachable-partial-candidate-reduction-from-header-merkle-and-untied-cases})$
 $(\text{rule trace-header-bound}[OF \text{ support}],$
 $\text{rule trace-merkle-bound}[OF \text{ support}],$
 $\text{rule trace-root-bound}[OF \text{ support}],$
 $\text{rule trace-query-bound}[OF \text{ support}],$
 $\text{rule trace-openings-bound}[OF \text{ support}],$
 $\text{rule total-bound}[OF \text{ support}])$

lemma *active-trace-fri-reachable-reduction-from-header-merkle-and-untied-query-openings*:
fixes *Ht Mt Qt Ot* ::
 $(\text{'f staged-proof-data} \Rightarrow \text{'f protocol-channel} \Rightarrow \text{prob})$
assumes *trace-header-bound*:
 $\bigwedge \text{data attacker-state.}$
 $\text{Some } (\text{data}, \text{attacker-state}) \in$
 $\text{set-dist } (\text{execute } (\text{checked-staged-transcript-program } A)$
 $\text{adversary-initial-state}) \implies$
 $\text{wp-event verify-monad}$
 $(\text{trace-fri-bad-with-header-tied-partial-candidate}$
 $(\text{verifier-state-from-adversary attacker-state}$
 $(\text{staged-proof-transcript data})))$
 $(\text{verifier-state-from-adversary attacker-state}$
 $(\text{staged-proof-transcript data})) \leq$
 $Ht \text{ data attacker-state}$
and *trace-merkle-bound*:
 $\bigwedge \text{data attacker-state.}$
 $\text{Some } (\text{data}, \text{attacker-state}) \in$
 $\text{set-dist } (\text{execute } (\text{checked-staged-transcript-program } A)$
 $\text{adversary-initial-state}) \implies$
 $\text{wp-event verify-monad}$
 $(\text{partial-merkle-inconsistency-bad}$
 $(\text{verifier-state-from-adversary attacker-state}$
 $(\text{staged-proof-transcript data})))$
 $(\text{verifier-state-from-adversary attacker-state}$
 $(\text{staged-proof-transcript data})) \leq$
 $Mt \text{ data attacker-state}$
and *trace-query-bound*:
 $\bigwedge \text{data attacker-state.}$
 $\text{Some } (\text{data}, \text{attacker-state}) \in$

$\text{set-dist } (\text{execute } (\text{checked-staged-transcript-program } A) \text{ adversary-initial-state}) \implies$
 $\text{wp-event verify-monad}$
 $(\text{trace-fri-header-tied-low-candidate-untied-query-gap}$
 $(\text{verifier-state-from-adversary attacker-state}$
 $(\text{staged-proof-transcript data})))$
 $(\text{verifier-state-from-adversary attacker-state}$
 $(\text{staged-proof-transcript data})) \leq$
 $Qt \text{ data attacker-state}$
and $\text{trace-openings-bound:}$
 $\wedge \text{data attacker-state.}$
 $\text{Some } (data, \text{attacker-state}) \in$
 $\text{set-dist } (\text{execute } (\text{checked-staged-transcript-program } A) \text{ adversary-initial-state}) \implies$
 $\text{wp-event verify-monad}$
 $(\text{trace-fri-header-tied-low-candidate-untied-openings-gap}$
 $(\text{verifier-state-from-adversary attacker-state}$
 $(\text{staged-proof-transcript data})))$
 $(\text{verifier-state-from-adversary attacker-state}$
 $(\text{staged-proof-transcript data})) \leq$
 $Ot \text{ data attacker-state}$
and total-bound:
 $\wedge \text{data attacker-state.}$
 $\text{Some } (data, \text{attacker-state}) \in$
 $\text{set-dist } (\text{execute } (\text{checked-staged-transcript-program } A) \text{ adversary-initial-state}) \implies$
 $Ht \text{ data attacker-state} +$
 $(Mt \text{ data attacker-state} +$
 $(Qt \text{ data attacker-state} + Ot \text{ data attacker-state})) \leq$
 trace-fri-error
and support:
 $\text{Some } (data, \text{attacker-state}) \in$
 $\text{set-dist } (\text{execute } (\text{checked-staged-transcript-program } A) \text{ adversary-initial-state})$
shows
 $\text{trace-fri-reachable-partial-candidate-reduction-assumption}$
 $(\text{verifier-state-from-adversary attacker-state}$
 $(\text{staged-proof-transcript data}))$
by $(\text{rule}$
 $\text{trace-fri-reachable-partial-candidate-reduction-from-header-merkle-and-untied-query-openings})$
 $(\text{rule trace-header-bound}[OF \text{ support}],$
 $\text{rule trace-merkle-bound}[OF \text{ support}],$
 $\text{rule trace-query-bound}[OF \text{ support}],$
 $\text{rule trace-openings-bound}[OF \text{ support}],$
 $\text{rule total-bound}[OF \text{ support}])$

lemma $\text{active-trace-fri-reachable-reduction-from-header-merkle-and-untied-openings:}$
fixes $Ht \ Mt \ Ot ::$
 $(\text{'f staged-proof-data} \Rightarrow \text{'f protocol-channel} \Rightarrow \text{prob})$

assumes *trace-header-bound*:

$$\begin{aligned} &\wedge \text{data attacker-state.} \\ &\text{Some (data, attacker-state)} \in \\ &\quad \text{set-dist (execute (checked-staged-transcript-program A)} \\ &\quad \quad \text{adversary-initial-state)} \implies \\ &\text{wp-event verify-monad} \\ &\quad (\text{trace-fri-bad-with-header-tied-partial-candidate} \\ &\quad \quad (\text{verifier-state-from-adversary attacker-state} \\ &\quad \quad \quad (\text{staged-proof-transcript data}))) \\ &\quad (\text{verifier-state-from-adversary attacker-state} \\ &\quad \quad (\text{staged-proof-transcript data})) \leq \\ &\text{Ht data attacker-state} \end{aligned}$$

and *trace-merkle-bound*:

$$\begin{aligned} &\wedge \text{data attacker-state.} \\ &\text{Some (data, attacker-state)} \in \\ &\quad \text{set-dist (execute (checked-staged-transcript-program A)} \\ &\quad \quad \text{adversary-initial-state)} \implies \\ &\text{wp-event verify-monad} \\ &\quad (\text{partial-merkle-inconsistency-bad} \\ &\quad \quad (\text{verifier-state-from-adversary attacker-state} \\ &\quad \quad \quad (\text{staged-proof-transcript data}))) \\ &\quad (\text{verifier-state-from-adversary attacker-state} \\ &\quad \quad (\text{staged-proof-transcript data})) \leq \\ &\text{Mt data attacker-state} \end{aligned}$$

and *trace-openings-bound*:

$$\begin{aligned} &\wedge \text{data attacker-state.} \\ &\text{Some (data, attacker-state)} \in \\ &\quad \text{set-dist (execute (checked-staged-transcript-program A)} \\ &\quad \quad \text{adversary-initial-state)} \implies \\ &\text{wp-event verify-monad} \\ &\quad (\text{trace-fri-header-tied-low-candidate-untied-openings-gap} \\ &\quad \quad (\text{verifier-state-from-adversary attacker-state} \\ &\quad \quad \quad (\text{staged-proof-transcript data}))) \\ &\quad (\text{verifier-state-from-adversary attacker-state} \\ &\quad \quad (\text{staged-proof-transcript data})) \leq \\ &\text{Ot data attacker-state} \end{aligned}$$

and *total-bound*:

$$\begin{aligned} &\wedge \text{data attacker-state.} \\ &\text{Some (data, attacker-state)} \in \\ &\quad \text{set-dist (execute (checked-staged-transcript-program A)} \\ &\quad \quad \text{adversary-initial-state)} \implies \\ &\text{Ht data attacker-state} + \\ &\quad (\text{Mt data attacker-state} + \text{Ot data attacker-state}) \leq \\ &\text{trace-fri-error} \end{aligned}$$

and *support*:

$$\begin{aligned} &\text{Some (data, attacker-state)} \in \\ &\quad \text{set-dist (execute (checked-staged-transcript-program A)} \\ &\quad \quad \text{adversary-initial-state)} \end{aligned}$$

shows

trace-fri-reachable-partial-candidate-reduction-assumption
 (verifier-state-from-adversary attacker-state
 (staged-proof-transcript data))
by (rule
 trace-fri-reachable-partial-candidate-reduction-from-header-merkle-and-untied-openings)
 (rule trace-header-bound[OF support],
 rule trace-merkle-bound[OF support],
 rule trace-openings-bound[OF support],
 rule total-bound[OF support])

lemma *active-trace-fri-reachable-reduction-from-header-merkle-hash-root-query*:

fixes *Ht Mt Kt Rt Qt* ::

(*'f staged-proof-data* $\Rightarrow$ *'f protocol-channel* $\Rightarrow$ *prob*)

assumes *trace-header-bound*:

$\wedge$ *data attacker-state*.

Some (*data*, *attacker-state*) $\in$
set-dist (*execute* (*checked-staged-transcript-program A*)
adversary-initial-state) $\Rightarrow$

wp-event verify-monad
 (*trace-fri-bad-with-header-tied-partial-candidate*
 (*verifier-state-from-adversary attacker-state*
 (*staged-proof-transcript data*)))
 (*verifier-state-from-adversary attacker-state*
 (*staged-proof-transcript data*)) $\leq$

Ht data attacker-state

and *trace-merkle-bound*:

$\wedge$ *data attacker-state*.

Some (*data*, *attacker-state*) $\in$
set-dist (*execute* (*checked-staged-transcript-program A*)
adversary-initial-state) $\Rightarrow$

wp-event verify-monad
 (*partial-merkle-inconsistency-bad*
 (*verifier-state-from-adversary attacker-state*
 (*staged-proof-transcript data*)))
 (*verifier-state-from-adversary attacker-state*
 (*staged-proof-transcript data*)) $\leq$

Mt data attacker-state

and *trace-hash-bound*:

$\wedge$ *data attacker-state*.

Some (*data*, *attacker-state*) $\in$
set-dist (*execute* (*checked-staged-transcript-program A*)
adversary-initial-state) $\Rightarrow$

wp-event verify-monad
 (*hash-map-output-collision-bad*
 (*verifier-state-from-adversary attacker-state*
 (*staged-proof-transcript data*)))
 (*verifier-state-from-adversary attacker-state*
 (*staged-proof-transcript data*)) $\leq$

Kt data attacker-state

and *trace-root-bound*:

$\wedge$ *data attacker-state*.

Some (*data*, *attacker-state*) $\in$
set-dist (*execute* (*checked-staged-transcript-program* *A*)
adversary-initial-state) $\implies$

wp-event verify-monad
(*trace-fri-header-tied-low-candidate-untied-root-gap*
(*verifier-state-from-adversary attacker-state*
(*staged-proof-transcript data*)))
(*verifier-state-from-adversary attacker-state*
(*staged-proof-transcript data*)) $\leq$

Rt data attacker-state

and *trace-query-bound*:

$\wedge$ *data attacker-state*.

Some (*data*, *attacker-state*) $\in$
set-dist (*execute* (*checked-staged-transcript-program* *A*)
adversary-initial-state) $\implies$

wp-event verify-monad
(*trace-fri-header-tied-low-candidate-untied-query-gap*
(*verifier-state-from-adversary attacker-state*
(*staged-proof-transcript data*)))
(*verifier-state-from-adversary attacker-state*
(*staged-proof-transcript data*)) $\leq$

Qt data attacker-state

and *total-bound*:

$\wedge$ *data attacker-state*.

Some (*data*, *attacker-state*) $\in$
set-dist (*execute* (*checked-staged-transcript-program* *A*)
adversary-initial-state) $\implies$

Ht data attacker-state +
(*Mt data attacker-state* +
(*Kt data attacker-state* +
(*Rt data attacker-state* + *Qt data attacker-state*))) $\leq$
trace-fri-error

and *support*:

Some (*data*, *attacker-state*) $\in$
set-dist (*execute* (*checked-staged-transcript-program* *A*)
adversary-initial-state)

shows

trace-fri-reachable-partial-candidate-reduction-assumption
(*verifier-state-from-adversary attacker-state*
(*staged-proof-transcript data*))

by (*rule*

trace-fri-reachable-partial-candidate-reduction-from-header-merkle-hash-root-query)
(*rule trace-header-bound*[*OF support*],
rule trace-merkle-bound[*OF support*],
rule trace-hash-bound[*OF support*],
rule trace-root-bound[*OF support*],
rule trace-query-bound[*OF support*],

rule total-bound[OF support])

end

end

theory *Soundness-FRI-Replay-Gap-Assignment*

imports

Soundness-FRI-Trace-Untied-Active

begin

Small adapters from the replay-gap residual predicates to the existing sampled-assignment conflict branches. The replay gaps are represented as sampled next, successor, or final conflicts, so they are charged to the same FRI sampled-assignment layer rather than treated as independent probabilistic obligations.

context *soundness*

begin

lemma *trace-fri-header-tied-next-value-replay-gap-imp-without-same:*

assumes *trace-fri-header-tied-next-value-replay-gap s out*

shows

*trace-fri-header-tied-sampled-assignment-conflict-without-same-layer
s out*

using *assms*

unfolding *trace-fri-header-tied-next-value-replay-gap-def*

by (*blast intro: trace-fri-header-tied-without-sameI-next*)

lemma *trace-fri-header-tied-successor-replay-gap-imp-without-same:*

assumes *trace-fri-header-tied-successor-replay-gap s out*

shows

*trace-fri-header-tied-sampled-assignment-conflict-without-same-layer
s out*

using *assms*

unfolding *trace-fri-header-tied-successor-replay-gap-def*

by (*blast intro: trace-fri-header-tied-without-sameI-successor*)

lemma *trace-fri-header-tied-final-value-replay-gap-imp-without-same:*

assumes *trace-fri-header-tied-final-value-replay-gap s out*

shows

*trace-fri-header-tied-sampled-assignment-conflict-without-same-layer
s out*

using *assms*

unfolding *trace-fri-header-tied-final-value-replay-gap-def*

by (*blast intro: trace-fri-header-tied-without-sameI-final*)

lemma *trace-fri-header-tied-without-same-imp-sampled-assignment-conflict:*

assumes

trace-fri-header-tied-sampled-assignment-conflict-without-same-layer

```

    s out
shows trace-fri-header-tied-sampled-assignment-conflict s out
using assms
unfolding
  trace-fri-header-tied-sampled-assignment-conflict-without-same-layer-def
  trace-fri-header-tied-sampled-assignment-conflict-def
  generic-fri-sampled-assignment-conflict-without-same-layer-def
  generic-fri-sampled-assignment-conflict-def
by blast

lemma wp-trace-fri-header-tied-next-value-replay-gap-bound-from-without-same:
fixes C :: prob
assumes conflict-bound:
  wp-event verify-monad
  (trace-fri-header-tied-sampled-assignment-conflict-without-same-layer
   s) s ≤ C
shows
  wp-event verify-monad
  (trace-fri-header-tied-next-value-replay-gap s) s ≤ C
by (rule order-trans[OF - conflict-bound])
  (rule wp-event-mono,
   rule trace-fri-header-tied-next-value-replay-gap-imp-without-same)

lemma wp-trace-fri-header-tied-successor-replay-gap-bound-from-without-same:
fixes C :: prob
assumes conflict-bound:
  wp-event verify-monad
  (trace-fri-header-tied-sampled-assignment-conflict-without-same-layer
   s) s ≤ C
shows
  wp-event verify-monad
  (trace-fri-header-tied-successor-replay-gap s) s ≤ C
by (rule order-trans[OF - conflict-bound])
  (rule wp-event-mono,
   rule trace-fri-header-tied-successor-replay-gap-imp-without-same)

lemma wp-trace-fri-header-tied-final-value-replay-gap-bound-from-without-same:
fixes C :: prob
assumes conflict-bound:
  wp-event verify-monad
  (trace-fri-header-tied-sampled-assignment-conflict-without-same-layer
   s) s ≤ C
shows
  wp-event verify-monad
  (trace-fri-header-tied-final-value-replay-gap s) s ≤ C
by (rule order-trans[OF - conflict-bound])
  (rule wp-event-mono,
   rule trace-fri-header-tied-final-value-replay-gap-imp-without-same)

```

lemma *wp-trace-fri-header-tied-without-same-bound-from-sampled-assignment-conflict:*

fixes $C :: \text{prob}$

assumes *conflict-bound:*

wp-event verify-monad

$(\text{trace-fri-header-tied-sampled-assignment-conflict } s) \ s \leq C$

shows

wp-event verify-monad

$(\text{trace-fri-header-tied-sampled-assignment-conflict-without-same-layer } s) \ s \leq C$

by (*rule order-trans[OF - conflict-bound]*)

(rule wp-event-mono,

rule trace-fri-header-tied-without-same-imp-sampled-assignment-conflict)

lemma *composition-fri-verifier-tied-next-value-replay-gap-imp-without-same:*

assumes *composition-fri-verifier-tied-next-value-replay-gap s out*

shows

composition-fri-verifier-tied-sampled-assignment-conflict-without-same-layer s out

using *assms*

unfolding *composition-fri-verifier-tied-next-value-replay-gap-def*

by (*blast intro: composition-fri-verifier-tied-without-sameI-next*)

lemma *composition-fri-verifier-tied-successor-replay-gap-imp-without-same:*

assumes *composition-fri-verifier-tied-successor-replay-gap s out*

shows

composition-fri-verifier-tied-sampled-assignment-conflict-without-same-layer s out

using *assms*

unfolding *composition-fri-verifier-tied-successor-replay-gap-def*

by (*blast intro: composition-fri-verifier-tied-without-sameI-successor*)

lemma *composition-fri-verifier-tied-final-value-replay-gap-imp-without-same:*

assumes *composition-fri-verifier-tied-final-value-replay-gap s out*

shows

composition-fri-verifier-tied-sampled-assignment-conflict-without-same-layer s out

using *assms*

unfolding *composition-fri-verifier-tied-final-value-replay-gap-def*

by (*blast intro: composition-fri-verifier-tied-without-sameI-final*)

lemma *composition-fri-verifier-tied-without-same-imp-sampled-assignment-conflict:*

assumes

composition-fri-verifier-tied-sampled-assignment-conflict-without-same-layer s out

shows *composition-fri-verifier-tied-sampled-assignment-conflict s out*

using *assms*

unfolding

composition-fri-verifier-tied-sampled-assignment-conflict-without-same-layer-def

composition-fri-verifier-tied-sampled-assignment-conflict-def

generic-fri-sampled-assignment-conflict-without-same-layer-def
generic-fri-sampled-assignment-conflict-def
by *blast*

lemma *wp-composition-fri-verifier-tied-next-value-replay-gap-bound-from-without-same:*

fixes $C :: \text{prob}$

assumes *conflict-bound:*

wp-event verify-monad

(*composition-fri-verifier-tied-sampled-assignment-conflict-without-same-layer*
 s) $s \leq C$

shows

wp-event verify-monad

(*composition-fri-verifier-tied-next-value-replay-gap* s) $s \leq C$

by (*rule order-trans*[*OF* - *conflict-bound*])

(*rule wp-event-mono*,

rule composition-fri-verifier-tied-next-value-replay-gap-imp-without-same)

lemma *wp-composition-fri-verifier-tied-successor-replay-gap-bound-from-without-same:*

fixes $C :: \text{prob}$

assumes *conflict-bound:*

wp-event verify-monad

(*composition-fri-verifier-tied-sampled-assignment-conflict-without-same-layer*
 s) $s \leq C$

shows

wp-event verify-monad

(*composition-fri-verifier-tied-successor-replay-gap* s) $s \leq C$

by (*rule order-trans*[*OF* - *conflict-bound*])

(*rule wp-event-mono*,

rule composition-fri-verifier-tied-successor-replay-gap-imp-without-same)

lemma *wp-composition-fri-verifier-tied-final-value-replay-gap-bound-from-without-same:*

fixes $C :: \text{prob}$

assumes *conflict-bound:*

wp-event verify-monad

(*composition-fri-verifier-tied-sampled-assignment-conflict-without-same-layer*
 s) $s \leq C$

shows

wp-event verify-monad

(*composition-fri-verifier-tied-final-value-replay-gap* s) $s \leq C$

by (*rule order-trans*[*OF* - *conflict-bound*])

(*rule wp-event-mono*,

rule composition-fri-verifier-tied-final-value-replay-gap-imp-without-same)

lemma *wp-composition-fri-verifier-tied-without-same-bound-from-sampled-assignment-conflict:*

fixes $C :: \text{prob}$

assumes *conflict-bound:*

wp-event verify-monad

(*composition-fri-verifier-tied-sampled-assignment-conflict* s) $s \leq C$

shows

wp-event verify-monad
 (*composition-fri-verifier-tied-sampled-assignment-conflict-without-same-layer*
s) $s \leq C$
by (rule *order-trans[OF - conflict-bound]*)
 (rule *wp-event-mono*,
 rule *composition-fri-verifier-tied-without-same-imp-sampled-assignment-conflict*)

lemma *trace-fri-header-tied-residual-obligations-from-without-same-bounds:*

fixes $C :: \text{prob}$

assumes *sampled:*

wp-event verify-monad
 (*trace-fri-bad-with-header-tied-sampled-layer-chain s*) $s \leq r-b$

and *zero:*

wp-event verify-monad
 (*trace-fri-header-tied-zero-round-final-obstruction s*) $s \leq z-b$

and *merkle:*

wp-event verify-monad (*partial-merkle-inconsistency-bad s*) $s \leq p-b$

and *structural:*

wp-event verify-monad
 (*trace-fri-header-tied-sibling-mismatch-structural-gap s*) $s \leq h-b$

and *without-same:*

wp-event verify-monad
 (*trace-fri-header-tied-sampled-assignment-conflict-without-same-layer*
s) $s \leq C$

and *total:*

$r-b + (((((p-b + h-b) + p-b) + C + C +$
 $(p-b + 0 + 0)) + p-b) + 0 + z-b)$
 $\leq \text{trace-fri-error}$

shows

trace-fri-header-tied-residual-obligations s
 $r-b \ z-b \ 0 \ p-b \ h-b \ p-b \ C \ C \ 0$

by (rule

trace-fri-header-tied-residual-obligations-from-bounds-without-head-slot-and-selected-zero)
 (rule *sampled*, rule *zero*, rule *merkle*, rule *structural*,
 rule *wp-trace-fri-header-tied-next-value-replay-gap-bound-from-without-same*
 [*OF without-same*],
 rule *wp-trace-fri-header-tied-successor-replay-gap-bound-from-without-same*
 [*OF without-same*],
 rule *total*)

lemma *composition-fri-verifier-tied-residual-obligations-from-without-same-bounds:*

fixes $C :: \text{prob}$

assumes *sampled:*

wp-event verify-monad
 (*composition-fri-bad-with-verifier-tied-sampled-layer-chain s*) s
 $\leq r-b$

and *merkle:*

wp-event verify-monad (*partial-merkle-inconsistency-bad s*) $s \leq p-b$

and *without-same:*

wp-event verify-monad
 (composition-fri-verifier-tied-sampled-assignment-conflict-without-same-layer
 s) *s* ≤ *C*
and total:
r-b + (((*p-b* + 0) + *C* + *C* + *C* + *p-b*) + 0)
 ≤ composition-fri-error
shows
 composition-fri-verifier-tied-residual-obligations *s r-b 0 p-b C C C*
by (rule composition-fri-verifier-tied-residual-obligations-from-bounds-slot-zero)
 (rule sampled, rule merkle,
 rule wp-composition-fri-verifier-tied-next-value-replay-gap-bound-from-without-same
 [*OF without-same*],
 rule wp-composition-fri-verifier-tied-successor-replay-gap-bound-from-without-same
 [*OF without-same*],
 rule wp-composition-fri-verifier-tied-final-value-replay-gap-bound-from-without-same
 [*OF without-same*],
 rule total)

lemma active-trace-fri-residual-obligations-from-without-same-bounds:

fixes

Rt Zt Pt Ht Ct ::

(*'f staged-proof-data* ⇒

'f protocol-channel ⇒ *prob*)

assumes trace-sampled-bound:

∧ *data attacker-state*.

Some (*data, attacker-state*) ∈

set-dist (execute (checked-staged-transcript-program *A*)

adversary-initial-state) ⇒

wp-event verify-monad

(trace-fri-bad-with-header-tied-sampled-layer-chain

(verifier-state-from-adversary *attacker-state*

(staged-proof-transcript *data*)))

(verifier-state-from-adversary *attacker-state*

(staged-proof-transcript *data*)) ≤

Rt data attacker-state

and trace-zero-bound:

∧ *data attacker-state*.

Some (*data, attacker-state*) ∈

set-dist (execute (checked-staged-transcript-program *A*)

adversary-initial-state) ⇒

wp-event verify-monad

(trace-fri-header-tied-zero-round-final-obstruction

(verifier-state-from-adversary *attacker-state*

(staged-proof-transcript *data*)))

(verifier-state-from-adversary *attacker-state*

(staged-proof-transcript *data*)) ≤

Zt data attacker-state

and trace-merkle-bound:

∧ *data attacker-state*.

Some (*data*, *attacker-state*) $\in$
set-dist (*execute* (*checked-staged-transcript-program* *A*)
adversary-initial-state) $\implies$
wp-event verify-monad
(partial-merkle-inconsistency-bad
(verifier-state-from-adversary attacker-state
(staged-proof-transcript data)))
(verifier-state-from-adversary attacker-state
(staged-proof-transcript data)) $\leq$
Pt data attacker-state
and *trace-structural-bound*:
 $\wedge$ *data attacker-state*.
Some (*data*, *attacker-state*) $\in$
set-dist (*execute* (*checked-staged-transcript-program* *A*)
adversary-initial-state) $\implies$
wp-event verify-monad
(trace-fri-header-tied-sibling-mismatch-structural-gap
(verifier-state-from-adversary attacker-state
(staged-proof-transcript data)))
(verifier-state-from-adversary attacker-state
(staged-proof-transcript data)) $\leq$
Ht data attacker-state
and *trace-without-same-bound*:
 $\wedge$ *data attacker-state*.
Some (*data*, *attacker-state*) $\in$
set-dist (*execute* (*checked-staged-transcript-program* *A*)
adversary-initial-state) $\implies$
wp-event verify-monad
(trace-fri-header-tied-sampled-assignment-conflict-without-same-layer
(verifier-state-from-adversary attacker-state
(staged-proof-transcript data)))
(verifier-state-from-adversary attacker-state
(staged-proof-transcript data)) $\leq$
Ct data attacker-state
and *trace-total-bound*:
 $\wedge$ *data attacker-state*.
Some (*data*, *attacker-state*) $\in$
set-dist (*execute* (*checked-staged-transcript-program* *A*)
adversary-initial-state) $\implies$
Rt data attacker-state +
(((((Pt data attacker-state + Ht data attacker-state) +
Pt data attacker-state) +
Ct data attacker-state + Ct data attacker-state +
(Pt data attacker-state + 0 + 0)) +
Pt data attacker-state) +
0 + Zt data attacker-state)
 $\leq$ *trace-fri-error*
shows *active-trace-fri-residual-obligations-for A Rt Zt*
 $(\lambda - . (0::prob)) \text{ Pt Ht Pt Ct Ct } (\lambda - . (0::prob))$


```

unfolding active-trace-fri-residual-obligations-for-def
proof (intro allI impI)
fix data attacker-state
assume support:
  Some (data, attacker-state) ∈
    set-dist (execute (checked-staged-transcript-program A)
      adversary-initial-state)
let ?s =
  verifier-state-from-adversary attacker-state
    (staged-proof-transcript data)
show trace-fri-header-tied-residual-obligations ?s
  (Rt data attacker-state) (Zt data attacker-state) 0
  (Pt data attacker-state) (Ht data attacker-state)
  (Pt data attacker-state) (Ct data attacker-state)
  (Ct data attacker-state) 0
by (rule trace-fri-header-tied-residual-obligations-from-without-same-bounds)
  (rule trace-sampled-bound[OF support],
   rule trace-zero-bound[OF support],
   rule trace-merkle-bound[OF support],
   rule trace-structural-bound[OF support],
   rule trace-without-same-bound[OF support],
   rule trace-total-bound[OF support])

```

qed

lemma *active-composition-fri-residual-obligations-from-without-same-bounds*:

fixes

Rc Pc Cc ::

('f staged-proof-data ⇒

'f protocol-channel ⇒ prob)

assumes *comp-sampled-bound*:

$\bigwedge$ *data attacker-state.*

Some (data, attacker-state) ∈

set-dist (execute (checked-staged-transcript-program A)

adversary-initial-state) ⇒

wp-event verify-monad

(composition-fri-bad-with-verifier-tied-sampled-layer-chain

(verifier-state-from-adversary attacker-state

(staged-proof-transcript data)))

(verifier-state-from-adversary attacker-state

(staged-proof-transcript data)) ≤

Rc data attacker-state

and *comp-merkle-bound*:

$\bigwedge$ *data attacker-state.*

Some (data, attacker-state) ∈

set-dist (execute (checked-staged-transcript-program A)

adversary-initial-state) ⇒

wp-event verify-monad

(partial-merkle-inconsistency-bad

(verifier-state-from-adversary attacker-state

```

      (staged-proof-transcript data)))
    (verifier-state-from-adversary attacker-state
      (staged-proof-transcript data)) ≤
    Pc data attacker-state
and comp-without-same-bound:
  ∧ data attacker-state.
    Some (data, attacker-state) ∈
      set-dist (execute (checked-staged-transcript-program A)
        adversary-initial-state) ⇒
    wp-event verify-monad
      (composition-fri-verifier-tied-sampled-assignment-conflict-without-same-layer
        (verifier-state-from-adversary attacker-state
          (staged-proof-transcript data)))
      (verifier-state-from-adversary attacker-state
        (staged-proof-transcript data)) ≤
    Cc data attacker-state
and comp-total-bound:
  ∧ data attacker-state.
    Some (data, attacker-state) ∈
      set-dist (execute (checked-staged-transcript-program A)
        adversary-initial-state) ⇒
    Rc data attacker-state +
      (((Pc data attacker-state + 0) +
        Cc data attacker-state + Cc data attacker-state +
        Cc data attacker-state + Pc data attacker-state) +
        0)
    ≤ composition-fri-error
shows active-composition-fri-residual-obligations-for A Rc
  (λ- -. (0::prob)) Pc Cc Cc Cc
unfolding active-composition-fri-residual-obligations-for-def
proof (intro allI impI)
fix data attacker-state
assume support:
  Some (data, attacker-state) ∈
    set-dist (execute (checked-staged-transcript-program A)
      adversary-initial-state)
let ?s =
  verifier-state-from-adversary attacker-state
    (staged-proof-transcript data)
show composition-fri-verifier-tied-residual-obligations ?s
  (Rc data attacker-state) 0 (Pc data attacker-state)
  (Cc data attacker-state) (Cc data attacker-state)
  (Cc data attacker-state)
by (rule composition-fri-verifier-tied-residual-obligations-from-without-same-bounds)
  (rule comp-sampled-bound[OF support],
    rule comp-merkle-bound[OF support],
    rule comp-without-same-bound[OF support],
    rule comp-total-bound[OF support])
qed

```

lemma *trace-fri-reachable-partial-candidate-reduction-from-header-without-same-and-untied:*
fixes $R\ Z\ P\ H\ C\ Q\ U :: \text{prob}$
assumes *sampld-bound:*
 $\text{wp-event verify-monad}$
 $(\text{trace-fri-bad-with-header-tied-sampled-layer-chain } s) \ s \leq R$
and *zero-bound:*
 $\text{wp-event verify-monad}$
 $(\text{trace-fri-header-tied-zero-round-final-obstruction } s) \ s \leq Z$
and *header-partial-merkle-bound:*
 $\text{wp-event verify-monad}$ $(\text{partial-merkle-inconsistency-bad } s) \ s \leq P$
and *structural-gap-bound:*
 $\text{wp-event verify-monad}$
 $(\text{trace-fri-header-tied-sibling-mismatch-structural-gap } s) \ s \leq H$
and *without-same-bound:*
 $\text{wp-event verify-monad}$
 $(\text{trace-fri-header-tied-sampled-assignment-conflict-without-same-layer } s) \ s \leq C$
and *gap-partial-merkle-bound:*
 $\text{wp-event verify-monad}$ $(\text{partial-merkle-inconsistency-bad } s) \ s \leq Q$
and *untied-bound:*
 $\text{wp-event verify-monad}$
 $(\text{trace-fri-header-tied-low-candidate-untied-reachable-witness-gap } s) \ s \leq U$
and *total-bound:*
 $R + (((((P + H) + P) + C + C + C) + P) + 0 + Z) + (Q + U) \leq \text{trace-fri-error}$
shows *trace-fri-reachable-partial-candidate-reduction-assumption* s
proof –
have *head-neq:*
 $\text{wp-event verify-monad}$
 $(\text{trace-fri-header-tied-base-head-value-neq-gap } s) \ s \leq P$
by (rule
 $\text{wp-trace-fri-header-tied-base-head-value-neq-gap-bound-from-partial-merkle}$
 $[\text{OF header-partial-merkle-bound}]$)
have *head-mismatch:*
 $\text{wp-event verify-monad}$
 $(\text{trace-fri-header-tied-base-head-value-mismatch-gap } s) \ s \leq P$
by (rule
 $\text{wp-trace-fri-header-tied-base-head-value-mismatch-gap-bound-from-neq-gap}$
 $[\text{OF head-neq}]$)
have *alignment:*
 $\text{wp-event verify-monad}$
 $(\text{trace-fri-header-tied-base-value-alignment-gap } s) \ s \leq P$
by (rule
 $\text{wp-trace-fri-header-tied-base-value-alignment-gap-bound-from-head-value-mismatch-gap}$
 $[\text{OF head-mismatch}]$)
have *next-bound:*
 $\text{wp-event verify-monad}$

(trace-fri-header-tied-next-value-replay-gap s) $s \leq C$
by (rule
 wp-trace-fri-header-tied-next-value-replay-gap-bound-from-without-same
 [OF without-same-bound])
have successor-bound:
 wp-event verify-monad
 (trace-fri-header-tied-successor-replay-gap s) $s \leq C$
by (rule
 wp-trace-fri-header-tied-successor-replay-gap-bound-from-without-same
 [OF without-same-bound])
have final-bound:
 wp-event verify-monad
 (trace-fri-header-tied-final-value-replay-gap s) $s \leq C$
by (rule
 wp-trace-fri-header-tied-final-value-replay-gap-bound-from-without-same
 [OF without-same-bound])
have header-bound:
 wp-event verify-monad
 (trace-fri-bad-with-header-tied-partial-candidate s) $s \leq$
 $R + (((((P + H) + P) + C + C + C) + P) + 0 + Z)$
by (rule
 wp-trace-fri-header-tied-bound-from-sampled-zero-slot-recorded-chunk-gap-and-residuals)
 (rule sampled-bound, rule zero-bound,
 rule wp-trace-fri-header-tied-authenticated-slot-recorded-chunk-gap-zero,
 rule header-partial-merkle-bound, rule structural-gap-bound,
 rule alignment, rule next-bound, rule successor-bound,
 rule final-bound)
show ?thesis
by (rule
 trace-fri-reachable-partial-candidate-reduction-from-header-merkle-and-untied)
 (rule header-bound, rule gap-partial-merkle-bound,
 rule untied-bound, rule total-bound)
qed

lemma trace-fri-reachable-partial-candidate-reduction-from-header-without-same-hash-root-query:
fixes $R\ Z\ P\ H\ C\ Q\ K\ Rt\ Qt :: \text{prob}$
assumes sampled-bound:
 wp-event verify-monad
 (trace-fri-bad-with-header-tied-sampled-layer-chain s) $s \leq R$
and zero-bound:
 wp-event verify-monad
 (trace-fri-header-tied-zero-round-final-obstruction s) $s \leq Z$
and header-partial-merkle-bound:
 wp-event verify-monad (partial-merkle-inconsistency-bad s) $s \leq P$
and structural-gap-bound:
 wp-event verify-monad
 (trace-fri-header-tied-sibling-mismatch-structural-gap s) $s \leq H$
and without-same-bound:
 wp-event verify-monad

$(\text{trace-fri-header-tied-sampled-assignment-conflict-without-same-layer } s) \leq C$
and *gap-partial-merkle-bound*:
 $\text{wp-event verify-monad } (\text{partial-merkle-inconsistency-bad } s) \leq Q$
and *hash-bound*:
 $\text{wp-event verify-monad } (\text{hash-map-output-collision-bad } s) \leq K$
and *root-bound*:
 $\text{wp-event verify-monad } (\text{trace-fri-header-tied-low-candidate-untied-root-gap } s) \leq Rt$
and *query-bound*:
 $\text{wp-event verify-monad } (\text{trace-fri-header-tied-low-candidate-untied-query-gap } s) \leq Qt$
and *total-bound*:
 $R + (((((P + H) + P) + C + C + C) + P) + 0 + Z) + (Q + (K + (Rt + Qt))) \leq \text{trace-fri-error}$
shows *trace-fri-reachable-partial-candidate-reduction-assumption* s
proof –
have *head-neq*:
 $\text{wp-event verify-monad } (\text{trace-fri-header-tied-base-head-value-neq-gap } s) \leq P$
by (rule
 $\text{wp-trace-fri-header-tied-base-head-value-neq-gap-bound-from-partial-merkle}$
 $[\text{OF header-partial-merkle-bound}])$
have *head-mismatch*:
 $\text{wp-event verify-monad } (\text{trace-fri-header-tied-base-head-value-mismatch-gap } s) \leq P$
by (rule
 $\text{wp-trace-fri-header-tied-base-head-value-mismatch-gap-bound-from-neq-gap}$
 $[\text{OF head-neq}])$
have *alignment*:
 $\text{wp-event verify-monad } (\text{trace-fri-header-tied-base-value-alignment-gap } s) \leq P$
by (rule
 $\text{wp-trace-fri-header-tied-base-value-alignment-gap-bound-from-head-value-mismatch-gap}$
 $[\text{OF head-mismatch}])$
have *next-bound*:
 $\text{wp-event verify-monad } (\text{trace-fri-header-tied-next-value-replay-gap } s) \leq C$
by (rule
 $\text{wp-trace-fri-header-tied-next-value-replay-gap-bound-from-without-same}$
 $[\text{OF without-same-bound}])$
have *successor-bound*:
 $\text{wp-event verify-monad } (\text{trace-fri-header-tied-successor-replay-gap } s) \leq C$
by (rule
 $\text{wp-trace-fri-header-tied-successor-replay-gap-bound-from-without-same}$
 $[\text{OF without-same-bound}])$
have *final-bound*:
 $\text{wp-event verify-monad}$

$(\text{trace-fri-header-tied-final-value-replay-gap } s) \ s \leq C$
by (rule
 $\text{wp-trace-fri-header-tied-final-value-replay-gap-bound-from-without-same}$
 $[OF \text{ without-same-bound}])$
have *header-bound*:
 $\text{wp-event verify-monad}$
 $(\text{trace-fri-bad-with-header-tied-partial-candidate } s) \ s \leq$
 $R + (((((P + H) + P) + C + C + C) + P) + 0 + Z)$
by (rule
 $\text{wp-trace-fri-header-tied-bound-from-sampled-zero-slot-recorded-chunk-gap-and-residuals})$
(rule *sampled-bound*, rule *zero-bound*,
rule *wp-trace-fri-header-tied-authenticated-slot-recorded-chunk-gap-zero*,
rule *header-partial-merkle-bound*, rule *structural-gap-bound*,
rule *alignment*, rule *next-bound*, rule *successor-bound*,
rule *final-bound*)
show *?thesis*
by (rule
 $\text{trace-fri-reachable-partial-candidate-reduction-from-header-merkle-hash-root-query})$
(rule *header-bound*, rule *gap-partial-merkle-bound*, rule *hash-bound*,
rule *root-bound*, rule *query-bound*, rule *total-bound*)
qed

lemma *active-trace-fri-reachable-reduction-from-header-without-same-and-untied*:
fixes *Rt Zt Pt Ht Ct Qt Ut* ::
 $(\text{'f staged-proof-data} \Rightarrow \text{'f protocol-channel} \Rightarrow \text{prob})$
assumes *trace-sampled-bound*:
 $\bigwedge \text{data attacker-state.}$
 $\text{Some } (data, \text{attacker-state}) \in$
 $\text{set-dist } (\text{execute } (\text{checked-staged-transcript-program } A)$
 $\text{adversary-initial-state}) \Rightarrow$
 $\text{wp-event verify-monad}$
 $(\text{trace-fri-bad-with-header-tied-sampled-layer-chain}$
 $(\text{verifier-state-from-adversary attacker-state}$
 $(\text{staged-proof-transcript data})))$
 $(\text{verifier-state-from-adversary attacker-state}$
 $(\text{staged-proof-transcript data})) \leq$
 $Rt \text{ data attacker-state}$
and *trace-zero-bound*:
 $\bigwedge \text{data attacker-state.}$
 $\text{Some } (data, \text{attacker-state}) \in$
 $\text{set-dist } (\text{execute } (\text{checked-staged-transcript-program } A)$
 $\text{adversary-initial-state}) \Rightarrow$
 $\text{wp-event verify-monad}$
 $(\text{trace-fri-header-tied-zero-round-final-obstruction}$
 $(\text{verifier-state-from-adversary attacker-state}$
 $(\text{staged-proof-transcript data})))$
 $(\text{verifier-state-from-adversary attacker-state}$
 $(\text{staged-proof-transcript data})) \leq$
 $Zt \text{ data attacker-state}$

and *trace-header-merkle-bound*:

$\wedge$ *data attacker-state*.

Some (*data*, *attacker-state*) $\in$
set-dist (*execute* (*checked-staged-transcript-program* *A*)
adversary-initial-state) $\implies$

wp-event verify-monad
(*partial-merkle-inconsistency-bad*
(*verifier-state-from-adversary attacker-state*
(*staged-proof-transcript data*)))
(*verifier-state-from-adversary attacker-state*
(*staged-proof-transcript data*)) $\leq$

Pt data attacker-state

and *trace-structural-bound*:

$\wedge$ *data attacker-state*.

Some (*data*, *attacker-state*) $\in$
set-dist (*execute* (*checked-staged-transcript-program* *A*)
adversary-initial-state) $\implies$

wp-event verify-monad
(*trace-fri-header-tied-sibling-mismatch-structural-gap*
(*verifier-state-from-adversary attacker-state*
(*staged-proof-transcript data*)))
(*verifier-state-from-adversary attacker-state*
(*staged-proof-transcript data*)) $\leq$

Ht data attacker-state

and *trace-without-same-bound*:

$\wedge$ *data attacker-state*.

Some (*data*, *attacker-state*) $\in$
set-dist (*execute* (*checked-staged-transcript-program* *A*)
adversary-initial-state) $\implies$

wp-event verify-monad
(*trace-fri-header-tied-sampled-assignment-conflict-without-same-layer*
(*verifier-state-from-adversary attacker-state*
(*staged-proof-transcript data*)))
(*verifier-state-from-adversary attacker-state*
(*staged-proof-transcript data*)) $\leq$

Ct data attacker-state

and *trace-gap-merkle-bound*:

$\wedge$ *data attacker-state*.

Some (*data*, *attacker-state*) $\in$
set-dist (*execute* (*checked-staged-transcript-program* *A*)
adversary-initial-state) $\implies$

wp-event verify-monad
(*partial-merkle-inconsistency-bad*
(*verifier-state-from-adversary attacker-state*
(*staged-proof-transcript data*)))
(*verifier-state-from-adversary attacker-state*
(*staged-proof-transcript data*)) $\leq$

Qt data attacker-state

and *trace-untied-bound*:

$\wedge$ data attacker-state.
 Some (data, attacker-state) $\in$
 set-dist (execute (checked-staged-transcript-program A)
 adversary-initial-state) $\implies$
 wp-event verify-monad
 (trace-fri-header-tied-low-candidate-untied-reachable-witness-gap
 (verifier-state-from-adversary attacker-state
 (staged-proof-transcript data)))
 (verifier-state-from-adversary attacker-state
 (staged-proof-transcript data)) $\leq$
 Ut data attacker-state
and trace-total-bound:
 $\wedge$ data attacker-state.
 Some (data, attacker-state) $\in$
 set-dist (execute (checked-staged-transcript-program A)
 adversary-initial-state) $\implies$
 Rt data attacker-state +
 (((((Pt data attacker-state + Ht data attacker-state) +
 Pt data attacker-state) +
 Ct data attacker-state + Ct data attacker-state +
 Ct data attacker-state) +
 Pt data attacker-state) +
 0 + Zt data attacker-state) +
 (Qt data attacker-state + Ut data attacker-state)
 $\leq$ trace-fri-error
and support:
 Some (data, attacker-state) $\in$
 set-dist (execute (checked-staged-transcript-program A)
 adversary-initial-state)
shows
 trace-fri-reachable-partial-candidate-reduction-assumption
 (verifier-state-from-adversary attacker-state
 (staged-proof-transcript data))
by (rule
 trace-fri-reachable-partial-candidate-reduction-from-header-without-same-and-untied)
 (rule trace-sampled-bound[OF support],
 rule trace-zero-bound[OF support],
 rule trace-header-merkle-bound[OF support],
 rule trace-structural-bound[OF support],
 rule trace-without-same-bound[OF support],
 rule trace-gap-merkle-bound[OF support],
 rule trace-untied-bound[OF support],
 rule trace-total-bound[OF support])

lemma active-trace-fri-reachable-reduction-from-header-without-same-hash-root-query:
fixes Rt Zt Pt Ht Ct Qt Kt Ut Vt ::
 ('f staged-proof-data $\implies$ 'f protocol-channel $\implies$ prob)
assumes trace-sampled-bound:
 $\wedge$ data attacker-state.

Some (data, attacker-state) ∈
set-dist (execute (checked-staged-transcript-program A)
adversary-initial-state) ⇒
wp-event verify-monad
(trace-fri-bad-with-header-tied-sampled-layer-chain
(verifier-state-from-adversary attacker-state
(staged-proof-transcript data)))
(verifier-state-from-adversary attacker-state
(staged-proof-transcript data)) ≤
Rt data attacker-state
and *trace-zero-bound:*
 $\wedge$ *data attacker-state.*
Some (data, attacker-state) ∈
set-dist (execute (checked-staged-transcript-program A)
adversary-initial-state) ⇒
wp-event verify-monad
(trace-fri-header-tied-zero-round-final-obstruction
(verifier-state-from-adversary attacker-state
(staged-proof-transcript data)))
(verifier-state-from-adversary attacker-state
(staged-proof-transcript data)) ≤
Zt data attacker-state
and *trace-header-merkle-bound:*
 $\wedge$ *data attacker-state.*
Some (data, attacker-state) ∈
set-dist (execute (checked-staged-transcript-program A)
adversary-initial-state) ⇒
wp-event verify-monad
(partial-merkle-inconsistency-bad
(verifier-state-from-adversary attacker-state
(staged-proof-transcript data)))
(verifier-state-from-adversary attacker-state
(staged-proof-transcript data)) ≤
Pt data attacker-state
and *trace-structural-bound:*
 $\wedge$ *data attacker-state.*
Some (data, attacker-state) ∈
set-dist (execute (checked-staged-transcript-program A)
adversary-initial-state) ⇒
wp-event verify-monad
(trace-fri-header-tied-sibling-mismatch-structural-gap
(verifier-state-from-adversary attacker-state
(staged-proof-transcript data)))
(verifier-state-from-adversary attacker-state
(staged-proof-transcript data)) ≤
Ht data attacker-state
and *trace-without-same-bound:*
 $\wedge$ *data attacker-state.*
Some (data, attacker-state) ∈

$\text{set-dist } (\text{execute } (\text{checked-staged-transcript-program } A) \text{ adversary-initial-state}) \implies$
 $\text{wp-event verify-monad}$
 $(\text{trace-fri-header-tied-sampled-assignment-conflict-without-same-layer}$
 $(\text{verifier-state-from-adversary attacker-state}$
 $(\text{staged-proof-transcript data})))$
 $(\text{verifier-state-from-adversary attacker-state}$
 $(\text{staged-proof-transcript data})) \leq$
 $\text{Ct data attacker-state}$
and $\text{trace-gap-merkle-bound:}$
 $\wedge \text{data attacker-state.}$
 $\text{Some } (data, \text{attacker-state}) \in$
 $\text{set-dist } (\text{execute } (\text{checked-staged-transcript-program } A) \text{ adversary-initial-state}) \implies$
 $\text{wp-event verify-monad}$
 $(\text{partial-merkle-inconsistency-bad}$
 $(\text{verifier-state-from-adversary attacker-state}$
 $(\text{staged-proof-transcript data})))$
 $(\text{verifier-state-from-adversary attacker-state}$
 $(\text{staged-proof-transcript data})) \leq$
 $\text{Qt data attacker-state}$
and trace-hash-bound:
 $\wedge \text{data attacker-state.}$
 $\text{Some } (data, \text{attacker-state}) \in$
 $\text{set-dist } (\text{execute } (\text{checked-staged-transcript-program } A) \text{ adversary-initial-state}) \implies$
 $\text{wp-event verify-monad}$
 $(\text{hash-map-output-collision-bad}$
 $(\text{verifier-state-from-adversary attacker-state}$
 $(\text{staged-proof-transcript data})))$
 $(\text{verifier-state-from-adversary attacker-state}$
 $(\text{staged-proof-transcript data})) \leq$
 $\text{Kt data attacker-state}$
and trace-root-bound:
 $\wedge \text{data attacker-state.}$
 $\text{Some } (data, \text{attacker-state}) \in$
 $\text{set-dist } (\text{execute } (\text{checked-staged-transcript-program } A) \text{ adversary-initial-state}) \implies$
 $\text{wp-event verify-monad}$
 $(\text{trace-fri-header-tied-low-candidate-untied-root-gap}$
 $(\text{verifier-state-from-adversary attacker-state}$
 $(\text{staged-proof-transcript data})))$
 $(\text{verifier-state-from-adversary attacker-state}$
 $(\text{staged-proof-transcript data})) \leq$
 $\text{Ut data attacker-state}$
and $\text{trace-query-bound:}$
 $\wedge \text{data attacker-state.}$
 $\text{Some } (data, \text{attacker-state}) \in$
 $\text{set-dist } (\text{execute } (\text{checked-staged-transcript-program } A)$

$\text{adversary-initial-state}) \implies$
 $\text{wp-event verify-monad}$
 $(\text{trace-fri-header-tied-low-candidate-untied-query-gap}$
 $(\text{verifier-state-from-adversary attacker-state}$
 $(\text{staged-proof-transcript data})))$
 $(\text{verifier-state-from-adversary attacker-state}$
 $(\text{staged-proof-transcript data})) \leq$
 $\text{Vt data attacker-state}$
and $\text{trace-total-bound:}$
 $\wedge \text{data attacker-state.}$
 $\text{Some (data, attacker-state) } \in$
 $\text{set-dist (execute (checked-staged-transcript-program A)}$
 $\text{adversary-initial-state}) \implies}$
 $\text{Rt data attacker-state} +$
 $(((((\text{Pt data attacker-state} + \text{Ht data attacker-state}) +$
 $\text{Pt data attacker-state}) +$
 $\text{Ct data attacker-state} + \text{Ct data attacker-state} +$
 $\text{Ct data attacker-state}) +$
 $\text{Pt data attacker-state}) +$
 $0 + \text{Zt data attacker-state}) +$
 $(\text{Qt data attacker-state} +$
 $(\text{Kt data attacker-state} +$
 $(\text{Ut data attacker-state} + \text{Vt data attacker-state})))$
 $\leq \text{trace-fri-error}$
and support:
 $\text{Some (data, attacker-state) } \in$
 $\text{set-dist (execute (checked-staged-transcript-program A)}$
 $\text{adversary-initial-state})$
shows
 $\text{trace-fri-reachable-partial-candidate-reduction-assumption}$
 $(\text{verifier-state-from-adversary attacker-state}$
 $(\text{staged-proof-transcript data}))$
by $(\text{rule}$
 $\text{trace-fri-reachable-partial-candidate-reduction-from-header-without-same-hash-root-query})$
 $(\text{rule trace-sampled-bound[OF support],}$
 $\text{rule trace-zero-bound[OF support],}$
 $\text{rule trace-header-merkle-bound[OF support],}$
 $\text{rule trace-structural-bound[OF support],}$
 $\text{rule trace-without-same-bound[OF support],}$
 $\text{rule trace-gap-merkle-bound[OF support],}$
 $\text{rule trace-hash-bound[OF support],}$
 $\text{rule trace-root-bound[OF support],}$
 $\text{rule trace-query-bound[OF support],}$
 $\text{rule trace-total-bound[OF support]})$

lemma $\text{composition-fri-reachable-partial-candidate-reduction-from-split-without-same-bounds:}$
fixes $R P C E Q T :: \text{prob}$
assumes sampled-bound:
 $\text{wp-event verify-monad}$

(*composition-fri-bad-with-verifier-tied-sampled-layer-chain* s) $s \leq R$
and *verifier-partial-merkle-bound*:
wp-event verify-monad (*partial-merkle-inconsistency-bad* s) $s \leq P$
and *without-same-bound*:
wp-event verify-monad
 (*composition-fri-verifier-tied-sampled-assignment-conflict-without-same-layer*
 s) $s \leq C$
and *empty-bound*:
wp-event verify-monad
 (*composition-fri-reachable-verifier-tie-empty-header-gap* s) $s \leq E$
and *gap-partial-merkle-bound*:
wp-event verify-monad (*partial-merkle-inconsistency-bad* s) $s \leq Q$
and *candidate-available-bound*:
wp-event verify-monad
 (*composition-fri-verifier-tied-candidate-available-gap* s) $s \leq T$
and *total-bound*:
 $R + (((P + 0) + C + C + C + P) + 0) + (E + (Q + T))$
 $\leq \text{composition-fri-error}$
shows *composition-fri-reachable-partial-candidate-reduction-assumption* s
by (*rule*
composition-fri-reachable-partial-candidate-reduction-from-split-residual-bounds-with-candidate-available-and-recon
(rule sampled-bound,
rule wp-composition-fri-verifier-tied-authenticated-slot-recorded-chunk-gap-zero,
rule verifier-partial-merkle-bound,
rule wp-composition-fri-verifier-tied-next-value-replay-gap-bound-from-without-same
[OF without-same-bound],
rule wp-composition-fri-verifier-tied-successor-replay-gap-bound-from-without-same
[OF without-same-bound],
rule wp-composition-fri-verifier-tied-final-value-replay-gap-bound-from-without-same
[OF without-same-bound],
rule empty-bound,
rule gap-partial-merkle-bound,
rule candidate-available-bound,
rule total-bound)

lemma *composition-fri-reachable-partial-candidate-reduction-from-split-sampled-assignment-bounds*:

fixes $R P C E Q T :: \text{prob}$

assumes *sampled-bound*:

wp-event verify-monad

(*composition-fri-bad-with-verifier-tied-sampled-layer-chain* s) $s \leq R$

and *verifier-partial-merkle-bound*:

wp-event verify-monad (*partial-merkle-inconsistency-bad* s) $s \leq P$

and *sampled-assignment-bound*:

wp-event verify-monad

(*composition-fri-verifier-tied-sampled-assignment-conflict* s) s
 $\leq C$

and *empty-bound*:

wp-event verify-monad

(*composition-fri-reachable-verifier-tie-empty-header-gap* s) $s \leq E$

and *gap-partial-merkle-bound*:
 $wp\text{-}event\ verify\text{-}monad\ (partial\text{-}merkle\text{-}inconsistency\text{-}bad\ s)\ s \leq Q$
and *candidate-available-bound*:
 $wp\text{-}event\ verify\text{-}monad\ (composition\text{-}fri\text{-}verifier\text{-}tied\text{-}candidate\text{-}available\text{-}gap\ s)\ s \leq T$
and *total-bound*:
 $R + (((P + 0) + C + C + C + P) + 0) + (E + (Q + T))$
 $\leq composition\text{-}fri\text{-}error$
shows *composition-fri-reachable-partial-candidate-reduction-assumption s*
by (*rule*
 $composition\text{-}fri\text{-}reachable\text{-}partial\text{-}candidate\text{-}reduction\text{-}from\text{-}split\text{-}without\text{-}same\text{-}bounds$)
(*rule* *sampled-bound*,
rule *verifier-partial-merkle-bound*,
rule *wp-composition-fri-verifier-tied-without-same-bound-from-sampled-assignment-conflict*
 $[OF\ sampled\text{-}assignment\text{-}bound]$,
rule *empty-bound*,
rule *gap-partial-merkle-bound*,
rule *candidate-available-bound*,
rule *total-bound*)

lemma *active-composition-fri-reachable-reduction-from-split-without-same-bounds*:
fixes $Rc\ Pc\ Cc\ Ec\ Qc\ Tc ::$
 $(\text{'f staged-proof-data} \Rightarrow \text{'f protocol-channel} \Rightarrow prob)$
assumes *comp-sampled-bound*:
 $\bigwedge data\ attacker\text{-}state.$
 $Some\ (data,\ attacker\text{-}state) \in$
 $set\text{-}dist\ (execute\ (checked\text{-}staged\text{-}transcript\text{-}program\ A)$
 $adversary\text{-}initial\text{-}state) \implies$
 $wp\text{-}event\ verify\text{-}monad$
 $(composition\text{-}fri\text{-}bad\text{-}with\text{-}verifier\text{-}tied\text{-}sampled\text{-}layer\text{-}chain$
 $(verifier\text{-}state\text{-}from\text{-}adversary\ attacker\text{-}state$
 $(staged\text{-}proof\text{-}transcript\ data)))$
 $(verifier\text{-}state\text{-}from\text{-}adversary\ attacker\text{-}state$
 $(staged\text{-}proof\text{-}transcript\ data)) \leq$
 $Rc\ data\ attacker\text{-}state$
and *comp-merkle-bound*:
 $\bigwedge data\ attacker\text{-}state.$
 $Some\ (data,\ attacker\text{-}state) \in$
 $set\text{-}dist\ (execute\ (checked\text{-}staged\text{-}transcript\text{-}program\ A)$
 $adversary\text{-}initial\text{-}state) \implies$
 $wp\text{-}event\ verify\text{-}monad$
 $(partial\text{-}merkle\text{-}inconsistency\text{-}bad$
 $(verifier\text{-}state\text{-}from\text{-}adversary\ attacker\text{-}state$
 $(staged\text{-}proof\text{-}transcript\ data)))$
 $(verifier\text{-}state\text{-}from\text{-}adversary\ attacker\text{-}state$
 $(staged\text{-}proof\text{-}transcript\ data)) \leq$
 $Pc\ data\ attacker\text{-}state$
and *comp-without-same-bound*:
 $\bigwedge data\ attacker\text{-}state.$

Some (data, attacker-state) ∈
set-dist (execute (checked-staged-transcript-program A)
adversary-initial-state) ⇒
wp-event verify-monad
(composition-fri-verifier-tied-sampled-assignment-conflict-without-same-layer
(verifier-state-from-adversary attacker-state
(staged-proof-transcript data)))
(verifier-state-from-adversary attacker-state
(staged-proof-transcript data)) ≤
Cc data attacker-state
and comp-empty-bound:
 $\wedge$ *data attacker-state.*
Some (data, attacker-state) ∈
set-dist (execute (checked-staged-transcript-program A)
adversary-initial-state) ⇒
wp-event verify-monad
(composition-fri-reachable-verifier-tie-empty-header-gap
(verifier-state-from-adversary attacker-state
(staged-proof-transcript data)))
(verifier-state-from-adversary attacker-state
(staged-proof-transcript data)) ≤
Ec data attacker-state
and comp-gap-merkle-bound:
 $\wedge$ *data attacker-state.*
Some (data, attacker-state) ∈
set-dist (execute (checked-staged-transcript-program A)
adversary-initial-state) ⇒
wp-event verify-monad
(partial-merkle-inconsistency-bad
(verifier-state-from-adversary attacker-state
(staged-proof-transcript data)))
(verifier-state-from-adversary attacker-state
(staged-proof-transcript data)) ≤
Qc data attacker-state
and comp-candidate-available-bound:
 $\wedge$ *data attacker-state.*
Some (data, attacker-state) ∈
set-dist (execute (checked-staged-transcript-program A)
adversary-initial-state) ⇒
wp-event verify-monad
(composition-fri-verifier-tied-candidate-available-gap
(verifier-state-from-adversary attacker-state
(staged-proof-transcript data)))
(verifier-state-from-adversary attacker-state
(staged-proof-transcript data)) ≤
Tc data attacker-state
and comp-total-bound:
 $\wedge$ *data attacker-state.*
Some (data, attacker-state) ∈

$set-dist \ (execute \ (checked-staged-transcript-program \ A) \ adversary-initial-state) \implies$
 $Rc \ data \ attacker-state +$
 $((Pc \ data \ attacker-state + 0) +$
 $Cc \ data \ attacker-state + Cc \ data \ attacker-state +$
 $Cc \ data \ attacker-state + Pc \ data \ attacker-state) +$
 $0) +$
 $(Ec \ data \ attacker-state +$
 $(Qc \ data \ attacker-state + Tc \ data \ attacker-state))$
 $\leq composition-fri-error$
and *support*:
 $Some \ (data, \ attacker-state) \in$
 $set-dist \ (execute \ (checked-staged-transcript-program \ A) \ adversary-initial-state)$
shows
 $composition-fri-reachable-partial-candidate-reduction-assumption$
 $(verifier-state-from-adversary \ attacker-state$
 $(staged-proof-transcript \ data))$
by (rule
 $composition-fri-reachable-partial-candidate-reduction-from-split-without-same-bounds)$
 $(rule \ comp-sampled-bound[OF \ support],$
 $rule \ comp-merkle-bound[OF \ support],$
 $rule \ comp-without-same-bound[OF \ support],$
 $rule \ comp-empty-bound[OF \ support],$
 $rule \ comp-gap-merkle-bound[OF \ support],$
 $rule \ comp-candidate-available-bound[OF \ support],$
 $rule \ comp-total-bound[OF \ support])$

lemma *active-fri-reductions-from-without-same-and-untied-bounds*:
fixes
 $Rt \ Zt \ Pt \ Ht \ Ct \ Qt \ Ut \ Rc \ Pc \ Cc \ Ec \ Qc \ Tc ::$
 $('f \ staged-proof-data \Rightarrow 'f \ protocol-channel \Rightarrow prob)$
assumes *trace-sampled-bound*:
 $\bigwedge data \ attacker-state.$
 $Some \ (data, \ attacker-state) \in$
 $set-dist \ (execute \ (checked-staged-transcript-program \ A) \ adversary-initial-state) \implies$
 $wp-event \ verify-monad$
 $(trace-fri-bad-with-header-tied-sampled-layer-chain$
 $(verifier-state-from-adversary \ attacker-state$
 $(staged-proof-transcript \ data)))$
 $(verifier-state-from-adversary \ attacker-state$
 $(staged-proof-transcript \ data)) \leq$
 $Rt \ data \ attacker-state$
and *trace-zero-bound*:
 $\bigwedge data \ attacker-state.$
 $Some \ (data, \ attacker-state) \in$
 $set-dist \ (execute \ (checked-staged-transcript-program \ A) \ adversary-initial-state) \implies$

wp-event verify-monad
 (trace-fri-header-tied-zero-round-final-obstruction
 (verifier-state-from-adversary attacker-state
 (staged-proof-transcript data)))
 (verifier-state-from-adversary attacker-state
 (staged-proof-transcript data)) ≤
 Zt data attacker-state
and trace-header-merkle-bound:
 ∧ data attacker-state.
 Some (data, attacker-state) ∈
 set-dist (execute (checked-staged-transcript-program A)
 adversary-initial-state) ⇒
wp-event verify-monad
 (partial-merkle-inconsistency-bad
 (verifier-state-from-adversary attacker-state
 (staged-proof-transcript data)))
 (verifier-state-from-adversary attacker-state
 (staged-proof-transcript data)) ≤
 Pt data attacker-state
and trace-structural-bound:
 ∧ data attacker-state.
 Some (data, attacker-state) ∈
 set-dist (execute (checked-staged-transcript-program A)
 adversary-initial-state) ⇒
wp-event verify-monad
 (trace-fri-header-tied-sibling-mismatch-structural-gap
 (verifier-state-from-adversary attacker-state
 (staged-proof-transcript data)))
 (verifier-state-from-adversary attacker-state
 (staged-proof-transcript data)) ≤
 Ht data attacker-state
and trace-without-same-bound:
 ∧ data attacker-state.
 Some (data, attacker-state) ∈
 set-dist (execute (checked-staged-transcript-program A)
 adversary-initial-state) ⇒
wp-event verify-monad
 (trace-fri-header-tied-sampled-assignment-conflict-without-same-layer
 (verifier-state-from-adversary attacker-state
 (staged-proof-transcript data)))
 (verifier-state-from-adversary attacker-state
 (staged-proof-transcript data)) ≤
 Ct data attacker-state
and trace-gap-merkle-bound:
 ∧ data attacker-state.
 Some (data, attacker-state) ∈
 set-dist (execute (checked-staged-transcript-program A)
 adversary-initial-state) ⇒
wp-event verify-monad

$(\text{partial-merkle-inconsistency-bad}$
 $(\text{verifier-state-from-adversary attacker-state}$
 $(\text{staged-proof-transcript data})))$
 $(\text{verifier-state-from-adversary attacker-state}$
 $(\text{staged-proof-transcript data})) \leq$
 $Qt \text{ data attacker-state}$
and $\text{trace-untied-bound:}$
 $\wedge \text{data attacker-state.}$
 $\text{Some (data, attacker-state)} \in$
 $\text{set-dist (execute (checked-staged-transcript-program A)}$
 $\text{adversary-initial-state)} \implies$
 $\text{wp-event verify-monad}$
 $(\text{trace-fri-header-tied-low-candidate-untied-reachable-witness-gap}$
 $(\text{verifier-state-from-adversary attacker-state}$
 $(\text{staged-proof-transcript data})))$
 $(\text{verifier-state-from-adversary attacker-state}$
 $(\text{staged-proof-transcript data})) \leq$
 $Ut \text{ data attacker-state}$
and $\text{trace-total-bound:}$
 $\wedge \text{data attacker-state.}$
 $\text{Some (data, attacker-state)} \in$
 $\text{set-dist (execute (checked-staged-transcript-program A)}$
 $\text{adversary-initial-state)} \implies$
 $Rt \text{ data attacker-state} +$
 $(((((Pt \text{ data attacker-state} + Ht \text{ data attacker-state}) +$
 $Pt \text{ data attacker-state}) +$
 $Ct \text{ data attacker-state} + Ct \text{ data attacker-state} +$
 $Ct \text{ data attacker-state}) +$
 $Pt \text{ data attacker-state}) +$
 $0 + Zt \text{ data attacker-state}) +$
 $(Qt \text{ data attacker-state} + Ut \text{ data attacker-state})$
 $\leq \text{trace-fri-error}$
and $\text{comp-sampled-bound:}$
 $\wedge \text{data attacker-state.}$
 $\text{Some (data, attacker-state)} \in$
 $\text{set-dist (execute (checked-staged-transcript-program A)}$
 $\text{adversary-initial-state)} \implies$
 $\text{wp-event verify-monad}$
 $(\text{composition-fri-bad-with-verifier-tied-sampled-layer-chain}$
 $(\text{verifier-state-from-adversary attacker-state}$
 $(\text{staged-proof-transcript data})))$
 $(\text{verifier-state-from-adversary attacker-state}$
 $(\text{staged-proof-transcript data})) \leq$
 $Rc \text{ data attacker-state}$
and $\text{comp-merkle-bound:}$
 $\wedge \text{data attacker-state.}$
 $\text{Some (data, attacker-state)} \in$
 $\text{set-dist (execute (checked-staged-transcript-program A)}$
 $\text{adversary-initial-state)} \implies$

$wp\text{-}event\ verify\text{-}monad$
 $(partial\text{-}merkle\text{-}inconsistency\text{-}bad$
 $(verifier\text{-}state\text{-}from\text{-}adversary\ attacker\text{-}state$
 $(staged\text{-}proof\text{-}transcript\ data)))$
 $(verifier\text{-}state\text{-}from\text{-}adversary\ attacker\text{-}state$
 $(staged\text{-}proof\text{-}transcript\ data)) \leq$
 $Pc\ data\ attacker\text{-}state$
and $comp\text{-}without\text{-}same\text{-}bound$:
 $\wedge data\ attacker\text{-}state.$
 $Some\ (data,\ attacker\text{-}state) \in$
 $set\text{-}dist\ (execute\ (checked\text{-}staged\text{-}transcript\text{-}program\ A)$
 $adversary\text{-}initial\text{-}state) \implies$
 $wp\text{-}event\ verify\text{-}monad$
 $(composition\text{-}fri\text{-}verifier\text{-}tied\text{-}sampled\text{-}assignment\text{-}conflict\text{-}without\text{-}same\text{-}layer$
 $(verifier\text{-}state\text{-}from\text{-}adversary\ attacker\text{-}state$
 $(staged\text{-}proof\text{-}transcript\ data)))$
 $(verifier\text{-}state\text{-}from\text{-}adversary\ attacker\text{-}state$
 $(staged\text{-}proof\text{-}transcript\ data)) \leq$
 $Cc\ data\ attacker\text{-}state$
and $comp\text{-}empty\text{-}bound$:
 $\wedge data\ attacker\text{-}state.$
 $Some\ (data,\ attacker\text{-}state) \in$
 $set\text{-}dist\ (execute\ (checked\text{-}staged\text{-}transcript\text{-}program\ A)$
 $adversary\text{-}initial\text{-}state) \implies$
 $wp\text{-}event\ verify\text{-}monad$
 $(composition\text{-}fri\text{-}reachable\text{-}verifier\text{-}tie\text{-}empty\text{-}header\text{-}gap$
 $(verifier\text{-}state\text{-}from\text{-}adversary\ attacker\text{-}state$
 $(staged\text{-}proof\text{-}transcript\ data)))$
 $(verifier\text{-}state\text{-}from\text{-}adversary\ attacker\text{-}state$
 $(staged\text{-}proof\text{-}transcript\ data)) \leq$
 $Ec\ data\ attacker\text{-}state$
and $comp\text{-}gap\text{-}merkle\text{-}bound$:
 $\wedge data\ attacker\text{-}state.$
 $Some\ (data,\ attacker\text{-}state) \in$
 $set\text{-}dist\ (execute\ (checked\text{-}staged\text{-}transcript\text{-}program\ A)$
 $adversary\text{-}initial\text{-}state) \implies$
 $wp\text{-}event\ verify\text{-}monad$
 $(partial\text{-}merkle\text{-}inconsistency\text{-}bad$
 $(verifier\text{-}state\text{-}from\text{-}adversary\ attacker\text{-}state$
 $(staged\text{-}proof\text{-}transcript\ data)))$
 $(verifier\text{-}state\text{-}from\text{-}adversary\ attacker\text{-}state$
 $(staged\text{-}proof\text{-}transcript\ data)) \leq$
 $Qc\ data\ attacker\text{-}state$
and $comp\text{-}candidate\text{-}available\text{-}bound$:
 $\wedge data\ attacker\text{-}state.$
 $Some\ (data,\ attacker\text{-}state) \in$
 $set\text{-}dist\ (execute\ (checked\text{-}staged\text{-}transcript\text{-}program\ A)$
 $adversary\text{-}initial\text{-}state) \implies$
 $wp\text{-}event\ verify\text{-}monad$

```

    (composition-fri-verifier-tied-candidate-available-gap
      (verifier-state-from-adversary attacker-state
        (staged-proof-transcript data)))
    (verifier-state-from-adversary attacker-state
      (staged-proof-transcript data)) ≤
  Tc data attacker-state
and comp-total-bound:
  ∧ data attacker-state.
    Some (data, attacker-state) ∈
      set-dist (execute (checked-staged-transcript-program A)
        adversary-initial-state) ⇒
  Rc data attacker-state +
    (((Pc data attacker-state + 0) +
      Cc data attacker-state + Cc data attacker-state +
      Cc data attacker-state + Pc data attacker-state) +
      0) +
    (Ec data attacker-state +
      (Qc data attacker-state + Tc data attacker-state))
    ≤ composition-fri-error
shows active-fri-reductions-for A
proof (rule active-fri-reductions-from-reachable)
fix data attacker-state
assume support:
  Some (data, attacker-state) ∈
    set-dist (execute (checked-staged-transcript-program A)
      adversary-initial-state)
have total:
  Rt data attacker-state +
    (Pt data attacker-state + Ht data attacker-state +
      Pt data attacker-state +
      Ct data attacker-state +
      Ct data attacker-state +
      Ct data attacker-state +
      Pt data attacker-state +
      0 +
      Zt data attacker-state) +
    (Qt data attacker-state + Ut data attacker-state)
    ≤ trace-fri-error
using trace-total-bound[OF support] by (simp add: add.assoc)
show trace-fri-reachable-partial-candidate-reduction-assumption
  (verifier-state-from-adversary attacker-state
    (staged-proof-transcript data))
using trace-sampled-bound[OF support]
  trace-zero-bound[OF support]
  trace-header-merkle-bound[OF support]
  trace-structural-bound[OF support]
  trace-without-same-bound[OF support]
  trace-gap-merkle-bound[OF support]
  trace-untied-bound[OF support]

```

```

    total
  by (rule
    trace-fri-reachable-partial-candidate-reduction-from-header-without-same-and-untied
    [where  $R = R_t$  data attacker-state
      and  $Z = Z_t$  data attacker-state
      and  $P = P_t$  data attacker-state
      and  $H = H_t$  data attacker-state
      and  $C = C_t$  data attacker-state
      and  $Q = Q_t$  data attacker-state
      and  $U = U_t$  data attacker-state])
  next
  fix data attacker-state
  assume support:
    Some (data, attacker-state) ∈
      set-dist (execute (checked-staged-transcript-program A)
        adversary-initial-state)
  have total:
    Rc data attacker-state +
      (Pc data attacker-state + 0 + Cc data attacker-state +
        Cc data attacker-state +
        Cc data attacker-state +
        Pc data attacker-state +
        0) +
      (Ec data attacker-state +
        (Qc data attacker-state + Tc data attacker-state))
    ≤ composition-fri-error
  using comp-total-bound[OF support] by (simp add: add.assoc)
  show composition-fri-reachable-partial-candidate-reduction-assumption
    (verifier-state-from-adversary attacker-state
      (staged-proof-transcript data))
  using comp-sampled-bound[OF support]
    comp-merkle-bound[OF support]
    comp-without-same-bound[OF support]
    comp-empty-bound[OF support]
    comp-gap-merkle-bound[OF support]
    comp-candidate-available-bound[OF support]
    total
  by (rule
    composition-fri-reachable-partial-candidate-reduction-from-split-without-same-bounds
    [where  $R = R_c$  data attacker-state
      and  $P = P_c$  data attacker-state
      and  $C = C_c$  data attacker-state
      and  $E = E_c$  data attacker-state
      and  $Q = Q_c$  data attacker-state
      and  $T = T_c$  data attacker-state])
qed

```

lemma *active-fri-reductions-from-without-same-hash-root-query-bounds:*
fixes

Rt Zt Pt Ht Ct Qt Kt Ut Vt Rc Pc Cc Ec Qc Tc ::
('f staged-proof-data ⇒ 'f protocol-channel ⇒ prob)
assumes *trace-sampled-bound:*
 $\wedge$ *data attacker-state.*
Some (data, attacker-state) ∈
set-dist (execute (checked-staged-transcript-program A)
adversary-initial-state) ⇒
wp-event verify-monad
(trace-fri-bad-with-header-tied-sampled-layer-chain
(verifier-state-from-adversary attacker-state
(staged-proof-transcript data)))
(verifier-state-from-adversary attacker-state
(staged-proof-transcript data)) ≤
Rt data attacker-state
and *trace-zero-bound:*
 $\wedge$ *data attacker-state.*
Some (data, attacker-state) ∈
set-dist (execute (checked-staged-transcript-program A)
adversary-initial-state) ⇒
wp-event verify-monad
(trace-fri-header-tied-zero-round-final-obstruction
(verifier-state-from-adversary attacker-state
(staged-proof-transcript data)))
(verifier-state-from-adversary attacker-state
(staged-proof-transcript data)) ≤
Zt data attacker-state
and *trace-header-merkle-bound:*
 $\wedge$ *data attacker-state.*
Some (data, attacker-state) ∈
set-dist (execute (checked-staged-transcript-program A)
adversary-initial-state) ⇒
wp-event verify-monad
(partial-merkle-inconsistency-bad
(verifier-state-from-adversary attacker-state
(staged-proof-transcript data)))
(verifier-state-from-adversary attacker-state
(staged-proof-transcript data)) ≤
Pt data attacker-state
and *trace-structural-bound:*
 $\wedge$ *data attacker-state.*
Some (data, attacker-state) ∈
set-dist (execute (checked-staged-transcript-program A)
adversary-initial-state) ⇒
wp-event verify-monad
(trace-fri-header-tied-sibling-mismatch-structural-gap
(verifier-state-from-adversary attacker-state
(staged-proof-transcript data)))
(verifier-state-from-adversary attacker-state
(staged-proof-transcript data)) ≤

Ht data attacker-state
and *trace-without-same-bound:*
 $\wedge$ *data attacker-state.*
Some (data, attacker-state) $\in$
set-dist (execute (checked-staged-transcript-program A)
adversary-initial-state) $\implies$
wp-event verify-monad
(trace-fri-header-tied-sampled-assignment-conflict-without-same-layer
(verifier-state-from-adversary attacker-state
(staged-proof-transcript data)))
(verifier-state-from-adversary attacker-state
(staged-proof-transcript data)) $\leq$
Ct data attacker-state
and *trace-gap-merkle-bound:*
 $\wedge$ *data attacker-state.*
Some (data, attacker-state) $\in$
set-dist (execute (checked-staged-transcript-program A)
adversary-initial-state) $\implies$
wp-event verify-monad
(partial-merkle-inconsistency-bad
(verifier-state-from-adversary attacker-state
(staged-proof-transcript data)))
(verifier-state-from-adversary attacker-state
(staged-proof-transcript data)) $\leq$
Qt data attacker-state
and *trace-hash-bound:*
 $\wedge$ *data attacker-state.*
Some (data, attacker-state) $\in$
set-dist (execute (checked-staged-transcript-program A)
adversary-initial-state) $\implies$
wp-event verify-monad
(hash-map-output-collision-bad
(verifier-state-from-adversary attacker-state
(staged-proof-transcript data)))
(verifier-state-from-adversary attacker-state
(staged-proof-transcript data)) $\leq$
Kt data attacker-state
and *trace-root-bound:*
 $\wedge$ *data attacker-state.*
Some (data, attacker-state) $\in$
set-dist (execute (checked-staged-transcript-program A)
adversary-initial-state) $\implies$
wp-event verify-monad
(trace-fri-header-tied-low-candidate-untied-root-gap
(verifier-state-from-adversary attacker-state
(staged-proof-transcript data)))
(verifier-state-from-adversary attacker-state
(staged-proof-transcript data)) $\leq$
Ut data attacker-state

and *trace-query-bound*:
 $\wedge$ *data attacker-state*.
Some (*data*, *attacker-state*) $\in$
set-dist (*execute* (*checked-staged-transcript-program* *A*)
adversary-initial-state) $\implies$
wp-event verify-monad
(*trace-fri-header-tied-low-candidate-untied-query-gap*
(*verifier-state-from-adversary attacker-state*
(*staged-proof-transcript data*)))
(*verifier-state-from-adversary attacker-state*
(*staged-proof-transcript data*)) $\leq$
Vt data attacker-state
and *trace-total-bound*:
 $\wedge$ *data attacker-state*.
Some (*data*, *attacker-state*) $\in$
set-dist (*execute* (*checked-staged-transcript-program* *A*)
adversary-initial-state) $\implies$
Rt data attacker-state +
((((*Pt data attacker-state* + *Ht data attacker-state*) +
Pt data attacker-state) +
Ct data attacker-state + *Ct data attacker-state* +
Ct data attacker-state) +
Pt data attacker-state) +
0 + *Zt data attacker-state*) +
(*Qt data attacker-state* +
(*Kt data attacker-state* +
(*Ut data attacker-state* + *Vt data attacker-state*)))
 $\leq$ *trace-fri-error*
and *comp-sampled-bound*:
 $\wedge$ *data attacker-state*.
Some (*data*, *attacker-state*) $\in$
set-dist (*execute* (*checked-staged-transcript-program* *A*)
adversary-initial-state) $\implies$
wp-event verify-monad
(*composition-fri-bad-with-verifier-tied-sampled-layer-chain*
(*verifier-state-from-adversary attacker-state*
(*staged-proof-transcript data*)))
(*verifier-state-from-adversary attacker-state*
(*staged-proof-transcript data*)) $\leq$
Rc data attacker-state
and *comp-merkle-bound*:
 $\wedge$ *data attacker-state*.
Some (*data*, *attacker-state*) $\in$
set-dist (*execute* (*checked-staged-transcript-program* *A*)
adversary-initial-state) $\implies$
wp-event verify-monad
(*partial-merkle-inconsistency-bad*
(*verifier-state-from-adversary attacker-state*
(*staged-proof-transcript data*)))

$(\text{verifier-state-from-adversary attacker-state}$
 $(\text{staged-proof-transcript data})) \leq$
 $Pc \text{ data attacker-state}$
and *comp-without-same-bound*:
 $\wedge \text{data attacker-state.}$
 $\text{Some (data, attacker-state)} \in$
 $\text{set-dist (execute (checked-staged-transcript-program A)}$
 $\text{adversary-initial-state)} \implies$
 $\text{wp-event verify-monad}$
 $(\text{composition-fri-verifier-tied-sampled-assignment-conflict-without-same-layer}$
 $(\text{verifier-state-from-adversary attacker-state}$
 $(\text{staged-proof-transcript data})))$
 $(\text{verifier-state-from-adversary attacker-state}$
 $(\text{staged-proof-transcript data})) \leq$
 $Cc \text{ data attacker-state}$
and *comp-empty-bound*:
 $\wedge \text{data attacker-state.}$
 $\text{Some (data, attacker-state)} \in$
 $\text{set-dist (execute (checked-staged-transcript-program A)}$
 $\text{adversary-initial-state)} \implies$
 $\text{wp-event verify-monad}$
 $(\text{composition-fri-reachable-verifier-tie-empty-header-gap}$
 $(\text{verifier-state-from-adversary attacker-state}$
 $(\text{staged-proof-transcript data})))$
 $(\text{verifier-state-from-adversary attacker-state}$
 $(\text{staged-proof-transcript data})) \leq$
 $Ec \text{ data attacker-state}$
and *comp-gap-merkle-bound*:
 $\wedge \text{data attacker-state.}$
 $\text{Some (data, attacker-state)} \in$
 $\text{set-dist (execute (checked-staged-transcript-program A)}$
 $\text{adversary-initial-state)} \implies$
 $\text{wp-event verify-monad}$
 $(\text{partial-merkle-inconsistency-bad}$
 $(\text{verifier-state-from-adversary attacker-state}$
 $(\text{staged-proof-transcript data})))$
 $(\text{verifier-state-from-adversary attacker-state}$
 $(\text{staged-proof-transcript data})) \leq$
 $Qc \text{ data attacker-state}$
and *comp-candidate-available-bound*:
 $\wedge \text{data attacker-state.}$
 $\text{Some (data, attacker-state)} \in$
 $\text{set-dist (execute (checked-staged-transcript-program A)}$
 $\text{adversary-initial-state)} \implies$
 $\text{wp-event verify-monad}$
 $(\text{composition-fri-verifier-tied-candidate-available-gap}$
 $(\text{verifier-state-from-adversary attacker-state}$
 $(\text{staged-proof-transcript data})))$
 $(\text{verifier-state-from-adversary attacker-state}$


```

    (staged-proof-transcript data)) ≤
    Tc data attacker-state
and comp-total-bound:
 $\wedge$ data attacker-state.
    Some (data, attacker-state) ∈
    set-dist (execute (checked-staged-transcript-program A)
    adversary-initial-state)  $\implies$ 
    Rc data attacker-state +
    (((Pc data attacker-state + 0) +
    Cc data attacker-state + Cc data attacker-state +
    Cc data attacker-state + Pc data attacker-state) +
    0) +
    (Ec data attacker-state +
    (Qc data attacker-state + Tc data attacker-state))
    ≤ composition-fri-error
shows active-fri-reductions-for A
apply (rule active-fri-reductions-from-without-same-and-untied-bounds
    [where Rt = Rt and Zt = Zt and Pt = Pt and Ht = Ht and Ct = Ct
    and Qt = Qt
    and Ut =
     $\lambda$ data attacker-state.
    Kt data attacker-state +
    (Ut data attacker-state + Vt data attacker-state)
    and Rc = Rc and Pc = Pc and Cc = Cc and Ec = Ec and Qc = Qc
    and Tc = Tc])
    apply (rule trace-sampled-bound, assumption)
    apply (rule trace-zero-bound, assumption)
    apply (rule trace-header-merkle-bound, assumption)
    apply (rule trace-structural-bound, assumption)
    apply (rule trace-without-same-bound, assumption)
    apply (rule trace-gap-merkle-bound, assumption)
    apply (rule
    wp-trace-fri-header-tied-low-candidate-untied-gap-bound-from-hash-root-query)
    apply (rule trace-hash-bound, assumption)
    apply (rule trace-root-bound, assumption)
    apply (rule trace-query-bound, assumption)
    apply (rule trace-total-bound, assumption)
    apply (rule comp-sampled-bound, assumption)
    apply (rule comp-merkle-bound, assumption)
    apply (rule comp-without-same-bound, assumption)
    apply (rule comp-empty-bound, assumption)
    apply (rule comp-gap-merkle-bound, assumption)
    apply (rule comp-candidate-available-bound, assumption)
    apply (rule comp-total-bound, assumption)
    done

end

end

```

```

theory Soundness-FRI-Composition-Replay-Gaps
  imports
    Soundness-FRI-Composition-Support
    Soundness-FRI-Generic-Low-Degree
begin

```

Narrow downstream layer for the remaining composition verifier-tied replay gaps. This avoids growing the composition residuals theory, which is already above the advisory size threshold.

```

context soundness
begin

```

```

lemma fri-same-mod-half-index-cases:
  fixes len i j :: nat
  assumes even-len:  $2 \text{ dvd } \text{len}$ 
    and i-bound:  $i < \text{len}$ 
    and j-bound:  $j < \text{len}$ 
    and same-mod:  $i \bmod (\text{len} \text{ div } 2) = j \bmod (\text{len} \text{ div } 2)$ 
  shows
     $i = j \vee$ 
     $i = \text{fri-sibling-index } \text{len } j \vee$ 
     $\text{fri-sibling-index } \text{len } i = j \vee$ 
     $\text{fri-sibling-index } \text{len } i = \text{fri-sibling-index } \text{len } j$ 
proof –
  let ?half =  $\text{len} \text{ div } 2$ 
  have len-eq:  $\text{len} = 2 * ?\text{half}$ 
    using even-len by simp
  have half-pos:  $0 < ?\text{half}$ 
    using i-bound len-eq by linarith
  consider
    (ff)  $i < ?\text{half}$   $j < ?\text{half}$ 
  | (fs)  $i < ?\text{half}$   $\neg j < ?\text{half}$ 
  | (sf)  $\neg i < ?\text{half}$   $j < ?\text{half}$ 
  | (ss)  $\neg i < ?\text{half}$   $\neg j < ?\text{half}$ 
    by blast
  then show ?thesis
proof cases
  case ff
    then have  $i = j$ 
      using same-mod half-pos by simp
    then show ?thesis by simp
next
  case fs
    have j-eq:  $j = \text{fri-sibling-index } \text{len } (j \bmod ?\text{half})$ 
      by (rule fri-sibling-index-mod-half-second-half
        [OF even-len j-bound fs(2)])
    have i-eq:  $j \bmod ?\text{half} = i$ 

```

```

    using same-mod fs(1) by simp
  have fri-sibling-index len i = j
    using j-eq i-eq by simp
  then show ?thesis by simp
next
case sf
have i-eq: i = fri-sibling-index len (i mod ?half)
  by (rule fri-sibling-index-mod-half-second-half
      [OF even-len i-bound sf(1)])
have j-eq: i mod ?half = j
  using same-mod sf(2) by simp
have i = fri-sibling-index len j
  using i-eq j-eq by simp
then show ?thesis by simp
next
case ss
have i-eq: i = fri-sibling-index len (i mod ?half)
  by (rule fri-sibling-index-mod-half-second-half
      [OF even-len i-bound ss(1)])
have j-eq: j = fri-sibling-index len (j mod ?half)
  by (rule fri-sibling-index-mod-half-second-half
      [OF even-len j-bound ss(2)])
have i = j
  using i-eq j-eq same-mod by simp
then show ?thesis by simp
qed
qed

lemma fri-same-mod-half-index-cases3:
  fixes len i j :: nat
  assumes even-len: 2 dvd len
    and i-bound: i < len
    and j-bound: j < len
    and same-mod: i mod (len div 2) = j mod (len div 2)
  shows
    i = j ∨
    i = fri-sibling-index len j ∨
    fri-sibling-index len i = j
proof -
  let ?half = len div 2
  have len-eq: len = 2 * ?half
    using even-len by simp
  have half-pos: 0 < ?half
    using i-bound len-eq by linarith
  consider
    (ff) i < ?half j < ?half
  | (fs) i < ?half ¬ j < ?half
  | (sf) ¬ i < ?half j < ?half
  | (ss) ¬ i < ?half ¬ j < ?half

```

```

    by blast
  then show ?thesis
proof cases
  case ff
  then have i = j
    using same-mod half-pos by simp
  then show ?thesis by simp
next
  case fs
  have j-eq: j = fri-sibling-index len (j mod ?half)
    by (rule fri-sibling-index-mod-half-second-half
        [OF even-len j-bound fs(2)])
  have i-eq: j mod ?half = i
    using same-mod fs(1) by simp
  have fri-sibling-index len i = j
    using j-eq i-eq by simp
  then show ?thesis by simp
next
  case sf
  have i-eq: i = fri-sibling-index len (i mod ?half)
    by (rule fri-sibling-index-mod-half-second-half
        [OF even-len i-bound sf(1)])
  have j-eq: i mod ?half = j
    using same-mod sf(2) by simp
  have i = fri-sibling-index len j
    using i-eq j-eq by simp
  then show ?thesis by simp
next
  case ss
  have i-eq: i = fri-sibling-index len (i mod ?half)
    by (rule fri-sibling-index-mod-half-second-half
        [OF even-len i-bound ss(1)])
  have j-eq: j = fri-sibling-index len (j mod ?half)
    by (rule fri-sibling-index-mod-half-second-half
        [OF even-len j-bound ss(2)])
  have i = j
    using i-eq j-eq same-mod by simp
  then show ?thesis by simp
qed
qed

lemma fri-sibling-index-involution:
  fixes len idx :: nat
  assumes even-len: 2 dvd len
    and idx-bound: idx < len
  shows fri-sibling-index len (fri-sibling-index len idx) = idx
proof -
  let ?half = len div 2
  have len-eq: len = 2 * ?half

```

```

    using even-len by simp
have half-pos: 0 < ?half
    using idx-bound len-eq by linarith
show ?thesis
proof (cases idx < ?half)
  case True
  then show ?thesis
    by (rule fri-sibling-index-involution-first-half[OF even-len])
next
  case False
  define k where k = idx - ?half
  have idx-eq: idx = ?half + k
    using False unfolding k-def by simp
  have k-bound: k < ?half
    using idx-bound len-eq idx-eq by linarith
  have sib-eq: fri-sibling-index len idx = k
  proof -
    have sum-eq: (?half + k + ?half) = k + len
      using len-eq by linarith
    have k-len: k < len
      using k-bound len-eq by linarith
    have (?half + k + ?half) mod len = (k + len) mod len
      by (simp only: sum-eq)
    also have ... = k
      using k-len by simp
    finally have mod-eq: (?half + k + ?half) mod len = k .
    then show ?thesis
      unfolding fri-sibling-index-def idx-eq
      using mod-eq by simp
  qed
have fri-sibling-index len (fri-sibling-index len idx) =
  fri-sibling-index len k
  using sib-eq by simp
also have ... = idx
  using fri-sibling-index-first-half[OF even-len k-bound]
  unfolding idx-eq by simp
finally show ?thesis .
qed
qed

```

```

lemma fri-sibling-fold-denominator-verifier:
  assumes len-pos: 0 < len
    and even-len: 2 dvd len
    and round: len * pw = clength * scale
  shows
    fri-fold-denominator ((h ^ fri-sibling-index len i) * shift) pw =
      - fri-fold-denominator ((h ^ i) * shift) pw
  using fri-sibling-domain-round[OF len-pos even-len round, of i]
  unfolding fri-sibling-index-def fri-fold-denominator-def

```

by *simp*

lemma *fri-fold-value-sibling-verifier*:

assumes *len-pos*: $0 < \text{len}$

and *even-len*: $2 \text{ dvd } \text{len}$

and *round*: $\text{len} * \text{pw} = \text{clength} * \text{scale}$

shows

fri-fold-value *b xn xp*

$(\text{fri-fold-denominator } ((h \wedge \text{fri-sibling-index len } i) * \text{shift}) \text{ pw}) =$

fri-fold-value *b xp xn*

$(\text{fri-fold-denominator } ((h \wedge i) * \text{shift}) \text{ pw})$

proof –

have *denom*:

fri-fold-denominator $((h \wedge \text{fri-sibling-index len } i) * \text{shift}) \text{ pw} =$

– *fri-fold-denominator* $((h \wedge i) * \text{shift}) \text{ pw}$

by (rule *fri-sibling-fold-denominator-verifier*

[*OF len-pos even-len round*])

show *?thesis*

unfolding *denom*

by (rule *fri-fold-value-swap-neg-denominator*)

qed

lemma *generic-fri-forced-next-value-eq-if-no-same-layer-conflict*:

assumes *no-same*:

$\neg$ *generic-fri-sampled-same-layer-opening-conflict* *roots challenges*
query-idxs round-layers

and *round-bound*: $\text{round-idx} < \text{length } \text{query-idxs}$

and *round-bound'*: $\text{round-idx}' < \text{length } \text{query-idxs}$

and *layer-bound*: $\text{layer-idx} < \text{length } \text{challenges}$

and *even-len*: $2 \text{ dvd } \text{fri-evidence-layer-len } \text{roots } \text{layer-idx}$

and *raw-bound*:

fri-evidence-layer-idx *roots query-idxs round-idx layer-idx* $<$
fri-evidence-layer-len *roots layer-idx*

and *raw-bound'*:

fri-evidence-layer-idx *roots query-idxs round-idx' layer-idx* $<$
fri-evidence-layer-len *roots layer-idx*

and *round-product*:

fri-evidence-layer-len *roots layer-idx* $* 2^{\text{layer-idx}} =$
clength $* \text{scale}$

and *same-next*:

fri-evidence-next-idx *roots query-idxs round-idx layer-idx* $=$
fri-evidence-next-idx *roots query-idxs round-idx' layer-idx*

and *forced*:

generic-fri-round-forced-next-value *roots challenges query-idxs*
round-layers round-idx layer-idx v

and *forced'*:

generic-fri-round-forced-next-value *roots challenges query-idxs*
round-layers round-idx' layer-idx v'

shows $v = v'$

```

proof –
  let ?len = fri-evidence-layer-len roots layer-idx
  let ?raw = fri-evidence-layer-idx roots query-idxs round-idx layer-idx
  let ?raw' =
    fri-evidence-layer-idx roots query-idxs round-idx' layer-idx
  let ?ch = challenges ! layer-idx
  let ?pw = 2 ^ layer-idx
  have len-pos: 0 < ?len
    using raw-bound by simp
  from forced obtain xp xp-path xn xn-path where step:
    fri-layer-step-evidence
      (roots ! layer-idx)
      ?ch
      ?len
      ?raw
      ?pw
      (fri-sibling-index ?len ?raw)
      xp xp-path xn xn-path
      (fri-evidence-next-idx roots query-idxs round-idx layer-idx)
      v
      (round-layers ! round-idx ! layer-idx)
    unfolding generic-fri-round-forced-next-value-def by blast
  have v-eq:
    v = fri-evidence-next-value roots challenges query-idxs round-idx
      layer-idx xp xn
    using fri-layer-step-evidenceD(3)[OF step]
    unfolding fri-evidence-next-value-def by simp
  have step-norm:
    fri-layer-step-evidence
      (roots ! layer-idx)
      ?ch
      ?len
      ?raw
      ?pw
      (fri-sibling-index ?len ?raw)
      xp xp-path xn xn-path
      (fri-evidence-next-idx roots query-idxs round-idx layer-idx)
      (fri-evidence-next-value roots challenges query-idxs round-idx
        layer-idx xp xn)
      (round-layers ! round-idx ! layer-idx)
    using step v-eq by simp
  from forced' obtain yp yp-path yn yn-path where step':
    fri-layer-step-evidence
      (roots ! layer-idx)
      ?ch
      ?len
      ?raw'
      ?pw
      (fri-sibling-index ?len ?raw')

```

```

  yp yp-path yn yn-path
  (fri-evidence-next-idx roots query-idxs round-idx' layer-idx)
  v'
  (round-layers ! round-idx' ! layer-idx)
unfolding generic-fri-round-forced-next-value-def by blast
have v'-eq:
  v' = fri-evidence-next-value roots challenges query-idxs round-idx'
    layer-idx yp yn
using fri-layer-step-evidenceD(3)[OF step']
unfolding fri-evidence-next-value-def by simp
have step'-norm:
  fri-layer-step-evidence
    (roots ! layer-idx)
    ?ch
    ?len
    ?raw'
    ?pw
    (fri-sibling-index ?len ?raw')
    yp yp-path yn yn-path
    (fri-evidence-next-idx roots query-idxs round-idx' layer-idx)
    (fri-evidence-next-value roots challenges query-idxs round-idx'
      layer-idx yp yn)
    (round-layers ! round-idx' ! layer-idx)
using step' v'-eq by simp
have same-mod: ?raw mod (?len div 2) = ?raw' mod (?len div 2)
using same-next unfolding fri-evidence-next-idx-def by simp
have idx-cases:
  ?raw = ?raw' ∨
  ?raw = fri-sibling-index ?len ?raw' ∨
  fri-sibling-index ?len ?raw = ?raw'
by (rule fri-same-mod-half-index-cases3
    [OF even-len raw-bound raw-bound' same-mod])
from idx-cases show ?thesis
proof
assume raw-eq: ?raw = ?raw'
have xp-eq: xp = yp
by (rule generic-fri-no-same-layer-opening-conflict-left-left
    [OF no-same round-bound round-bound' layer-bound step-norm
      step'-norm])
    (rule raw-eq)
have xn-eq: xn = yn
by (rule generic-fri-no-same-layer-opening-conflict-right-right
    [OF no-same round-bound round-bound' layer-bound step-norm
      step'-norm])
    (use raw-eq in simp)
show ?thesis
using v-eq v'-eq raw-eq xp-eq xn-eq
unfolding fri-evidence-next-value-def
by simp

```



```

next
  assume raw-or-sib:
    ?raw = fri-sibling-index ?len ?raw'  $\vee$ 
    fri-sibling-index ?len ?raw = ?raw'
  then show ?thesis
  proof
    assume raw-sib: ?raw = fri-sibling-index ?len ?raw'
    have sib-raw: fri-sibling-index ?len ?raw = ?raw'
      using raw-sib
      fri-sibling-index-involution[OF even-len raw-bound]
      by simp
    have xp-eq: xp = yn
      by (rule generic-fri-no-same-layer-opening-conflict-left-right
        [OF no-same round-bound round-bound' layer-bound step-norm
          step'-norm])
      (rule raw-sib)
    have xn-eq: xn = yp
      by (rule generic-fri-no-same-layer-opening-conflict-right-left
        [OF no-same round-bound round-bound' layer-bound step-norm
          step'-norm])
      (rule sib-raw)
    have fold-eq:
      fri-fold-value ?ch xp xn
      (fri-fold-denominator ((h  $\wedge$  ?raw) * shift) ?pw) =
      fri-fold-value ?ch yp yn
      (fri-fold-denominator ((h  $\wedge$  ?raw') * shift) ?pw)
    using fri-fold-value-sibling-verifier
      [OF len-pos even-len round-product,
        where i=?raw' and b=?ch and xn=yn and xp=yp]
      raw-sib xp-eq xn-eq
      by simp
    show ?thesis
      using v-eq v'-eq fold-eq
      unfolding fri-evidence-next-value-def
      by simp
  next
    assume sib-raw: fri-sibling-index ?len ?raw = ?raw'
    have raw-sib: ?raw = fri-sibling-index ?len ?raw'
      using sib-raw fri-sibling-index-involution[OF even-len raw-bound]
      by simp
    have xn-eq: xn = yp
      by (rule generic-fri-no-same-layer-opening-conflict-right-left
        [OF no-same round-bound round-bound' layer-bound step-norm
          step'-norm])
      (rule sib-raw)
    have xp-eq: xp = yn
      by (rule generic-fri-no-same-layer-opening-conflict-left-right
        [OF no-same round-bound round-bound' layer-bound step-norm
          step'-norm])

```

```

    (rule raw-sib)
  have fold-eq:
    fri-fold-value ?ch yp yn
    (fri-fold-denominator ((h ^ ?raw') * shift) ?pw) =
    fri-fold-value ?ch xp xn
    (fri-fold-denominator ((h ^ ?raw) * shift) ?pw)
  using fri-fold-value-sibling-verifier
  [OF len-pos even-len round-product,
   where i=?raw and b=?ch and xn=xn and xp=xp]
  sib-raw xp-eq xn-eq
  by simp
show ?thesis
  using v-eq v'-eq fold-eq
  unfolding fri-evidence-next-value-def
  by simp
qed
qed
qed

lemma accepted-fri-opening-transcript-composition-layer-arithmetic:
  assumes fri-openings:
    accepted-fri-opening-transcript s out trace-roots trace-bs trace-final
    dg composition-roots composition-bs composition-final query-idxs
    trace-round-layers composition-round-layers
  and degree-bound: to-nat dg ≤ maxDegree
  and layer-bound: layer-idx < length composition-bs
  shows 2 dvd fri-evidence-layer-len composition-roots layer-idx
  and fri-evidence-layer-len composition-roots layer-idx * 2 ^ layer-idx =
    clength * scale
proof -
  obtain N where eval-power: clength * scale = 2 ^ N
  using eval-domain-length-power by blast
  have bs-len: length composition-bs = length composition-roots
  using accepted-fri-opening-transcript-shapes(4)[OF fri-openings] .
  have roots-bound: layer-idx < length composition-roots
  using layer-bound bs-len by simp
  have roots-le: length composition-roots ≤ N
proof -
  have len: length composition-roots = ceil-log (to-nat dg + 1)
  using accepted-fri-opening-transcript-shapes(3)[OF fri-openings] .
  have to-nat dg + 1 ≤ clength * scale
  using degree-bound maxDegree-less-eval-domain by linarith
  also have ... = 2 ^ N
  by (rule eval-power)
  finally have ceil-log (to-nat dg + 1) ≤ N
  by (rule ceil-log-le-power)
  then show ?thesis
  using len by simp
qed

```

```

have suc-le: Suc layer-idx ≤ N
  using roots-bound roots-le by simp
have dvd-eval-suc: (2::nat) ^ Suc layer-idx dvd clength * scale
  using le-imp-power-dvd[OF suc-le] eval-power by simp
show 2 dvd fri-evidence-layer-len composition-roots layer-idx
  unfolding fri-evidence-layer-len-def
  by (rule fri-layer-lengths-even-if-dvd[OF roots-bound dvd-eval-suc])
have le-N: layer-idx ≤ N
  using roots-bound roots-le by simp
have dvd-eval: (2::nat) ^ layer-idx dvd clength * scale
  using le-imp-power-dvd[OF le-N] eval-power by simp
show fri-evidence-layer-len composition-roots layer-idx *
  2 ^ layer-idx = clength * scale
  unfolding fri-evidence-layer-len-def
  by (rule fri-layer-lengths-round-product-if-dvd
    [OF roots-bound dvd-eval])
qed

lemma accepted-fri-opening-transcript-composition-evidence-layer-idx-bound:
  assumes fri-openings:
    accepted-fri-opening-transcript s out trace-roots trace-bs trace-final
    dg composition-roots composition-bs composition-final query-idxs
    trace-round-layers composition-round-layers
  and degree-bound: to-nat dg ≤ maxDegree
  and round-bound: round-idx < length query-idxs
  and layer-bound: layer-idx < length composition-roots
  shows
    fri-evidence-layer-idx composition-roots query-idxs round-idx layer-idx <
      fri-evidence-layer-len composition-roots layer-idx
proof -
  obtain raw-idxs query-chunks query-state where
    len-raw: length raw-idxs = rounds
  and query-idxs-eq:
    query-idxs = map (λraw. index (to-nat raw)) raw-idxs
  using fri-openings
  by (auto simp: accepted-fri-opening-transcript-def)
have len-query: length query-idxs = rounds
  using accepted-fri-opening-transcript-shapes(7)[OF fri-openings] .
have round-raw-bound: round-idx < length raw-idxs
  using round-bound len-query len-raw by simp
have query-idx-eq:
  query-idxs ! round-idx = index (to-nat (raw-idxs ! round-idx))
  using query-idxs-eq round-raw-bound by simp
have raw-layer:
  0 < fri-layer-lengths (length composition-roots)
    (clength * scale) ! layer-idx ∧
  fri-layer-indices (length composition-roots)
    (index (to-nat (raw-idxs ! round-idx))) (clength * scale) !
    layer-idx <

```

```

    fri-layer-lengths (length composition-roots)
      (clength * scale) ! layer-idx
  by (rule accepted-fri-opening-transcript-composition-raw-layer-bound
      [OF fri-openings degree-bound layer-bound])
show ?thesis
  using raw-layer query-idx-eq
  unfolding fri-evidence-layer-idx-def fri-evidence-layer-len-def
  by simp
qed

```

lemma *generic-fri-forced-next-value-eq-from-recorded-step*:

assumes *forced*:

generic-fri-round-forced-next-value roots challenges query-idxs
round-layers round-idx layer-idx v

and *step*:

fri-layer-step-evidence
 (*roots ! layer-idx*)
 (*challenges ! layer-idx*)
 (*fri-evidence-layer-len roots layer-idx*)
 (*fri-evidence-layer-idx roots query-idxs round-idx layer-idx*)
 ($2 \wedge \text{layer-idx}$)
 (*fri-sibling-index (fri-evidence-layer-len roots layer-idx)*
 (*fri-evidence-layer-idx roots query-idxs round-idx layer-idx*))
xp xp-path xn xn-path
 (*fri-evidence-next-idx roots query-idxs round-idx layer-idx*)
v'
 (*round-layers ! round-idx ! layer-idx*)

shows $v = v'$

proof —

from *forced* **obtain** *yp yp-path yn yn-path* **where** *forced-step*:

fri-layer-step-evidence
 (*roots ! layer-idx*)
 (*challenges ! layer-idx*)
 (*fri-evidence-layer-len roots layer-idx*)
 (*fri-evidence-layer-idx roots query-idxs round-idx layer-idx*)
 ($2 \wedge \text{layer-idx}$)
 (*fri-sibling-index (fri-evidence-layer-len roots layer-idx)*
 (*fri-evidence-layer-idx roots query-idxs round-idx layer-idx*))
yp yp-path yn yn-path
 (*fri-evidence-next-idx roots query-idxs round-idx layer-idx*)
v
 (*round-layers ! round-idx ! layer-idx*)

unfolding *generic-fri-round-forced-next-value-def* **by** *blast*

have *yp-eq*: $yp = xp$

by (rule *fri-layer-opening-chunk-values-unique*(1))
 (rule *fri-layer-step-evidenceD*(4)[OF *forced-step*],
 rule *fri-layer-step-evidenceD*(4)[OF *step*])

have *yn-eq*: $yn = xn$

by (rule *fri-layer-opening-chunk-values-unique*(2))

```

    (rule fri-layer-step-evidenceD(4)[OF forced-step],
     rule fri-layer-step-evidenceD(4)[OF step])
  show ?thesis
  using fri-layer-step-evidenceD(3)[OF forced-step]
        fri-layer-step-evidenceD(3)[OF step]
        yp-eq yn-eq
  by simp
qed

lemma fri-layer-indices-tail-nth:
  fri-layer-indices (Suc j) (q mod (len div 2)) (len div 2) ! j =
    fri-layer-indices (Suc j) q len ! j mod
      (fri-layer-lengths (Suc j) len ! j div 2)
  by (induction j arbitrary: q len) simp-all

lemma fri-layer-indices-Suc-nth:
  assumes layer-bound: Suc j < n
  shows
    fri-layer-indices n q len ! Suc j =
      fri-layer-indices n q len ! j mod
        (fri-layer-lengths n len ! j div 2)
  proof -
    have idx-Suc:
      fri-layer-indices n q len ! Suc j =
        fri-layer-indices (Suc (Suc j)) q len ! Suc j
      using fri-layer-indices-prefix-nth[OF layer-bound, of q len] by simp
    have idx-j:
      fri-layer-indices n q len ! j =
        fri-layer-indices (Suc j) q len ! j
      using fri-layer-indices-prefix-nth[of j n q len] layer-bound by simp
    have len-j:
      fri-layer-lengths n len ! j =
        fri-layer-lengths (Suc j) len ! j
      using fri-layer-lengths-prefix-nth[of j n len] layer-bound by simp
    show ?thesis
      unfolding idx-Suc idx-j len-j
      using fri-layer-indices-tail-nth[of j q len]
      by simp
  qed

lemma fri-evidence-layer-idx-Suc:
  assumes layer-bound: Suc layer-idx < length roots
  shows
    fri-evidence-layer-idx roots query-idxs round-idx (Suc layer-idx) =
      fri-evidence-next-idx roots query-idxs round-idx layer-idx
  using fri-layer-indices-Suc-nth[OF layer-bound,
    of query-idxs ! round-idx clength * scale]
  unfolding fri-evidence-layer-idx-def fri-evidence-next-idx-def
    fri-evidence-layer-len-def

```

by simp

lemma *mfold-fri-layer-openings-successor-prefix-step*:

fixes *s s-suc* :: ('f, 'a) *protocol-channel-scheme*

assumes *prefix-suc*:

Some ((idx-suc, x-suc, len-suc, pw-suc), s-suc) ∈
set-dist

(execute
(mfold (idx, x, len, pw)
(take (Suc j) (map fri-layer-opening-step bfs))) s)

and *j-bound*: *j < length bfs*

obtains *idx-j x-j len-j pw-j b rt xp xp-path xn xn-path s-j* **where**

bfs ! j = (b, rt)

Some ((idx-j, x-j, len-j, pw-j), s-j) ∈
set-dist

(execute
(mfold (idx, x, len, pw)
(take j (map fri-layer-opening-step bfs))) s)

Some ((idx-suc, x-suc, len-suc, pw-suc), s-suc) ∈
set-dist

(execute
(fri-layer-opening-step (b, rt) (idx-j, x-j, len-j, pw-j))
s-j)

fri-layer-step-evidence rt b len-j idx-j pw-j
(fri-sibling-index len-j idx-j) xp xp-path xn xn-path
idx-suc x-suc ([xp] @ xp-path @ [xn] @ xn-path)

proof –

let *?bfs = take (Suc j) bfs*

have *j-bound-take*: *j < length ?bfs*

using *j-bound* **by** *simp*

have *take-map*:

take (Suc j) (map fri-layer-opening-step bfs) =
map fri-layer-opening-step ?bfs

by (*simp add: take-map*)

show *?thesis*

proof (*rule mfold-fri-layer-openings-decomp-at*
[OF prefix-suc[unfolded take-map] j-bound-take])

fix *idx-j x-j len-j pw-j out-j s-j s-j-suc*

assume *prefix*:

Some ((idx-j, x-j, len-j, pw-j), s-j) ∈
set-dist

(execute
(mfold (idx, x, len, pw)
(take j (map fri-layer-opening-step ?bfs))) s)

and *selected*:

Some (out-j, s-j-suc) ∈
set-dist

(execute
(fri-layer-opening-step (?bfs ! j)

```

      (idx-j, x-j, len-j, pw-j)) s-j)
and suffix:
  Some ((idx-suc, x-suc, len-suc, pw-suc), s-suc) ∈
    set-dist
      (execute
        (mfold out-j
          (drop (Suc j) (map fri-layer-opening-step ?bfs))) s-j-suc)
have drop-empty:
  drop (Suc j) (map fri-layer-opening-step ?bfs) = []
  by simp
have out-j-eq: out-j = (idx-suc, x-suc, len-suc, pw-suc)
  and s-suc-eq: s-suc = s-j-suc
  using suffix unfolding drop-empty by simp-all
have take-prefix-list: take j ?bfs = take j bfs
  by (simp add: take-take)
have take-prefix:
  take j (map fri-layer-opening-step ?bfs) =
    take j (map fri-layer-opening-step bfs)
proof (rule nth-equalityI)
  show length (take j (map fri-layer-opening-step ?bfs)) =
    length (take j (map fri-layer-opening-step bfs))
  by simp
next
  fix i
  assume i-bound: i < length (take j (map fri-layer-opening-step ?bfs))
  then have i < j
  by simp
  then show take j (map fri-layer-opening-step ?bfs) ! i =
    take j (map fri-layer-opening-step bfs) ! i
  using j-bound by simp
qed
have bfs-j: ?bfs ! j = bfs ! j
  using j-bound by simp
obtain b rt where bf-eq: bfs ! j = (b, rt)
  by (cases bfs ! j)
show ?thesis
proof (rule fri-layer-opening-step-outcome-evidence
  [OF selected[unfolded bfs-j bf-eq out-j-eq]])
  fix xp xp-path xn xn-path
  assume xp-eq: xp = x-j
  and next-idx-eq: idx-suc = idx-j mod (len-j div 2)
  and next-len-eq: len-suc = len-j div 2
  and next-pow-eq: pw-suc = pw-j + pw-j
  and step:
    fri-layer-step-evidence rt b len-j idx-j pw-j
    (fri-sibling-index len-j idx-j) xp xp-path xn xn-path
    idx-suc x-suc ([xp] @ xp-path @ [xn] @ xn-path)
have selected-suc:
  Some ((idx-suc, x-suc, len-suc, pw-suc), s-suc) ∈

```

```

    set-dist
    (execute
      (fri-layer-opening-step (b, rt)
        (idx-j, x-j, len-j, pw-j)) s-j)
    using selected[unfolded bfs-j bf-eq out-j-eq] s-suc-eq by simp
show ?thesis
  by (rule that[OF bf-eq prefix[unfolded take-prefix]
    selected-suc step])
qed
qed
qed

lemma mfold-fri-layer-opening-selected-recorded-step-evidence-at:
  fixes s t :: ('f, 'a) protocol-channel-scheme
  assumes outcome:
    Some (out, t) ∈
      set-dist
      (execute
        (mfold (idx, x, len, pw)
          (map fri-layer-opening-step bfs)) s)
  and layers:
    fri-layers-transcript (length bfs) len layer-chunks total-chunk
  and transcript:
    PTranscript s = total-chunk @ PTranscript t
  and initial-idx-bound: idx < len
  and all-layer-bounds:
     $\bigwedge k. k < \text{length } \text{bfs} \implies$ 
    0 < fri-layer-lengths (length bfs) len ! k  $\wedge$ 
    fri-layer-indices (length bfs) idx len ! k <
    fri-layer-lengths (length bfs) len ! k
  and j-bound: j < length bfs
  shows
     $\exists xp \text{ } xp\text{-path } xn \text{ } xn\text{-path } next\text{-value}.$ 
    fri-layer-step-evidence (snd (bfs ! j)) (fst (bfs ! j))
    (fri-layer-lengths (length bfs) len ! j)
    (fri-layer-indices (length bfs) idx len ! j)
    (pw * 2 ^ j)
    (fri-sibling-index (fri-layer-lengths (length bfs) len ! j)
      (fri-layer-indices (length bfs) idx len ! j))
    xp xp-path xn xn-path
    (fri-layer-indices (length bfs) idx len ! j mod
      (fri-layer-lengths (length bfs) len ! j div 2))
    next-value
    (layer-chunks ! j)
  proof -
    show ?thesis
      using outcome layers transcript initial-idx-bound all-layer-bounds
        j-bound
    proof (induction bfs arbitrary: idx x len pw s out t j layer-chunks total-chunk)

```



```

case Nil
then show ?case by simp
next
case (Cons bf bfs)
obtain b rt where bf-eq: bf = (b, rt)
  by (cases bf)
from Cons.prems(1)[unfolded bf-eq]
obtain out1 s1 where
  head:
    Some (out1, s1) ∈
      set-dist
        (execute (fri-layer-opening-step (b, rt)
          (idx, x, len, pw)) s)
  and tail:
    Some (out, t) ∈
      set-dist
        (execute (mfold out1 (map fri-layer-opening-step bfs)) s1)
  by (auto elim!: set-dist-bindE)
obtain first rest where chunks-eq: layer-chunks = first # rest
  using Cons.prems(6) Cons.prems(2)
  unfolding fri-layers-transcript-def by (cases layer-chunks) auto
have total-eq: total-chunk = first @ List.concat rest
  using Cons.prems(2) unfolding fri-layers-transcript-def chunks-eq
  by simp
have first-open:
  ∃ yp yp-path yn yn-path.
    fri-layer-opening-chunk len yp yp-path yn yn-path first
  using Cons.prems(2) unfolding fri-layers-transcript-def chunks-eq
  by auto
have rest-layers:
  fri-layers-transcript (length bfs) (len div 2) rest
    (List.concat rest)
  using Cons.prems(2) unfolding fri-layers-transcript-def chunks-eq
  by auto
obtain x1 xp xp-path xn xn-path head-chunk where out1-eq:
  out1 = (idx mod (len div 2), x1, len div 2, pw + pw)
  and head-chunk-def: head-chunk = [xp] @ xp-path @ [xn] @ xn-path
  and head-open:
    fri-layer-opening-chunk len xp xp-path xn xn-path head-chunk
  and head-transcript:
    PTranscript s = head-chunk @ PTranscript s1
  and head-step:
    fri-layer-step-evidence rt b len idx pw
      (fri-sibling-index len idx) xp xp-path xn xn-path
      (idx mod (len div 2)) x1 head-chunk
proof –
  have len-pos: 0 < len
    using Cons.prems(5)[of 0] by simp
  have idx-bound: idx < len

```

```

  by (rule Cons.premis(4))
show ?thesis
proof (rule fri-layer-opening-step-outcome-authenticated
  [OF len-pos idx-bound head])
fix  $xp'$   $xp\text{-path}'$   $xn'$   $xn\text{-path}'$   $x'$   $chunk$ 
assume out-eq:
  out1 = (idx mod (len div 2),  $x'$ , len div 2, pw + pw)
and xp-eq:  $xp' = x$ 
and x'-eq:
   $x' = \text{fri-fold-value } b \text{ } xp' \text{ } xn'$ 
  (fri-fold-denominator (( $h \wedge idx$ ) * shift) pw)
and chunk-eq:  $chunk = [xp'] @ xp\text{-path}' @ [xn'] @ xn\text{-path}'$ 
and chunk-open:
  fri-layer-opening-chunk len  $xp'$   $xp\text{-path}'$   $xn'$   $xn\text{-path}'$ 
  chunk
and tr: PTranscript  $s = chunk @ PTranscript s1$ 
assume
   $s \leq s1$ 
  PState  $s1 = \text{foldl concat (PState } s) \text{ } chunk$ 
  PQueryCounter  $s1 = PQueryCounter s$ 
  authenticated-opening-in  $s1$ 
    (opening-root = rt, opening-length = len,
     opening-index = idx, opening-value =  $xp'$ ,
     opening-path =  $xp\text{-path}'$ )
  authenticated-opening-in  $s1$ 
    (opening-root = rt, opening-length = len,
     opening-index = fri-sibling-index len idx,
     opening-value =  $xn'$ , opening-path =  $xn\text{-path}'$ )
have step:
  fri-layer-step-evidence rt b len idx pw
  (fri-sibling-index len idx)  $xp'$   $xp\text{-path}'$   $xn'$   $xn\text{-path}'$ 
  (idx mod (len div 2))  $x'$   $chunk$ 
  by (rule fri-layer-step-evidenceI)
  (use x'-eq chunk-open in simp-all)
show ?thesis
  by (rule that[OF out-eq chunk-eq chunk-open tr step])
qed
qed
from mfold-fri-layer-openings-outcome[OF tail[unfolded out1-eq]]
obtain tail-layers tail-chunk where
  tail-transcript:
    PTranscript  $s1 = tail\text{-chunk} @ PTranscript t$ 
  by blast
have head-len:
  length head-chunk = length first
proof -
  obtain  $yp$   $yp\text{-path}$   $yn$   $yn\text{-path}$  where first-open':
    fri-layer-opening-chunk len  $yp$   $yp\text{-path}$   $yn$   $yn\text{-path}$  first
  using first-open by blast

```

```

show ?thesis
  using fri-layer-opening-chunk-length[OF head-open]
    fri-layer-opening-chunk-length[OF first-open]
  by simp
qed
have chunks-total-eq:
  head-chunk @ tail-chunk = first @ List.concat rest
  using head-transcript tail-transcript Cons.prem5(3) total-eq by simp
have head-eq: head-chunk = first
  using arg-cong[OF chunks-total-eq,
    of take (length head-chunk)]
    head-len
  by simp
have tail-chunk-eq: tail-chunk = List.concat rest
  using chunks-total-eq head-eq by simp
have tail-transcript-rest:
  PTranscript s1 = List.concat rest @ PTranscript t
  using tail-transcript tail-chunk-eq by simp
show ?case
proof (cases j)
case 0
  have step:
    fri-layer-step-evidence (snd ((bf # bfs) ! j))
      (fst ((bf # bfs) ! j))
      (fri-layer-lengths (length (bf # bfs)) len ! j)
      (fri-layer-indices (length (bf # bfs)) idx len ! j)
      (pw * 2 ^ j)
      (fri-sibling-index
        (fri-layer-lengths (length (bf # bfs)) len ! j)
        (fri-layer-indices (length (bf # bfs)) idx len ! j))
      xp xp-path xn xn-path
      (fri-layer-indices (length (bf # bfs)) idx len ! j mod
        (fri-layer-lengths (length (bf # bfs)) len ! j div 2))
      x1 (layer-chunks ! j)
    using head-step 0 bf-eq head-eq chunks-eq by simp
  show ?thesis
    using step by blast
next
case (Suc k)
  have k-bound: k < length bfs
    using Cons.prem5(6) Suc by simp
  have tail-all-layer-bounds:
     $\bigwedge m. m < \text{length } \text{bfs} \implies$ 
     $0 < \text{fri-layer-lengths } (\text{length } \text{bfs}) \ (\text{len div } 2) ! m \wedge$ 
     $\text{fri-layer-indices } (\text{length } \text{bfs}) \ (\text{idx mod } (\text{len div } 2))$ 
     $(\text{len div } 2) ! m <$ 
     $\text{fri-layer-lengths } (\text{length } \text{bfs}) \ (\text{len div } 2) ! m$ 
    using Cons.prem5(5)[of Suc m for m] by simp
  have tail-initial-idx-bound:

```

```

    idx mod (len div 2) < len div 2
  using tail-all-layer-bounds[of 0] k-bound by (cases bfs) simp-all
from Cons.IH[OF tail[unfolded out1-eq] rest-layers
  tail-transcript-rest tail-initial-idx-bound tail-all-layer-bounds
  k-bound]
obtain txp txp-path txn txn-path tnext where tail-step:
  fri-layer-step-evidence (snd (bfs ! k)) (fst (bfs ! k))
    (fri-layer-lengths (length bfs) (len div 2) ! k)
    (fri-layer-indices (length bfs) (idx mod (len div 2))
      (len div 2) ! k)
    ((pw + pw) * 2 ^ k)
  (fri-sibling-index
    (fri-layer-lengths (length bfs) (len div 2) ! k)
    (fri-layer-indices (length bfs) (idx mod (len div 2))
      (len div 2) ! k))
  txp txp-path txn txn-path
  (fri-layer-indices (length bfs) (idx mod (len div 2))
    (len div 2) ! k mod
    (fri-layer-lengths (length bfs) (len div 2) ! k div 2))
  tnext (rest ! k)
  by blast
have pow-eq: (pw + pw) * 2 ^ k = pw * 2 ^ Suc k
  by (simp add: algebra-simps)
have step:
  fri-layer-step-evidence (snd ((bf # bfs) ! j))
    (fst ((bf # bfs) ! j))
    (fri-layer-lengths (length (bf # bfs)) len ! j)
    (fri-layer-indices (length (bf # bfs)) idx len ! j)
    (pw * 2 ^ j)
  (fri-sibling-index
    (fri-layer-lengths (length (bf # bfs)) len ! j)
    (fri-layer-indices (length (bf # bfs)) idx len ! j))
  txp txp-path txn txn-path
  (fri-layer-indices (length (bf # bfs)) idx len ! j mod
    (fri-layer-lengths (length (bf # bfs)) len ! j div 2))
  tnext (layer-chunks ! j)
  using tail-step Suc chunks-eq pow-eq by simp
show ?thesis
  using step by blast
qed
qed
qed

lemma mfold-fri-layer-opening-first-recorded-step-evidence:
  fixes s t :: ('f, 'a) protocol-channel-scheme
  assumes outcome:
    Some (out, t) ∈
      set-dist
      (execute

```

```

      (mfold (idx, x, len, pw)
        (map fri-layer-opening-step bfs)) s)
and layers:
  fri-layers-transcript (length bfs) len layer-chunks total-chunk
and transcript:
  PTranscript s = total-chunk @ PTranscript t
and nonempty: 0 < length bfs
shows
  ∃ xp-path xn xn-path next-value.
    fri-layer-step-evidence (snd (bfs ! 0)) (fst (bfs ! 0))
    (fri-layer-lengths (length bfs) len ! 0)
    (fri-layer-indices (length bfs) idx len ! 0)
    pw
    (fri-sibling-index (fri-layer-lengths (length bfs) len ! 0)
      (fri-layer-indices (length bfs) idx len ! 0))
    x xp-path xn xn-path
    (fri-layer-indices (length bfs) idx len ! 0 mod
      (fri-layer-lengths (length bfs) len ! 0 div 2))
    next-value
    (layer-chunks ! 0)
proof -
  obtain bf bfs' where bfs-eq: bfs = bf # bfs'
  using nonempty by (cases bfs) auto
  obtain b rt where bf-eq: bf = (b, rt)
  by (cases bf)
  from outcome[unfolded bfs-eq bf-eq]
  obtain out1 s1 where
    head:
      Some (out1, s1) ∈
        set-dist
          (execute (fri-layer-opening-step (b, rt)
            (idx, x, len, pw)) s)
  and tail:
    Some (out, t) ∈
      set-dist
        (execute (mfold out1 (map fri-layer-opening-step bfs') s1)
          by (auto elim!: set-dist-bindE))
  obtain first rest where chunks-eq: layer-chunks = first # rest
  using layers nonempty unfolding bfs-eq fri-layers-transcript-def
  by (cases layer-chunks) auto
  have total-eq: total-chunk = first @ List.concat rest
  using layers unfolding bfs-eq fri-layers-transcript-def chunks-eq
  by simp
  have first-open:
    ∃ yp yp-path yn yn-path.
      fri-layer-opening-chunk len yp yp-path yn yn-path first
  using layers unfolding bfs-eq fri-layers-transcript-def chunks-eq
  by auto
  obtain x1 xp xp-path xn xn-path head-chunk where out1-eq:

```

```

    out1 = (idx mod (len div 2), x1, len div 2, pw + pw)
and head-chunk-def: head-chunk = [xp] @ xp-path @ [xn] @ xn-path
and xp-eq: xp = x
and head-open:
    fri-layer-opening-chunk len xp xp-path xn xn-path head-chunk
and head-transcript:
    PTranscript s = head-chunk @ PTranscript s1
and head-step:
    fri-layer-step-evidence rt b len idx pw
    (fri-sibling-index len idx) xp xp-path xn xn-path
    (idx mod (len div 2)) x1 head-chunk
proof –
from fri-layer-opening-step-outcome[OF head]
obtain xp' xp-path' xn' xn-path' x' where out1-eq':
    out1 = (idx mod (len div 2), x', len div 2, pw + pw)
and xp'-eq: xp' = x
and x'-eq:
    x' = fri-fold-value b xp' xn'
    (fri-fold-denominator ((h ^ idx) * shift) pw)
and chunk-open:
    fri-layer-opening-chunk len xp' xp-path' xn' xn-path'
    ([xp'] @ xp-path' @ [xn'] @ xn-path')
and tr:
    PTranscript s =
    ([xp'] @ xp-path' @ [xn'] @ xn-path') @ PTranscript s1
by blast
have step:
    fri-layer-step-evidence rt b len idx pw
    (fri-sibling-index len idx) xp' xp-path' xn' xn-path'
    (idx mod (len div 2)) x'
    ([xp'] @ xp-path' @ [xn'] @ xn-path')
by (rule fri-layer-step-evidenceI)
    (use x'-eq chunk-open in simp-all)
show ?thesis
by (rule that[OF out1-eq' refl xp'-eq chunk-open tr step])
qed
from mfold-fri-layer-openings-outcome[OF tail[unfolded out1-eq]]
obtain tail-layers tail-chunk where
    tail-transcript:
    PTranscript s1 = tail-chunk @ PTranscript t
by blast
have head-len:
    length head-chunk = length first
proof –
obtain yp yp-path yn yn-path where first-open':
    fri-layer-opening-chunk len yp yp-path yn yn-path first
using first-open by blast
show ?thesis
using fri-layer-opening-chunk-length[OF head-open]

```

```

      fri-layer-opening-chunk-length[OF first-open']
    by simp
qed
have chunks-total-eq:
  head-chunk @ tail-chunk = first @ List.concat rest
  using head-transcript tail-transcript transcript total-eq by simp
have head-eq: head-chunk = first
  using arg-cong[OF chunks-total-eq,
    of take (length head-chunk)]
    head-len
  by simp
have step:
  fri-layer-step-evidence (snd (bfs ! 0)) (fst (bfs ! 0))
    (fri-layer-lengths (length bfs) len ! 0)
    (fri-layer-indices (length bfs) idx len ! 0)
  pw
  (fri-sibling-index (fri-layer-lengths (length bfs) len ! 0)
    (fri-layer-indices (length bfs) idx len ! 0))
  x xp-path xn xn-path
  (fri-layer-indices (length bfs) idx len ! 0 mod
    (fri-layer-lengths (length bfs) len ! 0 div 2))
  x1
  (layer-chunks ! 0)
  using head-step bfs-eq bf-eq chunks-eq head-eq xp-eq by simp
show ?thesis
  using step by blast
qed

lemma mfold-fri-layer-opening-successor-recorded-step-from-previous:
  fixes s t :: ('f, 'a) protocol-channel-scheme
  assumes outcome:
    Some (out, t) ∈
      set-dist
        (execute
          (mfold (idx, x, len, pw)
            (map fri-layer-opening-step bfs)) s)
  and layers:
    fri-layers-transcript (length bfs) len layer-chunks total-chunk
  and transcript:
    PTranscript s = total-chunk @ PTranscript t
  and initial-idx-bound: idx < len
  and all-layer-bounds:
     $\bigwedge k. k < \text{length } \text{bfs} \implies$ 
     $0 < \text{fri-layer-lengths } (\text{length } \text{bfs}) \text{ len } ! k \wedge$ 
     $\text{fri-layer-indices } (\text{length } \text{bfs}) \text{ idx len } ! k <$ 
     $\text{fri-layer-lengths } (\text{length } \text{bfs}) \text{ len } ! k$ 
  and j-bound: Suc j < length bfs
  and previous-step:
    fri-layer-step-evidence (snd (bfs ! j)) (fst (bfs ! j))

```

```

(fri-layer-lengths (length bfs) len ! j)
(fri-layer-indices (length bfs) idx len ! j)
(pw * 2 ^ j)
(fri-sibling-index
  (fri-layer-lengths (length bfs) len ! j)
  (fri-layer-indices (length bfs) idx len ! j))
xp xp-path xn xn-path
(fri-layer-indices (length bfs) idx len ! j mod
  (fri-layer-lengths (length bfs) len ! j div 2))
v
(layer-chunks ! j)
shows
  ∃ xp-path' xn' xn-path' next-value.
    fri-layer-step-evidence (snd (bfs ! Suc j)) (fst (bfs ! Suc j))
      (fri-layer-lengths (length bfs) len ! Suc j)
      (fri-layer-indices (length bfs) idx len ! Suc j)
      (pw * 2 ^ Suc j)
      (fri-sibling-index
        (fri-layer-lengths (length bfs) len ! Suc j)
        (fri-layer-indices (length bfs) idx len ! Suc j))
      v xp-path' xn' xn-path'
      (fri-layer-indices (length bfs) idx len ! Suc j mod
        (fri-layer-lengths (length bfs) len ! Suc j div 2))
      next-value
      (layer-chunks ! Suc j)
    using outcome layers transcript initial-idx-bound all-layer-bounds j-bound
    previous-step
  proof (induction bfs arbitrary: idx x len pw s out t j layer-chunks total-chunk
    xp xp-path xn xn-path v)
  case Nil
  then show ?case by simp
next
case (Cons bf bfs)
obtain b rt where bf-eq: bf = (b, rt)
by (cases bf)
from Cons.prem1[1][unfolded bf-eq]
obtain out1 s1 where
  head:
    Some (out1, s1) ∈
      set-dist
        (execute (fri-layer-opening-step (b, rt)
          (idx, x, len, pw)) s)
  and tail:
    Some (out, t) ∈
      set-dist
        (execute (mfold out1 (map fri-layer-opening-step bfs)) s1)
  by (auto elim!: set-dist-bindE)
obtain first rest where chunks-eq: layer-chunks = first # rest
using Cons.prem2 Cons.prem6

```



```

unfolding fri-layers-transcript-def by (cases layer-chunks) auto
have total-eq: total-chunk = first @ List.concat rest
using Cons.premis(2) unfolding fri-layers-transcript-def chunks-eq
by simp
have first-open:
   $\exists$  yp yp-path yn yn-path.
    fri-layer-opening-chunk len yp yp-path yn yn-path first
using Cons.premis(2) unfolding fri-layers-transcript-def chunks-eq
by auto
have rest-layers:
  fri-layers-transcript (length bfs) (len div 2) rest
    (List.concat rest)
using Cons.premis(2) unfolding fri-layers-transcript-def chunks-eq
by auto
obtain x1 hxp hxp-path hxn hxn-path head-chunk where out1-eq:
  out1 = (idx mod (len div 2), x1, len div 2, pw + pw)
and hxp-eq: hxp = x
and head-open:
  fri-layer-opening-chunk len hxp hxp-path hxn hxn-path head-chunk
and head-transcript:
  PTranscript s = head-chunk @ PTranscript s1
and head-step:
  fri-layer-step-evidence rt b len idx pw
    (fri-sibling-index len idx) hxp hxp-path hxn hxn-path
    (idx mod (len div 2)) x1 head-chunk
proof –
from fri-layer-opening-step-outcome[OF head]
obtain xp' xp-path' xn' xn-path' x' where out1-eq':
  out1 = (idx mod (len div 2), x', len div 2, pw + pw)
and xp'-eq: xp' = x
and x'-eq:
  x' = fri-fold-value b xp' xn'
    (fri-fold-denominator ((h ^ idx) * shift) pw)
and chunk-open:
  fri-layer-opening-chunk len xp' xp-path' xn' xn-path'
    ([xp'] @ xp-path' @ [xn'] @ xn-path')
and tr:
  PTranscript s =
    ([xp'] @ xp-path' @ [xn'] @ xn-path') @ PTranscript s1
by blast
have step:
  fri-layer-step-evidence rt b len idx pw
    (fri-sibling-index len idx) xp' xp-path' xn' xn-path'
    (idx mod (len div 2)) x'
    ([xp'] @ xp-path' @ [xn'] @ xn-path')
by (rule fri-layer-step-evidenceI)
    (use x'-eq chunk-open in simp-all)
show ?thesis
by (rule that[OF out1-eq' xp'-eq chunk-open tr step])

```

```

qed
from mfold-fri-layer-openings-outcome[OF tail[unfolded out1-eq]]
obtain tail-layers tail-chunk where
  tail-transcript:
    PTranscript s1 = tail-chunk @ PTranscript t
  by blast
have head-len:
  length head-chunk = length first
proof -
  obtain yp yp-path yn yn-path where first-open':
    fri-layer-opening-chunk len yp yp-path yn yn-path first
  using first-open by blast
show ?thesis
  using fri-layer-opening-chunk-length[OF head-open]
    fri-layer-opening-chunk-length[OF first-open']
  by simp
qed
have chunks-total-eq:
  head-chunk @ tail-chunk = first @ List.concat rest
  using head-transcript tail-transcript Cons.prem3 total-eq by simp
have head-eq: head-chunk = first
  using arg-cong[OF chunks-total-eq,
    of take (length head-chunk)]
  head-len
  by simp
have tail-chunk-eq: tail-chunk = List.concat rest
  using chunks-total-eq head-eq by simp
have tail-transcript-rest:
  PTranscript s1 = List.concat rest @ PTranscript t
  using tail-transcript tail-chunk-eq by simp
show ?case
proof (cases j)
  case 0
  have prev-head:
    fri-layer-step-evidence rt b len idx pw
    (fri-sibling-index len idx) xp xp-path xn xn-path
    (idx mod (len div 2)) v first
    using Cons.prem7 0 bf-eq chunks-eq by simp
  have v-eq-x1: v = x1
  proof -
    have xp-eq: xp = hxp
      by (rule fri-layer-opening-chunk-values-unique(1))
      (rule fri-layer-step-evidenceD(4)[OF prev-head],
        rule fri-layer-step-evidenceD(4)[OF head-step[unfolded head-eq]])
    have xn-eq: xn = hxn
      by (rule fri-layer-opening-chunk-values-unique(2))
      (rule fri-layer-step-evidenceD(4)[OF prev-head],
        rule fri-layer-step-evidenceD(4)[OF head-step[unfolded head-eq]])
    show ?thesis

```

```

    using fri-layer-step-evidenceD(3)[OF prev-head]
      fri-layer-step-evidenceD(3)[OF head-step[unfolded head-eq]]
      xp-eq xn-eq
    by simp
qed
have bfs-nonempty: 0 < length bfs
  using Cons.premis(6) 0 by simp
from mfold-fri-layer-opening-first-recorded-step-evidence
  [OF tail[unfolded out1-eq] rest-layers tail-transcript-rest
    bfs-nonempty]
obtain sxp-path sxn sxn-path snext where succ-step-tail:
  fri-layer-step-evidence (snd (bfs ! 0)) (fst (bfs ! 0))
    (fri-layer-lengths (length bfs) (len div 2) ! 0)
    (fri-layer-indices (length bfs) (idx mod (len div 2))
      (len div 2) ! 0)
    (pw + pw)
    (fri-sibling-index
      (fri-layer-lengths (length bfs) (len div 2) ! 0)
      (fri-layer-indices (length bfs) (idx mod (len div 2))
        (len div 2) ! 0))
  x1 sxp-path sxn sxn-path
    (fri-layer-indices (length bfs) (idx mod (len div 2))
      (len div 2) ! 0 mod
      (fri-layer-lengths (length bfs) (len div 2) ! 0 div 2))
  snext
    (rest ! 0)
by blast
have pow-eq: pw + pw = pw * 2 ^ Suc 0
  by (simp add: algebra-simps)
have succ-step:
  fri-layer-step-evidence (snd ((bf # bfs) ! Suc j))
    (fst ((bf # bfs) ! Suc j))
    (fri-layer-lengths (length (bf # bfs)) len ! Suc j)
    (fri-layer-indices (length (bf # bfs)) idx len ! Suc j)
    (pw * 2 ^ Suc j)
    (fri-sibling-index
      (fri-layer-lengths (length (bf # bfs)) len ! Suc j)
      (fri-layer-indices (length (bf # bfs)) idx len ! Suc j))
  v sxp-path sxn sxn-path
    (fri-layer-indices (length (bf # bfs)) idx len ! Suc j mod
      (fri-layer-lengths (length (bf # bfs)) len ! Suc j div 2))
  snext
    (layer-chunks ! Suc j)
  using succ-step-tail 0 chunks-eq v-eq-x1 pow-eq by simp
show ?thesis
  using succ-step by blast
next
case (Suc k)
have k-bound: Suc k < length bfs

```

```

using Cons.premis(6) Suc by simp
have tail-all-layer-bounds:
   $\wedge m. m < \text{length } \text{bfs} \implies$ 
     $0 < \text{fri-layer-lengths } (\text{length } \text{bfs}) (\text{len div } 2) ! m \wedge$ 
     $\text{fri-layer-indices } (\text{length } \text{bfs}) (\text{idx mod } (\text{len div } 2))$ 
     $(\text{len div } 2) ! m <$ 
     $\text{fri-layer-lengths } (\text{length } \text{bfs}) (\text{len div } 2) ! m$ 
using Cons.premis(5)[of Suc m for m] by simp
have tail-initial-idx-bound:
   $\text{idx mod } (\text{len div } 2) < \text{len div } 2$ 
using tail-all-layer-bounds[of 0] k-bound by (cases bfs) simp-all
have prev-tail:
  fri-layer-step-evidence (snd (bfs ! k)) (fst (bfs ! k))
    (fri-layer-lengths (length bfs) (len div 2) ! k)
    (fri-layer-indices (length bfs) (idx mod (len div 2))
      (len div 2) ! k)
    ((pw + pw) * 2 ^ k)
    (fri-sibling-index
      (fri-layer-lengths (length bfs) (len div 2) ! k)
      (fri-layer-indices (length bfs) (idx mod (len div 2))
        (len div 2) ! k))
    xp xp-path xn xn-path
    (fri-layer-indices (length bfs) (idx mod (len div 2))
      (len div 2) ! k mod
      (fri-layer-lengths (length bfs) (len div 2) ! k div 2))
    v (rest ! k)
proof -
have pow-eq:  $(pw + pw) * 2 ^ k = pw * (2 * 2 ^ k)$ 
by (simp add: algebra-simps)
have raw:
  fri-layer-step-evidence (snd (bfs ! k)) (fst (bfs ! k))
    (fri-layer-lengths (length bfs) (len div 2) ! k)
    (fri-layer-indices (length bfs) (idx mod (len div 2))
      (len div 2) ! k)
     $(pw * (2 * 2 ^ k))$ 
    (fri-sibling-index
      (fri-layer-lengths (length bfs) (len div 2) ! k)
      (fri-layer-indices (length bfs) (idx mod (len div 2))
        (len div 2) ! k))
    xp xp-path xn xn-path
    (fri-layer-indices (length bfs) (idx mod (len div 2))
      (len div 2) ! k mod
      (fri-layer-lengths (length bfs) (len div 2) ! k div 2))
    v (rest ! k)
using Cons.premis(7) Suc chunks-eq pow-eq by simp
show ?thesis
unfolding pow-eq by (rule raw)
qed
from Cons.IH[OF tail[unfolded out1-eq] rest-layers

```

```

tail-transcript-rest tail-initial-idx-bound tail-all-layer-bounds
k-bound prev-tail]
obtain sxp-path sn sn-path snext where succ-tail:
  fri-layer-step-evidence (snd (bfs ! Suc k))
    (fst (bfs ! Suc k))
    (fri-layer-lengths (length bfs) (len div 2) ! Suc k)
    (fri-layer-indices (length bfs) (idx mod (len div 2))
      (len div 2) ! Suc k)
    ((pw + pw) * 2 ^ Suc k)
    (fri-sibling-index
      (fri-layer-lengths (length bfs) (len div 2) ! Suc k)
      (fri-layer-indices (length bfs) (idx mod (len div 2))
        (len div 2) ! Suc k))
  v sxp-path sn sn-path
    (fri-layer-indices (length bfs) (idx mod (len div 2))
      (len div 2) ! Suc k mod
      (fri-layer-lengths (length bfs) (len div 2) ! Suc k div 2))
    snext (rest ! Suc k)
by blast
have pow-eg: (pw + pw) * (2 * 2 ^ k) = pw * (4 * 2 ^ k)
by (simp add: algebra-simps)
have succ-tail-raw:
  fri-layer-step-evidence (snd (bfs ! Suc k))
    (fst (bfs ! Suc k))
    (fri-layer-lengths (length bfs) (len div 2) ! Suc k)
    (fri-layer-indices (length bfs) (idx mod (len div 2))
      (len div 2) ! Suc k)
    ((pw + pw) * (2 * 2 ^ k))
    (fri-sibling-index
      (fri-layer-lengths (length bfs) (len div 2) ! Suc k)
      (fri-layer-indices (length bfs) (idx mod (len div 2))
        (len div 2) ! Suc k))
  v sxp-path sn sn-path
    (fri-layer-indices (length bfs) (idx mod (len div 2))
      (len div 2) ! Suc k mod
      (fri-layer-lengths (length bfs) (len div 2) ! Suc k div 2))
    snext (rest ! Suc k)
using succ-tail by simp
have succ-step:
  fri-layer-step-evidence (snd ((bf # bfs) ! Suc j))
    (fst ((bf # bfs) ! Suc j))
    (fri-layer-lengths (length (bf # bfs)) len ! Suc j)
    (fri-layer-indices (length (bf # bfs)) idx len ! Suc j)
    (pw * 2 ^ Suc j)
    (fri-sibling-index
      (fri-layer-lengths (length (bf # bfs)) len ! Suc j)
      (fri-layer-indices (length (bf # bfs)) idx len ! Suc j))
  v sxp-path sn sn-path
    (fri-layer-indices (length (bf # bfs)) idx len ! Suc j mod

```

```

      (fri-layer-lengths (length (bf # bfs)) len ! Suc j div 2))
    snext
    (layer-chunks ! Suc j)
  using succ-tail-raw Suc chunks-eq pow-eq
  by (simp add: algebra-simps)
show ?thesis
  using succ-step by blast
qed
qed

lemma verifier-query-round-after-index-composition-recorded-layer-step-evidence:
  fixes s t :: ('f, 'a) protocol-channel-scheme
  assumes outcome:
    Some ((), t) ∈
      set-dist
        (execute
          (verifier-query-round-after-index-program
            fr f-fl f-final as fl final raw) s)
  and recorded:
    query-round-fri-layer-transcripts (index (to-nat raw))
      (map snd f-fl) (map snd fl) round-chunk trace-layers
      composition-layers
  and transcript: PTranscript s = round-chunk @ PTranscript t
  and j-bound: j < length fl
  and all-layer-bounds:
     $\bigwedge k. k < \text{length } fl \implies$ 
     $0 < \text{fri-layer-lengths } (\text{length } fl) (\text{clength} * \text{scale}) ! k \wedge$ 
     $\text{fri-layer-indices } (\text{length } fl) (\text{index } (\text{to-nat } \text{raw}))$ 
     $(\text{clength} * \text{scale}) ! k <$ 
     $\text{fri-layer-lengths } (\text{length } fl) (\text{clength} * \text{scale}) ! k$ 
  shows
     $\exists xp \text{ } xp\text{-path } xn \text{ } xn\text{-path } \text{next-value}.$ 
    fri-layer-step-evidence (snd (fl ! j)) (fst (fl ! j))
      (fri-layer-lengths (length fl) (clength * scale) ! j)
      (fri-layer-indices (length fl) (index (to-nat raw)))
      (clength * scale) ! j)
    (2 ^ j)
    (fri-sibling-index
      (fri-layer-lengths (length fl) (clength * scale) ! j)
      (fri-layer-indices (length fl) (index (to-nat raw)))
      (clength * scale) ! j))
    xp xp-path xn xn-path
    (fri-layer-indices (length fl) (index (to-nat raw)))
    (clength * scale) ! j mod
    (fri-layer-lengths (length fl) (clength * scale) ! j div 2))
    next-value
    (composition-layers ! j)
  proof -
    let ?idx = index (to-nat raw)

```

```

from outcome obtain fv s1 f-out s2 s3 c-out s4 where
  query-decommit:
    Some (fv, s1) ∈
      set-dist (execute (mmap (check-decommit-on-query fr ?idx)) s)
  and trace-fri:
    Some (f-out, s2) ∈
      set-dist (execute
        (mfold (?idx, hd fv, clength * scale, 1)
          (receive-query-commits f-fl)) s1)
  and assert-trace:
    Some (( ), s3) ∈
      set-dist
        (execute
          (assert (case f-out of (-, f-x, -, -) ⇒ f-x = f-final)) s2)
  and comp-fri:
    Some (c-out, s4) ∈
      set-dist (execute
        (mfold
          (?idx, cp-eval as fv (h ^ ?idx * shift), clength * scale, 1)
          (receive-query-commits fl)) s3)
  and assert-comp:
    Some (( ), t) ∈
      set-dist
        (execute
          (assert (case c-out of (-, x, -, -) ⇒ x = final)) s4)
  unfolding verifier-query-round-after-index-program-def
  by (auto simp: Let-def split: prod.splits elim!: set-dist-bindE)
from check-decommit-on-query-outcome[OF query-decommit]
obtain query-paths query-chunk where
  query-chunk-shape:
    query-decommitment-transcript ?idx fv query-paths query-chunk
  and tr-s: PTranscript s = query-chunk @ PTranscript s1
  by blast
from receive-query-commits-layers-outcome[OF trace-fri]
obtain trace-layer-chunks trace-fri-chunk where
  trace-layers-actual:
    fri-layers-transcript (length f-fl) (clength * scale)
    trace-layer-chunks trace-fri-chunk
  and tr-s1: PTranscript s1 = trace-fri-chunk @ PTranscript s2
  by blast
have s3-eq: s3 = s2
  using assert-trace unfolding assert-def
  by (cases case f-out of (-, f-x, -, -) ⇒ f-x = f-final)
    (auto simp: throw-no-outcome)
from receive-query-commits-layers-outcome[OF comp-fri]
obtain comp-layer-chunks comp-fri-chunk where
  comp-layers-actual:
    fri-layers-transcript (length fl) (clength * scale)
    comp-layer-chunks comp-fri-chunk

```

```

and tr-s3: PTranscript s3 = comp-fri-chunk @ PTranscript s4
by blast
have t-eq: t = s4
using assert-comp unfolding assert-def
by (cases case c-out of (-, x, -, -)  $\Rightarrow$  x = final)
    (auto simp: throw-no-outcome)
let ?actual-chunk = query-chunk @ trace-fri-chunk @ comp-fri-chunk
have actual-recorded:
    query-round-fri-layer-transcripts ?idx (map snd f) (map snd fl)
    ?actual-chunk trace-layer-chunks comp-layer-chunks
unfolding query-round-fri-layer-transcripts-def
by (intro exI[of - query-chunk] exI[of - trace-fri-chunk]
    exI[of - comp-fri-chunk] exI[of - fv] exI[of - query-paths] conjI)
    (use query-chunk-shape trace-layers-actual comp-layers-actual in
     $\langle \text{simp-all add: mult.commute} \rangle$ )
have actual-transcript:
    PTranscript s = ?actual-chunk @ PTranscript t
using tr-s tr-s1 tr-s3 s3-eq t-eq by simp
have actual-chunk-eq: ?actual-chunk = round-chunk
using actual-transcript transcript by simp
have comp-layer-chunks-eq:
    comp-layer-chunks = composition-layers
by (rule query-round-fri-layer-transcripts-unique(2)
    [OF actual-recorded[unfolded actual-chunk-eq] recorded])
have idx-bound: ?idx < clength * scale
by (rule index-less-domain)
have map-eq:
    receive-query-commits fl = map fri-layer-opening-step fl
unfolding receive-query-commits-def fri-layer-opening-step-def
by (simp add: fun-eq-iff split: prod.splits)
from mfold-fri-layer-opening-selected-recorded-step-evidence-at
    [OF comp-fri[unfolded map-eq] comp-layers-actual tr-s3 idx-bound
    all-layer-bounds j-bound]
obtain xp xp-path xn xn-path next-value where step-actual:
    fri-layer-step-evidence (snd (fl ! j)) (fst (fl ! j))
    (fri-layer-lengths (length fl) (clength * scale) ! j)
    (fri-layer-indices (length fl) ?idx (clength * scale) ! j)
    (1 * 2 ^ j)
    (fri-sibling-index
    (fri-layer-lengths (length fl) (clength * scale) ! j)
    (fri-layer-indices (length fl) ?idx (clength * scale) ! j))
    xp xp-path xn xn-path
    (fri-layer-indices (length fl) ?idx (clength * scale) ! j mod
    (fri-layer-lengths (length fl) (clength * scale) ! j div 2))
    next-value
    (comp-layer-chunks ! j)
by blast
have step:
    fri-layer-step-evidence (snd (fl ! j)) (fst (fl ! j))

```



```

(fri-layer-lengths (length fl) (clength * scale) ! j)
(fri-layer-indices (length fl) ?idx (clength * scale) ! j)
(2 ^ j)
(fri-sibling-index
  (fri-layer-lengths (length fl) (clength * scale) ! j)
  (fri-layer-indices (length fl) ?idx (clength * scale) ! j))
xp xp-path xn xn-path
(fri-layer-indices (length fl) ?idx (clength * scale) ! j mod
  (fri-layer-lengths (length fl) (clength * scale) ! j div 2))
next-value
(composition-layers ! j)
using step-actual comp-layer-chunks-eq by simp
show ?thesis
using step by blast
qed

```

lemma *verifier-query-round-after-index-composition-successor-recorded-step-from-previous:*

fixes $s\ t :: ('f, 'a)\ \text{protocol-channel-scheme}$

assumes *outcome:*

Some $((\),\ t) \in$
set-dist
(execute
(verifier-query-round-after-index-program
fr f-fl f-final as fl final raw) s)

and *recorded:*

query-round-fri-layer-transcripts (index (to-nat raw))
(map snd f-fl) (map snd fl) round-chunk trace-layers
composition-layers

and *transcript:* $P\text{Transcript } s = \text{round-chunk } @\ P\text{Transcript } t$

and *j-bound:* $\text{Suc } j < \text{length } fl$

and *all-layer-bounds:*

$\bigwedge k. k < \text{length } fl \implies$
 $0 < \text{fri-layer-lengths } (\text{length } fl) (\text{clength} * \text{scale}) ! k \wedge$
 $\text{fri-layer-indices } (\text{length } fl) (\text{index } (\text{to-nat } \text{raw}))$
 $(\text{clength} * \text{scale}) ! k <$
 $\text{fri-layer-lengths } (\text{length } fl) (\text{clength} * \text{scale}) ! k$

and *previous-step:*

fri-layer-step-evidence (snd (fl ! j)) (fst (fl ! j))
*(fri-layer-lengths (length fl) (clength * scale) ! j)*
(fri-layer-indices (length fl) (index (to-nat raw)))
*(clength * scale) ! j)*
 $(2 ^ j)$
(fri-sibling-index
*(fri-layer-lengths (length fl) (clength * scale) ! j)*
(fri-layer-indices (length fl) (index (to-nat raw)))
*(clength * scale) ! j))*
xp xp-path xn xn-path
(fri-layer-indices (length fl) (index (to-nat raw)))
*(clength * scale) ! j mod*

```

      (fri-layer-lengths (length fl) (clength * scale) ! j div 2))
    v
    (composition-layers ! j)
  shows
    ∃ xp-path' xn' xn-path' next-value.
    fri-layer-step-evidence (snd (fl ! Suc j)) (fst (fl ! Suc j))
      (fri-layer-lengths (length fl) (clength * scale) ! Suc j)
      (fri-layer-indices (length fl) (index (to-nat raw))
        (clength * scale) ! Suc j)
      (2 ^ Suc j)
      (fri-sibling-index
        (fri-layer-lengths (length fl) (clength * scale) ! Suc j)
        (fri-layer-indices (length fl) (index (to-nat raw))
          (clength * scale) ! Suc j))
      v xp-path' xn' xn-path'
      (fri-layer-indices (length fl) (index (to-nat raw))
        (clength * scale) ! Suc j mod
        (fri-layer-lengths (length fl) (clength * scale) ! Suc j div 2))
      next-value
      (composition-layers ! Suc j)
  proof -
    let ?idx = index (to-nat raw)
    from outcome obtain fv s1 f-out s2 s3 c-out s4 where
      query-decommit:
        Some (fv, s1) ∈
          set-dist (execute (mmap (check-decommit-on-query fr ?idx)) s)
    and trace-fri:
      Some (f-out, s2) ∈
        set-dist (execute
          (mfold (?idx, hd fv, clength * scale, 1)
            (receive-query-commits f-fl)) s1)
    and assert-trace:
      Some ((), s3) ∈
        set-dist
          (execute
            (assert (case f-out of (-, f-x, -, -) ⇒ f-x = f-final)) s2)
    and comp-fri:
      Some (c-out, s4) ∈
        set-dist (execute
          (mfold
            (?idx, cp-eval as fv (h ^ ?idx * shift), clength * scale, 1)
            (receive-query-commits fl)) s3)
    and assert-comp:
      Some ((), t) ∈
        set-dist
          (execute
            (assert (case c-out of (-, x, -, -) ⇒ x = final)) s4)
    unfolding verifier-query-round-after-index-program-def
    by (auto simp: Let-def split: prod.splits elim!: set-dist-bindE)

```

```

from check-decommit-on-query-outcome[OF query-decommit]
obtain query-paths query-chunk where
  query-chunk-shape:
    query-decommitment-transcript ?idx fv query-paths query-chunk
    and tr-s: PTranscript s = query-chunk @ PTranscript s1
    by blast
from receive-query-commits-layers-outcome[OF trace-fri]
obtain trace-layer-chunks trace-fri-chunk where
  trace-layers-actual:
    fri-layers-transcript (length f-fl) (clength * scale)
    trace-layer-chunks trace-fri-chunk
    and tr-s1: PTranscript s1 = trace-fri-chunk @ PTranscript s2
    by blast
have s3-eq: s3 = s2
    using assert-trace unfolding assert-def
    by (cases case f-out of (-, f-x, -, -)  $\Rightarrow$  f-x = f-final)
      (auto simp: throw-no-outcome)
from receive-query-commits-layers-outcome[OF comp-fri]
obtain comp-layer-chunks comp-fri-chunk where
  comp-layers-actual:
    fri-layers-transcript (length fl) (clength * scale)
    comp-layer-chunks comp-fri-chunk
    and tr-s3: PTranscript s3 = comp-fri-chunk @ PTranscript s4
    by blast
have t-eq: t = s4
    using assert-comp unfolding assert-def
    by (cases case c-out of (-, x, -, -)  $\Rightarrow$  x = final)
      (auto simp: throw-no-outcome)
let ?actual-chunk = query-chunk @ trace-fri-chunk @ comp-fri-chunk
have actual-recorded:
  query-round-fri-layer-transcripts ?idx (map snd f-fl) (map snd fl)
  ?actual-chunk trace-layer-chunks comp-layer-chunks
  unfolding query-round-fri-layer-transcripts-def
  by (intro exI[of - query-chunk] exI[of - trace-fri-chunk]
    exI[of - comp-fri-chunk] exI[of - fv] exI[of - query-paths] conjI)
    (use query-chunk-shape trace-layers-actual comp-layers-actual in
       $\langle$ simp-all add: mult.commute $\rangle$ )
have actual-transcript:
  PTranscript s = ?actual-chunk @ PTranscript t
  using tr-s tr-s1 tr-s3 s3-eq t-eq by simp
have actual-chunk-eq: ?actual-chunk = round-chunk
  using actual-transcript transcript by simp
have comp-layer-chunks-eq:
  comp-layer-chunks = composition-layers
  by (rule query-round-fri-layer-transcripts-unique(2)
    [OF actual-recorded[unfolded actual-chunk-eq] recorded])
have idx-bound: ?idx < clength * scale
  by (rule index-less-domain)
have map-eq:

```

```

receive-query-commits fl = map fri-layer-opening-step fl
unfolding receive-query-commits-def fri-layer-opening-step-def
by (simp add: fun-eq-iff split: prod.splits)
have previous-step-actual:
  fri-layer-step-evidence (snd (fl ! j)) (fst (fl ! j))
    (fri-layer-lengths (length fl) (clength * scale) ! j)
    (fri-layer-indices (length fl) ?idx (clength * scale) ! j)
    (1 * 2 ^ j)
    (fri-sibling-index
      (fri-layer-lengths (length fl) (clength * scale) ! j)
      (fri-layer-indices (length fl) ?idx (clength * scale) ! j))
  xp xp-path xn xn-path
    (fri-layer-indices (length fl) ?idx (clength * scale) ! j mod
      (fri-layer-lengths (length fl) (clength * scale) ! j div 2))
  v
    (comp-layer-chunks ! j)
using previous-step comp-layer-chunks-eq by simp
from mfold-fri-layer-opening-successor-recorded-step-from-previous
  [OF comp-fri[unfolded map-eq] comp-layers-actual tr-s3 idx-bound
    all-layer-bounds j-bound previous-step-actual]
obtain sxp-path sxn sxn-path snext where succ-actual:
  fri-layer-step-evidence (snd (fl ! Suc j)) (fst (fl ! Suc j))
    (fri-layer-lengths (length fl) (clength * scale) ! Suc j)
    (fri-layer-indices (length fl) ?idx (clength * scale) ! Suc j)
    (1 * 2 ^ Suc j)
    (fri-sibling-index
      (fri-layer-lengths (length fl) (clength * scale) ! Suc j)
      (fri-layer-indices (length fl) ?idx (clength * scale) ! Suc j))
  v sxp-path sxn sxn-path
    (fri-layer-indices (length fl) ?idx (clength * scale) ! Suc j mod
      (fri-layer-lengths (length fl) (clength * scale) ! Suc j div 2))
  snext
    (comp-layer-chunks ! Suc j)
by blast
have succ:
  fri-layer-step-evidence (snd (fl ! Suc j)) (fst (fl ! Suc j))
    (fri-layer-lengths (length fl) (clength * scale) ! Suc j)
    (fri-layer-indices (length fl) ?idx (clength * scale) ! Suc j)
    (2 ^ Suc j)
    (fri-sibling-index
      (fri-layer-lengths (length fl) (clength * scale) ! Suc j)
      (fri-layer-indices (length fl) ?idx (clength * scale) ! Suc j))
  v sxp-path sxn sxn-path
    (fri-layer-indices (length fl) ?idx (clength * scale) ! Suc j mod
      (fri-layer-lengths (length fl) (clength * scale) ! Suc j div 2))
  snext
    (composition-layers ! Suc j)
using succ-actual comp-layer-chunks-eq by simp
show ?thesis

```

using succ by blast
qed

lemma *accepted-fri-opening-transcript-composition-successor-recorded-step-from-previous-at:*
assumes *fri-openings:*

accepted-fri-opening-transcript s out trace-roots trace-bs trace-final
dg composition-roots composition-bs composition-final query-idxs
trace-round-layers composition-round-layers

and *out-eq:* *out = Some (result, final-state)*
and *round-bound:* *i < rounds*
and *layer-bound:* *Suc j < length composition-roots*
and *all-layer-bounds:*

$\bigwedge \text{raw } k. k < \text{length } \text{composition-roots} \implies$
 $0 < \text{fri-layer-lengths } (\text{length } \text{composition-roots})$
 $(\text{clength} * \text{scale}) ! k \wedge$
 $\text{fri-layer-indices } (\text{length } \text{composition-roots}) (\text{index } (\text{to-nat } \text{raw}))$
 $(\text{clength} * \text{scale}) ! k <$
 $\text{fri-layer-lengths } (\text{length } \text{composition-roots}) (\text{clength} * \text{scale}) ! k$

and *previous-step:*

fri-layer-step-evidence (composition-roots ! j)
(composition-bs ! j)
(fri-evidence-layer-len composition-roots j)
(fri-evidence-layer-idx composition-roots query-idxs i j)
 $(2 \wedge j)$
(fri-sibling-index (fri-evidence-layer-len composition-roots j)
(fri-evidence-layer-idx composition-roots query-idxs i j))
xp xp-path xn xn-path
(fri-evidence-next-idx composition-roots query-idxs i j)
v
(composition-round-layers ! i ! j)

shows

$\exists \text{xp-path}' \text{xn}' \text{xn-path}' \text{next-value.}$
fri-layer-step-evidence (composition-roots ! Suc j)
(composition-bs ! Suc j)
(fri-evidence-layer-len composition-roots (Suc j))
(fri-evidence-layer-idx composition-roots query-idxs i (Suc j))
 $(2 \wedge \text{Suc } j)$
(fri-sibling-index
(fri-evidence-layer-len composition-roots (Suc j))
(fri-evidence-layer-idx composition-roots query-idxs i
(Suc j)))
v xp-path' xn' xn-path'
(fri-evidence-next-idx composition-roots query-idxs i (Suc j))
next-value
(composition-round-layers ! i ! Suc j)

proof –

from *fri-openings* **obtain** *result' final-state' fr f-fl as fl query-state*
raw-idxs query-chunks **where**
out-def: *out = Some (result', final-state')*

```

and trace-roots-eq: trace-roots = map snd f-fl
and composition-roots-eq: composition-roots = map snd fl
and composition-bs-eq: composition-bs = map fst fl
and outcome:
  Some (result', final-state') ∈
    set-dist (execute
      (ntimes
        (verifier-query-round-program fr f-fl trace-final as fl
          composition-final)
        rounds)
      query-state)
and len-raw: length raw-idxs = rounds
and query-idxs-eq:
  query-idxs = map (λraw. index (to-nat raw)) raw-idxs
and len-query-chunks: length query-chunks = rounds
and transcript-query:
  PTranscript query-state =
    List.concat query-chunks @ PTranscript final-state'
and query-chunk:
  ∧i. i < rounds ⇒
    verifier-query-round-chunk (query-idxs ! i)
    trace-roots composition-roots (query-chunks ! i)
and replay-lookup:
  ∧i. i < rounds ⇒
    fmlookup (HashMap final-state')
      (QueryIndexChallenge (PQueryCounter query-state + i)
        (state-after-query-chunks (PState query-state) query-chunks i)) =
      Some (raw-idxs ! i)
and recorded:
  ∧i. i < rounds ⇒
    query-round-fri-layer-transcripts (query-idxs ! i)
    trace-roots composition-roots (query-chunks ! i)
    (trace-round-layers ! i) (composition-round-layers ! i)
unfolding accepted-fri-opening-transcript-def by blast
have result-eq: result' = result
using out-def out-eq by simp
have final-state-eq: final-state' = final-state
using out-def out-eq by simp
have outcome':
  Some (result, final-state) ∈
    set-dist (execute
      (ntimes
        (verifier-query-round-program fr f-fl trace-final as fl
          composition-final)
        rounds)
      query-state)
using outcome result-eq final-state-eq by simp
let ?round =
  verifier-query-round-program fr f-fl trace-final as fl composition-final

```

```

from query-opening-ntimes-outcome-decomp-at[OF outcome' round-bound]
obtain prefix x suffix s-i s-suc where
  prefix:
    Some (prefix, s-i) ∈
      set-dist (execute (ntimes ?round i) query-state)
  and round:
    Some (x, s-suc) ∈ set-dist (execute ?round s-i)
  and suffix:
    Some (suffix, final-state) ∈
      set-dist (execute (ntimes ?round (rounds - Suc i)) s-suc)
  by blast
have x-eq: x = ()
  by (cases x) simp
show ?thesis
proof (rule query-opening-ntimes-prefix-transcript-state-alignment
  [OF prefix])
  fix prefix-query-idxs prefix-chunks
  assume len-prefix-idxs: length prefix-query-idxs = i
  and len-prefix-chunks: length prefix-chunks = i
  and transcript-prefix:
    PTranscript query-state =
      List.concat prefix-chunks @ PTranscript s-i
  and state-i:
    PState s-i =
      state-after-query-chunks (PState query-state) prefix-chunks i
  and counter-i: PQueryCounter s-i = PQueryCounter query-state + i
  and prefix-ext: query-state ≤ s-i
  and prefix-rounds:
     $\bigwedge k. k < i \implies$ 
      verifier-query-round-chunk (prefix-query-idxs ! k)
      (map snd f-fl) (map snd fl) (prefix-chunks ! k)
  from round[unfolded x-eq verifier-query-round-program-alt-def]
  obtain raw s0 where
    raw-out:
      Some (raw, s0) ∈
        set-dist (execute receive-query-index-challenge s-i)
  and after:
    Some ((), s-suc) ∈
      set-dist (execute
        (verifier-query-round-after-index-program fr f-fl trace-final as
          fl composition-final raw) s0)
  by (auto elim!: set-dist-bindE)
  have tr-s0: PTranscript s0 = PTranscript s-i
  using receive-query-index-challenge-outcome[OF raw-out] by simp
  show ?thesis
  proof (rule verifier-query-round-program-outcome[OF round[unfolded x-eq]])
    fix raw' idx chunk
    assume idx-eq: idx = index (to-nat raw')
    and chunk-shape:

```

```

    verifier-query-round-chunk idx (map snd f-fl) (map snd fl)
    chunk
and transcript-round:
    PTranscript s-i = chunk @ PTranscript s-suc
and state-suc:
    PState s-suc = foldl concat (PState s-i) chunk
and round-ext:  $s-i \leq s-suc$ 
and counter-suc:
    PQueryCounter s-suc = Suc (PQueryCounter s-i)
and lookup-suc:
    fmlookup (HashMap s-suc)
    (QueryIndexChallenge (PQueryCounter s-i) (PState s-i)) =
    Some raw'
have raw-eq': raw = raw'
proof -
  have lookup-s0:
    fmlookup (HashMap s0)
    (QueryIndexChallenge (PQueryCounter s-i) (PState s-i)) =
    Some raw
    using receive-query-index-challenge-outcome[OF raw-out] by simp
have s0-suc:  $s0 \leq s-suc$ 
    using verifier-query-round-after-index-program-outcome[OF after]
    by simp
have lookup-suc-raw:
    fmlookup (HashMap s-suc)
    (QueryIndexChallenge (PQueryCounter s-i) (PState s-i)) =
    Some raw
    using hash-extension-lookup[OF lookup-s0 s0-suc] .
show ?thesis
    using lookup-suc lookup-suc-raw by simp
qed
show ?thesis
proof (rule query-opening-ntimes-suffix-transcript-state-alignment
  [OF suffix])
  fix suffix-query-idxs suffix-chunks
  assume len-suffix-idxs:
    length suffix-query-idxs = rounds - Suc i
  and len-suffix-chunks:
    length suffix-chunks = rounds - Suc i
  and transcript-suffix:
    PTranscript s-suc =
    List.concat suffix-chunks @ PTranscript final-state
  and suffix-state:
    PState final-state =
    state-after-query-chunks (PState s-suc) suffix-chunks
    (rounds - Suc i)
  and suffix-counter:
    PQueryCounter final-state =
    PQueryCounter s-suc + (rounds - Suc i)

```



```

and suffix-ext:  $s\text{-suc} \leq \text{final-state}$ 
and suffix-rounds:
   $\bigwedge k. k < \text{rounds} - \text{Suc } i \implies$ 
    verifier-query-round-chunk (suffix-query-idxs ! k)
      (map snd f-fl) (map snd fl) (suffix-chunks ! k)
let ?selected-chunks = prefix-chunks @ chunk # suffix-chunks
have len-selected-chunks: length ?selected-chunks = rounds
  using len-prefix-chunks len-suffix-chunks round-bound by simp
have selected-chunk-lens:
   $\bigwedge k. k < \text{rounds} \implies$ 
    length (?selected-chunks ! k) =
      verifier-query-round-transcript-length (query-idxs ! k)
        (map snd f-fl) (map snd fl)
proof -
  fix k
  assume k-bound:  $k < \text{rounds}$ 
  show
    length (?selected-chunks ! k) =
      verifier-query-round-transcript-length (query-idxs ! k)
        (map snd f-fl) (map snd fl)
  proof (cases  $k < i$ )
    case True
    have nth-eq: ?selected-chunks ! k = prefix-chunks ! k
      using True len-prefix-chunks by (simp add: nth-append)
    have
      length (prefix-chunks ! k) =
        verifier-query-round-transcript-length
          (prefix-query-idxs ! k) (map snd f-fl) (map snd fl)
      by (rule verifier-query-round-chunk-length
        [OF prefix-rounds[OF True]])
    then show ?thesis
      using nth-eq
      by (simp add:
        verifier-query-round-transcript-length-index-irrelevant)
  next
  case False
  show ?thesis
  proof (cases  $k = i$ )
    case True
    have nth-eq: ?selected-chunks ! k = chunk
      using True len-prefix-chunks by (simp add: nth-append)
    have
      length chunk =
        verifier-query-round-transcript-length idx
          (map snd f-fl) (map snd fl)
      by (rule verifier-query-round-chunk-length[OF chunk-shape])
    then show ?thesis
      using nth-eq
      by (simp add:

```

```

      verifier-query-round-transcript-length-index-irrelevant)
next
  case False
  let ?k = k - Suc i
  have ge: Suc i ≤ k
    using ⟨¬ k < i⟩ False by linarith
  have k-bound': ?k < rounds - Suc i
    using ge k-bound by linarith
  have nth-eq: ?selected-chunks ! k = suffix-chunks ! ?k
    using ge len-prefix-chunks by (simp add: nth-append)
  have
    length (suffix-chunks ! ?k) =
      verifier-query-round-transcript-length
        (suffix-query-idxs ! ?k) (map snd f-fl) (map snd fl)
    by (rule verifier-query-round-chunk-length
        [OF suffix-rounds[OF k-bound']])
  then show ?thesis
    using nth-eq
    by (simp add:
        verifier-query-round-transcript-length-index-irrelevant)
qed
qed
qed
have query-chunk-f:
  ∧k. k < rounds ⇒
    verifier-query-round-chunk (query-idxs ! k)
      (map snd f-fl) (map snd fl) (query-chunks ! k)
  using query-chunk trace-roots-eq composition-roots-eq by simp
have query-chunk-lens:
  ∧k. k < rounds ⇒
    length (query-chunks ! k) =
      verifier-query-round-transcript-length (query-idxs ! k)
        (map snd f-fl) (map snd fl)
  using query-chunk-f verifier-query-round-chunk-length by blast
have selected-transcript:
  PTranscript query-state =
    List.concat ?selected-chunks @ PTranscript final-state
  using transcript-prefix transcript-round transcript-suffix by simp
have concat-eq: List.concat ?selected-chunks = List.concat query-chunks
  using selected-transcript transcript-query final-state-eq by simp
have chunks-eq: ?selected-chunks = query-chunks
proof -
  have same-lens:
    ∧k. k < length ?selected-chunks ⇒
      length (?selected-chunks ! k) = length (query-chunks ! k)
    using selected-chunk-lens query-chunk-lens len-selected-chunks
    by simp
  have concat-eq-empty:
    List.concat ?selected-chunks @ [] = List.concat query-chunks

```

```

    using concat-eq by simp
  have len-eq: length ?selected-chunks = length query-chunks
    using len-selected-chunks len-query-chunks by simp
  have ?selected-chunks = query-chunks  $\wedge$  ( $\square :: 'f \text{ list}$ ) =  $\square$ 
    by (rule concat-append-eq-concat-same-chunk-lengths
        [OF len-eq same-lens concat-eq-empty])
  then show ?thesis
    by simp
qed
have prefix-state-eq:
  state-after-query-chunks (PState query-state) prefix-chunks i =
  state-after-query-chunks (PState query-state) query-chunks i
proof -
  have
    state-after-query-chunks (PState query-state) ?selected-chunks i =
    state-after-query-chunks (PState query-state) prefix-chunks i
  by (rule state-after-query-chunks-append-prefix
      [OF len-prefix-chunks])
  then show ?thesis
    using chunks-eq by simp
qed
have lookup-final:
  fmlookup (HashMap final-state)
    (QueryIndexChallenge (PQueryCounter query-state + i)
      (state-after-query-chunks (PState query-state) query-chunks i)) =
  Some raw'
proof -
  have lookup-suc':
    fmlookup (HashMap s-suc)
      (QueryIndexChallenge (PQueryCounter query-state + i)
        (state-after-query-chunks (PState query-state)
          prefix-chunks i)) =
    Some raw'
  using lookup-suc state-i counter-i by simp
  have lookup-final-prefix:
    fmlookup (HashMap final-state)
      (QueryIndexChallenge (PQueryCounter query-state + i)
        (state-after-query-chunks (PState query-state)
          prefix-chunks i)) =
    Some raw'
  using hash-extension-lookup[OF lookup-suc' suffix-ext] .
  show ?thesis
    using lookup-final-prefix prefix-state-eq by simp
qed
have raw-idx-eq: raw' = raw-idxs ! i
  using lookup-final replay-lookup[OF round-bound] final-state-eq
  by simp
have idx-eq-query: idx = query-idxs ! i
  using idx-eq raw-idx-eq query-idxs-eq len-raw round-bound by simp

```

```

have selected-i: ?selected-chunks ! i = chunk
  using len-prefix-chunks by (simp add: nth-append)
have chunk-i-eq: chunk = query-chunks ! i
  using chunks-eq selected-i by simp
have recorded-i:
  query-round-fri-layer-transcripts (index (to-nat raw))
    (map snd f-fl) (map snd fl) chunk (trace-round-layers ! i)
    (composition-round-layers ! i)
  using recorded[OF round-bound] idx-eq idx-eq-query raw-eq'
    chunk-i-eq trace-roots-eq composition-roots-eq
  by simp
have tr-s0-suc:
  PTranscript s0 = chunk @ PTranscript s-suc
  using tr-s0 transcript-round by simp
have suc-j-bound-fl: Suc j < length fl
  using layer-bound composition-roots-eq by simp
have all-layer-bounds-fl:
   $\bigwedge k. k < \text{length } fl \implies$ 
  0 < fri-layer-lengths (length fl) (clength * scale) ! k  $\wedge$ 
  fri-layer-indices (length fl) (index (to-nat raw))
    (clength * scale) ! k <
  fri-layer-lengths (length fl) (clength * scale) ! k
  using all-layer-bounds composition-roots-eq by simp
have raw-idx-query: index (to-nat raw) = query-idxs ! i
  using idx-eq idx-eq-query raw-eq' by simp
have previous-step-fl:
  fri-layer-step-evidence (snd (fl ! j)) (fst (fl ! j))
    (fri-layer-lengths (length fl) (clength * scale) ! j)
    (fri-layer-indices (length fl) (index (to-nat raw))
      (clength * scale) ! j)
    (2 ^ j)
    (fri-sibling-index
      (fri-layer-lengths (length fl) (clength * scale) ! j)
      (fri-layer-indices (length fl) (index (to-nat raw))
        (clength * scale) ! j))
    xp xp-path xn xn-path
    (fri-layer-indices (length fl) (index (to-nat raw))
      (clength * scale) ! j mod
      (fri-layer-lengths (length fl) (clength * scale) ! j div 2))
  v
  (composition-round-layers ! i ! j)
using previous-step composition-roots-eq composition-bs-eq
  raw-idx-query layer-bound
unfolding fri-evidence-layer-len-def fri-evidence-layer-idx-def
  fri-evidence-next-idx-def
by simp
from verifier-query-round-after-index-composition-successor-recorded-step-from-previous
  [OF after recorded-i tr-s0-suc suc-j-bound-fl
    all-layer-bounds-fl previous-step-fl]

```

```

obtain sxp-path sxn sxn-path snext where succ-fl:
  fri-layer-step-evidence (snd (fl ! Suc j))
    (fst (fl ! Suc j))
    (fri-layer-lengths (length fl) (clength * scale) ! Suc j)
    (fri-layer-indices (length fl) (index (to-nat raw)))
      (clength * scale) ! Suc j)
    ( $2 \wedge \text{Suc } j$ )
    (fri-sibling-index
      (fri-layer-lengths (length fl) (clength * scale) ! Suc j)
      (fri-layer-indices (length fl) (index (to-nat raw)))
      (clength * scale) ! Suc j))
  v sxp-path sxn sxn-path
    (fri-layer-indices (length fl) (index (to-nat raw)))
      (clength * scale) ! Suc j mod
      (fri-layer-lengths (length fl) (clength * scale) ! Suc j
        div 2))
  snext
    (composition-round-layers ! i ! Suc j)
by blast
have succ:
  fri-layer-step-evidence (composition-roots ! Suc j)
    (composition-bs ! Suc j)
    (fri-evidence-layer-len composition-roots (Suc j))
    (fri-evidence-layer-idx composition-roots query-idxs i
      (Suc j))
    ( $2 \wedge \text{Suc } j$ )
    (fri-sibling-index
      (fri-evidence-layer-len composition-roots (Suc j))
      (fri-evidence-layer-idx composition-roots query-idxs i
        (Suc j))))
  v sxp-path sxn sxn-path
    (fri-evidence-next-idx composition-roots query-idxs i (Suc j))
  snext
    (composition-round-layers ! i ! Suc j)
using succ-fl composition-roots-eq composition-bs-eq
  raw-idx-query layer-bound
unfolding fri-evidence-layer-len-def fri-evidence-layer-idx-def
  fri-evidence-next-idx-def
by simp
show ?thesis
using succ by blast
qed
qed
qed
qed

lemma generic-fri-sampled-successor-conflict-imp-same-layer-with-source-steps:
assumes successor:
  generic-fri-sampled-successor-opening-conflict roots challenges

```

query-idxs round-layers
and *len-challenges: length challenges = length roots*
and *source-step:*
 $\bigwedge$ *source-round layer-idx v.*
source-round < length query-idxs $\implies$
Suc layer-idx < length challenges $\implies$
generic-fri-round-forced-next-value roots challenges query-idxs
round-layers source-round layer-idx v $\implies$
 $\exists$ *xp-path xn xn-path.*
fri-layer-step-evidence
(roots ! Suc layer-idx)
(challenges ! Suc layer-idx)
(fri-evidence-layer-len roots (Suc layer-idx))
(fri-evidence-layer-idx roots query-idxs source-round
(Suc layer-idx))
(2 $\wedge$ Suc layer-idx)
(fri-sibling-index
(fri-evidence-layer-len roots (Suc layer-idx))
(fri-evidence-layer-idx roots query-idxs source-round
(Suc layer-idx)))
v xp-path xn xn-path
(fri-evidence-next-idx roots query-idxs source-round
(Suc layer-idx))
(fri-evidence-next-value roots challenges query-idxs
source-round (Suc layer-idx) v xn)
(round-layers ! source-round ! Suc layer-idx)
shows
generic-fri-sampled-same-layer-opening-conflict roots challenges
query-idxs round-layers
proof –
from *generic-fri-sampled-successor-opening-conflictE[OF successor]*
obtain *source-round target-round layer-idx v xp xp-path xn xn-path*
where *source-bound: source-round < length query-idxs*
and *target-bound: target-round < length query-idxs*
and *layer-bound: Suc layer-idx < length challenges*
and *forced:*
generic-fri-round-forced-next-value roots challenges query-idxs
round-layers source-round layer-idx v
and *target-step:*
fri-layer-step-evidence
(roots ! Suc layer-idx)
(challenges ! Suc layer-idx)
(fri-evidence-layer-len roots (Suc layer-idx))
(fri-evidence-layer-idx roots query-idxs target-round
(Suc layer-idx))
(2 $\wedge$ Suc layer-idx)
(fri-sibling-index
(fri-evidence-layer-len roots (Suc layer-idx))
(fri-evidence-layer-idx roots query-idxs target-round

```

      (Suc layer-idx)))
    xp xp-path xn xn-path
    (fri-evidence-next-idx roots query-idxs target-round
      (Suc layer-idx))
    (fri-evidence-next-value roots challenges query-idxs target-round
      (Suc layer-idx) xp xn)
    (round-layers ! target-round ! Suc layer-idx)
  and conflict:
    (fri-evidence-next-idx roots query-idxs source-round layer-idx =
      fri-evidence-layer-idx roots query-idxs target-round
        (Suc layer-idx)  $\wedge$ 
      v  $\neq$  xp)  $\vee$ 
    (fri-evidence-next-idx roots query-idxs source-round layer-idx =
      fri-sibling-index
        (fri-evidence-layer-len roots (Suc layer-idx))
        (fri-evidence-layer-idx roots query-idxs target-round
          (Suc layer-idx))  $\wedge$ 
      v  $\neq$  xn)
  by blast
from source-step[OF source-bound layer-bound forced]
obtain vp-path vn vn-path where source-suc-step:
  fri-layer-step-evidence
    (roots ! Suc layer-idx)
    (challenges ! Suc layer-idx)
    (fri-evidence-layer-len roots (Suc layer-idx))
    (fri-evidence-layer-idx roots query-idxs source-round
      (Suc layer-idx))
    (2  $\wedge$  Suc layer-idx)
    (fri-sibling-index
      (fri-evidence-layer-len roots (Suc layer-idx))
      (fri-evidence-layer-idx roots query-idxs source-round
        (Suc layer-idx)))
  v vp-path vn vn-path
  (fri-evidence-next-idx roots query-idxs source-round
    (Suc layer-idx))
  (fri-evidence-next-value roots challenges query-idxs source-round
    (Suc layer-idx) v vn)
  (round-layers ! source-round ! Suc layer-idx)
by blast
have source-idx-suc:
  fri-evidence-layer-idx roots query-idxs source-round (Suc layer-idx) =
    fri-evidence-next-idx roots query-idxs source-round layer-idx
by (rule fri-evidence-layer-idx-Suc)
  (use layer-bound len-challenges in simp)
have layer-suc-bound: Suc layer-idx < length challenges
  using layer-bound .
have same-conflict:
  (fri-evidence-layer-idx roots query-idxs source-round (Suc layer-idx) =
    fri-evidence-layer-idx roots query-idxs target-round

```

```

      (Suc layer-idx) ∧
      v ≠ xp) ∨
      (fri-evidence-layer-idx roots query-idxs source-round (Suc layer-idx) =
        fri-sibling-index
        (fri-evidence-layer-len roots (Suc layer-idx))
        (fri-evidence-layer-idx roots query-idxs target-round
          (Suc layer-idx)) ∧
        v ≠ xn)
proof -
from conflict show ?thesis
proof
  assume c:
    fri-evidence-next-idx roots query-idxs source-round layer-idx =
    fri-evidence-layer-idx roots query-idxs target-round
      (Suc layer-idx) ∧
    v ≠ xp
  then show ?thesis
    using source-idx-suc by simp
next
  assume c:
    fri-evidence-next-idx roots query-idxs source-round layer-idx =
    fri-sibling-index
      (fri-evidence-layer-len roots (Suc layer-idx))
      (fri-evidence-layer-idx roots query-idxs target-round
        (Suc layer-idx)) ∧
    v ≠ xn
  then show ?thesis
    using source-idx-suc by simp
qed
qed
show ?thesis
  unfolding generic-fri-sampled-same-layer-opening-conflict-def
  apply (rule exI[where x=source-round])
  apply (rule exI[where x=target-round])
  apply (rule exI[where x=Suc layer-idx])
  apply (rule exI[where x=v])
  apply (rule exI[where x=vp-path])
  apply (rule exI[where x=vn])
  apply (rule exI[where x=vn-path])
  apply (rule exI[where x=xp])
  apply (rule exI[where x=xp-path])
  apply (rule exI[where x=xn])
  apply (rule exI[where x=xn-path])
  apply (intro conjI)
  using source-bound target-bound layer-suc-bound source-suc-step
    target-step same-conflict
  by blast+
qed

```


lemma *generic-fri-sampled-same-layer-conflict-imp-merkle-or-slot-gap*:
assumes *conflict*:
generic-fri-sampled-same-layer-opening-conflict roots challenges
query-idxs round-layers
and *len-challenges*: *length challenges = length roots*
and *auth-step*:
 $\bigwedge \text{round-idx layer-idx.}$
 $\text{round-idx} < \text{length query-idxs} \implies$
 $\text{layer-idx} < \text{length challenges} \implies$
generic-fri-transcript-step-with-authenticated-chunk roots
challenges query-idxs round-layers final-state round-idx layer-idx
shows
partial-merkle-inconsistency-bad s (Some (result, final-state)) $\vee$
generic-fri-authenticated-slot-recorded-chunk-gap roots query-idxs
round-layers final-state
proof –
from *generic-fri-sampled-same-layer-imp-authenticated-or-recorded-auth-gap*
 $[OF \text{ conflict, of final-state}]$
consider
generic-fri-sampled-same-layer-opening-conflict-with-authenticated-chunks
roots challenges query-idxs round-layers final-state
 $|$ *generic-fri-sampled-same-layer-recorded-auth-gap roots challenges*
query-idxs round-layers final-state
by *blast*
then show *?thesis*
proof *cases*
case 1
then have *partial-merkle-inconsistency-bad s (Some (result, final-state))*
by (rule
generic-fri-sampled-same-layer-opening-conflict-with-authenticated-chunks-imp-partial-merkle)
(rule *len-challenges*)
then show *?thesis* **by** *simp*
next
case 2
then have *auth-gap*:
generic-fri-sampled-same-layer-authenticated-recorded-auth-gap roots
challenges query-idxs round-layers final-state
by (rule *generic-fri-recorded-auth-gap-imp-authenticated-recorded-auth-gap*)
(rule *auth-step*)
have *generic-fri-authenticated-slot-recorded-chunk-gap roots query-idxs*
round-layers final-state
by (rule *generic-fri-authenticated-recorded-auth-gap-imp-slot-recorded-chunk-gap*
 $[OF \text{ auth-gap}]$)
then show *?thesis* **by** *simp*
qed
qed

lemma *composition-fri-verifier-tied-next-value-replay-gap-imp-merkle-or-slot-on-support*:
assumes *support*: *out* $\in$ *set-dist* (*execute verify-monad s*)

and *gap*:
composition-fri-verifier-tied-next-value-replay-gap s out
shows
partial-merkle-inconsistency-bad s out $\vee$
composition-fri-verifier-tied-authenticated-slot-recorded-chunk-gap
s out
proof –
from *gap* **have** *sampled*:
composition-fri-verifier-tied-sampled-next-value-conflict s out
unfolding *composition-fri-verifier-tied-next-value-replay-gap-def*
by *simp*
from *composition-fri-verifier-tied-sampled-next-value-conflictE*[*OF sampled*]
obtain *trace-roots trace-bs trace-final fri-dg composition-roots*
composition-bs composition-final fri-query-idxs trace-round-layers
composition-round-layers fr as trace-openings composition-openings
trace-table composition-table round-idx round-idx' layer-idx v v'
where *fri-openings*:
accepted-fri-opening-transcript s out trace-roots trace-bs
trace-final fri-dg composition-roots composition-bs composition-final
fri-query-idxs trace-round-layers composition-round-layers
and *aligned*:
accepted-with-partial-initial-openings-aligned s out fr trace-roots
trace-final as fri-dg composition-roots composition-final
fri-query-idxs trace-openings composition-openings
and *trace-cand*: *partial-trace-table-candidate trace-table trace-openings*
and *comp-cand*:
partial-composition-table-candidate composition-table
composition-openings
and *trace-low*: *trace-table-low-degree trace-table*
and *comp-not-low*:
 $\neg$ *composition-table-low-degree maxDegree composition-table*
and *round-bound*: *round-idx < length fri-query-idxs*
and *round-bound'*: *round-idx' < length fri-query-idxs*
and *layer-bound*: *layer-idx < length composition-bs*
and *same-idx*:
fri-evidence-next-idx composition-roots fri-query-idxs round-idx
layer-idx =
fri-evidence-next-idx composition-roots fri-query-idxs round-idx'
layer-idx
and *forced*:
generic-fri-round-forced-next-value composition-roots composition-bs
fri-query-idxs composition-round-layers round-idx layer-idx v
and *forced'*:
generic-fri-round-forced-next-value composition-roots composition-bs
fri-query-idxs composition-round-layers round-idx' layer-idx v'
and *neq*: $v \neq v'$
using *sampled*
by (*elim composition-fri-verifier-tied-sampled-next-value-conflictE*)
from *fri-openings* **obtain** *result final-state* **where** *out-eq*:

```

out = Some (result, final-state)
unfolding accepted-fri-opening-transcript-def by blast
have support-some:
  Some (result, final-state) ∈ set-dist (execute verify-monad s)
  using support out-eq by simp
have challenges:
  accepted-fri-challenges s (Some (result, final-state))
  trace-bs fri-dg composition-bs
  using accepted-fri-opening-transcript-challenges[OF fri-openings]
  out-eq by simp
have degree-bound: to-nat fri-dg ≤ maxDegree
  by (rule verify-monad-accepted-fri-challenges-degree-bound
    [OF support-some challenges])
have len-bs-roots: length composition-bs = length composition-roots
  using accepted-fri-opening-transcript-shapes(4)[OF fri-openings] .
have layer-bound-roots: layer-idx < length composition-roots
  using layer-bound len-bs-roots by simp
have len-query: length fri-query-idxs = rounds
  using accepted-fri-opening-transcript-shapes(7)[OF fri-openings] .
have round-bound-rounds: round-idx < rounds
  using round-bound len-query by simp
have round-bound-rounds': round-idx' < rounds
  using round-bound' len-query by simp
have raw-bound:
  fri-evidence-layer-idx composition-roots fri-query-idxs round-idx
  layer-idx <
  fri-evidence-layer-len composition-roots layer-idx
  by (rule
    accepted-fri-opening-transcript-composition-evidence-layer-idx-bound
    [OF fri-openings degree-bound round-bound layer-bound-roots])
have raw-bound':
  fri-evidence-layer-idx composition-roots fri-query-idxs round-idx'
  layer-idx <
  fri-evidence-layer-len composition-roots layer-idx
  by (rule
    accepted-fri-opening-transcript-composition-evidence-layer-idx-bound
    [OF fri-openings degree-bound round-bound' layer-bound-roots])
have even-len:
  2 dvd fri-evidence-layer-len composition-roots layer-idx
  by (rule accepted-fri-opening-transcript-composition-layer-arithmetic(1)
    [OF fri-openings degree-bound layer-bound])
have round-product:
  fri-evidence-layer-len composition-roots layer-idx * 2 ^ layer-idx =
  clength * scale
  by (rule accepted-fri-opening-transcript-composition-layer-arithmetic(2)
    [OF fri-openings degree-bound layer-bound])
have auth-step:
  ∧i j.
  i < length fri-query-idxs ⇒

```

```

     $j < \text{length } \text{composition-bs} \implies$ 
     $\text{generic-fri-transcript-step-with-authenticated-chunk } \text{composition-roots}$ 
     $\text{composition-bs } \text{fri-query-idxs } \text{composition-round-layers } \text{final-state } i \ j$ 
proof —
  fix  $i \ j$ 
  assume  $i\text{-bound}: i < \text{length } \text{fri-query-idxs}$ 
    and  $j\text{-bound}: j < \text{length } \text{composition-bs}$ 
  have  $i\text{-rounds}: i < \text{rounds}$ 
    using  $i\text{-bound } \text{len-query}$  by simp
  have  $j\text{-roots}: j < \text{length } \text{composition-roots}$ 
    using  $j\text{-bound } \text{len-bs-roots}$  by simp
  show
     $\text{generic-fri-transcript-step-with-authenticated-chunk}$ 
     $\text{composition-roots } \text{composition-bs } \text{fri-query-idxs}$ 
     $\text{composition-round-layers } \text{final-state } i \ j$ 
    by (rule
       $\text{accepted-fri-opening-transcript-composition-step-with-authenticated-chunk-at-raw}$ 
       $[\text{OF } \text{fri-openings } \text{out-eq } \text{degree-bound } i\text{-rounds } j\text{-roots}]$ )
qed
show ?thesis
proof (cases
   $\text{generic-fri-sampled-same-layer-opening-conflict } \text{composition-roots}$ 
   $\text{composition-bs } \text{fri-query-idxs } \text{composition-round-layers}$ )
case True
have  $\text{conflict-route}$ :
   $\text{partial-merkle-inconsistency-bad } s \ (\text{Some } (\text{result}, \text{final-state})) \vee$ 
   $\text{generic-fri-authenticated-slot-recorded-chunk-gap } \text{composition-roots}$ 
   $\text{fri-query-idxs } \text{composition-round-layers } \text{final-state}$ 
  by (rule  $\text{generic-fri-sampled-same-layer-conflict-imp-merkle-or-slot-gap}$ 
     $[\text{OF } \text{True } \text{len-bs-roots } \text{auth-step}]$ )
then show ?thesis
proof
  assume  $\text{partial-merkle-inconsistency-bad } s \ (\text{Some } (\text{result}, \text{final-state}))$ 
  then show ?thesis
    using  $\text{out-eq}$  by simp
next
  assume  $\text{slot-gap}$ :
   $\text{generic-fri-authenticated-slot-recorded-chunk-gap } \text{composition-roots}$ 
   $\text{fri-query-idxs } \text{composition-round-layers } \text{final-state}$ 
  have  $\text{route-slot-gap}$ :
   $\text{composition-fri-verifier-tied-authenticated-slot-recorded-chunk-gap}$ 
   $s \ \text{out}$ 
  unfolding
   $\text{composition-fri-verifier-tied-authenticated-slot-recorded-chunk-gap-def}$ 
  by (intro  $\text{exI } \text{conjI}$ )
    (rule  $\text{out-eq}$ , rule  $\text{fri-openings}$ , rule  $\text{aligned}$ , rule  $\text{trace-cand}$ ,
    rule  $\text{comp-cand}$ , rule  $\text{trace-low}$ , rule  $\text{comp-not-low}$ , rule  $\text{slot-gap}$ )
  then show ?thesis by simp
qed

```

```

next
case False
then have no-same:
  ¬ generic-fri-sampled-same-layer-opening-conflict
    composition-roots composition-bs fri-query-idxs
    composition-round-layers
  by simp
have v = v'
by (rule generic-fri-forced-next-value-eq-if-no-same-layer-conflict
  [OF no-same round-bound round-bound' layer-bound even-len
    raw-bound raw-bound' round-product same-idx forced forced'])
then show ?thesis
  using neq by simp
qed
qed

lemma composition-fri-verifier-tied-successor-replay-gap-imp-merkle-or-slot-on-support:
  assumes support: out ∈ set-dist (execute verify-monad s)
  and gap:
    composition-fri-verifier-tied-successor-replay-gap s out
  shows
    partial-merkle-inconsistency-bad s out ∨
    composition-fri-verifier-tied-authenticated-slot-recorded-chunk-gap
      s out
proof —
  from gap have sampled:
    composition-fri-verifier-tied-sampled-successor-opening-conflict s out
  unfolding composition-fri-verifier-tied-successor-replay-gap-def
  by simp
  obtain trace-roots trace-bs trace-final fri-dg composition-roots
    composition-bs composition-final fri-query-idxs trace-round-layers
    composition-round-layers fr as trace-openings composition-openings
    trace-table composition-table source-round target-round layer-idx v
    xp xp-path xn xn-path
  where fri-openings:
    accepted-fri-opening-transcript s out trace-roots trace-bs
      trace-final fri-dg composition-roots composition-bs composition-final
      fri-query-idxs trace-round-layers composition-round-layers
  and aligned:
    accepted-with-partial-initial-openings-aligned s out fr trace-roots
      trace-final as fri-dg composition-roots composition-final
      fri-query-idxs trace-openings composition-openings
  and trace-cand: partial-trace-table-candidate trace-table trace-openings
  and comp-cand:
    partial-composition-table-candidate composition-table
      composition-openings
  and trace-low: trace-table-low-degree trace-table
  and comp-not-low:
    ¬ composition-table-low-degree maxDegree composition-table

```

```

and source-bound: source-round < length fri-query-idxs
and target-bound: target-round < length fri-query-idxs
and layer-bound: Suc layer-idx < length composition-bs
and forced:
  generic-fri-round-forced-next-value composition-roots composition-bs
  fri-query-idxs composition-round-layers source-round layer-idx v
and target-step:
  fri-layer-step-evidence
    (composition-roots ! Suc layer-idx)
    (composition-bs ! Suc layer-idx)
    (fri-evidence-layer-len composition-roots (Suc layer-idx))
    (fri-evidence-layer-idx composition-roots fri-query-idxs
      target-round (Suc layer-idx))
    (2 ^ Suc layer-idx)
    (fri-sibling-index
      (fri-evidence-layer-len composition-roots (Suc layer-idx))
      (fri-evidence-layer-idx composition-roots fri-query-idxs
        target-round (Suc layer-idx)))
    xp xp-path xn xn-path
    (fri-evidence-next-idx composition-roots fri-query-idxs
      target-round (Suc layer-idx))
    (fri-evidence-next-value composition-roots composition-bs
      fri-query-idxs target-round (Suc layer-idx) xp xn)
    (composition-round-layers ! target-round ! Suc layer-idx)
and conflict:
  (fri-evidence-next-idx composition-roots fri-query-idxs source-round
    layer-idx =
    fri-evidence-layer-idx composition-roots fri-query-idxs target-round
      (Suc layer-idx) ∧
    v ≠ xp) ∨
  (fri-evidence-next-idx composition-roots fri-query-idxs source-round
    layer-idx =
    fri-sibling-index
      (fri-evidence-layer-len composition-roots (Suc layer-idx))
      (fri-evidence-layer-idx composition-roots fri-query-idxs
        target-round (Suc layer-idx)) ∧
    v ≠ xn)
using sampled
by (elim composition-fri-verifier-tied-sampled-successor-opening-conflictE)
from fri-openings obtain result final-state where out-eq:
  out = Some (result, final-state)
unfolding accepted-fri-opening-transcript-def by blast
have support-some:
  Some (result, final-state) ∈ set-dist (execute verify-monad s)
using support out-eq by simp
have challenges:
  accepted-fri-challenges s (Some (result, final-state))
  trace-bs fri-dg composition-bs
using accepted-fri-opening-transcript-challenges[OF fri-openings]

```

```

out-eq by simp
have degree-bound: to-nat fri-dg ≤ maxDegree
  by (rule verify-monad-accepted-fri-challenges-degree-bound
      [OF support-some challenges])
have len-bs-roots: length composition-bs = length composition-roots
  using accepted-fri-opening-transcript-shapes(4)[OF fri-openings] .
have len-query: length fri-query-idxs = rounds
  using accepted-fri-opening-transcript-shapes(7)[OF fri-openings] .
have sampled-generic:
  generic-fri-sampled-successor-opening-conflict composition-roots
    composition-bs fri-query-idxs composition-round-layers
  unfolding generic-fri-sampled-successor-opening-conflict-def
  by (intro exI conjI)
    (rule source-bound, rule target-bound, rule layer-bound,
     rule forced, rule target-step, rule conflict)
have all-layer-bounds:
  ∧ raw k. k < length composition-roots ⇒
    0 < fri-layer-lengths (length composition-roots)
      (length * scale) ! k ∧
    fri-layer-indices (length composition-roots) (index (to-nat raw))
      (length * scale) ! k <
    fri-layer-lengths (length composition-roots) (length * scale) ! k
  by (rule accepted-fri-opening-transcript-composition-raw-layer-bound
      [OF fri-openings degree-bound])
have source-step:
  ∧ source-round layer-idx v.
    source-round < length fri-query-idxs ⇒
    Suc layer-idx < length composition-bs ⇒
    generic-fri-round-forced-next-value composition-roots composition-bs
      fri-query-idxs composition-round-layers source-round layer-idx v ⇒
  ∃ xp-path xn xn-path.
    fri-layer-step-evidence
      (composition-roots ! Suc layer-idx)
      (composition-bs ! Suc layer-idx)
      (fri-evidence-layer-len composition-roots (Suc layer-idx))
      (fri-evidence-layer-idx composition-roots fri-query-idxs
        source-round (Suc layer-idx))
      (2 ^ Suc layer-idx)
      (fri-sibling-index
        (fri-evidence-layer-len composition-roots (Suc layer-idx))
        (fri-evidence-layer-idx composition-roots fri-query-idxs
          source-round (Suc layer-idx)))
    v xp-path xn xn-path
      (fri-evidence-next-idx composition-roots fri-query-idxs
        source-round (Suc layer-idx))
      (fri-evidence-next-value composition-roots composition-bs
        fri-query-idxs source-round (Suc layer-idx) v xn)
      (composition-round-layers ! source-round ! Suc layer-idx)
proof -

```

```

fix source-round layer-idx v
assume source-round-bound: source-round < length fri-query-idxs
  and layer-bound': Suc layer-idx < length composition-bs
  and forced':
    generic-fri-round-forced-next-value composition-roots composition-bs
    fri-query-idxs composition-round-layers source-round layer-idx v
have source-round-rounds: source-round < rounds
  using source-round-bound len-query by simp
have layer-bound-roots: Suc layer-idx < length composition-roots
  using layer-bound' len-bs-roots by simp
from forced' obtain fxp fxp-path fxn fxn-path where previous-step:
  fri-layer-step-evidence (composition-roots ! layer-idx)
    (composition-bs ! layer-idx)
    (fri-evidence-layer-len composition-roots layer-idx)
    (fri-evidence-layer-idx composition-roots fri-query-idxs
      source-round layer-idx)
    (2 ^ layer-idx)
    (fri-sibling-index
      (fri-evidence-layer-len composition-roots layer-idx)
      (fri-evidence-layer-idx composition-roots fri-query-idxs
        source-round layer-idx))
  fxp fxp-path fxn fxn-path
  (fri-evidence-next-idx composition-roots fri-query-idxs
    source-round layer-idx)
  v
  (composition-round-layers ! source-round ! layer-idx)
unfolding generic-fri-round-forced-next-value-def by blast
from accepted-fri-opening-transcript-composition-successor-recorded-step-from-previous-at
  [OF fri-openings out-eq source-round-rounds layer-bound-roots
    all-layer-bounds previous-step]
obtain sxp-path sxn sxn-path snext where succ:
  fri-layer-step-evidence (composition-roots ! Suc layer-idx)
    (composition-bs ! Suc layer-idx)
    (fri-evidence-layer-len composition-roots (Suc layer-idx))
    (fri-evidence-layer-idx composition-roots fri-query-idxs
      source-round (Suc layer-idx))
    (2 ^ Suc layer-idx)
    (fri-sibling-index
      (fri-evidence-layer-len composition-roots (Suc layer-idx))
      (fri-evidence-layer-idx composition-roots fri-query-idxs
        source-round (Suc layer-idx)))
  v sxp-path sxn sxn-path
  (fri-evidence-next-idx composition-roots fri-query-idxs
    source-round (Suc layer-idx))
  snext
  (composition-round-layers ! source-round ! Suc layer-idx)
by blast
have snext-eq:
  snext =

```



```

    fri-evidence-next-value composition-roots composition-bs
    fri-query-idxs source-round (Suc layer-idx) v sxn
using fri-layer-step-evidenceD(3)[OF succ]
unfolding fri-evidence-next-value-def by simp
show
  ∃ xp-path xn xn-path.
    fri-layer-step-evidence
      (composition-roots ! Suc layer-idx)
      (composition-bs ! Suc layer-idx)
      (fri-evidence-layer-len composition-roots (Suc layer-idx))
      (fri-evidence-layer-idx composition-roots fri-query-idxs
        source-round (Suc layer-idx))
      (2 ^ Suc layer-idx)
      (fri-sibling-index
        (fri-evidence-layer-len composition-roots (Suc layer-idx))
        (fri-evidence-layer-idx composition-roots fri-query-idxs
          source-round (Suc layer-idx)))
      v xp-path xn xn-path
      (fri-evidence-next-idx composition-roots fri-query-idxs
        source-round (Suc layer-idx))
      (fri-evidence-next-value composition-roots composition-bs
        fri-query-idxs source-round (Suc layer-idx) v xn)
      (composition-round-layers ! source-round ! Suc layer-idx)
    using succ snext-eq by blast
qed
have same-layer:
  generic-fri-sampled-same-layer-opening-conflict composition-roots
  composition-bs fri-query-idxs composition-round-layers
by (rule generic-fri-sampled-successor-conflict-imp-same-layer-with-source-steps
  [OF sampled-generic len-bs-roots source-step])
have auth-step:
  ∧ i j.
    i < length fri-query-idxs ⇒
    j < length composition-bs ⇒
    generic-fri-transcript-step-with-authenticated-chunk composition-roots
    composition-bs fri-query-idxs composition-round-layers final-state i j
proof –
  fix i j
  assume i-bound: i < length fri-query-idxs
  and j-bound: j < length composition-bs
  have i-rounds: i < rounds
  using i-bound len-query by simp
  have j-roots: j < length composition-roots
  using j-bound len-bs-roots by simp
  show
    generic-fri-transcript-step-with-authenticated-chunk
    composition-roots composition-bs fri-query-idxs
    composition-round-layers final-state i j
  by (rule

```

```

    accepted-fri-opening-transcript-composition-step-with-authenticated-chunk-at-raw
    [OF fri-openings out-eq degree-bound i-rounds j-roots])
qed
have route:
  partial-merkle-inconsistency-bad s (Some (result, final-state)) ∨
  generic-fri-authenticated-slot-recorded-chunk-gap composition-roots
  fri-query-idxs composition-round-layers final-state
  by (rule generic-fri-sampled-same-layer-conflict-imp-merkle-or-slot-gap
    [OF same-layer len-bs-roots auth-step])
then show ?thesis
proof
  assume partial-merkle-inconsistency-bad s (Some (result, final-state))
  then show ?thesis
    using out-eq by simp
next
  assume slot-gap:
    generic-fri-authenticated-slot-recorded-chunk-gap composition-roots
    fri-query-idxs composition-round-layers final-state
  have route-slot-gap:
    composition-fri-verifier-tied-authenticated-slot-recorded-chunk-gap
    s out
  unfolding
    composition-fri-verifier-tied-authenticated-slot-recorded-chunk-gap-def
  by (intro exI conjI)
    (rule out-eq, rule fri-openings, rule aligned, rule trace-cand,
    rule comp-cand, rule trace-low, rule comp-not-low, rule slot-gap)
  then show ?thesis by simp
qed
qed

lemma wp-composition-fri-verifier-tied-successor-replay-gap-bound-from-merkle-and-slot:
  fixes M A :: prob
  assumes merkle-bound:
    wp-event verify-monad (partial-merkle-inconsistency-bad s) s ≤ M
  and slot-bound:
    wp-event verify-monad
    (composition-fri-verifier-tied-authenticated-slot-recorded-chunk-gap s)
    s ≤ A
  shows
    wp-event verify-monad
    (composition-fri-verifier-tied-successor-replay-gap s) s
    ≤ M + A
proof -
  let ?P = partial-merkle-inconsistency-bad s
  let ?Q = composition-fri-verifier-tied-authenticated-slot-recorded-chunk-gap s
  have event-le:
    wp-event verify-monad
    (composition-fri-verifier-tied-successor-replay-gap s) s ≤
    wp-event verify-monad (λout. ?P out ∨ ?Q out) s

```

```

    by (rule wp-event-mono-on-support)
      (rule
        composition-fri-verifier-tied-successor-replay-gap-imp-merkle-or-slot-on-support)
  also have ...  $\leq$  wp-event verify-monad ?P s +
    wp-event verify-monad ?Q s
    by (rule wp-event-union-bound)
  also have ...  $\leq$  M + A
    by (intro add-mono merkle-bound slot-bound)
  finally show ?thesis .
qed

lemma wp-composition-fri-verifier-tied-next-value-replay-gap-bound-from-merkle-and-slot:
  fixes M A :: prob
  assumes merkle-bound:
    wp-event verify-monad (partial-merkle-inconsistency-bad s) s  $\leq$  M
  and slot-bound:
    wp-event verify-monad
      (composition-fri-verifier-tied-authenticated-slot-recorded-chunk-gap s)
      s  $\leq$  A
  shows
    wp-event verify-monad
      (composition-fri-verifier-tied-next-value-replay-gap s) s
       $\leq$  M + A
proof -
  let ?P = partial-merkle-inconsistency-bad s
  let ?Q = composition-fri-verifier-tied-authenticated-slot-recorded-chunk-gap s
  have event-le:
    wp-event verify-monad
      (composition-fri-verifier-tied-next-value-replay-gap s) s  $\leq$ 
    wp-event verify-monad ( $\lambda$ out. ?P out  $\vee$  ?Q out) s
  by (rule wp-event-mono-on-support)
    (rule
      composition-fri-verifier-tied-next-value-replay-gap-imp-merkle-or-slot-on-support)
  also have ...  $\leq$  wp-event verify-monad ?P s +
    wp-event verify-monad ?Q s
    by (rule wp-event-union-bound)
  also have ...  $\leq$  M + A
    by (intro add-mono merkle-bound slot-bound)
  finally show ?thesis .
qed

end

end

```

```

theory Soundness-FRI-Trace-Replay-Gaps
  imports
    Soundness-FRI-Trace-Residuals

```

begin

Narrow downstream layer for trace header-tied replay gaps. This mirrors the composition replay-gap layer without growing the already-large trace residual theory.

context *soundness*

begin

lemma *accepted-fri-opening-transcript-trace-layer-arithmetic:*

assumes *fri-openings:*

*accepted-fri-opening-transcript s out trace-roots trace-bs trace-final
dg composition-roots composition-bs composition-final query-idxs
trace-round-layers composition-round-layers*

and *layer-bound: layer-idx < length trace-bs*

shows *2 dvd fri-evidence-layer-len trace-roots layer-idx*

and *fri-evidence-layer-len trace-roots layer-idx * 2 ^ layer-idx =
clength * scale*

proof –

obtain *N where eval-power: clength * scale = 2 ^ N*

using *eval-domain-length-power* **by** *blast*

have *bs-len: length trace-bs = length trace-roots*

using *accepted-fri-opening-transcript-shapes(2)[OF fri-openings]* .

have *roots-bound: layer-idx < length trace-roots*

using *layer-bound bs-len* **by** *simp*

have *roots-le: length trace-roots ≤ N*

proof –

have *len: length trace-roots = ceil-log clength*

using *accepted-fri-opening-transcript-shapes(1)[OF fri-openings]* .

have *clength ≤ clength * scale*

using *scale-pos* **by** *simp*

also have *... = 2 ^ N*

by *(rule eval-power)*

finally have *ceil-log clength ≤ N*

by *(rule ceil-log-le-power)*

then show *?thesis*

using *len* **by** *simp*

qed

have *suc-le: Suc layer-idx ≤ N*

using *roots-bound roots-le* **by** *simp*

have *dvd-eval-suc: (2::nat) ^ Suc layer-idx dvd clength * scale*

using *le-imp-power-dvd[OF suc-le] eval-power* **by** *simp*

show *2 dvd fri-evidence-layer-len trace-roots layer-idx*

unfolding *fri-evidence-layer-len-def*

by *(rule fri-layer-lengths-even-if-dvd[OF roots-bound dvd-eval-suc])*

have *le-N: layer-idx ≤ N*

using *roots-bound roots-le* **by** *simp*

have *dvd-eval: (2::nat) ^ layer-idx dvd clength * scale*

using *le-imp-power-dvd[OF le-N] eval-power* **by** *simp*

show *fri-evidence-layer-len trace-roots layer-idx **

```

    2 ^ layer-idx = clength * scale
  unfolding fri-evidence-layer-len-def
  by (rule fri-layer-lengths-round-product-if-dvd
      [OF roots-bound dvd-eval])
qed

lemma accepted-fri-opening-transcript-trace-evidence-layer-idx-bound:
  assumes fri-openings:
    accepted-fri-opening-transcript s out trace-roots trace-bs trace-final
    dg composition-roots composition-bs composition-final query-idxs
    trace-round-layers composition-round-layers
  and round-bound: round-idx < length query-idxs
  and layer-bound: layer-idx < length trace-roots
  shows
    fri-evidence-layer-idx trace-roots query-idxs round-idx layer-idx <
    fri-evidence-layer-len trace-roots layer-idx
proof -
  obtain N where eval-power: clength * scale = 2 ^ N
  using eval-domain-length-power by blast
  have roots-le: length trace-roots ≤ N
proof -
  have len: length trace-roots = ceil-log clength
  using accepted-fri-opening-transcript-shapes(1)[OF fri-openings] .
  have clength ≤ clength * scale
  using scale-pos by simp
  also have ... = 2 ^ N
  by (rule eval-power)
  finally have ceil-log clength ≤ N
  by (rule ceil-log-le-power)
  then show ?thesis
  using len by simp
qed
have query-bound: query-idxs ! round-idx < clength * scale
  by (rule accepted-fri-opening-transcript-query-idx-bound
      [OF fri-openings])
  (use round-bound in simp)
have idx-bound:
  fri-layer-indices (length trace-roots) (query-idxs ! round-idx)
  (clength * scale) ! layer-idx <
  fri-layer-lengths (length trace-roots) (clength * scale) ! layer-idx
  by (rule fri-layer-indices-nth-bound
      [OF query-bound layer-bound eval-power roots-le])
  then show ?thesis
  unfolding fri-evidence-layer-idx-def fri-evidence-layer-len-def by simp
qed

lemma trace-fri-header-tied-next-value-replay-gap-imp-merkle-or-slot-on-support:
  assumes support: out ∈ set-dist (execute verify-monad s)
  and gap: trace-fri-header-tied-next-value-replay-gap s out

```

shows

partial-merkle-inconsistency-bad s out $\vee$
trace-fri-header-tied-authenticated-slot-recorded-chunk-gap s out

proof —

from *gap* **have** *sampled*:

trace-fri-header-tied-sampled-next-value-conflict s out

unfolding *trace-fri-header-tied-next-value-replay-gap-def* **by** *simp*

from *sampled* **obtain** *trace-roots trace-bs trace-final fri-dg*

composition-roots composition-bs composition-final fri-query-idxs

trace-round-layers composition-round-layers fr as rest trace-openings

trace-table

where *fri-openings*:

accepted-fri-opening-transcript s out trace-roots trace-bs trace-final

fri-dg composition-roots composition-bs composition-final

fri-query-idxs trace-round-layers composition-round-layers

and *header*:

verifier-header-transcript s fr trace-roots trace-final as fri-dg

composition-roots composition-final rest

and *partial*:

accepted-with-partial-trace-openings s out fr fri-query-idxs

trace-openings

and *cand*: *partial-trace-table-candidate trace-table trace-openings*

and *not-low*: $\neg$ *trace-table-low-degree trace-table*

and *generic*:

generic-fri-sampled-next-value-conflict trace-roots trace-bs

fri-query-idxs trace-round-layers

unfolding *trace-fri-header-tied-sampled-next-value-conflict-def* **by** *blast*

from *generic-fri-sampled-next-value-conflictE[OF generic]*

obtain *round-idx round-idx' layer-idx v v'*

where *round-bound*: *round-idx* < *length fri-query-idxs*

and *round-bound'*: *round-idx'* < *length fri-query-idxs*

and *layer-bound*: *layer-idx* < *length trace-bs*

and *same-idx*:

fri-evidence-next-idx trace-roots fri-query-idxs round-idx

layer-idx =

fri-evidence-next-idx trace-roots fri-query-idxs round-idx'

layer-idx

and *forced*:

generic-fri-round-forced-next-value trace-roots trace-bs

fri-query-idxs trace-round-layers round-idx layer-idx v

and *forced'*:

generic-fri-round-forced-next-value trace-roots trace-bs

fri-query-idxs trace-round-layers round-idx' layer-idx v'

and *neq*: *v* $\neq$ *v'*

by *blast*

from *fri-openings* **obtain** *result final-state* **where** *out-eq*:

out = *Some (result, final-state)*

unfolding *accepted-fri-opening-transcript-def* **by** *blast*

have *len-bs-roots*: *length trace-bs* = *length trace-roots*

```

    using accepted-fri-opening-transcript-shapes(2)[OF fri-openings] .
have layer-bound-roots: layer-idx < length trace-roots
    using layer-bound len-bs-roots by simp
have len-query: length fri-query-idxs = rounds
    using accepted-fri-opening-transcript-shapes(7)[OF fri-openings] .
have raw-bound:
    fri-evidence-layer-idx trace-roots fri-query-idxs round-idx layer-idx <
    fri-evidence-layer-len trace-roots layer-idx
    by (rule accepted-fri-opening-transcript-trace-evidence-layer-idx-bound
        [OF fri-openings round-bound layer-bound-roots])
have raw-bound':
    fri-evidence-layer-idx trace-roots fri-query-idxs round-idx' layer-idx <
    fri-evidence-layer-len trace-roots layer-idx
    by (rule accepted-fri-opening-transcript-trace-evidence-layer-idx-bound
        [OF fri-openings round-bound' layer-bound-roots])
have even-len: 2 dvd fri-evidence-layer-len trace-roots layer-idx
    by (rule accepted-fri-opening-transcript-trace-layer-arithmetic(1)
        [OF fri-openings layer-bound])
have round-product:
    fri-evidence-layer-len trace-roots layer-idx * 2 ^ layer-idx =
    clength * scale
    by (rule accepted-fri-opening-transcript-trace-layer-arithmetic(2)
        [OF fri-openings layer-bound])
have auth-step:
     $\bigwedge i j.$ 
     $i < \text{length } \text{fri-query-idxs} \implies$ 
     $j < \text{length } \text{trace-bs} \implies$ 
    generic-fri-transcript-step-with-authenticated-chunk trace-roots
    trace-bs fri-query-idxs trace-round-layers final-state i j
proof -
    fix i j
    assume i-bound:  $i < \text{length } \text{fri-query-idxs}$ 
    and j-bound:  $j < \text{length } \text{trace-bs}$ 
    have i-rounds:  $i < \text{rounds}$ 
    using i-bound len-query by simp
    have j-roots:  $j < \text{length } \text{trace-roots}$ 
    using j-bound len-bs-roots by simp
    show
    generic-fri-transcript-step-with-authenticated-chunk trace-roots
    trace-bs fri-query-idxs trace-round-layers final-state i j
    by (rule
        accepted-fri-opening-transcript-trace-step-with-authenticated-chunk-at-raw
        [OF fri-openings out-eq i-rounds j-roots])
qed
show ?thesis
proof (cases
    generic-fri-sampled-same-layer-opening-conflict trace-roots trace-bs
    fri-query-idxs trace-round-layers)
case True

```

```

have route:
  partial-merkle-inconsistency-bad s (Some (result, final-state)) ∨
  generic-fri-authenticated-slot-recorded-chunk-gap trace-roots
  fri-query-idxs trace-round-layers final-state
by (rule generic-fri-sampled-same-layer-conflict-imp-merkle-or-slot-gap
  [OF True len-bs-roots auth-step])
then show ?thesis
proof
  assume partial-merkle-inconsistency-bad s (Some (result, final-state))
  then show ?thesis
    using out-eq by simp
next
  assume slot-gap:
    generic-fri-authenticated-slot-recorded-chunk-gap trace-roots
    fri-query-idxs trace-round-layers final-state
  have route-slot-gap:
    trace-fri-header-tied-authenticated-slot-recorded-chunk-gap s out
  unfolding trace-fri-header-tied-authenticated-slot-recorded-chunk-gap-def
  by (intro exI conjI)
    (rule out-eq, rule fri-openings, rule header, rule partial, rule cand,
    rule not-low, rule slot-gap)
  then show ?thesis by simp
qed
next
  case False
  then have no-same:
    ¬ generic-fri-sampled-same-layer-opening-conflict trace-roots
    trace-bs fri-query-idxs trace-round-layers
  by simp
  have v = v'
  by (rule generic-fri-forced-next-value-eq-if-no-same-layer-conflict
  [OF no-same round-bound round-bound' layer-bound even-len
  raw-bound raw-bound' round-product same-idx forced forced'])
  then show ?thesis
    using neq by simp
qed
qed

lemma wp-trace-fri-header-tied-next-value-replay-gap-bound-from-merkle-and-slot:
  fixes M A :: prob
  assumes merkle-bound:
    wp-event verify-monad (partial-merkle-inconsistency-bad s) s ≤ M
  and slot-bound:
    wp-event verify-monad
    (trace-fri-header-tied-authenticated-slot-recorded-chunk-gap s) s ≤ A
  shows
    wp-event verify-monad
    (trace-fri-header-tied-next-value-replay-gap s) s ≤ M + A
proof –

```



```

let ?P = partial-merkle-inconsistency-bad s
let ?Q = trace-fri-header-tied-authenticated-slot-recorded-chunk-gap s
have event-le:
  wp-event verify-monad
    (trace-fri-header-tied-next-value-replay-gap s) s ≤
    wp-event verify-monad (λout. ?P out ∨ ?Q out) s
by (rule wp-event-mono-on-support)
  (rule
    trace-fri-header-tied-next-value-replay-gap-imp-merkle-or-slot-on-support)
also have ... ≤ wp-event verify-monad ?P s +
  wp-event verify-monad ?Q s
by (rule wp-event-union-bound)
also have ... ≤ M + A
by (intro add-mono merkle-bound slot-bound)
finally show ?thesis .
qed

lemma verifier-query-round-after-index-trace-successor-recorded-step-from-previous:
fixes s t :: ('f, 'a) protocol-channel-scheme
assumes outcome:
  Some ((), t) ∈
    set-dist
      (execute
        (verifier-query-round-after-index-program
          fr f-fl f-final as fl final raw) s)
and recorded:
  query-round-fri-layer-transcripts (index (to-nat raw))
    (map snd f-fl) (map snd fl) round-chunk trace-layers
    composition-layers
and transcript: PTranscript s = round-chunk @ PTranscript t
and j-bound: Suc j < length f-fl
and all-layer-bounds:
  ∧k. k < length f-fl ⇒
    0 < fri-layer-lengths (length f-fl) (clength * scale) ! k ∧
    fri-layer-indices (length f-fl) (index (to-nat raw))
      (clength * scale) ! k <
    fri-layer-lengths (length f-fl) (clength * scale) ! k
and previous-step:
  fri-layer-step-evidence (snd (f-fl ! j)) (fst (f-fl ! j))
    (fri-layer-lengths (length f-fl) (clength * scale) ! j)
    (fri-layer-indices (length f-fl) (index (to-nat raw))
      (clength * scale) ! j)
    (2 ^ j)
  (fri-sibling-index
    (fri-layer-lengths (length f-fl) (clength * scale) ! j)
    (fri-layer-indices (length f-fl) (index (to-nat raw))
      (clength * scale) ! j))
  xp xp-path xn xn-path
  (fri-layer-indices (length f-fl) (index (to-nat raw)))

```

```

      (clength * scale) ! j mod
      (fri-layer-lengths (length f-fl) (clength * scale) ! j div 2))
    v
    (trace-layers ! j)
  shows
    ∃ xp-path' xn' xn-path' next-value.
    fri-layer-step-evidence (snd (f-fl ! Suc j)) (fst (f-fl ! Suc j))
      (fri-layer-lengths (length f-fl) (clength * scale) ! Suc j)
      (fri-layer-indices (length f-fl) (index (to-nat raw))
        (clength * scale) ! Suc j)
      (2 ^ Suc j)
      (fri-sibling-index
        (fri-layer-lengths (length f-fl) (clength * scale) ! Suc j)
        (fri-layer-indices (length f-fl) (index (to-nat raw))
          (clength * scale) ! Suc j))
      v xp-path' xn' xn-path'
      (fri-layer-indices (length f-fl) (index (to-nat raw))
        (clength * scale) ! Suc j mod
        (fri-layer-lengths (length f-fl) (clength * scale) ! Suc j div 2))
      next-value
      (trace-layers ! Suc j)
  proof -
    let ?idx = index (to-nat raw)
    from outcome obtain fv s1 f-out s2 s3 c-out s4 where
      query-decommit:
        Some (fv, s1) ∈
          set-dist (execute (mmap (check-decommit-on-query fr ?idx)) s)
    and trace-fri:
        Some (f-out, s2) ∈
          set-dist (execute
            (mfold (?idx, hd fv, clength * scale, 1)
              (receive-query-commits f-fl)) s1)
    and assert-trace:
        Some ((), s3) ∈
          set-dist
            (execute
              (assert (case f-out of (-, f-x, -, -) ⇒ f-x = f-final)) s2)
    and comp-fri:
        Some (c-out, s4) ∈
          set-dist (execute
            (mfold
              (?idx, cp-eval as fv (h ^ ?idx * shift), clength * scale, 1)
              (receive-query-commits fl)) s3)
    and assert-comp:
        Some ((), t) ∈
          set-dist
            (execute
              (assert (case c-out of (-, x, -, -) ⇒ x = final)) s4)
    unfolding verifier-query-round-after-index-program-def

```

```

  by (auto simp: Let-def split: prod.splits elim!: set-dist-bindE)
from check-decommit-on-query-outcome[OF query-decommit]
obtain query-paths query-chunk where
  query-chunk-shape:
    query-decommitment-transcript ?idx fv query-paths query-chunk
  and tr-s: PTranscript s = query-chunk @ PTranscript s1
  by blast
from receive-query-commits-layers-outcome[OF trace-fri]
obtain trace-layer-chunks trace-fri-chunk where
  trace-layers-actual:
    fri-layers-transcript (length f-fl) (clength * scale)
    trace-layer-chunks trace-fri-chunk
  and tr-s1: PTranscript s1 = trace-fri-chunk @ PTranscript s2
  by blast
have s3-eq: s3 = s2
  using assert-trace unfolding assert-def
  by (cases case f-out of (-, f-x, -, -)  $\Rightarrow$  f-x = f-final)
    (auto simp: throw-no-outcome)
from receive-query-commits-layers-outcome[OF comp-fri]
obtain comp-layer-chunks comp-fri-chunk where
  comp-layers-actual:
    fri-layers-transcript (length fl) (clength * scale)
    comp-layer-chunks comp-fri-chunk
  and tr-s3: PTranscript s3 = comp-fri-chunk @ PTranscript s4
  by blast
have t-eq: t = s4
  using assert-comp unfolding assert-def
  by (cases case c-out of (-, x, -, -)  $\Rightarrow$  x = final)
    (auto simp: throw-no-outcome)
let ?actual-chunk = query-chunk @ trace-fri-chunk @ comp-fri-chunk
have actual-recorded:
  query-round-fri-layer-transcripts ?idx (map snd f-fl) (map snd fl)
  ?actual-chunk trace-layer-chunks comp-layer-chunks
  unfolding query-round-fri-layer-transcripts-def
  by (intro exI[of - query-chunk] exI[of - trace-fri-chunk]
    exI[of - comp-fri-chunk] exI[of - fv] exI[of - query-paths] conjI)
    (use query-chunk-shape trace-layers-actual comp-layers-actual in
      ⟨simp-all add: mult commute⟩)
have actual-transcript:
  PTranscript s = ?actual-chunk @ PTranscript t
  using tr-s tr-s1 tr-s3 s3-eq t-eq by simp
have actual-chunk-eq: ?actual-chunk = round-chunk
  using actual-transcript transcript by simp
have trace-layer-chunks-eq:
  trace-layer-chunks = trace-layers
  by (rule query-round-fri-layer-transcripts-unique(1)
    [OF actual-recorded[unfolded actual-chunk-eq] recorded])
have idx-bound: ?idx < clength * scale
  by (rule index-less-domain)

```

```

have map-eq:
  receive-query-commits f-fl = map fri-layer-opening-step f-fl
  unfolding receive-query-commits-def fri-layer-opening-step-def
  by (simp add: fun-eq-iff split: prod.splits)
have previous-step-actual:
  fri-layer-step-evidence (snd (f-fl ! j)) (fst (f-fl ! j))
    (fri-layer-lengths (length f-fl) (clength * scale) ! j)
    (fri-layer-indices (length f-fl) ?idx (clength * scale) ! j)
    (1 * 2 ^ j)
    (fri-sibling-index
      (fri-layer-lengths (length f-fl) (clength * scale) ! j)
      (fri-layer-indices (length f-fl) ?idx (clength * scale) ! j))
  xp xp-path xn xn-path
    (fri-layer-indices (length f-fl) ?idx (clength * scale) ! j mod
      (fri-layer-lengths (length f-fl) (clength * scale) ! j div 2))
  v
    (trace-layer-chunks ! j)
  using previous-step trace-layer-chunks-eq by simp
from mfold-fri-layer-opening-successor-recorded-step-from-previous
  [OF trace-fri[unfolded map-eq] trace-layers-actual tr-s1 idx-bound
    all-layer-bounds j-bound previous-step-actual]
obtain sxp-path sxn sxn-path snext where succ-actual:
  fri-layer-step-evidence (snd (f-fl ! Suc j)) (fst (f-fl ! Suc j))
    (fri-layer-lengths (length f-fl) (clength * scale) ! Suc j)
    (fri-layer-indices (length f-fl) ?idx (clength * scale) ! Suc j)
    (1 * 2 ^ Suc j)
    (fri-sibling-index
      (fri-layer-lengths (length f-fl) (clength * scale) ! Suc j)
      (fri-layer-indices (length f-fl) ?idx (clength * scale) ! Suc j))
  v sxp-path sxn sxn-path
    (fri-layer-indices (length f-fl) ?idx (clength * scale) ! Suc j mod
      (fri-layer-lengths (length f-fl) (clength * scale) ! Suc j div 2))
  snext
    (trace-layer-chunks ! Suc j)
  by blast
have succ:
  fri-layer-step-evidence (snd (f-fl ! Suc j)) (fst (f-fl ! Suc j))
    (fri-layer-lengths (length f-fl) (clength * scale) ! Suc j)
    (fri-layer-indices (length f-fl) ?idx (clength * scale) ! Suc j)
    (2 ^ Suc j)
    (fri-sibling-index
      (fri-layer-lengths (length f-fl) (clength * scale) ! Suc j)
      (fri-layer-indices (length f-fl) ?idx (clength * scale) ! Suc j))
  v sxp-path sxn sxn-path
    (fri-layer-indices (length f-fl) ?idx (clength * scale) ! Suc j mod
      (fri-layer-lengths (length f-fl) (clength * scale) ! Suc j div 2))
  snext
    (trace-layers ! Suc j)
  using succ-actual trace-layer-chunks-eq by simp

```

show ?thesis
 using succ by blast
 qed

lemma *accepted-fri-opening-transcript-trace-successor-recorded-step-from-previous-at:*

assumes *fri-openings:*

accepted-fri-opening-transcript s out trace-roots trace-bs trace-final
dg composition-roots composition-bs composition-final query-idxs
trace-round-layers composition-round-layers

and *out-eq:* *out = Some (result, final-state)*

and *round-bound:* *i < rounds*

and *layer-bound:* *Suc j < length trace-roots*

and *all-layer-bounds:*

$\bigwedge \text{raw } k. k < \text{length trace-roots} \implies$
 $0 < \text{fri-layer-lengths } (\text{length trace-roots})$
 $(\text{clength} * \text{scale}) ! k \wedge$
 $\text{fri-layer-indices } (\text{length trace-roots}) (\text{index } (\text{to-nat raw}))$
 $(\text{clength} * \text{scale}) ! k <$
 $\text{fri-layer-lengths } (\text{length trace-roots}) (\text{clength} * \text{scale}) ! k$

and *previous-step:*

fri-layer-step-evidence (trace-roots ! j)
(trace-bs ! j)
(fri-evidence-layer-len trace-roots j)
(fri-evidence-layer-idx trace-roots query-idxs i j)
 $(2 \wedge j)$
(fri-sibling-index (fri-evidence-layer-len trace-roots j)
(fri-evidence-layer-idx trace-roots query-idxs i j))
xp xp-path xn xn-path
(fri-evidence-next-idx trace-roots query-idxs i j)
v
(trace-round-layers ! i ! j)

shows

$\exists \text{xp-path}' \text{xn}' \text{xn-path}' \text{next-value}.$
fri-layer-step-evidence (trace-roots ! Suc j)
(trace-bs ! Suc j)
(fri-evidence-layer-len trace-roots (Suc j))
(fri-evidence-layer-idx trace-roots query-idxs i (Suc j))
 $(2 \wedge \text{Suc } j)$
(fri-sibling-index
(fri-evidence-layer-len trace-roots (Suc j))
(fri-evidence-layer-idx trace-roots query-idxs i
(Suc j)))
v xp-path' xn' xn-path'
(fri-evidence-next-idx trace-roots query-idxs i (Suc j))
next-value
(trace-round-layers ! i ! Suc j)

proof —

from *fri-openings* **obtain** *result' final-state' fr f-fl as fl query-state*
raw-idxs query-chunks **where**

```

out-def: out = Some (result', final-state')
and trace-roots-eq: trace-roots = map snd f-fl
and trace-bs-eq: trace-bs = map fst f-fl
and composition-roots-eq: composition-roots = map snd fl
and outcome:
  Some (result', final-state') ∈
    set-dist (execute
      (ntimes
        (verifier-query-round-program fr f-fl trace-final as fl
          composition-final)
        rounds)
      query-state)
and len-raw: length raw-idxs = rounds
and query-idxs-eq:
  query-idxs = map (λraw. index (to-nat raw)) raw-idxs
and len-query-chunks: length query-chunks = rounds
and transcript-query:
  PTranscript query-state =
    List.concat query-chunks @ PTranscript final-state'
and query-chunk:
  ∧i. i < rounds ⇒
    verifier-query-round-chunk (query-idxs ! i)
    trace-roots composition-roots (query-chunks ! i)
and replay-lookup:
  ∧i. i < rounds ⇒
    fmlookup (HashMap final-state')
    (QueryIndexChallenge (PQueryCounter query-state + i)
      (state-after-query-chunks (PState query-state) query-chunks i)) =
    Some (raw-idxs ! i)
and recorded:
  ∧i. i < rounds ⇒
    query-round-fri-layer-transcripts (query-idxs ! i)
    trace-roots composition-roots (query-chunks ! i)
    (trace-round-layers ! i) (composition-round-layers ! i)
unfolding accepted-fri-opening-transcript-def by blast
have result-eq: result' = result
  using out-def out-eq by simp
have final-state-eq: final-state' = final-state
  using out-def out-eq by simp
have outcome':
  Some (result, final-state) ∈
    set-dist (execute
      (ntimes
        (verifier-query-round-program fr f-fl trace-final as fl
          composition-final)
        rounds)
      query-state)
  using outcome result-eq final-state-eq by simp
let ?round =

```

```

    verifier-query-round-program fr f-fl trace-final as fl composition-final
from query-opening-ntimes-outcome-decomp-at[OF outcome' round-bound]
obtain prefix x suffix s-i s-suc where
  prefix:
    Some (prefix, s-i) ∈
      set-dist (execute (ntimes ?round i) query-state)
and round:
    Some (x, s-suc) ∈ set-dist (execute ?round s-i)
and suffix:
    Some (suffix, final-state) ∈
      set-dist (execute (ntimes ?round (rounds - Suc i)) s-suc)
by blast
have x-eq: x = ()
  by (cases x) simp
show ?thesis
proof (rule query-opening-ntimes-prefix-transcript-state-alignment
  [OF prefix])
  fix prefix-query-idxs prefix-chunks
  assume len-prefix-idxs: length prefix-query-idxs = i
  and len-prefix-chunks: length prefix-chunks = i
  and transcript-prefix:
    PTranscript query-state =
      List.concat prefix-chunks @ PTranscript s-i
  and state-i:
    PState s-i =
      state-after-query-chunks (PState query-state) prefix-chunks i
  and counter-i: PQueryCounter s-i = PQueryCounter query-state + i
  and prefix-ext: query-state ≤ s-i
  and prefix-rounds:
     $\bigwedge k. k < i \implies$ 
      verifier-query-round-chunk (prefix-query-idxs ! k)
      (map snd f-fl) (map snd fl) (prefix-chunks ! k)
  from round[unfolded x-eq verifier-query-round-program-alt-def]
  obtain raw s0 where
    raw-out:
      Some (raw, s0) ∈
        set-dist (execute receive-query-index-challenge s-i)
  and after:
    Some ((), s-suc) ∈
      set-dist (execute
        (verifier-query-round-after-index-program fr f-fl trace-final as
          fl composition-final raw) s0)
  by (auto elim!: set-dist-bindE)
  have tr-s0: PTranscript s0 = PTranscript s-i
  using receive-query-index-challenge-outcome[OF raw-out] by simp
  show ?thesis
  proof (rule verifier-query-round-program-outcome[OF round[unfolded x-eq]])
    fix raw' idx chunk
    assume idx-eq: idx = index (to-nat raw')

```

```

and chunk-shape:
  verifier-query-round-chunk idx (map snd f-fl) (map snd fl)
    chunk
and transcript-round:
  PTranscript s-i = chunk @ PTranscript s-suc
and lookup-suc:
  fmlookup (HashMap s-suc)
    (QueryIndexChallenge (PQueryCounter s-i) (PState s-i)) =
    Some raw'
have raw-eq': raw = raw'
proof –
  have lookup-s0:
    fmlookup (HashMap s0)
      (QueryIndexChallenge (PQueryCounter s-i) (PState s-i)) =
      Some raw
    using receive-query-index-challenge-outcome[OF raw-out] by simp
have s0-suc: s0 ≤ s-suc
  using verifier-query-round-after-index-program-outcome[OF after]
  by simp
have lookup-suc-raw:
  fmlookup (HashMap s-suc)
    (QueryIndexChallenge (PQueryCounter s-i) (PState s-i)) =
    Some raw
  using hash-extension-lookup[OF lookup-s0 s0-suc] .
show ?thesis
  using lookup-suc lookup-suc-raw by simp
qed
show ?thesis
proof (rule query-opening-ntimes-suffix-transcript-state-alignment
  [OF suffix])
fix suffix-query-idxs suffix-chunks
assume len-suffix-idxs:
  length suffix-query-idxs = rounds – Suc i
and len-suffix-chunks:
  length suffix-chunks = rounds – Suc i
and transcript-suffix:
  PTranscript s-suc =
    List.concat suffix-chunks @ PTranscript final-state
and suffix-ext: s-suc ≤ final-state
and suffix-rounds:
   $\bigwedge k. k < \text{rounds} - \text{Suc } i \implies$ 
    verifier-query-round-chunk (suffix-query-idxs ! k)
      (map snd f-fl) (map snd fl) (suffix-chunks ! k)
let ?selected-chunks = prefix-chunks @ chunk # suffix-chunks
have len-selected-chunks: length ?selected-chunks = rounds
  using len-prefix-chunks len-suffix-chunks round-bound by simp
have selected-chunk-lens:
   $\bigwedge k. k < \text{rounds} \implies$ 
    length (?selected-chunks ! k) =

```



```

    verifier-query-round-transcript-length (query-idxs ! k)
      (map snd f-fl) (map snd fl)
  proof -
    fix k
    assume k-bound: k < rounds
    show
      length (?selected-chunks ! k) =
        verifier-query-round-transcript-length (query-idxs ! k)
          (map snd f-fl) (map snd fl)
  proof (cases k < i)
    case True
    have nth-eq: ?selected-chunks ! k = prefix-chunks ! k
      using True len-prefix-chunks by (simp add: nth-append)
    have
      length (prefix-chunks ! k) =
        verifier-query-round-transcript-length
          (prefix-query-idxs ! k) (map snd f-fl) (map snd fl)
    by (rule verifier-query-round-chunk-length
        [OF prefix-rounds[OF True]])
    then show ?thesis
      using nth-eq
      by (simp add:
          verifier-query-round-transcript-length-index-irrelevant)
  next
    case False
    show ?thesis
  proof (cases k = i)
    case True
    have nth-eq: ?selected-chunks ! k = chunk
      using True len-prefix-chunks by (simp add: nth-append)
    have
      length chunk =
        verifier-query-round-transcript-length idx
          (map snd f-fl) (map snd fl)
    by (rule verifier-query-round-chunk-length[OF chunk-shape])
    then show ?thesis
      using nth-eq
      by (simp add:
          verifier-query-round-transcript-length-index-irrelevant)
  next
    case False
    let ?k = k - Suc i
    have ge: Suc i ≤ k
      using ⟨¬ k < i⟩ False by linarith
    have k-bound': ?k < rounds - Suc i
      using ge k-bound by linarith
    have nth-eq: ?selected-chunks ! k = suffix-chunks ! ?k
      using ge len-prefix-chunks by (simp add: nth-append)
    have

```

```

length (suffix-chunks ! ?k) =
  verifier-query-round-transcript-length
    (suffix-query-idxs ! ?k) (map snd f-fl) (map snd fl)
by (rule verifier-query-round-chunk-length
  [OF suffix-rounds[OF k-bound']])
then show ?thesis
  using nth-eq
  by (simp add:
    verifier-query-round-transcript-length-index-irrelevant)
qed
qed
qed
have query-chunk-f:
   $\bigwedge k. k < \text{rounds} \implies$ 
    verifier-query-round-chunk (query-idxs ! k)
      (map snd f-fl) (map snd fl) (query-chunks ! k)
  using query-chunk trace-roots-eq composition-roots-eq by simp
have query-chunk-lens:
   $\bigwedge k. k < \text{rounds} \implies$ 
    length (query-chunks ! k) =
      verifier-query-round-transcript-length (query-idxs ! k)
        (map snd f-fl) (map snd fl)
  using query-chunk-f verifier-query-round-chunk-length by blast
have selected-transcript:
  PTranscript query-state =
    List.concat ?selected-chunks @ PTranscript final-state
  using transcript-prefix transcript-round transcript-suffix by simp
have concat-eq: List.concat ?selected-chunks = List.concat query-chunks
  using selected-transcript transcript-query final-state-eq by simp
have chunks-eq: ?selected-chunks = query-chunks
proof -
  have same-lens:
     $\bigwedge k. k < \text{length } ?\text{selected-chunks} \implies$ 
      length (?selected-chunks ! k) = length (query-chunks ! k)
    using selected-chunk-lens query-chunk-lens len-selected-chunks
    by simp
  have concat-eq-empty:
    List.concat ?selected-chunks @ [] = List.concat query-chunks
    using concat-eq by simp
  have len-eq: length ?selected-chunks = length query-chunks
    using len-selected-chunks len-query-chunks by simp
  have ?selected-chunks = query-chunks  $\wedge$  ([] :: 'f list) = []
    by (rule concat-append-eq-concat-same-chunk-lengths
      [OF len-eq same-lens concat-eq-empty])
  then show ?thesis
    by simp
qed
have prefix-state-eq:
  state-after-query-chunks (PState query-state) prefix-chunks i =

```

```

    state-after-query-chunks (PState query-state) query-chunks i
proof —
  have
    state-after-query-chunks (PState query-state) ?selected-chunks i =
      state-after-query-chunks (PState query-state) prefix-chunks i
  by (rule state-after-query-chunks-append-prefix
    [OF len-prefix-chunks])
  then show ?thesis
    using chunks-eq by simp
qed
have lookup-final:
  fmlookup (HashMap final-state)
    (QueryIndexChallenge (PQueryCounter query-state + i)
      (state-after-query-chunks (PState query-state) query-chunks i)) =
    Some raw'
proof —
  have lookup-suc':
    fmlookup (HashMap s-suc)
      (QueryIndexChallenge (PQueryCounter query-state + i)
        (state-after-query-chunks (PState query-state)
          prefix-chunks i)) =
      Some raw'
    using lookup-suc state-i counter-i by simp
  have lookup-final-prefix:
    fmlookup (HashMap final-state)
      (QueryIndexChallenge (PQueryCounter query-state + i)
        (state-after-query-chunks (PState query-state)
          prefix-chunks i)) =
      Some raw'
    using hash-extension-lookup[OF lookup-suc' suffix-ext] .
  show ?thesis
    using lookup-final-prefix prefix-state-eq by simp
qed
have raw-idx-eq: raw' = raw-idxs ! i
  using lookup-final replay-lookup[OF round-bound] final-state-eq
  by simp
have idx-eq-query: idx = query-idxs ! i
  using idx-eq raw-idx-eq query-idxs-eq len-raw round-bound by simp
have selected-i: ?selected-chunks ! i = chunk
  using len-prefix-chunks by (simp add: nth-append)
have chunk-i-eq: chunk = query-chunks ! i
  using chunks-eq selected-i by simp
have recorded-i:
  query-round-fri-layer-transcripts (index (to-nat raw))
    (map snd f-fl) (map snd fl) chunk (trace-round-layers ! i)
    (composition-round-layers ! i)
  using recorded[OF round-bound] idx-eq idx-eq-query raw-eq'
    chunk-i-eq trace-roots-eq composition-roots-eq
  by simp

```

```

have tr-s0-suc:
  PTranscript s0 = chunk @ PTranscript s-suc
  using tr-s0 transcript-round by simp
have suc-j-bound-fl: Suc j < length f-fl
  using layer-bound trace-roots-eq by simp
have all-layer-bounds-fl:
   $\bigwedge k. k < \text{length } f\text{-fl} \implies$ 
     $0 < \text{fri-layer-lengths } (\text{length } f\text{-fl}) (\text{clength} * \text{scale}) ! k \wedge$ 
     $\text{fri-layer-indices } (\text{length } f\text{-fl}) (\text{index } (\text{to-nat } \text{raw}))$ 
     $(\text{clength} * \text{scale}) ! k <$ 
     $\text{fri-layer-lengths } (\text{length } f\text{-fl}) (\text{clength} * \text{scale}) ! k$ 
  using all-layer-bounds trace-roots-eq by simp
have raw-idx-query: index (to-nat raw) = query-idxs ! i
  using idx-eq idx-eq-query raw-eq' by simp
have previous-step-fl:
  fri-layer-step-evidence (snd (f-fl ! j)) (fst (f-fl ! j))
  (fri-layer-lengths (length f-fl) (clength * scale) ! j)
  (fri-layer-indices (length f-fl) (index (to-nat raw))
    (clength * scale) ! j)
   $(2 \wedge j)$ 
  (fri-sibling-index
    (fri-layer-lengths (length f-fl) (clength * scale) ! j)
    (fri-layer-indices (length f-fl) (index (to-nat raw))
      (clength * scale) ! j))
  xp xp-path xn xn-path
  (fri-layer-indices (length f-fl) (index (to-nat raw))
    (clength * scale) ! j mod
    (fri-layer-lengths (length f-fl) (clength * scale) ! j div 2))
  v
  (trace-round-layers ! i ! j)
using previous-step trace-roots-eq trace-bs-eq raw-idx-query
  layer-bound
unfolding fri-evidence-layer-len-def fri-evidence-layer-idx-def
  fri-evidence-next-idx-def
by simp
from verifier-query-round-after-index-trace-successor-recorded-step-from-previous
  [OF after recorded-i tr-s0-suc suc-j-bound-fl
    all-layer-bounds-fl previous-step-fl]
obtain xp-path sxn sxn-path snext where succ-fl:
  fri-layer-step-evidence (snd (f-fl ! Suc j))
  (fst (f-fl ! Suc j))
  (fri-layer-lengths (length f-fl) (clength * scale) ! Suc j)
  (fri-layer-indices (length f-fl) (index (to-nat raw))
    (clength * scale) ! Suc j)
   $(2 \wedge \text{Suc } j)$ 
  (fri-sibling-index
    (fri-layer-lengths (length f-fl) (clength * scale) ! Suc j)
    (fri-layer-indices (length f-fl) (index (to-nat raw))
      (clength * scale) ! Suc j))

```

```

    v sxp-path sxn sxn-path
    (fri-layer-indices (length f-fl) (index (to-nat raw))
      (clength * scale) ! Suc j mod
      (fri-layer-lengths (length f-fl) (clength * scale) ! Suc j
        div 2))
    snext
    (trace-round-layers ! i ! Suc j)
  by blast
have succ:
  fri-layer-step-evidence (trace-roots ! Suc j)
  (trace-bs ! Suc j)
  (fri-evidence-layer-len trace-roots (Suc j))
  (fri-evidence-layer-idx trace-roots query-idxs i (Suc j))
  (2 ^ Suc j)
  (fri-sibling-index
    (fri-evidence-layer-len trace-roots (Suc j))
    (fri-evidence-layer-idx trace-roots query-idxs i
      (Suc j)))
  v sxp-path sxn sxn-path
  (fri-evidence-next-idx trace-roots query-idxs i (Suc j))
  snext
  (trace-round-layers ! i ! Suc j)
using succ-fl trace-roots-eq trace-bs-eq raw-idx-query layer-bound
unfolding fri-evidence-layer-len-def fri-evidence-layer-idx-def
  fri-evidence-next-idx-def
by simp
show ?thesis
using succ by blast
qed
qed
qed
qed

lemma trace-fri-header-tied-successor-replay-gap-imp-merkle-or-slot-on-support:
  assumes support: out ∈ set-dist (execute verify-monad s)
  and gap: trace-fri-header-tied-successor-replay-gap s out
  shows
    partial-merkle-inconsistency-bad s out ∨
    trace-fri-header-tied-authenticated-slot-recorded-chunk-gap s out
proof -
  from gap have sampled:
    trace-fri-header-tied-sampled-successor-opening-conflict s out
  unfolding trace-fri-header-tied-successor-replay-gap-def by simp
  from sampled obtain trace-roots trace-bs trace-final fri-dg
    composition-roots composition-bs composition-final fri-query-idxs
    trace-round-layers composition-round-layers fr as rest trace-openings
    trace-table
  where fri-openings:
    accepted-fri-opening-transcript s out trace-roots trace-bs

```

```

    trace-final fri-dg composition-roots composition-bs composition-final
    fri-query-idxs trace-round-layers composition-round-layers
and header:
    verifier-header-transcript s fr trace-roots trace-final as fri-dg
    composition-roots composition-final rest
and partial:
    accepted-with-partial-trace-openings s out fr fri-query-idxs
    trace-openings
and cand: partial-trace-table-candidate trace-table trace-openings
and not-low:  $\neg$  trace-table-low-degree trace-table
and generic:
    generic-fri-sampled-successor-opening-conflict trace-roots trace-bs
    fri-query-idxs trace-round-layers
unfolding trace-fri-header-tied-sampled-successor-opening-conflict-def
by blast
from fri-openings obtain result final-state where out-eq:
    out = Some (result, final-state)
    unfolding accepted-fri-opening-transcript-def by blast
have len-bs-roots: length trace-bs = length trace-roots
    using accepted-fri-opening-transcript-shapes(2)[OF fri-openings] .
have len-query: length fri-query-idxs = rounds
    using accepted-fri-opening-transcript-shapes(7)[OF fri-openings] .
have all-layer-bounds:
 $\bigwedge$  raw k. k < length trace-roots  $\implies$ 
    0 < fri-layer-lengths (length trace-roots)
    (clength * scale) ! k  $\wedge$ 
    fri-layer-indices (length trace-roots) (index (to-nat raw))
    (clength * scale) ! k <
    fri-layer-lengths (length trace-roots) (clength * scale) ! k
by (rule accepted-fri-opening-transcript-trace-raw-layer-bound
    [OF fri-openings])
have source-step:
 $\bigwedge$  source-round layer-idx v.
    source-round < length fri-query-idxs  $\implies$ 
    Suc layer-idx < length trace-bs  $\implies$ 
    generic-fri-round-forced-next-value trace-roots trace-bs
    fri-query-idxs trace-round-layers source-round layer-idx v  $\implies$ 
 $\exists$  xp-path xn xn-path.
    fri-layer-step-evidence
    (trace-roots ! Suc layer-idx)
    (trace-bs ! Suc layer-idx)
    (fri-evidence-layer-len trace-roots (Suc layer-idx))
    (fri-evidence-layer-idx trace-roots fri-query-idxs
    source-round (Suc layer-idx))
    (2  $\wedge$  Suc layer-idx)
    (fri-sibling-index
    (fri-evidence-layer-len trace-roots (Suc layer-idx))
    (fri-evidence-layer-idx trace-roots fri-query-idxs
    source-round (Suc layer-idx)))

```

```

    v xp-path xn xn-path
    (fri-evidence-next-idx trace-roots fri-query-idxs
     source-round (Suc layer-idx))
    (fri-evidence-next-value trace-roots trace-bs
     fri-query-idxs source-round (Suc layer-idx) v xn)
    (trace-round-layers ! source-round ! Suc layer-idx)
  proof -
    fix source-round layer-idx v
    assume source-round-bound: source-round < length fri-query-idxs
    and layer-bound': Suc layer-idx < length trace-bs
    and forced:
      generic-fri-round-forced-next-value trace-roots trace-bs
      fri-query-idxs trace-round-layers source-round layer-idx v
    have source-round-rounds: source-round < rounds
    using source-round-bound len-query by simp
    have layer-bound-roots: Suc layer-idx < length trace-roots
    using layer-bound' len-bs-roots by simp
    from forced obtain fxp fxp-path fxn fxn-path where previous-step:
      fri-layer-step-evidence (trace-roots ! layer-idx)
      (trace-bs ! layer-idx)
      (fri-evidence-layer-len trace-roots layer-idx)
      (fri-evidence-layer-idx trace-roots fri-query-idxs
       source-round layer-idx)
      (2 ^ layer-idx)
      (fri-sibling-index
       (fri-evidence-layer-len trace-roots layer-idx)
       (fri-evidence-layer-idx trace-roots fri-query-idxs
        source-round layer-idx))
      fxp fxp-path fxn fxn-path
      (fri-evidence-next-idx trace-roots fri-query-idxs
       source-round layer-idx)
      v
      (trace-round-layers ! source-round ! layer-idx)
    unfolding generic-fri-round-forced-next-value-def by blast
    from accepted-fri-opening-transcript-trace-successor-recorded-step-from-previous-at
      [OF fri-openings out-eq source-round-rounds layer-bound-roots
       all-layer-bounds previous-step]
    obtain sxp-path sxn sxn-path snext where succ:
      fri-layer-step-evidence (trace-roots ! Suc layer-idx)
      (trace-bs ! Suc layer-idx)
      (fri-evidence-layer-len trace-roots (Suc layer-idx))
      (fri-evidence-layer-idx trace-roots fri-query-idxs
       source-round (Suc layer-idx))
      (2 ^ Suc layer-idx)
      (fri-sibling-index
       (fri-evidence-layer-len trace-roots (Suc layer-idx))
       (fri-evidence-layer-idx trace-roots fri-query-idxs
        source-round (Suc layer-idx)))
      v sxp-path sxn sxn-path

```

```

    (fri-evidence-next-idx trace-roots fri-query-idxs
     source-round (Suc layer-idx))
  snext
  (trace-round-layers ! source-round ! Suc layer-idx)
by blast
have snext-eq:
  snext =
    fri-evidence-next-value trace-roots trace-bs
    fri-query-idxs source-round (Suc layer-idx) v sn
using fri-layer-step-evidenceD(3)[OF succ]
unfolding fri-evidence-next-value-def by simp
show
  ∃ xp-path xn xn-path.
    fri-layer-step-evidence
      (trace-roots ! Suc layer-idx)
      (trace-bs ! Suc layer-idx)
      (fri-evidence-layer-len trace-roots (Suc layer-idx))
      (fri-evidence-layer-idx trace-roots fri-query-idxs
       source-round (Suc layer-idx))
      (2 ^ Suc layer-idx)
      (fri-sibling-index
       (fri-evidence-layer-len trace-roots (Suc layer-idx))
       (fri-evidence-layer-idx trace-roots fri-query-idxs
        source-round (Suc layer-idx)))
      v xp-path xn xn-path
      (fri-evidence-next-idx trace-roots fri-query-idxs
       source-round (Suc layer-idx))
      (fri-evidence-next-value trace-roots trace-bs
       fri-query-idxs source-round (Suc layer-idx) v xn)
      (trace-round-layers ! source-round ! Suc layer-idx)
    using succ snext-eq by blast
qed
have same-layer:
  generic-fri-sampled-same-layer-opening-conflict trace-roots trace-bs
  fri-query-idxs trace-round-layers
by (rule generic-fri-sampled-successor-conflict-imp-same-layer-with-source-steps
   [OF generic len-bs-roots source-step])
have auth-step:
  ∧ i j.
    i < length fri-query-idxs ⇒
    j < length trace-bs ⇒
    generic-fri-transcript-step-with-authenticated-chunk trace-roots
    trace-bs fri-query-idxs trace-round-layers final-state i j
proof -
  fix i j
  assume i-bound: i < length fri-query-idxs
  and j-bound: j < length trace-bs
  have i-rounds: i < rounds
  using i-bound len-query by simp

```



```

have j-roots:  $j < \text{length } \text{trace-roots}$ 
  using j-bound len-bs-roots by simp
show
  generic-fri-transcript-step-with-authenticated-chunk trace-roots
    trace-bs fri-query-idxs trace-round-layers final-state i j
  by (rule
    accepted-fri-opening-transcript-trace-step-with-authenticated-chunk-at-raw
    [OF fri-openings out-eq i-rounds j-roots])
qed
have route:
  partial-merkle-inconsistency-bad s (Some (result, final-state))  $\vee$ 
    generic-fri-authenticated-slot-recorded-chunk-gap trace-roots
      fri-query-idxs trace-round-layers final-state
  by (rule generic-fri-sampled-same-layer-conflict-imp-merkle-or-slot-gap
    [OF same-layer len-bs-roots auth-step])
then show ?thesis
proof
  assume partial-merkle-inconsistency-bad s (Some (result, final-state))
  then show ?thesis
    using out-eq by simp
next
  assume slot-gap:
    generic-fri-authenticated-slot-recorded-chunk-gap trace-roots
      fri-query-idxs trace-round-layers final-state
  have route-slot-gap:
    trace-fri-header-tied-authenticated-slot-recorded-chunk-gap s out
  unfolding trace-fri-header-tied-authenticated-slot-recorded-chunk-gap-def
  by (intro exI conjI)
    (rule out-eq, rule fri-openings, rule header, rule partial, rule cand,
    rule not-low, rule slot-gap)
  then show ?thesis by simp
qed
qed

lemma wp-trace-fri-header-tied-successor-replay-gap-bound-from-merkle-and-slot:
  fixes M A :: prob
  assumes merkle-bound:
    wp-event verify-monad (partial-merkle-inconsistency-bad s)  $s \leq M$ 
  and slot-bound:
    wp-event verify-monad
      (trace-fri-header-tied-authenticated-slot-recorded-chunk-gap s)  $s \leq A$ 
  shows
    wp-event verify-monad
      (trace-fri-header-tied-successor-replay-gap s)  $s \leq M + A$ 
proof –
  let ?P = partial-merkle-inconsistency-bad s
  let ?Q = trace-fri-header-tied-authenticated-slot-recorded-chunk-gap s
  have event-le:
    wp-event verify-monad

```

```

    (trace-fri-header-tied-successor-replay-gap s) s ≤
    wp-event verify-monad (λout. ?P out ∨ ?Q out) s
  by (rule wp-event-mono-on-support)
    (rule
      trace-fri-header-tied-successor-replay-gap-imp-merkle-or-slot-on-support)
  also have ... ≤ wp-event verify-monad ?P s +
    wp-event verify-monad ?Q s
  by (rule wp-event-union-bound)
  also have ... ≤ M + A
  by (intro add-mono merkle-bound slot-bound)
  finally show ?thesis .
qed

end

end

```

theory *Soundness-FRI-Trace-Sibling-Evidence*
imports *Soundness-FRI-Value-Aligned-Events*
begin

Proof-only refinement of the trace FRI sibling mismatch branch.

The existing trace candidate predicate records agreement with the trace-query openings only. The first trace FRI sibling value is read from the FRI layer transcript, so the sibling branch cannot be closed from the current candidate predicate alone. This theory isolates the exact missing evidence: a trace candidate must also agree with the recorded first-layer FRI sibling values.

context *soundness*
begin

definition *trace-table-agrees-with-recorded-first-fri-siblings*
 $:: 'f \text{ list} \Rightarrow 'f \text{ list} \Rightarrow 'f \text{ list} \Rightarrow \text{nat list} \Rightarrow$
 $'f \text{ list list list} \Rightarrow \text{bool}$

where

```

trace-table-agrees-with-recorded-first-fri-siblings trace-table
  trace-roots trace-bs query-idxs trace-round-layers  $\longleftrightarrow$ 
  (∀ round-idx xp xp-path xn xn-path.
    round-idx < length query-idxs  $\longrightarrow$ 
    0 < length trace-roots  $\longrightarrow$ 
    0 < length trace-bs  $\longrightarrow$ 
    fri-layer-step-evidence
      (trace-roots ! 0)
      (trace-bs ! 0)
      (fri-evidence-layer-len trace-roots 0)
      (fri-evidence-layer-idx trace-roots query-idxs round-idx 0)
      1
      (fri-sibling-index (fri-evidence-layer-len trace-roots 0)

```

$(\text{fri-evidence-layer-idx } \text{trace-roots } \text{query-idxs } \text{round-idx } 0))$
 $xp \text{ } xp\text{-path } xn \text{ } xn\text{-path}$
 $(\text{fri-evidence-next-idx } \text{trace-roots } \text{query-idxs } \text{round-idx } 0)$
 $(\text{fri-evidence-next-value } \text{trace-roots } \text{trace-bs } \text{query-idxs}$
 $\text{round-idx } 0 \text{ } xp \text{ } xn)$
 $(\text{trace-round-layers } ! \text{ round-idx } ! 0) \longrightarrow$
 $\text{trace-table } !$
 $\text{fri-sibling-index } (\text{fri-evidence-layer-len } \text{trace-roots } 0)$
 $(\text{fri-evidence-layer-idx } \text{trace-roots } \text{query-idxs } \text{round-idx } 0) =$
 $xn)$

definition *partial-trace-table-candidate-with-recorded-first-fri-siblings*

$:: 'f \text{ list} \Rightarrow 'f \text{ authenticated-opening list list} \Rightarrow$
 $'f \text{ list} \Rightarrow 'f \text{ list} \Rightarrow \text{nat list} \Rightarrow 'f \text{ list list list} \Rightarrow$
 bool

where

$\text{partial-trace-table-candidate-with-recorded-first-fri-siblings}$
 $\text{trace-table } \text{trace-openings } \text{trace-roots } \text{trace-bs } \text{query-idxs}$
 $\text{trace-round-layers} \longleftrightarrow$
 $\text{partial-trace-table-candidate } \text{trace-table } \text{trace-openings} \wedge$
 $\text{trace-table-agrees-with-recorded-first-fri-siblings } \text{trace-table}$
 $\text{trace-roots } \text{trace-bs } \text{query-idxs } \text{trace-round-layers}$

lemma *partial-trace-table-candidate-with-recorded-first-fri-siblingsD:*

assumes

$\text{partial-trace-table-candidate-with-recorded-first-fri-siblings}$
 $\text{trace-table } \text{trace-openings } \text{trace-roots } \text{trace-bs } \text{query-idxs}$
 $\text{trace-round-layers}$

shows $\text{partial-trace-table-candidate } \text{trace-table } \text{trace-openings}$

and $\text{trace-table-agrees-with-recorded-first-fri-siblings } \text{trace-table}$
 $\text{trace-roots } \text{trace-bs } \text{query-idxs } \text{trace-round-layers}$

using *assms*

unfolding *partial-trace-table-candidate-with-recorded-first-fri-siblings-def*

by *blast+*

definition *trace-fri-header-tied-recorded-sibling-candidate-mismatch*

$:: ('f, 'a) \text{ protocol-channel-scheme} \Rightarrow$
 $(\text{unit list} \times ('f, 'a) \text{ protocol-channel-scheme}) \text{ option} \Rightarrow \text{bool}$

where

$\text{trace-fri-header-tied-recorded-sibling-candidate-mismatch } s \text{ out}$
 $\longleftrightarrow$
 $(\exists \text{ trace-roots } \text{trace-bs } \text{trace-final } dg \text{ composition-roots}$
 $\text{composition-bs } \text{composition-final } \text{fri-query-idxs } \text{trace-round-layers}$
 $\text{composition-round-layers } fr \text{ as rest } \text{trace-openings } \text{trace-table}$
 $\text{round-idx } xp \text{ } xp\text{-path } xn \text{ } xn\text{-path } \text{result } \text{final-state}.$
 $\text{out} = \text{Some } (\text{result}, \text{final-state}) \wedge$
 $\text{accepted-fri-opening-transcript } s \text{ out } \text{trace-roots } \text{trace-bs } \text{trace-final}$
 $dg \text{ composition-roots } \text{composition-bs } \text{composition-final } \text{fri-query-idxs}$
 $\text{trace-round-layers } \text{composition-round-layers} \wedge$

verifier-header-transcript s fr trace-roots trace-final as dg
composition-roots composition-final rest $\wedge$
accepted-with-partial-trace-openings s out fr fri-query-idxs
trace-openings $\wedge$
partial-trace-table-candidate trace-table trace-openings $\wedge$
trace-table-agrees-with-recorded-first-fri-siblings trace-table
trace-roots trace-bs fri-query-idxs trace-round-layers $\wedge$
 $\neg$ *trace-table-low-degree trace-table* $\wedge$
round-idx < length fri-query-idxs $\wedge$
0 < length trace-bs $\wedge$
trace-openings ! round-idx $\neq []$ $\wedge$
opening-index (hd (trace-openings ! round-idx)) =
fri-evidence-layer-idx trace-roots fri-query-idxs round-idx 0 $\wedge$
opening-value (hd (trace-openings ! round-idx)) = xp $\wedge$
fri-layer-step-evidence
(trace-roots ! 0)
(trace-bs ! 0)
(fri-evidence-layer-len trace-roots 0)
(fri-evidence-layer-idx trace-roots fri-query-idxs round-idx 0)
1
(fri-sibling-index (fri-evidence-layer-len trace-roots 0)
(fri-evidence-layer-idx trace-roots fri-query-idxs round-idx 0))
xp xp-path xn xn-path
(fri-evidence-next-idx trace-roots fri-query-idxs round-idx 0)
(fri-evidence-next-value trace-roots trace-bs fri-query-idxs
round-idx 0 xp xn)
(trace-round-layers ! round-idx ! 0) $\wedge$
trace-table !
fri-sibling-index (fri-evidence-layer-len trace-roots 0)
(fri-evidence-layer-idx trace-roots fri-query-idxs round-idx 0)
 $\neq xn$)

definition *trace-fri-header-tied-recorded-sibling-candidate-missing*

$:: (f, 'a) \text{ protocol-channel-scheme} \Rightarrow$
 $(\text{unit list} \times (f, 'a) \text{ protocol-channel-scheme}) \text{ option} \Rightarrow \text{bool}$

where

trace-fri-header-tied-recorded-sibling-candidate-missing s out
 $\iff$
 $(\exists \text{ trace-roots trace-bs trace-final dg composition-roots}$
 $\text{ composition-bs composition-final fri-query-idxs trace-round-layers}$
 $\text{ composition-round-layers fr as rest trace-openings trace-table}$
 $\text{ round-idx xp xp-path xn xn-path result final-state.}$
 $\text{ out} = \text{Some (result, final-state)}) \wedge$
 $\text{ accepted-fri-opening-transcript s out trace-roots trace-bs trace-final}$
 $\text{ dg composition-roots composition-bs composition-final fri-query-idxs}$
 $\text{ trace-round-layers composition-round-layers} \wedge$
 $\text{ verifier-header-transcript s fr trace-roots trace-final as dg}$
 $\text{ composition-roots composition-final rest} \wedge$
 $\text{ accepted-with-partial-trace-openings s out fr fri-query-idxs}$

trace-openings $\wedge$
partial-trace-table-candidate *trace-table* *trace-openings* $\wedge$
 $\neg$ *trace-table-agrees-with-recorded-first-fri-siblings* *trace-table*
trace-roots *trace-bs* *fri-query-idxs* *trace-round-layers* $\wedge$
 $\neg$ *trace-table-low-degree* *trace-table* $\wedge$
 $\text{round-idx} < \text{length } \text{fri-query-idxs} \wedge$
 $0 < \text{length } \text{trace-bs} \wedge$
trace-openings ! $\text{round-idx} \neq [] \wedge$
 $\text{opening-index } (\text{hd } (\text{trace-openings} ! \text{round-idx})) =$
 $\text{fri-evidence-layer-idx } \text{trace-roots } \text{fri-query-idxs } \text{round-idx } 0 \wedge$
 $\text{opening-value } (\text{hd } (\text{trace-openings} ! \text{round-idx})) = xp \wedge$
fri-layer-step-evidence
 $(\text{trace-roots} ! 0)$
 $(\text{trace-bs} ! 0)$
 $(\text{fri-evidence-layer-len } \text{trace-roots } 0)$
 $(\text{fri-evidence-layer-idx } \text{trace-roots } \text{fri-query-idxs } \text{round-idx } 0)$
 1
 $(\text{fri-sibling-index } (\text{fri-evidence-layer-len } \text{trace-roots } 0)$
 $(\text{fri-evidence-layer-idx } \text{trace-roots } \text{fri-query-idxs } \text{round-idx } 0))$
 $xp \text{ } xp\text{-path } xn \text{ } xn\text{-path}$
 $(\text{fri-evidence-next-idx } \text{trace-roots } \text{fri-query-idxs } \text{round-idx } 0)$
 $(\text{fri-evidence-next-value } \text{trace-roots } \text{trace-bs } \text{fri-query-idxs}$
 $\text{round-idx } 0 \text{ } xp \text{ } xn)$
 $(\text{trace-round-layers} ! \text{round-idx} ! 0) \wedge$
trace-table !
 $\text{fri-sibling-index } (\text{fri-evidence-layer-len } \text{trace-roots } 0)$
 $(\text{fri-evidence-layer-idx } \text{trace-roots } \text{fri-query-idxs } \text{round-idx } 0)$
 $\neq xn)$

lemma *generic-trace-sampled-layer-chain-evidence-recorded-first-sibling-agreement:*

assumes *chain:*

generic-fri-sampled-layer-chain-evidence *trace-table-low-degree*
trace-table $(\text{clength} - 1)$ *trace-roots* *trace-bs* *trace-final*
fri-query-idxs *trace-round-layers* *doms* *layers*

shows

trace-table-agrees-with-recorded-first-fri-siblings *trace-table*
trace-roots *trace-bs* *fri-query-idxs* *trace-round-layers*

proof (*unfold trace-table-agrees-with-recorded-first-fri-siblings-def,*
intro allI impI)

fix *round-idx* *xp* *xp-path* *xn* *xn-path*

assume *round-bound:* $\text{round-idx} < \text{length } \text{fri-query-idxs}$

and *roots-nonempty:* $0 < \text{length } \text{trace-roots}$

and *challenges-nonempty:* $0 < \text{length } \text{trace-bs}$

and *step:*

fri-layer-step-evidence
 $(\text{trace-roots} ! 0)$
 $(\text{trace-bs} ! 0)$
 $(\text{fri-evidence-layer-len } \text{trace-roots } 0)$
 $(\text{fri-evidence-layer-idx } \text{trace-roots } \text{fri-query-idxs } \text{round-idx } 0)$

```

1
(fri-sibling-index (fri-evidence-layer-len trace-roots 0)
 (fri-evidence-layer-idx trace-roots fri-query-idxs round-idx 0))
xp xp-path xn xn-path
(fri-evidence-next-idx trace-roots fri-query-idxs round-idx 0)
(fri-evidence-next-value trace-roots trace-bs fri-query-idxs
 round-idx 0 xp xn)
(trace-round-layers ! round-idx ! 0)
have layer0: layers ! 0 = trace-table
  by (rule generic-fri-sampled-layer-chain-evidenceD(5)[OF chain])
have sampled:
  fri-sampled-table-fold
    (trace-bs ! 0)
    (fri-evidence-layer-len trace-roots 0)
    (fri-evidence-layer-idx trace-roots fri-query-idxs round-idx 0)
    (2 ^ 0)
    (doms ! 0)
    (layers ! 0)
    (trace-round-layers ! round-idx ! 0)
    (layers ! Suc 0 !
      fri-evidence-next-idx trace-roots fri-query-idxs round-idx 0)
  by (rule generic-fri-sampled-layer-chain-evidence-sample(2)
    [OF chain round-bound challenges-nonempty])
from sampled obtain yp yp-path yn yn-path
  where chunk:
    fri-layer-opening-chunk (fri-evidence-layer-len trace-roots 0)
    yp yp-path yn yn-path (trace-round-layers ! round-idx ! 0)
  and match:
    fri-opening-matches-table (fri-evidence-layer-len trace-roots 0)
    (fri-evidence-layer-idx trace-roots fri-query-idxs round-idx 0)
    (layers ! 0) yp yn
  by (elim fri-sampled-table-foldE)
have step-chunk:
  fri-layer-opening-chunk (fri-evidence-layer-len trace-roots 0)
  xp xp-path xn xn-path (trace-round-layers ! round-idx ! 0)
  by (rule fri-layer-step-evidenceD(4)[OF step])
have yn-eq: yn = xn
  using fri-layer-opening-chunk-values-unique(2)[OF chunk step-chunk]
  by simp
have (layers ! 0) !
  fri-sibling-index (fri-evidence-layer-len trace-roots 0)
  (fri-evidence-layer-idx trace-roots fri-query-idxs round-idx 0) =
  yn
  by (rule fri-opening-matches-tableD(4)[OF match])
then show
  trace-table !
  fri-sibling-index (fri-evidence-layer-len trace-roots 0)
  (fri-evidence-layer-idx trace-roots fri-query-idxs round-idx 0) =
  xn

```

using *layer0 yn-eq* by *simp*
qed

lemma *generic-trace-sampled-layer-chain-evidence-augments-partial-trace-candidate:*

assumes *cand: partial-trace-table-candidate trace-table trace-openings*

and chain:

generic-fri-sampled-layer-chain-evidence trace-table-low-degree
trace-table (clength - 1) trace-roots trace-bs trace-final
query-idxs trace-round-layers doms layers

shows

partial-trace-table-candidate-with-recorded-first-fri-siblings
trace-table trace-openings trace-roots trace-bs query-idxs
trace-round-layers

unfolding *partial-trace-table-candidate-with-recorded-first-fri-siblings-def*

by (*intro conjI cand*

generic-trace-sampled-layer-chain-evidence-recorded-first-sibling-agreement
[*OF chain*])

definition *trace-fri-bad-with-header-tied-augmented-partial-candidate*

:: (*f, 'a*) *protocol-channel-scheme* $\Rightarrow$

(*unit list* $\times$ (*f, 'a*) *protocol-channel-scheme*) *option* $\Rightarrow$ *bool*

where

trace-fri-bad-with-header-tied-augmented-partial-candidate s out

$\longleftrightarrow$

($\exists$ *trace-roots trace-bs trace-final dg composition-roots*
composition-bs composition-final fri-query-idxs trace-round-layers
composition-round-layers fr as rest trace-openings trace-table.
accepted-fri-opening-transcript s out trace-roots trace-bs
trace-final dg composition-roots composition-bs composition-final
fri-query-idxs trace-round-layers composition-round-layers $\wedge$
verifier-header-transcript s fr trace-roots trace-final as dg
composition-roots composition-final rest $\wedge$
accepted-with-partial-trace-openings s out fr fri-query-idxs
trace-openings $\wedge$
partial-trace-table-candidate-with-recorded-first-fri-siblings
trace-table trace-openings trace-roots trace-bs fri-query-idxs
trace-round-layers $\wedge$
 $\neg$ *trace-table-low-degree trace-table*)

definition *trace-fri-bad-with-header-tied-augmented-sampled-layer-chain*

:: (*f, 'a*) *protocol-channel-scheme* $\Rightarrow$

(*unit list* $\times$ (*f, 'a*) *protocol-channel-scheme*) *option* $\Rightarrow$ *bool*

where

trace-fri-bad-with-header-tied-augmented-sampled-layer-chain s out

$\longleftrightarrow$

($\exists$ *trace-roots trace-bs trace-final dg composition-roots*
composition-bs composition-final fri-query-idxs trace-round-layers
composition-round-layers fr as rest trace-openings trace-table
doms layers.

accepted-fri-opening-transcript s out trace-roots trace-bs
trace-final dg composition-roots composition-bs composition-final
fri-query-idxs trace-round-layers composition-round-layers $\wedge$
verifier-header-transcript s fr trace-roots trace-final as dg
composition-roots composition-final rest $\wedge$
accepted-with-partial-trace-openings s out fr fri-query-idxs
trace-openings $\wedge$
partial-trace-table-candidate-with-recorded-first-fri-siblings
trace-table trace-openings trace-roots trace-bs fri-query-idxs
trace-round-layers $\wedge$
 $\neg$ *trace-table-low-degree trace-table* $\wedge$
generic-fri-sampled-layer-chain-evidence trace-table-low-degree
trace-table (clength - 1) trace-roots trace-bs trace-final
fri-query-idxs trace-round-layers doms layers)

definition *trace-fri-header-tied-augmented-missing-sampled-layer-chain*
 $:: (f, 'a) \text{ protocol-channel-scheme} \Rightarrow$
 $(\text{unit list} \times (f, 'a) \text{ protocol-channel-scheme}) \text{ option} \Rightarrow \text{bool}$

where

trace-fri-header-tied-augmented-missing-sampled-layer-chain s out
 $\longleftrightarrow$
trace-fri-bad-with-header-tied-augmented-partial-candidate s out $\wedge$
 $\neg$ *trace-fri-bad-with-header-tied-augmented-sampled-layer-chain s out*

definition *trace-fri-header-tied-unaugmented-partial-candidate*
 $:: (f, 'a) \text{ protocol-channel-scheme} \Rightarrow$
 $(\text{unit list} \times (f, 'a) \text{ protocol-channel-scheme}) \text{ option} \Rightarrow \text{bool}$

where

trace-fri-header-tied-unaugmented-partial-candidate s out
 $\longleftrightarrow$
trace-fri-bad-with-header-tied-partial-candidate s out $\wedge$
 $\neg$ *trace-fri-bad-with-header-tied-augmented-partial-candidate s out*

lemma *trace-fri-bad-with-header-tied-augmented-partial-candidate-imp-plain:*

assumes

trace-fri-bad-with-header-tied-augmented-partial-candidate s out

shows *trace-fri-bad-with-header-tied-partial-candidate s out*

proof —

from *assms* **obtain** *trace-roots trace-bs trace-final dg composition-roots*
composition-bs composition-final fri-query-idxs trace-round-layers
composition-round-layers fr as rest trace-openings trace-table

where *fri*:

accepted-fri-opening-transcript s out trace-roots trace-bs
trace-final dg composition-roots composition-bs composition-final
fri-query-idxs trace-round-layers composition-round-layers

and *header*:

verifier-header-transcript s fr trace-roots trace-final as dg
composition-roots composition-final rest

and *partial*:

accepted-with-partial-trace-openings s out fr fri-query-idxs
 trace-openings
and cand:
 partial-trace-table-candidate-with-recorded-first-fri-siblings
 trace-table trace-openings trace-roots trace-bs fri-query-idxs
 trace-round-layers
and nonlow: $\neg$ trace-table-low-degree trace-table
unfolding trace-fri-bad-with-header-tied-augmented-partial-candidate-def
by blast
have plain: partial-trace-table-candidate trace-table trace-openings
by (rule
 partial-trace-table-candidate-with-recorded-first-fri-siblingsD(1)
 [OF cand])
show ?thesis
unfolding trace-fri-bad-with-header-tied-partial-candidate-def
by (intro exI conjI)
 (rule fri, rule header, rule partial, rule plain, rule nonlow)
qed

lemma trace-fri-bad-with-header-tied-partial-candidate-imp-augmented-or-unaugmented:
assumes trace-fri-bad-with-header-tied-partial-candidate s out
shows
 trace-fri-bad-with-header-tied-augmented-partial-candidate s out $\vee$
 trace-fri-header-tied-unaugmented-partial-candidate s out
using assms
unfolding trace-fri-header-tied-unaugmented-partial-candidate-def
by blast

lemma wp-trace-fri-header-tied-partial-candidate-bound-from-augmented-and-unaugmented:
assumes augmented-bound:
 wp-event verify-monad
 (trace-fri-bad-with-header-tied-augmented-partial-candidate s) $s \leq A$
and unaugmented-bound:
 wp-event verify-monad
 (trace-fri-header-tied-unaugmented-partial-candidate s) $s \leq U$
shows
 wp-event verify-monad
 (trace-fri-bad-with-header-tied-partial-candidate s) $s \leq A + U$

proof –
have wp-event verify-monad
 (trace-fri-bad-with-header-tied-partial-candidate s) $s \leq$
 wp-event verify-monad
 (λ out.
 trace-fri-bad-with-header-tied-augmented-partial-candidate s out $\vee$
 trace-fri-header-tied-unaugmented-partial-candidate s out) s
by (rule wp-event-mono)
 (rule
 trace-fri-bad-with-header-tied-partial-candidate-imp-augmented-or-unaugmented)

also have ... $\leq$
 $\quad wp\text{-event } verify\text{-monad}$
 $\quad (trace\text{-fri-bad-with-header-tied-augmented-partial-candidate } s) \ s +$
 $\quad wp\text{-event } verify\text{-monad}$
 $\quad (trace\text{-fri-header-tied-unaugmented-partial-candidate } s) \ s$
by (rule *wp-event-union-bound*)
also have ... $\leq A + U$
by (rule *add-mono[OF augmented-bound unaugmented-bound]*)
finally show ?thesis .
qed

lemma *trace-fri-bad-with-header-tied-augmented-sampled-layer-chain-imp-plain:*
assumes
 $\quad trace\text{-fri-bad-with-header-tied-augmented-sampled-layer-chain } s \ out$
shows $trace\text{-fri-bad-with-header-tied-sampled-layer-chain } s \ out$
proof –
from *assms* **obtain** $trace\text{-roots } trace\text{-bs } trace\text{-final } dg \ composition\text{-roots}$
 $\quad composition\text{-bs } composition\text{-final } fri\text{-query-idxs } trace\text{-round-layers}$
 $\quad composition\text{-round-layers } fr \ as \ rest \ trace\text{-openings } trace\text{-table } doms$
 $\quad layers$
where *fri*:
 $\quad accepted\text{-fri-opening-transcript } s \ out \ trace\text{-roots } trace\text{-bs}$
 $\quad \quad trace\text{-final } dg \ composition\text{-roots } composition\text{-bs } composition\text{-final}$
 $\quad \quad fri\text{-query-idxs } trace\text{-round-layers } composition\text{-round-layers}$
and *header*:
 $\quad verifier\text{-header-transcript } s \ fr \ trace\text{-roots } trace\text{-final } as \ dg$
 $\quad \quad composition\text{-roots } composition\text{-final } rest$
and *partial*:
 $\quad accepted\text{-with-partial-trace-openings } s \ out \ fr \ fri\text{-query-idxs}$
 $\quad \quad trace\text{-openings}$
and *cand*:
 $\quad partial\text{-trace-table-candidate-with-recorded-first-fri-siblings}$
 $\quad \quad trace\text{-table } trace\text{-openings } trace\text{-roots } trace\text{-bs } fri\text{-query-idxs}$
 $\quad \quad trace\text{-round-layers}$
and *nonlow*: $\neg \ trace\text{-table-low-degree } trace\text{-table}$
and *chain*:
 $\quad generic\text{-fri-sampled-layer-chain-evidence } trace\text{-table-low-degree}$
 $\quad \quad trace\text{-table } (clength - 1) \ trace\text{-roots } trace\text{-bs } trace\text{-final}$
 $\quad \quad fri\text{-query-idxs } trace\text{-round-layers } doms \ layers$
unfolding
 $\quad trace\text{-fri-bad-with-header-tied-augmented-sampled-layer-chain-def}$
by *blast*
have *plain*: $partial\text{-trace-table-candidate } trace\text{-table } trace\text{-openings}$
by (rule
 $\quad partial\text{-trace-table-candidate-with-recorded-first-fri-siblingsD}(1)$
 $\quad [OF \ cand])$
show ?thesis
unfolding $trace\text{-fri-bad-with-header-tied-sampled-layer-chain-def}$
by (intro *exI conjI*)

(rule fri, rule header, rule partial, rule plain, rule nonlow,
rule chain)

qed

lemma *trace-fri-bad-with-header-tied-sampled-layer-chain-imp-augmented:*

assumes *trace-fri-bad-with-header-tied-sampled-layer-chain s out*

shows *trace-fri-bad-with-header-tied-augmented-sampled-layer-chain s out*

proof –

from *assms* **obtain** *trace-roots trace-bs trace-final dg composition-roots
composition-bs composition-final fri-query-idxs trace-round-layers
composition-round-layers fr as rest trace-openings trace-table doms
layers*

where *fri*:

*accepted-fri-opening-transcript s out trace-roots trace-bs
trace-final dg composition-roots composition-bs composition-final
fri-query-idxs trace-round-layers composition-round-layers*

and *header*:

*verifier-header-transcript s fr trace-roots trace-final as dg
composition-roots composition-final rest*

and *partial*:

*accepted-with-partial-trace-openings s out fr fri-query-idxs
trace-openings*

and *cand*: *partial-trace-table-candidate trace-table trace-openings*

and *nonlow*: $\neg$ *trace-table-low-degree trace-table*

and *chain*:

*generic-fri-sampled-layer-chain-evidence trace-table-low-degree
trace-table (clength – 1) trace-roots trace-bs trace-final
fri-query-idxs trace-round-layers doms layers*

unfolding *trace-fri-bad-with-header-tied-sampled-layer-chain-def*

by *blast*

have *augmented*:

*partial-trace-table-candidate-with-recorded-first-fri-siblings
trace-table trace-openings trace-roots trace-bs fri-query-idxs
trace-round-layers*

by (rule

generic-trace-sampled-layer-chain-evidence-augments-partial-trace-candidate
[OF *cand chain*])

show *?thesis*

unfolding

trace-fri-bad-with-header-tied-augmented-sampled-layer-chain-def

by (intro *exI conjI*)

(rule *fri*, rule *header*, rule *partial*, rule *augmented*, rule *nonlow*,
rule *chain*)

qed

lemma *wp-trace-fri-header-tied-augmented-sampled-layer-chain-bound-from-plain:*

assumes *sampling-bound*:

wp-event verify-monad

(*trace-fri-bad-with-header-tied-sampled-layer-chain s*) $s \leq R$

shows
 $wp\text{-event } verify\text{-monad}$
 $(trace\text{-fri-bad-with-header-tied-augmented-sampled-layer-chain } s) \ s$
 $\leq R$
proof –
have $wp\text{-event } verify\text{-monad}$
 $(trace\text{-fri-bad-with-header-tied-augmented-sampled-layer-chain } s) \ s$
 $\leq wp\text{-event } verify\text{-monad}$
 $(trace\text{-fri-bad-with-header-tied-sampled-layer-chain } s) \ s$
by (rule $wp\text{-event-mono}$)
(rule
 $trace\text{-fri-bad-with-header-tied-augmented-sampled-layer-chain-imp-plain}$)
then show $?thesis$
by (rule $order\text{-trans}[OF - sampled\text{-bound}]$)
qed

lemma $wp\text{-trace-fri-header-tied-sampled-layer-chain-bound-from-augmented}$:

assumes $sampled\text{-bound}$:

$wp\text{-event } verify\text{-monad}$
 $(trace\text{-fri-bad-with-header-tied-augmented-sampled-layer-chain } s) \ s$
 $\leq R$

shows

$wp\text{-event } verify\text{-monad}$
 $(trace\text{-fri-bad-with-header-tied-sampled-layer-chain } s) \ s \leq R$

proof –

have $wp\text{-event } verify\text{-monad}$
 $(trace\text{-fri-bad-with-header-tied-sampled-layer-chain } s) \ s$
 $\leq wp\text{-event } verify\text{-monad}$
 $(trace\text{-fri-bad-with-header-tied-augmented-sampled-layer-chain } s) \ s$
by (rule $wp\text{-event-mono}$)
(rule $trace\text{-fri-bad-with-header-tied-sampled-layer-chain-imp-augmented}$)
then show $?thesis$
by (rule $order\text{-trans}[OF - sampled\text{-bound}]$)
qed

lemma $trace\text{-fri-header-tied-augmented-missing-sampled-imp-assignment-obstruction}$:

assumes $missing$:

$trace\text{-fri-header-tied-augmented-missing-sampled-layer-chain } s \ out$

shows

$trace\text{-fri-header-tied-sampled-layer-assignment-obstruction } s \ out$

proof –

have bad :

$trace\text{-fri-bad-with-header-tied-augmented-partial-candidate } s \ out$

and $not\text{-sampled}$:

$\neg trace\text{-fri-bad-with-header-tied-augmented-sampled-layer-chain } s \ out$

using $missing$

unfolding $trace\text{-fri-header-tied-augmented-missing-sampled-layer-chain-def}$

by $blast+$

from bad **obtain** $trace\text{-roots } trace\text{-bs } trace\text{-final } dg \ composition\text{-roots}$

```

    composition-bs composition-final fri-query-idxs trace-round-layers
    composition-round-layers fr as rest trace-openings trace-table
where fri:
    accepted-fri-opening-transcript s out trace-roots trace-bs
    trace-final dg composition-roots composition-bs composition-final
    fri-query-idxs trace-round-layers composition-round-layers
and header:
    verifier-header-transcript s fr trace-roots trace-final as dg
    composition-roots composition-final rest
and partial:
    accepted-with-partial-trace-openings s out fr fri-query-idxs
    trace-openings
and augmented:
    partial-trace-table-candidate-with-recorded-first-fri-siblings
    trace-table trace-openings trace-roots trace-bs fri-query-idxs
    trace-round-layers
and nonlow:  $\neg$  trace-table-low-degree trace-table
unfolding trace-fri-bad-with-header-tied-augmented-partial-candidate-def
by blast
have cand: partial-trace-table-candidate trace-table trace-openings
by (rule
    partial-trace-table-candidate-with-recorded-first-fri-siblingsD(1)
    [OF augmented])
have partial-generic:
    generic-fri-partial-evidence trace-table-low-degree trace-table
    (clength - 1) trace-roots trace-bs trace-final fri-query-idxs
    trace-round-layers
by (rule accepted-fri-opening-transcript-trace-generic-partial-evidence
    [OF fri])
have no-chain:
     $\bigwedge$  doms layers.  $\neg$ 
    generic-fri-sampled-layer-chain-evidence trace-table-low-degree
    trace-table (clength - 1) trace-roots trace-bs trace-final
    fri-query-idxs trace-round-layers doms layers
proof
fix doms layers
assume chain:
    generic-fri-sampled-layer-chain-evidence trace-table-low-degree
    trace-table (clength - 1) trace-roots trace-bs trace-final
    fri-query-idxs trace-round-layers doms layers
have trace-fri-bad-with-header-tied-augmented-sampled-layer-chain s out
unfolding
    trace-fri-bad-with-header-tied-augmented-sampled-layer-chain-def
by (intro exI conjI)
    (rule fri, rule header, rule partial, rule augmented, rule nonlow,
    rule chain)
then show False
using not-sampled by contradiction
qed

```

```

have obstruction:
  generic-fri-sampled-layer-assignment-obstruction
    trace-table-low-degree trace-table (clength - 1) trace-roots
    trace-bs trace-final fri-query-idxs trace-round-layers
  unfolding generic-fri-sampled-layer-assignment-obstruction-def
  using partial-generic no-chain by blast
show ?thesis
  unfolding trace-fri-header-tied-sampled-layer-assignment-obstruction-def
  by (intro exI conjI)
    (rule fri, rule header, rule partial, rule cand, rule nonlow,
     rule obstruction)
qed

lemma wp-trace-fri-header-tied-augmented-missing-sampled-bound-from-assignment-obstruction:
  assumes obstruction-bound:
    wp-event verify-monad
      (trace-fri-header-tied-sampled-layer-assignment-obstruction s) s
      ≤ M
  shows
    wp-event verify-monad
      (trace-fri-header-tied-augmented-missing-sampled-layer-chain s) s
      ≤ M
  proof -
    have wp-event verify-monad
      (trace-fri-header-tied-augmented-missing-sampled-layer-chain s) s
      ≤ wp-event verify-monad
        (trace-fri-header-tied-sampled-layer-assignment-obstruction s) s
    by (rule wp-event-mono)
      (rule
        trace-fri-header-tied-augmented-missing-sampled-imp-assignment-obstruction)
    then show ?thesis
      by (rule order-trans[OF - obstruction-bound])
qed

lemma wp-trace-fri-header-tied-augmented-partial-candidate-bound-from-sampled-and-missing:
  assumes sampled-bound:
    wp-event verify-monad
      (trace-fri-bad-with-header-tied-augmented-sampled-layer-chain s) s
      ≤ R
  and missing-bound:
    wp-event verify-monad
      (trace-fri-header-tied-augmented-missing-sampled-layer-chain s) s
      ≤ M
  shows
    wp-event verify-monad
      (trace-fri-bad-with-header-tied-augmented-partial-candidate s) s
      ≤ R + M
  proof -
    have wp-event verify-monad

```

$(\text{trace-fri-bad-with-header-tied-augmented-partial-candidate } s) \ s$
 $\leq \text{wp-event verify-monad}$
 $(\lambda \text{out.}$
 $\quad \text{trace-fri-bad-with-header-tied-augmented-sampled-layer-chain } s \ \text{out} \ \vee$
 $\quad \text{trace-fri-header-tied-augmented-missing-sampled-layer-chain } s \ \text{out}) \ s$
unfolding $\text{trace-fri-header-tied-augmented-missing-sampled-layer-chain-def}$
by $(\text{rule wp-event-mono}) \ \text{blast}$
also have $\dots \leq$
 $\quad \text{wp-event verify-monad}$
 $\quad (\text{trace-fri-bad-with-header-tied-augmented-sampled-layer-chain } s) \ s +$
 $\quad \text{wp-event verify-monad}$
 $\quad (\text{trace-fri-header-tied-augmented-missing-sampled-layer-chain } s) \ s$
by $(\text{rule wp-event-union-bound})$
also have $\dots \leq R + M$
by $(\text{rule add-mono}[OF \ \text{sampling-bound} \ \text{missing-bound}])$
finally show $?thesis$.
qed

lemma $\text{wp-trace-fri-header-tied-augmented-partial-candidate-bound-from-sampled-conflict-and-zero}$:
assumes sampling-bound :
 $\quad \text{wp-event verify-monad}$
 $\quad (\text{trace-fri-bad-with-header-tied-augmented-sampled-layer-chain } s) \ s$
 $\quad \leq R$
and conflict-bound :
 $\quad \text{wp-event verify-monad}$
 $\quad (\text{trace-fri-header-tied-sampled-assignment-conflict } s) \ s \leq C$
and zero-bound :
 $\quad \text{wp-event verify-monad}$
 $\quad (\text{trace-fri-header-tied-zero-round-final-obstruction } s) \ s \leq Z$
shows
 $\quad \text{wp-event verify-monad}$
 $\quad (\text{trace-fri-bad-with-header-tied-augmented-partial-candidate } s) \ s$
 $\quad \leq R + (C + Z)$
proof –
have obstruction-bound :
 $\quad \text{wp-event verify-monad}$
 $\quad (\text{trace-fri-header-tied-sampled-layer-assignment-obstruction } s) \ s$
 $\quad \leq C + Z$
by $(\text{rule}$
 $\quad \text{wp-trace-fri-header-tied-assignment-obstruction-bound-from-conflict-and-zero}$
 $\quad [OF \ \text{conflict-bound} \ \text{zero-bound}])$
have missing-bound :
 $\quad \text{wp-event verify-monad}$
 $\quad (\text{trace-fri-header-tied-augmented-missing-sampled-layer-chain } s) \ s$
 $\quad \leq C + Z$
by $(\text{rule}$
 $\quad \text{wp-trace-fri-header-tied-augmented-missing-sampled-bound-from-assignment-obstruction}$
 $\quad [OF \ \text{obstruction-bound}])$
show $?thesis$

by (rule
 wp-trace-fri-header-tied-augmented-partial-candidate-bound-from-sampled-and-missing
 [*OF sampled-bound missing-bound*])
 qed

lemma *trace-fri-header-tied-augmented-partial-candidate-imp-sampled-conflict-or-zero*:
 assumes *bad*:
 trace-fri-bad-with-header-tied-augmented-partial-candidate s out
 shows
 trace-fri-bad-with-header-tied-augmented-sampled-layer-chain s out $\vee$
 trace-fri-header-tied-sampled-assignment-conflict s out $\vee$
 trace-fri-header-tied-zero-round-final-obstruction s out
proof (cases
 trace-fri-bad-with-header-tied-augmented-sampled-layer-chain s out)
 case *True*
 then show ?thesis by blast
 next
 case *False*
 have *missing*:
 trace-fri-header-tied-augmented-missing-sampled-layer-chain s out
 unfolding *trace-fri-header-tied-augmented-missing-sampled-layer-chain-def*
 using *bad False* by blast
 have *obstruction*:
 trace-fri-header-tied-sampled-layer-assignment-obstruction s out
 by (rule
 trace-fri-header-tied-augmented-missing-sampled-imp-assignment-obstruction
 [*OF missing*])
 then show ?thesis
 using *trace-fri-header-tied-assignment-obstruction-imp-conflict-or-zero*
 by blast
 qed

lemma *trace-fri-header-tied-recorded-sibling-candidate-mismatch-false*:
 $\neg$ *trace-fri-header-tied-recorded-sibling-candidate-mismatch s out*
proof
 assume *bad*:
 trace-fri-header-tied-recorded-sibling-candidate-mismatch s out
from *bad* **obtain** *trace-roots trace-bs trace-final dg composition-roots*
 composition-bs composition-final fri-query-idxs trace-round-layers
 composition-round-layers fr as rest trace-openings trace-table
 round-idx xp xp-path xn xn-path result final-state **where**
 fri-openings:
 accepted-fri-opening-transcript s out trace-roots trace-bs trace-final
 dg composition-roots composition-bs composition-final fri-query-idxs
 trace-round-layers composition-round-layers
and *agrees*:
 trace-table-agrees-with-recorded-first-fri-siblings trace-table
 trace-roots trace-bs fri-query-idxs trace-round-layers
and *round-bound*: *round-idx < length fri-query-idxs*


```

and bs-nonempty:  $0 < \text{length } \text{trace-bs}$ 
and step:
  fri-layer-step-evidence
    (trace-roots ! 0)
    (trace-bs ! 0)
    (fri-evidence-layer-len trace-roots 0)
    (fri-evidence-layer-idx trace-roots fri-query-idxs round-idx 0)
    1
    (fri-sibling-index (fri-evidence-layer-len trace-roots 0)
      (fri-evidence-layer-idx trace-roots fri-query-idxs round-idx 0))
    xp xp-path xn xn-path
    (fri-evidence-next-idx trace-roots fri-query-idxs round-idx 0)
    (fri-evidence-next-value trace-roots trace-bs fri-query-idxs
      round-idx 0 xp xn)
    (trace-round-layers ! round-idx ! 0)
and neq:
  trace-table !
  fri-sibling-index (fri-evidence-layer-len trace-roots 0)
    (fri-evidence-layer-idx trace-roots fri-query-idxs round-idx 0)
     $\neq xn$ 
unfolding trace-fri-header-tied-recorded-sibling-candidate-mismatch-def
by blast
have roots-nonempty:  $0 < \text{length } \text{trace-roots}$ 
using accepted-fri-opening-transcript-shapes(2)[OF fri-openings]
  bs-nonempty by simp
have eq:
  trace-table !
  fri-sibling-index (fri-evidence-layer-len trace-roots 0)
    (fri-evidence-layer-idx trace-roots fri-query-idxs round-idx 0) =
  xn
using agrees round-bound roots-nonempty bs-nonempty step
unfolding trace-table-agrees-with-recorded-first-fri-siblings-def
by blast
show False
using eq neq by simp
qed

lemma trace-fri-header-tied-value-aligned-sibling-mismatch-imp-recorded-sibling-missing:
assumes mismatch:
  trace-fri-header-tied-value-aligned-sibling-mismatch s out
shows
  trace-fri-header-tied-recorded-sibling-candidate-missing s out
proof –
from mismatch obtain trace-roots trace-bs trace-final dg composition-roots
  composition-bs composition-final fri-query-idxs trace-round-layers
  composition-round-layers fr as rest trace-openings trace-table
  round-idx xp xp-path xn xn-path result final-state where
  out-eq: out = Some (result, final-state)
and fri-openings:

```

```

    accepted-fri-opening-transcript s out trace-roots trace-bs trace-final
    dg composition-roots composition-bs composition-final fri-query-idxs
    trace-round-layers composition-round-layers
and header:
    verifier-header-transcript s fr trace-roots trace-final as dg
    composition-roots composition-final rest
and partial:
    accepted-with-partial-trace-openings s out fr fri-query-idxs
    trace-openings
and cand: partial-trace-table-candidate trace-table trace-openings
and nonlow:  $\neg$  trace-table-low-degree trace-table
and round-bound: round-idx < length fri-query-idxs
and bs-nonempty:  $0 < \text{length trace-bs}$ 
and openings-nonempty: trace-openings ! round-idx  $\neq []$ 
and hd-idx:
    opening-index (hd (trace-openings ! round-idx)) =
    fri-evidence-layer-idx trace-roots fri-query-idxs round-idx 0
and hd-value:
    opening-value (hd (trace-openings ! round-idx)) = xp
and step:
    fri-layer-step-evidence
    (trace-roots ! 0)
    (trace-bs ! 0)
    (fri-evidence-layer-len trace-roots 0)
    (fri-evidence-layer-idx trace-roots fri-query-idxs round-idx 0)
    1
    (fri-sibling-index (fri-evidence-layer-len trace-roots 0)
    (fri-evidence-layer-idx trace-roots fri-query-idxs round-idx 0))
    xp xp-path xn xn-path
    (fri-evidence-next-idx trace-roots fri-query-idxs round-idx 0)
    (fri-evidence-next-value trace-roots trace-bs fri-query-idxs
    round-idx 0 xp xn)
    (trace-round-layers ! round-idx ! 0)
and neg:
    trace-table !
    fri-sibling-index (fri-evidence-layer-len trace-roots 0)
    (fri-evidence-layer-idx trace-roots fri-query-idxs round-idx 0)
     $\neq xn$ 
unfolding trace-fri-header-tied-value-aligned-sibling-mismatch-def
by auto
have roots-nonempty:  $0 < \text{length trace-roots}$ 
using accepted-fri-opening-transcript-shapes(2)[OF fri-openings]
    bs-nonempty by simp
have missing:
     $\neg$  trace-table-agrees-with-recorded-first-fri-siblings trace-table
    trace-roots trace-bs fri-query-idxs trace-round-layers
unfolding trace-table-agrees-with-recorded-first-fri-siblings-def
using round-bound roots-nonempty bs-nonempty step neg by blast
show ?thesis

```

unfolding *trace-fri-header-tied-recorded-sibling-candidate-missing-def*
by (*intro exI conjI*)
 (*rule out-eq, rule fri-openings, rule header, rule partial, rule cand,*
rule missing, rule nonlow, rule round-bound, rule bs-nonempty,
rule openings-nonempty, rule hd-idx, rule hd-value, rule step, rule neg)
qed

lemma *trace-fri-header-tied-sibling-mismatch-structural-gap-imp-recorded-sibling-missing:*
assumes *gap: trace-fri-header-tied-sibling-mismatch-structural-gap s out*
shows *trace-fri-header-tied-recorded-sibling-candidate-missing s out*
using *gap*
unfolding *trace-fri-header-tied-sibling-mismatch-structural-gap-def*
by (*blast intro:*
trace-fri-header-tied-value-aligned-sibling-mismatch-imp-recorded-sibling-missing)

lemma *trace-fri-header-tied-unaugmented-partial-candidate-imp-missing-or-base-gap:*
assumes *unaugmented:*
trace-fri-header-tied-unaugmented-partial-candidate s out
shows
trace-fri-header-tied-recorded-sibling-candidate-missing s out $\vee$
trace-fri-header-tied-base-value-alignment-gap s out

proof –
from *unaugmented* **have** *plain:*
trace-fri-bad-with-header-tied-partial-candidate s out
and *not-augmented:*
 $\neg$ *trace-fri-bad-with-header-tied-augmented-partial-candidate s out*
unfolding *trace-fri-header-tied-unaugmented-partial-candidate-def*
by *simp-all*
from *plain* **obtain** *trace-roots trace-bs trace-final dg composition-roots*
composition-bs composition-final fri-query-idxs trace-round-layers
composition-round-layers fr as rest trace-openings trace-table
where *fri-openings:*
accepted-fri-opening-transcript s out trace-roots trace-bs
trace-final dg composition-roots composition-bs composition-final
fri-query-idxs trace-round-layers composition-round-layers
and *header:*
verifier-header-transcript s fr trace-roots trace-final as dg
composition-roots composition-final rest
and *partial:*
accepted-with-partial-trace-openings s out fr fri-query-idxs
trace-openings
and *cand: partial-trace-table-candidate trace-table trace-openings*
and *not-low: $\neg$ trace-table-low-degree trace-table*
unfolding *trace-fri-bad-with-header-tied-partial-candidate-def*
by *blast*
have *not-agrees:*
 $\neg$ *trace-table-agrees-with-recorded-first-fri-siblings trace-table*
trace-roots trace-bs fri-query-idxs trace-round-layers
proof

```

assume agrees:
  trace-table-agrees-with-recorded-first-fri-siblings trace-table
  trace-roots trace-bs fri-query-idxs trace-round-layers
have augmented-candidate:
  partial-trace-table-candidate-with-recorded-first-fri-siblings
  trace-table trace-openings trace-roots trace-bs fri-query-idxs
  trace-round-layers
unfolding
  partial-trace-table-candidate-with-recorded-first-fri-siblings-def
by (intro conjI cand agrees)
have trace-fri-bad-with-header-tied-augmented-partial-candidate s out
unfolding trace-fri-bad-with-header-tied-augmented-partial-candidate-def
by (intro exI conjI)
  (rule fri-openings, rule header, rule partial,
   rule augmented-candidate, rule not-low)
then show False
  using not-augmented by contradiction
qed
from not-agrees obtain round-idx xp xp-path xn xn-path where
  round-bound: round-idx < length fri-query-idxs
  and roots-nonempty: 0 < length trace-roots
  and trace-bs-nonempty: 0 < length trace-bs
  and step:
    fri-layer-step-evidence
    (trace-roots ! 0)
    (trace-bs ! 0)
    (fri-evidence-layer-len trace-roots 0)
    (fri-evidence-layer-idx trace-roots fri-query-idxs round-idx 0)
    1
    (fri-sibling-index (fri-evidence-layer-len trace-roots 0)
     (fri-evidence-layer-idx trace-roots fri-query-idxs round-idx 0))
    xp xp-path xn xn-path
    (fri-evidence-next-idx trace-roots fri-query-idxs round-idx 0)
    (fri-evidence-next-value trace-roots trace-bs fri-query-idxs
     round-idx 0 xp xn)
    (trace-round-layers ! round-idx ! 0)
  and sibling-neq:
    trace-table !
    fri-sibling-index (fri-evidence-layer-len trace-roots 0)
    (fri-evidence-layer-idx trace-roots fri-query-idxs round-idx 0)
    ≠ xn
  unfolding trace-table-agrees-with-recorded-first-fri-siblings-def
  by blast
have no-match:
  ¬ fri-opening-matches-table (fri-evidence-layer-len trace-roots 0)
  (fri-evidence-layer-idx trace-roots fri-query-idxs round-idx 0)
  trace-table xp xn
proof
  assume match:

```

```

    fri-opening-matches-table (fri-evidence-layer-len trace-roots 0)
      (fri-evidence-layer-idx trace-roots fri-query-idxs round-idx 0)
      trace-table xp xn
  have trace-table !
    fri-sibling-index (fri-evidence-layer-len trace-roots 0)
      (fri-evidence-layer-idx trace-roots fri-query-idxs round-idx 0) =
      xn
    by (rule fri-opening-matches-tableD(4)[OF match])
  then show False
    using sibling-neq by contradiction
qed
have generic-base:
  generic-fri-sampled-base-opening-conflict trace-table trace-roots
    trace-bs fri-query-idxs trace-round-layers
  unfolding generic-fri-sampled-base-opening-conflict-def
  by (intro exI conjI)
    (rule trace-bs-nonempty, rule round-bound, rule step, rule no-match)
have sampled-base:
  trace-fri-header-tied-sampled-base-opening-conflict s out
  unfolding trace-fri-header-tied-sampled-base-opening-conflict-def
  by (intro exI conjI)
    (rule fri-openings, rule header, rule partial, rule cand, rule not-low,
      rule generic-base)
show ?thesis
proof (cases
  trace-fri-header-tied-value-aligned-base-opening-conflict s out)
case True
  have trace-fri-header-tied-value-aligned-sibling-mismatch s out
    by (rule trace-fri-header-tied-value-aligned-base-imp-sibling-mismatch
      [OF True])
  then show ?thesis
    by (blast intro:
      trace-fri-header-tied-value-aligned-sibling-mismatch-imp-recorded-sibling-missing)
next
case False
  then have trace-fri-header-tied-base-value-alignment-gap s out
    using sampled-base
    unfolding trace-fri-header-tied-base-value-alignment-gap-def
    by simp
  then show ?thesis by simp
qed
qed

lemma wp-trace-fri-header-tied-unaugmented-partial-candidate-bound-from-recorded-missing-and-base-gap:
  fixes M G :: prob
  assumes missing-bound:
    wp-event verify-monad
      (trace-fri-header-tied-recorded-sibling-candidate-missing s) s ≤ M
  and gap-bound:

```

$wp\text{-event verify-monad}$
 $(\text{trace-fri-header-tied-base-value-alignment-gap } s) \ s \leq G$
shows
 $wp\text{-event verify-monad}$
 $(\text{trace-fri-header-tied-unaugmented-partial-candidate } s) \ s \leq M + G$
proof –
have $wp\text{-event verify-monad}$
 $(\text{trace-fri-header-tied-unaugmented-partial-candidate } s) \ s \leq$
 $wp\text{-event verify-monad}$
 $(\lambda out.$
 $\text{trace-fri-header-tied-recorded-sibling-candidate-missing } s \ out \ \vee$
 $\text{trace-fri-header-tied-base-value-alignment-gap } s \ out) \ s$
by (rule $wp\text{-event-mono}$)
(rule
 $\text{trace-fri-header-tied-unaugmented-partial-candidate-imp-missing-or-base-gap}$)
also have $\dots \leq$
 $wp\text{-event verify-monad}$
 $(\text{trace-fri-header-tied-recorded-sibling-candidate-missing } s) \ s +$
 $wp\text{-event verify-monad}$
 $(\text{trace-fri-header-tied-base-value-alignment-gap } s) \ s$
by (rule $wp\text{-event-union-bound}$)
also have $\dots \leq M + G$
by (rule $\text{add-mono}[OF \ \text{missing-bound} \ \text{gap-bound}]$)
finally show $?thesis$.
qed

lemma $wp\text{-trace-fri-header-tied-sibling-structural-gap-bound-from-recorded-sibling-missing}$:
assumes missing-bound :
 $wp\text{-event verify-monad}$
 $(\text{trace-fri-header-tied-recorded-sibling-candidate-missing } s) \ s \leq M$
shows
 $wp\text{-event verify-monad}$
 $(\text{trace-fri-header-tied-sibling-mismatch-structural-gap } s) \ s \leq M$
proof –
have $wp\text{-event verify-monad}$
 $(\text{trace-fri-header-tied-sibling-mismatch-structural-gap } s) \ s \leq$
 $wp\text{-event verify-monad}$
 $(\text{trace-fri-header-tied-recorded-sibling-candidate-missing } s) \ s$
by (rule $wp\text{-event-mono}$)
(rule
 $\text{trace-fri-header-tied-sibling-mismatch-structural-gap-imp-recorded-sibling-missing}$)
then show $?thesis$
by (rule $\text{order-trans}[OF \ - \ \text{missing-bound}]$)
qed

lemma $\text{trace-fri-header-tied-sibling-structural-gap-without-recorded-missing-false}$:
assumes gap : $\text{trace-fri-header-tied-sibling-mismatch-structural-gap } s \ out$
and not-missing :
 $\neg \text{trace-fri-header-tied-recorded-sibling-candidate-missing } s \ out$

shows *False*

using *trace-fri-header-tied-sibling-mismatch-structural-gap-imp-recorded-sibling-missing*
[*OF gap*] *not-missing*

by *contradiction*

lemma *wp-trace-fri-header-tied-sibling-structural-gap-without-recorded-missing-zero:*

wp-event verify-monad

$(\lambda out. \text{trace-fri-header-tied-sibling-mismatch-structural-gap } s \text{ out} \wedge$
 $\neg \text{trace-fri-header-tied-recorded-sibling-candidate-missing } s \text{ out}) \ s$
 ≤ 0

proof –

have *wp-event verify-monad*

$(\lambda out. \text{trace-fri-header-tied-sibling-mismatch-structural-gap } s \text{ out} \wedge$
 $\neg \text{trace-fri-header-tied-recorded-sibling-candidate-missing } s \text{ out}) \ s$
 $\leq \text{wp-event verify-monad } (\lambda -. \text{False}) \ s$

by (*rule wp-event-mono*)

(*auto dest:*
trace-fri-header-tied-sibling-structural-gap-without-recorded-missing-false)

also have $\dots = 0$

unfolding *wp-event-def wp-def dist-expect-def* **by** *simp*

finally show *?thesis* .

qed

lemma *trace-fri-header-tied-value-aligned-sibling-mismatch-without-recorded-missing-imp-partial-merkle:*

assumes *mismatch:*

trace-fri-header-tied-value-aligned-sibling-mismatch s out

and *not-missing:*

$\neg \text{trace-fri-header-tied-recorded-sibling-candidate-missing } s \text{ out}$

shows *partial-merkle-inconsistency-bad s out*

proof –

have *split:*

partial-merkle-inconsistency-bad s out $\vee$
trace-fri-header-tied-sibling-mismatch-structural-gap s out

by (*rule*

trace-fri-header-tied-value-aligned-sibling-mismatch-imp-partial-merkle-or-structural-gap
[*OF mismatch*])

then show *?thesis*

using *not-missing*

trace-fri-header-tied-sibling-mismatch-structural-gap-imp-recorded-sibling-missing

by *blast*

qed

lemma *wp-trace-fri-header-tied-value-aligned-sibling-mismatch-without-recorded-missing-bound-from-partial-merkle:*

assumes *merkle-bound:*

wp-event verify-monad (*partial-merkle-inconsistency-bad s*) $s \leq P$

shows

wp-event verify-monad

$(\lambda out. \text{trace-fri-header-tied-value-aligned-sibling-mismatch } s \text{ out} \wedge$
 $\neg \text{trace-fri-header-tied-recorded-sibling-candidate-missing } s \text{ out}) \ s$

$\leq P$
proof –
have *wp-event verify-monad*
 $(\lambda out. \text{trace-fri-header-tied-value-aligned-sibling-mismatch } s \text{ out} \wedge$
 $\neg \text{trace-fri-header-tied-recorded-sibling-candidate-missing } s \text{ out}) \ s$
 $\leq \text{wp-event verify-monad } (\text{partial-merkle-inconsistency-bad } s) \ s$
by (*rule wp-event-mono*)
 $(\text{auto dest:}$
 $\text{trace-fri-header-tied-value-aligned-sibling-mismatch-without-recorded-missing-imp-partial-merkle})$
then show *?thesis*
by (*rule order-trans[OF - merkle-bound]*)
qed

lemma *wp-trace-fri-header-tied-value-aligned-base-without-recorded-missing-bound-from-partial-merkle:*

assumes *merkle-bound:*

wp-event verify-monad (partial-merkle-inconsistency-bad s) s $\leq P$

shows

wp-event verify-monad

$(\lambda out.$
 $\text{trace-fri-header-tied-value-aligned-base-opening-conflict } s \text{ out} \wedge$
 $\neg \text{trace-fri-header-tied-recorded-sibling-candidate-missing } s \text{ out}) \ s$
 $\leq P$

proof –

have *wp-event verify-monad*

$(\lambda out.$
 $\text{trace-fri-header-tied-value-aligned-base-opening-conflict } s \text{ out} \wedge$
 $\neg \text{trace-fri-header-tied-recorded-sibling-candidate-missing } s \text{ out}) \ s$
 $\leq \text{wp-event verify-monad}$
 $(\lambda out. \text{trace-fri-header-tied-value-aligned-sibling-mismatch } s \text{ out} \wedge$
 $\neg \text{trace-fri-header-tied-recorded-sibling-candidate-missing } s \text{ out}) \ s$

by (*rule wp-event-mono*)

$(\text{auto intro: trace-fri-header-tied-value-aligned-base-imp-sibling-mismatch})$

also have $\dots \leq P$

by (*rule*

wp-trace-fri-header-tied-value-aligned-sibling-mismatch-without-recorded-missing-bound-from-partial-merkle
 $[\text{OF merkle-bound}])$

finally show *?thesis* .

qed

lemma *wp-trace-fri-header-tied-sampled-base-without-recorded-missing-bound-from-partial-merkle-and-gap:*

fixes $P \ G :: \text{prob}$

assumes *merkle-bound:*

wp-event verify-monad (partial-merkle-inconsistency-bad s) s $\leq P$

and *gap-bound:*

wp-event verify-monad

$(\text{trace-fri-header-tied-base-value-alignment-gap } s) \ s \leq G$

shows

wp-event verify-monad

$(\lambda out.$

$\text{trace-fri-header-tied-sampled-base-opening-conflict } s \text{ out} \wedge$
 $\neg \text{trace-fri-header-tied-recorded-sibling-candidate-missing } s \text{ out}) \ s$
 $\leq P + G$
proof –
have *wp-event verify-monad*
 $(\lambda \text{out.}$
 $\text{trace-fri-header-tied-sampled-base-opening-conflict } s \text{ out} \wedge$
 $\neg \text{trace-fri-header-tied-recorded-sibling-candidate-missing } s \text{ out}) \ s$
 $\leq \text{wp-event verify-monad}$
 $(\lambda \text{out.}$
 $(\text{trace-fri-header-tied-value-aligned-base-opening-conflict } s \text{ out} \wedge$
 $\neg \text{trace-fri-header-tied-recorded-sibling-candidate-missing } s \text{ out}) \vee$
 $\text{trace-fri-header-tied-base-value-alignment-gap } s \text{ out}) \ s$
by (rule *wp-event-mono*)
 $(\text{auto dest:}$
 $\text{trace-fri-header-tied-sampled-base-imp-value-aligned-or-gap})$
also have ... $\leq$
 $\text{wp-event verify-monad}$
 $(\lambda \text{out.}$
 $\text{trace-fri-header-tied-value-aligned-base-opening-conflict } s \text{ out} \wedge$
 $\neg \text{trace-fri-header-tied-recorded-sibling-candidate-missing } s \text{ out}) \ s +$
 $\text{wp-event verify-monad}$
 $(\text{trace-fri-header-tied-base-value-alignment-gap } s) \ s$
by (rule *wp-event-union-bound*)
also have ... $\leq P + G$
by (intro *add-mono gap-bound*
 $\text{wp-trace-fri-header-tied-value-aligned-base-without-recorded-missing-bound-from-partial-merkle}$
 $[\text{OF merkle-bound}])$
finally show *?thesis* .
qed

lemma *trace-fri-header-tied-recorded-sibling-candidate-missingE*:
assumes *missing*:
 $\text{trace-fri-header-tied-recorded-sibling-candidate-missing } s \text{ out}$
obtains *result final-state trace-roots trace-bs trace-final dg*
 $\text{composition-roots composition-bs composition-final fri-query-idxs}$
 $\text{trace-round-layers composition-round-layers fr as rest trace-openings}$
 $\text{trace-table round-idx xp xp-path xn xn-path}$
where
 $\text{out} = \text{Some } (\text{result}, \text{final-state})$
 $\text{accepted-fri-opening-transcript } s \text{ out trace-roots trace-bs trace-final}$
 $\text{dg composition-roots composition-bs composition-final fri-query-idxs}$
 $\text{trace-round-layers composition-round-layers}$
 $\text{verifier-header-transcript } s \text{ fr trace-roots trace-final as dg}$
 $\text{composition-roots composition-final rest}$
 $\text{accepted-with-partial-trace-openings } s \text{ out fr fri-query-idxs}$
 trace-openings
 $\text{partial-trace-table-candidate trace-table trace-openings}$
 $\neg \text{trace-table-agrees-with-recorded-first-fri-siblings trace-table}$

```

    trace-roots trace-bs fri-query-idxs trace-round-layers
  ¬ trace-table-low-degree trace-table
  round-idx < length fri-query-idxs
  0 < length trace-bs
  trace-openings ! round-idx ≠ []
  opening-index (hd (trace-openings ! round-idx)) =
    fri-evidence-layer-idx trace-roots fri-query-idxs round-idx 0
  opening-value (hd (trace-openings ! round-idx)) = xp
  fri-layer-step-evidence
    (trace-roots ! 0)
    (trace-bs ! 0)
    (fri-evidence-layer-len trace-roots 0)
    (fri-evidence-layer-idx trace-roots fri-query-idxs round-idx 0)
    1
    (fri-sibling-index (fri-evidence-layer-len trace-roots 0)
      (fri-evidence-layer-idx trace-roots fri-query-idxs round-idx 0))
  xp xp-path xn xn-path
  (fri-evidence-next-idx trace-roots fri-query-idxs round-idx 0)
  (fri-evidence-next-value trace-roots trace-bs fri-query-idxs
    round-idx 0 xp xn)
  (trace-round-layers ! round-idx ! 0)
  trace-table !
  fri-sibling-index (fri-evidence-layer-len trace-roots 0)
    (fri-evidence-layer-idx trace-roots fri-query-idxs round-idx 0)
  ≠ xn
using missing
unfolding trace-fri-header-tied-recorded-sibling-candidate-missing-def
apply (elim exE conjE)
apply (rule that)
apply assumption+
done

```

lemma trace-fri-header-tied-recorded-sibling-candidate-missing-no-same-sampled-chain:

assumes missing:

trace-fri-header-tied-recorded-sibling-candidate-missing s out

obtains result final-state trace-roots trace-bs trace-final dg
 composition-roots composition-bs composition-final fri-query-idxs
 trace-round-layers composition-round-layers fr as rest trace-openings
 trace-table round-idx xp xp-path xn xn-path

where

out = Some (result, final-state)
 accepted-fri-opening-transcript s out trace-roots trace-bs trace-final
 dg composition-roots composition-bs composition-final fri-query-idxs
 trace-round-layers composition-round-layers
 verifier-header-transcript s fr trace-roots trace-final as dg
 composition-roots composition-final rest
 accepted-with-partial-trace-openings s out fr fri-query-idxs
 trace-openings
 partial-trace-table-candidate trace-table trace-openings

```

    ¬ trace-table-low-degree trace-table
  ∧ doms layers.
    ¬ generic-fri-sampled-layer-chain-evidence trace-table-low-degree
      trace-table (clength - 1) trace-roots trace-bs trace-final
      fri-query-idxs trace-round-layers doms layers
proof -
  from missing show thesis
proof (rule trace-fri-header-tied-recorded-sibling-candidate-missingE)
  fix result final-state trace-roots trace-bs trace-final dg
    composition-roots composition-bs composition-final fri-query-idxs
    trace-round-layers composition-round-layers fr as rest trace-openings
    trace-table round-idx xp xp-path xn xn-path
  assume out-eq: out = Some (result, final-state)
  and fri-openings:
    accepted-fri-opening-transcript s out trace-roots trace-bs
    trace-final dg composition-roots composition-bs composition-final
    fri-query-idxs trace-round-layers composition-round-layers
  and header:
    verifier-header-transcript s fr trace-roots trace-final as dg
    composition-roots composition-final rest
  and partial:
    accepted-with-partial-trace-openings s out fr fri-query-idxs
    trace-openings
  and candidate:
    partial-trace-table-candidate trace-table trace-openings
  and no-agree:
    ¬ trace-table-agrees-with-recorded-first-fri-siblings trace-table
    trace-roots trace-bs fri-query-idxs trace-round-layers
  and nonlow: ¬ trace-table-low-degree trace-table
  have no-chain:
    ∧ doms layers.
    ¬ generic-fri-sampled-layer-chain-evidence trace-table-low-degree
      trace-table (clength - 1) trace-roots trace-bs trace-final
      fri-query-idxs trace-round-layers doms layers
proof
  fix doms layers
  assume chain:
    generic-fri-sampled-layer-chain-evidence trace-table-low-degree
      trace-table (clength - 1) trace-roots trace-bs trace-final
      fri-query-idxs trace-round-layers doms layers
  have trace-table-agrees-with-recorded-first-fri-siblings trace-table
    trace-roots trace-bs fri-query-idxs trace-round-layers
  by (rule
    generic-trace-sampled-layer-chain-evidence-recorded-first-sibling-agreement
    [OF chain])
  then show False
  using no-agree by contradiction
qed
show thesis

```

by (rule that[*OF out-eq fri-openings header partial candidate nonlow no-chain*])
 qed
 qed

lemma *trace-fri-header-tied-recorded-sibling-candidate-missing-imp-assignment-obstruction:*
 assumes *missing:*
 trace-fri-header-tied-recorded-sibling-candidate-missing s out
 shows
 trace-fri-header-tied-sampled-layer-assignment-obstruction s out
proof (rule
 trace-fri-header-tied-recorded-sibling-candidate-missing-no-same-sampled-chain
 [*OF missing*])
fix *result final-state trace-roots trace-bs trace-final dg*
 composition-roots composition-bs composition-final fri-query-idxs
 trace-round-layers composition-round-layers fr as rest trace-openings
 trace-table round-idx xp xp-path xn xn-path
assume *fri-openings:*
 accepted-fri-opening-transcript s out trace-roots trace-bs trace-final
 dg composition-roots composition-bs composition-final fri-query-idxs
 trace-round-layers composition-round-layers
and *header:*
 verifier-header-transcript s fr trace-roots trace-final as dg
 composition-roots composition-final rest
and *partial:*
 accepted-with-partial-trace-openings s out fr fri-query-idxs
 trace-openings
and *cand:* *partial-trace-table-candidate trace-table trace-openings*
and *not-low:* $\neg$ *trace-table-low-degree trace-table*
and *no-chain:*
 $\bigwedge$ *doms layers.*
 $\neg$ *generic-fri-sampled-layer-chain-evidence trace-table-low-degree*
 trace-table (clength - 1) trace-roots trace-bs trace-final
 fri-query-idxs trace-round-layers doms layers
have *partial-generic:*
 generic-fri-partial-evidence trace-table-low-degree trace-table
 (clength - 1) trace-roots trace-bs trace-final fri-query-idxs
 trace-round-layers
by (rule *accepted-fri-opening-transcript-trace-generic-partial-evidence*
 [*OF fri-openings*])
have *obstruction:*
 generic-fri-sampled-layer-assignment-obstruction
 trace-table-low-degree trace-table (clength - 1) trace-roots
 trace-bs trace-final fri-query-idxs trace-round-layers
unfolding *generic-fri-sampled-layer-assignment-obstruction-def*
using *partial-generic no-chain* **by** *blast*
show *trace-fri-header-tied-sampled-layer-assignment-obstruction s out*
unfolding *trace-fri-header-tied-sampled-layer-assignment-obstruction-def*
by (*intro exI conjI*)

(rule *fri-openings*, rule *header*, rule *partial*, rule *cand*,
rule *not-low*, rule *obstruction*)

qed

lemma *trace-fri-header-tied-recorded-sibling-candidate-missing-imp-conflict-or-zero*:
assumes *missing*:
trace-fri-header-tied-recorded-sibling-candidate-missing s out
shows
trace-fri-header-tied-sampled-assignment-conflict s out $\vee$
trace-fri-header-tied-zero-round-final-obstruction s out
by (rule *trace-fri-header-tied-assignment-obstruction-imp-conflict-or-zero*)
(rule
trace-fri-header-tied-recorded-sibling-candidate-missing-imp-assignment-obstruction
[*OF missing*])

lemma *wp-trace-fri-header-tied-recorded-sibling-candidate-missing-bound-from-conflict-and-zero*:
assumes *conflict-bound*:
wp-event verify-monad
(*trace-fri-header-tied-sampled-assignment-conflict s*) $s \leq C$
and *zero-bound*:
wp-event verify-monad
(*trace-fri-header-tied-zero-round-final-obstruction s*) $s \leq Z$
shows
wp-event verify-monad
(*trace-fri-header-tied-recorded-sibling-candidate-missing s*) s
 $\leq C + Z$

proof –
have *wp-event verify-monad*
(*trace-fri-header-tied-recorded-sibling-candidate-missing s*) $s \leq$
wp-event verify-monad
($\lambda out. trace-fri-header-tied-sampled-assignment-conflict s out \vee$
trace-fri-header-tied-zero-round-final-obstruction s out) s
by (rule *wp-event-mono*)
(rule
trace-fri-header-tied-recorded-sibling-candidate-missing-imp-conflict-or-zero)
also have ... $\leq$
wp-event verify-monad
(*trace-fri-header-tied-sampled-assignment-conflict s*) $s +$
wp-event verify-monad
(*trace-fri-header-tied-zero-round-final-obstruction s*) s
by (rule *wp-event-union-bound*)
also have ... $\leq C + Z$
by (rule *add-mono*[*OF conflict-bound zero-bound*])
finally show ?thesis .

qed

end

end

theory *Soundness-FRI-Staged-Replay-Bounds*

imports

Soundness-FRI-Staged-Active-Bounds
Soundness-FRI-Replay-Gap-Assignment
Soundness-FRI-Composition-Replay-Gaps
Soundness-FRI-Trace-Replay-Gaps
Soundness-FRI-Trace-Sibling-Evidence

begin

Staged active FRI bounds that additionally use the replay-gap assignment adapters. This theory is deliberately downstream of both the staged prequery-accounting layer and the replay-gap layer.

context *soundness*

begin

lemma *wp-trace-fri-header-tied-without-same-without-recorded-missing-bound-from-partial-merkle-and-replay-gaps:*

fixes $P\ G\ N\ S\ F :: \text{prob}$

assumes *merkle-bound:*

wp-event verify-monad (partial-merkle-inconsistency-bad s) s $\leq P$

and *alignment-gap-bound:*

wp-event verify-monad

(trace-fri-header-tied-base-value-alignment-gap s) s $\leq G$

and *next-gap-bound:*

wp-event verify-monad

(trace-fri-header-tied-next-value-replay-gap s) s $\leq N$

and *successor-gap-bound:*

wp-event verify-monad

(trace-fri-header-tied-successor-replay-gap s) s $\leq S$

and *final-gap-bound:*

wp-event verify-monad

(trace-fri-header-tied-final-value-replay-gap s) s $\leq F$

shows

wp-event verify-monad

(λ out.

trace-fri-header-tied-sampled-assignment-conflict-without-same-layer

s out $\wedge$

$\neg \text{trace-fri-header-tied-recorded-sibling-candidate-missing } s \text{ out})\ s$

$\leq (P + G) + N + S + F$

proof –

have *base-bound:*

wp-event verify-monad

(λ out.

trace-fri-header-tied-sampled-base-opening-conflict s out $\wedge$

$\neg \text{trace-fri-header-tied-recorded-sibling-candidate-missing } s \text{ out})\ s$

$\leq P + G$

by *(rule*

wp-trace-fri-header-tied-sampled-base-without-recorded-missing-bound-from-partial-merkle-and-gap

$[OF \text{ merkle-bound alignment-gap-bound}]$
have $wp\text{-event verify-monad}$
 $(\lambda out.$
 $\text{trace-fri-header-tied-sampled-assignment-conflict-without-same-layer}$
 $s \text{ out} \wedge$
 $\neg \text{trace-fri-header-tied-recorded-sibling-candidate-missing } s \text{ out}) \ s$
 $\leq wp\text{-event verify-monad}$
 $(\lambda out.$
 $(\text{trace-fri-header-tied-sampled-base-opening-conflict } s \text{ out} \wedge$
 $\neg \text{trace-fri-header-tied-recorded-sibling-candidate-missing } s \text{ out}) \vee$
 $\text{trace-fri-header-tied-sampled-next-value-conflict } s \text{ out} \vee$
 $\text{trace-fri-header-tied-sampled-successor-opening-conflict } s \text{ out} \vee$
 $\text{trace-fri-header-tied-sampled-final-value-conflict } s \text{ out}) \ s$
by $(\text{rule } wp\text{-event-mono})$
 $(\text{auto dest: trace-fri-header-tied-without-same-imp-branch})$
also have $\dots \leq$
 $wp\text{-event verify-monad}$
 $(\lambda out.$
 $\text{trace-fri-header-tied-sampled-base-opening-conflict } s \text{ out} \wedge$
 $\neg \text{trace-fri-header-tied-recorded-sibling-candidate-missing } s \text{ out}) \ s +$
 $wp\text{-event verify-monad}$
 $(\text{trace-fri-header-tied-sampled-next-value-conflict } s) \ s +$
 $wp\text{-event verify-monad}$
 $(\text{trace-fri-header-tied-sampled-successor-opening-conflict } s) \ s +$
 $wp\text{-event verify-monad}$
 $(\text{trace-fri-header-tied-sampled-final-value-conflict } s) \ s$
by $(\text{rule } wp\text{-event-union-bound}_4)$
also have $\dots \leq (P + G) + N + S + F$
by $(\text{intro add-mono base-bound}$
 $wp\text{-trace-fri-header-tied-sampled-next-value-bound-from-replay-gap}$
 $[OF \text{ next-gap-bound}]$
 $wp\text{-trace-fri-header-tied-sampled-successor-bound-from-replay-gap}$
 $[OF \text{ successor-gap-bound}]$
 $wp\text{-trace-fri-header-tied-sampled-final-value-bound-from-replay-gap}$
 $[OF \text{ final-gap-bound}])$
finally show $?thesis$.
qed

lemma $wp\text{-trace-fri-header-tied-authenticated-conflict-without-recorded-missing-bound-from-partial-merkle-and-replay}$
fixes $P \ G \ N \ S \ F :: \text{prob}$
assumes merkle-bound:
 $wp\text{-event verify-monad } (\text{partial-merkle-inconsistency-bad } s) \ s \leq P$
and $\text{alignment-gap-bound:}$
 $wp\text{-event verify-monad}$
 $(\text{trace-fri-header-tied-base-value-alignment-gap } s) \ s \leq G$
and next-gap-bound:
 $wp\text{-event verify-monad}$
 $(\text{trace-fri-header-tied-next-value-replay-gap } s) \ s \leq N$
and $\text{successor-gap-bound:}$

$wp\text{-event verify-monad}$
 $(\text{trace-fri-header-tied-successor-replay-gap } s) \ s \leq S$
and final-gap-bound :
 $wp\text{-event verify-monad}$
 $(\text{trace-fri-header-tied-final-value-replay-gap } s) \ s \leq F$
shows
 $wp\text{-event verify-monad}$
 $(\lambda out.$
 $\text{trace-fri-header-tied-sampled-assignment-conflict-with-authenticated-same-layer}$
 $s \ out \wedge$
 $\neg \text{trace-fri-header-tied-recorded-sibling-candidate-missing } s \ out) \ s$
 $\leq ((P + G) + N + S + F) + P$
proof –
have without-bound :
 $wp\text{-event verify-monad}$
 $(\lambda out.$
 $\text{trace-fri-header-tied-sampled-assignment-conflict-without-same-layer}$
 $s \ out \wedge$
 $\neg \text{trace-fri-header-tied-recorded-sibling-candidate-missing } s \ out) \ s$
 $\leq (P + G) + N + S + F$
by (rule
 $wp\text{-trace-fri-header-tied-without-same-without-recorded-missing-bound-from-partial-merkle-and-replay-gaps}$
 $[OF \ \text{merkle-bound} \ \text{alignment-gap-bound} \ \text{next-gap-bound} \ \text{successor-gap-bound}$
 $\text{final-gap-bound}]$)
have $wp\text{-event verify-monad}$
 $(\lambda out.$
 $\text{trace-fri-header-tied-sampled-assignment-conflict-with-authenticated-same-layer}$
 $s \ out \wedge$
 $\neg \text{trace-fri-header-tied-recorded-sibling-candidate-missing } s \ out) \ s$
 $\leq wp\text{-event verify-monad}$
 $(\lambda out.$
 $(\text{trace-fri-header-tied-sampled-assignment-conflict-without-same-layer}$
 $s \ out \wedge$
 $\neg \text{trace-fri-header-tied-recorded-sibling-candidate-missing } s \ out) \vee$
 $\text{trace-fri-header-tied-same-layer-conflict-with-authenticated-chunks}$
 $s \ out) \ s$
by (rule $wp\text{-event-mono}$)
 $(\text{auto dest:}$
 $\text{trace-fri-header-tied-authenticated-conflict-imp-without-same-or-same-layer})$
also have $\dots \leq$
 $wp\text{-event verify-monad}$
 $(\lambda out.$
 $\text{trace-fri-header-tied-sampled-assignment-conflict-without-same-layer}$
 $s \ out \wedge$
 $\neg \text{trace-fri-header-tied-recorded-sibling-candidate-missing } s \ out) \ s +$
 $wp\text{-event verify-monad}$
 $(\text{trace-fri-header-tied-same-layer-conflict-with-authenticated-chunks } s)$
 s
by (rule $wp\text{-event-union-bound}$)

also have $\dots \leq ((P + G) + N + S + F) + P$
by (*intro add-mono without-bound*
wp-trace-fri-header-tied-same-layer-conflict-with-authenticated-chunks-bound-from-partial-merkle
[OF merkle-bound])
finally show *?thesis* .
qed

lemma *wp-trace-fri-header-tied-sampled-assignment-conflict-without-recorded-missing-bound-from-auth-gap-and-replay-gap*
fixes $A\ P\ G\ N\ S\ F :: \text{prob}$
assumes *auth-gap-bound*:
wp-event verify-monad
(trace-fri-header-tied-assignment-auth-gap s) s ≤ A
and *merkle-bound*:
wp-event verify-monad (partial-merkle-inconsistency-bad s) s ≤ P
and *alignment-gap-bound*:
wp-event verify-monad
(trace-fri-header-tied-base-value-alignment-gap s) s ≤ G
and *next-gap-bound*:
wp-event verify-monad
(trace-fri-header-tied-next-value-replay-gap s) s ≤ N
and *successor-gap-bound*:
wp-event verify-monad
(trace-fri-header-tied-successor-replay-gap s) s ≤ S
and *final-gap-bound*:
wp-event verify-monad
(trace-fri-header-tied-final-value-replay-gap s) s ≤ F
shows
wp-event verify-monad
(λout.
trace-fri-header-tied-sampled-assignment-conflict s out ∧
¬ trace-fri-header-tied-recorded-sibling-candidate-missing s out) s
≤ (((P + G) + N + S + F) + P) + A
proof –
have *auth-bound*:
wp-event verify-monad
(λout.
trace-fri-header-tied-sampled-assignment-conflict-with-authenticated-same-layer
s out ∧
¬ trace-fri-header-tied-recorded-sibling-candidate-missing s out) s
≤ ((P + G) + N + S + F) + P
by (*rule*
wp-trace-fri-header-tied-authenticated-conflict-without-recorded-missing-bound-from-partial-merkle-and-replay-gap
[OF merkle-bound alignment-gap-bound next-gap-bound successor-gap-bound
final-gap-bound])
have *wp-event verify-monad*
(λout.
trace-fri-header-tied-sampled-assignment-conflict s out ∧
¬ trace-fri-header-tied-recorded-sibling-candidate-missing s out) s
≤ wp-event verify-monad

```

    (λout.
      (trace-fri-header-tied-sampled-assignment-conflict-with-authenticated-same-layer
        s out ∧
        ¬ trace-fri-header-tied-recorded-sibling-candidate-missing s out) ∨
      trace-fri-header-tied-assignment-auth-gap s out) s
  by (rule wp-event-mono)
  (auto dest: trace-fri-header-tied-assignment-imp-authenticated-or-auth-gap)
also have ... ≤
  wp-event verify-monad
  (λout.
    trace-fri-header-tied-sampled-assignment-conflict-with-authenticated-same-layer
      s out ∧
    ¬ trace-fri-header-tied-recorded-sibling-candidate-missing s out) s +
  wp-event verify-monad
  (trace-fri-header-tied-assignment-auth-gap s) s
  by (rule wp-event-union-bound)
also have ... ≤ (((P + G) + N + S + F) + P) + A
  by (intro add-mono auth-bound auth-gap-bound)
finally show ?thesis .
qed

lemma wp-trace-fri-header-tied-sampled-assignment-conflict-bound-from-recorded-missing-and-complement:
fixes Missing Complement :: prob
assumes missing-bound:
  wp-event verify-monad
  (trace-fri-header-tied-recorded-sibling-candidate-missing s) s
  ≤ Missing
and complement-bound:
  wp-event verify-monad
  (λout.
    trace-fri-header-tied-sampled-assignment-conflict s out ∧
    ¬ trace-fri-header-tied-recorded-sibling-candidate-missing s out) s
  ≤ Complement
shows
  wp-event verify-monad
  (trace-fri-header-tied-sampled-assignment-conflict s) s
  ≤ Missing + Complement
proof –
have wp-event verify-monad
  (trace-fri-header-tied-sampled-assignment-conflict s) s ≤
  wp-event verify-monad
  (λout.
    trace-fri-header-tied-recorded-sibling-candidate-missing s out ∨
    (trace-fri-header-tied-sampled-assignment-conflict s out ∧
      ¬ trace-fri-header-tied-recorded-sibling-candidate-missing s out)) s
  by (rule wp-event-mono) blast
also have ... ≤
  wp-event verify-monad
  (trace-fri-header-tied-recorded-sibling-candidate-missing s) s +

```

$wp\text{-}event\ verify\text{-}monad$
 $(\lambda out.$
 $\quad trace\text{-}fri\text{-}header\text{-}tied\text{-}sampled\text{-}assignment\text{-}conflict\ s\ out \wedge$
 $\quad \neg trace\text{-}fri\text{-}header\text{-}tied\text{-}recorded\text{-}sibling\text{-}candidate\text{-}missing\ s\ out) \ s$
 $\text{by } (rule\ wp\text{-}event\text{-}union\text{-}bound)$
 $\text{also have } \dots \leq Missing + Complement$
 $\text{by } (intro\ add\text{-}mono\ missing\text{-}bound\ complement\text{-}bound)$
 $\text{finally show } ?thesis .$
 qed

lemma *checked-staged-security-trace-fri-header-tied-augmented-sampled-layer-chain-bound-from-fresh-and-prequery:*

assumes wf : *staged-budget-wellformed budgets*
and *controlled*: *staged-adversary-controlled budgets A*
and *finite-B*: *finite B*
and *cover*:
 $\wedge data\ attacker\text{-}state.$
 $\quad Some\ (data,\ attacker\text{-}state) \in$
 $\quad \quad set\text{-}dist\ (execute\ (checked\text{-}staged\text{-}transcript\text{-}program\ A)$
 $\quad \quad \quad adversary\text{-}initial\text{-}state) \implies$
 $\quad trace\text{-}fri\text{-}sampled\text{-}layer\text{-}chain\text{-}cover$
 $\quad \quad (verifier\text{-}state\text{-}from\text{-}adversary\ attacker\text{-}state$
 $\quad \quad \quad (staged\text{-}proof\text{-}transcript\ data))\ bad$
and *envelope*:
 $\wedge data\ attacker\text{-}state\ trace\text{-}table.$
 $\quad Some\ (data,\ attacker\text{-}state) \in$
 $\quad \quad set\text{-}dist\ (execute\ (checked\text{-}staged\text{-}transcript\text{-}program\ A)$
 $\quad \quad \quad adversary\text{-}initial\text{-}state) \implies$
 $\quad trace\text{-}fri\text{-}multiround\text{-}bad\text{-}sets\ bad\ trace\text{-}table \subseteq B$
and *fresh-bound*:
 $wp\text{-}event\ (checked\text{-}staged\text{-}security\text{-}experiment\text{-}with\text{-}data\text{-}state\ A)$
 $\quad (staged\text{-}security\text{-}with\text{-}data\text{-}state\text{-}verifier\text{-}event$
 $\quad \quad (\lambda s. trace\text{-}fri\text{-}challenge\text{-}list\text{-}fresh\text{-}hit\ s\ B))$
 $\quad adversary\text{-}initial\text{-}state \leq Fresh$

shows

$wp\text{-}event\ (checked\text{-}staged\text{-}security\text{-}experiment\text{-}with\text{-}data\text{-}state\ A)$
 $\quad (staged\text{-}security\text{-}with\text{-}data\text{-}state\text{-}verifier\text{-}event$
 $\quad \quad trace\text{-}fri\text{-}bad\text{-}with\text{-}header\text{-}tied\text{-}augmented\text{-}sampled\text{-}layer\text{-}chain)$
 $\quad adversary\text{-}initial\text{-}state \leq$
 $\quad Fresh +$
 $\quad \quad hash\text{-}relation\text{-}budget\text{-}value\ (card\ B * ceil\text{-}log\ clength)$
 $\quad \quad (staged\text{-}attacker\text{-}query\text{-}budget\ budgets +$
 $\quad \quad \quad staged\text{-}challenge\text{-}query\text{-}budget)$

proof —

have *sampled*:
 $wp\text{-}event\ (checked\text{-}staged\text{-}security\text{-}experiment\text{-}with\text{-}data\text{-}state\ A)$
 $\quad (staged\text{-}security\text{-}with\text{-}data\text{-}state\text{-}verifier\text{-}event$
 $\quad \quad trace\text{-}fri\text{-}bad\text{-}with\text{-}header\text{-}tied\text{-}sampled\text{-}layer\text{-}chain)$
 $\quad adversary\text{-}initial\text{-}state \leq$
 $\quad Fresh +$

```

    hash-relation-budget-value (card B * ceil-log clength)
      (staged-attacker-query-budget budgets +
       staged-challenge-query-budget)
  by (rule
    checked-staged-security-trace-fri-header-tied-sampled-layer-chain-bound-from-fresh-and-prequery
    [OF wf controlled finite-B cover envelope fresh-bound])
  have wp-event (checked-staged-security-experiment-with-data-state A)
    (staged-security-with-data-state-verifier-event
     trace-fri-bad-with-header-tied-augmented-sampled-layer-chain)
  adversary-initial-state ≤
  wp-event (checked-staged-security-experiment-with-data-state A)
    (staged-security-with-data-state-verifier-event
     trace-fri-bad-with-header-tied-sampled-layer-chain)
  adversary-initial-state
  unfolding staged-security-with-data-state-verifier-event-def
  by (rule wp-event-mono)
    (auto dest:
      trace-fri-bad-with-header-tied-augmented-sampled-layer-chain-imp-plain
      split: option.splits prod.splits)
  then show ?thesis
    by (rule order-trans[OF - sampled])
qed

lemma checked-staged-security-trace-fri-header-tied-partial-candidate-bound-from-augmented-and-unaugmented:
  assumes augmented-bound:
    wp-event (checked-staged-security-experiment-with-data-state A)
      (staged-security-with-data-state-verifier-event
       trace-fri-bad-with-header-tied-augmented-partial-candidate)
    adversary-initial-state ≤ Augmented
  and unaugmented-bound:
    ∧ data attacker-state.
    Some (data, attacker-state) ∈
      set-dist (execute (checked-staged-transcript-program A)
        adversary-initial-state) ⇒
    wp-event verify-monad
      (trace-fri-header-tied-unaugmented-partial-candidate
       (verifier-state-from-adversary attacker-state
        (staged-proof-transcript data)))
      (verifier-state-from-adversary attacker-state
       (staged-proof-transcript data)) ≤ Unaugmented
  shows
    wp-event (checked-staged-security-experiment-with-data-state A)
      (staged-security-with-data-state-verifier-event
       trace-fri-bad-with-header-tied-partial-candidate)
    adversary-initial-state ≤ Augmented + Unaugmented
proof -
  have unaugmented-staged:
    wp-event (checked-staged-security-experiment-with-data-state A)
      (staged-security-with-data-state-verifier-event

```

```

    trace-fri-header-tied-unaugmented-partial-candidate)
  adversary-initial-state ≤ Unaugmented
by (rule checked-staged-security-with-data-state-verifier-event-bound-from-cont)
  (auto simp: trace-fri-header-tied-unaugmented-partial-candidate-def
    unaugmented-bound)
have wp-event (checked-staged-security-experiment-with-data-state A)
  (staged-security-with-data-state-verifier-event
    trace-fri-bad-with-header-tied-partial-candidate)
  adversary-initial-state ≤
  wp-event (checked-staged-security-experiment-with-data-state A)
  (λout.
    staged-security-with-data-state-verifier-event
      trace-fri-bad-with-header-tied-augmented-partial-candidate out ∨
    staged-security-with-data-state-verifier-event
      trace-fri-header-tied-unaugmented-partial-candidate out)
  adversary-initial-state
unfolding staged-security-with-data-state-verifier-event-def
by (rule wp-event-mono-on-support)
  (auto dest:
    trace-fri-bad-with-header-tied-partial-candidate-imp-augmented-or-unaugmented
    split: option.splits prod.splits)
also have ... ≤
  wp-event (checked-staged-security-experiment-with-data-state A)
  (staged-security-with-data-state-verifier-event
    trace-fri-bad-with-header-tied-augmented-partial-candidate)
  adversary-initial-state +
  wp-event (checked-staged-security-experiment-with-data-state A)
  (staged-security-with-data-state-verifier-event
    trace-fri-header-tied-unaugmented-partial-candidate)
  adversary-initial-state
by (rule wp-event-union-bound)
also have ... ≤ Augmented + Unaugmented
by (rule add-mono[OF augmented-bound unaugmented-staged])
finally show ?thesis .
qed

```

lemma *checked-staged-security-trace-fri-header-tied-unaugmented-partial-candidate-bound-from-recorded-missing-and*

assumes *merkle-bound:*

$\bigwedge \text{data attacker-state.}$

Some (data, attacker-state) ∈
set-dist (execute (checked-staged-transcript-program A)
adversary-initial-state) ⇒

wp-event verify-monad
(partial-merkle-inconsistency-bad
(verifier-state-from-adversary attacker-state
(staged-proof-transcript data)))
(verifier-state-from-adversary attacker-state
(staged-proof-transcript data)) ≤ Merkle

and *missing-bound:*

$\wedge$ data attacker-state.
 Some (data, attacker-state) $\in$
 set-dist (execute (checked-staged-transcript-program A)
 adversary-initial-state) $\implies$
 wp-event verify-monad
 (trace-fri-header-tied-recorded-sibling-candidate-missing
 (verifier-state-from-adversary attacker-state
 (staged-proof-transcript data)))
 (verifier-state-from-adversary attacker-state
 (staged-proof-transcript data)) $\leq$ Missing
shows
 wp-event (checked-staged-security-experiment-with-data-state A)
 (staged-security-with-data-state-verifier-event
 trace-fri-header-tied-unaugmented-partial-candidate)
 adversary-initial-state $\leq$ Missing + Merkle
proof (rule checked-staged-security-with-data-state-verifier-event-bound-from-cont)
fix data attacker-state
assume support:
 Some (data, attacker-state) $\in$
 set-dist (execute (checked-staged-transcript-program A)
 adversary-initial-state)
let ?s =
 verifier-state-from-adversary attacker-state
 (staged-proof-transcript data)
have head-neq:
 wp-event verify-monad
 (trace-fri-header-tied-base-head-value-neq-gap ?s) ?s $\leq$ Merkle
by (rule
 wp-trace-fri-header-tied-base-head-value-neq-gap-bound-from-partial-merkle)
 (rule merkle-bound[OF support])
have head-mismatch:
 wp-event verify-monad
 (trace-fri-header-tied-base-head-value-mismatch-gap ?s) ?s $\leq$ Merkle
by (rule
 wp-trace-fri-header-tied-base-head-value-mismatch-gap-bound-from-neq-gap
 [OF head-neq])
have alignment:
 wp-event verify-monad
 (trace-fri-header-tied-base-value-alignment-gap ?s) ?s $\leq$ Merkle
by (rule
 wp-trace-fri-header-tied-base-value-alignment-gap-bound-from-head-value-mismatch-gap
 [OF head-mismatch])
show
 wp-event verify-monad
 (trace-fri-header-tied-unaugmented-partial-candidate ?s) ?s
 $\leq$ Missing + Merkle
by (rule
 wp-trace-fri-header-tied-unaugmented-partial-candidate-bound-from-recorded-missing-and-base-gap)
 (rule missing-bound[OF support], rule alignment)

```

next
  fix s
  show  $\neg$  trace-fri-header-tied-unaugmented-partial-candidate s None
  unfolding trace-fri-header-tied-unaugmented-partial-candidate-def
    trace-fri-bad-with-header-tied-partial-candidate-def
    accepted-fri-opening-transcript-def
  by simp
qed

lemma trace-fri-header-tied-reduction-from-augmented-and-recorded-missing-bounds:
  fixes Augmented Missing Merkle :: prob
  assumes augmented-bound:
    wp-event verify-monad
      (trace-fri-bad-with-header-tied-augmented-partial-candidate s) s
       $\leq$  Augmented
  and missing-bound:
    wp-event verify-monad
      (trace-fri-header-tied-recorded-sibling-candidate-missing s) s
       $\leq$  Missing
  and merkle-bound:
    wp-event verify-monad (partial-merkle-inconsistency-bad s) s
       $\leq$  Merkle
  and total-bound:
    Augmented + (Missing + Merkle)  $\leq$  trace-fri-error
  shows trace-fri-header-tied-reduction s
proof -
  have head-neq:
    wp-event verify-monad
      (trace-fri-header-tied-base-head-value-neq-gap s) s  $\leq$  Merkle
  by (rule
    wp-trace-fri-header-tied-base-head-value-neq-gap-bound-from-partial-merkle)
    (rule merkle-bound)
  have head-mismatch:
    wp-event verify-monad
      (trace-fri-header-tied-base-head-value-mismatch-gap s) s
       $\leq$  Merkle
  by (rule
    wp-trace-fri-header-tied-base-head-value-mismatch-gap-bound-from-neq-gap
    [OF head-neq])
  have alignment:
    wp-event verify-monad
      (trace-fri-header-tied-base-value-alignment-gap s) s  $\leq$  Merkle
  by (rule
    wp-trace-fri-header-tied-base-value-alignment-gap-bound-from-head-value-mismatch-gap
    [OF head-mismatch])
  have unaugmented:
    wp-event verify-monad
      (trace-fri-header-tied-unaugmented-partial-candidate s) s
       $\leq$  Missing + Merkle

```

```

by (rule
  wp-trace-fri-header-tied-unaugmented-partial-candidate-bound-from-recorded-missing-and-base-gap)
  (rule missing-bound, rule alignment)
have partial:
  wp-event verify-monad
    (trace-fri-bad-with-header-tied-partial-candidate s) s
    ≤ Augmented + (Missing + Merkle)
by (rule
  wp-trace-fri-header-tied-partial-candidate-bound-from-augmented-and-unaugmented)
  (rule augmented-bound, rule unaugmented)
show ?thesis
  unfolding trace-fri-header-tied-reduction-def
  by (rule order-trans[OF partial total-bound])
qed

```

lemma *active-trace-fri-header-tied-reductions-from-augmented-and-recorded-missing-bounds:*

fixes *Augmented Missing Merkle ::*

'f staged-proof-data ⇒ 'f protocol-channel ⇒ prob

assumes *augmented-bound:*

$\wedge$ *data attacker-state.*

Some (data, attacker-state) ∈
set-dist (execute (checked-staged-transcript-program A)
adversary-initial-state) ⇒
wp-event verify-monad
(trace-fri-bad-with-header-tied-augmented-partial-candidate
(verifier-state-from-adversary attacker-state
(staged-proof-transcript data)))
(verifier-state-from-adversary attacker-state
(staged-proof-transcript data)) ≤ Augmented data attacker-state

and *missing-bound:*

$\wedge$ *data attacker-state.*

Some (data, attacker-state) ∈
set-dist (execute (checked-staged-transcript-program A)
adversary-initial-state) ⇒
wp-event verify-monad
(trace-fri-header-tied-recorded-sibling-candidate-missing
(verifier-state-from-adversary attacker-state
(staged-proof-transcript data)))
(verifier-state-from-adversary attacker-state
(staged-proof-transcript data)) ≤ Missing data attacker-state

and *merkle-bound:*

$\wedge$ *data attacker-state.*

Some (data, attacker-state) ∈
set-dist (execute (checked-staged-transcript-program A)
adversary-initial-state) ⇒
wp-event verify-monad
(partial-merkle-inconsistency-bad
(verifier-state-from-adversary attacker-state
(staged-proof-transcript data)))

$(\text{verifier-state-from-adversary attacker-state}$
 $(\text{staged-proof-transcript data})) \leq \text{Merkle data attacker-state}$
and *total-bound*:
 $\wedge \text{data attacker-state.}$
 $\text{Some (data, attacker-state)} \in$
 $\text{set-dist (execute (checked-staged-transcript-program A)}$
 $\text{adversary-initial-state)} \implies$
 $\text{Augmented data attacker-state} +$
 $(\text{Missing data attacker-state} + \text{Merkle data attacker-state})$
 $\leq \text{trace-fri-error}$
and *support*:
 $\text{Some (data, attacker-state)} \in$
 $\text{set-dist (execute (checked-staged-transcript-program A)}$
 $\text{adversary-initial-state})$
shows
 $\text{trace-fri-header-tied-reduction}$
 $(\text{verifier-state-from-adversary attacker-state}$
 $(\text{staged-proof-transcript data}))$
by (*rule*
 $\text{trace-fri-header-tied-reduction-from-augmented-and-recorded-missing-bounds}$)
 $(\text{rule augmented-bound[OF support]}, \text{rule missing-bound[OF support]},$
 $\text{rule merkle-bound[OF support]}, \text{rule total-bound[OF support]})$

lemma *checked-staged-security-trace-fri-header-tied-partial-candidate-bound-from-fresh-prequery-without-same-residu*

assumes *wf*: *staged-budget-wellformed budgets*
and *controlled*: *staged-adversary-controlled budgets A*
and *finite-B*: *finite B*
and *cover*:
 $\wedge \text{data attacker-state.}$
 $\text{Some (data, attacker-state)} \in$
 $\text{set-dist (execute (checked-staged-transcript-program A)}$
 $\text{adversary-initial-state)} \implies$
 $\text{trace-fri-sampled-layer-chain-cover}$
 $(\text{verifier-state-from-adversary attacker-state}$
 $(\text{staged-proof-transcript data})) \text{ bad}$
and *envelope*:
 $\wedge \text{data attacker-state trace-table.}$
 $\text{Some (data, attacker-state)} \in$
 $\text{set-dist (execute (checked-staged-transcript-program A)}$
 $\text{adversary-initial-state)} \implies$
 $\text{trace-fri-multiround-bad-sets bad trace-table} \subseteq B$
and *fresh-bound*:
 $\text{wp-event (checked-staged-security-experiment-with-data-state A)}$
 $(\text{staged-security-with-data-state-verifier-event}$
 $(\lambda s. \text{trace-fri-challenge-list-fresh-hit s B}))$
 $\text{adversary-initial-state} \leq \text{Fresh}$
and *merkle-bound*:
 $\wedge \text{data attacker-state.}$
 $\text{Some (data, attacker-state)} \in$

```

    set-dist (execute (checked-staged-transcript-program A)
      adversary-initial-state)  $\implies$ 
wp-event verify-monad
  (partial-merkle-inconsistency-bad
    (verifier-state-from-adversary attacker-state
      (staged-proof-transcript data)))
  (verifier-state-from-adversary attacker-state
    (staged-proof-transcript data))  $\leq$  Merkle
and structural-bound:
 $\wedge$ data attacker-state.
  Some (data, attacker-state)  $\in$ 
    set-dist (execute (checked-staged-transcript-program A)
      adversary-initial-state)  $\implies$ 
wp-event verify-monad
  (trace-fri-header-tied-sibling-mismatch-structural-gap
    (verifier-state-from-adversary attacker-state
      (staged-proof-transcript data)))
  (verifier-state-from-adversary attacker-state
    (staged-proof-transcript data))  $\leq$  Structural
and zero-bound:
 $\wedge$ data attacker-state.
  Some (data, attacker-state)  $\in$ 
    set-dist (execute (checked-staged-transcript-program A)
      adversary-initial-state)  $\implies$ 
wp-event verify-monad
  (trace-fri-header-tied-zero-round-final-obstruction
    (verifier-state-from-adversary attacker-state
      (staged-proof-transcript data)))
  (verifier-state-from-adversary attacker-state
    (staged-proof-transcript data))  $\leq$  Zero
shows
wp-event (checked-staged-security-experiment-with-data-state A)
  (staged-security-with-data-state-verifier-event
    trace-fri-bad-with-header-tied-partial-candidate)
adversary-initial-state  $\leq$ 
(Fresh +
  hash-relation-budget-value (card B * ceil-log clength)
    (staged-attacker-query-budget budgets +
      staged-challenge-query-budget)) +
  (((((Merkle + Structural) + Merkle) + (Merkle + 0) + (Merkle + 0) +
    (Merkle + 0 + 0)) + Merkle) + 0 + Zero)
proof (rule
  checked-staged-security-trace-fri-header-tied-partial-candidate-bound-from-fresh-prequery-and-residual-gaps
    [OF wf controlled finite-B cover envelope fresh-bound])
fix data attacker-state
assume support:
  Some (data, attacker-state)  $\in$ 
    set-dist (execute (checked-staged-transcript-program A)
      adversary-initial-state)

```

```

let ?s =
  verifier-state-from-adversary attacker-state
    (staged-proof-transcript data)
show wp-event verify-monad
  (trace-fri-header-tied-authenticated-slot-recorded-chunk-gap ?s) ?s
  ≤ 0
  by (rule wp-trace-fri-header-tied-authenticated-slot-recorded-chunk-gap-zero)
show wp-event verify-monad (partial-merkle-inconsistency-bad ?s) ?s
  ≤ Merkle
  by (rule merkle-bound[OF support])
show wp-event verify-monad
  (trace-fri-header-tied-sibling-mismatch-structural-gap ?s) ?s
  ≤ Structural
  by (rule structural-bound[OF support])
have head-neq:
  wp-event verify-monad
    (trace-fri-header-tied-base-head-value-neq-gap ?s) ?s ≤ Merkle
  by (rule
    wp-trace-fri-header-tied-base-head-value-neq-gap-bound-from-partial-merkle)
    (rule merkle-bound[OF support])
have head-mismatch:
  wp-event verify-monad
    (trace-fri-header-tied-base-head-value-mismatch-gap ?s) ?s
    ≤ Merkle
  by (rule
    wp-trace-fri-header-tied-base-head-value-mismatch-gap-bound-from-neq-gap
      [OF head-neq])
show wp-event verify-monad
  (trace-fri-header-tied-base-value-alignment-gap ?s) ?s ≤ Merkle
  by (rule
    wp-trace-fri-header-tied-base-value-alignment-gap-bound-from-head-value-mismatch-gap
      [OF head-mismatch])
show wp-event verify-monad
  (trace-fri-header-tied-next-value-replay-gap ?s) ?s ≤ Merkle + 0
  by (rule
    wp-trace-fri-header-tied-next-value-replay-gap-bound-from-merkle-and-slot)
    (rule merkle-bound[OF support],
    rule wp-trace-fri-header-tied-authenticated-slot-recorded-chunk-gap-zero)
show wp-event verify-monad
  (trace-fri-header-tied-successor-replay-gap ?s) ?s ≤ Merkle + 0
  by (rule
    wp-trace-fri-header-tied-successor-replay-gap-bound-from-merkle-and-slot)
    (rule merkle-bound[OF support],
    rule wp-trace-fri-header-tied-authenticated-slot-recorded-chunk-gap-zero)
show wp-event verify-monad
  (trace-fri-header-tied-final-value-replay-gap ?s) ?s
  ≤ Merkle + 0 + 0
  by (rule
    wp-trace-fri-header-tied-final-value-replay-gap-bound-from-merkle-slot-and-selected-step)

```

```

(rule merkle-bound[OF support],
 rule wp-trace-fri-header-tied-authenticated-slot-recorded-chunk-gap-zero,
 rule wp-trace-fri-header-tied-final-value-selected-step-missing-gap-zero)
show wp-event verify-monad
  (trace-fri-header-tied-zero-round-final-obstruction ?s) ?s ≤ Zero
by (rule zero-bound[OF support])
qed

lemma checked-staged-security-trace-fri-header-tied-zero-round-bound-from-staged-checked-and-merkle:
assumes checked-bound:
  wp-event (checked-staged-security-experiment-with-data-state A)
  (staged-security-with-data-state-verifier-event
   trace-fri-zero-round-checked-final-obstruction)
  adversary-initial-state ≤ CheckedZero
and merkle-bound:
  ∧ data attacker-state.
  Some (data, attacker-state) ∈
    set-dist (execute (checked-staged-transcript-program A)
                     adversary-initial-state) ⇒
  wp-event verify-monad
    (partial-merkle-inconsistency-bad
     (verifier-state-from-adversary attacker-state
      (staged-proof-transcript data))))
  (verifier-state-from-adversary attacker-state
   (staged-proof-transcript data)) ≤ Merkle
shows
  wp-event (checked-staged-security-experiment-with-data-state A)
  (staged-security-with-data-state-verifier-event
   trace-fri-header-tied-zero-round-final-obstruction)
  adversary-initial-state ≤ CheckedZero + Merkle
proof –
have merkle-staged:
  wp-event (checked-staged-security-experiment-with-data-state A)
  (staged-security-with-data-state-verifier-event
   partial-merkle-inconsistency-bad)
  adversary-initial-state ≤ Merkle
by (rule checked-staged-security-with-data-state-verifier-event-bound-from-cont)
  (auto simp: partial-merkle-inconsistency-bad-def merkle-bound)
have split:
  wp-event (checked-staged-security-experiment-with-data-state A)
  (staged-security-with-data-state-verifier-event
   trace-fri-header-tied-zero-round-final-obstruction)
  adversary-initial-state ≤
  wp-event (checked-staged-security-experiment-with-data-state A)
  (λ out.
   staged-security-with-data-state-verifier-event
    trace-fri-zero-round-checked-final-obstruction out ∨
   staged-security-with-data-state-verifier-event
    partial-merkle-inconsistency-bad out)

```

adversary-initial-state
unfolding *staged-security-with-data-state-verifier-event-def*
by (rule *wp-event-mono-on-support*)
 (auto dest:
 trace-fri-header-tied-zero-round-final-obstruction-imp-checked-or-merkle
 split: option.splits prod.splits)
also have ... $\leq$
 wp-event (checked-staged-security-experiment-with-data-state A)
 (*staged-security-with-data-state-verifier-event*
 trace-fri-zero-round-checked-final-obstruction)
 adversary-initial-state +
 wp-event (checked-staged-security-experiment-with-data-state A)
 (*staged-security-with-data-state-verifier-event*
 partial-merkle-inconsistency-bad)
 adversary-initial-state
by (rule *wp-event-union-bound*)
also have ... $\leq$ *CheckedZero* + *Merkle*
by (intro *add-mono checked-bound merkle-staged*)
finally show ?thesis .
qed

lemma *checked-staged-security-trace-fri-header-tied-partial-candidate-bound-from-fresh-prequery-staged-checked-zero-*
assumes *wf: staged-budget-wellformed budgets*
and *controlled: staged-adversary-controlled budgets A*
and *finite-B: finite B*
and *cover:*
 $\bigwedge$ *data attacker-state.*
 Some (data, attacker-state) $\in$
 set-dist (execute (checked-staged-transcript-program A)
 adversary-initial-state) $\implies$
 trace-fri-sampled-layer-chain-cover
 (*verifier-state-from-adversary attacker-state*
 (*staged-proof-transcript data*)) *bad*
and *envelope:*
 $\bigwedge$ *data attacker-state trace-table.*
 Some (data, attacker-state) $\in$
 set-dist (execute (checked-staged-transcript-program A)
 adversary-initial-state) $\implies$
 trace-fri-multi-round-bad-sets bad trace-table $\subseteq B$
and *fresh-bound:*
wp-event (checked-staged-security-experiment-with-data-state A)
 (*staged-security-with-data-state-verifier-event*
 ($\lambda s. \text{trace-fri-challenge-list-fresh-hit } s \ B$))
 adversary-initial-state $\leq$ Fresh
and *merkle-bound:*
 $\bigwedge$ *data attacker-state.*
 Some (data, attacker-state) $\in$
 set-dist (execute (checked-staged-transcript-program A)
 adversary-initial-state) $\implies$

wp-event verify-monad
(partial-merkle-inconsistency-bad
(verifier-state-from-adversary attacker-state
(staged-proof-transcript data)))
(verifier-state-from-adversary attacker-state
(staged-proof-transcript data)) ≤ Merkle
and *structural-bound:*
 $\wedge$ *data attacker-state.*
Some (data, attacker-state) ∈
set-dist (execute (checked-staged-transcript-program A)
adversary-initial-state) ⇒
wp-event verify-monad
(trace-fri-header-tied-sibling-mismatch-structural-gap
(verifier-state-from-adversary attacker-state
(staged-proof-transcript data)))
(verifier-state-from-adversary attacker-state
(staged-proof-transcript data)) ≤ Structural
and *checked-zero-bound:*
wp-event (checked-staged-security-experiment-with-data-state A)
(staged-security-with-data-state-verifier-event
trace-fri-zero-round-checked-final-obstruction)
adversary-initial-state ≤ CheckedZero
shows
wp-event (checked-staged-security-experiment-with-data-state A)
(staged-security-with-data-state-verifier-event
trace-fri-bad-with-header-tied-partial-candidate)
adversary-initial-state ≤
(Fresh +
*hash-relation-budget-value (card B * ceil-log clength)*
(staged-attacker-query-budget budgets +
staged-challenge-query-budget)) +
(((((Merkle + Structural) + Merkle) + (Merkle + 0) + (Merkle + 0) +
(Merkle + 0 + 0)) + Merkle) + 0 + (CheckedZero + Merkle))
proof –
have *sampled:*
wp-event (checked-staged-security-experiment-with-data-state A)
(staged-security-with-data-state-verifier-event
trace-fri-bad-with-header-tied-sampled-layer-chain)
adversary-initial-state ≤
Fresh +
*hash-relation-budget-value (card B * ceil-log clength)*
(staged-attacker-query-budget budgets +
staged-challenge-query-budget)
by (rule
checked-staged-security-trace-fri-header-tied-sampled-layer-chain-bound-from-fresh-and-prequery
[OF wf controlled finite-B cover envelope fresh-bound])
have *conflict:*
wp-event (checked-staged-security-experiment-with-data-state A)
(staged-security-with-data-state-verifier-event

```

    trace-fri-header-tied-sampled-assignment-conflict)
  adversary-initial-state ≤
  (((((Merkle + Structural) + Merkle) + (Merkle + 0) +
    (Merkle + 0) + (Merkle + 0 + 0)) + Merkle) + 0)
proof (rule checked-staged-security-with-data-state-verifier-event-bound-from-cont)
fix data attacker-state
assume support:
  Some (data, attacker-state) ∈
    set-dist (execute (checked-staged-transcript-program A)
      adversary-initial-state)
let ?s =
  verifier-state-from-adversary attacker-state
  (staged-proof-transcript data)
have head-neq:
  wp-event verify-monad
    (trace-fri-header-tied-base-head-value-neq-gap ?s) ?s ≤ Merkle
  by (rule
    wp-trace-fri-header-tied-base-head-value-neq-gap-bound-from-partial-merkle)
  (rule merkle-bound[OF support])
have head-mismatch:
  wp-event verify-monad
    (trace-fri-header-tied-base-head-value-mismatch-gap ?s) ?s
    ≤ Merkle
  by (rule
    wp-trace-fri-header-tied-base-head-value-mismatch-gap-bound-from-neq-gap
    [OF head-neq])
have alignment:
  wp-event verify-monad
    (trace-fri-header-tied-base-value-alignment-gap ?s) ?s ≤ Merkle
  by (rule
    wp-trace-fri-header-tied-base-value-alignment-gap-bound-from-head-value-mismatch-gap
    [OF head-mismatch])
have next-bound:
  wp-event verify-monad
    (trace-fri-header-tied-next-value-replay-gap ?s) ?s ≤ Merkle + 0
  by (rule
    wp-trace-fri-header-tied-next-value-replay-gap-bound-from-merkle-and-slot)
  (rule merkle-bound[OF support],
    rule wp-trace-fri-header-tied-authenticated-slot-recorded-chunk-gap-zero)
have successor:
  wp-event verify-monad
    (trace-fri-header-tied-successor-replay-gap ?s) ?s ≤ Merkle + 0
  by (rule
    wp-trace-fri-header-tied-successor-replay-gap-bound-from-merkle-and-slot)
  (rule merkle-bound[OF support],
    rule wp-trace-fri-header-tied-authenticated-slot-recorded-chunk-gap-zero)
have final:
  wp-event verify-monad
    (trace-fri-header-tied-final-value-replay-gap ?s) ?s

```

```

    ≤ Merkle + 0 + 0
  by (rule
    wp-trace-fri-header-tied-final-value-replay-gap-bound-from-merkle-slot-and-selected-step)
    (rule merkle-bound[OF support],
    rule wp-trace-fri-header-tied-authenticated-slot-recorded-chunk-gap-zero,
    rule wp-trace-fri-header-tied-final-value-selected-step-missing-gap-zero)
  show wp-event verify-monad
    (trace-fri-header-tied-sampled-assignment-conflict ?s) ?s
    ≤ (((((Merkle + Structural) + Merkle) + (Merkle + 0) +
    (Merkle + 0) + (Merkle + 0 + 0)) + Merkle) + 0)
  by (rule
    wp-trace-fri-header-tied-sampled-assignment-conflict-bound-from-slot-recorded-chunk-gap-and-residuals)
    (rule wp-trace-fri-header-tied-authenticated-slot-recorded-chunk-gap-zero,
    rule merkle-bound[OF support],
    rule structural-bound[OF support],
    rule alignment, rule next-bound, rule successor, rule final)
next
fix s
show ¬ trace-fri-header-tied-sampled-assignment-conflict s None
  unfolding trace-fri-header-tied-sampled-assignment-conflict-def
    accepted-fri-opening-transcript-def
  by simp
qed
have zero:
  wp-event (checked-staged-security-experiment-with-data-state A)
    (staged-security-with-data-state-verifier-event
      trace-fri-header-tied-zero-round-final-obstruction)
  adversary-initial-state ≤ CheckedZero + Merkle
  by (rule
    checked-staged-security-trace-fri-header-tied-zero-round-bound-from-staged-checked-and-merkle)
    (rule checked-zero-bound, rule merkle-bound)
show ?thesis
  by (rule
    checked-staged-security-trace-fri-header-tied-partial-candidate-bound-from-sampled-conflict-and-staged-zero
      [OF sampled conflict zero])
qed

lemma checked-staged-security-trace-fri-header-tied-partial-candidate-bound-from-fresh-prequery-staged-checked-zero-
  assumes wf: staged-budget-wellformed budgets
  and controlled: staged-adversary-controlled budgets A
  and finite-B: finite B
  and cover:
    ∧ data attacker-state.
    Some (data, attacker-state) ∈
      set-dist (execute (checked-staged-transcript-program A)
        adversary-initial-state) ⇒
    trace-fri-sampled-layer-chain-cover
      (verifier-state-from-adversary attacker-state
        (staged-proof-transcript data)) bad

```


and *envelope*:
 $\wedge$ data attacker-state trace-table.
 Some (data, attacker-state) $\in$
 set-dist (execute (checked-staged-transcript-program A)
 adversary-initial-state) $\implies$
 trace-fri-multiround-bad-sets bad trace-table $\subseteq B$
and *fresh-bound*:
 wp-event (checked-staged-security-experiment-with-data-state A)
 (staged-security-with-data-state-verifier-event
 (λs . trace-fri-challenge-list-fresh-hit s B))
 adversary-initial-state $\leq$ Fresh
and *merkle-bound*:
 $\wedge$ data attacker-state.
 Some (data, attacker-state) $\in$
 set-dist (execute (checked-staged-transcript-program A)
 adversary-initial-state) $\implies$
 wp-event verify-monad
 (partial-merkle-inconsistency-bad
 (verifier-state-from-adversary attacker-state
 (staged-proof-transcript data)))
 (verifier-state-from-adversary attacker-state
 (staged-proof-transcript data)) $\leq$ Merkle
and *missing-bound*:
 $\wedge$ data attacker-state.
 Some (data, attacker-state) $\in$
 set-dist (execute (checked-staged-transcript-program A)
 adversary-initial-state) $\implies$
 wp-event verify-monad
 (trace-fri-header-tied-recorded-sibling-candidate-missing
 (verifier-state-from-adversary attacker-state
 (staged-proof-transcript data)))
 (verifier-state-from-adversary attacker-state
 (staged-proof-transcript data)) $\leq$ Missing
and *checked-zero-bound*:
 wp-event (checked-staged-security-experiment-with-data-state A)
 (staged-security-with-data-state-verifier-event
 trace-fri-zero-round-checked-final-obstruction)
 adversary-initial-state $\leq$ CheckedZero
shows
 wp-event (checked-staged-security-experiment-with-data-state A)
 (staged-security-with-data-state-verifier-event
 trace-fri-bad-with-header-tied-partial-candidate)
 adversary-initial-state $\leq$
 (Fresh +
 hash-relation-budget-value (card B * ceil-log clength)
 (staged-attacker-query-budget budgets +
 staged-challenge-query-budget)) +
 (((((Merkle + Missing) + Merkle) + (Merkle + 0) + (Merkle + 0) +
 (Merkle + 0 + 0)) + Merkle) + 0 + (CheckedZero + Merkle))

proof (rule
checked-staged-security-trace-fri-header-tied-partial-candidate-bound-from-fresh-prequery-staged-checked-zero-and-w
 $[OF\ wf\ controlled\ finite-B\ cover\ envelope\ fresh-bound\ merkle-bound - checked-zero-bound]$)
fix data attacker-state
assume support:
Some (data, attacker-state) $\in$
set-dist (execute (checked-staged-transcript-program A)
adversary-initial-state)
let ?s =
verifier-state-from-adversary attacker-state
(*staged-proof-transcript* data)
show wp-event verify-monad
(*trace-fri-header-tied-sibling-mismatch-structural-gap* ?s) ?s
 $\leq$ Missing
by (rule
wp-trace-fri-header-tied-sibling-structural-gap-bound-from-recorded-sibling-missing)
(rule missing-bound[OF support])
qed

lemma *checked-staged-security-trace-fri-header-tied-partial-candidate-bound-from-fresh-prequery-staged-checked-zero-*
assumes wf: *staged-budget-wellformed* budgets
and controlled: *staged-adversary-controlled* budgets A
and finite-B: *finite* B
and cover:
 $\wedge$ data attacker-state.
Some (data, attacker-state) $\in$
set-dist (execute (checked-staged-transcript-program A)
adversary-initial-state) $\implies$
trace-fri-sampled-layer-chain-cover
(*verifier-state-from-adversary* attacker-state
(*staged-proof-transcript* data)) bad
and envelope:
 $\wedge$ data attacker-state trace-table.
Some (data, attacker-state) $\in$
set-dist (execute (checked-staged-transcript-program A)
adversary-initial-state) $\implies$
trace-fri-multi-round-bad-sets bad trace-table $\subseteq$ B
and fresh-bound:
wp-event (checked-staged-security-experiment-with-data-state A)
(*staged-security-with-data-state-verifier-event*
($\lambda s.$ *trace-fri-challenge-list-fresh-hit* s B))
adversary-initial-state $\leq$ Fresh
and merkle-bound:
 $\wedge$ data attacker-state.
Some (data, attacker-state) $\in$
set-dist (execute (checked-staged-transcript-program A)
adversary-initial-state) $\implies$
wp-event verify-monad
(*partial-merkle-inconsistency-bad*)

$(\text{verifier-state-from-adversary attacker-state}$
 $(\text{staged-proof-transcript data}))$
 $(\text{verifier-state-from-adversary attacker-state}$
 $(\text{staged-proof-transcript data})) \leq \text{Merkle}$
and *missing-bound*:
 $\wedge \text{data attacker-state.}$
 $\text{Some (data, attacker-state)} \in$
 $\text{set-dist (execute (checked-staged-transcript-program A)}$
 $\text{adversary-initial-state)} \implies$
 $\text{wp-event verify-monad}$
 $(\text{trace-fri-header-tied-recorded-sibling-candidate-missing}$
 $(\text{verifier-state-from-adversary attacker-state}$
 $(\text{staged-proof-transcript data}))$
 $(\text{verifier-state-from-adversary attacker-state}$
 $(\text{staged-proof-transcript data})) \leq \text{Missing}$
and *checked-zero-bound*:
 $\text{wp-event (checked-staged-security-experiment-with-data-state A)}$
 $(\text{staged-security-with-data-state-verifier-event}$
 $\text{trace-fri-zero-round-checked-final-obstruction})$
 $\text{adversary-initial-state} \leq \text{CheckedZero}$
shows
 $\text{wp-event (checked-staged-security-experiment-with-data-state A)}$
 $(\text{staged-security-with-data-state-verifier-event}$
 $\text{trace-fri-bad-with-header-tied-partial-candidate})$
 $\text{adversary-initial-state} \leq$
 $(\text{Fresh} +$
 $\text{hash-relation-budget-value (card B * ceil-log clength)}$
 $(\text{staged-attacker-query-budget budgets} +$
 $\text{staged-challenge-query-budget})) +$
 $((\text{Missing} +$
 $((((\text{Merkle} + \text{Merkle}) + (\text{Merkle} + 0) + (\text{Merkle} + 0) +$
 $(\text{Merkle} + 0 + 0)) + \text{Merkle}) + 0)) +$
 $(\text{CheckedZero} + \text{Merkle}))$
proof –
have *sampled*:
 $\text{wp-event (checked-staged-security-experiment-with-data-state A)}$
 $(\text{staged-security-with-data-state-verifier-event}$
 $\text{trace-fri-bad-with-header-tied-sampled-layer-chain})$
 $\text{adversary-initial-state} \leq$
 $\text{Fresh} +$
 $\text{hash-relation-budget-value (card B * ceil-log clength)}$
 $(\text{staged-attacker-query-budget budgets} +$
 $\text{staged-challenge-query-budget})$
by (rule
 $\text{checked-staged-security-trace-fri-header-tied-sampled-layer-chain-bound-from-fresh-and-prequery}$
 $[\text{OF wf controlled finite-B cover envelope fresh-bound}])$
have *conflict*:
 $\text{wp-event (checked-staged-security-experiment-with-data-state A)}$
 $(\text{staged-security-with-data-state-verifier-event}$

```

    trace-fri-header-tied-sampled-assignment-conflict)
  adversary-initial-state ≤
  Missing +
  (((Merkle + Merkle) + (Merkle + 0) + (Merkle + 0) +
   (Merkle + 0 + 0)) + Merkle) + 0)
proof (rule checked-staged-security-with-data-state-verifier-event-bound-from-cont)
fix data attacker-state
assume support:
  Some (data, attacker-state) ∈
  set-dist (execute (checked-staged-transcript-program A)
    adversary-initial-state)
let ?s =
  verifier-state-from-adversary attacker-state
  (staged-proof-transcript data)
have head-neq:
  wp-event verify-monad
  (trace-fri-header-tied-base-head-value-neq-gap ?s) ?s ≤ Merkle
by (rule
  wp-trace-fri-header-tied-base-head-value-neq-gap-bound-from-partial-merkle)
  (rule merkle-bound[OF support])
have head-mismatch:
  wp-event verify-monad
  (trace-fri-header-tied-base-head-value-mismatch-gap ?s) ?s
  ≤ Merkle
by (rule
  wp-trace-fri-header-tied-base-head-value-mismatch-gap-bound-from-neq-gap
  [OF head-neq])
have alignment:
  wp-event verify-monad
  (trace-fri-header-tied-base-value-alignment-gap ?s) ?s ≤ Merkle
by (rule
  wp-trace-fri-header-tied-base-value-alignment-gap-bound-from-head-value-mismatch-gap
  [OF head-mismatch])
have next-bound:
  wp-event verify-monad
  (trace-fri-header-tied-next-value-replay-gap ?s) ?s ≤ Merkle + 0
by (rule
  wp-trace-fri-header-tied-next-value-replay-gap-bound-from-merkle-and-slot)
  (rule merkle-bound[OF support],
  rule wp-trace-fri-header-tied-authenticated-slot-recorded-chunk-gap-zero)
have successor:
  wp-event verify-monad
  (trace-fri-header-tied-successor-replay-gap ?s) ?s ≤ Merkle + 0
by (rule
  wp-trace-fri-header-tied-successor-replay-gap-bound-from-merkle-and-slot)
  (rule merkle-bound[OF support],
  rule wp-trace-fri-header-tied-authenticated-slot-recorded-chunk-gap-zero)
have final:
  wp-event verify-monad

```

$(\text{trace-fri-header-tied-final-value-replay-gap } ?s) ?s$
 $\leq \text{Merkle} + 0 + 0$
by (rule
 $\text{wp-trace-fri-header-tied-final-value-replay-gap-bound-from-merkle-slot-and-selected-step}$)
(rule merkle-bound[OF support],
rule wp-trace-fri-header-tied-authenticated-slot-recorded-chunk-gap-zero,
rule wp-trace-fri-header-tied-final-value-selected-step-missing-gap-zero)
have slot:
wp-event verify-monad
 $(\text{trace-fri-header-tied-authenticated-slot-recorded-chunk-gap } ?s) ?s$
 ≤ 0
by (rule wp-trace-fri-header-tied-authenticated-slot-recorded-chunk-gap-zero)
have authenticated-recorded-auth:
wp-event verify-monad
 $(\text{trace-fri-header-tied-same-layer-authenticated-recorded-auth-gap } ?s)$
 $?s \leq 0$
by (rule
 $\text{wp-trace-fri-header-tied-same-layer-authenticated-recorded-auth-gap-bound-from-slot-recorded-chunk-gap}$
[OF slot])
have recorded-auth:
wp-event verify-monad
 $(\text{trace-fri-header-tied-same-layer-recorded-auth-gap } ?s) ?s \leq 0$
by (rule
 $\text{wp-trace-fri-header-tied-same-layer-recorded-auth-gap-bound-from-authenticated-recorded-auth-gap}$
[OF authenticated-recorded-auth])
have same-auth:
wp-event verify-monad
 $(\text{trace-fri-header-tied-same-layer-auth-gap } ?s) ?s \leq 0$
by (rule
 $\text{wp-trace-fri-header-tied-same-layer-auth-gap-bound-from-recorded-auth-gap}$
[OF recorded-auth])
have auth-gap:
wp-event verify-monad
 $(\text{trace-fri-header-tied-assignment-auth-gap } ?s) ?s \leq 0$
by (rule
 $\text{wp-trace-fri-header-tied-assignment-auth-gap-bound-from-same-layer-auth-gap}$
[OF same-auth])
have complement:
wp-event verify-monad
 $(\lambda out.$
 $\text{trace-fri-header-tied-sampled-assignment-conflict } ?s \text{ out} \wedge$
 $\neg \text{trace-fri-header-tied-recorded-sibling-candidate-missing } ?s \text{ out})$
 $?s \leq$
 $(((((\text{Merkle} + \text{Merkle}) + (\text{Merkle} + 0) + (\text{Merkle} + 0) +$
 $(\text{Merkle} + 0 + 0)) + \text{Merkle}) + 0)$
by (rule
 $\text{wp-trace-fri-header-tied-sampled-assignment-conflict-without-recorded-missing-bound-from-auth-gap-and-replay-}$
(rule auth-gap, rule merkle-bound[OF support], rule alignment,
rule next-bound, rule successor, rule final)

```

show wp-event verify-monad
  (trace-fri-header-tied-sampled-assignment-conflict ?s) ?s
  ≤ Missing +
    (((Merkle + Merkle) + (Merkle + 0) + (Merkle + 0) +
      (Merkle + 0 + 0)) + Merkle) + 0)
  by (rule
    wp-trace-fri-header-tied-sampled-assignment-conflict-bound-from-recorded-missing-and-complement)
    (rule missing-bound[OF support], rule complement)
next
  fix s
  show ¬ trace-fri-header-tied-sampled-assignment-conflict s None
    unfolding trace-fri-header-tied-sampled-assignment-conflict-def
      accepted-fri-opening-transcript-def
    by simp
qed
have zero:
  wp-event (checked-staged-security-experiment-with-data-state A)
    (staged-security-with-data-state-verifier-event
      trace-fri-header-tied-zero-round-final-obstruction)
    adversary-initial-state ≤ CheckedZero + Merkle
  by (rule
    checked-staged-security-trace-fri-header-tied-zero-round-bound-from-staged-checked-and-merkle)
    (rule checked-zero-bound, rule merkle-bound)
show ?thesis
  by (rule
    checked-staged-security-trace-fri-header-tied-partial-candidate-bound-from-sampled-conflict-and-staged-zero
      [OF sampled conflict zero])
qed

lemma checked-staged-security-trace-fri-header-tied-augmented-partial-candidate-bound-from-fresh-prequery-staged-ch
assumes wf: staged-budget-wellformed budgets
and controlled: staged-adversary-controlled budgets A
and finite-B: finite B
and cover:
  ∧ data attacker-state.
    Some (data, attacker-state) ∈
      set-dist (execute (checked-staged-transcript-program A)
        adversary-initial-state) ⇒
      trace-fri-sampled-layer-chain-cover
        (verifier-state-from-adversary attacker-state
          (staged-proof-transcript data)) bad
and envelope:
  ∧ data attacker-state trace-table.
    Some (data, attacker-state) ∈
      set-dist (execute (checked-staged-transcript-program A)
        adversary-initial-state) ⇒
      trace-fri-multi-round-bad-sets bad trace-table ⊆ B
and fresh-bound:
  wp-event (checked-staged-security-experiment-with-data-state A)

```

(staged-security-with-data-state-verifier-event
($\lambda s. \text{trace-fri-challenge-list-fresh-hit } s \ B)$)
adversary-initial-state $\leq$ Fresh
and *merkle-bound:*
 $\wedge$ *data attacker-state.*
Some (data, attacker-state) $\in$
set-dist (execute (checked-staged-transcript-program A)
adversary-initial-state) $\implies$
wp-event verify-monad
(partial-merkle-inconsistency-bad
(verifier-state-from-adversary attacker-state
(staged-proof-transcript data)))
(verifier-state-from-adversary attacker-state
(staged-proof-transcript data)) $\leq$ Merkle
and *missing-bound:*
 $\wedge$ *data attacker-state.*
Some (data, attacker-state) $\in$
set-dist (execute (checked-staged-transcript-program A)
adversary-initial-state) $\implies$
wp-event verify-monad
(trace-fri-header-tied-recorded-sibling-candidate-missing
(verifier-state-from-adversary attacker-state
(staged-proof-transcript data)))
(verifier-state-from-adversary attacker-state
(staged-proof-transcript data)) $\leq$ Missing
and *checked-zero-bound:*
wp-event (checked-staged-security-experiment-with-data-state A)
(staged-security-with-data-state-verifier-event
trace-fri-zero-round-checked-final-obstruction)
adversary-initial-state $\leq$ CheckedZero
shows
wp-event (checked-staged-security-experiment-with-data-state A)
(staged-security-with-data-state-verifier-event
trace-fri-bad-with-header-tied-augmented-partial-candidate)
adversary-initial-state $\leq$
(Fresh +
*hash-relation-budget-value (card B * ceil-log clength)*
(staged-attacker-query-budget budgets +
staged-challenge-query-budget)) +
((Missing +
(((((Merkle + Merkle) + (Merkle + 0) + (Merkle + 0) +
(Merkle + 0 + 0)) + Merkle) + 0)) +
(CheckedZero + Merkle))
proof –
have *sampled:*
wp-event (checked-staged-security-experiment-with-data-state A)
(staged-security-with-data-state-verifier-event
trace-fri-bad-with-header-tied-augmented-sampled-layer-chain)
adversary-initial-state $\leq$

```

Fresh +
  hash-relation-budget-value (card B * ceil-log clength)
  (staged-attacker-query-budget budgets +
   staged-challenge-query-budget)
by (rule
  checked-staged-security-trace-fri-header-tied-augmented-sampled-layer-chain-bound-from-fresh-and-prequery
  [OF wf controlled finite-B cover envelope fresh-bound])
have conflict:
  wp-event (checked-staged-security-experiment-with-data-state A)
  (staged-security-with-data-state-verifier-event
   trace-fri-header-tied-sampled-assignment-conflict)
  adversary-initial-state ≤
  Missing +
  (((Merkle + Merkle) + (Merkle + 0) + (Merkle + 0) +
   (Merkle + 0 + 0)) + Merkle) + 0)
proof (rule checked-staged-security-with-data-state-verifier-event-bound-from-cont)
fix data attacker-state
assume support:
  Some (data, attacker-state) ∈
  set-dist (execute (checked-staged-transcript-program A)
   adversary-initial-state)
let ?s =
  verifier-state-from-adversary attacker-state
  (staged-proof-transcript data)
have head-neq:
  wp-event verify-monad
  (trace-fri-header-tied-base-head-value-neq-gap ?s) ?s ≤ Merkle
by (rule
  wp-trace-fri-header-tied-base-head-value-neq-gap-bound-from-partial-merkle)
(rule merkle-bound[OF support])
have head-mismatch:
  wp-event verify-monad
  (trace-fri-header-tied-base-head-value-mismatch-gap ?s) ?s
  ≤ Merkle
by (rule
  wp-trace-fri-header-tied-base-head-value-mismatch-gap-bound-from-neq-gap
  [OF head-neq])
have alignment:
  wp-event verify-monad
  (trace-fri-header-tied-base-value-alignment-gap ?s) ?s ≤ Merkle
by (rule
  wp-trace-fri-header-tied-base-value-alignment-gap-bound-from-head-value-mismatch-gap
  [OF head-mismatch])
have next-bound:
  wp-event verify-monad
  (trace-fri-header-tied-next-value-replay-gap ?s) ?s ≤ Merkle + 0
by (rule
  wp-trace-fri-header-tied-next-value-replay-gap-bound-from-merkle-and-slot)
(rule merkle-bound[OF support]),

```



```

    rule wp-trace-fri-header-tied-authenticated-slot-recorded-chunk-gap-zero)
have successor:
  wp-event verify-monad
    (trace-fri-header-tied-successor-replay-gap ?s) ?s ≤ Merkle + 0
  by (rule
    wp-trace-fri-header-tied-successor-replay-gap-bound-from-merkle-and-slot)
    (rule merkle-bound[OF support],
    rule wp-trace-fri-header-tied-authenticated-slot-recorded-chunk-gap-zero)
have final:
  wp-event verify-monad
    (trace-fri-header-tied-final-value-replay-gap ?s) ?s
    ≤ Merkle + 0 + 0
  by (rule
    wp-trace-fri-header-tied-final-value-replay-gap-bound-from-merkle-slot-and-selected-step)
    (rule merkle-bound[OF support],
    rule wp-trace-fri-header-tied-authenticated-slot-recorded-chunk-gap-zero,
    rule wp-trace-fri-header-tied-final-value-selected-step-missing-gap-zero)
have slot:
  wp-event verify-monad
    (trace-fri-header-tied-authenticated-slot-recorded-chunk-gap ?s) ?s
    ≤ 0
  by (rule wp-trace-fri-header-tied-authenticated-slot-recorded-chunk-gap-zero)
have authenticated-recorded-auth:
  wp-event verify-monad
    (trace-fri-header-tied-same-layer-authenticated-recorded-auth-gap ?s)
    ?s ≤ 0
  by (rule
    wp-trace-fri-header-tied-same-layer-authenticated-recorded-auth-gap-bound-from-slot-recorded-chunk-gap
    [OF slot])
have recorded-auth:
  wp-event verify-monad
    (trace-fri-header-tied-same-layer-recorded-auth-gap ?s) ?s ≤ 0
  by (rule
    wp-trace-fri-header-tied-same-layer-recorded-auth-gap-bound-from-authenticated-recorded-auth-gap
    [OF authenticated-recorded-auth])
have same-auth:
  wp-event verify-monad
    (trace-fri-header-tied-same-layer-auth-gap ?s) ?s ≤ 0
  by (rule
    wp-trace-fri-header-tied-same-layer-auth-gap-bound-from-recorded-auth-gap
    [OF recorded-auth])
have auth-gap:
  wp-event verify-monad
    (trace-fri-header-tied-assignment-auth-gap ?s) ?s ≤ 0
  by (rule
    wp-trace-fri-header-tied-assignment-auth-gap-bound-from-same-layer-auth-gap
    [OF same-auth])
have complement:
  wp-event verify-monad

```

```

(λout.
  trace-fri-header-tied-sampled-assignment-conflict ?s out ∧
  ¬ trace-fri-header-tied-recorded-sibling-candidate-missing ?s out)
?s ≤
(((Merkle + Merkle) + (Merkle + 0) + (Merkle + 0) +
 (Merkle + 0 + 0)) + Merkle) + 0)
by (rule
  wp-trace-fri-header-tied-sampled-assignment-conflict-without-recorded-missing-bound-from-auth-gap-and-replay-
  (rule auth-gap, rule merkle-bound[OF support], rule alignment,
   rule next-bound, rule successor, rule final)
show wp-event verify-monad
  (trace-fri-header-tied-sampled-assignment-conflict ?s) ?s
  ≤ Missing +
    (((Merkle + Merkle) + (Merkle + 0) + (Merkle + 0) +
     (Merkle + 0 + 0)) + Merkle) + 0)
by (rule
  wp-trace-fri-header-tied-sampled-assignment-conflict-bound-from-recorded-missing-and-complement)
  (rule missing-bound[OF support], rule complement)
next
fix s
show ¬ trace-fri-header-tied-sampled-assignment-conflict s None
  unfolding trace-fri-header-tied-sampled-assignment-conflict-def
    accepted-fri-opening-transcript-def
  by simp
qed
have zero:
  wp-event (checked-staged-security-experiment-with-data-state A)
    (staged-security-with-data-state-verifier-event
      trace-fri-header-tied-zero-round-final-obstruction)
  adversary-initial-state ≤ CheckedZero + Merkle
by (rule
  checked-staged-security-trace-fri-header-tied-zero-round-bound-from-staged-checked-and-merkle)
  (rule checked-zero-bound, rule merkle-bound)
have wp-event (checked-staged-security-experiment-with-data-state A)
  (staged-security-with-data-state-verifier-event
    trace-fri-bad-with-header-tied-augmented-partial-candidate)
  adversary-initial-state ≤
  wp-event (checked-staged-security-experiment-with-data-state A)
    (λout.
      staged-security-with-data-state-verifier-event
        trace-fri-bad-with-header-tied-augmented-sampled-layer-chain out ∨
      staged-security-with-data-state-verifier-event
        trace-fri-header-tied-sampled-assignment-conflict out ∨
      staged-security-with-data-state-verifier-event
        trace-fri-header-tied-zero-round-final-obstruction out)
    adversary-initial-state
  unfolding staged-security-with-data-state-verifier-event-def
  by (rule wp-event-mono-on-support)
  (auto dest:

```

```

    trace-fri-header-tied-augmented-partial-candidate-imp-sampled-conflict-or-zero
    split: option.splits prod.splits)
also have ... ≤
  wp-event (checked-staged-security-experiment-with-data-state A)
    (staged-security-with-data-state-verifier-event
      trace-fri-bad-with-header-tied-augmented-sampled-layer-chain)
  adversary-initial-state +
  wp-event (checked-staged-security-experiment-with-data-state A)
    (λout.
      staged-security-with-data-state-verifier-event
        trace-fri-header-tied-sampled-assignment-conflict out ∨
      staged-security-with-data-state-verifier-event
        trace-fri-header-tied-zero-round-final-obstruction out)
  adversary-initial-state
  by (rule wp-event-union-bound)
also have ... ≤
  wp-event (checked-staged-security-experiment-with-data-state A)
    (staged-security-with-data-state-verifier-event
      trace-fri-bad-with-header-tied-augmented-sampled-layer-chain)
  adversary-initial-state +
  (wp-event (checked-staged-security-experiment-with-data-state A)
    (staged-security-with-data-state-verifier-event
      trace-fri-header-tied-sampled-assignment-conflict)
    adversary-initial-state +
  wp-event (checked-staged-security-experiment-with-data-state A)
    (staged-security-with-data-state-verifier-event
      trace-fri-header-tied-zero-round-final-obstruction)
    adversary-initial-state)
  by (intro add-mono order-refl wp-event-union-bound)
also have ... ≤
  (Fresh +
    hash-relation-budget-value (card B * ceil-log clength)
      (staged-attacker-query-budget budgets +
        staged-challenge-query-budget)) +
  ((Missing +
    (((Merkle + Merkle) + (Merkle + 0) + (Merkle + 0) +
      (Merkle + 0 + 0)) + Merkle) + 0)) +
  (CheckedZero + Merkle))
  by (intro add-mono sampled conflict zero)
finally show ?thesis .
qed

lemma checked-staged-security-trace-fri-header-tied-partial-candidate-bound-from-augmented-route-and-unaugmented
  assumes wf: staged-budget-wellformed budgets
  and controlled: staged-adversary-controlled budgets A
  and finite-B: finite B
  and cover:
    ∧ data attacker-state.
    Some (data, attacker-state) ∈

```

$set-dist \ (execute \ (checked-staged-transcript-program \ A) \ adversary-initial-state) \implies$
 $trace-fri-sampled-layer-chain-cover$
 $(verifier-state-from-adversary \ attacker-state \ (staged-proof-transcript \ data)) \ bad$
and *envelope*:
 $\wedge data \ attacker-state \ trace-table.$
 $Some \ (data, \ attacker-state) \in$
 $set-dist \ (execute \ (checked-staged-transcript-program \ A) \ adversary-initial-state) \implies$
 $trace-fri-multiround-bad-sets \ bad \ trace-table \subseteq B$
and *fresh-bound*:
 $wp-event \ (checked-staged-security-experiment-with-data-state \ A)$
 $(staged-security-with-data-state-verifier-event$
 $(\lambda s. \ trace-fri-challenge-list-fresh-hit \ s \ B))$
 $adversary-initial-state \leq Fresh$
and *merkle-bound*:
 $\wedge data \ attacker-state.$
 $Some \ (data, \ attacker-state) \in$
 $set-dist \ (execute \ (checked-staged-transcript-program \ A) \ adversary-initial-state) \implies$
 $wp-event \ verify-monad$
 $(partial-merkle-inconsistency-bad$
 $(verifier-state-from-adversary \ attacker-state$
 $(staged-proof-transcript \ data)))$
 $(verifier-state-from-adversary \ attacker-state$
 $(staged-proof-transcript \ data)) \leq Merkle$
and *missing-bound*:
 $\wedge data \ attacker-state.$
 $Some \ (data, \ attacker-state) \in$
 $set-dist \ (execute \ (checked-staged-transcript-program \ A) \ adversary-initial-state) \implies$
 $wp-event \ verify-monad$
 $(trace-fri-header-tied-recorded-sibling-candidate-missing$
 $(verifier-state-from-adversary \ attacker-state$
 $(staged-proof-transcript \ data)))$
 $(verifier-state-from-adversary \ attacker-state$
 $(staged-proof-transcript \ data)) \leq Missing$
and *checked-zero-bound*:
 $wp-event \ (checked-staged-security-experiment-with-data-state \ A)$
 $(staged-security-with-data-state-verifier-event$
 $trace-fri-zero-round-checked-final-obstruction)$
 $adversary-initial-state \leq CheckedZero$
shows
 $wp-event \ (checked-staged-security-experiment-with-data-state \ A)$
 $(staged-security-with-data-state-verifier-event$
 $trace-fri-bad-with-header-tied-partial-candidate)$
 $adversary-initial-state \leq$
 $((Fresh +$

```

    hash-relation-budget-value (card B * ceil-log clength)
    (staged-attacker-query-budget budgets +
     staged-challenge-query-budget)) +
  ((Missing +
   (((Merkle + Merkle) + (Merkle + 0) + (Merkle + 0) +
    (Merkle + 0 + 0)) + Merkle) + 0)) +
  (CheckedZero + Merkle))) +
  (Missing + Merkle)
proof -
have augmented:
  wp-event (checked-staged-security-experiment-with-data-state A)
  (staged-security-with-data-state-verifier-event
   trace-fri-bad-with-header-tied-augmented-partial-candidate)
  adversary-initial-state ≤
  (Fresh +
   hash-relation-budget-value (card B * ceil-log clength)
   (staged-attacker-query-budget budgets +
    staged-challenge-query-budget)) +
  ((Missing +
   (((Merkle + Merkle) + (Merkle + 0) + (Merkle + 0) +
    (Merkle + 0 + 0)) + Merkle) + 0)) +
  (CheckedZero + Merkle))
by (rule
  checked-staged-security-trace-fri-header-tied-augmented-partial-candidate-bound-from-fresh-prequery-staged-check
  [OF wf controlled finite-B cover envelope fresh-bound merkle-bound
   missing-bound checked-zero-bound])
have unaugmented:
  wp-event (checked-staged-security-experiment-with-data-state A)
  (staged-security-with-data-state-verifier-event
   trace-fri-header-tied-unaugmented-partial-candidate)
  adversary-initial-state ≤ Missing + Merkle
by (rule
  checked-staged-security-trace-fri-header-tied-unaugmented-partial-candidate-bound-from-recorded-missing-and-me
  (erule merkle-bound, erule missing-bound))
have wp-event (checked-staged-security-experiment-with-data-state A)
  (staged-security-with-data-state-verifier-event
   trace-fri-bad-with-header-tied-partial-candidate)
  adversary-initial-state ≤
  wp-event (checked-staged-security-experiment-with-data-state A)
  (λout.
   staged-security-with-data-state-verifier-event
   trace-fri-bad-with-header-tied-augmented-partial-candidate out ∨
   staged-security-with-data-state-verifier-event
   trace-fri-header-tied-unaugmented-partial-candidate out)
  adversary-initial-state
unfolding staged-security-with-data-state-verifier-event-def
by (rule wp-event-mono-on-support)
  (auto dest:
   trace-fri-bad-with-header-tied-partial-candidate-imp-augmented-or-unaugmented

```

```

    split: option.splits prod.splits)
also have ...  $\leq$ 
  wp-event (checked-staged-security-experiment-with-data-state A)
    (staged-security-with-data-state-verifier-event
      trace-fri-bad-with-header-tied-augmented-partial-candidate)
    adversary-initial-state +
  wp-event (checked-staged-security-experiment-with-data-state A)
    (staged-security-with-data-state-verifier-event
      trace-fri-header-tied-unaugmented-partial-candidate)
    adversary-initial-state
by (rule wp-event-union-bound)
also have ...  $\leq$ 
  ((Fresh +
    hash-relation-budget-value (card B * ceil-log clength)
    (staged-attacker-query-budget budgets +
      staged-challenge-query-budget)) +
    ((Missing +
      (((Merkle + Merkle) + (Merkle + 0) + (Merkle + 0) +
        (Merkle + 0 + 0)) + Merkle) + 0)) +
      (CheckedZero + Merkle))) +
    (Missing + Merkle)
by (rule add-mono[OF augmented unaugmented])
finally show ?thesis .
qed

```

lemma checked-staged-security-trace-fri-header-tied-partial-candidate-bound-from-fresh-prequery-checked-zero-and-wi

assumes wf: staged-budget-wellformed budgets

and controlled: staged-adversary-controlled budgets A

and finite-B: finite B

and cover:

$\bigwedge$ data attacker-state.

Some (data, attacker-state) $\in$
 set-dist (execute (checked-staged-transcript-program A)
 adversary-initial-state) $\implies$
 trace-fri-sampled-layer-chain-cover
 (verifier-state-from-adversary attacker-state
 (staged-proof-transcript data)) bad

and envelope:

$\bigwedge$ data attacker-state trace-table.

Some (data, attacker-state) $\in$
 set-dist (execute (checked-staged-transcript-program A)
 adversary-initial-state) $\implies$
 trace-fri-multi-round-bad-sets bad trace-table $\subseteq B$

and fresh-bound:

wp-event (checked-staged-security-experiment-with-data-state A)
 (staged-security-with-data-state-verifier-event
 (λs . trace-fri-challenge-list-fresh-hit s B))
 adversary-initial-state $\leq$ Fresh

and merkle-bound:

$\wedge$ data attacker-state.
 Some (data, attacker-state) $\in$
 set-dist (execute (checked-staged-transcript-program A)
 adversary-initial-state) $\implies$
 wp-event verify-monad
 (partial-merkle-inconsistency-bad
 (verifier-state-from-adversary attacker-state
 (staged-proof-transcript data)))
 (verifier-state-from-adversary attacker-state
 (staged-proof-transcript data)) $\leq$ Merkle
and structural-bound:
 $\wedge$ data attacker-state.
 Some (data, attacker-state) $\in$
 set-dist (execute (checked-staged-transcript-program A)
 adversary-initial-state) $\implies$
 wp-event verify-monad
 (trace-fri-header-tied-sibling-mismatch-structural-gap
 (verifier-state-from-adversary attacker-state
 (staged-proof-transcript data)))
 (verifier-state-from-adversary attacker-state
 (staged-proof-transcript data)) $\leq$ Structural
and checked-zero-bound:
 $\wedge$ data attacker-state.
 Some (data, attacker-state) $\in$
 set-dist (execute (checked-staged-transcript-program A)
 adversary-initial-state) $\implies$
 wp-event verify-monad
 (trace-fri-zero-round-checked-final-obstruction
 (verifier-state-from-adversary attacker-state
 (staged-proof-transcript data)))
 (verifier-state-from-adversary attacker-state
 (staged-proof-transcript data)) $\leq$ CheckedZero
shows
 wp-event (checked-staged-security-experiment-with-data-state A)
 (staged-security-with-data-state-verifier-event
 trace-fri-bad-with-header-tied-partial-candidate)
 adversary-initial-state $\leq$
 (Fresh +
 hash-relation-budget-value (card B * ceil-log clength)
 (staged-attacker-query-budget budgets +
 staged-challenge-query-budget)) +
 (((((Merkle + Structural) + Merkle) + (Merkle + 0) + (Merkle + 0) +
 (Merkle + 0 + 0)) + Merkle) + 0 + (CheckedZero + Merkle))
proof (rule
 checked-staged-security-trace-fri-header-tied-partial-candidate-bound-from-fresh-prequery-without-same-residuals
 [OF wf controlled finite-B cover envelope fresh-bound merkle-bound
 structural-bound])
fix data attacker-state
assume support:

Some (data, attacker-state) ∈
set-dist (execute (checked-staged-transcript-program A)
adversary-initial-state)
show *wp-event verify-monad*
(trace-fri-header-tied-zero-round-final-obstruction
(verifier-state-from-adversary attacker-state
(staged-proof-transcript data)))
(verifier-state-from-adversary attacker-state
(staged-proof-transcript data)) ≤ CheckedZero + Merkle
by *(rule wp-trace-fri-header-tied-zero-round-bound-from-checked-and-merkle)*
(rule checked-zero-bound[OF support],
rule merkle-bound[OF support])
qed

lemma *checked-staged-security-composition-fri-verifier-tied-partial-candidate-bound-from-fresh-prequery-without-sam*
assumes *wf: staged-budget-wellformed budgets*
and *controlled: staged-adversary-controlled budgets A*
and *finite-B: ∧ dg. finite (B dg)*
and *cover:*
∧ data attacker-state.
Some (data, attacker-state) ∈
set-dist (execute (checked-staged-transcript-program A)
adversary-initial-state) ⇒
composition-fri-sampled-layer-chain-cover
(verifier-state-from-adversary attacker-state
(staged-proof-transcript data)) bad
and *envelope:*
∧ data attacker-state dg composition-table.
Some (data, attacker-state) ∈
set-dist (execute (checked-staged-transcript-program A)
adversary-initial-state) ⇒
composition-fri-multiround-bad-sets bad dg composition-table
⊆ B dg
and *fresh-bound:*
wp-event (checked-staged-security-experiment-with-data-state A)
(staged-security-with-data-state-verifier-event
(λs. composition-fri-challenge-list-fresh-hit s B))
adversary-initial-state ≤ Fresh
and *merkle-bound:*
∧ data attacker-state.
Some (data, attacker-state) ∈
set-dist (execute (checked-staged-transcript-program A)
adversary-initial-state) ⇒
wp-event verify-monad
(partial-merkle-inconsistency-bad
(verifier-state-from-adversary attacker-state
(staged-proof-transcript data)))
(verifier-state-from-adversary attacker-state
(staged-proof-transcript data)) ≤ Merkle

and *missing-bound*:
 $\wedge$ *data attacker-state*.
Some (*data*, *attacker-state*) $\in$
set-dist (*execute* (*checked-staged-transcript-program* *A*)
adversary-initial-state) $\implies$
wp-event verify-monad
(*composition-fri-verifier-tied-recorded-base-chunk-auth-missing*
(*verifier-state-from-adversary attacker-state*
(*staged-proof-transcript data*)))
(*verifier-state-from-adversary attacker-state*
(*staged-proof-transcript data*)) $\leq$ *Missing*
and *without-same-bound*:
 $\wedge$ *data attacker-state*.
Some (*data*, *attacker-state*) $\in$
set-dist (*execute* (*checked-staged-transcript-program* *A*)
adversary-initial-state) $\implies$
wp-event verify-monad
(*composition-fri-verifier-tied-sampled-assignment-conflict-without-same-layer*
(*verifier-state-from-adversary attacker-state*
(*staged-proof-transcript data*)))
(*verifier-state-from-adversary attacker-state*
(*staged-proof-transcript data*)) $\leq$ *WithoutSame*
shows
wp-event (*checked-staged-security-experiment-with-data-state A*)
(*staged-security-with-data-state-verifier-event*
composition-fri-bad-with-verifier-tied-partial-candidate)
adversary-initial-state $\leq$
(*Fresh* +
hash-relation-budget-value
 $(\sum dg \in (UNIV :: 'f \text{ set}).$
 $\text{card } (B \ dg) * \text{ceil-log } (\text{maxDegree} + 1))$
(*staged-attacker-query-budget budgets* +
staged-challenge-query-budget)) +
 $((\text{Merkle} + \text{Missing}) + (\text{Merkle} + 0) + (\text{Merkle} + 0) + \text{WithoutSame} +$
 $\text{Merkle}) + 0 + 0)$
proof (*rule*
checked-staged-security-composition-fri-verifier-tied-partial-candidate-bound-from-fresh-prequery-and-residual-gaps
 $[OF \text{ wf controlled finite-}B \text{ cover envelope fresh-bound}]$)
fix *data attacker-state*
assume *support*:
Some (*data*, *attacker-state*) $\in$
set-dist (*execute* (*checked-staged-transcript-program* *A*)
adversary-initial-state)
let *?s* =
verifier-state-from-adversary attacker-state
(*staged-proof-transcript data*)
show *wp-event verify-monad*
(*composition-fri-verifier-tied-authenticated-slot-recorded-chunk-gap ?s*)
?s ≤ 0

by (rule
 wp-composition-fri-verifier-tied-authenticated-slot-recorded-chunk-gap-zero)
 show wp-event verify-monad (partial-merkle-inconsistency-bad ?s) ?s
 ≤ Merkle
 by (rule merkle-bound[OF support])
 show wp-event verify-monad
 (composition-fri-verifier-tied-recorded-base-chunk-auth-missing ?s) ?s
 ≤ Missing
 by (rule missing-bound[OF support])
 show wp-event verify-monad
 (composition-fri-verifier-tied-next-value-replay-gap ?s) ?s
 ≤ Merkle + 0
 by (rule
 wp-composition-fri-verifier-tied-next-value-replay-gap-bound-from-merkle-and-slot)
 (rule merkle-bound[OF support],
 rule
 wp-composition-fri-verifier-tied-authenticated-slot-recorded-chunk-gap-zero)
 show wp-event verify-monad
 (composition-fri-verifier-tied-successor-replay-gap ?s) ?s
 ≤ Merkle + 0
 by (rule
 wp-composition-fri-verifier-tied-successor-replay-gap-bound-from-merkle-and-slot)
 (rule merkle-bound[OF support],
 rule
 wp-composition-fri-verifier-tied-authenticated-slot-recorded-chunk-gap-zero)
 show wp-event verify-monad
 (composition-fri-verifier-tied-final-value-replay-gap ?s) ?s
 ≤ WithoutSame
 by (rule
 wp-composition-fri-verifier-tied-final-value-replay-gap-bound-from-without-same)
 (rule without-same-bound[OF support])
 show wp-event verify-monad
 (composition-fri-verifier-tied-zero-round-final-obstruction ?s) ?s
 ≤ 0
 by (subst wp-composition-fri-verifier-tied-zero-round-final-obstruction-zero)
 simp
 qed

lemma checked-staged-security-composition-fri-verifier-tied-partial-candidate-bound-from-fresh-prequery-without-miss

assumes wf: staged-budget-wellformed budgets

and controlled: staged-adversary-controlled budgets A

and finite-B: $\bigwedge dg. \text{finite } (B \ dg)$

and cover:

$\bigwedge \text{data attacker-state.}$

Some (data, attacker-state) ∈

set-dist (execute (checked-staged-transcript-program A)

adversary-initial-state) $\implies$

composition-fri-sampled-layer-chain-cover

(verifier-state-from-adversary attacker-state

(staged-proof-transcript data)) bad
and envelope:
 $\wedge$ data attacker-state dg composition-table.
 Some (data, attacker-state) $\in$
 set-dist (execute (checked-staged-transcript-program A)
 adversary-initial-state) $\implies$
 composition-fri-multiround-bad-sets bad dg composition-table
 $\subseteq B$ dg
and fresh-bound:
 wp-event (checked-staged-security-experiment-with-data-state A)
 (staged-security-with-data-state-verifier-event
 (λs . composition-fri-challenge-list-fresh-hit s B))
 adversary-initial-state $\leq$ Fresh
and merkle-bound:
 $\wedge$ data attacker-state.
 Some (data, attacker-state) $\in$
 set-dist (execute (checked-staged-transcript-program A)
 adversary-initial-state) $\implies$
 wp-event verify-monad
 (partial-merkle-inconsistency-bad
 (verifier-state-from-adversary attacker-state
 (staged-proof-transcript data)))
 (verifier-state-from-adversary attacker-state
 (staged-proof-transcript data)) $\leq$ Merkle
and without-same-bound:
 $\wedge$ data attacker-state.
 Some (data, attacker-state) $\in$
 set-dist (execute (checked-staged-transcript-program A)
 adversary-initial-state) $\implies$
 wp-event verify-monad
 (composition-fri-verifier-tied-sampled-assignment-conflict-without-same-layer
 (verifier-state-from-adversary attacker-state
 (staged-proof-transcript data)))
 (verifier-state-from-adversary attacker-state
 (staged-proof-transcript data)) $\leq$ WithoutSame
shows
 wp-event (checked-staged-security-experiment-with-data-state A)
 (staged-security-with-data-state-verifier-event
 composition-fri-bad-with-verifier-tied-partial-candidate)
 adversary-initial-state $\leq$
 (Fresh +
 hash-relation-budget-value
 ($\sum dg \in (UNIV :: 'f \text{ set}).$
 card (B dg) * ceil-log (maxDegree + 1))
 (staged-attacker-query-budget budgets +
 staged-challenge-query-budget)) +
 (((Merkle + 0) + (Merkle + 0) + (Merkle + 0) + WithoutSame +
 Merkle) + 0 + 0)
proof (rule

checked-staged-security-composition-fri-verifier-tied-partial-candidate-bound-from-fresh-prequery-without-same-resid
 [where *Missing* = 0])
show *staged-budget-wellformed budgets*
 by (rule *wf*)
show *staged-adversary-controlled budgets A*
 by (rule *controlled*)
show $\bigwedge dg. \text{finite } (B \ dg)$
 by (rule *finite-B*)
show $\bigwedge \text{data attacker-state.}$
 Some (data, attacker-state)
 $\in \text{set-dist}$
 (*execute (checked-staged-transcript-program A)*
 adversary-initial-state) $\implies$
 composition-fri-sampled-layer-chain-cover
 (*verifier-state-from-adversary attacker-state*
 (*staged-proof-transcript data*)) *bad*
 by (rule *cover*)
show $\bigwedge \text{data attacker-state dg composition-table.}$
 Some (data, attacker-state)
 $\in \text{set-dist}$
 (*execute (checked-staged-transcript-program A)*
 adversary-initial-state) $\implies$
 composition-fri-multiround-bad-sets bad dg composition-table
 $\subseteq B \ dg$
 by (rule *envelope*)
show *wp-event (checked-staged-security-experiment-with-data-state A)*
 (*staged-security-with-data-state-verifier-event*
 ($\lambda s. \text{composition-fri-challenge-list-fresh-hit } s \ B$))
 adversary-initial-state $\leq \text{Fresh}$
 by (rule *fresh-bound*)
show $\bigwedge \text{data attacker-state.}$
 Some (data, attacker-state)
 $\in \text{set-dist}$
 (*execute (checked-staged-transcript-program A)*
 adversary-initial-state) $\implies$
 wp-event verify-monad
 (*partial-merkle-inconsistency-bad*
 (*verifier-state-from-adversary attacker-state*
 (*staged-proof-transcript data*)))
 (*verifier-state-from-adversary attacker-state*
 (*staged-proof-transcript data*)) $\leq \text{Merkle}$
 by (rule *merkle-bound*)
show $\bigwedge \text{data attacker-state.}$
 Some (data, attacker-state)
 $\in \text{set-dist}$
 (*execute (checked-staged-transcript-program A)*
 adversary-initial-state) $\implies$
 wp-event verify-monad
 (*composition-fri-verifier-tied-recorded-base-chunk-auth-missing*

$(\text{verifier-state-from-adversary attacker-state}$
 $(\text{staged-proof-transcript data}))$
 $(\text{verifier-state-from-adversary attacker-state}$
 $(\text{staged-proof-transcript data})) \leq 0$
by (*subst wp-composition-fri-verifier-tied-recorded-base-chunk-auth-missing-zero*)
simp
show $\bigwedge \text{data attacker-state.}$
Some (*data, attacker-state*)
 $\in \text{set-dist}$
 $(\text{execute } (\text{checked-staged-transcript-program } A)$
 $\text{adversary-initial-state}) \implies$
wp-event verify-monad
 $(\text{composition-fri-verifier-tied-sampled-assignment-conflict-without-same-layer}$
 $(\text{verifier-state-from-adversary attacker-state}$
 $(\text{staged-proof-transcript data}))$
 $(\text{verifier-state-from-adversary attacker-state}$
 $(\text{staged-proof-transcript data})) \leq \text{WithoutSame}$
by (*rule without-same-bound*)
qed

lemma *checked-staged-security-composition-fri-verifier-tied-partial-candidate-bound-from-fresh-prequery-zero-slot-mis*

assumes *wf: staged-budget-wellformed budgets*

and *controlled: staged-adversary-controlled budgets A*

and *finite-B: $\bigwedge dg. \text{finite } (B dg)$*

and *cover:*

$\bigwedge \text{data attacker-state.}$

Some (*data, attacker-state*) $\in$
 $\text{set-dist } (\text{execute } (\text{checked-staged-transcript-program } A)$
 $\text{adversary-initial-state}) \implies$

composition-fri-sampled-layer-chain-cover
 $(\text{verifier-state-from-adversary attacker-state}$
 $(\text{staged-proof-transcript data})) \text{ bad}$

and *envelope:*

$\bigwedge \text{data attacker-state dg composition-table.}$

Some (*data, attacker-state*) $\in$
 $\text{set-dist } (\text{execute } (\text{checked-staged-transcript-program } A)$
 $\text{adversary-initial-state}) \implies$
 $\text{composition-fri-multiround-bad-sets bad dg composition-table}$
 $\subseteq B dg$

and *fresh-bound:*

wp-event (*checked-staged-security-experiment-with-data-state A*)
 $(\text{staged-security-with-data-state-verifier-event}$
 $(\lambda s. \text{composition-fri-challenge-list-fresh-hit } s B))$
 $\text{adversary-initial-state} \leq \text{Fresh}$

and *merkle-bound:*

$\bigwedge \text{data attacker-state.}$

Some (*data, attacker-state*) $\in$
 $\text{set-dist } (\text{execute } (\text{checked-staged-transcript-program } A)$
 $\text{adversary-initial-state}) \implies$

$wp\text{-event } verify\text{-monad}$
 $(partial\text{-merkle-inconsistency-bad}$
 $(verify\text{-state-from-adversary } attacker\text{-state}$
 $(staged\text{-proof-transcript } data)))$
 $(verify\text{-state-from-adversary } attacker\text{-state}$
 $(staged\text{-proof-transcript } data)) \leq Merkle$
and $next\text{-bound}$:
 $\wedge data \text{ attacker-state.}$
 $Some (data, attacker\text{-state}) \in$
 $set\text{-dist } (execute (checked\text{-staged-transcript-program } A)$
 $adversary\text{-initial-state}) \implies$
 $wp\text{-event } verify\text{-monad}$
 $(composition\text{-fri-verifier-tied-next-value-replay-gap}$
 $(verify\text{-state-from-adversary } attacker\text{-state}$
 $(staged\text{-proof-transcript } data)))$
 $(verify\text{-state-from-adversary } attacker\text{-state}$
 $(staged\text{-proof-transcript } data)) \leq Next$
and $successor\text{-bound}$:
 $\wedge data \text{ attacker-state.}$
 $Some (data, attacker\text{-state}) \in$
 $set\text{-dist } (execute (checked\text{-staged-transcript-program } A)$
 $adversary\text{-initial-state}) \implies$
 $wp\text{-event } verify\text{-monad}$
 $(composition\text{-fri-verifier-tied-successor-replay-gap}$
 $(verify\text{-state-from-adversary } attacker\text{-state}$
 $(staged\text{-proof-transcript } data)))$
 $(verify\text{-state-from-adversary } attacker\text{-state}$
 $(staged\text{-proof-transcript } data)) \leq Successor$
and $final\text{-bound}$:
 $\wedge data \text{ attacker-state.}$
 $Some (data, attacker\text{-state}) \in$
 $set\text{-dist } (execute (checked\text{-staged-transcript-program } A)$
 $adversary\text{-initial-state}) \implies$
 $wp\text{-event } verify\text{-monad}$
 $(composition\text{-fri-verifier-tied-final-value-replay-gap}$
 $(verify\text{-state-from-adversary } attacker\text{-state}$
 $(staged\text{-proof-transcript } data)))$
 $(verify\text{-state-from-adversary } attacker\text{-state}$
 $(staged\text{-proof-transcript } data)) \leq Final$
shows
 $wp\text{-event } (checked\text{-staged-security-experiment-with-data-state } A)$
 $(staged\text{-security-with-data-state-verifier-event}$
 $composition\text{-fri-bad-with-verifier-tied-partial-candidate})$
 $adversary\text{-initial-state} \leq$
 $(Fresh +$
 $hash\text{-relation-budget-value}$
 $(\sum dg \in (UNIV :: 'f \text{ set}).$
 $card (B \ dg) * \text{ceil-log } (maxDegree + 1))$
 $(staged\text{-attacker-query-budget } budgets +$

$$\text{staged-challenge-query-budget})) +$$

$$(((\text{Merkle} + 0) + \text{Next} + \text{Successor} + \text{Final} + \text{Merkle}) + 0 + 0)$$

proof (rule
checked-staged-security-composition-fri-verifier-tied-partial-candidate-bound-from-fresh-prequery-and-residual-gaps
 [where Slot = 0 and Missing = 0 and Zero = 0])
show staged-budget-wellformed budgets
 by (rule wf)
show staged-adversary-controlled budgets A
 by (rule controlled)
show $\bigwedge dg. \text{finite } (B \ dg)$
 by (rule finite-B)
show $\bigwedge \text{data attacker-state.}$
 Some (data, attacker-state)
 $\in \text{set-dist}$
 (execute (checked-staged-transcript-program A)
 adversary-initial-state) $\implies$
 composition-fri-sampled-layer-chain-cover
 (verifier-state-from-adversary attacker-state
 (staged-proof-transcript data)) bad
 by (rule cover)
show $\bigwedge \text{data attacker-state dg composition-table.}$
 Some (data, attacker-state)
 $\in \text{set-dist}$
 (execute (checked-staged-transcript-program A)
 adversary-initial-state) $\implies$
 composition-fri-multiround-bad-sets bad dg composition-table
 $\subseteq B \ dg$
 by (rule envelope)
show wp-event (checked-staged-security-experiment-with-data-state A)
 (staged-security-with-data-state-verifier-event
 ($\lambda s. \text{composition-fri-challenge-list-fresh-hit } s \ B$))
 adversary-initial-state $\leq \text{Fresh}$
 by (rule fresh-bound)
show $\bigwedge \text{data attacker-state.}$
 Some (data, attacker-state)
 $\in \text{set-dist}$
 (execute (checked-staged-transcript-program A)
 adversary-initial-state) $\implies$
 wp-event verify-monad
 (composition-fri-verifier-tied-authenticated-slot-recorded-chunk-gap
 (verifier-state-from-adversary attacker-state
 (staged-proof-transcript data)))
 (verifier-state-from-adversary attacker-state
 (staged-proof-transcript data)) ≤ 0
 by (rule
 wp-composition-fri-verifier-tied-authenticated-slot-recorded-chunk-gap-zero)
show $\bigwedge \text{data attacker-state.}$
 Some (data, attacker-state)
 $\in \text{set-dist}$

```

      (execute (checked-staged-transcript-program A)
        adversary-initial-state)  $\implies$ 
wp-event verify-monad
  (partial-merkle-inconsistency-bad
    (verifier-state-from-adversary attacker-state
      (staged-proof-transcript data)))
  (verifier-state-from-adversary attacker-state
    (staged-proof-transcript data))  $\leq$  Merkle
by (rule merkle-bound)
show  $\bigwedge$  data attacker-state.
  Some (data, attacker-state)
 $\in$  set-dist
  (execute (checked-staged-transcript-program A)
    adversary-initial-state)  $\implies$ 
wp-event verify-monad
  (composition-fri-verifier-tied-recorded-base-chunk-auth-missing
    (verifier-state-from-adversary attacker-state
      (staged-proof-transcript data)))
  (verifier-state-from-adversary attacker-state
    (staged-proof-transcript data))  $\leq$  0
by (subst wp-composition-fri-verifier-tied-recorded-base-chunk-auth-missing-zero)
simp
show  $\bigwedge$  data attacker-state.
  Some (data, attacker-state)
 $\in$  set-dist
  (execute (checked-staged-transcript-program A)
    adversary-initial-state)  $\implies$ 
wp-event verify-monad
  (composition-fri-verifier-tied-next-value-replay-gap
    (verifier-state-from-adversary attacker-state
      (staged-proof-transcript data)))
  (verifier-state-from-adversary attacker-state
    (staged-proof-transcript data))  $\leq$  Next
by (rule next-bound)
show  $\bigwedge$  data attacker-state.
  Some (data, attacker-state)
 $\in$  set-dist
  (execute (checked-staged-transcript-program A)
    adversary-initial-state)  $\implies$ 
wp-event verify-monad
  (composition-fri-verifier-tied-successor-replay-gap
    (verifier-state-from-adversary attacker-state
      (staged-proof-transcript data)))
  (verifier-state-from-adversary attacker-state
    (staged-proof-transcript data))  $\leq$  Successor
by (rule successor-bound)
show  $\bigwedge$  data attacker-state.
  Some (data, attacker-state)
 $\in$  set-dist

```


$(\text{execute } (\text{checked-staged-transcript-program } A) \text{ adversary-initial-state}) \implies$
 $\text{wp-event verify-monad}$
 $(\text{composition-fri-verifier-tied-final-value-replay-gap}$
 $(\text{verifier-state-from-adversary attacker-state}$
 $(\text{staged-proof-transcript data})))$
 $(\text{verifier-state-from-adversary attacker-state}$
 $(\text{staged-proof-transcript data})) \leq \text{Final}$
by (rule final-bound)
show $\bigwedge \text{data attacker-state.}$
 $\text{Some } (data, \text{attacker-state})$
 $\in \text{set-dist}$
 $(\text{execute } (\text{checked-staged-transcript-program } A) \text{ adversary-initial-state}) \implies$
 $\text{wp-event verify-monad}$
 $(\text{composition-fri-verifier-tied-zero-round-final-obstruction}$
 $(\text{verifier-state-from-adversary attacker-state}$
 $(\text{staged-proof-transcript data})))$
 $(\text{verifier-state-from-adversary attacker-state}$
 $(\text{staged-proof-transcript data})) \leq 0$
by (subst wp-composition-fri-verifier-tied-zero-round-final-obstruction-zero)
 simp
qed

lemma *checked-staged-security-composition-fri-verifier-tied-partial-candidate-bound-from-fresh-prequery-zero-slot-mis*

assumes *wf: staged-budget-wellformed budgets*

and *controlled: staged-adversary-controlled budgets A*

and *finite-B: $\bigwedge dg. \text{finite } (B \ dg)$*

and *cover:*

$\bigwedge \text{data attacker-state.}$

$\text{Some } (data, \text{attacker-state}) \in$
 $\text{set-dist } (\text{execute } (\text{checked-staged-transcript-program } A) \text{ adversary-initial-state}) \implies$

$\text{composition-fri-sampled-layer-chain-cover}$
 $(\text{verifier-state-from-adversary attacker-state}$
 $(\text{staged-proof-transcript data})) \text{ bad}$

and *envelope:*

$\bigwedge \text{data attacker-state dg composition-table.}$

$\text{Some } (data, \text{attacker-state}) \in$
 $\text{set-dist } (\text{execute } (\text{checked-staged-transcript-program } A) \text{ adversary-initial-state}) \implies$
 $\text{composition-fri-multiround-bad-sets bad dg composition-table}$
 $\subseteq B \ dg$

and *fresh-bound:*

$\text{wp-event } (\text{checked-staged-security-experiment-with-data-state } A)$
 $(\text{staged-security-with-data-state-verifier-event}$
 $(\lambda s. \text{composition-fri-challenge-list-fresh-hit } s \ B))$
 $\text{adversary-initial-state} \leq \text{Fresh}$

and *merkle-bound:*

$\wedge \text{data attacker-state.}$
Some (data, attacker-state) $\in$
set-dist (execute (checked-staged-transcript-program A)
adversary-initial-state) $\implies$
wp-event verify-monad
(partial-merkle-inconsistency-bad
(verifier-state-from-adversary attacker-state
(staged-proof-transcript data)))
(verifier-state-from-adversary attacker-state
(staged-proof-transcript data)) $\leq$ Merkle
shows
wp-event (checked-staged-security-experiment-with-data-state A)
(staged-security-with-data-state-verifier-event
composition-fri-bad-with-verifier-tied-partial-candidate)
adversary-initial-state $\leq$
(Fresh +
hash-relation-budget-value
($\sum dg \in (\text{UNIV} :: 'f \text{ set}).$
*card (B dg) * ceil-log (maxDegree + 1))*
(staged-attacker-query-budget budgets +
staged-challenge-query-budget)) +
((Merkle + 0) + (Merkle + 0) + (Merkle + 0) + (Merkle + 0 + 0) +
Merkle) + 0 + 0)
proof (rule
checked-staged-security-composition-fri-verifier-tied-partial-candidate-bound-from-fresh-prequery-zero-slot-missing-a
show *staged-budget-wellformed budgets*
by (rule wf)
show *staged-adversary-controlled budgets A*
by (rule controlled)
show $\wedge dg. \text{finite } (B \ dg)$
by (rule finite-B)
show $\wedge \text{data attacker-state.}$
Some (data, attacker-state)
 $\in$ *set-dist*
(execute (checked-staged-transcript-program A)
adversary-initial-state) $\implies$
composition-fri-sampled-layer-chain-cover
(verifier-state-from-adversary attacker-state
(staged-proof-transcript data)) bad
by (rule cover)
show $\wedge \text{data attacker-state dg composition-table.}$
Some (data, attacker-state)
 $\in$ *set-dist*
(execute (checked-staged-transcript-program A)
adversary-initial-state) $\implies$
composition-fri-multiround-bad-sets bad dg composition-table
 $\subseteq B \ dg$
by (rule envelope)
show *wp-event (checked-staged-security-experiment-with-data-state A)*

```

    (staged-security-with-data-state-verifier-event
     (λs. composition-fri-challenge-list-fresh-hit s B))
    adversary-initial-state ≤ Fresh
  by (rule fresh-bound)
show ∧data attacker-state.
  Some (data, attacker-state)
  ∈ set-dist
    (execute (checked-staged-transcript-program A)
             adversary-initial-state) ⇒
  wp-event verify-monad
    (partial-merkle-inconsistency-bad
     (verifier-state-from-adversary attacker-state
      (staged-proof-transcript data)))
    (verifier-state-from-adversary attacker-state
     (staged-proof-transcript data)) ≤ Merkle
  by (rule merkle-bound)
show ∧data attacker-state.
  Some (data, attacker-state)
  ∈ set-dist
    (execute (checked-staged-transcript-program A)
             adversary-initial-state) ⇒
  wp-event verify-monad
    (composition-fri-verifier-tied-next-value-replay-gap
     (verifier-state-from-adversary attacker-state
      (staged-proof-transcript data)))
    (verifier-state-from-adversary attacker-state
     (staged-proof-transcript data)) ≤ Merkle + 0
proof -
  fix data attacker-state
  assume support:
    Some (data, attacker-state)
    ∈ set-dist
      (execute (checked-staged-transcript-program A)
               adversary-initial-state)
  let ?s =
    verifier-state-from-adversary attacker-state
    (staged-proof-transcript data)
  show wp-event verify-monad
    (composition-fri-verifier-tied-next-value-replay-gap ?s) ?s
    ≤ Merkle + 0
  by (rule
      wp-composition-fri-verifier-tied-next-value-replay-gap-bound-from-merkle-and-slot)
  (rule merkle-bound[OF support],
   rule
    wp-composition-fri-verifier-tied-authenticated-slot-recorded-chunk-gap-zero)
qed
show ∧data attacker-state.
  Some (data, attacker-state)
  ∈ set-dist

```

```

      (execute (checked-staged-transcript-program A)
        adversary-initial-state)  $\implies$ 
wp-event verify-monad
  (composition-fri-verifier-tied-successor-replay-gap
    (verifier-state-from-adversary attacker-state
      (staged-proof-transcript data)))
  (verifier-state-from-adversary attacker-state
    (staged-proof-transcript data))  $\leq$  Merkle + 0
proof –
  fix data attacker-state
  assume support:
    Some (data, attacker-state)
     $\in$  set-dist
      (execute (checked-staged-transcript-program A)
        adversary-initial-state)
  let ?s =
    verifier-state-from-adversary attacker-state
      (staged-proof-transcript data)
  show wp-event verify-monad
    (composition-fri-verifier-tied-successor-replay-gap ?s) ?s
     $\leq$  Merkle + 0
  by (rule
    wp-composition-fri-verifier-tied-successor-replay-gap-bound-from-merkle-and-slot)
    (rule merkle-bound[OF support],
    rule
      wp-composition-fri-verifier-tied-authenticated-slot-recorded-chunk-gap-zero)
qed
show  $\bigwedge$ data attacker-state.
  Some (data, attacker-state)
   $\in$  set-dist
    (execute (checked-staged-transcript-program A)
      adversary-initial-state)  $\implies$ 
wp-event verify-monad
  (composition-fri-verifier-tied-final-value-replay-gap
    (verifier-state-from-adversary attacker-state
      (staged-proof-transcript data)))
  (verifier-state-from-adversary attacker-state
    (staged-proof-transcript data))  $\leq$  Merkle + 0 + 0
proof –
  fix data attacker-state
  assume support:
    Some (data, attacker-state)
     $\in$  set-dist
      (execute (checked-staged-transcript-program A)
        adversary-initial-state)
  let ?s =
    verifier-state-from-adversary attacker-state
      (staged-proof-transcript data)
  show wp-event verify-monad

```

```

    (composition-fri-verifier-tied-final-value-replay-gap ?s) ?s
    ≤ Merkle + 0 + 0
  by (rule
    wp-composition-fri-verifier-tied-final-value-replay-gap-bound-from-merkle-slot-and-selected-step)
    (rule merkle-bound[OF support],
    rule
      wp-composition-fri-verifier-tied-authenticated-slot-recorded-chunk-gap-zero,
    rule
      wp-composition-fri-verifier-tied-final-value-selected-step-missing-gap-zero)
  qed
qed

lemma active-composition-fri-verifier-tied-reductions-from-sampled-and-merkle-bounds:
fixes Sampled Merkle ::
  'f staged-proof-data ⇒ 'f protocol-channel ⇒ prob
assumes sampled-bound:
  ∧data attacker-state.
  Some (data, attacker-state) ∈
    set-dist (execute (checked-staged-transcript-program A)
      adversary-initial-state) ⇒
  wp-event verify-monad
    (composition-fri-bad-with-verifier-tied-sampled-layer-chain
      (verifier-state-from-adversary attacker-state
        (staged-proof-transcript data)))
    (verifier-state-from-adversary attacker-state
      (staged-proof-transcript data)) ≤ Sampled data attacker-state
and merkle-bound:
  ∧data attacker-state.
  Some (data, attacker-state) ∈
    set-dist (execute (checked-staged-transcript-program A)
      adversary-initial-state) ⇒
  wp-event verify-monad
    (partial-merkle-inconsistency-bad
      (verifier-state-from-adversary attacker-state
        (staged-proof-transcript data)))
    (verifier-state-from-adversary attacker-state
      (staged-proof-transcript data)) ≤ Merkle data attacker-state
and total-bound:
  ∧data attacker-state.
  Some (data, attacker-state) ∈
    set-dist (execute (checked-staged-transcript-program A)
      adversary-initial-state) ⇒
  Sampled data attacker-state +
    (((Merkle data attacker-state + 0) +
      (Merkle data attacker-state + 0) +
      (Merkle data attacker-state + 0) +
      (Merkle data attacker-state + 0 + 0) +
      Merkle data attacker-state) + 0)
  ≤ composition-fri-error

```

and *support*:
Some (data, attacker-state) ∈
set-dist (execute (checked-staged-transcript-program A)
adversary-initial-state)
shows
composition-fri-verifier-tied-reduction
(verifier-state-from-adversary attacker-state
(staged-proof-transcript data))
proof –
let *?s* =
verifier-state-from-adversary attacker-state
(staged-proof-transcript data)
have *slot*:
wp-event verify-monad
(composition-fri-verifier-tied-authenticated-slot-recorded-chunk-gap
?s) ?s ≤ 0
by (rule
wp-composition-fri-verifier-tied-authenticated-slot-recorded-chunk-gap-zero)
have *next-gap*:
wp-event verify-monad
(composition-fri-verifier-tied-next-value-replay-gap ?s) ?s
≤ Merkle data attacker-state + 0
by (rule
wp-composition-fri-verifier-tied-next-value-replay-gap-bound-from-merkle-and-slot)
(rule merkle-bound[OF support], rule slot)
have *successor*:
wp-event verify-monad
(composition-fri-verifier-tied-successor-replay-gap ?s) ?s
≤ Merkle data attacker-state + 0
by (rule
wp-composition-fri-verifier-tied-successor-replay-gap-bound-from-merkle-and-slot)
(rule merkle-bound[OF support], rule slot)
have *final*:
wp-event verify-monad
(composition-fri-verifier-tied-final-value-replay-gap ?s) ?s
≤ Merkle data attacker-state + 0 + 0
by (rule
wp-composition-fri-verifier-tied-final-value-replay-gap-bound-from-merkle-slot-and-selected-step)
(rule merkle-bound[OF support], rule slot,
rule
wp-composition-fri-verifier-tied-final-value-selected-step-missing-gap-zero)
show *?thesis*
by (rule
composition-fri-verifier-tied-reduction-from-residual-bounds-with-recorded-missing-zero
[OF sampled-bound[OF support] slot merkle-bound[OF support]
next-gap successor final total-bound[OF support]])
qed
end

end

theory *Soundness-FRI-Full-Cover-Active-Bounds*

imports

Soundness-FRI-Obligation-Normalization

Soundness-FRI-Full-Cover-Domains

begin

Compact active-route bridges for the full-cover FRI split.

The imported layers already prove:

- a full-cover challenge-list/envelope bound;
- a missing-full-cover domain split;
- active trace/composition residual adapters.

This theory only packages those ingredients in the shape consumed by the same-run public soundness route. It does not add protocol assumptions or change the FRI model.

context *soundness*

begin

lemma *wp-trace-fri-header-tied-sampled-layer-chain-bound-from-envelope-domain-split:*

assumes *future: trace-fri-future-fresh s*

and *envelope:*

$\bigwedge \text{trace-table. trace-fri-multiround-bad-sets bad trace-table} \subseteq B$

and *subset: $B \subseteq \text{fri-challenge-space } (\text{ceil-log clength})$*

and *challenge-bound:*

$\text{nnreal } (\text{card } B) / \text{nnreal } (\text{CARD}(f) \wedge \text{ceil-log clength}) \leq R$

and *proximity:*

wp-event verify-monad

$(\text{trace-fri-sampled-full-cover-proximity-failure } s \text{ bad}) s \leq P$

and *index:*

wp-event verify-monad

$(\text{trace-fri-sampled-full-cover-index-failure } s) s \leq I$

and *domain-length:*

wp-event verify-monad

$(\text{trace-fri-sampled-full-cover-domain-length-failure } s) s \leq DL$

and *sampled-domain:*

wp-event verify-monad

$(\text{trace-fri-sampled-full-cover-sampled-domain-failure } s) s \leq SD$

shows

wp-event verify-monad

$(\text{trace-fri-bad-with-header-tied-sampled-layer-chain } s) s$

$\leq R + (P + I + (I + DL) + (I + SD))$

proof (*rule*

wp-trace-fri-header-tied-sampled-layer-chain-bound-from-full-cover-and-missing)

show *wp-event verify-monad*

$(\text{trace-fri-bad-with-full-cover-layer-chain } s \text{ bad}) \ s \leq R$
by (rule *wp-trace-fri-bad-with-full-cover-layer-chain-bound-from-envelope*
 $[\text{OF future envelope subset challenge-bound}]$)
show *wp-event verify-monad*
 $(\text{trace-fri-sampled-missing-full-cover } s \text{ bad}) \ s$
 $\leq P + I + (I + DL) + (I + SD)$
by (rule *wp-trace-fri-sampled-missing-full-cover-bound-from-domain-split*
 $[\text{OF proximity index domain-length sampled-domain}]$)
qed

lemma *wp-composition-fri-verifier-tied-sampled-layer-chain-bound-from-envelope-domain-split*:
assumes *future: composition-fri-future-fresh s*
and *envelope*:
 $\bigwedge dg \text{ composition-table.}$
 $\text{composition-fri-multi-round-bad-sets bad dg composition-table}$
 $\subseteq B \ dg$
and *subset*:
 $\bigwedge dg. B \ dg \subseteq \text{fri-challenge-space } (\text{ceil-log } (to\text{-nat } dg + 1))$
and *challenge-bound*:
 $\bigwedge dg. \text{nnreal } (\text{card } (B \ dg)) /$
 $\text{nnreal } (\text{CARD}(f) \wedge \text{ceil-log } (to\text{-nat } dg + 1)) \leq R$
and *proximity*:
wp-event verify-monad
 $(\text{composition-fri-sampled-full-cover-proximity-failure } s \text{ bad}) \ s$
 $\leq P$
and *index*:
wp-event verify-monad
 $(\text{composition-fri-sampled-full-cover-index-failure } s) \ s \leq I$
and *domain-length*:
wp-event verify-monad
 $(\text{composition-fri-sampled-full-cover-domain-length-failure } s) \ s$
 $\leq DL$
and *sampled-domain*:
wp-event verify-monad
 $(\text{composition-fri-sampled-full-cover-sampled-domain-failure } s) \ s$
 $\leq SD$
shows
wp-event verify-monad
 $(\text{composition-fri-bad-with-verifier-tied-sampled-layer-chain } s) \ s$
 $\leq R + (P + I + (I + DL) + (I + SD))$
proof (rule
wp-composition-fri-verifier-tied-sampled-layer-chain-bound-from-full-cover-and-missing)
show *wp-event verify-monad*
 $(\text{composition-fri-bad-with-full-cover-layer-chain } s \text{ bad}) \ s \leq R$
by (rule
wp-composition-fri-bad-with-full-cover-layer-chain-bound-from-envelope
 $[\text{OF future envelope subset challenge-bound}]$)
show *wp-event verify-monad*
 $(\text{composition-fri-sampled-missing-full-cover } s \text{ bad}) \ s$

$\leq P + I + (I + DL) + (I + SD)$
by (rule
wp-composition-fri-sampled-missing-full-cover-bound-from-domain-split
 $[OF \text{ proximity index domain-length sampled-domain}]$)
qed

lemma *wp-trace-fri-header-tied-sampled-layer-chain-bound-from-envelope-witness-domain:*
assumes *future: trace-fri-future-fresh s*
and *envelope:*
 $\bigwedge \text{trace-table. trace-fri-multiround-bad-sets bad trace-table} \subseteq B$
and *subset: $B \subseteq \text{fri-challenge-space } (\text{ceil-log clength})$*
and *challenge-bound:*
 $\text{nnreal } (\text{card } B) / \text{nnreal } (CARD('f) \wedge \text{ceil-log clength}) \leq R$
and *proximity:*
wp-event verify-monad
 $(\text{trace-fri-sampled-full-cover-proximity-failure } s \text{ bad}) \ s \leq P$
and *index:*
wp-event verify-monad
 $(\text{trace-fri-sampled-full-cover-index-failure } s) \ s \leq I$
and *witness-length:*
wp-event verify-monad
 $(\text{trace-fri-sampled-full-cover-witness-length-failure } s) \ s \leq W$
and *domain:*
wp-event verify-monad
 $(\text{trace-fri-sampled-full-cover-domain-failure } s) \ s \leq D$
shows
wp-event verify-monad
 $(\text{trace-fri-bad-with-header-tied-sampled-layer-chain } s) \ s$
 $\leq R + (P + I + W + D)$
proof (rule
wp-trace-fri-header-tied-sampled-layer-chain-bound-from-full-cover-and-missing)
show *wp-event verify-monad*
 $(\text{trace-fri-bad-with-full-cover-layer-chain } s \text{ bad}) \ s \leq R$
by (rule *wp-trace-fri-bad-with-full-cover-layer-chain-bound-from-envelope*
 $[OF \text{ future envelope subset challenge-bound}]$)
show *wp-event verify-monad*
 $(\text{trace-fri-sampled-missing-full-cover } s \text{ bad}) \ s \leq P + I + W + D$
by (rule
wp-trace-fri-sampled-missing-full-cover-bound-from-witness-length-subcases
 $[OF \text{ proximity index witness-length domain}]$)
qed

lemma *wp-composition-fri-verifier-tied-sampled-layer-chain-bound-from-envelope-witness-domain:*
assumes *future: composition-fri-future-fresh s*
and *envelope:*
 $\bigwedge dg \text{ composition-table.}$
 $\text{composition-fri-multiround-bad-sets bad dg composition-table}$
 $\subseteq B \ dg$
and *subset:*

$\bigwedge dg. B \ dg \subseteq \text{fri-challenge-space } (\text{ceil-log } (to\text{-nat } dg + 1))$
and *challenge-bound*:
 $\bigwedge dg. \text{nnreal } (\text{card } (B \ dg)) / \text{nnreal } (CARD('f) \wedge \text{ceil-log } (to\text{-nat } dg + 1)) \leq R$
and *proximity*:
 $\text{wp-event verify-monad } (composition\text{-fri-sampled-full-cover-proximity-failure } s \text{ bad}) \ s \leq P$
and *index*:
 $\text{wp-event verify-monad } (composition\text{-fri-sampled-full-cover-index-failure } s) \ s \leq I$
and *witness-length*:
 $\text{wp-event verify-monad } (composition\text{-fri-sampled-full-cover-witness-length-failure } s) \ s \leq W$
and *domain*:
 $\text{wp-event verify-monad } (composition\text{-fri-sampled-full-cover-domain-failure } s) \ s \leq D$
shows
 $\text{wp-event verify-monad } (composition\text{-fri-bad-with-verifier-tied-sampled-layer-chain } s) \ s \leq R + (P + I + W + D)$
proof (rule
 $\text{wp-composition-fri-verifier-tied-sampled-layer-chain-bound-from-full-cover-and-missing})$
show $\text{wp-event verify-monad } (composition\text{-fri-bad-with-full-cover-layer-chain } s \text{ bad}) \ s \leq R$
by (rule
 $\text{wp-composition-fri-bad-with-full-cover-layer-chain-bound-from-envelope } [OF \text{ future envelope subset challenge-bound}])$
show $\text{wp-event verify-monad } (composition\text{-fri-sampled-missing-full-cover } s \text{ bad}) \ s \leq P + I + W + D$
by (rule
 $\text{wp-composition-fri-sampled-missing-full-cover-bound-from-witness-length-subcases } [OF \text{ proximity index witness-length domain}])$
qed

lemma *trace-fri-header-tied-residual-obligations-from-envelope-domain-split-without-head-slot-and-selected-zero*:

assumes *future*: $\text{trace-fri-future-fresh } s$
and *envelope*:
 $\bigwedge \text{trace-table. trace-fri-multiround-bad-sets bad trace-table} \subseteq B$
and *subset*: $B \subseteq \text{fri-challenge-space } (\text{ceil-log } \text{clength})$
and *challenge-bound*:
 $\text{nnreal } (\text{card } B) / \text{nnreal } (CARD('f) \wedge \text{ceil-log } \text{clength}) \leq R$
and *proximity*:
 $\text{wp-event verify-monad } (\text{trace-fri-sampled-full-cover-proximity-failure } s \text{ bad}) \ s \leq P$
and *index*:
 $\text{wp-event verify-monad } (\text{trace-fri-sampled-full-cover-index-failure } s) \ s \leq I$

$(\text{trace-fri-sampled-full-cover-index-failure } s) \ s \leq I$
and *domain-length*:
 $\text{wp-event verify-monad}$
 $(\text{trace-fri-sampled-full-cover-domain-length-failure } s) \ s \leq DL$
and *sampled-domain*:
 $\text{wp-event verify-monad}$
 $(\text{trace-fri-sampled-full-cover-sampled-domain-failure } s) \ s \leq SD$
and *zero*:
 $\text{wp-event verify-monad}$
 $(\text{trace-fri-header-tied-zero-round-final-obstruction } s) \ s \leq z-b$
and *merkle*:
 $\text{wp-event verify-monad } (\text{partial-merkle-inconsistency-bad } s) \ s \leq p-b$
and *structural*:
 $\text{wp-event verify-monad}$
 $(\text{trace-fri-header-tied-sibling-mismatch-structural-gap } s) \ s$
 $\leq h-b$
and *next-bound*:
 $\text{wp-event verify-monad}$
 $(\text{trace-fri-header-tied-next-value-replay-gap } s) \ s \leq n-b$
and *successor*:
 $\text{wp-event verify-monad}$
 $(\text{trace-fri-header-tied-successor-replay-gap } s) \ s \leq s-b$
and *total*:
 $(R + (P + I + (I + DL) + (I + SD))) +$
 $(((((p-b + h-b) + p-b) + n-b + s-b +$
 $(p-b + 0 + 0)) + p-b) + 0 + z-b)$
 $\leq \text{trace-fri-error}$
shows
 $\text{trace-fri-header-tied-residual-obligations } s$
 $(R + (P + I + (I + DL) + (I + SD))) \ z-b \ 0 \ p-b \ h-b \ p-b$
 $n-b \ s-b \ 0$
by (*rule*
 $\text{trace-fri-header-tied-residual-obligations-from-bounds-without-head-slot-and-selected-zero}$)
(*rule*
 $\text{wp-trace-fri-header-tied-sampled-layer-chain-bound-from-envelope-domain-split}$
 $[\text{OF future envelope subset challenge-bound proximity index}$
 $\text{domain-length sampled-domain}]$,
 $\text{rule zero, rule merkle, rule structural, rule next-bound,}$
 $\text{rule successor, rule total}$)

lemma *composition-fri-verifier-tied-residual-obligations-from-envelope-domain-split-slot-zero*:
assumes *future: composition-fri-future-fresh* s
and *envelope*:
 $\bigwedge dg \ \text{composition-table.}$
 $\text{composition-fri-multiround-bad-sets bad } dg \ \text{composition-table}$
 $\subseteq B \ dg$
and *subset*:
 $\bigwedge dg. \ B \ dg \subseteq \text{fri-challenge-space } (\text{ceil-log } (\text{to-nat } dg + 1))$
and *challenge-bound*:

$\bigwedge dg. \text{nnreal}(\text{card}(B \text{ dg})) /$
 $\text{nnreal}(\text{CARD}(f) \wedge \text{ceil-log}(\text{to-nat } dg + 1)) \leq R$
and *proximity*:
 $\text{wp-event verify-monad}$
 $(\text{composition-fri-sampled-full-cover-proximity-failure } s \text{ bad}) \text{ } s$
 $\leq P$
and *index*:
 $\text{wp-event verify-monad}$
 $(\text{composition-fri-sampled-full-cover-index-failure } s) \text{ } s \leq I$
and *domain-length*:
 $\text{wp-event verify-monad}$
 $(\text{composition-fri-sampled-full-cover-domain-length-failure } s) \text{ } s$
 $\leq DL$
and *sampled-domain*:
 $\text{wp-event verify-monad}$
 $(\text{composition-fri-sampled-full-cover-sampled-domain-failure } s) \text{ } s$
 $\leq SD$
and *merkle*:
 $\text{wp-event verify-monad}(\text{partial-merkle-inconsistency-bad } s) \text{ } s \leq p\text{-}b$
and *next-bound*:
 $\text{wp-event verify-monad}$
 $(\text{composition-fri-verifier-tied-next-value-replay-gap } s) \text{ } s \leq n\text{-}b$
and *successor*:
 $\text{wp-event verify-monad}$
 $(\text{composition-fri-verifier-tied-successor-replay-gap } s) \text{ } s \leq s\text{-}b$
and *final-bound*:
 $\text{wp-event verify-monad}$
 $(\text{composition-fri-verifier-tied-final-value-replay-gap } s) \text{ } s \leq g\text{-}b$
and *total*:
 $(R + (P + I + (I + DL) + (I + SD))) +$
 $((p\text{-}b + 0) + n\text{-}b + s\text{-}b + g\text{-}b + p\text{-}b) + 0)$
 $\leq \text{composition-fri-error}$
shows
 $\text{composition-fri-verifier-tied-residual-obligations } s$
 $(R + (P + I + (I + DL) + (I + SD))) \text{ } 0 \text{ } p\text{-}b \text{ } n\text{-}b \text{ } s\text{-}b \text{ } g\text{-}b$
by (rule *composition-fri-verifier-tied-residual-obligations-from-bounds-slot-zero*)
(rule
 $\text{wp-composition-fri-verifier-tied-sampled-layer-chain-bound-from-envelope-domain-split}$
 $[\text{OF future envelope subset challenge-bound proximity index}$
 $\text{domain-length sampled-domain}]$,
rule *merkle*, rule *next-bound*, rule *successor*, rule *final-bound*,
rule *total*)

lemma *trace-fri-header-tied-residual-obligations-from-envelope-witness-domain-without-head-slot-and-selected-zero*:
assumes *future*: *trace-fri-future-fresh* *s*
and *envelope*:
 $\bigwedge \text{trace-table. trace-fri-multiround-bad-sets bad trace-table} \subseteq B$
and *subset*: $B \subseteq \text{fri-challenge-space}(\text{ceil-log } \text{clength})$
and *challenge-bound*:

$\text{nnreal } (\text{card } B) / \text{nnreal } (\text{CARD}(f) \wedge \text{ceil-log } \text{clength}) \leq R$
and *proximity*:
 $\text{wp-event verify-monad}$
 $(\text{trace-fri-sampled-full-cover-proximity-failure } s \text{ bad}) \ s \leq P$
and *index*:
 $\text{wp-event verify-monad}$
 $(\text{trace-fri-sampled-full-cover-index-failure } s) \ s \leq I$
and *witness-length*:
 $\text{wp-event verify-monad}$
 $(\text{trace-fri-sampled-full-cover-witness-length-failure } s) \ s \leq W$
and *domain*:
 $\text{wp-event verify-monad}$
 $(\text{trace-fri-sampled-full-cover-domain-failure } s) \ s \leq D$
and *zero*:
 $\text{wp-event verify-monad}$
 $(\text{trace-fri-header-tied-zero-round-final-obstruction } s) \ s \leq z\text{-b}$
and *merkle*:
 $\text{wp-event verify-monad } (\text{partial-merkle-inconsistency-bad } s) \ s \leq p\text{-b}$
and *structural*:
 $\text{wp-event verify-monad}$
 $(\text{trace-fri-header-tied-sibling-mismatch-structural-gap } s) \ s$
 $\leq h\text{-b}$
and *next-bound*:
 $\text{wp-event verify-monad}$
 $(\text{trace-fri-header-tied-next-value-replay-gap } s) \ s \leq n\text{-b}$
and *successor*:
 $\text{wp-event verify-monad}$
 $(\text{trace-fri-header-tied-successor-replay-gap } s) \ s \leq s\text{-b}$
and *total*:
 $(R + (P + I + W + D)) +$
 $(((((p\text{-b} + h\text{-b}) + p\text{-b}) + n\text{-b} + s\text{-b} +$
 $(p\text{-b} + 0 + 0)) + p\text{-b}) + 0 + z\text{-b})$
 $\leq \text{trace-fri-error}$
shows
 $\text{trace-fri-header-tied-residual-obligations } s$
 $(R + (P + I + W + D)) \ z\text{-b } 0 \ p\text{-b } h\text{-b } p\text{-b } n\text{-b } s\text{-b } 0$
by (rule
 $\text{trace-fri-header-tied-residual-obligations-from-bounds-without-head-slot-and-selected-zero})$
(rule
 $\text{wp-trace-fri-header-tied-sampled-layer-chain-bound-from-envelope-witness-domain}$
 $[\text{OF future envelope subset challenge-bound proximity index}$
 $\text{witness-length domain}]$,
rule zero, rule merkle, rule structural, rule next-bound,
rule successor, rule total)

lemma *composition-fri-verifier-tied-residual-obligations-from-envelope-witness-domain-slot-zero*:
assumes *future*: *composition-fri-future-fresh* s
and *envelope*:
 $\bigwedge dg \text{ composition-table.}$

$\text{composition-fri-multi-round-bad-sets bad dg composition-table}$
 $\subseteq B \text{ dg}$
and subset:
 $\bigwedge \text{dg. } B \text{ dg} \subseteq \text{fri-challenge-space (ceil-log (to-nat dg + 1))}$
and challenge-bound:
 $\bigwedge \text{dg. nnreal (card (B dg)) /}$
 $\text{nnreal (CARD('f) } \wedge \text{ ceil-log (to-nat dg + 1))} \leq R$
and proximity:
 $\text{wp-event verify-monad}$
 $(\text{composition-fri-sampled-full-cover-proximity-failure s bad}) s$
 $\leq P$
and index:
 $\text{wp-event verify-monad}$
 $(\text{composition-fri-sampled-full-cover-index-failure s}) s \leq I$
and witness-length:
 $\text{wp-event verify-monad}$
 $(\text{composition-fri-sampled-full-cover-witness-length-failure s}) s$
 $\leq W$
and domain:
 $\text{wp-event verify-monad}$
 $(\text{composition-fri-sampled-full-cover-domain-failure s}) s \leq D$
and merkle:
 $\text{wp-event verify-monad (partial-merkle-inconsistency-bad s)} s \leq p\text{-b}$
and next-bound:
 $\text{wp-event verify-monad}$
 $(\text{composition-fri-verifier-tied-next-value-replay-gap s}) s \leq n\text{-b}$
and successor:
 $\text{wp-event verify-monad}$
 $(\text{composition-fri-verifier-tied-successor-replay-gap s}) s \leq s\text{-b}$
and final-bound:
 $\text{wp-event verify-monad}$
 $(\text{composition-fri-verifier-tied-final-value-replay-gap s}) s \leq g\text{-b}$
and total:
 $(R + (P + I + W + D)) +$
 $((p\text{-b} + 0) + n\text{-b} + s\text{-b} + g\text{-b} + p\text{-b}) + 0)$
 $\leq \text{composition-fri-error}$
shows
 $\text{composition-fri-verifier-tied-residual-obligations s}$
 $(R + (P + I + W + D)) \ 0 \ p\text{-b} \ n\text{-b} \ s\text{-b} \ g\text{-b}$
by (rule $\text{composition-fri-verifier-tied-residual-obligations-from-bounds-slot-zero}$)
(rule
 $\text{wp-composition-fri-verifier-tied-sampled-layer-chain-bound-from-envelope-witness-domain}$
 $[\text{OF future envelope subset challenge-bound proximity index}$
 $\text{witness-length domain}]$,
rule merkle, rule next-bound, rule successor, rule final-bound,
rule total)

end

end

theory *Soundness-FRI-Full-Cover-Staged-Bounds*

imports

Soundness-FRI-Prequery-Composition-Sampled-Bounds

Soundness-FRI-Full-Cover-Active-Bounds

begin

Staged bounds for the full-cover FRI split.

Unlike the verifier-local full-cover envelope lemmas, these results work in the staged adversary experiment where future FRI keys may have been queried by the attacker. The full-cover part is therefore routed through the existing staged fresh/prequery accounting for FRI challenge-list hits.

context *soundness*

begin

lemma *trace-fri-bad-with-sampled-layer-chain-not-None:*

$\neg$ *trace-fri-bad-with-sampled-layer-chain* *s* *None*

unfolding *trace-fri-bad-with-sampled-layer-chain-def*

trace-fri-partial-candidate-opening-evidence-def

accepted-fri-opening-transcript-def

by *blast*

lemma *composition-fri-bad-with-sampled-layer-chain-not-None:*

$\neg$ *composition-fri-bad-with-sampled-layer-chain* *s* *None*

unfolding *composition-fri-bad-with-sampled-layer-chain-def*

composition-fri-partial-candidate-opening-evidence-def

accepted-fri-opening-transcript-def

by *blast*

lemma *trace-fri-sampled-missing-full-cover-not-None:*

$\neg$ *trace-fri-sampled-missing-full-cover* *s* *bad* *None*

unfolding *trace-fri-sampled-missing-full-cover-def*

using *trace-fri-bad-with-sampled-layer-chain-not-None* **by** *blast*

lemma *composition-fri-sampled-missing-full-cover-not-None:*

$\neg$ *composition-fri-sampled-missing-full-cover* *s* *bad* *None*

unfolding *composition-fri-sampled-missing-full-cover-def*

using *composition-fri-bad-with-sampled-layer-chain-not-None* **by** *blast*

lemma *checked-staged-security-trace-fri-full-cover-layer-chain-bound-from-fresh-and-prequery:*

assumes *wf: staged-budget-wellformed* *budgets*

and *controlled: staged-adversary-controlled* *budgets* *A*

and *finite-B: finite* *B*

and *envelope:*

$\bigwedge$ *data* *attacker-state* *trace-table.*

Some (*data*, *attacker-state*) $\in$

set-dist (*execute* (*checked-staged-transcript-program* *A*))

```

    adversary-initial-state)  $\implies$ 
    trace-fri-multiround-bad-sets bad trace-table  $\subseteq B$ 
and fresh-bound:
wp-event (checked-staged-security-experiment-with-data-state A)
  (staged-security-with-data-state-verifier-event
    ( $\lambda s$ . trace-fri-challenge-list-fresh-hit s B))
    adversary-initial-state  $\leq F$ 
shows
wp-event (checked-staged-security-experiment-with-data-state A)
  (staged-security-with-data-state-verifier-event
    ( $\lambda s$ . trace-fri-bad-with-full-cover-layer-chain s bad))
    adversary-initial-state
 $\leq F +$ 
    hash-relation-budget-value (card B * ceil-log clength)
    (staged-attacker-query-budget budgets +
      staged-challenge-query-budget)
proof –
have wp-event (checked-staged-security-experiment-with-data-state A)
  (staged-security-with-data-state-verifier-event
    ( $\lambda s$ . trace-fri-bad-with-full-cover-layer-chain s bad))
    adversary-initial-state  $\leq$ 
wp-event (checked-staged-security-experiment-with-data-state A)
  (staged-security-with-data-state-verifier-event
    ( $\lambda s$ . trace-fri-challenge-list-set-hit s B))
    adversary-initial-state
proof (rule wp-event-mono-on-support)
fix out
assume support:
  out  $\in$ 
    set-dist
    (execute (checked-staged-security-experiment-with-data-state A)
      adversary-initial-state)
and hit:
  staged-security-with-data-state-verifier-event
    ( $\lambda s$ . trace-fri-bad-with-full-cover-layer-chain s bad) out
from hit obtain data attacker-state result final-state where
  out-eq: out = Some (((data, attacker-state), result), final-state)
unfolding staged-security-with-data-state-verifier-event-def
by (cases out) (auto split: option.splits prod.splits)
have builder:
  Some (data, attacker-state)  $\in$ 
    set-dist (execute (checked-staged-transcript-program A)
      adversary-initial-state)
using checked-staged-security-experiment-with-data-state-outcomeE
  [OF support[unfolded out-eq]]
by blast
let ?s =
  verifier-state-from-adversary attacker-state
    (staged-proof-transcript data)

```



```

have full:
  trace-fri-bad-with-full-cover-layer-chain ?s bad
  (Some (result, final-state))
using hit unfolding staged-security-with-data-state-verifier-event-def
  out-eq by simp
have list-hit:
  trace-fri-challenge-list-set-hit ?s B
  (Some (result, final-state))
by (rule trace-fri-bad-with-full-cover-layer-chain-imp-list-hit
    [OF full])
  (rule envelope[OF builder])
show
  staged-security-with-data-state-verifier-event
  ( $\lambda s.$  trace-fri-challenge-list-set-hit s B) out
unfolding staged-security-with-data-state-verifier-event-def out-eq
using list-hit by simp
qed
also have ...  $\leq$ 
  F +
  hash-relation-budget-value (card B * ceil-log clength)
  (staged-attacker-query-budget budgets +
    staged-challenge-query-budget)
by (rule checked-staged-security-trace-fri-challenge-list-set-hit-bound
  [OF wf controlled finite-B fresh-bound])
finally show ?thesis .
qed

lemma checked-staged-security-composition-fri-full-cover-layer-chain-bound-from-fresh-and-prequery:
assumes wf: staged-budget-wellformed budgets
and controlled: staged-adversary-controlled budgets A
and finite-B:  $\bigwedge dg.$  finite (B dg)
and envelope:
 $\bigwedge$ data attacker-state dg composition-table.
  Some (data, attacker-state)  $\in$ 
    set-dist (execute (checked-staged-transcript-program A)
      adversary-initial-state)  $\implies$ 
    composition-fri-multiround-bad-sets bad dg composition-table
     $\subseteq$  B dg
and fresh-bound:
  wp-event (checked-staged-security-experiment-with-data-state A)
    (staged-security-with-data-state-verifier-event
      ( $\lambda s.$  composition-fri-challenge-list-fresh-hit s B))
  adversary-initial-state  $\leq$  F
shows
  wp-event (checked-staged-security-experiment-with-data-state A)
    (staged-security-with-data-state-verifier-event
      ( $\lambda s.$  composition-fri-bad-with-full-cover-layer-chain s bad))
  adversary-initial-state
   $\leq$  F +

```

```

    hash-relation-budget-value
    ( $\sum dg \in (UNIV :: 'f \text{ set}).$ 
      card (B dg) * ceil-log (maxDegree + 1))
    (staged-attacker-query-budget budgets +
      staged-challenge-query-budget)
proof –
  have wp-event (checked-staged-security-experiment-with-data-state A)
    (staged-security-with-data-state-verifier-event
      ( $\lambda s.$  composition-fri-bad-with-full-cover-layer-chain s bad))
    adversary-initial-state  $\leq$ 
    wp-event (checked-staged-security-experiment-with-data-state A)
      (staged-security-with-data-state-verifier-event
        ( $\lambda s.$  composition-fri-challenge-list-set-hit s B))
      adversary-initial-state
proof (rule wp-event-mono-on-support)
  fix out
  assume support:
    out  $\in$ 
      set-dist
        (execute (checked-staged-security-experiment-with-data-state A)
          adversary-initial-state)
  and hit:
    staged-security-with-data-state-verifier-event
      ( $\lambda s.$  composition-fri-bad-with-full-cover-layer-chain s bad) out
from hit obtain data attacker-state result final-state where
  out-eq: out = Some (((data, attacker-state), result), final-state)
  unfolding staged-security-with-data-state-verifier-event-def
by (cases out) (auto split: option.splits prod.splits)
have builder:
  Some (data, attacker-state)  $\in$ 
    set-dist (execute (checked-staged-transcript-program A)
      adversary-initial-state)
  using checked-staged-security-experiment-with-data-state-outcomeE
    [OF support[unfolded out-eq]]
by blast
let ?s =
  verifier-state-from-adversary attacker-state
    (staged-proof-transcript data)
have full:
  composition-fri-bad-with-full-cover-layer-chain ?s bad
    (Some (result, final-state))
  using hit unfolding staged-security-with-data-state-verifier-event-def
    out-eq by simp
have list-hit:
  composition-fri-challenge-list-set-hit ?s B
    (Some (result, final-state))
by (rule composition-fri-bad-with-full-cover-layer-chain-imp-list-hit
    [OF full])
  (rule envelope[OF builder])

```

```

show
  staged-security-with-data-state-verifier-event
    ( $\lambda s. \text{composition-fri-challenge-list-set-hit } s \ B$ ) out
unfolding staged-security-with-data-state-verifier-event-def out-eq
using list-hit by simp
qed
also have ...  $\leq$ 
   $F +$ 
    hash-relation-budget-value
    ( $\sum dg \in (UNIV :: 'f \text{ set}).$ 
       $\text{card } (B \ dg) * \text{ceil-log } (\text{maxDegree} + 1))$ 
    (staged-attacker-query-budget budgets +
      staged-challenge-query-budget)
by (rule
  checked-staged-security-composition-fri-challenge-list-set-hit-bound
  [OF wf controlled finite-B fresh-bound])
finally show ?thesis .
qed

lemma checked-staged-security-trace-fri-sampled-missing-full-cover-bound-from-domain-split:
assumes proximity:
   $\bigwedge \text{data attacker-state.}$ 
   $\text{Some } (data, \text{attacker-state}) \in$ 
     $\text{set-dist } (\text{execute } (\text{checked-staged-transcript-program } A)$ 
       $\text{adversary-initial-state}) \implies$ 
  wp-event verify-monad
    (trace-fri-sampled-full-cover-proximity-failure
      (verifier-state-from-adversary attacker-state
        (staged-proof-transcript data))) bad)
    (verifier-state-from-adversary attacker-state
      (staged-proof-transcript data))  $\leq P$ 
and index:
   $\bigwedge \text{data attacker-state.}$ 
   $\text{Some } (data, \text{attacker-state}) \in$ 
     $\text{set-dist } (\text{execute } (\text{checked-staged-transcript-program } A)$ 
       $\text{adversary-initial-state}) \implies$ 
  wp-event verify-monad
    (trace-fri-sampled-full-cover-index-failure
      (verifier-state-from-adversary attacker-state
        (staged-proof-transcript data))))
    (verifier-state-from-adversary attacker-state
      (staged-proof-transcript data))  $\leq I$ 
and domain-length:
   $\bigwedge \text{data attacker-state.}$ 
   $\text{Some } (data, \text{attacker-state}) \in$ 
     $\text{set-dist } (\text{execute } (\text{checked-staged-transcript-program } A)$ 
       $\text{adversary-initial-state}) \implies$ 
  wp-event verify-monad
    (trace-fri-sampled-full-cover-domain-length-failure

```

```

      (verifier-state-from-adversary attacker-state
       (staged-proof-transcript data)))
    (verifier-state-from-adversary attacker-state
     (staged-proof-transcript data)) ≤ DL
  and sampled-domain:
  ∧ data attacker-state.
    Some (data, attacker-state) ∈
      set-dist (execute (checked-staged-transcript-program A)
                      adversary-initial-state) ⇒
    wp-event verify-monad
      (trace-fri-sampled-full-cover-sampled-domain-failure
       (verifier-state-from-adversary attacker-state
        (staged-proof-transcript data)))
      (verifier-state-from-adversary attacker-state
       (staged-proof-transcript data)) ≤ SD
  shows
    wp-event (checked-staged-security-experiment-with-data-state A)
      (staged-security-with-data-state-verifier-event
       (λs. trace-fri-sampled-missing-full-cover s bad))
      adversary-initial-state ≤ P + I + (I + DL) + (I + SD)
  proof (rule checked-staged-security-with-data-state-verifier-event-bound-from-cont)
    show ∧s. ¬ trace-fri-sampled-missing-full-cover s bad None
      by (rule trace-fri-sampled-missing-full-cover-not-None)
  next
  fix data attacker-state
  assume support:
    Some (data, attacker-state) ∈
      set-dist (execute (checked-staged-transcript-program A)
                      adversary-initial-state)
  let ?s =
    verifier-state-from-adversary attacker-state
    (staged-proof-transcript data)
  show wp-event verify-monad
    (trace-fri-sampled-missing-full-cover ?s bad) ?s
    ≤ P + I + (I + DL) + (I + SD)
  by (rule wp-trace-fri-sampled-missing-full-cover-bound-from-domain-split)
    (rule proximity[OF support], rule index[OF support],
     rule domain-length[OF support], rule sampled-domain[OF support])
qed

```

lemma *checked-staged-security-trace-fri-sampled-layer-chain-bound-from-full-cover-fresh-and-domain-split:*

```

  assumes wf: staged-budget-wellformed budgets
  and controlled: staged-adversary-controlled budgets A
  and finite-B: finite B
  and envelope:
  ∧ data attacker-state trace-table.
    Some (data, attacker-state) ∈
      set-dist (execute (checked-staged-transcript-program A)
                      adversary-initial-state) ⇒

```

$\text{trace-fri-multiround-bad-sets bad trace-table} \subseteq B$
and *fresh-bound*:
 $\text{wp-event } (\text{checked-staged-security-experiment-with-data-state } A)$
 $(\text{staged-security-with-data-state-verifier-event}$
 $(\lambda s. \text{trace-fri-challenge-list-fresh-hit } s \ B))$
 $\text{adversary-initial-state} \leq F$
and *proximity*:
 $\wedge \text{data attacker-state.}$
 $\text{Some } (data, \text{attacker-state}) \in$
 $\text{set-dist } (\text{execute } (\text{checked-staged-transcript-program } A)$
 $\text{adversary-initial-state}) \implies$
 $\text{wp-event verify-monad}$
 $(\text{trace-fri-sampled-full-cover-proximity-failure}$
 $(\text{verifier-state-from-adversary attacker-state}$
 $(\text{staged-proof-transcript data})) \text{ bad})$
 $(\text{verifier-state-from-adversary attacker-state}$
 $(\text{staged-proof-transcript data})) \leq P$
and *index*:
 $\wedge \text{data attacker-state.}$
 $\text{Some } (data, \text{attacker-state}) \in$
 $\text{set-dist } (\text{execute } (\text{checked-staged-transcript-program } A)$
 $\text{adversary-initial-state}) \implies$
 $\text{wp-event verify-monad}$
 $(\text{trace-fri-sampled-full-cover-index-failure}$
 $(\text{verifier-state-from-adversary attacker-state}$
 $(\text{staged-proof-transcript data})))$
 $(\text{verifier-state-from-adversary attacker-state}$
 $(\text{staged-proof-transcript data})) \leq I$
and *domain-length*:
 $\wedge \text{data attacker-state.}$
 $\text{Some } (data, \text{attacker-state}) \in$
 $\text{set-dist } (\text{execute } (\text{checked-staged-transcript-program } A)$
 $\text{adversary-initial-state}) \implies$
 $\text{wp-event verify-monad}$
 $(\text{trace-fri-sampled-full-cover-domain-length-failure}$
 $(\text{verifier-state-from-adversary attacker-state}$
 $(\text{staged-proof-transcript data})))$
 $(\text{verifier-state-from-adversary attacker-state}$
 $(\text{staged-proof-transcript data})) \leq DL$
and *sampled-domain*:
 $\wedge \text{data attacker-state.}$
 $\text{Some } (data, \text{attacker-state}) \in$
 $\text{set-dist } (\text{execute } (\text{checked-staged-transcript-program } A)$
 $\text{adversary-initial-state}) \implies$
 $\text{wp-event verify-monad}$
 $(\text{trace-fri-sampled-full-cover-sampled-domain-failure}$
 $(\text{verifier-state-from-adversary attacker-state}$
 $(\text{staged-proof-transcript data})))$
 $(\text{verifier-state-from-adversary attacker-state}$

$(\text{staged-proof-transcript data}) \leq SD$
shows
 $\text{wp-event } (\text{checked-staged-security-experiment-with-data-state } A)$
 $(\text{staged-security-with-data-state-verifier-event}$
 $\text{trace-fri-bad-with-sampled-layer-chain})$
 $\text{adversary-initial-state}$
 $\leq$
 $(F +$
 $\text{hash-relation-budget-value } (\text{card } B * \text{ceil-log } \text{clength})$
 $(\text{staged-attacker-query-budget budgets} +$
 $\text{staged-challenge-query-budget})) +$
 $(P + I + (I + DL) + (I + SD))$
proof –
have *full-bound*:
 $\text{wp-event } (\text{checked-staged-security-experiment-with-data-state } A)$
 $(\text{staged-security-with-data-state-verifier-event}$
 $(\lambda s. \text{trace-fri-bad-with-full-cover-layer-chain } s \text{ bad}))$
 $\text{adversary-initial-state}$
 $\leq$
 $F +$
 $\text{hash-relation-budget-value } (\text{card } B * \text{ceil-log } \text{clength})$
 $(\text{staged-attacker-query-budget budgets} +$
 $\text{staged-challenge-query-budget})$
by (rule
 $\text{checked-staged-security-trace-fri-full-cover-layer-chain-bound-from-fresh-and-prequery}$
 $[\text{OF } \text{wf controlled finite-}B \text{ envelope fresh-bound}])$
have *missing-bound*:
 $\text{wp-event } (\text{checked-staged-security-experiment-with-data-state } A)$
 $(\text{staged-security-with-data-state-verifier-event}$
 $(\lambda s. \text{trace-fri-sampled-missing-full-cover } s \text{ bad}))$
 $\text{adversary-initial-state} \leq P + I + (I + DL) + (I + SD)$
by (rule
 $\text{checked-staged-security-trace-fri-sampled-missing-full-cover-bound-from-domain-split}$
 $[\text{OF } \text{proximity index domain-length sampled-domain}])$
have $\text{wp-event } (\text{checked-staged-security-experiment-with-data-state } A)$
 $(\text{staged-security-with-data-state-verifier-event}$
 $\text{trace-fri-bad-with-sampled-layer-chain})$
 $\text{adversary-initial-state} \leq$
 $\text{wp-event } (\text{checked-staged-security-experiment-with-data-state } A)$
 $(\lambda \text{out.}$
 $\text{staged-security-with-data-state-verifier-event}$
 $(\lambda s. \text{trace-fri-bad-with-full-cover-layer-chain } s \text{ bad}) \text{ out} \vee$
 $\text{staged-security-with-data-state-verifier-event}$
 $(\lambda s. \text{trace-fri-sampled-missing-full-cover } s \text{ bad}) \text{ out})$
 $\text{adversary-initial-state}$
by (rule *wp-event-mono-on-support*)
 $(\text{auto simp: staged-security-with-data-state-verifier-event-def}$
 $\text{dest: trace-fri-sampled-imp-full-cover-or-missing}$
 $\text{split: option.splits prod.splits})$

also have ... $\leq$
 $wp\text{-event } (checked\text{-staged}\text{-security}\text{-experiment}\text{-with}\text{-data}\text{-state } A)$
 $(staged\text{-security}\text{-with}\text{-data}\text{-state}\text{-verifier}\text{-event}$
 $(\lambda s. trace\text{-fri}\text{-bad}\text{-with}\text{-full}\text{-cover}\text{-layer}\text{-chain } s \text{ bad}))$
 $adversary\text{-initial}\text{-state} +$
 $wp\text{-event } (checked\text{-staged}\text{-security}\text{-experiment}\text{-with}\text{-data}\text{-state } A)$
 $(staged\text{-security}\text{-with}\text{-data}\text{-state}\text{-verifier}\text{-event}$
 $(\lambda s. trace\text{-fri}\text{-sampled}\text{-missing}\text{-full}\text{-cover } s \text{ bad}))$
 $adversary\text{-initial}\text{-state}$
by (rule $wp\text{-event}\text{-union}\text{-bound}$)
also have ... $\leq$
 $(F +$
 $hash\text{-relation}\text{-budget}\text{-value } (card \ B * \text{ceil}\text{-log } clength)$
 $(staged\text{-attacker}\text{-query}\text{-budget } budgets +$
 $staged\text{-challenge}\text{-query}\text{-budget})) +$
 $(P + I + (I + DL) + (I + SD))$
by (rule $add\text{-mono}[OF \text{ full}\text{-bound } missing\text{-bound}]$)
finally show ?thesis .
qed

lemma *checked-staged-security-trace-fri-canonical-sampled-layer-chain-bound-from-full-cover-fresh-and-domain-split:*

assumes wf : *staged-budget-wellformed* $budgets$
and *controlled*: *staged-adversary-controlled* $budgets \ A$
and *finite-B*: *finite* B
and *envelope*:
 $\bigwedge data \ attacker\text{-state} \ trace\text{-table}.$
 $Some \ (data, \ attacker\text{-state}) \in$
 $set\text{-dist } (execute \ (checked\text{-staged}\text{-transcript}\text{-program } A)$
 $adversary\text{-initial}\text{-state}) \implies$
 $trace\text{-fri}\text{-multiround}\text{-bad}\text{-sets } bad \ trace\text{-table} \subseteq B$
and *fresh-bound*:
 $wp\text{-event } (checked\text{-staged}\text{-security}\text{-experiment}\text{-with}\text{-data}\text{-state } A)$
 $(staged\text{-security}\text{-with}\text{-data}\text{-state}\text{-verifier}\text{-event}$
 $(\lambda s. trace\text{-fri}\text{-challenge}\text{-list}\text{-fresh}\text{-hit } s \ B))$
 $adversary\text{-initial}\text{-state} \leq F$
and *proximity*:
 $\bigwedge data \ attacker\text{-state}.$
 $Some \ (data, \ attacker\text{-state}) \in$
 $set\text{-dist } (execute \ (checked\text{-staged}\text{-transcript}\text{-program } A)$
 $adversary\text{-initial}\text{-state}) \implies$
 $wp\text{-event } verify\text{-monad}$
 $(trace\text{-fri}\text{-sampled}\text{-full}\text{-cover}\text{-proximity}\text{-failure}$
 $(verifier\text{-state}\text{-from}\text{-adversary } attacker\text{-state}$
 $(staged\text{-proof}\text{-transcript } data)) \text{ bad})$
 $(verifier\text{-state}\text{-from}\text{-adversary } attacker\text{-state}$
 $(staged\text{-proof}\text{-transcript } data)) \leq P$
and *index*:
 $\bigwedge data \ attacker\text{-state}.$
 $Some \ (data, \ attacker\text{-state}) \in$

$\text{set-dist } (\text{execute } (\text{checked-staged-transcript-program } A) \text{ adversary-initial-state}) \implies$
 $\text{wp-event verify-monad}$
 $(\text{trace-fri-sampled-full-cover-index-failure}$
 $(\text{verifier-state-from-adversary attacker-state}$
 $(\text{staged-proof-transcript data})))$
 $(\text{verifier-state-from-adversary attacker-state}$
 $(\text{staged-proof-transcript data})) \leq I$
and domain-length:
 $\wedge \text{data attacker-state.}$
 $\text{Some } (data, \text{attacker-state}) \in$
 $\text{set-dist } (\text{execute } (\text{checked-staged-transcript-program } A) \text{ adversary-initial-state}) \implies$
 $\text{wp-event verify-monad}$
 $(\text{trace-fri-sampled-full-cover-domain-length-failure}$
 $(\text{verifier-state-from-adversary attacker-state}$
 $(\text{staged-proof-transcript data})))$
 $(\text{verifier-state-from-adversary attacker-state}$
 $(\text{staged-proof-transcript data})) \leq DL$
and sampling-domain:
 $\wedge \text{data attacker-state.}$
 $\text{Some } (data, \text{attacker-state}) \in$
 $\text{set-dist } (\text{execute } (\text{checked-staged-transcript-program } A) \text{ adversary-initial-state}) \implies$
 $\text{wp-event verify-monad}$
 $(\text{trace-fri-sampled-full-cover-sampled-domain-failure}$
 $(\text{verifier-state-from-adversary attacker-state}$
 $(\text{staged-proof-transcript data})))$
 $(\text{verifier-state-from-adversary attacker-state}$
 $(\text{staged-proof-transcript data})) \leq SD$
shows
 $\text{wp-event } (\text{checked-staged-security-experiment-with-data-state } A)$
 $(\text{staged-security-with-data-state-verifier-event}$
 $\text{trace-fri-bad-with-canonical-sampled-layer-chain})$
 $\text{adversary-initial-state}$
 $\leq$
 $(F +$
 $\text{hash-relation-budget-value } (\text{card } B * \text{ceil-log clength})$
 $(\text{staged-attacker-query-budget budgets} +$
 $\text{staged-challenge-query-budget})) +$
 $(P + I + (I + DL) + (I + SD))$
proof –
have $\text{wp-event } (\text{checked-staged-security-experiment-with-data-state } A)$
 $(\text{staged-security-with-data-state-verifier-event}$
 $\text{trace-fri-bad-with-canonical-sampled-layer-chain})$
 $\text{adversary-initial-state} \leq$
 $\text{wp-event } (\text{checked-staged-security-experiment-with-data-state } A)$
 $(\text{staged-security-with-data-state-verifier-event}$
 $\text{trace-fri-bad-with-sampled-layer-chain})$

adversary-initial-state
by (rule *wp-event-mono-on-support*)
 (auto simp: *staged-security-with-data-state-verifier-event-def*
intro: trace-fri-bad-with-canonical-sampled-layer-chain-imp-sampled
split: option.splits prod.splits)
also have ... $\leq$
 ($F +$
hash-relation-budget-value (*card B * ceil-log clength*)
 (*staged-attacker-query-budget budgets +*
staged-challenge-query-budget)) +
 ($P + I + (I + DL) + (I + SD)$)
by (rule
checked-staged-security-trace-fri-sampled-layer-chain-bound-from-full-cover-fresh-and-domain-split
 [*OF wf controlled finite-B envelope fresh-bound proximity index*
domain-length sampled-domain])
finally show ?thesis .
qed

lemma *checked-staged-security-composition-fri-sampled-missing-full-cover-bound-from-domain-split:*

assumes *proximity:*

$\wedge$ *data attacker-state.*

Some (*data, attacker-state*) $\in$
set-dist (*execute* (*checked-staged-transcript-program A*)
adversary-initial-state) $\implies$

wp-event verify-monad

(*composition-fri-sampled-full-cover-proximity-failure*
(verifier-state-from-adversary attacker-state
(staged-proof-transcript data)) bad)
(verifier-state-from-adversary attacker-state
(staged-proof-transcript data)) $\leq P$

and *index:*

$\wedge$ *data attacker-state.*

Some (*data, attacker-state*) $\in$
set-dist (*execute* (*checked-staged-transcript-program A*)
adversary-initial-state) $\implies$

wp-event verify-monad

(*composition-fri-sampled-full-cover-index-failure*
(verifier-state-from-adversary attacker-state
(staged-proof-transcript data)))
(verifier-state-from-adversary attacker-state
(staged-proof-transcript data)) $\leq I$

and *domain-length:*

$\wedge$ *data attacker-state.*

Some (*data, attacker-state*) $\in$
set-dist (*execute* (*checked-staged-transcript-program A*)
adversary-initial-state) $\implies$

wp-event verify-monad

(*composition-fri-sampled-full-cover-domain-length-failure*
(verifier-state-from-adversary attacker-state

```

      (staged-proof-transcript data)))
    (verifier-state-from-adversary attacker-state
      (staged-proof-transcript data)) ≤ DL
  and sampled-domain:
  ∧ data attacker-state.
    Some (data, attacker-state) ∈
      set-dist (execute (checked-staged-transcript-program A)
        adversary-initial-state) ⇒
    wp-event verify-monad
      (composition-fri-sampled-full-cover-sampled-domain-failure
        (verifier-state-from-adversary attacker-state
          (staged-proof-transcript data)))
      (verifier-state-from-adversary attacker-state
        (staged-proof-transcript data)) ≤ SD
  shows
    wp-event (checked-staged-security-experiment-with-data-state A)
      (staged-security-with-data-state-verifier-event
        (λs. composition-fri-sampled-missing-full-cover s bad))
      adversary-initial-state ≤ P + I + (I + DL) + (I + SD)
  proof (rule checked-staged-security-with-data-state-verifier-event-bound-from-cont)
    show ∧s. ¬ composition-fri-sampled-missing-full-cover s bad None
      by (rule composition-fri-sampled-missing-full-cover-not-None)
  next
  fix data attacker-state
  assume support:
    Some (data, attacker-state) ∈
      set-dist (execute (checked-staged-transcript-program A)
        adversary-initial-state)
  let ?s =
    verifier-state-from-adversary attacker-state
      (staged-proof-transcript data)
  show wp-event verify-monad
    (composition-fri-sampled-missing-full-cover ?s bad) ?s
    ≤ P + I + (I + DL) + (I + SD)
  by (rule
    wp-composition-fri-sampled-missing-full-cover-bound-from-domain-split)
    (rule proximity[OF support], rule index[OF support],
      rule domain-length[OF support], rule sampled-domain[OF support])
qed

lemma checked-staged-security-composition-fri-sampled-layer-chain-bound-from-full-cover-fresh-and-domain-split:
  assumes wf: staged-budget-wellformed budgets
  and controlled: staged-adversary-controlled budgets A
  and finite-B: ∧dg. finite (B dg)
  and envelope:
    ∧ data attacker-state dg composition-table.
      Some (data, attacker-state) ∈
        set-dist (execute (checked-staged-transcript-program A)
          adversary-initial-state) ⇒

```

composition-fri-multiround-bad-sets bad dg composition-table
 $\subseteq B \text{ dg}$
and *fresh-bound*:
wp-event (checked-staged-security-experiment-with-data-state A)
(staged-security-with-data-state-verifier-event
($\lambda s. \text{composition-fri-challenge-list-fresh-hit } s \text{ } B$))
adversary-initial-state $\leq F$
and *proximity*:
 $\wedge$ *data attacker-state.*
Some (data, attacker-state) $\in$
set-dist (execute (checked-staged-transcript-program A)
adversary-initial-state) $\implies$
wp-event verify-monad
(composition-fri-sampled-full-cover-proximity-failure
(verifier-state-from-adversary attacker-state
(staged-proof-transcript data)) bad)
(verifier-state-from-adversary attacker-state
(staged-proof-transcript data)) $\leq P$
and *index*:
 $\wedge$ *data attacker-state.*
Some (data, attacker-state) $\in$
set-dist (execute (checked-staged-transcript-program A)
adversary-initial-state) $\implies$
wp-event verify-monad
(composition-fri-sampled-full-cover-index-failure
(verifier-state-from-adversary attacker-state
(staged-proof-transcript data)))
(verifier-state-from-adversary attacker-state
(staged-proof-transcript data)) $\leq I$
and *domain-length*:
 $\wedge$ *data attacker-state.*
Some (data, attacker-state) $\in$
set-dist (execute (checked-staged-transcript-program A)
adversary-initial-state) $\implies$
wp-event verify-monad
(composition-fri-sampled-full-cover-domain-length-failure
(verifier-state-from-adversary attacker-state
(staged-proof-transcript data)))
(verifier-state-from-adversary attacker-state
(staged-proof-transcript data)) $\leq DL$
and *sampled-domain*:
 $\wedge$ *data attacker-state.*
Some (data, attacker-state) $\in$
set-dist (execute (checked-staged-transcript-program A)
adversary-initial-state) $\implies$
wp-event verify-monad
(composition-fri-sampled-full-cover-sampled-domain-failure
(verifier-state-from-adversary attacker-state
(staged-proof-transcript data)))

$$\begin{aligned}
& (\text{verifier-state-from-adversary attacker-state} \\
& \quad (\text{staged-proof-transcript data})) \leq SD
\end{aligned}$$

shows

$$\begin{aligned}
& \text{wp-event } (\text{checked-staged-security-experiment-with-data-state } A) \\
& \quad (\text{staged-security-with-data-state-verifier-event} \\
& \quad \quad \text{composition-fri-bad-with-sampled-layer-chain}) \\
& \text{adversary-initial-state} \\
& \leq \\
& (F + \\
& \quad \text{hash-relation-budget-value} \\
& \quad (\sum dg \in (UNIV :: 'f \text{ set}). \\
& \quad \quad \text{card } (B \text{ dg}) * \text{ceil-log } (\text{maxDegree} + 1)) \\
& \quad (\text{staged-attacker-query-budget budgets} + \\
& \quad \quad \text{staged-challenge-query-budget})) + \\
& (P + I + (I + DL) + (I + SD))
\end{aligned}$$

proof –

have *full-bound*:

$$\begin{aligned}
& \text{wp-event } (\text{checked-staged-security-experiment-with-data-state } A) \\
& \quad (\text{staged-security-with-data-state-verifier-event} \\
& \quad \quad (\lambda s. \text{composition-fri-bad-with-full-cover-layer-chain } s \text{ bad})) \\
& \text{adversary-initial-state} \\
& \leq \\
& F + \\
& \quad \text{hash-relation-budget-value} \\
& \quad (\sum dg \in (UNIV :: 'f \text{ set}). \\
& \quad \quad \text{card } (B \text{ dg}) * \text{ceil-log } (\text{maxDegree} + 1)) \\
& \quad (\text{staged-attacker-query-budget budgets} + \\
& \quad \quad \text{staged-challenge-query-budget})
\end{aligned}$$

by (rule

$$\begin{aligned}
& \text{checked-staged-security-composition-fri-full-cover-layer-chain-bound-from-fresh-and-prequery} \\
& \quad [\text{OF wf controlled finite-B envelope fresh-bound}]
\end{aligned}$$

have *missing-bound*:

$$\begin{aligned}
& \text{wp-event } (\text{checked-staged-security-experiment-with-data-state } A) \\
& \quad (\text{staged-security-with-data-state-verifier-event} \\
& \quad \quad (\lambda s. \text{composition-fri-sampled-missing-full-cover } s \text{ bad})) \\
& \text{adversary-initial-state} \leq P + I + (I + DL) + (I + SD)
\end{aligned}$$

by (rule

$$\begin{aligned}
& \text{checked-staged-security-composition-fri-sampled-missing-full-cover-bound-from-domain-split} \\
& \quad [\text{OF proximity index domain-length sampled-domain}]
\end{aligned}$$

have *wp-event* (checked-staged-security-experiment-with-data-state A)

$$\begin{aligned}
& (\text{staged-security-with-data-state-verifier-event} \\
& \quad \text{composition-fri-bad-with-sampled-layer-chain}) \\
& \text{adversary-initial-state} \leq \\
& \text{wp-event } (\text{checked-staged-security-experiment-with-data-state } A) \\
& \quad (\lambda out. \\
& \quad \quad \text{staged-security-with-data-state-verifier-event} \\
& \quad \quad (\lambda s. \text{composition-fri-bad-with-full-cover-layer-chain } s \text{ bad}) \\
& \quad \quad out \vee \\
& \quad \quad \text{staged-security-with-data-state-verifier-event}
\end{aligned}$$

```

      (λs. composition-fri-sampled-missing-full-cover s bad) out)
    adversary-initial-state
  by (rule wp-event-mono-on-support)
    (auto simp: staged-security-with-data-state-verifier-event-def
      dest: composition-fri-sampled-imp-full-cover-or-missing
      split: option.splits prod.splits)
  also have ... ≤
    wp-event (checked-staged-security-experiment-with-data-state A)
      (staged-security-with-data-state-verifier-event
        (λs. composition-fri-bad-with-full-cover-layer-chain s bad))
    adversary-initial-state +
    wp-event (checked-staged-security-experiment-with-data-state A)
      (staged-security-with-data-state-verifier-event
        (λs. composition-fri-sampled-missing-full-cover s bad))
    adversary-initial-state
  by (rule wp-event-union-bound)
  also have ... ≤
    (F +
      hash-relation-budget-value
      (∑ dg ∈ (UNIV :: 'f set).
        card (B dg) * ceil-log (maxDegree + 1))
      (staged-attacker-query-budget budgets +
        staged-challenge-query-budget)) +
    (P + I + (I + DL) + (I + SD)))
  by (rule add-mono[OF full-bound missing-bound])
  finally show ?thesis .
qed

```

lemma *checked-staged-security-composition-fri-canonical-sampled-layer-chain-bound-from-full-cover-fresh-and-domain*

```

assumes wf: staged-budget-wellformed budgets
and controlled: staged-adversary-controlled budgets A
and finite-B: ∧ dg. finite (B dg)
and envelope:
  ∧ data attacker-state dg composition-table.
    Some (data, attacker-state) ∈
      set-dist (execute (checked-staged-transcript-program A)
        adversary-initial-state) ⇒
      composition-fri-multiround-bad-sets bad dg composition-table
      ⊆ B dg
and fresh-bound:
  wp-event (checked-staged-security-experiment-with-data-state A)
    (staged-security-with-data-state-verifier-event
      (λs. composition-fri-challenge-list-fresh-hit s B))
    adversary-initial-state ≤ F
and proximity:
  ∧ data attacker-state.
    Some (data, attacker-state) ∈
      set-dist (execute (checked-staged-transcript-program A)
        adversary-initial-state) ⇒

```

$$\begin{aligned} & \text{wp-event verify-monad} \\ & \quad (\text{composition-fri-sampled-full-cover-proximity-failure} \\ & \quad \quad (\text{verifier-state-from-adversary attacker-state} \\ & \quad \quad \quad (\text{staged-proof-transcript data})) \text{ bad}) \\ & \quad (\text{verifier-state-from-adversary attacker-state} \\ & \quad \quad (\text{staged-proof-transcript data})) \leq P \\ & \text{and index:} \\ & \wedge \text{data attacker-state.} \\ & \quad \text{Some (data, attacker-state)} \in \\ & \quad \quad \text{set-dist (execute (checked-staged-transcript-program A)} \\ & \quad \quad \quad \text{adversary-initial-state)} \implies \\ & \quad \text{wp-event verify-monad} \\ & \quad \quad (\text{composition-fri-sampled-full-cover-index-failure} \\ & \quad \quad \quad (\text{verifier-state-from-adversary attacker-state} \\ & \quad \quad \quad \quad (\text{staged-proof-transcript data}))) \\ & \quad \quad (\text{verifier-state-from-adversary attacker-state} \\ & \quad \quad \quad (\text{staged-proof-transcript data})) \leq I \\ & \text{and domain-length:} \\ & \wedge \text{data attacker-state.} \\ & \quad \text{Some (data, attacker-state)} \in \\ & \quad \quad \text{set-dist (execute (checked-staged-transcript-program A)} \\ & \quad \quad \quad \text{adversary-initial-state)} \implies \\ & \quad \text{wp-event verify-monad} \\ & \quad \quad (\text{composition-fri-sampled-full-cover-domain-length-failure} \\ & \quad \quad \quad (\text{verifier-state-from-adversary attacker-state} \\ & \quad \quad \quad \quad (\text{staged-proof-transcript data}))) \\ & \quad \quad (\text{verifier-state-from-adversary attacker-state} \\ & \quad \quad \quad (\text{staged-proof-transcript data})) \leq DL \\ & \text{and sampled-domain:} \\ & \wedge \text{data attacker-state.} \\ & \quad \text{Some (data, attacker-state)} \in \\ & \quad \quad \text{set-dist (execute (checked-staged-transcript-program A)} \\ & \quad \quad \quad \text{adversary-initial-state)} \implies \\ & \quad \text{wp-event verify-monad} \\ & \quad \quad (\text{composition-fri-sampled-full-cover-sampled-domain-failure} \\ & \quad \quad \quad (\text{verifier-state-from-adversary attacker-state} \\ & \quad \quad \quad \quad (\text{staged-proof-transcript data}))) \\ & \quad \quad (\text{verifier-state-from-adversary attacker-state} \\ & \quad \quad \quad (\text{staged-proof-transcript data})) \leq SD \\ & \text{shows} \\ & \quad \text{wp-event (checked-staged-security-experiment-with-data-state A)} \\ & \quad \quad (\text{staged-security-with-data-state-verifier-event} \\ & \quad \quad \quad \text{composition-fri-bad-with-canonical-sampled-layer-chain}) \\ & \quad \text{adversary-initial-state} \\ & \quad \leq \\ & \quad (F + \\ & \quad \quad \text{hash-relation-budget-value} \\ & \quad \quad (\sum dg \in (\text{UNIV} :: 'f \text{ set}). \\ & \quad \quad \quad \text{card (B dg) * ceil-log (maxDegree + 1)})) \end{aligned}$$

```

      (staged-attacker-query-budget budgets +
       staged-challenge-query-budget)) +
      (P + I + (I + DL) + (I + SD))
proof -
  have wp-event (checked-staged-security-experiment-with-data-state A)
    (staged-security-with-data-state-verifier-event
     composition-fri-bad-with-canonical-sampled-layer-chain)
    adversary-initial-state ≤
    wp-event (checked-staged-security-experiment-with-data-state A)
      (staged-security-with-data-state-verifier-event
       composition-fri-bad-with-sampled-layer-chain)
      adversary-initial-state
  by (rule wp-event-mono-on-support)
    (auto simp: staged-security-with-data-state-verifier-event-def
     intro: composition-fri-bad-with-canonical-sampled-layer-chain-imp-sampled
     split: option.splits prod.splits)
  also have ... ≤
    (F +
     hash-relation-budget-value
     (∑ dg ∈ (UNIV :: 'f set).
      card (B dg) * ceil-log (maxDegree + 1))
     (staged-attacker-query-budget budgets +
      staged-challenge-query-budget)) +
    (P + I + (I + DL) + (I + SD))
  by (rule
    checked-staged-security-composition-fri-sampled-layer-chain-bound-from-full-cover-fresh-and-domain-split
    [OF wf controlled finite-B envelope fresh-bound proximity index
     domain-length sampled-domain])
  finally show ?thesis .
qed

lemma checked-staged-security-trace-fri-header-tied-sampled-layer-chain-bound-from-full-cover-fresh-and-domain-split
assumes wf: staged-budget-wellformed budgets
and controlled: staged-adversary-controlled budgets A
and finite-B: finite B
and envelope:
  ∧ data attacker-state trace-table.
  Some (data, attacker-state) ∈
    set-dist (execute (checked-staged-transcript-program A)
      adversary-initial-state) ⇒
    trace-fri-multiround-bad-sets bad trace-table ⊆ B
and fresh-bound:
  wp-event (checked-staged-security-experiment-with-data-state A)
    (staged-security-with-data-state-verifier-event
     (λs. trace-fri-challenge-list-fresh-hit s B))
    adversary-initial-state ≤ F
and proximity:
  ∧ data attacker-state.
  Some (data, attacker-state) ∈

```

$\text{set-dist } (\text{execute } (\text{checked-staged-transcript-program } A) \text{ adversary-initial-state}) \implies$
 $\text{wp-event verify-monad}$
 $(\text{trace-fri-sampled-full-cover-proximity-failure}$
 $(\text{verifier-state-from-adversary attacker-state}$
 $(\text{staged-proof-transcript data})) \text{ bad})$
 $(\text{verifier-state-from-adversary attacker-state}$
 $(\text{staged-proof-transcript data})) \leq P$
and index:
 $\wedge \text{data attacker-state.}$
 $\text{Some } (data, \text{attacker-state}) \in$
 $\text{set-dist } (\text{execute } (\text{checked-staged-transcript-program } A) \text{ adversary-initial-state}) \implies$
 $\text{wp-event verify-monad}$
 $(\text{trace-fri-sampled-full-cover-index-failure}$
 $(\text{verifier-state-from-adversary attacker-state}$
 $(\text{staged-proof-transcript data})))$
 $(\text{verifier-state-from-adversary attacker-state}$
 $(\text{staged-proof-transcript data})) \leq I$
and domain-length:
 $\wedge \text{data attacker-state.}$
 $\text{Some } (data, \text{attacker-state}) \in$
 $\text{set-dist } (\text{execute } (\text{checked-staged-transcript-program } A) \text{ adversary-initial-state}) \implies$
 $\text{wp-event verify-monad}$
 $(\text{trace-fri-sampled-full-cover-domain-length-failure}$
 $(\text{verifier-state-from-adversary attacker-state}$
 $(\text{staged-proof-transcript data})))$
 $(\text{verifier-state-from-adversary attacker-state}$
 $(\text{staged-proof-transcript data})) \leq DL$
and sampled-domain:
 $\wedge \text{data attacker-state.}$
 $\text{Some } (data, \text{attacker-state}) \in$
 $\text{set-dist } (\text{execute } (\text{checked-staged-transcript-program } A) \text{ adversary-initial-state}) \implies$
 $\text{wp-event verify-monad}$
 $(\text{trace-fri-sampled-full-cover-sampled-domain-failure}$
 $(\text{verifier-state-from-adversary attacker-state}$
 $(\text{staged-proof-transcript data})))$
 $(\text{verifier-state-from-adversary attacker-state}$
 $(\text{staged-proof-transcript data})) \leq SD$
shows
 $\text{wp-event } (\text{checked-staged-security-experiment-with-data-state } A)$
 $(\text{staged-security-with-data-state-verifier-event}$
 $\text{trace-fri-bad-with-header-tied-sampled-layer-chain})$
 $\text{adversary-initial-state}$
 $\leq$
 $(F +$
 $\text{hash-relation-budget-value } (\text{card } B * \text{ceil-log clength}))$

$(\text{staged-attacker-query-budget } budgets + \text{staged-challenge-query-budget})) +$
 $(P + I + (I + DL) + (I + SD))$

proof –

have *full-bound*:

$\text{wp-event } (\text{checked-staged-security-experiment-with-data-state } A)$
 $(\text{staged-security-with-data-state-verifier-event } (\lambda s. \text{trace-fri-bad-with-full-cover-layer-chain } s \text{ bad}))$
 $\text{adversary-initial-state}$
 $\leq$
 $F +$
 $\text{hash-relation-budget-value } (\text{card } B * \text{ceil-log } \text{clength})$
 $(\text{staged-attacker-query-budget } budgets + \text{staged-challenge-query-budget})$

by (rule
 $\text{checked-staged-security-trace-fri-full-cover-layer-chain-bound-from-fresh-and-prequery}$
 $[OF \text{ wf controlled finite-}B \text{ envelope fresh-bound}]$)

have *missing-bound*:

$\text{wp-event } (\text{checked-staged-security-experiment-with-data-state } A)$
 $(\text{staged-security-with-data-state-verifier-event } (\lambda s. \text{trace-fri-sampled-missing-full-cover } s \text{ bad}))$
 $\text{adversary-initial-state} \leq P + I + (I + DL) + (I + SD)$

by (rule
 $\text{checked-staged-security-trace-fri-sampled-missing-full-cover-bound-from-domain-split}$
 $[OF \text{ proximity index domain-length sampled-domain}]$)

have $\text{wp-event } (\text{checked-staged-security-experiment-with-data-state } A)$
 $(\text{staged-security-with-data-state-verifier-event } \text{trace-fri-bad-with-header-tied-sampled-layer-chain})$
 $\text{adversary-initial-state} \leq$
 $\text{wp-event } (\text{checked-staged-security-experiment-with-data-state } A)$
 $(\lambda out.$
 $\text{staged-security-with-data-state-verifier-event}$
 $(\lambda s. \text{trace-fri-bad-with-full-cover-layer-chain } s \text{ bad}) \text{ out} \vee$
 $\text{staged-security-with-data-state-verifier-event}$
 $(\lambda s. \text{trace-fri-sampled-missing-full-cover } s \text{ bad}) \text{ out})$
 $\text{adversary-initial-state}$

by (rule *wp-event-mono-on-support*)
 $(\text{auto simp: staged-security-with-data-state-verifier-event-def}$
 $\text{dest! : trace-fri-header-tied-sampled-layer-chain-imp-sampled-layer-chain}$
 $\text{dest: trace-fri-sampled-imp-full-cover-or-missing}$
 $\text{split: option.splits prod.splits})$

also have $\dots \leq$
 $\text{wp-event } (\text{checked-staged-security-experiment-with-data-state } A)$
 $(\text{staged-security-with-data-state-verifier-event } (\lambda s. \text{trace-fri-bad-with-full-cover-layer-chain } s \text{ bad}))$
 $\text{adversary-initial-state} +$
 $\text{wp-event } (\text{checked-staged-security-experiment-with-data-state } A)$
 $(\text{staged-security-with-data-state-verifier-event } (\lambda s. \text{trace-fri-sampled-missing-full-cover } s \text{ bad}))$

adversary-initial-state
by (rule *wp-event-union-bound*)
also have ... $\leq$
 ($F +$
 hash-relation-budget-value (*card B * ceil-log clength*)
 (*staged-attacker-query-budget budgets +*
 staged-challenge-query-budget)) +
 ($P + I + (I + DL) + (I + SD)$)
by (rule *add-mono[OF full-bound missing-bound]*)
finally show ?thesis .
qed

lemma *checked-staged-security-composition-fri-verifier-tied-sampled-layer-chain-bound-from-full-cover-fresh-and-dom*

assumes *wf: staged-budget-wellformed budgets*
and *controlled: staged-adversary-controlled budgets A*
and *finite-B: $\bigwedge dg. \text{finite } (B \ dg)$*
and *envelope:*
 $\bigwedge \text{data attacker-state } dg \text{ composition-table.}$
 Some (*data, attacker-state*) $\in$
 set-dist (*execute* (*checked-staged-transcript-program A*)
 adversary-initial-state) $\implies$
 composition-fri-multiround-bad-sets bad dg composition-table
 $\subseteq B \ dg$
and *fresh-bound:*
wp-event (*checked-staged-security-experiment-with-data-state A*)
 (*staged-security-with-data-state-verifier-event*
 ($\lambda s. \text{composition-fri-challenge-list-fresh-hit } s \ B$))
 adversary-initial-state $\leq F$
and *proximity:*
 $\bigwedge \text{data attacker-state.}$
 Some (*data, attacker-state*) $\in$
 set-dist (*execute* (*checked-staged-transcript-program A*)
 adversary-initial-state) $\implies$
 wp-event verify-monad
 (*composition-fri-sampled-full-cover-proximity-failure*
 (*verifier-state-from-adversary attacker-state*
 (*staged-proof-transcript data*)) *bad*)
 (*verifier-state-from-adversary attacker-state*
 (*staged-proof-transcript data*)) $\leq P$
and *index:*
 $\bigwedge \text{data attacker-state.}$
 Some (*data, attacker-state*) $\in$
 set-dist (*execute* (*checked-staged-transcript-program A*)
 adversary-initial-state) $\implies$
 wp-event verify-monad
 (*composition-fri-sampled-full-cover-index-failure*
 (*verifier-state-from-adversary attacker-state*
 (*staged-proof-transcript data*)))
 (*verifier-state-from-adversary attacker-state*

$(\text{staged-proof-transcript data}) \leq I$
and *domain-length*:
 $\wedge \text{data attacker-state.}$
 $\text{Some (data, attacker-state)} \in$
 $\text{set-dist (execute (checked-staged-transcript-program A)}$
 $\text{adversary-initial-state)} \implies$
 $\text{wp-event verify-monad}$
 $(\text{composition-fri-sampled-full-cover-domain-length-failure}$
 $(\text{verifier-state-from-adversary attacker-state}$
 $(\text{staged-proof-transcript data})))$
 $(\text{verifier-state-from-adversary attacker-state}$
 $(\text{staged-proof-transcript data})) \leq DL$
and *sampling-domain*:
 $\wedge \text{data attacker-state.}$
 $\text{Some (data, attacker-state)} \in$
 $\text{set-dist (execute (checked-staged-transcript-program A)}$
 $\text{adversary-initial-state)} \implies$
 $\text{wp-event verify-monad}$
 $(\text{composition-fri-sampled-full-cover-sampled-domain-failure}$
 $(\text{verifier-state-from-adversary attacker-state}$
 $(\text{staged-proof-transcript data})))$
 $(\text{verifier-state-from-adversary attacker-state}$
 $(\text{staged-proof-transcript data})) \leq SD$
shows
 $\text{wp-event (checked-staged-security-experiment-with-data-state A)}$
 $(\text{staged-security-with-data-state-verifier-event}$
 $\text{composition-fri-bad-with-verifier-tied-sampled-layer-chain})$
 $\text{adversary-initial-state}$
 $\leq$
 $(F +$
 $\text{hash-relation-budget-value}$
 $(\sum dg \in (\text{UNIV} :: 'f \text{ set}).$
 $\text{card (B dg) * ceil-log (maxDegree + 1))}$
 $(\text{staged-attacker-query-budget budgets} +$
 $\text{staged-challenge-query-budget})) +$
 $(P + I + (I + DL) + (I + SD))$
proof –
have *full-bound*:
 $\text{wp-event (checked-staged-security-experiment-with-data-state A)}$
 $(\text{staged-security-with-data-state-verifier-event}$
 $(\lambda s. \text{composition-fri-bad-with-full-cover-layer-chain s bad}))$
 $\text{adversary-initial-state}$
 $\leq$
 $F +$
 $\text{hash-relation-budget-value}$
 $(\sum dg \in (\text{UNIV} :: 'f \text{ set}).$
 $\text{card (B dg) * ceil-log (maxDegree + 1))}$
 $(\text{staged-attacker-query-budget budgets} +$
 $\text{staged-challenge-query-budget})$

by (rule
 checked-staged-security-composition-fri-full-cover-layer-chain-bound-from-fresh-and-prequery
 [*OF wf controlled finite-B envelope fresh-bound*])
 have *missing-bound*:
 wp-event (*checked-staged-security-experiment-with-data-state A*)
 (*staged-security-with-data-state-verifier-event*
 ($\lambda s. \text{composition-fri-sampled-missing-full-cover } s \text{ bad}$))
 adversary-initial-state $\leq P + I + (I + DL) + (I + SD)$
 by (rule
 checked-staged-security-composition-fri-sampled-missing-full-cover-bound-from-domain-split
 [*OF proximity index domain-length sampled-domain*])
 have wp-event (*checked-staged-security-experiment-with-data-state A*)
 (*staged-security-with-data-state-verifier-event*
 composition-fri-bad-with-verifier-tied-sampled-layer-chain)
 adversary-initial-state $\leq$
 wp-event (*checked-staged-security-experiment-with-data-state A*)
 ($\lambda out.$
 staged-security-with-data-state-verifier-event
 ($\lambda s. \text{composition-fri-bad-with-full-cover-layer-chain } s \text{ bad}$)
 out $\vee$
 staged-security-with-data-state-verifier-event
 ($\lambda s. \text{composition-fri-sampled-missing-full-cover } s \text{ bad}$)
 out)
 adversary-initial-state
 by (rule *wp-event-mono-on-support*)
 (*auto simp: staged-security-with-data-state-verifier-event-def*
 dest!: composition-fri-verifier-tied-sampled-layer-chain-imp-sampled-layer-chain
 dest: composition-fri-sampled-imp-full-cover-or-missing
 split: option.splits prod.splits)
 also have ... $\leq$
 wp-event (*checked-staged-security-experiment-with-data-state A*)
 (*staged-security-with-data-state-verifier-event*
 ($\lambda s. \text{composition-fri-bad-with-full-cover-layer-chain } s \text{ bad}$))
 adversary-initial-state +
 wp-event (*checked-staged-security-experiment-with-data-state A*)
 (*staged-security-with-data-state-verifier-event*
 ($\lambda s. \text{composition-fri-sampled-missing-full-cover } s \text{ bad}$))
 adversary-initial-state
 by (rule *wp-event-union-bound*)
 also have ... $\leq$
 (*F* +
 hash-relation-budget-value
 ($\sum dg \in (UNIV :: 'f \text{ set}).$
 card (*B dg*) * *ceil-log* (*maxDegree* + 1))
 (*staged-attacker-query-budget budgets* +
 staged-challenge-query-budget)) +
 (*P* + *I* + (*I* + *DL*) + (*I* + *SD*))
 by (rule *add-mono[OF full-bound missing-bound]*)
 finally show ?thesis .

qed

lemma *checked-staged-security-trace-fri-sampled-missing-full-cover-bound-from-witness-domain:*

assumes *proximity:*

$\wedge$ *data attacker-state.*

Some (data, attacker-state) $\in$
set-dist (execute (checked-staged-transcript-program A)
adversary-initial-state) $\implies$

wp-event verify-monad

(trace-fri-sampled-full-cover-proximity-failure
(verifier-state-from-adversary attacker-state
(staged-proof-transcript data)) bad)

(verifier-state-from-adversary attacker-state
(staged-proof-transcript data)) $\leq P$

and *index:*

$\wedge$ *data attacker-state.*

Some (data, attacker-state) $\in$
set-dist (execute (checked-staged-transcript-program A)
adversary-initial-state) $\implies$

wp-event verify-monad

(trace-fri-sampled-full-cover-index-failure
(verifier-state-from-adversary attacker-state
(staged-proof-transcript data)))

(verifier-state-from-adversary attacker-state
(staged-proof-transcript data)) $\leq I$

and *witness-length:*

$\wedge$ *data attacker-state.*

Some (data, attacker-state) $\in$
set-dist (execute (checked-staged-transcript-program A)
adversary-initial-state) $\implies$

wp-event verify-monad

(trace-fri-sampled-full-cover-witness-length-failure
(verifier-state-from-adversary attacker-state
(staged-proof-transcript data)))

(verifier-state-from-adversary attacker-state
(staged-proof-transcript data)) $\leq W$

and *domain:*

$\wedge$ *data attacker-state.*

Some (data, attacker-state) $\in$
set-dist (execute (checked-staged-transcript-program A)
adversary-initial-state) $\implies$

wp-event verify-monad

(trace-fri-sampled-full-cover-domain-failure
(verifier-state-from-adversary attacker-state
(staged-proof-transcript data)))

(verifier-state-from-adversary attacker-state
(staged-proof-transcript data)) $\leq D$

shows

wp-event (checked-staged-security-experiment-with-data-state A)

```

    (staged-security-with-data-state-verifier-event
     (λs. trace-fri-sampled-missing-full-cover s bad))
    adversary-initial-state ≤ P + I + W + D
proof (rule checked-staged-security-with-data-state-verifier-event-bound-from-cont)
  show ∧s. ¬ trace-fri-sampled-missing-full-cover s bad None
    by (rule trace-fri-sampled-missing-full-cover-not-None)
next
  fix data attacker-state
  assume support:
    Some (data, attacker-state) ∈
      set-dist (execute (checked-staged-transcript-program A)
                    adversary-initial-state)
  let ?s =
    verifier-state-from-adversary attacker-state
    (staged-proof-transcript data)
  show wp-event verify-monad
    (trace-fri-sampled-missing-full-cover ?s bad) ?s ≤
      P + I + W + D
  by (rule
    wp-trace-fri-sampled-missing-full-cover-bound-from-witness-length-subcases)
    (rule proximity[OF support], rule index[OF support],
     rule witness-length[OF support], rule domain[OF support])
qed

lemma checked-staged-security-composition-fri-sampled-missing-full-cover-bound-from-witness-domain:
  assumes proximity:
    ∧data attacker-state.
      Some (data, attacker-state) ∈
        set-dist (execute (checked-staged-transcript-program A)
                      adversary-initial-state) ⇒
        wp-event verify-monad
          (composition-fri-sampled-full-cover-proximity-failure
           (verifier-state-from-adversary attacker-state
            (staged-proof-transcript data)) bad)
          (verifier-state-from-adversary attacker-state
           (staged-proof-transcript data)) ≤ P
  and index:
    ∧data attacker-state.
      Some (data, attacker-state) ∈
        set-dist (execute (checked-staged-transcript-program A)
                      adversary-initial-state) ⇒
        wp-event verify-monad
          (composition-fri-sampled-full-cover-index-failure
           (verifier-state-from-adversary attacker-state
            (staged-proof-transcript data)))
          (verifier-state-from-adversary attacker-state
           (staged-proof-transcript data)) ≤ I
  and witness-length:
    ∧data attacker-state.

```

Some (data, attacker-state) $\in$
 set-dist (execute (checked-staged-transcript-program A)
 adversary-initial-state) $\implies$
 wp-event verify-monad
 (composition-fri-sampled-full-cover-witness-length-failure
 (verifier-state-from-adversary attacker-state
 (staged-proof-transcript data)))
 (verifier-state-from-adversary attacker-state
 (staged-proof-transcript data)) $\leq W$
and domain:
 $\wedge$ data attacker-state.
 Some (data, attacker-state) $\in$
 set-dist (execute (checked-staged-transcript-program A)
 adversary-initial-state) $\implies$
 wp-event verify-monad
 (composition-fri-sampled-full-cover-domain-failure
 (verifier-state-from-adversary attacker-state
 (staged-proof-transcript data)))
 (verifier-state-from-adversary attacker-state
 (staged-proof-transcript data)) $\leq D$
shows
 wp-event (checked-staged-security-experiment-with-data-state A)
 (staged-security-with-data-state-verifier-event
 (λs . composition-fri-sampled-missing-full-cover s bad))
 adversary-initial-state $\leq P + I + W + D$
proof (rule checked-staged-security-with-data-state-verifier-event-bound-from-cont)
show $\wedge s$. $\neg$ composition-fri-sampled-missing-full-cover s bad None
by (rule composition-fri-sampled-missing-full-cover-not-None)
next
fix data attacker-state
assume support:
 Some (data, attacker-state) $\in$
 set-dist (execute (checked-staged-transcript-program A)
 adversary-initial-state)
let ?s =
 verifier-state-from-adversary attacker-state
 (staged-proof-transcript data)
show wp-event verify-monad
 (composition-fri-sampled-missing-full-cover ?s bad) ?s $\leq$
 P + I + W + D
by (rule
 wp-composition-fri-sampled-missing-full-cover-bound-from-witness-length-subcases)
 (rule proximity[OF support], rule index[OF support],
 rule witness-length[OF support], rule domain[OF support])
qed

lemma checked-staged-security-trace-fri-header-tied-sampled-layer-chain-bound-from-full-cover-fresh-and-witness-dom
assumes wf: staged-budget-wellformed budgets
and controlled: staged-adversary-controlled budgets A

and *finite-B*: *finite B*
and *envelope*:
 $\wedge$ *data attacker-state trace-table*.
Some (data, attacker-state) $\in$
set-dist (execute (checked-staged-transcript-program A)
adversary-initial-state) $\implies$
trace-fri-multiround-bad-sets bad trace-table $\subseteq B$
and *fresh-bound*:
wp-event (checked-staged-security-experiment-with-data-state A)
(staged-security-with-data-state-verifier-event
(λs . trace-fri-challenge-list-fresh-hit s B))
adversary-initial-state $\leq F$
and *proximity*:
 $\wedge$ *data attacker-state*.
Some (data, attacker-state) $\in$
set-dist (execute (checked-staged-transcript-program A)
adversary-initial-state) $\implies$
wp-event verify-monad
(trace-fri-sampled-full-cover-proximity-failure
(verifier-state-from-adversary attacker-state
(staged-proof-transcript data)) bad)
(verifier-state-from-adversary attacker-state
(staged-proof-transcript data)) $\leq P$
and *index*:
 $\wedge$ *data attacker-state*.
Some (data, attacker-state) $\in$
set-dist (execute (checked-staged-transcript-program A)
adversary-initial-state) $\implies$
wp-event verify-monad
(trace-fri-sampled-full-cover-index-failure
(verifier-state-from-adversary attacker-state
(staged-proof-transcript data)))
(verifier-state-from-adversary attacker-state
(staged-proof-transcript data)) $\leq I$
and *witness-length*:
 $\wedge$ *data attacker-state*.
Some (data, attacker-state) $\in$
set-dist (execute (checked-staged-transcript-program A)
adversary-initial-state) $\implies$
wp-event verify-monad
(trace-fri-sampled-full-cover-witness-length-failure
(verifier-state-from-adversary attacker-state
(staged-proof-transcript data)))
(verifier-state-from-adversary attacker-state
(staged-proof-transcript data)) $\leq W$
and *domain*:
 $\wedge$ *data attacker-state*.
Some (data, attacker-state) $\in$
set-dist (execute (checked-staged-transcript-program A)

$\text{adversary-initial-state}) \implies$
 $\text{wp-event verify-monad}$
 $(\text{trace-fri-sampled-full-cover-domain-failure}$
 $(\text{verifier-state-from-adversary attacker-state}$
 $(\text{staged-proof-transcript data})))$
 $(\text{verifier-state-from-adversary attacker-state}$
 $(\text{staged-proof-transcript data})) \leq D$
shows
 $\text{wp-event } (\text{checked-staged-security-experiment-with-data-state } A)$
 $(\text{staged-security-with-data-state-verifier-event}$
 $\text{trace-fri-bad-with-header-tied-sampled-layer-chain})$
 $\text{adversary-initial-state}$
 $\leq$
 $(F +$
 $\text{hash-relation-budget-value } (\text{card } B * \text{ceil-log } \text{clength})$
 $(\text{staged-attacker-query-budget budgets} +$
 $\text{staged-challenge-query-budget})) +$
 $(P + I + W + D)$
proof –
have *full-bound*:
 $\text{wp-event } (\text{checked-staged-security-experiment-with-data-state } A)$
 $(\text{staged-security-with-data-state-verifier-event}$
 $(\lambda s. \text{trace-fri-bad-with-full-cover-layer-chain } s \text{ bad}))$
 $\text{adversary-initial-state}$
 $\leq$
 $F +$
 $\text{hash-relation-budget-value } (\text{card } B * \text{ceil-log } \text{clength})$
 $(\text{staged-attacker-query-budget budgets} +$
 $\text{staged-challenge-query-budget})$
by (rule
 $\text{checked-staged-security-trace-fri-full-cover-layer-chain-bound-from-fresh-and-prequery}$
 $[\text{OF wf controlled finite-B envelope fresh-bound}])$
have *missing-bound*:
 $\text{wp-event } (\text{checked-staged-security-experiment-with-data-state } A)$
 $(\text{staged-security-with-data-state-verifier-event}$
 $(\lambda s. \text{trace-fri-sampled-missing-full-cover } s \text{ bad}))$
 $\text{adversary-initial-state} \leq P + I + W + D$
by (rule
 $\text{checked-staged-security-trace-fri-sampled-missing-full-cover-bound-from-witness-domain}$
 $[\text{OF proximity index witness-length domain}])$
have $\text{wp-event } (\text{checked-staged-security-experiment-with-data-state } A)$
 $(\text{staged-security-with-data-state-verifier-event}$
 $\text{trace-fri-bad-with-header-tied-sampled-layer-chain})$
 $\text{adversary-initial-state} \leq$
 $\text{wp-event } (\text{checked-staged-security-experiment-with-data-state } A)$
 $(\lambda \text{out.}$
 $\text{staged-security-with-data-state-verifier-event}$
 $(\lambda s. \text{trace-fri-bad-with-full-cover-layer-chain } s \text{ bad}) \text{ out} \vee$
 $\text{staged-security-with-data-state-verifier-event}$

```

      (λs. trace-fri-sampled-missing-full-cover s bad) out)
    adversary-initial-state
  by (rule wp-event-mono-on-support)
    (auto simp: staged-security-with-data-state-verifier-event-def
      dest!: trace-fri-header-tied-sampled-layer-chain-imp-sampled-layer-chain
      dest: trace-fri-sampled-imp-full-cover-or-missing
      split: option.splits prod.splits)
  also have ... ≤
    wp-event (checked-staged-security-experiment-with-data-state A)
      (staged-security-with-data-state-verifier-event
        (λs. trace-fri-bad-with-full-cover-layer-chain s bad))
    adversary-initial-state +
    wp-event (checked-staged-security-experiment-with-data-state A)
      (staged-security-with-data-state-verifier-event
        (λs. trace-fri-sampled-missing-full-cover s bad))
    adversary-initial-state
  by (rule wp-event-union-bound)
  also have ... ≤
    (F +
      hash-relation-budget-value (card B * ceil-log clength)
      (staged-attacker-query-budget budgets +
        staged-challenge-query-budget)) +
    (P + I + W + D)
  by (rule add-mono[OF full-bound missing-bound])
  finally show ?thesis .
qed

lemma checked-staged-security-trace-fri-sampled-layer-chain-bound-from-full-cover-fresh-and-witness-domain:
assumes wf: staged-budget-wellformed budgets
and controlled: staged-adversary-controlled budgets A
and finite-B: finite B
and envelope:
  ∧ data attacker-state trace-table.
  Some (data, attacker-state) ∈
    set-dist (execute (checked-staged-transcript-program A)
      adversary-initial-state) ⇒
    trace-fri-multi-round-bad-sets bad trace-table ⊆ B
and fresh-bound:
  wp-event (checked-staged-security-experiment-with-data-state A)
    (staged-security-with-data-state-verifier-event
      (λs. trace-fri-challenge-list-fresh-hit s B))
    adversary-initial-state ≤ F
and proximity:
  ∧ data attacker-state.
  Some (data, attacker-state) ∈
    set-dist (execute (checked-staged-transcript-program A)
      adversary-initial-state) ⇒
  wp-event verify-mono
    (trace-fri-sampled-full-cover-proximity-failure

```

$(\text{verifier-state-from-adversary attacker-state}$
 $(\text{staged-proof-transcript data})) \text{ bad})$
 $(\text{verifier-state-from-adversary attacker-state}$
 $(\text{staged-proof-transcript data})) \leq P$
and *index*:
 $\wedge \text{data attacker-state.}$
 $\text{Some (data, attacker-state)} \in$
 $\text{set-dist (execute (checked-staged-transcript-program A)}$
 $\text{adversary-initial-state)} \implies$
 $\text{wp-event verify-monad}$
 $(\text{trace-fri-sampled-full-cover-index-failure}$
 $(\text{verifier-state-from-adversary attacker-state}$
 $(\text{staged-proof-transcript data})))$
 $(\text{verifier-state-from-adversary attacker-state}$
 $(\text{staged-proof-transcript data})) \leq I$
and *witness-length*:
 $\wedge \text{data attacker-state.}$
 $\text{Some (data, attacker-state)} \in$
 $\text{set-dist (execute (checked-staged-transcript-program A)}$
 $\text{adversary-initial-state)} \implies$
 $\text{wp-event verify-monad}$
 $(\text{trace-fri-sampled-full-cover-witness-length-failure}$
 $(\text{verifier-state-from-adversary attacker-state}$
 $(\text{staged-proof-transcript data})))$
 $(\text{verifier-state-from-adversary attacker-state}$
 $(\text{staged-proof-transcript data})) \leq W$
and *domain*:
 $\wedge \text{data attacker-state.}$
 $\text{Some (data, attacker-state)} \in$
 $\text{set-dist (execute (checked-staged-transcript-program A)}$
 $\text{adversary-initial-state)} \implies$
 $\text{wp-event verify-monad}$
 $(\text{trace-fri-sampled-full-cover-domain-failure}$
 $(\text{verifier-state-from-adversary attacker-state}$
 $(\text{staged-proof-transcript data})))$
 $(\text{verifier-state-from-adversary attacker-state}$
 $(\text{staged-proof-transcript data})) \leq D$
shows
 $\text{wp-event (checked-staged-security-experiment-with-data-state A)}$
 $(\text{staged-security-with-data-state-verifier-event}$
 $\text{trace-fri-bad-with-sampled-layer-chain})$
 $\text{adversary-initial-state}$
 $\leq$
 $(F +$
 $\text{hash-relation-budget-value (card B * ceil-log clength)}$
 $(\text{staged-attacker-query-budget budgets} +$
 $\text{staged-challenge-query-budget})) +$
 $(P + I + W + D)$
proof –

have *full-bound*:

$$\begin{aligned} & \text{wp-event } (\text{checked-staged-security-experiment-with-data-state } A) \\ & \quad (\text{staged-security-with-data-state-verifier-event} \\ & \quad \quad (\lambda s. \text{trace-fri-bad-with-full-cover-layer-chain } s \text{ bad})) \\ & \text{adversary-initial-state} \\ & \leq \\ & F + \\ & \quad \text{hash-relation-budget-value } (\text{card } B * \text{ceil-log } \text{clength}) \\ & \quad (\text{staged-attacker-query-budget } \text{budgets} + \\ & \quad \quad \text{staged-challenge-query-budget}) \\ & \text{by (rule} \\ & \quad \text{checked-staged-security-trace-fri-full-cover-layer-chain-bound-from-fresh-and-prequery} \\ & \quad [\text{OF wf controlled finite-B envelope fresh-bound}]) \\ & \text{have } \text{missing-bound}: \\ & \text{wp-event } (\text{checked-staged-security-experiment-with-data-state } A) \\ & \quad (\text{staged-security-with-data-state-verifier-event} \\ & \quad \quad (\lambda s. \text{trace-fri-sampled-missing-full-cover } s \text{ bad})) \\ & \text{adversary-initial-state} \leq P + I + W + D \\ & \text{by (rule} \\ & \quad \text{checked-staged-security-trace-fri-sampled-missing-full-cover-bound-from-witness-domain} \\ & \quad [\text{OF proximity index witness-length domain}]) \\ & \text{have } \text{wp-event } (\text{checked-staged-security-experiment-with-data-state } A) \\ & \quad (\text{staged-security-with-data-state-verifier-event} \\ & \quad \quad \text{trace-fri-bad-with-sampled-layer-chain}) \\ & \text{adversary-initial-state} \leq \\ & \text{wp-event } (\text{checked-staged-security-experiment-with-data-state } A) \\ & \quad (\lambda \text{out.} \\ & \quad \quad \text{staged-security-with-data-state-verifier-event} \\ & \quad \quad \quad (\lambda s. \text{trace-fri-bad-with-full-cover-layer-chain } s \text{ bad}) \text{ out} \vee \\ & \quad \quad \text{staged-security-with-data-state-verifier-event} \\ & \quad \quad \quad (\lambda s. \text{trace-fri-sampled-missing-full-cover } s \text{ bad}) \text{ out}) \\ & \text{adversary-initial-state} \\ & \text{by (rule wp-event-mono-on-support)} \\ & \quad (\text{auto simp: staged-security-with-data-state-verifier-event-def} \\ & \quad \quad \text{dest: trace-fri-sampled-imp-full-cover-or-missing} \\ & \quad \quad \text{split: option.splits prod.splits}) \\ & \text{also have } \dots \leq \\ & \text{wp-event } (\text{checked-staged-security-experiment-with-data-state } A) \\ & \quad (\text{staged-security-with-data-state-verifier-event} \\ & \quad \quad (\lambda s. \text{trace-fri-bad-with-full-cover-layer-chain } s \text{ bad})) \\ & \text{adversary-initial-state} + \\ & \text{wp-event } (\text{checked-staged-security-experiment-with-data-state } A) \\ & \quad (\text{staged-security-with-data-state-verifier-event} \\ & \quad \quad (\lambda s. \text{trace-fri-sampled-missing-full-cover } s \text{ bad})) \\ & \text{adversary-initial-state} \\ & \text{by (rule wp-event-union-bound)} \\ & \text{also have } \dots \leq \\ & (F + \\ & \quad \text{hash-relation-budget-value } (\text{card } B * \text{ceil-log } \text{clength})
\end{aligned}$$

```

      (staged-attacker-query-budget budgets +
       staged-challenge-query-budget)) +
    (P + I + W + D)
  by (rule add-mono[OF full-bound missing-bound])
  finally show ?thesis .
qed

```

lemma *checked-staged-security-composition-fri-verifier-tied-sampled-layer-chain-bound-from-full-cover-fresh-and-witn*

```

assumes wf: staged-budget-wellformed budgets
and controlled: staged-adversary-controlled budgets A
and finite-B:  $\bigwedge dg. \text{finite } (B \ dg)$ 
and envelope:
 $\bigwedge \text{data attacker-state } dg \text{ composition-table.}$ 
  Some (data, attacker-state)  $\in$ 
    set-dist (execute (checked-staged-transcript-program A)
      adversary-initial-state)  $\implies$ 
    composition-fri-multiround-bad-sets bad dg composition-table
       $\subseteq B \ dg$ 
and fresh-bound:
wp-event (checked-staged-security-experiment-with-data-state A)
  (staged-security-with-data-state-verifier-event
    ( $\lambda s. \text{composition-fri-challenge-list-fresh-hit } s \ B$ ))
  adversary-initial-state  $\leq F$ 
and proximity:
 $\bigwedge \text{data attacker-state.}$ 
  Some (data, attacker-state)  $\in$ 
    set-dist (execute (checked-staged-transcript-program A)
      adversary-initial-state)  $\implies$ 
wp-event verify-monad
  (composition-fri-sampled-full-cover-proximity-failure
    (verifier-state-from-adversary attacker-state
      (staged-proof-transcript data)) bad)
  (verifier-state-from-adversary attacker-state
    (staged-proof-transcript data))  $\leq P$ 
and index:
 $\bigwedge \text{data attacker-state.}$ 
  Some (data, attacker-state)  $\in$ 
    set-dist (execute (checked-staged-transcript-program A)
      adversary-initial-state)  $\implies$ 
wp-event verify-monad
  (composition-fri-sampled-full-cover-index-failure
    (verifier-state-from-adversary attacker-state
      (staged-proof-transcript data)))
  (verifier-state-from-adversary attacker-state
    (staged-proof-transcript data))  $\leq I$ 
and witness-length:
 $\bigwedge \text{data attacker-state.}$ 
  Some (data, attacker-state)  $\in$ 
    set-dist (execute (checked-staged-transcript-program A)

```

$\text{adversary-initial-state}) \implies$
 $\text{wp-event verify-monad}$
 $(\text{composition-fri-sampled-full-cover-witness-length-failure}$
 $(\text{verifier-state-from-adversary attacker-state}$
 $(\text{staged-proof-transcript data})))$
 $(\text{verifier-state-from-adversary attacker-state}$
 $(\text{staged-proof-transcript data})) \leq W$
and domain:
 $\wedge \text{data attacker-state.}$
 $\text{Some (data, attacker-state)} \in$
 $\text{set-dist (execute (checked-staged-transcript-program A)}$
 $\text{adversary-initial-state}) \implies}$
 $\text{wp-event verify-monad}$
 $(\text{composition-fri-sampled-full-cover-domain-failure}$
 $(\text{verifier-state-from-adversary attacker-state}$
 $(\text{staged-proof-transcript data})))$
 $(\text{verifier-state-from-adversary attacker-state}$
 $(\text{staged-proof-transcript data})) \leq D$
shows
 $\text{wp-event (checked-staged-security-experiment-with-data-state A)}$
 $(\text{staged-security-with-data-state-verifier-event}$
 $\text{composition-fri-bad-with-verifier-tied-sampled-layer-chain})$
 $\text{adversary-initial-state}$
 $\leq$
 $(F +$
 $\text{hash-relation-budget-value}$
 $(\sum dg \in (\text{UNIV} :: 'f \text{ set}).$
 $\text{card (B dg) * ceil-log (maxDegree + 1))}$
 $(\text{staged-attacker-query-budget budgets} +$
 $\text{staged-challenge-query-budget})) +$
 $(P + I + W + D)$
proof –
have full-bound:
 $\text{wp-event (checked-staged-security-experiment-with-data-state A)}$
 $(\text{staged-security-with-data-state-verifier-event}$
 $(\lambda s. \text{composition-fri-bad-with-full-cover-layer-chain s bad}))$
 $\text{adversary-initial-state}$
 $\leq$
 $F +$
 $\text{hash-relation-budget-value}$
 $(\sum dg \in (\text{UNIV} :: 'f \text{ set}).$
 $\text{card (B dg) * ceil-log (maxDegree + 1))}$
 $(\text{staged-attacker-query-budget budgets} +$
 $\text{staged-challenge-query-budget})$
by (rule
 $\text{checked-staged-security-composition-fri-full-cover-layer-chain-bound-from-fresh-and-prequery}$
 $[\text{OF wf controlled finite-B envelope fresh-bound}])$
have missing-bound:
 $\text{wp-event (checked-staged-security-experiment-with-data-state A)}$

```

    (staged-security-with-data-state-verifier-event
     (λs. composition-fri-sampled-missing-full-cover s bad))
    adversary-initial-state ≤ P + I + W + D
  by (rule
    checked-staged-security-composition-fri-sampled-missing-full-cover-bound-from-witness-domain
    [OF proximity index witness-length domain])
  have wp-event (checked-staged-security-experiment-with-data-state A)
    (staged-security-with-data-state-verifier-event
     composition-fri-bad-with-verifier-tied-sampled-layer-chain)
    adversary-initial-state ≤
    wp-event (checked-staged-security-experiment-with-data-state A)
    (λout.
      staged-security-with-data-state-verifier-event
      (λs. composition-fri-bad-with-full-cover-layer-chain s bad)
      out ∨
      staged-security-with-data-state-verifier-event
      (λs. composition-fri-sampled-missing-full-cover s bad)
      out)
    adversary-initial-state
  by (rule wp-event-mono-on-support)
    (auto simp: staged-security-with-data-state-verifier-event-def
      dest!: composition-fri-verifier-tied-sampled-layer-chain-imp-sampled-layer-chain
      dest: composition-fri-sampled-imp-full-cover-or-missing
      split: option.splits prod.splits)
  also have ... ≤
    wp-event (checked-staged-security-experiment-with-data-state A)
    (staged-security-with-data-state-verifier-event
     (λs. composition-fri-bad-with-full-cover-layer-chain s bad))
    adversary-initial-state +
    wp-event (checked-staged-security-experiment-with-data-state A)
    (staged-security-with-data-state-verifier-event
     (λs. composition-fri-sampled-missing-full-cover s bad))
    adversary-initial-state
  by (rule wp-event-union-bound)
  also have ... ≤
    (F +
     hash-relation-budget-value
     (∑ dg ∈ (UNIV :: 'f set).
      card (B dg) * ceil-log (maxDegree + 1))
     (staged-attacker-query-budget budgets +
      staged-challenge-query-budget)) +
    (P + I + W + D)
  by (rule add-mono[OF full-bound missing-bound])
  finally show ?thesis .
qed

end

end

```

theory *Soundness-FRI-Derived-Active-Reductions*

imports

Soundness-FRI-Staged-Replay-Bounds

Soundness-FRI-Full-Cover-Staged-Bounds

begin

Derived active-FRI packaging.

This layer keeps the near-threshold staged replay theory stable. It packages the already proved trace and composition verifier-state reductions into the active-FRI interface consumed by the public route. It does not add protocol assumptions or change the FRI model.

context *soundness*

begin

lemma *trace-fri-header-tied-reduction-from-envelope-witness-domain-without-head-slot-and-selected-zero:*

assumes *future: trace-fri-future-fresh s*

and *envelope:*

$\bigwedge \text{trace-table. trace-fri-multiround-bad-sets bad trace-table} \subseteq B$

and *subset: $B \subseteq \text{fri-challenge-space } (\text{ceil-log clength})$*

and *challenge-bound:*

$\text{nnreal } (\text{card } B) / \text{nnreal } (\text{CARD}(f) \wedge \text{ceil-log clength}) \leq R$

and *proximity:*

wp-event verify-monad

$(\text{trace-fri-sampled-full-cover-proximity-failure } s \text{ bad}) s \leq P$

and *index:*

wp-event verify-monad

$(\text{trace-fri-sampled-full-cover-index-failure } s) s \leq I$

and *witness-length:*

wp-event verify-monad

$(\text{trace-fri-sampled-full-cover-witness-length-failure } s) s \leq W$

and *domain:*

wp-event verify-monad

$(\text{trace-fri-sampled-full-cover-domain-failure } s) s \leq D$

and *zero:*

wp-event verify-monad

$(\text{trace-fri-header-tied-zero-round-final-obstruction } s) s \leq z-b$

and *merkle:*

wp-event verify-monad (partial-merkle-inconsistency-bad s) s ≤ p-b

and *structural:*

wp-event verify-monad

$(\text{trace-fri-header-tied-sibling-mismatch-structural-gap } s) s \leq h-b$

and *next-bound:*

wp-event verify-monad

$(\text{trace-fri-header-tied-next-value-replay-gap } s) s \leq n-b$

and *successor:*

wp-event verify-monad

$(\text{trace-fri-header-tied-successor-replay-gap } s) \ s \leq s-b$
and total:
 $(R + (P + I + W + D)) +$
 $(((((p-b + h-b) + p-b) + n-b + s-b +$
 $(p-b + 0 + 0)) + p-b) + 0 + z-b)$
 $\leq \text{trace-fri-error}$
shows *trace-fri-header-tied-reduction s*
by (rule *trace-fri-header-tied-reduction-from-residual-obligations*)
(rule
trace-fri-header-tied-residual-obligations-from-envelope-witness-domain-without-head-slot-and-selected-zero
[*OF future envelope subset challenge-bound proximity index*
witness-length domain zero merkle structural next-bound successor
total])

lemma *composition-fri-verifier-tied-reduction-from-envelope-witness-domain-slot-zero:*

assumes *future: composition-fri-future-fresh s*
and envelope:
 $\bigwedge dg \ \text{composition-table.}$
 $\text{composition-fri-multi-round-bad-sets bad } dg \ \text{composition-table}$
 $\subseteq B \ dg$
and subset:
 $\bigwedge dg. \ B \ dg \subseteq \text{fri-challenge-space } (\text{ceil-log } (to\text{-nat } dg + 1))$
and challenge-bound:
 $\bigwedge dg. \ nnreal \ (\text{card } (B \ dg)) /$
 $nnreal \ (CARD('f) \wedge \text{ceil-log } (to\text{-nat } dg + 1)) \leq R$
and proximity:
 $wp\text{-event verify-monad}$
 $(\text{composition-fri-sampled-full-cover-proximity-failure } s \ \text{bad}) \ s$
 $\leq P$
and index:
 $wp\text{-event verify-monad}$
 $(\text{composition-fri-sampled-full-cover-index-failure } s) \ s \leq I$
and witness-length:
 $wp\text{-event verify-monad}$
 $(\text{composition-fri-sampled-full-cover-witness-length-failure } s) \ s$
 $\leq W$
and domain:
 $wp\text{-event verify-monad}$
 $(\text{composition-fri-sampled-full-cover-domain-failure } s) \ s \leq D$
and merkle:
 $wp\text{-event verify-monad } (\text{partial-merkle-inconsistency-bad } s) \ s \leq p-b$
and next-bound:
 $wp\text{-event verify-monad}$
 $(\text{composition-fri-verifier-tied-next-value-replay-gap } s) \ s \leq n-b$
and successor:
 $wp\text{-event verify-monad}$
 $(\text{composition-fri-verifier-tied-successor-replay-gap } s) \ s \leq s-b$
and final-bound:
 $wp\text{-event verify-monad}$

$(\text{composition-fri-verifier-tied-final-value-replay-gap } s) \ s \leq g\text{-}b$
and total:
 $(R + (P + I + W + D)) +$
 $((p\text{-}b + 0) + n\text{-}b + s\text{-}b + g\text{-}b + p\text{-}b) + 0)$
 $\leq \text{composition-fri-error}$
shows *composition-fri-verifier-tied-reduction* s
by (*rule composition-fri-verifier-tied-reduction-from-residual-obligations*)
(rule
composition-fri-verifier-tied-residual-obligations-from-envelope-witness-domain-slot-zero
[OF future envelope subset challenge-bound proximity index
witness-length domain merkle next-bound successor final-bound total])

lemma *composition-fri-verifier-tied-reduction-from-envelope-witness-domain-merkle:*
assumes *future: composition-fri-future-fresh* s
and envelope:
 $\bigwedge dg \text{ composition-table.}$
 $\text{composition-fri-multi-round-bad-sets bad } dg \text{ composition-table}$
 $\subseteq B \ dg$
and subset:
 $\bigwedge dg. B \ dg \subseteq \text{fri-challenge-space } (\text{ceil-log } (to\text{-nat } dg + 1))$
and challenge-bound:
 $\bigwedge dg. \text{nnreal } (\text{card } (B \ dg)) /$
 $\text{nnreal } (\text{CARD}'f \wedge \text{ceil-log } (to\text{-nat } dg + 1)) \leq R$
and proximity:
 $\text{wp-event verify-monad}$
 $(\text{composition-fri-sampled-full-cover-proximity-failure } s \text{ bad}) \ s$
 $\leq P$
and index:
 $\text{wp-event verify-monad}$
 $(\text{composition-fri-sampled-full-cover-index-failure } s) \ s \leq I$
and witness-length:
 $\text{wp-event verify-monad}$
 $(\text{composition-fri-sampled-full-cover-witness-length-failure } s) \ s$
 $\leq W$
and domain:
 $\text{wp-event verify-monad}$
 $(\text{composition-fri-sampled-full-cover-domain-failure } s) \ s \leq D$
and merkle:
 $\text{wp-event verify-monad } (\text{partial-merkle-inconsistency-bad } s) \ s \leq M$
and total:
 $(R + (P + I + W + D)) +$
 $((M + 0) + (M + 0) + (M + 0) + (M + 0 + 0) + M) + 0)$
 $\leq \text{composition-fri-error}$
shows *composition-fri-verifier-tied-reduction* s
proof –
have slot:
 $\text{wp-event verify-monad}$
 $(\text{composition-fri-verifier-tied-authenticated-slot-recorded-chunk-gap } s)$
 $s \leq 0$

by (rule
 wp-composition-fri-verifier-tied-authenticated-slot-recorded-chunk-gap-zero)
 have *next-bound*:
 wp-event verify-monad
 (*composition-fri-verifier-tied-next-value-replay-gap s*) $s \leq M + 0$
 by (rule
 wp-composition-fri-verifier-tied-next-value-replay-gap-bound-from-merkle-and-slot
 (rule *merkle*, rule *slot*)
 have *successor*:
 wp-event verify-monad
 (*composition-fri-verifier-tied-successor-replay-gap s*) $s \leq M + 0$
 by (rule
 wp-composition-fri-verifier-tied-successor-replay-gap-bound-from-merkle-and-slot
 (rule *merkle*, rule *slot*)
 have *final-bound*:
 wp-event verify-monad
 (*composition-fri-verifier-tied-final-value-replay-gap s*) s
 $\leq M + 0 + 0$
 by (rule
 wp-composition-fri-verifier-tied-final-value-replay-gap-bound-from-merkle-slot-and-selected-step
 (rule *merkle*, rule *slot*,
 rule
 wp-composition-fri-verifier-tied-final-value-selected-step-missing-gap-zero)
 show ?thesis
 by (rule
 composition-fri-verifier-tied-reduction-from-envelope-witness-domain-slot-zero
 [OF *future envelope subset challenge-bound proximity index*
 witness-length domain merkle next-bound successor final-bound total])
 qed

lemma *checked-staged-security-trace-fri-header-tied-partial-candidate-bound-from-full-cover-domain-conflict-and-zero*

assumes *wf*: *staged-budget-wellformed budgets*

and *controlled*: *staged-adversary-controlled budgets A*

and *finite-B*: *finite B*

and *envelope*:

$\bigwedge$ *data attacker-state trace-table.*

Some (*data*, *attacker-state*) $\in$

set-dist (*execute (checked-staged-transcript-program A)*

adversary-initial-state) $\implies$

trace-fri-multi-round-bad-sets bad trace-table $\subseteq B$

and *fresh-bound*:

wp-event (checked-staged-security-experiment-with-data-state A)

(*staged-security-with-data-state-verifier-event*

($\lambda s.$ *trace-fri-challenge-list-fresh-hit s B*))

adversary-initial-state $\leq F$

and *proximity*:

$\bigwedge$ *data attacker-state.*

Some (*data*, *attacker-state*) $\in$

set-dist (*execute (checked-staged-transcript-program A)*

$\text{adversary-initial-state}) \implies$
 $\text{wp-event verify-monad}$
 $(\text{trace-fri-sampled-full-cover-proximity-failure}$
 $(\text{verifier-state-from-adversary attacker-state}$
 $(\text{staged-proof-transcript data})) \text{ bad})$
 $(\text{verifier-state-from-adversary attacker-state}$
 $(\text{staged-proof-transcript data})) \leq P$
and index:
 $\wedge \text{data attacker-state.}$
 $\text{Some (data, attacker-state)} \in$
 $\text{set-dist (execute (checked-staged-transcript-program A)}$
 $\text{adversary-initial-state})} \implies$
 $\text{wp-event verify-monad}$
 $(\text{trace-fri-sampled-full-cover-index-failure}$
 $(\text{verifier-state-from-adversary attacker-state}$
 $(\text{staged-proof-transcript data})))$
 $(\text{verifier-state-from-adversary attacker-state}$
 $(\text{staged-proof-transcript data})) \leq I$
and domain-length:
 $\wedge \text{data attacker-state.}$
 $\text{Some (data, attacker-state)} \in$
 $\text{set-dist (execute (checked-staged-transcript-program A)}$
 $\text{adversary-initial-state})} \implies$
 $\text{wp-event verify-monad}$
 $(\text{trace-fri-sampled-full-cover-domain-length-failure}$
 $(\text{verifier-state-from-adversary attacker-state}$
 $(\text{staged-proof-transcript data})))$
 $(\text{verifier-state-from-adversary attacker-state}$
 $(\text{staged-proof-transcript data})) \leq DL$
and sampled-domain:
 $\wedge \text{data attacker-state.}$
 $\text{Some (data, attacker-state)} \in$
 $\text{set-dist (execute (checked-staged-transcript-program A)}$
 $\text{adversary-initial-state})} \implies$
 $\text{wp-event verify-monad}$
 $(\text{trace-fri-sampled-full-cover-sampled-domain-failure}$
 $(\text{verifier-state-from-adversary attacker-state}$
 $(\text{staged-proof-transcript data})))$
 $(\text{verifier-state-from-adversary attacker-state}$
 $(\text{staged-proof-transcript data})) \leq SD$
and conflict-bound:
 $\wedge \text{data attacker-state.}$
 $\text{Some (data, attacker-state)} \in$
 $\text{set-dist (execute (checked-staged-transcript-program A)}$
 $\text{adversary-initial-state})} \implies$
 $\text{wp-event verify-monad}$
 $(\text{trace-fri-header-tied-sampled-assignment-conflict}$
 $(\text{verifier-state-from-adversary attacker-state}$
 $(\text{staged-proof-transcript data})))$

$(\text{verifier-state-from-adversary attacker-state} \\ (\text{staged-proof-transcript data})) \leq C$
and zero-bound:
 $\wedge \text{data attacker-state.}$
 $\text{Some (data, attacker-state)} \in$
 $\text{set-dist (execute (checked-staged-transcript-program A)}$
 $\text{adversary-initial-state)} \implies$
 $\text{wp-event verify-monad}$
 $(\text{trace-fri-header-tied-zero-round-final-obstruction}$
 $(\text{verifier-state-from-adversary attacker-state}$
 $(\text{staged-proof-transcript data})))$
 $(\text{verifier-state-from-adversary attacker-state}$
 $(\text{staged-proof-transcript data})) \leq Z$
shows
 $\text{wp-event (checked-staged-security-experiment-with-data-state A)}$
 $(\text{staged-security-with-data-state-verifier-event}$
 $\text{trace-fri-bad-with-header-tied-partial-candidate})$
 $\text{adversary-initial-state} \leq$
 $((F +$
 $\text{hash-relation-budget-value (card B * ceil-log clength)}$
 $(\text{staged-attacker-query-budget budgets} +$
 $\text{staged-challenge-query-budget})) +$
 $(P + I + (I + DL) + (I + SD))) + (C + Z)$
proof (rule
 $\text{checked-staged-security-trace-fri-header-tied-partial-candidate-bound-from-sampled-conflict-and-staged-zero})$
show $\text{wp-event (checked-staged-security-experiment-with-data-state A)}$
 $(\text{staged-security-with-data-state-verifier-event}$
 $\text{trace-fri-bad-with-header-tied-sampled-layer-chain})$
 $\text{adversary-initial-state}$
 $\leq$
 $(F +$
 $\text{hash-relation-budget-value (card B * ceil-log clength)}$
 $(\text{staged-attacker-query-budget budgets} +$
 $\text{staged-challenge-query-budget})) +$
 $(P + I + (I + DL) + (I + SD))$
by (rule
 $\text{checked-staged-security-trace-fri-header-tied-sampled-layer-chain-bound-from-full-cover-fresh-and-domain-split}$
 $[\text{OF wf controlled finite-B envelope fresh-bound proximity index}$
 $\text{domain-length sampled-domain}]$)
show $\text{wp-event (checked-staged-security-experiment-with-data-state A)}$
 $(\text{staged-security-with-data-state-verifier-event}$
 $\text{trace-fri-header-tied-sampled-assignment-conflict})$
 $\text{adversary-initial-state} \leq C$
by (rule $\text{checked-staged-security-with-data-state-verifier-event-bound-from-cont}$)
 $(\text{auto simp: trace-fri-header-tied-sampled-assignment-conflict-def}$
 $\text{accepted-fri-opening-transcript-def conflict-bound})$
show $\text{wp-event (checked-staged-security-experiment-with-data-state A)}$
 $(\text{staged-security-with-data-state-verifier-event}$
 $\text{trace-fri-header-tied-zero-round-final-obstruction})$

$adversary-initial-state \leq Z$
by (rule *checked-staged-security-with-data-state-verifier-event-bound-from-cont*)
(auto simp: *trace-fri-header-tied-zero-round-final-obstruction-def*
accepted-fri-opening-transcript-def *zero-bound*)
qed

lemma *checked-staged-security-trace-fri-header-tied-partial-candidate-bound-from-full-cover-witness-domain-conflict-*
assumes *wf: staged-budget-wellformed budgets*
and *controlled: staged-adversary-controlled budgets A*
and *finite-B: finite B*
and *envelope:*
 $\bigwedge data \text{ attacker-state } trace\text{-}table.$
Some (data, attacker-state) $\in$
set-dist (execute (checked-staged-transcript-program A)
adversary-initial-state) $\implies$
trace-fri-multiround-bad-sets bad trace-table $\subseteq B$
and *fresh-bound:*
wp-event (checked-staged-security-experiment-with-data-state A)
(*staged-security-with-data-state-verifier-event*
($\lambda s. trace\text{-}fri\text{-}challenge\text{-}list\text{-}fresh\text{-}hit \ s \ B$))
adversary-initial-state $\leq F$
and *proximity:*
 $\bigwedge data \text{ attacker-state.}$
Some (data, attacker-state) $\in$
set-dist (execute (checked-staged-transcript-program A)
adversary-initial-state) $\implies$
wp-event verify-monad
(*trace-fri-sampled-full-cover-proximity-failure*
(*verifier-state-from-adversary attacker-state*
(*staged-proof-transcript data*)) bad)
(*verifier-state-from-adversary attacker-state*
(*staged-proof-transcript data*)) $\leq P$
and *index:*
 $\bigwedge data \text{ attacker-state.}$
Some (data, attacker-state) $\in$
set-dist (execute (checked-staged-transcript-program A)
adversary-initial-state) $\implies$
wp-event verify-monad
(*trace-fri-sampled-full-cover-index-failure*
(*verifier-state-from-adversary attacker-state*
(*staged-proof-transcript data*)))
(*verifier-state-from-adversary attacker-state*
(*staged-proof-transcript data*)) $\leq I$
and *witness-length:*
 $\bigwedge data \text{ attacker-state.}$
Some (data, attacker-state) $\in$
set-dist (execute (checked-staged-transcript-program A)
adversary-initial-state) $\implies$
wp-event verify-monad

$(\text{trace-fri-sampled-full-cover-witness-length-failure}$
 $(\text{verifier-state-from-adversary attacker-state}$
 $(\text{staged-proof-transcript data})))$
 $(\text{verifier-state-from-adversary attacker-state}$
 $(\text{staged-proof-transcript data})) \leq W$
and *domain*:
 $\wedge \text{data attacker-state.}$
 $\text{Some (data, attacker-state)} \in$
 $\text{set-dist (execute (checked-staged-transcript-program A)}$
 $\text{adversary-initial-state)} \implies$
 $\text{wp-event verify-monad}$
 $(\text{trace-fri-sampled-full-cover-domain-failure}$
 $(\text{verifier-state-from-adversary attacker-state}$
 $(\text{staged-proof-transcript data})))$
 $(\text{verifier-state-from-adversary attacker-state}$
 $(\text{staged-proof-transcript data})) \leq D$
and *conflict-bound*:
 $\wedge \text{data attacker-state.}$
 $\text{Some (data, attacker-state)} \in$
 $\text{set-dist (execute (checked-staged-transcript-program A)}$
 $\text{adversary-initial-state)} \implies$
 $\text{wp-event verify-monad}$
 $(\text{trace-fri-header-tied-sampled-assignment-conflict}$
 $(\text{verifier-state-from-adversary attacker-state}$
 $(\text{staged-proof-transcript data})))$
 $(\text{verifier-state-from-adversary attacker-state}$
 $(\text{staged-proof-transcript data})) \leq C$
and *zero-bound*:
 $\wedge \text{data attacker-state.}$
 $\text{Some (data, attacker-state)} \in$
 $\text{set-dist (execute (checked-staged-transcript-program A)}$
 $\text{adversary-initial-state)} \implies$
 $\text{wp-event verify-monad}$
 $(\text{trace-fri-header-tied-zero-round-final-obstruction}$
 $(\text{verifier-state-from-adversary attacker-state}$
 $(\text{staged-proof-transcript data})))$
 $(\text{verifier-state-from-adversary attacker-state}$
 $(\text{staged-proof-transcript data})) \leq Z$
shows
 $\text{wp-event (checked-staged-security-experiment-with-data-state A)}$
 $(\text{staged-security-with-data-state-verifier-event}$
 $\text{trace-fri-bad-with-header-tied-partial-candidate})$
 $\text{adversary-initial-state} \leq$
 $((F +$
 $\text{hash-relation-budget-value (card B * ceil-log clength)}$
 $(\text{staged-attacker-query-budget budgets} +$
 $\text{staged-challenge-query-budget})) +$
 $(P + I + W + D)) + (C + Z)$
proof (rule

checked-staged-security-trace-fri-header-tied-partial-candidate-bound-from-sampled-conflict-and-staged-zero)
show *wp-event* (*checked-staged-security-experiment-with-data-state A*)
 (*staged-security-with-data-state-verifier-event*
 trace-fri-bad-with-header-tied-sampled-layer-chain)
 adversary-initial-state
 $\leq$
 ($F +$
 hash-relation-budget-value (*card B * ceil-log clength*)
 (*staged-attacker-query-budget budgets +*
 staged-challenge-query-budget)) $+$
 ($P + I + W + D$)
by (*rule*
 checked-staged-security-trace-fri-header-tied-sampled-layer-chain-bound-from-full-cover-fresh-and-witness-domain
 [*OF wf controlled finite-B envelope fresh-bound proximity index*
 witness-length domain])
show *wp-event* (*checked-staged-security-experiment-with-data-state A*)
 (*staged-security-with-data-state-verifier-event*
 trace-fri-header-tied-sampled-assignment-conflict)
 adversary-initial-state $\leq C$
by (*rule* *checked-staged-security-with-data-state-verifier-event-bound-from-cont*)
 (*auto simp: trace-fri-header-tied-sampled-assignment-conflict-def*
 accepted-fri-opening-transcript-def conflict-bound)
show *wp-event* (*checked-staged-security-experiment-with-data-state A*)
 (*staged-security-with-data-state-verifier-event*
 trace-fri-header-tied-zero-round-final-obstruction)
 adversary-initial-state $\leq Z$
by (*rule* *checked-staged-security-with-data-state-verifier-event-bound-from-cont*)
 (*auto simp: trace-fri-header-tied-zero-round-final-obstruction-def*
 accepted-fri-opening-transcript-def zero-bound)
qed

lemma *checked-staged-security-composition-fri-verifier-tied-partial-candidate-bound-from-full-cover-domain-conflict-a*
assumes *wf: staged-budget-wellformed budgets*
and *controlled: staged-adversary-controlled budgets A*
and *finite-B: $\bigwedge dg. \text{finite } (B \ dg)$*
and *envelope:*
 $\bigwedge \text{data attacker-state } dg \text{ composition-table.}$
 Some (*data, attacker-state*) $\in$
 set-dist (*execute* (*checked-staged-transcript-program A*)
 adversary-initial-state) $\implies$
 composition-fri-multiround-bad-sets bad dg composition-table
 $\subseteq B \ dg$
and *fresh-bound:*
wp-event (*checked-staged-security-experiment-with-data-state A*)
 (*staged-security-with-data-state-verifier-event*
 ($\lambda s. \text{composition-fri-challenge-list-fresh-hit } s \ B$))
 adversary-initial-state $\leq F$
and *proximity:*
 $\bigwedge \text{data attacker-state.}$

Some (*data*, *attacker-state*) $\in$
set-dist (*execute* (*checked-staged-transcript-program* *A*)
adversary-initial-state) $\implies$
wp-event verify-monad
(*composition-fri-sampled-full-cover-proximity-failure*
(*verifier-state-from-adversary attacker-state*
(*staged-proof-transcript data*)) *bad*)
(*verifier-state-from-adversary attacker-state*
(*staged-proof-transcript data*)) $\leq P$

and *index*:

$\wedge$ *data attacker-state*.
Some (*data*, *attacker-state*) $\in$
set-dist (*execute* (*checked-staged-transcript-program* *A*)
adversary-initial-state) $\implies$
wp-event verify-monad
(*composition-fri-sampled-full-cover-index-failure*
(*verifier-state-from-adversary attacker-state*
(*staged-proof-transcript data*)))
(*verifier-state-from-adversary attacker-state*
(*staged-proof-transcript data*)) $\leq I$

and *domain-length*:

$\wedge$ *data attacker-state*.
Some (*data*, *attacker-state*) $\in$
set-dist (*execute* (*checked-staged-transcript-program* *A*)
adversary-initial-state) $\implies$
wp-event verify-monad
(*composition-fri-sampled-full-cover-domain-length-failure*
(*verifier-state-from-adversary attacker-state*
(*staged-proof-transcript data*)))
(*verifier-state-from-adversary attacker-state*
(*staged-proof-transcript data*)) $\leq DL$

and *sampled-domain*:

$\wedge$ *data attacker-state*.
Some (*data*, *attacker-state*) $\in$
set-dist (*execute* (*checked-staged-transcript-program* *A*)
adversary-initial-state) $\implies$
wp-event verify-monad
(*composition-fri-sampled-full-cover-sampled-domain-failure*
(*verifier-state-from-adversary attacker-state*
(*staged-proof-transcript data*)))
(*verifier-state-from-adversary attacker-state*
(*staged-proof-transcript data*)) $\leq SD$

and *conflict-bound*:

$\wedge$ *data attacker-state*.
Some (*data*, *attacker-state*) $\in$
set-dist (*execute* (*checked-staged-transcript-program* *A*)
adversary-initial-state) $\implies$
wp-event verify-monad
(*composition-fri-verifier-tied-sampled-assignment-conflict*

$$\begin{aligned} & (\text{verifier-state-from-adversary attacker-state} \\ & \quad (\text{staged-proof-transcript data}))) \\ & (\text{verifier-state-from-adversary attacker-state} \\ & \quad (\text{staged-proof-transcript data})) \leq C \end{aligned}$$

and zero-bound:

$\wedge$ data attacker-state.

Some (data, attacker-state) $\in$
 set-dist (execute (checked-staged-transcript-program A)
 adversary-initial-state) $\implies$

wp-event verify-monad
 (composition-fri-verifier-tied-zero-round-final-obstruction
 (verifier-state-from-adversary attacker-state
 (staged-proof-transcript data)))
 (verifier-state-from-adversary attacker-state
 (staged-proof-transcript data)) $\leq Z$

shows

wp-event (checked-staged-security-experiment-with-data-state A)
 (staged-security-with-data-state-verifier-event
 composition-fri-bad-with-verifier-tied-partial-candidate)
 adversary-initial-state $\leq$
 ((F +
 hash-relation-budget-value
 ($\sum dg \in (\text{UNIV} :: 'f \text{ set}).$
 card (B dg) * ceil-log (maxDegree + 1))
 (staged-attacker-query-budget budgets +
 staged-challenge-query-budget)) +
 (P + I + (I + DL) + (I + SD))) + (C + Z)

proof (rule
 checked-staged-security-composition-fri-verifier-tied-partial-candidate-bound-from-sampled-and-missing)

show wp-event (checked-staged-security-experiment-with-data-state A)
 (staged-security-with-data-state-verifier-event
 composition-fri-bad-with-verifier-tied-sampled-layer-chain)
 adversary-initial-state $\leq$
 (F +
 hash-relation-budget-value
 ($\sum dg \in (\text{UNIV} :: 'f \text{ set}).$
 card (B dg) * ceil-log (maxDegree + 1))
 (staged-attacker-query-budget budgets +
 staged-challenge-query-budget)) +
 (P + I + (I + DL) + (I + SD))

by (rule
 checked-staged-security-composition-fri-verifier-tied-sampled-layer-chain-bound-from-full-cover-fresh-and-domain
 [OF wf controlled finite-B envelope fresh-bound proximity index
 domain-length sampled-domain])

fix data attacker-state

assume support:
 Some (data, attacker-state) $\in$
 set-dist (execute (checked-staged-transcript-program A)
 adversary-initial-state)

show *wp-event verify-monad*
 (*composition-fri-verifier-tied-missing-sampled-layer-chain*
 (*verifier-state-from-adversary attacker-state*
 (*staged-proof-transcript data*)))
 (*verifier-state-from-adversary attacker-state*
 (*staged-proof-transcript data*))
 $\leq C + Z$
by (rule
wp-composition-fri-verifier-tied-missing-sampled-bound-from-assignment-obstruction)
 (rule
wp-composition-fri-verifier-tied-assignment-obstruction-bound-from-conflict-and-zero
 [*OF conflict-bound*[*OF support*] *zero-bound*[*OF support*]])

qed

lemma *checked-staged-security-composition-fri-verifier-tied-partial-candidate-bound-from-full-cover-witness-domain-c*

assumes *wf: staged-budget-wellformed budgets*
and *controlled: staged-adversary-controlled budgets A*
and *finite-B: $\bigwedge dg. \text{finite } (B \ dg)$*
and *envelope:*
 $\bigwedge \text{data attacker-state } dg \text{ composition-table.}$
 Some (data, attacker-state) $\in$
 set-dist (execute (checked-staged-transcript-program A)
 adversary-initial-state) $\implies$
 composition-fri-multiround-bad-sets bad dg composition-table
 $\subseteq B \ dg$
and *fresh-bound:*
wp-event (checked-staged-security-experiment-with-data-state A)
 (*staged-security-with-data-state-verifier-event*
 ($\lambda s. \text{composition-fri-challenge-list-fresh-hit } s \ B$))
 adversary-initial-state $\leq F$
and *proximity:*
 $\bigwedge \text{data attacker-state.}$
 Some (data, attacker-state) $\in$
 set-dist (execute (checked-staged-transcript-program A)
 adversary-initial-state) $\implies$
wp-event verify-monad
 (*composition-fri-sampled-full-cover-proximity-failure*
 (*verifier-state-from-adversary attacker-state*
 (*staged-proof-transcript data*)) bad)
 (*verifier-state-from-adversary attacker-state*
 (*staged-proof-transcript data*)) $\leq P$
and *index:*
 $\bigwedge \text{data attacker-state.}$
 Some (data, attacker-state) $\in$
 set-dist (execute (checked-staged-transcript-program A)
 adversary-initial-state) $\implies$
wp-event verify-monad
 (*composition-fri-sampled-full-cover-index-failure*
 (*verifier-state-from-adversary attacker-state*

$(\text{staged-proof-transcript data}))$
 $(\text{verifier-state-from-adversary attacker-state}$
 $(\text{staged-proof-transcript data})) \leq I$
and *witness-length*:
 $\wedge \text{data attacker-state.}$
 $\text{Some (data, attacker-state)} \in$
 $\text{set-dist (execute (checked-staged-transcript-program A)}$
 $\text{adversary-initial-state)} \implies$
 $\text{wp-event verify-monad}$
 $(\text{composition-fri-sampled-full-cover-witness-length-failure}$
 $(\text{verifier-state-from-adversary attacker-state}$
 $(\text{staged-proof-transcript data}))$
 $(\text{verifier-state-from-adversary attacker-state}$
 $(\text{staged-proof-transcript data})) \leq W$
and *domain*:
 $\wedge \text{data attacker-state.}$
 $\text{Some (data, attacker-state)} \in$
 $\text{set-dist (execute (checked-staged-transcript-program A)}$
 $\text{adversary-initial-state)} \implies$
 $\text{wp-event verify-monad}$
 $(\text{composition-fri-sampled-full-cover-domain-failure}$
 $(\text{verifier-state-from-adversary attacker-state}$
 $(\text{staged-proof-transcript data}))$
 $(\text{verifier-state-from-adversary attacker-state}$
 $(\text{staged-proof-transcript data})) \leq D$
and *conflict-bound*:
 $\wedge \text{data attacker-state.}$
 $\text{Some (data, attacker-state)} \in$
 $\text{set-dist (execute (checked-staged-transcript-program A)}$
 $\text{adversary-initial-state)} \implies$
 $\text{wp-event verify-monad}$
 $(\text{composition-fri-verifier-tied-sampled-assignment-conflict}$
 $(\text{verifier-state-from-adversary attacker-state}$
 $(\text{staged-proof-transcript data}))$
 $(\text{verifier-state-from-adversary attacker-state}$
 $(\text{staged-proof-transcript data})) \leq C$
and *zero-bound*:
 $\wedge \text{data attacker-state.}$
 $\text{Some (data, attacker-state)} \in$
 $\text{set-dist (execute (checked-staged-transcript-program A)}$
 $\text{adversary-initial-state)} \implies$
 $\text{wp-event verify-monad}$
 $(\text{composition-fri-verifier-tied-zero-round-final-obstruction}$
 $(\text{verifier-state-from-adversary attacker-state}$
 $(\text{staged-proof-transcript data}))$
 $(\text{verifier-state-from-adversary attacker-state}$
 $(\text{staged-proof-transcript data})) \leq Z$
shows
 $\text{wp-event (checked-staged-security-experiment-with-data-state A)}$

*(staged-security-with-data-state-verifier-event
composition-fri-bad-with-verifier-tied-partial-candidate)*
adversary-initial-state $\leq$
 $((F +$
hash-relation-budget-value
 $(\sum dg \in (UNIV :: 'f \text{ set}).$
 $\text{card } (B \ dg) * \text{ceil-log } (\text{maxDegree} + 1))$
 $(\text{staged-attacker-query-budget } \text{budgets} +$
 $\text{staged-challenge-query-budget})) +$
 $(P + I + W + D)) + (C + Z)$

proof (rule
checked-staged-security-composition-fri-verifier-tied-partial-candidate-bound-from-sampled-and-missing)
show *wp-event (checked-staged-security-experiment-with-data-state A)*
*(staged-security-with-data-state-verifier-event
composition-fri-bad-with-verifier-tied-sampled-layer-chain)*
adversary-initial-state $\leq$
 $(F +$
hash-relation-budget-value
 $(\sum dg \in (UNIV :: 'f \text{ set}).$
 $\text{card } (B \ dg) * \text{ceil-log } (\text{maxDegree} + 1))$
 $(\text{staged-attacker-query-budget } \text{budgets} +$
 $\text{staged-challenge-query-budget})) +$
 $(P + I + W + D)$

by (rule
checked-staged-security-composition-fri-verifier-tied-sampled-layer-chain-bound-from-full-cover-fresh-and-witness
 $[OF \ wf \ \text{controlled} \ \text{finite-}B \ \text{envelope} \ \text{fresh-bound} \ \text{proximity} \ \text{index}$
 $\text{witness-length} \ \text{domain}]$)

fix *data attacker-state*
assume *support:*
Some (data, attacker-state) $\in$
set-dist (execute (checked-staged-transcript-program A)
adversary-initial-state)

show *wp-event verify-monad*
(composition-fri-verifier-tied-missing-sampled-layer-chain
(verifier-state-from-adversary attacker-state
(staged-proof-transcript data)))
(verifier-state-from-adversary attacker-state
(staged-proof-transcript data))
 $\leq C + Z$

by (rule
wp-composition-fri-verifier-tied-missing-sampled-bound-from-assignment-obstruction)
(rule
wp-composition-fri-verifier-tied-assignment-obstruction-bound-from-conflict-and-zero
 $[OF \ \text{conflict-bound}[OF \ \text{support}] \ \text{zero-bound}[OF \ \text{support}]]$)

qed

lemma *checked-staged-security-trace-fri-empty-header-bound-from-full-cover-domain-conflict-and-zero:*
assumes *wf: staged-budget-wellformed budgets*
and *controlled: staged-adversary-controlled budgets A*

and *finite-B*: *finite B*
and *envelope*:
 $\wedge$ *data attacker-state trace-table*.
Some (data, attacker-state) $\in$
set-dist (execute (checked-staged-transcript-program A)
adversary-initial-state) $\implies$
trace-fri-multiround-bad-sets bad trace-table $\subseteq B$
and *fresh-bound*:
wp-event (checked-staged-security-experiment-with-data-state A)
(staged-security-with-data-state-verifier-event
(λs . trace-fri-challenge-list-fresh-hit s B))
adversary-initial-state $\leq F$
and *proximity*:
 $\wedge$ *data attacker-state*.
Some (data, attacker-state) $\in$
set-dist (execute (checked-staged-transcript-program A)
adversary-initial-state) $\implies$
wp-event verify-monad
(trace-fri-sampled-full-cover-proximity-failure
(verifier-state-from-adversary attacker-state
(staged-proof-transcript data)) bad)
(verifier-state-from-adversary attacker-state
(staged-proof-transcript data)) $\leq P$
and *index*:
 $\wedge$ *data attacker-state*.
Some (data, attacker-state) $\in$
set-dist (execute (checked-staged-transcript-program A)
adversary-initial-state) $\implies$
wp-event verify-monad
(trace-fri-sampled-full-cover-index-failure
(verifier-state-from-adversary attacker-state
(staged-proof-transcript data)))
(verifier-state-from-adversary attacker-state
(staged-proof-transcript data)) $\leq I$
and *domain-length*:
 $\wedge$ *data attacker-state*.
Some (data, attacker-state) $\in$
set-dist (execute (checked-staged-transcript-program A)
adversary-initial-state) $\implies$
wp-event verify-monad
(trace-fri-sampled-full-cover-domain-length-failure
(verifier-state-from-adversary attacker-state
(staged-proof-transcript data)))
(verifier-state-from-adversary attacker-state
(staged-proof-transcript data)) $\leq DL$
and *sampled-domain*:
 $\wedge$ *data attacker-state*.
Some (data, attacker-state) $\in$
set-dist (execute (checked-staged-transcript-program A)

$\text{adversary-initial-state}) \implies$
 $\text{wp-event verify-monad}$
 $(\text{trace-fri-sampled-full-cover-sampled-domain-failure}$
 $(\text{verifier-state-from-adversary attacker-state}$
 $(\text{staged-proof-transcript data})))$
 $(\text{verifier-state-from-adversary attacker-state}$
 $(\text{staged-proof-transcript data})) \leq SD$
and conflict-bound:
 $\wedge \text{data attacker-state.}$
 $\text{Some (data, attacker-state) } \in$
 $\text{set-dist (execute (checked-staged-transcript-program A)}$
 $\text{adversary-initial-state}) \implies}$
 $\text{wp-event verify-monad}$
 $(\text{trace-fri-sampled-assignment-conflict}$
 $(\text{verifier-state-from-adversary attacker-state}$
 $(\text{staged-proof-transcript data})))$
 $(\text{verifier-state-from-adversary attacker-state}$
 $(\text{staged-proof-transcript data})) \leq C$
and zero-bound:
 $\wedge \text{data attacker-state.}$
 $\text{Some (data, attacker-state) } \in$
 $\text{set-dist (execute (checked-staged-transcript-program A)}$
 $\text{adversary-initial-state}) \implies}$
 $\text{wp-event verify-monad}$
 $(\text{trace-fri-zero-round-final-obstruction}$
 $(\text{verifier-state-from-adversary attacker-state}$
 $(\text{staged-proof-transcript data})))$
 $(\text{verifier-state-from-adversary attacker-state}$
 $(\text{staged-proof-transcript data})) \leq Z$
shows
 $\text{wp-event (checked-staged-security-experiment-with-data-state A)}$
 $(\text{staged-security-with-data-state-verifier-event}$
 $\text{trace-fri-bad-with-empty-composition-header-candidates})$
 $\text{adversary-initial-state} \leq$
 $((F +$
 $\text{hash-relation-budget-value (card B * ceil-log clength)}$
 $(\text{staged-attacker-query-budget budgets} +$
 $\text{staged-challenge-query-budget})) +$
 $(P + I + (I + DL) + (I + SD))) + (C + Z)$
proof $(\text{rule checked-staged-security-trace-fri-empty-header-bound-from-sampled-and-missing})$
show $\text{wp-event (checked-staged-security-experiment-with-data-state A)}$
 $(\text{staged-security-with-data-state-verifier-event}$
 $\text{trace-fri-bad-with-sampled-layer-chain})$
 $\text{adversary-initial-state}$
 $\leq$
 $(F +$
 $\text{hash-relation-budget-value (card B * ceil-log clength)}$
 $(\text{staged-attacker-query-budget budgets} +$
 $\text{staged-challenge-query-budget})) +$

```

(P + I + (I + DL) + (I + SD))
by (rule
  checked-staged-security-trace-fri-sampled-layer-chain-bound-from-full-cover-fresh-and-domain-split
  [OF wf controlled finite-B envelope fresh-bound proximity index
    domain-length sampled-domain])
fix data attacker-state
assume support:
  Some (data, attacker-state) ∈
    set-dist (execute (checked-staged-transcript-program A)
      adversary-initial-state)
let ?s =
  verifier-state-from-adversary attacker-state
  (staged-proof-transcript data)
have obstruction:
  wp-event verify-monad
    (trace-fri-sampled-layer-assignment-obstruction ?s) ?s ≤ C + Z
proof -
  have wp-event verify-monad
    (trace-fri-sampled-layer-assignment-obstruction ?s) ?s ≤
  wp-event verify-monad
    (λout. trace-fri-sampled-assignment-conflict ?s out ∨
      trace-fri-zero-round-final-obstruction ?s out) ?s
  by (rule wp-event-mono)
  (rule
    trace-fri-sampled-layer-assignment-obstruction-imp-conflict-or-zero-round-final)
  also have ... ≤
    wp-event verify-monad (trace-fri-sampled-assignment-conflict ?s) ?s +
    wp-event verify-monad (trace-fri-zero-round-final-obstruction ?s) ?s
  by (rule wp-event-union-bound)
  also have ... ≤ C + Z
  by (rule add-mono[OF conflict-bound[OF support]
    zero-bound[OF support]])
  finally show ?thesis .
qed
show wp-event verify-monad
  (trace-fri-reachable-missing-sampled-layer-chain
    ?s) ?s ≤ C + Z
  by (rule wp-trace-fri-reachable-missing-sampled-bound-from-assignment-obstruction)
  (rule obstruction)
qed

lemma checked-staged-security-trace-fri-empty-header-bound-from-full-cover-witness-domain-conflict-and-zero:
  assumes wf: staged-budget-wellformed budgets
  and controlled: staged-adversary-controlled budgets A
  and finite-B: finite B
  and envelope:
    ∧ data attacker-state trace-table.
    Some (data, attacker-state) ∈
      set-dist (execute (checked-staged-transcript-program A)

```


$\text{adversary-initial-state}) \implies$
 $\text{trace-fri-multiround-bad-sets bad trace-table} \subseteq B$
and *fresh-bound*:
 $\text{wp-event } (\text{checked-staged-security-experiment-with-data-state } A)$
 $(\text{staged-security-with-data-state-verifier-event}$
 $(\lambda s. \text{trace-fri-challenge-list-fresh-hit } s \ B))$
 $\text{adversary-initial-state} \leq F$
and *proximity*:
 $\wedge \text{data attacker-state.}$
 $\text{Some } (data, \text{attacker-state}) \in$
 $\text{set-dist } (\text{execute } (\text{checked-staged-transcript-program } A)$
 $\text{adversary-initial-state}) \implies$
 $\text{wp-event verify-monad}$
 $(\text{trace-fri-sampled-full-cover-proximity-failure}$
 $(\text{verifier-state-from-adversary attacker-state}$
 $(\text{staged-proof-transcript data})) \text{ bad})$
 $(\text{verifier-state-from-adversary attacker-state}$
 $(\text{staged-proof-transcript data})) \leq P$
and *index*:
 $\wedge \text{data attacker-state.}$
 $\text{Some } (data, \text{attacker-state}) \in$
 $\text{set-dist } (\text{execute } (\text{checked-staged-transcript-program } A)$
 $\text{adversary-initial-state}) \implies$
 $\text{wp-event verify-monad}$
 $(\text{trace-fri-sampled-full-cover-index-failure}$
 $(\text{verifier-state-from-adversary attacker-state}$
 $(\text{staged-proof-transcript data})))$
 $(\text{verifier-state-from-adversary attacker-state}$
 $(\text{staged-proof-transcript data})) \leq I$
and *witness-length*:
 $\wedge \text{data attacker-state.}$
 $\text{Some } (data, \text{attacker-state}) \in$
 $\text{set-dist } (\text{execute } (\text{checked-staged-transcript-program } A)$
 $\text{adversary-initial-state}) \implies$
 $\text{wp-event verify-monad}$
 $(\text{trace-fri-sampled-full-cover-witness-length-failure}$
 $(\text{verifier-state-from-adversary attacker-state}$
 $(\text{staged-proof-transcript data})))$
 $(\text{verifier-state-from-adversary attacker-state}$
 $(\text{staged-proof-transcript data})) \leq W$
and *domain*:
 $\wedge \text{data attacker-state.}$
 $\text{Some } (data, \text{attacker-state}) \in$
 $\text{set-dist } (\text{execute } (\text{checked-staged-transcript-program } A)$
 $\text{adversary-initial-state}) \implies$
 $\text{wp-event verify-monad}$
 $(\text{trace-fri-sampled-full-cover-domain-failure}$
 $(\text{verifier-state-from-adversary attacker-state}$
 $(\text{staged-proof-transcript data})))$

$(\text{verifier-state-from-adversary attacker-state}$
 $(\text{staged-proof-transcript data})) \leq D$
and *conflict-bound*:
 $\wedge \text{data attacker-state.}$
 $\text{Some (data, attacker-state)} \in$
 $\text{set-dist (execute (checked-staged-transcript-program A)}$
 $\text{adversary-initial-state)} \implies$
 $\text{wp-event verify-monad}$
 $(\text{trace-fri-sampled-assignment-conflict}$
 $(\text{verifier-state-from-adversary attacker-state}$
 $(\text{staged-proof-transcript data})))$
 $(\text{verifier-state-from-adversary attacker-state}$
 $(\text{staged-proof-transcript data})) \leq C$
and *zero-bound*:
 $\wedge \text{data attacker-state.}$
 $\text{Some (data, attacker-state)} \in$
 $\text{set-dist (execute (checked-staged-transcript-program A)}$
 $\text{adversary-initial-state)} \implies$
 $\text{wp-event verify-monad}$
 $(\text{trace-fri-zero-round-final-obstruction}$
 $(\text{verifier-state-from-adversary attacker-state}$
 $(\text{staged-proof-transcript data})))$
 $(\text{verifier-state-from-adversary attacker-state}$
 $(\text{staged-proof-transcript data})) \leq Z$
shows
 $\text{wp-event (checked-staged-security-experiment-with-data-state A)}$
 $(\text{staged-security-with-data-state-verifier-event}$
 $\text{trace-fri-bad-with-empty-composition-header-candidates})$
 $\text{adversary-initial-state} \leq$
 $((F +$
 $\text{hash-relation-budget-value (card B * ceil-log clength)}$
 $(\text{staged-attacker-query-budget budgets} +$
 $\text{staged-challenge-query-budget})) +$
 $(P + I + W + D)) + (C + Z)$
proof (rule *checked-staged-security-trace-fri-empty-header-bound-from-sampled-and-missing*)
show $\text{wp-event (checked-staged-security-experiment-with-data-state A)}$
 $(\text{staged-security-with-data-state-verifier-event}$
 $\text{trace-fri-bad-with-sampled-layer-chain})$
 $\text{adversary-initial-state}$
 $\leq$
 $(F +$
 $\text{hash-relation-budget-value (card B * ceil-log clength)}$
 $(\text{staged-attacker-query-budget budgets} +$
 $\text{staged-challenge-query-budget})) +$
 $(P + I + W + D)$
by (rule
checked-staged-security-trace-fri-sampled-layer-chain-bound-from-full-cover-fresh-and-witness-domain
 $[\text{OF wf controlled finite-B envelope fresh-bound proximity index}$
 $\text{witness-length domain}]$)

```

fix data attacker-state
assume support:
  Some (data, attacker-state) ∈
    set-dist (execute (checked-staged-transcript-program A)
      adversary-initial-state)
let ?s =
  verifier-state-from-adversary attacker-state
    (staged-proof-transcript data)
have obstruction:
  wp-event verify-monad
    (trace-fri-sampled-layer-assignment-obstruction ?s) ?s ≤ C + Z
proof -
  have wp-event verify-monad
    (trace-fri-sampled-layer-assignment-obstruction ?s) ?s ≤
  wp-event verify-monad
    (λout. trace-fri-sampled-assignment-conflict ?s out ∨
      trace-fri-zero-round-final-obstruction ?s out) ?s
  by (rule wp-event-mono)
    (rule
      trace-fri-sampled-layer-assignment-obstruction-imp-conflict-or-zero-round-final)
  also have ... ≤
    wp-event verify-monad (trace-fri-sampled-assignment-conflict ?s) ?s +
    wp-event verify-monad (trace-fri-zero-round-final-obstruction ?s) ?s
  by (rule wp-event-union-bound)
  also have ... ≤ C + Z
  by (rule add-mono[OF conflict-bound[OF support]
    zero-bound[OF support]])
  finally show ?thesis .
qed
show wp-event verify-monad
  (trace-fri-reachable-missing-sampled-layer-chain
    ?s) ?s ≤ C + Z
  by (rule wp-trace-fri-reachable-missing-sampled-bound-from-assignment-obstruction)
    (rule obstruction)
qed

lemma active-fri-reductions-for-from-component-reductions:
assumes trace-header:
  ∧data attacker-state.
  Some (data, attacker-state) ∈
    set-dist (execute (checked-staged-transcript-program A)
      adversary-initial-state) ⇒
  trace-fri-header-tied-reduction
    (verifier-state-from-adversary attacker-state
      (staged-proof-transcript data))
and composition:
  ∧data attacker-state.
  Some (data, attacker-state) ∈
    set-dist (execute (checked-staged-transcript-program A)

```

```

    adversary-initial-state)  $\Rightarrow$ 
composition-fri-verifier-tied-reduction
  (verifier-state-from-adversary attacker-state
   (staged-proof-transcript data))
and empty-trace:
 $\wedge$ data attacker-state.
  Some (data, attacker-state)  $\in$ 
    set-dist (execute (checked-staged-transcript-program A)
                     adversary-initial-state)  $\Rightarrow$ 
trace-fri-empty-header-reduction
  (verifier-state-from-adversary attacker-state
   (staged-proof-transcript data))
shows active-fri-reductions-for A
unfolding active-fri-reductions-for-def
by (intro allI impI conjI)
  (erule trace-header, erule composition, erule empty-trace)

lemma trace-fri-empty-header-reduction-from-sampled-and-missing-bounds:
fixes R M :: prob
assumes sampled-bound:
  wp-event verify-monad
    (trace-fri-bad-with-sampled-layer-chain s) s  $\leq$  R
and missing-bound:
  wp-event verify-monad
    (trace-fri-reachable-missing-sampled-layer-chain s) s  $\leq$  M
and total-bound:
  R + M  $\leq$  trace-fri-error
shows trace-fri-empty-header-reduction s
proof –
have reachable:
  wp-event verify-monad
    (trace-fri-bad-with-reachable-partial-candidate s) s  $\leq$  R + M
by (rule wp-trace-fri-reachable-bound-from-sampled-and-missing
     [OF sampled-bound missing-bound])
have empty:
  wp-event verify-monad
    (trace-fri-bad-with-empty-composition-header-candidates s) s  $\leq$ 
    R + M
by (rule order-trans[OF - reachable])
  (rule wp-event-mono,
   rule
     trace-fri-bad-with-empty-composition-header-candidates-imp-reachable-partial-candidate)
show ?thesis
unfolding trace-fri-empty-header-reduction-def
by (rule order-trans[OF empty total-bound])
qed

lemma trace-fri-empty-header-reduction-from-sampled-and-assignment-obstruction-bounds:
fixes R M :: prob

```

assumes *sampld-bound*:
 $wp\text{-event verify-monad}$
 $(\text{trace-fri-bad-with-sampled-layer-chain } s) \ s \leq R$
and *obstruction-bound*:
 $wp\text{-event verify-monad}$
 $(\text{trace-fri-sampled-layer-assignment-obstruction } s) \ s \leq M$
and *total-bound*:
 $R + M \leq \text{trace-fri-error}$
shows *trace-fri-empty-header-reduction s*
proof (rule *trace-fri-empty-header-reduction-from-sampled-and-missing-bounds*)
show $wp\text{-event verify-monad}$
 $(\text{trace-fri-bad-with-sampled-layer-chain } s) \ s \leq R$
by (rule *sampld-bound*)
show $wp\text{-event verify-monad}$
 $(\text{trace-fri-reachable-missing-sampled-layer-chain } s) \ s \leq M$
by (rule
 $wp\text{-trace-fri-reachable-missing-sampled-bound-from-assignment-obstruction}$
 $[OF \ \text{obstruction-bound}]$)
show $R + M \leq \text{trace-fri-error}$
by (rule *total-bound*)
qed

lemma *wp-trace-fri-sampled-layer-assignment-obstruction-bound-from-conflict-and-zero*:
assumes *conflict-bound*:
 $wp\text{-event verify-monad}$
 $(\text{trace-fri-sampled-assignment-conflict } s) \ s \leq C$
and *zero-bound*:
 $wp\text{-event verify-monad}$
 $(\text{trace-fri-zero-round-final-obstruction } s) \ s \leq Z$
shows
 $wp\text{-event verify-monad}$
 $(\text{trace-fri-sampled-layer-assignment-obstruction } s) \ s \leq C + Z$
proof –
have $wp\text{-event verify-monad}$
 $(\text{trace-fri-sampled-layer-assignment-obstruction } s) \ s \leq$
 $wp\text{-event verify-monad}$
 $(\lambda out. \text{trace-fri-sampled-assignment-conflict } s \ out \ \vee$
 $\text{trace-fri-zero-round-final-obstruction } s \ out) \ s$
by (rule *wp-event-mono*)
 $(\text{rule } \text{trace-fri-sampled-layer-assignment-obstruction-imp-conflict-or-zero-round-final})$
also have $\dots \leq$
 $wp\text{-event verify-monad } (\text{trace-fri-sampled-assignment-conflict } s) \ s +$
 $wp\text{-event verify-monad } (\text{trace-fri-zero-round-final-obstruction } s) \ s$
by (rule *wp-event-union-bound*)
also have $\dots \leq C + Z$
by (rule *add-mono*[*OF conflict-bound zero-bound*])
finally show *?thesis* .
qed

lemma *trace-fri-empty-header-reduction-from-sampled-conflict-and-zero-bounds:*
fixes $R\ C\ Z :: \text{prob}$
assumes *sampled-bound:*
 $\text{wp-event verify-monad}$
 $(\text{trace-fri-bad-with-sampled-layer-chain } s) \ s \leq R$
and *conflict-bound:*
 $\text{wp-event verify-monad}$
 $(\text{trace-fri-sampled-assignment-conflict } s) \ s \leq C$
and *zero-bound:*
 $\text{wp-event verify-monad}$
 $(\text{trace-fri-zero-round-final-obstruction } s) \ s \leq Z$
and *total-bound:*
 $R + (C + Z) \leq \text{trace-fri-error}$
shows *trace-fri-empty-header-reduction* s
proof (rule
 trace-fri-empty-header-reduction-from-sampled-and-assignment-obstruction-bounds)
show $\text{wp-event verify-monad}$
 $(\text{trace-fri-bad-with-sampled-layer-chain } s) \ s \leq R$
by (rule *sampled-bound*)
show $\text{wp-event verify-monad}$
 $(\text{trace-fri-sampled-layer-assignment-obstruction } s) \ s \leq C + Z$
by (rule
 wp-trace-fri-sampled-layer-assignment-obstruction-bound-from-conflict-and-zero
 [*OF conflict-bound zero-bound*])
show $R + (C + Z) \leq \text{trace-fri-error}$
by (rule *total-bound*)
qed

lemma *trace-fri-header-tied-reduction-from-augmented-conflict-zero-merkle-bounds:*
fixes $A\ C\ Z\ M :: \text{prob}$
assumes *augmented-bound:*
 $\text{wp-event verify-monad}$
 $(\text{trace-fri-bad-with-header-tied-augmented-partial-candidate } s) \ s \leq A$
and *conflict-bound:*
 $\text{wp-event verify-monad}$
 $(\text{trace-fri-header-tied-sampled-assignment-conflict } s) \ s \leq C$
and *zero-bound:*
 $\text{wp-event verify-monad}$
 $(\text{trace-fri-header-tied-zero-round-final-obstruction } s) \ s \leq Z$
and *merkle-bound:*
 $\text{wp-event verify-monad}$ (*partial-merkle-inconsistency-bad* s) $s \leq M$
and *total-bound:*
 $A + ((C + Z) + M) \leq \text{trace-fri-error}$
shows *trace-fri-header-tied-reduction* s
proof –
have *missing-bound:*
 $\text{wp-event verify-monad}$
 $(\text{trace-fri-header-tied-recorded-sibling-candidate-missing } s) \ s$

$\leq C + Z$
by (rule
 $wp\text{-}trace\text{-}fri\text{-}header\text{-}tied\text{-}recorded\text{-}sibling\text{-}candidate\text{-}missing\text{-}bound\text{-}from\text{-}conflict\text{-}and\text{-}zero$)
(rule conflict-bound, rule zero-bound)
show ?thesis
by (rule
 $trace\text{-}fri\text{-}header\text{-}tied\text{-}reduction\text{-}from\text{-}augmented\text{-}and\text{-}recorded\text{-}missing\text{-}bounds$
[OF augmented-bound missing-bound merkle-bound total-bound])
qed

lemma active-fri-reductions-for-from-trace-augmented-composition-sampled-and-empty:

fixes TraceAug TraceMissing TraceMerkle CompSampled CompMerkle ::

'f staged-proof-data $\Rightarrow$ 'f protocol-channel $\Rightarrow$ prob

assumes trace-augmented-bound:

$\wedge$ data attacker-state.

Some (data, attacker-state) $\in$
set-dist (execute (checked-staged-transcript-program A)
adversary-initial-state) $\Rightarrow$
wp-event verify-monad
(trace-fri-bad-with-header-tied-augmented-partial-candidate
(verifier-state-from-adversary attacker-state
(staged-proof-transcript data)))
(verifier-state-from-adversary attacker-state
(staged-proof-transcript data)) $\leq$ TraceAug data attacker-state

and trace-missing-bound:

$\wedge$ data attacker-state.

Some (data, attacker-state) $\in$
set-dist (execute (checked-staged-transcript-program A)
adversary-initial-state) $\Rightarrow$
wp-event verify-monad
(trace-fri-header-tied-recorded-sibling-candidate-missing
(verifier-state-from-adversary attacker-state
(staged-proof-transcript data)))
(verifier-state-from-adversary attacker-state
(staged-proof-transcript data)) $\leq$
TraceMissing data attacker-state

and trace-merkle-bound:

$\wedge$ data attacker-state.

Some (data, attacker-state) $\in$
set-dist (execute (checked-staged-transcript-program A)
adversary-initial-state) $\Rightarrow$
wp-event verify-monad
(partial-merkle-inconsistency-bad
(verifier-state-from-adversary attacker-state
(staged-proof-transcript data)))
(verifier-state-from-adversary attacker-state
(staged-proof-transcript data)) $\leq$ TraceMerkle data attacker-state

and trace-total:

$\wedge$ data attacker-state.

Some (*data*, *attacker-state*) $\in$
set-dist (*execute* (*checked-staged-transcript-program* *A*)
adversary-initial-state) $\implies$
TraceAug *data* *attacker-state* +
(*TraceMissing* *data* *attacker-state* + *TraceMerkle* *data* *attacker-state*)
 $\leq$ *trace-fri-error*
and *composition-sampled-bound*:
 $\wedge$ *data* *attacker-state*.
Some (*data*, *attacker-state*) $\in$
set-dist (*execute* (*checked-staged-transcript-program* *A*)
adversary-initial-state) $\implies$
wp-event *verify-monad*
(*composition-fri-bad-with-verifier-tied-sampled-layer-chain*
(*verifier-state-from-adversary* *attacker-state*
(*staged-proof-transcript* *data*)))
(*verifier-state-from-adversary* *attacker-state*
(*staged-proof-transcript* *data*)) $\leq$
CompSampled *data* *attacker-state*
and *composition-merkle-bound*:
 $\wedge$ *data* *attacker-state*.
Some (*data*, *attacker-state*) $\in$
set-dist (*execute* (*checked-staged-transcript-program* *A*)
adversary-initial-state) $\implies$
wp-event *verify-monad*
(*partial-merkle-inconsistency-bad*
(*verifier-state-from-adversary* *attacker-state*
(*staged-proof-transcript* *data*)))
(*verifier-state-from-adversary* *attacker-state*
(*staged-proof-transcript* *data*)) $\leq$ *CompMerkle* *data* *attacker-state*
and *composition-total*:
 $\wedge$ *data* *attacker-state*.
Some (*data*, *attacker-state*) $\in$
set-dist (*execute* (*checked-staged-transcript-program* *A*)
adversary-initial-state) $\implies$
CompSampled *data* *attacker-state* +
(((*CompMerkle* *data* *attacker-state* + 0) +
(*CompMerkle* *data* *attacker-state* + 0) +
(*CompMerkle* *data* *attacker-state* + 0) +
(*CompMerkle* *data* *attacker-state* + 0 + 0) +
CompMerkle *data* *attacker-state*) + 0)
 $\leq$ *composition-fri-error*
and *empty-trace*:
 $\wedge$ *data* *attacker-state*.
Some (*data*, *attacker-state*) $\in$
set-dist (*execute* (*checked-staged-transcript-program* *A*)
adversary-initial-state) $\implies$
trace-fri-empty-header-reduction
(*verifier-state-from-adversary* *attacker-state*
(*staged-proof-transcript* *data*))


```

shows active-fri-reductions-for A
proof (rule active-fri-reductions-for-from-component-reductions)
fix data attacker-state
assume support:
  Some (data, attacker-state) ∈
    set-dist (execute (checked-staged-transcript-program A)
      adversary-initial-state)
show trace-fri-header-tied-reduction
  (verifier-state-from-adversary attacker-state
    (staged-proof-transcript data))
proof –
let ?s =
  verifier-state-from-adversary attacker-state
    (staged-proof-transcript data)
have aug:
  wp-event verify-monad
    (trace-fri-bad-with-header-tied-augmented-partial-candidate ?s)
    ?s ≤ TraceAug data attacker-state
by (rule trace-augmented-bound[OF support])
have missing:
  wp-event verify-monad
    (trace-fri-header-tied-recorded-sibling-candidate-missing ?s)
    ?s ≤ TraceMissing data attacker-state
by (rule trace-missing-bound[OF support])
have merkle:
  wp-event verify-monad (partial-merkle-inconsistency-bad ?s) ?s
    ≤ TraceMerkle data attacker-state
by (rule trace-merkle-bound[OF support])
have total:
  TraceAug data attacker-state +
    (TraceMissing data attacker-state + TraceMerkle data attacker-state)
    ≤ trace-fri-error
by (rule trace-total[OF support])
show ?thesis
by (rule
  trace-fri-header-tied-reduction-from-augmented-and-recorded-missing-bounds
    [OF aug missing merkle total])
qed
show composition-fri-verifier-tied-reduction
  (verifier-state-from-adversary attacker-state
    (staged-proof-transcript data))
by (rule active-composition-fri-verifier-tied-reductions-from-sampled-and-merkle-bounds
  [OF composition-sampled-bound composition-merkle-bound
    composition-total support])
show trace-fri-empty-header-reduction
  (verifier-state-from-adversary attacker-state
    (staged-proof-transcript data))
by (rule empty-trace[OF support])
qed

```

lemma *active-fri-reductions-for-from-trace-augmented-composition-sampled-and-empty-sampled:*
fixes *TraceAug TraceMissing TraceMerkle CompSampled CompMerkle EmptySampled EmptyMissing ::*

'f staged-proof-data $\Rightarrow$ 'f protocol-channel $\Rightarrow$ prob

assumes *trace-augmented-bound:*

$\wedge$ *data attacker-state.*

*Some (data, attacker-state) $\in$
 set-dist (execute (checked-staged-transcript-program A)
 adversary-initial-state) $\Rightarrow$
 wp-event verify-monad
 (trace-fri-bad-with-header-tied-augmented-partial-candidate
 (verifier-state-from-adversary attacker-state
 (staged-proof-transcript data)))
 (verifier-state-from-adversary attacker-state
 (staged-proof-transcript data)) $\leq$ TraceAug data attacker-state*

and *trace-missing-bound:*

$\wedge$ *data attacker-state.*

*Some (data, attacker-state) $\in$
 set-dist (execute (checked-staged-transcript-program A)
 adversary-initial-state) $\Rightarrow$
 wp-event verify-monad
 (trace-fri-header-tied-recorded-sibling-candidate-missing
 (verifier-state-from-adversary attacker-state
 (staged-proof-transcript data)))
 (verifier-state-from-adversary attacker-state
 (staged-proof-transcript data)) $\leq$
 TraceMissing data attacker-state*

and *trace-merkle-bound:*

$\wedge$ *data attacker-state.*

*Some (data, attacker-state) $\in$
 set-dist (execute (checked-staged-transcript-program A)
 adversary-initial-state) $\Rightarrow$
 wp-event verify-monad
 (partial-merkle-inconsistency-bad
 (verifier-state-from-adversary attacker-state
 (staged-proof-transcript data)))
 (verifier-state-from-adversary attacker-state
 (staged-proof-transcript data)) $\leq$ TraceMerkle data attacker-state*

and *trace-total:*

$\wedge$ *data attacker-state.*

*Some (data, attacker-state) $\in$
 set-dist (execute (checked-staged-transcript-program A)
 adversary-initial-state) $\Rightarrow$
 TraceAug data attacker-state +
 (TraceMissing data attacker-state + TraceMerkle data attacker-state)
 $\leq$ trace-fri-error*

and *composition-sampled-bound:*

$\wedge$ *data attacker-state.*

Some (*data*, *attacker-state*) $\in$
set-dist (*execute* (*checked-staged-transcript-program* *A*)
adversary-initial-state) $\implies$
wp-event verify-monad
(*composition-fri-bad-with-verifier-tied-sampled-layer-chain*
(*verifier-state-from-adversary attacker-state*
(*staged-proof-transcript data*)))
(*verifier-state-from-adversary attacker-state*
(*staged-proof-transcript data*)) $\leq$
CompSampled data attacker-state
and *composition-merkle-bound*:
 $\wedge$ *data attacker-state*.
Some (*data*, *attacker-state*) $\in$
set-dist (*execute* (*checked-staged-transcript-program* *A*)
adversary-initial-state) $\implies$
wp-event verify-monad
(*partial-merkle-inconsistency-bad*
(*verifier-state-from-adversary attacker-state*
(*staged-proof-transcript data*)))
(*verifier-state-from-adversary attacker-state*
(*staged-proof-transcript data*)) $\leq$ *CompMerkle data attacker-state*
and *composition-total*:
 $\wedge$ *data attacker-state*.
Some (*data*, *attacker-state*) $\in$
set-dist (*execute* (*checked-staged-transcript-program* *A*)
adversary-initial-state) $\implies$
CompSampled data attacker-state +
(((*CompMerkle data attacker-state* + 0) +
(*CompMerkle data attacker-state* + 0) +
(*CompMerkle data attacker-state* + 0) +
(*CompMerkle data attacker-state* + 0 + 0) +
CompMerkle data attacker-state) + 0)
 $\leq$ *composition-fri-error*
and *empty-sampled-bound*:
 $\wedge$ *data attacker-state*.
Some (*data*, *attacker-state*) $\in$
set-dist (*execute* (*checked-staged-transcript-program* *A*)
adversary-initial-state) $\implies$
wp-event verify-monad
(*trace-fri-bad-with-sampled-layer-chain*
(*verifier-state-from-adversary attacker-state*
(*staged-proof-transcript data*)))
(*verifier-state-from-adversary attacker-state*
(*staged-proof-transcript data*)) $\leq$
EmptySampled data attacker-state
and *empty-missing-bound*:
 $\wedge$ *data attacker-state*.
Some (*data*, *attacker-state*) $\in$
set-dist (*execute* (*checked-staged-transcript-program* *A*)

```

    adversary-initial-state)  $\impl$ 
wp-event verify-monad
  (trace-fri-reachable-missing-sampled-layer-chain
    (verifier-state-from-adversary attacker-state
      (staged-proof-transcript data)))
  (verifier-state-from-adversary attacker-state
    (staged-proof-transcript data))  $\leq$ 
  EmptyMissing data attacker-state
and empty-total:
 $\wedge$ data attacker-state.
  Some (data, attacker-state)  $\in$ 
    set-dist (execute (checked-staged-transcript-program A)
      adversary-initial-state)  $\impl$ 
    EmptySampled data attacker-state + EmptyMissing data attacker-state
     $\leq$  trace-fri-error
shows active-fri-reductions-for A
proof (rule active-fri-reductions-for-from-trace-augmented-composition-sampled-and-empty)
fix data attacker-state
assume support:
  Some (data, attacker-state)  $\in$ 
    set-dist (execute (checked-staged-transcript-program A)
      adversary-initial-state)
let ?s =
  verifier-state-from-adversary attacker-state
    (staged-proof-transcript data)
show wp-event verify-monad
  (trace-fri-bad-with-header-tied-augmented-partial-candidate ?s) ?s
   $\leq$  TraceAug data attacker-state
by (rule trace-augmented-bound[OF support])
show wp-event verify-monad
  (trace-fri-header-tied-recorded-sibling-candidate-missing ?s) ?s
   $\leq$  TraceMissing data attacker-state
by (rule trace-missing-bound[OF support])
show wp-event verify-monad (partial-merkle-inconsistency-bad ?s) ?s
   $\leq$  TraceMerkle data attacker-state
by (rule trace-merkle-bound[OF support])
show TraceAug data attacker-state +
  (TraceMissing data attacker-state + TraceMerkle data attacker-state)
   $\leq$  trace-fri-error
by (rule trace-total[OF support])
show wp-event verify-monad
  (composition-fri-bad-with-verifier-tied-sampled-layer-chain ?s) ?s
   $\leq$  CompSampled data attacker-state
by (rule composition-sampled-bound[OF support])
show wp-event verify-monad (partial-merkle-inconsistency-bad ?s) ?s
   $\leq$  CompMerkle data attacker-state
by (rule composition-merkle-bound[OF support])
show CompSampled data attacker-state +
  (((CompMerkle data attacker-state + 0) +

```

```

      (CompMerkle data attacker-state + 0) +
      (CompMerkle data attacker-state + 0) +
      (CompMerkle data attacker-state + 0 + 0) +
      CompMerkle data attacker-state) + 0)
    ≤ composition-fri-error
  by (rule composition-total[OF support])
show trace-fri-empty-header-reduction ?s
  by (rule trace-fri-empty-header-reduction-from-sampled-and-missing-bounds
      [OF empty-sampled-bound[OF support]
       empty-missing-bound[OF support] empty-total[OF support]])
qed

lemma active-fri-reductions-for-from-trace-augmented-composition-sampled-and-empty-assignment:
  fixes TraceAug TraceMissing TraceMerkle CompSampled CompMerkle EmptySampled Empty-
Obstruction ::
  'f staged-proof-data ⇒ 'f protocol-channel ⇒ prob
assumes trace-augmented-bound:
  ∧data attacker-state.
  Some (data, attacker-state) ∈
    set-dist (execute (checked-staged-transcript-program A)
      adversary-initial-state) ⇒
  wp-event verify-monad
    (trace-fri-bad-with-header-tied-augmented-partial-candidate
      (verifier-state-from-adversary attacker-state
        (staged-proof-transcript data)))
    (verifier-state-from-adversary attacker-state
      (staged-proof-transcript data)) ≤ TraceAug data attacker-state
and trace-missing-bound:
  ∧data attacker-state.
  Some (data, attacker-state) ∈
    set-dist (execute (checked-staged-transcript-program A)
      adversary-initial-state) ⇒
  wp-event verify-monad
    (trace-fri-header-tied-recorded-sibling-candidate-missing
      (verifier-state-from-adversary attacker-state
        (staged-proof-transcript data)))
    (verifier-state-from-adversary attacker-state
      (staged-proof-transcript data)) ≤
    TraceMissing data attacker-state
and trace-merkle-bound:
  ∧data attacker-state.
  Some (data, attacker-state) ∈
    set-dist (execute (checked-staged-transcript-program A)
      adversary-initial-state) ⇒
  wp-event verify-monad
    (partial-merkle-inconsistency-bad
      (verifier-state-from-adversary attacker-state
        (staged-proof-transcript data)))
    (verifier-state-from-adversary attacker-state
      (staged-proof-transcript data))

```

$(\text{staged-proof-transcript data})) \leq \text{TraceMerkle data attacker-state}$
and *trace-total*:
 $\wedge \text{data attacker-state.}$
 $\text{Some (data, attacker-state)} \in$
 $\text{set-dist (execute (checked-staged-transcript-program A)}$
 $\text{adversary-initial-state)} \implies$
 $\text{TraceAug data attacker-state} +$
 $(\text{TraceMissing data attacker-state} + \text{TraceMerkle data attacker-state})$
 $\leq \text{trace-fri-error}$
and *composition-sampled-bound*:
 $\wedge \text{data attacker-state.}$
 $\text{Some (data, attacker-state)} \in$
 $\text{set-dist (execute (checked-staged-transcript-program A)}$
 $\text{adversary-initial-state)} \implies$
 $\text{wp-event verify-monad}$
 $(\text{composition-fri-bad-with-verifier-tied-sampled-layer-chain}$
 $(\text{verifier-state-from-adversary attacker-state}$
 $(\text{staged-proof-transcript data})))$
 $(\text{verifier-state-from-adversary attacker-state}$
 $(\text{staged-proof-transcript data})) \leq$
 $\text{CompSampled data attacker-state}$
and *composition-merkle-bound*:
 $\wedge \text{data attacker-state.}$
 $\text{Some (data, attacker-state)} \in$
 $\text{set-dist (execute (checked-staged-transcript-program A)}$
 $\text{adversary-initial-state)} \implies$
 $\text{wp-event verify-monad}$
 $(\text{partial-merkle-inconsistency-bad}$
 $(\text{verifier-state-from-adversary attacker-state}$
 $(\text{staged-proof-transcript data})))$
 $(\text{verifier-state-from-adversary attacker-state}$
 $(\text{staged-proof-transcript data})) \leq \text{CompMerkle data attacker-state}$
and *composition-total*:
 $\wedge \text{data attacker-state.}$
 $\text{Some (data, attacker-state)} \in$
 $\text{set-dist (execute (checked-staged-transcript-program A)}$
 $\text{adversary-initial-state)} \implies$
 $\text{CompSampled data attacker-state} +$
 $((\text{CompMerkle data attacker-state} + 0) +$
 $(\text{CompMerkle data attacker-state} + 0) +$
 $(\text{CompMerkle data attacker-state} + 0) +$
 $(\text{CompMerkle data attacker-state} + 0 + 0) +$
 $\text{CompMerkle data attacker-state}) + 0)$
 $\leq \text{composition-fri-error}$
and *empty-sampled-bound*:
 $\wedge \text{data attacker-state.}$
 $\text{Some (data, attacker-state)} \in$
 $\text{set-dist (execute (checked-staged-transcript-program A)}$
 $\text{adversary-initial-state)} \implies$

```

wp-event verify-monad
  (trace-fri-bad-with-sampled-layer-chain
    (verifier-state-from-adversary attacker-state
      (staged-proof-transcript data)))
  (verifier-state-from-adversary attacker-state
    (staged-proof-transcript data)) ≤
  EmptySampled data attacker-state
and empty-obstruction-bound:
 $\wedge$  data attacker-state.
  Some (data, attacker-state) ∈
    set-dist (execute (checked-staged-transcript-program A)
      adversary-initial-state)  $\implies$ 
wp-event verify-monad
  (trace-fri-sampled-layer-assignment-obstruction
    (verifier-state-from-adversary attacker-state
      (staged-proof-transcript data)))
  (verifier-state-from-adversary attacker-state
    (staged-proof-transcript data)) ≤
  EmptyObstruction data attacker-state
and empty-total:
 $\wedge$  data attacker-state.
  Some (data, attacker-state) ∈
    set-dist (execute (checked-staged-transcript-program A)
      adversary-initial-state)  $\implies$ 
  EmptySampled data attacker-state + EmptyObstruction data attacker-state
  ≤ trace-fri-error
shows active-fri-reductions-for A
proof (rule active-fri-reductions-for-from-trace-augmented-composition-sampled-and-empty)
fix data attacker-state
assume support:
  Some (data, attacker-state) ∈
    set-dist (execute (checked-staged-transcript-program A)
      adversary-initial-state)
let ?s =
  verifier-state-from-adversary attacker-state
    (staged-proof-transcript data)
show wp-event verify-monad
  (trace-fri-bad-with-header-tied-augmented-partial-candidate ?s) ?s
  ≤ TraceAug data attacker-state
by (rule trace-augmented-bound[OF support])
show wp-event verify-monad
  (trace-fri-header-tied-recorded-sibling-candidate-missing ?s) ?s
  ≤ TraceMissing data attacker-state
by (rule trace-missing-bound[OF support])
show wp-event verify-monad (partial-merkle-inconsistency-bad ?s) ?s
  ≤ TraceMerkle data attacker-state
by (rule trace-merkle-bound[OF support])
show TraceAug data attacker-state +
  (TraceMissing data attacker-state + TraceMerkle data attacker-state)

```

```

    ≤ trace-fri-error
  by (rule trace-total[OF support])
show wp-event verify-monad
  (composition-fri-bad-with-verifier-tied-sampled-layer-chain ?s) ?s
  ≤ CompSampled data attacker-state
  by (rule composition-sampled-bound[OF support])
show wp-event verify-monad (partial-merkle-inconsistency-bad ?s) ?s
  ≤ CompMerkle data attacker-state
  by (rule composition-merkle-bound[OF support])
show CompSampled data attacker-state +
  (((CompMerkle data attacker-state + 0) +
    (CompMerkle data attacker-state + 0) +
    (CompMerkle data attacker-state + 0) +
    (CompMerkle data attacker-state + 0 + 0) +
    CompMerkle data attacker-state) + 0)
  ≤ composition-fri-error
  by (rule composition-total[OF support])
show trace-fri-empty-header-reduction ?s
  by (rule
    trace-fri-empty-header-reduction-from-sampled-and-assignment-obstruction-bounds
    [OF empty-sampled-bound[OF support]
     empty-obstruction-bound[OF support] empty-total[OF support]])
qed

```

lemma *active-fri-reductions-for-from-trace-augmented-composition-sampled-and-empty-conflict-zero:*

fixes *TraceAug TraceMissing TraceMerkle CompSampled CompMerkle EmptySampled EmptyConflict EmptyZero ::*

'f staged-proof-data ⇒ 'f protocol-channel ⇒ prob

assumes *trace-augmented-bound:*

$\wedge$ *data attacker-state.*

Some (data, attacker-state) ∈
set-dist (execute (checked-staged-transcript-program A)
adversary-initial-state) ⇒
wp-event verify-monad
(trace-fri-bad-with-header-tied-augmented-partial-candidate
(verifier-state-from-adversary attacker-state
(staged-proof-transcript data)))
(verifier-state-from-adversary attacker-state
(staged-proof-transcript data)) ≤ TraceAug data attacker-state

and *trace-missing-bound:*

$\wedge$ *data attacker-state.*

Some (data, attacker-state) ∈
set-dist (execute (checked-staged-transcript-program A)
adversary-initial-state) ⇒
wp-event verify-monad
(trace-fri-header-tied-recorded-sibling-candidate-missing
(verifier-state-from-adversary attacker-state
(staged-proof-transcript data)))
(verifier-state-from-adversary attacker-state

$(\text{staged-proof-transcript data}) \leq$
 $\text{TraceMissing data attacker-state}$
and *trace-merkle-bound*:
 $\wedge \text{data attacker-state.}$
 $\text{Some (data, attacker-state)} \in$
 $\text{set-dist (execute (checked-staged-transcript-program A)}$
 $\text{adversary-initial-state)} \implies$
 $\text{wp-event verify-monad}$
 $(\text{partial-merkle-inconsistency-bad}$
 $(\text{verifier-state-from-adversary attacker-state}$
 $(\text{staged-proof-transcript data})))$
 $(\text{verifier-state-from-adversary attacker-state}$
 $(\text{staged-proof-transcript data})) \leq \text{TraceMerkle data attacker-state}$
and *trace-total*:
 $\wedge \text{data attacker-state.}$
 $\text{Some (data, attacker-state)} \in$
 $\text{set-dist (execute (checked-staged-transcript-program A)}$
 $\text{adversary-initial-state)} \implies$
 $\text{TraceAug data attacker-state} +$
 $(\text{TraceMissing data attacker-state} + \text{TraceMerkle data attacker-state})$
 $\leq \text{trace-fri-error}$
and *composition-sampled-bound*:
 $\wedge \text{data attacker-state.}$
 $\text{Some (data, attacker-state)} \in$
 $\text{set-dist (execute (checked-staged-transcript-program A)}$
 $\text{adversary-initial-state)} \implies$
 $\text{wp-event verify-monad}$
 $(\text{composition-fri-bad-with-verifier-tied-sampled-layer-chain}$
 $(\text{verifier-state-from-adversary attacker-state}$
 $(\text{staged-proof-transcript data})))$
 $(\text{verifier-state-from-adversary attacker-state}$
 $(\text{staged-proof-transcript data})) \leq$
 $\text{CompSampled data attacker-state}$
and *composition-merkle-bound*:
 $\wedge \text{data attacker-state.}$
 $\text{Some (data, attacker-state)} \in$
 $\text{set-dist (execute (checked-staged-transcript-program A)}$
 $\text{adversary-initial-state)} \implies$
 $\text{wp-event verify-monad}$
 $(\text{partial-merkle-inconsistency-bad}$
 $(\text{verifier-state-from-adversary attacker-state}$
 $(\text{staged-proof-transcript data})))$
 $(\text{verifier-state-from-adversary attacker-state}$
 $(\text{staged-proof-transcript data})) \leq \text{CompMerkle data attacker-state}$
and *composition-total*:
 $\wedge \text{data attacker-state.}$
 $\text{Some (data, attacker-state)} \in$
 $\text{set-dist (execute (checked-staged-transcript-program A)}$
 $\text{adversary-initial-state)} \implies$

CompSampled data attacker-state +
((CompMerkle data attacker-state + 0) +
(CompMerkle data attacker-state + 0) +
(CompMerkle data attacker-state + 0) +
(CompMerkle data attacker-state + 0 + 0) +
CompMerkle data attacker-state) + 0)
 $\leq$ *composition-fri-error*
and *empty-sampled-bound:*
 $\wedge$ *data attacker-state.*
Some (data, attacker-state) $\in$
set-dist (execute (checked-staged-transcript-program A)
adversary-initial-state) $\implies$
wp-event verify-monad
(trace-fri-bad-with-sampled-layer-chain
(verifier-state-from-adversary attacker-state
(staged-proof-transcript data)))
(verifier-state-from-adversary attacker-state
(staged-proof-transcript data)) $\leq$
EmptySampled data attacker-state
and *empty-conflict-bound:*
 $\wedge$ *data attacker-state.*
Some (data, attacker-state) $\in$
set-dist (execute (checked-staged-transcript-program A)
adversary-initial-state) $\implies$
wp-event verify-monad
(trace-fri-sampled-assignment-conflict
(verifier-state-from-adversary attacker-state
(staged-proof-transcript data)))
(verifier-state-from-adversary attacker-state
(staged-proof-transcript data)) $\leq$
EmptyConflict data attacker-state
and *empty-zero-bound:*
 $\wedge$ *data attacker-state.*
Some (data, attacker-state) $\in$
set-dist (execute (checked-staged-transcript-program A)
adversary-initial-state) $\implies$
wp-event verify-monad
(trace-fri-zero-round-final-obstruction
(verifier-state-from-adversary attacker-state
(staged-proof-transcript data)))
(verifier-state-from-adversary attacker-state
(staged-proof-transcript data)) $\leq$
EmptyZero data attacker-state
and *empty-total:*
 $\wedge$ *data attacker-state.*
Some (data, attacker-state) $\in$
set-dist (execute (checked-staged-transcript-program A)
adversary-initial-state) $\implies$
EmptySampled data attacker-state +

```

      (EmptyConflict data attacker-state + EmptyZero data attacker-state)
    ≤ trace-fri-error
  shows active-fri-reductions-for A
proof (rule active-fri-reductions-for-from-trace-augmented-composition-sampled-and-empty-assignment
  [where EmptyObstruction =
    λdata attacker-state.
      EmptyConflict data attacker-state + EmptyZero data attacker-state])
fix data attacker-state
assume support:
  Some (data, attacker-state) ∈
    set-dist (execute (checked-staged-transcript-program A)
      adversary-initial-state)
let ?s =
  verifier-state-from-adversary attacker-state
  (staged-proof-transcript data)
show wp-event verify-monad
  (trace-fri-bad-with-header-tied-augmented-partial-candidate ?s) ?s
  ≤ TraceAug data attacker-state
  by (rule trace-augmented-bound[OF support])
show wp-event verify-monad
  (trace-fri-header-tied-recorded-sibling-candidate-missing ?s) ?s
  ≤ TraceMissing data attacker-state
  by (rule trace-missing-bound[OF support])
show wp-event verify-monad (partial-merkle-inconsistency-bad ?s) ?s
  ≤ TraceMerkle data attacker-state
  by (rule trace-merkle-bound[OF support])
show TraceAug data attacker-state +
  (TraceMissing data attacker-state + TraceMerkle data attacker-state)
  ≤ trace-fri-error
  by (rule trace-total[OF support])
show wp-event verify-monad
  (composition-fri-bad-with-verifier-tied-sampled-layer-chain ?s) ?s
  ≤ CompSampled data attacker-state
  by (rule composition-sampled-bound[OF support])
show wp-event verify-monad (partial-merkle-inconsistency-bad ?s) ?s
  ≤ CompMerkle data attacker-state
  by (rule composition-merkle-bound[OF support])
show CompSampled data attacker-state +
  (((CompMerkle data attacker-state + 0) +
    (CompMerkle data attacker-state + 0) +
    (CompMerkle data attacker-state + 0) +
    (CompMerkle data attacker-state + 0 + 0) +
    CompMerkle data attacker-state) + 0)
  ≤ composition-fri-error
  by (rule composition-total[OF support])
show wp-event verify-monad
  (trace-fri-bad-with-sampled-layer-chain ?s) ?s
  ≤ EmptySampled data attacker-state
  by (rule empty-sampled-bound[OF support])

```

show *wp-event verify-monad*
 (*trace-fri-sampled-layer-assignment-obstruction ?s*) ?*s*
 $\leq$ *EmptyConflict data attacker-state + EmptyZero data attacker-state*
by (rule
 wp-trace-fri-sampled-layer-assignment-obstruction-bound-from-conflict-and-zero)
 (rule *empty-conflict-bound*[*OF support*],
 rule *empty-zero-bound*[*OF support*])
show *EmptySampled data attacker-state +*
 (*EmptyConflict data attacker-state + EmptyZero data attacker-state*)
 $\leq$ *trace-fri-error*
by (rule *empty-total*[*OF support*])
qed

lemma *active-fri-reductions-for-from-trace-augmented-conflict-zero-composition-sampled-and-empty-conflict-zero:*
fixes *TraceAug TraceConflict TraceZero TraceMerkle CompSampled CompMerkle*
EmptySampled EmptyConflict EmptyZero ::
'f staged-proof-data $\Rightarrow$ 'f protocol-channel $\Rightarrow$ prob
assumes *trace-augmented-bound:*
 $\wedge$ *data attacker-state.*
Some (data, attacker-state) $\in$
set-dist (execute (checked-staged-transcript-program A)
adversary-initial-state) $\Rightarrow$
wp-event verify-monad
 (*trace-fri-bad-with-header-tied-augmented-partial-candidate*
 (*verifier-state-from-adversary attacker-state*
 (*staged-proof-transcript data*)))
 (*verifier-state-from-adversary attacker-state*
 (*staged-proof-transcript data*)) $\leq$ *TraceAug data attacker-state*
and *trace-conflict-bound:*
 $\wedge$ *data attacker-state.*
Some (data, attacker-state) $\in$
set-dist (execute (checked-staged-transcript-program A)
adversary-initial-state) $\Rightarrow$
wp-event verify-monad
 (*trace-fri-header-tied-sampled-assignment-conflict*
 (*verifier-state-from-adversary attacker-state*
 (*staged-proof-transcript data*)))
 (*verifier-state-from-adversary attacker-state*
 (*staged-proof-transcript data*)) $\leq$ *TraceConflict data attacker-state*
and *trace-zero-bound:*
 $\wedge$ *data attacker-state.*
Some (data, attacker-state) $\in$
set-dist (execute (checked-staged-transcript-program A)
adversary-initial-state) $\Rightarrow$
wp-event verify-monad
 (*trace-fri-header-tied-zero-round-final-obstruction*
 (*verifier-state-from-adversary attacker-state*
 (*staged-proof-transcript data*)))
 (*verifier-state-from-adversary attacker-state*

$(\text{staged-proof-transcript data})) \leq \text{TraceZero data attacker-state}$
and *trace-merkle-bound*:
 $\wedge \text{data attacker-state.}$
 $\text{Some (data, attacker-state)} \in$
 $\text{set-dist (execute (checked-staged-transcript-program A)}$
 $\text{adversary-initial-state)} \implies$
 $\text{wp-event verify-monad}$
 $(\text{partial-merkle-inconsistency-bad}$
 $(\text{verifier-state-from-adversary attacker-state}$
 $(\text{staged-proof-transcript data})))$
 $(\text{verifier-state-from-adversary attacker-state}$
 $(\text{staged-proof-transcript data})) \leq \text{TraceMerkle data attacker-state}$
and *trace-total*:
 $\wedge \text{data attacker-state.}$
 $\text{Some (data, attacker-state)} \in$
 $\text{set-dist (execute (checked-staged-transcript-program A)}$
 $\text{adversary-initial-state)} \implies$
 $\text{TraceAug data attacker-state} +$
 $((\text{TraceConflict data attacker-state} + \text{TraceZero data attacker-state}) +$
 $\text{TraceMerkle data attacker-state})$
 $\leq \text{trace-fri-error}$
and *composition-sampled-bound*:
 $\wedge \text{data attacker-state.}$
 $\text{Some (data, attacker-state)} \in$
 $\text{set-dist (execute (checked-staged-transcript-program A)}$
 $\text{adversary-initial-state)} \implies$
 $\text{wp-event verify-monad}$
 $(\text{composition-fri-bad-with-verifier-tied-sampled-layer-chain}$
 $(\text{verifier-state-from-adversary attacker-state}$
 $(\text{staged-proof-transcript data})))$
 $(\text{verifier-state-from-adversary attacker-state}$
 $(\text{staged-proof-transcript data})) \leq$
 $\text{CompSampled data attacker-state}$
and *composition-merkle-bound*:
 $\wedge \text{data attacker-state.}$
 $\text{Some (data, attacker-state)} \in$
 $\text{set-dist (execute (checked-staged-transcript-program A)}$
 $\text{adversary-initial-state)} \implies$
 $\text{wp-event verify-monad}$
 $(\text{partial-merkle-inconsistency-bad}$
 $(\text{verifier-state-from-adversary attacker-state}$
 $(\text{staged-proof-transcript data})))$
 $(\text{verifier-state-from-adversary attacker-state}$
 $(\text{staged-proof-transcript data})) \leq \text{CompMerkle data attacker-state}$
and *composition-total*:
 $\wedge \text{data attacker-state.}$
 $\text{Some (data, attacker-state)} \in$
 $\text{set-dist (execute (checked-staged-transcript-program A)}$
 $\text{adversary-initial-state)} \implies$

$\text{CompSampled data attacker-state} +$
 $((\text{CompMerkle data attacker-state} + 0) +$
 $(\text{CompMerkle data attacker-state} + 0) +$
 $(\text{CompMerkle data attacker-state} + 0) +$
 $(\text{CompMerkle data attacker-state} + 0 + 0) +$
 $\text{CompMerkle data attacker-state}) + 0)$
 $\leq \text{composition-fri-error}$
and empty-sampled-bound:
 $\wedge \text{data attacker-state.}$
 $\text{Some (data, attacker-state)} \in$
 $\text{set-dist (execute (checked-staged-transcript-program } A)$
 $\text{adversary-initial-state})} \implies$
 $\text{wp-event verify-monad}$
 $(\text{trace-fri-bad-with-sampled-layer-chain}$
 $(\text{verifier-state-from-adversary attacker-state}$
 $(\text{staged-proof-transcript data})))$
 $(\text{verifier-state-from-adversary attacker-state}$
 $(\text{staged-proof-transcript data})) \leq$
 $\text{EmptySampled data attacker-state}$
and empty-conflict-bound:
 $\wedge \text{data attacker-state.}$
 $\text{Some (data, attacker-state)} \in$
 $\text{set-dist (execute (checked-staged-transcript-program } A)$
 $\text{adversary-initial-state})} \implies$
 $\text{wp-event verify-monad}$
 $(\text{trace-fri-sampled-assignment-conflict}$
 $(\text{verifier-state-from-adversary attacker-state}$
 $(\text{staged-proof-transcript data})))$
 $(\text{verifier-state-from-adversary attacker-state}$
 $(\text{staged-proof-transcript data})) \leq$
 $\text{EmptyConflict data attacker-state}$
and empty-zero-bound:
 $\wedge \text{data attacker-state.}$
 $\text{Some (data, attacker-state)} \in$
 $\text{set-dist (execute (checked-staged-transcript-program } A)$
 $\text{adversary-initial-state})} \implies$
 $\text{wp-event verify-monad}$
 $(\text{trace-fri-zero-round-final-obstruction}$
 $(\text{verifier-state-from-adversary attacker-state}$
 $(\text{staged-proof-transcript data})))$
 $(\text{verifier-state-from-adversary attacker-state}$
 $(\text{staged-proof-transcript data})) \leq$
 $\text{EmptyZero data attacker-state}$
and empty-total:
 $\wedge \text{data attacker-state.}$
 $\text{Some (data, attacker-state)} \in$
 $\text{set-dist (execute (checked-staged-transcript-program } A)$
 $\text{adversary-initial-state})} \implies$
 $\text{EmptySampled data attacker-state} +$

$(\text{EmptyConflict data attacker-state} + \text{EmptyZero data attacker-state})$
 $\leq \text{trace-fri-error}$
shows *active-fri-reductions-for A*
proof (rule *active-fri-reductions-for-from-component-reductions*)
fix *data attacker-state*
assume *support*:
Some (data, attacker-state) ∈
set-dist (execute (checked-staged-transcript-program A)
adversary-initial-state)
let *?s =*
verifier-state-from-adversary attacker-state
(staged-proof-transcript data)
show *trace-fri-header-tied-reduction ?s*
by (rule
trace-fri-header-tied-reduction-from-augmented-conflict-zero-merkle-bounds)
(rule *trace-augmented-bound[OF support]*,
rule *trace-conflict-bound[OF support]*,
rule *trace-zero-bound[OF support]*,
rule *trace-merkle-bound[OF support]*,
rule *trace-total[OF support]*)
show *composition-fri-verifier-tied-reduction ?s*
by (rule *active-composition-fri-verifier-tied-reductions-from-sampled-and-merkle-bounds*
[*OF composition-sampled-bound composition-merkle-bound*
composition-total support])
show *trace-fri-empty-header-reduction ?s*
by (rule *trace-fri-empty-header-reduction-from-sampled-conflict-and-zero-bounds*)
(rule *empty-sampled-bound[OF support]*,
rule *empty-conflict-bound[OF support]*,
rule *empty-zero-bound[OF support]*,
rule *empty-total[OF support]*)
qed

lemma *active-fri-reductions-for-from-trace-augmented-composition-canonical-and-empty-conflict-zero*:
fixes *TraceAug TraceMissing TraceMerkle CompCanonical CompMerkle EmptyCanonical EmptyConflict EmptyZero ::*
'f staged-proof-data ⇒ 'f protocol-channel ⇒ prob
assumes *trace-augmented-bound*:
 $\bigwedge \text{data attacker-state.}$
Some (data, attacker-state) ∈
set-dist (execute (checked-staged-transcript-program A)
adversary-initial-state) ⇒
wp-event verify-monad
(trace-fri-bad-with-header-tied-augmented-partial-candidate
(verifier-state-from-adversary attacker-state
(staged-proof-transcript data)))
(verifier-state-from-adversary attacker-state
(staged-proof-transcript data)) ≤ TraceAug data attacker-state
and *trace-missing-bound*:
 $\bigwedge \text{data attacker-state.}$

Some (data, attacker-state) ∈
set-dist (execute (checked-staged-transcript-program A)
adversary-initial-state) ⇒
wp-event verify-monad
(trace-fri-header-tied-recorded-sibling-candidate-missing
(verifier-state-from-adversary attacker-state
(staged-proof-transcript data)))
(verifier-state-from-adversary attacker-state
(staged-proof-transcript data)) ≤
TraceMissing data attacker-state
and *trace-merkle-bound:*
 $\wedge$ *data attacker-state.*
Some (data, attacker-state) ∈
set-dist (execute (checked-staged-transcript-program A)
adversary-initial-state) ⇒
wp-event verify-monad
(partial-merkle-inconsistency-bad
(verifier-state-from-adversary attacker-state
(staged-proof-transcript data)))
(verifier-state-from-adversary attacker-state
(staged-proof-transcript data)) ≤ TraceMerkle data attacker-state
and *trace-total:*
 $\wedge$ *data attacker-state.*
Some (data, attacker-state) ∈
set-dist (execute (checked-staged-transcript-program A)
adversary-initial-state) ⇒
TraceAug data attacker-state +
(TraceMissing data attacker-state + TraceMerkle data attacker-state)
 $\leq$ *trace-fri-error*
and *composition-canonical-bound:*
 $\wedge$ *data attacker-state.*
Some (data, attacker-state) ∈
set-dist (execute (checked-staged-transcript-program A)
adversary-initial-state) ⇒
wp-event verify-monad
(composition-fri-bad-with-canonical-sampled-layer-chain
(verifier-state-from-adversary attacker-state
(staged-proof-transcript data)))
(verifier-state-from-adversary attacker-state
(staged-proof-transcript data)) ≤
CompCanonical data attacker-state
and *composition-merkle-bound:*
 $\wedge$ *data attacker-state.*
Some (data, attacker-state) ∈
set-dist (execute (checked-staged-transcript-program A)
adversary-initial-state) ⇒
wp-event verify-monad
(partial-merkle-inconsistency-bad
(verifier-state-from-adversary attacker-state

$(\text{staged-proof-transcript data}))$
 $(\text{verifier-state-from-adversary attacker-state}$
 $(\text{staged-proof-transcript data})) \leq \text{CompMerkle data attacker-state}$
and *composition-total*:
 $\wedge \text{data attacker-state.}$
 $\text{Some (data, attacker-state)} \in$
 $\text{set-dist (execute (checked-staged-transcript-program A)}$
 $\text{adversary-initial-state)} \implies$
 $\text{CompCanonical data attacker-state} +$
 $((\text{CompMerkle data attacker-state} + 0) +$
 $(\text{CompMerkle data attacker-state} + 0) +$
 $(\text{CompMerkle data attacker-state} + 0) +$
 $(\text{CompMerkle data attacker-state} + 0 + 0) +$
 $\text{CompMerkle data attacker-state}) + 0)$
 $\leq \text{composition-fri-error}$
and *empty-canonical-bound*:
 $\wedge \text{data attacker-state.}$
 $\text{Some (data, attacker-state)} \in$
 $\text{set-dist (execute (checked-staged-transcript-program A)}$
 $\text{adversary-initial-state)} \implies$
 $\text{wp-event verify-monad}$
 $(\text{trace-fri-bad-with-canonical-sampled-layer-chain}$
 $(\text{verifier-state-from-adversary attacker-state}$
 $(\text{staged-proof-transcript data})))$
 $(\text{verifier-state-from-adversary attacker-state}$
 $(\text{staged-proof-transcript data})) \leq$
 $\text{EmptyCanonical data attacker-state}$
and *empty-conflict-bound*:
 $\wedge \text{data attacker-state.}$
 $\text{Some (data, attacker-state)} \in$
 $\text{set-dist (execute (checked-staged-transcript-program A)}$
 $\text{adversary-initial-state)} \implies$
 $\text{wp-event verify-monad}$
 $(\text{trace-fri-sampled-assignment-conflict}$
 $(\text{verifier-state-from-adversary attacker-state}$
 $(\text{staged-proof-transcript data})))$
 $(\text{verifier-state-from-adversary attacker-state}$
 $(\text{staged-proof-transcript data})) \leq$
 $\text{EmptyConflict data attacker-state}$
and *empty-zero-bound*:
 $\wedge \text{data attacker-state.}$
 $\text{Some (data, attacker-state)} \in$
 $\text{set-dist (execute (checked-staged-transcript-program A)}$
 $\text{adversary-initial-state)} \implies$
 $\text{wp-event verify-monad}$
 $(\text{trace-fri-zero-round-final-obstruction}$
 $(\text{verifier-state-from-adversary attacker-state}$
 $(\text{staged-proof-transcript data})))$
 $(\text{verifier-state-from-adversary attacker-state}$

```

    (staged-proof-transcript data)) ≤
    EmptyZero data attacker-state
and empty-total:
  ∧ data attacker-state.
    Some (data, attacker-state) ∈
      set-dist (execute (checked-staged-transcript-program A)
        adversary-initial-state) ⇒
      EmptyCanonical data attacker-state +
      (EmptyConflict data attacker-state + EmptyZero data attacker-state)
    ≤ trace-fri-error
shows active-fri-reductions-for A
proof (rule
  active-fri-reductions-for-from-trace-augmented-composition-sampled-and-empty-conflict-zero)
fix data attacker-state
assume support:
  Some (data, attacker-state) ∈
    set-dist (execute (checked-staged-transcript-program A)
      adversary-initial-state)
let ?s =
  verifier-state-from-adversary attacker-state
  (staged-proof-transcript data)
show wp-event verify-monad
  (trace-fri-bad-with-header-tied-augmented-partial-candidate ?s) ?s
  ≤ TraceAug data attacker-state
by (rule trace-augmented-bound[OF support])
show wp-event verify-monad
  (trace-fri-header-tied-recorded-sibling-candidate-missing ?s) ?s
  ≤ TraceMissing data attacker-state
by (rule trace-missing-bound[OF support])
show wp-event verify-monad (partial-merkle-inconsistency-bad ?s) ?s
  ≤ TraceMerkle data attacker-state
by (rule trace-merkle-bound[OF support])
show TraceAug data attacker-state +
  (TraceMissing data attacker-state + TraceMerkle data attacker-state)
  ≤ trace-fri-error
by (rule trace-total[OF support])
show wp-event verify-monad
  (composition-fri-bad-with-verifier-tied-sampled-layer-chain ?s) ?s
  ≤ CompCanonical data attacker-state
by (rule
  wp-composition-fri-verifier-tied-sampled-layer-chain-bound-from-sampled)
  (rule wp-composition-fri-sampled-bound-from-canonical,
  rule composition-canonical-bound[OF support])
show wp-event verify-monad (partial-merkle-inconsistency-bad ?s) ?s
  ≤ CompMerkle data attacker-state
by (rule composition-merkle-bound[OF support])
show CompCanonical data attacker-state +
  (((CompMerkle data attacker-state + 0) +
  (CompMerkle data attacker-state + 0) +

```

```

      (CompMerkle data attacker-state + 0) +
      (CompMerkle data attacker-state + 0 + 0) +
      CompMerkle data attacker-state) + 0)
    ≤ composition-fri-error
  by (rule composition-total[OF support])
show wp-event verify-monad
  (trace-fri-bad-with-sampled-layer-chain ?s) ?s
  ≤ EmptyCanonical data attacker-state
  by (rule wp-trace-fri-sampled-bound-from-canonical)
  (rule empty-canonical-bound[OF support])
show wp-event verify-monad
  (trace-fri-sampled-assignment-conflict ?s) ?s
  ≤ EmptyConflict data attacker-state
  by (rule empty-conflict-bound[OF support])
show wp-event verify-monad
  (trace-fri-zero-round-final-obstruction ?s) ?s
  ≤ EmptyZero data attacker-state
  by (rule empty-zero-bound[OF support])
show EmptyCanonical data attacker-state +
  (EmptyConflict data attacker-state + EmptyZero data attacker-state)
  ≤ trace-fri-error
  by (rule empty-total[OF support])
qed

end

end

```

```

theory Soundness-FRI-Sampled-Interface
imports
  Soundness-FRI-Derived-Active-Reductions
begin

```

Sampled-query FRI interface.

This theory names the FRI bad-candidate event that matches the verifier's sampled evidence: authenticated partial openings, sampled fold checks on the canonical domains, final consistency, and a non-low-degree candidate. It is deliberately separate from the full-cover compatibility route, where sampled openings are temporarily strengthened to full fold-index coverage.

```

context soundness
begin

```

```

definition fri-query-index-list-space :: nat list set
where
  fri-query-index-list-space =
    {query-idxs. length query-idxs = rounds ∧
     set query-idxs ⊆ query-sample-space}

```

definition *trace-fri-query-index-list-set-hit*
 $:: ('f, 'a) \text{ protocol-channel-scheme} \Rightarrow \text{nat list set} \Rightarrow$
 $(\text{unit list} \times ('f, 'a) \text{ protocol-channel-scheme}) \text{ option} \Rightarrow \text{bool}$
where
 $\text{trace-fri-query-index-list-set-hit } s \ Q \ \text{out} \longleftrightarrow$
 $(\exists \text{ trace-roots trace-bs trace-final dg composition-roots}$
 $\text{composition-bs composition-final fri-query-idxs trace-round-layers}$
 $\text{composition-round-layers.}$
 $\text{accepted-fri-opening-transcript } s \ \text{out trace-roots trace-bs}$
 $\text{trace-final dg composition-roots composition-bs composition-final}$
 $\text{fri-query-idxs trace-round-layers composition-round-layers} \wedge$
 $\text{fri-query-idxs} \in Q)$

definition *composition-fri-query-index-list-set-hit*
 $:: ('f, 'a) \text{ protocol-channel-scheme} \Rightarrow \text{nat list set} \Rightarrow$
 $(\text{unit list} \times ('f, 'a) \text{ protocol-channel-scheme}) \text{ option} \Rightarrow \text{bool}$
where
 $\text{composition-fri-query-index-list-set-hit } s \ Q \ \text{out} \longleftrightarrow$
 $(\exists \text{ trace-roots trace-bs trace-final fri-dg composition-roots}$
 $\text{composition-bs composition-final fri-query-idxs trace-round-layers}$
 $\text{composition-round-layers.}$
 $\text{accepted-fri-opening-transcript } s \ \text{out trace-roots trace-bs}$
 $\text{trace-final fri-dg composition-roots composition-bs}$
 $\text{composition-final fri-query-idxs trace-round-layers}$
 $\text{composition-round-layers} \wedge$
 $\text{fri-query-idxs} \in Q)$

definition *query-index-list-entries* $:: \text{nat list set} \Rightarrow \text{nat set}$
where
 $\text{query-index-list-entries } Q = (\bigcup \text{ query-idxs} \in Q. \text{ set query-idxs})$

lemma *query-index-list-entries-subset-query-sample-space:*
assumes $Q \subseteq \text{fri-query-index-list-space}$
shows $\text{query-index-list-entries } Q \subseteq \text{query-sample-space}$
using *assms*
unfolding *query-index-list-entries-def fri-query-index-list-space-def*
by *auto*

lemma *finite-query-index-list-entries:*
assumes $Q \subseteq \text{fri-query-index-list-space}$
shows $\text{finite } (\text{query-index-list-entries } Q)$
using *query-index-list-entries-subset-query-sample-space[OF assms]*
by *(rule finite-subset) (rule finite-query-sample-space)*

lemma *query-index-list-entries-card-le:*
assumes *finite-Q: finite Q*
and *subset: $Q \subseteq \text{fri-query-index-list-space}$*
shows $\text{card } (\text{query-index-list-entries } Q) \leq \text{card } Q * \text{rounds}$

```

proof –
  have
     $\text{card } (\text{query-index-list-entries } Q) \leq (\sum \text{query-idxs} \in Q. \text{card } (\text{set query-idxs}))$ 
    unfolding query-index-list-entries-def
    by (rule card-UN-le) (rule finite-Q)
  also have  $\dots \leq (\sum \text{query-idxs} \in Q. \text{rounds})$ 
  proof (rule sum-mono)
    fix query-idxs
    assume  $\text{query-idxs} \in Q$ 
    then have  $\text{length query-idxs} = \text{rounds}$ 
      using subset unfolding fri-query-index-list-space-def by auto
    then show  $\text{card } (\text{set query-idxs}) \leq \text{rounds}$ 
      using card-length[of query-idxs] by simp
    qed
  also have  $\dots = \text{card } Q * \text{rounds}$ 
    using finite-Q by simp
  finally show ?thesis .
qed

definition generic-fri-sampled-query-candidate-evidence
  :: ('f list  $\Rightarrow$  bool)  $\Rightarrow$  ('f list  $\Rightarrow$  bool)  $\Rightarrow$ 
    ('f list  $\Rightarrow$  nat  $\Rightarrow$ 
      ('f list  $\Rightarrow$  'f list  $\Rightarrow$  'f  $\Rightarrow$  nat list  $\Rightarrow$ 
        ('f list list list  $\Rightarrow$  'f list list  $\Rightarrow$  bool)))
where
  generic-fri-sampled-query-candidate-evidence low-degree candidate-bad
    candidate-table degree-bound roots challenges final-value query-idxs
    round-layers layers  $\longleftrightarrow$ 
  generic-fri-canonical-sampled-layer-chain-evidence low-degree
    candidate-table degree-bound roots challenges final-value query-idxs
    round-layers layers  $\wedge$ 
    candidate-bad candidate-table

lemma generic-fri-sampled-query-candidate-evidenceI:
  assumes
    generic-fri-canonical-sampled-layer-chain-evidence low-degree
      candidate-table degree-bound roots challenges final-value query-idxs
      round-layers layers
      candidate-bad candidate-table
  shows
    generic-fri-sampled-query-candidate-evidence low-degree candidate-bad
      candidate-table degree-bound roots challenges final-value query-idxs
      round-layers layers
  using assms
  unfolding generic-fri-sampled-query-candidate-evidence-def by blast

lemma generic-fri-sampled-query-candidate-evidenceD:
  assumes

```

*generic-fri-sampled-query-candidate-evidence low-degree candidate-bad
candidate-table degree-bound roots challenges final-value query-idxs
round-layers layers*

shows

*generic-fri-canonical-sampled-layer-chain-evidence low-degree
candidate-table degree-bound roots challenges final-value query-idxs
round-layers layers
candidate-bad candidate-table*

using *assms*

unfolding *generic-fri-sampled-query-candidate-evidence-def* **by** *blast+*

definition *generic-fri-sampled-query-bad-pair-set*

$:: ('f \text{ list} \Rightarrow \text{bool}) \Rightarrow ('f \text{ list} \Rightarrow \text{bool}) \Rightarrow$
 $'f \text{ list} \Rightarrow \text{nat} \Rightarrow 'f \text{ list} \Rightarrow 'f \Rightarrow$
 $'f \text{ list list list} \Rightarrow 'f \text{ list list} \Rightarrow$
 $(\text{nat list} \times 'f \text{ list}) \text{ set}$

where

*generic-fri-sampled-query-bad-pair-set low-degree candidate-bad
candidate-table degree-bound roots final-value round-layers layers =*
 $\{(query-idxs, challenges).$
*generic-fri-sampled-query-candidate-evidence low-degree candidate-bad
candidate-table degree-bound roots challenges final-value query-idxs
round-layers layers}*

lemma *generic-fri-sampled-query-bad-pair-setI:*

assumes

*generic-fri-sampled-query-candidate-evidence low-degree candidate-bad
candidate-table degree-bound roots challenges final-value query-idxs
round-layers layers*

shows

$(query-idxs, challenges) \in$
*generic-fri-sampled-query-bad-pair-set low-degree candidate-bad
candidate-table degree-bound roots final-value round-layers layers*

using *assms* **unfolding** *generic-fri-sampled-query-bad-pair-set-def* **by** *simp*

lemma *generic-fri-sampled-query-bad-pair-setD:*

assumes

$(query-idxs, challenges) \in$
*generic-fri-sampled-query-bad-pair-set low-degree candidate-bad
candidate-table degree-bound roots final-value round-layers layers*

shows

*generic-fri-sampled-query-candidate-evidence low-degree candidate-bad
candidate-table degree-bound roots challenges final-value query-idxs
round-layers layers*

using *assms* **unfolding** *generic-fri-sampled-query-bad-pair-set-def* **by** *simp*

lemma *generic-fri-sampled-query-bad-pair-set-canonical-evidence:*

assumes *pair:*

$(query-idxs, challenges) \in$

generic-fri-sampled-query-bad-pair-set low-degree candidate-bad
candidate-table degree-bound roots final-value round-layers layers
shows
generic-fri-canonical-sampled-layer-chain-evidence low-degree
candidate-table degree-bound roots challenges final-value query-idxs
round-layers layers
proof —
have *evidence*:
generic-fri-sampled-query-candidate-evidence low-degree candidate-bad
candidate-table degree-bound roots challenges final-value query-idxs
round-layers layers
by (rule *generic-fri-sampled-query-bad-pair-setD*[*OF pair*])
show *?thesis*
by (rule *generic-fri-sampled-query-candidate-evidenceD*(1)[*OF evidence*])
qed

lemma *generic-fri-sampled-query-bad-pair-set-sampled-evidence*:
assumes *pair*:
(query-idxs, challenges) ∈
generic-fri-sampled-query-bad-pair-set low-degree candidate-bad
candidate-table degree-bound roots final-value round-layers layers
shows
generic-fri-sampled-layer-chain-evidence low-degree candidate-table
degree-bound roots challenges final-value query-idxs round-layers
(fri-canonical-domains (length challenges)) layers
proof —
have *canonical*:
generic-fri-canonical-sampled-layer-chain-evidence low-degree
candidate-table degree-bound roots challenges final-value query-idxs
round-layers layers
by (rule *generic-fri-sampled-query-bad-pair-set-canonical-evidence*
[*OF pair*])
show *?thesis*
by (rule *generic-fri-canonical-sampled-layer-chain-evidenceD*
[*OF canonical*])
qed

lemma *generic-fri-sampled-query-bad-pair-set-partial-evidence*:
assumes *pair*:
(query-idxs, challenges) ∈
generic-fri-sampled-query-bad-pair-set low-degree candidate-bad
candidate-table degree-bound roots final-value round-layers layers
shows
generic-fri-partial-evidence low-degree candidate-table degree-bound
roots challenges final-value query-idxs round-layers
proof —
have *sampled*:
generic-fri-sampled-layer-chain-evidence low-degree candidate-table
degree-bound roots challenges final-value query-idxs round-layers

(fri-canonical-domains (length challenges)) layers
 by (rule generic-fri-sampled-query-bad-pair-set-sampled-evidence
 [OF pair])
 show ?thesis
 by (rule generic-fri-sampled-layer-chain-evidenceD(1)[OF sampled])
 qed

lemma generic-fri-sampled-query-bad-pair-set-challenge-length:
 assumes pair:
 (query-idxs, challenges) ∈
 generic-fri-sampled-query-bad-pair-set low-degree candidate-bad
 candidate-table degree-bound roots final-value round-layers layers
 shows length challenges = fri-round-count-for-degree-bound degree-bound
proof –
 have evidence:
 generic-fri-sampled-query-candidate-evidence low-degree candidate-bad
 candidate-table degree-bound roots challenges final-value query-idxs
 round-layers layers
 by (rule generic-fri-sampled-query-bad-pair-setD[OF pair])
 have canonical:
 generic-fri-canonical-sampled-layer-chain-evidence low-degree
 candidate-table degree-bound roots challenges final-value query-idxs
 round-layers layers
 by (rule generic-fri-sampled-query-candidate-evidenceD(1)[OF evidence])
 have sampled:
 generic-fri-sampled-layer-chain-evidence low-degree candidate-table
 degree-bound roots challenges final-value query-idxs round-layers
 (fri-canonical-domains (length challenges)) layers
 by (rule generic-fri-canonical-sampled-layer-chain-evidenceD
 [OF canonical])
 have partial:
 generic-fri-partial-evidence low-degree candidate-table degree-bound
 roots challenges final-value query-idxs round-layers
 by (rule generic-fri-sampled-layer-chain-evidenceD(1)[OF sampled])
 show ?thesis
 using generic-fri-partial-evidence-shapes(1,2)[OF partial] by simp
 qed

lemma generic-fri-sampled-query-bad-pair-set-round-layers-length:
 assumes pair:
 (query-idxs, challenges) ∈
 generic-fri-sampled-query-bad-pair-set low-degree candidate-bad
 candidate-table degree-bound roots final-value round-layers layers
 shows length round-layers = length query-idxs
proof –
 have partial:
 generic-fri-partial-evidence low-degree candidate-table degree-bound
 roots challenges final-value query-idxs round-layers
 by (rule generic-fri-sampled-query-bad-pair-set-partial-evidence

[OF pair])
have *fri-all-round-layer-evidence* roots challenges query-idxs round-layers
 by (rule generic-fri-partial-evidence-shapes(3)[OF partial])
then show ?thesis
 unfolding *fri-all-round-layer-evidence-def* by simp
qed

lemma *generic-fri-sampled-query-bad-pair-set-sample*:

assumes *pair*:
 (query-idxs, challenges) ∈
 generic-fri-sampled-query-bad-pair-set low-degree candidate-bad
 candidate-table degree-bound roots final-value round-layers layers
and round-bound: round-idx < length query-idxs
and layer-bound: layer-idx < length challenges
shows
fri-evidence-next-idx roots query-idxs round-idx layer-idx <
 length (layers ! Suc layer-idx)
fri-sampled-table-fold
 (challenges ! layer-idx)
 (fri-evidence-layer-len roots layer-idx)
 (fri-evidence-layer-idx roots query-idxs round-idx layer-idx)
 (2 ^ layer-idx)
 (fri-canonical-domains (length challenges) ! layer-idx)
 (layers ! layer-idx)
 (round-layers ! round-idx ! layer-idx)
 (layers ! Suc layer-idx !
fri-evidence-next-idx roots query-idxs round-idx layer-idx)

proof —

have *sampled*:
 generic-fri-sampled-layer-chain-evidence low-degree candidate-table
 degree-bound roots challenges final-value query-idxs round-layers
 (fri-canonical-domains (length challenges)) layers
by (rule generic-fri-sampled-query-bad-pair-set-sampled-evidence
 [OF pair])
show *fri-evidence-next-idx* roots query-idxs round-idx layer-idx <
 length (layers ! Suc layer-idx)
by (rule generic-fri-sampled-layer-chain-evidence-sample(1)
 [OF sampled round-bound layer-bound])
show *fri-sampled-table-fold*
 (challenges ! layer-idx)
 (fri-evidence-layer-len roots layer-idx)
 (fri-evidence-layer-idx roots query-idxs round-idx layer-idx)
 (2 ^ layer-idx)
 (fri-canonical-domains (length challenges) ! layer-idx)
 (layers ! layer-idx)
 (round-layers ! round-idx ! layer-idx)
 (layers ! Suc layer-idx !
fri-evidence-next-idx roots query-idxs round-idx layer-idx)
by (rule generic-fri-sampled-layer-chain-evidence-sample(2))

[OF sampled round-bound layer-bound])
qed

lemma *generic-fri-sampled-query-bad-pair-set-candidate-bad*:
assumes *pair*:
 (*query-idxs*, *challenges*) ∈
 generic-fri-sampled-query-bad-pair-set *low-degree* *candidate-bad*
 candidate-table *degree-bound* *roots* *final-value* *round-layers* *layers*
shows *candidate-bad* *candidate-table*
proof –
have *evidence*:
 generic-fri-sampled-query-candidate-evidence *low-degree* *candidate-bad*
 candidate-table *degree-bound* *roots* *challenges* *final-value* *query-idxs*
 round-layers *layers*
by (rule *generic-fri-sampled-query-bad-pair-setD*[OF *pair*])
show ?thesis
by (rule *generic-fri-sampled-query-candidate-evidenceD*(2)[OF *evidence*])
qed

lemma *generic-fri-sampled-query-bad-pair-set-challenge-space*:
assumes
 (*query-idxs*, *challenges*) ∈
 generic-fri-sampled-query-bad-pair-set *low-degree* *candidate-bad*
 candidate-table *degree-bound* *roots* *final-value* *round-layers* *layers*
shows *challenges* ∈
 fri-challenge-space (*fri-round-count-for-degree-bound* *degree-bound*)
using *generic-fri-sampled-query-bad-pair-set-challenge-length*[OF *assms*]
unfolding *fri-challenge-space-def* **by** *simp*

lemma *generic-fri-sampled-query-bad-pair-set-subset-challenge-space*:
generic-fri-sampled-query-bad-pair-set *low-degree* *candidate-bad*
 candidate-table *degree-bound* *roots* *final-value* *round-layers* *layers*
 ⊆ (*UNIV* :: *nat list set*) ×
 fri-challenge-space (*fri-round-count-for-degree-bound* *degree-bound*)
by (auto intro: *generic-fri-sampled-query-bad-pair-set-challenge-space*)

definition *fri-query-challenge-pair-query-fiber*
 :: (*nat list* × '*f list*) *set* ⇒ '*f list* ⇒ *nat list set*
where
fri-query-challenge-pair-query-fiber *P* *challenges* =
 {*query-idxs*. (*query-idxs*, *challenges*) ∈ *P*}

lemma *finite-fri-query-index-list-space*:
finite *fri-query-index-list-space*
proof –
have *subset*:
fri-query-index-list-space ⊆
 {*query-idxs*. *set query-idxs* ⊆ *query-sample-space* ∧
 length query-idxs = *rounds*}

```

unfolding fri-query-index-list-space-def by auto
have finite-super:
  finite
  {query-idxs. set query-idxs  $\subseteq$  query-sample-space  $\wedge$ 
    length query-idxs = rounds}
  by (rule finite-lists-length-eq[OF finite-query-sample-space])
show ?thesis
  by (rule finite-subset[OF subset finite-super])
qed

lemma fri-query-challenge-pair-query-fiber-iff[simp]:
  query-idxs  $\in$  fri-query-challenge-pair-query-fiber  $P$  challenges  $\longleftrightarrow$ 
  (query-idxs, challenges)  $\in P$ 
unfolding fri-query-challenge-pair-query-fiber-def by simp

lemma fri-query-challenge-pair-query-projection-restricted:
  fst ' (P  $\cap$  (fri-query-index-list-space  $\times$  UNIV))
   $\subseteq$  fri-query-index-list-space
proof
  fix query-idxs
  assume query-idxs  $\in$ 
  fst ' (P  $\cap$  (fri-query-index-list-space  $\times$  UNIV))
  then show query-idxs  $\in$  fri-query-index-list-space
  by auto
qed

lemma finite-fri-query-challenge-pair-query-fiber-restricted:
  finite
  (fri-query-challenge-pair-query-fiber
    (P  $\cap$  (fri-query-index-list-space  $\times$  UNIV)) challenges)
proof –
  have
  fri-query-challenge-pair-query-fiber
  (P  $\cap$  (fri-query-index-list-space  $\times$  UNIV)) challenges
   $\subseteq$  fri-query-index-list-space
  unfolding fri-query-challenge-pair-query-fiber-def by blast
  then show ?thesis
  by (rule finite-subset) (rule finite-fri-query-index-list-space)
qed

definition trace-fri-query-challenge-pair-set-hit
  :: ('f, 'a) protocol-channel-scheme  $\Rightarrow$  (nat list  $\times$  'f list) set  $\Rightarrow$ 
  (unit list  $\times$  ('f, 'a) protocol-channel-scheme) option  $\Rightarrow$  bool
where
  trace-fri-query-challenge-pair-set-hit s P out  $\longleftrightarrow$ 
  ( $\exists$  trace-roots trace-bs trace-final dg composition-roots
    composition-bs composition-final fri-query-idxs trace-round-layers
    composition-round-layers.
    accepted-fri-opening-transcript s out trace-roots trace-bs

```

*trace-final dg composition-roots composition-bs composition-final
 fri-query-idxs trace-round-layers composition-round-layers* $\wedge$
(fri-query-idxs, trace-bs) $\in P$

definition *composition-fri-query-challenge-pair-set-hit*

$:: ('f, 'a) \text{ protocol-channel-scheme} \Rightarrow$
 $('f \Rightarrow (\text{nat list} \times 'f \text{ list}) \text{ set}) \Rightarrow$
 $(\text{unit list} \times ('f, 'a) \text{ protocol-channel-scheme}) \text{ option} \Rightarrow \text{bool}$

where

composition-fri-query-challenge-pair-set-hit s P out $\longleftrightarrow$
 $(\exists \text{ trace-roots trace-bs trace-final fri-dg composition-roots}$
 $\text{ composition-bs composition-final fri-query-idxs trace-round-layers}$
 $\text{ composition-round-layers.}$
 $\text{accepted-fri-opening-transcript s out trace-roots trace-bs}$
 $\text{ trace-final fri-dg composition-roots composition-bs}$
 $\text{ composition-final fri-query-idxs trace-round-layers}$
 $\text{ composition-round-layers} \wedge$
 $(\text{fri-query-idxs, composition-bs}) \in P \text{ fri-dg})$

lemma *accepted-fri-opening-transcript-query-index-list-space:*

assumes

*accepted-fri-opening-transcript s out trace-roots trace-bs trace-final
 dg composition-roots composition-bs composition-final fri-query-idxs
 trace-round-layers composition-round-layers*

shows *fri-query-idxs $\in$ fri-query-index-list-space*

proof –

have *len: length fri-query-idxs = rounds*

using *accepted-fri-opening-transcript-shapes[OF assms]* **by** *blast*

have *subset: set fri-query-idxs $\subseteq$ query-sample-space*

proof

fix *idx*

assume *idx $\in$ set fri-query-idxs*

then show *idx $\in$ query-sample-space*

by (*rule accepted-fri-opening-transcript-query-idx-in-sample-space*
 $[OF \text{ assms}]$)

qed

show *?thesis*

unfolding *fri-query-index-list-space-def*

using *len subset* **by** *simp*

qed

lemma *trace-fri-query-challenge-pair-set-hit-restrict-to-query-space:*

assumes *trace-fri-query-challenge-pair-set-hit s P out*

shows

trace-fri-query-challenge-pair-set-hit s
 $(P \cap (\text{fri-query-index-list-space} \times \text{UNIV})) \text{ out}$

proof –

from *assms* **obtain** *trace-roots trace-bs trace-final dg composition-roots
 composition-bs composition-final fri-query-idxs trace-round-layers*

composition-round-layers **where**
transcript:
accepted-fri-opening-transcript s out trace-roots trace-bs
trace-final dg composition-roots composition-bs composition-final
fri-query-idxs trace-round-layers composition-round-layers
and *pair*: (*fri-query-idxs*, *trace-bs*) $\in P$
unfolding *trace-fri-query-challenge-pair-set-hit-def* **by** *blast*
have *query-space*: *fri-query-idxs* $\in$ *fri-query-index-list-space*
by (*rule accepted-fri-opening-transcript-query-index-list-space*
 $[OF\ transcript]$)
show *?thesis*
unfolding *trace-fri-query-challenge-pair-set-hit-def*
using *transcript pair query-space* **by** *blast*
qed

lemma *composition-fri-query-challenge-pair-set-hit-restrict-to-query-space:*
assumes *composition-fri-query-challenge-pair-set-hit s P out*
shows
composition-fri-query-challenge-pair-set-hit s
 $(\lambda dg. P\ dg \cap (fri\text{-}query\text{-}index\text{-}list\text{-}space \times UNIV))\ out$
proof –
from *assms* **obtain** *trace-roots trace-bs trace-final fri-dg*
composition-roots composition-bs composition-final fri-query-idxs
trace-round-layers composition-round-layers **where**
transcript:
accepted-fri-opening-transcript s out trace-roots trace-bs
trace-final fri-dg composition-roots composition-bs
composition-final fri-query-idxs trace-round-layers
composition-round-layers
and *pair*: (*fri-query-idxs*, *composition-bs*) $\in P\ fri\text{-}dg$
unfolding *composition-fri-query-challenge-pair-set-hit-def* **by** *blast*
have *query-space*: *fri-query-idxs* $\in$ *fri-query-index-list-space*
by (*rule accepted-fri-opening-transcript-query-index-list-space*
 $[OF\ transcript]$)
show *?thesis*
unfolding *composition-fri-query-challenge-pair-set-hit-def*
using *transcript pair query-space* **by** *blast*
qed

lemma *trace-fri-query-challenge-pair-set-hit-imp-query-index-list-set-hit:*
assumes *hit: trace-fri-query-challenge-pair-set-hit s P out*
and *projection: fst ‘ P $\subseteq$ Q*
shows *trace-fri-query-index-list-set-hit s Q out*
proof –
from *hit* **obtain** *trace-roots trace-bs trace-final dg composition-roots*
composition-bs composition-final fri-query-idxs trace-round-layers
composition-round-layers
where *openings:*
accepted-fri-opening-transcript s out trace-roots trace-bs

trace-final dg composition-roots composition-bs composition-final
fri-query-idxs trace-round-layers composition-round-layers
and *pair*: (*fri-query-idxs*, *trace-bs*) $\in P$
unfolding *trace-fri-query-challenge-pair-set-hit-def* **by** *blast*
have *fri-query-idxs* $\in Q$
using *pair projection* **by** *force*
then show *?thesis*
unfolding *trace-fri-query-index-list-set-hit-def*
using *openings* **by** *blast*
qed

lemma *composition-fri-query-challenge-pair-set-hit-imp-query-index-list-set-hit*:
assumes *hit*: *composition-fri-query-challenge-pair-set-hit* *s P out*
and *projection*: $\bigwedge dg. \text{fst } ' P \ dg \subseteq Q$
shows *composition-fri-query-index-list-set-hit* *s Q out*
proof –
from *hit* **obtain** *trace-roots trace-bs trace-final fri-dg composition-roots*
composition-bs composition-final fri-query-idxs trace-round-layers
composition-round-layers
where *openings*:
accepted-fri-opening-transcript s out trace-roots trace-bs
trace-final fri-dg composition-roots composition-bs
composition-final fri-query-idxs trace-round-layers
composition-round-layers
and *pair*: (*fri-query-idxs*, *composition-bs*) $\in P$ *fri-dg*
unfolding *composition-fri-query-challenge-pair-set-hit-def* **by** *blast*
have *fri-query-idxs* $\in Q$
using *pair projection[of fri-dg]* **by** *force*
then show *?thesis*
unfolding *composition-fri-query-index-list-set-hit-def*
using *openings* **by** *blast*
qed

lemma *trace-fri-query-challenge-pair-set-hit-imp-challenge-list-set-hit*:
assumes *hit*: *trace-fri-query-challenge-pair-set-hit* *s P out*
and *projection*: *snd* ' $P \subseteq B$
shows *trace-fri-challenge-list-set-hit* *s B out*
proof –
from *hit* **obtain** *trace-roots trace-bs trace-final dg composition-roots*
composition-bs composition-final fri-query-idxs trace-round-layers
composition-round-layers
where *openings*:
accepted-fri-opening-transcript s out trace-roots trace-bs
trace-final dg composition-roots composition-bs composition-final
fri-query-idxs trace-round-layers composition-round-layers
and *pair*: (*fri-query-idxs*, *trace-bs*) $\in P$
unfolding *trace-fri-query-challenge-pair-set-hit-def* **by** *blast*
have *challenges*:
accepted-fri-challenges s out trace-bs dg composition-bs

```

    by (rule accepted-fri-opening-transcript-challenges[OF openings])
  have trace-bs ∈ B
    using pair projection by force
  then show ?thesis
    unfolding trace-fri-challenge-list-set-hit-def
    using challenges by blast
qed

lemma composition-fri-query-challenge-pair-set-hit-imp-challenge-list-set-hit:
  assumes hit: composition-fri-query-challenge-pair-set-hit s P out
    and projection:  $\bigwedge dg. \text{snd } 'P \ dg \subseteq B \ dg$ 
  shows composition-fri-challenge-list-set-hit s B out
proof -
  from hit obtain trace-roots trace-bs trace-final fri-dg composition-roots
    composition-bs composition-final fri-query-idxs trace-round-layers
    composition-round-layers
  where openings:
    accepted-fri-opening-transcript s out trace-roots trace-bs
    trace-final fri-dg composition-roots composition-bs
    composition-final fri-query-idxs trace-round-layers
    composition-round-layers
  and pair: (fri-query-idxs, composition-bs) ∈ P fri-dg
  unfolding composition-fri-query-challenge-pair-set-hit-def by blast
  have challenges:
    accepted-fri-challenges s out trace-bs fri-dg composition-bs
  by (rule accepted-fri-opening-transcript-challenges[OF openings])
  have composition-bs ∈ B fri-dg
    using pair projection[of fri-dg] by force
  then show ?thesis
    unfolding composition-fri-challenge-list-set-hit-def
    using challenges by blast
qed

definition trace-fri-sampled-query-pair-set-hit
  :: ('f list ⇒ 'f list ⇒ 'f ⇒ 'f list list list ⇒
    'f list list ⇒ (nat list × 'f list) set) ⇒
    ('f, 'a) protocol-channel-scheme ⇒
    (unit list × ('f, 'a) protocol-channel-scheme) option ⇒ bool
where
  trace-fri-sampled-query-pair-set-hit R s out  $\longleftrightarrow$ 
    (∃ trace-roots trace-bs trace-final dg composition-roots
      composition-bs composition-final fri-query-idxs trace-round-layers
      composition-round-layers fr candidate-query-idxs trace-openings
      trace-table layers.
      trace-fri-partial-candidate-opening-evidence s out trace-roots
      trace-bs trace-final dg composition-roots composition-bs
      composition-final fri-query-idxs trace-round-layers
      composition-round-layers fr candidate-query-idxs trace-openings
      trace-table ∧

```

$(\text{fri-query-idxs}, \text{trace-bs}) \in$
 $R \text{ trace-table trace-roots trace-final trace-round-layers layers}$

definition *composition-fri-sampled-query-pair-set-hit*

$:: ('f \Rightarrow 'f \text{ list} \Rightarrow 'f \text{ list} \Rightarrow 'f \Rightarrow$
 $'f \text{ list list list} \Rightarrow 'f \text{ list list} \Rightarrow$
 $(\text{nat list} \times 'f \text{ list}) \text{ set}) \Rightarrow$
 $('f, 'a) \text{ protocol-channel-scheme} \Rightarrow$
 $(\text{unit list} \times ('f, 'a) \text{ protocol-channel-scheme}) \text{ option} \Rightarrow \text{bool}$

where

$\text{composition-fri-sampled-query-pair-set-hit } R \text{ s out} \longleftrightarrow$
 $(\exists \text{ trace-roots trace-bs trace-final fri-dg}$
 $\text{opening-composition-roots composition-bs composition-final}$
 $\text{fri-query-idxs trace-round-layers composition-round-layers fr}$
 $\text{f-fri-roots f-final as dg composition-fri-roots final}$
 $\text{trace-query-idxs trace-openings composition-query-idxs}$
 $\text{composition-openings trace-table composition-table layers.}$
 $\text{composition-fri-partial-candidate-opening-evidence s out trace-roots}$
 $\text{trace-bs trace-final fri-dg opening-composition-roots composition-bs}$
 $\text{composition-final fri-query-idxs trace-round-layers}$
 $\text{composition-round-layers fr f-fri-roots f-final as dg}$
 $\text{composition-fri-roots final trace-query-idxs trace-openings}$
 $\text{composition-query-idxs composition-openings trace-table}$
 $\text{composition-table} \wedge$
 $(\text{fri-query-idxs}, \text{composition-bs}) \in$
 $R \text{ fri-dg composition-table opening-composition-roots composition-final}$
 $\text{composition-round-layers layers})$

lemma *trace-fri-sampled-query-pair-set-hit-imp-challenge-list-set-hit:*

assumes *hit:* $\text{trace-fri-sampled-query-pair-set-hit } R \text{ s out}$

and *projection:*

$\bigwedge \text{trace-table roots final round-layers layers.}$
 snd '

$R \text{ trace-table roots final round-layers layers} \subseteq B$

shows $\text{trace-fri-challenge-list-set-hit } s B \text{ out}$

proof —

from *hit* **obtain** $\text{trace-roots trace-bs trace-final dg composition-roots}$
 $\text{composition-bs composition-final fri-query-idxs trace-round-layers}$
 $\text{composition-round-layers fr candidate-query-idxs trace-openings}$
 $\text{trace-table layers}$

where *evidence:*

$\text{trace-fri-partial-candidate-opening-evidence s out trace-roots}$
 $\text{trace-bs trace-final dg composition-roots composition-bs}$
 $\text{composition-final fri-query-idxs trace-round-layers}$
 $\text{composition-round-layers fr candidate-query-idxs trace-openings}$
 trace-table

and *pair:*

$(\text{fri-query-idxs}, \text{trace-bs}) \in$
 $R \text{ trace-table trace-roots trace-final trace-round-layers layers}$

unfolding *trace-fri-sampled-query-pair-set-hit-def* **by** *blast*
have *challenges*:
accepted-fri-challenges s out trace-bs dg composition-bs
by (rule *accepted-fri-opening-transcript-challenges*)
(rule *trace-fri-partial-candidate-opening-evidenceD(1)[OF evidence]*)
have *trace-bs* $\in B$
using *pair projection[of trace-table trace-roots trace-final*
trace-round-layers layers] **by** *force*
then show *?thesis*
unfolding *trace-fri-challenge-list-set-hit-def*
using *challenges* **by** *blast*
qed

lemma *composition-fri-sampled-query-pair-set-hit-imp-challenge-list-set-hit*:
assumes *hit: composition-fri-sampled-query-pair-set-hit R s out*
and *projection*:
 $\bigwedge \text{fri-dg composition-table roots final round-layers layers.}$
snd '
R fri-dg composition-table roots final round-layers layers
 $\subseteq B \text{ fri-dg}$
shows *composition-fri-challenge-list-set-hit s B out*
proof –
from *hit* **obtain** *trace-roots trace-bs trace-final fri-dg*
opening-composition-roots composition-bs composition-final
fri-query-idxs trace-round-layers composition-round-layers fr
f-fri-roots f-final as dg composition-fri-roots final
trace-query-idxs trace-openings composition-query-idxs
composition-openings trace-table composition-table layers
where *evidence*:
composition-fri-partial-candidate-opening-evidence s out trace-roots
trace-bs trace-final fri-dg opening-composition-roots composition-bs
composition-final fri-query-idxs trace-round-layers
composition-round-layers fr f-fri-roots f-final as dg
composition-fri-roots final trace-query-idxs trace-openings
composition-query-idxs composition-openings trace-table
composition-table
and *pair*:
(*fri-query-idxs, composition-bs*) $\in$
R fri-dg composition-table opening-composition-roots
composition-final composition-round-layers layers
unfolding *composition-fri-sampled-query-pair-set-hit-def* **by** *auto*
have *challenges*:
accepted-fri-challenges s out trace-bs fri-dg composition-bs
by (rule *accepted-fri-opening-transcript-challenges*)
(rule *composition-fri-partial-candidate-opening-evidenceD(1)*
 $[OF \text{ evidence}]$)
have *composition-bs* $\in B \text{ fri-dg}$
using *pair projection[of fri-dg composition-table*
opening-composition-roots composition-final composition-round-layers

$layers]$ **by** *force*
then show *?thesis*
unfolding *composition-fri-challenge-list-set-hit-def*
using *challenges* **by** *blast*
qed

lemma *trace-fri-sampled-query-pair-set-hit-imp-query-index-list-set-hit:*
assumes *hit: trace-fri-sampled-query-pair-set-hit R s out*
and *projection:*
 $\bigwedge_{fst} trace-table\ roots\ final\ round-layers\ layers.$
 $R\ trace-table\ roots\ final\ round-layers\ layers \subseteq Q$
shows *trace-fri-query-index-list-set-hit s Q out*
proof –
from *hit* **obtain** *trace-roots trace-bs trace-final dg composition-roots*
composition-bs composition-final fri-query-idxs trace-round-layers
composition-round-layers fr candidate-query-idxs trace-openings
trace-table layers
where *evidence:*
trace-fri-partial-candidate-opening-evidence s out trace-roots
trace-bs trace-final dg composition-roots composition-bs
composition-final fri-query-idxs trace-round-layers
composition-round-layers fr candidate-query-idxs trace-openings
trace-table
and *pair:*
 $(fri-query-idxs, trace-bs) \in$
 $R\ trace-table\ trace-roots\ trace-final\ trace-round-layers\ layers$
unfolding *trace-fri-sampled-query-pair-set-hit-def* **by** *blast*
have *openings:*
accepted-fri-opening-transcript s out trace-roots trace-bs
trace-final dg composition-roots composition-bs composition-final
fri-query-idxs trace-round-layers composition-round-layers
by $(rule\ trace-fri-partial-candidate-opening-evidenceD(1)[OF\ evidence])$
have *fri-query-idxs $\in Q$*
using *pair projection[of trace-table trace-roots trace-final*
trace-round-layers layers] **by** *force*
then show *?thesis*
unfolding *trace-fri-query-index-list-set-hit-def*
using *openings* **by** *blast*
qed

lemma *composition-fri-sampled-query-pair-set-hit-imp-query-index-list-set-hit:*
assumes *hit: composition-fri-sampled-query-pair-set-hit R s out*
and *projection:*
 $\bigwedge_{fst} fri-dg\ composition-table\ roots\ final\ round-layers\ layers.$
 $R\ fri-dg\ composition-table\ roots\ final\ round-layers\ layers$
 $\subseteq Q$
shows *composition-fri-query-index-list-set-hit s Q out*

proof –

from *hit* **obtain** *trace-roots trace-bs trace-final fri-dg*
opening-composition-roots composition-bs composition-final
fri-query-idxs trace-round-layers composition-round-layers fr
f-fri-roots f-final as declared-dg composition-fri-roots final-value
trace-query-idxs trace-openings composition-query-idxs
composition-openings trace-table composition-table layers
where *evidence*:
composition-fri-partial-candidate-opening-evidence s out trace-roots
trace-bs trace-final fri-dg opening-composition-roots composition-bs
composition-final fri-query-idxs trace-round-layers
composition-round-layers fr f-fri-roots f-final as declared-dg
composition-fri-roots final-value trace-query-idxs trace-openings
composition-query-idxs composition-openings trace-table
composition-table
and *pair*:
(fri-query-idxs, composition-bs) ∈
R fri-dg composition-table opening-composition-roots
composition-final composition-round-layers layers
unfolding *composition-fri-sampled-query-pair-set-hit-def* **by** *auto*
have *openings*:
accepted-fri-opening-transcript s out trace-roots trace-bs
trace-final fri-dg opening-composition-roots composition-bs
composition-final fri-query-idxs trace-round-layers
composition-round-layers
by *(rule composition-fri-partial-candidate-opening-evidenceD(1)*
[OF evidence])
have *fri-query-idxs ∈ Q*
using *pair projection[of fri-dg composition-table*
opening-composition-roots composition-final composition-round-layers
layers] **by** *force*
then show *?thesis*
unfolding *composition-fri-query-index-list-set-hit-def*
using *openings* **by** *blast*
qed

lemma *trace-fri-sampled-query-pair-set-hit-imp-query-challenge-pair-set-hit*:

assumes *hit: trace-fri-sampled-query-pair-set-hit R s out*

and *projection*:

$\bigwedge \text{trace-table roots final round-layers layers.}$

$R \text{ trace-table roots final round-layers layers} \subseteq P$

shows *trace-fri-query-challenge-pair-set-hit s P out*

proof –

from *hit* **obtain** *trace-roots trace-bs trace-final dg composition-roots*
composition-bs composition-final fri-query-idxs trace-round-layers
composition-round-layers fr candidate-query-idxs trace-openings
trace-table layers
where *evidence*:
trace-fri-partial-candidate-opening-evidence s out trace-roots

trace-bs trace-final dg composition-roots composition-bs
composition-final fri-query-idxs trace-round-layers
composition-round-layers fr candidate-query-idxs trace-openings
trace-table
and *pair*:
(fri-query-idxs, trace-bs) ∈
R trace-table trace-roots trace-final trace-round-layers layers
unfolding *trace-fri-sampled-query-pair-set-hit-def* **by** *blast*
have *openings*:
accepted-fri-opening-transcript s out trace-roots trace-bs
trace-final dg composition-roots composition-bs composition-final
fri-query-idxs trace-round-layers composition-round-layers
by *(rule trace-fri-partial-candidate-opening-evidenceD(1)[OF evidence])*
have *(fri-query-idxs, trace-bs) ∈ P*
using *pair projection[of trace-table trace-roots trace-final*
trace-round-layers layers] **by** *blast*
then show *?thesis*
unfolding *trace-fri-query-challenge-pair-set-hit-def*
using *openings* **by** *blast*
qed

lemma *composition-fri-sampled-query-pair-set-hit-imp-query-challenge-pair-set-hit*:
assumes *hit: composition-fri-sampled-query-pair-set-hit R s out*
and *projection*:
 $\bigwedge$ *fri-dg composition-table roots final round-layers layers.*
R fri-dg composition-table roots final round-layers layers
 $\subseteq P$ *fri-dg*
shows *composition-fri-query-challenge-pair-set-hit s P out*
proof –
from *hit* **obtain** *trace-roots trace-bs trace-final fri-dg*
opening-composition-roots composition-bs composition-final
fri-query-idxs trace-round-layers composition-round-layers fr
f-fri-roots f-final as declared-dg composition-fri-roots final-value
trace-query-idxs trace-openings composition-query-idxs
composition-openings trace-table composition-table layers
where *evidence*:
composition-fri-partial-candidate-opening-evidence s out trace-roots
trace-bs trace-final fri-dg opening-composition-roots composition-bs
composition-final fri-query-idxs trace-round-layers
composition-round-layers fr f-fri-roots f-final as declared-dg
composition-fri-roots final-value trace-query-idxs trace-openings
composition-query-idxs composition-openings trace-table
composition-table
and *pair*:
(fri-query-idxs, composition-bs) ∈
R fri-dg composition-table opening-composition-roots
composition-final composition-round-layers layers
unfolding *composition-fri-sampled-query-pair-set-hit-def* **by** *auto*
have *openings*:

accepted-fri-opening-transcript s out trace-roots trace-bs
trace-final fri-dg opening-composition-roots composition-bs
composition-final fri-query-idxs trace-round-layers
composition-round-layers
by (rule *composition-fri-partial-candidate-opening-evidenceD(1)*
[*OF evidence*])
have (*fri-query-idxs, composition-bs*) $\in P$ *fri-dg*
using pair projection[*of fri-dg composition-table*
opening-composition-roots composition-final composition-round-layers
layers] **by** blast
then show ?thesis
unfolding *composition-fri-query-challenge-pair-set-hit-def*
using *openings* **by** blast
qed

lemma *trace-fri-sampled-query-pair-set-hit-imp-query-index-list-space-hit:*
assumes *hit: trace-fri-sampled-query-pair-set-hit R s out*
shows

trace-fri-query-index-list-set-hit s fri-query-index-list-space out
proof –
from *hit* **obtain** *trace-roots trace-bs trace-final dg composition-roots*
composition-bs composition-final fri-query-idxs trace-round-layers
composition-round-layers fr candidate-query-idxs trace-openings
trace-table layers
where *evidence:*
trace-fri-partial-candidate-opening-evidence s out trace-roots
trace-bs trace-final dg composition-roots composition-bs
composition-final fri-query-idxs trace-round-layers
composition-round-layers fr candidate-query-idxs trace-openings
trace-table
unfolding *trace-fri-sampled-query-pair-set-hit-def* **by** blast
have *openings:*
accepted-fri-opening-transcript s out trace-roots trace-bs
trace-final dg composition-roots composition-bs composition-final
fri-query-idxs trace-round-layers composition-round-layers
by (rule *trace-fri-partial-candidate-opening-evidenceD(1)*[*OF evidence*])
have *fri-query-idxs* $\in$ *fri-query-index-list-space*
by (rule *accepted-fri-opening-transcript-query-index-list-space*
[*OF openings*])
then show ?thesis
unfolding *trace-fri-query-index-list-set-hit-def*
using *openings* **by** blast
qed

lemma *composition-fri-sampled-query-pair-set-hit-imp-query-index-list-space-hit:*
assumes *hit: composition-fri-sampled-query-pair-set-hit R s out*
shows
composition-fri-query-index-list-set-hit s fri-query-index-list-space out
proof –

from *hit* **obtain** *trace-roots trace-bs trace-final fri-dg*
opening-composition-roots composition-bs composition-final
fri-query-idxs trace-round-layers composition-round-layers fr
f-fri-roots f-final as declared-dg composition-fri-roots final-value
trace-query-idxs trace-openings composition-query-idxs
composition-openings trace-table composition-table layers
where *evidence*:
composition-fri-partial-candidate-opening-evidence s out trace-roots
trace-bs trace-final fri-dg opening-composition-roots composition-bs
composition-final fri-query-idxs trace-round-layers
composition-round-layers fr f-fri-roots f-final as declared-dg
composition-fri-roots final-value trace-query-idxs trace-openings
composition-query-idxs composition-openings trace-table
composition-table
unfolding *composition-fri-sampled-query-pair-set-hit-def* **by** *auto*
have *openings*:
accepted-fri-opening-transcript s out trace-roots trace-bs
trace-final fri-dg opening-composition-roots composition-bs
composition-final fri-query-idxs trace-round-layers
composition-round-layers
by (rule *composition-fri-partial-candidate-opening-evidenceD(1)*
[*OF evidence*])
have *fri-query-idxs* $\in$ *fri-query-index-list-space*
by (rule *accepted-fri-opening-transcript-query-index-list-space*
[*OF openings*])
then show *?thesis*
unfolding *composition-fri-query-index-list-set-hit-def*
using *openings* **by** *blast*
qed

lemma *trace-fri-sampled-query-pair-set-hit-query-indices*:
assumes *hit: trace-fri-sampled-query-pair-set-hit R s out*
obtains *query-idxs challenges candidate-table roots final round-layers layers*
where
length query-idxs = rounds
 $\forall idx \in \text{set } query-idxs. idx \in query-sample-space$
 $\forall idx \in \text{set } query-idxs. idx < clength * scale$
 $(query-idxs, challenges) \in$
R candidate-table roots final round-layers layers
proof –
from *hit* **obtain** *trace-roots trace-bs trace-final dg composition-roots*
composition-bs composition-final fri-query-idxs trace-round-layers
composition-round-layers fr candidate-query-idxs trace-openings
candidate-table candidate-layers
where *evidence*:
trace-fri-partial-candidate-opening-evidence s out trace-roots
trace-bs trace-final dg composition-roots composition-bs
composition-final fri-query-idxs trace-round-layers
composition-round-layers fr candidate-query-idxs trace-openings

candidate-table
and *pair*:
 (*fri-query-idxs*, *trace-bs*) ∈
R candidate-table trace-roots trace-final trace-round-layers
candidate-layers
unfolding *trace-fri-sampled-query-pair-set-hit-def* **by** *blast*
have *fri-openings*:
accepted-fri-opening-transcript s out trace-roots trace-bs
trace-final dg composition-roots composition-bs composition-final
fri-query-idxs trace-round-layers composition-round-layers
by (*rule trace-fri-partial-candidate-opening-evidenceD(1)[OF evidence]*)
have *len*: *length fri-query-idxs = rounds*
using *accepted-fri-opening-transcript-shapes[OF fri-openings]* **by** *blast*
have *sample-space*: $\forall \text{idx} \in \text{set } \text{fri-query-idxs}. \text{idx} \in \text{query-sample-space}$
proof
fix *idx*
assume *idx* ∈ *set fri-query-idxs*
then show *idx* ∈ *query-sample-space*
by (*rule accepted-fri-opening-transcript-query-idx-in-sample-space*
[OF fri-openings])
qed
have *bounds*: $\forall \text{idx} \in \text{set } \text{fri-query-idxs}. \text{idx} < \text{clength} * \text{scale}$
using *fri-openings* **unfolding** *accepted-fri-opening-transcript-def*
by *blast*
show *?thesis*
by (*rule that[OF len sample-space bounds pair]*)
qed

lemma *composition-fri-sampled-query-pair-set-hit-query-indices*:
assumes *hit*: *composition-fri-sampled-query-pair-set-hit R s out*
obtains *query-idxs challenges dg candidate-table roots final round-layers*
layers
where
length query-idxs = rounds
 $\forall \text{idx} \in \text{set } \text{query-idxs}. \text{idx} \in \text{query-sample-space}$
 $\forall \text{idx} \in \text{set } \text{query-idxs}. \text{idx} < \text{clength} * \text{scale}$
(query-idxs, challenges) ∈
R dg candidate-table roots final round-layers layers
proof –

from *hit* **obtain** *trace-roots trace-bs trace-final fri-dg*
opening-composition-roots composition-bs composition-final
fri-query-idxs trace-round-layers composition-round-layers fr
f-fri-roots f-final as declared-dg composition-fri-roots final-value
trace-query-idxs trace-openings composition-query-idxs
composition-openings trace-table candidate-table candidate-layers
where *evidence*:
composition-fri-partial-candidate-opening-evidence s out trace-roots
trace-bs trace-final fri-dg opening-composition-roots composition-bs
composition-final fri-query-idxs trace-round-layers

```

    composition-round-layers fr f-fri-roots f-final as declared-dg
    composition-fri-roots final-value trace-query-idxs trace-openings
    composition-query-idxs composition-openings trace-table
    candidate-table
  and pair:
    (fri-query-idxs, composition-bs) ∈
      R fri-dg candidate-table opening-composition-roots
      composition-final composition-round-layers candidate-layers
  unfolding composition-fri-sampled-query-pair-set-hit-def by auto
have fri-openings:
  accepted-fri-opening-transcript s out trace-roots trace-bs
  trace-final fri-dg opening-composition-roots composition-bs
  composition-final fri-query-idxs trace-round-layers
  composition-round-layers
  by (rule composition-fri-partial-candidate-opening-evidenceD(1)
    [OF evidence])
have len: length fri-query-idxs = rounds
  using accepted-fri-opening-transcript-shapes[OF fri-openings] by blast
have sample-space: ∀ idx ∈ set fri-query-idxs. idx ∈ query-sample-space
proof
  fix idx
  assume idx ∈ set fri-query-idxs
  then show idx ∈ query-sample-space
    by (rule accepted-fri-opening-transcript-query-idx-in-sample-space
      [OF fri-openings])
qed
have bounds: ∀ idx ∈ set fri-query-idxs. idx < clength * scale
  using fri-openings unfolding accepted-fri-opening-transcript-def
  by blast
show ?thesis
  by (rule that[OF len sample-space bounds pair])
qed

lemma wp-trace-fri-sampled-query-pair-set-hit-bound-from-projection:
  assumes future: trace-fri-future-fresh s
  and projection:
    ∧ trace-table roots final round-layers layers.
    snd '
      R trace-table roots final round-layers layers ⊆ B
  and subset: B ⊆ fri-challenge-space (ceil-log clength)
  and bound:
    nnreal (card B) / nnreal (CARD('f) ^ ceil-log clength) ≤ C
  shows
    wp-event verify-monad
      (trace-fri-sampled-query-pair-set-hit R s) s ≤ C
proof -
  have wp-event verify-monad
    (trace-fri-sampled-query-pair-set-hit R s) s ≤
      wp-event verify-monad (trace-fri-challenge-list-set-hit s B) s

```



```

    by (rule wp-event-mono)
      (rule trace-fri-sampled-query-pair-set-hit-imp-challenge-list-set-hit
        [OF - projection])
  also have ... ≤ nnreal (card B) /
    nnreal (CARD('f) ^ ceil-log clength)
  by (rule wp-verify-monad-trace-fri-challenge-list-set-bound
    [OF future subset])
  also have ... ≤ C
  by (rule bound)
  finally show ?thesis .
qed

lemma wp-composition-fri-sampled-query-pair-set-hit-bound-from-projection:
  fixes C :: prob
  assumes future: composition-fri-future-fresh s
  and projection:
     $\bigwedge \text{fri-dg } \text{composition-table roots final round-layers layers.}$ 
    snd '
     $R \text{ fri-dg composition-table roots final round-layers layers}$ 
     $\subseteq B \text{ fri-dg}$ 
  and subset:
     $\bigwedge \text{dg. } B \text{ dg} \subseteq \text{fri-challenge-space (ceil-log (to-nat dg + 1))}$ 
  and bound:
     $\bigwedge \text{dg. nnreal (card (B dg)) /}$ 
     $\text{nnreal (CARD('f) ^ ceil-log (to-nat dg + 1))} \leq C$ 
  shows
    wp-event verify-monad
      (composition-fri-sampled-query-pair-set-hit R s) s ≤ C
proof -
  have wp-event verify-monad
    (composition-fri-sampled-query-pair-set-hit R s) s ≤
    wp-event verify-monad (composition-fri-challenge-list-set-hit s B) s
  by (rule wp-event-mono)
    (rule
      composition-fri-sampled-query-pair-set-hit-imp-challenge-list-set-hit
      [OF - projection])
  also have ... ≤ C
  by (rule wp-verify-monad-composition-fri-challenge-list-set-bound
    [OF future subset bound])
  finally show ?thesis .
qed

lemma wp-trace-fri-query-challenge-pair-set-hit-bound-from-challenge-projection:
  assumes future: trace-fri-future-fresh s
  and projection: snd ' P ⊆ B
  and subset: B ⊆ fri-challenge-space (ceil-log clength)
  and bound:
    nnreal (card B) / nnreal (CARD('f) ^ ceil-log clength) ≤ C
  shows

```

```

    wp-event verify-monad
      (trace-fri-query-challenge-pair-set-hit s P) s ≤ C
proof –
  have wp-event verify-monad
    (trace-fri-query-challenge-pair-set-hit s P) s ≤
    wp-event verify-monad (trace-fri-challenge-list-set-hit s B) s
  by (rule wp-event-mono)
    (rule trace-fri-query-challenge-pair-set-hit-imp-challenge-list-set-hit
      [OF - projection])
  also have ... ≤ nnreal (card B) /
    nnreal (CARD('f) ^ ceil-log clength)
  by (rule wp-verify-monad-trace-fri-challenge-list-set-bound
    [OF future subset])
  also have ... ≤ C
  by (rule bound)
  finally show ?thesis .
qed

lemma wp-composition-fri-query-challenge-pair-set-hit-bound-from-challenge-projection:
  fixes C :: prob
  assumes future: composition-fri-future-fresh s
  and projection:  $\bigwedge dg. \text{snd } 'P \ dg \subseteq B \ dg$ 
  and subset:
     $\bigwedge dg. B \ dg \subseteq \text{fri-challenge-space } (\text{ceil-log } (\text{to-nat } dg + 1))$ 
  and bound:
     $\bigwedge dg. \text{nnreal } (\text{card } (B \ dg)) /$ 
     $\text{nnreal } (\text{CARD}('f) ^ \text{ceil-log } (\text{to-nat } dg + 1)) \leq C$ 
  shows
    wp-event verify-monad
      (composition-fri-query-challenge-pair-set-hit s P) s ≤ C
proof –
  have wp-event verify-monad
    (composition-fri-query-challenge-pair-set-hit s P) s ≤
    wp-event verify-monad (composition-fri-challenge-list-set-hit s B) s
  by (rule wp-event-mono)
    (rule
      composition-fri-query-challenge-pair-set-hit-imp-challenge-list-set-hit
      [OF - projection])
  also have ... ≤ C
  by (rule wp-verify-monad-composition-fri-challenge-list-set-bound
    [OF future subset bound])
  finally show ?thesis .
qed

definition trace-fri-sampled-query-bad-candidate
  :: ('f, 'a) protocol-channel-scheme  $\Rightarrow$ 
    (unit list  $\times$  ('f, 'a) protocol-channel-scheme) option  $\Rightarrow$  bool
where
  trace-fri-sampled-query-bad-candidate s out  $\longleftrightarrow$ 

```

$(\exists \text{ trace-roots trace-bs trace-final dg composition-roots}$
 $\text{composition-bs composition-final fri-query-idxs trace-round-layers}$
 $\text{composition-round-layers fr candidate-query-idxs trace-openings}$
 $\text{trace-table layers.}$
 $\text{trace-fri-partial-candidate-opening-evidence s out trace-roots}$
 $\text{trace-bs trace-final dg composition-roots composition-bs}$
 $\text{composition-final fri-query-idxs trace-round-layers}$
 $\text{composition-round-layers fr candidate-query-idxs trace-openings}$
 $\text{trace-table} \wedge$
 $\text{generic-fri-sampled-query-candidate-evidence trace-table-low-degree}$
 $(\text{Not} \circ \text{trace-table-low-degree}) \text{ trace-table (clength} - 1)$
 $\text{trace-roots trace-bs trace-final fri-query-idxs trace-round-layers}$
 $\text{layers})$

definition *composition-fri-sampled-query-bad-candidate*

$:: ('f, 'a) \text{ protocol-channel-scheme} \Rightarrow$
 $(\text{unit list} \times ('f, 'a) \text{ protocol-channel-scheme}) \text{ option} \Rightarrow \text{bool}$

where

$\text{composition-fri-sampled-query-bad-candidate s out} \longleftrightarrow$
 $(\exists \text{ trace-roots trace-bs trace-final fri-dg}$
 $\text{opening-composition-roots composition-bs composition-final}$
 $\text{fri-query-idxs trace-round-layers composition-round-layers fr}$
 $\text{f-fri-roots f-final as dg composition-fri-roots final}$
 $\text{trace-query-idxs trace-openings composition-query-idxs}$
 $\text{composition-openings trace-table composition-table layers.}$
 $\text{composition-fri-partial-candidate-opening-evidence s out trace-roots}$
 $\text{trace-bs trace-final fri-dg opening-composition-roots composition-bs}$
 $\text{composition-final fri-query-idxs trace-round-layers}$
 $\text{composition-round-layers fr f-fri-roots f-final as dg}$
 $\text{composition-fri-roots final trace-query-idxs trace-openings}$
 $\text{composition-query-idxs composition-openings trace-table}$
 $\text{composition-table} \wedge$
 $\text{generic-fri-sampled-query-candidate-evidence}$
 $(\text{composition-table-low-degree (to-nat fri-dg)})$
 $(\text{Not} \circ \text{composition-table-low-degree maxDegree}) \text{ composition-table}$
 $(\text{to-nat fri-dg}) \text{ opening-composition-roots composition-bs}$
 $\text{composition-final fri-query-idxs composition-round-layers layers})$

definition *generic-fri-sampled-query-bad-pair-union*

$:: ('f \text{ list} \Rightarrow \text{bool}) \Rightarrow ('f \text{ list} \Rightarrow \text{bool}) \Rightarrow \text{nat} \Rightarrow$
 $(\text{nat list} \times 'f \text{ list}) \text{ set}$

where

$\text{generic-fri-sampled-query-bad-pair-union low-degree candidate-bad}$
 $\text{degree-bound} =$
 $\{\text{pair.} \exists \text{ candidate-table roots final round-layers layers.}$
 $\text{pair} \in$
 $\text{generic-fri-sampled-query-bad-pair-set low-degree candidate-bad}$
 $\text{candidate-table degree-bound}$
 $\text{roots final round-layers layers}\}$

definition *generic-fri-sampled-query-projection-fraction-bound*
 $:: ('f \text{ list} \Rightarrow \text{bool}) \Rightarrow ('f \text{ list} \Rightarrow \text{bool}) \Rightarrow \text{nat} \Rightarrow$
 $\text{prob} \Rightarrow \text{bool}$
where
generic-fri-sampled-query-projection-fraction-bound low-degree
candidate-bad degree-bound C $\longleftrightarrow$
 nnreal
 $(\text{card } (\text{snd } ' ($
 $(\text{generic-fri-sampled-query-bad-pair-union low-degree candidate-bad}$
 $\text{degree-bound} \cap (\text{fri-query-index-list-space} \times \text{UNIV})))) /$
 $\text{nnreal } (\text{CARD}('f) \wedge \text{fri-round-count-for-degree-bound degree-bound})$
 $\leq C$

lemma *generic-fri-sampled-query-projection-fraction-boundD:*
assumes
generic-fri-sampled-query-projection-fraction-bound low-degree
candidate-bad degree-bound C
shows
 nnreal
 $(\text{card } (\text{snd } ' ($
 $(\text{generic-fri-sampled-query-bad-pair-union low-degree candidate-bad}$
 $\text{degree-bound} \cap (\text{fri-query-index-list-space} \times \text{UNIV})))) /$
 $\text{nnreal } (\text{CARD}('f) \wedge \text{fri-round-count-for-degree-bound degree-bound})$
 $\leq C$
using *assms*
unfolding *generic-fri-sampled-query-projection-fraction-bound-def* .

lemma *generic-fri-sampled-query-projection-fraction-bound-mono:*
assumes
generic-fri-sampled-query-projection-fraction-bound low-degree
candidate-bad degree-bound C
and $C \leq D$
shows
generic-fri-sampled-query-projection-fraction-bound low-degree
candidate-bad degree-bound D
using *assms*
unfolding *generic-fri-sampled-query-projection-fraction-bound-def*
by (*rule order-trans*)

lemma *generic-fri-sampled-query-projection-fraction-bound-from-card:*
assumes *card-bound:*
 $\text{card } (\text{snd } ' ($
 $(\text{generic-fri-sampled-query-bad-pair-union low-degree candidate-bad}$
 $\text{degree-bound} \cap (\text{fri-query-index-list-space} \times \text{UNIV}))) \leq K$
and *fraction-bound:*
 $\text{nnreal } K /$
 $\text{nnreal } (\text{CARD}('f) \wedge \text{fri-round-count-for-degree-bound degree-bound})$
 $\leq C$

shows
generic-fri-sampled-query-projection-fraction-bound low-degree
candidate-bad degree-bound C

proof –
have *nnreal*
 (*card* (*snd* ‘
 (*generic-fri-sampled-query-bad-pair-union low-degree candidate-bad*
 degree-bound $\cap$ (*fri-query-index-list-space* $\times$ *UNIV*)))) /
nnreal (*CARD*('f) $\wedge$ *fri-round-count-for-degree-bound degree-bound*)
 $\leq$
nnreal K /
nnreal (*CARD*('f) $\wedge$ *fri-round-count-for-degree-bound degree-bound*)
using *card-bound* **by** (*rule nnreal-nat-divide-right-mono*)
also have ... $\leq C$
by (*rule fraction-bound*)
finally show *?thesis*
unfolding *generic-fri-sampled-query-projection-fraction-bound-def* .
qed

lemma *generic-fri-sampled-query-projection-fraction-bound-from-finite-cover*:
assumes *cover*:
snd ‘
 (*generic-fri-sampled-query-bad-pair-union low-degree candidate-bad*
 degree-bound $\cap$ (*fri-query-index-list-space* $\times$ *UNIV*)) $\subseteq B$
and *finite-B*: *finite B*
and *fraction-bound*:
nnreal (*card B*) /
nnreal (*CARD*('f) $\wedge$ *fri-round-count-for-degree-bound degree-bound*)
 $\leq C$
shows
generic-fri-sampled-query-projection-fraction-bound low-degree
candidate-bad degree-bound C

proof (*rule generic-fri-sampled-query-projection-fraction-bound-from-card*)
show *card* (*snd* ‘
 (*generic-fri-sampled-query-bad-pair-union low-degree candidate-bad*
 degree-bound $\cap$ (*fri-query-index-list-space* $\times$ *UNIV*)))
 $\leq$ *card B*
by (*rule card-mono*[*OF finite-B cover*])
show *nnreal* (*card B*) /
nnreal (*CARD*('f) $\wedge$ *fri-round-count-for-degree-bound degree-bound*)
 $\leq C$
by (*rule fraction-bound*)
qed

definition *trace-fri-sampled-query-bad-pair-union*
 $:: (\text{nat list} \times \text{'f list}) \text{ set}$
where
trace-fri-sampled-query-bad-pair-union =
generic-fri-sampled-query-bad-pair-union trace-table-low-degree

(Not $\circ$ trace-table-low-degree) (clength - 1)

definition *composition-fri-sampled-query-bad-pair-union*

$:: 'f \Rightarrow (\text{nat list} \times 'f \text{ list}) \text{ set}$

where

composition-fri-sampled-query-bad-pair-union fri-dg =

generic-fri-sampled-query-bad-pair-union

(*composition-table-low-degree* (to-nat fri-dg))

(Not $\circ$ *composition-table-low-degree* maxDegree) (to-nat fri-dg)

lemma *generic-fri-sampled-query-bad-pair-unionI*:

assumes

(query-idxs, challenges) $\in$

generic-fri-sampled-query-bad-pair-set low-degree candidate-bad

candidate-table degree-bound roots final round-layers layers

shows

(query-idxs, challenges) $\in$

generic-fri-sampled-query-bad-pair-union low-degree candidate-bad

degree-bound

using *assms*

unfolding *generic-fri-sampled-query-bad-pair-union-def* **by** *blast*

lemma *generic-fri-sampled-query-bad-pair-union-subset-challenge-space*:

generic-fri-sampled-query-bad-pair-union low-degree candidate-bad

degree-bound

$\subseteq (\text{UNIV} :: \text{nat list set}) \times$

fri-challenge-space (*fri-round-count-for-degree-bound* degree-bound)

unfolding *generic-fri-sampled-query-bad-pair-union-def*

by (auto intro: *generic-fri-sampled-query-bad-pair-set-challenge-space*)

lemma *generic-fri-sampled-query-bad-pair-union-challenge-projection*:

snd ‘

generic-fri-sampled-query-bad-pair-union low-degree candidate-bad

degree-bound

$\subseteq$ *fri-challenge-space*

(*fri-round-count-for-degree-bound* degree-bound)

using *generic-fri-sampled-query-bad-pair-union-subset-challenge-space*

[of low-degree candidate-bad degree-bound]

by *force*

Fallback sanity bound: the restricted projection is always a subset of the full challenge space. This lemma is intentionally not the main FRI low-degree theorem; the useful symbolic bounds must come from a sharper sampled-query FRI argument.

lemma *generic-fri-sampled-query-projection-fraction-bound-trivial*:

generic-fri-sampled-query-projection-fraction-bound low-degree

candidate-bad degree-bound 1

proof –

let ?S =

snd ‘

```

    (generic-fri-sampled-query-bad-pair-union low-degree candidate-bad
      degree-bound  $\cap$  (fri-query-index-list-space  $\times$  UNIV))
  let ?N = CARD('f)  $\wedge$  fri-round-count-for-degree-bound degree-bound
  have subset:
    ?S  $\subseteq$  fri-challenge-space
    (fri-round-count-for-degree-bound degree-bound)
  using generic-fri-sampled-query-bad-pair-union-challenge-projection
    [of low-degree candidate-bad degree-bound]
  by blast
  have card-le: card ?S  $\leq$  ?N
  using card-mono[OF finite-fri-challenge-space subset]
  by (simp add: card-fri-challenge-space)
  have ?N  $\neq$  0
  by simp
  then have nreal ?N / nreal ?N = (1::prob)
  by (rule nreal-nat-divide-self)
  moreover have nreal (card ?S) / nreal ?N  $\leq$  nreal ?N / nreal ?N
  using card-le by (rule nreal-nat-divide-right-mono)
  ultimately show ?thesis
  unfolding generic-fri-sampled-query-projection-fraction-bound-def
  by simp
qed

```

lemma trace-fri-sampled-query-bad-pair-union-challenge-projection:
 snd ‘ trace-fri-sampled-query-bad-pair-union $\subseteq$
 fri-challenge-space (ceil-log clength)
unfolding trace-fri-sampled-query-bad-pair-union-def
using generic-fri-sampled-query-bad-pair-union-challenge-projection
 [of trace-table-low-degree Not $\circ$ trace-table-low-degree
 clength $- 1$]
by (simp add: fri-round-count-for-degree-bound-def clength-pos)

lemma composition-fri-sampled-query-bad-pair-union-challenge-projection:
 snd ‘ composition-fri-sampled-query-bad-pair-union fri-dg $\subseteq$
 fri-challenge-space (ceil-log (to-nat fri-dg $+ 1$))
unfolding composition-fri-sampled-query-bad-pair-union-def
using generic-fri-sampled-query-bad-pair-union-challenge-projection
 [of composition-table-low-degree (to-nat fri-dg)
 Not $\circ$ composition-table-low-degree maxDegree to-nat fri-dg]
by (simp add: fri-round-count-for-degree-bound-def)

lemma trace-fri-restricted-sampled-query-bad-pair-union-challenge-projection:
 snd ‘
 (trace-fri-sampled-query-bad-pair-union $\cap$
 (fri-query-index-list-space $\times$ UNIV))
 $\subseteq$ fri-challenge-space (ceil-log clength)
using trace-fri-sampled-query-bad-pair-union-challenge-projection **by** blast

lemma composition-fri-restricted-sampled-query-bad-pair-union-challenge-projection:

snd ‘
 (composition-fri-sampled-query-bad-pair-union fri-dg $\cap$
 (fri-query-index-list-space $\times$ UNIV))
 $\subseteq$ fri-challenge-space (ceil-log (to-nat fri-dg + 1))
using composition-fri-sampled-query-bad-pair-union-challenge-projection
 [of fri-dg]
by blast

lemma trace-fri-sampled-query-bad-candidate-iff-pair-set-hit:
 trace-fri-sampled-query-bad-candidate *s out* $\longleftrightarrow$
 trace-fri-sampled-query-pair-set-hit
 (λ trace-table roots final round-layers layers.
 generic-fri-sampled-query-bad-pair-set trace-table-low-degree
 (Not $\circ$ trace-table-low-degree) trace-table (clength - 1)
 roots final round-layers layers)
s out
unfolding trace-fri-sampled-query-bad-candidate-def
 trace-fri-sampled-query-pair-set-hit-def
 generic-fri-sampled-query-bad-pair-set-def
by blast

lemma composition-fri-sampled-query-bad-candidate-iff-pair-set-hit:
 composition-fri-sampled-query-bad-candidate *s out* $\longleftrightarrow$
 composition-fri-sampled-query-pair-set-hit
 (λ fri-dg composition-table roots final round-layers layers.
 generic-fri-sampled-query-bad-pair-set
 (composition-table-low-degree (to-nat fri-dg))
 (Not $\circ$ composition-table-low-degree maxDegree)
 composition-table (to-nat fri-dg) roots final round-layers layers)
s out
unfolding composition-fri-sampled-query-bad-candidate-def
 composition-fri-sampled-query-pair-set-hit-def
 generic-fri-sampled-query-bad-pair-set-def
by auto

lemma trace-fri-sampled-query-bad-candidate-imp-query-challenge-pair-union-hit:
assumes trace-fri-sampled-query-bad-candidate *s out*
shows
 trace-fri-query-challenge-pair-set-hit *s*
 trace-fri-sampled-query-bad-pair-union *out*
proof –
have hit:
 trace-fri-sampled-query-pair-set-hit
 (λ trace-table roots final round-layers layers.
 generic-fri-sampled-query-bad-pair-set trace-table-low-degree
 (Not $\circ$ trace-table-low-degree) trace-table (clength - 1)
 roots final round-layers layers)
s out
using assms trace-fri-sampled-query-bad-candidate-iff-pair-set-hit


```

    by blast
  show ?thesis
proof (rule trace-fri-sampled-query-pair-set-hit-imp-query-challenge-pair-set-hit
  [OF hit])
  fix trace-table roots final round-layers layers
  show
    generic-fri-sampled-query-bad-pair-set trace-table-low-degree
      (Not  $\circ$  trace-table-low-degree) trace-table (clength - 1)
      roots final round-layers layers
     $\subseteq$  trace-fri-sampled-query-bad-pair-union
  unfolding trace-fri-sampled-query-bad-pair-union-def
  by (auto intro: generic-fri-sampled-query-bad-pair-unionI)
qed
qed

lemma composition-fri-sampled-query-bad-candidate-imp-query-challenge-pair-union-hit:
  assumes composition-fri-sampled-query-bad-candidate s out
  shows
    composition-fri-query-challenge-pair-set-hit s
    composition-fri-sampled-query-bad-pair-union out
proof -
  have hit:
    composition-fri-sampled-query-pair-set-hit
      ( $\lambda$ fri-dg composition-table roots final round-layers layers.
        generic-fri-sampled-query-bad-pair-set
          (composition-table-low-degree (to-nat fri-dg))
          (Not  $\circ$  composition-table-low-degree maxDegree)
          composition-table (to-nat fri-dg) roots final round-layers
          layers)
      s out
  using assms composition-fri-sampled-query-bad-candidate-iff-pair-set-hit
  by blast
  show ?thesis
proof (rule
    composition-fri-sampled-query-pair-set-hit-imp-query-challenge-pair-set-hit
    [OF hit])
  fix fri-dg composition-table roots final round-layers layers
  show
    generic-fri-sampled-query-bad-pair-set
      (composition-table-low-degree (to-nat fri-dg))
      (Not  $\circ$  composition-table-low-degree maxDegree)
      composition-table (to-nat fri-dg) roots final round-layers layers
     $\subseteq$  composition-fri-sampled-query-bad-pair-union fri-dg
  unfolding composition-fri-sampled-query-bad-pair-union-def
  by (auto intro: generic-fri-sampled-query-bad-pair-unionI)
qed
qed

lemma trace-fri-sampled-query-bad-candidate-imp-restricted-query-challenge-pair-union-hit:

```

assumes *trace-fri-sampled-query-bad-candidate s out*
shows
trace-fri-query-challenge-pair-set-hit s
(trace-fri-sampled-query-bad-pair-union $\cap$
(fri-query-index-list-space $\times$ UNIV)) out
by (rule *trace-fri-query-challenge-pair-set-hit-restrict-to-query-space*)
(rule
trace-fri-sampled-query-bad-candidate-imp-query-challenge-pair-union-hit
[OF *assms*])

lemma *composition-fri-sampled-query-bad-candidate-imp-restricted-query-challenge-pair-union-hit:*

assumes *composition-fri-sampled-query-bad-candidate s out*
shows
composition-fri-query-challenge-pair-set-hit s
(*ldg. composition-fri-sampled-query-bad-pair-union dg $\cap$*
*(fri-query-index-list-space $\times$ UNIV)) out
by (rule *composition-fri-query-challenge-pair-set-hit-restrict-to-query-space*)
(rule
composition-fri-sampled-query-bad-candidate-imp-query-challenge-pair-union-hit
[OF *assms*])*

lemma *wp-trace-fri-sampled-query-bad-candidate-iff-pair-set-hit:*

wp-event verify-monad (trace-fri-sampled-query-bad-candidate s) s =
wp-event verify-monad
(*trace-fri-sampled-query-pair-set-hit*
(λ *trace-table roots final round-layers layers.*
generic-fri-sampled-query-bad-pair-set trace-table-low-degree
(*Not $\circ$ trace-table-low-degree*) *trace-table (clength - 1)*
roots final round-layers layers)
s) s

proof (rule *antisym*)

show *wp-event verify-monad (trace-fri-sampled-query-bad-candidate s) s*
 $\leq$ *wp-event verify-monad*
(*trace-fri-sampled-query-pair-set-hit*
(λ *trace-table roots final round-layers layers.*
generic-fri-sampled-query-bad-pair-set trace-table-low-degree
(*Not $\circ$ trace-table-low-degree*) *trace-table (clength - 1)*
roots final round-layers layers)
s) s

by (rule *wp-event-mono*)

(use *trace-fri-sampled-query-bad-candidate-iff-pair-set-hit* **in** *blast*)

show *wp-event verify-monad*
(*trace-fri-sampled-query-pair-set-hit*
(λ *trace-table roots final round-layers layers.*
generic-fri-sampled-query-bad-pair-set trace-table-low-degree
(*Not $\circ$ trace-table-low-degree*) *trace-table (clength - 1)*
roots final round-layers layers)
s) s
 $\leq$ *wp-event verify-monad (trace-fri-sampled-query-bad-candidate s) s*

by (rule wp-event-mono)
 (use trace-fri-sampled-query-bad-candidate-iff-pair-set-hit in blast)
 qed

lemma wp-composition-fri-sampled-query-bad-candidate-iff-pair-set-hit:

wp-event verify-monad
 (composition-fri-sampled-query-bad-candidate s) s =
 wp-event verify-monad
 (composition-fri-sampled-query-pair-set-hit
 (λfri-dg composition-table roots final round-layers layers.
 generic-fri-sampled-query-bad-pair-set
 (composition-table-low-degree (to-nat fri-dg))
 (Not ∘ composition-table-low-degree maxDegree)
 composition-table (to-nat fri-dg) roots final round-layers
 layers)
 s) s

proof (rule antisym)

show wp-event verify-monad
 (composition-fri-sampled-query-bad-candidate s) s
 ≤ wp-event verify-monad
 (composition-fri-sampled-query-pair-set-hit
 (λfri-dg composition-table roots final round-layers layers.
 generic-fri-sampled-query-bad-pair-set
 (composition-table-low-degree (to-nat fri-dg))
 (Not ∘ composition-table-low-degree maxDegree)
 composition-table (to-nat fri-dg) roots final round-layers
 layers)
 s) s

by (rule wp-event-mono)
 (use composition-fri-sampled-query-bad-candidate-iff-pair-set-hit
 in blast)

show wp-event verify-monad
 (composition-fri-sampled-query-pair-set-hit
 (λfri-dg composition-table roots final round-layers layers.
 generic-fri-sampled-query-bad-pair-set
 (composition-table-low-degree (to-nat fri-dg))
 (Not ∘ composition-table-low-degree maxDegree)
 composition-table (to-nat fri-dg) roots final round-layers
 layers)
 s) s

≤ wp-event verify-monad
 (composition-fri-sampled-query-bad-candidate s) s

by (rule wp-event-mono)
 (use composition-fri-sampled-query-bad-candidate-iff-pair-set-hit
 in blast)

qed

lemma wp-trace-fri-sampled-query-bad-candidate-bound-by-query-challenge-union-hit:

wp-event verify-monad (trace-fri-sampled-query-bad-candidate s) s ≤

wp-event verify-monad
 (*trace-fri-query-challenge-pair-set-hit s*
 trace-fri-sampled-query-bad-pair-union) *s*
by (rule *wp-event-mono*)
 (rule
 trace-fri-sampled-query-bad-candidate-imp-query-challenge-pair-union-hit)

lemma *wp-composition-fri-sampled-query-bad-candidate-bound-by-query-challenge-union-hit*:
wp-event verify-monad (*composition-fri-sampled-query-bad-candidate s*) *s* ≤
wp-event verify-monad
 (*composition-fri-query-challenge-pair-set-hit s*
 composition-fri-sampled-query-bad-pair-union) *s*
by (rule *wp-event-mono*)
 (rule
 composition-fri-sampled-query-bad-candidate-imp-query-challenge-pair-union-hit)

lemma *wp-trace-fri-sampled-query-bad-candidate-bound-by-restricted-query-challenge-union-hit*:
wp-event verify-monad (*trace-fri-sampled-query-bad-candidate s*) *s* ≤
wp-event verify-monad
 (*trace-fri-query-challenge-pair-set-hit s*
 (*trace-fri-sampled-query-bad-pair-union* ∩
 (*fri-query-index-list-space* × *UNIV*))) *s*
by (rule *wp-event-mono*)
 (rule
 trace-fri-sampled-query-bad-candidate-imp-restricted-query-challenge-pair-union-hit)

lemma *wp-composition-fri-sampled-query-bad-candidate-bound-by-restricted-query-challenge-union-hit*:
wp-event verify-monad (*composition-fri-sampled-query-bad-candidate s*) *s* ≤
wp-event verify-monad
 (*composition-fri-query-challenge-pair-set-hit s*
 (λdg. *composition-fri-sampled-query-bad-pair-union dg* ∩
 (*fri-query-index-list-space* × *UNIV*))) *s*
by (rule *wp-event-mono*)
 (rule
 composition-fri-sampled-query-bad-candidate-imp-restricted-query-challenge-pair-union-hit)

lemma *wp-trace-fri-sampled-query-bad-candidate-bound-from-bad-pair-union-projection*:

assumes *future: trace-fri-future-fresh s*
and *bound*:
 nnreal (*card* (*snd* ‘ *trace-fri-sampled-query-bad-pair-union*)) /
 nnreal (*CARD*(*f*) ^ *ceil-log clength*) ≤ *C*

shows

wp-event verify-monad (*trace-fri-sampled-query-bad-candidate s*) *s* ≤ *C*

proof –

have *wp-event verify-monad* (*trace-fri-sampled-query-bad-candidate s*) *s*
 ≤ *wp-event verify-monad*
 (*trace-fri-query-challenge-pair-set-hit s*
 trace-fri-sampled-query-bad-pair-union) *s*
by (rule

$wp\text{-}trace\text{-}fri\text{-}sampled\text{-}query\text{-}bad\text{-}candidate\text{-}bound\text{-}by\text{-}query\text{-}challenge\text{-}union\text{-}hit)$
also have $\dots \leq C$
by (rule $wp\text{-}trace\text{-}fri\text{-}query\text{-}challenge\text{-}pair\text{-}set\text{-}hit\text{-}bound\text{-}from\text{-}challenge\text{-}projection$
 $[OF\ future\ subset\text{-}refl$
 $trace\text{-}fri\text{-}sampled\text{-}query\text{-}bad\text{-}pair\text{-}union\text{-}challenge\text{-}projection$
 $bound]$)
finally show $?thesis$.
qed

lemma $wp\text{-}composition\text{-}fri\text{-}sampled\text{-}query\text{-}bad\text{-}candidate\text{-}bound\text{-}from\text{-}bad\text{-}pair\text{-}union\text{-}projection$:
fixes $C :: prob$
assumes $future$: $composition\text{-}fri\text{-}future\text{-}fresh\ s$
and $bound$:
 $\bigwedge dg. nnreal$
 $(card\ (snd\ 'composition\text{-}fri\text{-}sampled\text{-}query\text{-}bad\text{-}pair\text{-}union\ dg)) /$
 $nnreal\ (CARD('f) \wedge \text{ceil}\text{-}log\ (to\text{-}nat\ dg + 1)) \leq C$
shows
 $wp\text{-}event\ verify\text{-}monad\ (composition\text{-}fri\text{-}sampled\text{-}query\text{-}bad\text{-}candidate\ s)\ s$
 $\leq C$
proof –
have $wp\text{-}event\ verify\text{-}monad$
 $(composition\text{-}fri\text{-}sampled\text{-}query\text{-}bad\text{-}candidate\ s)\ s \leq$
 $wp\text{-}event\ verify\text{-}monad$
 $(composition\text{-}fri\text{-}query\text{-}challenge\text{-}pair\text{-}set\text{-}hit\ s$
 $composition\text{-}fri\text{-}sampled\text{-}query\text{-}bad\text{-}pair\text{-}union)\ s$
by (rule
 $wp\text{-}composition\text{-}fri\text{-}sampled\text{-}query\text{-}bad\text{-}candidate\text{-}bound\text{-}by\text{-}query\text{-}challenge\text{-}union\text{-}hit)$
also have $\dots \leq C$
by (rule
 $wp\text{-}composition\text{-}fri\text{-}query\text{-}challenge\text{-}pair\text{-}set\text{-}hit\text{-}bound\text{-}from\text{-}challenge\text{-}projection$
 $[OF\ future\ subset\text{-}refl$
 $composition\text{-}fri\text{-}sampled\text{-}query\text{-}bad\text{-}pair\text{-}union\text{-}challenge\text{-}projection$
 $bound]$)
finally show $?thesis$.
qed

lemma $wp\text{-}trace\text{-}fri\text{-}sampled\text{-}query\text{-}bad\text{-}candidate\text{-}bound\text{-}from\text{-}restricted\text{-}bad\text{-}pair\text{-}union\text{-}projection$:
assumes $future$: $trace\text{-}fri\text{-}future\text{-}fresh\ s$
and $bound$:
 $nnreal$
 $(card\ (snd\ ' ($
 $(trace\text{-}fri\text{-}sampled\text{-}query\text{-}bad\text{-}pair\text{-}union \cap$
 $(fri\text{-}query\text{-}index\text{-}list\text{-}space \times UNIV)))) /$
 $nnreal\ (CARD('f) \wedge \text{ceil}\text{-}log\ clength) \leq C$
shows
 $wp\text{-}event\ verify\text{-}monad\ (trace\text{-}fri\text{-}sampled\text{-}query\text{-}bad\text{-}candidate\ s)\ s \leq C$
proof –
have $wp\text{-}event\ verify\text{-}monad\ (trace\text{-}fri\text{-}sampled\text{-}query\text{-}bad\text{-}candidate\ s)\ s$
 $\leq wp\text{-}event\ verify\text{-}monad$

$(\text{trace-fri-query-challenge-pair-set-hit } s$
 $(\text{trace-fri-sampled-query-bad-pair-union } \cap$
 $(\text{fri-query-index-list-space } \times \text{ UNIV}))) s$
by (rule
 $\text{wp-trace-fri-sampled-query-bad-candidate-bound-by-restricted-query-challenge-union-hit})$
also have $\dots \leq C$
by (rule $\text{wp-trace-fri-query-challenge-pair-set-hit-bound-from-challenge-projection}$
 $[\text{OF future subset-refl}$
 $\text{trace-fri-restricted-sampled-query-bad-pair-union-challenge-projection}$
 $\text{bound}]$)
finally show ?thesis .
qed

lemma $\text{wp-composition-fri-sampled-query-bad-candidate-bound-from-restricted-bad-pair-union-projection:}$
fixes $C :: \text{prob}$
assumes $\text{future: composition-fri-future-fresh } s$
and bound:
 $\bigwedge dg. \text{nnreal}$
 $(\text{card } (\text{snd } ($
 $(\text{composition-fri-sampled-query-bad-pair-union } dg \cap$
 $(\text{fri-query-index-list-space } \times \text{ UNIV})))) /$
 $\text{nnreal } (\text{CARD}(f) \wedge \text{ceil-log } (\text{to-nat } dg + 1)) \leq C$
shows
 $\text{wp-event verify-monad } (\text{composition-fri-sampled-query-bad-candidate } s) s$
 $\leq C$
proof –
have $\text{wp-event verify-monad}$
 $(\text{composition-fri-sampled-query-bad-candidate } s) s \leq$
 $\text{wp-event verify-monad}$
 $(\text{composition-fri-query-challenge-pair-set-hit } s$
 $(\lambda dg. \text{composition-fri-sampled-query-bad-pair-union } dg \cap$
 $(\text{fri-query-index-list-space } \times \text{ UNIV}))) s$
by (rule
 $\text{wp-composition-fri-sampled-query-bad-candidate-bound-by-restricted-query-challenge-union-hit})$
also have $\dots \leq C$
by (rule
 $\text{wp-composition-fri-query-challenge-pair-set-hit-bound-from-challenge-projection}$
 $[\text{OF future subset-refl}$
 $\text{composition-fri-restricted-sampled-query-bad-pair-union-challenge-projection}$
 $\text{bound}]$)
finally show ?thesis .
qed

lemma $\text{trace-fri-sampled-query-bad-pair-union-generic-projection-fraction-bound:}$
assumes
 $\text{generic-fri-sampled-query-projection-fraction-bound}$
 $\text{trace-table-low-degree } (\text{Not } \circ \text{ trace-table-low-degree}) (\text{clength} - 1) C$
shows
 nnreal

(card (snd ‘
 (trace-fri-sampled-query-bad-pair-union $\cap$
 (fri-query-index-list-space $\times$ UNIV)))) /
 nnreal (CARD('f) $\wedge$ ceil-log clength) $\leq C$
using *assms*
unfolding *generic-fri-sampled-query-projection-fraction-bound-def*
trace-fri-sampled-query-bad-pair-union-def
by (*simp add: fri-round-count-for-degree-bound-def clength-pos*)

lemma *composition-fri-sampled-query-bad-pair-union-generic-projection-fraction-bound:*

assumes
generic-fri-sampled-query-projection-fraction-bound
 (composition-table-low-degree (to-nat dg))
 (Not $\circ$ composition-table-low-degree maxDegree) (to-nat dg) C

shows

nnreal
 (card (snd ‘
 (composition-fri-sampled-query-bad-pair-union dg $\cap$
 (fri-query-index-list-space $\times$ UNIV)))) /
 nnreal (CARD('f) $\wedge$ ceil-log (to-nat dg + 1)) $\leq C$

using *assms*

unfolding *generic-fri-sampled-query-projection-fraction-bound-def*
composition-fri-sampled-query-bad-pair-union-def
by (*simp add: fri-round-count-for-degree-bound-def*)

lemma *wp-trace-fri-sampled-query-bad-candidate-bound-from-generic-projection-fraction:*

assumes *future: trace-fri-future-fresh s*
and *generic-bound:*
generic-fri-sampled-query-projection-fraction-bound
 trace-table-low-degree (Not $\circ$ trace-table-low-degree)
 (clength - 1) C

shows

wp-event verify-monad (trace-fri-sampled-query-bad-candidate s) s $\leq C$

by (*rule*

wp-trace-fri-sampled-query-bad-candidate-bound-from-restricted-bad-pair-union-projection
 [OF *future*])

(*rule*

trace-fri-sampled-query-bad-pair-union-generic-projection-fraction-bound
 [OF *generic-bound*])

lemma *wp-composition-fri-sampled-query-bad-candidate-bound-from-generic-projection-fraction:*

fixes $C :: \text{prob}$

assumes *future: composition-fri-future-fresh s*

and *generic-bound:*

$\bigwedge dg.$ *generic-fri-sampled-query-projection-fraction-bound*
 (composition-table-low-degree (to-nat dg))
 (Not $\circ$ composition-table-low-degree maxDegree) (to-nat dg) C

shows

wp-event verify-monad (composition-fri-sampled-query-bad-candidate s) s

$\leq C$
by (rule
wp-composition-fri-sampled-query-bad-candidate-bound-from-restricted-bad-pair-union-projection
[OF future])
(rule
composition-fri-sampled-query-bad-pair-union-generic-projection-fraction-bound
[OF generic-bound])

lemma *trace-fri-sampled-query-bad-candidate-imp-query-index-list-space-hit:*

assumes *trace-fri-sampled-query-bad-candidate s out*

shows

trace-fri-query-index-list-set-hit s fri-query-index-list-space out

using *assms trace-fri-sampled-query-bad-candidate-iff-pair-set-hit*

trace-fri-sampled-query-pair-set-hit-imp-query-index-list-space-hit

by *blast*

lemma *composition-fri-sampled-query-bad-candidate-imp-query-index-list-space-hit:*

assumes *composition-fri-sampled-query-bad-candidate s out*

shows

composition-fri-query-index-list-set-hit s fri-query-index-list-space out

using *assms composition-fri-sampled-query-bad-candidate-iff-pair-set-hit*

composition-fri-sampled-query-pair-set-hit-imp-query-index-list-space-hit

by *blast*

lemma *wp-trace-fri-sampled-query-bad-candidate-bound-by-query-index-space-hit:*

wp-event verify-monad (trace-fri-sampled-query-bad-candidate s) s ≤

wp-event verify-monad

(trace-fri-query-index-list-set-hit s fri-query-index-list-space) s

by (rule *wp-event-mono*)

(rule *trace-fri-sampled-query-bad-candidate-imp-query-index-list-space-hit*)

lemma *wp-composition-fri-sampled-query-bad-candidate-bound-by-query-index-space-hit:*

wp-event verify-monad (composition-fri-sampled-query-bad-candidate s) s ≤

wp-event verify-monad

(composition-fri-query-index-list-set-hit s fri-query-index-list-space) s

by (rule *wp-event-mono*)

(rule

composition-fri-sampled-query-bad-candidate-imp-query-index-list-space-hit)

lemma *trace-fri-sampled-query-bad-candidate-iff-canonical:*

trace-fri-sampled-query-bad-candidate s out $\longleftrightarrow$

trace-fri-bad-with-canonical-sampled-layer-chain s out

proof

assume *trace-fri-sampled-query-bad-candidate s out*

then show *trace-fri-bad-with-canonical-sampled-layer-chain s out*

unfolding *trace-fri-sampled-query-bad-candidate-def*

trace-fri-bad-with-canonical-sampled-layer-chain-def

generic-fri-sampled-query-candidate-evidence-def

by *blast*

next

assume *canonical: trace-fri-bad-with-canonical-sampled-layer-chain s out*

then obtain *trace-roots trace-bs trace-final dg composition-roots
composition-bs composition-final fri-query-idxs trace-round-layers
composition-round-layers fr candidate-query-idxs trace-openings
trace-table layers*

where *evidence:*

*trace-fri-partial-candidate-opening-evidence s out trace-roots
trace-bs trace-final dg composition-roots composition-bs
composition-final fri-query-idxs trace-round-layers
composition-round-layers fr candidate-query-idxs trace-openings
trace-table*

and *sampled:*

*generic-fri-canonical-sampled-layer-chain-evidence
trace-table-low-degree trace-table (clength - 1) trace-roots
trace-bs trace-final fri-query-idxs trace-round-layers layers*

unfolding *trace-fri-bad-with-canonical-sampled-layer-chain-def* **by** *blast*

have *not-low: (Not $\circ$ trace-table-low-degree) trace-table*

using *trace-fri-partial-candidate-opening-evidenceD(2)[OF evidence]*

by *(simp add: o-def trace-fri-partial-candidate-evidenceD(3))*

have *bad:*

*generic-fri-sampled-query-candidate-evidence trace-table-low-degree
(Not $\circ$ trace-table-low-degree) trace-table (clength - 1)
trace-roots trace-bs trace-final fri-query-idxs trace-round-layers
layers*

using *sampled not-low*

unfolding *generic-fri-sampled-query-candidate-evidence-def* **by** *blast*

show *trace-fri-sampled-query-bad-candidate s out*

unfolding *trace-fri-sampled-query-bad-candidate-def*

by *(intro exI conjI) (rule evidence, rule bad)*

qed

lemma *composition-fri-partial-candidate-opening-evidence-candidate-bad:*

assumes

*composition-fri-partial-candidate-opening-evidence s out trace-roots
trace-bs trace-final fri-dg opening-composition-roots composition-bs
composition-final fri-query-idxs trace-round-layers
composition-round-layers fr f-fri-roots f-final as dg
composition-fri-roots final trace-query-idxs trace-openings
composition-query-idxs composition-openings trace-table
composition-table*

shows *(Not $\circ$ composition-table-low-degree maxDegree) composition-table*

using *composition-fri-partial-candidate-opening-evidenceD(2)[OF assms]*

by *(simp add: o-def composition-fri-partial-candidate-evidenceD(5))*

lemma *composition-fri-sampled-query-bad-candidate-iff-canonical:*

composition-fri-sampled-query-bad-candidate s out $\longleftrightarrow$

composition-fri-bad-with-canonical-sampled-layer-chain s out

proof *(rule iffI)*

```

assume composition-fri-sampled-query-bad-candidate s out
then show composition-fri-bad-with-canonical-sampled-layer-chain s out
  unfolding composition-fri-sampled-query-bad-candidate-def
    composition-fri-bad-with-canonical-sampled-layer-chain-def
    generic-fri-sampled-query-candidate-evidence-def
  apply (elim exE conjE)
  apply (intro exI conjI)
  apply assumption
  apply assumption
  done
next
assume composition-fri-bad-with-canonical-sampled-layer-chain s out
then show composition-fri-sampled-query-bad-candidate s out
  unfolding composition-fri-sampled-query-bad-candidate-def
    composition-fri-bad-with-canonical-sampled-layer-chain-def
    generic-fri-sampled-query-candidate-evidence-def
  using composition-fri-partial-candidate-opening-evidence-candidate-bad
  apply (elim exE conjE)
  apply (intro exI conjI)
  apply assumption
  apply assumption
  apply (rule composition-fri-partial-candidate-opening-evidence-candidate-bad)
  apply assumption
  done
qed

lemma wp-trace-fri-sampled-query-bad-candidate-iff-canonical:
  wp-event verify-monad (trace-fri-sampled-query-bad-candidate s) s =
    wp-event verify-monad (trace-fri-bad-with-canonical-sampled-layer-chain s) s
proof (rule antisym)
  show wp-event verify-monad (trace-fri-sampled-query-bad-candidate s) s
     $\leq$  wp-event verify-monad (trace-fri-bad-with-canonical-sampled-layer-chain s) s
  by (rule wp-event-mono)
    (use trace-fri-sampled-query-bad-candidate-iff-canonical in blast)
  show wp-event verify-monad (trace-fri-bad-with-canonical-sampled-layer-chain s) s
     $\leq$  wp-event verify-monad (trace-fri-sampled-query-bad-candidate s) s
  by (rule wp-event-mono)
    (use trace-fri-sampled-query-bad-candidate-iff-canonical in blast)
qed

lemma wp-composition-fri-sampled-query-bad-candidate-iff-canonical:
  wp-event verify-monad (composition-fri-sampled-query-bad-candidate s) s =
    wp-event verify-monad (composition-fri-bad-with-canonical-sampled-layer-chain s) s
proof (rule antisym)

```

```

show wp-event verify-monad
  (composition-fri-sampled-query-bad-candidate s) s
  ≤ wp-event verify-monad
  (composition-fri-bad-with-canonical-sampled-layer-chain s) s
  by (rule wp-event-mono)
  (use composition-fri-sampled-query-bad-candidate-iff-canonical in blast)
show wp-event verify-monad
  (composition-fri-bad-with-canonical-sampled-layer-chain s) s
  ≤ wp-event verify-monad
  (composition-fri-sampled-query-bad-candidate s) s
  by (rule wp-event-mono)
  (use composition-fri-sampled-query-bad-candidate-iff-canonical in blast)
qed

```

The following bounds expose the current internal sampled-FRI interface: a sampled layer-chain cover plus a bad-set envelope implies a challenge-list hit, which is then bounded by the existing Fiat-Shamir challenge-list calculus. The cover and envelope parameters are internal proof-layer interfaces; they are not public protocol assumptions.

```

lemma trace-fri-sampled-query-bad-candidate-imp-list-hit:
  assumes sampled: trace-fri-sampled-query-bad-candidate s out
  and cover: trace-fri-sampled-layer-chain-cover s bad
  and envelope:
     $\bigwedge \text{trace-table. trace-fri-multiround-bad-sets bad trace-table} \subseteq B$ 
  shows trace-fri-challenge-list-set-hit s B out
proof –
  have canonical: trace-fri-bad-with-canonical-sampled-layer-chain s out
  using sampled trace-fri-sampled-query-bad-candidate-iff-canonical
  by blast
  then have sampled-chain: trace-fri-bad-with-sampled-layer-chain s out
  by (rule trace-fri-bad-with-canonical-sampled-layer-chain-imp-sampled)
  show ?thesis
  by (rule trace-fri-bad-with-sampled-layer-chain-imp-list-hit
    [OF sampled-chain cover envelope])
qed

```

```

lemma composition-fri-sampled-query-bad-candidate-imp-list-hit:
  assumes sampled: composition-fri-sampled-query-bad-candidate s out
  and cover: composition-fri-sampled-layer-chain-cover s bad
  and envelope:
     $\bigwedge dg \text{ composition-table. } \text{composition-fri-multiround-bad-sets bad dg composition-table} \subseteq B dg$ 
  shows composition-fri-challenge-list-set-hit s B out
proof –
  have canonical:
    composition-fri-bad-with-canonical-sampled-layer-chain s out
  using sampled composition-fri-sampled-query-bad-candidate-iff-canonical
  by blast
  then have sampled-chain:

```

composition-fri-bad-with-sampled-layer-chain s out
by (rule *composition-fri-bad-with-canonical-sampled-layer-chain-imp-sampled*)
show ?thesis
by (rule *composition-fri-bad-with-sampled-layer-chain-imp-list-hit*
[OF *sampling-chain cover envelope*])
qed

lemma *wp-trace-fri-sampled-query-bad-candidate-bound-from-envelope:*

assumes *future: trace-fri-future-fresh s*
and *cover: trace-fri-sampled-layer-chain-cover s bad*
and *envelope:*
 $\bigwedge \text{trace-table. trace-fri-multiround-bad-sets bad trace-table} \subseteq B$
and *subset: $B \subseteq \text{fri-challenge-space (ceil-log clength)}$*
and *bound:*
 $\text{nnreal (card B) / nnreal (CARD('f) ^ ceil-log clength)} \leq C$
shows
 $\text{wp-event verify-monad (trace-fri-sampled-query-bad-candidate s) s} \leq C$

proof –

have $\text{wp-event verify-monad (trace-fri-sampled-query-bad-candidate s) s}$
 $\leq \text{wp-event verify-monad (trace-fri-challenge-list-set-hit s B) s}$
by (rule *wp-event-mono*)
(rule *trace-fri-sampled-query-bad-candidate-imp-list-hit*
[OF - *cover envelope*])
also have $\dots \leq \text{nnreal (card B) /}$
 $\text{nnreal (CARD('f) ^ ceil-log clength)}$
by (rule *wp-verify-monad-trace-fri-challenge-list-set-bound*
[OF *future subset*])
also have $\dots \leq C$
by (rule *bound*)
finally show ?thesis .

qed

lemma *wp-composition-fri-sampled-query-bad-candidate-bound-from-envelope:*

assumes *future: composition-fri-future-fresh s*
and *cover: composition-fri-sampled-layer-chain-cover s bad*
and *envelope:*
 $\bigwedge dg \text{ composition-table.}$
 $\text{composition-fri-multiround-bad-sets bad dg composition-table}$
 $\subseteq B dg$
and *subset:*
 $\bigwedge dg. B dg \subseteq \text{fri-challenge-space (ceil-log (to-nat dg + 1))}$
and *bound:*
 $\bigwedge dg. \text{nnreal (card (B dg)) /}$
 $\text{nnreal (CARD('f) ^ ceil-log (to-nat dg + 1))} \leq C$

shows

$\text{wp-event verify-monad (composition-fri-sampled-query-bad-candidate s) s}$
 $\leq C$

proof –

have *wp-event verify-monad*

(composition-fri-sampled-query-bad-candidate s) s ≤
 wp-event verify-monad (composition-fri-challenge-list-set-hit s B) s
 by (rule wp-event-mono)
 (rule composition-fri-sampled-query-bad-candidate-imp-list-hit
 [OF - cover envelope])
 also have ... ≤ C
 by (rule wp-verify-monad-composition-fri-challenge-list-set-bound
 [OF future subset bound])
 finally show ?thesis .
 qed

lemma checked-staged-security-trace-fri-sampled-query-bad-candidate-bound:

assumes wf: staged-budget-wellformed budgets
and controlled: staged-adversary-controlled budgets A
and finite-B: finite B
and cover:
 ∧data attacker-state.
 Some (data, attacker-state) ∈
 set-dist (execute (checked-staged-transcript-program A)
 adversary-initial-state) ⇒
 trace-fri-sampled-layer-chain-cover
 (verifier-state-from-adversary attacker-state
 (staged-proof-transcript data)) bad
and envelope:
 ∧data attacker-state trace-table.
 Some (data, attacker-state) ∈
 set-dist (execute (checked-staged-transcript-program A)
 adversary-initial-state) ⇒
 trace-fri-multiround-bad-sets bad trace-table ⊆ B
and fresh-bound:
 wp-event (checked-staged-security-experiment-with-data-state A)
 (staged-security-with-data-state-verifier-event
 (λs. trace-fri-challenge-list-fresh-hit s B))
 adversary-initial-state ≤ F
shows
 wp-event (checked-staged-security-experiment-with-data-state A)
 (staged-security-with-data-state-verifier-event
 trace-fri-sampled-query-bad-candidate)
 adversary-initial-state ≤
 F +
 hash-relation-budget-value (card B * ceil-log clength)
 (staged-attacker-query-budget budgets + staged-challenge-query-budget)

proof –

have wp-event (checked-staged-security-experiment-with-data-state A)
 (staged-security-with-data-state-verifier-event
 trace-fri-sampled-query-bad-candidate)
 adversary-initial-state ≤
 wp-event (checked-staged-security-experiment-with-data-state A)
 (staged-security-with-data-state-verifier-event

```

      (λs. trace-fri-challenge-list-set-hit s B))
    adversary-initial-state
  proof (rule wp-event-mono-on-support)
    fix out
    assume support:
      out ∈
        set-dist
          (execute (checked-staged-security-experiment-with-data-state A)
            adversary-initial-state)
    and hit:
      staged-security-with-data-state-verifier-event
        trace-fri-sampled-query-bad-candidate out
  from hit obtain data attacker-state result final-state where
    out-eq: out = Some (((data, attacker-state), result), final-state)
  unfolding staged-security-with-data-state-verifier-event-def
  by (cases out) (auto split: option.splits prod.splits)
  have builder:
    Some (data, attacker-state) ∈
      set-dist (execute (checked-staged-transcript-program A)
        adversary-initial-state)
  using checked-staged-security-experiment-with-data-state-outcomeE
    [OF support[unfolded out-eq]]
  by blast
  let ?s =
    verifier-state-from-adversary attacker-state
      (staged-proof-transcript data)
  have sampled:
    trace-fri-sampled-query-bad-candidate ?s
      (Some (result, final-state))
  using hit unfolding staged-security-with-data-state-verifier-event-def
    out-eq by simp
  have list-hit:
    trace-fri-challenge-list-set-hit ?s B (Some (result, final-state))
  by (rule trace-fri-sampled-query-bad-candidate-imp-list-hit
    [OF sampled cover[OF builder]])
    (rule envelope[OF builder])
  show
    staged-security-with-data-state-verifier-event
      (λs. trace-fri-challenge-list-set-hit s B) out
  unfolding staged-security-with-data-state-verifier-event-def out-eq
  using list-hit by simp
qed
also have ... ≤
  F +
    hash-relation-budget-value (card B * ceil-log clength)
    (staged-attacker-query-budget budgets + staged-challenge-query-budget)
  by (rule checked-staged-security-trace-fri-challenge-list-set-hit-bound
    [OF wf controlled finite-B fresh-bound])
finally show ?thesis .

```

qed

lemma *checked-staged-security-composition-fri-sampled-query-bad-candidate-bound:*

assumes *wf: staged-budget-wellformed budgets*

and *controlled: staged-adversary-controlled budgets A*

and *finite-B: $\bigwedge dg. \text{finite } (B \ dg)$*

and *cover:*

$\bigwedge \text{data attacker-state.}$

*Some (data, attacker-state) $\in$
 set-dist (execute (checked-staged-transcript-program A)
 adversary-initial-state) $\implies$*

*composition-fri-sampled-layer-chain-cover
 (verifier-state-from-adversary attacker-state
 (staged-proof-transcript data)) bad*

and *envelope:*

$\bigwedge \text{data attacker-state dg composition-table.}$

*Some (data, attacker-state) $\in$
 set-dist (execute (checked-staged-transcript-program A)
 adversary-initial-state) $\implies$
 composition-fri-multiround-bad-sets bad dg composition-table
 $\subseteq B \ dg$*

and *fresh-bound:*

*wp-event (checked-staged-security-experiment-with-data-state A)
 (staged-security-with-data-state-verifier-event
 ($\lambda s. \text{composition-fri-challenge-list-fresh-hit } s \ B$))
 adversary-initial-state $\leq F$*

shows

*wp-event (checked-staged-security-experiment-with-data-state A)
 (staged-security-with-data-state-verifier-event
 composition-fri-sampled-query-bad-candidate)
 adversary-initial-state $\leq$
 F +
 hash-relation-budget-value
 ($\sum dg \in (UNIV :: 'f \text{ set}).$
 $\text{card } (B \ dg) * \text{ceil-log } (\text{maxDegree} + 1)$)
 (staged-attacker-query-budget budgets + staged-challenge-query-budget)*

proof –

have *wp-event (checked-staged-security-experiment-with-data-state A)*

*(staged-security-with-data-state-verifier-event
 composition-fri-sampled-query-bad-candidate)*

adversary-initial-state $\leq$

wp-event (checked-staged-security-experiment-with-data-state A)

*(staged-security-with-data-state-verifier-event
 ($\lambda s. \text{composition-fri-challenge-list-set-hit } s \ B$))*

adversary-initial-state

proof (rule *wp-event-mono-on-support*)

fix *out*

assume *support:*

out $\in$

```

    set-dist
    (execute (checked-staged-security-experiment-with-data-state A)
      adversary-initial-state)
  and hit:
    staged-security-with-data-state-verifier-event
    composition-fri-sampled-query-bad-candidate out
  from hit obtain data attacker-state result final-state where
    out-eq: out = Some (((data, attacker-state), result), final-state)
    unfolding staged-security-with-data-state-verifier-event-def
    by (cases out) (auto split: option.splits prod.splits)
  have builder:
    Some (data, attacker-state) ∈
      set-dist (execute (checked-staged-transcript-program A)
        adversary-initial-state)
    using checked-staged-security-experiment-with-data-state-outcomeE
      [OF support[unfolded out-eq]]
    by blast
  let ?s =
    verifier-state-from-adversary attacker-state
    (staged-proof-transcript data)
  have sampled:
    composition-fri-sampled-query-bad-candidate ?s
    (Some (result, final-state))
    using hit unfolding staged-security-with-data-state-verifier-event-def
      out-eq by simp
  have list-hit:
    composition-fri-challenge-list-set-hit ?s B
    (Some (result, final-state))
    by (rule composition-fri-sampled-query-bad-candidate-imp-list-hit
      [OF sampled cover[OF builder]])
    (rule envelope[OF builder])
  show
    staged-security-with-data-state-verifier-event
    (λs. composition-fri-challenge-list-set-hit s B) out
    unfolding staged-security-with-data-state-verifier-event-def out-eq
    using list-hit by simp
qed
also have ... ≤
  F +
  hash-relation-budget-value
  (∑ dg ∈ (UNIV :: 'f set).
    card (B dg) * ceil-log (maxDegree + 1))
  (staged-attacker-query-budget budgets + staged-challenge-query-budget)
  by (rule checked-staged-security-composition-fri-challenge-list-set-hit-bound
    [OF wf controlled finite-B fresh-bound])
  finally show ?thesis .
qed

```

lemma *active-fri-reductions-for-from-trace-augmented-composition-sampled-query-and-empty-conflict-zero:*

fixes *TraceAug TraceMissing TraceMerkle CompSampled CompMerkle EmptySampled EmptyConflict EmptyZero* ::

'f staged-proof-data $\Rightarrow$ 'f protocol-channel $\Rightarrow$ prob

assumes *trace-augmented-bound*:

$\wedge$ *data attacker-state*.

Some (*data*, *attacker-state*) $\in$
 set-dist (execute (checked-staged-transcript-program *A*)
 adversary-initial-state) $\Rightarrow$
 wp-event verify-monad
 (trace-fri-bad-with-header-tied-augmented-partial-candidate
 (verifier-state-from-adversary *attacker-state*
 (staged-proof-transcript *data*)))
 (verifier-state-from-adversary *attacker-state*
 (staged-proof-transcript *data*)) $\leq$ *TraceAug data attacker-state*

and *trace-missing-bound*:

$\wedge$ *data attacker-state*.

Some (*data*, *attacker-state*) $\in$
 set-dist (execute (checked-staged-transcript-program *A*)
 adversary-initial-state) $\Rightarrow$
 wp-event verify-monad
 (trace-fri-header-tied-recorded-sibling-candidate-missing
 (verifier-state-from-adversary *attacker-state*
 (staged-proof-transcript *data*)))
 (verifier-state-from-adversary *attacker-state*
 (staged-proof-transcript *data*)) $\leq$
TraceMissing data attacker-state

and *trace-merkle-bound*:

$\wedge$ *data attacker-state*.

Some (*data*, *attacker-state*) $\in$
 set-dist (execute (checked-staged-transcript-program *A*)
 adversary-initial-state) $\Rightarrow$
 wp-event verify-monad
 (partial-merkle-inconsistency-bad
 (verifier-state-from-adversary *attacker-state*
 (staged-proof-transcript *data*)))
 (verifier-state-from-adversary *attacker-state*
 (staged-proof-transcript *data*)) $\leq$ *TraceMerkle data attacker-state*

and *trace-total*:

$\wedge$ *data attacker-state*.

Some (*data*, *attacker-state*) $\in$
 set-dist (execute (checked-staged-transcript-program *A*)
 adversary-initial-state) $\Rightarrow$
TraceAug data attacker-state +
 (*TraceMissing data attacker-state* + *TraceMerkle data attacker-state*)
 $\leq$ *trace-fri-error*

and *composition-sampled-bound*:

$\wedge$ *data attacker-state*.

Some (*data*, *attacker-state*) $\in$
 set-dist (execute (checked-staged-transcript-program *A*)

$\text{adversary-initial-state}) \implies$
 $\text{wp-event verify-monad}$
 $(\text{composition-fri-sampled-query-bad-candidate}$
 $(\text{verifier-state-from-adversary attacker-state}$
 $(\text{staged-proof-transcript data})))$
 $(\text{verifier-state-from-adversary attacker-state}$
 $(\text{staged-proof-transcript data})) \leq$
 $\text{CompSampled data attacker-state}$
and $\text{composition-merkle-bound:}$
 $\wedge \text{data attacker-state.}$
 $\text{Some (data, attacker-state)} \in$
 $\text{set-dist (execute (checked-staged-transcript-program A)}$
 $\text{adversary-initial-state}) \implies}$
 $\text{wp-event verify-monad}$
 $(\text{partial-merkle-inconsistency-bad}$
 $(\text{verifier-state-from-adversary attacker-state}$
 $(\text{staged-proof-transcript data})))$
 $(\text{verifier-state-from-adversary attacker-state}$
 $(\text{staged-proof-transcript data})) \leq \text{CompMerkle data attacker-state}$
and $\text{composition-total:}$
 $\wedge \text{data attacker-state.}$
 $\text{Some (data, attacker-state)} \in$
 $\text{set-dist (execute (checked-staged-transcript-program A)}$
 $\text{adversary-initial-state}) \implies}$
 $\text{CompSampled data attacker-state} +$
 $((\text{CompMerkle data attacker-state} + 0) +$
 $(\text{CompMerkle data attacker-state} + 0) +$
 $(\text{CompMerkle data attacker-state} + 0) +$
 $(\text{CompMerkle data attacker-state} + 0 + 0) +$
 $\text{CompMerkle data attacker-state}) + 0)$
 $\leq \text{composition-fri-error}$
and $\text{empty-sampled-bound:}$
 $\wedge \text{data attacker-state.}$
 $\text{Some (data, attacker-state)} \in$
 $\text{set-dist (execute (checked-staged-transcript-program A)}$
 $\text{adversary-initial-state}) \implies}$
 $\text{wp-event verify-monad}$
 $(\text{trace-fri-sampled-query-bad-candidate}$
 $(\text{verifier-state-from-adversary attacker-state}$
 $(\text{staged-proof-transcript data})))$
 $(\text{verifier-state-from-adversary attacker-state}$
 $(\text{staged-proof-transcript data})) \leq$
 $\text{EmptySampled data attacker-state}$
and $\text{empty-conflict-bound:}$
 $\wedge \text{data attacker-state.}$
 $\text{Some (data, attacker-state)} \in$
 $\text{set-dist (execute (checked-staged-transcript-program A)}$
 $\text{adversary-initial-state}) \implies}$
 $\text{wp-event verify-monad}$

```

    (trace-fri-sampled-assignment-conflict
      (verifier-state-from-adversary attacker-state
        (staged-proof-transcript data)))
    (verifier-state-from-adversary attacker-state
      (staged-proof-transcript data)) ≤
    EmptyConflict data attacker-state
and empty-zero-bound:
  ∧ data attacker-state.
    Some (data, attacker-state) ∈
      set-dist (execute (checked-staged-transcript-program A)
        adversary-initial-state) ⇒
    wp-event verify-monad
      (trace-fri-zero-round-final-obstruction
        (verifier-state-from-adversary attacker-state
          (staged-proof-transcript data)))
      (verifier-state-from-adversary attacker-state
        (staged-proof-transcript data)) ≤
      EmptyZero data attacker-state
and empty-total:
  ∧ data attacker-state.
    Some (data, attacker-state) ∈
      set-dist (execute (checked-staged-transcript-program A)
        adversary-initial-state) ⇒
    EmptySampled data attacker-state +
      (EmptyConflict data attacker-state + EmptyZero data attacker-state)
    ≤ trace-fri-error
shows active-fri-reductions-for A
proof (rule
  active-fri-reductions-for-from-trace-augmented-composition-canonical-and-empty-conflict-zero
  [OF trace-augmented-bound trace-missing-bound trace-merkle-bound
    trace-total - composition-merkle-bound composition-total -
    empty-conflict-bound empty-zero-bound empty-total])
fix data attacker-state
assume support:
  Some (data, attacker-state) ∈
    set-dist (execute (checked-staged-transcript-program A)
      adversary-initial-state)
show wp-event verify-monad
  (composition-fri-bad-with-canonical-sampled-layer-chain
    (verifier-state-from-adversary attacker-state
      (staged-proof-transcript data)))
  (verifier-state-from-adversary attacker-state
    (staged-proof-transcript data))
  ≤ CompSampled data attacker-state
using composition-sampled-bound[OF support]
by (simp add: wp-composition-fri-sampled-query-bad-candidate-iff-canonical)
next
fix data attacker-state
assume support:

```

Some (data, attacker-state) ∈
set-dist (execute (checked-staged-transcript-program A)
adversary-initial-state)
show *wp-event verify-monad*
(trace-fri-bad-with-canonical-sampled-layer-chain
(verifier-state-from-adversary attacker-state
(staged-proof-transcript data)))
(verifier-state-from-adversary attacker-state
(staged-proof-transcript data)))
 $\leq$ *EmptySampled data attacker-state*
using *empty-sampled-bound[OF support]*
by *(simp add: wp-trace-fri-sampled-query-bad-candidate-iff-canonical)*
qed

definition *generic-fri-sampled-query-finite-cover*
 $:: ('f \text{ list} \Rightarrow \text{bool}) \Rightarrow ('f \text{ list} \Rightarrow \text{bool}) \Rightarrow \text{nat} \Rightarrow$
 $'f \text{ list set} \Rightarrow \text{bool}$

where

generic-fri-sampled-query-finite-cover low-degree candidate-bad
degree-bound B $\longleftrightarrow$
finite B $\wedge$
snd ' (generic-fri-sampled-query-bad-pair-union low-degree candidate-bad
degree-bound $\cap$ (fri-query-index-list-space $\times$ UNIV)) $\subseteq$ B

lemma *generic-fri-sampled-query-finite-coverD:*

assumes

generic-fri-sampled-query-finite-cover low-degree candidate-bad
degree-bound B

shows

finite B
snd ' (generic-fri-sampled-query-bad-pair-union low-degree candidate-bad
degree-bound $\cap$ (fri-query-index-list-space $\times$ UNIV)) $\subseteq$ B

using *assms*

unfolding *generic-fri-sampled-query-finite-cover-def* **by** *simp-all*

lemma *generic-fri-sampled-query-finite-cover-from-candidate-challenge-cover:*

assumes *finite-B: finite B*

and *cover:*

$\bigwedge$ *candidate-table roots final-value round-layers layers query-idxs*
challenges.

query-idxs ∈ fri-query-index-list-space $\implies$

generic-fri-sampled-query-candidate-evidence low-degree candidate-bad
candidate-table degree-bound roots challenges final-value query-idxs
round-layers layers $\implies$

challenges ∈ B

shows

generic-fri-sampled-query-finite-cover low-degree candidate-bad

degree-bound B
proof –
have *subset*:
snd ‘
(generic-fri-sampled-query-bad-pair-union low-degree candidate-bad
degree-bound $\cap$ (fri-query-index-list-space $\times$ UNIV)) $\subseteq$ B
proof
fix *challenges*
assume *challenges $\in$ snd ‘*
(generic-fri-sampled-query-bad-pair-union low-degree candidate-bad
degree-bound $\cap$ (fri-query-index-list-space $\times$ UNIV))
then obtain *pair* **where** *pair-mem*:
pair $\in$ generic-fri-sampled-query-bad-pair-union low-degree
candidate-bad degree-bound $\cap$ (fri-query-index-list-space $\times$ UNIV)
and *challenges-eq*: *challenges = snd pair*
by *blast*
obtain *query-idxs challenges'* **where** *pair-eq*: *pair = (query-idxs, challenges')*
by *(cases pair)*
from *pair-mem pair-eq challenges-eq*
obtain *candidate-table roots final-value round-layers layers*
where *query-space*: *query-idxs $\in$ fri-query-index-list-space*
and *pair*:
(query-idxs, challenges) $\in$
generic-fri-sampled-query-bad-pair-set low-degree candidate-bad
candidate-table degree-bound roots final-value round-layers layers
unfolding *generic-fri-sampled-query-bad-pair-union-def* **by** *auto*
have *evidence*:
generic-fri-sampled-query-candidate-evidence low-degree candidate-bad
candidate-table degree-bound roots challenges final-value query-idxs
round-layers layers
by *(rule generic-fri-sampled-query-bad-pair-setD[OF pair])*
show *challenges $\in$ B*
by *(rule cover[OF query-space evidence])*
qed
show *?thesis*
unfolding *generic-fri-sampled-query-finite-cover-def*
using *finite-B subset* **by** *simp*
qed

lemma *generic-fri-sampled-query-projection-fraction-bound-from-cover*:
assumes *cover*:
generic-fri-sampled-query-finite-cover low-degree candidate-bad
degree-bound B
and *fraction-bound*:
nnreal (card B) /
nnreal (CARD('f) $\wedge$ fri-round-count-for-degree-bound degree-bound)
 $\leq C$
shows
generic-fri-sampled-query-projection-fraction-bound low-degree

candidate-bad degree-bound C
by (rule generic-fri-sampled-query-projection-fraction-bound-from-finite-cover)
(rule generic-fri-sampled-query-finite-coverD(2)[OF cover],
rule generic-fri-sampled-query-finite-coverD(1)[OF cover],
rule fraction-bound)

lemma trace-fri-sampled-query-bad-candidate-bound-from-generic-cover:
assumes future: trace-fri-future-fresh s
and cover:
generic-fri-sampled-query-finite-cover trace-table-low-degree
(Not ◦ trace-table-low-degree) (clength - 1) B
and fraction-bound:
nnreal (card B) / nnreal (CARD('f) ^ ceil-log clength) ≤ C
shows
wp-event verify-monad (trace-fri-sampled-query-bad-candidate s) s ≤ C
proof (rule
wp-trace-fri-sampled-query-bad-candidate-bound-from-generic-projection-fraction
[OF future])
show
generic-fri-sampled-query-projection-fraction-bound
trace-table-low-degree (Not ◦ trace-table-low-degree)
(clength - 1) C
by (rule generic-fri-sampled-query-projection-fraction-bound-from-cover
[OF cover])
(use fraction-bound **in**
⟨simp add: fri-round-count-for-degree-bound-def clength-pos⟩)

qed

lemma composition-fri-sampled-query-bad-candidate-bound-from-generic-cover:
fixes C :: prob
assumes future: composition-fri-future-fresh s
and cover:
 $\bigwedge dg.$ generic-fri-sampled-query-finite-cover
(composition-table-low-degree (to-nat dg))
(Not ◦ composition-table-low-degree maxDegree) (to-nat dg) (B dg)
and fraction-bound:
 $\bigwedge dg.$ nnreal (card (B dg)) /
nnreal (CARD('f) ^ ceil-log (to-nat dg + 1)) ≤ C
shows
wp-event verify-monad (composition-fri-sampled-query-bad-candidate s) s
≤ C
proof (rule
wp-composition-fri-sampled-query-bad-candidate-bound-from-generic-projection-fraction
[OF future])
fix dg
show
generic-fri-sampled-query-projection-fraction-bound
(composition-table-low-degree (to-nat dg))
(Not ◦ composition-table-low-degree maxDegree) (to-nat dg) C

```

    by (rule generic-fri-sampled-query-projection-fraction-bound-from-cover
        [OF cover[of dg]])
    (use fraction-bound[of dg] in
        ⟨simp add: fri-round-count-for-degree-bound-def⟩)
qed

end

end

```

```

theory Soundness-FRI-Query-Aware-Interface
imports Soundness-FRI-Sampled-Interface
begin

```

Query-aware sampled FRI interface.

This theory names the joint query/challenge boundary used by the sampled FRI low-degree reduction. The definitions are generic and are instantiated by the trace and composition FRI routes.

```

context soundness
begin

```

```

definition generic-fri-sampled-query-restricted-bad-pairs
  :: ('f list  $\Rightarrow$  bool)  $\Rightarrow$  ('f list  $\Rightarrow$  bool)  $\Rightarrow$  nat  $\Rightarrow$ 
    (nat list  $\times$  'f list) set

```

```

where
  generic-fri-sampled-query-restricted-bad-pairs low-degree candidate-bad
    degree-bound =
  generic-fri-sampled-query-bad-pair-union low-degree candidate-bad
    degree-bound  $\cap$  (fri-query-index-list-space  $\times$  UNIV)

```

```

definition generic-fri-sampled-query-pair-fraction-bound
  :: ('f list  $\Rightarrow$  bool)  $\Rightarrow$  ('f list  $\Rightarrow$  bool)  $\Rightarrow$  nat  $\Rightarrow$ 
    prob  $\Rightarrow$  bool

```

```

where
  generic-fri-sampled-query-pair-fraction-bound low-degree candidate-bad
    degree-bound C  $\longleftrightarrow$ 
  nnreal
    (card
      (generic-fri-sampled-query-restricted-bad-pairs low-degree
        candidate-bad degree-bound)) /
  nnreal
    (card fri-query-index-list-space *
      CARD('f)  $\wedge$  fri-round-count-for-degree-bound degree-bound)
   $\leq$  C

```

```

definition generic-fri-sampled-query-challenge-projection
  :: ('f list  $\Rightarrow$  bool)  $\Rightarrow$  ('f list  $\Rightarrow$  bool)  $\Rightarrow$  nat  $\Rightarrow$ 
    'f list set

```

where

generic-fri-sampled-query-challenge-projection low-degree candidate-bad
degree-bound =
snd ‘
generic-fri-sampled-query-restricted-bad-pairs low-degree
candidate-bad degree-bound

definition *generic-fri-sampled-query-query-fiber*

$:: ('f \text{ list} \Rightarrow \text{bool}) \Rightarrow ('f \text{ list} \Rightarrow \text{bool}) \Rightarrow \text{nat} \Rightarrow$
 $'f \text{ list} \Rightarrow \text{nat list set}$

where

generic-fri-sampled-query-query-fiber low-degree candidate-bad
degree-bound challenges =
fri-query-challenge-pair-query-fiber
(generic-fri-sampled-query-restricted-bad-pairs low-degree
candidate-bad degree-bound) challenges

definition *query-rounds-index-list-hit*

$:: ('f, 'a) \text{ protocol-channel-scheme} \Rightarrow \text{nat list} \Rightarrow \text{nat} \Rightarrow$
 $(\text{unit list} \times ('f, 'a) \text{ protocol-channel-scheme}) \text{ option} \Rightarrow \text{bool}$

where

query-rounds-index-list-hit s query-idxs n out $\longleftrightarrow$
 $(\exists \text{ results } t \text{ raw-idxs.}$
 $\text{out} = \text{Some (results, } t) \wedge$
 $\text{length raw-idxs} = n \wedge$
 $\text{length query-idxs} = n \wedge$
 $\text{query-idxs} = \text{map } (\lambda \text{ raw. index (to-nat raw)) raw-idxs} \wedge$
 $(\forall i < n. \exists x.$
 $\text{fmlookup (HashMap } t)$
 $(\text{QueryIndexChallenge (PQueryCounter } s + i) x) =$
 $\text{Some (raw-idxs ! } i)))$

lemma *card-fri-query-index-list-space:*

card fri-query-index-list-space = card query-sample-space ^ rounds
unfolding *fri-query-index-list-space-def*
using *card-lists-length-eq[OF finite-query-sample-space, of rounds]*
by (*simp add: conj-commute*)

lemma *query-rounds-index-list-hit-None[simp]:*

$\neg \text{query-rounds-index-list-hit } s \text{ query-idxs } n \text{ None}$
unfolding *query-rounds-index-list-hit-def* **by** *simp*

lemma *query-rounds-index-list-hit-tail:*

assumes *head:*
 $\text{Some } ((), s1) \in$
 set-dist
 $(\text{execute}$
 $(\text{verifier-query-round-program fr f-fl f-final as fl final}) s)$
and *tail:*


```

    Some (results, t) ∈
      set-dist
        (execute
          (ntimes
            (verifier-query-round-program fr f-fl f-final as fl final) n)
          s1)
  and hit:
    query-rounds-index-list-hit s query-idxs (Suc n)
      (Some (u # results, t))
  shows
    query-rounds-index-list-hit s1 (tl query-idxs) n
      (Some (results, t))
  proof -
  from hit obtain raw-idxs where
    len-raw: length raw-idxs = Suc n
  and len-query: length query-idxs = Suc n
  and query-eq: query-idxs = map (λraw. index (to-nat raw)) raw-idxs
  and lookups:
    ∧ i. i < Suc n ⇒ ∃ x.
      fmlookup (HashMap t)
        (QueryIndexChallenge (PQueryCounter s + i) x) =
          Some (raw-idxs ! i)
  unfolding query-rounds-index-list-hit-def by auto
  have counter-s1: PQueryCounter s1 = Suc (PQueryCounter s)
  using verifier-query-round-program-outcome[OF head] by blast
  let ?raw-tail = tl raw-idxs
  have len-tail-raw: length ?raw-tail = n
  using len-raw by simp
  have len-tail-query: length (tl query-idxs) = n
  using len-query by simp
  have tail-eq:
    tl query-idxs = map (λraw. index (to-nat raw)) ?raw-tail
  using query-eq by (cases raw-idxs; cases query-idxs; simp)
  have tail-lookups:
    ∀ i < n. ∃ x.
      fmlookup (HashMap t)
        (QueryIndexChallenge (PQueryCounter s1 + i) x) =
          Some (?raw-tail ! i)
  proof (intro allI impI)
  fix i
  assume i-bound: i < n
  then obtain x where lookup:
    fmlookup (HashMap t)
      (QueryIndexChallenge (PQueryCounter s + Suc i) x) =
        Some (raw-idxs ! Suc i)
  using lookups[of Suc i] by auto
  have counter-eq: PQueryCounter s1 + i = PQueryCounter s + Suc i
  using counter-s1 by presburger
  have tail-raw: ?raw-tail ! i = raw-idxs ! Suc i

```

```

    using i-bound len-raw by (cases raw-idxs) auto
show  $\exists x$ .
  fmlookup (HashMap t)
    (QueryIndexChallenge (PQueryCounter s1 + i) x) =
      Some (?raw-tail ! i)
  by (rule exI[where x=x])
    (use lookup counter-eq tail-raw in simp)
qed
show ?thesis
  unfolding query-rounds-index-list-hit-def
  by (intro exI[of - results] exI[of - t] exI[of - ?raw-tail] conjI)
    (use len-tail-raw len-tail-query tail-eq tail-lookups in simp-all)
qed

lemma query-rounds-index-list-hit-head-index:
  assumes future: query-future-fresh s
  and head:
    Some ((), s1)  $\in$ 
      set-dist
        (execute
          (verifier-query-round-program fr f-fl f-final as fl final) s)
  and tail:
    Some (results, t)  $\in$ 
      set-dist
        (execute
          (ntimes
            (verifier-query-round-program fr f-fl f-final as fl final) n)
          s1)
  and hit:
    query-rounds-index-list-hit s query-idxs (Suc n)
      (Some (u # results, t))
  shows
    query-round-any-index-set-hit s {query-idxs ! 0} (Some ((), s1))
proof -
  from head[unfolded verifier-query-round-program-alt-def]
  obtain raw s0 where
    recv: Some (raw, s0)  $\in$  set-dist (execute receive-query-index-challenge s)
  and after:
    Some ((), s1)  $\in$ 
      set-dist
        (execute
          (verifier-query-round-after-index-program fr f-fl f-final as fl
            final raw) s0)
  by (auto elim!: set-dist-bindE)
  from hit obtain raw-idxs x where
    len-raw: length raw-idxs = Suc n
  and len-query: length query-idxs = Suc n
  and query-eq: query-idxs = map ( $\lambda$ raw. index (to-nat raw)) raw-idxs
  and lookup-t:

```

```

    fmlookup (HashMap t)
      (QueryIndexChallenge (PQueryCounter s) x) = Some (raw-idxs ! 0)
  unfolding query-rounds-index-list-hit-def by auto
  have counter-s1: PQueryCounter s1 = Suc (PQueryCounter s)
    using verifier-query-round-program-outcome[OF head] by blast
  have past: PQueryCounter s < PQueryCounter s1
    using counter-s1 by simp
  have lookup-s1:
    fmlookup (HashMap s1) (QueryIndexChallenge (PQueryCounter s) x) =
      Some (raw-idxs ! 0)
    using ntimes-verifier-query-round-program-preserves-past-query-lookup
      [OF past tail, of x] lookup-t
    by simp
  have lookup-s0-x:
    fmlookup (HashMap s0) (QueryIndexChallenge (PQueryCounter s) x) =
      Some (raw-idxs ! 0)
    using verifier-query-round-after-index-program-preserves-query-lookup
      [OF after, of PQueryCounter s x] lookup-s1
    by simp
  have lookup-s0-current:
    fmlookup (HashMap s0)
      (QueryIndexChallenge (PQueryCounter s) (PState s)) = Some raw
    using receive-query-index-challenge-outcome[OF recv] by blast
  have key-eq:
    QueryIndexChallenge (PQueryCounter s) x =
      QueryIndexChallenge (PQueryCounter s) (PState s)
  proof (rule ccontr)
    assume neg:
      QueryIndexChallenge (PQueryCounter s) x ≠
        QueryIndexChallenge (PQueryCounter s) (PState s)
    have lookup-old:
      fmlookup (HashMap s0) (QueryIndexChallenge (PQueryCounter s) x) =
        fmlookup (HashMap s) (QueryIndexChallenge (PQueryCounter s) x)
      by (rule receive-query-index-challenge-preserves-other-lookup
        [OF recv neg])
    have fmlookup (HashMap s) (QueryIndexChallenge (PQueryCounter s) x) =
      None
      using future unfolding query-future-fresh-def by simp
    then show False
      using lookup-s0-x lookup-old by simp
  qed
  have raw-eq: raw-idxs ! 0 = raw
    using lookup-s0-x lookup-s0-current key-eq by simp
  have index (to-nat raw) = query-idxs ! 0
    using query-eq raw-eq len-raw by simp
  then show ?thesis
    unfolding query-round-any-index-set-hit-def
    using lookup-s1 raw-eq query-eq len-raw by auto
qed

```

```

lemma wp-ntimes-verifier-query-round-program-index-list-bound:
  assumes future: query-future-fresh s
    and raw-bound: query-index-raw-preimage-bound
    and len-query: length query-idxs = n
    and subset: set query-idxs  $\subseteq$  query-sample-space
  shows
    wp-event
      (ntimes (verifier-query-round-program fr f-fl f-final as fl final) n)
      (query-rounds-index-list-hit s query-idxs n) s  $\leq$ 
      (1 / nnreal (card query-sample-space))  $^n$ 
    using future len-query subset
  proof (induction n arbitrary: s query-idxs)
    case 0
    show ?case
      using wp-event-le-1
      [of ntimes (verifier-query-round-program fr f-fl f-final as fl final) 0
        query-rounds-index-list-hit s query-idxs 0 s]
      by simp
    next
    case (Suc n)
    let ?prog = verifier-query-round-program fr f-fl f-final as fl final
    let ?p = 1 / nnreal (card query-sample-space)
    let ?Q = query-rounds-index-list-hit s query-idxs (Suc n)
    let ?Head = query-round-any-index-set-hit s {query-idxs ! 0}
    have query0-in: query-idxs ! 0  $\in$  query-sample-space
      using Suc.premis(2,3) by (auto intro: nth-mem)
    have singleton-subset: {query-idxs ! 0}  $\subseteq$  query-sample-space
      using query0-in by simp
    have head-bound:
      wp-event ?prog ?Head s  $\leq$  ?p
    proof -
      have base:
        wp-event ?prog ?Head s  $\leq$ 
          nnreal (card {query-idxs ! 0}) /
            nnreal (card query-sample-space)
        by (rule wp-verifier-query-round-program-any-index-set-bound
          [OF Suc.premis(1) raw-bound singleton-subset])
      then show ?thesis
        by simp
    qed
    show ?case
      unfolding ntimes.simps
    proof (rule order-trans)
      show
        wp-event
          (?prog  $\gg$  (λu. ntimes ?prog n  $\gg$  (λxs. return (u # xs))))
          ?Q s
         $\leq$  wp-event ?prog ?Head s * (?p  $^n$ )

```

```

proof (rule wp-event-bind-bound-by-head-and-cont)
  show  $\neg ?Q \text{ None}$ 
    by simp
next
  fix  $u \ s1$ 
  assume head:
     $\text{Some } (u, s1) \in \text{set-dist } (\text{execute } ?\text{prog } s)$ 
    and not-head:  $\neg ?\text{Head } (\text{Some } (u, s1))$ 
  have u-eq:  $u = ()$ 
    by (cases  $u$ ) simp
  show
    wp-event
     $(\text{ntimes } ?\text{prog } n \gg (\lambda xs. \text{return } (u \# xs)))$ 
     $?Q \ s1 = 0$ 
proof (rule ccontr)
  assume nonzero:
    wp-event
     $(\text{ntimes } ?\text{prog } n \gg (\lambda xs. \text{return } (u \# xs)))$ 
     $?Q \ s1 \neq 0$ 
  have positive:
     $0 <$ 
    wp-event
     $(\text{ntimes } ?\text{prog } n \gg (\lambda xs. \text{return } (u \# xs)))$ 
     $?Q \ s1$ 
  using nonzero by simp
from wp-event-pos-imp-exists-support[OF positive]
obtain out where out-support:
  out  $\in$ 
  set-dist
   $(\text{execute } (\text{ntimes } ?\text{prog } n \gg (\lambda xs. \text{return } (u \# xs))) \ s1)$ 
and hit:  $?Q \ \text{out}$ 
by blast
from hit obtain  $ys \ t0$  raw-idxs where out-some:  $\text{out} = \text{Some } (ys, t0)$ 
  unfolding query-rounds-index-list-hit-def by blast
have  $\exists \text{ results } t.$ 
   $\text{Some } (\text{results}, t) \in \text{set-dist } (\text{execute } (\text{ntimes } ?\text{prog } n) \ s1) \wedge$ 
   $\text{out} = \text{Some } (u \# \text{results}, t)$ 
proof (rule set-dist-bindE[OF out-support[unfolded out-some]])
  fix results  $t$ 
  assume tail:
     $\text{Some } (\text{results}, t) \in \text{set-dist } (\text{execute } (\text{ntimes } ?\text{prog } n) \ s1)$ 
    and ret:
     $\text{Some } (ys, t0) \in \text{set-dist } (\text{execute } (\text{return } (u \# \text{results})) \ t)$ 
  have out-eq:  $\text{out} = \text{Some } (u \# \text{results}, t)$ 
    using ret out-some
  unfolding set-dist-def return.rep-eq dist-return-def
    dist-delta-dist delta-map-def
  by simp

```

```

  show ?thesis
    using tail out-eq by blast
qed
then obtain results t where
  tail:
    Some (results, t) ∈ set-dist (execute (ntimes ?prog n) s1)
  and out-eq: out = Some (u # results, t)
  by blast
have head-hit: ?Head (Some (u, s1))
  unfolding u-eq
  by (rule query-rounds-index-list-hit-head-index
    [OF Suc.prem1(1) head[unfolded u-eq] tail])
    (use hit out-eq u-eq in simp)
then show False
  using not-head by simp
qed
next
fix u s1
assume head:
  Some (u, s1) ∈ set-dist (execute ?prog s)
  and head-hit: ?Head (Some (u, s1))
have u-eq: u = ()
  by (cases u) simp
have future-s1: query-future-fresh s1
  by (rule verifier-query-round-program-preserves-query-future-fresh
    [OF Suc.prem1(1) head[unfolded u-eq]])
have tail-len: length (tl query-idxs) = n
  using Suc.prem2(2) by simp
have tail-subset: set (tl query-idxs) ⊆ query-sample-space
  using Suc.prem3(3) by (cases query-idxs) auto
have tail-bound:
  wp-event (ntimes ?prog n)
    (query-rounds-index-list-hit s1 (tl query-idxs) n) s1 ≤
    ?p ^ n
  by (rule Suc.IH[OF future-s1 tail-len tail-subset])
show
  wp-event
    (ntimes ?prog n ≫ (λxs. return (u # xs)))
    ?Q s1 ≤ ?p ^ n
proof (rule order-trans[OF - tail-bound])
  show
    wp-event
      (ntimes ?prog n ≫ (λxs. return (u # xs)))
      ?Q s1
    ≤ wp-event (ntimes ?prog n)
      (query-rounds-index-list-hit s1 (tl query-idxs) n) s1
  proof (rule wp-event-bind-bound-by-head-event)
    show
      wp-event (ntimes ?prog n)

```

```

      (query-rounds-index-list-hit s1 (tl query-idxs) n) s1
    ≤ wp-event (ntimes ?prog n)
      (query-rounds-index-list-hit s1 (tl query-idxs) n) s1
  by simp
next
  show
    ?Q None  $\implies$ 
      query-rounds-index-list-hit s1 (tl query-idxs) n None
  by simp
next
  fix results t out
  assume tail:
    Some (results, t)  $\in$  set-dist (execute (ntimes ?prog n) s1)
  and ret:
    out  $\in$  set-dist (execute (return (u # results)) t)
  and hit: ?Q out
  have out-eq: out = Some (u # results, t)
  using ret
  unfolding set-dist-def return.rep-eq dist-return-def
    dist-delta-dist delta-map-def
  by simp
  show query-rounds-index-list-hit s1 (tl query-idxs) n
    (Some (results, t))
  by (rule query-rounds-index-list-hit-tail
    [OF head[unfolded u-eq] tail])
    (use hit out-eq u-eq in simp)
qed
qed
qed
next
  show wp-event ?prog ?Head s * ?p  $\wedge$  n  $\leq$  ?p  $\wedge$  Suc n
  using head-bound by (simp add: mult-right-mono)
qed
qed

lemma wp-ntimes-verifier-query-round-program-index-list-set-bound:
  assumes future: query-future-fresh s
  and raw-bound: query-index-raw-preimage-bound
  and finite-Q: finite Q
  and lengths:  $\bigwedge$  query-idxs. query-idxs  $\in$  Q  $\implies$  length query-idxs = n
  and subsets:
     $\bigwedge$  query-idxs. query-idxs  $\in$  Q  $\implies$ 
      set query-idxs  $\subseteq$  query-sample-space
  shows
    wp-event
      (ntimes (verifier-query-round-program fr f-fl f-final as fl final) n)
      ( $\lambda$ out.  $\exists$  query-idxs  $\in$  Q.
        query-rounds-index-list-hit s query-idxs n out) s  $\leq$ 
      nnreal (card Q) *

```

$(1 / \text{nnreal } (\text{card } \text{query-sample-space})) ^ n$
proof –
have
wp-event
 $(\text{ntimes } (\text{verifier-query-round-program } \text{fr } f\text{-fl } f\text{-final } \text{as } fl \text{ final}) \text{ } n)$
 $(\lambda \text{out. } \exists \text{ query-idxs } \in Q.$
 $\text{query-rounds-index-list-hit } s \text{ query-idxs } n \text{ out}) \text{ } s \leq$
 $(\sum \text{ query-idxs } \in Q.$
 $(1 / \text{nnreal } (\text{card } \text{query-sample-space})) ^ n)$
by $(\text{rule } \text{wp-event-finite-UN-bound} [OF \text{ finite-}Q])$
 $(\text{rule } \text{wp-ntimes-verifier-query-round-program-index-list-bound}$
 $[OF \text{ future raw-bound lengths subsets}])$
also have ... =
 $\text{nnreal } (\text{card } Q) *$
 $(1 / \text{nnreal } (\text{card } \text{query-sample-space})) ^ n$
using *finite-Q* **by** *simp*
finally show *?thesis* .
qed

lemma *wp-ntimes-verifier-query-round-program-fri-index-list-set-bound:*
assumes *future: query-future-fresh s*
and *raw-bound: query-index-raw-preimage-bound*
and *subset: $Q \subseteq \text{fri-query-index-list-space}$*
shows
wp-event
 $(\text{ntimes}$
 $(\text{verifier-query-round-program } \text{fr } f\text{-fl } f\text{-final } \text{as } fl \text{ final}) \text{ rounds})$
 $(\lambda \text{out. } \exists \text{ query-idxs } \in Q.$
 $\text{query-rounds-index-list-hit } s \text{ query-idxs } \text{rounds out}) \text{ } s \leq$
 $\text{nnreal } (\text{card } Q) *$
 $(1 / \text{nnreal } (\text{card } \text{query-sample-space})) ^ \text{rounds}$
proof –
have *finite-Q: finite Q*
using *subset* **by** $(\text{rule } \text{finite-subset}) (\text{rule } \text{finite-fri-query-index-list-space})$
show *?thesis*
proof $(\text{rule } \text{wp-ntimes-verifier-query-round-program-index-list-set-bound}$
 $[OF \text{ future raw-bound finite-}Q])$
fix *query-idxs*
assume $\text{query-idxs } \in Q$
then show $\text{length } \text{query-idxs} = \text{rounds}$
using *subset* **unfolding** *fri-query-index-list-space-def* **by** *auto*
next
fix *query-idxs*
assume $\text{query-idxs } \in Q$
then show $\text{set } \text{query-idxs} \subseteq \text{query-sample-space}$
using *subset* **unfolding** *fri-query-index-list-space-def* **by** *auto*
qed
qed

lemma *generic-fri-sampled-query-query-fiber-entries-subset:*
query-index-list-entries
 (generic-fri-sampled-query-query-fiber low-degree candidate-bad
 degree-bound challenges)
 $\subseteq$ *query-sample-space*
unfolding *query-index-list-entries-def*
generic-fri-sampled-query-query-fiber-def
generic-fri-sampled-query-restricted-bad-pairs-def
fri-query-challenge-pair-query-fiber-def
fri-query-index-list-space-def
by *auto*

lemma *finite-generic-fri-sampled-query-query-fiber-entries:*
finite
 (*query-index-list-entries*
 (generic-fri-sampled-query-query-fiber low-degree candidate-bad
 degree-bound challenges))
using *generic-fri-sampled-query-query-fiber-entries-subset*
by (rule *finite-subset*) *simp*

lemma *generic-fri-sampled-query-restricted-bad-pairs-subset:*
generic-fri-sampled-query-restricted-bad-pairs low-degree candidate-bad
 degree-bound
 $\subseteq$ *fri-query-index-list-space* $\times$
fri-challenge-space
 (*fri-round-count-for-degree-bound* degree-bound)
unfolding *generic-fri-sampled-query-restricted-bad-pairs-def*
using *generic-fri-sampled-query-bad-pair-union-subset-challenge-space*
 [of low-degree candidate-bad degree-bound]
by *blast*

lemma *finite-generic-fri-sampled-query-restricted-bad-pairs:*
finite
 (*generic-fri-sampled-query-restricted-bad-pairs* low-degree candidate-bad
 degree-bound)
proof (rule *finite-subset*)
show
generic-fri-sampled-query-restricted-bad-pairs low-degree candidate-bad
 degree-bound
 $\subseteq$ *fri-query-index-list-space* $\times$
fri-challenge-space
 (*fri-round-count-for-degree-bound* degree-bound)
by (rule *generic-fri-sampled-query-restricted-bad-pairs-subset*)
show
finite
 (*fri-query-index-list-space* $\times$
fri-challenge-space
 (*fri-round-count-for-degree-bound* degree-bound))
by (*simp* add: *finite-fri-query-index-list-space*)

$\text{finite-fri-challenge-space})$
qed
lemma *generic-fri-sampled-query-pair-fraction-boundD*:
assumes
 $\text{generic-fri-sampled-query-pair-fraction-bound low-degree candidate-bad}$
 $\text{degree-bound } C$
shows
 nnreal
 $(\text{card}$
 $(\text{generic-fri-sampled-query-restricted-bad-pairs low-degree}$
 $\text{candidate-bad degree-bound})) /$
 nnreal
 $(\text{card fri-query-index-list-space} *$
 $\text{CARD}(f) \wedge \text{fri-round-count-for-degree-bound degree-bound})$
 $\leq C$
using *assms*
unfolding *generic-fri-sampled-query-pair-fraction-bound-def* .

lemma *generic-fri-sampled-query-challenge-projection-subset*:
 $\text{generic-fri-sampled-query-challenge-projection low-degree candidate-bad}$
 degree-bound
 $\subseteq \text{fri-challenge-space}$
 $(\text{fri-round-count-for-degree-bound degree-bound})$
unfolding *generic-fri-sampled-query-challenge-projection-def*
using *generic-fri-sampled-query-restricted-bad-pairs-subset*
 $[\text{of low-degree candidate-bad degree-bound}]$
by *force*

lemma *generic-fri-sampled-query-query-fiber-subset*:
 $\text{generic-fri-sampled-query-query-fiber low-degree candidate-bad}$
 $\text{degree-bound challenges}$
 $\subseteq \text{fri-query-index-list-space}$
unfolding *generic-fri-sampled-query-query-fiber-def*
 $\text{generic-fri-sampled-query-restricted-bad-pairs-def}$
 $\text{fri-query-challenge-pair-query-fiber-def}$
by *auto*

lemma *finite-generic-fri-sampled-query-query-fiber*:
 finite
 $(\text{generic-fri-sampled-query-query-fiber low-degree candidate-bad}$
 $\text{degree-bound challenges})$
using *generic-fri-sampled-query-query-fiber-subset*
 $[\text{of low-degree candidate-bad degree-bound challenges}]$
by $(\text{rule finite-subset}) (\text{rule finite-fri-query-index-list-space})$

lemma *generic-fri-sampled-query-query-fiber-entries-card-le*:
 card
 $(\text{query-index-list-entries}$

```

    (generic-fri-sampled-query-query-fiber low-degree candidate-bad
     degree-bound challenges))
  ≤
  card
  (generic-fri-sampled-query-query-fiber low-degree candidate-bad
   degree-bound challenges) * rounds
by (rule query-index-list-entries-card-le)
  (rule finite-generic-fri-sampled-query-query-fiber,
   rule generic-fri-sampled-query-query-fiber-subset)

lemma generic-fri-sampled-query-restricted-bad-pairs-as-fiber-Sigma:
  generic-fri-sampled-query-restricted-bad-pairs low-degree candidate-bad
  degree-bound =
  (λ(challenges, query-idxs). (query-idxs, challenges)) ‘
  (SIGMA challenges:
   fri-challenge-space (fri-round-count-for-degree-bound degree-bound).
   generic-fri-sampled-query-query-fiber low-degree candidate-bad
   degree-bound challenges)
unfolding generic-fri-sampled-query-query-fiber-def
  fri-query-challenge-pair-query-fiber-def
using generic-fri-sampled-query-restricted-bad-pairs-subset
by force

lemma generic-fri-sampled-query-restricted-bad-pairs-card-eq-fiber-sum:
  card
  (generic-fri-sampled-query-restricted-bad-pairs low-degree candidate-bad
   degree-bound) =
  (∑ challenges ∈
   fri-challenge-space (fri-round-count-for-degree-bound degree-bound).
   card
   (generic-fri-sampled-query-query-fiber low-degree candidate-bad
    degree-bound challenges))

proof –
  let ?S =
    SIGMA challenges:
      fri-challenge-space (fri-round-count-for-degree-bound degree-bound).
      generic-fri-sampled-query-query-fiber low-degree candidate-bad
      degree-bound challenges
  let ?swap = λ(challenges, query-idxs). (query-idxs, challenges)
  have finite-S: finite ?S
    by (rule finite-SigmaI)
      (rule finite-fri-challenge-space,
       rule finite-generic-fri-sampled-query-query-fiber)
  have inj: inj-on ?swap ?S
    unfolding inj-on-def by auto
  have finite-challenge-space:
    finite
    (fri-challenge-space
     (fri-round-count-for-degree-bound degree-bound))

```

```

by (rule finite-fri-challenge-space)
have finite-fibers:
   $\bigwedge \text{challenges. challenges} \in$ 
    fri-challenge-space (fri-round-count-for-degree-bound degree-bound)
   $\implies$ 
    finite
    (generic-fri-sampled-query-query-fiber low-degree candidate-bad
     degree-bound challenges)
by (rule finite-generic-fri-sampled-query-query-fiber)
have card-S:
  card ?S =
    ( $\sum \text{challenges} \in$ 
     fri-challenge-space (fri-round-count-for-degree-bound degree-bound).
     card
     (generic-fri-sampled-query-query-fiber low-degree candidate-bad
      degree-bound challenges))
by (rule card-SigmaI)
  (use finite-challenge-space finite-fibers in auto)
have card
  (generic-fri-sampled-query-restricted-bad-pairs low-degree
   candidate-bad degree-bound) =
  card (?swap ‘ ?S)
unfolding generic-fri-sampled-query-restricted-bad-pairs-as-fiber-Sigma
by simp
also have ... = card ?S
by (rule card-image[OF inj])
also have ... =
  ( $\sum \text{challenges} \in$ 
   fri-challenge-space (fri-round-count-for-degree-bound degree-bound).
   card
   (generic-fri-sampled-query-query-fiber low-degree candidate-bad
    degree-bound challenges))
by (rule card-S)
finally show ?thesis .
qed

lemma generic-fri-sampled-query-pair-fraction-bound-from-fiber-sum:
assumes fraction-bound:
  nnreal
  ( $\sum \text{challenges} \in$ 
   fri-challenge-space (fri-round-count-for-degree-bound degree-bound).
   card
   (generic-fri-sampled-query-query-fiber low-degree candidate-bad
    degree-bound challenges)) /
  nnreal
  (card fri-query-index-list-space *
   CARD('f)  $\wedge$  fri-round-count-for-degree-bound degree-bound)
   $\leq C$ 
shows

```

generic-fri-sampled-query-pair-fraction-bound low-degree candidate-bad degree-bound C
proof –
have *card-eq*:
card
(generic-fri-sampled-query-restricted-bad-pairs low-degree candidate-bad degree-bound) =
($\sum$ challenges $\in$
fri-challenge-space (fri-round-count-for-degree-bound degree-bound).
card
(generic-fri-sampled-query-query-fiber low-degree candidate-bad degree-bound challenges))
by (rule *generic-fri-sampled-query-restricted-bad-pairs-card-eq-fiber-sum*)
show *?thesis*
unfolding *generic-fri-sampled-query-pair-fraction-bound-def*
using *fraction-bound card-eq* **by** *simp*
qed

lemma *generic-fri-sampled-query-pair-fraction-bound-iff-fiber-sum*:
generic-fri-sampled-query-pair-fraction-bound low-degree candidate-bad degree-bound C
 $\longleftrightarrow$
nnreal
($\sum$ challenges $\in$
fri-challenge-space (fri-round-count-for-degree-bound degree-bound).
card
(generic-fri-sampled-query-query-fiber low-degree candidate-bad degree-bound challenges)) /
nnreal
*(card fri-query-index-list-space **
CARD('f) ^ fri-round-count-for-degree-bound degree-bound)
 $\leq C$
unfolding *generic-fri-sampled-query-pair-fraction-bound-def*
by (*simp add*:
generic-fri-sampled-query-restricted-bad-pairs-card-eq-fiber-sum)

lemma *trace-fri-sampled-query-restricted-bad-pairs-card-eq-fiber-sum*:
card
(trace-fri-sampled-query-bad-pair-union $\cap$
(fri-query-index-list-space $\times$ UNIV)) =
($\sum$ challenges $\in$
fri-challenge-space (fri-round-count-for-degree-bound (clength – 1)).
card
(generic-fri-sampled-query-query-fiber trace-table-low-degree
(Not $\circ$ trace-table-low-degree) (clength – 1) challenges))
proof –
have *generic*:
card
(generic-fri-sampled-query-restricted-bad-pairs trace-table-low-degree

$(\text{Not} \circ \text{trace-table-low-degree}) (\text{clength} - 1)) =$
 $(\sum \text{challenges} \in$
 $\text{fri-challenge-space} (\text{fri-round-count-for-degree-bound} (\text{clength} - 1)).$
 card
 $(\text{generic-fri-sampled-query-query-fiber trace-table-low-degree}$
 $(\text{Not} \circ \text{trace-table-low-degree}) (\text{clength} - 1) \text{ challenges}))$
by (rule generic-fri-sampled-query-restricted-bad-pairs-card-eq-fiber-sum)
then show ?thesis
unfolding trace-fri-sampled-query-bad-pair-union-def
 $\text{generic-fri-sampled-query-restricted-bad-pairs-def}$
by simp
qed

lemma composition-fri-sampled-query-restricted-bad-pairs-card-eq-fiber-sum:

card
 $(\text{composition-fri-sampled-query-bad-pair-union dg} \cap$
 $(\text{fri-query-index-list-space} \times \text{UNIV})) =$
 $(\sum \text{challenges} \in$
 $\text{fri-challenge-space} (\text{fri-round-count-for-degree-bound} (\text{to-nat dg})).$
 card
 $(\text{generic-fri-sampled-query-query-fiber}$
 $(\text{composition-table-low-degree} (\text{to-nat dg}))$
 $(\text{Not} \circ \text{composition-table-low-degree maxDegree}) (\text{to-nat dg})$
 $\text{challenges}))$

proof —

have generic:

card
 $(\text{generic-fri-sampled-query-restricted-bad-pairs}$
 $(\text{composition-table-low-degree} (\text{to-nat dg}))$
 $(\text{Not} \circ \text{composition-table-low-degree maxDegree}) (\text{to-nat dg})) =$
 $(\sum \text{challenges} \in$
 $\text{fri-challenge-space} (\text{fri-round-count-for-degree-bound} (\text{to-nat dg})).$
 card
 $(\text{generic-fri-sampled-query-query-fiber}$
 $(\text{composition-table-low-degree} (\text{to-nat dg}))$
 $(\text{Not} \circ \text{composition-table-low-degree maxDegree}) (\text{to-nat dg})$
 $\text{challenges}))$
by (rule generic-fri-sampled-query-restricted-bad-pairs-card-eq-fiber-sum)
then show ?thesis
unfolding composition-fri-sampled-query-bad-pair-union-def
 $\text{generic-fri-sampled-query-restricted-bad-pairs-def}$
by simp
qed

lemma trace-fri-sampled-query-pair-fraction-bound-iff-fiber-sum:

$\text{generic-fri-sampled-query-pair-fraction-bound trace-table-low-degree}$
 $(\text{Not} \circ \text{trace-table-low-degree}) (\text{clength} - 1) C$
 $\longleftrightarrow$
 nnreal

$(\sum \text{challenges} \in$
 $\text{fri-challenge-space } (\text{fri-round-count-for-degree-bound } (\text{clength} - 1)).$
 card
 $(\text{generic-fri-sampled-query-query-fiber trace-table-low-degree}$
 $(\text{Not} \circ \text{trace-table-low-degree}) (\text{clength} - 1) \text{ challenges})) /$
 nnreal
 $(\text{card fri-query-index-list-space} *$
 $\text{CARD('f')} \wedge \text{fri-round-count-for-degree-bound } (\text{clength} - 1))$
 $\leq C$
by (rule generic-fri-sampled-query-pair-fraction-bound-iff-fiber-sum)

lemma composition-fri-sampled-query-pair-fraction-bound-iff-fiber-sum:

$\text{generic-fri-sampled-query-pair-fraction-bound}$
 $(\text{composition-table-low-degree } (\text{to-nat } dg))$
 $(\text{Not} \circ \text{composition-table-low-degree } \text{maxDegree}) (\text{to-nat } dg) C$
 $\longleftrightarrow$
 nnreal
 $(\sum \text{challenges} \in$
 $\text{fri-challenge-space } (\text{fri-round-count-for-degree-bound } (\text{to-nat } dg)).$
 card
 $(\text{generic-fri-sampled-query-query-fiber}$
 $(\text{composition-table-low-degree } (\text{to-nat } dg))$
 $(\text{Not} \circ \text{composition-table-low-degree } \text{maxDegree}) (\text{to-nat } dg)$
 $\text{challenges})) /$
 nnreal
 $(\text{card fri-query-index-list-space} *$
 $\text{CARD('f')} \wedge \text{fri-round-count-for-degree-bound } (\text{to-nat } dg))$
 $\leq C$
by (rule generic-fri-sampled-query-pair-fraction-bound-iff-fiber-sum)

lemma trace-fri-query-index-list-set-hit-imp-query-rounds-any-index-set-hit:

assumes hit: trace-fri-query-index-list-set-hit s Q out

shows

$\text{query-rounds-any-index-set-hit } s (\text{query-index-list-entries } Q) \text{ rounds}$
 out

proof –

from hit **obtain** trace-roots trace-bs trace-final dg composition-roots
 $\text{composition-bs composition-final query-idxs trace-round-layers}$
 $\text{composition-round-layers}$ **where**

openings:

$\text{accepted-fri-opening-transcript } s \text{ out trace-roots trace-bs}$
 $\text{trace-final dg composition-roots composition-bs composition-final}$
 $\text{query-idxs trace-round-layers composition-round-layers}$

and query-in: query-idxs $\in Q$

unfolding trace-fri-query-index-list-set-hit-def **by** blast

from openings **obtain** result final-state query-start-state rest raw-idxs
 $\text{query-chunks trailing}$ **where**

out-eq: out = Some (result, final-state)

and len-raw: length raw-idxs = rounds

```

and query-idxs-eq: query-idxs = map ( $\lambda$ raw. index (to-nat raw)) raw-idxs
and lookup:
   $\wedge i. i < \text{rounds} \implies$ 
    fmlookup (HashMap final-state)
      (QueryIndexChallenge (PQueryCounter s + i)
        (state-after-query-chunks query-start-state query-chunks i)) =
        Some (raw-idxs ! i)
unfolding accepted-fri-opening-transcript-def
  verifier-query-indices-derived-def
by blast
have zero-bound: 0 < rounds
  by (rule rounds-positive)
have query-len: length query-idxs = rounds
  using query-idxs-eq len-raw by simp
have idx-in-entries:
  query-idxs ! 0  $\in$  query-index-list-entries Q
  using query-in zero-bound query-len
  unfolding query-index-list-entries-def
  by (auto intro: nth-mem)
have idx-eq: query-idxs ! 0 = index (to-nat (raw-idxs ! 0))
  using query-idxs-eq len-raw zero-bound by simp
show ?thesis
  unfolding query-rounds-any-index-set-hit-def
  apply (rule exI[where x=result])
  apply (rule exI[where x=final-state])
  apply (rule exI[where x=0])
  apply (rule exI[where x=state-after-query-chunks query-start-state
    query-chunks 0])
  apply (rule exI[where x=raw-idxs ! 0])
  using out-eq zero-bound lookup[OF zero-bound] idx-in-entries idx-eq
  by simp
qed

lemma composition-fri-query-index-list-set-hit-imp-query-rounds-any-index-set-hit:
  assumes hit: composition-fri-query-index-list-set-hit s Q out
  shows
    query-rounds-any-index-set-hit s (query-index-list-entries Q) rounds
    out
proof –
  from hit obtain trace-roots trace-bs trace-final dg composition-roots
    composition-bs composition-final query-idxs trace-round-layers
    composition-round-layers where
    openings:
      accepted-fri-opening-transcript s out trace-roots trace-bs
        trace-final dg composition-roots composition-bs composition-final
        query-idxs trace-round-layers composition-round-layers
  and query-in: query-idxs  $\in$  Q
  unfolding composition-fri-query-index-list-set-hit-def by blast
  from openings obtain result final-state query-start-state rest raw-idxs

```



```

    query-chunks trailing where
    out-eq: out = Some (result, final-state)
    and len-raw: length raw-idxs = rounds
    and query-idxs-eq: query-idxs = map ( $\lambda$ raw. index (to-nat raw)) raw-idxs
    and lookup:
       $\wedge i. i < rounds \implies$ 
        fmllookup (HashMap final-state)
          (QueryIndexChallenge (PQueryCounter s + i)
            (state-after-query-chunks query-start-state query-chunks i)) =
            Some (raw-idxs ! i)
    unfolding accepted-fri-opening-transcript-def
    verifier-query-indices-derived-def
    by blast
    have zero-bound:  $0 < rounds$ 
    by (rule rounds-positive)
    have query-len: length query-idxs = rounds
    using query-idxs-eq len-raw by simp
    have idx-in-entries:
      query-idxs ! 0  $\in$  query-index-list-entries Q
    using query-in zero-bound query-len
    unfolding query-index-list-entries-def
    by (auto intro: nth-mem)
    have idx-eq: query-idxs ! 0 = index (to-nat (raw-idxs ! 0))
    using query-idxs-eq len-raw zero-bound by simp
    show ?thesis
    unfolding query-rounds-any-index-set-hit-def
    apply (rule exI[where x=result])
    apply (rule exI[where x=final-state])
    apply (rule exI[where x=0])
    apply (rule exI[where x=state-after-query-chunks query-start-state
      query-chunks 0])
    apply (rule exI[where x=raw-idxs ! 0])
    using out-eq zero-bound lookup[OF zero-bound] idx-in-entries idx-eq
    by simp
qed

lemma wp-trace-fri-query-index-list-set-hit-bound-from-entries:
  assumes future: query-future-fresh s
  and raw-bound: query-index-raw-preimage-bound
  and subset:  $Q \subseteq \text{fri-query-index-list-space}$ 
shows
  wp-event verify-monad (trace-fri-query-index-list-set-hit s Q)  $s \leq$ 
    nnreal rounds *
    (nnreal (card (query-index-list-entries Q)) /
      nnreal (card query-sample-space))
proof –
  have event-bound:
    wp-event verify-monad
      (query-rounds-any-index-set-hit s (query-index-list-entries Q) rounds)

```

```

    s ≤
    nnreal rounds *
    (nnreal (card (query-index-list-entries Q)) /
     nnreal (card query-sample-space))
  by (rule wp-verify-monad-query-rounds-any-index-set-bound
      [OF future raw-bound
       query-index-list-entries-subset-query-sample-space[OF subset]])
  show ?thesis
  by (rule order-trans[OF - event-bound])
      (rule wp-event-mono,
       rule trace-fri-query-index-list-set-hit-imp-query-rounds-any-index-set-hit)
qed

lemma wp-composition-fri-query-index-list-set-hit-bound-from-entries:
  assumes future: query-future-fresh s
  and raw-bound: query-index-raw-preimage-bound
  and subset: Q ⊆ fri-query-index-list-space
  shows
    wp-event verify-monad (composition-fri-query-index-list-set-hit s Q) s ≤
    nnreal rounds *
    (nnreal (card (query-index-list-entries Q)) /
     nnreal (card query-sample-space))
proof -
  have event-bound:
    wp-event verify-monad
      (query-rounds-any-index-set-hit s (query-index-list-entries Q) rounds)
    s ≤
    nnreal rounds *
    (nnreal (card (query-index-list-entries Q)) /
     nnreal (card query-sample-space))
  by (rule wp-verify-monad-query-rounds-any-index-set-bound
      [OF future raw-bound
       query-index-list-entries-subset-query-sample-space[OF subset]])
  show ?thesis
  by (rule order-trans[OF - event-bound])
      (rule wp-event-mono,
       rule composition-fri-query-index-list-set-hit-imp-query-rounds-any-index-set-hit)
qed

lemma generic-fri-sampled-query-pair-fraction-bound-mono:
  assumes
    generic-fri-sampled-query-pair-fraction-bound low-degree candidate-bad
    degree-bound C
  and C ≤ D
  shows
    generic-fri-sampled-query-pair-fraction-bound low-degree candidate-bad
    degree-bound D
  using assms
  unfolding generic-fri-sampled-query-pair-fraction-bound-def

```

by (rule order-trans)

lemma *generic-fri-sampled-query-pair-fraction-bound-from-card:*

assumes *card-bound:*

card
 (*generic-fri-sampled-query-restricted-bad-pairs low-degree*
candidate-bad degree-bound) $\leq K$

and *fraction-bound:*

nnreal K /
nnreal
 (*card fri-query-index-list-space **
CARD('f) ^ fri-round-count-for-degree-bound degree-bound)
 $\leq C$

shows

generic-fri-sampled-query-pair-fraction-bound low-degree candidate-bad
degree-bound C

proof –

have *nnreal*

(*card*
 (*generic-fri-sampled-query-restricted-bad-pairs low-degree*
candidate-bad degree-bound)) /
nnreal
 (*card fri-query-index-list-space **
CARD('f) ^ fri-round-count-for-degree-bound degree-bound)

$\leq$

nnreal K /
nnreal
 (*card fri-query-index-list-space **
CARD('f) ^ fri-round-count-for-degree-bound degree-bound)

using *card-bound* **by** (rule *nnreal-nat-divide-right-mono*)

also have ... $\leq C$

by (rule *fraction-bound*)

finally show *?thesis*

unfolding *generic-fri-sampled-query-pair-fraction-bound-def* .

qed

lemma *generic-fri-sampled-query-restricted-bad-pairs-subset-Sigma:*

assumes *projection:*

generic-fri-sampled-query-challenge-projection low-degree candidate-bad
degree-bound $\subseteq B$

shows

generic-fri-sampled-query-restricted-bad-pairs low-degree candidate-bad
degree-bound

$\subseteq$

($\lambda(challenges, query-idxs). (query-idxs, challenges)$) ‘
 (*SIGMA challenges:B.*

generic-fri-sampled-query-query-fiber low-degree candidate-bad
degree-bound challenges)

proof

```

fix pair
assume pair-mem:
  pair ∈
    generic-fri-sampled-query-restricted-bad-pairs low-degree candidate-bad
    degree-bound
obtain query-idxs challenges where pair-eq: pair = (query-idxs, challenges)
  by (cases pair)
have challenges-in: challenges ∈ B
  using projection pair-mem pair-eq
  unfolding generic-fri-sampled-query-challenge-projection-def
  by force
have query-in:
  query-idxs ∈
    generic-fri-sampled-query-query-fiber low-degree candidate-bad
    degree-bound challenges
  using pair-mem pair-eq
  unfolding generic-fri-sampled-query-query-fiber-def
    fri-query-challenge-pair-query-fiber-def
  by simp
have (challenges, query-idxs) ∈
  (SIGMA challenges:B.
    generic-fri-sampled-query-query-fiber low-degree candidate-bad
    degree-bound challenges)
  using challenges-in query-in by simp
then show
  pair ∈
    ( $\lambda(challenges, query-idxs). (query-idxs, challenges)$ ) ‘
    (SIGMA challenges:B.
      generic-fri-sampled-query-query-fiber low-degree candidate-bad
      degree-bound challenges)
  using pair-eq by force
qed

lemma generic-fri-sampled-query-restricted-bad-pairs-card-le-from-fibers:
assumes finite-B: finite B
and projection:
  generic-fri-sampled-query-challenge-projection low-degree
  candidate-bad degree-bound  $\subseteq B$ 
and fiber-bound:
   $\bigwedge challenges. challenges \in B \implies$ 
    card
      (generic-fri-sampled-query-query-fiber low-degree candidate-bad
        degree-bound challenges)  $\leq K$ 
shows
  card
    (generic-fri-sampled-query-restricted-bad-pairs low-degree
      candidate-bad degree-bound)
   $\leq \text{card } B * K$ 
proof –

```

```

have subset:
  generic-fri-sampled-query-restricted-bad-pairs low-degree candidate-bad
  degree-bound
 $\subseteq$ 
  ( $\lambda(\text{challenges}, \text{query-idxs}). (\text{query-idxs}, \text{challenges})$ ) ‘
  (SIGMA challenges:B.
    generic-fri-sampled-query-query-fiber low-degree candidate-bad
    degree-bound challenges)
  by (rule generic-fri-sampled-query-restricted-bad-pairs-subset-Sigma
    [OF projection])
have finite-fibers:
   $\bigwedge \text{challenges}. \text{challenges} \in B \implies$ 
  finite
  (generic-fri-sampled-query-query-fiber low-degree candidate-bad
  degree-bound challenges)
  by (rule finite-generic-fri-sampled-query-query-fiber)
have card-sigma:
  card
  (SIGMA challenges:B.
    generic-fri-sampled-query-query-fiber low-degree candidate-bad
    degree-bound challenges)
   $\leq \text{card } B * K$ 
  using finite-B finite-fibers fiber-bound
proof (induction rule: finite-induct)
  case empty
  then show ?case by simp
next
  case (insert challenges B)
  have fiber-le:
  card
  (generic-fri-sampled-query-query-fiber low-degree candidate-bad
  degree-bound challenges)  $\leq K$ 
  by (rule insert.premis(2)) simp
have tail-le:
  card
  (SIGMA challenges:B.
    generic-fri-sampled-query-query-fiber low-degree candidate-bad
    degree-bound challenges)
   $\leq \text{card } B * K$ 
  by (rule insert.IH) (use insert.premis in auto)
have card
  (SIGMA challenges:insert challenges B.
    generic-fri-sampled-query-query-fiber low-degree candidate-bad
    degree-bound challenges)
  =
  card
  (generic-fri-sampled-query-query-fiber low-degree candidate-bad
  degree-bound challenges) +
  card

```

```

    (SIGMA challenges:B.
      generic-fri-sampled-query-query-fiber low-degree candidate-bad
      degree-bound challenges)
  using insert.hyps insert.premis by simp
also have ...  $\leq K + \text{card } B * K$ 
  using fiber-le tail-le by simp
also have ... = card (insert challenges B) * K
  using insert.hyps by simp
finally show ?case .
qed
have finite-sigma:
  finite
  (SIGMA challenges:B.
    generic-fri-sampled-query-query-fiber low-degree candidate-bad
    degree-bound challenges)
  by (rule finite-SigmaI[OF finite-B finite-fibers])
then have finite-swap:
  finite
  (( $\lambda(\text{challenges}, \text{query-idxs}). (\text{query-idxs}, \text{challenges})$ ) ‘
    (SIGMA challenges:B.
      generic-fri-sampled-query-query-fiber low-degree candidate-bad
      degree-bound challenges))
  by simp
have card
  (generic-fri-sampled-query-restricted-bad-pairs low-degree candidate-bad
    degree-bound)
 $\leq$ 
card
  (( $\lambda(\text{challenges}, \text{query-idxs}). (\text{query-idxs}, \text{challenges})$ ) ‘
    (SIGMA challenges:B.
      generic-fri-sampled-query-query-fiber low-degree candidate-bad
      degree-bound challenges))
  by (rule card-mono[OF finite-swap subset])
also have ...  $\leq$ 
card
  (SIGMA challenges:B.
    generic-fri-sampled-query-query-fiber low-degree candidate-bad
    degree-bound challenges)
  by (rule card-image-le[OF finite-sigma])
also have ...  $\leq \text{card } B * K$ 
  by (rule card-sigma)
finally show ?thesis .
qed

```

```

lemma generic-fri-sampled-query-pair-fraction-bound-from-projection-and-fibers:
  assumes finite-B: finite B
  and projection:
    generic-fri-sampled-query-challenge-projection low-degree
    candidate-bad degree-bound  $\subseteq B$ 

```

```

and fiber-bound:
   $\bigwedge \text{challenges. challenges} \in B \implies$ 
    card
      (generic-fri-sampled-query-query-fiber low-degree candidate-bad
       degree-bound challenges)  $\leq K$ 
and fraction-bound:
  nnreal (card B * K) /
    nnreal
      (card fri-query-index-list-space * CARD('f) ^ fri-round-count-for-degree-bound degree-bound)
     $\leq C$ 
shows
  generic-fri-sampled-query-pair-fraction-bound low-degree candidate-bad
  degree-bound C
proof (rule generic-fri-sampled-query-pair-fraction-bound-from-card)
show
  card
    (generic-fri-sampled-query-restricted-bad-pairs low-degree
     candidate-bad degree-bound)
   $\leq \text{card } B * K$ 
by (rule generic-fri-sampled-query-restricted-bad-pairs-card-le-from-fibers
      [OF finite-B projection fiber-bound])
show
  nnreal (card B * K) /
    nnreal
      (card fri-query-index-list-space * CARD('f) ^ fri-round-count-for-degree-bound degree-bound)
     $\leq C$ 
by (rule fraction-bound)
qed

end

end

```

```

theory Soundness-FRI-Query-Exact-Verifier
  imports Soundness-FRI-Query-Aware-Interface
begin

```

Exact verifier-level consumption of sampled query-index lists.

The query-aware interface defines the strict query-list events and proves exact bounds for the raw verifier query-round sequence. This narrow layer lifts those bounds through the verifier decomposition.

```

context soundness
begin

```

```

lemma query-rounds-index-list-hit-counter-eq:
  assumes counter: PQueryCounter t = PQueryCounter s

```

and *hit*: *query-rounds-index-list-hit s query-idxs n out*
shows *query-rounds-index-list-hit t query-idxs n out*
using *assms* **unfolding** *query-rounds-index-list-hit-def* **by** *auto*

lemma *wp-verifier-after-composition-fri-query-index-list-set-bound*:

assumes *future*: *query-future-fresh t*
and *counter*: *PQueryCounter t = PQueryCounter s*
and *raw-bound*: *query-index-raw-preimage-bound*
and *subset*: $Q \subseteq \text{fri-query-index-list-space}$

shows

wp-event (verifier-after-composition-fri header)
 $(\lambda \text{out. } \exists \text{ query-idxs} \in Q.$
 $\text{query-rounds-index-list-hit } s \text{ query-idxs rounds out}) \ t \leq$
 $\text{nnreal (card } Q) * (1 / \text{nnreal (card query-sample-space)}) \wedge \text{rounds}$

proof –

obtain *fr f-fl f-final as dg fl* **where** *header-eq*:

header = (fr, f-fl, f-final, as, dg, fl)

by *(cases header) auto*

have *after-eq*:

verifier-after-composition-fri header =
 $(\text{read} \gg=$
 $(\lambda \text{final.}$
 ntimes
 $(\text{verifier-query-round-program } fr \text{ f-fl f-final as fl final})$
 $\text{rounds}))$

unfolding *header-eq* *verifier-after-composition-fri-def* **by** *simp*

show *?thesis*

unfolding *after-eq*

proof *(rule wp-event-bind-bound-by-cont)*

show $\neg (\exists \text{ query-idxs} \in Q.$
 $\text{query-rounds-index-list-hit } s \text{ query-idxs rounds None})$
by *simp*

next

fix *final query-state*

assume *read-final*:

Some (final, query-state) ∈ set-dist (execute read t)

have *future-query*: *query-future-fresh query-state*

by *(rule read-preserves-query-future-fresh[OF future read-final])*

from *read-outcome[OF read-final]* **obtain** *rest* **where**

counter-query-t: *PQueryCounter query-state = PQueryCounter t*

by *blast*

have *counter-query*: *PQueryCounter query-state = PQueryCounter s*

using *counter-query-t counter* **by** *simp*

have *event-mono*:

wp-event
 $(\text{ntimes}$
 $(\text{verifier-query-round-program } fr \text{ f-fl f-final as fl final})$
 $\text{rounds})$


```

    (λout. ∃ query-idxs ∈ Q.
      query-rounds-index-list-hit s query-idxs rounds out)
    query-state ≤
wp-event
  (ntimes
    (verifier-query-round-program fr f-fl f-final as fl final)
    rounds)
  (λout. ∃ query-idxs ∈ Q.
    query-rounds-index-list-hit query-state query-idxs rounds out)
  query-state
proof (rule wp-event-mono)
  fix out
  assume ∃ query-idxs ∈ Q.
    query-rounds-index-list-hit s query-idxs rounds out
  then obtain query-idxs where
    query-in: query-idxs ∈ Q
  and hit: query-rounds-index-list-hit s query-idxs rounds out
  by blast
  have query-rounds-index-list-hit query-state query-idxs rounds out
  by (rule query-rounds-index-list-hit-counter-eq[OF counter-query hit])
  then show ∃ query-idxs ∈ Q.
    query-rounds-index-list-hit query-state query-idxs rounds out
  using query-in by blast
qed
also have ... ≤
  nnreal (card Q) *
  (1 / nnreal (card query-sample-space)) ^ rounds
by (rule wp-ntimes-verifier-query-round-program-fri-index-list-set-bound
  [OF future-query raw-bound subset])
finally show
  wp-event
  (ntimes
    (verifier-query-round-program fr f-fl f-final as fl final)
    rounds)
  (λout. ∃ query-idxs ∈ Q.
    query-rounds-index-list-hit s query-idxs rounds out)
  query-state ≤
  nnreal (card Q) *
  (1 / nnreal (card query-sample-space)) ^ rounds .
qed
qed

lemma wp-verify-monad-query-index-list-set-bound:
assumes future: query-future-fresh s
and raw-bound: query-index-raw-preimage-bound
and subset: Q ⊆ fri-query-index-list-space
shows
  wp-event verify-monad
  (λout. ∃ query-idxs ∈ Q.

```

```

      query-rounds-index-list-hit s query-idxs rounds out) s ≤
      nnreal (card Q) *
      (1 / nnreal (card query-sample-space)) ^ rounds
unfolding verify-monad-composition-fri-decomposition
proof (rule wp-event-bind-bound-by-cont)
  show ¬ (∃ query-idxs ∈ Q.
    query-rounds-index-list-hit s query-idxs rounds None)
  by simp
next
  fix header t
  assume prefix:
    Some (header, t) ∈
      set-dist (execute verifier-composition-fri-prefix s)
  obtain fr f-fl f-final as dg fl where header-eq:
    header = (fr, f-fl, f-final, as, dg, fl)
  by (cases header) auto
  have future-t: query-future-fresh t
  by (rule verifier-composition-fri-prefix-preserves-query-future-fresh
    [OF future prefix])
  have counter-t: PQueryCounter t = PQueryCounter s
  using verifier-composition-fri-prefix-outcome[OF prefix[unfolded header-eq]]
  by simp
  show
    wp-event (verifier-after-composition-fri header)
    (λout. ∃ query-idxs ∈ Q.
      query-rounds-index-list-hit s query-idxs rounds out) t ≤
      nnreal (card Q) *
      (1 / nnreal (card query-sample-space)) ^ rounds
  by (rule wp-verifier-after-composition-fri-query-index-list-set-bound
    [OF future-t counter-t raw-bound subset])
qed

end

end

```

```

theory Soundness-FRI-Query-Exact-Bounds
imports Soundness-FRI-Query-Exact-Verifier
begin

```

Exact sampled-query bounds for the FRI query-list events.

This layer connects the verifier-level query-index-list consumption theorem to the trace/composition sampled FRI events. It stays separate from the raw verifier decomposition to keep theory finalization predictable.

```

context soundness
begin

```

```

lemma wp-event-finite-union-bound-fri-exact:

```

```

fixes  $B :: 'i \Rightarrow \text{prob}$ 
assumes  $\text{finite: finite } I$ 
  and  $\text{bounds: } \bigwedge i. i \in I \implies \text{wp-event } m (E i) s \leq B i$ 
shows
   $\text{wp-event } m (\lambda \text{out}. \exists i \in I. E i \text{ out}) s \leq (\sum_{i \in I}. B i)$ 
using  $\text{finite bounds}$ 
proof ( $\text{induction } I$ )
  case  $\text{empty}$ 
  show  $?case$ 
    unfolding  $\text{wp-event-def wp-def dist-expect-def}$  by  $\text{simp}$ 
next
  case ( $\text{insert } i I$ )
  have  $\text{split:}$ 
     $\text{wp-event } m (\lambda \text{out}. \exists j \in \text{insert } i I. E j \text{ out}) s \leq$ 
     $\text{wp-event } m (E i) s +$ 
     $\text{wp-event } m (\lambda \text{out}. \exists j \in I. E j \text{ out}) s$ 
  proof  $-$ 
    have  $\text{wp-event } m (\lambda \text{out}. \exists j \in \text{insert } i I. E j \text{ out}) s \leq$ 
     $\text{wp-event } m (\lambda \text{out}. E i \text{ out} \vee (\exists j \in I. E j \text{ out})) s$ 
    by ( $\text{rule wp-event-mono}$ )  $\text{auto}$ 
    also have  $\dots \leq$ 
     $\text{wp-event } m (E i) s +$ 
     $\text{wp-event } m (\lambda \text{out}. \exists j \in I. E j \text{ out}) s$ 
    by ( $\text{rule wp-event-union-bound}$ )
    finally show  $?thesis .$ 
  qed
  have  $\text{tail:}$ 
     $\text{wp-event } m (\lambda \text{out}. \exists j \in I. E j \text{ out}) s \leq (\sum_{j \in I}. B j)$ 
    by ( $\text{rule insert.IH}$ ) ( $\text{use insert.premis in blast}$ )
  have  $\text{wp-event } m (E i) s +$ 
     $\text{wp-event } m (\lambda \text{out}. \exists j \in I. E j \text{ out}) s \leq$ 
     $B i + (\sum_{j \in I}. B j)$ 
    by ( $\text{intro add-mono tail insert.premis}$ )  $\text{simp}$ 
  also have  $\dots = (\sum_{j \in \text{insert } i I}. B j)$ 
    using  $\text{insert.hyps}$  by  $\text{simp}$ 
  finally show  $?case$ 
    by ( $\text{rule order-trans[OF split]}$ )
qed

lemma  $\text{weighted-query-fiber-sum-eq:}$ 
  fixes  $I :: 'i \text{ set}$  and  $f :: 'i \Rightarrow \text{nat}$ 
  assumes  $\text{fin: finite } I$ 
    and  $A\text{-pos: } 0 < A$ 
    and  $B\text{-pos: } 0 < B$ 
  shows
     $(\sum x \in I. (1 / \text{nnreal } A) * ((1 / \text{nnreal } B) ^ \text{rounds} * \text{nnreal } (f x))) =$ 
     $\text{nnreal } (\sum x \in I. f x) / (\text{nnreal } A * \text{nnreal } B ^ \text{rounds})$ 
proof  $-$ 

```

```

have sum-nnreal:
  ( $\sum x \in I. \text{nnreal } (f x)$ ) = nnreal ( $\sum x \in I. f x$ )
  using fin by induction simp-all
have
  ( $\sum x \in I.$ 
    ( $1 / \text{nnreal } A$ ) * (( $1 / \text{nnreal } B$ )  $\wedge^{\text{rounds}}$  * nnreal ( $f x$ ))) =
    ( $\sum x \in I. \text{nnreal } (f x) / (\text{nnreal } A * \text{nnreal } B \wedge^{\text{rounds}})$ )
  using A-pos B-pos
  by (intro sum.cong refl)
    (transfer, simp add: field-simps power-divide)
also have ... =
  ( $\sum x \in I. \text{nnreal } (f x)$ ) / (nnreal A * nnreal B  $\wedge^{\text{rounds}}$ )
  by (rule sum-divide-nnreal[OF fin])
also have ... =
  nnreal ( $\sum x \in I. f x$ ) / (nnreal A * nnreal B  $\wedge^{\text{rounds}}$ )
  by (simp add: sum-nnreal)
finally show ?thesis .
qed

lemma accepted-fri-opening-transcript-imp-query-rounds-index-list-hit:
  assumes openings:
    accepted-fri-opening-transcript s out trace-roots trace-bs trace-final
    dg composition-roots composition-bs composition-final query-idxs
    trace-round-layers composition-round-layers
  shows query-rounds-index-list-hit s query-idxs rounds out
proof –
  from openings obtain result final-state query-start-state rest raw-idxs
    query-chunks trailing where
    out-eq: out = Some (result, final-state)
    and len-raw: length raw-idxs = rounds
    and query-idxs-eq: query-idxs = map ( $\lambda \text{raw}. \text{index } (\text{to-nat raw})$ ) raw-idxs
    and lookup:
       $\bigwedge i. i < \text{rounds} \implies$ 
      fmlookup (HashMap final-state)
      (QueryIndexChallenge (PQueryCounter s + i)
        (state-after-query-chunks query-start-state query-chunks i)) =
      Some (raw-idxs ! i)
  unfolding accepted-fri-opening-transcript-def
    verifier-query-indices-derived-def
  by blast
show ?thesis
  unfolding query-rounds-index-list-hit-def
  apply (rule exI[where x=result])
  apply (rule exI[where x=final-state])
  apply (rule exI[where x=raw-idxs])
  using out-eq len-raw query-idxs-eq lookup
  by auto
qed

```

lemma *trace-fri-query-index-list-set-hit-imp-query-rounds-index-list-hit:*

assumes *hit: trace-fri-query-index-list-set-hit s Q out*

shows

$\exists \text{query-idxs} \in Q.$

query-rounds-index-list-hit s query-idxs rounds out

proof –

from *hit* **obtain** *trace-roots trace-bs trace-final dg composition-roots*

composition-bs composition-final query-idxs trace-round-layers

composition-round-layers **where**

openings:

accepted-fri-opening-transcript s out trace-roots trace-bs

trace-final dg composition-roots composition-bs composition-final

query-idxs trace-round-layers composition-round-layers

and *query-in: query-idxs $\in$ Q*

unfolding *trace-fri-query-index-list-set-hit-def* **by** *blast*

have *query-rounds-index-list-hit s query-idxs rounds out*

by (*rule accepted-fri-opening-transcript-imp-query-rounds-index-list-hit*
[*OF openings*])

then show *?thesis*

using *query-in* **by** *blast*

qed

lemma *composition-fri-query-index-list-set-hit-imp-query-rounds-index-list-hit:*

assumes *hit: composition-fri-query-index-list-set-hit s Q out*

shows

$\exists \text{query-idxs} \in Q.$

query-rounds-index-list-hit s query-idxs rounds out

proof –

from *hit* **obtain** *trace-roots trace-bs trace-final dg composition-roots*

composition-bs composition-final query-idxs trace-round-layers

composition-round-layers **where**

openings:

accepted-fri-opening-transcript s out trace-roots trace-bs

trace-final dg composition-roots composition-bs composition-final

query-idxs trace-round-layers composition-round-layers

and *query-in: query-idxs $\in$ Q*

unfolding *composition-fri-query-index-list-set-hit-def* **by** *blast*

have *query-rounds-index-list-hit s query-idxs rounds out*

by (*rule accepted-fri-opening-transcript-imp-query-rounds-index-list-hit*
[*OF openings*])

then show *?thesis*

using *query-in* **by** *blast*

qed

lemma *wp-trace-fri-query-index-list-set-hit-exact-bound:*

assumes *future: query-future-fresh s*

and *raw-bound: query-index-raw-preimage-bound*

and *subset: $Q \subseteq \text{fri-query-index-list-space}$*

shows

$wp\text{-}event\ verify\text{-}monad\ (trace\text{-}fri\text{-}query\text{-}index\text{-}list\text{-}set\text{-}hit\ s\ Q)\ s \leq$
 $nnreal\ (card\ Q) *$
 $(1 / nnreal\ (card\ query\text{-}sample\text{-}space)) \wedge rounds$

proof –

have *event-bound*:
 $wp\text{-}event\ verify\text{-}monad$
 $(\lambda out. \exists query\text{-}idxs \in Q.$
 $query\text{-}rounds\text{-}index\text{-}list\text{-}hit\ s\ query\text{-}idxs\ rounds\ out)\ s \leq$
 $nnreal\ (card\ Q) *$
 $(1 / nnreal\ (card\ query\text{-}sample\text{-}space)) \wedge rounds$

by (*rule* *wp-verify-monad-query-index-list-set-bound*
 $[OF\ future\ raw\text{-}bound\ subset]$)

show *?thesis*
by (*rule* *order-trans*[*OF* - *event-bound*])
 $(rule\ wp\text{-}event\text{-}mono,$
 $rule\ trace\text{-}fri\text{-}query\text{-}index\text{-}list\text{-}set\text{-}hit\text{-}imp\text{-}query\text{-}rounds\text{-}index\text{-}list\text{-}hit)$

qed

lemma *wp-composition-fri-query-index-list-set-hit-exact-bound*:

assumes *future: query-future-fresh s*
and *raw-bound: query-index-raw-preimage-bound*
and *subset: $Q \subseteq fri\text{-}query\text{-}index\text{-}list\text{-}space$*

shows

$wp\text{-}event\ verify\text{-}monad\ (composition\text{-}fri\text{-}query\text{-}index\text{-}list\text{-}set\text{-}hit\ s\ Q)\ s \leq$
 $nnreal\ (card\ Q) *$
 $(1 / nnreal\ (card\ query\text{-}sample\text{-}space)) \wedge rounds$

proof –

have *event-bound*:
 $wp\text{-}event\ verify\text{-}monad$
 $(\lambda out. \exists query\text{-}idxs \in Q.$
 $query\text{-}rounds\text{-}index\text{-}list\text{-}hit\ s\ query\text{-}idxs\ rounds\ out)\ s \leq$
 $nnreal\ (card\ Q) *$
 $(1 / nnreal\ (card\ query\text{-}sample\text{-}space)) \wedge rounds$

by (*rule* *wp-verify-monad-query-index-list-set-bound*
 $[OF\ future\ raw\text{-}bound\ subset]$)

show *?thesis*
by (*rule* *order-trans*[*OF* - *event-bound*])
 $(rule\ wp\text{-}event\text{-}mono,$
 $rule\ composition\text{-}fri\text{-}query\text{-}index\text{-}list\text{-}set\text{-}hit\text{-}imp\text{-}query\text{-}rounds\text{-}index\text{-}list\text{-}hit)$

qed

lemma *wp-trace-fri-query-challenge-pair-set-hit-bound-from-query-projection*:

assumes *future: query-future-fresh s*
and *raw-bound: query-index-raw-preimage-bound*
and *projection: fst ‘ $P \subseteq Q$*
and *subset: $Q \subseteq fri\text{-}query\text{-}index\text{-}list\text{-}space$*

shows

$wp\text{-}event\ verify\text{-}monad\ (trace\text{-}fri\text{-}query\text{-}challenge\text{-}pair\text{-}set\text{-}hit\ s\ P)\ s \leq$
 $nnreal\ (card\ Q) *$

$(1 / \text{nnreal} (\text{card } \text{query-sample-space})) \wedge \text{rounds}$
proof –
have *event-bound*:
 $\text{wp-event verify-monad } (\text{trace-fri-query-index-list-set-hit } s \ Q) \ s \leq$
 $\text{nnreal} (\text{card } Q) *$
 $(1 / \text{nnreal} (\text{card } \text{query-sample-space})) \wedge \text{rounds}$
by (rule *wp-trace-fri-query-index-list-set-hit-exact-bound*
 $[\text{OF future raw-bound subset}]$)
show ?thesis
by (rule *order-trans*[*OF - event-bound*])
(rule *wp-event-mono*,
rule *trace-fri-query-challenge-pair-set-hit-imp-query-index-list-set-hit*
 $[\text{OF - projection}]$)
qed

lemma
wp-composition-fri-query-challenge-pair-set-hit-bound-from-query-projection:
assumes *future: query-future-fresh s*
and *raw-bound: query-index-raw-preimage-bound*
and *projection: $\bigwedge dg. \text{fst } ' P \ dg \subseteq Q$*
and *subset: $Q \subseteq \text{fri-query-index-list-space}$*
shows
 $\text{wp-event verify-monad}$
 $(\text{composition-fri-query-challenge-pair-set-hit } s \ P) \ s \leq$
 $\text{nnreal} (\text{card } Q) *$
 $(1 / \text{nnreal} (\text{card } \text{query-sample-space})) \wedge \text{rounds}$
proof –
have *event-bound*:
 $\text{wp-event verify-monad } (\text{composition-fri-query-index-list-set-hit } s \ Q) \ s$
 $\leq$
 $\text{nnreal} (\text{card } Q) *$
 $(1 / \text{nnreal} (\text{card } \text{query-sample-space})) \wedge \text{rounds}$
by (rule *wp-composition-fri-query-index-list-set-hit-exact-bound*
 $[\text{OF future raw-bound subset}]$)
show ?thesis
by (rule *order-trans*[*OF - event-bound*])
(rule *wp-event-mono*,
rule *composition-fri-query-challenge-pair-set-hit-imp-query-index-list-set-hit*
 $[\text{OF - projection}]$)
qed

lemma *wp-trace-fri-sampled-query-pair-set-hit-bound-from-query-projection*:
assumes *future: query-future-fresh s*
and *raw-bound: query-index-raw-preimage-bound*
and *projection:*
 $\bigwedge \text{trace-table roots final round-layers layers.}$
 $\text{fst } ' R \ \text{trace-table roots final round-layers layers} \subseteq Q$
and *subset: $Q \subseteq \text{fri-query-index-list-space}$*
shows

$wp\text{-}event\ verify\text{-}monad\ (trace\text{-}fri\text{-}sampled\text{-}query\text{-}pair\text{-}set\text{-}hit\ R\ s)\ s \leq$
 $nnreal\ (card\ Q) *$
 $(1 / nnreal\ (card\ query\text{-}sample\text{-}space)) \wedge rounds$

proof –

have *event-bound*:

$wp\text{-}event\ verify\text{-}monad\ (trace\text{-}fri\text{-}query\text{-}index\text{-}list\text{-}set\text{-}hit\ s\ Q)\ s \leq$
 $nnreal\ (card\ Q) *$
 $(1 / nnreal\ (card\ query\text{-}sample\text{-}space)) \wedge rounds$

by (rule *wp-trace-fri-query-index-list-set-hit-exact-bound*
 $[OF\ future\ raw\text{-}bound\ subset]$)

show *?thesis*

by (rule *order-trans* $[OF - event\text{-}bound]$)
(rule *wp-event-mono*,
rule *trace-fri-sampled-query-pair-set-hit-imp-query-index-list-set-hit*
 $[OF - projection]$)

qed

lemma *wp-composition-fri-sampled-query-pair-set-hit-bound-from-query-projection*:

assumes *future: query-future-fresh s*
and *raw-bound: query-index-raw-preimage-bound*
and *projection:*
 $\bigwedge dg\ composition\text{-}table\ roots\ final\ round\text{-}layers\ layers.$
 $fst\ 'R\ dg\ composition\text{-}table\ roots\ final\ round\text{-}layers\ layers \subseteq Q$
and *subset: $Q \subseteq fri\text{-}query\text{-}index\text{-}list\text{-}space$*

shows

$wp\text{-}event\ verify\text{-}monad$
 $(composition\text{-}fri\text{-}sampled\text{-}query\text{-}pair\text{-}set\text{-}hit\ R\ s)\ s \leq$
 $nnreal\ (card\ Q) *$
 $(1 / nnreal\ (card\ query\text{-}sample\text{-}space)) \wedge rounds$

proof –

have *event-bound*:

$wp\text{-}event\ verify\text{-}monad\ (composition\text{-}fri\text{-}query\text{-}index\text{-}list\text{-}set\text{-}hit\ s\ Q)\ s$
 $\leq$
 $nnreal\ (card\ Q) *$
 $(1 / nnreal\ (card\ query\text{-}sample\text{-}space)) \wedge rounds$

by (rule *wp-composition-fri-query-index-list-set-hit-exact-bound*
 $[OF\ future\ raw\text{-}bound\ subset]$)

show *?thesis*

by (rule *order-trans* $[OF - event\text{-}bound]$)
(rule *wp-event-mono*,
rule *composition-fri-sampled-query-pair-set-hit-imp-query-index-list-set-hit*
 $[OF - projection]$)

qed

lemma *wp-verifier-after-trace-fri-query-index-list-set-bound*:

assumes *future: query-future-fresh t*
and *counter: PQueryCounter t = PQueryCounter s*
and *raw-bound: query-index-raw-preimage-bound*
and *subset: $Q \subseteq fri\text{-}query\text{-}index\text{-}list\text{-}space$*


```

shows
  wp-event (verifier-after-trace-fri header)
    (λout. ∃ query-idxs ∈ Q.
      query-rounds-index-list-hit s query-idxs rounds out) t ≤
      nnreal (card Q) *
      (1 / nnreal (card query-sample-space)) ^ rounds
proof -
  obtain fr f-fl where header-eq: header = (fr, f-fl)
  by (cases header) auto
  let ?E =
    λout. ∃ query-idxs ∈ Q.
      query-rounds-index-list-hit s query-idxs rounds out
  let ?C =
    nnreal (card Q) *
    (1 / nnreal (card query-sample-space)) ^ rounds
  have after-eq:
    verifier-after-trace-fri header =
      (read ≥=
        (λf-final.
          mmap (replicate (length spec) alpha-round) ≥=
            (λas. read ≥=
              (λdg. assert (to-nat dg ≤ maxDegree) ≥=
                (λ-. ntimes receive-composition-fri-commits
                  (ceil-log (to-nat dg + 1)) ≥=
                    (λfl. read ≥=
                      (λfinal.
                        ntimes
                          (verifier-query-round-program fr f-fl f-final as fl
                            final)
                        rounds))))))))))
  unfolding header-eq verifier-after-trace-fri-def by simp
  show ?thesis
  unfolding after-eq
  proof (rule wp-event-bind-bound-by-cont)
    show ¬ ?E None
    by simp
  next
    fix f-final s1
    assume read-f-final: Some (f-final, s1) ∈ set-dist (execute read t)
    have future-s1: query-future-fresh s1
    by (rule read-preserves-query-future-fresh[OF future read-f-final])
    from read-outcome[OF read-f-final] obtain rest1 where
      counter-s1-t: PQueryCounter s1 = PQueryCounter t
    by blast
    have counter-s1: PQueryCounter s1 = PQueryCounter s
    using counter-s1-t counter by simp
    show
      wp-event
        (mmap (replicate (length spec) alpha-round) ≥=

```

```

    (λas. read ≫=
      (λdg. assert (to-nat dg ≤ maxDegree) ≫=
        (λ-. ntimes receive-composition-fri-commits
          (ceil-log (to-nat dg + 1)) ≫=
            (λfl. read ≫=
              (λfinal.
                ntimes
                  (verifier-query-round-program fr f-fl f-final as fl
                    final)
                rounds)))))) ?E s1 ≤ ?C
  proof (rule wp-event-bind-bound-by-cont)
  show ¬ ?E None
  by simp
next
  fix as s2
  assume alpha-out:
    Some (as, s2) ∈
      set-dist
        (execute (mmap (replicate (length spec) alpha-round)) s1)
  have future-s2: query-future-fresh s2
  by (rule mmap-alpha-round-preserves-query-future-fresh
    [OF future-s1 alpha-out])
  have counter-s2-s1: PQueryCounter s2 = PQueryCounter s1
  using mmap-alpha-round-outcome[OF alpha-out] by simp
  have counter-s2: PQueryCounter s2 = PQueryCounter s
  using counter-s2-s1 counter-s1 by simp
  show
    wp-event
      (read ≫=
        (λdg. assert (to-nat dg ≤ maxDegree) ≫=
          (λ-. ntimes receive-composition-fri-commits
            (ceil-log (to-nat dg + 1)) ≫=
              (λfl. read ≫=
                (λfinal.
                  ntimes
                    (verifier-query-round-program fr f-fl f-final as fl
                      final)
                  rounds)))))) ?E s2 ≤ ?C
  proof (rule wp-event-bind-bound-by-cont)
  show ¬ ?E None
  by simp
next
  fix dg s3
  assume read-dg: Some (dg, s3) ∈ set-dist (execute read s2)
  have future-s3: query-future-fresh s3
  by (rule read-preserves-query-future-fresh[OF future-s2 read-dg])
  from read-outcome[OF read-dg] obtain rest3 where
    counter-s3-s2: PQueryCounter s3 = PQueryCounter s2
  by blast

```

```

have counter-s3: PQueryCounter s3 = PQueryCounter s
  using counter-s3-s2 counter-s2 by simp
show
  wp-event
    (assert (to-nat dg ≤ maxDegree) >>=
      (λ-. ntimes receive-composition-fri-commits
        (ceil-log (to-nat dg + 1)) >>=
          (λfl. read >>=
            (λfinal.
              ntimes
                (verifier-query-round-program fr f-fl f-final as fl
                  final)
              rounds)))) ?E s3 ≤ ?C
    )
proof (rule wp-event-bind-bound-by-cont)
  show ¬ ?E None
    by simp
next
  fix unit s4
  assume assert-out:
    Some (unit, s4) ∈
      set-dist (execute (assert (to-nat dg ≤ maxDegree)) s3)
  have s4-eq: s4 = s3
    using assert-out unfolding assert-def
    by (cases to-nat dg ≤ maxDegree) (auto simp: throw-no-outcome)
  have future-s4: query-future-fresh s4
    using future-s3 s4-eq by simp
  have counter-s4: PQueryCounter s4 = PQueryCounter s
    using counter-s3 s4-eq by simp
  show
    wp-event
      (ntimes receive-composition-fri-commits
        (ceil-log (to-nat dg + 1)) >>=
          (λfl. read >>=
            (λfinal.
              ntimes
                (verifier-query-round-program fr f-fl f-final as fl
                  final)
              rounds)))) ?E s4 ≤ ?C
    )
proof (rule wp-event-bind-bound-by-cont)
  show ¬ ?E None
    by simp
next
  fix fl s5
  assume comp-fri:
    Some (fl, s5) ∈
      set-dist
        (execute
          (ntimes receive-composition-fri-commits
            (ceil-log (to-nat dg + 1)))) s4)

```

```

have future-s5: query-future-fresh s5
  by (rule ntimes-receive-composition-fri-commits-preserves-query-future-fresh
    [OF comp-fri future-s4])
have counter-s5-s4: PQueryCounter s5 = PQueryCounter s4
  using ntimes-receive-composition-fri-commits-outcome[OF comp-fri]
  by simp
have counter-s5: PQueryCounter s5 = PQueryCounter s
  using counter-s5-s4 counter-s4 by simp
show
  wp-event
    (read  $\gg$ 
      ( $\lambda$ final.
        ntimes
          (verifier-query-round-program fr f-fl f-final as fl
            final)
          rounds)) ?E s5  $\leq$  ?C
proof (rule wp-event-bind-bound-by-cont)
  show  $\neg$  ?E None
    by simp
next
  fix final query-state
  assume read-final:
    Some (final, query-state)  $\in$  set-dist (execute read s5)
  have future-query: query-future-fresh query-state
    by (rule read-preserves-query-future-fresh
      [OF future-s5 read-final])
  from read-outcome[OF read-final] obtain rest where
    counter-query-s5:
      PQueryCounter query-state = PQueryCounter s5
    by blast
  have counter-query:
    PQueryCounter query-state = PQueryCounter s
    using counter-query-s5 counter-s5 by simp
  have event-mono:
    wp-event
      (ntimes
        (verifier-query-round-program fr f-fl f-final as fl
          final)
        rounds)
      ?E query-state  $\leq$ 
    wp-event
      (ntimes
        (verifier-query-round-program fr f-fl f-final as fl
          final)
        rounds)
      ( $\lambda$ out.  $\exists$  query-idxs  $\in$  Q.
        query-rounds-index-list-hit query-state query-idxs
          rounds out)
      query-state

```

```

proof (rule wp-event-mono)
  fix out
  assume  $\exists \text{ query-idxs} \in Q$ .
    query-rounds-index-list-hit s query-idxs rounds out
  then obtain query-idxs where
    query-in: query-idxs  $\in Q$ 
  and hit:
    query-rounds-index-list-hit s query-idxs rounds out
  by blast
have
  query-rounds-index-list-hit query-state query-idxs rounds
    out
  by (rule query-rounds-index-list-hit-counter-eq
    [OF counter-query hit])
then show  $\exists \text{ query-idxs} \in Q$ .
  query-rounds-index-list-hit query-state query-idxs rounds
    out
  using query-in by blast
qed
also have  $\dots \leq ?C$ 
  by (rule wp-ntimes-verifier-query-round-program-fri-index-list-set-bound
    [OF future-query raw-bound subset])
finally show
  wp-event
    (ntimes
      (verifier-query-round-program fr f-fl f-final as fl
        final)
      rounds)
  ?E query-state  $\leq ?C$  .
qed
qed
qed
qed
qed
qed
qed
qed

lemma verifier-trace-fri-prefix-preserves-query-future-fresh-exact:
  assumes future: query-future-fresh s
  and prefix:
    Some (header, t)  $\in$  set-dist (execute verifier-trace-fri-prefix s)
  shows query-future-fresh t
proof –
  obtain fr f-fl where header-eq: header = (fr, f-fl)
  by (cases header) auto
  from prefix[unfolded verifier-trace-fri-prefix-def header-eq]
  obtain s1 where
    read-fr: Some (fr, s1)  $\in$  set-dist (execute read s)
  and trace-fri:

```

```

    Some (f-fl, t) ∈
      set-dist
        (execute (ntimes receive-trace-fri-commits (ceil-log clength))
          s1)
    by (auto elim!: set-dist-bindE)
  have future-s1: query-future-fresh s1
    by (rule read-preserves-query-future-fresh[OF future read-fr])
  show ?thesis
    by (rule ntimes-receive-trace-fri-commits-preserves-query-future-fresh
      [OF trace-fri future-s1])
qed

lemma wp-trace-fri-query-challenge-pair-set-hit-fixed-challenge-bound:
  assumes trace-future: trace-fri-future-fresh s
    and query-future: query-future-fresh s
    and raw-bound: query-index-raw-preimage-bound
    and challenges-space:
      challenges ∈ fri-challenge-space (ceil-log clength)
    and projection:
      fst ‘ (P ∩ (UNIV × {challenges})) ⊆ Q
    and subset: Q ⊆ fri-query-index-list-space
  shows
    wp-event verify-monad
      (λout. trace-fri-challenge-list-set-hit s {challenges} out ∧
        trace-fri-query-challenge-pair-set-hit s P out) s ≤
      (1 / nnreal (CARD('f) ^ ceil-log clength)) *
      (nnreal (card Q) *
        (1 / nnreal (card query-sample-space)) ^ rounds)
  proof -
    let ?Hit =
      λout. trace-fri-challenge-list-set-hit s {challenges} out ∧
        trace-fri-query-challenge-pair-set-hit s P out
    let ?Head =
      λout. case out of
        None ⇒ False
        | Some ((-, f-fl), -) ⇒ map fst f-fl ∈ {challenges}
    let ?D =
      nnreal (card Q) *
      (1 / nnreal (card query-sample-space)) ^ rounds
    have bind-bound:
      wp-event (verifier-trace-fri-prefix ≫ verifier-after-trace-fri)
        ?Hit s ≤ wp-event verifier-trace-fri-prefix ?Head s * ?D
    proof (rule wp-event-bind-bound-by-head-and-cont)
      show ¬ ?Hit None
        unfolding trace-fri-challenge-list-set-hit-def
          trace-fri-query-challenge-pair-set-hit-def
          accepted-fri-opening-transcript-def
        by blast
    next

```

```

fix header prefix-state
assume prefix:
  Some (header, prefix-state) ∈
    set-dist (execute verifier-trace-fri-prefix s)
and not-head:
  ¬ (case Some (header, prefix-state) of None ⇒ False
    | Some (-, f-fl), -) ⇒ map fst f-fl ∈ {challenges})
obtain fr f-fl where header-eq: header = (fr, f-fl)
  by (cases header) auto
have neg: map fst f-fl ≠ challenges
  using not-head unfolding header-eq by simp
have zero-le:
  wp-event (verifier-after-trace-fri header)
    (λout. trace-fri-challenge-list-set-hit s {challenges} out ∧
      trace-fri-query-challenge-pair-set-hit s P out) prefix-state
    ≤ 0
proof (rule order-trans)
show
  wp-event (verifier-after-trace-fri header)
    (λout. trace-fri-challenge-list-set-hit s {challenges} out ∧
      trace-fri-query-challenge-pair-set-hit s P out) prefix-state
    ≤
  wp-event (verifier-after-trace-fri header)
    (λ-. False) prefix-state
proof (rule wp-event-mono-on-support)
fix out
assume suffix:
  out ∈
    set-dist (execute (verifier-after-trace-fri header) prefix-state)
and hit:
  trace-fri-challenge-list-set-hit s {challenges} out ∧
  trace-fri-query-challenge-pair-set-hit s P out
from hit obtain trace-bs fri-dg composition-bs where
  challenges-hit:
    accepted-fri-challenges s out trace-bs fri-dg composition-bs
and trace-in: trace-bs ∈ {challenges}
unfolding trace-fri-challenge-list-set-hit-def
by blast
from challenges-hit obtain result final-state where out-eq:
  out = Some (result, final-state)
unfolding accepted-fri-challenges-def by blast
have prefix':
  Some ((fr, f-fl), prefix-state) ∈
    set-dist (execute verifier-trace-fri-prefix s)
using prefix header-eq by simp
have suffix':
  Some (result, final-state) ∈
    set-dist
      (execute (verifier-after-trace-fri (fr, f-fl)) prefix-state)

```

```

    using suffix header-eq out-eq by simp
  from verifier-after-trace-fri-accepted-fri-challenges[OF prefix' suffix']
  obtain dg' comp-bs' where challenges-prefix:
    accepted-fri-challenges s (Some (result, final-state))
    (map fst f-fl) dg' comp-bs'
  by blast
  have trace-eq: trace-bs = map fst f-fl
  using accepted-fri-challenges-unique
    [OF challenges-prefix challenges-hit[unfolded out-eq]]
  by simp
  show False
  using neq trace-eq trace-in by simp
qed
next
show
  wp-event (verifier-after-trace-fri header)
  (λ-. False) prefix-state ≤ 0
  unfolding wp-event-def wp-def dist-expect-def by simp
qed
show
  wp-event (verifier-after-trace-fri header)
  (λout. trace-fri-challenge-list-set-hit s {challenges} out ∧
    trace-fri-query-challenge-pair-set-hit s P out) prefix-state = 0
  by (rule antisym[OF zero-le]) simp
next
fix header prefix-state
assume prefix:
  Some (header, prefix-state) ∈
    set-dist (execute verifier-trace-fri-prefix s)
and head:
  (case Some (header, prefix-state) of None ⇒ False
   | Some ((-, f-fl), -) ⇒ map fst f-fl ∈ {challenges})
obtain fr f-fl where header-eq: header = (fr, f-fl)
  by (cases header) auto
have f-fl-eq: map fst f-fl = challenges
  using head unfolding header-eq by simp
have future-prefix: query-future-fresh prefix-state
  by (rule verifier-trace-fri-prefix-preserves-query-future-fresh-exact
    [OF query-future prefix])
have counter-prefix: PQueryCounter prefix-state = PQueryCounter s
  using verifier-trace-fri-prefix-outcome[OF prefix[unfolded header-eq]]
  by simp
have event-bound:
  wp-event (verifier-after-trace-fri header)
  (λout. ∃ query-idxs ∈ Q.
    query-rounds-index-list-hit s query-idxs rounds out)
  prefix-state ≤ ?D
  by (rule wp-verifier-after-trace-fri-query-index-list-set-bound
    [OF future-prefix counter-prefix raw-bound subset])

```


have *hit-to-query*:

$$\begin{aligned} & wp\text{-}event \ (verifier\text{-}after\text{-}trace\text{-}fri \ header) \\ & \ (\lambda out. \ trace\text{-}fri\text{-}challenge\text{-}list\text{-}set\text{-}hit \ s \ \{challenges\} \ out \wedge \\ & \ \ \ \ trace\text{-}fri\text{-}query\text{-}challenge\text{-}pair\text{-}set\text{-}hit \ s \ P \ out) \ prefix\text{-}state \\ & \leq \\ & wp\text{-}event \ (verifier\text{-}after\text{-}trace\text{-}fri \ header) \\ & \ (\lambda out. \ \exists \ query\text{-}idxs \in Q. \\ & \ \ \ \ query\text{-}rounds\text{-}index\text{-}list\text{-}hit \ s \ query\text{-}idxs \ rounds \ out) \ prefix\text{-}state \end{aligned}$$
proof (rule *wp-event-mono-on-support*)
fix *out*
assume *suffix*:

$$out \in$$

$$set\text{-}dist \ (execute \ (verifier\text{-}after\text{-}trace\text{-}fri \ header) \ prefix\text{-}state)$$
and *hit*:

$$\begin{aligned} & trace\text{-}fri\text{-}challenge\text{-}list\text{-}set\text{-}hit \ s \ \{challenges\} \ out \wedge \\ & trace\text{-}fri\text{-}query\text{-}challenge\text{-}pair\text{-}set\text{-}hit \ s \ P \ out \end{aligned}$$
from *hit* **obtain** *trace-roots trace-bs trace-final fri-dg*

$$\begin{aligned} & composition\text{-}roots \ composition\text{-}bs \ composition\text{-}final \ query\text{-}idxs \\ & trace\text{-}round\text{-}layers \ composition\text{-}round\text{-}layers \ \mathbf{where} \end{aligned}$$
openings:

$$\begin{aligned} & accepted\text{-}fri\text{-}opening\text{-}transcript \ s \ out \ trace\text{-}roots \ trace\text{-}bs \\ & trace\text{-}final \ fri\text{-}dg \ composition\text{-}roots \ composition\text{-}bs \\ & composition\text{-}final \ query\text{-}idxs \ trace\text{-}round\text{-}layers \\ & composition\text{-}round\text{-}layers \end{aligned}$$
and *pair*: $(query\text{-}idxs, \ trace\text{-}bs) \in P$
unfolding *trace-fri-query-challenge-pair-set-hit-def* **by** *blast*
from *openings* **obtain** *result final-state* **where** *out-eq*:

$$out = Some \ (result, \ final\text{-}state)$$
unfolding *accepted-fri-opening-transcript-def* **by** *blast*
have *prefix'*:

$$\begin{aligned} & Some \ ((fr, \ f\text{-}fl), \ prefix\text{-}state) \in \\ & set\text{-}dist \ (execute \ verifier\text{-}trace\text{-}fri\text{-}prefix \ s) \end{aligned}$$
using *prefix header-eq* **by** *simp*
have *suffix'*:

$$\begin{aligned} & Some \ (result, \ final\text{-}state) \in \\ & set\text{-}dist \\ & \ (execute \ (verifier\text{-}after\text{-}trace\text{-}fri \ (fr, \ f\text{-}fl)) \ prefix\text{-}state) \end{aligned}$$
using *suffix header-eq out-eq* **by** *simp*
from *verifier-after-trace-fri-accepted-fri-challenges*[*OF prefix' suffix'*]
obtain *dg' comp-bs'* **where** *challenges-prefix*:

$$\begin{aligned} & accepted\text{-}fri\text{-}challenges \ s \ (Some \ (result, \ final\text{-}state)) \\ & \ (map \ fst \ f\text{-}fl) \ dg' \ comp\text{-}bs' \end{aligned}$$
by *blast*
have *challenges-openings*:

$$\begin{aligned} & accepted\text{-}fri\text{-}challenges \ s \ out \ trace\text{-}bs \ fri\text{-}dg \ composition\text{-}bs \\ & \ \mathbf{using} \ openings \\ & \ \mathbf{unfolding} \ accepted\text{-}fri\text{-}opening\text{-}transcript\text{-}def \\ & \ \mathbf{apply} \ (elim \ exE \ conjE) \\ & \ \mathbf{apply} \ assumption \end{aligned}$$

```

done
have trace-eq: trace-bs = map fst f-fl
using accepted-fri-challenges-unique
[OF challenges-prefix challenges-openings[unfolded out-eq]]
by simp
have query-in: query-idxs ∈ Q
proof -
  have query-idxs ∈ fst ‘ (P ∩ (UNIV × {challenges}))
    using pair trace-eq f-fl-eq by force
  then show ?thesis
    using projection by blast
qed
have query-hit:
  query-rounds-index-list-hit s query-idxs rounds out
  by (rule accepted-fri-opening-transcript-imp-query-rounds-index-list-hit
    [OF openings])
show ∃ query-idxs ∈ Q.
  query-rounds-index-list-hit s query-idxs rounds out
  using query-in query-hit by blast
qed
show
  wp-event (verifier-after-trace-fri header)
  (λout. trace-fri-challenge-list-set-hit s {challenges} out ∧
    trace-fri-query-challenge-pair-set-hit s P out) prefix-state
  ≤ ?D
  by (rule order-trans[OF hit-to-query event-bound])
qed
have head-bound:
  wp-event verifier-trace-fri-prefix ?Head s ≤
    1 / nnreal (CARD('f) ^ ceil-log clength)
proof -
  have wp-event verifier-trace-fri-prefix ?Head s ≤
    nnreal (card {challenges}) / nnreal (CARD('f) ^ ceil-log clength)
  by (rule wp-verifier-trace-fri-prefix-challenge-space-set-bound
    [where B = {challenges}, OF trace-future])
    (use challenges-space in auto)
  then show ?thesis by simp
qed
show ?thesis
  unfolding verify-monad-trace-fri-decomposition
  apply (rule order-trans)
  apply (rule bind-bound)
  apply (rule mult-right-mono[OF head-bound])
  apply simp
done
qed

lemma wp-trace-fri-query-challenge-pair-set-hit-bound-from-query-fiber-sum:
  assumes trace-future: trace-fri-future-fresh s

```

```

and query-future: query-future-fresh s
and raw-bound: query-index-raw-preimage-bound
and projection:
   $\bigwedge \text{challenges.}$ 
   $\text{fst } ' (P \cap (UNIV \times \{\text{challenges}\})) \subseteq$ 
   $Q \text{ challenges}$ 
and subset:  $\bigwedge \text{challenges. } Q \text{ challenges} \subseteq \text{fri-query-index-list-space}$ 
shows
  wp-event verify-monad
   $(\text{trace-fri-query-challenge-pair-set-hit } s \ P) \ s \leq$ 
   $(\sum \text{challenges} \in \text{fri-challenge-space } (\text{ceil-log } \text{clength}).$ 
   $(1 / \text{nnreal } (\text{CARD}('f) \wedge \text{ceil-log } \text{clength})) *$ 
   $(\text{nnreal } (\text{card } (Q \text{ challenges})) *$ 
   $(1 / \text{nnreal } (\text{card } \text{query-sample-space})) \wedge \text{rounds}))$ 
proof –
  let ?I = fri-challenge-space (ceil-log clength)
  let ?E =
     $\lambda \text{challenges out.}$ 
     $\text{trace-fri-challenge-list-set-hit } s \ \{\text{challenges}\} \ \text{out} \wedge$ 
     $\text{trace-fri-query-challenge-pair-set-hit } s \ P \ \text{out}$ 
  let ?B =
     $\lambda \text{challenges.}$ 
     $(1 / \text{nnreal } (\text{CARD}('f) \wedge \text{ceil-log } \text{clength})) *$ 
     $(\text{nnreal } (\text{card } (Q \text{ challenges})) *$ 
     $(1 / \text{nnreal } (\text{card } \text{query-sample-space})) \wedge \text{rounds})$ 
  have split:
    wp-event verify-monad
     $(\text{trace-fri-query-challenge-pair-set-hit } s \ P) \ s \leq$ 
    wp-event verify-monad
     $(\lambda \text{out. } \exists \text{challenges} \in ?I. ?E \text{ challenges out}) \ s$ 
  proof (rule wp-event-mono)
  fix out
  assume hit: trace-fri-query-challenge-pair-set-hit s P out
  from hit obtain trace-roots trace-bs trace-final fri-dg
    composition-roots composition-bs composition-final query-idxs
    trace-round-layers composition-round-layers where
  openings:
    accepted-fri-opening-transcript s out trace-roots trace-bs
    trace-final fri-dg composition-roots composition-bs
    composition-final query-idxs trace-round-layers
    composition-round-layers
  unfolding trace-fri-query-challenge-pair-set-hit-def by blast
  have challenges:
    accepted-fri-challenges s out trace-bs fri-dg composition-bs
  using openings
  unfolding accepted-fri-opening-transcript-def
  apply (elim exE conjE)
  apply assumption
  done

```

```

have trace-space: trace-bs  $\in$  ?I
  by (rule accepted-fri-challenges-trace-space[OF challenges])
have challenge-hit:
  trace-fri-challenge-list-set-hit s {trace-bs} out
  unfolding trace-fri-challenge-list-set-hit-def
  using challenges by blast
show  $\exists$  challenges  $\in$  ?I. ?E challenges out
  using trace-space challenge-hit hit by blast
qed
also have ...  $\leq$  ( $\sum$  challenges  $\in$  ?I. ?B challenges)
proof (rule wp-event-finite-union-bound-fri-exact)
  show finite ?I
  by (rule finite-fri-challenge-space)
next
  fix challenges
  assume challenges: challenges  $\in$  ?I
  show
    wp-event verify-monad (?E challenges) s  $\leq$  ?B challenges
  by (rule wp-trace-fri-query-challenge-pair-set-hit-fixed-challenge-bound
    [OF trace-future query-future raw-bound challenges
      projection subset])
qed
finally show ?thesis .
qed

lemma wp-trace-fri-query-challenge-pair-set-hit-bound-from-query-fibers:
  fixes C :: prob
  assumes future: query-future-fresh s
  and raw-bound: query-index-raw-preimage-bound
  and projection:
     $\bigwedge$  challenges.
    fst ‘ (P  $\cap$  (UNIV  $\times$  {challenges}))  $\subseteq$ 
      Q challenges
  and subset:  $\bigwedge$  challenges. Q challenges  $\subseteq$  fri-query-index-list-space
  and bound:
     $\bigwedge$  challenges.
    nnreal (card (Q challenges)) *
      (1 / nnreal (card query-sample-space))  $\wedge$  rounds  $\leq$  C
  shows
    wp-event verify-monad
      (trace-fri-query-challenge-pair-set-hit s P) s  $\leq$  C
  unfolding verify-monad-trace-fri-decomposition
proof (rule wp-event-bind-bound-by-cont)
  show  $\neg$  trace-fri-query-challenge-pair-set-hit s P None
  unfolding trace-fri-query-challenge-pair-set-hit-def
    accepted-fri-opening-transcript-def
  by blast
next
  fix header prefix-state

```

```

assume prefix:
  Some (header, prefix-state) ∈
    set-dist (execute verifier-trace-fri-prefix s)
obtain fr f-fl where header-eq: header = (fr, f-fl)
  by (cases header) auto
have future-prefix: query-future-fresh prefix-state
  by (rule verifier-trace-fri-prefix-preserves-query-future-fresh-exact
    [OF future prefix])
have counter-prefix: PQueryCounter prefix-state = PQueryCounter s
  using verifier-trace-fri-prefix-outcome[OF prefix[unfolded header-eq]]
  by simp
let ?Q = Q (map fst f-fl)
have event-bound:
  wp-event (verifier-after-trace-fri header)
    (λout. ∃ query-idxs ∈ ?Q.
      query-rounds-index-list-hit s query-idxs rounds out)
    prefix-state ≤
      nnreal (card ?Q) *
        (1 / nnreal (card query-sample-space)) ^ rounds
  by (rule wp-verifier-after-trace-fri-query-index-list-set-bound
    [OF future-prefix counter-prefix raw-bound subset])
have pair-to-query:
  wp-event (verifier-after-trace-fri header)
    (trace-fri-query-challenge-pair-set-hit s P) prefix-state ≤
  wp-event (verifier-after-trace-fri header)
    (λout. ∃ query-idxs ∈ ?Q.
      query-rounds-index-list-hit s query-idxs rounds out) prefix-state
proof (rule wp-event-mono-on-support)
fix out
assume suffix:
  out ∈
    set-dist (execute (verifier-after-trace-fri header) prefix-state)
  and hit: trace-fri-query-challenge-pair-set-hit s P out
from hit obtain trace-roots trace-bs trace-final fri-dg
  composition-roots composition-bs composition-final query-idxs
  trace-round-layers composition-round-layers where
openings:
  accepted-fri-opening-transcript s out trace-roots trace-bs
  trace-final fri-dg composition-roots composition-bs
  composition-final query-idxs trace-round-layers
  composition-round-layers
and pair: (query-idxs, trace-bs) ∈ P
unfolding trace-fri-query-challenge-pair-set-hit-def by blast
from openings obtain result final-state where out-eq:
  out = Some (result, final-state)
unfolding accepted-fri-opening-transcript-def by blast
have prefix':
  Some ((fr, f-fl), prefix-state) ∈
    set-dist (execute verifier-trace-fri-prefix s)

```

```

    using prefix header-eq by simp
have suffix':
  Some (result, final-state) ∈
    set-dist
      (execute (verifier-after-trace-fri (fr, f-fl)) prefix-state)
    using suffix header-eq out-eq by simp
from verifier-after-trace-fri-accepted-fri-challenges[OF prefix' suffix']
obtain dg' comp-bs' where challenges-prefix:
  accepted-fri-challenges s (Some (result, final-state))
    (map fst f-fl) dg' comp-bs'
  by blast
have challenges-openings:
  accepted-fri-challenges s out trace-bs fri-dg composition-bs
  using openings
  unfolding accepted-fri-opening-transcript-def
  apply (elim exE conjE)
  apply assumption
  done
have trace-eq: trace-bs = map fst f-fl
  using accepted-fri-challenges-unique
    [OF challenges-prefix challenges-openings[unfolded out-eq]]
  by simp
have query-in: query-idxs ∈ ?Q
proof -
  have query-idxs ∈
    fst ' (P ∩ (UNIV × {map fst f-fl}))
    using pair trace-eq by force
  then show ?thesis
    using projection[of map fst f-fl] by blast
qed
have query-hit:
  query-rounds-index-list-hit s query-idxs rounds out
  by (rule accepted-fri-opening-transcript-imp-query-rounds-index-list-hit
    [OF openings])
show ∃ query-idxs ∈ ?Q.
  query-rounds-index-list-hit s query-idxs rounds out
  using query-in query-hit by blast
qed
show
  wp-event (verifier-after-trace-fri header)
    (trace-fri-query-challenge-pair-set-hit s P) prefix-state ≤ C
  by (rule order-trans[OF pair-to-query])
    (rule order-trans[OF event-bound bound])
qed

lemma wp-trace-fri-query-challenge-pair-set-hit-bound-from-challenge-cover-and-query-fibers:
  fixes C-challenge C-query :: prob
  assumes trace-future: trace-fri-future-fresh s
  and query-future: query-future-fresh s

```

and *raw-bound*: *query-index-raw-preimage-bound*
and *challenge-subset*: $B \subseteq \text{fri-challenge-space } (\text{ceil-log } \text{clength})$
and *challenge-bound*:
 $\text{nnreal } (\text{card } B) / \text{nnreal } (\text{CARD}(f) \wedge \text{ceil-log } \text{clength}) \leq$
 $C\text{-challenge}$
and *projection*:
 $\bigwedge \text{challenges.}$
 $\text{fst } ' ((P \cap (\text{UNIV} \times (- B))) \cap$
 $(\text{UNIV} \times \{\text{challenges}\})) \subseteq Q \text{ challenges}$
and *subset*: $\bigwedge \text{challenges. } Q \text{ challenges} \subseteq \text{fri-query-index-list-space}$
and *query-bound*:
 $\bigwedge \text{challenges.}$
 $\text{nnreal } (\text{card } (Q \text{ challenges})) *$
 $(1 / \text{nnreal } (\text{card } \text{query-sample-space})) \wedge \text{rounds} \leq C\text{-query}$
shows
 $\text{wp-event verify-monad}$
 $(\text{trace-fri-query-challenge-pair-set-hit } s P) s \leq$
 $C\text{-challenge} + C\text{-query}$
proof –
let $?Outside = P \cap (\text{UNIV} \times (- B))$
have *split*:
 $\text{wp-event verify-monad}$
 $(\text{trace-fri-query-challenge-pair-set-hit } s P) s \leq$
 $\text{wp-event verify-monad}$
 $(\lambda \text{out. trace-fri-challenge-list-set-hit } s B \text{ out} \vee$
 $\text{trace-fri-query-challenge-pair-set-hit } s ?Outside \text{ out}) s$
proof (rule *wp-event-mono*)
fix *out*
assume *hit*: $\text{trace-fri-query-challenge-pair-set-hit } s P \text{ out}$
from *hit* **obtain** *trace-roots trace-bs trace-final fri-dg*
 $\text{composition-roots composition-bs composition-final query-idxs}$
 $\text{trace-round-layers composition-round-layers}$ **where**
openings:
 $\text{accepted-fri-opening-transcript } s \text{ out trace-roots trace-bs}$
 $\text{trace-final fri-dg composition-roots composition-bs}$
 $\text{composition-final query-idxs trace-round-layers}$
 $\text{composition-round-layers}$
and *pair*: $(\text{query-idxs}, \text{trace-bs}) \in P$
unfolding *trace-fri-query-challenge-pair-set-hit-def* **by** *blast*
show
 $\text{trace-fri-challenge-list-set-hit } s B \text{ out} \vee$
 $\text{trace-fri-query-challenge-pair-set-hit } s ?Outside \text{ out}$
proof (cases $\text{trace-bs} \in B$)
case *True*
have *challenges*:
 $\text{accepted-fri-challenges } s \text{ out trace-bs fri-dg composition-bs}$
using *openings*
unfolding *accepted-fri-opening-transcript-def*
apply (*elim exE conjE*)

```

    apply assumption
  done
then have trace-fri-challenge-list-set-hit s B out
  using True unfolding trace-fri-challenge-list-set-hit-def by blast
then show ?thesis by simp
next
case False
have (query-idxs, trace-bs) ∈ ?Outside
  using pair False by simp
then have trace-fri-query-challenge-pair-set-hit s ?Outside out
  unfolding trace-fri-query-challenge-pair-set-hit-def
  using openings by blast
then show ?thesis by simp
qed
qed
also have ... ≤
  wp-event verify-monad (trace-fri-challenge-list-set-hit s B) s +
  wp-event verify-monad
    (trace-fri-query-challenge-pair-set-hit s ?Outside) s
  by (rule wp-event-union-bound)
also have ... ≤ C-challenge + C-query
proof (rule add-mono)
  show wp-event verify-monad (trace-fri-challenge-list-set-hit s B) s
    ≤ C-challenge
  by (rule order-trans[
    OF wp-verify-monad-trace-fri-challenge-list-set-bound
    [OF trace-future challenge-subset] challenge-bound])
  show
    wp-event verify-monad
      (trace-fri-query-challenge-pair-set-hit s ?Outside) s ≤
      C-query
  by (rule wp-trace-fri-query-challenge-pair-set-hit-bound-from-query-fibers
    [OF query-future raw-bound projection subset query-bound])
qed
finally show ?thesis .
qed

lemma
  wp-composition-fri-query-challenge-pair-set-hit-bound-from-query-fibers:
  fixes C :: prob
  assumes future: query-future-fresh s
  and raw-bound: query-index-raw-preimage-bound
  and projection:
     $\bigwedge dg \text{ challenges. } fst \text{ ' } (P \ dg \cap (UNIV \times \{challenges\})) \subseteq Q \ dg \text{ challenges}$ 
  and subset:  $\bigwedge dg \text{ challenges. } Q \ dg \text{ challenges} \subseteq fri\text{-}query\text{-}index\text{-}list\text{-}space$ 
  and bound:
     $\bigwedge dg \text{ challenges. }$ 

```



```

      nnreal (card (Q dg challenges)) *
      (1 / nnreal (card query-sample-space)) ^ rounds ≤ C
shows
  wp-event verify-monad
    (composition-fri-query-challenge-pair-set-hit s P) s ≤ C
unfolding verify-monad-composition-fri-decomposition
proof (rule wp-event-bind-bound-by-cont)
  show ¬ composition-fri-query-challenge-pair-set-hit s P None
  unfolding composition-fri-query-challenge-pair-set-hit-def
    accepted-fri-opening-transcript-def
  by blast
next
fix header prefix-state
assume prefix:
  Some (header, prefix-state) ∈
    set-dist (execute verifier-composition-fri-prefix s)
obtain fr f-fl f-final as dg fl where header-eq:
  header = (fr, f-fl, f-final, as, dg, fl)
by (cases header) auto
have future-prefix: query-future-fresh prefix-state
by (rule verifier-composition-fri-prefix-preserves-query-future-fresh
    [OF future prefix])
have counter-prefix: PQueryCounter prefix-state = PQueryCounter s
using verifier-composition-fri-prefix-outcome[OF prefix[unfolded header-eq]]
by simp
let ?Q = Q dg (map fst fl)
have event-bound:
  wp-event (verifier-after-composition-fri header)
    (λout. ∃ query-idxs ∈ ?Q.
      query-rounds-index-list-hit s query-idxs rounds out)
    prefix-state ≤
      nnreal (card ?Q) *
      (1 / nnreal (card query-sample-space)) ^ rounds
by (rule wp-verifier-after-composition-fri-query-index-list-set-bound
    [OF future-prefix counter-prefix raw-bound subset])
have pair-to-query:
  wp-event (verifier-after-composition-fri header)
    (composition-fri-query-challenge-pair-set-hit s P) prefix-state ≤
  wp-event (verifier-after-composition-fri header)
    (λout. ∃ query-idxs ∈ ?Q.
      query-rounds-index-list-hit s query-idxs rounds out) prefix-state
proof (rule wp-event-mono-on-support)
  fix out
  assume suffix:
    out ∈
      set-dist (execute (verifier-after-composition-fri header)
        prefix-state)
    and hit: composition-fri-query-challenge-pair-set-hit s P out
  from hit obtain trace-roots trace-bs trace-final fri-dg

```

composition-roots composition-bs composition-final query-idxs
trace-round-layers composition-round-layers **where**
openings:
accepted-fri-opening-transcript s out trace-roots trace-bs
trace-final fri-dg composition-roots composition-bs
composition-final query-idxs trace-round-layers
composition-round-layers
and *pair*: (*query-idxs*, *composition-bs*) $\in P$ *fri-dg*
unfolding *composition-fri-query-challenge-pair-set-hit-def* **by** *blast*
from *openings* **obtain** *result final-state* **where** *out-eq*:
out = Some (result, final-state)
unfolding *accepted-fri-opening-transcript-def* **by** *blast*
have *prefix'*:
Some ((fr, f-fl, f-final, as, dg, fl), prefix-state) $\in$
set-dist (execute verifier-composition-fri-prefix s)
using *prefix header-eq* **by** *simp*
have *suffix'*:
Some (result, final-state) $\in$
set-dist
(execute
(verifier-after-composition-fri
(fr, f-fl, f-final, as, dg, fl))
prefix-state)
using *suffix header-eq out-eq* **by** *simp*
have *challenges-prefix*:
accepted-fri-challenges s (Some (result, final-state))
(map fst f-fl) dg (map fst fl)
by (*rule verifier-after-composition-fri-accepted-fri-challenges*
[OF prefix' suffix'])
have *challenges-openings*:
accepted-fri-challenges s out trace-bs fri-dg composition-bs
using *openings*
unfolding *accepted-fri-opening-transcript-def*
apply (*elim exE conjE*)
apply *assumption*
done
have *eqs*: *fri-dg = dg $\wedge$ composition-bs = map fst fl*
using *accepted-fri-challenges-unique*
[OF challenges-prefix challenges-openings[unfolded out-eq]]
by *simp*
have *query-in*: *query-idxs $\in ?Q$*
proof –
have *query-idxs $\in$*
fst ‘ (P dg $\cap$ (UNIV $\times$ {map fst fl}))
using *pair eqs* **by** *force*
then show *?thesis*
using *projection[of dg map fst fl]* **by** *blast*
qed
have *query-hit*:

```

    query-rounds-index-list-hit s query-idxs rounds out
  by (rule accepted-fri-opening-transcript-imp-query-rounds-index-list-hit
      [OF openings])
show  $\exists$  query-idxs  $\in$  ?Q.
    query-rounds-index-list-hit s query-idxs rounds out
    using query-in query-hit by blast
qed
show
  wp-event (verifier-after-composition-fri header)
    (composition-fri-query-challenge-pair-set-hit s P) prefix-state  $\leq$  C
  by (rule order-trans[OF pair-to-query])
    (rule order-trans[OF event-bound bound])
qed

lemma wp-composition-fri-query-challenge-pair-set-hit-fixed-challenge-bound:
  assumes composition-future: composition-fri-future-fresh s
    and query-future: query-future-fresh s
    and raw-bound: query-index-raw-preimage-bound
    and challenges-space:
      challenges  $\in$  fri-challenge-space (ceil-log (to-nat dg + 1))
    and projection:
      fst ' (P dg  $\cap$  (UNIV  $\times$  {challenges}))  $\subseteq$  Q
    and subset: Q  $\subseteq$  fri-query-index-list-space
  shows
    wp-event verify-monad
      ( $\lambda$ out.
        composition-fri-challenge-list-set-hit s
          ( $\lambda$ dg'. if dg' = dg then {challenges} else {})) out  $\wedge$ 
        composition-fri-query-challenge-pair-set-hit s P out) s  $\leq$ 
      (1 / nnreal (CARD('f)  $\wedge$  ceil-log (to-nat dg + 1))) *
      (nnreal (card Q) *
        (1 / nnreal (card query-sample-space))  $\wedge$  rounds)
  proof -
    let ?B =  $\lambda$ dg'. if dg' = dg then {challenges} else {}
    let ?Hit =
       $\lambda$ out. composition-fri-challenge-list-set-hit s ?B out  $\wedge$ 
        composition-fri-query-challenge-pair-set-hit s P out
    let ?Head =
       $\lambda$ out. case out of
        None  $\Rightarrow$  False
      | Some ((-, -, -, dg', fl), -)  $\Rightarrow$ 
        dg' = dg  $\wedge$  map fst fl = challenges
    let ?D =
      nnreal (card Q) *
      (1 / nnreal (card query-sample-space))  $\wedge$  rounds
    have bind-bound:
      wp-event
        (verifier-composition-fri-prefix  $\gg$  verifier-after-composition-fri)
        ?Hit s  $\leq$  wp-event verifier-composition-fri-prefix ?Head s * ?D

```

```

proof (rule wp-event-bind-bound-by-head-and-cont)
show  $\neg ?Hit \text{ None}$ 
  unfolding composition-fri-challenge-list-set-hit-def
    composition-fri-query-challenge-pair-set-hit-def
    accepted-fri-opening-transcript-def
  by blast
next
fix header prefix-state
assume prefix:
  Some (header, prefix-state)  $\in$ 
    set-dist (execute verifier-composition-fri-prefix s)
and not-head:
   $\neg$  (case Some (header, prefix-state) of None  $\Rightarrow$  False
    | Some ((-, -, -, -, dg', fl), -)  $\Rightarrow$ 
      dg' = dg  $\wedge$  map fst fl = challenges)
obtain fr f-fl f-final as dg' fl where header-eq:
  header = (fr, f-fl, f-final, as, dg', fl)
by (cases header) auto
have not-fixed: dg'  $\neq$  dg  $\vee$  map fst fl  $\neq$  challenges
  using not-head unfolding header-eq by simp
have zero-le:
  wp-event (verifier-after-composition-fri header)
    ( $\lambda$ out.
      composition-fri-challenge-list-set-hit s
        ( $\lambda$ dg'. if dg' = dg then {challenges} else {})) out  $\wedge$ 
      composition-fri-query-challenge-pair-set-hit s P out) prefix-state
     $\leq$  0
proof (rule order-trans)
show
  wp-event (verifier-after-composition-fri header)
    ( $\lambda$ out.
      composition-fri-challenge-list-set-hit s
        ( $\lambda$ dg'. if dg' = dg then {challenges} else {})) out  $\wedge$ 
      composition-fri-query-challenge-pair-set-hit s P out) prefix-state
     $\leq$ 
  wp-event (verifier-after-composition-fri header)
    ( $\lambda$ -. False) prefix-state
proof (rule wp-event-mono-on-support)
fix out
assume suffix:
  out  $\in$ 
    set-dist
      (execute (verifier-after-composition-fri header) prefix-state)
and hit:
  composition-fri-challenge-list-set-hit s
    ( $\lambda$ dg'. if dg' = dg then {challenges} else {})) out  $\wedge$ 
    composition-fri-query-challenge-pair-set-hit s P out
from hit obtain trace-bs fri-dg composition-bs where
  challenges-hit:

```

```

    accepted-fri-challenges s out trace-bs fri-dg composition-bs
and comp-in:
    composition-bs ∈
      (if fri-dg = dg then {challenges} else {})
unfolding composition-fri-challenge-list-set-hit-def
by blast
from challenges-hit obtain result final-state where out-eq:
    out = Some (result, final-state)
unfolding accepted-fri-challenges-def by blast
have prefix':
    Some ((fr, f-fl, f-final, as, dg', fl), prefix-state) ∈
      set-dist (execute verifier-composition-fri-prefix s)
using prefix header-eq by simp
have suffix':
    Some (result, final-state) ∈
      set-dist
        (execute
          (verifier-after-composition-fri
            (fr, f-fl, f-final, as, dg', fl))
          prefix-state)
using suffix header-eq out-eq by simp
have challenges-prefix:
    accepted-fri-challenges s (Some (result, final-state))
      (map fst f-fl) dg' (map fst fl)
by (rule verifier-after-composition-fri-accepted-fri-challenges
      [OF prefix' suffix'])
have eqs: fri-dg = dg' ∧ composition-bs = map fst fl
using accepted-fri-challenges-unique
  [OF challenges-prefix challenges-hit[unfolded out-eq]]
by simp
have fixed: fri-dg = dg ∧ composition-bs = challenges
using comp-in by (cases fri-dg = dg) auto
show False
using not-fixed eqs fixed by simp
qed
next
show
  wp-event (verifier-after-composition-fri header)
    (λ-. False) prefix-state ≤ 0
unfolding wp-event-def wp-def dist-expect-def by simp
qed
show
  wp-event (verifier-after-composition-fri header)
    (λout.
      composition-fri-challenge-list-set-hit s
        (λdg'. if dg' = dg then {challenges} else {}) out ∧
        composition-fri-query-challenge-pair-set-hit s P out) prefix-state
    = 0
by (rule antisym[OF zero-le]) simp

```

```

next
fix header prefix-state
assume prefix:
  Some (header, prefix-state) ∈
    set-dist (execute verifier-composition-fri-prefix s)
and head:
  (case Some (header, prefix-state) of None ⇒ False
   | Some ((-, -, -, dg', fl), -) ⇒
     dg' = dg ∧ map fst fl = challenges)
obtain fr f-fl f-final as dg' fl where header-eq:
  header = (fr, f-fl, f-final, as, dg', fl)
by (cases header) auto
have dg-eq: dg' = dg and fl-eq: map fst fl = challenges
  using head unfolding header-eq by simp-all
have future-prefix: query-future-fresh prefix-state
  by (rule verifier-composition-fri-prefix-preserves-query-future-fresh
      [OF query-future prefix])
have counter-prefix: PQueryCounter prefix-state = PQueryCounter s
  using verifier-composition-fri-prefix-outcome[OF prefix[unfolded header-eq]]
  by simp
have event-bound:
  wp-event (verifier-after-composition-fri header)
    (λout. ∃ query-idxs ∈ Q.
      query-rounds-index-list-hit s query-idxs rounds out)
    prefix-state ≤ ?D
  by (rule wp-verifier-after-composition-fri-query-index-list-set-bound
      [OF future-prefix counter-prefix raw-bound subset])
have hit-to-query:
  wp-event (verifier-after-composition-fri header)
    (λout.
      composition-fri-challenge-list-set-hit s
        (λdg'. if dg' = dg then {challenges} else {}) out ∧
        composition-fri-query-challenge-pair-set-hit s P out) prefix-state
    ≤
  wp-event (verifier-after-composition-fri header)
    (λout. ∃ query-idxs ∈ Q.
      query-rounds-index-list-hit s query-idxs rounds out) prefix-state
proof (rule wp-event-mono-on-support)
fix out
assume suffix:
  out ∈
    set-dist (execute (verifier-after-composition-fri header)
      prefix-state)
and hit:
  composition-fri-challenge-list-set-hit s
    (λdg'. if dg' = dg then {challenges} else {}) out ∧
    composition-fri-query-challenge-pair-set-hit s P out
from hit obtain trace-roots trace-bs trace-final fri-dg
  composition-roots composition-bs composition-final query-idxs

```

trace-round-layers composition-round-layers **where**
openings:
accepted-fri-opening-transcript s out trace-roots trace-bs
trace-final fri-dg composition-roots composition-bs
composition-final query-idxs trace-round-layers
composition-round-layers
and *pair*: (*query-idxs*, *composition-bs*) $\in P$ *fri-dg*
unfolding *composition-fri-query-challenge-pair-set-hit-def* **by** *blast*
from *openings* **obtain** *result final-state* **where** *out-eq*:
out = Some (result, final-state)
unfolding *accepted-fri-opening-transcript-def* **by** *blast*
have *prefix'*:
Some ((fr, f-fl, f-final, as, dg', fl), prefix-state) $\in$
set-dist (execute verifier-composition-fri-prefix s)
using *prefix header-eq* **by** *simp*
have *suffix'*:
Some (result, final-state) $\in$
set-dist
(execute
(verifier-after-composition-fri
(fr, f-fl, f-final, as, dg', fl))
prefix-state)
using *suffix header-eq out-eq* **by** *simp*
have *challenges-prefix*:
accepted-fri-challenges s (Some (result, final-state))
(map fst f-fl) dg' (map fst fl)
by (*rule verifier-after-composition-fri-accepted-fri-challenges*
[OF prefix' suffix'])
have *challenges-openings*:
accepted-fri-challenges s out trace-bs fri-dg composition-bs
using *openings*
unfolding *accepted-fri-opening-transcript-def*
apply (*elim exE conjE*)
apply *assumption*
done
have *eqs*: *fri-dg = dg' $\wedge$ composition-bs = map fst fl*
using *accepted-fri-challenges-unique*
[OF challenges-prefix challenges-openings[unfolded out-eq]]
by *simp*
have *query-in*: *query-idxs $\in Q$*
proof –
have *query-idxs $\in$ fst ‘ (P dg $\cap$ (UNIV $\times$ {challenges}))*
using *pair eqs dg-eq fl-eq* **by** *force*
then show *?thesis*
using *projection* **by** *blast*
qed
have *query-hit*:
query-rounds-index-list-hit s query-idxs rounds out
by (*rule accepted-fri-opening-transcript-imp-query-rounds-index-list-hit*

```

    [OF openings])
  show  $\exists \text{ query-idxs} \in Q.$ 
    query-rounds-index-list-hit  $s$  query-idxs rounds out
    using query-in query-hit by blast
qed
show
  wp-event (verifier-after-composition-fri header)
    ( $\lambda \text{out}.$ 
      composition-fri-challenge-list-set-hit  $s$ 
      ( $\lambda dg'. \text{ if } dg' = dg \text{ then } \{\text{challenges}\} \text{ else } \{\}$ ) out  $\wedge$ 
      composition-fri-query-challenge-pair-set-hit  $s P$  out) prefix-state
       $\leq ?D$ 
    by (rule order-trans[OF hit-to-query event-bound])
qed
have head-bound:
  wp-event verifier-composition-fri-prefix ?Head  $s \leq$ 
    1 / nnreal (CARD('f) ^ ceil-log (to-nat dg + 1))
proof (rule order-trans)
  let ?SetHead =
     $\lambda \text{out}.$  case out of
      None  $\Rightarrow$  False
      | Some ((-, -, -, dg', fl), -)  $\Rightarrow$  map fst fl  $\in ?B dg'$ 
  show wp-event verifier-composition-fri-prefix ?Head  $s \leq$ 
    wp-event verifier-composition-fri-prefix ?SetHead  $s$ 
  by (rule wp-event-mono-on-support)
    (auto split: option.splits prod.splits)
  show wp-event verifier-composition-fri-prefix ?SetHead  $s \leq$ 
    1 / nnreal (CARD('f) ^ ceil-log (to-nat dg + 1))
  by (rule wp-verifier-composition-fri-prefix-challenge-space-set-bound
      [OF composition-future])
    (use challenges-space in auto)
qed
show ?thesis
  unfolding verify-monad-composition-fri-decomposition
  apply (rule order-trans)
  apply (rule bind-bound)
  apply (rule mult-right-mono[OF head-bound])
  apply simp
done
qed

lemma wp-composition-fri-query-challenge-pair-set-hit-bound-from-query-fiber-sum:
  assumes composition-future: composition-fri-future-fresh  $s$ 
  and query-future: query-future-fresh  $s$ 
  and raw-bound: query-index-raw-preimage-bound
  and projection:
     $\bigwedge dg \text{ challenges}.$ 
      fst ' ( $P dg \cap (UNIV \times \{\text{challenges}\})$ )  $\subseteq$ 
         $Q dg \text{ challenges}$ 

```


and *subset*:
 $\bigwedge dg \text{ challenges. } Q \text{ dg challenges} \subseteq \text{fri-query-index-list-space}$
shows
 $\text{wp-event verify-monad}$
 $(\text{composition-fri-query-challenge-pair-set-hit } s \text{ } P) \text{ } s \leq$
 $(\sum dg \in \text{UNIV.}$
 $\sum \text{challenges} \in \text{fri-challenge-space } (\text{ceil-log } (\text{to-nat } dg + 1)).$
 $(1 / \text{nnreal } (\text{CARD}('f) \wedge \text{ceil-log } (\text{to-nat } dg + 1))) *$
 $(\text{nnreal } (\text{card } (Q \text{ dg challenges})) *$
 $(1 / \text{nnreal } (\text{card query-sample-space})) \wedge \text{rounds}))$
proof –
let $?I =$
 $\text{SIGMA } dg:\text{UNIV. } \text{fri-challenge-space } (\text{ceil-log } (\text{to-nat } dg + 1))$
let $?E =$
 $\lambda p \text{ out.}$
 $\text{composition-fri-challenge-list-set-hit } s$
 $(\lambda dg'. \text{ if } dg' = \text{fst } p \text{ then } \{\text{snd } p\} \text{ else } \{\}) \text{ out} \wedge$
 $\text{composition-fri-query-challenge-pair-set-hit } s \text{ } P \text{ out}$
let $?B =$
 $\lambda p.$
 $(1 / \text{nnreal } (\text{CARD}('f) \wedge \text{ceil-log } (\text{to-nat } (\text{fst } p) + 1))) *$
 $(\text{nnreal } (\text{card } (Q (\text{fst } p) (\text{snd } p)))) *$
 $(1 / \text{nnreal } (\text{card query-sample-space})) \wedge \text{rounds})$
have *split*:
 $\text{wp-event verify-monad}$
 $(\text{composition-fri-query-challenge-pair-set-hit } s \text{ } P) \text{ } s \leq$
 $\text{wp-event verify-monad}$
 $(\lambda out. \exists p \in ?I. ?E \text{ } p \text{ out}) \text{ } s$
proof (rule *wp-event-mono*)
fix *out*
assume *hit*: $\text{composition-fri-query-challenge-pair-set-hit } s \text{ } P \text{ out}$
from *hit* **obtain** *trace-roots trace-bs trace-final fri-dg*
 $\text{composition-roots composition-bs composition-final query-idxs}$
 $\text{trace-round-layers composition-round-layers}$ **where**
openings:
 $\text{accepted-fri-opening-transcript } s \text{ out trace-roots trace-bs}$
 $\text{trace-final fri-dg composition-roots composition-bs}$
 $\text{composition-final query-idxs trace-round-layers}$
 $\text{composition-round-layers}$
unfolding $\text{composition-fri-query-challenge-pair-set-hit-def}$ **by** *blast*
have *challenges*:
 $\text{accepted-fri-challenges } s \text{ out trace-bs fri-dg composition-bs}$
using *openings*
unfolding $\text{accepted-fri-opening-transcript-def}$
apply (*elim exE conjE*)
apply *assumption*
done
have *comp-space*:
 $\text{composition-bs} \in \text{fri-challenge-space } (\text{ceil-log } (\text{to-nat } \text{fri-dg} + 1))$

```

    by (rule accepted-fri-challenges-composition-space[OF challenges])
  have challenge-hit:
    composition-fri-challenge-list-set-hit s
    ( $\lambda dg'. \text{if } dg' = \text{fri-dg} \text{ then } \{\text{composition-bs}\} \text{ else } \{\}$ ) out
  unfolding composition-fri-challenge-list-set-hit-def
  using challenges by auto
  have (fri-dg, composition-bs)  $\in ?I$ 
  using comp-space by simp
  then show  $\exists p \in ?I. ?E p$  out
  proof (intro bexI[where  $x = (\text{fri-dg}, \text{composition-bs})$ ])
    show  $?E (\text{fri-dg}, \text{composition-bs})$  out
    using challenge-hit hit by (simp only: fst-conv snd-conv)
    show (fri-dg, composition-bs)  $\in ?I$ 
    using comp-space by simp
  qed
qed
also have  $\dots \leq (\sum p \in ?I. ?B p)$ 
proof (rule wp-event-finite-union-bound-fri-exact)
  show finite ?I
  by (simp add: finite-fri-challenge-space)
next
fix p
assume p-in:  $p \in ?I$ 
obtain dg challenges where p-eq:  $p = (\text{dg}, \text{challenges})$ 
by (cases p)
have challenges:
  challenges  $\in \text{fri-challenge-space } (\text{ceil-log } (\text{to-nat dg} + 1))$ 
  using p-in p-eq by simp
have fixed-bound:
  wp-event verify-monad
  ( $\lambda \text{out}. \text{composition-fri-challenge-list-set-hit } s$ 
  ( $\lambda dg'. \text{if } dg' = \text{dg} \text{ then } \{\text{challenges}\} \text{ else } \{\}$ ) out  $\wedge$ 
  composition-fri-query-challenge-pair-set-hit  $s P$  out)  $s$ 
   $\leq 1 / \text{nnreal } (\text{CARD}(f) \wedge \text{ceil-log } (\text{to-nat dg} + 1)) *$ 
  ( $\text{nnreal } (\text{card } (Q \text{ dg challenges})) *$ 
  ( $1 / \text{nnreal } (\text{card query-sample-space})) \wedge \text{rounds}$ )
  by (rule
    wp-composition-fri-query-challenge-pair-set-hit-fixed-challenge-bound
    [OF composition-future query-future raw-bound challenges
      projection subset])
show wp-event verify-monad ( $?E p$ )  $s \leq ?B p$ 
  unfolding p-eq using fixed-bound by (simp only: fst-conv snd-conv)
qed
also have  $\dots =$ 
  ( $\sum dg \in \text{UNIV}. \sum \text{challenges} \in \text{fri-challenge-space } (\text{ceil-log } (\text{to-nat dg} + 1)).$ 
  ( $1 / \text{nnreal } (\text{CARD}(f) \wedge \text{ceil-log } (\text{to-nat dg} + 1)) *$ 
  ( $\text{nnreal } (\text{card } (Q \text{ dg challenges})) *$ 
  ( $1 / \text{nnreal } (\text{card query-sample-space})) \wedge \text{rounds}$ ))

```

by (subst sum.Sigma) (simp-all add: finite-fri-challenge-space case-prod-beta)
 finally show ?thesis .
 qed

lemma wp-composition-fri-query-challenge-pair-set-hit-bound-from-challenge-cover-and-query-fibers:

fixes C-challenge C-query :: prob
assumes composition-future: composition-fri-future-fresh s
and query-future: query-future-fresh s
and raw-bound: query-index-raw-preimage-bound
and challenge-subset:
 $\bigwedge dg. B \ dg \subseteq \text{fri-challenge-space } (\text{ceil-log } (to\text{-nat } dg + 1))$
and challenge-bound:
 $\bigwedge dg. \text{nnreal } (\text{card } (B \ dg)) / \text{nnreal } (CARD('f) \wedge \text{ceil-log } (to\text{-nat } dg + 1)) \leq C\text{-challenge}$
and projection:
 $\bigwedge dg \text{ challenges. } \text{fst } ' ((P \ dg \cap (UNIV \times (- \ B \ dg))) \cap (UNIV \times \{\text{challenges}\})) \subseteq Q \ dg \text{ challenges}$
and subset:
 $\bigwedge dg \text{ challenges. } Q \ dg \text{ challenges} \subseteq \text{fri-query-index-list-space}$
and query-bound:
 $\bigwedge dg \text{ challenges. } \text{nnreal } (\text{card } (Q \ dg \text{ challenges})) * (1 / \text{nnreal } (\text{card } \text{query-sample-space})) \wedge \text{rounds} \leq C\text{-query}$

shows

wp-event verify-monad
 $(\text{composition-fri-query-challenge-pair-set-hit } s \ P) \ s \leq C\text{-challenge} + C\text{-query}$

proof –

let ?Outside = $\lambda dg. P \ dg \cap (UNIV \times (- \ B \ dg))$

have split:

wp-event verify-monad
 $(\text{composition-fri-query-challenge-pair-set-hit } s \ P) \ s \leq$
 wp-event verify-monad
 $(\lambda out. \text{composition-fri-challenge-list-set-hit } s \ B \ out \vee \text{composition-fri-query-challenge-pair-set-hit } s \ ?Outside \ out) \ s$

proof (rule wp-event-mono)

fix out

assume hit: composition-fri-query-challenge-pair-set-hit s P out

from hit **obtain** trace-roots trace-bs trace-final fri-dg
 composition-roots composition-bs composition-final query-idxs
 trace-round-layers composition-round-layers **where**

openings:

accepted-fri-opening-transcript s out trace-roots trace-bs
 trace-final fri-dg composition-roots composition-bs
 composition-final query-idxs trace-round-layers
 composition-round-layers

and pair: (query-idxs, composition-bs) $\in P \ \text{fri-dg}$

unfolding composition-fri-query-challenge-pair-set-hit-def **by** blast

```

show
  composition-fri-challenge-list-set-hit s B out ∨
  composition-fri-query-challenge-pair-set-hit s ?Outside out
proof (cases composition-bs ∈ B fri-dg)
  case True
  have challenges:
    accepted-fri-challenges s out trace-bs fri-dg composition-bs
  using openings
  unfolding accepted-fri-opening-transcript-def
  apply (elim exE conjE)
  apply assumption
  done
  then have composition-fri-challenge-list-set-hit s B out
    using True unfolding composition-fri-challenge-list-set-hit-def
    by blast
  then show ?thesis by simp
next
case False
have (query-idxs, composition-bs) ∈ ?Outside fri-dg
  using pair False by simp
then have
  composition-fri-query-challenge-pair-set-hit s ?Outside out
  unfolding composition-fri-query-challenge-pair-set-hit-def
  using openings by blast
then show ?thesis by simp
qed
qed
also have ... ≤
  wp-event verify-monad (composition-fri-challenge-list-set-hit s B) s +
  wp-event verify-monad
  (composition-fri-query-challenge-pair-set-hit s ?Outside) s
  by (rule wp-event-union-bound)
also have ... ≤ C-challenge + C-query
proof (rule add-mono)
  show
    wp-event verify-monad (composition-fri-challenge-list-set-hit s B) s
    ≤ C-challenge
  by (rule wp-verify-monad-composition-fri-challenge-list-set-bound
    [OF composition-future challenge-subset challenge-bound])
  show
    wp-event verify-monad
    (composition-fri-query-challenge-pair-set-hit s ?Outside) s ≤
    C-query
  by (rule
    wp-composition-fri-query-challenge-pair-set-hit-bound-from-query-fibers
    [OF query-future raw-bound projection subset query-bound])
qed
finally show ?thesis .
qed

```

lemma *wp-trace-fri-sampled-query-bad-candidate-bound-from-challenge-cover-and-query-fibers:*

fixes *C-challenge C-query :: prob*

assumes *trace-future: trace-fri-future-fresh s*

and *query-future: query-future-fresh s*

and *raw-bound: query-index-raw-preimage-bound*

and *challenge-subset: $B \subseteq \text{fri-challenge-space } (\text{ceil-log } \text{clength})$*

and *challenge-bound:*

nnreal (card B) / nnreal (CARD('f) ^ ceil-log clength) ≤
C-challenge

and *projection:*

∧ challenges.

fst ‘

((trace-fri-sampled-query-bad-pair-union ∩
(fri-query-index-list-space × UNIV)) ∩
(UNIV × (− B))) ∩
(UNIV × {challenges})) ⊆ Q challenges

and *subset: ∧ challenges. Q challenges ⊆ fri-query-index-list-space*

and *query-bound:*

∧ challenges.

*nnreal (card (Q challenges)) **
(1 / nnreal (card query-sample-space)) ^ rounds ≤ C-query

shows

wp-event verify-monad (trace-fri-sampled-query-bad-candidate s) s ≤
C-challenge + C-query

proof −

let *?P =*

trace-fri-sampled-query-bad-pair-union ∩
(fri-query-index-list-space × UNIV)

have *wp-event verify-monad (trace-fri-sampled-query-bad-candidate s) s*

≤ wp-event verify-monad

(trace-fri-query-challenge-pair-set-hit s ?P) s

by *(rule*

wp-trace-fri-sampled-query-bad-candidate-bound-by-restricted-query-challenge-union-hit)

also have *... ≤ C-challenge + C-query*

by *(rule*

wp-trace-fri-query-challenge-pair-set-hit-bound-from-challenge-cover-and-query-fibers
[OF trace-future query-future raw-bound challenge-subset
challenge-bound projection subset query-bound])

finally show *?thesis .*

qed

lemma *wp-composition-fri-sampled-query-bad-candidate-bound-from-challenge-cover-and-query-fibers:*

fixes *C-challenge C-query :: prob*

assumes *composition-future: composition-fri-future-fresh s*

and *query-future: query-future-fresh s*

and *raw-bound: query-index-raw-preimage-bound*

and *challenge-subset:*

∧ dg. B dg ⊆ fri-challenge-space (ceil-log (to-nat dg + 1))

and challenge-bound:
 $\bigwedge dg. \text{nnreal} (\text{card } (B \text{ dg})) / \text{nnreal} (\text{CARD}('f) \wedge \text{ceil-log } (\text{to-nat } dg + 1)) \leq C\text{-challenge}$
and projection:
 $\bigwedge dg \text{ challenges.}$
 fst '
 $((\text{composition-fri-sampled-query-bad-pair-union } dg \cap (\text{fri-query-index-list-space} \times \text{UNIV})) \cap (\text{UNIV} \times (- B \text{ dg}))) \cap (\text{UNIV} \times \{\text{challenges}\}) \subseteq Q \text{ dg challenges}$
and subset:
 $\bigwedge dg \text{ challenges. } Q \text{ dg challenges} \subseteq \text{fri-query-index-list-space}$
and query-bound:
 $\bigwedge dg \text{ challenges.}$
 $\text{nnreal} (\text{card } (Q \text{ dg challenges})) * (1 / \text{nnreal} (\text{card query-sample-space})) \wedge \text{rounds} \leq C\text{-query}$
shows
 $\text{wp-event verify-monad}$
 $(\text{composition-fri-sampled-query-bad-candidate } s) s \leq C\text{-challenge} + C\text{-query}$
proof –
let $?P =$
 $\bigwedge dg. \text{composition-fri-sampled-query-bad-pair-union } dg \cap (\text{fri-query-index-list-space} \times \text{UNIV})$
have $\text{wp-event verify-monad}$
 $(\text{composition-fri-sampled-query-bad-candidate } s) s \leq$
 $\text{wp-event verify-monad}$
 $(\text{composition-fri-query-challenge-pair-set-hit } s ?P) s$
by (rule
 $\text{wp-composition-fri-sampled-query-bad-candidate-bound-by-restricted-query-challenge-union-hit})$
also have $\dots \leq C\text{-challenge} + C\text{-query}$
by (rule
 $\text{wp-composition-fri-query-challenge-pair-set-hit-bound-from-challenge-cover-and-query-fibers}$
 $[\text{OF composition-future query-future raw-bound challenge-subset challenge-bound projection subset query-bound}])$
finally show $?thesis$.
qed

lemma $\text{wp-trace-fri-sampled-query-bad-candidate-bound-from-generic-challenge-cover-and-fibers:}$

fixes $C\text{-challenge } C\text{-query} :: \text{prob}$
assumes $\text{trace-future: trace-fri-future-fresh } s$
and $\text{query-future: query-future-fresh } s$
and $\text{raw-bound: query-index-raw-preimage-bound}$
and $\text{challenge-subset: } B \subseteq \text{fri-challenge-space } (\text{ceil-log } \text{clength})$
and challenge-bound:
 $\text{nnreal} (\text{card } B) / \text{nnreal} (\text{CARD}('f) \wedge \text{ceil-log } \text{clength}) \leq C\text{-challenge}$
and query-bound:
 $\bigwedge \text{challenges.}$

```

    nnreal
      (card
        (generic-fri-sampled-query-query-fiber trace-table-low-degree
          (Not  $\circ$  trace-table-low-degree) (clength - 1) challenges)) *
        (1 / nnreal (card query-sample-space)) ^ rounds  $\leq$  C-query
  shows
    wp-event verify-monad (trace-fri-sampled-query-bad-candidate s) s  $\leq$ 
      C-challenge + C-query
  proof (rule
    wp-trace-fri-sampled-query-bad-candidate-bound-from-challenge-cover-and-query-fibers
    [OF trace-future query-future raw-bound challenge-subset challenge-bound])
  fix challenges
  show
    fst ‘
      (((trace-fri-sampled-query-bad-pair-union  $\cap$ 
        (fri-query-index-list-space  $\times$  UNIV))  $\cap$ 
        (UNIV  $\times$  (- B)))  $\cap$ 
        (UNIV  $\times$  {challenges}))
       $\subseteq$ 
      generic-fri-sampled-query-query-fiber trace-table-low-degree
        (Not  $\circ$  trace-table-low-degree) (clength - 1) challenges
  unfolding trace-fri-sampled-query-bad-pair-union-def
    generic-fri-sampled-query-query-fiber-def
    generic-fri-sampled-query-restricted-bad-pairs-def
    fri-query-challenge-pair-query-fiber-def
  by auto
  next
  fix challenges
  show
    generic-fri-sampled-query-query-fiber trace-table-low-degree
      (Not  $\circ$  trace-table-low-degree) (clength - 1) challenges
       $\subseteq$  fri-query-index-list-space
    by (rule generic-fri-sampled-query-query-fiber-subset)
  next
  fix challenges
  show
    nnreal
      (card
        (generic-fri-sampled-query-query-fiber trace-table-low-degree
          (Not  $\circ$  trace-table-low-degree) (clength - 1) challenges)) *
        (1 / nnreal (card query-sample-space)) ^ rounds  $\leq$  C-query
    by (rule query-bound)
  qed

```

lemma wp-trace-fri-sampled-query-bad-candidate-bound-from-pair-fraction:
assumes trace-future: trace-fri-future-fresh s
and query-future: query-future-fresh s
and raw-bound: query-index-raw-preimage-bound
and fraction:

generic-fri-sampled-query-pair-fraction-bound trace-table-low-degree
 $(\text{Not} \circ \text{trace-table-low-degree}) (\text{clength} - 1) C$

shows

wp-event verify-monad (trace-fri-sampled-query-bad-candidate s) s $\leq C$

proof –

let $?P =$

trace-fri-sampled-query-bad-pair-union $\cap$
(fri-query-index-list-space $\times$ *UNIV)*

let $?Q =$

$\lambda \text{challenges.}$
generic-fri-sampled-query-query-fiber trace-table-low-degree
 $(\text{Not} \circ \text{trace-table-low-degree}) (\text{clength} - 1) \text{challenges}$

let $?I = \text{fri-challenge-space } (\text{ceil-log } \text{clength})$

have *sampling-to-pair*:

wp-event verify-monad (trace-fri-sampled-query-bad-candidate s) s
 $\leq \text{wp-event verify-monad}$
 $(\text{trace-fri-query-challenge-pair-set-hit } s ?P) s$

by (rule
wp-trace-fri-sampled-query-bad-candidate-bound-by-restricted-query-challenge-union-hit)

also have $\dots \leq$

$(\sum \text{challenges} \in ?I.$
 $(1 / \text{nnreal } (\text{CARD}('f) \wedge \text{ceil-log } \text{clength})) *$
 $(\text{nnreal } (\text{card } (?Q \text{ challenges})) *$
 $(1 / \text{nnreal } (\text{card } \text{query-sample-space})) \wedge \text{rounds}))$

proof (rule
wp-trace-fri-query-challenge-pair-set-hit-bound-from-query-fiber-sum
 $[OF \text{ trace-future query-future raw-bound}]$)

fix *challenges*

show $\text{fst } ' (?P \cap (UNIV \times \{ \text{challenges} \})) \subseteq$
 $?Q \text{ challenges}$

unfolding *trace-fri-sampled-query-bad-pair-union-def*
generic-fri-sampled-query-query-fiber-def
generic-fri-sampled-query-restricted-bad-pairs-def
fri-query-challenge-pair-query-fiber-def

by *auto*

next

fix *challenges*

show $?Q \text{ challenges} \subseteq \text{fri-query-index-list-space}$

by (rule *generic-fri-sampled-query-query-fiber-subset*)

qed

also have $\dots \leq C$

proof –

have *round-eq*:

fri-round-count-for-degree-bound $(\text{clength} - \text{Suc } 0) = \text{ceil-log } \text{clength}$

unfolding *fri-round-count-for-degree-bound-def*

by (*simp add: clength-pos*)

have *fiber-fraction*:

nnreal
 $(\sum \text{challenges} \in ?I. \text{card } (?Q \text{ challenges})) /$


```

    (nnreal (CARD('f) ^ ceil-log clength) *
      nnreal query-sample-space-size ^ rounds)
  ≤ C
using fraction
unfolding trace-fri-sampled-query-pair-fraction-bound-iff-fiber-sum
by (simp add: round-eq card-fri-query-index-list-space
      query-sample-space-size-pos algebra-simps)
have prob-eq:
  (∑ challenges ∈ ?I.
    (1 / nnreal (CARD('f) ^ ceil-log clength)) *
    (nnreal (card (?Q challenges)) *
      (1 / nnreal (card query-sample-space)) ^ rounds)) =
  nnreal
  (∑ challenges ∈ ?I. card (?Q challenges)) /
  (nnreal (CARD('f) ^ ceil-log clength) *
    nnreal query-sample-space-size ^ rounds)
using weighted-query-fiber-sum-eq
  [where I = ?I
    and f = λchallenges. card (?Q challenges)
    and A = CARD('f) ^ ceil-log clength
    and B = query-sample-space-size]
by (simp add: finite-fri-challenge-space query-sample-space-size-pos
      card-query-sample-space algebra-simps)
show ?thesis
  using fiber-fraction unfolding prob-eq .
qed
finally show ?thesis .
qed

lemma wp-composition-fri-sampled-query-bad-candidate-bound-from-generic-challenge-cover-and-fibers:
fixes C-challenge C-query :: prob
assumes composition-future: composition-fri-future-fresh s
and query-future: query-future-fresh s
and raw-bound: query-index-raw-preimage-bound
and challenge-subset:
  ∧dg. B dg ⊆ fri-challenge-space (ceil-log (to-nat dg + 1))
and challenge-bound:
  ∧dg. nnreal (card (B dg)) /
    nnreal (CARD('f) ^ ceil-log (to-nat dg + 1)) ≤ C-challenge
and query-bound:
  ∧dg challenges.
    nnreal
      (card
        (generic-fri-sampled-query-query-fiber
          (composition-table-low-degree (to-nat dg))
          (Not ∘ composition-table-low-degree maxDegree) (to-nat dg)
          challenges)) *
      (1 / nnreal (card query-sample-space)) ^ rounds ≤ C-query
shows

```

```

wp-event verify-monad
  (composition-fri-sampled-query-bad-candidate s) s ≤
  C-challenge + C-query
proof (rule
  wp-composition-fri-sampled-query-bad-candidate-bound-from-challenge-cover-and-query-fibers
  [OF composition-future query-future raw-bound challenge-subset
  challenge-bound])
fix dg challenges
show
  fst ‘
    (((composition-fri-sampled-query-bad-pair-union dg ∩
      (fri-query-index-list-space × UNIV)) ∩
      (UNIV × (− B dg))) ∩
      (UNIV × {challenges}))
    ⊆
    generic-fri-sampled-query-query-fiber
    (composition-table-low-degree (to-nat dg))
    (Not ∘ composition-table-low-degree maxDegree) (to-nat dg)
    challenges
unfolding composition-fri-sampled-query-bad-pair-union-def
    generic-fri-sampled-query-query-fiber-def
    generic-fri-sampled-query-restricted-bad-pairs-def
    fri-query-challenge-pair-query-fiber-def
by auto
next
fix dg challenges
show
  generic-fri-sampled-query-query-fiber
  (composition-table-low-degree (to-nat dg))
  (Not ∘ composition-table-low-degree maxDegree) (to-nat dg)
  challenges
  ⊆ fri-query-index-list-space
by (rule generic-fri-sampled-query-query-fiber-subset)
next
fix dg challenges
show
  nnreal
  (card
    (generic-fri-sampled-query-query-fiber
      (composition-table-low-degree (to-nat dg))
      (Not ∘ composition-table-low-degree maxDegree) (to-nat dg)
      challenges)) *
    (1 / nnreal (card query-sample-space)) ^ rounds ≤ C-query
by (rule query-bound)
qed

lemma wp-composition-fri-sampled-query-bad-candidate-bound-from-pair-fraction-sum:
  fixes C :: 'f ⇒ prob
  assumes composition-future: composition-fri-future-fresh s

```

```

and query-future: query-future-fresh s
and raw-bound: query-index-raw-preimage-bound
and fraction:
   $\wedge dg.$  generic-fri-sampled-query-pair-fraction-bound
    (composition-table-low-degree (to-nat dg))
    (Not  $\circ$  composition-table-low-degree maxDegree) (to-nat dg) (C dg)
shows
  wp-event verify-monad
    (composition-fri-sampled-query-bad-candidate s) s  $\leq$ 
    ( $\sum dg \in UNIV.$  C dg)
proof -
  let ?P =
     $\lambda dg.$  composition-fri-sampled-query-bad-pair-union dg  $\cap$ 
    (fri-query-index-list-space  $\times$  UNIV)
  let ?Q =
     $\lambda dg$  challenges.
    generic-fri-sampled-query-query-fiber
      (composition-table-low-degree (to-nat dg))
      (Not  $\circ$  composition-table-low-degree maxDegree) (to-nat dg)
    challenges
  have sampled-to-pair:
    wp-event verify-monad (composition-fri-sampled-query-bad-candidate s) s
       $\leq$  wp-event verify-monad
        (composition-fri-query-challenge-pair-set-hit s ?P) s
    by (rule
      wp-composition-fri-sampled-query-bad-candidate-bound-by-restricted-query-challenge-union-hit)
  also have ...  $\leq$ 
    ( $\sum dg \in UNIV.$ 
       $\sum challenges \in fri-challenge-space$  (ceil-log (to-nat dg + 1)).
      (1 / nnreal (CARD('f)  $\wedge$  ceil-log (to-nat dg + 1))) *
      (nnreal (card (?Q dg challenges)) *
      (1 / nnreal (card query-sample-space))  $\wedge$  rounds))
  proof (rule
    wp-composition-fri-query-challenge-pair-set-hit-bound-from-query-fiber-sum
    [OF composition-future query-future raw-bound])
  fix dg challenges
  show fst ' (?P dg  $\cap$  (UNIV  $\times$  {challenges}))  $\subseteq$ 
    ?Q dg challenges
    unfolding composition-fri-sampled-query-bad-pair-union-def
      generic-fri-sampled-query-query-fiber-def
      generic-fri-sampled-query-restricted-bad-pairs-def
      fri-query-challenge-pair-query-fiber-def
    by auto
  next
  fix dg challenges
  show ?Q dg challenges  $\subseteq$  fri-query-index-list-space
    by (rule generic-fri-sampled-query-query-fiber-subset)
  qed
  also have ...  $\leq$  ( $\sum dg \in UNIV.$  C dg)

```

```

proof (rule sum-mono)
  fix dg :: 'f
  assume dg ∈ UNIV
  let ?I = fri-challenge-space (ceil-log (to-nat dg + 1))
  have round-eq:
    fri-round-count-for-degree-bound (to-nat dg) =
      ceil-log (to-nat dg + 1)
  unfolding fri-round-count-for-degree-bound-def by simp
  have fiber-fraction:
    nnreal
      (∑ challenges ∈ ?I. card (?Q dg challenges)) /
      (nnreal (CARD('f) ^ ceil-log (to-nat dg + 1)) *
        nnreal query-sample-space-size ^ rounds)
    ≤ C dg
  using fraction[of dg]
  unfolding composition-fri-sampled-query-pair-fraction-bound-iff-fiber-sum
  by (simp add: round-eq card-fri-query-index-list-space
    query-sample-space-size-pos algebra-simps)
  have prob-eq:
    (∑ challenges ∈ ?I.
      (1 / nnreal (CARD('f) ^ ceil-log (to-nat dg + 1))) *
      (nnreal (card (?Q dg challenges)) *
        (1 / nnreal (card query-sample-space)) ^ rounds)) =
    nnreal
      (∑ challenges ∈ ?I. card (?Q dg challenges)) /
      (nnreal (CARD('f) ^ ceil-log (to-nat dg + 1)) *
        nnreal query-sample-space-size ^ rounds)
  using weighted-query-fiber-sum-eq
  [where I = ?I
    and f = λchallenges. card (?Q dg challenges)
    and A = CARD('f) ^ ceil-log (to-nat dg + 1)
    and B = query-sample-space-size]
  by (simp add: finite-fri-challenge-space query-sample-space-size-pos
    card-query-sample-space algebra-simps)
  show
    (∑ challenges ∈ ?I.
      (1 / nnreal (CARD('f) ^ ceil-log (to-nat dg + 1))) *
      (nnreal (card (?Q dg challenges)) *
        (1 / nnreal (card query-sample-space)) ^ rounds))
    ≤ C dg
  using fiber-fraction unfolding prob-eq .
qed
finally show ?thesis .
qed

end

end

```

```

theory Soundness-FRI-Query-Index-Staged-Bounds
  imports Soundness-FRI-Sampled-Interface
begin

```

Staged query-index accounting for FRI query-list events.

The verifier-local exact query-list bounds still assume query freshness of the transferred verifier state. This layer charges accepted FRI query-list hits to the checked staged transcript query-index target event, using only the query indices actually derived by the verifier.

```

lemma pair-second-set-cover-split:
  assumes  $(x, y) \in P$ 
  shows  $y \in B \vee (x, y) \in P \cap (UNIV \times (- B))$ 
proof (cases  $y \in B$ )
  case True
  then show ?thesis ..
next
  case False
  then have  $(x, y) \in P \cap (UNIV \times (- B))$ 
  using assms by simp
  then show ?thesis ..
qed

```

```

context soundness
begin

```

```

lemma accepted-fri-opening-transcript-query-headerE:
  assumes
    accepted-fri-opening-transcript s out trace-roots trace-bs trace-final
    dg composition-roots composition-bs composition-final query-idxs
    trace-round-layers composition-round-layers
  obtains fr as query-state where
    verifier-header-transcript s fr trace-roots trace-final as dg
    composition-roots composition-final (PTranscript query-state)
    PState query-state =
      verifier-header-state s fr trace-roots trace-final as dg
      composition-roots composition-final
    verifier-query-indices-derived s out
      (verifier-header-state s fr trace-roots trace-final as dg
       composition-roots composition-final)
      (PTranscript query-state) trace-roots composition-roots query-idxs
proof -
  show ?thesis
  using assms
  unfolding accepted-fri-opening-transcript-def
  apply (elim exE conjE)
  apply (rule that)
  by (auto intro: Channel-Core.protocol-channel.select-convs(1))
qed

```

lemma *accepted-fri-opening-transcript-imp-staged-query-index-set-hit*:

assumes *wf*: *staged-budget-wellformed budgets*

and *controlled*: *staged-adversary-controlled budgets A*

and *builder*:

Some (data, attacker-state) ∈
set-dist
(execute (checked-staged-transcript-program A)
adversary-initial-state)

and *openings*:

accepted-fri-opening-transcript
(verifier-state-from-adversary attacker-state
(staged-proof-transcript data))
(Some (result, final-state)) trace-roots trace-bs trace-final dg
composition-roots composition-bs composition-final query-idxs
trace-round-layers composition-round-layers

and *query-in*: *query-idxs ∈ Q*

shows *staged-security-with-data-query-index-set-hit*
(query-index-list-entries Q) (Some ((data, result), final-state))

proof –

let *?s =*
verifier-state-from-adversary attacker-state
(staged-proof-transcript data)

from *accepted-fri-opening-transcript-query-headerE[OF openings]*

obtain *fr* as *query-state* **where**

header:

verifier-header-transcript ?s fr trace-roots trace-final as dg
composition-roots composition-final (PTranscript query-state)

and *query-state-hash*:

PState query-state =
verifier-header-state ?s fr trace-roots trace-final as dg
composition-roots composition-final

and *derived*:

verifier-query-indices-derived ?s (Some (result, final-state))
(verifier-header-state ?s fr trace-roots trace-final as dg
composition-roots composition-final)
(PTranscript query-state) trace-roots composition-roots query-idxs

by *blast*

have *staged-header*:

verifier-header-transcript ?s
(staged-trace-root data)
(staged-trace-fri-roots data)
(staged-trace-final data)
(staged-alphas data)
(staged-degree data)
(staged-composition-fri-roots data)
(staged-composition-final data)
(List.concat (staged-query-chunks data))

by *(rule checked-staged-transcript-program-outcome-header-transcript*

```

    [OF wf controlled builder])
have header-eq:
  fr = staged-trace-root data ∧
  trace-roots = staged-trace-fri-roots data ∧
  trace-final = staged-trace-final data ∧
  as = staged-alphas data ∧
  dg = staged-degree data ∧
  composition-roots = staged-composition-fri-roots data ∧
  composition-final = staged-composition-final data ∧
  PTranscript query-state = List.concat (staged-query-chunks data)
using verifier-header-transcript-unique[OF header staged-header]
by simp
have query-start-eq:
  verifier-header-state ?s fr trace-roots trace-final as dg
  composition-roots composition-final =
  staged-query-start-hash data
using query-state-hash header-eq
unfolding staged-query-start-hash-def
  staged-composition-fri-start-hash-def
  staged-trace-fri-start-hash-def
  verifier-header-state-def verifier-header-messages-def
  verifier-state-from-adversary-def
by simp
have query-counter: PQueryCounter ?s = 0
unfolding verifier-state-from-adversary-def by simp
from verifier-query-indices-derivedE[OF derived]
obtain result' final-state' raw-idxs query-chunks trailing where
  out-derived: Some (result, final-state) = Some (result', final-state')
and len-raw: length raw-idxs = rounds
and query-idxs-eq:
  query-idxs = map (λraw. index (to-nat raw)) raw-idxs
and len-chunks: length query-chunks = rounds
and concat-eq:
  List.concat query-chunks @ trailing = PTranscript query-state
and round-chunks:
  ∧ i. i < rounds ⇒
    verifier-query-round-chunk (query-idxs ! i)
    trace-roots composition-roots (query-chunks ! i)
and lookups':
  ∧ i. i < rounds ⇒
    fmlookup (HashMap final-state')
    (QueryIndexChallenge (PQueryCounter ?s + i)
    (state-after-query-chunks
    (verifier-header-state ?s fr trace-roots trace-final as dg
    composition-roots composition-final)
    query-chunks i)) =
    Some (raw-idxs ! i)
by metis
have final-state'-eq: final-state' = final-state

```

```

using out-derived by simp
have lookups:
   $\wedge i. i < \text{rounds} \implies$ 
    fmlookup (HashMap final-state)
      (QueryIndexChallenge (PQueryCounter ?s + i)
        (state-after-query-chunks
          (verifier-header-state ?s fr trace-roots trace-final as dg
            composition-roots composition-final)
          query-chunks i)) =
      Some (raw-idxs ! i)
using lookups' final-state'-eq by simp
have len-query: length query-idxs = rounds
using len-raw query-idxs-eq by simp
have parser-chunk-len:
   $\wedge i. i < \text{rounds} \implies$ 
    length (query-chunks ! i) =
      verifier-query-round-transcript-length (query-idxs ! i)
        (staged-trace-fri-roots data)
        (staged-composition-fri-roots data)
proof -
  fix i
  assume i-bound:  $i < \text{rounds}$ 
  show
    length (query-chunks ! i) =
      verifier-query-round-transcript-length (query-idxs ! i)
        (staged-trace-fri-roots data)
        (staged-composition-fri-roots data)
    using verifier-query-round-chunk-length[OF round-chunks[OF i-bound]]
      header-eq
    by simp
qed
have concat-staged:
  List.concat query-chunks @ trailing =
    List.concat (staged-query-chunks data)
using concat-eq header-eq by simp
have chunks-eq: query-chunks = staged-query-chunks data
using selected-query-chunks-eq-staged
  [OF builder len-query len-chunks concat-staged parser-chunk-len]
by simp
from rounds-positive obtain i where i-bound:  $i < \text{rounds}$ 
by auto
have idx-in: query-idxs ! i  $\in$  query-index-list-entries Q
using query-in i-bound len-query unfolding query-index-list-entries-def
by (auto intro: nth-mem)
have lookup:
  fmlookup (HashMap final-state)
    (QueryIndexChallenge i
      (state-after-query-chunks (staged-query-start-hash data)
        (staged-query-chunks data) i)) =

```



```

    Some (raw-idxs ! i)
  using lookups[OF i-bound] query-counter query-start-eq chunks-eq
  by simp
  have raw-hit:
    index (to-nat (raw-idxs ! i)) ∈ query-index-list-entries Q
  using idx-in query-idxs-eq i-bound len-raw by simp
  show ?thesis
    unfolding staged-security-with-data-query-index-set-hit-def
    using i-bound lookup raw-hit by auto
qed

lemma checked-staged-security-trace-fri-query-index-list-set-hit-bound:
  assumes wf: staged-budget-wellformed budgets
    and controlled: staged-adversary-controlled budgets A
  shows
    wp-event (checked-staged-security-experiment-with-data-state A)
      (staged-security-with-data-state-verifier-event
        (λs. trace-fri-query-index-list-set-hit s Q))
    adversary-initial-state ≤
    staged-phase-target-error
      (query-index-raw-preimage (query-index-list-entries Q))
      (staged-attacker-query-budget budgets +
        staged-challenge-query-budget)
  proof -
    let ?B = query-index-list-entries Q
    let ?projected =
      λout. case out of None ⇒
        staged-security-with-data-query-index-set-hit ?B None
      | Some (((data, -), result), t) ⇒
        staged-security-with-data-query-index-set-hit ?B
          (Some ((data, result), t))
    have event-le:
      wp-event (checked-staged-security-experiment-with-data-state A)
        (staged-security-with-data-state-verifier-event
          (λs. trace-fri-query-index-list-set-hit s Q))
      adversary-initial-state ≤
      wp-event (checked-staged-security-experiment-with-data-state A)
        ?projected adversary-initial-state
    proof (rule wp-event-mono-on-support)
      fix out
      assume out-support:
        out ∈
        set-dist
          (execute (checked-staged-security-experiment-with-data-state A)
            adversary-initial-state)
      assume hit:
        staged-security-with-data-state-verifier-event
          (λs. trace-fri-query-index-list-set-hit s Q) out
      show ?projected out

```

```

proof (cases out)
  case None
  then show ?thesis
    using hit
    unfolding staged-security-with-data-state-verifier-event-def
      staged-security-with-data-query-index-set-hit-def
    by simp
next
  case (Some pair)
  then obtain data attacker-state result final-state where out-eq:
    out = Some (((data, attacker-state), result), final-state)
    by (cases pair) (auto split: prod.splits)
  let ?s =
    verifier-state-from-adversary attacker-state
      (staged-proof-transcript data)
  have builder:
    Some (data, attacker-state) ∈
      set-dist
        (execute (checked-staged-transcript-program A)
          adversary-initial-state)
    using out-support
    unfolding out-eq checked-staged-security-experiment-with-data-state-def
    by (auto elim!: set-dist-bindE)
  from hit out-eq have trace-hit:
    trace-fri-query-index-list-set-hit ?s Q
      (Some (result, final-state))
    unfolding staged-security-with-data-state-verifier-event-def by simp
  from trace-hit obtain trace-roots trace-bs trace-final dg
    composition-roots composition-bs composition-final query-idxs
    trace-round-layers composition-round-layers where
    fri:
      accepted-fri-opening-transcript ?s (Some (result, final-state))
      trace-roots trace-bs trace-final dg composition-roots
      composition-bs composition-final query-idxs trace-round-layers
      composition-round-layers
    and query-in: query-idxs ∈ Q
    unfolding trace-fri-query-index-list-set-hit-def by blast
  have staged-hit:
    staged-security-with-data-query-index-set-hit
      (query-index-list-entries Q)
      (Some ((data, result), final-state))
    by (rule
      accepted-fri-opening-transcript-imp-staged-query-index-set-hit
      [OF wf controlled builder fri query-in])
  show ?thesis
    unfolding out-eq using staged-hit by simp
qed
qed
also have

```

```

wp-event (checked-staged-security-experiment-with-data-state A)
  ?projected adversary-initial-state =
wp-event (checked-staged-security-experiment-with-data A)
  (staged-security-with-data-query-index-set-hit ?B)
  adversary-initial-state
using checked-staged-security-experiment-with-data-event-from-data-state
  [of A staged-security-with-data-query-index-set-hit ?B]
by simp
also have ... ≤
  staged-phase-target-error
    (query-index-raw-preimage (query-index-list-entries Q))
    (staged-attacker-query-budget budgets + staged-challenge-query-budget)
by (rule checked-staged-security-with-data-query-index-set-hit-bound
  [OF wf controlled])
finally show ?thesis .
qed

```

lemma *checked-staged-security-composition-fri-query-index-list-set-hit-bound:*

assumes *wf: staged-budget-wellformed budgets*

and *controlled: staged-adversary-controlled budgets A*

shows

```

wp-event (checked-staged-security-experiment-with-data-state A)
  (staged-security-with-data-state-verifier-event
    (λs. composition-fri-query-index-list-set-hit s Q))
  adversary-initial-state ≤
  staged-phase-target-error
    (query-index-raw-preimage (query-index-list-entries Q))
    (staged-attacker-query-budget budgets +
      staged-challenge-query-budget)

```

proof –

let *?B = query-index-list-entries Q*

let *?projected =*

```

λout. case out of None ⇒
  staged-security-with-data-query-index-set-hit ?B None
| Some (((data, -), result), t) ⇒
  staged-security-with-data-query-index-set-hit ?B
  (Some ((data, result), t))

```

have *event-le:*

```

wp-event (checked-staged-security-experiment-with-data-state A)
  (staged-security-with-data-state-verifier-event
    (λs. composition-fri-query-index-list-set-hit s Q))
  adversary-initial-state ≤
wp-event (checked-staged-security-experiment-with-data-state A)
  ?projected adversary-initial-state

```

proof (rule *wp-event-mono-on-support*)

fix *out*

assume *out-support:*

```

out ∈
  set-dist

```

```

      (execute (checked-staged-security-experiment-with-data-state A)
        adversary-initial-state)
assume hit:
  staged-security-with-data-state-verifier-event
  (λs. composition-fri-query-index-list-set-hit s Q) out
show ?projected out
proof (cases out)
  case None
  then show ?thesis
    using hit
    unfolding staged-security-with-data-state-verifier-event-def
      staged-security-with-data-query-index-set-hit-def
    by simp
next
  case (Some pair)
  then obtain data attacker-state result final-state where out-eq:
    out = Some (((data, attacker-state), result), final-state)
    by (cases pair) (auto split: prod.splits)
  let ?s =
    verifier-state-from-adversary attacker-state
    (staged-proof-transcript data)
  have builder:
    Some (data, attacker-state) ∈
      set-dist
      (execute (checked-staged-transcript-program A)
        adversary-initial-state)
    using out-support
    unfolding out-eq checked-staged-security-experiment-with-data-state-def
    by (auto elim!: set-dist-bindE)
  from hit out-eq have composition-hit:
    composition-fri-query-index-list-set-hit ?s Q
    (Some (result, final-state))
    unfolding staged-security-with-data-state-verifier-event-def by simp
  from composition-hit obtain trace-roots trace-bs trace-final dg
    composition-roots composition-bs composition-final query-idxs
    trace-round-layers composition-round-layers where
    fri:
      accepted-fri-opening-transcript ?s (Some (result, final-state))
      trace-roots trace-bs trace-final dg composition-roots
      composition-bs composition-final query-idxs trace-round-layers
      composition-round-layers
    and query-in: query-idxs ∈ Q
    unfolding composition-fri-query-index-list-set-hit-def by blast
  have staged-hit:
    staged-security-with-data-query-index-set-hit
      (query-index-list-entries Q)
      (Some ((data, result), final-state))
    apply (rule
      accepted-fri-opening-transcript-imp-staged-query-index-set-hit

```

```

    [OF wf controlled builder fri query-in])
  .
  show ?thesis
    unfolding out-eq using staged-hit by simp
qed
qed
also have
  wp-event (checked-staged-security-experiment-with-data-state A)
    ?projected adversary-initial-state =
  wp-event (checked-staged-security-experiment-with-data A)
    (staged-security-with-data-query-index-set-hit ?B)
    adversary-initial-state
  using checked-staged-security-experiment-with-data-event-from-data-state
    [of A staged-security-with-data-query-index-set-hit ?B]
  by simp
also have ... ≤
  staged-phase-target-error
    (query-index-raw-preimage (query-index-list-entries Q))
    (staged-attacker-query-budget budgets + staged-challenge-query-budget)
  by (rule checked-staged-security-with-data-query-index-set-hit-bound
    [OF wf controlled])
  finally show ?thesis .
qed

lemma checked-staged-security-trace-fri-query-challenge-pair-set-hit-bound-from-query-projection:
  assumes wf: staged-budget-wellformed budgets
  and controlled: staged-adversary-controlled budgets A
  and projection: fst ' P ⊆ Q
  shows
    wp-event (checked-staged-security-experiment-with-data-state A)
      (staged-security-with-data-state-verifier-event
        (λs. trace-fri-query-challenge-pair-set-hit s P))
      adversary-initial-state ≤
      staged-phase-target-error
        (query-index-raw-preimage (query-index-list-entries Q))
        (staged-attacker-query-budget budgets +
          staged-challenge-query-budget)
  proof (rule order-trans)
  show wp-event (checked-staged-security-experiment-with-data-state A)
    (staged-security-with-data-state-verifier-event
      (λs. trace-fri-query-challenge-pair-set-hit s P))
    adversary-initial-state ≤
  wp-event (checked-staged-security-experiment-with-data-state A)
    (staged-security-with-data-state-verifier-event
      (λs. trace-fri-query-index-list-set-hit s Q))
    adversary-initial-state
  by (rule wp-event-mono-on-support)
    (auto simp: staged-security-with-data-state-verifier-event-def
      intro:

```

$\text{trace-fri-query-challenge-pair-set-hit-imp-query-index-list-set-hit}$
 $[OF - \text{projection}]$
 $\text{split: option.splits prod.splits})$

next

show $\text{wp-event (checked-staged-security-experiment-with-data-state } A)$
 $(\text{staged-security-with-data-state-verifier-event}$
 $(\lambda s. \text{trace-fri-query-index-list-set-hit } s \ Q))$
 $\text{adversary-initial-state}$
 $\leq \text{staged-phase-target-error}$
 $(\text{query-index-raw-preimage (query-index-list-entries } Q))$
 $(\text{staged-attacker-query-budget budgets} + \text{staged-challenge-query-budget})$

apply $(\text{rule checked-staged-security-trace-fri-query-index-list-set-hit-bound}$
 $[OF \text{ wf controlled}])$.

qed

lemma *checked-staged-security-composition-fri-query-challenge-pair-set-hit-bound-from-query-projection:*

assumes $\text{wf: staged-budget-wellformed budgets}$
and $\text{controlled: staged-adversary-controlled budgets } A$
and $\text{projection: } \bigwedge dg. \text{fst } 'P \ dg \subseteq Q$

shows

$\text{wp-event (checked-staged-security-experiment-with-data-state } A)$
 $(\text{staged-security-with-data-state-verifier-event}$
 $(\lambda s. \text{composition-fri-query-challenge-pair-set-hit } s \ P))$
 $\text{adversary-initial-state} \leq$
 $\text{staged-phase-target-error}$
 $(\text{query-index-raw-preimage (query-index-list-entries } Q))$
 $(\text{staged-attacker-query-budget budgets} +$
 $\text{staged-challenge-query-budget})$

proof $(\text{rule order-trans})$

show $\text{wp-event (checked-staged-security-experiment-with-data-state } A)$
 $(\text{staged-security-with-data-state-verifier-event}$
 $(\lambda s. \text{composition-fri-query-challenge-pair-set-hit } s \ P))$
 $\text{adversary-initial-state} \leq$

$\text{wp-event (checked-staged-security-experiment-with-data-state } A)$
 $(\text{staged-security-with-data-state-verifier-event}$
 $(\lambda s. \text{composition-fri-query-index-list-set-hit } s \ Q))$
 $\text{adversary-initial-state}$

by $(\text{rule wp-event-mono-on-support})$
 $(\text{auto simp: staged-security-with-data-state-verifier-event-def}$
 intro:
 $\text{composition-fri-query-challenge-pair-set-hit-imp-query-index-list-set-hit}$
 $[OF - \text{projection}]$
 $\text{split: option.splits prod.splits})$

next

show $\text{wp-event (checked-staged-security-experiment-with-data-state } A)$
 $(\text{staged-security-with-data-state-verifier-event}$
 $(\lambda s. \text{composition-fri-query-index-list-set-hit } s \ Q))$
 $\text{adversary-initial-state}$
 $\leq \text{staged-phase-target-error}$

(query-index-row-preimage (query-index-list-entries Q))
 (staged-attacker-query-budget budgets + staged-challenge-query-budget)
 by (rule
 checked-staged-security-composition-fri-query-index-list-set-hit-bound
 [OF wf controlled])
 qed

lemma checked-staged-security-trace-fri-sampled-query-pair-set-hit-bound-from-query-projection:
assumes wf: staged-budget-wellformed budgets
and controlled: staged-adversary-controlled budgets A
and projection:
 $\bigwedge$ trace-table roots final round-layers layers.
 $\text{fst } R \text{ trace-table roots final round-layers layers} \subseteq Q$
shows
 wp-event (checked-staged-security-experiment-with-data-state A)
 (staged-security-with-data-state-verifier-event
 ($\lambda s.$ trace-fri-sampled-query-pair-set-hit R s))
 adversary-initial-state $\leq$
 staged-phase-target-error
 (query-index-row-preimage (query-index-list-entries Q))
 (staged-attacker-query-budget budgets +
 staged-challenge-query-budget)
proof (rule order-trans)
show wp-event (checked-staged-security-experiment-with-data-state A)
 (staged-security-with-data-state-verifier-event
 ($\lambda s.$ trace-fri-sampled-query-pair-set-hit R s))
 adversary-initial-state $\leq$
 wp-event (checked-staged-security-experiment-with-data-state A)
 (staged-security-with-data-state-verifier-event
 ($\lambda s.$ trace-fri-query-index-list-set-hit s Q))
 adversary-initial-state
by (rule wp-event-mono-on-support)
 (auto simp: staged-security-with-data-state-verifier-event-def
 intro:
 trace-fri-sampled-query-pair-set-hit-imp-query-index-list-set-hit
 [OF - projection]
 split: option.splits prod.splits)
next
show wp-event (checked-staged-security-experiment-with-data-state A)
 (staged-security-with-data-state-verifier-event
 ($\lambda s.$ trace-fri-query-index-list-set-hit s Q))
 adversary-initial-state
 $\leq$ staged-phase-target-error
 (query-index-row-preimage (query-index-list-entries Q))
 (staged-attacker-query-budget budgets + staged-challenge-query-budget)
by (rule checked-staged-security-trace-fri-query-index-list-set-hit-bound
 [OF wf controlled])
 qed

lemma *checked-staged-security-composition-fri-sampled-query-pair-set-hit-bound-from-query-projection:*

assumes *wf: staged-budget-wellformed budgets*

and *controlled: staged-adversary-controlled budgets A*

and *projection:*

$\wedge$ *fri-dg composition-table roots final round-layers layers.*

fst ‘ R fri-dg composition-table roots final round-layers layers

$\subseteq Q$

shows

wp-event (checked-staged-security-experiment-with-data-state A)

(staged-security-with-data-state-verifier-event

($\lambda s.$ composition-fri-sampled-query-pair-set-hit R s))

adversary-initial-state $\leq$

staged-phase-target-error

(query-index-raw-preimage (query-index-list-entries Q))

(staged-attacker-query-budget budgets +

staged-challenge-query-budget)

proof *(rule order-trans)*

show *wp-event (checked-staged-security-experiment-with-data-state A)*

(staged-security-with-data-state-verifier-event

($\lambda s.$ composition-fri-sampled-query-pair-set-hit R s))

adversary-initial-state $\leq$

wp-event (checked-staged-security-experiment-with-data-state A)

(staged-security-with-data-state-verifier-event

($\lambda s.$ composition-fri-query-index-list-set-hit s Q))

adversary-initial-state

by *(rule wp-event-mono-on-support)*

(auto simp: staged-security-with-data-state-verifier-event-def

intro:

composition-fri-sampled-query-pair-set-hit-imp-query-index-list-set-hit

[OF - projection]

split: option.splits prod.splits)

next

show *wp-event (checked-staged-security-experiment-with-data-state A)*

(staged-security-with-data-state-verifier-event

($\lambda s.$ composition-fri-query-index-list-set-hit s Q))

adversary-initial-state

$\leq$ *staged-phase-target-error*

(query-index-raw-preimage (query-index-list-entries Q))

(staged-attacker-query-budget budgets + staged-challenge-query-budget)

by *(rule*

checked-staged-security-composition-fri-query-index-list-set-hit-bound

[OF wf controlled])

qed

lemma *trace-fri-query-challenge-pair-set-hit-split-by-pair-cover:*

assumes *hit: trace-fri-query-challenge-pair-set-hit s P out*

shows

trace-fri-query-challenge-pair-set-hit s

($P \cap (UNIV \times B)$) out $\vee$


```

      trace-fri-query-challenge-pair-set-hit s
      (P  $\cap$  (UNIV  $\times$  ( $-$  B))) out
proof -
  show ?thesis
  using hit
  unfolding trace-fri-query-challenge-pair-set-hit-def
  apply (elim exE conjE)
  apply (subgoal-tac
    (fri-query-idxs, trace-bs)  $\in$  P  $\cap$  (UNIV  $\times$  B)  $\vee$ 
    (fri-query-idxs, trace-bs)  $\in$  P  $\cap$  (UNIV  $\times$  ( $-$  B)))
  apply (erule disjE)
  apply (rule disjI1)
  apply (intro exI conjI)
  apply assumption
  apply assumption
  apply (rule disjI2)
  apply (intro exI conjI)
  apply assumption
  apply assumption
  by simp
qed

lemma composition-fri-query-challenge-pair-set-hit-split-by-pair-cover:
  assumes hit: composition-fri-query-challenge-pair-set-hit s P out
  shows
    composition-fri-query-challenge-pair-set-hit s
    ( $\lambda dg. P dg \cap$  (UNIV  $\times$  B dg)) out  $\vee$ 
    composition-fri-query-challenge-pair-set-hit s
    ( $\lambda dg. P dg \cap$  (UNIV  $\times$  ( $-$  B dg))) out
proof -
  show ?thesis
  using hit
  unfolding composition-fri-query-challenge-pair-set-hit-def
  apply (elim exE conjE)
  apply (subgoal-tac
    (fri-query-idxs, composition-bs)  $\in$  P fri-dg  $\cap$  (UNIV  $\times$  B fri-dg)  $\vee$ 
    (fri-query-idxs, composition-bs)  $\in$ 
    P fri-dg  $\cap$  (UNIV  $\times$  ( $-$  B fri-dg)))
  apply (erule disjE)
  apply (rule disjI1)
  apply (intro exI conjI)
  apply assumption
  apply assumption
  apply (rule disjI2)
  apply (intro exI conjI)
  apply assumption
  apply assumption
  by auto
qed

```

lemma *checked-staged-security-trace-fri-query-challenge-pair-set-hit-bound-from-challenge-cover-and-query-projection*

assumes *wf*: staged-budget-wellformed budgets

and *controlled*: staged-adversary-controlled budgets *A*

and *finite-B*: finite *B*

and *fresh-bound*:

wp-event (*checked-staged-security-experiment-with-data-state A*)

(*staged-security-with-data-state-verifier-event*

($\lambda s. \text{trace-fri-challenge-list-fresh-hit } s \ B$))

adversary-initial-state $\leq F$

and *projection*:

fst ' ($P \cap (UNIV \times (- B))$) $\subseteq Q$

shows

wp-event (*checked-staged-security-experiment-with-data-state A*)

(*staged-security-with-data-state-verifier-event*

($\lambda s. \text{trace-fri-query-challenge-pair-set-hit } s \ P$))

adversary-initial-state $\leq$

F +

hash-relation-budget-value (*card B* * *ceil-log clength*)

(*staged-attacker-query-budget budgets* + *staged-challenge-query-budget*) +

staged-phase-target-error

(*query-index-row-preimage* (*query-index-list-entries Q*))

(*staged-attacker-query-budget budgets* +

staged-challenge-query-budget)

proof –

let *?Inside* = $P \cap (UNIV \times B)$

let *?Outside* = $P \cap (UNIV \times (- B))$

let *?challenge-event* =

staged-security-with-data-state-verifier-event

($\lambda s. \text{trace-fri-challenge-list-set-hit } s \ B$)

let *?inside-event* =

staged-security-with-data-state-verifier-event

($\lambda s. \text{trace-fri-query-challenge-pair-set-hit } s \ ?\text{Inside}$)

let *?outside-event* =

staged-security-with-data-state-verifier-event

($\lambda s. \text{trace-fri-query-challenge-pair-set-hit } s \ ?\text{Outside}$)

let *?hbudget* =

hash-relation-budget-value (*card B* * *ceil-log clength*)

(*staged-attacker-query-budget budgets* + *staged-challenge-query-budget*)

let *?qbudget* =

staged-phase-target-error

(*query-index-row-preimage* (*query-index-list-entries Q*))

(*staged-attacker-query-budget budgets* + *staged-challenge-query-budget*)

have *split*:

wp-event (*checked-staged-security-experiment-with-data-state A*)

(*staged-security-with-data-state-verifier-event*

($\lambda s. \text{trace-fri-query-challenge-pair-set-hit } s \ P$))

adversary-initial-state $\leq$

wp-event (*checked-staged-security-experiment-with-data-state A*)

```

  (λout.
    ?inside-event out ∨ ?outside-event out)
  adversary-initial-state
proof (rule wp-event-mono-on-support)
fix out
assume hit:
  staged-security-with-data-state-verifier-event
  (λs. trace-fri-query-challenge-pair-set-hit s P) out
then show
  ?inside-event out ∨ ?outside-event out
unfolding staged-security-with-data-state-verifier-event-def
by (auto
  dest: trace-fri-query-challenge-pair-set-hit-split-by-pair-cover
  split: option.splits prod.splits)
qed
have union-bound:
  wp-event (checked-staged-security-experiment-with-data-state A)
  (λout. ?inside-event out ∨ ?outside-event out)
  adversary-initial-state ≤
  wp-event (checked-staged-security-experiment-with-data-state A)
  ?inside-event adversary-initial-state +
  wp-event (checked-staged-security-experiment-with-data-state A)
  ?outside-event adversary-initial-state
by (rule wp-event-union-bound)
have split-union-bound:
  wp-event (checked-staged-security-experiment-with-data-state A)
  (staged-security-with-data-state-verifier-event
  (λs. trace-fri-query-challenge-pair-set-hit s P))
  adversary-initial-state ≤
  wp-event (checked-staged-security-experiment-with-data-state A)
  ?inside-event adversary-initial-state +
  wp-event (checked-staged-security-experiment-with-data-state A)
  ?outside-event adversary-initial-state
apply (rule order-trans)
apply (rule split)
apply (rule union-bound)
done
have sum-bound:
  wp-event (checked-staged-security-experiment-with-data-state A)
  ?inside-event adversary-initial-state +
  wp-event (checked-staged-security-experiment-with-data-state A)
  ?outside-event adversary-initial-state ≤ (F + ?hbudget) + ?qbudget
proof –
have challenge-bound:
  wp-event (checked-staged-security-experiment-with-data-state A)
  ?challenge-event adversary-initial-state ≤ F + ?hbudget
by (rule checked-staged-security-trace-fri-challenge-list-set-hit-bound
  [OF wf controlled finite-B fresh-bound])
have inside-projection: snd ‘ ?Inside ⊆ B

```

```

by force
have inside-to-challenge:
  wp-event (checked-staged-security-experiment-with-data-state A)
    ?inside-event adversary-initial-state ≤
  wp-event (checked-staged-security-experiment-with-data-state A)
    ?challenge-event adversary-initial-state
by (rule wp-event-mono-on-support)
  (auto simp: staged-security-with-data-state-verifier-event-def
    intro:
      trace-fri-query-challenge-pair-set-hit-imp-challenge-list-set-hit
      [OF - inside-projection]
      split: option.splits prod.splits)
have inside-bound:
  wp-event (checked-staged-security-experiment-with-data-state A)
    ?inside-event adversary-initial-state ≤ F + ?hbudget
apply (rule order-trans)
apply (rule inside-to-challenge)
apply (rule challenge-bound)
done
have outside-bound:
  wp-event (checked-staged-security-experiment-with-data-state A)
    ?outside-event adversary-initial-state ≤ ?qbudget
by (rule
  checked-staged-security-trace-fri-query-challenge-pair-set-hit-bound-from-query-projection
  [OF wf controlled projection])
show ?thesis
by (rule add-mono[OF inside-bound outside-bound])
qed
show ?thesis
using order-trans[OF split-union-bound sum-bound]
by (simp add: add.assoc)
qed

lemma checked-staged-security-composition-fri-query-challenge-pair-set-hit-bound-from-challenge-cover-and-query-pr
assumes wf: staged-budget-wellformed budgets
and controlled: staged-adversary-controlled budgets A
and finite-B:  $\bigwedge dg. \text{finite } (B \ dg)$ 
and fresh-bound:
  wp-event (checked-staged-security-experiment-with-data-state A)
    (staged-security-with-data-state-verifier-event
      ( $\lambda s. \text{composition-fri-challenge-list-fresh-hit } s \ B$ ))
    adversary-initial-state ≤ F
and projection:
   $\bigwedge dg. \text{fst } ' (P \ dg \cap (UNIV \times (- \ B \ dg))) \subseteq Q$ 
shows
  wp-event (checked-staged-security-experiment-with-data-state A)
    (staged-security-with-data-state-verifier-event
      ( $\lambda s. \text{composition-fri-query-challenge-pair-set-hit } s \ P$ ))
    adversary-initial-state ≤

```

```

F +
hash-relation-budget-value
  ( $\sum dg \in (UNIV :: 'f \text{ set}).$ 
     $\text{card } (B \text{ } dg) * \text{ceil-log } (\text{maxDegree} + 1))$ 
  (staged-attacker-query-budget budgets +
    staged-challenge-query-budget) +
staged-phase-target-error
  (query-index-raw-preimage (query-index-list-entries Q))
  (staged-attacker-query-budget budgets +
    staged-challenge-query-budget)
proof –
  let ?Inside =  $\lambda dg. P \text{ } dg \cap (UNIV \times B \text{ } dg)$ 
  let ?Outside =  $\lambda dg. P \text{ } dg \cap (UNIV \times (- B \text{ } dg))$ 
  let ?challenge-event =
    staged-security-with-data-state-verifier-event
    ( $\lambda s. \text{composition-fri-challenge-list-set-hit } s \text{ } B$ )
  let ?inside-event =
    staged-security-with-data-state-verifier-event
    ( $\lambda s. \text{composition-fri-query-challenge-pair-set-hit } s \text{ } ?Inside$ )
  let ?outside-event =
    staged-security-with-data-state-verifier-event
    ( $\lambda s. \text{composition-fri-query-challenge-pair-set-hit } s \text{ } ?Outside$ )
  let ?hbudget =
    hash-relation-budget-value
    ( $\sum dg \in (UNIV :: 'f \text{ set}).$ 
       $\text{card } (B \text{ } dg) * \text{ceil-log } (\text{maxDegree} + 1))$ 
    (staged-attacker-query-budget budgets + staged-challenge-query-budget)
  let ?qbudget =
    staged-phase-target-error
    (query-index-raw-preimage (query-index-list-entries Q))
    (staged-attacker-query-budget budgets + staged-challenge-query-budget)
  have split:
    wp-event (checked-staged-security-experiment-with-data-state A)
    (staged-security-with-data-state-verifier-event
      ( $\lambda s. \text{composition-fri-query-challenge-pair-set-hit } s \text{ } P$ ))
    adversary-initial-state  $\leq$ 
    wp-event (checked-staged-security-experiment-with-data-state A)
    ( $\lambda out.$ 
      ?inside-event out  $\vee$  ?outside-event out)
    adversary-initial-state
  proof (rule wp-event-mono-on-support)
  fix out
  assume hit:
    staged-security-with-data-state-verifier-event
    ( $\lambda s. \text{composition-fri-query-challenge-pair-set-hit } s \text{ } P$ ) out
  then show
    ?inside-event out  $\vee$  ?outside-event out
  unfolding staged-security-with-data-state-verifier-event-def
  by (auto)

```

```

    dest: composition-fri-query-challenge-pair-set-hit-split-by-pair-cover
    split: option.splits prod.splits)
qed
have union-bound:
  wp-event (checked-staged-security-experiment-with-data-state A)
    (λout. ?inside-event out ∨ ?outside-event out)
    adversary-initial-state ≤
  wp-event (checked-staged-security-experiment-with-data-state A)
    ?inside-event adversary-initial-state +
  wp-event (checked-staged-security-experiment-with-data-state A)
    ?outside-event adversary-initial-state
  by (rule wp-event-union-bound)
have split-union-bound:
  wp-event (checked-staged-security-experiment-with-data-state A)
    (staged-security-with-data-state-verifier-event
      (λs. composition-fri-query-challenge-pair-set-hit s P))
    adversary-initial-state ≤
  wp-event (checked-staged-security-experiment-with-data-state A)
    ?inside-event adversary-initial-state +
  wp-event (checked-staged-security-experiment-with-data-state A)
    ?outside-event adversary-initial-state
  apply (rule order-trans)
  apply (rule split)
  apply (rule union-bound)
done
have sum-bound:
  wp-event (checked-staged-security-experiment-with-data-state A)
    ?inside-event adversary-initial-state +
  wp-event (checked-staged-security-experiment-with-data-state A)
    ?outside-event adversary-initial-state ≤ (F + ?hbudget) + ?qbudget
proof -
  have challenge-bound:
    wp-event (checked-staged-security-experiment-with-data-state A)
      ?challenge-event adversary-initial-state ≤ F + ?hbudget
    by (rule
      checked-staged-security-composition-fri-challenge-list-set-hit-bound
      [OF wf controlled finite-B fresh-bound])
  have inside-projection:  $\bigwedge dg. \text{snd } dg \subseteq B \text{ } dg$ 
    by force
  have inside-to-challenge:
    wp-event (checked-staged-security-experiment-with-data-state A)
      ?inside-event adversary-initial-state ≤
    wp-event (checked-staged-security-experiment-with-data-state A)
      ?challenge-event adversary-initial-state
    by (rule wp-event-mono-on-support)
    (auto simp: staged-security-with-data-state-verifier-event-def
      intro:
        composition-fri-query-challenge-pair-set-hit-imp-challenge-list-set-hit
        [OF - inside-projection])

```

```

      split: option.splits prod.splits)
have inside-bound:
  wp-event (checked-staged-security-experiment-with-data-state A)
    ?inside-event adversary-initial-state  $\leq F + ?hbudget$ 
  apply (rule order-trans)
  apply (rule inside-to-challenge)
  apply (rule challenge-bound)
  done
have outside-bound:
  wp-event (checked-staged-security-experiment-with-data-state A)
    ?outside-event adversary-initial-state  $\leq ?qbudget$ 
  by (rule
    checked-staged-security-composition-fri-query-challenge-pair-set-hit-bound-from-query-projection
    [OF wf controlled projection])
show ?thesis
  by (rule add-mono[OF inside-bound outside-bound])
qed
show ?thesis
  using order-trans[OF split-union-bound sum-bound]
  by (simp add: add.assoc)
qed

lemma checked-staged-security-trace-fri-sampled-query-bad-candidate-bound-from-challenge-cover-and-query-projection
assumes wf: staged-budget-wellformed budgets
and controlled: staged-adversary-controlled budgets A
and finite-B: finite B
and fresh-bound:
  wp-event (checked-staged-security-experiment-with-data-state A)
    (staged-security-with-data-state-verifier-event
      ( $\lambda s.$  trace-fri-challenge-list-fresh-hit s B))
    adversary-initial-state  $\leq F$ 
and projection:
  fst ‘
    ((trace-fri-sampled-query-bad-pair-union  $\cap$ 
      (fri-query-index-list-space  $\times$  UNIV))  $\cap$ 
      (UNIV  $\times$  ( $-$  B)))  $\subseteq Q$ 
shows
  wp-event (checked-staged-security-experiment-with-data-state A)
    (staged-security-with-data-state-verifier-event
      trace-fri-sampled-query-bad-candidate)
    adversary-initial-state  $\leq$ 
    F +
    hash-relation-budget-value (card B * ceil-log clength)
    (staged-attacker-query-budget budgets + staged-challenge-query-budget) +
    staged-phase-target-error
    (query-index-raw-preimage (query-index-list-entries Q))
    (staged-attacker-query-budget budgets +
      staged-challenge-query-budget)
proof –

```

```

let ?P =
  trace-fri-sampled-query-bad-pair-union  $\cap$ 
    (fri-query-index-list-space  $\times$  UNIV)
have wp-event (checked-staged-security-experiment-with-data-state A)
  (staged-security-with-data-state-verifier-event
    trace-fri-sampled-query-bad-candidate)
  adversary-initial-state  $\leq$ 
wp-event (checked-staged-security-experiment-with-data-state A)
  (staged-security-with-data-state-verifier-event
    ( $\lambda s$ . trace-fri-query-challenge-pair-set-hit s ?P))
  adversary-initial-state
by (rule wp-event-mono-on-support)
  (auto simp: staged-security-with-data-state-verifier-event-def
    intro:
      trace-fri-sampled-query-bad-candidate-imp-restricted-query-challenge-pair-union-hit
      split: option.splits prod.splits)
also have ...  $\leq$ 
  F +
  hash-relation-budget-value (card B * ceil-log clength)
  (staged-attacker-query-budget budgets +
    staged-challenge-query-budget) +
  staged-phase-target-error
  (query-index-raw-preimage (query-index-list-entries Q))
  (staged-attacker-query-budget budgets +
    staged-challenge-query-budget)
by (rule
  checked-staged-security-trace-fri-query-challenge-pair-set-hit-bound-from-challenge-cover-and-query-projection
    [OF wf controlled finite-B fresh-bound projection])
finally show ?thesis .
qed

lemma checked-staged-security-composition-fri-sampled-query-bad-candidate-bound-from-challenge-cover-and-query-p
assumes wf: staged-budget-wellformed budgets
and controlled: staged-adversary-controlled budgets A
and finite-B:  $\bigwedge dg$ . finite (B dg)
and fresh-bound:
wp-event (checked-staged-security-experiment-with-data-state A)
  (staged-security-with-data-state-verifier-event
    ( $\lambda s$ . composition-fri-challenge-list-fresh-hit s B))
  adversary-initial-state  $\leq$  F
and projection:
 $\bigwedge dg$ . fst ‘
  ((composition-fri-sampled-query-bad-pair-union dg  $\cap$ 
    (fri-query-index-list-space  $\times$  UNIV))  $\cap$ 
    (UNIV  $\times$  ( $-$  B dg)))  $\subseteq$  Q
shows
wp-event (checked-staged-security-experiment-with-data-state A)
  (staged-security-with-data-state-verifier-event
    composition-fri-sampled-query-bad-candidate)

```



```

    adversary-initial-state ≤
    F +
    hash-relation-budget-value
    (∑ dg ∈ (UNIV :: 'f set).
      card (B dg) * ceil-log (maxDegree + 1))
    (staged-attacker-query-budget budgets +
      staged-challenge-query-budget) +
    staged-phase-target-error
    (query-index-raw-preimage (query-index-list-entries Q))
    (staged-attacker-query-budget budgets +
      staged-challenge-query-budget)
proof -
  let ?P =
    λdg. composition-fri-sampled-query-bad-pair-union dg ∩
      (fri-query-index-list-space × UNIV)
  have wp-event (checked-staged-security-experiment-with-data-state A)
    (staged-security-with-data-state-verifier-event
      composition-fri-sampled-query-bad-candidate)
    adversary-initial-state ≤
  wp-event (checked-staged-security-experiment-with-data-state A)
    (staged-security-with-data-state-verifier-event
      (λs. composition-fri-query-challenge-pair-set-hit s ?P))
    adversary-initial-state
  by (rule wp-event-mono-on-support)
    (auto simp: staged-security-with-data-state-verifier-event-def
      intro:
        composition-fri-sampled-query-bad-candidate-imp-restricted-query-challenge-pair-union-hit
        split: option.splits prod.splits)
  also have ... ≤
    F +
    hash-relation-budget-value
    (∑ dg ∈ (UNIV :: 'f set).
      card (B dg) * ceil-log (maxDegree + 1))
    (staged-attacker-query-budget budgets +
      staged-challenge-query-budget) +
    staged-phase-target-error
    (query-index-raw-preimage (query-index-list-entries Q))
    (staged-attacker-query-budget budgets +
      staged-challenge-query-budget)
  by (rule
    checked-staged-security-composition-fri-query-challenge-pair-set-hit-bound-from-challenge-cover-and-query-projection
    [OF wf controlled finite-B fresh-bound projection])
  finally show ?thesis .
qed

lemma checked-staged-security-trace-fri-sampled-query-bad-candidate-bound-from-generic-cover:
assumes wf: staged-budget-wellformed budgets
and controlled: staged-adversary-controlled budgets A
and cover:

```

```

generic-fri-sampled-query-finite-cover trace-table-low-degree
  (Not  $\circ$  trace-table-low-degree) (clength - 1) B
and fresh-bound:
wp-event (checked-staged-security-experiment-with-data-state A)
  (staged-security-with-data-state-verifier-event
    ( $\lambda s$ . trace-fri-challenge-list-fresh-hit s B))
adversary-initial-state  $\leq F$ 
shows
wp-event (checked-staged-security-experiment-with-data-state A)
  (staged-security-with-data-state-verifier-event
    trace-fri-sampled-query-bad-candidate)
adversary-initial-state  $\leq$ 
  F +
  hash-relation-budget-value (card B * ceil-log clength)
  (staged-attacker-query-budget budgets + staged-challenge-query-budget)
proof -
have finite-B: finite B
  by (rule generic-fri-sampled-query-finite-coverD(1)[OF cover])
have projection:
  snd '
    (trace-fri-sampled-query-bad-pair-union  $\cap$ 
      (fri-query-index-list-space  $\times$  UNIV))  $\subseteq B$ 
  using generic-fri-sampled-query-finite-coverD(2)[OF cover]
  unfolding trace-fri-sampled-query-bad-pair-union-def by simp
have wp-event (checked-staged-security-experiment-with-data-state A)
  (staged-security-with-data-state-verifier-event
    trace-fri-sampled-query-bad-candidate)
  adversary-initial-state  $\leq$ 
wp-event (checked-staged-security-experiment-with-data-state A)
  (staged-security-with-data-state-verifier-event
    ( $\lambda s$ . trace-fri-challenge-list-set-hit s B))
  adversary-initial-state
by (rule wp-event-mono-on-support)
  (auto simp: staged-security-with-data-state-verifier-event-def
    intro:
      trace-fri-query-challenge-pair-set-hit-imp-challenge-list-set-hit
      [OF trace-fri-sampled-query-bad-candidate-imp-restricted-query-challenge-pair-union-hit
        projection]
      split: option.splits prod.splits)
also have ...  $\leq$ 
  F +
  hash-relation-budget-value (card B * ceil-log clength)
  (staged-attacker-query-budget budgets +
    staged-challenge-query-budget)
  by (rule checked-staged-security-trace-fri-challenge-list-set-hit-bound
    [OF wf controlled finite-B fresh-bound])
finally show ?thesis .
qed

```

lemma *checked-staged-security-composition-fri-sampled-query-bad-candidate-bound-from-generic-cover:*

assumes *wf*: staged-budget-wellformed budgets

and *controlled*: staged-adversary-controlled budgets *A*

and *cover*:

$\bigwedge dg.$ *generic-fri-sampled-query-finite-cover*

(*composition-table-low-degree* (*to-nat dg*))

(*Not* $\circ$ *composition-table-low-degree maxDegree*) (*to-nat dg*) (*B dg*)

and *fresh-bound*:

wp-event (*checked-staged-security-experiment-with-data-state A*)

(*staged-security-with-data-state-verifier-event*

($\lambda s.$ *composition-fri-challenge-list-fresh-hit s B*))

adversary-initial-state $\leq F$

shows

wp-event (*checked-staged-security-experiment-with-data-state A*)

(*staged-security-with-data-state-verifier-event*

composition-fri-sampled-query-bad-candidate)

adversary-initial-state $\leq$

F +

hash-relation-budget-value

($\sum dg \in (UNIV :: 'f \text{ set}).$

card (*B dg*) * *ceil-log* (*maxDegree* + 1))

(*staged-attacker-query-budget budgets* +

staged-challenge-query-budget)

proof –

have *finite-B*: $\bigwedge dg.$ *finite* (*B dg*)

by (*rule generic-fri-sampled-query-finite-coverD*(1)[*OF cover*])

have *projection*:

$\bigwedge dg.$ *snd* ‘

(*composition-fri-sampled-query-bad-pair-union dg* $\cap$

(*fri-query-index-list-space* $\times UNIV$)) $\subseteq B dg$

proof –

fix *dg*

show *snd* ‘

(*composition-fri-sampled-query-bad-pair-union dg* $\cap$

(*fri-query-index-list-space* $\times UNIV$)) $\subseteq B dg$

using *generic-fri-sampled-query-finite-coverD*(2)[*OF cover*[*of dg*]]

unfolding *composition-fri-sampled-query-bad-pair-union-def* **by** *simp*

qed

have *wp-event* (*checked-staged-security-experiment-with-data-state A*)

(*staged-security-with-data-state-verifier-event*

composition-fri-sampled-query-bad-candidate)

adversary-initial-state $\leq$

wp-event (*checked-staged-security-experiment-with-data-state A*)

(*staged-security-with-data-state-verifier-event*

($\lambda s.$ *composition-fri-challenge-list-set-hit s B*))

adversary-initial-state

by (*rule wp-event-mono-on-support*)

(*auto simp*: *staged-security-with-data-state-verifier-event-def*

intro:

```

      composition-fri-query-challenge-pair-set-hit-imp-challenge-list-set-hit
    [OF composition-fri-sampled-query-bad-candidate-imp-restricted-query-challenge-pair-union-hit
      projection]
    split: option.splits prod.splits)
  also have ... ≤
    F +
    hash-relation-budget-value
    (∑ dg ∈ (UNIV :: 'f set).
      card (B dg) * ceil-log (maxDegree + 1))
    (staged-attacker-query-budget budgets +
      staged-challenge-query-budget)
  by (rule checked-staged-security-composition-fri-challenge-list-set-hit-bound
    [OF wf controlled finite-B fresh-bound])
  finally show ?thesis .
qed

lemma checked-staged-security-trace-fri-sampled-layer-chain-bound-from-sampled-query:
  assumes sampled-query:
    wp-event (checked-staged-security-experiment-with-data-state A)
    (staged-security-with-data-state-verifier-event
      trace-fri-sampled-query-bad-candidate)
    adversary-initial-state ≤ R
  shows
    wp-event (checked-staged-security-experiment-with-data-state A)
    (staged-security-with-data-state-verifier-event
      trace-fri-bad-with-sampled-layer-chain)
    adversary-initial-state ≤ R
proof -
  have missing:
    wp-event (checked-staged-security-experiment-with-data-state A)
    (staged-security-with-data-state-verifier-event
      trace-fri-sampled-missing-canonical-sampled-layer-chain)
    adversary-initial-state ≤ 0
  proof (rule checked-staged-security-with-data-state-verifier-event-bound-from-cont)
    show ∧ s. ¬ trace-fri-sampled-missing-canonical-sampled-layer-chain s None
    unfolding trace-fri-sampled-missing-canonical-sampled-layer-chain-def
      trace-fri-bad-with-sampled-layer-chain-def
      accepted-fri-opening-transcript-def
      trace-fri-partial-candidate-opening-evidence-def
      trace-fri-partial-candidate-evidence-def
    by blast
  next
    fix data attacker-state
    assume
      Some (data, attacker-state) ∈
        set-dist (execute (checked-staged-transcript-program A)
          adversary-initial-state)
    show wp-event verify-monad
      (trace-fri-sampled-missing-canonical-sampled-layer-chain

```

```

      (verifier-state-from-adversary attacker-state
       (staged-proof-transcript data)))
    (verifier-state-from-adversary attacker-state
     (staged-proof-transcript data)) ≤ 0
  by (rule wp-trace-fri-missing-canonical-sampled-zero)
qed
have wp-event (checked-staged-security-experiment-with-data-state A)
  (staged-security-with-data-state-verifier-event
   trace-fri-bad-with-sampled-layer-chain)
  adversary-initial-state ≤
wp-event (checked-staged-security-experiment-with-data-state A)
  (λout.
   staged-security-with-data-state-verifier-event
    trace-fri-sampled-query-bad-candidate out ∨
   staged-security-with-data-state-verifier-event
    trace-fri-sampled-missing-canonical-sampled-layer-chain out)
  adversary-initial-state
by (rule wp-event-mono-on-support)
  (auto simp: staged-security-with-data-state-verifier-event-def
   trace-fri-sampled-query-bad-candidate-iff-canonical
   dest: trace-fri-sampled-layer-chain-canonical-or-missing
   split: option.splits prod.splits)
also have ... ≤
wp-event (checked-staged-security-experiment-with-data-state A)
  (staged-security-with-data-state-verifier-event
   trace-fri-sampled-query-bad-candidate)
  adversary-initial-state +
wp-event (checked-staged-security-experiment-with-data-state A)
  (staged-security-with-data-state-verifier-event
   trace-fri-sampled-missing-canonical-sampled-layer-chain)
  adversary-initial-state
by (rule wp-event-union-bound)
also have ... ≤ R + 0
by (rule add-mono[OF sampled-query missing])
finally show ?thesis
by simp
qed

```

lemma *checked-staged-security-composition-fri-verifier-tied-sampled-layer-chain-bound-from-sampled-query:*

assumes *sampled-query:*

```

wp-event (checked-staged-security-experiment-with-data-state A)
  (staged-security-with-data-state-verifier-event
   composition-fri-sampled-query-bad-candidate)
adversary-initial-state ≤ R

```

shows

```

wp-event (checked-staged-security-experiment-with-data-state A)
  (staged-security-with-data-state-verifier-event
   composition-fri-bad-with-verifier-tied-sampled-layer-chain)
adversary-initial-state ≤ R

```

```

proof —
  have missing:
    wp-event (checked-staged-security-experiment-with-data-state A)
      (staged-security-with-data-state-verifier-event
        composition-fri-sampled-missing-canonical-sampled-layer-chain)
      adversary-initial-state  $\leq 0$ 
  proof (rule checked-staged-security-with-data-state-verifier-event-bound-from-cont)
  show
     $\bigwedge s. \neg \text{composition-fri-sampled-missing-canonical-sampled-layer-chain}$ 
      s None
  unfolding composition-fri-sampled-missing-canonical-sampled-layer-chain-def
    composition-fri-bad-with-sampled-layer-chain-def
    accepted-fri-opening-transcript-def
    composition-fri-partial-candidate-opening-evidence-def
    composition-fri-partial-candidate-evidence-def
  by blast
  next
  fix data attacker-state
  assume
    Some (data, attacker-state) ∈
      set-dist (execute (checked-staged-transcript-program A)
        adversary-initial-state)
  show wp-event verify-monad
    (composition-fri-sampled-missing-canonical-sampled-layer-chain
      (verifier-state-from-adversary attacker-state
        (staged-proof-transcript data)))
    (verifier-state-from-adversary attacker-state
      (staged-proof-transcript data))  $\leq 0$ 
  by (rule wp-composition-fri-missing-canonical-sampled-zero)
  qed
  have wp-event (checked-staged-security-experiment-with-data-state A)
    (staged-security-with-data-state-verifier-event
      composition-fri-bad-with-verifier-tied-sampled-layer-chain)
    adversary-initial-state  $\leq$ 
  wp-event (checked-staged-security-experiment-with-data-state A)
    ( $\lambda out.$ 
      staged-security-with-data-state-verifier-event
        composition-fri-sampled-query-bad-candidate out  $\vee$ 
      staged-security-with-data-state-verifier-event
        composition-fri-sampled-missing-canonical-sampled-layer-chain out)
    adversary-initial-state
  by (rule wp-event-mono-on-support)
    (auto simp: staged-security-with-data-state-verifier-event-def
      composition-fri-sampled-query-bad-candidate-iff-canonical
      dest!: composition-fri-verifier-tied-sampled-layer-chain-imp-sampled-layer-chain
      dest: composition-fri-sampled-layer-chain-canonical-or-missing
      split: option.splits prod.splits)
  also have ...  $\leq$ 
    wp-event (checked-staged-security-experiment-with-data-state A)

```

```

    (staged-security-with-data-state-verifier-event
     composition-fri-sampled-query-bad-candidate)
  adversary-initial-state +
  wp-event (checked-staged-security-experiment-with-data-state A)
  (staged-security-with-data-state-verifier-event
   composition-fri-sampled-missing-canonical-sampled-layer-chain)
  adversary-initial-state
by (rule wp-event-union-bound)
also have ...  $\leq R + 0$ 
  by (rule add-mono[OF sampled-query missing])
finally show ?thesis
  by simp
qed

lemma checked-staged-security-trace-fri-empty-header-bound-from-sampled-query-conflict-and-zero:
assumes sampled-query:
  wp-event (checked-staged-security-experiment-with-data-state A)
  (staged-security-with-data-state-verifier-event
   trace-fri-sampled-query-bad-candidate)
  adversary-initial-state  $\leq R$ 
and conflict-bound:
 $\bigwedge$ data attacker-state.
  Some (data, attacker-state)  $\in$ 
    set-dist (execute (checked-staged-transcript-program A)
      adversary-initial-state)  $\implies$ 
  wp-event verify-monad
    (trace-fri-sampled-assignment-conflict
     (verifier-state-from-adversary attacker-state
      (staged-proof-transcript data))))
  (verifier-state-from-adversary attacker-state
   (staged-proof-transcript data))  $\leq C$ 
and zero-bound:
 $\bigwedge$ data attacker-state.
  Some (data, attacker-state)  $\in$ 
    set-dist (execute (checked-staged-transcript-program A)
      adversary-initial-state)  $\implies$ 
  wp-event verify-monad
    (trace-fri-zero-round-final-obstruction
     (verifier-state-from-adversary attacker-state
      (staged-proof-transcript data))))
  (verifier-state-from-adversary attacker-state
   (staged-proof-transcript data))  $\leq Z$ 
shows
  wp-event (checked-staged-security-experiment-with-data-state A)
  (staged-security-with-data-state-verifier-event
   trace-fri-bad-with-empty-composition-header-candidates)
  adversary-initial-state  $\leq R + (C + Z)$ 
proof (rule checked-staged-security-trace-fri-empty-header-bound-from-sampled-and-missing)
show wp-event (checked-staged-security-experiment-with-data-state A)

```

```

    (staged-security-with-data-state-verifier-event
     trace-fri-bad-with-sampled-layer-chain)
    adversary-initial-state  $\leq R$ 
  by (rule
      checked-staged-security-trace-fri-sampled-layer-chain-bound-from-sampled-query
      [OF sampled-query])
next
  fix data attacker-state
  assume support:
    Some (data, attacker-state)  $\in$ 
      set-dist (execute (checked-staged-transcript-program A)
                  adversary-initial-state)
  let ?s =
    verifier-state-from-adversary attacker-state
    (staged-proof-transcript data)
  have obstruction:
    wp-event verify-monad
      (trace-fri-sampled-layer-assignment-obstruction ?s) ?s  $\leq C + Z$ 
  by (rule
      wp-trace-fri-sampled-layer-assignment-obstruction-bound-from-conflict-and-zero)
      (rule conflict-bound[OF support], rule zero-bound[OF support])
  show wp-event verify-monad
    (trace-fri-reachable-missing-sampled-layer-chain ?s) ?s  $\leq C + Z$ 
  by (rule wp-trace-fri-reachable-missing-sampled-bound-from-assignment-obstruction)
      (rule obstruction)
qed

lemma wp-composition-fri-verifier-tied-missing-sampled-layer-chain-bound-from-residual-gaps:
  assumes slot-bound:
    wp-event verify-monad
      (composition-fri-verifier-tied-authenticated-slot-recorded-chunk-gap s) s
       $\leq \text{Slot}$ 
  and merkle-bound:
    wp-event verify-monad (partial-merkle-inconsistency-bad s) s
       $\leq \text{Merkle}$ 
  and missing-bound:
    wp-event verify-monad
      (composition-fri-verifier-tied-recorded-base-chunk-auth-missing s) s
       $\leq \text{Missing}$ 
  and next-bound:
    wp-event verify-monad
      (composition-fri-verifier-tied-next-value-replay-gap s) s
       $\leq \text{Next}$ 
  and successor-bound:
    wp-event verify-monad
      (composition-fri-verifier-tied-successor-replay-gap s) s
       $\leq \text{Successor}$ 
  and final-bound:
    wp-event verify-monad

```



```

    (composition-fri-verifier-tied-final-value-replay-gap s) s
    ≤ Final
and zero-bound:
    wp-event verify-monad
    (composition-fri-verifier-tied-zero-round-final-obstruction s) s
    ≤ Zero
shows
    wp-event verify-monad
    (composition-fri-verifier-tied-missing-sampled-layer-chain s) s
    ≤
    (((Merkle + Missing) + Next + Successor + Final + Merkle) +
     Slot + Zero)
proof –
have conflict:
    wp-event verify-monad
    (composition-fri-verifier-tied-sampled-assignment-conflict s) s
    ≤ ((Merkle + Missing) + Next + Successor + Final + Merkle) + Slot
by (rule
    wp-composition-fri-verifier-tied-sampled-assignment-conflict-bound-from-slot-recorded-chunk-gap-and-residuals
    [OF slot-bound merkle-bound missing-bound next-bound successor-bound
     final-bound])
have obstruction:
    wp-event verify-monad
    (composition-fri-verifier-tied-sampled-layer-assignment-obstruction s) s
    ≤
    (((Merkle + Missing) + Next + Successor + Final + Merkle) +
     Slot + Zero)
by (rule
    wp-composition-fri-verifier-tied-assignment-obstruction-bound-from-conflict-and-zero)
    (rule conflict, rule zero-bound)
show ?thesis
by (rule
    wp-composition-fri-verifier-tied-missing-sampled-bound-from-assignment-obstruction)
    (rule obstruction)
qed

lemma checked-staged-security-composition-fri-verifier-tied-partial-candidate-bound-from-sampled-query-and-residual
assumes sampled-query:
    wp-event (checked-staged-security-experiment-with-data-state A)
    (staged-security-with-data-state-verifier-event
     composition-fri-sampled-query-bad-candidate)
    adversary-initial-state ≤ R
and slot-bound:
    ∧ data attacker-state.
    Some (data, attacker-state) ∈
    set-dist (execute (checked-staged-transcript-program A)
     adversary-initial-state) ⇒
    wp-event verify-monad
    (composition-fri-verifier-tied-authenticated-slot-recorded-chunk-gap

```

$(\text{verifier-state-from-adversary attacker-state}$
 $(\text{staged-proof-transcript data}))$
 $(\text{verifier-state-from-adversary attacker-state}$
 $(\text{staged-proof-transcript data})) \leq \text{Slot}$
and merkle-bound:
 $\wedge \text{data attacker-state.}$
 $\text{Some (data, attacker-state)} \in$
 $\text{set-dist (execute (checked-staged-transcript-program A)}$
 $\text{adversary-initial-state)} \implies$
 $\text{wp-event verify-monad}$
 $(\text{partial-merkle-inconsistency-bad}$
 $(\text{verifier-state-from-adversary attacker-state}$
 $(\text{staged-proof-transcript data}))$
 $(\text{verifier-state-from-adversary attacker-state}$
 $(\text{staged-proof-transcript data})) \leq \text{Merkle}$
and missing-bound:
 $\wedge \text{data attacker-state.}$
 $\text{Some (data, attacker-state)} \in$
 $\text{set-dist (execute (checked-staged-transcript-program A)}$
 $\text{adversary-initial-state)} \implies$
 $\text{wp-event verify-monad}$
 $(\text{composition-fri-verifier-tied-recorded-base-chunk-auth-missing}$
 $(\text{verifier-state-from-adversary attacker-state}$
 $(\text{staged-proof-transcript data}))$
 $(\text{verifier-state-from-adversary attacker-state}$
 $(\text{staged-proof-transcript data})) \leq \text{Missing}$
and next-bound:
 $\wedge \text{data attacker-state.}$
 $\text{Some (data, attacker-state)} \in$
 $\text{set-dist (execute (checked-staged-transcript-program A)}$
 $\text{adversary-initial-state)} \implies$
 $\text{wp-event verify-monad}$
 $(\text{composition-fri-verifier-tied-next-value-replay-gap}$
 $(\text{verifier-state-from-adversary attacker-state}$
 $(\text{staged-proof-transcript data}))$
 $(\text{verifier-state-from-adversary attacker-state}$
 $(\text{staged-proof-transcript data})) \leq \text{Next}$
and successor-bound:
 $\wedge \text{data attacker-state.}$
 $\text{Some (data, attacker-state)} \in$
 $\text{set-dist (execute (checked-staged-transcript-program A)}$
 $\text{adversary-initial-state)} \implies$
 $\text{wp-event verify-monad}$
 $(\text{composition-fri-verifier-tied-successor-replay-gap}$
 $(\text{verifier-state-from-adversary attacker-state}$
 $(\text{staged-proof-transcript data}))$
 $(\text{verifier-state-from-adversary attacker-state}$
 $(\text{staged-proof-transcript data})) \leq \text{Successor}$
and final-bound:

$\wedge$ data attacker-state.
 Some (data, attacker-state) $\in$
 set-dist (execute (checked-staged-transcript-program A)
 adversary-initial-state) $\implies$
 wp-event verify-monad
 (composition-fri-verifier-tied-final-value-replay-gap
 (verifier-state-from-adversary attacker-state
 (staged-proof-transcript data)))
 (verifier-state-from-adversary attacker-state
 (staged-proof-transcript data)) $\leq$ Final
and zero-bound:
 $\wedge$ data attacker-state.
 Some (data, attacker-state) $\in$
 set-dist (execute (checked-staged-transcript-program A)
 adversary-initial-state) $\implies$
 wp-event verify-monad
 (composition-fri-verifier-tied-zero-round-final-obstruction
 (verifier-state-from-adversary attacker-state
 (staged-proof-transcript data)))
 (verifier-state-from-adversary attacker-state
 (staged-proof-transcript data)) $\leq$ Zero
shows
 wp-event (checked-staged-security-experiment-with-data-state A)
 (staged-security-with-data-state-verifier-event
 composition-fri-bad-with-verifier-tied-partial-candidate)
 adversary-initial-state $\leq$
 R + (((Merkle + Missing) + Next + Successor + Final + Merkle) +
 Slot + Zero)
proof –
have sampled:
 wp-event (checked-staged-security-experiment-with-data-state A)
 (staged-security-with-data-state-verifier-event
 composition-fri-bad-with-verifier-tied-sampled-layer-chain)
 adversary-initial-state $\leq$ R
by (rule
 checked-staged-security-composition-fri-verifier-tied-sampled-layer-chain-bound-from-sampled-query
 [OF sampled-query])
have missing:
 $\wedge$ data attacker-state.
 Some (data, attacker-state) $\in$
 set-dist (execute (checked-staged-transcript-program A)
 adversary-initial-state) $\implies$
 wp-event verify-monad
 (composition-fri-verifier-tied-missing-sampled-layer-chain
 (verifier-state-from-adversary attacker-state
 (staged-proof-transcript data)))
 (verifier-state-from-adversary attacker-state
 (staged-proof-transcript data)) $\leq$
 (((Merkle + Missing) + Next + Successor + Final + Merkle) +

```

    Slot + Zero)
proof -
  fix data attacker-state
  assume support:
    Some (data, attacker-state) ∈
      set-dist (execute (checked-staged-transcript-program A)
        adversary-initial-state)
  let ?s =
    verifier-state-from-adversary attacker-state
    (staged-proof-transcript data)
  show wp-event verify-monad
    (composition-fri-verifier-tied-missing-sampled-layer-chain ?s) ?s
    ≤
    (((Merkle + Missing) + Next + Successor + Final + Merkle) +
      Slot + Zero)
  by (rule
    wp-composition-fri-verifier-tied-missing-sampled-layer-chain-bound-from-residual-gaps)
    (rule slot-bound[OF support],
    rule merkle-bound[OF support],
    rule missing-bound[OF support],
    rule next-bound[OF support],
    rule successor-bound[OF support],
    rule final-bound[OF support],
    rule zero-bound[OF support])
  qed
show ?thesis
  by (rule
    checked-staged-security-composition-fri-verifier-tied-partial-candidate-bound-from-sampled-and-missing
    [OF sampled missing])
  qed

end

end

```

```

theory Soundness-Public-Route-Active-Base
imports
  Soundness-Public-Route-Single-Query
  Soundness-FRI-Query-Exact-Bounds
  Soundness-FRI-Query-Index-Staged-Bounds
begin

```

Base active-FRI public-route endpoint.

This theory is deliberately upstream of the downstream FRI route adapters. It exposes the common route theorem from staged active FRI event bounds, without importing the final same-run endpoint theory. This avoids an import cycle when later route layers are consumed by the final public endpoint.

```

context soundness

```

begin

definition *active-route-transcript-pre-error-for*

where

active-route-transcript-pre-error-for $A =$
 $\text{wp-event } (\text{checked-staged-security-experiment-with-data-state } A)$
 $\text{staged-security-with-data-state-transcript-pre-hit}$
 $\text{adversary-initial-state}$

definition *active-route-current-empty-query-error-for*

where

active-route-current-empty-query-error-for $A =$
 $\text{wp-event } (\text{checked-staged-security-experiment-with-data-state } A)$
 $(\text{staged-security-with-data-state-current-empty-query-opening-hit-at } 0)$
 $\text{adversary-initial-state}$

definition *active-route-current-prefix-authenticated-error-for*

where

active-route-current-prefix-authenticated-error-for $A \ i =$
 wp-event
 $(\text{checked-staged-security-experiment-with-query-prefix-data-state } A \ i)$
 $(\text{checked-staged-security-with-query-prefix-current-prefix-authenticated-hit-at}$
 $i)$
 $\text{adversary-initial-state}$

theorem *stark-soundness-from-current-public-prefix-current-query-relevant-drift-route-from-active-fri-bounds:*

fixes *trace-fri-error'* *composition-fri-error'*

empty-trace-fri-error' :: prob

assumes *false-statement: $\neg \text{exists-valid-trace}$*

and *wf: staged-budget-wellformed budgets*

and *controlled: staged-adversary-controlled budgets A*

and *trace-fri-bound:*

$\text{wp-event } (\text{checked-staged-security-experiment-with-data-state } A)$
 $(\text{staged-security-with-data-state-verifier-event}$
 $\text{trace-fri-bad-with-header-tied-partial-candidate})$
 $\text{adversary-initial-state} \leq \text{trace-fri-error}'$

and *comp-fri-bound:*

$\text{wp-event } (\text{checked-staged-security-experiment-with-data-state } A)$
 $(\text{staged-security-with-data-state-verifier-event}$
 $\text{composition-fri-bad-with-verifier-tied-partial-candidate})$
 $\text{adversary-initial-state} \leq \text{composition-fri-error}'$

and *empty-trace-fri-bound:*

$\text{wp-event } (\text{checked-staged-security-experiment-with-data-state } A)$
 $(\text{staged-security-with-data-state-verifier-event}$
 $\text{trace-fri-bad-with-empty-composition-header-candidates})$
 $\text{adversary-initial-state} \leq \text{empty-trace-fri-error}'$

shows *checked-staged-adversary-acceptance-probability A $\leq$*

hash-collision-budget-value 0

(staged-attacker-query-budget budgets +

$$\begin{aligned}
& \text{staged-challenge-query-budget} + \text{verifier-hash-query-budget}) + \\
& (\text{composition-error-bound} + \\
& \quad \text{staged-phase-relation-error size} \\
& \quad (\text{staged-alpha-search-queries budgets } 0) + \\
& \text{hash-collision-budget-value } 0 \\
& \quad (\text{staged-alpha-search-queries budgets } 0) + \\
& \quad \text{staged-phase-relation-error size} \\
& \quad (\text{staged-alpha-search-queries budgets } 0) + \\
& (\text{active-route-transcript-pre-error-for } A + \\
& \quad \text{staged-concrete-transcript-target-error-bound})) + \\
& (\sum i < \text{rounds}. \\
& \quad \text{active-route-current-prefix-authenticated-error-for } A \ i + \\
& \quad (\text{active-route-transcript-pre-error-for } A + \\
& \quad \quad \text{staged-concrete-transcript-target-error-bound}) + \\
& \quad (\text{active-route-transcript-pre-error-for } A + \\
& \quad \quad \text{staged-concrete-transcript-target-error-bound})) + \\
& \text{trace-fri-error}' + \text{composition-fri-error}' + \\
& (\text{composition-error-bound} + \\
& \quad \text{staged-phase-relation-error size} \\
& \quad (\text{staged-alpha-search-queries budgets } 0) + \\
& \text{hash-collision-budget-value } 0 \\
& \quad (\text{staged-alpha-search-queries budgets } 0) + \\
& \quad \text{staged-phase-relation-error size} \\
& \quad (\text{staged-alpha-search-queries budgets } 0) + \\
& (\text{active-route-transcript-pre-error-for } A + \\
& \quad \text{staged-concrete-transcript-target-error-bound}) + \\
& \text{empty-trace-fri-error}' + \text{active-route-current-empty-query-error-for } A)
\end{aligned}$$

proof –
have *transcript-new-bound*:

$$\begin{aligned}
& \text{wp-event } (\text{checked-staged-security-experiment-with-data-state } A) \\
& \quad \text{staged-security-with-data-state-transcript-new-hit} \\
& \quad \text{adversary-initial-state} \leq \text{staged-concrete-transcript-target-error-bound} \\
& \text{by } (\text{rule checked-staged-security-with-data-state-transcript-new-hit-bound} \\
& \quad [\text{OF wf controlled}])
\end{aligned}$$
have *data-pre-bound*:

$$\begin{aligned}
& \text{wp-event } (\text{checked-staged-security-experiment-with-data-state } A) \\
& \quad \text{staged-security-with-data-state-transcript-pre-hit} \\
& \quad \text{adversary-initial-state} \leq \text{active-route-transcript-pre-error-for } A \\
& \text{by } (\text{simp add: active-route-transcript-pre-error-for-def})
\end{aligned}$$
have *current-empty-bound*:

$$\begin{aligned}
& \text{wp-event } (\text{checked-staged-security-experiment-with-data-state } A) \\
& \quad (\text{staged-security-with-data-state-current-empty-query-opening-hit-at } 0) \\
& \quad \text{adversary-initial-state} \leq \text{active-route-current-empty-query-error-for } A \\
& \text{by } (\text{simp add: active-route-current-empty-query-error-for-def})
\end{aligned}$$
have *prefix-bound*:

$$\begin{aligned}
& \bigwedge i. i < \text{rounds} \implies \\
& \quad \text{wp-event} \\
& \quad (\text{checked-staged-security-experiment-with-query-prefix-data-state } A \ i) \\
& \quad (\text{checked-staged-security-with-query-prefix-current-prefix-authenticated-hit-at}
\end{aligned}$$

```

    i)
    adversary-initial-state ≤
    active-route-current-prefix-authenticated-error-for A i
  by (simp add: active-route-current-prefix-authenticated-error-for-def)
have trace-path-bound:
  ∧i. i < rounds ⇒
    wp-event
      (checked-staged-security-experiment-with-query-prefix-data-state A i)
      (checked-staged-security-with-query-prefix-current-trace-path-output-hit-at
        i)
      adversary-initial-state ≤
      active-route-transcript-pre-error-for A +
      staged-concrete-transcript-target-error-bound
  by (rule
    checked-staged-security-with-query-prefix-current-trace-path-output-hit-at-bound-from-transcript
    [OF - data-pre-bound transcript-new-bound])
have composition-path-bound:
  ∧i. i < rounds ⇒
    wp-event
      (checked-staged-security-experiment-with-query-prefix-data-state A i)
      (checked-staged-security-with-query-prefix-current-composition-path-output-hit-at
        i)
      adversary-initial-state ≤
      active-route-transcript-pre-error-for A +
      staged-concrete-transcript-target-error-bound
  by (rule
    checked-staged-security-with-query-prefix-current-composition-path-output-hit-at-bound-from-transcript
    [OF - data-pre-bound transcript-new-bound])
define C :: nat ⇒ prob where
  C =
    (λi. active-route-current-prefix-authenticated-error-for A i +
      (active-route-transcript-pre-error-for A +
        staged-concrete-transcript-target-error-bound) +
      (active-route-transcript-pre-error-for A +
        staged-concrete-transcript-target-error-bound))
have round-bound:
  ∧i. i < rounds ⇒
    wp-event
      (checked-staged-security-experiment-with-query-prefix-data-state A i)
      (checked-staged-security-with-query-prefix-current-query-partial-opening-hit-at
        i)
      adversary-initial-state ≤ C i
proof -
  fix i
  assume i-bound: i < rounds
  show
    wp-event
      (checked-staged-security-experiment-with-query-prefix-data-state A i)
      (checked-staged-security-with-query-prefix-current-query-partial-opening-hit-at
        i)

```

```

      i)
      adversary-initial-state  $\leq C \ i$ 
unfolding C-def
by (rule
  checked-staged-security-with-query-prefix-current-query-partial-opening-hit-at-bound-from-prefix-and-structured-
    [OF wf controlled i-bound prefix-bound[OF i-bound]
      trace-path-bound[OF i-bound] composition-path-bound[OF i-bound]])
qed
have route:
  checked-staged-adversary-acceptance-probability  $A \leq$ 
  hash-collision-budget-value 0
  (staged-attacker-query-budget budgets +
    staged-challenge-query-budget + verifier-hash-query-budget) +
  (composition-error-bound +
    staged-phase-relation-error size
    (staged-alpha-search-queries budgets 0) +
    hash-collision-budget-value 0
    (staged-alpha-search-queries budgets 0) +
    staged-phase-relation-error size
    (staged-alpha-search-queries budgets 0) +
    (active-route-transcript-pre-error-for A +
      staged-concrete-transcript-target-error-bound)) +
  trace-fri-error' + composition-fri-error' +
  ( $\sum i < \text{rounds. } C \ i$ ) +
  (composition-error-bound +
    staged-phase-relation-error size
    (staged-alpha-search-queries budgets 0) +
    hash-collision-budget-value 0
    (staged-alpha-search-queries budgets 0) +
    staged-phase-relation-error size
    (staged-alpha-search-queries budgets 0) +
    (active-route-transcript-pre-error-for A +
      staged-concrete-transcript-target-error-bound) +
    empty-trace-fri-error' +
    active-route-current-empty-query-error-for A)
by (rule
  stark-soundness-from-transcript-indexed-partial-opening-route-from-sampled-transcript-relevant-drift-only-single-
    [OF false-statement wf controlled round-bound data-pre-bound
      current-empty-bound trace-fri-bound comp-fri-bound
      empty-trace-fri-bound])
then show ?thesis
unfolding C-def by (simp add: algebra-simps)
qed

end

end

```



```

theory Soundness-FRI-Sampled-Interface-Conflict-Zero
  imports Soundness-FRI-Sampled-Interface
begin

```

Conflict/zero active-FRI wrapper for the sampled-query interface.

This narrow layer keeps the sampled-interface theory from growing further while exposing the cleaner trace route that does not carry a separate recorded-sibling-missing component.

```

context soundness
begin

```

lemma *active-fri-reductions-for-from-trace-augmented-conflict-zero-composition-sampled-query-and-empty-conflict-zero*

```

fixes TraceAug TraceConflict TraceZero TraceMerkle CompSampled CompMerkle
  EmptySampled EmptyConflict EmptyZero ::
  'f staged-proof-data  $\Rightarrow$  'f protocol-channel  $\Rightarrow$  prob

```

assumes *trace-augmented-bound*:

```

 $\wedge$ data attacker-state.
  Some (data, attacker-state)  $\in$ 
    set-dist (execute (checked-staged-transcript-program A)
      adversary-initial-state)  $\impl$ 
    wp-event verify-monad
      (trace-fri-bad-with-header-tied-augmented-partial-candidate
        (verifier-state-from-adversary attacker-state
          (staged-proof-transcript data)))
      (verifier-state-from-adversary attacker-state
        (staged-proof-transcript data))  $\leq$  TraceAug data attacker-state

```

and *trace-conflict-bound*:

```

 $\wedge$ data attacker-state.
  Some (data, attacker-state)  $\in$ 
    set-dist (execute (checked-staged-transcript-program A)
      adversary-initial-state)  $\impl$ 
    wp-event verify-monad
      (trace-fri-header-tied-sampled-assignment-conflict
        (verifier-state-from-adversary attacker-state
          (staged-proof-transcript data)))
      (verifier-state-from-adversary attacker-state
        (staged-proof-transcript data))  $\leq$  TraceConflict data attacker-state

```

and *trace-zero-bound*:

```

 $\wedge$ data attacker-state.
  Some (data, attacker-state)  $\in$ 
    set-dist (execute (checked-staged-transcript-program A)
      adversary-initial-state)  $\impl$ 
    wp-event verify-monad
      (trace-fri-header-tied-zero-round-final-obstruction
        (verifier-state-from-adversary attacker-state
          (staged-proof-transcript data)))
      (verifier-state-from-adversary attacker-state
        (staged-proof-transcript data))  $\leq$  TraceZero data attacker-state

```

and *trace-merkle-bound*:

$\wedge$ data attacker-state.
 Some (data, attacker-state) $\in$
 set-dist (execute (checked-staged-transcript-program A)
 adversary-initial-state) $\implies$
 wp-event verify-monad
 (partial-merkle-inconsistency-bad
 (verifier-state-from-adversary attacker-state
 (staged-proof-transcript data)))
 (verifier-state-from-adversary attacker-state
 (staged-proof-transcript data)) $\leq$ TraceMerkle data attacker-state
and trace-total:
 $\wedge$ data attacker-state.
 Some (data, attacker-state) $\in$
 set-dist (execute (checked-staged-transcript-program A)
 adversary-initial-state) $\implies$
 TraceAug data attacker-state +
 ((TraceConflict data attacker-state + TraceZero data attacker-state) +
 TraceMerkle data attacker-state)
 $\leq$ trace-fri-error
and composition-sampled-bound:
 $\wedge$ data attacker-state.
 Some (data, attacker-state) $\in$
 set-dist (execute (checked-staged-transcript-program A)
 adversary-initial-state) $\implies$
 wp-event verify-monad
 (composition-fri-sampled-query-bad-candidate
 (verifier-state-from-adversary attacker-state
 (staged-proof-transcript data)))
 (verifier-state-from-adversary attacker-state
 (staged-proof-transcript data)) $\leq$
 CompSampled data attacker-state
and composition-merkle-bound:
 $\wedge$ data attacker-state.
 Some (data, attacker-state) $\in$
 set-dist (execute (checked-staged-transcript-program A)
 adversary-initial-state) $\implies$
 wp-event verify-monad
 (partial-merkle-inconsistency-bad
 (verifier-state-from-adversary attacker-state
 (staged-proof-transcript data)))
 (verifier-state-from-adversary attacker-state
 (staged-proof-transcript data)) $\leq$ CompMerkle data attacker-state
and composition-total:
 $\wedge$ data attacker-state.
 Some (data, attacker-state) $\in$
 set-dist (execute (checked-staged-transcript-program A)
 adversary-initial-state) $\implies$
 CompSampled data attacker-state +
 (((CompMerkle data attacker-state + 0) +

$(\text{CompMerkle data attacker-state} + 0) +$
 $(\text{CompMerkle data attacker-state} + 0) +$
 $(\text{CompMerkle data attacker-state} + 0 + 0) +$
 $\text{CompMerkle data attacker-state}) + 0)$
 $\leq \text{composition-fri-error}$
and empty-sampled-bound:
 $\wedge \text{data attacker-state.}$
 $\text{Some (data, attacker-state)} \in$
 $\text{set-dist (execute (checked-staged-transcript-program A)}$
 $\text{adversary-initial-state)} \implies$
 $\text{wp-event verify-monad}$
 $(\text{trace-fri-sampled-query-bad-candidate}$
 $(\text{verifier-state-from-adversary attacker-state}$
 $(\text{staged-proof-transcript data})))$
 $(\text{verifier-state-from-adversary attacker-state}$
 $(\text{staged-proof-transcript data})) \leq$
 $\text{EmptySampled data attacker-state}$
and empty-conflict-bound:
 $\wedge \text{data attacker-state.}$
 $\text{Some (data, attacker-state)} \in$
 $\text{set-dist (execute (checked-staged-transcript-program A)}$
 $\text{adversary-initial-state)} \implies$
 $\text{wp-event verify-monad}$
 $(\text{trace-fri-sampled-assignment-conflict}$
 $(\text{verifier-state-from-adversary attacker-state}$
 $(\text{staged-proof-transcript data})))$
 $(\text{verifier-state-from-adversary attacker-state}$
 $(\text{staged-proof-transcript data})) \leq$
 $\text{EmptyConflict data attacker-state}$
and empty-zero-bound:
 $\wedge \text{data attacker-state.}$
 $\text{Some (data, attacker-state)} \in$
 $\text{set-dist (execute (checked-staged-transcript-program A)}$
 $\text{adversary-initial-state)} \implies$
 $\text{wp-event verify-monad}$
 $(\text{trace-fri-zero-round-final-obstruction}$
 $(\text{verifier-state-from-adversary attacker-state}$
 $(\text{staged-proof-transcript data})))$
 $(\text{verifier-state-from-adversary attacker-state}$
 $(\text{staged-proof-transcript data})) \leq$
 $\text{EmptyZero data attacker-state}$
and empty-total:
 $\wedge \text{data attacker-state.}$
 $\text{Some (data, attacker-state)} \in$
 $\text{set-dist (execute (checked-staged-transcript-program A)}$
 $\text{adversary-initial-state)} \implies$
 $\text{EmptySampled data attacker-state} +$
 $(\text{EmptyConflict data attacker-state} + \text{EmptyZero data attacker-state})$
 $\leq \text{trace-fri-error}$

shows *active-fri-reductions-for A*
proof (rule
active-fri-reductions-for-from-trace-augmented-conflict-zero-composition-sampled-and-empty-conflict-zero)
fix *data attacker-state*
assume *support*:
Some (data, attacker-state) ∈
set-dist (execute (checked-staged-transcript-program A)
adversary-initial-state)
let *?s =*
verifier-state-from-adversary attacker-state
(staged-proof-transcript data)
show *wp-event verify-monad*
(trace-fri-bad-with-header-tied-augmented-partial-candidate ?s) ?s
 $\leq$ *TraceAug data attacker-state*
by (rule *trace-augmented-bound[OF support]*)
show *wp-event verify-monad*
(trace-fri-header-tied-sampled-assignment-conflict ?s) ?s
 $\leq$ *TraceConflict data attacker-state*
by (rule *trace-conflict-bound[OF support]*)
show *wp-event verify-monad*
(trace-fri-header-tied-zero-round-final-obstruction ?s) ?s
 $\leq$ *TraceZero data attacker-state*
by (rule *trace-zero-bound[OF support]*)
show *wp-event verify-monad (partial-merkle-inconsistency-bad ?s) ?s*
 $\leq$ *TraceMerkle data attacker-state*
by (rule *trace-merkle-bound[OF support]*)
show *TraceAug data attacker-state +*
((TraceConflict data attacker-state + TraceZero data attacker-state) +
TraceMerkle data attacker-state)
 $\leq$ *trace-fri-error*
by (rule *trace-total[OF support]*)
show *wp-event verify-monad*
(composition-fri-bad-with-verifier-tied-sampled-layer-chain ?s) ?s
 $\leq$ *CompSampled data attacker-state*
by (rule *wp-composition-fri-verifier-tied-sampled-layer-chain-bound-from-sampled*)
(rule *wp-composition-fri-sampled-bound-from-canonical,*
insert composition-sampled-bound[OF support],
simp add: wp-composition-fri-sampled-query-bad-candidate-iff-canonical)
show *wp-event verify-monad (partial-merkle-inconsistency-bad ?s) ?s*
 $\leq$ *CompMerkle data attacker-state*
by (rule *composition-merkle-bound[OF support]*)
show *CompSampled data attacker-state +*
((CompMerkle data attacker-state + 0) +
(CompMerkle data attacker-state + 0) +
(CompMerkle data attacker-state + 0) +
(CompMerkle data attacker-state + 0 + 0) +
CompMerkle data attacker-state) + 0)
 $\leq$ *composition-fri-error*
by (rule *composition-total[OF support]*)

```

show wp-event verify-monad
  (trace-fri-bad-with-sampled-layer-chain ?s) ?s
  ≤ EmptySampled data attacker-state
by (rule wp-trace-fri-sampled-bound-from-canonical)
  (insert empty-sampled-bound[OF support],
    simp add: wp-trace-fri-sampled-query-bad-candidate-iff-canonical)
show wp-event verify-monad
  (trace-fri-sampled-assignment-conflict ?s) ?s
  ≤ EmptyConflict data attacker-state
by (rule empty-conflict-bound[OF support])
show wp-event verify-monad
  (trace-fri-zero-round-final-obstruction ?s) ?s
  ≤ EmptyZero data attacker-state
by (rule empty-zero-bound[OF support])
show EmptySampled data attacker-state +
  (EmptyConflict data attacker-state + EmptyZero data attacker-state)
  ≤ trace-fri-error
by (rule empty-total[OF support])
qed

end

end

```

```

theory Soundness-Public-Route-Same-Run
imports
  Soundness-Public-Route-Active-Base
  Soundness-FRI-Sampled-Interface-Conflict-Zero
begin

```

Public-route infrastructure for the staged soundness theorem. This layer assembles the checked current-empty and transcript-empty reductions and states them in the plain locale *soundness*. FRI losses are expressed as concrete staged verifier-event probabilities that are bounded in later FRI layers.

```

context soundness
begin

```

```

lemma current-query-route-bound-from-active-fri-bounds-internal:
fixes trace-fri-error' composition-fri-error'
  empty-trace-fri-error' :: prob
assumes false-statement: ¬ exists-valid-trace
and wf: staged-budget-wellformed budgets
and controlled: staged-adversary-controlled budgets A
and trace-fri-bound:
  wp-event (checked-staged-security-experiment-with-data-state A)
  (staged-security-with-data-state-verifier-event
    trace-fri-bad-with-header-tied-partial-candidate)
  adversary-initial-state ≤ trace-fri-error'

```

and *comp-fri-bound*:
 $\text{wp-event } (\text{checked-staged-security-experiment-with-data-state } A)$
 $(\text{staged-security-with-data-state-verifier-event}$
 $\text{composition-fri-bad-with-verifier-tied-partial-candidate})$
 $\text{adversary-initial-state} \leq \text{composition-fri-error}'$
and *empty-trace-fri-bound*:
 $\text{wp-event } (\text{checked-staged-security-experiment-with-data-state } A)$
 $(\text{staged-security-with-data-state-verifier-event}$
 $\text{trace-fri-bad-with-empty-composition-header-candidates})$
 $\text{adversary-initial-state} \leq \text{empty-trace-fri-error}'$
shows *checked-staged-adversary-acceptance-probability* $A \leq$
 $\text{hash-collision-budget-value } 0$
 $(\text{staged-attacker-query-budget budgets} +$
 $\text{staged-challenge-query-budget} + \text{verifier-hash-query-budget}) +$
 $(\text{composition-error-bound} +$
 $\text{staged-phase-relation-error size}$
 $(\text{staged-alpha-search-queries budgets } 0) +$
 $\text{hash-collision-budget-value } 0$
 $(\text{staged-alpha-search-queries budgets } 0) +$
 $\text{staged-phase-relation-error size}$
 $(\text{staged-alpha-search-queries budgets } 0) +$
 $(1 + \text{staged-concrete-transcript-target-error-bound})) +$
 $(\sum_{i < \text{rounds.}}$
 $1 +$
 $(1 + \text{staged-concrete-transcript-target-error-bound}) +$
 $(1 + \text{staged-concrete-transcript-target-error-bound})) +$
 $\text{trace-fri-error}' + \text{composition-fri-error}' +$
 $(\text{composition-error-bound} +$
 $\text{staged-phase-relation-error size}$
 $(\text{staged-alpha-search-queries budgets } 0) +$
 $\text{hash-collision-budget-value } 0$
 $(\text{staged-alpha-search-queries budgets } 0) +$
 $\text{staged-phase-relation-error size}$
 $(\text{staged-alpha-search-queries budgets } 0) +$
 $(1 + \text{staged-concrete-transcript-target-error-bound}) +$
 $\text{empty-trace-fri-error}' + 1)$
proof –
let *?exact* =
 $\text{hash-collision-budget-value } 0$
 $(\text{staged-attacker-query-budget budgets} +$
 $\text{staged-challenge-query-budget} + \text{verifier-hash-query-budget}) +$
 $(\text{composition-error-bound} +$
 $\text{staged-phase-relation-error size}$
 $(\text{staged-alpha-search-queries budgets } 0) +$
 $\text{hash-collision-budget-value } 0$
 $(\text{staged-alpha-search-queries budgets } 0) +$
 $\text{staged-phase-relation-error size}$
 $(\text{staged-alpha-search-queries budgets } 0) +$
 $(\text{active-route-transcript-pre-error-for } A +$

$\text{staged-concrete-transcript-target-error-bound})) +$
 $(\sum i < \text{rounds}.$
 $\text{active-route-current-prefix-authenticated-error-for } A \ i +$
 $(\text{active-route-transcript-pre-error-for } A +$
 $\text{staged-concrete-transcript-target-error-bound}) +$
 $(\text{active-route-transcript-pre-error-for } A +$
 $\text{staged-concrete-transcript-target-error-bound})) +$
 $\text{trace-fri-error}' + \text{composition-fri-error}' +$
 $(\text{composition-error-bound} +$
 $\text{staged-phase-relation-error size}$
 $(\text{staged-alpha-search-queries budgets } 0) +$
 $\text{hash-collision-budget-value } 0$
 $(\text{staged-alpha-search-queries budgets } 0) +$
 $\text{staged-phase-relation-error size}$
 $(\text{staged-alpha-search-queries budgets } 0) +$
 $(\text{active-route-transcript-pre-error-for } A +$
 $\text{staged-concrete-transcript-target-error-bound}) +$
 $\text{empty-trace-fri-error}' + \text{active-route-current-empty-query-error-for } A)$
have *route*:
 $\text{checked-staged-adversary-acceptance-probability } A \leq ?\text{exact}$
by (rule
 $\text{stark-soundness-from-current-public-prefix-current-query-relevant-drift-route-from-active-fri-bounds}$
 $[\text{OF false-statement wf controlled trace-fri-bound comp-fri-bound}$
 $\text{empty-trace-fri-bound}]$)
have *exact-le-route*:
 $? \text{exact} \leq$
 $\text{hash-collision-budget-value } 0$
 $(\text{staged-attacker-query-budget budgets} +$
 $\text{staged-challenge-query-budget} + \text{verifier-hash-query-budget}) +$
 $(\text{composition-error-bound} +$
 $\text{staged-phase-relation-error size}$
 $(\text{staged-alpha-search-queries budgets } 0) +$
 $\text{hash-collision-budget-value } 0$
 $(\text{staged-alpha-search-queries budgets } 0) +$
 $\text{staged-phase-relation-error size}$
 $(\text{staged-alpha-search-queries budgets } 0) +$
 $(1 + \text{staged-concrete-transcript-target-error-bound})) +$
 $(\sum i < \text{rounds}.$
 $1 +$
 $(1 + \text{staged-concrete-transcript-target-error-bound}) +$
 $(1 + \text{staged-concrete-transcript-target-error-bound})) +$
 $\text{trace-fri-error}' + \text{composition-fri-error}' +$
 $(\text{composition-error-bound} +$
 $\text{staged-phase-relation-error size}$
 $(\text{staged-alpha-search-queries budgets } 0) +$
 $\text{hash-collision-budget-value } 0$
 $(\text{staged-alpha-search-queries budgets } 0) +$
 $\text{staged-phase-relation-error size}$
 $(\text{staged-alpha-search-queries budgets } 0) +$

$(1 + \text{staged-concrete-transcript-target-error-bound}) +$
 $\text{empty-trace-fri-error}' + 1)$
unfolding *active-route-transcript-pre-error-for-def*
active-route-current-prefix-authenticated-error-for-def
active-route-current-empty-query-error-for-def
by (*intro add-mono sum-mono order-refl wp-event-le-1*)
show *?thesis*
by (*rule order-trans[OF route exact-le-route]*)
qed

theorem *stark-soundness-from-current-public-prefix-current-query-relevant-drift-route-from-staged-active-fri-bounds:*

fixes *trace-bad* ::
 $'f \text{ list} \Rightarrow \text{nat} \Rightarrow 'f \text{ list} \Rightarrow 'f \text{ set}$
and *composition-bad* ::
 $'f \Rightarrow 'f \text{ list} \Rightarrow \text{nat} \Rightarrow 'f \text{ list} \Rightarrow 'f \text{ set}$
and *trace-B* :: $'f \text{ list set}$
and *composition-B* :: $'f \Rightarrow 'f \text{ list set}$
and *trace-fresh trace-merkle trace-missing empty-fresh*
empty-assignment composition-fresh composition-merkle :: *prob*
and *trace-fri-error'* *composition-fri-error'*
empty-trace-fri-error' :: *prob*
assumes *false-statement*: $\neg \text{exists-valid-trace}$
and *wf*: *staged-budget-wellformed budgets*
and *controlled*: *staged-adversary-controlled budgets A*
and *trace-finite*: *finite trace-B*
and *trace-cover*:
 $\bigwedge \text{data attacker-state.}$
 $\text{Some}(\text{data}, \text{attacker-state}) \in$
 $\text{set-dist}(\text{execute}(\text{checked-staged-transcript-program } A)$
 $\text{adversary-initial-state}) \implies$
 $\text{trace-fri-sampled-layer-chain-cover}$
 $(\text{verifier-state-from-adversary attacker-state}$
 $(\text{staged-proof-transcript data})) \text{ trace-bad}$
and *trace-envelope*:
 $\bigwedge \text{data attacker-state trace-table.}$
 $\text{Some}(\text{data}, \text{attacker-state}) \in$
 $\text{set-dist}(\text{execute}(\text{checked-staged-transcript-program } A)$
 $\text{adversary-initial-state}) \implies$
 $\text{trace-fri-multi-round-bad-sets trace-bad trace-table} \subseteq \text{trace-B}$
and *trace-fresh-bound*:
 $\text{wp-event}(\text{checked-staged-security-experiment-with-data-state } A)$
 $(\text{staged-security-with-data-state-verifier-event}$
 $(\lambda s. \text{trace-fri-challenge-list-fresh-hit } s \text{ trace-B}))$
 $\text{adversary-initial-state} \leq \text{trace-fresh}$
and *trace-merkle-bound*:
 $\bigwedge \text{data attacker-state.}$
 $\text{Some}(\text{data}, \text{attacker-state}) \in$
 $\text{set-dist}(\text{execute}(\text{checked-staged-transcript-program } A)$
 $\text{adversary-initial-state}) \implies$

$wp\text{-event } verify\text{-monad}$
 $(partial\text{-merkle-inconsistency-bad}$
 $(verifier\text{-state-from-adversary } attacker\text{-state}$
 $(staged\text{-proof-transcript } data)))$
 $(verifier\text{-state-from-adversary } attacker\text{-state}$
 $(staged\text{-proof-transcript } data)) \leq trace\text{-merkle}$
and $trace\text{-missing-bound}$:
 $\wedge data \text{ attacker-state.}$
 $Some (data, attacker\text{-state}) \in$
 $set\text{-dist } (execute (checked\text{-staged-transcript-program } A)$
 $adversary\text{-initial-state}) \implies$
 $wp\text{-event } verify\text{-monad}$
 $(trace\text{-fri-header-tied-recorded-sibling-candidate-missing}$
 $(verifier\text{-state-from-adversary } attacker\text{-state}$
 $(staged\text{-proof-transcript } data)))$
 $(verifier\text{-state-from-adversary } attacker\text{-state}$
 $(staged\text{-proof-transcript } data)) \leq trace\text{-missing}$
and $trace\text{-total}$:
 $(trace\text{-fresh } +$
 $hash\text{-relation-budget-value } (card \text{ trace-B } * \text{ ceil-log } clength)$
 $(staged\text{-attacker-query-budget } budgets +$
 $staged\text{-challenge-query-budget})) +$
 $((trace\text{-missing } +$
 $(((((trace\text{-merkle } + trace\text{-merkle}) + (trace\text{-merkle } + 0) +$
 $(trace\text{-merkle } + 0) + (trace\text{-merkle } + 0 + 0)) +$
 $trace\text{-merkle}) + 0))) +$
 $(1 + trace\text{-merkle})) +$
 $(trace\text{-missing } + trace\text{-merkle})$
 $\leq trace\text{-fri-error'}$
and $empty\text{-fresh-bound}$:
 $wp\text{-event } (checked\text{-staged-security-experiment-with-data-state } A)$
 $(staged\text{-security-with-data-state-verifier-event}$
 $(\lambda s. trace\text{-fri-challenge-list-fresh-hit } s \text{ trace-B}))$
 $adversary\text{-initial-state} \leq empty\text{-fresh}$
and $empty\text{-assignment-bound}$:
 $\wedge data \text{ attacker-state.}$
 $Some (data, attacker\text{-state}) \in$
 $set\text{-dist } (execute (checked\text{-staged-transcript-program } A)$
 $adversary\text{-initial-state}) \implies$
 $wp\text{-event } verify\text{-monad}$
 $(trace\text{-fri-sampled-layer-assignment-obstruction}$
 $(verifier\text{-state-from-adversary } attacker\text{-state}$
 $(staged\text{-proof-transcript } data)))$
 $(verifier\text{-state-from-adversary } attacker\text{-state}$
 $(staged\text{-proof-transcript } data)) \leq empty\text{-assignment}$
and $empty\text{-total}$:
 $(empty\text{-fresh } +$
 $hash\text{-relation-budget-value } (card \text{ trace-B } * \text{ ceil-log } clength)$
 $(staged\text{-attacker-query-budget } budgets +$

$\leq \text{empty-trace-fri-error}' + \text{empty-assignment}$
 $\leq \text{empty-trace-fri-error}'$
and *composition-finite*: $\bigwedge dg. \text{finite } (\text{composition-B } dg)$
and *composition-cover*:
 $\bigwedge \text{data attacker-state.}$
 $\text{Some } (data, \text{attacker-state}) \in$
 $\text{set-dist } (\text{execute } (\text{checked-staged-transcript-program } A)$
 $\text{adversary-initial-state}) \implies$
 $\text{composition-fri-sampled-layer-chain-cover}$
 $(\text{verifier-state-from-adversary attacker-state}$
 $(\text{staged-proof-transcript data})) \text{ composition-bad}$
and *composition-envelope*:
 $\bigwedge \text{data attacker-state dg composition-table.}$
 $\text{Some } (data, \text{attacker-state}) \in$
 $\text{set-dist } (\text{execute } (\text{checked-staged-transcript-program } A)$
 $\text{adversary-initial-state}) \implies$
 $\text{composition-fri-multiround-bad-sets composition-bad dg composition-table}$
 $\subseteq \text{composition-B } dg$
and *composition-fresh-bound*:
 $\text{wp-event } (\text{checked-staged-security-experiment-with-data-state } A)$
 $(\text{staged-security-with-data-state-verifier-event}$
 $(\lambda s. \text{composition-fri-challenge-list-fresh-hit } s \text{ composition-B}))$
 $\text{adversary-initial-state} \leq \text{composition-fresh}$
and *composition-merkle-bound*:
 $\bigwedge \text{data attacker-state.}$
 $\text{Some } (data, \text{attacker-state}) \in$
 $\text{set-dist } (\text{execute } (\text{checked-staged-transcript-program } A)$
 $\text{adversary-initial-state}) \implies$
 $\text{wp-event verify-monad}$
 $(\text{partial-merkle-inconsistency-bad}$
 $(\text{verifier-state-from-adversary attacker-state}$
 $(\text{staged-proof-transcript data})))$
 $(\text{verifier-state-from-adversary attacker-state}$
 $(\text{staged-proof-transcript data})) \leq \text{composition-merkle}$
and *composition-total*:
 $(\text{composition-fresh} +$
 $\text{hash-relation-budget-value}$
 $(\sum dg \in (\text{UNIV} :: 'f \text{ set}).$
 $\text{card } (\text{composition-B } dg) * \text{ceil-log } (\text{maxDegree} + 1))$
 $(\text{staged-attacker-query-budget budgets} +$
 $\text{staged-challenge-query-budget})) +$
 $((\text{composition-merkle} + 0) +$
 $(\text{composition-merkle} + 0) + (\text{composition-merkle} + 0) +$
 $(\text{composition-merkle} + 0 + 0) + \text{composition-merkle}) + 0 + 0)$
 $\leq \text{composition-fri-error}'$
shows $\text{checked-staged-adversary-acceptance-probability } A \leq$
 $\text{hash-collision-budget-value } 0$
 $(\text{staged-attacker-query-budget budgets} +$
 $\text{staged-challenge-query-budget} + \text{verifier-hash-query-budget}) +$

(composition-error-bound +
 staged-phase-relation-error size
 (staged-alpha-search-queries budgets 0) +
 hash-collision-budget-value 0
 (staged-alpha-search-queries budgets 0) +
 staged-phase-relation-error size
 (staged-alpha-search-queries budgets 0) +
 (1 + staged-concrete-transcript-target-error-bound)) +
 (∑ i<rounds.
 1 +
 (1 + staged-concrete-transcript-target-error-bound) +
 (1 + staged-concrete-transcript-target-error-bound)) +
 trace-fri-error' + composition-fri-error' +
 (composition-error-bound +
 staged-phase-relation-error size
 (staged-alpha-search-queries budgets 0) +
 hash-collision-budget-value 0
 (staged-alpha-search-queries budgets 0) +
 staged-phase-relation-error size
 (staged-alpha-search-queries budgets 0) +
 (1 + staged-concrete-transcript-target-error-bound) +
 empty-trace-fri-error' + 1)
proof –
have trace-fri-bound:
 wp-event (checked-staged-security-experiment-with-data-state A)
 (staged-security-with-data-state-verifier-event
 trace-fri-bad-with-header-tied-partial-candidate)
 adversary-initial-state ≤ trace-fri-error'
proof (rule order-trans)
show wp-event (checked-staged-security-experiment-with-data-state A)
 (staged-security-with-data-state-verifier-event
 trace-fri-bad-with-header-tied-partial-candidate)
 adversary-initial-state ≤
 ((trace-fresh +
 hash-relation-budget-value (card trace-B * ceil-log clength)
 (staged-attacker-query-budget budgets +
 staged-challenge-query-budget)) +
 ((trace-missing +
 (((trace-merkle + trace-merkle) + (trace-merkle + 0) +
 (trace-merkle + 0) + (trace-merkle + 0 + 0)) +
 trace-merkle) + 0)) +
 (1 + trace-merkle))) +
 (trace-missing + trace-merkle)
by (rule
 checked-staged-security-trace-fri-header-tied-partial-candidate-bound-from-augmented-route-and-unaugmented-s
 [OF wf controlled trace-finite trace-cover trace-envelope
 trace-fresh-bound trace-merkle-bound trace-missing-bound
 wp-event-le-1])
show ... ≤ trace-fri-error'

```

    by (rule trace-total)
qed
have empty-trace-fri-bound:
  wp-event (checked-staged-security-experiment-with-data-state A)
    (staged-security-with-data-state-verifier-event
      trace-fri-bad-with-empty-composition-header-candidates)
  adversary-initial-state  $\leq$  empty-trace-fri-error'
proof (rule order-trans)
  show wp-event (checked-staged-security-experiment-with-data-state A)
    (staged-security-with-data-state-verifier-event
      trace-fri-bad-with-empty-composition-header-candidates)
  adversary-initial-state  $\leq$ 
    (empty-fresh +
      hash-relation-budget-value (card trace-B * ceil-log clength)
      (staged-attacker-query-budget budgets +
        staged-challenge-query-budget)) + empty-assignment
  by (rule
    checked-staged-security-trace-fri-empty-header-bound-from-fresh-prequery-and-assignment-obstruction
      [OF wf controlled trace-finite trace-cover trace-envelope
        empty-fresh-bound empty-assignment-bound])
  show ...  $\leq$  empty-trace-fri-error'
  by (rule empty-total)
qed
have comp-fri-bound:
  wp-event (checked-staged-security-experiment-with-data-state A)
    (staged-security-with-data-state-verifier-event
      composition-fri-bad-with-verifier-tied-partial-candidate)
  adversary-initial-state  $\leq$  composition-fri-error'
proof (rule order-trans)
  show wp-event (checked-staged-security-experiment-with-data-state A)
    (staged-security-with-data-state-verifier-event
      composition-fri-bad-with-verifier-tied-partial-candidate)
  adversary-initial-state  $\leq$ 
    (composition-fresh +
      hash-relation-budget-value
        ( $\sum dg \in (UNIV :: 'f \text{ set}).$ 
          card (composition-B dg) * ceil-log (maxDegree + 1))
      (staged-attacker-query-budget budgets +
        staged-challenge-query-budget)) +
    (((composition-merkle + 0) +
      (composition-merkle + 0) + (composition-merkle + 0) +
      (composition-merkle + 0 + 0) + composition-merkle) + 0 + 0)
  by (rule
    checked-staged-security-composition-fri-verifier-tied-partial-candidate-bound-from-fresh-prequery-zero-slot-missin
      [OF wf controlled composition-finite composition-cover
        composition-envelope composition-fresh-bound composition-merkle-bound])
  show ...  $\leq$  composition-fri-error'
  by (rule composition-total)
qed

```

show ?thesis
by (rule
 current-query-route-bound-from-active-fri-bounds-internal
 [OF false-statement wf controlled trace-fri-bound comp-fri-bound
 empty-trace-fri-bound])
qed

theorem stark-soundness-from-current-public-prefix-current-query-relevant-drift-route-from-active-fri-residual-obliga

assumes false-statement: $\neg$ exists-valid-trace

and wf: staged-budget-wellformed budgets

and controlled: staged-adversary-controlled budgets A

and residuals:

active-fri-residual-obligations-for A Rt Zt At Pt Ht Gt Nt St Ft
 Rc Ac Pc Nc Sc Gc

shows checked-staged-adversary-acceptance-probability A $\leq$

hash-collision-budget-value 0

(staged-attacker-query-budget budgets +
 staged-challenge-query-budget + verifier-hash-query-budget) +

(composition-error-bound +
 staged-phase-relation-error size

(staged-alpha-search-queries budgets 0) +

hash-collision-budget-value 0

(staged-alpha-search-queries budgets 0) +

staged-phase-relation-error size

(staged-alpha-search-queries budgets 0) +

(1 + staged-concrete-transcript-target-error-bound)) +

($\sum$ i < rounds.

1 +

(1 + staged-concrete-transcript-target-error-bound) +

(1 + staged-concrete-transcript-target-error-bound)) +

trace-fri-error + composition-fri-error +

(composition-error-bound +

staged-phase-relation-error size

(staged-alpha-search-queries budgets 0) +

hash-collision-budget-value 0

(staged-alpha-search-queries budgets 0) +

staged-phase-relation-error size

(staged-alpha-search-queries budgets 0) +

(1 + staged-concrete-transcript-target-error-bound) +

trace-fri-error + 1)

proof –

have trace-fri-bound:

wp-event (checked-staged-security-experiment-with-data-state A)

(staged-security-with-data-state-verifier-event

trace-fri-bad-with-header-tied-partial-candidate)

adversary-initial-state $\leq$ trace-fri-error

by (rule active-fri-residual-obligations-staged-trace-bound[OF residuals])

have comp-fri-bound:

wp-event (checked-staged-security-experiment-with-data-state A)

(staged-security-with-data-state-verifier-event
 composition-fri-bad-with-verifier-tied-partial-candidate)
 adversary-initial-state $\leq$ composition-fri-error
 by (rule
 active-fri-residual-obligations-staged-composition-bound[OF residuals])
have empty-trace-fri-bound:
 wp-event (checked-staged-security-experiment-with-data-state A)
 (staged-security-with-data-state-verifier-event
 trace-fri-bad-with-empty-composition-header-candidates)
 adversary-initial-state $\leq$ trace-fri-error
 by (rule
 active-fri-residual-obligations-staged-empty-header-bound[OF residuals])
show ?thesis
 by (rule
 current-query-route-bound-from-active-fri-bounds-internal
 [OF false-statement wf controlled trace-fri-bound comp-fri-bound
 empty-trace-fri-bound])
qed

theorem stark-soundness-from-current-public-prefix-current-query-relevant-drift-route-from-active-fri-residual-compo
assumes false-statement: $\neg$ exists-valid-trace
and wf: staged-budget-wellformed budgets
and controlled: staged-adversary-controlled budgets A
and trace-residuals:
 active-trace-fri-residual-obligations-for A Rt Zt At Pt Ht Gt Nt St Ft
and composition-residuals:
 active-composition-fri-residual-obligations-for A Rc Ac Pc Nc Sc Gc
and empty-trace: active-empty-trace-fri-reduction-for A
shows checked-staged-adversary-acceptance-probability A $\leq$
 hash-collision-budget-value 0
 (staged-attacker-query-budget budgets +
 staged-challenge-query-budget + verifier-hash-query-budget) +
 (composition-error-bound +
 staged-phase-relation-error size
 (staged-alpha-search-queries budgets 0) +
 hash-collision-budget-value 0
 (staged-alpha-search-queries budgets 0) +
 staged-phase-relation-error size
 (staged-alpha-search-queries budgets 0) +
 (1 + staged-concrete-transcript-target-error-bound)) +
 ($\sum$ i < rounds.
 1 +
 (1 + staged-concrete-transcript-target-error-bound) +
 (1 + staged-concrete-transcript-target-error-bound)) +
 trace-fri-error + composition-fri-error +
 (composition-error-bound +
 staged-phase-relation-error size
 (staged-alpha-search-queries budgets 0) +
 hash-collision-budget-value 0

(staged-alpha-search-queries budgets 0) +
 staged-phase-relation-error size
 (staged-alpha-search-queries budgets 0) +
 (1 + staged-concrete-transcript-target-error-bound) +
 trace-fri-error + 1)
proof –
have residuals:
 active-fri-residual-obligations-for A Rt Zt At Pt Ht Gt Nt St Ft
 Rc Ac Pc Nc Sc Gc
by (rule active-fri-residual-obligations-from-components
 [OF trace-residuals composition-residuals empty-trace])
show ?thesis
by (rule
 stark-soundness-from-current-public-prefix-current-query-relevant-drift-route-from-active-fri-residual-obligations
 [OF false-statement wf controlled residuals])
qed

theorem stark-soundness-from-current-public-prefix-current-query-relevant-drift-route-from-active-fri-reductions:
assumes false-statement: $\neg$ exists-valid-trace
and wf: staged-budget-wellformed budgets
and controlled: staged-adversary-controlled budgets A
and active-fri: active-fri-reductions-for A
shows checked-staged-adversary-acceptance-probability A $\leq$
 hash-collision-budget-value 0
 (staged-attacker-query-budget budgets +
 staged-challenge-query-budget + verifier-hash-query-budget) +
 (composition-error-bound +
 staged-phase-relation-error size
 (staged-alpha-search-queries budgets 0) +
 hash-collision-budget-value 0
 (staged-alpha-search-queries budgets 0) +
 staged-phase-relation-error size
 (staged-alpha-search-queries budgets 0) +
 (1 + staged-concrete-transcript-target-error-bound)) +
 ($\sum i < \text{rounds.}$
 1 +
 (1 + staged-concrete-transcript-target-error-bound) +
 (1 + staged-concrete-transcript-target-error-bound)) +
 trace-fri-error + composition-fri-error +
 (composition-error-bound +
 staged-phase-relation-error size
 (staged-alpha-search-queries budgets 0) +
 hash-collision-budget-value 0
 (staged-alpha-search-queries budgets 0) +
 staged-phase-relation-error size
 (staged-alpha-search-queries budgets 0) +
 (1 + staged-concrete-transcript-target-error-bound) +
 trace-fri-error + 1)
proof –

have *trace-fri-bound*:
wp-event (*checked-staged-security-experiment-with-data-state A*)
(*staged-security-with-data-state-verifier-event*
trace-fri-bad-with-header-tied-partial-candidate)
adversary-initial-state $\leq$ *trace-fri-error*
by (rule *active-fri-reductions-staged-trace-bound*[*OF active-fri*])
have *comp-fri-bound*:
wp-event (*checked-staged-security-experiment-with-data-state A*)
(*staged-security-with-data-state-verifier-event*
composition-fri-bad-with-verifier-tied-partial-candidate)
adversary-initial-state $\leq$ *composition-fri-error*
by (rule *active-fri-reductions-staged-composition-bound*[*OF active-fri*])
have *empty-trace-fri-bound*:
wp-event (*checked-staged-security-experiment-with-data-state A*)
(*staged-security-with-data-state-verifier-event*
trace-fri-bad-with-empty-composition-header-candidates)
adversary-initial-state $\leq$ *trace-fri-error*
by (rule *active-fri-reductions-staged-empty-header-bound*[*OF active-fri*])
show *?thesis*
by (rule
current-query-route-bound-from-active-fri-bounds-internal
[*OF false-statement wf controlled trace-fri-bound comp-fri-bound*
empty-trace-fri-bound])
qed

theorem *stark-soundness-from-current-public-prefix-current-query-relevant-drift-route-from-derived-active-fri-composition*

fixes *TraceAug TraceMissing TraceMerkle CompSampled CompMerkle EmptySampled EmptyMissing* ::

'f staged-proof-data $\Rightarrow$ 'f protocol-channel $\Rightarrow$ prob

assumes *false-statement*: $\neg$ *exists-valid-trace*

and *wf*: *staged-budget-wellformed budgets*

and *controlled*: *staged-adversary-controlled budgets A*

and *trace-augmented-bound*:

$\bigwedge$ *data attacker-state*.

Some (data, attacker-state) $\in$

set-dist (execute (checked-staged-transcript-program A)

adversary-initial-state) $\Rightarrow$

wp-event verify-monad

(trace-fri-bad-with-header-tied-augmented-partial-candidate

(verifier-state-from-adversary attacker-state

(staged-proof-transcript data)))

(verifier-state-from-adversary attacker-state

(staged-proof-transcript data)) $\leq$ TraceAug data attacker-state

and *trace-missing-bound*:

$\bigwedge$ *data attacker-state*.

Some (data, attacker-state) $\in$

set-dist (execute (checked-staged-transcript-program A)

adversary-initial-state) $\Rightarrow$

wp-event verify-monad

$(\text{trace-fri-header-tied-recorded-sibling-candidate-missing}$
 $(\text{verifier-state-from-adversary attacker-state}$
 $(\text{staged-proof-transcript data})))$
 $(\text{verifier-state-from-adversary attacker-state}$
 $(\text{staged-proof-transcript data})) \leq$
 $\text{TraceMissing data attacker-state}$
and *trace-merkle-bound*:
 $\wedge \text{data attacker-state.}$
 $\text{Some (data, attacker-state)} \in$
 $\text{set-dist (execute (checked-staged-transcript-program A)}$
 $\text{adversary-initial-state)} \implies$
 $\text{wp-event verify-monad}$
 $(\text{partial-merkle-inconsistency-bad}$
 $(\text{verifier-state-from-adversary attacker-state}$
 $(\text{staged-proof-transcript data})))$
 $(\text{verifier-state-from-adversary attacker-state}$
 $(\text{staged-proof-transcript data})) \leq \text{TraceMerkle data attacker-state}$
and *trace-total*:
 $\wedge \text{data attacker-state.}$
 $\text{Some (data, attacker-state)} \in$
 $\text{set-dist (execute (checked-staged-transcript-program A)}$
 $\text{adversary-initial-state)} \implies$
 $\text{TraceAug data attacker-state} +$
 $(\text{TraceMissing data attacker-state} + \text{TraceMerkle data attacker-state})$
 $\leq \text{trace-fri-error}$
and *composition-sampled-bound*:
 $\wedge \text{data attacker-state.}$
 $\text{Some (data, attacker-state)} \in$
 $\text{set-dist (execute (checked-staged-transcript-program A)}$
 $\text{adversary-initial-state)} \implies$
 $\text{wp-event verify-monad}$
 $(\text{composition-fri-bad-with-verifier-tied-sampled-layer-chain}$
 $(\text{verifier-state-from-adversary attacker-state}$
 $(\text{staged-proof-transcript data})))$
 $(\text{verifier-state-from-adversary attacker-state}$
 $(\text{staged-proof-transcript data})) \leq$
 $\text{CompSampled data attacker-state}$
and *composition-merkle-bound*:
 $\wedge \text{data attacker-state.}$
 $\text{Some (data, attacker-state)} \in$
 $\text{set-dist (execute (checked-staged-transcript-program A)}$
 $\text{adversary-initial-state)} \implies$
 $\text{wp-event verify-monad}$
 $(\text{partial-merkle-inconsistency-bad}$
 $(\text{verifier-state-from-adversary attacker-state}$
 $(\text{staged-proof-transcript data})))$
 $(\text{verifier-state-from-adversary attacker-state}$
 $(\text{staged-proof-transcript data})) \leq \text{CompMerkle data attacker-state}$
and *composition-total*:

$\wedge$ data attacker-state.
 Some (data, attacker-state) $\in$
 set-dist (execute (checked-staged-transcript-program A)
 adversary-initial-state) $\implies$
 CompSampled data attacker-state +
 (((CompMerkle data attacker-state + 0) +
 (CompMerkle data attacker-state + 0) +
 (CompMerkle data attacker-state + 0) +
 (CompMerkle data attacker-state + 0 + 0) +
 CompMerkle data attacker-state) + 0)
 $\leq$ composition-fri-error
and empty-sampled-bound:
 $\wedge$ data attacker-state.
 Some (data, attacker-state) $\in$
 set-dist (execute (checked-staged-transcript-program A)
 adversary-initial-state) $\implies$
 wp-event verify-monad
 (trace-fri-bad-with-sampled-layer-chain
 (verifier-state-from-adversary attacker-state
 (staged-proof-transcript data)))
 (verifier-state-from-adversary attacker-state
 (staged-proof-transcript data)) $\leq$
 EmptySampled data attacker-state
and empty-missing-bound:
 $\wedge$ data attacker-state.
 Some (data, attacker-state) $\in$
 set-dist (execute (checked-staged-transcript-program A)
 adversary-initial-state) $\implies$
 wp-event verify-monad
 (trace-fri-reachable-missing-sampled-layer-chain
 (verifier-state-from-adversary attacker-state
 (staged-proof-transcript data)))
 (verifier-state-from-adversary attacker-state
 (staged-proof-transcript data)) $\leq$
 EmptyMissing data attacker-state
and empty-total:
 $\wedge$ data attacker-state.
 Some (data, attacker-state) $\in$
 set-dist (execute (checked-staged-transcript-program A)
 adversary-initial-state) $\implies$
 EmptySampled data attacker-state + EmptyMissing data attacker-state
 $\leq$ trace-fri-error
shows checked-staged-adversary-acceptance-probability A $\leq$
 hash-collision-budget-value 0
 (staged-attacker-query-budget budgets +
 staged-challenge-query-budget + verifier-hash-query-budget) +
 (composition-error-bound +
 staged-phase-relation-error size
 (staged-alpha-search-queries budgets 0) +

```

hash-collision-budget-value 0
  (staged-alpha-search-queries budgets 0) +
staged-phase-relation-error size
  (staged-alpha-search-queries budgets 0) +
(1 + staged-concrete-transcript-target-error-bound)) +
(∑ i<rounds.
  1 +
  (1 + staged-concrete-transcript-target-error-bound) +
  (1 + staged-concrete-transcript-target-error-bound)) +
trace-fri-error + composition-fri-error +
(composition-error-bound +
  staged-phase-relation-error size
  (staged-alpha-search-queries budgets 0) +
hash-collision-budget-value 0
  (staged-alpha-search-queries budgets 0) +
staged-phase-relation-error size
  (staged-alpha-search-queries budgets 0) +
(1 + staged-concrete-transcript-target-error-bound) +
trace-fri-error + 1)
proof -
have active-fri: active-fri-reductions-for A
by (rule
  active-fri-reductions-for-from-trace-augmented-composition-sampled-and-empty-sampled
  [OF trace-augmented-bound trace-missing-bound trace-merkle-bound
    trace-total composition-sampled-bound composition-merkle-bound
    composition-total empty-sampled-bound empty-missing-bound
    empty-total])
show ?thesis
by (rule
  stark-soundness-from-current-public-prefix-current-query-relevant-drift-route-from-active-fri-reductions
  [OF false-statement wf controlled active-fri])
qed

theorem stark-soundness-from-current-public-prefix-current-query-relevant-drift-route-from-derived-active-fri-conflict
fixes TraceAug TraceMissing TraceMerkle CompSampled CompMerkle EmptySampled Empty-
Conflict EmptyZero ::
  'f staged-proof-data ⇒ 'f protocol-channel ⇒ prob
assumes false-statement: ¬ exists-valid-trace
and wf: staged-budget-wellformed budgets
and controlled: staged-adversary-controlled budgets A
and trace-augmented-bound:
  ∧ data attacker-state.
  Some (data, attacker-state) ∈
    set-dist (execute (checked-staged-transcript-program A)
      adversary-initial-state) ⇒
wp-event verify-monad
  (trace-fri-bad-with-header-tied-augmented-partial-candidate
    (verifier-state-from-adversary attacker-state
      (staged-proof-transcript data)))

```

$(\text{verifier-state-from-adversary attacker-state}$
 $(\text{staged-proof-transcript data})) \leq \text{TraceAug data attacker-state}$
and *trace-missing-bound*:
 $\wedge \text{data attacker-state.}$
 $\text{Some (data, attacker-state)} \in$
 $\text{set-dist (execute (checked-staged-transcript-program A)}$
 $\text{adversary-initial-state)} \implies$
 $\text{wp-event verify-monad}$
 $(\text{trace-fri-header-tied-recorded-sibling-candidate-missing}$
 $(\text{verifier-state-from-adversary attacker-state}$
 $(\text{staged-proof-transcript data})))$
 $(\text{verifier-state-from-adversary attacker-state}$
 $(\text{staged-proof-transcript data})) \leq$
 $\text{TraceMissing data attacker-state}$
and *trace-merkle-bound*:
 $\wedge \text{data attacker-state.}$
 $\text{Some (data, attacker-state)} \in$
 $\text{set-dist (execute (checked-staged-transcript-program A)}$
 $\text{adversary-initial-state)} \implies$
 $\text{wp-event verify-monad}$
 $(\text{partial-merkle-inconsistency-bad}$
 $(\text{verifier-state-from-adversary attacker-state}$
 $(\text{staged-proof-transcript data})))$
 $(\text{verifier-state-from-adversary attacker-state}$
 $(\text{staged-proof-transcript data})) \leq \text{TraceMerkle data attacker-state}$
and *trace-total*:
 $\wedge \text{data attacker-state.}$
 $\text{Some (data, attacker-state)} \in$
 $\text{set-dist (execute (checked-staged-transcript-program A)}$
 $\text{adversary-initial-state)} \implies$
 $\text{TraceAug data attacker-state} +$
 $(\text{TraceMissing data attacker-state} + \text{TraceMerkle data attacker-state})$
 $\leq \text{trace-fri-error}$
and *composition-sampled-bound*:
 $\wedge \text{data attacker-state.}$
 $\text{Some (data, attacker-state)} \in$
 $\text{set-dist (execute (checked-staged-transcript-program A)}$
 $\text{adversary-initial-state)} \implies$
 $\text{wp-event verify-monad}$
 $(\text{composition-fri-bad-with-verifier-tied-sampled-layer-chain}$
 $(\text{verifier-state-from-adversary attacker-state}$
 $(\text{staged-proof-transcript data})))$
 $(\text{verifier-state-from-adversary attacker-state}$
 $(\text{staged-proof-transcript data})) \leq$
 $\text{CompSampled data attacker-state}$
and *composition-merkle-bound*:
 $\wedge \text{data attacker-state.}$
 $\text{Some (data, attacker-state)} \in$
 $\text{set-dist (execute (checked-staged-transcript-program A)}$

$\text{adversary-initial-state}) \implies$
 $\text{wp-event verify-monad}$
 $(\text{partial-merkle-inconsistency-bad}$
 $(\text{verifier-state-from-adversary attacker-state}$
 $(\text{staged-proof-transcript data})))$
 $(\text{verifier-state-from-adversary attacker-state}$
 $(\text{staged-proof-transcript data})) \leq \text{CompMerkle data attacker-state}$
and $\text{composition-total:}$
 $\wedge \text{data attacker-state.}$
 $\text{Some (data, attacker-state)} \in$
 $\text{set-dist (execute (checked-staged-transcript-program A)}$
 $\text{adversary-initial-state}) \implies}$
 $\text{CompSampled data attacker-state} +$
 $((\text{CompMerkle data attacker-state} + 0) +$
 $(\text{CompMerkle data attacker-state} + 0) +$
 $(\text{CompMerkle data attacker-state} + 0) +$
 $(\text{CompMerkle data attacker-state} + 0 + 0) +$
 $\text{CompMerkle data attacker-state}) + 0)$
 $\leq \text{composition-fri-error}$
and $\text{empty-sampled-bound:}$
 $\wedge \text{data attacker-state.}$
 $\text{Some (data, attacker-state)} \in$
 $\text{set-dist (execute (checked-staged-transcript-program A)}$
 $\text{adversary-initial-state}) \implies}$
 $\text{wp-event verify-monad}$
 $(\text{trace-fri-bad-with-sampled-layer-chain}$
 $(\text{verifier-state-from-adversary attacker-state}$
 $(\text{staged-proof-transcript data})))$
 $(\text{verifier-state-from-adversary attacker-state}$
 $(\text{staged-proof-transcript data})) \leq$
 $\text{EmptySampled data attacker-state}$
and $\text{empty-conflict-bound:}$
 $\wedge \text{data attacker-state.}$
 $\text{Some (data, attacker-state)} \in$
 $\text{set-dist (execute (checked-staged-transcript-program A)}$
 $\text{adversary-initial-state}) \implies}$
 $\text{wp-event verify-monad}$
 $(\text{trace-fri-sampled-assignment-conflict}$
 $(\text{verifier-state-from-adversary attacker-state}$
 $(\text{staged-proof-transcript data})))$
 $(\text{verifier-state-from-adversary attacker-state}$
 $(\text{staged-proof-transcript data})) \leq$
 $\text{EmptyConflict data attacker-state}$
and empty-zero-bound:
 $\wedge \text{data attacker-state.}$
 $\text{Some (data, attacker-state)} \in$
 $\text{set-dist (execute (checked-staged-transcript-program A)}$
 $\text{adversary-initial-state}) \implies}$
 $\text{wp-event verify-monad}$

(trace-fri-zero-round-final-obstruction
 (verifier-state-from-adversary attacker-state
 (staged-proof-transcript data)))
 (verifier-state-from-adversary attacker-state
 (staged-proof-transcript data)) ≤
 EmptyZero data attacker-state
and empty-total:
 ∧ data attacker-state.
 Some (data, attacker-state) ∈
 set-dist (execute (checked-staged-transcript-program A)
 adversary-initial-state) ⇒
 EmptySampled data attacker-state +
 (EmptyConflict data attacker-state + EmptyZero data attacker-state)
 ≤ trace-fri-error
shows checked-staged-adversary-acceptance-probability A ≤
 hash-collision-budget-value 0
 (staged-attacker-query-budget budgets +
 staged-challenge-query-budget + verifier-hash-query-budget) +
 (composition-error-bound +
 staged-phase-relation-error size
 (staged-alpha-search-queries budgets 0) +
 hash-collision-budget-value 0
 (staged-alpha-search-queries budgets 0) +
 staged-phase-relation-error size
 (staged-alpha-search-queries budgets 0) +
 (1 + staged-concrete-transcript-target-error-bound)) +
 (∑ i < rounds.
 1 +
 (1 + staged-concrete-transcript-target-error-bound) +
 (1 + staged-concrete-transcript-target-error-bound)) +
 trace-fri-error + composition-fri-error +
 (composition-error-bound +
 staged-phase-relation-error size
 (staged-alpha-search-queries budgets 0) +
 hash-collision-budget-value 0
 (staged-alpha-search-queries budgets 0) +
 staged-phase-relation-error size
 (staged-alpha-search-queries budgets 0) +
 (1 + staged-concrete-transcript-target-error-bound) +
 trace-fri-error + 1)
proof –
have active-fri: active-fri-reductions-for A
by (rule
 active-fri-reductions-for-from-trace-augmented-composition-sampled-and-empty-conflict-zero
 [OF trace-augmented-bound trace-missing-bound trace-merkle-bound
 trace-total composition-sampled-bound composition-merkle-bound
 composition-total empty-sampled-bound empty-conflict-bound
 empty-zero-bound empty-total])
show ?thesis

by (rule
 stark-soundness-from-current-public-prefix-current-query-relevant-drift-route-from-active-fri-reductions
 [*OF false-statement wf controlled active-fri*])
 qed

theorem *stark-soundness-from-current-public-prefix-current-query-relevant-drift-route-from-derived-active-fri-canonical*
fixes *TraceAug TraceMissing TraceMerkle CompCanonical CompMerkle EmptyCanonical EmptyConflict EmptyZero* ::
'f staged-proof-data ⇒ 'f protocol-channel ⇒ prob
assumes *false-statement: ¬ exists-valid-trace*
and *wf: staged-budget-wellformed budgets*
and *controlled: staged-adversary-controlled budgets A*
and *trace-augmented-bound:*
 $\bigwedge \text{data attacker-state.}$
Some (data, attacker-state) ∈
set-dist (execute (checked-staged-transcript-program A)
adversary-initial-state) ⇒
wp-event verify-monad
(trace-fri-bad-with-header-tied-augmented-partial-candidate
(verifier-state-from-adversary attacker-state
(staged-proof-transcript data)))
(verifier-state-from-adversary attacker-state
(staged-proof-transcript data)) ≤ TraceAug data attacker-state
and *trace-missing-bound:*
 $\bigwedge \text{data attacker-state.}$
Some (data, attacker-state) ∈
set-dist (execute (checked-staged-transcript-program A)
adversary-initial-state) ⇒
wp-event verify-monad
(trace-fri-header-tied-recorded-sibling-candidate-missing
(verifier-state-from-adversary attacker-state
(staged-proof-transcript data)))
(verifier-state-from-adversary attacker-state
(staged-proof-transcript data)) ≤
TraceMissing data attacker-state
and *trace-merkle-bound:*
 $\bigwedge \text{data attacker-state.}$
Some (data, attacker-state) ∈
set-dist (execute (checked-staged-transcript-program A)
adversary-initial-state) ⇒
wp-event verify-monad
(partial-merkle-inconsistency-bad
(verifier-state-from-adversary attacker-state
(staged-proof-transcript data)))
(verifier-state-from-adversary attacker-state
(staged-proof-transcript data)) ≤ TraceMerkle data attacker-state
and *trace-total:*
 $\bigwedge \text{data attacker-state.}$
Some (data, attacker-state) ∈

$\text{set-dist } (\text{execute } (\text{checked-staged-transcript-program } A) \text{ adversary-initial-state}) \implies$
 $\text{TraceAug data attacker-state} +$
 $(\text{TraceMissing data attacker-state} + \text{TraceMerkle data attacker-state})$
 $\leq \text{trace-fri-error}$
and *composition-canonical-bound*:
 $\wedge \text{data attacker-state.}$
 $\text{Some } (data, \text{attacker-state}) \in$
 $\text{set-dist } (\text{execute } (\text{checked-staged-transcript-program } A) \text{ adversary-initial-state}) \implies$
 $\text{wp-event verify-monad}$
 $(\text{composition-fri-bad-with-canonical-sampled-layer-chain}$
 $(\text{verifier-state-from-adversary attacker-state}$
 $(\text{staged-proof-transcript data})))$
 $(\text{verifier-state-from-adversary attacker-state}$
 $(\text{staged-proof-transcript data})) \leq$
 $\text{CompCanonical data attacker-state}$
and *composition-merkle-bound*:
 $\wedge \text{data attacker-state.}$
 $\text{Some } (data, \text{attacker-state}) \in$
 $\text{set-dist } (\text{execute } (\text{checked-staged-transcript-program } A) \text{ adversary-initial-state}) \implies$
 $\text{wp-event verify-monad}$
 $(\text{partial-merkle-inconsistency-bad}$
 $(\text{verifier-state-from-adversary attacker-state}$
 $(\text{staged-proof-transcript data})))$
 $(\text{verifier-state-from-adversary attacker-state}$
 $(\text{staged-proof-transcript data})) \leq \text{CompMerkle data attacker-state}$
and *composition-total*:
 $\wedge \text{data attacker-state.}$
 $\text{Some } (data, \text{attacker-state}) \in$
 $\text{set-dist } (\text{execute } (\text{checked-staged-transcript-program } A) \text{ adversary-initial-state}) \implies$
 $\text{CompCanonical data attacker-state} +$
 $((\text{CompMerkle data attacker-state} + 0) +$
 $(\text{CompMerkle data attacker-state} + 0) +$
 $(\text{CompMerkle data attacker-state} + 0) +$
 $(\text{CompMerkle data attacker-state} + 0 + 0) +$
 $\text{CompMerkle data attacker-state}) + 0)$
 $\leq \text{composition-fri-error}$
and *empty-canonical-bound*:
 $\wedge \text{data attacker-state.}$
 $\text{Some } (data, \text{attacker-state}) \in$
 $\text{set-dist } (\text{execute } (\text{checked-staged-transcript-program } A) \text{ adversary-initial-state}) \implies$
 $\text{wp-event verify-monad}$
 $(\text{trace-fri-bad-with-canonical-sampled-layer-chain}$
 $(\text{verifier-state-from-adversary attacker-state}$
 $(\text{staged-proof-transcript data})))$

$(\text{verifier-state-from-adversary attacker-state}$
 $(\text{staged-proof-transcript data})) \leq$
 $\text{EmptyCanonical data attacker-state}$
and *empty-conflict-bound*:
 $\wedge \text{data attacker-state.}$
 $\text{Some (data, attacker-state)} \in$
 $\text{set-dist (execute (checked-staged-transcript-program A)}$
 $\text{adversary-initial-state)} \implies$
 $\text{wp-event verify-monad}$
 $(\text{trace-fri-sampled-assignment-conflict}$
 $(\text{verifier-state-from-adversary attacker-state}$
 $(\text{staged-proof-transcript data})))$
 $(\text{verifier-state-from-adversary attacker-state}$
 $(\text{staged-proof-transcript data})) \leq$
 $\text{EmptyConflict data attacker-state}$
and *empty-zero-bound*:
 $\wedge \text{data attacker-state.}$
 $\text{Some (data, attacker-state)} \in$
 $\text{set-dist (execute (checked-staged-transcript-program A)}$
 $\text{adversary-initial-state)} \implies$
 $\text{wp-event verify-monad}$
 $(\text{trace-fri-zero-round-final-obstruction}$
 $(\text{verifier-state-from-adversary attacker-state}$
 $(\text{staged-proof-transcript data})))$
 $(\text{verifier-state-from-adversary attacker-state}$
 $(\text{staged-proof-transcript data})) \leq$
 $\text{EmptyZero data attacker-state}$
and *empty-total*:
 $\wedge \text{data attacker-state.}$
 $\text{Some (data, attacker-state)} \in$
 $\text{set-dist (execute (checked-staged-transcript-program A)}$
 $\text{adversary-initial-state)} \implies$
 $\text{EmptyCanonical data attacker-state} +$
 $(\text{EmptyConflict data attacker-state} + \text{EmptyZero data attacker-state})$
 $\leq \text{trace-fri-error}$
shows *checked-staged-adversary-acceptance-probability A* $\leq$
 $\text{hash-collision-budget-value } 0$
 $(\text{staged-attacker-query-budget budgets} +$
 $\text{staged-challenge-query-budget} + \text{verifier-hash-query-budget}) +$
 $(\text{composition-error-bound} +$
 $\text{staged-phase-relation-error size}$
 $(\text{staged-alpha-search-queries budgets } 0) +$
 $\text{hash-collision-budget-value } 0$
 $(\text{staged-alpha-search-queries budgets } 0) +$
 $\text{staged-phase-relation-error size}$
 $(\text{staged-alpha-search-queries budgets } 0) +$
 $(1 + \text{staged-concrete-transcript-target-error-bound})) +$
 $(\sum_{i < \text{rounds.}}$
 $1 +$

$(1 + \text{staged-concrete-transcript-target-error-bound}) +$
 $(1 + \text{staged-concrete-transcript-target-error-bound})) +$
 $\text{trace-fri-error} + \text{composition-fri-error} +$
 $(\text{composition-error-bound} +$
 $\text{staged-phase-relation-error size}$
 $(\text{staged-alpha-search-queries budgets } 0) +$
 $\text{hash-collision-budget-value } 0$
 $(\text{staged-alpha-search-queries budgets } 0) +$
 $\text{staged-phase-relation-error size}$
 $(\text{staged-alpha-search-queries budgets } 0) +$
 $(1 + \text{staged-concrete-transcript-target-error-bound}) +$
 $\text{trace-fri-error} + 1)$

proof –

have *active-fri: active-fri-reductions-for A*

by (rule
active-fri-reductions-for-from-trace-augmented-composition-canonical-and-empty-conflict-zero
 $[OF \text{ trace-augmented-bound trace-missing-bound trace-merkle-bound}$
 $\text{trace-total composition-canonical-bound composition-merkle-bound}$
 $\text{composition-total empty-canonical-bound empty-conflict-bound}$
 $\text{empty-zero-bound empty-total}]$)

show *?thesis*

by (rule
stark-soundness-from-current-public-prefix-current-query-relevant-drift-route-from-active-fri-reductions
 $[OF \text{ false-statement wf controlled active-fri}]$)

qed

theorem *stark-soundness-from-current-public-prefix-current-query-relevant-drift-route-from-sampled-query-fri-compo*

fixes *TraceAug TraceConflict TraceZero TraceMerkle CompSampled CompMerkle EmptySam-*
pled EmptyConflict EmptyZero ::

'f staged-proof-data $\Rightarrow$ 'f protocol-channel $\Rightarrow$ prob

assumes *false-statement: $\neg$ exists-valid-trace*

and *wf: staged-budget-wellformed budgets*

and *controlled: staged-adversary-controlled budgets A*

and *trace-augmented-bound:*

$\bigwedge \text{data attacker-state.}$
 $\text{Some (data, attacker-state)} \in$
 $\text{set-dist (execute (checked-staged-transcript-program A)}$
 $\text{adversary-initial-state)} \implies$
 $\text{wp-event verify-monad}$
 $(\text{trace-fri-bad-with-header-tied-augmented-partial-candidate}$
 $(\text{verifier-state-from-adversary attacker-state}$
 $(\text{staged-proof-transcript data})))$
 $(\text{verifier-state-from-adversary attacker-state}$
 $(\text{staged-proof-transcript data})) \leq \text{TraceAug data attacker-state}$

and *trace-conflict-bound:*

$\bigwedge \text{data attacker-state.}$
 $\text{Some (data, attacker-state)} \in$
 $\text{set-dist (execute (checked-staged-transcript-program A)}$
 $\text{adversary-initial-state)} \implies$

$\text{wp-event verify-monad}$
 $(\text{trace-fri-header-tied-sampled-assignment-conflict}$
 $(\text{verifier-state-from-adversary attacker-state}$
 $(\text{staged-proof-transcript data})))$
 $(\text{verifier-state-from-adversary attacker-state}$
 $(\text{staged-proof-transcript data})) \leq \text{TraceConflict data attacker-state}$
and trace-zero-bound:
 $\wedge \text{data attacker-state.}$
 $\text{Some (data, attacker-state)} \in$
 $\text{set-dist (execute (checked-staged-transcript-program A)}$
 $\text{adversary-initial-state)} \implies$
 $\text{wp-event verify-monad}$
 $(\text{trace-fri-header-tied-zero-round-final-obstruction}$
 $(\text{verifier-state-from-adversary attacker-state}$
 $(\text{staged-proof-transcript data})))$
 $(\text{verifier-state-from-adversary attacker-state}$
 $(\text{staged-proof-transcript data})) \leq \text{TraceZero data attacker-state}$
and $\text{trace-merkle-bound:}$
 $\wedge \text{data attacker-state.}$
 $\text{Some (data, attacker-state)} \in$
 $\text{set-dist (execute (checked-staged-transcript-program A)}$
 $\text{adversary-initial-state)} \implies$
 $\text{wp-event verify-monad}$
 $(\text{partial-merkle-inconsistency-bad}$
 $(\text{verifier-state-from-adversary attacker-state}$
 $(\text{staged-proof-transcript data})))$
 $(\text{verifier-state-from-adversary attacker-state}$
 $(\text{staged-proof-transcript data})) \leq \text{TraceMerkle data attacker-state}$
and trace-total:
 $\wedge \text{data attacker-state.}$
 $\text{Some (data, attacker-state)} \in$
 $\text{set-dist (execute (checked-staged-transcript-program A)}$
 $\text{adversary-initial-state)} \implies$
 $\text{TraceAug data attacker-state} +$
 $((\text{TraceConflict data attacker-state} + \text{TraceZero data attacker-state}) +$
 $\text{TraceMerkle data attacker-state})$
 $\leq \text{trace-fri-error}$
and $\text{composition-sampled-bound:}$
 $\wedge \text{data attacker-state.}$
 $\text{Some (data, attacker-state)} \in$
 $\text{set-dist (execute (checked-staged-transcript-program A)}$
 $\text{adversary-initial-state)} \implies$
 $\text{wp-event verify-monad}$
 $(\text{composition-fri-sampled-query-bad-candidate}$
 $(\text{verifier-state-from-adversary attacker-state}$
 $(\text{staged-proof-transcript data})))$
 $(\text{verifier-state-from-adversary attacker-state}$
 $(\text{staged-proof-transcript data})) \leq$
 $\text{CompSampled data attacker-state}$

and *composition-merkle-bound*:
 $\wedge$ *data attacker-state*.
Some (*data*, *attacker-state*) $\in$
set-dist (*execute* (*checked-staged-transcript-program* *A*)
adversary-initial-state) $\implies$
wp-event verify-monad
(*partial-merkle-inconsistency-bad*
(*verifier-state-from-adversary attacker-state*
(*staged-proof-transcript data*)))
(*verifier-state-from-adversary attacker-state*
(*staged-proof-transcript data*)) $\leq$ *CompMerkle data attacker-state*
and *composition-total*:
 $\wedge$ *data attacker-state*.
Some (*data*, *attacker-state*) $\in$
set-dist (*execute* (*checked-staged-transcript-program* *A*)
adversary-initial-state) $\implies$
CompSampled data attacker-state +
(((*CompMerkle data attacker-state* + 0) +
(*CompMerkle data attacker-state* + 0) +
(*CompMerkle data attacker-state* + 0) +
(*CompMerkle data attacker-state* + 0 + 0) +
CompMerkle data attacker-state) + 0)
 $\leq$ *composition-fri-error*
and *empty-sampled-bound*:
 $\wedge$ *data attacker-state*.
Some (*data*, *attacker-state*) $\in$
set-dist (*execute* (*checked-staged-transcript-program* *A*)
adversary-initial-state) $\implies$
wp-event verify-monad
(*trace-fri-sampled-query-bad-candidate*
(*verifier-state-from-adversary attacker-state*
(*staged-proof-transcript data*)))
(*verifier-state-from-adversary attacker-state*
(*staged-proof-transcript data*)) $\leq$
EmptySampled data attacker-state
and *empty-conflict-bound*:
 $\wedge$ *data attacker-state*.
Some (*data*, *attacker-state*) $\in$
set-dist (*execute* (*checked-staged-transcript-program* *A*)
adversary-initial-state) $\implies$
wp-event verify-monad
(*trace-fri-sampled-assignment-conflict*
(*verifier-state-from-adversary attacker-state*
(*staged-proof-transcript data*)))
(*verifier-state-from-adversary attacker-state*
(*staged-proof-transcript data*)) $\leq$
EmptyConflict data attacker-state
and *empty-zero-bound*:
 $\wedge$ *data attacker-state*.

Some (data, attacker-state) ∈
set-dist (execute (checked-staged-transcript-program A)
adversary-initial-state) ⇒
wp-event verify-monad
(trace-fri-zero-round-final-obstruction
(verifier-state-from-adversary attacker-state
(staged-proof-transcript data)))
(verifier-state-from-adversary attacker-state
(staged-proof-transcript data)) ≤
EmptyZero data attacker-state
and *empty-total:*
 $\wedge$ *data attacker-state.*
Some (data, attacker-state) ∈
set-dist (execute (checked-staged-transcript-program A)
adversary-initial-state) ⇒
EmptySampled data attacker-state +
(EmptyConflict data attacker-state + EmptyZero data attacker-state)
 $\leq$ *trace-fri-error*
shows *checked-staged-adversary-acceptance-probability A ≤*
hash-collision-budget-value 0
(staged-attacker-query-budget budgets +
staged-challenge-query-budget + verifier-hash-query-budget) +
(composition-error-bound +
staged-phase-relation-error size
(staged-alpha-search-queries budgets 0) +
hash-collision-budget-value 0
(staged-alpha-search-queries budgets 0) +
staged-phase-relation-error size
(staged-alpha-search-queries budgets 0) +
(1 + staged-concrete-transcript-target-error-bound)) +
($\sum i < \text{rounds}.$
1 +
(1 + staged-concrete-transcript-target-error-bound) +
(1 + staged-concrete-transcript-target-error-bound)) +
trace-fri-error + composition-fri-error +
(composition-error-bound +
staged-phase-relation-error size
(staged-alpha-search-queries budgets 0) +
hash-collision-budget-value 0
(staged-alpha-search-queries budgets 0) +
staged-phase-relation-error size
(staged-alpha-search-queries budgets 0) +
(1 + staged-concrete-transcript-target-error-bound) +
trace-fri-error + 1)
proof –
have *active-fri: active-fri-reductions-for A*
by (rule
active-fri-reductions-for-from-trace-augmented-conflict-zero-composition-sampled-query-and-empty-conflict-zero
 $[OF$ *trace-augmented-bound trace-conflict-bound trace-zero-bound*

```

    trace-merkle-bound trace-total composition-sampled-bound composition-merkle-bound
    composition-total empty-sampled-bound empty-conflict-bound
    empty-zero-bound empty-total])
show ?thesis
by (rule
  stark-soundness-from-current-public-prefix-current-query-relevant-drift-route-from-active-fri-reductions
  [OF false-statement wf controlled active-fri])
qed

theorem stark-soundness-from-current-public-prefix-current-query-relevant-drift-route-from-sampled-query-projection
fixes TraceAug TraceConflict TraceZero TraceMerkle CompSampled CompMerkle EmptySam-
pled EmptyConflict EmptyZero ::
  'f staged-proof-data  $\Rightarrow$  'f protocol-channel  $\Rightarrow$  prob
assumes false-statement:  $\neg$  exists-valid-trace
and wf: staged-budget-wellformed budgets
and controlled: staged-adversary-controlled budgets A
and trace-augmented-bound:
 $\wedge$ data attacker-state.
  Some (data, attacker-state)  $\in$ 
    set-dist (execute (checked-staged-transcript-program A)
      adversary-initial-state)  $\implies$ 
wp-event verify-monad
  (trace-fri-bad-with-header-tied-augmented-partial-candidate
    (verifier-state-from-adversary attacker-state
      (staged-proof-transcript data))))
  (verifier-state-from-adversary attacker-state
    (staged-proof-transcript data))  $\leq$  TraceAug data attacker-state
and trace-conflict-bound:
 $\wedge$ data attacker-state.
  Some (data, attacker-state)  $\in$ 
    set-dist (execute (checked-staged-transcript-program A)
      adversary-initial-state)  $\implies$ 
wp-event verify-monad
  (trace-fri-header-tied-sampled-assignment-conflict
    (verifier-state-from-adversary attacker-state
      (staged-proof-transcript data))))
  (verifier-state-from-adversary attacker-state
    (staged-proof-transcript data))  $\leq$  TraceConflict data attacker-state
and trace-zero-bound:
 $\wedge$ data attacker-state.
  Some (data, attacker-state)  $\in$ 
    set-dist (execute (checked-staged-transcript-program A)
      adversary-initial-state)  $\implies$ 
wp-event verify-monad
  (trace-fri-header-tied-zero-round-final-obstruction
    (verifier-state-from-adversary attacker-state
      (staged-proof-transcript data))))
  (verifier-state-from-adversary attacker-state
    (staged-proof-transcript data))  $\leq$  TraceZero data attacker-state

```

and *trace-merkle-bound*:
 $\wedge$ *data attacker-state*.
Some (*data*, *attacker-state*) $\in$
set-dist (*execute* (*checked-staged-transcript-program* *A*)
adversary-initial-state) $\implies$
wp-event verify-monad
(*partial-merkle-inconsistency-bad*
(*verifier-state-from-adversary attacker-state*
(*staged-proof-transcript data*)))
(*verifier-state-from-adversary attacker-state*
(*staged-proof-transcript data*)) $\leq$ *TraceMerkle data attacker-state*
and *trace-total*:
 $\wedge$ *data attacker-state*.
Some (*data*, *attacker-state*) $\in$
set-dist (*execute* (*checked-staged-transcript-program* *A*)
adversary-initial-state) $\implies$
TraceAug data attacker-state +
((*TraceConflict data attacker-state* + *TraceZero data attacker-state*) +
TraceMerkle data attacker-state)
 $\leq$ *trace-fri-error*
and *composition-sampled-future*:
 $\wedge$ *data attacker-state*.
Some (*data*, *attacker-state*) $\in$
set-dist (*execute* (*checked-staged-transcript-program* *A*)
adversary-initial-state) $\implies$
composition-fri-future-fresh
(*verifier-state-from-adversary attacker-state*
(*staged-proof-transcript data*))
and *composition-sampled-generic*:
 $\wedge$ *data attacker-state dg*.
Some (*data*, *attacker-state*) $\in$
set-dist (*execute* (*checked-staged-transcript-program* *A*)
adversary-initial-state) $\implies$
generic-fri-sampled-query-projection-fraction-bound
(*composition-table-low-degree* (*to-nat dg*))
(*Not* $\circ$ *composition-table-low-degree maxDegree*) (*to-nat dg*)
(*CompSampled data attacker-state*)
and *composition-merkle-bound*:
 $\wedge$ *data attacker-state*.
Some (*data*, *attacker-state*) $\in$
set-dist (*execute* (*checked-staged-transcript-program* *A*)
adversary-initial-state) $\implies$
wp-event verify-monad
(*partial-merkle-inconsistency-bad*
(*verifier-state-from-adversary attacker-state*
(*staged-proof-transcript data*)))
(*verifier-state-from-adversary attacker-state*
(*staged-proof-transcript data*)) $\leq$ *CompMerkle data attacker-state*
and *composition-total*:

$\wedge$ data attacker-state.
 Some (data, attacker-state) $\in$
 set-dist (execute (checked-staged-transcript-program A)
 adversary-initial-state) $\implies$
 CompSampled data attacker-state +
 (((CompMerkle data attacker-state + 0) +
 (CompMerkle data attacker-state + 0) +
 (CompMerkle data attacker-state + 0) +
 (CompMerkle data attacker-state + 0 + 0) +
 CompMerkle data attacker-state) + 0)
 $\leq$ composition-fri-error
and empty-sampled-future:
 $\wedge$ data attacker-state.
 Some (data, attacker-state) $\in$
 set-dist (execute (checked-staged-transcript-program A)
 adversary-initial-state) $\implies$
 trace-fri-future-fresh
 (verifier-state-from-adversary attacker-state
 (staged-proof-transcript data))
and empty-sampled-generic:
 $\wedge$ data attacker-state.
 Some (data, attacker-state) $\in$
 set-dist (execute (checked-staged-transcript-program A)
 adversary-initial-state) $\implies$
 generic-fri-sampled-query-projection-fraction-bound
 trace-table-low-degree (Not $\circ$ trace-table-low-degree)
 (clength - 1) (EmptySampled data attacker-state)
and empty-conflict-bound:
 $\wedge$ data attacker-state.
 Some (data, attacker-state) $\in$
 set-dist (execute (checked-staged-transcript-program A)
 adversary-initial-state) $\implies$
 wp-event verify-monad
 (trace-fri-sampled-assignment-conflict
 (verifier-state-from-adversary attacker-state
 (staged-proof-transcript data)))
 (verifier-state-from-adversary attacker-state
 (staged-proof-transcript data)) $\leq$
 EmptyConflict data attacker-state
and empty-zero-bound:
 $\wedge$ data attacker-state.
 Some (data, attacker-state) $\in$
 set-dist (execute (checked-staged-transcript-program A)
 adversary-initial-state) $\implies$
 wp-event verify-monad
 (trace-fri-zero-round-final-obstruction
 (verifier-state-from-adversary attacker-state
 (staged-proof-transcript data)))
 (verifier-state-from-adversary attacker-state

$(\text{staged-proof-transcript data})) \leq$
 $\text{EmptyZero data attacker-state}$
and empty-total :
 $\wedge \text{data attacker-state.}$
 $\text{Some (data, attacker-state)} \in$
 $\text{set-dist (execute (checked-staged-transcript-program A)}$
 $\text{adversary-initial-state)} \implies$
 $\text{EmptySampled data attacker-state} +$
 $(\text{EmptyConflict data attacker-state} + \text{EmptyZero data attacker-state})$
 $\leq \text{trace-fri-error}$
shows $\text{checked-staged-adversary-acceptance-probability A} \leq$
 $\text{hash-collision-budget-value 0}$
 $(\text{staged-attacker-query-budget budgets} +$
 $\text{staged-challenge-query-budget} + \text{verifier-hash-query-budget}) +$
 $(\text{composition-error-bound} +$
 $\text{staged-phase-relation-error size}$
 $(\text{staged-alpha-search-queries budgets 0}) +$
 $\text{hash-collision-budget-value 0}$
 $(\text{staged-alpha-search-queries budgets 0}) +$
 $\text{staged-phase-relation-error size}$
 $(\text{staged-alpha-search-queries budgets 0}) +$
 $(1 + \text{staged-concrete-transcript-target-error-bound})) +$
 $(\sum_{i < \text{rounds.}}$
 $1 +$
 $(1 + \text{staged-concrete-transcript-target-error-bound}) +$
 $(1 + \text{staged-concrete-transcript-target-error-bound})) +$
 $\text{trace-fri-error} + \text{composition-fri-error} +$
 $(\text{composition-error-bound} +$
 $\text{staged-phase-relation-error size}$
 $(\text{staged-alpha-search-queries budgets 0}) +$
 $\text{hash-collision-budget-value 0}$
 $(\text{staged-alpha-search-queries budgets 0}) +$
 $\text{staged-phase-relation-error size}$
 $(\text{staged-alpha-search-queries budgets 0}) +$
 $(1 + \text{staged-concrete-transcript-target-error-bound}) +$
 $\text{trace-fri-error} + 1)$
proof (rule
 $\text{stark-soundness-from-current-public-prefix-current-query-relevant-drift-route-from-sampled-query-fri-components}$
 $[\text{OF false-statement wf controlled trace-augmented-bound}$
 $\text{trace-conflict-bound trace-zero-bound trace-merkle-bound trace-total -}$
 $\text{composition-merkle-bound composition-total - empty-conflict-bound}$
 $\text{empty-zero-bound empty-total}]$)
fix $\text{data attacker-state}$
assume support :
 $\text{Some (data, attacker-state)} \in$
 $\text{set-dist (execute (checked-staged-transcript-program A)}$
 $\text{adversary-initial-state})$
let $?s =$
 $\text{verifier-state-from-adversary attacker-state}$

```

    (staged-proof-transcript data)
  show
    wp-event verify-monad (composition-fri-sampled-query-bad-candidate ?s)
      ?s ≤ CompSampled data attacker-state
  by (rule
    wp-composition-fri-sampled-query-bad-candidate-bound-from-generic-projection-fraction
      [OF composition-sampled-future[OF support]])
    (rule composition-sampled-generic[OF support])
next
  fix data attacker-state
  assume support:
    Some (data, attacker-state) ∈
      set-dist (execute (checked-staged-transcript-program A)
        adversary-initial-state)
  let ?s =
    verifier-state-from-adversary attacker-state
      (staged-proof-transcript data)
  show
    wp-event verify-monad (trace-fri-sampled-query-bad-candidate ?s)
      ?s ≤ EmptySampled data attacker-state
  by (rule
    wp-trace-fri-sampled-query-bad-candidate-bound-from-generic-projection-fraction
      [OF empty-sampled-future[OF support]])
    (rule empty-sampled-generic[OF support])
qed

```

theorem stark-soundness-from-current-public-prefix-current-query-relevant-drift-route-from-trace-active-and-staged-s

```

  fixes composition-B :: 'f ⇒ 'f list set
  and composition-Q empty-Q :: nat list set
  and CompSampled CompMerkle EmptySampled EmptyConflict EmptyZero
    composition-fresh empty-fresh :: prob
  assumes false-statement: ¬ exists-valid-trace
  and wf: staged-budget-wellformed budgets
  and controlled: staged-adversary-controlled budgets A
  and trace-fri-bound:
    wp-event (checked-staged-security-experiment-with-data-state A)
      (staged-security-with-data-state-verifier-event
        trace-fri-bad-with-header-tied-partial-candidate)
        adversary-initial-state ≤ trace-fri-error
  and composition-finite:  $\bigwedge dg. \text{finite } (composition-B \ dg)$ 
  and composition-fresh-bound:
    wp-event (checked-staged-security-experiment-with-data-state A)
      (staged-security-with-data-state-verifier-event
        ( $\lambda s. \text{composition-fri-challenge-list-fresh-hit } s \text{ composition-B}$ ))
        adversary-initial-state ≤ composition-fresh
  and composition-projection:
     $\bigwedge dg. \text{fst } \langle$ 
      ((composition-fri-sampled-query-bad-pair-union dg  $\cap$ 
        (fri-query-index-list-space  $\times$  UNIV))  $\cap$ 

```

$(UNIV \times (- \text{composition-}B \text{ dg}))) \subseteq \text{composition-}Q$
and *composition-sampled-total*:
composition-fresh +
hash-relation-budget-value
 $(\sum dg \in (UNIV :: 'f \text{ set}).$
 $\text{card } (\text{composition-}B \text{ dg}) * \text{ceil-log } (\text{maxDegree} + 1))$
 $(\text{staged-attacker-query-budget budgets} +$
 $\text{staged-challenge-query-budget}) +$
staged-phase-target-error
 $(\text{query-index-raw-preimage}$
 $(\text{query-index-list-entries } \text{composition-}Q))$
 $(\text{staged-attacker-query-budget budgets} +$
 $\text{staged-challenge-query-budget})$
 $\leq \text{CompSampled}$
and *composition-merkle-bound*:
 $\wedge \text{data attacker-state.}$
 $\text{Some } (\text{data}, \text{attacker-state}) \in$
 $\text{set-dist } (\text{execute } (\text{checked-staged-transcript-program } A)$
 $\text{adversary-initial-state}) \implies$
 $\text{wp-event verify-monad}$
 $(\text{partial-merkle-inconsistency-bad}$
 $(\text{verifier-state-from-adversary attacker-state}$
 $(\text{staged-proof-transcript data})))$
 $(\text{verifier-state-from-adversary attacker-state}$
 $(\text{staged-proof-transcript data})) \leq \text{CompMerkle}$
and *composition-total*:
 $\text{CompSampled} +$
 $((\text{CompMerkle} + 0) + (\text{CompMerkle} + 0) + (\text{CompMerkle} + 0) +$
 $(\text{CompMerkle} + 0 + 0) + \text{CompMerkle}) + 0 + 0)$
 $\leq \text{composition-fri-error}$
and *empty-finite*: *finite empty-B*
and *empty-fresh-bound*:
 $\text{wp-event } (\text{checked-staged-security-experiment-with-data-state } A)$
 $(\text{staged-security-with-data-state-verifier-event}$
 $(\lambda s. \text{trace-fri-challenge-list-fresh-hit } s \text{ empty-B}))$
 $\text{adversary-initial-state} \leq \text{empty-fresh}$
and *empty-projection*:
 fst '
 $((\text{trace-fri-sampled-query-bad-pair-union} \cap$
 $(\text{fri-query-index-list-space} \times UNIV)) \cap$
 $(UNIV \times (- \text{empty-B}))) \subseteq \text{empty-Q}$
and *empty-sampled-total*:
empty-fresh +
hash-relation-budget-value $(\text{card } \text{empty-B} * \text{ceil-log } \text{clength})$
 $(\text{staged-attacker-query-budget budgets} +$
 $\text{staged-challenge-query-budget}) +$
staged-phase-target-error
 $(\text{query-index-raw-preimage } (\text{query-index-list-entries } \text{empty-Q}))$
 $(\text{staged-attacker-query-budget budgets} +$

$\leq \text{EmptySampled}$
and *empty-conflict-bound*:
 $\wedge \text{data attacker-state.}$
 $\text{Some (data, attacker-state)} \in$
 $\text{set-dist (execute (checked-staged-transcript-program } A)$
 $\text{adversary-initial-state)} \implies$
 $\text{wp-event verify-monad}$
 $(\text{trace-fri-sampled-assignment-conflict}$
 $(\text{verifier-state-from-adversary attacker-state}$
 $(\text{staged-proof-transcript data})))$
 $(\text{verifier-state-from-adversary attacker-state}$
 $(\text{staged-proof-transcript data})) \leq \text{EmptyConflict}$
and *empty-zero-bound*:
 $\wedge \text{data attacker-state.}$
 $\text{Some (data, attacker-state)} \in$
 $\text{set-dist (execute (checked-staged-transcript-program } A)$
 $\text{adversary-initial-state)} \implies$
 $\text{wp-event verify-monad}$
 $(\text{trace-fri-zero-round-final-obstruction}$
 $(\text{verifier-state-from-adversary attacker-state}$
 $(\text{staged-proof-transcript data})))$
 $(\text{verifier-state-from-adversary attacker-state}$
 $(\text{staged-proof-transcript data})) \leq \text{EmptyZero}$
and *empty-total*:
 $\text{EmptySampled} + (\text{EmptyConflict} + \text{EmptyZero}) \leq \text{trace-fri-error}$
shows *checked-staged-adversary-acceptance-probability* $A \leq$
 $\text{hash-collision-budget-value } 0$
 $(\text{staged-attacker-query-budget budgets} +$
 $\text{staged-challenge-query-budget} + \text{verifier-hash-query-budget}) +$
 $(\text{composition-error-bound} +$
 $\text{staged-phase-relation-error size}$
 $(\text{staged-alpha-search-queries budgets } 0) +$
 $\text{hash-collision-budget-value } 0$
 $(\text{staged-alpha-search-queries budgets } 0) +$
 $\text{staged-phase-relation-error size}$
 $(\text{staged-alpha-search-queries budgets } 0) +$
 $(1 + \text{staged-concrete-transcript-target-error-bound})) +$
 $(\sum_{i < \text{rounds.}}$
 $1 +$
 $(1 + \text{staged-concrete-transcript-target-error-bound}) +$
 $(1 + \text{staged-concrete-transcript-target-error-bound})) +$
 $\text{trace-fri-error} + \text{composition-fri-error} +$
 $(\text{composition-error-bound} +$
 $\text{staged-phase-relation-error size}$
 $(\text{staged-alpha-search-queries budgets } 0) +$
 $\text{hash-collision-budget-value } 0$
 $(\text{staged-alpha-search-queries budgets } 0) +$
 $\text{staged-phase-relation-error size}$

(staged-alpha-search-queries budgets 0) +
 (1 + staged-concrete-transcript-target-error-bound) +
 trace-fri-error + 1)

proof –

have composition-sampled:

wp-event (checked-staged-security-experiment-with-data-state A)
 (staged-security-with-data-state-verifier-event
 composition-fri-sampled-query-bad-candidate)
 adversary-initial-state ≤ CompSampled

by (rule order-trans
 [OF checked-staged-security-composition-fri-sampled-query-bad-candidate-bound-from-challenge-cover-and-query-
 [OF wf controlled composition-finite composition-fresh-bound
 composition-projection]
 composition-sampled-total])

have comp-fri-bound:

wp-event (checked-staged-security-experiment-with-data-state A)
 (staged-security-with-data-state-verifier-event
 composition-fri-bad-with-verifier-tied-partial-candidate)
 adversary-initial-state ≤ composition-fri-error

proof –

have wp-event (checked-staged-security-experiment-with-data-state A)
 (staged-security-with-data-state-verifier-event
 composition-fri-bad-with-verifier-tied-partial-candidate)
 adversary-initial-state ≤
 CompSampled +
 (((CompMerkle + 0) + (CompMerkle + 0) + (CompMerkle + 0) +
 (CompMerkle + 0 + 0) + CompMerkle) + 0 + 0)

apply (rule
 checked-staged-security-composition-fri-verifier-tied-partial-candidate-bound-from-sampled-query-and-residual-g
 [OF composition-sampled])

apply (rule wp-composition-fri-verifier-tied-authenticated-slot-recorded-chunk-gap-zero)

apply (rule composition-merkle-bound)

apply assumption

apply (rule wp-composition-fri-verifier-tied-recorded-base-chunk-auth-missing-zero)

apply (rule wp-composition-fri-verifier-tied-next-value-replay-gap-bound-from-merkle-and-slot)

apply (rule composition-merkle-bound)

apply assumption

apply (rule wp-composition-fri-verifier-tied-authenticated-slot-recorded-chunk-gap-zero)

apply (rule wp-composition-fri-verifier-tied-successor-replay-gap-bound-from-merkle-and-slot)

apply (rule composition-merkle-bound)

apply assumption

apply (rule wp-composition-fri-verifier-tied-authenticated-slot-recorded-chunk-gap-zero)

apply (rule wp-composition-fri-verifier-tied-final-value-replay-gap-bound-from-merkle-slot-and-selected-step)

apply (rule composition-merkle-bound)

apply assumption

apply (rule wp-composition-fri-verifier-tied-authenticated-slot-recorded-chunk-gap-zero)

apply (rule wp-composition-fri-verifier-tied-final-value-selected-step-missing-gap-zero)

apply (subst wp-composition-fri-verifier-tied-zero-round-final-obstruction-zero)

apply simp

```

    done
  then show ?thesis
    by (rule order-trans[OF - composition-total])
qed
have empty-sampled:
  wp-event (checked-staged-security-experiment-with-data-state A)
    (staged-security-with-data-state-verifier-event
      trace-fri-sampled-query-bad-candidate)
  adversary-initial-state  $\leq$  EmptySampled
by (rule order-trans
  [OF checked-staged-security-trace-fri-sampled-query-bad-candidate-bound-from-challenge-cover-and-query-project
    [OF wf controlled empty-finite empty-fresh-bound empty-projection]
    empty-sampled-total])
have empty-trace-fri-bound:
  wp-event (checked-staged-security-experiment-with-data-state A)
    (staged-security-with-data-state-verifier-event
      trace-fri-bad-with-empty-composition-header-candidates)
  adversary-initial-state  $\leq$  trace-fri-error
proof -
  have wp-event (checked-staged-security-experiment-with-data-state A)
    (staged-security-with-data-state-verifier-event
      trace-fri-bad-with-empty-composition-header-candidates)
    adversary-initial-state  $\leq$  EmptySampled + (EmptyConflict + EmptyZero)
  by (rule
    checked-staged-security-trace-fri-empty-header-bound-from-sampled-query-conflict-and-zero
    [OF empty-sampled empty-conflict-bound empty-zero-bound])
  then show ?thesis
    by (rule order-trans[OF - empty-total])
qed
show ?thesis
  by (rule
    current-query-route-bound-from-active-fri-bounds-internal
    [OF false-statement wf controlled trace-fri-bound comp-fri-bound
      empty-trace-fri-bound])
qed

theorem stark-soundness-from-current-public-prefix-current-query-relevant-drift-route-from-full-cover-domain-conflict
fixes trace-bad ::
  'f list  $\Rightarrow$  nat  $\Rightarrow$  'f list  $\Rightarrow$  'f set
and composition-bad ::
  'f  $\Rightarrow$  'f list  $\Rightarrow$  nat  $\Rightarrow$  'f list  $\Rightarrow$  'f set
and trace-B :: 'f list set
and composition-B :: 'f  $\Rightarrow$  'f list set
and trace-fresh trace-proximity trace-index trace-domain-length
  trace-sampled-domain trace-conflict trace-zero
  composition-fresh composition-proximity composition-index
  composition-domain-length composition-sampled-domain
  composition-conflict composition-zero
  empty-conflict empty-zero :: prob

```

assumes *false-statement*: $\neg \text{exists-valid-trace}$
and *wf*: *staged-budget-wellformed budgets*
and *controlled*: *staged-adversary-controlled budgets A*
and *trace-finite*: *finite trace-B*
and *trace-envelope*:
 $\wedge \text{data attacker-state trace-table.}$
 $\text{Some (data, attacker-state)} \in$
 $\text{set-dist (execute (checked-staged-transcript-program A)}$
 $\text{adversary-initial-state)} \implies$
 $\text{trace-fri-multiround-bad-sets trace-bad trace-table} \subseteq \text{trace-B}$
and *trace-fresh-bound*:
 $\text{wp-event (checked-staged-security-experiment-with-data-state A)}$
 $(\text{staged-security-with-data-state-verifier-event}$
 $(\lambda s. \text{trace-fri-challenge-list-fresh-hit s trace-B}))$
 $\text{adversary-initial-state} \leq \text{trace-fresh}$
and *trace-proximity-bound*:
 $\wedge \text{data attacker-state.}$
 $\text{Some (data, attacker-state)} \in$
 $\text{set-dist (execute (checked-staged-transcript-program A)}$
 $\text{adversary-initial-state)} \implies$
 $\text{wp-event verify-monad}$
 $(\text{trace-fri-sampled-full-cover-proximity-failure}$
 $(\text{verifier-state-from-adversary attacker-state}$
 $(\text{staged-proof-transcript data})) \text{ trace-bad})$
 $(\text{verifier-state-from-adversary attacker-state}$
 $(\text{staged-proof-transcript data})) \leq \text{trace-proximity}$
and *trace-index-bound*:
 $\wedge \text{data attacker-state.}$
 $\text{Some (data, attacker-state)} \in$
 $\text{set-dist (execute (checked-staged-transcript-program A)}$
 $\text{adversary-initial-state)} \implies$
 $\text{wp-event verify-monad}$
 $(\text{trace-fri-sampled-full-cover-index-failure}$
 $(\text{verifier-state-from-adversary attacker-state}$
 $(\text{staged-proof-transcript data})))$
 $(\text{verifier-state-from-adversary attacker-state}$
 $(\text{staged-proof-transcript data})) \leq \text{trace-index}$
and *trace-domain-length-bound*:
 $\wedge \text{data attacker-state.}$
 $\text{Some (data, attacker-state)} \in$
 $\text{set-dist (execute (checked-staged-transcript-program A)}$
 $\text{adversary-initial-state)} \implies$
 $\text{wp-event verify-monad}$
 $(\text{trace-fri-sampled-full-cover-domain-length-failure}$
 $(\text{verifier-state-from-adversary attacker-state}$
 $(\text{staged-proof-transcript data})))$
 $(\text{verifier-state-from-adversary attacker-state}$
 $(\text{staged-proof-transcript data})) \leq \text{trace-domain-length}$
and *trace-sampled-domain-bound*:

$\wedge$ *data attacker-state*.
Some (*data*, *attacker-state*) $\in$
set-dist (*execute* (*checked-staged-transcript-program* *A*)
adversary-initial-state) $\implies$
wp-event verify-monad
(*trace-fri-sampled-full-cover-sampled-domain-failure*
(*verifier-state-from-adversary attacker-state*
(*staged-proof-transcript data*)))
(*verifier-state-from-adversary attacker-state*
(*staged-proof-transcript data*)) $\leq$ *trace-sampled-domain*
and *trace-conflict-bound*:
 $\wedge$ *data attacker-state*.
Some (*data*, *attacker-state*) $\in$
set-dist (*execute* (*checked-staged-transcript-program* *A*)
adversary-initial-state) $\implies$
wp-event verify-monad
(*trace-fri-header-tied-sampled-assignment-conflict*
(*verifier-state-from-adversary attacker-state*
(*staged-proof-transcript data*)))
(*verifier-state-from-adversary attacker-state*
(*staged-proof-transcript data*)) $\leq$ *trace-conflict*
and *trace-zero-bound*:
 $\wedge$ *data attacker-state*.
Some (*data*, *attacker-state*) $\in$
set-dist (*execute* (*checked-staged-transcript-program* *A*)
adversary-initial-state) $\implies$
wp-event verify-monad
(*trace-fri-header-tied-zero-round-final-obstruction*
(*verifier-state-from-adversary attacker-state*
(*staged-proof-transcript data*)))
(*verifier-state-from-adversary attacker-state*
(*staged-proof-transcript data*)) $\leq$ *trace-zero*
and *trace-total*:
((*trace-fresh* +
hash-relation-budget-value (*card trace-B* * *ceil-log clength*)
(*staged-attacker-query-budget budgets* +
staged-challenge-query-budget)) +
(*trace-proximity* + *trace-index* +
(*trace-index* + *trace-domain-length*) +
(*trace-index* + *trace-sampled-domain*))) +
(*trace-conflict* + *trace-zero*)
 $\leq$ *trace-fri-error*
and *composition-finite*: $\wedge$ *dg. finite* (*composition-B dg*)
and *composition-envelope*:
 $\wedge$ *data attacker-state dg composition-table*.
Some (*data*, *attacker-state*) $\in$
set-dist (*execute* (*checked-staged-transcript-program* *A*)
adversary-initial-state) $\implies$
composition-fri-multiround-bad-sets composition-bad dg composition-table

$\subseteq \text{composition-}B \text{ dg}$
and *composition-fresh-bound*:
wp-event (*checked-staged-security-experiment-with-data-state A*)
 (*staged-security-with-data-state-verifier-event*
 ($\lambda s. \text{composition-fri-challenge-list-fresh-hit } s \text{ composition-}B$))
adversary-initial-state $\leq$ *composition-fresh*
and *composition-proximity-bound*:
 $\wedge$ *data attacker-state*.
 Some (*data*, *attacker-state*) $\in$
 set-dist (*execute* (*checked-staged-transcript-program A*)
adversary-initial-state) $\implies$
wp-event verify-monad
 (*composition-fri-sampled-full-cover-proximity-failure*
 (*verifier-state-from-adversary attacker-state*
 (*staged-proof-transcript data*)) *composition-bad*)
 (*verifier-state-from-adversary attacker-state*
 (*staged-proof-transcript data*)) $\leq$ *composition-proximity*
and *composition-index-bound*:
 $\wedge$ *data attacker-state*.
 Some (*data*, *attacker-state*) $\in$
 set-dist (*execute* (*checked-staged-transcript-program A*)
adversary-initial-state) $\implies$
wp-event verify-monad
 (*composition-fri-sampled-full-cover-index-failure*
 (*verifier-state-from-adversary attacker-state*
 (*staged-proof-transcript data*)))
 (*verifier-state-from-adversary attacker-state*
 (*staged-proof-transcript data*)) $\leq$ *composition-index*
and *composition-domain-length-bound*:
 $\wedge$ *data attacker-state*.
 Some (*data*, *attacker-state*) $\in$
 set-dist (*execute* (*checked-staged-transcript-program A*)
adversary-initial-state) $\implies$
wp-event verify-monad
 (*composition-fri-sampled-full-cover-domain-length-failure*
 (*verifier-state-from-adversary attacker-state*
 (*staged-proof-transcript data*)))
 (*verifier-state-from-adversary attacker-state*
 (*staged-proof-transcript data*)) $\leq$ *composition-domain-length*
and *composition-sampled-domain-bound*:
 $\wedge$ *data attacker-state*.
 Some (*data*, *attacker-state*) $\in$
 set-dist (*execute* (*checked-staged-transcript-program A*)
adversary-initial-state) $\implies$
wp-event verify-monad
 (*composition-fri-sampled-full-cover-sampled-domain-failure*
 (*verifier-state-from-adversary attacker-state*
 (*staged-proof-transcript data*)))
 (*verifier-state-from-adversary attacker-state*

$(\text{staged-proof-transcript data})) \leq \text{composition-sampled-domain}$
and *composition-conflict-bound*:
 $\wedge \text{data attacker-state.}$
 $\text{Some (data, attacker-state)} \in$
 $\text{set-dist (execute (checked-staged-transcript-program A)}$
 $\text{adversary-initial-state)} \implies$
 $\text{wp-event verify-monad}$
 $(\text{composition-fri-verifier-tied-sampled-assignment-conflict}$
 $(\text{verifier-state-from-adversary attacker-state}$
 $(\text{staged-proof-transcript data})))$
 $(\text{verifier-state-from-adversary attacker-state}$
 $(\text{staged-proof-transcript data})) \leq \text{composition-conflict}$
and *composition-zero-bound*:
 $\wedge \text{data attacker-state.}$
 $\text{Some (data, attacker-state)} \in$
 $\text{set-dist (execute (checked-staged-transcript-program A)}$
 $\text{adversary-initial-state)} \implies$
 $\text{wp-event verify-monad}$
 $(\text{composition-fri-verifier-tied-zero-round-final-obstruction}$
 $(\text{verifier-state-from-adversary attacker-state}$
 $(\text{staged-proof-transcript data})))$
 $(\text{verifier-state-from-adversary attacker-state}$
 $(\text{staged-proof-transcript data})) \leq \text{composition-zero}$
and *composition-total*:
 $((\text{composition-fresh} +$
 $\text{hash-relation-budget-value}$
 $(\sum dg \in (\text{UNIV} :: 'f \text{ set}).$
 $\text{card (composition-B dg) * ceil-log (maxDegree + 1))}$
 $(\text{staged-attacker-query-budget budgets} +$
 $\text{staged-challenge-query-budget})) +$
 $(\text{composition-proximity} + \text{composition-index} +$
 $(\text{composition-index} + \text{composition-domain-length}) +$
 $(\text{composition-index} + \text{composition-sampled-domain}))) +$
 $(\text{composition-conflict} + \text{composition-zero})$
 $\leq \text{composition-fri-error}$
and *empty-conflict-bound*:
 $\wedge \text{data attacker-state.}$
 $\text{Some (data, attacker-state)} \in$
 $\text{set-dist (execute (checked-staged-transcript-program A)}$
 $\text{adversary-initial-state)} \implies$
 $\text{wp-event verify-monad}$
 $(\text{trace-fri-sampled-assignment-conflict}$
 $(\text{verifier-state-from-adversary attacker-state}$
 $(\text{staged-proof-transcript data})))$
 $(\text{verifier-state-from-adversary attacker-state}$
 $(\text{staged-proof-transcript data})) \leq \text{empty-conflict}$
and *empty-zero-bound*:
 $\wedge \text{data attacker-state.}$
 $\text{Some (data, attacker-state)} \in$

$\text{set-dist } (\text{execute } (\text{checked-staged-transcript-program } A) \text{ adversary-initial-state}) \implies$
 $\text{wp-event verify-monad}$
 $(\text{trace-fri-zero-round-final-obstruction}$
 $(\text{verifier-state-from-adversary attacker-state}$
 $(\text{staged-proof-transcript data})))$
 $(\text{verifier-state-from-adversary attacker-state}$
 $(\text{staged-proof-transcript data})) \leq \text{empty-zero}$
and empty-total:
 $((\text{trace-fresh} +$
 $\text{hash-relation-budget-value } (\text{card trace-B} * \text{ceil-log clength})$
 $(\text{staged-attacker-query-budget budgets} +$
 $\text{staged-challenge-query-budget})) +$
 $(\text{trace-proximity} + \text{trace-index} +$
 $(\text{trace-index} + \text{trace-domain-length}) +$
 $(\text{trace-index} + \text{trace-sampled-domain}))) +$
 $(\text{empty-conflict} + \text{empty-zero})$
 $\leq \text{trace-fri-error}$
shows $\text{checked-staged-adversary-acceptance-probability } A \leq$
 $\text{hash-collision-budget-value } 0$
 $(\text{staged-attacker-query-budget budgets} +$
 $\text{staged-challenge-query-budget} + \text{verifier-hash-query-budget}) +$
 $(\text{composition-error-bound} +$
 $\text{staged-phase-relation-error size}$
 $(\text{staged-alpha-search-queries budgets } 0)) +$
 $\text{hash-collision-budget-value } 0$
 $(\text{staged-alpha-search-queries budgets } 0) +$
 $\text{staged-phase-relation-error size}$
 $(\text{staged-alpha-search-queries budgets } 0) +$
 $(1 + \text{staged-concrete-transcript-target-error-bound})) +$
 $(\sum_{i < \text{rounds.}} 1 +$
 $(1 + \text{staged-concrete-transcript-target-error-bound}) +$
 $(1 + \text{staged-concrete-transcript-target-error-bound})) +$
 $\text{trace-fri-error} + \text{composition-fri-error} +$
 $(\text{composition-error-bound} +$
 $\text{staged-phase-relation-error size}$
 $(\text{staged-alpha-search-queries budgets } 0)) +$
 $\text{hash-collision-budget-value } 0$
 $(\text{staged-alpha-search-queries budgets } 0) +$
 $\text{staged-phase-relation-error size}$
 $(\text{staged-alpha-search-queries budgets } 0) +$
 $(1 + \text{staged-concrete-transcript-target-error-bound}) +$
 $\text{trace-fri-error} + 1)$
proof –
have trace-fri-bound:
 $\text{wp-event } (\text{checked-staged-security-experiment-with-data-state } A)$
 $(\text{staged-security-with-data-state-verifier-event}$
 $\text{trace-fri-bad-with-header-tied-partial-candidate})$

$adversary-initial-state \leq trace-fri-error$
proof –
have $wp-event$ ($checked-staged-security-experiment-with-data-state\ A$)
 $(staged-security-with-data-state-verifier-event$
 $trace-fri-bad-with-header-tied-partial-candidate)$
 $adversary-initial-state \leq$
 $((trace-fresh +$
 $hash-relation-budget-value\ (card\ trace-B * ceil-log\ clength)$
 $(staged-attacker-query-budget\ budgets +$
 $staged-challenge-query-budget)) +$
 $(trace-proximity + trace-index +$
 $(trace-index + trace-domain-length) +$
 $(trace-index + trace-sampled-domain))) +$
 $(trace-conflict + trace-zero)$
by ($rule$
 $checked-staged-security-trace-fri-header-tied-partial-candidate-bound-from-full-cover-domain-conflict-and-zero$
 $[where\ bad = trace-bad];$
 $(rule\ wf \mid rule\ controlled \mid rule\ trace-finite \mid$
 $rule\ trace-envelope \mid rule\ trace-fresh-bound \mid$
 $rule\ trace-proximity-bound \mid rule\ trace-index-bound \mid$
 $rule\ trace-domain-length-bound \mid rule\ trace-sampled-domain-bound \mid$
 $rule\ trace-conflict-bound \mid rule\ trace-zero-bound))$
then show $?thesis$
by ($rule\ order-trans[OF - trace-total]$)
qed
have $comp-fri-bound$:
 $wp-event$ ($checked-staged-security-experiment-with-data-state\ A$)
 $(staged-security-with-data-state-verifier-event$
 $composition-fri-bad-with-verifier-tied-partial-candidate)$
 $adversary-initial-state \leq composition-fri-error$
proof –
have $wp-event$ ($checked-staged-security-experiment-with-data-state\ A$)
 $(staged-security-with-data-state-verifier-event$
 $composition-fri-bad-with-verifier-tied-partial-candidate)$
 $adversary-initial-state \leq$
 $((composition-fresh +$
 $hash-relation-budget-value$
 $(\sum dg \in (UNIV :: 'f\ set).$
 $card\ (composition-B\ dg) * ceil-log\ (maxDegree + 1))$
 $(staged-attacker-query-budget\ budgets +$
 $staged-challenge-query-budget)) +$
 $(composition-proximity + composition-index +$
 $(composition-index + composition-domain-length) +$
 $(composition-index + composition-sampled-domain))) +$
 $(composition-conflict + composition-zero)$
by ($rule$
 $checked-staged-security-composition-fri-verifier-tied-partial-candidate-bound-from-full-cover-domain-conflict-and-zero$
 $[where\ bad = composition-bad];$
 $(rule\ wf \mid rule\ controlled \mid rule\ composition-finite \mid$

```

    rule composition-envelope | rule composition-fresh-bound |
    rule composition-proximity-bound | rule composition-index-bound |
    rule composition-domain-length-bound |
    rule composition-sampled-domain-bound |
    rule composition-conflict-bound | rule composition-zero-bound))
  then show ?thesis
    by (rule order-trans[OF - composition-total])
qed
have empty-trace-fri-bound:
  wp-event (checked-staged-security-experiment-with-data-state A)
  (staged-security-with-data-state-verifier-event
   trace-fri-bad-with-empty-composition-header-candidates)
  adversary-initial-state ≤ trace-fri-error
proof -
  have wp-event (checked-staged-security-experiment-with-data-state A)
    (staged-security-with-data-state-verifier-event
     trace-fri-bad-with-empty-composition-header-candidates)
    adversary-initial-state ≤
      ((trace-fresh +
        hash-relation-budget-value (card trace-B * ceil-log clength)
        (staged-attacker-query-budget budgets +
         staged-challenge-query-budget)) +
        (trace-proximity + trace-index +
         (trace-index + trace-domain-length) +
         (trace-index + trace-sampled-domain))) +
        (empty-conflict + empty-zero)
  by (rule
      checked-staged-security-trace-fri-empty-header-bound-from-full-cover-domain-conflict-and-zero
      [where bad = trace-bad];
      (rule wf | rule controlled | rule trace-finite |
       rule trace-envelope | rule trace-fresh-bound |
       rule trace-proximity-bound | rule trace-index-bound |
       rule trace-domain-length-bound | rule trace-sampled-domain-bound |
       rule empty-conflict-bound | rule empty-zero-bound))
  then show ?thesis
    by (rule order-trans[OF - empty-total])
qed
show ?thesis
  by (rule
      current-query-route-bound-from-active-fri-bounds-internal
      [OF false-statement wf controlled trace-fri-bound comp-fri-bound
       empty-trace-fri-bound])
qed

theorem stark-soundness-from-current-public-prefix-current-query-relevant-drift-route-from-full-cover-domain-conflict
fixes trace-bad ::
  'f list ⇒ nat ⇒ 'f list ⇒ 'f set
and composition-bad ::
  'f ⇒ 'f list ⇒ nat ⇒ 'f list ⇒ 'f set

```

and *trace-fresh trace-proximity trace-index trace-domain-length*
trace-sampled-domain trace-conflict trace-zero
composition-fresh composition-proximity composition-index
composition-domain-length composition-sampled-domain
composition-conflict composition-zero
empty-conflict empty-zero :: prob
assumes *false-statement: $\neg$ exists-valid-trace*
and *wf: staged-budget-wellformed budgets*
and *controlled: staged-adversary-controlled budgets A*
and *trace-fresh-bound:*
wp-event (checked-staged-security-experiment-with-data-state A)
(staged-security-with-data-state-verifier-event
($\lambda s.$
trace-fri-challenge-list-fresh-hit s
(fri-challenge-space (ceil-log clength))))
adversary-initial-state $\leq$ trace-fresh
and *trace-proximity-bound:*
 $\wedge$ *data attacker-state.*
Some (data, attacker-state) $\in$
set-dist (execute (checked-staged-transcript-program A)
adversary-initial-state) $\impl$
wp-event verify-monad
(trace-fri-sampled-full-cover-proximity-failure
(verifier-state-from-adversary attacker-state
(staged-proof-transcript data)) trace-bad)
(verifier-state-from-adversary attacker-state
(staged-proof-transcript data)) $\leq$ trace-proximity
and *trace-index-bound:*
 $\wedge$ *data attacker-state.*
Some (data, attacker-state) $\in$
set-dist (execute (checked-staged-transcript-program A)
adversary-initial-state) $\impl$
wp-event verify-monad
(trace-fri-sampled-full-cover-index-failure
(verifier-state-from-adversary attacker-state
(staged-proof-transcript data))))
(verifier-state-from-adversary attacker-state
(staged-proof-transcript data)) $\leq$ trace-index
and *trace-domain-length-bound:*
 $\wedge$ *data attacker-state.*
Some (data, attacker-state) $\in$
set-dist (execute (checked-staged-transcript-program A)
adversary-initial-state) $\impl$
wp-event verify-monad
(trace-fri-sampled-full-cover-domain-length-failure
(verifier-state-from-adversary attacker-state
(staged-proof-transcript data))))
(verifier-state-from-adversary attacker-state
(staged-proof-transcript data)) $\leq$ trace-domain-length

and *trace-sampled-domain-bound*:
 $\wedge$ *data attacker-state*.
Some (*data*, *attacker-state*) $\in$
set-dist (*execute* (*checked-staged-transcript-program* *A*)
adversary-initial-state) $\implies$
wp-event verify-monad
(*trace-fri-sampled-full-cover-sampled-domain-failure*
(*verifier-state-from-adversary attacker-state*
(*staged-proof-transcript data*)))
(*verifier-state-from-adversary attacker-state*
(*staged-proof-transcript data*)) $\leq$ *trace-sampled-domain*
and *trace-conflict-bound*:
 $\wedge$ *data attacker-state*.
Some (*data*, *attacker-state*) $\in$
set-dist (*execute* (*checked-staged-transcript-program* *A*)
adversary-initial-state) $\implies$
wp-event verify-monad
(*trace-fri-header-tied-sampled-assignment-conflict*
(*verifier-state-from-adversary attacker-state*
(*staged-proof-transcript data*)))
(*verifier-state-from-adversary attacker-state*
(*staged-proof-transcript data*)) $\leq$ *trace-conflict*
and *trace-zero-bound*:
 $\wedge$ *data attacker-state*.
Some (*data*, *attacker-state*) $\in$
set-dist (*execute* (*checked-staged-transcript-program* *A*)
adversary-initial-state) $\implies$
wp-event verify-monad
(*trace-fri-header-tied-zero-round-final-obstruction*
(*verifier-state-from-adversary attacker-state*
(*staged-proof-transcript data*)))
(*verifier-state-from-adversary attacker-state*
(*staged-proof-transcript data*)) $\leq$ *trace-zero*
and *trace-total*:
(*trace-fresh* +
hash-relation-budget-value
(*card* (*fri-challenge-space* (*ceil-log clength*))) * *ceil-log clength*)
(*staged-attacker-query-budget budgets* +
staged-challenge-query-budget)) +
(*trace-proximity* + *trace-index* +
(*trace-index* + *trace-domain-length*) +
(*trace-index* + *trace-sampled-domain*))) +
(*trace-conflict* + *trace-zero*)
 $\leq$ *trace-fri-error*
and *composition-fresh-bound*:
wp-event (*checked-staged-security-experiment-with-data-state A*)
(*staged-security-with-data-state-verifier-event*
(λs .
composition-fri-challenge-list-fresh-hit s

$(\lambda dg. \text{fri-challenge-space } (\text{ceil-log } (\text{to-nat } dg + 1))))$
 $\text{adversary-initial-state} \leq \text{composition-fresh}$
and $\text{composition-proximity-bound}$:
 $\wedge \text{data attacker-state.}$
 $\text{Some } (data, \text{attacker-state}) \in$
 $\text{set-dist } (\text{execute } (\text{checked-staged-transcript-program } A)$
 $\text{adversary-initial-state}) \implies$
 $\text{wp-event verify-monad}$
 $(\text{composition-fri-sampled-full-cover-proximity-failure}$
 $(\text{verifier-state-from-adversary attacker-state}$
 $(\text{staged-proof-transcript data})) \text{ composition-bad})$
 $(\text{verifier-state-from-adversary attacker-state}$
 $(\text{staged-proof-transcript data})) \leq \text{composition-proximity}$
and $\text{composition-index-bound}$:
 $\wedge \text{data attacker-state.}$
 $\text{Some } (data, \text{attacker-state}) \in$
 $\text{set-dist } (\text{execute } (\text{checked-staged-transcript-program } A)$
 $\text{adversary-initial-state}) \implies$
 $\text{wp-event verify-monad}$
 $(\text{composition-fri-sampled-full-cover-index-failure}$
 $(\text{verifier-state-from-adversary attacker-state}$
 $(\text{staged-proof-transcript data})))$
 $(\text{verifier-state-from-adversary attacker-state}$
 $(\text{staged-proof-transcript data})) \leq \text{composition-index}$
and $\text{composition-domain-length-bound}$:
 $\wedge \text{data attacker-state.}$
 $\text{Some } (data, \text{attacker-state}) \in$
 $\text{set-dist } (\text{execute } (\text{checked-staged-transcript-program } A)$
 $\text{adversary-initial-state}) \implies$
 $\text{wp-event verify-monad}$
 $(\text{composition-fri-sampled-full-cover-domain-length-failure}$
 $(\text{verifier-state-from-adversary attacker-state}$
 $(\text{staged-proof-transcript data})))$
 $(\text{verifier-state-from-adversary attacker-state}$
 $(\text{staged-proof-transcript data})) \leq \text{composition-domain-length}$
and $\text{composition-sampled-domain-bound}$:
 $\wedge \text{data attacker-state.}$
 $\text{Some } (data, \text{attacker-state}) \in$
 $\text{set-dist } (\text{execute } (\text{checked-staged-transcript-program } A)$
 $\text{adversary-initial-state}) \implies$
 $\text{wp-event verify-monad}$
 $(\text{composition-fri-sampled-full-cover-sampled-domain-failure}$
 $(\text{verifier-state-from-adversary attacker-state}$
 $(\text{staged-proof-transcript data})))$
 $(\text{verifier-state-from-adversary attacker-state}$
 $(\text{staged-proof-transcript data})) \leq \text{composition-sampled-domain}$
and $\text{composition-conflict-bound}$:
 $\wedge \text{data attacker-state.}$
 $\text{Some } (data, \text{attacker-state}) \in$

$\text{set-dist } (\text{execute } (\text{checked-staged-transcript-program } A) \text{ adversary-initial-state}) \implies$
 $\text{wp-event verify-monad}$
 $(\text{composition-fri-verifier-tied-sampled-assignment-conflict}$
 $(\text{verifier-state-from-adversary attacker-state}$
 $(\text{staged-proof-transcript data})))$
 $(\text{verifier-state-from-adversary attacker-state}$
 $(\text{staged-proof-transcript data})) \leq \text{composition-conflict}$
and $\text{composition-zero-bound:}$
 $\wedge \text{data attacker-state.}$
 $\text{Some } (data, \text{attacker-state}) \in$
 $\text{set-dist } (\text{execute } (\text{checked-staged-transcript-program } A) \text{ adversary-initial-state}) \implies$
 $\text{wp-event verify-monad}$
 $(\text{composition-fri-verifier-tied-zero-round-final-obstruction}$
 $(\text{verifier-state-from-adversary attacker-state}$
 $(\text{staged-proof-transcript data})))$
 $(\text{verifier-state-from-adversary attacker-state}$
 $(\text{staged-proof-transcript data})) \leq \text{composition-zero}$
and $\text{composition-total:}$
 $((\text{composition-fresh } +$
 $\text{hash-relation-budget-value}$
 $(\sum dg \in (\text{UNIV} :: 'f \text{ set}).$
 $\text{card } (\text{fri-challenge-space } (\text{ceil-log } (\text{to-nat } dg + 1))) *$
 $\text{ceil-log } (\text{maxDegree} + 1))) *$
 $(\text{staged-attacker-query-budget budgets } +$
 $\text{staged-challenge-query-budget})) +$
 $(\text{composition-proximity } + \text{composition-index } +$
 $(\text{composition-index } + \text{composition-domain-length}) +$
 $(\text{composition-index } + \text{composition-sampled-domain}))) +$
 $(\text{composition-conflict } + \text{composition-zero})$
 $\leq \text{composition-fri-error}$
and $\text{empty-conflict-bound:}$
 $\wedge \text{data attacker-state.}$
 $\text{Some } (data, \text{attacker-state}) \in$
 $\text{set-dist } (\text{execute } (\text{checked-staged-transcript-program } A) \text{ adversary-initial-state}) \implies$
 $\text{wp-event verify-monad}$
 $(\text{trace-fri-sampled-assignment-conflict}$
 $(\text{verifier-state-from-adversary attacker-state}$
 $(\text{staged-proof-transcript data})))$
 $(\text{verifier-state-from-adversary attacker-state}$
 $(\text{staged-proof-transcript data})) \leq \text{empty-conflict}$
and empty-zero-bound:
 $\wedge \text{data attacker-state.}$
 $\text{Some } (data, \text{attacker-state}) \in$
 $\text{set-dist } (\text{execute } (\text{checked-staged-transcript-program } A) \text{ adversary-initial-state}) \implies$
 $\text{wp-event verify-monad}$

$(\text{trace-fri-zero-round-final-obstruction}$
 $(\text{verifier-state-from-adversary attacker-state}$
 $(\text{staged-proof-transcript data})))$
 $(\text{verifier-state-from-adversary attacker-state}$
 $(\text{staged-proof-transcript data})) \leq \text{empty-zero}$
and empty-total :
 $((\text{trace-fresh} +$
 $\text{hash-relation-budget-value}$
 $(\text{card} (\text{fri-challenge-space} (\text{ceil-log clength})) * \text{ceil-log clength})$
 $(\text{staged-attacker-query-budget budgets} +$
 $\text{staged-challenge-query-budget})) +$
 $(\text{trace-proximity} + \text{trace-index} +$
 $(\text{trace-index} + \text{trace-domain-length}) +$
 $(\text{trace-index} + \text{trace-sampled-domain}))) +$
 $(\text{empty-conflict} + \text{empty-zero})$
 $\leq \text{trace-fri-error}$
shows $\text{checked-staged-adversary-acceptance-probability } A \leq$
 $\text{hash-collision-budget-value } 0$
 $(\text{staged-attacker-query-budget budgets} +$
 $\text{staged-challenge-query-budget} + \text{verifier-hash-query-budget}) +$
 $(\text{composition-error-bound} +$
 $\text{staged-phase-relation-error size}$
 $(\text{staged-alpha-search-queries budgets } 0) +$
 $\text{hash-collision-budget-value } 0$
 $(\text{staged-alpha-search-queries budgets } 0) +$
 $\text{staged-phase-relation-error size}$
 $(\text{staged-alpha-search-queries budgets } 0) +$
 $(1 + \text{staged-concrete-transcript-target-error-bound})) +$
 $(\sum_{i < \text{rounds.}}$
 $1 +$
 $(1 + \text{staged-concrete-transcript-target-error-bound}) +$
 $(1 + \text{staged-concrete-transcript-target-error-bound})) +$
 $\text{trace-fri-error} + \text{composition-fri-error} +$
 $(\text{composition-error-bound} +$
 $\text{staged-phase-relation-error size}$
 $(\text{staged-alpha-search-queries budgets } 0) +$
 $\text{hash-collision-budget-value } 0$
 $(\text{staged-alpha-search-queries budgets } 0) +$
 $\text{staged-phase-relation-error size}$
 $(\text{staged-alpha-search-queries budgets } 0) +$
 $(1 + \text{staged-concrete-transcript-target-error-bound}) +$
 $\text{trace-fri-error} + 1)$
proof (rule
 $\text{stark-soundness-from-current-public-prefix-current-query-relevant-drift-route-from-full-cover-domain-conflict-zero-c}$
 $[\text{where } \text{trace-B} = \text{fri-challenge-space} (\text{ceil-log clength})$
and $\text{composition-B} =$
 $\lambda dg. \text{fri-challenge-space} (\text{ceil-log} (\text{to-nat } dg + 1))$
and $\text{trace-bad} = \text{trace-bad}$
and $\text{composition-bad} = \text{composition-bad}]$)

```

show  $\neg$  exists-valid-trace
  by (rule false-statement)
show staged-budget-wellformed budgets
  by (rule wf)
show staged-adversary-controlled budgets A
  by (rule controlled)
show finite (fri-challenge-space (ceil-log clength))
  by simp
show  $\bigwedge$ data attacker-state trace-table.
  Some (data, attacker-state)  $\in$ 
    set-dist (execute (checked-staged-transcript-program A)
      adversary-initial-state)  $\implies$ 
    trace-fri-multiround-bad-sets trace-bad trace-table
     $\subseteq$  fri-challenge-space (ceil-log clength)
  by (rule trace-fri-multiround-bad-sets-subset)
show wp-event (checked-staged-security-experiment-with-data-state A)
  (staged-security-with-data-state-verifier-event
    ( $\lambda s.$ 
      trace-fri-challenge-list-fresh-hit s
      (fri-challenge-space (ceil-log clength))))
  adversary-initial-state  $\leq$  trace-fresh
  by (rule trace-fresh-bound)
show  $\bigwedge$ data attacker-state.
  Some (data, attacker-state)  $\in$ 
    set-dist (execute (checked-staged-transcript-program A)
      adversary-initial-state)  $\implies$ 
  wp-event verify-monad
    (trace-fri-sampled-full-cover-proximity-failure
      (verifier-state-from-adversary attacker-state
        (staged-proof-transcript data)) trace-bad)
    (verifier-state-from-adversary attacker-state
      (staged-proof-transcript data))  $\leq$  trace-proximity
  by (rule trace-proximity-bound)
show  $\bigwedge$ data attacker-state.
  Some (data, attacker-state)  $\in$ 
    set-dist (execute (checked-staged-transcript-program A)
      adversary-initial-state)  $\implies$ 
  wp-event verify-monad
    (trace-fri-sampled-full-cover-index-failure
      (verifier-state-from-adversary attacker-state
        (staged-proof-transcript data)))
    (verifier-state-from-adversary attacker-state
      (staged-proof-transcript data))  $\leq$  trace-index
  by (rule trace-index-bound)
show  $\bigwedge$ data attacker-state.
  Some (data, attacker-state)  $\in$ 
    set-dist (execute (checked-staged-transcript-program A)
      adversary-initial-state)  $\implies$ 
  wp-event verify-monad

```

```

    (trace-fri-sampled-full-cover-domain-length-failure
     (verifier-state-from-adversary attacker-state
      (staged-proof-transcript data)))
    (verifier-state-from-adversary attacker-state
     (staged-proof-transcript data)) ≤ trace-domain-length
  by (rule trace-domain-length-bound)
show ∧ data attacker-state.
  Some (data, attacker-state) ∈
    set-dist (execute (checked-staged-transcript-program A)
               adversary-initial-state) ⇒
  wp-event verify-monad
    (trace-fri-sampled-full-cover-sampled-domain-failure
     (verifier-state-from-adversary attacker-state
      (staged-proof-transcript data)))
    (verifier-state-from-adversary attacker-state
     (staged-proof-transcript data)) ≤ trace-sampled-domain
  by (rule trace-sampled-domain-bound)
show ∧ data attacker-state.
  Some (data, attacker-state) ∈
    set-dist (execute (checked-staged-transcript-program A)
               adversary-initial-state) ⇒
  wp-event verify-monad
    (trace-fri-header-tied-sampled-assignment-conflict
     (verifier-state-from-adversary attacker-state
      (staged-proof-transcript data)))
    (verifier-state-from-adversary attacker-state
     (staged-proof-transcript data)) ≤ trace-conflict
  by (rule trace-conflict-bound)
show ∧ data attacker-state.
  Some (data, attacker-state) ∈
    set-dist (execute (checked-staged-transcript-program A)
               adversary-initial-state) ⇒
  wp-event verify-monad
    (trace-fri-header-tied-zero-round-final-obstruction
     (verifier-state-from-adversary attacker-state
      (staged-proof-transcript data)))
    (verifier-state-from-adversary attacker-state
     (staged-proof-transcript data)) ≤ trace-zero
  by (rule trace-zero-bound)
show ((trace-fresh +
      hash-relation-budget-value
      (card (fri-challenge-space (ceil-log clength)) * ceil-log clength)
      (staged-attacker-query-budget budgets +
       staged-challenge-query-budget)) +
     (trace-proximity + trace-index +
      (trace-index + trace-domain-length) +
      (trace-index + trace-sampled-domain))) +
     (trace-conflict + trace-zero)
    ≤ trace-fri-error

```

```

  by (rule trace-total)
show  $\bigwedge dg. \text{finite } (\text{fri-challenge-space } (\text{ceil-log } (\text{to-nat } dg + 1)))$ 
  by simp
show  $\bigwedge \text{data attacker-state } dg \text{ composition-table.}$ 
  Some (data, attacker-state)  $\in$ 
    set-dist (execute (checked-staged-transcript-program A)
      adversary-initial-state)  $\implies$ 
    composition-fri-multiround-bad-sets composition-bad dg
    composition-table
     $\subseteq$  fri-challenge-space (ceil-log (to-nat dg + 1))
  by (rule composition-fri-multiround-bad-sets-subset)
show wp-event (checked-staged-security-experiment-with-data-state A)
  (staged-security-with-data-state-verifier-event
    ( $\lambda s.$ 
      composition-fri-challenge-list-fresh-hit s
      ( $\lambda dg. \text{fri-challenge-space } (\text{ceil-log } (\text{to-nat } dg + 1)))$ ))
    adversary-initial-state  $\leq$  composition-fresh
  by (rule composition-fresh-bound)
show  $\bigwedge \text{data attacker-state.}$ 
  Some (data, attacker-state)  $\in$ 
    set-dist (execute (checked-staged-transcript-program A)
      adversary-initial-state)  $\implies$ 
    wp-event verify-monad
      (composition-fri-sampled-full-cover-proximity-failure
        (verifier-state-from-adversary attacker-state
          (staged-proof-transcript data)) composition-bad)
        (verifier-state-from-adversary attacker-state
          (staged-proof-transcript data))  $\leq$  composition-proximity
  by (rule composition-proximity-bound)
show  $\bigwedge \text{data attacker-state.}$ 
  Some (data, attacker-state)  $\in$ 
    set-dist (execute (checked-staged-transcript-program A)
      adversary-initial-state)  $\implies$ 
    wp-event verify-monad
      (composition-fri-sampled-full-cover-index-failure
        (verifier-state-from-adversary attacker-state
          (staged-proof-transcript data)))
        (verifier-state-from-adversary attacker-state
          (staged-proof-transcript data))  $\leq$  composition-index
  by (rule composition-index-bound)
show  $\bigwedge \text{data attacker-state.}$ 
  Some (data, attacker-state)  $\in$ 
    set-dist (execute (checked-staged-transcript-program A)
      adversary-initial-state)  $\implies$ 
    wp-event verify-monad
      (composition-fri-sampled-full-cover-domain-length-failure
        (verifier-state-from-adversary attacker-state
          (staged-proof-transcript data)))
        (verifier-state-from-adversary attacker-state

```

$(\text{staged-proof-transcript data}) \leq \text{composition-domain-length}$
by (rule composition-domain-length-bound)
show $\bigwedge \text{data attacker-state.}$
 $\text{Some (data, attacker-state)} \in$
 $\text{set-dist (execute (checked-staged-transcript-program A)}$
 $\text{adversary-initial-state)} \implies$
 $\text{wp-event verify-monad}$
 $(\text{composition-fri-sampled-full-cover-sampled-domain-failure}$
 $(\text{verifier-state-from-adversary attacker-state}$
 $(\text{staged-proof-transcript data})))$
 $(\text{verifier-state-from-adversary attacker-state}$
 $(\text{staged-proof-transcript data})) \leq \text{composition-sampled-domain}$
by (rule composition-sampled-domain-bound)
show $\bigwedge \text{data attacker-state.}$
 $\text{Some (data, attacker-state)} \in$
 $\text{set-dist (execute (checked-staged-transcript-program A)}$
 $\text{adversary-initial-state)} \implies$
 $\text{wp-event verify-monad}$
 $(\text{composition-fri-verifier-tied-sampled-assignment-conflict}$
 $(\text{verifier-state-from-adversary attacker-state}$
 $(\text{staged-proof-transcript data})))$
 $(\text{verifier-state-from-adversary attacker-state}$
 $(\text{staged-proof-transcript data})) \leq \text{composition-conflict}$
by (rule composition-conflict-bound)
show $\bigwedge \text{data attacker-state.}$
 $\text{Some (data, attacker-state)} \in$
 $\text{set-dist (execute (checked-staged-transcript-program A)}$
 $\text{adversary-initial-state)} \implies$
 $\text{wp-event verify-monad}$
 $(\text{composition-fri-verifier-tied-zero-round-final-obstruction}$
 $(\text{verifier-state-from-adversary attacker-state}$
 $(\text{staged-proof-transcript data})))$
 $(\text{verifier-state-from-adversary attacker-state}$
 $(\text{staged-proof-transcript data})) \leq \text{composition-zero}$
by (rule composition-zero-bound)
show $((\text{composition-fresh} +$
 $\text{hash-relation-budget-value}$
 $(\sum dg \in (\text{UNIV} :: 'f \text{ set}).$
 $\text{card (fri-challenge-space (ceil-log (to-nat dg + 1)))} *$
 $\text{ceil-log (maxDegree + 1))}$
 $(\text{staged-attacker-query-budget budgets} +$
 $\text{staged-challenge-query-budget})) +$
 $(\text{composition-proximity} + \text{composition-index} +$
 $(\text{composition-index} + \text{composition-domain-length}) +$
 $(\text{composition-index} + \text{composition-sampled-domain}))) +$
 $(\text{composition-conflict} + \text{composition-zero})$
 $\leq \text{composition-fri-error}$
by (rule composition-total)
show $\bigwedge \text{data attacker-state.}$

Some (data, attacker-state) ∈
set-dist (execute (checked-staged-transcript-program A)
adversary-initial-state) ⇒
wp-event verify-monad
(trace-fri-sampled-assignment-conflict
(verifier-state-from-adversary attacker-state
(staged-proof-transcript data)))
(verifier-state-from-adversary attacker-state
(staged-proof-transcript data)) ≤ empty-conflict
by (rule empty-conflict-bound)
show $\bigwedge$ data attacker-state.
Some (data, attacker-state) ∈
set-dist (execute (checked-staged-transcript-program A)
adversary-initial-state) ⇒
wp-event verify-monad
(trace-fri-zero-round-final-obstruction
(verifier-state-from-adversary attacker-state
(staged-proof-transcript data)))
(verifier-state-from-adversary attacker-state
(staged-proof-transcript data)) ≤ empty-zero
by (rule empty-zero-bound)
show ((trace-fresh +
hash-relation-budget-value
(card (fri-challenge-space (ceil-log clength)) * ceil-log clength)
(staged-attacker-query-budget budgets +
staged-challenge-query-budget)) +
(trace-proximity + trace-index +
(trace-index + trace-domain-length) +
(trace-index + trace-sampled-domain))) +
(empty-conflict + empty-zero)
≤ trace-fri-error
by (rule empty-total)
qed

definition concrete-trace-fri-header-error-for
where

concrete-trace-fri-header-error-for A =
wp-event (checked-staged-security-experiment-with-data-state A)
(staged-security-with-data-state-verifier-event
trace-fri-bad-with-header-tied-partial-candidate)
adversary-initial-state

definition concrete-composition-fri-error-for
where

concrete-composition-fri-error-for A =
wp-event (checked-staged-security-experiment-with-data-state A)
(staged-security-with-data-state-verifier-event
composition-fri-bad-with-verifier-tied-partial-candidate)
adversary-initial-state

definition *concrete-trace-fri-empty-error-for*

where

concrete-trace-fri-empty-error-for $A =$
wp-event (*checked-staged-security-experiment-with-data-state* A)
 (*staged-security-with-data-state-verifier-event*
trace-fri-bad-with-empty-composition-header-candidates)
adversary-initial-state

theorem *stark-soundness-from-current-public-prefix-current-query-relevant-drift-route:*

assumes *false-statement*: $\neg$ *exists-valid-trace*

and *wf*: *staged-budget-wellformed* *budgets*

and *controlled*: *staged-adversary-controlled* *budgets* A

shows *checked-staged-adversary-acceptance-probability* $A \leq$

hash-collision-budget-value 0
 (*staged-attacker-query-budget* *budgets* $+$
staged-challenge-query-budget $+$ *verifier-hash-query-budget*) $+$
 (*composition-error-bound* $+$
staged-phase-relation-error *size*
 (*staged-alpha-search-queries* *budgets* 0) $+$
hash-collision-budget-value 0
 (*staged-alpha-search-queries* *budgets* 0) $+$
staged-phase-relation-error *size*
 (*staged-alpha-search-queries* *budgets* 0) $+$
 ($1 +$ *staged-concrete-transcript-target-error-bound*)) $+$
 ($\sum i < \text{rounds.}$
 $1 +$
 ($1 +$ *staged-concrete-transcript-target-error-bound*) $+$
 ($1 +$ *staged-concrete-transcript-target-error-bound*)) $+$
concrete-trace-fri-header-error-for $A +$
concrete-composition-fri-error-for $A +$
 (*composition-error-bound* $+$
staged-phase-relation-error *size*
 (*staged-alpha-search-queries* *budgets* 0) $+$
hash-collision-budget-value 0
 (*staged-alpha-search-queries* *budgets* 0) $+$
staged-phase-relation-error *size*
 (*staged-alpha-search-queries* *budgets* 0) $+$
 ($1 +$ *staged-concrete-transcript-target-error-bound*) $+$
concrete-trace-fri-empty-error-for $A + 1$)

proof $-$

have *trace-fri-bound*:

wp-event (*checked-staged-security-experiment-with-data-state* A)
 (*staged-security-with-data-state-verifier-event*
trace-fri-bad-with-header-tied-partial-candidate)
adversary-initial-state $\leq$ *concrete-trace-fri-header-error-for* A

by (*simp add*: *concrete-trace-fri-header-error-for-def*)

have *comp-fri-bound*:

wp-event (*checked-staged-security-experiment-with-data-state* A)

(staged-security-with-data-state-verifier-event
 composition-fri-bad-with-verifier-tied-partial-candidate)
 adversary-initial-state $\leq$ concrete-composition-fri-error-for A
 by (simp add: concrete-composition-fri-error-for-def)
have empty-trace-fri-bound:
 wp-event (checked-staged-security-experiment-with-data-state A)
 (staged-security-with-data-state-verifier-event
 trace-fri-bad-with-empty-composition-header-candidates)
 adversary-initial-state $\leq$ concrete-trace-fri-empty-error-for A
 by (simp add: concrete-trace-fri-empty-error-for-def)
show ?thesis
 by (rule
 current-query-route-bound-from-active-fri-bounds-internal
 [OF false-statement wf controlled trace-fri-bound comp-fri-bound
 empty-trace-fri-bound])
qed

theorem stark-soundness-same-run-endpoint:
assumes false-statement: $\neg$ exists-valid-trace
and wf: staged-budget-wellformed budgets
and controlled: staged-adversary-controlled budgets A
shows checked-staged-adversary-acceptance-probability $A \leq$
 hash-collision-budget-value 0
 (staged-attacker-query-budget budgets +
 staged-challenge-query-budget + verifier-hash-query-budget) +
 (composition-error-bound +
 staged-phase-relation-error size
 (staged-alpha-search-queries budgets 0) +
 hash-collision-budget-value 0
 (staged-alpha-search-queries budgets 0) +
 staged-phase-relation-error size
 (staged-alpha-search-queries budgets 0) +
 (1 + staged-concrete-transcript-target-error-bound)) +
 ($\sum i < \text{rounds.}$
 1 +
 (1 + staged-concrete-transcript-target-error-bound) +
 (1 + staged-concrete-transcript-target-error-bound)) +
 concrete-trace-fri-header-error-for A +
 concrete-composition-fri-error-for A +
 (composition-error-bound +
 staged-phase-relation-error size
 (staged-alpha-search-queries budgets 0) +
 hash-collision-budget-value 0
 (staged-alpha-search-queries budgets 0) +
 staged-phase-relation-error size
 (staged-alpha-search-queries budgets 0) +
 (1 + staged-concrete-transcript-target-error-bound) +
 concrete-trace-fri-empty-error-for A + 1)
by (rule

stark-soundness-from-current-public-prefix-current-query-relevant-drift-route
[OF false-statement wf controlled])

end

hide-fact

soundness.stark-soundness-same-run-endpoint
soundness.stark-soundness-from-current-public-prefix-current-query-relevant-drift-route
soundness.stark-soundness-from-current-public-prefix-current-query-relevant-drift-route-from-staged-active-fri-bound
soundness.stark-soundness-from-current-public-prefix-current-query-relevant-drift-route-from-active-fri-bounds
soundness.stark-soundness-from-current-public-prefix-current-query-relevant-drift-route-from-active-fri-reductions
soundness.stark-soundness-from-current-public-prefix-current-query-relevant-drift-route-from-derived-active-fri-conf
soundness.stark-soundness-from-current-public-prefix-current-query-relevant-drift-route-from-derived-active-fri-conf
soundness.stark-soundness-from-current-public-prefix-current-query-relevant-drift-route-from-derived-active-fri-cano
soundness.stark-soundness-from-current-public-prefix-current-query-relevant-drift-route-from-sampled-query-fri-conf
soundness.stark-soundness-from-current-public-prefix-current-query-relevant-drift-route-from-sampled-query-projecti
soundness.stark-soundness-from-current-public-prefix-current-query-relevant-drift-route-from-trace-active-and-staged
soundness.stark-soundness-from-current-public-prefix-current-query-relevant-drift-route-from-full-cover-domain-conf
soundness.stark-soundness-from-current-public-prefix-current-query-relevant-drift-route-from-full-cover-domain-conf
soundness.stark-soundness-from-current-public-prefix-current-query-relevant-drift-route-from-active-fri-residual-obli
soundness.stark-soundness-from-current-public-prefix-current-query-relevant-drift-route-from-active-fri-residual-com

end

theory *Soundness-Public-Route-Concrete-FRI-Endpoint*

imports *Soundness-Public-Route-Same-Run*

begin

Public-route endpoint with concrete FRI losses expanded one step below the top-level bad-event probabilities.

This theory is retained as infrastructure for the current sharpened endpoint: downstream theories reuse the reachable verifier-event envelope definitions and one-step concrete FRI bounds. The active public theorem is exported by `Soundness_FRI_Trace_Restricted_Query_Endpoint.th`

context *soundness*

begin

definition *reachable-verifier-event-bound-for*

where

reachable-verifier-event-bound-for A E =
Max (insert 0
((λout. case out of
None ⇒ 0
| Some y ⇒
let s =
verifier-state-from-adversary (snd y)
(staged-proof-transcript (fst y))
in wp-event verify-monad (E s) s) ‘
set-dist

(execute (checked-staged-transcript-program A)
adversary-initial-state)))

lemma *reachable-verifier-event-bound-for-ge*:

assumes *support*:

Some (data, attacker-state) ∈
set-dist
(execute (checked-staged-transcript-program A)
adversary-initial-state)

shows

wp-event verify-monad
(E
(verifier-state-from-adversary attacker-state
(staged-proof-transcript data)))
(verifier-state-from-adversary attacker-state
(staged-proof-transcript data)))
≤ reachable-verifier-event-bound-for A E

proof –

let ?F =

(λout. case out of
None ⇒ 0
| Some y ⇒
let s =
verifier-state-from-adversary (snd y)
(staged-proof-transcript (fst y))
in wp-event verify-monad (E s) s)

let ?S =

insert 0
(?F ‘
set-dist
(execute (checked-staged-transcript-program A)
adversary-initial-state)))

have *finite-S*: finite ?S

by *simp*

have *mem*: ?F (Some (data, attacker-state)) ∈ ?S

using *support* **by** *blast*

have *le*: ?F (Some (data, attacker-state)) ≤ Max ?S

by (rule *Max-ge*[OF *finite-S mem*])

have *F-eq*:

?F (Some (data, attacker-state)) =
wp-event verify-monad
(E
(verifier-state-from-adversary attacker-state
(staged-proof-transcript data)))
(verifier-state-from-adversary attacker-state
(staged-proof-transcript data)))

by (*simp add: Let-def*)

show ?thesis

unfolding *reachable-verifier-event-bound-for-def*

using le *F-eq* by *simp*
qed

definition *trace-fri-full-challenge-bad*

where

trace-fri-full-challenge-bad - - - = *UNIV*

definition *composition-fri-full-challenge-bad*

where

composition-fri-full-challenge-bad - - - = *UNIV*

definition *concrete-trace-fri-full-cover-domain-error-for*

where

concrete-trace-fri-full-cover-domain-error-for budgets *A* =
 ((*wp-event* (*checked-staged-security-experiment-with-data-state* *A*)
 (*staged-security-with-data-state-verifier-event*
 ($\lambda s.$
 trace-fri-challenge-list-fresh-hit *s*
 (*fri-challenge-space* (*ceil-log* *clength*))))
 adversary-initial-state +
 hash-relation-budget-value
 (*card* (*fri-challenge-space* (*ceil-log* *clength*))) * *ceil-log* *clength*)
 (*staged-attacker-query-budget* budgets +
 staged-challenge-query-budget)) +
 (*reachable-verifier-event-bound-for* *A*
 ($\lambda s.$
 trace-fri-sampled-full-cover-proximity-failure *s*
 trace-fri-full-challenge-bad) +
 reachable-verifier-event-bound-for *A*
 trace-fri-sampled-full-cover-index-failure +
 (*reachable-verifier-event-bound-for* *A*
 trace-fri-sampled-full-cover-index-failure +
 reachable-verifier-event-bound-for *A*
 trace-fri-sampled-full-cover-domain-length-failure) +
 (*reachable-verifier-event-bound-for* *A*
 trace-fri-sampled-full-cover-index-failure +
 reachable-verifier-event-bound-for *A*
 trace-fri-sampled-full-cover-sampled-domain-failure))))

definition *concrete-trace-fri-header-error-for'*

where

concrete-trace-fri-header-error-for' budgets *A* =
concrete-trace-fri-full-cover-domain-error-for budgets *A* +
 (*reachable-verifier-event-bound-for* *A*
 trace-fri-header-tied-sampled-assignment-conflict +
 reachable-verifier-event-bound-for *A*
 trace-fri-header-tied-zero-round-final-obstruction)

definition *concrete-trace-fri-empty-error-for'*

where

concrete-trace-fri-empty-error-for' budgets $A =$
concrete-trace-fri-full-cover-domain-error-for budgets $A +$
 (*reachable-verifier-event-bound-for* A
trace-fri-sampled-assignment-conflict $+$
reachable-verifier-event-bound-for A
trace-fri-zero-round-final-obstruction)

definition *concrete-composition-fri-full-cover-domain-error-for*

where

concrete-composition-fri-full-cover-domain-error-for budgets $A =$
 ((*wp-event* (*checked-staged-security-experiment-with-data-state* A)
 (*staged-security-with-data-state-verifier-event*
 ($\lambda s.$
 composition-fri-challenge-list-fresh-hit s
 ($\lambda dg.$ *fri-challenge-space* (*ceil-log* (*to-nat* $dg + 1$))))))
adversary-initial-state $+$
hash-relation-budget-value
 ($\sum dg \in (UNIV :: 'f \text{ set}).$
 card (*fri-challenge-space* (*ceil-log* (*to-nat* $dg + 1$))) $*$
 ceil-log (*maxDegree* $+ 1$))
 (*staged-attacker-query-budget* budgets $+$
 staged-challenge-query-budget) $+$
 (*reachable-verifier-event-bound-for* A
 ($\lambda s.$
 composition-fri-sampled-full-cover-proximity-failure s
 composition-fri-full-challenge-bad) $+$
reachable-verifier-event-bound-for A
 composition-fri-sampled-full-cover-index-failure $+$
 (*reachable-verifier-event-bound-for* A
 composition-fri-sampled-full-cover-index-failure $+$
 reachable-verifier-event-bound-for A
 composition-fri-sampled-full-cover-domain-length-failure) $+$
 (*reachable-verifier-event-bound-for* A
 composition-fri-sampled-full-cover-index-failure $+$
 reachable-verifier-event-bound-for A
 composition-fri-sampled-full-cover-sampled-domain-failure)))

definition *concrete-composition-fri-error-for'*

where

concrete-composition-fri-error-for' budgets $A =$
concrete-composition-fri-full-cover-domain-error-for budgets $A +$
 (*reachable-verifier-event-bound-for* A
 composition-fri-verifier-tied-sampled-assignment-conflict $+$
reachable-verifier-event-bound-for A
 composition-fri-verifier-tied-zero-round-final-obstruction)

lemma *concrete-trace-fri-header-error-for'-bound:*

assumes *wf*: *staged-budget-wellformed* budgets

and *controlled*: staged-adversary-controlled budgets A
shows
 $wp\text{-}event$ (*checked-staged-security-experiment-with-data-state* A)
 (*staged-security-with-data-state-verifier-event*
trace-fri-bad-with-header-tied-partial-candidate)
 $adversary\text{-}initial\text{-}state \leq concrete\text{-}trace\text{-}fri\text{-}header\text{-}error\text{-}for'$ budgets A
unfolding *concrete-trace-fri-header-error-for'-def*
concrete-trace-fri-full-cover-domain-error-for-def
by (*rule*
checked-staged-security-trace-fri-header-tied-partial-candidate-bound-from-full-cover-domain-conflict-and-zero
 [**where** $bad = trace\text{-}fri\text{-}full\text{-}challenge\text{-}bad$
and $B = fri\text{-}challenge\text{-}space$ ($ceil\text{-}log$ $clength$),
 OF wf *controlled*])
 (*auto intro!*: *reachable-verifier-event-bound-for-ge*
dest: *trace-fri-multiround-bad-sets-subset*[*THEN subsetD*])

lemma *concrete-trace-fri-empty-error-for'-bound*:
assumes wf : *staged-budget-wellformed* budgets
and *controlled*: staged-adversary-controlled budgets A
shows
 $wp\text{-}event$ (*checked-staged-security-experiment-with-data-state* A)
 (*staged-security-with-data-state-verifier-event*
trace-fri-bad-with-empty-composition-header-candidates)
 $adversary\text{-}initial\text{-}state \leq concrete\text{-}trace\text{-}fri\text{-}empty\text{-}error\text{-}for'$ budgets A
unfolding *concrete-trace-fri-empty-error-for'-def*
concrete-trace-fri-full-cover-domain-error-for-def
by (*rule*
checked-staged-security-trace-fri-empty-header-bound-from-full-cover-domain-conflict-and-zero
 [**where** $bad = trace\text{-}fri\text{-}full\text{-}challenge\text{-}bad$
and $B = fri\text{-}challenge\text{-}space$ ($ceil\text{-}log$ $clength$),
 OF wf *controlled*])
 (*auto intro!*: *reachable-verifier-event-bound-for-ge*
dest: *trace-fri-multiround-bad-sets-subset*[*THEN subsetD*])

lemma *concrete-composition-fri-error-for'-bound*:
assumes wf : *staged-budget-wellformed* budgets
and *controlled*: staged-adversary-controlled budgets A
shows
 $wp\text{-}event$ (*checked-staged-security-experiment-with-data-state* A)
 (*staged-security-with-data-state-verifier-event*
composition-fri-bad-with-verifier-tied-partial-candidate)
 $adversary\text{-}initial\text{-}state \leq concrete\text{-}composition\text{-}fri\text{-}error\text{-}for'$ budgets A
unfolding *concrete-composition-fri-error-for'-def*
concrete-composition-fri-full-cover-domain-error-for-def
by (*rule*
checked-staged-security-composition-fri-verifier-tied-partial-candidate-bound-from-full-cover-domain-conflict-and-zero
 [**where** $bad = composition\text{-}fri\text{-}full\text{-}challenge\text{-}bad$
and $B = \lambda dg. fri\text{-}challenge\text{-}space$ ($ceil\text{-}log$ ($to\text{-}nat$ $dg + 1$)),
 OF wf *controlled*])

(auto intro!: reachable-verifier-event-bound-for-ge
 dest: composition-fri-multi-round-bad-sets-subset[THEN subsetD])

theorem stark-soundness-from-current-public-prefix-current-query-relevant-drift-route:

assumes false-statement: $\neg$ exists-valid-trace

and wf: staged-budget-wellformed budgets

and controlled: staged-adversary-controlled budgets A

shows checked-staged-adversary-acceptance-probability A $\leq$

hash-collision-budget-value 0

(staged-attacker-query-budget budgets +
 staged-challenge-query-budget + verifier-hash-query-budget) +

(composition-error-bound +

staged-phase-relation-error size

(staged-alpha-search-queries budgets 0) +

hash-collision-budget-value 0

(staged-alpha-search-queries budgets 0) +

staged-phase-relation-error size

(staged-alpha-search-queries budgets 0) +

(1 + staged-concrete-transcript-target-error-bound)) +

($\sum$ i < rounds.

1 +

(1 + staged-concrete-transcript-target-error-bound) +

(1 + staged-concrete-transcript-target-error-bound)) +

concrete-trace-fri-header-error-for' budgets A +

concrete-composition-fri-error-for' budgets A +

(composition-error-bound +

staged-phase-relation-error size

(staged-alpha-search-queries budgets 0) +

hash-collision-budget-value 0

(staged-alpha-search-queries budgets 0) +

staged-phase-relation-error size

(staged-alpha-search-queries budgets 0) +

(1 + staged-concrete-transcript-target-error-bound) +

concrete-trace-fri-empty-error-for' budgets A + 1)

by (rule

current-query-route-bound-from-active-fri-bounds-internal

[OF false-statement wf controlled

concrete-trace-fri-header-error-for'-bound[OF wf controlled]

concrete-composition-fri-error-for'-bound[OF wf controlled]

concrete-trace-fri-empty-error-for'-bound[OF wf controlled]])

theorem stark-soundness-concrete-fri-endpoint:

assumes false-statement: $\neg$ exists-valid-trace

and wf: staged-budget-wellformed budgets

and controlled: staged-adversary-controlled budgets A

shows checked-staged-adversary-acceptance-probability A $\leq$

hash-collision-budget-value 0

(staged-attacker-query-budget budgets +

staged-challenge-query-budget + verifier-hash-query-budget) +

```

    (composition-error-bound +
      staged-phase-relation-error size
      (staged-alpha-search-queries budgets 0) +
      hash-collision-budget-value 0
      (staged-alpha-search-queries budgets 0) +
      staged-phase-relation-error size
      (staged-alpha-search-queries budgets 0) +
      (1 + staged-concrete-transcript-target-error-bound)) +
    (∑ i<rounds.
      1 +
      (1 + staged-concrete-transcript-target-error-bound) +
      (1 + staged-concrete-transcript-target-error-bound)) +
    concrete-trace-fri-header-error-for' budgets A +
    concrete-composition-fri-error-for' budgets A +
    (composition-error-bound +
      staged-phase-relation-error size
      (staged-alpha-search-queries budgets 0) +
      hash-collision-budget-value 0
      (staged-alpha-search-queries budgets 0) +
      staged-phase-relation-error size
      (staged-alpha-search-queries budgets 0) +
      (1 + staged-concrete-transcript-target-error-bound) +
      concrete-trace-fri-empty-error-for' budgets A + 1)
  by (rule
    stark-soundness-from-current-public-prefix-current-query-relevant-drift-route
    [OF false-statement wf controlled])

end

hide-fact
  soundness.stark-soundness-concrete-fri-endpoint
  soundness.stark-soundness-from-current-public-prefix-current-query-relevant-drift-route

end

theory Soundness-FRI-Query-Fiber-Staged-Bounds
  imports
    Soundness-FRI-Query-Index-Staged-Bounds
    Soundness-FRI-Query-Aware-Interface
begin

Staged sampled-query FRI bounds from generic query fibers.
The checked staged query-index accounting currently consumes a fixed query target.
This layer derives such a target from the union of the generic per-challenge query fibers
outside a chosen challenge-list cover. It keeps the query-aware FRI boundary explicit
without changing the protocol or the random-oracle model.

context soundness
begin

```


definition *generic-fri-sampled-query-outside-query-cover*

$:: ('f \text{ list} \Rightarrow \text{bool}) \Rightarrow ('f \text{ list} \Rightarrow \text{bool}) \Rightarrow \text{nat} \Rightarrow$
 $\quad 'f \text{ list set} \Rightarrow \text{nat list set}$

where

generic-fri-sampled-query-outside-query-cover low-degree candidate-bad
degree-bound B =
 $(\bigcup \text{challenges} \in$
 $\quad \text{fri-challenge-space (fri-round-count-for-degree-bound degree-bound)} - B.$
generic-fri-sampled-query-query-fiber low-degree candidate-bad
degree-bound challenges)

definition *generic-fri-sampled-query-missing-full-cover-fiber*

$:: ('f \text{ list} \Rightarrow \text{bool}) \Rightarrow ('f \text{ list} \Rightarrow \text{bool}) \Rightarrow \text{nat} \Rightarrow$
 $\quad 'f \text{ list} \Rightarrow \text{nat list set}$

where

generic-fri-sampled-query-missing-full-cover-fiber low-degree candidate-bad
degree-bound challenges =
 $\{ \text{query-idxs} \in \text{fri-query-index-list-space}.$
 $\quad \exists \text{candidate-table roots final-value round-layers layers}.$
 $\quad \text{generic-fri-sampled-query-candidate-evidence low-degree candidate-bad}$
 $\quad \text{candidate-table degree-bound roots challenges final-value query-idxs}$
 $\quad \text{round-layers layers} \wedge$
 $\quad \neg \text{fri-sampled-layers-cover-fold-indices roots query-idxs} \}$

lemma *generic-fri-sampled-query-missing-full-cover-fiber-subset:*

generic-fri-sampled-query-missing-full-cover-fiber low-degree candidate-bad
degree-bound challenges $\subseteq$ fri-query-index-list-space

unfolding *generic-fri-sampled-query-missing-full-cover-fiber-def* **by** *auto*

lemma *finite-generic-fri-sampled-query-missing-full-cover-fiber:*

finite
 $(\text{generic-fri-sampled-query-missing-full-cover-fiber low-degree}$
 $\quad \text{candidate-bad degree-bound challenges})$

using *generic-fri-sampled-query-missing-full-cover-fiber-subset*

by $(\text{rule finite-subset}) (\text{rule finite-fri-query-index-list-space})$

lemma *generic-fri-sampled-query-query-fiber-subset-missing-full-cover:*

assumes *full-cover-imp:*

$\bigwedge \text{candidate-table roots final-value round-layers layers query-idxs}.$
 $\text{query-idxs} \in \text{fri-query-index-list-space} \implies$
 $\text{generic-fri-sampled-query-candidate-evidence low-degree candidate-bad}$
 $\text{candidate-table degree-bound roots challenges final-value query-idxs}$
 $\text{round-layers layers} \implies$
 $\text{fri-sampled-layers-cover-fold-indices roots query-idxs} \implies$
 $\text{challenges} \in B$

and *challenges-notin: challenges $\notin B$*

shows

generic-fri-sampled-query-query-fiber low-degree candidate-bad

degree-bound challenges $\subseteq$
generic-fri-sampled-query-missing-full-cover-fiber low-degree
candidate-bad degree-bound challenges

proof

fix *query-idxs*

assume *query-in*:

query-idxs $\in$
generic-fri-sampled-query-query-fiber low-degree candidate-bad
degree-bound challenges

then have *query-space*: *query-idxs* $\in$ *fri-query-index-list-space*

using *generic-fri-sampled-query-query-fiber-subset* **by** *blast*

from *query-in* **obtain** *candidate-table roots final-value round-layers layers*

where *evidence*:

generic-fri-sampled-query-candidate-evidence low-degree candidate-bad
candidate-table degree-bound roots challenges final-value query-idxs
round-layers layers

unfolding *generic-fri-sampled-query-query-fiber-def*
generic-fri-sampled-query-restricted-bad-pairs-def
generic-fri-sampled-query-bad-pair-union-def
generic-fri-sampled-query-bad-pair-set-def
fri-query-challenge-pair-query-fiber-def

by *auto*

have *not-cover*: $\neg$ *fri-sampled-layers-cover-fold-indices roots query-idxs*

proof

assume *cover*: *fri-sampled-layers-cover-fold-indices roots query-idxs*

have *challenges* $\in B$

by (*rule full-cover-imp*[*OF query-space evidence cover*])

then show *False*

using *challenges-notin* **by** *simp*

qed

show *query-idxs* $\in$
generic-fri-sampled-query-missing-full-cover-fiber low-degree
candidate-bad degree-bound challenges

unfolding *generic-fri-sampled-query-missing-full-cover-fiber-def*

by (*intro CollectI conjI exI*)
(*rule query-space, rule evidence, rule not-cover*)

qed

lemma *generic-fri-sampled-query-query-fiber-entries-card-le-missing-full-cover*:

assumes *full-cover-imp*:

$\bigwedge$ *candidate-table roots final-value round-layers layers query-idxs*.
query-idxs $\in$ *fri-query-index-list-space* $\implies$
generic-fri-sampled-query-candidate-evidence low-degree candidate-bad
candidate-table degree-bound roots challenges final-value query-idxs
round-layers layers $\implies$
fri-sampled-layers-cover-fold-indices roots query-idxs $\implies$
challenges $\in B$

and *challenges-notin*: *challenges* $\notin B$

shows

```

card
  (query-index-list-entries
    (generic-fri-sampled-query-query-fiber low-degree candidate-bad
      degree-bound challenges))
≤
card
  (query-index-list-entries
    (generic-fri-sampled-query-missing-full-cover-fiber low-degree
      candidate-bad degree-bound challenges))
proof –
have subset:
  generic-fri-sampled-query-query-fiber low-degree candidate-bad
    degree-bound challenges ⊆
  generic-fri-sampled-query-missing-full-cover-fiber low-degree
    candidate-bad degree-bound challenges
by (rule generic-fri-sampled-query-query-fiber-subset-missing-full-cover
  [OF full-cover-imp challenges-notin])
have entries-subset:
  query-index-list-entries
    (generic-fri-sampled-query-query-fiber low-degree candidate-bad
      degree-bound challenges)
    ⊆
  query-index-list-entries
    (generic-fri-sampled-query-missing-full-cover-fiber low-degree
      candidate-bad degree-bound challenges)
using subset unfolding query-index-list-entries-def by blast
show ?thesis
by (rule card-mono[OF - entries-subset])
  (rule finite-subset[OF - finite-query-sample-space],
    unfold query-index-list-entries-def
      generic-fri-sampled-query-missing-full-cover-fiber-def
      fri-query-index-list-space-def, auto)
qed

lemma generic-fri-sampled-query-outside-query-cover-subset:
  generic-fri-sampled-query-outside-query-cover low-degree candidate-bad
    degree-bound B ⊆ fri-query-index-list-space
unfolding generic-fri-sampled-query-outside-query-cover-def
by (auto intro: generic-fri-sampled-query-query-fiber-subset[THEN subsetD])

lemma finite-generic-fri-sampled-query-outside-query-cover:
  finite
    (generic-fri-sampled-query-outside-query-cover low-degree candidate-bad
      degree-bound B)
using generic-fri-sampled-query-outside-query-cover-subset
by (rule finite-subset) (rule finite-fri-query-index-list-space)

lemma generic-fri-sampled-query-outside-query-cover-card-le-from-fibers:
  card

```

$(\text{generic-fri-sampled-query-outside-query-cover low-degree candidate-bad}$
 $\text{degree-bound } B)$
 $\leq$
 $(\sum_{\text{card}} \text{challenges} \in$
 $\text{fri-challenge-space (fri-round-count-for-degree-bound degree-bound)} - B.$
 card
 $(\text{generic-fri-sampled-query-query-fiber low-degree candidate-bad}$
 $\text{degree-bound challenges}))$
unfolding $\text{generic-fri-sampled-query-outside-query-cover-def}$
by $(\text{rule card-UN-le}) \text{ simp}$

lemma $\text{generic-fri-sampled-query-outside-query-cover-entries-card-le-from-fibers:}$
 card

$(\text{query-index-list-entries}$
 $(\text{generic-fri-sampled-query-outside-query-cover low-degree candidate-bad}$
 $\text{degree-bound } B))$
 $\leq$
 $(\sum_{\text{card}} \text{challenges} \in$
 $\text{fri-challenge-space (fri-round-count-for-degree-bound degree-bound)} - B.$
 card
 $(\text{generic-fri-sampled-query-query-fiber low-degree candidate-bad}$
 $\text{degree-bound challenges})) * \text{rounds}$

proof –

let $?Q =$
 $\text{generic-fri-sampled-query-outside-query-cover low-degree candidate-bad}$
 $\text{degree-bound } B$
let $?K =$
 $(\sum_{\text{card}} \text{challenges} \in$
 $\text{fri-challenge-space (fri-round-count-for-degree-bound degree-bound)} - B.$
 card
 $(\text{generic-fri-sampled-query-query-fiber low-degree candidate-bad}$
 $\text{degree-bound challenges}))$
have $\text{entries-le: card (query-index-list-entries ?Q) } \leq \text{card ?Q} * \text{rounds}$
by $(\text{rule query-index-list-entries-card-le})$
 $(\text{rule finite-generic-fri-sampled-query-outside-query-cover},$
 $\text{rule generic-fri-sampled-query-outside-query-cover-subset})$
have $\text{cover-le: card ?Q } \leq ?K$
by $(\text{rule generic-fri-sampled-query-outside-query-cover-card-le-from-fibers})$
have $\text{card ?Q} * \text{rounds } \leq ?K * \text{rounds}$
by $(\text{rule mult-right-mono[OF cover-le]}) \text{ simp}$
then show $?thesis$
using $\text{entries-le by linarith}$
qed

lemma $\text{trace-fri-sampled-query-outside-projection-subset:}$

fst '
 $((\text{trace-fri-sampled-query-bad-pair-union} \cap$
 $(\text{fri-query-index-list-space} \times \text{UNIV})) \cap$
 $(\text{UNIV} \times (- B)))$

$\subseteq$
generic-fri-sampled-query-outside-query-cover trace-table-low-degree
 $(\text{Not} \circ \text{trace-table-low-degree}) (\text{clength} - 1) B$
unfolding *trace-fri-sampled-query-bad-pair-union-def*
generic-fri-sampled-query-outside-query-cover-def
generic-fri-sampled-query-query-fiber-def
generic-fri-sampled-query-restricted-bad-pairs-def
fri-query-challenge-pair-query-fiber-def
using *generic-fri-sampled-query-bad-pair-union-subset-challenge-space*
 $[\text{of trace-table-low-degree Not} \circ \text{trace-table-low-degree clength} - 1]$
by force

definition *composition-fri-sampled-query-outside-query-cover*
 $:: ('f \Rightarrow 'f \text{ list set}) \Rightarrow \text{nat list set}$

where
composition-fri-sampled-query-outside-query-cover B =
 $(\bigcup dg \in (UNIV :: 'f \text{ set}).$
generic-fri-sampled-query-outside-query-cover
 $(\text{composition-table-low-degree } (\text{to-nat } dg))$
 $(\text{Not} \circ \text{composition-table-low-degree maxDegree})$
 $(\text{to-nat } dg) (B \ dg))$

lemma *composition-fri-sampled-query-outside-query-cover-subset:*
composition-fri-sampled-query-outside-query-cover B
 $\subseteq \text{fri-query-index-list-space}$
unfolding *composition-fri-sampled-query-outside-query-cover-def*
by (*auto intro:*
generic-fri-sampled-query-outside-query-cover-subset[THEN subsetD])

lemma *finite-composition-fri-sampled-query-outside-query-cover:*
finite (composition-fri-sampled-query-outside-query-cover B)
using *composition-fri-sampled-query-outside-query-cover-subset*
by (*rule finite-subset*) (*rule finite-fri-query-index-list-space*)

lemma *composition-fri-sampled-query-outside-query-cover-card-le-from-fibers:*
card (composition-fri-sampled-query-outside-query-cover B)
 $\leq$
 $(\sum dg \in (UNIV :: 'f \text{ set}).$
 $\sum \text{challenges} \in$
 $\text{fri-challenge-space } (\text{fri-round-count-for-degree-bound } (\text{to-nat } dg)) -$
 $B \ dg.$
card
 $(\text{generic-fri-sampled-query-query-fiber}$
 $(\text{composition-table-low-degree } (\text{to-nat } dg))$
 $(\text{Not} \circ \text{composition-table-low-degree maxDegree})$
 $(\text{to-nat } dg) \text{ challenges}))$

proof –
let $?Q = \text{composition-fri-sampled-query-outside-query-cover } B$
let $?G = \lambda dg.$

```

generic-fri-sampled-query-outside-query-cover
  (composition-table-low-degree (to-nat dg))
  (Not ◦ composition-table-low-degree maxDegree)
  (to-nat dg) (B dg)
let ?K = λdg.
  (∑ challenges ∈
    fri-challenge-space (fri-round-count-for-degree-bound (to-nat dg)) -
    B dg.
    card
      (generic-fri-sampled-query-query-fiber
        (composition-table-low-degree (to-nat dg))
        (Not ◦ composition-table-low-degree maxDegree)
        (to-nat dg) challenges))
have card ?Q ≤ (∑ dg ∈ (UNIV :: 'f set). card (?G dg))
  unfolding composition-fri-sampled-query-outside-query-cover-def
  by (rule card-UN-le) simp
also have ... ≤ (∑ dg ∈ (UNIV :: 'f set). ?K dg)
  by (rule sum-mono)
    (rule generic-fri-sampled-query-outside-query-cover-card-le-from-fibers)
finally show ?thesis .
qed

lemma composition-fri-sampled-query-outside-query-cover-entries-card-le-from-fibers:
  card
    (query-index-list-entries
      (composition-fri-sampled-query-outside-query-cover B))
  ≤
    (∑ dg ∈ (UNIV :: 'f set).
      ∑ challenges ∈
        fri-challenge-space (fri-round-count-for-degree-bound (to-nat dg)) -
        B dg.
        card
          (generic-fri-sampled-query-query-fiber
            (composition-table-low-degree (to-nat dg))
            (Not ◦ composition-table-low-degree maxDegree)
            (to-nat dg) challenges)) * rounds
proof -
let ?Q = composition-fri-sampled-query-outside-query-cover B
let ?K =
  (∑ dg ∈ (UNIV :: 'f set).
    ∑ challenges ∈
      fri-challenge-space (fri-round-count-for-degree-bound (to-nat dg)) -
      B dg.
      card
        (generic-fri-sampled-query-query-fiber
          (composition-table-low-degree (to-nat dg))
          (Not ◦ composition-table-low-degree maxDegree)
          (to-nat dg) challenges))
have entries-le: card (query-index-list-entries ?Q) ≤ card ?Q * rounds

```

```

by (rule query-index-list-entries-card-le)
  (rule finite-composition-fri-sampled-query-outside-query-cover,
   rule composition-fri-sampled-query-outside-query-cover-subset)
have cover-le: card ?Q ≤ ?K
  by (rule composition-fri-sampled-query-outside-query-cover-card-le-from-fibers)
have card ?Q * rounds ≤ ?K * rounds
  by (rule mult-right-mono[OF cover-le]) simp
then show ?thesis
  using entries-le by linarith
qed

```

lemma *composition-fri-sampled-query-outside-projection-subset:*

```

fst ‘
  ((composition-fri-sampled-query-bad-pair-union dg ∩
    (fri-query-index-list-space × UNIV)) ∩
   (UNIV × (− B dg)))
  ⊆ composition-fri-sampled-query-outside-query-cover B
unfolding composition-fri-sampled-query-bad-pair-union-def
  composition-fri-sampled-query-outside-query-cover-def
  generic-fri-sampled-query-outside-query-cover-def
  generic-fri-sampled-query-query-fiber-def
  generic-fri-sampled-query-restricted-bad-pairs-def
  fri-query-challenge-pair-query-fiber-def
using generic-fri-sampled-query-bad-pair-union-subset-challenge-space
  [of composition-table-low-degree (to-nat dg)
   Not ∘ composition-table-low-degree maxDegree to-nat dg]
by force

```

lemma *checked-staged-security-trace-fri-sampled-query-bad-candidate-bound-from-generic-challenge-cover-and-outsides*

```

assumes wf: staged-budget-wellformed budgets
and controlled: staged-adversary-controlled budgets A
and finite-B: finite B
and fresh-bound:
  wp-event (checked-staged-security-experiment-with-data-state A)
  (staged-security-with-data-state-verifier-event
   (λs. trace-fri-challenge-list-fresh-hit s B))
  adversary-initial-state ≤ F
shows
  wp-event (checked-staged-security-experiment-with-data-state A)
  (staged-security-with-data-state-verifier-event
   trace-fri-sampled-query-bad-candidate)
  adversary-initial-state ≤
  F +
  hash-relation-budget-value (card B * ceil-log clength)
  (staged-attacker-query-budget budgets + staged-challenge-query-budget) +
  staged-phase-target-error
  (query-index-raw-preimage
   (query-index-list-entries
    (generic-fri-sampled-query-outside-query-cover

```

$\text{trace-table-low-degree } (\text{Not } \circ \text{ trace-table-low-degree})$
 $(\text{clength} - 1) B)))$
 $(\text{staged-attacker-query-budget budgets} +$
 $\text{staged-challenge-query-budget})$
by (rule
 $\text{checked-staged-security-trace-fri-sampled-query-bad-candidate-bound-from-challenge-cover-and-query-projection}$
 $[\text{OF wf controlled finite-B fresh-bound}$
 $\text{trace-fri-sampled-query-outside-projection-subset}])$

lemma $\text{checked-staged-security-trace-fri-sampled-query-bad-candidate-bound-from-generic-challenge-cover-and-outsid}$

assumes $\text{wf: staged-budget-wellformed budgets}$

and $\text{controlled: staged-adversary-controlled budgets } A$

and $\text{finite-B: finite } B$

and fresh-bound:

$\text{wp-event } (\text{checked-staged-security-experiment-with-data-state } A)$

$(\text{staged-security-with-data-state-verifier-event}$

$(\lambda s. \text{trace-fri-challenge-list-fresh-hit } s B))$

$\text{adversary-initial-state} \leq F$

and query-fraction:

nnreal

$(\text{card}$

$(\text{query-index-list-entries}$

$(\text{generic-fri-sampled-query-outside-query-cover}$

$\text{trace-table-low-degree } (\text{Not } \circ \text{ trace-table-low-degree})$

$(\text{clength} - 1) B))) /$

$\text{nnreal } (\text{card } \text{query-sample-space}) \leq Q$

shows

$\text{wp-event } (\text{checked-staged-security-experiment-with-data-state } A)$

$(\text{staged-security-with-data-state-verifier-event}$

$\text{trace-fri-sampled-query-bad-candidate})$

$\text{adversary-initial-state} \leq$

$F +$

$\text{hash-relation-budget-value } (\text{card } B * \text{ceil-log } \text{clength})$

$(\text{staged-attacker-query-budget budgets} + \text{staged-challenge-query-budget}) +$

$\text{nnreal } (\text{staged-attacker-query-budget budgets} +$

$\text{staged-challenge-query-budget}) * Q$

proof –

let $?Qset =$

$\text{generic-fri-sampled-query-outside-query-cover } \text{trace-table-low-degree}$

$(\text{Not } \circ \text{ trace-table-low-degree}) (\text{clength} - 1) B$

have base:

$\text{wp-event } (\text{checked-staged-security-experiment-with-data-state } A)$

$(\text{staged-security-with-data-state-verifier-event}$

$\text{trace-fri-sampled-query-bad-candidate})$

$\text{adversary-initial-state} \leq$

$F +$

$\text{hash-relation-budget-value } (\text{card } B * \text{ceil-log } \text{clength})$

$(\text{staged-attacker-query-budget budgets} + \text{staged-challenge-query-budget}) +$

$\text{staged-phase-target-error}$


```

    (query-index-row-preimage (query-index-list-entries ?Qset))
    (staged-attacker-query-budget budgets +
     staged-challenge-query-budget)
  by (rule
    checked-staged-security-trace-fri-sampled-query-bad-candidate-bound-from-generic-challenge-cover-and-outside-fil
    [OF wf controlled finite-B fresh-bound])
  have subset:
    query-index-list-entries ?Qset  $\subseteq$  query-sample-space
  by (rule query-index-list-entries-subset-query-sample-space)
    (rule generic-fri-sampled-query-outside-query-cover-subset)
  have target-bound:
    staged-phase-target-error
      (query-index-row-preimage (query-index-list-entries ?Qset))
      (staged-attacker-query-budget budgets +
       staged-challenge-query-budget)
     $\leq$ 
    nnreal (staged-attacker-query-budget budgets +
      staged-challenge-query-budget) * Q
  by (rule staged-phase-query-index-target-error-query-bound
    [OF query-index-row-preimage-bound-from-sampler-wellformed subset
     query-fraction])
  show ?thesis
    by (rule order-trans[OF base])
      (use target-bound in simp)
qed

```

lemma *checked-staged-security-trace-fri-sampled-query-bad-candidate-bound-from-generic-challenge-cover-and-outside-fil*

assumes wf: staged-budget-wellformed budgets
and controlled: staged-adversary-controlled budgets A
and finite-B: finite B
and fresh-bound:
 wp-event (checked-staged-security-experiment-with-data-state A)
 (staged-security-with-data-state-verifier-event
 ($\lambda s.$ trace-fri-challenge-list-fresh-hit s B))
 adversary-initial-state $\leq F$
and query-fraction:
 nnreal
 (($\sum$ challenges $\in$
 fri-challenge-space (fri-round-count-for-degree-bound (clength - 1)) -
 B.
 card
 (generic-fri-sampled-query-query-fiber trace-table-low-degree
 (Not $\circ$ trace-table-low-degree) (clength - 1) challenges)) *
 rounds) /
 nnreal (card query-sample-space) $\leq Q$
shows
 wp-event (checked-staged-security-experiment-with-data-state A)
 (staged-security-with-data-state-verifier-event
 trace-fri-sampled-query-bad-candidate)

$adversary-initial-state \leq$
 $F +$
 $hash-relation-budget-value (card\ B * ceil-log\ clength)$
 $(staged-attacker-query-budget\ budgets + staged-challenge-query-budget) +$
 $nnreal (staged-attacker-query-budget\ budgets +$
 $staged-challenge-query-budget) * Q$
proof (rule
 $checked-staged-security-trace-fri-sampled-query-bad-candidate-bound-from-generic-challenge-cover-and-outside-fiber$
 $[OF\ wf\ controlled\ finite-B\ fresh-bound])$
let $?Entries =$
 $query-index-list-entries$
 $(generic-fri-sampled-query-outside-query-cover\ trace-table-low-degree$
 $(Not \circ trace-table-low-degree) (clength - 1) B)$
let $?K =$
 $(\sum challenges \in$
 $fri-challenge-space (fri-round-count-for-degree-bound (clength - 1)) -$
 $B.$
 $card$
 $(generic-fri-sampled-query-query-fiber\ trace-table-low-degree$
 $(Not \circ trace-table-low-degree) (clength - 1) challenges)) *$
 $rounds$
have $card-le: card\ ?Entries \leq ?K$
by (rule
 $generic-fri-sampled-query-outside-query-cover-entries-card-le-from-fibers)$
have $nnreal (card\ ?Entries) / nnreal (card\ query-sample-space)$
 $\leq nnreal\ ?K / nnreal (card\ query-sample-space)$
using $card-le$ **by** (rule $nnreal-nat-divide-right-mono$)
also have $\dots \leq Q$
by (rule $query-fraction$)
finally show
 $nnreal (card\ ?Entries) / nnreal (card\ query-sample-space) \leq Q .$
qed

lemma $checked-staged-security-composition-fri-sampled-query-bad-candidate-bound-from-generic-challenge-cover-and$
assumes $wf: staged-budget-wellformed\ budgets$
and $controlled: staged-adversary-controlled\ budgets\ A$
and $finite-B: \bigwedge dg. finite\ (B\ dg)$
and $fresh-bound:$
 $wp-event (checked-staged-security-experiment-with-data-state\ A)$
 $(staged-security-with-data-state-verifier-event$
 $(\lambda s. composition-fri-challenge-list-fresh-hit\ s\ B))$
 $adversary-initial-state \leq F$
shows
 $wp-event (checked-staged-security-experiment-with-data-state\ A)$
 $(staged-security-with-data-state-verifier-event$
 $composition-fri-sampled-query-bad-candidate)$
 $adversary-initial-state \leq$
 $F +$
 $hash-relation-budget-value$

$(\sum dg \in (UNIV :: 'f \text{ set}).$
 $\text{card } (B \text{ dg}) * \text{ceil-log } (\text{maxDegree} + 1))$
 $(\text{staged-attacker-query-budget budgets} +$
 $\text{staged-challenge-query-budget}) +$
 $\text{staged-phase-target-error}$
 $(\text{query-index-raw-preimage}$
 $(\text{query-index-list-entries}$
 $(\text{composition-fri-sampled-query-outside-query-cover } B)))$
 $(\text{staged-attacker-query-budget budgets} +$
 $\text{staged-challenge-query-budget})$
by (rule
 $\text{checked-staged-security-composition-fri-sampled-query-bad-candidate-bound-from-challenge-cover-and-query-projection}$
 $[OF \text{ wf controlled finite-}B \text{ fresh-bound}$
 $\text{composition-fri-sampled-query-outside-projection-subset}]$)

lemma $\text{checked-staged-security-composition-fri-sampled-query-bad-candidate-bound-from-generic-challenge-cover-and-query-projection}$

assumes $\text{wf: staged-budget-wellformed budgets}$

and $\text{controlled: staged-adversary-controlled budgets } A$

and $\text{finite-}B: \bigwedge dg. \text{finite } (B \text{ dg})$

and fresh-bound:

$\text{wp-event } (\text{checked-staged-security-experiment-with-data-state } A)$

$(\text{staged-security-with-data-state-verifier-event}$

$(\lambda s. \text{composition-fri-challenge-list-fresh-hit } s \text{ } B))$

$\text{adversary-initial-state} \leq F$

and query-fraction:

nnreal

$(\text{card}$

$(\text{query-index-list-entries}$

$(\text{composition-fri-sampled-query-outside-query-cover } B))) /$

$\text{nnreal } (\text{card } \text{query-sample-space}) \leq Q$

shows

$\text{wp-event } (\text{checked-staged-security-experiment-with-data-state } A)$

$(\text{staged-security-with-data-state-verifier-event}$

$\text{composition-fri-sampled-query-bad-candidate})$

$\text{adversary-initial-state} \leq$

$F +$

$\text{hash-relation-budget-value}$

$(\sum dg \in (UNIV :: 'f \text{ set}).$

$\text{card } (B \text{ dg}) * \text{ceil-log } (\text{maxDegree} + 1))$

$(\text{staged-attacker-query-budget budgets} +$

$\text{staged-challenge-query-budget}) +$

$\text{nnreal } (\text{staged-attacker-query-budget budgets} +$

$\text{staged-challenge-query-budget}) * Q$

proof –

let $?Qset = \text{composition-fri-sampled-query-outside-query-cover } B$

have base:

$\text{wp-event } (\text{checked-staged-security-experiment-with-data-state } A)$

$(\text{staged-security-with-data-state-verifier-event}$

$\text{composition-fri-sampled-query-bad-candidate})$

```

adversary-initial-state ≤
F +
hash-relation-budget-value
  (∑ dg ∈ (UNIV :: 'f set).
    card (B dg) * ceil-log (maxDegree + 1))
  (staged-attacker-query-budget budgets +
    staged-challenge-query-budget) +
staged-phase-target-error
  (query-index-row-preimage (query-index-list-entries ?Qset))
  (staged-attacker-query-budget budgets +
    staged-challenge-query-budget)
by (rule
  checked-staged-security-composition-fri-sampled-query-bad-candidate-bound-from-generic-challenge-cover-and-ou
  [OF wf controlled finite-B fresh-bound])
have subset:
  query-index-list-entries ?Qset ⊆ query-sample-space
by (rule query-index-list-entries-subset-query-sample-space)
  (rule composition-fri-sampled-query-outside-query-cover-subset)
have target-bound:
  staged-phase-target-error
    (query-index-row-preimage (query-index-list-entries ?Qset))
    (staged-attacker-query-budget budgets +
      staged-challenge-query-budget)
  ≤
  nnreal (staged-attacker-query-budget budgets +
    staged-challenge-query-budget) * Q
by (rule staged-phase-query-index-target-error-query-bound
  [OF query-index-row-preimage-bound-from-sampler-wellformed subset
    query-fraction])
show ?thesis
by (rule order-trans[OF base])
  (use target-bound in simp)
qed

```

lemma *checked-staged-security-composition-fri-sampled-query-bad-candidate-bound-from-generic-challenge-cover-and*
assumes *wf*: *staged-budget-wellformed budgets*
and *controlled*: *staged-adversary-controlled budgets A*
and *finite-B*: $\bigwedge dg. \text{finite } (B \ dg)$
and *fresh-bound*:
wp-event (*checked-staged-security-experiment-with-data-state A*)
 (*staged-security-with-data-state-verifier-event*
 ($\lambda s. \text{composition-fri-challenge-list-fresh-hit } s \ B$))
adversary-initial-state ≤ *F*
and *query-fraction*:
nnreal
 ((∑ *dg* ∈ (*UNIV* :: 'f set).
 ∑ *challenges* ∈
fri-challenge-space (*fri-round-count-for-degree-bound* (*to-nat dg*)) –
B dg).

```

    card
      (generic-fri-sampled-query-query-fiber
        (composition-table-low-degree (to-nat dg))
        (Not ∘ composition-table-low-degree maxDegree)
        (to-nat dg) challenges)) * rounds) /
    nnreal (card query-sample-space) ≤ Q
  shows
    wp-event (checked-staged-security-experiment-with-data-state A)
      (staged-security-with-data-state-verifier-event
        composition-fri-sampled-query-bad-candidate)
    adversary-initial-state ≤
      F +
      hash-relation-budget-value
        (∑ dg ∈ (UNIV :: 'f set).
          card (B dg) * ceil-log (maxDegree + 1))
        (staged-attacker-query-budget budgets +
          staged-challenge-query-budget) +
      nnreal (staged-attacker-query-budget budgets +
        staged-challenge-query-budget) * Q
  proof (rule
    checked-staged-security-composition-fri-sampled-query-bad-candidate-bound-from-generic-challenge-cover-and-outside
    [OF wf controlled finite-B fresh-bound])
    let ?Entries =
      query-index-list-entries
        (composition-fri-sampled-query-outside-query-cover B)
    let ?K =
      (∑ dg ∈ (UNIV :: 'f set).
        ∑ challenges ∈
          fri-challenge-space (fri-round-count-for-degree-bound (to-nat dg)) -
            B dg.
          card
            (generic-fri-sampled-query-query-fiber
              (composition-table-low-degree (to-nat dg))
              (Not ∘ composition-table-low-degree maxDegree)
              (to-nat dg) challenges)) * rounds)
    have card-le: card ?Entries ≤ ?K
    by (rule
      composition-fri-sampled-query-outside-query-cover-entries-card-le-from-fibers)
    have nnreal (card ?Entries) / nnreal (card query-sample-space)
      ≤ nnreal ?K / nnreal (card query-sample-space)
    using card-le by (rule nnreal-nat-divide-right-mono)
    also have ... ≤ Q
    by (rule query-fraction)
    finally show
      nnreal (card ?Entries) / nnreal (card query-sample-space) ≤ Q .
  qed
end

```

end

theory *Soundness-FRI-Query-List-Prequery*
imports
 Stark-Core.Merkle-Tree
 Soundness-Core-Events
 Soundness-FRI-Query-Exact-Bounds
 Soundness-FRI-Query-Index-Staged-Bounds
begin

Query-list prequery accounting for FRI. The lemmas in this theory relate verifier-generated query index lists to random-oracle budget events used in the staged soundness proof.

context *soundness*
begin

lemma *set-dist-return-valueD*:
 assumes $\text{Some } (y, t) \in \text{set-dist } (\text{execute } (\text{return } x) \ s)$
 shows $y = x$
 using *assms*
 unfolding *set-dist-def return.rep-eq dist-return-def*
 by (*simp add: dist-delta-dist delta-map-def*)

lemma *set-dist-return-stateD*:
 assumes $\text{Some } (y, t) \in \text{set-dist } (\text{execute } (\text{return } x) \ s)$
 shows $t = s$
 using *assms*
 unfolding *set-dist-def return.rep-eq dist-return-def*
 by (*simp add: dist-delta-dist delta-map-def*)

lemma *set-dist-bind-return-consE*:
 assumes *support*:
 $\text{out} \in \text{set-dist } (\text{execute } (m \gg (\lambda xs. \text{return } (u \# xs))) \ s)$
 and *out-eq*: $\text{out} = \text{Some } (ys, t0)$
 obtains *results t where*
 $\text{Some } (\text{results}, t) \in \text{set-dist } (\text{execute } m \ s)$
 $ys = u \# \text{results}$
 $t0 = t$
 $\text{out} = \text{Some } (u \# \text{results}, t)$
proof –
 from *support[unfolded out-eq]*
 obtain *results t where tail*:
 $\text{Some } (\text{results}, t) \in \text{set-dist } (\text{execute } m \ s)$
 and *ret*:
 $\text{Some } (ys, t0) \in \text{set-dist } (\text{execute } (\text{return } (u \# \text{results})) \ t)$
 by (*auto elim!: set-dist-bindE*)
 have *ys-eq*: $ys = u \# \text{results}$
 by (*rule set-dist-return-valueD[OF ret]*)
 have *t0-eq*: $t0 = t$

by (rule set-dist-return-stateD[OF ret])
 have out-eq': out = Some (u # results, t)
 using out-eq ys-eq t0-eq by simp
 show ?thesis
 by (rule that[OF tail ys-eq t0-eq out-eq'])
 qed

lemma set-dist-bind-return-cons-SomeE:
 assumes support:
 Some (ys, t) ∈ set-dist (execute (m ≫ (λxs. return (u # xs))) s)
 obtains xs where
 Some (xs, t) ∈ set-dist (execute m s)
 ys = u # xs
proof –
 from support obtain xs t0 where tail:
 Some (xs, t0) ∈ set-dist (execute m s)
 and ys-eq: ys = u # xs
 and t-eq: t = t0
 by (elim set-dist-bind-return-consE) simp
 have tail-t: Some (xs, t) ∈ set-dist (execute m s)
 by (subst t-eq, rule tail)
 show ?thesis
 by (rule that[OF tail-t ys-eq])
 qed

lemma wp-event-nonzero-imp-exists-support:
 assumes wp-event m P s ≠ 0
 obtains out where out ∈ set-dist (execute m s) and P out
 by (rule wp-event-pos-imp-exists-support)
 (rule zero-less-iff-neq-zero[THEN iffD2], rule assms)

lemma wp-event-false [simp]:
 wp-event m (λ-. False) s = 0
 unfolding wp-event-def wp-def dist-expect-def
 by simp

lemma wp-event-bind-return-cons-mono:
 fixes m :: ('a list, 's) state-monad
 assumes none-imp: Q None ⇒ R None
 and some-imp:
 $\bigwedge xs t. \text{Some } (xs, t) \in \text{set-dist } (\text{execute } m \text{ } s) \implies$
 $Q (\text{Some } (u \# xs, t)) \implies R (\text{Some } (xs, t))$
 shows
 wp-event (m ≫ (λxs. return (u # xs))) Q s ≤
 wp-event m R s
proof (rule wp-event-bind-bound-by-head-event)
 show wp-event m R s ≤ wp-event m R s
 by (rule order.refl)
 next

```

show  $Q \text{ None} \implies R \text{ None}$ 
  by (rule none-imp)
next
  fix  $xs \ t \ out$ 
  assume  $support: \text{Some } (xs, t) \in \text{set-dist } (\text{execute } m \ s)$ 
  and  $ret: out \in \text{set-dist } (\text{execute } (\text{return } (u \ \# \ xs)) \ t)$ 
  and  $q: Q \ out$ 
  have  $out\text{-eq}: out = \text{Some } (u \ \# \ xs, t)$ 
  using  $ret$ 
  unfolding  $\text{set-dist-def return.rep-eq dist-return-def}$ 
   $\text{dist-delta-dist delta-map-def}$ 
  by  $simp$ 
  have  $q\text{-some}: Q \ (\text{Some } (u \ \# \ xs, t))$ 
  using  $q$  by ( $simp \ only: out\text{-eq}$ )
  show  $R \ (\text{Some } (xs, t))$ 
  by (rule some-imp[OF support q-some])
qed

lemma  $eq\text{-Suc-imp-less-nat}$ :
  assumes  $m = \text{Suc } n$ 
  shows  $n < m$ 
  by (subst assms, rule lessI)

lemma  $list\text{-length-SucE}$ :
  assumes  $\text{length } xs = \text{Suc } n$ 
  obtains  $x \ xs' \ \text{where } xs = x \ \# \ xs'$ 
  using  $assms$  by (cases  $xs$ )  $simp\text{-all}$ 

lemma  $state\text{-after-query-chunks-cons-shift}$ :
  assumes  $s1 = \text{state-after-query-chunks } s \ (\text{chunk } \# \ \text{chunks}) \ 1$ 
  shows
     $\text{state-after-query-chunks } s1 \ \text{chunks } n =$ 
     $\text{state-after-query-chunks } s \ (\text{chunk } \# \ \text{chunks}) \ (\text{Suc } n)$ 
  using  $assms$  unfolding  $\text{state-after-query-chunks-def}$  by  $simp$ 

lemma  $state\text{-after-query-chunks-cons-shift-index}$ :
  assumes  $s1 = \text{state-after-query-chunks } s \ (\text{chunk } \# \ \text{chunks}) \ 1$ 
  shows
     $\text{state-after-query-chunks } s1 \ \text{chunks } i =$ 
     $\text{state-after-query-chunks } s \ (\text{chunk } \# \ \text{chunks}) \ (\text{Suc } i)$ 
  using  $assms$  unfolding  $\text{state-after-query-chunks-def}$  by  $simp$ 

lemma  $state\text{-after-query-chunks-one-cons}$ :
   $\text{state-after-query-chunks } s \ (\text{chunk } \# \ \text{chunks}) \ 1 = \text{foldl concat } s \ \text{chunk}$ 
  unfolding  $\text{state-after-query-chunks-def}$  by  $simp$ 

end

```

Query-list prequery accounting for sampled FRI.

The exact verifier-local sampled-query bounds are list-valued: they bound the probability of drawing a whole query-index list from a target set. In the staged adversary experiment the verifier reuses the adversary's oracle map, so an exact staged theorem needs a fresh/prequeried split for the whole query-index list. This layer provides the conservative relation-budget side of that split without changing the oracle model.

context *soundness*
begin

lemma *verifier-query-round-program-alt-outcomeE*:

assumes *outcome*:
Some $(((), t) \in$
set-dist
(execute
(verifier-query-round-program fr f-fl f-final as fl final) s)
obtains *raw s0* **where**
Some $(raw, s0) \in$ *set-dist (execute receive-query-index-challenge s)*
Some $(((), t) \in$
set-dist
(execute
(verifier-query-round-after-index-program fr f-fl f-final as fl
final raw) s0)
using *outcome[unfolded verifier-query-round-program-alt-def]*
by *(auto elim!: set-dist-bindE)*

lemma *verifier-query-round-parts-counter*:

assumes *recv*:
Some $(raw, s0) \in$ *set-dist (execute receive-query-index-challenge s)*
and *after*:
Some $(((), t) \in$
set-dist
(execute
(verifier-query-round-after-index-program fr f-fl f-final as fl
final raw) s0)
shows *PQueryCounter t = Suc (PQueryCounter s)*
proof –
have *counter-s0*: *PQueryCounter s0 = Suc (PQueryCounter s)*
using *receive-query-index-challenge-counter-outcome[OF recv]* **by** *simp*
have *counter-t-s0*: *PQueryCounter t = PQueryCounter s0*
by *(rule conjunct2[OF verifier-query-round-after-index-program-outcome*
[OF after]])
show *?thesis*
proof *(rule HOL.trans)*
show *PQueryCounter t = PQueryCounter s0*
by *(rule counter-t-s0)*
show *PQueryCounter s0 = Suc (PQueryCounter s)*
by *(rule counter-s0)*
qed
qed

definition *query-index-raw-list-preimage* :: *nat list set* $\Rightarrow$ *'f list set*
where

query-index-raw-list-preimage *Q* =
 $\{ \text{rows}. \text{length } \text{rows} = \text{rounds} \wedge$
 $\text{map } (\lambda \text{raw}. \text{index } (\text{to-nat } \text{raw})) \text{ rows} \in Q \}$

definition *query-index-raw-list-position-values*
:: *nat list set* $\Rightarrow$ *nat* $\Rightarrow$ *'f set*
where

query-index-raw-list-position-values *Q i* =
 $\{ \text{raw}. \exists \text{rows} \in \text{query-index-raw-list-preimage } Q.$
 $i < \text{rounds} \wedge \text{raw} = \text{rows} ! i \}$

definition *query-index-raw-list-relation-fiber-bound*
:: *nat list set* $\Rightarrow$ *nat*
where

query-index-raw-list-relation-fiber-bound *Q* =
 $(\sum i < \text{rounds}. \text{card } (\text{query-index-raw-list-position-values } Q i))$

definition *staged-transcript-query-index-list-set-hit*
:: *nat list set* $\Rightarrow$
('f staged-proof-data $\times$ *'f protocol-channel)* *option* $\Rightarrow$ *bool*
where

staged-transcript-query-index-list-set-hit *Q out* $\longleftrightarrow$
(case out of
 $\text{None} \Rightarrow \text{False}$
| $\text{Some } (\text{data}, \text{attacker-state}) \Rightarrow$
 $(\exists \text{raw-idxs } \text{query-idxs}.$
 $\text{length } \text{raw-idxs} = \text{rounds} \wedge$
 $\text{query-idxs} \in Q \wedge$
 $\text{query-idxs} = \text{map } (\lambda \text{raw}. \text{index } (\text{to-nat } \text{raw})) \text{ raw-idxs} \wedge$
 $(\forall i < \text{rounds}.$
 $\text{fmlookup } (\text{HashMap } \text{attacker-state})$
 $(\text{QueryIndexChallenge } i$
 $(\text{state-after-query-chunks}$
 $(\text{staged-query-start-hash } \text{data})$
 $(\text{staged-query-chunks } \text{data}) i)) =$
 $\text{Some } (\text{raw-idxs} ! i))))$

definition *staged-security-with-data-query-index-list-set-hit*
:: *nat list set* $\Rightarrow$
 $((('f \text{ staged-proof-data} \times 'f \text{ protocol-channel}) \times$
 $\text{unit list}) \times 'f \text{ protocol-channel}) \text{ option} \Rightarrow \text{bool}$
where

staged-security-with-data-query-index-list-set-hit *Q out* $\longleftrightarrow$
(case out of
 $\text{None} \Rightarrow \text{False}$
| $\text{Some } (((\text{data}, -), \text{result}), \text{final-state}) \Rightarrow$

$(\exists \text{raw-idxs query-idxs.}$
 $\text{length raw-idxs} = \text{rounds} \wedge$
 $\text{query-idxs} \in Q \wedge$
 $\text{query-idxs} = \text{map } (\lambda \text{raw. index (to-nat raw)}) \text{ raw-idxs} \wedge$
 $(\forall i < \text{rounds.}$
 $\text{fmlookup (HashMap final-state)}$
 $\text{(QueryIndexChallenge i}$
 $\text{(state-after-query-chunks}$
 $\text{(staged-query-start-hash data)}$
 $\text{(staged-query-chunks data) i})) =$
 $\text{Some (raw-idxs ! i))})$

definition *staged-security-with-data-state-query-index-list-path-fresh*

$:: \text{nat list set} \Rightarrow$
 $((('f \text{ staged-proof-data} \times 'f \text{ protocol-channel}) \times$
 $\text{unit list}) \times 'f \text{ protocol-channel}) \text{ option} \Rightarrow \text{bool}$

where

$\text{staged-security-with-data-state-query-index-list-path-fresh } Q \text{ out} \longleftrightarrow$
 $(\text{case out of}$
 $\text{None} \Rightarrow \text{False}$
 $| \text{Some } (((\text{data}, \text{attacker-state}), \text{result}), \text{final-state}) \Rightarrow$
 $(\exists \text{raw-idxs query-idxs.}$
 $\text{length raw-idxs} = \text{rounds} \wedge$
 $\text{query-idxs} \in Q \wedge$
 $\text{query-idxs} = \text{map } (\lambda \text{raw. index (to-nat raw)}) \text{ raw-idxs} \wedge$
 $(\forall i < \text{rounds.}$
 $\text{fmlookup (HashMap final-state)}$
 $\text{(QueryIndexChallenge i}$
 $\text{(state-after-query-chunks}$
 $\text{(staged-query-start-hash data)}$
 $\text{(staged-query-chunks data) i})) =$
 $\text{Some (raw-idxs ! i)} \wedge$
 fmlookup
 (HashMap
 $\text{(verifier-state-from-adversary attacker-state}$
 $\text{(staged-proof-transcript data))})$
 $\text{(QueryIndexChallenge i}$
 $\text{(state-after-query-chunks}$
 $\text{(staged-query-start-hash data)}$
 $\text{(staged-query-chunks data) i})) =$
 $\text{None}))$

definition *checked-staged-query-index-list-relation*

$:: 'f \text{ staged-adversary} \Rightarrow \text{nat list set} \Rightarrow$
 $'f \text{ protocol-hash-input} \Rightarrow 'f \Rightarrow \text{bool}$

where

$\text{checked-staged-query-index-list-relation } A \text{ } Q \text{ } x \text{ } y \longleftrightarrow$
 $(\exists \text{data attacker-state raw-idxs i.}$
 $\text{Some (data, attacker-state)} \in$

```

    set-dist
      (execute (checked-staged-transcript-program A)
        adversary-initial-state) ∧
    raw-idxs ∈ query-index-raw-list-preimage Q ∧
    i < rounds ∧
    x = QueryIndexChallenge i
      (state-after-query-chunks
        (staged-query-start-hash data)
        (staged-query-chunks data) i) ∧
    y = raw-idxs ! i)

```

definition *staged-security-with-data-state-query-index-list-prequery-hit*

```

:: 'f staged-adversary ⇒ nat list set ⇒
  (((('f staged-proof-data × 'f protocol-channel) ×
    unit list) × 'f protocol-channel) option ⇒ bool

```

where

```

staged-security-with-data-state-query-index-list-prequery-hit A Q out
  ⟷
  (case out of
    None ⇒ False
  | Some (((data, attacker-state), result), final-state) ⇒
    hash-relation-hit
      (checked-staged-query-index-list-relation A Q)
      adversary-initial-state attacker-state)

```

definition *query-index-list-path-fresh*

```

:: ('f, 'a) protocol-channel-scheme ⇒ nat ⇒ 'f list list ⇒
  'f list ⇒ bool

```

where

```

query-index-list-path-fresh s n chunks raws ⟷
  length raws = n ∧
  length chunks = n ∧
  (∀ i < n.
    fmllookup (HashMap s)
      (QueryIndexChallenge (PQueryCounter s + i)
        (state-after-query-chunks (PState s) chunks i)) = None)

```

lemma *query-index-list-path-fresh-head-fresh:*

assumes

```

  query-index-list-path-fresh s (Suc n) chunks (raw # raws)

```

shows

```

  fmllookup (HashMap s)
    (QueryIndexChallenge (PQueryCounter s) (PState s)) = None

```

using *assms* **unfolding** *query-index-list-path-fresh-def*

```

  state-after-query-chunks-def

```

by *auto*

lemma *query-index-list-path-fresh-lookupD:*

assumes *fresh:* *query-index-list-path-fresh s n chunks raws*

and *i-bound*: $i < n$
shows
 $\text{fmlookup } (\text{HashMap } s)$
 $(\text{QueryIndexChallenge } (P\text{QueryCounter } s + i)$
 $(\text{state-after-query-chunks } (P\text{State } s) \text{ chunks } i)) = \text{None}$
proof –
have *all*:
 $\forall j < n.$
 $\text{fmlookup } (\text{HashMap } s)$
 $(\text{QueryIndexChallenge } (P\text{QueryCounter } s + j)$
 $(\text{state-after-query-chunks } (P\text{State } s) \text{ chunks } j)) = \text{None}$
using *fresh unfolding query-index-list-path-fresh-def*
by (*elim conjE*)
show *?thesis*
by (*rule all[rule-format, OF i-bound]*)
qed

definition *query-rounds-raw-list-path-hit*
 $:: (f, 'a) \text{ protocol-channel-scheme} \Rightarrow 'f \text{ list} \Rightarrow \text{nat} \Rightarrow$
 $(\text{unit list} \times (f, 'a) \text{ protocol-channel-scheme}) \text{ option} \Rightarrow \text{bool}$
where
 $\text{query-rounds-raw-list-path-hit } s \text{ rows } n \text{ out} \longleftrightarrow$
 $(\text{case out of}$
 $\text{None} \Rightarrow \text{False}$
 $| \text{Some } (\text{results}, t) \Rightarrow$
 $(\exists \text{chunks.}$
 $\text{length rows} = n \wedge$
 $\text{length chunks} = n \wedge$
 $P\text{State } t = \text{state-after-query-chunks } (P\text{State } s) \text{ chunks } n \wedge$
 $\text{query-index-list-path-fresh } s \text{ } n \text{ chunks rows} \wedge$
 $(\forall i < n.$
 $\text{fmlookup } (\text{HashMap } t)$
 $(\text{QueryIndexChallenge } (P\text{QueryCounter } s + i)$
 $(\text{state-after-query-chunks } (P\text{State } s) \text{ chunks } i)) =$
 $\text{Some } (\text{rows } ! i)))$

lemma *query-rounds-raw-list-path-hitI*:
assumes *len-rows*: $\text{length rows} = n$
and *len-chunks*: $\text{length chunks} = n$
and *state*:
 $P\text{State } t = \text{state-after-query-chunks } (P\text{State } s) \text{ chunks } n$
and *fresh*: $\text{query-index-list-path-fresh } s \text{ } n \text{ chunks rows}$
and *lookup*:
 $\bigwedge i. i < n \implies$
 $\text{fmlookup } (\text{HashMap } t)$
 $(\text{QueryIndexChallenge } (P\text{QueryCounter } s + i)$
 $(\text{state-after-query-chunks } (P\text{State } s) \text{ chunks } i)) =$
 $\text{Some } (\text{rows } ! i)$
shows $\text{query-rounds-raw-list-path-hit } s \text{ rows } n (\text{Some } (\text{results}, t))$

proof –
have *rhs-eq*:
 $query_rounds_raw_list_path_hit\ s\ raws\ n\ (Some\ (results,\ t)) =$
 $(\exists\ chunks.$
 $\quad length\ raws = n \wedge$
 $\quad length\ chunks = n \wedge$
 $\quad PState\ t = state_after_query_chunks\ (PState\ s)\ chunks\ n \wedge$
 $\quad query_index_list_path_fresh\ s\ n\ chunks\ raws \wedge$
 $\quad (\forall\ i < n.$
 $\quad \quad fmlookup\ (HashMap\ t)$
 $\quad \quad (QueryIndexChallenge\ (PQueryCounter\ s + i)$
 $\quad \quad \quad (state_after_query_chunks\ (PState\ s)\ chunks\ i)) =$
 $\quad \quad Some\ (raws\ !\ i)))$
unfolding *query-rounds-raw-list-path-hit-def* **by** *simp*
show *?thesis*
by (*subst rhs-eq*, *rule exI[of - chunks]*,
intro conjI allI impI,
rule len-raws, *rule len-chunks*, *rule state*,
rule fresh, *rule lookup*)
qed

lemma *query-rounds-raw-list-path-hit-cons-SomeE*:
assumes
 $query_rounds_raw_list_path_hit\ s\ (raw\ \# \ raws)\ (Suc\ n)$
 $(Some\ (ys,\ t))$
obtains *chunks* **where**
 $length\ raws = n$
 $length\ chunks = Suc\ n$
 $PState\ t = state_after_query_chunks\ (PState\ s)\ chunks\ (Suc\ n)$
 $query_index_list_path_fresh\ s\ (Suc\ n)\ chunks\ (raw\ \# \ raws)$
 $\bigwedge i. i < Suc\ n \implies$
 $\quad fmlookup\ (HashMap\ t)$
 $\quad (QueryIndexChallenge\ (PQueryCounter\ s + i)$
 $\quad \quad (state_after_query_chunks\ (PState\ s)\ chunks\ i)) =$
 $\quad Some\ ((raw\ \# \ raws)\ !\ i)$

proof –
have *hit-eq*:
 $query_rounds_raw_list_path_hit\ s\ (raw\ \# \ raws)\ (Suc\ n)$
 $(Some\ (ys,\ t)) =$
 $(\exists\ chunks.$
 $\quad length\ (raw\ \# \ raws) = Suc\ n \wedge$
 $\quad length\ chunks = Suc\ n \wedge$
 $\quad PState\ t = state_after_query_chunks\ (PState\ s)\ chunks\ (Suc\ n) \wedge$
 $\quad query_index_list_path_fresh\ s\ (Suc\ n)\ chunks\ (raw\ \# \ raws) \wedge$
 $\quad (\forall\ i < Suc\ n.$
 $\quad \quad fmlookup\ (HashMap\ t)$
 $\quad \quad (QueryIndexChallenge\ (PQueryCounter\ s + i)$
 $\quad \quad \quad (state_after_query_chunks\ (PState\ s)\ chunks\ i)) =$
 $\quad \quad Some\ ((raw\ \# \ raws)\ !\ i)))$

```

unfolding query-rounds-raw-list-path-hit-def by simp
have ex-chunks:
   $\exists$  chunks.
    length (raw # rows) = Suc n  $\wedge$ 
    length chunks = Suc n  $\wedge$ 
    PState t = state-after-query-chunks (PState s) chunks (Suc n)  $\wedge$ 
    query-index-list-path-fresh s (Suc n) chunks (raw # rows)  $\wedge$ 
    ( $\forall i < \text{Suc } n.$ 
      fmllookup (HashMap t)
        (QueryIndexChallenge (PQueryCounter s + i)
          (state-after-query-chunks (PState s) chunks i)) =
        Some ((raw # rows) ! i))
    using assms by (simp only: hit-eq)
then obtain chunks where len-rows-full: length (raw # rows) = Suc n
and len-chunks: length chunks = Suc n
and state: PState t = state-after-query-chunks (PState s) chunks (Suc n)
and fresh: query-index-list-path-fresh s (Suc n) chunks (raw # rows)
and lookup-all:
   $\forall i < \text{Suc } n.$ 
    fmllookup (HashMap t)
      (QueryIndexChallenge (PQueryCounter s + i)
        (state-after-query-chunks (PState s) chunks i)) =
      Some ((raw # rows) ! i)
  by blast
have len-rows: length rows = n
  using len-rows-full by simp
have lookup:
   $\bigwedge i. i < \text{Suc } n \implies$ 
    fmllookup (HashMap t)
      (QueryIndexChallenge (PQueryCounter s + i)
        (state-after-query-chunks (PState s) chunks i)) =
      Some ((raw # rows) ! i)
  using lookup-all by blast
show ?thesis
  by (rule that[OF len-rows len-chunks state fresh lookup])
qed

```

```

definition query-round-any-raw-set-hit
  :: ('f, 'a) protocol-channel-scheme  $\Rightarrow$  'f set  $\Rightarrow$ 
    (unit  $\times$  ('f, 'a) protocol-channel-scheme) option  $\Rightarrow$  bool
where
  query-round-any-raw-set-hit s B out  $\longleftrightarrow$ 
    (case out of
      None  $\Rightarrow$  False
    | Some (_, t)  $\Rightarrow$ 
      ( $\exists$  raw  $\in$  B.
        fmllookup (HashMap t)
          (QueryIndexChallenge (PQueryCounter s) (PState s)) =
          Some raw))

```

lemma *query-round-any-raw-set-hit-None*:
 $\neg$ *query-round-any-raw-set-hit* *s B None*
unfolding *query-round-any-raw-set-hit-def* **by** *simp*

lemma *query-round-any-raw-set-hit-NoneE*:
assumes *query-round-any-raw-set-hit* *s B None*
shows *P*
using *assms query-round-any-raw-set-hit-None* **by** *contradiction*

lemma *query-round-any-raw-set-hitI*:
assumes *raw* $\in$ *B*
and *fmlookup* (*HashMap* *t*)
(*QueryIndexChallenge* (*PQueryCounter* *s*) (*PState* *s*)) = *Some raw*
shows *query-round-any-raw-set-hit* *s B* (*Some* (*u*, *t*))

proof –
have *hit-eq*:
query-round-any-raw-set-hit *s B* (*Some* (*u*, *t*)) =
($\exists$ *raw* $\in$ *B*.
fmlookup (*HashMap* *t*)
(*QueryIndexChallenge* (*PQueryCounter* *s*) (*PState* *s*)) = *Some raw*)
unfolding *query-round-any-raw-set-hit-def* **by** *simp*
show *?thesis*
by (*subst hit-eq*, *rule* *beI*[*of* - *raw*], *rule* *assms*(2), *rule* *assms*(1))

qed

lemma *query-round-any-raw-set-hit-SomeE*:
assumes *query-round-any-raw-set-hit* *s B* (*Some* (*u*, *t*))
obtains *raw* **where**
raw $\in$ *B*
fmlookup (*HashMap* *t*)
(*QueryIndexChallenge* (*PQueryCounter* *s*) (*PState* *s*)) = *Some raw*

proof –
have *hit-eq*:
query-round-any-raw-set-hit *s B* (*Some* (*u*, *t*)) =
($\exists$ *raw* $\in$ *B*.
fmlookup (*HashMap* *t*)
(*QueryIndexChallenge* (*PQueryCounter* *s*) (*PState* *s*)) = *Some raw*)
unfolding *query-round-any-raw-set-hit-def* **by** *simp*
have $\exists$ *raw* $\in$ *B*.
fmlookup (*HashMap* *t*)
(*QueryIndexChallenge* (*PQueryCounter* *s*) (*PState* *s*)) = *Some raw*
using *assms* **by** (*simp* *only*: *hit-eq*)
then show *?thesis*
by (*blast intro*: *that*)

qed

lemma *receive-query-index-challenge-current-lookupD*:
assumes *Some* (*raw*, *s0*) $\in$ *set-dist* (*execute receive-query-index-challenge* *s*)

shows
 $fmlookup (HashMap s0)$
 $(QueryIndexChallenge (PQueryCounter s) (PState s)) = Some\ raw$
proof –
have *recv-info*:
 $s \leq s0 \wedge$
 $PState\ s0 = PState\ s \wedge$
 $PTranscript\ s0 = PTranscript\ s \wedge$
 $fmlookup (HashMap s0)$
 $(QueryIndexChallenge (PQueryCounter s) (PState s)) = Some\ raw$
by (rule *receive-query-index-challenge-outcome*[*OF* *assms*])
show *?thesis*
by (rule *conjunct2*[*OF* *conjunct2*[*OF* *conjunct2*[*OF* *recv-info*]]])
qed

lemma *verifier-query-round-after-index-program-preserves-current-lookupD*:
assumes *after*:
 $Some\ (u, t) \in$
 $set-dist$
 $(execute$
 $(verifier-query-round-after-index-program\ fr\ f-fl\ f-final\ as\ fl$
 $final\ raw)\ s0)$
and *lookup*:
 $fmlookup (HashMap s0)$
 $(QueryIndexChallenge (PQueryCounter s) (PState s)) = Some\ raw$
shows
 $fmlookup (HashMap t)$
 $(QueryIndexChallenge (PQueryCounter s) (PState s)) = Some\ raw$
proof –
let *?key* = $QueryIndexChallenge (PQueryCounter s) (PState s)$
have *u-eq*: $u = ()$
by (*cases* *u*) *simp*
have *preserve*:
 $fmlookup (HashMap t)\ ?key = fmlookup (HashMap s0)\ ?key$
by (rule *verifier-query-round-after-index-program-preserves-query-lookup*
 $[OF\ after[unfolded\ u-eq]]$)
show *?thesis*
proof (rule *HOL.trans*)
show $fmlookup (HashMap t)\ ?key = fmlookup (HashMap s0)\ ?key$
by (rule *preserve*)
show $fmlookup (HashMap s0)\ ?key = Some\ raw$
by (rule *lookup*)
qed
qed

lemma *wp-receive-query-index-challenge-raw-set-le*:
assumes *fresh*:
 $fmlookup (HashMap s)$
 $(QueryIndexChallenge (PQueryCounter s) (PState s)) = None$

```

shows
  wp-event receive-query-index-challenge
    ( $\lambda out. case\ out\ of\ None \Rightarrow False \mid Some\ (raw, -) \Rightarrow raw \in B$ )
     $s \leq nnreal\ (card\ B) / nnreal\ size$ 
proof -
  have eq:
    wp-event receive-query-index-challenge
      ( $\lambda out. case\ out\ of\ None \Rightarrow False \mid Some\ (raw, -) \Rightarrow raw \in B$ )
       $s = nnreal\ (card\ B) / nnreal\ size$ 
    by (rule wp-receive-query-index-challenge-fresh-set[OF fresh])
  show ?thesis
    by (subst eq, rule order-refl)
qed

lemma finite-query-index-raw-list-preimage[simp]:
  finite (query-index-raw-list-preimage Q)
proof -
  have query-index-raw-list-preimage Q  $\subseteq \{xs. length\ xs = rounds\}$ 
    unfolding query-index-raw-list-preimage-def by auto
  then show ?thesis
    apply (rule finite-subset) by (simp add: finite-list-length)
qed

lemma finite-query-index-raw-list-position-values[simp]:
  finite (query-index-raw-list-position-values Q i)
proof -
  have query-index-raw-list-position-values Q i  $\subseteq UNIV$ 
    by simp
  then show ?thesis
    by (rule finite-subset) simp
qed

lemma query-index-list-path-fresh-tail-lookup:
  assumes fresh:
    query-index-list-path-fresh s (Suc n) (chunk # chunks) (raw # rows)
  and head:
    Some (( ), s1)  $\in$ 
      set-dist
        (execute
          (verifier-query-round-program fr f-fl f-final as fl final) s)
  and head-chunk:
    PState s1 = foldl concat (PState s) chunk
  and head-counter:
    PQueryCounter s1 = Suc (PQueryCounter s)
  and i-bound:  $i < n$ 
shows
  fmlookup (HashMap s1)
    (QueryIndexChallenge (PQueryCounter s1 + i)
      (state-after-query-chunks (PState s1) chunks i)) = None

```

```

proof –
  from verifier-query-round-program-alt-outcomeE[OF head]
  obtain raw-idx s0 where recv:
    Some (raw-idx, s0) ∈ set-dist (execute receive-query-index-challenge s)
  and after:
    Some ((), s1) ∈
      set-dist
      (execute
        (verifier-query-round-after-index-program fr f-fl f-final as fl
          final raw-idx) s0)
  by blast
  let ?tail-key =
    QueryIndexChallenge (PQueryCounter s1 + i)
    (state-after-query-chunks (PState s1) chunks i)
  let ?full-key =
    QueryIndexChallenge (PQueryCounter s + Suc i)
    (state-after-query-chunks (PState s) (chunk # chunks) (Suc i))
  have key-eq: ?tail-key = ?full-key
    using head-chunk head-counter
    unfolding state-after-query-chunks-def
    by simp
  have suc-i-bound: Suc i < Suc n
    using i-bound by simp
  have full-none:
    fmlookup (HashMap s) ?full-key = None
    by (rule query-index-list-path-fresh-lookupD[OF fresh suc-i-bound])
  have neq:
    ?full-key ≠ QueryIndexChallenge (PQueryCounter s) (PState s)
    by simp
  have lookup-s0:
    fmlookup (HashMap s0) ?full-key = None
  proof –
    have preserve:
      fmlookup (HashMap s0) ?full-key =
        fmlookup (HashMap s) ?full-key
      by (rule receive-query-index-challenge-preserves-other-lookup
        [OF recv neq])
    show ?thesis
    proof (rule HOL.trans)
      show fmlookup (HashMap s0) ?full-key =
        fmlookup (HashMap s) ?full-key
      by (rule preserve)
      show fmlookup (HashMap s) ?full-key = None
      by (rule full-none)
    qed
  qed
  have lookup-s1:
    fmlookup (HashMap s1) ?full-key = None
  proof –

```

```

have preserve:
  fmlookup (HashMap s1) ?full-key =
    fmlookup (HashMap s0) ?full-key
by (rule verifier-query-round-after-index-program-preserves-query-lookup
    [OF after])
show ?thesis
proof (rule HOL.trans)
  show fmlookup (HashMap s1) ?full-key =
    fmlookup (HashMap s0) ?full-key
    by (rule preserve)
  show fmlookup (HashMap s0) ?full-key = None
    by (rule lookup-s0)
qed
qed
show ?thesis
  unfolding key-eq by (rule lookup-s1)
qed

lemma query-index-list-path-fresh-tail:
assumes fresh:
  query-index-list-path-fresh s (Suc n) (chunk # chunks) (raw # raws)
and head:
  Some ((), s1) ∈
    set-dist
      (execute
        (verifier-query-round-program fr f-fl f-final as fl final) s)
and head-chunk:
  PState s1 = foldl concat (PState s) chunk
and head-counter:
  PQueryCounter s1 = Suc (PQueryCounter s)
shows query-index-list-path-fresh s1 n chunks raws
proof –
have len-raws: length raws = n
  using fresh unfolding query-index-list-path-fresh-def by simp
have len-chunks: length chunks = n
  using fresh unfolding query-index-list-path-fresh-def by simp
have lookup-none:
   $\bigwedge i. i < n \implies$ 
    fmlookup (HashMap s1)
      (QueryIndexChallenge (PQueryCounter s1 + i)
        (state-after-query-chunks (PState s1) chunks i)) = None
  by (rule query-index-list-path-fresh-tail-lookup
    [OF fresh head head-chunk head-counter])
show ?thesis
  unfolding query-index-list-path-fresh-def
  using len-raws len-chunks lookup-none by simp
qed

lemma query-rounds-first-tail-key-fresh-after-head:

```

```

assumes fresh:
  query-index-list-path-fresh s (Suc n) chunks (raw # rows)
and head:
  Some ((), s1) ∈
    set-dist
      (execute
        (verifier-query-round-program fr f-fl f-final as fl final) s)
and one-bound: 1 < Suc n
and head-counter: PQueryCounter s1 = Suc (PQueryCounter s)
shows
  fmlookup (HashMap s1)
    (QueryIndexChallenge (PQueryCounter s1)
      (state-after-query-chunks (PState s) chunks 1)) = None
proof –
  let ?y = state-after-query-chunks (PState s) chunks 1
  let ?full-key = QueryIndexChallenge (PQueryCounter s + 1) ?y
  have full-none-s: fmlookup (HashMap s) ?full-key = None
    by (rule query-index-list-path-fresh-lookupD [OF fresh one-bound])
  from verifier-query-round-program-alt-outcomeE [OF head]
  obtain r s0 where recv:
    Some (r, s0) ∈ set-dist (execute receive-query-index-challenge s)
  and after:
    Some ((), s1) ∈
      set-dist
        (execute
          (verifier-query-round-after-index-program fr f-fl f-final as fl
            final r) s0)
  by blast
  have neq:
    ?full-key ≠ QueryIndexChallenge (PQueryCounter s) (PState s)
  by simp
  have s0-none:
    fmlookup (HashMap s0) ?full-key = None
  proof –
    have preserve:
      fmlookup (HashMap s0) ?full-key =
        fmlookup (HashMap s) ?full-key
      by (rule receive-query-index-challenge-preserves-other-lookup
        [OF recv neq])
    show ?thesis
    proof (rule HOL.trans)
      show fmlookup (HashMap s0) ?full-key =
        fmlookup (HashMap s) ?full-key
      by (rule preserve)
      show fmlookup (HashMap s) ?full-key = None
      by (rule full-none-s)
    qed
  qed
  have s1-none: fmlookup (HashMap s1) ?full-key = None

```

```

proof –
  have preserve:
    fmlookup (HashMap s1) ?full-key =
      fmlookup (HashMap s0) ?full-key
    by (rule verifier-query-round-after-index-program-preserves-query-lookup
      [OF after])
  show ?thesis
  proof (rule HOL.trans)
    show fmlookup (HashMap s1) ?full-key =
      fmlookup (HashMap s0) ?full-key
    by (rule preserve)
    show fmlookup (HashMap s0) ?full-key = None
    by (rule s0-none)
  qed
qed
show ?thesis
  using head-counter s1-none by simp
qed

lemma query-rounds-raw-list-path-hit-tail-head-state:
  assumes head:
    Some ((), s1) ∈
      set-dist
        (execute
          (verifier-query-round-program fr f-fl f-final as fl final) s)
  and tail:
    Some (results, t) ∈
      set-dist
        (execute
          (ntimes
            (verifier-query-round-program fr f-fl f-final as fl final) n)
          s1)
  and len-chunks-suc: length chunks = Suc n
  and state-t:
    PState t = state-after-query-chunks (PState s) chunks (Suc n)
  and fresh:
    query-index-list-path-fresh s (Suc n) chunks (raw # raws)
  and lookups:
     $\bigwedge i. i < \text{Suc } n \implies$ 
      fmlookup (HashMap t)
        (QueryIndexChallenge (PQueryCounter s + i)
          (state-after-query-chunks (PState s) chunks i)) =
        Some ((raw # raws) ! i)
  and chunks-eq: chunks = chunk # chunks-tail
  and head-counter: PQueryCounter s1 = Suc (PQueryCounter s)
  shows PState s1 = state-after-query-chunks (PState s) chunks 1
proof (cases n)
  case 0
  then have results = [] t = s1

```

```

    using tail by simp-all
  then show ?thesis
    using state-t 0 unfolding chunks-eq state-after-query-chunks-def by simp
next
case (Suc m)
from tail[unfolded Suc ntimes.simps] obtain u' s2 rest t0 where tail-head:
  Some (u', s2) ∈
    set-dist
      (execute
        (verifier-query-round-program fr f-fl f-final as fl final) s1)
and tail-rest0:
  Some (rest, t0) ∈
    set-dist
      (execute
        (ntimes
          (verifier-query-round-program fr f-fl f-final as fl final) m)
        s2)
and ret:
  Some (results, t) ∈ set-dist (execute (return (u' # rest)) t0)
by (auto elim!: set-dist-bindE)
have results-eq: results = u' # rest
  by (rule set-dist-return-valueD[OF ret])
have t-eq: t = t0
  by (rule set-dist-return-stateD[OF ret])
have tail-rest:
  Some (rest, t) ∈
    set-dist
      (execute
        (ntimes
          (verifier-query-round-program fr f-fl f-final as fl final) m)
        s2)
  by (subst t-eq, rule tail-rest0)
have u'-eq: u' = ()
  by (cases u') simp
let ?y = state-after-query-chunks (PState s) chunks 1
let ?key = QueryIndexChallenge (PQueryCounter s1) ?y
have full-none-s1:
  fmlookup (HashMap s1) ?key = None
  by (rule query-rounds-first-tail-key-fresh-after-head
    [OF fresh head - head-counter])
  (use Suc in simp)
have lookup-t:
  fmlookup (HashMap t) ?key = Some (raws ! 0)
  using lookups[of 1] Suc head-counter
  unfolding chunks-eq state-after-query-chunks-def by simp
have key-eq:
  ?key = QueryIndexChallenge (PQueryCounter s1) (PState s1)
proof (rule ccontr)
  assume neg:

```

```

    ?key ≠ QueryIndexChallenge (PQueryCounter s1) (PState s1)
have tail-head-unit:
  Some ((), s2) ∈
    set-dist
      (execute
        (verifier-query-round-program fr f-fl f-final as fl final) s1)
  using tail-head u'-eq by simp
from verifier-query-round-program-alt-outcomeE[OF tail-head-unit]
obtain r s0 where recv:
  Some (r, s0) ∈ set-dist (execute receive-query-index-challenge s1)
and after:
  Some ((), s2) ∈
    set-dist
      (execute
        (verifier-query-round-after-index-program fr f-fl f-final as fl
          final r) s0)
  by blast
have s0-none:
  fmlookup (HashMap s0) ?key = None
  using receive-query-index-challenge-preserves-other-lookup[OF recv neq]
  full-none-s1
  by simp
have s2-none: fmlookup (HashMap s2) ?key = None
proof –
  have preserve:
    fmlookup (HashMap s2) ?key = fmlookup (HashMap s0) ?key
    by (rule verifier-query-round-after-index-program-preserves-query-lookup
      [OF after])
  show ?thesis
  proof (rule HOL.trans)
    show fmlookup (HashMap s2) ?key = fmlookup (HashMap s0) ?key
    by (rule preserve)
    show fmlookup (HashMap s0) ?key = None
    by (rule s0-none)
  qed
qed
from verifier-query-round-program-outcome[OF tail-head[unfolded u'-eq]]
obtain r2::'f and idx2::nat and chunk2::'f list where
  counter-s2: PQueryCounter s2 = Suc (PQueryCounter s1)
  by blast
have lookup-pres:
  fmlookup (HashMap t) ?key = fmlookup (HashMap s2) ?key
  by (rule ntimes-verifier-query-round-program-preserves-past-query-lookup
    [OF - tail-rest])
  (use counter-s2 in simp)
show False
  using lookup-t s2-none lookup-pres by simp
qed
then show ?thesis

```



```

    by simp
qed

end

context soundness
begin

lemma query-rounds-raw-list-path-hit-tail:
  assumes head:
    Some (( ), s1) ∈
      set-dist
        (execute
          (verifier-query-round-program fr f-fl f-final as fl final) s)
  and tail:
    Some (results, t) ∈
      set-dist
        (execute
          (ntimes
            (verifier-query-round-program fr f-fl f-final as fl final) n)
          s1)
  and hit:
    query-rounds-raw-list-path-hit s (raw # rows) (Suc n)
      (Some (u # results, t))
  shows query-rounds-raw-list-path-hit s1 rows n (Some (results, t))
proof -
  from query-rounds-raw-list-path-hit-cons-SomeE[OF hit] obtain chunks where
    len-rows: length rows = n
  and len-chunks-suc: length chunks = Suc n
  and state-t:
    PState t = state-after-query-chunks (PState s) chunks (Suc n)
  and fresh:
    query-index-list-path-fresh s (Suc n) chunks (raw # rows)
  and lookups:
     $\bigwedge i. i < \text{Suc } n \implies$ 
      fmlookup (HashMap t)
        (QueryIndexChallenge (PQueryCounter s + i)
          (state-after-query-chunks (PState s) chunks i)) =
        Some ((raw # rows) ! i)
  by blast
  obtain chunk chunks-tail where chunks-eq: chunks = chunk # chunks-tail
  using len-chunks-suc by (cases chunks) auto
  have len-chunks-tail: length chunks-tail = n
  using len-chunks-suc unfolding chunks-eq by simp
  from verifier-query-round-program-outcome[OF head]
  obtain raw0::'f and idx::nat and chunk0::'f list where
    head-counter: PQueryCounter s1 = Suc (PQueryCounter s)
  by blast
  have head-state:

```

```

PState s1 = state-after-query-chunks (PState s) chunks 1
by (rule query-rounds-raw-list-path-hit-tail-head-state
  [OF head tail len-chunks-suc state-t fresh lookups chunks-eq
    head-counter])
have fresh-tail:
  query-index-list-path-fresh s1 n chunks-tail raws
proof -
  have fresh-chunks:
    query-index-list-path-fresh s (Suc n) (chunk # chunks-tail)
      (raw # raws)
  by (subst chunks-eq[symmetric], rule fresh)
  have head-chunk: PState s1 = foldl concat (PState s) chunk
proof -
  have head-state-cons:
    PState s1 =
      state-after-query-chunks (PState s) (chunk # chunks-tail) 1
  by (subst chunks-eq[symmetric], rule head-state)
  show ?thesis
  by (subst state-after-query-chunks-one-cons[symmetric],
    rule head-state-cons)
qed
show ?thesis
  by (rule query-index-list-path-fresh-tail
    [OF fresh-chunks head head-chunk head-counter])
qed
have state-tail:
  PState t =
    state-after-query-chunks (PState s1) chunks-tail n
proof -
  have shift:
    state-after-query-chunks (PState s1) chunks-tail n =
      state-after-query-chunks (PState s) (chunk # chunks-tail) (Suc n)
  by (rule state-after-query-chunks-cons-shift)
  (use head-state chunks-eq in simp)
  show ?thesis
  by (subst shift, subst chunks-eq[symmetric], rule state-t)
qed
have lookup-tail:
   $\bigwedge i. i < n \implies$ 
  fmlookup (HashMap t)
    (QueryIndexChallenge (PQueryCounter s1 + i)
      (state-after-query-chunks (PState s1) chunks-tail i)) =
    Some (raws ! i)
proof -
  fix i
  assume i-bound: i < n
  have key-eq:
    QueryIndexChallenge (PQueryCounter s1 + i)
      (state-after-query-chunks (PState s1) chunks-tail i) =

```

```

    QueryIndexChallenge (PQueryCounter s + Suc i)
      (state-after-query-chunks (PState s) chunks (Suc i))
  proof -
    have st-shift:
      state-after-query-chunks (PState s1) chunks-tail i =
        state-after-query-chunks (PState s) (chunk # chunks-tail) (Suc i)
    by (rule state-after-query-chunks-cons-shift-index)
      (use head-state chunks-eq in simp)
    show ?thesis
      using head-counter st-shift unfolding chunks-eq by simp
  qed
  show fmlookup (HashMap t)
    (QueryIndexChallenge (PQueryCounter s1 + i)
      (state-after-query-chunks (PState s1) chunks-tail i)) =
    Some (rows ! i)
  using lookups[of Suc i] i-bound key-eq by simp
  qed
  show ?thesis
    by (rule query-rounds-raw-list-path-hitI
      [OF len-rows len-chunks-tail state-tail fresh-tail lookup-tail])
  qed
end

context soundness
begin

lemma query-rounds-raw-list-path-hit-head:
  assumes head:
    Some ((), s1) ∈
      set-dist
        (execute
          (verifier-query-round-program fr f-fl f-final as fl final) s)
  and tail:
    Some (results, t) ∈
      set-dist
        (execute
          (ntimes
            (verifier-query-round-program fr f-fl f-final as fl final) n)
          s1)
  and hit:
    query-rounds-raw-list-path-hit s (raw # rows) (Suc n)
      (Some (u # results, t))
  shows query-round-any-raw-set-hit s {raw} (Some ((), s1))
proof -
  have lookup-t:
    fmlookup (HashMap t)
      (QueryIndexChallenge (PQueryCounter s) (PState s)) = Some raw
  proof -

```

```

from query-rounds-raw-list-path-hit-cons-SomeE[OF hit]
obtain chunks where lookup:
   $\bigwedge i. i < \text{Suc } n \implies$ 
    fmlookup (HashMap t)
      (QueryIndexChallenge (PQueryCounter s + i)
        (state-after-query-chunks (PState s) chunks i)) =
      Some ((raw # rows) ! i)
  by blast
have lookup0:
  fmlookup (HashMap t)
    (QueryIndexChallenge (PQueryCounter s + 0)
      (state-after-query-chunks (PState s) chunks 0)) =
    Some ((raw # rows) ! 0)
  using lookup[of 0] by simp
show ?thesis
  using lookup0 unfolding state-after-query-chunks-def by simp
qed
from verifier-query-round-program-alt-outcomeE[OF head]
obtain raw0 s0 where
  recv:
    Some (raw0, s0)  $\in$  set-dist (execute receive-query-index-challenge s)
  and after:
    Some ((), s1)  $\in$ 
      set-dist
        (execute
          (verifier-query-round-after-index-program fr f-fl f-final as fl
            final raw0) s0)
  by blast
have counter-s1: PQueryCounter s1 = Suc (PQueryCounter s)
  by (rule verifier-query-round-parts-counter[OF recv after])
have past: PQueryCounter s < PQueryCounter s1
  by (rule eq-Suc-imp-less-nat[OF counter-s1])
have lookup-s1:
  fmlookup (HashMap s1)
    (QueryIndexChallenge (PQueryCounter s) (PState s)) = Some raw
proof –
  let ?key = QueryIndexChallenge (PQueryCounter s) (PState s)
  have preserve:
    fmlookup (HashMap t) ?key = fmlookup (HashMap s1) ?key
  by (rule ntimes-verifier-query-round-program-preserves-past-query-lookup
    [OF past tail])
  show ?thesis
proof (rule HOL.trans)
  show fmlookup (HashMap s1) ?key = fmlookup (HashMap t) ?key
  by (rule HOL.sym[OF preserve])
  show fmlookup (HashMap t) ?key = Some raw
  by (rule lookup-t)
qed
qed

```

show ?thesis
 by (rule query-round-any-raw-set-hitI[OF singletonI lookup-s1])
qed

lemma verifier-query-round-program-raw-set-hit-imp-raw-in-Some:
assumes recv:
 Some (raw, s0) ∈ set-dist (execute receive-query-index-challenge s)
and after:
 Some (u, t) ∈
 set-dist
 (execute
 (verifier-query-round-after-index-program fr f-fl f-final as fl
 final raw) s0)
and hit: query-round-any-raw-set-hit s B (*Some (u, t)*)
shows raw ∈ B
proof –
from query-round-any-raw-set-hit-SomeE[OF hit]
obtain raw' where
 raw'-in: raw' ∈ B
and lookup-t:
 fmlookup (HashMap t)
 (QueryIndexChallenge (PQueryCounter s) (PState s)) = *Some raw'*
by blast
have lookup-s0:
 fmlookup (HashMap s0)
 (QueryIndexChallenge (PQueryCounter s) (PState s)) = *Some raw*
by (rule receive-query-index-challenge-current-lookupD[OF recv])
have lookup-t-current:
 fmlookup (HashMap t)
 (QueryIndexChallenge (PQueryCounter s) (PState s)) = *Some raw*
by (rule verifier-query-round-after-index-program-preserves-current-lookupD
 [OF after lookup-s0])
have some-eq: *Some raw = Some raw'*
 using lookup-t-current lookup-t **by** simp
have raw-eq: raw = raw'
 using some-eq **by** simp
show ?thesis
 by (subst raw-eq, rule raw'-in)
qed

lemma verifier-query-round-program-raw-set-hit-imp-raw-in:
assumes recv:
 Some (raw, s0) ∈ set-dist (execute receive-query-index-challenge s)
and after:
 out ∈
 set-dist
 (execute
 (verifier-query-round-after-index-program fr f-fl f-final as fl
 final raw) s0)

```

    and hit: query-round-any-raw-set-hit s B out
  shows raw ∈ B
proof -
  have rhs:
    (case out of
      None ⇒ False
    | Some (-, t) ⇒
      (∃ raw' ∈ B.
        fmlookup (HashMap t)
          (QueryIndexChallenge (PQueryCounter s) (PState s)) =
            Some raw'))
  using hit by (simp only: query-round-any-raw-set-hit-def)
  obtain u t where out-eq: out = Some (u, t)
proof (cases out)
  case None
  have False
  using rhs None by simp
  then show ?thesis
  by (rule FalseE)
next
  case (Some pair)
  then obtain a b where out = Some (a, b)
  by (cases pair) auto
  then show ?thesis
  by (rule that)
qed
have after-some:
  Some (u, t) ∈
    set-dist
      (execute
        (verifier-query-round-after-index-program fr f-fl f-final as fl
          final raw) s0)
  by (subst out-eq[symmetric]) (rule after)
have hit-some: query-round-any-raw-set-hit s B (Some (u, t))
  by (subst out-eq[symmetric]) (rule hit)
show ?thesis
  by (rule verifier-query-round-program-raw-set-hit-imp-raw-in-Some
    [OF recv after-some hit-some])
qed

lemma wp-verifier-query-round-program-any-raw-set-bound:
  assumes fresh:
    fmlookup (HashMap s)
      (QueryIndexChallenge (PQueryCounter s) (PState s)) = None
  shows
    wp-event (verifier-query-round-program fr f-fl f-final as fl final)
      (query-round-any-raw-set-hit s B) s ≤
      nnreal (card B) / nnreal size
  unfolding verifier-query-round-program-alt-def

```

```

proof (rule wp-event-bind-bound-by-head-event)
  show wp-event receive-query-index-challenge
    ( $\lambda out. \text{case out of None} \Rightarrow \text{False} \mid \text{Some (raw, -)} \Rightarrow \text{raw} \in B$ )
     $s \leq \text{nnreal (card B)} / \text{nnreal size}$ 
    by (rule wp-receive-query-index-challenge-raw-set-le[OF fresh])
next
  show query-round-any-raw-set-hit s B None  $\Rightarrow$ 
    ( $\text{case None of None} \Rightarrow \text{False} \mid \text{Some (raw, -)} \Rightarrow \text{raw} \in B$ )
    by (rule query-round-any-raw-set-hit-NoneE)
next
  fix raw s0 out
  assume recv:
     $\text{Some (raw, s0)} \in \text{set-dist (execute receive-query-index-challenge s)}$ 
  and out:
     $out \in$ 
     $\text{set-dist}$ 
    ( $\text{execute}$ 
      ( $\text{verifier-query-round-after-index-program fr f-fl f-final as fl}$ 
         $\text{final raw} \text{) s0}$ )
    and hit:  $\text{query-round-any-raw-set-hit s B out}$ 
  show ( $\text{case Some (raw, s0) of None} \Rightarrow \text{False} \mid \text{Some (raw, -)} \Rightarrow \text{raw} \in B$ )
proof –
  have raw-in:  $\text{raw} \in B$ 
    by (rule verifier-query-round-program-raw-set-hit-imp-raw-in
      [where  $s=s$  and  $B=B$  and  $\text{raw}=\text{raw}$  and  $s0=s0$  and  $\text{out}=\text{out}$ 
        and  $\text{fr}=\text{fr}$  and  $\text{f-fl}=\text{f-fl}$  and  $\text{f-final}=\text{f-final}$  and  $\text{as}=\text{as}$ 
        and  $\text{fl}=\text{fl}$  and  $\text{final}=\text{final}$ , OF recv out hit])
  then show ?thesis
    by simp
qed
qed

lemma query-rounds-raw-list-path-hit-SomeE:
  assumes query-rounds-raw-list-path-hit s raws n out
  obtains results t where  $\text{out} = \text{Some (results, t)}$ 
proof –
  show ?thesis
proof (cases out)
  case None
  have none-eq:  $\text{query-rounds-raw-list-path-hit s raws n None} = \text{False}$ 
    unfolding query-rounds-raw-list-path-hit-def by simp
  have hit-none:  $\text{query-rounds-raw-list-path-hit s raws n None}$ 
    by (subst None[symmetric], rule assms)
  then show ?thesis
    using hit-none by (simp only: none-eq)
next
  case (Some pair)
  then obtain results t where  $\text{out} = \text{Some (results, t)}$ 
    by (cases pair) auto

```

```

    then show ?thesis
      by (rule that)
  qed
qed

lemma query-rounds-raw-list-path-hit-some-fresh-currentD:
  assumes hit:
    query-rounds-raw-list-path-hit s (raw # rows-tail) (Suc n)
    (Some (ys, t))
  shows
    fmllookup (HashMap s)
      (QueryIndexChallenge (PQueryCounter s) (PState s)) = None
  proof -
    have some-eq:
      query-rounds-raw-list-path-hit s (raw # rows-tail) (Suc n)
      (Some (ys, t)) =
      (∃ chunks.
        length (raw # rows-tail) = Suc n ∧
        length chunks = Suc n ∧
        PState t = state-after-query-chunks (PState s) chunks (Suc n) ∧
        query-index-list-path-fresh s (Suc n) chunks (raw # rows-tail) ∧
        (∀ i < Suc n.
          fmllookup (HashMap t)
            (QueryIndexChallenge (PQueryCounter s + i)
              (state-after-query-chunks (PState s) chunks i)) =
            Some ((raw # rows-tail) ! i)))
    unfolding query-rounds-raw-list-path-hit-def by simp
    have ex-chunks:
      ∃ chunks.
        length (raw # rows-tail) = Suc n ∧
        length chunks = Suc n ∧
        PState t = state-after-query-chunks (PState s) chunks (Suc n) ∧
        query-index-list-path-fresh s (Suc n) chunks (raw # rows-tail) ∧
        (∀ i < Suc n.
          fmllookup (HashMap t)
            (QueryIndexChallenge (PQueryCounter s + i)
              (state-after-query-chunks (PState s) chunks i)) =
            Some ((raw # rows-tail) ! i))
    using hit by (simp only: some-eq)
    then obtain chunks where fresh-path:
      query-index-list-path-fresh s (Suc n) chunks (raw # rows-tail)
    by (elim exE conjE)
    show ?thesis
      by (rule query-index-list-path-fresh-head-fresh[OF fresh-path])
  qed

```

```

lemma query-rounds-raw-list-path-hit-some-head-lookupD:
  assumes hit:
    query-rounds-raw-list-path-hit s (raw # rows) (Suc n)

```



```

    (Some (ys, t))
  shows
    fmlookup (HashMap t)
      (QueryIndexChallenge (PQueryCounter s) (PState s)) = Some raw
  proof -
    have some-eq:
      query-rounds-row-list-path-hit s (raw # rows) (Suc n)
        (Some (ys, t)) =
        (∃ chunks.
          length (raw # rows) = Suc n ∧
          length chunks = Suc n ∧
          PState t = state-after-query-chunks (PState s) chunks (Suc n) ∧
          query-index-list-path-fresh s (Suc n) chunks (raw # rows) ∧
          (∀ i < Suc n.
            fmlookup (HashMap t)
              (QueryIndexChallenge (PQueryCounter s + i)
                (state-after-query-chunks (PState s) chunks i)) =
              Some ((raw # rows) ! i)))
    unfolding query-rounds-row-list-path-hit-def by simp
    have ex-chunks:
      ∃ chunks.
        length (raw # rows) = Suc n ∧
        length chunks = Suc n ∧
        PState t = state-after-query-chunks (PState s) chunks (Suc n) ∧
        query-index-list-path-fresh s (Suc n) chunks (raw # rows) ∧
        (∀ i < Suc n.
          fmlookup (HashMap t)
            (QueryIndexChallenge (PQueryCounter s + i)
              (state-after-query-chunks (PState s) chunks i)) =
          Some ((raw # rows) ! i))
    using hit by (simp only: some-eq)
    then obtain chunks where lookup-all:
      ∀ i < Suc n.
        fmlookup (HashMap t)
          (QueryIndexChallenge (PQueryCounter s + i)
            (state-after-query-chunks (PState s) chunks i)) =
        Some ((raw # rows) ! i)
    by (elim exE conjE)
    have lookup0:
      fmlookup (HashMap t)
        (QueryIndexChallenge (PQueryCounter s + 0)
          (state-after-query-chunks (PState s) chunks 0)) =
      Some ((raw # rows) ! 0)
    proof -
      have 0 < Suc n
        by simp
      from lookup-all this show ?thesis
        by blast
    qed
  qed

```

```

show ?thesis
  using lookup0 unfolding state-after-query-chunks-def by simp
qed

lemma query-rounds-raw-list-path-hit-bind-headD:
  assumes head:
    Some (u, s1) ∈
      set-dist
        (execute
          (verifier-query-round-program fr f-fl f-final as fl final) s)
  and support:
    out ∈
      set-dist
        (execute
          (ntimes (verifier-query-round-program fr f-fl f-final as fl final) n
            >>= (λxs. return (u # xs))) s1)
  and hit:
    query-rounds-raw-list-path-hit s (raw # raws-tail) (Suc n) out
  shows query-round-any-raw-set-hit s {raw} (Some (u, s1))
proof -
  let ?tail =
    ntimes (verifier-query-round-program fr f-fl f-final as fl final) n
  from hit obtain ys t0 where out-some: out = Some (ys, t0)
  by (rule query-rounds-raw-list-path-hit-SomeE)
  from support out-some obtain results t where tail:
    Some (results, t) ∈ set-dist (execute ?tail s1)
  and out-eq: out = Some (u # results, t)
  by (elim set-dist-bind-return-consE)
  have u-eq: u = ()
  by (cases u) simp
  have head-unit:
    Some ((), s1) ∈
      set-dist
        (execute
          (verifier-query-round-program fr f-fl f-final as fl final) s)
  using head u-eq by simp
  have hit-head-out:
    query-rounds-raw-list-path-hit s (raw # raws-tail) (Suc n)
      (Some (u # results, t))
  by (subst out-eq[symmetric], rule hit)
  have query-round-any-raw-set-hit s {raw} (Some ((), s1))
  by (rule query-rounds-raw-list-path-hit-head
    [OF head-unit tail hit-head-out])
  then show ?thesis
  using u-eq by simp
qed

lemma query-rounds-raw-list-path-hit-cont-zero-if-not-head:
  assumes head:

```

```

    Some (u, s1) ∈
      set-dist
        (execute
          (verifier-query-round-program fr f-fl f-final as fl final) s)
  and not-head:
    ¬ (fmlookup (HashMap s)
      (QueryIndexChallenge (PQueryCounter s) (PState s)) = None ∧
      query-round-any-raw-set-hit s {raw} (Some (u, s1)))
  shows
    wp-event
      (ntimes (verifier-query-round-program fr f-fl f-final as fl final) n
        ≫ (λxs. return (u # xs)))
      (query-rounds-raw-list-path-hit s (raw # rows-tail) (Suc n))
      s1 = 0
  proof (rule ccontr)
    let ?tail =
      ntimes (verifier-query-round-program fr f-fl f-final as fl final) n
    assume nonzero:
      wp-event (?tail ≫ (λxs. return (u # xs)))
      (query-rounds-raw-list-path-hit s (raw # rows-tail) (Suc n))
      s1 ≠ 0
    from wp-event-nonzero-imp-exists-support[OF nonzero]
    obtain out where out-support:
      out ∈ set-dist (execute (?tail ≫ (λxs. return (u # xs))) s1)
    and hit:
      query-rounds-raw-list-path-hit s (raw # rows-tail) (Suc n) out
    by blast
    from hit obtain ys t0 where out-some: out = Some (ys, t0)
    by (rule query-rounds-raw-list-path-hit-SomeE)
    have hit-some:
      query-rounds-raw-list-path-hit s (raw # rows-tail) (Suc n)
      (Some (ys, t0))
    by (subst out-some[symmetric], rule hit)
    have fresh-current:
      fmlookup (HashMap s)
      (QueryIndexChallenge (PQueryCounter s) (PState s)) = None
    by (rule query-rounds-raw-list-path-hit-some-fresh-currentD[OF hit-some])
    have raw-head:
      query-round-any-raw-set-hit s {raw} (Some (u, s1))
    by (rule query-rounds-raw-list-path-hit-bind-headD[OF head out-support hit])
    show False
    using not-head fresh-current raw-head by blast
  qed

```

lemma query-rounds-raw-list-path-hit-cont-le-tail:

assumes head:

```

  Some (u, s1) ∈
    set-dist
      (execute

```

```

      (verifier-query-round-program fr f-fl f-final as fl final) s)
shows
  wp-event
    (ntimes (verifier-query-round-program fr f-fl f-final as fl final) n
      >>= (λxs. return (u # xs)))
    (query-rounds-raw-list-path-hit s (raw # rows-tail) (Suc n))
    s1
  ≤ wp-event
    (ntimes (verifier-query-round-program fr f-fl f-final as fl final) n)
    (query-rounds-raw-list-path-hit s1 rows-tail n) s1
proof -
  let ?tail =
    ntimes (verifier-query-round-program fr f-fl f-final as fl final) n
  have u-eq: u = ()
  by (cases u) simp
  show ?thesis
  proof (rule wp-event-bind-bound-by-head-event)
    show wp-event ?tail
      (query-rounds-raw-list-path-hit s1 rows-tail n) s1
    ≤ wp-event ?tail
      (query-rounds-raw-list-path-hit s1 rows-tail n) s1
    by simp
  next
  show
    query-rounds-raw-list-path-hit s (raw # rows-tail) (Suc n) None
    ⇒ query-rounds-raw-list-path-hit s1 rows-tail n None
    unfolding query-rounds-raw-list-path-hit-def by simp
  next
  fix results t out
  assume tail:
    Some (results, t) ∈ set-dist (execute ?tail s1)
  and ret: out ∈ set-dist (execute (return (u # results)) t)
  and hit:
    query-rounds-raw-list-path-hit s (raw # rows-tail) (Suc n) out
  from ret obtain out-eq:
    out = Some (u # results, t)
  unfolding set-dist-def return.rep-eq dist-return-def
    dist-delta-dist delta-map-def
  by simp
  show query-rounds-raw-list-path-hit s1 rows-tail n
    (Some (results, t))
  by (rule query-rounds-raw-list-path-hit-tail
    [OF head[unfolded u-eq] tail])
    (use hit out-eq u-eq in simp)
qed
qed

lemma wp-ntimes-verifier-query-round-program-raw-list-path-hit-bound:
  assumes len-rows: length rows = n

```

```

shows
  wp-event
    (ntimes (verifier-query-round-program fr f-fl f-final as fl final) n)
    (query-rounds-raw-list-path-hit s raws n) s ≤
    (1 / nnreal size) ^ n
using len-raws
proof (induction n arbitrary: s raws)
  case 0
  then show ?case
    unfolding query-rounds-raw-list-path-hit-def
      query-index-list-path-fresh-def state-after-query-chunks-def
    by (simp add: wp-event-def wpsimps)
next
  case (Suc n)
  obtain raw raws-tail where raws-eq: raws = raw # raws-tail
    by (rule list-length-SucE[OF Suc.prem])
  have len-tail: length raws-tail = n
    using Suc.prem unfolding raws-eq by simp
  let ?prog = verifier-query-round-program fr f-fl f-final as fl final
  let ?tail =
    ntimes ?prog n
  let ?Q = query-rounds-raw-list-path-hit s raws (Suc n)
  let ?fresh =
    fmlookup (HashMap s)
      (QueryIndexChallenge (PQueryCounter s) (PState s)) = None
  let ?Head =
    λout. ?fresh ∧ query-round-any-raw-set-hit s {raw} out
  have head-bound:
    wp-event ?prog ?Head s ≤ 1 / nnreal size
  proof (cases ?fresh)
    case True
    have mono:
      wp-event ?prog ?Head s ≤
        wp-event ?prog (query-round-any-raw-set-hit s {raw}) s
      by (rule wp-event-mono) simp
    also have ... ≤ nnreal (card {raw}) / nnreal size
      by (rule wp-verifier-query-round-program-any-raw-set-bound[OF True])
    also have ... = 1 / nnreal size
      by simp
    finally show ?thesis .
  next
    case False
    have impossible: ?Head = (λ-. False)
      using False by auto
    have zero: wp-event ?prog ?Head s = 0
      by (simp only: impossible wp-event-false)
    then show ?thesis
      by simp
  qed

```

```

show ?case
  unfolding ntimes.simps rows-eq
proof (rule order-trans)
  have none-Q:
     $\neg$  query-rounds-raw-list-path-hit s (raw # rows-tail) (Suc n) None
  by (simp add: query-rounds-raw-list-path-hit-def)
show
  wp-event
    (?prog  $\gg$  ( $\lambda u.$  ?tail  $\gg$  ( $\lambda xs.$  return (u # xs))))
    (query-rounds-raw-list-path-hit s (raw # rows-tail) (Suc n)) s
   $\leq$  wp-event ?prog ?Head s * ((1 / nnreal size)  $^n$ )
proof (rule wp-event-bind-bound-by-head-and-cont
  [where m = ?prog
    and k =  $\lambda u.$  ?tail  $\gg$  ( $\lambda xs.$  return (u # xs))
    and Q = query-rounds-raw-list-path-hit s (raw # rows-tail) (Suc n)
    and P = ?Head
    and D = (1 / nnreal size)  $^n$ ,
    OF none-Q])
  fix u s1
  assume head:
    Some (u, s1)  $\in$  set-dist (execute ?prog s)
  and not-head:  $\neg$  ?Head (Some (u, s1))
show
  wp-event (?tail  $\gg$  ( $\lambda xs.$  return (u # xs)))
    (query-rounds-raw-list-path-hit s (raw # rows-tail) (Suc n))
    s1 = 0
by (rule query-rounds-raw-list-path-hit-cont-zero-if-not-head
  [OF head not-head])
next
  fix u s1
  assume head:
    Some (u, s1)  $\in$  set-dist (execute ?prog s)
  and head-hit: ?Head (Some (u, s1))
  have tail-bound:
    wp-event ?tail
      (query-rounds-raw-list-path-hit s1 rows-tail n) s1  $\leq$ 
      (1 / nnreal size)  $^n$ 
  by (rule Suc.IH[OF len-tail])
show
  wp-event (?tail  $\gg$  ( $\lambda xs.$  return (u # xs)))
    (query-rounds-raw-list-path-hit s (raw # rows-tail) (Suc n))
    s1  $\leq$  (1 / nnreal size)  $^n$ 
proof (rule order-trans[OF - tail-bound])
  show
    wp-event (?tail  $\gg$  ( $\lambda xs.$  return (u # xs)))
      (query-rounds-raw-list-path-hit s (raw # rows-tail) (Suc n))
      s1
     $\leq$  wp-event ?tail
      (query-rounds-raw-list-path-hit s1 rows-tail n) s1

```

```

proof (rule wp-event-bind-return-cons-mono
  [where  $m = ?tail$ 
    and  $Q =$ 
      query-rounds-raw-list-path-hit  $s$  (raw # rows-tail) (Suc  $n$ )
    and  $R =$  query-rounds-raw-list-path-hit  $s1$  rows-tail  $n$ 
    and  $u = u$ ])
show
  query-rounds-raw-list-path-hit  $s$  (raw # rows-tail) (Suc  $n$ )
  None  $\implies$ 
  query-rounds-raw-list-path-hit  $s1$  rows-tail  $n$  None
  unfolding query-rounds-raw-list-path-hit-def by simp
next
  fix  $xs\ t$ 
  assume tail:
    Some  $(xs, t) \in set-dist (execute\ ?tail\ s1)$ 
  and hit:
    query-rounds-raw-list-path-hit  $s$  (raw # rows-tail) (Suc  $n$ )
    (Some  $(u \# xs, t)$ )
  have u-eq:  $u = ()$ 
    by (cases  $u$ ) simp
  show query-rounds-raw-list-path-hit  $s1$  rows-tail  $n$ 
    (Some  $(xs, t)$ )
    by (rule query-rounds-raw-list-path-hit-tail
      [OF head[unfolded u-eq] tail])
      (use hit u-eq in simp)
  qed
  qed
  qed
next
  show wp-event ?prog ?Head  $s * (1 / nnreal\ size) ^ n$ 
     $\leq (1 / nnreal\ size) ^ {Suc\ n}$ 
    using head-bound by (simp add: mult-right-mono)
  qed
  qed

lemma wp-ntimes-verifier-query-round-program-raw-list-preimage-path-hit-bound:
shows
  wp-event
    (ntimes (verifier-query-round-program fr f-fl f-final as fl final)
      rounds)
  ( $\lambda out. \exists rows \in query-index-raw-list-preimage\ Q.$ 
    query-rounds-raw-list-path-hit  $s$  rows rounds out)
   $s \leq$ 
  nnreal (card (query-index-raw-list-preimage  $Q$ )) *
   $(1 / nnreal\ size) ^ {rounds}$ 
proof -
  have finite-rows: finite (query-index-raw-list-preimage  $Q$ )
    by (rule finite-query-index-raw-list-preimage)
  have

```

wp-event
 (ntimes (verifier-query-round-program fr f-fl f-final as fl final)
 rounds)
 (λout. ∃ rows ∈ query-index-raw-list-preimage Q.
 query-rounds-raw-list-path-hit s rows rounds out)
 s ≤
 (∑ rows ∈ query-index-raw-list-preimage Q.
 (1 / nnreal size) ^ rounds)
by (rule wp-event-finite-UN-bound[OF finite-rows])
 (rule wp-ntimes-verifier-query-round-program-raw-list-path-hit-bound,
 simp add: query-index-raw-list-preimage-def)
also have ... =
 nnreal (card (query-index-raw-list-preimage Q)) *
 (1 / nnreal size) ^ rounds
using finite-rows **by** simp
finally show ?thesis .
qed

lemma checked-staged-query-index-list-relation-fiber-card-bound:
 card {y. checked-staged-query-index-list-relation A Q x y}
 ≤ query-index-raw-list-relation-fiber-bound Q
proof –
 let ?values =
 (∪ i < rounds. query-index-raw-list-position-values Q i)
have subset:
 {y. checked-staged-query-index-list-relation A Q x y} ⊆ ?values
unfolding checked-staged-query-index-list-relation-def
 query-index-raw-list-position-values-def
by blast
have finite-values: finite ?values
by simp
have card {y. checked-staged-query-index-list-relation A Q x y}
 ≤ card ?values
by (rule card-mono[OF finite-values subset])
also have ... ≤
 (∑ i < rounds. card (query-index-raw-list-position-values Q i))
apply (rule card-UN-le) **by** simp
also have ... = query-index-raw-list-relation-fiber-bound Q
unfolding query-index-raw-list-relation-fiber-bound-def **by** simp
finally show ?thesis .
qed

lemma hash-relation-program-checked-staged-transcript-query-index-list:
assumes wf: staged-budget-wellformed budgets
and controlled: staged-adversary-controlled budgets A
shows
 hash-relation-program
 (checked-staged-query-index-list-relation A Q)
 (query-index-raw-list-relation-fiber-bound Q)

(staged-attacker-query-budget budgets + staged-challenge-query-budget)
 (checked-staged-transcript-program A)
by (rule hash-relation-program-checked-staged-transcript-program
 [OF wf controlled])
 (rule checked-staged-query-index-list-relation-fiber-card-bound)

lemma staged-transcript-query-index-list-set-hit-imp-relation-hit:

assumes support:

Some (data, attacker-state) ∈
 set-dist
 (execute (checked-staged-transcript-program A)
 adversary-initial-state)

and hit:

staged-transcript-query-index-list-set-hit Q
 (Some (data, attacker-state))

shows

hash-relation-hit
 (checked-staged-query-index-list-relation A Q)
 adversary-initial-state attacker-state

proof –

from hit **obtain** raw-idxs query-idxs **where**

len-row: length raw-idxs = rounds

and query-in: query-idxs ∈ Q

and query-idxs-eq:

query-idxs = map (λraw. index (to-nat raw)) raw-idxs

and lookup:

∧ i. i < rounds ⇒
 fmlookup (HashMap attacker-state)
 (QueryIndexChallenge i
 (state-after-query-chunks
 (staged-query-start-hash data)
 (staged-query-chunks data) i)) =
 Some (raw-idxs ! i)

unfolding staged-transcript-query-index-list-set-hit-def **by** auto

have raw-in: raw-idxs ∈ query-index-raw-list-preimage Q

unfolding query-index-raw-list-preimage-def

using len-row query-in query-idxs-eq **by** blast

let ?x =

QueryIndexChallenge 0
 (state-after-query-chunks
 (staged-query-start-hash data) (staged-query-chunks data) 0)

have zero-bound: 0 < rounds

by (rule rounds-positive)

have initial-none: fmlookup (HashMap adversary-initial-state) ?x = None

by (simp add: adversary-initial-state-def)

have final-lookup:

fmlookup (HashMap attacker-state) ?x = Some (raw-idxs ! 0)
using lookup[OF zero-bound] **by** simp

have rel:

```

checked-staged-query-index-list-relation A Q ?x (raw-idxs ! 0)
unfolding checked-staged-query-index-list-relation-def
by (intro exI[of - data] exI[of - attacker-state]
      exI[of - raw-idxs] exI[of - 0] conjI)
      (use support raw-in zero-bound in simp-all)
show ?thesis
  unfolding hash-relation-hit-def
  by (intro exI[of - ?x] exI[of - raw-idxs ! 0] conjI)
      (use initial-none final-lookup rel in simp-all)
qed

lemma staged-transcript-query-index-list-set-hit-imp-relation-hit-event:
assumes support:
  out ∈
    set-dist
      (execute (checked-staged-transcript-program A)
        adversary-initial-state)
and hit: staged-transcript-query-index-list-set-hit Q out
shows
  hash-relation-hit-event
    (checked-staged-query-index-list-relation A Q)
    adversary-initial-state out
proof (cases out)
case None
then show ?thesis
  using hit unfolding staged-transcript-query-index-list-set-hit-def
    hash-relation-hit-event-def
by simp
next
case (Some result)
then obtain data attacker-state where out-eq:
  out = Some (data, attacker-state)
by (cases result) simp
have support-some:
  Some (data, attacker-state) ∈
    set-dist
      (execute (checked-staged-transcript-program A)
        adversary-initial-state)
using support by (simp only: out-eq)
have hit-some:
  staged-transcript-query-index-list-set-hit Q
    (Some (data, attacker-state))
using hit by (simp only: out-eq)
have rel-hit:
  hash-relation-hit
    (checked-staged-query-index-list-relation A Q)
    adversary-initial-state attacker-state
by (rule staged-transcript-query-index-list-set-hit-imp-relation-hit
      [OF support-some hit-some])

```

```

show ?thesis
  unfolding out-eq hash-relation-hit-event-def
  using rel-hit by simp
qed

lemma staged-transcript-query-index-list-set-hit-bound:
  assumes wf: staged-budget-wellformed budgets
  and controlled: staged-adversary-controlled budgets A
  shows
    wp-event (checked-staged-transcript-program A)
      (staged-transcript-query-index-list-set-hit Q)
    adversary-initial-state ≤
    hash-relation-budget-value
      (query-index-raw-list-relation-fiber-bound Q)
      (staged-attacker-query-budget budgets +
        staged-challenge-query-budget)
proof –
  have event-mono:
    wp-event (checked-staged-transcript-program A)
      (staged-transcript-query-index-list-set-hit Q)
    adversary-initial-state ≤
    wp-event (checked-staged-transcript-program A)
      (hash-relation-hit-event
        (checked-staged-query-index-list-relation A Q)
        adversary-initial-state)
    adversary-initial-state
  proof (rule wp-event-mono-on-support)
  fix out
  assume support:
    out ∈
    set-dist
      (execute (checked-staged-transcript-program A)
        adversary-initial-state)
  and hit: staged-transcript-query-index-list-set-hit Q out
  show hash-relation-hit-event
    (checked-staged-query-index-list-relation A Q)
    adversary-initial-state out
  by (rule staged-transcript-query-index-list-set-hit-imp-relation-hit-event
    [OF support hit])
qed

have relation-bound:
  wp-event (checked-staged-transcript-program A)
    (hash-relation-hit-event
      (checked-staged-query-index-list-relation A Q)
      adversary-initial-state)
  adversary-initial-state ≤
  hash-relation-budget-value
    (query-index-raw-list-relation-fiber-bound Q)
    (staged-attacker-query-budget budgets +

```

```

    staged-challenge-query-budget)
proof —
  have program:
    hash-relation-program
      (checked-staged-query-index-list-relation A Q)
      (query-index-raw-list-relation-fiber-bound Q)
      (staged-attacker-query-budget budgets + staged-challenge-query-budget)
      (checked-staged-transcript-program A)
    by (rule hash-relation-program-checked-staged-transcript-query-index-list
      [OF wf controlled])
  then show ?thesis
    unfolding hash-relation-program-def hash-relation-budget-def by blast
  qed
show ?thesis
  by (rule order-trans[OF event-mono relation-bound])
qed

lemma checked-staged-security-with-data-query-index-list-set-bound-from-transcript:
  assumes wf: staged-budget-wellformed budgets
  and controlled: staged-adversary-controlled budgets A
  and transcript-bound:
    wp-event (checked-staged-transcript-program A)
    (staged-transcript-query-index-list-set-hit Q)
    adversary-initial-state  $\leq C$ 
  shows
    wp-event (checked-staged-security-experiment-with-data-state A)
    (staged-security-with-data-query-index-list-set-hit Q)
    adversary-initial-state  $\leq C$ 
  unfolding checked-staged-security-experiment-with-data-state-def
proof (rule wp-event-bind-bound-by-head-event[OF transcript-bound])
  show staged-security-with-data-query-index-list-set-hit Q None  $\implies$ 
    staged-transcript-query-index-list-set-hit Q None
  unfolding staged-security-with-data-query-index-list-set-hit-def
    staged-transcript-query-index-list-set-hit-def
  by simp
next
  fix data attacker-state out
  assume builder:
    Some (data, attacker-state)  $\in$ 
      set-dist
        (execute (checked-staged-transcript-program A)
          adversary-initial-state)
  and cont:
    out  $\in$ 
      set-dist
        (execute
          (get  $\gg$ 
            ( $\lambda s.$  put
              (verifier-state-from-adversary s

```

```

      (staged-proof-transcript data)) >>=
    (λ-.
      verify-monad >>=
        (λresult. return ((data, s), result))))))
  attacker-state)
and hit: staged-security-with-data-query-index-list-set-hit Q out
from checked-staged-transcript-program-outcome-query-lookups
  [OF wf controlled builder]
obtain attacker-raw-idxs attacker-query-idxs where
  len-attacker-raw: length attacker-raw-idxs = rounds
and attacker-query-idxs-eq:
  attacker-query-idxs =
    map (λraw. index (to-nat raw)) attacker-raw-idxs
and lookup-attacker:
  ∀ j < rounds.
    fmllookup (HashMap attacker-state)
      (QueryIndexChallenge j
        (state-after-query-chunks
          (staged-query-start-hash data)
          (staged-query-chunks data) j)) =
      Some (attacker-raw-idxs ! j)
by blast
from hit obtain data' attacker-state' result final-state raw-idxs query-idxs where
  out-eq: out = Some (((data', attacker-state'), result), final-state))
and len-raw: length raw-idxs = rounds
and query-in: query-idxs ∈ Q
and query-idxs-eq:
  query-idxs = map (λraw. index (to-nat raw)) raw-idxs
and lookup-final:
  ∧ i. i < rounds ⇒
    fmllookup (HashMap final-state)
      (QueryIndexChallenge i
        (state-after-query-chunks
          (staged-query-start-hash data')
          (staged-query-chunks data') i)) =
      Some (raw-idxs ! i)
unfolding staged-security-with-data-query-index-list-set-hit-def
by (auto split: option.splits prod.splits)
from cont[unfolded out-eq] obtain verifier-result where
  verifier:
    Some (verifier-result, final-state) ∈
      set-dist
        (execute verify-monad
          (verifier-state-from-adversary attacker-state
            (staged-proof-transcript data)))
and data'-eq: data' = data
and attacker-state'-eq: attacker-state' = attacker-state
by (auto elim!: set-dist-bindE)
have ext:

```

```

    verifier-state-from-adversary attacker-state
      (staged-proof-transcript data) ≤ final-state
  by (rule verify-monad-hash-extends[OF verifier])
have raw-eq:
  ∧i. i < rounds ⇒ attacker-raw-idxs ! i = raw-idxs ! i
proof -
  fix i
  assume i-bound: i < rounds
  have lookup-initial:
    fmllookup
      (HashMap
        (verifier-state-from-adversary attacker-state
          (staged-proof-transcript data)))
      (QueryIndexChallenge i
        (state-after-query-chunks
          (staged-query-start-hash data)
          (staged-query-chunks data) i)) =
        Some (attacker-raw-idxs ! i)
  using lookup-attacker i-bound by simp
  have lookup-final-from-initial:
    fmllookup (HashMap final-state)
      (QueryIndexChallenge i
        (state-after-query-chunks
          (staged-query-start-hash data)
          (staged-query-chunks data) i)) =
        Some (attacker-raw-idxs ! i)
  by (rule hash-extension-lookup[OF lookup-initial ext])
  show attacker-raw-idxs ! i = raw-idxs ! i
  using lookup-final-from-initial lookup-final[OF i-bound] data'-eq
  by simp
qed
have query-idxs-attacker:
  query-idxs = map (λraw. index (to-nat raw)) attacker-raw-idxs
proof (rule nth-equalityI)
  show length query-idxs =
    length (map (λraw. index (to-nat raw)) attacker-raw-idxs)
  using len-raw len-attacker-raw query-idxs-eq by simp
next
  fix i
  assume i-bound:
    i < length query-idxs
  then have i-rounds: i < rounds
  using len-raw query-idxs-eq by simp
  then show query-idxs ! i =
    map (λraw. index (to-nat raw)) attacker-raw-idxs ! i
  using query-idxs-eq raw-eq[OF i-rounds] len-raw apply simp
  using len-attacker-raw by auto
qed
show staged-transcript-query-index-list-set-hit Q

```

```

    (Some (data, attacker-state))
  unfolding staged-transcript-query-index-list-set-hit-def
  using len-attacker-raw query-in query-idxs-attacker lookup-attacker
  by auto
qed

lemma checked-staged-security-with-data-query-index-list-set-hit-bound:
  assumes wf: staged-budget-wellformed budgets
    and controlled: staged-adversary-controlled budgets A
  shows
    wp-event (checked-staged-security-experiment-with-data-state A)
      (staged-security-with-data-query-index-list-set-hit Q)
    adversary-initial-state ≤
    hash-relation-budget-value
      (query-index-raw-list-relation-fiber-bound Q)
      (staged-attacker-query-budget budgets +
        staged-challenge-query-budget)
  by (rule checked-staged-security-with-data-query-index-list-set-bound-from-transcript
    [OF wf controlled staged-transcript-query-index-list-set-hit-bound
      [OF wf controlled]])

lemma checked-staged-security-with-data-state-query-index-list-fresh-or-prequeried:
  assumes support:
    Some (((data, attacker-state), result), final-state) ∈
    set-dist
      (execute (checked-staged-security-experiment-with-data-state A)
        adversary-initial-state)
  and hit:
    staged-security-with-data-query-index-list-set-hit Q
    (Some (((data, attacker-state), result), final-state))
  shows
    staged-security-with-data-state-query-index-list-path-fresh Q
    (Some (((data, attacker-state), result), final-state)) ∨
    hash-relation-hit
      (checked-staged-query-index-list-relation A Q)
      adversary-initial-state attacker-state
  proof -
  from checked-staged-security-experiment-with-data-state-outcomeE[OF support]
  obtain builder verifier where
    builder:
      Some (data, attacker-state) ∈
      set-dist
        (execute (checked-staged-transcript-program A)
          adversary-initial-state)
  and verifier:
    Some (result, final-state) ∈
    set-dist
      (execute verify-monad
        (verifier-state-from-adversary attacker-state

```

```

      (staged-proof-transcript data)))
  by blast
from hit obtain raw-idxs query-idxs where
  len-raw: length raw-idxs = rounds
  and query-in: query-idxs ∈ Q
  and query-idxs-eq:
    query-idxs = map (λraw. index (to-nat raw)) raw-idxs
  and lookup-final:
    ∧ i. i < rounds ⇒
      fmllookup (HashMap final-state)
        (QueryIndexChallenge i
          (state-after-query-chunks
            (staged-query-start-hash data)
            (staged-query-chunks data) i)) =
        Some (raw-idxs ! i)
  unfolding staged-security-with-data-query-index-list-set-hit-def
  by auto
let ?vstate =
  verifier-state-from-adversary attacker-state
  (staged-proof-transcript data)
have raw-in: raw-idxs ∈ query-index-raw-list-preimage Q
  unfolding query-index-raw-list-preimage-def
  using len-raw query-in query-idxs-eq by blast
have ext: ?vstate ≤ final-state
  by (rule verify-monad-hash-extends[OF verifier])
show ?thesis
proof (cases
  ∀ i < rounds.
    fmllookup (HashMap ?vstate)
      (QueryIndexChallenge i
        (state-after-query-chunks
          (staged-query-start-hash data)
          (staged-query-chunks data) i)) =
      None)
  case True
  then have
    staged-security-with-data-state-query-index-list-path-fresh Q
      (Some (((data, attacker-state), result), final-state))
    unfolding staged-security-with-data-state-query-index-list-path-fresh-def
    using len-raw query-in query-idxs-eq lookup-final
    by (auto intro: exI[of - raw-idxs] exI[of - query-idxs])
  then show ?thesis by simp
next
  case False
  then obtain i old where
    i-bound: i < rounds
    and lookup-initial:
      fmllookup (HashMap ?vstate)
        (QueryIndexChallenge i

```



```

      (state-after-query-chunks
        (staged-query-start-hash data)
        (staged-query-chunks data) i)) =
    Some old
  by (metis option.exhaust)
let ?key =
  QueryIndexChallenge i
  (state-after-query-chunks
    (staged-query-start-hash data) (staged-query-chunks data) i)
have lookup-final-old:
  fmlookup (HashMap final-state) ?key = Some old
  by (rule hash-extension-lookup[OF lookup-initial ext])
have old-eq: old = raw-idxs ! i
  using lookup-final-old lookup-final[OF i-bound] by simp
have lookup-attacker:
  fmlookup (HashMap attacker-state) ?key = Some (raw-idxs ! i)
  using lookup-initial old-eq
  unfolding verifier-state-from-adversary-def by simp
have relation:
  checked-staged-query-index-list-relation A Q ?key (raw-idxs ! i)
  unfolding checked-staged-query-index-list-relation-def
  by (intro exI[of - data] exI[of - attacker-state]
    exI[of - raw-idxs] exI[of - i] conjI)
    (use builder raw-in i-bound in simp-all)
have initial-none:
  fmlookup (HashMap adversary-initial-state) ?key = None
  by (simp add: adversary-initial-state-def)
have
  hash-relation-hit
    (checked-staged-query-index-list-relation A Q)
    adversary-initial-state attacker-state
  unfolding hash-relation-hit-def
  by (intro exI[of - ?key] exI[of - raw-idxs ! i] conjI)
    (use initial-none lookup-attacker relation in simp-all)
then show ?thesis by simp
qed
qed

lemma checked-staged-security-with-data-state-query-index-list-prequery-hit-bound:
  assumes wf: staged-budget-wellformed budgets
  and controlled: staged-adversary-controlled budgets A
  shows
    wp-event (checked-staged-security-experiment-with-data-state A)
      (staged-security-with-data-state-query-index-list-prequery-hit A Q)
      adversary-initial-state ≤
      hash-relation-budget-value
        (query-index-raw-list-relation-fiber-bound Q)
        (staged-attacker-query-budget budgets +
          staged-challenge-query-budget)

```

```

unfolding checked-staged-security-experiment-with-data-state-def
proof (rule wp-event-bind-bound-by-head-event)
show wp-event (checked-staged-transcript-program A)
  (hash-relation-hit-event
   (checked-staged-query-index-list-relation A Q)
   adversary-initial-state)
  adversary-initial-state
  ≤ hash-relation-budget-value
    (query-index-row-list-relation-fiber-bound Q)
    (staged-attacker-query-budget budgets + staged-challenge-query-budget)
proof –
  have program:
    hash-relation-program
    (checked-staged-query-index-list-relation A Q)
    (query-index-row-list-relation-fiber-bound Q)
    (staged-attacker-query-budget budgets + staged-challenge-query-budget)
    (checked-staged-transcript-program A)
  by (rule hash-relation-program-checked-staged-transcript-query-index-list
        [OF wf controlled])
  then show ?thesis
    unfolding hash-relation-program-def hash-relation-budget-def by blast
qed
next
show
  staged-security-with-data-state-query-index-list-prequery-hit A Q None
  ⇒
  hash-relation-hit-event
  (checked-staged-query-index-list-relation A Q)
  adversary-initial-state None
unfolding staged-security-with-data-state-query-index-list-prequery-hit-def
  hash-relation-hit-event-def
by simp
next
fix data attacker-state out
assume cont:
  out ∈
  set-dist
  (execute
   (get ≥=
    (λs. put
     (verifier-state-from-adversary s
      (staged-proof-transcript data)) ≥=
     (λ-. verify-monad ≥=
      (λresult. return ((data, s), result))))))
   attacker-state)
assume prequery:
  staged-security-with-data-state-query-index-list-prequery-hit A Q out
show
  hash-relation-hit-event

```

```

    (checked-staged-query-index-list-relation A Q)
    adversary-initial-state (Some (data, attacker-state))
proof (cases out)
  case None
  then show ?thesis
    using prequery
    unfolding staged-security-with-data-state-query-index-list-prequery-hit-def
    by simp
next
  case (Some packed)
  from cont Some have cont-some:
    Some packed ∈
      set-dist
      (execute
        (get >=>
          (λs. put
            (verifier-state-from-adversary s
              (staged-proof-transcript data)) >=>
            (λ-. verify-monad >=>
              (λresult. return ((data, s), result))))))
          attacker-state)
    by simp
  obtain packed-value packed-state where packed-pair:
    packed = (packed-value, packed-state)
  by (cases packed)
  from cont-some[unfolded packed-pair] obtain captured get-state where get-out:
    Some (captured, get-state) ∈ set-dist (execute get attacker-state)
  and after-get0:
    Some (packed-value, packed-state) ∈
      set-dist
      (execute
        (put
          (verifier-state-from-adversary captured
            (staged-proof-transcript data)) >=>
          (λ-. verify-monad >=>
            (λresult. return ((data, captured), result))))
          get-state)
    by (rule set-dist-bindE) (rule that)
  have captured-eq: captured = attacker-state
  and get-state-eq: get-state = attacker-state
  using get-out
  unfolding set-dist-def get.rep-eq dist-get-def
  by (simp-all add: dist-delta-dist delta-map-def)
  have after-get:
    Some packed ∈
      set-dist
      (execute
        (put
          (verifier-state-from-adversary attacker-state

```

```

      (staged-proof-transcript data)) >>=
      (λ-. verify-monad >>=
        (λresult. return ((data, attacker-state), result))))
    attacker-state)
  using after-get0 captured-eq get-state-eq packed-pair by simp
from after-get[unfolded packed-pair] obtain unit put-state where put-out:
  Some (unit, put-state) ∈
    set-dist
      (execute
        (put
          (verifier-state-from-adversary attacker-state
            (staged-proof-transcript data))))
        attacker-state)
  and after-put0:
    Some (packed-value, packed-state) ∈
      set-dist
        (execute
          (verify-monad >>=
            (λresult. return ((data, attacker-state), result))))
          put-state)
  by (rule set-dist-bindE) (rule that)
have put-state-eq:
  put-state =
    verifier-state-from-adversary attacker-state
      (staged-proof-transcript data)
  using put-out
  unfolding set-dist-def put.rep-eq dist-put-def
  by (simp add: dist-delta-dist delta-map-def)
have after-put:
  Some packed ∈
    set-dist
      (execute
        (verify-monad >>=
          (λresult. return ((data, attacker-state), result))))
        (verifier-state-from-adversary attacker-state
          (staged-proof-transcript data)))
  using after-put0 put-state-eq packed-pair by simp
from after-put[unfolded packed-pair] obtain result final-state where verify-out:
  Some (result, final-state) ∈
    set-dist
      (execute verify-monad
        (verifier-state-from-adversary attacker-state
          (staged-proof-transcript data)))
  and ret-out:
    Some (packed-value, packed-state) ∈
      set-dist (execute (return ((data, attacker-state), result)) final-state)
  by (rule set-dist-bindE) (rule that)
have packed-eq: packed = (((data, attacker-state), result), final-state)
proof -

```

```

have value-eq: packed-value = ((data, attacker-state), result)
  by (rule set-dist-return-valueD[OF ret-out])
have state-eq: packed-state = final-state
  by (rule set-dist-return-stateD[OF ret-out])
show ?thesis
  using packed-pair value-eq state-eq by simp
qed
have out-eq:
  out = Some (((data, attacker-state), result), final-state)
  using Some packed-eq by simp
from prequery have hit:
  hash-relation-hit
    (checked-staged-query-index-list-relation A Q)
    adversary-initial-state attacker-state
  unfolding out-eq
    staged-security-with-data-state-query-index-list-prequery-hit-def
  by simp
then show ?thesis
  using hit unfolding hash-relation-hit-event-def by simp
qed
qed

lemma checked-staged-security-with-data-state-query-index-list-hit-bound-from-path-fresh:
assumes wf: staged-budget-wellformed budgets
and controlled: staged-adversary-controlled budgets A
and path-fresh-bound:
  wp-event (checked-staged-security-experiment-with-data-state A)
    (staged-security-with-data-state-query-index-list-path-fresh Q)
    adversary-initial-state  $\leq C$ 
shows
  wp-event (checked-staged-security-experiment-with-data-state A)
    (staged-security-with-data-state-query-index-list-set-hit Q)
    adversary-initial-state  $\leq$ 
    C +
    hash-relation-budget-value
      (query-index-raw-list-relation-fiber-bound Q)
      (staged-attacker-query-budget budgets +
        staged-challenge-query-budget)

proof –
have event-mono:
  wp-event (checked-staged-security-experiment-with-data-state A)
    (staged-security-with-data-state-query-index-list-set-hit Q)
    adversary-initial-state  $\leq$ 
  wp-event (checked-staged-security-experiment-with-data-state A)
    ( $\lambda$ out.
      staged-security-with-data-state-query-index-list-path-fresh Q out  $\vee$ 
      staged-security-with-data-state-query-index-list-prequery-hit A Q out)
    adversary-initial-state
proof (rule wp-event-mono-on-support)

```

```

fix out
assume support:
  out ∈
    set-dist
      (execute (checked-staged-security-experiment-with-data-state A)
        adversary-initial-state)
  and hit: staged-security-with-data-query-index-list-set-hit Q out
show
  staged-security-with-data-state-query-index-list-path-fresh Q out ∨
  staged-security-with-data-state-query-index-list-prequery-hit A Q out
proof (cases out)
  case None
  then show ?thesis
    using hit unfolding staged-security-with-data-query-index-list-set-hit-def
    by simp
next
  case (Some packed)
  then obtain data attacker-state result final-state where out-eq:
    out = Some (((data, attacker-state), result), final-state)
    by (cases packed) (auto split: prod.splits)
  have split:
    staged-security-with-data-state-query-index-list-path-fresh Q
      (Some (((data, attacker-state), result), final-state)) ∨
    hash-relation-hit
      (checked-staged-query-index-list-relation A Q)
    adversary-initial-state attacker-state
  apply (rule
    checked-staged-security-with-data-state-query-index-list-fresh-or-prequeried)
  using support hit unfolding out-eq by simp-all
  then show ?thesis
    unfolding out-eq
    staged-security-with-data-state-query-index-list-prequery-hit-def
    by simp
  qed
qed
have union-bound:
  wp-event (checked-staged-security-experiment-with-data-state A)
    (λout.
      staged-security-with-data-state-query-index-list-path-fresh Q out ∨
      staged-security-with-data-state-query-index-list-prequery-hit A Q out)
  adversary-initial-state ≤
  wp-event (checked-staged-security-experiment-with-data-state A)
    (staged-security-with-data-state-query-index-list-path-fresh Q)
  adversary-initial-state +
  wp-event (checked-staged-security-experiment-with-data-state A)
    (staged-security-with-data-state-query-index-list-prequery-hit A Q)
  adversary-initial-state
  by (rule wp-event-union-bound)
have prequery-bound:

```

```

wp-event (checked-staged-security-experiment-with-data-state A)
  (staged-security-with-data-state-query-index-list-prequery-hit A Q)
  adversary-initial-state ≤
  hash-relation-budget-value
    (query-index-raw-list-relation-fiber-bound Q)
    (staged-attacker-query-budget budgets +
      staged-challenge-query-budget)
by (rule
  checked-staged-security-with-data-state-query-index-list-prequery-hit-bound
  [OF wf controlled])
have wp-event (checked-staged-security-experiment-with-data-state A)
  (staged-security-with-data-state-query-index-list-set-hit Q)
  adversary-initial-state ≤
  wp-event (checked-staged-security-experiment-with-data-state A)
    (staged-security-with-data-state-query-index-list-path-fresh Q)
    adversary-initial-state +
  wp-event (checked-staged-security-experiment-with-data-state A)
    (staged-security-with-data-state-query-index-list-prequery-hit A Q)
    adversary-initial-state
by (rule order-trans[OF event-mono union-bound])
also have ... ≤
  C +
  hash-relation-budget-value
    (query-index-raw-list-relation-fiber-bound Q)
    (staged-attacker-query-budget budgets +
      staged-challenge-query-budget)
by (rule add-mono[OF path-fresh-bound prequery-bound])
finally show ?thesis .
qed

```

lemma *accepted-fri-opening-transcript-imp-accepted-transcript-shape:*
assumes *fri:*
accepted-fri-opening-transcript s out trace-roots trace-bs trace-final
dg composition-roots composition-bs composition-final query-idxs
trace-round-layers composition-round-layers
shows $\exists$ *as*. *accepted-transcript-shape s out as query-idxs*
using *fri*
unfolding *accepted-fri-opening-transcript-def*
accepted-transcript-shape-def
by *blast*

lemma *checked-staged-security-trace-fri-query-index-list-set-hit-bound-list:*
assumes *wf:* *staged-budget-wellformed budgets*
and *controlled:* *staged-adversary-controlled budgets A*
shows
wp-event (checked-staged-security-experiment-with-data-state A)
(staged-security-with-data-state-verifier-event
*(λ s. *trace-fri-query-index-list-set-hit s Q*))*
adversary-initial-state ≤

```

    hash-relation-budget-value
      (query-index-raw-list-relation-fiber-bound Q)
      (staged-attacker-query-budget budgets +
        staged-challenge-query-budget)
  proof (rule order-trans)
  show wp-event (checked-staged-security-experiment-with-data-state A)
    (staged-security-with-data-state-verifier-event
      (λs. trace-fri-query-index-list-set-hit s Q))
    adversary-initial-state ≤
  wp-event (checked-staged-security-experiment-with-data-state A)
    (staged-security-with-data-query-index-list-set-hit Q)
    adversary-initial-state
  proof (rule wp-event-mono-on-support)
  fix out
  assume support:
    out ∈
      set-dist
      (execute (checked-staged-security-experiment-with-data-state A)
        adversary-initial-state)
  and hit:
    staged-security-with-data-state-verifier-event
      (λs. trace-fri-query-index-list-set-hit s Q) out
  show staged-security-with-data-query-index-list-set-hit Q out
  proof (cases out)
  case None
  then show ?thesis
    using hit
    unfolding staged-security-with-data-state-verifier-event-def
      staged-security-with-data-query-index-list-set-hit-def
    by simp
  next
  case (Some packed)
  then obtain data attacker-state result final-state where out-eq:
    out = Some (((data, attacker-state), result), final-state)
    by (cases packed) (auto split: prod.splits)
  let ?s =
    verifier-state-from-adversary attacker-state
      (staged-proof-transcript data)
  from hit out-eq have trace-hit:
    trace-fri-query-index-list-set-hit ?s Q
    (Some (result, final-state))
  unfolding staged-security-with-data-state-verifier-event-def by simp
  from trace-hit obtain trace-roots trace-bs trace-final dg
    composition-roots composition-bs composition-final query-idxs
    trace-round-layers composition-round-layers where
  fri:
    accepted-fri-opening-transcript ?s (Some (result, final-state))
    trace-roots trace-bs trace-final dg composition-roots
    composition-bs composition-final query-idxs trace-round-layers

```



```

    composition-round-layers
  and query-in: query-idxs ∈ Q
  unfolding trace-fri-query-index-list-set-hit-def by blast
from accepted-fri-opening-transcript-imp-accepted-transcript-shape[OF fri]
obtain as where shape:
  accepted-transcript-shape ?s (Some (result, final-state)) as
  query-idxs
  by blast
from checked-staged-security-with-data-state-accepted-shape-query-keys
  [OF wf controlled support[unfolded out-eq] shape]
obtain raw-idxs where
  len-raw: length raw-idxs = rounds
  and query-idxs-eq:
    query-idxs = map (λraw. index (to-nat raw)) raw-idxs
  and lookup:
    ∧i. i < rounds ⇒
      fmllookup (HashMap final-state)
        (QueryIndexChallenge i
          (state-after-query-chunks
            (staged-query-start-hash data)
            (staged-query-chunks data) i)) =
        Some (raw-idxs ! i)
    by blast
show ?thesis
  unfolding out-eq staged-security-with-data-query-index-list-set-hit-def
  using len-raw query-in query-idxs-eq lookup by auto
qed
qed
next
show wp-event (checked-staged-security-experiment-with-data-state A)
  (staged-security-with-data-query-index-list-set-hit Q)
  adversary-initial-state
  ≤ hash-relation-budget-value
  (query-index-raw-list-relation-fiber-bound Q)
  (staged-attacker-query-budget budgets + staged-challenge-query-budget)
  by (rule checked-staged-security-with-data-query-index-list-set-hit-bound
    [OF wf controlled])
qed

lemma checked-staged-security-composition-fri-query-index-list-set-hit-bound-list:
  assumes wf: staged-budget-wellformed budgets
  and controlled: staged-adversary-controlled budgets A
  shows
    wp-event (checked-staged-security-experiment-with-data-state A)
      (staged-security-with-data-state-verifier-event
        (λs. composition-fri-query-index-list-set-hit s Q))
    adversary-initial-state ≤
    hash-relation-budget-value
    (query-index-raw-list-relation-fiber-bound Q)

```

```

      (staged-attacker-query-budget budgets +
       staged-challenge-query-budget)
proof (rule order-trans)
show wp-event (checked-staged-security-experiment-with-data-state A)
  (staged-security-with-data-state-verifier-event
   (λs. composition-fri-query-index-list-set-hit s Q))
  adversary-initial-state ≤
  wp-event (checked-staged-security-experiment-with-data-state A)
  (staged-security-with-data-query-index-list-set-hit Q)
  adversary-initial-state
proof (rule wp-event-mono-on-support)
fix out
assume support:
  out ∈
  set-dist
  (execute (checked-staged-security-experiment-with-data-state A)
   adversary-initial-state)
and hit:
  staged-security-with-data-state-verifier-event
  (λs. composition-fri-query-index-list-set-hit s Q) out
show staged-security-with-data-query-index-list-set-hit Q out
proof (cases out)
case None
then show ?thesis
  using hit
  unfolding staged-security-with-data-state-verifier-event-def
    staged-security-with-data-query-index-list-set-hit-def
  by simp
next
case (Some packed)
then obtain data attacker-state result final-state where out-eq:
  out = Some (((data, attacker-state), result), final-state)
  by (cases packed) (auto split: prod.splits)
let ?s =
  verifier-state-from-adversary attacker-state
  (staged-proof-transcript data)
from hit out-eq have composition-hit:
  composition-fri-query-index-list-set-hit ?s Q
  (Some (result, final-state))
  unfolding staged-security-with-data-state-verifier-event-def by simp
from composition-hit obtain trace-roots trace-bs trace-final dg
  composition-roots composition-bs composition-final query-idxs
  trace-round-layers composition-round-layers where
  fri:
  accepted-fri-opening-transcript ?s (Some (result, final-state))
  trace-roots trace-bs trace-final dg composition-roots
  composition-bs composition-final query-idxs trace-round-layers
  composition-round-layers
and query-in: query-idxs ∈ Q

```

```

    unfolding composition-fri-query-index-list-set-hit-def by blast
from accepted-fri-opening-transcript-imp-accepted-transcript-shape[OF fri]
obtain as where shape:
    accepted-transcript-shape ?s (Some (result, final-state)) as
        query-idxs
    by blast
from checked-staged-security-with-data-state-accepted-shape-query-keys
    [OF wf controlled support[unfolded out-eq] shape]
obtain raw-idxs where
    len-raw: length raw-idxs = rounds
    and query-idxs-eq:
        query-idxs = map ( $\lambda$ raw. index (to-nat raw)) raw-idxs
    and lookup:
         $\bigwedge i. i < \text{rounds} \implies$ 
            fmlookup (HashMap final-state)
                (QueryIndexChallenge i
                    (state-after-query-chunks
                        (staged-query-start-hash data)
                        (staged-query-chunks data) i)) =
                Some (raw-idxs ! i)
    by blast
show ?thesis
    unfolding out-eq staged-security-with-data-query-index-list-set-hit-def
    using len-raw query-in query-idxs-eq lookup by auto
qed
qed
next
show wp-event (checked-staged-security-experiment-with-data-state A)
    (staged-security-with-data-query-index-list-set-hit Q)
    adversary-initial-state
     $\leq$  hash-relation-budget-value
    (query-index-raw-list-relation-fiber-bound Q)
    (staged-attacker-query-budget budgets + staged-challenge-query-budget)
by (rule checked-staged-security-with-data-query-index-list-set-hit-bound
    [OF wf controlled])
qed

lemma checked-staged-security-trace-fri-query-challenge-pair-set-hit-bound-list:
assumes wf: staged-budget-wellformed budgets
and controlled: staged-adversary-controlled budgets A
and projection: fst ‘  $P \subseteq Q$ 
shows
    wp-event (checked-staged-security-experiment-with-data-state A)
    (staged-security-with-data-state-verifier-event
        ( $\lambda s. \text{trace-fri-query-challenge-pair-set-hit } s \ P$ ))
    adversary-initial-state  $\leq$ 
    hash-relation-budget-value
    (query-index-raw-list-relation-fiber-bound Q)
    (staged-attacker-query-budget budgets +

```

staged-challenge-query-budget)

proof (rule order-trans)

show *wp-event* (*checked-staged-security-experiment-with-data-state A*)
 (*staged-security-with-data-state-verifier-event*
 ($\lambda s. \text{trace-fri-query-challenge-pair-set-hit } s \ P$))
 adversary-initial-state $\leq$
wp-event (*checked-staged-security-experiment-with-data-state A*)
 (*staged-security-with-data-state-verifier-event*
 ($\lambda s. \text{trace-fri-query-index-list-set-hit } s \ Q$))
 adversary-initial-state

by (rule *wp-event-mono-on-support*)
 (auto simp: *staged-security-with-data-state-verifier-event-def*
 intro:
 trace-fri-query-challenge-pair-set-hit-imp-query-index-list-set-hit
 [*OF - projection*]
 split: *option.splits prod.splits*)

next

show *wp-event* (*checked-staged-security-experiment-with-data-state A*)
 (*staged-security-with-data-state-verifier-event*
 ($\lambda s. \text{trace-fri-query-index-list-set-hit } s \ Q$))
 adversary-initial-state
 $\leq$ *hash-relation-budget-value*
 (*query-index-raw-list-relation-fiber-bound Q*)
 (*staged-attacker-query-budget budgets* + *staged-challenge-query-budget*)

by (rule *checked-staged-security-trace-fri-query-index-list-set-hit-bound-list*
 [*OF wf controlled*])

qed

lemma *checked-staged-security-composition-fri-query-challenge-pair-set-hit-bound-list:*

assumes *wf: staged-budget-wellformed budgets*
and *controlled: staged-adversary-controlled budgets A*
and *projection: $\bigwedge dg. \text{fst } ' P \ dg \subseteq Q$*

shows

wp-event (*checked-staged-security-experiment-with-data-state A*)
 (*staged-security-with-data-state-verifier-event*
 ($\lambda s. \text{composition-fri-query-challenge-pair-set-hit } s \ P$))
 adversary-initial-state $\leq$
hash-relation-budget-value
 (*query-index-raw-list-relation-fiber-bound Q*)
 (*staged-attacker-query-budget budgets* +
 staged-challenge-query-budget)

proof (rule order-trans)

show *wp-event* (*checked-staged-security-experiment-with-data-state A*)
 (*staged-security-with-data-state-verifier-event*
 ($\lambda s. \text{composition-fri-query-challenge-pair-set-hit } s \ P$))
 adversary-initial-state $\leq$
wp-event (*checked-staged-security-experiment-with-data-state A*)
 (*staged-security-with-data-state-verifier-event*
 ($\lambda s. \text{composition-fri-query-index-list-set-hit } s \ Q$))

adversary-initial-state
by (rule *wp-event-mono-on-support*)
 (auto simp: *staged-security-with-data-state-verifier-event-def*
 intro:
 composition-fri-query-challenge-pair-set-hit-imp-query-index-list-set-hit
 [*OF* - *projection*]
 split: *option.splits prod.splits*)
next
show *wp-event (checked-staged-security-experiment-with-data-state A)*
 (*staged-security-with-data-state-verifier-event*
 ($\lambda s.$ *composition-fri-query-index-list-set-hit s Q*))
adversary-initial-state
 $\leq$ *hash-relation-budget-value*
 (*query-index-raw-list-relation-fiber-bound Q*)
 (*staged-attacker-query-budget budgets* + *staged-challenge-query-budget*)
by (rule
 checked-staged-security-composition-fri-query-index-list-set-hit-bound-list
 [*OF wf controlled*])
qed

lemma *checked-staged-security-trace-fri-query-index-list-set-hit-bound-from-path-fresh:*
assumes *wf: staged-budget-wellformed budgets*
and *controlled: staged-adversary-controlled budgets A*
and *path-fresh-bound:*
 wp-event (checked-staged-security-experiment-with-data-state A)
 (*staged-security-with-data-state-query-index-list-path-fresh Q*)
 adversary-initial-state $\leq C$
shows
 wp-event (checked-staged-security-experiment-with-data-state A)
 (*staged-security-with-data-state-verifier-event*
 ($\lambda s.$ *trace-fri-query-index-list-set-hit s Q*))
 adversary-initial-state $\leq$
 C +
 hash-relation-budget-value
 (*query-index-raw-list-relation-fiber-bound Q*)
 (*staged-attacker-query-budget budgets* +
 staged-challenge-query-budget)
proof (rule *order-trans*)
show *wp-event (checked-staged-security-experiment-with-data-state A)*
 (*staged-security-with-data-state-verifier-event*
 ($\lambda s.$ *trace-fri-query-index-list-set-hit s Q*))
adversary-initial-state $\leq$
wp-event (checked-staged-security-experiment-with-data-state A)
 (*staged-security-with-data-query-index-list-set-hit Q*)
adversary-initial-state
proof (rule *wp-event-mono-on-support*)
fix *out*
assume *support:*
 out $\in$

```

    set-dist
    (execute (checked-staged-security-experiment-with-data-state A)
      adversary-initial-state)
  and hit:
    staged-security-with-data-state-verifier-event
    (λs. trace-fri-query-index-list-set-hit s Q) out
show staged-security-with-data-query-index-list-set-hit Q out
proof (cases out)
  case None
  then show ?thesis
    using hit
    unfolding staged-security-with-data-state-verifier-event-def
      staged-security-with-data-query-index-list-set-hit-def
    by simp
next
case (Some packed)
then obtain data attacker-state result final-state where out-eq:
  out = Some (((data, attacker-state), result), final-state)
  by (cases packed) (auto split: prod.splits)
let ?s =
  verifier-state-from-adversary attacker-state
  (staged-proof-transcript data)
from hit out-eq have trace-hit:
  trace-fri-query-index-list-set-hit ?s Q
  (Some (result, final-state))
  unfolding staged-security-with-data-state-verifier-event-def by simp
from trace-hit obtain trace-roots trace-bs trace-final dg
  composition-roots composition-bs composition-final query-idxs
  trace-round-layers composition-round-layers where
  fri:
    accepted-fri-opening-transcript ?s (Some (result, final-state))
    trace-roots trace-bs trace-final dg composition-roots
    composition-bs composition-final query-idxs trace-round-layers
    composition-round-layers
  and query-in: query-idxs ∈ Q
  unfolding trace-fri-query-index-list-set-hit-def by blast
from accepted-fri-opening-transcript-imp-accepted-transcript-shape[OF fri]
obtain as where shape:
  accepted-transcript-shape ?s (Some (result, final-state)) as
  query-idxs
  by blast
from checked-staged-security-with-data-state-accepted-shape-query-keys
  [OF wf controlled support[unfolded out-eq] shape]
obtain raw-idxs where
  len-raw: length raw-idxs = rounds
  and query-idxs-eq:
    query-idxs = map (λraw. index (to-nat raw)) raw-idxs
  and lookup:
    ∧i. i < rounds ⇒

```

```

      fmlookup (HashMap final-state)
        (QueryIndexChallenge i
          (state-after-query-chunks
            (staged-query-start-hash data)
            (staged-query-chunks data) i)) =
          Some (raw-idxs ! i)
    by blast
  show ?thesis
    unfolding out-eq staged-security-with-data-query-index-list-set-hit-def
    using len-raw query-in query-idxs-eq lookup by auto
qed
qed
next
  show wp-event (checked-staged-security-experiment-with-data-state A)
    (staged-security-with-data-query-index-list-set-hit Q)
    adversary-initial-state
    ≤ C +
      hash-relation-budget-value
        (query-index-raw-list-relation-fiber-bound Q)
        (staged-attacker-query-budget budgets +
          staged-challenge-query-budget)
    by (rule
      checked-staged-security-with-data-state-query-index-list-hit-bound-from-path-fresh
      [OF wf controlled path-fresh-bound])
qed

lemma checked-staged-security-composition-fri-query-index-list-set-hit-bound-from-path-fresh:
  assumes wf: staged-budget-wellformed budgets
  and controlled: staged-adversary-controlled budgets A
  and path-fresh-bound:
    wp-event (checked-staged-security-experiment-with-data-state A)
      (staged-security-with-data-state-query-index-list-path-fresh Q)
      adversary-initial-state ≤ C
  shows
    wp-event (checked-staged-security-experiment-with-data-state A)
      (staged-security-with-data-state-verifier-event
        (λs. composition-fri-query-index-list-set-hit s Q))
      adversary-initial-state ≤
        C +
          hash-relation-budget-value
            (query-index-raw-list-relation-fiber-bound Q)
            (staged-attacker-query-budget budgets +
              staged-challenge-query-budget)
proof (rule order-trans)
  show wp-event (checked-staged-security-experiment-with-data-state A)
    (staged-security-with-data-state-verifier-event
      (λs. composition-fri-query-index-list-set-hit s Q))
    adversary-initial-state ≤
      wp-event (checked-staged-security-experiment-with-data-state A)

```

```

    (staged-security-with-data-query-index-list-set-hit Q)
    adversary-initial-state
proof (rule wp-event-mono-on-support)
  fix out
  assume support:
    out ∈
      set-dist
      (execute (checked-staged-security-experiment-with-data-state A)
        adversary-initial-state)
  and hit:
    staged-security-with-data-state-verifier-event
    (λs. composition-fri-query-index-list-set-hit s Q) out
show staged-security-with-data-query-index-list-set-hit Q out
proof (cases out)
  case None
  then show ?thesis
    using hit
    unfolding staged-security-with-data-state-verifier-event-def
      staged-security-with-data-query-index-list-set-hit-def
    by simp
next
  case (Some packed)
  then obtain data attacker-state result final-state where out-eq:
    out = Some (((data, attacker-state), result), final-state)
    by (cases packed) (auto split: prod.splits)
  let ?s =
    verifier-state-from-adversary attacker-state
    (staged-proof-transcript data)
  from hit out-eq have composition-hit:
    composition-fri-query-index-list-set-hit ?s Q
    (Some (result, final-state))
    unfolding staged-security-with-data-state-verifier-event-def by simp
  from composition-hit obtain trace-roots trace-bs trace-final dg
    composition-roots composition-bs composition-final query-idxs
    trace-round-layers composition-round-layers where
    fri:
      accepted-fri-opening-transcript ?s (Some (result, final-state))
      trace-roots trace-bs trace-final dg composition-roots
      composition-bs composition-final query-idxs trace-round-layers
      composition-round-layers
    and query-in: query-idxs ∈ Q
    unfolding composition-fri-query-index-list-set-hit-def by blast
  from accepted-fri-opening-transcript-imp-accepted-transcript-shape[OF fri]
  obtain as where shape:
    accepted-transcript-shape ?s (Some (result, final-state)) as
    query-idxs
    by blast
  from checked-staged-security-with-data-state-accepted-shape-query-keys
    [OF wf controlled support[unfolded out-eq] shape]

```



```

obtain raw-idxs where
  len-raw: length raw-idxs = rounds
and query-idxs-eq:
  query-idxs = map ( $\lambda$ raw. index (to-nat raw)) raw-idxs
and lookup:
   $\bigwedge i. i < \text{rounds} \implies$ 
    fmlookup (HashMap final-state)
      (QueryIndexChallenge i
        (state-after-query-chunks
          (staged-query-start-hash data)
          (staged-query-chunks data) i)) =
        Some (raw-idxs ! i)
by blast
show ?thesis
  unfolding out-eq staged-security-with-data-query-index-list-set-hit-def
  using len-raw query-in query-idxs-eq lookup by auto
qed
qed
next
show wp-event (checked-staged-security-experiment-with-data-state A)
  (staged-security-with-data-query-index-list-set-hit Q)
  adversary-initial-state
   $\leq C +$ 
    hash-relation-budget-value
    (query-index-raw-list-relation-fiber-bound Q)
    (staged-attacker-query-budget budgets +
      staged-challenge-query-budget)
by (rule
  checked-staged-security-with-data-state-query-index-list-hit-bound-from-path-fresh
  [OF wf controlled path-fresh-bound])
qed

lemma checked-staged-security-trace-fri-query-challenge-pair-set-hit-bound-from-path-fresh:
assumes wf: staged-budget-wellformed budgets
and controlled: staged-adversary-controlled budgets A
and projection: fst ‘  $P \subseteq Q$ 
and path-fresh-bound:
  wp-event (checked-staged-security-experiment-with-data-state A)
  (staged-security-with-data-state-query-index-list-path-fresh Q)
  adversary-initial-state  $\leq C$ 
shows
  wp-event (checked-staged-security-experiment-with-data-state A)
  (staged-security-with-data-state-verifier-event
    ( $\lambda s. \text{trace-fri-query-challenge-pair-set-hit } s \ P$ ))
  adversary-initial-state  $\leq$ 
    C +
    hash-relation-budget-value
    (query-index-raw-list-relation-fiber-bound Q)
    (staged-attacker-query-budget budgets +

```

staged-challenge-query-budget)

proof (*rule order-trans*)

show *wp-event* (*checked-staged-security-experiment-with-data-state A*)
 (*staged-security-with-data-state-verifier-event*
 ($\lambda s. \text{trace-fri-query-challenge-pair-set-hit } s \ P$))
adversary-initial-state $\leq$
wp-event (*checked-staged-security-experiment-with-data-state A*)
 (*staged-security-with-data-state-verifier-event*
 ($\lambda s. \text{trace-fri-query-index-list-set-hit } s \ Q$))
adversary-initial-state

by (*rule wp-event-mono-on-support*)
 (*auto simp: staged-security-with-data-state-verifier-event-def*
intro:
trace-fri-query-challenge-pair-set-hit-imp-query-index-list-set-hit
[OF - projection]
split: option.splits prod.splits)

next

show *wp-event* (*checked-staged-security-experiment-with-data-state A*)
 (*staged-security-with-data-state-verifier-event*
 ($\lambda s. \text{trace-fri-query-index-list-set-hit } s \ Q$))
adversary-initial-state
 $\leq C +$
hash-relation-budget-value
 (*query-index-raw-list-relation-fiber-bound Q*)
 (*staged-attacker-query-budget budgets +*
staged-challenge-query-budget)

by (*rule*
checked-staged-security-trace-fri-query-index-list-set-hit-bound-from-path-fresh
[OF wf controlled path-fresh-bound])

qed

lemma *checked-staged-security-composition-fri-query-challenge-pair-set-hit-bound-from-path-fresh:*

assumes *wf: staged-budget-wellformed budgets*

and *controlled: staged-adversary-controlled budgets A*

and *projection: $\bigwedge dg. \text{fst } dg \subseteq P \implies dg \subseteq Q$*

and *path-fresh-bound:*

wp-event (*checked-staged-security-experiment-with-data-state A*)
 (*staged-security-with-data-state-query-index-list-path-fresh Q*)
adversary-initial-state $\leq C$

shows

wp-event (*checked-staged-security-experiment-with-data-state A*)
 (*staged-security-with-data-state-verifier-event*
 ($\lambda s. \text{composition-fri-query-challenge-pair-set-hit } s \ P$))
adversary-initial-state $\leq$
 $C +$
hash-relation-budget-value
 (*query-index-raw-list-relation-fiber-bound Q*)
 (*staged-attacker-query-budget budgets +*
staged-challenge-query-budget)

```

proof (rule order-trans)
  show wp-event (checked-staged-security-experiment-with-data-state A)
    (staged-security-with-data-state-verifier-event
      ( $\lambda s$ . composition-fri-query-challenge-pair-set-hit s P))
    adversary-initial-state  $\leq$ 
    wp-event (checked-staged-security-experiment-with-data-state A)
      (staged-security-with-data-state-verifier-event
        ( $\lambda s$ . composition-fri-query-index-list-set-hit s Q))
      adversary-initial-state
  by (rule wp-event-mono-on-support)
    (auto simp: staged-security-with-data-state-verifier-event-def
      intro:
        composition-fri-query-challenge-pair-set-hit-imp-query-index-list-set-hit
        [OF - projection]
        split: option.splits prod.splits)
next
  show wp-event (checked-staged-security-experiment-with-data-state A)
    (staged-security-with-data-state-verifier-event
      ( $\lambda s$ . composition-fri-query-index-list-set-hit s Q))
    adversary-initial-state
     $\leq C +$ 
    hash-relation-budget-value
      (query-index-raw-list-relation-fiber-bound Q)
      (staged-attacker-query-budget budgets +
        staged-challenge-query-budget)
  by (rule
    checked-staged-security-composition-fri-query-index-list-set-hit-bound-from-path-fresh
    [OF wf controlled path-fresh-bound])
qed

end

end

```

```

theory Soundness-FRI-Query-List-Exact-Product
  imports Soundness-FRI-Query-List-Prequery
begin

```

Exact product accounting for sampled FRI query-index lists.

The preceding layer proves the product theorem in raw field space. This layer converts raw-list preimages to query-index-list targets using the current exact-uniformity assumptions for the modulo sampler.

```

context soundness
begin

```

```

definition verifier-after-composition-fri-raw-list-path-hit
  :: ('f, 'a) protocol-channel-scheme  $\Rightarrow$  nat list set  $\Rightarrow$ 
    (unit list  $\times$  ('f, 'a) protocol-channel-scheme) option  $\Rightarrow$  bool

```

where

verifier-after-composition-fri-raw-list-path-hit $s \ Q \ out \longleftrightarrow$
 $(\exists \text{final query-state rows.}$
 $\text{Some } (final, \text{query-state}) \in \text{set-dist } (\text{execute read } s) \wedge$
 $\text{rows} \in \text{query-index-raw-list-preimage } Q \wedge$
 $\text{query-rounds-raw-list-path-hit query-state rows rounds out})$

lemma *verifier-after-composition-fri-raw-list-path-hitI*:

assumes *read-final*:

$\text{Some } (final, \text{query-state}) \in \text{set-dist } (\text{execute read } s)$

and *raw-in*: $\text{rows} \in \text{query-index-raw-list-preimage } Q$

and *len-rows*: $\text{length rows} = \text{rounds}$

and *len-chunks*: $\text{length chunks} = \text{rounds}$

and *state*:

$PState \ t = \text{state-after-query-chunks } (PState \ \text{query-state}) \ \text{chunks} \ \text{rounds}$

and *fresh*:

$\text{query-index-list-path-fresh query-state rounds chunks rows}$

and *lookup*:

$\bigwedge i. i < \text{rounds} \implies$

$\text{fmlookup } (HashMap \ t)$

$(QueryIndexChallenge \ (PQueryCounter \ \text{query-state} + i)$

$(\text{state-after-query-chunks } (PState \ \text{query-state}) \ \text{chunks} \ i)) =$

$\text{Some } (\text{rows} \ ! \ i)$

and *out-eq*: $\text{out} = \text{Some } (\text{results}, \ t)$

shows *verifier-after-composition-fri-raw-list-path-hit* $s \ Q \ out$

proof –

have *path*: $\text{query-rounds-raw-list-path-hit query-state rows rounds out}$

unfolding *out-eq*

by (rule *query-rounds-raw-list-path-hitI*

[*OF len-rows len-chunks state fresh lookup*])

show *?thesis*

unfolding *verifier-after-composition-fri-raw-list-path-hit-def*

by (intro *exI*[*of - final*] *exI*[*of - query-state*] *exI*[*of - rows*]

conjI read-final raw-in path)

qed

lemma *staged-security-with-data-state-query-index-list-path-fresh-SomeE*:

assumes *hit*:

$\text{staged-security-with-data-state-query-index-list-path-fresh } Q$

$(\text{Some } (((data, \text{attacker-state}), \text{result}), \text{final-state}))$

obtains *raw-idxs query-idxs* **where**

$\text{length raw-idxs} = \text{rounds}$

$\text{query-idxs} \in Q$

$\text{query-idxs} = \text{map } (\lambda \text{raw. index } (\text{to-nat raw})) \ \text{raw-idxs}$

$\bigwedge i. i < \text{rounds} \implies$

$\text{fmlookup } (HashMap \ \text{final-state})$

$(QueryIndexChallenge \ i$

$(\text{state-after-query-chunks}$

$(\text{staged-query-start-hash } data)$

```

      (staged-query-chunks data) i)) =
    Some (raw-idxs ! i)
  ∧ i. i < rounds ⇒
    fmllookup
      (HashMap
        (verifier-state-from-adversary attacker-state
          (staged-proof-transcript data)))
      (QueryIndexChallenge i
        (state-after-query-chunks
          (staged-query-start-hash data)
          (staged-query-chunks data) i)) =
    None
proof —
have hit-ex:
  ∃ raw-idxs query-idxs.
    length raw-idxs = rounds ∧
    query-idxs ∈ Q ∧
    query-idxs = map (λ raw. index (to-nat raw)) raw-idxs ∧
    (∀ i < rounds.
      fmllookup (HashMap final-state)
        (QueryIndexChallenge i
          (state-after-query-chunks
            (staged-query-start-hash data)
            (staged-query-chunks data) i)) =
        Some (raw-idxs ! i) ∧
      fmllookup
        (HashMap
          (verifier-state-from-adversary attacker-state
            (staged-proof-transcript data)))
        (QueryIndexChallenge i
          (state-after-query-chunks
            (staged-query-start-hash data)
            (staged-query-chunks data) i)) =
        None)
using hit
unfolding staged-security-with-data-state-query-index-list-path-fresh-def
by simp
from hit-ex obtain raw-idxs query-idxs where len:
  length raw-idxs = rounds
and in-Q: query-idxs ∈ Q
and map-eq:
  query-idxs = map (λ raw. index (to-nat raw)) raw-idxs
and lookups:
  ∀ i < rounds.
    fmllookup (HashMap final-state)
      (QueryIndexChallenge i
        (state-after-query-chunks
          (staged-query-start-hash data)
          (staged-query-chunks data) i)) =

```

```

    Some (raw-idxs ! i) ∧
    fmlookup
      (HashMap
        (verifier-state-from-adversary attacker-state
          (staged-proof-transcript data)))
      (QueryIndexChallenge i
        (state-after-query-chunks
          (staged-query-start-hash data)
          (staged-query-chunks data) i)) =
    None
  by blast
show ?thesis
proof (rule that[OF len in-Q map-eq])
  fix i
  assume i < rounds
  then show
    fmlookup (HashMap final-state)
      (QueryIndexChallenge i
        (state-after-query-chunks
          (staged-query-start-hash data)
          (staged-query-chunks data) i)) =
    Some (raw-idxs ! i)
  using lookups by blast
next
  fix i
  assume i < rounds
  then show
    fmlookup
      (HashMap
        (verifier-state-from-adversary attacker-state
          (staged-proof-transcript data)))
      (QueryIndexChallenge i
        (state-after-query-chunks
          (staged-query-start-hash data)
          (staged-query-chunks data) i)) =
    None
  using lookups by blast
qed
qed

lemma staged-security-with-data-state-query-index-list-path-fresh-None[simp]:
  ¬ staged-security-with-data-state-query-index-list-path-fresh Q None
  unfolding staged-security-with-data-state-query-index-list-path-fresh-def
  by simp

lemma staged-security-with-data-state-query-index-list-path-fresh-caseD:
  assumes hit:
    staged-security-with-data-state-query-index-list-path-fresh Q
    (case out of

```

```

    None  $\Rightarrow$  None
  | Some (result, final-state)  $\Rightarrow$ 
    Some (((data, attacker-state), result), final-state))
shows
 $\exists$  result final-state raw-idxs query-idxs.
  out = Some (result, final-state)  $\wedge$ 
  length raw-idxs = rounds  $\wedge$ 
  query-idxs  $\in Q$   $\wedge$ 
  query-idxs = map ( $\lambda$ raw. index (to-nat raw)) raw-idxs  $\wedge$ 
  ( $\forall i < \text{rounds}$ .
    fmllookup (HashMap final-state)
      (QueryIndexChallenge i
        (state-after-query-chunks
          (staged-query-start-hash data)
          (staged-query-chunks data) i)) =
      Some (raw-idxs ! i))  $\wedge$ 
  ( $\forall i < \text{rounds}$ .
    fmllookup
      (HashMap
        (verifier-state-from-adversary attacker-state
          (staged-proof-transcript data)))
      (QueryIndexChallenge i
        (state-after-query-chunks
          (staged-query-start-hash data)
          (staged-query-chunks data) i)) =
      None)
proof (cases out)
case None
then show ?thesis
using hit by simp
next
case (Some pair)
then obtain result final-state where out-eq:
  out = Some (result, final-state)
by (cases pair) simp
have hit-ex:
 $\exists$  raw-idxs query-idxs.
  length raw-idxs = rounds  $\wedge$ 
  query-idxs  $\in Q$   $\wedge$ 
  query-idxs = map ( $\lambda$ raw. index (to-nat raw)) raw-idxs  $\wedge$ 
  ( $\forall i < \text{rounds}$ .
    fmllookup (HashMap final-state)
      (QueryIndexChallenge i
        (state-after-query-chunks
          (staged-query-start-hash data)
          (staged-query-chunks data) i)) =
      Some (raw-idxs ! i))  $\wedge$ 
  fmllookup
    (HashMap

```

```

      (verifier-state-from-adversary attacker-state
        (staged-proof-transcript data)))
    (QueryIndexChallenge i
      (state-after-query-chunks
        (staged-query-start-hash data)
        (staged-query-chunks data) i)) =
    None)
  using hit out-eq
  unfolding staged-security-with-data-state-query-index-list-path-fresh-def
  by simp
  from hit-ex obtain raw-idxs query-idxs where
    length raw-idxs = rounds
    query-idxs ∈ Q
    query-idxs = map (λraw. index (to-nat raw)) raw-idxs
    ∀ i < rounds.
      fmllookup (HashMap final-state)
        (QueryIndexChallenge i
          (state-after-query-chunks
            (staged-query-start-hash data)
            (staged-query-chunks data) i)) =
      Some (raw-idxs ! i) ∧
      fmllookup
        (HashMap
          (verifier-state-from-adversary attacker-state
            (staged-proof-transcript data)))
        (QueryIndexChallenge i
          (state-after-query-chunks
            (staged-query-start-hash data)
            (staged-query-chunks data) i)) =
      None
  by blast
  then show ?thesis
  using out-eq by blast
qed

```

lemma *staged-security-with-data-state-query-index-list-path-fresh-caseE*:

```

assumes hit:
  staged-security-with-data-state-query-index-list-path-fresh Q
  (case out of
    None ⇒ None
  | Some (result, final-state) ⇒
    Some (((data, attacker-state), result), final-state))
obtains result final-state raw-idxs query-idxs where
  out = Some (result, final-state)
  length raw-idxs = rounds
  query-idxs ∈ Q
  query-idxs = map (λraw. index (to-nat raw)) raw-idxs
  ∧ i. i < rounds ⇒
    fmllookup (HashMap final-state)

```



```

      (QueryIndexChallenge i
        (state-after-query-chunks
          (staged-query-start-hash data)
          (staged-query-chunks data) i)) =
      Some (raw-idxs ! i)
     $\bigwedge i. i < \text{rounds} \implies$ 
    fmlookup
      (HashMap
        (verifier-state-from-adversary attacker-state
          (staged-proof-transcript data)))
      (QueryIndexChallenge i
        (state-after-query-chunks
          (staged-query-start-hash data)
          (staged-query-chunks data) i)) =
      None
  proof (cases out)
  case None
  then have False
    using hit by simp
  then show ?thesis by simp
next
case (Some pair)
then obtain result final-state where out-eq:
  out = Some (result, final-state)
  by (cases pair) simp
have hit-some:
  staged-security-with-data-state-query-index-list-path-fresh Q
    (Some (((data, attacker-state), result), final-state))
  using hit out-eq by simp
from staged-security-with-data-state-query-index-list-path-fresh-SomeE
  [OF hit-some]
obtain raw-idxs query-idxs where
  len: length raw-idxs = rounds
  and in-Q: query-idxs  $\in$  Q
  and map-eq: query-idxs = map ( $\lambda$ raw. index (to-nat raw)) raw-idxs
  and lookup:
     $\bigwedge i. i < \text{rounds} \implies$ 
    fmlookup (HashMap final-state)
      (QueryIndexChallenge i
        (state-after-query-chunks
          (staged-query-start-hash data)
          (staged-query-chunks data) i)) =
      Some (raw-idxs ! i)
  and fresh:
     $\bigwedge i. i < \text{rounds} \implies$ 
    fmlookup
      (HashMap
        (verifier-state-from-adversary attacker-state
          (staged-proof-transcript data)))

```

```

      (QueryIndexChallenge i
        (state-after-query-chunks
          (staged-query-start-hash data)
          (staged-query-chunks data) i)) =
    None
  by blast
show ?thesis
  by (rule that[OF out-eq len in-Q map-eq lookup fresh])
qed

lemma verifier-header-transcript-from-prefix-read:
  assumes len-trace: length f-roots = ceil-log clength
    and len-alpha: length as = length spec
    and len-comp: length comp-roots = ceil-log (to-nat dg + 1)
    and transcript-prefix:
      PTranscript s =
        [fr] @ f-roots @ [f-final] @ as @ [dg] @
        comp-roots @ PTranscript prefix-state
    and read-prefix: PTranscript prefix-state = final # rest
    and query-rest: PTranscript query-state = rest
  shows
    verifier-header-transcript s fr f-roots f-final as dg
      comp-roots final (PTranscript query-state)
proof -
  have transcript:
    PTranscript s =
      verifier-header-messages fr f-roots f-final as dg comp-roots final @
      PTranscript query-state
  using transcript-prefix read-prefix query-rest
  unfolding verifier-header-messages-def
  by simp
show ?thesis
  unfolding verifier-header-transcript-def
  using transcript len-trace len-alpha len-comp by simp
qed

```

```

lemma verifier-composition-fri-prefix-outcome-basicD:
  assumes outcome:
    Some ((fr, f-fl, f-final, as, dg, fl), t) ∈
      set-dist (execute verifier-composition-fri-prefix s)
  shows
    length f-fl = ceil-log clength ∧
    length as = length spec ∧
    length fl = ceil-log (to-nat dg + 1) ∧
    PTranscript s =
      [fr] @ map snd f-fl @ [f-final] @ as @ [dg] @
      map snd fl @ PTranscript t ∧
    PState t =
      foldl concat

```

```

    (concat
      (foldl concat
        (concat
          (foldl concat (concat (PState s) fr) (map snd f-fl))
            f-final)
          as)
      dg)
    (map snd fl) ∧
    PQueryCounter t = PQueryCounter s
proof –
  have full: length f-fl = ceil-log clength ∧
    length as = length spec ∧
    length fl = ceil-log (to-nat dg + 1) ∧
    PTranscript s =
      [fr] @ map snd f-fl @ [f-final] @ as @ [dg] @
      map snd fl @ PTranscript t ∧
    PState t =
      foldl concat
        (concat
          (foldl concat
            (concat
              (foldl concat (concat (PState s) fr) (map snd f-fl))
                f-final)
              as)
          dg)
        (map snd fl) ∧
    s ≤ t ∧
    PQueryCounter t = PQueryCounter s ∧
    (∀ i < length f-fl.
      fmllookup (HashMap t)
        (TraceFriChallenge (PTraceFriCounter s + i)
          (foldl concat (concat (PState s) fr)
            (take (Suc i) (map snd f-fl)))) =
        Some (fst (f-fl ! i))) ∧
    (∀ i < length fl.
      fmllookup (HashMap t)
        (CompositionFriChallenge (PCompositionFriCounter s + i)
          (foldl concat
            (concat
              (foldl concat
                (concat
                  (foldl concat (concat (PState s) fr) (map snd f-fl))
                    f-final)
                  as)
              dg)
            (take (Suc i) (map snd fl)))) =
        Some (fst (fl ! i)))
  by (rule verifier-composition-fri-prefix-outcome[OF outcome])
then show ?thesis by simp

```

qed

lemma *alpha-round-preserves-query-lookup*:

assumes *outcome*: $\text{Some } (a, t) \in \text{set-dist } (\text{execute alpha-round } s)$

shows

$\text{fmlookup } (\text{HashMap } t) (\text{QueryIndexChallenge } i \ x) =$
 $\text{fmlookup } (\text{HashMap } s) (\text{QueryIndexChallenge } i \ x)$

proof –

from *outcome* **obtain** *a0 s0 a1 s1 s2* **where**

rand: $\text{Some } (a0, s0) \in \text{set-dist } (\text{execute receive-alpha-challenge } s)$

and *read-a*: $\text{Some } (a1, s1) \in \text{set-dist } (\text{execute read } s0)$

and *assert-a*: $\text{Some } ((), s2) \in \text{set-dist } (\text{execute } (\text{assert } (a0 = a1)) \ s1)$

and *ret-a*: $\text{Some } (a, t) \in \text{set-dist } (\text{execute } (\text{return } a1) \ s2)$

unfolding *alpha-round-def*

by (*auto simp: Let-def elim!: set-dist-bindE*)

have *neq*:

$\text{QueryIndexChallenge } i \ x \neq$

$\text{AlphaChallenge } (\text{PAlphaCounter } s) (\text{PState } s)$

by *simp*

have *rand-lookup*:

$\text{fmlookup } (\text{HashMap } s0) (\text{QueryIndexChallenge } i \ x) =$

$\text{fmlookup } (\text{HashMap } s) (\text{QueryIndexChallenge } i \ x)$

by (*rule receive-alpha-challenge-preserves-other-lookup[OF rand neq]*)

have *read-lookup*:

$\text{HashMap } s1 = \text{HashMap } s0$

by (*rule read-preserves-hash-map[OF read-a]*)

have *s2-eq*: $s2 = s1$

using *assert-a* **unfolding** *assert-def*

by (*cases a0 = a1*) (*auto simp: throw-no-outcome*)

have *t-eq*: $t = s2$

using *ret-a* **by** *simp*

show *?thesis*

using *rand-lookup read-lookup* **unfolding** *t-eq s2-eq* **by** *simp*

qed

lemma *receive-trace-fri-commits-preserves-query-lookup*:

assumes *outcome*:

$\text{Some } ((b, r), t) \in \text{set-dist } (\text{execute receive-trace-fri-commits } s)$

shows

$\text{fmlookup } (\text{HashMap } t) (\text{QueryIndexChallenge } i \ x) =$

$\text{fmlookup } (\text{HashMap } s) (\text{QueryIndexChallenge } i \ x)$

proof –

from *outcome* **obtain** *s-read* **where**

read-r: $\text{Some } (r, s\text{-read}) \in \text{set-dist } (\text{execute read } s)$

and *rand-b*:

$\text{Some } (b, t) \in \text{set-dist } (\text{execute receive-trace-fri-challenge } s\text{-read})$

unfolding *receive-trace-fri-commits-def receive-fri-commits-with-def*

by (*auto elim!: set-dist-bindE*)

have *read-lookup*:

```

    HashMap s-read = HashMap s
  by (rule read-preserves-hash-map[OF read-r])
have neq:
  QueryIndexChallenge i x ≠
    TraceFriChallenge (PTraceFriCounter s-read) (PState s-read)
  by simp
have rand-lookup:
  fmlookup (HashMap t) (QueryIndexChallenge i x) =
    fmlookup (HashMap s-read) (QueryIndexChallenge i x)
  by (rule receive-trace-fri-challenge-preserves-other-lookup
      [OF rand-b neq])
show ?thesis
  using rand-lookup read-lookup by simp
qed

lemma receive-composition-fri-commits-preserves-query-lookup:
  assumes outcome:
    Some ((b, r), t) ∈
      set-dist (execute receive-composition-fri-commits s)
  shows
    fmlookup (HashMap t) (QueryIndexChallenge i x) =
      fmlookup (HashMap s) (QueryIndexChallenge i x)
proof -
  from outcome obtain s-read where
    read-r: Some (r, s-read) ∈ set-dist (execute read s)
  and rand-b:
    Some (b, t) ∈
      set-dist (execute receive-composition-fri-challenge s-read)
  unfolding receive-composition-fri-commits-def receive-fri-commits-with-def
  by (auto elim!: set-dist-bindE)
have read-lookup:
  HashMap s-read = HashMap s
  by (rule read-preserves-hash-map[OF read-r])
have neq:
  QueryIndexChallenge i x ≠
    CompositionFriChallenge
      (PCompositionFriCounter s-read) (PState s-read)
  by simp
have rand-lookup:
  fmlookup (HashMap t) (QueryIndexChallenge i x) =
    fmlookup (HashMap s-read) (QueryIndexChallenge i x)
  by (rule receive-composition-fri-challenge-preserves-other-lookup
      [OF rand-b neq])
show ?thesis
  using rand-lookup read-lookup by simp
qed

lemma verifier-composition-fri-prefix-preserves-query-lookup:
  assumes outcome:

```

```

    Some ((fr, f-fl, f-final, as, dg, fl), t) ∈
      set-dist (execute verifier-composition-fri-prefix s)
  shows
    fmlookup (HashMap t) (QueryIndexChallenge i x) =
      fmlookup (HashMap s) (QueryIndexChallenge i x)
  proof -
    let ?key = QueryIndexChallenge i x
  from outcome obtain s1 s2 s3 s4 s5 s6 s7 where
    read-fr: Some (fr, s1) ∈ set-dist (execute read s)
  and trace-fri:
    Some (f-fl, s2) ∈
      set-dist (execute
        (ntimes receive-trace-fri-commits (ceil-log clength)) s1)
  and read-f-final: Some (f-final, s3) ∈ set-dist (execute read s2)
  and alpha-out:
    Some (as, s4) ∈
      set-dist (execute (mmap (replicate (length spec) alpha-round)) s3)
  and read-dg: Some (dg, s5) ∈ set-dist (execute read s4)
  and degree-assert:
    Some ((), s6) ∈
      set-dist (execute (assert (to-nat dg ≤ maxDegree)) s5)
  and comp-fri:
    Some (fl, s7) ∈
      set-dist (execute
        (ntimes receive-composition-fri-commits
          (ceil-log (to-nat dg + 1)))) s6)
  and ret:
    Some ((fr, f-fl, f-final, as, dg, fl), t) ∈
      set-dist (execute (return (fr, f-fl, f-final, as, dg, fl)) s7)
  unfolding verifier-composition-fri-prefix-def
  by (auto elim!: set-dist-bindE)
  have lookup-s1:
    fmlookup (HashMap s1) ?key = fmlookup (HashMap s) ?key
  using read-preserves-hash-map[OF read-fr] by simp
  have lookup-s2:
    fmlookup (HashMap s2) ?key = fmlookup (HashMap s1) ?key
  proof (rule ntimes-preserves-lookup[OF - trace-fri])
    fix y u v
    assume out: Some (y, v) ∈ set-dist (execute receive-trace-fri-commits u)
    obtain b r where y-eq: y = (b, r)
    by (cases y) simp
    show fmlookup (HashMap v) ?key = fmlookup (HashMap u) ?key
    by (rule receive-trace-fri-commits-preserves-query-lookup
      [OF out[unfolded y-eq]])
  qed
  have lookup-s3:
    fmlookup (HashMap s3) ?key = fmlookup (HashMap s2) ?key
  using read-preserves-hash-map[OF read-f-final] by simp
  have lookup-s4:

```

```

  fmlookup (HashMap s4) ?key = fmlookup (HashMap s3) ?key
proof (rule mmap-preserves-lookup[OF - alpha-out])
  fix m y u v
  assume m-in: m ∈ set (replicate (length spec) alpha-round)
  and out: Some (y, v) ∈ set-dist (execute m u)
  then have m-eq: m = alpha-round
  by simp
  show fmlookup (HashMap v) ?key = fmlookup (HashMap u) ?key
  using alpha-round-preserves-query-lookup[OF out[unfolded m-eq]] .
qed
have lookup-s5:
  fmlookup (HashMap s5) ?key = fmlookup (HashMap s4) ?key
  using read-preserves-hash-map[OF read-dg] by simp
have s6-eq: s6 = s5
  using degree-assert unfolding assert-def
  by (cases to-nat dg ≤ maxDegree) (auto simp: throw-no-outcome)
have lookup-s7:
  fmlookup (HashMap s7) ?key = fmlookup (HashMap s6) ?key
proof (rule ntimes-preserves-lookup[OF - comp-fri])
  fix y u v
  assume out: Some (y, v) ∈
    set-dist (execute receive-composition-fri-commits u)
  obtain b r where y-eq: y = (b, r)
  by (cases y) simp
  show fmlookup (HashMap v) ?key = fmlookup (HashMap u) ?key
  by (rule receive-composition-fri-commits-preserves-query-lookup
    [OF out[unfolded y-eq]])
qed
have t-eq: t = s7
  using ret by simp
show ?thesis
  using lookup-s1 lookup-s2 lookup-s3 lookup-s4 lookup-s5 lookup-s7
  unfolding t-eq s6-eq by simp
qed

lemma query-index-raw-singleton-card-uniform:
  assumes b-in: b ∈ query-sample-space
  shows card (query-index-raw-preimage {b}) =
    size div query-sample-space-size
proof -
  have subset: {b} ⊆ query-sample-space
  using b-in by simp
  have card (query-index-raw-preimage {b}) =
    (size div query-sample-space-size) * card {b}
  unfolding card-query-index-raw-preimage-eq-nat-preimage
  apply (rule card-query-index-nat-preimage-uniform-range)
  apply (rule to-nat-range)
  using query-sample-space-size-dvd subset
  unfolding query-sample-space-size-def by simp-all

```

```

then show ?thesis
  by simp
qed

lemma query-index-row-list-fixed-preimage-card-bound:
  assumes len: length query-idxs = rounds
  and subset: set query-idxs  $\subseteq$  query-sample-space
  shows card {rows.
    length rows = rounds  $\wedge$ 
    map ( $\lambda$ raw. index (to-nat raw)) rows = query-idxs}
     $\leq$  (size div query-sample-space-size)  $\wedge$  rounds
  using len subset
proof (induction rounds arbitrary: query-idxs)
  case 0
  then have {rows.
    length rows = 0  $\wedge$ 
    map ( $\lambda$ raw. index (to-nat raw)) rows = query-idxs} = {}
  by auto
  then show ?case by simp
next
  case (Suc n)
  obtain q qs where query-eq: query-idxs = q # qs
  using Suc.premis(1) by (cases query-idxs) auto
  have len-qs: length qs = n
  using Suc.premis(1) unfolding query-eq by simp
  have q-in: q  $\in$  query-sample-space
  using Suc.premis(2) unfolding query-eq by simp
  have qs-subset: set qs  $\subseteq$  query-sample-space
  using Suc.premis(2) unfolding query-eq by simp
  let ?A = {rows.
    length rows = Suc n  $\wedge$ 
    map ( $\lambda$ raw. index (to-nat raw)) rows = q # qs}
  let ?B = query-index-row-preimage {q}
  let ?C = {rows.
    length rows = n  $\wedge$ 
    map ( $\lambda$ raw. index (to-nat raw)) rows = qs}
  have A-eq: ?A = ( $\lambda$ (raw, rows). raw # rows) ' (?B  $\times$  ?C)
  proof
    show ?A  $\subseteq$  ( $\lambda$ (raw, rows). raw # rows) ' (?B  $\times$  ?C)
  proof
    fix rows
    assume rows-in: rows  $\in$  ?A
    then obtain raw tail where rows-eq: rows = raw # tail
    by (cases rows) auto
    have raw-in: raw  $\in$  ?B
    using rows-in unfolding rows-eq query-index-row-preimage-def by simp
    have tail-in: tail  $\in$  ?C
    using rows-in unfolding rows-eq by simp
    show rows  $\in$  ( $\lambda$ (raw, rows). raw # rows) ' (?B  $\times$  ?C)
  end
end

```



```

    unfolding rows-eq by (intro image-eqI[of - - (raw, tail)])
      (use raw-in tail-in in simp-all)
  qed
next
  show  $(\lambda(\text{raw}, \text{rows}). \text{raw} \# \text{rows}) \text{ ' } (?B \times ?C) \subseteq ?A$ 
    unfolding query-index-raw-preimage-def by auto
  qed
  have inj: inj-on  $(\lambda(\text{raw}, \text{rows}). \text{raw} \# \text{rows}) (?B \times ?C)$ 
    unfolding inj-on-def by auto
  have finite-B: finite ?B
    by simp
  have finite-C: finite ?C
  proof -
    have  $?C \subseteq \{xs. \text{length } xs = n\}$ 
      by auto
    then show ?thesis
      by (auto intro: List.finite-list-length)
  qed
  have card ?A = card  $((\lambda(\text{raw}, \text{rows}). \text{raw} \# \text{rows}) \text{ ' } (?B \times ?C))$ 
    unfolding A-eq by simp
  also have ... = card  $(?B \times ?C)$ 
    by (rule card-image[OF inj])
  also have ... = card ?B * card ?C
    using finite-B finite-C by simp
  also have ...  $\leq$ 
     $(\text{size div query-sample-space-size}) * (\text{size div query-sample-space-size}) ^ n$ 
  proof (rule mult-mono)
    show card ?B  $\leq \text{size div query-sample-space-size}$ 
      using query-index-raw-singleton-card-uniform[OF q-in] by simp
    show card ?C  $\leq (\text{size div query-sample-space-size}) ^ n$ 
      by (rule Suc.IH[OF len-qs qs-subset])
    show  $0 \leq \text{size div query-sample-space-size}$  by simp
    show  $0 \leq \text{card } ?C$  by simp
  qed
  also have ... =  $(\text{size div query-sample-space-size}) ^ \text{Suc } n$ 
    by simp
  finally show ?case
    unfolding query-eq .
  qed

lemma query-index-raw-list-preimage-card-bound:
  assumes subset:  $Q \subseteq \text{fri-query-index-list-space}$ 
  shows card  $(\text{query-index-raw-list-preimage } Q)$ 
     $\leq \text{card } Q * (\text{size div query-sample-space-size}) ^ \text{rounds}$ 
  proof -
    let ?fiber =  $\lambda \text{query-idxs}. \{\text{rows}.$ 
       $\text{length } \text{rows} = \text{rounds} \wedge$ 
       $\text{map } (\lambda \text{raw}. \text{index } (\text{to-nat } \text{raw})) \text{ rows} = \text{query-idxs}\}$ 

```

```

have finite-Q: finite Q
  using subset by (rule finite-subset) (rule finite-fri-query-index-list-space)
have preimage-eq:
  query-index-raw-list-preimage Q = ( $\bigcup$  query-idxs  $\in$  Q. ?fiber query-idxs)
  unfolding query-index-raw-list-preimage-def by auto
have finite-fiber:  $\bigwedge$  query-idxs. finite (?fiber query-idxs)
proof -
  fix query-idxs
  have ?fiber query-idxs  $\subseteq$  {xs. length xs = rounds}
    by auto
  then show finite (?fiber query-idxs)
    by (auto intro: List.finite-list-length)
qed
have card (query-index-raw-list-preimage Q) =
  card ( $\bigcup$  query-idxs  $\in$  Q. ?fiber query-idxs)
  unfolding preimage-eq ..
also have ...  $\leq$  ( $\sum$  query-idxs  $\in$  Q. card (?fiber query-idxs))
  using finite-Q by (auto intro: card-UN-le)
also have ...  $\leq$ 
  ( $\sum$  query-idxs  $\in$  Q.
    (size div query-sample-space-size) ^ rounds)
proof (rule sum-mono)
  fix query-idxs
  assume query-in: query-idxs  $\in$  Q
  have len: length query-idxs = rounds
    using query-in subset unfolding fri-query-index-list-space-def by auto
  have entries: set query-idxs  $\subseteq$  query-sample-space
    using query-in subset unfolding fri-query-index-list-space-def by auto
  show card (?fiber query-idxs)
     $\leq$  (size div query-sample-space-size) ^ rounds
    by (rule query-index-raw-list-fixed-preimage-card-bound[OF len entries])
qed
also have ... = card Q * (size div query-sample-space-size) ^ rounds
  using finite-Q by simp
finally show ?thesis .
qed

lemma query-index-raw-list-preimage-probability-bound:
  assumes subset: Q  $\subseteq$  fri-query-index-list-space
  shows
    nnreal (card (query-index-raw-list-preimage Q)) *
      (1 / nnreal size) ^ rounds
     $\leq$  nnreal (card Q) *
      (1 / nnreal (card query-sample-space)) ^ rounds
proof -
  let ?d = size div query-sample-space-size
  have card-le:
    card (query-index-raw-list-preimage Q)
       $\leq$  card Q * ?d ^ rounds

```

```

    by (rule query-index-raw-list-preimage-card-bound[OF subset])
  have dvd: query-sample-space-size dvd size
    using query-sample-space-size-dvd
    unfolding query-sample-space-size-def by simp
  have size-eq: size = ?d * query-sample-space-size
    using dvd query-sample-space-size-pos by (simp add: dvd-eq-mod-eq-0)
  have size-pos: 0 < size
    using size-card by simp
  have d-pos: 0 < ?d
  proof (cases ?d)
    case 0
    then show ?thesis
      using size-eq size-pos by simp
  next
    case (Suc n)
    then show ?thesis by simp
  qed
  have le:
    nnreal (card (query-index-raw-list-preimage Q)) *
      (1 / nnreal size) ^ rounds
    ≤ nnreal (card Q * ?d ^ rounds) *
      (1 / nnreal size) ^ rounds
    using card-le apply (simp add: mult-right-mono)
    by (metis mult-right-mono of-nat-le-iff of-nat-mult of-nat-power zero-least)
  have prod-eq:
    nnreal (card Q * ?d ^ rounds) *
      (1 / nnreal size) ^ rounds =
    nnreal (card Q) *
      (1 / nnreal (card query-sample-space)) ^ rounds
  proof -
    have card-space-eq:
      card query-sample-space = query-sample-space-size
    by (rule card-query-sample-space)
    have base-eq:
      nnreal ?d * (1 / nnreal size) =
      1 / nnreal (card query-sample-space)
    proof (rule nn2real-eq-iff[THEN iffD1])
      have real-base:
        real ?d / real size = 1 / real query-sample-space-size
      proof -
        have real-size-eq:
          real size = real ?d * real query-sample-space-size
        using arg-cong[OF size-eq, of real]
        by (simp add: of-nat-mult)
        have real ?d / real size =
          real ?d / (real ?d * real query-sample-space-size)
        using real-size-eq by simp
        also have ... = 1 / real query-sample-space-size
        using d-pos query-sample-space-size-pos

```

```

    by (simp add: field-simps)
  finally show ?thesis .
qed
show nn2real (nnreal ?d * (1 / nnreal size)) =
  nn2real (1 / nnreal (card query-sample-space))
proof -
  have lhs-eq:
    nn2real (nnreal ?d * (1 / nnreal size)) =
      real ?d / real size
  proof -
    have nn2real (nnreal ?d * (1 / nnreal size)) =
      real ?d * (1 / real size)
    by (simp only: nn2real-mult nn2real-nnreal
      nn2real-divide nn2real-1)
    also have ... = real ?d / real size
    by (simp add: divide-inverse)
    finally show ?thesis .
  qed
  have rhs-eq:
    nn2real (1 / nnreal (card query-sample-space)) =
      1 / real query-sample-space-size
  unfolding card-space-eq
  by (simp only: nn2real-divide nn2real-1 nn2real-nnreal)
show ?thesis
using real-base lhs-eq rhs-eq by simp
qed
qed
have
  nnreal (card Q * ?d ^ rounds) * (1 / nnreal size) ^ rounds =
  nnreal (card Q) * (nnreal ?d * (1 / nnreal size)) ^ rounds
by (simp add: of-nat-mult of-nat-power power-mult-distrib
  ac-simps)
also have ... =
  nnreal (card Q) *
  (1 / nnreal (card query-sample-space)) ^ rounds
using base-eq by simp
finally show ?thesis .
qed
show ?thesis
using le prod-eq by simp
qed

lemma wp-ntimes-verifier-query-round-program-query-list-path-product-bound:
  assumes subset:  $Q \subseteq \text{fri-query-index-list-space}$ 
  shows
    wp-event
      (ntimes (verifier-query-round-program fr f-fl f-final as fl final)
        rounds)
      ( $\lambda \text{out}. \exists \text{raws} \in \text{query-index-raw-list-preimage } Q.$ 

```

```

      query-rounds-raw-list-path-hit s raws rounds out)
    s ≤
    nnreal (card Q) *
    (1 / nnreal (card query-sample-space)) ^ rounds
  proof -
    have raw-bound:
      wp-event
      (ntimes (verifier-query-round-program fr f-fl f-final as fl final)
       rounds)
      (λout. ∃ raws ∈ query-index-raw-list-preimage Q.
       query-rounds-raw-list-path-hit s raws rounds out)
      s ≤
      nnreal (card (query-index-raw-list-preimage Q)) *
      (1 / nnreal size) ^ rounds
    by (rule wp-ntimes-verifier-query-round-program-raw-list-preimage-path-hit-bound)
  also have ... ≤
    nnreal (card Q) *
    (1 / nnreal (card query-sample-space)) ^ rounds
    by (rule query-index-raw-list-preimage-probability-bound[OF subset])
  finally show ?thesis .
qed

```

```

lemma read-outcome-unique-state:
  assumes out1: Some (x, t) ∈ set-dist (execute read s)
    and out2: Some (y, u) ∈ set-dist (execute read s)
  shows x = y ∧ t = u
proof -
  from read-outcome[OF out1] obtain rest where
    tr: PTranscript s = x # rest
  by blast
  have out1-eq:
    t = s(PState := concat (PState s) x, PTranscript := rest)
    using read-nonempty-outcome[OF out1 tr] by simp
  have out2-eq:
    y = x ∧
    u = s(PState := concat (PState s) x, PTranscript := rest)
    using read-nonempty-outcome[OF out2 tr] by simp
  then show ?thesis
    using out1-eq by simp
qed

```

```

lemma wp-verifier-after-composition-fri-raw-list-path-product-bound:
  assumes subset: Q ⊆ fri-query-index-list-space
  shows
    wp-event (verifier-after-composition-fri header)
    (verifier-after-composition-fri-raw-list-path-hit s Q) s ≤
    nnreal (card Q) *
    (1 / nnreal (card query-sample-space)) ^ rounds
proof -

```

```

obtain fr f-fl f-final as dg fl where header-eq:
  header = (fr, f-fl, f-final, as, dg, fl)
by (cases header) auto
have after-eq:
  verifier-after-composition-fri header =
    (read  $\gg$ 
      ( $\lambda$ final.
        ntimes
          (verifier-query-round-program fr f-fl f-final as fl final)
          rounds))
unfolding header-eq verifier-after-composition-fri-def by simp
show ?thesis
unfolding after-eq
proof (rule wp-event-bind-bound-by-cont)
show  $\neg$  verifier-after-composition-fri-raw-list-path-hit s Q None
unfolding verifier-after-composition-fri-raw-list-path-hit-def
  query-rounds-raw-list-path-hit-def
by simp
next
fix final query-state
assume read-final:
  Some (final, query-state)  $\in$  set-dist (execute read s)
have mono:
  wp-event
    (ntimes
      (verifier-query-round-program fr f-fl f-final as fl final)
      rounds)
    (verifier-after-composition-fri-raw-list-path-hit s Q)
    query-state  $\leq$ 
  wp-event
    (ntimes
      (verifier-query-round-program fr f-fl f-final as fl final)
      rounds)
    ( $\lambda$ out.  $\exists$  rows  $\in$  query-index-raw-list-preimage Q.
      query-rounds-raw-list-path-hit query-state rows rounds out)
    query-state
proof (rule wp-event-mono-on-support)
fix out
assume hit:
  verifier-after-composition-fri-raw-list-path-hit s Q out
from hit obtain final' query-state' rows where
  read':
    Some (final', query-state')  $\in$  set-dist (execute read s)
and raw-in: rows  $\in$  query-index-raw-list-preimage Q
and path:
  query-rounds-raw-list-path-hit query-state' rows rounds out
unfolding verifier-after-composition-fri-raw-list-path-hit-def
by blast
have query-state' = query-state

```

```

    using read-outcome-unique-state[OF read' read-final] by simp
  then show  $\exists \text{raws} \in \text{query-index-raw-list-preimage } Q.$ 
    query-rounds-raw-list-path-hit query-state raws rounds out
    using raw-in path by blast
qed
also have ...  $\leq$ 
  nnreal (card Q) *
  (1 / nnreal (card query-sample-space))  $\wedge$  rounds
  by (rule
    wp-ntimes-verifier-query-round-program-query-list-path-product-bound
    [OF subset])
finally show
  wp-event
  (ntimes
    (verifier-query-round-program fr f-fl f-final as fl final)
    rounds)
  (verifier-after-composition-fri-raw-list-path-hit s Q)
  query-state
 $\leq$  nnreal (card Q) *
  (1 / nnreal (card query-sample-space))  $\wedge$  rounds .
qed
qed

lemma verifier-after-composition-fri-query-index-list-path-fresh-imp-raw-hit-after-prefix:
assumes wf: staged-budget-wellformed budgets
and controlled: staged-adversary-controlled budgets A
and builder:
  Some (data, attacker-state)  $\in$ 
    set-dist
      (execute (checked-staged-transcript-program A)
        adversary-initial-state)
and prefix:
  Some (header, prefix-state)  $\in$ 
    set-dist
      (execute verifier-composition-fri-prefix
        (verifier-state-from-adversary attacker-state
          (staged-proof-transcript data)))
and suffix:
  out  $\in$  set-dist
    (execute (verifier-after-composition-fri header) prefix-state)
and hit:
  staged-security-with-data-state-query-index-list-path-fresh Q
  (case out of
    None  $\Rightarrow$  None
    | Some (result, final-state)  $\Rightarrow$ 
      Some (((data, attacker-state), result), final-state))
shows verifier-after-composition-fri-raw-list-path-hit prefix-state Q out
proof (cases out)
case None

```

```

then show ?thesis
  using hit by simp
next
case (Some result-state)
then obtain result final-state where
  out-eq: out = Some (result, final-state)
  by (cases result-state) simp
let ?s =
  verifier-state-from-adversary attacker-state
  (staged-proof-transcript data)
have hit-some:
  staged-security-with-data-state-query-index-list-path-fresh Q
  (Some (((data, attacker-state), result), final-state))
  using hit out-eq by simp
from staged-security-with-data-state-query-index-list-path-fresh-SomeE
  [OF hit-some]
obtain raw-idxs-hit query-idxs-hit where
  len-hit: length raw-idxs-hit = rounds
  and query-hit-in: query-idxs-hit ∈ Q
  and query-hit-eq:
    query-idxs-hit = map (λraw. index (to-nat raw)) raw-idxs-hit
  and lookup-hit:
    ∧ i. i < rounds ⇒
      fmllookup (HashMap final-state)
      (QueryIndexChallenge i
      (state-after-query-chunks
      (staged-query-start-hash data)
      (staged-query-chunks data) i)) =
      Some (raw-idxs-hit ! i)
  and fresh-hit:
    ∧ i. i < rounds ⇒
      fmllookup (HashMap ?s)
      (QueryIndexChallenge i
      (state-after-query-chunks
      (staged-query-start-hash data)
      (staged-query-chunks data) i)) =
      None
  by blast
obtain fr f-fl f-final as dg fl where header-eq:
  header = (fr, f-fl, f-final, as, dg, fl)
  by (cases header) auto
from suffix[unfolded out-eq header-eq verifier-after-composition-fri-def]
obtain final query-state where
  read-final:
    Some (final, query-state) ∈ set-dist (execute read prefix-state)
  and query-out:
    Some (result, final-state) ∈
    set-dist
    (execute

```



```

      (ntimes
        (verifier-query-round-program fr f-fl f-final as fl final)
        rounds)
      query-state)
  by (auto elim!: set-dist-bindE)
from ntimes-verifier-query-rounds-outcome[OF query-out]
obtain raw-idxs query-idxs query-chunks where
  len-raw: length raw-idxs = rounds
  and query-idxs-eq:
    query-idxs = map (λraw. index (to-nat raw)) raw-idxs
  and len-chunks: length query-chunks = rounds
  and tr-query:
    PTranscript query-state =
      List.concat query-chunks @ PTranscript final-state
  and state-final:
    PState final-state =
      state-after-query-chunks (PState query-state) query-chunks rounds
  and round-chunks:
    ∧ i. i < rounds ⇒
      verifier-query-round-chunk (query-idxs ! i)
      (map snd f-fl) (map snd fl) (query-chunks ! i)
  and lookups:
    ∧ i. i < rounds ⇒
      fmllookup (HashMap final-state)
      (QueryIndexChallenge (PQueryCounter query-state + i)
        (state-after-query-chunks
          (PState query-state) query-chunks i)) =
        Some (raw-idxs ! i)
  by blast
from read-outcome[OF read-final] have query-counter:
  PQueryCounter query-state = PQueryCounter prefix-state
  by blast
have prefix-basic:
  length f-fl = ceil-log clength ∧
  length as = length spec ∧
  length fl = ceil-log (to-nat dg + 1) ∧
  PTranscript ?s =
    [fr] @ map snd f-fl @ [f-final] @ as @ [dg] @
    map snd fl @ PTranscript prefix-state ∧
  PState prefix-state =
    foldl concat
      (concat
        (foldl concat
          (concat
            (foldl concat (concat (PState ?s) fr) (map snd f-fl))
            f-final)
          as)
        dg)
      (map snd fl) ∧

```

```

    PQueryCounter prefix-state = PQueryCounter ?s
  by (rule verifier-composition-fri-prefix-outcome-basicD
      [OF prefix[unfolded header-eq]])
  have prefix-counter: PQueryCounter prefix-state = PQueryCounter ?s
    using prefix-basic by simp
  have local-counter: PQueryCounter query-state = 0
    using query-counter prefix-counter
    unfolding verifier-state-from-adversary-def by simp
  have prefix-res:
    length f-fl = ceil-log clength ∧
    length as = length spec ∧
    length fl = ceil-log (to-nat dg + 1) ∧
    PTranscript ?s =
      [fr] @ map snd f-fl @ [f-final] @ as @ [dg] @
      map snd fl @ PTranscript prefix-state ∧
    PState prefix-state =
      foldl concat
        (concat
          (foldl concat
            (concat
              (foldl concat (concat (PState ?s) fr) (map snd f-fl))
              f-final)
            as)
          dg)
        (map snd fl)
    using prefix-basic by simp
  from read-outcome[OF read-final] obtain rest-query where
    tr-prefix: PTranscript prefix-state = final # rest-query
  and state-query:
    PState query-state = concat (PState prefix-state) final
  and tr-read-query: PTranscript query-state = rest-query
  by blast
  have lookup-query:
    ∧i x. fmllookup (HashMap query-state)
      (QueryIndexChallenge i x) =
      fmllookup (HashMap ?s) (QueryIndexChallenge i x)
  proof -
    have query-state-eq:
      query-state =
        prefix-state(PState := concat (PState prefix-state) final,
          PTranscript := rest-query)
    using read-nonempty-outcome[OF read-final tr-prefix] by simp
  have hash-prefix:
    ∧i x. fmllookup (HashMap prefix-state)
      (QueryIndexChallenge i x) =
      fmllookup (HashMap ?s) (QueryIndexChallenge i x)
  by (rule verifier-composition-fri-prefix-preserves-query-lookup
      [OF prefix[unfolded header-eq]])
  show ∧i x. fmllookup (HashMap query-state)

```

```

    (QueryIndexChallenge i x) =
      fmllookup (HashMap ?s) (QueryIndexChallenge i x)
    using query-state-eq hash-prefix by simp
  qed
  have len-trace-actual: length (map snd f-fl) = ceil-log clength
    using prefix-res by simp
  have len-alpha-actual: length as = length spec
    using prefix-res by simp
  have len-comp-actual:
    length (map snd fl) = ceil-log (to-nat dg + 1)
    using prefix-res by simp
  have transcript-prefix-actual:
    PTranscript ?s =
      [fr] @ map snd f-fl @ [f-final] @ as @ [dg] @
      map snd fl @ PTranscript prefix-state
    using prefix-res by simp
  have header-actual:
    verifier-header-transcript ?s fr (map snd f-fl) f-final as dg
      (map snd fl) final (PTranscript query-state)
    by (rule verifier-header-transcript-from-prefix-read
      [OF len-trace-actual len-alpha-actual len-comp-actual
        transcript-prefix-actual tr-prefix tr-read-query])
  have staged-header:
    verifier-header-transcript ?s
      (staged-trace-root data)
      (staged-trace-fri-roots data)
      (staged-trace-final data)
      (staged-alphas data)
      (staged-degree data)
      (staged-composition-fri-roots data)
      (staged-composition-final data)
      (List.concat (staged-query-chunks data))
    by (rule checked-staged-transcript-program-outcome-header-transcript
      [OF wf controlled builder])
  have header-unique:
    fr = staged-trace-root data ∧
    map snd f-fl = staged-trace-fri-roots data ∧
    f-final = staged-trace-final data ∧
    as = staged-alphas data ∧
    dg = staged-degree data ∧
    map snd fl = staged-composition-fri-roots data ∧
    final = staged-composition-final data ∧
    PTranscript query-state = List.concat (staged-query-chunks data)
    using verifier-header-transcript-unique[OF header-actual staged-header]
    by simp
  have prefix-state-query:
    PState query-state = staged-query-start-hash data
    using state-query prefix-res header-unique
    unfolding staged-query-start-hash-def

```

```

    staged-composition-fri-start-hash-def
    staged-trace-fri-start-hash-def
  by simp
have chunks-eq: query-chunks = staged-query-chunks data
proof -
  from checked-staged-transcript-program-query-chunks-match-verifier-lengths
    [OF builder]
  obtain staged-query-idxs where
    staged-match:
      staged-query-chunks-match-verifier-lengths data staged-query-idxs
  by blast
have query-idxs-len: length query-idxs = rounds
  using query-idxs-eq len-raw by simp
have staged-match-query-idxs:
  staged-query-chunks-match-verifier-lengths data query-idxs
  by (rule staged-query-chunks-match-verifier-lengths-transfer
    [OF staged-match query-idxs-len])
have parser-chunk-len:
   $\bigwedge i. i < rounds \implies$ 
    length (query-chunks ! i) =
      verifier-query-round-transcript-length (query-idxs ! i)
      (staged-trace-fri-roots data)
      (staged-composition-fri-roots data)
proof -
  fix i
  assume i-bound: i < rounds
  show
    length (query-chunks ! i) =
      verifier-query-round-transcript-length (query-idxs ! i)
      (staged-trace-fri-roots data)
      (staged-composition-fri-roots data)
  using verifier-query-round-chunk-length
    [OF round-chunks[OF i-bound]] header-unique
  by simp
qed
have concat-staged:
  List.concat query-chunks @ PTranscript final-state =
    List.concat (staged-query-chunks data)
  using tr-query header-unique by simp
then show ?thesis
  using query-chunks-eq-staged-if-matching-lengths
    [OF len-chunks concat-staged parser-chunk-len staged-match-query-idxs]
  by simp
qed
have raw-eq:
   $\bigwedge i. i < rounds \implies$  raw-idxs ! i = raw-idxs-hit ! i
  using lookups lookup-hit local-counter prefix-state-query chunks-eq
  by simp
have raw-idxs-eq: raw-idxs = raw-idxs-hit

```

```

    by (rule nth-equalityI)
      (use len-raw len-hit raw-eq in simp-all)
  have raw-in: raw-idxs  $\in$  query-index-raw-list-preimage Q
    unfolding query-index-raw-list-preimage-def
    using len-raw raw-idxs-eq query-hit-in query-hit-eq by blast
  have local-fresh:
    query-index-list-path-fresh query-state rounds query-chunks raw-idxs
    unfolding query-index-list-path-fresh-def
    using len-raw len-chunks fresh-hit local-counter prefix-state-query
      chunks-eq lookup-query
    by simp
  show ?thesis
    by (rule verifier-after-composition-fri-raw-list-path-hitI
      [where final=final and query-state=query-state and raws=raw-idxs
        and chunks=query-chunks and t=final-state and results=result,
        OF read-final raw-in len-raw len-chunks state-final local-fresh
        lookups out-eq])
qed

lemma verifier-after-composition-fri-query-index-list-path-fresh-product-bound-after-prefix:
  assumes wf: staged-budget-wellformed budgets
    and controlled: staged-adversary-controlled budgets A
    and subset:  $Q \subseteq \text{fri-query-index-list-space}$ 
    and builder:
      Some (data, attacker-state)  $\in$ 
        set-dist
          (execute (checked-staged-transcript-program A)
            adversary-initial-state)
    and prefix:
      Some (header, prefix-state)  $\in$ 
        set-dist
          (execute verifier-composition-fri-prefix
            (verifier-state-from-adversary attacker-state
              (staged-proof-transcript data))))
  shows
    wp-event (verifier-after-composition-fri header)
      ( $\lambda$ out.
        staged-security-with-data-state-query-index-list-path-fresh Q
          (case out of
            None  $\Rightarrow$  None
            | Some (result, final-state)  $\Rightarrow$ 
              Some (((data, attacker-state), result), final-state)))
        prefix-state
         $\leq$  nnreal (card Q) *
          (1 / nnreal (card query-sample-space))  $\wedge$  rounds
  proof -
    let ?s =
      verifier-state-from-adversary attacker-state
        (staged-proof-transcript data)

```

```

let ?P =
   $\lambda out.$ 
    staged-security-with-data-state-query-index-list-path-fresh Q
    (case out of
      None  $\Rightarrow$  None
      | Some (result, final-state)  $\Rightarrow$ 
        Some (((data, attacker-state), result), final-state))
have event-mono:
  wp-event (verifier-after-composition-fri header)
    ?P prefix-state
 $\leq$ 
  wp-event (verifier-after-composition-fri header)
    (verifier-after-composition-fri-raw-list-path-hit prefix-state Q)
    prefix-state
proof (rule wp-event-mono-on-support)
fix out
assume suffix:
  out  $\in$ 
    set-dist
    (execute (verifier-after-composition-fri header) prefix-state)
and hit:
  staged-security-with-data-state-query-index-list-path-fresh Q
  (case out of
    None  $\Rightarrow$  None
    | Some (result, final-state)  $\Rightarrow$ 
      Some (((data, attacker-state), result), final-state))
show verifier-after-composition-fri-raw-list-path-hit prefix-state Q out
by (rule
  verifier-after-composition-fri-query-index-list-path-fresh-imp-raw-hit-after-prefix
  [OF wf controlled builder prefix suffix hit])
qed
also have ...  $\leq$ 
  nnreal (card Q) *
  (1 / nnreal (card query-sample-space))  $\wedge$  rounds
by (rule wp-verifier-after-composition-fri-raw-list-path-product-bound
  [OF subset])
finally show ?thesis .
qed

lemma verify-monad-query-index-list-path-fresh-product-bound-after-builder:
assumes wf: staged-budget-wellformed budgets
and controlled: staged-adversary-controlled budgets A
and subset:  $Q \subseteq \text{fri-query-index-list-space}$ 
and builder:
  Some (data, attacker-state)  $\in$ 
    set-dist
    (execute (checked-staged-transcript-program A)
      adversary-initial-state)
shows

```

```

wp-event verify-monad
  (λout.
    staged-security-with-data-state-query-index-list-path-fresh Q
      (case out of
        None ⇒ None
        | Some (result, final-state) ⇒
          Some (((data, attacker-state), result), final-state)))
    (verifier-state-from-adversary attacker-state
      (staged-proof-transcript data))
    ≤ nnreal (card Q) *
      (1 / nnreal (card query-sample-space)) ^ rounds
proof -
  let ?s =
    verifier-state-from-adversary attacker-state
      (staged-proof-transcript data)
note verify-decomp = verify-monad-composition-fri-decomposition
show ?thesis
  unfolding verify-decomp
proof (rule wp-event-bind-bound-by-cont)
show ¬ staged-security-with-data-state-query-index-list-path-fresh Q
  (case None of None ⇒ None
  | Some (result, final-state) ⇒
    Some (((data, attacker-state), result), final-state))
  by simp
next
fix header prefix-state
assume prefix:
  Some (header, prefix-state) ∈
    set-dist (execute verifier-composition-fri-prefix ?s)
show
  wp-event (verifier-after-composition-fri header)
    (λout.
      staged-security-with-data-state-query-index-list-path-fresh Q
        (case out of
          None ⇒ None
          | Some (result, final-state) ⇒
            Some (((data, attacker-state), result), final-state)))
      prefix-state
    ≤ nnreal (card Q) *
      (1 / nnreal (card query-sample-space)) ^ rounds
  by (rule
    verifier-after-composition-fri-query-index-list-path-fresh-product-bound-after-prefix
    [OF wf controlled subset builder prefix])
qed
qed

lemma checked-staged-security-with-data-state-query-index-list-path-fresh-product-bound:
assumes wf: staged-budget-wellformed budgets
and controlled: staged-adversary-controlled budgets A

```

and *subset*: $Q \subseteq \text{fri-query-index-list-space}$
shows
 $\text{wp-event } (\text{checked-staged-security-experiment-with-data-state } A)$
 $(\text{staged-security-with-data-state-query-index-list-path-fresh } Q)$
 $\text{adversary-initial-state} \leq$
 $\text{nnreal } (\text{card } Q) *$
 $(1 / \text{nnreal } (\text{card } \text{query-sample-space})) \wedge \text{rounds}$
proof –
let $?P = \text{staged-security-with-data-state-query-index-list-path-fresh } Q$
have *none*: $\neg ?P \text{ None}$
by *simp*
have *cont-bound*:
 $\wedge \text{data attacker-state.}$
 $\text{Some } (\text{data}, \text{attacker-state}) \in$
 set-dist
 $(\text{execute } (\text{checked-staged-transcript-program } A)$
 $\text{adversary-initial-state}) \implies$
 $\text{wp-event verify-monad}$
 $(\lambda \text{out.}$
 $?P$
 $(\text{case out of}$
 $\text{None} \implies \text{None}$
 $| \text{Some } (\text{result}, \text{final-state}) \implies$
 $\text{Some } (((\text{data}, \text{attacker-state}), \text{result}), \text{final-state})))$
 $(\text{verifier-state-from-adversary attacker-state}$
 $(\text{staged-proof-transcript data}))$
 $\leq \text{nnreal } (\text{card } Q) *$
 $(1 / \text{nnreal } (\text{card } \text{query-sample-space})) \wedge \text{rounds}$
by (*rule verify-monad-query-index-list-path-fresh-product-bound-after-builder*
 $[\text{OF wf controlled subset}]$)
show *?thesis*
by (*rule checked-staged-security-with-data-state-bound-from-data-cont*
 $[\text{OF none cont-bound}]$)
qed

lemma *checked-staged-security-with-data-state-query-index-list-hit-exact-product-bound*:
assumes *wf*: *staged-budget-wellformed budgets*
and *controlled*: *staged-adversary-controlled budgets A*
and *subset*: $Q \subseteq \text{fri-query-index-list-space}$
shows
 $\text{wp-event } (\text{checked-staged-security-experiment-with-data-state } A)$
 $(\text{staged-security-with-data-query-index-list-set-hit } Q)$
 $\text{adversary-initial-state} \leq$
 $\text{nnreal } (\text{card } Q) *$
 $(1 / \text{nnreal } (\text{card } \text{query-sample-space})) \wedge \text{rounds} +$
 $\text{hash-relation-budget-value}$
 $(\text{query-index-raw-list-relation-fiber-bound } Q)$
 $(\text{staged-attacker-query-budget budgets} +$
 $\text{staged-challenge-query-budget})$

by (rule
checked-staged-security-with-data-state-query-index-list-hit-bound-from-path-fresh
 [OF wf controlled
checked-staged-security-with-data-state-query-index-list-path-fresh-product-bound
 [OF wf controlled subset]])

lemma *checked-staged-security-trace-fri-query-index-list-set-hit-exact-product-bound:*

assumes wf: *staged-budget-wellformed budgets*
and controlled: *staged-adversary-controlled budgets A*
and subset: $Q \subseteq \text{fri-query-index-list-space}$

shows

wp-event (checked-staged-security-experiment-with-data-state A)
(staged-security-with-data-state-verifier-event
($\lambda s. \text{trace-fri-query-index-list-set-hit } s \ Q$))
adversary-initial-state $\leq$
*nnreal (card Q) **
(1 / nnreal (card query-sample-space)) $\wedge$ rounds +
hash-relation-budget-value
(query-index-raw-list-relation-fiber-bound Q)
(staged-attacker-query-budget budgets +
staged-challenge-query-budget)

by (rule
checked-staged-security-trace-fri-query-index-list-set-hit-bound-from-path-fresh
 [OF wf controlled
checked-staged-security-with-data-state-query-index-list-path-fresh-product-bound
 [OF wf controlled subset]])

lemma *checked-staged-security-composition-fri-query-index-list-set-hit-exact-product-bound:*

assumes wf: *staged-budget-wellformed budgets*
and controlled: *staged-adversary-controlled budgets A*
and subset: $Q \subseteq \text{fri-query-index-list-space}$

shows

wp-event (checked-staged-security-experiment-with-data-state A)
(staged-security-with-data-state-verifier-event
($\lambda s. \text{composition-fri-query-index-list-set-hit } s \ Q$))
adversary-initial-state $\leq$
*nnreal (card Q) **
(1 / nnreal (card query-sample-space)) $\wedge$ rounds +
hash-relation-budget-value
(query-index-raw-list-relation-fiber-bound Q)
(staged-attacker-query-budget budgets +
staged-challenge-query-budget)

by (rule
checked-staged-security-composition-fri-query-index-list-set-hit-bound-from-path-fresh
 [OF wf controlled
checked-staged-security-with-data-state-query-index-list-path-fresh-product-bound
 [OF wf controlled subset]])

lemma *checked-staged-security-trace-fri-query-challenge-pair-set-hit-exact-product-bound:*

assumes *wf*: staged-budget-wellformed budgets
and *controlled*: staged-adversary-controlled budgets *A*
and *projection*: $\text{fst } P \subseteq Q$
and *subset*: $Q \subseteq \text{fri-query-index-list-space}$
shows
wp-event (*checked-staged-security-experiment-with-data-state A*)
(*staged-security-with-data-state-verifier-event*
($\lambda s. \text{trace-fri-query-challenge-pair-set-hit } s \ P$))
adversary-initial-state $\leq$
nnreal (*card Q*) *
($1 / \text{nnreal } (\text{card query-sample-space})$) $\wedge$ *rounds* +
hash-relation-budget-value
(*query-index-row-list-relation-fiber-bound Q*)
(*staged-attacker-query-budget budgets* +
staged-challenge-query-budget)
by (*rule*
checked-staged-security-trace-fri-query-challenge-pair-set-hit-bound-from-path-fresh
[*OF wf controlled projection*
checked-staged-security-with-data-state-query-index-list-path-fresh-product-bound
[*OF wf controlled subset*]])

lemma *checked-staged-security-composition-fri-query-challenge-pair-set-hit-exact-product-bound*:

assumes *wf*: staged-budget-wellformed budgets
and *controlled*: staged-adversary-controlled budgets *A*
and *projection*: $\bigwedge dg. \text{fst } P \ dg \subseteq Q$
and *subset*: $Q \subseteq \text{fri-query-index-list-space}$
shows
wp-event (*checked-staged-security-experiment-with-data-state A*)
(*staged-security-with-data-state-verifier-event*
($\lambda s. \text{composition-fri-query-challenge-pair-set-hit } s \ P$))
adversary-initial-state $\leq$
nnreal (*card Q*) *
($1 / \text{nnreal } (\text{card query-sample-space})$) $\wedge$ *rounds* +
hash-relation-budget-value
(*query-index-row-list-relation-fiber-bound Q*)
(*staged-attacker-query-budget budgets* +
staged-challenge-query-budget)
by (*rule*
checked-staged-security-composition-fri-query-challenge-pair-set-hit-bound-from-path-fresh
[*OF wf controlled projection*
checked-staged-security-with-data-state-query-index-list-path-fresh-product-bound
[*OF wf controlled subset*]])

end

end

theory *Soundness-FRI-Query-Fiber-Exact-Staged-Bounds*

```

imports
  Soundness-FRI-Query-Fiber-Staged-Bounds
  Soundness-FRI-Query-List-Exact-Product

```

```

begin

```

Exact query-list staged bounds for the outside-query-cover branch of sampled FRI query events.

This layer keeps the challenge-cover split and uses exact product query-list accounting for the outside branch.

```

context soundness
begin

```

lemma *checked-staged-security-trace-fri-query-challenge-pair-set-hit-bound-from-challenge-cover-and-query-list-exact*

```

assumes wf: staged-budget-wellformed budgets

```

```

and controlled: staged-adversary-controlled budgets A

```

```

and finite-B: finite B

```

```

and fresh-bound:

```

```

  wp-event (checked-staged-security-experiment-with-data-state A)

```

```

    (staged-security-with-data-state-verifier-event

```

```

      ( $\lambda s.$  trace-fri-challenge-list-fresh-hit s B))

```

```

    adversary-initial-state  $\leq$  F

```

```

and projection:

```

```

  fst ' $(P \cap (UNIV \times (- B))) \subseteq Q$ 

```

```

and subset:  $Q \subseteq \text{fri-query-index-list-space}$ 

```

```

shows

```

```

  wp-event (checked-staged-security-experiment-with-data-state A)

```

```

    (staged-security-with-data-state-verifier-event

```

```

      ( $\lambda s.$  trace-fri-query-challenge-pair-set-hit s P))

```

```

    adversary-initial-state  $\leq$ 

```

```

      F +

```

```

      hash-relation-budget-value (card B * ceil-log clength)

```

```

      (staged-attacker-query-budget budgets + staged-challenge-query-budget) +

```

```

      (nnreal (card Q) *

```

```

        (1 / nnreal (card query-sample-space))  $\wedge$  rounds +

```

```

      hash-relation-budget-value

```

```

      (query-index-raw-list-relation-fiber-bound Q)

```

```

      (staged-attacker-query-budget budgets +

```

```

        staged-challenge-query-budget))

```

```

proof -

```

```

  let ?Outside =  $P \cap (UNIV \times (- B))$ 

```

```

  have split:

```

```

    wp-event (checked-staged-security-experiment-with-data-state A)

```

```

      (staged-security-with-data-state-verifier-event

```

```

        ( $\lambda s.$  trace-fri-query-challenge-pair-set-hit s P))

```

```

      adversary-initial-state  $\leq$ 

```

```

    wp-event (checked-staged-security-experiment-with-data-state A)

```

```

      ( $\lambda out.$ 

```

```

        staged-security-with-data-state-verifier-event

```

```

        ( $\lambda s.$  trace-fri-challenge-list-set-hit s B) out  $\vee$ 

```

```

    staged-security-with-data-state-verifier-event
      (λs. trace-fri-query-challenge-pair-set-hit s ?Outside) out)
  adversary-initial-state
proof (rule wp-event-mono-on-support)
  fix out
  assume hit:
    staged-security-with-data-state-verifier-event
      (λs. trace-fri-query-challenge-pair-set-hit s P) out
  then show
    staged-security-with-data-state-verifier-event
      (λs. trace-fri-challenge-list-set-hit s B) out ∨
    staged-security-with-data-state-verifier-event
      (λs. trace-fri-query-challenge-pair-set-hit s ?Outside) out
proof (cases out)
  case None
  then show ?thesis
    using hit unfolding staged-security-with-data-state-verifier-event-def
    by simp
next
  case (Some full)
  then obtain data attacker-state result final-state where out-eq:
    out = Some (((data, attacker-state), result), final-state)
    by (cases full) (auto split: prod.splits)
  let ?s =
    verifier-state-from-adversary attacker-state
      (staged-proof-transcript data)
  have pair-hit:
    trace-fri-query-challenge-pair-set-hit ?s P
      (Some (result, final-state))
    using hit out-eq
    unfolding staged-security-with-data-state-verifier-event-def by simp
  from pair-hit obtain trace-roots trace-bs trace-final dg
    composition-roots composition-bs composition-final fri-query-idxs
    trace-round-layers composition-round-layers
  where openings:
    accepted-fri-opening-transcript ?s (Some (result, final-state))
    trace-roots trace-bs trace-final dg composition-roots
    composition-bs composition-final fri-query-idxs
    trace-round-layers composition-round-layers
  and pair: (fri-query-idxs, trace-bs) ∈ P
  unfolding trace-fri-query-challenge-pair-set-hit-def by blast
show ?thesis
proof (cases trace-bs ∈ B)
  case True
  have challenges:
    accepted-fri-challenges ?s (Some (result, final-state))
    trace-bs dg composition-bs
    by (rule accepted-fri-opening-transcript-challenges[OF openings])
  have trace-fri-challenge-list-set-hit ?s B

```

```

      (Some (result, final-state))
    unfolding trace-fri-challenge-list-set-hit-def
    using challenges True by blast
  then show ?thesis
    using out-eq unfolding staged-security-with-data-state-verifier-event-def
    by simp
next
case False
have outside-pair: (fri-query-idxs, trace-bs) ∈ ?Outside
  using pair False by simp
have trace-fri-query-challenge-pair-set-hit ?s ?Outside
  (Some (result, final-state))
  unfolding trace-fri-query-challenge-pair-set-hit-def
  using openings outside-pair by blast
then show ?thesis
  using out-eq unfolding staged-security-with-data-state-verifier-event-def
  by simp
qed
qed
qed
also have ... ≤
  wp-event (checked-staged-security-experiment-with-data-state A)
    (staged-security-with-data-state-verifier-event
      (λs. trace-fri-challenge-list-set-hit s B))
    adversary-initial-state +
  wp-event (checked-staged-security-experiment-with-data-state A)
    (staged-security-with-data-state-verifier-event
      (λs. trace-fri-query-challenge-pair-set-hit s ?Outside))
    adversary-initial-state
  by (rule wp-event-union-bound)
also have ... ≤
  (F +
    hash-relation-budget-value (card B * ceil-log clength)
      (staged-attacker-query-budget budgets +
        staged-challenge-query-budget)) +
  (nnreal (card Q) *
    (1 / nnreal (card query-sample-space)) ^ rounds +
    hash-relation-budget-value
      (query-index-raw-list-relation-fiber-bound Q)
      (staged-attacker-query-budget budgets +
        staged-challenge-query-budget))
proof (rule add-mono)
show wp-event (checked-staged-security-experiment-with-data-state A)
  (staged-security-with-data-state-verifier-event
    (λs. trace-fri-challenge-list-set-hit s B))
  adversary-initial-state
≤ F +
  hash-relation-budget-value (card B * ceil-log clength)
    (staged-attacker-query-budget budgets +

```

```

      staged-challenge-query-budget)
  by (rule checked-staged-security-trace-fri-challenge-list-set-hit-bound
      [OF wf controlled finite-B fresh-bound])
show wp-event (checked-staged-security-experiment-with-data-state A)
  (staged-security-with-data-state-verifier-event
   (λs. trace-fri-query-challenge-pair-set-hit s ?Outside))
adversary-initial-state
≤ nnreal (card Q) *
  (1 / nnreal (card query-sample-space)) ^ rounds +
hash-relation-budget-value
  (query-index-raw-list-relation-fiber-bound Q)
  (staged-attacker-query-budget budgets +
   staged-challenge-query-budget)
  by (rule
      checked-staged-security-trace-fri-query-challenge-pair-set-hit-exact-product-bound
      [OF wf controlled projection subset])
qed
finally show ?thesis
  by (simp add: add.assoc)
qed

lemma checked-staged-security-composition-fri-query-challenge-pair-set-hit-bound-from-challenge-cover-and-query-lis
assumes wf: staged-budget-wellformed budgets
and controlled: staged-adversary-controlled budgets A
and finite-B:  $\bigwedge dg. \text{finite } (B \ dg)$ 
and fresh-bound:
wp-event (checked-staged-security-experiment-with-data-state A)
  (staged-security-with-data-state-verifier-event
   (λs. composition-fri-challenge-list-fresh-hit s B))
adversary-initial-state ≤ F
and projection:
 $\bigwedge dg. \text{fst } ' (P \ dg \cap (UNIV \times (- \ B \ dg))) \subseteq Q$ 
and subset:  $Q \subseteq \text{fri-query-index-list-space}$ 
shows
wp-event (checked-staged-security-experiment-with-data-state A)
  (staged-security-with-data-state-verifier-event
   (λs. composition-fri-query-challenge-pair-set-hit s P))
adversary-initial-state ≤
F +
hash-relation-budget-value
  ( $\sum dg \in (UNIV :: 'f \ \text{set}).$ 
   card (B dg) * ceil-log (maxDegree + 1))
  (staged-attacker-query-budget budgets +
   staged-challenge-query-budget) +
(nnreal (card Q) *
  (1 / nnreal (card query-sample-space)) ^ rounds +
hash-relation-budget-value
  (query-index-raw-list-relation-fiber-bound Q)
  (staged-attacker-query-budget budgets +

```

```

    staged-challenge-query-budget))
proof -
  let ?Outside =  $\lambda dg. P \ dg \cap (UNIV \times (- \ B \ dg))$ 
  have split:
    wp-event (checked-staged-security-experiment-with-data-state A)
      (staged-security-with-data-state-verifier-event
        ( $\lambda s. \text{composition-fri-query-challenge-pair-set-hit } s \ P$ ))
    adversary-initial-state  $\leq$ 
    wp-event (checked-staged-security-experiment-with-data-state A)
      ( $\lambda out.$ 
        staged-security-with-data-state-verifier-event
          ( $\lambda s. \text{composition-fri-challenge-list-set-hit } s \ B$ ) out  $\vee$ 
        staged-security-with-data-state-verifier-event
          ( $\lambda s. \text{composition-fri-query-challenge-pair-set-hit } s \ ?Outside$ ) out)
    adversary-initial-state
proof (rule wp-event-mono-on-support)
  fix out
  assume hit:
    staged-security-with-data-state-verifier-event
      ( $\lambda s. \text{composition-fri-query-challenge-pair-set-hit } s \ P$ ) out
  then show
    staged-security-with-data-state-verifier-event
      ( $\lambda s. \text{composition-fri-challenge-list-set-hit } s \ B$ ) out  $\vee$ 
    staged-security-with-data-state-verifier-event
      ( $\lambda s. \text{composition-fri-query-challenge-pair-set-hit } s \ ?Outside$ ) out
proof (cases out)
  case None
  then show ?thesis
    using hit unfolding staged-security-with-data-state-verifier-event-def
    by simp
  next
  case (Some full)
  then obtain data attacker-state result final-state where out-eq:
    out = Some (((data, attacker-state), result), final-state)
    by (cases full) (auto split: prod.splits)
  let ?s =
    verifier-state-from-adversary attacker-state
      (staged-proof-transcript data)
  have pair-hit:
    composition-fri-query-challenge-pair-set-hit ?s P
      (Some (result, final-state))
    using hit out-eq
    unfolding staged-security-with-data-state-verifier-event-def by simp
  from pair-hit obtain trace-roots trace-bs trace-final fri-dg
    composition-roots composition-bs composition-final fri-query-idxs
    trace-round-layers composition-round-layers
  where openings:
    accepted-fri-opening-transcript ?s (Some (result, final-state))
    trace-roots trace-bs trace-final fri-dg composition-roots

```

```

    composition-bs composition-final fri-query-idxs
    trace-round-layers composition-round-layers
and pair: (fri-query-idxs, composition-bs)  $\in P$  fri-dg
    unfolding composition-fri-query-challenge-pair-set-hit-def by blast
show ?thesis
proof (cases composition-bs  $\in B$  fri-dg)
  case True
  have challenges:
    accepted-fri-challenges ?s (Some (result, final-state))
    trace-bs fri-dg composition-bs
  by (rule accepted-fri-opening-transcript-challenges[OF openings])
  have composition-fri-challenge-list-set-hit ?s B
    (Some (result, final-state))
  unfolding composition-fri-challenge-list-set-hit-def
  using challenges True by blast
  then show ?thesis
    using out-eq unfolding staged-security-with-data-state-verifier-event-def
    by simp
  next
  case False
  have outside-pair:
    (fri-query-idxs, composition-bs)  $\in$  ?Outside fri-dg
  using pair False by simp
  have composition-fri-query-challenge-pair-set-hit ?s ?Outside
    (Some (result, final-state))
  unfolding composition-fri-query-challenge-pair-set-hit-def
  using openings outside-pair by blast
  then show ?thesis
    using out-eq unfolding staged-security-with-data-state-verifier-event-def
    by simp
  qed
qed
qed
also have ...  $\leq$ 
  wp-event (checked-staged-security-experiment-with-data-state A)
    (staged-security-with-data-state-verifier-event
      ( $\lambda s$ . composition-fri-challenge-list-set-hit s B))
  adversary-initial-state +
  wp-event (checked-staged-security-experiment-with-data-state A)
    (staged-security-with-data-state-verifier-event
      ( $\lambda s$ . composition-fri-query-challenge-pair-set-hit s ?Outside))
  adversary-initial-state
by (rule wp-event-union-bound)
also have ...  $\leq$ 
  (F +
    hash-relation-budget-value
    ( $\sum dg \in (UNIV :: 'f \text{ set}).$ 
      card (B dg) * ceil-log (maxDegree + 1))
    (staged-attacker-query-budget budgets +

```


$$\begin{aligned} & \text{staged-challenge-query-budget})) + \\ & (\text{nnreal } (\text{card } Q) * \\ & (1 / \text{nnreal } (\text{card } \text{query-sample-space})) ^ \wedge \text{rounds} + \\ & \text{hash-relation-budget-value} \\ & (\text{query-index-raw-list-relation-fiber-bound } Q) \\ & (\text{staged-attacker-query-budget } \text{budgets} + \\ & \text{staged-challenge-query-budget})) \end{aligned}$$

proof (rule *add-mono*)

show *wp-event (checked-staged-security-experiment-with-data-state A)*
(staged-security-with-data-state-verifier-event
*(λs . *composition-fri-challenge-list-set-hit s B*))*
adversary-initial-state
 $\leq F +$
hash-relation-budget-value
 $(\sum dg \in (\text{UNIV} :: 'f \text{ set}).$
 $\text{card } (B \ dg) * \text{ceil-log } (\text{maxDegree} + 1))$
 $(\text{staged-attacker-query-budget } \text{budgets} +$
 $\text{staged-challenge-query-budget})$

by (rule
checked-staged-security-composition-fri-challenge-list-set-hit-bound
[OF wf controlled finite-B fresh-bound])

show *wp-event (checked-staged-security-experiment-with-data-state A)*
(staged-security-with-data-state-verifier-event
*(λs . *composition-fri-query-challenge-pair-set-hit s ?Outside*))*
adversary-initial-state
 $\leq \text{nnreal } (\text{card } Q) *$
 $(1 / \text{nnreal } (\text{card } \text{query-sample-space})) ^ \wedge \text{rounds} +$
hash-relation-budget-value
 $(\text{query-index-raw-list-relation-fiber-bound } Q)$
 $(\text{staged-attacker-query-budget } \text{budgets} +$
 $\text{staged-challenge-query-budget})$

by (rule
checked-staged-security-composition-fri-query-challenge-pair-set-hit-exact-product-bound
[OF wf controlled projection subset])

qed

finally show *?thesis*

by (*simp add: add.assoc*)

qed

lemma *checked-staged-security-trace-fri-sampled-query-bad-candidate-bound-from-generic-challenge-cover-and-outsides*

assumes *wf: staged-budget-wellformed budgets*

and *controlled: staged-adversary-controlled budgets A*

and *finite-B: finite B*

and *fresh-bound:*

wp-event (checked-staged-security-experiment-with-data-state A)
(staged-security-with-data-state-verifier-event
*(λs . *trace-fri-challenge-list-fresh-hit s B*))*
adversary-initial-state $\leq F$

shows

```

wp-event (checked-staged-security-experiment-with-data-state A)
  (staged-security-with-data-state-verifier-event
    trace-fri-sampled-query-bad-candidate)
adversary-initial-state ≤
F +
hash-relation-budget-value (card B * ceil-log clength)
  (staged-attacker-query-budget budgets + staged-challenge-query-budget) +
(nnreal
  (card
    (generic-fri-sampled-query-outside-query-cover
      trace-table-low-degree (Not ∘ trace-table-low-degree)
      (clength - 1) B)) *
    (1 / nnreal (card query-sample-space)) ^ rounds +
  hash-relation-budget-value
    (query-index-row-list-relation-fiber-bound
      (generic-fri-sampled-query-outside-query-cover
        trace-table-low-degree (Not ∘ trace-table-low-degree)
        (clength - 1) B))
    (staged-attacker-query-budget budgets +
      staged-challenge-query-budget))
proof -
let ?P =
  trace-fri-sampled-query-bad-pair-union ∩
  (fri-query-index-list-space × UNIV)
let ?Q =
  generic-fri-sampled-query-outside-query-cover trace-table-low-degree
  (Not ∘ trace-table-low-degree) (clength - 1) B
have sampled-to-pair:
  wp-event (checked-staged-security-experiment-with-data-state A)
    (staged-security-with-data-state-verifier-event
      trace-fri-sampled-query-bad-candidate)
  adversary-initial-state ≤
  wp-event (checked-staged-security-experiment-with-data-state A)
    (staged-security-with-data-state-verifier-event
      (λs. trace-fri-query-challenge-pair-set-hit s ?P))
  adversary-initial-state
by (rule wp-event-mono-on-support)
  (auto simp: staged-security-with-data-state-verifier-event-def
    intro:
      trace-fri-sampled-query-bad-candidate-imp-restricted-query-challenge-pair-union-hit
      split: option.splits prod.splits)
also have ... ≤
  F +
  hash-relation-budget-value (card B * ceil-log clength)
    (staged-attacker-query-budget budgets + staged-challenge-query-budget) +
  (nnreal (card ?Q) *
    (1 / nnreal (card query-sample-space)) ^ rounds +
  hash-relation-budget-value
    (query-index-row-list-relation-fiber-bound ?Q)

```

(staged-attacker-query-budget budgets +
 staged-challenge-query-budget))
 by (rule
 checked-staged-security-trace-fri-query-challenge-pair-set-hit-bound-from-challenge-cover-and-query-list-exact
 [OF wf controlled finite-B fresh-bound
 trace-fri-sampled-query-outside-projection-subset
 generic-fri-sampled-query-outside-query-cover-subset])
 finally show ?thesis .
 qed

lemma checked-staged-security-composition-fri-sampled-query-bad-candidate-bound-from-generic-challenge-cover-and
assumes wf: staged-budget-wellformed budgets
and controlled: staged-adversary-controlled budgets A
and finite-B: $\bigwedge dg. \text{finite } (B \ dg)$
and fresh-bound:
 wp-event (checked-staged-security-experiment-with-data-state A)
 (staged-security-with-data-state-verifier-event
 ($\lambda s. \text{composition-fri-challenge-list-fresh-hit } s \ B$))
 adversary-initial-state $\leq F$
shows
 wp-event (checked-staged-security-experiment-with-data-state A)
 (staged-security-with-data-state-verifier-event
 composition-fri-sampled-query-bad-candidate)
 adversary-initial-state $\leq$
 F +
 hash-relation-budget-value
 ($\sum dg \in (UNIV :: 'f \text{ set}).$
 card (B dg) * ceil-log (maxDegree + 1))
 (staged-attacker-query-budget budgets +
 staged-challenge-query-budget) +
 (nnreal (card (composition-fri-sampled-query-outside-query-cover B)) *
 (1 / nnreal (card query-sample-space)) ^ rounds +
 hash-relation-budget-value
 (query-index-raw-list-relation-fiber-bound
 (composition-fri-sampled-query-outside-query-cover B))
 (staged-attacker-query-budget budgets +
 staged-challenge-query-budget))

proof –
 let ?P =
 $\lambda dg. \text{composition-fri-sampled-query-bad-pair-union } dg \cap$
 (fri-query-index-list-space $\times$ UNIV)
 let ?Q = composition-fri-sampled-query-outside-query-cover B
have sampled-to-pair:
 wp-event (checked-staged-security-experiment-with-data-state A)
 (staged-security-with-data-state-verifier-event
 composition-fri-sampled-query-bad-candidate)
 adversary-initial-state $\leq$
 wp-event (checked-staged-security-experiment-with-data-state A)
 (staged-security-with-data-state-verifier-event

```

      (λs. composition-fri-query-challenge-pair-set-hit s ?P))
    adversary-initial-state
  by (rule wp-event-mono-on-support)
    (auto simp: staged-security-with-data-state-verifier-event-def
      intro:
        composition-fri-sampled-query-bad-candidate-imp-restricted-query-challenge-pair-union-hit
        split: option.splits prod.splits)
  also have ... ≤
    F +
    hash-relation-budget-value
    (∑ dg ∈ (UNIV :: 'f set).
      card (B dg) * ceil-log (maxDegree + 1))
    (staged-attacker-query-budget budgets +
      staged-challenge-query-budget) +
    (nnreal (card ?Q) *
      (1 / nnreal (card query-sample-space)) ^ rounds +
      hash-relation-budget-value
      (query-index-row-list-relation-fiber-bound ?Q)
      (staged-attacker-query-budget budgets +
        staged-challenge-query-budget))
  by (rule
    checked-staged-security-composition-fri-query-challenge-pair-set-hit-bound-from-challenge-cover-and-query-list-ex
    [OF wf controlled finite-B fresh-bound
      composition-fri-sampled-query-outside-projection-subset
      composition-fri-sampled-query-outside-query-cover-subset])
  finally show ?thesis .
qed

end

end

```

```

theory Soundness-FRI-Query-Challenge-Fresh-Split
imports Soundness-FRI-Query-Fiber-Exact-Staged-Bounds
begin

```

Staged split for FRI query/challenge pair events.

The exact pair-fraction route must keep the query/challenge pair event until the challenge-list probability has been accounted for. This layer separates only the challenge prequery side event, leaving the fresh branch as the original pair hit conjoined with path freshness of the relevant FRI challenge keys.

```

context soundness
begin

```

```

lemma verifier-after-trace-fri-concrete-header-transcript:
fixes s :: ('f, 'a) protocol-channel-scheme
assumes prefix:
  Some ((fr, f-fl), prefix-state) ∈

```

```

    set-dist (execute verifier-trace-fri-prefix s)
and suffix:
    Some (result, final-state) ∈
      set-dist
        (execute (verifier-after-trace-fri (fr, f-fl)) prefix-state)
obtains f-final as dg fl final query-state where
    verifier-header-transcript s fr (map snd f-fl) f-final as dg
      (map snd fl) final (PTranscript query-state)
    Some (result, final-state) ∈
      set-dist
        (execute
          (ntimes
            (verifier-query-round-program fr f-fl f-final as fl final)
            rounds)
          query-state)
proof –
have prefix-res:
    PTranscript s = [fr] @ map snd f-fl @ PTranscript prefix-state ∧
    PState prefix-state =
      foldl concat (concat (PState s) fr) (map snd f-fl)
    using verifier-trace-fri-prefix-outcome[OF prefix] by simp
have len-f-fl: length f-fl = ceil-log clength
    using verifier-trace-fri-prefix-outcome[OF prefix] by simp
from suffix obtain f-final s3 as s4 dg s5 s6 fl s7 final query-state where
    read-trace-final:
      Some (f-final, s3) ∈ set-dist (execute read prefix-state)
and alpha-out:
      Some (as, s4) ∈
        set-dist (execute (mmap (replicate (length spec) alpha-round)) s3)
and read-dg: Some (dg, s5) ∈ set-dist (execute read s4)
and degree-assert:
      Some ((), s6) ∈
        set-dist (execute (assert (to-nat dg ≤ maxDegree)) s5)
and comp-fri:
      Some (fl, s7) ∈
        set-dist (execute
          (ntimes receive-composition-fri-commits
            (ceil-log (to-nat dg + 1))) s6)
and read-final:
      Some (final, query-state) ∈ set-dist (execute read s7)
and query-out:
      Some (result, final-state) ∈
        set-dist (execute
          (ntimes (verifier-query-round-program fr f-fl f-final as fl final)
            rounds)
          query-state)
    unfolding verifier-after-trace-fri-def
    by (auto elim!: set-dist-bindE)
from read-outcome[OF read-trace-final] obtain rest3 where

```

```

  tr-prefix: PTranscript prefix-state = f-final # rest3
  and tr-s3: PTranscript s3 = rest3
  by blast
have alpha-res:
  length as = length spec ∧
  PTranscript s3 = as @ PTranscript s4
  using mmap-alpha-round-outcome[OF alpha-out] by simp
from read-outcome[OF read-dg] obtain rest5 where
  tr-s4: PTranscript s4 = dg # rest5
  and tr-s5: PTranscript s5 = rest5
  by blast
have s6-eq: s6 = s5
  using degree-assert unfolding assert-def
  by (cases to-nat dg ≤ maxDegree) (auto simp: throw-no-outcome)
have comp-res:
  length fl = ceil-log (to-nat dg + 1) ∧
  PTranscript s6 = map snd fl @ PTranscript s7
  using ntimes-receive-composition-fri-commits-outcome[OF comp-fri]
  by simp
from read-outcome[OF read-final] obtain rest-query where
  tr-s7: PTranscript s7 = final # rest-query
  and tr-query: PTranscript query-state = rest-query
  by blast
have header:
  verifier-header-transcript s fr (map snd f-fl) f-final as dg
  (map snd fl) final (PTranscript query-state)
  using prefix-res len-f-fl tr-prefix tr-s3 alpha-res tr-s4 tr-s5 s6-eq comp-res
  tr-s7 tr-query
  unfolding verifier-header-transcript-def verifier-header-messages-def
  by simp
show thesis
  by (rule that[OF header query-out])
qed

```

lemma *verifier-after-trace-fri-header-agrees-with-accepted-challenges:*

```

  assumes prefix:
    Some ((fr, f-fl), prefix-state) ∈
      set-dist (execute verifier-trace-fri-prefix s)
  and suffix:
    Some (result, final-state) ∈
      set-dist
        (execute (verifier-after-trace-fri (fr, f-fl)) prefix-state)
  and accepted:
    accepted-fri-challenges s (Some (result, final-state)) trace-bs dg
    comp-bs
  shows trace-bs = map fst f-fl
proof –
  from verifier-after-trace-fri-accepted-fri-challenges[OF prefix suffix]
  obtain dg' comp-bs' where replay:

```

```

    accepted-fri-challenges s (Some (result, final-state))
      (map fst f-fl) dg' comp-bs'
  by blast
show ?thesis
  using accepted-fri-challenges-unique[OF replay accepted] by simp
qed

lemma verifier-after-trace-fri-header-roots-agree-with-fresh-hit:
  assumes prefix:
    Some ((fr, f-fl), prefix-state) ∈
      set-dist (execute verifier-trace-fri-prefix s)
  and suffix:
    Some (result, final-state) ∈
      set-dist
        (execute (verifier-after-trace-fri (fr, f-fl)) prefix-state)
  and fresh:
    trace-fri-challenge-list-fresh-hit s B
    (Some (result, final-state))
  shows trace-fri-challenge-path-fresh s fr (map snd f-fl)
    (length (map snd f-fl))
proof -
  from fresh obtain trace-bs dg comp-bs fr' trace-roots' trace-final' as'
    composition-roots' final' rest' where
  accepted:
    accepted-fri-challenges s (Some (result, final-state)) trace-bs dg
    comp-bs
  and header-hit:
    verifier-header-transcript s fr' trace-roots' trace-final' as' dg
    composition-roots' final' rest'
  and fresh-path:
    trace-fri-challenge-path-fresh s fr' trace-roots'
    (length trace-roots')
  unfolding trace-fri-challenge-list-fresh-hit-def by blast
  from verifier-after-trace-fri-accepted-fri-challenges[OF prefix suffix]
  obtain dg0 comp-bs0 where replay:
    accepted-fri-challenges s (Some (result, final-state))
      (map fst f-fl) dg0 comp-bs0
  by blast
  have dg-eq: dg = dg0
  using accepted-fri-challenges-unique[OF accepted replay] by simp
  from verifier-after-trace-fri-concrete-header-transcript[OF prefix suffix]
  obtain f-final0::'f and as0:: 'f list and dg1::'f and fl0:: ('f × 'f) list and final0:: 'f and
    query-state0:: ('f, 'a) protocol-channel-scheme where
  header-concrete:
    verifier-header-transcript s fr (map snd f-fl) f-final0 as0 dg1
      (map snd fl0) final0 (PTranscript query-state0)
  by metis
  from accepted-fri-challenges-headerE[OF replay]
  obtain fr0::'f and f-fl0:: ('f × 'f) list and f-final0'::'f and as0':: 'f list and fl0':: ('f × 'f)

```

```

list and final0'::'f and rest0':: 'f list where
  header-replay:
    verifier-header-transcript s fr0 (map snd f-fl0) f-final0' as0'
    dg0 (map snd fl0') final0' rest0'
  by metis
have replay-unique:
  fr0 = fr  $\wedge$  map snd f-fl0 = map snd f-fl  $\wedge$  dg0 = dg1
  using verifier-header-transcript-unique[OF header-replay header-concrete]
  by simp
from accepted-fri-challenges-headerE[OF accepted]
obtain fr1::'f and f-fl1:: ('f  $\times$  'f) list and f-final1::'f and as1:: 'f list and fl1:: ('f  $\times$  'f) list
and final1::'f and rest1 where
  header-accepted:
    verifier-header-transcript s fr1 (map snd f-fl1) f-final1 as1 dg
    (map snd fl1) final1 rest1
  by metis
have accepted-unique-header:
  fr1 = fr0  $\wedge$  map snd f-fl1 = map snd f-fl0
  using verifier-header-transcript-unique[OF header-accepted header-replay]
  dg-eq
  by simp
have hit-unique:
  fr' = fr1  $\wedge$  trace-roots' = map snd f-fl1
  using verifier-header-transcript-unique[OF header-hit header-accepted]
  by simp
show ?thesis
  using fresh-path hit-unique accepted-unique-header replay-unique by simp
qed

```

lemma *verifier-after-composition-fri-concrete-header-transcript:*

fixes $s :: ('f, 'a)$ *protocol-channel-scheme*

assumes *prefix:*

Some ((fr, f-fl, f-final, as, dg, fl), prefix-state) $\in$
set-dist (execute verifier-composition-fri-prefix s)

and *suffix:*

Some (result, final-state) $\in$
set-dist
(execute
(verifier-after-composition-fri
(fr, f-fl, f-final, as, dg, fl))
prefix-state)

obtains *final query-state* **where**

verifier-header-transcript s fr (map snd f-fl) f-final as dg
(map snd fl) final (PTranscript query-state)

Some (result, final-state) $\in$

set-dist
(execute
(ntimes
(verifier-query-round-program fr f-fl f-final as fl final)


```

        rounds)
        query-state)
proof –
  have prefix-res:
    PTranscript s =
      [fr] @ map snd f-fl @ [f-final] @ as @ [dg] @
        map snd fl @ PTranscript prefix-state
    using verifier-composition-fri-prefix-outcome[OF prefix] by simp
from suffix obtain final query-state where
  read-final:
    Some (final, query-state) ∈
      set-dist (execute read prefix-state)
and query-out:
    Some (result, final-state) ∈
      set-dist
        (execute
          (ntimes
            (verifier-query-round-program fr f-fl f-final as fl final)
            rounds)
          query-state)
    unfolding verifier-after-composition-fri-def
    by (auto elim! set-dist-bindE)
from read-outcome[OF read-final] obtain rest-query where
  tr-prefix: PTranscript prefix-state = final # rest-query
  and tr-query: PTranscript query-state = rest-query
  by blast
have len-f-fl: length f-fl = ceil-log clength
  using verifier-composition-fri-prefix-outcome[OF prefix] by simp
have len-as: length as = length spec
  using verifier-composition-fri-prefix-outcome[OF prefix] by simp
have len-fl: length fl = ceil-log (to-nat dg + 1)
  using verifier-composition-fri-prefix-outcome[OF prefix] by simp
have header:
  verifier-header-transcript s fr (map snd f-fl) f-final as dg
    (map snd fl) final (PTranscript query-state)
  using prefix-res len-f-fl len-as len-fl tr-prefix tr-query
  unfolding verifier-header-transcript-def verifier-header-messages-def
  by simp
show thesis
  by (rule that[OF header query-out])
qed

```

```

lemma verifier-after-composition-fri-header-agrees-with-accepted-challenges:
assumes prefix:
  Some ((fr, f-fl, f-final, as, dg, fl), prefix-state) ∈
    set-dist (execute verifier-composition-fri-prefix s)
and suffix:
  Some (result, final-state) ∈
    set-dist

```

```

      (execute
        (verifier-after-composition-fri
          (fr, f-fl, f-final, as, dg, fl))
        prefix-state)
    and accepted:
      accepted-fri-challenges s (Some (result, final-state)) trace-bs dg'
      comp-bs
    shows dg' = dg ∧ comp-bs = map fst fl
  proof –
    have replay:
      accepted-fri-challenges s (Some (result, final-state))
        (map fst f-fl) dg (map fst fl)
    by (rule verifier-after-composition-fri-accepted-fri-challenges
        [OF prefix suffix])
    show ?thesis
      using accepted-fri-challenges-unique[OF replay accepted] by simp
  qed

lemma verifier-after-composition-fri-header-roots-agree-with-fresh-hit:
  assumes prefix:
    Some ((fr, f-fl, f-final, as, dg, fl), prefix-state) ∈
      set-dist (execute verifier-composition-fri-prefix s)
  and suffix:
    Some (result, final-state) ∈
      set-dist
        (execute
          (verifier-after-composition-fri
            (fr, f-fl, f-final, as, dg, fl))
          prefix-state)
  and fresh:
    composition-fri-challenge-list-fresh-hit s B
    (Some (result, final-state))
  shows composition-fri-challenge-path-fresh s fr (map snd f-fl) f-final
    as dg (map snd fl) (length (map snd fl))
  proof –
    from fresh obtain trace-bs dg' comp-bs fr' trace-roots' trace-final' as'
      composition-roots' final' rest' where
      accepted:
        accepted-fri-challenges s (Some (result, final-state)) trace-bs dg'
        comp-bs
    and header-hit:
      verifier-header-transcript s fr' trace-roots' trace-final' as' dg'
      composition-roots' final' rest'
    and fresh-path:
      composition-fri-challenge-path-fresh s fr' trace-roots' trace-final'
      as' dg' composition-roots' (length composition-roots')
    unfolding composition-fri-challenge-list-fresh-hit-def by blast
    from verifier-after-composition-fri-concrete-header-transcript[OF prefix suffix]
    obtain final0:: 'f and query-state0:: ('f, 'a) protocol-channel-scheme where

```

```

header-concrete:
  verifier-header-transcript s fr (map snd f-fl) f-final as dg
  (map snd fl) final0 (PTranscript query-state0)
by blast
have fr' = fr ∧
  trace-roots' = map snd f-fl ∧
  trace-final' = f-final ∧
  as' = as ∧
  dg' = dg ∧
  composition-roots' = map snd fl
using verifier-header-transcript-unique[OF header-hit header-concrete]
by simp
then show ?thesis
  using fresh-path by simp
qed

```

definition *staged-security-trace-fri-pair-challenge-fresh*

```

:: (nat list × 'f list) set ⇒ 'f list set ⇒
  (((('f staged-proof-data × 'f protocol-channel) ×
    unit list) × 'f protocol-channel) option ⇒ bool

```

where

```

staged-security-trace-fri-pair-challenge-fresh P B out ⟷
  staged-security-with-data-state-verifier-event
    (λs. trace-fri-query-challenge-pair-set-hit s P) out ∧
  staged-security-with-data-state-verifier-event
    (λs. trace-fri-challenge-list-fresh-hit s B) out

```

definition *staged-security-composition-fri-pair-challenge-fresh*

```

:: ('f ⇒ (nat list × 'f list) set) ⇒
  ('f ⇒ 'f list set) ⇒
  (((('f staged-proof-data × 'f protocol-channel) ×
    unit list) × 'f protocol-channel) option ⇒ bool

```

where

```

staged-security-composition-fri-pair-challenge-fresh P B out ⟷
  staged-security-with-data-state-verifier-event
    (λs. composition-fri-query-challenge-pair-set-hit s P) out ∧
  staged-security-with-data-state-verifier-event
    (λs. composition-fri-challenge-list-fresh-hit s B) out

```

lemma *checked-staged-security-trace-fri-query-challenge-pair-set-hit-split-challenge-fresh:*

assumes *projection:* $\text{snd} \, 'P \subseteq B$

shows

```

wp-event (checked-staged-security-experiment-with-data-state A)
  (staged-security-with-data-state-verifier-event
    (λs. trace-fri-query-challenge-pair-set-hit s P))
adversary-initial-state ≤
wp-event (checked-staged-security-experiment-with-data-state A)
  (staged-security-trace-fri-pair-challenge-fresh P B)
adversary-initial-state +

```

```

    wp-event (checked-staged-security-experiment-with-data-state A)
      (staged-security-with-data-state-verifier-event
        (λs. trace-fri-challenge-list-prequery-hit s B))
      adversary-initial-state
  proof -
  have event-mono:
    wp-event (checked-staged-security-experiment-with-data-state A)
      (staged-security-with-data-state-verifier-event
        (λs. trace-fri-query-challenge-pair-set-hit s P))
      adversary-initial-state ≤
    wp-event (checked-staged-security-experiment-with-data-state A)
      (λout.
        staged-security-trace-fri-pair-challenge-fresh P B out ∨
        staged-security-with-data-state-verifier-event
          (λs. trace-fri-challenge-list-prequery-hit s B) out)
      adversary-initial-state
  proof (rule wp-event-mono-on-support)
  fix out
  assume hit:
    staged-security-with-data-state-verifier-event
      (λs. trace-fri-query-challenge-pair-set-hit s P) out
  show
    staged-security-trace-fri-pair-challenge-fresh P B out ∨
    staged-security-with-data-state-verifier-event
      (λs. trace-fri-challenge-list-prequery-hit s B) out
  proof (cases out)
  case None
  then show ?thesis
    using hit unfolding staged-security-with-data-state-verifier-event-def
    by simp
  next
  case (Some packed)
  then obtain data attacker-state result final-state where out-eq:
    out = Some (((data, attacker-state), result), final-state)
    by (cases packed) (auto split: prod.splits)
  let ?s =
    verifier-state-from-adversary attacker-state
      (staged-proof-transcript data)
  from hit out-eq have pair-hit:
    trace-fri-query-challenge-pair-set-hit ?s P
    (Some (result, final-state))
  unfolding staged-security-with-data-state-verifier-event-def by simp
  have challenge-hit:
    trace-fri-challenge-list-set-hit ?s B
    (Some (result, final-state))
  by (rule trace-fri-query-challenge-pair-set-hit-imp-challenge-list-set-hit
    [OF pair-hit projection])
  from trace-fri-challenge-list-set-hit-imp-fresh-or-prequery
    [OF challenge-hit]

```

```

show ?thesis
proof
  assume fresh:
    trace-fri-challenge-list-fresh-hit ?s B
    (Some (result, final-state))
  then have staged-security-trace-fri-pair-challenge-fresh P B out
    using hit out-eq unfolding
      staged-security-trace-fri-pair-challenge-fresh-def
      staged-security-with-data-state-verifier-event-def
    by simp
  then show ?thesis by simp
next
  assume prequery:
    trace-fri-challenge-list-prequery-hit ?s B
    (Some (result, final-state))
  then have
    staged-security-with-data-state-verifier-event
    (λs. trace-fri-challenge-list-prequery-hit s B) out
    unfolding out-eq staged-security-with-data-state-verifier-event-def
    by simp
  then show ?thesis by simp
qed
qed
qed
also have ... ≤
  wp-event (checked-staged-security-experiment-with-data-state A)
  (staged-security-trace-fri-pair-challenge-fresh P B)
  adversary-initial-state +
  wp-event (checked-staged-security-experiment-with-data-state A)
  (staged-security-with-data-state-verifier-event
    (λs. trace-fri-challenge-list-prequery-hit s B))
  adversary-initial-state
  by (rule wp-event-union-bound)
finally show ?thesis .
qed

lemma checked-staged-security-trace-fri-query-challenge-pair-set-hit-bound-from-challenge-fresh:
assumes wf: staged-budget-wellformed budgets
and controlled: staged-adversary-controlled budgets A
and finite-B: finite B
and projection: snd ' P ⊆ B
and fresh-branch-bound:
  wp-event (checked-staged-security-experiment-with-data-state A)
  (staged-security-trace-fri-pair-challenge-fresh P B)
  adversary-initial-state ≤ F
shows
  wp-event (checked-staged-security-experiment-with-data-state A)
  (staged-security-with-data-state-verifier-event
    (λs. trace-fri-query-challenge-pair-set-hit s P))

```

$adversary-initial-state \leq$
 $F +$
 $hash-relation-budget-value (card\ B * ceil-log\ clength)$
 $(staged-attacker-query-budget\ budgets + staged-challenge-query-budget)$
proof –
have *split*:
 $wp-event (checked-staged-security-experiment-with-data-state\ A)$
 $(staged-security-with-data-state-verifier-event$
 $(\lambda s. trace-fri-query-challenge-pair-set-hit\ s\ P))$
 $adversary-initial-state \leq$
 $wp-event (checked-staged-security-experiment-with-data-state\ A)$
 $(staged-security-trace-fri-pair-challenge-fresh\ P\ B)$
 $adversary-initial-state +$
 $wp-event (checked-staged-security-experiment-with-data-state\ A)$
 $(staged-security-with-data-state-verifier-event$
 $(\lambda s. trace-fri-challenge-list-prequery-hit\ s\ B))$
 $adversary-initial-state$
by (rule
 $checked-staged-security-trace-fri-query-challenge-pair-set-hit-split-challenge-fresh$
 $[OF\ projection])$
have *prequery-bound*:
 $wp-event (checked-staged-security-experiment-with-data-state\ A)$
 $(staged-security-with-data-state-verifier-event$
 $(\lambda s. trace-fri-challenge-list-prequery-hit\ s\ B))$
 $adversary-initial-state \leq$
 $hash-relation-budget-value (card\ B * ceil-log\ clength)$
 $(staged-attacker-query-budget\ budgets + staged-challenge-query-budget)$
by (rule $checked-staged-security-trace-fri-verifier-prequery-hit-bound$
 $[OF\ wf\ controlled\ finite-B])$
have $wp-event (checked-staged-security-experiment-with-data-state\ A)$
 $(staged-security-with-data-state-verifier-event$
 $(\lambda s. trace-fri-query-challenge-pair-set-hit\ s\ P))$
 $adversary-initial-state \leq$
 $F +$
 $hash-relation-budget-value (card\ B * ceil-log\ clength)$
 $(staged-attacker-query-budget\ budgets + staged-challenge-query-budget)$
by (rule $order-trans[OF\ split]$)
 $(rule\ add-mono[OF\ fresh-branch-bound\ prequery-bound])$
then show *?thesis* .
qed

lemma *checked-staged-security-composition-fri-query-challenge-pair-set-hit-split-challenge-fresh*:
assumes *projection*: $\bigwedge dg. snd\ 'P\ dg \subseteq B\ dg$
shows

$wp-event (checked-staged-security-experiment-with-data-state\ A)$
 $(staged-security-with-data-state-verifier-event$
 $(\lambda s. composition-fri-query-challenge-pair-set-hit\ s\ P))$
 $adversary-initial-state \leq$
 $wp-event (checked-staged-security-experiment-with-data-state\ A)$

```

    (staged-security-composition-fri-pair-challenge-fresh P B)
    adversary-initial-state +
    wp-event (checked-staged-security-experiment-with-data-state A)
    (staged-security-with-data-state-verifier-event
      (λs. composition-fri-challenge-list-prequery-hit s B))
    adversary-initial-state
proof –
  have event-mono:
    wp-event (checked-staged-security-experiment-with-data-state A)
    (staged-security-with-data-state-verifier-event
      (λs. composition-fri-query-challenge-pair-set-hit s P))
    adversary-initial-state ≤
    wp-event (checked-staged-security-experiment-with-data-state A)
    (λout.
      staged-security-composition-fri-pair-challenge-fresh P B out ∨
      staged-security-with-data-state-verifier-event
        (λs. composition-fri-challenge-list-prequery-hit s B) out)
    adversary-initial-state
proof (rule wp-event-mono-on-support)
  fix out
  assume hit:
    staged-security-with-data-state-verifier-event
      (λs. composition-fri-query-challenge-pair-set-hit s P) out
  show
    staged-security-composition-fri-pair-challenge-fresh P B out ∨
    staged-security-with-data-state-verifier-event
      (λs. composition-fri-challenge-list-prequery-hit s B) out
  proof (cases out)
  case None
  then show ?thesis
    using hit unfolding staged-security-with-data-state-verifier-event-def
    by simp
  next
  case (Some packed)
  then obtain data attacker-state result final-state where out-eq:
    out = Some (((data, attacker-state), result), final-state)
    by (cases packed) (auto split: prod.splits)
  let ?s =
    verifier-state-from-adversary attacker-state
    (staged-proof-transcript data)
  from hit out-eq have pair-hit:
    composition-fri-query-challenge-pair-set-hit ?s P
    (Some (result, final-state))
  unfolding staged-security-with-data-state-verifier-event-def by simp
  have challenge-hit:
    composition-fri-challenge-list-set-hit ?s B
    (Some (result, final-state))
  by (rule
    composition-fri-query-challenge-pair-set-hit-imp-challenge-list-set-hit

```

```

    [OF pair-hit projection])
  from composition-fri-challenge-list-set-hit-imp-fresh-or-prequery
    [OF challenge-hit]
  show ?thesis
  proof
    assume fresh:
      composition-fri-challenge-list-fresh-hit ?s B
      (Some (result, final-state))
    then have
      staged-security-composition-fri-pair-challenge-fresh P B out
    using hit out-eq unfolding
      staged-security-composition-fri-pair-challenge-fresh-def
      staged-security-with-data-state-verifier-event-def
    by simp
    then show ?thesis by simp
  next
    assume prequery:
      composition-fri-challenge-list-prequery-hit ?s B
      (Some (result, final-state))
    then have
      staged-security-with-data-state-verifier-event
      (λs. composition-fri-challenge-list-prequery-hit s B) out
    unfolding out-eq staged-security-with-data-state-verifier-event-def
    by simp
    then show ?thesis by simp
  qed
  qed
  qed
  also have ... ≤
    wp-event (checked-staged-security-experiment-with-data-state A)
    (staged-security-composition-fri-pair-challenge-fresh P B)
    adversary-initial-state +
    wp-event (checked-staged-security-experiment-with-data-state A)
    (staged-security-with-data-state-verifier-event
      (λs. composition-fri-challenge-list-prequery-hit s B))
    adversary-initial-state
  by (rule wp-event-union-bound)
  finally show ?thesis .
  qed

lemma checked-staged-security-composition-fri-query-challenge-pair-set-hit-bound-from-challenge-fresh:
  assumes wf: staged-budget-wellformed budgets
  and controlled: staged-adversary-controlled budgets A
  and finite-B:  $\bigwedge dg. \text{finite } (B \ dg)$ 
  and projection:  $\bigwedge dg. \text{snd } 'P \ dg \subseteq B \ dg$ 
  and fresh-branch-bound:
    wp-event (checked-staged-security-experiment-with-data-state A)
    (staged-security-composition-fri-pair-challenge-fresh P B)
    adversary-initial-state ≤ F

```


shows

wp-event (*checked-staged-security-experiment-with-data-state A*)
 (*staged-security-with-data-state-verifier-event*
 ($\lambda s. \text{composition-fri-query-challenge-pair-set-hit } s \ P$))
adversary-initial-state $\leq$
 $F +$
hash-relation-budget-value
 $(\sum dg \in (UNIV :: 'f \text{ set}).$
 $\text{card } (B \ dg) * \text{ceil-log } (\text{maxDegree} + 1))$
 $(\text{staged-attacker-query-budget budgets} + \text{staged-challenge-query-budget})$

proof –

have *split*:

wp-event (*checked-staged-security-experiment-with-data-state A*)
 (*staged-security-with-data-state-verifier-event*
 ($\lambda s. \text{composition-fri-query-challenge-pair-set-hit } s \ P$))
adversary-initial-state $\leq$
wp-event (*checked-staged-security-experiment-with-data-state A*)
 (*staged-security-composition-fri-pair-challenge-fresh P B*)
adversary-initial-state $+$
wp-event (*checked-staged-security-experiment-with-data-state A*)
 (*staged-security-with-data-state-verifier-event*
 ($\lambda s. \text{composition-fri-challenge-list-prequery-hit } s \ B$))
adversary-initial-state
by (*rule*
checked-staged-security-composition-fri-query-challenge-pair-set-hit-split-challenge-fresh
 $[OF \ \text{projection}]$)

have *prequery-bound*:

wp-event (*checked-staged-security-experiment-with-data-state A*)
 (*staged-security-with-data-state-verifier-event*
 ($\lambda s. \text{composition-fri-challenge-list-prequery-hit } s \ B$))
adversary-initial-state $\leq$
hash-relation-budget-value
 $(\sum dg \in (UNIV :: 'f \text{ set}).$
 $\text{card } (B \ dg) * \text{ceil-log } (\text{maxDegree} + 1))$
 $(\text{staged-attacker-query-budget budgets} + \text{staged-challenge-query-budget})$
by (*rule checked-staged-security-composition-fri-verifier-prequery-hit-bound*
 $[OF \ wf \ \text{controlled } \text{finite-}B]$)

have *wp-event* (*checked-staged-security-experiment-with-data-state A*)

(*staged-security-with-data-state-verifier-event*
 ($\lambda s. \text{composition-fri-query-challenge-pair-set-hit } s \ P$))
adversary-initial-state $\leq$
 $F +$
hash-relation-budget-value
 $(\sum dg \in (UNIV :: 'f \text{ set}).$
 $\text{card } (B \ dg) * \text{ceil-log } (\text{maxDegree} + 1))$
 $(\text{staged-attacker-query-budget budgets} + \text{staged-challenge-query-budget})$

by (*rule order-trans* $[OF \ \text{split}]$)
 (*rule add-mono* $[OF \ \text{fresh-branch-bound prequery-bound}]$)

then show *?thesis* .

qed

end

end

theory *Soundness-FRI-Query-Challenge-Actual-Product*
 imports *Soundness-FRI-Query-Challenge-Fresh-Split*
begin

Verifier-local pair-product bounds with actual FRI challenge-path freshness.

This replaces the global trace-FRI future-fresh premise in the fixed challenge product bound by the concrete freshness evidence carried by the trace FRI challenge-list freshness event.

context *soundness*
begin

lemma *verifier-trace-fri-prefix-preserves-query-lookup*:

assumes *outcome*:

Some ((fr, f-fl), t) ∈
 set-dist (execute verifier-trace-fri-prefix s)

shows

fmlookup (HashMap t) (QueryIndexChallenge i x) =
 fmlookup (HashMap s) (QueryIndexChallenge i x)

proof –

let *?key = QueryIndexChallenge i x*

from *outcome* **obtain** *s1 s2* **where**

read-fr: Some (fr, s1) ∈ set-dist (execute read s)

and *trace-fri*:

Some (f-fl, s2) ∈
 set-dist (execute
 (ntimes receive-trace-fri-commits (ceil-log clength)) s1)

and *ret*:

Some ((fr, f-fl), t) ∈ set-dist (execute (return (fr, f-fl)) s2)

unfolding *verifier-trace-fri-prefix-def*

by (*auto elim!: set-dist-bindE*)

have *lookup-s1*:

fmlookup (HashMap s1) ?key = fmlookup (HashMap s) ?key

using *read-preserves-hash-map[OF read-fr]* **by** *simp*

have *lookup-s2*:

fmlookup (HashMap s2) ?key = fmlookup (HashMap s1) ?key

proof (*rule ntimes-preserves-lookup[OF - trace-fri]*)

fix *y u v*

assume *out: Some (y, v) ∈ set-dist (execute receive-trace-fri-commits u)*

obtain *b r* **where** *y-eq: y = (b, r)*

by (*cases y*) *simp*

show *fmlookup (HashMap v) ?key = fmlookup (HashMap u) ?key*

by (*rule receive-trace-fri-commits-preserves-query-lookup*)

```

      [OF out[unfolded y-eq]])
qed
have t-eq: t = s2
  using ret by simp
show ?thesis
  using lookup-s1 lookup-s2 unfolding t-eq by simp
qed

lemma wp-trace-fri-query-challenge-pair-set-hit-fixed-challenge-actual-fresh-bound:
  assumes query-future: query-future-fresh s
  and raw-bound: query-index-raw-preimage-bound
  and challenges-space:
    challenges ∈ fri-challenge-space (ceil-log clength)
  and projection:
    fst ‘(Pairs ∩ (UNIV × {challenges})) ⊆ Queries
  and subset: Queries ⊆ fri-query-index-list-space
  shows
    wp-event verify-monad
      (λout. trace-fri-challenge-list-fresh-hit s {challenges} out ∧
        trace-fri-query-challenge-pair-set-hit s Pairs out) s ≤
      (1 / nnreal (CARD('f) ^ ceil-log clength)) *
      (nnreal (card Queries) *
        (1 / nnreal (card query-sample-space)) ^ rounds)
  proof -
    let ?Hit =
      λout. trace-fri-challenge-list-fresh-hit s {challenges} out ∧
        trace-fri-query-challenge-pair-set-hit s Pairs out
    let ?Head =
      λout. case out of
        None ⇒ False
      | Some ((fr, f-fl), -) ⇒
        map fst f-fl = challenges ∧
        trace-fri-challenge-path-fresh s fr (map snd f-fl)
        (length (map snd f-fl))
    let ?D =
      nnreal (card Queries) *
      (1 / nnreal (card query-sample-space)) ^ rounds
    have bind-bound:
      wp-event verify-monad ?Hit s ≤
        wp-event verifier-trace-fri-prefix ?Head s * ?D
    unfolding verify-monad-trace-fri-decomposition
  proof (rule wp-event-bind-bound-by-head-and-cont)
    show ¬ ?Hit None
      unfolding trace-fri-challenge-list-fresh-hit-def
        trace-fri-query-challenge-pair-set-hit-def
        accepted-fri-challenges-def
        accepted-fri-opening-transcript-def
      by blast
  next

```

```

fix header prefix-state
assume prefix:
  Some (header, prefix-state) ∈
    set-dist (execute verifier-trace-fri-prefix s)
  and not-head:  $\neg$  ?Head (Some (header, prefix-state))
obtain fr f-fl where header-eq: header = (fr, f-fl)
  by (cases header) auto
have not-head':
  map fst f-fl  $\neq$  challenges  $\vee$ 
     $\neg$  trace-fri-challenge-path-fresh s fr (map snd f-fl)
    (length (map snd f-fl))
  using not-head unfolding header-eq by simp
have zero-le:
  wp-event (verifier-after-trace-fri header) ?Hit prefix-state  $\leq 0$ 
proof (rule order-trans)
show wp-event (verifier-after-trace-fri header) ?Hit prefix-state  $\leq$ 
  wp-event (verifier-after-trace-fri header)
  ( $\lambda$ -. False) prefix-state
proof (rule wp-event-mono-on-support)
  fix out
  assume suffix:
    out ∈
      set-dist (execute (verifier-after-trace-fri header) prefix-state)
  and hit: ?Hit out
from hit obtain result :: unit list and final-state where out-eq:
  out = Some (result, final-state)
  unfolding trace-fri-challenge-list-fresh-hit-def
  accepted-fri-challenges-def
  by blast
have prefix':
  Some ((fr, f-fl), prefix-state) ∈
    set-dist (execute verifier-trace-fri-prefix s)
  using prefix header-eq by simp
have suffix':
  Some (result, final-state) ∈
    set-dist
      (execute (verifier-after-trace-fri (fr, f-fl)) prefix-state)
  using suffix header-eq out-eq by simp
from hit obtain trace-roots trace-bs trace-final fri-dg
  composition-roots composition-bs composition-final query-idxs
  trace-round-layers composition-round-layers where
  openings:
    accepted-fri-opening-transcript s out trace-roots trace-bs
    trace-final fri-dg composition-roots composition-bs
    composition-final query-idxs trace-round-layers
    composition-round-layers
  unfolding trace-fri-query-challenge-pair-set-hit-def by blast
have pair-challenges:
  accepted-fri-challenges s out trace-bs fri-dg composition-bs

```

```

    by (rule accepted-fri-opening-transcript-challenges[OF openings])
  from hit obtain trace-bs' dg' comp-bs' where
    fresh-challenges:
      accepted-fri-challenges s out trace-bs' dg' comp-bs'
    and trace-bs'-in: trace-bs' ∈ {challenges}
    unfolding trace-fri-challenge-list-fresh-hit-def by blast
  have trace-bs-eq: trace-bs = trace-bs'
    using accepted-fri-challenges-unique
      [OF pair-challenges[unfolded out-eq]
        fresh-challenges[unfolded out-eq]]
    by simp
  have prefix-trace-eq: trace-bs = map fst f-fl
    by (rule verifier-after-trace-fri-header-agrees-with-accepted-challenges
      [OF prefix' suffix' pair-challenges[unfolded out-eq]])
  have fresh-hit:
    trace-fri-challenge-list-fresh-hit s {challenges}
      (Some (result, final-state))
    using hit out-eq by simp
  have concrete-fresh:
    trace-fri-challenge-path-fresh s fr (map snd f-fl)
      (length (map snd f-fl))
    by (rule verifier-after-trace-fri-header-roots-agree-with-fresh-hit
      [OF prefix' suffix' fresh-hit])
  have head-holds:
    map fst f-fl = challenges ∧
      trace-fri-challenge-path-fresh s fr (map snd f-fl)
      (length (map snd f-fl))
    using prefix-trace-eq trace-bs-eq trace-bs'-in concrete-fresh by simp
  show False
    using not-head' head-holds by blast
qed
next
show wp-event (verifier-after-trace-fri header)
  (λ-. False) prefix-state ≤ 0
  unfolding wp-event-def wp-def dist-expect-def by simp
qed
show wp-event (verifier-after-trace-fri header) ?Hit prefix-state = 0
  by (rule antisym[OF zero-le]) simp
next
fix header prefix-state
assume prefix:
  Some (header, prefix-state) ∈
    set-dist (execute verifier-trace-fri-prefix s)
  and head: ?Head (Some (header, prefix-state))
obtain fr f-fl where header-eq: header = (fr, f-fl)
  by (cases header) auto
have f-fl-eq: map fst f-fl = challenges
  using head unfolding header-eq by simp
have future-prefix: query-future-fresh prefix-state

```

```

by (rule verifier-trace-fri-prefix-preserves-query-future-fresh-exact
  [OF query-future prefix])
have counter-prefix: PQueryCounter prefix-state = PQueryCounter s
using verifier-trace-fri-prefix-outcome[OF prefix[unfolded header-eq]]
by simp
have event-bound:
  wp-event (verifier-after-trace-fri header)
    (λout. ∃ query-idxs ∈ Queries.
      query-rounds-index-list-hit s query-idxs rounds out)
    prefix-state ≤ ?D
by (rule wp-verifier-after-trace-fri-query-index-list-set-bound
  [OF future-prefix counter-prefix raw-bound subset])
have hit-to-query:
  wp-event (verifier-after-trace-fri header) ?Hit prefix-state ≤
    wp-event (verifier-after-trace-fri header)
      (λout. ∃ query-idxs ∈ Queries.
        query-rounds-index-list-hit s query-idxs rounds out) prefix-state
proof (rule wp-event-mono-on-support)
fix out
assume suffix:
  out ∈
    set-dist (execute (verifier-after-trace-fri header) prefix-state)
and hit: ?Hit out
from hit obtain trace-roots trace-bs trace-final fri-dg
  composition-roots composition-bs composition-final query-idxs
  trace-round-layers composition-round-layers where
  openings:
    accepted-fri-opening-transcript s out trace-roots trace-bs
    trace-final fri-dg composition-roots composition-bs
    composition-final query-idxs trace-round-layers
    composition-round-layers
and pair: (query-idxs, trace-bs) ∈ Pairs
unfolding trace-fri-query-challenge-pair-set-hit-def by blast
from openings obtain result :: unit list and final-state where out-eq:
  out = Some (result, final-state)
unfolding accepted-fri-opening-transcript-def by blast
have prefix':
  Some ((fr, f-fl), prefix-state) ∈
    set-dist (execute verifier-trace-fri-prefix s)
using prefix header-eq by simp
have suffix':
  Some (result, final-state) ∈
    set-dist
      (execute (verifier-after-trace-fri (fr, f-fl)) prefix-state)
using suffix header-eq out-eq by simp
have challenges-openings:
  accepted-fri-challenges s out trace-bs fri-dg composition-bs
by (rule accepted-fri-opening-transcript-challenges[OF openings])
have trace-eq: trace-bs = map fst f-fl

```

```

    by (rule verifier-after-trace-fri-header-agrees-with-accepted-challenges
        [OF prefix' suffix' challenges-openings[unfolded out-eq]])
  have query-in: query-idxs ∈ Queries
  proof -
    have query-idxs ∈ fst ‘ (Pairs ∩ (UNIV × {challenges}))
      using pair trace-eq f-fl-eq by force
    then show ?thesis
      using projection by blast
  qed
  have query-hit:
    query-rounds-index-list-hit s query-idxs rounds out
  by (rule accepted-fri-opening-transcript-imp-query-rounds-index-list-hit
      [OF openings])
  show ∃ query-idxs ∈ Queries.
    query-rounds-index-list-hit s query-idxs rounds out
  using query-in query-hit by blast
  qed
  show wp-event (verifier-after-trace-fri header) ?Hit prefix-state ≤ ?D
  by (rule order-trans[OF hit-to-query event-bound])
  qed
  have head-bound:
    wp-event verifier-trace-fri-prefix ?Head s ≤
      1 / nnreal (CARD('f) ^ ceil-log clength)
  proof -
    let ?HeadSet =
      λout. case out of
        None ⇒ False
      | Some ((fr, f-fl), -) ⇒
        map fst f-fl ∈ {challenges} ∧
        trace-fri-challenge-path-fresh s fr (map snd f-fl)
        (length (map snd f-fl))
    have subset: {challenges} ⊆ fri-challenge-space (ceil-log clength)
      using challenges-space by simp
    have mono:
      wp-event verifier-trace-fri-prefix ?Head s ≤
        wp-event verifier-trace-fri-prefix ?HeadSet s
    by (rule wp-event-mono) (auto split: option.splits prod.splits)
    have wp-event verifier-trace-fri-prefix ?Head s ≤
      nnreal (card {challenges}) /
      nnreal (CARD('f) ^ ceil-log clength)
    by (rule order-trans[OF mono])
      (rule wp-verifier-trace-fri-prefix-challenge-list-actual-fresh-bound
        [OF subset])
    then show ?thesis
      by simp
  qed
  have wp-event verify-monad ?Hit s ≤
    wp-event verifier-trace-fri-prefix ?Head s * ?D
  by (rule bind-bound)

```

also have ... $\leq$
 (1 / nnreal (CARD('f) $\wedge$ ceil-log clength)) * ?D
 by (rule mult-right-mono[OF head-bound]) simp
 finally show ?thesis .
 qed

lemma wp-trace-fri-query-challenge-pair-set-hit-challenge-fresh-fiber-sum:
assumes query-future: query-future-fresh s
and raw-bound: query-index-raw-preimage-bound
and challenge-subset: $B \subseteq \text{fri-challenge-space (ceil-log clength)}$
and projection:
 $\bigwedge \text{challenges. challenges} \in B \implies$
 $\text{fst } '(P \cap (UNIV \times \{\text{challenges}\})) \subseteq Q \text{ challenges}$
and subset:
 $\bigwedge \text{challenges. challenges} \in B \implies$
 $Q \text{ challenges} \subseteq \text{fri-query-index-list-space}$
shows
 wp-event verify-monad
 ($\lambda \text{out. trace-fri-challenge-list-fresh-hit } s \text{ } B \text{ out} \wedge$
 $\text{trace-fri-query-challenge-pair-set-hit } s \text{ } P \text{ out}) \text{ } s \leq$
 ($\sum \text{challenges} \in B.$
 (1 / nnreal (CARD('f) $\wedge$ ceil-log clength)) *
 (nnreal (card (Q challenges)) *
 (1 / nnreal (card query-sample-space)) $\wedge$ rounds))

proof –
 let ?E =
 $\lambda \text{challenges out.}$
 $\text{trace-fri-challenge-list-fresh-hit } s \text{ } \{\text{challenges}\} \text{ out} \wedge$
 $\text{trace-fri-query-challenge-pair-set-hit } s \text{ } P \text{ out}$
 let ?C =
 $\lambda \text{challenges.}$
 (1 / nnreal (CARD('f) $\wedge$ ceil-log clength)) *
 (nnreal (card (Q challenges)) *
 (1 / nnreal (card query-sample-space)) $\wedge$ rounds)
have split:
 wp-event verify-monad
 ($\lambda \text{out. trace-fri-challenge-list-fresh-hit } s \text{ } B \text{ out} \wedge$
 $\text{trace-fri-query-challenge-pair-set-hit } s \text{ } P \text{ out}) \text{ } s \leq$
 wp-event verify-monad
 ($\lambda \text{out. } \exists \text{challenges} \in B. ?E \text{ challenges out}) \text{ } s$
proof (rule wp-event-mono)
fix out
assume hit:
 $\text{trace-fri-challenge-list-fresh-hit } s \text{ } B \text{ out} \wedge$
 $\text{trace-fri-query-challenge-pair-set-hit } s \text{ } P \text{ out}$
then obtain trace-bs :: 'f list **and** dg :: 'f **and** comp-bs :: 'f list
and fr :: 'f **and** trace-roots :: 'f list **and** trace-final :: 'f
and as :: 'f list **and** composition-roots :: 'f list


```

    and final :: 'f and rest :: 'f list where
accepted:
    accepted-fri-challenges s out trace-bs dg comp-bs
and header:
    verifier-header-transcript s fr trace-roots trace-final as dg
    composition-roots final rest
and trace-bs-in: trace-bs ∈ B
and fresh-path:
    trace-fri-challenge-path-fresh s fr trace-roots
    (length trace-roots)
unfolding trace-fri-challenge-list-fresh-hit-def by blast
have singleton-fresh:
    trace-fri-challenge-list-fresh-hit s {trace-bs} out
    unfolding trace-fri-challenge-list-fresh-hit-def
proof (intro exI conjI)
    show accepted-fri-challenges s out trace-bs dg comp-bs
        by (rule accepted)
    show verifier-header-transcript s fr trace-roots trace-final as dg
        composition-roots final rest
        by (rule header)
    show trace-bs ∈ {trace-bs}
        by simp
    show trace-fri-challenge-path-fresh s fr trace-roots
        (length trace-roots)
        by (rule fresh-path)
qed

show ∃ challenges ∈ B. ?E challenges out
    using singleton-fresh hit trace-bs-in
    by (intro bexI[of - trace-bs]) simp-all
qed
also have ... ≤ (∑ challenges ∈ B. ?C challenges)
proof (rule wp-event-finite-union-bound-fri-exact)
    show finite B
        using challenge-subset by (rule finite-subset)
        (rule finite-fri-challenge-space)
next
fix challenges
assume challenges-in: challenges ∈ B
have challenges-space:
    challenges ∈ fri-challenge-space (ceil-log clength)
    using challenge-subset challenges-in by blast
show wp-event verify-monad (?E challenges) s ≤ ?C challenges
    by (rule
        wp-trace-fri-query-challenge-pair-set-hit-fixed-challenge-actual-fresh-bound
        [OF query-future raw-bound challenges-space
            projection[OF challenges-in] subset[OF challenges-in]])
qed
finally show ?thesis .

```

qed

lemma *wp-composition-fri-query-challenge-pair-set-hit-fixed-challenge-actual-fresh-bound:*

assumes *query-future: query-future-fresh s*
and *raw-bound: query-index-raw-preimage-bound*
and *challenges-space:*
 challenges ∈ fri-challenge-space (ceil-log (to-nat dg + 1))
and *projection:*
 fst ‘ (Pairs dg ∩ (UNIV × {challenges})) ⊆ Queries
and *subset: Queries ⊆ fri-query-index-list-space*
shows
 wp-event verify-monad
 ($\lambda out.$
 composition-fri-challenge-list-fresh-hit s
 ($\lambda dg'. \text{if } dg' = dg \text{ then } \{challenges\} \text{ else } \{\}$) *out* $\wedge$
 composition-fri-query-challenge-pair-set-hit s Pairs out) *s* $\leq$
 ($1 / \text{nnreal } (CARD('f) \wedge \text{ceil-log } (to-nat dg + 1))$) *
 ($\text{nnreal } (card \text{ Queries})$) *
 ($1 / \text{nnreal } (card \text{ query-sample-space})$) $\wedge$ *rounds*)

proof –

let *?B* = $\lambda dg'. \text{if } dg' = dg \text{ then } \{challenges\} \text{ else } \{\}$
let *?Hit* =
 $\lambda out. \text{composition-fri-challenge-list-fresh-hit } s \text{ ?B } out \wedge$
 composition-fri-query-challenge-pair-set-hit s Pairs out
let *?Head* =
 $\lambda out. \text{case } out \text{ of}$
 None $\Rightarrow$ *False*
 | *Some ((fr, f-fl, f-final, as, dg', fl), -)* $\Rightarrow$
 $dg' = dg \wedge \text{map } fst \text{ fl} = challenges \wedge$
 composition-fri-challenge-path-fresh s fr (map snd f-fl)
 f-final as dg' (map snd fl) (length (map snd fl))
let *?D* =
 $\text{nnreal } (card \text{ Queries})$ *
 ($1 / \text{nnreal } (card \text{ query-sample-space})$) $\wedge$ *rounds*

have *bind-bound:*

wp-event verify-monad ?Hit s $\leq$
 wp-event verifier-composition-fri-prefix ?Head s * *?D*
unfolding *verify-monad-composition-fri-decomposition*

proof (*rule wp-event-bind-bound-by-head-and-cont*)

show $\neg ?Hit \text{ None}$

unfolding *composition-fri-challenge-list-fresh-hit-def*
 composition-fri-query-challenge-pair-set-hit-def
 accepted-fri-challenges-def
 accepted-fri-opening-transcript-def

by *blast*

next

fix *header prefix-state*

assume *prefix:*

Some (header, prefix-state) ∈

```

    set-dist (execute verifier-composition-fri-prefix s)
  and not-head:  $\neg$  ?Head (Some (header, prefix-state))
obtain fr f-fl f-final as dg' fl where header-eq:
  header = (fr, f-fl, f-final, as, dg', fl)
by (cases header) auto
have not-head':
  dg'  $\neq$  dg  $\vee$  map fst fl  $\neq$  challenges  $\vee$ 
     $\neg$  composition-fri-challenge-path-fresh s fr (map snd f-fl)
    f-final as dg' (map snd fl) (length (map snd fl))
using not-head unfolding header-eq by simp
have zero-le:
  wp-event (verifier-after-composition-fri header) ?Hit prefix-state  $\leq$  0
proof (rule order-trans)
show wp-event (verifier-after-composition-fri header) ?Hit prefix-state  $\leq$ 
  wp-event (verifier-after-composition-fri header)
    ( $\lambda$ -. False) prefix-state
proof (rule wp-event-mono-on-support)
fix out
assume suffix:
  out  $\in$ 
    set-dist
    (execute (verifier-after-composition-fri header) prefix-state)
and hit: ?Hit out
from hit obtain result :: unit list and final-state where out-eq:
  out = Some (result, final-state)
unfolding composition-fri-challenge-list-fresh-hit-def
  accepted-fri-challenges-def
by blast
have prefix':
  Some ((fr, f-fl, f-final, as, dg', fl), prefix-state)  $\in$ 
    set-dist (execute verifier-composition-fri-prefix s)
using prefix header-eq by simp
have suffix':
  Some (result, final-state)  $\in$ 
    set-dist
    (execute
      (verifier-after-composition-fri
        (fr, f-fl, f-final, as, dg', fl))
      prefix-state)
using suffix header-eq out-eq by simp
from hit obtain trace-roots trace-bs trace-final fri-dg
  composition-roots composition-bs composition-final query-idxs
  trace-round-layers composition-round-layers where
  openings:
    accepted-fri-opening-transcript s out trace-roots trace-bs
    trace-final fri-dg composition-roots composition-bs
    composition-final query-idxs trace-round-layers
    composition-round-layers
unfolding composition-fri-query-challenge-pair-set-hit-def by blast

```

```

have pair-challenges:
  accepted-fri-challenges s out trace-bs fri-dg composition-bs
  by (rule accepted-fri-opening-transcript-challenges[OF openings])
from hit obtain trace-bs' dg'' comp-bs' where
  fresh-challenges:
    accepted-fri-challenges s out trace-bs' dg'' comp-bs'
  and comp-bs'-in:
    comp-bs' ∈ (if dg'' = dg then {challenges} else {})
  unfolding composition-fri-challenge-list-fresh-hit-def by blast
have challenge-eqs:
  fri-dg = dg'' ∧ composition-bs = comp-bs'
  using accepted-fri-challenges-unique
    [OF pair-challenges[unfolded out-eq]
     fresh-challenges[unfolded out-eq]]
  by simp
have prefix-eqs:
  fri-dg = dg' ∧ composition-bs = map fst fl
  by (rule verifier-after-composition-fri-header-agrees-with-accepted-challenges
    [OF prefix' suffix' pair-challenges[unfolded out-eq]])
have fresh-hit:
  composition-fri-challenge-list-fresh-hit s ?B
  (Some (result, final-state))
  using hit out-eq by simp
have concrete-fresh:
  composition-fri-challenge-path-fresh s fr (map snd f-fl)
  f-final as dg' (map snd fl) (length (map snd fl))
  by (rule verifier-after-composition-fri-header-roots-agree-with-fresh-hit
    [OF prefix' suffix' fresh-hit])
have dg-eq': dg'' = dg
  using comp-bs'-in by (cases dg'' = dg) auto
have comp-bs'-eq: comp-bs' = map fst fl
  using challenge-eqs prefix-eqs by simp
have comp-bs'-challenges: comp-bs' = challenges
  using comp-bs'-in dg-eq' by simp
have dg'-eq: dg' = dg
  using challenge-eqs prefix-eqs dg-eq' by simp
have fl-challenges: map fst fl = challenges
  using comp-bs'-eq comp-bs'-challenges by simp
have head-holds:
  dg' = dg ∧ map fst fl = challenges ∧
    composition-fri-challenge-path-fresh s fr (map snd f-fl)
    f-final as dg' (map snd fl) (length (map snd fl))
  using dg'-eq fl-challenges concrete-fresh by simp
show False
  using not-head' head-holds by blast
qed
next
show wp-event (verifier-after-composition-fri header)
  (λ-. False) prefix-state ≤ 0

```

```

    unfolding wp-event-def wp-def dist-expect-def by simp
qed
show wp-event (verifier-after-composition-fri header) ?Hit prefix-state = 0
  by (rule antisym[OF zero-le]) simp
next
fix header prefix-state
assume prefix:
  Some (header, prefix-state) ∈
    set-dist (execute verifier-composition-fri-prefix s)
  and head: ?Head (Some (header, prefix-state))
obtain fr f-fl f-final as dg' fl where header-eq:
  header = (fr, f-fl, f-final, as, dg', fl)
  by (cases header) auto
have dg-eq: dg' = dg and fl-eq: map fst fl = challenges
  using head unfolding header-eq by simp-all
have future-prefix: query-future-fresh prefix-state
  by (rule verifier-composition-fri-prefix-preserves-query-future-fresh
    [OF query-future prefix])
have counter-prefix: PQueryCounter prefix-state = PQueryCounter s
  using verifier-composition-fri-prefix-outcome[OF prefix[unfolded header-eq]]
  by simp
have event-bound:
  wp-event (verifier-after-composition-fri header)
    (λout. ∃ query-idxs ∈ Queries.
      query-rounds-index-list-hit s query-idxs rounds out)
    prefix-state ≤ ?D
  by (rule wp-verifier-after-composition-fri-query-index-list-set-bound
    [OF future-prefix counter-prefix raw-bound subset])
have hit-to-query:
  wp-event (verifier-after-composition-fri header) ?Hit prefix-state ≤
    wp-event (verifier-after-composition-fri header)
      (λout. ∃ query-idxs ∈ Queries.
        query-rounds-index-list-hit s query-idxs rounds out) prefix-state
proof (rule wp-event-mono-on-support)
fix out
assume suffix:
  out ∈
    set-dist
      (execute (verifier-after-composition-fri header) prefix-state)
  and hit: ?Hit out
from hit obtain trace-roots trace-bs trace-final fri-dg
  composition-roots composition-bs composition-final query-idxs
  trace-round-layers composition-round-layers where
openings:
  accepted-fri-opening-transcript s out trace-roots trace-bs
  trace-final fri-dg composition-roots composition-bs
  composition-final query-idxs trace-round-layers
  composition-round-layers
and pair: (query-idxs, composition-bs) ∈ Pairs fri-dg

```

```

unfolding composition-fri-query-challenge-pair-set-hit-def by blast
from openings obtain result :: unit list and final-state where out-eq:
  out = Some (result, final-state)
unfolding accepted-fri-opening-transcript-def by blast
have prefix':
  Some ((fr, f-fl, f-final, as, dg', fl), prefix-state) ∈
    set-dist (execute verifier-composition-fri-prefix s)
using prefix header-eq by simp
have suffix':
  Some (result, final-state) ∈
    set-dist
      (execute
        (verifier-after-composition-fri
          (fr, f-fl, f-final, as, dg', fl))
        prefix-state)
using suffix header-eq out-eq by simp
have challenges-openings:
  accepted-fri-challenges s out trace-bs fri-dg composition-bs
by (rule accepted-fri-opening-transcript-challenges[OF openings])
have eqs: fri-dg = dg' ∧ composition-bs = map fst fl
by (rule verifier-after-composition-fri-header-agrees-with-accepted-challenges
  [OF prefix' suffix' challenges-openings[unfolded out-eq]])
have query-in: query-idxs ∈ Queries
proof –
  have query-idxs ∈ fst ‘ (Pairs dg ∩ (UNIV × {challenges}))
    using pair eqs dg-eq fl-eq by force
  then show ?thesis
    using projection by blast
qed
have query-hit:
  query-rounds-index-list-hit s query-idxs rounds out
by (rule accepted-fri-opening-transcript-imp-query-rounds-index-list-hit
  [OF openings])
show  $\exists$  query-idxs ∈ Queries.
  query-rounds-index-list-hit s query-idxs rounds out
using query-in query-hit by blast
qed
show wp-event (verifier-after-composition-fri header) ?Hit prefix-state ≤
  ?D
by (rule order-trans[OF hit-to-query event-bound])
qed
have head-bound:
  wp-event verifier-composition-fri-prefix ?Head s ≤
     $1 / \text{nnreal } (\text{CARD}('f) \wedge \text{ceil-log } (\text{to-nat } dg + 1))$ 
proof –
  let ?B = λdg'. if dg' = dg then {challenges} else {}
  let ?HeadSet =
     $\lambda out. \text{case out of}$ 
    None ⇒ False

```

```

| Some ((fr, f-fl, f-final, as, dg', fl), -) ⇒
  map fst fl ∈ ?B dg' ∧
  composition-fri-challenge-path-fresh s fr (map snd f-fl)
  f-final as dg' (map snd fl) (length (map snd fl))
have subset:
  ∧ dg'. ?B dg' ⊆
    fri-challenge-space (ceil-log (to-nat dg' + 1))
  using challenges-space by auto
have bound:
  ∧ dg'. nnreal (card (?B dg')) /
    nnreal (CARD('f) ^ ceil-log (to-nat dg' + 1)) ≤
    1 / nnreal (CARD('f) ^ ceil-log (to-nat dg + 1))
  using challenges-space by auto
have mono:
  wp-event verifier-composition-fri-prefix ?Head s ≤
    wp-event verifier-composition-fri-prefix ?HeadSet s
  by (rule wp-event-mono) (auto split: option.splits prod.splits)
show ?thesis
  by (rule order-trans[OF mono])
    (rule wp-verifier-composition-fri-prefix-challenge-list-actual-fresh-bound
      [OF subset bound])
qed
have wp-event verify-mono ?Hit s ≤
  wp-event verifier-composition-fri-prefix ?Head s * ?D
  by (rule bind-bound)
also have ... ≤
  (1 / nnreal (CARD('f) ^ ceil-log (to-nat dg + 1))) * ?D
  by (rule mult-right-mono[OF head-bound]) simp
finally show ?thesis .
qed

lemma wp-composition-fri-query-challenge-pair-set-hit-challenge-fresh-fiber-sum:
  assumes query-future: query-future-fresh s
  and raw-bound: query-index-raw-preimage-bound
  and challenge-subset:
    ∧ dg. B dg ⊆ fri-challenge-space (ceil-log (to-nat dg + 1))
  and projection:
    ∧ dg challenges.
      challenges ∈ B dg ⇒
      fst ' (P dg ∩ (UNIV × {challenges})) ⊆
        Q dg challenges
  and subset:
    ∧ dg challenges. challenges ∈ B dg ⇒
      Q dg challenges ⊆ fri-query-index-list-space
  shows
    wp-event verify-mono
    (λout. composition-fri-challenge-list-fresh-hit s B out ∧
      composition-fri-query-challenge-pair-set-hit s P out) s ≤
    (∑ dg ∈ UNIV.

```

```

       $\sum challenges \in B \text{ dg.}$ 
       $(1 / \text{nnreal} (\text{CARD}('f) \wedge \text{ceil-log} (\text{to-nat } dg + 1))) *$ 
       $(\text{nnreal} (\text{card } (Q \text{ dg } challenges)) *$ 
       $(1 / \text{nnreal} (\text{card } \text{query-sample-space})) \wedge \text{rounds}))$ 
proof –
  let  $?I = \text{SIGMA } dg:UNIV. B \text{ dg}$ 
  let  $?E =$ 
     $\lambda p \text{ out.}$ 
     $\text{composition-fri-challenge-list-fresh-hit } s$ 
     $(\lambda dg'. \text{ if } dg' = \text{fst } p \text{ then } \{\text{snd } p\} \text{ else } \{\}) \text{ out} \wedge$ 
     $\text{composition-fri-query-challenge-pair-set-hit } s \text{ } P \text{ out}$ 
  let  $?C =$ 
     $\lambda p.$ 
     $(1 / \text{nnreal} (\text{CARD}('f) \wedge \text{ceil-log} (\text{to-nat } (\text{fst } p) + 1))) *$ 
     $(\text{nnreal} (\text{card } (Q (\text{fst } p) (\text{snd } p)))) *$ 
     $(1 / \text{nnreal} (\text{card } \text{query-sample-space})) \wedge \text{rounds})$ 
  have split:
     $\text{wp-event verify-monad}$ 
     $(\lambda out. \text{composition-fri-challenge-list-fresh-hit } s \text{ } B \text{ out} \wedge$ 
     $\text{composition-fri-query-challenge-pair-set-hit } s \text{ } P \text{ out}) \text{ } s \leq$ 
     $\text{wp-event verify-monad}$ 
     $(\lambda out. \exists p \in ?I. ?E \text{ } p \text{ out}) \text{ } s$ 
  proof (rule wp-event-mono)
  fix out
  assume hit:
     $\text{composition-fri-challenge-list-fresh-hit } s \text{ } B \text{ out} \wedge$ 
     $\text{composition-fri-query-challenge-pair-set-hit } s \text{ } P \text{ out}$ 
  then obtain trace-bs dg comp-bs fr trace-roots trace-final as
     $\text{composition-roots final rest}$  where
    accepted:
       $\text{accepted-fri-challenges } s \text{ out trace-bs dg comp-bs}$ 
    and comp-bs-in:  $\text{comp-bs} \in B \text{ dg}$ 
    and header:
       $\text{verifier-header-transcript } s \text{ fr trace-roots trace-final as dg}$ 
       $\text{composition-roots final rest}$ 
    and fresh-path:
       $\text{composition-fri-challenge-path-fresh } s \text{ fr trace-roots trace-final}$ 
       $\text{as dg composition-roots (length composition-roots)}$ 
    unfolding composition-fri-challenge-list-fresh-hit-def by blast
  have singleton-fresh:
     $\text{composition-fri-challenge-list-fresh-hit } s$ 
     $(\lambda dg'. \text{ if } dg' = dg \text{ then } \{\text{comp-bs}\} \text{ else } \{\}) \text{ out}$ 
    unfolding composition-fri-challenge-list-fresh-hit-def
  proof (intro exI conjI)
  show  $\text{accepted-fri-challenges } s \text{ out trace-bs dg comp-bs}$ 
    by (rule accepted)
  show  $\text{verifier-header-transcript } s \text{ fr trace-roots trace-final as dg}$ 
     $\text{composition-roots final rest}$ 
    by (rule header)

```



```

show comp-bs ∈ (if dg = dg then {comp-bs} else {})
  by simp
show composition-fri-challenge-path-fresh s fr trace-roots trace-final
  as dg composition-roots (length composition-roots)
  by (rule fresh-path)
qed
have p-in: (dg, comp-bs) ∈ ?I
  using comp-bs-in by simp
have singleton-fresh':
  composition-fri-challenge-list-fresh-hit s
  (λdg'. if dg' = fst (dg, comp-bs) then {snd (dg, comp-bs)}
    else {}) out
  using singleton-fresh by (simp only: fst-conv snd-conv)
have e-hit: ?E (dg, comp-bs) out
  using singleton-fresh' hit by simp
show ∃ p ∈ ?I. ?E p out
  using p-in e-hit by blast
qed
also have ... ≤ (∑ p ∈ ?I. ?C p)
proof (rule wp-event-finite-union-bound-fri-exact)
  show finite ?I
    apply (rule finite-SigmaI)
    using challenge-subset finite-fri-challenge-space
    apply (auto intro: finite-subset)
    by (meson finite-fri-challenge-space rev-finite-subset)
next
fix p
assume p-in: p ∈ ?I
obtain dg challenges where p-eq: p = (dg, challenges)
  by (cases p) auto
have challenges-in: challenges ∈ B dg
  using p-in p-eq by simp
have challenges-space:
  challenges ∈ fri-challenge-space (ceil-log (to-nat dg + 1))
  using challenge-subset challenges-in by blast
show wp-event verify-monad (?E p) s ≤ ?C p
  unfolding p-eq
apply (rule wp-composition-fri-query-challenge-pair-set-hit-fixed-challenge-actual-fresh-bound)
  using query-future raw-bound challenges-space
  projection[OF challenges-in] subset[OF challenges-in]
  by auto
qed
finally have bound:
  wp-event verify-monad
  (λout. composition-fri-challenge-list-fresh-hit s B out ∧
    composition-fri-query-challenge-pair-set-hit s P out) s ≤
  (∑ p ∈ ?I. ?C p) .
have finite-B: ∧dg. finite (B dg)
  by (meson challenge-subset finite-fri-challenge-space finite-subset)

```

```

have sigma-eq:
  ( $\sum p \in ?I. ?C\ p$ ) =
    ( $\sum dg \in UNIV.$ 
       $\sum challenges \in B\ dg.$ 
        ( $1 / nnreal\ (CARD(f) \wedge \text{ceil-log}\ (to\text{-nat}\ dg + 1))$ ) *
        ( $nnreal\ (card\ (Q\ dg\ challenges))$ ) *
        ( $1 / nnreal\ (card\ query\text{-sample}\text{-space})$ )  $\wedge$  rounds))
  by (subst sum.Sigma) (simp-all add: finite-B case-prod-beta)
show ?thesis
  using bound sigma-eq by simp
qed

definition staged-security-trace-fri-pair-challenge-query-path-fresh
  :: (nat list  $\times$  'f list) set  $\Rightarrow$  'f list set  $\Rightarrow$ 
    nat list set  $\Rightarrow$ 
    (((f staged-proof-data  $\times$  'f protocol-channel)  $\times$ 
      unit list)  $\times$  'f protocol-channel) option  $\Rightarrow$  bool

where
  staged-security-trace-fri-pair-challenge-query-path-fresh P B Q out
     $\longleftrightarrow$ 
    staged-security-trace-fri-pair-challenge-fresh P B out  $\wedge$ 
    staged-security-with-data-state-query-index-list-path-fresh Q out

definition staged-security-composition-fri-pair-challenge-query-path-fresh
  :: (f  $\Rightarrow$  (nat list  $\times$  'f list) set)  $\Rightarrow$ 
    (f  $\Rightarrow$  'f list set)  $\Rightarrow$  nat list set  $\Rightarrow$ 
    (((f staged-proof-data  $\times$  'f protocol-channel)  $\times$ 
      unit list)  $\times$  'f protocol-channel) option  $\Rightarrow$  bool

where
  staged-security-composition-fri-pair-challenge-query-path-fresh P B Q out
     $\longleftrightarrow$ 
    staged-security-composition-fri-pair-challenge-fresh P B out  $\wedge$ 
    staged-security-with-data-state-query-index-list-path-fresh Q out

definition verifier-after-trace-fri-raw-list-path-hit
  :: 'f  $\times$  (f  $\times$  'f) list  $\Rightarrow$ 
    (f, 'a) protocol-channel-scheme  $\Rightarrow$  nat list set  $\Rightarrow$ 
    (unit list  $\times$  (f, 'a) protocol-channel-scheme) option  $\Rightarrow$  bool

where
  verifier-after-trace-fri-raw-list-path-hit header s Q out  $\longleftrightarrow$ 
    ( $\exists fr\ f\text{-}fl\ f\text{-}final\ s3\ as\ s4\ dg\ s5\ s6\ fl\ s7\ final\ query\text{-}state\ raws.$ 
      header = (fr, f-fl)  $\wedge$ 
      Some (f-final, s3)  $\in$  set-dist (execute read s)  $\wedge$ 
      Some (as, s4)  $\in$ 
        set-dist (execute (mmap (replicate (length spec) alpha-round)) s3)  $\wedge$ 
      Some (dg, s5)  $\in$  set-dist (execute read s4)  $\wedge$ 
      Some ((), s6)  $\in$ 
        set-dist (execute (assert (to-nat dg  $\leq$  maxDegree)) s5)  $\wedge$ 
      Some (fl, s7)  $\in$ 

```

set-dist (*execute*
 (*ntimes receive-composition-fri-commits*
 (*ceil-log (to-nat dg + 1)*)) *s6*) $\wedge$
Some (*final*, *query-state*) $\in$ *set-dist* (*execute read s7*) $\wedge$
rows $\in$ *query-index-row-list-preimage Q* $\wedge$
query-rounds-row-list-path-hit query-state rows rounds out)

lemma *verifier-after-trace-fri-staged-query-path-fresh-imp-row-list-path-hit:*

assumes *wf: staged-budget-wellformed budgets*

and *controlled: staged-adversary-controlled budgets A*

and *builder:*

Some (*data*, *attacker-state*) $\in$
set-dist
 (*execute (checked-staged-transcript-program A)*
adversary-initial-state)

and *prefix:*

Some ((*fr*, *f-fl*), *prefix-state*) $\in$
set-dist
 (*execute verifier-trace-fri-prefix*
 (*verifier-state-from-adversary attacker-state*
 (*staged-proof-transcript data*)))

and *suffix:*

out $\in$
set-dist
 (*execute (verifier-after-trace-fri (fr, f-fl)) prefix-state*)

and *hit:*

staged-security-with-data-state-query-index-list-path-fresh Q
 (*case out of*
None $\Rightarrow$ *None*
 | *Some* (*result*, *final-state*) $\Rightarrow$
Some (((*data*, *attacker-state*), *result*), *final-state*))

shows *verifier-after-trace-fri-row-list-path-hit (fr, f-fl) prefix-state Q out*

proof (*cases out*)

case *None*

then show *?thesis*

using *hit*

unfolding *staged-security-with-data-state-query-index-list-path-fresh-def*

by *simp*

next

case (*Some pair*)

then obtain *result :: unit list* **and** *final-state* **where** *out-eq:*

out = *Some* (*result*, *final-state*)

by (*cases pair*) *simp*

let *?s* =

verifier-state-from-adversary attacker-state
 (*staged-proof-transcript data*)

from *suffix*[*unfolded out-eq verifier-after-trace-fri-def*]

obtain *f-final s3 as s4 dg s5 s6 fl s7 final query-state* **where**

read-trace-final:

```

    Some (f-final, s3) ∈ set-dist (execute read prefix-state)
and alpha-out:
    Some (as, s4) ∈
      set-dist (execute (mmap (replicate (length spec) alpha-round)) s3)
and read-dg: Some (dg, s5) ∈ set-dist (execute read s4)
and degree-assert:
    Some ((), s6) ∈
      set-dist (execute (assert (to-nat dg ≤ maxDegree)) s5)
and comp-fri:
    Some (fl, s7) ∈
      set-dist (execute
        (ntimes receive-composition-fri-commits
          (ceil-log (to-nat dg + 1))) s6)
and read-final:
    Some (final, query-state) ∈ set-dist (execute read s7)
and query-out:
    Some (result, final-state) ∈
      set-dist
        (execute
          (ntimes
            (verifier-query-round-program fr f-fl f-final as fl final)
            rounds)
          query-state)
by (auto elim!: set-dist-bindE)
from ntimes-verifier-query-rounds-outcome[OF query-out]
obtain raw-idxs query-idxs query-chunks where
  len-raw: length raw-idxs = rounds
and query-idxs-eq:
  query-idxs = map (λraw. index (to-nat raw)) raw-idxs
and len-chunks: length query-chunks = rounds
and tr-query:
  PTranscript query-state =
    List.concat query-chunks @ PTranscript final-state
and state-final:
  PState final-state =
    state-after-query-chunks (PState query-state) query-chunks rounds
and round-chunks:
  ∧ i. i < rounds ⇒
    verifier-query-round-chunk (query-idxs ! i)
      (map snd f-fl) (map snd fl) (query-chunks ! i)
and lookups:
  ∧ i. i < rounds ⇒
    fmlookup (HashMap final-state)
      (QueryIndexChallenge (PQueryCounter query-state + i)
        (state-after-query-chunks
          (PState query-state) query-chunks i)) =
      Some (raw-idxs ! i)
by blast
have trace-prefix-res:

```

```

length f-fl = ceil-log clength ∧
  PTranscript ?s = [fr] @ map snd f-fl @ PTranscript prefix-state ∧
  PState prefix-state =
    foldl concat (concat (PState ?s) fr) (map snd f-fl) ∧
  PQueryCounter prefix-state = PQueryCounter ?s
using verifier-trace-fri-prefix-outcome[OF prefix] by simp
from read-outcome[OF read-trace-final] obtain rest3 where
  tr-prefix: PTranscript prefix-state = f-final # rest3
and tr-s3: PTranscript s3 = rest3
and state-s3: PState s3 = concat (PState prefix-state) f-final
and counter-s3: PQueryCounter s3 = PQueryCounter prefix-state
by blast
have alpha-res:
  length as = length spec ∧
  PTranscript s3 = as @ PTranscript s4 ∧
  PState s4 = foldl concat (PState s3) as ∧
  PQueryCounter s4 = PQueryCounter s3
using mmap-alpha-round-outcome[OF alpha-out] by simp
from read-outcome[OF read-dg] obtain rest5 where
  tr-s4: PTranscript s4 = dg # rest5
and tr-s5: PTranscript s5 = rest5
and state-s5: PState s5 = concat (PState s4) dg
and counter-s5: PQueryCounter s5 = PQueryCounter s4
by blast
have s6-eq: s6 = s5
using degree-assert unfolding assert-def
by (cases to-nat dg ≤ maxDegree) (auto simp: throw-no-outcome)
have comp-res:
  length fl = ceil-log (to-nat dg + 1) ∧
  PTranscript s6 = map snd fl @ PTranscript s7 ∧
  PState s7 = foldl concat (PState s6) (map snd fl) ∧
  PQueryCounter s7 = PQueryCounter s6
using ntimes-receive-composition-fri-commits-outcome[OF comp-fri]
by simp
from read-outcome[OF read-final] obtain rest-query where
  tr-s7: PTranscript s7 = final # rest-query
and tr-read-query: PTranscript query-state = rest-query
and state-query: PState query-state = concat (PState s7) final
and counter-query-s7: PQueryCounter query-state = PQueryCounter s7
by blast
have local-counter: PQueryCounter query-state = 0
using trace-prefix-res counter-s3 alpha-res counter-s5 s6-eq comp-res
  counter-query-s7
unfolding verifier-state-from-adversary-def by simp
have prefix-lookup:
  ∧ i x. fmllookup (HashMap prefix-state)
    (QueryIndexChallenge i x) =
    fmllookup (HashMap ?s) (QueryIndexChallenge i x)
by (rule verifier-trace-fri-prefix-preserves-query-lookup[OF prefix])

```

```

have s3-lookup:
  HashMap s3 = HashMap prefix-state
  by (rule read-preserves-hash-map[OF read-trace-final])
have s4-lookup:
   $\bigwedge i x. \text{fmlookup } (\text{HashMap } s4) (\text{QueryIndexChallenge } i x) =$ 
    fmlookup (HashMap s3) (QueryIndexChallenge i x)
proof -
  fix i x
  show fmlookup (HashMap s4) (QueryIndexChallenge i x) =
    fmlookup (HashMap s3) (QueryIndexChallenge i x)
proof (rule mmap-preserves-lookup[OF - alpha-out])
  fix m y u v
  assume m-in:  $m \in \text{set } (\text{replicate } (\text{length spec}) \text{ alpha-round})$ 
  and out:  $\text{Some } (y, v) \in \text{set-dist } (\text{execute } m u)$ 
  then have m-eq:  $m = \text{alpha-round}$ 
  by simp
  show fmlookup (HashMap v) (QueryIndexChallenge i x) =
    fmlookup (HashMap u) (QueryIndexChallenge i x)
  using alpha-round-preserves-query-lookup[OF out[unfolded m-eq]] .
qed
qed
have s5-lookup:
  HashMap s5 = HashMap s4
  by (rule read-preserves-hash-map[OF read-dg])
have s7-lookup:
   $\bigwedge i x. \text{fmlookup } (\text{HashMap } s7) (\text{QueryIndexChallenge } i x) =$ 
    fmlookup (HashMap s6) (QueryIndexChallenge i x)
proof -
  fix i x
  show fmlookup (HashMap s7) (QueryIndexChallenge i x) =
    fmlookup (HashMap s6) (QueryIndexChallenge i x)
proof (rule ntimes-preserves-lookup[OF - comp-fri])
  fix y u v
  assume out:
     $\text{Some } (y, v) \in$ 
     $\text{set-dist } (\text{execute } \text{receive-composition-fri-commits } u)$ 
  obtain b r where y-eq:  $y = (b, r)$ 
  by (cases y) simp
  show fmlookup (HashMap v) (QueryIndexChallenge i x) =
    fmlookup (HashMap u) (QueryIndexChallenge i x)
  by (rule receive-composition-fri-commits-preserves-query-lookup
    [OF out[unfolded y-eq]])
qed
qed
have query-lookup:
  HashMap query-state = HashMap s7
  by (rule read-preserves-hash-map[OF read-final])
have header-actual:
  verifier-header-transcript ?s fr (map snd f-fl) f-final as dg

```

```

    (map snd fl) final (PTranscript query-state)
using trace-prefix-res tr-prefix tr-s3 alpha-res tr-s4 tr-s5 s6-eq
    comp-res tr-s7 tr-read-query
unfolding verifier-header-transcript-def verifier-header-messages-def
by simp
have staged-header:
  verifier-header-transcript ?s
    (staged-trace-root data)
    (staged-trace-fri-roots data)
    (staged-trace-final data)
    (staged-alphas data)
    (staged-degree data)
    (staged-composition-fri-roots data)
    (staged-composition-final data)
    (List.concat (staged-query-chunks data))
by (rule checked-staged-transcript-program-outcome-header-transcript
    [OF wf controlled builder])
have header-unique:
  fr = staged-trace-root data ∧
  map snd f-fl = staged-trace-fri-roots data ∧
  f-final = staged-trace-final data ∧
  as = staged-alphas data ∧
  dg = staged-degree data ∧
  map snd fl = staged-composition-fri-roots data ∧
  final = staged-composition-final data ∧
  PTranscript query-state = List.concat (staged-query-chunks data)
using verifier-header-transcript-unique[OF header-actual staged-header]
by simp
have prefix-state-query:
  PState query-state = staged-query-start-hash data
using trace-prefix-res state-s3 alpha-res state-s5 s6-eq comp-res
  state-query header-unique
unfolding staged-query-start-hash-def
  staged-composition-fri-start-hash-def
  staged-trace-fri-start-hash-def
by simp
have chunks-eq: query-chunks = staged-query-chunks data
proof –
from checked-staged-transcript-program-query-chunks-match-verifier-lengths
  [OF builder]
obtain staged-query-idxs where
  staged-match:
    staged-query-chunks-match-verifier-lengths data staged-query-idxs
by blast
have query-idxs-len: length query-idxs = rounds
using query-idxs-eq len-raw by simp
have staged-match-query-idxs:
  staged-query-chunks-match-verifier-lengths data query-idxs
by (rule staged-query-chunks-match-verifier-lengths-transfer

```

```

    [OF staged-match query-idxs-len])
have parser-chunk-len:
   $\bigwedge i. i < \text{rounds} \implies$ 
    length (query-chunks ! i) =
      verifier-query-round-transcript-length (query-idxs ! i)
        (staged-trace-fri-roots data)
        (staged-composition-fri-roots data)
proof -
  fix i
  assume i-bound: i < rounds
  show
    length (query-chunks ! i) =
      verifier-query-round-transcript-length (query-idxs ! i)
        (staged-trace-fri-roots data)
        (staged-composition-fri-roots data)
  using verifier-query-round-chunk-length[OF round-chunks[OF i-bound]]
    header-unique
  by simp
qed
have concat-staged:
  List.concat query-chunks @ PTranscript final-state =
    List.concat (staged-query-chunks data)
  using tr-query header-unique by simp
then show ?thesis
  using query-chunks-eq-staged-if-matching-lengths
    [OF len-chunks concat-staged parser-chunk-len
      staged-match-query-idxs]
  by simp
qed
from hit out-eq obtain raw-idxs-hit query-idxs-hit where
  query-hit-in: query-idxs-hit  $\in Q$ 
  and len-hit: length raw-idxs-hit = rounds
  and query-hit-eq:
    query-idxs-hit =
      map ( $\lambda \text{raw}. \text{index } (\text{to-nat } \text{raw})$ ) raw-idxs-hit
  and lookup-hit:
     $\bigwedge i. i < \text{rounds} \implies$ 
      fmllookup (HashMap final-state)
        (QueryIndexChallenge i
          (state-after-query-chunks
            (staged-query-start-hash data)
            (staged-query-chunks data) i)) =
        Some (raw-idxs-hit ! i)
  and fresh-hit:
     $\bigwedge i. i < \text{rounds} \implies$ 
      fmllookup (HashMap ?s)
        (QueryIndexChallenge i
          (state-after-query-chunks
            (staged-query-start-hash data)

```



```

      (staged-query-chunks data) i)) =
    None
  unfolding staged-security-with-data-state-query-index-list-path-fresh-def
  by auto
  have raw-eq:
     $\bigwedge i. i < \text{rounds} \implies \text{raw-idxs} ! i = \text{raw-idxs-hit} ! i$ 
    using lookups lookup-hit local-counter prefix-state-query chunks-eq
    by simp
  have raw-idxs-eq:  $\text{raw-idxs} = \text{raw-idxs-hit}$ 
    by (rule nth-equalityI)
    (use len-raw len-hit raw-eq in simp-all)
  have raw-in:  $\text{raw-idxs} \in \text{query-index-raw-list-preimage } Q$ 
    unfolding query-index-raw-list-preimage-def
    using len-raw raw-idxs-eq query-hit-in query-hit-eq by blast
  have query-lookup-start:
     $\bigwedge i x. \text{fmlookup } (\text{HashMap } \text{query-state}) (\text{QueryIndexChallenge } i x) =$ 
     $\text{fmlookup } (\text{HashMap } ?s) (\text{QueryIndexChallenge } i x)$ 
    using prefix-lookup s3-lookup s4-lookup s5-lookup s6-eq
    s7-lookup query-lookup by simp
  have local-fresh:
    query-index-list-path-fresh query-state rounds query-chunks raw-idxs
    unfolding query-index-list-path-fresh-def
  proof (intro conjI allI impI)
    show  $\text{length } \text{raw-idxs} = \text{rounds}$ 
    using len-raw .
  next
    show  $\text{length } \text{query-chunks} = \text{rounds}$ 
    using len-chunks .
  next
    fix i
    assume i-bound:  $i < \text{rounds}$ 
    let ?x =
      state-after-query-chunks (staged-query-start-hash data)
      (staged-query-chunks data) i
    have fmlookup (HashMap query-state)
      (QueryIndexChallenge (PQueryCounter query-state + i)
        (state-after-query-chunks (PState query-state) query-chunks i)) =
      fmlookup (HashMap ?s) (QueryIndexChallenge i ?x)
    using local-counter prefix-state-query chunks-eq
    query-lookup-start[of i ?x] by simp
    also have ... = None
    by (rule fresh-hit[OF i-bound])
    finally show
      fmlookup (HashMap query-state)
        (QueryIndexChallenge (PQueryCounter query-state + i)
          (state-after-query-chunks (PState query-state) query-chunks i)) =
      None .
  qed
  have path:

```

```

    query-rounds-raw-list-path-hit query-state raw-idxs rounds out
unfolding query-rounds-raw-list-path-hit-def out-eq
using len-raw len-chunks state-final local-fresh lookups
by auto
have witness:
  (fr, f-fl) = (fr, f-fl) ∧
  Some (f-final, s3) ∈ set-dist (execute read prefix-state) ∧
  Some (as, s4) ∈
    set-dist (execute (mmap (replicate (length spec) alpha-round)) s3) ∧
  Some (dg, s5) ∈ set-dist (execute read s4) ∧
  Some ((), s6) ∈
    set-dist (execute (assert (to-nat dg ≤ maxDegree)) s5) ∧
  Some (fl, s7) ∈
    set-dist (execute
      (ntimes receive-composition-fri-commits
        (ceil-log (to-nat dg + 1))) s6) ∧
  Some (final, query-state) ∈ set-dist (execute read s7) ∧
  raw-idxs ∈ query-index-raw-list-preimage Q ∧
  query-rounds-raw-list-path-hit query-state raw-idxs rounds out
using read-trace-final alpha-out read-dg degree-assert comp-fri
  read-final raw-in path by simp
show ?thesis
unfolding verifier-after-trace-fri-raw-list-path-hit-def
using witness by blast
qed

lemma verifier-after-composition-fri-staged-query-path-fresh-imp-raw-list-path-hit:
assumes wf: staged-budget-wellformed budgets
and controlled: staged-adversary-controlled budgets A
and builder:
  Some (data, attacker-state) ∈
    set-dist
      (execute (checked-staged-transcript-program A)
        adversary-initial-state)
and prefix:
  Some (header, prefix-state) ∈
    set-dist
      (execute
        verifier-composition-fri-prefix
        (verifier-state-from-adversary attacker-state
          (staged-proof-transcript data)))
and suffix:
  out ∈
    set-dist
      (execute (verifier-after-composition-fri header) prefix-state)
and hit:
  staged-security-with-data-state-query-index-list-path-fresh Q
  (case out of
    None ⇒ None

```

```

    | Some (result, final-state)  $\Rightarrow$ 
      Some (((data, attacker-state), result), final-state))
  shows verifier-after-composition-fri-raw-list-path-hit prefix-state Q out
proof (cases out)
case None
then show ?thesis
  using hit
  unfolding staged-security-with-data-state-query-index-list-path-fresh-def
  by simp
next
case (Some pair)
then obtain result :: unit list and final-state where out-eq:
  out = Some (result, final-state)
  by (cases pair) simp
let ?s =
  verifier-state-from-adversary attacker-state
  (staged-proof-transcript data)
obtain fr f-fl f-final as dg fl where header-eq:
  header = (fr, f-fl, f-final, as, dg, fl)
  by (cases header) auto
from suffix[unfolded out-eq header-eq verifier-after-composition-fri-def]
obtain final query-state where
  read-final:
    Some (final, query-state)  $\in$  set-dist (execute read prefix-state)
  and query-out:
    Some (result, final-state)  $\in$ 
      set-dist
      (execute
        (ntimes
          (verifier-query-round-program fr f-fl f-final as fl final)
          rounds)
        query-state)
      by (auto elim!: set-dist-bindE)
from ntimes-verifier-query-rounds-outcome[OF query-out]
obtain raw-idxs query-idxs query-chunks where
  len-raw: length raw-idxs = rounds
  and query-idxs-eq:
    query-idxs = map ( $\lambda$ raw. index (to-nat raw)) raw-idxs
  and len-chunks: length query-chunks = rounds
  and tr-query:
    PTranscript query-state =
      List.concat query-chunks @ PTranscript final-state
  and state-final:
    PState final-state =
      state-after-query-chunks (PState query-state) query-chunks rounds
  and round-chunks:
     $\bigwedge i. i < rounds \Rightarrow$ 
      verifier-query-round-chunk (query-idxs ! i)
      (map snd f-fl) (map snd fl) (query-chunks ! i)

```

```

and lookups:
   $\bigwedge i. i < \text{rounds} \implies$ 
     $\text{fmlookup } (\text{HashMap } \text{final-state})$ 
       $(\text{QueryIndexChallenge } (P\text{QueryCounter } \text{query-state} + i)$ 
         $(\text{state-after-query-chunks}$ 
           $(P\text{State } \text{query-state}) \text{ query-chunks } i)) =$ 
           $\text{Some } (\text{raw-idxs } ! i)$ 
        by blast
from read-outcome[OF read-final] have query-counter:
   $P\text{QueryCounter } \text{query-state} = P\text{QueryCounter } \text{prefix-state}$ 
  by blast
have prefix-counter:
   $P\text{QueryCounter } \text{prefix-state} = P\text{QueryCounter } ?s$ 
  using verifier-composition-fri-prefix-outcome
    [OF prefix[unfolded header-eq]]
  by simp
have local-counter:  $P\text{QueryCounter } \text{query-state} = 0$ 
  using query-counter prefix-counter
  unfolding verifier-state-from-adversary-def by simp
have prefix-lookup:
   $\bigwedge i x. \text{fmlookup } (\text{HashMap } \text{prefix-state})$ 
     $(\text{QueryIndexChallenge } i x) =$ 
     $\text{fmlookup } (\text{HashMap } ?s) (\text{QueryIndexChallenge } i x)$ 
  by (rule verifier-composition-fri-prefix-preserves-query-lookup
    [OF prefix[unfolded header-eq]])
have read-lookup:  $\text{HashMap } \text{query-state} = \text{HashMap } \text{prefix-state}$ 
  by (rule read-preserves-hash-map[OF read-final])
have prefix-res:
   $\text{length } f\text{-fl} = \text{ceil-log } \text{clength} \wedge$ 
   $\text{length } as = \text{length } \text{spec} \wedge$ 
   $\text{length } fl = \text{ceil-log } (\text{to-nat } dg + 1) \wedge$ 
   $P\text{Transcript } ?s =$ 
   $[fr] @ \text{map } \text{snd } f\text{-fl} @ [f\text{-final}] @ as @ [dg] @$ 
   $\text{map } \text{snd } fl @ P\text{Transcript } \text{prefix-state} \wedge$ 
   $P\text{State } \text{prefix-state} =$ 
   $\text{foldl } \text{concat}$ 
   $(\text{concat}$ 
     $(\text{foldl } \text{concat}$ 
       $(\text{concat}$ 
         $(\text{foldl } \text{concat } (\text{concat } (P\text{State } ?s) fr) (\text{map } \text{snd } f\text{-fl}))$ 
         $f\text{-final})$ 
         $as)$ 
         $dg)$ 
         $(\text{map } \text{snd } fl)$ 
      using verifier-composition-fri-prefix-outcome
        [OF prefix[unfolded header-eq]]
      by simp
from read-outcome[OF read-final] obtain rest-query where
   $tr\text{-prefix}: P\text{Transcript } \text{prefix-state} = \text{final} \# \text{rest-query}$ 

```

```

and state-query: PState query-state = concat (PState prefix-state) final
and tr-read-query: PTranscript query-state = rest-query
by blast
have header-actual:
  verifier-header-transcript ?s fr (map snd f-fl) f-final as dg
  (map snd fl) final (PTranscript query-state)
unfolding verifier-header-transcript-def verifier-header-messages-def
using prefix-res tr-prefix tr-read-query by simp
have staged-header:
  verifier-header-transcript ?s
  (staged-trace-root data)
  (staged-trace-fri-roots data)
  (staged-trace-final data)
  (staged-alphas data)
  (staged-degree data)
  (staged-composition-fri-roots data)
  (staged-composition-final data)
  (List.concat (staged-query-chunks data))
by (rule checked-staged-transcript-program-outcome-header-transcript
  [OF wf controlled builder])
have header-unique:
  fr = staged-trace-root data ∧
  map snd f-fl = staged-trace-fri-roots data ∧
  f-final = staged-trace-final data ∧
  as = staged-alphas data ∧
  dg = staged-degree data ∧
  map snd fl = staged-composition-fri-roots data ∧
  final = staged-composition-final data ∧
  PTranscript query-state = List.concat (staged-query-chunks data)
using verifier-header-transcript-unique[OF header-actual staged-header]
by simp
have prefix-state-query:
  PState query-state = staged-query-start-hash data
using state-query prefix-res header-unique
unfolding staged-query-start-hash-def
  staged-composition-fri-start-hash-def
  staged-trace-fri-start-hash-def
by simp
have chunks-eq: query-chunks = staged-query-chunks data
proof –
from checked-staged-transcript-program-query-chunks-match-verifier-lengths
  [OF builder]
obtain staged-query-idxs where
  staged-match:
    staged-query-chunks-match-verifier-lengths data staged-query-idxs
by blast
have query-idxs-len: length query-idxs = rounds
using query-idxs-eq len-raw by simp
have staged-match-query-idxs:

```

```

    staged-query-chunks-match-verifier-lengths data query-idxs
  by (rule staged-query-chunks-match-verifier-lengths-transfer
      [OF staged-match query-idxs-len])
have parser-chunk-len:
   $\bigwedge i. i < \text{rounds} \implies$ 
    length (query-chunks ! i) =
      verifier-query-round-transcript-length (query-idxs ! i)
        (staged-trace-fri-roots data)
        (staged-composition-fri-roots data)
proof -
  fix i
  assume i-bound: i < rounds
  show
    length (query-chunks ! i) =
      verifier-query-round-transcript-length (query-idxs ! i)
        (staged-trace-fri-roots data)
        (staged-composition-fri-roots data)
  using verifier-query-round-chunk-length[OF round-chunks[OF i-bound]]
    header-unique
  by simp
qed
have concat-staged:
  List.concat query-chunks @ PTranscript final-state =
    List.concat (staged-query-chunks data)
  using tr-query header-unique by simp
then show ?thesis
  using query-chunks-eq-staged-if-matching-lengths
    [OF len-chunks concat-staged parser-chunk-len
      staged-match-query-idxs]
  by simp
qed
from hit out-eq obtain raw-idxs-hit query-idxs-hit where
  query-hit-in: query-idxs-hit  $\in$  Q
  and len-hit: length raw-idxs-hit = rounds
  and query-hit-eq:
    query-idxs-hit =
      map ( $\lambda$ raw. index (to-nat raw)) raw-idxs-hit
  and lookup-hit:
     $\bigwedge i. i < \text{rounds} \implies$ 
      fmllookup (HashMap final-state)
        (QueryIndexChallenge i
          (state-after-query-chunks
            (staged-query-start-hash data)
            (staged-query-chunks data) i)) =
        Some (raw-idxs-hit ! i)
  and fresh-hit:
     $\bigwedge i. i < \text{rounds} \implies$ 
      fmllookup (HashMap ?s)
        (QueryIndexChallenge i

```

```

      (state-after-query-chunks
       (staged-query-start-hash data)
       (staged-query-chunks data) i)) =
    None
  unfolding staged-security-with-data-state-query-index-list-path-fresh-def
  by auto
  have raw-eq:
     $\bigwedge i. i < \text{rounds} \implies \text{raw-idxs} ! i = \text{raw-idxs-hit} ! i$ 
  using lookups lookup-hit local-counter prefix-state-query chunks-eq
  by simp
  have raw-idxs-eq:  $\text{raw-idxs} = \text{raw-idxs-hit}$ 
  by (rule nth-equalityI)
    (use len-raw len-hit raw-eq in simp-all)
  have raw-in:  $\text{raw-idxs} \in \text{query-index-raw-list-preimage } Q$ 
  unfolding query-index-raw-list-preimage-def
  using len-raw raw-idxs-eq query-hit-in query-hit-eq by blast
  have query-lookup-start:
     $\bigwedge i x. \text{fmlookup } (\text{HashMap } \text{query-state}) (\text{QueryIndexChallenge } i x) =$ 
     $\text{fmlookup } (\text{HashMap } ?s) (\text{QueryIndexChallenge } i x)$ 
  using prefix-lookup read-lookup by simp
  have local-fresh:
    query-index-list-path-fresh query-state rounds query-chunks raw-idxs
  unfolding query-index-list-path-fresh-def
  proof (intro conjI allI impI)
    show  $\text{length } \text{raw-idxs} = \text{rounds}$ 
    using len-raw .
  next
    show  $\text{length } \text{query-chunks} = \text{rounds}$ 
    using len-chunks .
  next
    fix i
    assume i-bound:  $i < \text{rounds}$ 
    let ?x =
      state-after-query-chunks (staged-query-start-hash data)
      (staged-query-chunks data) i
    have fmlookup (HashMap query-state)
      (QueryIndexChallenge (PQueryCounter query-state + i)
        (state-after-query-chunks (PState query-state) query-chunks i)) =
      fmlookup (HashMap ?s) (QueryIndexChallenge i ?x)
    using local-counter prefix-state-query chunks-eq
      query-lookup-start[of i ?x] by simp
    also have ... = None
    by (rule fresh-hit[OF i-bound])
  finally show
    fmlookup (HashMap query-state)
      (QueryIndexChallenge (PQueryCounter query-state + i)
        (state-after-query-chunks (PState query-state) query-chunks i)) =
    None .
qed

```

```

have path:
  query-rounds-raw-list-path-hit query-state raw-idxs rounds out
  unfolding query-rounds-raw-list-path-hit-def out-eq
  using len-raw len-chunks state-final local-fresh lookups
  by auto
show ?thesis
  unfolding verifier-after-composition-fri-raw-list-path-hit-def
  by (intro exI[of - final] exI[of - query-state]
      exI[of - raw-idxs] conjI)
      (use read-final raw-in path in simp-all)
qed

lemma staged-security-trace-fri-pair-challenge-fresh-imp-query-index-list-hit:
assumes wf: staged-budget-wellformed budgets
and controlled: staged-adversary-controlled budgets A
and support:
  out ∈
    set-dist
    (execute (checked-staged-security-experiment-with-data-state A)
      adversary-initial-state)
and projection: fst ‘ P ⊆ Q
and hit: staged-security-trace-fri-pair-challenge-fresh P B out
shows staged-security-with-data-query-index-list-set-hit Q out
proof (cases out)
case None
then show ?thesis
  using hit
  unfolding staged-security-trace-fri-pair-challenge-fresh-def
    staged-security-with-data-state-verifier-event-def
    staged-security-with-data-query-index-list-set-hit-def
  by simp
next
case (Some packed)
then obtain data attacker-state result final-state where out-eq:
  out = Some (((data, attacker-state), result), final-state)
  by (cases packed) (auto split: prod.splits)
let ?s =
  verifier-state-from-adversary attacker-state
    (staged-proof-transcript data)
from hit out-eq have pair-hit:
  trace-fri-query-challenge-pair-set-hit ?s P
    (Some (result, final-state))
  unfolding staged-security-trace-fri-pair-challenge-fresh-def
    staged-security-with-data-state-verifier-event-def
  by simp
have trace-hit:
  trace-fri-query-index-list-set-hit ?s Q
    (Some (result, final-state))
  by (rule trace-fri-query-challenge-pair-set-hit-imp-query-index-list-set-hit

```



```

    [OF pair-hit projection])
from trace-hit obtain trace-roots trace-bs trace-final dg
    composition-roots composition-bs composition-final query-idxs
    trace-round-layers composition-round-layers where
    fri:
    accepted-fri-opening-transcript ?s (Some (result, final-state))
    trace-roots trace-bs trace-final dg composition-roots
    composition-bs composition-final query-idxs trace-round-layers
    composition-round-layers
    and query-in: query-idxs ∈ Q
    unfolding trace-fri-query-index-list-set-hit-def by blast
from accepted-fri-opening-transcript-imp-accepted-transcript-shape[OF fri]
obtain as where shape:
    accepted-transcript-shape ?s (Some (result, final-state)) as query-idxs
    by blast
from checked-staged-security-with-data-state-accepted-shape-query-keys
    [OF wf controlled support[unfolded out-eq] shape]
obtain raw-idxs where
    len-raw: length raw-idxs = rounds
    and query-idxs-eq:
    query-idxs = map (λraw. index (to-nat raw)) raw-idxs
    and lookup:
    ∧ i. i < rounds ⇒
    fmlookup (HashMap final-state)
    (QueryIndexChallenge i
    (state-after-query-chunks
    (staged-query-start-hash data)
    (staged-query-chunks data) i)) =
    Some (raw-idxs ! i)
    by blast
show ?thesis
    unfolding out-eq staged-security-with-data-query-index-list-set-hit-def
    using len-raw query-in query-idxs-eq lookup by auto
qed

lemma staged-security-composition-fri-pair-challenge-fresh-imp-query-index-list-hit:
assumes wf: staged-budget-wellformed budgets
and controlled: staged-adversary-controlled budgets A
and support:
    out ∈
    set-dist
    (execute (checked-staged-security-experiment-with-data-state A)
    adversary-initial-state)
and projection: ∧ dg. fst ‘ P dg ⊆ Q
and hit: staged-security-composition-fri-pair-challenge-fresh P B out
shows staged-security-with-data-query-index-list-set-hit Q out
proof (cases out)
case None
then show ?thesis

```

```

using hit
unfolding staged-security-composition-fri-pair-challenge-fresh-def
  staged-security-with-data-state-verifier-event-def
  staged-security-with-data-query-index-list-set-hit-def
by simp
next
case (Some packed)
then obtain data attacker-state result final-state where out-eq:
  out = Some (((data, attacker-state), result), final-state)
  by (cases packed) (auto split: prod.splits)
let ?s =
  verifier-state-from-adversary attacker-state
    (staged-proof-transcript data)
from hit out-eq have pair-hit:
  composition-fri-query-challenge-pair-set-hit ?s P
  (Some (result, final-state))
unfolding staged-security-composition-fri-pair-challenge-fresh-def
  staged-security-with-data-state-verifier-event-def
by simp
have composition-hit:
  composition-fri-query-index-list-set-hit ?s Q
  (Some (result, final-state))
by (rule
  composition-fri-query-challenge-pair-set-hit-imp-query-index-list-set-hit
  [OF pair-hit projection])
from composition-hit obtain trace-roots trace-bs trace-final dg
  composition-roots composition-bs composition-final query-idxs
  trace-round-layers composition-round-layers where
  fri:
  accepted-fri-opening-transcript ?s (Some (result, final-state))
  trace-roots trace-bs trace-final dg composition-roots
  composition-bs composition-final query-idxs trace-round-layers
  composition-round-layers
and query-in: query-idxs ∈ Q
unfolding composition-fri-query-index-list-set-hit-def by blast
from accepted-fri-opening-transcript-imp-accepted-transcript-shape[OF fri]
obtain as where shape:
  accepted-transcript-shape ?s (Some (result, final-state)) as query-idxs
by blast
from checked-staged-security-with-data-state-accepted-shape-query-keys
  [OF wf controlled support[unfolded out-eq] shape]
obtain raw-idxs where
  len-raw: length raw-idxs = rounds
and query-idxs-eq:
  query-idxs = map (λraw. index (to-nat raw)) raw-idxs
and lookup:
  ∧ i. i < rounds ⇒
  fmlookup (HashMap final-state)
    (QueryIndexChallenge i

```

```

      (state-after-query-chunks
       (staged-query-start-hash data)
       (staged-query-chunks data) i)) =
    Some (raw-idxs ! i)
  by blast
show ?thesis
  unfolding out-eq staged-security-with-data-query-index-list-set-hit-def
  using len-raw query-in query-idxs-eq lookup by auto
qed

lemma checked-staged-security-trace-fri-pair-challenge-fresh-bound-from-query-path-fresh:
  assumes wf: staged-budget-wellformed budgets
  and controlled: staged-adversary-controlled budgets A
  and projection: fst ' P ⊆ Q
  and path-branch-bound:
    wp-event (checked-staged-security-experiment-with-data-state A)
      (staged-security-trace-fri-pair-challenge-query-path-fresh P B Q)
    adversary-initial-state ≤ C
  shows
    wp-event (checked-staged-security-experiment-with-data-state A)
      (staged-security-trace-fri-pair-challenge-fresh P B)
    adversary-initial-state ≤
      C +
      hash-relation-budget-value
        (query-index-raw-list-relation-fiber-bound Q)
        (staged-attacker-query-budget budgets +
         staged-challenge-query-budget)
proof -
  have event-mono:
    wp-event (checked-staged-security-experiment-with-data-state A)
      (staged-security-trace-fri-pair-challenge-fresh P B)
    adversary-initial-state ≤
    wp-event (checked-staged-security-experiment-with-data-state A)
      (λout.
        staged-security-trace-fri-pair-challenge-query-path-fresh P B Q out ∨
        staged-security-with-data-state-query-index-list-prequery-hit A Q out)
    adversary-initial-state
  proof (rule wp-event-mono-on-support)
    fix out
    assume support:
      out ∈
      set-dist
      (execute (checked-staged-security-experiment-with-data-state A)
        adversary-initial-state)
    and hit: staged-security-trace-fri-pair-challenge-fresh P B out
    have query-hit:
      staged-security-with-data-state-query-index-list-set-hit Q out
    by (rule
      staged-security-trace-fri-pair-challenge-fresh-imp-query-index-list-hit

```

```

    [OF wf controlled support projection hit])
show
  staged-security-trace-fri-pair-challenge-query-path-fresh P B Q out ∨
  staged-security-with-data-state-query-index-list-prequery-hit A Q out
proof (cases out)
case None
then show ?thesis
  using query-hit
  unfolding staged-security-with-data-query-index-list-set-hit-def
  by simp
next
case (Some packed)
then obtain data attacker-state result final-state where out-eq:
  out = Some (((data, attacker-state), result), final-state)
  by (cases packed) (auto split: prod.splits)
have split:
  staged-security-with-data-state-query-index-list-path-fresh Q
    (Some (((data, attacker-state), result), final-state)) ∨
  hash-relation-hit
    (checked-staged-query-index-list-relation A Q)
  adversary-initial-state attacker-state
apply (rule
  checked-staged-security-with-data-state-query-index-list-fresh-or-prequeried)
using support query-hit unfolding out-eq by simp-all
then show ?thesis
proof
  assume fresh:
    staged-security-with-data-state-query-index-list-path-fresh Q
      (Some (((data, attacker-state), result), final-state))
  have pair-fresh:
    staged-security-trace-fri-pair-challenge-fresh P B
      (Some (((data, attacker-state), result), final-state))
  using hit out-eq by simp
  then have
    staged-security-trace-fri-pair-challenge-query-path-fresh P B Q
      out
  unfolding out-eq
    staged-security-trace-fri-pair-challenge-query-path-fresh-def
  using fresh pair-fresh by simp
  then show ?thesis ..
next
  assume prequery:
    hash-relation-hit
      (checked-staged-query-index-list-relation A Q)
    adversary-initial-state attacker-state
  then have
    staged-security-with-data-state-query-index-list-prequery-hit A Q
      out
  unfolding out-eq

```

```

    staged-security-with-data-state-query-index-list-prequery-hit-def
  by simp
  then show ?thesis ..
qed
qed
qed
have union-bound:
  wp-event (checked-staged-security-experiment-with-data-state A)
    (λout.
      staged-security-trace-fri-pair-challenge-query-path-fresh P B Q out ∨
      staged-security-with-data-state-query-index-list-prequery-hit A Q out)
  adversary-initial-state ≤
  wp-event (checked-staged-security-experiment-with-data-state A)
    (staged-security-trace-fri-pair-challenge-query-path-fresh P B Q)
  adversary-initial-state +
  wp-event (checked-staged-security-experiment-with-data-state A)
    (staged-security-with-data-state-query-index-list-prequery-hit A Q)
  adversary-initial-state
  by (rule wp-event-union-bound)
have prequery-bound:
  wp-event (checked-staged-security-experiment-with-data-state A)
    (staged-security-with-data-state-query-index-list-prequery-hit A Q)
  adversary-initial-state ≤
  hash-relation-budget-value
    (query-index-raw-list-relation-fiber-bound Q)
    (staged-attacker-query-budget budgets +
      staged-challenge-query-budget)
  by (rule
    checked-staged-security-with-data-state-query-index-list-prequery-hit-bound
    [OF wf controlled])
have wp-event (checked-staged-security-experiment-with-data-state A)
  (staged-security-trace-fri-pair-challenge-fresh P B)
  adversary-initial-state ≤
  wp-event (checked-staged-security-experiment-with-data-state A)
    (staged-security-trace-fri-pair-challenge-query-path-fresh P B Q)
  adversary-initial-state +
  wp-event (checked-staged-security-experiment-with-data-state A)
    (staged-security-with-data-state-query-index-list-prequery-hit A Q)
  adversary-initial-state
  by (rule order-trans[OF event-mono union-bound])
also have ... ≤
  C +
  hash-relation-budget-value
    (query-index-raw-list-relation-fiber-bound Q)
    (staged-attacker-query-budget budgets +
      staged-challenge-query-budget)
  by (rule add-mono[OF path-branch-bound prequery-bound])
finally show ?thesis .
qed

```

lemma *checked-staged-security-composition-fri-pair-challenge-fresh-bound-from-query-path-fresh:*

assumes *wf: staged-budget-wellformed budgets*

and *controlled: staged-adversary-controlled budgets A*

and *projection: $\bigwedge dg. \text{fst } dg \subseteq Q$*

and *path-branch-bound:*

wp-event (checked-staged-security-experiment-with-data-state A)

(staged-security-composition-fri-pair-challenge-query-path-fresh P B Q)

adversary-initial-state $\leq C$

shows

wp-event (checked-staged-security-experiment-with-data-state A)

(staged-security-composition-fri-pair-challenge-fresh P B)

adversary-initial-state $\leq$

C +

hash-relation-budget-value

(query-index-row-list-relation-fiber-bound Q)

(staged-attacker-query-budget budgets +

staged-challenge-query-budget)

proof –

have *event-mono:*

wp-event (checked-staged-security-experiment-with-data-state A)

(staged-security-composition-fri-pair-challenge-fresh P B)

adversary-initial-state $\leq$

wp-event (checked-staged-security-experiment-with-data-state A)

($\lambda out.$

staged-security-composition-fri-pair-challenge-query-path-fresh P B Q out $\vee$

staged-security-with-data-state-query-index-list-prequery-hit A Q out)

adversary-initial-state

proof (rule *wp-event-mono-on-support*)

fix *out*

assume *support:*

out $\in$

set-dist

(execute (checked-staged-security-experiment-with-data-state A)

adversary-initial-state)

and *hit: staged-security-composition-fri-pair-challenge-fresh P B out*

have *query-hit:*

staged-security-with-data-query-index-list-set-hit Q out

by (rule

staged-security-composition-fri-pair-challenge-fresh-imp-query-index-list-hit

[OF wf controlled support projection hit])

show

staged-security-composition-fri-pair-challenge-query-path-fresh P B Q out $\vee$

staged-security-with-data-state-query-index-list-prequery-hit A Q out

proof (cases *out*)

case *None*

then show *?thesis*

using *query-hit*

unfolding *staged-security-with-data-query-index-list-set-hit-def*

```

    by simp
next
case (Some packed)
then obtain data attacker-state result final-state where out-eq:
  out = Some (((data, attacker-state), result), final-state)
  by (cases packed) (auto split: prod.splits)
have split:
  staged-security-with-data-state-query-index-list-path-fresh Q
    (Some (((data, attacker-state), result), final-state)) ∨
  hash-relation-hit
    (checked-staged-query-index-list-relation A Q)
  adversary-initial-state attacker-state
apply (rule
  checked-staged-security-with-data-state-query-index-list-fresh-or-prequeried)
using support query-hit unfolding out-eq by simp-all
then show ?thesis
proof
  assume fresh:
    staged-security-with-data-state-query-index-list-path-fresh Q
    (Some (((data, attacker-state), result), final-state))
  have pair-fresh:
    staged-security-composition-fri-pair-challenge-fresh P B
    (Some (((data, attacker-state), result), final-state))
  using hit out-eq by simp
  then have
    staged-security-composition-fri-pair-challenge-query-path-fresh
      P B Q out
  unfolding out-eq
    staged-security-composition-fri-pair-challenge-query-path-fresh-def
  using fresh pair-fresh by simp
  then show ?thesis ..
next
assume prequery:
  hash-relation-hit
    (checked-staged-query-index-list-relation A Q)
  adversary-initial-state attacker-state
then have
  staged-security-with-data-state-query-index-list-prequery-hit A Q
  out
  unfolding out-eq
    staged-security-with-data-state-query-index-list-prequery-hit-def
  by simp
then show ?thesis ..
qed
qed
qed
have union-bound:
  wp-event (checked-staged-security-experiment-with-data-state A)
  (λout.

```

```

    staged-security-composition-fri-pair-challenge-query-path-fresh P B Q out ∨
    staged-security-with-data-state-query-index-list-prequery-hit A Q out)
  adversary-initial-state ≤
  wp-event (checked-staged-security-experiment-with-data-state A)
    (staged-security-composition-fri-pair-challenge-query-path-fresh P B Q)
    adversary-initial-state +
  wp-event (checked-staged-security-experiment-with-data-state A)
    (staged-security-with-data-state-query-index-list-prequery-hit A Q)
    adversary-initial-state
  by (rule wp-event-union-bound)
have prequery-bound:
  wp-event (checked-staged-security-experiment-with-data-state A)
    (staged-security-with-data-state-query-index-list-prequery-hit A Q)
    adversary-initial-state ≤
    hash-relation-budget-value
      (query-index-raw-list-relation-fiber-bound Q)
      (staged-attacker-query-budget budgets +
        staged-challenge-query-budget)
  by (rule
    checked-staged-security-with-data-state-query-index-list-prequery-hit-bound
    [OF wf controlled])
have wp-event (checked-staged-security-experiment-with-data-state A)
  (staged-security-composition-fri-pair-challenge-fresh P B)
  adversary-initial-state ≤
  wp-event (checked-staged-security-experiment-with-data-state A)
    (staged-security-composition-fri-pair-challenge-query-path-fresh P B Q)
    adversary-initial-state +
  wp-event (checked-staged-security-experiment-with-data-state A)
    (staged-security-with-data-state-query-index-list-prequery-hit A Q)
    adversary-initial-state
  by (rule order-trans[OF event-mono union-bound])
also have ... ≤
  C +
  hash-relation-budget-value
    (query-index-raw-list-relation-fiber-bound Q)
    (staged-attacker-query-budget budgets +
      staged-challenge-query-budget)
  by (rule add-mono[OF path-branch-bound prequery-bound])
finally show ?thesis by simp
qed

lemma checked-staged-security-composition-fri-pair-challenge-query-path-fresh-fixed-bound:
  assumes wf: staged-budget-wellformed budgets
  and controlled: staged-adversary-controlled budgets A
  and challenges-space:
    challenges ∈ fri-challenge-space (ceil-log (to-nat dg + 1))
  and projection:
    fst ‘ (P dg ∩ (UNIV × {challenges})) ⊆ Q
  and subset: Q ⊆ fri-query-index-list-space

```


shows

wp-event (checked-staged-security-experiment-with-data-state A)
(staged-security-composition-fri-pair-challenge-query-path-fresh P
($\lambda dg'. \text{ if } dg' = dg \text{ then } \{\text{challenges}\} \text{ else } \{\}\} Q)$
adversary-initial-state $\leq$
*(1 / nnreal (CARD('f) $\wedge$ ceil-log (to-nat dg + 1))) **
*(nnreal (card Q) **
(1 / nnreal (card query-sample-space)) $\wedge$ rounds)

proof (*rule checked-staged-security-with-data-state-bound-from-data-cont*)
let *?B = $\lambda dg'. \text{ if } dg' = dg \text{ then } \{\text{challenges}\} \text{ else } \{\}$*
let *?D =*
*nnreal (card Q) **
(1 / nnreal (card query-sample-space)) $\wedge$ rounds
show $\neg$ *staged-security-composition-fri-pair-challenge-query-path-fresh*
P ?B Q None
unfolding
staged-security-composition-fri-pair-challenge-query-path-fresh-def
staged-security-composition-fri-pair-challenge-fresh-def
staged-security-with-data-state-verifier-event-def
by *simp*
next
fix *data attacker-state*
assume *builder:*
Some (data, attacker-state) $\in$
set-dist
(execute (checked-staged-transcript-program A)
adversary-initial-state)
let *?s =*
verifier-state-from-adversary attacker-state
(staged-proof-transcript data)
let *?B = $\lambda dg'. \text{ if } dg' = dg \text{ then } \{\text{challenges}\} \text{ else } \{\}$*
let *?Event =*
 $\lambda out.$
staged-security-composition-fri-pair-challenge-query-path-fresh P ?B Q
(case out of
None $\Rightarrow$ None
| Some (result, final-state) $\Rightarrow$
Some (((data, attacker-state), result), final-state)))
let *?Head =*
 $\lambda out.$ case out of
None $\Rightarrow$ False
| Some ((fr, f-fl, f-final, as, dg', fl), -) $\Rightarrow$
dg' = dg $\wedge$ map fst fl = challenges $\wedge$
composition-fri-challenge-path-fresh ?s fr (map snd f-fl)
f-final as dg' (map snd fl) (length (map snd fl)))
show *wp-event verify-monad ?Event ?s $\leq$*
*(1 / nnreal (CARD('f) $\wedge$ ceil-log (to-nat dg + 1))) **
*(nnreal (card Q) **
(1 / nnreal (card query-sample-space)) $\wedge$ rounds)

```

unfolding verify-monad-composition-fri-decomposition
proof (rule order-trans)
show wp-event
  (verifier-composition-fri-prefix  $\gg=$ 
    verifier-after-composition-fri)
  ?Event ?s
 $\leq$  wp-event verifier-composition-fri-prefix ?Head ?s *
  (nnreal (card Q) *
    (1 / nnreal (card query-sample-space))  $\wedge$  rounds)
proof (rule wp-event-bind-bound-by-head-and-cont)
show  $\neg$  ?Event None
  unfolding
    staged-security-composition-fri-pair-challenge-query-path-fresh-def
    staged-security-composition-fri-pair-challenge-fresh-def
    staged-security-with-data-state-verifier-event-def
  by simp
next
fix header prefix-state
assume prefix:
  Some (header, prefix-state)  $\in$ 
    set-dist (execute verifier-composition-fri-prefix ?s)
  and not-head:  $\neg$  ?Head (Some (header, prefix-state))
obtain fr f-fl f-final as dg' fl where header-eq:
  header = (fr, f-fl, f-final, as, dg', fl)
by (cases header) auto
have not-head':
  dg'  $\neq$  dg  $\vee$  map fst fl  $\neq$  challenges  $\vee$ 
   $\neg$  composition-fri-challenge-path-fresh ?s fr (map snd f-fl)
  f-final as dg' (map snd fl) (length (map snd fl))
using not-head unfolding header-eq by simp
show wp-event (verifier-after-composition-fri header) ?Event
  prefix-state = 0
proof (rule antisym)
show wp-event (verifier-after-composition-fri header) ?Event
  prefix-state  $\leq$  0
proof (rule order-trans)
show wp-event (verifier-after-composition-fri header) ?Event
  prefix-state  $\leq$ 
  wp-event (verifier-after-composition-fri header)
  ( $\lambda$ -. False) prefix-state
proof (rule wp-event-mono-on-support)
fix out
assume suffix:
  out  $\in$ 
    set-dist
    (execute (verifier-after-composition-fri header)
      prefix-state)
  and hit: ?Event out
show False

```

```

proof (cases out)
  case None
  then show ?thesis
    using hit
    unfolding
      staged-security-composition-fri-pair-challenge-query-path-fresh-def
      staged-security-composition-fri-pair-challenge-fresh-def
      staged-security-with-data-state-verifier-event-def
    by simp
next
  case (Some pair)
  then obtain result :: unit list and final-state where out-eq:
    out = Some (result, final-state)
  by (cases pair) simp
from hit out-eq have fresh:
  composition-fri-challenge-list-fresh-hit ?s ?B
  (Some (result, final-state))
and pair-hit:
  composition-fri-query-challenge-pair-set-hit ?s P
  (Some (result, final-state))
unfolding
  staged-security-composition-fri-pair-challenge-query-path-fresh-def
  staged-security-composition-fri-pair-challenge-fresh-def
  staged-security-with-data-state-verifier-event-def
by simp-all
have prefix':
  Some ((fr, f-fl, f-final, as, dg', fl), prefix-state) ∈
  set-dist (execute verifier-composition-fri-prefix ?s)
using prefix header-eq by simp
have suffix':
  Some (result, final-state) ∈
  set-dist
  (execute
    (verifier-after-composition-fri
      (fr, f-fl, f-final, as, dg', fl))
    prefix-state)
using suffix header-eq out-eq by simp
from pair-hit obtain trace-roots trace-bs trace-final fri-dg
composition-roots composition-bs composition-final query-idxs
trace-round-layers composition-round-layers where
openings:
  accepted-fri-opening-transcript ?s
  (Some (result, final-state)) trace-roots trace-bs
trace-final fri-dg composition-roots composition-bs
composition-final query-idxs trace-round-layers
composition-round-layers
unfolding composition-fri-query-challenge-pair-set-hit-def
by blast
have pair-challenges:

```

```

    accepted-fri-challenges ?s (Some (result, final-state))
    trace-bs fri-dg composition-bs
  by (rule accepted-fri-opening-transcript-challenges
    [OF openings])
from fresh obtain trace-bs' dg'' comp-bs' where
  fresh-challenges:
    accepted-fri-challenges ?s (Some (result, final-state))
    trace-bs' dg'' comp-bs'
  and comp-bs'-in:
    comp-bs' ∈
    (if dg'' = dg then {challenges} else {})
  unfolding composition-fri-challenge-list-fresh-hit-def
  by blast
have challenge-eqs:
  fri-dg = dg'' ∧ composition-bs = comp-bs'
  using accepted-fri-challenges-unique
  [OF pair-challenges fresh-challenges]
  by simp
have prefix-eqs:
  fri-dg = dg' ∧ composition-bs = map fst fl
  by (rule
    verifier-after-composition-fri-header-agrees-with-accepted-challenges
    [OF prefix' suffix' pair-challenges])
have concrete-fresh:
  composition-fri-challenge-path-fresh ?s fr (map snd f-fl)
  f-final as dg' (map snd fl) (length (map snd fl))
  by (rule
    verifier-after-composition-fri-header-roots-agree-with-fresh-hit
    [OF prefix' suffix' fresh])
have dg''-eq: dg'' = dg
  using comp-bs'-in by (cases dg'' = dg) auto
have dg'-eq: dg' = dg
  using challenge-eqs prefix-eqs dg''-eq by simp
have fl-challenges: map fst fl = challenges
  using comp-bs'-in prefix-eqs challenge-eqs dg''-eq by simp
have head-holds:
  dg' = dg ∧ map fst fl = challenges ∧
  composition-fri-challenge-path-fresh ?s fr (map snd f-fl)
  f-final as dg' (map snd fl) (length (map snd fl))
  using dg'-eq fl-challenges concrete-fresh by simp
show ?thesis
  using not-head' head-holds by blast
qed
qed
next
show wp-event (verifier-after-composition-fri header)
  (λ-. False) prefix-state ≤ 0
  unfolding wp-event-def wp-def dist-expect-def by simp
qed

```

```

next
  show  $0 \leq$ 
    wp-event (verifier-after-composition-fri header) ?Event
    prefix-state
  by simp
qed
next
fix header prefix-state
assume prefix:
  Some (header, prefix-state)  $\in$ 
    set-dist (execute verifier-composition-fri-prefix ?s)
  and head: ?Head (Some (header, prefix-state))
have event-mono:
  wp-event (verifier-after-composition-fri header) ?Event
  prefix-state  $\leq$ 
  wp-event (verifier-after-composition-fri header)
    (verifier-after-composition-fri-raw-list-path-hit prefix-state Q)
  prefix-state
proof (rule wp-event-mono-on-support)
  fix out
  assume suffix:
    out  $\in$ 
    set-dist
      (execute (verifier-after-composition-fri header) prefix-state)
  and hit: ?Event out
  from hit have path-hit:
    staged-security-with-data-state-query-index-list-path-fresh Q
    (case out of
      None  $\Rightarrow$  None
    | Some (result, final-state)  $\Rightarrow$ 
      Some (((data, attacker-state), result), final-state))
  unfolding
    staged-security-composition-fri-pair-challenge-query-path-fresh-def
  by simp
show
  verifier-after-composition-fri-raw-list-path-hit prefix-state Q
  out
  by (rule
    verifier-after-composition-fri-staged-query-path-fresh-imp-raw-list-path-hit
    [OF wf controlled builder prefix suffix path-hit])
qed
also have ...  $\leq$ 
  nnreal (card Q) *
    (1 / nnreal (card query-sample-space))  $\wedge$  rounds
  by (rule wp-verifier-after-composition-fri-raw-list-path-product-bound
    [OF subset])
finally show
  wp-event (verifier-after-composition-fri header) ?Event
  prefix-state

```

```

    ≤ nnreal (card Q) *
      (1 / nnreal (card query-sample-space)) ^ rounds .
qed
next
show wp-event verifier-composition-fri-prefix ?Head ?s *
  (nnreal (card Q) *
    (1 / nnreal (card query-sample-space)) ^ rounds)
≤
  (1 / nnreal (CARD('f) ^ ceil-log (to-nat dg + 1))) *
  (nnreal (card Q) *
    (1 / nnreal (card query-sample-space)) ^ rounds)
proof (rule mult-right-mono)
show wp-event verifier-composition-fri-prefix ?Head ?s ≤
  1 / nnreal (CARD('f) ^ ceil-log (to-nat dg + 1))
proof -
  let ?B = λdg'. if dg' = dg then {challenges} else {}
  let ?HeadSet =
    λout. case out of
      None ⇒ False
    | Some ((fr, f-fl, f-final, as, dg', fl), -) ⇒
      map fst fl ∈ ?B dg' ∧
      composition-fri-challenge-path-fresh ?s fr (map snd f-fl)
      f-final as dg' (map snd fl) (length (map snd fl))
  have mono:
    wp-event verifier-composition-fri-prefix ?Head ?s ≤
    wp-event verifier-composition-fri-prefix ?HeadSet ?s
  by (rule wp-event-mono) (auto split: option.splits prod.splits)
  have subset:
    ∧dg'. ?B dg' ⊆
      fri-challenge-space (ceil-log (to-nat dg' + 1))
  using challenges-space by auto
  have bound:
    ∧dg'. nnreal (card (?B dg')) /
      nnreal (CARD('f) ^ ceil-log (to-nat dg' + 1)) ≤
      1 / nnreal (CARD('f) ^ ceil-log (to-nat dg + 1))
  using challenges-space by auto
  have set-bound:
    wp-event verifier-composition-fri-prefix ?HeadSet ?s ≤
      1 / nnreal (CARD('f) ^ ceil-log (to-nat dg + 1))
  by (rule
    wp-verifier-composition-fri-prefix-challenge-list-actual-fresh-bound
    [OF subset bound])
  show ?thesis
  by (rule order-trans[OF mono set-bound])
qed
show 0 ≤ nnreal (card Q) *
  (1 / nnreal (card query-sample-space)) ^ rounds
  by simp
qed

```

qed
qed

lemma *checked-staged-security-composition-fri-pair-challenge-query-path-fresh-fiber-sum:*

assumes *wf: staged-budget-wellformed budgets*

and *controlled: staged-adversary-controlled budgets A*

and *challenge-subset:*

$\bigwedge dg. B \ dg \subseteq \text{fri-challenge-space } (\text{ceil-log } (\text{to-nat } dg + 1))$

and *projection:*

$\bigwedge dg \text{ challenges.}$

$\text{challenges} \in B \ dg \implies$

$\text{fst } ' (P \ dg \cap (UNIV \times \{\text{challenges}\})) \subseteq$

$Q \ dg \ \text{challenges}$

and *subset:*

$\bigwedge dg \ \text{challenges. } \text{challenges} \in B \ dg \implies$

$Q \ dg \ \text{challenges} \subseteq \text{fri-query-index-list-space}$

shows

wp-event (checked-staged-security-experiment-with-data-state A)

(staged-security-composition-fri-pair-challenge-query-path-fresh P B

$(\bigcup dg \in UNIV. \bigcup \text{challenges} \in B \ dg. Q \ dg \ \text{challenges}))$

adversary-initial-state $\leq$

$(\sum dg \in UNIV.$

$\sum \text{challenges} \in B \ dg.$

$(1 / \text{nnreal } (\text{CARD}('f) \wedge \text{ceil-log } (\text{to-nat } dg + 1))) *$

$(\text{nnreal } (\text{card } (Q \ dg \ \text{challenges}))) *$

$(1 / \text{nnreal } (\text{card query-sample-space})) \wedge \text{rounds}))$

proof –

let *?Q* = $(\bigcup dg \in UNIV. \bigcup \text{challenges} \in B \ dg. Q \ dg \ \text{challenges})$

let *?I* = *SIGMA dg:UNIV. B dg*

let *?E* =

$\lambda p.$

staged-security-composition-fri-pair-challenge-query-path-fresh P

$(\lambda dg'. \text{if } dg' = \text{fst } p \text{ then } \{\text{snd } p\} \text{ else } \{\})$

$(Q \ (\text{fst } p) \ (\text{snd } p))$

let *?C* =

$\lambda p.$

$(1 / \text{nnreal } (\text{CARD}('f) \wedge \text{ceil-log } (\text{to-nat } (\text{fst } p) + 1))) *$

$(\text{nnreal } (\text{card } (Q \ (\text{fst } p) \ (\text{snd } p)))) *$

$(1 / \text{nnreal } (\text{card query-sample-space})) \wedge \text{rounds})$

have *split:*

wp-event (checked-staged-security-experiment-with-data-state A)

(staged-security-composition-fri-pair-challenge-query-path-fresh P B ?Q)

adversary-initial-state $\leq$

wp-event (checked-staged-security-experiment-with-data-state A)

$(\lambda out. \exists p \in ?I. ?E \ p \ out)$

adversary-initial-state

proof (rule *wp-event-mono-on-support*)

fix *out*

```

assume support:
  out ∈
    set-dist
      (execute (checked-staged-security-experiment-with-data-state A)
        adversary-initial-state)
  and hit:
    staged-security-composition-fri-pair-challenge-query-path-fresh P B ?Q
    out
show  $\exists p \in ?I. ?E\ p\ out$ 
proof (cases out)
  case None
  then show ?thesis
    using hit
    unfolding
      staged-security-composition-fri-pair-challenge-query-path-fresh-def
      staged-security-composition-fri-pair-challenge-fresh-def
      staged-security-with-data-state-verifier-event-def
    by simp
next
  case (Some packed)
  then obtain data attacker-state result final-state where out-eq:
    out = Some (((data, attacker-state), result), final-state)
    by (cases packed) (auto split: prod.splits)
  let ?s =
    verifier-state-from-adversary attacker-state
    (staged-proof-transcript data)
  from hit out-eq have pair-hit:
    composition-fri-query-challenge-pair-set-hit ?s P
    (Some (result, final-state))
  and fresh:
    composition-fri-challenge-list-fresh-hit ?s B
    (Some (result, final-state))
  and path:
    staged-security-with-data-state-query-index-list-path-fresh ?Q out
  unfolding
    staged-security-composition-fri-pair-challenge-query-path-fresh-def
    staged-security-composition-fri-pair-challenge-fresh-def
    staged-security-with-data-state-verifier-event-def
  by simp-all
  from fresh obtain trace-bs dg comp-bs fr trace-roots trace-final as
    composition-roots final rest where
    accepted:
      accepted-fri-challenges ?s (Some (result, final-state))
      trace-bs dg comp-bs
  and comp-bs-in: comp-bs ∈ B dg
  and header:
    verifier-header-transcript ?s fr trace-roots trace-final as dg
    composition-roots final rest
  and fresh-path:

```


composition-fri-challenge-path-fresh ?s fr trace-roots
trace-final as dg composition-roots (length composition-roots)
unfolding *composition-fri-challenge-list-fresh-hit-def* **by** *blast*
from *pair-hit* **obtain** *trace-roots' trace-bs' trace-final' fri-dg*
composition-roots' composition-bs' composition-final' query-idxs
trace-round-layers composition-round-layers **where**
openings:
accepted-fri-opening-transcript ?s
(Some (result, final-state)) trace-roots' trace-bs'
trace-final' fri-dg composition-roots' composition-bs'
composition-final' query-idxs trace-round-layers
composition-round-layers
and *pair: (query-idxs, composition-bs') ∈ P fri-dg*
unfolding *composition-fri-query-challenge-pair-set-hit-def* **by** *blast*
have *pair-challenges:*
accepted-fri-challenges ?s (Some (result, final-state))
trace-bs' fri-dg composition-bs'
by *(rule accepted-fri-opening-transcript-challenges[OF openings])*
have *eqs: fri-dg = dg ∧ composition-bs' = comp-bs*
using *accepted-fri-challenges-unique[OF pair-challenges accepted]*
by *simp*
have *query-in: query-idxs ∈ Q dg comp-bs*
proof –
have *query-idxs ∈*
fst ‘ (P dg ∩ (UNIV × {comp-bs}))
using *pair eqs* **by** *force*
then show *?thesis*
using *projection[OF comp-bs-in]* **by** *blast*
qed
from *path out-eq* **obtain** *raw-idxs query-idxs-path* **where**
len-raw: length raw-idxs = rounds
and *query-path-in: query-idxs-path ∈ ?Q*
and *query-path-eq:*
query-idxs-path =
map (λraw. index (to-nat raw)) raw-idxs
and *lookup:*
 $\bigwedge i. i < \text{rounds} \implies$
fmlookup (HashMap final-state)
(QueryIndexChallenge i
(state-after-query-chunks
(staged-query-start-hash data)
(staged-query-chunks data) i)) =
Some (raw-idxs ! i)
and *fresh-lookup:*
 $\bigwedge i. i < \text{rounds} \implies$
fmlookup (HashMap ?s)
(QueryIndexChallenge i
(state-after-query-chunks
(staged-query-start-hash data)

```

      (staged-query-chunks data) i)) =
    None
  unfolding staged-security-with-data-state-query-index-list-path-fresh-def
  by auto
from accepted-fri-opening-transcript-imp-accepted-transcript-shape[OF openings]
obtain as-shape where shape:
  accepted-transcript-shape ?s (Some (result, final-state))
  as-shape query-idxs
  by blast
from checked-staged-security-with-data-state-accepted-shape-query-keys
  [OF wf controlled support[unfolded out-eq] shape]
obtain raw-idxs-pair where
  len-raw-pair: length raw-idxs-pair = rounds
  and query-idxs-eq:
    query-idxs = map (λraw. index (to-nat raw)) raw-idxs-pair
  and lookup-pair:
    ∧ i. i < rounds ⇒
      fmllookup (HashMap final-state)
        (QueryIndexChallenge i
          (state-after-query-chunks
            (staged-query-start-hash data)
            (staged-query-chunks data) i)) =
        Some (raw-idxs-pair ! i)
  by blast
have raw-eq:
  ∧ i. i < rounds ⇒ raw-idxs ! i = raw-idxs-pair ! i
  using lookup lookup-pair by simp
have raw-idxs-eq: raw-idxs = raw-idxs-pair
  by (rule nth-equalityI)
  (use len-raw len-raw-pair raw-eq in simp-all)
have path-witness:
  length raw-idxs = rounds ∧
  query-idxs ∈ Q dg comp-bs ∧
  query-idxs = map (λraw. index (to-nat raw)) raw-idxs ∧
  (∀ i < rounds.
    fmllookup (HashMap final-state)
      (QueryIndexChallenge i
        (state-after-query-chunks
          (staged-query-start-hash data)
          (staged-query-chunks data) i)) =
      Some (raw-idxs ! i) ∧
    fmllookup (HashMap ?s)
      (QueryIndexChallenge i
        (state-after-query-chunks
          (staged-query-start-hash data)
          (staged-query-chunks data) i)) =
      None)
using len-raw raw-idxs-eq query-in query-idxs-eq lookup
  fresh-lookup by auto

```

```

have path-body:
   $\exists$  raw-idxs query-idxs.
    length raw-idxs = rounds  $\wedge$ 
    query-idxs  $\in Q$  dg comp-bs  $\wedge$ 
    query-idxs = map ( $\lambda$ raw. index (to-nat raw)) raw-idxs  $\wedge$ 
    ( $\forall i < \text{rounds}$ .
      fmllookup (HashMap final-state)
        (QueryIndexChallenge i
          (state-after-query-chunks
            (staged-query-start-hash data)
            (staged-query-chunks data) i)) =
        Some (raw-idxs ! i)  $\wedge$ 
      fmllookup (HashMap ?s)
        (QueryIndexChallenge i
          (state-after-query-chunks
            (staged-query-start-hash data)
            (staged-query-chunks data) i)) =
        None)
    using path-witness by blast
have path-single:
  staged-security-with-data-state-query-index-list-path-fresh
    (Q dg comp-bs) out
unfolding out-eq
  staged-security-with-data-state-query-index-list-path-fresh-def
using path-body by simp
have fresh-single:
  composition-fri-challenge-list-fresh-hit ?s
    ( $\lambda$ dg'. if dg' = dg then {comp-bs} else {})
    (Some (result, final-state))
proof –
  have witness:
    accepted-fri-challenges ?s (Some (result, final-state))
      trace-bs dg comp-bs  $\wedge$ 
      verifier-header-transcript ?s fr trace-roots trace-final as dg
      composition-roots final rest  $\wedge$ 
      comp-bs  $\in$  (if dg = dg then {comp-bs} else {})  $\wedge$ 
      composition-fri-challenge-path-fresh ?s fr trace-roots
      trace-final as dg composition-roots (length composition-roots)
    using accepted header fresh-path by simp
  show ?thesis
    unfolding composition-fri-challenge-list-fresh-hit-def
    using witness by blast
qed
have singleton:
  ?E (dg, comp-bs) out
proof –
  have fresh-single-pair:
    composition-fri-challenge-list-fresh-hit ?s
      ( $\lambda$ dg'. if dg' = dg then {snd (dg, comp-bs)} else {})

```

```

    (Some (result, final-state))
  proof -
    have
      (λdg'. if dg' = dg then {comp-bs} else {}) =
      (λdg'. if dg' = dg then {snd (dg, comp-bs)} else {})
    by (rule ext) (simp add: prod.sel)
    then show ?thesis
      using fresh-single by simp
  qed
  show ?thesis
  unfolding out-eq
    staged-security-composition-fri-pair-challenge-query-path-fresh-def
    staged-security-composition-fri-pair-challenge-fresh-def
    staged-security-with-data-state-verifier-event-def
  using pair-hit fresh-single-pair path-single out-eq by simp
  qed
  show ?thesis
    by (intro bexI[of - (dg, comp-bs)])
      (use comp-bs-in singleton in simp-all)
  qed
  qed
  also have ... ≤ (∑ p ∈ ?I. ?C p)
  proof (rule wp-event-finite-union-bound-fri-exact)
    show finite ?I
      by (rule finite-SigmaI)
        (use challenge-subset finite-fri-challenge-space in
          ‹auto intro: finite-subset[OF challenge-subset]›)
  next
    fix p
    assume p-in: p ∈ ?I
    obtain dg challenges where p-eq: p = (dg, challenges)
      by (cases p) auto
    have challenges-in: challenges ∈ B dg
      using p-in p-eq by simp
    have challenges-space:
      challenges ∈ fri-challenge-space (ceil-log (to-nat dg + 1))
      using challenge-subset[of dg] challenges-in by blast
    have fixed-bound:
      wp-event (checked-staged-security-experiment-with-data-state A)
        (staged-security-composition-fri-pair-challenge-query-path-fresh P
          (λdg'. if dg' = dg then {challenges} else {})
          (Q dg challenges))
      adversary-initial-state ≤
      (1 / nnreal (CARD('f) ^ ceil-log (to-nat dg + 1))) *
      (nnreal (card (Q dg challenges)) *
        (1 / nnreal (card query-sample-space)) ^ rounds)
    by (rule
      checked-staged-security-composition-fri-pair-challenge-query-path-fresh-fixed-bound
      [where dg = dg and challenges = challenges

```

```

    and  $Q = Q \text{ dg challenges}$  and  $P = P$ ,
     $OF \text{ wf controlled challenges-space}$ 
     $\text{projection}[OF \text{ challenges-in}] \text{ subset}[OF \text{ challenges-in}]$ )
  have event-eq:
    ?E  $p =$ 
      staged-security-composition-fri-pair-challenge-query-path-fresh  $P$ 
      ( $\lambda dg'. \text{ if } dg' = dg \text{ then } \{\text{challenges}\} \text{ else } \{\}$ )
      ( $Q \text{ dg challenges}$ )
    unfolding  $p\text{-eq}$ 
      staged-security-composition-fri-pair-challenge-query-path-fresh-def
      staged-security-composition-fri-pair-challenge-fresh-def
      composition-fri-challenge-list-fresh-hit-def
    by (rule ext) (simp add: prod.sel)
  have cost-eq:
    ?C  $p =$ 
      ( $1 / \text{nnreal} (\text{CARD}('f) \wedge \text{ceil-log} (\text{to-nat } dg + 1))) *$ 
      ( $\text{nnreal} (\text{card } (Q \text{ dg challenges})) *$ 
      ( $1 / \text{nnreal} (\text{card query-sample-space})) \wedge \text{rounds}$ )
    unfolding  $p\text{-eq}$  by simp
  show wp-event (checked-staged-security-experiment-with-data-state  $A$ )
    ( $?E p$ ) adversary-initial-state  $\leq ?C p$ 
    using fixed-bound unfolding event-eq cost-eq .
qed
finally have bound:
  wp-event (checked-staged-security-experiment-with-data-state  $A$ )
    (staged-security-composition-fri-pair-challenge-query-path-fresh  $P$ 
       $B ?Q$ )
    adversary-initial-state  $\leq (\sum p \in ?I. ?C p)$  .
have finite-B:  $\bigwedge dg. \text{ finite } (B \text{ dg})$ 
  by (meson challenge-subset finite-fri-challenge-space finite-subset)
have sigma-eq:
  ( $\sum p \in ?I. ?C p$ ) =
  ( $\sum dg \in UNIV. \sum challenges \in B \text{ dg.}$ 
    ( $1 / \text{nnreal} (\text{CARD}('f) \wedge \text{ceil-log} (\text{to-nat } dg + 1))) *$ 
    ( $\text{nnreal} (\text{card } (Q \text{ dg challenges})) *$ 
    ( $1 / \text{nnreal} (\text{card query-sample-space})) \wedge \text{rounds}$ ))
  by (subst sum.Sigma) (simp-all add: finite-B case-prod-beta)
show ?thesis
  using bound sigma-eq by simp
qed

end

end

```

```

theory Soundness-FRI-Query-Challenge-Trace-Staged-Product
  imports Soundness-FRI-Query-Challenge-Actual-Product

```

begin

Trace-side staged product accounting for the challenge-fresh/query-path-fresh FRI pair branch.

This theory is intentionally narrow. The preceding layer contains the verifier-local actual-fresh product bounds and the raw-list path bridges. Here we package the trace bridge into staged fixed-challenge and finite-cover product bounds.

context *soundness*

begin

lemma *verifier-after-trace-fri-query-index-list-path-fresh-imp-actual-raw-path:*

assumes *wf: staged-budget-wellformed budgets*

and *controlled: staged-adversary-controlled budgets A*

and *builder:*

Some (data, attacker-state) ∈
set-dist
(execute (checked-staged-transcript-program A)
adversary-initial-state)

and *prefix:*

Some ((fr, f-fl), prefix-state) ∈
set-dist
(execute verifier-trace-fri-prefix
(verifier-state-from-adversary attacker-state
(staged-proof-transcript data))))

and *read-trace-final:*

Some (f-final, s3) ∈ set-dist (execute read prefix-state)

and *alpha-out:*

Some (as, s4) ∈
set-dist (execute (mmap (replicate (length spec) alpha-round)) s3)

and *read-dg: Some (dg, s5) ∈ set-dist (execute read s4)*

and *degree-assert:*

Some ((), s6) ∈
set-dist (execute (assert (to-nat dg ≤ maxDegree)) s5)

and *comp-fri:*

Some (fl, s7) ∈
set-dist (execute
(ntimes receive-composition-fri-commits
(ceil-log (to-nat dg + 1))) s6)

and *read-final:*

Some (final, query-state) ∈ set-dist (execute read s7)

and *query-out:*

Some (result, final-state) ∈
set-dist
(execute
(ntimes
(verifier-query-round-program fr f-fl f-final as fl final)
rounds)
query-state)

and *hit:*

staged-security-with-data-state-query-index-list-path-fresh Q
 (Some (((data, attacker-state), result), final-state))
shows $\exists$ raws $\in$ *query-index-raw-list-preimage* Q .
query-rounds-raw-list-path-hit *query-state* raws rounds
 (Some (result, final-state))
proof –
 let ?s =
 verifier-state-from-adversary *attacker-state*
 (*staged-proof-transcript* data)
from *ntimes-verifier-query-rounds-outcome*[*OF* *query-out*]
obtain *raw-idxs* *query-idxs* *query-chunks* **where**
 len-raw: *length* *raw-idxs* = rounds
and *query-idxs-eq*:
 query-idxs = *map* (λ raw. *index* (*to-nat* raw)) *raw-idxs*
and *len-chunks*: *length* *query-chunks* = rounds
and *tr-query*:
 PTranscript *query-state* =
 List.concat *query-chunks* @ *PTranscript* *final-state*
and *state-final*:
 PState *final-state* =
 state-after-query-chunks (*PState* *query-state*) *query-chunks* rounds
and *round-chunks*:
 $\bigwedge i. i < \text{rounds} \implies$
 verifier-query-round-chunk (*query-idxs* ! i)
 (*map* *snd* *f-fl*) (*map* *snd* *fl*) (*query-chunks* ! i)
and *lookups*:
 $\bigwedge i. i < \text{rounds} \implies$
 fmlookup (*HashMap* *final-state*)
 (*QueryIndexChallenge* (*PQueryCounter* *query-state* + i)
 (*state-after-query-chunks*
 (*PState* *query-state*) *query-chunks* i)) =
 Some (*raw-idxs* ! i)
by *blast*
have *trace-prefix-res*:
 length *f-fl* = *ceil-log* *clength* $\wedge$
 PTranscript ?s = [*fr*] @ *map* *snd* *f-fl* @ *PTranscript* *prefix-state* $\wedge$
 PState *prefix-state* =
 foldl *concat* (*concat* (*PState* ?s) *fr*) (*map* *snd* *f-fl*) $\wedge$
 PQueryCounter *prefix-state* = *PQueryCounter* ?s
using *verifier-trace-fri-prefix-outcome*[*OF* *prefix*] **by** *simp*
from *read-outcome*[*OF* *read-trace-final*] **obtain** *rest3* **where**
 tr-prefix: *PTranscript* *prefix-state* = *f-final* # *rest3*
and *tr-s3*: *PTranscript* *s3* = *rest3*
and *state-s3*: *PState* *s3* = *concat* (*PState* *prefix-state*) *f-final*
and *counter-s3*: *PQueryCounter* *s3* = *PQueryCounter* *prefix-state*
by *blast*
have *alpha-res*:
 length *as* = *length* *spec* $\wedge$
 PTranscript *s3* = *as* @ *PTranscript* *s4* $\wedge$

```

    PState s4 = foldl concat (PState s3) as ∧
    PQueryCounter s4 = PQueryCounter s3
    using mmap-alpha-round-outcome[OF alpha-out] by simp
  from read-outcome[OF read-dg] obtain rest5 where
    tr-s4: PTranscript s4 = dg # rest5
    and tr-s5: PTranscript s5 = rest5
    and state-s5: PState s5 = concat (PState s4) dg
    and counter-s5: PQueryCounter s5 = PQueryCounter s4
  by blast
  have s6-eq: s6 = s5
    using degree-assert unfolding assert-def
    by (cases to-nat dg ≤ maxDegree) (auto simp: throw-no-outcome)
  have comp-res:
    length fl = ceil-log (to-nat dg + 1) ∧
    PTranscript s6 = map snd fl @ PTranscript s7 ∧
    PState s7 = foldl concat (PState s6) (map snd fl) ∧
    PQueryCounter s7 = PQueryCounter s6
    using ntimes-receive-composition-fri-commits-outcome[OF comp-fri]
    by simp
  from read-outcome[OF read-final] obtain rest-query where
    tr-s7: PTranscript s7 = final # rest-query
    and tr-read-query: PTranscript query-state = rest-query
    and state-query: PState query-state = concat (PState s7) final
    and counter-query-s7: PQueryCounter query-state = PQueryCounter s7
  by blast
  have local-counter: PQueryCounter query-state = 0
    using trace-prefix-res counter-s3 alpha-res counter-s5 s6-eq comp-res
    counter-query-s7
    unfolding verifier-state-from-adversary-def by simp
  have prefix-lookup:
    ∧i x. fmlookup (HashMap prefix-state)
      (QueryIndexChallenge i x) =
      fmlookup (HashMap ?s) (QueryIndexChallenge i x)
    by (rule verifier-trace-fri-prefix-preserves-query-lookup[OF prefix])
  have s3-lookup:
    HashMap s3 = HashMap prefix-state
    by (rule read-preserves-hash-map[OF read-trace-final])
  have s4-lookup:
    ∧i x. fmlookup (HashMap s4) (QueryIndexChallenge i x) =
      fmlookup (HashMap s3) (QueryIndexChallenge i x)
  proof -
    fix i x
    show fmlookup (HashMap s4) (QueryIndexChallenge i x) =
      fmlookup (HashMap s3) (QueryIndexChallenge i x)
    proof (rule mmap-preserves-lookup[OF - alpha-out])
      fix m y u v
      assume m-in: m ∈ set (replicate (length spec) alpha-round)
      and out: Some (y, v) ∈ set-dist (execute m u)
      then have m-eq: m = alpha-round

```



```

    by simp
  show fmllookup (HashMap v) (QueryIndexChallenge i x) =
    fmllookup (HashMap u) (QueryIndexChallenge i x)
    using alpha-round-preserves-query-lookup[OF out[unfolded m-eq]] .
qed
qed
have s5-lookup:
  HashMap s5 = HashMap s4
  by (rule read-preserves-hash-map[OF read-dg])
have s7-lookup:
   $\bigwedge i x. \text{fmllookup (HashMap s7) (QueryIndexChallenge i x) =}$ 
    fmllookup (HashMap s6) (QueryIndexChallenge i x)
proof -
  fix i x
  show fmllookup (HashMap s7) (QueryIndexChallenge i x) =
    fmllookup (HashMap s6) (QueryIndexChallenge i x)
  proof (rule ntimes-preserves-lookup[OF - comp-fri])
    fix y u v
    assume out:
      Some (y, v)  $\in$ 
        set-dist (execute receive-composition-fri-commits u)
    obtain b r where y-eq: y = (b, r)
    by (cases y) simp
    show fmllookup (HashMap v) (QueryIndexChallenge i x) =
      fmllookup (HashMap u) (QueryIndexChallenge i x)
    by (rule receive-composition-fri-commits-preserves-query-lookup
        [OF out[unfolded y-eq]])
  qed
qed
qed
have query-lookup:
  HashMap query-state = HashMap s7
  by (rule read-preserves-hash-map[OF read-final])
have header-actual:
  verifier-header-transcript ?s fr (map snd f-fl) f-final as dg
  (map snd fl) final (PTranscript query-state)
  using trace-prefix-res tr-prefix tr-s3 alpha-res tr-s4 tr-s5 s6-eq
    comp-res tr-s7 tr-read-query
  unfolding verifier-header-transcript-def verifier-header-messages-def
  by simp
have staged-header:
  verifier-header-transcript ?s
    (staged-trace-root data)
    (staged-trace-fri-roots data)
    (staged-trace-final data)
    (staged-alphas data)
    (staged-degree data)
    (staged-composition-fri-roots data)
    (staged-composition-final data)
    (List.concat (staged-query-chunks data))

```

```

  by (rule checked-staged-transcript-program-outcome-header-transcript
    [OF wf controlled builder])
have header-unique:
  fr = staged-trace-root data ∧
  map snd f-fl = staged-trace-fri-roots data ∧
  f-final = staged-trace-final data ∧
  as = staged-alphas data ∧
  dg = staged-degree data ∧
  map snd fl = staged-composition-fri-roots data ∧
  final = staged-composition-final data ∧
  PTranscript query-state = List.concat (staged-query-chunks data)
using verifier-header-transcript-unique[OF header-actual staged-header]
by simp
have prefix-state-query:
  PState query-state = staged-query-start-hash data
using trace-prefix-res state-s3 alpha-res state-s5 s6-eq comp-res
  state-query header-unique
unfolding staged-query-start-hash-def
  staged-composition-fri-start-hash-def
  staged-trace-fri-start-hash-def
by simp
have chunks-eq: query-chunks = staged-query-chunks data
proof -
  from checked-staged-transcript-program-query-chunks-match-verifier-lengths
    [OF builder]
  obtain staged-query-idxs where
    staged-match:
      staged-query-chunks-match-verifier-lengths data staged-query-idxs
  by blast
have query-idxs-len: length query-idxs = rounds
  using query-idxs-eq len-row by simp
have staged-match-query-idxs:
  staged-query-chunks-match-verifier-lengths data query-idxs
  by (rule staged-query-chunks-match-verifier-lengths-transfer
    [OF staged-match query-idxs-len])
have parser-chunk-len:
  ∧ i. i < rounds ⇒
    length (query-chunks ! i) =
      verifier-query-round-transcript-length (query-idxs ! i)
        (staged-trace-fri-roots data)
        (staged-composition-fri-roots data)
proof -
  fix i
  assume i-bound: i < rounds
  show
    length (query-chunks ! i) =
      verifier-query-round-transcript-length (query-idxs ! i)
        (staged-trace-fri-roots data)
        (staged-composition-fri-roots data)

```

```

    using verifier-query-round-chunk-length[OF round-chunks[OF i-bound]]
      header-unique
    by simp
qed
have concat-staged:
  List.concat query-chunks @ PTranscript final-state =
    List.concat (staged-query-chunks data)
  using tr-query header-unique by simp
then show ?thesis
  using query-chunks-eq-staged-if-matching-lengths
    [OF len-chunks concat-staged parser-chunk-len
      staged-match-query-idxs]
  by simp
qed
from hit obtain raw-idxs-hit query-idxs-hit where
  query-hit-in: query-idxs-hit ∈ Q
  and len-hit: length raw-idxs-hit = rounds
  and query-hit-eq:
    query-idxs-hit =
      map (λraw. index (to-nat raw)) raw-idxs-hit
  and lookup-hit:
    ∧ i. i < rounds ⇒
      fmllookup (HashMap final-state)
        (QueryIndexChallenge i
          (state-after-query-chunks
            (staged-query-start-hash data)
            (staged-query-chunks data) i)) =
        Some (raw-idxs-hit ! i)
  and fresh-hit:
    ∧ i. i < rounds ⇒
      fmllookup (HashMap ?s)
        (QueryIndexChallenge i
          (state-after-query-chunks
            (staged-query-start-hash data)
            (staged-query-chunks data) i)) =
        None
  unfolding staged-security-with-data-state-query-index-list-path-fresh-def
  by auto
have raw-eq:
  ∧ i. i < rounds ⇒ raw-idxs ! i = raw-idxs-hit ! i
  using lookups lookup-hit local-counter prefix-state-query chunks-eq
  by simp
have raw-idxs-eq: raw-idxs = raw-idxs-hit
  by (rule nth-equalityI)
  (use len-raw len-hit raw-eq in simp-all)
have raw-in: raw-idxs ∈ query-index-raw-list-preimage Q
  unfolding query-index-raw-list-preimage-def
  using len-raw raw-idxs-eq query-hit-in query-hit-eq by blast
have query-lookup-start:

```

```

 $\wedge i x.$  fmlookup (HashMap query-state) (QueryIndexChallenge i x) =
  fmlookup (HashMap ?s) (QueryIndexChallenge i x)
using prefix-lookup s3-lookup s4-lookup s5-lookup s6-eq
  s7-lookup query-lookup by simp
have local-fresh:
  query-index-list-path-fresh query-state rounds query-chunks raw-idxs
  unfolding query-index-list-path-fresh-def
proof (intro conjI allI impI)
  show length raw-idxs = rounds
    using len-raw .
next
  show length query-chunks = rounds
    using len-chunks .
next
  fix i
  assume i-bound:  $i < \text{rounds}$ 
  let ?x =
    state-after-query-chunks (staged-query-start-hash data)
    (staged-query-chunks data) i
  have fmlookup (HashMap query-state)
    (QueryIndexChallenge (PQueryCounter query-state + i)
      (state-after-query-chunks (PState query-state) query-chunks i)) =
    fmlookup (HashMap ?s) (QueryIndexChallenge i ?x)
  using local-counter prefix-state-query chunks-eq
    query-lookup-start[of i ?x] by simp
  also have ... = None
    by (rule fresh-hit[OF i-bound])
  finally show
    fmlookup (HashMap query-state)
      (QueryIndexChallenge (PQueryCounter query-state + i)
        (state-after-query-chunks (PState query-state) query-chunks i)) =
      None .
qed
have path:
  query-rounds-raw-list-path-hit query-state raw-idxs rounds
    (Some (result, final-state))
  unfolding query-rounds-raw-list-path-hit-def
  using len-raw len-chunks state-final local-fresh lookups
  by auto
show ?thesis
  using raw-in path by blast
qed

lemma verifier-after-trace-fri-query-index-list-path-fresh-product-bound-after-prefix:
assumes wf: staged-budget-wellformed budgets
and controlled: staged-adversary-controlled budgets A
and subset:  $Q \subseteq \text{fri-query-index-list-space}$ 
and builder:
  Some (data, attacker-state)  $\in$ 

```

```

    set-dist
      (execute (checked-staged-transcript-program A)
        adversary-initial-state)
  and prefix:
    Some (header, prefix-state) ∈
      set-dist
        (execute verifier-trace-fri-prefix
          (verifier-state-from-adversary attacker-state
            (staged-proof-transcript data)))
  shows
    wp-event (verifier-after-trace-fri header)
      (λout.
        staged-security-with-data-state-query-index-list-path-fresh Q
          (case out of
            None ⇒ None
            | Some (result, final-state) ⇒
              Some (((data, attacker-state), result), final-state)))
        prefix-state
        ≤ nnreal (card Q) *
          (1 / nnreal (card query-sample-space)) ^ rounds
  proof -
  obtain fr f-fl where header-eq: header = (fr, f-fl)
  by (cases header) auto
  let ?P =
    λout.
      staged-security-with-data-state-query-index-list-path-fresh Q
        (case out of
          None ⇒ None
          | Some (result, final-state) ⇒
            Some (((data, attacker-state), result), final-state))
  let ?C =
    nnreal (card Q) *
      (1 / nnreal (card query-sample-space)) ^ rounds
  have prefix':
    Some ((fr, f-fl), prefix-state) ∈
      set-dist
        (execute verifier-trace-fri-prefix
          (verifier-state-from-adversary attacker-state
            (staged-proof-transcript data)))
  using prefix header-eq by simp
  have after-eq:
    verifier-after-trace-fri header =
      (read ≧=
        (λf-final.
          mmap (replicate (length spec) alpha-round) ≧=
            (λas. read ≧=
              (λdg. assert (to-nat dg ≤ maxDegree) ≧=
                (λ-. ntimes receive-composition-fri-commits
                  (ceil-log (to-nat dg + 1)) ≧=

```

```

      (λfl. read >=>
        (λfinal.
          ntimes
            (verifier-query-round-program fr f-fl f-final as fl
              final)
            rounds))))))
  unfolding header-eq verifier-after-trace-fri-def by simp
show ?thesis
  unfolding after-eq
proof (rule wp-event-bind-bound-by-cont)
  show ¬ ?P None
    unfolding staged-security-with-data-state-query-index-list-path-fresh-def
    by simp
next
fix f-final s3
assume read-trace-final:
  Some (f-final, s3) ∈ set-dist (execute read prefix-state)
show
  wp-event
    (mmap (replicate (length spec) alpha-round) >=>
      (λas. read >=>
        (λdg. assert (to-nat dg ≤ maxDegree) >=>
          (λ-. ntimes receive-composition-fri-commits
            (ceil-log (to-nat dg + 1)) >=>
              (λfl. read >=>
                (λfinal.
                  ntimes
                    (verifier-query-round-program fr f-fl f-final as fl
                      final)
                    rounds)))))) ?P s3 ≤ ?C
      )
    )
proof (rule wp-event-bind-bound-by-cont)
  show ¬ ?P None
    unfolding staged-security-with-data-state-query-index-list-path-fresh-def
    by simp
next
fix as s4
assume alpha-out:
  Some (as, s4) ∈
    set-dist
      (execute (mmap (replicate (length spec) alpha-round)) s3)
show
  wp-event
    (read >=>
      (λdg. assert (to-nat dg ≤ maxDegree) >=>
        (λ-. ntimes receive-composition-fri-commits
          (ceil-log (to-nat dg + 1)) >=>
            (λfl. read >=>
              (λfinal.
                ntimes

```

```

      (verifier-query-round-program fr f-fl f-final as fl
       final)
      rounds)))) ?P s4 ≤ ?C
proof (rule wp-event-bind-bound-by-cont)
  show ¬ ?P None
    unfolding staged-security-with-data-state-query-index-list-path-fresh-def
    by simp
next
  fix dg s5
  assume read-dg:
    Some (dg, s5) ∈ set-dist (execute read s4)
  show
    wp-event
      (assert (to-nat dg ≤ maxDegree) >>=
       (λ-. ntimes receive-composition-fri-commits
        (ceil-log (to-nat dg + 1)) >>=
        (λfl. read >>=
         (λfinal.
          ntimes
            (verifier-query-round-program fr f-fl f-final as fl
             final)
            rounds)))) ?P s5 ≤ ?C
proof (rule wp-event-bind-bound-by-cont)
  show ¬ ?P None
    unfolding staged-security-with-data-state-query-index-list-path-fresh-def
    by simp
next
  fix unit s6
  assume degree-assert:
    Some (unit, s6) ∈
      set-dist (execute (assert (to-nat dg ≤ maxDegree)) s5)
  show
    wp-event
      (ntimes receive-composition-fri-commits
       (ceil-log (to-nat dg + 1)) >>=
       (λfl. read >>=
        (λfinal.
         ntimes
           (verifier-query-round-program fr f-fl f-final as fl
            final)
           rounds)))) ?P s6 ≤ ?C
proof (rule wp-event-bind-bound-by-cont)
  show ¬ ?P None
    unfolding staged-security-with-data-state-query-index-list-path-fresh-def
    by simp
next
  fix fl s7
  assume comp-fri:
    Some (fl, s7) ∈

```

```

    set-dist
    (execute
      (ntimes receive-composition-fri-commits
        (ceil-log (to-nat dg + 1))) s6)
  show
    wp-event
    (read  $\gg$ 
      ( $\lambda$ final.
        ntimes
          (verifier-query-round-program fr f-fl f-final as fl
            final)
          rounds)) ?P s7  $\leq$  ?C
  proof (rule wp-event-bind-bound-by-cont)
  show  $\neg$  ?P None
    unfolding staged-security-with-data-state-query-index-list-path-fresh-def
    by simp
  next
  fix final query-state
  assume read-final:
    Some (final, query-state)  $\in$  set-dist (execute read s7)
  have degree-assert-unit:
    Some ((), s6)  $\in$ 
      set-dist (execute (assert (to-nat dg  $\leq$  maxDegree)) s5)
  using degree-assert by (cases unit) simp
  have mono:
    wp-event
    (ntimes
      (verifier-query-round-program fr f-fl f-final as fl final)
      rounds)
    ?P query-state  $\leq$ 
    wp-event
    (ntimes
      (verifier-query-round-program fr f-fl f-final as fl final)
      rounds)
    ( $\lambda$ out.  $\exists$  rows  $\in$  query-index-raw-list-preimage Q.
      query-rounds-raw-list-path-hit query-state rows rounds out)
    query-state
  proof (rule wp-event-mono-on-support)
  fix out
  assume query-out:
    out  $\in$ 
      set-dist
      (execute
        (ntimes
          (verifier-query-round-program fr f-fl f-final as fl
            final)
          rounds)
        query-state)
  and hit: ?P out

```



```

show  $\exists \text{raws} \in \text{query-index-raw-list-preimage } Q.$ 
  query-rounds-raw-list-path-hit query-state raws rounds out
proof (cases out)
  case None
  then show ?thesis
    using hit
    unfolding staged-security-with-data-state-query-index-list-path-fresh-def
    by simp
next
  case (Some pair)
  then obtain result final-state where out-eq:
    out = Some (result, final-state)
    by (cases pair) simp
  have query-out':
    Some (result, final-state) ∈
      set-dist
      (execute
        (ntimes
          (verifier-query-round-program fr f-fl f-final as fl
            final)
          rounds)
        query-state)
    using query-out out-eq by simp
  have hit':
    staged-security-with-data-state-query-index-list-path-fresh Q
    (Some (((data, attacker-state), result), final-state))
    using hit out-eq by simp
  show ?thesis
    using
      verifier-after-trace-fri-query-index-list-path-fresh-imp-actual-raw-path
      [OF wf controlled builder prefix' read-trace-final
        alpha-out read-dg degree-assert-unit comp-fri read-final
        query-out' hit']
    unfolding out-eq .
  qed
qed
also have  $\dots \leq ?C$ 
  by (rule
    wp-ntimes-verifier-query-round-program-query-list-path-product-bound
    [OF subset])
finally show
  wp-event
  (ntimes
    (verifier-query-round-program fr f-fl f-final as fl final)
    rounds)
  ?P query-state ≤ ?C .
qed
qed
qed

```

```

qed
qed
qed
qed

lemma checked-staged-security-trace-fri-pair-challenge-query-path-fresh-fixed-bound:
  assumes wf: staged-budget-wellformed budgets
  and controlled: staged-adversary-controlled budgets A
  and challenges-space:
    challenges  $\in$  fri-challenge-space (ceil-log clength)
  and projection:
    fst ' (P  $\cap$  (UNIV  $\times$  {challenges}))  $\subseteq$  Q
  and subset: Q  $\subseteq$  fri-query-index-list-space
  shows
    wp-event (checked-staged-security-experiment-with-data-state A)
      (staged-security-trace-fri-pair-challenge-query-path-fresh P
        {challenges} Q)
    adversary-initial-state  $\leq$ 
      (1 / nnreal (CARD('f) ^ ceil-log clength)) *
      (nnreal (card Q) *
        (1 / nnreal (card query-sample-space)) ^ rounds)
  proof (rule checked-staged-security-with-data-state-bound-from-data-cont)
    let ?D =
      nnreal (card Q) *
      (1 / nnreal (card query-sample-space)) ^ rounds
    show  $\neg$  staged-security-trace-fri-pair-challenge-query-path-fresh
      P {challenges} Q None
    unfolding staged-security-trace-fri-pair-challenge-query-path-fresh-def
      staged-security-trace-fri-pair-challenge-fresh-def
      staged-security-with-data-state-verifier-event-def
    by simp
  next
    fix data attacker-state
    assume builder:
      Some (data, attacker-state)  $\in$ 
        set-dist
          (execute (checked-staged-transcript-program A)
            adversary-initial-state)
    let ?s =
      verifier-state-from-adversary attacker-state
      (staged-proof-transcript data)
    let ?Event =
       $\lambda$ out.
        staged-security-trace-fri-pair-challenge-query-path-fresh P
          {challenges} Q
        (case out of
          None  $\Rightarrow$  None
          | Some (result, final-state)  $\Rightarrow$ 
            Some (((data, attacker-state), result), final-state))

```

```

let ?Head =
  λout. case out of
    None ⇒ False
  | Some ((fr, f-fl), -) ⇒
    map fst f-fl = challenges ∧
    trace-fri-challenge-path-fresh ?s fr (map snd f-fl)
    (length (map snd f-fl))
show wp-event verify-monad ?Event ?s ≤
  (1 / nnreal (CARD('f) ^ ceil-log clength)) *
  (nnreal (card Q) *
    (1 / nnreal (card query-sample-space)) ^ rounds)
unfolding verify-monad-trace-fri-decomposition
proof (rule order-trans)
show wp-event
  (verifier-trace-fri-prefix ≫= verifier-after-trace-fri)
  ?Event ?s
≤ wp-event verifier-trace-fri-prefix ?Head ?s *
  (nnreal (card Q) *
    (1 / nnreal (card query-sample-space)) ^ rounds)
proof (rule wp-event-bind-bound-by-head-and-cont)
show ¬ ?Event None
  unfolding staged-security-trace-fri-pair-challenge-query-path-fresh-def
    staged-security-trace-fri-pair-challenge-fresh-def
    staged-security-with-data-state-verifier-event-def
  by simp
next
fix header prefix-state
assume prefix:
  Some (header, prefix-state) ∈
    set-dist (execute verifier-trace-fri-prefix ?s)
  and not-head: ¬ ?Head (Some (header, prefix-state))
obtain fr f-fl where header-eq: header = (fr, f-fl)
  by (cases header) auto
have not-head':
  map fst f-fl ≠ challenges ∨
  ¬ trace-fri-challenge-path-fresh ?s fr (map snd f-fl)
  (length (map snd f-fl))
  using not-head unfolding header-eq by simp
show wp-event (verifier-after-trace-fri header) ?Event prefix-state =
  0
proof (rule antisym)
show wp-event (verifier-after-trace-fri header) ?Event
  prefix-state ≤ 0
proof (rule order-trans)
show wp-event (verifier-after-trace-fri header) ?Event
  prefix-state ≤
  wp-event (verifier-after-trace-fri header)
  (λ-. False) prefix-state
proof (rule wp-event-mono-on-support)

```

```

fix out
assume suffix:
  out ∈
    set-dist
      (execute (verifier-after-trace-fri header) prefix-state)
  and hit: ?Event out
show False
proof (cases out)
  case None
  then show ?thesis
  using hit
  unfolding
    staged-security-trace-fri-pair-challenge-query-path-fresh-def
    staged-security-trace-fri-pair-challenge-fresh-def
    staged-security-with-data-state-verifier-event-def
  by simp
next
case (Some pair)
then obtain result final-state where out-eq:
  out = Some (result, final-state)
  by (cases pair) simp
from hit out-eq have fresh:
  trace-fri-challenge-list-fresh-hit ?s {challenges}
  (Some (result, final-state))
and pair-hit:
  trace-fri-query-challenge-pair-set-hit ?s P
  (Some (result, final-state))
unfolding
  staged-security-trace-fri-pair-challenge-query-path-fresh-def
  staged-security-trace-fri-pair-challenge-fresh-def
  staged-security-with-data-state-verifier-event-def
by simp-all
have prefix':
  Some ((fr, f-fl), prefix-state) ∈
    set-dist (execute verifier-trace-fri-prefix ?s)
using prefix header-eq by simp
have suffix':
  Some (result, final-state) ∈
    set-dist
      (execute (verifier-after-trace-fri (fr, f-fl))
        prefix-state)
using suffix header-eq out-eq by simp
from pair-hit obtain trace-roots trace-bs trace-final fri-dg
  composition-roots composition-bs composition-final query-idxs
  trace-round-layers composition-round-layers where
openings:
  accepted-fri-opening-transcript ?s
  (Some (result, final-state)) trace-roots trace-bs
  trace-final fri-dg composition-roots composition-bs

```

```

      composition-final query-idxs trace-round-layers
      composition-round-layers
    unfolding trace-fri-query-challenge-pair-set-hit-def by blast
  have pair-challenges:
    accepted-fri-challenges ?s (Some (result, final-state))
    trace-bs fri-dg composition-bs
  by (rule accepted-fri-opening-transcript-challenges
    [OF openings])
  from fresh obtain trace-bs' dg' comp-bs' where
    fresh-challenges:
      accepted-fri-challenges ?s (Some (result, final-state))
      trace-bs' dg' comp-bs'
  and trace-bs'-in: trace-bs' ∈ {challenges}
  unfolding trace-fri-challenge-list-fresh-hit-def by blast
  have trace-bs-eq: trace-bs = trace-bs'
  using accepted-fri-challenges-unique
    [OF pair-challenges fresh-challenges]
  by simp
  have prefix-trace-eq: trace-bs = map fst f-fl
  by (rule
    verifier-after-trace-fri-header-agrees-with-accepted-challenges
    [OF prefix' suffix' pair-challenges])
  have concrete-fresh:
    trace-fri-challenge-path-fresh ?s fr (map snd f-fl)
    (length (map snd f-fl))
  by (rule verifier-after-trace-fri-header-roots-agree-with-fresh-hit
    [OF prefix' suffix' fresh])
  have head-holds:
    map fst f-fl = challenges ∧
    trace-fri-challenge-path-fresh ?s fr (map snd f-fl)
    (length (map snd f-fl))
  using prefix-trace-eq trace-bs-eq trace-bs'-in
    concrete-fresh by simp
  show ?thesis
  using not-head' head-holds by blast
qed
qed
next
  show wp-event (verifier-after-trace-fri header)
    (λ-. False) prefix-state ≤ 0
  unfolding wp-event-def wp-def dist-expect-def by simp
qed
next
  show 0 ≤ wp-event (verifier-after-trace-fri header) ?Event
    prefix-state
  by simp
qed
next
  fix header prefix-state

```

```

assume prefix:
  Some (header, prefix-state) ∈
    set-dist (execute verifier-trace-fri-prefix ?s)
  and head: ?Head (Some (header, prefix-state))
obtain fr f-fl where header-eq: header = (fr, f-fl)
  by (cases header) auto
have event-mono:
  wp-event (verifier-after-trace-fri header) ?Event prefix-state ≤
    wp-event (verifier-after-trace-fri header)
    (λout.
      staged-security-with-data-state-query-index-list-path-fresh Q
      (case out of
        None ⇒ None
        | Some (result, final-state) ⇒
          Some (((data, attacker-state), result), final-state)))
      prefix-state
    )
  by (rule wp-event-mono-on-support)
    (auto simp: staged-security-trace-fri-pair-challenge-query-path-fresh-def)
also have ... ≤
  nnreal (card Q) *
    (1 / nnreal (card query-sample-space)) ^ rounds
  by (rule
    verifier-after-trace-fri-query-index-list-path-fresh-product-bound-after-prefix
    [OF wf controlled subset builder prefix])
finally show wp-event (verifier-after-trace-fri header) ?Event
  prefix-state ≤
    nnreal (card Q) *
      (1 / nnreal (card query-sample-space)) ^ rounds .
qed
next
show wp-event verifier-trace-fri-prefix ?Head ?s *
  (nnreal (card Q) *
    (1 / nnreal (card query-sample-space)) ^ rounds)
  ≤
  (1 / nnreal (CARD('f) ^ ceil-log clength)) *
  (nnreal (card Q) *
    (1 / nnreal (card query-sample-space)) ^ rounds)
proof (rule mult-right-mono)
show wp-event verifier-trace-fri-prefix ?Head ?s ≤
  1 / nnreal (CARD('f) ^ ceil-log clength)
proof –
  let ?HeadSet =
    λout. case out of
      None ⇒ False
      | Some ((fr, f-fl), -) ⇒
        map fst f-fl ∈ {challenges} ∧
        trace-fri-challenge-path-fresh ?s fr (map snd f-fl)
        (length (map snd f-fl))
  have mono:

```

```

wp-event verifier-trace-fri-prefix ?Head ?s ≤
  wp-event verifier-trace-fri-prefix ?HeadSet ?s
by (rule wp-event-mono) (auto split: option.splits prod.splits)
have set-bound:
  wp-event verifier-trace-fri-prefix ?HeadSet ?s ≤
    nnreal (card {challenges}) /
    nnreal (CARD('f) ^ ceil-log clength)
by (rule
  wp-verifier-trace-fri-prefix-challenge-list-actual-fresh-bound
  [where B = {challenges}])
  (use challenges-space in auto)
show ?thesis
  using order-trans[OF mono set-bound] by simp
qed
show 0 ≤ nnreal (card Q) *
  (1 / nnreal (card query-sample-space)) ^ rounds
  by simp
qed
qed
qed
qed

lemma checked-staged-security-trace-fri-pair-challenge-query-path-fresh-fiber-sum:
  assumes wf: staged-budget-wellformed budgets
  and controlled: staged-adversary-controlled budgets A
  and challenge-subset: B ⊆ fri-challenge-space (ceil-log clength)
  and projection:
    ∧challenges.
      challenges ∈ B ⇒
        fst ' (P ∩ (UNIV × {challenges})) ⊆ Q challenges
  and subset:
    ∧challenges. challenges ∈ B ⇒
      Q challenges ⊆ fri-query-index-list-space
  shows
    wp-event (checked-staged-security-experiment-with-data-state A)
      (staged-security-trace-fri-pair-challenge-query-path-fresh P B
        (⋃ challenges ∈ B. Q challenges))
      adversary-initial-state ≤
      (∑ challenges ∈ B.
        (1 / nnreal (CARD('f) ^ ceil-log clength)) *
        (nnreal (card (Q challenges)) *
        (1 / nnreal (card query-sample-space)) ^ rounds))
proof -
  let ?Q = (⋃ challenges ∈ B. Q challenges)
  let ?E =
    λchallenges.
      staged-security-trace-fri-pair-challenge-query-path-fresh P
        {challenges} (Q challenges)
  let ?C =
    λchallenges.

```

```

(1 / nnreal (CARD('f) ^ ceil-log clength)) *
  (nnreal (card (Q challenges)) *
    (1 / nnreal (card query-sample-space)) ^ rounds)
have split:
  wp-event (checked-staged-security-experiment-with-data-state A)
    (staged-security-trace-fri-pair-challenge-query-path-fresh P B ?Q)
  adversary-initial-state ≤
  wp-event (checked-staged-security-experiment-with-data-state A)
    (λout. ∃ challenges ∈ B. ?E challenges out)
  adversary-initial-state
proof (rule wp-event-mono-on-support)
fix out
assume support:
  out ∈
  set-dist
  (execute (checked-staged-security-experiment-with-data-state A)
    adversary-initial-state)
and hit:
  staged-security-trace-fri-pair-challenge-query-path-fresh P B ?Q out
show ∃ challenges ∈ B. ?E challenges out
proof (cases out)
case None
then show ?thesis
  using hit
  unfolding staged-security-trace-fri-pair-challenge-query-path-fresh-def
    staged-security-trace-fri-pair-challenge-fresh-def
    staged-security-with-data-state-verifier-event-def
  by simp
next
case (Some packed)
then obtain data attacker-state result final-state where out-eq:
  out = Some (((data, attacker-state), result), final-state)
  by (cases packed) (auto split: prod.splits)
let ?s =
  verifier-state-from-adversary attacker-state
  (staged-proof-transcript data)
from hit out-eq have pair-hit:
  trace-fri-query-challenge-pair-set-hit ?s P
  (Some (result, final-state))
and fresh:
  trace-fri-challenge-list-fresh-hit ?s B
  (Some (result, final-state))
and path:
  staged-security-with-data-state-query-index-list-path-fresh ?Q out
unfolding staged-security-trace-fri-pair-challenge-query-path-fresh-def
  staged-security-trace-fri-pair-challenge-fresh-def
  staged-security-with-data-state-verifier-event-def
by simp-all
from fresh obtain trace-bs dg comp-bs fr trace-roots trace-final as

```



```

    composition-roots final rest where
accepted:
    accepted-fri-challenges ?s (Some (result, final-state))
    trace-bs dg comp-bs
and trace-bs-in: trace-bs ∈ B
and header:
    verifier-header-transcript ?s fr trace-roots trace-final as dg
    composition-roots final rest
and fresh-path:
    trace-fri-challenge-path-fresh ?s fr trace-roots
    (length trace-roots)
unfolding trace-fri-challenge-list-fresh-hit-def by blast
from pair-hit obtain trace-roots' trace-bs' trace-final' fri-dg
    composition-roots' composition-bs' composition-final' query-idxs
    trace-round-layers composition-round-layers where
openings:
    accepted-fri-opening-transcript ?s
    (Some (result, final-state)) trace-roots' trace-bs'
    trace-final' fri-dg composition-roots' composition-bs'
    composition-final' query-idxs trace-round-layers
    composition-round-layers
and pair: (query-idxs, trace-bs') ∈ P
unfolding trace-fri-query-challenge-pair-set-hit-def by blast
have pair-challenges:
    accepted-fri-challenges ?s (Some (result, final-state))
    trace-bs' fri-dg composition-bs'
by (rule accepted-fri-opening-transcript-challenges[OF openings])
have trace-eq: trace-bs' = trace-bs
using accepted-fri-challenges-unique[OF pair-challenges accepted]
by simp
have query-in: query-idxs ∈ Q trace-bs
proof –
    have query-idxs ∈ fst ' (P ∩ (UNIV × {trace-bs}))
    using pair trace-eq by force
    then show ?thesis
    using projection[OF trace-bs-in] by blast
qed
from path out-eq obtain raw-idxs query-idxs-path where
    len-raw: length raw-idxs = rounds
and query-path-in: query-idxs-path ∈ ?Q
and query-path-eq:
    query-idxs-path =
    map (λraw. index (to-nat raw)) raw-idxs
and lookup:
    ∧i. i < rounds ⇒
    fmlookup (HashMap final-state)
    (QueryIndexChallenge i
    (state-after-query-chunks
    (staged-query-start-hash data)

```

```

      (staged-query-chunks data) i)) =
    Some (raw-idxs ! i)
and fresh-lookup:
   $\bigwedge i. i < \text{rounds} \implies$ 
    fmllookup (HashMap ?s)
      (QueryIndexChallenge i
        (state-after-query-chunks
          (staged-query-start-hash data)
          (staged-query-chunks data) i)) =
      None
unfolding staged-security-with-data-state-query-index-list-path-fresh-def
by auto
from accepted-fri-opening-transcript-imp-accepted-transcript-shape[OF openings]
obtain as-shape where shape:
  accepted-transcript-shape ?s (Some (result, final-state))
  as-shape query-idxs
by blast
from checked-staged-security-with-data-state-accepted-shape-query-keys
  [OF wf controlled support[unfolded out-eq] shape]
obtain raw-idxs-pair where
  len-raw-pair: length raw-idxs-pair = rounds
and query-idxs-eq:
  query-idxs = map ( $\lambda \text{raw}. \text{index } (\text{to-nat } \text{raw})$ ) raw-idxs-pair
and lookup-pair:
   $\bigwedge i. i < \text{rounds} \implies$ 
    fmllookup (HashMap final-state)
      (QueryIndexChallenge i
        (state-after-query-chunks
          (staged-query-start-hash data)
          (staged-query-chunks data) i)) =
      Some (raw-idxs-pair ! i)
by blast
have raw-eq:
   $\bigwedge i. i < \text{rounds} \implies \text{raw-idxs } ! i = \text{raw-idxs-pair } ! i$ 
using lookup lookup-pair by simp
have raw-idxs-eq: raw-idxs = raw-idxs-pair
by (rule nth-equalityI)
  (use len-raw len-raw-pair raw-eq in simp-all)
have path-witness:
  length raw-idxs = rounds  $\wedge$ 
  query-idxs  $\in Q$  trace-bs  $\wedge$ 
  query-idxs = map ( $\lambda \text{raw}. \text{index } (\text{to-nat } \text{raw})$ ) raw-idxs  $\wedge$ 
  ( $\forall i < \text{rounds}.$ 
    fmllookup (HashMap final-state)
      (QueryIndexChallenge i
        (state-after-query-chunks
          (staged-query-start-hash data)
          (staged-query-chunks data) i)) =
      Some (raw-idxs ! i)  $\wedge$ 

```

```

    fmllookup (HashMap ?s)
      (QueryIndexChallenge i
        (state-after-query-chunks
          (staged-query-start-hash data)
          (staged-query-chunks data) i)) =
        None)
  using len-raw raw-idxs-eq query-in query-idxs-eq lookup
    fresh-lookup by auto
have path-body:
  ∃ raw-idxs query-idxs.
    length raw-idxs = rounds ∧
    query-idxs ∈ Q trace-bs ∧
    query-idxs = map (λraw. index (to-nat raw)) raw-idxs ∧
    (∀ i < rounds.
      fmllookup (HashMap final-state)
        (QueryIndexChallenge i
          (state-after-query-chunks
            (staged-query-start-hash data)
            (staged-query-chunks data) i)) =
          Some (raw-idxs ! i) ∧
      fmllookup (HashMap ?s)
        (QueryIndexChallenge i
          (state-after-query-chunks
            (staged-query-start-hash data)
            (staged-query-chunks data) i)) =
          None)
  using path-witness by blast
have path-single:
  staged-security-with-data-state-query-index-list-path-fresh
    (Q trace-bs) out
  unfolding out-eq
    staged-security-with-data-state-query-index-list-path-fresh-def
  using path-body by simp
have fresh-single:
  trace-fri-challenge-list-fresh-hit ?s {trace-bs}
    (Some (result, final-state))
proof -
  have witness:
    accepted-fri-challenges ?s (Some (result, final-state))
      trace-bs dg comp-bs ∧
    verifier-header-transcript ?s fr trace-roots trace-final as dg
      composition-roots final rest ∧
    trace-bs ∈ {trace-bs} ∧
    trace-fri-challenge-path-fresh ?s fr trace-roots
      (length trace-roots)
  using accepted header fresh-path by simp
show ?thesis
  unfolding trace-fri-challenge-list-fresh-hit-def
  using witness by blast

```

```

qed
have singleton:
  ?E trace-bs out
  unfolding out-eq
    staged-security-trace-fri-pair-challenge-query-path-fresh-def
    staged-security-trace-fri-pair-challenge-fresh-def
    staged-security-with-data-state-verifier-event-def
  using pair-hit fresh-single path-single out-eq by simp
show ?thesis
  using singleton trace-bs-in by blast
qed
qed
also have ...  $\leq (\sum \text{challenges} \in B. ?C \text{ challenges})$ 
proof (rule wp-event-finite-union-bound-fri-exact)
show finite B
  using challenge-subset by (rule finite-subset)
  (rule finite-fri-challenge-space)
next
fix challenges
assume challenges-in:  $\text{challenges} \in B$ 
have challenges-space:
   $\text{challenges} \in \text{fri-challenge-space } (\text{ceil-log clength})$ 
  using challenge-subset challenges-in by blast
show wp-event (checked-staged-security-experiment-with-data-state A)
  ( $?E \text{ challenges}$ ) adversary-initial-state  $\leq ?C \text{ challenges}$ 
  by (rule
    checked-staged-security-trace-fri-pair-challenge-query-path-fresh-fixed-bound
    [OF wf controlled challenges-space
      projection[OF challenges-in] subset[OF challenges-in]])
qed
finally show ?thesis .
qed

lemma checked-staged-security-trace-fri-query-challenge-pair-set-hit-challenge-weighted-bound:
assumes wf: staged-budget-wellformed budgets
and controlled: staged-adversary-controlled budgets A
and challenge-subset:  $B \subseteq \text{fri-challenge-space } (\text{ceil-log clength})$ 
and challenge-projection: snd ' P  $\subseteq B$ 
and query-projection:
   $\bigwedge \text{challenges.}$ 
   $\text{challenges} \in B \implies$ 
  fst ' (P  $\cap (\text{UNIV} \times \{\text{challenges}\})) \subseteq Q \text{ challenges}$ 
and subset:
   $\bigwedge \text{challenges.}$   $\text{challenges} \in B \implies$ 
   $Q \text{ challenges} \subseteq \text{fri-query-index-list-space}$ 
shows
wp-event (checked-staged-security-experiment-with-data-state A)
  (staged-security-with-data-state-verifier-event
    ( $\lambda s. \text{trace-fri-query-challenge-pair-set-hit } s \text{ P}$ ))

```

```

adversary-initial-state ≤
  (∑ challenges ∈ B.
    (1 / nnreal (CARD('f) ^ ceil-log clength)) *
    (nnreal (card (Q challenges)) *
    (1 / nnreal (card query-sample-space)) ^ rounds)) +
  hash-relation-budget-value
    (query-index-row-list-relation-fiber-bound
      (∪ challenges ∈ B. Q challenges))
    (staged-attacker-query-budget budgets +
    staged-challenge-query-budget) +
  hash-relation-budget-value (card B * ceil-log clength)
    (staged-attacker-query-budget budgets +
    staged-challenge-query-budget)
proof -
  let ?Q = (∪ challenges ∈ B. Q challenges)
  let ?Path =
    (∑ challenges ∈ B.
      (1 / nnreal (CARD('f) ^ ceil-log clength)) *
      (nnreal (card (Q challenges)) *
      (1 / nnreal (card query-sample-space)) ^ rounds))
  let ?QueryPre =
    hash-relation-budget-value
      (query-index-row-list-relation-fiber-bound ?Q)
      (staged-attacker-query-budget budgets +
      staged-challenge-query-budget)
  let ?ChallengePre =
    hash-relation-budget-value (card B * ceil-log clength)
      (staged-attacker-query-budget budgets +
      staged-challenge-query-budget)
  have finite-B: finite B
    using challenge-subset by (rule finite-subset)
    (rule finite-fri-challenge-space)
  have fst-projection: fst ' P ⊆ ?Q
proof
  fix q
  assume q-in: q ∈ fst ' P
  then obtain c where pair: (q, c) ∈ P
    by auto
  have c-in: c ∈ B
    using challenge-projection pair by force
  have q ∈ fst ' (P ∩ (UNIV × {c}))
    using pair by force
  then have q ∈ Q c
    using query-projection[OF c-in] by blast
  then show q ∈ ?Q
    using c-in by blast
qed
  have path-bound:
    wp-event (checked-staged-security-experiment-with-data-state A)

```

$(\text{staged-security-trace-fri-pair-challenge-query-path-fresh } P \ B \ ?Q)$
 $\text{adversary-initial-state} \leq ?Path$
by (rule
 $\text{checked-staged-security-trace-fri-pair-challenge-query-path-fresh-fiber-sum}$
 $[OF \ wf \ controlled \ challenge\text{-}subset \ query\text{-}projection \ subset])$
have *fresh-bound*:
 $\text{wp-event } (\text{checked-staged-security-experiment-with-data-state } A)$
 $(\text{staged-security-trace-fri-pair-challenge-fresh } P \ B)$
 $\text{adversary-initial-state} \leq ?Path + ?QueryPre$
by (rule
 $\text{checked-staged-security-trace-fri-pair-challenge-fresh-bound-from-query-path-fresh}$
 $[OF \ wf \ controlled \ fst\text{-}projection \ path\text{-}bound])$
have *pair-bound*:
 $\text{wp-event } (\text{checked-staged-security-experiment-with-data-state } A)$
 $(\text{staged-security-with-data-state-verifier-event}$
 $(\lambda s. \text{trace-fri-query-challenge-pair-set-hit } s \ P))$
 $\text{adversary-initial-state} \leq$
 $(?Path + ?QueryPre) + ?ChallengePre$
by (rule
 $\text{checked-staged-security-trace-fri-query-challenge-pair-set-hit-bound-from-challenge-fresh}$
 $[OF \ wf \ controlled \ finite\text{-}B \ challenge\text{-}projection \ fresh\text{-}bound])$
then show *?thesis*
by (*simp add: add.assoc*)
qed

lemma *checked-staged-security-composition-fri-query-challenge-pair-set-hit-challenge-weighted-bound*:
assumes *wf*: *staged-budget-wellformed budgets*
and *controlled*: *staged-adversary-controlled budgets A*
and *challenge-subset*:
 $\bigwedge dg. B \ dg \subseteq \text{fri-challenge-space } (\text{ceil-log } (to\text{-}nat \ dg + 1))$
and *challenge-projection*: $\bigwedge dg. \text{snd } 'P \ dg \subseteq B \ dg$
and *query-projection*:
 $\bigwedge dg \text{ challenges.}$
 $\text{challenges} \in B \ dg \implies$
 $\text{fst } '(P \ dg \cap (UNIV \times \{\text{challenges}\})) \subseteq$
 $Q \ dg \text{ challenges}$
and *subset*:
 $\bigwedge dg \text{ challenges. } \text{challenges} \in B \ dg \implies$
 $Q \ dg \text{ challenges} \subseteq \text{fri-query-index-list-space}$
shows
 $\text{wp-event } (\text{checked-staged-security-experiment-with-data-state } A)$
 $(\text{staged-security-with-data-state-verifier-event}$
 $(\lambda s. \text{composition-fri-query-challenge-pair-set-hit } s \ P))$
 $\text{adversary-initial-state} \leq$
 $(\sum dg \in UNIV.$
 $\sum \text{challenges} \in B \ dg.$
 $(1 / \text{nnreal } (CARD('f) \wedge \text{ceil-log } (to\text{-}nat \ dg + 1))) *$
 $(\text{nnreal } (\text{card } (Q \ dg \text{ challenges}))) *$
 $(1 / \text{nnreal } (\text{card } \text{query-sample-space})) \wedge \text{rounds})) +$

```

hash-relation-budget-value
  (query-index-raw-list-relation-fiber-bound
    ( $\bigcup dg \in UNIV. \bigcup challenges \in B dg. Q dg challenges$ ))
    ( $staged-attacker-query-budget budgets + staged-challenge-query-budget$ ) +
hash-relation-budget-value
  ( $\sum dg \in (UNIV :: 'f set).$ 
     $card (B dg) * ceil-log (maxDegree + 1)$ )
    ( $staged-attacker-query-budget budgets + staged-challenge-query-budget$ )
proof -
let ?Q =
  ( $\bigcup dg \in UNIV. \bigcup challenges \in B dg. Q dg challenges$ )
let ?Path =
  ( $\sum dg \in UNIV. \sum challenges \in B dg.$ 
    ( $1 / nnreal (CARD('f) ^ ceil-log (to-nat dg + 1))$ ) *
    ( $nnreal (card (Q dg challenges))$ ) *
    ( $1 / nnreal (card query-sample-space) ^ rounds$ ))
let ?QueryPre =
  hash-relation-budget-value
    (query-index-raw-list-relation-fiber-bound ?Q)
    ( $staged-attacker-query-budget budgets + staged-challenge-query-budget$ )
let ?ChallengePre =
  hash-relation-budget-value
    ( $\sum dg \in (UNIV :: 'f set).$ 
       $card (B dg) * ceil-log (maxDegree + 1)$ )
    ( $staged-attacker-query-budget budgets + staged-challenge-query-budget$ )
have finite-B:  $\bigwedge dg. finite (B dg)$ 
  by (rule finite-subset[OF challenge-subset])
  (rule finite-fri-challenge-space)
have fst-projection:  $\bigwedge dg. fst ' P dg \subseteq ?Q$ 
proof
  fix dg q
  assume q-in:  $q \in fst ' P dg$ 
  then obtain c where pair:  $(q, c) \in P dg$ 
  by auto
  have c-in:  $c \in B dg$ 
  using challenge-projection[of dg] pair by force
  have q  $\in fst ' (P dg \cap (UNIV \times \{c\}))$ 
  using pair by force
  then have q  $\in Q dg c$ 
  using query-projection[OF c-in] by blast
  then show q  $\in ?Q$ 
  using c-in by blast
qed

```

have *path-bound*:
wp-event (*checked-staged-security-experiment-with-data-state* *A*)
(*staged-security-composition-fri-pair-challenge-query-path-fresh* *P*
B ?Q)
adversary-initial-state $\leq$ *?Path*
by (*rule*
checked-staged-security-composition-fri-pair-challenge-query-path-fresh-fiber-sum
[*OF wf controlled challenge-subset query-projection subset*])
have *fresh-bound*:
wp-event (*checked-staged-security-experiment-with-data-state* *A*)
(*staged-security-composition-fri-pair-challenge-fresh* *P B*)
adversary-initial-state $\leq$ *?Path* + *?QueryPre*
by (*rule*
checked-staged-security-composition-fri-pair-challenge-fresh-bound-from-query-path-fresh
[*OF wf controlled fst-projection path-bound*])
have *pair-bound*:
wp-event (*checked-staged-security-experiment-with-data-state* *A*)
(*staged-security-with-data-state-verifier-event*
($\lambda s.$ *composition-fri-query-challenge-pair-set-hit* *s P*))
adversary-initial-state $\leq$
(*?Path* + *?QueryPre*) + *?ChallengePre*
by (*rule*
checked-staged-security-composition-fri-query-challenge-pair-set-hit-bound-from-challenge-fresh
[*OF wf controlled finite-B challenge-projection fresh-bound*])
then show *?thesis*
by (*simp add: add.assoc*)
qed

lemma *checked-staged-security-trace-fri-sampled-query-bad-candidate-bound-from-pair-fraction-challenge-weighted*:

assumes *wf*: *staged-budget-wellformed* *budgets*
and *controlled*: *staged-adversary-controlled* *budgets* *A*
and *fraction*:
generic-fri-sampled-query-pair-fraction-bound *trace-table-low-degree*
(*Not* $\circ$ *trace-table-low-degree*) (*clength* - 1) *C*

shows

wp-event (*checked-staged-security-experiment-with-data-state* *A*)
(*staged-security-with-data-state-verifier-event*
trace-fri-sampled-query-bad-candidate)
adversary-initial-state $\leq$
C +
hash-relation-budget-value
(*query-index-raw-list-relation-fiber-bound*
($\bigcup$ *challenges* $\in$ *fri-challenge-space* (*ceil-log clength*)).
generic-fri-sampled-query-query-fiber *trace-table-low-degree*
(*Not* $\circ$ *trace-table-low-degree*) (*clength* - 1) *challenges*))
(*staged-attacker-query-budget* *budgets* +
staged-challenge-query-budget) +
hash-relation-budget-value
(*card* (*fri-challenge-space* (*ceil-log clength*))) * *ceil-log clength*)


```

      (staged-attacker-query-budget budgets +
       staged-challenge-query-budget)
proof -
  let ?P =
    trace-fri-sampled-query-bad-pair-union  $\cap$ 
    (fri-query-index-list-space  $\times$  UNIV)
  let ?B = fri-challenge-space (ceil-log clength)
  let ?Q =
     $\lambda$ challenges.
      generic-fri-sampled-query-query-fiber trace-table-low-degree
      (Not  $\circ$  trace-table-low-degree) (clength - 1) challenges
  let ?Path =
    ( $\sum$  challenges  $\in$  ?B.
     (1 / nnreal (CARD('f)  $\wedge$  ceil-log clength)) *
     (nnreal (card (?Q challenges)) *
      (1 / nnreal (card query-sample-space))  $\wedge$  rounds))
  let ?QueryPre =
    hash-relation-budget-value
    (query-index-raw-list-relation-fiber-bound
     ( $\bigcup$  challenges  $\in$  ?B. ?Q challenges))
    (staged-attacker-query-budget budgets +
     staged-challenge-query-budget)
  let ?ChallengePre =
    hash-relation-budget-value
    (card ?B * ceil-log clength)
    (staged-attacker-query-budget budgets +
     staged-challenge-query-budget)
  have sampled-to-pair:
    wp-event (checked-staged-security-experiment-with-data-state A)
    (staged-security-with-data-state-verifier-event
     trace-fri-sampled-query-bad-candidate)
    adversary-initial-state  $\leq$ 
    wp-event (checked-staged-security-experiment-with-data-state A)
    (staged-security-with-data-state-verifier-event
     ( $\lambda s$ . trace-fri-query-challenge-pair-set-hit s ?P))
    adversary-initial-state
  by (rule wp-event-mono-on-support)
    (auto simp: staged-security-with-data-state-verifier-event-def
     intro:
      trace-fri-sampled-query-bad-candidate-imp-restricted-query-challenge-pair-union-hit
      split: option.splits prod.splits)
  also have ...  $\leq$  ?Path + ?QueryPre + ?ChallengePre
proof -
  have challenge-subset:
    ?B  $\subseteq$  fri-challenge-space (ceil-log clength)
  by simp
  have challenge-projection: snd ' ?P  $\subseteq$  ?B
  by (rule trace-fri-restricted-sampled-query-bad-pair-union-challenge-projection)
  have query-projection:

```

```

 $\bigwedge \text{challenges. challenges} \in ?B \implies$ 
   $\text{fst } ' ( (?P \cap (UNIV \times \{\text{challenges}\})) ) \subseteq ?Q \text{ challenges}$ 
by (auto
  intro: generic-fri-sampled-query-query-fiber-subset
  simp: generic-fri-sampled-query-query-fiber-def
        generic-fri-sampled-query-restricted-bad-pairs-def
        fri-query-challenge-pair-query-fiber-def
        trace-fri-sampled-query-bad-pair-union-def)
have subset-Q:
   $\bigwedge \text{challenges. challenges} \in ?B \implies$ 
     $?Q \text{ challenges} \subseteq \text{fri-query-index-list-space}$ 
  by (rule generic-fri-sampled-query-query-fiber-subset)
show ?thesis
  by (rule
    checked-staged-security-trace-fri-query-challenge-pair-set-hit-challenge-weighted-bound
    [OF wf controlled challenge-subset challenge-projection
      query-projection subset-Q])
qed
also have ...  $\leq C + ?QueryPre + ?ChallengePre$ 
proof -
  have round-eq:
    fri-round-count-for-degree-bound (clength - Suc 0) = ceil-log clength
  unfolding fri-round-count-for-degree-bound-def
  by (simp add: clength-pos)
  have fiber-fraction:
    nnreal
      (  $\sum \text{challenges} \in ?B. \text{card } (?Q \text{ challenges})$  ) /
      (nnreal (CARD('f) ^ ceil-log clength) *
        nnreal query-sample-space-size ^ rounds)
     $\leq C$ 
  using fraction
  unfolding trace-fri-sampled-query-pair-fraction-bound-iff-fiber-sum
  by (simp add: round-eq card-fri-query-index-list-space
    query-sample-space-size-pos algebra-simps)
  have prob-eq:
    ?Path =
      nnreal
        (  $\sum \text{challenges} \in ?B. \text{card } (?Q \text{ challenges})$  ) /
        (nnreal (CARD('f) ^ ceil-log clength) *
          nnreal query-sample-space-size ^ rounds)
  using weighted-query-fiber-sum-eq
  [where I = ?B
    and f =  $\lambda \text{challenges. card } (?Q \text{ challenges})$ 
    and A =  $\text{CARD('f) ^ ceil-log clength}$ 
    and B = query-sample-space-size]
  by (simp add: finite-fri-challenge-space query-sample-space-size-pos
    card-query-sample-space algebra-simps)
  have ?Path  $\leq C$ 
  using fiber-fraction unfolding prob-eq .

```

```

    then show ?thesis
    by (simp add: add-mono)
qed
finally show ?thesis .
qed

```

lemma *checked-staged-security-composition-fri-sampled-query-bad-candidate-bound-from-pair-fraction-challenge-weig*

fixes $C :: 'f \Rightarrow \text{prob}$

assumes wf : *staged-budget-wellformed budgets*

and $controlled$: *staged-adversary-controlled budgets A*

and $fraction$:

$\wedge dg.$ *generic-fri-sampled-query-pair-fraction-bound*
(composition-table-low-degree (to-nat dg))
(Not $\circ$ composition-table-low-degree maxDegree) (to-nat dg) (C dg)

shows

$wp\text{-}event$ (*checked-staged-security-experiment-with-data-state A*)
(staged-security-with-data-state-verifier-event
composition-fri-sampled-query-bad-candidate)
adversary-initial-state $\leq$
($\sum dg \in UNIV. C dg$) +
hash-relation-budget-value
(query-index-raw-list-relation-fiber-bound
($\bigcup dg \in UNIV.$
 $\bigcup challenges \in fri\text{-}challenge\text{-}space$ ($ceil\text{-}log$ ($to\text{-}nat dg + 1$)).
generic-fri-sampled-query-query-fiber
(composition-table-low-degree (to-nat dg))
(Not $\circ$ composition-table-low-degree maxDegree)
(to-nat dg) challenges))
(staged-attacker-query-budget budgets +
staged-challenge-query-budget) +
hash-relation-budget-value
($\sum dg \in (UNIV :: 'f \text{ set}).$
*card ($fri\text{-}challenge\text{-}space$ ($ceil\text{-}log$ ($to\text{-}nat dg + 1$))) **
ceil-log (maxDegree + 1))
(staged-attacker-query-budget budgets +
staged-challenge-query-budget)

proof –

let $?P =$

$\lambda dg.$ *composition-fri-sampled-query-bad-pair-union dg $\cap$*
(fri-query-index-list-space $\times UNIV$)

let $?B =$

$\lambda dg.$ *fri-challenge-space (ceil-log (to-nat dg + 1))*

let $?Q =$

λdg *challenges.*
generic-fri-sampled-query-query-fiber
(composition-table-low-degree (to-nat dg))
(Not $\circ$ composition-table-low-degree maxDegree)
(to-nat dg) challenges

let $?Path =$

```

( $\sum dg \in UNIV.$ 
 $\sum challenges \in ?B dg.$ 
 $(1 / \text{nnreal} (CARD('f) \wedge \text{ceil-log} (to\text{-nat} dg + 1))) *$ 
 $(\text{nnreal} (\text{card} (?Q dg challenges)) *$ 
 $(1 / \text{nnreal} (\text{card query-sample-space})) \wedge \text{rounds}))$ 
let ?QueryPre =
  hash-relation-budget-value
  (query-index-raw-list-relation-fiber-bound
   ( $\bigcup dg \in UNIV. \bigcup challenges \in ?B dg. ?Q dg challenges$ ))
  (staged-attacker-query-budget budgets +
   staged-challenge-query-budget)
let ?ChallengePre =
  hash-relation-budget-value
  ( $\sum dg \in (UNIV :: 'f \text{ set}). \text{card} (?B dg) * \text{ceil-log} (\text{maxDegree} + 1)$ )
  (staged-attacker-query-budget budgets +
   staged-challenge-query-budget)
have sampled-to-pair:
  wp-event (checked-staged-security-experiment-with-data-state A)
  (staged-security-with-data-state-verifier-event
   composition-fri-sampled-query-bad-candidate)
  adversary-initial-state  $\leq$ 
  wp-event (checked-staged-security-experiment-with-data-state A)
  (staged-security-with-data-state-verifier-event
   ( $\lambda s. \text{composition-fri-query-challenge-pair-set-hit } s ?P$ ))
  adversary-initial-state
by (rule wp-event-mono-on-support)
  (auto simp: staged-security-with-data-state-verifier-event-def
   intro:
    composition-fri-sampled-query-bad-candidate-imp-restricted-query-challenge-pair-union-hit
    split: option.splits prod.splits)
also have ...  $\leq ?Path + ?QueryPre + ?ChallengePre$ 
proof -
have challenge-subset:
   $\bigwedge dg. ?B dg \subseteq \text{fri-challenge-space} (\text{ceil-log} (to\text{-nat} dg + 1))$ 
by simp
have challenge-projection:  $\bigwedge dg. \text{snd} ' ?P dg \subseteq ?B dg$ 
by (rule
  composition-fri-restricted-sampled-query-bad-pair-union-challenge-projection)
have query-projection:
   $\bigwedge dg \text{ challenges. challenges} \in ?B dg \implies$ 
  fst ' ( $?P dg \cap (UNIV \times \{\text{challenges}\})$ )  $\subseteq$ 
   $?Q dg \text{ challenges}$ 
by (auto
  intro: generic-fri-sampled-query-query-fiber-subset
  simp: generic-fri-sampled-query-query-fiber-def
  generic-fri-sampled-query-restricted-bad-pairs-def
  fri-query-challenge-pair-query-fiber-def
  composition-fri-sampled-query-bad-pair-union-def)
have subset-Q:

```

```

 $\bigwedge dg \text{ challenges. challenges} \in ?B \text{ dg} \implies$ 
   $?Q \text{ dg challenges} \subseteq \text{fri-query-index-list-space}$ 
by (rule generic-fri-sampled-query-query-fiber-subset)
show ?thesis
by (rule
  checked-staged-security-composition-fri-query-challenge-pair-set-hit-challenge-weighted-bound
  [OF wf controlled challenge-subset challenge-projection
  query-projection subset-Q])
qed
also have ...  $\leq (\sum dg \in UNIV. C \text{ dg}) + ?QueryPre + ?ChallengePre$ 
proof –
  have path-le:
     $?Path \leq (\sum dg \in UNIV. C \text{ dg})$ 
proof (rule sum-mono)
  fix dg :: 'f
  assume dg  $\in UNIV$ 
  let ?I = ?B dg
  have round-eq:
    fri-round-count-for-degree-bound (to-nat dg) =
      ceil-log (to-nat dg + 1)
  unfolding fri-round-count-for-degree-bound-def by simp
have fiber-fraction:
  nnreal
    ( $\sum \text{challenges} \in ?I. \text{card } (?Q \text{ dg challenges})$ ) /
    (nnreal (CARD('f) ^ ceil-log (to-nat dg + 1)) *
      nnreal query-sample-space-size ^ rounds)
     $\leq C \text{ dg}$ 
  using fraction[of dg]
  unfolding composition-fri-sampled-query-pair-fraction-bound-iff-fiber-sum
  by (simp add: round-eq card-fri-query-index-list-space
    query-sample-space-size-pos algebra-simps)
have prob-eq:
  ( $\sum \text{challenges} \in ?I. (1 / \text{nnreal } (\text{CARD}('f) ^ \text{ceil-log } (\text{to-nat dg} + 1))) * (1 / \text{nnreal } (\text{card } (?Q \text{ dg challenges}))) * (1 / \text{nnreal } (\text{card query-sample-space})) ^ \text{rounds})$ ) =
  nnreal
    ( $\sum \text{challenges} \in ?I. \text{card } (?Q \text{ dg challenges})$ ) /
    (nnreal (CARD('f) ^ ceil-log (to-nat dg + 1)) *
      nnreal query-sample-space-size ^ rounds)
  using weighted-query-fiber-sum-eq
  [where I = ?I
    and f =  $\lambda \text{challenges. card } (?Q \text{ dg challenges})$ 
    and A =  $\text{CARD}('f) ^ \text{ceil-log } (\text{to-nat dg} + 1)$ 
    and B = query-sample-space-size]
  by (simp add: finite-fri-challenge-space query-sample-space-size-pos
    card-query-sample-space algebra-simps)
show
  ( $\sum \text{challenges} \in ?I.$ 

```

```

      (1 / nnreal (CARD('f) ^ ceil-log (to-nat dg + 1))) *
      (nnreal (card (?Q dg challenges)) *
      (1 / nnreal (card query-sample-space)) ^ rounds))
    ≤ C dg
  using fiber-fraction unfolding prob-eq .
qed
then show ?thesis
  by (simp add: add-mono)
qed
finally show ?thesis .
qed

end

end

```

```

theory Soundness-FRI-Sampled-Challenge-Cover
imports
  Soundness-FRI-Query-Challenge-Trace-Staged-Product
  Soundness-FRI-Internal-Bounds
begin

```

Generic sampled-query FRI challenge-cover reductions.

This layer is deliberately independent of trace/composition naming. The intended later FRI low-degree proof has to show that every sampled bad candidate produces a challenge list in an algebraic multiround bad set. The lemmas here turn exactly that fact, plus local round cardinality bounds, into the finite-cover/fraction interface consumed by the staged route.

```

lemma nnreal-nat-mult-divide-cancel-left:
  fixes a b c :: nat
  assumes 0 < b
  shows nnreal (a * b) / nnreal (b * c) = nnreal a / nnreal c
proof -
  have b-ne: (real b) ≠ 0
    using assms by simp
  have nn2real (nnreal (a * b) / nnreal (b * c)) =
    real (a * b) / real (b * c)
    by (simp only: nn2real-divide nn2real-nnreal)
  also have ... = (real a * real b) / (real b * real c)
    by (simp add: of-nat-mult)
  also have ... = real a / real c
    using b-ne by (simp add: field-simps)
  also have ... = nn2real (nnreal a / nnreal c)
    by (simp only: nn2real-divide nn2real-nnreal)
  finally have
    nn2real (nnreal (a * b) / nnreal (b * c)) =
      nn2real (nnreal a / nnreal c) .

```

then show *?thesis*
by (*simp only: nn2real-eq-iff*)
qed

context *soundness*
begin

definition *trace-fri-sampled-query-no-full-cover-failure-candidate*
 $:: ('f, 'a) \text{ protocol-channel-scheme} \Rightarrow$
 $('f \text{ list} \Rightarrow \text{nat} \Rightarrow 'f \text{ list} \Rightarrow 'f \text{ set}) \Rightarrow$
 $(\text{unit list} \times ('f, 'a) \text{ protocol-channel-scheme}) \text{ option} \Rightarrow \text{bool}$

where

$\text{trace-fri-sampled-query-no-full-cover-failure-candidate } s \text{ bad out}$
 $\longleftrightarrow$
 $(\exists \text{ trace-roots trace-bs trace-final dg composition-roots}$
 $\text{ composition-bs composition-final fri-query-idxs trace-round-layers}$
 $\text{ composition-round-layers fr candidate-query-idxs trace-openings}$
 $\text{ trace-table layers.}$
 $\text{trace-fri-partial-candidate-opening-evidence } s \text{ out trace-roots}$
 $\text{ trace-bs trace-final dg composition-roots composition-bs}$
 $\text{ composition-final fri-query-idxs trace-round-layers}$
 $\text{ composition-round-layers fr candidate-query-idxs trace-openings}$
 $\text{ trace-table} \wedge$
 $\text{generic-fri-sampled-query-candidate-evidence trace-table-low-degree}$
 $(\text{Not} \circ \text{trace-table-low-degree}) \text{ trace-table (clength - 1)}$
 $\text{ trace-roots trace-bs trace-final fri-query-idxs trace-round-layers}$
 $\text{ layers} \wedge$
 $\neg \text{generic-fri-sampled-full-cover-failure trace-table-low-degree}$
 $\text{ trace-table (clength - 1) trace-roots trace-bs trace-final}$
 $\text{ fri-query-idxs trace-round-layers}$
 $(\text{fri-canonical-domains (length trace-bs)}) \text{ layers}$
 $(\text{bad trace-table}))$

definition *composition-fri-sampled-query-no-full-cover-failure-candidate*
 $:: ('f, 'a) \text{ protocol-channel-scheme} \Rightarrow$
 $('f \Rightarrow 'f \text{ list} \Rightarrow \text{nat} \Rightarrow 'f \text{ list} \Rightarrow 'f \text{ set}) \Rightarrow$
 $(\text{unit list} \times ('f, 'a) \text{ protocol-channel-scheme}) \text{ option} \Rightarrow \text{bool}$

where

$\text{composition-fri-sampled-query-no-full-cover-failure-candidate } s \text{ bad out}$
 $\longleftrightarrow$
 $(\exists \text{ trace-roots trace-bs trace-final fri-dg}$
 $\text{ opening-composition-roots composition-bs composition-final}$
 $\text{ fri-query-idxs trace-round-layers composition-round-layers fr}$
 $\text{ f-fri-roots f-final as dg composition-fri-roots final}$
 $\text{ trace-query-idxs trace-openings composition-query-idxs}$
 $\text{ composition-openings trace-table composition-table layers.}$
 $\text{composition-fri-partial-candidate-opening-evidence } s \text{ out trace-roots}$
 $\text{ trace-bs trace-final fri-dg opening-composition-roots composition-bs}$
 $\text{ composition-final fri-query-idxs trace-round-layers}$

composition-round-layers *fr* *f-fri-roots* *f-final* *as dg*
composition-fri-roots *final* *trace-query-idxs* *trace-openings*
composition-query-idxs *composition-openings* *trace-table*
composition-table $\wedge$
generic-fri-sampled-query-candidate-evidence
(*composition-table-low-degree* (*to-nat* *fri-dg*))
(*Not* $\circ$ *composition-table-low-degree* *maxDegree*) *composition-table*
(*to-nat* *fri-dg*) *opening-composition-roots* *composition-bs*
composition-final *fri-query-idxs* *composition-round-layers* *layers* $\wedge$
 $\neg$ *generic-fri-sampled-full-cover-failure*
(*composition-table-low-degree* (*to-nat* *fri-dg*)) *composition-table*
(*to-nat* *fri-dg*) *opening-composition-roots* *composition-bs*
composition-final *fri-query-idxs* *composition-round-layers*
(*fri-canonical-domains* (*length* *composition-bs*)) *layers*
(*bad* *fri-dg* *composition-table*)

definition *trace-fri-sampled-query-no-full-cover-failure-pairs*

$:: ('f \text{ list} \Rightarrow \text{nat} \Rightarrow 'f \text{ list} \Rightarrow 'f \text{ set}) \Rightarrow$
 $(\text{nat list} \times 'f \text{ list}) \text{ set}$

where

trace-fri-sampled-query-no-full-cover-failure-pairs *bad* =
 $\{(\text{query-idxs}, \text{challenges}).$
 $\exists$ *candidate-table* *roots* *final-value* *round-layers* *layers*.
generic-fri-sampled-query-candidate-evidence *trace-table-low-degree*
(*Not* $\circ$ *trace-table-low-degree*) *candidate-table* (*clength* $- 1$)
roots *challenges* *final-value* *query-idxs* *round-layers* *layers* $\wedge$
 $\neg$ *generic-fri-sampled-full-cover-failure* *trace-table-low-degree*
candidate-table (*clength* $- 1$) *roots* *challenges* *final-value*
query-idxs *round-layers* (*fri-canonical-domains* (*length* *challenges*))
layers (*bad* *candidate-table*) $\}$
 $\cap (\text{fri-query-index-list-space} \times \text{UNIV})$

definition *trace-fri-sampled-query-no-full-cover-failure-query-fiber*

$:: ('f \text{ list} \Rightarrow \text{nat} \Rightarrow 'f \text{ list} \Rightarrow 'f \text{ set}) \Rightarrow$
 $'f \text{ list} \Rightarrow \text{nat list set}$

where

trace-fri-sampled-query-no-full-cover-failure-query-fiber *bad*
challenges =
fri-query-challenge-pair-query-fiber
(*trace-fri-sampled-query-no-full-cover-failure-pairs* *bad*)
challenges

definition *composition-fri-sampled-query-no-full-cover-failure-pairs*

$:: ('f \Rightarrow 'f \text{ list} \Rightarrow \text{nat} \Rightarrow 'f \text{ list} \Rightarrow 'f \text{ set}) \Rightarrow$
 $'f \Rightarrow (\text{nat list} \times 'f \text{ list}) \text{ set}$

where

composition-fri-sampled-query-no-full-cover-failure-pairs *bad* *dg* =
 $\{(\text{query-idxs}, \text{challenges}).$
 $\exists$ *candidate-table* *roots* *final-value* *round-layers* *layers*.

generic-fri-sampled-query-candidate-evidence
 (composition-table-low-degree (to-nat dg))
 (Not $\circ$ composition-table-low-degree maxDegree)
 candidate-table (to-nat dg) roots challenges final-value
 query-idxs round-layers layers $\wedge$
 $\neg$ generic-fri-sampled-full-cover-failure
 (composition-table-low-degree (to-nat dg)) candidate-table
 (to-nat dg) roots challenges final-value query-idxs round-layers
 (fri-canonical-domains (length challenges)) layers
 (bad dg candidate-table)
 $\cap$ (fri-query-index-list-space $\times$ UNIV)

definition composition-fri-sampled-query-no-full-cover-failure-query-fiber
 $:: ('f \Rightarrow 'f \text{ list} \Rightarrow \text{nat} \Rightarrow 'f \text{ list} \Rightarrow 'f \text{ set}) \Rightarrow$
 $'f \Rightarrow 'f \text{ list} \Rightarrow \text{nat list set}$

where

composition-fri-sampled-query-no-full-cover-failure-query-fiber bad dg
 challenges =
 fri-query-challenge-pair-query-fiber
 (composition-fri-sampled-query-no-full-cover-failure-pairs bad dg)
 challenges

lemma trace-fri-sampled-query-no-full-cover-failure-pairs-subset:
 trace-fri-sampled-query-no-full-cover-failure-pairs bad
 $\subseteq$ trace-fri-sampled-query-bad-pair-union $\cap$
 (fri-query-index-list-space $\times$ UNIV)

proof

fix pair

assume pair-in:

pair $\in$ trace-fri-sampled-query-no-full-cover-failure-pairs bad

obtain query-idxs challenges **where** pair-eq: pair = (query-idxs, challenges)

by (cases pair)

from pair-in pair-eq **obtain** candidate-table roots final-value round-layers

layers **where** query-space: query-idxs $\in$ fri-query-index-list-space

and evidence:

generic-fri-sampled-query-candidate-evidence trace-table-low-degree
 (Not $\circ$ trace-table-low-degree) candidate-table (clength - 1)
 roots challenges final-value query-idxs round-layers layers

unfolding trace-fri-sampled-query-no-full-cover-failure-pairs-def

by blast

have pair-set:

(query-idxs, challenges) $\in$

generic-fri-sampled-query-bad-pair-set trace-table-low-degree
 (Not $\circ$ trace-table-low-degree) candidate-table (clength - 1)
 roots final-value round-layers layers

by (rule generic-fri-sampled-query-bad-pair-setI[OF evidence])

then have union:

(query-idxs, challenges) $\in$ trace-fri-sampled-query-bad-pair-union

unfolding trace-fri-sampled-query-bad-pair-union-def

by (rule generic-fri-sampled-query-bad-pair-unionI)
 show
 pair $\in$ trace-fri-sampled-query-bad-pair-union $\cap$
 (fri-query-index-list-space $\times$ UNIV)
 using pair-eq query-space union by simp
 qed

lemma trace-fri-sampled-query-no-full-cover-failure-pairs-challenge-projection:
 snd ‘ trace-fri-sampled-query-no-full-cover-failure-pairs bad
 $\subseteq$ fri-challenge-space (ceil-log clength)
 using trace-fri-sampled-query-no-full-cover-failure-pairs-subset
 trace-fri-restricted-sampled-query-bad-pair-union-challenge-projection
 by blast

lemma trace-fri-sampled-query-no-full-cover-failure-query-fiber-subset:
 trace-fri-sampled-query-no-full-cover-failure-query-fiber bad challenges
 $\subseteq$ fri-query-index-list-space
 unfolding trace-fri-sampled-query-no-full-cover-failure-query-fiber-def
 using trace-fri-sampled-query-no-full-cover-failure-pairs-subset
 by (auto simp: fri-query-challenge-pair-query-fiber-def)

lemma composition-fri-sampled-query-no-full-cover-failure-pairs-subset:
 composition-fri-sampled-query-no-full-cover-failure-pairs bad dg
 $\subseteq$ composition-fri-sampled-query-bad-pair-union dg $\cap$
 (fri-query-index-list-space $\times$ UNIV)
proof
 fix pair
 assume pair-in:
 pair $\in$ composition-fri-sampled-query-no-full-cover-failure-pairs
 bad dg
obtain query-idxs challenges **where** pair-eq: pair = (query-idxs, challenges)
by (cases pair)
from pair-in pair-eq **obtain** candidate-table roots final-value round-layers
 layers **where** query-space: query-idxs $\in$ fri-query-index-list-space
and evidence:
 generic-fri-sampled-query-candidate-evidence
 (composition-table-low-degree (to-nat dg))
 (Not $\circ$ composition-table-low-degree maxDegree)
 candidate-table (to-nat dg) roots challenges final-value
 query-idxs round-layers layers
unfolding composition-fri-sampled-query-no-full-cover-failure-pairs-def
by blast
have pair-set:
 (query-idxs, challenges) $\in$
 generic-fri-sampled-query-bad-pair-set
 (composition-table-low-degree (to-nat dg))
 (Not $\circ$ composition-table-low-degree maxDegree)
 candidate-table (to-nat dg) roots final-value round-layers layers
by (rule generic-fri-sampled-query-bad-pair-setI[OF evidence])

then have union:
 (query-idxs, challenges) ∈
 composition-fri-sampled-query-bad-pair-union dg
unfolding composition-fri-sampled-query-bad-pair-union-def
by (rule generic-fri-sampled-query-bad-pair-unionI)
show
 pair ∈ composition-fri-sampled-query-bad-pair-union dg ∩
 (fri-query-index-list-space × UNIV)
using pair-eq query-space union **by** simp
qed

lemma composition-fri-sampled-query-no-full-cover-failure-pairs-challenge-projection:
 snd ‘ composition-fri-sampled-query-no-full-cover-failure-pairs bad dg
 ⊆ fri-challenge-space (ceil-log (to-nat dg + 1))
using composition-fri-sampled-query-no-full-cover-failure-pairs-subset
 composition-fri-restricted-sampled-query-bad-pair-union-challenge-projection
by blast

lemma composition-fri-sampled-query-no-full-cover-failure-query-fiber-subset:
 composition-fri-sampled-query-no-full-cover-failure-query-fiber bad dg
 challenges
 ⊆ fri-query-index-list-space
unfolding composition-fri-sampled-query-no-full-cover-failure-query-fiber-def
using composition-fri-sampled-query-no-full-cover-failure-pairs-subset
by (auto simp: fri-query-challenge-pair-query-fiber-def)

lemma trace-fri-sampled-query-no-full-cover-failure-candidate-imp-pair-hit:
assumes
 trace-fri-sampled-query-no-full-cover-failure-candidate s bad out
shows
 trace-fri-query-challenge-pair-set-hit s
 (trace-fri-sampled-query-no-full-cover-failure-pairs bad) out
proof –
from assms **obtain** trace-roots trace-bs trace-final dg composition-roots
 composition-bs composition-final fri-query-idxs trace-round-layers
 composition-round-layers fr candidate-query-idxs trace-openings
 trace-table layers **where** evidence:
 trace-fri-partial-candidate-opening-evidence s out trace-roots
 trace-bs trace-final dg composition-roots composition-bs
 composition-final fri-query-idxs trace-round-layers
 composition-round-layers fr candidate-query-idxs trace-openings
 trace-table
and candidate:
 generic-fri-sampled-query-candidate-evidence trace-table-low-degree
 (Not ∘ trace-table-low-degree) trace-table (clength – 1)
 trace-roots trace-bs trace-final fri-query-idxs trace-round-layers
 layers
and no-failure:
 ¬ generic-fri-sampled-full-cover-failure trace-table-low-degree

```

    trace-table (clength - 1) trace-roots trace-bs trace-final
    fri-query-idxs trace-round-layers
    (fri-canonical-domains (length trace-bs)) layers
    (bad trace-table)
unfolding trace-fri-sampled-query-no-full-cover-failure-candidate-def
by fastforce
have transcript:
    accepted-fri-opening-transcript s out trace-roots trace-bs trace-final
    dg composition-roots composition-bs composition-final fri-query-idxs
    trace-round-layers composition-round-layers
using evidence
unfolding trace-fri-partial-candidate-opening-evidence-def by blast
have query-space:
    fri-query-idxs ∈ fri-query-index-list-space
by (rule accepted-fri-opening-transcript-query-index-list-space
    [OF transcript])
have pair:
    (fri-query-idxs, trace-bs) ∈
    trace-fri-sampled-query-no-full-cover-failure-pairs bad
unfolding trace-fri-sampled-query-no-full-cover-failure-pairs-def
using candidate no-failure query-space by blast
show ?thesis
unfolding trace-fri-query-challenge-pair-set-hit-def
using transcript pair by blast
qed

lemma composition-fri-sampled-query-no-full-cover-failure-candidate-imp-pair-hit:
assumes
    composition-fri-sampled-query-no-full-cover-failure-candidate s bad out
shows
    composition-fri-query-challenge-pair-set-hit s
    (composition-fri-sampled-query-no-full-cover-failure-pairs bad) out
proof —
from assms obtain trace-roots trace-bs trace-final fri-dg
    opening-composition-roots composition-bs composition-final
    fri-query-idxs trace-round-layers composition-round-layers fr
    f-fri-roots f-final as dg composition-fri-roots final
    trace-query-idxs trace-openings composition-query-idxs
    composition-openings trace-table composition-table layers where
    evidence:
    composition-fri-partial-candidate-opening-evidence s out trace-roots
    trace-bs trace-final fri-dg opening-composition-roots composition-bs
    composition-final fri-query-idxs trace-round-layers
    composition-round-layers fr f-fri-roots f-final as dg
    composition-fri-roots final trace-query-idxs trace-openings
    composition-query-idxs composition-openings trace-table
    composition-table
and candidate:
    generic-fri-sampled-query-candidate-evidence

```

```

    (composition-table-low-degree (to-nat fri-dg))
    (Not ∘ composition-table-low-degree maxDegree) composition-table
    (to-nat fri-dg) opening-composition-roots composition-bs
    composition-final fri-query-idxs composition-round-layers layers
and no-failure:
  ¬ generic-fri-sampled-full-cover-failure
    (composition-table-low-degree (to-nat fri-dg)) composition-table
    (to-nat fri-dg) opening-composition-roots composition-bs
    composition-final fri-query-idxs composition-round-layers
    (fri-canonical-domains (length composition-bs)) layers
    (bad fri-dg composition-table)
unfolding composition-fri-sampled-query-no-full-cover-failure-candidate-def
by fastforce
have transcript:
  accepted-fri-opening-transcript s out trace-roots trace-bs trace-final
  fri-dg opening-composition-roots composition-bs composition-final
  fri-query-idxs trace-round-layers composition-round-layers
using evidence
unfolding composition-fri-partial-candidate-opening-evidence-def by blast
have query-space:
  fri-query-idxs ∈ fri-query-index-list-space
by (rule accepted-fri-opening-transcript-query-index-list-space
    [OF transcript])
have pair:
  (fri-query-idxs, composition-bs) ∈
    composition-fri-sampled-query-no-full-cover-failure-pairs bad fri-dg
unfolding composition-fri-sampled-query-no-full-cover-failure-pairs-def
using candidate no-failure query-space by blast
show ?thesis
unfolding composition-fri-query-challenge-pair-set-hit-def
using transcript pair by blast
qed

lemma trace-fri-sampled-query-no-full-cover-failure-candidate-not-None[simp]:
  ¬ trace-fri-sampled-query-no-full-cover-failure-candidate s bad None
unfolding trace-fri-sampled-query-no-full-cover-failure-candidate-def
  trace-fri-partial-candidate-opening-evidence-def
  accepted-fri-opening-transcript-def
by simp

lemma composition-fri-sampled-query-no-full-cover-failure-candidate-not-None[simp]:
  ¬ composition-fri-sampled-query-no-full-cover-failure-candidate s bad
  None
unfolding composition-fri-sampled-query-no-full-cover-failure-candidate-def
  composition-fri-partial-candidate-opening-evidence-def
  accepted-fri-opening-transcript-def
by simp

lemma trace-fri-sampled-query-no-full-cover-failure-candidate-imp-bad-candidate:

```

assumes
trace-fri-sampled-query-no-full-cover-failure-candidate s bad out
shows *trace-fri-sampled-query-bad-candidate s out*
using *assms*
unfolding *trace-fri-sampled-query-no-full-cover-failure-candidate-def*
trace-fri-sampled-query-bad-candidate-def
by *force*

lemma *composition-fri-sampled-query-no-full-cover-failure-candidate-imp-bad-candidate:*

assumes
composition-fri-sampled-query-no-full-cover-failure-candidate s bad out
shows *composition-fri-sampled-query-bad-candidate s out*
using *assms*
unfolding
composition-fri-sampled-query-no-full-cover-failure-candidate-def
composition-fri-sampled-query-bad-candidate-def
by *force*

lemma *trace-fri-sampled-query-bad-candidate-imp-no-full-cover-or-failure:*

assumes *sampled: trace-fri-sampled-query-bad-candidate s out*
shows
trace-fri-sampled-query-no-full-cover-failure-candidate s bad out $\vee$
trace-fri-sampled-full-cover-failure s bad out

proof —

from *sampled* **obtain** *trace-roots trace-bs trace-final dg composition-roots*
composition-bs composition-final fri-query-idxs trace-round-layers
composition-round-layers fr candidate-query-idxs trace-openings
trace-table layers

where *evidence:*

trace-fri-partial-candidate-opening-evidence s out trace-roots
trace-bs trace-final dg composition-roots composition-bs
composition-final fri-query-idxs trace-round-layers
composition-round-layers fr candidate-query-idxs trace-openings
trace-table

and *candidate:*

generic-fri-sampled-query-candidate-evidence trace-table-low-degree
(Not $\circ$ trace-table-low-degree) trace-table (clength $- 1$)
trace-roots trace-bs trace-final fri-query-idxs trace-round-layers
layers

unfolding *trace-fri-sampled-query-bad-candidate-def* **by** *blast*

have *canonical:*

generic-fri-canonical-sampled-layer-chain-evidence trace-table-low-degree
trace-table (clength $- 1$) trace-roots trace-bs trace-final
fri-query-idxs trace-round-layers layers

by *(rule generic-fri-sampled-query-candidate-evidenceD(1)*
[OF candidate])

have *chain:*

generic-fri-sampled-layer-chain-evidence trace-table-low-degree
trace-table (clength $- 1$) trace-roots trace-bs trace-final

```

    fri-query-idxs trace-round-layers
    (fri-canonical-domains (length trace-bs)) layers
  proof -
    have sampled-chain:
      generic-fri-sampled-layer-chain-evidence trace-table-low-degree
      trace-table (clength - 1) trace-roots trace-bs trace-final
      fri-query-idxs trace-round-layers
      (fri-canonical-domains (length trace-bs)) layers
    by (rule generic-fri-canonical-sampled-layer-chain-evidenceD
        [OF canonical])
    then show ?thesis .
  qed
  show ?thesis
  proof (cases
    generic-fri-sampled-full-cover-failure trace-table-low-degree
    trace-table (clength - 1) trace-roots trace-bs trace-final
    fri-query-idxs trace-round-layers
    (fri-canonical-domains (length trace-bs)) layers
    (bad trace-table))
  case False
  then have
    trace-fri-sampled-query-no-full-cover-failure-candidate s bad out
    unfolding trace-fri-sampled-query-no-full-cover-failure-candidate-def
    using evidence candidate by blast
  then show ?thesis by simp
  next
  case True
  then have trace-fri-sampled-full-cover-failure s bad out
    unfolding trace-fri-sampled-full-cover-failure-def
    using evidence chain by blast
  then show ?thesis by simp
  qed
qed

lemma composition-fri-sampled-query-bad-candidate-imp-no-full-cover-or-failure:
  assumes sampled: composition-fri-sampled-query-bad-candidate s out
  shows
    composition-fri-sampled-query-no-full-cover-failure-candidate s bad out  $\vee$ 
    composition-fri-sampled-full-cover-failure s bad out
  proof -
    from sampled obtain trace-roots trace-bs trace-final fri-dg
      opening-composition-roots composition-bs composition-final
      fri-query-idxs trace-round-layers composition-round-layers fr
      f-fri-roots f-final as dg composition-fri-roots final
      trace-query-idxs trace-openings composition-query-idxs
      composition-openings trace-table composition-table layers
    where evidence:
      composition-fri-partial-candidate-opening-evidence s out trace-roots
      trace-bs trace-final fri-dg opening-composition-roots composition-bs

```

```

    composition-final fri-query-idxs trace-round-layers
    composition-round-layers fr f-fri-roots f-final as dg
    composition-fri-roots final trace-query-idxs trace-openings
    composition-query-idxs composition-openings trace-table
    composition-table
and candidate:
    generic-fri-sampled-query-candidate-evidence
      (composition-table-low-degree (to-nat fri-dg))
      (Not o composition-table-low-degree maxDegree) composition-table
      (to-nat fri-dg) opening-composition-roots composition-bs
      composition-final fri-query-idxs composition-round-layers layers
unfolding composition-fri-sampled-query-bad-candidate-def by meson
have canonical:
    generic-fri-canonical-sampled-layer-chain-evidence
      (composition-table-low-degree (to-nat fri-dg)) composition-table
      (to-nat fri-dg) opening-composition-roots composition-bs
      composition-final fri-query-idxs composition-round-layers layers
by (rule generic-fri-sampled-query-candidate-evidenceD(1)
      [OF candidate])
have chain:
    generic-fri-sampled-layer-chain-evidence
      (composition-table-low-degree (to-nat fri-dg)) composition-table
      (to-nat fri-dg) opening-composition-roots composition-bs
      composition-final fri-query-idxs composition-round-layers
      (fri-canonical-domains (length composition-bs)) layers
proof —
have sampled-chain:
    generic-fri-sampled-layer-chain-evidence
      (composition-table-low-degree (to-nat fri-dg)) composition-table
      (to-nat fri-dg) opening-composition-roots composition-bs
      composition-final fri-query-idxs composition-round-layers
      (fri-canonical-domains (length composition-bs)) layers
by (rule generic-fri-canonical-sampled-layer-chain-evidenceD
      [OF canonical])
then show ?thesis .
qed
show ?thesis
proof (cases
    generic-fri-sampled-full-cover-failure
      (composition-table-low-degree (to-nat fri-dg)) composition-table
      (to-nat fri-dg) opening-composition-roots composition-bs
      composition-final fri-query-idxs composition-round-layers
      (fri-canonical-domains (length composition-bs)) layers
      (bad fri-dg composition-table))
case False
then have
    composition-fri-sampled-query-no-full-cover-failure-candidate s bad out
unfolding
    composition-fri-sampled-query-no-full-cover-failure-candidate-def

```



```

    using evidence candidate by meson
  then show ?thesis by simp
next
case True
then have composition-fri-sampled-full-cover-failure s bad out
  unfolding composition-fri-sampled-full-cover-failure-def
  using evidence chain by meson
then show ?thesis by simp
qed
qed

```

lemma *wp-trace-fri-sampled-query-bad-candidate-bound-from-no-full-cover-or-failure:*

assumes *no-failure-bound:*

wp-event verify-monad

(trace-fri-sampled-query-no-full-cover-failure-candidate s bad) s

$\leq N$

and *failure-bound:*

wp-event verify-monad

(trace-fri-sampled-full-cover-failure s bad) s $\leq F$

shows

wp-event verify-monad (trace-fri-sampled-query-bad-candidate s) s

$\leq N + F$

proof –

have *wp-event verify-monad (trace-fri-sampled-query-bad-candidate s) s*

$\leq$ *wp-event verify-monad*

(λ out.

trace-fri-sampled-query-no-full-cover-failure-candidate s bad out $\vee$

trace-fri-sampled-full-cover-failure s bad out) s

by *(rule wp-event-mono)*

(rule trace-fri-sampled-query-bad-candidate-imp-no-full-cover-or-failure)

also have ... $\leq$

wp-event verify-monad

(trace-fri-sampled-query-no-full-cover-failure-candidate s bad) s +

wp-event verify-monad

(trace-fri-sampled-full-cover-failure s bad) s

by *(rule wp-event-union-bound)*

also have ... $\leq N + F$

by *(rule add-mono[OF no-failure-bound failure-bound])*

finally show ?thesis .

qed

lemma *wp-composition-fri-sampled-query-bad-candidate-bound-from-no-full-cover-or-failure:*

assumes *no-failure-bound:*

wp-event verify-monad

(composition-fri-sampled-query-no-full-cover-failure-candidate s bad) s

$\leq N$

and *failure-bound:*

wp-event verify-monad

(composition-fri-sampled-full-cover-failure s bad) s $\leq F$

shows

$$wp\text{-event } verify\text{-monad } (composition\text{-fri-sampled-query-bad-candidate } s) \ s$$

$$\leq N + F$$
proof –
have $wp\text{-event } verify\text{-monad}$

$$(composition\text{-fri-sampled-query-bad-candidate } s) \ s$$

$$\leq wp\text{-event } verify\text{-monad}$$

$$(\lambda out.$$

$$composition\text{-fri-sampled-query-no-full-cover-failure-candidate } s \ bad$$

$$out \ \vee$$

$$composition\text{-fri-sampled-full-cover-failure } s \ bad \ out) \ s$$
by $(rule \ wp\text{-event-mono})$
 $(rule$

$$composition\text{-fri-sampled-query-bad-candidate-imp-no-full-cover-or-failure})$$
also have $\dots \leq$

$$wp\text{-event } verify\text{-monad}$$

$$(composition\text{-fri-sampled-query-no-full-cover-failure-candidate } s \ bad)$$

$$s +$$

$$wp\text{-event } verify\text{-monad}$$

$$(composition\text{-fri-sampled-full-cover-failure } s \ bad) \ s$$
by $(rule \ wp\text{-event-union-bound})$
also have $\dots \leq N + F$
by $(rule \ add\text{-mono}[OF \ no\text{-failure-bound} \ failure\text{-bound}])$
finally show $?thesis$.
qed

lemma *checked-staged-security-trace-fri-sampled-query-bad-candidate-bound-from-no-full-cover-or-failure:*

assumes *no-failure-bound:*

$\wedge data \ attacker\text{-state}.$

$Some \ (data, \ attacker\text{-state}) \in$
 $set\text{-dist} \ (execute \ (checked\text{-staged-transcript-program } A)$
 $adversary\text{-initial-state}) \implies$

$wp\text{-event } verify\text{-monad}$
 $(trace\text{-fri-sampled-query-no-full-cover-failure-candidate}$
 $(verifier\text{-state-from-adversary } attacker\text{-state}$
 $(staged\text{-proof-transcript } data)) \ bad)$
 $(verifier\text{-state-from-adversary } attacker\text{-state}$
 $(staged\text{-proof-transcript } data)) \leq N$

and *failure-bound:*

$\wedge data \ attacker\text{-state}.$

$Some \ (data, \ attacker\text{-state}) \in$
 $set\text{-dist} \ (execute \ (checked\text{-staged-transcript-program } A)$
 $adversary\text{-initial-state}) \implies$

$wp\text{-event } verify\text{-monad}$
 $(trace\text{-fri-sampled-full-cover-failure}$
 $(verifier\text{-state-from-adversary } attacker\text{-state}$
 $(staged\text{-proof-transcript } data)) \ bad)$
 $(verifier\text{-state-from-adversary } attacker\text{-state}$
 $(staged\text{-proof-transcript } data)) \leq F$

shows
 $wp\text{-}event \ (checked\text{-}staged\text{-}security\text{-}experiment\text{-}with\text{-}data\text{-}state \ A)$
 $(staged\text{-}security\text{-}with\text{-}data\text{-}state\text{-}verifier\text{-}event$
 $\quad trace\text{-}fri\text{-}sampled\text{-}query\text{-}bad\text{-}candidate)$
 $adversary\text{-}initial\text{-}state \leq N + F$
proof (rule checked-staged-security-with-data-state-verifier-event-bound-from-cont)
show $\bigwedge s. \neg trace\text{-}fri\text{-}sampled\text{-}query\text{-}bad\text{-}candidate \ s \ None$
unfolding trace-fri-sampled-query-bad-candidate-def
 $trace\text{-}fri\text{-}partial\text{-}candidate\text{-}opening\text{-}evidence\text{-}def$
 $accepted\text{-}fri\text{-}opening\text{-}transcript\text{-}def$
by simp
next
fix data attacker-state
assume support:
 $Some \ (data, \ attacker\text{-}state) \in$
 $set\text{-}dist \ (execute \ (checked\text{-}staged\text{-}transcript\text{-}program \ A)$
 $\quad adversary\text{-}initial\text{-}state)$
let ?s =
 $verifier\text{-}state\text{-}from\text{-}adversary \ attacker\text{-}state$
 $(staged\text{-}proof\text{-}transcript \ data)$
show $wp\text{-}event \ verify\text{-}monad \ (trace\text{-}fri\text{-}sampled\text{-}query\text{-}bad\text{-}candidate \ ?s) \ ?s$
 $\leq N + F$
by (rule
 $wp\text{-}trace\text{-}fri\text{-}sampled\text{-}query\text{-}bad\text{-}candidate\text{-}bound\text{-}from\text{-}no\text{-}full\text{-}cover\text{-}or\text{-}failure)$
 $(rule \ no\text{-}failure\text{-}bound[OF \ support], \ rule \ failure\text{-}bound[OF \ support])$
qed

lemma checked-staged-security-composition-fri-sampled-query-bad-candidate-bound-from-no-full-cover-or-failure:
assumes no-failure-bound:
 $\bigwedge data \ attacker\text{-}state.$
 $Some \ (data, \ attacker\text{-}state) \in$
 $set\text{-}dist \ (execute \ (checked\text{-}staged\text{-}transcript\text{-}program \ A)$
 $\quad adversary\text{-}initial\text{-}state) \implies$
 $wp\text{-}event \ verify\text{-}monad$
 $(composition\text{-}fri\text{-}sampled\text{-}query\text{-}no\text{-}full\text{-}cover\text{-}failure\text{-}candidate$
 $\quad (verifier\text{-}state\text{-}from\text{-}adversary \ attacker\text{-}state$
 $\quad \quad (staged\text{-}proof\text{-}transcript \ data)) \ bad)$
 $(verifier\text{-}state\text{-}from\text{-}adversary \ attacker\text{-}state$
 $\quad \quad (staged\text{-}proof\text{-}transcript \ data)) \leq N$
and failure-bound:
 $\bigwedge data \ attacker\text{-}state.$
 $Some \ (data, \ attacker\text{-}state) \in$
 $set\text{-}dist \ (execute \ (checked\text{-}staged\text{-}transcript\text{-}program \ A)$
 $\quad adversary\text{-}initial\text{-}state) \implies$
 $wp\text{-}event \ verify\text{-}monad$
 $(composition\text{-}fri\text{-}sampled\text{-}full\text{-}cover\text{-}failure$
 $\quad (verifier\text{-}state\text{-}from\text{-}adversary \ attacker\text{-}state$
 $\quad \quad (staged\text{-}proof\text{-}transcript \ data)) \ bad)$
 $(verifier\text{-}state\text{-}from\text{-}adversary \ attacker\text{-}state$

```

      (staged-proof-transcript data)) ≤ F
shows
  wp-event (checked-staged-security-experiment-with-data-state A)
    (staged-security-with-data-state-verifier-event
      composition-fri-sampled-query-bad-candidate)
  adversary-initial-state ≤ N + F
proof (rule checked-staged-security-with-data-state-verifier-event-bound-from-cont)
  show  $\bigwedge s. \neg \text{composition-fri-sampled-query-bad-candidate } s \text{ None}$ 
  unfolding composition-fri-sampled-query-bad-candidate-def
    composition-fri-partial-candidate-opening-evidence-def
    accepted-fri-opening-transcript-def
  by simp
next
  fix data attacker-state
  assume support:
    Some (data, attacker-state) ∈
      set-dist (execute (checked-staged-transcript-program A)
        adversary-initial-state)
  let ?s =
    verifier-state-from-adversary attacker-state
      (staged-proof-transcript data)
  show wp-event verify-monad
    (composition-fri-sampled-query-bad-candidate ?s) ?s ≤ N + F
  by (rule
    wp-composition-fri-sampled-query-bad-candidate-bound-from-no-full-cover-or-failure)
    (rule no-failure-bound[OF support], rule failure-bound[OF support])
qed

lemma checked-staged-security-trace-fri-sampled-query-bad-candidate-bound-from-staged-no-full-cover-or-failure:
assumes no-failure-bound:
  wp-event (checked-staged-security-experiment-with-data-state A)
    (staged-security-with-data-state-verifier-event
      ( $\lambda s. \text{trace-fri-sampled-query-no-full-cover-failure-candidate } s \text{ bad}$ ))
  adversary-initial-state ≤ N
and failure-bound:
  wp-event (checked-staged-security-experiment-with-data-state A)
    (staged-security-with-data-state-verifier-event
      ( $\lambda s. \text{trace-fri-sampled-full-cover-failure } s \text{ bad}$ ))
  adversary-initial-state ≤ F
shows
  wp-event (checked-staged-security-experiment-with-data-state A)
    (staged-security-with-data-state-verifier-event
      trace-fri-sampled-query-bad-candidate)
  adversary-initial-state ≤ N + F
proof –
  have wp-event (checked-staged-security-experiment-with-data-state A)
    (staged-security-with-data-state-verifier-event
      trace-fri-sampled-query-bad-candidate)
  adversary-initial-state ≤

```

```

wp-event (checked-staged-security-experiment-with-data-state A)
  (staged-security-with-data-state-verifier-event
    (λs out.
      trace-fri-sampled-query-no-full-cover-failure-candidate s bad
      out ∨
      trace-fri-sampled-full-cover-failure s bad out))
  adversary-initial-state
by (rule wp-event-mono)
  (auto simp: staged-security-with-data-state-verifier-event-def
    dest: trace-fri-sampled-query-bad-candidate-imp-no-full-cover-or-failure
    split: option.splits prod.splits)
also have ... ≤
  wp-event (checked-staged-security-experiment-with-data-state A)
    (λout.
      staged-security-with-data-state-verifier-event
        (λs. trace-fri-sampled-query-no-full-cover-failure-candidate s
          bad) out ∨
      staged-security-with-data-state-verifier-event
        (λs. trace-fri-sampled-full-cover-failure s bad) out)
    adversary-initial-state
by (rule wp-event-mono-on-support)
  (auto simp: staged-security-with-data-state-verifier-event-def
    split: option.splits prod.splits)
also have ... ≤
  wp-event (checked-staged-security-experiment-with-data-state A)
    (staged-security-with-data-state-verifier-event
      (λs. trace-fri-sampled-query-no-full-cover-failure-candidate s
        bad))
    adversary-initial-state +
  wp-event (checked-staged-security-experiment-with-data-state A)
    (staged-security-with-data-state-verifier-event
      (λs. trace-fri-sampled-full-cover-failure s bad))
    adversary-initial-state
by (rule wp-event-union-bound)
also have ... ≤ N + F
  by (rule add-mono[OF no-failure-bound failure-bound])
finally show ?thesis .
qed

```

lemma *checked-staged-security-composition-fri-sampled-query-bad-candidate-bound-from-staged-no-full-cover-or-failure*

assumes *no-failure-bound*:

```

wp-event (checked-staged-security-experiment-with-data-state A)
  (staged-security-with-data-state-verifier-event
    (λs.
      composition-fri-sampled-query-no-full-cover-failure-candidate s
      bad))

```

adversary-initial-state ≤ N

and *failure-bound*:

```

wp-event (checked-staged-security-experiment-with-data-state A)

```

(staged-security-with-data-state-verifier-event
 (λs. composition-fri-sampled-full-cover-failure s bad))
 adversary-initial-state ≤ F

shows

wp-event (checked-staged-security-experiment-with-data-state A)
 (staged-security-with-data-state-verifier-event
 composition-fri-sampled-query-bad-candidate)
 adversary-initial-state ≤ N + F

proof –

have wp-event (checked-staged-security-experiment-with-data-state A)
 (staged-security-with-data-state-verifier-event
 composition-fri-sampled-query-bad-candidate)
 adversary-initial-state ≤
 wp-event (checked-staged-security-experiment-with-data-state A)
 (staged-security-with-data-state-verifier-event
 (λs out.
 composition-fri-sampled-query-no-full-cover-failure-candidate s
 bad out ∨
 composition-fri-sampled-full-cover-failure s bad out))
 adversary-initial-state

by (rule wp-event-mono)
 (auto simp: staged-security-with-data-state-verifier-event-def
 dest:
 composition-fri-sampled-query-bad-candidate-imp-no-full-cover-or-failure
 split: option.splits prod.splits)

also have ... ≤
 wp-event (checked-staged-security-experiment-with-data-state A)
 (λout.
 staged-security-with-data-state-verifier-event
 (λs.
 composition-fri-sampled-query-no-full-cover-failure-candidate s
 bad) out ∨
 staged-security-with-data-state-verifier-event
 (λs. composition-fri-sampled-full-cover-failure s bad) out)
 adversary-initial-state

by (rule wp-event-mono-on-support)
 (auto simp: staged-security-with-data-state-verifier-event-def
 split: option.splits prod.splits)

also have ... ≤
 wp-event (checked-staged-security-experiment-with-data-state A)
 (staged-security-with-data-state-verifier-event
 (λs.
 composition-fri-sampled-query-no-full-cover-failure-candidate s
 bad))
 adversary-initial-state +
 wp-event (checked-staged-security-experiment-with-data-state A)
 (staged-security-with-data-state-verifier-event
 (λs. composition-fri-sampled-full-cover-failure s bad))
 adversary-initial-state

```

    by (rule wp-event-union-bound)
  also have ...  $\leq N + F$ 
    by (rule add-mono[OF no-failure-bound failure-bound])
  finally show ?thesis .
qed

lemma checked-staged-security-trace-fri-sampled-query-no-full-cover-failure-candidate-bound-from-pair-fraction-chall
  assumes wf: staged-budget-wellformed budgets
    and controlled: staged-adversary-controlled budgets A
    and fraction:
      generic-fri-sampled-query-pair-fraction-bound trace-table-low-degree
        (Not  $\circ$  trace-table-low-degree) (clength - 1) C
  shows
    wp-event (checked-staged-security-experiment-with-data-state A)
      (staged-security-with-data-state-verifier-event
        ( $\lambda s$ . trace-fri-sampled-query-no-full-cover-failure-candidate s bad))
    adversary-initial-state  $\leq$ 
      C +
      hash-relation-budget-value
        (query-index-raw-list-relation-fiber-bound
          ( $\bigcup$  challenges  $\in$  fri-challenge-space (ceil-log clength).
            generic-fri-sampled-query-query-fiber trace-table-low-degree
              (Not  $\circ$  trace-table-low-degree) (clength - 1) challenges))
        (staged-attacker-query-budget budgets +
          staged-challenge-query-budget) +
      hash-relation-budget-value
        (card (fri-challenge-space (ceil-log clength)) * ceil-log clength)
        (staged-attacker-query-budget budgets +
          staged-challenge-query-budget)
  proof -
  have wp-event (checked-staged-security-experiment-with-data-state A)
    (staged-security-with-data-state-verifier-event
      ( $\lambda s$ . trace-fri-sampled-query-no-full-cover-failure-candidate s bad))
    adversary-initial-state
     $\leq$  wp-event (checked-staged-security-experiment-with-data-state A)
      (staged-security-with-data-state-verifier-event
        trace-fri-sampled-query-bad-candidate)
      adversary-initial-state
  by (rule wp-event-mono)
    (auto simp: staged-security-with-data-state-verifier-event-def
      intro:
        trace-fri-sampled-query-no-full-cover-failure-candidate-imp-bad-candidate
        split: option.splits prod.splits)
  also have ...  $\leq$ 
    C +
    hash-relation-budget-value
      (query-index-raw-list-relation-fiber-bound
        ( $\bigcup$  challenges  $\in$  fri-challenge-space (ceil-log clength).
          generic-fri-sampled-query-query-fiber trace-table-low-degree
            (Not  $\circ$  trace-table-low-degree) (clength - 1) challenges))
      (staged-attacker-query-budget budgets +
        staged-challenge-query-budget)

```

```

      (Not  $\circ$  trace-table-low-degree) (clength - 1) challenges))
    (staged-attacker-query-budget budgets +
      staged-challenge-query-budget) +
    hash-relation-budget-value
    (card (fri-challenge-space (ceil-log clength)) * ceil-log clength)
    (staged-attacker-query-budget budgets +
      staged-challenge-query-budget)
  by (rule
    checked-staged-security-trace-fri-sampled-query-bad-candidate-bound-from-pair-fraction-challenge-weighted
    [OF wf controlled fraction])
  finally show ?thesis .
qed

```

lemma *checked-staged-security-trace-fri-sampled-query-no-full-cover-failure-candidate-bound-from-restricted-pairs:*
assumes wf: staged-budget-wellformed budgets
and controlled: staged-adversary-controlled budgets A
shows

```

wp-event (checked-staged-security-experiment-with-data-state A)
  (staged-security-with-data-state-verifier-event
    ( $\lambda s$ . trace-fri-sampled-query-no-full-cover-failure-candidate s bad))
adversary-initial-state  $\leq$ 
  ( $\sum$  challenges  $\in$  fri-challenge-space (ceil-log clength).
    (1 / nnreal (CARD('f) ^ ceil-log clength)) *
    (nnreal
      (card
        (trace-fri-sampled-query-no-full-cover-failure-query-fiber
          bad challenges)) *
        (1 / nnreal (card query-sample-space)) ^ rounds)) +
  hash-relation-budget-value
  (query-index-raw-list-relation-fiber-bound
    ( $\bigcup$  challenges  $\in$  fri-challenge-space (ceil-log clength).
      trace-fri-sampled-query-no-full-cover-failure-query-fiber
        bad challenges))
  (staged-attacker-query-budget budgets +
    staged-challenge-query-budget) +
  hash-relation-budget-value
  (card (fri-challenge-space (ceil-log clength)) * ceil-log clength)
  (staged-attacker-query-budget budgets +
    staged-challenge-query-budget)

```

proof –

```

let ?P = trace-fri-sampled-query-no-full-cover-failure-pairs bad
let ?B = fri-challenge-space (ceil-log clength)
let ?Q =
  trace-fri-sampled-query-no-full-cover-failure-query-fiber bad
have event-le:
  wp-event (checked-staged-security-experiment-with-data-state A)
    (staged-security-with-data-state-verifier-event
      ( $\lambda s$ . trace-fri-sampled-query-no-full-cover-failure-candidate s bad))
  adversary-initial-state  $\leq$ 

```



```

wp-event (checked-staged-security-experiment-with-data-state A)
  (staged-security-with-data-state-verifier-event
    ( $\lambda s.$  trace-fri-query-challenge-pair-set-hit s ?P))
  adversary-initial-state
by (rule wp-event-mono-on-support)
  (auto simp: staged-security-with-data-state-verifier-event-def
    intro:
      trace-fri-sampled-query-no-full-cover-failure-candidate-imp-pair-hit
      split: option.splits prod.splits)
have pair-bound:
  wp-event (checked-staged-security-experiment-with-data-state A)
    (staged-security-with-data-state-verifier-event
      ( $\lambda s.$  trace-fri-query-challenge-pair-set-hit s ?P))
    adversary-initial-state  $\leq$ 
    ( $\sum$  challenges  $\in$  ?B.
      (1 / nnreal (CARD('f)  $\wedge$  ceil-log clength)) *
      (nnreal (card (?Q challenges)) *
        (1 / nnreal (card query-sample-space))  $\wedge$  rounds)) +
    hash-relation-budget-value
      (query-index-row-list-relation-fiber-bound
        ( $\bigcup$  challenges  $\in$  ?B. ?Q challenges))
      (staged-attacker-query-budget budgets +
        staged-challenge-query-budget) +
    hash-relation-budget-value (card ?B * ceil-log clength)
      (staged-attacker-query-budget budgets +
        staged-challenge-query-budget)
proof (rule
  checked-staged-security-trace-fri-query-challenge-pair-set-hit-challenge-weighted-bound
  [OF wf controlled])
show ?B  $\subseteq$  fri-challenge-space (ceil-log clength)
  by simp
next
show snd ' ?P  $\subseteq$  ?B
  by (rule
    trace-fri-sampled-query-no-full-cover-failure-pairs-challenge-projection)
next
fix challenges
assume challenges  $\in$  ?B
show fst ' (?P  $\cap$  (UNIV  $\times$  {challenges}))  $\subseteq$ 
  ?Q challenges
  unfolding trace-fri-sampled-query-no-full-cover-failure-query-fiber-def
    fri-query-challenge-pair-query-fiber-def
  by force
next
fix challenges
assume challenges  $\in$  ?B
show ?Q challenges  $\subseteq$  fri-query-index-list-space
  by (rule
    trace-fri-sampled-query-no-full-cover-failure-query-fiber-subset)

```

```

qed
show ?thesis
  by (rule order-trans[OF event-le pair-bound])
qed

lemma checked-staged-security-composition-fri-sampled-query-no-full-cover-failure-candidate-bound-from-restricted-p
  assumes wf: staged-budget-wellformed budgets
  and controlled: staged-adversary-controlled budgets A
  shows
    wp-event (checked-staged-security-experiment-with-data-state A)
      (staged-security-with-data-state-verifier-event
        (λs.
          composition-fri-sampled-query-no-full-cover-failure-candidate s
            bad))
    adversary-initial-state ≤
    (∑ dg ∈ UNIV.
      ∑ challenges ∈ fri-challenge-space (ceil-log (to-nat dg + 1)).
        (1 / nnreal (CARD('f) ^ ceil-log (to-nat dg + 1))) *
        (nnreal
          (card
            (composition-fri-sampled-query-no-full-cover-failure-query-fiber
              bad dg challenges)) *
            (1 / nnreal (card query-sample-space)) ^ rounds)) +
      hash-relation-budget-value
        (query-index-row-list-relation-fiber-bound
          (∪ dg ∈ UNIV.
            ∪ challenges ∈ fri-challenge-space (ceil-log (to-nat dg + 1)).
              composition-fri-sampled-query-no-full-cover-failure-query-fiber
                bad dg challenges))
          (staged-attacker-query-budget budgets +
            staged-challenge-query-budget) +
          hash-relation-budget-value
            (∑ dg ∈ (UNIV :: 'f set).
              card (fri-challenge-space (ceil-log (to-nat dg + 1))) *
              ceil-log (maxDegree + 1))
            (staged-attacker-query-budget budgets +
              staged-challenge-query-budget)
        )
    proof -
    let ?P =
      composition-fri-sampled-query-no-full-cover-failure-pairs bad
    let ?B =
      λdg. fri-challenge-space (ceil-log (to-nat dg + 1))
    let ?Q =
      composition-fri-sampled-query-no-full-cover-failure-query-fiber bad
    have event-le:
      wp-event (checked-staged-security-experiment-with-data-state A)
        (staged-security-with-data-state-verifier-event
          (λs.
            composition-fri-sampled-query-no-full-cover-failure-candidate s

```

```

    bad))
  adversary-initial-state ≤
wp-event (checked-staged-security-experiment-with-data-state A)
  (staged-security-with-data-state-verifier-event
    (λs. composition-fri-query-challenge-pair-set-hit s ?P))
  adversary-initial-state
by (rule wp-event-mono-on-support)
  (auto simp: staged-security-with-data-state-verifier-event-def
    intro:
      composition-fri-sampled-query-no-full-cover-failure-candidate-imp-pair-hit
      split: option.splits prod.splits)
have pair-bound:
wp-event (checked-staged-security-experiment-with-data-state A)
  (staged-security-with-data-state-verifier-event
    (λs. composition-fri-query-challenge-pair-set-hit s ?P))
  adversary-initial-state ≤
  (∑ dg ∈ UNIV.
    ∑ challenges ∈ ?B dg.
      (1 / nnreal (CARD('f) ^ ceil-log (to-nat dg + 1))) *
      (nnreal (card (?Q dg challenges)) *
        (1 / nnreal (card query-sample-space)) ^ rounds)) +
  hash-relation-budget-value
  (query-index-raw-list-relation-fiber-bound
    (∪ dg ∈ UNIV. ∪ challenges ∈ ?B dg.
      ?Q dg challenges))
  (staged-attacker-query-budget budgets +
    staged-challenge-query-budget) +
  hash-relation-budget-value
  (∑ dg ∈ (UNIV :: 'f set).
    card (?B dg) * ceil-log (maxDegree + 1))
  (staged-attacker-query-budget budgets +
    staged-challenge-query-budget)
proof (rule
  checked-staged-security-composition-fri-query-challenge-pair-set-hit-challenge-weighted-bound
  [OF wf controlled])
  fix dg
  show ?B dg ⊆ fri-challenge-space (ceil-log (to-nat dg + 1))
  by simp
next
  fix dg
  show snd ' ?P dg ⊆ ?B dg
  by (rule
    composition-fri-sampled-query-no-full-cover-failure-pairs-challenge-projection)
next
  fix dg challenges
  assume challenges ∈ ?B dg
  show fst ' (?P dg ∩ (UNIV × {challenges})) ⊆
    ?Q dg challenges
  unfolding

```

```

    composition-fri-sampled-query-no-full-cover-failure-query-fiber-def
    fri-query-challenge-pair-query-fiber-def
  by force
next
fix dg challenges
assume challenges ∈ ?B dg
show ?Q dg challenges ⊆ fri-query-index-list-space
  by (rule
    composition-fri-sampled-query-no-full-cover-failure-query-fiber-subset)
qed
show ?thesis
  by (rule order-trans[OF event-le pair-bound])
qed

lemma checked-staged-security-composition-fri-sampled-query-no-full-cover-failure-candidate-bound-from-pair-fraction
  fixes C :: 'f ⇒ prob
  assumes wf: staged-budget-wellformed budgets
  and controlled: staged-adversary-controlled budgets A
  and fraction:
    ∧ dg. generic-fri-sampled-query-pair-fraction-bound
      (composition-table-low-degree (to-nat dg))
      (Not ∘ composition-table-low-degree maxDegree) (to-nat dg) (C dg)
  shows
    wp-event (checked-staged-security-experiment-with-data-state A)
      (staged-security-with-data-state-verifier-event
        (λs.
          composition-fri-sampled-query-no-full-cover-failure-candidate s
            bad))
    adversary-initial-state ≤
      (∑ dg ∈ UNIV. C dg) +
      hash-relation-budget-value
        (query-index-row-list-relation-fiber-bound
          (⋃ dg ∈ UNIV.
            ⋃ challenges ∈ fri-challenge-space (ceil-log (to-nat dg + 1)).
              generic-fri-sampled-query-query-fiber
                (composition-table-low-degree (to-nat dg))
                (Not ∘ composition-table-low-degree maxDegree)
                (to-nat dg) challenges))
          (staged-attacker-query-budget budgets +
            staged-challenge-query-budget) +
          hash-relation-budget-value
            (∑ dg ∈ (UNIV :: 'f set).
              card (fri-challenge-space (ceil-log (to-nat dg + 1))) *
                ceil-log (maxDegree + 1))
            (staged-attacker-query-budget budgets +
              staged-challenge-query-budget))
  proof —
  have wp-event (checked-staged-security-experiment-with-data-state A)
    (staged-security-with-data-state-verifier-event

```

```

    (λs.
      composition-fri-sampled-query-no-full-cover-failure-candidate s
      bad))
  adversary-initial-state
≤ wp-event (checked-staged-security-experiment-with-data-state A)
  (staged-security-with-data-state-verifier-event
    composition-fri-sampled-query-bad-candidate)
  adversary-initial-state
by (rule wp-event-mono)
  (auto simp: staged-security-with-data-state-verifier-event-def
    intro:
      composition-fri-sampled-query-no-full-cover-failure-candidate-imp-bad-candidate
      split: option.splits prod.splits)
also have ... ≤
  (∑ dg ∈ UNIV. C dg) +
  hash-relation-budget-value
  (query-index-row-list-relation-fiber-bound
    (∪ dg ∈ UNIV.
      ∪ challenges ∈ fri-challenge-space (ceil-log (to-nat dg + 1)).
        generic-fri-sampled-query-query-fiber
          (composition-table-low-degree (to-nat dg))
          (Not ∘ composition-table-low-degree maxDegree)
          (to-nat dg) challenges))
    (staged-attacker-query-budget budgets +
      staged-challenge-query-budget) +
  hash-relation-budget-value
  (∑ dg ∈ (UNIV :: 'f set).
    card (fri-challenge-space (ceil-log (to-nat dg + 1))) *
    ceil-log (maxDegree + 1))
  (staged-attacker-query-budget budgets +
    staged-challenge-query-budget)
by (rule
  checked-staged-security-composition-fri-sampled-query-bad-candidate-bound-from-pair-fraction-challenge-weighted
  [OF wf controlled fraction])
finally show ?thesis .
qed

lemma finite-generic-fri-bad-challenge-lists:
  finite (generic-fri-bad-challenge-lists n bad)
proof (rule finite-subset)
  show generic-fri-bad-challenge-lists n bad ⊆ fri-challenge-space n
    by (rule generic-fri-bad-challenge-lists-subset)
  show finite (fri-challenge-space n)
    by (rule finite-fri-challenge-space)
qed

lemma generic-fri-sampled-query-finite-cover-from-bad-challenge-lists:
  assumes cover:
    ∧ candidate-table roots final-value round-layers layers query-idxs

```

challenges.
query-idxs $\in$ *fri-query-index-list-space* $\implies$
generic-fri-sampled-query-candidate-evidence *low-degree* *candidate-bad*
candidate-table *degree-bound* *roots* *challenges* *final-value* *query-idxs*
round-layers *layers* $\implies$
challenges $\in$
generic-fri-bad-challenge-lists
(fri-round-count-for-degree-bound degree-bound) *bad*
shows
generic-fri-sampled-query-finite-cover *low-degree* *candidate-bad*
degree-bound
(generic-fri-bad-challenge-lists
(fri-round-count-for-degree-bound degree-bound) *bad)*
proof (rule *generic-fri-sampled-query-finite-cover-from-candidate-challenge-cover*)
show *finite*
(generic-fri-bad-challenge-lists
(fri-round-count-for-degree-bound degree-bound) *bad)*
by (rule *finite-generic-fri-bad-challenge-lists*)
fix *candidate-table* *roots* *final-value* *round-layers* *layers* *query-idxs*
challenges
assume *query-space*: *query-idxs* $\in$ *fri-query-index-list-space*
and *evidence*:
generic-fri-sampled-query-candidate-evidence *low-degree* *candidate-bad*
candidate-table *degree-bound* *roots* *challenges* *final-value* *query-idxs*
round-layers *layers*
show *challenges* $\in$
generic-fri-bad-challenge-lists
(fri-round-count-for-degree-bound degree-bound) *bad*
by (rule *cover[OF query-space evidence]*)
qed

lemma *generic-fri-sampled-query-projection-fraction-bound-from-bad-challenge-round-bounds*:
assumes *cover*:
 $\bigwedge$ *candidate-table* *roots* *final-value* *round-layers* *layers* *query-idxs*
challenges.
query-idxs $\in$ *fri-query-index-list-space* $\implies$
generic-fri-sampled-query-candidate-evidence *low-degree* *candidate-bad*
candidate-table *degree-bound* *roots* *challenges* *final-value* *query-idxs*
round-layers *layers* $\implies$
challenges $\in$
generic-fri-bad-challenge-lists
(fri-round-count-for-degree-bound degree-bound) *bad*
and *local-bound*:
 $\bigwedge i$ *prefix.*
i < *fri-round-count-for-degree-bound degree-bound* $\implies$
length prefix = *i* $\implies$
card (bad i prefix) $\leq$ *bd i*
and *fraction-bound*:
nnreal

$$\begin{aligned}
& (\sum i \in \{..<\text{fri-round-count-for-degree-bound degree-bound}\}. \\
& \quad \text{CARD}(f) \wedge i * \text{bd } i * \\
& \quad \text{CARD}(f) \wedge \\
& \quad (\text{fri-round-count-for-degree-bound degree-bound} - \text{Suc } i)) / \\
& \quad \text{nnreal } (\text{CARD}(f) \wedge \text{fri-round-count-for-degree-bound degree-bound}) \\
& \leq C
\end{aligned}$$

shows

generic-fri-sampled-query-projection-fraction-bound low-degree
candidate-bad degree-bound C

proof –

let $?n = \text{fri-round-count-for-degree-bound degree-bound}$
let $?B = \text{generic-fri-bad-challenge-lists } ?n \text{ bad}$
let $?K =$

$$\begin{aligned}
& (\sum i \in \{..<?n\}. \\
& \quad \text{CARD}(f) \wedge i * \text{bd } i * \text{CARD}(f) \wedge (?n - \text{Suc } i))
\end{aligned}$$

have *cover-B*:
generic-fri-sampled-query-finite-cover low-degree candidate-bad
degree-bound ?B
by (rule *generic-fri-sampled-query-finite-cover-from-bad-challenge-lists*
[OF *cover*])

have *card-bound*: *card ?B ≤ ?K*
by (rule *generic-fri-bad-challenge-lists-card-bound-from-round-bounds*)
(rule *local-bound*)

have *ratio-bound*:
 $\text{nnreal } (\text{card } ?B) / \text{nnreal } (\text{CARD}(f) \wedge ?n) \leq C$

proof –

have $\text{nnreal } (\text{card } ?B) / \text{nnreal } (\text{CARD}(f) \wedge ?n)$
 $\leq \text{nnreal } ?K / \text{nnreal } (\text{CARD}(f) \wedge ?n)$
using *card-bound* **by** (rule *nnreal-nat-divide-right-mono*)

also have $\dots \leq C$
by (rule *fraction-bound*)

finally show *?thesis* .

qed

show *?thesis*
by (rule *generic-fri-sampled-query-projection-fraction-bound-from-cover*
[OF *cover-B ratio-bound*])

qed

lemma *generic-fri-sampled-query-projection-fraction-bound-from-one-step-disagreement-subset*:
assumes *cover*:
 $\bigwedge \text{candidate-table roots final-value round-layers layers query-idxs}$
challenges.
 $\text{query-idxs} \in \text{fri-query-index-list-space} \implies$
generic-fri-sampled-query-candidate-evidence low-degree candidate-bad
candidate-table degree-bound roots challenges final-value query-idxs
round-layers layers $\implies$
challenges $\in$
generic-fri-bad-challenge-lists
(fri-round-count-for-degree-bound degree-bound) bad

and *local-subset*:
 $\bigwedge i \text{ prefix.}$
 $i < \text{fri-round-count-for-degree-bound degree-bound} \implies$
 $\text{length prefix} = i \implies$
 $\text{bad } i \text{ prefix} \subseteq$
 $\text{fri-one-step-disagreement-challenges}$
 $(\text{len } i) (\text{pw } i) (\text{layer } i \text{ prefix}) (\text{claimed } i \text{ prefix}) (\text{domains } i \text{ prefix})$
and *fraction-bound*:
 nnreal
 $(\sum i \in \{..<\text{fri-round-count-for-degree-bound degree-bound}\}).$
 $\text{CARD}('f) \wedge i * (\text{len } i \text{ div } 2) *$
 $\text{CARD}('f) \wedge$
 $(\text{fri-round-count-for-degree-bound degree-bound} - \text{Suc } i)) /$
 $\text{nnreal } (\text{CARD}('f) \wedge \text{fri-round-count-for-degree-bound degree-bound})$
 $\leq C$
shows
 $\text{generic-fri-sampled-query-projection-fraction-bound low-degree}$
 $\text{candidate-bad degree-bound } C$
proof (rule
 $\text{generic-fri-sampled-query-projection-fraction-bound-from-bad-challenge-round-bounds}$
 $[\text{OF cover - fraction-bound}])$
fix $i :: \text{nat}$ **and** $\text{prefix} :: 'f \text{ list}$
assume $i: i < \text{fri-round-count-for-degree-bound degree-bound}$
and $\text{prefix}: \text{length prefix} = i$
have $\text{card } (\text{bad } i \text{ prefix}) \leq$
 card
 $(\text{fri-one-step-disagreement-challenges}$
 $(\text{len } i) (\text{pw } i) (\text{layer } i \text{ prefix}) (\text{claimed } i \text{ prefix})$
 $(\text{domains } i \text{ prefix}))$
by (rule card-mono) ($\text{simp-all add: local-subset}[\text{OF } i \text{ prefix}]$)
also have $\dots \leq \text{len } i \text{ div } 2$
by (rule $\text{fri-one-step-disagreement-challenges-card-bound-local}$)
finally show $\text{card } (\text{bad } i \text{ prefix}) \leq \text{len } i \text{ div } 2 .$
qed

lemma *generic-fri-sampled-query-projection-fraction-bound-from-one-step-disagreement-subset-exact*:
assumes *cover*:
 $\bigwedge \text{candidate-table roots final-value round-layers layers query-idxs}$
 challenges.
 $\text{query-idxs} \in \text{fri-query-index-list-space} \implies$
 $\text{generic-fri-sampled-query-candidate-evidence low-degree candidate-bad}$
 $\text{candidate-table degree-bound roots challenges final-value query-idxs}$
 $\text{round-layers layers} \implies$
 $\text{challenges} \in$
 $\text{generic-fri-bad-challenge-lists}$
 $(\text{fri-round-count-for-degree-bound degree-bound}) \text{ bad}$
and *local-subset*:
 $\bigwedge i \text{ prefix.}$
 $i < \text{fri-round-count-for-degree-bound degree-bound} \implies$


```

    length prefix = i  $\implies$ 
    bad i prefix  $\subseteq$ 
      fri-one-step-disagreement-challenges
      (len i) (pw i) (layer i prefix) (claimed i prefix) (domains i prefix)
  shows
    generic-fri-sampled-query-projection-fraction-bound low-degree
    candidate-bad degree-bound
    (fri-one-step-disagreement-round-fraction
     (fri-round-count-for-degree-bound degree-bound) len)
  proof (rule
    generic-fri-sampled-query-projection-fraction-bound-from-one-step-disagreement-subset)
  fix candidate-table roots final-value round-layers layers query-idxs challenges
  assume query-idxs: query-idxs  $\in$  fri-query-index-list-space
  and evidence:
    generic-fri-sampled-query-candidate-evidence low-degree candidate-bad
    candidate-table degree-bound roots challenges final-value query-idxs
    round-layers layers
  show challenges  $\in$ 
    generic-fri-bad-challenge-lists
    (fri-round-count-for-degree-bound degree-bound) bad
  by (rule cover[OF query-idxs evidence])
next
  fix i :: nat and prefix :: 'f list
  assume i: i < fri-round-count-for-degree-bound degree-bound
  and prefix: length prefix = i
  show bad i prefix  $\subseteq$ 
    fri-one-step-disagreement-challenges
    (len i) (pw i) (layer i prefix) (claimed i prefix) (domains i prefix)
  by (rule local-subset[OF i prefix])
next
  show nnreal
    ( $\sum i \in \{..< \text{fri-round-count-for-degree-bound degree-bound}\}.$ 
     CARD('f)  $\wedge$  i * (len i div 2) *
     CARD('f)  $\wedge$ 
     (fri-round-count-for-degree-bound degree-bound - Suc i)) /
    nnreal (CARD('f)  $\wedge$  fri-round-count-for-degree-bound degree-bound)
     $\leq$  fri-one-step-disagreement-round-fraction
    (fri-round-count-for-degree-bound degree-bound) len
  by (rule fri-one-step-disagreement-round-fraction-refl-bound)
qed

lemma generic-fri-sampled-query-pair-fraction-bound-from-bad-challenge-round-bounds:
  assumes cover:
     $\bigwedge$  candidate-table roots final-value round-layers layers query-idxs
      challenges.
    query-idxs  $\in$  fri-query-index-list-space  $\implies$ 
    generic-fri-sampled-query-candidate-evidence low-degree candidate-bad
    candidate-table degree-bound roots challenges final-value query-idxs
    round-layers layers  $\implies$ 

```

```

    challenges ∈
      generic-fri-bad-challenge-lists
      (fri-round-count-for-degree-bound degree-bound) bad
  and local-bound:
    ∧i prefix.
      i < fri-round-count-for-degree-bound degree-bound ⇒
        length prefix = i ⇒
          card (bad i prefix) ≤ bd i
  and fraction-bound:
    nnreal
      (∑ i ∈ {..fri-round-count-for-degree-bound degree-bound}.
        CARD('f) ^ i * bd i *
        CARD('f) ^
          (fri-round-count-for-degree-bound degree-bound - Suc i)) /
      nnreal (CARD('f) ^ fri-round-count-for-degree-bound degree-bound)
    ≤ C
  shows
    generic-fri-sampled-query-pair-fraction-bound low-degree candidate-bad
    degree-bound C
proof -
  let ?n = fri-round-count-for-degree-bound degree-bound
  let ?B = generic-fri-bad-challenge-lists ?n bad
  let ?K =
    (∑ i ∈ {..?n}.
      CARD('f) ^ i * bd i * CARD('f) ^ (?n - Suc i))
  have projection:
    generic-fri-sampled-query-challenge-projection low-degree
    candidate-bad degree-bound ⊆ ?B
proof
  fix challenges
  assume challenges-in:
    challenges ∈
      generic-fri-sampled-query-challenge-projection low-degree
      candidate-bad degree-bound
  then obtain query-idxs where pair:
    (query-idxs, challenges) ∈
      generic-fri-sampled-query-restricted-bad-pairs low-degree
      candidate-bad degree-bound
  unfolding generic-fri-sampled-query-challenge-projection-def by force
  then have query-space: query-idxs ∈ fri-query-index-list-space
  unfolding generic-fri-sampled-query-restricted-bad-pairs-def by simp
  have pair-union:
    (query-idxs, challenges) ∈
      generic-fri-sampled-query-bad-pair-union low-degree candidate-bad
      degree-bound
  using pair
  unfolding generic-fri-sampled-query-restricted-bad-pairs-def by simp
  then obtain candidate-table roots final-value round-layers layers where
    pair-set:

```

```

    (query-idxs, challenges) ∈
      generic-fri-sampled-query-bad-pair-set low-degree candidate-bad
      candidate-table degree-bound roots final-value round-layers layers
unfolding generic-fri-sampled-query-bad-pair-union-def by auto
have evidence:
  generic-fri-sampled-query-candidate-evidence low-degree
  candidate-bad candidate-table degree-bound roots challenges
  final-value query-idxs round-layers layers
by (rule generic-fri-sampled-query-bad-pair-setD[OF pair-set])
show challenges ∈ ?B
by (rule cover[OF query-space evidence])
qed
have card-bound: card ?B ≤ ?K
by (rule generic-fri-bad-challenge-lists-card-bound-from-round-bounds)
    (rule local-bound)
have ratio-bound:
  nnreal (card ?B * card fri-query-index-list-space) /
  nnreal (card fri-query-index-list-space * CARD('f) ^ ?n)
  ≤ C
proof –
have card-q-pos: 0 < card fri-query-index-list-space
by (simp add: card-fri-query-index-list-space
  query-sample-space-size-pos)
then have card-q-ne: card fri-query-index-list-space ≠ 0
by simp
have nnreal (card ?B * card fri-query-index-list-space) /
  nnreal (card fri-query-index-list-space * CARD('f) ^ ?n) =
  nnreal (card ?B) / nnreal (CARD('f) ^ ?n)
by (rule nnreal-nat-mult-divide-cancel-left[OF card-q-pos])
also have ... ≤ nnreal ?K / nnreal (CARD('f) ^ ?n)
using card-bound by (rule nnreal-nat-divide-right-mono)
also have ... ≤ C
by (rule fraction-bound)
finally show ?thesis .
qed
show ?thesis
by (rule generic-fri-sampled-query-pair-fraction-bound-from-projection-and-fibers
  [OF finite-generic-fri-bad-challenge-lists projection - ratio-bound])
  (meson card-mono finite-fri-query-index-list-space
  generic-fri-sampled-query-query-fiber-subset)
qed

lemma generic-fri-sampled-query-pair-fraction-bound-from-one-step-disagreement-subset:
assumes cover:
  ∧ candidate-table roots final-value round-layers layers query-idxs
  challenges.
  query-idxs ∈ fri-query-index-list-space ⇒
  generic-fri-sampled-query-candidate-evidence low-degree candidate-bad
  candidate-table degree-bound roots challenges final-value query-idxs

```

round-layers layers $\implies$
challenges $\in$
generic-fri-bad-challenge-lists
(fri-round-count-for-degree-bound degree-bound) bad
and *local-subset*:
 $\bigwedge i$ *prefix*.
 $i < \text{fri-round-count-for-degree-bound degree-bound} \implies$
 $\text{length prefix} = i \implies$
 $\text{bad } i \text{ prefix} \subseteq$
fri-one-step-disagreement-challenges
(len i) (pw i) (layer i prefix) (claimed i prefix) (domains i prefix)
and *fraction-bound*:
 nnreal
 $(\sum i \in \{..<\text{fri-round-count-for-degree-bound degree-bound}\}.$
 $\text{CARD}(f) \wedge i * (\text{len } i \text{ div } 2) *$
 $\text{CARD}(f) \wedge$
 $(\text{fri-round-count-for-degree-bound degree-bound} - \text{Suc } i)) /$
 $\text{nnreal } (\text{CARD}(f) \wedge \text{fri-round-count-for-degree-bound degree-bound})$
 $\leq C$
shows
generic-fri-sampled-query-pair-fraction-bound low-degree candidate-bad
degree-bound C
proof (rule
generic-fri-sampled-query-pair-fraction-bound-from-bad-challenge-round-bounds
[OF cover - fraction-bound])
fix $i :: \text{nat}$ **and** $\text{prefix} :: 'f \text{ list}$
assume $i: i < \text{fri-round-count-for-degree-bound degree-bound}$
and $\text{prefix}: \text{length prefix} = i$
have $\text{card } (\text{bad } i \text{ prefix}) \leq$
 card
(fri-one-step-disagreement-challenges
(len i) (pw i) (layer i prefix) (claimed i prefix)
(domains i prefix))
by (rule *card-mono*) (*simp-all add: local-subset[OF i prefix]*)
also have $\dots \leq \text{len } i \text{ div } 2$
by (rule *fri-one-step-disagreement-challenges-card-bound-local*)
finally show $\text{card } (\text{bad } i \text{ prefix}) \leq \text{len } i \text{ div } 2$.
qed

lemma *generic-fri-sampled-query-pair-fraction-bound-from-one-step-disagreement-subset-exact*:
assumes *cover*:
 $\bigwedge \text{candidate-table roots final-value round-layers layers query-idxs}$
 $\text{challenges}.$
 $\text{query-idxs} \in \text{fri-query-index-list-space} \implies$
generic-fri-sampled-query-candidate-evidence low-degree candidate-bad
candidate-table degree-bound roots challenges final-value query-idxs
round-layers layers $\implies$
challenges $\in$
generic-fri-bad-challenge-lists

```

    (fri-round-count-for-degree-bound degree-bound) bad
and local-subset:
   $\bigwedge i \text{ prefix.}$ 
     $i < \text{fri-round-count-for-degree-bound degree-bound} \implies$ 
     $\text{length prefix} = i \implies$ 
     $\text{bad } i \text{ prefix} \subseteq$ 
     $\text{fri-one-step-disagreement-challenges}$ 
     $(\text{len } i) (\text{pw } i) (\text{layer } i \text{ prefix}) (\text{claimed } i \text{ prefix}) (\text{domains } i \text{ prefix})$ 
shows
  generic-fri-sampled-query-pair-fraction-bound low-degree candidate-bad
  degree-bound
  (fri-one-step-disagreement-round-fraction
    (fri-round-count-for-degree-bound degree-bound) len)
proof (rule
  generic-fri-sampled-query-pair-fraction-bound-from-one-step-disagreement-subset)
fix candidate-table roots final-value round-layers layers query-idxs challenges
assume query-idxs: query-idxs  $\in$  fri-query-index-list-space
and evidence:
  generic-fri-sampled-query-candidate-evidence low-degree candidate-bad
  candidate-table degree-bound roots challenges final-value query-idxs
  round-layers layers
show challenges  $\in$ 
  generic-fri-bad-challenge-lists
  (fri-round-count-for-degree-bound degree-bound) bad
by (rule cover[OF query-idxs evidence])
next
fix i :: nat and prefix :: 'f list
assume i:  $i < \text{fri-round-count-for-degree-bound degree-bound}$ 
and prefix:  $\text{length prefix} = i$ 
show  $\text{bad } i \text{ prefix} \subseteq$ 
  fri-one-step-disagreement-challenges
  (len i) (pw i) (layer i prefix) (claimed i prefix) (domains i prefix)
by (rule local-subset[OF i prefix])
next
show nnreal
  ( $\sum i \in \{..<\text{fri-round-count-for-degree-bound degree-bound}\}.$ 
     $\text{CARD('f)} ^ i * (\text{len } i \text{ div } 2) *$ 
     $\text{CARD('f)} ^$ 
     $(\text{fri-round-count-for-degree-bound degree-bound} - \text{Suc } i)) /$ 
  nnreal ( $\text{CARD('f)} ^ \text{fri-round-count-for-degree-bound degree-bound}$ )
   $\leq \text{fri-one-step-disagreement-round-fraction}$ 
  (fri-round-count-for-degree-bound degree-bound) len
by (rule fri-one-step-disagreement-round-fraction-refl-bound)
qed

lemma trace-fri-sampled-query-pair-fraction-bound-from-bad-challenge-round-bounds:
assumes cover:
   $\bigwedge \text{candidate-table roots final-value round-layers layers query-idxs}$ 
  challenges.

```

$query_idxs \in fri_query_index_list_space \implies$
 $generic_fri_sampled_query_candidate_evidence\ trace_table_low_degree$
 $(Not \circ trace_table_low_degree)\ candidate_table\ (clength - 1)$
 $roots\ challenges\ final_value\ query_idxs\ round_layers\ layers \implies$
 $challenges \in$
 $generic_fri_bad_challenge_lists$
 $(fri_round_count_for_degree_bound\ (clength - 1))\ bad$
and $local_bound:$
 $\bigwedge i\ prefix.$
 $i < fri_round_count_for_degree_bound\ (clength - 1) \implies$
 $length\ prefix = i \implies$
 $card\ (bad\ i\ prefix) \leq bd\ i$
and $fraction_bound:$
 $nnreal$
 $(\sum i \in \{..<fri_round_count_for_degree_bound\ (clength - 1)\}.$
 $CARD('f) \wedge^i * bd\ i *$
 $CARD('f) \wedge$
 $(fri_round_count_for_degree_bound\ (clength - 1) - Suc\ i)) /$
 $nnreal\ (CARD('f) \wedge fri_round_count_for_degree_bound\ (clength - 1))$
 $\leq C$
shows
 $generic_fri_sampled_query_pair_fraction_bound\ trace_table_low_degree$
 $(Not \circ trace_table_low_degree)\ (clength - 1)\ C$
by $(rule$
 $generic_fri_sampled_query_pair_fraction_bound_from_bad_challenge_round_bounds$
 $[OF\ cover\ local_bound\ fraction_bound])$

lemma $generic_fri_sampled_query_pair_fraction_bound_from_full_cover_round_bounds:$

assumes $full_cover:$

$\bigwedge candidate_table\ roots\ final_value\ round_layers\ layers\ query_idxs$
 $challenges.$
 $query_idxs \in fri_query_index_list_space \implies$
 $generic_fri_sampled_query_candidate_evidence\ low_degree\ candidate_bad$
 $candidate_table\ degree_bound\ roots\ challenges\ final_value\ query_idxs$
 $round_layers\ layers \implies$
 $\exists doms.\ generic_fri_sampled_layer_chain_full_cover\ low_degree$
 $candidate_table\ degree_bound\ roots\ challenges\ final_value\ query_idxs$
 $round_layers\ doms\ layers\ bad$

and $local_bound:$

$\bigwedge i\ prefix.$
 $i < fri_round_count_for_degree_bound\ degree_bound \implies$
 $length\ prefix = i \implies$
 $card\ (bad\ i\ prefix) \leq bd\ i$

and $fraction_bound:$

$nnreal$
 $(\sum i \in \{..<fri_round_count_for_degree_bound\ degree_bound\}.$
 $CARD('f) \wedge^i * bd\ i *$
 $CARD('f) \wedge$
 $(fri_round_count_for_degree_bound\ degree_bound - Suc\ i)) /$

$$\text{nnreal } (CARD('f) \wedge \text{fri-round-count-for-degree-bound degree-bound})$$

$$\leq C$$

shows

generic-fri-sampled-query-pair-fraction-bound low-degree candidate-bad degree-bound C

proof (rule

generic-fri-sampled-query-pair-fraction-bound-from-bad-challenge-round-bounds
 $[OF - \text{local-bound fraction-bound}]$)

fix *candidate-table roots final-value round-layers layers query-idxs challenges*

assume *query-space: query-idxs $\in$ fri-query-index-list-space*

and *evidence:*

generic-fri-sampled-query-candidate-evidence low-degree candidate-bad candidate-table degree-bound roots challenges final-value query-idxs round-layers layers

obtain *doms where cover:*

generic-fri-sampled-layer-chain-full-cover low-degree candidate-table degree-bound roots challenges final-value query-idxs round-layers doms layers bad

using *full-cover* $[OF \text{ query-space evidence}]$ **by** *blast*

have *bad:*

challenges $\in$ generic-fri-bad-challenge-lists (length challenges) bad

by (rule *generic-fri-sampled-layer-chain-full-cover-bad-challenge-list*
 $[OF \text{ cover}]$)

have *canonical:*

generic-fri-canonical-sampled-layer-chain-evidence low-degree candidate-table degree-bound roots challenges final-value query-idxs round-layers layers

by (rule *generic-fri-sampled-query-candidate-evidenceD(1)* $[OF \text{ evidence}]$)

have *sampled:*

generic-fri-sampled-layer-chain-evidence low-degree candidate-table degree-bound roots challenges final-value query-idxs round-layers (fri-canonical-domains (length challenges)) layers

by (rule *generic-fri-canonical-sampled-layer-chain-evidenceD*
 $[OF \text{ canonical}]$)

have *partial:*

generic-fri-partial-evidence low-degree candidate-table degree-bound roots challenges final-value query-idxs round-layers

by (rule *generic-fri-sampled-layer-chain-evidenceD(1)* $[OF \text{ sampled}]$)

have *len:*

length challenges = fri-round-count-for-degree-bound degree-bound

using *generic-fri-partial-evidence-shapes(1,2)* $[OF \text{ partial}]$ **by** *simp*

show *challenges $\in$*

generic-fri-bad-challenge-lists
(fri-round-count-for-degree-bound degree-bound) bad

using *bad by (simp add: len)*

qed

lemma *generic-fri-sampled-query-candidate-evidence-full-cover-if-no-failure:*

assumes *evidence*:
generic-fri-sampled-query-candidate-evidence low-degree candidate-bad
candidate-table degree-bound roots challenges final-value query-idxs
round-layers layers
and *no-failure*:
 $\neg$ *generic-fri-sampled-full-cover-failure low-degree candidate-table*
degree-bound roots challenges final-value query-idxs round-layers
(fri-canonical-domains (length challenges)) layers bad
shows
 $\exists$ *doms. generic-fri-sampled-layer-chain-full-cover low-degree*
candidate-table degree-bound roots challenges final-value query-idxs
round-layers doms layers bad
proof –
have *canonical*:
generic-fri-canonical-sampled-layer-chain-evidence low-degree
candidate-table degree-bound roots challenges final-value query-idxs
round-layers layers
by (rule *generic-fri-sampled-query-candidate-evidenceD(1)[OF evidence]*)
have *sampled*:
generic-fri-sampled-layer-chain-evidence low-degree candidate-table
degree-bound roots challenges final-value query-idxs round-layers
(fri-canonical-domains (length challenges)) layers
by (rule *generic-fri-canonical-sampled-layer-chain-evidenceD*
 $[OF\ canonical]$)
show *?thesis*
using *sampled no-failure*
unfolding *generic-fri-sampled-layer-chain-full-cover-def*
generic-fri-sampled-full-cover-failure-def
by *blast*
qed

lemma *generic-fri-sampled-query-pair-fraction-bound-from-no-full-cover-failure-round-bounds*:
assumes *no-failure*:
 $\bigwedge$ *candidate-table roots final-value round-layers layers query-idxs*
challenges.
query-idxs $\in$ fri-query-index-list-space $\implies$
generic-fri-sampled-query-candidate-evidence low-degree candidate-bad
candidate-table degree-bound roots challenges final-value query-idxs
round-layers layers $\implies$
 $\neg$ *generic-fri-sampled-full-cover-failure low-degree candidate-table*
degree-bound roots challenges final-value query-idxs round-layers
(fri-canonical-domains (length challenges)) layers bad
and *local-bound*:
 $\bigwedge$ *i prefix.*
i < fri-round-count-for-degree-bound degree-bound $\implies$
length prefix = i $\implies$
card (bad i prefix) $\leq$ bd i
and *fraction-bound*:
nnreal

$$\begin{aligned}
& (\sum i \in \{..<\text{fri-round-count-for-degree-bound degree-bound}\}. \\
& \quad \text{CARD}('f) \wedge i * \text{bd } i * \\
& \quad \text{CARD}('f) \wedge \\
& \quad (\text{fri-round-count-for-degree-bound degree-bound} - \text{Suc } i)) / \\
& \quad \text{nnreal } (\text{CARD}('f) \wedge \text{fri-round-count-for-degree-bound degree-bound}) \\
& \leq C \\
& \text{shows} \\
& \quad \text{generic-fri-sampled-query-pair-fraction-bound low-degree candidate-bad} \\
& \quad \text{degree-bound } C \\
& \text{proof (rule} \\
& \quad \text{generic-fri-sampled-query-pair-fraction-bound-from-full-cover-round-bounds} \\
& \quad [\text{OF - local-bound fraction-bound}]) \\
& \text{fix candidate-table roots final-value round-layers layers query-idxs} \\
& \quad \text{challenges} \\
& \text{assume query-space: query-idxs} \in \text{fri-query-index-list-space} \\
& \text{and evidence:} \\
& \quad \text{generic-fri-sampled-query-candidate-evidence low-degree} \\
& \quad \text{candidate-bad candidate-table degree-bound roots challenges} \\
& \quad \text{final-value query-idxs round-layers layers} \\
& \text{show } \exists \text{ doms. generic-fri-sampled-layer-chain-full-cover low-degree} \\
& \quad \text{candidate-table degree-bound roots challenges final-value query-idxs} \\
& \quad \text{round-layers doms layers bad} \\
& \text{by (rule generic-fri-sampled-query-candidate-evidence-full-cover-if-no-failure} \\
& \quad [\text{OF evidence no-failure[OF query-space evidence]}]) \\
& \text{qed} \\
& \text{lemma trace-fri-sampled-query-pair-fraction-bound-from-full-cover-round-bounds:} \\
& \text{assumes full-cover:} \\
& \quad \bigwedge \text{candidate-table roots final-value round-layers layers query-idxs} \\
& \quad \text{challenges.} \\
& \quad \text{query-idxs} \in \text{fri-query-index-list-space} \implies \\
& \quad \text{generic-fri-sampled-query-candidate-evidence trace-table-low-degree} \\
& \quad (\text{Not } \circ \text{ trace-table-low-degree}) \text{ candidate-table (clength - 1)} \\
& \quad \text{roots challenges final-value query-idxs round-layers layers} \implies \\
& \quad \exists \text{ doms. generic-fri-sampled-layer-chain-full-cover} \\
& \quad \text{trace-table-low-degree candidate-table (clength - 1) roots} \\
& \quad \text{challenges final-value query-idxs round-layers doms layers bad} \\
& \text{and local-bound:} \\
& \quad \bigwedge i \text{ prefix.} \\
& \quad i < \text{fri-round-count-for-degree-bound (clength - 1)} \implies \\
& \quad \text{length prefix} = i \implies \\
& \quad \text{card (bad } i \text{ prefix)} \leq \text{bd } i \\
& \text{and fraction-bound:} \\
& \quad \text{nnreal} \\
& \quad (\sum i \in \{..<\text{fri-round-count-for-degree-bound (clength - 1)}\}. \\
& \quad \quad \text{CARD}('f) \wedge i * \text{bd } i * \\
& \quad \quad \text{CARD}('f) \wedge \\
& \quad \quad (\text{fri-round-count-for-degree-bound (clength - 1) - Suc } i)) / \\
& \quad \text{nnreal } (\text{CARD}('f) \wedge \text{fri-round-count-for-degree-bound (clength - 1)})
\end{aligned}$$

$$\leq C$$

shows

generic-fri-sampled-query-pair-fraction-bound trace-table-low-degree
 (Not $\circ$ trace-table-low-degree) (clength - 1) C

by (rule

generic-fri-sampled-query-pair-fraction-bound-from-full-cover-round-bounds
 [OF full-cover local-bound fraction-bound])

lemma *trace-fri-sampled-query-pair-fraction-bound-from-no-full-cover-failure-round-bounds:*

assumes *no-failure:*

$\bigwedge$ candidate-table roots final-value round-layers layers query-idxs
 challenges.

query-idxs $\in$ fri-query-index-list-space $\implies$
generic-fri-sampled-query-candidate-evidence trace-table-low-degree

(Not $\circ$ trace-table-low-degree) candidate-table (clength - 1)

roots challenges final-value query-idxs round-layers layers $\implies$

$\neg$ *generic-fri-sampled-full-cover-failure trace-table-low-degree*
 candidate-table (clength - 1) roots challenges final-value
 query-idxs round-layers (fri-canonical-domains (length challenges))
 layers bad

and *local-bound:*

$\bigwedge i$ prefix.

$i < \text{fri-round-count-for-degree-bound (clength - 1)} \implies$

length prefix = $i \implies$

card (bad i prefix) $\leq$ bd i

and *fraction-bound:*

nnreal

$(\sum i \in \{..<\text{fri-round-count-for-degree-bound (clength - 1)}\}).$

CARD('f) $\wedge i * \text{bd } i *$

CARD('f) $\wedge$

(fri-round-count-for-degree-bound (clength - 1) - Suc i)) /

nnreal (CARD('f) $\wedge$ fri-round-count-for-degree-bound (clength - 1))

$$\leq C$$

shows

generic-fri-sampled-query-pair-fraction-bound trace-table-low-degree
 (Not $\circ$ trace-table-low-degree) (clength - 1) C

by (rule

generic-fri-sampled-query-pair-fraction-bound-from-no-full-cover-failure-round-bounds
 [OF no-failure local-bound fraction-bound])

lemma *composition-fri-sampled-query-pair-fraction-bound-from-bad-challenge-round-bounds:*

fixes dg :: 'f

assumes *cover:*

$\bigwedge$ candidate-table roots final-value round-layers layers query-idxs
 challenges.

query-idxs $\in$ fri-query-index-list-space $\implies$

generic-fri-sampled-query-candidate-evidence

(composition-table-low-degree (to-nat dg))

(Not $\circ$ composition-table-low-degree maxDegree) candidate-table

```

    (to-nat dg) roots challenges final-value query-idxs round-layers
    layers  $\implies$ 
    challenges  $\in$ 
      generic-fri-bad-challenge-lists
      (fri-round-count-for-degree-bound (to-nat dg)) (bad dg)
  and local-bound:
     $\bigwedge dg \ i \text{ prefix.}$ 
     $i < \text{fri-round-count-for-degree-bound (to-nat dg)} \implies$ 
     $\text{length prefix} = i \implies$ 
     $\text{card (bad dg i prefix)} \leq \text{bd dg } i$ 
  and fraction-bound:
     $\bigwedge dg.$ 
     $\text{nnreal}$ 
     $(\sum i \in \{..<\text{fri-round-count-for-degree-bound (to-nat dg)}\}.$ 
       $\text{CARD('f)} \wedge i * \text{bd dg } i *$ 
       $\text{CARD('f)} \wedge$ 
       $(\text{fri-round-count-for-degree-bound (to-nat dg)} - \text{Suc } i)) /$ 
       $\text{nnreal (CARD('f)} \wedge \text{fri-round-count-for-degree-bound (to-nat dg))}$ 
       $\leq C \text{ dg}$ 
  shows
    generic-fri-sampled-query-pair-fraction-bound
    (composition-table-low-degree (to-nat dg))
    (Not  $\circ$  composition-table-low-degree maxDegree) (to-nat dg) (C dg)
  proof (rule
    generic-fri-sampled-query-pair-fraction-bound-from-bad-challenge-round-bounds)
  fix candidate-table roots final-value round-layers layers query-idxs
  challenges
  assume query-space: query-idxs  $\in$  fri-query-index-list-space
  and evidence:
    generic-fri-sampled-query-candidate-evidence
    (composition-table-low-degree (to-nat dg))
    (Not  $\circ$  composition-table-low-degree maxDegree) candidate-table
    (to-nat dg) roots challenges final-value query-idxs round-layers
    layers
  show challenges  $\in$ 
    generic-fri-bad-challenge-lists
    (fri-round-count-for-degree-bound (to-nat dg)) (bad dg)
  by (rule cover[OF query-space evidence])
next
  fix i and prefix :: 'f list
  assume i:  $i < \text{fri-round-count-for-degree-bound (to-nat dg)}$ 
  and len:  $\text{length prefix} = i$ 
  show  $\text{card (bad dg i prefix)} \leq \text{bd dg } i$ 
  by (rule local-bound[OF i len])
next
  show  $\text{nnreal}$ 
     $(\sum i \in \{..<\text{fri-round-count-for-degree-bound (to-nat dg)}\}.$ 
       $\text{CARD('f)} \wedge i * \text{bd dg } i *$ 
       $\text{CARD('f)} \wedge$ 

```

$$\frac{(\text{fri-round-count-for-degree-bound } (\text{to-nat } dg) - \text{Suc } i))}{\text{nnreal } (\text{CARD}('f) \wedge \text{fri-round-count-for-degree-bound } (\text{to-nat } dg))}$$

$$\leq C \text{ } dg$$
by (rule fraction-bound)

qed

lemma *composition-fri-sampled-query-pair-fraction-bound-from-full-cover-round-bounds:*

fixes $dg :: 'f$

assumes *full-cover:*

$\wedge$ candidate-table roots final-value round-layers layers query-idxs
challenges.

query-idxs $\in$ fri-query-index-list-space $\implies$

generic-fri-sampled-query-candidate-evidence

(composition-table-low-degree (to-nat dg))

(Not $\circ$ composition-table-low-degree maxDegree) candidate-table

(to-nat dg) roots challenges final-value query-idxs round-layers

layers $\implies$

$\exists$ doms. generic-fri-sampled-layer-chain-full-cover

(composition-table-low-degree (to-nat dg)) candidate-table

(to-nat dg) roots challenges final-value query-idxs round-layers doms

layers (bad dg)

and local-bound:

$\wedge dg$ i prefix.

$i < \text{fri-round-count-for-degree-bound } (\text{to-nat } dg) \implies$

length prefix = i $\implies$

card (bad dg i prefix) $\leq$ bd dg i

and fraction-bound:

$\wedge dg$.

nnreal

$(\sum i \in \{..<\text{fri-round-count-for-degree-bound } (\text{to-nat } dg)\}).$

$\text{CARD}('f) \wedge i * \text{bd } dg \text{ } i *$

$\text{CARD}('f) \wedge$

$(\text{fri-round-count-for-degree-bound } (\text{to-nat } dg) - \text{Suc } i)) /$

$\text{nnreal } (\text{CARD}('f) \wedge \text{fri-round-count-for-degree-bound } (\text{to-nat } dg))$

$\leq C \text{ } dg$

shows

generic-fri-sampled-query-pair-fraction-bound

(composition-table-low-degree (to-nat dg))

(Not $\circ$ composition-table-low-degree maxDegree) (to-nat dg) (C dg)

proof (rule

generic-fri-sampled-query-pair-fraction-bound-from-full-cover-round-bounds)

fix candidate-table roots final-value round-layers layers query-idxs

challenges

assume query-space: query-idxs $\in$ fri-query-index-list-space

and evidence:

generic-fri-sampled-query-candidate-evidence

(composition-table-low-degree (to-nat dg))

(Not $\circ$ composition-table-low-degree maxDegree) candidate-table

(to-nat dg) roots challenges final-value query-idxs round-layers

layers
show $\exists$ doms. *generic-fri-sampled-layer-chain-full-cover*
 (composition-table-low-degree (to-nat dg)) candidate-table
 (to-nat dg) roots challenges final-value query-idxs round-layers doms
 layers (bad dg)
 by (rule full-cover[OF query-space evidence])
next
fix *i* and *prefix* :: 'f list
assume *i*: *i* < fri-round-count-for-degree-bound (to-nat dg)
 and *len*: length *prefix* = *i*
show card (bad dg *i* *prefix*) $\leq$ bd dg *i*
 by (rule local-bound[OF *i len*])
next
show *nnreal*
 ($\sum i \in \{..<\text{fri-round-count-for-degree-bound (to-nat dg)}\}$.
 CARD('f) $\wedge$ *i* * bd dg *i* *
 CARD('f) $\wedge$
 (fri-round-count-for-degree-bound (to-nat dg) - Suc *i*)) /
nnreal (CARD('f) $\wedge$ fri-round-count-for-degree-bound (to-nat dg))
 $\leq$ *C dg*
 by (rule fraction-bound)
qed

lemma *composition-fri-sampled-query-pair-fraction-bound-from-no-full-cover-failure-round-bounds*:

fixes dg :: 'f
assumes *no-failure*:
 $\bigwedge$ candidate-table roots final-value round-layers layers query-idxs
 challenges.
 query-idxs $\in$ fri-query-index-list-space $\implies$
 generic-fri-sampled-query-candidate-evidence
 (composition-table-low-degree (to-nat dg))
 (Not $\circ$ composition-table-low-degree maxDegree) candidate-table
 (to-nat dg) roots challenges final-value query-idxs round-layers
 layers $\implies$
 $\neg$ generic-fri-sampled-full-cover-failure
 (composition-table-low-degree (to-nat dg)) candidate-table
 (to-nat dg) roots challenges final-value query-idxs round-layers
 (fri-canonical-domains (length challenges)) layers (bad dg)
and *local-bound*:
 $\bigwedge$ dg *i* *prefix*.
i < fri-round-count-for-degree-bound (to-nat dg) $\implies$
 length *prefix* = *i* $\implies$
 card (bad dg *i* *prefix*) $\leq$ bd dg *i*
and *fraction-bound*:
 $\bigwedge$ dg.
nnreal
 ($\sum i \in \{..<\text{fri-round-count-for-degree-bound (to-nat dg)}\}$.
 CARD('f) $\wedge$ *i* * bd dg *i* *
 CARD('f) $\wedge$

$$\frac{(\text{fri-round-count-for-degree-bound } (to\text{-nat } dg) - \text{Suc } i)) /}{nnreal (CARD('f) \wedge \text{fri-round-count-for-degree-bound } (to\text{-nat } dg))} \leq C \text{ } dg$$

shows

generic-fri-sampled-query-pair-fraction-bound
(composition-table-low-degree (to-nat dg))
(Not o composition-table-low-degree maxDegree) (to-nat dg) (C dg)

proof (rule

generic-fri-sampled-query-pair-fraction-bound-from-no-full-cover-failure-round-bounds)

fix *candidate-table roots final-value round-layers layers query-idxs*
challenges

assume *query-space: query-idxs ∈ fri-query-index-list-space*

and *evidence:*

generic-fri-sampled-query-candidate-evidence
(composition-table-low-degree (to-nat dg))
(Not o composition-table-low-degree maxDegree) candidate-table
(to-nat dg) roots challenges final-value query-idxs round-layers
layers

show $\neg$ *generic-fri-sampled-full-cover-failure*
(composition-table-low-degree (to-nat dg)) candidate-table
(to-nat dg) roots challenges final-value query-idxs round-layers
(fri-canonical-domains (length challenges)) layers (bad dg)

by (rule *no-failure[OF query-space evidence]*)

next

fix *i* **and** *prefix :: 'f list*

assume *i: i < fri-round-count-for-degree-bound (to-nat dg)*

and *len: length prefix = i*

show *card (bad dg i prefix) ≤ bd dg i*

by (rule *local-bound[OF i len]*)

next

show *nnreal*

$$\left(\sum i \in \{..<\text{fri-round-count-for-degree-bound } (to\text{-nat } dg)\}. \right.$$

$$CARD('f) \wedge i * bd \text{ } dg \text{ } i *$$

$$CARD('f) \wedge$$

$$\left. (\text{fri-round-count-for-degree-bound } (to\text{-nat } dg) - \text{Suc } i)) / \right.$$

$$nnreal (CARD('f) \wedge \text{fri-round-count-for-degree-bound } (to\text{-nat } dg))$$

$$\leq C \text{ } dg$$

by (rule *fraction-bound*)

qed

lemma *wp-trace-fri-sampled-query-bad-candidate-bound-from-bad-challenge-round-bounds:*

assumes *future: trace-fri-future-fresh s*

and *cover:*

$\bigwedge$ *candidate-table roots final-value round-layers layers query-idxs*
challenges.
query-idxs ∈ fri-query-index-list-space $\implies$
generic-fri-sampled-query-candidate-evidence trace-table-low-degree
(Not o trace-table-low-degree) candidate-table (clength - 1)
roots challenges final-value query-idxs round-layers layers $\implies$

challenges $\in$
generic-fri-bad-challenge-lists
(fri-round-count-for-degree-bound (clength - 1)) bad
and *local-bound*:
 $\bigwedge i$ *prefix*.
 $i < \text{fri-round-count-for-degree-bound (clength - 1)} \implies$
 $\text{length prefix} = i \implies$
 $\text{card (bad i prefix)} \leq \text{bd i}$
and *fraction-bound*:
 nnreal
 $(\sum i \in \{..<\text{fri-round-count-for-degree-bound (clength - 1)}\}.$
 $\text{CARD('f)} \wedge i * \text{bd i} *$
 $\text{CARD('f)} \wedge$
 $(\text{fri-round-count-for-degree-bound (clength - 1)} - \text{Suc i})) /$
 $\text{nnreal (CARD('f)} \wedge \text{fri-round-count-for-degree-bound (clength - 1))}$
 $\leq C$
shows
 $\text{wp-event verify-monad (trace-fri-sampled-query-bad-candidate s) s}$
 $\leq C$
proof (*rule*
 $\text{wp-trace-fri-sampled-query-bad-candidate-bound-from-generic-projection-fraction}$
 $[\text{OF future}]$)
show
 $\text{generic-fri-sampled-query-projection-fraction-bound}$
 $\text{trace-table-low-degree (Not } \circ \text{ trace-table-low-degree)}$
 $(\text{clength} - 1) C$
by (*rule*
 $\text{generic-fri-sampled-query-projection-fraction-bound-from-bad-challenge-round-bounds}$
 $[\text{OF cover local-bound fraction-bound}]$)
qed

lemma *wp-trace-fri-sampled-query-bad-candidate-bound-from-one-step-disagreement-subset*:
assumes *future: trace-fri-future-fresh s*
and *cover*:
 $\bigwedge \text{candidate-table roots final-value round-layers layers query-idxs}$
 challenges.
 $\text{query-idxs} \in \text{fri-query-index-list-space} \implies$
 $\text{generic-fri-sampled-query-candidate-evidence trace-table-low-degree}$
 $(\text{Not } \circ \text{ trace-table-low-degree}) \text{ candidate-table (clength - 1)}$
 $\text{roots challenges final-value query-idxs round-layers layers} \implies$
 $\text{challenges} \in$
 $\text{generic-fri-bad-challenge-lists}$
 $(\text{fri-round-count-for-degree-bound (clength - 1)) bad}$
and *local-subset*:
 $\bigwedge i$ *prefix*.
 $i < \text{fri-round-count-for-degree-bound (clength - 1)} \implies$
 $\text{length prefix} = i \implies$
 $\text{bad i prefix} \subseteq$
 $\text{fri-one-step-disagreement-challenges}$

$(\text{len } i) (\text{pw } i) (\text{layer } i \text{ prefix}) (\text{claimed } i \text{ prefix}) (\text{domains } i \text{ prefix})$
and *fraction-bound*:
 nnreal
 $(\sum i \in \{..<\text{fri-round-count-for-degree-bound } (\text{clength} - 1)\}.$
 $\text{CARD}(f) \wedge i * (\text{len } i \text{ div } 2) *$
 $\text{CARD}(f) \wedge$
 $(\text{fri-round-count-for-degree-bound } (\text{clength} - 1) - \text{Suc } i)) /$
 $\text{nnreal } (\text{CARD}(f) \wedge \text{fri-round-count-for-degree-bound } (\text{clength} - 1))$
 $\leq C$
shows
 $\text{wp-event verify-monad } (\text{trace-fri-sampled-query-bad-candidate } s) s$
 $\leq C$
proof (*rule*
 $\text{wp-trace-fri-sampled-query-bad-candidate-bound-from-generic-projection-fraction}$
 $[\text{OF future}]$)
show
 $\text{generic-fri-sampled-query-projection-fraction-bound}$
 $\text{trace-table-low-degree } (\text{Not } \circ \text{trace-table-low-degree})$
 $(\text{clength} - 1) C$
by (*rule*
 $\text{generic-fri-sampled-query-projection-fraction-bound-from-one-step-disagreement-subset}$
 $[\text{OF cover local-subset fraction-bound}]$)
qed

lemma *wp-trace-fri-sampled-query-bad-candidate-bound-from-one-step-disagreement-subset-exact*:
assumes *future: trace-fri-future-fresh s*
and *cover*:
 $\bigwedge \text{candidate-table roots final-value round-layers layers query-idxs}$
 challenges.
 $\text{query-idxs} \in \text{fri-query-index-list-space} \implies$
 $\text{generic-fri-sampled-query-candidate-evidence trace-table-low-degree}$
 $(\text{Not } \circ \text{trace-table-low-degree}) \text{ candidate-table } (\text{clength} - 1)$
 $\text{roots challenges final-value query-idxs round-layers layers} \implies$
 $\text{challenges} \in$
 $\text{generic-fri-bad-challenge-lists}$
 $(\text{fri-round-count-for-degree-bound } (\text{clength} - 1)) \text{ bad}$
and *local-subset*:
 $\bigwedge i \text{ prefix.}$
 $i < \text{fri-round-count-for-degree-bound } (\text{clength} - 1) \implies$
 $\text{length prefix} = i \implies$
 $\text{bad } i \text{ prefix} \subseteq$
 $\text{fri-one-step-disagreement-challenges}$
 $(\text{len } i) (\text{pw } i) (\text{layer } i \text{ prefix}) (\text{claimed } i \text{ prefix}) (\text{domains } i \text{ prefix})$
shows
 $\text{wp-event verify-monad } (\text{trace-fri-sampled-query-bad-candidate } s) s$
 $\leq$
 $\text{fri-one-step-disagreement-round-fraction}$
 $(\text{fri-round-count-for-degree-bound } (\text{clength} - 1)) \text{ len}$
proof (*rule* *wp-trace-fri-sampled-query-bad-candidate-bound-from-one-step-disagreement-subset*)


```

show trace-fri-future-fresh s
  by (rule future)
next
fix candidate-table roots final-value round-layers layers query-idxs challenges
assume query-idxs: query-idxs  $\in$  fri-query-index-list-space
and evidence:
  generic-fri-sampled-query-candidate-evidence trace-table-low-degree
    (Not  $\circ$  trace-table-low-degree) candidate-table (clength  $- 1$ )
    roots challenges final-value query-idxs round-layers layers
show challenges  $\in$ 
  generic-fri-bad-challenge-lists
    (fri-round-count-for-degree-bound (clength  $- 1$ )) bad
by (rule cover[OF query-idxs evidence])
next
fix i :: nat and prefix :: 'f list
assume i: i < fri-round-count-for-degree-bound (clength  $- 1$ )
and prefix: length prefix = i
show bad i prefix  $\subseteq$ 
  fri-one-step-disagreement-challenges
    (len i) (pw i) (layer i prefix) (claimed i prefix) (domains i prefix)
by (rule local-subset[OF i prefix])
next
show nnreal
  ( $\sum i \in \{.. < \text{fri-round-count-for-degree-bound } (\text{clength} - 1)\}$ .
    CARD('f)  $\wedge$  i * (len i div 2) *
    CARD('f)  $\wedge$ 
      (fri-round-count-for-degree-bound (clength  $- 1$ )  $-$  Suc i) /
    nnreal (CARD('f)  $\wedge$  fri-round-count-for-degree-bound (clength  $- 1$ ))
   $\leq$  fri-one-step-disagreement-round-fraction
    (fri-round-count-for-degree-bound (clength  $- 1$ )) len
by (rule fri-one-step-disagreement-round-fraction-refl-bound)
qed

lemma wp-trace-fri-sampled-query-bad-candidate-bound-from-canonical-one-step-disagreement-subset:
assumes future: trace-fri-future-fresh s
and cover:
   $\bigwedge$  candidate-table roots final-value round-layers layers query-idxs
    challenges.
    query-idxs  $\in$  fri-query-index-list-space  $\implies$ 
    generic-fri-sampled-query-candidate-evidence trace-table-low-degree
      (Not  $\circ$  trace-table-low-degree) candidate-table (clength  $- 1$ )
      roots challenges final-value query-idxs round-layers layers  $\implies$ 
    challenges  $\in$ 
      generic-fri-bad-challenge-lists
        (fri-round-count-for-degree-bound (clength  $- 1$ )) bad
and local-subset:
   $\bigwedge$  i prefix.
    i < fri-round-count-for-degree-bound (clength  $- 1$ )  $\implies$ 
    length prefix = i  $\implies$ 

```

$bad\ i\ prefix \subseteq$
 $fri-one-step-disagreement-challenges$
 $(trace-fri-canonical-layer-len\ i)\ (pw\ i)$
 $(layer\ i\ prefix)\ (claimed\ i\ prefix)\ (domains\ i\ prefix)$
shows
 $wp-event\ verify-monad\ (trace-fri-sampled-query-bad-candidate\ s)\ s$
 $\leq\ concrete-trace-fri-bad-challenge-fraction$
using
 $wp-trace-fri-sampled-query-bad-candidate-bound-from-one-step-disagreement-subset-exact$
 $[OF\ future\ cover\ local-subset]$
by (*simp add: concrete-trace-fri-bad-challenge-fraction-def*)

lemma *wp-composition-fri-sampled-query-bad-candidate-bound-from-bad-challenge-round-bounds:*
assumes *future: composition-fri-future-fresh s*
and *cover:*
 $\bigwedge dg\ candidate-table\ roots\ final-value\ round-layers\ layers\ query-idxs$
 $challenges.$
 $query-idxs \in fri-query-index-list-space \implies$
 $generic-fri-sampled-query-candidate-evidence$
 $(composition-table-low-degree\ (to-nat\ dg))$
 $(Not \circ composition-table-low-degree\ maxDegree)\ candidate-table$
 $(to-nat\ dg)\ roots\ challenges\ final-value\ query-idxs\ round-layers$
 $layers \implies$
 $challenges \in$
 $generic-fri-bad-challenge-lists$
 $(fri-round-count-for-degree-bound\ (to-nat\ dg))\ (bad\ dg)$
and *local-bound:*
 $\bigwedge dg\ i\ prefix.$
 $i < fri-round-count-for-degree-bound\ (to-nat\ dg) \implies$
 $length\ prefix = i \implies$
 $card\ (bad\ dg\ i\ prefix) \leq bd\ dg\ i$
and *fraction-bound:*
 $\bigwedge dg.$
 $nnreal$
 $(\sum i \in \{..<fri-round-count-for-degree-bound\ (to-nat\ dg)\}).$
 $CARD('f) \wedge i * bd\ dg\ i *$
 $CARD('f) \wedge$
 $(fri-round-count-for-degree-bound\ (to-nat\ dg) - Suc\ i)) /$
 $nnreal\ (CARD('f) \wedge fri-round-count-for-degree-bound\ (to-nat\ dg))$
 $\leq C$
shows
 $wp-event\ verify-monad\ (composition-fri-sampled-query-bad-candidate\ s)\ s$
 $\leq C$
proof (*rule*
 $wp-composition-fri-sampled-query-bad-candidate-bound-from-generic-projection-fraction$
 $[OF\ future])$
fix $dg :: 'f$
show
 $generic-fri-sampled-query-projection-fraction-bound$

```

    (composition-table-low-degree (to-nat dg))
    (Not ◦ composition-table-low-degree maxDegree)
    (to-nat dg) C
proof (rule
  generic-fri-sampled-query-projection-fraction-bound-from-bad-challenge-round-bounds)
fix candidate-table roots final-value round-layers layers query-idxs
  challenges
assume query-space: query-idxs ∈ fri-query-index-list-space
and evidence:
  generic-fri-sampled-query-candidate-evidence
  (composition-table-low-degree (to-nat dg))
  (Not ◦ composition-table-low-degree maxDegree) candidate-table
  (to-nat dg) roots challenges final-value query-idxs round-layers
  layers
show challenges ∈
  generic-fri-bad-challenge-lists
  (fri-round-count-for-degree-bound (to-nat dg)) (bad dg)
by (rule cover[OF query-space evidence])
next
fix i and prefix :: 'f list
assume i: i < fri-round-count-for-degree-bound (to-nat dg)
and len: length prefix = i
show card (bad dg i prefix) ≤ bd dg i
by (rule local-bound[OF i len])
next
show nreal
  (∑ i ∈ {.. $\text{fri-round-count-for-degree-bound (to-nat dg)}$ }.
    CARD('f) ^ i * bd dg i *
    CARD('f) ^
      (fri-round-count-for-degree-bound (to-nat dg) - Suc i)) /
  nreal (CARD('f) ^ fri-round-count-for-degree-bound (to-nat dg))
  ≤ C
by (rule fraction-bound)
qed
qed

lemma wp-composition-fri-sampled-query-bad-candidate-bound-from-one-step-disagreement-subset:
assumes future: composition-fri-future-fresh s
and cover:
  ∧ dg candidate-table roots final-value round-layers layers query-idxs
  challenges.
  query-idxs ∈ fri-query-index-list-space ⇒
  generic-fri-sampled-query-candidate-evidence
  (composition-table-low-degree (to-nat dg))
  (Not ◦ composition-table-low-degree maxDegree) candidate-table
  (to-nat dg) roots challenges final-value query-idxs round-layers
  layers ⇒
  challenges ∈
  generic-fri-bad-challenge-lists

```

```

    (fri-round-count-for-degree-bound (to-nat dg)) (bad dg)
and local-subset:
   $\bigwedge dg \ i \ prefix.$ 
     $i < \text{fri-round-count-for-degree-bound } (to\text{-nat } dg) \implies$ 
     $length \ prefix = i \implies$ 
     $bad \ dg \ i \ prefix \subseteq$ 
     $\text{fri-one-step-disagreement-challenges}$ 
     $(len \ dg \ i) \ (pw \ dg \ i) \ (layer \ dg \ i \ prefix)$ 
     $(claimed \ dg \ i \ prefix) \ (domains \ dg \ i \ prefix)$ 
and fraction-bound:
   $\bigwedge dg.$ 
     $nnreal$ 
     $(\sum i \in \{..<\text{fri-round-count-for-degree-bound } (to\text{-nat } dg)\}.$ 
     $CARD('f) \wedge i * (len \ dg \ i \ div \ 2) *$ 
     $CARD('f) \wedge$ 
     $(\text{fri-round-count-for-degree-bound } (to\text{-nat } dg) - Suc \ i)) /$ 
     $nnreal \ (CARD('f) \wedge \text{fri-round-count-for-degree-bound } (to\text{-nat } dg))$ 
     $\leq C$ 
shows
   $wp\text{-event } verify\text{-monad } (composition\text{-fri-sampled-query-bad-candidate } s) \ s$ 
   $\leq C$ 
proof (rule
   $wp\text{-composition-fri-sampled-query-bad-candidate-bound-from-generic-projection-fraction}$ 
  [OF future])
fix dg :: 'f
show
   $generic\text{-fri-sampled-query-projection-fraction-bound}$ 
   $(composition\text{-table-low-degree } (to\text{-nat } dg))$ 
   $(Not \circ composition\text{-table-low-degree } maxDegree)$ 
   $(to\text{-nat } dg) \ C$ 
proof (rule
   $generic\text{-fri-sampled-query-projection-fraction-bound-from-one-step-disagreement-subset}$ )
fix candidate-table roots final-value round-layers layers query-idxs
  challenges
assume query-space: query-idxs  $\in$  fri-query-index-list-space
and evidence:
   $generic\text{-fri-sampled-query-candidate-evidence}$ 
   $(composition\text{-table-low-degree } (to\text{-nat } dg))$ 
   $(Not \circ composition\text{-table-low-degree } maxDegree) \ candidate\text{-table}$ 
   $(to\text{-nat } dg) \ roots \ challenges \ final\text{-value} \ query\text{-idxs} \ round\text{-layers}$ 
  layers
show challenges  $\in$ 
   $generic\text{-fri-bad-challenge-lists}$ 
   $(\text{fri-round-count-for-degree-bound } (to\text{-nat } dg)) \ (bad \ dg)$ 
by (rule cover[OF query-space evidence])
next
fix i and prefix :: 'f list
assume i:  $i < \text{fri-round-count-for-degree-bound } (to\text{-nat } dg)$ 
and prefix:  $length \ prefix = i$ 

```

```

show bad dg i prefix  $\subseteq$ 
  fri-one-step-disagreement-challenges
    (len dg i) (pw dg i) (layer dg i prefix)
    (claimed dg i prefix) (domains dg i prefix)
  by (rule local-subset[OF i prefix])
next
show nnreal
  ( $\sum i \in \{..<\text{fri-round-count-for-degree-bound } (to\text{-nat } dg)\}$ ).
   $CARD('f) \wedge i * (len\ dg\ i\ div\ 2) * CARD('f) \wedge$ 
    ( $\text{fri-round-count-for-degree-bound } (to\text{-nat } dg) - Suc\ i$ ) /
  nnreal ( $CARD('f) \wedge \text{fri-round-count-for-degree-bound } (to\text{-nat } dg)$ )
 $\leq C$ 
  by (rule fraction-bound)
qed
qed

lemma wp-composition-fri-sampled-query-bad-candidate-bound-from-one-step-disagreement-subset-exact-bound:
assumes future: composition-fri-future-fresh s
and cover:
 $\bigwedge dg$  candidate-table roots final-value round-layers layers query-idxs
  challenges.
  query-idxs  $\in$  fri-query-index-list-space  $\implies$ 
generic-fri-sampled-query-candidate-evidence
  (composition-table-low-degree (to-nat dg))
  (Not  $\circ$  composition-table-low-degree maxDegree) candidate-table
  (to-nat dg) roots challenges final-value query-idxs round-layers
  layers  $\implies$ 
challenges  $\in$ 
  generic-fri-bad-challenge-lists
  ( $\text{fri-round-count-for-degree-bound } (to\text{-nat } dg)$ ) (bad dg)
and local-subset:
 $\bigwedge dg$  i prefix.
   $i < \text{fri-round-count-for-degree-bound } (to\text{-nat } dg) \implies$ 
   $length\ prefix = i \implies$ 
  bad dg i prefix  $\subseteq$ 
  fri-one-step-disagreement-challenges
    (len dg i) (pw dg i) (layer dg i prefix)
    (claimed dg i prefix) (domains dg i prefix)
and fraction-bound:
 $\bigwedge dg.$ 
  fri-one-step-disagreement-round-fraction
  ( $\text{fri-round-count-for-degree-bound } (to\text{-nat } dg)$ ) (len dg)
 $\leq C$ 
shows
  wp-event verify-monad (composition-fri-sampled-query-bad-candidate s) s
 $\leq C$ 
proof (rule
  wp-composition-fri-sampled-query-bad-candidate-bound-from-one-step-disagreement-subset

```

```

[OF future cover local-subset])
fix dg
show nreal
  (∑ i ∈ {..fri-round-count-for-degree-bound (to-nat dg)}.
    CARD('f) ^ i * (len dg i div 2) *
    CARD('f) ^
      (fri-round-count-for-degree-bound (to-nat dg) - Suc i)) /
  nreal (CARD('f) ^ fri-round-count-for-degree-bound (to-nat dg))
  ≤ C
using fraction-bound[of dg]
by (simp add: fri-one-step-disagreement-round-fraction-def
  fri-bad-challenge-round-fraction-def
  fri-one-step-disagreement-round-mass-def
  fri-bad-challenge-round-mass-def)
qed

lemma wp-composition-fri-sampled-query-bad-candidate-bound-from-canonical-one-step-disagreement-subset:
  assumes future: composition-fri-future-fresh s
  and cover:
    ∧ dg candidate-table roots final-value round-layers layers query-idxs
      challenges.
    query-idxs ∈ fri-query-index-list-space ⇒
    generic-fri-sampled-query-candidate-evidence
      (composition-table-low-degree (to-nat dg))
      (Not ∘ composition-table-low-degree maxDegree) candidate-table
      (to-nat dg) roots challenges final-value query-idxs round-layers
      layers ⇒
    challenges ∈
      generic-fri-bad-challenge-lists
      (fri-round-count-for-degree-bound (to-nat dg)) (bad dg)
  and local-subset:
    ∧ dg i prefix.
    i < fri-round-count-for-degree-bound (to-nat dg) ⇒
    length prefix = i ⇒
    bad dg i prefix ⊆
      fri-one-step-disagreement-challenges
      (composition-fri-canonical-layer-len dg i) (pw dg i)
      (layer dg i prefix) (claimed dg i prefix) (domains dg i prefix)
  shows
    wp-event verify-monad (composition-fri-sampled-query-bad-candidate s) s
    ≤ concrete-composition-fri-bad-challenge-fraction
proof (rule
  wp-composition-fri-sampled-query-bad-candidate-bound-from-one-step-disagreement-subset-exact-bound)
show composition-fri-future-fresh s
  by (rule future)
next
fix dg candidate-table roots final-value round-layers layers query-idxs
  challenges
assume query-idxs: query-idxs ∈ fri-query-index-list-space

```

```

and evidence:
  generic-fri-sampled-query-candidate-evidence
    (composition-table-low-degree (to-nat dg))
    (Not  $\circ$  composition-table-low-degree maxDegree) candidate-table
    (to-nat dg) roots challenges final-value query-idxs round-layers
    layers
show challenges  $\in$ 
  generic-fri-bad-challenge-lists
    (fri-round-count-for-degree-bound (to-nat dg)) (bad dg)
by (rule cover[OF query-idxs evidence])
next
  fix dg and i :: nat and prefix :: 'f list
  assume i: i < fri-round-count-for-degree-bound (to-nat dg)
    and prefix: length prefix = i
  show bad dg i prefix  $\subseteq$ 
    fri-one-step-disagreement-challenges
      (composition-fri-canonical-layer-len dg i) (pw dg i)
      (layer dg i prefix) (claimed dg i prefix) (domains dg i prefix)
    by (rule local-subset[OF i prefix])
next
  fix dg
  show fri-one-step-disagreement-round-fraction
    (fri-round-count-for-degree-bound (to-nat dg))
    (composition-fri-canonical-layer-len dg)
     $\leq$  concrete-composition-fri-bad-challenge-fraction
  by (rule concrete-composition-fri-bad-challenge-fraction-ge)
qed

end

end

```

```

theory Soundness-FRI-Zero-Round-Query-List-Bounds
imports
  Soundness-FRI-Zero-Round-Target
  Soundness-FRI-Query-List-Exact-Product
begin

```

Exact staged query-list bounds for fixed zero-round FRI targets.

The target table and final value must already be fixed before query sampling. This theory deliberately does not close the remaining candidate/verifier alignment gap; it only packages the exact-product query accounting for the fixed targets isolated in *Soundness-FRI-Zero-Round-Target*.

```

context soundness
begin

```

```

definition fri-zero-round-first-query-index-lists
  :: 'f list  $\Rightarrow$  'f  $\Rightarrow$  nat list set

```

where

fri-zero-round-first-query-index-lists table final =
 $\{ \text{query-idxs} \in \text{fri-query-index-list-space} .$
 $\text{query-idxs} ! 0 \in \text{fri-final-value-agreement-set table final} \}$

lemma *fri-zero-round-first-query-index-lists-subset*:

fri-zero-round-first-query-index-lists table final
 $\subseteq \text{fri-query-index-list-space}$
unfolding *fri-zero-round-first-query-index-lists-def* **by** *blast*

lemma *finite-fri-zero-round-first-query-index-lists[simp]*:

finite (fri-zero-round-first-query-index-lists table final)
by (*rule finite-subset*
 $[OF \text{ fri-zero-round-first-query-index-lists-subset}]$)
 $(\text{rule finite-fri-query-index-list-space})$

lemma *fri-zero-round-first-query-index-listsD*:

assumes *query-idxs* $\in$
fri-zero-round-first-query-index-lists table final
shows *query-idxs* $\in \text{fri-query-index-list-space}$
and *query-idxs* ! 0 $\in \text{fri-final-value-agreement-set table final}$
using *assms*
unfolding *fri-zero-round-first-query-index-lists-def* **by** *blast+*

lemma *fri-zero-round-first-query-index-lists-card-le*:

card (fri-zero-round-first-query-index-lists table final) $\leq$
card (fri-final-value-agreement-set table final) *
card query-sample-space $\wedge (\text{rounds} - 1)$

proof –

let *?T* = *fri-zero-round-first-query-index-lists table final*

let *?Tail* =

$\{ xs. \text{set } xs \subseteq \text{query-sample-space} \wedge$
 $\text{length } xs = \text{rounds} - 1 \}$

let *?F* = $\lambda \text{query-idxs}. (\text{query-idxs} ! 0, \text{tl } \text{query-idxs})$

have *image-subset*:

?F ‘ *?T* $\subseteq$
 $\text{fri-final-value-agreement-set table final} \times \text{?Tail}$

proof

fix *y*

assume *y* $\in \text{?F ‘ ?T}$

then obtain *query-idxs* **where** *query-idxs-in*: *query-idxs* $\in \text{?T}$

and *y-def*: *y* = *?F query-idxs*

by *blast*

have *query-space*: *query-idxs* $\in \text{fri-query-index-list-space}$

and *first-in*:

query-idxs ! 0 $\in \text{fri-final-value-agreement-set table final}$

using *fri-zero-round-first-query-index-listsD*[*OF query-idxs-in*]

by *blast+*

have *len*: *length query-idxs* = *rounds*


```

    and set-subset: set query-idxs  $\subseteq$  query-sample-space
    using query-space unfolding fri-query-index-list-space-def by auto
have tail-len: length (tl query-idxs) = rounds - 1
    using len rounds-positive by (cases query-idxs) simp-all
have tail-set: set (tl query-idxs)  $\subseteq$  set query-idxs
    by (cases query-idxs) auto
have tail-subset: set (tl query-idxs)  $\subseteq$  query-sample-space
    using tail-set set-subset by blast
show  $y \in \text{fri-final-value-agreement-set table final} \times ?\text{Tail}$ 
    using y-def first-in tail-len tail-subset by simp
qed
have inj: inj-on  $?F ?T$ 
proof (rule inj-onI)
  fix xs ys
  assume xs-in:  $xs \in ?T$ 
    and ys-in:  $ys \in ?T$ 
    and eq:  $?F xs = ?F ys$ 
  have xs-len: length xs = rounds
    and ys-len: length ys = rounds
    using fri-zero-round-first-query-index-listsD(1)[OF xs-in]
      fri-zero-round-first-query-index-listsD(1)[OF ys-in]
    unfolding fri-query-index-list-space-def by auto
  have xs-ne:  $xs \neq []$ 
    and ys-ne:  $ys \neq []$ 
    using xs-len ys-len rounds-positive by auto
  have head-eq:  $xs ! 0 = ys ! 0$ 
    using eq by simp
  have tail-eq:  $tl xs = tl ys$ 
    using eq by simp
  from xs-ne obtain  $x xs'$  where xs-def:  $xs = x \# xs'$ 
    by (cases xs) auto
  from ys-ne obtain  $y ys'$  where ys-def:  $ys = y \# ys'$ 
    by (cases ys) auto
  show  $xs = ys$ 
    using head-eq tail-eq unfolding xs-def ys-def by simp
qed
have card  $?T = \text{card } (?F ' ?T)$ 
  using inj by (rule card-image[symmetric])
also have ...  $\leq$ 
  card (fri-final-value-agreement-set table final  $\times$   $?Tail$ )
proof (rule card-mono)
  show finite (fri-final-value-agreement-set table final  $\times$   $?Tail$ )
    by (intro finite-cartesian-product finite-fri-final-value-agreement-set
      finite-lists-length-eq[OF finite-query-sample-space])
  show  $?F ' ?T \subseteq \text{fri-final-value-agreement-set table final} \times ?Tail$ 
    by (rule image-subset)
qed
also have ... =
  card (fri-final-value-agreement-set table final) * card  $?Tail$ 

```

by (simp add: card-cartesian-product)
 also have card ?Tail = card query-sample-space $^{\wedge}$ (rounds - 1)
 by (rule card-lists-length-eq[OF finite-query-sample-space])
 finally show ?thesis .
 qed

lemma fri-zero-round-first-query-index-lists-card-bound-from-small:

assumes small:

card (fri-final-value-agreement-set table final) <
 query-sample-space-size

shows card (fri-zero-round-first-query-index-lists table final) $\leq$
 (query-sample-space-size - 1) *
 query-sample-space-size $^{\wedge}$ (rounds - 1)

proof -

have agreement-le:

card (fri-final-value-agreement-set table final) $\leq$
 query-sample-space-size - 1

using small **by** linarith

have card (fri-zero-round-first-query-index-lists table final) $\leq$
 card (fri-final-value-agreement-set table final) *
 card query-sample-space $^{\wedge}$ (rounds - 1)

by (rule fri-zero-round-first-query-index-lists-card-le)

also have ... $\leq$
 (query-sample-space-size - 1) *
 query-sample-space-size $^{\wedge}$ (rounds - 1)

using agreement-le **by** simp

finally show ?thesis .

qed

definition trace-fri-zero-round-query-list-target-hit

:: (f, 'a) protocol-channel-scheme $\Rightarrow$
 (unit list $\times$ (f, 'a) protocol-channel-scheme) option $\Rightarrow$ bool

where

trace-fri-zero-round-query-list-target-hit s out $\longleftrightarrow$
 ($\exists$ trace-table trace-final.
 trace-fri-query-index-list-set-hit s
 (fri-zero-round-first-query-index-lists trace-table trace-final)
 out $\wedge$
 card (fri-final-value-agreement-set trace-table trace-final) <
 query-sample-space-size)

lemma trace-fri-zero-round-query-list-target-hitE:

assumes trace-fri-zero-round-query-list-target-hit s out

obtains trace-table trace-final **where**

trace-fri-query-index-list-set-hit s
 (fri-zero-round-first-query-index-lists trace-table trace-final)
 out

card (fri-final-value-agreement-set trace-table trace-final) <
 query-sample-space-size

```

using assms
unfolding trace-fri-zero-round-query-list-target-hit-def by blast

lemma trace-fri-zero-round-checked-first-query-target-hit-imp-query-list-target:
  assumes trace-fri-zero-round-checked-first-query-target-hit s out
  shows trace-fri-zero-round-query-list-target-hit s out
proof –
  from assms obtain trace-roots trace-bs trace-final dg composition-roots
    composition-bs composition-final fri-query-idxs trace-round-layers
    composition-round-layers fr trace-openings trace-table
  where fri-openings:
    accepted-fri-opening-transcript s out trace-roots trace-bs
    trace-final dg composition-roots composition-bs composition-final
    fri-query-idxs trace-round-layers composition-round-layers
  and first-hit:
    fri-query-idxs ! 0 ∈
    fri-final-value-agreement-set trace-table trace-final
  and card-lt:
    card (fri-final-value-agreement-set trace-table trace-final) <
    query-sample-space-size
  unfolding trace-fri-zero-round-checked-first-query-target-hit-def
  by blast
  have query-space:
    fri-query-idxs ∈ fri-query-index-list-space
  by (rule accepted-fri-opening-transcript-query-index-list-space
    [OF fri-openings])
  have query-target:
    fri-query-idxs ∈
    fri-zero-round-first-query-index-lists trace-table trace-final
  using query-space first-hit
  unfolding fri-zero-round-first-query-index-lists-def by simp
  have hit:
    trace-fri-query-index-list-set-hit s
    (fri-zero-round-first-query-index-lists trace-table trace-final) out
  unfolding trace-fri-query-index-list-set-hit-def
  using fri-openings query-target by blast
  show ?thesis
  unfolding trace-fri-zero-round-query-list-target-hit-def
  using hit card-lt by blast
qed

lemma trace-fri-zero-round-candidate-first-query-target-hit-imp-query-list-target-or-alignment-gap:
  assumes trace-fri-zero-round-candidate-first-query-target-hit s out
  shows
    trace-fri-zero-round-query-list-target-hit s out ∨
    trace-fri-zero-round-candidate-first-query-alignment-gap s out
proof (rule trace-fri-zero-round-candidate-first-query-target-hitE[OF assms])
  fix trace-roots trace-bs trace-final dg composition-roots
    composition-bs composition-final fri-query-idxs trace-round-layers

```

```

    composition-round-layers fr candidate-query-idxs trace-openings
    trace-table
assume evidence:
    trace-fri-partial-candidate-opening-evidence s out trace-roots
    trace-bs trace-final dg composition-roots composition-bs
    composition-final fri-query-idxs trace-round-layers
    composition-round-layers fr candidate-query-idxs trace-openings
    trace-table
and rounds-empty: length trace-bs = 0
and not-final:
    ¬ fri-final-constant-consistent trace-table trace-final
and candidate-first-hit:
    candidate-query-idxs ! 0 ∈
    fri-final-value-agreement-set trace-table trace-final
and card-lt:
    card (fri-final-value-agreement-set trace-table trace-final) <
    query-sample-space-size
have fri-openings:
    accepted-fri-opening-transcript s out trace-roots trace-bs
    trace-final dg composition-roots composition-bs composition-final
    fri-query-idxs trace-round-layers composition-round-layers
by (rule trace-fri-partial-candidate-opening-evidenceD(1)
    [OF evidence])
show ?thesis
proof (cases fri-query-idxs ! 0 ∈
    fri-final-value-agreement-set trace-table trace-final)
case True
have query-space:
    fri-query-idxs ∈ fri-query-index-list-space
by (rule accepted-fri-opening-transcript-query-index-list-space
    [OF fri-openings])
have query-target:
    fri-query-idxs ∈
    fri-zero-round-first-query-index-lists trace-table trace-final
using query-space True
unfolding fri-zero-round-first-query-index-lists-def by simp
have hit:
    trace-fri-query-index-list-set-hit s
    (fri-zero-round-first-query-index-lists trace-table trace-final)
    out
unfolding trace-fri-query-index-list-set-hit-def
using fri-openings query-target by blast
have trace-fri-zero-round-query-list-target-hit s out
unfolding trace-fri-zero-round-query-list-target-hit-def
using hit card-lt by blast
then show ?thesis by blast
next
case False
have gap:

```

```

    trace-fri-zero-round-candidate-first-query-alignment-gap s out
unfolding trace-fri-zero-round-candidate-first-query-alignment-gap-def
by (intro exI conjI)
    (rule evidence, rule rounds-empty, rule not-final,
     rule candidate-first-hit, rule False, rule card-lt)
then show ?thesis by blast
qed
qed

lemma trace-fri-zero-round-final-obstruction-imp-query-list-target-or-candidate-gaps:
assumes trace-fri-zero-round-final-obstruction s out
shows
    trace-fri-zero-round-query-list-target-hit s out  $\vee$ 
    trace-fri-zero-round-candidate-first-query-value-gap s out  $\vee$ 
    trace-fri-zero-round-candidate-first-query-alignment-gap s out
proof (rule trace-fri-zero-round-final-obstruction-agreement-set-card-lt
    [OF assms])
fix trace-roots trace-bs trace-final dg composition-roots
    composition-bs composition-final fri-query-idxs trace-round-layers
    composition-round-layers fr candidate-query-idxs trace-openings
    trace-table
assume evidence:
    trace-fri-partial-candidate-opening-evidence s out trace-roots
    trace-bs trace-final dg composition-roots composition-bs
    composition-final fri-query-idxs trace-round-layers
    composition-round-layers fr candidate-query-idxs trace-openings
    trace-table
and rounds-empty: length trace-bs = 0
and not-final:
     $\neg$  fri-final-constant-consistent trace-table trace-final
and card-lt:
    card (fri-final-value-agreement-set trace-table trace-final) <
    query-sample-space-size
show ?thesis
proof (cases candidate-query-idxs ! 0  $\in$ 
    fri-final-value-agreement-set trace-table trace-final)
case True
have candidate-hit:
    trace-fri-zero-round-candidate-first-query-target-hit s out
unfolding trace-fri-zero-round-candidate-first-query-target-hit-def
by (intro exI conjI)
    (rule evidence, rule rounds-empty, rule not-final, rule True,
     rule card-lt)
then show ?thesis
using
    trace-fri-zero-round-candidate-first-query-target-hit-imp-query-list-target-or-alignment-gap
by blast
next
case False

```

```

have value-gap:
  trace-fri-zero-round-candidate-first-query-value-gap s out
unfolding trace-fri-zero-round-candidate-first-query-value-gap-def
by (intro exI conjI)
  (rule evidence, rule rounds-empty, rule not-final, rule False,
   rule card-lt)
then show ?thesis by blast
qed
qed

lemma wp-trace-fri-zero-round-final-obstruction-bound-from-query-list-target-and-candidate-gaps:
assumes target-bound:
  wp-event verify-monad
  (trace-fri-zero-round-query-list-target-hit s) s ≤ Q
and value-bound:
  wp-event verify-monad
  (trace-fri-zero-round-candidate-first-query-value-gap s) s ≤ V
and alignment-bound:
  wp-event verify-monad
  (trace-fri-zero-round-candidate-first-query-alignment-gap s) s ≤ A
shows
  wp-event verify-monad
  (trace-fri-zero-round-final-obstruction s) s ≤ Q + V + A
proof –
have event-le:
  wp-event verify-monad
  (trace-fri-zero-round-final-obstruction s) s ≤
  wp-event verify-monad
  (λout.
    trace-fri-zero-round-query-list-target-hit s out ∨
    trace-fri-zero-round-candidate-first-query-value-gap s out ∨
    trace-fri-zero-round-candidate-first-query-alignment-gap s out) s
by (rule wp-event-mono)
  (rule trace-fri-zero-round-final-obstruction-imp-query-list-target-or-candidate-gaps)
also have ... ≤
  wp-event verify-monad
  (trace-fri-zero-round-query-list-target-hit s) s +
  wp-event verify-monad
  (λout.
    trace-fri-zero-round-candidate-first-query-value-gap s out ∨
    trace-fri-zero-round-candidate-first-query-alignment-gap s out) s
by (rule wp-event-union-bound)
also have ... ≤
  wp-event verify-monad
  (trace-fri-zero-round-query-list-target-hit s) s +
  (wp-event verify-monad
    (trace-fri-zero-round-candidate-first-query-value-gap s) s +
    wp-event verify-monad
    (trace-fri-zero-round-candidate-first-query-alignment-gap s) s)

```

```

    by (intro add-mono order.refl wp-event-union-bound)
  also have ...  $\leq Q + (V + A)$ 
    by (intro add-mono target-bound value-bound alignment-bound)
  also have ...  $= Q + V + A$ 
    by (simp add: add.assoc)
  finally show ?thesis .
qed

lemma checked-staged-security-trace-fri-zero-round-query-list-target-exact-bound:
  assumes wf: staged-budget-wellformed budgets
    and controlled: staged-adversary-controlled budgets A
  shows
    wp-event (checked-staged-security-experiment-with-data-state A)
      (staged-security-with-data-state-verifier-event
        ( $\lambda s.$ 
          trace-fri-query-index-list-set-hit s
            (fri-zero-round-first-query-index-lists table final)))
    adversary-initial-state  $\leq$ 
      nnreal
        (card (fri-zero-round-first-query-index-lists table final)) *
        (1 / nnreal (card query-sample-space))  $^{\wedge}$  rounds +
      hash-relation-budget-value
        (query-index-raw-list-relation-fiber-bound
          (fri-zero-round-first-query-index-lists table final))
        (staged-attacker-query-budget budgets +
          staged-challenge-query-budget)
  by (rule checked-staged-security-trace-fri-query-index-list-set-hit-exact-product-bound
    [OF wf controlled fri-zero-round-first-query-index-lists-subset])

lemma checked-staged-security-trace-fri-zero-round-query-list-target-small-bound:
  assumes wf: staged-budget-wellformed budgets
    and controlled: staged-adversary-controlled budgets A
    and small:
      card (fri-final-value-agreement-set table final) <
        query-sample-space-size
  shows
    wp-event (checked-staged-security-experiment-with-data-state A)
      (staged-security-with-data-state-verifier-event
        ( $\lambda s.$ 
          trace-fri-query-index-list-set-hit s
            (fri-zero-round-first-query-index-lists table final)))
    adversary-initial-state  $\leq$ 
      nnreal
        ((query-sample-space-size - 1) *
          query-sample-space-size  $^{\wedge}$  (rounds - 1)) *
        (1 / nnreal (card query-sample-space))  $^{\wedge}$  rounds +
      hash-relation-budget-value
        (query-index-raw-list-relation-fiber-bound
          (fri-zero-round-first-query-index-lists table final))

```

(staged-attacker-query-budget budgets +
 staged-challenge-query-budget)

proof –

have *exact*:

wp-event (checked-staged-security-experiment-with-data-state *A*)
 (staged-security-with-data-state-verifier-event
 (λ*s*.
 trace-fri-query-index-list-set-hit *s*
 (fri-zero-round-first-query-index-lists table final)))
adversary-initial-state ≤
nnreal
 (card (fri-zero-round-first-query-index-lists table final)) *
 (1 / *nnreal* (card query-sample-space)) ^ rounds +
hash-relation-budget-value
 (query-index-row-list-relation-fiber-bound
 (fri-zero-round-first-query-index-lists table final))
 (staged-attacker-query-budget budgets +
 staged-challenge-query-budget)

by (rule
 checked-staged-security-trace-fri-zero-round-query-list-target-exact-bound
 [OF *wf controlled*])

have *card-le*:

card (fri-zero-round-first-query-index-lists table final) ≤
 (query-sample-space-size - 1) *
 query-sample-space-size ^ (rounds - 1)

by (rule fri-zero-round-first-query-index-lists-card-bound-from-small
 [OF *small*])

have *product-le*:

nnreal
 (card (fri-zero-round-first-query-index-lists table final)) *
 (1 / *nnreal* (card query-sample-space)) ^ rounds ≤
nnreal
 ((query-sample-space-size - 1) *
 query-sample-space-size ^ (rounds - 1)) *
 (1 / *nnreal* (card query-sample-space)) ^ rounds

proof –

have *card-nn*:

nnreal (card (fri-zero-round-first-query-index-lists table final)) ≤
nnreal
 ((query-sample-space-size - 1) *
 query-sample-space-size ^ (rounds - 1))

using *of-nat-mono*[OF *card-le*, **where** '*a* = *nnreal*] **by** *simp*

show *?thesis*

by (rule *mult-right-mono*[OF *card-nn*]) *simp*

qed

show *?thesis*

by (rule *order-trans*[OF *exact add-mono*[OF *product-le order-refl*]])

qed

end

end

theory *Soundness-FRI-Zero-Round-Agreement-Set-Bounds*
 imports *Soundness-FRI-Zero-Round-Query-List-Bounds*
begin

Global agreement-set bound for the zero-round trace FRI target.

The verifier-local event *trace-fri-zero-round-query-list-target-hit* is existential in a candidate table and final value. The query-list target depends on these witnesses only through the agreement set inside *query-sample-space*. This layer replaces the arbitrary-list existential by a finite union over agreement sets, preserving the exact query-list product accounting for each fixed set.

context *soundness*
begin

definition *fri-zero-round-first-query-index-lists-for-set*
 $:: \text{nat set} \Rightarrow \text{nat list set}$

where

fri-zero-round-first-query-index-lists-for-set $S =$
 $\{\text{query-idxs} \in \text{fri-query-index-list-space}. \text{query-idxs} ! 0 \in S\}$

definition *fri-zero-round-small-agreement-sets* $:: \text{nat set set}$

where

fri-zero-round-small-agreement-sets $=$
 $\{S. S \subseteq \text{query-sample-space} \wedge \text{card } S < \text{query-sample-space-size}\}$

definition *trace-fri-zero-round-agreement-set-target-error-for*

where

trace-fri-zero-round-agreement-set-target-error-for $\text{budgets} =$
 $(\sum S \in \text{fri-zero-round-small-agreement-sets}.$
 $\text{nnreal } (\text{card } (\text{fri-zero-round-first-query-index-lists-for-set } S)) *$
 $(1 / \text{nnreal } (\text{card } \text{query-sample-space})) ^ \text{rounds} +$
 $\text{hash-relation-budget-value}$
 $(\text{query-index-raw-list-relation-fiber-bound}$
 $(\text{fri-zero-round-first-query-index-lists-for-set } S))$
 $(\text{staged-attacker-query-budget } \text{budgets} +$
 $\text{staged-challenge-query-budget}))$

lemma *finite-fri-zero-round-small-agreement-sets[simp]:*

finite fri-zero-round-small-agreement-sets

proof –

have *fri-zero-round-small-agreement-sets* $\subseteq \text{Pow } \text{query-sample-space}$

unfolding *fri-zero-round-small-agreement-sets-def* **by** *blast*

then show *?thesis*

by (rule *finite-subset*) *simp*

qed

lemma *fri-zero-round-first-query-index-lists-for-set-subset:*
 fri-zero-round-first-query-index-lists-for-set S
 $\subseteq$ *fri-query-index-list-space*
unfolding *fri-zero-round-first-query-index-lists-for-set-def* **by** *blast*

lemma *trace-fri-zero-round-table-target-eq-agreement-set-target:*
 fri-zero-round-first-query-index-lists table final =
 fri-zero-round-first-query-index-lists-for-set
 (*fri-final-value-agreement-set table final*)
unfolding *fri-zero-round-first-query-index-lists-def*
 fri-zero-round-first-query-index-lists-for-set-def
by *simp*

lemma *checked-staged-security-trace-fri-zero-round-agreement-set-target-bound:*
assumes *wf: staged-budget-wellformed budgets*
and *controlled: staged-adversary-controlled budgets A*
and *S-in: S ∈ fri-zero-round-small-agreement-sets*
shows
 wp-event (checked-staged-security-experiment-with-data-state A)
 (*staged-security-with-data-state-verifier-event*
 ($\lambda s.$
 trace-fri-query-index-list-set-hit s
 (*fri-zero-round-first-query-index-lists-for-set S*)))
 adversary-initial-state $\leq$
 *nnreal (card (fri-zero-round-first-query-index-lists-for-set S)) **
 (*1 / nnreal (card query-sample-space)*) $^$ *rounds* +
 hash-relation-budget-value
 (*query-index-row-list-relation-fiber-bound*
 (*fri-zero-round-first-query-index-lists-for-set S*))
 (*staged-attacker-query-budget budgets* +
 staged-challenge-query-budget)
by (*rule checked-staged-security-trace-fri-query-index-list-set-hit-exact-product-bound*
 [*OF wf controlled fri-zero-round-first-query-index-lists-for-set-subset*])

lemma *checked-staged-security-trace-fri-zero-round-query-list-target-bound-from-agreement-sets:*
assumes *wf: staged-budget-wellformed budgets*
and *controlled: staged-adversary-controlled budgets A*
shows
 wp-event (checked-staged-security-experiment-with-data-state A)
 (*staged-security-with-data-state-verifier-event*
 trace-fri-zero-round-query-list-target-hit)
 adversary-initial-state $\leq$
 trace-fri-zero-round-agreement-set-target-error-for budgets
proof –
 let *?M = checked-staged-security-experiment-with-data-state A*
 let *?A = fri-zero-round-small-agreement-sets*
 let *?P =*

```

λS. staged-security-with-data-state-verifier-event
  (λs.
    trace-fri-query-index-list-set-hit s
    (fri-zero-round-first-query-index-lists-for-set S))
have event-le:
  wp-event ?M
    (staged-security-with-data-state-verifier-event
      trace-fri-zero-round-query-list-target-hit)
  adversary-initial-state ≤
  wp-event ?M (λout. ∃ S ∈ ?A. ?P S out)
  adversary-initial-state
proof (rule wp-event-mono-on-support)
fix out
assume
  out ∈ set-dist (execute ?M adversary-initial-state)
and hit:
  staged-security-with-data-state-verifier-event
    trace-fri-zero-round-query-list-target-hit out
show ∃ S ∈ ?A. ?P S out
proof (cases out)
case None
then show ?thesis
  using hit unfolding staged-security-with-data-state-verifier-event-def
    trace-fri-zero-round-query-list-target-hit-def
  by simp
next
case (Some out-data)
then obtain data attacker-state result final-state where out-eq:
  out = Some (((data, attacker-state), result), final-state)
  by (cases out-data, auto split: prod.splits)
let ?s =
  verifier-state-from-adversary attacker-state
    (staged-proof-transcript data)
have local-hit:
  trace-fri-zero-round-query-list-target-hit ?s
    (Some (result, final-state))
  using hit unfolding out-eq staged-security-with-data-state-verifier-event-def
  by simp
then obtain trace-table trace-final where query-hit:
  trace-fri-query-index-list-set-hit ?s
    (fri-zero-round-first-query-index-lists trace-table trace-final)
    (Some (result, final-state))
and small:
  card (fri-final-value-agreement-set trace-table trace-final)
    < query-sample-space-size
  by (rule trace-fri-zero-round-query-list-target-hitE)
let ?S = fri-final-value-agreement-set trace-table trace-final
have S-in: ?S ∈ ?A
  unfolding fri-zero-round-small-agreement-sets-def

```

```

    using fri-final-value-agreement-set-subset small by simp
  have query-hit':
    trace-fri-query-index-list-set-hit ?s
      (fri-zero-round-first-query-index-lists-for-set ?S)
      (Some (result, final-state))
    using query-hit
    by (simp add:
        trace-fri-zero-round-table-target-eq-agreement-set-target)
  have ?P ?S out
    unfolding out-eq staged-security-with-data-state-verifier-event-def
    using query-hit' by simp
  show ?thesis
    using S-in ⟨?P ?S out⟩ by blast
qed
qed
have union-bound:
  wp-event ?M (λout. ∃ S ∈ ?A. ?P S out)
  adversary-initial-state ≤
  (∑ S ∈ ?A.
    nnreal (card (fri-zero-round-first-query-index-lists-for-set S)) *
    (1 / nnreal (card query-sample-space)) ^ rounds +
    hash-relation-budget-value
    (query-index-raw-list-relation-fiber-bound
      (fri-zero-round-first-query-index-lists-for-set S))
    (staged-attacker-query-budget budgets +
      staged-challenge-query-budget))
proof (rule wp-event-finite-UN-bound)
  show finite ?A
    by simp
next
fix S
assume S-in: S ∈ ?A
show wp-event ?M (?P S) adversary-initial-state ≤
  nnreal (card (fri-zero-round-first-query-index-lists-for-set S)) *
  (1 / nnreal (card query-sample-space)) ^ rounds +
  hash-relation-budget-value
  (query-index-raw-list-relation-fiber-bound
    (fri-zero-round-first-query-index-lists-for-set S))
  (staged-attacker-query-budget budgets +
    staged-challenge-query-budget)
by (rule checked-staged-security-trace-fri-zero-round-agreement-set-target-bound
    [OF wf controlled S-in])
qed
show ?thesis
  by (rule order-trans[OF event-le])
  (use union-bound in
    ⟨simp add: trace-fri-zero-round-agreement-set-target-error-for-def⟩)
qed

```

end

end

theory *Soundness-FRI-Trace-Empty-Verifier-Tied*

imports

Soundness-FRI-Trace-Untied-Cases

Soundness-FRI-Zero-Round-Query-List-Bounds

begin

Verifier-tied normalization for the empty-composition-header trace FRI branch.

The broad empty-header branch can carry a non-low candidate built from existential partial openings. This layer separates that branch into the verifier-checked zero-round obstruction, Merkle inconsistency, or the existing candidate-transfer residual. It deliberately avoids complete-table reconstruction from sampled openings.

context *soundness*

begin

lemma *trace-zero-round-candidate-low-degree-if-final-consistent:*

assumes *rounds-empty: length trace-bs = 0*

and *fri-openings:*

accepted-fri-opening-transcript s out trace-roots trace-bs

trace-final dg composition-roots composition-bs composition-final

fri-query-idxs trace-round-layers composition-round-layers

and *candidate:*

partial-trace-table-candidate trace-table trace-openings

and *final:*

fri-final-constant-consistent trace-table trace-final

shows *trace-table-low-degree trace-table*

proof –

have *len-trace-bs: length trace-bs = ceil-log clength*

using *accepted-fri-opening-transcript-shapes(1,2)[OF fri-openings]*

by *simp*

have *ceil-zero: ceil-log clength = 0*

using *rounds-empty len-trace-bs* **by** *simp*

have *clength-le-one: clength ≤ 1*

using *ceil-zero unfolding ceil-log-def*

by *(cases clength ≤ 1) simp-all*

have *clength-eq: clength = 1*

using *clength-le-one clength-pos* **by** *simp*

have *len-table: length trace-table = length eval-domain*

using *candidate eval-domain-length*

unfolding *partial-trace-table-candidate-def*

by *simp*

have *table-eq:*

trace-table = map (poly [:trace-final:]) eval-domain

proof *(rule nth-equalityI)*

show *length trace-table =*

```

    length (map (poly [:trace-final:]) eval-domain)
  using len-table by simp
next
  fix i
  assume i-bound: i < length trace-table
  have trace-table ! i = trace-final
    by (rule fri-final-constant-consistent-nth[OF final i-bound])
  then show trace-table ! i =
    map (poly [:trace-final:]) eval-domain ! i
    using i-bound len-table by simp
qed
show ?thesis
  unfolding trace-table-low-degree-def
  by (intro exI[of - [:trace-final:]] conjI)
    (use clength-eq table-eq in simp-all)
qed

lemma trace-fri-zero-round-final-obstruction-imp-checked-merkle-transfer-or-header:
  assumes support: out ∈ set-dist (execute verify-monad s)
  and obstruction: trace-fri-zero-round-final-obstruction s out
  shows
    trace-fri-zero-round-checked-final-obstruction s out ∨
    partial-merkle-inconsistency-bad s out ∨
    trace-fri-header-tied-candidate-transfer-gap s out ∨
    trace-fri-bad-with-header-tied-partial-candidate s out
proof -
  from trace-fri-zero-round-final-obstructionE[OF obstruction]
  obtain trace-roots trace-bs trace-final dg composition-roots
    composition-bs composition-final fri-query-idxs trace-round-layers
    composition-round-layers fr0 candidate-query-idxs trace-openings0
    trace-table0 where evidence:
    trace-fri-partial-candidate-opening-evidence s out trace-roots
    trace-bs trace-final dg composition-roots composition-bs
    composition-final fri-query-idxs trace-round-layers
    composition-round-layers fr0 candidate-query-idxs trace-openings0
    trace-table0
  and rounds-empty: length trace-bs = 0
  and not-final0:
    ¬ fri-final-constant-consistent trace-table0 trace-final
  by blast
  have fri-openings:
    accepted-fri-opening-transcript s out trace-roots trace-bs
    trace-final dg composition-roots composition-bs composition-final
    fri-query-idxs trace-round-layers composition-round-layers
  using trace-fri-partial-candidate-opening-evidenceD(1)[OF evidence] .
  from fri-openings obtain result final-state where out-eq:
    out = Some (result, final-state)
  unfolding accepted-fri-opening-transcript-def by blast
  from accepted-fri-opening-transcript-headerE[OF fri-openings]

```

```

obtain fr as rest where header:
  verifier-header-transcript s fr trace-roots trace-final as dg
  composition-roots composition-final rest
by blast
show ?thesis
proof (cases partial-merkle-inconsistency-bad s out)
  case True
  then show ?thesis by simp
next
  case no-merkle: False
  from accepted-fri-opening-transcript-trace-openings-for-query-indices
    [OF fri-openings header out-eq]
  obtain trace-openings where partial:
    accepted-with-partial-trace-openings s out fr fri-query-idxs
    trace-openings
  by blast
  obtain trace-table where candidate:
    partial-trace-table-candidate trace-table trace-openings
  using accepted-with-partial-trace-openings-candidate-if-no-partial-merkle-bad
    [OF partial[unfolded out-eq] no-merkle[unfolded out-eq]]
  by blast
  show ?thesis
  proof (cases trace-table-low-degree trace-table)
    case True
    then show ?thesis
    proof (cases trace-fri-bad-with-header-tied-partial-candidate s out)
      case True
      then show ?thesis by simp
    next
    case no-header: False
    have reachable:
      trace-fri-bad-with-reachable-partial-candidate s out
    using trace-fri-zero-round-final-obstruction-imp-header-tied-or-gap
      [OF obstruction] no-header
    unfolding trace-fri-reachable-header-tie-gap-def by simp
    have gap: trace-fri-reachable-header-tie-gap s out
    unfolding trace-fri-reachable-header-tie-gap-def
    using reachable no-header by simp
    have transfer:
      trace-fri-header-tied-candidate-transfer-gap s out
    by (rule trace-fri-header-candidate-low-imp-transfer-gap
      [OF gap fri-openings header partial candidate True])
    then show ?thesis by simp
  qed
next
  case not-low: False
  have not-final:
     $\neg$  fri-final-constant-consistent trace-table trace-final
  proof

```

```

assume final:
  fri-final-constant-consistent trace-table trace-final
have low:
  trace-table-low-degree trace-table
  by (rule
    trace-zero-round-candidate-low-degree-if-final-consistent
    [OF rounds-empty fri-openings candidate final])
then show False
  using not-low by contradiction
qed
have header-zero:
  trace-fri-header-tied-zero-round-final-obstruction s out
  unfolding trace-fri-header-tied-zero-round-final-obstruction-def
  by (intro exI conjI)
    (rule fri-openings, rule header, rule partial, rule candidate,
     rule not-low, rule rounds-empty, rule not-final)
then show ?thesis
  using
    trace-fri-header-tied-zero-round-final-obstruction-imp-checked-or-merkle
  by blast
qed
qed
qed

lemma wp-trace-fri-zero-round-checked-final-obstruction-bound-from-query-list-target:
  assumes target-bound:
    wp-event verify-monad
    (trace-fri-zero-round-query-list-target-hit s)  $s \leq Q$ 
  shows
    wp-event verify-monad
    (trace-fri-zero-round-checked-final-obstruction s)  $s \leq Q$ 
proof –
  have wp-event verify-monad
    (trace-fri-zero-round-checked-final-obstruction s)  $s \leq$ 
    wp-event verify-monad
    (trace-fri-zero-round-checked-first-query-target-hit s)  $s$ 
  by (rule wp-event-mono)
    (rule trace-fri-zero-round-checked-final-obstruction-imp-first-query-target-hit)
  also have ...  $\leq$ 
    wp-event verify-monad
    (trace-fri-zero-round-query-list-target-hit s)  $s$ 
  by (rule wp-event-mono)
    (rule trace-fri-zero-round-checked-first-query-target-hit-imp-query-list-target)
  also have ...  $\leq Q$ 
  by (rule target-bound)
  finally show ?thesis .
qed

```

lemma *wp-trace-fri-zero-round-final-obstruction-bound-from-checked-merkle-transfer-header*:

fixes $H\ M\ Q\ T :: \text{prob}$
assumes *checked-bound*:
 $\text{wp-event verify-monad}$
 $(\text{trace-fri-zero-round-checked-final-obstruction } s) \ s \leq Q$
and *merkle-bound*:
 $\text{wp-event verify-monad } (\text{partial-merkle-inconsistency-bad } s) \ s \leq M$
and *transfer-bound*:
 $\text{wp-event verify-monad}$
 $(\text{trace-fri-header-tied-candidate-transfer-gap } s) \ s \leq T$
and *header-bound*:
 $\text{wp-event verify-monad}$
 $(\text{trace-fri-bad-with-header-tied-partial-candidate } s) \ s \leq H$
shows
 $\text{wp-event verify-monad}$
 $(\text{trace-fri-zero-round-final-obstruction } s) \ s \leq Q + M + T + H$
proof –
 have $\text{wp-event verify-monad}$
 $(\text{trace-fri-zero-round-final-obstruction } s) \ s \leq$
 $\text{wp-event verify-monad}$
 $(\lambda \text{out.}$
 $\text{trace-fri-zero-round-checked-final-obstruction } s \ \text{out} \ \vee$
 $\text{partial-merkle-inconsistency-bad } s \ \text{out} \ \vee$
 $\text{trace-fri-header-tied-candidate-transfer-gap } s \ \text{out} \ \vee$
 $\text{trace-fri-bad-with-header-tied-partial-candidate } s \ \text{out}) \ s$
 by $(\text{rule wp-event-mono-on-support})$
 $(\text{rule trace-fri-zero-round-final-obstruction-imp-checked-merkle-transfer-or-header})$
 also have $\dots \leq$
 $\text{wp-event verify-monad}$
 $(\text{trace-fri-zero-round-checked-final-obstruction } s) \ s +$
 $\text{wp-event verify-monad } (\text{partial-merkle-inconsistency-bad } s) \ s +$
 $\text{wp-event verify-monad}$
 $(\text{trace-fri-header-tied-candidate-transfer-gap } s) \ s +$
 $\text{wp-event verify-monad}$
 $(\text{trace-fri-bad-with-header-tied-partial-candidate } s) \ s$
 by $(\text{rule wp-event-union-bound4})$
 also have $\dots \leq Q + M + T + H$
 by $(\text{intro add-mono checked-bound merkle-bound transfer-bound}$
 $\text{header-bound})$
 finally show $?thesis$.
qed

lemma *trace-fri-header-tied-candidate-transfer-gap-imp-header-or-untied*:
assumes *transfer*: $\text{trace-fri-header-tied-candidate-transfer-gap } s \ \text{out}$
shows
 $\text{trace-fri-bad-with-header-tied-partial-candidate } s \ \text{out} \ \vee$
 $\text{trace-fri-header-tied-low-candidate-untied-reachable-witness-gap } s \ \text{out}$
proof $(\text{cases trace-fri-bad-with-header-tied-partial-candidate } s \ \text{out})$
 case *True*
 then show $?thesis$ **by** *simp*

```

next
case no-header: False
from transfer obtain trace-roots trace-bs trace-final fri-dg
  composition-roots composition-bs composition-final fri-query-idxs
  trace-round-layers composition-round-layers fr as rest trace-openings
  trace-table where
raw: trace-fri-bad-with-reachable-partial-candidate s out
and fri-openings:
  accepted-fri-opening-transcript s out trace-roots trace-bs
  trace-final fri-dg composition-roots composition-bs
  composition-final fri-query-idxs trace-round-layers
  composition-round-layers
and header:
  verifier-header-transcript s fr trace-roots trace-final as fri-dg
  composition-roots composition-final rest
and partial:
  accepted-with-partial-trace-openings s out fr fri-query-idxs
  trace-openings
and candidate:
  partial-trace-table-candidate trace-table trace-openings
and low: trace-table-low-degree trace-table
unfolding trace-fri-header-tied-candidate-transfer-gap-def by blast
have gap: trace-fri-reachable-header-tie-gap s out
unfolding trace-fri-reachable-header-tie-gap-def
using raw no-header by simp
have low-gap: trace-fri-header-tied-low-candidate-gap s out
unfolding trace-fri-header-tied-low-candidate-gap-def
by (intro exI conjI)
  (rule gap, rule fri-openings, rule header, rule partial, rule candidate,
   rule low)
have untied:
  trace-fri-header-tied-low-candidate-untied-reachable-witness-gap s out
by (rule trace-fri-header-tied-low-candidate-gap-imp-untied-reachable-witness
  [OF low-gap])
then show ?thesis by simp
qed

lemma wp-trace-fri-header-tied-candidate-transfer-gap-bound-from-header-and-untied:
fixes H U :: prob
assumes header-bound:
  wp-event verify-monad
  (trace-fri-bad-with-header-tied-partial-candidate s) s ≤ H
and untied-bound:
  wp-event verify-monad
  (trace-fri-header-tied-low-candidate-untied-reachable-witness-gap s)
  s ≤ U
shows
  wp-event verify-monad
  (trace-fri-header-tied-candidate-transfer-gap s) s ≤ H + U

```

```

proof –
  have wp-event verify-monad
    (trace-fri-header-tied-candidate-transfer-gap s) s ≤
    wp-event verify-monad
      ( $\lambda out.$ 
        trace-fri-bad-with-header-tied-partial-candidate s out  $\vee$ 
        trace-fri-header-tied-low-candidate-untied-reachable-witness-gap
          s out) s
  by (rule wp-event-mono)
    (rule trace-fri-header-tied-candidate-transfer-gap-imp-header-or-untied)
  also have ... ≤
    wp-event verify-monad
      (trace-fri-bad-with-header-tied-partial-candidate s) s +
    wp-event verify-monad
      (trace-fri-header-tied-low-candidate-untied-reachable-witness-gap s)
      s
  by (rule wp-event-union-bound)
  also have ... ≤ H + U
  by (rule add-mono[OF header-bound untied-bound])
  finally show ?thesis .
qed

end

end

```

```

theory Soundness-Public-Route-Concrete-FRI-Sharpened
imports
  Soundness-Public-Route-Concrete-FRI-Endpoint
  Soundness-FRI-Query-Fiber-Staged-Bounds
  Soundness-FRI-Prequery-Challenge-Bounds
  Soundness-FRI-Sampled-Challenge-Cover
  Soundness-FRI-Staged-Replay-Bounds
  Soundness-FRI-Zero-Round-Agreement-Set-Bounds
  Soundness-FRI-Trace-Empty-Verifier-Tied
begin

```

First-stage API for sharpening the concrete FRI endpoint.

The public endpoint already removed the FRI soundness assumptions by using explicit staged verifier-event error definitions. This theory factors those definitions into named challenge, full-cover/domain, conflict, and zero-round components. At this stage the components are definitionally equal to the endpoint terms; later proof layers can replace individual components by sharper product/fiber or residual sums without touching the public theorem interface.

```

context soundness
begin

```

definition *trace-fri-challenge-prequery-error-for*

where

trace-fri-challenge-prequery-error-for budgets $A =$
 (wp-event (checked-staged-security-experiment-with-data-state A)
 (staged-security-with-data-state-verifier-event
 ($\lambda s.$
 trace-fri-challenge-list-fresh-hit s
 (*fri-challenge-space* (ceil-log clength))))
 adversary-initial-state +
 hash-relation-budget-value
 (card (*fri-challenge-space* (ceil-log clength)) * ceil-log clength)
 (staged-attacker-query-budget budgets +
 staged-challenge-query-budget))

definition *trace-fri-full-cover-domain-residual-error-for*

where

trace-fri-full-cover-domain-residual-error-for $A =$
 (reachable-verifier-event-bound-for A
 ($\lambda s.$
 trace-fri-sampled-full-cover-proximity-failure s
 trace-fri-full-challenge-bad) +
 reachable-verifier-event-bound-for A
trace-fri-sampled-full-cover-index-failure +
 (reachable-verifier-event-bound-for A
 trace-fri-sampled-full-cover-index-failure +
 reachable-verifier-event-bound-for A
 trace-fri-sampled-full-cover-domain-length-failure) +
 (reachable-verifier-event-bound-for A
 trace-fri-sampled-full-cover-index-failure +
 reachable-verifier-event-bound-for A
 trace-fri-sampled-full-cover-sampled-domain-failure))

definition *sharpened-trace-fri-full-cover-domain-error-for*

where

sharpened-trace-fri-full-cover-domain-error-for budgets $A =$
trace-fri-challenge-prequery-error-for budgets A +
trace-fri-full-cover-domain-residual-error-for A

definition *trace-fri-header-conflict-zero-error-for*

where

trace-fri-header-conflict-zero-error-for $A =$
 (reachable-verifier-event-bound-for A
 trace-fri-header-tied-sampled-assignment-conflict +
 reachable-verifier-event-bound-for A
 trace-fri-header-tied-zero-round-final-obstruction)

definition *trace-fri-header-replay-conflict-error-for*

where

trace-fri-header-replay-conflict-error-for $A =$

(let $M = \text{reachable-verifier-event-bound-for } A$
 $\text{partial-merkle-inconsistency-bad};$
 $H = \text{reachable-verifier-event-bound-for } A$
 $\text{trace-fri-header-tied-recorded-sibling-candidate-missing}$
in $H + (((M + M) + (M + 0) + (M + 0) + (M + 0 + 0)) + M) + 0))$

definition *trace-fri-controlled-merkle-error-for*

where

trace-fri-controlled-merkle-error-for budgets =
 $\text{hash-collision-budget-value } 0$
 $(\text{staged-attacker-query-budget budgets} +$
 $\text{staged-challenge-query-budget} + \text{verifier-hash-query-budget})$

definition *trace-fri-header-replay-conflict-zero-error-for*

where

trace-fri-header-replay-conflict-zero-error-for budgets $A =$
 $\text{trace-fri-header-replay-conflict-error-for } A +$
 $(\text{trace-fri-zero-round-agreement-set-target-error-for budgets} +$
 $\text{trace-fri-controlled-merkle-error-for budgets})$

definition *sharpened-trace-fri-header-error-for*

where

sharpened-trace-fri-header-error-for budgets $A =$
 $\text{sharpened-trace-fri-full-cover-domain-error-for budgets } A +$
 $\text{trace-fri-header-conflict-zero-error-for } A$

definition *trace-fri-empty-conflict-zero-error-for*

where

trace-fri-empty-conflict-zero-error-for $A =$
 $(\text{reachable-verifier-event-bound-for } A$
 $\text{trace-fri-sampled-assignment-conflict} +$
 $\text{reachable-verifier-event-bound-for } A$
 $\text{trace-fri-zero-round-final-obstruction})$

definition *trace-fri-empty-small-conflict-zero-error-for*

where

trace-fri-empty-small-conflict-zero-error-for budgets $A =$
 $(\text{reachable-verifier-event-bound-for } A$
 $\text{trace-fri-sampled-assignment-conflict} +$
 $\text{trace-fri-zero-round-agreement-set-target-error-for budgets} +$
 $\text{trace-fri-controlled-merkle-error-for budgets} +$
 $\text{reachable-verifier-event-bound-for } A$
 $\text{trace-fri-bad-with-header-tied-partial-candidate} +$
 $(\text{trace-fri-controlled-merkle-error-for budgets} +$
 $(\text{reachable-verifier-event-bound-for } A$
 $\text{trace-fri-header-tied-low-candidate-untied-root-gap} +$
 $\text{reachable-verifier-event-bound-for } A$
 $\text{trace-fri-header-tied-low-candidate-untied-query-gap})) +$
 $\text{reachable-verifier-event-bound-for } A$

trace-fri-bad-with-header-tied-partial-candidate)

definition *sharpened-trace-fri-empty-error-for*

where

sharpened-trace-fri-empty-error-for budgets $A =$
sharpened-trace-fri-full-cover-domain-error-for budgets $A +$
trace-fri-empty-conflict-zero-error-for A

definition *composition-fri-challenge-prequery-error-for*

where

composition-fri-challenge-prequery-error-for budgets $A =$
 (*wp-event* (*checked-staged-security-experiment-with-data-state* A)
 (*staged-security-with-data-state-verifier-event*
 ($\lambda s.$
 composition-fri-challenge-list-fresh-hit s
 ($\lambda dg. \text{fri-challenge-space } (\text{ceil-log } (\text{to-nat } dg + 1))$)))
adversary-initial-state +
hash-relation-budget-value
 ($\sum dg \in (UNIV :: 'f \text{ set}).$
 card (*fri-challenge-space* ($\text{ceil-log } (\text{to-nat } dg + 1)$)) *
 ceil-log (*maxDegree* + 1))
 (*staged-attacker-query-budget* budgets +
 staged-challenge-query-budget))

definition *composition-fri-full-cover-domain-residual-error-for*

where

composition-fri-full-cover-domain-residual-error-for $A =$
 (*reachable-verifier-event-bound-for* A
 ($\lambda s.$
 composition-fri-sampled-full-cover-proximity-failure s
 composition-fri-full-challenge-bad) +
reachable-verifier-event-bound-for A
 composition-fri-sampled-full-cover-index-failure +
 (*reachable-verifier-event-bound-for* A
 composition-fri-sampled-full-cover-index-failure +
 reachable-verifier-event-bound-for A
 composition-fri-sampled-full-cover-domain-length-failure) +
 (*reachable-verifier-event-bound-for* A
 composition-fri-sampled-full-cover-index-failure +
 reachable-verifier-event-bound-for A
 composition-fri-sampled-full-cover-sampled-domain-failure))

definition *sharpened-composition-fri-full-cover-domain-error-for*

where

sharpened-composition-fri-full-cover-domain-error-for budgets $A =$
composition-fri-challenge-prequery-error-for budgets $A +$
composition-fri-full-cover-domain-residual-error-for A

definition *composition-fri-conflict-zero-error-for*

where

composition-fri-conflict-zero-error-for $A =$
 (*reachable-verifier-event-bound-for* A
 composition-fri-verifier-tied-sampled-assignment-conflict +
 reachable-verifier-event-bound-for A
 composition-fri-verifier-tied-zero-round-final-obstruction)

definition *sharpened-composition-fri-error-for*

where

sharpened-composition-fri-error-for *budgets* $A =$
sharpened-composition-fri-full-cover-domain-error-for *budgets* $A +$
composition-fri-conflict-zero-error-for A

definition *composition-one-step-query-fraction-for*

$:: ('f \Rightarrow \text{nat} \Rightarrow 'f \text{ list} \Rightarrow 'f \text{ set}) \Rightarrow \text{prob}$

where

composition-one-step-query-fraction-for *comp-bad* =
 nnreal
 (($\sum dg \in (\text{UNIV} :: 'f \text{ set}).$
 $\sum \text{challenges} \in$
 fri-challenge-space (*fri-round-count-for-degree-bound* (*to-nat* dg)) -
 generic-fri-bad-challenge-lists
 (*fri-round-count-for-degree-bound* (*to-nat* dg))
 (*comp-bad* dg).
 card
 (*generic-fri-sampled-query-query-fiber*
 (*composition-table-low-degree* (*to-nat* dg))
 (*Not* $\circ$ *composition-table-low-degree* *maxDegree*)
 (*to-nat* dg) *challenges*)) * *rounds*) /
 nnreal (*card* *query-sample-space*)

definition *composition-one-step-challenge-hash-count* $:: \text{nat}$

where

composition-one-step-challenge-hash-count =
 ($\sum dg \in (\text{UNIV} :: 'f \text{ set}).$
 fri-one-step-disagreement-round-mass
 (*fri-round-count-for-degree-bound* (*to-nat* dg))
 (*composition-fri-canonical-layer-len* dg) *
 ceil-log (*maxDegree* + 1))

definition *composition-one-step-sampled-error-for*

where

composition-one-step-sampled-error-for *budgets* *comp-bad* =
concrete-composition-fri-bad-challenge-fraction +
hash-relation-budget-value
 composition-one-step-challenge-hash-count
 (*staged-attacker-query-budget* *budgets* +
 staged-challenge-query-budget) +
 nnreal (*staged-attacker-query-budget* *budgets* +

staged-challenge-query-budget) *
composition-one-step-query-fraction-for comp-bad

definition *composition-one-step-reachable-merkle-error-for*
where

composition-one-step-reachable-merkle-error-for A =
reachable-verifier-event-bound-for A partial-merkle-inconsistency-bad

definition *composition-one-step-controlled-merkle-error-for*
where

composition-one-step-controlled-merkle-error-for budgets =
hash-collision-budget-value 0
 (*staged-attacker-query-budget budgets* +
staged-challenge-query-budget + *verifier-hash-query-budget*)

definition *composition-one-step-verifier-tied-error-for*
where

composition-one-step-verifier-tied-error-for budgets A comp-bad =
composition-one-step-sampled-error-for budgets comp-bad +
 (((*composition-one-step-reachable-merkle-error-for A* + 0) +
 (*composition-one-step-reachable-merkle-error-for A* + 0) +
 (*composition-one-step-reachable-merkle-error-for A* + 0) +
 (*composition-one-step-reachable-merkle-error-for A* + 0 + 0) +
composition-one-step-reachable-merkle-error-for A) + 0 + 0)

definition *composition-one-step-verifier-tied-controlled-error-for*
where

composition-one-step-verifier-tied-controlled-error-for budgets comp-bad =
composition-one-step-sampled-error-for budgets comp-bad +
 (((*composition-one-step-controlled-merkle-error-for budgets* + 0) +
 (*composition-one-step-controlled-merkle-error-for budgets* + 0) +
 (*composition-one-step-controlled-merkle-error-for budgets* + 0) +
 (*composition-one-step-controlled-merkle-error-for budgets* + 0 + 0) +
composition-one-step-controlled-merkle-error-for budgets) + 0 + 0)

definition *trace-one-step-query-fraction-for*

:: (nat ⇒ 'f list ⇒ 'f set) ⇒ prob

where

trace-one-step-query-fraction-for empty-bad =
nnreal
 (($\sum$ challenges ∈
fri-challenge-space (fri-round-count-for-degree-bound (clength - 1)) -
generic-fri-bad-challenge-lists
 (*fri-round-count-for-degree-bound (clength - 1)) empty-bad.*
card
 (*generic-fri-sampled-query-query-fiber trace-table-low-degree*
 (*Not* ○ *trace-table-low-degree*) (clength - 1) challenges)) *
rounds) /
nnreal (card query-sample-space)

definition *trace-one-step-challenge-hash-count* :: nat

where

trace-one-step-challenge-hash-count =
fri-one-step-disagreement-round-mass
 (*fri-round-count-for-degree-bound* (*clength* - 1))
trace-fri-canonical-layer-len *
ceil-log clength

definition *trace-one-step-disagreement-bad-for*

:: (nat $\Rightarrow$ nat) $\Rightarrow$
 (nat $\Rightarrow$ 'f list $\Rightarrow$ 'f list) $\Rightarrow$
 (nat $\Rightarrow$ 'f list $\Rightarrow$ 'f list) $\Rightarrow$
 (nat $\Rightarrow$ 'f list $\Rightarrow$ 'f list) $\Rightarrow$
 nat $\Rightarrow$ 'f list $\Rightarrow$ 'f set

where

trace-one-step-disagreement-bad-for pw layer claimed doms i prefix =
fri-one-step-disagreement-challenges
 (*trace-fri-canonical-layer-len* i) (pw i)
 (layer i prefix) (claimed i prefix) (doms i prefix)

definition *composition-one-step-disagreement-bad-for*

:: ('f $\Rightarrow$ nat $\Rightarrow$ nat) $\Rightarrow$
 ('f $\Rightarrow$ nat $\Rightarrow$ 'f list $\Rightarrow$ 'f list) $\Rightarrow$
 ('f $\Rightarrow$ nat $\Rightarrow$ 'f list $\Rightarrow$ 'f list) $\Rightarrow$
 ('f $\Rightarrow$ nat $\Rightarrow$ 'f list $\Rightarrow$ 'f list) $\Rightarrow$
 'f $\Rightarrow$ nat $\Rightarrow$ 'f list $\Rightarrow$ 'f set

where

composition-one-step-disagreement-bad-for pw layer claimed doms dg i prefix =
fri-one-step-disagreement-challenges
 (*composition-fri-canonical-layer-len* dg i) (pw dg i)
 (layer dg i prefix) (claimed dg i prefix) (doms dg i prefix)

definition *trace-one-step-sampled-error-for*

where

trace-one-step-sampled-error-for budgets empty-bad =
concrete-trace-fri-bad-challenge-fraction +
hash-relation-budget-value
trace-one-step-challenge-hash-count
 (*staged-attacker-query-budget* budgets +
staged-challenge-query-budget) +
 nnreal (*staged-attacker-query-budget* budgets +
staged-challenge-query-budget) *
trace-one-step-query-fraction-for empty-bad

definition *trace-empty-one-step-error-for*

where

trace-empty-one-step-error-for budgets A empty-bad =
trace-one-step-sampled-error-for budgets empty-bad +

trace-fri-empty-small-conflict-zero-error-for budgets A

definition *trace-header-one-step-error-for*

where

trace-header-one-step-error-for budgets A trace-bad =
trace-one-step-sampled-error-for budgets trace-bad +
trace-fri-header-replay-conflict-zero-error-for budgets A

definition *trace-one-step-split-pw :: nat ⇒ nat*

where

trace-one-step-split-pw - = 0

definition *trace-one-step-split-layer*

:: nat ⇒ 'f list ⇒ 'f list

where

trace-one-step-split-layer i - =
replicate (trace-fri-canonical-layer-len i) 0

definition *trace-one-step-split-claimed*

:: nat ⇒ 'f list ⇒ 'f list

where

trace-one-step-split-claimed i - =
replicate (trace-fri-canonical-layer-len i) 1

definition *trace-one-step-split-domain*

:: nat ⇒ 'f list ⇒ 'f list

where

trace-one-step-split-domain i - =
replicate (trace-fri-canonical-layer-len i) 1

definition *trace-one-step-split-bad*

:: nat ⇒ 'f list ⇒ 'f set

where

trace-one-step-split-bad =
trace-one-step-disagreement-bad-for
trace-one-step-split-pw trace-one-step-split-layer
trace-one-step-split-claimed trace-one-step-split-domain

definition *composition-one-step-split-pw*

:: 'f ⇒ nat ⇒ nat

where

composition-one-step-split-pw - - = 0

definition *composition-one-step-split-layer*

:: 'f ⇒ nat ⇒ 'f list ⇒ 'f list

where

composition-one-step-split-layer dg i - =
replicate (composition-fri-canonical-layer-len dg i) 0

definition *composition-one-step-split-claimed*

$:: 'f \Rightarrow \text{nat} \Rightarrow 'f \text{ list} \Rightarrow 'f \text{ list}$

where

composition-one-step-split-claimed dg i - =
 replicate (*composition-fri-canonical-layer-len* dg i) 1

definition *composition-one-step-split-domain*

$:: 'f \Rightarrow \text{nat} \Rightarrow 'f \text{ list} \Rightarrow 'f \text{ list}$

where

composition-one-step-split-domain dg i - =
 replicate (*composition-fri-canonical-layer-len* dg i) 1

definition *composition-one-step-split-bad*

$:: 'f \Rightarrow \text{nat} \Rightarrow 'f \text{ list} \Rightarrow 'f \text{ set}$

where

composition-one-step-split-bad =
 composition-one-step-disagreement-bad-for
 composition-one-step-split-pw composition-one-step-split-layer
 composition-one-step-split-claimed composition-one-step-split-domain

definition *concrete-trace-header-one-step-error-for*

where

concrete-trace-header-one-step-error-for budgets A =
 trace-header-one-step-error-for budgets A
 trace-one-step-split-bad

definition *concrete-composition-one-step-error-for*

where

concrete-composition-one-step-error-for budgets A =
 composition-one-step-verifier-tied-controlled-error-for budgets
 composition-one-step-split-bad

definition *concrete-trace-empty-one-step-error-for*

where

concrete-trace-empty-one-step-error-for budgets A =
 trace-empty-one-step-error-for budgets A
 trace-one-step-split-bad

lemma *sharpened-trace-fri-full-cover-domain-error-for-eq:*

sharpened-trace-fri-full-cover-domain-error-for budgets A =
 concrete-trace-fri-full-cover-domain-error-for budgets A

by (*simp add:*

sharpened-trace-fri-full-cover-domain-error-for-def
 trace-fri-challenge-prequery-error-for-def
 trace-fri-full-cover-domain-residual-error-for-def
 concrete-trace-fri-full-cover-domain-error-for-def)

lemma *sharpened-trace-fri-header-error-for-eq:*

sharpened-trace-fri-header-error-for budgets A =

concrete-trace-fri-header-error-for' budgets A
by (*simp add:*
sharpened-trace-fri-header-error-for-def
sharpened-trace-fri-full-cover-domain-error-for-eq
trace-fri-header-conflict-zero-error-for-def
concrete-trace-fri-header-error-for'-def)

lemma *sharpened-trace-fri-empty-error-for-eq:*
sharpened-trace-fri-empty-error-for budgets $A =$
concrete-trace-fri-empty-error-for' budgets A
by (*simp add:*
sharpened-trace-fri-empty-error-for-def
sharpened-trace-fri-full-cover-domain-error-for-eq
trace-fri-empty-conflict-zero-error-for-def
concrete-trace-fri-empty-error-for'-def)

lemma *sharpened-composition-fri-full-cover-domain-error-for-eq:*
sharpened-composition-fri-full-cover-domain-error-for budgets $A =$
concrete-composition-fri-full-cover-domain-error-for budgets A
by (*simp add:*
sharpened-composition-fri-full-cover-domain-error-for-def
composition-fri-challenge-prequery-error-for-def
composition-fri-full-cover-domain-residual-error-for-def
concrete-composition-fri-full-cover-domain-error-for-def)

lemma *sharpened-composition-fri-error-for-eq:*
sharpened-composition-fri-error-for budgets $A =$
concrete-composition-fri-error-for' budgets A
by (*simp add:*
sharpened-composition-fri-error-for-def
sharpened-composition-fri-full-cover-domain-error-for-eq
composition-fri-conflict-zero-error-for-def
concrete-composition-fri-error-for'-def)

lemma *sharpened-trace-fri-header-error-for-bound:*
assumes *wf: staged-budget-wellformed* budgets
and *controlled: staged-adversary-controlled* budgets A
shows
wp-event (*checked-staged-security-experiment-with-data-state* A)
(staged-security-with-data-state-verifier-event
trace-fri-bad-with-header-tied-partial-candidate)
adversary-initial-state $\leq$ *sharpened-trace-fri-header-error-for* budgets A
using *concrete-trace-fri-header-error-for'-bound*[*OF wf controlled*]
by (*simp add: sharpened-trace-fri-header-error-for-eq*)

lemma *sharpened-trace-fri-empty-error-for-bound:*
assumes *wf: staged-budget-wellformed* budgets
and *controlled: staged-adversary-controlled* budgets A
shows

$wp\text{-}event$ ($checked\text{-}staged\text{-}security\text{-}experiment\text{-}with\text{-}data\text{-}state$ A)
 ($staged\text{-}security\text{-}with\text{-}data\text{-}state\text{-}verifier\text{-}event$
 $trace\text{-}fri\text{-}bad\text{-}with\text{-}empty\text{-}composition\text{-}header\text{-}candidates$)
 $adversary\text{-}initial\text{-}state \leq sharpened\text{-}trace\text{-}fri\text{-}empty\text{-}error\text{-}for$ budgets A
using $concrete\text{-}trace\text{-}fri\text{-}empty\text{-}error\text{-}for'\text{-}bound[OF$ wf controlled]
by ($simp$ add: $sharpened\text{-}trace\text{-}fri\text{-}empty\text{-}error\text{-}for\text{-}eq$)

lemma $sharpened\text{-}composition\text{-}fri\text{-}error\text{-}for\text{-}bound$:
assumes wf : $staged\text{-}budget\text{-}wellformed$ budgets
and $controlled$: $staged\text{-}adversary\text{-}controlled$ budgets A
shows
 $wp\text{-}event$ ($checked\text{-}staged\text{-}security\text{-}experiment\text{-}with\text{-}data\text{-}state$ A)
 ($staged\text{-}security\text{-}with\text{-}data\text{-}state\text{-}verifier\text{-}event$
 $composition\text{-}fri\text{-}bad\text{-}with\text{-}verifier\text{-}tied\text{-}partial\text{-}candidate$)
 $adversary\text{-}initial\text{-}state \leq sharpened\text{-}composition\text{-}fri\text{-}error\text{-}for$ budgets A
using $concrete\text{-}composition\text{-}fri\text{-}error\text{-}for'\text{-}bound[OF$ wf controlled]
by ($simp$ add: $sharpened\text{-}composition\text{-}fri\text{-}error\text{-}for\text{-}eq$)

lemma $composition\text{-}one\text{-}step\text{-}sampled\text{-}error\text{-}for\text{-}bound$:
fixes $comp\text{-}bad :: 'f \Rightarrow nat \Rightarrow 'f\text{ list} \Rightarrow 'f\text{ set}$
assumes wf : $staged\text{-}budget\text{-}wellformed$ budgets
and $controlled$: $staged\text{-}adversary\text{-}controlled$ budgets A
and $comp\text{-}subset$:
 $\bigwedge dg\ i\ prefix.$
 $i < fri\text{-}round\text{-}count\text{-}for\text{-}degree\text{-}bound$ ($to\text{-}nat\ dg$) $\implies$
 $length\ prefix = i \implies$
 $comp\text{-}bad\ dg\ i\ prefix \subseteq$
 $fri\text{-}one\text{-}step\text{-}disagreement\text{-}challenges$
 ($composition\text{-}fri\text{-}canonical\text{-}layer\text{-}len\ dg\ i$) ($comp\text{-}pw\ dg\ i$)
 ($comp\text{-}layer\ dg\ i\ prefix$) ($comp\text{-}claimed\ dg\ i\ prefix$)
 ($comp\text{-}dom\ dg\ i\ prefix$)

shows
 $wp\text{-}event$ ($checked\text{-}staged\text{-}security\text{-}experiment\text{-}with\text{-}data\text{-}state$ A)
 ($staged\text{-}security\text{-}with\text{-}data\text{-}state\text{-}verifier\text{-}event$
 $composition\text{-}fri\text{-}sampled\text{-}query\text{-}bad\text{-}candidate$)
 $adversary\text{-}initial\text{-}state$
 $\leq composition\text{-}one\text{-}step\text{-}sampled\text{-}error\text{-}for$ budgets $comp\text{-}bad$

proof –
let $?B =$
 $\lambda dg.$ $generic\text{-}fri\text{-}bad\text{-}challenge\text{-}lists$
 ($fri\text{-}round\text{-}count\text{-}for\text{-}degree\text{-}bound$ ($to\text{-}nat\ dg$)) ($comp\text{-}bad\ dg$)
let $?Q = composition\text{-}one\text{-}step\text{-}query\text{-}fraction\text{-}for$ $comp\text{-}bad$
have $round\text{-}bound$:
 $\bigwedge dg\ i\ prefix.$
 $i < fri\text{-}round\text{-}count\text{-}for\text{-}degree\text{-}bound$ ($to\text{-}nat\ dg$) $\implies$
 $length\ prefix = i \implies$
 $card$ ($comp\text{-}bad\ dg\ i\ prefix$)
 $\leq composition\text{-}fri\text{-}canonical\text{-}layer\text{-}len\ dg\ i\ div\ 2$
proof –

```

fix dg i and prefix :: 'f list
assume i: i < fri-round-count-for-degree-bound (to-nat dg)
  and prefix-len: length prefix = i
have card (comp-bad dg i prefix) ≤
  card
    (fri-one-step-disagreement-challenges
      (composition-fri-canonical-layer-len dg i) (comp-pw dg i)
      (comp-layer dg i prefix) (comp-claimed dg i prefix)
      (comp-dom dg i prefix))
  by (rule card-mono) (simp-all add: comp-subset[OF i prefix-len])
also have ... ≤ composition-fri-canonical-layer-len dg i div 2
  by (rule fri-one-step-disagreement-challenges-card-bound-local)
finally show card (comp-bad dg i prefix)
  ≤ composition-fri-canonical-layer-len dg i div 2 .
qed
have fraction-bound:
   $\bigwedge dg. \text{nnreal}$ 
   $(\sum i \in \{.. < \text{fri-round-count-for-degree-bound (to-nat dg)}\}. \text{CARD('f)}^i * \text{composition-fri-canonical-layer-len dg i div 2}) * \text{CARD('f)}^{\text{fri-round-count-for-degree-bound (to-nat dg) - Suc i}} / \text{nnreal (CARD('f)}^{\text{fri-round-count-for-degree-bound (to-nat dg)}})$ 
  ≤ concrete-composition-fri-bad-challenge-fraction
proof -
  fix dg
  show nnreal
   $(\sum i \in \{.. < \text{fri-round-count-for-degree-bound (to-nat dg)}\}. \text{CARD('f)}^i * \text{composition-fri-canonical-layer-len dg i div 2}) * \text{CARD('f)}^{\text{fri-round-count-for-degree-bound (to-nat dg) - Suc i}} / \text{nnreal (CARD('f)}^{\text{fri-round-count-for-degree-bound (to-nat dg)}})$ 
  ≤ concrete-composition-fri-bad-challenge-fraction
  using concrete-composition-fri-bad-challenge-fraction-ge[of dg]
  by (simp add: fri-one-step-disagreement-round-fraction-def
    fri-bad-challenge-round-fraction-def
    fri-one-step-disagreement-round-mass-def
    fri-bad-challenge-round-mass-def)
qed
have fresh-bound:
  wp-event (checked-staged-security-experiment-with-data-state A)
  (staged-security-with-data-state-verifier-event
    (λs. composition-fri-challenge-list-fresh-hit s ?B))
  adversary-initial-state
  ≤ concrete-composition-fri-bad-challenge-fraction
by (rule
  checked-staged-security-composition-fri-bad-challenge-list-fresh-hit-bound-from-round-bounds
  [OF round-bound fraction-bound])

```

```

have fiber-fraction:
  nnreal
  (( $\sum_{dg \in UNIV} \sum_{challenges \in \text{fri-challenge-space } (\text{fri-round-count-for-degree-bound } (to\text{-nat } dg)) - ?B \text{ } dg} \text{card } (\text{generic-fri-sampled-query-query-fiber } (\text{composition-table-low-degree } (to\text{-nat } dg)) (\text{Not } \circ \text{ composition-table-low-degree } \text{maxDegree}) (to\text{-nat } dg) \text{ challenges})) * \text{rounds}) / \text{nnreal } (\text{card } \text{query-sample-space}) \leq ?Q$ )
  by (simp add: composition-one-step-query-fraction-for-def)
have sampled:
  wp-event (checked-staged-security-experiment-with-data-state A)
  (staged-security-with-data-state-verifier-event
    composition-fri-sampled-query-bad-candidate)
  adversary-initial-state
  ≤
  concrete-composition-fri-bad-challenge-fraction +
  hash-relation-budget-value
  ( $\sum_{dg \in UNIV} \text{card } (?B \text{ } dg) * \text{ceil-log } (\text{maxDegree} + 1)$ )
  (staged-attacker-query-budget budgets +
    staged-challenge-query-budget) +
  nnreal
  (staged-attacker-query-budget budgets +
    staged-challenge-query-budget) * ?Q
  by (rule
    checked-staged-security-composition-fri-sampled-query-bad-candidate-bound-from-generic-challenge-cover-and-ou
    [OF wf controlled finite-generic-fri-bad-challenge-lists
      fresh-bound fiber-fraction])
have card-le:
  ( $\sum_{dg \in UNIV} \text{card } (?B \text{ } dg) * \text{ceil-log } (\text{maxDegree} + 1)$ )
  ≤ composition-one-step-challenge-hash-count
proof (unfold composition-one-step-challenge-hash-count-def, rule sum-mono)
fix dg :: 'f
have raw:
  card (?B dg) ≤
  ( $\sum_{i \in \{..<\text{fri-round-count-for-degree-bound } (to\text{-nat } dg)\}} \text{CARD}('f) \wedge i * (\text{composition-fri-canonical-layer-len } dg \text{ } i \text{ div } 2) * \text{CARD}('f) \wedge (\text{fri-round-count-for-degree-bound } (to\text{-nat } dg) - \text{Suc } i)$ )
proof (rule generic-fri-bad-challenge-lists-card-bound-from-round-bounds)
fix i :: nat and prefix :: 'f list
assume i: i < fri-round-count-for-degree-bound (to-nat dg)
and prefix-len: length prefix = i
show card (comp-bad dg i prefix)

```

```

    ≤ composition-fri-canonical-layer-len dg i div 2
  by (rule round-bound[OF i prefix-len])
qed
have card (?B dg) ≤
  fri-one-step-disagreement-round-mass
    (fri-round-count-for-degree-bound (to-nat dg))
    (composition-fri-canonical-layer-len dg)
  using raw
  by (simp add: fri-one-step-disagreement-round-mass-def
    fri-bad-challenge-round-mass-def)
then show card (?B dg) * ceil-log (maxDegree + 1)
  ≤ fri-one-step-disagreement-round-mass
    (fri-round-count-for-degree-bound (to-nat dg))
    (composition-fri-canonical-layer-len dg) *
    ceil-log (maxDegree + 1)
  by (rule mult-right-mono) simp
qed
have budget-le:
  hash-relation-budget-value
    (∑ dg ∈ UNIV. card (?B dg) * ceil-log (maxDegree + 1))
    (staged-attacker-query-budget budgets +
     staged-challenge-query-budget)
  ≤
  hash-relation-budget-value
    composition-one-step-challenge-hash-count
    (staged-attacker-query-budget budgets +
     staged-challenge-query-budget)
  by (rule hash-relation-budget-value-mono-left[OF card-le])
show ?thesis
  by (rule order-trans[OF sampled])
    (use budget-le in
     ⟨simp add: composition-one-step-sampled-error-for-def⟩)
qed

lemma trace-one-step-sampled-error-for-bound:
  fixes empty-bad :: nat ⇒ 'f list ⇒ 'f set
  assumes wf: staged-budget-wellformed budgets
  and controlled: staged-adversary-controlled budgets A
  and empty-subset:
    ∧ i prefix.
    i < fri-round-count-for-degree-bound (clength - 1) ⇒
    length prefix = i ⇒
    empty-bad i prefix ⊆
    fri-one-step-disagreement-challenges
      (trace-fri-canonical-layer-len i) (empty-pw i)
      (empty-layer i prefix) (empty-claimed i prefix)
      (empty-dom i prefix)
  shows
    wp-event (checked-staged-security-experiment-with-data-state A)

```



```

    (staged-security-with-data-state-verifier-event
      trace-fri-sampled-query-bad-candidate)
    adversary-initial-state
  ≤ trace-one-step-sampled-error-for budgets empty-bad
proof –
  let ?B =
    generic-fri-bad-challenge-lists
    (fri-round-count-for-degree-bound (clength – 1)) empty-bad
  let ?Q = trace-one-step-query-fraction-for empty-bad
  have round-bound:
    ∧ i prefix.
      i < fri-round-count-for-degree-bound (clength – 1) ⇒
        length prefix = i ⇒
          card (empty-bad i prefix) ≤ trace-fri-canonical-layer-len i div 2
  proof –
    fix i :: nat and prefix :: 'f list
    assume i: i < fri-round-count-for-degree-bound (clength – 1)
      and prefix-len: length prefix = i
    have card (empty-bad i prefix) ≤
      card
        (fri-one-step-disagreement-challenges
          (trace-fri-canonical-layer-len i) (empty-pw i)
          (empty-layer i prefix) (empty-claimed i prefix)
          (empty-dom i prefix))
      by (rule card-mono) (simp-all add: empty-subset[OF i prefix-len])
    also have ... ≤ trace-fri-canonical-layer-len i div 2
      by (rule fri-one-step-disagreement-challenges-card-bound-local)
    finally show card (empty-bad i prefix)
      ≤ trace-fri-canonical-layer-len i div 2 .
  qed
  have fresh-bound:
    wp-event (checked-staged-security-experiment-with-data-state A)
      (staged-security-with-data-state-verifier-event
        (λs. trace-fri-challenge-list-fresh-hit s ?B))
      adversary-initial-state
    ≤ concrete-trace-fri-bad-challenge-fraction
  proof –
    have base:
      wp-event (checked-staged-security-experiment-with-data-state A)
        (staged-security-with-data-state-verifier-event
          (λs. trace-fri-challenge-list-fresh-hit s ?B))
        adversary-initial-state
      ≤ nnreal
        (∑ i ∈ {..

```

```

    checked-staged-security-trace-fri-bad-challenge-list-fresh-hit-bound-from-round-bounds
    [OF round-bound])
show ?thesis
by (rule order-trans[OF base])
    (simp add: concrete-trace-fri-bad-challenge-fraction-def
      fri-one-step-disagreement-round-fraction-def
      fri-bad-challenge-round-fraction-def
      fri-one-step-disagreement-round-mass-def
      fri-bad-challenge-round-mass-def)
qed
have fiber-fraction:
  nnreal
  (( $\sum$  challenges $\in$ 
    fri-challenge-space (fri-round-count-for-degree-bound (clength - 1)) -
    ?B.
    card
    (generic-fri-sampled-query-query-fiber trace-table-low-degree
      (Not  $\circ$  trace-table-low-degree) (clength - 1) challenges)) *
    rounds) /
    nnreal (card query-sample-space)  $\leq$  ?Q
by (simp add: trace-one-step-query-fraction-for-def)
have sampled:
  wp-event (checked-staged-security-experiment-with-data-state A)
  (staged-security-with-data-state-verifier-event
    trace-fri-sampled-query-bad-candidate)
  adversary-initial-state
   $\leq$ 
  concrete-trace-fri-bad-challenge-fraction +
  hash-relation-budget-value (card ?B * ceil-log clength)
  (staged-attacker-query-budget budgets +
    staged-challenge-query-budget) +
  nnreal
  (staged-attacker-query-budget budgets +
    staged-challenge-query-budget) * ?Q
by (rule
  checked-staged-security-trace-fri-sampled-query-bad-candidate-bound-from-generic-challenge-cover-and-outside-fil
  [OF wf controlled finite-generic-fri-bad-challenge-lists
    fresh-bound fiber-fraction])
have card-le:
  card ?B * ceil-log clength  $\leq$  trace-one-step-challenge-hash-count
proof -
have raw:
  card ?B  $\leq$ 
  ( $\sum i \in \{..< \text{fri-round-count-for-degree-bound (clength - 1)}\}$ .
    CARD('f)  $\wedge$  i * (trace-fri-canonical-layer-len i div 2) *
    CARD('f)  $\wedge$ 
    (fri-round-count-for-degree-bound (clength - 1) - Suc i))
proof (rule generic-fri-bad-challenge-lists-card-bound-from-round-bounds)
fix i :: nat and prefix :: 'f list

```

```

assume  $i: i < \text{fri-round-count-for-degree-bound } (\text{clength} - 1)$ 
  and  $\text{prefix-len}: \text{length prefix} = i$ 
show  $\text{card } (\text{empty-bad } i \text{ prefix})$ 
   $\leq \text{trace-fri-canonical-layer-len } i \text{ div } 2$ 
  by (rule round-bound[OF  $i$  prefix-len])
qed
have  $\text{card } ?B \leq$ 
   $\text{fri-one-step-disagreement-round-mass}$ 
   $(\text{fri-round-count-for-degree-bound } (\text{clength} - 1))$ 
   $\text{trace-fri-canonical-layer-len}$ 
using raw
by (simp add: fri-one-step-disagreement-round-mass-def
  fri-bad-challenge-round-mass-def)
then show ?thesis
  unfolding trace-one-step-challenge-hash-count-def
  by (rule mult-right-mono) simp
qed
have budget-le:
   $\text{hash-relation-budget-value } (\text{card } ?B * \text{ceil-log clength})$ 
   $(\text{staged-attacker-query-budget budgets} +$ 
   $\text{staged-challenge-query-budget})$ 
   $\leq$ 
   $\text{hash-relation-budget-value trace-one-step-challenge-hash-count}$ 
   $(\text{staged-attacker-query-budget budgets} +$ 
   $\text{staged-challenge-query-budget})$ 
  by (rule hash-relation-budget-value-mono-left[OF card-le])
show ?thesis
  by (rule order-trans[OF sampled])
  (use budget-le in ⟨simp add: trace-one-step-sampled-error-for-def⟩)
qed

lemma checked-staged-security-trace-fri-empty-header-bound-from-staged-sampled-conflict-and-zero:
assumes sampled-query:
   $\text{wp-event } (\text{checked-staged-security-experiment-with-data-state } A)$ 
   $(\text{staged-security-with-data-state-verifier-event}$ 
   $\text{trace-fri-sampled-query-bad-candidate})$ 
   $\text{adversary-initial-state} \leq R$ 
and conflict-bound:
   $\text{wp-event } (\text{checked-staged-security-experiment-with-data-state } A)$ 
   $(\text{staged-security-with-data-state-verifier-event}$ 
   $\text{trace-fri-sampled-assignment-conflict})$ 
   $\text{adversary-initial-state} \leq C$ 
and zero-bound:
   $\text{wp-event } (\text{checked-staged-security-experiment-with-data-state } A)$ 
   $(\text{staged-security-with-data-state-verifier-event}$ 
   $\text{trace-fri-zero-round-final-obstruction})$ 
   $\text{adversary-initial-state} \leq Z$ 
shows
   $\text{wp-event } (\text{checked-staged-security-experiment-with-data-state } A)$ 

```

```

    (staged-security-with-data-state-verifier-event
     trace-fri-bad-with-empty-composition-header-candidates)
    adversary-initial-state  $\leq R + (C + Z)$ 
proof –
  let ?M = checked-staged-security-experiment-with-data-state A
  let ?sampled =
    staged-security-with-data-state-verifier-event
    trace-fri-bad-with-sampled-layer-chain
  let ?missing =
    staged-security-with-data-state-verifier-event
    trace-fri-reachable-missing-sampled-layer-chain
  let ?conflict =
    staged-security-with-data-state-verifier-event
    trace-fri-sampled-assignment-conflict
  let ?zero =
    staged-security-with-data-state-verifier-event
    trace-fri-zero-round-final-obstruction
  have sampled:
    wp-event ?M ?sampled adversary-initial-state  $\leq R$ 
  by (rule
    checked-staged-security-trace-fri-sampled-layer-chain-bound-from-sampled-query
    [OF sampled-query])
  have missing-split:
    wp-event ?M ?missing adversary-initial-state  $\leq$ 
    wp-event ?M ( $\lambda$ out. ?conflict out  $\vee$  ?zero out)
    adversary-initial-state
  proof (rule wp-event-mono-on-support)
  fix out
  assume support:
    out  $\in$  set-dist (execute ?M adversary-initial-state)
    and missing: ?missing out
  show ?conflict out  $\vee$  ?zero out
  proof (cases out)
  case None
  then show ?thesis
    using missing unfolding staged-security-with-data-state-verifier-event-def
    by simp
  next
  case (Some out-data)
  then obtain data attacker-state result final-state where out-eq:
    out = Some (((data, attacker-state), result), final-state)
  by (cases out-data, auto split: prod.splits)
  let ?s =
    verifier-state-from-adversary attacker-state
    (staged-proof-transcript data)
  from checked-staged-security-experiment-with-data-state-outcomeE
    [OF support[unfolded out-eq]]
  have verify-support:
    Some (result, final-state)  $\in$  set-dist (execute verify-monad ?s)

```

```

    by blast
  have missing-local:
    trace-fri-reachable-missing-sampled-layer-chain ?s
      (Some (result, final-state))
    using missing unfolding out-eq
      staged-security-with-data-state-verifier-event-def
    by simp
  have obstruction:
    trace-fri-sampled-layer-assignment-obstruction ?s
      (Some (result, final-state))
    by (rule
      trace-fri-reachable-missing-sampled-imp-assignment-obstruction-on-support
      [OF verify-support missing-local])
  then have
    trace-fri-sampled-assignment-conflict ?s
      (Some (result, final-state)) ∨
    trace-fri-zero-round-final-obstruction ?s
      (Some (result, final-state))
    by (rule
      trace-fri-sampled-layer-assignment-obstruction-imp-conflict-or-zero-round-final)
  then show ?thesis
    unfolding out-eq staged-security-with-data-state-verifier-event-def
    by simp
qed
qed
have missing-bound:
  wp-event ?M ?missing adversary-initial-state ≤ C + Z
proof -
  have wp-event ?M ?missing adversary-initial-state ≤
    wp-event ?M (λout. ?conflict out ∨ ?zero out)
      adversary-initial-state
    by (rule missing-split)
  also have ... ≤
    wp-event ?M ?conflict adversary-initial-state +
    wp-event ?M ?zero adversary-initial-state
    by (rule wp-event-union-bound)
  also have ... ≤ C + Z
    by (rule add-mono[OF conflict-bound zero-bound])
  finally show ?thesis .
qed
have split:
  wp-event ?M
    (staged-security-with-data-state-verifier-event
      trace-fri-bad-with-empty-composition-header-candidates)
  adversary-initial-state ≤
  wp-event ?M (λout. ?sampled out ∨ ?missing out)
    adversary-initial-state
  by (rule wp-event-mono-on-support)
  (auto simp: staged-security-with-data-state-verifier-event-def

```

```

    dest:
      trace-fri-bad-with-empty-composition-header-candidates-imp-reachable-partial-candidate
      trace-fri-reachable-imp-sampled-or-missing-layer-chain
    split: option.splits prod.splits)
  also have ... ≤
    wp-event ?M ?sampled adversary-initial-state +
    wp-event ?M ?missing adversary-initial-state
  by (rule wp-event-union-bound)
  also have ... ≤ R + (C + Z)
  by (rule add-mono[OF sampled missing-bound])
  finally show ?thesis .
qed

lemma checked-staged-security-trace-fri-header-tied-candidate-transfer-gap-bound-from-staged-header-and-untied:
  fixes H U :: prob
  assumes header-bound:
    wp-event (checked-staged-security-experiment-with-data-state A)
    (staged-security-with-data-state-verifier-event
     trace-fri-bad-with-header-tied-partial-candidate)
    adversary-initial-state ≤ H
  and untied-bound:
    wp-event (checked-staged-security-experiment-with-data-state A)
    (staged-security-with-data-state-verifier-event
     trace-fri-header-tied-low-candidate-untied-reachable-witness-gap)
    adversary-initial-state ≤ U
  shows
    wp-event (checked-staged-security-experiment-with-data-state A)
    (staged-security-with-data-state-verifier-event
     trace-fri-header-tied-candidate-transfer-gap)
    adversary-initial-state ≤ H + U
proof -
  let ?M = checked-staged-security-experiment-with-data-state A
  let ?header =
    staged-security-with-data-state-verifier-event
    trace-fri-bad-with-header-tied-partial-candidate
  let ?untied =
    staged-security-with-data-state-verifier-event
    trace-fri-header-tied-low-candidate-untied-reachable-witness-gap
  have wp-event ?M
    (staged-security-with-data-state-verifier-event
     trace-fri-header-tied-candidate-transfer-gap)
    adversary-initial-state ≤
    wp-event ?M (λout. ?header out ∨ ?untied out)
    adversary-initial-state
  unfolding staged-security-with-data-state-verifier-event-def
  by (rule wp-event-mono-on-support)
  (auto dest:
    trace-fri-header-tied-candidate-transfer-gap-imp-header-or-untied
    split: option.splits prod.splits)

```

also have ... $\leq$
 $wp\text{-event } ?M \text{ ?header adversary-initial-state} +$
 $wp\text{-event } ?M \text{ ?untied adversary-initial-state}$
 by (rule wp-event-union-bound)
 also have ... $\leq H + U$
 by (rule add-mono[OF header-bound untied-bound])
 finally show ?thesis .
 qed

lemma *checked-staged-security-trace-fri-zero-round-final-obstruction-bound-from-staged-checked-merkle-transfer-head*

fixes $H M Q T :: \text{prob}$

assumes *checked-bound*:

$wp\text{-event } (\text{checked-staged-security-experiment-with-data-state } A)$
 (staged-security-with-data-state-verifier-event
 trace-fri-zero-round-checked-final-obstruction)
 $\text{adversary-initial-state} \leq Q$

and *merkle-bound*:

$wp\text{-event } (\text{checked-staged-security-experiment-with-data-state } A)$
 (staged-security-with-data-state-verifier-event
 partial-merkle-inconsistency-bad)
 $\text{adversary-initial-state} \leq M$

and *transfer-bound*:

$wp\text{-event } (\text{checked-staged-security-experiment-with-data-state } A)$
 (staged-security-with-data-state-verifier-event
 trace-fri-header-tied-candidate-transfer-gap)
 $\text{adversary-initial-state} \leq T$

and *header-bound*:

$wp\text{-event } (\text{checked-staged-security-experiment-with-data-state } A)$
 (staged-security-with-data-state-verifier-event
 trace-fri-bad-with-header-tied-partial-candidate)
 $\text{adversary-initial-state} \leq H$

shows

$wp\text{-event } (\text{checked-staged-security-experiment-with-data-state } A)$
 (staged-security-with-data-state-verifier-event
 trace-fri-zero-round-final-obstruction)
 $\text{adversary-initial-state} \leq Q + M + T + H$

proof –

let $?M = \text{checked-staged-security-experiment-with-data-state } A$

let $?checked =$

staged-security-with-data-state-verifier-event
 trace-fri-zero-round-checked-final-obstruction

let $?merkle =$

staged-security-with-data-state-verifier-event
 partial-merkle-inconsistency-bad

let $?transfer =$

staged-security-with-data-state-verifier-event
 trace-fri-header-tied-candidate-transfer-gap

let $?header =$

staged-security-with-data-state-verifier-event

```

    trace-fri-bad-with-header-tied-partial-candidate
have wp-event ?M
  (staged-security-with-data-state-verifier-event
   trace-fri-zero-round-final-obstruction)
  adversary-initial-state ≤
  wp-event ?M
  (λout.
   ?checked out ∨ ?merkle out ∨ ?transfer out ∨ ?header out)
  adversary-initial-state
proof (rule wp-event-mono-on-support)
fix out
assume support:
  out ∈ set-dist (execute ?M adversary-initial-state)
and obstruction:
  staged-security-with-data-state-verifier-event
  trace-fri-zero-round-final-obstruction out
show ?checked out ∨ ?merkle out ∨ ?transfer out ∨ ?header out
proof (cases out)
  case None
  then show ?thesis
    using obstruction
    unfolding staged-security-with-data-state-verifier-event-def
    by simp
next
  case (Some out-data)
  then obtain data attacker-state result final-state where out-eq:
    out = Some (((data, attacker-state), result), final-state)
    by (cases out-data, auto split: prod.splits)
  let ?s =
    verifier-state-from-adversary attacker-state
    (staged-proof-transcript data)
  from checked-staged-security-experiment-with-data-state-outcomeE
    [OF support[unfolded out-eq]]
  have verify-support:
    Some (result, final-state) ∈ set-dist (execute verify-monad ?s)
    by blast
  have obstruction-local:
    trace-fri-zero-round-final-obstruction ?s
    (Some (result, final-state))
    using obstruction
    unfolding out-eq staged-security-with-data-state-verifier-event-def
    by simp
  have
    trace-fri-zero-round-checked-final-obstruction ?s
    (Some (result, final-state)) ∨
    partial-merkle-inconsistency-bad ?s
    (Some (result, final-state)) ∨
    trace-fri-header-tied-candidate-transfer-gap ?s
    (Some (result, final-state)) ∨

```



```

    trace-fri-bad-with-header-tied-partial-candidate ?s
    (Some (result, final-state))
  by (rule
    trace-fri-zero-round-final-obstruction-imp-checked-merkle-transfer-or-header
    [OF verify-support obstruction-local])
  then show ?thesis
    unfolding out-eq staged-security-with-data-state-verifier-event-def
    by simp
qed
qed
also have ... ≤
  wp-event ?M ?checked adversary-initial-state +
  wp-event ?M ?merkle adversary-initial-state +
  wp-event ?M ?transfer adversary-initial-state +
  wp-event ?M ?header adversary-initial-state
  by (rule wp-event-union-bound4)
also have ... ≤ Q + M + T + H
  by (intro add-mono checked-bound merkle-bound transfer-bound
    header-bound)
finally show ?thesis .
qed

lemma checked-staged-security-trace-fri-zero-round-checked-final-obstruction-bound-from-agreement-sets:
  assumes wf: staged-budget-wellformed budgets
  and controlled: staged-adversary-controlled budgets A
  shows
    wp-event (checked-staged-security-experiment-with-data-state A)
    (staged-security-with-data-state-verifier-event
      trace-fri-zero-round-checked-final-obstruction)
    adversary-initial-state ≤
    trace-fri-zero-round-agreement-set-target-error-for budgets
proof -
  have event-le:
    wp-event (checked-staged-security-experiment-with-data-state A)
    (staged-security-with-data-state-verifier-event
      trace-fri-zero-round-checked-final-obstruction)
    adversary-initial-state ≤
    wp-event (checked-staged-security-experiment-with-data-state A)
    (staged-security-with-data-state-verifier-event
      trace-fri-zero-round-query-list-target-hit)
    adversary-initial-state
  unfolding staged-security-with-data-state-verifier-event-def
  by (rule wp-event-mono-on-support)
  (auto
    intro:
      trace-fri-zero-round-checked-first-query-target-hit-imp-query-list-target
    dest:
      trace-fri-zero-round-checked-final-obstruction-imp-first-query-target-hit
    split: option.splits prod.splits)

```

```

show ?thesis
  by (rule order-trans[OF event-le])
    (rule checked-staged-security-trace-fri-zero-round-query-list-target-bound-from-agreement-sets
      [OF wf controlled])
qed

lemma checked-staged-security-trace-fri-zero-round-final-obstruction-bound-from-agreement-sets:
assumes wf: staged-budget-wellformed budgets
and controlled: staged-adversary-controlled budgets A
shows
  wp-event (checked-staged-security-experiment-with-data-state A)
    (staged-security-with-data-state-verifier-event
      trace-fri-zero-round-final-obstruction)
  adversary-initial-state ≤
  trace-fri-zero-round-agreement-set-target-error-for budgets +
  trace-fri-controlled-merkle-error-for budgets +
  reachable-verifier-event-bound-for A
  trace-fri-bad-with-header-tied-partial-candidate +
  (trace-fri-controlled-merkle-error-for budgets +
    (reachable-verifier-event-bound-for A
      trace-fri-header-tied-low-candidate-untied-root-gap +
      reachable-verifier-event-bound-for A
      trace-fri-header-tied-low-candidate-untied-query-gap)) +
  reachable-verifier-event-bound-for A
  trace-fri-bad-with-header-tied-partial-candidate
proof –
have checked:
  wp-event (checked-staged-security-experiment-with-data-state A)
    (staged-security-with-data-state-verifier-event
      trace-fri-zero-round-checked-final-obstruction)
  adversary-initial-state ≤
  trace-fri-zero-round-agreement-set-target-error-for budgets
by (rule checked-staged-security-trace-fri-zero-round-checked-final-obstruction-bound-from-agreement-sets
    [OF wf controlled])
have merkle:
  wp-event (checked-staged-security-experiment-with-data-state A)
    (staged-security-with-data-state-verifier-event
      partial-merkle-inconsistency-bad)
  adversary-initial-state ≤
  trace-fri-controlled-merkle-error-for budgets
unfolding trace-fri-controlled-merkle-error-for-def
by (rule
  checked-staged-security-with-data-state-partial-merkle-inconsistency-bad-bound-controlled
    [OF wf controlled])
have header:
  wp-event (checked-staged-security-experiment-with-data-state A)
    (staged-security-with-data-state-verifier-event
      trace-fri-bad-with-header-tied-partial-candidate)
  adversary-initial-state ≤

```

```

    reachable-verifier-event-bound-for A
    trace-fri-bad-with-header-tied-partial-candidate
proof (rule checked-staged-security-with-data-state-verifier-event-bound-from-cont)
  show  $\bigwedge s. \neg \text{trace-fri-bad-with-header-tied-partial-candidate } s \text{ None}$ 
    unfolding trace-fri-bad-with-header-tied-partial-candidate-def
      accepted-fri-opening-transcript-def
    by simp
next
  fix data attacker-state
  assume support: Some (data, attacker-state)  $\in$ 
    set-dist
    (execute (checked-staged-transcript-program A)
      adversary-initial-state)
  show wp-event verify-monad
    (trace-fri-bad-with-header-tied-partial-candidate
      (verifier-state-from-adversary attacker-state
        (staged-proof-transcript data)))
    (verifier-state-from-adversary attacker-state
      (staged-proof-transcript data))
     $\leq$  reachable-verifier-event-bound-for A
      trace-fri-bad-with-header-tied-partial-candidate
    by (rule reachable-verifier-event-bound-for-ge[OF support])
qed
have hash:
  wp-event (checked-staged-security-experiment-with-data-state A)
    (staged-security-with-data-state-verifier-event
      hash-map-output-collision-bad)
  adversary-initial-state  $\leq$ 
    trace-fri-controlled-merkle-error-for budgets
  unfolding trace-fri-controlled-merkle-error-for-def
  by (rule
    checked-staged-security-with-data-state-hash-map-output-collision-bad-bound-controlled
    [OF wf controlled])
have root:
  wp-event (checked-staged-security-experiment-with-data-state A)
    (staged-security-with-data-state-verifier-event
      trace-fri-header-tied-low-candidate-untied-root-gap)
  adversary-initial-state  $\leq$ 
    reachable-verifier-event-bound-for A
      trace-fri-header-tied-low-candidate-untied-root-gap
proof (rule checked-staged-security-with-data-state-verifier-event-bound-from-cont)
  show  $\bigwedge s. \neg \text{trace-fri-header-tied-low-candidate-untied-root-gap } s \text{ None}$ 
    unfolding trace-fri-header-tied-low-candidate-untied-root-gap-def
      trace-fri-header-tied-low-candidate-untied-reachable-witness-case-def
      accepted-fri-opening-transcript-def
    by simp
next
  fix data attacker-state
  assume support: Some (data, attacker-state)  $\in$ 

```

```

    set-dist
      (execute (checked-staged-transcript-program A)
        adversary-initial-state)
  show wp-event verify-monad
    (trace-fri-header-tied-low-candidate-untied-root-gap
      (verifier-state-from-adversary attacker-state
        (staged-proof-transcript data)))
    (verifier-state-from-adversary attacker-state
      (staged-proof-transcript data))
  ≤ reachable-verifier-event-bound-for A
    trace-fri-header-tied-low-candidate-untied-root-gap
  by (rule reachable-verifier-event-bound-for-ge[OF support])
qed
have query:
  wp-event (checked-staged-security-experiment-with-data-state A)
    (staged-security-with-data-state-verifier-event
      trace-fri-header-tied-low-candidate-untied-query-gap)
  adversary-initial-state ≤
    reachable-verifier-event-bound-for A
      trace-fri-header-tied-low-candidate-untied-query-gap
proof (rule checked-staged-security-with-data-state-verifier-event-bound-from-cont)
  show  $\bigwedge s. \neg \text{trace-fri-header-tied-low-candidate-untied-query-gap } s \text{ None}$ 
  unfolding trace-fri-header-tied-low-candidate-untied-query-gap-def
    trace-fri-header-tied-low-candidate-untied-reachable-witness-case-def
    accepted-fri-opening-transcript-def
  by simp
next
  fix data attacker-state
  assume support: Some (data, attacker-state) ∈
    set-dist
      (execute (checked-staged-transcript-program A)
        adversary-initial-state)
  show wp-event verify-monad
    (trace-fri-header-tied-low-candidate-untied-query-gap
      (verifier-state-from-adversary attacker-state
        (staged-proof-transcript data)))
    (verifier-state-from-adversary attacker-state
      (staged-proof-transcript data))
  ≤ reachable-verifier-event-bound-for A
    trace-fri-header-tied-low-candidate-untied-query-gap
  by (rule reachable-verifier-event-bound-for-ge[OF support])
qed
have untied:
  wp-event (checked-staged-security-experiment-with-data-state A)
    (staged-security-with-data-state-verifier-event
      trace-fri-header-tied-low-candidate-untied-reachable-witness-gap)
  adversary-initial-state ≤
    trace-fri-controlled-merkle-error-for budgets +
    (reachable-verifier-event-bound-for A

```

```

    trace-fri-header-tied-low-candidate-untied-root-gap +
    reachable-verifier-event-bound-for A
    trace-fri-header-tied-low-candidate-untied-query-gap)
proof -
  let ?M = checked-staged-security-experiment-with-data-state A
  let ?untied =
    staged-security-with-data-state-verifier-event
    trace-fri-header-tied-low-candidate-untied-reachable-witness-gap
  let ?hash =
    staged-security-with-data-state-verifier-event
    hash-map-output-collision-bad
  let ?root =
    staged-security-with-data-state-verifier-event
    trace-fri-header-tied-low-candidate-untied-root-gap
  let ?query =
    staged-security-with-data-state-verifier-event
    trace-fri-header-tied-low-candidate-untied-query-gap
  have split:
    wp-event ?M ?untied adversary-initial-state ≤
      wp-event ?M (λout. ?hash out ∨ ?root out ∨ ?query out)
      adversary-initial-state
  unfolding staged-security-with-data-state-verifier-event-def
  by (rule wp-event-mono-on-support)
    (auto
     dest:
      trace-fri-header-tied-low-candidate-untied-gap-imp-hash-root-or-query
      split: option.splits prod.splits)
  also have ... ≤
    wp-event ?M ?hash adversary-initial-state +
    wp-event ?M (λout. ?root out ∨ ?query out)
    adversary-initial-state
  by (rule wp-event-union-bound)
  also have ... ≤
    wp-event ?M ?hash adversary-initial-state +
    (wp-event ?M ?root adversary-initial-state +
     wp-event ?M ?query adversary-initial-state)
  by (intro add-mono order-refl wp-event-union-bound)
  also have ... ≤
    trace-fri-controlled-merkle-error-for budgets +
    (reachable-verifier-event-bound-for A
     trace-fri-header-tied-low-candidate-untied-root-gap +
     reachable-verifier-event-bound-for A
     trace-fri-header-tied-low-candidate-untied-query-gap)
  by (intro add-mono hash root query)
  finally show ?thesis .
qed
have transfer:
  wp-event (checked-staged-security-experiment-with-data-state A)
    (staged-security-with-data-state-verifier-event

```

$\text{trace-fri-header-tied-candidate-transfer-gap})$
 $\text{adversary-initial-state} \leq$
 $\text{reachable-verifier-event-bound-for } A$
 $\text{trace-fri-bad-with-header-tied-partial-candidate} +$
 $(\text{trace-fri-controlled-merkle-error-for budgets} +$
 $(\text{reachable-verifier-event-bound-for } A$
 $\text{trace-fri-header-tied-low-candidate-untied-root-gap} +$
 $\text{reachable-verifier-event-bound-for } A$
 $\text{trace-fri-header-tied-low-candidate-untied-query-gap}))$
by (rule
 $\text{checked-staged-security-trace-fri-header-tied-candidate-transfer-gap-bound-from-staged-header-and-untied}$
 $[\text{OF header untied}]$)
have raw:
 $\text{wp-event } (\text{checked-staged-security-experiment-with-data-state } A)$
 $(\text{staged-security-with-data-state-verifier-event}$
 $\text{trace-fri-zero-round-final-obstruction})$
 $\text{adversary-initial-state} \leq$
 $\text{trace-fri-zero-round-agreement-set-target-error-for budgets} +$
 $\text{trace-fri-controlled-merkle-error-for budgets} +$
 $(\text{reachable-verifier-event-bound-for } A$
 $\text{trace-fri-bad-with-header-tied-partial-candidate} +$
 $(\text{trace-fri-controlled-merkle-error-for budgets} +$
 $(\text{reachable-verifier-event-bound-for } A$
 $\text{trace-fri-header-tied-low-candidate-untied-root-gap} +$
 $\text{reachable-verifier-event-bound-for } A$
 $\text{trace-fri-header-tied-low-candidate-untied-query-gap}))) +$
 $\text{reachable-verifier-event-bound-for } A$
 $\text{trace-fri-bad-with-header-tied-partial-candidate}$
by (rule
 $\text{checked-staged-security-trace-fri-zero-round-final-obstruction-bound-from-staged-checked-merkle-transfer-header}$
 $[\text{OF checked merkle transfer header}]$)
show ?thesis
by (rule order-trans[OF raw]) (simp add: algebra-simps)
qed

lemma $\text{checked-staged-security-composition-fri-verifier-tied-partial-candidate-bound-from-staged-sampled-conflict-and}$
assumes sampled-bound :

$\text{wp-event } (\text{checked-staged-security-experiment-with-data-state } A)$
 $(\text{staged-security-with-data-state-verifier-event}$
 $\text{composition-fri-bad-with-verifier-tied-sampled-layer-chain})$
 $\text{adversary-initial-state} \leq R$

and conflict-bound :

$\text{wp-event } (\text{checked-staged-security-experiment-with-data-state } A)$
 $(\text{staged-security-with-data-state-verifier-event}$
 $\text{composition-fri-verifier-tied-sampled-assignment-conflict})$
 $\text{adversary-initial-state} \leq C$

and zero-bound :

$\text{wp-event } (\text{checked-staged-security-experiment-with-data-state } A)$
 $(\text{staged-security-with-data-state-verifier-event}$

```

    composition-fri-verifier-tied-zero-round-final-obstruction)
  adversary-initial-state ≤ Z
shows
  wp-event (checked-staged-security-experiment-with-data-state A)
    (staged-security-with-data-state-verifier-event
      composition-fri-bad-with-verifier-tied-partial-candidate)
  adversary-initial-state ≤ R + (C + Z)
proof -
  let ?M = checked-staged-security-experiment-with-data-state A
  let ?sampled =
    staged-security-with-data-state-verifier-event
      composition-fri-bad-with-verifier-tied-sampled-layer-chain
  let ?missing =
    staged-security-with-data-state-verifier-event
      composition-fri-verifier-tied-missing-sampled-layer-chain
  let ?conflict =
    staged-security-with-data-state-verifier-event
      composition-fri-verifier-tied-sampled-assignment-conflict
  let ?zero =
    staged-security-with-data-state-verifier-event
      composition-fri-verifier-tied-zero-round-final-obstruction
  have missing-split:
    wp-event ?M ?missing adversary-initial-state ≤
      wp-event ?M (λout. ?conflict out ∨ ?zero out)
      adversary-initial-state
  proof (rule wp-event-mono-on-support)
    fix out
    assume support:
      out ∈ set-dist (execute ?M adversary-initial-state)
    and missing: ?missing out
    show ?conflict out ∨ ?zero out
  proof (cases out)
    case None
    then show ?thesis
      using missing unfolding staged-security-with-data-state-verifier-event-def
      by simp
  next
    case (Some out-data)
    then obtain data attacker-state result final-state where out-eq:
      out = Some (((data, attacker-state), result), final-state)
    by (cases out-data, auto split: prod.splits)
    let ?s =
      verifier-state-from-adversary attacker-state
      (staged-proof-transcript data)
    from checked-staged-security-experiment-with-data-state-outcomeE
      [OF support[unfolded out-eq]]
    have verify-support:
      Some (result, final-state) ∈ set-dist (execute verify-monad ?s)
    by blast

```

```

have missing-local:
  composition-fri-verifier-tied-missing-sampled-layer-chain ?s
    (Some (result, final-state))
using missing unfolding out-eq
  staged-security-with-data-state-verifier-event-def
by simp
have obstruction:
  composition-fri-verifier-tied-sampled-layer-assignment-obstruction
    ?s (Some (result, final-state))
by (rule
  composition-fri-verifier-tied-missing-sampled-imp-assignment-obstruction
  [OF missing-local])
then have
  composition-fri-verifier-tied-sampled-assignment-conflict ?s
    (Some (result, final-state)) ∨
  composition-fri-verifier-tied-zero-round-final-obstruction ?s
    (Some (result, final-state))
by (rule
  composition-fri-verifier-tied-assignment-obstruction-imp-conflict-or-zero-on-support
  [OF verify-support])
then show ?thesis
  unfolding out-eq staged-security-with-data-state-verifier-event-def
by simp
qed
qed
have missing-bound:
  wp-event ?M ?missing adversary-initial-state ≤ C + Z
proof –
  have wp-event ?M ?missing adversary-initial-state ≤
    wp-event ?M (λout. ?conflict out ∨ ?zero out)
      adversary-initial-state
    by (rule missing-split)
  also have ... ≤
    wp-event ?M ?conflict adversary-initial-state +
    wp-event ?M ?zero adversary-initial-state
    by (rule wp-event-union-bound)
  also have ... ≤ C + Z
    by (rule add-mono[OF conflict-bound zero-bound])
  finally show ?thesis .
qed
have split:
  wp-event ?M
    (staged-security-with-data-state-verifier-event
      composition-fri-bad-with-verifier-tied-partial-candidate)
  adversary-initial-state ≤
  wp-event ?M (λout. ?sampled out ∨ ?missing out)
    adversary-initial-state
unfolding composition-fri-verifier-tied-missing-sampled-layer-chain-def
by (rule wp-event-mono-on-support)

```



```

    (auto simp: staged-security-with-data-state-verifier-event-def
      split: option.splits prod.splits)
  also have ... ≤
    wp-event ?M ?sampled adversary-initial-state +
    wp-event ?M ?missing adversary-initial-state
  by (rule wp-event-union-bound)
  also have ... ≤ R + (C + Z)
  by (rule add-mono[OF sampled-bound missing-bound])
  finally show ?thesis .
qed

lemma checked-staged-security-partial-merkle-inconsistency-bad-bound-by-composition-one-step-controlled-merkle-err
  assumes wf: staged-budget-wellformed budgets
    and controlled: staged-adversary-controlled budgets A
  shows
    wp-event (checked-staged-security-experiment-with-data-state A)
      (staged-security-with-data-state-verifier-event
        partial-merkle-inconsistency-bad)
      adversary-initial-state ≤
        composition-one-step-controlled-merkle-error-for budgets
  unfolding composition-one-step-controlled-merkle-error-for-def
  by (rule
    checked-staged-security-with-data-state-partial-merkle-inconsistency-bad-bound-controlled
    [OF wf controlled])

lemma checked-staged-security-composition-fri-verifier-tied-authenticated-slot-recorded-chunk-gap-zero:
  wp-event (checked-staged-security-experiment-with-data-state A)
    (staged-security-with-data-state-verifier-event
      composition-fri-verifier-tied-authenticated-slot-recorded-chunk-gap)
    adversary-initial-state ≤ (0::prob)
proof (rule checked-staged-security-with-data-state-verifier-event-bound-from-cont)
  show  $\bigwedge s. \neg$ 
    composition-fri-verifier-tied-authenticated-slot-recorded-chunk-gap s None
  unfolding
    composition-fri-verifier-tied-authenticated-slot-recorded-chunk-gap-def
  by simp
next
  fix data attacker-state
  assume Some (data, attacker-state) ∈
    set-dist
      (execute (checked-staged-transcript-program A)
        adversary-initial-state)
  show wp-event verify-monad
    (composition-fri-verifier-tied-authenticated-slot-recorded-chunk-gap
      (verifier-state-from-adversary attacker-state
        (staged-proof-transcript data)))
    (verifier-state-from-adversary attacker-state
      (staged-proof-transcript data))
    ≤ 0

```

```

    by (rule
      wp-composition-fri-verifier-tied-authenticated-slot-recorded-chunk-gap-zero)
qed

lemma checked-staged-security-composition-fri-verifier-tied-next-value-replay-gap-bound-by-controlled-merkle:
  assumes wf: staged-budget-wellformed budgets
    and controlled: staged-adversary-controlled budgets A
  shows
    wp-event (checked-staged-security-experiment-with-data-state A)
      (staged-security-with-data-state-verifier-event
        composition-fri-verifier-tied-next-value-replay-gap)
      adversary-initial-state ≤
      composition-one-step-controlled-merkle-error-for budgets + 0
proof -
  let ?M = checked-staged-security-experiment-with-data-state A
  let ?merkle =
    staged-security-with-data-state-verifier-event
    partial-merkle-inconsistency-bad
  let ?slot =
    staged-security-with-data-state-verifier-event
    composition-fri-verifier-tied-authenticated-slot-recorded-chunk-gap
  have split:
    wp-event ?M
      (staged-security-with-data-state-verifier-event
        composition-fri-verifier-tied-next-value-replay-gap)
      adversary-initial-state ≤
      wp-event ?M (λout. ?merkle out ∨ ?slot out)
      adversary-initial-state
  proof (rule wp-event-mono-on-support)
    fix out
    assume support:
      out ∈ set-dist (execute ?M adversary-initial-state)
    and gap:
      staged-security-with-data-state-verifier-event
      composition-fri-verifier-tied-next-value-replay-gap out
    show ?merkle out ∨ ?slot out
  proof (cases out)
    case None
    then show ?thesis
      using gap unfolding staged-security-with-data-state-verifier-event-def
      by simp
  next
    case (Some out-data)
    then obtain data attacker-state result final-state where out-eq:
      out = Some (((data, attacker-state), result), final-state)
    by (cases out-data, auto split: prod.splits)
    let ?s =
      verifier-state-from-adversary attacker-state
      (staged-proof-transcript data)

```

```

from checked-staged-security-experiment-with-data-state-outcomeE
  [OF support[unfolded out-eq]]
have verify-support:
  Some (result, final-state) ∈ set-dist (execute verify-monad ?s)
  by blast
have gap-local:
  composition-fri-verifier-tied-next-value-replay-gap ?s
  (Some (result, final-state))
using gap unfolding out-eq
  staged-security-with-data-state-verifier-event-def
by simp
have
  partial-merkle-inconsistency-bad ?s
  (Some (result, final-state)) ∨
  composition-fri-verifier-tied-authenticated-slot-recorded-chunk-gap
  ?s (Some (result, final-state))
by (rule
  composition-fri-verifier-tied-next-value-replay-gap-imp-merkle-or-slot-on-support
  [OF verify-support gap-local])
then show ?thesis
  unfolding out-eq staged-security-with-data-state-verifier-event-def
  by simp
qed
qed
also have ... ≤
  wp-event ?M ?merkle adversary-initial-state +
  wp-event ?M ?slot adversary-initial-state
by (rule wp-event-union-bound)
also have ... ≤
  composition-one-step-controlled-merkle-error-for budgets + 0
by (intro add-mono
  checked-staged-security-partial-merkle-inconsistency-bad-bound-by-composition-one-step-controlled-merkle-error
  [OF wf controlled]
  checked-staged-security-composition-fri-verifier-tied-authenticated-slot-recorded-chunk-gap-zero)
finally show ?thesis .
qed

lemma checked-staged-security-composition-fri-verifier-tied-successor-replay-gap-bound-by-controlled-merkle:
assumes wf: staged-budget-wellformed budgets
and controlled: staged-adversary-controlled budgets A
shows
  wp-event (checked-staged-security-experiment-with-data-state A)
  (staged-security-with-data-state-verifier-event
  composition-fri-verifier-tied-successor-replay-gap)
  adversary-initial-state ≤
  composition-one-step-controlled-merkle-error-for budgets + 0
proof -
let ?M = checked-staged-security-experiment-with-data-state A
let ?merkle =

```

```

    staged-security-with-data-state-verifier-event
    partial-merkle-inconsistency-bad
  let ?slot =
    staged-security-with-data-state-verifier-event
    composition-fri-verifier-tied-authenticated-slot-recorded-chunk-gap
  have split:
    wp-event ?M
    (staged-security-with-data-state-verifier-event
     composition-fri-verifier-tied-successor-replay-gap)
    adversary-initial-state ≤
    wp-event ?M (λout. ?merkle out ∨ ?slot out)
    adversary-initial-state
  proof (rule wp-event-mono-on-support)
    fix out
    assume support:
      out ∈ set-dist (execute ?M adversary-initial-state)
    and gap:
      staged-security-with-data-state-verifier-event
      composition-fri-verifier-tied-successor-replay-gap out
    show ?merkle out ∨ ?slot out
    proof (cases out)
      case None
      then show ?thesis
        using gap unfolding staged-security-with-data-state-verifier-event-def
        by simp
    next
      case (Some out-data)
      then obtain data attacker-state result final-state where out-eq:
        out = Some (((data, attacker-state), result), final-state)
        by (cases out-data, auto split: prod.splits)
      let ?s =
        verifier-state-from-adversary attacker-state
        (staged-proof-transcript data)
      from checked-staged-security-experiment-with-data-state-outcomeE
        [OF support[unfolded out-eq]]
      have verify-support:
        Some (result, final-state) ∈ set-dist (execute verify-monad ?s)
        by blast
      have gap-local:
        composition-fri-verifier-tied-successor-replay-gap ?s
        (Some (result, final-state))
      using gap unfolding out-eq
        staged-security-with-data-state-verifier-event-def
        by simp
      have
        partial-merkle-inconsistency-bad ?s
        (Some (result, final-state)) ∨
        composition-fri-verifier-tied-authenticated-slot-recorded-chunk-gap
        ?s (Some (result, final-state))

```

```

    by (rule
      composition-fri-verifier-tied-successor-replay-gap-imp-merkle-or-slot-on-support
      [OF verify-support gap-local])
  then show ?thesis
    unfolding out-eq staged-security-with-data-state-verifier-event-def
    by simp
qed
qed
also have ... ≤
  wp-event ?M ?merkle adversary-initial-state +
  wp-event ?M ?slot adversary-initial-state
  by (rule wp-event-union-bound)
also have ... ≤
  composition-one-step-controlled-merkle-error-for budgets + 0
  by (intro add-mono
    checked-staged-security-partial-merkle-inconsistency-bad-bound-by-composition-one-step-controlled-merkle-error
    [OF wf controlled]
    checked-staged-security-composition-fri-verifier-tied-authenticated-slot-recorded-chunk-gap-zero)
  finally show ?thesis .
qed

lemma checked-staged-security-composition-fri-verifier-tied-final-value-selected-step-missing-gap-zero:
  wp-event (checked-staged-security-experiment-with-data-state A)
  (staged-security-with-data-state-verifier-event
    composition-fri-verifier-tied-final-value-selected-step-missing-gap)
  adversary-initial-state ≤ (0::prob)
proof (rule checked-staged-security-with-data-state-verifier-event-bound-from-cont)
  show  $\bigwedge s. \neg$ 
    composition-fri-verifier-tied-final-value-selected-step-missing-gap s None
  unfolding
    composition-fri-verifier-tied-final-value-selected-step-missing-gap-def
    composition-fri-verifier-tied-final-value-authenticated-replay-gap-def
    accepted-fri-opening-transcript-def
  by simp
next
  fix data attacker-state
  assume Some (data, attacker-state) ∈
    set-dist
    (execute (checked-staged-transcript-program A)
      adversary-initial-state)
  show wp-event verify-monad
    (composition-fri-verifier-tied-final-value-selected-step-missing-gap
      (verifier-state-from-adversary attacker-state
        (staged-proof-transcript data)))
    (verifier-state-from-adversary attacker-state
      (staged-proof-transcript data))
    ≤ 0
  by (rule
    wp-composition-fri-verifier-tied-final-value-selected-step-missing-gap-zero)

```

qed

lemma *checked-staged-security-composition-fri-verifier-tied-final-value-replay-gap-bound-by-controlled-merkle:*

assumes *wf: staged-budget-wellformed budgets*

and *controlled: staged-adversary-controlled budgets A*

shows

wp-event (checked-staged-security-experiment-with-data-state A)
(staged-security-with-data-state-verifier-event
composition-fri-verifier-tied-final-value-replay-gap)
adversary-initial-state $\leq$
composition-one-step-controlled-merkle-error-for budgets + 0 + 0

proof –

let *?M = checked-staged-security-experiment-with-data-state A*

let *?merkle =*

staged-security-with-data-state-verifier-event
partial-merkle-inconsistency-bad

let *?slot =*

staged-security-with-data-state-verifier-event
composition-fri-verifier-tied-authenticated-slot-recorded-chunk-gap

let *?selected =*

staged-security-with-data-state-verifier-event
composition-fri-verifier-tied-final-value-selected-step-missing-gap

have *split:*

wp-event ?M
(staged-security-with-data-state-verifier-event
composition-fri-verifier-tied-final-value-replay-gap)
adversary-initial-state $\leq$
wp-event ?M
(λ out. ?merkle out $\vee$?slot out $\vee$?selected out)
adversary-initial-state

proof (rule *wp-event-mono-on-support*)

fix *out*

assume *support:*

out $\in$ *set-dist (execute ?M adversary-initial-state)*

and *gap:*

staged-security-with-data-state-verifier-event
composition-fri-verifier-tied-final-value-replay-gap out

show *?merkle out $\vee$?slot out $\vee$?selected out*

proof (cases *out*)

case *None*

then show *?thesis*

using *gap unfolding staged-security-with-data-state-verifier-event-def*
by *simp*

next

case (*Some out-data*)

then obtain *data attacker-state result final-state* **where** *out-eq:*

out = Some (((data, attacker-state), result), final-state)

by (cases *out-data*, auto *split: prod.splits*)

let *?s =*

```

    verifier-state-from-adversary attacker-state
    (staged-proof-transcript data)
from checked-staged-security-experiment-with-data-state-outcomeE
    [OF support[unfolded out-eq]]
have verify-support:
    Some (result, final-state) ∈ set-dist (execute verify-monad ?s)
    by blast
have gap-local:
    composition-fri-verifier-tied-final-value-replay-gap ?s
    (Some (result, final-state))
    using gap unfolding out-eq
    staged-security-with-data-state-verifier-event-def
    by simp
have
    partial-merkle-inconsistency-bad ?s
    (Some (result, final-state)) ∨
    composition-fri-verifier-tied-authenticated-slot-recorded-chunk-gap
    ?s (Some (result, final-state)) ∨
    composition-fri-verifier-tied-final-value-authenticated-replay-gap
    ?s (Some (result, final-state))
    by (rule
    composition-fri-verifier-tied-final-value-replay-gap-imp-merkle-or-slot-or-authenticated-on-support
    [OF verify-support gap-local])
then show ?thesis
proof
    assume partial-merkle-inconsistency-bad ?s
    (Some (result, final-state))
    then show ?thesis
    unfolding out-eq staged-security-with-data-state-verifier-event-def
    by simp
next
    assume tail:
    composition-fri-verifier-tied-authenticated-slot-recorded-chunk-gap
    ?s (Some (result, final-state)) ∨
    composition-fri-verifier-tied-final-value-authenticated-replay-gap
    ?s (Some (result, final-state))
    then show ?thesis
    proof
    assume
    composition-fri-verifier-tied-authenticated-slot-recorded-chunk-gap
    ?s (Some (result, final-state))
    then show ?thesis
    unfolding out-eq staged-security-with-data-state-verifier-event-def
    by simp
next
    assume auth:
    composition-fri-verifier-tied-final-value-authenticated-replay-gap
    ?s (Some (result, final-state))
    have selected:

```

```

    composition-fri-verifier-tied-final-value-selected-step-missing-gap
      ?s (Some (result, final-state))
    by (rule
      composition-fri-verifier-tied-final-value-authenticated-replay-gap-imp-selected-step-missing
        [OF auth])
    then show ?thesis
      unfolding out-eq staged-security-with-data-state-verifier-event-def
      by simp
  qed
qed
qed
qed
also have ... ≤
  wp-event ?M ?merkle adversary-initial-state +
  wp-event ?M ?slot adversary-initial-state +
  wp-event ?M ?selected adversary-initial-state
proof -
  have wp-event ?M
    (λout. ?merkle out ∨ ?slot out ∨ ?selected out)
    adversary-initial-state ≤
    wp-event ?M ?merkle adversary-initial-state +
    wp-event ?M (λout. ?slot out ∨ ?selected out)
    adversary-initial-state
  by (rule wp-event-union-bound)
  also have ... ≤
    wp-event ?M ?merkle adversary-initial-state +
    (wp-event ?M ?slot adversary-initial-state +
    wp-event ?M ?selected adversary-initial-state)
  by (intro add-mono order-refl wp-event-union-bound)
  finally show ?thesis
    by (simp add: add.assoc)
qed
also have ... ≤
  composition-one-step-controlled-merkle-error-for budgets + 0 + 0
  by (intro add-mono
    checked-staged-security-partial-merkle-inconsistency-bad-bound-by-composition-one-step-controlled-merkle-error
      [OF wf controlled]
    checked-staged-security-composition-fri-verifier-tied-authenticated-slot-recorded-chunk-gap-zero
    checked-staged-security-composition-fri-verifier-tied-final-value-selected-step-missing-gap-zero)
  finally show ?thesis .
qed

lemma checked-staged-security-composition-fri-verifier-tied-recorded-base-chunk-auth-missing-zero:
  wp-event (checked-staged-security-experiment-with-data-state A)
  (staged-security-with-data-state-verifier-event
    composition-fri-verifier-tied-recorded-base-chunk-auth-missing)
  adversary-initial-state ≤ (0::prob)
proof (rule checked-staged-security-with-data-state-verifier-event-bound-from-cont)
  show ∧s. ¬

```



```

    composition-fri-verifier-tied-recorded-base-chunk-auth-missing s None
  unfolding
    composition-fri-verifier-tied-recorded-base-chunk-auth-missing-def
    accepted-fri-opening-transcript-def
  by simp
next
fix data attacker-state
assume Some (data, attacker-state) ∈
  set-dist
    (execute (checked-staged-transcript-program A)
      adversary-initial-state)
show wp-event verify-monad
  (composition-fri-verifier-tied-recorded-base-chunk-auth-missing
    (verifier-state-from-adversary attacker-state
      (staged-proof-transcript data)))
  (verifier-state-from-adversary attacker-state
    (staged-proof-transcript data))
≤ 0
by (rule
  wp-composition-fri-verifier-tied-recorded-base-chunk-auth-missing-zero)
qed

lemma checked-staged-security-composition-fri-verifier-tied-sampled-assignment-conflict-bound-by-controlled-merkle:
  assumes wf: staged-budget-wellformed budgets
    and controlled: staged-adversary-controlled budgets A
  shows
    wp-event (checked-staged-security-experiment-with-data-state A)
      (staged-security-with-data-state-verifier-event
        composition-fri-verifier-tied-sampled-assignment-conflict)
      adversary-initial-state ≤
      (((composition-one-step-controlled-merkle-error-for budgets + 0) +
        (composition-one-step-controlled-merkle-error-for budgets + 0) +
        (composition-one-step-controlled-merkle-error-for budgets + 0) +
        (composition-one-step-controlled-merkle-error-for budgets + 0 + 0) +
        composition-one-step-controlled-merkle-error-for budgets) + 0)
proof -
  let ?M = checked-staged-security-experiment-with-data-state A
  let ?conflict =
    staged-security-with-data-state-verifier-event
      composition-fri-verifier-tied-sampled-assignment-conflict
  let ?merkle =
    staged-security-with-data-state-verifier-event
      partial-merkle-inconsistency-bad
  let ?slot =
    staged-security-with-data-state-verifier-event
      composition-fri-verifier-tied-authenticated-slot-recorded-chunk-gap
  let ?next =
    staged-security-with-data-state-verifier-event
      composition-fri-verifier-tied-next-value-replay-gap

```

```

let ?successor =
  staged-security-with-data-state-verifier-event
  composition-fri-verifier-tied-successor-replay-gap
let ?final =
  staged-security-with-data-state-verifier-event
  composition-fri-verifier-tied-final-value-replay-gap
have split:
  wp-event ?M ?conflict adversary-initial-state ≤
  wp-event ?M
  (λout.
    ?merkle out ∨ ?slot out ∨ ?next out ∨
    ?successor out ∨ ?final out ∨ ?merkle out ∨ ?slot out)
  adversary-initial-state
proof (rule wp-event-mono-on-support)
fix out
assume support:
  out ∈ set-dist (execute ?M adversary-initial-state)
  and conflict: ?conflict out
show ?merkle out ∨ ?slot out ∨ ?next out ∨
  ?successor out ∨ ?final out ∨ ?merkle out ∨ ?slot out
proof (cases out)
case None
then show ?thesis
  using conflict unfolding staged-security-with-data-state-verifier-event-def
  by simp
next
case (Some out-data)
then obtain data attacker-state result final-state where out-eq:
  out = Some (((data, attacker-state), result), final-state)
  by (cases out-data, auto split: prod.splits)
let ?s =
  verifier-state-from-adversary attacker-state
  (staged-proof-transcript data)
from checked-staged-security-experiment-with-data-state-outcomeE
  [OF support[unfolded out-eq]]
have verify-support:
  Some (result, final-state) ∈ set-dist (execute verify-monad ?s)
  by blast
have conflict-local:
  composition-fri-verifier-tied-sampled-assignment-conflict ?s
  (Some (result, final-state))
  using conflict unfolding out-eq
  staged-security-with-data-state-verifier-event-def
  by simp
have local-split:
  partial-merkle-inconsistency-bad ?s
  (Some (result, final-state)) ∨
  composition-fri-verifier-tied-authenticated-slot-recorded-chunk-gap
  ?s (Some (result, final-state)) ∨

```

```

composition-fri-verifier-tied-next-value-replay-gap
  ?s (Some (result, final-state)) ∨
composition-fri-verifier-tied-successor-replay-gap
  ?s (Some (result, final-state)) ∨
composition-fri-verifier-tied-final-value-replay-gap
  ?s (Some (result, final-state)) ∨
partial-merkle-inconsistency-bad ?s
  (Some (result, final-state)) ∨
composition-fri-verifier-tied-authenticated-slot-recorded-chunk-gap
  ?s (Some (result, final-state))
proof –
  have auth-or-gap:
    composition-fri-verifier-tied-sampled-assignment-conflict-with-authenticated-same-layer
      ?s (Some (result, final-state)) ∨
    composition-fri-verifier-tied-assignment-auth-gap
      ?s (Some (result, final-state))
  by (rule
    composition-fri-verifier-tied-assignment-imp-authenticated-or-auth-gap
    [OF conflict-local])
  then show ?thesis
proof
  assume auth:
    composition-fri-verifier-tied-sampled-assignment-conflict-with-authenticated-same-layer
      ?s (Some (result, final-state))
  have without-or-same:
    composition-fri-verifier-tied-sampled-assignment-conflict-without-same-layer
      ?s (Some (result, final-state)) ∨
    composition-fri-verifier-tied-same-layer-conflict-with-authenticated-chunks
      ?s (Some (result, final-state))
  by (rule
    composition-fri-verifier-tied-authenticated-conflict-imp-without-same-or-same-layer
    [OF auth])
  then show ?thesis
proof
  assume without:
    composition-fri-verifier-tied-sampled-assignment-conflict-without-same-layer
      ?s (Some (result, final-state))
  then have branches:
    composition-fri-verifier-tied-sampled-base-opening-conflict
      ?s (Some (result, final-state)) ∨
    composition-fri-verifier-tied-sampled-next-value-conflict
      ?s (Some (result, final-state)) ∨
    composition-fri-verifier-tied-sampled-successor-opening-conflict
      ?s (Some (result, final-state)) ∨
    composition-fri-verifier-tied-sampled-final-value-conflict
      ?s (Some (result, final-state))
  using composition-fri-verifier-tied-without-same-iff-branches
  by blast
  then show ?thesis

```

```

proof
  assume base:
    composition-fri-verifier-tied-sampled-base-opening-conflict
    ?s (Some (result, final-state))
  have base-split:
    composition-fri-verifier-tied-authenticated-base-opening-conflict
    ?s (Some (result, final-state)) ∨
    composition-fri-verifier-tied-base-chunk-auth-gap
    ?s (Some (result, final-state))
  by (rule
    composition-fri-verifier-tied-sampled-base-imp-authenticated-or-auth-gap
    [OF base])
  then show ?thesis
proof
  assume auth-base:
    composition-fri-verifier-tied-authenticated-base-opening-conflict
    ?s (Some (result, final-state))
  then have
    partial-merkle-inconsistency-bad ?s
    (Some (result, final-state))
  by (rule
    composition-fri-verifier-tied-authenticated-base-conflict-imp-partial-merkle)
  then show ?thesis by simp
next
  assume gap:
    composition-fri-verifier-tied-base-chunk-auth-gap
    ?s (Some (result, final-state))
  then have missing:
    composition-fri-verifier-tied-recorded-base-chunk-auth-missing
    ?s (Some (result, final-state))
  by (rule
    composition-fri-verifier-tied-base-chunk-auth-gap-imp-recorded-missing)
  then have
    composition-fri-verifier-tied-authenticated-slot-recorded-chunk-gap
    ?s (Some (result, final-state))
  by (rule
    composition-fri-verifier-tied-recorded-base-chunk-auth-missing-imp-slot-recorded-chunk-gap)
  then show ?thesis by simp
qed
next
  assume rest:
    composition-fri-verifier-tied-sampled-next-value-conflict
    ?s (Some (result, final-state)) ∨
    composition-fri-verifier-tied-sampled-successor-opening-conflict
    ?s (Some (result, final-state)) ∨
    composition-fri-verifier-tied-sampled-final-value-conflict
    ?s (Some (result, final-state))
  then show ?thesis
proof

```

```

assume next-conflict:
  composition-fri-verifier-tied-sampled-next-value-conflict
    ?s (Some (result, final-state))
then have
  composition-fri-verifier-tied-next-value-replay-gap
    ?s (Some (result, final-state))
  by (rule
    composition-fri-verifier-tied-sampled-next-value-imp-replay-gap)
then show ?thesis by simp
next
assume tail:
  composition-fri-verifier-tied-sampled-successor-opening-conflict
    ?s (Some (result, final-state)) ∨
  composition-fri-verifier-tied-sampled-final-value-conflict
    ?s (Some (result, final-state))
then show ?thesis
proof
  assume successor:
    composition-fri-verifier-tied-sampled-successor-opening-conflict
      ?s (Some (result, final-state))
  then have
    composition-fri-verifier-tied-successor-replay-gap
      ?s (Some (result, final-state))
    by (rule
      composition-fri-verifier-tied-sampled-successor-imp-replay-gap)
  then show ?thesis by simp
next
assume final:
  composition-fri-verifier-tied-sampled-final-value-conflict
    ?s (Some (result, final-state))
  then have
    composition-fri-verifier-tied-final-value-replay-gap
      ?s (Some (result, final-state))
    by (rule
      composition-fri-verifier-tied-sampled-final-value-imp-replay-gap)
  then show ?thesis by simp
qed
qed
qed
next
assume same:
  composition-fri-verifier-tied-same-layer-conflict-with-authenticated-chunks
    ?s (Some (result, final-state))
then have
  partial-merkle-inconsistency-bad ?s
    (Some (result, final-state))
  by (rule
    composition-fri-verifier-tied-same-layer-conflict-with-authenticated-chunks-imp-partial-merkle)
then show ?thesis by simp

```

```

qed
next
  assume gap:
    composition-fri-verifier-tied-assignment-auth-gap
    ?s (Some (result, final-state))
  have same-gap:
    composition-fri-verifier-tied-same-layer-auth-gap
    ?s (Some (result, final-state))
  by (rule
    composition-fri-verifier-tied-assignment-auth-gap-imp-same-layer-auth-gap
    [OF gap])
  have recorded-gap:
    composition-fri-verifier-tied-same-layer-recorded-auth-gap
    ?s (Some (result, final-state))
  by (rule
    composition-fri-verifier-tied-same-layer-auth-gap-imp-recorded-auth-gap
    [OF same-gap])
  have authenticated-gap:
    composition-fri-verifier-tied-same-layer-authenticated-recorded-auth-gap
    ?s (Some (result, final-state))
  by (rule
    composition-fri-verifier-tied-same-layer-recorded-auth-gap-imp-authenticated-recorded-auth-gap-on-support
    [OF verify-support recorded-gap])
  have
    composition-fri-verifier-tied-authenticated-slot-recorded-chunk-gap
    ?s (Some (result, final-state))
  by (rule
    composition-fri-verifier-tied-same-layer-authenticated-recorded-auth-gap-imp-slot-recorded-chunk-gap
    [OF authenticated-gap])
  then show ?thesis by simp
qed
qed
then show ?thesis
  unfolding out-eq staged-security-with-data-state-verifier-event-def
  by simp
qed
qed
have next-bound:
  wp-event ?M ?next adversary-initial-state ≤
    composition-one-step-controlled-merkle-error-for budgets
  using
    checked-staged-security-composition-fri-verifier-tied-next-value-replay-gap-bound-by-controlled-merkle
    [OF wf controlled]
  by simp
have successor-bound:
  wp-event ?M ?successor adversary-initial-state ≤
    composition-one-step-controlled-merkle-error-for budgets
  using
    checked-staged-security-composition-fri-verifier-tied-successor-replay-gap-bound-by-controlled-merkle

```

```

[OF wf controlled]
by simp
have final-bound:
  wp-event ?M ?final adversary-initial-state ≤
    composition-one-step-controlled-merkle-error-for budgets
using
  checked-staged-security-composition-fri-verifier-tied-final-value-replay-gap-bound-by-controlled-merkle
  [OF wf controlled]
by simp
have split-bound:
  wp-event ?M
  (λout.
    ?merkle out ∨ ?slot out ∨ ?next out ∨
    ?successor out ∨ ?final out ∨ ?merkle out ∨ ?slot out)
  adversary-initial-state ≤
    wp-event ?M ?merkle adversary-initial-state +
    (wp-event ?M ?slot adversary-initial-state +
    (wp-event ?M ?next adversary-initial-state +
    (wp-event ?M ?successor adversary-initial-state +
    (wp-event ?M ?final adversary-initial-state +
    wp-event ?M ?merkle adversary-initial-state))))))
proof -
  let ?nodup =
    λout.
      ?merkle out ∨ ?slot out ∨ ?next out ∨
      ?successor out ∨ ?final out ∨ ?merkle out
  have dedup:
    wp-event ?M
    (λout.
      ?merkle out ∨ ?slot out ∨ ?next out ∨
      ?successor out ∨ ?final out ∨ ?merkle out ∨ ?slot out)
    adversary-initial-state ≤
      wp-event ?M ?nodup adversary-initial-state
  by (rule wp-event-mono) auto
  have union-bound:
    wp-event ?M ?nodup adversary-initial-state ≤
      wp-event ?M ?merkle adversary-initial-state +
      (wp-event ?M ?slot adversary-initial-state +
      (wp-event ?M ?next adversary-initial-state +
      (wp-event ?M ?successor adversary-initial-state +
      (wp-event ?M ?final adversary-initial-state +
      wp-event ?M ?merkle adversary-initial-state))))))
  proof -
    let ?tail-final = λout. ?final out ∨ ?merkle out
    let ?tail-successor = λout. ?successor out ∨ ?tail-final out
    let ?tail-next = λout. ?next out ∨ ?tail-successor out
    let ?tail-slot = λout. ?slot out ∨ ?tail-next out
    have final-bound':
      wp-event ?M ?tail-final adversary-initial-state ≤

```

$wp\text{-event } ?M \text{ ?final adversary-initial-state} +$
 $wp\text{-event } ?M \text{ ?merkle adversary-initial-state}$
by (rule *wp-event-union-bound*)
have *successor-bound'*:
 $wp\text{-event } ?M \text{ ?tail-successor adversary-initial-state} \leq$
 $wp\text{-event } ?M \text{ ?successor adversary-initial-state} +$
 $(wp\text{-event } ?M \text{ ?final adversary-initial-state} +$
 $wp\text{-event } ?M \text{ ?merkle adversary-initial-state})$
proof –
have $wp\text{-event } ?M \text{ ?tail-successor adversary-initial-state} \leq$
 $wp\text{-event } ?M \text{ ?successor adversary-initial-state} +$
 $wp\text{-event } ?M \text{ ?tail-final adversary-initial-state}$
by (rule *wp-event-union-bound*)
also have $\dots \leq$
 $wp\text{-event } ?M \text{ ?successor adversary-initial-state} +$
 $(wp\text{-event } ?M \text{ ?final adversary-initial-state} +$
 $wp\text{-event } ?M \text{ ?merkle adversary-initial-state})$
by (intro *add-mono order-refl final-bound'*)
finally show *?thesis* .
qed
have *next-bound'*:
 $wp\text{-event } ?M \text{ ?tail-next adversary-initial-state} \leq$
 $wp\text{-event } ?M \text{ ?next adversary-initial-state} +$
 $(wp\text{-event } ?M \text{ ?successor adversary-initial-state} +$
 $(wp\text{-event } ?M \text{ ?final adversary-initial-state} +$
 $wp\text{-event } ?M \text{ ?merkle adversary-initial-state}))$
proof –
have $wp\text{-event } ?M \text{ ?tail-next adversary-initial-state} \leq$
 $wp\text{-event } ?M \text{ ?next adversary-initial-state} +$
 $wp\text{-event } ?M \text{ ?tail-successor adversary-initial-state}$
by (rule *wp-event-union-bound*)
also have $\dots \leq$
 $wp\text{-event } ?M \text{ ?next adversary-initial-state} +$
 $(wp\text{-event } ?M \text{ ?successor adversary-initial-state} +$
 $(wp\text{-event } ?M \text{ ?final adversary-initial-state} +$
 $wp\text{-event } ?M \text{ ?merkle adversary-initial-state}))$
by (intro *add-mono order-refl successor-bound'*)
finally show *?thesis* .
qed
have *slot-bound'*:
 $wp\text{-event } ?M \text{ ?tail-slot adversary-initial-state} \leq$
 $wp\text{-event } ?M \text{ ?slot adversary-initial-state} +$
 $(wp\text{-event } ?M \text{ ?next adversary-initial-state} +$
 $(wp\text{-event } ?M \text{ ?successor adversary-initial-state} +$
 $(wp\text{-event } ?M \text{ ?final adversary-initial-state} +$
 $wp\text{-event } ?M \text{ ?merkle adversary-initial-state})))$
proof –
have $wp\text{-event } ?M \text{ ?tail-slot adversary-initial-state} \leq$
 $wp\text{-event } ?M \text{ ?slot adversary-initial-state} +$


```

    wp-event ?M ?tail-next adversary-initial-state
  by (rule wp-event-union-bound)
also have ... ≤
  wp-event ?M ?slot adversary-initial-state +
  (wp-event ?M ?next adversary-initial-state +
  (wp-event ?M ?successor adversary-initial-state +
  (wp-event ?M ?final adversary-initial-state +
  wp-event ?M ?merkle adversary-initial-state)))
  by (intro add-mono order-refl next-bound')
  finally show ?thesis .
qed
have wp-event ?M ?nodup adversary-initial-state ≤
  wp-event ?M ?merkle adversary-initial-state +
  wp-event ?M ?tail-slot adversary-initial-state
  by (rule wp-event-union-bound)
also have ... ≤
  wp-event ?M ?merkle adversary-initial-state +
  (wp-event ?M ?slot adversary-initial-state +
  (wp-event ?M ?next adversary-initial-state +
  (wp-event ?M ?successor adversary-initial-state +
  (wp-event ?M ?final adversary-initial-state +
  wp-event ?M ?merkle adversary-initial-state))))
  by (intro add-mono order-refl slot-bound')
  finally show ?thesis .
qed
show ?thesis
  by (rule order-trans[OF dedup union-bound])
qed
have split-sum-bound:
  wp-event ?M ?merkle adversary-initial-state +
  (wp-event ?M ?slot adversary-initial-state +
  (wp-event ?M ?next adversary-initial-state +
  (wp-event ?M ?successor adversary-initial-state +
  (wp-event ?M ?final adversary-initial-state +
  wp-event ?M ?merkle adversary-initial-state)))) ≤
  composition-one-step-controlled-merkle-error-for budgets +
  (0 +
  (composition-one-step-controlled-merkle-error-for budgets +
  (composition-one-step-controlled-merkle-error-for budgets +
  (composition-one-step-controlled-merkle-error-for budgets +
  composition-one-step-controlled-merkle-error-for budgets))))
  by (intro add-mono
  checked-staged-security-partial-merkle-inconsistency-bad-bound-by-composition-one-step-controlled-merkle-error
  [OF wf controlled]
  checked-staged-security-composition-fri-verifier-tied-authenticated-slot-recorded-chunk-gap-zero
  next-bound successor-bound final-bound)
have wp-event ?M ?conflict adversary-initial-state ≤
  wp-event ?M
  (λout.

```

```

      ?merkle out ∨ ?slot out ∨ ?next out ∨
      ?successor out ∨ ?final out ∨ ?merkle out ∨ ?slot out)
    adversary-initial-state
  by (rule split)
also have ... ≤
  wp-event ?M ?merkle adversary-initial-state +
  (wp-event ?M ?slot adversary-initial-state +
  (wp-event ?M ?next adversary-initial-state +
  (wp-event ?M ?successor adversary-initial-state +
  (wp-event ?M ?final adversary-initial-state +
  wp-event ?M ?merkle adversary-initial-state))))
  by (rule split-bound)
also have ... ≤
  composition-one-step-controlled-merkle-error-for budgets +
  (0 +
  (composition-one-step-controlled-merkle-error-for budgets +
  (composition-one-step-controlled-merkle-error-for budgets +
  (composition-one-step-controlled-merkle-error-for budgets +
  composition-one-step-controlled-merkle-error-for budgets))))
  by (rule split-sum-bound)
finally show ?thesis
  by (simp add: add.assoc)
qed

```

lemma *composition-one-step-verifier-tied-controlled-error-for-bound:*

```

fixes comp-bad :: 'f ⇒ nat ⇒ 'f list ⇒ 'f set
assumes wf: staged-budget-wellformed budgets
and controlled: staged-adversary-controlled budgets A
and comp-subset:
  ∧ dg i prefix.
    i < fri-round-count-for-degree-bound (to-nat dg) ⇒
    length prefix = i ⇒
    comp-bad dg i prefix ⊆
    fri-one-step-disagreement-challenges
      (composition-fri-canonical-layer-len dg i) (comp-pw dg i)
      (comp-layer dg i prefix) (comp-claimed dg i prefix)
      (comp-dom dg i prefix)

```

shows

```

  wp-event (checked-staged-security-experiment-with-data-state A)
  (staged-security-with-data-state-verifier-event
  composition-fri-bad-with-verifier-tied-partial-candidate)
  adversary-initial-state
≤ composition-one-step-verifier-tied-controlled-error-for budgets
  comp-bad

```

proof —

have *sampled-query:*

```

  wp-event (checked-staged-security-experiment-with-data-state A)
  (staged-security-with-data-state-verifier-event
  composition-fri-sampled-query-bad-candidate)

```

```

    adversary-initial-state
  ≤ composition-one-step-sampled-error-for budgets comp-bad
  by (rule composition-one-step-sampled-error-for-bound
    [OF wf controlled comp-subset])
  have sampled:
    wp-event (checked-staged-security-experiment-with-data-state A)
      (staged-security-with-data-state-verifier-event
        composition-fri-bad-with-verifier-tied-sampled-layer-chain)
      adversary-initial-state
    ≤ composition-one-step-sampled-error-for budgets comp-bad
  by (rule
    checked-staged-security-composition-fri-verifier-tied-sampled-layer-chain-bound-from-sampled-query
    [OF sampled-query])
  have conflict:
    wp-event (checked-staged-security-experiment-with-data-state A)
      (staged-security-with-data-state-verifier-event
        composition-fri-verifier-tied-sampled-assignment-conflict)
      adversary-initial-state ≤
      (((composition-one-step-controlled-merkle-error-for budgets + 0) +
        (composition-one-step-controlled-merkle-error-for budgets + 0) +
        (composition-one-step-controlled-merkle-error-for budgets + 0) +
        (composition-one-step-controlled-merkle-error-for budgets + 0 + 0) +
        composition-one-step-controlled-merkle-error-for budgets) + 0)
  by (rule
    checked-staged-security-composition-fri-verifier-tied-sampled-assignment-conflict-bound-by-controlled-merkle
    [OF wf controlled])
  have zero:
    wp-event (checked-staged-security-experiment-with-data-state A)
      (staged-security-with-data-state-verifier-event
        composition-fri-verifier-tied-zero-round-final-obstruction)
      adversary-initial-state ≤ (0::prob)
  proof (rule checked-staged-security-with-data-state-verifier-event-bound-from-cont)
    show  $\bigwedge s. \neg$ 
      composition-fri-verifier-tied-zero-round-final-obstruction s None
    unfolding composition-fri-verifier-tied-zero-round-final-obstruction-def
      accepted-fri-opening-transcript-def
    by simp
  next
  fix data attacker-state
  assume support: Some (data, attacker-state) ∈
    set-dist
    (execute (checked-staged-transcript-program A)
      adversary-initial-state)
  show wp-event verify-monad
    (composition-fri-verifier-tied-zero-round-final-obstruction
      (verifier-state-from-adversary attacker-state
        (staged-proof-transcript data)))
    (verifier-state-from-adversary attacker-state
      (staged-proof-transcript data))

```

```

    ≤ 0
  by (subst
      wp-composition-fri-verifier-tied-zero-round-final-obstruction-zero)
    simp
qed
have branch:
  wp-event (checked-staged-security-experiment-with-data-state A)
    (staged-security-with-data-state-verifier-event
      composition-fri-bad-with-verifier-tied-partial-candidate)
  adversary-initial-state ≤
  composition-one-step-sampled-error-for budgets comp-bad +
  (((composition-one-step-controlled-merkle-error-for budgets + 0) +
    (composition-one-step-controlled-merkle-error-for budgets + 0) +
    (composition-one-step-controlled-merkle-error-for budgets + 0) +
    (composition-one-step-controlled-merkle-error-for budgets + 0 + 0) +
    composition-one-step-controlled-merkle-error-for budgets) + 0) + 0)
  by (rule
      checked-staged-security-composition-fri-verifier-tied-partial-candidate-bound-from-staged-sampled-conflict-and-zero
      [OF sampled conflict zero])
  show ?thesis
  using branch
  by (simp add: composition-one-step-verifier-tied-controlled-error-for-def)
qed

lemma composition-one-step-verifier-tied-error-for-bound:
  fixes comp-bad :: 'f ⇒ nat ⇒ 'f list ⇒ 'f set
  assumes wf: staged-budget-wellformed budgets
  and controlled: staged-adversary-controlled budgets A
  and comp-subset:
    ∧ dg i prefix.
    i < fri-round-count-for-degree-bound (to-nat dg) ⇒
    length prefix = i ⇒
    comp-bad dg i prefix ⊆
    fri-one-step-disagreement-challenges
      (composition-fri-canonical-layer-len dg i) (comp-pw dg i)
      (comp-layer dg i prefix) (comp-claimed dg i prefix)
      (comp-dom dg i prefix)
  shows
    wp-event (checked-staged-security-experiment-with-data-state A)
      (staged-security-with-data-state-verifier-event
        composition-fri-bad-with-verifier-tied-partial-candidate)
    adversary-initial-state
    ≤ composition-one-step-verifier-tied-error-for budgets A comp-bad
proof -
  have sampled:
    wp-event (checked-staged-security-experiment-with-data-state A)
      (staged-security-with-data-state-verifier-event
        composition-fri-sampled-query-bad-candidate)
    adversary-initial-state

```

$\leq$ *composition-one-step-sampled-error-for budgets comp-bad*
by (rule *composition-one-step-sampled-error-for-bound*
[*OF wf controlled comp-subset*])
have *merkle-bound*:
 $\wedge$ *data attacker-state*.
Some (data, attacker-state) $\in$
set-dist (execute (checked-staged-transcript-program A)
adversary-initial-state) $\implies$
wp-event verify-monad
(partial-merkle-inconsistency-bad
(verifier-state-from-adversary attacker-state
(staged-proof-transcript data)))
(verifier-state-from-adversary attacker-state
(staged-proof-transcript data))
 $\leq$ *composition-one-step-reachable-merkle-error-for A*
unfolding *composition-one-step-reachable-merkle-error-for-def*
by (rule *reachable-verifier-event-bound-for-ge*, *assumption*)
have *branch*:
wp-event (checked-staged-security-experiment-with-data-state A)
(staged-security-with-data-state-verifier-event
composition-fri-bad-with-verifier-tied-partial-candidate)
adversary-initial-state
 $\leq$ *composition-one-step-sampled-error-for budgets comp-bad* +
(((*composition-one-step-reachable-merkle-error-for A* + 0) +
(*composition-one-step-reachable-merkle-error-for A* + 0) +
(*composition-one-step-reachable-merkle-error-for A* + 0) +
(*composition-one-step-reachable-merkle-error-for A* + 0 + 0) +
composition-one-step-reachable-merkle-error-for A) + 0 + 0)
apply (rule
checked-staged-security-composition-fri-verifier-tied-partial-candidate-bound-from-sampled-query-and-residual-gap
[*OF sampled*])
apply (rule *wp-composition-fri-verifier-tied-authenticated-slot-recorded-chunk-gap-zero*)
apply (rule *merkle-bound*)
apply *assumption*
apply (rule *wp-composition-fri-verifier-tied-recorded-base-chunk-auth-missing-zero*)
apply (rule *wp-composition-fri-verifier-tied-next-value-replay-gap-bound-from-merkle-and-slot*)
apply (rule *merkle-bound*)
apply *assumption*
apply (rule *wp-composition-fri-verifier-tied-authenticated-slot-recorded-chunk-gap-zero*)
apply (rule *wp-composition-fri-verifier-tied-successor-replay-gap-bound-from-merkle-and-slot*)
apply (rule *merkle-bound*)
apply *assumption*
apply (rule *wp-composition-fri-verifier-tied-authenticated-slot-recorded-chunk-gap-zero*)
apply (rule *wp-composition-fri-verifier-tied-final-value-replay-gap-bound-from-merkle-slot-and-selected-step*)
apply (rule *merkle-bound*)
apply *assumption*
apply (rule *wp-composition-fri-verifier-tied-authenticated-slot-recorded-chunk-gap-zero*)
apply (rule *wp-composition-fri-verifier-tied-final-value-selected-step-missing-gap-zero*)
apply (subst *wp-composition-fri-verifier-tied-zero-round-final-obstruction-zero*)

```

    apply simp
  done
show ?thesis
  using branch
  by (simp add: composition-one-step-verifier-tied-error-for-def)
qed

lemma trace-empty-one-step-error-for-bound:
  fixes empty-bad :: nat  $\Rightarrow$  'f list  $\Rightarrow$  'f set
  assumes wf: staged-budget-wellformed budgets
  and controlled: staged-adversary-controlled budgets A
  and empty-subset:
     $\bigwedge i$  prefix.
       $i < \text{fri-round-count-for-degree-bound } (\text{clength} - 1) \Rightarrow$ 
       $\text{length prefix} = i \Rightarrow$ 
       $\text{empty-bad } i \text{ prefix} \subseteq$ 
       $\text{fri-one-step-disagreement-challenges}$ 
       $(\text{trace-fri-canonical-layer-len } i) (\text{empty-pw } i)$ 
       $(\text{empty-layer } i \text{ prefix}) (\text{empty-claimed } i \text{ prefix})$ 
       $(\text{empty-dom } i \text{ prefix})$ 
  shows
    wp-event (checked-staged-security-experiment-with-data-state A)
      (staged-security-with-data-state-verifier-event
        trace-fri-bad-with-empty-composition-header-candidates)
      adversary-initial-state
     $\leq$  trace-empty-one-step-error-for budgets A empty-bad
  proof -
    have sampled:
      wp-event (checked-staged-security-experiment-with-data-state A)
        (staged-security-with-data-state-verifier-event
          trace-fri-sampled-query-bad-candidate)
        adversary-initial-state
       $\leq$  trace-one-step-sampled-error-for budgets empty-bad
    by (rule trace-one-step-sampled-error-for-bound
      [OF wf controlled empty-subset])
    have conflict-bound:
      wp-event (checked-staged-security-experiment-with-data-state A)
        (staged-security-with-data-state-verifier-event
          trace-fri-sampled-assignment-conflict)
        adversary-initial-state
       $\leq$  reachable-verifier-event-bound-for A
        trace-fri-sampled-assignment-conflict
  proof (rule checked-staged-security-with-data-state-verifier-event-bound-from-cont)
    show  $\bigwedge s. \neg \text{trace-fri-sampled-assignment-conflict } s \text{ None}$ 
    unfolding trace-fri-sampled-assignment-conflict-def
      trace-fri-partial-candidate-opening-evidence-def
      accepted-fri-opening-transcript-def
    by simp
  next

```

```

fix data attacker-state
assume support: Some (data, attacker-state) ∈
  set-dist
    (execute (checked-staged-transcript-program A)
      adversary-initial-state)
show wp-event verify-monad
  (trace-fri-sampled-assignment-conflict
    (verifier-state-from-adversary attacker-state
      (staged-proof-transcript data)))
  (verifier-state-from-adversary attacker-state
    (staged-proof-transcript data))
  ≤ reachable-verifier-event-bound-for A
    trace-fri-sampled-assignment-conflict
  by (rule reachable-verifier-event-bound-for-ge[OF support])
qed
have zero-bound:
  wp-event (checked-staged-security-experiment-with-data-state A)
    (staged-security-with-data-state-verifier-event
      trace-fri-zero-round-final-obstruction)
    adversary-initial-state
  ≤ trace-fri-zero-round-agreement-set-target-error-for budgets +
    trace-fri-controlled-merkle-error-for budgets +
    reachable-verifier-event-bound-for A
    trace-fri-bad-with-header-tied-partial-candidate +
    (trace-fri-controlled-merkle-error-for budgets +
      (reachable-verifier-event-bound-for A
        trace-fri-header-tied-low-candidate-untied-root-gap +
        reachable-verifier-event-bound-for A
          trace-fri-header-tied-low-candidate-untied-query-gap)) +
    reachable-verifier-event-bound-for A
    trace-fri-bad-with-header-tied-partial-candidate
  by (rule checked-staged-security-trace-fri-zero-round-final-obstruction-bound-from-agreement-sets
    [OF wf controlled])
have branch:
  wp-event (checked-staged-security-experiment-with-data-state A)
    (staged-security-with-data-state-verifier-event
      trace-fri-bad-with-empty-composition-header-candidates)
    adversary-initial-state
  ≤ trace-one-step-sampled-error-for budgets empty-bad +
    (reachable-verifier-event-bound-for A
      trace-fri-sampled-assignment-conflict +
      trace-fri-zero-round-agreement-set-target-error-for budgets +
      trace-fri-controlled-merkle-error-for budgets +
      reachable-verifier-event-bound-for A
        trace-fri-bad-with-header-tied-partial-candidate +
      (trace-fri-controlled-merkle-error-for budgets +
        (reachable-verifier-event-bound-for A
          trace-fri-header-tied-low-candidate-untied-root-gap +
          reachable-verifier-event-bound-for A
            trace-fri-header-tied-low-candidate-untied-query-gap)))

```

```

    trace-fri-header-tied-low-candidate-untied-query-gap))) +
    reachable-verifier-event-bound-for A
    trace-fri-bad-with-header-tied-partial-candidate)
proof —
  have raw:
    wp-event (checked-staged-security-experiment-with-data-state A)
      (staged-security-with-data-state-verifier-event
        trace-fri-bad-with-empty-composition-header-candidates)
    adversary-initial-state
  ≤ trace-one-step-sampled-error-for budgets empty-bad +
    (reachable-verifier-event-bound-for A
      trace-fri-sampled-assignment-conflict +
      (trace-fri-zero-round-agreement-set-target-error-for budgets +
        trace-fri-controlled-merkle-error-for budgets +
        reachable-verifier-event-bound-for A
          trace-fri-bad-with-header-tied-partial-candidate +
          (trace-fri-controlled-merkle-error-for budgets +
            (reachable-verifier-event-bound-for A
              trace-fri-header-tied-low-candidate-untied-root-gap +
              reachable-verifier-event-bound-for A
                trace-fri-header-tied-low-candidate-untied-query-gap))) +
            reachable-verifier-event-bound-for A
              trace-fri-bad-with-header-tied-partial-candidate)))
    by (rule
      checked-staged-security-trace-fri-empty-header-bound-from-staged-sampled-conflict-and-zero
        [OF sampled conflict-bound zero-bound])
  show ?thesis
    by (rule order-trans[OF raw]) (simp add: algebra-simps)
qed
show ?thesis
  using branch
  by (simp add: trace-empty-one-step-error-for-def
    trace-fri-empty-small-conflict-zero-error-for-def)
qed

lemma trace-header-one-step-error-for-bound:
  fixes trace-bad :: nat ⇒ 'f list ⇒ 'f set
  assumes wf: staged-budget-wellformed budgets
  and controlled: staged-adversary-controlled budgets A
  and trace-subset:
    ∧ i prefix.
    i < fri-round-count-for-degree-bound (clength - 1) ⇒
    length prefix = i ⇒
    trace-bad i prefix ⊆
    fri-one-step-disagreement-challenges
      (trace-fri-canonical-layer-len i) (empty-pw i)
      (empty-layer i prefix) (empty-claimed i prefix)
      (empty-dom i prefix)
  shows

```


$wp\text{-event}$ (*checked-staged-security-experiment-with-data-state* A)
 (*staged-security-with-data-state-verifier-event*
trace-fri-bad-with-header-tied-partial-candidate)
adversary-initial-state
 $\leq$ *trace-header-one-step-error-for* budgets A *trace-bad*

proof –

have *sampling-query*:

$wp\text{-event}$ (*checked-staged-security-experiment-with-data-state* A)
 (*staged-security-with-data-state-verifier-event*
trace-fri-sampled-query-bad-candidate)
adversary-initial-state
 $\leq$ *trace-one-step-sampled-error-for* budgets *trace-bad*

by (*rule trace-one-step-sampled-error-for-bound*
 $[OF\ wf\ controlled\ trace\ subset]$)

have *sampling-plain*:

$wp\text{-event}$ (*checked-staged-security-experiment-with-data-state* A)
 (*staged-security-with-data-state-verifier-event*
trace-fri-bad-with-sampled-layer-chain)
adversary-initial-state
 $\leq$ *trace-one-step-sampled-error-for* budgets *trace-bad*

by (*rule*
checked-staged-security-trace-fri-sampled-layer-chain-bound-from-sampled-query
 $[OF\ sampling\ query]$)

have *sampling-header*:

$wp\text{-event}$ (*checked-staged-security-experiment-with-data-state* A)
 (*staged-security-with-data-state-verifier-event*
trace-fri-bad-with-header-tied-sampled-layer-chain)
adversary-initial-state
 $\leq$ *trace-one-step-sampled-error-for* budgets *trace-bad*

proof (*rule order-trans* $[OF - sampling\ plain]$)

show $wp\text{-event}$ (*checked-staged-security-experiment-with-data-state* A)
 (*staged-security-with-data-state-verifier-event*
trace-fri-bad-with-header-tied-sampled-layer-chain)
adversary-initial-state
 $\leq$ $wp\text{-event}$ (*checked-staged-security-experiment-with-data-state* A)
 (*staged-security-with-data-state-verifier-event*
trace-fri-bad-with-sampled-layer-chain)
adversary-initial-state

by (*rule wp-event-mono*)
 (*auto simp: staged-security-with-data-state-verifier-event-def*
dest: trace-fri-header-tied-sampled-layer-chain-imp-sampled-layer-chain
split: option.splits prod.splits)

qed

have *conflict-bound*:

$wp\text{-event}$ (*checked-staged-security-experiment-with-data-state* A)
 (*staged-security-with-data-state-verifier-event*
trace-fri-header-tied-sampled-assignment-conflict)
adversary-initial-state
 $\leq$ *trace-fri-header-replay-conflict-error-for* A

```

proof (rule checked-staged-security-with-data-state-verifier-event-bound-from-cont)
  fix data attacker-state
  assume support:
    Some (data, attacker-state) ∈
      set-dist (execute (checked-staged-transcript-program A)
        adversary-initial-state)
  let ?s =
    verifier-state-from-adversary attacker-state
    (staged-proof-transcript data)
  let ?M =
    reachable-verifier-event-bound-for A partial-merkle-inconsistency-bad
  let ?H =
    reachable-verifier-event-bound-for A
    trace-fri-header-tied-recorded-sibling-candidate-missing
  have merkle-bound:
    wp-event verify-monad (partial-merkle-inconsistency-bad ?s) ?s
      ≤ ?M
    by (rule reachable-verifier-event-bound-for-ge[OF support])
  have recorded-missing-bound:
    wp-event verify-monad
      (trace-fri-header-tied-recorded-sibling-candidate-missing ?s) ?s
      ≤ ?H
    by (rule reachable-verifier-event-bound-for-ge[OF support])
  have head-neq:
    wp-event verify-monad
      (trace-fri-header-tied-base-head-value-neq-gap ?s) ?s ≤ ?M
    by (rule
      wp-trace-fri-header-tied-base-head-value-neq-gap-bound-from-partial-merkle)
      (rule merkle-bound)
  have head-mismatch:
    wp-event verify-monad
      (trace-fri-header-tied-base-head-value-mismatch-gap ?s) ?s
      ≤ ?M
    by (rule
      wp-trace-fri-header-tied-base-head-value-mismatch-gap-bound-from-neq-gap
      [OF head-neq])
  have alignment:
    wp-event verify-monad
      (trace-fri-header-tied-base-value-alignment-gap ?s) ?s ≤ ?M
    by (rule
      wp-trace-fri-header-tied-base-value-alignment-gap-bound-from-head-value-mismatch-gap
      [OF head-mismatch])
  have next-bound:
    wp-event verify-monad
      (trace-fri-header-tied-next-value-replay-gap ?s) ?s ≤ ?M + 0
    by (rule
      wp-trace-fri-header-tied-next-value-replay-gap-bound-from-merkle-and-slot)
      (rule merkle-bound,
      rule wp-trace-fri-header-tied-authenticated-slot-recorded-chunk-gap-zero)

```

have *successor*:
 wp-event verify-monad
 (trace-fri-header-tied-successor-replay-gap ?s) ?s ≤ ?M + 0
by (rule
 wp-trace-fri-header-tied-successor-replay-gap-bound-from-merkle-and-slot)
 (rule merkle-bound,
 rule wp-trace-fri-header-tied-authenticated-slot-recorded-chunk-gap-zero)
have *final*:
 wp-event verify-monad
 (trace-fri-header-tied-final-value-replay-gap ?s) ?s ≤ ?M + 0 + 0
by (rule
 wp-trace-fri-header-tied-final-value-replay-gap-bound-from-merkle-slot-and-selected-step)
 (rule merkle-bound,
 rule wp-trace-fri-header-tied-authenticated-slot-recorded-chunk-gap-zero,
 rule wp-trace-fri-header-tied-final-value-selected-step-missing-gap-zero)
have *slot*:
 wp-event verify-monad
 (trace-fri-header-tied-authenticated-slot-recorded-chunk-gap ?s) ?s
 ≤ 0
by (rule wp-trace-fri-header-tied-authenticated-slot-recorded-chunk-gap-zero)
have *authenticated-recorded-auth*:
 wp-event verify-monad
 (trace-fri-header-tied-same-layer-authenticated-recorded-auth-gap ?s)
 ?s ≤ 0
by (rule
 wp-trace-fri-header-tied-same-layer-authenticated-recorded-auth-gap-bound-from-slot-recorded-chunk-gap
 [OF slot])
have *recorded-auth*:
 wp-event verify-monad
 (trace-fri-header-tied-same-layer-recorded-auth-gap ?s) ?s ≤ 0
by (rule
 wp-trace-fri-header-tied-same-layer-recorded-auth-gap-bound-from-authenticated-recorded-auth-gap
 [OF authenticated-recorded-auth])
have *same-auth*:
 wp-event verify-monad
 (trace-fri-header-tied-same-layer-auth-gap ?s) ?s ≤ 0
by (rule
 wp-trace-fri-header-tied-same-layer-auth-gap-bound-from-recorded-auth-gap
 [OF recorded-auth])
have *auth-gap*:
 wp-event verify-monad
 (trace-fri-header-tied-assignment-auth-gap ?s) ?s ≤ 0
by (rule
 wp-trace-fri-header-tied-assignment-auth-gap-bound-from-same-layer-auth-gap
 [OF same-auth])
have *complement*:
 wp-event verify-monad
 (λout.
 trace-fri-header-tied-sampled-assignment-conflict ?s out ∧

```

    ¬ trace-fri-header-tied-recorded-sibling-candidate-missing ?s out)
  ?s ≤
    (((?M + ?M) + (?M + 0) + (?M + 0) + (?M + 0 + 0)) + ?M) + 0)
by (rule
  wp-trace-fri-header-tied-sampled-assignment-conflict-without-recorded-missing-bound-from-auth-gap-and-replay-
    (rule auth-gap, rule merkle-bound, rule alignment,
      rule next-bound, rule successor, rule final)
show wp-event verify-monad
  (trace-fri-header-tied-sampled-assignment-conflict ?s) ?s
≤ trace-fri-header-replay-conflict-error-for A
unfolding trace-fri-header-replay-conflict-error-for-def Let-def
by (rule
  wp-trace-fri-header-tied-sampled-assignment-conflict-bound-from-recorded-missing-and-complement)
  (rule recorded-missing-bound, rule complement)
next
fix s
show ¬ trace-fri-header-tied-sampled-assignment-conflict s None
unfolding trace-fri-header-tied-sampled-assignment-conflict-def
  accepted-fri-opening-transcript-def
by simp
qed
have zero-bound:
  wp-event (checked-staged-security-experiment-with-data-state A)
    (staged-security-with-data-state-verifier-event
      trace-fri-header-tied-zero-round-final-obstruction)
    adversary-initial-state
≤ trace-fri-zero-round-agreement-set-target-error-for budgets +
  trace-fri-controlled-merkle-error-for budgets
proof –
have checked:
  wp-event (checked-staged-security-experiment-with-data-state A)
    (staged-security-with-data-state-verifier-event
      trace-fri-zero-round-checked-final-obstruction)
    adversary-initial-state
≤ trace-fri-zero-round-agreement-set-target-error-for budgets
by (rule checked-staged-security-trace-fri-zero-round-checked-final-obstruction-bound-from-agreement-sets
  [OF wf controlled])
have merkle:
  wp-event (checked-staged-security-experiment-with-data-state A)
    (staged-security-with-data-state-verifier-event
      partial-merkle-inconsistency-bad)
    adversary-initial-state
≤ trace-fri-controlled-merkle-error-for budgets
unfolding trace-fri-controlled-merkle-error-for-def
by (rule
  checked-staged-security-with-data-state-partial-merkle-inconsistency-bad-bound-controlled
  [OF wf controlled])
have split:
  wp-event (checked-staged-security-experiment-with-data-state A)

```

```

    (staged-security-with-data-state-verifier-event
     trace-fri-header-tied-zero-round-final-obstruction)
    adversary-initial-state
  ≤ wp-event (checked-staged-security-experiment-with-data-state A)
  (λout.
    staged-security-with-data-state-verifier-event
    trace-fri-zero-round-checked-final-obstruction out ∨
    staged-security-with-data-state-verifier-event
    partial-merkle-inconsistency-bad out)
    adversary-initial-state
unfolding staged-security-with-data-state-verifier-event-def
by (rule wp-event-mono-on-support)
  (auto dest:
    trace-fri-header-tied-zero-round-final-obstruction-imp-checked-or-merkle
    split: option.splits prod.splits)
also have ... ≤
  wp-event (checked-staged-security-experiment-with-data-state A)
  (staged-security-with-data-state-verifier-event
   trace-fri-zero-round-checked-final-obstruction)
  adversary-initial-state +
  wp-event (checked-staged-security-experiment-with-data-state A)
  (staged-security-with-data-state-verifier-event
   partial-merkle-inconsistency-bad)
  adversary-initial-state
by (rule wp-event-union-bound)
also have ... ≤
  trace-fri-zero-round-agreement-set-target-error-for budgets +
  trace-fri-controlled-merkle-error-for budgets
by (rule add-mono[OF checked merkle])
finally show ?thesis .
qed
have branch:
  wp-event (checked-staged-security-experiment-with-data-state A)
  (staged-security-with-data-state-verifier-event
   trace-fri-bad-with-header-tied-partial-candidate)
  adversary-initial-state
  ≤ trace-one-step-sampled-error-for budgets trace-bad +
    (trace-fri-header-replay-conflict-error-for A +
     (trace-fri-zero-round-agreement-set-target-error-for budgets +
      trace-fri-controlled-merkle-error-for budgets))
by (rule
  checked-staged-security-trace-fri-header-tied-partial-candidate-bound-from-sampled-conflict-and-staged-zero
  [OF sampled-header conflict-bound zero-bound])
show ?thesis
using branch
by (simp add: trace-header-one-step-error-for-def
  trace-fri-header-replay-conflict-zero-error-for-def)
qed

```

definition *trace-empty-header-refined-zero-error-for*

where

trace-empty-header-refined-zero-error-for budgets *A* *empty-bad* =
 (*trace-fri-zero-round-agreement-set-target-error-for* budgets +
trace-fri-controlled-merkle-error-for budgets +
 (*trace-header-one-step-error-for* budgets *A* *empty-bad* +
 (*trace-fri-controlled-merkle-error-for* budgets +
 (*reachable-verifier-event-bound-for* *A*
trace-fri-header-tied-low-candidate-untied-root-gap +
reachable-verifier-event-bound-for *A*
trace-fri-header-tied-low-candidate-untied-query-gap))) +
trace-header-one-step-error-for budgets *A* *empty-bad*)

definition *trace-empty-header-refined-one-step-error-for*

where

trace-empty-header-refined-one-step-error-for budgets *A* *empty-bad* =
trace-one-step-sampled-error-for budgets *empty-bad* +
 (*reachable-verifier-event-bound-for* *A*
trace-fri-sampled-assignment-conflict +
trace-empty-header-refined-zero-error-for budgets *A* *empty-bad*)

definition *concrete-trace-empty-header-refined-one-step-error-for*

where

concrete-trace-empty-header-refined-one-step-error-for budgets *A* =
trace-empty-header-refined-one-step-error-for budgets *A*
trace-one-step-split-bad

lemma *checked-staged-security-trace-fri-zero-round-final-obstruction-bound-from-agreement-sets-and-header:*

fixes *H* :: *prob*

assumes *wf*: *staged-budget-wellformed* budgets

and *controlled*: *staged-adversary-controlled* budgets *A*

and *header-bound*:

wp-event (*checked-staged-security-experiment-with-data-state* *A*)
 (*staged-security-with-data-state-verifier-event*
trace-fri-bad-with-header-tied-partial-candidate)
adversary-initial-state ≤ *H*

shows

wp-event (*checked-staged-security-experiment-with-data-state* *A*)
 (*staged-security-with-data-state-verifier-event*
trace-fri-zero-round-final-obstruction)
adversary-initial-state ≤
trace-fri-zero-round-agreement-set-target-error-for budgets +
trace-fri-controlled-merkle-error-for budgets +
 (*H* +
 (*trace-fri-controlled-merkle-error-for* budgets +
 (*reachable-verifier-event-bound-for* *A*
trace-fri-header-tied-low-candidate-untied-root-gap +
reachable-verifier-event-bound-for *A*
trace-fri-header-tied-low-candidate-untied-query-gap))) +

H

proof –

have *checked*:

wp-event (*checked-staged-security-experiment-with-data-state A*)
(staged-security-with-data-state-verifier-event
trace-fri-zero-round-checked-final-obstruction)
adversary-initial-state $\leq$
trace-fri-zero-round-agreement-set-target-error-for budgets

by (*rule*
checked-staged-security-trace-fri-zero-round-checked-final-obstruction-bound-from-agreement-sets
[OF wf controlled])

have *merkle*:

wp-event (*checked-staged-security-experiment-with-data-state A*)
(staged-security-with-data-state-verifier-event
partial-merkle-inconsistency-bad)
adversary-initial-state $\leq$
trace-fri-controlled-merkle-error-for budgets

unfolding *trace-fri-controlled-merkle-error-for-def*

by (*rule*
checked-staged-security-with-data-state-partial-merkle-inconsistency-bad-bound-controlled
[OF wf controlled])

have *hash*:

wp-event (*checked-staged-security-experiment-with-data-state A*)
(staged-security-with-data-state-verifier-event
hash-map-output-collision-bad)
adversary-initial-state $\leq$
trace-fri-controlled-merkle-error-for budgets

unfolding *trace-fri-controlled-merkle-error-for-def*

by (*rule*
checked-staged-security-with-data-state-hash-map-output-collision-bad-bound-controlled
[OF wf controlled])

have *root*:

wp-event (*checked-staged-security-experiment-with-data-state A*)
(staged-security-with-data-state-verifier-event
trace-fri-header-tied-low-candidate-untied-root-gap)
adversary-initial-state $\leq$
reachable-verifier-event-bound-for A
trace-fri-header-tied-low-candidate-untied-root-gap

proof (*rule checked-staged-security-with-data-state-verifier-event-bound-from-cont*)

show $\bigwedge s. \neg \text{trace-fri-header-tied-low-candidate-untied-root-gap } s \text{ None}$

unfolding *trace-fri-header-tied-low-candidate-untied-root-gap-def*
trace-fri-header-tied-low-candidate-untied-reachable-witness-case-def
accepted-fri-opening-transcript-def

by *simp*

next

fix *data attacker-state*

assume *support: Some (data, attacker-state) ∈*
set-dist
(execute (checked-staged-transcript-program A))

```

    adversary-initial-state)
show wp-event verify-monad
  (trace-fri-header-tied-low-candidate-untied-root-gap
   (verifier-state-from-adversary attacker-state
    (staged-proof-transcript data)))
  (verifier-state-from-adversary attacker-state
   (staged-proof-transcript data))
  ≤ reachable-verifier-event-bound-for A
    trace-fri-header-tied-low-candidate-untied-root-gap
  by (rule reachable-verifier-event-bound-for-ge[OF support])
qed
have query:
  wp-event (checked-staged-security-experiment-with-data-state A)
    (staged-security-with-data-state-verifier-event
     trace-fri-header-tied-low-candidate-untied-query-gap)
  adversary-initial-state ≤
  reachable-verifier-event-bound-for A
    trace-fri-header-tied-low-candidate-untied-query-gap
proof (rule checked-staged-security-with-data-state-verifier-event-bound-from-cont)
show  $\bigwedge s. \neg \text{trace-fri-header-tied-low-candidate-untied-query-gap } s \text{ None}$ 
unfolding trace-fri-header-tied-low-candidate-untied-query-gap-def
  trace-fri-header-tied-low-candidate-untied-reachable-witness-case-def
  accepted-fri-opening-transcript-def
by simp
next
fix data attacker-state
assume support: Some (data, attacker-state) ∈
  set-dist
  (execute (checked-staged-transcript-program A)
   adversary-initial-state)
show wp-event verify-monad
  (trace-fri-header-tied-low-candidate-untied-query-gap
   (verifier-state-from-adversary attacker-state
    (staged-proof-transcript data)))
  (verifier-state-from-adversary attacker-state
   (staged-proof-transcript data))
  ≤ reachable-verifier-event-bound-for A
    trace-fri-header-tied-low-candidate-untied-query-gap
  by (rule reachable-verifier-event-bound-for-ge[OF support])
qed
let ?M = checked-staged-security-experiment-with-data-state A
let ?untied =
  staged-security-with-data-state-verifier-event
  trace-fri-header-tied-low-candidate-untied-reachable-witness-gap
let ?hash =
  staged-security-with-data-state-verifier-event
  hash-map-output-collision-bad
let ?root =
  staged-security-with-data-state-verifier-event

```



```

    trace-fri-header-tied-low-candidate-untied-root-gap
  let ?query =
    staged-security-with-data-state-verifier-event
    trace-fri-header-tied-low-candidate-untied-query-gap
  have untied:
    wp-event ?M ?untied adversary-initial-state ≤
      trace-fri-controlled-merkle-error-for budgets +
      (reachable-verifier-event-bound-for A
        trace-fri-header-tied-low-candidate-untied-root-gap +
        reachable-verifier-event-bound-for A
        trace-fri-header-tied-low-candidate-untied-query-gap)
  proof -
    have wp-event ?M ?untied adversary-initial-state ≤
      wp-event ?M (λout. ?hash out ∨ ?root out ∨ ?query out)
      adversary-initial-state
    unfolding staged-security-with-data-state-verifier-event-def
    by (rule wp-event-mono-on-support)
      (auto
        dest:
          trace-fri-header-tied-low-candidate-untied-gap-imp-hash-root-or-query
          split: option.splits prod.splits)
    also have ... ≤
      wp-event ?M ?hash adversary-initial-state +
      wp-event ?M (λout. ?root out ∨ ?query out)
      adversary-initial-state
    by (rule wp-event-union-bound)
    also have ... ≤
      wp-event ?M ?hash adversary-initial-state +
      (wp-event ?M ?root adversary-initial-state +
        wp-event ?M ?query adversary-initial-state)
    by (intro add-mono order-refl wp-event-union-bound)
    also have ... ≤
      trace-fri-controlled-merkle-error-for budgets +
      (reachable-verifier-event-bound-for A
        trace-fri-header-tied-low-candidate-untied-root-gap +
        reachable-verifier-event-bound-for A
        trace-fri-header-tied-low-candidate-untied-query-gap)
    by (intro add-mono hash root query)
    finally show ?thesis .
  qed
  have transfer:
    wp-event (checked-staged-security-experiment-with-data-state A)
      (staged-security-with-data-state-verifier-event
        trace-fri-header-tied-candidate-transfer-gap)
    adversary-initial-state ≤
      H +
      (trace-fri-controlled-merkle-error-for budgets +
        (reachable-verifier-event-bound-for A
          trace-fri-header-tied-low-candidate-untied-root-gap +

```

```

    reachable-verifier-event-bound-for A
    trace-fri-header-tied-low-candidate-untied-query-gap))
  by (rule
    checked-staged-security-trace-fri-header-tied-candidate-transfer-gap-bound-from-staged-header-and-untied
    [OF header-bound untied])
  show ?thesis
  by (rule
    checked-staged-security-trace-fri-zero-round-final-obstruction-bound-from-staged-checked-merkle-transfer-header
    [OF checked merkle transfer header-bound])
qed

lemma trace-empty-header-refined-one-step-error-for-bound:
  fixes empty-bad :: nat  $\Rightarrow$  'f list  $\Rightarrow$  'f set
  assumes wf: staged-budget-wellformed budgets
  and controlled: staged-adversary-controlled budgets A
  and empty-subset:
     $\bigwedge i$  prefix.
       $i < \text{fri-round-count-for-degree-bound } (\text{clength} - 1) \Rightarrow$ 
       $\text{length prefix} = i \Rightarrow$ 
       $\text{empty-bad } i \text{ prefix} \subseteq$ 
      fri-one-step-disagreement-challenges
        (trace-fri-canonical-layer-len i) (empty-pw i)
        (empty-layer i prefix) (empty-claimed i prefix)
        (empty-dom i prefix)
  shows
    wp-event (checked-staged-security-experiment-with-data-state A)
      (staged-security-with-data-state-verifier-event
        trace-fri-bad-with-empty-composition-header-candidates)
      adversary-initial-state
     $\leq$  trace-empty-header-refined-one-step-error-for budgets A empty-bad
  proof -
    have sampled:
      wp-event (checked-staged-security-experiment-with-data-state A)
        (staged-security-with-data-state-verifier-event
          trace-fri-sampled-query-bad-candidate)
        adversary-initial-state
       $\leq$  trace-one-step-sampled-error-for budgets empty-bad
    by (rule trace-one-step-sampled-error-for-bound
      [OF wf controlled empty-subset])
    have conflict-bound:
      wp-event (checked-staged-security-experiment-with-data-state A)
        (staged-security-with-data-state-verifier-event
          trace-fri-sampled-assignment-conflict)
        adversary-initial-state
       $\leq$  reachable-verifier-event-bound-for A
        trace-fri-sampled-assignment-conflict
  proof (rule checked-staged-security-with-data-state-verifier-event-bound-from-cont)
    show  $\bigwedge s. \neg \text{trace-fri-sampled-assignment-conflict } s \text{ None}$ 
    unfolding trace-fri-sampled-assignment-conflict-def

```

```

    trace-fri-partial-candidate-opening-evidence-def
    accepted-fri-opening-transcript-def
  by simp
next
fix data attacker-state
assume support: Some (data, attacker-state) ∈
  set-dist
  (execute (checked-staged-transcript-program A)
    adversary-initial-state)
show wp-event verify-monad
  (trace-fri-sampled-assignment-conflict
    (verifier-state-from-adversary attacker-state
      (staged-proof-transcript data)))
  (verifier-state-from-adversary attacker-state
    (staged-proof-transcript data))
≤ reachable-verifier-event-bound-for A
  trace-fri-sampled-assignment-conflict
  by (rule reachable-verifier-event-bound-for-ge[OF support])
qed
have header-bound:
  wp-event (checked-staged-security-experiment-with-data-state A)
    (staged-security-with-data-state-verifier-event
      trace-fri-bad-with-header-tied-partial-candidate)
    adversary-initial-state
≤ trace-header-one-step-error-for budgets A empty-bad
  by (rule trace-header-one-step-error-for-bound
    [OF wf controlled empty-subset])
have zero-bound:
  wp-event (checked-staged-security-experiment-with-data-state A)
    (staged-security-with-data-state-verifier-event
      trace-fri-zero-round-final-obstruction)
    adversary-initial-state
≤ trace-empty-header-refined-zero-error-for budgets A empty-bad
  unfolding trace-empty-header-refined-zero-error-for-def
  by (rule
    checked-staged-security-trace-fri-zero-round-final-obstruction-bound-from-agreement-sets-and-header
      [OF wf controlled header-bound])
have branch:
  wp-event (checked-staged-security-experiment-with-data-state A)
    (staged-security-with-data-state-verifier-event
      trace-fri-bad-with-empty-composition-header-candidates)
    adversary-initial-state
≤ trace-one-step-sampled-error-for budgets empty-bad +
  (reachable-verifier-event-bound-for A
    trace-fri-sampled-assignment-conflict +
    trace-empty-header-refined-zero-error-for budgets A empty-bad)
  by (rule
    checked-staged-security-trace-fri-empty-header-bound-from-staged-sampled-conflict-and-zero
      [OF sampled conflict-bound zero-bound])

```

show *?thesis*
using *branch*
by (*simp add: trace-empty-header-refined-one-step-error-for-def*)
qed

theorem *stark-soundness-from-current-public-prefix-current-query-relevant-drift-route-from-one-step-fri-components*

fixes *trace-bad empty-bad :: nat $\Rightarrow$ 'f list $\Rightarrow$ 'f set*
and *comp-bad :: 'f $\Rightarrow$ nat $\Rightarrow$ 'f list $\Rightarrow$ 'f set*
assumes *false-statement: $\neg$ exists-valid-trace*
and *wf: staged-budget-wellformed budgets*
and *controlled: staged-adversary-controlled budgets A*
and *trace-subset:*
 $\bigwedge i$ *prefix.*
 $i < \text{fri-round-count-for-degree-bound } (\text{clength } - 1) \Rightarrow$
 $\text{length prefix} = i \Rightarrow$
 $\text{trace-bad } i \text{ prefix} \subseteq$
 $\text{fri-one-step-disagreement-challenges}$
 $(\text{trace-fri-canonical-layer-len } i) (\text{empty-pw } i)$
 $(\text{empty-layer } i \text{ prefix}) (\text{empty-claimed } i \text{ prefix})$
 $(\text{empty-dom } i \text{ prefix})$
and *comp-subset:*
 $\bigwedge dg$ *i prefix.*
 $i < \text{fri-round-count-for-degree-bound } (\text{to-nat } dg) \Rightarrow$
 $\text{length prefix} = i \Rightarrow$
 $\text{comp-bad } dg \text{ } i \text{ prefix} \subseteq$
 $\text{fri-one-step-disagreement-challenges}$
 $(\text{composition-fri-canonical-layer-len } dg \text{ } i) (\text{comp-pw } dg \text{ } i)$
 $(\text{comp-layer } dg \text{ } i \text{ prefix}) (\text{comp-claimed } dg \text{ } i \text{ prefix})$
 $(\text{comp-dom } dg \text{ } i \text{ prefix})$
and *empty-subset:*
 $\bigwedge i$ *prefix.*
 $i < \text{fri-round-count-for-degree-bound } (\text{clength } - 1) \Rightarrow$
 $\text{length prefix} = i \Rightarrow$
 $\text{empty-bad } i \text{ prefix} \subseteq$
 $\text{fri-one-step-disagreement-challenges}$
 $(\text{trace-fri-canonical-layer-len } i) (\text{empty-pw } i)$
 $(\text{empty-layer } i \text{ prefix}) (\text{empty-claimed } i \text{ prefix})$
 $(\text{empty-dom } i \text{ prefix})$
shows *checked-staged-adversary-acceptance-probability A $\leq$*
 $\text{hash-collision-budget-value } 0$
 $(\text{staged-attacker-query-budget budgets } +$
 $\text{staged-challenge-query-budget } + \text{verifier-hash-query-budget}) +$
 $(\text{composition-error-bound } +$
 $\text{staged-phase-relation-error size}$
 $(\text{staged-alpha-search-queries budgets } 0) +$
 $\text{hash-collision-budget-value } 0$
 $(\text{staged-alpha-search-queries budgets } 0) +$
 $\text{staged-phase-relation-error size}$
 $(\text{staged-alpha-search-queries budgets } 0) +$

(active-route-transcript-pre-error-for A +
 staged-concrete-transcript-target-error-bound)) +
 ($\sum i < \text{rounds}$.
 active-route-current-prefix-authenticated-error-for A i +
 (active-route-transcript-pre-error-for A +
 staged-concrete-transcript-target-error-bound) +
 (active-route-transcript-pre-error-for A +
 staged-concrete-transcript-target-error-bound)) +
 trace-header-one-step-error-for budgets A trace-bad +
 composition-one-step-verifier-tied-controlled-error-for budgets
 comp-bad +
 (composition-error-bound +
 staged-phase-relation-error size
 (staged-alpha-search-queries budgets 0) +
 hash-collision-budget-value 0
 (staged-alpha-search-queries budgets 0) +
 staged-phase-relation-error size
 (staged-alpha-search-queries budgets 0) +
 (active-route-transcript-pre-error-for A +
 staged-concrete-transcript-target-error-bound) +
 trace-empty-header-refined-one-step-error-for budgets A empty-bad +
 active-route-current-empty-query-error-for A)

proof —

have trace-bound:

wp-event (checked-staged-security-experiment-with-data-state A)
 (staged-security-with-data-state-verifier-event
 trace-fri-bad-with-header-tied-partial-candidate)
 adversary-initial-state
 $\leq$ trace-header-one-step-error-for budgets A trace-bad

by (rule trace-header-one-step-error-for-bound
 [OF wf controlled trace-subset])

have comp-bound:

wp-event (checked-staged-security-experiment-with-data-state A)
 (staged-security-with-data-state-verifier-event
 composition-fri-bad-with-verifier-tied-partial-candidate)
 adversary-initial-state
 $\leq$ composition-one-step-verifier-tied-controlled-error-for budgets
 comp-bad

by (rule composition-one-step-verifier-tied-controlled-error-for-bound
 [OF wf controlled comp-subset])

have empty-bound:

wp-event (checked-staged-security-experiment-with-data-state A)
 (staged-security-with-data-state-verifier-event
 trace-fri-bad-with-empty-composition-header-candidates)
 adversary-initial-state
 $\leq$ trace-empty-header-refined-one-step-error-for budgets A empty-bad

by (rule trace-empty-header-refined-one-step-error-for-bound
 [OF wf controlled empty-subset])

show ?thesis

by (rule
stark-soundness-from-current-public-prefix-current-query-relevant-drift-route-from-active-fri-bounds
 [OF false-statement wf controlled trace-bound comp-bound empty-bound])
 qed

theorem *stark-soundness-from-current-public-prefix-current-query-relevant-drift-route-one-step-endpoint:*
 assumes false-statement: $\neg$ exists-valid-trace
 and wf: staged-budget-wellformed budgets
 and controlled: staged-adversary-controlled budgets A
 shows checked-staged-adversary-acceptance-probability $A \leq$
 hash-collision-budget-value 0
 (staged-attacker-query-budget budgets +
 staged-challenge-query-budget + verifier-hash-query-budget) +
 (composition-error-bound +
 staged-phase-relation-error size
 (staged-alpha-search-queries budgets 0) +
 hash-collision-budget-value 0
 (staged-alpha-search-queries budgets 0) +
 staged-phase-relation-error size
 (staged-alpha-search-queries budgets 0) +
 (active-route-transcript-pre-error-for A +
 staged-concrete-transcript-target-error-bound)) +
 ($\sum i < \text{rounds.}$
 active-route-current-prefix-authenticated-error-for A i +
 (active-route-transcript-pre-error-for A +
 staged-concrete-transcript-target-error-bound) +
 (active-route-transcript-pre-error-for A +
 staged-concrete-transcript-target-error-bound)) +
 concrete-trace-header-one-step-error-for budgets A +
 concrete-composition-one-step-error-for budgets A +
 (composition-error-bound +
 staged-phase-relation-error size
 (staged-alpha-search-queries budgets 0) +
 hash-collision-budget-value 0
 (staged-alpha-search-queries budgets 0) +
 staged-phase-relation-error size
 (staged-alpha-search-queries budgets 0) +
 (active-route-transcript-pre-error-for A +
 staged-concrete-transcript-target-error-bound) +
 concrete-trace-empty-header-refined-one-step-error-for budgets A +
 active-route-current-empty-query-error-for A)
 unfolding concrete-trace-header-one-step-error-for-def
 concrete-composition-one-step-error-for-def
 concrete-trace-empty-header-refined-one-step-error-for-def
 by (rule
stark-soundness-from-current-public-prefix-current-query-relevant-drift-route-from-one-step-fri-components
 [where trace-bad = trace-one-step-split-bad
 and comp-bad = composition-one-step-split-bad
 and empty-bad = trace-one-step-split-bad

and *empty-pw* = *trace-one-step-split-pw*
and *empty-layer* = *trace-one-step-split-layer*
and *empty-claimed* = *trace-one-step-split-claimed*
and *empty-dom* = *trace-one-step-split-domain*
and *comp-pw* = *composition-one-step-split-pw*
and *comp-layer* = *composition-one-step-split-layer*
and *comp-claimed* = *composition-one-step-split-claimed*
and *comp-dom* = *composition-one-step-split-domain*,
OF false-statement wf controlled])
(*simp-all add*:
trace-one-step-split-bad-def
composition-one-step-split-bad-def
trace-one-step-disagreement-bad-for-def
composition-one-step-disagreement-bad-for-def)

theorem *stark-soundness-from-current-public-prefix-current-query-relevant-drift-route-sharpened*:

assumes *false-statement*: $\neg$ *exists-valid-trace*
and *wf*: *staged-budget-wellformed budgets*
and *controlled*: *staged-adversary-controlled budgets A*
shows *checked-staged-adversary-acceptance-probability A* $\leq$
hash-collision-budget-value 0
(*staged-attacker-query-budget budgets* +
staged-challenge-query-budget + *verifier-hash-query-budget*) +
(*composition-error-bound* +
staged-phase-relation-error size
(*staged-alpha-search-queries budgets 0*) +
hash-collision-budget-value 0
(*staged-alpha-search-queries budgets 0*) +
staged-phase-relation-error size
(*staged-alpha-search-queries budgets 0*) +
(*active-route-transcript-pre-error-for A* +
staged-concrete-transcript-target-error-bound)) +
($\sum i < \text{rounds}$.
active-route-current-prefix-authenticated-error-for A i +
(*active-route-transcript-pre-error-for A* +
staged-concrete-transcript-target-error-bound) +
(*active-route-transcript-pre-error-for A* +
staged-concrete-transcript-target-error-bound)) +
sharpened-trace-fri-header-error-for budgets A +
sharpened-composition-fri-error-for budgets A +
(*composition-error-bound* +
staged-phase-relation-error size
(*staged-alpha-search-queries budgets 0*) +
hash-collision-budget-value 0
(*staged-alpha-search-queries budgets 0*) +
staged-phase-relation-error size
(*staged-alpha-search-queries budgets 0*) +
(*active-route-transcript-pre-error-for A* +
staged-concrete-transcript-target-error-bound) +

sharpened-trace-fri-empty-error-for budgets A +
active-route-current-empty-query-error-for A)
by (rule
stark-soundness-from-current-public-prefix-current-query-relevant-drift-route-from-active-fri-bounds
[OF false-statement wf controlled
sharpened-trace-fri-header-error-for-bound[OF wf controlled]
sharpened-composition-fri-error-for-bound[OF wf controlled]
sharpened-trace-fri-empty-error-for-bound[OF wf controlled]])

theorem *stark-soundness-concrete-fri-sharpened-endpoint:*
assumes *false-statement: $\neg$ exists-valid-trace*
and *wf: staged-budget-wellformed budgets*
and *controlled: staged-adversary-controlled budgets A*
shows *checked-staged-adversary-acceptance-probability A $\leq$*
hash-collision-budget-value 0
(staged-attacker-query-budget budgets +
staged-challenge-query-budget + verifier-hash-query-budget) +
(composition-error-bound +
staged-phase-relation-error size
(staged-alpha-search-queries budgets 0) +
hash-collision-budget-value 0
(staged-alpha-search-queries budgets 0) +
staged-phase-relation-error size
(staged-alpha-search-queries budgets 0) +
(active-route-transcript-pre-error-for A +
staged-concrete-transcript-target-error-bound)) +
($\sum i < \text{rounds.}$
active-route-current-prefix-authenticated-error-for A i +
(active-route-transcript-pre-error-for A +
staged-concrete-transcript-target-error-bound) +
(active-route-transcript-pre-error-for A +
staged-concrete-transcript-target-error-bound)) +
concrete-trace-header-one-step-error-for budgets A +
concrete-composition-one-step-error-for budgets A +
(composition-error-bound +
staged-phase-relation-error size
(staged-alpha-search-queries budgets 0) +
hash-collision-budget-value 0
(staged-alpha-search-queries budgets 0) +
staged-phase-relation-error size
(staged-alpha-search-queries budgets 0) +
(active-route-transcript-pre-error-for A +
staged-concrete-transcript-target-error-bound) +
concrete-trace-empty-header-refined-one-step-error-for budgets A +
active-route-current-empty-query-error-for A)
by (rule
stark-soundness-from-current-public-prefix-current-query-relevant-drift-route-one-step-endpoint
[OF false-statement wf controlled])

end

hide-fact

soundness.stark-soundness-concrete-fri-sharpened-endpoint

soundness.stark-soundness-from-current-public-prefix-current-query-relevant-drift-route-sharpened

end

theory *Soundness-FRI-Canonical-Full-Cover-Failure*

imports *Soundness-FRI-Sampled-Challenge-Cover*

begin

Canonical full-cover failure events for sampled-query FRI.

The broad sampled full-cover failure predicates quantify over arbitrary domain witnesses.

The sampled-query split, however, produces the failure on the canonical FRI domains.

This layer names that narrower event so later bounds do not have to prove facts about unused arbitrary domain entries.

context *soundness*

begin

definition *trace-fri-sampled-canonical-full-cover-failure*

$:: ('f, 'a) \text{ protocol-channel-scheme} \Rightarrow$
 $('f \text{ list} \Rightarrow \text{nat} \Rightarrow 'f \text{ list} \Rightarrow 'f \text{ set}) \Rightarrow$
 $(\text{unit list} \times ('f, 'a) \text{ protocol-channel-scheme}) \text{ option} \Rightarrow \text{bool}$

where

trace-fri-sampled-canonical-full-cover-failure s bad out $\longleftrightarrow$
 $(\exists \text{ trace-roots trace-bs trace-final dg composition-roots}$
 $\text{ composition-bs composition-final fri-query-idxs trace-round-layers}$
 $\text{ composition-round-layers fr candidate-query-idxs trace-openings}$
 $\text{ trace-table layers.}$
trace-fri-partial-candidate-opening-evidence s out trace-roots
trace-bs trace-final dg composition-roots composition-bs
composition-final fri-query-idxs trace-round-layers
composition-round-layers fr candidate-query-idxs trace-openings
trace-table $\wedge$
generic-fri-sampled-query-candidate-evidence trace-table-low-degree
 $(\text{Not} \circ \text{trace-table-low-degree}) \text{ trace-table } (\text{clength} - 1)$
trace-roots trace-bs trace-final fri-query-idxs trace-round-layers
layers $\wedge$
generic-fri-sampled-full-cover-failure trace-table-low-degree
 $\text{ trace-table } (\text{clength} - 1) \text{ trace-roots trace-bs trace-final}$
 $\text{ fri-query-idxs trace-round-layers}$
 $(\text{fri-canonical-domains } (\text{length trace-bs})) \text{ layers}$
 $(\text{bad trace-table}))$

definition *composition-fri-sampled-canonical-full-cover-failure*

$:: ('f, 'a) \text{ protocol-channel-scheme} \Rightarrow$
 $('f \Rightarrow 'f \text{ list} \Rightarrow \text{nat} \Rightarrow 'f \text{ list} \Rightarrow 'f \text{ set}) \Rightarrow$

$(\text{unit list} \times ('f, 'a) \text{ protocol-channel-scheme}) \text{ option} \Rightarrow \text{bool}$
where
composition-fri-sampled-canonical-full-cover-failure s bad out
 $\longleftrightarrow$
 $(\exists \text{ trace-roots trace-bs trace-final fri-dg}$
 $\text{ opening-composition-roots composition-bs composition-final}$
 $\text{ fri-query-idxs trace-round-layers composition-round-layers fr}$
 $\text{ f-fri-roots f-final as dg composition-fri-roots final}$
 $\text{ trace-query-idxs trace-openings composition-query-idxs}$
 $\text{ composition-openings trace-table composition-table layers.}$
 $\text{ composition-fri-partial-candidate-opening-evidence s out trace-roots}$
 $\text{ trace-bs trace-final fri-dg opening-composition-roots composition-bs}$
 $\text{ composition-final fri-query-idxs trace-round-layers}$
 $\text{ composition-round-layers fr f-fri-roots f-final as dg}$
 $\text{ composition-fri-roots final trace-query-idxs trace-openings}$
 $\text{ composition-query-idxs composition-openings trace-table}$
 $\text{ composition-table} \wedge$
 $\text{ generic-fri-sampled-query-candidate-evidence}$
 $\text{ (composition-table-low-degree (to-nat fri-dg))}$
 $\text{ (Not } \circ \text{ composition-table-low-degree maxDegree) composition-table}$
 $\text{ (to-nat fri-dg) opening-composition-roots composition-bs}$
 $\text{ composition-final fri-query-idxs composition-round-layers layers} \wedge$
 $\text{ generic-fri-sampled-full-cover-failure}$
 $\text{ (composition-table-low-degree (to-nat fri-dg)) composition-table}$
 $\text{ (to-nat fri-dg) opening-composition-roots composition-bs}$
 $\text{ composition-final fri-query-idxs composition-round-layers}$
 $\text{ (fri-canonical-domains (length composition-bs)) layers}$
 $\text{ (bad fri-dg composition-table))}$

lemma *trace-fri-sampled-canonical-full-cover-failure-not-None[simp]:*

$\neg \text{ trace-fri-sampled-canonical-full-cover-failure s bad None}$

unfolding *trace-fri-sampled-canonical-full-cover-failure-def*

trace-fri-partial-candidate-opening-evidence-def

accepted-fri-opening-transcript-def

by *simp*

lemma *composition-fri-sampled-canonical-full-cover-failure-not-None[simp]:*

$\neg \text{ composition-fri-sampled-canonical-full-cover-failure s bad None}$

unfolding *composition-fri-sampled-canonical-full-cover-failure-def*

composition-fri-partial-candidate-opening-evidence-def

accepted-fri-opening-transcript-def

by *simp*

lemma *trace-fri-sampled-canonical-full-cover-failure-imp-broad:*

assumes *trace-fri-sampled-canonical-full-cover-failure s bad out*

shows *trace-fri-sampled-full-cover-failure s bad out*

proof —

from *assms* **obtain** *trace-roots trace-bs trace-final dg composition-roots*

composition-bs composition-final fri-query-idxs trace-round-layers

composition-round-layers fr candidate-query-idxs trace-openings
trace-table layers
where *evidence:*
trace-fri-partial-candidate-opening-evidence s out trace-roots
trace-bs trace-final dg composition-roots composition-bs
composition-final fri-query-idxs trace-round-layers
composition-round-layers fr candidate-query-idxs trace-openings
trace-table
and *candidate:*
generic-fri-sampled-query-candidate-evidence trace-table-low-degree
(Not $\circ$ trace-table-low-degree) trace-table (clength $- 1$)
trace-roots trace-bs trace-final fri-query-idxs trace-round-layers
layers
and *failure:*
generic-fri-sampled-full-cover-failure trace-table-low-degree
trace-table (clength $- 1$) trace-roots trace-bs trace-final
fri-query-idxs trace-round-layers
(fri-canonical-domains (length trace-bs)) layers
(bad trace-table)
unfolding *trace-fri-sampled-canonical-full-cover-failure-def* **by** *blast*
have *canonical:*
generic-fri-canonical-sampled-layer-chain-evidence trace-table-low-degree
trace-table (clength $- 1$) trace-roots trace-bs trace-final
fri-query-idxs trace-round-layers layers
by *(rule generic-fri-sampled-query-candidate-evidenceD(1)[OF candidate])*
have *sampled:*
generic-fri-sampled-layer-chain-evidence trace-table-low-degree
trace-table (clength $- 1$) trace-roots trace-bs trace-final
fri-query-idxs trace-round-layers
(fri-canonical-domains (length trace-bs)) layers
by *(rule generic-fri-canonical-sampled-layer-chain-evidenceD*
[OF canonical])
show *?thesis*
unfolding *trace-fri-sampled-full-cover-failure-def*
using *evidence sampled failure* **by** *blast*
qed

lemma *composition-fri-sampled-canonical-full-cover-failure-imp-broad:*
assumes *composition-fri-sampled-canonical-full-cover-failure s bad out*
shows *composition-fri-sampled-full-cover-failure s bad out*
proof $-$
from *assms* **obtain** *trace-roots trace-bs trace-final fri-dg*
opening-composition-roots composition-bs composition-final
fri-query-idxs trace-round-layers composition-round-layers fr
f-fri-roots f-final as dg composition-fri-roots final
trace-query-idxs trace-openings composition-query-idxs
composition-openings trace-table composition-table layers
where *evidence:*
composition-fri-partial-candidate-opening-evidence s out trace-roots

trace-bs trace-final fri-dg opening-composition-roots composition-bs
composition-final fri-query-idxs trace-round-layers
composition-round-layers fr f-fri-roots f-final as dg
composition-fri-roots final trace-query-idxs trace-openings
composition-query-idxs composition-openings trace-table
composition-table
and *candidate:*
generic-fri-sampled-query-candidate-evidence
(composition-table-low-degree (to-nat fri-dg))
(Not o composition-table-low-degree maxDegree) composition-table
(to-nat fri-dg) opening-composition-roots composition-bs
composition-final fri-query-idxs composition-round-layers layers
and *failure:*
generic-fri-sampled-full-cover-failure
(composition-table-low-degree (to-nat fri-dg)) composition-table
(to-nat fri-dg) opening-composition-roots composition-bs
composition-final fri-query-idxs composition-round-layers
(fri-canonical-domains (length composition-bs)) layers
(bad fri-dg composition-table)
unfolding *composition-fri-sampled-canonical-full-cover-failure-def*
by *meson*
have *canonical:*
generic-fri-canonical-sampled-layer-chain-evidence
(composition-table-low-degree (to-nat fri-dg)) composition-table
(to-nat fri-dg) opening-composition-roots composition-bs
composition-final fri-query-idxs composition-round-layers layers
by *(rule generic-fri-sampled-query-candidate-evidenceD(1)[OF candidate])*
have *sampled:*
generic-fri-sampled-layer-chain-evidence
(composition-table-low-degree (to-nat fri-dg)) composition-table
(to-nat fri-dg) opening-composition-roots composition-bs
composition-final fri-query-idxs composition-round-layers
(fri-canonical-domains (length composition-bs)) layers
by *(rule generic-fri-canonical-sampled-layer-chain-evidenceD*
[OF canonical])
show *?thesis*
unfolding *composition-fri-sampled-full-cover-failure-def*
using *evidence sampled failure by meson*
qed

lemma *generic-fri-canonical-sampled-layer-chain-next-idx-current-bound:*
assumes *chain:*
generic-fri-sampled-layer-chain-evidence low-degree candidate-table
degree-bound roots challenges final-value query-idxs round-layers
(fri-canonical-domains (length challenges)) layers
and *round-bound: round-idx < length query-idxs*
and *layer-bound: layer-idx < length challenges*
shows
fri-evidence-next-idx roots query-idxs round-idx layer-idx <

```

    length (fri-canonical-domain-at layer-idx)
proof –
  have fold:
    fri-sampled-table-fold (challenges ! layer-idx)
      (fri-evidence-layer-len roots layer-idx)
      (fri-evidence-layer-idx roots query-idxs round-idx layer-idx)
      (2 ^ layer-idx)
      (fri-canonical-domains (length challenges) ! layer-idx)
      (layers ! layer-idx)
      (round-layers ! round-idx ! layer-idx)
      (layers ! Suc layer-idx !
        fri-evidence-next-idx roots query-idxs round-idx layer-idx)
  by (rule generic-fri-sampled-layer-chain-evidence-sample(2)
    [OF chain round-bound layer-bound])
from fold obtain xp xp-path xn xn-path where match:
  fri-opening-matches-table
    (fri-evidence-layer-len roots layer-idx)
    (fri-evidence-layer-idx roots query-idxs round-idx layer-idx)
    (layers ! layer-idx) xp xn
  by (elim fri-sampled-table-foldE)
have partial:
  generic-fri-partial-evidence low-degree candidate-table degree-bound
    roots challenges final-value query-idxs round-layers
  by (rule generic-fri-sampled-layer-chain-evidenceD(1)[OF chain])
have roots-len: length challenges = length roots
  by (rule generic-fri-partial-evidence-shapes(2)[OF partial])
have roots-bound: layer-idx < length roots
  using layer-bound roots-len by simp
have len-eq:
  fri-evidence-layer-len roots layer-idx =
    length (fri-canonical-domain-at layer-idx)
  unfolding fri-evidence-layer-len-def fri-canonical-domain-at-length
  using fri-layer-lengths-nth-div[OF roots-bound, of clength * scale]
  by simp
have raw-bound:
  fri-evidence-layer-idx roots query-idxs round-idx layer-idx <
    length (fri-canonical-domain-at layer-idx)
  using fri-opening-matches-tableD(1)[OF match] unfolding len-eq .
show ?thesis
proof (cases fri-evidence-layer-len roots layer-idx div 2 = 0)
  case True
  then show ?thesis
    using raw-bound unfolding fri-evidence-next-idx-def by simp
next
  case False
  have half-bound:
    fri-evidence-layer-len roots layer-idx div 2 ≤
      length (fri-canonical-domain-at layer-idx)
  unfolding len-eq by simp

```

```

have
  fri-evidence-next-idx roots query-idxs round-idx layer-idx <
    fri-evidence-layer-len roots layer-idx div 2
unfolding fri-evidence-next-idx-def
by (rule mod-less-divisor) (use False in simp)
then show ?thesis
  using half-bound by linarith
qed
qed

lemma generic-fri-canonical-sampled-layer-chain-next-idx-successor-bound:
assumes chain:
  generic-fri-sampled-layer-chain-evidence low-degree candidate-table
  degree-bound roots challenges final-value query-idxs round-layers
  (fri-canonical-domains (length challenges)) layers
and no-shape:
  ¬ generic-fri-full-cover-shape-failure roots challenges
  (fri-canonical-domains (length challenges)) layers
and round-bound: round-idx < length query-idxs
and layer-bound: layer-idx < length roots
shows
  fri-evidence-next-idx roots query-idxs round-idx layer-idx <
    length (fri-canonical-domains (length challenges) ! Suc layer-idx)
proof -
have partial:
  generic-fri-partial-evidence low-degree candidate-table degree-bound
  roots challenges final-value query-idxs round-layers
by (rule generic-fri-sampled-layer-chain-evidenceD(1)[OF chain])
have roots-len: length challenges = length roots
by (rule generic-fri-partial-evidence-shapes(2)[OF partial])
have layer-challenge-bound: layer-idx < length challenges
using layer-bound roots-len by simp
have no-arith:
  ¬ generic-fri-full-cover-arithmetic-shape-failure roots challenges
using no-shape
unfolding generic-fri-full-cover-shape-failure-def
  generic-fri-full-cover-arithmetic-shape-failure-def
by blast
have even-len:
  2 dvd fri-evidence-layer-len roots layer-idx
using no-arith layer-challenge-bound
unfolding generic-fri-full-cover-arithmetic-shape-failure-def by blast
have roots-bound: layer-idx < length roots
by (rule layer-bound)
have len-eq:
  fri-evidence-layer-len roots layer-idx =
    length (fri-canonical-domain-at layer-idx)
unfolding fri-evidence-layer-len-def fri-canonical-domain-at-length
using fri-layer-lengths-nth-div[OF roots-bound, of clength * scale]

```

```

  by simp
have next-dom-eq:
  fri-canonical-domains (length challenges) ! Suc layer-idx =
    fri-canonical-domain-at (Suc layer-idx)
  by (rule fri-canonical-domains-nth)
    (use layer-challenge-bound in simp)
have next-len:
  length (fri-canonical-domains (length challenges) ! Suc layer-idx) =
    fri-evidence-layer-len roots layer-idx div 2
proof -
  have length (fri-canonical-domains (length challenges) ! Suc layer-idx) =
    (clength * scale) div 2 ^ Suc layer-idx
    unfolding next-dom-eq fri-canonical-domain-at-length by simp
  also have ... = ((clength * scale) div 2 ^ layer-idx) div 2
    by (metis div-mult2-eq mult.commute power-Suc)
  also have ... = fri-evidence-layer-len roots layer-idx div 2
    unfolding fri-evidence-layer-len-def
    using fri-layer-lengths-nth-div[OF roots-bound, of clength * scale]
    by simp
  finally show ?thesis .
qed
have raw-current:
  fri-evidence-next-idx roots query-idxs round-idx layer-idx <
    length (fri-canonical-domain-at layer-idx)
  by (rule generic-fri-canonical-sampled-layer-chain-next-idx-current-bound
    [OF chain round-bound layer-challenge-bound])
have half-pos:
  0 < fri-evidence-layer-len roots layer-idx div 2
  using raw-current even-len unfolding len-eq by auto
have
  fri-evidence-next-idx roots query-idxs round-idx layer-idx <
    fri-evidence-layer-len roots layer-idx div 2
  unfolding fri-evidence-next-idx-def
  by (rule mod-less-divisor[OF half-pos])
then show ?thesis
  unfolding next-len .
qed

lemma generic-fri-canonical-sampled-domain-failure-imp-shape:
  assumes chain:
    generic-fri-sampled-layer-chain-evidence low-degree candidate-table
    degree-bound roots challenges final-value query-idxs round-layers
    (fri-canonical-domains (length challenges)) layers
  and sampled-domain:
    generic-fri-full-cover-sampled-domain-failure roots query-idxs
    (fri-canonical-domains (length challenges))
  shows
    generic-fri-full-cover-shape-failure roots challenges
    (fri-canonical-domains (length challenges)) layers

```

proof (rule *ccontr*)
assume *no-shape*:
 $\neg$ generic-fri-full-cover-shape-failure roots challenges
(fri-canonical-domains (length challenges)) layers
have *partial*:
generic-fri-partial-evidence low-degree candidate-table degree-bound
roots challenges final-value query-idxs round-layers
by (rule generic-fri-sampled-layer-chain-evidenceD(1)[OF chain])
have roots-len: length challenges = length roots
by (rule generic-fri-partial-evidence-shapes(2)[OF partial])
have roots-bound: length roots $\leq$ length challenges
using roots-len **by** simp
have *no-sampled*:
 $\neg$ generic-fri-full-cover-sampled-domain-failure roots query-idxs
(fri-canonical-domains (length challenges))
proof (rule fri-canonical-domains-no-sampled-domain-failure
[OF roots-bound])
fix round-idx layer-idx
assume round-bound: round-idx < length query-idxs
and layer-bound: layer-idx < length roots
show fri-evidence-next-idx roots query-idxs round-idx layer-idx <
length (fri-canonical-domain-at layer-idx)
by (rule generic-fri-canonical-sampled-layer-chain-next-idx-current-bound
[OF chain round-bound])
(use layer-bound roots-len **in** simp)
next
fix round-idx layer-idx
assume round-bound: round-idx < length query-idxs
and layer-bound: layer-idx < length roots
show fri-evidence-next-idx roots query-idxs round-idx layer-idx <
length (fri-canonical-domains (length challenges) ! Suc layer-idx)
by (rule
generic-fri-canonical-sampled-layer-chain-next-idx-successor-bound
[OF chain no-shape round-bound layer-bound])
qed
show False
using sampled-domain no-sampled **by** contradiction
qed

lemma generic-fri-canonical-domain-failure-imp-index-or-shape:
assumes chain:
generic-fri-sampled-layer-chain-evidence low-degree candidate-table
degree-bound roots challenges final-value query-idxs round-layers
(fri-canonical-domains (length challenges)) layers
and domain:
generic-fri-full-cover-domain-failure roots challenges
(fri-canonical-domains (length challenges))
shows
generic-fri-full-cover-index-failure roots query-idxs $\vee$

generic-fri-full-cover-shape-failure roots challenges
(fri-canonical-domains (length challenges)) layers
proof (*cases generic-fri-full-cover-index-failure roots query-idxs*)
case *True*
then show *?thesis by simp*
next
case *no-index: False*
have *partial:*
generic-fri-partial-evidence low-degree candidate-table degree-bound
roots challenges final-value query-idxs round-layers
by (*rule generic-fri-sampled-layer-chain-evidenceD(1)[OF chain]*)
have *roots-len: length challenges = length roots*
by (*rule generic-fri-partial-evidence-shapes(2)[OF partial]*)
have *sampled-domain:*
generic-fri-full-cover-sampled-domain-failure roots query-idxs
(fri-canonical-domains (length challenges))
using *generic-fri-full-cover-domain-failure-imp-index-or-sampled-domain*
[OF roots-len domain, of query-idxs] no-index
by *blast*
then have *shape:*
generic-fri-full-cover-shape-failure roots challenges
(fri-canonical-domains (length challenges)) layers
by (*rule generic-fri-canonical-sampled-domain-failure-imp-shape*
[OF chain])
then show *?thesis by simp*
qed

lemma *trace-fri-sampled-canonical-full-cover-failure-split-no-domain:*

assumes *trace-fri-sampled-canonical-full-cover-failure s bad out*

shows

trace-fri-sampled-full-cover-proximity-failure s bad out $\vee$
trace-fri-sampled-full-cover-index-failure s out $\vee$
trace-fri-sampled-full-cover-shape-failure s out $\vee$
trace-fri-sampled-full-cover-start-low-failure s out

proof —

from *assms* **obtain** *trace-roots trace-bs trace-final dg composition-roots*
composition-bs composition-final fri-query-idxs trace-round-layers
composition-round-layers fr candidate-query-idxs trace-openings
trace-table layers

where *evidence:*

trace-fri-partial-candidate-opening-evidence s out trace-roots
trace-bs trace-final dg composition-roots composition-bs
composition-final fri-query-idxs trace-round-layers
composition-round-layers fr candidate-query-idxs trace-openings
trace-table

and *candidate:*

generic-fri-sampled-query-candidate-evidence trace-table-low-degree
(Not $\circ$ trace-table-low-degree) trace-table (clength — 1)
trace-roots trace-bs trace-final fri-query-idxs trace-round-layers

```

    layers
  and failure:
    generic-fri-sampled-full-cover-failure trace-table-low-degree
      trace-table (clength - 1) trace-roots trace-bs trace-final
      fri-query-idxs trace-round-layers
      (fri-canonical-domains (length trace-bs)) layers
      (bad trace-table)
  unfolding trace-fri-sampled-canonical-full-cover-failure-def by blast
have canonical:
  generic-fri-canonical-sampled-layer-chain-evidence
    trace-table-low-degree trace-table (clength - 1) trace-roots
    trace-bs trace-final fri-query-idxs trace-round-layers layers
  by (rule generic-fri-sampled-query-candidate-evidenceD(1)
      [OF candidate])
have chain:
  generic-fri-sampled-layer-chain-evidence trace-table-low-degree
    trace-table (clength - 1) trace-roots trace-bs trace-final
    fri-query-idxs trace-round-layers
    (fri-canonical-domains (length trace-bs)) layers
  by (rule generic-fri-canonical-sampled-layer-chain-evidenceD
      [OF canonical])
have cases:
  generic-fri-full-cover-proximity-failure (clength - 1) trace-bs
    (fri-canonical-domains (length trace-bs)) layers
    (bad trace-table) ∨
  generic-fri-full-cover-index-failure trace-roots fri-query-idxs ∨
  generic-fri-full-cover-shape-failure trace-roots trace-bs
    (fri-canonical-domains (length trace-bs)) layers ∨
  generic-fri-full-cover-domain-failure trace-roots trace-bs
    (fri-canonical-domains (length trace-bs)) ∨
  generic-fri-full-cover-start-low-failure (clength - 1)
    trace-table ∨
  generic-fri-full-cover-final-low-failure (clength - 1) trace-bs
    (fri-canonical-domains (length trace-bs)) layers
  by (rule generic-fri-sampled-full-cover-failure-cases[OF failure])
from cases show ?thesis
proof
  assume proximity:
    generic-fri-full-cover-proximity-failure (clength - 1) trace-bs
      (fri-canonical-domains (length trace-bs)) layers
      (bad trace-table)
  then show ?thesis
    unfolding trace-fri-sampled-full-cover-proximity-failure-def
    using evidence chain by blast
next
  assume rest:
    generic-fri-full-cover-index-failure trace-roots fri-query-idxs ∨
    generic-fri-full-cover-shape-failure trace-roots trace-bs
      (fri-canonical-domains (length trace-bs)) layers ∨

```

```

generic-fri-full-cover-domain-failure trace-roots trace-bs
(fri-canonical-domains (length trace-bs)) ∨
generic-fri-full-cover-start-low-failure (clength - 1)
trace-table ∨
generic-fri-full-cover-final-low-failure (clength - 1) trace-bs
(fri-canonical-domains (length trace-bs)) layers
then show ?thesis
proof
  assume index:
    generic-fri-full-cover-index-failure trace-roots fri-query-idxs
  then show ?thesis
    unfolding trace-fri-sampled-full-cover-index-failure-def
    using evidence chain by blast
next
  assume rest':
    generic-fri-full-cover-shape-failure trace-roots trace-bs
    (fri-canonical-domains (length trace-bs)) layers ∨
    generic-fri-full-cover-domain-failure trace-roots trace-bs
    (fri-canonical-domains (length trace-bs)) ∨
    generic-fri-full-cover-start-low-failure (clength - 1)
    trace-table ∨
    generic-fri-full-cover-final-low-failure (clength - 1) trace-bs
    (fri-canonical-domains (length trace-bs)) layers
  then show ?thesis
  proof
    assume shape:
      generic-fri-full-cover-shape-failure trace-roots trace-bs
      (fri-canonical-domains (length trace-bs)) layers
    then show ?thesis
      unfolding trace-fri-sampled-full-cover-shape-failure-def
      using evidence chain by blast
next
  assume rest'':
    generic-fri-full-cover-domain-failure trace-roots trace-bs
    (fri-canonical-domains (length trace-bs)) ∨
    generic-fri-full-cover-start-low-failure (clength - 1)
    trace-table ∨
    generic-fri-full-cover-final-low-failure (clength - 1)
    trace-bs (fri-canonical-domains (length trace-bs)) layers
  then show ?thesis
  proof
    assume domain:
      generic-fri-full-cover-domain-failure trace-roots trace-bs
      (fri-canonical-domains (length trace-bs))
    have index-or-shape:
      generic-fri-full-cover-index-failure trace-roots
      fri-query-idxs ∨
      generic-fri-full-cover-shape-failure trace-roots trace-bs
      (fri-canonical-domains (length trace-bs)) layers

```

```

    by (rule generic-fri-canonical-domain-failure-imp-index-or-shape
        [OF chain domain])
  then show ?thesis
proof
  assume generic-fri-full-cover-index-failure trace-roots
    fri-query-idxs
  then show ?thesis
    unfolding trace-fri-sampled-full-cover-index-failure-def
    using evidence chain by blast
next
  assume generic-fri-full-cover-shape-failure trace-roots
    trace-bs (fri-canonical-domains (length trace-bs)) layers
  then show ?thesis
    unfolding trace-fri-sampled-full-cover-shape-failure-def
    using evidence chain by blast
qed
next
  assume rest''':
    generic-fri-full-cover-start-low-failure (clength - 1)
    trace-table  $\vee$ 
    generic-fri-full-cover-final-low-failure (clength - 1)
    trace-bs (fri-canonical-domains (length trace-bs)) layers
  then show ?thesis
proof
  assume start:
    generic-fri-full-cover-start-low-failure (clength - 1)
    trace-table
  then show ?thesis
    unfolding trace-fri-sampled-full-cover-start-low-failure-def
    using evidence chain by blast
next
  assume final:
    generic-fri-full-cover-final-low-failure (clength - 1)
    trace-bs (fri-canonical-domains (length trace-bs)) layers
  have final-event:
    trace-fri-sampled-full-cover-final-low-failure s out
  unfolding trace-fri-sampled-full-cover-final-low-failure-def
  using evidence chain final by blast
  then have shape-event:
    trace-fri-sampled-full-cover-shape-failure s out
  by (rule trace-fri-sampled-full-cover-final-low-imp-shape)
  then show ?thesis by simp
qed
qed
qed
qed
qed
qed

```

lemma *composition-fri-sampled-canonical-full-cover-failure-split-no-domain:*

assumes *composition-fri-sampled-canonical-full-cover-failure s bad out*

shows

composition-fri-sampled-full-cover-proximity-failure s bad out $\vee$

composition-fri-sampled-full-cover-index-failure s out $\vee$

composition-fri-sampled-full-cover-shape-failure s out $\vee$

composition-fri-sampled-full-cover-start-low-failure s out

proof –

from *assms* **obtain** *trace-roots trace-bs trace-final fri-dg*

opening-composition-roots composition-bs composition-final

fri-query-idxs trace-round-layers composition-round-layers fr

f-fri-roots f-final as dg composition-fri-roots final

trace-query-idxs trace-openings composition-query-idxs

composition-openings trace-table composition-table layers

where *evidence:*

composition-fri-partial-candidate-opening-evidence s out trace-roots

trace-bs trace-final fri-dg opening-composition-roots composition-bs

composition-final fri-query-idxs trace-round-layers

composition-round-layers fr f-fri-roots f-final as dg

composition-fri-roots final trace-query-idxs trace-openings

composition-query-idxs composition-openings trace-table

composition-table

and *candidate:*

generic-fri-sampled-query-candidate-evidence

(composition-table-low-degree (to-nat fri-dg))

(Not $\circ$ composition-table-low-degree maxDegree) composition-table

(to-nat fri-dg) opening-composition-roots composition-bs

composition-final fri-query-idxs composition-round-layers layers

and *failure:*

generic-fri-sampled-full-cover-failure

(composition-table-low-degree (to-nat fri-dg)) composition-table

(to-nat fri-dg) opening-composition-roots composition-bs

composition-final fri-query-idxs composition-round-layers

(fri-canonical-domains (length composition-bs)) layers

(bad fri-dg composition-table)

unfolding *composition-fri-sampled-canonical-full-cover-failure-def*

by *meson*

have *canonical:*

generic-fri-canonical-sampled-layer-chain-evidence

(composition-table-low-degree (to-nat fri-dg)) composition-table

(to-nat fri-dg) opening-composition-roots composition-bs

composition-final fri-query-idxs composition-round-layers layers

by *(rule generic-fri-sampled-query-candidate-evidenceD(1)*

[OF candidate])

have *chain:*

generic-fri-sampled-layer-chain-evidence

(composition-table-low-degree (to-nat fri-dg)) composition-table

(to-nat fri-dg) opening-composition-roots composition-bs

composition-final fri-query-idxs composition-round-layers

```

    (fri-canonical-domains (length composition-bs)) layers
  by (rule generic-fri-canonical-sampled-layer-chain-evidenceD
    [OF canonical])
have cases:
generic-fri-full-cover-proximity-failure (to-nat fri-dg)
  composition-bs (fri-canonical-domains (length composition-bs))
  layers (bad fri-dg composition-table) ∨
generic-fri-full-cover-index-failure opening-composition-roots
  fri-query-idxs ∨
generic-fri-full-cover-shape-failure opening-composition-roots
  composition-bs (fri-canonical-domains (length composition-bs))
  layers ∨
generic-fri-full-cover-domain-failure opening-composition-roots
  composition-bs (fri-canonical-domains (length composition-bs)) ∨
generic-fri-full-cover-start-low-failure (to-nat fri-dg)
  composition-table ∨
generic-fri-full-cover-final-low-failure (to-nat fri-dg) composition-bs
  (fri-canonical-domains (length composition-bs)) layers
  by (rule generic-fri-sampled-full-cover-failure-cases[OF failure])
from cases show ?thesis
proof
  assume proximity:
    generic-fri-full-cover-proximity-failure (to-nat fri-dg)
    composition-bs (fri-canonical-domains (length composition-bs))
    layers (bad fri-dg composition-table)
  then show ?thesis
    unfolding composition-fri-sampled-full-cover-proximity-failure-def
    using evidence chain by meson
next
  assume rest:
    generic-fri-full-cover-index-failure opening-composition-roots
    fri-query-idxs ∨
    generic-fri-full-cover-shape-failure opening-composition-roots
    composition-bs (fri-canonical-domains (length composition-bs))
    layers ∨
    generic-fri-full-cover-domain-failure opening-composition-roots
    composition-bs (fri-canonical-domains (length composition-bs)) ∨
    generic-fri-full-cover-start-low-failure (to-nat fri-dg)
    composition-table ∨
    generic-fri-full-cover-final-low-failure (to-nat fri-dg)
    composition-bs (fri-canonical-domains (length composition-bs))
    layers
  then show ?thesis
proof
  assume index:
    generic-fri-full-cover-index-failure opening-composition-roots
    fri-query-idxs
  then show ?thesis
    unfolding composition-fri-sampled-full-cover-index-failure-def

```

```

    using evidence chain by meson
next
assume rest':
  generic-fri-full-cover-shape-failure opening-composition-roots
    composition-bs (fri-canonical-domains (length composition-bs))
    layers ∨
  generic-fri-full-cover-domain-failure opening-composition-roots
    composition-bs (fri-canonical-domains (length composition-bs)) ∨
  generic-fri-full-cover-start-low-failure (to-nat fri-dg)
    composition-table ∨
  generic-fri-full-cover-final-low-failure (to-nat fri-dg)
    composition-bs (fri-canonical-domains (length composition-bs))
    layers
then show ?thesis
proof
  assume shape:
    generic-fri-full-cover-shape-failure opening-composition-roots
      composition-bs (fri-canonical-domains (length composition-bs))
      layers
  then show ?thesis
  unfolding composition-fri-sampled-full-cover-shape-failure-def
  using evidence chain by meson
next
assume rest'':
  generic-fri-full-cover-domain-failure opening-composition-roots
    composition-bs (fri-canonical-domains (length composition-bs)) ∨
  generic-fri-full-cover-start-low-failure (to-nat fri-dg)
    composition-table ∨
  generic-fri-full-cover-final-low-failure (to-nat fri-dg)
    composition-bs (fri-canonical-domains (length composition-bs))
    layers
then show ?thesis
proof
  assume domain:
    generic-fri-full-cover-domain-failure opening-composition-roots
      composition-bs (fri-canonical-domains (length composition-bs))
  have index-or-shape:
    generic-fri-full-cover-index-failure opening-composition-roots
      fri-query-idxs ∨
    generic-fri-full-cover-shape-failure opening-composition-roots
      composition-bs (fri-canonical-domains (length composition-bs))
    layers
  by (rule generic-fri-canonical-domain-failure-imp-index-or-shape
    [OF chain domain])
then show ?thesis
proof
  assume generic-fri-full-cover-index-failure
    opening-composition-roots fri-query-idxs
  then show ?thesis

```

```

    unfolding composition-fri-sampled-full-cover-index-failure-def
    using evidence chain by meson
  next
    assume generic-fri-full-cover-shape-failure
      opening-composition-roots composition-bs
      (fri-canonical-domains (length composition-bs)) layers
    then show ?thesis
      unfolding composition-fri-sampled-full-cover-shape-failure-def
      using evidence chain by meson
  qed
next
  assume rest''':
    generic-fri-full-cover-start-low-failure (to-nat fri-dg)
    composition-table ∨
    generic-fri-full-cover-final-low-failure (to-nat fri-dg)
    composition-bs (fri-canonical-domains (length composition-bs))
    layers
  then show ?thesis
  proof
    assume start:
      generic-fri-full-cover-start-low-failure (to-nat fri-dg)
      composition-table
    then show ?thesis
      unfolding
        composition-fri-sampled-full-cover-start-low-failure-def
      using evidence chain by meson
  next
    assume final:
      generic-fri-full-cover-final-low-failure (to-nat fri-dg)
      composition-bs (fri-canonical-domains (length composition-bs))
      layers
    have final-event:
      composition-fri-sampled-full-cover-final-low-failure s out
    unfolding
      composition-fri-sampled-full-cover-final-low-failure-def
    using evidence chain final by meson
    then have shape-event:
      composition-fri-sampled-full-cover-shape-failure s out
      by (rule composition-fri-sampled-full-cover-final-low-imp-shape)
    then show ?thesis by simp
  qed
qed
qed
qed
qed
qed
qed

lemma wp-trace-fri-sampled-canonical-full-cover-failure-bound-from-shape-subcases:
  assumes proximity:

```


wp-event verify-monad
 (trace-fri-sampled-full-cover-proximity-failure s bad) $s \leq P$
and index:
wp-event verify-monad
 (trace-fri-sampled-full-cover-index-failure s) $s \leq I$
and shape:
wp-event verify-monad
 (trace-fri-sampled-full-cover-shape-failure s) $s \leq Sh$
shows
wp-event verify-monad
 (trace-fri-sampled-canonical-full-cover-failure s bad) s
 $\leq P + I + Sh$
proof –
have *wp-event verify-monad*
 (trace-fri-sampled-canonical-full-cover-failure s bad) s
 $\leq$ *wp-event verify-monad*
 ($\lambda out.$
 trace-fri-sampled-full-cover-proximity-failure s bad out $\vee$
 trace-fri-sampled-full-cover-index-failure s out $\vee$
 trace-fri-sampled-full-cover-shape-failure s out $\vee$
 trace-fri-sampled-full-cover-start-low-failure s out) s
by (rule *wp-event-mono*)
 (rule trace-fri-sampled-canonical-full-cover-failure-split-no-domain)
also have ... $\leq$
wp-event verify-monad
 (trace-fri-sampled-full-cover-proximity-failure s bad) s +
wp-event verify-monad
 (trace-fri-sampled-full-cover-index-failure s) s +
wp-event verify-monad
 (trace-fri-sampled-full-cover-shape-failure s) s +
wp-event verify-monad
 (trace-fri-sampled-full-cover-start-low-failure s) s
by (rule *wp-event-union-bound4*)
also have ... $\leq P + I + Sh + 0$
using *proximity index shape*
wp-trace-fri-sampled-full-cover-start-low-failure-zero[of s]
by (*intro add-mono*)
finally show *?thesis*
by *simp*
qed

lemma *wp-composition-fri-sampled-canonical-full-cover-failure-bound-from-shape-subcases:*
assumes *proximity:*
wp-event verify-monad
 (composition-fri-sampled-full-cover-proximity-failure s bad) $s \leq P$
and index:
wp-event verify-monad
 (composition-fri-sampled-full-cover-index-failure s) $s \leq I$
and shape:

$wp\text{-event verify-monad}$
 $(\text{composition-fri-sampled-full-cover-shape-failure } s) \ s \leq Sh$
shows
 $wp\text{-event verify-monad}$
 $(\text{composition-fri-sampled-canonical-full-cover-failure } s \text{ bad}) \ s$
 $\leq P + I + Sh$
proof –
have $wp\text{-event verify-monad}$
 $(\text{composition-fri-sampled-canonical-full-cover-failure } s \text{ bad}) \ s$
 $\leq wp\text{-event verify-monad}$
 $(\lambda out.$
 $\text{composition-fri-sampled-full-cover-proximity-failure } s \text{ bad out} \vee$
 $\text{composition-fri-sampled-full-cover-index-failure } s \text{ out} \vee$
 $\text{composition-fri-sampled-full-cover-shape-failure } s \text{ out} \vee$
 $\text{composition-fri-sampled-full-cover-start-low-failure } s \text{ out}) \ s$
by $(\text{rule } wp\text{-event-mono})$
 $(\text{rule}$
 $\text{composition-fri-sampled-canonical-full-cover-failure-split-no-domain})$
also have $\dots \leq$
 $wp\text{-event verify-monad}$
 $(\text{composition-fri-sampled-full-cover-proximity-failure } s \text{ bad}) \ s +$
 $wp\text{-event verify-monad}$
 $(\text{composition-fri-sampled-full-cover-index-failure } s) \ s +$
 $wp\text{-event verify-monad}$
 $(\text{composition-fri-sampled-full-cover-shape-failure } s) \ s +$
 $wp\text{-event verify-monad}$
 $(\text{composition-fri-sampled-full-cover-start-low-failure } s) \ s$
by $(\text{rule } wp\text{-event-union-bound}_4)$
also have $\dots \leq P + I + Sh + 0$
using $\text{proximity index shape}$
 $wp\text{-composition-fri-sampled-full-cover-start-low-failure-zero}[of \ s]$
by (intro add-mono)
finally show $?thesis$
by simp
qed

lemma $wp\text{-trace-fri-sampled-canonical-full-cover-failure-bound-from-witness-shape}$:
assumes proximity :
 $wp\text{-event verify-monad}$
 $(\text{trace-fri-sampled-full-cover-proximity-failure } s \text{ bad}) \ s \leq P$
and index :
 $wp\text{-event verify-monad}$
 $(\text{trace-fri-sampled-full-cover-index-failure } s) \ s \leq I$
and witness :
 $wp\text{-event verify-monad}$
 $(\text{trace-fri-sampled-full-cover-witness-length-failure } s) \ s \leq W$
shows
 $wp\text{-event verify-monad}$
 $(\text{trace-fri-sampled-canonical-full-cover-failure } s \text{ bad}) \ s$

$\leq P + I + W$
by (rule
wp-trace-fri-sampled-canonical-full-cover-failure-bound-from-shape-subcases
[OF proximity index])
(rule *wp-trace-fri-sampled-full-cover-shape-bound-from-witness-length*
[OF witness])

lemma *wp-composition-fri-sampled-canonical-full-cover-failure-bound-from-witness-shape:*

assumes *proximity:*

wp-event verify-monad
(composition-fri-sampled-full-cover-proximity-failure s bad) s $\leq P$

and *index:*

wp-event verify-monad
(composition-fri-sampled-full-cover-index-failure s) s $\leq I$

and *witness:*

wp-event verify-monad
(composition-fri-sampled-full-cover-witness-length-failure s) s
 $\leq W$

shows

wp-event verify-monad
(composition-fri-sampled-canonical-full-cover-failure s bad) s
 $\leq P + I + W$

by (rule

wp-composition-fri-sampled-canonical-full-cover-failure-bound-from-shape-subcases
[OF proximity index])

(rule

wp-composition-fri-sampled-full-cover-shape-bound-from-witness-length
[OF witness])

lemma *checked-staged-security-trace-fri-sampled-canonical-full-cover-failure-bound-from-shape-subcases:*

assumes *proximity:*

$\wedge$ *data attacker-state.*

Some (data, attacker-state) ∈
set-dist (execute (checked-staged-transcript-program A)
adversary-initial-state) ⇒

wp-event verify-monad
(trace-fri-sampled-full-cover-proximity-failure
(verifier-state-from-adversary attacker-state
(staged-proof-transcript data)) bad)
(verifier-state-from-adversary attacker-state
(staged-proof-transcript data)) ≤ P

and *index:*

$\wedge$ *data attacker-state.*

Some (data, attacker-state) ∈
set-dist (execute (checked-staged-transcript-program A)
adversary-initial-state) ⇒

wp-event verify-monad
(trace-fri-sampled-full-cover-index-failure
(verifier-state-from-adversary attacker-state

```

      (staged-proof-transcript data)))
    (verifier-state-from-adversary attacker-state
      (staged-proof-transcript data)) ≤ I
and shape:
   $\wedge$  data attacker-state.
    Some (data, attacker-state) ∈
      set-dist (execute (checked-staged-transcript-program A)
        adversary-initial-state)  $\implies$ 
    wp-event verify-monad
      (trace-fri-sampled-full-cover-shape-failure
        (verifier-state-from-adversary attacker-state
          (staged-proof-transcript data)))
      (verifier-state-from-adversary attacker-state
        (staged-proof-transcript data)) ≤ Sh
shows
  wp-event (checked-staged-security-experiment-with-data-state A)
    (staged-security-with-data-state-verifier-event
      ( $\lambda$ s. trace-fri-sampled-canonical-full-cover-failure s bad))
    adversary-initial-state ≤ P + I + Sh
proof (rule checked-staged-security-with-data-state-verifier-event-bound-from-cont)
show
   $\wedge$ s.  $\neg$  trace-fri-sampled-canonical-full-cover-failure s bad None
by simp
next
fix data attacker-state
assume support:
  Some (data, attacker-state) ∈
    set-dist (execute (checked-staged-transcript-program A)
      adversary-initial-state)
let ?s =
  verifier-state-from-adversary attacker-state
    (staged-proof-transcript data)
show wp-event verify-monad
  (trace-fri-sampled-canonical-full-cover-failure ?s bad) ?s
  ≤ P + I + Sh
by (rule
  wp-trace-fri-sampled-canonical-full-cover-failure-bound-from-shape-subcases)
  (rule proximity[OF support], rule index[OF support],
    rule shape[OF support])
qed

lemma checked-staged-security-trace-fri-sampled-canonical-full-cover-failure-bound-from-witness-shape:
assumes proximity:
   $\wedge$  data attacker-state.
    Some (data, attacker-state) ∈
      set-dist (execute (checked-staged-transcript-program A)
        adversary-initial-state)  $\implies$ 
    wp-event verify-monad
      (trace-fri-sampled-full-cover-proximity-failure

```

```

    (verifier-state-from-adversary attacker-state
      (staged-proof-transcript data)) bad)
  (verifier-state-from-adversary attacker-state
    (staged-proof-transcript data)) ≤ P
and index:
  ∧ data attacker-state.
    Some (data, attacker-state) ∈
      set-dist (execute (checked-staged-transcript-program A)
        adversary-initial-state) ⇒
    wp-event verify-monad
      (trace-fri-sampled-full-cover-index-failure
        (verifier-state-from-adversary attacker-state
          (staged-proof-transcript data)))
      (verifier-state-from-adversary attacker-state
        (staged-proof-transcript data)) ≤ I
and witness:
  ∧ data attacker-state.
    Some (data, attacker-state) ∈
      set-dist (execute (checked-staged-transcript-program A)
        adversary-initial-state) ⇒
    wp-event verify-monad
      (trace-fri-sampled-full-cover-witness-length-failure
        (verifier-state-from-adversary attacker-state
          (staged-proof-transcript data)))
      (verifier-state-from-adversary attacker-state
        (staged-proof-transcript data)) ≤ W
shows
  wp-event (checked-staged-security-experiment-with-data-state A)
    (staged-security-with-data-state-verifier-event
      (λs. trace-fri-sampled-canonical-full-cover-failure s bad))
    adversary-initial-state ≤ P + I + W
proof (rule checked-staged-security-with-data-state-verifier-event-bound-from-cont)
show
  ∧ s. ¬ trace-fri-sampled-canonical-full-cover-failure s bad None
by simp
next
fix data attacker-state
assume support:
  Some (data, attacker-state) ∈
    set-dist (execute (checked-staged-transcript-program A)
      adversary-initial-state)
let ?s =
  verifier-state-from-adversary attacker-state
    (staged-proof-transcript data)
show wp-event verify-monad
  (trace-fri-sampled-canonical-full-cover-failure ?s bad) ?s
  ≤ P + I + W
by (rule
  wp-trace-fri-sampled-canonical-full-cover-failure-bound-from-witness-shape)

```

(rule proximity[OF support], rule index[OF support],
rule witness[OF support])

qed

lemma checked-staged-security-composition-fri-sampled-canonical-full-cover-failure-bound-from-shape-subcases:

assumes proximity:

$\wedge$ data attacker-state.

Some (data, attacker-state) $\in$
set-dist (execute (checked-staged-transcript-program A)
adversary-initial-state) $\implies$

wp-event verify-monad
(composition-fri-sampled-full-cover-proximity-failure
(verifier-state-from-adversary attacker-state
(staged-proof-transcript data)) bad)
(verifier-state-from-adversary attacker-state
(staged-proof-transcript data)) $\leq P$

and index:

$\wedge$ data attacker-state.

Some (data, attacker-state) $\in$
set-dist (execute (checked-staged-transcript-program A)
adversary-initial-state) $\implies$

wp-event verify-monad
(composition-fri-sampled-full-cover-index-failure
(verifier-state-from-adversary attacker-state
(staged-proof-transcript data)))
(verifier-state-from-adversary attacker-state
(staged-proof-transcript data)) $\leq I$

and shape:

$\wedge$ data attacker-state.

Some (data, attacker-state) $\in$
set-dist (execute (checked-staged-transcript-program A)
adversary-initial-state) $\implies$

wp-event verify-monad
(composition-fri-sampled-full-cover-shape-failure
(verifier-state-from-adversary attacker-state
(staged-proof-transcript data)))
(verifier-state-from-adversary attacker-state
(staged-proof-transcript data)) $\leq Sh$

shows

wp-event (checked-staged-security-experiment-with-data-state A)
(staged-security-with-data-state-verifier-event
($\lambda s.$ composition-fri-sampled-canonical-full-cover-failure s bad))
adversary-initial-state $\leq P + I + Sh$

proof (rule checked-staged-security-with-data-state-verifier-event-bound-from-cont)

show

$\wedge s. \neg$ composition-fri-sampled-canonical-full-cover-failure s bad
None

by simp

next

```

fix data attacker-state
assume support:
  Some (data, attacker-state) ∈
    set-dist (execute (checked-staged-transcript-program A)
      adversary-initial-state)
let ?s =
  verifier-state-from-adversary attacker-state
    (staged-proof-transcript data)
show wp-event verify-monad
  (composition-fri-sampled-canonical-full-cover-failure ?s bad) ?s
  ≤ P + I + Sh
by (rule
  wp-composition-fri-sampled-canonical-full-cover-failure-bound-from-shape-subcases)
  (rule proximity[OF support], rule index[OF support],
  rule shape[OF support])
qed

lemma checked-staged-security-composition-fri-sampled-canonical-full-cover-failure-bound-from-witness-shape:
assumes proximity:
  ∧data attacker-state.
  Some (data, attacker-state) ∈
    set-dist (execute (checked-staged-transcript-program A)
      adversary-initial-state) ⇒
  wp-event verify-monad
  (composition-fri-sampled-full-cover-proximity-failure
    (verifier-state-from-adversary attacker-state
      (staged-proof-transcript data)) bad)
  (verifier-state-from-adversary attacker-state
    (staged-proof-transcript data)) ≤ P
and index:
  ∧data attacker-state.
  Some (data, attacker-state) ∈
    set-dist (execute (checked-staged-transcript-program A)
      adversary-initial-state) ⇒
  wp-event verify-monad
  (composition-fri-sampled-full-cover-index-failure
    (verifier-state-from-adversary attacker-state
      (staged-proof-transcript data)))
  (verifier-state-from-adversary attacker-state
    (staged-proof-transcript data)) ≤ I
and witness:
  ∧data attacker-state.
  Some (data, attacker-state) ∈
    set-dist (execute (checked-staged-transcript-program A)
      adversary-initial-state) ⇒
  wp-event verify-monad
  (composition-fri-sampled-full-cover-witness-length-failure
    (verifier-state-from-adversary attacker-state
      (staged-proof-transcript data)))

```

$(\text{verifier-state-from-adversary attacker-state}$
 $(\text{staged-proof-transcript data})) \leq W$

shows

$\text{wp-event } (\text{checked-staged-security-experiment-with-data-state } A)$
 $(\text{staged-security-with-data-state-verifier-event}$
 $(\lambda s. \text{composition-fri-sampled-canonical-full-cover-failure } s \text{ bad}))$
 $\text{adversary-initial-state} \leq P + I + W$

proof (rule checked-staged-security-with-data-state-verifier-event-bound-from-cont)

show

$\bigwedge s. \neg \text{composition-fri-sampled-canonical-full-cover-failure } s \text{ bad}$
 None

by simp

next

fix data attacker-state

assume support:

$\text{Some } (data, \text{attacker-state}) \in$
 $\text{set-dist } (\text{execute } (\text{checked-staged-transcript-program } A)$
 $\text{adversary-initial-state})$

let ?s =

$\text{verifier-state-from-adversary attacker-state}$
 $(\text{staged-proof-transcript data})$

show wp-event verify-monad

$(\text{composition-fri-sampled-canonical-full-cover-failure } ?s \text{ bad}) \text{ ?s}$
 $\leq P + I + W$

by (rule

$\text{wp-composition-fri-sampled-canonical-full-cover-failure-bound-from-witness-shape})$
 $(\text{rule proximity}[OF \text{ support}], \text{rule index}[OF \text{ support}],$
 $\text{rule witness}[OF \text{ support}])$

qed

lemma trace-fri-sampled-query-bad-candidate-imp-no-full-cover-or-canonical-failure:

assumes sampled: trace-fri-sampled-query-bad-candidate s out

shows

$\text{trace-fri-sampled-query-no-full-cover-failure-candidate } s \text{ bad out} \vee$
 $\text{trace-fri-sampled-canonical-full-cover-failure } s \text{ bad out}$

proof –

from sampled **obtain** trace-roots trace-bs trace-final dg composition-roots

$\text{composition-bs composition-final fri-query-idxs trace-round-layers}$
 $\text{composition-round-layers fr candidate-query-idxs trace-openings}$
 $\text{trace-table layers}$

where evidence:

$\text{trace-fri-partial-candidate-opening-evidence } s \text{ out trace-roots}$
 $\text{trace-bs trace-final dg composition-roots composition-bs}$
 $\text{composition-final fri-query-idxs trace-round-layers}$
 $\text{composition-round-layers fr candidate-query-idxs trace-openings}$
 trace-table

and candidate:

$\text{generic-fri-sampled-query-candidate-evidence trace-table-low-degree}$
 $(\text{Not } \circ \text{ trace-table-low-degree}) \text{ trace-table } (\text{clength} - 1)$


```

    trace-roots trace-bs trace-final fri-query-idxs trace-round-layers
    layers
  unfolding trace-fri-sampled-query-bad-candidate-def by blast
show ?thesis
proof (cases
  generic-fri-sampled-full-cover-failure trace-table-low-degree
    trace-table (clength - 1) trace-roots trace-bs trace-final
    fri-query-idxs trace-round-layers
    (fri-canonical-domains (length trace-bs)) layers
    (bad trace-table))
case False
then have
  trace-fri-sampled-query-no-full-cover-failure-candidate s bad out
  unfolding trace-fri-sampled-query-no-full-cover-failure-candidate-def
  using evidence candidate by blast
then show ?thesis by simp
next
case True
then have
  trace-fri-sampled-canonical-full-cover-failure s bad out
  unfolding trace-fri-sampled-canonical-full-cover-failure-def
  using evidence candidate by blast
then show ?thesis by simp
qed
qed

lemma composition-fri-sampled-query-bad-candidate-imp-no-full-cover-or-canonical-failure:
  assumes sampled: composition-fri-sampled-query-bad-candidate s out
  shows
    composition-fri-sampled-query-no-full-cover-failure-candidate s bad out  $\vee$ 
    composition-fri-sampled-canonical-full-cover-failure s bad out
proof -
  from sampled obtain trace-roots trace-bs trace-final fri-dg
    opening-composition-roots composition-bs composition-final
    fri-query-idxs trace-round-layers composition-round-layers fr
    f-fri-roots f-final as dg composition-fri-roots final
    trace-query-idxs trace-openings composition-query-idxs
    composition-openings trace-table composition-table layers
  where evidence:
    composition-fri-partial-candidate-opening-evidence s out trace-roots
    trace-bs trace-final fri-dg opening-composition-roots composition-bs
    composition-final fri-query-idxs trace-round-layers
    composition-round-layers fr f-fri-roots f-final as dg
    composition-fri-roots final trace-query-idxs trace-openings
    composition-query-idxs composition-openings trace-table
    composition-table
  and candidate:
    generic-fri-sampled-query-candidate-evidence
    (composition-table-low-degree (to-nat fri-dg))

```

```

    (Not  $\circ$  composition-table-low-degree maxDegree) composition-table
    (to-nat fri-dg) opening-composition-roots composition-bs
    composition-final fri-query-idxs composition-round-layers layers
unfolding composition-fri-sampled-query-bad-candidate-def by meson
show ?thesis
proof (cases
  generic-fri-sampled-full-cover-failure
    (composition-table-low-degree (to-nat fri-dg)) composition-table
    (to-nat fri-dg) opening-composition-roots composition-bs
    composition-final fri-query-idxs composition-round-layers
    (fri-canonical-domains (length composition-bs)) layers
    (bad fri-dg composition-table))
  case False
  then have
    composition-fri-sampled-query-no-full-cover-failure-candidate s bad out
  unfolding
    composition-fri-sampled-query-no-full-cover-failure-candidate-def
  using evidence candidate by meson
  then show ?thesis by simp
next
  case True
  then have
    composition-fri-sampled-canonical-full-cover-failure s bad out
  unfolding composition-fri-sampled-canonical-full-cover-failure-def
  using evidence candidate by meson
  then show ?thesis by simp
qed
qed

lemma wp-trace-fri-sampled-query-bad-candidate-bound-from-no-full-cover-or-canonical-failure:
assumes no-failure-bound:
  wp-event verify-monad
    (trace-fri-sampled-query-no-full-cover-failure-candidate s bad) s
     $\leq N$ 
and failure-bound:
  wp-event verify-monad
    (trace-fri-sampled-canonical-full-cover-failure s bad) s  $\leq F$ 
shows
  wp-event verify-monad (trace-fri-sampled-query-bad-candidate s) s
     $\leq N + F$ 
proof –
have wp-event verify-monad (trace-fri-sampled-query-bad-candidate s) s
     $\leq$  wp-event verify-monad
      ( $\lambda$ out.
        trace-fri-sampled-query-no-full-cover-failure-candidate s bad out  $\vee$ 
        trace-fri-sampled-canonical-full-cover-failure s bad out) s
by (rule wp-event-mono)
    (rule
      trace-fri-sampled-query-bad-candidate-imp-no-full-cover-or-canonical-failure)

```

also have ... $\leq$
 $wp\text{-event verify-monad}$
 $(trace\text{-fri-sampled-query-no-full-cover-failure-candidate } s \text{ bad}) \ s +$
 $wp\text{-event verify-monad}$
 $(trace\text{-fri-sampled-canonical-full-cover-failure } s \text{ bad}) \ s$
by (rule *wp-event-union-bound*)
also have ... $\leq N + F$
by (rule *add-mono[OF no-failure-bound failure-bound]*)
finally show ?thesis .
qed

lemma *wp-composition-fri-sampled-query-bad-candidate-bound-from-no-full-cover-or-canonical-failure:*

assumes *no-failure-bound:*

$wp\text{-event verify-monad}$
 $(composition\text{-fri-sampled-query-no-full-cover-failure-candidate } s \text{ bad}) \ s$
 $\leq N$

and *failure-bound:*

$wp\text{-event verify-monad}$
 $(composition\text{-fri-sampled-canonical-full-cover-failure } s \text{ bad}) \ s \leq F$

shows

$wp\text{-event verify-monad } (composition\text{-fri-sampled-query-bad-candidate } s)$
 $\leq N + F$

proof –

have $wp\text{-event verify-monad } (composition\text{-fri-sampled-query-bad-candidate } s) \ s$
 $\leq wp\text{-event verify-monad}$

$(\lambda out.$
 $composition\text{-fri-sampled-query-no-full-cover-failure-candidate } s \text{ bad}$
 $out \vee$
 $composition\text{-fri-sampled-canonical-full-cover-failure } s \text{ bad } out) \ s$

by (rule *wp-event-mono*)

(rule
 $composition\text{-fri-sampled-query-bad-candidate-imp-no-full-cover-or-canonical-failure}$)

also have ... $\leq$

$wp\text{-event verify-monad}$
 $(composition\text{-fri-sampled-query-no-full-cover-failure-candidate } s \text{ bad})$
 $s +$

$wp\text{-event verify-monad}$
 $(composition\text{-fri-sampled-canonical-full-cover-failure } s \text{ bad}) \ s$

by (rule *wp-event-union-bound*)

also have ... $\leq N + F$

by (rule *add-mono[OF no-failure-bound failure-bound]*)

finally show ?thesis .

qed

lemma *checked-staged-security-trace-fri-sampled-query-bad-candidate-bound-from-no-full-cover-or-canonical-failure:*

assumes *no-failure-bound:*

$wp\text{-event } (checked\text{-staged-security-experiment-with-data-state } A)$
 $(staged\text{-security-with-data-state-verifier-event}$
 $(\lambda s. trace\text{-fri-sampled-query-no-full-cover-failure-candidate } s \text{ bad}))$

$adversary-initial-state \leq N$
and *failure-bound*:
 $wp-event \ (checked-staged-security-experiment-with-data-state \ A)$
 $(staged-security-with-data-state-verifier-event$
 $(\lambda s. trace-fri-sampled-canonical-full-cover-failure \ s \ bad))$
 $adversary-initial-state \leq F$
shows
 $wp-event \ (checked-staged-security-experiment-with-data-state \ A)$
 $(staged-security-with-data-state-verifier-event$
 $trace-fri-sampled-query-bad-candidate)$
 $adversary-initial-state \leq N + F$
proof –
have $wp-event \ (checked-staged-security-experiment-with-data-state \ A)$
 $(staged-security-with-data-state-verifier-event$
 $trace-fri-sampled-query-bad-candidate)$
 $adversary-initial-state \leq$
 $wp-event \ (checked-staged-security-experiment-with-data-state \ A)$
 $(\lambda out.$
 $staged-security-with-data-state-verifier-event$
 $(\lambda s. trace-fri-sampled-query-no-full-cover-failure-candidate \ s$
 $bad) \ out \ \vee$
 $staged-security-with-data-state-verifier-event$
 $(\lambda s. trace-fri-sampled-canonical-full-cover-failure \ s \ bad) \ out)$
 $adversary-initial-state$
by (rule *wp-event-mono*)
 $(auto \ simp: staged-security-with-data-state-verifier-event-def$
 $dest:$
 $trace-fri-sampled-query-bad-candidate-imp-no-full-cover-or-canonical-failure$
 $split: option.splits \ prod.splits)$
also have $\dots \leq$
 $wp-event \ (checked-staged-security-experiment-with-data-state \ A)$
 $(staged-security-with-data-state-verifier-event$
 $(\lambda s. trace-fri-sampled-query-no-full-cover-failure-candidate \ s \ bad))$
 $adversary-initial-state +$
 $wp-event \ (checked-staged-security-experiment-with-data-state \ A)$
 $(staged-security-with-data-state-verifier-event$
 $(\lambda s. trace-fri-sampled-canonical-full-cover-failure \ s \ bad))$
 $adversary-initial-state$
by (rule *wp-event-union-bound*)
also have $\dots \leq N + F$
by (rule *add-mono[OF no-failure-bound failure-bound]*)
finally show *?thesis* .
qed

lemma *checked-staged-security-composition-fri-sampled-query-bad-candidate-bound-from-no-full-cover-or-canonical-f*
assumes *no-failure-bound*:

$wp-event \ (checked-staged-security-experiment-with-data-state \ A)$
 $(staged-security-with-data-state-verifier-event$
 $(\lambda s.$

```

      composition-fri-sampled-query-no-full-cover-failure-candidate s
      bad))
    adversary-initial-state ≤ N
  and failure-bound:
    wp-event (checked-staged-security-experiment-with-data-state A)
      (staged-security-with-data-state-verifier-event
        (λs. composition-fri-sampled-canonical-full-cover-failure s bad))
    adversary-initial-state ≤ F
  shows
    wp-event (checked-staged-security-experiment-with-data-state A)
      (staged-security-with-data-state-verifier-event
        composition-fri-sampled-query-bad-candidate)
    adversary-initial-state ≤ N + F
  proof -
  have wp-event (checked-staged-security-experiment-with-data-state A)
    (staged-security-with-data-state-verifier-event
      composition-fri-sampled-query-bad-candidate)
    adversary-initial-state ≤
  wp-event (checked-staged-security-experiment-with-data-state A)
    (λout.
      staged-security-with-data-state-verifier-event
        (λs.
          composition-fri-sampled-query-no-full-cover-failure-candidate s
          bad) out ∨
          staged-security-with-data-state-verifier-event
            (λs. composition-fri-sampled-canonical-full-cover-failure s bad)
            out)
      adversary-initial-state
  by (rule wp-event-mono)
    (auto simp: staged-security-with-data-state-verifier-event-def
      dest:
        composition-fri-sampled-query-bad-candidate-imp-no-full-cover-or-canonical-failure
        split: option.splits prod.splits)
  also have ... ≤
    wp-event (checked-staged-security-experiment-with-data-state A)
      (staged-security-with-data-state-verifier-event
        (λs.
          composition-fri-sampled-query-no-full-cover-failure-candidate s
          bad))
      adversary-initial-state +
    wp-event (checked-staged-security-experiment-with-data-state A)
      (staged-security-with-data-state-verifier-event
        (λs. composition-fri-sampled-canonical-full-cover-failure s bad))
      adversary-initial-state
  by (rule wp-event-union-bound)
  also have ... ≤ N + F
  by (rule add-mono[OF no-failure-bound failure-bound])
  finally show ?thesis .
qed

```

end

end

theory *Soundness-FRI-Trace-Transcript-Consistent*
 imports *Soundness-FRI-Trace-Candidate-Blocker*
begin

Trace-only verifier-consumed FRI evidence.

The broad reachable trace FRI predicate is root-parametric: it accepts any authenticated trace root present in the final verifier state. This layer names the verifier-consumed alternative, where the trace root and query indices are those appearing in the accepted FRI opening transcript. It does not change the protocol or the public theorem interface.

context *soundness*
begin

definition *accepted-with-partial-trace-openings-verifier-consumed*

$:: ('f, 'a) \text{ protocol-channel-scheme} \Rightarrow$
 $(\text{unit list} \times ('f, 'a) \text{ protocol-channel-scheme}) \text{ option} \Rightarrow$
 $'f \text{ list} \Rightarrow 'f \text{ list} \Rightarrow 'f \Rightarrow 'f \Rightarrow$
 $'f \text{ list} \Rightarrow 'f \text{ list} \Rightarrow 'f \Rightarrow \text{nat list} \Rightarrow$
 $'f \text{ list list list} \Rightarrow 'f \text{ list list list} \Rightarrow$
 $'f \Rightarrow 'f \text{ list} \Rightarrow$
 $'f \text{ authenticated-opening list list} \Rightarrow \text{bool}$

where

accepted-with-partial-trace-openings-verifier-consumed s out
 $\text{trace-roots trace-bs trace-final dg composition-roots composition-bs}$
 $\text{composition-final query-idxs trace-round-layers}$
 $\text{composition-round-layers fr as trace-openings} \longleftrightarrow$
accepted-fri-opening-transcript s out trace-roots trace-bs trace-final
 $\text{dg composition-roots composition-bs composition-final query-idxs}$
 $\text{trace-round-layers composition-round-layers} \wedge$
 $(\exists \text{ rest.}$
 $\text{verifier-header-transcript s fr trace-roots trace-final as dg}$
 $\text{composition-roots composition-final rest}) \wedge$
 $\text{accepted-with-partial-trace-openings s out fr query-idxs trace-openings}$

lemma *accepted-with-partial-trace-openings-verifier-consumedD:*

assumes

accepted-with-partial-trace-openings-verifier-consumed s out
 $\text{trace-roots trace-bs trace-final dg composition-roots composition-bs}$
 $\text{composition-final query-idxs trace-round-layers}$
 $\text{composition-round-layers fr as trace-openings}$

shows

accepted-fri-opening-transcript s out trace-roots trace-bs trace-final
 $\text{dg composition-roots composition-bs composition-final query-idxs}$
 $\text{trace-round-layers composition-round-layers}$

$\exists$ rest. verifier-header-transcript s fr trace-roots trace-final as dg
 composition-roots composition-final rest
 accepted-with-partial-trace-openings s out fr query-idxs trace-openings
using *assms*
unfolding *accepted-with-partial-trace-openings-verifier-consumed-def*
by *simp-all*

definition *trace-fri-bad-with-verifier-consumed-partial-candidate*

$:: (f, 'a) \text{ protocol-channel-scheme} \Rightarrow$
 $(\text{unit list} \times (f, 'a) \text{ protocol-channel-scheme}) \text{ option} \Rightarrow \text{bool}$

where

trace-fri-bad-with-verifier-consumed-partial-candidate s out
 $\longleftrightarrow$
 $(\exists \text{ trace-roots trace-bs trace-final dg composition-roots}$
 $\text{ composition-bs composition-final fri-query-idxs trace-round-layers}$
 $\text{ composition-round-layers fr as trace-openings trace-table.}$
 $\text{ accepted-with-partial-trace-openings-verifier-consumed s out}$
 $\text{ trace-roots trace-bs trace-final dg composition-roots composition-bs}$
 $\text{ composition-final fri-query-idxs trace-round-layers}$
 $\text{ composition-round-layers fr as trace-openings} \wedge$
 $\text{ partial-trace-table-candidate trace-table trace-openings} \wedge$
 $\neg \text{ trace-table-low-degree trace-table})$

lemma *trace-fri-bad-with-verifier-consumed-imp-header-tied:*

assumes *trace-fri-bad-with-verifier-consumed-partial-candidate* s out

shows *trace-fri-bad-with-header-tied-partial-candidate* s out

proof —

from *assms* **obtain** *trace-roots trace-bs trace-final dg composition-roots*
 $\text{ composition-bs composition-final fri-query-idxs trace-round-layers}$
 $\text{ composition-round-layers fr as trace-openings trace-table}$ **where**
consumed:

$\text{ accepted-with-partial-trace-openings-verifier-consumed s out}$
 $\text{ trace-roots trace-bs trace-final dg composition-roots composition-bs}$
 $\text{ composition-final fri-query-idxs trace-round-layers}$
 $\text{ composition-round-layers fr as trace-openings}$

and *candidate:*

$\text{ partial-trace-table-candidate trace-table trace-openings}$

and *not-low:* $\neg \text{ trace-table-low-degree trace-table}$

unfolding *trace-fri-bad-with-verifier-consumed-partial-candidate-def*

by *blast*

have *fri-openings:*

$\text{ accepted-fri-opening-transcript s out trace-roots trace-bs trace-final}$
 $\text{ dg composition-roots composition-bs composition-final fri-query-idxs}$
 $\text{ trace-round-layers composition-round-layers}$

by (rule *accepted-with-partial-trace-openings-verifier-consumedD(1)*)
 $[\text{OF consumed}]$

from *accepted-with-partial-trace-openings-verifier-consumedD(2)[OF consumed]*

obtain rest **where** *header:*

$\text{ verifier-header-transcript s fr trace-roots trace-final as dg}$

composition-roots composition-final rest
by *blast*
have *partial:*
accepted-with-partial-trace-openings s out fr fri-query-idxs
trace-openings
by (*rule accepted-with-partial-trace-openings-verifier-consumedD(3)*
[OF consumed])
show *?thesis*
unfolding *trace-fri-bad-with-header-tied-partial-candidate-def*
by (*intro exI conjI*)
(rule fri-openings, rule header, rule partial, rule candidate,
rule not-low)
qed

lemma *trace-fri-bad-with-header-tied-imp-verifier-consumed:*
assumes *trace-fri-bad-with-header-tied-partial-candidate s out*
shows *trace-fri-bad-with-verifier-consumed-partial-candidate s out*
proof –
from *assms* **obtain** *trace-roots trace-bs trace-final dg composition-roots*
composition-bs composition-final fri-query-idxs trace-round-layers
composition-round-layers fr as rest trace-openings trace-table **where**
fri-openings:
accepted-fri-opening-transcript s out trace-roots trace-bs
trace-final dg composition-roots composition-bs composition-final
fri-query-idxs trace-round-layers composition-round-layers
and *header:*
verifier-header-transcript s fr trace-roots trace-final as dg
composition-roots composition-final rest
and *partial:*
accepted-with-partial-trace-openings s out fr fri-query-idxs
trace-openings
and *candidate:*
partial-trace-table-candidate trace-table trace-openings
and *not-low: $\neg$ trace-table-low-degree trace-table*
unfolding *trace-fri-bad-with-header-tied-partial-candidate-def*
by *blast*
have *consumed:*
accepted-with-partial-trace-openings-verifier-consumed s out
trace-roots trace-bs trace-final dg composition-roots composition-bs
composition-final fri-query-idxs trace-round-layers
composition-round-layers fr as trace-openings
unfolding *accepted-with-partial-trace-openings-verifier-consumed-def*
by (*intro conjI exI*)
(rule fri-openings, rule header, rule partial)
show *?thesis*
unfolding *trace-fri-bad-with-verifier-consumed-partial-candidate-def*
by (*intro exI conjI*)
(rule consumed, rule candidate, rule not-low)
qed

lemma *trace-fri-bad-with-verifier-consumed-iff-header-tied:*
trace-fri-bad-with-verifier-consumed-partial-candidate s out
 $\longleftrightarrow$ *trace-fri-bad-with-header-tied-partial-candidate s out*
proof
 assume *trace-fri-bad-with-verifier-consumed-partial-candidate s out*
 then show *trace-fri-bad-with-header-tied-partial-candidate s out*
 by (rule *trace-fri-bad-with-verifier-consumed-imp-header-tied*)
 next
 assume *trace-fri-bad-with-header-tied-partial-candidate s out*
 then show *trace-fri-bad-with-verifier-consumed-partial-candidate s out*
 by (rule *trace-fri-bad-with-header-tied-imp-verifier-consumed*)
qed

lemma *trace-fri-header-tied-reduction-verifier-consumed-bound:*
 assumes *trace-fri-header-tied-reduction s*
 shows
wp-event verify-monad
 (*trace-fri-bad-with-verifier-consumed-partial-candidate s*) $s \leq$
trace-fri-error
proof –
 have *mono:*
wp-event verify-monad
 (*trace-fri-bad-with-verifier-consumed-partial-candidate s*) $s \leq$
wp-event verify-monad
 (*trace-fri-bad-with-header-tied-partial-candidate s*) s
 by (rule *wp-event-mono*)
 (rule *trace-fri-bad-with-verifier-consumed-imp-header-tied*)
 also have $\dots \leq$ *trace-fri-error*
 by (rule *trace-fri-header-tied-reductionD[OF assms]*)
 finally show ?thesis .
qed

lemma *trace-fri-verifier-consumed-reduction-staged-bound:*
 assumes *reductions:*
 $\bigwedge$ *data attacker-state.*
Some (data, attacker-state) \in
set-dist (execute (checked-staged-transcript-program A)
adversary-initial-state) \implies
trace-fri-header-tied-reduction
(verifier-state-from-adversary attacker-state
(staged-proof-transcript data))
 shows
wp-event (checked-staged-security-experiment-with-data-state A)
(staged-security-with-data-state-verifier-event
trace-fri-bad-with-verifier-consumed-partial-candidate)
adversary-initial-state \leq trace-fri-error
proof –
 have *mono:*

```

wp-event (checked-staged-security-experiment-with-data-state A)
  (staged-security-with-data-state-verifier-event
    trace-fri-bad-with-verifier-consumed-partial-candidate)
adversary-initial-state ≤
wp-event (checked-staged-security-experiment-with-data-state A)
  (staged-security-with-data-state-verifier-event
    trace-fri-bad-with-header-tied-partial-candidate)
adversary-initial-state
proof (rule wp-event-mono)
  fix out
  assume consumed:
    staged-security-with-data-state-verifier-event
    trace-fri-bad-with-verifier-consumed-partial-candidate out
  then show
    staged-security-with-data-state-verifier-event
    trace-fri-bad-with-header-tied-partial-candidate out
proof (cases out)
  case None
  then show ?thesis
    using consumed
    unfolding staged-security-with-data-state-verifier-event-def
    by simp
next
  case (Some payload)
  obtain data attacker-state result final-state where payload:
    payload = (((data, attacker-state), result), final-state)
  by (cases payload) auto
  have consumed':
    trace-fri-bad-with-verifier-consumed-partial-candidate
      (verifier-state-from-adversary attacker-state
        (staged-proof-transcript data))
      (Some (result, final-state))
  using consumed Some payload
  unfolding staged-security-with-data-state-verifier-event-def
  by simp
  have header:
    trace-fri-bad-with-header-tied-partial-candidate
      (verifier-state-from-adversary attacker-state
        (staged-proof-transcript data))
      (Some (result, final-state))
  by (rule trace-fri-bad-with-verifier-consumed-imp-header-tied
    [OF consumed'])
  show ?thesis
    using Some payload header
    unfolding staged-security-with-data-state-verifier-event-def
    by simp
qed
qed
also have ... ≤ trace-fri-error

```

```

    by (rule trace-fri-header-tied-reduction-staged-bound[OF reductions])
  finally show ?thesis .
qed

end

end

```

```

theory Soundness-FRI-Trace-Empty-Sampled-Assignment-Split
imports
  Soundness-Public-Route-Concrete-FRI-Sharpened
  Soundness-FRI-Canonical-Full-Cover-Failure
  Soundness-FRI-Trace-Transcript-Consistent
  Soundness-Conceptual-Query-Empty-Current
begin

```

First split for the remaining plain trace empty-branch sampled-assignment conflict. The plain event is intentionally broader than the low-degree sampled-query bad-candidate event: it does not require the conflicting candidate table to be non-low-degree. This layer separates the aligned verifier-tied part from the residual that still needs a Merkle/replay explanation.

```

context soundness
begin

```

```

lemma prob-le-refl:  $(x::prob) \leq x$ 
  by transfer simp

```

```

definition trace-fri-sampled-assignment-verifier-untied-gap
  :: ('f, 'a) protocol-channel-scheme  $\Rightarrow$ 
    (unit list  $\times$  ('f, 'a) protocol-channel-scheme) option  $\Rightarrow$  bool

```

```

where
  trace-fri-sampled-assignment-verifier-untied-gap s out  $\longleftrightarrow$ 
    trace-fri-sampled-assignment-conflict s out  $\wedge$ 
     $\neg$  trace-fri-verifier-tied-sampled-assignment-conflict s out

```

```

lemma trace-fri-sampled-assignment-conflict-imp-verifier-tied-or-untied:
assumes trace-fri-sampled-assignment-conflict s out
shows
  trace-fri-verifier-tied-sampled-assignment-conflict s out  $\vee$ 
  trace-fri-sampled-assignment-verifier-untied-gap s out
using assms
unfolding trace-fri-sampled-assignment-verifier-untied-gap-def by blast

```

```

lemma wp-trace-fri-sampled-assignment-conflict-bound-from-verifier-tied-and-untied:
assumes verifier-tied-bound:
  wp-event verify-monad
  (trace-fri-verifier-tied-sampled-assignment-conflict s)  $s \leq V$ 
and untied-bound:

```

$wp\text{-}event\ verify\text{-}monad$
 $(trace\text{-}fri\text{-}sampled\text{-}assignment\text{-}verifier\text{-}untied\text{-}gap\ s)\ s \leq U$
shows
 $wp\text{-}event\ verify\text{-}monad$
 $(trace\text{-}fri\text{-}sampled\text{-}assignment\text{-}conflict\ s)\ s \leq V + U$
proof –
have $wp\text{-}event\ verify\text{-}monad$
 $(trace\text{-}fri\text{-}sampled\text{-}assignment\text{-}conflict\ s)\ s \leq$
 $wp\text{-}event\ verify\text{-}monad$
 $(\lambda out.$
 $\quad trace\text{-}fri\text{-}verifier\text{-}tied\text{-}sampled\text{-}assignment\text{-}conflict\ s\ out \vee$
 $\quad trace\text{-}fri\text{-}sampled\text{-}assignment\text{-}verifier\text{-}untied\text{-}gap\ s\ out)\ s$
by $(rule\ wp\text{-}event\text{-}mono)$
 $(rule\ trace\text{-}fri\text{-}sampled\text{-}assignment\text{-}conflict\text{-}imp\text{-}verifier\text{-}tied\text{-}or\text{-}untied)$
also have $\dots \leq$
 $wp\text{-}event\ verify\text{-}monad$
 $(trace\text{-}fri\text{-}verifier\text{-}tied\text{-}sampled\text{-}assignment\text{-}conflict\ s)\ s +$
 $wp\text{-}event\ verify\text{-}monad$
 $(trace\text{-}fri\text{-}sampled\text{-}assignment\text{-}verifier\text{-}untied\text{-}gap\ s)\ s$
by $(rule\ wp\text{-}event\text{-}union\text{-}bound)$
also have $\dots \leq V + U$
by $(rule\ add\text{-}mono[OF\ verifier\text{-}tied\text{-}bound\ untied\text{-}bound])$
finally show $?thesis$.
qed

lemma *checked-staged-security-trace-fri-sampled-assignment-conflict-bound-from-verifier-tied-and-untied:*
assumes *verifier-tied-bound:*
 $wp\text{-}event\ (checked\text{-}staged\text{-}security\text{-}experiment\text{-}with\text{-}data\text{-}state\ A)$
 $(staged\text{-}security\text{-}with\text{-}data\text{-}state\text{-}verifier\text{-}event$
 $\quad trace\text{-}fri\text{-}verifier\text{-}tied\text{-}sampled\text{-}assignment\text{-}conflict)$
 $adversary\text{-}initial\text{-}state \leq V$
and *untied-bound:*
 $wp\text{-}event\ (checked\text{-}staged\text{-}security\text{-}experiment\text{-}with\text{-}data\text{-}state\ A)$
 $(staged\text{-}security\text{-}with\text{-}data\text{-}state\text{-}verifier\text{-}event$
 $\quad trace\text{-}fri\text{-}sampled\text{-}assignment\text{-}verifier\text{-}untied\text{-}gap)$
 $adversary\text{-}initial\text{-}state \leq U$
shows
 $wp\text{-}event\ (checked\text{-}staged\text{-}security\text{-}experiment\text{-}with\text{-}data\text{-}state\ A)$
 $(staged\text{-}security\text{-}with\text{-}data\text{-}state\text{-}verifier\text{-}event$
 $\quad trace\text{-}fri\text{-}sampled\text{-}assignment\text{-}conflict)$
 $adversary\text{-}initial\text{-}state \leq V + U$
proof –
let $?M = checked\text{-}staged\text{-}security\text{-}experiment\text{-}with\text{-}data\text{-}state\ A$
let $?plain =$
 $staged\text{-}security\text{-}with\text{-}data\text{-}state\text{-}verifier\text{-}event$
 $trace\text{-}fri\text{-}sampled\text{-}assignment\text{-}conflict$
let $?tied =$
 $staged\text{-}security\text{-}with\text{-}data\text{-}state\text{-}verifier\text{-}event$
 $trace\text{-}fri\text{-}verifier\text{-}tied\text{-}sampled\text{-}assignment\text{-}conflict$

```

let ?untied =
  staged-security-with-data-state-verifier-event
  trace-fri-sampled-assignment-verifier-untied-gap
have wp-event ?M ?plain adversary-initial-state ≤
  wp-event ?M (λout. ?tied out ∨ ?untied out)
  adversary-initial-state
unfolding staged-security-with-data-state-verifier-event-def
by (rule wp-event-mono)
  (auto dest:
    trace-fri-sampled-assignment-conflict-imp-verifier-tied-or-untied
    split: option.splits prod.splits)
also have ... ≤
  wp-event ?M ?tied adversary-initial-state +
  wp-event ?M ?untied adversary-initial-state
by (rule wp-event-union-bound)
also have ... ≤ V + U
by (rule add-mono[OF verifier-tied-bound untied-bound])
finally show ?thesis .
qed

definition trace-fri-verifier-tied-sampled-assignment-conflict-without-same-layer
  :: ('f, 'a) protocol-channel-scheme ⇒
    (unit list × ('f, 'a) protocol-channel-scheme) option ⇒ bool
where
  trace-fri-verifier-tied-sampled-assignment-conflict-without-same-layer
    s out ⟷
    (∃ trace-roots trace-bs trace-final dg composition-roots
      composition-bs composition-final fri-query-idxs trace-round-layers
      composition-round-layers fr trace-openings trace-table.
      accepted-fri-opening-transcript s out trace-roots trace-bs
        trace-final dg composition-roots composition-bs composition-final
          fri-query-idxs trace-round-layers composition-round-layers ∧
      accepted-with-partial-trace-openings s out fr fri-query-idxs
        trace-openings ∧
      partial-trace-table-candidate trace-table trace-openings ∧
      ¬ trace-table-low-degree trace-table ∧
      generic-fri-sampled-assignment-conflict-without-same-layer
        trace-table trace-roots trace-bs trace-final fri-query-idxs
          trace-round-layers)

definition trace-fri-verifier-tied-same-layer-auth-gap
  :: ('f, 'a) protocol-channel-scheme ⇒
    (unit list × ('f, 'a) protocol-channel-scheme) option ⇒ bool
where
  trace-fri-verifier-tied-same-layer-auth-gap s out ⟷
    (∃ trace-roots trace-bs trace-final dg composition-roots
      composition-bs composition-final fri-query-idxs trace-round-layers
      composition-round-layers fr trace-openings trace-table result
        final-state.

```

accepted-fri-opening-transcript s out trace-roots trace-bs
trace-final dg composition-roots composition-bs composition-final
fri-query-idxs trace-round-layers composition-round-layers $\wedge$
out = Some (result, final-state) $\wedge$
accepted-with-partial-trace-openings s out fr fri-query-idxs
trace-openings $\wedge$
partial-trace-table-candidate trace-table trace-openings $\wedge$
 $\neg$ *trace-table-low-degree trace-table* $\wedge$
generic-fri-sampled-same-layer-opening-conflict trace-roots trace-bs
fri-query-idxs trace-round-layers $\wedge$
 $\neg$ *generic-fri-sampled-same-layer-opening-conflict-with-authenticated-chunks*
trace-roots trace-bs fri-query-idxs trace-round-layers final-state)

lemma *trace-fri-verifier-tied-conflict-imp-authenticated-or-auth-gap:*

assumes *trace-fri-verifier-tied-sampled-assignment-conflict s out*

shows

trace-fri-verifier-tied-sampled-assignment-conflict-with-authenticated-same-layer
s out $\vee$

trace-fri-verifier-tied-same-layer-auth-gap s out

proof —

from *assms* **obtain** *trace-roots trace-bs trace-final dg composition-roots*
composition-bs composition-final fri-query-idxs trace-round-layers
composition-round-layers fr trace-openings trace-table

where *fri-openings:*

accepted-fri-opening-transcript s out trace-roots trace-bs
trace-final dg composition-roots composition-bs composition-final
fri-query-idxs trace-round-layers composition-round-layers

and *partial:*

accepted-with-partial-trace-openings s out fr fri-query-idxs
trace-openings

and *cand:* *partial-trace-table-candidate trace-table trace-openings*

and *not-low:* $\neg$ *trace-table-low-degree trace-table*

and *conflict:*

generic-fri-sampled-assignment-conflict trace-table trace-roots
trace-bs trace-final fri-query-idxs trace-round-layers

unfolding *trace-fri-verifier-tied-sampled-assignment-conflict-def*

by *blast*

from *fri-openings* **obtain** *result final-state* **where** *out-eq:*

out = Some (result, final-state)

unfolding *accepted-fri-opening-transcript-def* **by** *blast*

have *split:*

generic-fri-sampled-assignment-conflict-with-authenticated-same-layer
trace-table trace-roots trace-bs trace-final fri-query-idxs
trace-round-layers final-state $\vee$

(*generic-fri-sampled-same-layer-opening-conflict trace-roots trace-bs*
fri-query-idxs trace-round-layers $\wedge$

$\neg$ *generic-fri-sampled-same-layer-opening-conflict-with-authenticated-chunks*
trace-roots trace-bs fri-query-idxs trace-round-layers final-state)

by (*rule*

```

    generic-fri-sampled-assignment-conflict-imp-authenticated-or-same-layer-auth-gap
    [OF conflict])
  then show ?thesis
proof
  assume auth:
    generic-fri-sampled-assignment-conflict-with-authenticated-same-layer
    trace-table trace-roots trace-bs trace-final fri-query-idxs
    trace-round-layers final-state
  have trace-fri-verifier-tied-sampled-assignment-conflict-with-authenticated-same-layer
    s out
  unfolding
    trace-fri-verifier-tied-sampled-assignment-conflict-with-authenticated-same-layer-def
  by (intro exI conjI)
    (rule fri-openings, rule out-eq, rule partial, rule cand,
     rule not-low, rule auth)
  then show ?thesis by simp
next
  assume gap:
    generic-fri-sampled-same-layer-opening-conflict trace-roots trace-bs
    fri-query-idxs trace-round-layers  $\wedge$ 
     $\neg$  generic-fri-sampled-same-layer-opening-conflict-with-authenticated-chunks
    trace-roots trace-bs fri-query-idxs trace-round-layers final-state
  have trace-fri-verifier-tied-same-layer-auth-gap s out
  unfolding trace-fri-verifier-tied-same-layer-auth-gap-def
  by (intro exI conjI)
    (rule fri-openings, rule out-eq, rule partial, rule cand,
     rule not-low, use gap in simp-all)
  then show ?thesis by simp
qed
qed

lemma wp-trace-fri-verifier-tied-conflict-bound-from-authenticated-and-auth-gap:
  fixes A G :: prob
  assumes auth-bound:
    wp-event verify-monad
    (trace-fri-verifier-tied-sampled-assignment-conflict-with-authenticated-same-layer s)
     $s \leq A$ 
  and gap-bound:
    wp-event verify-monad
    (trace-fri-verifier-tied-same-layer-auth-gap s)  $s \leq G$ 
  shows
    wp-event verify-monad
    (trace-fri-verifier-tied-sampled-assignment-conflict s)  $s \leq A + G$ 
proof -
  have wp-event verify-monad
    (trace-fri-verifier-tied-sampled-assignment-conflict s)  $s \leq$ 
  wp-event verify-monad
    ( $\lambda$ out.
    trace-fri-verifier-tied-sampled-assignment-conflict-with-authenticated-same-layer

```

$s \text{ out} \vee$
 $\text{trace-fri-verifier-tied-same-layer-auth-gap } s \text{ out}) \ s$
by (rule *wp-event-mono*)
(rule *trace-fri-verifier-tied-conflict-imp-authenticated-or-auth-gap*)
also have $\dots \leq$
 $\text{wp-event verify-monad}$
 $(\text{trace-fri-verifier-tied-sampled-assignment-conflict-with-authenticated-same-layer } s)$
 $s +$
 $\text{wp-event verify-monad } (\text{trace-fri-verifier-tied-same-layer-auth-gap } s) \ s$
by (rule *wp-event-union-bound*)
also have $\dots \leq A + G$
by (rule *add-mono[OF auth-bound gap-bound]*)
finally show *?thesis* .
qed

lemma *trace-fri-verifier-tied-authenticated-conflict-imp-without-same-or-same-layer:*
assumes
 $\text{trace-fri-verifier-tied-sampled-assignment-conflict-with-authenticated-same-layer}$
 $s \text{ out}$
shows
 $\text{trace-fri-verifier-tied-sampled-assignment-conflict-without-same-layer}$
 $s \text{ out} \vee$
 $\text{trace-fri-verifier-tied-same-layer-conflict-with-authenticated-chunks}$
 $s \text{ out}$

proof —
from *assms* **obtain** *trace-roots trace-bs trace-final dg composition-roots*
composition-bs composition-final fri-query-idxs trace-round-layers
composition-round-layers fr trace-openings trace-table result
final-state
where *fri-openings:*
 $\text{accepted-fri-opening-transcript } s \text{ out } \text{trace-roots } \text{trace-bs}$
 $\text{trace-final } dg \text{ composition-roots } \text{composition-bs } \text{composition-final}$
 $\text{fri-query-idxs } \text{trace-round-layers } \text{composition-round-layers}$
and *out-eq:* $\text{out} = \text{Some } (\text{result}, \text{final-state})$
and *partial:*
 $\text{accepted-with-partial-trace-openings } s \text{ out } \text{fr } \text{fri-query-idxs}$
 trace-openings
and *cand:* $\text{partial-trace-table-candidate } \text{trace-table } \text{trace-openings}$
and *not-low:* $\neg \text{trace-table-low-degree } \text{trace-table}$
and *conflict:*
 $\text{generic-fri-sampled-assignment-conflict-with-authenticated-same-layer}$
 $\text{trace-table } \text{trace-roots } \text{trace-bs } \text{trace-final } \text{fri-query-idxs}$
 $\text{trace-round-layers } \text{final-state}$
unfolding
 $\text{trace-fri-verifier-tied-sampled-assignment-conflict-with-authenticated-same-layer-def}$
by *blast*
have *split:*
 $\text{generic-fri-sampled-assignment-conflict-without-same-layer}$
 $\text{trace-table } \text{trace-roots } \text{trace-bs } \text{trace-final } \text{fri-query-idxs}$


```

    trace-round-layers  $\vee$ 
    generic-fri-sampled-same-layer-opening-conflict-with-authenticated-chunks
    trace-roots trace-bs fri-query-idxs trace-round-layers final-state
  by (rule
    generic-fri-sampled-assignment-conflict-with-authenticated-same-layer-split
    [OF conflict])
  then show ?thesis
proof
  assume without:
    generic-fri-sampled-assignment-conflict-without-same-layer
    trace-table trace-roots trace-bs trace-final fri-query-idxs
    trace-round-layers
  have trace-fri-verifier-tied-sampled-assignment-conflict-without-same-layer
    s out
  unfolding
    trace-fri-verifier-tied-sampled-assignment-conflict-without-same-layer-def
  by (intro exI conjI)
    (rule fri-openings, rule partial, rule cand, rule not-low,
     rule without)
  then show ?thesis by simp
next
  assume same:
    generic-fri-sampled-same-layer-opening-conflict-with-authenticated-chunks
    trace-roots trace-bs fri-query-idxs trace-round-layers final-state
  have trace-fri-verifier-tied-same-layer-conflict-with-authenticated-chunks
    s out
  unfolding
    trace-fri-verifier-tied-same-layer-conflict-with-authenticated-chunks-def
  by (intro exI conjI)
    (rule fri-openings, rule out-eq, rule partial, rule cand,
     rule not-low, rule same)
  then show ?thesis by simp
qed
qed

lemma wp-trace-fri-verifier-tied-authenticated-conflict-bound-from-without-same-and-merkle:
  assumes without-bound:
    wp-event verify-monad
    (trace-fri-verifier-tied-sampled-assignment-conflict-without-same-layer s)
     $s \leq W$ 
  and merkle-bound:
    wp-event verify-monad (partial-merkle-inconsistency-bad s)  $s \leq M$ 
  shows
    wp-event verify-monad
    (trace-fri-verifier-tied-sampled-assignment-conflict-with-authenticated-same-layer s)
     $s \leq W + M$ 
proof -
  have wp-event verify-monad
    (trace-fri-verifier-tied-sampled-assignment-conflict-with-authenticated-same-layer s)

```

```

s ≤
wp-event verify-monad
(λout.
  trace-fri-verifier-tied-sampled-assignment-conflict-without-same-layer
    s out ∨
  trace-fri-verifier-tied-same-layer-conflict-with-authenticated-chunks
    s out) s
by (rule wp-event-mono)
(rule
  trace-fri-verifier-tied-authenticated-conflict-imp-without-same-or-same-layer)
also have ... ≤
wp-event verify-monad
  (trace-fri-verifier-tied-sampled-assignment-conflict-without-same-layer s)
s +
wp-event verify-monad
  (trace-fri-verifier-tied-same-layer-conflict-with-authenticated-chunks s)
s
by (rule wp-event-union-bound)
also have ... ≤ W + M
by (rule add-mono[OF without-bound
  wp-trace-fri-verifier-tied-same-layer-conflict-with-authenticated-chunks-bound-from-partial-merkle
  [OF merkle-bound]])
finally show ?thesis .
qed

```

lemma *checked-staged-security-trace-fri-verifier-tied-conflict-bound-from-authenticated-and-auth-gap:*

assumes *auth-bound:*

```

wp-event (checked-staged-security-experiment-with-data-state A)
(staged-security-with-data-state-verifier-event
  trace-fri-verifier-tied-sampled-assignment-conflict-with-authenticated-same-layer)
adversary-initial-state ≤ Auth

```

and *gap-bound:*

```

wp-event (checked-staged-security-experiment-with-data-state A)
(staged-security-with-data-state-verifier-event
  trace-fri-verifier-tied-same-layer-auth-gap)
adversary-initial-state ≤ Gap

```

shows

```

wp-event (checked-staged-security-experiment-with-data-state A)
(staged-security-with-data-state-verifier-event
  trace-fri-verifier-tied-sampled-assignment-conflict)
adversary-initial-state ≤ Auth + Gap

```

proof –

let *?M* = *checked-staged-security-experiment-with-data-state A*

let *?conflict* =

```

staged-security-with-data-state-verifier-event
  trace-fri-verifier-tied-sampled-assignment-conflict

```

let *?auth* =

```

staged-security-with-data-state-verifier-event
  trace-fri-verifier-tied-sampled-assignment-conflict-with-authenticated-same-layer

```

```

let ?gap =
  staged-security-with-data-state-verifier-event
  trace-fri-verifier-tied-same-layer-auth-gap
have wp-event ?M ?conflict adversary-initial-state ≤
  wp-event ?M (λout. ?auth out ∨ ?gap out) adversary-initial-state
unfolding staged-security-with-data-state-verifier-event-def
by (rule wp-event-mono)
  (auto dest:
    trace-fri-verifier-tied-conflict-imp-authenticated-or-auth-gap
    split: option.splits prod.splits)
also have ... ≤
  wp-event ?M ?auth adversary-initial-state +
  wp-event ?M ?gap adversary-initial-state
by (rule wp-event-union-bound)
also have ... ≤ Auth + Gap
by (rule add-mono[OF auth-bound gap-bound])
finally show ?thesis .
qed

lemma checked-staged-security-trace-fri-verifier-tied-authenticated-conflict-bound-from-without-same-and-merkle:
assumes without-bound:
  wp-event (checked-staged-security-experiment-with-data-state A)
  (staged-security-with-data-state-verifier-event
    trace-fri-verifier-tied-sampled-assignment-conflict-without-same-layer)
  adversary-initial-state ≤ W
and merkle-bound:
  wp-event (checked-staged-security-experiment-with-data-state A)
  (staged-security-with-data-state-verifier-event
    partial-merkle-inconsistency-bad)
  adversary-initial-state ≤ M
shows
  wp-event (checked-staged-security-experiment-with-data-state A)
  (staged-security-with-data-state-verifier-event
    trace-fri-verifier-tied-sampled-assignment-conflict-with-authenticated-same-layer)
  adversary-initial-state ≤ W + M
proof –
let ?M = checked-staged-security-experiment-with-data-state A
let ?auth =
  staged-security-with-data-state-verifier-event
  trace-fri-verifier-tied-sampled-assignment-conflict-with-authenticated-same-layer
let ?without =
  staged-security-with-data-state-verifier-event
  trace-fri-verifier-tied-sampled-assignment-conflict-without-same-layer
let ?merkle =
  staged-security-with-data-state-verifier-event
  partial-merkle-inconsistency-bad
have wp-event ?M ?auth adversary-initial-state ≤
  wp-event ?M (λout. ?without out ∨ ?merkle out)
  adversary-initial-state

```

```

proof (rule wp-event-mono)
  fix out
  assume auth: ?auth out
  show ?without out  $\vee$  ?merkle out
  proof (cases out)
    case None
    then show ?thesis
      using auth
      unfolding staged-security-with-data-state-verifier-event-def
        trace-fri-verifier-tied-sampled-assignment-conflict-with-authenticated-same-layer-def
      by simp
  next
  case (Some packed)
  then obtain data attacker-state result final-state where out-eq:
    out = Some (((data, attacker-state), result), final-state)
    by (cases packed, auto split: prod.splits)
  let ?s =
    verifier-state-from-adversary attacker-state
    (staged-proof-transcript data)
  have local-auth:
    trace-fri-verifier-tied-sampled-assignment-conflict-with-authenticated-same-layer
    ?s (Some (result, final-state))
    using auth
    unfolding out-eq staged-security-with-data-state-verifier-event-def
    by simp
  have split:
    trace-fri-verifier-tied-sampled-assignment-conflict-without-same-layer
    ?s (Some (result, final-state))  $\vee$ 
    trace-fri-verifier-tied-same-layer-conflict-with-authenticated-chunks
    ?s (Some (result, final-state))
    by (rule
      trace-fri-verifier-tied-authenticated-conflict-imp-without-same-or-same-layer
      [OF local-auth])
  then show ?thesis
  proof
    assume without:
      trace-fri-verifier-tied-sampled-assignment-conflict-without-same-layer
      ?s (Some (result, final-state))
    then show ?thesis
      unfolding out-eq staged-security-with-data-state-verifier-event-def
      by simp
  next
  assume same:
    trace-fri-verifier-tied-same-layer-conflict-with-authenticated-chunks
    ?s (Some (result, final-state))
  have partial-merkle-inconsistency-bad ?s
    (Some (result, final-state))
    by (rule
      trace-fri-verifier-tied-same-layer-conflict-with-authenticated-chunks-imp-partial-merkle

```

```

      [OF same])
    then show ?thesis
      unfolding out-eq staged-security-with-data-state-verifier-event-def
      by simp
    qed
  qed
qed
also have ... ≤
  wp-event ?M ?without adversary-initial-state +
  wp-event ?M ?merkle adversary-initial-state
  by (rule wp-event-union-bound)
also have ... ≤ W + M
  by (rule add-mono[OF without-bound merkle-bound])
finally show ?thesis .
qed

```

lemma *checked-staged-security-trace-fri-verifier-tied-conflict-bound-from-without-same-merkle-and-auth-gap:*

fixes $W\ M\ G :: \text{prob}$

assumes *without-bound:*

```

  wp-event (checked-staged-security-experiment-with-data-state A)
    (staged-security-with-data-state-verifier-event
      trace-fri-verifier-tied-sampled-assignment-conflict-without-same-layer)
  adversary-initial-state ≤ W

```

and *merkle-bound:*

```

  wp-event (checked-staged-security-experiment-with-data-state A)
    (staged-security-with-data-state-verifier-event
      partial-merkle-inconsistency-bad)
  adversary-initial-state ≤ M

```

and *gap-bound:*

```

  wp-event (checked-staged-security-experiment-with-data-state A)
    (staged-security-with-data-state-verifier-event
      trace-fri-verifier-tied-same-layer-auth-gap)
  adversary-initial-state ≤ G

```

shows

```

  wp-event (checked-staged-security-experiment-with-data-state A)
    (staged-security-with-data-state-verifier-event
      trace-fri-verifier-tied-sampled-assignment-conflict)
  adversary-initial-state ≤ (W + M) + G

```

proof (rule

checked-staged-security-trace-fri-verifier-tied-conflict-bound-from-authenticated-and-auth-gap)

show *wp-event (checked-staged-security-experiment-with-data-state A)*

```

  (staged-security-with-data-state-verifier-event
    trace-fri-verifier-tied-sampled-assignment-conflict-with-authenticated-same-layer)
  adversary-initial-state ≤ W + M

```

by (rule

checked-staged-security-trace-fri-verifier-tied-authenticated-conflict-bound-from-without-same-and-merkle
[OF without-bound merkle-bound])

show *wp-event (checked-staged-security-experiment-with-data-state A)*

```

  (staged-security-with-data-state-verifier-event

```

$\text{trace-fri-verifier-tied-same-layer-auth-gap})$
 $\text{adversary-initial-state} \leq G$
by (rule gap-bound)
qed

definition *trace-fri-empty-sampled-assignment-structural-error-for*
where

$\text{trace-fri-empty-sampled-assignment-structural-error-for budgets } A =$
 $((\text{reachable-verifier-event-bound-for } A$
 $\quad \text{trace-fri-verifier-tied-sampled-assignment-conflict-without-same-layer} +$
 $\quad \text{trace-fri-controlled-merkle-error-for budgets}) +$
 $\text{reachable-verifier-event-bound-for } A$
 $\quad \text{trace-fri-verifier-tied-same-layer-auth-gap}) +$
 $\text{reachable-verifier-event-bound-for } A$
 $\quad \text{trace-fri-sampled-assignment-verifier-untied-gap}$

definition *trace-empty-header-openings-zero-error-for*
where

$\text{trace-empty-header-openings-zero-error-for budgets } A \text{ empty-bad} =$
 $(\text{trace-fri-zero-round-agreement-set-target-error-for budgets} +$
 $\text{trace-fri-controlled-merkle-error-for budgets} +$
 $(\text{trace-header-one-step-error-for budgets } A \text{ empty-bad} +$
 $\text{reachable-verifier-event-bound-for } A$
 $\quad \text{trace-fri-header-tied-low-candidate-untied-openings-gap}) +$
 $\text{trace-header-one-step-error-for budgets } A \text{ empty-bad})$

definition *trace-fri-header-tied-low-candidate-nonconsumed-openings-gap*

$:: ('f, 'a) \text{ protocol-channel-scheme} \Rightarrow$
 $(\text{unit list} \times ('f, 'a) \text{ protocol-channel-scheme}) \text{ option} \Rightarrow \text{bool}$

where

$\text{trace-fri-header-tied-low-candidate-nonconsumed-openings-gap } s \text{ out}$
 $\iff$
 $(\exists \text{ trace-roots trace-bs trace-final fri-dg composition-roots}$
 $\quad \text{composition-bs composition-final (fri-query-idxs} :: \text{nat list)}$
 $\quad \text{trace-round-layers composition-round-layers (fr} :: 'f) \text{ as}$
 $\quad \text{trace-openings trace-table (fr0} :: 'f) \text{ (query-idxs0} :: \text{nat list)}$
 $\quad \text{trace-openings0 trace-table0.}$
 $\text{trace-fri-reachable-header-tie-gap } s \text{ out} \wedge$
 $\text{accepted-with-partial-trace-openings-verifier-consumed } s \text{ out}$
 $\quad \text{trace-roots trace-bs trace-final fri-dg composition-roots}$
 $\quad \text{composition-bs composition-final fri-query-idxs trace-round-layers}$
 $\quad \text{composition-round-layers fr as trace-openings} \wedge$
 $\text{partial-trace-table-candidate trace-table trace-openings} \wedge$
 $\text{trace-table-low-degree trace-table} \wedge$
 $\text{trace-fri-partial-candidate-evidence } s \text{ out fr0 query-idxs0}$
 $\quad \text{trace-openings0 trace-table0} \wedge$
 $(\text{fr0} \neq \text{fr} \vee \text{query-idxs0} \neq \text{fri-query-idxs}) \wedge$
 $\text{trace-openings0} \neq \text{trace-openings})$

definition *trace-fri-header-tied-low-candidate-nonconsumed-root-gap*

$:: ('f, 'a) \text{ protocol-channel-scheme} \Rightarrow$
 $(\text{unit list} \times ('f, 'a) \text{ protocol-channel-scheme}) \text{ option} \Rightarrow \text{bool}$

where

trace-fri-header-tied-low-candidate-nonconsumed-root-gap $s \text{ out}$
 $\longleftrightarrow$
 $(\exists \text{ trace-roots trace-bs trace-final fri-dg composition-roots}$
 $\text{ composition-bs composition-final (fri-query-idxs} :: \text{ nat list)}$
 $\text{ trace-round-layers composition-round-layers (fr} :: 'f) \text{ as}$
 $\text{ trace-openings trace-table (fr0} :: 'f) (\text{query-idxs0} :: \text{ nat list)}$
 $\text{ trace-openings0 trace-table0.}$
 $\text{ trace-fri-reachable-header-tie-gap } s \text{ out} \wedge$
 $\text{ accepted-with-partial-trace-openings-verifier-consumed } s \text{ out}$
 $\text{ trace-roots trace-bs trace-final fri-dg composition-roots}$
 $\text{ composition-bs composition-final fri-query-idxs trace-round-layers}$
 $\text{ composition-round-layers fr as trace-openings} \wedge$
 $\text{ partial-trace-table-candidate trace-table trace-openings} \wedge$
 $\text{ trace-table-low-degree trace-table} \wedge$
 $\text{ trace-fri-partial-candidate-evidence } s \text{ out fr0 query-idxs0}$
 $\text{ trace-openings0 trace-table0} \wedge$
 $\text{ fr0} \neq \text{ fr} \wedge$
 $\text{ trace-openings0} \neq \text{ trace-openings})$

definition *trace-fri-header-tied-low-candidate-nonconsumed-query-gap*

$:: ('f, 'a) \text{ protocol-channel-scheme} \Rightarrow$
 $(\text{unit list} \times ('f, 'a) \text{ protocol-channel-scheme}) \text{ option} \Rightarrow \text{bool}$

where

trace-fri-header-tied-low-candidate-nonconsumed-query-gap $s \text{ out}$
 $\longleftrightarrow$
 $(\exists \text{ trace-roots trace-bs trace-final fri-dg composition-roots}$
 $\text{ composition-bs composition-final (fri-query-idxs} :: \text{ nat list)}$
 $\text{ trace-round-layers composition-round-layers (fr} :: 'f) \text{ as}$
 $\text{ trace-openings trace-table (fr0} :: 'f) (\text{query-idxs0} :: \text{ nat list)}$
 $\text{ trace-openings0 trace-table0.}$
 $\text{ trace-fri-reachable-header-tie-gap } s \text{ out} \wedge$
 $\text{ accepted-with-partial-trace-openings-verifier-consumed } s \text{ out}$
 $\text{ trace-roots trace-bs trace-final fri-dg composition-roots}$
 $\text{ composition-bs composition-final fri-query-idxs trace-round-layers}$
 $\text{ composition-round-layers fr as trace-openings} \wedge$
 $\text{ partial-trace-table-candidate trace-table trace-openings} \wedge$
 $\text{ trace-table-low-degree trace-table} \wedge$
 $\text{ trace-fri-partial-candidate-evidence } s \text{ out fr0 query-idxs0}$
 $\text{ trace-openings0 trace-table0} \wedge$
 $\text{ query-idxs0} \neq \text{ fri-query-idxs} \wedge$
 $\text{ trace-openings0} \neq \text{ trace-openings})$

definition *trace-fri-header-tied-low-candidate-nonconsumed-irrelevant-root-gap*

$:: ('f, 'a) \text{ protocol-channel-scheme} \Rightarrow$
 $(\text{unit list} \times ('f, 'a) \text{ protocol-channel-scheme}) \text{ option} \Rightarrow \text{bool}$

where

trace-fri-header-tied-low-candidate-nonconsumed-irrelevant-root-gap s out
 $\longleftrightarrow$
 $(\exists \text{ trace-roots trace-bs trace-final fri-dg composition-roots}$
 $\text{ composition-bs composition-final (fri-query-idxs :: nat list)}$
 $\text{ trace-round-layers composition-round-layers (fr :: 'f) as}$
 $\text{ trace-openings trace-table (fr0 :: 'f) (query-idxs0 :: nat list)}$
 $\text{ trace-openings0 trace-table0.}$
 $\text{ trace-fri-reachable-header-tie-gap s out} \wedge$
 $\text{ accepted-with-partial-trace-openings-verifier-consumed s out}$
 $\text{ trace-roots trace-bs trace-final fri-dg composition-roots}$
 $\text{ composition-bs composition-final fri-query-idxs trace-round-layers}$
 $\text{ composition-round-layers fr as trace-openings} \wedge$
 $\text{ partial-trace-table-candidate trace-table trace-openings} \wedge$
 $\text{ trace-table-low-degree trace-table} \wedge$
 $\text{ trace-fri-partial-candidate-evidence s out fr0 query-idxs0}$
 $\text{ trace-openings0 trace-table0} \wedge$
 $\text{ fr0} \neq \text{ fr} \wedge$
 $\text{ query-idxs0} = \text{ fri-query-idxs} \wedge$
 $\text{ trace-openings0} \neq \text{ trace-openings})$

definition *trace-empty-header-nonnormalized-zero-error-for*

where

trace-empty-header-nonnormalized-zero-error-for budgets A empty-bad =
 $(\text{trace-fri-zero-round-agreement-set-target-error-for budgets} +$
 $\text{trace-fri-controlled-merkle-error-for budgets} +$
 $(\text{trace-fri-controlled-merkle-error-for budgets} +$
 $((\text{reachable-verifier-event-bound-for A}$
 $\text{ trace-fri-header-tied-low-candidate-nonconsumed-query-gap} +$
 $\text{reachable-verifier-event-bound-for A}$
 $\text{ trace-fri-header-tied-low-candidate-nonconsumed-irrelevant-root-gap}) +$
 $\text{reachable-verifier-event-bound-for A}$
 $\text{ trace-fri-header-tied-low-candidate-nonconsumed-query-gap})) +$
 $\text{trace-header-one-step-error-for budgets A empty-bad})$

definition *trace-fri-zero-round-verifier-consumed-low-candidate-gap*

$:: ('f, 'a) \text{ protocol-channel-scheme} \Rightarrow$
 $(\text{unit list} \times ('f, 'a) \text{ protocol-channel-scheme}) \text{ option} \Rightarrow \text{bool}$

where

trace-fri-zero-round-verifier-consumed-low-candidate-gap s out
 $\longleftrightarrow$
 $(\exists \text{ trace-roots trace-bs trace-final fri-dg composition-roots}$
 $\text{ composition-bs composition-final fri-query-idxs trace-round-layers}$
 $\text{ composition-round-layers fr as trace-openings trace-table.}$
 $\text{ trace-fri-zero-round-final-obstruction s out} \wedge$
 $\text{ trace-fri-reachable-header-tie-gap s out} \wedge$
 $\text{ accepted-with-partial-trace-openings-verifier-consumed s out}$
 $\text{ trace-roots trace-bs trace-final fri-dg composition-roots}$
 $\text{ composition-bs composition-final fri-query-idxs trace-round-layers}$

*composition-round-layers fr as trace-openings $\wedge$
length trace-bs = 0 $\wedge$
partial-trace-table-candidate trace-table trace-openings $\wedge$
trace-table-low-degree trace-table)*

lemma *trace-fri-header-tied-low-candidate-nonconsumed-root-gapE:*
assumes *trace-fri-header-tied-low-candidate-nonconsumed-root-gap s out*
obtains *trace-roots trace-bs trace-final fri-dg composition-roots
composition-bs composition-final fri-query-idxs trace-round-layers
composition-round-layers fr as trace-openings trace-table
fr0 query-idxs0 trace-openings0 trace-table0 where
trace-fri-reachable-header-tie-gap s out
accepted-with-partial-trace-openings-verifier-consumed s out
trace-roots trace-bs trace-final fri-dg composition-roots
composition-bs composition-final fri-query-idxs trace-round-layers
composition-round-layers fr as trace-openings
partial-trace-table-candidate trace-table trace-openings
trace-table-low-degree trace-table
trace-fri-partial-candidate-evidence s out fr0 query-idxs0
trace-openings0 trace-table0
fr0 $\neq$ fr
trace-openings0 $\neq$ trace-openings*
using *assms*
unfolding *trace-fri-header-tied-low-candidate-nonconsumed-root-gap-def*
by *blast*

lemma *trace-fri-header-tied-low-candidate-nonconsumed-query-gapE:*
assumes *trace-fri-header-tied-low-candidate-nonconsumed-query-gap s out*
obtains *trace-roots trace-bs trace-final fri-dg composition-roots
composition-bs composition-final fri-query-idxs trace-round-layers
composition-round-layers fr as trace-openings trace-table
fr0 query-idxs0 trace-openings0 trace-table0 where
trace-fri-reachable-header-tie-gap s out
accepted-with-partial-trace-openings-verifier-consumed s out
trace-roots trace-bs trace-final fri-dg composition-roots
composition-bs composition-final fri-query-idxs trace-round-layers
composition-round-layers fr as trace-openings
partial-trace-table-candidate trace-table trace-openings
trace-table-low-degree trace-table
trace-fri-partial-candidate-evidence s out fr0 query-idxs0
trace-openings0 trace-table0
query-idxs0 $\neq$ fri-query-idxs
trace-openings0 $\neq$ trace-openings*
using *assms*
unfolding *trace-fri-header-tied-low-candidate-nonconsumed-query-gap-def*
by *blast*

lemma *trace-fri-header-tied-low-candidate-nonconsumed-irrelevant-root-gapE:*
assumes

trace-fri-header-tied-low-candidate-nonconsumed-irrelevant-root-gap s out
obtains *trace-roots trace-bs trace-final fri-dg composition-roots*
composition-bs composition-final fri-query-idxs trace-round-layers
composition-round-layers fr as trace-openings trace-table
fr0 query-idxs0 trace-openings0 trace-table0 **where**
trace-fri-reachable-header-tie-gap s out
accepted-with-partial-trace-openings-verifier-consumed s out
trace-roots trace-bs trace-final fri-dg composition-roots
composition-bs composition-final fri-query-idxs trace-round-layers
composition-round-layers fr as trace-openings
partial-trace-table-candidate trace-table trace-openings
trace-table-low-degree trace-table
trace-fri-partial-candidate-evidence s out fr0 query-idxs0
trace-openings0 trace-table0
fr0 ≠ fr
query-idxs0 = fri-query-idxs
trace-openings0 ≠ trace-openings
using *assms*
unfolding
trace-fri-header-tied-low-candidate-nonconsumed-irrelevant-root-gap-def
by *blast*

lemma *trace-fri-zero-round-verifier-consumed-low-candidate-gapE:*

assumes

trace-fri-zero-round-verifier-consumed-low-candidate-gap s out

obtains *trace-roots trace-bs trace-final fri-dg composition-roots*
composition-bs composition-final fri-query-idxs trace-round-layers
composition-round-layers fr as trace-openings trace-table **where**

trace-fri-zero-round-final-obstruction s out

trace-fri-reachable-header-tie-gap s out

accepted-with-partial-trace-openings-verifier-consumed s out

trace-roots trace-bs trace-final fri-dg composition-roots

composition-bs composition-final fri-query-idxs trace-round-layers

composition-round-layers fr as trace-openings

length trace-bs = 0

partial-trace-table-candidate trace-table trace-openings

trace-table-low-degree trace-table

using *assms*

unfolding *trace-fri-zero-round-verifier-consumed-low-candidate-gap-def*

by *blast*

lemma *trace-fri-zero-round-verifier-consumed-low-candidate-gap-imp-candidate-available:*

assumes

trace-fri-zero-round-verifier-consumed-low-candidate-gap s out

shows *trace-fri-header-tied-candidate-available-gap s out*

proof (rule *trace-fri-zero-round-verifier-consumed-low-candidate-gapE*
 $[OF\ assms]$)

fix *trace-roots trace-bs trace-final fri-dg composition-roots*

composition-bs composition-final fri-query-idxs trace-round-layers

composition-round-layers fr as trace-openings trace-table
assume *gap: trace-fri-reachable-header-tie-gap s out*
and *consumed:*
accepted-with-partial-trace-openings-verifier-consumed s out
trace-roots trace-bs trace-final fri-dg composition-roots
composition-bs composition-final fri-query-idxs trace-round-layers
composition-round-layers fr as trace-openings
and *candidate:*
partial-trace-table-candidate trace-table trace-openings
have *fri-openings:*
accepted-fri-opening-transcript s out trace-roots trace-bs
trace-final fri-dg composition-roots composition-bs composition-final
fri-query-idxs trace-round-layers composition-round-layers
by (rule *accepted-with-partial-trace-openings-verifier-consumedD(1)*
[*OF consumed*])
from *accepted-with-partial-trace-openings-verifier-consumedD(2)*
[*OF consumed*]
obtain *rest where header:*
verifier-header-transcript s fr trace-roots trace-final as fri-dg
composition-roots composition-final rest
by *blast*
have *partial:*
accepted-with-partial-trace-openings s out fr fri-query-idxs
trace-openings
by (rule *accepted-with-partial-trace-openings-verifier-consumedD(3)*
[*OF consumed*])
show *trace-fri-header-tied-candidate-available-gap s out*
unfolding *trace-fri-header-tied-candidate-available-gap-def*
by (*intro exI conjI*)
(*rule gap, rule fri-openings, rule header, rule partial,*
rule candidate)
qed

lemma *trace-fri-zero-round-verifier-consumed-low-candidate-gap-imp-untied-openings:*
assumes
trace-fri-zero-round-verifier-consumed-low-candidate-gap s out
shows *trace-fri-header-tied-low-candidate-untied-openings-gap s out*
proof (rule *trace-fri-zero-round-verifier-consumed-low-candidate-gapE*
[*OF assms*])
fix *trace-roots trace-bs trace-final fri-dg composition-roots*
composition-bs composition-final fri-query-idxs trace-round-layers
composition-round-layers fr as trace-openings trace-table
assume *gap: trace-fri-reachable-header-tie-gap s out*
and *consumed:*
accepted-with-partial-trace-openings-verifier-consumed s out
trace-roots trace-bs trace-final fri-dg composition-roots
composition-bs composition-final fri-query-idxs trace-round-layers
composition-round-layers fr as trace-openings
and *candidate:*

```

    partial-trace-table-candidate trace-table trace-openings
  and low: trace-table-low-degree trace-table
have fri-openings:
  accepted-fri-opening-transcript s out trace-roots trace-bs trace-final
  fri-dg composition-roots composition-bs composition-final
  fri-query-idxs trace-round-layers composition-round-layers
  by (rule accepted-with-partial-trace-openings-verifier-consumedD(1)
      [OF consumed])
from accepted-with-partial-trace-openings-verifier-consumedD(2)
  [OF consumed]
obtain rest where header:
  verifier-header-transcript s fr trace-roots trace-final as fri-dg
  composition-roots composition-final rest
  by blast
have partial:
  accepted-with-partial-trace-openings s out fr fri-query-idxs
  trace-openings
  by (rule accepted-with-partial-trace-openings-verifier-consumedD(3)
      [OF consumed])
have low-gap: trace-fri-header-tied-low-candidate-gap s out
  unfolding trace-fri-header-tied-low-candidate-gap-def
  by (intro exI conjI)
    (rule gap, rule fri-openings, rule header, rule partial,
     rule candidate, rule low)
have untied:
  trace-fri-header-tied-low-candidate-untied-reachable-witness-gap s out
  by (rule trace-fri-header-tied-low-candidate-gap-imp-untied-reachable-witness
      [OF low-gap])
show trace-fri-header-tied-low-candidate-untied-openings-gap s out
  by (rule trace-fri-header-tied-low-candidate-untied-gap-imp-openings
      [OF untied])
qed

lemma accepted-with-partial-trace-openings-query-index-list-space:
  assumes partial:
    accepted-with-partial-trace-openings s out fr query-idxs trace-openings
  shows query-idxs ∈ fri-query-index-list-space
proof -
  have len: length query-idxs = rounds
  by (rule accepted-with-partial-trace-openings-shapes(1)[OF partial])
  have subset: set query-idxs ⊆ query-sample-space
  proof
    fix idx
    assume idx-in: idx ∈ set query-idxs
    then obtain i where i-bound: i < length query-idxs
      and idx-eq: query-idxs ! i = idx
      unfolding in-set-conv-nth by blast
    have i-round: i < rounds
    using i-bound len by simp

```

obtain *result final-state* **where** *out-eq: out = Some (result, final-state)*
and *auth:*
 $\bigwedge j. j < \text{rounds} \implies$
 $\text{partial-authenticated-table } fr \text{ (scale * clength)}$
 $(\text{trace-openings ! } j) \text{ final-state}$
using *partial unfolding accepted-with-partial-trace-openings-def* **by** *blast*
have *indices:*
 $\text{map opening-index (trace-openings ! } i) = \text{powers-scaled idx}$
using *accepted-with-partial-trace-openings-shapes(3)[OF partial i-round]*
 idx-eq **by** *simp*
show $\text{idx} \in \text{query-sample-space}$
by (rule *partial-authenticated-trace-openings-query-index-in-sample-space*
 $[\text{OF indices auth}[\text{OF } i\text{-round}]]$)
qed
show *?thesis*
unfolding *fri-query-index-list-space-def*
using *len subset* **by** *simp*
qed

lemma *trace-fri-header-tied-low-candidate-nonconsumed-root-gap-shapes:*
assumes *trace-fri-header-tied-low-candidate-nonconsumed-root-gap s out*
obtains *fri-query-idxs query-idxs0* **where**
 $\text{fri-query-idxs} \in \text{fri-query-index-list-space}$
 $\text{query-idxs0} \in \text{fri-query-index-list-space}$
proof —
from *assms* **obtain** *trace-roots trace-bs trace-final fri-dg composition-roots*
 $\text{composition-bs composition-final fri-query-idxs trace-round-layers}$
 $\text{composition-round-layers fr as trace-openings trace-table}$
 $\text{fr0 query-idxs0 trace-openings0 trace-table0}$ **where**
consumed:
 $\text{accepted-with-partial-trace-openings-verifier-consumed } s \text{ out}$
 $\text{trace-roots trace-bs trace-final fri-dg composition-roots}$
 $\text{composition-bs composition-final fri-query-idxs trace-round-layers}$
 $\text{composition-round-layers fr as trace-openings}$
and *evidence0:*
 $\text{trace-fri-partial-candidate-evidence } s \text{ out fr0 query-idxs0}$
 $\text{trace-openings0 trace-table0}$
and *mismatch: fr0 ≠ fr*
by (rule *trace-fri-header-tied-low-candidate-nonconsumed-root-gapE*)
have *fri-openings:*
 $\text{accepted-fri-opening-transcript } s \text{ out trace-roots trace-bs trace-final}$
 $\text{fri-dg composition-roots composition-bs composition-final fri-query-idxs}$
 $\text{trace-round-layers composition-round-layers}$
by (rule *accepted-with-partial-trace-openings-verifier-consumedD(1)*
 $[\text{OF consumed}]$)
have *verifier-space:*
 $\text{fri-query-idxs} \in \text{fri-query-index-list-space}$
by (rule *accepted-fri-opening-transcript-query-index-list-space*
 $[\text{OF fri-openings}]$)

have *alternate-space*: *query-idxs0* $\in$ *fri-query-index-list-space*
by (rule *accepted-with-partial-trace-openings-query-index-list-space*)
(rule *trace-fri-partial-candidate-evidenceD(1)[OF evidence0]*)
show *?thesis*
by (rule *that[OF verifier-space alternate-space]*)
qed

lemma *trace-fri-header-tied-low-candidate-nonconsumed-query-gap-shapes*:
assumes *trace-fri-header-tied-low-candidate-nonconsumed-query-gap s out*
obtains *fri-query-idxs query-idxs0* **where**
fri-query-idxs $\in$ *fri-query-index-list-space*
query-idxs0 $\in$ *fri-query-index-list-space*
query-idxs0 $\neq$ *fri-query-idxs*
proof –
from *assms* **obtain** *trace-roots trace-bs trace-final fri-dg composition-roots*
composition-bs composition-final fri-query-idxs trace-round-layers
composition-round-layers fr as trace-openings trace-table
fr0 query-idxs0 trace-openings0 trace-table0 **where**
consumed:
accepted-with-partial-trace-openings-verifier-consumed s out
trace-roots trace-bs trace-final fri-dg composition-roots
composition-bs composition-final fri-query-idxs trace-round-layers
composition-round-layers fr as trace-openings
and *evidence0*:
trace-fri-partial-candidate-evidence s out fr0 query-idxs0
trace-openings0 trace-table0
and *mismatch*: *query-idxs0* $\neq$ *fri-query-idxs*
by (rule *trace-fri-header-tied-low-candidate-nonconsumed-query-gapE*)
have *fri-openings*:
accepted-fri-opening-transcript s out trace-roots trace-bs trace-final
fri-dg composition-roots composition-bs composition-final fri-query-idxs
trace-round-layers composition-round-layers
by (rule *accepted-with-partial-trace-openings-verifier-consumedD(1)*)
[*OF consumed*])
have *verifier-space*:
fri-query-idxs $\in$ *fri-query-index-list-space*
by (rule *accepted-fri-opening-transcript-query-index-list-space*)
[*OF fri-openings*])
have *alternate-space*: *query-idxs0* $\in$ *fri-query-index-list-space*
by (rule *accepted-with-partial-trace-openings-query-index-list-space*)
(rule *trace-fri-partial-candidate-evidenceD(1)[OF evidence0]*)
show *?thesis*
by (rule *that[of fri-query-idxs query-idxs0,*
OF verifier-space alternate-space mismatch])
qed

lemma *trace-fri-header-tied-low-candidate-nonconsumed-irrelevant-root-gap-shapes*:
assumes
trace-fri-header-tied-low-candidate-nonconsumed-irrelevant-root-gap s out

obtains *fri-query-idxs query-idxs0* **where**
fri-query-idxs $\in$ *fri-query-index-list-space*
query-idxs0 $\in$ *fri-query-index-list-space*
query-idxs0 = *fri-query-idxs*

proof –

from *assms* **obtain** *trace-roots trace-bs trace-final fri-dg composition-roots*
composition-bs composition-final fri-query-idxs trace-round-layers
composition-round-layers fr as trace-openings trace-table
fr0 query-idxs0 trace-openings0 trace-table0 **where**
consumed:
accepted-with-partial-trace-openings-verifier-consumed s out
trace-roots trace-bs trace-final fri-dg composition-roots
composition-bs composition-final fri-query-idxs trace-round-layers
composition-round-layers fr as trace-openings
and *evidence0:*
trace-fri-partial-candidate-evidence s out fr0 query-idxs0
trace-openings0 trace-table0
and *mismatch:* *fr0* $\neq$ *fr*
and *query-eq:* *query-idxs0* = *fri-query-idxs*
by (rule *trace-fri-header-tied-low-candidate-nonconsumed-irrelevant-root-gapE*)
have *fri-openings:*
accepted-fri-opening-transcript s out trace-roots trace-bs trace-final
fri-dg composition-roots composition-bs composition-final fri-query-idxs
trace-round-layers composition-round-layers
by (rule *accepted-with-partial-trace-openings-verifier-consumedD*(1)
[*OF consumed*])
have *verifier-space:*
fri-query-idxs $\in$ *fri-query-index-list-space*
by (rule *accepted-fri-opening-transcript-query-index-list-space*
[*OF fri-openings*])
have *alternate-space:* *query-idxs0* $\in$ *fri-query-index-list-space*
by (rule *accepted-with-partial-trace-openings-query-index-list-space*)
(rule *trace-fri-partial-candidate-evidenceD*(1)[*OF evidence0*])
show *?thesis*
by (rule *that*[*OF verifier-space alternate-space query-eq*])
qed

lemma *trace-fri-header-tied-low-candidate-nonconsumed-root-gap-imp-query-or-irrelevant-root:*
assumes *trace-fri-header-tied-low-candidate-nonconsumed-root-gap s out*
shows
trace-fri-header-tied-low-candidate-nonconsumed-query-gap s out $\vee$
trace-fri-header-tied-low-candidate-nonconsumed-irrelevant-root-gap
s out

proof –

from *assms* **obtain** *trace-roots trace-bs trace-final fri-dg composition-roots*
composition-bs composition-final fri-query-idxs trace-round-layers
composition-round-layers fr as trace-openings trace-table
fr0 query-idxs0 trace-openings0 trace-table0 **where**
gap: *trace-fri-reachable-header-tie-gap s out*

and *consumed*:
accepted-with-partial-trace-openings-verifier-consumed s out
trace-roots trace-bs trace-final fri-dg composition-roots
composition-bs composition-final fri-query-idxs trace-round-layers
composition-round-layers fr as trace-openings
and *candidate*: *partial-trace-table-candidate trace-table trace-openings*
and *low*: *trace-table-low-degree trace-table*
and *evidence0*:
trace-fri-partial-candidate-evidence s out fr0 query-idxs0
trace-openings0 trace-table0
and *root-mismatch*: *fr0 $\neq$ fr*
and *openings-mismatch*: *trace-openings0 $\neq$ trace-openings*
by (*rule trace-fri-header-tied-low-candidate-nonconsumed-root-gapE*)
show *?thesis*
proof (*cases query-idxs0 $\neq$ fri-query-idxs*)
case *True*
have *trace-fri-header-tied-low-candidate-nonconsumed-query-gap s out*
unfolding *trace-fri-header-tied-low-candidate-nonconsumed-query-gap-def*
by (*intro exI conjI*)
(rule gap, rule consumed, rule candidate, rule low, rule evidence0,
rule True, rule openings-mismatch)
then show *?thesis* **by** *simp*
next
case *False*
then have *query-eq: query-idxs0 = fri-query-idxs*
by *simp*
have *trace-fri-header-tied-low-candidate-nonconsumed-irrelevant-root-gap*
s out
unfolding
trace-fri-header-tied-low-candidate-nonconsumed-irrelevant-root-gap-def
by (*intro exI conjI*)
(rule gap, rule consumed, rule candidate, rule low, rule evidence0,
rule root-mismatch, rule query-eq, rule openings-mismatch)
then show *?thesis* **by** *simp*
qed
qed

lemma *checked-staged-security-trace-fri-header-tied-low-candidate-nonconsumed-root-gap-bound-from-query-and-irrel*

assumes *query-bound*:

wp-event (checked-staged-security-experiment-with-data-state A)

(staged-security-with-data-state-verifier-event

trace-fri-header-tied-low-candidate-nonconsumed-query-gap)

adversary-initial-state $\leq Q$

and *irrelevant-bound*:

wp-event (checked-staged-security-experiment-with-data-state A)

(staged-security-with-data-state-verifier-event

trace-fri-header-tied-low-candidate-nonconsumed-irrelevant-root-gap)

adversary-initial-state $\leq I$

shows


```

wp-event (checked-staged-security-experiment-with-data-state A)
  (staged-security-with-data-state-verifier-event
    trace-fri-header-tied-low-candidate-nonconsumed-root-gap)
  adversary-initial-state  $\leq Q + I$ 
proof –
  let ?M = checked-staged-security-experiment-with-data-state A
  let ?root =
    staged-security-with-data-state-verifier-event
    trace-fri-header-tied-low-candidate-nonconsumed-root-gap
  let ?query =
    staged-security-with-data-state-verifier-event
    trace-fri-header-tied-low-candidate-nonconsumed-query-gap
  let ?irrelevant =
    staged-security-with-data-state-verifier-event
    trace-fri-header-tied-low-candidate-nonconsumed-irrelevant-root-gap
  have wp-event ?M ?root adversary-initial-state  $\leq$ 
    wp-event ?M ( $\lambda$ out. ?query out  $\vee$  ?irrelevant out)
    adversary-initial-state
  unfolding staged-security-with-data-state-verifier-event-def
  by (rule wp-event-mono-on-support)
  (auto dest:
    trace-fri-header-tied-low-candidate-nonconsumed-root-gap-imp-query-or-irrelevant-root
    split: option.splits prod.splits)
  also have ...  $\leq$ 
    wp-event ?M ?query adversary-initial-state +
    wp-event ?M ?irrelevant adversary-initial-state
  by (rule wp-event-union-bound)
  also have ...  $\leq Q + I$ 
  by (rule add-mono[OF query-bound irrelevant-bound])
  finally show ?thesis .
qed

```

lemma trace-fri-header-tied-low-candidate-nonconsumed-openings-gap-imp-root-or-query:

assumes trace-fri-header-tied-low-candidate-nonconsumed-openings-gap s out

shows

trace-fri-header-tied-low-candidate-nonconsumed-root-gap s out $\vee$
 trace-fri-header-tied-low-candidate-nonconsumed-query-gap s out

proof –

from assms **obtain** trace-roots trace-bs trace-final fri-dg composition-roots

composition-bs composition-final fri-query-idxs

trace-round-layers composition-round-layers fr as

trace-openings trace-table fr0 query-idxs0 trace-openings0 trace-table0

where

gap: trace-fri-reachable-header-tie-gap s out

and consumed:

accepted-with-partial-trace-openings-verifier-consumed s out

trace-roots trace-bs trace-final fri-dg composition-roots

composition-bs composition-final fri-query-idxs trace-round-layers

composition-round-layers fr as trace-openings

```

and candidate: partial-trace-table-candidate trace-table trace-openings
and low: trace-table-low-degree trace-table
and evidence0:
  trace-fri-partial-candidate-evidence s out fr0 query-idxs0
  trace-openings0 trace-table0
and mismatch: fr0  $\neq$  fr  $\vee$  query-idxs0  $\neq$  fri-query-idxs
and openings-mismatch: trace-openings0  $\neq$  trace-openings
unfolding trace-fri-header-tied-low-candidate-nonconsumed-openings-gap-def
by blast
from mismatch show ?thesis
proof
  assume root-mismatch: fr0  $\neq$  fr
  have trace-fri-header-tied-low-candidate-nonconsumed-root-gap s out
    unfolding trace-fri-header-tied-low-candidate-nonconsumed-root-gap-def
    apply (rule exI[where x=trace-roots])
    apply (rule exI[where x=trace-bs])
    apply (rule exI[where x=trace-final])
    apply (rule exI[where x=fri-dg])
    apply (rule exI[where x=composition-roots])
    apply (rule exI[where x=composition-bs])
    apply (rule exI[where x=composition-final])
    apply (rule exI[where x=fri-query-idxs])
    apply (rule exI[where x=trace-round-layers])
    apply (rule exI[where x=composition-round-layers])
    apply (rule exI[where x=fr])
    apply (rule exI[where x=as])
    apply (rule exI[where x=trace-openings])
    apply (rule exI[where x=trace-table])
    apply (rule exI[where x=fr0])
    apply (rule exI[where x=query-idxs0])
    apply (rule exI[where x=trace-openings0])
    apply (rule exI[where x=trace-table0])
    using gap consumed candidate low evidence0 root-mismatch openings-mismatch
    by blast
  then show ?thesis by simp
next
  assume query-mismatch: query-idxs0  $\neq$  fri-query-idxs
  have trace-fri-header-tied-low-candidate-nonconsumed-query-gap s out
    unfolding trace-fri-header-tied-low-candidate-nonconsumed-query-gap-def
    apply (rule exI[where x=trace-roots])
    apply (rule exI[where x=trace-bs])
    apply (rule exI[where x=trace-final])
    apply (rule exI[where x=fri-dg])
    apply (rule exI[where x=composition-roots])
    apply (rule exI[where x=composition-bs])
    apply (rule exI[where x=composition-final])
    apply (rule exI[where x=fri-query-idxs])
    apply (rule exI[where x=trace-round-layers])
    apply (rule exI[where x=composition-round-layers])

```

```

    apply (rule exI[where x=fr])
    apply (rule exI[where x=as])
    apply (rule exI[where x=trace-openings])
    apply (rule exI[where x=trace-table])
    apply (rule exI[where x=fr0])
    apply (rule exI[where x=query-idxs0])
    apply (rule exI[where x=trace-openings0])
    apply (rule exI[where x=trace-table0])
    using gap consumed candidate low evidence0 query-mismatch openings-mismatch
    by blast
  then show ?thesis by simp
qed
qed

lemma trace-fri-header-tied-low-candidate-nonconsumed-root-gap-imp-untied-root-gap:
  assumes trace-fri-header-tied-low-candidate-nonconsumed-root-gap s out
  shows trace-fri-header-tied-low-candidate-untied-root-gap s out
proof -
  from assms obtain trace-roots trace-bs trace-final fri-dg
    composition-roots composition-bs composition-final fri-query-idxs
    trace-round-layers composition-round-layers fr as trace-openings
    trace-table fr0 query-idxs0 trace-openings0 trace-table0
  where gap: trace-fri-reachable-header-tie-gap s out
    and consumed:
      accepted-with-partial-trace-openings-verifier-consumed s out
      trace-roots trace-bs trace-final fri-dg composition-roots
      composition-bs composition-final fri-query-idxs trace-round-layers
      composition-round-layers fr as trace-openings
    and candidate: partial-trace-table-candidate trace-table trace-openings
    and low: trace-table-low-degree trace-table
    and evidence0:
      trace-fri-partial-candidate-evidence s out fr0 query-idxs0
      trace-openings0 trace-table0
    and root-mismatch: fr0  $\neq$  fr
  by (rule trace-fri-header-tied-low-candidate-nonconsumed-root-gapE)
  have transcript:
    accepted-fri-opening-transcript s out trace-roots trace-bs
    trace-final fri-dg composition-roots composition-bs
    composition-final fri-query-idxs trace-round-layers
    composition-round-layers
  and header-ex:
     $\exists$  rest. verifier-header-transcript s fr trace-roots trace-final as
    fri-dg composition-roots composition-final rest
  and accepted:
    accepted-with-partial-trace-openings s out fr fri-query-idxs
    trace-openings
  by (rule accepted-with-partial-trace-openings-verifier-consumedD
    [OF consumed])+
  from header-ex obtain rest where header:

```

```

    verifier-header-transcript s fr trace-roots trace-final as fri-dg
      composition-roots composition-final rest
  by blast
have accepted0:
  accepted-with-partial-trace-openings s out fr0 query-idxs0
    trace-openings0
  and candidate0: partial-trace-table-candidate trace-table0 trace-openings0
  and not-low0:  $\neg$  trace-table-low-degree trace-table0
  by (rule trace-fri-partial-candidate-evidenceD[OF evidence0])+
show ?thesis
  unfolding trace-fri-header-tied-low-candidate-untied-root-gap-def
  by (rule trace-fri-header-tied-low-candidate-untied-reachable-witness-caseI
      [OF gap transcript header accepted candidate low accepted0 candidate0
        not-low0])
    (simp add: root-mismatch)
qed

lemma trace-fri-header-tied-low-candidate-nonconsumed-query-gap-imp-untied-query-gap:
  assumes trace-fri-header-tied-low-candidate-nonconsumed-query-gap s out
  shows trace-fri-header-tied-low-candidate-untied-query-gap s out
proof -
  from assms obtain trace-roots trace-bs trace-final fri-dg
    composition-roots composition-bs composition-final fri-query-idxs
    trace-round-layers composition-round-layers fr as trace-openings
    trace-table fr0 query-idxs0 trace-openings0 trace-table0
  where gap: trace-fri-reachable-header-tie-gap s out
  and consumed:
    accepted-with-partial-trace-openings-verifier-consumed s out
    trace-roots trace-bs trace-final fri-dg composition-roots
    composition-bs composition-final fri-query-idxs trace-round-layers
    composition-round-layers fr as trace-openings
  and candidate: partial-trace-table-candidate trace-table trace-openings
  and low: trace-table-low-degree trace-table
  and evidence0:
    trace-fri-partial-candidate-evidence s out fr0 query-idxs0
    trace-openings0 trace-table0
  and query-mismatch: query-idxs0  $\neq$  fri-query-idxs
  by (rule trace-fri-header-tied-low-candidate-nonconsumed-query-gapE)
have transcript:
  accepted-fri-opening-transcript s out trace-roots trace-bs
  trace-final fri-dg composition-roots composition-bs
  composition-final fri-query-idxs trace-round-layers
  composition-round-layers
  and header-ex:
     $\exists$  rest. verifier-header-transcript s fr trace-roots trace-final as
    fri-dg composition-roots composition-final rest
  and accepted:
    accepted-with-partial-trace-openings s out fr fri-query-idxs
    trace-openings

```

by (rule *accepted-with-partial-trace-openings-verifier-consumedD*
 [OF consumed]))+
from *header-ex* **obtain** *rest* **where** *header*:
 verifier-header-transcript *s* fr *trace-roots* *trace-final* as *fri-dg*
 composition-roots composition-final *rest*
by *blast*
have *accepted0*:
 accepted-with-partial-trace-openings *s* out fr0 *query-idxs0*
 trace-openings0
and *candidate0*: partial-trace-table-candidate *trace-table0* *trace-openings0*
and *not-low0*: $\neg$ trace-table-low-degree *trace-table0*
by (rule *trace-fri-partial-candidate-evidenceD*[OF *evidence0*]))+
show ?thesis
unfolding *trace-fri-header-tied-low-candidate-untied-query-gap-def*
by (rule *trace-fri-header-tied-low-candidate-untied-reachable-witness-caseI*
 [OF *gap* transcript *header* *accepted* *candidate* *low* *accepted0* *candidate0*
 not-low0]))
 (simp add: *query-mismatch*)
qed

lemma *checked-staged-security-trace-fri-header-tied-low-candidate-nonconsumed-openings-gap-bound-from-root-query*

assumes *root-bound*:

wp-event (*checked-staged-security-experiment-with-data-state A*)
 (*staged-security-with-data-state-verifier-event*
 trace-fri-header-tied-low-candidate-nonconsumed-root-gap)
adversary-initial-state $\leq R$

and *query-bound*:

wp-event (*checked-staged-security-experiment-with-data-state A*)
 (*staged-security-with-data-state-verifier-event*
 trace-fri-header-tied-low-candidate-nonconsumed-query-gap)
adversary-initial-state $\leq Q$

shows

wp-event (*checked-staged-security-experiment-with-data-state A*)
 (*staged-security-with-data-state-verifier-event*
 trace-fri-header-tied-low-candidate-nonconsumed-openings-gap)
adversary-initial-state $\leq R + Q$

proof –

let ?*M* = *checked-staged-security-experiment-with-data-state A*

let ?*nonconsumed* =

staged-security-with-data-state-verifier-event
 trace-fri-header-tied-low-candidate-nonconsumed-openings-gap

let ?*root* =

staged-security-with-data-state-verifier-event
 trace-fri-header-tied-low-candidate-nonconsumed-root-gap

let ?*query* =

staged-security-with-data-state-verifier-event
 trace-fri-header-tied-low-candidate-nonconsumed-query-gap

have *split*:

wp-event ?*M* ?*nonconsumed* *adversary-initial-state* $\leq$

```

    wp-event ?M ( $\lambda$ out. ?root out  $\vee$  ?query out) adversary-initial-state
unfolding staged-security-with-data-state-verifier-event-def
by (rule wp-event-mono-on-support)
  (auto dest:
    trace-fri-header-tied-low-candidate-nonconsumed-openings-gap-imp-root-or-query
    split: option.splits prod.splits)
also have ...  $\leq$ 
  wp-event ?M ?root adversary-initial-state +
  wp-event ?M ?query adversary-initial-state
by (rule wp-event-union-bound)
also have ...  $\leq R + Q$ 
by (rule add-mono[OF root-bound query-bound])
finally show ?thesis .
qed

lemma trace-fri-zero-round-final-obstruction-imp-checked-merkle-low-consumed-or-header:
assumes support: out  $\in$  set-dist (execute verify-monad s)
and obstruction: trace-fri-zero-round-final-obstruction s out
shows
  trace-fri-zero-round-checked-final-obstruction s out  $\vee$ 
  partial-merkle-inconsistency-bad s out  $\vee$ 
  trace-fri-zero-round-verifier-consumed-low-candidate-gap s out  $\vee$ 
  trace-fri-bad-with-header-tied-partial-candidate s out
proof –
from trace-fri-zero-round-final-obstructionE[OF obstruction]
obtain trace-roots trace-bs trace-final dg composition-roots
  composition-bs composition-final fri-query-idxs trace-round-layers
  composition-round-layers fr0 candidate-query-idxs trace-openings0
  trace-table0 where evidence:
  trace-fri-partial-candidate-opening-evidence s out trace-roots
  trace-bs trace-final dg composition-roots composition-bs
  composition-final fri-query-idxs trace-round-layers
  composition-round-layers fr0 candidate-query-idxs trace-openings0
  trace-table0
and rounds-empty: length trace-bs = 0
by (rule trace-fri-zero-round-final-obstructionE[OF obstruction])
have fri-openings:
  accepted-fri-opening-transcript s out trace-roots trace-bs
  trace-final dg composition-roots composition-bs composition-final
  fri-query-idxs trace-round-layers composition-round-layers
using trace-fri-partial-candidate-opening-evidenceD(1)[OF evidence] .
from fri-openings obtain result final-state where out-eq:
  out = Some (result, final-state)
unfolding accepted-fri-opening-transcript-def by blast
from accepted-fri-opening-transcript-headerE[OF fri-openings]
obtain fr as rest where header:
  verifier-header-transcript s fr trace-roots trace-final as dg
  composition-roots composition-final rest
by blast

```

```

show ?thesis
proof (cases partial-merkle-inconsistency-bad s out)
  case True
  then show ?thesis by simp
next
  case no-merkle: False
  from accepted-fri-opening-transcript-trace-openings-for-query-indices
    [OF fri-openings header out-eq]
  obtain trace-openings where partial:
    accepted-with-partial-trace-openings s out fr fri-query-idxs
    trace-openings
  by blast
  obtain trace-table where candidate:
    partial-trace-table-candidate trace-table trace-openings
  using accepted-with-partial-trace-openings-candidate-if-no-partial-merkle-bad
    [OF partial[unfolded out-eq] no-merkle[unfolded out-eq]]
  by blast
show ?thesis
proof (cases trace-table-low-degree trace-table)
  case True
  then show ?thesis
  proof (cases trace-fri-bad-with-header-tied-partial-candidate s out)
    case True
    then show ?thesis by simp
  next
    case no-header: False
    have reachable:
      trace-fri-bad-with-reachable-partial-candidate s out
    using trace-fri-zero-round-final-obstruction-imp-header-tied-or-gap
      [OF obstruction] no-header
    unfolding trace-fri-reachable-header-tie-gap-def by simp
    have gap: trace-fri-reachable-header-tie-gap s out
    unfolding trace-fri-reachable-header-tie-gap-def
    using reachable no-header by simp
    have consumed:
      accepted-with-partial-trace-openings-verifier-consumed s out
      trace-roots trace-bs trace-final dg composition-roots
      composition-bs composition-final fri-query-idxs
      trace-round-layers composition-round-layers fr as
      trace-openings
    unfolding accepted-with-partial-trace-openings-verifier-consumed-def
    by (intro conjI exI)
      (rule fri-openings, rule header, rule partial)
    have low-gap:
      trace-fri-zero-round-verifier-consumed-low-candidate-gap s out
    unfolding
      trace-fri-zero-round-verifier-consumed-low-candidate-gap-def
    by (intro exI conjI)
      (rule obstruction, rule gap, rule consumed, rule rounds-empty,

```

```

      rule candidate, rule True)
    then show ?thesis by simp
  qed
next
case not-low: False
have not-final:
   $\neg$  fri-final-constant-consistent trace-table trace-final
proof
  assume final:
    fri-final-constant-consistent trace-table trace-final
  have low:
    trace-table-low-degree trace-table
  by (rule
    trace-zero-round-candidate-low-degree-if-final-consistent
    [OF rounds-empty fri-openings candidate final])
  then show False
    using not-low by contradiction
  qed
have header-zero:
  trace-fri-header-tied-zero-round-final-obstruction s out
  unfolding trace-fri-header-tied-zero-round-final-obstruction-def
  by (intro exI conjI)
    (rule fri-openings, rule header, rule partial, rule candidate,
     rule not-low, rule rounds-empty, rule not-final)
then show ?thesis
  using
    trace-fri-header-tied-zero-round-final-obstruction-imp-checked-or-merkle
  by blast
qed
qed
qed

lemma checked-staged-security-trace-fri-zero-round-final-obstruction-bound-from-staged-checked-merkle-low-consumed
  fixes Q M L H :: prob
  assumes checked-bound:
    wp-event (checked-staged-security-experiment-with-data-state A)
      (staged-security-with-data-state-verifier-event
        trace-fri-zero-round-checked-final-obstruction)
    adversary-initial-state  $\leq$  Q
  and merkle-bound:
    wp-event (checked-staged-security-experiment-with-data-state A)
      (staged-security-with-data-state-verifier-event
        partial-merkle-inconsistency-bad)
    adversary-initial-state  $\leq$  M
  and low-consumed-bound:
    wp-event (checked-staged-security-experiment-with-data-state A)
      (staged-security-with-data-state-verifier-event
        trace-fri-zero-round-verifier-consumed-low-candidate-gap)
    adversary-initial-state  $\leq$  L

```



```

and header-bound:
  wp-event (checked-staged-security-experiment-with-data-state A)
    (staged-security-with-data-state-verifier-event
      trace-fri-bad-with-header-tied-partial-candidate)
  adversary-initial-state  $\leq H$ 
shows
  wp-event (checked-staged-security-experiment-with-data-state A)
    (staged-security-with-data-state-verifier-event
      trace-fri-zero-round-final-obstruction)
  adversary-initial-state  $\leq Q + M + L + H$ 
proof –
  let ?M = checked-staged-security-experiment-with-data-state A
  let ?zero =
    staged-security-with-data-state-verifier-event
    trace-fri-zero-round-final-obstruction
  let ?checked =
    staged-security-with-data-state-verifier-event
    trace-fri-zero-round-checked-final-obstruction
  let ?merkle =
    staged-security-with-data-state-verifier-event
    partial-merkle-inconsistency-bad
  let ?low =
    staged-security-with-data-state-verifier-event
    trace-fri-zero-round-verifier-consumed-low-candidate-gap
  let ?header =
    staged-security-with-data-state-verifier-event
    trace-fri-bad-with-header-tied-partial-candidate
  have wp-event ?M ?zero adversary-initial-state  $\leq$ 
    wp-event ?M
    ( $\lambda$ out. ?checked out  $\vee$  ?merkle out  $\vee$  ?low out  $\vee$ 
      ?header out)
    adversary-initial-state
  proof (rule wp-event-mono-on-support)
  fix out
  assume support:
    out  $\in$  set-dist (execute ?M adversary-initial-state)
    and zero: ?zero out
  show ?checked out  $\vee$  ?merkle out  $\vee$  ?low out  $\vee$  ?header out
  proof (cases out)
  case None
  then show ?thesis
    using zero unfolding staged-security-with-data-state-verifier-event-def
    by simp
  next
  case (Some out-data)
  then obtain data attacker-state result final-state where out-eq:
    out = Some (((data, attacker-state), result), final-state)
    by (cases out-data, auto split: prod.splits)
  let ?s =

```

```

    verifier-state-from-adversary attacker-state
    (staged-proof-transcript data)
from checked-staged-security-experiment-with-data-state-outcomeE
    [OF support[unfolded out-eq]]
have verify-support:
    Some (result, final-state) ∈ set-dist (execute verify-monad ?s)
by blast
have zero-local:
    trace-fri-zero-round-final-obstruction ?s
    (Some (result, final-state))
using zero unfolding out-eq
    staged-security-with-data-state-verifier-event-def
by simp
have
    trace-fri-zero-round-checked-final-obstruction ?s
    (Some (result, final-state)) ∨
    partial-merkle-inconsistency-bad ?s
    (Some (result, final-state)) ∨
    trace-fri-zero-round-verifier-consumed-low-candidate-gap ?s
    (Some (result, final-state)) ∨
    trace-fri-bad-with-header-tied-partial-candidate ?s
    (Some (result, final-state))
by (rule
    trace-fri-zero-round-final-obstruction-imp-checked-merkle-low-consumed-or-header
    [OF verify-support zero-local])
then show ?thesis
    unfolding out-eq staged-security-with-data-state-verifier-event-def
    by simp
qed
qed
also have ... ≤
    wp-event ?M ?checked adversary-initial-state +
    wp-event ?M ?merkle adversary-initial-state +
    wp-event ?M ?low adversary-initial-state +
    wp-event ?M ?header adversary-initial-state
by (rule wp-event-union-bound4)
also have ... ≤ Q + M + L + H
by (intro add-mono checked-bound merkle-bound low-consumed-bound
    header-bound)
finally show ?thesis .
qed

lemma trace-fri-header-tied-low-candidate-untied-nonnormalized-openings-gap-imp-nonconsumed:
assumes
    trace-fri-header-tied-low-candidate-untied-nonnormalized-openings-gap
    s out
shows
    trace-fri-header-tied-low-candidate-nonconsumed-openings-gap s out
proof —

```

from *assms* **obtain** *trace-roots trace-bs trace-final fri-dg*
composition-roots composition-bs composition-final fri-query-idxs
trace-round-layers composition-round-layers fr as rest trace-openings
trace-table fr0 query-idxs0 trace-openings0 trace-table0 **where**
gap: trace-fri-reachable-header-tie-gap s out
and *fri-openings:*
accepted-fri-opening-transcript s out trace-roots trace-bs
trace-final fri-dg composition-roots composition-bs
composition-final fri-query-idxs trace-round-layers
composition-round-layers
and *header:*
verifier-header-transcript s fr trace-roots trace-final as fri-dg
composition-roots composition-final rest
and *partial:*
accepted-with-partial-trace-openings s out fr fri-query-idxs
trace-openings
and *candidate:*
partial-trace-table-candidate trace-table trace-openings
and *low: trace-table-low-degree trace-table*
and *partial0:*
accepted-with-partial-trace-openings s out fr0 query-idxs0
trace-openings0
and *candidate0:*
partial-trace-table-candidate trace-table0 trace-openings0
and *not-low0: $\neg$ trace-table-low-degree trace-table0*
and *mismatch:*
fr0 $\neq$ fr $\vee$ query-idxs0 $\neq$ fri-query-idxs
and *openings-mismatch:*
trace-openings0 $\neq$ trace-openings
unfolding
trace-fri-header-tied-low-candidate-untied-nonnormalized-openings-gap-def
trace-fri-header-tied-low-candidate-untied-reachable-witness-case-def
by *blast*
have *consumed:*
accepted-with-partial-trace-openings-verifier-consumed s out
trace-roots trace-bs trace-final fri-dg composition-roots
composition-bs composition-final fri-query-idxs trace-round-layers
composition-round-layers fr as trace-openings
unfolding *accepted-with-partial-trace-openings-verifier-consumed-def*
by (*intro conjI exI*)
(*rule fri-openings, rule header, rule partial*)
have *evidence0:*
trace-fri-partial-candidate-evidence s out fr0 query-idxs0
trace-openings0 trace-table0
unfolding *trace-fri-partial-candidate-evidence-def*
by (*intro conjI*)
(*rule partial0, rule candidate0, rule not-low0*)
show *?thesis*
unfolding *trace-fri-header-tied-low-candidate-nonconsumed-openings-gap-def*

by (intro exI conjI)
 (rule gap, rule consumed, rule candidate, rule low, rule evidence0,
 rule mismatch, rule openings-mismatch)
 qed

lemma *checked-staged-security-trace-fri-header-tied-low-candidate-untied-nonnormalized-openings-gap-bound-from-n*
assumes *nonconsumed-bound:*
wp-event (checked-staged-security-experiment-with-data-state A)
(staged-security-with-data-state-verifier-event
trace-fri-header-tied-low-candidate-nonconsumed-openings-gap)
adversary-initial-state ≤ N
shows
wp-event (checked-staged-security-experiment-with-data-state A)
(staged-security-with-data-state-verifier-event
trace-fri-header-tied-low-candidate-untied-nonnormalized-openings-gap)
adversary-initial-state ≤ N
proof (rule order-trans[*OF - nonconsumed-bound*])
show *wp-event (checked-staged-security-experiment-with-data-state A)*
(staged-security-with-data-state-verifier-event
trace-fri-header-tied-low-candidate-untied-nonnormalized-openings-gap)
adversary-initial-state ≤
wp-event (checked-staged-security-experiment-with-data-state A)
(staged-security-with-data-state-verifier-event
trace-fri-header-tied-low-candidate-nonconsumed-openings-gap)
adversary-initial-state
unfolding *staged-security-with-data-state-verifier-event-def*
by (rule *wp-event-mono-on-support*)
 (auto dest:
trace-fri-header-tied-low-candidate-untied-nonnormalized-openings-gap-imp-nonconsumed
split: option.splits prod.splits)
 qed

lemma *checked-staged-security-trace-fri-sampled-assignment-conflict-bound-structural:*
assumes *wf: staged-budget-wellformed budgets*
and *controlled: staged-adversary-controlled budgets A*
shows
wp-event (checked-staged-security-experiment-with-data-state A)
(staged-security-with-data-state-verifier-event
trace-fri-sampled-assignment-conflict)
adversary-initial-state ≤
trace-fri-empty-sampled-assignment-structural-error-for budgets A
proof –
have *without:*
wp-event (checked-staged-security-experiment-with-data-state A)
(staged-security-with-data-state-verifier-event
trace-fri-verifier-tied-sampled-assignment-conflict-without-same-layer)
adversary-initial-state ≤
reachable-verifier-event-bound-for A
trace-fri-verifier-tied-sampled-assignment-conflict-without-same-layer

by (rule *checked-staged-security-with-data-state-verifier-event-bound-from-cont*)
 (simp add:
 trace-fri-verifier-tied-sampled-assignment-conflict-without-same-layer-def
 accepted-fri-opening-transcript-def,
 rule *reachable-verifier-event-bound-for-ge*)
have merkle:
 wp-event (checked-staged-security-experiment-with-data-state A)
 (staged-security-with-data-state-verifier-event
 partial-merkle-inconsistency-bad)
 adversary-initial-state $\leq$
 trace-fri-controlled-merkle-error-for budgets
unfolding trace-fri-controlled-merkle-error-for-def
by (rule
 checked-staged-security-with-data-state-partial-merkle-inconsistency-bad-bound-controlled
 [OF wf controlled])
have gap:
 wp-event (checked-staged-security-experiment-with-data-state A)
 (staged-security-with-data-state-verifier-event
 trace-fri-verifier-tied-same-layer-auth-gap)
 adversary-initial-state $\leq$
 reachable-verifier-event-bound-for A
 trace-fri-verifier-tied-same-layer-auth-gap
by (rule *checked-staged-security-with-data-state-verifier-event-bound-from-cont*)
 (simp add: *trace-fri-verifier-tied-same-layer-auth-gap-def*
 accepted-fri-opening-transcript-def,
 rule *reachable-verifier-event-bound-for-ge*)
have tied:
 wp-event (checked-staged-security-experiment-with-data-state A)
 (staged-security-with-data-state-verifier-event
 trace-fri-verifier-tied-sampled-assignment-conflict)
 adversary-initial-state $\leq$
 (reachable-verifier-event-bound-for A
 trace-fri-verifier-tied-sampled-assignment-conflict-without-same-layer +
 trace-fri-controlled-merkle-error-for budgets) +
 reachable-verifier-event-bound-for A
 trace-fri-verifier-tied-same-layer-auth-gap
by (rule
 checked-staged-security-trace-fri-verifier-tied-conflict-bound-from-without-same-merkle-and-auth-gap
 [OF without merkle gap])
have untied:
 wp-event (checked-staged-security-experiment-with-data-state A)
 (staged-security-with-data-state-verifier-event
 trace-fri-sampled-assignment-verifier-untied-gap)
 adversary-initial-state $\leq$
 reachable-verifier-event-bound-for A
 trace-fri-sampled-assignment-verifier-untied-gap
by (rule *checked-staged-security-with-data-state-verifier-event-bound-from-cont*)
 (simp add: *trace-fri-sampled-assignment-verifier-untied-gap-def*
 trace-fri-sampled-assignment-conflict-def)

```

    trace-fri-partial-candidate-opening-evidence-def
    accepted-fri-opening-transcript-def,
    rule reachable-verifier-event-bound-for-ge)
show ?thesis
  unfolding trace-fri-empty-sampled-assignment-structural-error-for-def
  by (rule
    checked-staged-security-trace-fri-sampled-assignment-conflict-bound-from-verifier-tied-and-untied
    [OF tied untied])
qed

lemma checked-staged-security-trace-fri-zero-round-final-obstruction-bound-from-agreement-sets-header-and-openings
  fixes H :: prob
  assumes wf: staged-budget-wellformed budgets
  and controlled: staged-adversary-controlled budgets A
  and header-bound:
    wp-event (checked-staged-security-experiment-with-data-state A)
    (staged-security-with-data-state-verifier-event
      trace-fri-bad-with-header-tied-partial-candidate)
    adversary-initial-state ≤ H
  shows
    wp-event (checked-staged-security-experiment-with-data-state A)
    (staged-security-with-data-state-verifier-event
      trace-fri-zero-round-final-obstruction)
    adversary-initial-state ≤
    trace-fri-zero-round-agreement-set-target-error-for budgets +
    trace-fri-controlled-merkle-error-for budgets +
    (H +
      reachable-verifier-event-bound-for A
      trace-fri-header-tied-low-candidate-untied-openings-gap) +
    H
proof -
  have checked:
    wp-event (checked-staged-security-experiment-with-data-state A)
    (staged-security-with-data-state-verifier-event
      trace-fri-zero-round-checked-final-obstruction)
    adversary-initial-state ≤
    trace-fri-zero-round-agreement-set-target-error-for budgets
  by (rule
    checked-staged-security-trace-fri-zero-round-checked-final-obstruction-bound-from-agreement-sets
    [OF wf controlled])
  have merkle:
    wp-event (checked-staged-security-experiment-with-data-state A)
    (staged-security-with-data-state-verifier-event
      partial-merkle-inconsistency-bad)
    adversary-initial-state ≤
    trace-fri-controlled-merkle-error-for budgets
  unfolding trace-fri-controlled-merkle-error-for-def
  by (rule
    checked-staged-security-with-data-state-partial-merkle-inconsistency-bad-bound-controlled

```

[OF wf controlled])

have *openings*:

wp-event (*checked-staged-security-experiment-with-data-state A*)
 (*staged-security-with-data-state-verifier-event*
trace-fri-header-tied-low-candidate-untied-openings-gap)
adversary-initial-state $\leq$
reachable-verifier-event-bound-for A
trace-fri-header-tied-low-candidate-untied-openings-gap

proof (rule *checked-staged-security-with-data-state-verifier-event-bound-from-cont*)

show $\bigwedge s. \neg$
trace-fri-header-tied-low-candidate-untied-openings-gap s None
by (*simp add: trace-fri-header-tied-low-candidate-untied-openings-gap-def*
trace-fri-header-tied-low-candidate-untied-reachable-witness-case-def
accepted-fri-opening-transcript-def)

next

fix *data attacker-state*

assume *support*:

Some (data, attacker-state) ∈
set-dist
(execute (checked-staged-transcript-program A)
adversary-initial-state)

show *wp-event verify-monad*
(trace-fri-header-tied-low-candidate-untied-openings-gap
(verifier-state-from-adversary attacker-state
(staged-proof-transcript data)))
(verifier-state-from-adversary attacker-state
(staged-proof-transcript data)))
 $\leq$ *reachable-verifier-event-bound-for A*
trace-fri-header-tied-low-candidate-untied-openings-gap
by (rule *reachable-verifier-event-bound-for-ge[OF support]*)

qed

have *untied*:

wp-event (*checked-staged-security-experiment-with-data-state A*)
 (*staged-security-with-data-state-verifier-event*
trace-fri-header-tied-low-candidate-untied-reachable-witness-gap)
adversary-initial-state $\leq$
reachable-verifier-event-bound-for A
trace-fri-header-tied-low-candidate-untied-openings-gap

proof –

have *wp-event* (*checked-staged-security-experiment-with-data-state A*)
 (*staged-security-with-data-state-verifier-event*
trace-fri-header-tied-low-candidate-untied-reachable-witness-gap)
adversary-initial-state $\leq$
wp-event (*checked-staged-security-experiment-with-data-state A*)
 (*staged-security-with-data-state-verifier-event*
trace-fri-header-tied-low-candidate-untied-openings-gap)
adversary-initial-state
unfolding *staged-security-with-data-state-verifier-event-def*
by (rule *wp-event-mono-on-support*)

```

      (auto dest: trace-fri-header-tied-low-candidate-untied-gap-imp-openings
        split: option.splits prod.splits)
    also have ... ≤
      reachable-verifier-event-bound-for A
      trace-fri-header-tied-low-candidate-untied-openings-gap
    by (rule openings)
  finally show ?thesis .
qed
have transfer:
  wp-event (checked-staged-security-experiment-with-data-state A)
    (staged-security-with-data-state-verifier-event
      trace-fri-header-tied-candidate-transfer-gap)
  adversary-initial-state ≤
    H +
    reachable-verifier-event-bound-for A
    trace-fri-header-tied-low-candidate-untied-openings-gap
  by (rule
    checked-staged-security-trace-fri-header-tied-candidate-transfer-gap-bound-from-staged-header-and-untied
      [OF header-bound untied])
show ?thesis
  by (rule
    checked-staged-security-trace-fri-zero-round-final-obstruction-bound-from-staged-checked-merkle-transfer-header
      [OF checked merkle transfer header-bound])
qed

lemma checked-staged-security-trace-fri-header-tied-low-candidate-untied-openings-gap-bound-from-hash-and-nonnorm
  assumes wf: staged-budget-wellformed budgets
  and controlled: staged-adversary-controlled budgets A
  shows
    wp-event (checked-staged-security-experiment-with-data-state A)
      (staged-security-with-data-state-verifier-event
        trace-fri-header-tied-low-candidate-untied-openings-gap)
    adversary-initial-state ≤
      trace-fri-controlled-merkle-error-for budgets +
      ((reachable-verifier-event-bound-for A
        trace-fri-header-tied-low-candidate-nonconsumed-query-gap +
        reachable-verifier-event-bound-for A
        trace-fri-header-tied-low-candidate-nonconsumed-irrelevant-root-gap) +
        reachable-verifier-event-bound-for A
        trace-fri-header-tied-low-candidate-nonconsumed-query-gap)
  proof -
    let ?M = checked-staged-security-experiment-with-data-state A
    let ?openings =
      staged-security-with-data-state-verifier-event
      trace-fri-header-tied-low-candidate-untied-openings-gap
    let ?hash =
      staged-security-with-data-state-verifier-event
      hash-map-output-collision-bad
    let ?nonnormal =

```



```

    staged-security-with-data-state-verifier-event
    trace-fri-header-tied-low-candidate-untied-nonnormalized-openings-gap
let ?nonconsumed =
    staged-security-with-data-state-verifier-event
    trace-fri-header-tied-low-candidate-nonconsumed-openings-gap
let ?root =
    staged-security-with-data-state-verifier-event
    trace-fri-header-tied-low-candidate-nonconsumed-root-gap
let ?query =
    staged-security-with-data-state-verifier-event
    trace-fri-header-tied-low-candidate-nonconsumed-query-gap
let ?irrelevant =
    staged-security-with-data-state-verifier-event
    trace-fri-header-tied-low-candidate-nonconsumed-irrelevant-root-gap
have hash:
    wp-event ?M ?hash adversary-initial-state ≤
    trace-fri-controlled-merkle-error-for budgets
unfolding trace-fri-controlled-merkle-error-for-def
by (rule
    checked-staged-security-with-data-state-hash-map-output-collision-bad-bound-controlled
    [OF wf controlled])
have root:
    wp-event ?M ?root adversary-initial-state ≤
    reachable-verifier-event-bound-for A
    trace-fri-header-tied-low-candidate-nonconsumed-root-gap
by (rule checked-staged-security-with-data-state-verifier-event-bound-from-cont)
    (simp add: trace-fri-header-tied-low-candidate-nonconsumed-root-gap-def
    accepted-with-partial-trace-openings-verifier-consumed-def
    trace-fri-partial-candidate-evidence-def
    accepted-fri-opening-transcript-def,
    rule reachable-verifier-event-bound-for-ge)
have query:
    wp-event ?M ?query adversary-initial-state ≤
    reachable-verifier-event-bound-for A
    trace-fri-header-tied-low-candidate-nonconsumed-query-gap
by (rule checked-staged-security-with-data-state-verifier-event-bound-from-cont)
    (simp add: trace-fri-header-tied-low-candidate-nonconsumed-query-gap-def
    accepted-with-partial-trace-openings-verifier-consumed-def
    trace-fri-partial-candidate-evidence-def
    accepted-fri-opening-transcript-def,
    rule reachable-verifier-event-bound-for-ge)
have irrelevant:
    wp-event ?M ?irrelevant adversary-initial-state ≤
    reachable-verifier-event-bound-for A
    trace-fri-header-tied-low-candidate-nonconsumed-irrelevant-root-gap
by (rule checked-staged-security-with-data-state-verifier-event-bound-from-cont)
    (simp add:
    trace-fri-header-tied-low-candidate-nonconsumed-irrelevant-root-gap-def
    accepted-with-partial-trace-openings-verifier-consumed-def

```

```

    trace-fri-partial-candidate-evidence-def
    accepted-fri-opening-transcript-def,
    rule reachable-verifier-event-bound-for-ge)
have root-refined:
  wp-event ?M ?root adversary-initial-state ≤
    reachable-verifier-event-bound-for A
    trace-fri-header-tied-low-candidate-nonconsumed-query-gap +
    reachable-verifier-event-bound-for A
    trace-fri-header-tied-low-candidate-nonconsumed-irrelevant-root-gap
by (rule
  checked-staged-security-trace-fri-header-tied-low-candidate-nonconsumed-root-gap-bound-from-query-and-irrelevant
  [OF query irrelevant])
have nonconsumed-root-query:
  wp-event ?M ?nonconsumed adversary-initial-state ≤
    (reachable-verifier-event-bound-for A
    trace-fri-header-tied-low-candidate-nonconsumed-query-gap +
    reachable-verifier-event-bound-for A
    trace-fri-header-tied-low-candidate-nonconsumed-irrelevant-root-gap) +
    reachable-verifier-event-bound-for A
    trace-fri-header-tied-low-candidate-nonconsumed-query-gap
by (rule
  checked-staged-security-trace-fri-header-tied-low-candidate-nonconsumed-openings-gap-bound-from-root-query
  [OF root-refined query])
have nonnormal-root-query:
  wp-event ?M ?nonnormal adversary-initial-state ≤
    (reachable-verifier-event-bound-for A
    trace-fri-header-tied-low-candidate-nonconsumed-query-gap +
    reachable-verifier-event-bound-for A
    trace-fri-header-tied-low-candidate-nonconsumed-irrelevant-root-gap) +
    reachable-verifier-event-bound-for A
    trace-fri-header-tied-low-candidate-nonconsumed-query-gap
by (rule
  checked-staged-security-trace-fri-header-tied-low-candidate-untied-nonnormalized-openings-gap-bound-from-nonconsumed-root-query
  [OF nonconsumed-root-query])
have wp-event ?M ?openings adversary-initial-state ≤
  wp-event ?M (λout. ?hash out ∨ ?nonnormal out)
  adversary-initial-state
unfolding staged-security-with-data-state-verifier-event-def
by (rule wp-event-mono-on-support)
  (auto
  dest:
    trace-fri-header-tied-low-candidate-untied-openings-gap-imp-normalized-or-nonnormalized
    trace-fri-header-tied-low-candidate-untied-normalized-openings-gap-imp-hash-collision
  split: option.splits prod.splits)
also have ... ≤
  wp-event ?M ?hash adversary-initial-state +
  wp-event ?M ?nonnormal adversary-initial-state
by (rule wp-event-union-bound)
also have ... ≤

```

```

    trace-fri-controlled-merkle-error-for budgets +
    ((reachable-verifier-event-bound-for A
      trace-fri-header-tied-low-candidate-nonconsumed-query-gap +
      reachable-verifier-event-bound-for A
      trace-fri-header-tied-low-candidate-nonconsumed-irrelevant-root-gap) +
      reachable-verifier-event-bound-for A
      trace-fri-header-tied-low-candidate-nonconsumed-query-gap)
  by (rule add-mono[OF hash nonnormal-root-query])
  finally show ?thesis .
qed

lemma checked-staged-security-trace-fri-zero-round-verifier-consumed-low-candidate-gap-bound-from-hash-and-nonnormal-root-query
  assumes wf: staged-budget-wellformed budgets
  and controlled: staged-adversary-controlled budgets A
  shows
    wp-event (checked-staged-security-experiment-with-data-state A)
      (staged-security-with-data-state-verifier-event
        trace-fri-zero-round-verifier-consumed-low-candidate-gap)
    adversary-initial-state ≤
    trace-fri-controlled-merkle-error-for budgets +
    ((reachable-verifier-event-bound-for A
      trace-fri-header-tied-low-candidate-nonconsumed-query-gap +
      reachable-verifier-event-bound-for A
      trace-fri-header-tied-low-candidate-nonconsumed-irrelevant-root-gap) +
      reachable-verifier-event-bound-for A
      trace-fri-header-tied-low-candidate-nonconsumed-query-gap)
  proof -
    have wp-event (checked-staged-security-experiment-with-data-state A)
      (staged-security-with-data-state-verifier-event
        trace-fri-zero-round-verifier-consumed-low-candidate-gap)
    adversary-initial-state ≤
    wp-event (checked-staged-security-experiment-with-data-state A)
      (staged-security-with-data-state-verifier-event
        trace-fri-header-tied-low-candidate-untied-openings-gap)
    adversary-initial-state
  unfolding staged-security-with-data-state-verifier-event-def
  by (rule wp-event-mono-on-support)
  (auto
    dest:
      trace-fri-zero-round-verifier-consumed-low-candidate-gap-imp-untied-openings
      split: option.splits prod.splits)
  also have ... ≤
    trace-fri-controlled-merkle-error-for budgets +
    ((reachable-verifier-event-bound-for A
      trace-fri-header-tied-low-candidate-nonconsumed-query-gap +
      reachable-verifier-event-bound-for A
      trace-fri-header-tied-low-candidate-nonconsumed-irrelevant-root-gap) +
      reachable-verifier-event-bound-for A
      trace-fri-header-tied-low-candidate-nonconsumed-query-gap)

```

by (rule
 checked-staged-security-trace-fri-header-tied-low-candidate-untied-openings-gap-bound-from-hash-and-nonnormal
 [*OF wf controlled*])
 finally show ?thesis .
 qed

lemma *checked-staged-security-trace-fri-zero-round-final-obstruction-bound-from-agreement-sets-header-and-nonnormal*

fixes $H :: \text{prob}$

assumes *wf*: *staged-budget-wellformed budgets*

and *controlled*: *staged-adversary-controlled budgets A*

and *header-bound*:

wp-event (*checked-staged-security-experiment-with-data-state A*)

(*staged-security-with-data-state-verifier-event*

trace-fri-bad-with-header-tied-partial-candidate)

adversary-initial-state $\leq H$

shows

wp-event (*checked-staged-security-experiment-with-data-state A*)

(*staged-security-with-data-state-verifier-event*

trace-fri-zero-round-final-obstruction)

adversary-initial-state $\leq$

trace-fri-zero-round-agreement-set-target-error-for budgets +

trace-fri-controlled-merkle-error-for budgets +

(*trace-fri-controlled-merkle-error-for budgets* +

((*reachable-verifier-event-bound-for A*

trace-fri-header-tied-low-candidate-nonconsumed-query-gap +

reachable-verifier-event-bound-for A

trace-fri-header-tied-low-candidate-nonconsumed-irrelevant-root-gap) +

reachable-verifier-event-bound-for A

trace-fri-header-tied-low-candidate-nonconsumed-query-gap)) +

H

proof –

have *checked*:

wp-event (*checked-staged-security-experiment-with-data-state A*)

(*staged-security-with-data-state-verifier-event*

trace-fri-zero-round-checked-final-obstruction)

adversary-initial-state $\leq$

trace-fri-zero-round-agreement-set-target-error-for budgets

by (rule

checked-staged-security-trace-fri-zero-round-checked-final-obstruction-bound-from-agreement-sets

[*OF wf controlled*])

have *merkle*:

wp-event (*checked-staged-security-experiment-with-data-state A*)

(*staged-security-with-data-state-verifier-event*

partial-merkle-inconsistency-bad)

adversary-initial-state $\leq$

trace-fri-controlled-merkle-error-for budgets

unfolding *trace-fri-controlled-merkle-error-for-def*

by (rule

checked-staged-security-with-data-state-partial-merkle-inconsistency-bad-bound-controlled

[OF wf controlled])

have openings:

wp-event (*checked-staged-security-experiment-with-data-state A*)
 (*staged-security-with-data-state-verifier-event*
trace-fri-zero-round-verifier-consumed-low-candidate-gap)
adversary-initial-state $\leq$
trace-fri-controlled-merkle-error-for budgets +
 ((*reachable-verifier-event-bound-for A*
trace-fri-header-tied-low-candidate-nonconsumed-query-gap +
reachable-verifier-event-bound-for A
trace-fri-header-tied-low-candidate-nonconsumed-irrelevant-root-gap) +
reachable-verifier-event-bound-for A
trace-fri-header-tied-low-candidate-nonconsumed-query-gap)
by (rule
checked-staged-security-trace-fri-zero-round-verifier-consumed-low-candidate-gap-bound-from-hash-and-nonnorm
 [OF wf controlled])

have zero:

wp-event (*checked-staged-security-experiment-with-data-state A*)
 (*staged-security-with-data-state-verifier-event*
trace-fri-zero-round-final-obstruction)
adversary-initial-state $\leq$
trace-fri-zero-round-agreement-set-target-error-for budgets +
trace-fri-controlled-merkle-error-for budgets +
 (*trace-fri-controlled-merkle-error-for budgets* +
 ((*reachable-verifier-event-bound-for A*
trace-fri-header-tied-low-candidate-nonconsumed-query-gap +
reachable-verifier-event-bound-for A
trace-fri-header-tied-low-candidate-nonconsumed-irrelevant-root-gap) +
reachable-verifier-event-bound-for A
trace-fri-header-tied-low-candidate-nonconsumed-query-gap))) +
H
by (rule
checked-staged-security-trace-fri-zero-round-final-obstruction-bound-from-staged-checked-merkle-low-consumed-he
 [OF checked merkle openings header-bound])

show ?thesis
by (rule zero)

qed

definition *trace-empty-header-sampled-split-one-step-error-for*

where

trace-empty-header-sampled-split-one-step-error-for budgets A empty-bad =
trace-one-step-sampled-error-for budgets empty-bad +
 (*trace-fri-empty-sampled-assignment-structural-error-for budgets A* +
trace-empty-header-openings-zero-error-for budgets A empty-bad)

definition *concrete-trace-empty-header-sampled-split-one-step-error-for*

where

concrete-trace-empty-header-sampled-split-one-step-error-for budgets A =
trace-empty-header-sampled-split-one-step-error-for budgets A

trace-one-step-split-bad

lemma *trace-empty-header-sampled-split-one-step-error-for-bound:*

fixes *empty-bad* :: nat $\Rightarrow$ 'f list $\Rightarrow$ 'f set

assumes *wf*: *staged-budget-wellformed budgets*

and *controlled*: *staged-adversary-controlled budgets A*

and *empty-subset*:

$\bigwedge i$ *prefix*.

$i < \text{fri-round-count-for-degree-bound } (\text{clength} - 1) \Rightarrow$

$\text{length prefix} = i \Rightarrow$

$\text{empty-bad } i \text{ prefix} \subseteq$

$\text{fri-one-step-disagreement-challenges}$

$(\text{trace-fri-canonical-layer-len } i) (\text{empty-pw } i)$

$(\text{empty-layer } i \text{ prefix}) (\text{empty-claimed } i \text{ prefix})$

$(\text{empty-dom } i \text{ prefix})$

shows

$\text{wp-event } (\text{checked-staged-security-experiment-with-data-state } A)$

$(\text{staged-security-with-data-state-verifier-event}$

$\text{trace-fri-bad-with-empty-composition-header-candidates})$

$\text{adversary-initial-state}$

$\leq \text{trace-empty-header-sampled-split-one-step-error-for budgets } A \text{ empty-bad}$

proof –

have *sampled*:

$\text{wp-event } (\text{checked-staged-security-experiment-with-data-state } A)$

$(\text{staged-security-with-data-state-verifier-event}$

$\text{trace-fri-sampled-query-bad-candidate})$

$\text{adversary-initial-state}$

$\leq \text{trace-one-step-sampled-error-for budgets empty-bad}$

by (rule *trace-one-step-sampled-error-for-bound*

[OF *wf controlled empty-subset*])

have *conflict-bound*:

$\text{wp-event } (\text{checked-staged-security-experiment-with-data-state } A)$

$(\text{staged-security-with-data-state-verifier-event}$

$\text{trace-fri-sampled-assignment-conflict})$

$\text{adversary-initial-state}$

$\leq \text{trace-fri-empty-sampled-assignment-structural-error-for budgets } A$

by (rule

checked-staged-security-trace-fri-sampled-assignment-conflict-bound-structural

[OF *wf controlled*])

have *header-bound*:

$\text{wp-event } (\text{checked-staged-security-experiment-with-data-state } A)$

$(\text{staged-security-with-data-state-verifier-event}$

$\text{trace-fri-bad-with-header-tied-partial-candidate})$

$\text{adversary-initial-state}$

$\leq \text{trace-header-one-step-error-for budgets } A \text{ empty-bad}$

by (rule *trace-header-one-step-error-for-bound*

[OF *wf controlled empty-subset*])

have *zero-bound*:

$\text{wp-event } (\text{checked-staged-security-experiment-with-data-state } A)$

(staged-security-with-data-state-verifier-event
 trace-fri-zero-round-final-obstruction)
 adversary-initial-state
 $\leq$ trace-empty-header-openings-zero-error-for budgets A empty-bad
unfolding trace-empty-header-openings-zero-error-for-def
by (rule
 checked-staged-security-trace-fri-zero-round-final-obstruction-bound-from-agreement-sets-header-and-openings
 [OF wf controlled header-bound])
have branch:
 wp-event (checked-staged-security-experiment-with-data-state A)
 (staged-security-with-data-state-verifier-event
 trace-fri-bad-with-empty-composition-header-candidates)
 adversary-initial-state
 $\leq$ trace-one-step-sampled-error-for budgets empty-bad +
 (trace-fri-empty-sampled-assignment-structural-error-for budgets A +
 trace-empty-header-openings-zero-error-for budgets A empty-bad)
by (rule
 checked-staged-security-trace-fri-empty-header-bound-from-staged-sampled-conflict-and-zero
 [OF sampled conflict-bound zero-bound])
show ?thesis
using branch
by (simp add: trace-empty-header-sampled-split-one-step-error-for-def)
qed

theorem stark-soundness-from-current-public-prefix-current-query-relevant-drift-route-from-sampled-split-one-step-fr
fixes trace-bad empty-bad :: nat $\Rightarrow$ 'f list $\Rightarrow$ 'f set
and comp-bad :: 'f $\Rightarrow$ nat $\Rightarrow$ 'f list $\Rightarrow$ 'f set
assumes false-statement: $\neg$ exists-valid-trace
and wf: staged-budget-wellformed budgets
and controlled: staged-adversary-controlled budgets A
and trace-subset:
 $\bigwedge i$ prefix.
 $i < \text{fri-round-count-for-degree-bound } (\text{clength} - 1) \implies$
 $\text{length prefix} = i \implies$
 $\text{trace-bad } i \text{ prefix} \subseteq$
 fri-one-step-disagreement-challenges
 (trace-fri-canonical-layer-len i) (empty-pw i)
 (empty-layer i prefix) (empty-claimed i prefix)
 (empty-dom i prefix)
and comp-subset:
 $\bigwedge dg$ i prefix.
 $i < \text{fri-round-count-for-degree-bound } (\text{to-nat } dg) \implies$
 $\text{length prefix} = i \implies$
 $\text{comp-bad } dg \ i \text{ prefix} \subseteq$
 fri-one-step-disagreement-challenges
 (composition-fri-canonical-layer-len dg i) (comp-pw dg i)
 (comp-layer dg i prefix) (comp-claimed dg i prefix)
 (comp-dom dg i prefix)
and empty-subset:

$\wedge i \text{ prefix.}$
 $i < \text{fri-round-count-for-degree-bound } (\text{clength} - 1) \implies$
 $\text{length prefix} = i \implies$
 $\text{empty-bad } i \text{ prefix} \subseteq$
 $\text{fri-one-step-disagreement-challenges}$
 $(\text{trace-fri-canonical-layer-len } i) (\text{empty-pw } i)$
 $(\text{empty-layer } i \text{ prefix}) (\text{empty-claimed } i \text{ prefix})$
 $(\text{empty-dom } i \text{ prefix})$
shows $\text{checked-staged-adversary-acceptance-probability } A \leq$
 $\text{hash-collision-budget-value } 0$
 $(\text{staged-attacker-query-budget budgets} +$
 $\text{staged-challenge-query-budget} + \text{verifier-hash-query-budget}) +$
 $(\text{composition-error-bound} +$
 $\text{staged-phase-relation-error size}$
 $(\text{staged-alpha-search-queries budgets } 0) +$
 $\text{hash-collision-budget-value } 0$
 $(\text{staged-alpha-search-queries budgets } 0) +$
 $\text{staged-phase-relation-error size}$
 $(\text{staged-alpha-search-queries budgets } 0) +$
 $(\text{active-route-transcript-pre-error-for } A +$
 $\text{staged-concrete-transcript-target-error-bound})) +$
 $(\sum i < \text{rounds.}$
 $\text{active-route-current-prefix-authenticated-error-for } A \ i +$
 $(\text{active-route-transcript-pre-error-for } A +$
 $\text{staged-concrete-transcript-target-error-bound}) +$
 $(\text{active-route-transcript-pre-error-for } A +$
 $\text{staged-concrete-transcript-target-error-bound})) +$
 $\text{trace-header-one-step-error-for budgets } A \text{ trace-bad} +$
 $\text{composition-one-step-verifier-tied-controlled-error-for budgets}$
 $\text{comp-bad} +$
 $(\text{composition-error-bound} +$
 $\text{staged-phase-relation-error size}$
 $(\text{staged-alpha-search-queries budgets } 0) +$
 $\text{hash-collision-budget-value } 0$
 $(\text{staged-alpha-search-queries budgets } 0) +$
 $\text{staged-phase-relation-error size}$
 $(\text{staged-alpha-search-queries budgets } 0) +$
 $(\text{active-route-transcript-pre-error-for } A +$
 $\text{staged-concrete-transcript-target-error-bound}) +$
 $\text{trace-empty-header-sampled-split-one-step-error-for budgets } A \text{ empty-bad} +$
 $\text{active-route-current-empty-query-error-for } A)$
proof –
have trace-bound:
 $\text{wp-event } (\text{checked-staged-security-experiment-with-data-state } A)$
 $(\text{staged-security-with-data-state-verifier-event}$
 $\text{trace-fri-bad-with-header-tied-partial-candidate})$
 $\text{adversary-initial-state}$
 $\leq \text{trace-header-one-step-error-for budgets } A \text{ trace-bad}$
by $(\text{rule trace-header-one-step-error-for-bound})$

[OF wf controlled trace-subset])
have comp-bound:
 wp-event (checked-staged-security-experiment-with-data-state A)
 (staged-security-with-data-state-verifier-event
 composition-fri-bad-with-verifier-tied-partial-candidate)
 adversary-initial-state
 $\leq$ composition-one-step-verifier-tied-controlled-error-for budgets
 comp-bad
by (rule composition-one-step-verifier-tied-controlled-error-for-bound
 [OF wf controlled comp-subset])
have empty-bound:
 wp-event (checked-staged-security-experiment-with-data-state A)
 (staged-security-with-data-state-verifier-event
 trace-fri-bad-with-empty-composition-header-candidates)
 adversary-initial-state
 $\leq$ trace-empty-header-sampled-split-one-step-error-for budgets A empty-bad
by (rule trace-empty-header-sampled-split-one-step-error-for-bound
 [OF wf controlled empty-subset])
show ?thesis
by (rule
 stark-soundness-from-current-public-prefix-current-query-relevant-drift-route-from-active-fri-bounds
 [OF false-statement wf controlled trace-bound comp-bound empty-bound])
qed

theorem stark-soundness-from-current-public-prefix-current-query-relevant-drift-route-sampled-split-endpoint:

assumes false-statement: $\neg$ exists-valid-trace
and wf: staged-budget-wellformed budgets
and controlled: staged-adversary-controlled budgets A
shows checked-staged-adversary-acceptance-probability A $\leq$
 hash-collision-budget-value 0
 (staged-attacker-query-budget budgets +
 staged-challenge-query-budget + verifier-hash-query-budget) +
 (composition-error-bound +
 staged-phase-relation-error size
 (staged-alpha-search-queries budgets 0) +
 hash-collision-budget-value 0
 (staged-alpha-search-queries budgets 0) +
 staged-phase-relation-error size
 (staged-alpha-search-queries budgets 0) +
 (active-route-transcript-pre-error-for A +
 staged-concrete-transcript-target-error-bound)) +
 ($\sum$ i < rounds.
 active-route-current-prefix-authenticated-error-for A i +
 (active-route-transcript-pre-error-for A +
 staged-concrete-transcript-target-error-bound) +
 (active-route-transcript-pre-error-for A +
 staged-concrete-transcript-target-error-bound)) +
 concrete-trace-header-one-step-error-for budgets A +
 concrete-composition-one-step-error-for budgets A +

(composition-error-bound +
 staged-phase-relation-error size
 (staged-alpha-search-queries budgets 0) +
 hash-collision-budget-value 0
 (staged-alpha-search-queries budgets 0) +
 staged-phase-relation-error size
 (staged-alpha-search-queries budgets 0) +
 (active-route-transcript-pre-error-for A +
 staged-concrete-transcript-target-error-bound) +
 concrete-trace-empty-header-sampled-split-one-step-error-for budgets A +
 active-route-current-empty-query-error-for A)
unfolding concrete-trace-header-one-step-error-for-def
 concrete-composition-one-step-error-for-def
 concrete-trace-empty-header-sampled-split-one-step-error-for-def
by (rule
 stark-soundness-from-current-public-prefix-current-query-relevant-drift-route-from-sampled-split-one-step-fri-comp
 [where trace-bad = trace-one-step-split-bad
 and comp-bad = composition-one-step-split-bad
 and empty-bad = trace-one-step-split-bad
 and empty-pw = trace-one-step-split-pw
 and empty-layer = trace-one-step-split-layer
 and empty-claimed = trace-one-step-split-claimed
 and empty-dom = trace-one-step-split-domain
 and comp-pw = composition-one-step-split-pw
 and comp-layer = composition-one-step-split-layer
 and comp-claimed = composition-one-step-split-claimed
 and comp-dom = composition-one-step-split-domain,
 OF false-statement wf controlled])
 (simp-all add:
 trace-one-step-split-bad-def
 composition-one-step-split-bad-def
 trace-one-step-disagreement-bad-for-def
 composition-one-step-disagreement-bad-for-def))

definition active-route-current-prefix-conceptual-residual-error-for
where

active-route-current-prefix-conceptual-residual-error-for
 (A :: 'f staged-adversary) (i :: nat) =
 wp-event
 (checked-staged-security-experiment-with-query-prefix-data-state A i)
 checked-staged-security-with-query-prefix-conceptual-context-residual-at
 adversary-initial-state

definition active-route-current-prefix-conceptual-hit-residual-error-for
where

active-route-current-prefix-conceptual-hit-residual-error-for
 (A :: 'f staged-adversary) (i :: nat) =
 wp-event
 (checked-staged-security-experiment-with-query-prefix-data-state A i)

$(\lambda out.$
 $\quad checked_staged_security_with_query_prefix_current_prefix_authenticated_hit_at$
 $\quad\quad i \ out \wedge$
 $\quad checked_staged_security_with_query_prefix_conceptual_context_residual_at$
 $\quad\quad out)$
 $adversary_initial_state$

definition *active-route-current-prefix-conceptual-bound-for*
where

$active_route_current_prefix_conceptual_bound_for$
 $(budgets :: staged_budgets) (A :: 'f staged_adversary)$
 $(i :: nat) =$
 $staged_phase_relation_error \ size$
 $(staged_query_search_queries \ budgets \ i + 1) +$
 $query_error_bound +$
 $active_route_current_prefix_conceptual_hit_residual_error_for \ A \ i$

lemma *active-route-current-prefix-authenticated-error-for-le-conceptual-bound:*

assumes $wf: staged_budget_wellformed \ budgets$
and $controlled: staged_adversary_controlled \ budgets \ A$
and $i_bound: i < rounds$

shows

$active_route_current_prefix_authenticated_error_for \ A \ i \leq$
 $active_route_current_prefix_conceptual_bound_for \ budgets \ A \ i$

proof —

let $?M = checked_staged_security_experiment_with_query_prefix_data_state \ A \ i$

let $?Hit =$

$checked_staged_security_with_query_prefix_current_prefix_authenticated_hit_at$
 i

let $?Conceptual =$

$checked_staged_security_with_query_prefix_conceptual_target_hit$

let $?Residual =$

$\lambda out. ?Hit \ out \wedge$
 $checked_staged_security_with_query_prefix_conceptual_context_residual_at$
 out

have *event-le:*

$wp_event \ ?M \ ?Hit \ adversary_initial_state \leq$
 $wp_event \ ?M \ (\lambda out. ?Conceptual \ out \vee ?Residual \ out)$
 $adversary_initial_state$

proof (*rule wp-event-mono-on-support*)

fix out

assume $support: out \in set_dist \ (execute \ ?M \ adversary_initial_state)$

and $hit: ?Hit \ out$

from $checked_staged_security_with_query_prefix_current_prefix_authenticated_hit_imp_conceptual_or_residual_on_supp$
 $[OF \ wf \ controlled \ i_bound \ support \ hit]$

show $?Conceptual \ out \vee ?Residual \ out$

using hit **by** *blast*

qed

have *conceptual-bound:*

```

wp-event ?M ?Conceptual adversary-initial-state ≤
  staged-phase-relation-error size
  (staged-query-search-queries budgets i + 1) + query-error-bound
by (rule
  checked-staged-security-with-query-prefix-conceptual-target-hit-bound
  [OF wf controlled i-bound])
have union-bound:
  wp-event ?M (λout. ?Conceptual out ∨ ?Residual out)
    adversary-initial-state ≤
    wp-event ?M ?Conceptual adversary-initial-state +
    wp-event ?M ?Residual adversary-initial-state
  by (rule wp-event-union-bound)
have residual-bound:
  wp-event ?M ?Residual adversary-initial-state ≤
    active-route-current-prefix-conceptual-hit-residual-error-for A i
  by (simp add:
    active-route-current-prefix-conceptual-hit-residual-error-for-def)
have sum-bound:
  wp-event ?M ?Conceptual adversary-initial-state +
  wp-event ?M ?Residual adversary-initial-state ≤
    staged-phase-relation-error size
    (staged-query-search-queries budgets i + 1) +
    query-error-bound +
    active-route-current-prefix-conceptual-hit-residual-error-for A i
  by (intro add-mono conceptual-bound residual-bound)
show ?thesis
  unfolding active-route-current-prefix-authenticated-error-for-def
    active-route-current-prefix-conceptual-bound-for-def
  by (rule order-trans[OF event-le])
  (rule order-trans[OF union-bound sum-bound])
qed

definition active-route-current-empty-conceptual-residual-error-for
  :: 'f itself ⇒ 'f staged-adversary ⇒ prob
where
  active-route-current-empty-conceptual-residual-error-for
    (- :: 'f itself)
    (A :: 'f staged-adversary) =
  wp-event
    (checked-staged-security-experiment-with-query-prefix-data-state A 0)
    (checked-staged-security-with-query-prefix-conceptual-empty-context-residual-hit-at
      (0 :: nat))
    adversary-initial-state

definition active-route-current-empty-conceptual-bound-for
where
  active-route-current-empty-conceptual-bound-for
    (budgets :: staged-budgets) (A :: 'f staged-adversary) =
    staged-phase-relation-error size

```

$(\text{staged-query-search-queries budgets } 0 + 1) +$
 $\text{query-error-bound} +$
 $\text{active-route-current-empty-conceptual-residual-error-for TYPE('f) } A +$
 $(\text{active-route-transcript-pre-error-for } A +$
 $\text{staged-concrete-transcript-target-error-bound})$

lemma *active-route-current-empty-query-error-for-le-conceptual-bound:*

assumes *wf: staged-budget-wellformed budgets*

and *controlled: staged-adversary-controlled budgets A*

shows

$\text{active-route-current-empty-query-error-for } A \leq$
 $\text{active-route-current-empty-conceptual-bound-for budgets } A$

proof —

have *i-bound: 0 < rounds*

using *rounds-positive* **by** *simp*

have *residual-bound:*

wp-event

$(\text{checked-staged-security-experiment-with-query-prefix-data-state } A \ 0)$

$(\text{checked-staged-security-with-query-prefix-conceptual-empty-context-residual-hit-at}$
 $(0 :: \text{nat}))$

$\text{adversary-initial-state} \leq$

$\text{active-route-current-empty-conceptual-residual-error-for TYPE('f) } A$

unfolding *active-route-current-empty-conceptual-residual-error-for-def*

by $(\text{rule wp-event-mono})$ *simp*

have *pre-bound:*

wp-event (checked-staged-security-experiment-with-data-state A)

staged-security-with-data-state-transcript-pre-hit

$\text{adversary-initial-state} \leq \text{active-route-transcript-pre-error-for } A$

by $(\text{simp add: active-route-transcript-pre-error-for-def})$

have *new-bound:*

wp-event (checked-staged-security-experiment-with-data-state A)

staged-security-with-data-state-transcript-new-hit

$\text{adversary-initial-state} \leq \text{staged-concrete-transcript-target-error-bound}$

by $(\text{rule checked-staged-security-with-data-state-transcript-new-hit-bound}$
 $[\text{OF wf controlled}])$

show *?thesis*

unfolding *active-route-current-empty-query-error-for-def*

active-route-current-empty-conceptual-bound-for-def

by $(\text{rule}$

$\text{checked-staged-security-with-data-state-current-empty-query-opening-hit-at-bound-from-conceptual-empty-residual}$
 $[\text{OF wf controlled i-bound residual-bound pre-bound new-bound}])$

qed

definition *trace-no-full-cover-sampled-query-error-for*

where

trace-no-full-cover-sampled-query-error-for budgets bad =

$(\sum \text{challenges} \in \text{fri-challenge-space } (\text{ceil-log clength}).$

$(1 / \text{nnreal } (\text{CARD('f) } ^{\wedge} \text{ceil-log clength})) *$

$(\text{nnreal}$

$$\begin{aligned}
& (card \\
& \quad (trace-fri-sampled-query-no-full-cover-failure-query-fiber \\
& \quad \quad bad \ challenges)) * \\
& \quad (1 / nreal (card \ query-sample-space)) ^ \wedge rounds)) + \\
& hash-relation-budget-value \\
& \quad (query-index-row-list-relation-fiber-bound \\
& \quad \quad (\bigcup challenges \in fri-challenge-space \ (ceil-log \ clength). \\
& \quad \quad \quad trace-fri-sampled-query-no-full-cover-failure-query-fiber \\
& \quad \quad \quad \quad bad \ challenges)) \\
& \quad (staged-attacker-query-budget \ budgets + staged-challenge-query-budget) + \\
& hash-relation-budget-value \\
& \quad (card \ (fri-challenge-space \ (ceil-log \ clength)) * ceil-log \ clength) \\
& \quad (staged-attacker-query-budget \ budgets + staged-challenge-query-budget)
\end{aligned}$$

definition *trace-canonical-full-cover-shape-failure-error-for*
where

$$\begin{aligned}
& trace-canonical-full-cover-shape-failure-error-for \ A \ bad = \\
& \quad reachable-verifier-event-bound-for \ A \\
& \quad \quad (\lambda s. trace-fri-sampled-full-cover-proximity-failure \ s \ bad) + \\
& \quad reachable-verifier-event-bound-for \ A \\
& \quad \quad trace-fri-sampled-full-cover-index-failure + \\
& \quad reachable-verifier-event-bound-for \ A \\
& \quad \quad trace-fri-sampled-full-cover-shape-failure
\end{aligned}$$

definition *trace-restricted-sampled-query-error-for*
where

$$\begin{aligned}
& trace-restricted-sampled-query-error-for \ budgets \ A \ bad = \\
& \quad trace-no-full-cover-sampled-query-error-for \ budgets \ bad + \\
& \quad trace-canonical-full-cover-shape-failure-error-for \ A \ bad
\end{aligned}$$

lemma *trace-restricted-sampled-query-error-for-bound:*

assumes *wf*: staged-budget-wellformed budgets
and *controlled*: staged-adversary-controlled budgets A
shows

$$\begin{aligned}
& wp-event \ (checked-staged-security-experiment-with-data-state \ A) \\
& \quad (staged-security-with-data-state-verifier-event \\
& \quad \quad trace-fri-sampled-query-bad-candidate) \\
& \quad adversary-initial-state \\
& \leq trace-restricted-sampled-query-error-for \ budgets \ A \ bad
\end{aligned}$$

proof –

have *no-full*:

$$\begin{aligned}
& wp-event \ (checked-staged-security-experiment-with-data-state \ A) \\
& \quad (staged-security-with-data-state-verifier-event \\
& \quad \quad (\lambda s. trace-fri-sampled-query-no-full-cover-failure-candidate \ s \ bad)) \\
& \quad adversary-initial-state
\end{aligned}$$

$$\leq trace-no-full-cover-sampled-query-error-for \ budgets \ bad$$

unfolding *trace-no-full-cover-sampled-query-error-for-def*

by (rule

checked-staged-security-trace-fri-sampled-query-no-full-cover-failure-candidate-bound-from-restricted-pairs

```

    [OF wf controlled])
have failure:
  wp-event (checked-staged-security-experiment-with-data-state A)
    (staged-security-with-data-state-verifier-event
      (λs. trace-fri-sampled-canonical-full-cover-failure s bad))
    adversary-initial-state
  ≤ trace-canonical-full-cover-shape-failure-error-for A bad
unfolding trace-canonical-full-cover-shape-failure-error-for-def
proof (rule
  checked-staged-security-trace-fri-sampled-canonical-full-cover-failure-bound-from-shape-subcases)
fix data attacker-state
assume support:
  Some (data, attacker-state) ∈
    set-dist (execute (checked-staged-transcript-program A)
      adversary-initial-state)
show wp-event verify-monad
  (trace-fri-sampled-full-cover-proximity-failure
    (verifier-state-from-adversary attacker-state
      (staged-proof-transcript data)) bad)
  (verifier-state-from-adversary attacker-state
    (staged-proof-transcript data))
  ≤ reachable-verifier-event-bound-for A
  (λs. trace-fri-sampled-full-cover-proximity-failure s bad)
by (rule reachable-verifier-event-bound-for-ge[OF support])
next
fix data attacker-state
assume support:
  Some (data, attacker-state) ∈
    set-dist (execute (checked-staged-transcript-program A)
      adversary-initial-state)
show wp-event verify-monad
  (trace-fri-sampled-full-cover-index-failure
    (verifier-state-from-adversary attacker-state
      (staged-proof-transcript data)))
  (verifier-state-from-adversary attacker-state
    (staged-proof-transcript data))
  ≤ reachable-verifier-event-bound-for A
  trace-fri-sampled-full-cover-index-failure
by (rule reachable-verifier-event-bound-for-ge[OF support])
next
fix data attacker-state
assume support:
  Some (data, attacker-state) ∈
    set-dist (execute (checked-staged-transcript-program A)
      adversary-initial-state)
show wp-event verify-monad
  (trace-fri-sampled-full-cover-shape-failure
    (verifier-state-from-adversary attacker-state
      (staged-proof-transcript data)))

```

(verifier-state-from-adversary attacker-state
 (staged-proof-transcript data))
 $\leq$ reachable-verifier-event-bound-for A
 trace-fri-sampled-full-cover-shape-failure
 by (rule reachable-verifier-event-bound-for-ge[OF support])
 qed
 have sampled:
 wp-event (checked-staged-security-experiment-with-data-state A)
 (staged-security-with-data-state-verifier-event
 trace-fri-sampled-query-bad-candidate)
 adversary-initial-state
 $\leq$ trace-no-full-cover-sampled-query-error-for budgets bad +
 trace-canonical-full-cover-shape-failure-error-for A bad
 by (rule
 checked-staged-security-trace-fri-sampled-query-bad-candidate-bound-from-no-full-cover-or-canonical-failure
 [OF no-full failure])
 show ?thesis
 using sampled
 by (simp add: trace-restricted-sampled-query-error-for-def)
 qed

definition concrete-sampled-split-stark-soundness-bound-for
where

concrete-sampled-split-stark-soundness-bound-for budgets A =
 hash-collision-budget-value 0
 (staged-attacker-query-budget budgets +
 staged-challenge-query-budget + verifier-hash-query-budget) +
 (composition-error-bound +
 staged-phase-relation-error size
 (staged-alpha-search-queries budgets 0) +
 hash-collision-budget-value 0
 (staged-alpha-search-queries budgets 0) +
 staged-phase-relation-error size
 (staged-alpha-search-queries budgets 0) +
 (active-route-transcript-pre-error-for A +
 staged-concrete-transcript-target-error-bound)) +
 ($\sum i < \text{rounds.}$
 active-route-current-prefix-conceptual-bound-for budgets A i +
 (active-route-transcript-pre-error-for A +
 staged-concrete-transcript-target-error-bound) +
 (active-route-transcript-pre-error-for A +
 staged-concrete-transcript-target-error-bound)) +
 concrete-trace-header-one-step-error-for budgets A +
 concrete-composition-one-step-error-for budgets A +
 (composition-error-bound +
 staged-phase-relation-error size
 (staged-alpha-search-queries budgets 0) +
 hash-collision-budget-value 0
 (staged-alpha-search-queries budgets 0) +

staged-phase-relation-error size
(staged-alpha-search-queries budgets 0) +
(active-route-transcript-pre-error-for A +
staged-concrete-transcript-target-error-bound) +
concrete-trace-empty-header-sampled-split-one-step-error-for budgets A +
active-route-current-empty-conceptual-bound-for budgets A)

theorem *stark-soundness-trace-empty-sampled-split-endpoint:*

assumes *false-statement: $\neg$ exists-valid-trace*

and *wf: staged-budget-wellformed budgets*

and *controlled: staged-adversary-controlled budgets A*

shows *checked-staged-adversary-acceptance-probability A $\leq$*
concrete-sampled-split-stark-soundness-bound-for budgets A

proof –

have *old-bound:*

checked-staged-adversary-acceptance-probability A $\leq$
hash-collision-budget-value 0
(staged-attacker-query-budget budgets +
staged-challenge-query-budget + verifier-hash-query-budget) +
(composition-error-bound +
staged-phase-relation-error size
(staged-alpha-search-queries budgets 0) +
hash-collision-budget-value 0
(staged-alpha-search-queries budgets 0) +
staged-phase-relation-error size
(staged-alpha-search-queries budgets 0) +
(active-route-transcript-pre-error-for A +
staged-concrete-transcript-target-error-bound)) +
($\sum i < \text{rounds}.$
active-route-current-prefix-authenticated-error-for A i +
(active-route-transcript-pre-error-for A +
staged-concrete-transcript-target-error-bound) +
(active-route-transcript-pre-error-for A +
staged-concrete-transcript-target-error-bound)) +
concrete-trace-header-one-step-error-for budgets A +
concrete-composition-one-step-error-for budgets A +
(composition-error-bound +
staged-phase-relation-error size
(staged-alpha-search-queries budgets 0) +
hash-collision-budget-value 0
(staged-alpha-search-queries budgets 0) +
staged-phase-relation-error size
(staged-alpha-search-queries budgets 0) +
(active-route-transcript-pre-error-for A +
staged-concrete-transcript-target-error-bound) +
concrete-trace-empty-header-sampled-split-one-step-error-for budgets A +
active-route-current-empty-query-error-for A)

by (rule

stark-soundness-from-current-public-prefix-current-query-relevant-drift-route-sampled-split-endpoint

[OF false-statement wf controlled])

have *prefix-terms*:

($\sum i < \text{rounds}$.
 active-route-current-prefix-authenticated-error-for A i +
 (active-route-transcript-pre-error-for A +
 staged-concrete-transcript-target-error-bound) +
 (active-route-transcript-pre-error-for A +
 staged-concrete-transcript-target-error-bound))
 $\leq$
 ($\sum i < \text{rounds}$.
 active-route-current-prefix-conceptual-bound-for budgets A i +
 (active-route-transcript-pre-error-for A +
 staged-concrete-transcript-target-error-bound) +
 (active-route-transcript-pre-error-for A +
 staged-concrete-transcript-target-error-bound))
by (intro sum-mono add-mono order-refl
 active-route-current-prefix-authenticated-error-for-le-conceptual-bound
 [OF wf controlled])
 simp

have *empty-term*:

active-route-current-empty-query-error-for $A \leq$
 active-route-current-empty-conceptual-bound-for budgets A
by (rule active-route-current-empty-query-error-for-le-conceptual-bound
 [OF wf controlled])

have *old-le-new*:

hash-collision-budget-value 0
 (staged-attacker-query-budget budgets +
 staged-challenge-query-budget + verifier-hash-query-budget) +
 (composition-error-bound +
 staged-phase-relation-error size
 (staged-alpha-search-queries budgets 0) +
 hash-collision-budget-value 0
 (staged-alpha-search-queries budgets 0) +
 staged-phase-relation-error size
 (staged-alpha-search-queries budgets 0) +
 (active-route-transcript-pre-error-for A +
 staged-concrete-transcript-target-error-bound)) +
 ($\sum i < \text{rounds}$.
 active-route-current-prefix-authenticated-error-for A i +
 (active-route-transcript-pre-error-for A +
 staged-concrete-transcript-target-error-bound) +
 (active-route-transcript-pre-error-for A +
 staged-concrete-transcript-target-error-bound)) +
 concrete-trace-header-one-step-error-for budgets A +
 concrete-composition-one-step-error-for budgets A +
 (composition-error-bound +
 staged-phase-relation-error size
 (staged-alpha-search-queries budgets 0) +
 hash-collision-budget-value 0

```

    (staged-alpha-search-queries budgets 0) +
    staged-phase-relation-error size
    (staged-alpha-search-queries budgets 0) +
    (active-route-transcript-pre-error-for A +
     staged-concrete-transcript-target-error-bound) +
    concrete-trace-empty-header-sampled-split-one-step-error-for budgets A +
    active-route-current-empty-query-error-for A)
  ≤ concrete-sampled-split-stark-soundness-bound-for budgets A
unfolding concrete-sampled-split-stark-soundness-bound-for-def
by (intro add-mono order-refl prefix-terms empty-term)
show ?thesis
by (rule order-trans[OF old-bound old-le-new])
qed

end

hide-fact
  soundness.stark-soundness-trace-empty-sampled-split-endpoint
  soundness.stark-soundness-from-current-public-prefix-current-query-relevant-drift-route-sampled-split-endpoint

end

theory Soundness-FRI-Trace-Restricted-Query-Endpoint
  imports Soundness-FRI-Trace-Empty-Sampled-Assignment-Split
begin

Public entry point for the staged STARK soundness theorem. The theorem in this theory combines the staged adversary experiment, Merkle authentication reductions, query bounds, composition randomization bounds, and the FRI query/challenge accounting into one explicit probability bound.

The bound uses restricted trace and composition product-accounting terms and the proved zero-round trace empty-header estimates from the preceding FRI theories.

context soundness
begin

definition trace-header-restricted-one-step-error-for
where
  trace-header-restricted-one-step-error-for budgets A trace-bad =
    trace-restricted-sampled-query-error-for budgets A trace-bad +
    trace-fri-header-replay-conflict-zero-error-for budgets A

definition concrete-trace-header-restricted-one-step-error-for
where
  concrete-trace-header-restricted-one-step-error-for budgets A =
    trace-header-restricted-one-step-error-for budgets A
    (λ-. trace-one-step-split-bad)

```

lemma *trace-fri-header-tied-sampled-assignment-conflict-replay-bound:*
wp-event (checked-staged-security-experiment-with-data-state A)
(staged-security-with-data-state-verifier-event
trace-fri-header-tied-sampled-assignment-conflict)
adversary-initial-state
 $\leq$ *trace-fri-header-replay-conflict-error-for A*

proof (rule *checked-staged-security-with-data-state-verifier-event-bound-from-cont*)
fix *data attacker-state*
assume *support:*
Some (data, attacker-state) ∈
set-dist (execute (checked-staged-transcript-program A)
adversary-initial-state)
let *?s =*
verifier-state-from-adversary attacker-state
(staged-proof-transcript data)
let *?M =*
reachable-verifier-event-bound-for A partial-merkle-inconsistency-bad
let *?H =*
reachable-verifier-event-bound-for A
trace-fri-header-tied-recorded-sibling-candidate-missing
have *merkle-bound:*
wp-event verify-monad (partial-merkle-inconsistency-bad ?s) ?s
 $\leq$ *?M*
by (rule *reachable-verifier-event-bound-for-ge[OF support]*)
have *recorded-missing-bound:*
wp-event verify-monad
(trace-fri-header-tied-recorded-sibling-candidate-missing ?s) ?s
 $\leq$ *?H*
by (rule *reachable-verifier-event-bound-for-ge[OF support]*)
have *head-neq:*
wp-event verify-monad
(trace-fri-header-tied-base-head-value-neq-gap ?s) ?s ≤ ?M
by (rule
wp-trace-fri-header-tied-base-head-value-neq-gap-bound-from-partial-merkle)
(rule merkle-bound)
have *head-mismatch:*
wp-event verify-monad
(trace-fri-header-tied-base-head-value-mismatch-gap ?s) ?s
 $\leq$ *?M*
by (rule
wp-trace-fri-header-tied-base-head-value-mismatch-gap-bound-from-neq-gap
[OF head-neq])
have *alignment:*
wp-event verify-monad
(trace-fri-header-tied-base-value-alignment-gap ?s) ?s ≤ ?M
by (rule
wp-trace-fri-header-tied-base-value-alignment-gap-bound-from-head-value-mismatch-gap
[OF head-mismatch])
have *next-bound:*

wp-event verify-monad
 (trace-fri-header-tied-next-value-replay-gap ?s) ?s ≤ ?M + 0
by (rule
 wp-trace-fri-header-tied-next-value-replay-gap-bound-from-merkle-and-slot)
 (rule merkle-bound,
 rule wp-trace-fri-header-tied-authenticated-slot-recorded-chunk-gap-zero)
have successor:
wp-event verify-monad
 (trace-fri-header-tied-successor-replay-gap ?s) ?s ≤ ?M + 0
by (rule
 wp-trace-fri-header-tied-successor-replay-gap-bound-from-merkle-and-slot)
 (rule merkle-bound,
 rule wp-trace-fri-header-tied-authenticated-slot-recorded-chunk-gap-zero)
have final:
wp-event verify-monad
 (trace-fri-header-tied-final-value-replay-gap ?s) ?s ≤ ?M + 0 + 0
by (rule
 wp-trace-fri-header-tied-final-value-replay-gap-bound-from-merkle-slot-and-selected-step)
 (rule merkle-bound,
 rule wp-trace-fri-header-tied-authenticated-slot-recorded-chunk-gap-zero,
 rule wp-trace-fri-header-tied-final-value-selected-step-missing-gap-zero)
have slot:
wp-event verify-monad
 (trace-fri-header-tied-authenticated-slot-recorded-chunk-gap ?s) ?s
 ≤ 0
by (rule wp-trace-fri-header-tied-authenticated-slot-recorded-chunk-gap-zero)
have authenticated-recorded-auth:
wp-event verify-monad
 (trace-fri-header-tied-same-layer-authenticated-recorded-auth-gap ?s)
 ?s ≤ 0
by (rule
 wp-trace-fri-header-tied-same-layer-authenticated-recorded-auth-gap-bound-from-slot-recorded-chunk-gap
 [OF slot])
have recorded-auth:
wp-event verify-monad
 (trace-fri-header-tied-same-layer-recorded-auth-gap ?s) ?s ≤ 0
by (rule
 wp-trace-fri-header-tied-same-layer-recorded-auth-gap-bound-from-authenticated-recorded-auth-gap
 [OF authenticated-recorded-auth])
have same-auth:
wp-event verify-monad
 (trace-fri-header-tied-same-layer-auth-gap ?s) ?s ≤ 0
by (rule
 wp-trace-fri-header-tied-same-layer-auth-gap-bound-from-recorded-auth-gap
 [OF recorded-auth])
have auth-gap:
wp-event verify-monad
 (trace-fri-header-tied-assignment-auth-gap ?s) ?s ≤ 0
by (rule

$wp\text{-}trace\text{-}fri\text{-}header\text{-}tied\text{-}assignment\text{-}auth\text{-}gap\text{-}bound\text{-}from\text{-}same\text{-}layer\text{-}auth\text{-}gap$
 $[OF\ same\text{-}auth])$
have complement:
 $wp\text{-}event\ verify\text{-}monad$
 $(\lambda out.$
 $\quad trace\text{-}fri\text{-}header\text{-}tied\text{-}sampled\text{-}assignment\text{-}conflict\ ?s\ out \wedge$
 $\quad \neg\ trace\text{-}fri\text{-}header\text{-}tied\text{-}recorded\text{-}sibling\text{-}candidate\text{-}missing\ ?s\ out)$
 $\ ?s \leq$
 $\ (((?M + ?M) + (?M + 0) + (?M + 0) + (?M + 0 + 0)) + ?M) + 0)$
by (rule
 $wp\text{-}trace\text{-}fri\text{-}header\text{-}tied\text{-}sampled\text{-}assignment\text{-}conflict\text{-}without\text{-}recorded\text{-}missing\text{-}bound\text{-}from\text{-}auth\text{-}gap\text{-}and\text{-}replay\text{-}g$
 $(rule\ auth\text{-}gap, rule\ merkle\text{-}bound, rule\ alignment,$
 $rule\ next\text{-}bound, rule\ successor, rule\ final)$
show $wp\text{-}event\ verify\text{-}monad$
 $(trace\text{-}fri\text{-}header\text{-}tied\text{-}sampled\text{-}assignment\text{-}conflict\ ?s)\ ?s$
 $\leq trace\text{-}fri\text{-}header\text{-}replay\text{-}conflict\text{-}error\text{-}for\ A$
unfolding $trace\text{-}fri\text{-}header\text{-}replay\text{-}conflict\text{-}error\text{-}for\text{-}def\ Let\text{-}def$
by (rule
 $wp\text{-}trace\text{-}fri\text{-}header\text{-}tied\text{-}sampled\text{-}assignment\text{-}conflict\text{-}bound\text{-}from\text{-}recorded\text{-}missing\text{-}and\text{-}complement)$
 $(rule\ recorded\text{-}missing\text{-}bound, rule\ complement)$
next
fix s
show $\neg\ trace\text{-}fri\text{-}header\text{-}tied\text{-}sampled\text{-}assignment\text{-}conflict\ s\ None$
unfolding $trace\text{-}fri\text{-}header\text{-}tied\text{-}sampled\text{-}assignment\text{-}conflict\text{-}def$
 $accepted\text{-}fri\text{-}opening\text{-}transcript\text{-}def$
by $simp$
qed

lemma $trace\text{-}fri\text{-}header\text{-}tied\text{-}zero\text{-}round\text{-}final\text{-}obstruction\text{-}agreement\text{-}bound$:
assumes wf : $staged\text{-}budget\text{-}wellformed\ budgets$
and $controlled$: $staged\text{-}adversary\text{-}controlled\ budgets\ A$
shows
 $wp\text{-}event\ (checked\text{-}staged\text{-}security\text{-}experiment\text{-}with\text{-}data\text{-}state\ A)$
 $(staged\text{-}security\text{-}with\text{-}data\text{-}state\text{-}verifier\text{-}event$
 $\quad trace\text{-}fri\text{-}header\text{-}tied\text{-}zero\text{-}round\text{-}final\text{-}obstruction)$
 $adversary\text{-}initial\text{-}state$
 $\leq trace\text{-}fri\text{-}zero\text{-}round\text{-}agreement\text{-}set\text{-}target\text{-}error\text{-}for\ budgets +$
 $trace\text{-}fri\text{-}controlled\text{-}merkle\text{-}error\text{-}for\ budgets$
proof –
have $checked$:
 $wp\text{-}event\ (checked\text{-}staged\text{-}security\text{-}experiment\text{-}with\text{-}data\text{-}state\ A)$
 $(staged\text{-}security\text{-}with\text{-}data\text{-}state\text{-}verifier\text{-}event$
 $\quad trace\text{-}fri\text{-}zero\text{-}round\text{-}checked\text{-}final\text{-}obstruction)$
 $adversary\text{-}initial\text{-}state$
 $\leq trace\text{-}fri\text{-}zero\text{-}round\text{-}agreement\text{-}set\text{-}target\text{-}error\text{-}for\ budgets$
by (rule
 $checked\text{-}staged\text{-}security\text{-}trace\text{-}fri\text{-}zero\text{-}round\text{-}checked\text{-}final\text{-}obstruction\text{-}bound\text{-}from\text{-}agreement\text{-}sets$
 $[OF\ wf\ controlled])$
have $merkle$:

```

wp-event (checked-staged-security-experiment-with-data-state A)
  (staged-security-with-data-state-verifier-event
    partial-merkle-inconsistency-bad)
  adversary-initial-state
≤ trace-fri-controlled-merkle-error-for budgets
unfolding trace-fri-controlled-merkle-error-for-def
by (rule
  checked-staged-security-with-data-state-partial-merkle-inconsistency-bad-bound-controlled
  [OF wf controlled])
have split:
wp-event (checked-staged-security-experiment-with-data-state A)
  (staged-security-with-data-state-verifier-event
    trace-fri-header-tied-zero-round-final-obstruction)
  adversary-initial-state
≤ wp-event (checked-staged-security-experiment-with-data-state A)
  (λout.
    staged-security-with-data-state-verifier-event
      trace-fri-zero-round-checked-final-obstruction out ∨
    staged-security-with-data-state-verifier-event
      partial-merkle-inconsistency-bad out)
  adversary-initial-state
unfolding staged-security-with-data-state-verifier-event-def
by (rule wp-event-mono-on-support)
  (auto dest:
    trace-fri-header-tied-zero-round-final-obstruction-imp-checked-or-merkle
    split: option.splits prod.splits)
also have ... ≤
wp-event (checked-staged-security-experiment-with-data-state A)
  (staged-security-with-data-state-verifier-event
    trace-fri-zero-round-checked-final-obstruction)
  adversary-initial-state +
wp-event (checked-staged-security-experiment-with-data-state A)
  (staged-security-with-data-state-verifier-event
    partial-merkle-inconsistency-bad)
  adversary-initial-state
by (rule wp-event-union-bound)
also have ... ≤
  trace-fri-zero-round-agreement-set-target-error-for budgets +
  trace-fri-controlled-merkle-error-for budgets
by (rule add-mono[OF checked merkle])
finally show ?thesis .
qed

lemma trace-header-restricted-one-step-error-for-bound:
fixes trace-bad :: 'f list ⇒ nat ⇒ 'f list ⇒ 'f set
assumes wf: staged-budget-wellformed budgets
and controlled: staged-adversary-controlled budgets A
shows
  wp-event (checked-staged-security-experiment-with-data-state A)

```

(staged-security-with-data-state-verifier-event
 trace-fri-bad-with-header-tied-partial-candidate)
 adversary-initial-state
 $\leq$ trace-header-restricted-one-step-error-for budgets A trace-bad
proof –
have sampled-query:
 wp-event (checked-staged-security-experiment-with-data-state A)
 (staged-security-with-data-state-verifier-event
 trace-fri-sampled-query-bad-candidate)
 adversary-initial-state
 $\leq$ trace-restricted-sampled-query-error-for budgets A trace-bad
by (rule trace-restricted-sampled-query-error-for-bound[OF wf controlled])
have sampled-plain:
 wp-event (checked-staged-security-experiment-with-data-state A)
 (staged-security-with-data-state-verifier-event
 trace-fri-bad-with-sampled-layer-chain)
 adversary-initial-state
 $\leq$ trace-restricted-sampled-query-error-for budgets A trace-bad
by (rule
 checked-staged-security-trace-fri-sampled-layer-chain-bound-from-sampled-query
 [OF sampled-query])
have sampled-header:
 wp-event (checked-staged-security-experiment-with-data-state A)
 (staged-security-with-data-state-verifier-event
 trace-fri-bad-with-header-tied-sampled-layer-chain)
 adversary-initial-state
 $\leq$ trace-restricted-sampled-query-error-for budgets A trace-bad
proof (rule order-trans[OF - sampled-plain])
show wp-event (checked-staged-security-experiment-with-data-state A)
 (staged-security-with-data-state-verifier-event
 trace-fri-bad-with-header-tied-sampled-layer-chain)
 adversary-initial-state
 $\leq$ wp-event (checked-staged-security-experiment-with-data-state A)
 (staged-security-with-data-state-verifier-event
 trace-fri-bad-with-sampled-layer-chain)
 adversary-initial-state
by (rule wp-event-mono)
 (auto simp: staged-security-with-data-state-verifier-event-def
 dest: trace-fri-header-tied-sampled-layer-chain-imp-sampled-layer-chain
 split: option.splits prod.splits)
qed
have conflict-bound:
 wp-event (checked-staged-security-experiment-with-data-state A)
 (staged-security-with-data-state-verifier-event
 trace-fri-header-tied-sampled-assignment-conflict)
 adversary-initial-state
 $\leq$ trace-fri-header-replay-conflict-error-for A
by (rule trace-fri-header-tied-sampled-assignment-conflict-replay-bound)
have zero-bound:

$wp\text{-}event$ (*checked-staged-security-experiment-with-data-state* A)
 (*staged-security-with-data-state-verifier-event*
trace-fri-header-tied-zero-round-final-obstruction)
adversary-initial-state
 $\leq$ *trace-fri-zero-round-agreement-set-target-error-for budgets* +
trace-fri-controlled-merkle-error-for budgets
by (*rule*
trace-fri-header-tied-zero-round-final-obstruction-agreement-bound
 $[OF\ wf\ controlled]$)
have *branch*:
 $wp\text{-}event$ (*checked-staged-security-experiment-with-data-state* A)
 (*staged-security-with-data-state-verifier-event*
trace-fri-bad-with-header-tied-partial-candidate)
adversary-initial-state
 $\leq$ *trace-restricted-sampled-query-error-for budgets* A *trace-bad* +
 (*trace-fri-header-replay-conflict-error-for* A +
 (*trace-fri-zero-round-agreement-set-target-error-for budgets* +
trace-fri-controlled-merkle-error-for budgets))
by (*rule*
checked-staged-security-trace-fri-header-tied-partial-candidate-bound-from-sampled-conflict-and-staged-zero
 $[OF\ sampled\text{-}header\ conflict\text{-}bound\ zero\text{-}bound]$)
show *?thesis*
using *branch*
by (*simp add: trace-header-restricted-one-step-error-for-def*
trace-fri-header-replay-conflict-zero-error-for-def)
qed

definition *trace-empty-header-restricted-openings-zero-error-for*
where

trace-empty-header-restricted-openings-zero-error-for budgets A *empty-bad* =
 (*trace-fri-zero-round-agreement-set-target-error-for budgets* +
trace-fri-controlled-merkle-error-for budgets +
 (*trace-header-restricted-one-step-error-for budgets* A ($\lambda\cdot$. *empty-bad*) +
reachable-verifier-event-bound-for A
trace-fri-header-tied-low-candidate-untied-openings-gap) +
trace-header-restricted-one-step-error-for budgets A ($\lambda\cdot$. *empty-bad*))

definition *trace-empty-header-restricted-nonnormalized-zero-error-for*
where

trace-empty-header-restricted-nonnormalized-zero-error-for budgets A *empty-bad* =
 (*trace-fri-zero-round-agreement-set-target-error-for budgets* +
trace-fri-controlled-merkle-error-for budgets +
 (*trace-fri-controlled-merkle-error-for budgets* +
 ((*reachable-verifier-event-bound-for* A
trace-fri-header-tied-low-candidate-nonconsumed-query-gap +
reachable-verifier-event-bound-for A
trace-fri-header-tied-low-candidate-nonconsumed-irrelevant-root-gap) +
reachable-verifier-event-bound-for A
trace-fri-header-tied-low-candidate-nonconsumed-query-gap)) +

trace-header-restricted-one-step-error-for budgets A (λ-. empty-bad))

definition *trace-empty-header-restricted-sampled-split-one-step-error-for*
where

trace-empty-header-restricted-sampled-split-one-step-error-for budgets A empty-bad =
trace-restricted-sampled-query-error-for budgets A (λ-. empty-bad) +
(trace-fri-empty-sampled-assignment-structural-error-for budgets A +
min (trace-empty-header-restricted-openings-zero-error-for budgets A empty-bad)
(trace-empty-header-restricted-nonnormalized-zero-error-for budgets A empty-bad))

definition *concrete-trace-empty-header-restricted-sampled-split-one-step-error-for*
where

concrete-trace-empty-header-restricted-sampled-split-one-step-error-for budgets A =
trace-empty-header-restricted-sampled-split-one-step-error-for budgets A
trace-one-step-split-bad

lemma *trace-empty-header-restricted-sampled-split-one-step-error-for-bound:*

fixes *empty-bad :: nat ⇒ 'f list ⇒ 'f set*

assumes *wf: staged-budget-wellformed budgets*

and *controlled: staged-adversary-controlled budgets A*

shows

wp-event (checked-staged-security-experiment-with-data-state A)
(staged-security-with-data-state-verifier-event
trace-fri-bad-with-empty-composition-header-candidates)
adversary-initial-state
 $\leq$ *trace-empty-header-restricted-sampled-split-one-step-error-for*
budgets A empty-bad

proof –

have *sampled:*

wp-event (checked-staged-security-experiment-with-data-state A)
(staged-security-with-data-state-verifier-event
trace-fri-sampled-query-bad-candidate)
adversary-initial-state
 $\leq$ *trace-restricted-sampled-query-error-for budgets A (λ-. empty-bad)*
by (rule *trace-restricted-sampled-query-error-for-bound*[OF *wf controlled*])

have *conflict-bound:*

wp-event (checked-staged-security-experiment-with-data-state A)
(staged-security-with-data-state-verifier-event
trace-fri-sampled-assignment-conflict)
adversary-initial-state
 $\leq$ *trace-fri-empty-sampled-assignment-structural-error-for budgets A*
by (rule
checked-staged-security-trace-fri-sampled-assignment-conflict-bound-structural
[OF *wf controlled*])

have *header-bound:*

wp-event (checked-staged-security-experiment-with-data-state A)
(staged-security-with-data-state-verifier-event
trace-fri-bad-with-header-tied-partial-candidate)
adversary-initial-state

```

≤ trace-header-restricted-one-step-error-for budgets A (λ-. empty-bad)
by (rule trace-header-restricted-one-step-error-for-bound
    [OF wf controlled])
have zero-openings-bound:
  wp-event (checked-staged-security-experiment-with-data-state A)
    (staged-security-with-data-state-verifier-event
     trace-fri-zero-round-final-obstruction)
    adversary-initial-state
≤ trace-empty-header-restricted-openings-zero-error-for budgets A empty-bad
unfolding trace-empty-header-restricted-openings-zero-error-for-def
by (rule
    checked-staged-security-trace-fri-zero-round-final-obstruction-bound-from-agreement-sets-header-and-openings
    [OF wf controlled header-bound])
have zero-nonnormalized-bound:
  wp-event (checked-staged-security-experiment-with-data-state A)
    (staged-security-with-data-state-verifier-event
     trace-fri-zero-round-final-obstruction)
    adversary-initial-state
≤ trace-empty-header-restricted-nonnormalized-zero-error-for budgets A empty-bad
unfolding trace-empty-header-restricted-nonnormalized-zero-error-for-def
by (rule
    checked-staged-security-trace-fri-zero-round-final-obstruction-bound-from-agreement-sets-header-and-nonnormalized
    [OF wf controlled header-bound])
have zero-bound:
  wp-event (checked-staged-security-experiment-with-data-state A)
    (staged-security-with-data-state-verifier-event
     trace-fri-zero-round-final-obstruction)
    adversary-initial-state
≤ min (trace-empty-header-restricted-openings-zero-error-for budgets A empty-bad)
    (trace-empty-header-restricted-nonnormalized-zero-error-for budgets A empty-bad)
using zero-openings-bound zero-nonnormalized-bound by simp
have branch:
  wp-event (checked-staged-security-experiment-with-data-state A)
    (staged-security-with-data-state-verifier-event
     trace-fri-bad-with-empty-composition-header-candidates)
    adversary-initial-state
≤ trace-restricted-sampled-query-error-for budgets A (λ-. empty-bad) +
    (trace-fri-empty-sampled-assignment-structural-error-for budgets A +
     min (trace-empty-header-restricted-openings-zero-error-for budgets A empty-bad)
         (trace-empty-header-restricted-nonnormalized-zero-error-for budgets A empty-bad))
by (rule
    checked-staged-security-trace-fri-empty-header-bound-from-staged-sampled-conflict-and-zero
    [OF sampled conflict-bound zero-bound])
show ?thesis
using branch
by (simp add:
    trace-empty-header-restricted-sampled-split-one-step-error-for-def)
qed

```

definition *composition-no-full-cover-sampled-query-error-for*

where

$$\begin{aligned}
& \text{composition-no-full-cover-sampled-query-error-for budgets bad} = \\
& (\sum dg \in UNIV. \\
& \quad \sum \text{challenges} \in \text{fri-challenge-space } (\text{ceil-log } (\text{to-nat } dg + 1)). \\
& \quad (1 / \text{nnreal } (\text{CARD}('f) \wedge \text{ceil-log } (\text{to-nat } dg + 1))) * \\
& \quad (\text{nnreal} \\
& \quad \quad (\text{card} \\
& \quad \quad \quad (\text{composition-fri-sampled-query-no-full-cover-failure-query-fiber} \\
& \quad \quad \quad \quad \text{bad dg challenges})) * \\
& \quad \quad (1 / \text{nnreal } (\text{card query-sample-space})) \wedge \text{rounds})) + \\
& \text{hash-relation-budget-value} \\
& (\text{query-index-raw-list-relation-fiber-bound} \\
& \quad (\bigcup dg \in UNIV. \\
& \quad \quad \bigcup \text{challenges} \in \text{fri-challenge-space } (\text{ceil-log } (\text{to-nat } dg + 1)). \\
& \quad \quad \text{composition-fri-sampled-query-no-full-cover-failure-query-fiber} \\
& \quad \quad \quad \text{bad dg challenges})) \\
& \quad (\text{staged-attacker-query-budget budgets} + \text{staged-challenge-query-budget}) + \\
& \text{hash-relation-budget-value} \\
& (\sum dg \in (UNIV :: 'f \text{ set}). \\
& \quad \text{card } (\text{fri-challenge-space } (\text{ceil-log } (\text{to-nat } dg + 1))) * \\
& \quad \quad \text{ceil-log } (\text{maxDegree} + 1)) \\
& \quad (\text{staged-attacker-query-budget budgets} + \text{staged-challenge-query-budget})
\end{aligned}$$

definition *composition-canonical-full-cover-shape-failure-error-for*

where

$$\begin{aligned}
& \text{composition-canonical-full-cover-shape-failure-error-for } A \text{ bad} = \\
& \text{reachable-verifier-event-bound-for } A \\
& \quad (\lambda s. \text{composition-fri-sampled-full-cover-proximity-failure } s \text{ bad}) + \\
& \text{reachable-verifier-event-bound-for } A \\
& \quad \text{composition-fri-sampled-full-cover-index-failure} + \\
& \text{reachable-verifier-event-bound-for } A \\
& \quad \text{composition-fri-sampled-full-cover-shape-failure}
\end{aligned}$$

definition *composition-restricted-sampled-query-error-for*

where

$$\begin{aligned}
& \text{composition-restricted-sampled-query-error-for budgets } A \text{ bad} = \\
& \text{composition-no-full-cover-sampled-query-error-for budgets bad} + \\
& \text{composition-canonical-full-cover-shape-failure-error-for } A \text{ bad}
\end{aligned}$$

lemma *composition-restricted-sampled-query-error-for-bound:*

assumes *wf: staged-budget-wellformed budgets*

and controlled: *staged-adversary-controlled budgets A*

shows

$$\begin{aligned}
& \text{wp-event } (\text{checked-staged-security-experiment-with-data-state } A) \\
& \quad (\text{staged-security-with-data-state-verifier-event} \\
& \quad \quad \text{composition-fri-sampled-query-bad-candidate}) \\
& \quad \text{adversary-initial-state} \\
& \leq \text{composition-restricted-sampled-query-error-for budgets } A \text{ bad}
\end{aligned}$$

```

proof –
  have no-full:
    wp-event (checked-staged-security-experiment-with-data-state A)
      (staged-security-with-data-state-verifier-event
        ( $\lambda s.$ 
          composition-fri-sampled-query-no-full-cover-failure-candidate s
            bad))
      adversary-initial-state
     $\leq$  composition-no-full-cover-sampled-query-error-for budgets bad
  unfolding composition-no-full-cover-sampled-query-error-for-def
  by (rule
    checked-staged-security-composition-fri-sampled-query-no-full-cover-failure-candidate-bound-from-restricted-pairs
      [OF wf controlled])
  have failure:
    wp-event (checked-staged-security-experiment-with-data-state A)
      (staged-security-with-data-state-verifier-event
        ( $\lambda s.$  composition-fri-sampled-canonical-full-cover-failure s bad))
      adversary-initial-state
     $\leq$  composition-canonical-full-cover-shape-failure-error-for A bad
  unfolding composition-canonical-full-cover-shape-failure-error-for-def
  proof (rule
    checked-staged-security-composition-fri-sampled-canonical-full-cover-failure-bound-from-shape-subcases)
  fix data attacker-state
  assume support:
    Some (data, attacker-state)  $\in$ 
      set-dist (execute (checked-staged-transcript-program A)
        adversary-initial-state)
  show wp-event verify-monad
    (composition-fri-sampled-full-cover-proximity-failure
      (verifier-state-from-adversary attacker-state
        (staged-proof-transcript data)) bad)
    (verifier-state-from-adversary attacker-state
      (staged-proof-transcript data))
     $\leq$  reachable-verifier-event-bound-for A
    ( $\lambda s.$  composition-fri-sampled-full-cover-proximity-failure s bad)
  by (rule reachable-verifier-event-bound-for-ge[OF support])
  next
  fix data attacker-state
  assume support:
    Some (data, attacker-state)  $\in$ 
      set-dist (execute (checked-staged-transcript-program A)
        adversary-initial-state)
  show wp-event verify-monad
    (composition-fri-sampled-full-cover-index-failure
      (verifier-state-from-adversary attacker-state
        (staged-proof-transcript data))))
    (verifier-state-from-adversary attacker-state
      (staged-proof-transcript data))
     $\leq$  reachable-verifier-event-bound-for A

```

```

    composition-fri-sampled-full-cover-index-failure
  by (rule reachable-verifier-event-bound-for-ge[OF support])
next
fix data attacker-state
assume support:
  Some (data, attacker-state) ∈
    set-dist (execute (checked-staged-transcript-program A)
      adversary-initial-state)
show wp-event verify-monad
  (composition-fri-sampled-full-cover-shape-failure
    (verifier-state-from-adversary attacker-state
      (staged-proof-transcript data)))
  (verifier-state-from-adversary attacker-state
    (staged-proof-transcript data))
≤ reachable-verifier-event-bound-for A
  composition-fri-sampled-full-cover-shape-failure
  by (rule reachable-verifier-event-bound-for-ge[OF support])
qed
have sampled:
  wp-event (checked-staged-security-experiment-with-data-state A)
    (staged-security-with-data-state-verifier-event
      composition-fri-sampled-query-bad-candidate)
    adversary-initial-state
≤ composition-no-full-cover-sampled-query-error-for budgets bad +
  composition-canonical-full-cover-shape-failure-error-for A bad
  by (rule
    checked-staged-security-composition-fri-sampled-query-bad-candidate-bound-from-no-full-cover-or-canonical-failure
      [OF no-full failure])
show ?thesis
  using sampled
  by (simp add: composition-restricted-sampled-query-error-for-def)
qed

definition composition-restricted-verifier-tied-controlled-error-for
where
  composition-restricted-verifier-tied-controlled-error-for budgets A comp-bad =
    composition-restricted-sampled-query-error-for budgets A comp-bad +
    (((composition-one-step-controlled-merkle-error-for budgets + 0) +
      (composition-one-step-controlled-merkle-error-for budgets + 0) +
      (composition-one-step-controlled-merkle-error-for budgets + 0) +
      (composition-one-step-controlled-merkle-error-for budgets + 0 + 0) +
      composition-one-step-controlled-merkle-error-for budgets) + 0 + 0)

definition concrete-composition-restricted-one-step-error-for
where
  concrete-composition-restricted-one-step-error-for budgets A =
    composition-restricted-verifier-tied-controlled-error-for budgets A
    (λdg -. composition-one-step-split-bad dg)

```

lemma *composition-restricted-verifier-tied-controlled-error-for-bound:*

assumes *wf: staged-budget-wellformed budgets*

and *controlled: staged-adversary-controlled budgets A*

shows

wp-event (checked-staged-security-experiment-with-data-state A)
(staged-security-with-data-state-verifier-event
composition-fri-bad-with-verifier-tied-partial-candidate)
adversary-initial-state
 $\leq$ *composition-restricted-verifier-tied-controlled-error-for budgets A*
comp-bad

proof —

have *sampled-query:*

wp-event (checked-staged-security-experiment-with-data-state A)
(staged-security-with-data-state-verifier-event
composition-fri-sampled-query-bad-candidate)
adversary-initial-state
 $\leq$ *composition-restricted-sampled-query-error-for budgets A comp-bad*
by (rule *composition-restricted-sampled-query-error-for-bound*
 $[OF\ wf\ controlled]$)

have *sampled:*

wp-event (checked-staged-security-experiment-with-data-state A)
(staged-security-with-data-state-verifier-event
composition-fri-bad-with-verifier-tied-sampled-layer-chain)
adversary-initial-state
 $\leq$ *composition-restricted-sampled-query-error-for budgets A comp-bad*
by (rule
checked-staged-security-composition-fri-verifier-tied-sampled-layer-chain-bound-from-sampled-query
 $[OF\ sampled-query]$)

have *conflict:*

wp-event (checked-staged-security-experiment-with-data-state A)
(staged-security-with-data-state-verifier-event
composition-fri-verifier-tied-sampled-assignment-conflict)
adversary-initial-state $\leq$
 $(((composition-one-step-controlled-merkle-error-for\ budgets + 0) +$
 $(composition-one-step-controlled-merkle-error-for\ budgets + 0) +$
 $(composition-one-step-controlled-merkle-error-for\ budgets + 0) +$
 $(composition-one-step-controlled-merkle-error-for\ budgets + 0 + 0) +$
 $composition-one-step-controlled-merkle-error-for\ budgets) + 0)$
by (rule
checked-staged-security-composition-fri-verifier-tied-sampled-assignment-conflict-bound-by-controlled-merkle
 $[OF\ wf\ controlled]$)

have *zero:*

wp-event (checked-staged-security-experiment-with-data-state A)
(staged-security-with-data-state-verifier-event
composition-fri-verifier-tied-zero-round-final-obstruction)
adversary-initial-state $\leq (0::prob)$
by (rule *checked-staged-security-with-data-state-verifier-event-bound-from-cont*)
 $(simp-all\ add:$
composition-fri-verifier-tied-zero-round-final-obstruction-false

$wp_composition_fri_verifier_tied_zero_round_final_obstruction_zero$)
have *branch*:
 wp_event ($checked_staged_security_experiment_with_data_state$ A)
 $(staged_security_with_data_state_verifier_event$
 $composition_fri_bad_with_verifier_tied_partial_candidate)$
 $adversary_initial_state \leq$
 $composition_restricted_sampled_query_error_for$ $budgets$ A $comp_bad$ $+$
 $((((composition_one_step_controlled_merkle_error_for$ $budgets$ $+$ 0) $+$
 $(composition_one_step_controlled_merkle_error_for$ $budgets$ $+$ 0) $+$
 $(composition_one_step_controlled_merkle_error_for$ $budgets$ $+$ 0) $+$
 $(composition_one_step_controlled_merkle_error_for$ $budgets$ $+$ 0 $+$ 0) $+$
 $composition_one_step_controlled_merkle_error_for$ $budgets$) $+$ 0) $+$ 0)
by (*rule*
 $checked_staged_security_composition_fri_verifier_tied_partial_candidate_bound_from_staged_sampled_conflict_and_zen$
 $[OF$ $sampled$ $conflict$ $zero]$)
show *?thesis*
using *branch*
by (*simp* *add*:
 $composition_restricted_verifier_tied_controlled_error_for_def$)
qed

definition *active-route-current-prefix-conceptual-prefix-collision-hit-at*
where

$active_route_current_prefix_conceptual_prefix_collision_hit_at$ i out
 $\longleftrightarrow$
 $checked_staged_security_with_query_prefix_current_prefix_authenticated_hit_at$
 i $out \wedge$
 $(case$ out of
 $None \Rightarrow False$
 $| Some$ $(((((prefix, prefix_state), raw), raw_state),$
 $((data, attacker_state), result)), final_state) \Rightarrow$
 $hash_map_output_collision$ $prefix_state)$

definition *active-route-current-prefix-conceptual-algebraic-residual-hit-at*
where

$active_route_current_prefix_conceptual_algebraic_residual_hit_at$ i out
 $\longleftrightarrow$
 $checked_staged_security_with_query_prefix_current_prefix_authenticated_hit_at$
 i $out \wedge$
 $(case$ out of
 $None \Rightarrow False$
 $| Some$ $(((((prefix, prefix_state), raw), raw_state),$
 $((data, attacker_state), result)), final_state) \Rightarrow$
 $\neg$ $trace_table_low_degree$
 $(query_prefix_trace_conceptual_table$ $prefix$ $prefix_state) \vee$
 $\neg$ $composition_table_low_degree$ $maxDegree$
 $(query_prefix_composition_conceptual_table$ $prefix$ $prefix_state) \vee$
 $all_queries_consistent$
 $(query_prefix_trace_conceptual_table$ $prefix$ $prefix_state)$

(query-prefix-composition-conceptual-table prefix prefix-state)
 (sqp-alphas prefix))

definition *active-route-current-prefix-conceptual-algebraic-residual-error-for*
where

active-route-current-prefix-conceptual-algebraic-residual-error-for A i =
wp-event
 (checked-staged-security-experiment-with-query-prefix-data-state A i)
 (active-route-current-prefix-conceptual-algebraic-residual-hit-at i)
adversary-initial-state

definition *active-route-current-prefix-conceptual-trace-low-degree-bad-at*
where

active-route-current-prefix-conceptual-trace-low-degree-bad-at i out
 $\longleftrightarrow$
active-route-current-prefix-conceptual-algebraic-residual-hit-at i out $\wedge$
 (case out of
 None $\Rightarrow$ False
 | Some (((((prefix, prefix-state), raw), raw-state),
 ((data, attacker-state), result)), final-state) $\Rightarrow$
 $\neg$ trace-table-low-degree
 (query-prefix-trace-conceptual-table prefix prefix-state))

definition *active-route-current-prefix-conceptual-composition-low-degree-bad-at*
where

active-route-current-prefix-conceptual-composition-low-degree-bad-at i out
 $\longleftrightarrow$
active-route-current-prefix-conceptual-algebraic-residual-hit-at i out $\wedge$
 (case out of
 None $\Rightarrow$ False
 | Some (((((prefix, prefix-state), raw), raw-state),
 ((data, attacker-state), result)), final-state) $\Rightarrow$
 $\neg$ composition-table-low-degree maxDegree
 (query-prefix-composition-conceptual-table prefix prefix-state))

definition *active-route-current-prefix-conceptual-alpha-bad-set-hit-at*
where

active-route-current-prefix-conceptual-alpha-bad-set-hit-at i out
 $\longleftrightarrow$
active-route-current-prefix-conceptual-algebraic-residual-hit-at i out $\wedge$
 (case out of
 None $\Rightarrow$ False
 | Some (((((prefix, prefix-state), raw), raw-state),
 ((data, attacker-state), result)), final-state) $\Rightarrow$
 sqp-alphas prefix $\in$
 composition-trace-bad-alpha-space
 (query-prefix-trace-conceptual-table prefix prefix-state))

definition *active-route-current-prefix-conceptual-trace-low-degree-error-for*

where

active-route-current-prefix-conceptual-trace-low-degree-error-for A i =
wp-event
(checked-staged-security-experiment-with-query-prefix-data-state A i)
(active-route-current-prefix-conceptual-trace-low-degree-bad-at i)
adversary-initial-state

definition *active-route-current-prefix-conceptual-composition-low-degree-error-for*
where

active-route-current-prefix-conceptual-composition-low-degree-error-for A i =
wp-event
(checked-staged-security-experiment-with-query-prefix-data-state A i)
(active-route-current-prefix-conceptual-composition-low-degree-bad-at i)
adversary-initial-state

definition *active-route-current-prefix-conceptual-alpha-bad-set-error-for*
where

active-route-current-prefix-conceptual-alpha-bad-set-error-for A i =
wp-event
(checked-staged-security-experiment-with-query-prefix-data-state A i)
(active-route-current-prefix-conceptual-alpha-bad-set-hit-at i)
adversary-initial-state

lemma *active-route-current-prefix-conceptual-alpha-bad-set-hit-at-imp-data-state-alpha-prefix-bad-set-hit-on-support:*

assumes *support:*

Some (((((prefix, prefix-state), raw), raw-state),
((data, attacker-state), result)), final-state) ∈
set-dist
(execute
(checked-staged-security-experiment-with-query-prefix-data-state A i)
adversary-initial-state)

and *candidate:*

∧ alpha-prefix alpha-prefix-state.
alpha-prefix =
(sqp-trace-root prefix, sqp-trace-fri-roots prefix,
sqp-trace-fri-challenges prefix, sqp-trace-final prefix) ⇒
Some (alpha-prefix, alpha-prefix-state) ∈
set-dist
(execute (staged-alpha-prefix-program A)
adversary-initial-state) ⇒
query-prefix-trace-conceptual-table prefix prefix-state ∈
alpha-prefix-trace-table-candidates alpha-prefix-state
(sqp-trace-root prefix)

and *hit:*

active-route-current-prefix-conceptual-alpha-bad-set-hit-at i
(Some (((((prefix, prefix-state), raw), raw-state),
((data, attacker-state), result)), final-state))

shows

checked-staged-security-with-data-state-alpha-prefix-bad-set-hit A

```

    composition-trace-bad-alpha-space
    (Some (((data, attacker-state), result), final-state))
proof –
obtain alpha-prefix alpha-prefix-state where alpha-prefix-eq:
  alpha-prefix =
    (sqp-trace-root prefix, sqp-trace-fri-roots prefix,
     sqp-trace-fri-challenges prefix, sqp-trace-final prefix)
and alpha-support:
  Some (alpha-prefix, alpha-prefix-state) ∈
    set-dist
    (execute (staged-alpha-prefix-program A) adversary-initial-state)
by (rule checked-staged-security-with-query-prefix-data-state-alpha-prefix-support
    [OF support])
have table-candidate:
  query-prefix-trace-conceptual-table prefix prefix-state ∈
    alpha-prefix-trace-table-candidates alpha-prefix-state
    (sqp-trace-root prefix)
by (rule candidate[OF alpha-prefix-eq alpha-support])
have alphas-eq: staged-alphas data = sqp-alphas prefix
by (rule checked-staged-security-with-query-prefix-data-state-alphas-eq
    [OF support])
have alpha-bad:
  sqp-alphas prefix ∈
    composition-trace-bad-alpha-space
    (query-prefix-trace-conceptual-table prefix prefix-state)
using hit
unfolding active-route-current-prefix-conceptual-alpha-bad-set-hit-at-def
by simp
have staged-alpha-bad:
  staged-alphas data ∈
    composition-trace-bad-alpha-space
    (query-prefix-trace-conceptual-table prefix prefix-state)
using alpha-bad alphas-eq by simp
have staged-alpha-in-space: staged-alphas data ∈ alpha-space
using staged-alpha-bad composition-trace-bad-alpha-space-subset-alpha-space
by blast
have union-hit:
  staged-alphas data ∈
    alpha-prefix-union-bad-sets alpha-prefix-state
    composition-trace-bad-alpha-space (staged-trace-root data)
proof –
have root-eq: staged-trace-root data = sqp-trace-root prefix
using support
unfolding
  checked-staged-security-experiment-with-query-prefix-data-state-def
  checked-staged-after-query-prefix-receive-with-verifier-def
  checked-staged-after-query-prefix-receive-def
by (auto elim!: set-dist-bindE split: prod.splits)
have table-candidate':

```

```

    query-prefix-trace-conceptual-table prefix prefix-state ∈
      alpha-prefix-trace-table-candidates alpha-prefix-state
      (staged-trace-root data)
    using table-candidate root-eq by simp
  show ?thesis
    unfolding alpha-prefix-union-bad-sets-def
    using table-candidate' staged-alpha-bad staged-alpha-in-space
    by auto
qed
have prefix-components:
  staged-trace-root data = fst alpha-prefix ∧
  staged-trace-fri-roots data = fst (snd alpha-prefix) ∧
  staged-trace-fri-challenges data = fst (snd (snd alpha-prefix)) ∧
  staged-trace-final data = snd (snd (snd alpha-prefix))
using support alpha-prefix-eq
unfolding
  checked-staged-security-experiment-with-query-prefix-data-state-def
  checked-staged-after-query-prefix-receive-with-verifier-def
  checked-staged-after-query-prefix-receive-def
by (auto elim!: set-dist-bindE split: prod.splits)
show ?thesis
  unfolding checked-staged-security-with-data-state-alpha-prefix-bad-set-hit-def
  using alpha-support prefix-components union-hit
  by auto
qed

lemma active-route-current-prefix-conceptual-alpha-bad-set-error-for-bound-from-data-state-alpha-prefix:
  assumes i-bound: i < rounds
  and candidate:
    ∧ prefix prefix-state raw raw-state data attacker-state result
      final-state alpha-prefix alpha-prefix-state.
    Some (((((prefix, prefix-state), raw), raw-state),
      ((data, attacker-state), result)), final-state) ∈
      set-dist
      (execute
        (checked-staged-security-experiment-with-query-prefix-data-state A i)
        adversary-initial-state) ⇒
    alpha-prefix =
      (sqp-trace-root prefix, sqp-trace-fri-roots prefix,
        sqp-trace-fri-challenges prefix, sqp-trace-final prefix) ⇒
    Some (alpha-prefix, alpha-prefix-state) ∈
      set-dist
      (execute (staged-alpha-prefix-program A)
        adversary-initial-state) ⇒
    query-prefix-trace-conceptual-table prefix prefix-state ∈
      alpha-prefix-trace-table-candidates alpha-prefix-state
      (sqp-trace-root prefix)
  and data-state-bound:
    wp-event (checked-staged-security-experiment-with-data-state A)

```

```

    (checked-staged-security-with-data-state-alpha-prefix-bad-set-hit A
     composition-trace-bad-alpha-space)
    adversary-initial-state ≤ C
  shows
    active-route-current-prefix-conceptual-alpha-bad-set-error-for A i ≤ C
proof –
  let ?Data =
    checked-staged-security-with-data-state-alpha-prefix-bad-set-hit A
    composition-trace-bad-alpha-space
  let ?Projected =
    λout. case out of
      None ⇒ ?Data None
    | Some (packed, t) ⇒ ?Data (Some (snd packed, t))
  let ?E = active-route-current-prefix-conceptual-alpha-bad-set-hit-at i
  have event-le:
    wp-event
      (checked-staged-security-experiment-with-query-prefix-data-state A i)
      ?E adversary-initial-state ≤
    wp-event
      (checked-staged-security-experiment-with-query-prefix-data-state A i)
      ?Projected adversary-initial-state
  proof (rule wp-event-mono-on-support)
  fix out
  assume support:
    out ∈
      set-dist
        (execute
          (checked-staged-security-experiment-with-query-prefix-data-state A i)
          adversary-initial-state)
  and hit: ?E out
  show ?Projected out
  proof (cases out)
  case None
  then show ?thesis
    using hit
    unfolding active-route-current-prefix-conceptual-alpha-bad-set-hit-at-def
    by simp
  next
  case (Some packed)
  then obtain prefix prefix-state raw raw-state data attacker-state result
    final-state where out-eq:
    out =
      Some (((((prefix, prefix-state), raw), raw-state),
        ((data, attacker-state), result)), final-state)
  by (cases packed) (auto split: prod.splits)
  have support':
    Some (((((prefix, prefix-state), raw), raw-state),
      ((data, attacker-state), result)), final-state) ∈
    set-dist

```

```

      (execute
        (checked-staged-security-experiment-with-query-prefix-data-state A i)
        adversary-initial-state)
    using support unfolding out-eq .
  have hit':
    active-route-current-prefix-conceptual-alpha-bad-set-hit-at i
      (Some (((((prefix, prefix-state), raw), raw-state),
        ((data, attacker-state), result)), final-state))
    using hit unfolding out-eq .
  have data-hit:
    ?Data (Some (((data, attacker-state), result), final-state))
  by (rule
    active-route-current-prefix-conceptual-alpha-bad-set-hit-at-imp-data-state-alpha-prefix-bad-set-hit-on-support
      [OF support' - hit'])
    (rule candidate[OF support'])
  show ?thesis
    unfolding out-eq using data-hit by simp
qed
qed
have projection:
  wp-event
    (checked-staged-security-experiment-with-query-prefix-data-state A i)
    ?Projected adversary-initial-state =
  wp-event (checked-staged-security-experiment-with-data-state A)
    ?Data adversary-initial-state
  using checked-staged-security-experiment-with-query-prefix-data-state-projection-event
    [OF i-bound, of A ?Data]
  by simp
show ?thesis
  unfolding active-route-current-prefix-conceptual-alpha-bad-set-error-for-def
  by (rule order-trans[OF event-le])
    (simp add: projection data-state-bound)
qed

```

definition *active-route-current-prefix-split-algebraic-residual-bound-for*
where

active-route-current-prefix-split-algebraic-residual-bound-for A i =
active-route-current-prefix-conceptual-trace-low-degree-error-for A i +
active-route-current-prefix-conceptual-composition-low-degree-error-for A i +
active-route-current-prefix-conceptual-alpha-bad-set-error-for A i

definition *active-route-current-prefix-split-conceptual-bound-for*
where

active-route-current-prefix-split-conceptual-bound-for budgets A i =
staged-phase-relation-error size
(staged-query-search-queries budgets i + 1) +
query-error-bound +
hash-collision-budget-value 0
(staged-query-search-queries budgets i + 1) +

active-route-current-prefix-split-algebraic-residual-bound-for A i

lemma *active-route-current-prefix-conceptual-prefix-collision-hit-at-bound:*

assumes *wf: staged-budget-wellformed budgets*

and *controlled: staged-adversary-controlled budgets A*

and *i-bound: i < rounds*

shows

wp-event

(checked-staged-security-experiment-with-query-prefix-data-state A i)

(active-route-current-prefix-conceptual-prefix-collision-hit-at i)

adversary-initial-state ≤

hash-collision-budget-value 0

(staged-query-search-queries budgets i + 1)

proof –

have *i-le: i ≤ rounds*

using *i-bound* **by** *simp*

have *prefix-bound:*

wp-event (checked-staged-query-prefix-receive-with-state A i)

checked-staged-query-prefix-hash-map-output-collision

adversary-initial-state ≤

hash-collision-budget-value 0

(staged-query-search-queries budgets i + 1)

by *(rule checked-staged-query-prefix-hash-map-output-collision-bound*
[OF wf controlled i-le])

show *?thesis*

proof *(rule*

checked-staged-security-experiment-with-query-prefix-data-state-bound-by-prefix

[OF prefix-bound])

show

active-route-current-prefix-conceptual-prefix-collision-hit-at i None

⇒ checked-staged-query-prefix-hash-map-output-collision None

unfolding

active-route-current-prefix-conceptual-prefix-collision-hit-at-def

checked-staged-query-prefix-hash-map-output-collision-def

by *simp*

next

fix *prefix-out t out*

assume

Some (prefix-out, t) ∈

set-dist

(execute (checked-staged-query-prefix-receive-with-state A i)

adversary-initial-state)

and *cont:*

out ∈

set-dist

(execute

(get ≫=

(λprefix-state.

checked-staged-after-query-prefix-receive-with-verifier A i

```

      prefix-out  $\gg$ 
      ( $\lambda$ result. return ((prefix-out, prefix-state), result))))
    t)
  and event:
    active-route-current-prefix-conceptual-prefix-collision-hit-at i out
show
  checked-staged-query-prefix-hash-map-output-collision
  (Some (prefix-out, t))
using cont event
unfolding
  active-route-current-prefix-conceptual-prefix-collision-hit-at-def
  checked-staged-query-prefix-hash-map-output-collision-def
by (auto elim!: set-dist-bindE split: option.splits prod.splits)
qed
qed

lemma active-route-current-prefix-conceptual-hit-residual-split:
  assumes
    checked-staged-security-with-query-prefix-current-prefix-authenticated-hit-at
    i out
  and
    checked-staged-security-with-query-prefix-conceptual-context-residual-at
    out
  shows
    active-route-current-prefix-conceptual-prefix-collision-hit-at i out  $\vee$ 
    active-route-current-prefix-conceptual-algebraic-residual-hit-at i out
  using assms
  unfolding
    active-route-current-prefix-conceptual-prefix-collision-hit-at-def
    active-route-current-prefix-conceptual-algebraic-residual-hit-at-def
    checked-staged-security-with-query-prefix-conceptual-context-residual-at-def
  by (cases out) (auto split: prod.splits)

lemma active-route-current-prefix-conceptual-algebraic-residual-split-on-support:
  assumes false-statement:  $\neg$  exists-valid-trace
  and support:
    Some (((((prefix, prefix-state), raw), raw-state),
      ((data, attacker-state), result)), final-state)  $\in$ 
    set-dist
    (execute
      (checked-staged-security-experiment-with-query-prefix-data-state A i)
      adversary-initial-state)
  and hit:
    active-route-current-prefix-conceptual-algebraic-residual-hit-at i
    (Some (((((prefix, prefix-state), raw), raw-state),
      ((data, attacker-state), result)), final-state))
  shows
    active-route-current-prefix-conceptual-trace-low-degree-bad-at i
    (Some (((((prefix, prefix-state), raw), raw-state),

```



```

      ((data, attacker-state), result)), final-state)) ∨
    active-route-current-prefix-conceptual-composition-low-degree-bad-at i
    (Some (((((prefix, prefix-state), raw), raw-state),
      ((data, attacker-state), result)), final-state)) ∨
    active-route-current-prefix-conceptual-alpha-bad-set-hit-at i
    (Some (((((prefix, prefix-state), raw), raw-state),
      ((data, attacker-state), result)), final-state))
proof -
  let ?trace = query-prefix-trace-conceptual-table prefix prefix-state
  let ?composition =
    query-prefix-composition-conceptual-table prefix prefix-state
  from hit have residual:
    ¬ trace-table-low-degree ?trace ∨
    ¬ composition-table-low-degree maxDegree ?composition ∨
    all-queries-consistent ?trace ?composition (sqp-alphas prefix)
  unfolding
    active-route-current-prefix-conceptual-algebraic-residual-hit-at-def
  by simp
  show ?thesis
  proof (cases trace-table-low-degree ?trace)
  case False
  then show ?thesis
    using hit
    unfolding active-route-current-prefix-conceptual-trace-low-degree-bad-at-def
    by simp
  next
  case trace-low: True
  show ?thesis
  proof (cases composition-table-low-degree maxDegree ?composition)
  case False
  then show ?thesis
    using hit
    unfolding
      active-route-current-prefix-conceptual-composition-low-degree-bad-at-def
    by simp
  next
  case composition-low: True
  from residual trace-low composition-low have all-queries:
    all-queries-consistent ?trace ?composition (sqp-alphas prefix)
  by simp
  have prefix-support:
    Some ((prefix, prefix-state), prefix-state) ∈
      set-dist
        (execute (checked-staged-query-prefix-with-state A i)
          adversary-initial-state)
  by (rule
    checked-staged-security-with-query-prefix-data-state-prefix-support
    [OF support])
  have alpha-len: length (sqp-alphas prefix) = length spec

```

```

    by (rule checked-staged-query-prefix-with-state-alphas-length
        [OF prefix-support])
  have alpha-bad:
    sqp-alphas prefix  $\in$  composition-trace-bad-alpha-space ?trace
    by (rule
        low-degree-all-queries-consistent-imp-composition-trace-bad-alpha-space
        [OF false-statement alpha-len trace-low composition-low
            all-queries])
  then show ?thesis
    using hit
    unfolding
      active-route-current-prefix-conceptual-alpha-bad-set-hit-at-def
    by simp
qed
qed
qed

lemma active-route-current-prefix-conceptual-algebraic-residual-bound-from-split:
  assumes false-statement:  $\neg$  exists-valid-trace
  shows
    active-route-current-prefix-conceptual-algebraic-residual-error-for A i
       $\leq$  active-route-current-prefix-split-algebraic-residual-bound-for A i
  proof -
    let ?M = checked-staged-security-experiment-with-query-prefix-data-state A i
    let ?Algebraic =
      active-route-current-prefix-conceptual-algebraic-residual-hit-at i
    let ?Trace =
      active-route-current-prefix-conceptual-trace-low-degree-bad-at i
    let ?Composition =
      active-route-current-prefix-conceptual-composition-low-degree-bad-at i
    let ?Alpha =
      active-route-current-prefix-conceptual-alpha-bad-set-hit-at i
    have event-le:
      wp-event ?M ?Algebraic adversary-initial-state  $\leq$ 
      wp-event ?M ( $\lambda$ out. ?Trace out  $\vee$  ?Composition out  $\vee$  ?Alpha out)
      adversary-initial-state
    proof (rule wp-event-mono-on-support)
      fix out
      assume support: out  $\in$  set-dist (execute ?M adversary-initial-state)
      and hit: ?Algebraic out
      show ?Trace out  $\vee$  ?Composition out  $\vee$  ?Alpha out
      proof (cases out)
        case None
        then show ?thesis
          using hit
          unfolding
            active-route-current-prefix-conceptual-algebraic-residual-hit-at-def
          by simp
      next

```

```

case (Some packed)
then obtain prefix prefix-state raw raw-state data attacker-state result
final-state where out-eq:
  out =
    Some (((((prefix, prefix-state), raw), raw-state),
      ((data, attacker-state), result)), final-state)
  by (cases packed) (auto split: prod.splits)
have support':
  Some (((((prefix, prefix-state), raw), raw-state),
    ((data, attacker-state), result)), final-state)  $\in$ 
set-dist (execute ?M adversary-initial-state)
  using support unfolding out-eq .
have hit':
  ?Algebraic
  (Some (((((prefix, prefix-state), raw), raw-state),
    ((data, attacker-state), result)), final-state))
  using hit unfolding out-eq .
show ?thesis
  unfolding out-eq
  by (rule
    active-route-current-prefix-conceptual-algebraic-residual-split-on-support
    [OF false-statement support' hit'])
qed
qed
have union-bound:
  wp-event ?M ( $\lambda out. ?Trace\ out \vee ?Composition\ out \vee ?Alpha\ out$ )
  adversary-initial-state  $\leq$ 
  wp-event ?M ?Trace adversary-initial-state +
  wp-event ?M ?Composition adversary-initial-state +
  wp-event ?M ?Alpha adversary-initial-state
proof –
  have wp-event ?M
    ( $\lambda out. ?Trace\ out \vee ?Composition\ out \vee ?Alpha\ out$ )
    adversary-initial-state  $\leq$ 
    wp-event ?M ?Trace adversary-initial-state +
    wp-event ?M ( $\lambda out. ?Composition\ out \vee ?Alpha\ out$ )
    adversary-initial-state
    by (rule wp-event-union-bound)
  also have ...  $\leq$ 
    wp-event ?M ?Trace adversary-initial-state +
    (wp-event ?M ?Composition adversary-initial-state +
    wp-event ?M ?Alpha adversary-initial-state)
    by (intro add-mono order-refl wp-event-union-bound)
  finally show ?thesis
    by (simp add: add.assoc)
qed
show ?thesis
  unfolding
    active-route-current-prefix-conceptual-algebraic-residual-error-for-def

```

active-route-current-prefix-split-algebraic-residual-bound-for-def
active-route-current-prefix-conceptual-trace-low-degree-error-for-def
active-route-current-prefix-conceptual-composition-low-degree-error-for-def
active-route-current-prefix-conceptual-alpha-bad-set-error-for-def
 by (rule order-trans[*OF event-le union-bound*])
 qed

lemma *active-route-current-prefix-authenticated-error-for-le-split-conceptual-bound:*

assumes *wf: staged-budget-wellformed budgets*
and *controlled: staged-adversary-controlled budgets A*
and *i-bound: i < rounds*
and *false-statement: $\neg$ exists-valid-trace*

shows

active-route-current-prefix-authenticated-error-for A i $\leq$
active-route-current-prefix-split-conceptual-bound-for budgets A i

proof –

let *?M = checked-staged-security-experiment-with-query-prefix-data-state A i*

let *?Hit =*

checked-staged-security-with-query-prefix-current-prefix-authenticated-hit-at
i

let *?Conceptual =*

checked-staged-security-with-query-prefix-conceptual-target-hit

let *?Collision =*

active-route-current-prefix-conceptual-prefix-collision-hit-at i

let *?Algebraic =*

active-route-current-prefix-conceptual-algebraic-residual-hit-at i

have *event-le:*

wp-event ?M ?Hit adversary-initial-state $\leq$
wp-event ?M
(λ out. ?Conceptual out $\vee$?Collision out $\vee$?Algebraic out)
adversary-initial-state

proof (rule *wp-event-mono-on-support*)

fix *out*

assume *support: out $\in$ set-dist (execute ?M adversary-initial-state)*

and *hit: ?Hit out*

have *conceptual-or-residual:*

?Conceptual out $\vee$
checked-staged-security-with-query-prefix-conceptual-context-residual-at
out

by (rule

checked-staged-security-with-query-prefix-current-prefix-authenticated-hit-imp-conceptual-or-residual-on-support
 [where *budgets=*budgets **and** *A=A* **and** *i=i* **and** *out=out*,
OF wf controlled i-bound support hit])

show *?Conceptual out $\vee$?Collision out $\vee$?Algebraic out*

using *conceptual-or-residual*

proof (elim *disjE*)

assume *conceptual: ?Conceptual out*

then show *?thesis* **by** *simp*

next

```

assume residual:
  checked-staged-security-with-query-prefix-conceptual-context-residual-at
  out
have ?Collision out  $\vee$  ?Algebraic out
  by (rule active-route-current-prefix-conceptual-hit-residual-split
    [OF hit residual])
  then show ?thesis by simp
qed
qed
have conceptual-bound:
  wp-event ?M ?Conceptual adversary-initial-state  $\leq$ 
    staged-phase-relation-error size
    (staged-query-search-queries budgets i + 1) + query-error-bound
  by (rule
    checked-staged-security-with-query-prefix-conceptual-target-hit-bound
    [OF wf controlled i-bound])
have collision-bound:
  wp-event ?M ?Collision adversary-initial-state  $\leq$ 
    hash-collision-budget-value 0
    (staged-query-search-queries budgets i + 1)
  by (rule
    active-route-current-prefix-conceptual-prefix-collision-hit-at-bound
    [OF wf controlled i-bound])
have algebraic-bound0:
  active-route-current-prefix-conceptual-algebraic-residual-error-for A i  $\leq$ 
    active-route-current-prefix-split-algebraic-residual-bound-for A i
  by (rule
    active-route-current-prefix-conceptual-algebraic-residual-bound-from-split
    [OF false-statement])
have algebraic-bound:
  wp-event ?M ?Algebraic adversary-initial-state  $\leq$ 
    active-route-current-prefix-split-algebraic-residual-bound-for A i
  using algebraic-bound0
  unfolding active-route-current-prefix-conceptual-algebraic-residual-error-for-def
  by simp
have union-bound:
  wp-event ?M
    ( $\lambda out. ?Conceptual out \vee ?Collision out \vee ?Algebraic out$ )
    adversary-initial-state  $\leq$ 
    wp-event ?M ?Conceptual adversary-initial-state +
    wp-event ?M ?Collision adversary-initial-state +
    wp-event ?M ?Algebraic adversary-initial-state
proof –
  have
    wp-event ?M
      ( $\lambda out. ?Conceptual out \vee ?Collision out \vee ?Algebraic out$ )
      adversary-initial-state  $\leq$ 
      wp-event ?M ?Conceptual adversary-initial-state +
      wp-event ?M ( $\lambda out. ?Collision out \vee ?Algebraic out$ )

```

```

    adversary-initial-state
  by (rule wp-event-union-bound)
also have ... ≤
  wp-event ?M ?Conceptual adversary-initial-state +
  (wp-event ?M ?Collision adversary-initial-state +
   wp-event ?M ?Algebraic adversary-initial-state)
  by (intro add-mono order-refl wp-event-union-bound)
finally show ?thesis
  by (simp add: add.assoc)
qed
have sum-bound:
  wp-event ?M ?Conceptual adversary-initial-state +
  wp-event ?M ?Collision adversary-initial-state +
  wp-event ?M ?Algebraic adversary-initial-state ≤
  staged-phase-relation-error size
  (staged-query-search-queries budgets i + 1) +
  query-error-bound +
  hash-collision-budget-value 0
  (staged-query-search-queries budgets i + 1) +
  active-route-current-prefix-split-algebraic-residual-bound-for A i
proof -
  have tail-bound:
    wp-event ?M ?Collision adversary-initial-state +
    wp-event ?M ?Algebraic adversary-initial-state ≤
    hash-collision-budget-value 0
    (staged-query-search-queries budgets i + 1) +
    active-route-current-prefix-split-algebraic-residual-bound-for A i
  by (intro add-mono collision-bound algebraic-bound)
  have
    wp-event ?M ?Conceptual adversary-initial-state +
    (wp-event ?M ?Collision adversary-initial-state +
     wp-event ?M ?Algebraic adversary-initial-state) ≤
    (staged-phase-relation-error size
     (staged-query-search-queries budgets i + 1) +
     query-error-bound) +
    (hash-collision-budget-value 0
     (staged-query-search-queries budgets i + 1) +
     active-route-current-prefix-split-algebraic-residual-bound-for A i)
  by (intro add-mono conceptual-bound tail-bound)
  then show ?thesis
    by (simp add: add.assoc)
qed
show ?thesis
  unfolding active-route-current-prefix-authenticated-error-for-def
    active-route-current-prefix-split-conceptual-bound-for-def
  by (rule order-trans[OF event-le])
    (rule order-trans[OF union-bound sum-bound])
qed

```

definition *active-route-current-empty-conceptual-prefix-collision-hit-at*

where

active-route-current-empty-conceptual-prefix-collision-hit-at
 (- :: 'f itself) out
 $\longleftrightarrow$
checked-staged-security-with-query-prefix-conceptual-empty-context-residual-hit-at
 0 out $\wedge$
 (case out of
 None $\Rightarrow$ False
 | Some (((((prefix, prefix-state), raw), raw-state),
 ((data, attacker-state), result)), final-state) $\Rightarrow$
 hash-map-output-collision prefix-state)

definition *active-route-current-empty-conceptual-trace-low-degree-bad-at*

where

active-route-current-empty-conceptual-trace-low-degree-bad-at
 (- :: 'f itself) out
 $\longleftrightarrow$
checked-staged-security-with-query-prefix-conceptual-empty-context-residual-hit-at
 0 out $\wedge$
 (case out of
 None $\Rightarrow$ False
 | Some (((((prefix, prefix-state), raw), raw-state),
 ((data, attacker-state), result)), final-state) $\Rightarrow$
 $\neg$ trace-table-low-degree
 (query-prefix-trace-conceptual-table prefix prefix-state))

definition *active-route-current-empty-conceptual-alpha-bad-set-hit-at*

where

active-route-current-empty-conceptual-alpha-bad-set-hit-at
 (- :: 'f itself) out
 $\longleftrightarrow$
checked-staged-security-with-query-prefix-conceptual-empty-context-residual-hit-at
 0 out $\wedge$
 (case out of
 None $\Rightarrow$ False
 | Some (((((prefix, prefix-state), raw), raw-state),
 ((data, attacker-state), result)), final-state) $\Rightarrow$
 sqp-alphas prefix $\in$
 composition-trace-bad-alpha-space
 (query-prefix-trace-conceptual-table prefix prefix-state))

definition *active-route-current-empty-conceptual-prefix-collision-error-for*

where

active-route-current-empty-conceptual-prefix-collision-error-for A =
 wp-event
 (checked-staged-security-experiment-with-query-prefix-data-state A 0)
 (active-route-current-empty-conceptual-prefix-collision-hit-at
 TYPE('f))

adversary-initial-state

definition *active-route-current-empty-conceptual-trace-low-degree-error-for*
where

active-route-current-empty-conceptual-trace-low-degree-error-for $A =$
wp-event
(*checked-staged-security-experiment-with-query-prefix-data-state* A 0)
(*active-route-current-empty-conceptual-trace-low-degree-bad-at*
 $TYPE('f')$)
adversary-initial-state

definition *active-route-current-empty-conceptual-alpha-bad-set-error-for*
where

active-route-current-empty-conceptual-alpha-bad-set-error-for $A =$
wp-event
(*checked-staged-security-experiment-with-query-prefix-data-state* A 0)
(*active-route-current-empty-conceptual-alpha-bad-set-hit-at*
 $TYPE('f')$)
adversary-initial-state

lemma *active-route-current-empty-conceptual-alpha-bad-set-hit-at-imp-data-state-alpha-prefix-bad-set-hit-on-support*:

assumes *support*:

Some ((((*prefix*, *prefix-state*), *raw*), *raw-state*),
((*data*, *attacker-state*), *result*)), *final-state*) $\in$
set-dist
(*execute*
(*checked-staged-security-experiment-with-query-prefix-data-state* A 0)
adversary-initial-state)

and *candidate*:

$\bigwedge$ *alpha-prefix* *alpha-prefix-state*.
alpha-prefix =
(*sqp-trace-root* *prefix*, *sqp-trace-fri-roots* *prefix*,
sqp-trace-fri-challenges *prefix*, *sqp-trace-final* *prefix*) $\implies$
Some (*alpha-prefix*, *alpha-prefix-state*) $\in$
set-dist
(*execute* (*staged-alpha-prefix-program* A)
adversary-initial-state) $\implies$
query-prefix-trace-conceptual-table *prefix* *prefix-state* $\in$
alpha-prefix-trace-table-candidates *alpha-prefix-state*
(*sqp-trace-root* *prefix*)

and *hit*:

active-route-current-empty-conceptual-alpha-bad-set-hit-at
 $TYPE('f')$
(*Some* ((((*prefix*, *prefix-state*), *raw*), *raw-state*),
((*data*, *attacker-state*), *result*)), *final-state*))

shows

checked-staged-security-with-data-state-alpha-prefix-bad-set-hit A
composition-trace-bad-alpha-space
(*Some* (((*data*, *attacker-state*), *result*), *final-state*))

proof –

obtain *alpha-prefix alpha-prefix-state* **where** *alpha-prefix-eq*:

alpha-prefix =

(*sqp-trace-root prefix*, *sqp-trace-fri-roots prefix*,
sqp-trace-fri-challenges prefix, *sqp-trace-final prefix*)

and *alpha-support*:

Some (alpha-prefix, alpha-prefix-state) ∈
set-dist
(execute (staged-alpha-prefix-program A) adversary-initial-state)

by (*rule checked-staged-security-with-query-prefix-data-state-alpha-prefix-support*
[*OF support*])

have *table-candidate*:

query-prefix-trace-conceptual-table prefix prefix-state ∈
alpha-prefix-trace-table-candidates alpha-prefix-state
(*sqp-trace-root prefix*)

by (*rule candidate*[*OF alpha-prefix-eq alpha-support*])

have *alphas-eq*: *staged-alphas data = sqp-alphas prefix*

by (*rule checked-staged-security-with-query-prefix-data-state-alphas-eq*
[*OF support*])

have *alpha-bad*:

sqp-alphas prefix ∈
composition-trace-bad-alpha-space
(*query-prefix-trace-conceptual-table prefix prefix-state*)

using *hit*

unfolding *active-route-current-empty-conceptual-alpha-bad-set-hit-at-def*

by *simp*

have *staged-alpha-bad*:

staged-alphas data ∈
composition-trace-bad-alpha-space
(*query-prefix-trace-conceptual-table prefix prefix-state*)

using *alpha-bad alphas-eq* **by** *simp*

have *staged-alpha-in-space*: *staged-alphas data ∈ alpha-space*

using *staged-alpha-bad composition-trace-bad-alpha-space-subset-alpha-space*

by *blast*

have *root-eq*: *staged-trace-root data = sqp-trace-root prefix*

using *support*

unfolding

checked-staged-security-experiment-with-query-prefix-data-state-def
checked-staged-after-query-prefix-receive-with-verifier-def
checked-staged-after-query-prefix-receive-def

by (*auto elim!*: *set-dist-bindE split: prod.splits*)

have *union-hit*:

staged-alphas data ∈
alpha-prefix-union-bad-sets alpha-prefix-state
composition-trace-bad-alpha-space (staged-trace-root data)

proof –

have *table-candidate'*:

query-prefix-trace-conceptual-table prefix prefix-state ∈
alpha-prefix-trace-table-candidates alpha-prefix-state

```

    (staged-trace-root data)
  using table-candidate root-eq by simp
show ?thesis
unfolding alpha-prefix-union-bad-sets-def
  using table-candidate' staged-alpha-bad staged-alpha-in-space
  by auto
qed
have prefix-components:
  staged-trace-root data = fst alpha-prefix ∧
  staged-trace-fri-roots data = fst (snd alpha-prefix) ∧
  staged-trace-fri-challenges data = fst (snd (snd alpha-prefix)) ∧
  staged-trace-final data = snd (snd (snd alpha-prefix))
using support alpha-prefix-eq
unfolding
  checked-staged-security-experiment-with-query-prefix-data-state-def
  checked-staged-after-query-prefix-receive-with-verifier-def
  checked-staged-after-query-prefix-receive-def
  by (auto elim!: set-dist-bindE split: prod.splits)
show ?thesis
unfolding checked-staged-security-with-data-state-alpha-prefix-bad-set-hit-def
  using alpha-support prefix-components union-hit
  by auto
qed

lemma active-route-current-empty-conceptual-alpha-bad-set-error-for-bound-from-data-state-alpha-prefix:
  assumes candidate:
    ∧ prefix prefix-state raw raw-state data attacker-state result
    final-state alpha-prefix alpha-prefix-state.
    Some (((((prefix, prefix-state), raw), raw-state),
      ((data, attacker-state), result)), final-state) ∈
      set-dist
      (execute
        (checked-staged-security-experiment-with-query-prefix-data-state A 0)
        adversary-initial-state) ⇒
    alpha-prefix =
      (sqp-trace-root prefix, sqp-trace-fri-roots prefix,
        sqp-trace-fri-challenges prefix, sqp-trace-final prefix) ⇒
    Some (alpha-prefix, alpha-prefix-state) ∈
      set-dist
      (execute (staged-alpha-prefix-program A)
        adversary-initial-state) ⇒
    query-prefix-trace-conceptual-table prefix prefix-state ∈
      alpha-prefix-trace-table-candidates alpha-prefix-state
      (sqp-trace-root prefix)
  and data-state-bound:
    wp-event (checked-staged-security-experiment-with-data-state A)
      (checked-staged-security-with-data-state-alpha-prefix-bad-set-hit A
        composition-trace-bad-alpha-space)
    adversary-initial-state ≤ C

```

shows
active-route-current-empty-conceptual-alpha-bad-set-error-for $A \leq C$

proof –

let $?Data =$
checked-staged-security-with-data-state-alpha-prefix-bad-set-hit A
composition-trace-bad-alpha-space

let $?Projected =$
 $\lambda out. \text{case } out \text{ of}$
 $\quad None \Rightarrow ?Data \text{ None}$
 $\quad | \text{Some } (packed, t) \Rightarrow ?Data \text{ (Some (snd packed}, t))$

let $?E =$
active-route-current-empty-conceptual-alpha-bad-set-hit-at
 $TYPE('f)$

have *event-le*:
wp-event
 $(\text{checked-staged-security-experiment-with-query-prefix-data-state } A \ 0)$
 $?E \text{ adversary-initial-state} \leq$
wp-event
 $(\text{checked-staged-security-experiment-with-query-prefix-data-state } A \ 0)$
 $?Projected \text{ adversary-initial-state}$

proof (rule *wp-event-mono-on-support*)

fix *out*

assume *support*:
 $out \in$
set-dist
 $(\text{execute}$
 $\quad (\text{checked-staged-security-experiment-with-query-prefix-data-state } A \ 0)$
 $\quad \text{adversary-initial-state})$

and *hit*: $?E \text{ out}$

show $?Projected \text{ out}$

proof (cases *out*)

case *None*

then show $?thesis$
using *hit*
unfolding *active-route-current-empty-conceptual-alpha-bad-set-hit-at-def*
by *simp*

next

case $(\text{Some } packed)$

then obtain *prefix prefix-state raw raw-state data attacker-state result*
final-state **where** *out-eq*:
 $out =$
 $\text{Some } (((((\text{prefix}, \text{prefix-state}), \text{raw}), \text{raw-state}),$
 $\quad ((\text{data}, \text{attacker-state}), \text{result})), \text{final-state})$

by (cases *packed*) (auto split: *prod.splits*)

have *support'*:
 $\text{Some } (((((\text{prefix}, \text{prefix-state}), \text{raw}), \text{raw-state}),$
 $\quad ((\text{data}, \text{attacker-state}), \text{result})), \text{final-state}) \in$
set-dist
 $(\text{execute}$

```

      (checked-staged-security-experiment-with-query-prefix-data-state A 0)
      adversary-initial-state)
    using support unfolding out-eq .
  have hit':
    active-route-current-empty-conceptual-alpha-bad-set-hit-at
      TYPE('f)
      (Some (((((prefix, prefix-state), raw), raw-state),
        ((data, attacker-state), result)), final-state))
    using hit unfolding out-eq .
  have data-hit:
    ?Data (Some (((data, attacker-state), result), final-state))
  by (rule
    active-route-current-empty-conceptual-alpha-bad-set-hit-at-imp-data-state-alpha-prefix-bad-set-hit-on-support
      [OF support' - hit'])
    (rule candidate[OF support'])
  show ?thesis
    unfolding out-eq using data-hit by simp
qed
qed
have projection:
  wp-event
    (checked-staged-security-experiment-with-query-prefix-data-state A 0)
    ?Projected adversary-initial-state =
  wp-event (checked-staged-security-experiment-with-data-state A)
    ?Data adversary-initial-state
  using checked-staged-security-experiment-with-query-prefix-data-state-projection-event
    [OF rounds-positive, of A ?Data]
  by simp
show ?thesis
  unfolding active-route-current-empty-conceptual-alpha-bad-set-error-for-def
  by (rule order-trans[OF event-le])
    (simp add: projection data-state-bound)
qed

```

definition *active-route-current-empty-split-residual-bound-for*
where

active-route-current-empty-split-residual-bound-for A =
active-route-current-empty-conceptual-prefix-collision-error-for A +
active-route-current-empty-conceptual-trace-low-degree-error-for A +
active-route-current-empty-conceptual-alpha-bad-set-error-for A

definition *active-route-current-empty-split-conceptual-bound-for*
where

active-route-current-empty-split-conceptual-bound-for budgets A =
staged-phase-relation-error size
(staged-query-search-queries budgets 0 + 1) +
query-error-bound +
active-route-current-empty-split-residual-bound-for A +
(active-route-transcript-pre-error-for A +

staged-concrete-transcript-target-error-bound)

lemma *active-route-current-empty-conceptual-prefix-collision-hit-at-bound:*
assumes *wf: staged-budget-wellformed budgets*
and *controlled: staged-adversary-controlled budgets A*
shows
active-route-current-empty-conceptual-prefix-collision-error-for A ≤
hash-collision-budget-value 0
(staged-query-search-queries budgets 0 + 1)
proof –
have *i-le: 0 ≤ rounds*
by *simp*
have *prefix-bound:*
wp-event (checked-staged-query-prefix-receive-with-state A 0)
checked-staged-query-prefix-hash-map-output-collision
adversary-initial-state ≤
hash-collision-budget-value 0
(staged-query-search-queries budgets 0 + 1)
by *(rule checked-staged-query-prefix-hash-map-output-collision-bound*
[OF wf controlled i-le])
show *?thesis*
unfolding *active-route-current-empty-conceptual-prefix-collision-error-for-def*
proof *(rule*
checked-staged-security-experiment-with-query-prefix-data-state-bound-by-prefix
[OF prefix-bound])
show
active-route-current-empty-conceptual-prefix-collision-hit-at
TYPE('f) None
 $\implies$ *checked-staged-query-prefix-hash-map-output-collision None*
unfolding
active-route-current-empty-conceptual-prefix-collision-hit-at-def
checked-staged-query-prefix-hash-map-output-collision-def
by *simp*
next
fix *prefix-out t out*
assume
Some (prefix-out, t) ∈
set-dist
(execute (checked-staged-query-prefix-receive-with-state A 0)
adversary-initial-state)
and *cont:*
out ∈
set-dist
(execute
(get $\gg$
(λ prefix-state.
checked-staged-after-query-prefix-receive-with-verifier A 0
prefix-out $\gg$
(λ result. return ((prefix-out, prefix-state), result))))

```

    t)
  and event:
    active-route-current-empty-conceptual-prefix-collision-hit-at
      TYPE('f) out
  show
    checked-staged-query-prefix-hash-map-output-collision
      (Some (prefix-out, t))
  using cont event
  unfolding
    active-route-current-empty-conceptual-prefix-collision-hit-at-def
    checked-staged-query-prefix-hash-map-output-collision-def
  by (auto elim!: set-dist-bindE split: option.splits prod.splits)
qed
qed

lemma active-route-current-empty-conceptual-residual-split-on-support:
  assumes false-statement:  $\neg$  exists-valid-trace
  and support:
    Some (((((prefix, prefix-state), raw), raw-state),
      ((data, attacker-state), result)), final-state)  $\in$ 
      set-dist
        (execute
          (checked-staged-security-experiment-with-query-prefix-data-state A 0)
          adversary-initial-state)
  and hit:
    checked-staged-security-with-query-prefix-conceptual-empty-context-residual-hit-at
      0
    (Some (((((prefix, prefix-state), raw), raw-state),
      ((data, attacker-state), result)), final-state))
  shows
    active-route-current-empty-conceptual-prefix-collision-hit-at
      TYPE('f)
    (Some (((((prefix, prefix-state), raw), raw-state),
      ((data, attacker-state), result)), final-state))  $\vee$ 
    active-route-current-empty-conceptual-trace-low-degree-bad-at
      TYPE('f)
    (Some (((((prefix, prefix-state), raw), raw-state),
      ((data, attacker-state), result)), final-state))  $\vee$ 
    active-route-current-empty-conceptual-alpha-bad-set-hit-at
      TYPE('f)
    (Some (((((prefix, prefix-state), raw), raw-state),
      ((data, attacker-state), result)), final-state))
  proof -
    let ?trace = query-prefix-trace-conceptual-table prefix prefix-state
    let ?composition = replicate (scale * clength) (sqp-composition-final prefix)
    from hit have residual:
      hash-map-output-collision prefix-state  $\vee$ 
       $\neg$  trace-table-low-degree ?trace  $\vee$ 
      all-queries-consistent ?trace ?composition (sqp-alphas prefix)
  
```

```

unfolding
  checked-staged-security-with-query-prefix-conceptual-empty-context-residual-hit-at-def
  checked-staged-security-with-query-prefix-conceptual-empty-context-residual-at-def
by simp
show ?thesis
proof (cases hash-map-output-collision prefix-state)
  case True
  then show ?thesis
    using hit
    unfolding
      active-route-current-empty-conceptual-prefix-collision-hit-at-def
    by simp
next
  case clean: False
  show ?thesis
  proof (cases trace-table-low-degree ?trace)
    case False
    then show ?thesis
      using hit
      unfolding
        active-route-current-empty-conceptual-trace-low-degree-bad-at-def
      by simp
  next
  case trace-low: True
  from residual clean trace-low have all-queries:
    all-queries-consistent ?trace ?composition (sqp-alphas prefix)
    by simp
  have prefix-support:
    Some ((prefix, prefix-state), prefix-state) ∈
      set-dist
      (execute (checked-staged-query-prefix-with-state A 0)
        adversary-initial-state)
    by (rule
      checked-staged-security-with-query-prefix-data-state-prefix-support
      [OF support])
  have alpha-len: length (sqp-alphas prefix) = length spec
    by (rule checked-staged-query-prefix-with-state-alphas-length
      [OF prefix-support])
  have alpha-bad:
    sqp-alphas prefix ∈ composition-trace-bad-alpha-space ?trace
    by (rule
      low-degree-all-queries-consistent-imp-composition-trace-bad-alpha-space
      [OF false-statement alpha-len trace-low
        constant-composition-table-low-degree all-queries])
  then show ?thesis
    using hit
    unfolding
      active-route-current-empty-conceptual-alpha-bad-set-hit-at-def
    by simp

```

qed
 qed
 qed

lemma *active-route-current-empty-conceptual-residual-bound-from-split:*

assumes *false-statement:* $\neg \text{exists-valid-trace}$

shows

active-route-current-empty-conceptual-residual-error-for TYPE('f) A $\leq$
active-route-current-empty-split-residual-bound-for A

proof –

let *?M* = *checked-staged-security-experiment-with-query-prefix-data-state A 0*

let *?Residual* =

$\lambda out.$
checked-staged-security-with-query-prefix-conceptual-empty-context-residual-hit-at
0 out

let *?Collision* =

active-route-current-empty-conceptual-prefix-collision-hit-at
TYPE('f)

let *?Trace* =

active-route-current-empty-conceptual-trace-low-degree-bad-at
TYPE('f)

let *?Alpha* =

active-route-current-empty-conceptual-alpha-bad-set-hit-at
TYPE('f)

have *event-le:*

wp-event ?M ?Residual adversary-initial-state $\leq$
wp-event ?M ($\lambda out.$ *?Collision out* $\vee$ *?Trace out* $\vee$ *?Alpha out*)
adversary-initial-state

proof (*rule wp-event-mono-on-support*)

fix *out*

assume *support:* *out* $\in$ *set-dist* (*execute ?M adversary-initial-state*)

and *hit:* *?Residual out*

show *?Collision out* $\vee$ *?Trace out* $\vee$ *?Alpha out*

proof (*cases out*)

case *None*

then show *?thesis*

using *hit*

unfolding

checked-staged-security-with-query-prefix-conceptual-empty-context-residual-hit-at-def
checked-staged-security-with-query-prefix-conceptual-empty-context-residual-at-def

by *simp*

next

case (*Some packed*)

then obtain *prefix prefix-state raw raw-state data attacker-state result*

final-state **where** *out-eq:*

out =

Some (((((*prefix*, *prefix-state*), *raw*), *raw-state*),
 ((*data*, *attacker-state*), *result*)), *final-state*)

by (*cases packed*) (*auto split: prod.splits*)


```

have support':
  Some (((((prefix, prefix-state), raw), raw-state),
    ((data, attacker-state), result)), final-state) ∈
  set-dist (execute ?M adversary-initial-state)
  using support unfolding out-eq .
have hit':
  ?Residual
    (Some (((((prefix, prefix-state), raw), raw-state),
      ((data, attacker-state), result)), final-state))
  using hit unfolding out-eq .
show ?thesis
  unfolding out-eq
  by (rule
    active-route-current-empty-conceptual-residual-split-on-support
    [OF false-statement support' hit'])
qed
qed
have union-bound:
  wp-event ?M ( $\lambda out. ?Collision\ out \vee ?Trace\ out \vee ?Alpha\ out$ )
    adversary-initial-state ≤
  wp-event ?M ?Collision adversary-initial-state +
  wp-event ?M ?Trace adversary-initial-state +
  wp-event ?M ?Alpha adversary-initial-state
proof –
  have
    wp-event ?M
      ( $\lambda out. ?Collision\ out \vee ?Trace\ out \vee ?Alpha\ out$ )
        adversary-initial-state ≤
    wp-event ?M ?Collision adversary-initial-state +
    wp-event ?M ( $\lambda out. ?Trace\ out \vee ?Alpha\ out$ )
      adversary-initial-state
    by (rule wp-event-union-bound)
  also have ... ≤
    wp-event ?M ?Collision adversary-initial-state +
    (wp-event ?M ?Trace adversary-initial-state +
    wp-event ?M ?Alpha adversary-initial-state)
    by (intro add-mono order-refl wp-event-union-bound)
  finally show ?thesis
    by (simp add: add.assoc)
qed
show ?thesis
  unfolding
    active-route-current-empty-conceptual-residual-error-for-def
    active-route-current-empty-split-residual-bound-for-def
    active-route-current-empty-conceptual-prefix-collision-error-for-def
    active-route-current-empty-conceptual-trace-low-degree-error-for-def
    active-route-current-empty-conceptual-alpha-bad-set-error-for-def
  by (rule order-trans[OF event-le union-bound])
qed

```

lemma *active-route-current-empty-query-error-for-le-split-conceptual-bound:*

assumes *false-statement: $\neg$ exists-valid-trace*

and *wf: staged-budget-wellformed budgets*

and *controlled: staged-adversary-controlled budgets A*

shows

active-route-current-empty-query-error-for A $\leq$

active-route-current-empty-split-conceptual-bound-for budgets A

proof –

have *i-bound: 0 < rounds*

using *rounds-positive* **by** *simp*

have *residual-bound:*

wp-event

(checked-staged-security-experiment-with-query-prefix-data-state A 0)

(checked-staged-security-with-query-prefix-conceptual-empty-context-residual-hit-at 0)

adversary-initial-state $\leq$

active-route-current-empty-split-residual-bound-for A

using *active-route-current-empty-conceptual-residual-bound-from-split*

[OF false-statement]

unfolding *active-route-current-empty-conceptual-residual-error-for-def .*

have *pre-bound:*

wp-event (checked-staged-security-experiment-with-data-state A)

staged-security-with-data-state-transcript-pre-hit

adversary-initial-state $\leq$ active-route-transcript-pre-error-for A

by *(simp add: active-route-transcript-pre-error-for-def)*

have *new-bound:*

wp-event (checked-staged-security-experiment-with-data-state A)

staged-security-with-data-state-transcript-new-hit

adversary-initial-state $\leq$ staged-concrete-transcript-target-error-bound

by *(rule checked-staged-security-with-data-state-transcript-new-hit-bound*

[OF wf controlled])

show *?thesis*

unfolding *active-route-current-empty-query-error-for-def*

active-route-current-empty-split-conceptual-bound-for-def

by *(rule*

checked-staged-security-with-data-state-current-empty-query-opening-hit-at-bound-from-conceptual-empty-residual

[OF wf controlled i-bound residual-bound pre-bound new-bound])

qed

theorem *stark-soundness-from-current-public-prefix-current-query-relevant-drift-route-restricted-trace-endpoint:*

assumes *false-statement: $\neg$ exists-valid-trace*

and *wf: staged-budget-wellformed budgets*

and *controlled: staged-adversary-controlled budgets A*

shows *checked-staged-adversary-acceptance-probability A $\leq$*

hash-collision-budget-value 0

(staged-attacker-query-budget budgets +

staged-challenge-query-budget + verifier-hash-query-budget) +

(composition-error-bound +

staged-phase-relation-error size
 (*staged-alpha-search-queries budgets 0*) +
hash-collision-budget-value 0
 (*staged-alpha-search-queries budgets 0*) +
staged-phase-relation-error size
 (*staged-alpha-search-queries budgets 0*) +
 (*active-route-transcript-pre-error-for A* +
 staged-concrete-transcript-target-error-bound)) +
 ($\sum i < \text{rounds}$.
 active-route-current-prefix-split-conceptual-bound-for budgets A i +
 (*active-route-transcript-pre-error-for A* +
 staged-concrete-transcript-target-error-bound) +
 (*active-route-transcript-pre-error-for A* +
 staged-concrete-transcript-target-error-bound)) +
concrete-trace-header-restricted-one-step-error-for budgets A +
concrete-composition-restricted-one-step-error-for budgets A +
 (*composition-error-bound* +
 staged-phase-relation-error size
 (*staged-alpha-search-queries budgets 0*) +
 hash-collision-budget-value 0
 (*staged-alpha-search-queries budgets 0*) +
 staged-phase-relation-error size
 (*staged-alpha-search-queries budgets 0*) +
 (*active-route-transcript-pre-error-for A* +
 staged-concrete-transcript-target-error-bound) +
 concrete-trace-empty-header-restricted-sampled-split-one-step-error-for budgets A +
 active-route-current-empty-split-conceptual-bound-for budgets A)
proof –
have *trace-bound*:
 wp-event (*checked-staged-security-experiment-with-data-state A*)
 (*staged-security-with-data-state-verifier-event*
 trace-fri-bad-with-header-tied-partial-candidate)
 adversary-initial-state
 $\leq$ *concrete-trace-header-restricted-one-step-error-for budgets A*
unfolding *concrete-trace-header-restricted-one-step-error-for-def*
by (*rule trace-header-restricted-one-step-error-for-bound*
 [*OF wf controlled*])
have *comp-bound*:
 wp-event (*checked-staged-security-experiment-with-data-state A*)
 (*staged-security-with-data-state-verifier-event*
 composition-fri-bad-with-verifier-tied-partial-candidate)
 adversary-initial-state
 $\leq$ *concrete-composition-restricted-one-step-error-for budgets A*
unfolding *concrete-composition-restricted-one-step-error-for-def*
by (*rule composition-restricted-verifier-tied-controlled-error-for-bound*
 [*OF wf controlled*])
have *empty-bound*:
 wp-event (*checked-staged-security-experiment-with-data-state A*)
 (*staged-security-with-data-state-verifier-event*

trace-fri-bad-with-empty-composition-header-candidates)
adversary-initial-state
 $\leq$ *concrete-trace-empty-header-restricted-sampled-split-one-step-error-for*
budgets A
unfolding
concrete-trace-empty-header-restricted-sampled-split-one-step-error-for-def
by (*rule trace-empty-header-restricted-sampled-split-one-step-error-for-bound*
[OF wf controlled])
have *route*:
checked-staged-adversary-acceptance-probability A $\leq$
hash-collision-budget-value 0
(staged-attacker-query-budget budgets +
staged-challenge-query-budget + verifier-hash-query-budget) +
(composition-error-bound +
staged-phase-relation-error size
(staged-alpha-search-queries budgets 0) +
hash-collision-budget-value 0
(staged-alpha-search-queries budgets 0) +
staged-phase-relation-error size
(staged-alpha-search-queries budgets 0) +
(active-route-transcript-pre-error-for A +
staged-concrete-transcript-target-error-bound)) +
($\sum i < \text{rounds}.$
active-route-current-prefix-authenticated-error-for A i +
(active-route-transcript-pre-error-for A +
staged-concrete-transcript-target-error-bound) +
(active-route-transcript-pre-error-for A +
staged-concrete-transcript-target-error-bound)) +
concrete-trace-header-restricted-one-step-error-for budgets A +
concrete-composition-restricted-one-step-error-for budgets A +
(composition-error-bound +
staged-phase-relation-error size
(staged-alpha-search-queries budgets 0) +
hash-collision-budget-value 0
(staged-alpha-search-queries budgets 0) +
staged-phase-relation-error size
(staged-alpha-search-queries budgets 0) +
(active-route-transcript-pre-error-for A +
staged-concrete-transcript-target-error-bound) +
concrete-trace-empty-header-restricted-sampled-split-one-step-error-for budgets A +
active-route-current-empty-query-error-for A)
by (*rule*
stark-soundness-from-current-public-prefix-current-query-relevant-drift-route-from-active-fri-bounds
[OF false-statement wf controlled trace-bound comp-bound empty-bound])
have *prefix-terms*:
($\sum i < \text{rounds}.$
active-route-current-prefix-authenticated-error-for A i +
(active-route-transcript-pre-error-for A +
staged-concrete-transcript-target-error-bound) +

(active-route-transcript-pre-error-for A +
 staged-concrete-transcript-target-error-bound))
 $\leq$
 ($\sum i < \text{rounds}$.
 active-route-current-prefix-split-conceptual-bound-for budgets A i +
 (active-route-transcript-pre-error-for A +
 staged-concrete-transcript-target-error-bound) +
 (active-route-transcript-pre-error-for A +
 staged-concrete-transcript-target-error-bound))
by (intro sum-mono add-mono order-refl
 active-route-current-prefix-authenticated-error-for-le-split-conceptual-bound
 [OF wf controlled - false-statement])
 simp
have empty-term:
 active-route-current-empty-query-error-for $A \leq$
 active-route-current-empty-split-conceptual-bound-for budgets A
by (rule active-route-current-empty-query-error-for-le-split-conceptual-bound
 [OF false-statement wf controlled])
have route-le:
 hash-collision-budget-value 0
 (staged-attacker-query-budget budgets +
 staged-challenge-query-budget + verifier-hash-query-budget) +
 (composition-error-bound +
 staged-phase-relation-error size
 (staged-alpha-search-queries budgets 0) +
 hash-collision-budget-value 0
 (staged-alpha-search-queries budgets 0) +
 staged-phase-relation-error size
 (staged-alpha-search-queries budgets 0) +
 (active-route-transcript-pre-error-for A +
 staged-concrete-transcript-target-error-bound)) +
 ($\sum i < \text{rounds}$.
 active-route-current-prefix-authenticated-error-for A i +
 (active-route-transcript-pre-error-for A +
 staged-concrete-transcript-target-error-bound) +
 (active-route-transcript-pre-error-for A +
 staged-concrete-transcript-target-error-bound)) +
 concrete-trace-header-restricted-one-step-error-for budgets A +
 concrete-composition-restricted-one-step-error-for budgets A +
 (composition-error-bound +
 staged-phase-relation-error size
 (staged-alpha-search-queries budgets 0) +
 hash-collision-budget-value 0
 (staged-alpha-search-queries budgets 0) +
 staged-phase-relation-error size
 (staged-alpha-search-queries budgets 0) +
 (active-route-transcript-pre-error-for A +
 staged-concrete-transcript-target-error-bound) +
 concrete-trace-empty-header-restricted-sampled-split-one-step-error-for budgets A +

$$\begin{aligned}
& \text{active-route-current-empty-query-error-for } A) \\
\leq & \text{hash-collision-budget-value } 0 \\
& (\text{staged-attacker-query-budget budgets} + \\
& \quad \text{staged-challenge-query-budget} + \text{verifier-hash-query-budget}) + \\
& (\text{composition-error-bound} + \\
& \quad \text{staged-phase-relation-error size} \\
& \quad (\text{staged-alpha-search-queries budgets } 0) + \\
& \text{hash-collision-budget-value } 0 \\
& \quad (\text{staged-alpha-search-queries budgets } 0) + \\
& \text{staged-phase-relation-error size} \\
& \quad (\text{staged-alpha-search-queries budgets } 0) + \\
& (\text{active-route-transcript-pre-error-for } A + \\
& \quad \text{staged-concrete-transcript-target-error-bound})) + \\
& (\sum_{i < \text{rounds}}. \\
& \quad \text{active-route-current-prefix-split-conceptual-bound-for budgets } A \ i + \\
& \quad (\text{active-route-transcript-pre-error-for } A + \\
& \quad \quad \text{staged-concrete-transcript-target-error-bound}) + \\
& \quad (\text{active-route-transcript-pre-error-for } A + \\
& \quad \quad \text{staged-concrete-transcript-target-error-bound})) + \\
& \text{concrete-trace-header-restricted-one-step-error-for budgets } A + \\
& \quad \text{concrete-composition-restricted-one-step-error-for budgets } A + \\
& (\text{composition-error-bound} + \\
& \quad \text{staged-phase-relation-error size} \\
& \quad (\text{staged-alpha-search-queries budgets } 0) + \\
& \text{hash-collision-budget-value } 0 \\
& \quad (\text{staged-alpha-search-queries budgets } 0) + \\
& \text{staged-phase-relation-error size} \\
& \quad (\text{staged-alpha-search-queries budgets } 0) + \\
& (\text{active-route-transcript-pre-error-for } A + \\
& \quad \text{staged-concrete-transcript-target-error-bound}) + \\
& \text{concrete-trace-empty-header-restricted-sampled-split-one-step-error-for budgets } A + \\
& \quad \text{active-route-current-empty-split-conceptual-bound-for budgets } A) \\
& \text{by (intro add-mono order-refl prefix-terms empty-term)} \\
& \text{show ?thesis} \\
& \text{by (rule order-trans[OF route route-le])} \\
& \text{qed}
\end{aligned}$$

definition *concrete-restricted-trace-stark-soundness-bound-for*
where

$$\begin{aligned}
& \text{concrete-restricted-trace-stark-soundness-bound-for budgets } A = \\
& \text{hash-collision-budget-value } 0 \\
& (\text{staged-attacker-query-budget budgets} + \\
& \quad \text{staged-challenge-query-budget} + \text{verifier-hash-query-budget}) + \\
& (\text{composition-error-bound} + \\
& \quad \text{staged-phase-relation-error size} \\
& \quad (\text{staged-alpha-search-queries budgets } 0) + \\
& \text{hash-collision-budget-value } 0 \\
& \quad (\text{staged-alpha-search-queries budgets } 0) +
\end{aligned}$$

staged-phase-relation-error size
 (*staged-alpha-search-queries budgets 0*) +
 (*active-route-transcript-pre-error-for A* +
 staged-concrete-transcript-target-error-bound)) +
 ($\sum i < \text{rounds}$.
 active-route-current-prefix-split-conceptual-bound-for budgets A i +
 (*active-route-transcript-pre-error-for A* +
 staged-concrete-transcript-target-error-bound) +
 (*active-route-transcript-pre-error-for A* +
 staged-concrete-transcript-target-error-bound))) +
concrete-trace-header-restricted-one-step-error-for budgets A +
concrete-composition-restricted-one-step-error-for budgets A +
 (*composition-error-bound* +
 staged-phase-relation-error size
 (*staged-alpha-search-queries budgets 0*) +
 hash-collision-budget-value 0
 (*staged-alpha-search-queries budgets 0*) +
 staged-phase-relation-error size
 (*staged-alpha-search-queries budgets 0*) +
 (*active-route-transcript-pre-error-for A* +
 staged-concrete-transcript-target-error-bound) +
 concrete-trace-empty-header-restricted-sampled-split-one-step-error-for budgets A +
 active-route-current-empty-split-conceptual-bound-for budgets A)

theorem *stark-soundness*:

assumes *false-statement*: $\neg \text{exists-valid-trace}$

and *wf*: *staged-budget-wellformed budgets*

and *controlled*: *staged-adversary-controlled budgets A*

shows *checked-staged-adversary-acceptance-probability A* $\leq$
concrete-restricted-trace-stark-soundness-bound-for budgets A

unfolding *concrete-restricted-trace-stark-soundness-bound-for-def*

by (rule

stark-soundness-from-current-public-prefix-current-query-relevant-drift-route-restricted-trace-endpoint
 [*OF false-statement wf controlled*])

end

end